

An Isabelle/HOL Formalization of a STARK Protocol

Diego Marmsoler

August 6, 2026

Department of Computer Science, University of Exeter, Exeter, UK
`d.marmsoler@exeter.ac.uk`

Contents

1	Core Infrastructure and Protocol	1
1.1	Finite Non-negative Real Numbers	1
1.2	Type definition	1
1.3	Basic operations	2
1.4	Algebra and order instances	4
1.5	Representation simp rules	4
1.6	Conversions from natural numbers	5
1.7	Arithmetic simp rules	6
1.8	Finite sums	7
1.9	Probability Distributions	8
1.10	Finite Set Aggregation	9
1.10.1	Finite set sums	9
1.10.2	Summing pairs by key	9
1.10.3	Squashing pairs by key	10
1.11	Probability Maps	11
1.11.1	Summing probability maps	11
1.11.2	Dirac delta distribution	12
1.11.3	Normalization	12
1.11.4	Maps from finite sets	12
1.11.5	Raw items	19
1.12	Probability distributions	19
1.12.1	Delta Distributions	20
1.12.2	Uniform Distributions	20
1.12.3	Die	21
1.12.4	Coin	21
1.13	Distribution Items	21

1.14	Mapper	33
1.15	Relator	33
1.16	BNF Setup	34
1.17	Code Generator Setup	48
1.17.1	Examples	49
1.18	Finite Map setup	52
1.19	Hashing	53
1.20	Weakest Precondition Calculus	59
1.21	Code Generator Setup	59
1.21.1	Enums	59
1.22	Channels	60
1.23	Boolean Interpretation	126
1.24	GF5 Interpretation	128
1.25	The Field GF(5)	129
1.26	Merkle Trees	133
1.27	Weakest Precondition Calculus	134
1.28	Boolean interpretation	135
1.29	GF5 interpretation	136
1.30	Domain-separated protocol Merkle interface	136
1.31	Sanity Check	137
1.31.1	Length	149
1.32	Probabilistic State Monad	152
1.32.1	Bind	152
1.32.2	Return	163
1.32.3	Monad laws	163
1.33	Monad Operations	165
1.33.1	Basic Operators	165
1.33.2	Probabilistic Choice	166
1.34	Weakest Precondition Calculus	166
1.34.1	Examples	178
1.35	Code generator	178
1.35.1	Some basic examples	180
1.36	Stark Protocol	182
1.37	Preliminaries	182
1.37.1	Filter list by predicate over length	182
1.37.2	Basic polynomial definitions	185
1.38	Stark Specifications	186
1.38.1	Lemmas	188
1.38.2	Basic definitions	189
1.38.3	Composition polynomial	199
1.39	Prover	199
1.39.1	Trace Polynomial	200
1.39.2	FRI Commitments	200
1.39.3	Decommitments	217

1.39.4 Stark Proof	217
1.40 Verifier	218
1.40.1 Validate Committments	220
1.40.2 Validate Decommittments	221
2 Completeness	222
3 Executable Example	650
3.1 Square Sequence Example	650
3.2 Polynomial Constraints	651
3.3 Interpreting the stark locale	652
3.4 Interpretation of prover locale	656
3.5 Interpretation of verifier locale	657
3.6 Creating and Verifying Proof	657
4 Soundness	657
4.1 Proof structure for soundness	661
4.2 Algebraic FRI Soundness Interface	1027
4.3 Global FRI table bindings	1080
4.4 Partial Merkle Openings	1871

1 Core Infrastructure and Protocol

```

theory NNReal
  imports
    Complex-Main
    HOL-Library.FSet
    HOL-Library.Conditional-Parametricity
begin

```

1.1 Finite Non-negative Real Numbers

HOL-Probability already provides the extended non-negative real numbers (usually called *ennreal*). That type is ideal for measure theory: non-negative integrals and measures may naturally take the value infinity.

The probabilistic state monad in this development only uses finite, discrete probability weights. For that purpose it is useful to have a smaller type of finite non-negative real numbers, implemented as a subtype of *real*. This avoids proof obligations about infinity while keeping non-negativity as a type invariant.

1.2 Type definition

```

typedef nnreal = {x :: real. 0 ≤ x}

```

morphisms *nn2real real2nn*
by *auto*

setup-lifting *type-definition-nnreal*

lemma *nn2real-nonneg[simp]*: $0 \leq nn2real\ x$
using *nn2real[of x]* **by** *simp*

lemma *real2nn-nn2real[simp]*: $real2nn\ (nn2real\ x) = x$
by (*rule nn2real-inverse*)

lemma *nn2real-real2nn[simp]*:
assumes $0 \leq x$
shows $nn2real\ (real2nn\ x) = x$
using *assms* **by** (*auto intro: real2nn-inverse*)

1.3 Basic operations

instantiation *nnreal* :: *zero*
begin

lift-definition *zero-nnreal* :: *nnreal* **is** *0* **by** *simp*

instance ..

end

instantiation *nnreal* :: *one*
begin

lift-definition *one-nnreal* :: *nnreal* **is** *1* **by** *simp*

instance ..

end

instantiation *nnreal* :: *plus*
begin

lift-definition *plus-nnreal* :: *nnreal* $\Rightarrow$ *nnreal* $\Rightarrow$ *nnreal* **is** $(+)$ **by** *simp*

instance ..

end

instantiation *nnreal* :: *times*
begin

lift-definition *times-nnreal* :: *nnreal* $\Rightarrow$ *nnreal* $\Rightarrow$ *nnreal* **is** $(*)$ **by** *simp*


```

instance ..

end

instantiation nnreal :: ord
begin

lift-definition less-eq-nnreal :: nnreal  $\Rightarrow$  nnreal  $\Rightarrow$  bool is ( $\leq$ ) .

lift-definition less-nnreal :: nnreal  $\Rightarrow$  nnreal  $\Rightarrow$  bool is ( $<$ ) .

instance ..

end

instantiation nnreal :: minus
begin

lift-definition minus-nnreal :: nnreal  $\Rightarrow$  nnreal  $\Rightarrow$  nnreal
  is  $\lambda x y. \max 0 (x - y)$ 
  by simp

instance ..

end

instantiation nnreal :: divide
begin

lift-definition divide-nnreal :: nnreal  $\Rightarrow$  nnreal  $\Rightarrow$  nnreal is ( $/$ ) by simp

instance ..

end

instantiation nnreal :: inverse
begin

lift-definition inverse-nnreal :: nnreal  $\Rightarrow$  nnreal is inverse by simp

instance ..

end

instantiation nnreal :: equal
begin

lift-definition equal-nnreal :: nnreal  $\Rightarrow$  nnreal  $\Rightarrow$  bool is ( $=$ ) .

```

instance by (*intro-classes*, *transfer*, *auto*)

end

1.4 Algebra and order instances

instantiation *nnreal* :: *linordered-semidom*
begin

instance

by (*intro-classes*; *transfer*, (*simp*|*linarith*|*algebra*))
(*metis* *max.absorb2* *max.order-iff* *mult-nonneg-nonpos* *nle-le* *right-diff-distrib'* *zero-le-mult-iff*)

end

instantiation *nnreal* :: *comm-monoid-diff*
begin

instance

by *intro-classes* (*transfer*; *auto* *simp*: *max-def*)

end

1.5 Representation simp rules

lemma *nn2real-eq-iff*[*simp*]: $nn2real\ x = nn2real\ y \longleftrightarrow x = y$
by *transfer simp*

lemma *nn2real-le-iff*[*simp*]: $nn2real\ x \leq nn2real\ y \longleftrightarrow x \leq y$
by *transfer simp*

lemma *nn2real-less-iff*[*simp*]: $nn2real\ x < nn2real\ y \longleftrightarrow x < y$
by *transfer simp*

lemma *nn2real-0*[*simp*]: $nn2real\ 0 = 0$
by *transfer simp*

lemma *nn2real-1*[*simp*]: $nn2real\ 1 = 1$
by *transfer simp*

lemma *nn2real-add*[*simp*]: $nn2real\ (x + y) = nn2real\ x + nn2real\ y$
by *transfer simp*

lemma *nn2real-mult*[*simp*]: $nn2real\ (x * y) = nn2real\ x * nn2real\ y$
by *transfer simp*

lemma *nn2real-minus*[*simp*]:
 $nn2real\ (x - y) = max\ 0\ (nn2real\ x - nn2real\ y)$
by *transfer simp*

lemma *nn2real-inverse[simp]*: $nn2real\ (inverse\ x) = inverse\ (nn2real\ x)$
by *transfer simp*

lemma *nn2real-divide[simp]*: $nn2real\ (x / y) = nn2real\ x / nn2real\ y$
by *transfer simp*

lemma *nn2real-numeral[simp]*: $nn2real\ (numeral\ n) = numeral\ n$
by (*induct n*) (*simp-all only: numeral.simps nn2real-1 nn2real-add*)

lemma *nn2real-eq-transfer-aux[transfer-rule]*:
Transfer.Rel (rel-fun (=) (rel-fun pcr-nnreal (=))) ($\lambda x\ y. nn2real\ x = y$) (=)
by (*smt (verit) Rel-def cr-nnreal-def nn2real-eq-iff nnreal.pcr-cr-eq rel-funI*)

lemma *real2nn-0[simp]*: $real2nn\ 0 = 0$
apply *transfer-start*
apply *transfer-step*
apply *transfer-step*
apply *transfer-step*
apply (*rule nn2real-eq-transfer-aux*)
apply *transfer-end*
by *simp*

lemma *real2nn-1[simp]*: $real2nn\ 1 = 1$
apply *transfer-start*
apply *transfer-step*
apply *transfer-step*
apply *transfer-step*
apply (*rule nn2real-eq-transfer-aux*)
apply *transfer-end*
by *simp*

lemma *real2nn-add[simp]*:
assumes $0 \leq x\ 0 \leq y$
shows $real2nn\ (x + y) = real2nn\ x + real2nn\ y$
by (*metis assms(1,2) nn2real-add nn2real-real2nn real2nn-nn2real*)

lemma *real2nn-mult[simp]*:
assumes $0 \leq x\ 0 \leq y$
shows $real2nn\ (x * y) = real2nn\ x * real2nn\ y$
by (*simp add: assms(1,2) times-nnreal-def*)

1.6 Conversions from natural numbers

lift-definition *nnreal* :: $nat \Rightarrow nnreal$ **is** *of-nat* **by** *linarith*

parametric-constant *of-nat-transfer[transfer-rule]*: *of-nat-def*

lemma *nn2real-nnreal[simp]*: $\text{nn2real } (\text{nnreal } n) = \text{real } n$
by *transfer simp*

lemma *real2nn-eq-transfer-aux[transfer-rule]*:
 $\text{Transfer.Rel } (\text{rel-fun } \text{pcr-nnreal } (\text{rel-fun } (=) (=))) (\lambda y x. x = \text{real2nn } y) (=)$
by (*smt (verit) Rel-abs Rel-def cr-nnreal-def nnreal.pcr-cr-eq*
real2nn-nn2real)

lemma *nnreal-of-nat[simp]*: $\text{nnreal } n = \text{of-nat } n$
by *transfer simp*

lemma *nnreal-leq-simp[simp]*: $\text{nnreal } x \leq \text{nnreal } y \longleftrightarrow x \leq y$
by *transfer simp*

lemma *nnreal-less-simp[simp]*: $\text{nnreal } x < \text{nnreal } y \longleftrightarrow x < y$
by *transfer simp*

lemma *nnreal-0[simp]*: $\text{nnreal } 0 = 0$
by *transfer simp*

lemma *nnreal-Suc[simp]*: $\text{nnreal } (\text{Suc } n) = \text{nnreal } n + 1$
by *transfer simp*

lemma *nnreal-eq-0-iff[simp]*: $\text{nnreal } n = 0 \longleftrightarrow n = 0$
by *transfer simp*

1.7 Arithmetic simp rules

lemma *zero-least[simp]*: $0 \leq (x::\text{nnreal})$
by *transfer simp*

lemma *zero-less-iff-neq-zero[simp]*: $0 < (x::\text{nnreal}) \longleftrightarrow x \neq 0$
using *zero-least[of x]* **by** (*auto simp: le-less*)

lemma *nnreal-add-eq-0-iff[simp]*:
 $(x + y = 0) \longleftrightarrow x = 0 \wedge y = (0::\text{nnreal})$
by *transfer auto*

lemma *nnreal-mult-eq-0-iff[simp]*:
 $(x * y = 0) \longleftrightarrow x = 0 \vee y = (0::\text{nnreal})$
by *transfer auto*

lemma *div-one-nnreal[simp]*: $(p::\text{nnreal}) / 1 = p$
by *transfer simp*

lemma *divide-self*: $(a::\text{nnreal}) \neq 0 \implies a / a = 1$
by *transfer simp*

lemma *times-div-1*:

assumes $x \neq 0$
shows $x * (1 / x) = (1::nnreal)$
using *assms* **by** *transfer simp*

lemma *zero-div[simp]*: $0 / (q::nnreal) = 0$
by *transfer simp*

lemma *divide-eq-0-nnreal[simp]*:
 $((a::nnreal) / q = 0) \longleftrightarrow a = 0 \vee q = 0$
by *transfer auto*

lemma *add-divide-nnreal*:
 $((a::nnreal) + b) / q = a / q + b / q$
by *transfer (simp add: add-divide-distrib)*

lemma *minus-nnreal-code[code]*:
 $nn2real (x - y) = (if\ x \geq y\ then\ nn2real\ x - nn2real\ y\ else\ 0)$
by *transfer auto*

1.8 Finite sums

lemma *fsum-fset-sum*:
 $fsum\ f\ A = (\sum_{x \in fset\ A} f\ x)$
unfolding *fsum-def* **apply** *transfer*
by (*metis fsum.F.rep-eq fsum-def*)

lemma *fsum-divide-nnreal*:
 $fsum\ (\lambda x. f\ x / q)\ A = fsum\ f\ A / (q::nnreal)$
proof (*induct A*)
case *empty*
show *?case*
by (*simp add: fsum-fset-sum*)
next
case (*insert x A*)
then show *?case*
by (*simp add: fsum-fset-sum add-divide-nnreal*)
qed

lemma *sum-divide-nnreal*:
assumes *finite A*
shows $(\sum_{x \in A} f\ x / (q::nnreal)) = (\sum_{x \in A} f\ x) / q$
using *assms*
proof *induct*
case *empty*
show *?case* **by** *simp*
next
case (*insert x A*)
then show *?case*
by (*simp add: add-divide-nnreal*)

qed

lemma *fsum-mult-left-nnreal*:

fsum ($\lambda x. (c::nnreal) * f x$) *A* = *c* * *fsum* *f* *A*

unfolding *fsum-fset-sum*

by (*simp add: sum-distrib-left*)

lemma *sum-constant-nnreal*:

assumes *finite S*

shows ($\sum_{x \in S}. c$) = *nnreal* (*card S*) * (*c::nnreal*)

using *assms*

proof *induction*

case *empty*

show ?*case* **by** *simp*

next

case (*insert x S*)

then show ?*case*

by (*simp add: algebra-simps*)

qed

lemma *sum-1-nnreal*:

assumes *finite S*

shows ($\sum_{x \in S}. (1::nnreal)$) = *nnreal* (*card S*)

using *assms*

proof *induction*

case *empty*

show ?*case* **by** *simp*

next

case (*insert x S*)

then show ?*case* **by** (*simp add: Groups.add-ac(2)*)

qed

end

theory *Prob-Dist*

imports

NNReal

HOL-Library.Monad-Syntax

HOL-Library.FSet

begin

1.9 Probability Distributions

This is a formalization of a datatype for finite probability distributions.

The mathematical presentation uses finitely supported probability maps. This gives a direct specification of distributions and *bind*: a distribution is a partial map whose domain is finite and whose weights sum to one, and *bind* computes the probability of a

result by summing over the finite support of the first computation.

The most direct code equation for `bind` is not executable for arbitrary result and state types, however. The map operation `dom` needs to enumerate the support of a map, and Isabelle’s generated code can only do that for types in the `enum` class. Thus a purely map-based implementation of `bind` would impose an unwanted executability restriction on the types used by the monad.

To avoid this, the theory keeps the map-based definitions as the abstract specification, but provides alternative executable equations in terms of finite sets. The finite-set representation explicitly stores the support and weights, so generated code can iterate over those stored elements without requiring the underlying types to be enumerable. This makes the executable version suitable for arbitrary result and state types.

A previous list-based version has the same basic idea, but converting finite sets to lists in a canonical way requires an ordering on the element type, so that approach introduces a *linorder* constraint. The finite-set implementation used here is therefore more general: it avoids both the *enum* constraint of map-domain enumeration and the *linorder* constraint of canonical list conversion.

The theory also proofs monadic laws for probability distributions and registers the type as a BNF.

1.10 Finite Set Aggregation

1.10.1 Finite set sums

lemma *fsum-ffilter-neutral*:

assumes $\bigwedge x. x \in A \implies \neg P x \implies f x = 0$
shows $fsum\ f\ (ffilter\ P\ A) = fsum\ f\ A$
unfolding *fsum-fset-sum*
apply (*rule sum.mono-neutral-left*)
using *assms* **by** *auto*

lemma *fsum-ffilter-if*:

$fsum\ f\ (ffilter\ P\ A) = fsum\ (\lambda x. \text{if } P\ x \text{ then } f\ x \text{ else } 0)\ A$

proof –

have $fsum\ f\ (ffilter\ P\ A) = (\sum x \in \{x \in fset\ A. P\ x\}. f\ x)$
apply (*simp add: fsum-fset-sum*)
by (*metis Collect-conj-eq Int-commute Collect-mem-eq*)
also have $\dots = (\sum x \in fset\ A. \text{if } P\ x \text{ then } f\ x \text{ else } 0)$
by (*rule sum.inter-filter*) *simp*
also have $\dots = fsum\ (\lambda x. \text{if } P\ x \text{ then } f\ x \text{ else } 0)\ A$
by (*simp add: fsum-fset-sum*)
finally show *?thesis* .

qed

1.10.2 Summing pairs by key

definition *sum-fset-x* :: $'a \Rightarrow ('a \times 'w :: comm-monoid-add)\ fset \Rightarrow 'w$ **where**
 $sum-fset-x\ x\ xs = fsum\ snd\ (ffilter\ (\lambda p. fst\ p = x)\ xs)$

```

lemma sum-fset-x-notin-fst:
  assumes  $x \notin \text{fimage fst } A$ 
  shows  $\text{sum-fset-x } x \ A = 0$ 
proof -
  have  $\text{ffilter } (\lambda p. \text{fst } p = x) \ A = \{\}\}$ 
    using assms
    by (rule-tac fset-eqI) auto
  then show ?thesis
    unfolding sum-fset-x-def by (simp add: fsum-fset-sum)
qed

```

```

lemma sum-fset-x-unique:
  assumes uniq: inj-on fst (fset A)
  assumes mem: (x, p) ∈ A
  shows  $\text{sum-fset-x } x \ A = p$ 
proof -
  have  $\text{ffilter } (\lambda q. \text{fst } q = x) \ A = \{(x, p)\}$ 
    using uniq mem
    apply (rule-tac fset-eqI) apply (auto simp: inj-on-def) by fastforce
  then show ?thesis
    unfolding sum-fset-x-def
    by (simp add: fsum-fset-sum)
qed

```

1.10.3 Squashing pairs by key

definition *squish* :: $('a \times 'w :: \text{comm-monoid-add}) \text{fset} \Rightarrow ('a \times 'w) \text{fset}$ **where**
squish xs = fimage $(\lambda x. (x, \text{sum-fset-x } x \ xs))$ *(fimage fst xs)*

```

lemma squish-fst-unique:
  assumes  $(x, p) \in \text{squish } A$   $(x, q) \in \text{squish } A$ 
  shows  $p = q$ 
  using assms unfolding squish-def by auto

```

```

lemma squish-inj-on-fst:
  inj-on fst (fset (squish A))
  unfolding squish-def inj-on-def
  by auto

```

```

lemma squish-id-if-inj-on-fst:
  assumes uniq: inj-on fst (fset A)
  shows  $\text{squish } A = A$ 
proof (rule fset-eqI)
  fix  $z :: 'a \times 'b$ 
  obtain  $x \ p$  where  $z = (x, p)$ 
    by (cases z)
  show  $(z \in \text{squish } A) = (z \in A)$ 
proof

```



```

    assume  $z \mid\in\mid$  squish  $A$ 
    then obtain  $q$  where  $q: (x, q) \mid\in\mid A$  and  $p: p = \text{sum-fset-}x\ x\ A$ 
      unfolding  $z$  squish-def by auto
    then have  $\text{sum-fset-}x\ x\ A = q$ 
      using uniq by (simp add: sum-fset-x-unique)
    then show  $z \mid\in\mid A$ 
      using  $q\ p$  unfolding  $z$  by simp
  next
  assume  $zA: z \mid\in\mid A$ 
  then have  $\text{sum-fset-}x\ x\ A = p$ 
    using uniq unfolding  $z$  by (simp add: sum-fset-x-unique)
  then show  $z \mid\in\mid$  squish  $A$ 
    using  $zA$  unfolding  $z$  squish-def by force
qed
qed

lemma fsum-squish:
  fsum snd (squish A) = fsum snd A
proof -
  have inj: inj-on ( $\lambda x. (x, \text{sum-fset-}x\ x\ A)$ ) (fset ' fset  $A$ )
    by (auto intro!: inj-onI)

  have fsum snd (squish A) =
    ( $\sum x \in \text{fst ' fset } A. \text{sum-fset-}x\ x\ A$ )
    unfolding squish-def
    apply (simp add: fsum-fset-sum fimage.rep-eq sum.reindex[OF inj] o-def)
    by (smt (verit) image-cong image-image inj split-pairs sum.reindex-cong)
  also have ... =
    ( $\sum x \in \text{fst ' fset } A. \sum p \in \{p \in \text{fset } A. \text{fst } p = x\}. \text{snd } p$ )
    unfolding sum-fset-x-def
    apply (intro sum.cong refl) apply (simp add: fsum-fset-sum Int-def)
    by (metis (lifting) Collect-cong)
  also have ... = ( $\sum p \in \text{fset } A. \text{snd } p$ )
    by (rule sum.group) auto
  also have ... = fsum snd A
    by (simp add: fsum-fset-sum)
  finally show ?thesis .
qed

```

1.11 Probability Maps

1.11.1 Summing probability maps

definition *sum-map* :: ($'a \rightarrow \text{nnreal}$) $\Rightarrow$ *nnreal* **where**
sum-map $f \equiv (\sum i \in \text{dom } f. \text{the } (f\ i))$

type-synonym *prob* = *nnreal*

Probability weights are represented by *nnreal*: finite, non-negative real numbers. Keeping non-negativity as a type invariant avoids carrying explicit $0 \leq p$ assumptions through

sums, bind, map, expectation, and normalization proofs.

Plain *real* would admit negative weights, so every constructor and intermediate probability calculation would need extra invariants and proof obligations to rule them out. Strictly positive reals are also not a good fit: the development needs a zero weight for absent probabilities, empty sums, pruning zero-mass outcomes, and intermediate aggregation before the stored support is normalized.

The separate invariant on the distribution type below that excludes zero weights from the domain is therefore intentional. The weight type supports zero where computations need it, while distributions store only the non-zero support.

abbreviation *some-map*

where

some-map $\equiv$ (let $x = \text{SOME } x :: 'a$. True in (Map.empty($x := \text{Some } 1$)))

1.11.2 Dirac delta distribution

definition *delta-map* $:: 'a \Rightarrow 'a \multimap \text{prob}$

where

delta-map $x = \text{Map.empty}(x := \text{Some } 1)$

1.11.3 Normalization

definition *norm-fset* $:: ('a \times \text{prob}) \text{fset} \Rightarrow ('a \times \text{prob}) \text{fset}$

where

norm-fset $xs =$
 (if $\text{fsum } \text{snd } xs = 0$
 then $\{(\text{undefined}, 1)\}$
 else
 (let
 $xs' = \text{squish } xs;$
 $q = \text{fsum } \text{snd } xs'$
 in $\text{ffilter } (\lambda p. \text{snd } p \neq 0) (\text{fimage } (\lambda(x, p). (x, p / q)) xs'))$

1.11.4 Maps from finite sets

definition *map-of-val* $:: ('a \times 'b) \text{fset} \Rightarrow 'a \Rightarrow 'b$ **where**

map-of-val $A \ x =$
 $\text{snd } (\text{fthe-elem } (\text{ffilter } (\lambda p. \text{fst } p = x) A))$

definition *map-of* $:: ('a \times 'b) \text{fset} \Rightarrow 'a \multimap 'b$ **where**

map-of $A \ x =$
 (if $x \in \text{fimage } \text{fst } A$ then $\text{Some } (\text{map-of-val } A \ x)$ else None)

lemma *dom-map-of-subset*:

$\text{dom } (\text{map-of } A) \subseteq \text{fset } (\text{fimage } \text{fst } A)$

unfolding *map-of-def*

by (*auto split:if-split-asm*)

lemma *finite-dom-map-of*[*simp*]:

```

finite (dom (map-of A))
using dom-map-of-subset[of A]
by (meson finite-fset finite-subset)

```

lemma *map-of-singleton*:

```

map-of {|(x, y)|} = Map.empty(x := Some y)

```

proof

```

fix z
show map-of {|(x, y)|} z = (Map.empty(x := Some y)) z
proof (cases z = x)
  case True
    have filt: ffilter (λp. fst p = x) {|(x, y)|} = {|(x, y)|}
      by (rule fset-eqI) auto
    show ?thesis
      using True
      unfolding map-of-def map-of-val-def
      by (simp add: filt)
  case False
    then show ?thesis
      unfolding map-of-def
      by simp
qed
qed

```

lemma *map-of-val-unique-key*:

```

assumes uniq: ∧y z. (x, y) |∈| A ⟹ (x, z) |∈| A ⟹ y = z
assumes mem: (x, y) |∈| A
shows map-of-val A x = y

```

proof –

```

have filt: ffilter (λp. fst p = x) A = {|(x, y)|}
proof (rule fset-eqI)
  fix p
  show (p |∈| ffilter (λp. fst p = x) A) = (p |∈| {|(x, y)|})
  proof
    assume p: p |∈| ffilter (λp. fst p = x) A
    then obtain z where p-eq: p = (x, z) and pA: (x, z) |∈| A
      by (cases p) auto
    have z = y
      using uniq[OF pA mem] .
    then show p |∈| {|(x, y)|}
      using p-eq by simp
  next
    assume p |∈| {|(x, y)|}
    then show p |∈| ffilter (λp. fst p = x) A
      using mem by auto
  qed
qed

```

```

show ?thesis
  unfolding map-of-val-def
  by (simp add: filt)
qed

lemma map-of-nonzero-on-dom:
  assumes uniq:  $\bigwedge x y z. (x, y) \in A \implies (x, z) \in A \implies y = z$ 
  assumes nonzero:  $\bigwedge x p. (x, p) \in A \implies p \neq 0$ 
  assumes x:  $x \in \text{dom} (\text{map-of } A)$ 
  shows the  $(\text{map-of } A \ x) \neq 0$ 
proof -
  from x have xin:  $x \in \text{fimage fst } A$ 
  unfolding map-of-def by (auto split: if-splits)
  then obtain p where xp:  $(x, p) \in A$ 
  by auto
  have map-of-val A x = p
  by (rule map-of-val-unique-key[OF uniq xp])
  then have map-of A x = Some p
  using xin unfolding map-of-def by simp
  then show ?thesis
  using nonzero[OF xp] by simp
qed

definition map-of-fset ::  $('a \times \text{prob}) \text{fset} \Rightarrow 'a \rightarrow \text{prob}$  where
  map-of-fset A =
    (let q = fsum snd A in
     if q = 0 then Map.empty(undefined := Some 1)
     else map-of (norm-fset A))

lemma finite-dom-map-of-fset: finite (dom (map-of-fset A))
  unfolding map-of-fset-def
  by (simp add: Let-def)

lemma norm-fset-fst-unique:
  assumes (x, p)  $\in \text{norm-fset } A$  (x, q)  $\in \text{norm-fset } A$ 
  shows p = q
proof (cases fsum snd A = 0)
  case True
  then show ?thesis
  using assms unfolding norm-fset-def by auto
next
  case False
  from assms False obtain r s where
    r:  $(x, r) \in \text{squish } A$  p = r / fsum snd (squish A)
    and s:  $(x, s) \in \text{squish } A$  q = s / fsum snd (squish A)
  unfolding norm-fset-def Let-def by auto
  then have r = s
  using squish-fst-unique by fast
  then show ?thesis

```

```

    using r s by simp
qed

lemma norm-fset-nonzero-snd:
  assumes  $(x, p) \in \text{norm-fset } A$ 
  shows  $p \neq 0$ 
  using assms
  unfolding norm-fset-def
  by (auto simp: Let-def split: if-splits prod.splits)

lemma fsum-norm-fset:
   $\text{fsum snd (norm-fset } A) = 1$ 
proof (cases  $\text{fsum snd } A = 0$ )
  case True
  then show ?thesis
    unfolding norm-fset-def by (simp add: fsum-fset-sum)
next
  case False
  define S where  $S = \text{squish } A$ 
  define q where  $q = \text{fsum snd } S$ 

  have q0:  $q \neq 0$ 
  using False fsum-squish unfolding S-def q-def apply simp
  by (simp add: fsum-squish)

  have norm:
     $\text{norm-fset } A =$ 
     $\text{ffilter } (\lambda p. \text{snd } p \neq 0) (\text{fimage } (\lambda p. (\text{fst } p, \text{snd } p / q)) S)$ 
  using False unfolding norm-fset-def S-def q-def
  by (simp add: Let-def case-prod-unfold)

  have inj-norm:  $\text{inj-on } (\lambda p. (\text{fst } p, \text{snd } p / q)) (\text{fset } S)$ 
proof (rule inj-onI)
  fix a b
  assume aS:  $a \in \text{fset } S$  and bS:  $b \in \text{fset } S$ 
  and eq:  $(\text{fst } a, \text{snd } a / q) = (\text{fst } b, \text{snd } b / q)$ 
  obtain x p where  $a = (x, p)$  by (cases a)
  obtain y r where  $b = (y, r)$  by (cases b)
  from eq have xy:  $x = y$ 
  by (simp add: a b)
  have p = r
  using squish-fst-unique[of x p A r] aS bS xy
  unfolding a b S-def by simp
  then show  $a = b$ 
  using xy by (simp add: a b)
qed

have  $\text{fsum snd (norm-fset } A) =$ 
 $\text{fsum snd (ffilter } (\lambda p. \text{snd } p \neq 0) (\text{fimage } (\lambda p. (\text{fst } p, \text{snd } p / q)) S))$ 

```

```

    by (simp add: norm)
  also have ... = fsum snd (fimage (λp. (fst p, snd p / q)) S)
    by (rule fsum-ffilter-neutral) auto
  also have ... = fsum (λp. snd p / q) S
    using inj-norm
    by (simp add: fsum-fset-sum fimage.rep-eq sum.reindex o-def)
  also have ... = q / q
    by (simp add: fsum-divide-nnreal q-def)
  also have ... = 1
    by (rule divide-self[OF q0])
  finally show ?thesis .
qed

```

```

lemma norm-fset-singleton-1:
  norm-fset {|(x, 1)|} = {|(x, 1::nnreal)|}
proof -
  let ?A = {|(x, 1::nnreal)|}

```

```

  have total: fsum snd ?A = 1
    by (simp add: fsum-fset-sum)

```

```

  have sx: sum-fset-x x ?A = 1
    unfolding sum-fset-x-def
    by (simp add: fsum-fset-sum)

```

```

  have squish: squish ?A = ?A
proof (rule fset-eqI)
  fix z
  show (z |∈| squish ?A) = (z |∈| ?A)
    unfolding squish-def
    using sx by auto
qed

```

```

  have img:
    fimage (λ(x, p). (x, p / fsum snd ?A)) (squish ?A) = ?A
proof (rule fset-eqI)
  fix z
  show (z |∈| fimage (λ(x, p). (x, p / fsum snd ?A)) (squish ?A)) =
    (z |∈| ?A)
    using total squish by auto
qed

```

```

  show ?thesis
    unfolding norm-fset-def
    using total squish img
    apply (simp add: Let-def) by fastforce
qed

```

```

lemma sum-map-of:

```

```

assumes uniq:  $\bigwedge x\ y\ z. (x, y) \in A \implies (x, z) \in A \implies y = z$ 
shows sum-map (map-of A) = fsum snd A
proof –
  have dom: dom (map-of A) = fset (fimage fst A)
    unfolding map-of-def apply auto
    apply (metis option.discI)
    by force

  have inj: inj-on fst (fset A)
proof (rule inj-onI)
  fix p q
  assume pA:  $p \in \text{fset } A$  and qA:  $q \in \text{fset } A$  and fst: fst p = fst q
  obtain x y where p:  $p = (x, y)$  by (cases p)
  obtain z where q:  $q = (x, z)$ 
    using fst p by (cases q) auto
  have y = z
    using uniq[of x y z] pA qA unfolding p q by simp
  then show p = q
    unfolding p q by simp
qed

have val:  $p \in A \implies \text{the } (\text{map-of } A \text{ } (\text{fst } p)) = \text{snd } p$  for p
proof –
  assume pA:  $p \in A$ 
  have filt: ffilter ( $\lambda q. \text{fst } q = \text{fst } p$ ) A =  $\{p\}$ 
proof (rule fset-eqI)
  fix q
  show  $(q \in \text{ffilter } (\lambda q. \text{fst } q = \text{fst } p) \ A) = (q \in \{p\})$ 
proof
  assume qf:  $q \in \text{ffilter } (\lambda q. \text{fst } q = \text{fst } p) \ A$ 
  then have qA:  $q \in A$  and fq: fst q = fst p by auto
  obtain x y where p:  $p = (x, y)$  by (cases p)
  obtain z where q:  $q = (x, z)$ 
    using fq p by (cases q) auto
  have z = y
    using uniq[of x z y] qA pA unfolding p q by simp
  then show  $q \in \{p\}$ 
    unfolding p q by simp
next
  assume  $q \in \{p\}$ 
  then show  $q \in \text{ffilter } (\lambda q. \text{fst } q = \text{fst } p) \ A$ 
    using pA by auto
qed
qed
then show ?thesis
  using pA unfolding map-of-def map-of-val-def by auto
qed

have sum-map (map-of A) =

```

```

    (∑ x∈fset (fimage fst A). the (map-of A x))
  unfolding sum-map-def dom by simp
also have ... = (∑ p∈fset A. the (map-of A (fst p)))
  using inj by (simp add: fimage.rep-eq sum.reindex)
also have ... = (∑ p∈fset A. snd p)
  using val by (intro sum.cong) auto
also have ... = fsum snd A
  by (simp add: fsum-fset-sum)
finally show ?thesis .
qed

lemma sum-map-1: sum-map (map-of-fset A) = 1
proof (cases fsum snd A = 0)
  case True
  then show ?thesis
    unfolding map-of-fset-def sum-map-def by (simp add: Let-def)
next
  case False
  have sum-map (map-of-fset A) = sum-map (map-of (norm-fset A))
    using False unfolding map-of-fset-def by (simp add: Let-def)
  also have ... = fsum snd (norm-fset A)
    by (rule sum-map-of) (rule norm-fset-fst-unique)
  also have ... = 1
    by (rule fsum-norm-fset)
  finally show ?thesis .
qed

```

```

lemma map-of-fset-nonzero-on-dom:
  assumes x ∈ dom (map-of-fset A)
  shows the (map-of-fset A x) ≠ 0
proof (cases fsum snd A = 0)
  case True
  then show ?thesis
    using assms unfolding map-of-fset-def by (simp add: Let-def)
next
  case False
  then have map-eq: map-of-fset A = map-of (norm-fset A)
    unfolding map-of-fset-def by (simp add: Let-def)
  show ?thesis
    unfolding map-eq
  proof (rule map-of-nonzero-on-dom)
    show ∧ x y z. (x, y) |∈| norm-fset A ⇒ (x, z) |∈| norm-fset A ⇒ y = z
      by (rule norm-fset-fst-unique)
    show ∧ x p. (x, p) |∈| norm-fset A ⇒ p ≠ 0
      by (rule norm-fset-nonzero-snd)
    show x ∈ dom (map-of (norm-fset A))
      using assms map-eq by simp
  qed
qed

```


lemma *delta-map-map-of-fset-singleton*:

delta-map $x = \text{map-of-fset } \{|(x, 1)|\}$

proof –

let $?A = \{|(x, 1::\text{nnreal})|\}$

have *total*: *fsum snd* $?A = 1$

by (*simp add: fsum-fset-sum*)

have *norm*: *norm-fset* $?A = ?A$

by (*rule norm-fset-singleton-1*)

have *map-of-fset* $?A = \text{map-of } ?A$

unfolding *map-of-fset-def*

using *total norm*

by (*simp add: Let-def*)

also have $\dots = \text{Map.empty}(x := \text{Some } 1)$

by (*rule map-of-singleton*)

finally show *?thesis*

unfolding *delta-map-def* **by** *simp*

qed

abbreviation *option-default where* *option-default* $a \equiv \text{case-option } (0::\text{nnreal}) \text{ id } a$

lemma *option-default-if-zero[simp]*:

option-default (*if* $p = 0$ *then* *None* *else* *Some* p) = p

by (*cases* $p = (0::\text{nnreal})$) *simp-all*

1.11.5 Raw items

definition *raw-items* :: $('a \rightarrow \text{prob}) \Rightarrow ('a \times \text{prob}) \text{ set}$ **where**

raw-items $f = \text{image } (\lambda x. (x, \text{the } (f x))) (\text{dom } f)$

1.12 Probability distributions

typedef $'a \text{ dist} =$

$\{f. \text{finite } (\text{dom } f) \wedge \text{sum-map } f = 1 \wedge (\forall x \in \text{dom } f. \text{the } (f x) \neq 0)\}$

$:: ('a \rightarrow \text{prob}) \text{ set}$

morphisms *dist abs-dist*

proof

fix $x::'a$

show *some-map* $\in$

$\{f. \text{finite } (\text{dom } f) \wedge \text{sum-map } f = 1 \wedge (\forall x \in \text{dom } f. \text{the } (f x) \neq 0)\}$

unfolding *sum-map-def* **by** *simp*

qed

setup-lifting *type-definition-dist*

```

lemma norm-fset-inj-on-fst:
  inj-on fst (fset (norm-fset A))
proof (cases fsum snd A = 0)
  case True
  then show ?thesis
  unfolding norm-fset-def by (simp add: inj-on-def)
next
  case False
  then show ?thesis
  using squish-inj-on-fst[of A]
  unfolding inj-on-def
  unfolding norm-fset-def apply (auto simp: Let-def split: prod.splits)
  by fastforce
qed

```

```

lift-definition dist-of-fset :: ('a × prob) fset ⇒ 'a dist
is map-of-fset
  by (auto simp add: finite-dom-map-of-fset sum-map-1 map-of-fset-nonzero-on-dom)

```

1.12.1 Delta Distributions

```

definition delta-dist :: 'a ⇒ 'a dist
  where delta-dist x = dist-of-fset {|(x, 1)|}

lemma dist-delta-dist:
  dist (delta-dist x) = delta-map x
  unfolding delta-dist-def
  by (simp add: dist-of-fset.rep-eq delta-map-map-of-fset-singleton)

```

1.12.2 Uniform Distributions

```

definition dist-uniform :: 'a fset ⇒ 'a dist
  where dist-uniform xs = dist-of-fset ((λx. (x, 1::prob)) |q xs)

lemma fsum-snd-uniform-ones:
  fsum snd ((λx. (x, 1::nnreal)) |q xs) = nnreal (fcard xs)
proof –
  have inj: inj-on (λx. (x, 1::nnreal)) (fset xs)
  by (auto intro!: inj-onI)

  have fsum snd ((λx. (x, 1::nnreal)) |q xs) =
    (∑ x∈fset xs. (1::nnreal))
  by (simp add: fsum-fset-sum fimage.rep-eq sum.reindex[OF inj] o-def)
  also have ... = nnreal (card (fset xs))
  by (rule sum-1-nnreal) simp
  also have ... = nnreal (fcard xs)
  by (metis fcard.rep-eq)
  finally show ?thesis .
qed

```

```

lemma norm-fset-uniform-ones:
  assumes  $xs \neq \{\}\{\}$ 
  shows
     $norm\text{-}fset ((\lambda x. (x, 1::nnreal)) \mid^{\dagger} xs) =$ 
     $((\lambda x. (x, 1 / nnreal (fcard\ xs))) \mid^{\dagger} xs)$ 
proof -
  let  $?A = ((\lambda x. (x, 1::nnreal)) \mid^{\dagger} xs)$ 
  let  $?B = ((\lambda x. (x, 1 / nnreal (fcard\ xs))) \mid^{\dagger} xs)$ 

  have total:  $fsum\ snd\ ?A = nnreal\ (fcard\ xs)$ 
    by (rule fsum-snd-uniform-ones)

  have nonzero:  $fsum\ snd\ ?A \neq 0$ 
    using assms total by simp

  have inj: inj-on fst (fset  $?A$ )
    by (auto simp: fimage.rep-eq inj-on-def)

  have squish-A: squish  $?A = ?A$ 
    by (rule squish-id-if-inj-on-fst[OF inj])

  have img:
     $fimage\ (\lambda(x, p). (x, p / fsum\ snd\ ?A))\ ?A = ?B$ 
    using total
    by (rule-tac fset-eqI) (auto simp: fimage.rep-eq)

  have filt:
     $ffilter\ (\lambda p. snd\ p \neq 0)\ ?B = ?B$ 
    using assms
    by (rule-tac fset-eqI) auto

  show ?thesis
    unfolding norm-fset-def
    using nonzero squish-A img filt
    by (simp add: Let-def)
qed

```

1.12.3 Die

```

definition die ::  $nat \Rightarrow nat\ dist$ 
  where  $die\ n = dist\text{-}uniform\ (fset\text{-}of\text{-}list\ ([1..<Suc\ n]))$ 

```

1.12.4 Coin

```

definition coin ::  $prob \Rightarrow 'a \Rightarrow 'a \Rightarrow 'a\ dist$ 
  where  $coin\ f\ x\ y = dist\text{-}of\text{-}fset\ \{|(x,f),(y,1-f)|\}$ 

```

1.13 Distribution Items

context

```

  includes fset.lifting
begin

lift-definition items :: 'a dist  $\Rightarrow$  ('a  $\times$  prob) fset
is raw-items unfolding raw-items-def by simp

end

lemma map-of-val-unique:
  assumes uniq: inj-on fst (fset A)
  assumes mem: (x, y)  $\in$  A
  shows map-of-val A x = y
proof -
  have filt: ffilter ( $\lambda p$ . fst p = x) A = {(x, y)|}
  proof (rule fset-eqI)
    fix p
    show (p  $\in$  ffilter ( $\lambda p$ . fst p = x) A) = (p  $\in$  {(x, y)|})
  proof
    assume p: p  $\in$  ffilter ( $\lambda p$ . fst p = x) A
    then have pA: p  $\in$  A and fst-p: fst p = x
      by auto
    have p = (x, y)
      using uniq pA mem fst-p
      unfolding inj-on-def
      by fastforce
    then show p  $\in$  {(x, y)|}
      by simp
  next
    assume p  $\in$  {(x, y)|}
    then show p  $\in$  ffilter ( $\lambda p$ . fst p = x) A
      using mem by auto
  qed
  qed

  show ?thesis
    unfolding map-of-val-def
    by (simp add: filt)
qed

lemma raw-items-map-of-unique:
  assumes uniq: inj-on fst (fset A)
  shows raw-items (map-of A) = fset A
proof
  show raw-items (map-of A)  $\subseteq$  fset A
  proof
    fix z
    assume z: z  $\in$  raw-items (map-of A)
    then obtain x where
      xdom: x  $\in$  dom (map-of A)

```

```

    and z-eq: z = (x, the (map-of A x))
    unfolding raw-items-def by auto

from xdom have xin: x |∈| fimage fst A
  unfolding map-of-def by (auto split: if-splits)

then obtain y where xy: (x, y) |∈| A
  by auto

have the (map-of A x) = y
  using xin map-of-val-unique[OF uniq xy]
  unfolding map-of-def by simp

then show z ∈ fset A
  using z-eq xy by simp
qed
next
show fset A ⊆ raw-items (map-of A)
proof
  fix z
  assume zA: z ∈ fset A
  obtain x y where z-eq: z = (x, y)
    by (cases z)

  have xy: (x, y) |∈| A
    using zA z-eq by simp

  have xin: x |∈| fimage fst A
    using xy apply auto by force

  have map-x: map-of A x = Some y
    using xin map-of-val-unique[OF uniq xy]
    unfolding map-of-def by simp

  then have x ∈ dom (map-of A)
    by auto

  moreover have z = (x, the (map-of A x))
    using z-eq map-x by simp

  ultimately show z ∈ raw-items (map-of A)
    unfolding raw-items-def by auto
qed
qed

lemma raw-items-map-of-fset:
  raw-items (map-of-fset xs) = fset (norm-fset xs)
proof (cases fsum snd xs = 0)
  case True

```

```

then show ?thesis
  unfolding raw-items-def map-of-fset-def norm-fset-def
  by (simp add: Let-def)
next
case False
have raw-items (map-of-fset xs) = raw-items (map-of (norm-fset xs))
  using False
  unfolding map-of-fset-def
  by (simp add: Let-def)
also have ... = fset (norm-fset xs)
  by (rule raw-items-map-of-unique) (rule norm-fset-inj-on-fst)
finally show ?thesis .
qed

lemma fset-eq-transfer-aux[transfer-rule]:
  Transfer.Rel
  (rel-fun (pcr-fset (rel-prod (=) (=)))
    (rel-fun (rel-fset (rel-prod (=) (=))) (=)))
  (λa b. a = fset b) (=)
  unfolding rel-fun-def
  by (simp add: Rel-def cr-fset-def fset.pcr-cr-eq fset.rel-eq fset-cong prod.rel-eq)

definition sum-fset-by ::
  'a ⇒ ('i ⇒ 'a) ⇒ ('i ⇒ prob) ⇒ 'i fset ⇒ prob
where
  sum-fset-by x key weight I =
    fsum weight (ffilter (λi. key i = x) I)

definition map-of-fset-by ::
  ('i ⇒ 'a) ⇒ ('i ⇒ prob) ⇒ 'i fset ⇒ 'a → prob
where
  map-of-fset-by key weight I =
    (let q = fsum weight I in
     if q = 0 then Map.empty(undefined := Some 1)
     else (λx.
       let p = sum-fset-by x key weight I / q
       in if p = 0 then None else Some p))

lemma fsum-by-total:
  fsum (λx. sum-fset-by x key weight I) (fimage key I) = fsum weight I
proof -
  have fsum (λx. sum-fset-by x key weight I) (fimage key I) =
    (∑ x∈key ' fset I. ∑ i∈{i ∈ fset I. key i = x}. weight i)
  unfolding sum-fset-by-def
  apply (simp add: fsum-fset-sum fimage.rep-eq)
  by (metis Int-commute Collect-conj-eq Collect-mem-eq)
also have ... = (∑ i∈fset I. weight i)
  apply (rule sum.group) by auto
also have ... = fsum weight I

```

```

    by (simp add: fsum-fset-sum)
  finally show ?thesis .
qed

lemma finite-dom-map-of-fset-by[simp]:
  finite (dom (map-of-fset-by key weight I))
proof -
  have dom (map-of-fset-by key weight I)  $\subseteq$  fset (fimage key I)  $\cup$  {undefined}
    unfolding map-of-fset-by-def sum-fset-by-def
    apply (auto simp: Let-def fsum-fset-sum)
    by (metis fmember-filter[of -  $\lambda uub.$  key uub = - I] filter-fset[of  $\lambda uub.$  key uub = - I]
image-iff[of - key fset I] option.discI[of None]
    sum.not-neutral-contains-not-neutral[of weight fset (ffilter ( $\lambda uub.$  key uub = -) I)])
  then show ?thesis
    by (simp add: finite-subset)
qed

lemma sum-fset-by-notin:
  assumes  $x \notin$  fset (fimage key I)
  shows sum-fset-by x key weight I = 0
  using assms
  unfolding sum-fset-by-def
  apply (simp add: fsum-fset-sum fimage.rep-eq)
  by (smt (verit) Int-emptyI image-iff mem-Collect-eq nnreal-0 sum.empty)

lemma dom-map-of-fset-by:
  assumes  $q$ : fsum weight I  $\neq$  0
  shows
    dom (map-of-fset-by key weight I) =
      { $x \in$  fset (fimage key I). sum-fset-by x key weight I  $\neq$  0}
proof
  show dom (map-of-fset-by key weight I)
     $\subseteq$  { $x \in$  fset (fimage key I). sum-fset-by x key weight I  $\neq$  0}
  proof
    fix x
    assume  $x$ :  $x \in$  dom (map-of-fset-by key weight I)
    then have nz: sum-fset-by x key weight I  $\neq$  0
      using  $q$ 
      unfolding map-of-fset-by-def
      by (auto simp: Let-def split: if-splits)

    moreover have  $x \in$  fset (fimage key I)
      using nz sum-fset-by-notin[of x key I weight]
      by blast

    ultimately show  $x \in$  { $x \in$  fset (fimage key I). sum-fset-by x key weight I  $\neq$  0}
      by simp
  qed
next

```

```

show  $\{x \in \text{fset } (\text{fimage key } I). \text{sum-fset-by } x \text{ key weight } I \neq 0\}$ 
   $\subseteq \text{dom } (\text{map-of-fset-by key weight } I)$ 
using  $q$ 
unfolding  $\text{map-of-fset-by-def}$ 
by  $(\text{auto simp: Let-def})$ 
qed

lemma  $\text{sum-map-map-of-fset-by}[simp]:$ 
   $\text{sum-map } (\text{map-of-fset-by key weight } I) = 1$ 
proof  $(\text{cases fsum weight } I = 0)$ 
  case  $\text{True}$ 
  then show  $?thesis$ 
    unfolding  $\text{map-of-fset-by-def sum-map-def}$ 
    by  $\text{simp}$ 
next
  case  $\text{False}$ 
  let  $?q = \text{fsum weight } I$ 
  let  $?S = \{x \in \text{fset } (\text{fimage key } I). \text{sum-fset-by } x \text{ key weight } I \neq 0\}$ 

  have  $\text{dom: dom } (\text{map-of-fset-by key weight } I) = ?S$ 
    using  $\text{False by (rule dom-map-of-fset-by)}$ 

  have  $\text{val:}$ 
     $\bigwedge x. x \in ?S \implies$ 
     $\text{the } (\text{map-of-fset-by key weight } I \ x) =$ 
     $\text{sum-fset-by } x \text{ key weight } I \ / \ ?q$ 
    using  $\text{False}$ 
    unfolding  $\text{map-of-fset-by-def}$ 
    by  $(\text{auto simp: Let-def})$ 

  have  $\text{sum-map } (\text{map-of-fset-by key weight } I) =$ 
     $(\sum x \in ?S. \text{the } (\text{map-of-fset-by key weight } I \ x))$ 
    unfolding  $\text{sum-map-def dom by simp}$ 
  also have  $\dots =$ 
     $(\sum x \in ?S. \text{sum-fset-by } x \text{ key weight } I \ / \ ?q)$ 
    by  $(\text{intro sum.cong refl val})$ 
  also have  $\dots =$ 
     $(\sum x \in \text{fset } (\text{fimage key } I). \text{sum-fset-by } x \text{ key weight } I \ / \ ?q)$ 
    apply  $(\text{rule sum.mono-neutral-left})$ 
    apply  $\text{simp}$ 
    apply  $\text{auto}$ 
    done
  also have  $\dots =$ 
     $(\sum x \in \text{fset } (\text{fimage key } I). \text{sum-fset-by } x \text{ key weight } I) \ / \ ?q$ 
    by  $(\text{rule sum-divide-nnreal}) \text{ simp}$ 
  also have  $\dots = ?q \ / \ ?q$ 
    using  $\text{fsum-by-total}[of \text{key weight } I]$ 
    by  $(\text{simp add: fsum-fset-sum})$ 
  also have  $\dots = 1$ 

```


by (rule divide-self[OF False])
 finally show ?thesis .
 qed

definition *squish-by* ::
 ($'i \Rightarrow 'a \Rightarrow ('i \Rightarrow \text{prob}) \Rightarrow 'i \text{ fset} \Rightarrow ('a \times \text{prob}) \text{ fset}$)
where
squish-by key weight I =
 fimage ($\lambda x. (x, \text{sum-fset-by } x \text{ key weight } I)$) (fimage key I)

definition *dist-of-fset-by* ::
 ($'i \Rightarrow 'a \Rightarrow ('i \Rightarrow \text{prob}) \Rightarrow 'i \text{ fset} \Rightarrow 'a \text{ dist}$)
where
dist-of-fset-by key weight I =
 dist-of-fset (squish-by key weight I)

lemma *fimage-fst-squish-by[simp]*:
 fimage fst (squish-by key weight I) = fimage key I
unfolding *squish-by-def*
by (rule fset-eqI) auto

lemma *squish-by-inj-on-fst*:
 inj-on fst (fset (squish-by key weight I))
unfolding *squish-by-def* *inj-on-def*
by auto

lemma *fsum-snd-squish-by*:
 fsum snd (squish-by key weight I) = fsum weight I

proof –
have *inj*:
 inj-on ($\lambda x. (x, \text{sum-fset-by } x \text{ key weight } I)$) (fset (fimage key I))
by (auto intro!: inj-onI)

have *fsum snd (squish-by key weight I) =*
fsum ($\lambda x. \text{sum-fset-by } x \text{ key weight } I$) (fimage key I)
unfolding *squish-by-def*
apply (simp add: fsum-fset-sum fimage.rep-eq sum.reindex[OF inj] o-def)
by (smt (verit) fset.set-map image-cong image-image inj sndI sum.reindex-cong)
also have ... = *fsum weight I*
by (rule fsum-by-total)
finally show ?thesis .

qed

lemma *sum-fset-x-squish-by*:
 sum-fset-x x (squish-by key weight I) = sum-fset-by x key weight I
proof (cases x | $\in$ | fimage key I)
case True
then have ($x, \text{sum-fset-by } x \text{ key weight } I$) | $\in$ | squish-by key weight I

```

    unfolding squish-by-def by auto
  then show ?thesis
    using squish-by-inj-on-fst
    by (metis squish-by-inj-on-fst  $\langle (x, \text{sum-fset-by } x \text{ key weight } I) \mid \in \mid \text{squish-by key weight } I \rangle$ 
    sum-fset-x-unique)
next
  case False
  then show ?thesis
    using sum-fset-x-notin-fst[of  $x$  squish-by key weight  $I$ ]
    sum-fset-by-notin[of  $x$  key  $I$  weight]
    by simp
qed

lemma norm-fset-squish-by-1:
  assumes total: fsum weight  $I = 1$ 
  shows
    norm-fset (squish-by key weight  $I$ ) =
      ffilter ( $\lambda p. \text{snd } p \neq 0$ ) (squish-by key weight  $I$ )
proof -
  let ?A = squish-by key weight  $I$ 

  have total-A: fsum snd ?A = 1
    using total by (simp add: fsum-snd-squish-by)

  have squish-A: squish ?A = ?A
    by (rule squish-id-if-inj-on-fst) (rule squish-by-inj-on-fst)

  have div-A: fimage ( $\lambda(x, p). (x, p / \text{fsum snd } ?A)$ ) ?A = ?A
    using total-A
    by (rule-tac fset-eqI) auto

  show ?thesis
    unfolding norm-fset-def
    by (simp add: total-A squish-A div-A Let-def)
qed

lemma map-of-filtered-unique:
  assumes uniq: inj-on fst (fset  $A$ )
  shows
    map-of (ffilter ( $\lambda p. \text{snd } p \neq 0$ )  $A$ )  $x =$ 
      (if sum-fset-x  $x$   $A = 0$  then None else Some (sum-fset-x  $x$   $A$ ))
proof (cases  $x \mid \in \mid \text{fimage fst } A$ )
  case False
  then have sx: sum-fset-x  $x$   $A = 0$ 
    by (rule sum-fset-x-notin-fst)
  moreover have  $x \not\mid \in \mid \text{fimage fst } A$  (ffilter ( $\lambda p. \text{snd } p \neq 0$ )  $A$ )
    using False by auto
  ultimately show ?thesis
    unfolding map-of-def by simp

```

```

next
  case True
  then obtain p where pA: (x, p) |∈| A
    by auto
  have sx: sum-fset-x x A = p
    using uniq pA by (rule sum-fset-x-unique)

  show ?thesis
  proof (cases p = 0)
    case True
    have x |∉| fimage fst (ffilter (λp. snd p ≠ 0) A)
      using uniq pA True
      apply (auto simp: inj-on-def) by fastforce
    then show ?thesis
      using sx True unfolding map-of-def by simp
  next
    case False
    have filt:
      ffilter (λq. fst q = x) (ffilter (λp. snd p ≠ 0) A) = {(x, p)}
      using uniq pA False
      apply (rule-tac fset-eqI) apply (auto simp: inj-on-def) by fastforce
    have xin: x |∈| fimage fst (ffilter (λp. snd p ≠ 0) A)
      using pA False apply auto by force
    have val:
      map-of-val (ffilter (λp. snd p ≠ 0) A) x = p
      unfolding map-of-val-def
      by (simp add: filt)
    show ?thesis
      using sx False xin val
      unfolding map-of-def by simp
  qed
qed

lemma map-of-fset-squish-by-1:
  assumes total: fsum weight I = 1
  shows map-of-fset (squish-by key weight I) = map-of-fset-by key weight I
proof
  fix x
  let ?A = squish-by key weight I

  have total-A: fsum snd ?A = 1
    using total by (simp add: fsum-snd-squish-by)

  have map-of-fset ?A x =
    map-of (ffilter (λp. snd p ≠ 0) ?A) x
    unfolding map-of-fset-def
    using total-A norm-fset-squish-by-1[OF total]
    apply simp by (rule arg-cong2)
  also have ... =

```

(if sum-fset-x x ?A = 0 then None else Some (sum-fset-x x ?A))
 by (rule map-of-filtered-unique) (rule squish-by-inj-on-fst)
 also have ... =
 (if sum-fset-by x key weight I = 0 then None else Some (sum-fset-by x key weight I))
 by (simp add: sum-fset-x-squish-by)
 also have ... = map-of-fset-by key weight I x
 using total
 unfolding map-of-fset-by-def
 by simp
 finally show map-of-fset ?A x = map-of-fset-by key weight I x .
 qed

lemma dist-of-fset-by-rep-eq-1:
 assumes total: fsum weight I = 1
 shows dist (dist-of-fset-by key weight I) = map-of-fset-by key weight I
 unfolding dist-of-fset-by-def dist-of-fset.rep-eq
 using map-of-fset-squish-by-1[OF total] .

lemma sum-fset-by-id:
 sum-fset-by x id weight I = (if x |∈| I then weight x else 0)
 unfolding sum-fset-by-def fsum-fset-sum
 by (simp add: ffilter.rep-eq)

lemma dist-of-fset-by-id-apply:
 assumes total: fsum weight I = 1
 shows dist (dist-of-fset-by id weight I) x =
 (let p = (if x |∈| I then weight x else 0) in
 if p = 0 then None else Some p)
 using dist-of-fset-by-rep-eq-1[OF total, of id] total
 by (simp add: map-of-fset-by-def sum-fset-by-id)

lemma sum-map-dist[simp]:
 sum-map (dist d) = 1
 by transfer simp

lemma finite-dom-dist[simp]:
 finite (dom (dist d))
 by transfer simp

lemma dist-nonzero-on-dom[simp]:
 assumes x ∈ dom (dist d)
 shows the (dist d x) ≠ 0
 using assms by transfer simp

lemma fset-items:
 fset (items d) = (λx. (x, the (dist d x))) ‘ dom (dist d)
 apply transfer apply (simp add: raw-items-def)
 by (metis fset.rep-transfer Rel-def)

lemma *fsum-items*:
 $fsum\ h\ (items\ d) = (\sum x \in dom\ (dist\ d). h\ (x, the\ (dist\ d\ x)))$
proof –
have *inj*: *inj-on* $(\lambda x. (x, the\ (dist\ d\ x)))\ (dom\ (dist\ d))$
by *(auto intro! inj-onI)*
show *?thesis*
unfolding *fsum-fset-sum fset-items*
by *(subst sum.reindex[OF inj]) simp*
qed

lemma *fsum-items-key*:
 $fsum\ (\lambda xp. if\ fst\ xp = x\ then\ snd\ xp\ else\ 0)\ (items\ d) =$
 $option-default\ (dist\ d\ x)$
unfolding *fsum-items*
apply *simp*
using *id-apply* **by** *fastforce*

lemma *fsum-snd-items[simp]*:
 $fsum\ snd\ (items\ d) = 1$
unfolding *fsum-items sum-map-def* **apply** *simp*
by *(metis sum-map-def sum-map-dist)*

lemma *fsum-fbind-fimage-Pair*:
 $fsum\ h\ (fbind\ A\ (\lambda a. fimage\ (\lambda b. (a, b))\ (B\ a))) =$
 $fsum\ (\lambda a. fsum\ (\lambda b. h\ (a, b))\ (B\ a))\ A$
proof –
have *set-eq*:
 $fset\ (fbind\ A\ (\lambda a. fimage\ (\lambda b. (a, b))\ (B\ a))) =$
 $(SIGMA\ a:fset\ A. fset\ (B\ a))$
by *(auto simp: fbind.rep-eq fimage.rep-eq Set.bind-def)*
have $sum\ h\ (SIGMA\ a:fset\ A. fset\ (B\ a)) =$
 $(\sum a \in fset\ A. \sum b \in fset\ (B\ a). h\ (a, b))$
by *(subst sum.Sigma) auto*
then show *?thesis*
unfolding *fsum-fset-sum*
by *(simp add: set-eq)*
qed

lemma *fsum-fbind-fimage-Pair-right*:
 $fsum\ h\ (fbind\ A\ (\lambda a. fimage\ (\lambda b. (b, a))\ (B\ a))) =$
 $fsum\ (\lambda a. fsum\ (\lambda b. h\ (b, a))\ (B\ a))\ A$
proof –
have *set-eq*:
 $fset\ (fbind\ A\ (\lambda a. fimage\ (\lambda b. (b, a))\ (B\ a))) =$
 $(\bigcup a \in fset\ A. (\lambda b. (b, a))\ ' fset\ (B\ a))$
by *(auto simp: fbind.rep-eq fimage.rep-eq Set.bind-def)*
have $sum\ h\ (\bigcup a \in fset\ A. (\lambda b. (b, a))\ ' fset\ (B\ a)) =$

$(\sum_{a \in fset\ A} \text{sum } h\ ((\lambda b. (b, a))\ 'fset\ (B\ a)))$
by (*subst sum.UNION-disjoint*) *auto*
also have ... = $(\sum_{a \in fset\ A} \sum_{b \in fset\ (B\ a)} h\ (b, a))$
by (*intro sum.cong refl, subst sum.reindex*) (*auto simp: inj-on-def*)
finally have *sum-eq*:
 $\text{sum } h\ (\bigcup_{a \in fset\ A} (\lambda b. (b, a))\ 'fset\ (B\ a)) =$
 $(\sum_{a \in fset\ A} \sum_{b \in fset\ (B\ a)} h\ (b, a))\ .$
then show *?thesis*
unfolding *fsum-fset-sum*
by (*simp add: set-eq*)
qed

lemma *sum-fset-by-bind-items*:

sum-fset-by x
 $(\lambda(ms, ks). \text{fst } ks)$
 $(\lambda(ms, ks). \text{snd } ms * \text{snd } ks)$
 $(\text{fbind } (items\ d)\ (\lambda ms.$
 $\text{case } ms \text{ of } ((a, s'), p) \Rightarrow$
 $\text{fimage } (\lambda ks. (ms, ks))\ (items\ (k\ a\ s')))) =$
 $(\sum_{i \in dom\ (dist\ d)}.$
 $\text{the } (dist\ d\ i) * \text{option-default } (dist\ (k\ (\text{fst } i)\ (\text{snd } i))\ x))$
unfolding *sum-fset-by-def*
apply (*simp add: fsum-ffilter-if fsum-fbind-fimage-Pair case-prod-unfold*)
unfolding *fsum-items*
apply (*intro sum.cong refl*)
apply (*simp add: fsum-mult-left-nnreal fsum-items-key*)
by (*simp add: dom-def option.case-eq-if*)

lemma *fsum-items-mult-left*:

$fsum\ (\lambda x. c * \text{snd } x)\ (items\ d) = c$
proof –
have $fsum\ (\lambda x. c * \text{snd } x)\ (items\ d) =$
 $c * fsum\ \text{snd } (items\ d)$
by (*rule fsum-mult-left-nnreal*)
also have ... = c
by *simp*
finally show *?thesis* .
qed

lemma *fsum-bind-items-total*:

fsum
 $(\lambda(ms, ks). \text{snd } ms * \text{snd } ks)$
 $(\text{fbind } (items\ d)\ (\lambda ms.$
 $\text{case } ms \text{ of } ((a, s'), p) \Rightarrow$
 $\text{fimage } (\lambda ks. (ms, ks))\ (items\ (k\ a\ s')))) = 1$
proof –
have *fsum*
 $(\lambda(ms, ks). \text{snd } ms * \text{snd } ks)$
 $(\text{fbind } (items\ d)\ (\lambda ms.$

```

      case ms of ((a, s'), p) ⇒
        fimage (λks. (ms, ks)) (items (k a s')) =
fsum
  (λms. fsum (λks. snd ms * snd ks)
    (items (k (fst (fst ms)) (snd (fst ms)))))
  (items d)
by (simp add: fsum-fbind-fimage-Pair case-prod-unfold)

also have ... = fsum snd (items d)
by (intro fsum.cong refl) (simp add: fsum-items-mult-left)

also have ... = 1
by simp

finally show ?thesis .
qed

```

1.14 Mapper

definition *dist-map*:: ('a ⇒ 'b) ⇒ 'a dist ⇒ 'b dist
where
dist-map f a = *dist-of-fset-by* (f ∘ fst) snd (items a)

definition *set-dist* :: 'a dist ⇒ 'a set
where
set-dist d = *dom* (dist d)

lemma *set-dist-dist-of-fset-by-id-subset*:
assumes *total*: fsum weight I = 1
shows *set-dist* (dist-of-fset-by id weight I) ⊆ fset I
using *dist-of-fset-by-id-apply*[OF *total*]
unfolding *set-dist-def*
by (auto simp: Let-def split: if-splits)

1.15 Relator

definition *rel-dist* :: ('a ⇒ 'b ⇒ bool) ⇒ 'a dist ⇒ 'b dist ⇒ bool
where
rel-dist R p q ⟷
 (∃ r :: ('a × 'b) dist.
 set-dist r ⊆ {(x, y). R x y} ∧
 dist-map fst r = p ∧
 dist-map snd r = q)

lemma *nnreal-mult-div-cancel*:
fixes a b :: nnreal
assumes b ≠ 0
shows a * b / b = a
using *assms* **by** transfer simp

1.16 BNF Setup

definition *glue-items* ::

$('a \times 'b) \text{ dist} \Rightarrow ('b \times 'c) \text{ dist} \Rightarrow$
 $((('a \times 'b) \times \text{prob}) \times (('b \times 'c) \times \text{prob})) \text{ fset}$

where

$\text{glue-items } p \ q =$
 $\text{fbind } (\text{items } p)$
 $(\lambda ab. \text{fimage } (\lambda bc. (ab, bc))$
 $(\text{ffilter } (\lambda bc. \text{snd } (\text{fst } ab) = \text{fst } (\text{fst } bc)) (\text{items } q)))$

definition *glue-weight* ::

$('a \times 'b) \text{ dist} \Rightarrow$
 $((('a \times 'b) \times \text{prob}) \times (('b \times 'c) \times \text{prob})) \Rightarrow \text{prob}$

where

$\text{glue-weight } p \ x =$
 $\text{snd } (\text{fst } x) * \text{snd } (\text{snd } x) /$
 $\text{the } (\text{dist } (\text{dist-map } \text{snd } p) (\text{snd } (\text{fst } (\text{fst } x))))$

definition *glue-key* ::

$((('a \times 'b) \times \text{prob}) \times (('b \times 'c) \times \text{prob})) \Rightarrow 'a \times 'c$

where

$\text{glue-key } x = (\text{fst } (\text{fst } (\text{fst } x)), \text{snd } (\text{fst } (\text{snd } x)))$

definition *glue-source* ::

$('a \times 'b) \text{ dist} \Rightarrow ('b \times 'c) \text{ dist} \Rightarrow$
 $((('a \times 'b) \times \text{prob}) \times (('b \times 'c) \times \text{prob})) \text{ dist}$

where

$\text{glue-source } p \ q = \text{dist-of-fset-by id } (\text{glue-weight } p) (\text{glue-items } p \ q)$

definition *glue-dist* :: $('a \times 'b) \text{ dist} \Rightarrow ('b \times 'c) \text{ dist} \Rightarrow ('a \times 'c) \text{ dist}$

where

$\text{glue-dist } p \ q = \text{dist-map glue-key } (\text{glue-source } p \ q)$

lemma *dist-map-rep-eq*:

$\text{dist } (\text{dist-map } f \ d) = \text{map-of-fset-by } (f \circ \text{fst}) \ \text{snd } (\text{items } d)$
unfolding *dist-map-def*
by (*rule dist-of-fset-by-rep-eq-1*) *simp*

lemma *sum-fset-by-items*:

$\text{sum-fset-by } x \ (f \circ \text{fst}) \ \text{snd } (\text{items } d) =$
 $(\sum_{y \in \{y \in \text{dom } (\text{dist } d). f \ y = x\}} \text{the } (\text{dist } d \ y))$

proof –

let $?g = \lambda y. (y, \text{the } (\text{dist } d \ y))$
let $?S = \{y \in \text{dom } (\text{dist } d). f \ y = x\}$
have *inj*: *inj-on* $?g \ \{y \in \text{dom } (\text{dist } d). f \ y = x\}$
by (*auto intro!*: *inj-onI*)
have *filt*:
 $\{i \in ?g \text{ ‘ dom } (\text{dist } d). (f \circ \text{fst}) \ i = x\} =$
 $?g \text{ ‘ } \{y \in \text{dom } (\text{dist } d). f \ y = x\}$


```

  by auto
have set-eq:
  {i. (f ∘ fst) i = x} ∩ fset (items d) = ?g ‘ ?S
  by (auto simp: fset-items)
have sum snd ({i. (f ∘ fst) i = x} ∩ fset (items d)) =
  sum snd (?g ‘ ?S)
  using set-eq by simp
also have ... = (∑ y∈?S. snd (?g y))
  by (simp add: sum.reindex[OF inj])
also have ... = (∑ y∈?S. the (dist d y))
  by simp
finally have sum-eq:
  sum snd ({i. (f ∘ fst) i = x} ∩ fset (items d)) =
    (∑ y∈?S. the (dist d y)) .
have set-eq':
  {i. i |∈| items d ∧ (f ∘ fst) i = x} = ?g ‘ ?S
  by (auto simp: fset-items)
have sum-eq':
  (∑ i | i |∈| items d ∧ (f ∘ fst) i = x. snd i) =
    (∑ y∈?S. the (dist d y))
  using set-eq' by (simp add: sum.reindex[OF inj])
show ?thesis
  unfolding sum-fset-by-def fsum-fset-sum
  using sum-eq by (simp add: ffilter.rep-eq Int-def)
qed

lemma dist-map-apply:
  dist (dist-map f d) x =
    (let p = (∑ y∈{y ∈ dom (dist d). f y = x}. the (dist d y))
    in if p = 0 then None else Some p)
proof -
  have total: fsum snd (items d) ≠ 0
    by simp
  show ?thesis
    unfolding dist-map-rep-eq map-of-fset-by-def sum-fset-by-items
    using total by (simp add: Let-def)
qed

lemma fsum-items-filter-dist-map:
  fsum snd (ffilter (λi. f (fst i) = x) (items d)) =
    (∑ y∈{y ∈ dom (dist d). f y = x}. the (dist d y))
  using sum-fset-by-items[of x f d]
  by (simp add: sum-fset-by-def comp-def)

lemma fsum-items-filter-dist-map-on-dom:
  assumes x ∈ dom (dist (dist-map f d))
  shows fsum snd (ffilter (λi. f (fst i) = x) (items d)) =
    the (dist (dist-map f d) x)
proof -

```

```

let ?p = (∑ y ∈ {y ∈ dom (dist d). f y = x}. the (dist d y))
have dist (dist-map f d) x = Some ?p
  using assms by (simp add: dist-map-apply Let-def domIff split: if-splits)
then show ?thesis
  by (simp add: fsum-items-filter-dist-map)
qed

```

```

lemma dist-map-dist-of-fset-by-id-apply:
  assumes total: fsum weight I = 1
  shows dist (dist-map f (dist-of-fset-by id weight I)) x =
    (let p = fsum weight (ffilter (λi. f i = x) I)
     in if p = 0 then None else Some p)

```

```

proof -
  let ?d = dist-of-fset-by id weight I
  let ?D = {y ∈ fset I. weight y ≠ 0}
  let ?S = {y ∈ dom (dist ?d). f y = x}
  let ?T = {y ∈ fset I. f y = x}
  have T-eq: ?T = {i. f i = x} ∩ fset I
    by auto
  have dom-eq: dom (dist ?d) = ?D
    using dist-of-fset-by-id-apply[OF total]
    by (auto simp: Let-def split: if-splits)
  have sum-eq:
    (∑ y ∈ ?S. the (dist ?d y)) = (∑ y ∈ ?T. weight y)
  proof -
    have (∑ y ∈ ?S. the (dist ?d y)) = (∑ y ∈ {y ∈ ?T. weight y ≠ 0}. weight y)
      using dist-of-fset-by-id-apply[OF total]
      by (intro sum.cong) (auto simp: dom-eq Let-def split: if-splits)
    also have ... = (∑ y ∈ ?T. weight y)
      by (rule sum.mono-neutral-left) auto
    finally show ?thesis .
  qed
  qed
  show ?thesis
    unfolding dist-map-apply fsum-fset-sum
    using sum-eq by (simp add: ffilter.rep-eq Let-def T-eq)
qed

```

```

lemma set-dist-dist-map:
  set-dist (dist-map f d) = f ' set-dist d
proof -
  have sum-nonzero:
    (∑ y ∈ {y ∈ dom (dist d). f y = f x}. the (dist d y)) ≠ 0
  if x: x ∈ dom (dist d) for x
  proof -
    let ?S = {y ∈ dom (dist d). f y = f x}
    have fin: finite ?S
      by simp
    have xS: x ∈ ?S
      using x by simp

```

```

moreover have  $0 < \text{the } (\text{dist } d \ x)$ 
  using  $x$  by simp
ultimately show ?thesis
  using sum-pos2[OF  $\text{fin } xS$ , of  $\lambda y. \text{the } (\text{dist } d \ y)$ ]
  by auto
qed

show ?thesis
proof
  show  $\text{set-dist } (\text{dist-map } f \ d) \subseteq f \text{ ' set-dist } d$ 
  proof
    fix  $z$ 
    assume  $z: z \in \text{set-dist } (\text{dist-map } f \ d)$ 
    let  $?S = \{y \in \text{dom } (\text{dist } d). f \ y = z\}$ 
    have  $\text{fin: finite } ?S$ 
    by simp
    have  $p\text{-ne: } (\sum y \in ?S. \text{the } (\text{dist } d \ y)) \neq 0$ 
    using  $z$  unfolding set-dist-def dist-map-apply
    by (auto simp: Let-def split: if-splits)
    have  $\exists y. y \in ?S$ 
    proof (rule ccontr)
      assume  $\neg (\exists y. y \in ?S)$ 
      then have  $?S = \{\}$ 
      by auto
      then show False
      using  $p\text{-ne}$  by (metis sum.empty)
    qed
    then obtain  $y$  where  $y: y \in ?S$ 
    by blast
    then show  $z \in f \text{ ' set-dist } d$ 
    unfolding set-dist-def by auto
  qed
next
  show  $f \text{ ' set-dist } d \subseteq \text{set-dist } (\text{dist-map } f \ d)$ 
  proof
    fix  $z$ 
    assume  $z \in f \text{ ' set-dist } d$ 
    then obtain  $x$  where  $x: x \in \text{dom } (\text{dist } d)$  and  $z: z = f \ x$ 
    unfolding set-dist-def by auto
    have  $(\sum y \in \{y \in \text{dom } (\text{dist } d). f \ y = z\}. \text{the } (\text{dist } d \ y)) \neq 0$ 
    using sum-nonzero[OF  $x$ ]  $z$  by simp
    then show  $z \in \text{set-dist } (\text{dist-map } f \ d)$ 
    unfolding set-dist-def dist-map-apply
    by (simp add: Let-def domIff)
  qed
qed
qed

```

lemma *fsum-glue-weight-inner*:

```

fixes  $p :: ('a \times 'b) \text{ dist}$  and  $q :: ('b \times 'c) \text{ dist}$ 
assumes  $\text{mid}: \text{dist-map snd } p = \text{dist-map fst } q$ 
and  $\text{ab}: \text{ab} \in \text{items } p$ 
shows  $\text{fsum } (\lambda bc. \text{glue-weight } p (\text{ab}, bc))$ 
 $(\text{ffilter } (\lambda bc. \text{snd } (\text{fst } ab) = \text{fst } (\text{fst } bc)) (\text{items } q)) = \text{snd } ab$ 
proof –
  let  $?b = \text{snd } (\text{fst } ab)$ 
  let  $?den = \text{the } (\text{dist } (\text{dist-map snd } p) ?b)$ 
  have  $\text{fst-ab-dom}: \text{fst } ab \in \text{dom } (\text{dist } p)$ 
  using  $\text{ab}$  by  $(\text{auto simp: fset-items})$ 
  have  $\text{b-dom}: ?b \in \text{dom } (\text{dist } (\text{dist-map snd } p))$ 
  using  $\text{fst-ab-dom}$   $\text{set-dist-dist-map[of snd } p]$ 
  unfolding  $\text{set-dist-def}$  by  $\text{auto}$ 
  then have  $\text{den-ne}: ?den \neq 0$ 
  by  $\text{simp}$ 
  have  $\text{sum-q}: \text{fsum snd } (\text{ffilter } (\lambda bc. \text{fst } (\text{fst } bc) = ?b) (\text{items } q)) = ?den$ 
  using  $\text{fsum-items-filter-dist-map-on-dom[of ?b fst } q]$   $\text{mid b-dom}$ 
  by  $\text{simp}$ 
  have  $\text{fsum } (\lambda bc. \text{glue-weight } p (\text{ab}, bc))$ 
 $(\text{ffilter } (\lambda bc. \text{snd } (\text{fst } ab) = \text{fst } (\text{fst } bc)) (\text{items } q)) =$ 
 $\text{fsum } (\lambda bc. \text{snd } ab * \text{snd } bc / ?den)$ 
 $(\text{ffilter } (\lambda bc. \text{fst } (\text{fst } bc) = ?b) (\text{items } q))$ 
  unfolding  $\text{glue-weight-def}$  by  $(\text{simp add: eq-commute})$ 
  also have  $\dots =$ 
 $\text{snd } ab * \text{fsum snd } (\text{ffilter } (\lambda bc. \text{fst } (\text{fst } bc) = ?b) (\text{items } q)) / ?den$ 
  by  $(\text{simp add: fsum-divide-nnreal fsum-mult-left-nnreal})$ 
  also have  $\dots = \text{snd } ab$ 
  using  $\text{sum-q den-ne}$  by  $(\text{simp add: nnreal-mult-div-cancel})$ 
  finally show  $?thesis$  .
qed

```

```

lemma  $\text{fsum-glue-weight-inner-left}$ :
fixes  $p :: ('a \times 'b) \text{ dist}$  and  $q :: ('b \times 'c) \text{ dist}$ 
assumes  $\text{mid}: \text{dist-map snd } p = \text{dist-map fst } q$ 
and  $\text{bc}: \text{bc} \in \text{items } q$ 
shows  $\text{fsum } (\lambda ab. \text{glue-weight } p (ab, bc))$ 
 $(\text{ffilter } (\lambda ab. \text{snd } (\text{fst } ab) = \text{fst } (\text{fst } bc)) (\text{items } p)) = \text{snd } bc$ 
proof –
  let  $?b = \text{fst } (\text{fst } bc)$ 
  let  $?den = \text{the } (\text{dist } (\text{dist-map snd } p) ?b)$ 
  have  $\text{fst-bc-dom}: \text{fst } bc \in \text{dom } (\text{dist } q)$ 
  using  $\text{bc}$  by  $(\text{auto simp: fset-items})$ 
  have  $\text{b-dom-q}: ?b \in \text{dom } (\text{dist } (\text{dist-map fst } q))$ 
  using  $\text{fst-bc-dom}$   $\text{set-dist-dist-map[of fst } q]$ 
  unfolding  $\text{set-dist-def}$  by  $\text{auto}$ 
  then have  $\text{b-dom-p}: ?b \in \text{dom } (\text{dist } (\text{dist-map snd } p))$ 
  using  $\text{mid}$  by  $\text{simp}$ 
  then have  $\text{den-ne}: ?den \neq 0$ 

```

```

  by simp
have sum-p:
  fsum snd (ffilter (λab. snd (fst ab) = ?b) (items p)) = ?den
  using fsum-items-filter-dist-map-on-dom[of ?b snd p] b-dom-p
  by simp
have fsum (λab. glue-weight p (ab, bc))
  (ffilter (λab. snd (fst ab) = fst (fst bc)) (items p)) =
  fsum (λab. snd bc * snd ab / ?den)
    (ffilter (λab. snd (fst ab) = ?b) (items p))
  unfolding glue-weight-def by (simp add: mult.commute)
also have ... =
  snd bc * fsum snd (ffilter (λab. snd (fst ab) = ?b) (items p)) / ?den
  by (simp add: fsum-divide-nnreal fsum-mult-left-nnreal)
also have ... = snd bc
  using sum-p den-ne by (simp add: nnreal-mult-div-cancel)
finally show ?thesis .
qed

```

```

lemma glue-items-commute:
  glue-items p q =
  fbind (items q)
    (λbc. fimage (λab. (ab, bc))
      (ffilter (λab. snd (fst ab) = fst (fst bc)) (items p)))
  unfolding glue-items-def
  by (rule fset-eqI) (auto simp: fbind.rep-eq fimage.rep-eq Set.bind-def)

```

```

lemma fsum-glue-weight:
  fixes p :: ('a × 'b) dist and q :: ('b × 'c) dist
  assumes mid: dist-map snd p = dist-map fst q
  shows fsum (glue-weight p) (glue-items p q) = 1
proof -
  have fsum (glue-weight p) (glue-items p q) =
    fsum (λab. fsum (λbc. glue-weight p (ab, bc))
      (ffilter (λbc. snd (fst ab) = fst (fst bc)) (items q))) (items p)
    unfolding glue-items-def
    by (simp add: fsum-fbind-fimage-Pair)
  also have ... = fsum snd (items p)
    by (intro fsum.cong refl) (simp add: fsum-glue-weight-inner[OF mid])
  also have ... = 1
    by simp
  finally show ?thesis .
qed

```

```

lemma fsum-glue-weight-fst:
  fixes p :: ('a × 'b) dist and q :: ('b × 'c) dist
  assumes mid: dist-map snd p = dist-map fst q
  shows fsum (glue-weight p)
    (ffilter (λi. fst (glue-key i) = a) (glue-items p q)) =
    fsum snd (ffilter (λab. fst (fst ab) = a) (items p))

```

proof –
have $fsum\ (glue-weight\ p)$
 $(ffilter\ (\lambda i. fst\ (glue-key\ i) = a)\ (glue-items\ p\ q)) =$
 $fsum\ (\lambda ab. fsum\ (\lambda bc.$
 $\quad if\ fst\ (fst\ ab) = a\ then\ glue-weight\ p\ (ab,\ bc)\ else\ 0)$
 $\quad (ffilter\ (\lambda bc. snd\ (fst\ ab) = fst\ (fst\ bc))\ (items\ q)))\ (items\ p)$
unfolding $glue-items-def$
by ($simp\ add: fsum-ffilter-if\ fsum-fbind-fimage-Pair\ glue-key-def$)
also have $\dots = fsum\ (\lambda ab. if\ fst\ (fst\ ab) = a\ then\ snd\ ab\ else\ 0)\ (items\ p)$
proof ($intro\ fsum.cong\ refl$)
fix ab
assume $ab: ab \in items\ p$
show $fsum\ (\lambda bc. if\ fst\ (fst\ ab) = a\ then\ glue-weight\ p\ (ab,\ bc)\ else\ 0)$
 $(ffilter\ (\lambda bc. snd\ (fst\ ab) = fst\ (fst\ bc))\ (items\ q)) =$
 $(if\ fst\ (fst\ ab) = a\ then\ snd\ ab\ else\ 0)$
proof ($cases\ fst\ (fst\ ab) = a$)
case $True$
then show $?thesis$
using $fsum-glue-weight-inner[OF\ mid\ ab]$ **by** $simp$
next
case $False$
then show $?thesis$
by ($simp\ add: fsum-fset-sum$)
qed
qed
also have $\dots = fsum\ snd\ (ffilter\ (\lambda ab. fst\ (fst\ ab) = a)\ (items\ p))$
by ($simp\ add: fsum-ffilter-if$)
finally show $?thesis$.
qed

lemma $fsum-glue-weight-snd$:
fixes $p :: ('a \times 'b)\ dist$ **and** $q :: ('b \times 'c)\ dist$
assumes $mid: dist-map\ snd\ p = dist-map\ fst\ q$
shows $fsum\ (glue-weight\ p)$
 $(ffilter\ (\lambda i. snd\ (glue-key\ i) = c)\ (glue-items\ p\ q)) =$
 $fsum\ snd\ (ffilter\ (\lambda bc. snd\ (fst\ bc) = c)\ (items\ q))$
proof –
have $fsum\ (glue-weight\ p)$
 $(ffilter\ (\lambda i. snd\ (glue-key\ i) = c)\ (glue-items\ p\ q)) =$
 $fsum\ (\lambda bc. fsum\ (\lambda ab.$
 $\quad if\ snd\ (fst\ bc) = c\ then\ glue-weight\ p\ (ab,\ bc)\ else\ 0)$
 $\quad (ffilter\ (\lambda ab. snd\ (fst\ ab) = fst\ (fst\ bc))\ (items\ p)))\ (items\ q)$
unfolding $glue-items-commute$
by ($simp\ add: fsum-ffilter-if\ fsum-fbind-fimage-Pair-right\ glue-key-def$)
also have $\dots = fsum\ (\lambda bc. if\ snd\ (fst\ bc) = c\ then\ snd\ bc\ else\ 0)\ (items\ q)$
proof ($intro\ fsum.cong\ refl$)
fix bc
assume $bc: bc \in items\ q$
show $fsum\ (\lambda ab. if\ snd\ (fst\ bc) = c\ then\ glue-weight\ p\ (ab,\ bc)\ else\ 0)$

```

      (ffilter (λab. snd (fst ab) = fst (fst bc)) (items p)) =
      (if snd (fst bc) = c then snd bc else 0)
proof (cases snd (fst bc) = c)
  case True
  then show ?thesis
    using fsum-glue-weight-inner-left[OF mid bc] by simp
next
  case False
  then show ?thesis
    by (simp add: fsum-fset-sum)
qed
qed
also have ... = fsum snd (ffilter (λbc. snd (fst bc) = c) (items q))
  by (simp add: fsum-ffilter-if)
finally show ?thesis .
qed

```

```

lemma dist-map-id:
  dist-map id = id
proof (rule ext, rule dist-inject[THEN iffD1], rule ext)
  fix d::'a dist and x::'a
  show dist.dist (dist-map id d) x = dist.dist (id d) x
  proof (cases x ∈ dom (dist d))
  case True
  have S: {y ∈ dom (dist d). id y = x} = {x}
    using True by auto
  have D: dist d x = Some (the (dist d x))
    using True by auto
  have NZ: the (dist d x) ≠ 0
    using True by simp
  have dist (dist-map id d) x =
    (let p = (∑ y ∈ {y ∈ dom (dist d). id y = x}. the (dist d y))
     in if p = 0 then None else Some p)
    by (rule dist-map-apply)
  also have ... = Some (the (dist d x))
    using S NZ by simp
  also have ... = dist d x
    using D by simp
  finally show ?thesis by simp
next
  case False
  have S: {y ∈ dom (dist d). id y = x} = {}
    using False by auto
  have S': {y. y = x ∧ y ∈ dom (dist d)} = {}
    using False by auto
  have p0: (∑ y | y = x ∧ y ∈ dom (dist d). the (dist d y)) = 0
    by (simp add: S')
  have D: dist d x = None
    using False by auto

```

```

have dist (dist-map id d) x =
  (let p = ( $\sum y \in \{y \in \text{dom } (\text{dist } d). \text{ id } y = x\}. \text{ the } (\text{dist } d \ y)$ )
    in if p = 0 then None else Some p)
  by (rule dist-map-apply)
also have ... = None
  using S S' p0 by simp
also have ... = dist d x
  using D by simp
finally show ?thesis by simp
qed
qed

lemma dist-map-delta-dist[simp]:
  dist-map f (delta-dist x) = delta-dist (f x)
proof (rule dist-inject[THEN iffD1], rule ext)
  fix y
  show dist (dist-map f (delta-dist x)) y = dist (delta-dist (f x)) y
  proof (cases f x = y)
    case True
    have  $\{z \in \text{dom } (\text{dist } (\text{delta-dist } x)). f \ z = y\} = \{x\}$ 
    using True by (auto simp: dist-delta-dist delta-map-def)
    then show ?thesis
    using True by (simp add: dist-map-apply dist-delta-dist delta-map-def)
  next
  case False
  have  $\{z \in \text{dom } (\text{dist } (\text{delta-dist } x)). f \ z = y\} = \{\}$ 
  using False by (auto simp: dist-delta-dist delta-map-def)
  then show ?thesis
  using False by (simp add: dist-map-apply dist-delta-dist delta-map-def)
  qed
qed

lemma dist-map-cong:
  assumes  $\bigwedge x. x \in \text{set-dist } d \implies f \ x = g \ x$ 
  shows dist-map f d = dist-map g d
proof (rule dist-inject[THEN iffD1], rule ext)
  fix y
  have  $\{x \in \text{dom } (\text{dist } d). f \ x = y\} = \{x \in \text{dom } (\text{dist } d). g \ x = y\}$ 
  using assms unfolding set-dist-def by (auto simp: domIff)
  then show dist (dist-map f d) y = dist (dist-map g d) y
  by (simp add: dist-map-apply)
qed

lemma sum-partition-by-image:
  fixes w :: 'a  $\Rightarrow$  nnreal
  assumes fin: finite M
  shows
    ( $\sum y \in \{y \in f \text{ ' } M. g \ y = x\}. \sum z \in \{z \in M. f \ z = y\}. w \ z$ ) =
    ( $\sum z \in \{z \in M. g \ (f \ z) = x\}. w \ z$ )

```



```

proof –
  let ?Y = {y ∈ f ‘ M. g y = x}
  have finY: finite ?Y
    using fin by simp
  have finM: finite M
    by (rule fin)
  have filter-sum:
    (∑ z∈{z ∈ M. f z = y}. w z) =
    (∑ z∈M. if f z = y then w z else 0) for y
  proof –
    have (∑ z∈M. if f z = y then w z else 0) =
      (∑ z∈{z ∈ M. f z = y}. w z)
      using fin
      by (intro sum.mono-neutral-cong-right) auto
    then show ?thesis by simp
  qed

  have (∑ y∈?Y. ∑ z∈{z ∈ M. f z = y}. w z) =
    (∑ y∈?Y. ∑ z∈M. if f z = y then w z else 0)
    by (rule sum.cong) (simp-all add: filter-sum)
  also have ... =
    (∑ z∈M. ∑ y∈?Y. if f z = y then w z else 0)
    by (rule sum.swap)
  also have ... =
    (∑ z∈M. if g (f z) = x then w z else 0)
  proof (intro sum.cong refl)
    fix z
    assume z: z ∈ M
    show (∑ y∈?Y. if f z = y then w z else 0) =
      (if g (f z) = x then w z else 0)
    proof (cases g (f z) = x)
      case True
        then have fz: f z ∈ ?Y
          using z by simp
        have (∑ y∈?Y. if f z = y then w z else 0) =
          (∑ y∈?Y. if y = f z then w z else 0)
          by (intro sum.cong refl) simp
        also have ... = w z
          using finY fz by (simp)
        finally show ?thesis
          using True by simp
      case False
        then have ∧y. y ∈ ?Y ⇒ f z ≠ y
          by auto
        then show ?thesis
          using False by simp
    qed
  qed

```

```

also have ... = ( $\sum z \in \{z \in M. g (f z) = x\}. w z$ )
  using finM by (rule sum.inter-filter[symmetric])
finally show ?thesis .
qed

lemma dist-map-comp:
  dist-map ( $g \circ f$ ) = dist-map  $g \circ$  dist-map  $f$ 
proof (rule ext)
  fix  $d$ 
  show dist-map ( $g \circ f$ )  $d$  = (dist-map  $g \circ$  dist-map  $f$ )  $d$ 
  proof (rule dist-inject[THEN iffD1], rule ext)
    fix  $x$ 
    let ? $M$  = dom (dist  $d$ )
    let ? $lhs$  = ( $\sum y \in \{y \in \text{dom} (\text{dist} (\text{dist-map } f \ d)). g \ y = x\}. \text{the} (\text{dist} (\text{dist-map } f \ d) \ y))$ )
    let ? $rhs$  = ( $\sum z \in \{z \in ?M. (g \circ f) \ z = x\}. \text{the} (\text{dist } d \ z)$ )

    have dom-map: dom (dist (dist-map  $f \ d$ )) =  $f \ ' \ ?M$ 
      using set-dist-dist-map[of f d]
      unfolding set-dist-def by simp

    have lhs-sum:
      ? $lhs$  =
        ( $\sum y \in \{y \in f \ ' \ ?M. g \ y = x\}. \sum z \in \{z \in ?M. f \ z = y\}. \text{the} (\text{dist } d \ z)$ )
    proof -
      have Aeq:
         $\{y \in \text{dom} (\text{dist} (\text{dist-map } f \ d)). g \ y = x\} = \{y \in f \ ' \ ?M. g \ y = x\}$ 
        by (simp add: dom-map)
      show ?thesis
        unfolding Aeq
    proof (intro sum.cong refl)
      fix  $y$ 
      assume  $y \in \{y \in f \ ' \ ?M. g \ y = x\}$ 
      then have y-dom:  $y \in \text{dom} (\text{dist} (\text{dist-map } f \ d))$ 
        by (simp add: dom-map)
      have nz:
        ( $\sum z \in \{z \in ?M. f \ z = y\}. \text{the} (\text{dist } d \ z)$ )  $\neq 0$ 
        using y-dom unfolding dist-map-apply
        by (auto simp: Let-def split: if-splits)
      have dist (dist-map  $f \ d$ )  $y$  =
        Some ( $\sum z \in \{z \in ?M. f \ z = y\}. \text{the} (\text{dist } d \ z)$ )
        unfolding dist-map-apply using nz by simp
      then show the (dist (dist-map  $f \ d$ )  $y$ ) =
        ( $\sum z \in \{z \in ?M. f \ z = y\}. \text{the} (\text{dist } d \ z)$ )
        by simp
    qed
  qed

```

```

have part:
  ( $\sum_{y \in \{y \in f^{-1} ?M. g y = x\}.}$ 
     $\sum_{z \in \{z \in ?M. f z = y\}.}$  the (dist d z)) =
    ( $\sum_{z \in \{z \in ?M. g (f z) = x\}.}$  the (dist d z))
  by (rule sum-partition-by-image) simp
have sums-eq: ?lhs = ?rhs
  using lhs-sum part by (simp add: comp-def)

have left:
  dist (dist-map ( $\lambda z. g (f z)$ ) d) x =
    (let p = ?rhs in if p = 0 then None else Some p)
  by (simp add: dist-map-apply)
have right:
  dist (dist-map g (dist-map f d)) x =
    (let p = ?lhs in if p = 0 then None else Some p)
  by (simp add: dist-map-apply)

show dist (dist-map (g  $\circ$  f) d) x =
  dist ((dist-map g  $\circ$  dist-map f) d) x
  unfolding comp-def
  using left right sums-eq by simp
qed
qed

lemma dist-map-fst-glue-dist:
  fixes p :: ('a  $\times$  'b) dist and q :: ('b  $\times$  'c) dist
  assumes mid: dist-map snd p = dist-map fst q
  shows dist-map fst (glue-dist p q) = dist-map fst p
proof (rule dist-inject[THEN iffD1], rule ext)
  fix a
  have total: fsum (glue-weight p) (glue-items p q) = 1
  by (rule fsum-glue-weight[OF mid])
  have comp-fst:
    dist-map fst (glue-dist p q) = dist-map (fst  $\circ$  glue-key) (glue-source p q)
    unfolding glue-dist-def
    by (metis comp-apply dist-map-comp)
  have left:
    dist (dist-map fst (glue-dist p q)) a =
      (let s = fsum (glue-weight p)
        (ffilter ( $\lambda i. \text{fst } (\text{glue-key } i) = a$ ) (glue-items p q))
        in if s = 0 then None else Some s)
      unfolding comp-fst glue-source-def
      using dist-map-dist-of-fset-by-id-apply[OF total, of fst  $\circ$  glue-key a]
      by (simp add: comp-def)
  have right:
    dist (dist-map fst p) a =
      (let s = fsum snd (ffilter ( $\lambda ab. \text{fst } (\text{fst } ab) = a$ ) (items p))
        in if s = 0 then None else Some s)

```

```

    by (simp add: dist-map-apply fsum-items-filter-dist-map[symmetric])
  show dist (dist-map fst (glue-dist p q)) a = dist (dist-map fst p) a
    using left right fsum-glue-weight-fst[OF mid, of a] by simp
qed

lemma dist-map-snd-glue-dist:
  fixes p :: ('a × 'b) dist and q :: ('b × 'c) dist
  assumes mid: dist-map snd p = dist-map fst q
  shows dist-map snd (glue-dist p q) = dist-map snd q
proof (rule dist-inject[THEN iffD1], rule ext)
  fix c
  have total: fsum (glue-weight p) (glue-items p q) = 1
    by (rule fsum-glue-weight[OF mid])
  have comp-snd:
    dist-map snd (glue-dist p q) = dist-map (snd ∘ glue-key) (glue-source p q)
  unfolding glue-dist-def
  by (metis comp-apply dist-map-comp)
  have left:
    dist (dist-map snd (glue-dist p q)) c =
      (let s = fsum (glue-weight p)
        (ffilter (λi. snd (glue-key i) = c) (glue-items p q))
        in if s = 0 then None else Some s)
  unfolding comp-snd glue-source-def
  using dist-map-dist-of-fset-by-id-apply[OF total, of snd ∘ glue-key c]
  by (simp add: comp-def)
  have right:
    dist (dist-map snd q) c =
      (let s = fsum snd (ffilter (λbc. snd (fst bc) = c) (items q))
        in if s = 0 then None else Some s)
  by (simp add: dist-map-apply fsum-items-filter-dist-map[symmetric])
  show dist (dist-map snd (glue-dist p q)) c = dist (dist-map snd q) c
    using left right fsum-glue-weight-snd[OF mid, of c] by simp
qed

```

```

lemma set-dist-glue-dist-subset:
  fixes p :: ('a × 'b) dist and q :: ('b × 'c) dist
  assumes mid: dist-map snd p = dist-map fst q
  and Rp: set-dist p ⊆ {(a, b). R a b}
  and Sq: set-dist q ⊆ {(b, c). S b c}
  shows set-dist (glue-dist p q) ⊆ {(a, c). (R OO S) a c}
proof
  fix ac
  assume ac: ac ∈ set-dist (glue-dist p q)
  let ?src = glue-source p q
  have total: fsum (glue-weight p) (glue-items p q) = 1
    by (rule fsum-glue-weight[OF mid])
  obtain x where x-src: x ∈ set-dist ?src and ac-x: ac = glue-key x
  using ac set-dist-dist-map[of glue-key ?src]
  unfolding glue-dist-def by auto

```

```

have x-I:  $x \in \text{fset } (\text{glue-items } p \ q)$ 
  using x-src set-dist-dist-of-fset-by-id-subset[OF total]
  unfolding glue-source-def by auto
then obtain ab bc where  $x = (ab, bc)$ 
  and ab:  $ab \in \text{items } p$ 
  and bc:  $bc \in \text{items } q$ 
  and match:  $\text{snd } (\text{fst } ab) = \text{fst } (\text{fst } bc)$ 
  unfolding glue-items-def
  by (auto simp: fbind.rep-eq fimage.rep-eq Set.bind-def)
have ab-supp:  $\text{fst } ab \in \text{set-dist } p$ 
  using ab unfolding set-dist-def by (auto simp: fset-items)
have bc-supp:  $\text{fst } bc \in \text{set-dist } q$ 
  using bc unfolding set-dist-def by (auto simp: fset-items)
show  $ac \in \{(a, c). (R \ OO \ S) \ a \ c\}$ 
  using Rp Sq ab-supp bc-supp match ac-x
  unfolding x glue-key-def
  by (cases fst ab; cases fst bc; auto)
qed

lemma rel-dist-OO:
   $(\text{rel-dist } R \ OO \ \text{rel-dist } S) \leq \text{rel-dist } (R \ OO \ S)$ 
proof
  fix x z
  assume  $(\text{rel-dist } R \ OO \ \text{rel-dist } S) \ x \ z$ 
  then obtain y where  $xy: \text{rel-dist } R \ x \ y$  and  $yz: \text{rel-dist } S \ y \ z$ 
    by auto
  obtain p where  $pR: \text{set-dist } p \subseteq \{(a, b). R \ a \ b\}$ 
    and p-fst:  $\text{dist-map } \text{fst } p = x$ 
    and p-snd:  $\text{dist-map } \text{snd } p = y$ 
    using xy unfolding rel-dist-def by auto
  obtain q where  $qS: \text{set-dist } q \subseteq \{(b, c). S \ b \ c\}$ 
    and q-fst:  $\text{dist-map } \text{fst } q = y$ 
    and q-snd:  $\text{dist-map } \text{snd } q = z$ 
    using yz unfolding rel-dist-def by auto
  have mid:  $\text{dist-map } \text{snd } p = \text{dist-map } \text{fst } q$ 
    using p-snd q-fst by simp
  have  $\text{rel-dist } (R \ OO \ S) \ x \ z$ 
    unfolding rel-dist-def
  proof (intro exI conjI)
    show  $\text{set-dist } (\text{glue-dist } p \ q) \subseteq \{(a, c). (R \ OO \ S) \ a \ c\}$ 
      by (rule set-dist-glue-dist-subset[OF mid pR qS])
    show  $\text{dist-map } \text{fst } (\text{glue-dist } p \ q) = x$ 
      using dist-map-fst-glue-dist[OF mid] p-fst by simp
    show  $\text{dist-map } \text{snd } (\text{glue-dist } p \ q) = z$ 
      using dist-map-snd-glue-dist[OF mid] q-snd by simp
  qed
  then show  $\text{rel-dist } (R \ OO \ S) \ x \ z$  .
qed

```

```

bnf dist: 'a dist
  map: dist-map
  sets: set-dist
  bd: natLeq
  wits: delta-dist
  rel: rel-dist
proof –
  show dist-map id = id
    by (rule dist-map-id)
next
  show  $\bigwedge f g. \text{dist-map } (g \circ f) = \text{dist-map } g \circ \text{dist-map } f$ 
    by (rule dist-map-comp)
next
  show  $\bigwedge f g. (\bigwedge z. z \in \text{set-dist } x \implies f z = g z) \implies \text{dist-map } f x = \text{dist-map } g x$ 
    by (rule dist-map-cong)
next
  show  $\bigwedge f. \text{set-dist} \circ \text{dist-map } f = (') f \circ \text{set-dist}$ 
    by (simp add: fun-eq-iff set-dist-dist-map)
next
  show card-order natLeq
    by (rule natLeq-card-order)
next
  show BNF-Cardinal-Arithmetic.cinfinite natLeq
    by (rule natLeq-cinfinite)
next
  show regularCard natLeq
    by (rule regularCard-natLeq)
next
  show  $\bigwedge x. \text{ordLess2 } (\text{card-of } (\text{set-dist } x)) \text{ natLeq}$ 
    unfolding set-dist-def
    by (simp add: finite-iff-ordLess-natLeq[symmetric])
next
  show  $\bigwedge R S. \text{rel-dist } R \text{ OO rel-dist } S \leq \text{rel-dist } (R \text{ OO } S)$ 
    by (rule rel-dist-OO)
next
  show  $\bigwedge R. \text{rel-dist } R =$ 
     $(\lambda x y. \exists z. \text{set-dist } z \subseteq \{(x, y). R x y\} \wedge$ 
     $\text{dist-map fst } z = x \wedge \text{dist-map snd } z = y)$ 
    by (auto simp: fun-eq-iff rel-dist-def)
next
  show  $\bigwedge b z. b \in \text{set-dist } (\text{delta-dist } z) \implies b = z$ 
    unfolding set-dist-def by (simp add: dist-delta-dist delta-map-def)
qed

```

1.17 Code Generator Setup

We implement distributions using finite sets of key-value pairs. Note that we use a code datatype here since we want to use a different datatype for implementation as the raw datatype which was used to define the type.

code-datatype *dist-of-fset*

lemma *delta-dist-code*[*code*]:
 $\text{delta-dist } x = \text{dist-of-fset } \{|(x, 1)|\}$
unfolding *delta-dist-def* **by** *simp*

declare [[*code drop: squish*]]

lemma *squish-code*[*code*]:
 $\text{squish } xs = \text{fbind } (\text{fimage } \text{fst } xs) (\lambda x. \{|(x, \text{sum-fset-}x \ x \ xs)|\})$
unfolding *squish-def*
by (*rule fset-eqI*) (*auto simp: fbind.rep-eq fimage.rep-eq Set.bind-def*)

lemma *dist-code*[*code*]: $\text{dist } (\text{dist-of-fset } A) = (\text{let } q = \text{fsum } \text{snd } A \text{ in}$
 $\text{if } q = 0 \text{ then Map.empty(undefined := Some 1)}$
 $\text{else map-of } (\text{norm-fset } A))$
by (*simp add: dist-of-fset.rep-eq map-of-fset-def*)

lemma *dist-uniform-code*[*code*]:
 $\text{dist-uniform } xs = \text{dist-of-fset } ((\lambda x. (x, 1::\text{prob})) \mid^{\cdot} xs)$
unfolding *dist-uniform-def* **by** *simp*

lemma *items-code*[*code*]:
 $\text{items } (\text{dist-of-fset } xs) = \text{norm-fset } xs$
apply *transfer-start*
apply *transfer-step*
apply *transfer-step*
apply *transfer-step*
apply (*rule fset-eq-transfer-aux*)
apply *transfer-step*
apply *transfer-end*
by (*rule raw-items-map-of-fset*)

declare [[*code drop: squish-by*]]

lemma *squish-by-code*[*code*]:
 $\text{squish-by } \text{key } \text{weight } I =$
 $\text{fbind } (\text{fimage } \text{key } I) (\lambda x. \{|(x, \text{sum-fset-by } x \ \text{key } \text{weight } I)|\})$
unfolding *squish-by-def*
by (*rule fset-eqI*) (*auto simp: fbind.rep-eq fimage.rep-eq Set.bind-def*)

1.17.1 Examples

thm *eq-transfer*
instantiation *dist::(type) equal*
begin
lift-definition *equal-dist::'a dist $\Rightarrow$ 'a dist $\Rightarrow$ bool*
is (=) .
instance **by** (*intro-classes, transfer, simp*)

end

lemma *eq-fset-set-transfer*:

assumes *bi-unique A*

shows *Transfer.Rel (rel-fun (pcr-fset A) (rel-fun (pcr-fset A) (==))) (==) (==)*

using *assms FSet.fset.bi-unique eq-transfer*

unfolding *Transfer.Rel-def*

by *blast*

lemma *eq-iff-raw-items-eq*:

fixes *a b*

assumes *finite (dom a) $\wedge$ sum-map a = 1 $\wedge$ ($\forall x \in \text{dom } a. \text{the } (a \ x) \neq 0$)*

and *finite (dom b) $\wedge$ sum-map b = 1 $\wedge$ ($\forall x \in \text{dom } b. \text{the } (b \ x) \neq 0$)*

shows *(a = b) = (raw-items a = raw-items b)*

proof

assume *a = b*

then show *raw-items a = raw-items b*

by *simp*

next

assume *raw-eq: raw-items a = raw-items b*

show *a = b*

proof

fix *x*

show *a x = b x*

proof (*cases x $\in$ dom a*)

case *True*

then have *(x, the (a x)) $\in$ raw-items a*

unfolding *raw-items-def* **by** *blast*

then have *(x, the (a x)) $\in$ raw-items b*

using *raw-eq* **by** *simp*

then have *x $\in$ dom b the (a x) = the (b x)*

unfolding *raw-items-def* **by** *auto*

then show *?thesis*

using *True* **by** (*cases a x; cases b x*) *auto*

next

case *False*

show *?thesis*

proof (*cases x $\in$ dom b*)

case *True*

then have *(x, the (b x)) $\in$ raw-items b*

unfolding *raw-items-def* **by** *blast*

then have *(x, the (b x)) $\in$ raw-items a*

using *raw-eq* **by** *simp*

then have *x $\in$ dom a*

unfolding *raw-items-def* **by** *auto*

then show *?thesis*

using *False* **by** *simp*

next

case *False*


```

    then show ?thesis
    using ⟨ $x \notin \text{dom } a$ ⟩ by (simp add: domIff)
qed
qed
qed
qed

lemma equal-dist-code[code]:
  equal-class.equal a b = (items a = items b)
  apply transfer-start
    apply transfer-step
    apply transfer-step
    apply (rule eq-fset-set-transfer)
    apply transfer-step
    apply transfer-step
    apply transfer-step
    apply transfer-step
    apply transfer-end
  by (rule eq-iff-raw-items-eq)

lemma
  dist-of-fset {|(True, 1), (False, 1), (False, 1)|}
  = dist-of-fset {|(True, 0.5), (False, 0.5)|}
  by eval

lemma
  dist-of-fset {|(1::nat, 1), (2, 1), (2, 1)|}
  = dist-of-fset {|(1::nat, 1), (2, 1), (2, 1)|}
  by eval

lemma dist-uniform {|1,2,3::nat|} = dist-of-fset {|(1, 1), (2, 1), (3, 1)|}
  by eval

lemma
  delta-dist (True) = dist-of-fset {|(True, 1)|}
  by eval

lemma coin 0.5 (1::nat) 0 = dist-of-fset {|(1, (1 / 2)), (0, (1 / 2))|}
  by eval

lemma die 6 = dist-of-fset {|(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (6, 1)|}
  by eval

end

theory Hash-Monad
  imports Main Prob-Monad HOL-Library.Finite-Map HOL.Enum

```

begin

1.18 Finite Map setup

definition *fmap-of-fun* :: (*'a*::*finite* $\Rightarrow$ *'b option*) $\Rightarrow$ (*'a*, *'b*) *fmap* **where**

fmap-of-fun *f* = *Abs-fmap* *f*

lemma *fmlookup-fmap-of-fun*[*simp*]:

fmlookup (*fmap-of-fun* *f*) = *f*

unfolding *fmap-of-fun-def*

by (*simp* *add*: *Abs-fmap-inverse*)

lemma *fmap-of-fun-fmlookup*[*simp*]:

fmap-of-fun (*fmlookup* *m*) = *m*

by (*rule* *fmap-ext*) *simp*

instantiation *fmap* :: (*enum*, *enum*) *enum*

begin

definition *enum-fmap* :: (*'a*, *'b*) *fmap* *list* **where**

enum-fmap = *map* *fmap-of-fun* (*enum-class.enum* :: (*'a* $\Rightarrow$ *'b option*) *list*)

definition *enum-all-fmap* :: ((*'a*, *'b*) *fmap* $\Rightarrow$ *bool*) $\Rightarrow$ *bool* **where**

enum-all-fmap *P* =

enum-class.enum-all ($\lambda f :: 'a \Rightarrow 'b \text{ option. } P \text{ (fmap-of-fun } f)$)

definition *enum-ex-fmap* :: ((*'a*, *'b*) *fmap* $\Rightarrow$ *bool*) $\Rightarrow$ *bool* **where**

enum-ex-fmap *P* =

enum-class.enum-ex ($\lambda f :: 'a \Rightarrow 'b \text{ option. } P \text{ (fmap-of-fun } f)$)

instance

proof *standard*

show *UNIV* = *set* (*enum-class.enum* :: (*'a*, *'b*) *fmap* *list*)

unfolding *enum-fmap-def*

proof (*rule* *subset-antisym*)

show *UNIV* $\subseteq$ *set* (*map* *fmap-of-fun*

(*enum-class.enum* :: (*'a* $\Rightarrow$ *'b option*) *list*))

by (*metis* *UNIV-I* *enum-UNIV* *fmap-of-fun-fmlookup* *image-eqI* *set-map* *subsetI*)

show *set* (*map* *fmap-of-fun*

(*enum-class.enum* :: (*'a* $\Rightarrow$ *'b option*) *list*)) $\subseteq$ *UNIV*

by *simp*

qed

show *distinct* (*enum-class.enum* :: (*'a*, *'b*) *fmap* *list*)

proof –

have *inj-on* *fmap-of-fun* (*set* (*enum-class.enum* :: (*'a* $\Rightarrow$ *'b option*) *list*))

by (*rule* *inj-onI*) (*metis* *fmlookup-fmap-of-fun*)

then show *?thesis*

by (*simp* *add*: *enum-fmap-def* *enum-distinct* *distinct-map*)

qed

show *enum-class.enum-all* *P* = *Ball* *UNIV* *P* **for** *P* :: (*'a*, *'b*) *fmap* $\Rightarrow$ *bool*

apply (*auto* *simp*: *enum-all-fmap-def* *enum-all-UNIV*)

by (*metis* *fmap-of-fun-fmlookup*)

show *enum-class.enum-ex* *P* = *Bex* *UNIV* *P* **for** *P* :: (*'a*, *'b*) *fmap* $\Rightarrow$ *bool*

apply (*auto* *simp*: *enum-ex-fmap-def* *enum-ex-UNIV*)

by (*metis* *fmap-of-fun-fmlookup*)

```
qed
end
```

1.19 Hashing

Protocols should not encode semantically different random-oracle queries as the same raw value before hashing. The STARK protocol uses field-valued hash outputs and messages, but Merkle leaves, Merkle internal nodes, and Fiat-Shamir challenge queries need distinct input domains. The datatype below provides that shared domain-separated input vocabulary.

```
datatype 'a protocol-hash-input =
  MerkleLeaf 'a
| MerkleNode 'a 'a
| FiatShamirChallenge 'a
| TraceFriChallenge nat 'a
| CompositionFriChallenge nat 'a
| AlphaChallenge nat 'a
| QueryIndexChallenge nat 'a
```

```
record ('a, 'b) hash =
  HashMap :: ('a, 'b) fmap
```

```
instantiation hash-ext :: (enum, enum, enum) enum
begin
```

```
definition enum-hash-ext :: ('a, 'b, 'c) hash-ext list where
  enum-hash-ext =
    map ( $\lambda(m :: ('a, 'b) \text{fmap}, z :: 'c). (\text{hash-ext } m \ z :: ('a, 'b, 'c) \text{hash-ext}))$ 
    (List.product
      (enum-class.enum :: ('a, 'b) fmap list)
      (enum-class.enum :: 'c list))
```

```
definition enum-all-hash-ext :: (('a, 'b, 'c) hash-ext  $\Rightarrow$  bool)  $\Rightarrow$  bool where
  enum-all-hash-ext P =
    enum-class.enum-all ( $\lambda m :: ('a, 'b) \text{fmap}.$ 
      enum-class.enum-all ( $\lambda z :: 'c. P (\text{hash-ext } m \ z :: ('a, 'b, 'c) \text{hash-ext}))$ )
```

```
definition enum-ex-hash-ext :: (('a, 'b, 'c) hash-ext  $\Rightarrow$  bool)  $\Rightarrow$  bool where
  enum-ex-hash-ext P =
    enum-class.enum-ex ( $\lambda m :: ('a, 'b) \text{fmap}.$ 
      enum-class.enum-ex ( $\lambda z :: 'c. P (\text{hash-ext } m \ z :: ('a, 'b, 'c) \text{hash-ext}))$ )
```

```
instance
```

```
proof standard
```

```
  show UNIV = set (enum-class.enum :: ('a, 'b, 'c) hash-ext list)
  unfolding enum-hash-ext-def
```

```
proof (rule subset-antisym)
```

```
  show UNIV  $\subseteq$ 
    set (map ( $\lambda(m :: ('a, 'b) \text{fmap}, z :: 'c). (\text{hash-ext } m \ z :: ('a, 'b, 'c) \text{hash-ext}))$ 
      (List.product
        (enum-class.enum :: ('a, 'b) fmap list)
```

```

    (enum-class.enum :: 'c list)))
  by (auto, case-tac x, auto simp: enum-UNIV)
show set (map (λ(m :: ('a, 'b) fmap, z :: 'c).
  (hash-ext m z :: ('a, 'b, 'c) hash-ext))
  (List.product
    (enum-class.enum :: ('a, 'b) fmap list)
    (enum-class.enum :: 'c list))) ⊆ UNIV
  by simp
qed
show distinct (enum-class.enum :: ('a, 'b, 'c) hash-ext list)
  by (auto simp: enum-hash-ext-def enum-distinct distinct-map inj-on-def
    intro!: distinct-product)
show enum-class.enum-all P = Ball UNIV P for P :: ('a, 'b, 'c) hash-ext ⇒ bool
  unfolding enum-all-hash-ext-def
  by (auto simp: enum-all-UNIV; metis hash.ext-surjective)
show enum-class.enum-ex P = Bex UNIV P for P :: ('a, 'b, 'c) hash-ext ⇒ bool
  unfolding enum-ex-hash-ext-def
  by (auto simp: enum-ex-UNIV; metis hash.ext-surjective)
qed
end

type-synonym ('a, 'b, 'c, 'd) hash-monad = ('c, ('a, 'b, 'd) hash-scheme) state-monad

definition hash-default :: 'a::finite dist
  where hash-default = dist-uniform (fUNIV :: 'a fset)

definition option-default-dist :: 'a::finite option ⇒ 'a dist
  where option-default-dist a = case-option hash-default delta-dist a

definition hash-dist :: 'a ⇒ ('a, 'b::finite, 'c) hash-scheme ⇒ 'b dist
  where
    hash-dist a s = option-default-dist (fmlookup (HashMap s) a)

definition modify-HashMap :: 'a ⇒ 'b ⇒ ('a, 'b, unit, 'd) hash-monad where
  modify-HashMap x a = modify (λs. s(HashMap := fmupd x a (HashMap s)))

definition apply-hash :: 'b ⇒ ('b, 'a, 'a::finite, 'c) hash-monad where
  apply-hash x = lift (hash-dist x)

definition hash :: 'b ⇒ ('b, 'a, 'a::finite, 'c) hash-monad
  where
    hash x ≡
    do {
      a ← apply-hash x;
      modify-HashMap x a;
      return a
    }

instantiation fmap :: (type, type) ord

```

begin

definition $less\text{-}eq\ a\ b \equiv (\forall x. fmlookup\ a\ x = None \vee fmlookup\ a\ x = fmlookup\ b\ x)$

definition $less\ a\ b \equiv less\text{-}eq\ a\ b \wedge (\exists x. fmlookup\ a\ x = None \wedge fmlookup\ b\ x \neq None)$

instance ..

end

instantiation $hash\text{-}ext :: (type, type, type)\ ord$

begin

definition $less\text{-}eq\ a\ b \equiv (HashMap\ a) \leq (HashMap\ b)$

definition $less\ a\ b \equiv (HashMap\ a) < (HashMap\ b)$

instance ..

end

definition $less\text{-}eq\text{-}hash :: ('a, 'b)\ hash \Rightarrow ('a, 'b)\ hash \Rightarrow bool$

where $less\text{-}eq\text{-}hash\ (a::('a, 'b)\ hash)\ (b::('a, 'b)\ hash) \equiv (HashMap\ a) \leq (HashMap\ b)$

definition $less\text{-}hash :: ('a, 'b)\ hash \Rightarrow ('a, 'b)\ hash \Rightarrow bool$

where $less\text{-}hash\ (a::('a, 'b)\ hash)\ (b::('a, 'b)\ hash) \equiv (HashMap\ a) < (HashMap\ b)$

lemma $dist\text{-}expect\text{-}eq\text{-}1$:

assumes $\bigwedge x. x \in dom\ (dist\ d) \implies Q\ x = 1$

shows $dist\text{-}expect\ d\ Q = 1$

proof –

have $dist\text{-}expect\ d\ Q = (\sum x \in dom\ (dist\ d). the\ (dist\ d\ x))$

unfolding $dist\text{-}expect\text{-}def$

by $(intro\ sum.cong\ refl)\ (simp\ add: assms)$

also have $\dots = sum\text{-}map\ (dist\ d)$

unfolding $sum\text{-}map\text{-}def$ **by** $simp$

also have $\dots = 1$

by $simp$

finally show $?thesis$.

qed

lemma $hash\text{-}expect\text{-}known$:

assumes $fmlookup\ (HashMap\ s)\ x = Some\ y$

shows

$dist\text{-}expect\ (lift\text{-}dist\ (hash\text{-}dist\ x)\ s)$

$(\lambda r. case\ r\ of\ None \Rightarrow 0 \mid Some\ (z, u) \Rightarrow if\ y = z\ then\ 1\ else\ 0) = 1$

proof –

have $lift\text{-}dist\ (hash\text{-}dist\ x)\ s = delta\text{-}dist\ (Some\ (y, s))$

using $assms$

```

    by (simp add: lift-dist-def hash-dist-def option-default-dist-def)
  then show ?thesis
    by simp
qed

lemma hash-expect-known-extension:
  assumes fmllookup (HashMap s0) x = Some y
  and s0 ≤ t
  shows
    dist-expect (lift-dist (hash-dist x) t)
      (λr. case r of None ⇒ 0 | Some (z, u) ⇒ if y = z then 1 else 0) = 1
proof -
  have fmllookup (HashMap t) x = Some y
  using assms
  unfolding less-eq-hash-ext-def less-eq-fmap-def
  by (metis option.distinct(1))
  then show ?thesis
    by (rule hash-expect-known)
qed

```

```

lemma hash-expect-after-update-extension:
  assumes s0(HashMap := fmupd x y (HashMap s0)) ≤ t
  shows
    dist-expect (lift-dist (hash-dist x) t)
      (λr. case r of None ⇒ 0 | Some (z, u) ⇒ if y = z then 1 else 0) = 1
  using assms
  by (intro hash-expect-known-extension) simp-all

```

```

lemma hash-expect-after-update-extension-dist-map:
  assumes s0(HashMap := fmupd x y (HashMap s0)) ≤ t
  shows
    dist-expect (dist-map (λz. Some (z, t)) (hash-dist x t))
      (λr. case r of None ⇒ 0 | Some (z, u) ⇒ if y = z then 1 else 0) = 1
  using hash-expect-after-update-extension[OF assms]
  by (simp add: lift-dist-def)

```

```

lemma hash-dist-expect-after-update-extension:
  assumes s0(HashMap := fmupd x y (HashMap s0)) ≤ t
  shows dist-expect (hash-dist x t) (λz. if y = z then 1 else 0) = 1
  using hash-expect-after-update-extension[OF assms]
  by (simp add: lift-dist-def dist-expect-map)

```

```

lemma None-notin-dist-map-Some[simp]:
  None ∉ dom (dist (dist-map (λx. Some (x, s)) d))
  using set-dist-dist-map[of λx. Some (x, s) d]
  unfolding set-dist-def
  by auto

```

```

lemma hash-update-extends:

```

```

assumes  $y \in \text{dom } (\text{dist } (\text{hash-dist } x \ s))$ 
shows  $s \leq s \langle \text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s) \rangle$ 
unfolding less-eq-hash-ext-def less-eq-fmap-def
proof (intro allI)
  fix  $z$ 
  show  $\text{fmlookup } (\text{HashMap } s) \ z = \text{None} \vee$ 
     $\text{fmlookup } (\text{HashMap } s) \ z =$ 
       $\text{fmlookup } (\text{HashMap } (s \langle \text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s) \rangle)) \ z$ 
  proof (cases  $\text{fmlookup } (\text{HashMap } s) \ x$ )
    case None
    then show ?thesis
    by (cases  $z = x$ ) simp-all
  next
  case (Some old)
  then have  $y = \text{old}$ 
  using assms
  unfolding hash-dist-def option-default-dist-def
  by (simp add: dist-delta-dist delta-map-def)
  with Some show ?thesis
  by (cases  $z = x$ ) simp-all
qed
qed

lemma dist-expect-then-hash-after-extension:
assumes  $\bigwedge r \ t.$ 
   $\text{Some } (r, t) \in \text{dom } (\text{dist } d) \implies$ 
   $s0 \langle \text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s0) \rangle \leq t$ 
shows
  dist-expect  $d$ 
   $(\lambda r. \text{case } r \text{ of } \text{None} \Rightarrow 1 \mid \text{Some } (r, t) \Rightarrow$ 
     $\text{dist-expect } (\text{lift-dist } (\text{hash-dist } x) \ t)$ 
     $(\lambda z. \text{case } z \text{ of } \text{None} \Rightarrow 0 \mid \text{Some } (z, u) \Rightarrow \text{if } y = z \text{ then } 1 \text{ else } 0)) = 1$ 
proof (rule dist-expect-eq-1)
  fix  $z$ 
  assume  $z: z \in \text{dom } (\text{dist } d)$ 
  show  $(\text{case } z \text{ of } \text{None} \Rightarrow 1 \mid \text{Some } (r, t) \Rightarrow$ 
     $\text{dist-expect } (\text{lift-dist } (\text{hash-dist } x) \ t)$ 
     $(\lambda z. \text{case } z \text{ of } \text{None} \Rightarrow 0 \mid \text{Some } (z, u) \Rightarrow \text{if } y = z \text{ then } 1 \text{ else } 0)) = 1$ 
proof (cases  $z$ )
  case None
  then show ?thesis by simp
next
  case (Some rt)
  then obtain  $r \ t$  where  $rt: z = \text{Some } (r, t)$ 
  by (cases rt) simp
  then show ?thesis
  using assms[of  $r \ t$ ]  $z$ 
  by (simp add: hash-expect-after-update-extension)
qed

```

qed

lemma *dist-expect-then-hash-after-extension-dist-map:*

assumes $\bigwedge r. t.$

$\text{Some } (r, t) \in \text{dom } (\text{dist } d) \implies$

$s0(\text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s0)) \leq t$

shows

dist-expect d

$(\lambda r. \text{case } r \text{ of } \text{None} \Rightarrow 1 \mid \text{Some } (r, t) \Rightarrow$

$\text{dist-expect } (\text{dist-map } (\lambda z. \text{Some } (z, t)) (\text{hash-dist } x \ t))$

$(\lambda z. \text{case } z \text{ of } \text{None} \Rightarrow 0 \mid \text{Some } (z, u) \Rightarrow \text{if } y = z \text{ then } 1 \text{ else } 0)) = 1$

proof –

show *?thesis*

proof (rule *dist-expect-eq-1*)

fix z

assume $z: z \in \text{dom } (\text{dist } d)$

show $(\text{case } z \text{ of } \text{None} \Rightarrow 1 \mid \text{Some } (r, t) \Rightarrow$

$\text{dist-expect } (\text{dist-map } (\lambda z. \text{Some } (z, t)) (\text{hash-dist } x \ t))$

$(\lambda z. \text{case } z \text{ of } \text{None} \Rightarrow 0 \mid \text{Some } (z, u) \Rightarrow \text{if } y = z \text{ then } 1 \text{ else } 0)) = 1$

proof (cases z)

case *None*

then show *?thesis* **by** *simp*

next

case $(\text{Some } rt)$

then obtain $r \ t$ **where** $rt: z = \text{Some } (r, t)$

by (cases rt) *simp*

then show *?thesis*

proof –

have *dist-expect (lift-dist (hash-dist x) t)*

$(\lambda z. \text{case } z \text{ of } \text{None} \Rightarrow 0 \mid \text{Some } (z, u) \Rightarrow \text{if } y = z \text{ then } 1 \text{ else } 0) = 1$

using *assms[of r t] z rt*

by (*simp add: hash-expect-after-update-extension*)

then show *?thesis*

using rt **by** (*simp add: lift-dist-def*)

qed

qed

qed

qed

lemma *send-intermediate-send-same:*

assumes *intermediate-no-fail:*

$\bigwedge a \ s'. \text{None} \notin$

$\text{dom } (\text{dist } (\text{execute } (\text{between } a)$

$(s'(\text{HashMap} := \text{fmupd } x \ a \ (\text{HashMap } s')))))$

assumes *intermediate-extends:*

$\bigwedge a \ s' \ r \ t.$

$\text{Some } (r, t) \in$

$\text{dom } (\text{dist } (\text{execute } (\text{between } a)$

$(s'(\text{HashMap} := \text{fmupd } x \ a \ (\text{HashMap } s'))))) \implies$


```

    s'(|HashMap := fmupd x a (HashMap s')|) ≤ t
shows
  wp-event
    (do {
      a ← hash x;
      - ← between a;
      b ← hash x;
      return (a = b)
    })
    (λr. case r of None ⇒ False | Some (ok, -) ⇒ ok)
    s = 1
unfolding wp-event-def hash-def apply-hash-def modify-HashMap-def
apply (simp add: wpsimps)
apply (rule dist-expect-eq-1)
apply (simp add: wp-def)
apply (rule dist-expect-eq-1)
apply (case-tac xaa)
apply (simp add: intermediate-no-fail)
apply (clarsimp simp: wpsimps split: prod.splits)
apply (rule hash-dist-expect-after-update-extension)
by (rule-tac a=xa and s'=s and r=x1 and t=x2 in intermediate-extends) (simp add: domIff)

```

1.20 Weakest Precondition Calculus

```

lemma wp-hash[wpsimps]:
  wp (hash x) Q s =
    dist-expect (hash-dist x s)
      (λxa. Q (Some (xa, s(|HashMap := fmupd x xa (HashMap s')|))))
by (simp add: hash-def wpsimps apply-hash-def modify-HashMap-def)

```

```

lemma wp-hashI[wp]:
  assumes P ≤
    dist-expect (hash-dist x s)
      (λxa. Q (Some (xa, s(|HashMap := fmupd x xa (HashMap s')|))))
  shows P ≤ wp (hash x) Q s
  using assms by (simp add: wpsimps)

```

1.21 Code Generator Setup

hash-default does not execute since it is using *fUNIV*. Thus, we provide an alternative implementation for enumerable datatypes which can be used for execution.

1.21.1 Enums

```

definition hash-default-enum :: 'a::enum dist
  where hash-default-enum = hash-default

```

```

declare hash-default-enum-def[symmetric,code]

```

```

lemma hash-default-enum-code[code]:
  hash-default-enum = dist-uniform (fset-of-list (Enum.enum :: 'a::enum list))
proof -
  have Abs-fset (UNIV :: 'a set) = fset-of-list (Enum.enum :: 'a list)
    by (rule fset-inject[THEN iffD1])
    (simp add: Abs-fset-inverse fset-of-list.rep-eq enum-UNIV)
  then have (fUNIV :: 'a fset) = fset-of-list (Enum.enum :: 'a list)
    by (simp add: FSet.top-fset.abs-eq)
  then show ?thesis
    unfolding hash-default-enum-def hash-default-def by simp
qed

value fmupd (1::nat) (2::nat) fmempty

value execute (hash True) (HashMap = (fmempty::(bool, bool) fmap))

value execute (apply-hash True) (HashMap = (fmempty::(bool, bool) fmap))

value execute (modify-HashMap True True) (HashMap = (fmempty::(bool, bool) fmap))

value execute (put True) False

value execute (
  do {
    a ← hash True;
    x ← hash False;
    b ← hash True;
    return (a = b)
  }) (HashMap = (fmempty::(bool, bool) fmap))

end

theory Channel-Core
  imports Hash-Monad Merkle-Tree
begin

1.22 Channels

lemma no-failure-bindI:
  assumes m-no-fail: None ∉ dom (dist (execute m s))
  and k-no-fail:
    ∧x t. Some (x, t) ∈ set-dist (execute m s) ⇒
      None ∉ dom (dist (execute (k x) t))
  shows None ∉ dom (dist (execute (m ⋈ k) s))
proof
  assume None ∈ dom (dist (execute (m ⋈ k) s))
  then have none:

```

```

None ∈ dom
  (map-bind (dist ∘ execute m)
    (bind-cont-map (map-fun id dist ∘ execute ∘ k)) s)
unfolding sm-bind.rep-eq dist-bind.rep-eq
by (simp add: o-def map-fun-def)
then have
  None ∈
    (⋃ i ∈ dom (dist (execute m s)).
      dom (bind-cont-map (map-fun id dist ∘ execute ∘ k) i))
using dom-map-bind[of dist ∘ execute m
  bind-cont-map (map-fun id dist ∘ execute ∘ k) s]
by (simp add: o-def)
then obtain i where
  i-dom: i ∈ dom (dist (execute m s))
and none-i:
  None ∈ dom (bind-cont-map (map-fun id dist ∘ execute ∘ k) i)
by auto
show False
proof (cases i)
  case None
  then show ?thesis
    using i-dom m-no-fail by simp
next
  case (Some xt)
  then obtain x t where xt: i = Some (x, t)
  by (cases xt) auto
  have i-set: Some (x, t) ∈ set-dist (execute m s)
  using i-dom unfolding xt set-dist-def .
  have None ∉ dom (dist (execute (k x) t))
  by (rule k-no-fail[OF i-set])
  moreover have
    dom (bind-cont-map (map-fun id dist ∘ execute ∘ k) (Some (x, t))) =
      dom (dist (execute (k x) t))
  unfolding bind-cont-map-def by simp
  ultimately show ?thesis
    using none-i unfolding xt by simp
qed
qed

lemma no-failure-return[simp]:
  None ∉ dom (dist (execute (return x) s))
  unfolding return.rep-eq dist-return-def dist-delta-dist delta-map-def
  by simp

lemma no-failure-get[simp]:
  None ∉ dom (dist (execute get s))
  unfolding get.rep-eq dist-get-def dist-delta-dist delta-map-def
  by simp

```

lemma *no-failure-put*[simp]:
 $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (\text{put } s') s))$
unfolding *put.rep-eq dist-put-def dist-delta-dist delta-map-def*
by *simp*

lemma *no-failure-lift*[simp]:
 $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (\text{lift } d) s))$
unfolding *lift.rep-eq lift-dist-def set-dist-def*
using *set-dist-dist-map*[of $\lambda a. \text{Some } (a, s) d s$]
unfolding *set-dist-def* **by** *auto*

lemma *no-failure-modify*[simp]:
 $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (\text{modify } f) s))$
unfolding *modify-def*
by (*rule no-failure-bindI*) *simp-all*

lemma *throw-no-Some-set-dist*:
 $\text{Some } x \notin \text{set-dist } (\text{execute } (\text{throw} :: ('a, 's) \text{ state-monad}) s)$
unfolding *set-dist-def throw.rep-eq dist-throw-def dist-delta-dist*
 delta-map-def
by *simp*

record *'a channel* = (*'a, 'a*) *hash* +
State :: *'a*
Transcript :: *'a list*

type-synonym (*'a, 'b, 'c*) *c-monad* = (*'b, ('a, 'c) channel-scheme*) *state-monad*

definition *receive-random-field-element* :: (*'a, 'a::finite, 'b*) *c-monad*
where
receive-random-field-element $\equiv$
do {
 $s \leftarrow \text{get};$
 $r \leftarrow \text{hash } (\text{State } s);$
return *r*
}

record *'a protocol-channel* = (*'a protocol-hash-input, 'a*) *hash* +
PState :: *'a*
PTranscript :: *'a list*
PTraceFriCounter :: *nat*
PCompositionFriCounter :: *nat*
PAlphaCounter :: *nat*
PQueryCounter :: *nat*

type-synonym (*'a, 'b, 'c*) *protocol-c-monad* =
(*'b, ('a, 'c) protocol-channel-scheme*) *state-monad*

definition *protocol-receive-tagged-random-field-element*

$:: ('a \Rightarrow 'a \text{ protocol-hash-input}) \Rightarrow$
 $('a, 'a::\text{finite}, 'b) \text{ protocol-c-monad}$

where

$\text{protocol-receive-tagged-random-field-element tag} \equiv$
 $\text{do } \{$
 $\quad s \leftarrow \text{get};$
 $\quad r \leftarrow \text{hash } (\text{tag } (\text{PState } s));$
 $\quad \text{return } r$
 $\}$

definition *protocol-receive-random-field-element*

$:: ('a, 'a::\text{finite}, 'b) \text{ protocol-c-monad}$

where

$\text{protocol-receive-random-field-element} \equiv$
 $\text{protocol-receive-tagged-random-field-element FiatShamirChallenge}$

definition *protocol-receive-counted-tagged-random-field-element*

$:: ((('a, 'b) \text{ protocol-channel-scheme}) \Rightarrow \text{nat}) \Rightarrow$
 $((('a, 'b) \text{ protocol-channel-scheme}) \Rightarrow ('a, 'b) \text{ protocol-channel-scheme}) \Rightarrow$
 $(\text{nat} \Rightarrow 'a \Rightarrow 'a \text{ protocol-hash-input}) \Rightarrow$
 $('a, 'a::\text{finite}, 'b) \text{ protocol-c-monad}$

where

$\text{protocol-receive-counted-tagged-random-field-element counter bump tag} \equiv$
 $\text{do } \{$
 $\quad s \leftarrow \text{get};$
 $\quad r \leftarrow \text{hash } (\text{tag } (\text{counter } s) (\text{PState } s));$
 $\quad \text{modify bump};$
 $\quad \text{return } r$
 $\}$

definition *protocol-send*

$:: ('a \Rightarrow 'a \Rightarrow 'a) \Rightarrow 'a \Rightarrow$
 $('a, \text{unit}, 'b) \text{ protocol-c-monad}$

where

$\text{protocol-send absorb } x \equiv$
 modify
 $(\lambda s. s \langle$
 $\quad \text{PState} := \text{absorb } (\text{PState } s) \ x,$
 $\quad \text{PTranscript} := x \ \# \ \text{PTranscript } s \rangle)$

definition *protocol-read*

$:: ('a \Rightarrow 'a \Rightarrow 'a) \Rightarrow ('a, 'a, 'b) \text{ protocol-c-monad}$

where

$\text{protocol-read absorb} \equiv$
 $\text{do } \{$
 $\quad s \leftarrow \text{get};$
 $\quad \text{assert } (\text{PTranscript } s \neq []);$
 $\quad \text{modify}$
 $\quad (\lambda s. s \langle$

```

    PState := absorb (PState s) (hd (PTranscript s)),
    PTranscript := tl (PTranscript s));
  return (hd (PTranscript s))
}

```

lemma *protocol-hash-no-failure*:

```

  None ∉ dom (dist (execute (hash x) s))
unfolding hash-def apply-hash-def modify-HashMap-def
by (intro no-failure-bindI) simp-all

```

lemma *protocol-receive-tagged-random-field-element-no-failure*:

```

  None ∉
    dom (dist
      (execute (protocol-receive-tagged-random-field-element tag) s))
unfolding protocol-receive-tagged-random-field-element-def
by (intro no-failure-bindI) (simp-all add: protocol-hash-no-failure)

```

lemma *protocol-receive-random-field-element-no-failure*:

```

  None ∉
    dom (dist (execute protocol-receive-random-field-element s))
unfolding protocol-receive-random-field-element-def
by (rule protocol-receive-tagged-random-field-element-no-failure)

```

lemma *protocol-receive-counted-tagged-random-field-element-no-failure*:

```

  None ∉
    dom (dist
      (execute
        (protocol-receive-counted-tagged-random-field-element counter bump tag) s))
unfolding protocol-receive-counted-tagged-random-field-element-def
by (intro no-failure-bindI) (simp-all add: protocol-hash-no-failure)

```

lemma *protocol-send-outcome*:

```

assumes Some ((), t) ∈ set-dist (execute (protocol-send absorb x) s)
shows
  t = s⟦
    PState := absorb (PState s) x,
    PTranscript := x # PTranscript s⟧
using assms
unfolding protocol-send-def modify-def
by (auto elim!: protocol-merkle.set-dist-bindE)

```

lemma *protocol-read-cons-outcome*:

```

assumes Some (y, t) ∈
  set-dist (execute (protocol-read absorb)
    (s⟦PTranscript := x # xs⟧))
shows
  y = x ∧
  t = s⟦
    PState := absorb (PState s) x,

```

```

    PTranscript := xs)
using assms
unfolding protocol-read-def modify-def assert-def
by (auto elim!: protocol-merkle.set-dist-bindE)

lemma protocol-hash-channel-preserves:
  assumes Some (h, t) ∈ set-dist (execute (hash x) s)
  shows PState t = PState s
    and PTranscript t = PTranscript s
    and PTraceFriCounter t = PTraceFriCounter s
    and PCompositionFriCounter t = PCompositionFriCounter s
    and PAlphaCounter t = PAlphaCounter s
    and PQueryCounter t = PQueryCounter s
proof -
  from assms obtain a s1 where
    a: Some (a, s1) ∈ set-dist (execute (apply-hash x) s)
    and rest: Some (h, t) ∈
      set-dist (execute (modify-HashMap x a >>= (λ-. return a)) s1)
  proof -
    have Some (h, t) ∈
      set-dist (execute
        (apply-hash x >>= (λa. modify-HashMap x a >>= (λ-. return a)))) s)
    using assms unfolding hash-def .
    then show ?thesis
      by (rule protocol-merkle.set-dist-bindE) (rule that)
  qed
  have a-img: Some (a, s1) ∈ (λa. Some (a, s)) ‘ set-dist (hash-dist x s)
  using a
  unfolding apply-hash-def lift.rep-eq lift-dist-def
  by (simp add: set-dist-dist-map)
  then have s1: s1 = s
  by auto
  from rest obtain u s2 where
    mod: Some (u, s2) ∈ set-dist (execute (modify-HashMap x a) s1)
    and ret: Some (h, t) ∈ set-dist (execute (return a) s2)
    by (elim protocol-merkle.set-dist-bindE)
  from mod have s2: s2 = s1 (HashMap := fupd x a (HashMap s1))
  unfolding modify-HashMap-def modify-def
  by (auto elim!: protocol-merkle.set-dist-bindE)
  from ret have t: t = s2
  by auto
  show PState t = PState s
  unfolding t s2 s1 by simp
  show PTranscript t = PTranscript s
  unfolding t s2 s1 by simp
  show PTraceFriCounter t = PTraceFriCounter s
  unfolding t s2 s1 by simp
  show PCompositionFriCounter t = PCompositionFriCounter s
  unfolding t s2 s1 by simp

```

```

show  $P\text{AlphaCounter } t = P\text{AlphaCounter } s$ 
  unfolding  $t \ s2 \ s1$  by simp
show  $P\text{QueryCounter } t = P\text{QueryCounter } s$ 
  unfolding  $t \ s2 \ s1$  by simp
qed

lemma protocol-hash-preserves-other-lookup:
  assumes outcome:  $\text{Some } (h, t) \in \text{set-dist } (\text{execute } (\text{hash } x) \ s)$ 
  and neg:  $y \neq x$ 
  shows  $\text{fmlookup } (\text{HashMap } t) \ y = \text{fmlookup } (\text{HashMap } s) \ y$ 
proof –
  from outcome obtain  $a \ s1$  where
     $a$ :  $\text{Some } (a, s1) \in \text{set-dist } (\text{execute } (\text{apply-hash } x) \ s)$ 
    and rest:  $\text{Some } (h, t) \in$ 
       $\text{set-dist } (\text{execute } (\text{modify-HashMap } x \ a \gg (\lambda-. \text{return } a)) \ s1)$ 
  proof –
    have  $\text{Some } (h, t) \in$ 
       $\text{set-dist } (\text{execute } (\text{apply-hash } x \gg (\lambda a. \text{modify-HashMap } x \ a \gg (\lambda-. \text{return } a))) \ s)$ 
    using outcome unfolding hash-def .
    then show ?thesis
      by (rule protocol-merkle.set-dist-bindE) (rule that)
  qed
have  $s1: s1 = s$ 
  using  $a$ 
  unfolding apply-hash-def lift.rep-eq lift-dist-def
  by (auto simp: set-dist-dist-map)
from rest obtain  $u \ s2$  where
   $\text{mod}$ :  $\text{Some } (u, s2) \in \text{set-dist } (\text{execute } (\text{modify-HashMap } x \ a) \ s1)$ 
  and  $\text{ret}$ :  $\text{Some } (h, t) \in \text{set-dist } (\text{execute } (\text{return } a) \ s2)$ 
  by (elim protocol-merkle.set-dist-bindE)
have  $s2: s2 = s1 \ (\text{HashMap} := \text{fmupd } x \ a \ (\text{HashMap } s1))$ 
  using  $\text{mod}$  unfolding modify-HashMap-def modify-def
  by (auto elim!: protocol-merkle.set-dist-bindE)
have  $t: t = s2$ 
  using  $\text{ret}$  by auto
show ?thesis
  unfolding  $t \ s2 \ s1$  using neg by simp
qed

lemma protocol-receive-random-field-element-outcome:
  assumes outcome:
     $\text{Some } (r, t) \in$ 
       $\text{set-dist } (\text{execute } \text{protocol-receive-random-field-element } s)$ 
  shows
     $s \leq t \wedge$ 
     $P\text{State } t = P\text{State } s \wedge$ 
     $P\text{Transcript } t = P\text{Transcript } s \wedge$ 
     $\text{fmlookup } (\text{HashMap } t) \ (\text{FiatShamirChallenge } (P\text{State } s)) = \text{Some } r$ 

```


proof –

from *outcome* **obtain** *s0* **where**

get-s: *Some* (*s0*, *s0*) $\in$ *set-dist* (*execute get s*)

and *rest*:

Some (*r*, *t*) $\in$

set-dist

(*execute* (*hash* (*FiatShamirChallenge* (*PState s0*))) $\gg$ *return*) *s0*)

unfolding *protocol-receive-random-field-element-def*

protocol-receive-tagged-random-field-element-def

by (*auto elim!*: *protocol-merkle.set-dist-bindE*)

have *s0*: *s0* = *s*

using *get-s* **by** *simp*

have *hash-out*:

Some (*r*, *t*) $\in$

set-dist (*execute* (*hash* (*FiatShamirChallenge* (*PState s*)))) *s*)

using *rest* **unfolding** *s0* **by** *simp*

show *?thesis*

using *protocol-merkle.hash-outcome*[*OF hash-out*]

protocol-hash-channel-preserves[*OF hash-out*]

by *simp*

qed

lemma *protocol-receive-tagged-random-field-element-outcome*:

assumes *outcome*:

Some (*r*, *t*) $\in$

set-dist

(*execute* (*protocol-receive-tagged-random-field-element tag*) *s*)

shows

s $\leq$ *t* $\wedge$

PState t = *PState s* $\wedge$

PTranscript t = *PTranscript s* $\wedge$

fmlookup (*HashMap t*) (*tag* (*PState s*)) = *Some r*

proof –

from *outcome* **obtain** *s0* **where**

get-s: *Some* (*s0*, *s0*) $\in$ *set-dist* (*execute get s*)

and *rest*:

Some (*r*, *t*) $\in$

set-dist (*execute* (*hash* (*tag* (*PState s0*))) $\gg$ *return*) *s0*)

unfolding *protocol-receive-tagged-random-field-element-def*

by (*auto elim!*: *protocol-merkle.set-dist-bindE*)

have *s0*: *s0* = *s*

using *get-s* **by** *simp*

have *hash-out*:

Some (*r*, *t*) $\in$ *set-dist* (*execute* (*hash* (*tag* (*PState s*)))) *s*)

using *rest* **unfolding** *s0* **by** *simp*

show *?thesis*

using *protocol-merkle.hash-outcome*[*OF hash-out*]

protocol-hash-channel-preserves[*OF hash-out*]

by *simp*

qed

lemma *protocol-receive-tagged-random-field-element-preserves-other-lookup:*

assumes *outcome:*

Some (r, t) ∈

set-dist

(execute (protocol-receive-tagged-random-field-element tag) s)

and *neg: y ≠ tag (PState s)*

shows *fmlookup (HashMap t) y = fmlookup (HashMap s) y*

proof –

from *outcome* **obtain** *s0* **where**

get-s: Some (s0, s0) ∈ set-dist (execute get s)

and *rest:*

Some (r, t) ∈

set-dist (execute (hash (tag (PState s0))) >>= return) s0)

unfolding *protocol-receive-tagged-random-field-element-def*

by *(auto elim!: protocol-merkle.set-dist-bindE)*

have *s0: s0 = s*

using *get-s* **by** *simp*

from *rest* **obtain** *h u* **where**

hash-out: Some (h, u) ∈ set-dist (execute (hash (tag (PState s)))) s)

and *ret: Some (r, t) ∈ set-dist (execute (return h) u)*

unfolding *s0* **by** *(elim protocol-merkle.set-dist-bindE)*

have *t: t = u* **and** *r: r = h*

using *ret* **by** *auto*

show *?thesis*

unfolding *t*

by *(rule protocol-hash-preserves-other-lookup[OF hash-out neg])*

qed

lemma *protocol-receive-counted-tagged-random-field-element-outcome:*

assumes *outcome:*

Some (r, t) ∈

set-dist

(execute

(protocol-receive-counted-tagged-random-field-element counter bump tag) s)

and *bump-hash: ∧u. HashMap (bump u) = HashMap u*

and *bump-state: ∧u. PState (bump u) = PState u*

and *bump-transcript: ∧u. PTranscript (bump u) = PTranscript u*

shows

s ≤ t ∧

PState t = PState s ∧

PTranscript t = PTranscript s ∧

fmlookup (HashMap t) (tag (counter s) (PState s)) = Some r

proof –

from *outcome* **obtain** *s0 h u v* **where**

get-s: Some (s0, s0) ∈ set-dist (execute get s)

and *hash-out:*

Some (h, u) ∈

```

    set-dist (execute (hash (tag (counter s0) (PState s0))) s0)
  and mod:
    Some ((), v) ∈ set-dist (execute (modify bump) u)
  and ret: Some (r, t) ∈ set-dist (execute (return h) v)
  unfolding protocol-receive-counted-tagged-random-field-element-def
  by (auto elim!: protocol-merkle.set-dist-bindE)
have s0: s0 = s
  using get-s by simp
have v: v = bump u
  using mod unfolding modify-def
  by (auto elim!: protocol-merkle.set-dist-bindE)
have t: t = v and r: r = h
  using ret by auto
have ext-su: s ≤ u
  using protocol-merkle.hash-outcome[OF hash-out[unfolded s0]]
  by simp
have lookup-u:
  fmlookup (HashMap u) (tag (counter s) (PState s)) = Some h
  using protocol-merkle.hash-outcome[OF hash-out[unfolded s0]]
  by simp
have channel-u:
  PState u = PState s ∧ PTranscript u = PTranscript s
  using protocol-hash-channel-preserves[OF hash-out[unfolded s0]]
  by simp
have hash-t: HashMap t = HashMap u
  unfolding t v by (simp add: bump-hash)
show ?thesis
  using ext-su lookup-u channel-u
  unfolding t v r less-eq-hash-ext-def
  by (simp add: bump-hash bump-state bump-transcript)
qed

lemma protocol-receive-counted-tagged-random-field-element-preserves-other-lookup:
  assumes outcome:
    Some (r, t) ∈
      set-dist
        (execute
          (protocol-receive-counted-tagged-random-field-element counter bump tag) s)
  and neq: y ≠ tag (counter s) (PState s)
  and bump-hash:  $\bigwedge u. \text{HashMap (bump } u) = \text{HashMap } u$ 
  shows fmlookup (HashMap t) y = fmlookup (HashMap s) y
proof -
  from outcome obtain s0 h u v where
    get-s: Some (s0, s0) ∈ set-dist (execute get s)
  and hash-out:
    Some (h, u) ∈
      set-dist (execute (hash (tag (counter s0) (PState s0))) s0)
  and mod:
    Some ((), v) ∈ set-dist (execute (modify bump) u)

```

```

and ret: Some (r, t) ∈ set-dist (execute (return h) v)
unfolding protocol-receive-counted-tagged-random-field-element-def
by (auto elim!: protocol-merkle.set-dist-bindE)
have s0: s0 = s
  using get-s by simp
have v: v = bump u
  using mod unfolding modify-def
  by (auto elim!: protocol-merkle.set-dist-bindE)
have t: t = v
  using ret by auto
have hash-pres:
  fmlookup (HashMap u) y = fmlookup (HashMap s) y
  by (rule protocol-hash-preserves-other-lookup[OF hash-out[unfolded s0] neg])
show ?thesis
  unfolding t v using hash-pres by (simp add: bump-hash)
qed

```

```

lemma protocol-merkle-create-no-failure:
  None ∉ dom (dist (execute (protocol-merkle.create xs) s))
proof (induction xs arbitrary: s rule: protocol-merkle.create.induct)
  case 1
  then show ?case by simp
next
  case (2 x)
  then show ?case
    by (simp add: no-failure-bindI protocol-hash-no-failure)
next
  case (3 v vb vc)
  then show ?case
    by (simp add: Let-def no-failure-bindI protocol-hash-no-failure)
qed

```

```

lemma protocol-merkle-create-preserves-channel:
  assumes outcome:
    Some (r, t) ∈ set-dist (execute (protocol-merkle.create xs) s)
  shows PState t = PState s
    and PTranscript t = PTranscript s
    and PTraceFriCounter t = PTraceFriCounter s
    and PCompositionFriCounter t = PCompositionFriCounter s
    and PAlphaCounter t = PAlphaCounter s
    and PQueryCounter t = PQueryCounter s
proof –
  have both:
    PState t = PState s ∧
    PTranscript t = PTranscript s ∧
    PTraceFriCounter t = PTraceFriCounter s ∧
    PCompositionFriCounter t = PCompositionFriCounter s ∧
    PAlphaCounter t = PAlphaCounter s ∧
    PQueryCounter t = PQueryCounter s

```

```

if outcome:
  Some (r, t) ∈ set-dist (execute (protocol-merkle.create xs) s)
for xs r s t
using outcome
proof (induction xs arbitrary: r s t rule: protocol-merkle.create.induct)
  case 1
  then show ?case by simp
next
  case (2 x)
  from 2.prems obtain h u where
    h: Some (h, u) ∈ set-dist (execute (hash x) s)
    and ret:
      Some (r, t) ∈
        set-dist (execute (return ⟨⟨⟩, h, ⟨⟩⟩) u)
    by (auto elim!: protocol-merkle.set-dist-bindE)
  have PState u = PState s ∧
    PTranscript u = PTranscript s ∧
    PTraceFriCounter u = PTraceFriCounter s ∧
    PCompositionFriCounter u = PCompositionFriCounter s ∧
    PAlphaCounter u = PAlphaCounter s ∧
    PQueryCounter u = PQueryCounter s
    using protocol-hash-channel-preserves[OF h] by simp
  then show ?case
    using ret by simp
next
  case (3 v vb vc)
  from 3.prems obtain l s1 rn s2 val s3 where
    l:
      Some (l, s1) ∈
        set-dist (execute
          (protocol-merkle.create
            (v # take (length vc div 2) (vb # vc))) s)
    and rn:
      Some (rn, s2) ∈
        set-dist (execute
          (protocol-merkle.create
            (drop (length vc div 2) (vb # vc))) s1)
    and val:
      Some (val, s3) ∈
        set-dist (execute (hash (MerkleNode (value l) (value rn))) s2)
    and ret:
      Some (r, t) ∈
        set-dist (execute (return ⟨l, val, rn⟩) s3)
    by (auto simp: Let-def elim!: protocol-merkle.set-dist-bindE)
  have s1:
    PState s1 = PState s ∧
    PTranscript s1 = PTranscript s ∧
    PTraceFriCounter s1 = PTraceFriCounter s ∧
    PCompositionFriCounter s1 = PCompositionFriCounter s ∧

```

```

    PAlphaCounter s1 = PAlphaCounter s ∧
    PQueryCounter s1 = PQueryCounter s
  using 3.IH(1)[of length (v # vb # vc) div 2 l s1 s] l
  by simp
  have s2:
    PState s2 = PState s1 ∧
    PTranscript s2 = PTranscript s1 ∧
    PTraceFriCounter s2 = PTraceFriCounter s1 ∧
    PCompositionFriCounter s2 = PCompositionFriCounter s1 ∧
    PAlphaCounter s2 = PAlphaCounter s1 ∧
    PQueryCounter s2 = PQueryCounter s1
  using 3.IH(2)[of length (v # vb # vc) div 2 rn s2 s1] rn
  by simp
  have s3:
    PState s3 = PState s2 ∧
    PTranscript s3 = PTranscript s2 ∧
    PTraceFriCounter s3 = PTraceFriCounter s2 ∧
    PCompositionFriCounter s3 = PCompositionFriCounter s2 ∧
    PAlphaCounter s3 = PAlphaCounter s2 ∧
    PQueryCounter s3 = PQueryCounter s2
  using protocol-hash-channel-preserves[OF val] by simp
  show ?case
    using s1 s2 s3 ret by simp
  qed
  show PState t = PState s
    using both[OF assms] by simp
  show PTranscript t = PTranscript s
    using both[OF assms] by simp
  show PTraceFriCounter t = PTraceFriCounter s
    using both[OF assms] by simp
  show PCompositionFriCounter t = PCompositionFriCounter s
    using both[OF assms] by simp
  show PAlphaCounter t = PAlphaCounter s
    using both[OF assms] by simp
  show PQueryCounter t = PQueryCounter s
    using both[OF assms] by simp
  qed

lemma protocol-merkle-create-preserves-non-merkle-lookup:
  assumes outcome:
    Some (r, t) ∈ set-dist (execute (protocol-merkle.create xs) s)
  and not-leaf:  $\bigwedge x. x \in \text{set } xs \implies y \neq x$ 
  and not-node:  $\bigwedge x z. y \neq \text{MerkleNode } x z$ 
  shows fmlookup (HashMap t) y = fmlookup (HashMap s) y
  using outcome not-leaf
proof (induction xs arbitrary: r s t rule: protocol-merkle.create.induct)
  case 1
  then show ?case
    by simp

```

```

next
  case (2 x)
  from 2.premis(1) obtain h u where
    h: Some (h, u) ∈ set-dist (execute (hash x) s)
  and ret:
    Some (r, t) ∈
      set-dist (execute (return ⟨⟨⟩, h, ⟨⟩⟩) u)
  by (auto elim!: protocol-merkle.set-dist-bindE)
  have h-pres: fmlookup (HashMap u) y = fmlookup (HashMap s) y
  by (rule protocol-hash-preserves-other-lookup[OF h])
    (rule 2.premis(2), simp)
  show ?case
    using h-pres ret by simp
next
  case (3 v vb vc)
  let ?left = v # take (length vc div 2) (vb # vc)
  let ?right = drop (length vc div 2) (vb # vc)
  have left-eq:
    ?left = take (length (v # vb # vc) div 2) (v # vb # vc)
  by simp
  have right-eq:
    ?right = drop (length (v # vb # vc) div 2) (v # vb # vc)
  by simp
  from 3.premis(1) obtain l s1 rn s2 val s3 where
    l:
      Some (l, s1) ∈
        set-dist (execute (protocol-merkle.create ?left) s)
  and rn:
    Some (rn, s2) ∈
      set-dist (execute (protocol-merkle.create ?right) s1)
  and val:
    Some (val, s3) ∈
      set-dist (execute (hash (MerkleNode (value l) (value rn))) s2)
  and ret:
    Some (r, t) ∈
      set-dist (execute (return ⟨l, val, rn⟩) s3)
  by (auto simp: Let-def elim!: protocol-merkle.set-dist-bindE)
  have left-not-leaf:  $\bigwedge x. x \in \text{set } ?\text{left} \implies y \neq x$ 
  proof -
    fix x
    assume mem:  $x \in \text{set } ?\text{left}$ 
    show  $y \neq x$ 
    proof (cases  $x = v$ )
      case True
      then show ?thesis
        using 3.premis(2)[of x] by simp
    next
      case False
      then have  $x \in \text{set } (\text{take } (\text{length } vc \text{ div } 2) (vb \# vc))$ 

```

```

    using mem by simp
  then have  $x \in \text{set } (vb \# vc)$ 
    by (rule in-set-takeD)
  then show ?thesis
    using 3.premis(2)[of x] by simp
qed
qed
have right-not-leaf:  $\bigwedge x. x \in \text{set } ?right \implies y \neq x$ 
proof -
  fix x
  assume mem:  $x \in \text{set } ?right$ 
  then have  $x \in \text{set } (vb \# vc)$ 
    by (rule in-set-dropD)
  then show  $y \neq x$ 
    using 3.premis(2)[of x] by simp
qed
have l':
  Some (l, s1)  $\in$ 
    set-dist (execute
      (protocol-merkle.create
        (take (length (v # vb # vc) div 2) (v # vb # vc))) s)
  using l by (simp only: left-eq[symmetric])
have rn':
  Some (rn, s2)  $\in$ 
    set-dist (execute
      (protocol-merkle.create
        (drop (length (v # vb # vc) div 2) (v # vb # vc))) s1)
  using rn by (simp only: right-eq[symmetric])
have left-not-leaf':
 $\bigwedge x. x \in \text{set } (\text{take } (\text{length } (v \# vb \# vc) \text{ div } 2) (v \# vb \# vc)) \implies$ 
 $y \neq x$ 
proof -
  fix x
  assume mem:  $x \in \text{set } (\text{take } (\text{length } (v \# vb \# vc) \text{ div } 2) (v \# vb \# vc))$ 
  have  $x \in \text{set } ?left$ 
    using mem by (simp only: left-eq)
  then show  $y \neq x$ 
    by (rule left-not-leaf)
qed
have right-not-leaf':
 $\bigwedge x. x \in \text{set } (\text{drop } (\text{length } (v \# vb \# vc) \text{ div } 2) (v \# vb \# vc)) \implies$ 
 $y \neq x$ 
proof -
  fix x
  assume mem:  $x \in \text{set } (\text{drop } (\text{length } (v \# vb \# vc) \text{ div } 2) (v \# vb \# vc))$ 
  have  $x \in \text{set } ?right$ 
    using mem by (simp only: right-eq)
  then show  $y \neq x$ 
    by (rule right-not-leaf)

```



```

qed
have left-pres: fmllookup (HashMap s1) y = fmllookup (HashMap s) y
by (rule 3.IH(1))
  [of length (v # vb # vc) div 2 l s1 s,
   OF refl l' left-not-leaf']
have right-pres: fmllookup (HashMap s2) y = fmllookup (HashMap s1) y
by (rule 3.IH(2))
  [of length (v # vb # vc) div 2 rn s2 s1,
   OF refl rn' right-not-leaf']
have hash-pres: fmllookup (HashMap s3) y = fmllookup (HashMap s2) y
by (rule protocol-hash-preserves-other-lookup[OF val])
  (rule not-node)
show ?case
  using left-pres right-pres hash-pres ret by simp
qed

lemma protocol-create-preserves-non-merkle-lookup:
  assumes outcome: Some (r, t) ∈ set-dist (execute (protocol-create xs) s)
  and not-leaf:  $\bigwedge x. y \neq \text{MerkleLeaf } x$ 
  and not-node:  $\bigwedge x z. y \neq \text{MerkleNode } x z$ 
  shows fmllookup (HashMap t) y = fmllookup (HashMap s) y
proof -
  have outcome':
    Some (r, t) ∈
      set-dist (execute (protocol-merkle.create (map MerkleLeaf xs)) s)
  using outcome unfolding protocol-create-def .
  show ?thesis
  by (rule protocol-merkle-create-preserves-non-merkle-lookup[OF outcome'])
    (use not-leaf not-node in auto)
qed

locale protocol-channel-compat =
  fixes concat :: 'a::finite  $\Rightarrow$  'a  $\Rightarrow$  'a
begin

definition create
  :: 'a list  $\Rightarrow$  ('a protocol-hash-input, 'a, 'a tree, 'b) hash-monad
  where create xs = protocol-create xs

definition check-authentication-path
  :: nat  $\Rightarrow$  nat  $\Rightarrow$  'a  $\Rightarrow$  'a list  $\Rightarrow$ 
    ('a protocol-hash-input, 'a, 'a, 'b) hash-monad
  where
    check-authentication-path len idx leaf path =
      protocol-check-authentication-path len idx leaf path

definition created-tree
  :: 'a list  $\Rightarrow$  'a tree  $\Rightarrow$ 
    ('a protocol-hash-input, 'a, 'b) hash-scheme  $\Rightarrow$  bool

```

where *created-tree xs r s* $\longleftrightarrow$ *protocol-created-tree xs r s*

definition *receive-random-field-element*

$:: ('a, 'a, 'b) \text{ protocol-c-monad}$

where

receive-random-field-element = *protocol-receive-random-field-element*

definition *receive-tagged-random-field-element*

$:: ('a \Rightarrow 'a \text{ protocol-hash-input}) \Rightarrow$

$('a, 'a, 'b) \text{ protocol-c-monad}$

where

receive-tagged-random-field-element tag =

protocol-receive-tagged-random-field-element tag

definition *receive-trace-fri-challenge*

$:: ('a, 'a, 'b) \text{ protocol-c-monad}$

where

receive-trace-fri-challenge =

protocol-receive-counted-tagged-random-field-element

PTraceFriCounter

$(\lambda s. s \downarrow PTraceFriCounter := Suc (PTraceFriCounter s))$

TraceFriChallenge

definition *receive-composition-fri-challenge*

$:: ('a, 'a, 'b) \text{ protocol-c-monad}$

where

receive-composition-fri-challenge =

protocol-receive-counted-tagged-random-field-element

PCompositionFriCounter

$(\lambda s. s \downarrow PCompositionFriCounter := Suc (PCompositionFriCounter s))$

CompositionFriChallenge

definition *receive-alpha-challenge*

$:: ('a, 'a, 'b) \text{ protocol-c-monad}$

where

receive-alpha-challenge =

protocol-receive-counted-tagged-random-field-element

PAlphaCounter

$(\lambda s. s \downarrow PAlphaCounter := Suc (PAlphaCounter s))$

AlphaChallenge

definition *receive-query-index-challenge*

$:: ('a, 'a, 'b) \text{ protocol-c-monad}$

where

receive-query-index-challenge =

protocol-receive-counted-tagged-random-field-element

PQueryCounter

$(\lambda s. s \downarrow PQueryCounter := Suc (PQueryCounter s))$

QueryIndexChallenge

definition *send* :: ('a $\Rightarrow$ ('a, unit, 'b) protocol-c-monad
where *send* *x* = protocol-send concat *x*

definition *read* :: ('a, 'a, 'b) protocol-c-monad
where *read* = protocol-read concat

lemmas *set-dist-bindE* = protocol-merkle.set-dist-bindE

lemmas *hash-outcome* = protocol-merkle.hash-outcome

lemmas *hash-ext-refl* = protocol-merkle.hash-ext-refl

lemmas *hash-ext-trans* = protocol-merkle.hash-ext-trans

lemmas *hash-extension-lookup* = protocol-merkle.hash-extension-lookup

lemma *hash-no-failure*:

None $\notin$ dom (dist (execute (hash *x*) *s*))

by (rule protocol-hash-no-failure)

lemmas *hash-channel-preserves* = protocol-hash-channel-preserves

lemma *send-no-failure*:

None $\notin$ dom (dist (execute (send *x*) *s*))

unfolding send-def protocol-send-def

by simp

lemma *create-no-failure*:

None $\notin$ dom (dist (execute (create *xs*) *s*))

unfolding create-def protocol-create-def

by (rule protocol-merkle-create-no-failure)

lemma *created-tree-mono*:

assumes created-tree *xs* *r* *s0*

and *s0* $\leq$ *s*

shows created-tree *xs* *r* *s*

using assms

unfolding created-tree-def

by (rule protocol-created-tree-mono)

lemma *create-outcome*:

assumes Some (*r*, *t*) $\in$ set-dist (execute (create *xs*) *s*)

shows *s* $\leq$ *t* $\wedge$ created-tree *xs* *r* *t*

using protocol-create-outcome[OF assms[unfolded create-def]]

unfolding created-tree-def **by** simp

lemma *create-get-authentication-path-len*:

assumes Some (*r*, *t*) $\in$ set-dist (execute (create *xs*) *s*)

and 0 < *lgth*

and *lgth* $\leq$ length *xs*

shows length (get-authentication-path *lgth* *idx* *r*) = floor-log *lgth*

using protocol-merkle.create-get-authentication-path-len

$[OF \text{ assms}(1)[\text{unfolded create-def protocol-create-def}] \text{ assms}(2)]$
 $\text{assms}(3)$
by *simp*

lemma *create-preserves-channel:*

assumes *Some* $(r, t) \in \text{set-dist}(\text{execute}(\text{create } xs) \ s)$
shows $PState \ t = PState \ s$
and $PTranscript \ t = PTranscript \ s$
and $PTraceFriCounter \ t = PTraceFriCounter \ s$
and $PCompositionFriCounter \ t = PCompositionFriCounter \ s$
and $PAlphaCounter \ t = PAlphaCounter \ s$
and $PQueryCounter \ t = PQueryCounter \ s$
using *protocol-merkle-create-preserves-channel*
 $[OF \text{ assms}[\text{unfolded create-def protocol-create-def}]]$
by *simp-all*

lemma *create-preserves-non-merkle-lookup:*

assumes *outcome: Some* $(r, t) \in \text{set-dist}(\text{execute}(\text{create } xs) \ s)$
and *not-leaf:* $\bigwedge x. y \neq \text{MerkleLeaf } x$
and *not-node:* $\bigwedge x \ z. y \neq \text{MerkleNode } x \ z$
shows $\text{fmlookup}(\text{HashMap } t) \ y = \text{fmlookup}(\text{HashMap } s) \ y$
using *protocol-create-preserves-non-merkle-lookup*
 $[OF \text{ outcome}[\text{unfolded create-def}] \text{ not-leaf not-node}]$
by *simp*

lemma *create-preserves-challenge-lookups:*

assumes *outcome: Some* $(r, t) \in \text{set-dist}(\text{execute}(\text{create } xs) \ s)$
shows
 $\text{fmlookup}(\text{HashMap } t) (\text{AlphaChallenge } i \ x) =$
 $\text{fmlookup}(\text{HashMap } s) (\text{AlphaChallenge } i \ x)$
 $\text{fmlookup}(\text{HashMap } t) (\text{QueryIndexChallenge } i \ x) =$
 $\text{fmlookup}(\text{HashMap } s) (\text{QueryIndexChallenge } i \ x)$
 $\text{fmlookup}(\text{HashMap } t) (\text{TraceFriChallenge } i \ x) =$
 $\text{fmlookup}(\text{HashMap } s) (\text{TraceFriChallenge } i \ x)$
 $\text{fmlookup}(\text{HashMap } t) (\text{CompositionFriChallenge } i \ x) =$
 $\text{fmlookup}(\text{HashMap } s) (\text{CompositionFriChallenge } i \ x)$

proof –

show $\text{fmlookup}(\text{HashMap } t) (\text{AlphaChallenge } i \ x) =$
 $\text{fmlookup}(\text{HashMap } s) (\text{AlphaChallenge } i \ x)$
by (rule *create-preserves-non-merkle-lookup*[*OF outcome*]) *simp-all*
show $\text{fmlookup}(\text{HashMap } t) (\text{QueryIndexChallenge } i \ x) =$
 $\text{fmlookup}(\text{HashMap } s) (\text{QueryIndexChallenge } i \ x)$
by (rule *create-preserves-non-merkle-lookup*[*OF outcome*]) *simp-all*
show $\text{fmlookup}(\text{HashMap } t) (\text{TraceFriChallenge } i \ x) =$
 $\text{fmlookup}(\text{HashMap } s) (\text{TraceFriChallenge } i \ x)$
by (rule *create-preserves-non-merkle-lookup*[*OF outcome*]) *simp-all*
show $\text{fmlookup}(\text{HashMap } t) (\text{CompositionFriChallenge } i \ x) =$
 $\text{fmlookup}(\text{HashMap } s) (\text{CompositionFriChallenge } i \ x)$
by (rule *create-preserves-non-merkle-lookup*[*OF outcome*]) *simp-all*

qed

lemma *check-created-tree-outcome*:
assumes *created*: *created-tree xs r s0*
and *length-xs*: $\text{length } xs = 2^n$
and *i-bound*: $i < \text{length } xs$
and *v*: $v = xs ! i$
and *ext*: $s0 \leq s$
and *check*:
Some (h, t) ∈ set-dist (execute
(check-authentication-path (length xs) i v
(get-authentication-path (length xs) i r)) s)
shows $h = \text{value } r \wedge s \leq t$
using *protocol-check-created-tree-outcome*
[OF created[unfolded created-tree-def] length-xs i-bound v ext
check[unfolded check-authentication-path-def]]
by *simp*

lemma *receive-random-field-element-no-failure*:
None ∉ dom (dist (execute receive-random-field-element s))
unfolding *receive-random-field-element-def*
by *(rule protocol-receive-random-field-element-no-failure)*

lemma *receive-tagged-random-field-element-no-failure*:
None ∉ dom (dist (execute (receive-tagged-random-field-element tag) s))
unfolding *receive-tagged-random-field-element-def*
by *(rule protocol-receive-tagged-random-field-element-no-failure)*

lemma *receive-trace-fri-challenge-no-failure*:
None ∉ dom (dist (execute receive-trace-fri-challenge s))
unfolding *receive-trace-fri-challenge-def*
by *(rule protocol-receive-counted-tagged-random-field-element-no-failure)*

lemma *receive-composition-fri-challenge-no-failure*:
None ∉ dom (dist (execute receive-composition-fri-challenge s))
unfolding *receive-composition-fri-challenge-def*
by *(rule protocol-receive-counted-tagged-random-field-element-no-failure)*

lemma *receive-alpha-challenge-no-failure*:
None ∉ dom (dist (execute receive-alpha-challenge s))
unfolding *receive-alpha-challenge-def*
by *(rule protocol-receive-counted-tagged-random-field-element-no-failure)*

lemma *receive-query-index-challenge-no-failure*:
None ∉ dom (dist (execute receive-query-index-challenge s))
unfolding *receive-query-index-challenge-def*
by *(rule protocol-receive-counted-tagged-random-field-element-no-failure)*

lemma *receive-random-field-element-outcome:*

assumes $\text{Some } (r, t) \in$
 $\text{set-dist } (\text{execute } \text{receive-random-field-element } s)$
shows
 $s \leq t \wedge$
 $P\text{State } t = P\text{State } s \wedge$
 $P\text{Transcript } t = P\text{Transcript } s \wedge$
 $\text{fmlookup } (\text{HashMap } t) (\text{FiatShamirChallenge } (P\text{State } s)) = \text{Some } r$
using *protocol-receive-random-field-element-outcome*
 $[\text{OF } \text{assms}[\text{unfolded } \text{receive-random-field-element-def}]]$
by *simp*

lemma *receive-tagged-random-field-element-outcome:*

assumes $\text{Some } (r, t) \in$
 $\text{set-dist } (\text{execute } (\text{receive-tagged-random-field-element } \text{tag}) s)$
shows
 $s \leq t \wedge$
 $P\text{State } t = P\text{State } s \wedge$
 $P\text{Transcript } t = P\text{Transcript } s \wedge$
 $\text{fmlookup } (\text{HashMap } t) (\text{tag } (P\text{State } s)) = \text{Some } r$
using *protocol-receive-tagged-random-field-element-outcome*
 $[\text{OF } \text{assms}[\text{unfolded } \text{receive-tagged-random-field-element-def}]]$
by *simp*

lemma *receive-tagged-random-field-element-preserves-other-lookup:*

assumes $\text{Some } (r, t) \in$
 $\text{set-dist } (\text{execute } (\text{receive-tagged-random-field-element } \text{tag}) s)$
and $y \neq \text{tag } (P\text{State } s)$
shows $\text{fmlookup } (\text{HashMap } t) y = \text{fmlookup } (\text{HashMap } s) y$
using *protocol-receive-tagged-random-field-element-preserves-other-lookup*
 $[\text{OF } \text{assms}(1)[\text{unfolded } \text{receive-tagged-random-field-element-def}] \text{ assms}(2)]$
by *simp*

lemma *receive-trace-fri-challenge-outcome:*

assumes $\text{Some } (r, t) \in \text{set-dist } (\text{execute } \text{receive-trace-fri-challenge } s)$
shows
 $s \leq t \wedge$
 $P\text{State } t = P\text{State } s \wedge$
 $P\text{Transcript } t = P\text{Transcript } s \wedge$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{TraceFriChallenge } (P\text{TraceFriCounter } s) (P\text{State } s)) = \text{Some } r$
using *protocol-receive-counted-tagged-random-field-element-outcome*
 $[\text{OF } \text{assms}[\text{unfolded } \text{receive-trace-fri-challenge-def}]]$
by *simp*

lemma *receive-composition-fri-challenge-outcome:*

assumes $\text{Some } (r, t) \in$
 $\text{set-dist } (\text{execute } \text{receive-composition-fri-challenge } s)$
shows

$s \leq t \wedge$
 $PState\ t = PState\ s \wedge$
 $PTranscript\ t = PTranscript\ s \wedge$
 $fmlookup\ (HashMap\ t)$
 $(CompositionFriChallenge\ (PCompositionFriCounter\ s)\ (PState\ s)) =$
 $Some\ r$
using *protocol-receive-counted-tagged-random-field-element-outcome*
 $[OF\ assms[unfolding\ receive-composition-fri-challenge-def]]$
by *simp*

lemma *receive-alpha-challenge-outcome:*

assumes $Some\ (r, t) \in set-dist\ (execute\ receive-alpha-challenge\ s)$
shows
 $s \leq t \wedge$
 $PState\ t = PState\ s \wedge$
 $PTranscript\ t = PTranscript\ s \wedge$
 $fmlookup\ (HashMap\ t)$
 $(AlphaChallenge\ (PAlphaCounter\ s)\ (PState\ s)) = Some\ r$
using *protocol-receive-counted-tagged-random-field-element-outcome*
 $[OF\ assms[unfolding\ receive-alpha-challenge-def]]$
by *simp*

lemma *receive-query-index-challenge-outcome:*

assumes $Some\ (r, t) \in set-dist\ (execute\ receive-query-index-challenge\ s)$
shows
 $s \leq t \wedge$
 $PState\ t = PState\ s \wedge$
 $PTranscript\ t = PTranscript\ s \wedge$
 $fmlookup\ (HashMap\ t)$
 $(QueryIndexChallenge\ (PQueryCounter\ s)\ (PState\ s)) = Some\ r$
using *protocol-receive-counted-tagged-random-field-element-outcome*
 $[OF\ assms[unfolding\ receive-query-index-challenge-def]]$
by *simp*

lemma *receive-trace-fri-challenge-preserves-other-lookup:*

assumes $Some\ (r, t) \in set-dist\ (execute\ receive-trace-fri-challenge\ s)$
and $y \neq TraceFriChallenge\ (PTraceFriCounter\ s)\ (PState\ s)$
shows $fmlookup\ (HashMap\ t)\ y = fmlookup\ (HashMap\ s)\ y$
using *protocol-receive-counted-tagged-random-field-element-preserves-other-lookup*
 $[OF\ assms(1)[unfolding\ receive-trace-fri-challenge-def]\ assms(2)]$
by *simp*

lemma *receive-composition-fri-challenge-preserves-other-lookup:*

assumes $Some\ (r, t) \in$
 $set-dist\ (execute\ receive-composition-fri-challenge\ s)$
and $y \neq CompositionFriChallenge\ (PCompositionFriCounter\ s)\ (PState\ s)$
shows $fmlookup\ (HashMap\ t)\ y = fmlookup\ (HashMap\ s)\ y$
using *protocol-receive-counted-tagged-random-field-element-preserves-other-lookup*
 $[OF\ assms(1)[unfolding\ receive-composition-fri-challenge-def]\ assms(2)]$

by simp

lemma *receive-alpha-challenge-preserves-other-lookup:*

assumes *Some* $(r, t) \in \text{set-dist}$ (execute *receive-alpha-challenge* s)
and $y \neq \text{AlphaChallenge}$ ($\text{PAlphaCounter } s$) ($\text{PState } s$)
shows fmlookup ($\text{HashMap } t$) $y = \text{fmlookup}$ ($\text{HashMap } s$) y
using *protocol-receive-counted-tagged-random-field-element-preserves-other-lookup*
 $[\text{OF } \text{assms}(1)[\text{unfolded } \text{receive-alpha-challenge-def}] \text{ assms}(2)]$
by simp

lemma *receive-query-index-challenge-preserves-other-lookup:*

assumes *Some* $(r, t) \in \text{set-dist}$ (execute *receive-query-index-challenge* s)
and $y \neq \text{QueryIndexChallenge}$ ($\text{PQueryCounter } s$) ($\text{PState } s$)
shows fmlookup ($\text{HashMap } t$) $y = \text{fmlookup}$ ($\text{HashMap } s$) y
using *protocol-receive-counted-tagged-random-field-element-preserves-other-lookup*
 $[\text{OF } \text{assms}(1)[\text{unfolded } \text{receive-query-index-challenge-def}] \text{ assms}(2)]$
by simp

lemma *receive-trace-fri-challenge-counter-outcome:*

assumes *Some* $(r, t) \in \text{set-dist}$ (execute *receive-trace-fri-challenge* s)
shows
 $\text{PTraceFriCounter } t = \text{Suc } (\text{PTraceFriCounter } s) \wedge$
 $\text{PCompositionFriCounter } t = \text{PCompositionFriCounter } s \wedge$
 $\text{PAlphaCounter } t = \text{PAlphaCounter } s \wedge$
 $\text{PQueryCounter } t = \text{PQueryCounter } s$

proof –

from *assms* **obtain** $s0\ h\ u\ v$ **where**

get-s: *Some* $(s0, s0) \in \text{set-dist}$ (execute *get* s)

and *hash-out*:

Some $(h, u) \in$
 set-dist (execute
 $(\text{hash } (\text{TraceFriChallenge } (\text{PTraceFriCounter } s0) (\text{PState } s0))) s0)$

and *mod*:

Some $((), v) \in$
 set-dist (execute
 $(\text{modify } (\lambda s. s(\text{PTraceFriCounter} := \text{Suc } (\text{PTraceFriCounter } s)))) u)$

and *ret*: *Some* $(r, t) \in \text{set-dist}$ (execute (return h) v)

unfolding *receive-trace-fri-challenge-def*

protocol-receive-counted-tagged-random-field-element-def

by (auto elim!: *protocol-merkle.set-dist-bindE*)

have $s0: s0 = s$

using *get-s* **by** simp

have $v: v = u(\text{PTraceFriCounter} := \text{Suc } (\text{PTraceFriCounter } u))$

using *mod* **unfolding** *modify-def*

by (auto elim!: *protocol-merkle.set-dist-bindE*)

have $t: t = v$

using *ret* **by** simp

have *counters-u*:

$\text{PTraceFriCounter } u = \text{PTraceFriCounter } s \wedge$


```

    PCompositionFriCounter u = PCompositionFriCounter s ∧
    PAlphaCounter u = PAlphaCounter s ∧
    PQueryCounter u = PQueryCounter s
  using protocol-hash-channel-preserves[OF hash-out[unfolded s0]]
  by simp
show ?thesis
  unfolding t v using counters-u by simp
qed

lemma receive-composition-fri-challenge-counter-outcome:
  assumes Some (r, t) ∈ set-dist (execute receive-composition-fri-challenge s)
  shows
    PTraceFriCounter t = PTraceFriCounter s ∧
    PCompositionFriCounter t = Suc (PCompositionFriCounter s) ∧
    PAlphaCounter t = PAlphaCounter s ∧
    PQueryCounter t = PQueryCounter s
proof -
  from assms obtain s0 h u v where
    get-s: Some (s0, s0) ∈ set-dist (execute get s)
  and hash-out:
    Some (h, u) ∈
      set-dist (execute
        (hash (CompositionFriChallenge
          (PCompositionFriCounter s0) (PState s0))) s0)
  and mod:
    Some ((), v) ∈
      set-dist (execute
        (modify (λs. s(|
          PCompositionFriCounter := Suc (PCompositionFriCounter s)|)) u)
  and ret: Some (r, t) ∈ set-dist (execute (return h) v)
  unfolding receive-composition-fri-challenge-def
    protocol-receive-counted-tagged-random-field-element-def
  by (auto elim!: protocol-merkle.set-dist-bindE)
  have s0: s0 = s
  using get-s by simp
  have v: v = u(|
    PCompositionFriCounter := Suc (PCompositionFriCounter u)|)
  using mod unfolding modify-def
  by (auto elim!: protocol-merkle.set-dist-bindE)
  have t: t = v
  using ret by simp
  have counters-u:
    PTraceFriCounter u = PTraceFriCounter s ∧
    PCompositionFriCounter u = PCompositionFriCounter s ∧
    PAlphaCounter u = PAlphaCounter s ∧
    PQueryCounter u = PQueryCounter s
  using protocol-hash-channel-preserves[OF hash-out[unfolded s0]]
  by simp
show ?thesis

```

unfolding $t\ v$ **using** $\text{counters-}u$ **by** simp
qed

lemma *receive-alpha-challenge-counter-outcome:*

assumes $\text{Some } (r, t) \in \text{set-dist } (\text{execute } \text{receive-alpha-challenge } s)$

shows

$P\text{TraceFriCounter } t = P\text{TraceFriCounter } s \wedge$
 $P\text{CompositionFriCounter } t = P\text{CompositionFriCounter } s \wedge$
 $P\text{AlphaCounter } t = \text{Suc } (P\text{AlphaCounter } s) \wedge$
 $P\text{QueryCounter } t = P\text{QueryCounter } s$

proof –

from assms **obtain** $s0\ h\ u\ v$ **where**

$\text{get-s: } \text{Some } (s0, s0) \in \text{set-dist } (\text{execute } \text{get } s)$

and hash-out:

$\text{Some } (h, u) \in$
 $\text{set-dist } (\text{execute } (\text{hash } (\text{AlphaChallenge } (P\text{AlphaCounter } s0) (P\text{State } s0))) s0)$

and mod:

$\text{Some } ((), v) \in$
 $\text{set-dist } (\text{execute } (\text{modify } (\lambda s. s(P\text{AlphaCounter } := \text{Suc } (P\text{AlphaCounter } s)))) u)$

and $\text{ret: } \text{Some } (r, t) \in \text{set-dist } (\text{execute } (\text{return } h) v)$

unfolding *receive-alpha-challenge-def*

protocol-receive-counted-tagged-random-field-element-def

by $(\text{auto elim!: protocol-merkle.set-dist-bindE})$

have $s0: s0 = s$

using get-s **by** simp

have $v: v = u(P\text{AlphaCounter } := \text{Suc } (P\text{AlphaCounter } u))$

using mod **unfolding** *modify-def*

by $(\text{auto elim!: protocol-merkle.set-dist-bindE})$

have $t: t = v$

using ret **by** simp

have $\text{counters-}u:$

$P\text{TraceFriCounter } u = P\text{TraceFriCounter } s \wedge$
 $P\text{CompositionFriCounter } u = P\text{CompositionFriCounter } s \wedge$
 $P\text{AlphaCounter } u = P\text{AlphaCounter } s \wedge$
 $P\text{QueryCounter } u = P\text{QueryCounter } s$

using *protocol-hash-channel-preserves* $[OF \text{ hash-out}[\text{unfolded } s0]]$

by simp

show *?thesis*

unfolding $t\ v$ **using** $\text{counters-}u$ **by** simp

qed

lemma *receive-random-field-element-counter-outcome:*

assumes $\text{Some } (r, t) \in \text{set-dist } (\text{execute } \text{receive-random-field-element } s)$

shows

$P\text{TraceFriCounter } t = P\text{TraceFriCounter } s \wedge$
 $P\text{CompositionFriCounter } t = P\text{CompositionFriCounter } s \wedge$
 $P\text{AlphaCounter } t = P\text{AlphaCounter } s \wedge$

$PQueryCounter\ t = PQueryCounter\ s$
proof –
from *assms* **obtain** $s0$ **where**
 $get\text{-}s: Some\ (s0, s0) \in set\text{-}dist\ (execute\ get\ s)$
and *hash-out*:
 $Some\ (r, t) \in$
 $set\text{-}dist\ (execute\ (hash\ (FiatShamirChallenge\ (PState\ s0)) \gg= return)\ s0)$
unfolding *receive-random-field-element-def*
 $protocol\text{-}receive\text{-}random\text{-}field\text{-}element\text{-}def$
 $protocol\text{-}receive\text{-}tagged\text{-}random\text{-}field\text{-}element\text{-}def$
by $(auto\ elim!: protocol\text{-}merkle.set\text{-}dist\text{-}bindE)$
have $s0: s0 = s$
using *get-s* **by** *simp*
from *hash-out* **obtain** $h\ u$ **where**
hash-u:
 $Some\ (h, u) \in$
 $set\text{-}dist\ (execute\ (hash\ (FiatShamirChallenge\ (PState\ s)))\ s)$
and *ret*: $Some\ (r, t) \in set\text{-}dist\ (execute\ (return\ h)\ u)$
unfolding $s0$ **by** $(auto\ elim!: protocol\text{-}merkle.set\text{-}dist\text{-}bindE)$
have $t: t = u$
using *ret* **by** *simp*
show *?thesis*
using *protocol-hash-channel-preserves*[*OF hash-u*]
unfolding t **by** *simp*
qed

lemma *receive-query-index-challenge-counter-outcome*:

assumes $Some\ (r, t) \in set\text{-}dist\ (execute\ receive\text{-}query\text{-}index\text{-}challenge\ s)$

shows

$PTraceFriCounter\ t = PTraceFriCounter\ s \wedge$

$PCompositionFriCounter\ t = PCompositionFriCounter\ s \wedge$

$PAlphaCounter\ t = PAlphaCounter\ s \wedge$

$PQueryCounter\ t = Suc\ (PQueryCounter\ s)$

proof –

from *assms* **obtain** $s0\ h\ u\ v$ **where**

$get\text{-}s: Some\ (s0, s0) \in set\text{-}dist\ (execute\ get\ s)$

and *hash-out*:

$Some\ (h, u) \in$

$set\text{-}dist\ (execute$

$(hash\ (QueryIndexChallenge\ (PQueryCounter\ s0)\ (PState\ s0)))\ s0)$

and *mod*:

$Some\ ((), v) \in$

$set\text{-}dist\ (execute$

$(modify\ (\lambda s. s(PQueryCounter := Suc\ (PQueryCounter\ s))))\ u)$

and *ret*: $Some\ (r, t) \in set\text{-}dist\ (execute\ (return\ h)\ v)$

unfolding *receive-query-index-challenge-def*

$protocol\text{-}receive\text{-}counted\text{-}tagged\text{-}random\text{-}field\text{-}element\text{-}def$

by $(auto\ elim!: protocol\text{-}merkle.set\text{-}dist\text{-}bindE)$

have $s0: s0 = s$

```

    using get-s by simp
  have v: v = u(PQueryCounter := Suc (PQueryCounter u))
    using mod unfolding modify-def
    by (auto elim!: protocol-merkle.set-dist-bindE)
  have t: t = v
    using ret by simp
  have counters-u:
    PTraceFriCounter u = PTraceFriCounter s ∧
    PCompositionFriCounter u = PCompositionFriCounter s ∧
    PAlphaCounter u = PAlphaCounter s ∧
    PQueryCounter u = PQueryCounter s
    using protocol-hash-channel-preserves[OF hash-out[unfolded s0]]
    by simp
  show ?thesis
    unfolding t v using counters-u by simp
qed

```

lemma *bind-preserves-lookup*:

```

  fixes m :: ('x, ('a, 'b) protocol-channel-scheme) state-monad
    and k :: 'x ⇒ ('y, ('a, 'b) protocol-channel-scheme) state-monad
  assumes outcome: Some (y, t) ∈ set-dist (execute (m ≫ k) s)
    and m-pres:
      ∧ x u. Some (x, u) ∈ set-dist (execute m s) ⇒
        fmlookup (HashMap u) key = fmlookup (HashMap s) key
    and k-pres:
      ∧ x u y t. Some (x, u) ∈ set-dist (execute m s) ⇒
        Some (y, t) ∈ set-dist (execute (k x) u) ⇒
          fmlookup (HashMap t) key = fmlookup (HashMap u) key
  shows fmlookup (HashMap t) key = fmlookup (HashMap s) key

```

proof –

```

  from outcome obtain x u where
    m-out: Some (x, u) ∈ set-dist (execute m s)
    and k-out: Some (y, t) ∈ set-dist (execute (k x) u)
    by (elim set-dist-bindE)
  have fmlookup (HashMap t) key = fmlookup (HashMap u) key
    by (rule k-pres[OF m-out k-out])
  also have ... = fmlookup (HashMap s) key
    by (rule m-pres[OF m-out])
  finally show ?thesis .

```

qed

lemma *mmap-preserves-lookup*:

```

  fixes ms :: ('x, ('a, 'b) protocol-channel-scheme) state-monad list
  assumes step:
    ∧ m x s t. m ∈ set ms ⇒
      Some (x, t) ∈ set-dist (execute m s) ⇒
        fmlookup (HashMap t) key = fmlookup (HashMap s) key
    and outcome: Some (xs, t) ∈ set-dist (execute (mmap ms) s)
  shows fmlookup (HashMap t) key = fmlookup (HashMap s) key

```

```

    using step outcome
  proof (induction ms arbitrary: xs s t)
    case Nil
    then show ?case
      by simp
  next
    case (Cons m ms)
  from Cons.prem1(2) obtain x u ys where
    m-out: Some (x, u) ∈ set-dist (execute m s)
    and tail-out: Some (ys, t) ∈ set-dist (execute (mmap ms) u)
    and xs-eq: xs = x # ys
    by (auto elim!: set-dist-bindE)
  have head-pres: fmlookup (HashMap u) key = fmlookup (HashMap s) key
    by (rule Cons.prem1(1)) (use m-out in simp-all)
  have step-tail:
     $\bigwedge m x s t. m \in \text{set } ms \implies$ 
     $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m s) \implies$ 
     $\text{fmlookup } (\text{HashMap } t) \text{ key} = \text{fmlookup } (\text{HashMap } s) \text{ key}$ 
  proof -
    fix m x s t
    assume mem: m ∈ set ms
    and out: Some (x, t) ∈ set-dist (execute m s)
    show fmlookup (HashMap t) key = fmlookup (HashMap s) key
      by (rule Cons.prem1(1)) (use mem out in simp-all)
    qed
  have tail-pres: fmlookup (HashMap t) key = fmlookup (HashMap u) key
    by (rule Cons.IH[OF step-tail tail-out])
  show ?case
    using head-pres tail-pres by simp
  qed

lemma mfold2-preserves-lookup:
  fixes f :: 'x ⇒ (unit, ('a, 'b) protocol-channel-scheme) state-monad
  assumes step:
     $\bigwedge x s t. x \in \text{set } xs \implies$ 
     $\text{Some } ((), t) \in \text{set-dist } (\text{execute } (f x) s) \implies$ 
     $\text{fmlookup } (\text{HashMap } t) \text{ key} = \text{fmlookup } (\text{HashMap } s) \text{ key}$ 
    and outcome: Some ((), t) ∈ set-dist (execute (mfold2 f xs) s)
  shows fmlookup (HashMap t) key = fmlookup (HashMap s) key
    using step outcome
  proof (induction xs arbitrary: s t)
    case Nil
    then show ?case
      by simp
  next
    case (Cons x xs)
  from Cons.prem1(2) obtain u where
    head-out: Some ((), u) ∈ set-dist (execute (f x) s)
    and tail-out: Some ((), t) ∈ set-dist (execute (mfold2 f xs) u)

```

```

  by (auto elim!: set-dist-bindE)
have head-pres: fmlookup (HashMap u) key = fmlookup (HashMap s) key
  by (rule Cons.prem1) (use head-out in simp-all)
have step-tail:
   $\bigwedge x s t. x \in \text{set } xs \implies$ 
    Some  $(((), t) \in \text{set-dist } (\text{execute } (f x) s) \implies$ 
      fmlookup (HashMap t) key = fmlookup (HashMap s) key
proof -
  fix x s t
  assume mem:  $x \in \text{set } xs$ 
  and out: Some  $(((), t) \in \text{set-dist } (\text{execute } (f x) s)$ 
  show fmlookup (HashMap t) key = fmlookup (HashMap s) key
    by (rule Cons.prem1) (use mem out in simp-all)
qed
have tail-pres: fmlookup (HashMap t) key = fmlookup (HashMap u) key
  by (rule Cons.IH[OF step-tail tail-out])
show ?case
  using head-pres tail-pres by simp
qed

```

```

lemma mfold-preserves-lookup:
  fixes ms ::  $('x \Rightarrow ('a, 'b) \text{ protocol-channel-scheme}) \text{ state-monad} \text{ list}$ 
  assumes step:
     $\bigwedge m a x s t. m \in \text{set } ms \implies$ 
      Some  $(x, t) \in \text{set-dist } (\text{execute } (m a) s) \implies$ 
        fmlookup (HashMap t) key = fmlookup (HashMap s) key
    and outcome: Some  $(x, t) \in \text{set-dist } (\text{execute } (\text{mfold } a ms) s)$ 
  shows fmlookup (HashMap t) key = fmlookup (HashMap s) key
  using step outcome
proof (induction ms arbitrary: a x s t)
  case Nil
  then show ?case
    by simp
next
  case (Cons m ms)
  from Cons.prem2 obtain y u where
    head-out: Some  $(y, u) \in \text{set-dist } (\text{execute } (m a) s)$ 
    and tail-out: Some  $(x, t) \in \text{set-dist } (\text{execute } (\text{mfold } y ms) u)$ 
    by (auto elim!: set-dist-bindE)
  have head-pres: fmlookup (HashMap u) key = fmlookup (HashMap s) key
    by (rule Cons.prem1) (use head-out in simp-all)
  have step-tail:
     $\bigwedge m a x s t. m \in \text{set } ms \implies$ 
      Some  $(x, t) \in \text{set-dist } (\text{execute } (m a) s) \implies$ 
        fmlookup (HashMap t) key = fmlookup (HashMap s) key
  proof -
    fix m a x s t
    assume mem:  $m \in \text{set } ms$ 
    and out: Some  $(x, t) \in \text{set-dist } (\text{execute } (m a) s)$ 

```

show $\text{fmlookup } (\text{HashMap } t) \text{ key} = \text{fmlookup } (\text{HashMap } s) \text{ key}$
by (rule $\text{Cons.prem}(1)$) (use mem out in simp-all)
qed
have $\text{tail-pres: fmlookup } (\text{HashMap } t) \text{ key} = \text{fmlookup } (\text{HashMap } u) \text{ key}$
by (rule $\text{Cons.IH}[OF \text{ step-tail tail-out}]$)
show ?case
using head-pres tail-pres **by** simp
qed

lemma *ntimes-preserves-lookup*:
fixes $m :: ('x, ('a, 'b) \text{ protocol-channel-scheme}) \text{ state-monad}$
assumes *step*:
 $\bigwedge x s t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m s) \implies$
 $\text{fmlookup } (\text{HashMap } t) \text{ key} = \text{fmlookup } (\text{HashMap } s) \text{ key}$
and *outcome*: $\text{Some } (xs, t) \in \text{set-dist } (\text{execute } (\text{ntimes } m n) s)$
shows $\text{fmlookup } (\text{HashMap } t) \text{ key} = \text{fmlookup } (\text{HashMap } s) \text{ key}$
using *step outcome*
proof (induction n arbitrary: $xs s t$)
case 0
then show ?case
by simp
next
case (Suc n)
from $\text{Suc.prem}(2)$ **obtain** $x u ys$ **where**
 $\text{head-out: Some } (x, u) \in \text{set-dist } (\text{execute } m s)$
and $\text{tail-out: Some } (ys, t) \in \text{set-dist } (\text{execute } (\text{ntimes } m n) u)$
and $xs\text{-eq: } xs = x \# ys$
by (auto elim!: set-dist-bindE)
have $\text{head-pres: fmlookup } (\text{HashMap } u) \text{ key} = \text{fmlookup } (\text{HashMap } s) \text{ key}$
by (rule $\text{Suc.prem}(1)[OF \text{ head-out}]$)
have $\text{tail-pres: fmlookup } (\text{HashMap } t) \text{ key} = \text{fmlookup } (\text{HashMap } u) \text{ key}$
by (rule $\text{Suc.IH}[OF \text{ Suc.prem}(1) \text{ tail-out}]$)
show ?case
using head-pres tail-pres **by** simp
qed

lemma *receive-random-field-element-known-outcome*:
assumes *lookup*:
 $\text{fmlookup } (\text{HashMap } s) (\text{FiatShamirChallenge } (PState s)) = \text{Some } r$
and *outcome*:
 $\text{Some } (r', t) \in \text{set-dist } (\text{execute receive-random-field-element } s)$
shows
 $r' = r \wedge PState t = PState s \wedge$
 $PTranscript t = PTranscript s \wedge s \leq t$
proof –
have *out*:
 $s \leq t \wedge$
 $PState t = PState s \wedge$
 $PTranscript t = PTranscript s \wedge$

$fmlookup (HashMap\ t) (FiatShamirChallenge\ (PState\ s)) = Some\ r'$
by (rule receive-random-field-element-outcome[OF outcome])
have old:
 $fmlookup (HashMap\ t) (FiatShamirChallenge\ (PState\ s)) = Some\ r$
using lookup out
by (meson protocol-merkle.hash-extension-lookup)
show ?thesis
using out old **by** auto
qed

lemma receive-tagged-random-field-element-known-outcome:

assumes lookup:
 $fmlookup (HashMap\ s) (tag\ (PState\ s)) = Some\ r$
and outcome:
 $Some\ (r', t) \in set-dist\ (execute\ (receive-tagged-random-field-element\ tag)\ s)$
shows
 $r' = r \wedge PState\ t = PState\ s \wedge$
 $PTranscript\ t = PTranscript\ s \wedge s \leq t$

proof –
have out:
 $s \leq t \wedge$
 $PState\ t = PState\ s \wedge$
 $PTranscript\ t = PTranscript\ s \wedge$
 $fmlookup (HashMap\ t) (tag\ (PState\ s)) = Some\ r'$
by (rule receive-tagged-random-field-element-outcome[OF outcome])
have old:
 $fmlookup (HashMap\ t) (tag\ (PState\ s)) = Some\ r$
using lookup out
by (meson protocol-merkle.hash-extension-lookup)
show ?thesis
using out old **by** auto
qed

lemma receive-trace-fri-challenge-known-outcome:

assumes fmlookup (HashMap s)
 $(TraceFriChallenge\ (PTraceFriCounter\ s) (PState\ s)) = Some\ r$
and $Some\ (r', t) \in set-dist\ (execute\ receive-trace-fri-challenge\ s)$
shows
 $r' = r \wedge PState\ t = PState\ s \wedge$
 $PTranscript\ t = PTranscript\ s \wedge s \leq t$

proof –
have out:
 $s \leq t \wedge$
 $PState\ t = PState\ s \wedge$
 $PTranscript\ t = PTranscript\ s \wedge$
 $fmlookup (HashMap\ t)$
 $(TraceFriChallenge\ (PTraceFriCounter\ s) (PState\ s)) = Some\ r'$
by (rule receive-trace-fri-challenge-outcome[OF assms(2)])

have *old*:
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{TraceFriChallenge } (P\text{TraceFriCounter } s) (P\text{State } s)) = \text{Some } r$
using *assms*(1) *out*
by (*meson protocol-merkle.hash-extension-lookup*)
show ?*thesis*
using *out old* **by** *auto*
qed

lemma *receive-composition-fri-challenge-known-outcome*:
assumes $\text{fmlookup } (\text{HashMap } s)$
 $(\text{CompositionFriChallenge } (P\text{CompositionFriCounter } s) (P\text{State } s)) = \text{Some } r$
and $\text{Some } (r', t) \in \text{set-dist } (\text{execute receive-composition-fri-challenge } s)$
shows
 $r' = r \wedge P\text{State } t = P\text{State } s \wedge$
 $P\text{Transcript } t = P\text{Transcript } s \wedge s \leq t$
proof –
have *out*:
 $s \leq t \wedge$
 $P\text{State } t = P\text{State } s \wedge$
 $P\text{Transcript } t = P\text{Transcript } s \wedge$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{CompositionFriChallenge } (P\text{CompositionFriCounter } s) (P\text{State } s)) =$
 $\text{Some } r'$
by (*rule receive-composition-fri-challenge-outcome*[*OF assms*(2)])
have *old*:
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{CompositionFriChallenge } (P\text{CompositionFriCounter } s) (P\text{State } s)) =$
 $\text{Some } r$
using *assms*(1) *out*
by (*meson protocol-merkle.hash-extension-lookup*)
show ?*thesis*
using *out old* **by** *auto*
qed

lemma *receive-alpha-challenge-known-outcome*:
assumes $\text{fmlookup } (\text{HashMap } s)$
 $(\text{AlphaChallenge } (P\text{AlphaCounter } s) (P\text{State } s)) = \text{Some } r$
and $\text{Some } (r', t) \in \text{set-dist } (\text{execute receive-alpha-challenge } s)$
shows
 $r' = r \wedge P\text{State } t = P\text{State } s \wedge$
 $P\text{Transcript } t = P\text{Transcript } s \wedge s \leq t$
proof –
have *out*:
 $s \leq t \wedge$
 $P\text{State } t = P\text{State } s \wedge$
 $P\text{Transcript } t = P\text{Transcript } s \wedge$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{AlphaChallenge } (P\text{AlphaCounter } s) (P\text{State } s)) = \text{Some } r'$

```

    by (rule receive-alpha-challenge-outcome[OF assms(2)])
  have old:
    fmlookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s) (PState s)) = Some r
    using assms(1) out
    by (meson protocol-merkle.hash-extension-lookup)
  show ?thesis
    using out old by auto
qed

```

lemma *receive-query-index-challenge-known-outcome:*

```

  assumes fmlookup (HashMap s)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some r
  and Some (r', t) ∈ set-dist (execute receive-query-index-challenge s)
  shows
    r' = r ∧ PState t = PState s ∧
    PTranscript t = PTranscript s ∧ s ≤ t
  proof -
    have out:
      s ≤ t ∧
      PState t = PState s ∧
      PTranscript t = PTranscript s ∧
      fmlookup (HashMap t)
        (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some r'
    by (rule receive-query-index-challenge-outcome[OF assms(2)])
  have old:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some r
    using assms(1) out
    by (meson protocol-merkle.hash-extension-lookup)
  show ?thesis
    using out old by auto
qed

```

lemma *read-no-failure:*

```

  assumes PTranscript s ≠ []
  shows None ∉ dom (dist (execute read s))
  using assms
  unfolding read-def protocol-read-def assert-def
  by (intro no-failure-bindI) simp-all

```

lemma *read-hash-extends:*

```

  assumes Some (x, t) ∈ set-dist (execute read s)
  shows s ≤ t
  proof (cases PTranscript s)
    case Nil
  from assms obtain s0 u where
    get-s: Some (s0, s0) ∈ set-dist (execute get s)
    and assert-u:

```

```

    Some ( $\emptyset$ ,  $u$ )  $\in$ 
      set-dist (execute (assert (PTranscript  $s0 \neq \emptyset$ ))  $s0$ )
  unfolding read-def protocol-read-def
  by (auto elim!: set-dist-bindE)
have  $s0 = s$ 
  using get-s by simp
with Nil assert-u have False
  unfolding assert-def
  using throw-no-Some-set-dist by force
then show ?thesis by simp
next
case (Cons a list)
then show ?thesis
  using assms
  unfolding read-def protocol-read-def assert-def
    less-eq-hash-ext-def less-eq-fmap-def modify-def
  by (auto elim!: set-dist-bindE)
qed

lemma receive-random-field-element-extends:
  assumes Some ( $r$ ,  $t$ )  $\in$ 
    set-dist (execute receive-random-field-element  $s$ )
  shows  $s \leq t$ 
  using receive-random-field-element-outcome[OF assms] by simp

lemma receive-tagged-random-field-element-extends:
  assumes Some ( $r$ ,  $t$ )  $\in$ 
    set-dist (execute (receive-tagged-random-field-element tag)  $s$ )
  shows  $s \leq t$ 
  using receive-tagged-random-field-element-outcome[OF assms] by simp

lemma receive-trace-fri-challenge-extends:
  assumes Some ( $r$ ,  $t$ )  $\in$  set-dist (execute receive-trace-fri-challenge  $s$ )
  shows  $s \leq t$ 
  using receive-trace-fri-challenge-outcome[OF assms] by simp

lemma receive-composition-fri-challenge-extends:
  assumes Some ( $r$ ,  $t$ )  $\in$ 
    set-dist (execute receive-composition-fri-challenge  $s$ )
  shows  $s \leq t$ 
  using receive-composition-fri-challenge-outcome[OF assms] by simp

lemma receive-alpha-challenge-extends:
  assumes Some ( $r$ ,  $t$ )  $\in$  set-dist (execute receive-alpha-challenge  $s$ )
  shows  $s \leq t$ 
  using receive-alpha-challenge-outcome[OF assms] by simp

lemma receive-query-index-challenge-extends:
  assumes Some ( $r$ ,  $t$ )  $\in$  set-dist (execute receive-query-index-challenge  $s$ )

```

shows $s \leq t$
using *receive-query-index-challenge-outcome*[*OF assms*] **by** *simp*

lemma *send-outcome*:

assumes *Some* $(((), t) \in \text{set-dist } (\text{execute } (\text{send } x) s))$
shows
 $t = s \langle \langle$
 $\quad PState := \text{concat } (PState s) x,$
 $\quad PTranscript := x \# PTranscript s \rangle \rangle$
using *protocol-send-outcome*[*OF assms*[*unfolded send-def*]] .

lemma *read-cons-outcome*:

assumes *Some* $(y, t) \in \text{set-dist } (\text{execute read } (s \langle \langle PTranscript := x \# xs \rangle \rangle))$
shows
 $y = x \wedge$
 $t = s \langle \langle$
 $\quad PState := \text{concat } (PState s) x,$
 $\quad PTranscript := xs \rangle \rangle$
using *protocol-read-cons-outcome*[*OF assms*[*unfolded read-def*]] .

lemma *read-nonempty-outcome*:

assumes *out*: *Some* $(y, t) \in \text{set-dist } (\text{execute read } s)$
and *tr*: $PTranscript s = x \# xs$
shows
 $y = x \wedge$
 $t = s \langle \langle PState := \text{concat } (PState s) x, PTranscript := xs \rangle \rangle$

proof –

have *s-eq*: $s \langle \langle PTranscript := x \# xs \rangle \rangle = s$
using *tr* **by** (*cases s*) *simp*
have *out'*: *Some* $(y, t) \in \text{set-dist } (\text{execute read } (s \langle \langle PTranscript := x \# xs \rangle \rangle))$
using *out* **unfolding** *s-eq* .

show *?thesis*

using *read-cons-outcome*[*OF out'*] *tr* **by** *simp*

qed

lemma *random-then-read-cons-outcome*:

assumes *lookup*: *fmlookup* (*HashMap s*) (*FiatShamirChallenge* (*PState s*)) = *Some x*
and *outcome*:
 $\text{Some } ((r, y), t) \in \text{set-dist } (\text{execute } (do \{$
 $\quad r \leftarrow \text{receive-random-field-element};$
 $\quad y \leftarrow \text{read};$
 $\quad \text{return } (r, y)$
 $\}) (s \langle \langle PTranscript := x \# xs \rangle \rangle))$

shows

$r = x \wedge$

```

    y = x ∧
    PState t = concat (PState s) x ∧
    PTranscript t = xs
proof –
  let ?s = s(PTranscript := x # xs)
  have outcome':
    Some ((r, y), t) ∈
      set-dist (execute
        (receive-random-field-element >>=
          (λr. read >>= (λy. return (r, y))))) ?s)
  using outcome by simp
  from outcome' obtain r-state where
    rand:
      Some (r, r-state) ∈ set-dist (execute receive-random-field-element ?s)
  and read-y:
    Some (y, t) ∈ set-dist (execute read r-state)
  by (auto elim!: set-dist-bindE)
  have lookup': fmlookup (HashMap ?s) (FiatShamirChallenge (PState ?s)) = Some x
  using lookup by simp
  have rand-res:
    r = x ∧ PState r-state = PState s ∧ PTranscript r-state = x # xs
  using receive-random-field-element-known-outcome[OF lookup' rand] by simp
  have r-state-update: r-state(PTranscript := x # xs) = r-state
  using rand-res by simp
  have read-y':
    Some (y, t) ∈ set-dist (execute read (r-state(PTranscript := x # xs)))
  using read-y unfolding r-state-update .
  have read-res:
    y = x ∧ PState t = concat (PState s) x ∧ PTranscript t = xs
  using read-cons-outcome[OF read-y'] rand-res by simp
  show ?thesis
  using rand-res read-res by simp
qed

```

lemma random-then-read-assert-no-failure:

assumes lookup:

fmlookup (HashMap s) (FiatShamirChallenge (PState s)) = Some x

shows

None ∉ dom (dist (execute

(do {

 r ← receive-random-field-element;

 y ← read;

 assert (r = y);

 return y

}) (s(PTranscript := x # xs))))

proof –

let ?s = s(PTranscript := x # xs)

let ?prefix =

do {

```

  r ← receive-random-field-element;
  y ← read;
  return (r, y)
}
have prefix-no-failure:
  None ∉ dom (dist (execute ?prefix ?s))
proof (rule no-failure-bindI)
  show None ∉ dom (dist (execute receive-random-field-element ?s))
    by (rule receive-random-field-element-no-failure)
  fix r t
  assume rand: Some (r, t) ∈ set-dist (execute receive-random-field-element ?s)
  have lookup-s:
    fmlookup (HashMap ?s) (FiatShamirChallenge (PState ?s)) = Some x
    using lookup by simp
  have tr-t: PTranscript t = x # xs
    using receive-random-field-element-known-outcome[OF lookup-s rand] by simp
  show None ∉ dom (dist (execute (read ≫ (λy. return (r, y))) t))
  proof (rule no-failure-bindI)
    show None ∉ dom (dist (execute read t))
      by (rule read-no-failure) (simp add: tr-t)
    fix y u
    assume Some (y, u) ∈ set-dist (execute read t)
    show None ∉ dom (dist (execute (return (r, y)) u))
      by simp
  qed
qed
have full:
  do {
    r ← receive-random-field-element;
    y ← read;
    assert (r = y);
    return y
  } =
  (?prefix ≫ (λ(r, y). assert (r = y) ≫ (λ-. return y)))
  by (simp add: sm-bind-assoc split: prod.splits)
show ?thesis
  unfolding full
proof (rule no-failure-bindI[OF prefix-no-failure])
  fix ry t
  assume prefix:
    Some (ry, t) ∈ set-dist (execute ?prefix ?s)
  obtain r y where ry: ry = (r, y)
    by (cases ry)
  have replay: r = y
    using random-then-read-cons-outcome[OF lookup prefix[unfolded ry]] by simp
  show None ∉ dom
    (dist (execute ((case ry of (r, y) ⇒ assert (r = y) ≫ (λ-. return y))) t))
    unfolding ry replay assert-def by simp
qed

```

qed

lemma *read-then-random-cons-outcome:*

assumes *lookup:*

fmlookup (*HashMap* *s*)

(*FiatShamirChallenge* (*concat* (*PState* *s*) *x*)) = *Some* *r*

and *outcome:*

Some ((*y*, *r'*), *t*) ∈

set-dist (*execute*

(*do* {

y ← *read*;

r ← *receive-random-field-element*;

return (*y*, *r*)

}) (*s*(*PTranscript* := *x* # *xs*)))

shows

y = *x* ∧

r' = *r* ∧

PState *t* = *concat* (*PState* *s*) *x* ∧

PTranscript *t* = *xs*

proof –

let *?s* = *s*(*PTranscript* := *x* # *xs*)

have *outcome'*:

Some ((*y*, *r'*), *t*) ∈

set-dist (*execute*

(*read* ≫=

(λ*y*. *receive-random-field-element* ≫= (λ*r*. *return* (*y*, *r*)))) *?s*)

using *outcome* **by** *simp*

from *outcome'* **obtain** *read-state* **where**

read-y: *Some* (*y*, *read-state*) ∈ *set-dist* (*execute* *read* *?s*)

and *rand*:

Some (*r'*, *t*) ∈ *set-dist* (*execute* *receive-random-field-element* *read-state*)

by (*auto elim!*: *set-dist-bindE*)

have *read-res*:

y = *x* ∧

read-state = *s*(*PState* := *concat* (*PState* *s*) *x*, *PTranscript* := *xs*)

using *read-cons-outcome*[*OF* *read-y*] **by** *simp*

have *rand-res*:

r' = *r* ∧

PState *t* = *concat* (*PState* *s*) *x* ∧

PTranscript *t* = *xs*

using *receive-random-field-element-known-outcome*[*OF* - *rand*] *lookup* *read-res* **by** *simp*

show *?thesis*

using *read-res* *rand-res* **by** *simp*

qed

lemma *mfold2-send-outcome:*

assumes *Some* (*()*, *t*) ∈ *set-dist* (*execute* (*mfold2* *send* *xs*) *s*)

shows

t = *s*(

```

    PState := foldl concat (PState s) xs,
    PTranscript := rev xs @ PTranscript s)
  using assms
proof (induction xs arbitrary: s t)
  case Nil
  then show ?case by simp
next
  case (Cons x xs)
  from Cons.prem1 obtain s1 where
    send-x: Some ((), s1) ∈ set-dist (execute (send x) s)
    and rest: Some ((), t) ∈ set-dist (execute (mfold2 send xs) s1)
    by (auto elim!: set-dist-bindE)
  have s1:
    s1 = s(PState := concat (PState s) x,
      PTranscript := x # PTranscript s)
  using send-outcome[OF send-x] .
  show ?case
  using Cons.IH[OF rest] unfolding s1 by simp
qed

lemma ntimes-read-outcome:
  assumes Some (ys, t) ∈
    set-dist (execute (ntimes read (length xs)) (s(PTranscript := xs)))
  shows
    ys = xs ∧
    t = s(PState := foldl concat (PState s) xs, PTranscript := [])
  using assms
proof (induction xs arbitrary: s ys t)
  case Nil
  then show ?case by simp
next
  case (Cons x xs)
  have prem1':
    Some (ys, t) ∈
      set-dist (execute
        (read ≫ (λy. ntimes read (length xs) ≫ (λys. return (y # ys))))
        (s(PTranscript := x # xs)))
  using Cons.prem1 by simp
  from prem1' obtain y s1 where
    read-x: Some (y, s1) ∈ set-dist (execute read (s(PTranscript := x # xs)))
    and rest1:
      Some (ys, t) ∈
        set-dist (execute (ntimes read (length xs) ≫ (λys. return (y # ys))) s1)
  by (rule set-dist-bindE) blast
  from rest1 obtain ys' s2 where
    reads: Some (ys', s2) ∈ set-dist (execute (ntimes read (length xs)) s1)
    and ret: Some (ys, t) ∈ set-dist (execute (return (y # ys')) s2)
    by (auto elim!: set-dist-bindE)
  have read-res:

```



```

    y = x ∧ s1 = s(PState := concat (PState s) x, PTranscript := xs)
    using read-cons-outcome[OF read-x] .
  let ?s0 = s(PState := concat (PState s) x)
  have s1: s1 = ?s0(PTranscript := xs)
    using read-res by simp
  have tail:
    ys' = xs ∧
    s2 = ?s0(PState := foldl concat (PState ?s0) xs, PTranscript := [])
    using Cons.IH[OF reads[unfolded s1]] .
  have ys-t: ys = y # ys' ∧ t = s2
    using ret by simp
  show ?case
    using read-res tail ys-t by simp
qed

theorem honest-transcript-replay-mfold2-ntimes:
  assumes init: PTranscript s = []
  and sent: Some ((), sent) ∈ set-dist (execute (mfold2 send xs) s)
  and replay:
    Some (ys, replayed) ∈
    set-dist (execute
      (ntimes read (length xs))
      (sent(PState := PState s, PTranscript := rev (PTranscript sent))))
  shows ys = xs ∧ replayed = sent(PTranscript := [])
proof –
  have sent-eq:
    sent =
    s(
      PState := foldl concat (PState s) xs,
      PTranscript := rev xs
    )
    using mfold2-send-outcome[OF sent] init by simp
  let ?replay-start = sent(PState := PState s)
  have start-eq:
    sent(PState := PState s, PTranscript := rev (PTranscript sent)) =
    ?replay-start(PTranscript := xs)
    using sent-eq by simp
  have replay-out:
    ys = xs ∧
    replayed =
    ?replay-start(
      PState := foldl concat (PState ?replay-start) xs,
      PTranscript := []
    )
    using ntimes-read-outcome[OF replay[unfolded start-eq]] .
  then show ?thesis
    using sent-eq by simp
qed

```

lemma *ntimes-read-prefix-outcome*:

assumes $\text{Some } (ys, t) \in$
 $\text{set-dist } (\text{execute } (\text{ntimes read } (\text{length } xs)) \ (s(P\text{Transcript} := xs @ \text{rest})))$

shows
 $ys = xs \wedge$
 $t = s(P\text{State} := \text{foldl concat } (P\text{State } s) \ xs, \ P\text{Transcript} := \text{rest})$

using *assms*

proof (*induction xs arbitrary: s ys t*)

case *Nil*

then show *?case* **by** *simp*

next

case (*Cons x xs*)

have *prems'*:

$\text{Some } (ys, t) \in$
 $\text{set-dist } (\text{execute}$
 $(\text{read} \gg (\lambda y. \text{ntimes read } (\text{length } xs) \gg (\lambda ys. \text{return } (y \# ys))))$
 $(s(P\text{Transcript} := x \# xs @ \text{rest})))$

using *Cons.prems* **by** *simp*

from *prems'* **obtain** $y \ s1$ **where**
 $\text{read-}x: \text{Some } (y, s1) \in \text{set-dist } (\text{execute read } (s(P\text{Transcript} := x \# xs @ \text{rest})))$

and *rest1*:

$\text{Some } (ys, t) \in$
 $\text{set-dist } (\text{execute } (\text{ntimes read } (\text{length } xs) \gg (\lambda ys. \text{return } (y \# ys))) \ s1)$

by (*rule set-dist-bindE*) *blast*

from *rest1* **obtain** $ys' \ s2$ **where**
 $\text{reads}: \text{Some } (ys', s2) \in \text{set-dist } (\text{execute } (\text{ntimes read } (\text{length } xs)) \ s1)$
and *ret*: $\text{Some } (ys, t) \in \text{set-dist } (\text{execute } (\text{return } (y \# ys')) \ s2)$
by (*auto elim!: set-dist-bindE*)

have *read-res*:

$y = x \wedge s1 = s(P\text{State} := \text{concat } (P\text{State } s) \ x, \ P\text{Transcript} := xs @ \text{rest})$

using *read-cons-outcome[OF read-x]* .

let $?s0 = s(P\text{State} := \text{concat } (P\text{State } s) \ x)$

have $s1: s1 = ?s0(P\text{Transcript} := xs @ \text{rest})$

using *read-res* **by** *simp*

have *tail*:

$ys' = xs \wedge$
 $s2 = ?s0(P\text{State} := \text{foldl concat } (P\text{State } ?s0) \ xs, \ P\text{Transcript} := \text{rest})$

using *Cons.IH[OF reads[unfolded s1]]* .

have $ys\text{-}t: ys = y \# ys' \wedge t = s2$

using *ret* **by** *simp*

show *?case*

using *read-res tail ys-t* **by** *simp*

qed

lemma *ntimes-read-no-failure*:

assumes $n \leq \text{length } (P\text{Transcript } s)$

shows $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (\text{ntimes read } n) \ s))$

using *assms*

proof (*induction n arbitrary: s*)

```

case 0
then show ?case by simp
next
case (Suc n)
have nonempty: PTranscript s  $\neq$  []
  using Suc.premis by (cases PTranscript s) auto
have nf-read: None  $\notin$  dom (dist (execute read s))
  by (rule read-no-failure[OF nonempty])
have nf-cont:
 $\bigwedge x t. \text{Some } (x, t) \in \text{set-dist } (\text{execute read } s) \implies$ 
  None  $\notin$  dom
  (dist (execute
    (ntimes read n  $\gg$  ( $\lambda xs. \text{return } (x \# xs)$ )) t))
proof -
  fix x t
  assume out: Some (x, t)  $\in$  set-dist (execute read s)
  obtain y ys where tr: PTranscript s = y # ys
  using nonempty by (cases PTranscript s) auto
  have t-eq:
    t = s[PState := concat (PState s) y, PTranscript := ys]
  using read-nonempty-outcome[OF out tr] by simp
  have n  $\leq$  length ys
  using Suc.premis tr by simp
  then have nf-tail: None  $\notin$  dom (dist (execute (ntimes read n) t))
  using Suc.IH[of t] unfolding t-eq by simp
  show None  $\notin$  dom
    (dist (execute
      (ntimes read n  $\gg$  ( $\lambda xs. \text{return } (x \# xs)$ )) t))
  by (rule no-failure-bindI[OF nf-tail]) simp
qed
show ?case
  unfolding ntimes.simps
  by (rule no-failure-bindI[OF nf-read nf-cont])
qed

lemma ntimes-read-hash-extends:
  assumes Some (xs, t)  $\in$  set-dist (execute (ntimes read n) s)
  shows s  $\leq$  t
  using assms
proof (induction n arbitrary: xs s t)
  case 0
  then show ?case by (simp add: hash-ext-refl)
next
  case (Suc n)
  from Suc.premis obtain x ys u where
    read: Some (x, u)  $\in$  set-dist (execute read s)
    and reads: Some (ys, t)  $\in$  set-dist (execute (ntimes read n) u)
    by (auto elim!: set-dist-bindE)
  have s  $\leq$  u

```

```

    by (rule read-hash-extends[OF read])
  moreover have  $u \leq t$ 
    by (rule Suc.IH[OF reads])
  ultimately show ?case
    by (rule hash-ext-trans)
qed

lemma check-authentication-path-no-failure:
  None  $\notin$  dom (dist (execute (check-authentication-path len i v path) s))
  unfolding check-authentication-path-def protocol-check-authentication-path-def
proof (induction path arbitrary: len i v s)
  case Nil
  then show ?case
    by (simp add: protocol-hash-no-failure)
next
  case (Cons a path)
  show ?case
  proof (cases  $i < \text{len div } 2$ )
    case True
    then show ?thesis
      by (simp add: no-failure-bindI protocol-hash-no-failure Cons.IH)
  next
    case False
    then show ?thesis
      by (simp add: no-failure-bindI protocol-hash-no-failure Cons.IH)
  qed
qed

lemma check-authentication-path-hash-extends:
  assumes outcome:
    Some (h, t)  $\in$ 
      set-dist (execute (check-authentication-path len i v path) s)
  shows  $s \leq t$ 
  using outcome
  unfolding check-authentication-path-def protocol-check-authentication-path-def
proof (induction path arbitrary: len i v s h t)
  case Nil
  then have hash-step:
    Some (h, t)  $\in$  set-dist (execute (hash (MerkleLeaf v)) s)
    by simp
  show ?case
    by (rule hash-outcome(1)[OF hash-step])
next
  case (Cons a path)
  show ?case
  proof (cases  $i < \text{len div } 2$ )
    case True
    from Cons.premis obtain x u where
      rec:

```

```

    Some (x, u) ∈
      set-dist (execute
        (protocol-merkle.check-authentication-path
          (len div 2) i (MerkleLeaf v) path) s)
  and hash:
    Some (h, t) ∈ set-dist (execute (hash (MerkleNode x a)) u)
    using True by (auto elim!: set-dist-bindE)
  have s ≤ u
    by (rule Cons.IH[OF rec])
  moreover have u ≤ t
    using hash-outcome(1)[OF hash] .
  ultimately show ?thesis
    by (rule hash-ext-trans)
next
case False
from Cons.premis obtain x u where
  rec:
    Some (x, u) ∈
      set-dist (execute
        (protocol-merkle.check-authentication-path
          (len div 2) (i - len div 2) (MerkleLeaf v) path) s)
  and hash:
    Some (h, t) ∈ set-dist (execute (hash (MerkleNode a x)) u)
    using False by (auto elim!: set-dist-bindE)
  have s ≤ u
    by (rule Cons.IH[OF rec])
  moreover have u ≤ t
    using hash-outcome(1)[OF hash] .
  ultimately show ?thesis
    by (rule hash-ext-trans)
qed
qed

```

lemma *check-authentication-path-preserves-channel:*
assumes Some (h, t) ∈
 set-dist (execute (check-authentication-path len i v path) s)
shows PState t = PState s
 and PTranscript t = PTranscript s
 and PTraceFriCounter t = PTraceFriCounter s
 and PCompositionFriCounter t = PCompositionFriCounter s
 and PAlphaCounter t = PAlphaCounter s
 and PQueryCounter t = PQueryCounter s
proof –
 have both:
 PState t = PState s ∧
 PTranscript t = PTranscript s ∧
 PTraceFriCounter t = PTraceFriCounter s ∧
 PCompositionFriCounter t = PCompositionFriCounter s ∧
 PAlphaCounter t = PAlphaCounter s ∧
 PQueryCounter t = PQueryCounter s

```

    PQueryCounter t = PQueryCounter s
  if outcome:
    Some (h, t) ∈
      set-dist (execute (check-authentication-path len i v path) s)
  for len i v s h t
  using outcome
  unfolding check-authentication-path-def protocol-check-authentication-path-def
proof (induction path arbitrary: len i v s h t)
  case Nil
  have hash-step:
    Some (h, t) ∈ set-dist (execute (hash (MerkleLeaf v)) s)
    using Nil.prem by simp
  show ?case
  using hash-channel-preserves(1)[OF hash-step]
    hash-channel-preserves(2)[OF hash-step]
    hash-channel-preserves(3)[OF hash-step]
    hash-channel-preserves(4)[OF hash-step]
    hash-channel-preserves(5)[OF hash-step]
    hash-channel-preserves(6)[OF hash-step]
  by simp
next
  case (Cons a path)
  show ?case
  proof (cases i < len div 2)
    case True
    from Cons.prem obtain x u where
      rec:
        Some (x, u) ∈
          set-dist (execute
            (protocol-merkle.check-authentication-path
              (len div 2) i (MerkleLeaf v) path) s)
    and hash:
      Some (h, t) ∈ set-dist (execute (hash (MerkleNode x a)) u)
    using True by (auto elim!: set-dist-bindE)
    have rec-pres:
      PState u = PState s ∧
      PTranscript u = PTranscript s ∧
      PTraceFriCounter u = PTraceFriCounter s ∧
      PCompositionFriCounter u = PCompositionFriCounter s ∧
      PAlphaCounter u = PAlphaCounter s ∧
      PQueryCounter u = PQueryCounter s
    using Cons.IH[OF rec] .
    have hash-pres:
      PState t = PState u ∧
      PTranscript t = PTranscript u ∧
      PTraceFriCounter t = PTraceFriCounter u ∧
      PCompositionFriCounter t = PCompositionFriCounter u ∧
      PAlphaCounter t = PAlphaCounter u ∧
      PQueryCounter t = PQueryCounter u
  
```

```

using hash-channel-preserves(1)[OF hash]
  hash-channel-preserves(2)[OF hash]
  hash-channel-preserves(3)[OF hash]
  hash-channel-preserves(4)[OF hash]
  hash-channel-preserves(5)[OF hash]
  hash-channel-preserves(6)[OF hash] by simp
show ?thesis
  using rec-pres hash-pres by simp
next
case False
from Cons.premis obtain x u where
  rec:
    Some (x, u) ∈
      set-dist (execute
        (protocol-merkle.check-authentication-path
          (len div 2) (i - len div 2) (MerkleLeaf v) path) s)
and hash:
    Some (h, t) ∈ set-dist (execute (hash (MerkleNode a x)) u)
  using False by (auto elim!: set-dist-bindE)
have rec-pres:
  PState u = PState s ∧
  PTranscript u = PTranscript s ∧
  PTraceFriCounter u = PTraceFriCounter s ∧
  PCompositionFriCounter u = PCompositionFriCounter s ∧
  PAlphaCounter u = PAlphaCounter s ∧
  PQueryCounter u = PQueryCounter s
  using Cons.IH[OF rec] .
have hash-pres:
  PState t = PState u ∧
  PTranscript t = PTranscript u ∧
  PTraceFriCounter t = PTraceFriCounter u ∧
  PCompositionFriCounter t = PCompositionFriCounter u ∧
  PAlphaCounter t = PAlphaCounter u ∧
  PQueryCounter t = PQueryCounter u
  using hash-channel-preserves(1)[OF hash]
    hash-channel-preserves(2)[OF hash]
    hash-channel-preserves(3)[OF hash]
    hash-channel-preserves(4)[OF hash]
    hash-channel-preserves(5)[OF hash]
    hash-channel-preserves(6)[OF hash] by simp
show ?thesis
  using rec-pres hash-pres by simp
qed
qed
show PState t = PState s
  using both[OF assms] by simp
show PTranscript t = PTranscript s
  using both[OF assms] by simp
show PTraceFriCounter t = PTraceFriCounter s

```

```

    using both[OF assms] by simp
  show PCompositionFriCounter t = PCompositionFriCounter s
    using both[OF assms] by simp
  show PAlphaCounter t = PAlphaCounter s
    using both[OF assms] by simp
  show PQueryCounter t = PQueryCounter s
    using both[OF assms] by simp
qed

lemma check-authentication-path-preserves-non-merkle-lookup:
  assumes outcome: Some (h, t) ∈
    set-dist (execute (check-authentication-path len i v path) s)
    and not-leaf:  $\bigwedge x. y \neq \text{MerkleLeaf } x$ 
    and not-node:  $\bigwedge x z. y \neq \text{MerkleNode } x z$ 
  shows fmlookup (HashMap t) y = fmlookup (HashMap s) y
  using outcome
  unfolding check-authentication-path-def protocol-check-authentication-path-def
proof (induction path arbitrary: len i v s h t)
  case Nil
  have hash-step:
    Some (h, t) ∈ set-dist (execute (hash (MerkleLeaf v)) s)
  using Nil.premis by simp
  show ?case
  by (rule protocol-hash-preserves-other-lookup[OF hash-step])
    (rule not-leaf)
next
  case (Cons a path)
  show ?case
proof (cases i < len div 2)
  case True
  from Cons.premis obtain x u where
    rec:
      Some (x, u) ∈
        set-dist (execute
          (protocol-merkle.check-authentication-path
            (len div 2) i (MerkleLeaf v) path) s)
    and hash-step:
      Some (h, t) ∈ set-dist (execute (hash (MerkleNode x a)) u)
  using True by (auto elim!: set-dist-bindE)
  have rec-pres:
    fmlookup (HashMap u) y = fmlookup (HashMap s) y
  by (rule Cons.IH[OF rec])
  have hash-pres:
    fmlookup (HashMap t) y = fmlookup (HashMap u) y
  by (rule protocol-hash-preserves-other-lookup[OF hash-step])
    (rule not-node)
  show ?thesis
  using hash-pres rec-pres by simp
next

```



```

case False
from Cons.prems obtain x u where
  rec:
    Some (x, u) ∈
      set-dist (execute
        (protocol-merkle.check-authentication-path
          (len div 2) (i - len div 2) (MerkleLeaf v) path) s)
    and hash-step:
      Some (h, t) ∈ set-dist (execute (hash (MerkleNode a x)) u)
    using False by (auto elim!: set-dist-bindE)
have rec-pres:
  fmlookup (HashMap u) y = fmlookup (HashMap s) y
  by (rule Cons.IH[OF rec])
have hash-pres:
  fmlookup (HashMap t) y = fmlookup (HashMap u) y
  by (rule protocol-hash-preserves-other-lookup[OF hash-step])
    (rule not-node)
show ?thesis
  using hash-pres rec-pres by simp
qed
qed

```

lemma *check-authentication-path-preserves-challenge-lookups*:

assumes *outcome*: *Some* (*h, t*) ∈
set-dist (*execute* (*check-authentication-path len i v path*) *s*)

shows

fmlookup (*HashMap t*) (*AlphaChallenge j x*) =
fmlookup (*HashMap s*) (*AlphaChallenge j x*)
fmlookup (*HashMap t*) (*QueryIndexChallenge j x*) =
fmlookup (*HashMap s*) (*QueryIndexChallenge j x*)
fmlookup (*HashMap t*) (*TraceFriChallenge j x*) =
fmlookup (*HashMap s*) (*TraceFriChallenge j x*)
fmlookup (*HashMap t*) (*CompositionFriChallenge j x*) =
fmlookup (*HashMap s*) (*CompositionFriChallenge j x*)

proof –

show *fmlookup* (*HashMap t*) (*AlphaChallenge j x*) =
fmlookup (*HashMap s*) (*AlphaChallenge j x*)
by (*rule check-authentication-path-preserves-non-merkle-lookup*[
OF outcome]) *simp-all*
show *fmlookup* (*HashMap t*) (*QueryIndexChallenge j x*) =
fmlookup (*HashMap s*) (*QueryIndexChallenge j x*)
by (*rule check-authentication-path-preserves-non-merkle-lookup*[
OF outcome]) *simp-all*
show *fmlookup* (*HashMap t*) (*TraceFriChallenge j x*) =
fmlookup (*HashMap s*) (*TraceFriChallenge j x*)
by (*rule check-authentication-path-preserves-non-merkle-lookup*[
OF outcome]) *simp-all*
show *fmlookup* (*HashMap t*) (*CompositionFriChallenge j x*) =
fmlookup (*HashMap s*) (*CompositionFriChallenge j x*)

```

    by (rule check-authentication-path-preserves-non-merkle-lookup[
      OF outcome]) simp-all
qed

lemma honest-query-decommitment-no-failure:
  assumes created: created-tree xs tree s0
    and len-pow: length xs = 2 ^ n
    and idx: i < length xs
    and leaf: v = xs ! i
    and ext: s0 ≤ s
    and path-len: length (get-authentication-path (length xs) i tree) = floor-log (length xs)
    and root: fr = value tree
  shows
    None ∉ dom (dist (execute
      (do {
        gh ← read;
        gh-path ← ntimes read (floor-log (length xs));
        ap ← check-authentication-path (length xs) i gh gh-path;
        assert (ap = fr);
        return gh
      }) (s[PTranscript := v # get-authentication-path (length xs) i tree @ rest])))
  proof -
    let ?path = get-authentication-path (length xs) i tree
    let ?s = s[PTranscript := v # ?path @ rest]
    let ?prefix =
      do {
        gh ← read;
        gh-path ← ntimes read (floor-log (length xs));
        ap ← check-authentication-path (length xs) i gh gh-path;
        return (gh, gh-path, ap)
      }
    have nf-read: None ∉ dom (dist (execute read ?s))
    by (rule read-no-failure) simp
    have nf-after-read:
      ∧ gh s1. Some (gh, s1) ∈ set-dist (execute read ?s) ⇒
        None ∉ dom
          (dist (execute
            (ntimes read (floor-log (length xs))) ⋈
              (λ gh-path. check-authentication-path (length xs) i gh gh-path ⋈
                (λ ap. return (gh, gh-path, ap)))) s1))
    proof -
      fix gh s1
      assume read-gh: Some (gh, s1) ∈ set-dist (execute read ?s)
      have read-gh-res:
        s1 = s[PState := concat (PState s) v, PTranscript := ?path @ rest]
      using read-cons-outcome[OF read-gh] by simp
      have enough-path: floor-log (length xs) ≤ length (PTranscript s1)
      using path-len unfolding read-gh-res by simp
      have nf-reads:

```

$None \notin dom (dist (execute (ntimes read (floor-log (length xs))) s1))$
by (rule *ntimes-read-no-failure*[*OF enough-path*])
have *nf-after-reads*:
 $\bigwedge qh-path\ s2. Some (qh-path, s2) \in$
 $set-dist (execute (ntimes read (floor-log (length xs))) s1) \implies$
 $None \notin dom$
 $(dist (execute$
 $(check-authentication-path (length xs) i\ qh\ qh-path \gg=$
 $(\lambda ap. return (qh, qh-path, ap))) s2))$
proof –
fix *qh-path s2*
assume $Some (qh-path, s2) \in$
 $set-dist (execute (ntimes read (floor-log (length xs))) s1)$
have *nf-check*:
 $None \notin dom$
 $(dist (execute (check-authentication-path (length xs) i\ qh\ qh-path) s2))$
by (rule *check-authentication-path-no-failure*)
show $None \notin dom$
 $(dist (execute$
 $(check-authentication-path (length xs) i\ qh\ qh-path \gg=$
 $(\lambda ap. return (qh, qh-path, ap))) s2))$
by (rule *no-failure-bindI*[*OF nf-check*]) *simp*
qed
show $None \notin dom$
 $(dist (execute$
 $(ntimes read (floor-log (length xs)) \gg=$
 $(\lambda qh-path. check-authentication-path (length xs) i\ qh\ qh-path \gg=$
 $(\lambda ap. return (qh, qh-path, ap)))) s1))$
by (rule *no-failure-bindI*[*OF nf-reads nf-after-reads*])
qed
have *prefix-nf*:
 $None \notin dom (dist (execute ?prefix ?s))$
by (rule *no-failure-bindI*[*OF nf-read nf-after-read*])
have *prefix-out*:
 $\bigwedge res\ t. Some (res, t) \in set-dist (execute ?prefix ?s) \implies$
 $case\ res\ of\ (qh, qh-path, ap) \Rightarrow ap = fr$
proof –
fix *res t*
assume $out: Some (res, t) \in set-dist (execute ?prefix ?s)$
obtain *qh qh-path ap* **where** *res-eq*: $res = (qh, qh-path, ap)$
by (*cases res*) *auto*
from *out*[*unfolded res-eq*] **obtain** *s1 s2* **where**
 $read-qh: Some (qh, s1) \in set-dist (execute read ?s)$
and *reads*:
 $Some (qh-path, s2) \in$
 $set-dist (execute (ntimes read (floor-log (length xs))) s1)$
and *check*:
 $Some (ap, t) \in$
 $set-dist (execute (check-authentication-path (length xs) i\ qh\ qh-path) s2)$

```

  by (auto elim!: set-dist-bindE)
have qh-eq: qh = v
  using read-cons-outcome[OF read-qh] by simp
have s1-eq:
  s1 = s(PState := concat (PState s) v, PTranscript := ?path @ rest)
  using read-cons-outcome[OF read-qh] by simp
have reads-out:
  qh-path = ?path
  using ntimes-read-prefix-outcome[OF reads[unfolded path-len[symmetric] s1-eq]]
  by simp
have s-start: s ≤ ?s
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have start-s1: ?s ≤ s1
  by (rule read-hash-extends[OF read-qh])
have s1-s2: s1 ≤ s2
  by (rule ntimes-read-hash-extends[OF reads])
have s0-s2: s0 ≤ s2
  using ext s-start start-s1 s1-s2 by (meson hash-ext-trans)
have ap = value tree
  using check-created-tree-outcome
  [OF created len-pow idx leaf s0-s2 check[unfolded qh-eq reads-out]]
  by simp
then show case res of (qh, qh-path, ap) ⇒ ap = fr
  unfolding res-eq root by simp
qed
have full:
  do {
    qh ← read;
    qh-path ← ntimes read (floor-log (length xs));
    ap ← check-authentication-path (length xs) i qh qh-path;
    assert (ap = fr);
    return qh
  } =
  (?prefix ≫ (λ(qh, qh-path, ap). assert (ap = fr) ≫ (λ-. return qh)))
  by (simp add: sm-bind-assoc split: prod.splits)
show ?thesis
  unfolding full
proof (rule no-failure-bindI[OF prefix-nf])
  fix res t
  assume out: Some (res, t) ∈ set-dist (execute ?prefix ?s)
  obtain qh qh-path ap where res-eq: res = (qh, qh-path, ap)
    by (cases res) auto
  have ap-eq: ap = fr
    using prefix-out[OF out] unfolding res-eq by simp
  show None ∉ dom
    (dist (execute
      ((case res of (qh, qh-path, ap) ⇒
        assert (ap = fr) ≫ (λ-. return qh))) t))
  unfolding res-eq ap-eq assert-def by simp

```

qed
qed

end

end

theory *Channel-Transcript*
 imports *Channel-Core*
begin

locale *channel* = *merkle-tree* +
 constrains *concat* :: '*a*::*finite* $\Rightarrow$ '*a* $\Rightarrow$ '*a*
begin

definition *send*:: '*a* $\Rightarrow$ ('*a*, *unit*, '*b*) *c-monad*
where
 send *x* $\equiv$
 modify ($\lambda s. s \ll \text{State} := \text{concat} (\text{State } s) \ x, \text{Transcript} := x \ \# (\text{Transcript } s) \gg$)

definition *send2*:: '*a* $\Rightarrow$ ('*a*, *unit*, '*b*) *c-monad*
where
 send2 *x* $\equiv$
 modify ($\lambda s. s \ll \text{Transcript} := x \ \# (\text{Transcript } s) \gg$)

The idea is that read recovers the state of write

definition *read* :: ('*a*, '*a*, '*b*) *c-monad*
where
 read $\equiv$
 do {
 s $\leftarrow$ *get*;
 assert (*Transcript* *s* $\neq$ []);
 modify ($\lambda s. s \ll \text{State} := \text{concat} (\text{State } s) (\text{hd } (\text{Transcript } s)), \text{Transcript} := \text{tl } (\text{Transcript } s) \gg$);
 return (*hd* (*Transcript* *s*))
 }

lemma *send-outcome*:
 assumes *Some* (*()*, *t*) $\in$ *set-dist* (*execute* (*send* *x*) *s*)
 shows *t* = *s* $\ll \text{State} := \text{concat} (\text{State } s) \ x, \text{Transcript} := x \ \# \text{Transcript } s \gg$
 using *assms*
 unfolding *send-def modify-def*
 by (*auto elim!*: *set-dist-bindE*)

lemma *read-cons-outcome*:
 assumes *Some* (*y*, *t*) $\in$ *set-dist* (*execute* *read* (*s* $\ll \text{Transcript} := x \ \# \text{xs} \gg$))
 shows
 y = *x* $\wedge$
 t = *s* $\ll \text{State} := \text{concat} (\text{State } s) \ x, \text{Transcript} := \text{xs} \gg$

```

using assms
unfolding read-def modify-def assert-def
by (auto elim! set-dist-bindE)

lemma read-nonempty-outcome:
  assumes out: Some (y, t) ∈ set-dist (execute read s)
  and tr: Transcript s = x # xs
  shows
     $y = x \wedge$ 
     $t = s \langle \text{State} := \text{concat} (\text{State } s) \ x, \text{Transcript} := xs \rangle$ 
proof –
  have s-eq: s ⟨ Transcript := x # xs ⟩ = s
  using tr by (cases s simp)
  have out': Some (y, t) ∈
    set-dist (execute read (s ⟨ Transcript := x # xs ⟩))
  using out unfolding s-eq .
  show ?thesis
  using read-cons-outcome[OF out'] tr by simp
qed

lemma read-no-failure:
  assumes Transcript s ≠ []
  shows None ∉ dom (dist (execute read s))
  using assms
  unfolding read-def assert-def
  by (intro no-failure-bindI simp-all)

lemma hash-no-failure:
  None ∉ dom (dist (execute (hash x) s))
  unfolding hash-def apply-hash-def modify-HashMap-def
  by (intro no-failure-bindI simp-all)

lemma receive-random-field-element-no-failure:
  None ∉ dom (dist (execute receive-random-field-element s))
  unfolding receive-random-field-element-def
  by (intro no-failure-bindI (simp-all add: hash-no-failure))

lemma receive-random-field-element-extends:
  assumes Some (r, t) ∈ set-dist (execute receive-random-field-element s)
  shows  $s \leq t$ 
proof –
  from assms obtain s0 where
    get-s: Some (s0, s0) ∈ set-dist (execute get s)
  and rest:
    Some (r, t) ∈
    set-dist (execute (hash (State s0) >>= return) s0)
  unfolding receive-random-field-element-def
  by (auto elim! set-dist-bindE)
  have s0: s0 = s

```

```

    using get-s by simp
  have hash-out:
    Some (r, t) ∈ set-dist (execute (hash (State s)) s)
    using rest unfolding s0 by simp
  show ?thesis
    using hash-outcome(1)[OF hash-out] .
qed

lemma hash-channel-preserves:
  assumes Some (h, t) ∈ set-dist (execute (hash x) s)
  shows State t = State s
    and Transcript t = Transcript s
proof -
  from assms obtain a s1 where
    a: Some (a, s1) ∈ set-dist (execute (apply-hash x) s)
    and rest: Some (h, t) ∈
      set-dist (execute (modify-HashMap x a >>= (λ-. return a)) s1)
  proof -
    have Some (h, t) ∈
      set-dist (execute
        (apply-hash x >>= (λa. modify-HashMap x a >>= (λ-. return a))) s)
      using assms unfolding hash-def .
    then show ?thesis
      by (rule set-dist-bindE) (rule that)
  qed
  have a-img: Some (a, s1) ∈ (λa. Some (a, s)) ‘ set-dist (hash-dist x s)
    using a
    unfolding apply-hash-def lift.rep-eq lift-dist-def
    by (simp add: set-dist-dist-map)
  then have s1: s1 = s
    by auto
  from rest obtain u s2 where
    mod: Some (u, s2) ∈ set-dist (execute (modify-HashMap x a) s1)
    and ret: Some (h, t) ∈ set-dist (execute (return a) s2)
    by (elim set-dist-bindE)
  from mod have s2: s2 = s1 (HashMap := fmapd x a (HashMap s1))
    unfolding modify-HashMap-def modify-def
    by (auto elim!: set-dist-bindE)
  from ret have t: t = s2
    by auto
  show State t = State s
    unfolding t s2 s1 by simp
  show Transcript t = Transcript s
    unfolding t s2 s1 by simp
qed

lemma receive-random-field-element-known-outcome:
  assumes lookup: fmllookup (HashMap s) (State s) = Some r
  and outcome: Some (r', t) ∈ set-dist (execute receive-random-field-element s)

```

shows
 $r' = r \wedge \text{State } t = \text{State } s \wedge \text{Transcript } t = \text{Transcript } s \wedge s \leq t$
proof –
from *outcome* **obtain** $s0$ **where**
 $\text{get-}s: \text{Some } (s0, s0) \in \text{set-dist } (\text{execute } \text{get } s)$
and *rest*:
 $\text{Some } (r', t) \in$
 $\text{set-dist } (\text{execute } (\text{hash } (\text{State } s0)) \gg \text{return}) s0)$
unfolding *receive-random-field-element-def*
by (*auto elim!*: *set-dist-bindE*)
have $s0: s0 = s$
using *get-s* **by** *simp*
have *hash-out*:
 $\text{Some } (r', t) \in \text{set-dist } (\text{execute } (\text{hash } (\text{State } s)) s)$
using *rest* **unfolding** $s0$ **by** *simp*
have *ext*: $s \leq t$
using *hash-outcome(1)*[*OF hash-out*] .
have *lookup-r'*: $\text{fmlookup } (\text{HashMap } t) (\text{State } s) = \text{Some } r'$
using *hash-outcome(2)*[*OF hash-out*] .
have *lookup-r*: $\text{fmlookup } (\text{HashMap } t) (\text{State } s) = \text{Some } r$
using *hash-extension-lookup*[*OF lookup ext*] .
moreover **have** $\text{State } t = \text{State } s \wedge \text{Transcript } t = \text{Transcript } s$
using *hash-channel-preserves*[*OF hash-out*] **by** *simp*
ultimately show *?thesis*
using *ext lookup-r'* **by** *auto*
qed

lemma *random-then-read-cons-outcome*:
assumes *lookup*: $\text{fmlookup } (\text{HashMap } s) (\text{State } s) = \text{Some } x$
and *outcome*:
 $\text{Some } ((r, y), t) \in$
 $\text{set-dist } (\text{execute}$
 $\text{do } \{$
 $r \leftarrow \text{receive-random-field-element};$
 $y \leftarrow \text{read};$
 $\text{return } (r, y)$
 $\}) (s \ll \text{Transcript } := x \# xs))$
shows
 $r = x \wedge$
 $y = x \wedge$
 $\text{State } t = \text{concat } (\text{State } s) x \wedge$
 $\text{Transcript } t = xs$

proof –
let $?s = s \ll \text{Transcript } := x \# xs$
have *outcome'*:
 $\text{Some } ((r, y), t) \in$
 $\text{set-dist } (\text{execute}$
 $(\text{receive-random-field-element} \gg$
 $(\lambda r. \text{read} \gg (\lambda y. \text{return } (r, y)))) ?s)$


```

using outcome by simp
from outcome' obtain r-state where
  rand:
    Some (r, r-state) ∈ set-dist (execute receive-random-field-element ?s)
and read-y:
    Some (y, t) ∈ set-dist (execute read r-state)
by (auto elim!: set-dist-bindE)
have lookup': fmllookup (HashMap ?s) (State ?s) = Some x
  using lookup by simp
have rand-res:
  r = x ∧ State r-state = State s ∧ Transcript r-state = x # xs
  using receive-random-field-element-known-outcome[OF lookup' rand] by simp
have r-state-update: r-state(Transcript := x # xs) = r-state
  using rand-res by simp
have read-y':
  Some (y, t) ∈ set-dist (execute read (r-state(Transcript := x # xs)))
  using read-y unfolding r-state-update .
have read-res:
  y = x ∧ State t = concat (State s) x ∧ Transcript t = xs
  using read-cons-outcome[OF read-y'] rand-res by simp
show ?thesis
  using rand-res read-res by simp
qed

lemma random-then-read-assert-no-failure:
assumes lookup: fmllookup (HashMap s) (State s) = Some x
shows
  None ∉ dom (dist (execute
    (do {
      r ← receive-random-field-element;
      y ← read;
      assert (r = y);
      return y
    }) (s(Transcript := x # xs))))
proof −
let ?s = s(Transcript := x # xs)
let ?prefix =
  do {
    r ← receive-random-field-element;
    y ← read;
    return (r, y)
  }
have prefix-no-failure:
  None ∉ dom (dist (execute ?prefix ?s))
proof (rule no-failure-bindI)
show None ∉ dom (dist (execute receive-random-field-element ?s))
  by (rule receive-random-field-element-no-failure)
fix r t
assume rand: Some (r, t) ∈ set-dist (execute receive-random-field-element ?s)

```

```

have lookup-s: fmlookup (HashMap ?s) (State ?s) = Some x
  using lookup by simp
have tr-t: Transcript t = x # xs
  using receive-random-field-element-known-outcome[OF lookup-s rand] by simp
show None  $\notin$  dom (dist (execute (read  $\gg$  ( $\lambda y$ . return (r, y))) t))
proof (rule no-failure-bindI)
  show None  $\notin$  dom (dist (execute read t))
    by (rule read-no-failure) (simp add: tr-t)
  fix y u
  assume Some (y, u)  $\in$  set-dist (execute read t)
  show None  $\notin$  dom (dist (execute (return (r, y)) u))
    by simp
qed
qed
have full:
  do {
    r  $\leftarrow$  receive-random-field-element;
    y  $\leftarrow$  read;
    assert (r = y);
    return y
  } =
  (?prefix  $\gg$  ( $\lambda(r, y)$ . assert (r = y)  $\gg$  ( $\lambda-$ . return y)))
  by (simp add: sm-bind-assoc split: prod.splits)
show ?thesis
  unfolding full
proof (rule no-failure-bindI[OF prefix-no-failure])
  fix ry t
  assume prefix:
    Some (ry, t)  $\in$  set-dist (execute ?prefix ?s)
  obtain r y where ry: ry = (r, y)
    by (cases ry)
  have replay: r = y
    using random-then-read-cons-outcome[OF lookup prefix[unfolded ry]] by simp
  show None  $\notin$  dom
    (dist (execute ((case ry of (r, y)  $\Rightarrow$  assert (r = y)  $\gg$  ( $\lambda-$ . return y))) t))
    unfolding ry replay assert-def by simp
qed
qed

lemma read-then-random-cons-outcome:
assumes lookup:
  fmlookup (HashMap s) (concat (State s) x) = Some r
and outcome:
  Some ((y, r'), t)  $\in$ 
  set-dist (execute
    (do {
      y  $\leftarrow$  read;
      r  $\leftarrow$  receive-random-field-element;
      return (y, r)
    }

```

```

    }) (s(| Transcript := x # xs |))
  shows
    y = x ∧
    r' = r ∧
    State t = concat (State s) x ∧
    Transcript t = xs
  proof -
    let ?s = s(| Transcript := x # xs |)
    have outcome':
      Some ((y, r'), t) ∈
        set-dist (execute
          (read ≫=
            (λy. receive-random-field-element ≫= (λr. return (y, r)))) ?s)
    using outcome' by simp
    from outcome' obtain read-state where
      read-y: Some (y, read-state) ∈ set-dist (execute read ?s)
    and rand:
      Some (r', t) ∈ set-dist (execute receive-random-field-element read-state)
    by (auto elim!: set-dist-bindE)
    have read-res:
      y = x ∧
      read-state = s(| State := concat (State s) x, Transcript := xs |)
    using read-cons-outcome[OF read-y] by simp
    have rand-res:
      r' = r ∧
      State t = concat (State s) x ∧
      Transcript t = xs
    using receive-random-field-element-known-outcome[OF - rand] lookup read-res by simp
    show ?thesis
      using read-res rand-res by simp
  qed

  lemma mfold2-send-outcome:
    assumes Some ((), t) ∈ set-dist (execute (mfold2 send xs) s)
    shows
      t =
        s(|
          State := foldl concat (State s) xs,
          Transcript := rev xs @ Transcript s
        |)
    using assms
  proof (induction xs arbitrary: s t)
    case Nil
    then show ?case by simp
  next
    case (Cons x xs)
    from Cons.premis obtain s1 where
      send-x: Some ((), s1) ∈ set-dist (execute (send x) s)
    and rest: Some ((), t) ∈ set-dist (execute (mfold2 send xs) s1)

```

```

    by (auto elim!: set-dist-bindE)
  have s1:
    s1 = s⟦State := concat (State s) x, Transcript := x # Transcript s⟧
    using send-outcome[OF send-x] .
  show ?case
    using Cons.IH[OF rest] unfolding s1 by simp
qed

lemma ntimes-read-outcome:
  assumes Some (ys, t) ∈ set-dist (execute (ntimes read (length xs)) (s⟦Transcript := xs⟧))
  shows
    ys = xs ∧
    t = s⟦State := foldl concat (State s) xs, Transcript := []⟧
  using assms
proof (induction xs arbitrary: s ys t)
  case Nil
  then show ?case by simp
next
  case (Cons x xs)
  have prems':
    Some (ys, t) ∈
    set-dist (execute
      (read ≫ (λy. ntimes read (length xs) ≫ (λys. return (y # ys))))
      (s⟦Transcript := x # xs⟧))
  using Cons.prems by simp
  from prems' obtain y s1 where
    read-x: Some (y, s1) ∈ set-dist (execute read (s⟦Transcript := x # xs⟧))
  and rest1:
    Some (ys, t) ∈
    set-dist (execute (ntimes read (length xs) ≫ (λys. return (y # ys))) s1)
  by (rule set-dist-bindE) blast
  from rest1 obtain ys' s2 where
    reads: Some (ys', s2) ∈ set-dist (execute (ntimes read (length xs)) s1)
  and ret: Some (ys, t) ∈ set-dist (execute (return (y # ys')) s2)
  by (auto elim!: set-dist-bindE)
  have read-res:
    y = x ∧ s1 = s⟦State := concat (State s) x, Transcript := xs⟧
    using read-cons-outcome[OF read-x] .
  let ?s0 = s⟦State := concat (State s) x⟧
  have s1: s1 = ?s0⟦Transcript := xs⟧
    using read-res by simp
  have tail:
    ys' = xs ∧
    s2 = ?s0⟦State := foldl concat (State ?s0) xs, Transcript := []⟧
    using Cons.IH[OF reads[unfolded s1]] .
  have ys-t: ys = y # ys' ∧ t = s2
    using ret by simp
  show ?case
    using read-res tail ys-t by simp

```

qed

lemma *ntimes-read-prefix-outcome*:

assumes *Some* (*ys*, *t*) ∈

set-dist (*execute* (*ntimes read* (*length xs*)) (*s*⌊*Transcript* := *xs* @ *rest*⌋))

shows

ys = *xs* ∧

t = *s*⌊*State* := *foldl concat* (*State s*) *xs*, *Transcript* := *rest*⌋

using *assms*

proof (*induction xs arbitrary: s ys t*)

case *Nil*

then show *?case* **by** *simp*

next

case (*Cons x xs*)

have *prems'*:

Some (*ys*, *t*) ∈

set-dist (*execute*

(*read* $\gg$ ($\lambda y. \text{ntimes read } (\text{length } xs) \gg (\lambda ys. \text{return } (y \# ys))$))

(*s*⌊*Transcript* := *x* # *xs* @ *rest*⌋))

using *Cons.prems* **by** *simp*

from *prems'* **obtain** *y s1* **where**

read-x: *Some* (*y*, *s1*) ∈ *set-dist* (*execute read* (*s*⌊*Transcript* := *x* # *xs* @ *rest*⌋))

and *rest1*:

Some (*ys*, *t*) ∈

set-dist (*execute* (*ntimes read* (*length xs*) $\gg$ ($\lambda ys. \text{return } (y \# ys)$)) *s1*)

by (*rule set-dist-bindE*) *blast*

from *rest1* **obtain** *ys' s2* **where**

reads: *Some* (*ys'*, *s2*) ∈ *set-dist* (*execute* (*ntimes read* (*length xs*)) *s1*)

and *ret*: *Some* (*ys*, *t*) ∈ *set-dist* (*execute* (*return* (*y* # *ys'*)) *s2*)

by (*auto elim!*: *set-dist-bindE*)

have *read-res*:

y = *x* ∧ *s1* = *s*⌊*State* := *concat* (*State s*) *x*, *Transcript* := *xs* @ *rest*⌋

using *read-cons-outcome*[*OF read-x*] .

let *?s0* = *s*⌊*State* := *concat* (*State s*) *x*⌋

have *s1*: *s1* = *?s0*⌊*Transcript* := *xs* @ *rest*⌋

using *read-res* **by** *simp*

have *tail*:

ys' = *xs* ∧

s2 = *?s0*⌊*State* := *foldl concat* (*State ?s0*) *xs*, *Transcript* := *rest*⌋

using *Cons.IH*[*OF reads*[*unfolded s1*]] .

have *ys-t*: *ys* = *y* # *ys'* ∧ *t* = *s2*

using *ret* **by** *simp*

show *?case*

using *read-res tail ys-t* **by** *simp*

qed

theorem *honest-transcript-replay-mfold2-ntimes*:

assumes *init*: *Transcript s* = []

and *sent*: *Some* (*()*, *sent*) ∈ *set-dist* (*execute* (*mfold2 send xs*) *s*)

```

and replay:
  Some (ys, replayed)  $\in$ 
    set-dist (execute
      (ntimes read (length xs))
      (sent(State := State s, Transcript := rev (Transcript sent))))
shows ys = xs  $\wedge$  replayed = sent(Transcript := [])
proof –
have sent-eq:
  sent =
    s(
      State := foldl concat (State s) xs,
      Transcript := rev xs
    )
using mfold2-send-outcome[OF sent] init by simp
let ?replay-start = sent(State := State s)
have start-eq:
  sent(State := State s, Transcript := rev (Transcript sent)) =
    ?replay-start(Transcript := xs)
using sent-eq by simp
have replay-out:
  ys = xs  $\wedge$ 
    replayed =
      ?replay-start(
        State := foldl concat (State ?replay-start) xs,
        Transcript := []
      )
using ntimes-read-outcome[OF replay[unfolded start-eq]] .
then show ?thesis
using sent-eq by simp
qed

```

```

lemma ntimes-read-no-failure:
assumes  $n \leq \text{length } (\text{Transcript } s)$ 
shows  $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (\text{ntimes read } n) s))$ 
using assms
proof (induction n arbitrary: s)
case 0
then show ?case by simp
next
case (Suc n)
have nonempty: Transcript s  $\neq []$ 
using Suc.prems by (cases Transcript s) auto
have nf-read:  $\text{None} \notin \text{dom } (\text{dist } (\text{execute read } s))$ 
by (rule read-no-failure[OF nonempty])
have nf-cont:
 $\bigwedge x t. \text{Some } (x, t) \in \text{set-dist } (\text{execute read } s) \implies$ 
   $\text{None} \notin \text{dom}$ 
   $(\text{dist } (\text{execute}$ 
     $(\text{ntimes read } n \gg (\lambda xs. \text{return } (x \# xs))) t))$ 

```

```

proof –
  fix  $x\ t$ 
  assume  $out: \text{Some } (x, t) \in \text{set-dist } (\text{execute read } s)$ 
  obtain  $y\ ys$  where  $tr: \text{Transcript } s = y \# ys$ 
    using nonempty by (cases Transcript s) auto
  have  $t\text{-eq}: t = s(\text{State} := \text{concat } (\text{State } s)\ y, \text{Transcript} := ys)$ 
    using read-nonempty-outcome[OF out tr] by simp
  have  $n \leq \text{length } ys$ 
    using Suc.premis tr by simp
  then have  $nf\text{-tail}: \text{None} \notin \text{dom } (\text{dist } (\text{execute } (ntimes \text{ read } n)\ t))$ 
    using Suc.IH[of t] unfolding  $t\text{-eq}$  by simp
  show  $\text{None} \notin \text{dom}$ 
    ( $\text{dist } (\text{execute } (ntimes \text{ read } n \gg (\lambda xs. \text{return } (x \# xs)))\ t)$ )
    by (rule no-failure-bindI[OF nf-tail]) simp
  qed
show ?case
  by (simp add: no-failure-bindI[OF nf-read nf-cont])
qed

lemma check-authentication-path-no-failure:
   $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (\text{check-authentication-path len } i\ v\ \text{path})\ s))$ 
proof (induction path arbitrary: len i v s)
  case Nil
  then show ?case
    by (simp add: hash-no-failure)
next
  case (Cons a path)
  show ?case
  proof (cases i < len div 2)
  case True
  then show ?thesis
    by (simp add: no-failure-bindI hash-no-failure Cons.IH)
  next
  case False
  then show ?thesis
    by (simp add: no-failure-bindI hash-no-failure Cons.IH)
  qed
qed

lemma check-authentication-path-preserves-channel:
  assumes  $\text{Some } (h, t) \in \text{set-dist } (\text{execute } (\text{check-authentication-path len } i\ v\ \text{path})\ s)$ 
  shows  $\text{State } t = \text{State } s$ 
    and  $\text{Transcript } t = \text{Transcript } s$ 
proof –
  have both:
     $\text{State } t = \text{State } s \wedge \text{Transcript } t = \text{Transcript } s$ 
  if outcome:
     $\text{Some } (h, t) \in \text{set-dist } (\text{execute } (\text{check-authentication-path len } i\ v\ \text{path})\ s)$ 

```

```

for len i v s h t
using outcome
proof (induction path arbitrary: len i v s h t)
  case Nil
  have hash-step: Some (h, t) ∈ set-dist (execute (hash v) s)
    using Nil.prems by simp
  show ?case
    using hash-channel-preserves(1)[OF hash-step]
      hash-channel-preserves(2)[OF hash-step]
    by simp
next
  case (Cons a path)
  show ?case
  proof (cases i < len div 2)
    case True
    from Cons.prems obtain x s1 where
      rest:
        Some (x, s1) ∈
          set-dist (execute (check-authentication-path (len div 2) i v path) s)
        and hash-step: Some (h, t) ∈ set-dist (execute (hash (concat x a)) s1)
        using True by (auto elim! set-dist-bindE)
    have rec: State s1 = State s ∧ Transcript s1 = Transcript s
      using Cons.IH[OF rest] .
    have hash-pres: State t = State s1 ∧ Transcript t = Transcript s1
      using hash-channel-preserves(1)[OF hash-step]
        hash-channel-preserves(2)[OF hash-step]
      by simp
    show ?thesis
      using rec hash-pres by simp
  next
  case False
  from Cons.prems obtain x s1 where
    rest:
      Some (x, s1) ∈
        set-dist (execute (check-authentication-path (len div 2) (i - len div 2) v path) s)
      and hash-step: Some (h, t) ∈ set-dist (execute (hash (concat a x)) s1)
      using False by (auto elim! set-dist-bindE)
    have rec: State s1 = State s ∧ Transcript s1 = Transcript s
      using Cons.IH[OF rest] .
    have hash-pres: State t = State s1 ∧ Transcript t = Transcript s1
      using hash-channel-preserves(1)[OF hash-step]
        hash-channel-preserves(2)[OF hash-step]
      by simp
    show ?thesis
      using rec hash-pres by simp
  qed
qed
show State t = State s
  using both[OF assms] by simp

```



```

show Transcript t = Transcript s
  using both[OF assms] by simp
qed

lemma honest-query-decommitment-no-failure:
  assumes created: created-tree xs tree s0
    and len-pow: length xs =  $2^n$ 
    and idx: i < length xs
    and leaf: v = xs ! i
    and ext: s0 ≤ s
    and path-len: length (get-authentication-path (length xs) i tree) = floor-log (length xs)
    and root: fr = value tree
  shows
    None ∉ dom (dist (execute
      (do {
        qh ← read;
        qh-path ← ntimes read (floor-log (length xs));
        ap ← check-authentication-path (length xs) i qh qh-path;
        assert (ap = fr);
        return qh
      }) (s | Transcript := v # get-authentication-path (length xs) i tree @ rest))))))
  proof –
    let ?path = get-authentication-path (length xs) i tree
    let ?s = s | Transcript := v # ?path @ rest
    let ?prefix =
      do {
        qh ← read;
        qh-path ← ntimes read (floor-log (length xs));
        ap ← check-authentication-path (length xs) i qh qh-path;
        return (qh, qh-path, ap)
      }
    have nf-read: None ∉ dom (dist (execute read ?s))
      by (rule read-no-failure) simp
    have nf-after-read:
       $\bigwedge qh\ s1. \text{Some } (qh, s1) \in \text{set-dist } (\text{execute read } ?s) \implies$ 
      None ∉ dom
      (dist (execute
        (ntimes read (floor-log (length xs)))  $\gg$ 
        ( $\lambda qh\text{-path}. \text{check-authentication-path } (\text{length } xs) \ i \ qh \ qh\text{-path} \gg$ 
          ( $\lambda ap. \text{return } (qh, qh\text{-path}, ap)$ )))) s1))
    proof –
      fix qh s1
      assume read-qh: Some (qh, s1) ∈ set-dist (execute read ?s)
      have read-qh-res:
        s1 = s | State := concat (State s) v, Transcript := ?path @ rest
        using read-cons-outcome[OF read-qh] by simp
      have enough-path: floor-log (length xs) ≤ length (Transcript s1)
        using path-len unfolding read-qh-res by simp
      have nf-reads:

```

```

  None  $\notin$  dom (dist (execute (ntimes read (floor-log (length xs))) s1))
  by (rule ntimes-read-no-failure[OF enough-path])
have nf-after-reads:
   $\bigwedge$  qh-path s2. Some (qh-path, s2)  $\in$ 
    set-dist (execute (ntimes read (floor-log (length xs))) s1)  $\implies$ 
    None  $\notin$  dom
      (dist (execute
        (check-authentication-path (length xs) i qh qh-path  $\gg$ 
          ( $\lambda$  ap. return (qh, qh-path, ap))) s2))
proof -
  fix qh-path s2
  assume Some (qh-path, s2)  $\in$ 
    set-dist (execute (ntimes read (floor-log (length xs))) s1)
  have nf-check:
    None  $\notin$  dom
      (dist (execute (check-authentication-path (length xs) i qh qh-path) s2))
  by (rule check-authentication-path-no-failure)
  show None  $\notin$  dom
    (dist (execute
      (check-authentication-path (length xs) i qh qh-path  $\gg$ 
        ( $\lambda$  ap. return (qh, qh-path, ap))) s2))
  proof (rule no-failure-bindI[OF nf-check])
    fix ap t
    assume Some (ap, t)  $\in$ 
      set-dist (execute (check-authentication-path (length xs) i qh qh-path) s2)
    show None  $\notin$  dom (dist (execute (return (qh, qh-path, ap)) t))
    by simp
  qed
qed
show None  $\notin$  dom
  (dist (execute
    (ntimes read (floor-log (length xs))  $\gg$ 
      ( $\lambda$  qh-path. check-authentication-path (length xs) i qh qh-path  $\gg$ 
        ( $\lambda$  ap. return (qh, qh-path, ap)))) s1))
  using no-failure-bindI[OF nf-reads nf-after-reads] .
qed
have prefix-no-failure: None  $\notin$  dom (dist (execute ?prefix ?s))
  using no-failure-bindI[OF nf-read nf-after-read] .
have full:
  do {
    qh  $\leftarrow$  read;
    qh-path  $\leftarrow$  ntimes read (floor-log (length xs));
    ap  $\leftarrow$  check-authentication-path (length xs) i qh qh-path;
    assert (ap = fr);
    return qh
  } =
  (?prefix  $\gg$  ( $\lambda$ (qh, qh-path, ap). assert (ap = fr)  $\gg$  ( $\lambda$ -. return qh)))
  by (simp add: sm-bind-assoc split: prod.splits)
show ?thesis

```

```

unfolding full
proof (rule no-failure-bindI[OF prefix-no-failure])
  fix out t
  assume out:
    Some (out, t) ∈ set-dist (execute ?prefix ?s)
  obtain gh gh-path ap where out-eq: out = (gh, gh-path, ap)
  by (cases out) auto
  from out[unfolded out-eq] obtain s1 s2 where
    read-leaf: Some (gh, s1) ∈ set-dist (execute read ?s)
  and read-path:
    Some (gh-path, s2) ∈
      set-dist (execute (ntimes read (floor-log (length xs))) s1)
  and check:
    Some (ap, t) ∈
      set-dist (execute
        (check-authentication-path (length xs) i gh gh-path) s2)
  by (auto elim!: set-dist-bindE)
  have read-leaf-res:
    gh = v ∧
    s1 = s(State := concat (State s) v, Transcript := ?path @ rest)
  using read-cons-outcome[OF read-leaf] by simp
  let ?s-leaf = s(State := concat (State s) v)
  have s1-eq: s1 = ?s-leaf(Transcript := ?path @ rest)
  using read-leaf-res by simp
  have path-replay:
    gh-path = ?path ∧
    s2 = ?s-leaf(
      State := foldl concat (State ?s-leaf) ?path,
      Transcript := rest)
  using ntimes-read-prefix-outcome[OF read-path[unfolded path-len[symmetric] s1-eq]]
  by simp
  have s0-s2: s0 ≤ s2
  using ext read-leaf-res path-replay
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  have check-honest:
    Some (ap, t) ∈
      set-dist (execute
        (check-authentication-path (length xs) i v ?path) s2)
  using check read-leaf-res path-replay by simp
  have ap-eq: ap = value tree
  using check-created-tree-outcome[
    OF created len-pow idx leaf s0-s2 check-honest]
  by simp
  then show None ∉ dom
    (dist (execute
      ((case out of (gh, gh-path, ap) ⇒ assert (ap = fr) ≫= (λ-. return gh))) t))
  unfolding out-eq root assert-def by simp
qed
qed

```

end

end

```
theory Channel-Instances
  imports Channel-Transcript
begin
```

1.23 Boolean Interpretation

```
global-interpretation test-channel:
```

```
  channel concat-bool
```

```
  defines test-send = test-channel.send
```

```
    and test-send2 = test-channel.send2
```

```
    and test-read = test-channel.read
```

```
  .
```

```
value execute (receive-random-field-element) ( $\lambda$ HashMap = (fmempty::(bool, bool) fmap), State = True, Transcript=[])
```

```
value execute (test-send True) ( $\lambda$ HashMap = (fmempty::(bool, bool) fmap), State = True, Transcript=[])
```

The idea is that if we process a transcript then we recover the state and get the same random element. To this end we need to execute the monad on the final HashMap, the initial state and the reversed transcript. Each read then modifies the state in the same way as a send which leads to the same result for the hash value used to create the random value.

```
value
```

```
  execute (
```

```
    do {
```

```
      test-send True;
```

```
      r1  $\leftarrow$  receive-random-field-element;
```

```
      test-send False;
```

```
      r2  $\leftarrow$  receive-random-field-element;
```

```
      return r1
```

```
    }) ( $\lambda$ HashMap = (fmempty::(bool, bool) fmap), State = True, Transcript=[])
```

```
lemma
```

```
  execute (
```

```
    do {
```

```
      test-read;
```

```
      r1  $\leftarrow$  receive-random-field-element;
```

```
      test-read;
```

```
      r2  $\leftarrow$  receive-random-field-element;
```

```
      return (r1  $\wedge$   $\neg$  r2)
```

} ($\text{HashMap} = \text{fmap-of-list } [(True, True), (False, False)], \text{State} = True, \text{Transcript} = \text{rev } [False, True]$)

=

$\text{dist-of-fset } (\{ | (\text{Some } (True, (\text{HashMap} = \text{fmap-of-list } [(True, True), (False, False)], \text{State} = False, \text{Transcript} = [])), 1) | \})$

by eval

lemma

execute (

do {

test-read;

$r1 \leftarrow \text{receive-random-field-element};$

test-read;

$r2 \leftarrow \text{receive-random-field-element};$

return ($r1 \wedge r2$)

} ($\text{HashMap} = \text{fmap-of-list } [(True, True), (False, True)], \text{State} = True, \text{Transcript} = \text{rev } [False, True]$)

=

$\text{dist-of-fset } (\{ | (\text{Some } (True, (\text{HashMap} = \text{fmap-of-list } [(True, True), (False, True)], \text{State} = False, \text{Transcript} = [])), 1) | \})$

by eval

lemma

execute (

do {

test-read;

$r1 \leftarrow \text{receive-random-field-element};$

test-read;

$r2 \leftarrow \text{receive-random-field-element};$

return ($\neg r1 \wedge r2$)

} ($\text{HashMap} = \text{fmap-of-list } [(True, False), (False, True)], \text{State} = True, \text{Transcript} = \text{rev } [False, True]$)

=

$\text{dist-of-fset } (\{ | (\text{Some } (True, (\text{HashMap} = \text{fmap-of-list } [(True, False), (False, True)], \text{State} = False, \text{Transcript} = [])), 1) | \})$

by eval

lemma

execute (

do {

test-read;

$r1 \leftarrow \text{receive-random-field-element};$

test-read;

$r2 \leftarrow \text{receive-random-field-element};$

return ($\neg r1 \wedge \neg r2$)

} ($\text{HashMap} = \text{fmap-of-list } [(True, False), (False, False)], \text{State} = True, \text{Transcript} = \text{rev } [False, True]$)

=

$\text{dist-of-fset } (\{ | (\text{Some } (True, (\text{HashMap} = \text{fmap-of-list } [(True, False), (False, False)], \text{State} =$

```
False, Transcript = []), 1)|})
  by eval
```

1.24 GF5 Interpretation

global-interpretation *gf5-channel*:

```
channel concat-gf5
defines gf5-send = gf5-channel.send
  and gf5-send2 = gf5-channel.send2
  and gf5-read = gf5-channel.read
.
```

lemma *execute (receive-random-field-element)* $(\text{HashMap} = (\text{fmempty}::(\text{gf5}, \text{gf5}) \text{fmap}), \text{State} = 0, \text{Transcript} = [])$

```
=
dist-of-fset (
  { |(Some (4, (HashMap = fmap-of-list [(0, 4)], State = 0, Transcript = [])), (1 / 5)),
    (Some (3, (HashMap = fmap-of-list [(0, 3)], State = 0, Transcript = [])), (1 / 5)),
    (Some (2, (HashMap = fmap-of-list [(0, 2)], State = 0, Transcript = [])), (1 / 5)),
    (Some (1, (HashMap = fmap-of-list [(0, 1)], State = 0, Transcript = [])), (1 / 5)),
    (Some (0, (HashMap = fmap-of-list [(0, 0)], State = 0, Transcript = [])), (1 / 5))|})
  by eval
```

lemma *execute (gf5-send 1)* $(\text{HashMap} = (\text{fmempty}::(\text{gf5}, \text{gf5}) \text{fmap}), \text{State} = 0, \text{Transcript} = [])$

```
=
dist-of-fset ({ |(Some ((), (HashMap = fmap-of-list [], State = 1, Transcript = [1])), 1)|})
  by eval
```

end

theory *Channel*

imports *Channel-Instances*

begin

Compatibility umbrella for the channel development. The imported theories define the protocol state, transcript operations, Fiat-Shamir oracle calls, and concrete field instances used by the STARK prover and verifier.

end

theory *Galois-Field-5*

imports *HOL-Number-Theory.Number-Theory*

HOL-Library.Cardinality

Berlekamp-Zassenhaus.Finite-Field

begin

1.25 The Field GF(5)

This theory provides a small executable finite field instance used by the square-sequence example. The instance is intentionally small so that example proof generation and verification remain feasible inside Isabelle.

```
typedef gf5 = {0 .. 4 :: int}  
by auto
```

```
setup-lifting type-definition-gf5
```

```
lift-definition gf5-0:: gf5  
is 0 by simp
```

```
lift-definition gf5-1:: gf5  
is 1 by simp
```

```
lift-definition gf5-2:: gf5  
is 2 by simp
```

```
lift-definition gf5-3:: gf5  
is 3 by simp
```

```
lift-definition gf5-4:: gf5  
is 4 by simp
```

```
lemma [simp]: gf5-0  $\neq$  gf5-1  
by transfer simp
```

```
lemma [simp]: gf5-0  $\neq$  gf5-2  
by transfer simp
```

```
lemma [simp]: gf5-0  $\neq$  gf5-3  
by transfer simp
```

```
lemma [simp]: gf5-0  $\neq$  gf5-4  
by transfer simp
```

```
lemma [simp]: gf5-1  $\neq$  gf5-2  
by transfer simp
```

```
lemma [simp]: gf5-1  $\neq$  gf5-3  
by transfer simp
```

```
lemma [simp]: gf5-1  $\neq$  gf5-4  
by transfer simp
```

```
lemma [simp]: gf5-2  $\neq$  gf5-3  
by transfer simp
```

```

lemma [simp]:  $gf5-2 \neq gf5-4$ 
  by transfer simp

lemma [simp]:  $gf5-3 \neq gf5-4$ 
  by transfer simp

instantiation  $gf5 :: field$ 
begin
lift-definition  $zero-gf5 :: gf5$  is 0 by auto
lift-definition  $one-gf5 :: gf5$  is 1 by auto
lift-definition  $uminus-gf5 :: gf5 \Rightarrow gf5$  is  $\lambda x. (5 - x) \bmod 5$  by auto
lift-definition  $plus-gf5 :: gf5 \Rightarrow gf5 \Rightarrow gf5$  is  $\lambda x y. (x + y) \bmod 5$  by auto
definition  $minus-gf5 :: gf5 \Rightarrow gf5 \Rightarrow gf5$  where  $minus-gf5\ x\ y = (x + (uminus\ y))$ 
lift-definition  $times-gf5 :: gf5 \Rightarrow gf5 \Rightarrow gf5$  is  $\lambda x y. (x * y) \bmod 5$  by auto
lift-definition  $inverse-gf5 :: gf5 \Rightarrow gf5$  is  $\lambda x. \text{if } x = 2 \text{ then } 3 \text{ else if } x = 3 \text{ then } 2 \text{ else } x$  by
  auto
definition  $divide-gf5 :: gf5 \Rightarrow gf5 \Rightarrow gf5$  where  $divide-gf5\ x\ y = (x * inverse\ y)$ 

instance
  apply standard
    apply (transfer, simp add: mod-mult-left-eq mod-mult-right-eq mult.assoc)
    apply (transfer, simp add: mult.commute)
    apply (transfer, simp)
    apply (transfer, presburger)
    apply (transfer, presburger)
    apply (transfer, simp)
    apply (transfer, presburger)
    apply (simp add: minus-gf5-def)
    apply (transfer, simp add: distrib-right mod-add-eq mod-mult-left-eq)
    apply (transfer, simp)
    apply (transfer, (case-tac a=1; auto), (case-tac a=4; auto))
    apply (simp add: divide-gf5-def)
  by (transfer, presburger)
end

lift-definition  $to-nat::gf5 \Rightarrow nat$ 
is nat .

lift-definition  $of-nat::nat \Rightarrow gf5$ 
is  $\lambda x::nat. \text{int } (x \bmod 5)$  by auto

lemma  $of-nat-to-nat[simp]: of-nat\ (to-nat\ x) = x$ 
  apply transfer by fastforce

lemma  $gf5-cases[cases\ type:gf5]:$ 
  fixes  $x::gf5$ 
  obtains
     $x = gf5-0 \mid x = gf5-1 \mid x = gf5-2 \mid x = gf5-3 \mid x = gf5-4$ 
  apply transfer

```



```

by fastforce

lemma gf5-Univ: (UNIV::gf5 set) = {gf5-0,gf5-1,gf5-2,gf5-3,gf5-4}
  using gf5-cases by blast

instantiation gf5 :: enum
begin

definition enum-gf5 :: gf5 list
  where enum-gf5 = [gf5-0, gf5-1, gf5-2, gf5-3, gf5-4]

definition enum-all-gf5 :: (gf5  $\Rightarrow$  bool)  $\Rightarrow$  bool
  where enum-all-gf5 P  $\equiv$  P gf5-0  $\wedge$  P gf5-1  $\wedge$  P gf5-2  $\wedge$  P gf5-3  $\wedge$  P gf5-4

definition enum-ex-gf5 :: (gf5  $\Rightarrow$  bool)  $\Rightarrow$  bool
  where enum-ex-gf5 P  $\equiv$  P gf5-0  $\vee$  P gf5-1  $\vee$  P gf5-2  $\vee$  P gf5-3  $\vee$  P gf5-4

instance
  apply (standard)
  by (auto simp add: gf5-Univ enum-gf5-def enum-all-gf5-def enum-ex-gf5-def)
end

instance gf5 :: finite-field
proof
  interpret type-definition Rep-gf5 Abs-gf5 {0 .. 4 :: int}
  by (rule type-definition-gf5)
  show finite (UNIV :: gf5 set)
  by simp
qed

instantiation gf5 :: equal
begin
lift-definition equal-gf5 :: gf5  $\Rightarrow$  gf5  $\Rightarrow$  bool
is (=) parametric eq-transfer .

instance
  by (standard) (transfer, simp)
end

lemma card-5-iff: card S = 5  $\longleftrightarrow$  ( $\exists v w x y z. S = \{v,w,x,y,z\}$ 
 $\wedge v \neq w \wedge v \neq x \wedge v \neq y \wedge v \neq z$ 
 $\wedge w \neq x \wedge w \neq y \wedge w \neq z$ 
 $\wedge x \neq y \wedge x \neq z$ 
 $\wedge y \neq z$ )
  by (fastforce simp: card-Suc-eq numeral-eq-Suc)

lemma gf5-size[simp]: CARD (gf5) = 5
proof -
  have ( $\exists v w x y z. \{gf5-0, gf5-1, gf5-2, gf5-3, gf5-4\} = \{v, w, x, y, z\} \wedge v \neq w \wedge v \neq x \wedge v$ 

```

```

 $\neq y \wedge v \neq z \wedge w \neq x \wedge w \neq y \wedge w \neq z \wedge x \neq y \wedge x \neq z \wedge y \neq z)$ 
  by (metis gf5-0.rep-eq gf5-1.rep-eq gf5-2.rep-eq gf5-3.rep-eq gf5-4.rep-eq numeral-eq-iff nu-
meral-eq-one-iff semiring-norm(85) verit-eq-simplify(12,14,8) zero-neq-numeral)
  then show ?thesis using card-5-iff gf5-Univ by fastforce
qed

```

Proth numbers: https://en.wikipedia.org/wiki/Proth_prime, they are useful for FTTs

```

class proth = finite +
  assumes proth-card:  $\exists k n. \text{CARD}(a) = k * 2^n + 1$ 

```

```

class proth-field = finite-field + proth + prime-card

```

```

instantiation gf5::proth-field
begin
instance proof
  show normalization-semidom-class.prime CARD(gf5) by simp
next
  show  $\exists k n. \text{CARD}(gf5) = k * 2^n + 1$ 
  apply (simp)
  by (metis power-even-eq power2-eq-square numeral-Bit0-eq-double)
qed

end

```

```

lemma gen-2-gf5:
  fixes x::gf5
  assumes x  $\neq$  gf5-0
  shows  $\exists i. gf5-2^i = x$ 
proof (cases x)
  case 1
  with assms show ?thesis by satx
next
  case 2
  then show ?thesis
    by (metis gf5-1-def one-gf5-def power-0)
next
  case 3
  then show ?thesis
    by (meson power-Suc0-right)
next
  case 4
  moreover have  $gf5-2 * gf5-2 * gf5-2 = gf5-3$ 
    by transfer simp
  moreover have  $((gf5-2::gf5)^3) = gf5-2 * gf5-2 * gf5-2$ 
    by (rule power3-eq-cube)
  ultimately show ?thesis by metis
next
  case 5

```

```

moreover have  $gf5-2 * gf5-2 = gf5-4$ 
  by transfer simp
moreover have  $((gf5-2::gf5)^2) = gf5-2 * gf5-2$ 
  by (rule power2-eq-square)
ultimately show ?thesis by metis
qed

lemma to-nat (0::gf5) = 0 by eval
lemma to-nat (1::gf5) = 1 by eval
lemma to-nat (2::gf5) = 2 by eval
lemma to-nat (7::gf5) = 2 by eval
lemma ((of-nat 0)::gf5) = 0 by eval
lemma ((of-nat 1)::gf5) = 1 by eval
lemma ((of-nat 2)::gf5) = 2 by eval
lemma ((of-nat 3)::gf5) = 3 by eval

lemma gf5-0: 0 = gf5-0 apply transfer by presburger

end

```

```

theory Merkle-Tree
imports
  HOL-Library.Tree
  HOL-Library.Discrete-Functions
  Galois-Field-5
  Hash-Monad
begin

```

1.26 Merkle Trees

This theory defines Merkle tree creation, authentication paths, and checking in the probabilistic hash monad. The STARK prover uses these commitments for trace, composition, and FRI layers, while the verifier checks sampled openings against the recorded roots.

```

fun get-authentication-path::nat ⇒ nat ⇒ 'a tree ⇒ 'a list
where
  get-authentication-path lgth idx (Node l v r) =
    (if lgth = 1 then [] else
      (if idx < lgth div 2
        then (value r) # (get-authentication-path (lgth div 2) idx l)
        else (value l) # (get-authentication-path (lgth div 2) (idx - (lgth div 2)) r)))

locale merkle-tree =
  fixes concat :: 'a::finite ⇒ 'a ⇒ 'b
begin

fun create::'b list ⇒ ('b, 'a, 'a tree, 'c) hash-monad
where
  create [] = return ⟨⟩

```

```

| create [x] =
  do {
    h ← hash x;
    return ⟨⟨⟩, h, ⟨⟩⟩
  }
| create xs =
  do {
    let i = length xs div 2;
    ln ← create (take i xs);
    rn ← create (drop i xs);
    val ← hash (concat (value ln) (value rn));
    return ⟨ln, val, rn⟩
  }

fun check-authentication-path:: nat ⇒ nat ⇒ 'b ⇒ 'a list ⇒ ('b, 'a, 'a, 'c) hash-monad
where
  check-authentication-path lgth idx leaf-data [] = hash leaf-data
| check-authentication-path lgth idx leaf-data (a#ap) =
  (if idx < lgth div 2
   then
     do {
       x ← check-authentication-path (lgth div 2) idx leaf-data ap;
       hash (concat x a)
     }
   else
     do {
       x ← check-authentication-path (lgth div 2) (idx - (lgth div 2)) leaf-data ap;
       hash (concat a x)
     })

end

```

1.27 Weakest Precondition Calculus

context merkle-tree
begin

lemma wp-check-authentication-path[wp]:
assumes $ap = [] \implies P \leq wp \text{ (hash leaf-data) } Q \text{ s}$
assumes $\bigwedge a \text{ list.}$
 $ap = a \# list \implies$
 $idx < lgth \text{ div } 2 \implies$
 $P \leq wp \text{ (check-authentication-path (lgth div 2) idx leaf-data list } \gg= (\lambda x. \text{hash (concat x$
 $a))) \text{ } Q \text{ s}$
assumes $\bigwedge a \text{ list.}$
 $ap = a \# list \implies$
 $\neg idx < lgth \text{ div } 2 \implies$
 $P \leq wp \text{ (check-authentication-path (lgth div 2) (idx - lgth div 2) leaf-data list } \gg=$
 $(\lambda x. \text{hash (concat a x)}))$

$Q\ s$

shows $P \leq wp\ (check\text{-}authentication\text{-}path\ lgth\ idx\ leaf\text{-}data\ ap)\ Q\ s$
using *assms* **by** (*cases* *ap*) (*auto*)

lemma *wp-create*[*wp*]:

assumes $xs = [] \implies P \leq wp\ (return\ \langle \rangle)\ Q\ s$
assumes $\bigwedge x. xs = [x] \implies P \leq wp\ (hash\ x \gg (\lambda h. return\ (\langle \rangle, h, \langle \rangle)))\ Q\ s$
assumes $\bigwedge v\ vb\ vc.$
 $xs = v \# vb \# vc \implies$
 $P \leq wp\ (local.create\ (v \# take\ (length\ vc\ div\ 2)\ (vb \# vc)) \gg$
 $(\lambda ln. local.create\ (drop\ (length\ vc\ div\ 2)\ (vb \# vc)) \gg$
 $(\lambda rn. hash\ (concat\ (value\ ln)\ (value\ rn)) \gg (\lambda val. return\ (\ln, val, rn))))$
 $Q\ s$
shows $P \leq wp\ (create\ xs)\ Q\ s$
using *assms* **by** (*cases* *xs* *rule:create.cases*) (*auto*)

end

1.28 Boolean interpretation

definition *concat-bool*:: *bool* $\Rightarrow$ *bool* $\Rightarrow$ *bool*

where

concat-bool = ($\wedge$)

global-interpretation *test-merkle*:

merkle-tree concat-bool

defines *test-create* = *test-merkle.create*

and *test-check-authentication-path* = *test-merkle.check-authentication-path*

.

lemma

execute (*test-create* [*True*, *True*, *True*, *True*]) ($\llbracket HashMap = (fmempty::(bool, bool)\ fmap) \rrbracket =$
 $dist\text{-}of\text{-}fset\ ($
 $\{ \llbracket (Some\ (\langle \langle \langle \rangle, False, \langle \rangle), True, \langle \rangle, False, \langle \rangle \rangle, False, \langle \langle \rangle, False, \langle \rangle), True, \langle \rangle, False, \langle \rangle \rangle),$
 $\llbracket HashMap = fmap\text{-}of\text{-}list\ [(True, False), (False, True)] \rrbracket \rrbracket,$
 $(1\ /\ 4) \rrbracket,$
 $(Some\ (\langle \langle \langle \rangle, False, \langle \rangle), False, \langle \rangle, False, \langle \rangle \rangle, False, \langle \langle \rangle, False, \langle \rangle), False, \langle \rangle, False, \langle \rangle \rangle),$
 $\llbracket HashMap = fmap\text{-}of\text{-}list\ [(True, False), (False, False)] \rrbracket \rrbracket,$
 $(1\ /\ 4) \rrbracket,$
 $(Some\ (\langle \langle \langle \rangle, True, \langle \rangle), True, \langle \rangle, True, \langle \rangle \rangle, True, \langle \langle \rangle, True, \langle \rangle), True, \langle \rangle, True, \langle \rangle \rangle),$
 $\llbracket HashMap = fmap\text{-}of\text{-}list\ [(True, True)] \rrbracket \rrbracket, (1\ /\ 2) \rrbracket \rrbracket \}$
by *eval*

lemma

execute (*test-create* [*True*, *False*, *True*, *False*]) ($\llbracket HashMap = (fmempty::(bool, bool)\ fmap) \rrbracket =$
 $dist\text{-}of\text{-}fset\ ($
 $\{ \llbracket (Some\ (\langle \langle \langle \rangle, False, \langle \rangle), True, \langle \rangle, True, \langle \rangle \rangle, False, \langle \langle \rangle, False, \langle \rangle), True, \langle \rangle, True, \langle \rangle \rangle),$
 $\llbracket HashMap = fmap\text{-}of\text{-}list\ [(True, False), (False, True)] \rrbracket \rrbracket,$
 $(1\ /\ 4) \rrbracket,$

```

    (Some (<<<<<, False, <>), False, <>, False, <>, False, <>)), False, <<<<<, False, <>), False, <>, False, <>)),
    (HashMap = fmap-of-list [(True, False), (False, False)])),
    (1 / 4)),
    (Some (<<<<<, True, <>), True, <>, True, <>, True, <>)), True, <<<<<, True, <>), True, <>, True, <>)),
    (HashMap = fmap-of-list [(True, True), (False, True)])), (1 / 4)),
    (Some (<<<<<, True, <>), False, <>, False, <>)), False, <<<<<, True, <>), False, <>, False, <>)),
    (HashMap = fmap-of-list [(True, True), (False, False)])),
    (1 / 4)))]))
  by eval

```

lemma *execute*

```

  (do {
    let x = [True, True, True, True];
    r ← test-create x;
    let ap = get-authentication-path (length x) 2 r;
    r2 ← test-check-authentication-path (length x) 2 True ap;
    return (r2 = value r)
  }) (HashMap = (fmempty::(bool, bool) fmap))
=
dist-of-fset (
  {(Some (True, (HashMap = fmap-of-list [(True, True)])), (1 / 2)), (Some (True, (HashMap
= fmap-of-list [(True, False), (False, False)])), (1 / 4)),
  (Some (True, (HashMap = fmap-of-list [(True, False), (False, True)])), (1 / 4))]}
  by eval

```

1.29 GF5 interpretation

definition *concat-gf5:: gf5 ⇒ gf5 ⇒ gf5*

where

concat-gf5 = (+)

global-interpretation *merkle-gf5:*

merkle-tree concat-gf5

defines *gf5-create* = *merkle-gf5.create*

and *gf5-check-authentication-path* = *merkle-gf5.check-authentication-path*

.

value *execute* (*gf5-create* [1, 2, 3]) (HashMap = (fmempty::(gf5, gf5) fmap))

1.30 Domain-separated protocol Merkle interface

The generic Merkle locale already separates hash outputs from hash inputs: leaves have a separate input type, hash outputs have the field type, and internal nodes are encoded by the locale parameter. Instantiating that parameter with the constructor `MerkleNode` and wrapping leaves with `MerkleLeaf` gives the STARK protocol a domain-separated Merkle API without duplicating the existing Merkle proofs.

global-interpretation *protocol-merkle:*

merkle-tree ($\lambda x y. \text{MerkleNode } x y$)

```

defines protocol-merkle-create-raw = protocol-merkle.create
and protocol-merkle-check-authentication-path-raw =
  protocol-merkle.check-authentication-path

```

```

definition protocol-create
:: 'a::finite list  $\Rightarrow$ 
  ('a protocol-hash-input, 'a, 'a tree, 'c) hash-monad
where
  protocol-create xs =
    protocol-merkle.create (map MerkleLeaf xs)

```

```

definition protocol-check-authentication-path
:: nat  $\Rightarrow$  nat  $\Rightarrow$  'a::finite  $\Rightarrow$  'a list  $\Rightarrow$ 
  ('a protocol-hash-input, 'a, 'a, 'c) hash-monad
where
  protocol-check-authentication-path len idx leaf path =
    protocol-merkle.check-authentication-path len idx (MerkleLeaf leaf) path

```

1.31 Sanity Check

```

context merkle-tree
begin

```

The created-tree predicate is a proof-only invariant connecting the result of create to the final hash-map state. It is needed because create samples hashes probabilistically, while authentication-path checking later recomputes hashes. To prove the recomputation succeeds with probability 1, the proof needs to know those sampled hashes are already recorded in the hash map, so future calls to hash replay the same values instead of sampling fresh ones. So the created-tree predicate is not part of the Merkle API. It is a ghost invariant used to bridge "this tree was produced by create" and "checking its authentication path will replay the same root hash."

```

fun created-tree::'b list  $\Rightarrow$  'a tree  $\Rightarrow$  ('b, 'a, 'c) hash-scheme  $\Rightarrow$  bool
where
  created-tree [] t s  $\longleftrightarrow$  t =  $\langle \rangle$ 
| created-tree [x] t s  $\longleftrightarrow$ 
  ( $\exists h. t = \langle \rangle, h, \langle \rangle \wedge$  fmllookup (HashMap s) x = Some h)
| created-tree xs t s  $\longleftrightarrow$ 
  (let i = length xs div 2 in
    $\exists l h r. t = \langle l, h, r \rangle \wedge$ 
    created-tree (take i xs) l s  $\wedge$ 
    created-tree (drop i xs) r s  $\wedge$ 
    fmllookup (HashMap s) (concat (value l) (value r)) = Some h)

```

```

declare created-tree.simps[simp del]

```

```

lemma set-dist-bindE:
assumes Some (y, t)  $\in$  set-dist (execute (m  $\gg$  f) s)

```

```

obtains  $x\ u$  where  $\text{Some } (x, u) \in \text{set-dist } (\text{execute } m\ s)$ 
and  $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (f\ x)\ u)$ 
proof –
  let  $?K = \text{bind-cont-map } (\text{map-fun } \text{id } \text{dist} \circ \text{execute} \circ f)$ 
  have  $\text{bind-dom}$ :
     $\text{Some } (y, t) \in \text{dom } (\text{map-bind } (\text{dist} \circ \text{execute } m)\ ?K\ s)$ 
    using  $\text{assms}$ 
    unfolding  $\text{set-dist-def } \text{sm-bind.rep-eq } \text{dist-bind.rep-eq}$ 
    by  $(\text{simp add: o-def map-fun-def})$ 
  then have  $\text{union}$ :
     $\text{Some } (y, t) \in$ 
     $(\bigcup_{i \in \text{dom } (\text{dist } (\text{execute } m\ s))} \text{dom } (?K\ i))$ 
    using  $\text{dom-map-bind}[of\ \text{dist} \circ \text{execute } m\ ?K\ s]$ 
    by  $\text{auto}$ 
  then obtain  $i$  where  $i\text{-dom}$ :  $i \in \text{dom } (\text{dist } (\text{execute } m\ s))$ 
    and  $y\text{-dom}$ :  $\text{Some } (y, t) \in \text{dom } (?K\ i)$ 
    by  $\text{auto}$ 
  obtain  $x\ u$  where  $i$ :  $i = \text{Some } (x, u)$ 
  proof  $(\text{cases } i)$ 
    case  $\text{None}$ 
    then have  $\text{False}$ 
      using  $y\text{-dom}$ 
      unfolding  $\text{bind-cont-map-def } \text{delta-map-def}$ 
      by  $\text{simp}$ 
    then show  $?thesis \dots$ 
  next
    case  $(\text{Some } p)$ 
    then obtain  $a\ b$  where  $i = \text{Some } (a, b)$ 
      by  $(\text{cases } p)\ \text{auto}$ 
    then show  $?thesis$ 
      by  $(\text{rule that})$ 
  qed
  have  $\text{Some } (x, u) \in \text{set-dist } (\text{execute } m\ s)$ 
    using  $i\text{-dom } i$  unfolding  $\text{set-dist-def}$  by  $\text{simp}$ 
  moreover have  $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (f\ x)\ u)$ 
    using  $y\text{-dom } i$ 
    unfolding  $\text{set-dist-def } \text{bind-cont-map-def}$ 
    by  $(\text{simp add: o-def map-fun-def})$ 
  ultimately show  $?thesis$ 
    by  $(\text{rule that})$ 
qed

lemma  $\text{set-dist-return-iff}[\text{simp}]$ :
   $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (\text{return } x)\ s) \longleftrightarrow y = x \wedge t = s$ 
  unfolding  $\text{set-dist-def } \text{return.rep-eq } \text{dist-return-def}$ 
  by  $(\text{simp add: dist-delta-dist } \text{delta-map-def})$ 

lemma  $\text{set-dist-get-iff}[\text{simp}]$ :
   $\text{Some } (y, t) \in \text{set-dist } (\text{execute } \text{get } s) \longleftrightarrow y = s \wedge t = s$ 

```


unfolding *set-dist-def get.rep-eq dist-get-def*
by (*simp add: dist-delta-dist delta-map-def*)

lemma *set-dist-put-iff[simp]*:

Some (y, t) ∈ set-dist (execute (put s') s) ⟷ y = () ∧ t = s'

unfolding *set-dist-def put.rep-eq dist-put-def*
by (*simp add: dist-delta-dist delta-map-def*)

lemma *hash-outcome*:

assumes *Some (h, t) ∈ set-dist (execute (hash x) s)*

shows $s \leq t$

and *fmlookup (HashMap t) x = Some h*

proof –

from *assms* **obtain** *a s1* **where**

a: Some (a, s1) ∈ set-dist (execute (apply-hash x) s)

and *rest: Some (h, t) ∈*

set-dist (execute (modify-HashMap x a >>= (λ-. return a)) s1)

proof –

have *Some (h, t) ∈*

set-dist (execute

(apply-hash x >>= (λa. modify-HashMap x a >>= (λ-. return a))) s)

using *assms* **unfolding** *hash-def* .

then show *?thesis*

by (*rule set-dist-bindE*) (*rule that*)

qed

have *a-img: Some (a, s1) ∈ (λa. Some (a, s)) ‘ set-dist (hash-dist x s)*

using *a*

unfolding *apply-hash-def lift.rep-eq lift-dist-def*

by (*simp add: set-dist-dist-map*)

then have *s1: s1 = s*

by *auto*

from *a-img* **have** *a-dom: a ∈ dom (dist (hash-dist x s))*

unfolding *set-dist-def* **by** *auto*

from *rest* **obtain** *u s2* **where**

mod: Some (u, s2) ∈ set-dist (execute (modify-HashMap x a) s1)

and *ret: Some (h, t) ∈ set-dist (execute (return a) s2)*

by (*elim set-dist-bindE*)

from *mod* **have** *s2: s2 = s1 (HashMap := fmupd x a (HashMap s1))*

unfolding *modify-HashMap-def modify-def*

by (*auto elim!: set-dist-bindE*)

from *ret* **have** *h: h = a* **and** *t: t = s2*

by *auto*

show $s \leq t$

using *a-dom* **unfolding** *s1 s2 h t* **by** (*rule hash-update-extends*)

show *fmlookup (HashMap t) x = Some h*

using *a-dom* **unfolding** *s1 s2 h t hash-dist-def option-default-dist-def*

by (*cases fmlookup (HashMap s) x*) (*auto simp: dist-delta-dist delta-map-def*)

qed

```

lemma hash-extension-lookup:
  assumes fmlookup (HashMap s0) x = Some y
    and s0 ≤ s
  shows fmlookup (HashMap s) x = Some y
  using assms
  unfolding less-eq-hash-ext-def less-eq-fmap-def
  by (metis option.distinct(1))

lemma hash-ext-trans:
  assumes s0 ≤ s1 and s1 ≤ s2
  shows s0 ≤ (s2::('b, 'a, 'c) hash-scheme)
  using assms
  unfolding less-eq-hash-ext-def less-eq-fmap-def
  by (metis order.trans)

lemma hash-ext-refl:
  s ≤ (s::('b, 'a, 'c) hash-scheme)
  unfolding less-eq-hash-ext-def less-eq-fmap-def
  by simp

lemma created-tree-mono:
  assumes created-tree xs t s0
    and s0 ≤ s
  shows created-tree xs t s
  using assms
  by (induction xs t s0 arbitrary: s rule: created-tree.induct)
    (auto simp: Let-def created-tree.simps dest: hash-extension-lookup)

lemma create-outcome:
  assumes Some (r, t) ∈ set-dist (execute (create xs) s)
  shows s ≤ t ∧ created-tree xs r t
  using assms
proof (induction xs arbitrary: r s t rule: measure-induct-rule[of length])
case (less xs)
show ?case
  proof (cases xs rule: create.cases)
    case 1
    with less.prems show ?thesis
    by (auto simp: created-tree.simps intro: hash-ext-refl)
  next
    case (2 x)
    from less.prems[unfolded 2] obtain h s1 where
      h: Some (h, s1) ∈ set-dist (execute (hash x) s)
      and ret: Some (r, t) ∈
        set-dist (execute (return ⟨⟨⟩, h, ⟨⟩⟩) s1)
      by (auto elim!: set-dist-bindE)
    then show ?thesis
    using hash-outcome unfolding 2 by (auto simp: created-tree.simps)
  next

```

```

case ( $\exists v \text{ } vb \text{ } vc$ )
let  $?i = \text{length } xs \text{ div } 2$ 
have  $\text{len-ge}: 2 \leq \text{length } xs$ 
  unfolding  $\exists$  by simp
have  $\text{take-less}: \text{length } (\text{take } ?i \text{ } xs) < \text{length } xs$ 
  using  $\text{len-ge}$  by simp
have  $\text{drop-less}: \text{length } (\text{drop } ?i \text{ } xs) < \text{length } xs$ 
  using  $\text{len-ge}$  by simp
have  $\text{take-eq}: \text{take } ?i \text{ } xs = v \# \text{take } (\text{length } vc \text{ div } 2) \text{ } (vb \# vc)$ 
  unfolding  $\exists$  by simp
have  $\text{drop-eq}: \text{drop } ?i \text{ } xs = \text{drop } (\text{length } vc \text{ div } 2) \text{ } (vb \# vc)$ 
  unfolding  $\exists$  by simp
have  $\text{split-create}: \text{Some } (r, t) \in \text{set-dist } (\text{execute } (\text{create } (\text{take } ?i \text{ } xs)) \gg=$ 
   $(\lambda l. \text{create } (\text{drop } ?i \text{ } xs)) \gg=$ 
   $(\lambda rn. \text{hash } (\text{concat } (\text{value } l) (\text{value } rn))) \gg=$ 
   $(\lambda val. \text{return } \langle l, val, rn \rangle))) s$ 
  using less.prems unfolding  $\exists$   $\text{take-eq}$   $\text{drop-eq}$  by simp
from  $\text{split-create}$  obtain  $l \text{ } s1$  where
   $l: \text{Some } (l, s1) \in \text{set-dist } (\text{execute } (\text{create } (\text{take } ?i \text{ } xs)) s)$ 
  and  $\text{rest1}: \text{Some } (r, t) \in$ 
   $\text{set-dist } (\text{execute } (\text{create } (\text{drop } ?i \text{ } xs)) \gg=$ 
   $(\lambda rn. \text{hash } (\text{concat } (\text{value } l) (\text{value } rn))) \gg=$ 
   $(\lambda val. \text{return } \langle l, val, rn \rangle))) s1$ 
  by (rule set-dist-bindE) (rule that)
from  $\text{rest1}$  obtain  $rn \text{ } s2$  where
   $rn: \text{Some } (rn, s2) \in \text{set-dist } (\text{execute } (\text{create } (\text{drop } ?i \text{ } xs)) s1)$ 
  and  $\text{rest2}: \text{Some } (r, t) \in$ 
   $\text{set-dist } (\text{execute } (\text{hash } (\text{concat } (\text{value } l) (\text{value } rn))) \gg=$ 
   $(\lambda val. \text{return } \langle l, val, rn \rangle))) s2$ 
  by (rule set-dist-bindE) (rule that)
from  $\text{rest2}$  obtain  $val \text{ } s3$  where
   $val: \text{Some } (val, s3) \in \text{set-dist } (\text{execute } (\text{hash } (\text{concat } (\text{value } l) (\text{value } rn)))) s2$ 
  and  $\text{ret}: \text{Some } (r, t) \in \text{set-dist } (\text{execute } (\text{return } \langle l, val, rn \rangle)) s3$ 
  by (auto elim!: set-dist-bindE)
have  $\text{left-out}: s \leq s1 \wedge \text{created-tree } (\text{take } ?i \text{ } xs) \text{ } l \text{ } s1$ 
  by (rule less.IH[OF take-less l])
have  $\text{right-out}: s1 \leq s2 \wedge \text{created-tree } (\text{drop } ?i \text{ } xs) \text{ } rn \text{ } s2$ 
  by (rule less.IH[OF drop-less rn])
have  $s2 \leq s3$  and  $\text{root-lookup}: \text{fmlookup } (\text{HashMap } s3) (\text{concat } (\text{value } l) (\text{value } rn)) = \text{Some } val$ 
  using hash-outcome[OF val] by auto
from  $\text{ret}$  have  $r: r = \langle l, val, rn \rangle$  and  $t: t = s3$ 
  by auto
have  $s1 \leq s3$ 
  using  $\text{right-out } s2 \leq s3$  by (auto intro: hash-ext-trans)
have  $st3: s \leq s3$ 

```

```

    using left-out s1-s3 by (auto intro: hash-ext-trans)
  have created-tree (take ?i xs) l s3
    using left-out s1-s3 by (auto intro: created-tree-mono)
  moreover have created-tree (drop ?i xs) rn s3
    using right-out s2-s3 by (auto intro: created-tree-mono)
  ultimately have created-tree xs r t
    using root-lookup unfolding r t 3 by (auto simp: created-tree.simps Let-def)
  with st3 show ?thesis
    unfolding t by simp
qed
qed

lemma hash-replay:
  assumes fmlookup (HashMap s0) x = Some y
    and s0 ≤ s
  shows wp-event (hash x) (λr. case r of None ⇒ False | Some (z, -) ⇒ z = y) s = 1
proof -
  have flip:
    (λr. case r of None ⇒ 0 | Some (z, u) ⇒ if z = y then 1 else 0) =
    (λr. case r of None ⇒ 0 | Some (z, u) ⇒ if y = z then 1 else 0)
  by (rule ext) (auto split: option.splits prod.splits)
  have dist-expect (lift-dist (hash-dist x) s)
    (λr. case r of None ⇒ 0 | Some (z, u) ⇒ if z = y then 1 else 0) = 1
  using hash-expect-known-extension[OF assms]
  unfolding flip .
  then show ?thesis
    unfolding wp-event-def wp-hash lift-dist-def
    by (simp add: dist-expect-map)
qed

lemma singleton-index:
  assumes xs = [x]
    and i < length xs
  shows i = 0 and xs ! i = x
  using assms by (cases i; auto)+

lemma check-created-tree-outcome:
  assumes created-tree xs r s0
    and length xs = 2 ^ n
    and i < length xs
    and v = xs ! i
    and s0 ≤ s
    and Some (h, t) ∈ set-dist (execute
      (check-authentication-path (length xs) i v
        (get-authentication-path (length xs) i r)) s)
  shows h = value r ∧ s ≤ t
  using assms
proof (induction n arbitrary: xs r i v s0 s h t)
  case 0

```

```

have len1: length xs = 1
proof -
  have length xs = (2::nat) ^ 0
    using 0.premis(2) .
  also have ... = 1
    by (rule power-0)
  finally show ?thesis .
qed
obtain x where xs: xs = [x]
proof (cases xs)
  case Nil
  then show ?thesis
    using len1 by simp
next
  case (Cons y ys)
  then have Suc (length ys) = Suc 0
    using len1 by (simp only: length-Cons)
  then have length ys = 0
    by simp
  then have ys = []
    by (simp only: length-0-conv)
  then show ?thesis
    using Cons that by simp
qed
then obtain val where r: r = ⟨⟨⟩, val, ⟨⟩⟩
  and lookup: fmlookup (HashMap s0) x = Some val
  using 0.premis(1) by (auto simp: created-tree.simps)
have i0: i = 0
  using singleton-index(1)[OF xs 0.premis(3)] by assumption
have v-x: v = x
  using 0.premis(4) singleton-index(2)[OF xs 0.premis(3)] by simp
have outcome-hash: Some (h, t) ∈ set-dist (execute (hash x) s)
  using 0.premis(6) unfolding xs r v-x by simp
have s-t: s ≤ t
  by (rule hash-outcome(1)[OF outcome-hash])
have s0-t: s0 ≤ t
  using 0.premis(5) s-t by (meson hash-ext-trans)
have fmlookup (HashMap t) x = Some val
  by (rule hash-extension-lookup[OF lookup s0-t])
moreover have fmlookup (HashMap t) x = Some h
  by (rule hash-outcome(2)[OF outcome-hash])
ultimately show h = value r ∧ s ≤ t
  unfolding r using s-t by auto
next
  case (Suc n)
  let ?m = 2 ^ n
  let ?lxs = take ?m xs
  let ?rxs = drop ?m xs
  have len: length xs = 2 * ?m

```

```

    using Suc.premis(2) by simp
  then have mid: length xs div 2 = ?m
    by simp
  obtain a b cs where xs-cons: xs = a # b # cs
    using len by (cases xs; cases tl xs) auto
  have m-eq:  $2^n = \text{Suc} (\text{length cs div } 2)$ 
    using mid unfolding xs-cons by simp
  have take-mid-eq: ?lxs = a # take (length cs div 2) (b # cs)
    unfolding xs-cons m-eq by simp
  have drop-mid-eq: ?rxs = drop (length cs div 2) (b # cs)
    unfolding xs-cons m-eq by simp
  from Suc.premis(1) obtain l root rr where
    r:  $r = \langle l, \text{root}, \text{rr} \rangle$ 
    and l-created': created-tree (a # take (length cs div 2) (b # cs)) l s0
    and r-created': created-tree (drop (length cs div 2) (b # cs)) rr s0
    and lookup: fmllookup (HashMap s0) (concat (value l) (value rr)) = Some root
    unfolding xs-cons
    by (auto simp: created-tree.simps Let-def mid intro: that)
  have l-created: created-tree ?lxs l s0
    using l-created' unfolding take-mid-eq .
  have r-created: created-tree ?rxs rr s0
    using r-created' unfolding drop-mid-eq .
  have len-l: length ?lxs =  $2^n$ 
    using len by simp
  have len-r: length ?rxs =  $2^n$ 
    using len by simp
  show  $h = \text{value } r \wedge s \leq t$ 
  proof (cases  $i < ?m$ )
    case True
    have v-l:  $v = ?lxs ! i$ 
      using Suc.premis(3,4) True len by (simp add: nth-take)
    have i-l:  $i < \text{length } ?lxs$ 
      using True len-l by simp
    have split-check:  $\text{Some } (h, t) \in \text{set-dist (execute (check-authentication-path (length ?lxs) i v (get-authentication-path (length ?lxs) i l)) \gg (\lambda x. \text{hash (concat x (value rr))})) s}$ 
      using Suc.premis(6) True unfolding r
      by (simp add: mid len-l len)
    from split-check obtain x s1 where
      check-l:  $\text{Some } (x, s1) \in \text{set-dist (execute (check-authentication-path (length ?lxs) i v (get-authentication-path (length ?lxs) i l)) s)}$ 
      and hash-root:  $\text{Some } (h, t) \in \text{set-dist (execute (hash (concat x (value rr))) s1)}$ 
      by (rule set-dist-bindE) (rule that)
    have left-out:  $x = \text{value } l \wedge s \leq s1$ 
      by (rule Suc.IH[OF l-created len-l i-l v-l Suc.premis(5) check-l])
    have x:  $x = \text{value } l$ 
      using left-out by simp
  
```

```

have s-s1:  $s \leq s1$ 
  using left-out by simp
have s1-t:  $s1 \leq t$ 
  using hash-outcome(1)[OF hash-root] .
have s0-t:  $s0 \leq t$ 
  using Suc.prem(5) s-s1 s1-t by (meson hash-ext-trans)
have fmlookup (HashMap t) (concat (value l) (value rr)) = Some root
  by (rule hash-extension-lookup[OF lookup s0-t])
moreover have fmlookup (HashMap t) (concat (value l) (value rr)) = Some h
  using hash-outcome(2)[OF hash-root] unfolding x .
ultimately show  $h = \text{value } r \wedge s \leq t$ 
  unfolding r using s-s1 s1-t by (auto intro: hash-ext-trans)
next
case False
have i-ge:  $?m \leq i$ 
  using False by simp
have i-r:  $i - ?m < \text{length } ?rxs$ 
  using Suc.prem(3) len i-ge by simp
have v-r:  $v = ?rxs ! (i - ?m)$ 
  using Suc.prem(3,4) i-ge len
  by (simp add: nth-drop)
have split-check:  $\text{Some } (h, t) \in \text{set-dist } (\text{execute } (\text{check-authentication-path } (\text{length } ?rxs) (i - ?m) v) (\text{get-authentication-path } (\text{length } ?rxs) (i - ?m) rr)) \gg (\lambda x. \text{hash } (\text{concat } (\text{value } l) x)) s$ 
  using Suc.prem(6) False unfolding r
  by (simp add: mid len-r len)
from split-check obtain x s1 where
  check-r:  $\text{Some } (x, s1) \in \text{set-dist } (\text{execute } (\text{check-authentication-path } (\text{length } ?rxs) (i - ?m) v) (\text{get-authentication-path } (\text{length } ?rxs) (i - ?m) rr)) s$ 
  and hash-root:  $\text{Some } (h, t) \in \text{set-dist } (\text{execute } (\text{hash } (\text{concat } (\text{value } l) x)) s1)$ 
  by (rule set-dist-bindE) (rule that)
have right-out:  $x = \text{value } rr \wedge s \leq s1$ 
  by (rule Suc.IH[OF r-created len-r i-r v-r Suc.prem(5) check-r])
have x:  $x = \text{value } rr$ 
  using right-out by simp
have s-s1:  $s \leq s1$ 
  using right-out by simp
have s1-t:  $s1 \leq t$ 
  using hash-outcome(1)[OF hash-root] .
have s0-t:  $s0 \leq t$ 
  using Suc.prem(5) s-s1 s1-t by (meson hash-ext-trans)
have fmlookup (HashMap t) (concat (value l) (value rr)) = Some root
  by (rule hash-extension-lookup[OF lookup s0-t])
moreover have fmlookup (HashMap t) (concat (value l) (value rr)) = Some h
  using hash-outcome(2)[OF hash-root] unfolding x .
ultimately show  $h = \text{value } r \wedge s \leq t$ 
  unfolding r using s-s1 s1-t by (auto intro: hash-ext-trans)

```

qed
qed

lemma *check-created-tree*:

assumes *created-tree xs r s0*

and $\text{length } xs = 2^n$

and $i < \text{length } xs$

and $v = xs ! i$

and $s0 \leq s$

and $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (\text{check-authentication-path } (\text{length } xs) i v) (\text{get-authentication-path } (\text{length } xs) i r)) s))$

shows *wp-event*

$(\text{check-authentication-path } (\text{length } xs) i v$

$(\text{get-authentication-path } (\text{length } xs) i r)) s)$

$(\lambda \text{out. case out of None} \Rightarrow \text{False} \mid \text{Some } (r', -) \Rightarrow r' = \text{value } r) s = 1$

unfolding *wp-event-def wp-def*

proof (*rule dist-expect-eq-1*)

fix *p*

assume $p: p \in \text{dom } (\text{dist } (\text{execute } (\text{check-authentication-path } (\text{length } xs) i v)$

$(\text{get-authentication-path } (\text{length } xs) i r)) s))$

obtain *a b* **where** *p-eq*: $p = \text{Some } (a, b)$

using *p assms(6)*

by (*cases p*) *auto*

have *support*: $\text{Some } (a, b) \in \text{set-dist } (\text{execute } (\text{check-authentication-path } (\text{length } xs) i v)$

$(\text{get-authentication-path } (\text{length } xs) i r)) s)$

using *p unfolding p-eq set-dist-def* .

then have *a-eq*: $a = \text{value } r$

using *check-created-tree-outcome[OF assms(1-5) support]* **by** *simp*

have *goal*: $(\text{if } (\text{case } p \text{ of None} \Rightarrow \text{False} \mid \text{Some } (r', -) \Rightarrow r' = \text{value } r) \text{ then } 1 \text{ else } 0) = 1$

using *a-eq* **by** (*simp add: p-eq*)

from *goal* **show** $(\text{if } (\text{case } p \text{ of None} \Rightarrow \text{False} \mid \text{Some } (r', -) \Rightarrow r' = \text{value } r) \text{ then } 1 \text{ else } 0) = 1$.

1 .

qed

proposition *create-check*:

assumes $\text{length } x = 2^n$

and $i < \text{length } x$

and $v = x ! i$

and $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (\text{do } \{$

$r \leftarrow \text{create } x;$

$\text{let } ap = \text{get-authentication-path } (\text{length } x) i r;$

$r2 \leftarrow \text{check-authentication-path } (\text{length } x) i v ap;$

$\text{return } (r2 = \text{value } r)$

$\}) (\text{HashMap} = \text{fmempty}))$

shows $1 \leq \text{wp-event}$


```

(do {
  r ← create x;
  let ap = get-authentication-path (length x) i r;
  r2 ← check-authentication-path (length x) i v ap;
  return (r2 = value r)
}) (λout. case out of None ⇒ False | Some (r, -) ⇒ r) (HashMap = fmempty)
proof -
let ?s0 = (HashMap = fmempty)
let ?m = do {
  r ← create x;
  let ap = get-authentication-path (length x) i r;
  r2 ← check-authentication-path (length x) i v ap;
  return (r2 = value r)
}
have wp-event ?m (λout. case out of None ⇒ False | Some (r, -) ⇒ r) ?s0 = 1
  unfolding wp-event-def wp-def
proof (rule dist-expect-eq-1)
fix p
assume p: p ∈ dom (dist (execute ?m ?s0))
obtain ok sf where p-eq: p = Some (ok, sf)
  using p assms(4)
  by (cases p) auto
have p-set: Some (ok, sf) ∈ set-dist (execute ?m ?s0)
  using p unfolding p-eq set-dist-def .
from p-set obtain r s1 where
  create-r: Some (r, s1) ∈ set-dist (execute (create x) ?s0)
  and rest: Some (ok, sf) ∈ set-dist (execute
    (let ap = get-authentication-path (length x) i r in
      check-authentication-path (length x) i v ap >>=
        (λr2. return (r2 = value r))) s1)
  by (rule set-dist-bindE) (rule that)
from rest obtain r2 s2 where
  check-r: Some (r2, s2) ∈ set-dist (execute
    (check-authentication-path (length x) i v
      (get-authentication-path (length x) i r)) s1)
  and ret: Some (ok, sf) ∈ set-dist (execute (return (r2 = value r)) s2)
  by (auto simp: Let-def elim!: set-dist-bindE)
have created: created-tree x r s1
  using create-outcome[OF create-r] by simp
have r2 = value r
  using check-created-tree-outcome[
    OF created assms(1-3) hash-ext-refl check-r] by simp
with ret have ok: ok
  by auto
have goal: (if (case p of None ⇒ False | Some (r, -) ⇒ r) then 1 else 0) = 1
  using ok by (simp add: p-eq)
from goal show (if (case p of None ⇒ False | Some (r, -) ⇒ r) then 1 else 0) = 1 .
qed
then show ?thesis

```

by simp
qed

end

definition *protocol-created-tree*
 $:: 'a::\text{finite list} \Rightarrow 'a \text{ tree} \Rightarrow$
 $('a \text{ protocol-hash-input}, 'a, 'c) \text{ hash-scheme} \Rightarrow \text{bool}$
where
 $\text{protocol-created-tree } xs \ r \ s \longleftrightarrow$
 $\text{protocol-merkle.created-tree } (\text{map MerkleLeaf } xs) \ r \ s$

lemma *protocol-created-tree-mono*:
assumes *protocol-created-tree* $xs \ r \ s0$
and $s0 \leq s$
shows *protocol-created-tree* $xs \ r \ s$
using *assms*
unfolding *protocol-created-tree-def*
by (rule *protocol-merkle.created-tree-mono*)

lemma *protocol-create-outcome*:
assumes $\text{Some } (r, t) \in \text{set-dist } (\text{execute } (\text{protocol-create } xs) \ s)$
shows $s \leq t \wedge \text{protocol-created-tree } xs \ r \ t$
proof –
have *raw*:
 $\text{Some } (r, t) \in$
 $\text{set-dist } (\text{execute } (\text{protocol-merkle.create } (\text{map MerkleLeaf } xs)) \ s)$
using *assms* **unfolding** *protocol-create-def* .
show ?thesis
using *protocol-merkle.create-outcome*[OF *raw*]
unfolding *protocol-created-tree-def* **by** *simp*
qed

lemma *protocol-check-created-tree-outcome*:
assumes *created*: *protocol-created-tree* $xs \ r \ s0$
and *length-xs*: $\text{length } xs = 2^n$
and *i-bound*: $i < \text{length } xs$
and *v*: $v = xs ! i$
and *ext*: $s0 \leq s$
and *check*:
 $\text{Some } (r2, t) \in \text{set-dist } (\text{execute}$
 $(\text{protocol-check-authentication-path } (\text{length } xs) \ i \ v$
 $(\text{get-authentication-path } (\text{length } xs) \ i \ r)) \ s)$
shows $r2 = \text{value } r \wedge s \leq t$
proof –
have *raw-created*:
 $\text{protocol-merkle.created-tree } (\text{map MerkleLeaf } xs) \ r \ s0$
using *created* **unfolding** *protocol-created-tree-def* .
have *raw-check*:

```

Some (r2, t) ∈ set-dist (execute
  (protocol-merkle.check-authentication-path
    (length (map MerkleLeaf xs)) i (MerkleLeaf v)
    (get-authentication-path (length (map MerkleLeaf xs)) i r)) s)
using check unfolding protocol-check-authentication-path-def by simp
have leaf-eq: MerkleLeaf v = map MerkleLeaf xs ! i
using i-bound v by simp
show ?thesis
by (rule protocol-merkle.check-created-tree-outcome
  [OF raw-created - - leaf-eq ext raw-check])
  (use length-xs i-bound in simp-all)
qed

```

1.31.1 Length

```

context merkle-tree
begin

```

```

lemma create-get-authentication-path-len:
  assumes Some (r, t) ∈ set-dist (execute (create xs) s)
    and 0 < lgth
    and lgth ≤ length xs
  shows length (get-authentication-path lgth idx r) = floor-log lgth
  using assms
proof (induction xs arbitrary: r s t lgth idx rule: measure-induct-rule[of length])
  case (less xs)
  show ?case
  proof (cases xs)
  case Nil
  then show ?thesis
    using less.premis by simp
  next
  case (Cons x ys)
  have xs-Cons: xs = x # ys
    using Cons by simp
  show ?thesis
  proof (cases ys)
  case Nil
  then obtain h s1 where r: r = ⟨⟨⟩, h, ⟨⟩⟩
    using less.premis xs-Cons by (auto elim!: set-dist-bindE)
  have lgth = 1
    using less.premis xs-Cons Nil by simp
  then show ?thesis
    unfolding r by (simp add: floor-log-Suc-zero)
  next
  case (Cons y zs)
  let ?i = length xs div 2
  have xs-eq: xs = x # y # zs
    using xs-Cons ⟨ys = y # zs⟩ by simp

```

```

have len-ge:  $2 \leq \text{length } xs$ 
  using xs-eq by simp
have take-eq:  $\text{take } ?i \, xs = x \# \text{take } (\text{length } zs \, \text{div } 2) \, (y \# zs)$ 
  unfolding xs-eq by simp
have drop-eq:  $\text{drop } ?i \, xs = \text{drop } (\text{length } zs \, \text{div } 2) \, (y \# zs)$ 
  unfolding xs-eq by simp
have split-create:  $\text{Some } (r, t) \in \text{set-dist } (\text{execute } (\text{create } (\text{take } ?i \, xs) \gg=$ 
   $(\lambda l. \text{create } (\text{drop } ?i \, xs) \gg=$ 
   $(\lambda rn. \text{hash } (\text{concat } (\text{value } l) (\text{value } rn)) \gg=$ 
   $(\lambda val. \text{return } \langle l, val, rn \rangle))) s)$ 
  using less.premis(1) unfolding xs-eq take-eq drop-eq by simp
from split-create obtain l s1 where
  l:  $\text{Some } (l, s1) \in \text{set-dist } (\text{execute } (\text{create } (\text{take } ?i \, xs)) s)$ 
  and rest1:  $\text{Some } (r, t) \in$ 
   $\text{set-dist } (\text{execute } (\text{create } (\text{drop } ?i \, xs) \gg=$ 
   $(\lambda rn. \text{hash } (\text{concat } (\text{value } l) (\text{value } rn)) \gg=$ 
   $(\lambda val. \text{return } \langle l, val, rn \rangle))) s1)$ 
  by (rule set-dist-bindE) (rule that)
from rest1 obtain rr s2 where
  rr:  $\text{Some } (rr, s2) \in \text{set-dist } (\text{execute } (\text{create } (\text{drop } ?i \, xs)) s1)$ 
  and rest2:  $\text{Some } (r, t) \in$ 
   $\text{set-dist } (\text{execute } (\text{hash } (\text{concat } (\text{value } l) (\text{value } rr)) \gg=$ 
   $(\lambda val. \text{return } \langle l, val, rr \rangle))) s2)$ 
  by (rule set-dist-bindE) (rule that)
from rest2 obtain val s3 where
  val:  $\text{Some } (val, s3) \in \text{set-dist } (\text{execute } (\text{hash } (\text{concat } (\text{value } l) (\text{value } rr)))) s2)$ 
  and ret:  $\text{Some } (r, t) \in \text{set-dist } (\text{execute } (\text{return } \langle l, val, rr \rangle) s3)$ 
  by (auto elim!: set-dist-bindE)
have r:  $r = \langle l, val, rr \rangle$ 
  using ret by simp
show ?thesis
proof (cases lgth = 1)
  case True
  then show ?thesis
    unfolding r by (simp add: floor-log-Suc-zero)
next
  case False
  have lgth-ge:  $2 \leq \text{lgth}$ 
    using False less.premis by simp
  have half-pos:  $0 < \text{lgth div } 2$ 
    using lgth-ge by simp
  have take-less:  $\text{length } (\text{take } ?i \, xs) < \text{length } xs$ 
    using len-ge by simp
  have drop-less:  $\text{length } (\text{drop } ?i \, xs) < \text{length } xs$ 
    using len-ge by simp
  have half-le-i:  $\text{lgth div } 2 \leq ?i$ 

```

```

    using less.prem3 by simp
  have half-le-take: lgth div 2 ≤ length (take ?i xs)
    using half-le-i by simp
  have half-le-drop: lgth div 2 ≤ length (drop ?i xs)
    using half-le-i len-ge by simp
  have left-len:
    length (get-authentication-path (lgth div 2) idx l) = floor-log (lgth div 2)
    by (rule less.IH[OF take-less l half-pos half-le-take])
  have right-len:
    length (get-authentication-path (lgth div 2) (idx - lgth div 2) rr) =
      floor-log (lgth div 2)
    by (rule less.IH[OF drop-less rr half-pos half-le-drop])
  have floor-eq: floor-log lgth = Suc (floor-log (lgth div 2))
    using floor-log-rec[OF lgth-ge] .
  show ?thesis
  proof (cases idx < lgth div 2)
    case True
    have length (get-authentication-path lgth idx r) =
      Suc (floor-log (lgth div 2))
      using False True left-len unfolding r by simp
    then show ?thesis
      by (simp only: floor-eq)
    next
    case False
    have length (get-authentication-path lgth idx r) =
      Suc (floor-log (lgth div 2))
      using <lgth ≠ 1> False right-len unfolding r by simp
    then show ?thesis
      by (simp only: floor-eq)
  qed
qed
qed
qed
qed
qed

lemma get-authentication-path-len:
  assumes execute (create xs) s = d
    and x ∈ dom (dist d)
    and x = Some (r, h)
    and xs ≠ []
  shows length (get-authentication-path (length xs) idx r) = floor-log (length xs)
proof -
  have support: Some (r, h) ∈ set-dist (execute (create xs) s)
    using assms(1-3) unfolding set-dist-def by simp
  show ?thesis
    by (rule create-get-authentication-path-len[OF support])
      (use assms(4) in simp-all)
qed

```

end

end

```
theory Prob-Monad
  imports
    Prob-Dist
begin
```

1.32 Probabilistic State Monad

This theory defines the finite probabilistic state monad used throughout the formalization. It includes monadic combinators, failure, assertions, and a weakest-precondition calculus for probabilistic events.

```
typedef ('a, 's) state-monad = UNIV::('s  $\Rightarrow$  (('a  $\times$  's) option) dist) set
  morphisms execute create
  by simp
```

```
lemma execute-create: execute (create f) = f using create-inverse by simp
```

```
setup-lifting type-definition-state-monad
```

1.32.1 Bind

```
definition map-bind :: ('s  $\Rightarrow$  ('a  $\rightarrow$  prob))  $\Rightarrow$  ('a  $\Rightarrow$  ('b  $\rightarrow$  prob))  $\Rightarrow$  ('s  $\Rightarrow$  ('b  $\rightarrow$  prob))
```

```
where
```

```
  map-bind m k s =
    ( $\lambda$ b.
      let p =
        ( $\sum_{i \in \text{dom } (m \ s)}$ 
          (the (m s i))
            * (option-default (k i b)))
        in if p = 0 then None else Some p)
```

```
lemma map-bind-nonzero-on-dom:
```

```
  assumes  $x \in \text{dom } (\text{map-bind } m \ k \ s)$ 
```

```
  shows the (map-bind m k s x)  $\neq$  0
```

```
proof -
```

```
  show ?thesis
```

```
  using assms
```

```
  unfolding map-bind-def
```

```
  by (auto simp: Let-def split: if-splits)
```

```
qed
```

```
lemma option-default-map-bind:
```

```
  option-default (map-bind m k s x) =
```

```
    ( $\sum_{i \in \text{dom } (m \ s)}$ 
      the (m s i) * option-default (k i x))
```

```

unfolding map-bind-def by (simp add: Let-def)

lemma map-bind-eq-from-option-default:
  assumes
    option-default (map-bind m k s x) =
      option-default (map-bind m' k' s' x)
  shows map-bind m k s x = map-bind m' k' s' x
proof -
  show ?thesis
    using assms
    unfolding map-bind-def
    by (simp add: Let-def)
qed

lemma map-bind-cong:
  assumes m-eq: m s = m' s
  assumes k-eq:  $\bigwedge i. i \in \text{dom } (m s) \implies k i = k' i$ 
  shows map-bind m k s = map-bind m' k' s
proof
  fix x
  have dom-eq:  $\text{dom } (m' s) = \text{dom } (m s)$ 
    using m-eq by simp
  show map-bind m k s x = map-bind m' k' s x
    unfolding map-bind-def Let-def
    using m-eq dom-eq k-eq
    by (intro if-cong refl arg-cong[where f=Some] sum.cong) auto
qed

lemma map-bind-cong-state:
  assumes m-eq: m s = m' s'
  assumes k-eq:  $\bigwedge i. i \in \text{dom } (m s) \implies k i = k' i$ 
  shows map-bind m k s = map-bind m' k' s'
proof
  fix x
  have dom-eq:  $\text{dom } (m' s') = \text{dom } (m s)$ 
    using m-eq by simp
  show map-bind m k s x = map-bind m' k' s' x
    unfolding map-bind-def Let-def
    using m-eq dom-eq k-eq
    by (intro if-cong refl arg-cong[where f=Some] sum.cong) auto
qed

lemma dom-map-bind:
   $\text{dom } (\text{map-bind } m k s) =$ 
   $\{x \in (\bigcup i \in \text{dom } (m s). \text{dom } (k i)).$ 
   $(\sum i \in \text{dom } (m s).$ 
   $\text{the } (m s i) * \text{option-default } (k i x)) \neq 0\}$ 
proof -
  let ?M =  $\text{dom } (m s)$ 

```

```

let ?K = λi. k i
let ?D = ⋃ i∈?M. dom (?K i)
let ?p = λx. ∑ i∈?M. the (m s i) * option-default (?K i x)

have outside-zero: x ∉ ?D ⇒ ?p x = 0 for x
proof -
  assume x: x ∉ ?D
  have ∧i. i ∈ ?M ⇒ option-default (?K i x) = 0
    using x by (simp add: domIff)
  then show ?p x = 0
    by simp
qed

show ?thesis
proof
  show dom (map-bind m k s) ⊆ {x ∈ ?D. ?p x ≠ 0}
  proof
    fix x
    assume x-dom: x ∈ dom (map-bind m k s)

    have px-ne: ?p x ≠ 0
      using x-dom
      unfolding map-bind-def
      by (fastforce simp: Let-def split: if-splits)

    moreover have x ∈ ?D
      using px-ne outside-zero by blast

    ultimately show x ∈ {x ∈ ?D. ?p x ≠ 0}
      by simp
    qed
  next
  show {x ∈ ?D. ?p x ≠ 0} ⊆ dom (map-bind m k s)
  proof
    fix x
    assume x: x ∈ {x ∈ ?D. ?p x ≠ 0}

    show x ∈ dom (map-bind m k s)
      using x
      unfolding map-bind-def
      apply (simp add: Let-def) by auto
    qed
  qed
qed

```

```

lemma sum-option-default-superset:
  assumes fin-D: finite D
  assumes dom-subset: dom f ⊆ D
  shows (∑ x∈D. option-default (f x)) = sum-map f

```



```

proof –
  have  $(\sum_{x \in D}. \text{option-default } (f \ x)) =$ 
     $(\sum_{x \in D \cap \text{dom } f}. \text{option-default } (f \ x))$ 
    using fin-D
    apply (intro sum.mono-neutral-right)
    apply auto
    by (metis not-None-eq option.case-eq-if)
  also have  $\dots = (\sum_{x \in \text{dom } f}. \text{option-default } (f \ x))$ 
    using dom-subset by (simp add: inf.absorb2)
  also have  $\dots = (\sum_{x \in \text{dom } f}. \text{the } (f \ x))$ 
    by (intro sum.cong) auto
  finally show ?thesis
    unfolding sum-map-def .
qed

```

```

lemma sum-option-default-mult-superset:
  assumes fin-D: finite D
  assumes dom-subset:  $\text{dom } f \subseteq D$ 
  shows  $(\sum_{x \in D}. \text{option-default } (f \ x) * g \ x) =$ 
     $(\sum_{x \in \text{dom } f}. \text{the } (f \ x) * g \ x)$ 
proof –
  have  $(\sum_{x \in D}. \text{option-default } (f \ x) * g \ x) =$ 
     $(\sum_{x \in \text{dom } f}. \text{option-default } (f \ x) * g \ x)$ 
    using fin-D dom-subset
    by (intro sum.mono-neutral-right) (auto simp: dom-def)
  also have  $\dots = (\sum_{x \in \text{dom } f}. \text{the } (f \ x) * g \ x)$ 
    by (intro sum.cong) auto
  finally show ?thesis .
qed

```

```

lemma the-map-bind-on-dom:
  assumes  $x \in \text{dom } (\text{map-bind } m \ k \ s)$ 
  shows
     $\text{the } (\text{map-bind } m \ k \ s \ x) =$ 
       $(\sum_{i \in \text{dom } (m \ s)}. \text{the } (m \ s \ i) * \text{option-default } (k \ i \ x))$ 
proof –
  show ?thesis
    using assms
    unfolding map-bind-def
    by (fastforce simp: Let-def split: if-splits)
qed

```

```

lemma finite-dom-bind:
  assumes fin-m: finite (dom (m s))
  assumes fin-k:  $\forall i \in \text{dom } (m \ s). \text{finite } (\text{dom } (k \ i))$ 
  shows finite (dom (map-bind m k s))
proof –
  let  $?D = \bigcup_{i \in \text{dom } (m \ s)}. \text{dom } (k \ i)$ 

```

```

have fin-D: finite ?D
  using fin-m fin-k by auto

have finite
  {x ∈ ?D.
    (∑ i ∈ dom (m s).
      the (m s i) * option-default (k i x)) ≠ 0}
  using fin-D fin-m by simp

then show ?thesis
  by (simp add: dom-map-bind)
qed

lemma sum-map-bind-1:
  assumes fin-m: finite (dom (m s))
  assumes fin-k: ∀ i ∈ dom (m s). finite (dom (k i))
  assumes sum-m: sum-map (m s) = 1
  assumes sum-k: ∀ i ∈ dom (m s). sum-map (k i) = 1
  shows sum-map (map-bind m k s) = 1
proof -
  let ?M = dom (m s)
  let ?K = λ i. k i
  let ?D = ⋃ i ∈ ?M. dom (?K i)
  let ?p = λ x. ∑ i ∈ ?M. the (m s i) * option-default (?K i x)

  have fin-D: finite ?D
    using fin-m fin-k by auto

  have sum-map (map-bind m k s) =
    (∑ x ∈ dom (map-bind m k s). the (map-bind m k s x))
  unfolding sum-map-def by simp

  also have ... = (∑ x ∈ dom (map-bind m k s). ?p x)
    by (intro sum.cong) (simp-all add: the-map-bind-on-dom)

  also have ... = (∑ x ∈ {x ∈ ?D. ?p x ≠ 0}. ?p x)
    by (simp add: dom-map-bind)

  also have ... = (∑ x ∈ ?D. ?p x)
    using fin-D
    apply (intro sum.mono-neutral-cong-left) using assms(1) by auto

  also have ... = (∑ i ∈ ?M. ∑ x ∈ ?D.
    the (m s i) * option-default (?K i x))
    by (rule sum.swap)

  also have ... = (∑ i ∈ ?M.
    the (m s i) * (∑ x ∈ ?D. option-default (?K i x)))

```

```

    by (simp add: sum-distrib-left)

also have ... = ( $\sum i \in ?M. \text{the } (m \ s \ i) * \text{sum-map } (?K \ i)$ )
proof (intro sum.cong refl)
  fix i
  assume i:  $i \in ?M$ 
  have dom  $(?K \ i) \subseteq ?D$ 
    using i by auto
  then show
     $\text{the } (m \ s \ i) * (\sum x \in ?D. \text{option-default } (?K \ i \ x)) =$ 
     $\text{the } (m \ s \ i) * \text{sum-map } (?K \ i)$ 
    using sum-option-default-superset[OF fin-D, of  $?K \ i$ ]
    by simp
qed

also have ... = ( $\sum i \in ?M. \text{the } (m \ s \ i) * 1$ )
  using sum-k by simp

also have ... = sum-map  $(m \ s)$ 
  unfolding sum-map-def by simp

also have ... = 1
  using sum-m .

finally show ?thesis .
qed

lemma map-bind-delta-left:
  fixes k :: 'a  $\Rightarrow$  'b  $\rightarrow$  prob
  assumes clean:  $\bigwedge x. x \in \text{dom } (k \ a) \implies \text{the } (k \ a \ x) \neq 0$ 
  shows map-bind  $(\lambda-. \text{delta-map } a) \ k \ s = k \ a$ 
proof
  fix x :: 'b
  show map-bind  $(\lambda-. \text{delta-map } a) \ k \ s \ x = k \ a \ x$ 
  proof (cases k a x)
    case None
    then show ?thesis
      unfolding map-bind-def delta-map-def
      by (simp add: Let-def)
  next
    case (Some p)
    then have  $p \neq 0$ 
      using clean[of x] by auto
    then show ?thesis
      unfolding map-bind-def delta-map-def
      using Some by (simp add: Let-def)
  qed
qed

```

```

lemma map-bind-delta-left-state:
  fixes  $k :: 'a \Rightarrow 'b \rightarrow \text{prob}$ 
  assumes clean:  $\bigwedge x. x \in \text{dom } (k \text{ (f s)}) \implies \text{the } (k \text{ (f s)} x) \neq 0$ 
  shows map-bind  $(\lambda t. \text{delta-map } (f t))$   $k s = k \text{ (f s)}$ 
proof
  fix  $x :: 'b$ 
  show map-bind  $(\lambda t. \text{delta-map } (f t))$   $k s x = k \text{ (f s)} x$ 
  proof (cases  $k \text{ (f s)} x$ )
    case None
    then show ?thesis
      unfolding map-bind-def delta-map-def
      by (simp add: Let-def)
    next
    case (Some  $p$ )
    then have  $p \neq 0$ 
      using clean[of  $x$ ] by auto
    then show ?thesis
      unfolding map-bind-def delta-map-def
      using Some by (simp add: Let-def)
  qed
qed

lemma map-bind-delta-right:
  fixes  $m :: 's \Rightarrow 'a \rightarrow \text{prob}$ 
  assumes fin: finite (dom ( $m s$ ))
  assumes clean:  $\bigwedge x. x \in \text{dom } (m s) \implies \text{the } (m s x) \neq 0$ 
  shows map-bind  $m (\lambda a. \text{delta-map } a)$   $s = m s$ 
proof
  fix  $x :: 'a$ 
  have sum-delta:
     $(\sum_{i \in \text{dom } (m s)}. \text{the } (m s i) * \text{option-default } (\text{delta-map } i x)) =$ 
     $(\text{if } x \in \text{dom } (m s) \text{ then the } (m s x) \text{ else } 0)$ 
  proof –
    have  $(\sum_{i \in \text{dom } (m s)}. \text{the } (m s i) * \text{option-default } (\text{delta-map } i x)) =$ 
     $(\sum_{i \in \text{dom } (m s)}. \text{if } i = x \text{ then the } (m s i) \text{ else } 0)$ 
    by (intro sum.cong refl) (simp add: delta-map-def)
    also have ... =  $(\text{if } x \in \text{dom } (m s) \text{ then the } (m s x) \text{ else } 0)$ 
    using fin by (simp add: sum.delta)
    finally show ?thesis .
  qed
  show map-bind  $m (\lambda a. \text{delta-map } a)$   $s x = m s x$ 
  proof (cases  $x \in \text{dom } (m s)$ )
    case True
    then have  $m s x = \text{Some } (\text{the } (m s x))$ 
    by auto

```

```

moreover have the (m s x) ≠ 0
  using True by (rule clean)
ultimately show ?thesis
  unfolding map-bind-def
  using sum-delta True by (simp add: Let-def)
next
  case False
  then have m s x = None
    by auto
  then show ?thesis
    unfolding map-bind-def
    using sum-delta False by (simp add: Let-def)
qed
qed

lemma map-bind-assoc:
  fixes m :: 's ⇒ 'a → prob
    and k :: 'a ⇒ 'b → prob
    and h :: 'b ⇒ 'c → prob
  assumes fin-m: finite (dom (m s))
  assumes fin-k:  $\bigwedge i. i \in \text{dom } (m s) \implies \text{finite } (\text{dom } (k i))$ 
  shows
    map-bind (map-bind m k) h s =
      map-bind m ( $\lambda a. \text{map-bind } (\lambda -. k a) h ()$ ) s
proof
  fix x :: 'c

  let ?M = dom (m s)
  let ?K =  $\lambda i. k i$ 
  let ?D =  $\bigcup i \in ?M. \text{dom } (?K i)$ 
  let ?H =  $\lambda j. \text{option-default } (h j x)$ 
  let ?p =  $\lambda j. \sum i \in ?M. \text{the } (m s i) * \text{option-default } (?K i j)$ 

  have fin-D: finite ?D
    using fin-m fin-k by auto

  have left-sum:
    ( $\sum j \in \text{dom } (\text{map-bind } m k s). \text{the } (\text{map-bind } m k s j) * ?H j$ ) =
    ( $\sum j \in ?D. ?p j * ?H j$ )
  proof -
    have ( $\sum j \in \text{dom } (\text{map-bind } m k s). \text{the } (\text{map-bind } m k s j) * ?H j$ ) =
      ( $\sum j \in \{j \in ?D. ?p j \neq 0\}. \text{the } (\text{map-bind } m k s j) * ?H j$ )
      by (simp add: dom-map-bind)
    also have ... =
      ( $\sum j \in \{j \in ?D. ?p j \neq 0\}. ?p j * ?H j$ )
      by (intro sum.cong refl) (simp add: dom-map-bind the-map-bind-on-dom)

```

```

also have ... = (∑ j ∈ ?D. ?p j * ?H j)
  using fin-D
  by (intro sum.mono-neutral-left) auto
finally show ?thesis .
qed

have swap-sum:
  (∑ j ∈ ?D. ?p j * ?H j) =
    (∑ i ∈ ?M.
      the (m s i) *
        (∑ j ∈ dom (?K i).
          the (?K i j) * ?H j))
proof -
  have (∑ j ∈ ?D. ?p j * ?H j) =
    (∑ j ∈ ?D. ∑ i ∈ ?M.
      the (m s i) * option-default (?K i j) * ?H j)
    by (simp add: sum-distrib-left algebra-simps)
  also have ... =
    (∑ i ∈ ?M. ∑ j ∈ ?D.
      the (m s i) * option-default (?K i j) * ?H j)
    by (rule sum.swap)
  also have ... =
    (∑ i ∈ ?M.
      the (m s i) *
        (∑ j ∈ ?D. option-default (?K i j) * ?H j))
    by (simp add: sum-distrib-left algebra-simps)
  also have ... =
    (∑ i ∈ ?M.
      the (m s i) *
        (∑ j ∈ dom (?K i). the (?K i j) * ?H j))
  proof (intro sum.cong refl)
    fix i
    assume i: i ∈ ?M
    have dom (?K i) ⊆ ?D
      using i by auto
    then show
      the (m s i) *
        (∑ j ∈ ?D. option-default (?K i j) * ?H j) =
        the (m s i) *
          (∑ j ∈ dom (?K i). the (?K i j) * ?H j)
      using sum-option-default-mult-superset[OF fin-D, of ?K i ?H]
      by simp
    qed
  finally show ?thesis .
qed

have sums-eq:
  (∑ j ∈ dom (map-bind m k s).
    the (map-bind m k s j) * ?H j) =

```

```

  (∑ i ∈ ?M.
    the (m s i) *
      option-default (map-bind (λ-. k i) h () x))
proof -
  have (∑ j ∈ dom (map-bind m k s).
    the (map-bind m k s j) * ?H j) =
    (∑ i ∈ ?M.
      the (m s i) *
        (∑ j ∈ dom (?K i). the (?K i j) * ?H j))
    using left-sum swap-sum by simp
  also have ... =
    (∑ i ∈ ?M.
      the (m s i) *
        option-default (map-bind (λ-. k i) h () x))
    by (intro sum.cong refl) (simp add: option-default-map-bind)
  finally show ?thesis .
qed

```

```

show
  map-bind (map-bind m k) h s x =
    map-bind m (λa. map-bind (λ-. k a) h ()) s x
proof (rule map-bind-eq-from-option-default)
  show
    option-default (map-bind (map-bind m k) h s x) =
      option-default (map-bind m (λa. map-bind (λ-. k a) h ()) s x)
    using sums-eq
    by (simp add: option-default-map-bind)
  qed
qed

```

```

definition bind-cont-map ::
  ('a ⇒ 's ⇒ (('b × 's) option → prob)) ⇒
  ('a × 's) option ⇒ (('b × 's) option → prob)
where
  bind-cont-map k r =
    (case r of
      None ⇒ delta-map None
    | Some (a, s') ⇒ k a s')

```

```

lemma bind-cont-map-return:
  bind-cont-map (λa s. delta-map (Some (a, s))) r = delta-map r
unfolding bind-cont-map-def
by (cases r) auto

```

```

lemma finite-dom-bind-cont-mapI:
  assumes ⋀ a s. finite (dom (k a s))
  shows finite (dom (bind-cont-map k r))
  using assms
  unfolding bind-cont-map-def

```

```

by (cases r) (auto simp: delta-map-def split: prod.splits)

lemma sum-map-bind-cont-mapI:
  assumes  $\bigwedge a s. \text{sum-map } (k a s) = 1$ 
  shows  $\text{sum-map } (\text{bind-cont-map } k r) = 1$ 
  using assms
  unfolding bind-cont-map-def sum-map-def
  by (cases r) (auto simp: delta-map-def split: prod.splits)

lift-definition dist-bind ::
  ( $'s \Rightarrow (('a \times 's) \text{ option}) \text{ dist}$ )  $\Rightarrow$ 
  ( $'a \Rightarrow ('s \Rightarrow (('b \times 's) \text{ option}) \text{ dist})$ )  $\Rightarrow$ 
  ( $'s \Rightarrow (('b \times 's) \text{ option}) \text{ dist}$ )
is  $\lambda m k. \text{map-bind } m (\text{bind-cont-map } k)$ 
proof -
  fix m ::  $'s \Rightarrow (('a \times 's) \text{ option} \rightarrow \text{prob})$ 
  and k ::  $'a \Rightarrow 's \Rightarrow (('b \times 's) \text{ option} \rightarrow \text{prob})$ 
  and s ::  $'s$ 
  assume m:
     $\bigwedge s. \text{finite } (\text{dom } (m s)) \wedge \text{sum-map } (m s) = 1 \wedge$ 
     $(\forall x \in \text{dom } (m s). \text{the } (m s x) \neq 0)$ 
  assume k:
     $\bigwedge a s. \text{finite } (\text{dom } (k a s)) \wedge \text{sum-map } (k a s) = 1 \wedge$ 
     $(\forall x \in \text{dom } (k a s). \text{the } (k a s x) \neq 0)$ 
  show  $\text{finite } (\text{dom } (\text{map-bind } m (\text{bind-cont-map } k) s)) \wedge$ 
     $\text{sum-map } (\text{map-bind } m (\text{bind-cont-map } k) s) = 1 \wedge$ 
     $(\forall x \in \text{dom } (\text{map-bind } m (\text{bind-cont-map } k) s).$ 
       $\text{the } (\text{map-bind } m (\text{bind-cont-map } k) s x) \neq 0)$ 
  proof (intro conjI ballI)
    show  $\text{finite } (\text{dom } (\text{map-bind } m (\text{bind-cont-map } k) s))$ 
    by (rule finite-dom-bind) (use m k in
       $\langle \text{auto intro!}; \text{finite-dom-bind-cont-mapI simp: bind-cont-map-def delta-map-def split: option.splits prod.splits} \rangle$ )
    show  $\text{sum-map } (\text{map-bind } m (\text{bind-cont-map } k) s) = 1$ 
    by (rule sum-map-bind-1) (use m k in
       $\langle \text{auto intro!}; \text{sum-map-bind-cont-mapI simp: bind-cont-map-def delta-map-def sum-map-def split: option.splits prod.splits} \rangle$ )
    fix x
    assume  $x \in \text{dom } (\text{map-bind } m (\text{bind-cont-map } k) s)$ 
    then show  $\text{the } (\text{map-bind } m (\text{bind-cont-map } k) s x) \neq 0$ 
    by (rule map-bind-nonzero-on-dom)
  qed
qed

lift-definition
  sm-bind :: ( $'a, 's$ )  $\text{state-monad} \Rightarrow ('a \Rightarrow ('b, 's) \text{state-monad}) \Rightarrow ('b, 's) \text{state-monad}$ 
(infixl >>= 60)
is dist-bind .

```


1.32.2 Return

definition $\text{dist-return} :: 'a \Rightarrow ('s \Rightarrow (('a \times 's) \text{ option}) \text{ dist})$
where $\text{dist-return } a = (\lambda s. \text{delta-dist } (\text{Some } (a, s)))$

lift-definition $\text{return} :: 'a \Rightarrow ('a, 's) \text{ state-monad}$
is dist-return .

definition $\text{dist-throw} :: 's \Rightarrow (('a \times 's) \text{ option}) \text{ dist}$
where $\text{dist-throw} = (\lambda -. \text{delta-dist } \text{None})$

lift-definition $\text{throw} :: ('a, 's) \text{ state-monad}$
is dist-throw .

adhoc-overloading $\text{Monad-Syntax.bind} \Rightarrow \text{sm-bind}$

1.32.3 Monad laws

lemma $\text{dist-bind-return-left}$:
 $\text{dist-bind } (\text{dist-return } a) k = k a$
proof
fix s
show $\text{dist-bind } (\text{dist-return } a) k s = k a s$
apply $(\text{rule dist-inject}[\text{THEN iffD1}])$
unfolding $\text{dist-bind.rep-eq dist-return-def}$
apply $(\text{simp add: o-def map-fun-def dist-delta-dist})$
apply $(\text{subst map-bind-delta-left-state}[\text{where } f = \lambda t. \text{Some } (a, t)])$
by $(\text{auto simp: bind-cont-map-def split: option.splits prod.splits})$
qed

lemma $\text{sm-bind-return-left}[\text{simp}]$:
 $\text{sm-bind } (\text{return } a) k = k a$
by $\text{transfer } (\text{rule dist-bind-return-left})$

lemma $\text{dist-bind-return-right}$:
 $\text{dist-bind } m (\lambda a. \text{dist-return } a) = m$
proof
fix s
have cont :
 $\text{bind-cont-map } (\lambda x xa. \text{delta-map } (\text{Some } (x, xa))) = (\lambda r. \text{delta-map } r)$
by $(\text{rule ext}) (\text{simp add: bind-cont-map-return})$
show $\text{dist-bind } m (\lambda a. \text{dist-return } a) s = m s$
apply $(\text{rule dist-inject}[\text{THEN iffD1}])$
unfolding $\text{dist-bind.rep-eq dist-return-def}$
apply $(\text{simp add: o-def map-fun-def dist-delta-dist})$
apply (subst cont)
by $(\text{simp add: map-bind-delta-right})$
qed

lemma $\text{sm-bind-return-right}[\text{simp}]$:

```

sm-bind m return = m
by transfer (rule dist-bind-return-right)

lemma dist-bind-assoc:
  dist-bind (dist-bind m k) h =
    dist-bind m (λa. dist-bind (k a) h)
proof
  fix s
  let ?M = λs. dist (m s)
  let ?K = bind-cont-map (λa t. dist (k a t))
  let ?H = bind-cont-map (λa t. dist (h a t))
  let ?R = bind-cont-map (λa t. map-bind (λu. dist (k a u)) ?H t)
  have assoc:
    map-bind (map-bind ?M ?K) ?H s =
      map-bind ?M (λr. map-bind (λ-. ?K r) ?H ()) s
    by (rule map-bind-assoc) (auto intro!: finite-dom-bind-cont-mapI)
  have cont:
    map-bind (λ-. ?K r) ?H () = ?R r for r
  proof (cases r)
    case None
    then show ?thesis
      apply (simp add: bind-cont-map-def)
      apply (subst map-bind-delta-left[where a=None])
      apply (auto simp: bind-cont-map-def delta-map-def split: option.splits prod.splits)
      done
    next
    case (Some as)
    obtain a t where r: r = Some (a, t)
    using Some by (cases as) simp
    show ?thesis
      unfolding r bind-cont-map-def
      apply (simp add: map-fun-def)
      by (rule map-bind-cong-state) simp-all
  qed
  show
    dist-bind (dist-bind m k) h s =
      dist-bind m (λa. dist-bind (k a) h) s
  apply (rule dist-inject[THEN iffD1])
  unfolding dist-bind.rep-eq
  using assoc
  apply (simp add: o-def map-fun-def dist-bind.rep-eq)
  apply (rule map-bind-cong)
  apply simp
  using cont
  by simp
qed

```

```

lemma sm-bind-assoc:
  sm-bind (sm-bind m k) h = sm-bind m (λa. sm-bind (k a) h)

```

by transfer (rule dist-bind-assoc)

1.33 Monad Operations

1.33.1 Basic Operators

definition *dist-get* :: 's $\Rightarrow$ (('s $\times$'s) option) dist
where
dist-get = (λs . delta-dist (Some (s,s)))

lift-definition *get* :: ('s, 's) state-monad
is *dist-get* .

definition *dist-put* :: 's $\Rightarrow$'s $\Rightarrow$ (unit $\times$'s) option) dist **where**
dist-put s = (λ -. delta-dist (Some ((),s)))

lift-definition *put* :: 's $\Rightarrow$ (unit, 's) state-monad
is *dist-put* .

definition *lift-dist* :: ('s $\Rightarrow$ 'a dist) $\Rightarrow$'s $\Rightarrow$ (('a $\times$'s) option) dist
where
lift-dist d = (λs . dist-map (λa . Some (a, s)) (d s))

lift-definition *lift* :: ('s $\Rightarrow$ 'a dist) $\Rightarrow$ ('a, 's) state-monad
is *lift-dist* .

definition *modify* :: ('s $\Rightarrow$'s) $\Rightarrow$ (unit, 's) state-monad **where**
modify f = *get* $\gg$ ($\lambda s::'s$. *put* (f s))

definition *assert* :: bool $\Rightarrow$ (unit, 's) state-monad **where**
assert b = (if b then return () else throw)

primrec *ntimes* :: ('a, 's) state-monad $\Rightarrow$ nat $\Rightarrow$ ('a list, 's) state-monad
where
ntimes m 0 = return []
| *ntimes* m (Suc n) =
do {
x $\leftarrow$ m;
xs $\leftarrow$ *ntimes* m n;
return (x # xs)
}

primrec *mfold2* :: ('a $\Rightarrow$ (unit, 's) state-monad) $\Rightarrow$ 'a list $\Rightarrow$ (unit, 's) state-monad
where
mfold2 m [] = return ()
| *mfold2* m (x # xs) =
do {
m x;
mfold2 m xs
}

primrec *mmap* :: ('a,'s) state-monad list $\Rightarrow$ ('a list, 's) state-monad

where

```

mmap [] = return []
| mmap (m # ms) =
  do {
    x  $\leftarrow$  m;
    xs  $\leftarrow$  mmap ms;
    return (x # xs)
  }

```

primrec *mfold* :: 'a $\Rightarrow$ ('a $\Rightarrow$ ('a,'s) state-monad) list $\Rightarrow$ ('a, 's) state-monad

where

```

mfold a [] = return a
| mfold a (m # ms) =
  do {
    x  $\leftarrow$  m a;
    mfold x ms
  }

```

1.33.2 Probabilistic Choice

definition *die* :: nat $\Rightarrow$ (nat, 's) state-monad

where *die* n = create (λs . dist-uniform (fset-of-list (map (λx . Some (x,s)) [1..*Suc* n])))

definition *coin* :: prob $\Rightarrow$ 'a $\Rightarrow$ 'a $\Rightarrow$ ('a, 's) state-monad

where *coin* f x y = create (λs . dist-of-fset {|(Some (x,s),f),(Some (y,s),1-f)|})

1.34 Weakest Precondition Calculus

definition *dist-expect* :: 'a dist $\Rightarrow$ ('a $\Rightarrow$ prob) $\Rightarrow$ prob **where**

dist-expect d Q = ($\sum x \in \text{dom } (dist\ d)$. the (dist d x) * Q x)

lemma *dist-expect-items*:

dist-expect d Q = fsum ($\lambda(x, p)$. p * Q x) (items d)

unfolding *dist-expect-def* fsum-items

by *simp*

lemma *dist-expect-return*[*simp*]:

dist-expect (delta-dist x) Q = Q x

unfolding *dist-expect-def*

by (*simp add: dist-delta-dist delta-map-def*)

lemma *dist-expect-0*[*simp*]:

dist-expect d (λ -. 0) = 0

unfolding *dist-expect-def* **by** *simp*

lemma *dist-expect-indicator*:

dist-expect d (λx . if x = a then 1 else 0) =

option-default (dist d a)

proof –
have $\text{dist-expect } d \ (\lambda x. \text{ if } x = a \text{ then } 1 \text{ else } 0) =$
 $(\sum_{x \in \text{dom } (dist\ d)}. \text{ if } x = a \text{ then the } (dist\ d\ x) \text{ else } 0)$
unfolding dist-expect-def
by $(intro\ sum.cong\ refl)\ simp$
also have $\dots = (\text{if } a \in \text{dom } (dist\ d) \text{ then the } (dist\ d\ a) \text{ else } 0)$
by $(simp\ add: sum.delta)$
also have $\dots = \text{option-default } (dist\ d\ a)$
by $(cases\ dist\ d\ a)\ auto$
finally show $?thesis$.
qed

lemma dist-expect-None :
 $\text{dist-expect } d \ (\lambda r. \text{ case } r \text{ of } None \Rightarrow 1 \mid Some\ - \Rightarrow 0) =$
 $\text{option-default } (dist\ d\ None)$

proof –
have $eq: (\lambda r. \text{ case } r \text{ of } None \Rightarrow 1 \mid Some\ - \Rightarrow 0) =$
 $(\lambda r. \text{ if } r = None \text{ then } 1 \text{ else } 0)$
by $(rule\ ext)\ (simp\ split: option.splits)$
have $\text{dist-expect } d \ (\lambda r. \text{ case } r \text{ of } None \Rightarrow 1 \mid Some\ - \Rightarrow 0) =$
 $\text{dist-expect } d \ (\lambda r. \text{ if } r = None \text{ then } 1 \text{ else } 0)$
by $(simp\ add: eq)$
also have $\dots = \text{option-default } (dist\ d\ None)$
by $(rule\ dist-expect-indicator)$
finally show $?thesis$.
qed

lemma dist-expect-map :
 $\text{dist-expect } (dist\text{-map } f\ d)\ Q = \text{dist-expect } d \ (\lambda x. Q\ (f\ x))$

proof –
let $?M = \text{dom } (dist\ d)$
have $\text{dom-map: dom } (dist\ (dist\text{-map } f\ d)) = f\ ' ?M$
using $\text{set-dist-dist-map}[of\ f\ d]$
unfolding set-dist-def **by** $simp$

have $\text{dist-expect } (dist\text{-map } f\ d)\ Q =$
 $(\sum_{x \in f\ ' ?M}. \text{ the } (dist\ (dist\text{-map } f\ d)\ x) * Q\ x)$
unfolding $\text{dist-expect-def dom-map}$ **by** $simp$

also have $\dots =$
 $(\sum_{x \in f\ ' ?M}. \text{ the } (dist\ d\ y)) * Q\ x)$
proof $(intro\ sum.cong\ refl)$

fix x
assume $x: x \in f\ ' ?M$
then have $x\text{-dom: } x \in \text{dom } (dist\ (dist\text{-map } f\ d))$
using dom-map **by** $simp$
have $\text{the } (dist\ (dist\text{-map } f\ d)\ x) =$
 $fsum\ snd\ (ffilter\ (\lambda i. f\ (fst\ i) = x)\ (items\ d))$
using $fsum\ items\ filter\ dist\ map\ on\ dom[OF\ x\text{-dom}]$

```

    by simp
  also have ... = (∑ y ∈ {y ∈ ?M. f y = x}. the (dist d y))
    by (rule fsum-items-filter-dist-map)
  finally show
    the (dist (dist-map f d) x) * Q x =
      (∑ y ∈ {y ∈ ?M. f y = x}. the (dist d y)) * Q x
    by simp
qed
also have ... =
  (∑ x ∈ f' ?M.
    (∑ y ∈ {y ∈ ?M. f y = x}. the (dist d y) * Q (f y)))
proof (intro sum.cong refl)
  fix x
  show
    (∑ y ∈ {y ∈ ?M. f y = x}. the (dist d y)) * Q x =
      (∑ y ∈ {y ∈ ?M. f y = x}. the (dist d y) * Q (f y))
    by (subst sum-distrib-right) (intro sum.cong refl, auto)
qed
also have ... = (∑ y ∈ ?M. the (dist d y) * Q (f y))
  by (rule sum.group) auto
also have ... = dist-expect d (λx. Q (f x))
  unfolding dist-expect-def by simp
finally show ?thesis .
qed

lemma fimage-fst-norm-fset-subset:
  assumes fsum snd A ≠ 0
  shows fset (fimage fst (norm-fset A)) ⊆ fset (fimage fst A)
proof
  fix x
  assume x ∈ fset (fimage fst (norm-fset A))
  then obtain p where xp: (x, p) ∈ norm-fset A
    by auto
  then show x ∈ fset (fimage fst A)
    using assms
    unfolding norm-fset-def squish-def
    by (force simp: Let-def split: prod.splits)
qed

lemma dom-dist-of-fset-subset:
  assumes fsum snd A ≠ 0
  shows dom (dist (dist-of-fset A)) ⊆ fset (fimage fst A)
proof -
  have dist-eq: dist (dist-of-fset A) = map-of (norm-fset A)
    using assms
    unfolding dist-of-fset.rep-eq map-of-fset-def
    by (simp add: Let-def)
  have dom (dist (dist-of-fset A)) ⊆ fset (fimage fst (norm-fset A))
    unfolding dist-eq

```

```

    by (rule dom-map-of-subset)
  also have ...  $\subseteq$  fset (fimage fst A)
    by (rule fimage-fst-norm-fset-subset[OF assms])
  finally show ?thesis .
qed

lemma dom-dist-uniform-subset:
  assumes  $xs \neq \{\mid\}$ 
  shows dom (dist (dist-uniform xs))  $\subseteq$  fset xs
proof -
  let ?A = (( $\lambda x. (x, 1::prob)$ )  $\mid$   $\mid$  xs)
  have total: fsum snd ?A  $\neq 0$ 
    using assms by (simp add: fsum-snd-uniform-ones)
  have dom (dist (dist-uniform xs))  $\subseteq$  fset (fimage fst ?A)
    unfolding dist-uniform-def
    by (rule dom-dist-of-fset-subset[OF total])
  also have ...  $\subseteq$  fset xs
    by auto
  finally show ?thesis .
qed

lemma fsum-snd-coin-nonzero:
  fixes p :: prob
  shows fsum snd  $\{|(a, p), (b, 1 - p)|\} \neq 0$ 
proof -
  have one-minus-zero: ( $1::prob$ ) - 0 = 1
    by transfer simp
  have  $p \neq 0 \vee 1 - p \neq 0$ 
  proof (cases p = 0)
    case True
    have  $1 - p = 1$ 
      using True one-minus-zero by simp
    then show ?thesis
      by simp
  next
    case False
    then show ?thesis
      by simp
  qed
  then show ?thesis
  proof
    assume p:  $p \neq 0$ 
    have  $(a, p) \in \{|(a, p), (b, 1 - p)|\}$ 
      by simp
    then show ?thesis
      using p
      unfolding fsum-fset-sum
      by (auto simp: sum-nonneg-eq-0-iff)
  next

```

```

assume  $q: 1 - p \neq 0$ 
have  $(b, 1 - p) \in \text{fset } \{|(a, p), (b, 1 - p)|\}$ 
  by simp
then show ?thesis
  using  $q$ 
  unfolding fsum-fset-sum
  by (auto simp: sum-nonneg-eq-0-iff)
qed
qed

lemma dist-expect-bind:
  dist-expect (dist-bind m k s) Q =
    dist-expect (m s)
    ( $\lambda r. \text{case } r \text{ of } \text{None} \Rightarrow Q \text{ None} \mid \text{Some } (x, s') \Rightarrow \text{dist-expect } (k \ x \ s') \ Q$ )
proof -
  let  $?dm = \text{dist} \circ m$ 
  let  $?dk = \text{map-fun id dist} \circ k$ 
  let  $?M = \text{dom } (?dm \ s)$ 
  let  $?K = \text{bind-cont-map } ?dk$ 
  let  $?D = \bigcup i \in ?M. \text{dom } (?K \ i)$ 
  let  $?B = \text{map-bind } ?dm \ ?K \ s$ 

  have fin-D: finite ?D
    by (auto simp: bind-cont-map-def delta-map-def split: option.splits prod.splits)

  have dom-B-subset: dom ?B  $\subseteq$  ?D
    by (auto simp: dom-map-bind)

  have dist-expect (dist-bind m k s) Q =
    ( $\sum x \in \text{dom } ?B. \text{the } (?B \ x) * Q \ x$ )
    unfolding dist-expect-def dist-bind.rep-eq by simp
  also have  $\dots = (\sum x \in ?D. \text{option-default } (?B \ x) * Q \ x)$ 
    using sum-option-default-mult-superset[OF fin-D dom-B-subset, of Q]
    by simp
  also have  $\dots =$ 
    ( $\sum x \in ?D. \sum i \in ?M. \text{the } (\text{dist } (m \ s) \ i) * \text{option-default } (?K \ i \ x) * Q \ x$ )
    by (intro sum.cong refl) (simp add: option-default-map-bind)
  also have  $\dots =$ 
    ( $\sum x \in ?D. \sum i \in ?M. Q \ x * (\text{the } (\text{dist } (m \ s) \ i) * \text{option-default } (?K \ i \ x))$ )
proof (intro sum.cong refl)
  fix  $x$ 
  assume  $x \in ?D$ 
  have
    ( $\sum i \in ?M. \text{the } (\text{dist } (m \ s) \ i) * \text{option-default } (?K \ i \ x) * Q \ x =$ 
      ( $\sum i \in ?M. (\text{the } (\text{dist } (m \ s) \ i) * \text{option-default } (?K \ i \ x)) * Q \ x$ )
      by (simp add: sum-distrib-right)
    also have  $\dots =$ 

```



```

    (∑ i ∈ ?M. Q x * (the (dist (m s) i) * option-default (?K i x)))
  by (intro sum.cong refl) (simp add: algebra-simps)
finally show
  (∑ i ∈ ?M. the (dist (m s) i) * option-default (?K i x)) * Q x =
  (∑ i ∈ ?M. Q x * (the (dist (m s) i) * option-default (?K i x))) .
qed
also have ... =
  (∑ i ∈ ?M. ∑ x ∈ ?D.
    Q x * (the (dist (m s) i) * option-default (?K i x)))
  by (rule sum.swap)
also have ... =
  (∑ i ∈ ?M.
    the (dist (m s) i) *
    (∑ x ∈ ?D. option-default (?K i x) * Q x))
  by (simp add: sum-distrib-left algebra-simps)
also have ... =
  (∑ i ∈ ?M.
    the (dist (m s) i) *
    (∑ x ∈ dom (?K i). the (?K i x) * Q x))
proof (intro sum.cong refl)
  fix i
  assume i: i ∈ ?M
  have dom (?K i) ⊆ ?D
    using i by auto
  then show
    the (dist (m s) i) *
    (∑ x ∈ ?D. option-default (?K i x) * Q x) =
    the (dist (m s) i) *
    (∑ x ∈ dom (?K i). the (?K i x) * Q x)
    using sum-option-default-mult-superset[OF fin-D, of ?K i Q]
    by simp
qed
also have ... =
  dist-expect (m s)
  (λr. case r of None ⇒ Q None | Some (x, s') ⇒ dist-expect (k x s') Q)
unfolding dist-expect-def bind-cont-map-def
apply (rule sum.cong)
apply simp
apply (rename-tac i)
apply (case-tac i)
apply (auto simp: case-prod-unfold dist-delta-dist delta-map-def split: prod.splits)
done
finally show ?thesis .
qed

definition wp :: ('a, 's) state-monad ⇒ (('a × 's) option ⇒ prob) ⇒ 's ⇒ prob where
  wp m Q s =
    dist-expect (execute m s)
    (λr. Q r)

```

lemma *wp-items*:

wp m Q s =
*fsum (λ(r, p). p * Q r) (items (execute m s))*
unfolding *wp-def dist-expect-items*
by (*simp add: case-prod-unfold*)

definition *wp-event* :: (*'a*, *'s*) *state-monad* $\Rightarrow$ ((*'a* $\times$ *'s*) *option* $\Rightarrow$ *bool*) $\Rightarrow$ *'s* $\Rightarrow$ *prob* **where**
wp-event m P s = wp m (λx. if P x then 1 else 0) s

definition *outcome-prob* ::

(*'a*, *'s*) *state-monad* $\Rightarrow$ *'s* $\Rightarrow$ *'a* $\Rightarrow$ *'s* $\Rightarrow$ *prob* **where**
outcome-prob m s x s' = option-default (dist (execute m s) (Some (x, s')))

definition *failure-prob* :: (*'a*, *'s*) *state-monad* $\Rightarrow$ *'s* $\Rightarrow$ *prob* **where**

failure-prob m s = option-default (dist (execute m s) None)

definition *success-prob* :: (*'a*, *'s*) *state-monad* $\Rightarrow$ *'s* $\Rightarrow$ *prob* **where**

success-prob m s = wp m (λ-. 1) s

definition *wp-error* :: (*'a*, *'s*) *state-monad* $\Rightarrow$ *'s* $\Rightarrow$ *prob* **where**

wp-error m s =
wp m (λr. case r of None $\Rightarrow$ 1 | Some - $\Rightarrow$ 0) s

definition *wp-success* :: (*'a*, *'s*) *state-monad* $\Rightarrow$ (*'a* $\Rightarrow$ *'s* $\Rightarrow$ *prob*) $\Rightarrow$ *'s* $\Rightarrow$ *prob* **where**

wp-success m Q s =
wp m (λr. case r of None $\Rightarrow$ 0 | Some (x, s') $\Rightarrow$ Q x s') s

named-theorems *wpsimps*

named-theorems *wp*

lemma *wp-return*[*wpsimps*]:

wp (return x) Q s = Q (Some (x, s))
unfolding *wp-def return.rep-eq dist-return-def*
by *simp*

lemma *wp-bind*[*wpsimps*]:

wp (m $\gg$ k) Q s =
wp m (λr. case r of
None $\Rightarrow$ Q None
| Some (x, s') $\Rightarrow$ wp (k x) Q s') s
unfolding *wp-def sm-bind.rep-eq*
apply (*simp add: dist-expect-bind*)
apply (*rule arg-cong[where f=λF. dist-expect (execute m s) F]*)
by (*rule ext*) (*auto split: option.splits prod.splits*)

lemma *wp-get*[*wpsimps*]:

wp get Q s = Q (Some (s, s))
unfolding *wp-def get.rep-eq dist-get-def* **by** *simp*

lemma *wp-put*[*wpsimps*]:
 $wp\ (put\ s')\ Q\ s = Q\ (Some\ ((),\ s'))$
unfolding *wp-def put.rep-eq dist-put-def* **by** *simp*

lemma *wp-lift*[*wpsimps*]:
 $wp\ (lift\ d)\ Q\ s = dist-expect\ (d\ s)\ (\lambda x. Q\ (Some\ (x,\ s)))$
unfolding *wp-def lift.rep-eq lift-dist-def*
by (*simp add: dist-expect-map*)

lemma *wp-modify*[*wpsimps*]:
 $wp\ (modify\ f)\ Q\ s = Q\ (Some\ ((),\ f\ s))$
unfolding *modify-def* **by** (*simp add: wpsimps*)

lemma *wp-throw*[*wpsimps*]:
 $wp\ throw\ Q\ s = Q\ None$
unfolding *wp-def throw.rep-eq dist-throw-def* **by** *simp*

lemma *wp-create*[*wpsimps*]:
 $wp\ (create\ f)\ Q\ s = dist-expect\ (f\ s)\ Q$
unfolding *wp-def execute-create* **by** *simp*

lemma *wp-die*[*wpsimps*]:
 $wp\ (die\ n)\ Q\ s =$
 $dist-expect$
 $(dist-uniform\ (fset-of-list\ (map\ (\lambda x. Some\ (x,\ s))\ [1..<Suc\ n])))\ Q$
unfolding *die-def* **by** (*simp add: wpsimps*)

lemma *wp-coin*[*wpsimps*]:
 $wp\ (coin\ p\ x\ y)\ Q\ s =$
 $dist-expect\ (dist-of-fset\ \{|(Some\ (x,\ s),\ p),\ (Some\ (y,\ s),\ 1 - p)|\})\ Q$
unfolding *coin-def* **by** (*simp add: wpsimps*)

lemma *wp-returnI*[*wp*]:
assumes $P \leq Q\ (Some\ (x,\ s))$
shows $P \leq wp\ (return\ x)\ Q\ s$
using *assms* **by** (*simp add: wpsimps*)

lemma *wp-bindI*[*wp*]:
assumes $P \leq wp\ m\ (\lambda r. case\ r\ of$
 $None \Rightarrow Q\ None$
 $| Some\ (x,\ s') \Rightarrow wp\ (k\ x)\ Q\ s')\ s$
shows $P \leq wp\ (m\ \gg\ k)\ Q\ s$
using *assms* **by** (*simp add: wpsimps*)

lemma *wp-getI*[*wp*]:
assumes $P \leq Q\ (Some\ (s,\ s))$
shows $P \leq wp\ get\ Q\ s$
using *assms* **by** (*simp add: wpsimps*)

lemma *wp-putI*[*wp*]:
assumes $P \leq Q \text{ (Some } ((), s^{\wedge}) \text{)}$
shows $P \leq wp \text{ (put } s^{\wedge}) Q s$
using *assms* **by** (*simp add: wpsimps*)

lemma *wp-modifyI*[*wp*]:
assumes $P \leq Q \text{ (Some } ((), f s) \text{)}$
shows $P \leq wp \text{ (modify } f) Q s$
using *assms* **by** (*simp add: wpsimps*)

lemma *wp-liftI*[*wp*]:
assumes $P \leq dist-expect \text{ (d s) } (\lambda x. Q \text{ (Some } (x, s)))$
shows $P \leq wp \text{ (lift d) } Q s$
using *assms* **by** (*simp add: wpsimps*)

lemma *wp-throwI*[*wp*]:
assumes $P \leq Q \text{ None}$
shows $P \leq wp \text{ throw } Q s$
using *assms* **by** (*simp add: wpsimps*)

lemma *wp-createI*[*wp*]:
assumes $P \leq dist-expect \text{ (f s) } Q$
shows $P \leq wp \text{ (create } f) Q s$
using *assms* **by** (*simp add: wpsimps*)

lemma *wp-dieI*[*wp*]:
assumes $P \leq$
 $dist-expect$
 $(dist-uniform \text{ (fset-of-list (map } (\lambda x. Some \text{ (x, s)) } [1..<Suc \text{ n}]))) Q$
shows $P \leq wp \text{ (die n) } Q s$
using *assms* **by** (*simp add: wpsimps*)

lemma *wp-coinI*[*wp*]:
assumes $P \leq dist-expect \text{ (dist-of-fset } \{|(Some \text{ (x, s), } p), (Some \text{ (y, s), } 1 - p)|\}) Q$
shows $P \leq wp \text{ (coin p x y) } Q s$
using *assms* **by** (*simp add: wpsimps*)

lemma *wp-error-eq-failure-prob*:
 $wp-error \text{ m s} = failure-prob \text{ m s}$
unfolding *wp-error-def wp-def failure-prob-def*
by (*simp add: dist-expect-None*)

lemma *wp-error-return*[*wpsimps*]:
 $wp-error \text{ (return x) s} = 0$
unfolding *wp-error-def* **by** (*simp add: wpsimps*)

lemma *wp-error-bind*[*wpsimps*]:
 $wp-error \text{ (m } \gg k) s =$

```

    wp m (λr. case r of
      None ⇒ 1
    | Some (x, s') ⇒ wp-error (k x) s') s
  unfolding wp-error-def
  apply (simp add: wpsimps)
  apply (rule arg-cong[where f=λF. wp m F s])
  by (rule ext) (auto split: option.splits prod.splits)

lemma wp-error-get[wpsimps]:
  wp-error get s = 0
  unfolding wp-error-def by (simp add: wpsimps)

lemma wp-error-put[wpsimps]:
  wp-error (put s') s = 0
  unfolding wp-error-def by (simp add: wpsimps)

lemma wp-error-modify[wpsimps]:
  wp-error (modify f) s = 0
  unfolding wp-error-def by (simp add: wpsimps)

lemma wp-error-lift[wpsimps]:
  wp-error (lift d) s = 0
  unfolding wp-error-def by (simp add: wpsimps)

lemma wp-error-throw[wpsimps]:
  wp-error throw s = 1
  unfolding wp-error-def by (simp add: wpsimps)

lemma wp-error-create[wpsimps]:
  wp-error (create f) s = option-default (dist (f s) None)
  by (simp add: wp-error-eq-failure-prob failure-prob-def execute-create)

lemma wp-error-coin[wpsimps]:
  wp-error (coin p x y) s = 0
proof -
  let ?A = {|(Some (x, s), p), (Some (y, s), 1 - p)|}
  have total: fsum snd ?A ≠ 0
    by (rule fsum-snd-coin-nonzero)
  have None ∉ dom (dist (dist-of-fset ?A))
    using dom-dist-of-fset-subset[OF total]
    by auto
  then show ?thesis
    unfolding wp-error-def
    by (simp add: wpsimps dist-expect-None domIff)
qed

lemma wp-error-die-pos[wpsimps]:
  assumes 0 < n
  shows wp-error (die n) s = 0

```

proof –
 let $?xs = \text{fset-of-list } (\text{map } (\lambda x. \text{Some } (x, s)) [1..<\text{Suc } n])$
 have $xs\text{-nonempty}: ?xs \neq \{\}\}$
 using *assms* **by** *auto*
 have $none\text{-}xs: \text{None} \notin \text{fset } ?xs$
 by (*auto simp: fset-of-list.rep-eq*)
 have $\text{supp}: \text{dom } (\text{dist } (\text{dist-uniform } ?xs)) \subseteq \text{fset } ?xs$
 by (*rule dom-dist-uniform-subset[OF xs-nonempty]*)
 have $\text{None} \notin \text{dom } (\text{dist } (\text{dist-uniform } ?xs))$
 using *supp none-xs* **by** *blast*
 then show *?thesis*
 unfolding *wp-error-def*
 by (*simp add: wpsimps dist-expect-None domIff*)
qed

lemma *wp-success-return[wpsimps]*:
 $\text{wp-success } (\text{return } x) \ Q \ s = Q \ x \ s$
 unfolding *wp-success-def* **by** (*simp add: wpsimps*)

lemma *wp-success-bind[wpsimps]*:
 $\text{wp-success } (m \gg k) \ Q \ s =$
 $\text{wp-success } m \ (\lambda x \ s'. \text{wp-success } (k \ x) \ Q \ s') \ s$
 unfolding *wp-success-def*
 apply (*simp add: wpsimps*)
 apply (*rule arg-cong[where f= $\lambda F. \text{wp } m \ F \ s$]*)
 by (*rule ext*) (*auto split: option.splits prod.splits*)

lemma *wp-success-get[wpsimps]*:
 $\text{wp-success } \text{get} \ Q \ s = Q \ s \ s$
 unfolding *wp-success-def* **by** (*simp add: wpsimps*)

lemma *wp-success-put[wpsimps]*:
 $\text{wp-success } (\text{put } s') \ Q \ s = Q \ () \ s'$
 unfolding *wp-success-def* **by** (*simp add: wpsimps*)

lemma *wp-success-modify[wpsimps]*:
 $\text{wp-success } (\text{modify } f) \ Q \ s = Q \ () \ (f \ s)$
 unfolding *wp-success-def* **by** (*simp add: wpsimps*)

lemma *wp-success-lift[wpsimps]*:
 $\text{wp-success } (\text{lift } d) \ Q \ s = \text{dist-expect } (d \ s) \ (\lambda x. Q \ x \ s)$
 unfolding *wp-success-def* **by** (*simp add: wpsimps*)

lemma *wp-success-throw[wpsimps]*:
 $\text{wp-success } \text{throw} \ Q \ s = 0$
 unfolding *wp-success-def* **by** (*simp add: wpsimps*)

lemma *wp-error-return-check*:
 $\text{wp-error } (\text{return } x) \ s = 0$

by (*simp add: wpsimps*)

lemma *wp-error-throw-check*:
 $wp_error\ throw\ s = 1$
by (*simp add: wpsimps*)

lemma *wp-throw-error-post-check*:
 $wp\ throw\ (\lambda r. case\ r\ of\ None \Rightarrow 1 \mid Some\ - \Rightarrow 0)\ s = 1$
by (*simp add: wpsimps*)

lemma *wp-error-bind-check*:
 $wp_error\ (m \gg k)\ s =$
 $wp\ m\ (\lambda r. case\ r\ of\ None \Rightarrow 1 \mid Some\ (x, s') \Rightarrow wp_error\ (k\ x)\ s')\ s$
by (*simp add: wpsimps*)

lemma *failure-prob-throw[simp]*:
 $failure_prob\ throw\ s = 1$
unfolding *failure-prob-def throw.rep-eq dist-throw-def dist-delta-dist delta-map-def*
by *simp*

lemma *failure-prob-return[simp]*:
 $failure_prob\ (return\ x)\ s = 0$
unfolding *failure-prob-def return.rep-eq dist-return-def dist-delta-dist delta-map-def*
by *simp*

lemma *success-prob-return[simp]*:
 $success_prob\ (return\ x)\ s = 1$
unfolding *success-prob-def* **by** (*simp add: wpsimps*)

lemma *outcome-prob-notin-dom*:
assumes $Some\ (x, s') \notin dom\ (dist\ (execute\ m\ s))$
shows $outcome_prob\ m\ s\ x\ s' = 0$
using *assms* **unfolding** *outcome-prob-def* **by** (*simp add: domIff*)

lemma *all-outcomes-above-thresholdI*:
assumes $\bigwedge x\ s'. outcome_prob\ m\ s\ x\ s' > t \implies P\ x\ s'$
shows $\forall x\ s'. outcome_prob\ m\ s\ x\ s' > t \longrightarrow P\ x\ s'$
using *assms* **by** *blast*

lemma *all-outcomes-above-threshold-domI*:
assumes
 $\bigwedge x\ s'. Some\ (x, s') \in dom\ (dist\ (execute\ m\ s)) \implies$
 $outcome_prob\ m\ s\ x\ s' > t \implies P\ x\ s'$
shows $\forall x\ s'. outcome_prob\ m\ s\ x\ s' > t \longrightarrow P\ x\ s'$
proof (*intro allI impI*)
fix $x\ s'$
assume *gt*: $outcome_prob\ m\ s\ x\ s' > t$
show $P\ x\ s'$
proof (*cases* $Some\ (x, s') \in dom\ (dist\ (execute\ m\ s))$)

```

    case True
    then show ?thesis
      using assms gt by blast
next
case False
then have outcome-prob m s x s' = 0
  by (rule outcome-prob-notin-dom)
then have False
  using gt zero-least[of t] by order
then show ?thesis
  by simp
qed
qed

```

1.34.1 Examples

lemma *return-outcomes-above-zero-example:*

```

 $\forall y t. \text{outcome-prob } (\text{return } x) s y t > 0 \longrightarrow y = x \wedge t = s$ 
unfolding outcome-prob-def return.rep-eq dist-return-def dist-delta-dist delta-map-def
by (simp add: domIff)

```

lemma *return-threshold-example:*

```

 $\forall y t. \text{outcome-prob } (\text{return } x) s y t > 0 \longrightarrow y = x$ 
using return-outcomes-above-zero-example[of x s] by blast

```

1.35 Code generator

lemmas

```

sm-bind-def[code]
execute-create[code]

```

lemma *sum-fset-by-bind-items-generic:*

```

sum-fset-by x
  ( $\lambda(ms, ks). \text{fst } ks$ )
  ( $\lambda(ms, ks). \text{snd } ms * \text{snd } ks$ )
  (fbind (items d) ( $\lambda ms.$ 
    fimage ( $\lambda ks. (ms, ks)$ ) (items (k (fst ms)))))) =
  ( $\sum_{i \in \text{dom } (dist d).}$ 
    the (dist d i) * option-default (dist (k i) x))
unfolding sum-fset-by-def
apply (simp add: fsum-ffilter-if fsum-fbind-fimage-Pair case-prod-unfold)
unfolding fsum-items
apply (intro sum.cong refl)
apply (simp add: fsum-mult-left-nnreal fsum-items-key)
by (simp add: dom-def option.case-eq-if)

```

lemma *fsum-bind-items-total-generic:*

```

fsum
  ( $\lambda(ms, ks). \text{snd } ms * \text{snd } ks$ )
  (fbind (items d) ( $\lambda ms.$ 

```



```

      fimage (λks. (ms, ks)) (items (k (fst ms)))))) = 1
proof -
  have fsum
    (λ(ms, ks). snd ms * snd ks)
    (fbind (items d) (λms.
      fimage (λks. (ms, ks)) (items (k (fst ms)))))) =
    fsum
      (λms. fsum (λks. snd ms * snd ks) (items (k (fst ms))))
      (items d)
  by (simp add: fsum-fbind-fimage-Pair case-prod-unfold)

  also have ... = fsum snd (items d)
  by (intro fsum.cong refl) (simp add: fsum-items-mult-left)

  also have ... = 1
  by simp

  finally show ?thesis .
qed

lemma dist-bind-code[code]:
  fixes m :: 's ⇒ (('a × 's) option) dist
  and k :: 'a ⇒ 's ⇒ (('b × 's) option) dist
  shows
    dist-bind m k s =
      dist-of-fset-by
        (λ(ms, ks). fst ks)
        (λ(ms, ks). snd ms * snd ks)
        (fbind (items (m s)) (λms.
          fimage (λks. (ms, ks))
            (items (case fst ms of None ⇒ delta-dist None | Some (x, s') ⇒ k x s'))))
proof (rule dist-inject[THEN iffD1])
  let ?I =
    fbind (items (m s)) (λms.
      fimage (λks. (ms, ks))
        (items (case fst ms of None ⇒ delta-dist None | Some (x, s') ⇒ k x s'))))
  let ?key = λ(ms, ks). fst ks
  let ?weight = λ(ms, ks). snd ms * snd ks
  let ?next =
    (λr :: ('a × 's) option.
      case r of None ⇒ delta-dist (None :: ('b × 's) option)
      | Some (x, s') ⇒ k x s')

  have next-eq:
    bind-cont-map (map-fun id dist ∘ k) i = dist (?next i) for i
  by (cases i) (auto simp: bind-cont-map-def dist-delta-dist o-def map-fun-def split: prod.splits)

  have next-eq-raw:
    bind-cont-map (λx xa. dist (k x xa)) i = dist (?next i) for i

```

```

by (cases i) (auto simp: bind-cont-map-def dist-delta-dist split: prod.splits)

have total: fsum ?weight ?I = 1
  by (rule fsum-bind-items-total-generic)

show dist (dist-bind m k s) =
  dist (dist-of-fset-by ?key ?weight ?I)
proof
  fix x :: ('b × 's) option

  have by-sum:
    sum-fset-by x ?key ?weight ?I =
      (∑ i∈dom (dist (m s)).
        the (dist (m s) i) *
        option-default (dist (?next i) x))
  by (rule sum-fset-by-bind-items-generic[of x m s ?next])

  have rhs:
    dist (dist-of-fset-by ?key ?weight ?I) x =
      map-of-fset-by ?key ?weight ?I x
  using dist-of-fset-by-rep-eq-1[OF total]
  by metis

  have dist (dist-bind m k s) x =
    (let p =
      (∑ i∈dom (dist (m s)).
        the (dist (m s) i) *
        option-default (dist (?next i) x))
    in if p = 0 then None else Some p)
  unfolding dist-bind.rep-eq map-bind-def
  by (simp add: Let-def o-def map-fun-def next-eq next-eq-raw split: option.splits prod.splits)
  also have ... = map-of-fset-by ?key ?weight ?I x
  using total by-sum
  unfolding map-of-fset-by-def
  by (simp add: Let-def)
  also have ... = dist (dist-of-fset-by ?key ?weight ?I) x
  using rhs by simp
  finally show
    dist (dist-bind m k s) x =
      dist (dist-of-fset-by ?key ?weight ?I) x .

qed
qed

```

1.35.1 Some basic examples

```

fun mymap1::bool ⇒ ((bool × bool) option → prob)
where
  mymap1 True = Map.map-of [(Some (True,True), 0.5),(Some (False,True), 0.5)]

```

```

| mymap1 False = Map.map-of [(Some (False,True), 0.5),(Some (False,False), 0.5)]

fun mymap2::(bool × bool) option ⇒ ((bool × bool) option → prob)
where
  mymap2 None = delta-map None
| mymap2 (Some (True, True)) = Map.map-of [(Some (True,False), 0.5),(Some (False,False),
0.5)]
| mymap2 (Some (False, True)) = Map.map-of [(Some (False,False), 0.5),(Some (False,True),
0.5)]
| mymap2 (Some (True, False)) = Map.map-of [(Some (True,False), 0.5),(Some (False,False),
0.5)]
| mymap2 (Some (False, False)) = Map.map-of [(Some (False,False), 0.5),(Some (True,False),
0.5)]

value map (map-bind mymap1 mymap2 True) [Some (True,False),Some (False,False),Some
(True,True),Some (False,True),None]

value (execute ((return 1)::(nat,nat) state-monad) 0)

value (execute (
  do {
    x ← return (1::nat);
    return (2::nat);
    return x
  } ::(nat,bool) state-monad) False)

value (execute (
  do {
    number ← die 6;
    if number = 6
      then coin 0.5 1 0
      else coin 0.1 1 0
  } ::(nat,bool) state-monad) False)

value execute (coin 0.5 True False) True

value wp-event (coin 0.5 True False) (λa. ∃ b. a = Some (True, b)) True

value execute (
  do {
    number ← coin 0.5 True False;
    if number
      then coin 0.2 True False
      else coin 0.7 True False
  }) False

value wp-event (
  do {
    number ← coin 0.5 True False;

```

```

    if number
      then coin 0.5 True False
      else coin 0.5 True False
  }) (λa. ∃ b. a = Some (b, False)) False

end

theory Stark
imports
  Main
  Merkle-Tree
  Channel
  Polynomial-Interpolation.Lagrange-Interpolation

begin

```

1.36 Stark Protocol

This theory defines the executable prover and verifier monads for the STARK protocol formalized in this entry. It also fixes the common domain parameters, transcript operations, Merkle commitments, query sampler, and FRI folding routines used by the completeness and soundness developments.

The formalization follows the structure of the StarkWare STARK tutorial and the algebraic framework described in the STARK literature. The field class uses Proth fields because their multiplicative groups provide the power-of-two subgroups used for FFT-style evaluation domains.

1.37 Preliminaries

definition *ceil-log* :: *nat* ⇒ *nat* **where**
ceil-log *n* = (if *n* ≤ 1 then 0 else Suc (floor-log (*n* - 1)))

1.37.1 Filter list by predicate over length

There exists *nths* in List which does what we want but it cannot be executed. Thus, we introduce a new function with a corresponding code equation which makes it executable.

definition *nths-pred*
where
nths-pred *xs* *P* ≡ *nths* *xs* (Collect *P*)

lemma *nths-pred-nths*[code]:
nths-pred *xs* *P* = map fst (filter (λ*p*. *P* (snd *p*)) (zip *xs* ([0..*length xs*])))
unfolding *nths-pred-def* *nths-def* **by** *simp*

lemma *nths-pred-alt-def*[*simp*]:

nths-pred *xs P* = *nths xs (Collect P)*
unfolding *nths-pred-def* ..

lemma *nths-pred* ([5,6,3,4]::nat list) even = [5,3] **by** *eval*
lemma *nths-pred* ([5,6,3]::nat list) even = [5,3] **by** *eval*
lemma *nths-pred* ([5,6,3]::nat list) odd = [6] **by** *eval*
lemma *nths-pred* ([6,3]::nat list) even = [6] **by** *eval*
lemma *nths-pred* ([6,3]::nat list) odd = [3] **by** *eval*

lemma *nths-pred-length-leq*:
length (nths-pred xs P) ≤ length xs
unfolding *nths-pred-def nths-def*
by (*metis length-filter-le length-map map-fst-zip map-nth*)

Many properties can be more easily proven if we use *zip xs (rev [0..<length xs])*. Thus, we introduce the following lemmas which can be used to convert the lemmas back to *nths-pred*.

lemma *filter-zip-lcomp*:
assumes *xs ≠ []*
shows
filter (λp. P (snd p)) (zip xs [0..<length xs]) = [(xs!i,i). i ← filter P [0..<length xs]]
using *assms*
proof (*induction xs rule: rev-induct*)
case *Nil*
then show ?*case* **by** *simp*
next
case (*snoc x xs*)
then show ?*case*
proof (*cases P (length xs)*)
case *True*
then have
filter (λp. P (snd p)) (zip (xs @ [x]) [0..<length (xs @ [x])])
= (*filter (λp. P (snd p)) (zip (xs) [0..<(length xs)])*) @ [(*x, (length xs)*)]
using *snoc.prem*s **by** *auto*
also from *snoc.IH*
have ... = *map (λi. (xs ! i, i)) (filter P [0..<length xs])* @ [(*x, (length xs)*)]
by *fastforce*
also have ... = *map (λi. ((xs @ [x]) ! i, i)) (filter P [0..<length (xs @ [x])])*
using *snoc.prem*s *True* **by** (*auto simp add: nth-append-left*)
finally show ?*thesis* .
next
case *False*
then have
filter (λp. P (snd p)) (zip (xs @ [x]) [0..<length (xs @ [x])])
= (*filter (λp. P (snd p)) (zip (xs) [0..<(length xs)])*)
using *snoc.prem*s **by** *auto*
also from *snoc* **have** ... = *map (λi. (xs ! i, i)) (filter P [0..<length xs])* **by** *fastforce*
also have ... = *map (λi. ((xs @ [x]) ! i, i)) (filter P [0..<length ((xs @ [x])])*)
using *snoc.prem*s *False* **by** (*auto simp add: nth-append-left*)

```

    finally show ?thesis .
qed
qed

lemma filter-zip-rev-lcomp:
  assumes  $xs \neq []$ 
  shows
    filter ( $\lambda p. P (snd\ p)$ ) (zip  $xs$  (rev [0.. $length\ xs$ ]))
    = [( $xs!(length\ xs - (Suc\ i)), i$ ) .  $i \leftarrow filter\ P\ (rev\ [0.. $length\ xs$ ])]$ ]
  using assms
proof (induction  $xs$ )
  case Nil
  then show ?case by simp
next
  case (Cons  $x\ xs$ )
  then show ?case
proof (cases  $P\ (length\ xs)$ )
  case True
  then have
    filter ( $\lambda p. P (snd\ p)$ ) (zip ( $x \# xs$ ) (rev [0.. $length\ (x \# xs)$ ]))
    = ( $x, (length\ xs)$ ) # (filter ( $\lambda p. P (snd\ p)$ ) (zip  $xs$  (rev [0.. $length\ xs$ ]))))
    using Cons.prems by auto
  also from Cons.IH have
    ... = ( $x, (length\ xs)$ ) # map ( $\lambda i. (xs!\ (length\ xs - (Suc\ i)), i)$ ) (filter  $P$  (rev [0.. $length\ xs$ ]))
  by fastforce
  also have
    ... = map ( $\lambda i. ((x \# xs)!\ (length\ (x \# xs) - Suc\ i), i)$ ) (filter  $P$  (rev [0.. $length\ (x \# xs)$ ]))
    using Cons.prems True by simp
  finally show ?thesis .
next
  case False
  then have
    filter ( $\lambda p. P (snd\ p)$ ) (zip ( $x \# xs$ ) (rev [0.. $length\ (x \# xs)$ ]))
    = filter ( $\lambda p. P (snd\ p)$ ) (zip  $xs$  (rev [0.. $length\ xs$ ]))
    using Cons.prems by auto
  also from Cons.IH
  have ... = map ( $\lambda i. (xs!\ (length\ xs - (Suc\ i)), i)$ ) (filter  $P$  (rev [0.. $length\ xs$ ]))
    by fastforce
  also have ... = map ( $\lambda i. ((x \# xs)!\ (length\ (x \# xs) - Suc\ i), i)$ ) (filter  $P$  (rev [0.. $length\ (x \# xs)$ ]))
    using Cons.prems False by simp
  finally show ?thesis .
qed
qed

```

```

lemma length-filter-zip-length-rev:
  length (filter ( $\lambda p. P (snd\ p)$ ) (zip  $xs$  ([0.. $length\ xs$ ]))))

```

```

    = length (filter (λp. P (snd p)) (zip xs (rev [0.. $\text{length } xs$ ])))
proof (cases xs)
  case Nil
  then show ?thesis by simp
next
  case (Cons a list)
  then have length (map (λi. (xs ! i, i)) (filter P [0.. $\text{length } xs$ ]))
    = length (map (λi. (xs ! (length xs - Suc i), i)) (filter P (rev [0.. $\text{length } xs$ ])))
    by (metis length-map length-rev rev-filter)
  then show ?thesis using filter-zip-lcomp[of xs P] filter-zip-rev-lcomp[of xs P]
    by fastforce
qed

```

The following two lemmas are example of what can be easily proven if we use *rev* $[0.. $\text{length } xs$]$

```

lemma nths-pred-even-length-eq:
  length (nths-pred xs even) = (Suc (length xs)) div 2
proof -
  have length (map fst (filter (λp. even (snd p)) (zip xs (rev [0.. $\text{length } xs$ ])))) = Suc (length xs) div 2
    by (induction xs, auto)
  then show ?thesis using length-filter-zip-length-rev[of even xs] by (simp add: nths-def)
qed

```

```

lemma nths-pred-odd-length-eq:
  length (nths-pred xs odd) = (length xs) div 2
proof -
  have length (map fst (filter (λp. odd (snd p)) (zip xs (rev [0.. $\text{length } xs$ ])))) = length xs div 2
    by (induction xs, auto)
  then show ?thesis using length-filter-zip-length-rev[of odd xs] by (simp add: nths-def)
qed

```

```

lemma nths-pred-even-length-less:
  assumes length xs > 1
  shows length (nths-pred xs even) < length xs
  using assms nths-pred-even-length-eq[of xs]
  by linarith

```

```

lemma nths-pred-odd-length-less:
  assumes length xs > 1
  shows length (nths-pred xs odd) < length xs
  using assms nths-pred-odd-length-eq[of xs]
  by linarith

```

1.37.2 Basic polynomial definitions

```

definition XP
where
  XP x ≡ monom x 1

```

definition *CP*

where

$CP\ x \equiv monom\ x\ 0$

definition *prod*

where

$prod\ xs \equiv fold\ (\lambda s. (*)\ (XP\ 1 - CP\ s))\ xs\ 1$

1.38 Stark Specifications

definition *degrees*

where $degrees\ clength\ spec = insert\ 0\ \{d * (clength - 1) - length\ rs \mid a\ rs\ d. (a, rs, d) \in set\ spec\}$

definition *exact-order* :: $'a::monoid-mult \Rightarrow nat \Rightarrow bool$

where

$exact-order\ a\ n \longleftrightarrow$

$0 < n \wedge a \wedge^n = 1 \wedge (\forall m. 0 < m \wedge m < n \longrightarrow a \wedge^m \neq 1)$

locale *stark* =

protocol-channel-compat +

constrains $\omega::'f::proth-field$

and $shift::'f$

and $g::'f$

and $h::'f$

and $vals::'f\ list$

and $concat :: 'f \Rightarrow 'f \Rightarrow 'f$

fixes $\omega::'f::proth-field$ — This is assumed to be a generator of $'f$

and $shift::'f$ — This shifts H to the evaluation-domain coset.

and $scale::nat$ — This is the scaling or blow-up factor

and $clength::nat$

— The length of the computation, it does need to be a power of two or else needs to be padded to the next power of 2

and $powers::nat$

— The number of powers of the generator required for the spec $[0..<powers]$. This is basically the size of the list which is passed to the first element of *spec*

and $to-nat::'f \Rightarrow nat$

and $of-nat::nat \Rightarrow 'f$

— Functions to transfer from $'f$ to the field and back

and $size::nat$

— The size of $'f$

and $spec:: (('f\ poly\ list \Rightarrow 'f\ poly) \times nat\ list \times nat)\ list$

— The specification consists of a list of three elements: A constraint polynomial, i.e., a function which creates a polynomial from a list of polynomials (the input here is assumed to be the trace polynomial scaled by powers of the generator); A list of roots for a given constraint polynomial; The degree of the constraint polynomial (actually this could be computed but for now we give it explicitly)

and $rounds::nat$

assumes *size-card*: $size = CARD('f)$ — This is just there since $CARD$ cannot be computed

and *clength-pow*: $\exists k. clength = 2^k$

— We assume the trace has correct length. In reality this is not required since we can just add padding.

and *scale-pow*: $\exists k. scale = 2^k$ — This needs to be a power of two to make definition of h well-defined

and *omega-order*: *exact-order omega* ($size - 1$)

— *omega* is a primitive generator of the multiplicative group.

and *eval-domain-dvd*: $clength * scale \text{ dvd } size - 1$

and *eval-domain-nontrivial*: $1 < clength * scale$

and *of-nat-to-nat*: $\forall x. \text{of-nat } (\text{to-nat } x) = x$

and *to-nat-range*:

$range \text{ to-nat} = \{0..<size\}$

— Exact-uniformity assumption for the current query sampler, which samples a field element and maps it through *to-nat* and modulo reduction. A direct sampler over the query sample space, or a rejection sampler with a proved distribution, would remove this divisibility condition from the protocol locale.

and *powers-le-clength*: $powers \leq clength$

and *powers-pos*: $0 < powers$

and *query-sample-space-size-dvd*:

$clength * scale - Max (set [0..<powers]) * scale \text{ dvd } size$

— Exact-uniformity assumption for modulo reduction into the current query sample space.

It is specific to the current field-element encoding based sampler.

and *shift-nonzero*: $shift \neq 0$

and *two-nonzero*: $(2::'f) \neq 0$

and *spec-degree-fits*:

$\bigwedge c \text{ roots } d.$

$(c, roots, d) \in set \text{ spec} \implies$

$d * (clength - 1) - length \text{ roots} < clength * scale$

and *constraint-degree-wellformed-raw*:

$\bigwedge f c \text{ roots } d.$

$degree \text{ f} < clength \implies$

$(c, roots, d) \in set \text{ spec} \implies$

$degree$

$(c \text{ (map}$

$(\lambda p. f \circ_p XP ((\omega \wedge ((size - 1) \text{ div } clength)) \wedge p))$

$[0..<powers]))$

$\leq d * (clength - 1)$

and *spec-roots-in-range*:

$\bigwedge c \text{ roots } d \text{ r.}$

$(c, roots, d) \in set \text{ spec} \implies$

$r \in set \text{ roots} \implies$

$r < clength$

and *spec-roots-distinct*:

$\bigwedge c \text{ roots } d.$

$(c, roots, d) \in set \text{ spec} \implies$

$distinct \text{ roots}$

and *shift-notin-H-raw*:

$shift \notin$

```

      set (map (( $\wedge$ ) (omega  $\wedge$  ((size - 1) div (clength * scale))))
        [0.. $\leq$ scale * clength])
    and to-nat-of-nat-bounded:  $\bigwedge n. n \leq \text{Max } (\text{degrees } \text{clength } \text{spec}) \implies \text{to-nat } (\text{of-nat } n) = n$ 
begin

```

1.38.1 Lemmas

```

lemma scale-pos: 0 < scale
  using scale-pow
  by auto

```

```

lemma clength-pos: 0 < clength
  using clength-pow by auto

```

```

lemma eval-domain-size-pos: 0 < clength * scale
  using eval-domain-nontrivial by linarith

```

```

lemma of-nat-to-nat-field:
  of-nat (to-nat x) = x
  using of-nat-to-nat by simp

```

```

lemma to-nat-inj:
  inj to-nat
proof (rule injI)
  fix x y
  assume eq: to-nat x = to-nat y
  have of-nat (to-nat x) = of-nat (to-nat y)
    using eq by simp
  then show x = y
    using of-nat-to-nat-field by simp
qed

```

```

lemma finite-range-to-nat[simp]:
  finite (range to-nat)
  by simp

```

```

lemma card-range-to-nat:
  card (range to-nat) = size
proof -
  have range-eq: range to-nat = to-nat ' (UNIV :: 'f set)
    by blast
  have inj-univ: inj-on to-nat (UNIV :: 'f set)
    using to-nat-inj by (simp add: inj-on-def inj-def)
  have card-img:
    card (to-nat ' (UNIV :: 'f set)) = card (UNIV :: 'f set)
    by (rule card-image[OF inj-univ])
  have card-univ: card (UNIV :: 'f set) = size
    using size-card by simp
  show ?thesis

```

using *range-eq card-img card-univ* **by** *simp*
qed

lemma *eval-domain-length-power*: $\exists n. \text{clength} * \text{scale} = 2^n$

proof —

obtain $k\ l$ **where** $\text{clength} = 2^k$ **and** $\text{scale} = 2^l$
using *clength-pow scale-pow* **by** *blast*
then have $\text{clength} * \text{scale} = 2^{(k + l)}$
by (*simp add: power-add*)
then show *?thesis*
by *blast*

qed

lemma *omega-power-size-minus-one*:

$\text{omega}^{(\text{size} - 1)} = 1$

using *omega-order unfolding exact-order-def* **by** *simp*

lemma *omega-nonzero-derived*:

$\text{omega} \neq 0$

proof

assume *omega-zero*: $\text{omega} = 0$
have *size-pos*: $0 < \text{size} - 1$
using *omega-order unfolding exact-order-def* **by** *simp*
then obtain n **where** *size-eq*: $\text{size} - 1 = \text{Suc } n$
by (*cases size - 1*) *auto*
have $(0 :: 'f) = 1$
using *omega-power-size-minus-one omega-zero size-eq* **by** *simp*
then show *False*
by *simp*

qed

1.38.2 Basic definitions

G is a subgroup of $'f^x$ (the multiplicative group of $'f$). It is generated by generator g and contains exactly as many elements as the length of the (padded) execution trace.

definition g

where

$g \equiv \text{omega}^{((\text{size} - 1) \text{ div } \text{clength})}$

definition *trace-powers-of* :: $'f \text{ poly} \Rightarrow 'f \text{ poly list}$

where *trace-powers-of* $f = \text{map } (\lambda p. f \circ_p XP (g \wedge p)) [0..<\text{powers}]$

lemma *trace-powers-of-length*[*simp*]:

$\text{length } (\text{trace-powers-of } f) = \text{powers}$

unfolding *trace-powers-of-def* **by** *simp*

lemma *constraint-degree-wellformed*:

— This locale fact records a syntactic degree side condition for *spec*. A stronger syntactic degree-analysis interface could derive it in a later refinement of the specification layer.

assumes $\text{degree } f < \text{clength}$
and $(c, \text{roots}, d) \in \text{set spec}$
shows $\text{degree } (c (\text{trace-powers-of } f)) \leq d * (\text{clength} - 1)$
using $\text{constraint-degree-wellformed-raw}[OF \text{ assms}]$
unfolding $\text{trace-powers-of-def } g\text{-def}$.

definition G

where

$G = \text{map } ((\frown) g) [0..<\text{clength}]$

H is again subgroup of f^x . Its size is the size of G scaled by the scaling factor scale . It is generated by generator h .

definition h

where

$h \equiv \text{omega}^\wedge((\text{size} - 1) \text{ div } (\text{clength} * \text{scale}))$

definition H

where

$H = \text{map } ((\frown) h) [0..<\text{scale} * \text{clength}]$ — Since both scale and clength are powers of two their product is a power of two as well

lemma $\text{shift-notin-}H$:

$\text{shift} \notin \text{set } H$

using $\text{shift-notin-}H\text{-raw}$

unfolding $H\text{-def } h\text{-def}$.

We generate a Coset(<https://en.wikipedia.org/wiki/Coset>) of H for our evaluation domain by multiplying each element with a nonzero shift.

definition eval-domain

where

$\text{eval-domain} = \text{map } ((* \text{shift}) H$

This function creates the indices which need to be evaluated for the de-commitment phase. Note that the verifier needs to obtain values of f for all the powers in powers to compute the constraint polynomial. Also, in the flat representation a power actually means that the index is multiplied by the scale factor.

definition powers-scaled

where

$\text{powers-scaled } \text{idx} \equiv \text{map } (\lambda x. \text{idx} + (x * \text{scale})) [0 ..< \text{powers}]$

definition index

where

$\text{index } x = x \text{ mod } ((\text{clength} * \text{scale}) - ((\text{Max } (\text{set } [0 ..< \text{powers}])) * \text{scale}))$

definition $g\text{-map}$

where

$g\text{-map } rs = (\text{map } ((\frown) g) rs)$

```

lemma domain-alignment:
   $g = h \wedge \text{scale}$ 
proof –
  let ?N = clength * scale
  obtain q where q-def:  $\text{size} - 1 = ?N * q$ 
    using eval-domain-dvd unfolding dvd-def by blast
  have div-mult:
     $(\text{size} - 1) \text{ div } \text{clength} = ((\text{size} - 1) \text{ div } (\text{clength} * \text{scale})) * \text{scale}$ 
    using q-def scale-pos clength-pos
    by (simp add: mult.assoc)
  have  $h \wedge \text{scale} = \omega^{\wedge}(((\text{size} - 1) \text{ div } (\text{clength} * \text{scale})) * \text{scale})$ 
    unfolding h-def by (simp add: power-mult)
  also have ... =  $\omega^{\wedge}((\text{size} - 1) \text{ div } \text{clength})$ 
    using div-mult by simp
  finally show ?thesis
    unfolding g-def by simp
qed

lemma g-exact-order:
  exact-order g clength
proof –
  let ?N = clength
  let ?M = size - 1
  have dvd-clength: clength dvd ?M
    using eval-domain-dvd by (meson dvd-mult-left dvd-trans)
  obtain q where q-def:  $?M = ?N * q$ 
    using dvd-clength unfolding dvd-def by blast
  have N-pos:  $0 < ?N$ 
    using clength-pos .
  have q-pos:  $0 < q$ 
    using q-def omega-order unfolding exact-order-def by (cases q) auto
  have gN:  $g \wedge ?N = 1$ 
proof –
  have  $g \wedge ?N = \omega^{\wedge}(((\text{size} - 1) \text{ div } ?N) * ?N)$ 
    unfolding g-def by (simp add: power-mult)
  also have ... =  $\omega^{\wedge}(\text{size} - 1)$ 
    using q-def N-pos by (simp add: mult.commute)
  also have ... = 1
    by (rule omega-power-size-minus-one)
  finally show ?thesis .
qed
have no-smaller:  $\bigwedge m. 0 < m \wedge m < ?N \implies g \wedge m \neq 1$ 
proof
  fix m
  assume m-bounds:  $0 < m \wedge m < ?N$ 
  assume gm:  $g \wedge m = 1$ 
  have exp-lt:  $q * m < ?M$ 
    using m-bounds q-def q-pos by (simp add: mult-less-mono2)
  have exp-pos:  $0 < q * m$ 

```

```

    using m-bounds q-pos by simp
  have g-eq:  $g = \omega^q$ 
    using q-def N-pos unfolding g-def by simp
  have  $\omega^{q * m} = 1$ 
    using gm g-eq by (simp add: power-mult)
  then show False
    using omega-order exp-pos exp-lt unfolding exact-order-def by blast
qed
show ?thesis
  unfolding exact-order-def using N-pos gN no-smaller by blast
qed

```

```

lemma g-nonzero:
   $g \neq 0$ 
  unfolding g-def using omega-nonzero-derived by simp

```

```

lemma g-power-inj-on-range:
  assumes i-bound:  $i < \text{clength}$ 
    and j-bound:  $j < \text{clength}$ 
    and eq:  $g^i = g^j$ 
  shows  $i = j$ 
proof (rule ccontr)
  assume neg:  $i \neq j$ 
  consider  $i < j \mid j < i$ 
    using neg by linarith
  then show False
proof cases
  case 1
  have diff-pos:  $0 < j - i$ 
    using 1 by simp
  have diff-lt:  $j - i < \text{clength}$ 
    using j-bound by simp
  have gi-nonzero:  $g^i \neq 0$ 
    using g-nonzero by simp
  have j-decomp:  $j = i + (j - i)$ 
    using 1 by simp
  have mult-eq:  $g^j = g^i * g^{(j - i)}$ 
    by (subst j-decomp) (simp add: power-add)
  have cancel-eq:  $g^i * 1 = g^i * g^{(j - i)}$ 
    using eq mult-eq by simp
  have  $g^i = 0 \vee 1 = g^{(j - i)}$ 
    using cancel-eq by (simp only: mult-cancel-left)
  then have  $g^{(j - i)} = 1$ 
    using g-nonzero by auto
  then show False
    using g-exact-order diff-pos diff-lt unfolding exact-order-def by blast
next
  case 2
  have diff-pos:  $0 < i - j$ 

```

```

    using 2 by simp
  have diff-lt:  $i - j < \text{clength}$ 
    using i-bound by simp
  have gj-nonzero:  $g^j \neq 0$ 
    using g-nonzero by simp
  have i-decomp:  $i = j + (i - j)$ 
    using 2 by simp
  have mult-eq:  $g^i = g^j * g^{(i - j)}$ 
    by (subst i-decomp) (simp add: power-add)
  have cancel-eq:  $g^j * 1 = g^j * g^{(i - j)}$ 
    using eq mult-eq by simp
  have  $g^j = 0 \vee 1 = g^{(i - j)}$ 
    using cancel-eq by (simp only: mult-cancel-left)
  then have  $g^{(i - j)} = 1$ 
    using g-nonzero by auto
  then show False
    using g-exact-order diff-pos diff-lt unfolding exact-order-def by blast
qed

```

lemma *h-order*:

$h^{(\text{clength} * \text{scale})} = 1$

proof –

let $?N = \text{clength} * \text{scale}$

obtain q **where** $q\text{-def}$: $\text{size} - 1 = ?N * q$

using eval-domain-dvd unfolding dvd-def by blast

have $h^{?N} = \text{omega}^{(((\text{size} - 1) \text{ div } ?N) * ?N)}$

unfolding h-def by (simp add: power-mult)

also have $\dots = \text{omega}^{(\text{size} - 1)}$

using q-def eval-domain-size-pos by (simp add: mult.commute)

also have $\dots = 1$

by (rule omega-power-size-minus-one)

finally show ?thesis .

qed

lemma *h-exact-order*:

$\text{exact-order } h (\text{clength} * \text{scale})$

proof –

let $?N = \text{clength} * \text{scale}$

let $?M = \text{size} - 1$

obtain q **where** $q\text{-def}$: $?M = ?N * q$

using eval-domain-dvd unfolding dvd-def by blast

have $N\text{-pos}$: $0 < ?N$

using eval-domain-size-pos .

have $q\text{-pos}$: $0 < q$

using q-def omega-order unfolding exact-order-def by (cases q) auto

have hN : $h^{?N} = 1$

by (rule h-order)

have no-smaller: $\bigwedge m. 0 < m \wedge m < ?N \implies h^m \neq 1$

```

proof
  fix m
  assume m-bounds:  $0 < m \wedge m < ?N$ 
  assume hm:  $h \wedge m = 1$ 
  have exp-lt:  $q * m < ?M$ 
    using m-bounds q-def q-pos by (simp add: mult-less-mono2)
  have exp-pos:  $0 < q * m$ 
    using m-bounds q-pos by simp
  have h-eq:  $h = \text{omega} \wedge q$ 
    using q-def N-pos unfolding h-def by simp
  have omega- $\wedge$ ( $q * m$ ) = 1
    using hm h-eq by (simp add: power-mult)
  then show False
    using omega-order exp-pos exp-lt unfolding exact-order-def by blast
qed
show ?thesis
  unfolding exact-order-def using N-pos hN no-smaller by blast
qed

lemma square-eq-one-imp-eq-one-or-neg-one:
  fixes x :: 'f
  assumes  $x * x = 1$ 
  shows  $x = 1 \vee x = -1$ 
proof -
  have  $(x - 1) * (x + 1) = 0$ 
    using assms by (simp add: algebra-simps)
  then have  $x - 1 = 0 \vee x + 1 = 0$ 
    by (simp add: mult-eq-0-iff)
  then show ?thesis
proof
  assume  $x - 1 = 0$ 
  then show ?thesis by simp
next
  assume  $x + 1 = 0$ 
  then have  $x = -1$ 
    by (simp add: add-eq-0-iff2)
  then show ?thesis by simp
qed
qed

lemma h-half-order:
   $h \wedge ((\text{clength} * \text{scale}) \text{div } 2) = -1$ 
proof -
  let ?N =  $\text{clength} * \text{scale}$ 
  obtain k l where clength-eq:  $\text{clength} = 2 \wedge k$  and scale-eq:  $\text{scale} = 2 \wedge l$ 
    using clength-pow scale-pow by blast
  have N-power:  $?N = 2 \wedge (k + l)$ 
    using clength-eq scale-eq by (simp add: power-add)
  have N-gt-one:  $1 < ?N$ 

```



```

    using eval-domain-nontrivial .
  then have exp-pos:  $0 < ?N \text{ div } 2$ 
    by simp
  have exp-lt:  $?N \text{ div } 2 < ?N$ 
    using N-gt-one by simp
  have h-exp-ne-one:  $h ^{(?N \text{ div } 2)} \neq 1$ 
    using h-exact-order exp-pos exp-lt unfolding exact-order-def by blast
  have square-one:  $(h ^{(?N \text{ div } 2)}) * (h ^{(?N \text{ div } 2)}) = 1$ 
  proof -
    have even-N: even  $?N$ 
      using N-power N-gt-one by (cases  $k + l$ ) simp-all
    have  $(h ^{(?N \text{ div } 2)}) * (h ^{(?N \text{ div } 2)}) = h ^{(?N \text{ div } 2 + ?N \text{ div } 2)}$ 
      by (simp add: power-add)
    also have  $\dots = h ^{?N}$ 
  proof -
    have  $?N \text{ div } 2 + ?N \text{ div } 2 = ?N$ 
      using even-two-times-div-two[OF even-N] by presburger
    then show ?thesis
      by simp
  qed
  also have  $\dots = 1$ 
    by (rule h-order)
  finally show ?thesis .
qed
show ?thesis
  using square-eq-one-imp-eq-one-or-neg-one[OF square-one] h-exp-ne-one by blast
qed

```

lemma *h-power-mod-domain*:

$$h ^{(k \bmod (\text{clength} * \text{scale}))} = h ^k$$

proof -

let $?N = \text{clength} * \text{scale}$

have order: $h ^{?N} = 1$

using h-order .

have k-eq: $k = (k \text{ div } ?N) * ?N + k \bmod ?N$

using div-mult-mod-eq[of k $?N$] by (simp add: mult.commute)

have period: $h ^{((k \text{ div } ?N) * ?N)} = 1$

proof -

have $h ^{((k \text{ div } ?N) * ?N)} = h ^{(?N * (k \text{ div } ?N))}$

by (simp add: mult.commute)

also have $\dots = (h ^{?N}) ^{(k \text{ div } ?N)}$

by (simp only: power-mult)

also have $\dots = 1$

using order by simp

finally show ?thesis .

qed

have $h ^k = h ^{((k \text{ div } ?N) * ?N + k \bmod ?N)}$

using k-eq by simp

also have $\dots = h ^{((k \text{ div } ?N) * ?N)} * h ^{(k \bmod ?N)}$

```

    by (simp add: power-add)
  also have ... =  $h \wedge (k \bmod ?N)$ 
    using period by simp
  finally show ?thesis
    by simp
qed

lemma h-power-in-H:
   $h \wedge k \in \text{set } H$ 
proof -
  let ?N = clength * scale
  have mod-bound:  $k \bmod ?N < \text{scale} * \text{clength}$ 
    using eval-domain-size-pos
    by (simp add: mult.commute)
  have  $h \wedge (k \bmod ?N) \in \text{set } H$ 
    unfolding H-def using mod-bound by simp
  then show ?thesis
    using h-power-mod-domain[of k] by simp
qed

lemma eval-coset-disjoint-from-H:
  assumes idx-bound:  $i < \text{clength} * \text{scale}$ 
  shows  $h \wedge i * \text{shift} \notin \text{set } H$ 
proof
  let ?N = clength * scale
  assume in-H:  $h \wedge i * \text{shift} \in \text{set } H$ 
  then obtain j where j-bound:  $j < ?N$ 
    and eq:  $h \wedge i * \text{shift} = h \wedge j$ 
    unfolding H-def by (auto simp: mult.commute)
  have h-nonzero:  $h \neq 0$ 
    unfolding h-def using omega-nonzero-derived by simp
  have h-i-nonzero:  $h \wedge i \neq 0$ 
    using h-nonzero by simp
  have inverse-eq:  $\text{inverse } (h \wedge i) = h \wedge (?N - i)$ 
proof (rule inverse-unique)
  have  $h \wedge i * h \wedge (?N - i) = h \wedge (i + (?N - i))$ 
    by (simp add: power-add)
  also have ... =  $h \wedge ?N$ 
    using idx-bound by simp
  also have ... = 1
    using h-order .
  finally show  $h \wedge i * h \wedge (?N - i) = 1$  .
qed
have shift-eq:  $\text{shift} = h \wedge j * \text{inverse } (h \wedge i)$ 
proof -
  have eq':  $\text{shift} * h \wedge i = h \wedge j$ 
    using eq by (simp only: mult.commute)
  have  $\text{shift} = \text{shift} * (h \wedge i * \text{inverse } (h \wedge i))$ 
    using h-i-nonzero by simp

```

```

    also have ... = shift * h ^ i * inverse (h ^ i)
      by (simp add: mult.assoc)
    also have ... = h ^ j * inverse (h ^ i)
      using eq' by simp
    finally show ?thesis .
  qed
  have shift = h ^ (j + (?N - i))
    using shift-eq inverse-eq by (simp add: power-add)
  then have shift ∈ set H
    using h-power-in-H[of j + (?N - i)] by simp
  then show False
    using shift-notin-H by contradiction
  qed

lemma clength-lt-size:
  clength < size
proof -
  have size-minus-pos: 0 < size - 1
    using omega-order unfolding exact-order-def by simp
  have size-minus-ge: clength * scale ≤ size - 1
    using eval-domain-dvd eval-domain-size-pos size-minus-pos
    by (meson dvd-imp-le)
  have clength ≤ clength * scale
    using scale-pos by simp
  also have ... ≤ size - 1
    using size-minus-ge .
  finally show ?thesis
    using size-minus-pos by linarith
  qed

definition maxDegree
  where maxDegree = Max (degrees clength spec)

lemma maxDegree-code[code]:
  maxDegree = foldl (λx (-,rs,d). max x (d * (clength - 1) - length rs)) 0 spec
proof -
  let ?deg = λ(-, rs, d). d * (clength - 1) - length rs
  have Max-insert-max:
    finite A ⟹ Max (insert (max x y) A) = Max (insert x (insert y A))
    for A :: nat set and x y
    by (cases A = {}) (auto)
  have deg-set: {d * (clength - 1) - length rs | a rs d. (a,rs,d) ∈ set spec} =
    ?deg ' set spec
    by force
  have aux: foldl (λx s. max x (?deg s)) n spec = Max (insert n (?deg ' set spec))
    for n
  proof (induction spec arbitrary: n)
    case Nil
    then show ?case by simp
  end

```

```

next
  case (Cons s spec)
  then show ?case
    by (simp add: Max-insert-max)
qed
from aux[of 0] show ?thesis
  unfolding maxDegree-def using deg-set by (simp add: degrees-def split-beta')
qed

```

```

lemma maxDegree-less-eval-domain:
  maxDegree < clength * scale
proof -
  have finite-degrees: finite (degrees clength spec)
  proof -
    have {d * (clength - 1) - length roots | c roots d. (c, roots, d) ∈ set spec} =
      (λ(c, roots, d). d * (clength - 1) - length roots) ‘ set spec
    by force
  then show ?thesis
    unfolding degrees-def by simp
  qed
  have nonempty-degrees: degrees clength spec ≠ {}
  unfolding degrees-def by simp
  have all-lt: ∧x. x ∈ degrees clength spec ⇒ x < clength * scale
  unfolding degrees-def
  using eval-domain-size-pos spec-degree-fits
  by force
  then show ?thesis
    unfolding maxDegree-def
    using finite-degrees nonempty-degrees
    by (simp add: Max-less-iff)
qed

```

```

lemma g-map-distinct:
  assumes spec-entry: (c, roots, d) ∈ set spec
  shows distinct (g-map roots)
proof -
  have roots-distinct: distinct roots
  by (rule spec-roots-distinct[OF spec-entry])
  have roots-in-range: ∧r. r ∈ set roots ⇒ r < clength
  by (rule spec-roots-in-range[OF spec-entry])
  have inj: inj-on (( $\cap$ ) g) (set roots)
  using roots-in-range by (auto intro!: inj-onI dest!: g-power-inj-on-range)
  show ?thesis
    unfolding g-map-def
    using roots-distinct inj by (simp add: distinct-map)
  qed

```

```

lemma g-powers-in-H:
  assumes r-bound: r < clength

```

```

  shows  $g \wedge r \in \text{set } H$ 
proof -
  have idx-bound:  $\text{scale} * r < \text{scale} * \text{clength}$ 
    using r-bound scale-pos by simp
  have  $g \wedge r = h \wedge (\text{scale} * r)$ 
    using domain-alignment by (simp add: power-mult)
  then show ?thesis
    unfolding H-def using idx-bound by simp
qed

```

```

lemma g-map-subset-H:
  assumes spec-entry:  $(c, \text{roots}, d) \in \text{set spec}$ 
  shows  $\text{set } (g\text{-map roots}) \subseteq \text{set } H$ 
  unfolding g-map-def
  using spec-roots-in-range[OF spec-entry] g-powers-in-H
  by auto

```

```

lemma query-domain-disjoint:
  assumes spec-entry:  $(c, \text{roots}, d) \in \text{set spec}$ 
    and idx-bound:  $i < \text{clength} * \text{scale}$ 
  shows  $h \wedge i * \text{shift} \notin \text{set } (g\text{-map roots})$ 
  using eval-coset-disjoint-from-H[OF idx-bound] g-map-subset-H[OF spec-entry]
  by auto

```

1.38.3 Composition polynomial

The composition polynomial is a random linear combination of the constraint polynomials divided by their roots. Note that in reality we don't compute the product polynomial as here. Instead we use a trick to optimize the computation (described here under Succinctness: <https://medium.com/starkware/arithmeticization-ii-403c3b3f4355>) Also note that it needs to be parametrized by *f* since the prover uses it with the real *f* whereas the verifier uses another *f*. Maybe we need different versions for prover and verifier later on.

```

definition cp
where
   $cp \text{ as } fs \equiv \text{fold } (\lambda(a, (p, rs, -)). (+) ((CP \ a) * ((p \ fs) \text{ div } (\text{prod } (g\text{-map } rs)))))) (\text{zip as spec}) \ 0$ 
end

```

1.39 Prover

Only the prover knows the actual execution trace.

```

locale prover = stark +
  constrains omega::'f::proth-field
    and shift::'f
    and concat :: 'f  $\Rightarrow$  'f  $\Rightarrow$  'f
    and spec :: (('f poly list  $\Rightarrow$  'f poly)  $\times$  nat list  $\times$  nat) list

```

fixes *vals::'f list* — This is the actual computation trace

assumes *trace-length: clength = length vals*

begin

1.39.1 Trace Polynomial

The interpolated polynomial from the trace over G . We do it using Lagrange interpolation. In practice it is usually done using IFFT (inverse FFT – Fast Fourier) but could not find it in the AFP.

definition *f*

where

f $\equiv$ *lagrange-interpolation-poly* (*zip G vals*)

Now we evaluate the trace polynomial on the larger domain, creating a Reed-Solomon error correction code.

definition *f-eval*

where

f-eval $\equiv$ *map* (*poly f*) *eval-domain*

Similar as the trace polynomial we evaluate the composition polynomial on the larger domain.

definition *f-powers*

where

f-powers $\equiv$ *map* ($\lambda p. f \circ_p XP (g \hat{p})$) [*0* ..< *powers*]

definition *cp-eval*

where

cp-eval as $\equiv$ *map* (*poly* (*cp as f-powers*)) *eval-domain*

1.39.2 FRI Commitments

definition *next-fri-domain* :: *'f list* $\Rightarrow$ *'f list*

where

next-fri-domain fd $\equiv$ *map* ($\lambda x. x * x$) (*take* ((*length fd*) *div* (*2::nat*)) *fd*)

definition *next-fri-polynomial* :: *'f poly* $\Rightarrow$ *'f* $\Rightarrow$ *'f poly*

where

next-fri-polynomial p b $\equiv$

let

c = *coeffs p*;

odd-coefficients = *nths-pred c odd*;

even-coefficients = *nths-pred c even*;

odd = *CP b * poly-of-list odd-coefficients*;

even = *poly-of-list even-coefficients*
in odd + *even*

lemma *degree-Poly-less*:

assumes *degree p > 0*

and *length xs < length (coeffs p)*

shows *degree (Poly xs) < degree p*

proof –

from *assms(2)* **have** *length (strip-while (HOL.eq 0) xs) < length (coeffs p)*

by (*meson dual-order.strict-trans1 length-strip-while-le verit-comp-simplify1(3)*)

then have *degree (Poly (strip-while (HOL.eq 0) xs)) < degree p*

using *length-coeffs[of (Poly (nth-pred (coeffs p) even))]*

by (*metis Poly-coeffs add-less-cancel-right assms(1) coeffs-Poly degree-0 length-coeffs not-gr-zero*)

then show *degree (Poly xs) < degree p*

by (*metis Poly-coeffs coeffs-Poly*)

qed

lemma *degree-less-degree*:

assumes *degree p > 0*

and *next-fri-polynomial p b = p'*

shows *degree p' < degree p*

proof –

have *: *length (coeffs p) > 1* **using** *assms length-coeffs*

by (*simp add: degree-eq-length-coeffs*)

define *odd-coefficients* **where** *odd-coefficients* $\equiv$ *nth-pred (coeffs p) odd*

define *oddp* **where** *oddp* $\equiv$ *CP b * Poly odd-coefficients*

define *even-coefficients* **where** *even-coefficients* $\equiv$ *nth-pred (coeffs p) even*

define *evenp* **where** *evenp* $\equiv$ *Poly even-coefficients*

from * **have** *length odd-coefficients < length (coeffs p)*

using *nth-pred-odd-length-less odd-coefficients-def* **by** *simp*

then have *degree (Poly (odd-coefficients)) < degree p*

using *assms(1) degree-Poly-less* **by** *metis*

moreover have *degree (CP b) = 0* **unfolding** *CP-def* **by** (*simp add: monom-0*)

ultimately have *degree oddp < degree p*

unfolding *oddp-def* **using** *degree-mult-le[of CP b (Poly odd-coefficients)]* **by** *simp*

moreover from * **have** *length even-coefficients < length (coeffs p)*

using *nth-pred-even-length-less even-coefficients-def* **by** *simp*

then have *degree (Poly (even-coefficients)) < degree p*

using *assms(1) degree-Poly-less* **by** *metis*

then have *degree evenp < degree p* **unfolding** *evenp-def* **by** *satx*

moreover have *p' = oddp + evenp*

using *assms*

unfolding *oddp-def evenp-def odd-coefficients-def even-coefficients-def next-fri-polynomial-def*

by (*auto simp add: Let-def*)

ultimately show *?thesis* **by** (*simp add: degree-add-less*)
qed

definition *next-fri-layer*:: 'f poly $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$ 'f poly $\times$ 'f list $\times$ 'f list
where

next-fri-layer p d b $\equiv$
 let
 next-poly = *next-fri-polynomial* p b;
 next-domain = *next-fri-domain* d;
 next-layer = map (*poly next-poly*) *next-domain*
 in (*next-poly*, *next-domain*, *next-layer*)

primrec *fri-commit-with* ::

('f, 'f, unit) protocol-c-monad
 $\Rightarrow$ nat
 $\Rightarrow$ 'f poly list
 $\Rightarrow$ 'f list list
 $\Rightarrow$ 'f list list
 $\Rightarrow$ 'f tree list
 $\Rightarrow$ ('f, 'f poly list $\times$ 'f list list $\times$ 'f list list $\times$ 'f tree list, unit) protocol-c-monad

where

fri-commit-with receive-challenge 0 ps ds ls ms =
 return (ps, ds, ls, ms)
| *fri-commit-with* receive-challenge (Suc n) ps ds ls ms =
 do {
 send (value (last ms));
 b $\leftarrow$ receive-challenge;
 let (*next-poly*, *next-domain*, *next-layer*) =
 next-fri-layer (last ps) (last ds) b;
 m $\leftarrow$ create *next-layer*;
 fri-commit-with receive-challenge n
 (ps @ [next-poly])
 (ds @ [next-domain])
 (ls @ [next-layer])
 (ms @ [m])
 }

primrec *fri-commit* ::

nat
 $\Rightarrow$ 'f poly list
 $\Rightarrow$ 'f list list
 $\Rightarrow$ 'f list list
 $\Rightarrow$ 'f tree list
 $\Rightarrow$ ('f, 'f poly list $\times$ 'f list list $\times$ 'f list list $\times$ 'f tree list, unit) protocol-c-monad

where

fri-commit 0 ps ds ls ms = return (ps, ds, ls, ms)
| *fri-commit* (Suc n) ps ds ls ms =
 do {
 send (value (last ms));


```

  b ← receive-random-field-element;
  let (next-poly, next-domain, next-layer) = next-fri-layer (last ps) (last ds) b;
  m ← create next-layer;
  fri-commit n
    (ps @ [next-poly])
    (ds @ [next-domain])
    (ls @ [next-layer])
    (ms @ [m])
}

```

definition *trace-fri-commit*

where

```

  trace-fri-commit =
    fri-commit-with receive-trace-fri-challenge

```

definition *composition-fri-commit*

where

```

  composition-fri-commit =
    fri-commit-with receive-composition-fri-challenge

```

lemma *fri-commit-with-receive-random-field-element:*

```

  fri-commit-with receive-random-field-element n ps ds ls ms =
    fri-commit n ps ds ls ms
  by (induction n arbitrary: ps ds ls ms) simp-all

```

lemma *fri-commit-lengths:*

```

  assumes Some ((ps', ds', ls', ms'), t) ∈
    set-dist (execute (fri-commit n ps ds ls ms) s)

```

shows

```

  length ps' = length ps + n ∧
  length ds' = length ds + n ∧
  length ls' = length ls + n ∧
  length ms' = length ms + n

```

using *assms*

proof (induction n arbitrary: ps ds ls ms s ps' ds' ls' ms' t)

case 0

then show ?case **by** *simp*

next

case (Suc n)

from *Suc.premis* **obtain** s1 b s2 next-poly next-domain next-layer m s3 **where**

send-root: Some ((), s1) ∈ set-dist (execute (send (value (last ms)))) s)

and *rand*: Some (b, s2) ∈ set-dist (execute receive-random-field-element s1)

and *next-layer-eq*: next-fri-layer (last ps) (last ds) b = (next-poly, next-domain, next-layer)

and *create-m*: Some (m, s3) ∈ set-dist (execute (create next-layer) s2)

and *rest*: Some ((ps', ds', ls', ms'), t) ∈

set-dist (execute

(fri-commit n

(ps @ [next-poly])

(ds @ [next-domain])

```

      (ls @ [next-layer])
      (ms @ [m])) s3)
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
show ?case
  using Suc.IH[OF rest] by simp
qed

lemma fri-commit-extends:
  assumes Some ((ps', ds', ls', ms'), t) ∈
    set-dist (execute (fri-commit n ps ds ls ms) s)
  shows s ≤ t
  using assms
proof (induction n arbitrary: ps ds ls ms s ps' ds' ls' ms' t)
  case 0
  then show ?case
    using hash-ext-refl by simp
next
  case (Suc n)
  from Suc.premis obtain s1 b s2 next-poly next-domain next-layer m s3 where
    send-root: Some ((), s1) ∈ set-dist (execute (send (value (last ms))) s)
    and rand: Some (b, s2) ∈ set-dist (execute receive-random-field-element s1)
    and next-layer-eq: next-fri-layer (last ps) (last ds) b = (next-poly, next-domain, next-layer)
    and create-m: Some (m, s3) ∈ set-dist (execute (create next-layer) s2)
    and rest: Some ((ps', ds', ls', ms'), t) ∈
      set-dist (execute
        (fri-commit n
          (ps @ [next-poly])
          (ds @ [next-domain])
          (ls @ [next-layer])
          (ms @ [m])) s3)
    by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
  have s-s1: s ≤ s1
    using send-outcome[OF send-root]
    unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  have s1-s2: s1 ≤ s2
    using receive-random-field-element-extends[OF rand] .
  have s2-s3: s2 ≤ s3
    using create-outcome[OF create-m] by simp
  have s3-t: s3 ≤ t
    using Suc.IH[OF rest] .
  show ?case
    using s-s1 s1-s2 s2-s3 s3-t by (meson hash-ext-trans)
qed

```

```

lemma fri-commit-preserves-prefix:
  assumes outcome:
    Some ((ps', ds', ls', ms'), t) ∈
      set-dist (execute (fri-commit n ps ds ls ms) s)
  shows

```

```

    take (length ps) ps' = ps ∧
    take (length ds) ds' = ds ∧
    take (length ls) ls' = ls ∧
    take (length ms) ms' = ms
  using outcome
proof (induction n arbitrary: ps ds ls ms s ps' ds' ls' ms' t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.premis obtain s1 b s2 next-poly next-domain next-layer m s3 where
    send-root: Some ((), s1) ∈ set-dist (execute (send (value (last ms))) s)
    and rand: Some (b, s2) ∈ set-dist (execute receive-random-field-element s1)
    and next-layer-eq: next-fri-layer (last ps) (last ds) b = (next-poly, next-domain, next-layer)
    and create-m: Some (m, s3) ∈ set-dist (execute (create next-layer) s2)
    and rest: Some ((ps', ds', ls', ms'), t) ∈
      set-dist (execute
        (fri-commit n
          (ps @ [next-poly])
          (ds @ [next-domain])
          (ls @ [next-layer])
          (ms @ [m])) s3)
    by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
  have ih: take (length (ps @ [next-poly])) ps' = ps @ [next-poly] ∧
    take (length (ds @ [next-domain])) ds' = ds @ [next-domain] ∧
    take (length (ls @ [next-layer])) ls' = ls @ [next-layer] ∧
    take (length (ms @ [m])) ms' = ms @ [m]
  using Suc.IH[OF rest] .
  have take (length ps) ps' = ps
  proof -
    have take (length ps) ps' =
      take (length ps) (take (length (ps @ [next-poly])) ps')
    by (simp add: take-take)
    also have ... = ps
    using ih by simp
    finally show ?thesis .
  qed
  moreover have take (length ds) ds' = ds
  proof -
    have take (length ds) ds' =
      take (length ds) (take (length (ds @ [next-domain])) ds')
    by (simp add: take-take)
    also have ... = ds
    using ih by simp
    finally show ?thesis .
  qed
  moreover have take (length ls) ls' = ls
  proof -
    have take (length ls) ls' =

```

```

      take (length ls) (take (length (ls @ [next-layer])) ls')
    by (simp add: take-take)
  also have ... = ls
    using ih by simp
  finally show ?thesis .
qed
moreover have take (length ms) ms' = ms
proof -
  have take (length ms) ms' =
    take (length ms) (take (length (ms @ [m])) ms')
  by (simp add: take-take)
  also have ... = ms
    using ih by simp
  finally show ?thesis .
qed
ultimately show ?case
  by simp
qed

lemma fri-commit-layers-are-evaluations:
  assumes init:
     $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$ 
     $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$ 
  and outcome:
    Some ((ps', ds', ls', ms'), t)  $\in$ 
    set-dist (execute (fri-commit n ps ds ls ms) s)
  and lens: length ps = length ds length ds = length ls
  shows
     $\bigwedge i. i < \text{length } ps' \implies i < \text{length } ds' \implies i < \text{length } ls' \implies$ 
     $ls' ! i = \text{map } (\text{poly } (ps' ! i)) (ds' ! i)$ 
  using init outcome lens
proof (induction n arbitrary: ps ds ls ms s ps' ds' ls' ms' t i)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.prem5 obtain s1 b s2 next-poly next-domain next-layer m s3 where
    send-root: Some ((), s1)  $\in$  set-dist (execute (send (value (last ms))) s)
    and rand: Some (b, s2)  $\in$  set-dist (execute receive-random-field-element s1)
    and next-layer-eq: next-fri-layer (last ps) (last ds) b = (next-poly, next-domain, next-layer)
    and create-m: Some (m, s3)  $\in$  set-dist (execute (create next-layer) s2)
    and rest: Some ((ps', ds', ls', ms'), t)  $\in$ 
      set-dist (execute
        (fri-commit n
          (ps @ [next-poly])
          (ds @ [next-domain])
          (ls @ [next-layer])
          (ms @ [m])) s3)
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)

```

```

have next-eval: next-layer = map (poly next-poly) next-domain
  using next-layer-eq unfolding next-fri-layer-def by (auto simp: Let-def split: prod.splits)
have init':
   $\bigwedge i. i < \text{length } (ps @ [next-poly]) \implies$ 
   $i < \text{length } (ds @ [next-domain]) \implies$ 
   $i < \text{length } (ls @ [next-layer]) \implies$ 
   $(ls @ [next-layer]) ! i =$ 
   $\text{map } (\text{poly } ((ps @ [next-poly]) ! i)) ((ds @ [next-domain]) ! i)$ 
proof -
  fix i
  assume i-ps:  $i < \text{length } (ps @ [next-poly])$ 
  and i-ds:  $i < \text{length } (ds @ [next-domain])$ 
  and i-ls:  $i < \text{length } (ls @ [next-layer])$ 
  show  $(ls @ [next-layer]) ! i =$ 
   $\text{map } (\text{poly } ((ps @ [next-poly]) ! i)) ((ds @ [next-domain]) ! i)$ 
proof (cases  $i < \text{length } ps$ )
  case True
  then show ?thesis
  using Suc.premis(4)[of i] Suc.premis(6,7)
  by (simp add: nth-append)
next
  case False
  then have i-eq:  $i = \text{length } ps$ 
  using i-ps by simp
  then show ?thesis
  using Suc.premis(6,7) next-eval by (simp add: nth-append)
qed
qed
show ?case
  using Suc.IH[OF - - init' rest] Suc.premis(1-3,6,7) by simp
qed

lemma fri-commit-created-trees:
assumes init:
   $\bigwedge i. i < \text{length } ls \implies i < \text{length } ms \implies$ 
   $\text{created-tree } (ls ! i) (ms ! i) (s::'f \text{ protocol-channel})$ 
and outcome:
   $\text{Some } ((ps', ds', ls', ms'), t) \in$ 
   $\text{set-dist } (\text{execute } (\text{fri-commit } n \ ps \ ds \ ls \ ms) \ s)$ 
and lens:  $\text{length } ls = \text{length } ms$ 
shows  $\bigwedge i. i < \text{length } ls' \implies i < \text{length } ms' \implies$ 
   $\text{created-tree } (ls' ! i) (ms' ! i) \ t$ 
using init outcome lens
proof (induction n arbitrary: ps ds ls ms s ps' ds' ls' ms' t i)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.premis(4) obtain s1 b s2 next-poly next-domain next-layer m s3 where

```

```

send-root: Some ((, s1) ∈ set-dist (execute (send (value (last ms))) s)
and rand: Some (b, s2) ∈ set-dist (execute receive-random-field-element s1)
and next-layer-eq: next-fri-layer (last ps) (last ds) b = (next-poly, next-domain, next-layer)
and create-m: Some (m, s3) ∈ set-dist (execute (create next-layer) s2)
and rest: Some ((ps', ds', ls', ms'), t) ∈
  set-dist (execute
    (fri-commit n
      (ps @ [next-poly])
      (ds @ [next-domain])
      (ls @ [next-layer])
      (ms @ [m])) s3)
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have s-s1: s ≤ s1
  using send-outcome[OF send-root]
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have s1-s2: s1 ≤ s2
  using receive-random-field-element-extends[OF rand] .
have s2-s3: s2 ≤ s3
  using create-outcome[OF create-m] by simp
have s-s3: s ≤ s3
  using s-s1 s1-s2 s2-s3 by (meson hash-ext-trans)
have created-m: created-tree next-layer m s3
  using create-outcome[OF create-m] by simp
have init':
  ∧i. i < length (ls @ [next-layer]) ⇒
    i < length (ms @ [m]) ⇒
    created-tree ((ls @ [next-layer]) ! i) ((ms @ [m]) ! i) s3
proof -
  fix i
  assume i-ls: i < length (ls @ [next-layer])
  and i-ms: i < length (ms @ [m])
  show created-tree ((ls @ [next-layer]) ! i) ((ms @ [m]) ! i) s3
  proof (cases i < length ls)
    case True
    then have created-tree (ls ! i) (ms ! i) s
      using Suc.premis(3)[of i] Suc.premis(5) by simp
    then show ?thesis
      using True Suc.premis(5) s-s3
      by (simp add: nth-append created-tree-mono)
  next
    case False
    then have i = length ls
      using i-ls by simp
    then show ?thesis
      using Suc.premis(5) created-m by (simp add: nth-append)
  qed
qed
show ?case
  using Suc.IH[OF Suc.premis(1,2) init' rest] Suc.premis(5) by simp

```

qed

lemma *fri-commit-with-lengths:*

assumes *Some ((ps', ds', ls', ms'), t) ∈ set-dist (execute (fri-commit-with receive-challenge n ps ds ls ms) s)*

shows

*length ps' = length ps + n ∧
length ds' = length ds + n ∧
length ls' = length ls + n ∧
length ms' = length ms + n*

using *assms*

proof (*induction n arbitrary: ps ds ls ms s ps' ds' ls' ms' t*)

case 0

then show *?case* **by** *simp*

next

case (*Suc n*)

from *Suc.premis* **obtain** *s1 b s2 next-poly next-domain next-layer m s3* **where**

send-root: Some ((), s1) ∈ set-dist (execute (send (value (last ms)))) s)

and *rand: Some (b, s2) ∈ set-dist (execute receive-challenge s1)*

and *next-layer-eq:*

*next-fri-layer (last ps) (last ds) b =
(next-poly, next-domain, next-layer)*

and *create-m: Some (m, s3) ∈ set-dist (execute (create next-layer) s2)*

and *rest: Some ((ps', ds', ls', ms'), t) ∈*

*set-dist (execute
(fri-commit-with receive-challenge n
(ps @ [next-poly])
(ds @ [next-domain])
(ls @ [next-layer])
(ms @ [m])) s3)*

by (*auto simp: Let-def split: prod.splits elim!: set-dist-bindE*)

show *?case*

using *Suc.IH[OF rest]* **by** *simp*

qed

lemma *fri-commit-with-extends:*

assumes *receiver-extends:*

*∧ b s t. Some (b, t) ∈ set-dist (execute receive-challenge s) ⇒
s ≤ t*

and *outcome:*

*Some ((ps', ds', ls', ms'), t) ∈
set-dist (execute (fri-commit-with receive-challenge n ps ds ls ms) s)*

shows *s ≤ t*

using *outcome*

proof (*induction n arbitrary: ps ds ls ms s ps' ds' ls' ms' t*)

case 0

then show *?case*

using *hash-ext-refl* **by** *simp*

next

case (*Suc n*)
from *Suc.prem*s **obtain** *s1 b s2 next-poly next-domain next-layer m s3* **where**
send-root: *Some* (*()*, *s1*) $\in$ *set-dist* (*execute* (*send* (*value* (*last ms*)))) *s*)
and *rand*: *Some* (*b*, *s2*) $\in$ *set-dist* (*execute receive-challenge s1*)
and *next-layer-eq*:
next-fri-layer (*last ps*) (*last ds*) *b* =
(*next-poly*, *next-domain*, *next-layer*)
and *create-m*: *Some* (*m*, *s3*) $\in$ *set-dist* (*execute* (*create next-layer*) *s2*)
and *rest*: *Some* (*(ps', ds', ls', ms')*, *t*) $\in$
set-dist (*execute*
(*fri-commit-with receive-challenge n*
(*ps* @ [*next-poly*])
(*ds* @ [*next-domain*])
(*ls* @ [*next-layer*])
(*ms* @ [*m*])) *s3*)
by (*auto simp: Let-def split: prod.splits elim!: set-dist-bindE*)
have *s-s1*: *s* $\leq$ *s1*
using *send-outcome*[*OF send-root*]
unfolding *less-eq-hash-ext-def less-eq-fmap-def* **by** *simp*
have *s1-s2*: *s1* $\leq$ *s2*
using *receiver-extends*[*OF rand*] .
have *s2-s3*: *s2* $\leq$ *s3*
using *create-outcome*[*OF create-m*] **by** *simp*
have *s3-t*: *s3* $\leq$ *t*
using *Suc.IH*[*OF rest*] .
show ?*case*
using *s-s1 s1-s2 s2-s3 s3-t* **by** (*meson hash-ext-trans*)
qed

lemma *fri-commit-with-preserves-prefix*:
assumes *outcome*:
Some (*(ps', ds', ls', ms')*, *t*) $\in$
set-dist (*execute* (*fri-commit-with receive-challenge n ps ds ls ms*) *s*)
shows
take (*length ps*) *ps'* = *ps* $\wedge$
take (*length ds*) *ds'* = *ds* $\wedge$
take (*length ls*) *ls'* = *ls* $\wedge$
take (*length ms*) *ms'* = *ms*
using *outcome*
proof (*induction n arbitrary: ps ds ls ms s ps' ds' ls' ms' t*)
case 0
then show ?*case* **by** *simp*
next
case (*Suc n*)
from *Suc.prem*s **obtain** *s1 b s2 next-poly next-domain next-layer m s3* **where**
send-root: *Some* (*()*, *s1*) $\in$ *set-dist* (*execute* (*send* (*value* (*last ms*)))) *s*)
and *rand*: *Some* (*b*, *s2*) $\in$ *set-dist* (*execute receive-challenge s1*)
and *next-layer-eq*:
next-fri-layer (*last ps*) (*last ds*) *b* =


```

    (next-poly, next-domain, next-layer)
and create-m: Some (m, s3) ∈ set-dist (execute (create next-layer) s2)
and rest: Some ((ps', ds', ls', ms'), t) ∈
  set-dist (execute
    (fri-commit-with receive-challenge n
      (ps @ [next-poly])
      (ds @ [next-domain])
      (ls @ [next-layer])
      (ms @ [m])) s3)
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have ih: take (length (ps @ [next-poly])) ps' = ps @ [next-poly] ∧
  take (length (ds @ [next-domain])) ds' = ds @ [next-domain] ∧
  take (length (ls @ [next-layer])) ls' = ls @ [next-layer] ∧
  take (length (ms @ [m])) ms' = ms @ [m]
using Suc.IH[OF rest] .
have take (length ps) ps' = ps
proof -
  have take (length ps) ps' =
    take (length ps) (take (length (ps @ [next-poly])) ps')
    by (simp add: take-take)
  also have ... = ps
    using ih by simp
  finally show ?thesis .
qed
moreover have take (length ds) ds' = ds
proof -
  have take (length ds) ds' =
    take (length ds) (take (length (ds @ [next-domain])) ds')
    by (simp add: take-take)
  also have ... = ds
    using ih by simp
  finally show ?thesis .
qed
moreover have take (length ls) ls' = ls
proof -
  have take (length ls) ls' =
    take (length ls) (take (length (ls @ [next-layer])) ls')
    by (simp add: take-take)
  also have ... = ls
    using ih by simp
  finally show ?thesis .
qed
moreover have take (length ms) ms' = ms
proof -
  have take (length ms) ms' =
    take (length ms) (take (length (ms @ [m])) ms')
    by (simp add: take-take)
  also have ... = ms
    using ih by simp

```

```

    finally show ?thesis .
qed
ultimately show ?case
  by simp
qed

lemma fri-commit-with-layers-are-evaluations:
  assumes init:
     $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$ 
     $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$ 
  and outcome:
     $\text{Some } ((ps', ds', ls', ms'), t) \in$ 
     $\text{set-dist } (\text{execute } (\text{fri-commit-with receive-challenge } n \ ps \ ds \ ls \ ms) \ s)$ 
  and lens:  $\text{length } ps = \text{length } ds \ \text{length } ds = \text{length } ls$ 
  shows
     $\bigwedge i. i < \text{length } ps' \implies i < \text{length } ds' \implies i < \text{length } ls' \implies$ 
     $ls' ! i = \text{map } (\text{poly } (ps' ! i)) (ds' ! i)$ 
  using init outcome lens
proof (induction n arbitrary: ps ds ls ms s ps' ds' ls' ms' t i)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.prem5 obtain s1 b s2 next-poly next-domain next-layer m s3 where
    send-root:  $\text{Some } ((), s1) \in \text{set-dist } (\text{execute } (\text{send } (\text{value } (\text{last } ms))) \ s)$ 
    and rand:  $\text{Some } (b, s2) \in \text{set-dist } (\text{execute receive-challenge } s1)$ 
    and next-layer-eq:
       $\text{next-fri-layer } (\text{last } ps) (\text{last } ds) \ b =$ 
       $(\text{next-poly}, \text{next-domain}, \text{next-layer})$ 
    and create-m:  $\text{Some } (m, s3) \in \text{set-dist } (\text{execute } (\text{create next-layer}) \ s2)$ 
    and rest:  $\text{Some } ((ps', ds', ls', ms'), t) \in$ 
     $\text{set-dist } (\text{execute}$ 
       $(\text{fri-commit-with receive-challenge } n$ 
         $(ps @ [\text{next-poly}])$ 
         $(ds @ [\text{next-domain}])$ 
         $(ls @ [\text{next-layer}])$ 
         $(ms @ [m])) \ s3)$ 
    by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
  have next-eval:  $\text{next-layer} = \text{map } (\text{poly next-poly}) \ \text{next-domain}$ 
  using next-layer-eq unfolding next-fri-layer-def
  by (auto simp: Let-def split: prod.splits)
  have init':
     $\bigwedge i. i < \text{length } (ps @ [\text{next-poly}]) \implies$ 
     $i < \text{length } (ds @ [\text{next-domain}]) \implies$ 
     $i < \text{length } (ls @ [\text{next-layer}]) \implies$ 
     $(ls @ [\text{next-layer}]) ! i =$ 
     $\text{map } (\text{poly } ((ps @ [\text{next-poly}]) ! i)) ((ds @ [\text{next-domain}]) ! i)$ 
  proof -
    fix i

```

```

assume  $i$ -ps:  $i < \text{length } (ps @ [\text{next-poly}])$ 
and  $i$ -ds:  $i < \text{length } (ds @ [\text{next-domain}])$ 
and  $i$ -ls:  $i < \text{length } (ls @ [\text{next-layer}])$ 
show  $(ls @ [\text{next-layer}]) ! i =$ 
   $\text{map } (\text{poly } ((ps @ [\text{next-poly}]) ! i)) ((ds @ [\text{next-domain}]) ! i)$ 
proof ( $\text{cases } i < \text{length } ps$ )
  case True
  then show ?thesis
    using  $\text{Suc.prem}(4)[\text{of } i] \text{ Suc.prem}(6,7)$ 
    by ( $\text{simp add: nth-append}$ )
  next
  case False
  then have  $i$ -eq:  $i = \text{length } ps$ 
    using  $i$ -ps by simp
  then show ?thesis
    using  $\text{Suc.prem}(6,7) \text{ next-eval}$  by ( $\text{simp add: nth-append}$ )
  qed
qed
show ?case
  using  $\text{Suc.IH}[OF \text{ - - init' rest}] \text{ Suc.prem}(1-3,6,7)$  by simp
qed

```

lemma *fri-commit-with-created-trees*:

```

assumes receiver-extends:
   $\bigwedge b \ s \ t. \text{Some } (b, t) \in \text{set-dist } (\text{execute receive-challenge } s) \implies$ 
     $s \leq t$ 
and init:
   $\bigwedge i. i < \text{length } ls \implies i < \text{length } ms \implies$ 
     $\text{created-tree } (ls ! i) (ms ! i) (s::'f \text{ protocol-channel})$ 
and outcome:
   $\text{Some } ((ps', ds', ls', ms'), t) \in$ 
     $\text{set-dist } (\text{execute } (\text{fri-commit-with receive-challenge } n \ ps \ ds \ ls \ ms) \ s)$ 
and lens:  $\text{length } ls = \text{length } ms$ 
shows  $\bigwedge i. i < \text{length } ls' \implies i < \text{length } ms' \implies$ 
   $\text{created-tree } (ls' ! i) (ms' ! i) \ t$ 
using init outcome lens
proof (induction n arbitrary: ps ds ls ms s ps' ds' ls' ms' t i)
  case 0
  then show ?case by simp
next
  case ( $\text{Suc } n$ )
from  $\text{Suc.prem}(4)$  obtain  $s1 \ b \ s2 \ \text{next-poly} \ \text{next-domain} \ \text{next-layer} \ m \ s3$  where
  send-root:  $\text{Some } ((), s1) \in \text{set-dist } (\text{execute } (\text{send } (\text{value } (\text{last } ms)))) \ s)$ 
and rand:  $\text{Some } (b, s2) \in \text{set-dist } (\text{execute receive-challenge } s1)$ 
and next-layer-eq:
   $\text{next-fri-layer } (\text{last } ps) (\text{last } ds) \ b =$ 
     $(\text{next-poly}, \text{next-domain}, \text{next-layer})$ 
and create-m:  $\text{Some } (m, s3) \in \text{set-dist } (\text{execute } (\text{create next-layer}) \ s2)$ 
and rest:  $\text{Some } ((ps', ds', ls', ms'), t) \in$ 

```

```

    set-dist (execute
      (fri-commit-with receive-challenge n
        (ps @ [next-poly])
        (ds @ [next-domain])
        (ls @ [next-layer])
        (ms @ [m])) s3)
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have s-s1: s ≤ s1
  using send-outcome[OF send-root]
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have s1-s2: s1 ≤ s2
  using receiver-extends[OF rand] .
have s2-s3: s2 ≤ s3
  using create-outcome[OF create-m] by simp
have s-s3: s ≤ s3
  using s-s1 s1-s2 s2-s3 by (meson hash-ext-trans)
have created-m: created-tree next-layer m s3
  using create-outcome[OF create-m] by simp
have init':
  ∧i. i < length (ls @ [next-layer]) ⇒
    i < length (ms @ [m]) ⇒
      created-tree ((ls @ [next-layer]) ! i) ((ms @ [m]) ! i) s3
proof -
  fix i
  assume i-ls: i < length (ls @ [next-layer])
    and i-ms: i < length (ms @ [m])
  show created-tree ((ls @ [next-layer]) ! i) ((ms @ [m]) ! i) s3
  proof (cases i < length ls)
    case True
    then have created-tree (ls ! i) (ms ! i) s
      using Suc.premis(3)[of i] Suc.premis(5) by simp
    then show ?thesis
      using True Suc.premis(5) s-s3
      by (simp add: nth-append created-tree-mono)
    next
    case False
    then have i = length ls
      using i-ls by simp
    then show ?thesis
      using Suc.premis(5) created-m by (simp add: nth-append)
  qed
qed
show ?case
  using Suc.IH[OF Suc.premis(1,2) init' rest] Suc.premis(5) by simp
qed

```

lemma trace-fri-commit-extends:
 assumes Some ((ps', ds', ls', ms'), t) ∈
 set-dist (execute (trace-fri-commit n ps ds ls ms) s)

shows $s \leq t$
unfolding *trace-fri-commit-def*
by (*rule fri-commit-with-extends*
 $[OF \text{ receive-trace-fri-challenge-extends } \text{assms}[\text{unfolded trace-fri-commit-def}]]$)

lemma *trace-fri-commit-lengths*:
assumes *Some* $((ps', ds', ls', ms'), t) \in$
 $\text{set-dist } (\text{execute } (\text{trace-fri-commit } n \text{ } ps \text{ } ds \text{ } ls \text{ } ms) \text{ } s)$
shows
 $\text{length } ps' = \text{length } ps + n \wedge$
 $\text{length } ds' = \text{length } ds + n \wedge$
 $\text{length } ls' = \text{length } ls + n \wedge$
 $\text{length } ms' = \text{length } ms + n$
by (*rule fri-commit-with-lengths* $[OF \text{ assms}[\text{unfolded trace-fri-commit-def}]]$)

lemma *trace-fri-commit-preserves-prefix*:
assumes *Some* $((ps', ds', ls', ms'), t) \in$
 $\text{set-dist } (\text{execute } (\text{trace-fri-commit } n \text{ } ps \text{ } ds \text{ } ls \text{ } ms) \text{ } s)$
shows
 $\text{take } (\text{length } ps) \text{ } ps' = ps \wedge$
 $\text{take } (\text{length } ds) \text{ } ds' = ds \wedge$
 $\text{take } (\text{length } ls) \text{ } ls' = ls \wedge$
 $\text{take } (\text{length } ms) \text{ } ms' = ms$
by (*rule fri-commit-with-preserves-prefix*
 $[OF \text{ assms}[\text{unfolded trace-fri-commit-def}]]$)

lemma *trace-fri-commit-layers-are-evaluations*:
assumes *init*:
 $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$
 $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$
and *outcome*:
 $\text{Some } ((ps', ds', ls', ms'), t) \in$
 $\text{set-dist } (\text{execute } (\text{trace-fri-commit } n \text{ } ps \text{ } ds \text{ } ls \text{ } ms) \text{ } s)$
and *lens*: $\text{length } ps = \text{length } ds \text{ } \text{length } ds = \text{length } ls$
shows
 $\bigwedge i. i < \text{length } ps' \implies i < \text{length } ds' \implies i < \text{length } ls' \implies$
 $ls' ! i = \text{map } (\text{poly } (ps' ! i)) (ds' ! i)$
by (*rule fri-commit-with-layers-are-evaluations*
 $[OF \text{ init outcome}[\text{unfolded trace-fri-commit-def} \text{ } lens]]$)

lemma *trace-fri-commit-created-trees*:
assumes *init*:
 $\bigwedge i. i < \text{length } ls \implies i < \text{length } ms \implies$
 $\text{created-tree } (ls ! i) (ms ! i) (s::'f \text{ protocol-channel})$
and *outcome*:
 $\text{Some } ((ps', ds', ls', ms'), t) \in$
 $\text{set-dist } (\text{execute } (\text{trace-fri-commit } n \text{ } ps \text{ } ds \text{ } ls \text{ } ms) \text{ } s)$
and *lens*: $\text{length } ls = \text{length } ms$
shows $\bigwedge i. i < \text{length } ls' \implies i < \text{length } ms' \implies$

$created-tree (ls' ! i) (ms' ! i) t$
by (*rule fri-commit-with-created-trees*
 $[OF\ receive-trace-fri-challenge-extends\ init$
 $outcome[unfolded\ trace-fri-commit-def]\ lens])$

lemma *composition-fri-commit-extends:*

assumes $Some ((ps', ds', ls', ms'), t) \in$
 $set-dist (execute (composition-fri-commit\ n\ ps\ ds\ ls\ ms)\ s)$
shows $s \leq t$
unfolding *composition-fri-commit-def*
by (*rule fri-commit-with-extends*
 $[OF\ receive-composition-fri-challenge-extends$
 $assms[unfolded\ composition-fri-commit-def]])$

lemma *composition-fri-commit-lengths:*

assumes $Some ((ps', ds', ls', ms'), t) \in$
 $set-dist (execute (composition-fri-commit\ n\ ps\ ds\ ls\ ms)\ s)$
shows
 $length\ ps' = length\ ps + n \wedge$
 $length\ ds' = length\ ds + n \wedge$
 $length\ ls' = length\ ls + n \wedge$
 $length\ ms' = length\ ms + n$
by (*rule fri-commit-with-lengths*
 $[OF\ assms[unfolded\ composition-fri-commit-def]])$

lemma *composition-fri-commit-preserves-prefix:*

assumes $Some ((ps', ds', ls', ms'), t) \in$
 $set-dist (execute (composition-fri-commit\ n\ ps\ ds\ ls\ ms)\ s)$
shows
 $take (length\ ps)\ ps' = ps \wedge$
 $take (length\ ds)\ ds' = ds \wedge$
 $take (length\ ls)\ ls' = ls \wedge$
 $take (length\ ms)\ ms' = ms$
by (*rule fri-commit-with-preserves-prefix*
 $[OF\ assms[unfolded\ composition-fri-commit-def]])$

lemma *composition-fri-commit-layers-are-evaluations:*

assumes *init:*
 $\bigwedge i. i < length\ ps \implies i < length\ ds \implies i < length\ ls \implies$
 $ls ! i = map\ (poly\ (ps ! i))\ (ds ! i)$
and *outcome:*
 $Some ((ps', ds', ls', ms'), t) \in$
 $set-dist (execute (composition-fri-commit\ n\ ps\ ds\ ls\ ms)\ s)$
and *lens:* $length\ ps = length\ ds\ length\ ds = length\ ls$
shows
 $\bigwedge i. i < length\ ps' \implies i < length\ ds' \implies i < length\ ls' \implies$
 $ls' ! i = map\ (poly\ (ps' ! i))\ (ds' ! i)$
by (*rule fri-commit-with-layers-are-evaluations*
 $[OF\ init\ outcome[unfolded\ composition-fri-commit-def]\ lens])$

lemma *composition-fri-commit-created-trees*:
assumes *init*:
 $\bigwedge i. i < \text{length } ls \implies i < \text{length } ms \implies$
 $\text{created-tree } (ls ! i) (ms ! i) (s::'f \text{ protocol-channel})$
and *outcome*:
 $\text{Some } ((ps', ds', ls', ms'), t) \in$
 $\text{set-dist } (\text{execute } (\text{composition-fri-commit } n \text{ ps ds ls ms}) s)$
and *lens*: $\text{length } ls = \text{length } ms$
shows $\bigwedge i. i < \text{length } ls' \implies i < \text{length } ms' \implies$
 $\text{created-tree } (ls' ! i) (ms' ! i) t$
by (*rule fri-commit-with-created-trees*
 $[\text{OF receive-composition-fri-challenge-extends init}$
 $\text{outcome}[\text{unfolded composition-fri-commit-def}] \text{ lens}]$)

1.39.3 Decommitments

definition *decommit-on-fri-layers*

where

decommit-on-fri-layers fs =
 map
 $(\lambda(l, m) \text{ idx. do } \{$
 $\text{let } len = \text{length } l;$
 $\text{let } idx' = \text{idx mod } len;$
 $\text{let } sidx = (idx' + (len \text{ div } 2)) \text{ mod } len;$
 $\text{send } (l ! idx');$
 $\text{mfold2 send } (\text{get-authentication-path } len \text{ idx'} m);$
 $\text{send } (l ! sidx);$
 $\text{mfold2 send } (\text{get-authentication-path } len \text{ sidx } m);$
 $\text{return } idx'$
 $\}) \text{ fs}$

definition *decommit-on-query*

where

decommit-on-query idx f-merkle $\equiv$
 map
 $(\lambda i. \text{do } \{$
 $\text{send } (f\text{-eval} ! i);$
 $\text{mfold2 send } (\text{get-authentication-path } (\text{length } (f\text{-eval})) i \text{ f-merkle})$
 $\})$
 $(\text{powers-scaled } idx)$

1.39.4 Stark Proof

The following is a STARK proof for vals.

definition *prover-monad*

where

prover-monad $\equiv$ ($\text{do } \{$
 $f\text{-merkle} \leftarrow \text{create } f\text{-eval};$

```

    send (value f-merkle); — Send the root of a Merkle tree which contains the evaluations over
the larger domain.
    let f-nrounds = ceil-log clength;
    (f-ps, f-ds, f-ls, f-ms) ←
      trace-fri-commit f-nrounds [f] [eval-domain] [f-eval] [f-merkle];
    send (hd (last f-ls)); — Send the final constant for the trace FRI proof.
    as ← mmap (replicate (length spec) (
      do {
        a ← receive-alpha-challenge;
        send a;
        return a
      })); — Send the random factors (alphas) for the composition polynomial.
    let cp' = cp as f-powers;
    send (of-nat (degree cp')); — Send the degree of the composition polynomial.
    cp-merkle ← create (cp-eval as);
    let nrounds = ceil-log (degree cp' + 1);
    (ps, ds, ls, ms) ←
      composition-fri-commit nrounds [cp'] [eval-domain] [cp-eval as] [cp-merkle];
    — Again, we create a Merkle tree which contains the evaluations over the larger domain.
    send (hd (last ls)); — Send the final constant before the verifier samples the query index.
    ntimes (do {
      idx ← receive-query-index-challenge; — Check the validity for a random index. This should
be repeated multiple times.
      let idx' = index (to-nat idx); — Make sure index is not out of bounds
      mmap (decommit-on-query idx' f-merkle);
      mfold idx' (decommit-on-fri-layers (butlast (zip f-ls f-ms))); — Check that the trace com-
mitment is low-degree.
      mfold idx' (decommit-on-fri-layers (butlast (zip ls ms))); — We don't de-commit on the
constant polynomial
      return ()
    }) rounds
  })

```

definition *sproof*

where

sproof ≡

execute prover-monad

(\HashMap = fmempty, PState = 0, PTranscript = [],
 PTraceFriCounter = 0, PCompositionFriCounter = 0,
 PAlphaCounter = 0, PQueryCounter = 0)

end

1.40 Verifier

locale *verifier* = stark +

constrains *omega::'f::proth-field*


```

and shift::'f
and concat :: 'f ⇒ 'f ⇒ 'f
and spec :: (('f poly list ⇒ 'f poly) × nat list × nat) list
fixes spec2:: (('f list ⇒ 'f) × nat list) list
assumes specs-aligned:
  list-all2
    (λ(c, roots, d) (c2, roots2)).
      roots2 = roots ∧
      (∀ qs x.
        powers ≤ length qs ⟶
        c2 (map (λq. poly q x) qs) = poly (c qs) x))
    spec spec2

```

begin

```

lemma spec2-length:
  length spec2 = length spec
using list-all2-lengthD[OF specs-aligned] by simp

```

```

lemma specs-agree:
assumes pair:
  ((c, roots, d), (c2, roots2)) ∈ set (zip spec spec2)
and qs-len: powers ≤ length qs
shows
  roots2 = roots ∧
  c2 (map (λq. poly q x) qs) = poly (c qs) x

```

proof –

```

obtain i where i-bound: i < length spec i < length spec2
and spec-i: spec ! i = (c, roots, d)
and spec2-i: spec2 ! i = (c2, roots2)
using pair by (auto simp: set-zip)
have aligned:
  (λ(c, roots, d) (c2, roots2)).
    roots2 = roots ∧
    (∀ qs x.
      powers ≤ length qs ⟶
      c2 (map (λq. poly q x) qs) = poly (c qs) x))
    (spec ! i) (spec2 ! i)
by (rule list-all2-nthD[OF specs-aligned i-bound(1)])
show ?thesis
using aligned spec-i spec2-i qs-len by simp
qed

```

definition *cp-eval*

where

```

  cp-eval as fs x ≡ fold (λ(a, (p, rs)). (+) (a * ((p fs) div poly (prod (g-map rs)) x))) (zip as spec2) 0

```

1.40.1 Validate Committments

definition *receive-fri-commits*

where

```

receive-fri-commits =
  do {
    r ← read;
    b ← receive-random-field-element;
    return (b,r)
  }
```

definition *receive-fri-commits-with*

where

```

receive-fri-commits-with receive-challenge =
  do {
    r ← read;
    b ← receive-challenge;
    return (b,r)
  }
```

definition *receive-trace-fri-commits*

where

```

receive-trace-fri-commits =
  receive-fri-commits-with receive-trace-fri-challenge
```

definition *receive-composition-fri-commits*

where

```

receive-composition-fri-commits =
  receive-fri-commits-with receive-composition-fri-challenge
```

lemma *receive-fri-commits-with-receive-random-field-element*:

receive-fri-commits-with receive-random-field-element = *receive-fri-commits*

unfolding *receive-fri-commits-with-def receive-fri-commits-def* **by** *simp*

lemma *receive-fri-commits-with-no-failure*:

assumes *receiver-nf*:

$\bigwedge s. \text{None} \notin \text{dom} (\text{dist} (\text{execute } \text{receive-challenge } s))$

and *tr*: $P\text{Transcript } s \neq []$

shows

$\text{None} \notin \text{dom} (\text{dist} (\text{execute} (\text{receive-fri-commits-with receive-challenge}) s))$

using *tr*

unfolding *receive-fri-commits-with-def*

by (*intro no-failure-bindI*)

(*simp-all add: read-no-failure receiver-nf*)

lemma *receive-trace-fri-commits-no-failure*:

assumes $P\text{Transcript } s \neq []$

shows $\text{None} \notin \text{dom} (\text{dist} (\text{execute } \text{receive-trace-fri-commits } s))$

unfolding *receive-trace-fri-commits-def*

by (*rule receive-fri-commits-with-no-failure*)
 (*rule receive-trace-fri-challenge-no-failure*, *rule assms*)

lemma *receive-composition-fri-commits-no-failure*:
assumes $P\text{Transcript } s \neq []$
shows $\text{None} \notin \text{dom } (\text{dist } (\text{execute } \text{receive-composition-fri-commits } s))$
unfolding *receive-composition-fri-commits-def*
by (*rule receive-fri-commits-with-no-failure*)
 (*rule receive-composition-fri-challenge-no-failure*, *rule assms*)

1.40.2 Validate Decommitments

definition *validate*

where

validate lgth idx leaf-data l rv $\equiv$ *check-authentication-path lgth idx leaf-data l = rv*

definition *check-decommit-on-query*

where

check-decommit-on-query fr idx $\equiv$
map
 ($\lambda i.$ *do* {
 qh $\leftarrow$ *read*;
 let *len* = *scale* * *clength*; — *scale* * *clength* is length of *f-eval*
 qh-path $\leftarrow$ *ntimes read* (*floor-log len*);
 ap $\leftarrow$ *check-authentication-path len i qh qh-path*;
 assert (*ap* = *fr*);
 return *qh*
 })
 (*powers-scaled idx*)

definition *receive-query-commits*

where

receive-query-commits fl =
map ($\lambda(b,f)$ (*i,x,len,pow*)).
do {
 xp $\leftarrow$ *read*;
 xp' $\leftarrow$ *ntimes read* (*floor-log len*);
 xn $\leftarrow$ *read*;
 xn' $\leftarrow$ *ntimes read* (*floor-log len*);
 assert (*xp* = *x*);
 ap $\leftarrow$ *check-authentication-path len i xp xp'*;
 assert (*ap* = *f*);
 let *sidx* = (*i* + *len div 2*) mod *len*;
 ap $\leftarrow$ *check-authentication-path len sidx xn xn'*;
 assert (*ap* = *f*);
 let *gp* = (*xp* + *xn*) div 2;
 let *hp* = (*xp* - *xn*) div ($2 * ((h \hat{=} i) * \text{shift}) \hat{=} \text{pow}$);
 let *x* = *gp* + *b* * *hp*;
 return (*i* mod (*len div 2*), *x*, *len div 2*, *pow+pow*)

}) *fl*

definition *verify-monad*

where

```

verify-monad  $\equiv$  (do {
  fr  $\leftarrow$  read;
  f-fl  $\leftarrow$  ntimes receive-trace-fri-commits (ceil-log clength);
  f-final  $\leftarrow$  read;
  as  $\leftarrow$  mmap (replicate (length spec) (
    do {
      a0  $\leftarrow$  receive-alpha-challenge;
      let a0' = a0;
      a1  $\leftarrow$  read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    }
  ));
  dg  $\leftarrow$  read;
  assert (to-nat dg  $\leq$  maxDegree); — Degree check
  fl  $\leftarrow$  ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1));
  final  $\leftarrow$  read;
  ntimes (do {
    idx  $\leftarrow$  receive-query-index-challenge;
    let idx' = index (to-nat idx); — Make sure index is not out of bounds
    fv  $\leftarrow$  mmap (check-decommit-on-query fr idx');

    (f-i, f-x, f-len, f-pow)  $\leftarrow$  mfold (idx', hd fv, clength * scale, 1) (receive-query-commits f-fl);
    assert (f-x = f-final);

    (i,x,len,pow)  $\leftarrow$  mfold (idx', cp-eval as fv (h  $\wedge$  idx' * shift), clength * scale, 1) (receive-query-commits fl);

    assert (x = final) — Check that the final FRI value is the committed constant polynomial
  }) rounds
})

```

definition *verify*

where

verify p $\equiv$ *execute verify-monad* (*p* $\parallel$ *PState* := 0)

end

end

2 Completeness

```

theory Completeness-Core
  imports Stark-Core.Stark
begin

```

Core verification locale, honest-trace predicates, and WP/no-failure infrastructure.

```

locale verification =
  p: prover concat omega shift scale clength powers to-nat of-nat size spec rounds vals +
  v: verifier concat omega shift scale clength powers to-nat of-nat size spec rounds spec2
  for concat :: 'f::proth-field  $\Rightarrow$  'f  $\Rightarrow$  'f
  and omega :: 'f
  and shift :: 'f
  and scale :: nat
  and clength :: nat
  and powers :: nat
  and of-nat:: nat  $\Rightarrow$  'f
  and to-nat:: 'f  $\Rightarrow$  nat
  and size:: nat
  and spec:: (('f poly list  $\Rightarrow$  'f poly)  $\times$  nat list  $\times$  nat) list
  and rounds :: nat
  and spec2:: (('f list  $\Rightarrow$  'f)  $\times$  nat list) list
  and vals :: 'f list
  +
  assumes constraint-degree-bound:
     $\bigwedge c$  roots d.
      (c, roots, d)  $\in$  set spec  $\implies$ 
        degree (c p.f-powers)  $\leq d * (clength - 1)$ 
begin

fun root-check
where
  root-check p [] = True
| root-check p (r#rs) = (poly p r = 0  $\wedge$  root-check p rs)

```

```

definition exec
where exec  $\equiv$  do {
  fr  $\leftarrow$  p.prover-monad;
  modify ( $\lambda c.$  c(PState := 0, PTranscript:=rev (PTranscript c),
    PTraceFriCounter := 0, PCompositionFriCounter := 0,
    PAlphaCounter := 0, PQueryCounter := 0));
  v.verify-monad
}

```

```

definition init-state
where init-state  $\equiv$ 
  ( $\lambda$  (HashMap = fmempty, PState = 0, PTranscript = [],
    PTraceFriCounter = 0, PCompositionFriCounter = 0,
    PAlphaCounter = 0, PQueryCounter = 0))

```

```

definition verifier-replay-state

```

where *verifier-replay-state* $s \equiv$
 $s(PState := 0, PTranscript := \text{rev } (PTranscript\ s),$
 $PTraceFriCounter := 0, PCompositionFriCounter := 0,$
 $PAlphaCounter := 0, PQueryCounter := 0)$

definition *constraint-roots-vanish*

where
 $\text{constraint-roots-vanish } c\ roots \longleftrightarrow$
 $(\forall r \in \text{set } (p.g\text{-map } roots). \text{poly } (c\ p.f\text{-powers})\ r = 0)$

definition *declared-degree-sound*

where
 $\text{declared-degree-sound } c\ roots\ d \longleftrightarrow$
 $\text{degree } (c\ p.f\text{-powers}) \leq d * (\text{clength} - 1)$

definition *roots-distinct*

where
 $\text{roots-distinct } roots \longleftrightarrow \text{distinct } (p.g\text{-map } roots)$

definition *honest-trace-algebra*

where
 $\text{honest-trace-algebra} \longleftrightarrow$
 $(\forall (c, roots, d) \in \text{set spec.}$
 $\text{constraint-roots-vanish } c\ roots)$

definition *honest-trace-valid*

where
 $\text{honest-trace-valid} \longleftrightarrow \text{honest-trace-algebra}$

definition *no-failure*

where $\text{no-failure } m\ s \longleftrightarrow \text{None} \notin \text{dom } (\text{dist } (\text{execute } m\ s))$

definition *transcript-extends*

where $\text{transcript-extends } t\ s \longleftrightarrow (\exists xs. PTranscript\ t = xs @ PTranscript\ s)$

lemma *declared-degree-sound-from-constraint-degree-bound:*

assumes $(c, roots, d) \in \text{set spec}$
shows $\text{declared-degree-sound } c\ roots\ d$
using $\text{constraint-degree-bound}[OF\ \text{assms}]$
unfolding $\text{declared-degree-sound-def}$.

lemma *roots-distinct-from-spec-roots-distinct:*

assumes $(c, roots, d) \in \text{set spec}$
shows $\text{roots-distinct } roots$
using $p.g\text{-map-distinct}[OF\ \text{assms}]$
unfolding $\text{roots-distinct-def}$.

lemma *maxDegree-less-eval-domain:*

$p.\text{maxDegree} < \text{clength} * \text{scale}$

by (rule p.maxDegree-less-eval-domain)

lemma wp-error-assert[wpsimps]:
 wp-error (assert b) s = (if b then 0 else 1)
 unfolding assert-def by (simp add: wpsimps)

lemma no-failure-assert-bindI:
 assumes b
 and None $\notin$ dom (dist (execute (k ()) s))
 shows None $\notin$ dom (dist (execute (assert b $\ggg$ k) s))
 using assms unfolding assert-def by simp

lemma no-failure-bind-returnI:
 assumes None $\notin$ dom (dist (execute m s))
 shows None $\notin$ dom (dist (execute (m $\ggg$ (λx . return (f x))) s))
 by (rule no-failure-bindI[OF assms]) simp

lemma no-failure-mfoldI:
 assumes step:
 $\bigwedge m a s. m \in \text{set } ms \implies$
 $\text{None} \notin \text{dom} (\text{dist} (\text{execute} (m a) s))$
 shows None $\notin$ dom (dist (execute (mfold a ms) s))
 using step
proof (induction ms arbitrary: a s)
 case Nil
 then show ?case by simp
next
 case (Cons m ms)
 show ?case
 unfolding mfold.simps
proof (rule no-failure-bindI)
 show None $\notin$ dom (dist (execute (m a) s))
 using Cons.prem by simp
next
 fix x t
 assume Some (x, t) $\in$ set-dist (execute (m a) s)
 show None $\notin$ dom (dist (execute (mfold x ms) t))
 by (rule Cons.IH) (use Cons.prem in simp)
qed
qed

lemma no-failure-mfold-invariantI:
 assumes init: I a s
 and step-nf:
 $\bigwedge m a s. m \in \text{set } ms \implies I a s \implies$
 $\text{None} \notin \text{dom} (\text{dist} (\text{execute} (m a) s))$
 and step-inv:
 $\bigwedge m a s x t. m \in \text{set } ms \implies I a s \implies$
 $\text{Some} (x, t) \in \text{set-dist} (\text{execute} (m a) s) \implies I x t$

```

shows None  $\notin$  dom (dist (execute (mfold a ms) s))
using init step-nf step-inv
proof (induction ms arbitrary: a s)
  case Nil
  then show ?case by simp
next
  case (Cons m ms)
  show ?case
    unfolding mfold.simps
  proof (rule no-failure-bindI)
    show None  $\notin$  dom (dist (execute (m a) s))
    using Cons.prem by simp
  next
    fix x t
    assume outcome: Some (x, t)  $\in$  set-dist (execute (m a) s)
    have inv-xt: I x t
      using Cons.prem(1) Cons.prem(3)[of m a s x t] outcome by simp
    show None  $\notin$  dom (dist (execute (mfold x ms) t))
    proof (rule Cons.IH[OF inv-xt])
      fix m' a' s'
      assume m'  $\in$  set ms and I a' s'
      then show None  $\notin$  dom (dist (execute (m' a') s'))
      using Cons.prem by simp
    next
      fix m' a' s' x' t'
      assume m'-in: m'  $\in$  set ms and inv: I a' s'
      and out: Some (x', t')  $\in$  set-dist (execute (m' a') s')
      show I x' t'
      using Cons.prem(3)[of m' a' s' x' t'] m'-in inv out by simp
    qed
  qed
qed

lemma no-failure-mfold-map-index-fromI:
  assumes init: I k a s
  and step-nf:
     $\bigwedge j x a s. k \leq j \implies j < k + \text{length } xs \implies$ 
     $x = xs ! (j - k) \implies I j a s \implies$ 
    None  $\notin$  dom (dist (execute (F x a) s))
  and step-inv:
     $\bigwedge j x a s y t. k \leq j \implies j < k + \text{length } xs \implies$ 
     $x = xs ! (j - k) \implies I j a s \implies$ 
    Some (y, t)  $\in$  set-dist (execute (F x a) s)  $\implies$ 
    I (Suc j) y t
  shows None  $\notin$  dom (dist (execute (mfold a (map F xs) s)))
  using init step-nf step-inv
proof (induction xs arbitrary: k a s)
  case Nil
  then show ?case by simp

```



```

next
  case (Cons x xs)
  show ?case
    unfolding mfold.simps list.map
  proof (rule no-failure-bindI)
    show None  $\notin$  dom (dist (execute (F x a) s))
      by (rule Cons.premis(2)[of k x a s]) (use Cons.premis(1) in simp-all)
  next
    fix y t
    assume step-out: Some (y, t)  $\in$  set-dist (execute (F x a) s)
    have inv-next: I (Suc k) y t
      by (rule Cons.premis(3)[of k x a s y t])
        (use Cons.premis(1) step-out in simp-all)
    show None  $\notin$  dom (dist (execute (mfold y (map F xs)) t))
    proof (rule Cons.IH[OF inv-next])
      fix j x' a' s'
      assume ge: Suc k  $\leq$  j
        and lt: j < Suc k + length xs
        and x'-eq: x' = xs ! (j - Suc k)
        and inv: I j a' s'
      have j-gt: k < j
        using ge by simp
      have j-lt: j < k + length (x # xs)
        using lt by simp
      have nth-eq: x' = (x # xs) ! (j - k)
        using x'-eq j-gt by simp
      show None  $\notin$  dom (dist (execute (F x' a') s'))
        by (rule Cons.premis(2)[OF - j-lt nth-eq inv]) (use ge in simp)
    next
      fix j x' a' s' y' t'
      assume ge: Suc k  $\leq$  j
        and lt: j < Suc k + length xs
        and x'-eq: x' = xs ! (j - Suc k)
        and inv: I j a' s'
        and out: Some (y', t')  $\in$  set-dist (execute (F x' a') s')
      have j-gt: k < j
        using ge by simp
      have j-lt: j < k + length (x # xs)
        using lt by simp
      have nth-eq: x' = (x # xs) ! (j - k)
        using x'-eq j-gt by simp
      show I (Suc j) y' t'
        by (rule Cons.premis(3)[OF - j-lt nth-eq inv out]) (use ge in simp)
    qed
  qed
qed

lemma no-failure-mfold-map-indexI:
  assumes init: I 0 a s

```

```

and step-nf:
   $\bigwedge j \ x \ a \ s. \ j < \text{length } xs \implies$ 
     $x = xs ! j \implies I \ j \ a \ s \implies$ 
     $\text{None} \notin \text{dom} (\text{dist} (\text{execute} (F \ x \ a) \ s))$ 
and step-inv:
   $\bigwedge j \ x \ a \ s \ y \ t. \ j < \text{length } xs \implies$ 
     $x = xs ! j \implies I \ j \ a \ s \implies$ 
     $\text{Some} (y, t) \in \text{set-dist} (\text{execute} (F \ x \ a) \ s) \implies$ 
     $I \ (\text{Suc } j) \ y \ t$ 
shows  $\text{None} \notin \text{dom} (\text{dist} (\text{execute} (\text{mfold } a \ (\text{map } F \ xs)) \ s))$ 
by (rule no-failure-mfold-map-index-fromI[
  where  $I=I$  and  $k=0$  and  $xs=xs$  and  $F=F$ , OF init])
  (use step-nf step-inv in simp-all)

lemma no-failure-mmapI:
  assumes step:  $\bigwedge m \ s. \ m \in \text{set } ms \implies \text{None} \notin \text{dom} (\text{dist} (\text{execute } m \ s))$ 
  shows  $\text{None} \notin \text{dom} (\text{dist} (\text{execute} (\text{mmap } ms) \ s))$ 
  using step
proof (induction ms arbitrary: s)
  case Nil
  then show ?case by simp
next
  case (Cons m ms)
  show ?case
    unfolding mmap.simps
  proof (rule no-failure-bindI)
    show  $\text{None} \notin \text{dom} (\text{dist} (\text{execute } m \ s))$ 
    using Cons.prems by simp
  next
    fix  $x \ t$ 
    assume  $\text{Some} (x, t) \in \text{set-dist} (\text{execute } m \ s)$ 
    show  $\text{None} \notin \text{dom} (\text{dist} (\text{execute} (\text{mmap } ms \gg (\lambda xs. \text{return } (x \# xs))) \ t))$ 
    by (rule no-failure-bind-returnI)
      (rule Cons.IH, use Cons.prems in simp)
  qed
qed

lemma no-failure-mfold2I:
  assumes step:  $\bigwedge x \ s. \ x \in \text{set } xs \implies \text{None} \notin \text{dom} (\text{dist} (\text{execute} (f \ x) \ s))$ 
  shows  $\text{None} \notin \text{dom} (\text{dist} (\text{execute} (\text{mfold2 } f \ xs) \ s))$ 
  using step
proof (induction xs arbitrary: s)
  case Nil
  then show ?case by simp
next
  case (Cons x xs)
  show ?case
    unfolding mfold2.simps
  proof (rule no-failure-bindI)

```

```

  show  $\text{None} \notin \text{dom} (\text{dist} (\text{execute} (f\ x)\ s))$ 
    using Cons.prems by simp
next
  fix  $y\ t$ 
  assume  $\text{Some} (y, t) \in \text{set-dist} (\text{execute} (f\ x)\ s)$ 
  show  $\text{None} \notin \text{dom} (\text{dist} (\text{execute} (\text{mfold2}\ f\ xs)\ t))$ 
    by (rule Cons.IH) (use Cons.prems in simp)
qed
qed

lemma no-failure-ntimesI:
  assumes step:  $\bigwedge s. \text{None} \notin \text{dom} (\text{dist} (\text{execute}\ m\ s))$ 
  shows  $\text{None} \notin \text{dom} (\text{dist} (\text{execute} (\text{ntimes}\ m\ n)\ s))$ 
  using assms
proof (induction  $n$  arbitrary:  $s$ )
  case 0
  then show ?case by simp
next
  case (Suc  $n$ )
  show ?case
    unfolding ntimes.simps
  proof (rule no-failure-bindI[OF Suc.prems])
    fix  $x\ t$ 
    assume  $\text{Some} (x, t) \in \text{set-dist} (\text{execute}\ m\ s)$ 
    show  $\text{None} \notin \text{dom} (\text{dist} (\text{execute} (\text{ntimes}\ m\ n \gg (\lambda xs. \text{return} (x \# xs)))\ t))$ 
      by (rule no-failure-bind-returnI) (rule Suc.IH[OF Suc.prems])
    qed
  qed

lemma no-failure-ntimes-invariantI:
  assumes init:  $I\ s$ 
  and step-nf:  $\bigwedge s. I\ s \implies \text{None} \notin \text{dom} (\text{dist} (\text{execute}\ m\ s))$ 
  and step-inv:  $\bigwedge x\ s\ t. I\ s \implies \text{Some} (x, t) \in \text{set-dist} (\text{execute}\ m\ s) \implies I\ t$ 
  shows  $\text{None} \notin \text{dom} (\text{dist} (\text{execute} (\text{ntimes}\ m\ n)\ s))$ 
  using init
proof (induction  $n$  arbitrary:  $s$ )
  case 0
  then show ?case by simp
next
  case (Suc  $n$ )
  show ?case
    unfolding ntimes.simps
  proof (rule no-failure-bindI[OF step-nf[OF Suc.prems]])
    fix  $x\ t$ 
    assume step:  $\text{Some} (x, t) \in \text{set-dist} (\text{execute}\ m\ s)$ 
    have  $I\ t$ 
      by (rule step-inv[OF Suc.prems step])
    then show  $\text{None} \notin \text{dom} (\text{dist} (\text{execute} (\text{ntimes}\ m\ n \gg (\lambda xs. \text{return} (x \# xs)))\ t))$ 

```

```

    by (rule no-failure-bind-returnI[OF Suc.IH])
  qed
qed

lemma ntimes-hash-extends:
  fixes m :: ('a, ('f, 'b) protocol-channel-scheme) state-monad
  assumes step-ext:
     $\bigwedge x (s::('f, 'b) \text{ protocol-channel-scheme}) t.$ 
     $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies s \leq t$ 
  and outcome:  $\text{Some } (xs, t) \in \text{set-dist } (\text{execute } (\text{ntimes } m \ n) \ s)$ 
  shows  $s \leq t$ 
  using outcome
proof (induction n arbitrary: xs s t)
  case 0
  then show ?case
    using p.hash-ext-refl[of s] by simp
next
  case (Suc n)
  have expanded:
     $\text{Some } (xs, t) \in$ 
     $\text{set-dist } (\text{execute } (m \gg (\lambda x. \text{ntimes } m \ n \gg (\lambda ys. \text{return } (x \# ys)))) \ s)$ 
  using Suc.prem by simp
  show ?case
proof (rule p.set-dist-bindE[OF expanded])
  fix x u
  assume step:  $\text{Some } (x, u) \in \text{set-dist } (\text{execute } m \ s)$ 
  and rest:
     $\text{Some } (xs, t) \in$ 
     $\text{set-dist } (\text{execute } (\text{ntimes } m \ n \gg (\lambda ys. \text{return } (x \# ys))) \ u)$ 
  show ?thesis
proof (rule p.set-dist-bindE[OF rest])
  fix ys v
  assume tail-v:  $\text{Some } (ys, v) \in \text{set-dist } (\text{execute } (\text{ntimes } m \ n) \ u)$ 
  and ret:  $\text{Some } (xs, t) \in \text{set-dist } (\text{execute } (\text{return } (x \# ys)) \ v)$ 
  have tail:  $\text{Some } (ys, t) \in \text{set-dist } (\text{execute } (\text{ntimes } m \ n) \ u)$ 
  using tail-v ret by simp
  have  $s \leq u$ 
  by (rule step-ext[OF step])
  moreover have  $u \leq t$ 
  by (rule Suc.IH[OF tail])
  ultimately show ?thesis
    unfolding less-eq-hash-ext-def less-eq-fmap-def
    by (metis option.distinct(1))
  qed
qed
qed

```

lemma ntimes-transcript-extends:

```

assumes step-ext:
   $\bigwedge x s t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m s) \implies \text{transcript-extends } t s$ 
and outcome:  $\text{Some } (xs, t) \in \text{set-dist } (\text{execute } (\text{ntimes } m n) s)$ 
shows transcript-extends  $t s$ 
using outcome
proof (induction  $n$  arbitrary:  $xs s t$ )
  case 0
  then show ?case
    unfolding transcript-extends-def by simp
next
  case (Suc  $n$ )
  have expanded:
     $\text{Some } (xs, t) \in$ 
       $\text{set-dist } (\text{execute } (m \gg (\lambda x. \text{ntimes } m n \gg (\lambda ys. \text{return } (x \# ys)))) s)$ 
    using Suc.prems by simp
  show ?case
  proof (rule p.set-dist-bindE[OF expanded])
    fix  $x u$ 
    assume step:  $\text{Some } (x, u) \in \text{set-dist } (\text{execute } m s)$ 
    and rest:
       $\text{Some } (xs, t) \in$ 
         $\text{set-dist } (\text{execute } (\text{ntimes } m n \gg (\lambda ys. \text{return } (x \# ys))) u)$ 
    show ?thesis
    proof (rule p.set-dist-bindE[OF rest])
      fix  $ys v$ 
      assume tail-v:  $\text{Some } (ys, v) \in \text{set-dist } (\text{execute } (\text{ntimes } m n) u)$ 
      and ret:  $\text{Some } (xs, t) \in \text{set-dist } (\text{execute } (\text{return } (x \# ys)) v)$ 
      have tail:  $\text{Some } (ys, t) \in \text{set-dist } (\text{execute } (\text{ntimes } m n) u)$ 
      using tail-v ret by simp
      have transcript-extends  $u s$ 
      by (rule step-ext[OF step])
      moreover have transcript-extends  $t u$ 
      by (rule Suc.IH[OF tail])
      ultimately show ?thesis
      unfolding transcript-extends-def by auto
    qed
  qed
qed

```

```

lemma ntimes-Suc-outcomeE:
  assumes outcome:  $\text{Some } (xs, t) \in \text{set-dist } (\text{execute } (\text{ntimes } m (\text{Suc } n)) s)$ 
  obtains  $x ys u$  where
     $\text{Some } (x, u) \in \text{set-dist } (\text{execute } m s)$ 
     $\text{Some } (ys, t) \in \text{set-dist } (\text{execute } (\text{ntimes } m n) u)$ 
     $xs = x \# ys$ 
proof –
  have expanded:
     $\text{Some } (xs, t) \in$ 

```

```

    set-dist (execute (m  $\gg$  ( $\lambda x. ntimes\ m\ n \gg (\lambda ys. return\ (x\ \# \ ys))$ ))) s)
  using outcome by simp
show ?thesis
proof (rule p.set-dist-bindE[OF expanded])
  fix x u
  assume step: Some (x, u)  $\in$  set-dist (execute m s)
  and rest:
    Some (xs, t)  $\in$ 
      set-dist (execute (ntimes m n  $\gg (\lambda ys. return\ (x\ \# \ ys))$ ) u)
  show ?thesis
proof (rule p.set-dist-bindE[OF rest])
  fix ys v
  assume tail-v: Some (ys, v)  $\in$  set-dist (execute (ntimes m n) u)
  and ret: Some (xs, t)  $\in$  set-dist (execute (return (x  $\#$  ys)) v)
  have tail: Some (ys, t)  $\in$  set-dist (execute (ntimes m n) u)
  using tail-v ret by simp
  have xs-eq: xs = x  $\#$  ys
  using ret by simp
  show ?thesis
  by (rule that[OF step tail xs-eq])
qed
qed
qed

```

definition prover-query-round

where

```

prover-query-round f-merkle f-ls f-ms ls ms  $\equiv$ 
do {
  idx  $\leftarrow$  p.receive-query-index-challenge;
  mmap (p.decommit-on-query (p.index (to-nat idx)) f-merkle);
  mfold (p.index (to-nat idx)) (p.decommit-on-fri-layers (butlast (zip f-ls f-ms)));
  mfold (p.index (to-nat idx)) (p.decommit-on-fri-layers (butlast (zip ls ms)));
  return ()
}

```

definition verifier-query-round

where

```

verifier-query-round fr f-fl f-final as fl final  $\equiv$ 
do {
  idx  $\leftarrow$  p.receive-query-index-challenge;
  let idx' = p.index (to-nat idx);
  fv  $\leftarrow$  mmap (v.check-decommit-on-query fr idx');
  (f-i, f-x, f-len, f-pw)  $\leftarrow$ 
    mfold
      (idx', hd fv, clength * scale, 1)
      (v.receive-query-commits f-fl);
  assert (f-x = f-final);
  (i, x, len, pw)  $\leftarrow$ 
    mfold

```

```

      (idx', v.cp-eval as fv (p.h  $\hat{\wedge}$  idx' * shift), clength * scale, 1)
      (v.receive-query-commits fl);
    assert (x = final)
  }

```

lemma *send-no-failure*:

```

  None  $\notin$  dom (dist (execute (p.send x) s))
  by (rule p.send-no-failure)

```

lemma *create-no-failure*:

```

  None  $\notin$  dom (dist (execute (p.create xs) s))
  by (rule p.create-no-failure)

```

lemma *fri-commit-no-failure*:

```

  None  $\notin$  dom (dist (execute (p.fri-commit n ps ds ls ms) s))

```

proof (induction n arbitrary: ps ds ls ms s)

case 0

then show ?case by simp

next

case (Suc n)

show ?case

unfolding p.fri-commit.simps

proof (rule no-failure-bindI[OF send-no-failure])

fix u s1

assume Some (u, s1) $\in$ set-dist (execute (p.send (value (last ms))) s)

show None $\notin$ dom (dist (execute

(p.receive-random-field-element $\gg$

(λ b. let (next-poly, next-domain, next-layer) =

p.next-fri-layer (last ps) (last ds) b

in p.create next-layer $\gg$

(λ m. p.fri-commit n

(ps @ [next-poly]) (ds @ [next-domain])

(ls @ [next-layer]) (ms @ [m]))) s1))

proof (rule no-failure-bindI[OF p.receive-random-field-element-no-failure])

fix b s2

assume Some (b, s2) $\in$ set-dist (execute p.receive-random-field-element s1)

obtain next-poly next-domain next-layer **where** next-layer-eq:

p.next-fri-layer (last ps) (last ds) b =

(next-poly, next-domain, next-layer)

by (cases p.next-fri-layer (last ps) (last ds) b)

show None $\notin$ dom (dist (execute

(let (next-poly, next-domain, next-layer) =

p.next-fri-layer (last ps) (last ds) b

in p.create next-layer $\gg$

(λ m. p.fri-commit n

(ps @ [next-poly]) (ds @ [next-domain])

(ls @ [next-layer]) (ms @ [m]))) s2))

unfolding next-layer-eq split Let-def

proof (rule no-failure-bindI[OF create-no-failure])

```

    fix m s3
    assume Some (m, s3) ∈ set-dist (execute (p.create next-layer) s2)
    show None ∉ dom (dist (execute
      (p.fri-commit n (ps @ [next-poly]) (ds @ [next-domain])
        (ls @ [next-layer]) (ms @ [m])) s3))
    by (rule Suc.IH)
  qed
qed
qed
qed

lemma fri-commit-with-no-failure:
  assumes receiver-no-failure:
    ∧s. None ∉ dom (dist (execute receive-challenge s))
  shows
    None ∉ dom (dist
      (execute (p.fri-commit-with receive-challenge n ps ds ls ms) s))
proof (induction n arbitrary: ps ds ls ms s)
  case 0
  then show ?case by simp
next
  case (Suc n)
  show ?case
    unfolding p.fri-commit-with.simps
  proof (rule no-failure-bindI[OF send-no-failure])
    fix u s1
    assume Some (u, s1) ∈ set-dist (execute (p.send (value (last ms))) s)
    show None ∉ dom (dist (execute
      (receive-challenge >>=
        (λb. let (next-poly, next-domain, next-layer) =
          p.next-fri-layer (last ps) (last ds) b
          in p.create next-layer >>=
            (λm. p.fri-commit-with receive-challenge n
              (ps @ [next-poly]) (ds @ [next-domain])
                (ls @ [next-layer]) (ms @ [m])))) s1)))
  proof (rule no-failure-bindI[OF receiver-no-failure])
    fix b s2
    assume Some (b, s2) ∈ set-dist (execute receive-challenge s1)
    obtain next-poly next-domain next-layer where next-layer-eq:
      p.next-fri-layer (last ps) (last ds) b =
        (next-poly, next-domain, next-layer)
    by (cases p.next-fri-layer (last ps) (last ds) b)
    show None ∉ dom (dist (execute
      (let (next-poly, next-domain, next-layer) =
        p.next-fri-layer (last ps) (last ds) b
      in p.create next-layer >>=
        (λm. p.fri-commit-with receive-challenge n
          (ps @ [next-poly]) (ds @ [next-domain])
            (ls @ [next-layer]) (ms @ [m])))) s2))
  end
end

```


unfolding *next-layer-eq split Let-def*
proof (rule no-failure-bindI[OF create-no-failure])
fix $m\ s3$
assume $\text{Some } (m, s3) \in \text{set-dist } (\text{execute } (p.\text{create next-layer})\ s2)$
show $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (p.\text{fri-commit-with receive-challenge } n$
 $(ps @ [\text{next-poly}])\ (ds @ [\text{next-domain}])$
 $(ls @ [\text{next-layer}])\ (ms @ [m]))\ s3))$
by (rule *Suc.IH*)
qed
qed
qed
qed

lemma *trace-fri-commit-no-failure:*
 $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (p.\text{trace-fri-commit } n\ ps\ ds\ ls\ ms)\ s))$
unfolding *p.trace-fri-commit-def*
by (rule *fri-commit-with-no-failure*)
(rule *p.receive-trace-fri-challenge-no-failure*)

lemma *composition-fri-commit-no-failure:*
 $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (p.\text{composition-fri-commit } n\ ps\ ds\ ls\ ms)\ s))$
unfolding *p.composition-fri-commit-def*
by (rule *fri-commit-with-no-failure*)
(rule *p.receive-composition-fri-challenge-no-failure*)

lemma *honest-alpha-send-round-no-failure:*
 $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (do \{$
 $a \leftarrow p.\text{receive-alpha-challenge};$
 $p.\text{send } a;$
 $\text{return } a$
 $\})\ s))$
by (intro *no-failure-bindI*)
(simp-all add: *p.receive-alpha-challenge-no-failure send-no-failure*)

lemma *prover-decommit-on-query-step-no-failure:*
 $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (do \{$
 $p.\text{send } x;$
 $\text{mfold2 } p.\text{send path}$
 $\})\ s))$
proof (rule no-failure-bindI[OF send-no-failure])
fix $u\ t$
assume $\text{Some } (u, t) \in \text{set-dist } (\text{execute } (p.\text{send } x)\ s)$
show $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (\text{mfold2 } p.\text{send path})\ t))$
by (rule no-failure-mfold2I) (simp add: *send-no-failure*)
qed

lemma *prover-decommit-on-query-no-failure*:
 $\text{None} \notin \text{dom} (\text{dist} (\text{execute} (\text{mmap} (p.\text{decommit-on-query } \text{idx } \text{f-merkle})) s))$
unfolding *p.decommit-on-query-def*
by (*rule no-failure-mmapI*)
(auto intro!: *prover-decommit-on-query-step-no-failure*)

lemma *prover-decommit-on-fri-layer-no-failure*:
 $\text{None} \notin \text{dom} (\text{dist} (\text{execute}$
(do {
let $\text{len} = \text{length } l$;
let $\text{idx}' = \text{idx} \bmod \text{len}$;
let $\text{sid}x = (\text{idx}' + (\text{len} \text{div } 2)) \bmod \text{len}$;
 $p.\text{send } (l ! \text{idx}')$;
 $\text{mfold2 } p.\text{send } (\text{get-authentication-path } \text{len } \text{idx}' m)$;
 $p.\text{send } (l ! \text{sid}x)$;
 $\text{mfold2 } p.\text{send } (\text{get-authentication-path } \text{len } \text{sid}x m)$;
return idx'
}) s))
unfolding *Let-def*
proof (*rule no-failure-bindI[OF send-no-failure]*)
fix $u \ s1$
assume $\text{Some } (u, s1) \in \text{set-dist } (\text{execute } (p.\text{send } (l ! (\text{idx} \bmod \text{length } l))) s)$
show $\text{None} \notin \text{dom} (\text{dist} (\text{execute}$
($\text{mfold2 } p.\text{send } (\text{get-authentication-path } (\text{length } l) (\text{idx} \bmod \text{length } l) m) \gg=$
($\lambda-. p.\text{send } (l ! ((\text{idx} \bmod \text{length } l + \text{length } l \text{div } 2) \bmod \text{length } l)) \gg=$
($\lambda-. \text{mfold2 } p.\text{send}$
($\text{get-authentication-path } (\text{length } l)$
($((\text{idx} \bmod \text{length } l + \text{length } l \text{div } 2) \bmod \text{length } l) m) \gg=$
($\lambda-. \text{return } (\text{idx} \bmod \text{length } l)))) s1))$
proof (*rule no-failure-bindI*)
show $\text{None} \notin \text{dom} (\text{dist} (\text{execute}$
($\text{mfold2 } p.\text{send } (\text{get-authentication-path } (\text{length } l) (\text{idx} \bmod \text{length } l) m) s1))$
by (*rule no-failure-mfold2I*) (*simp add: send-no-failure*)
next
fix $u2 \ s2$
assume $\text{Some } (u2, s2) \in \text{set-dist } (\text{execute}$
($\text{mfold2 } p.\text{send } (\text{get-authentication-path } (\text{length } l) (\text{idx} \bmod \text{length } l) m) s1)$
show $\text{None} \notin \text{dom} (\text{dist} (\text{execute}$
($p.\text{send } (l ! ((\text{idx} \bmod \text{length } l + \text{length } l \text{div } 2) \bmod \text{length } l)) \gg=$
($\lambda-. \text{mfold2 } p.\text{send}$
($\text{get-authentication-path } (\text{length } l)$
($((\text{idx} \bmod \text{length } l + \text{length } l \text{div } 2) \bmod \text{length } l) m) \gg=$
($\lambda-. \text{return } (\text{idx} \bmod \text{length } l)))) s2))$
proof (*rule no-failure-bindI[OF send-no-failure]*)
fix $u3 \ s3$
assume $\text{Some } (u3, s3) \in \text{set-dist } (\text{execute}$
($p.\text{send } (l ! ((\text{idx} \bmod \text{length } l + \text{length } l \text{div } 2) \bmod \text{length } l)) s2)$
show $\text{None} \notin \text{dom} (\text{dist} (\text{execute}$
($\text{mfold2 } p.\text{send}$

```

    (get-authentication-path (length l)
      ((idx mod length l + length l div 2) mod length l) m) >>=
    (λ-. return (idx mod length l))) s3))
  by (rule no-failure-bind-returnI)
    (rule no-failure-mfold2I, simp add: send-no-failure)
qed
qed
qed

lemma prover-decommit-on-fri-layers-no-failure:
  None ∉ dom (dist (execute (mfold idx (p.decommit-on-fri-layers fs)) s))
unfolding p.decommit-on-fri-layers-def
by (rule no-failure-mfoldI)
    (auto intro!: prover-decommit-on-fri-layer-no-failure)

lemma prover-query-round-no-failure:
  None ∉ dom (dist (execute (prover-query-round f-merkle f-ls f-ms ls ms) s))
unfolding prover-query-round-def
proof (rule no-failure-bindI[OF p.receive-query-index-challenge-no-failure])
  fix idx s1
assume Some (idx, s1) ∈ set-dist (execute p.receive-query-index-challenge s)
show None ∉ dom (dist (execute
  (mmap (p.decommit-on-query (p.index (to-nat idx)) f-merkle) >>=
    (λ-. mfold (p.index (to-nat idx))
      (p.decommit-on-fri-layers (butlast (zip f-ls f-ms))) >>=
      (λ-. mfold (p.index (to-nat idx))
        (p.decommit-on-fri-layers (butlast (zip ls ms))) >>=
        (λ-. return ()))))) s1))
proof (rule no-failure-bindI[OF prover-decommit-on-query-no-failure])
  fix u2 s2
assume Some (u2, s2) ∈ set-dist (execute
  (mmap (p.decommit-on-query (p.index (to-nat idx)) f-merkle)) s1)
show None ∉ dom (dist (execute
  (mfold (p.index (to-nat idx))
    (p.decommit-on-fri-layers (butlast (zip f-ls f-ms))) >>=
    (λ-. mfold (p.index (to-nat idx))
      (p.decommit-on-fri-layers (butlast (zip ls ms))) >>=
      (λ-. return ()))) s2))
proof (rule no-failure-bindI[OF prover-decommit-on-fri-layers-no-failure])
  fix u3 s3
assume Some (u3, s3) ∈ set-dist (execute
  (mfold (p.index (to-nat idx))
    (p.decommit-on-fri-layers (butlast (zip f-ls f-ms)))) s2)
show None ∉ dom (dist (execute
  (mfold (p.index (to-nat idx))
    (p.decommit-on-fri-layers (butlast (zip ls ms))) >>=
    (λ-. return ())) s3))
by (rule no-failure-bind-returnI)
    (rule prover-decommit-on-fri-layers-no-failure)

```

qed
 qed
 qed

lemma *prover-after-fri-layers-tail-no-failure:*

None $\notin$ *dom* (*dist* (*execute*
 (*do* {
 p.send (*hd* (*last* *ls*));
 ntimes (*prover-query-round* *f-merkle* *f-ls* *f-ms* *ls* *ms*) rounds
 }) *s*))

proof (*rule* *no-failure-bindI*[*OF* *send-no-failure*])

fix *u s1*
assume *Some* (*u, s1*) $\in$ *set-dist* (*execute* (*p.send* (*hd* (*last* *ls*))) *s*)
show *None* $\notin$ *dom* (*dist* (*execute*
 (*ntimes* (*prover-query-round* *f-merkle* *f-ls* *f-ms* *ls* *ms*) rounds) *s1*))
by (*rule* *no-failure-ntimesI*) (*rule* *prover-query-round-no-failure*)

qed

lemma *prover-after-fri-commit-no-failure:*

None $\notin$ *dom* (*dist* (*execute*
 ((*case* *fri-data* of (*ps, ds, ls, ms*) $\Rightarrow$
 do {
 p.send (*hd* (*last* *ls*));
 ntimes (*prover-query-round* *f-merkle* *f-ls* *f-ms* *ls* *ms*) rounds
 })) *s*))

proof –

obtain *ps ds ls ms* **where** *fri-data-eq*: *fri-data* = (*ps, ds, ls, ms*)
by (*cases* *fri-data*)
show *?thesis*
unfolding *fri-data-eq* *case-prod-unfold*
by (*rule* *prover-after-fri-layers-tail-no-failure*)

qed

lemma *prover-after-fri-commit-named-no-failure:*

None $\notin$ *dom* (*dist* (*execute*
 ((*case* *fri-data* of (*ps, ds, ls, ms*) $\Rightarrow$
 do {
 p.send (*hd* (*last* *ls*));
 ntimes (*prover-query-round* *f-merkle* *f-ls* *f-ms* *ls* *ms*) rounds
 })) *s*))

proof –

obtain *ps ds ls ms* **where** *fri-data-eq*: *fri-data* = (*ps, ds, ls, ms*)
by (*cases* *fri-data*)
show *?thesis*
unfolding *fri-data-eq*
apply (*simp* *add*: *case-prod-unfold*)
apply (*rule* *no-failure-bindI*[*OF* *send-no-failure*])
apply (*rule* *no-failure-ntimesI*)
apply (*rule* *prover-query-round-no-failure*)

done
qed

lemma *prover-after-alphas-no-failure:*

None $\notin$ *dom* (*dist* (*execute*
 (*do* {
p.send (*of-nat* (*degree* (*p.cp as p.f-powers*)));
cp-merkle $\leftarrow$ *p.create* (*p.cp-eval as*);
(ps, ds, ls, ms) $\leftarrow$
 p.composition-fri-commit (*ceil-log* (*degree* (*p.cp as p.f-powers*) + 1))
 [*p.cp as p.f-powers*] [*p.eval-domain*] [*p.cp-eval as*] [*cp-merkle*];
p.send (*hd* (*last ls*));
ntimes (*prover-query-round f-merkle f-ls f-ms ls ms*) *rounds*
 }) *s*))

unfolding *Let-def*

by (*intro no-failure-bindI no-failure-bind-returnI*)

(*simp-all add: create-no-failure composition-fri-commit-no-failure*
send-no-failure
p.receive-random-field-element-no-failure prover-decommit-on-query-no-failure
p.receive-alpha-challenge-no-failure p.receive-query-index-challenge-no-failure
prover-decommit-on-fri-layers-no-failure prover-after-fri-commit-no-failure
prover-after-fri-commit-named-no-failure)

lemma *prover-after-trace-fri-commit-no-failure:*

None $\notin$ *dom* (*dist* (*execute*
 ((*case trace-fri-data of* (*f-ps, f-ds, f-ls, f-ms*) $\Rightarrow$
 do {
 p.send (*hd* (*last f-ls*));
 as $\leftarrow$ *mmap* (*replicate* (*length spec*)
 (*do* {
 a $\leftarrow$ *p.receive-alpha-challenge*;
 p.send a;
 return a
 }));
 p.send (*of-nat* (*degree* (*p.cp as p.f-powers*)));
 cp-merkle $\leftarrow$ *p.create* (*p.cp-eval as*);
 (ps, ds, ls, ms) $\leftarrow$
 p.composition-fri-commit (*ceil-log* (*degree* (*p.cp as p.f-powers*) + 1))
 [*p.cp as p.f-powers*] [*p.eval-domain*] [*p.cp-eval as*] [*cp-merkle*];
 p.send (*hd* (*last ls*));
 ntimes (*prover-query-round f-merkle f-ls f-ms ls ms*) *rounds*
 })) *s*))

proof –

obtain *f-ps f-ds f-ls f-ms* **where** *trace-fri-data-eq:*

trace-fri-data = (*f-ps, f-ds, f-ls, f-ms*)

by (*cases trace-fri-data*)

show *?thesis*

unfolding *trace-fri-data-eq case-prod-unfold Let-def*

by (*intro no-failure-bindI no-failure-mmapI*)

```

    (simp-all add: send-no-failure honest-alpha-send-round-no-failure
      prover-after-alphas-no-failure create-no-failure
      composition-fri-commit-no-failure
      p.receive-alpha-challenge-no-failure p.receive-query-index-challenge-no-failure
      no-failure-ntimesI prover-query-round-no-failure split: prod.splits)
qed

```

```

lemma prover-monad-no-failure:
  None  $\notin$  dom (dist (execute p.prover-monad init-state))
  unfolding p.prover-monad-def Let-def
  apply (rule no-failure-bindI)
  apply (rule create-no-failure)
  apply (rule no-failure-bindI)
  apply (rule send-no-failure)
  apply (rule no-failure-bindI)
  apply (rule trace-fri-commit-no-failure)
  apply (simp only: prover-query-round-def[symmetric])
  apply (rule prover-after-trace-fri-commit-no-failure)
  done

```

```

lemma mfold-map-index-outcome-invariant-fromI:
  assumes init: I k a s
  and step-inv:
     $\bigwedge j \ x \ a \ s \ y \ t. \ k \leq j \implies j < k + \text{length } xs \implies$ 
     $x = xs \ ! \ (j - k) \implies I \ j \ a \ s \implies$ 
     $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (F \ x \ a) \ s) \implies$ 
     $I \ (\text{Suc } j) \ y \ t$ 
  and outcome:
     $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (\text{mfold } a \ (\text{map } F \ xs)) \ s)$ 
  shows I (k + length xs) y t
  using init step-inv outcome
proof (induction xs arbitrary: k a s y t)
  case Nil
  then show ?case by simp
next
  case (Cons x xs)
  from Cons.prem1(3) obtain y0 t0 where
    head:  $\text{Some } (y0, t0) \in \text{set-dist } (\text{execute } (F \ x \ a) \ s)$ 
  and tail:
     $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (\text{mfold } y0 \ (\text{map } F \ xs)) \ t0)$ 
  unfolding mfold.simps by (auto elim!: p.set-dist-bindE)
  have inv-next: I (Suc k) y0 t0
  by (rule Cons.prem1(2)[of k x a s y0 t0])
    (use Cons.prem1(1) head in simp-all)
  have tail-inv: I (Suc k + length xs) y t
  proof (rule Cons.IH[OF inv-next - tail])
    fix j x' a' s' y' t'
    assume ge:  $\text{Suc } k \leq j$ 
    and lt:  $j < \text{Suc } k + \text{length } xs$ 

```

```

    and  $x'$ -eq:  $x' = xs \ ! \ (j - \text{Suc } k)$ 
    and  $inv$ :  $I \ j \ a' \ s'$ 
    and  $out$ :  $\text{Some } (y', t') \in \text{set-dist } (\text{execute } (F \ x' \ a') \ s')$ 
  have  $j$ -gt:  $k < j$ 
    using  $ge$  by simp
  have  $j$ -lt:  $j < k + \text{length } (x \ \# \ xs)$ 
    using  $lt$  by simp
  have  $nth$ -eq:  $x' = (x \ \# \ xs) \ ! \ (j - k)$ 
    using  $x'$ -eq  $j$ -gt by simp
  show  $I \ (\text{Suc } j) \ y' \ t'$ 
    by (rule Cons.premis(2)[OF -  $j$ -lt  $nth$ -eq  $inv$   $out$ ]) (use  $ge$  in simp)
qed
then show ?case by simp
qed

```

lemma *mfold-map-index-outcome-invariantI*:

```

  assumes  $init$ :  $I \ 0 \ a \ s$ 
  and  $step$ - $inv$ :
     $\bigwedge j \ x \ a \ s \ y \ t. \ j < \text{length } xs \implies$ 
       $x = xs \ ! \ j \implies I \ j \ a \ s \implies$ 
       $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (F \ x \ a) \ s) \implies$ 
       $I \ (\text{Suc } j) \ y \ t$ 
  and  $outcome$ :
     $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (\text{mfold } a \ (\text{map } F \ xs)) \ s)$ 
  shows  $I \ (\text{length } xs) \ y \ t$ 
  using mfold-map-index-outcome-invariant-fromI[
    where  $I=I$  and  $k=0$  and  $xs=xs$  and  $F=F$ , OF  $init$  -  $outcome$ ]
     $step$ - $inv$ 
  by simp

```

lemma *set-dist-bindI*:

```

  assumes  $\text{Some } (x, u) \in \text{set-dist } (\text{execute } m \ s)$ 
  and  $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (f \ x) \ u)$ 
  shows  $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (m \gg f) \ s)$ 
proof -
  let ? $K$  =  $\text{bind-cont-map } (\text{map-fun } id \ dist \circ \text{execute } \circ f)$ 
  have  $x$ -dom:  $\text{Some } (x, u) \in \text{dom } (dist \ (\text{execute } m \ s))$ 
    using assms(1) unfolding set-dist-def .
  have  $y$ -dom':  $\text{Some } (y, t) \in \text{dom } (dist \ (\text{execute } (f \ x) \ u))$ 
    using assms(2) unfolding set-dist-def .
  have  $y$ -dom:  $\text{Some } (y, t) \in \text{dom } (?K \ (\text{Some } (x, u)))$ 
    using assms(2)
    unfolding set-dist-def bind-cont-map-def
    by (simp add: o-def map-fun-def)
  have  $x$ -pos:  $0 < the \ (dist \ (\text{execute } m \ s) \ (\text{Some } (x, u)))$ 
    using  $x$ -dom by simp
  have  $y$ -pos:  $0 < the \ (dist \ (\text{execute } (f \ x) \ u) \ (\text{Some } (y, t)))$ 
    using  $y$ -dom' by simp
  have  $y$ -opt-pos:

```

```

0 < option-default (?K (Some (x, u)) (Some (y, t)))
using y-dom' y-pos
by (cases dist (execute (f x) u) (Some (y, t)))
    (auto simp: bind-cont-map-def o-def map-fun-def domIff)
have term-pos:
0 <
    the (dist (execute m s) (Some (x, u))) *
    option-default (?K (Some (x, u)) (Some (y, t)))
using x-pos y-opt-pos by simp
have sum-nonzero:
(∑ i∈dom (dist (execute m s)).
    the (dist (execute m s) i) *
    option-default (?K i (Some (y, t)))) ≠ 0
proof –
have 0 <
(∑ i∈dom (dist (execute m s)).
    the (dist (execute m s) i) *
    option-default (?K i (Some (y, t))))
using sum-pos2[OF finite-dom-dist x-dom, of
    λi. the (dist (execute m s) i) *
    option-default (?K i (Some (y, t)))]
    term-pos
by auto
then show ?thesis by simp
qed
have Some (y, t) ∈
{z ∈ (∪ i∈dom (dist (execute m s)). dom (?K i)).
    (∑ i∈dom (dist (execute m s)).
        the (dist (execute m s) i) * option-default (?K i z)) ≠ 0}
using x-dom y-dom sum-nonzero by auto
then show ?thesis
unfolding set-dist-def sm-bind.rep-eq dist-bind.rep-eq
by (simp add: o-def map-fun-def dom-map-bind)
qed

lemma set-dist-throw-empty[simp]:
    Some x ∉ set-dist (execute throw s)
unfolding throw.rep-eq dist-throw-def dist-delta-dist delta-map-def set-dist-def
by simp

lemma assert-outcomeD:
assumes outcome: Some ((), t) ∈ set-dist (execute (assert b) s)
shows b t = s
proof –
show b
proof (cases b)
case True
then show ?thesis .
next

```



```

    case False
  then show ?thesis
    using outcome by (simp add: assert-def)
qed
then show  $t = s$ 
  using outcome by (simp add: assert-def)
qed

lemma assert-hash-extends:
  assumes Some  $((), t) \in$ 
    set-dist (execute (assert b) ( $s::('f, 'a)$  protocol-channel-scheme))
  shows  $s \leq t$ 
  using assert-outcomeD(2)[OF assms] p.hash-ext-refl[of s] by simp

lemma modify-verifier-replay-state-outcome:
  assumes Some  $(u, t) \in$ 
    set-dist (execute
      (modify ( $\lambda c. c(|PState := 0, PTranscript := rev (PTranscript c),$ 
        PTraceFriCounter := 0, PCompositionFriCounter := 0,
        PAlphaCounter := 0, PQueryCounter := 0|))) s)
  shows  $u = () \wedge t = \text{verifier-replay-state } s$ 
  using assms
  unfolding verifier-replay-state-def modify-def
  by (auto elim!: p.set-dist-bindE)

lemma transcript-extends-refl[simp]:
  transcript-extends s s
  unfolding transcript-extends-def by simp

lemma transcript-extends-trans:
  assumes transcript-extends u s
    and transcript-extends t u
  shows transcript-extends t s
  using assms unfolding transcript-extends-def by auto

lemma read-hash-extends:
  assumes Some  $(x, t) \in$  set-dist (execute p.read s)
  shows  $s \leq t$ 
  by (rule p.read-hash-extends[OF assms])

lemma ntimes-read-hash-extends:
  assumes Some  $(xs, t) \in$  set-dist (execute (ntimes p.read n) s)
  shows  $s \leq t$ 
  using assms
proof (induction n arbitrary: xs s t)
  case 0
  then show ?case
    by (simp add: p.hash-ext-refl)
next

```

case (*Suc n*)
from *Suc.prem*s **obtain** *x ys u* **where**
 read: *Some (x, u) ∈ set-dist (execute p.read s)*
 and *reads*:
 Some (ys, t) ∈ set-dist (execute (ntimes p.read n) u)
 by (*auto elim!*: *p.set-dist-bindE*)
have *s ≤ u*
 using *read* **by** (*rule read-hash-extends*)
moreover **have** *u ≤ t*
 using *reads* **by** (*rule Suc.IH*)
ultimately show *?case*
 by (*rule p.hash-ext-trans*)
qed

lemma *check-authentication-path-hash-extends*:
 assumes *outcome*:
 Some (h, t) ∈
 set-dist (execute (p.check-authentication-path len i v path) s)
 shows *s ≤ t*
 by (*rule p.check-authentication-path-hash-extends[OF outcome]*)

lemma *honest-trace-valid-raw-algebra*:
 assumes *honest-trace-valid*
 shows
 $\forall (c, \text{roots}, d) \in \text{set spec.}$
 $(\forall r \in \text{set } (p.g\text{-map roots}). \text{poly } (c \text{ p.f-powers}) r = 0) \wedge$
 $\text{degree } (c \text{ p.f-powers}) \leq d * (\text{clength} - 1) \wedge$
 $\text{distinct } (p.g\text{-map roots})$
 using *assms*
 unfolding *honest-trace-valid-def honest-trace-algebra-def*
 constraint-roots-vanish-def
 using *constraint-degree-bound p.g-map-distinct*
 by *force*

lemma *honest-trace-valid-algebra*:
 assumes *honest-trace-valid*
 shows *honest-trace-algebra*
 using *assms* **unfolding** *honest-trace-valid-def* **by** *simp*

lemma *straight-line-honest-transcript-replay*:
 assumes *PTranscript s = []*
 and *Some ((), sent) ∈ set-dist (execute (mfold2 p.send xs) s)*
 and *Some (ys, replayed) ∈*
 set-dist (execute
 (ntimes p.read (length xs))
 (sent(PState := PState s, PTranscript := rev (PTranscript sent))))
 shows *ys = xs ∧ replayed = sent(PTranscript := [])*
 using *p.honest-transcript-replay-mfold2-ntimes[OF assms]* .

lemma *honest-alpha-round-replay-no-failure*:

assumes *lookup*: *fmlookup* (*HashMap s*) (*AlphaChallenge (PAlphaCounter s) (PState s)*) = *Some a*

shows

$None \notin dom (dist (execute$
 $(do \{$
 $a0 \leftarrow p.receive-alpha-challenge;$
 $let a0' = a0;$
 $a1 \leftarrow p.read;$
 $let a1' = a1;$
 $assert (a0' = a1');$
 $return a1'$
 $\}) (s(PTranscript := a \# rest))))$

proof –

let *?start* = *s(PTranscript := a # rest)*

let *?prefix* =

$do \{$
 $a0 \leftarrow p.receive-alpha-challenge;$
 $a1 \leftarrow p.read;$
 $return (a0, a1)$
 $\}$

have *lookup-start*:

fmlookup (*HashMap ?start*) (*AlphaChallenge (PAlphaCounter ?start) (PState ?start)*) = *Some a*

using *lookup by simp*

have *prefix-no-failure*:

$None \notin dom (dist (execute ?prefix ?start))$

proof (*rule no-failure-bindI*)

show $None \notin dom (dist (execute p.receive-alpha-challenge ?start))$

by (*rule p.receive-alpha-challenge-no-failure*)

next

fix *a0 t*

assume *rand*: *Some* (*a0, t*) $\in set-dist (execute p.receive-alpha-challenge ?start)$

have *tr-t*: *PTranscript t* = *a # rest*

using *p.receive-alpha-challenge-known-outcome[OF lookup-start rand]* **by** *simp*

show $None \notin dom (dist (execute (p.read \gg= (\lambda a1. return (a0, a1))) t))$

proof (*rule no-failure-bindI*)

show $None \notin dom (dist (execute p.read t))$

by (*rule p.read-no-failure*) (*simp add: tr-t*)

next

fix *a1 u*

assume *Some* (*a1, u*) $\in set-dist (execute p.read t)$

show $None \notin dom (dist (execute (return (a0, a1)) u))$

by *simp*

qed

qed

have *full*:

$(do \{$
 $a0 \leftarrow p.receive-alpha-challenge;$

```

    let a0' = a0;
    a1 ← p.read;
    let a1' = a1;
    assert (a0' = a1');
    return a1'
  }) =
  (?prefix ≫ (λ(a0, a1). assert (a0 = a1) ≫ (λ-. return a1)))
  by (simp add: Let-def sm-bind-assoc split: prod.splits)
show ?thesis
  unfolding full
proof (rule no-failure-bindI[OF prefix-no-failure])
  fix aa t
  assume prefix: Some (aa, t) ∈ set-dist (execute ?prefix ?start)
  obtain a0 a1 where aa-eq: aa = (a0, a1)
    by (cases aa)
  from prefix[unfolded aa-eq] obtain r-state where
    rand:
      Some (a0, r-state) ∈ set-dist (execute p.receive-alpha-challenge ?start)
  and read:
      Some (a1, t) ∈ set-dist (execute p.read r-state)
  by (auto elim!: p.set-dist-bindE)
  have rand-res:
    a0 = a ∧ PState r-state = PState s ∧ PTranscript r-state = a # rest
  using p.receive-alpha-challenge-known-outcome[OF lookup-start rand] by simp
  have read-start:
    r-state = r-state(PTranscript := a # rest)
  using rand-res by simp
  have read':
    Some (a1, t) ∈ set-dist (execute p.read (r-state(PTranscript := a # rest)))
  by (subst read-start[symmetric]) (rule read)
  have a1 = a
    using p.read-cons-outcome[OF read'] rand-res by simp
  then show None ∉ dom
    (dist (execute ((case aa of (a0, a1) ⇒ assert (a0 = a1) ≫ (λ-. return a1))) t))
  using rand-res unfolding aa-eq assert-def by simp
qed
qed

```

lemma honest-alpha-round-replay-outcome:

assumes lookup: fmlookup (HashMap s) (AlphaChallenge (PAlphaCounter s) (PState s)) =
Some a

and outcome:

Some (y, t) ∈ set-dist (execute
 (do {
 a0 ← p.receive-alpha-challenge;
 let a0' = a0;
 a1 ← p.read;
 let a1' = a1;
 assert (a0' = a1');

```

    return a1'
  }) (s(PTranscript := a # rest)))
shows
  y = a ∧
  PState t = concat (PState s) a ∧
  PTranscript t = rest ∧
  PTraceFriCounter t = PTraceFriCounter s ∧
  PCompositionFriCounter t = PCompositionFriCounter s ∧
  PAlphaCounter t = Suc (PAlphaCounter s) ∧
  PQueryCounter t = PQueryCounter s ∧
  s ≤ t
proof –
  let ?start = s(PTranscript := a # rest)
  from outcome obtain a0 r-state read-state where
    rand:
      Some (a0, r-state) ∈ set-dist (execute p.receive-alpha-challenge ?start)
    and read:
      Some (y, read-state) ∈ set-dist (execute p.read r-state)
    and assert-ok: Some ((), t) ∈ set-dist (execute (assert (a0 = y)) read-state)
    by (auto simp: Let-def elim!: p.set-dist-bindE)
  have lookup-start: fmlookup (HashMap ?start) (AlphaChallenge (PAlphaCounter ?start) (PState
    ?start)) = Some a
    using lookup by simp
  have rand-known:
    a0 = a ∧
    PState r-state = PState s ∧
    PTranscript r-state = a # rest ∧
    ?start ≤ r-state
    using p.receive-alpha-challenge-known-outcome[OF lookup-start rand] by simp
  have read-start:
    r-state = r-state(PTranscript := a # rest)
    using rand-known by simp
  have read':
    Some (y, read-state) ∈
      set-dist (execute p.read (r-state(PTranscript := a # rest)))
    by (subst read-start[symmetric]) (rule read)
  have read-res:
    y = a ∧
    read-state =
      r-state(PState := concat (PState r-state) a, PTranscript := rest)
    using p.read-cons-outcome[OF read'] by simp
  have t-eq: t = read-state
    using assert-ok rand-known read-res
    unfolding assert-def by simp
  have alpha-counter:
    PAlphaCounter t = Suc (PAlphaCounter s)
    using p.receive-alpha-challenge-counter-outcome[OF rand] rand-known read-res
    unfolding t-eq by simp
  have other-counters:

```

```

PTraceFriCounter t = PTraceFriCounter s ∧
PCompositionFriCounter t = PCompositionFriCounter s ∧
PQueryCounter t = PQueryCounter s
using p.receive-alpha-challenge-counter-outcome[OF rand] rand-known read-res
unfolding t-eq by simp
have s-t: s ≤ t
proof –
  have s-start: s ≤ ?start
    unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  have start-r: ?start ≤ r-state
    using rand-known by simp
  have r-read: r-state ≤ read-state
    using read-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  show ?thesis
    using s-start start-r r-read unfolding t-eq
    by (meson p.hash-ext-trans)
qed
show ?thesis
  using rand-known read-res t-eq alpha-counter other-counters s-t by simp
qed

lemma honest-alpha-round-replay-parts-outcome:
  assumes lookup: fmlookup (HashMap s) (AlphaChallenge (PAlphaCounter s) (PState s)) =
Some a
  and rand:
    Some (a0, r-state) ∈
      set-dist (execute p.receive-alpha-challenge (s(PTranscript := a # rest))))
  and read:
    Some (y, read-state) ∈ set-dist (execute p.read r-state)
  and assert-ok:
    Some ((), t) ∈ set-dist (execute (assert (a0 = y)) read-state)
  shows
    y = a ∧
    PState t = concat (PState s) a ∧
    PTranscript t = rest ∧
    PTraceFriCounter t = PTraceFriCounter s ∧
    PCompositionFriCounter t = PCompositionFriCounter s ∧
    PAlphaCounter t = Suc (PAlphaCounter s) ∧
    PQueryCounter t = PQueryCounter s ∧
    s ≤ t
  proof –
    let ?start = s(PTranscript := a # rest)
    have lookup-start: fmlookup (HashMap ?start) (AlphaChallenge (PAlphaCounter ?start) (PState
    ?start)) = Some a
      using lookup by simp
    have rand-known:
      a0 = a ∧
      PState r-state = PState s ∧
      PTranscript r-state = a # rest ∧

```

```

    ?start ≤ r-state
  using p.receive-alpha-challenge-known-outcome[OF lookup-start rand] by simp
have read-start:
  r-state = r-state(PTranscript := a # rest)
  using rand-known by simp
have read':
  Some (y, read-state) ∈
    set-dist (execute p.read (r-state(PTranscript := a # rest)))
  by (subst read-start[symmetric]) (rule read)
have read-res:
  y = a ∧
  read-state =
    r-state(PState := concat (PState r-state) a, PTranscript := rest)
  using p.read-cons-outcome[OF read'] by simp
have t-eq: t = read-state
  using assert-ok rand-known read-res
  unfolding assert-def by simp
have alpha-counter:
  PAlphaCounter t = Suc (PAlphaCounter s)
  using p.receive-alpha-challenge-counter-outcome[OF rand] rand-known read-res
  unfolding t-eq by simp
have other-counters:
  PTraceFriCounter t = PTraceFriCounter s ∧
  PCompositionFriCounter t = PCompositionFriCounter s ∧
  PQueryCounter t = PQueryCounter s
  using p.receive-alpha-challenge-counter-outcome[OF rand] rand-known read-res
  unfolding t-eq by simp
have s-t: s ≤ t
proof -
  have s-start: s ≤ ?start
    unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  have start-r: ?start ≤ r-state
    using rand-known by simp
  have r-read: r-state ≤ read-state
    using read-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  show ?thesis
    using s-start start-r r-read unfolding t-eq
    by (meson p.hash-ext-trans)
qed
show ?thesis
  using rand-known read-res t-eq alpha-counter other-counters s-t by simp
qed

lemma honest-receive-fri-commit-replay-outcome:
  assumes lookup: fmlookup (HashMap s) (FiatShamirChallenge (concat (PState s) fri-root)) =
    Some challenge
  and outcome:
    Some ((b, r), t) ∈
      set-dist (execute v.receive-fri-commits (s(PTranscript := fri-root # rest)))

```

shows
 $r = \text{fri-root} \wedge$
 $b = \text{challenge} \wedge$
 $P\text{State } t = \text{concat } (P\text{State } s) \text{ fri-root} \wedge$
 $P\text{Transcript } t = \text{rest} \wedge$
 $s \leq t$
proof –
from *outcome* **obtain** u **where**
 $\text{read-root: } \text{Some } (r, u) \in \text{set-dist } (\text{execute } p.\text{read } (s \parallel P\text{Transcript} := \text{fri-root} \# \text{rest}))$
and $\text{rand: } \text{Some } (b, t) \in \text{set-dist } (\text{execute } p.\text{receive-random-field-element } u)$
unfolding $v.\text{receive-fri-commits-def}$
by $(\text{auto elim!}: p.\text{set-dist-bindE})$
have read-res:
 $r = \text{fri-root} \wedge$
 $u = s \parallel P\text{State} := \text{concat } (P\text{State } s) \text{ fri-root}, P\text{Transcript} := \text{rest}$
using $p.\text{read-cons-outcome}[OF \text{ read-root}]$ **by** *simp*
have $\text{lookup-u: } \text{fmlookup } (\text{HashMap } u) (\text{FiatShamirChallenge } (P\text{State } u)) = \text{Some challenge}$
using lookup read-res **by** *simp*
have rand-res:
 $b = \text{challenge} \wedge P\text{State } t = \text{concat } (P\text{State } s) \text{ fri-root} \wedge P\text{Transcript } t = \text{rest} \wedge u \leq t$
using $p.\text{receive-random-field-element-known-outcome}[OF \text{ lookup-u rand}]$ read-res **by** *simp*
have $s-u: s \leq u$
using read-res **unfolding** $\text{less-eq-hash-ext-def less-eq-fmap-def}$ **by** *simp*
have $s-t: s \leq t$
using $s-u \text{ rand-res}$ **by** $(\text{meson } p.\text{hash-ext-trans})$
show *?thesis*
using $\text{read-res rand-res s-t}$ **by** *simp*
qed

lemma *honest-receive-fri-commit-no-failure:*
assumes $P\text{Transcript } s \neq []$
shows
 $\text{None} \notin \text{dom } (\text{dist } (\text{execute } v.\text{receive-fri-commits } s))$
using *assms*
unfolding $v.\text{receive-fri-commits-def}$
by $(\text{intro no-failure-bindI})$ $(\text{simp-all add: } p.\text{read-no-failure } p.\text{receive-random-field-element-no-failure})$

lemma *receive-fri-commit-preserves-query-counter:*
assumes *outcome:*
 $\text{Some } (br, t) \in \text{set-dist } (\text{execute } v.\text{receive-fri-commits } s)$
shows $P\text{QueryCounter } t = P\text{QueryCounter } s$
proof –
from *outcome* **obtain** $r u b$ **where**
 $\text{read-root: } \text{Some } (r, u) \in \text{set-dist } (\text{execute } p.\text{read } s)$
and $\text{rand: } \text{Some } (b, t) \in \text{set-dist } (\text{execute } p.\text{receive-random-field-element } u)$
unfolding $v.\text{receive-fri-commits-def}$
by $(\text{auto elim!}: p.\text{set-dist-bindE})$
have $q\text{-read: } P\text{QueryCounter } u = P\text{QueryCounter } s$
proof $(\text{cases } P\text{Transcript } s)$


```

    case Nil
    then have False
      using read-root
      unfolding p.read-def protocol-read-def assert-def
      by (auto elim!: p.set-dist-bindE)
    then show ?thesis by simp
  next
    case (Cons x xs)
    then show ?thesis
      using p.read-nonempty-outcome[OF read-root Cons] by simp
  qed
  have q-rand: PQueryCounter t = PQueryCounter u
    using p.receive-random-field-element-counter-outcome[OF rand] by simp
  show ?thesis
    using q-read q-rand by simp
qed

```

lemma *receive-fri-commits-preserves-query-counter*:

```

  assumes outcome:
    Some (brs, t) ∈ set-dist (execute (ntimes v.receive-fri-commits n) s)
  shows PQueryCounter t = PQueryCounter s
  using outcome
proof (induction n arbitrary: brs s t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.premis obtain br u brs' where
    head: Some (br, u) ∈ set-dist (execute v.receive-fri-commits s)
    and tail:
      Some (brs', t) ∈
        set-dist (execute (ntimes v.receive-fri-commits n) u)
    by (auto elim!: p.set-dist-bindE)
  have q-head: PQueryCounter u = PQueryCounter s
    by (rule receive-fri-commit-preserves-query-counter[OF head])
  have q-tail: PQueryCounter t = PQueryCounter u
    by (rule Suc.IH[OF tail])
  show ?case
    using q-head q-tail by simp
qed

```

lemma *honest-receive-fri-commits-list-outcome*:

```

  assumes lookups:
    ∀ i < length roots.
      fmlookup (HashMap s) (FiatShamirChallenge (foldl concat (PState s) (take (Suc i) roots)))
  =
    Some (challenges ! i)
  and len: length challenges = length roots
  and outcome:

```

```

    Some (brs, t) ∈
      set-dist (execute (ntimes v.receive-fri-commits (length roots))
        (s(PTranscript := roots @ rest)))
  shows
    brs = zip challenges roots ∧
    PState t = foldl concat (PState s) roots ∧
    PTranscript t = rest ∧
    s ≤ t
  using lookups len outcome
proof (induction roots arbitrary: challenges brs s t rest)
  case Nil
  have brs-eq: brs = []
  using Nil.premis(3) by simp
  have t-eq: t = s(PTranscript := rest)
  using Nil.premis(3) by simp
  have s-t: s ≤ t
  unfolding t-eq less-eq-hash-ext-def less-eq-fmap-def by simp
  show ?case
  using brs-eq t-eq s-t Nil.premis(2) by simp
next
  case (Cons root roots)
  from Cons.premis(2) obtain challenge challenges' where challenges-eq:
    challenges = challenge # challenges'
  by (cases challenges) auto
  from Cons.premis(3) obtain br u brs' where
    head:
      Some (br, u) ∈
        set-dist (execute v.receive-fri-commits
          (s(PTranscript := root # (roots @ rest))))
    and tail:
      Some (brs', t) ∈
        set-dist (execute (ntimes v.receive-fri-commits (length roots)) u)
  and brs-eq: brs = br # brs'
  by (auto elim!: p.set-dist-bindE)
  have lookup-root:
    fmlookup (HashMap s) (FiatShamirChallenge (concat (PState s) root)) = Some challenge
  using spec[OF Cons.premis(1), of 0] challenges-eq by simp
  obtain b r where br-eq: br = (b, r)
  by (cases br)
  have head-out:
    r = root ∧
    b = challenge ∧
    PState u = concat (PState s) root ∧
    PTranscript u = roots @ rest ∧
    s ≤ u
  using honest-receive-fri-commit-replay-outcome[
    OF lookup-root head[unfolded br-eq]]
  unfolding br-eq by simp
  have tail-lookups:

```

```

   $\forall i < \text{length roots}.$ 
   $\text{fmlookup (HashMap } u) (\text{FiatShamirChallenge (foldl concat (PState } u) (\text{take (Suc } i) \text{ roots})))$ 
=
   $\text{Some (challenges' ! } i)$ 
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{length roots}$ 
  have old-lookup:
     $\text{fmlookup (HashMap } s)$ 
     $(\text{FiatShamirChallenge (foldl concat (PState } s) (\text{take (Suc (Suc } i)) (\text{root \# roots})))) =$ 
     $\text{Some ((challenge \# challenges') ! Suc } i)$ 
    using spec[OF Cons.prems(1), of Suc i] i-bound challenges-eq by simp
  have key-eq:
     $\text{foldl concat (PState } s) (\text{take (Suc (Suc } i)) (\text{root \# roots})) =$ 
     $\text{foldl concat (PState } u) (\text{take (Suc } i) \text{ roots})$ 
    using head-out by simp
  show  $\text{fmlookup (HashMap } u) (\text{FiatShamirChallenge (foldl concat (PState } u) (\text{take (Suc } i)$ 
 $\text{roots}))) =$ 
     $\text{Some (challenges' ! } i)$ 
    using p.hash-extension-lookup[OF old-lookup, of u] head-out key-eq i-bound
    by simp
qed
have u-update:  $u(\text{PTranscript := roots @ rest}) = u$ 
  using head-out by simp
have tail':
   $\text{Some (brs', } t) \in$ 
   $\text{set-dist (execute (ntimes } v.\text{receive-fri-commits (length roots))$ 
   $(u(\text{PTranscript := roots @ rest})))$ 
  using tail unfolding u-update .
have tail-len:  $\text{length challenges' = length roots}$ 
  using Cons.prems(2) challenges-eq by simp
have tail-out:
   $\text{brs' = zip challenges' roots} \wedge$ 
   $\text{PState } t = \text{foldl concat (PState } u) \text{ roots} \wedge$ 
   $\text{PTranscript } t = \text{rest} \wedge$ 
   $u \leq t$ 
  using Cons.IH[OF tail-lookups tail-len tail] .
have s-t:  $s \leq t$ 
  using head-out tail-out by (meson p.hash-ext-trans)
show ?case
  using brs-eq br-eq head-out tail-out s-t challenges-eq by simp
qed

lemma honest-receive-fri-commits-list-no-failure:
assumes lookups:
   $\forall i < \text{length roots}.$ 
   $\text{fmlookup (HashMap } s) (\text{FiatShamirChallenge (foldl concat (PState } s) (\text{take (Suc } i) \text{ roots})))$ 
=
   $\text{Some (challenges ! } i)$ 

```

```

    and len: length challenges = length roots
shows
  None  $\notin$  dom (dist (execute
    (ntimes v.receive-fri-commits (length roots))
    (s(PTranscript := roots @ rest))))
using lookups len
proof (induction roots arbitrary: challenges s rest)
  case Nil
  then show ?case by simp
next
  case (Cons root roots)
  from Cons.prem1(2) obtain challenge challenges' where challenges-eq:
    challenges = challenge # challenges'
  by (cases challenges) auto
  have lookup-root:
    fmlookup (HashMap s) (FiatShamirChallenge (concat (PState s) root)) = Some challenge
  using spec[OF Cons.prem1(1), of 0] challenges-eq by simp
  have head-nf:
    None  $\notin$  dom (dist (execute v.receive-fri-commits
      (s(PTranscript := root # (roots @ rest)))))
  by (rule honest-receive-fri-commit-no-failure) simp
  have case-eq:
    ?case =
      (None  $\notin$  dom (dist (execute
        (v.receive-fri-commits  $\gg$ 
          ( $\lambda$ br. ntimes v.receive-fri-commits (length roots)  $\gg$ 
            ( $\lambda$ brs. return (br # brs)))))
        (s(PTranscript := root # (roots @ rest)))))
  by simp
  show ?case
  unfolding case-eq
  proof (rule no-failure-bindI[OF head-nf])
    fix br u
    assume head:
      Some (br, u)  $\in$ 
        set-dist (execute v.receive-fri-commits
          (s(PTranscript := root # roots @ rest)))
    obtain b r where br-eq: br = (b, r)
    by (cases br)
    have head-out:
      r = root  $\wedge$ 
      b = challenge  $\wedge$ 
      PState u = concat (PState s) root  $\wedge$ 
      PTranscript u = roots @ rest  $\wedge$ 
      s  $\leq$  u
    using honest-receive-fri-commit-replay-outcome[
      OF lookup-root head[unfolded br-eq]]
    unfolding br-eq by simp
  have tail-lookups:

```

```

     $\forall i < \text{length roots}.$ 
     $\text{fmlookup} (\text{HashMap } u) (\text{FiatShamirChallenge} (\text{foldl concat} (\text{PState } u) (\text{take} (\text{Suc } i) \text{ roots})))$ 
  =
     $\text{Some} (\text{challenges}' ! i)$ 
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{length roots}$ 
  have old-lookup:
     $\text{fmlookup} (\text{HashMap } s)$ 
     $(\text{FiatShamirChallenge} (\text{foldl concat} (\text{PState } s) (\text{take} (\text{Suc} (\text{Suc } i)) (\text{root} \# \text{ roots})))) =$ 
     $\text{Some} ((\text{challenge} \# \text{ challenges}') ! \text{Suc } i)$ 
  using spec[OF Cons.prem1, of Suc i] i-bound challenges-eq by simp
  have key-eq:
     $\text{foldl concat} (\text{PState } s) (\text{take} (\text{Suc} (\text{Suc } i)) (\text{root} \# \text{ roots})) =$ 
     $\text{foldl concat} (\text{PState } u) (\text{take} (\text{Suc } i) \text{ roots})$ 
  using head-out by simp
  show  $\text{fmlookup} (\text{HashMap } u) (\text{FiatShamirChallenge} (\text{foldl concat} (\text{PState } u) (\text{take} (\text{Suc } i) \text{ roots}))) =$ 
     $\text{Some} (\text{challenges}' ! i)$ 
  using p.hash-extension-lookup[OF old-lookup, of u] head-out key-eq i-bound
  by simp
qed
have tail-len:  $\text{length challenges}' = \text{length roots}$ 
using Cons.prem2 challenges-eq by simp
have tail-nf-start:
   $\text{None} \notin \text{dom} (\text{dist} (\text{execute}$ 
     $(\text{ntimes } v.\text{receive-fri-commits} (\text{length roots}))$ 
     $(u(P\text{Transcript} := \text{roots} @ \text{rest}))))$ 
  using Cons.IH[OF tail-lookups tail-len, of rest] .
have u-update:  $u(P\text{Transcript} := \text{roots} @ \text{rest}) = u$ 
using head-out by simp
have tail-nf:
   $\text{None} \notin \text{dom} (\text{dist} (\text{execute}$ 
     $(\text{ntimes } v.\text{receive-fri-commits} (\text{length roots})) u))$ 
  using tail-nf-start unfolding u-update .
show  $\text{None} \notin \text{dom} (\text{dist} (\text{execute}$ 
     $(\text{ntimes } v.\text{receive-fri-commits} (\text{length roots}) \gg=$ 
     $(\lambda \text{brs}. \text{return} (\text{br} \# \text{brs}))) u))$ 
  by (rule no-failure-bindI[OF tail-nf]) simp
qed
qed

lemma honest-receive-fri-commit-with-no-failure:
assumes receiver-nf:
   $\bigwedge s. \text{None} \notin \text{dom} (\text{dist} (\text{execute } \text{receive-challenge } s))$ 
and tr:  $P\text{Transcript } s \neq []$ 
shows
   $\text{None} \notin \text{dom} (\text{dist}$ 
     $(\text{execute } (v.\text{receive-fri-commits-with } \text{receive-challenge } s)))$ 

```

```

using tr
unfolding v.receive-fri-commits-with-def
by (intro no-failure-bindI)
  (simp-all add: p.read-no-failure receiver-nf)

lemma honest-receive-trace-fri-commit-replay-outcome:
assumes lookup:
  fmlookup (HashMap s)
  (TraceFriChallenge (PTraceFriCounter s)
    (concat (PState s) fri-root)) = Some challenge
and outcome:
  Some ((b, r), t) ∈
    set-dist (execute
      v.receive-trace-fri-commits
      (s(PTranscript := fri-root # rest)))
shows
  r = fri-root ∧
  b = challenge ∧
  PState t = concat (PState s) fri-root ∧
  PTranscript t = rest ∧
  PTraceFriCounter t = Suc (PTraceFriCounter s) ∧
  s ≤ t
proof -
from outcome obtain u where
  read-root:
    Some (r, u) ∈
      set-dist (execute p.read
        (s(PTranscript := fri-root # rest)))
and rand:
  Some (b, t) ∈ set-dist (execute p.receive-trace-fri-challenge u)
unfolding v.receive-trace-fri-commits-def v.receive-fri-commits-with-def
by (auto elim!: p.set-dist-bindE)
have read-res:
  r = fri-root ∧
  u = s(PState := concat (PState s) fri-root, PTranscript := rest)
using p.read-cons-outcome[OF read-root] by simp
have lookup-u:
  fmlookup (HashMap u) (TraceFriChallenge (PTraceFriCounter u) (PState u)) = Some chal-
  lenge
using lookup read-res by simp
have rand-res:
  b = challenge ∧ PState t = concat (PState s) fri-root ∧
  PTranscript t = rest ∧ u ≤ t
using p.receive-trace-fri-challenge-known-outcome[OF lookup-u rand]
  read-res by simp
have rand-counter:
  PTraceFriCounter t = Suc (PTraceFriCounter s)
using p.receive-trace-fri-challenge-counter-outcome[OF rand] read-res
by simp

```

```

have s-u:  $s \leq u$ 
  using read-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have s-t:  $s \leq t$ 
  using s-u rand-res by (meson p.hash-ext-trans)
show ?thesis
  using read-res rand-res rand-counter s-t by simp
qed

lemma honest-receive-composition-fri-commit-replay-outcome:
  assumes lookup:
    fmlookup (HashMap s)
    (CompositionFriChallenge (PCompositionFriCounter s)
      (concat (PState s) fri-root)) =
      Some challenge
  and outcome:
    Some ((b, r), t) ∈
      set-dist (execute
        v.receive-composition-fri-commits
        (s(PTranscript := fri-root # rest)))
  shows
    r = fri-root ∧
    b = challenge ∧
    PState t = concat (PState s) fri-root ∧
    PTranscript t = rest ∧
    PCompositionFriCounter t = Suc (PCompositionFriCounter s) ∧
    s ≤ t
proof –
  from outcome obtain u where
    read-root:
      Some (r, u) ∈
        set-dist (execute p.read
          (s(PTranscript := fri-root # rest)))
  and rand:
    Some (b, t) ∈
      set-dist (execute p.receive-composition-fri-challenge u)
  unfolding v.receive-composition-fri-commits-def
    v.receive-fri-commits-with-def
  by (auto elim!: p.set-dist-bindE)
have read-res:
  r = fri-root ∧
  u = s(PState := concat (PState s) fri-root, PTranscript := rest)
  using p.read-cons-outcome[OF read-root] by simp
have lookup-u:
  fmlookup (HashMap u) (CompositionFriChallenge (PCompositionFriCounter u) (PState u))
=
  Some challenge
  using lookup read-res by simp
have rand-res:
  b = challenge ∧ PState t = concat (PState s) fri-root ∧

```

```

    PTranscript t = rest ∧ u ≤ t
  using p.receive-composition-fri-challenge-known-outcome[OF lookup-u rand]
    read-res by simp
have rand-counter:
  PCompositionFriCounter t = Suc (PCompositionFriCounter s)
  using p.receive-composition-fri-challenge-counter-outcome[OF rand] read-res
  by simp
have s-u: s ≤ u
  using read-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have s-t: s ≤ t
  using s-u rand-res by (meson p.hash-ext-trans)
show ?thesis
  using read-res rand-res rand-counter s-t by simp
qed

```

```

lemma read-preserves-counters:
  assumes outcome: Some (r, t) ∈ set-dist (execute p.read s)
  shows
    PTraceFriCounter t = PTraceFriCounter s ∧
    PCompositionFriCounter t = PCompositionFriCounter s ∧
    PAlphaCounter t = PAlphaCounter s ∧
    PQueryCounter t = PQueryCounter s
proof (cases PTranscript s)
case Nil
  then have False
    using outcome
    unfolding p.read-def protocol-read-def assert-def
    by (auto elim!: p.set-dist-bindE)
  then show ?thesis by simp
next
case (Cons x xs)
  then show ?thesis
    using p.read-nonempty-outcome[OF outcome Cons] by simp
qed

```

```

lemma ntimes-read-preserves-query-counter:
  assumes outcome: Some (xs, t) ∈ set-dist (execute (ntimes p.read n) s)
  shows PQueryCounter t = PQueryCounter s
  using outcome
proof (induction n arbitrary: xs s t)
case 0
  then show ?case by simp
next
case (Suc n)
  from Suc.premis obtain x u xs' where
    head: Some (x, u) ∈ set-dist (execute p.read s)
    and tail: Some (xs', t) ∈ set-dist (execute (ntimes p.read n) u)
    by (auto elim!: p.set-dist-bindE)
  have q-head: PQueryCounter u = PQueryCounter s

```



```

    using read-preserves-counters[OF head] by simp
  have q-tail: PQueryCounter t = PQueryCounter u
    by (rule Suc.IH[OF tail])
  show ?case
    using q-head q-tail by simp
qed

lemma receive-trace-fri-commit-preserves-other-counters:
  assumes outcome:
    Some (br, t) ∈ set-dist (execute v.receive-trace-fri-commits s)
  shows
    PCompositionFriCounter t = PCompositionFriCounter s ∧
    PAlphaCounter t = PAlphaCounter s ∧
    PQueryCounter t = PQueryCounter s
proof –
  from outcome obtain r u b where
    read-root: Some (r, u) ∈ set-dist (execute p.read s)
  and rand: Some (b, t) ∈ set-dist (execute p.receive-trace-fri-challenge u)
  unfolding v.receive-trace-fri-commits-def v.receive-fri-commits-with-def
  by (auto elim!: p.set-dist-bindE)
  have read-res:
    PCompositionFriCounter u = PCompositionFriCounter s ∧
    PAlphaCounter u = PAlphaCounter s ∧
    PQueryCounter u = PQueryCounter s
  using read-preserves-counters[OF read-root] by simp
  show ?thesis
    using p.receive-trace-fri-challenge-counter-outcome[OF rand] read-res
    by simp
qed

lemma receive-composition-fri-commit-preserves-other-counters:
  assumes outcome:
    Some (br, t) ∈ set-dist (execute v.receive-composition-fri-commits s)
  shows
    PTraceFriCounter t = PTraceFriCounter s ∧
    PAlphaCounter t = PAlphaCounter s ∧
    PQueryCounter t = PQueryCounter s
proof –
  from outcome obtain r u b where
    read-root: Some (r, u) ∈ set-dist (execute p.read s)
  and rand:
    Some (b, t) ∈ set-dist (execute p.receive-composition-fri-challenge u)
  unfolding v.receive-composition-fri-commits-def
    v.receive-fri-commits-with-def
  by (auto elim!: p.set-dist-bindE)
  have read-res:
    PTraceFriCounter u = PTraceFriCounter s ∧
    PAlphaCounter u = PAlphaCounter s ∧
    PQueryCounter u = PQueryCounter s

```

```

    using read-preserves-counters[OF read-root] by simp
  show ?thesis
    using p.receive-composition-fri-challenge-counter-outcome[OF rand]
      read-res
    by simp
qed

```

lemma *receive-trace-fri-commits-preserves-other-counters:*

```

  assumes outcome:
    Some (brs, t) ∈
      set-dist (execute (ntimes v.receive-trace-fri-commits n) s)
  shows
    PCompositionFriCounter t = PCompositionFriCounter s ∧
    PAlphaCounter t = PAlphaCounter s ∧
    PQueryCounter t = PQueryCounter s
  using outcome
proof (induction n arbitrary: brs s t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.premis obtain br u brs' where
    head: Some (br, u) ∈ set-dist (execute v.receive-trace-fri-commits s)
  and tail:
    Some (brs', t) ∈
      set-dist (execute (ntimes v.receive-trace-fri-commits n) u)
  by (auto elim!: p.set-dist-bindE)
  have head-pres:
    PCompositionFriCounter u = PCompositionFriCounter s ∧
    PAlphaCounter u = PAlphaCounter s ∧
    PQueryCounter u = PQueryCounter s
  using receive-trace-fri-commit-preserves-other-counters[OF head] .
  have tail-pres:
    PCompositionFriCounter t = PCompositionFriCounter u ∧
    PAlphaCounter t = PAlphaCounter u ∧
    PQueryCounter t = PQueryCounter u
  using Suc.IH[OF tail] .
  show ?case
    using head-pres tail-pres by simp
qed

```

lemma *receive-composition-fri-commits-preserves-other-counters:*

```

  assumes outcome:
    Some (brs, t) ∈
      set-dist (execute (ntimes v.receive-composition-fri-commits n) s)
  shows
    PTraceFriCounter t = PTraceFriCounter s ∧
    PAlphaCounter t = PAlphaCounter s ∧
    PQueryCounter t = PQueryCounter s

```

```

using outcome
proof (induction n arbitrary: brs s t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.prems obtain br u brs' where
    head:
      Some (br, u) ∈
        set-dist (execute v.receive-composition-fri-commits s)
    and tail:
      Some (brs', t) ∈
        set-dist (execute (ntimes v.receive-composition-fri-commits n) u)
    by (auto elim!: p.set-dist-bindE)
  have head-pres:
    PTraceFriCounter u = PTraceFriCounter s ∧
    PAlphaCounter u = PAlphaCounter s ∧
    PQueryCounter u = PQueryCounter s
    using receive-composition-fri-commit-preserves-other-counters[OF head] .
  have tail-pres:
    PTraceFriCounter t = PTraceFriCounter u ∧
    PAlphaCounter t = PAlphaCounter u ∧
    PQueryCounter t = PQueryCounter u
    using Suc.IH[OF tail] .
  show ?case
    using head-pres tail-pres by simp
qed

```

```

lemma honest-receive-fri-commits-list-outcome-with:
  fixes counter :: ('f, 'a) protocol-channel-scheme ⇒ nat
  and tag :: nat ⇒ 'f ⇒ 'f protocol-hash-input
  assumes step:
     $\bigwedge s \text{ root rest challenge } b \ r \ t.$ 
     $\text{fmlookup } (\text{HashMap } s) \ (\text{tag } (\text{counter } s) \ (\text{concat } (\text{PState } s) \ \text{root})) =$ 
      Some challenge  $\implies$ 
    Some ((b, r), t) ∈
      set-dist (execute receive-commits
        (s(PTranscript := root # rest)))  $\implies$ 
     $r = \text{root} \wedge$ 
     $b = \text{challenge} \wedge$ 
     $\text{PState } t = \text{concat } (\text{PState } s) \ \text{root} \wedge$ 
     $\text{PTranscript } t = \text{rest} \wedge$ 
     $\text{counter } t = \text{Suc } (\text{counter } s) \wedge$ 
     $s \leq t$ 
  and counter-update:
     $\bigwedge s \ tr. \text{counter } (s(\text{PTranscript} := tr)) = \text{counter } s$ 
  and lookups:
     $\forall i < \text{length roots}.$ 
     $\text{fmlookup } (\text{HashMap } s)$ 

```

```

      (tag (counter s + i)
        (foldl concat (PState s) (take (Suc i) roots))) =
      Some (challenges ! i)
    and len: length challenges = length roots
    and outcome:
      Some (brs, t) ∈
        set-dist (execute (ntimes receive-commits (length roots))
          (s(PTranscript := roots @ rest)))
  shows
    brs = zip challenges roots ∧
    PState t = foldl concat (PState s) roots ∧
    PTranscript t = rest ∧
    counter t = counter s + length roots ∧
    s ≤ t
  using lookups len outcome
proof (induction roots arbitrary: challenges brs s t rest)
case Nil
  have brs-eq: brs = []
  using Nil.premis(3) by simp
  have t-eq: t = s(PTranscript := rest)
  using Nil.premis(3) by simp
  have s-t: s ≤ t
  unfolding t-eq less-eq-hash-ext-def less-eq-fmap-def by simp
  have counter-t: counter t = counter s
  using t-eq counter-update by simp
  show ?case
  using brs-eq t-eq s-t counter-t Nil.premis(2) by simp
next
case (Cons root roots)
from Cons.premis(2) obtain challenge challenges' where challenges-eq:
  challenges = challenge # challenges'
by (cases challenges) auto
from Cons.premis(3) obtain br u brs' where
  head:
    Some (br, u) ∈
      set-dist (execute receive-commits
        (s(PTranscript := root # (roots @ rest))))
  and tail:
    Some (brs', t) ∈
      set-dist (execute (ntimes receive-commits (length roots)) u)
  and brs-eq: brs = br # brs'
by (auto elim!: p.set-dist-bindE)
have lookup-root:
  fmllookup (HashMap s)
    (tag (counter s) (concat (PState s) root)) = Some challenge
  using spec[OF Cons.premis(1), of 0] challenges-eq by simp
obtain b r where br-eq: br = (b, r)
by (cases br)
have head-out:

```

```

r = root ∧
b = challenge ∧
PState u = concat (PState s) root ∧
PTranscript u = roots @ rest ∧
counter u = Suc (counter s) ∧
s ≤ u
using step[OF lookup-root head[unfolded br-eq]]
unfolding br-eq by simp
have tail-lookups:
  ∀ i < length roots.
    fmlookup (HashMap u)
      (tag (counter u + i)
        (foldl concat (PState u) (take (Suc i) roots))) =
      Some (challenges' ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have old-lookup:
    fmlookup (HashMap s)
      (tag (counter s + Suc i)
        (foldl concat (PState s) (take (Suc (Suc i)) (root # roots)))) =
      Some ((challenge # challenges') ! Suc i)
    using spec[OF Cons.prems(1), of Suc i] i-bound challenges-eq by simp
  have key-eq:
    foldl concat (PState s) (take (Suc (Suc i)) (root # roots)) =
      foldl concat (PState u) (take (Suc i) roots)
    using head-out by simp
  show fmlookup (HashMap u)
    (tag (counter u + i)
      (foldl concat (PState u) (take (Suc i) roots))) =
    Some (challenges' ! i)
    using p.hash-extension-lookup[OF old-lookup, of u] head-out key-eq i-bound
    by simp
qed
have u-update: u(PTranscript := roots @ rest) = u
  using head-out by simp
have tail':
  Some (brs', t) ∈
    set-dist (execute (ntimes receive-commits (length roots))
      (u(PTranscript := roots @ rest)))
  using tail unfolding u-update .
have tail-len: length challenges' = length roots
  using Cons.prems(2) challenges-eq by simp
have tail-out:
  brs' = zip challenges' roots ∧
  PState t = foldl concat (PState u) roots ∧
  PTranscript t = rest ∧
  counter t = counter u + length roots ∧
  u ≤ t

```

```

    using Cons.IH[OF tail-lookups tail-len tail] .
  have s-t:  $s \leq t$ 
    using head-out tail-out by (meson p.hash-ext-trans)
  show ?case
    using brs-eq br-eq head-out tail-out s-t challenges-eq by simp
qed

lemma honest-receive-fri-commits-list-no-failure-with:
  fixes counter :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  nat
    and tag :: nat  $\Rightarrow$  'f  $\Rightarrow$  'f protocol-hash-input
  assumes step:
     $\bigwedge s$  root rest challenge b r t.
    fmlookup (HashMap s) (tag (counter s) (concat (PState s) root)) =
      Some challenge  $\Rightarrow$ 
      Some ((b, r), t)  $\in$ 
        set-dist (execute receive-commits
          (s(PTranscript := root # rest)))  $\Rightarrow$ 
          r = root  $\wedge$ 
          b = challenge  $\wedge$ 
          PState t = concat (PState s) root  $\wedge$ 
          PTranscript t = rest  $\wedge$ 
          counter t = Suc (counter s)  $\wedge$ 
          s  $\leq$  t
    and step-nf:
       $\bigwedge s$ . PTranscript s  $\neq [] \Rightarrow$ 
        None  $\notin$  dom (dist (execute receive-commits s))
    and lookups:
       $\forall i < \text{length roots}$ .
      fmlookup (HashMap s)
        (tag (counter s + i)
          (foldl concat (PState s) (take (Suc i) roots))) =
        Some (challenges ! i)
    and len: length challenges = length roots
  shows
    None  $\notin$  dom (dist (execute
      (ntimes receive-commits (length roots))
      (s(PTranscript := roots @ rest))))
    using lookups len
  proof (induction roots arbitrary: challenges s rest)
    case Nil
      then show ?case by simp
    next
      case (Cons root roots)
        from Cons.premis(2) obtain challenge challenges' where challenges-eq:
          challenges = challenge # challenges'
          by (cases challenges) auto
        have lookup-root:
          fmlookup (HashMap s)
            (tag (counter s) (concat (PState s) root)) = Some challenge

```

```

using spec[OF Cons.prem1, of 0] challenges-eq by simp
have head-nf:
  None  $\notin$  dom (dist (execute receive-commits
    (s(PTranscript := root # (roots @ rest))))))
by (rule step-nf) simp
have case-eq:
  ?case =
    (None  $\notin$  dom (dist (execute
      (receive-commits  $\gg$ 
        ( $\lambda$ br. ntimes receive-commits (length roots)  $\gg$ 
          ( $\lambda$ brs. return (br # brs))))
      (s(PTranscript := root # (roots @ rest))))))
by simp
show ?case
unfolding case-eq
proof (rule no-failure-bindI[OF head-nf])
fix br u
assume head:
  Some (br, u)  $\in$ 
    set-dist (execute receive-commits
      (s(PTranscript := root # roots @ rest)))
obtain b r where br-eq: br = (b, r)
by (cases br)
have head-out:
  r = root  $\wedge$ 
  b = challenge  $\wedge$ 
  PState u = concat (PState s) root  $\wedge$ 
  PTranscript u = roots @ rest  $\wedge$ 
  counter u = Suc (counter s)  $\wedge$ 
  s  $\leq$  u
using step[OF lookup-root head[unfolded br-eq]]
unfolding br-eq by simp
have tail-lookups:
   $\forall i < \text{length roots}.$ 
    fmllookup (HashMap u)
      (tag (counter u + i)
        (foldl concat (PState u) (take (Suc i) roots))) =
      Some (challenges' ! i)
proof (intro allI impI)
fix i
assume i-bound: i < length roots
have old-lookup:
  fmllookup (HashMap s)
    (tag (counter s + Suc i)
      (foldl concat (PState s) (take (Suc (Suc i)) (root # roots)))) =
    Some ((challenge # challenges') ! Suc i)
using spec[OF Cons.prem1, of Suc i] i-bound challenges-eq by simp
have key-eq:
  foldl concat (PState s) (take (Suc (Suc i)) (root # roots)) =

```

```

    foldl concat (PState u) (take (Suc i) roots)
  using head-out by simp
show fmlookup (HashMap u)
  (tag (counter u + i)
    (foldl concat (PState u) (take (Suc i) roots))) =
  Some (challenges' ! i)
  using p.hash-extension-lookup[OF old-lookup, of u] head-out key-eq i-bound
  by simp
qed
have tail-len: length challenges' = length roots
  using Cons.premis(2) challenges-eq by simp
have tail-nf-start:
  None  $\notin$  dom (dist (execute
    (ntimes receive-commits (length roots))
    (u(PTranscript := roots @ rest))))
  using Cons.IH[OF tail-lookups tail-len, of rest] .
have u-update: u(PTranscript := roots @ rest) = u
  using head-out by simp
have tail-nf:
  None  $\notin$  dom (dist (execute
    (ntimes receive-commits (length roots)) u)
  using tail-nf-start unfolding u-update .
show None  $\notin$  dom (dist (execute
  (ntimes receive-commits (length roots))  $\gg$ 
  ( $\lambda$ brs. return (br # brs))) u)
  by (rule no-failure-bindI[OF tail-nf]) simp
qed
qed

lemma honest-receive-trace-fri-commits-list-outcome:
  assumes lookups:
     $\forall i < \text{length roots.}$ 
    fmlookup (HashMap s)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (PState s) (take (Suc i) roots))) =
      Some (challenges ! i)
  and len: length challenges = length roots
  and outcome:
    Some (brs, t)  $\in$ 
    set-dist (execute (ntimes v.receive-trace-fri-commits (length roots))
      (s(PTranscript := roots @ rest)))
  shows
    brs = zip challenges roots  $\wedge$ 
    PState t = foldl concat (PState s) roots  $\wedge$ 
    PTranscript t = rest  $\wedge$ 
    s  $\leq$  t
proof -
  have counter-update:
     $\bigwedge s \text{ tr. } P\text{TraceFriCounter } (s(P\text{Transcript} := \text{tr})) = P\text{TraceFriCounter } s$ 

```



```

    by simp
  have res:
    brs = zip challenges roots ∧
    PState t = foldl concat (PState s) roots ∧
    PTranscript t = rest ∧
    PTraceFriCounter t = PTraceFriCounter s + length roots ∧
    s ≤ t
  by (rule honest-receive-fri-commits-list-outcome-with
    [where counter=PTraceFriCounter and tag=TraceFriChallenge,
      OF honest-receive-trace-fri-commit-replay-outcome counter-update lookups len outcome])
  then show ?thesis by simp
qed

```

lemma honest-receive-composition-fri-commits-list-outcome:

assumes lookups:

$\forall i < \text{length roots}.$

fmlookup (HashMap s)
 (CompositionFriChallenge
 (PCompositionFriCounter s + i)
 (foldl concat (PState s) (take (Suc i) roots))) =
 Some (challenges ! i)

and len: length challenges = length roots

and outcome:

Some (brs, t) ∈
 set-dist (execute
 (ntimes v.receive-composition-fri-commits (length roots))
 (s(PTranscript := roots @ rest)))

shows

brs = zip challenges roots ∧
 PState t = foldl concat (PState s) roots ∧
 PTranscript t = rest ∧
 s ≤ t

proof –

have counter-update:

$\bigwedge s \text{ tr}. PCompositionFriCounter$
 (s(PTranscript := tr)) = PCompositionFriCounter s

by simp

have res:

brs = zip challenges roots ∧
 PState t = foldl concat (PState s) roots ∧
 PTranscript t = rest ∧
 PCompositionFriCounter t = PCompositionFriCounter s + length roots ∧
 s ≤ t

by (rule honest-receive-fri-commits-list-outcome-with

[where counter=PCompositionFriCounter and tag=CompositionFriChallenge,
 OF honest-receive-composition-fri-commit-replay-outcome
 counter-update lookups len outcome])

then show ?thesis **by** simp

qed

lemma *honest-receive-trace-fri-commits-list-no-failure*:

assumes *lookups*:

$\forall i < \text{length roots}.$

$\text{fmlookup } (\text{HashMap } s)$

$(\text{TraceFriChallenge } (P\text{TraceFriCounter } s + i)$

$(\text{foldl concat } (P\text{State } s) (\text{take } (\text{Suc } i) \text{ roots}))) =$

$\text{Some } (\text{challenges } ! i)$

and *len*: $\text{length challenges} = \text{length roots}$

shows

$\text{None} \notin \text{dom } (\text{dist } (\text{execute}$

$(\text{ntimes } v.\text{receive-trace-fri-commits } (\text{length roots}))$

$(s(P\text{Transcript} := \text{roots } @ \text{rest}))))$

using *lookups len*

proof (*induction roots arbitrary: challenges s rest*)

case *Nil*

then show *?case* **by** *simp*

next

case (*Cons root roots*)

from *Cons.prem1(2)* **obtain** *challenge challenges'* **where** *challenges-eq*:

$\text{challenges} = \text{challenge } \# \text{ challenges'}$

by (*cases challenges*) *auto*

have *lookup-root*:

$\text{fmlookup } (\text{HashMap } s)$

$(\text{TraceFriChallenge } (P\text{TraceFriCounter } s) (\text{concat } (P\text{State } s) \text{ root})) =$

$\text{Some } \text{challenge}$

using *spec[OF Cons.prem1(1), of 0]* *challenges-eq* **by** *simp*

have *head-nf*:

$\text{None} \notin \text{dom } (\text{dist } (\text{execute } v.\text{receive-trace-fri-commits}$

$(s(P\text{Transcript} := \text{root } \# (\text{roots } @ \text{rest}))))$

by (*rule v.receive-trace-fri-commits-no-failure*) *simp*

have *case-eq*:

?case =

$(\text{None} \notin \text{dom } (\text{dist } (\text{execute}$

$(v.\text{receive-trace-fri-commits } \gg=$

$(\lambda br. \text{ntimes } v.\text{receive-trace-fri-commits } (\text{length roots}) \gg=$

$(\lambda brs. \text{return } (br \# brs))))$

$(s(P\text{Transcript} := \text{root } \# (\text{roots } @ \text{rest}))))$

by *simp*

show *?case*

unfolding *case-eq*

proof (*rule no-failure-bindI[OF head-nf]*)

fix *br u*

assume *head*:

$\text{Some } (br, u) \in$

$\text{set-dist } (\text{execute } v.\text{receive-trace-fri-commits}$

$(s(P\text{Transcript} := \text{root } \# \text{roots } @ \text{rest})))$

obtain *b r* **where** *br-eq*: $br = (b, r)$

by (*cases br*)

```

have head-out:
  r = root ∧
  b = challenge ∧
  PState u = concat (PState s) root ∧
  PTranscript u = roots @ rest ∧
  PTraceFriCounter u = Suc (PTraceFriCounter s) ∧
  s ≤ u
using honest-receive-trace-fri-commit-replay-outcome[
  OF lookup-root head[unfolded br-eq]]
unfolding br-eq by simp
have tail-lookups:
  ∀ i < length roots.
    fmllookup (HashMap u)
      (TraceFriChallenge
        (PTraceFriCounter u + i)
        (foldl concat (PState u) (take (Suc i) roots))) =
      Some (challenges' ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have old-lookup:
    fmllookup (HashMap s)
      (TraceFriChallenge
        (PTraceFriCounter s + Suc i)
        (foldl concat (PState s) (take (Suc (Suc i)) (root # roots)))) =
    Some ((challenge # challenges') ! Suc i)
  using spec[OF Cons.prem1(1), of Suc i] i-bound challenges-eq
  by simp
  have key-eq:
    foldl concat (PState s) (take (Suc (Suc i)) (root # roots)) =
    foldl concat (PState u) (take (Suc i) roots)
  using head-out by simp
  show fmllookup (HashMap u)
    (TraceFriChallenge
      (PTraceFriCounter u + i)
      (foldl concat (PState u) (take (Suc i) roots))) =
    Some (challenges' ! i)
  using p.hash-extension-lookup[OF old-lookup, of u] head-out key-eq
  i-bound
  by simp
qed
have tail-len: length challenges' = length roots
  using Cons.prem2(2) challenges-eq by simp
have tail-nf-start:
  None ∉ dom (dist (execute
    (ntimes v.receive-trace-fri-commits (length roots))
    (u(PTranscript := roots @ rest))))
  using Cons.IH[OF tail-lookups tail-len, of rest] .
have u-update: u(PTranscript := roots @ rest) = u

```

```

    using head-out by simp
  have tail-nf:
    None  $\notin$  dom (dist (execute
      (ntimes v.receive-trace-fri-commits (length roots)) u))
    using tail-nf-start unfolding u-update .
  show None  $\notin$  dom (dist (execute
    (ntimes v.receive-trace-fri-commits (length roots)  $\gg$ 
      ( $\lambda$ brs. return (br # brs))) u))
    by (rule no-failure-bindI[OF tail-nf]) simp
qed
qed

lemma honest-receive-composition-fri-commits-list-no-failure:
  assumes lookups:
     $\forall i < \text{length roots.}$ 
    fmlookup (HashMap s)
      (CompositionFriChallenge
        (PCompositionFriCounter s + i)
        (foldl concat (PState s) (take (Suc i) roots))) =
      Some (challenges ! i)
  and len: length challenges = length roots
  shows
    None  $\notin$  dom (dist (execute
      (ntimes v.receive-composition-fri-commits (length roots))
      (s(PTranscript := roots @ rest))))
    using lookups len
  proof (induction roots arbitrary: challenges s rest)
    case Nil
    then show ?case by simp
  next
    case (Cons root roots)
    from Cons.prem1(2) obtain challenge challenges' where challenges-eq:
      challenges = challenge # challenges'
    by (cases challenges) auto
  have lookup-root:
    fmlookup (HashMap s)
      (CompositionFriChallenge (PCompositionFriCounter s)
        (concat (PState s) root)) =
      Some challenge
    using spec[OF Cons.prem1(1), of 0] challenges-eq by simp
  have head-nf:
    None  $\notin$  dom (dist (execute v.receive-composition-fri-commits
      (s(PTranscript := root # (roots @ rest))))))
    by (rule v.receive-composition-fri-commits-no-failure) simp
  have case-eq:
    ?case =
      (None  $\notin$  dom (dist (execute
        (v.receive-composition-fri-commits  $\gg$ 
          ( $\lambda$ br. ntimes v.receive-composition-fri-commits (length roots)  $\gg$ 

```

```

      (λbrs. return (br # brs))))
    (s(PTranscript := root # (roots @ rest))))))
  by simp
show ?case
  unfolding case-eq
proof (rule no-failure-bindI[OF head-nf])
  fix br u
  assume head:
    Some (br, u) ∈
      set-dist (execute v.receive-composition-fri-commits
        (s(PTranscript := root # roots @ rest)))
  obtain b r where br-eq: br = (b, r)
  by (cases br)
  have head-out:
    r = root ∧
    b = challenge ∧
    PState u = concat (PState s) root ∧
    PTranscript u = roots @ rest ∧
    PCompositionFriCounter u = Suc (PCompositionFriCounter s) ∧
    s ≤ u
  using honest-receive-composition-fri-commit-replay-outcome[
    OF lookup-root head[unfolded br-eq]]
  unfolding br-eq by simp
  have tail-lookups:
    ∀ i < length roots.
      fmlookup (HashMap u)
        (CompositionFriChallenge
          (PCompositionFriCounter u + i)
          (foldl concat (PState u) (take (Suc i) roots))) =
        Some (challenges' ! i)
  proof (intro allI impI)
    fix i
    assume i-bound: i < length roots
    have old-lookup:
      fmlookup (HashMap s)
        (CompositionFriChallenge
          (PCompositionFriCounter s + Suc i)
          (foldl concat (PState s) (take (Suc (Suc i)) (root # roots)))) =
        Some ((challenge # challenges') ! Suc i)
    using spec[OF Cons.prem1, of Suc i] i-bound challenges-eq
    by simp
    have key-eq:
      foldl concat (PState s) (take (Suc (Suc i)) (root # roots)) =
      foldl concat (PState u) (take (Suc i) roots)
    using head-out by simp
    show fmlookup (HashMap u)
      (CompositionFriChallenge
        (PCompositionFriCounter u + i)
        (foldl concat (PState u) (take (Suc i) roots))) =

```

```

    Some (challenges' ! i)
  using p.hash-extension-lookup[OF old-lookup, of u] head-out key-eq
    i-bound
  by simp
qed
have tail-len: length challenges' = length roots
  using Cons.premis(2) challenges-eq by simp
have tail-nf-start:
  None  $\notin$  dom (dist (execute
    (ntimes v.receive-composition-fri-commits (length roots))
    (u(PTranscript := roots @ rest))))
  using Cons.IH[OF tail-lookups tail-len, of rest] .
have u-update: u(PTranscript := roots @ rest) = u
  using head-out by simp
have tail-nf:
  None  $\notin$  dom (dist (execute
    (ntimes v.receive-composition-fri-commits (length roots)) u))
  using tail-nf-start unfolding u-update .
show None  $\notin$  dom (dist (execute
  (ntimes v.receive-composition-fri-commits (length roots)  $\gg$ 
    ( $\lambda$ brs. return (br # brs)))) u))
  by (rule no-failure-bindI[OF tail-nf]) simp
qed
qed

```

lemma honest-fri-commit-lengths:

```

  assumes Some ((ps', ds', ls', ms'), t)  $\in$ 
    set-dist (execute (p.fri-commit n ps ds ls ms) s)
  shows
    length ps' = length ps + n  $\wedge$ 
    length ds' = length ds + n  $\wedge$ 
    length ls' = length ls + n  $\wedge$ 
    length ms' = length ms + n
  using p.fri-commit-lengths[OF assms] .

```

lemma honest-fri-commit-extends:

```

  assumes Some ((ps', ds', ls', ms'), t)  $\in$ 
    set-dist (execute (p.fri-commit n ps ds ls ms) s)
  shows s  $\leq$  t
  using p.fri-commit-extends[OF assms] .

```

lemma honest-fri-commit-layers-are-evaluations:

```

  assumes init:
     $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$ 
    ls ! i = map (poly (ps ! i)) (ds ! i)
  and outcome:
    Some ((ps', ds', ls', ms'), t)  $\in$ 
    set-dist (execute (p.fri-commit n ps ds ls ms) s)
  and lens: length ps = length ds length ds = length ls

```

and *bounds*: $i < \text{length } ps' \ i < \text{length } ds' \ i < \text{length } ls'$
shows $ls' ! i = \text{map } (\text{poly } (ps' ! i)) \ (ds' ! i)$
using $p.\text{fri-commit-layers-are-evaluations}[OF \ \text{init} \ \text{outcome} \ \text{lens} \ \text{bounds}]$.

lemma *honest-fri-commit-created-trees*:

assumes *init*:

$\bigwedge i. i < \text{length } ls \implies i < \text{length } ms \implies p.\text{created-tree } (ls ! i) \ (ms ! i) \ s$

and *outcome*:

$\text{Some } ((ps', ds', ls', ms'), t) \in$
 $\text{set-dist } (\text{execute } (p.\text{fri-commit } n \ ps \ ds \ ls \ ms) \ s)$

and *lens*: $\text{length } ls = \text{length } ms$

and *bounds*: $i < \text{length } ls' \ i < \text{length } ms'$

shows $p.\text{created-tree } (ls' ! i) \ (ms' ! i) \ t$

using $p.\text{fri-commit-created-trees}[OF \ \text{init} \ \text{outcome} \ \text{lens} \ \text{bounds}]$.

lemma *honest-prover-monad-outcome*:

assumes *outcome*:

$\text{Some } (final, t) \in \text{set-dist } (\text{execute } p.\text{prover-monad } \text{init-state})$

obtains $f\text{-merkle } s1 \ s2 \ f\text{-nrounds } f\text{-ps } f\text{-ds } f\text{-ls } f\text{-ms } s\text{-trace-fri}$

$s\text{-trace-final } as \ s3 \ cp' \ s4 \ cp\text{-merkle } s5 \ n\text{rounds } ps \ ds \ ls \ ms \ s6 \ s\text{-final}$

where

$\text{Some } (f\text{-merkle}, s1) \in$

$\text{set-dist } (\text{execute } (p.\text{create } p.f\text{-eval}) \ \text{init-state})$

$p.\text{created-tree } p.f\text{-eval } f\text{-merkle } s1$

$\text{Some } ((), s2) \in \text{set-dist } (\text{execute } (p.\text{send } (\text{value } f\text{-merkle})) \ s1)$

$f\text{-nrounds} = \text{ceil-log } cl\text{length}$

$\text{Some } ((f\text{-ps}, f\text{-ds}, f\text{-ls}, f\text{-ms}), s\text{-trace-fri}) \in$

$\text{set-dist } (\text{execute}$

$(p.\text{trace-fri-commit } f\text{-nrounds } [p.f] \ [p.\text{eval-domain}] \ [p.f\text{-eval}]$
 $[f\text{-merkle}]) \ s2)$

$\text{length } f\text{-ps} = \text{Suc } f\text{-nrounds}$

$\text{length } f\text{-ds} = \text{Suc } f\text{-nrounds}$

$\text{length } f\text{-ls} = \text{Suc } f\text{-nrounds}$

$\text{length } f\text{-ms} = \text{Suc } f\text{-nrounds}$

$\bigwedge i. i < \text{length } f\text{-ps} \implies i < \text{length } f\text{-ds} \implies i < \text{length } f\text{-ls} \implies$

$f\text{-ls} ! i = \text{map } (\text{poly } (f\text{-ps} ! i)) \ (f\text{-ds} ! i)$

$\bigwedge i. i < \text{length } f\text{-ls} \implies i < \text{length } f\text{-ms} \implies$

$p.\text{created-tree } (f\text{-ls} ! i) \ (f\text{-ms} ! i) \ s\text{-trace-fri}$

$\text{Some } ((), s\text{-trace-final}) \in \text{set-dist } (\text{execute } (p.\text{send } (\text{hd } (\text{last } f\text{-ls}))) \ s\text{-trace-fri})$

$\text{Some } (as, s3) \in$

$\text{set-dist } (\text{execute}$

$(\text{mmap } (\text{replicate } (\text{length } \text{spec})$

$(\text{do } \{$

$a \leftarrow p.\text{receive-alpha-challenge};$

$p.\text{send } a;$

$\text{return } a$

$\}))) \ s\text{-trace-final})$

$cp' = p.cp \ as \ p.f\text{-powers}$

$\text{Some } ((), s4) \in \text{set-dist } (\text{execute } (p.\text{send } (\text{of-nat } (\text{degree } cp'))) \ s3)$

Some (*cp-merkle*, *s5*) ∈ *set-dist* (*execute* (*p.create* (*p.cp-eval as*)) *s4*)
p.created-tree (*p.cp-eval as*) *cp-merkle s5*
nrounds = *ceil-log* (*degree cp' + 1*)
Some ((*ps*, *ds*, *ls*, *ms*), *s6*) ∈
set-dist (*execute*
(*p.composition-fri-commit nrounds [cp'] [p.eval-domain]*
[*p.cp-eval as*] [*cp-merkle*]) *s5*)
length ps = *Suc nrounds*
length ds = *Suc nrounds*
length ls = *Suc nrounds*
length ms = *Suc nrounds*
 $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$
 $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$
 $\bigwedge i. i < \text{length } ls \implies i < \text{length } ms \implies$
p.created-tree (*ls ! i*) (*ms ! i*) *s6*
Some (*()*, *s-final*) ∈ *set-dist* (*execute* (*p.send* (*hd* (*last ls*)))) *s6*)
Some (*final*, *t*) ∈
set-dist (*execute*
(*ntimes* (*prover-query-round f-merkle f-ls f-ms ls ms*) *rounds*) *s-final*)
proof –
from *outcome* **obtain** *f-merkle s1 s2 f-ps f-ds f-ls f-ms s-trace-fri*
s-trace-final as s3 s4 cp-merkle s5 ps ds ls ms s6 s-final **where**
create-f:
Some (*f-merkle*, *s1*) ∈
set-dist (*execute* (*p.create p.f-eval*) *init-state*)
and *send-f*:
Some (*()*, *s2*) ∈ *set-dist* (*execute* (*p.send* (*value f-merkle*)) *s1*)
and *trace-fri*:
Some ((*f-ps*, *f-ds*, *f-ls*, *f-ms*), *s-trace-fri*) ∈
set-dist (*execute*
(*p.trace-fri-commit* (*ceil-log clength*)
[*p.f*] [*p.eval-domain*] [*p.f-eval*] [*f-merkle*] *s2*)
and *trace-final-send*:
Some (*()*, *s-trace-final*) ∈
set-dist (*execute* (*p.send* (*hd* (*last f-ls*)))) *s-trace-fri*)
and *alphas*:
Some (*as*, *s3*) ∈
set-dist (*execute*
(*mmap* (*replicate* (*length spec*)
(*do* {
a ← *p.receive-alpha-challenge*;
p.send a;
return a
}))) *s-trace-final*)
and *send-degree*:
Some (*()*, *s4*) ∈
set-dist (*execute* (*p.send* (*of-nat* (*degree* (*p.cp as p.f-powers*)))) *s3*)
and *create-cp*:
Some (*cp-merkle*, *s5*) ∈


```

    set-dist (execute (p.create (p.cp-eval as)) s4)
and fri:
  Some ((ps, ds, ls, ms), s6) ∈
    set-dist (execute
      (p.composition-fri-commit
        (ceil-log (degree (p.cp as p.f-powers) + 1))
        [p.cp as p.f-powers] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
and final-send:
  Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
and round-tail:
  Some (final, t) ∈
    set-dist (execute
      (ntimes (prover-query-round f-merkle f-ls f-ms ls ms) rounds) s-final)
unfolding p.prover-monad-def
by (auto simp: Let-def init-state-def prover-query-round-def
    split: prod.splits elim!: p.set-dist-bindE)
let ?cp' = p.cp as p.f-powers
let ?nrounds = ceil-log (degree ?cp' + 1)
have created-f: p.created-tree p.f-eval f-merkle s1
  using p.create-outcome[OF create-f] by simp
let ?f-nrounds = ceil-log clength
have trace-fri-lengths:
  length f-ps = Suc ?f-nrounds
  length f-ds = Suc ?f-nrounds
  length f-ls = Suc ?f-nrounds
  length f-ms = Suc ?f-nrounds
  using p.trace-fri-commit-lengths[OF trace-fri] by simp-all
have trace-fri-layers:
  ∧i. i < length f-ps ⇒ i < length f-ds ⇒ i < length f-ls ⇒
    f-ls ! i = map (poly (f-ps ! i)) (f-ds ! i)
proof (rule p.trace-fri-commit-layers-are-evaluations[OF - trace-fri])
  fix i
  assume i < length [p.f] i < length [p.eval-domain] i < length [p.f-eval]
  then show [p.f-eval] ! i =
    map (poly ([p.f] ! i)) ([p.eval-domain] ! i)
  unfolding p.f-eval-def by simp
qed simp-all
have trace-fri-created:
  ∧i. i < length f-ls ⇒ i < length f-ms ⇒
    p.created-tree (f-ls ! i) (f-ms ! i) s-trace-fri
proof (rule p.trace-fri-commit-created-trees[OF - trace-fri])
  fix i
  assume i < length [p.f-eval] i < length [f-merkle]
  then show p.created-tree ([p.f-eval] ! i) ([f-merkle] ! i) s2
proof -
  have s1 ≤ s2
  using p.send-outcome[OF send-f]
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
then have p.created-tree p.f-eval f-merkle s2

```

```

    using p.created-tree-mono[OF created-f] by blast
  then show ?thesis
    using ⟨i < length [p.f-eval]⟩ ⟨i < length [f-merkle]⟩ by simp
qed
qed simp-all
have created-cp: p.created-tree (p.cp-eval as) cp-merkle s5
  using p.create-outcome[OF create-cp] by simp
have fri-lengths:
  length ps = Suc ?nrounds
  length ds = Suc ?nrounds
  length ls = Suc ?nrounds
  length ms = Suc ?nrounds
  using p.composition-fri-commit-lengths[OF fri] by simp-all
have fri-layers:
  ∧i. i < length ps ⇒ i < length ds ⇒ i < length ls ⇒
    ls ! i = map (poly (ps ! i)) (ds ! i)
proof (rule p.composition-fri-commit-layers-are-evaluations[OF - fri])
  fix i
  assume i < length [?cp] i < length [p.eval-domain] i < length [p.cp-eval as]
  then show [p.cp-eval as] ! i =
    map (poly ([?cp] ! i)) ([p.eval-domain] ! i)
    unfolding p.cp-eval-def by simp
qed simp-all
have fri-created:
  ∧i. i < length ls ⇒ i < length ms ⇒
    p.created-tree (ls ! i) (ms ! i) s6
proof (rule p.composition-fri-commit-created-trees[OF - fri])
  fix i
  assume i < length [p.cp-eval as] i < length [cp-merkle]
  then show p.created-tree ([p.cp-eval as] ! i) ([cp-merkle] ! i) s5
    using created-cp by simp
qed simp-all
show ?thesis
  by (rule that[
    OF create-f created-f send-f refl trace-fri trace-fri-lengths(1)
    trace-fri-lengths(2) trace-fri-lengths(3) trace-fri-lengths(4)
    trace-fri-layers trace-fri-created trace-final-send alphas refl send-degree create-cp created-cp
    refl fri fri-lengths(1) fri-lengths(2) fri-lengths(3) fri-lengths(4)
    fri-layers fri-created final-send round-tail])
qed

```

lemma *honest-verifier-monad-outcome*:

assumes *outcome*:

Some (result, t) ∈ set-dist (execute v.verify-monad s)

obtains *fr s1 f-fl s-fri f-final s-fri-final as s2 dg s3 s4 fl s5 final s6*

where

Some (fr, s1) ∈ set-dist (execute p.read s)

Some (f-fl, s-fri) ∈

set-dist (execute

```

    (ntimes v.receive-trace-fri-commits (ceil-log clength)) s1)
Some (f-final, s-fri-final) ∈ set-dist (execute p.read s-fri)
Some (as, s2) ∈
  set-dist (execute
    (mmap (replicate (length spec)
      (do {
        a0 ← p.receive-alpha-challenge;
        let a0' = a0;
        a1 ← p.read;
        let a1' = a1;
        assert (a0' = a1');
        return a1'
      }))) s-fri-final)
Some (dg, s3) ∈ set-dist (execute p.read s2)
Some ((), s4) ∈
  set-dist (execute (assert (to-nat dg ≤ p.maxDegree)) s3)
to-nat dg ≤ p.maxDegree
s4 = s3
Some (fl, s5) ∈
  set-dist (execute
    (ntimes v.receive-composition-fri-commits
      (ceil-log (to-nat dg + 1))) s4)
Some (final, s6) ∈ set-dist (execute p.read s5)
Some (result, t) ∈
  set-dist (execute
    (ntimes (verifier-query-round fr f-fl f-final as fl final) rounds) s6)
proof –
from outcome obtain fr s1 f-fl s-fri f-final s-fri-final as s2 dg s3 s4 fl s5 final s6 where
  read-fr: Some (fr, s1) ∈ set-dist (execute p.read s)
and trace-fri-reads:
  Some (f-fl, s-fri) ∈
    set-dist (execute
      (ntimes v.receive-trace-fri-commits (ceil-log clength)) s1)
and read-trace-final:
  Some (f-final, s-fri-final) ∈ set-dist (execute p.read s-fri)
and alphas:
  Some (as, s2) ∈
    set-dist (execute
      (mmap (replicate (length spec)
        (do {
          a0 ← p.receive-alpha-challenge;
          let a0' = a0;
          a1 ← p.read;
          let a1' = a1;
          assert (a0' = a1');
          return a1'
        }))) s-fri-final)
and read-dg: Some (dg, s3) ∈ set-dist (execute p.read s2)
and degree-assert:

```

```

    Some ( $\langle \rangle$ ,  $s_4$ )  $\in$ 
      set-dist (execute (assert ( $\text{to-nat } dg \leq p.\text{maxDegree}$ ))  $s_3$ )
and fri-reads:
    Some ( $fl$ ,  $s_5$ )  $\in$ 
      set-dist (execute
        (ntimes  $v.\text{receive-composition-fri-commits}$ 
          ( $\text{ceil-log } (\text{to-nat } dg + 1)$ ))  $s_4$ )
and read-final: Some ( $final$ ,  $s_6$ )  $\in$  set-dist (execute  $p.\text{read } s_5$ )
and round-tail:
    Some ( $result$ ,  $t$ )  $\in$ 
      set-dist (execute
        (ntimes ( $\text{verifier-query-round } fr\ f\text{-}fl\ f\text{-}final\ \text{as } fl\ final$ ) rounds)  $s_6$ )
unfolding  $v.\text{verify-monad-def}$ 
by (auto simp: Let-def verifier-query-round-def
      split: prod.splits elim!:  $p.\text{set-dist-bindE}$ )
have degree-ok:  $\text{to-nat } dg \leq p.\text{maxDegree}$ 
using degree-assert
by (cases  $\text{to-nat } dg \leq p.\text{maxDegree}$ ) (auto simp: assert-def)
have  $s_4\text{-eq}$ :  $s_4 = s_3$ 
using degree-assert
by (cases  $\text{to-nat } dg \leq p.\text{maxDegree}$ ) (auto simp: assert-def)
show ?thesis
by (rule that[
  OF read-fr trace-fri-reads read-trace-final alphas read-dg degree-assert degree-ok  $s_4\text{-eq}$ 
  fri-reads
  read-final round-tail])
qed

```

```

lemma exec-success-splits:
assumes outcome:
  Some ( $result$ ,  $t$ )  $\in$  set-dist (execute exec init-state)
obtains prover-result prover-state replay-state
where
  Some ( $prover\text{-}result$ ,  $prover\text{-}state$ )  $\in$ 
    set-dist (execute  $p.\text{prover-monad init-state}$ )
  Some ( $\langle \rangle$ ,  $replay\text{-}state$ )  $\in$ 
    set-dist (execute
      (modify ( $\lambda c.$   $c[PState := 0, PTranscript := \text{rev } (PTranscript\ c),$ 
         $PTraceFriCounter := 0, PCompositionFriCounter := 0,$ 
         $PAlphaCounter := 0, PQueryCounter := 0]$ ))
      prover-state)
  replay-state = verifier-replay-state prover-state
  Some ( $result$ ,  $t$ )  $\in$  set-dist (execute  $v.\text{verify-monad replay-state}$ )
proof –
from outcome obtain prover-result prover-state replay-unit replay-state where
  prover:
    Some ( $prover\text{-}result$ ,  $prover\text{-}state$ )  $\in$ 
      set-dist (execute  $p.\text{prover-monad init-state}$ )
and reset:

```

```

Some (replay-unit, replay-state) ∈
  set-dist (execute
    (modify (λc. c(PState := 0, PTranscript := rev (PTranscript c),
      PTraceFriCounter := 0, PCompositionFriCounter := 0,
      PAlphaCounter := 0, PQueryCounter := 0)))
    prover-state)
and verifier:
  Some (result, t) ∈ set-dist (execute v.verify-monad replay-state)
unfolding exec-def
by (auto elim!: p.set-dist-bindE)
have reset-res:
  replay-unit = () ∧ replay-state = verifier-replay-state prover-state
using modify-verifier-replay-state-outcome[OF reset] .
show ?thesis
by (rule that[OF prover - - verifier])
  (use reset reset-res in simp-all)
qed

```

lemma exec-success-prover-outcome:

assumes outcome:

Some (result, t) ∈ set-dist (execute exec init-state)

obtains prover-result prover-state replay-state

f-merkle s1 s2 f-nrounds f-ps f-ds f-ls f-ms s-trace-fri

s-trace-final as s3 cp' s4 cp-merkle s5 nrounds ps ds ls ms s6 s-final

where

Some (prover-result, prover-state) ∈

set-dist (execute p.prover-monad init-state)

replay-state = verifier-replay-state prover-state

Some (f-merkle, s1) ∈

set-dist (execute (p.create p.f-eval) init-state)

p.created-tree p.f-eval f-merkle s1

Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)

f-nrounds = ceil-log clength

Some ((f-ps, f-ds, f-ls, f-ms), s-trace-fri) ∈

set-dist (execute

(p.trace-fri-commit f-nrounds [p.f] [p.eval-domain] [p.f-eval]
[f-merkle]) s2)

length f-ps = Suc f-nrounds

length f-ds = Suc f-nrounds

length f-ls = Suc f-nrounds

length f-ms = Suc f-nrounds

∧ i. i < length f-ps ⇒ i < length f-ds ⇒ i < length f-ls ⇒

f-ls ! i = map (poly (f-ps ! i)) (f-ds ! i)

∧ i. i < length f-ls ⇒ i < length f-ms ⇒

p.created-tree (f-ls ! i) (f-ms ! i) s-trace-fri

Some ((), s-trace-final) ∈ set-dist (execute (p.send (hd (last f-ls))) s-trace-fri)

Some (as, s3) ∈

set-dist (execute

(mmap (replicate (length spec)

```

    (do {
      a ← p.receive-alpha-challenge;
      p.send a;
      return a
    }))) s-trace-final)
cp' = p.cp as p.f-powers
Some ((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp')))) s3)
Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
p.created-tree (p.cp-eval as) cp-merkle s5
nrounds = ceil-log (degree cp' + 1)
Some ((ps, ds, ls, ms), s6) ∈
  set-dist (execute
    (p.composition-fri-commit nrounds [cp'] [p.eval-domain]
      [p.cp-eval as] [cp-merkle]) s5)
length ps = Suc nrounds
length ds = Suc nrounds
length ls = Suc nrounds
length ms = Suc nrounds
∧ i. i < length ps ⇒ i < length ds ⇒ i < length ls ⇒
  ls ! i = map (poly (ps ! i)) (ds ! i)
∧ i. i < length ls ⇒ i < length ms ⇒
  p.created-tree (ls ! i) (ms ! i) s6
Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
Some (prover-result, prover-state) ∈
  set-dist (execute
    (ntimes (prover-query-round f-merkle f-ls f-ms ls ms) rounds) s-final)
proof –
from exec-success-splits[OF outcome] obtain prover-result prover-state replay-state where
  prover:
    Some (prover-result, prover-state) ∈
      set-dist (execute p.prover-monad init-state)
and replay: replay-state = verifier-replay-state prover-state
by blast
show ?thesis
proof (rule honest-prover-monad-outcome[OF prover])
fix f-merkle s1 s2 f-nrounds f-ps f-ds f-ls f-ms s-trace-fri
  s-trace-final as s3 cp' s4 cp-merkle s5 nrounds ps ds ls ms s6 s-final
assume create-f:
  Some (f-merkle, s1) ∈
    set-dist (execute (p.create p.f-eval) init-state)
and created-f: p.created-tree p.f-eval f-merkle s1
and send-f: Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and f-nrounds-def: f-nrounds = ceil-log clength
and trace-fri:
  Some ((f-ps, f-ds, f-ls, f-ms), s-trace-fri) ∈
    set-dist (execute
      (p.trace-fri-commit f-nrounds [p.f] [p.eval-domain] [p.f-eval]
        [f-merkle]) s2)
and trace-len-ps: length f-ps = Suc f-nrounds

```

and *trace-len-ds*: $\text{length } f\text{-ds} = \text{Suc } f\text{-nrounds}$
and *trace-len-ls*: $\text{length } f\text{-ls} = \text{Suc } f\text{-nrounds}$
and *trace-len-ms*: $\text{length } f\text{-ms} = \text{Suc } f\text{-nrounds}$
and *trace-layers*:
 $\bigwedge i. i < \text{length } f\text{-ps} \implies i < \text{length } f\text{-ds} \implies i < \text{length } f\text{-ls} \implies$
 $f\text{-ls} ! i = \text{map } (\text{poly } (f\text{-ps} ! i)) (f\text{-ds} ! i)$
and *trace-trees*:
 $\bigwedge i. i < \text{length } f\text{-ls} \implies i < \text{length } f\text{-ms} \implies$
 $p.\text{created-tree } (f\text{-ls} ! i) (f\text{-ms} ! i) \text{ s-trace-fri}$
and *trace-final-send*:
 $\text{Some } ((), \text{s-trace-final}) \in \text{set-dist } (\text{execute } (p.\text{send } (\text{hd } (\text{last } f\text{-ls})))) \text{ s-trace-fri}$
and *alphas*:
 $\text{Some } (as, s3) \in$
 $\text{set-dist } (\text{execute}$
 $(\text{mmap } (\text{replicate } (\text{length } \text{spec}))$
 $(\text{do } \{$
 $a \leftarrow p.\text{receive-alpha-challenge};$
 $p.\text{send } a;$
 $\text{return } a$
 $\}))) \text{ s-trace-final})$
and *cp'-def*: $cp' = p.\text{cp } as \text{ p.f-powers}$
and *send-degree*:
 $\text{Some } ((), s4) \in \text{set-dist } (\text{execute } (p.\text{send } (\text{of-nat } (\text{degree } cp')))) s3)$
and *create-cp*:
 $\text{Some } (cp\text{-merkle}, s5) \in \text{set-dist } (\text{execute } (p.\text{create } (p.\text{cp-eval } as))) s4)$
and *created-cp*: $p.\text{created-tree } (p.\text{cp-eval } as) \text{ cp-merkle } s5$
and *nrounds-def*: $\text{nrounds} = \text{ceil-log } (\text{degree } cp' + 1)$
and *fri*:
 $\text{Some } ((ps, ds, ls, ms), s6) \in$
 $\text{set-dist } (\text{execute}$
 $(p.\text{composition-fri-commit } \text{nrounds } [cp'] [p.\text{eval-domain}]$
 $[p.\text{cp-eval } as] [cp\text{-merkle}] s5)$
and *len-ps*: $\text{length } ps = \text{Suc } \text{nrounds}$
and *len-ds*: $\text{length } ds = \text{Suc } \text{nrounds}$
and *len-ls*: $\text{length } ls = \text{Suc } \text{nrounds}$
and *len-ms*: $\text{length } ms = \text{Suc } \text{nrounds}$
and *layers*:
 $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$
 $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$
and *trees*:
 $\bigwedge i. i < \text{length } ls \implies i < \text{length } ms \implies$
 $p.\text{created-tree } (ls ! i) (ms ! i) s6$
and *final-send*:
 $\text{Some } ((), \text{s-final}) \in \text{set-dist } (\text{execute } (p.\text{send } (\text{hd } (\text{last } ls)))) s6)$
and *round-tail*:
 $\text{Some } (\text{prover-result}, \text{prover-state}) \in$
 $\text{set-dist } (\text{execute}$
 $(\text{ntimes } (\text{prover-query-round } f\text{-merkle } f\text{-ls } f\text{-ms } ls \text{ ms}) \text{ rounds}) \text{ s-final})$
show *?thesis*

```

using prover replay create-f created-f send-f f-nrounds-def trace-fri
  trace-len-ps trace-len-ds trace-len-ls trace-len-ms trace-layers
  trace-trees trace-final-send alphas cp'-def send-degree
  create-cp created-cp nrounds-def fri len-ps len-ds len-ls len-ms
  layers trees final-send round-tail
by (rule that)
qed
qed

lemma exec-success-verifier-replay-outcome:
assumes outcome:
  Some (result, t) ∈ set-dist (execute exec init-state)
obtains prover-result prover-state replay-state
  fr s1 f-fl s-fri f-final s-fri-final as s2 dg s3 s4 fl s5 final s6
where
  Some (prover-result, prover-state) ∈
    set-dist (execute p.prover-monad init-state)
  replay-state = verifier-replay-state prover-state
  Some (fr, s1) ∈ set-dist (execute p.read replay-state)
  Some (f-fl, s-fri) ∈
    set-dist (execute
      (ntimes v.receive-trace-fri-commits (ceil-log clength)) s1)
  Some (f-final, s-fri-final) ∈ set-dist (execute p.read s-fri)
  Some (as, s2) ∈
    set-dist (execute
      (mmap (replicate (length spec)
        (do {
          a0 ← p.receive-alpha-challenge;
          let a0' = a0;
          a1 ← p.read;
          let a1' = a1;
          assert (a0' = a1');
          return a1'
        })))) s-fri-final)
  Some (dg, s3) ∈ set-dist (execute p.read s2)
  Some ((), s4) ∈
    set-dist (execute (assert (to-nat dg ≤ p.maxDegree)) s3)
  to-nat dg ≤ p.maxDegree
  s4 = s3
  Some (fl, s5) ∈
    set-dist (execute
      (ntimes v.receive-composition-fri-commits
        (ceil-log (to-nat dg + 1))) s4)
  Some (final, s6) ∈ set-dist (execute p.read s5)
  Some (result, t) ∈
    set-dist (execute
      (ntimes (verifier-query-round fr f-fl f-final as fl final) rounds) s6)
proof –
from exec-success-splits[OF outcome] obtain prover-result prover-state replay-state where

```



```

prover:
  Some (prover-result, prover-state) ∈
    set-dist (execute p.prover-monad init-state)
and replay: replay-state = verifier-replay-state prover-state
and verifier:
  Some (result, t) ∈ set-dist (execute v.verify-monad replay-state)
by blast
show ?thesis
proof (rule honest-verifier-monad-outcome[OF verifier])
  fix fr s1 f-fl s-fri f-final s-fri-final as s2 dg s3 s4 fl s5 final s6
  assume read-fr: Some (fr, s1) ∈ set-dist (execute p.read replay-state)
  and trace-fri-reads:
    Some (f-fl, s-fri) ∈
      set-dist (execute
        (ntimes v.receive-trace-fri-commits (ceil-log clength)) s1)
  and read-trace-final:
    Some (f-final, s-fri-final) ∈ set-dist (execute p.read s-fri)
  and alphas:
    Some (as, s2) ∈
      set-dist (execute
        (mmap (replicate (length spec)
          (do {
            a0 ← p.receive-alpha-challenge;
            let a0' = a0;
            a1 ← p.read;
            let a1' = a1;
            assert (a0' = a1');
            return a1'
          }))) s-fri-final)
  and read-dg: Some (dg, s3) ∈ set-dist (execute p.read s2)
  and degree-assert:
    Some ((), s4) ∈
      set-dist (execute (assert (to-nat dg ≤ p.maxDegree)) s3)
  and degree-ok: to-nat dg ≤ p.maxDegree
  and s4-eq: s4 = s3
  and fri-reads:
    Some (fl, s5) ∈
      set-dist (execute
        (ntimes v.receive-composition-fri-commits
          (ceil-log (to-nat dg + 1))) s4)
  and read-final: Some (final, s6) ∈ set-dist (execute p.read s5)
  and round-tail:
    Some (result, t) ∈
      set-dist (execute
        (ntimes (verifier-query-round fr f-fl f-final as fl final) rounds) s6)
show ?thesis
by (rule that[
  OF prover replay read-fr trace-fri-reads read-trace-final alphas read-dg degree-assert
  degree-ok s4-eq

```

```

      fri-reads read-final round-tail])
    qed
  qed
end
end

```

```

theory Completeness-Algebra
  imports Completeness-Core
begin

```

Deterministic polynomial and degree facts used by honest completeness.

```

context verification
begin

```

```

lemma degree-message-decodes:
  assumes degree  $q \leq p.maxDegree$ 
  shows to-nat (of-nat (degree  $q$ )) = degree  $q$ 
  using assms by (simp add: p.maxDegree-def degrees-def p.to-nat-of-nat-bounded)

```

```

lemma degree-assert-hol:
  assumes degree  $q \leq p.maxDegree$ 
  shows to-nat (of-nat (degree  $q$ ))  $\leq p.maxDegree$ 
  using assms degree-message-decodes[OF assms] by simp

```

```

lemma root-factor-pCons:
  fixes  $r :: 'f$ 
  shows  $XP\ 1 - CP\ r = pCons\ (-\ r)\ 1$ 
proof -
  have  $xp: XP\ (1::'f) = pCons\ 0\ 1$ 
    unfolding XP-def by (simp add: monom-0 monom-Suc)
  have  $cp: CP\ r = pCons\ r\ 0$ 
    unfolding CP-def by (simp add: monom-0)
  show ?thesis
    unfolding  $xp\ cp$ 
    by (simp only: diff-pCons pCons-eq-iff)
      (simp only: diff-conv-add-uminus add-0-left diff-0-right minus-zero add-0-right)
qed

```

```

lemma poly-root-factor:
  fixes  $r\ x :: 'f$ 
  shows poly (XP 1 - CP r)  $x = x - r$ 
  unfolding root-factor-pCons by (simp add: algebra-simps)

```

```

lemma root-factor-nonzero:
  fixes  $r :: 'f$ 
  shows  $XP\ 1 - CP\ r \neq 0$ 

```

unfolding *root-factor-pCons* **by** *simp*

lemma *root-factor-degree*:

fixes $r :: 'f$

shows $\text{degree } (XP\ 1 - CP\ r) = 1$

unfolding *root-factor-pCons* **by** *simp*

lemma *prod-fold-acc*:

fixes $\text{roots} :: 'f\ \text{list}$

shows $\text{fold } (\lambda s\ \text{acc}. (XP\ 1 - CP\ s) * \text{acc})\ \text{roots}\ \text{acc} =$

$\text{fold } (\lambda s\ \text{acc}. (XP\ 1 - CP\ s) * \text{acc})\ \text{roots}\ (1 :: 'f\ \text{poly}) * \text{acc}$

proof (*induction* roots *arbitrary*: acc)

case *Nil*

then show *?case* **by** *simp*

next

case ($\text{Cons}\ r\ \text{roots}$)

let $?F = \lambda s\ \text{acc}. (XP\ 1 - CP\ s) * \text{acc}$

let $?fac = XP\ 1 - CP\ r$

have *lhs*:

$\text{fold } ?F\ (r \# \text{roots})\ \text{acc} =$

$\text{fold } ?F\ \text{roots}\ 1 * (?fac * \text{acc})$

using *Cons.IH*[*of* $?fac * \text{acc}$] **by** *simp*

have $\text{fold } ?F\ (r \# \text{roots})\ \text{acc} =$

$(\text{fold } ?F\ \text{roots}\ 1 * ?fac) * \text{acc}$

using *lhs* **by** (*simp only*: *mult.assoc*)

also have $\dots = \text{fold } ?F\ \text{roots}\ ?fac * \text{acc}$

using *Cons.IH*[*of* $?fac$, *symmetric*] **by** *simp*

also have $\dots = \text{fold } ?F\ (r \# \text{roots})\ 1 * \text{acc}$

by *simp*

finally show *?case* .

qed

lemma *prod-Cons*:

fixes $r :: 'f$ **and** $\text{roots} :: 'f\ \text{list}$

shows $\text{prod } (r \# \text{roots}) = (XP\ 1 - CP\ r) * (\text{prod } \text{roots} :: 'f\ \text{poly})$

proof –

let $?F = \lambda s\ \text{acc}. (XP\ 1 - CP\ s) * \text{acc}$

have $\text{prod } (r \# \text{roots}) = \text{fold } ?F\ \text{roots}\ ((XP\ 1 - CP\ r) * 1)$

unfolding *prod-def* **by** *simp*

also have $\dots = \text{fold } ?F\ \text{roots}\ 1 * ((XP\ 1 - CP\ r) * 1)$

by (*rule prod-fold-acc*)

also have $\dots = (\text{prod } \text{roots} :: 'f\ \text{poly}) * (XP\ 1 - CP\ r)$

unfolding *prod-def* **by** *simp*

also have $\dots = (XP\ 1 - CP\ r) * (\text{prod } \text{roots} :: 'f\ \text{poly})$

by (*rule mult.commute*)

finally show *?thesis* .

qed

lemma *prod-roots-nonzero*:

```

    fixes roots :: 'f list
    shows prod roots  $\neq$  (0 :: 'f poly)
  proof (induction roots)
    case Nil
    then show ?case
      unfolding prod-def by simp
  next
    case (Cons r roots)
    then show ?case
      unfolding prod-Cons
      using root-factor-nonzero[of r]
      by simp
  qed

lemma degree-prod-roots:
  fixes roots :: 'f list
  shows degree (prod roots :: 'f poly) = length roots
  proof (induction roots)
    case Nil
    then show ?case
      unfolding prod-def by simp
  next
    case (Cons r roots)
    have degree (prod (r # roots) :: 'f poly) =
      degree ((XP 1 - CP r) * prod roots :: 'f poly)
      unfolding prod-Cons by simp
    also have ... = degree (XP 1 - CP r :: 'f poly) + degree (prod roots :: 'f poly)
      by (rule degree-mult-eq)
      (use root-factor-nonzero[of r] prod-roots-nonzero[of roots] in simp-all)
    also have ... = Suc (length roots)
      using Cons.IH root-factor-degree[of r] by simp
    finally show ?case
      by simp
  qed

lemma prod-roots-dvd:
  fixes roots :: 'f list
  assumes vanish:  $\bigwedge r. r \in \text{set roots} \implies \text{poly } q \ r = 0$ 
    and distinct: distinct roots
  shows prod roots dvd q
  using assms
  proof (induction roots arbitrary: q)
    case Nil
    then show ?case
      unfolding prod-def by simp
  next
    case (Cons r roots)
    have r-root: poly q r = 0
      using Cons.prem1 by simp

```

```

have fac-dvd: (XP 1 - CP r) dvd q
proof -
  have pCons (- r) 1 dvd q
  proof -
    have [:- r, 1:] dvd q
      using r-root by (simp add: poly-eq-0-iff-dvd)
    moreover have [:- r, 1:] = pCons (- r) 1
      by simp
    ultimately show ?thesis
      by simp
  qed
then show ?thesis
  unfolding root-factor-pCons .
qed
then obtain q' where q-def: q = (XP 1 - CP r) * q'
  unfolding dvd-def by auto
have roots-distinct: distinct roots
  using Cons.premis(2) by simp
have vanish-q':  $\bigwedge x. x \in \text{set roots} \implies \text{poly } q' x = 0$ 
proof -
  fix x
  assume x-in:  $x \in \text{set roots}$ 
  have x-ne:  $x \neq r$ 
    using Cons.premis(2) x-in by auto
  have 0 = poly q x
    using Cons.premis(1) x-in by simp
  also have ... = poly (XP 1 - CP r) x * poly q' x
    unfolding q-def by simp
  finally have zero-eq:  $0 = \text{poly } (XP 1 - CP r) x * \text{poly } q' x$  .
  then have prod-zero:  $\text{poly } (XP 1 - CP r) x * \text{poly } q' x = 0$ 
    by simp
  moreover have poly (XP 1 - CP r) x  $\neq 0$ 
  proof -
    have poly (XP 1 - CP r) x = x - r
      by (rule poly-root-factor)
    then show ?thesis
      using x-ne by simp
  qed
  ultimately show poly q' x = 0
    by (metis mult-eq-0-iff)
qed
have prod-tail-dvd: prod roots dvd q'
  by (rule Cons.IH[OF vanish-q' roots-distinct])
then obtain k where q'-def:  $q' = \text{prod roots} * k$ 
  unfolding dvd-def by auto
have q = ((XP 1 - CP r) * prod roots) * k
  unfolding q-def q'-def by (simp add: mult.assoc)
also have ... = prod (r # roots) * k
  using prod-Cons[of r roots] by simp

```

```

    finally have  $q = \text{prod } (r \# \text{roots}) * k$  .
    then show ?case
      unfolding dvd-def by blast
qed

lemma prod-eval-nonzero:
  fixes roots :: 'f list
  assumes x-not-root:  $x \notin \text{set roots}$ 
  shows poly (prod roots :: 'f poly)  $x \neq 0$ 
  using x-not-root
proof (induction roots)
  case Nil
  then show ?case
    unfolding prod-def by simp
next
  case (Cons r roots)
  have  $x \neq r$ 
    using Cons.prem by simp
  moreover have poly (prod roots :: 'f poly)  $x \neq 0$ 
    using Cons by simp
  ultimately show ?case
    unfolding prod-Cons
    using poly-root-factor[of r x] by simp
qed

lemma h-power-mod-domain:
   $p.h \wedge (k \bmod (\text{clength} * \text{scale})) = p.h \wedge k$ 
  by (rule p.h-power-mod-domain)

lemma h-power-in-H:
   $p.h \wedge k \in \text{set } p.H$ 
  by (rule p.h-power-in-H)

lemma eval-coset-disjoint-from-H:
  assumes idx-bound:  $i < \text{clength} * \text{scale}$ 
  shows  $p.h \wedge i * \text{shift} \notin \text{set } p.H$ 
  by (rule p.eval-coset-disjoint-from-H[OF idx-bound])

lemma g-powers-in-H:
  assumes r-bound:  $r < \text{clength}$ 
  shows  $p.g \wedge r \in \text{set } p.H$ 
  by (rule p.g-powers-in-H[OF r-bound])

lemma g-map-subset-H:
  assumes spec-entry:  $(c, \text{roots}, d) \in \text{set spec}$ 
  shows  $\text{set } (p.g\text{-map roots}) \subseteq \text{set } p.H$ 
  by (rule p.g-map-subset-H[OF spec-entry])

lemma query-domain-disjoint:

```

```

assumes spec-entry:  $(c, \text{roots}, d) \in \text{set spec}$ 
and idx-bound:  $i < \text{clength} * \text{scale}$ 
shows  $p.h \wedge i * \text{shift} \notin \text{set } (p.g\text{-map roots})$ 
by (rule p.query-domain-disjoint[OF spec-entry idx-bound])

lemma honest-trace-algebra-prod-roots-dvd:
assumes hta: honest-trace-algebra
and spec-entry:  $(c, \text{roots}, d) \in \text{set spec}$ 
shows  $\text{prod } (p.g\text{-map roots}) \text{ dvd } c \text{ p.f-powers}$ 
proof –
have vanish:
 $\bigwedge r. r \in \text{set } (p.g\text{-map roots}) \implies \text{poly } (c \text{ p.f-powers}) r = 0$ 
using hta spec-entry
unfolding honest-trace-algebra-def constraint-roots-vanish-def
by force
have distinct:  $\text{distinct } (p.g\text{-map roots})$ 
by (rule p.g-map-distinct[OF spec-entry])
show ?thesis
by (rule prod-roots-dvd[OF vanish distinct])
qed

lemma poly-div-eval-dvd:
fixes  $q \ r :: 'f \text{ poly}$ 
assumes dvd:  $r \text{ dvd } q$ 
and denom:  $\text{poly } r \ x \neq 0$ 
shows  $\text{poly } (q \text{ div } r) \ x = \text{poly } q \ x \text{ div } \text{poly } r \ x$ 
proof –
have q-eq:  $q \text{ div } r * r = q$ 
using dvd by simp
then have  $\text{poly } (q \text{ div } r) \ x * \text{poly } r \ x = \text{poly } q \ x$ 
by (metis poly-mult)
then show ?thesis
using denom by (simp add: field-simps)
qed

lemma honest-trace-algebra-quotient-eval:
assumes hta: honest-trace-algebra
and spec-entry:  $(c, \text{roots}, d) \in \text{set spec}$ 
and idx-bound:  $\text{idx} < \text{clength} * \text{scale}$ 
shows

$$\begin{aligned} &\text{poly } ((c \text{ p.f-powers}) \text{ div } \text{prod } (p.g\text{-map roots})) (p.h \wedge \text{idx} * \text{shift}) = \\ &\text{poly } (c \text{ p.f-powers}) (p.h \wedge \text{idx} * \text{shift}) \text{ div } \\ &\text{poly } (\text{prod } (p.g\text{-map roots})) (p.h \wedge \text{idx} * \text{shift}) \end{aligned}$$

proof –
have dvd:  $\text{prod } (p.g\text{-map roots}) \text{ dvd } c \text{ p.f-powers}$ 
by (rule honest-trace-algebra-prod-roots-dvd[OF hta spec-entry])
have denom:  $\text{poly } (\text{prod } (p.g\text{-map roots})) (p.h \wedge \text{idx} * \text{shift}) \neq 0$ 
by (rule prod-eval-nonzero)
(use query-domain-disjoint[OF spec-entry idx-bound] in simp)

```

```

show ?thesis
  by (rule poly-div-eval-dvd[OF dvd denom])
qed

lemma degree-div-dvd-le:
  fixes  $q\ r :: 'f\ poly$ 
  assumes dvd:  $r\ dvd\ q$ 
  and r-nonzero:  $r \neq 0$ 
  shows  $\text{degree } (q\ \text{div } r) \leq \text{degree } q - \text{degree } r$ 
proof (cases  $q\ \text{div } r = 0$ )
  case True
    then show ?thesis by simp
  next
    case False
    have q-eq:  $q\ \text{div } r * r = q$ 
      using dvd by simp
    have  $\text{degree } q = \text{degree } (q\ \text{div } r * r)$ 
      using q-eq by simp
    also have ... =  $\text{degree } (q\ \text{div } r) + \text{degree } r$ 
      by (rule degree-mult-eq[OF False r-nonzero])
    finally show ?thesis
      by simp
  qed

lemma honest-trace-algebra-quotient-degree:
  assumes honest-trace-algebra
  and  $(c, \text{roots}, d) \in \text{set spec}$ 
  shows
     $\text{degree } ((c\ \text{p.f-powers})\ \text{div } (\text{prod } (p.g\text{-map } \text{roots}))) \leq$ 
     $d * (\text{clength} - 1) - \text{length } \text{roots}$ 
proof -
  let ?q =  $c\ \text{p.f-powers}$ 
  let ?roots =  $p.g\text{-map } \text{roots}$ 
  have vanish:  $\bigwedge r. r \in \text{set } ?\text{roots} \implies \text{poly } ?q\ r = 0$ 
    using assms
    unfolding honest-trace-algebra-def constraint-roots-vanish-def
    by force
  have deg:  $\text{degree } ?q \leq d * (\text{clength} - 1)$ 
    by (rule constraint-degree-bound[OF assms(2)])
  have distinct:  $\text{distinct } ?\text{roots}$ 
    by (rule p.g-map-distinct[OF assms(2)])
  have dvd:  $\text{prod } ?\text{roots}\ dvd\ ?q$ 
    by (rule prod-roots-dvd[OF vanish distinct])
  have qdiv-le:
     $\text{degree } (?q\ \text{div } \text{prod } ?\text{roots}) \leq \text{degree } ?q - \text{degree } (\text{prod } ?\text{roots})$ 
    by (rule degree-div-dvd-le[OF dvd prod-roots-nonzero])
  have roots-len:  $\text{length } ?\text{roots} = \text{length } \text{roots}$ 
    unfolding p.g-map-def by simp
  have prod-deg:  $\text{degree } (\text{prod } ?\text{roots}) = \text{length } \text{roots}$ 

```



```

    using degree-prod-roots[of ?roots] roots-len by simp
  have degree (?q div prod ?roots) ≤ degree ?q - length roots
    using qdiv-le prod-deg by simp
  also have ... ≤ d * (clength - 1) - length roots
    using deg by simp
  finally show ?thesis .
qed

```

```

lemma maxDegree-ge-spec:
  assumes (c, roots, d) ∈ set spec
  shows d * (clength - 1) - length roots ≤ p.maxDegree
proof -
  let ?S = {d * (clength - 1) - length rs | a rs d. (a,rs,d) ∈ set spec}
  have finite-S: finite ?S
  proof -
    have ?S = (λ(-, rs, d). d * (clength - 1) - length rs) ` set spec
      by force
    then show ?thesis by simp
  qed
  have elem: d * (clength - 1) - length roots ∈ ?S
    using assms by force
  have d * (clength - 1) - length roots ≤ Max (insert 0 ?S)
    by (rule Max-ge) (use finite-S elem in auto)
  then show ?thesis
    unfolding p.maxDegree-def degrees-def .
qed

```

```

lemma composition-term-degree:
  assumes hta: honest-trace-algebra
  and spec-entry: (c, roots, d) ∈ set spec
  shows
    degree (CP a * ((c p.f-powers) div (prod (p.g-map roots)))) ≤ p.maxDegree
proof -
  have qdeg:
    degree ((c p.f-powers) div (prod (p.g-map roots))) ≤
      d * (clength - 1) - length roots
    using hta spec-entry by (rule honest-trace-algebra-quotient-degree)
  have degree (CP a * ((c p.f-powers) div (prod (p.g-map roots)))) ≤
    degree (CP a) + degree ((c p.f-powers) div (prod (p.g-map roots)))
    by (rule degree-mult-le)
  also have ... ≤ d * (clength - 1) - length roots
    using qdeg unfolding CP-def by (simp add: monom-0)
  also have ... ≤ p.maxDegree
    using spec-entry by (rule maxDegree-ge-spec)
  finally show ?thesis .
qed

```

```

lemma cp-fold-degree-aux:
  assumes terms:

```

```

 $\bigwedge a \ c \ roots \ d.$ 
   $(a, (c, roots, d)) \in set \ xs \implies$ 
     $degree \ (CP \ a * ((c \ p.f-powers) \ div \ (prod \ (p.g-map \ roots)))) \leq p.maxDegree$ 
  and  $acc: degree \ acc \leq p.maxDegree$ 
shows
  degree
    (fold
       $(\lambda(a, (c, roots, -)).$ 
         $(+) \ (CP \ a * ((c \ p.f-powers) \ div \ (prod \ (p.g-map \ roots))))$ 
       $xs \ acc) \leq p.maxDegree$ 
    using terms acc
proof (induction xs arbitrary: acc)
  case Nil
  then show ?case by simp
next
  case (Cons x xs)
  obtain  $a \ c \ roots \ d$  where  $x: x = (a, (c, roots, d))$ 
  by (cases x) (auto split: prod.splits)
  have term-deg:
     $degree \ (CP \ a * ((c \ p.f-powers) \ div \ (prod \ (p.g-map \ roots)))) \leq p.maxDegree$ 
  using Cons.prem1[of a c roots d] unfolding x by simp
  have acc':
    degree
       $((\lambda(a, (c, roots, -)).$ 
         $(+) \ (CP \ a * ((c \ p.f-powers) \ div \ (prod \ (p.g-map \ roots))))$ 
       $x \ acc) \leq p.maxDegree$ 
  using term-deg Cons.prem2
  unfolding x
  by (auto intro: degree-add-le)
  show ?case
  using Cons.IH[OF - acc'] Cons.prem1
  by auto
qed

lemma composition-degree-from-honest-trace-algebra:
  assumes honest-trace-algebra
  shows  $degree \ (p.cp \ as \ p.f-powers) \leq p.maxDegree$ 
  unfolding p.cp-def
proof (rule cp-fold-degree-aux)
  fix  $a \ c \ roots \ d$ 
  assume  $(a, (c, roots, d)) \in set \ (zip \ as \ spec)$ 
  then have spec-entry:  $(c, roots, d) \in set \ spec$ 
  by (auto dest!: set-zip-rightD)
  show  $degree \ (CP \ a * ((c \ p.f-powers) \ div \ (prod \ (p.g-map \ roots)))) \leq p.maxDegree$ 
  by (rule composition-term-degree[OF assms spec-entry])
qed simp

lemma composition-degree-from-honest-trace-valid:
  assumes honest-trace-valid

```

shows $\text{degree } (p.\text{cp as } p.\text{f-powers}) \leq p.\text{maxDegree}$
using *assms*
unfolding *honest-trace-valid-def*
by (*intro composition-degree-from-honest-trace-algebra*) *simp*

lemma *honest-degree-message-assert-hol*:
assumes *honest-trace-valid*
shows $\text{to-nat } (\text{of-nat } (\text{degree } (p.\text{cp as } p.\text{f-powers}))) \leq p.\text{maxDegree}$
by (*rule degree-assert-hol[OF composition-degree-from-honest-trace-valid[OF assms]]*)

lemma *honest-degree-message-rounds*:
assumes *honest-trace-valid*
shows $\text{ceil-log } (\text{to-nat } (\text{of-nat } (\text{degree } (p.\text{cp as } p.\text{f-powers}))) + 1) =$
 $\text{ceil-log } (\text{degree } (p.\text{cp as } p.\text{f-powers}) + 1)$
using *degree-message-decodes[OF composition-degree-from-honest-trace-valid[OF assms]]*
by *simp*

end

end

theory *Completeness-Transcript*
imports *Completeness-Algebra*
begin

Transcript-extension and hash-extension lemmas for honest prover execution.

context *verification*
begin

lemma *transcript-send-outcome*:
assumes $\text{Some } ((), t) \in \text{set-dist } (\text{execute } (p.\text{send } x) s)$
shows $t = s \llbracket P\text{State} := \text{concat } (P\text{State } s), P\text{Transcript} := x \# P\text{Transcript } s \rrbracket$
using *p.send-outcome[OF assms]* .

lemma *send-transcript-extends*:
assumes $\text{Some } ((), t) \in \text{set-dist } (\text{execute } (p.\text{send } x) s)$
shows *transcript-extends t s*
using *transcript-send-outcome[OF assms]*
unfolding *transcript-extends-def* **by** *auto*

lemma *send-hash-extends*:
assumes $\text{Some } ((), t) \in \text{set-dist } (\text{execute } (p.\text{send } x) s)$
shows $s \leq t$
using *p.send-outcome[OF assms]*
unfolding *less-eq-hash-ext-def less-eq-fmap-def* **by** *simp*

lemma *transcript-read-cons-outcome*:
assumes $\text{Some } (y, t) \in \text{set-dist } (\text{execute } p.\text{read } (s \llbracket P\text{Transcript} := x \# xs \rrbracket))$

shows

$y = x \wedge$
 $t = s(PState := \text{concat } (PState\ s)\ x, PTranscript := xs)$
using $p.read-cons-outcome[OF\ assms]$.

lemma *create-preserves-transcript:*

assumes $outcome: \text{Some } (r, t) \in \text{set-dist } (\text{execute } (p.create\ xs)\ s)$
shows $PTranscript\ t = PTranscript\ s$
by ($\text{rule } p.create-preserves-channel(2)[OF\ outcome]$)

lemma *create-preserves-state:*

assumes $outcome: \text{Some } (r, t) \in \text{set-dist } (\text{execute } (p.create\ xs)\ s)$
shows $PState\ t = PState\ s$
by ($\text{rule } p.create-preserves-channel(1)[OF\ outcome]$)

lemma *create-transcript-extends:*

assumes $\text{Some } (r, t) \in \text{set-dist } (\text{execute } (p.create\ xs)\ s)$
shows $\text{transcript-extends } t\ s$
using $\text{create-preserves-transcript}[OF\ assms]$
unfolding $\text{transcript-extends-def}$ **by** *auto*

lemma *create-hash-extends:*

assumes $\text{Some } (r, t) \in \text{set-dist } (\text{execute } (p.create\ xs)\ s)$
shows $s \leq t$
using $p.create-outcome[OF\ assms]$ **by** *simp*

lemma *receive-random-field-element-preserves-transcript:*

assumes $\text{Some } (a, t) \in \text{set-dist } (\text{execute } p.receive-random-field-element\ s)$
shows $PTranscript\ t = PTranscript\ s$
using $p.receive-random-field-element-outcome[OF\ assms]$ **by** *simp*

lemma *receive-random-field-element-transcript-extends:*

assumes $\text{Some } (a, t) \in \text{set-dist } (\text{execute } p.receive-random-field-element\ s)$
shows $\text{transcript-extends } t\ s$
using $\text{receive-random-field-element-preserves-transcript}[OF\ assms]$
unfolding $\text{transcript-extends-def}$ **by** *auto*

lemma *receive-alpha-challenge-preserves-transcript:*

assumes $\text{Some } (a, t) \in \text{set-dist } (\text{execute } p.receive-alpha-challenge\ s)$
shows $PTranscript\ t = PTranscript\ s$
using $p.receive-alpha-challenge-outcome[OF\ assms]$ **by** *simp*

lemma *receive-alpha-challenge-transcript-extends:*

assumes $\text{Some } (a, t) \in \text{set-dist } (\text{execute } p.receive-alpha-challenge\ s)$
shows $\text{transcript-extends } t\ s$
using $\text{receive-alpha-challenge-preserves-transcript}[OF\ assms]$
unfolding $\text{transcript-extends-def}$ **by** *auto*

lemma *receive-query-index-challenge-preserves-transcript:*

assumes *Some* $(a, t) \in \text{set-dist}$ (*execute* $p.\text{receive-query-index-challenge}$ s)
shows $P\text{Transcript } t = P\text{Transcript } s$
using $p.\text{receive-query-index-challenge-outcome}[OF \text{ assms}]$ **by** *simp*

lemma *receive-query-index-challenge-transcript-extends:*

assumes *Some* $(a, t) \in \text{set-dist}$ (*execute* $p.\text{receive-query-index-challenge}$ s)
shows *transcript-extends* t s
using *receive-query-index-challenge-preserves-transcript* $[OF \text{ assms}]$
unfolding *transcript-extends-def* **by** *auto*

lemma *receive-random-field-element-outcome:*

assumes *outcome: Some* $(a, t) \in \text{set-dist}$ (*execute* $p.\text{receive-random-field-element}$ s)
shows
 $s \leq t \wedge$
 $P\text{State } t = P\text{State } s \wedge$
 $P\text{Transcript } t = P\text{Transcript } s \wedge$
 $\text{fmlookup } (\text{HashMap } t) (\text{FiatShamirChallenge } (P\text{State } s)) = \text{Some } a$
by (*rule* $p.\text{receive-random-field-element-outcome}[OF \text{ outcome}]$)

lemma *bind-transcript-extends:*

assumes *outcome: Some* $(y, t) \in \text{set-dist}$ (*execute* $(m \gg k)$ s)
and *m-ext:* $\bigwedge x u. \text{Some } (x, u) \in \text{set-dist}$ (*execute* m s) $\implies$
 $\text{transcript-extends } u$ s
and *k-ext:* $\bigwedge x u y t. \text{Some } (x, u) \in \text{set-dist}$ (*execute* m s) $\implies$
 $\text{Some } (y, t) \in \text{set-dist}$ (*execute* $(k \ x)$ u) $\implies$
 $\text{transcript-extends } t$ u

shows *transcript-extends* t s

proof –

from *outcome* **obtain** x u **where**

$mx: \text{Some } (x, u) \in \text{set-dist}$ (*execute* m s)

and $ky: \text{Some } (y, t) \in \text{set-dist}$ (*execute* $(k \ x)$ u)

by (*auto elim!*: $p.\text{set-dist-bindE}$)

show *?thesis*

using $m\text{-ext}[OF \text{ mx}]$ $k\text{-ext}[OF \text{ mx ky}]$ **by** (*rule* *transcript-extends-trans*)

qed

lemma *mmap-transcript-extends:*

assumes *outcome: Some* $(xs, t) \in \text{set-dist}$ (*execute* $(\text{mmap } ms)$ s)
and *step:* $\bigwedge m x u v. m \in \text{set } ms \implies$
 $\text{Some } (x, v) \in \text{set-dist}$ (*execute* m u) $\implies$
 $\text{transcript-extends } v$ u

shows *transcript-extends* t s

using *outcome step*

proof (*induction* ms *arbitrary:* xs s t)

case *Nil*

then show *?case* **by** *simp*

next

case $(\text{Cons } m \ ms)$

from $\text{Cons.prem}(1)$ **obtain** x u xs' **where**

```

  head: Some (x, u) ∈ set-dist (execute m s)
  and tail: Some (xs', t) ∈ set-dist (execute (mmap ms) u)
  by (auto elim!: p.set-dist-bindE)
have m-in: m ∈ set (m # ms)
  by simp
have head-ext: transcript-extends u s
  using Cons.prem(2)[OF m-in head] .
have tail-step:
   $\bigwedge m' x' u' v'. m' \in \text{set } ms \implies$ 
    Some (x', v') ∈ set-dist (execute m' u')  $\implies$ 
      transcript-extends v' u'
  using Cons.prem(2) by auto
have tail-ext: transcript-extends t u
  by (rule Cons.IH[OF tail tail-step])
show ?case
  using head-ext tail-ext by (rule transcript-extends-trans)
qed

lemma mfold-transcript-extends:
  assumes outcome: Some (x, t) ∈ set-dist (execute (mfold a ms) s)
  and step:  $\bigwedge m a0 y u v. m \in \text{set } ms \implies$ 
    Some (y, v) ∈ set-dist (execute (m a0) u)  $\implies$ 
      transcript-extends v u
  shows transcript-extends t s
  using outcome step
proof (induction ms arbitrary: a x s t)
  case Nil
  then show ?case by simp
next
  case (Cons m ms)
  from Cons.prem(1) obtain y u where
    head: Some (y, u) ∈ set-dist (execute (m a) s)
    and tail: Some (x, t) ∈ set-dist (execute (mfold y ms) u)
    by (auto elim!: p.set-dist-bindE)
  have m-in: m ∈ set (m # ms)
    by simp
  have head-ext: transcript-extends u s
    using Cons.prem(2)[OF m-in head] .
  have tail-step:
     $\bigwedge m' a0 y' u' v'. m' \in \text{set } ms \implies$ 
      Some (y', v') ∈ set-dist (execute (m' a0) u')  $\implies$ 
        transcript-extends v' u'
  proof -
    fix m' a0 y' u' v'
    assume m'-in-tail: m' ∈ set ms
    and out: Some (y', v') ∈ set-dist (execute (m' a0) u')
    have m'-in: m' ∈ set (m # ms)
      using m'-in-tail by simp
    show transcript-extends v' u'

```

```

    using Cons.premis(2)[OF m'-in out] .
qed
have tail-ext: transcript-extends t u
  by (rule Cons.IH[OF tail tail-step])
show ?case
  using head-ext tail-ext by (rule transcript-extends-trans)
qed

lemma mmap-hash-extends:
  fixes s t :: ('f, 'more) protocol-channel-scheme
  assumes outcome: Some (xs, t) ∈ set-dist (execute (mmap ms) s)
  and step:  $\bigwedge m x u v. m \in \text{set } ms \implies$ 
    Some (x, v) ∈ set-dist (execute m u)  $\implies$ 
    u ≤ v
  shows s ≤ t
  using outcome step
proof (induction ms arbitrary: xs s t)
  case Nil
  have t-eq: t = s
    using Nil.premis(1) by simp
  show ?case
    unfolding t-eq less-eq-hash-ext-def less-eq-fmap-def by simp
next
  case (Cons m ms)
  from Cons.premis(1) obtain x u xs' where
    head: Some (x, u) ∈ set-dist (execute m s)
    and tail: Some (xs', t) ∈ set-dist (execute (mmap ms) u)
    by (auto elim!: p.set-dist-bindE)
  have m-in: m ∈ set (m # ms)
    by simp
  have head-ext: s ≤ u
    using Cons.premis(2)[OF m-in head] .
  have tail-step:
     $\bigwedge m' x' u' v'. m' \in \text{set } ms \implies$ 
    Some (x', v') ∈ set-dist (execute m' u')  $\implies$ 
    u' ≤ v'
  proof -
    fix m' x' u' v'
    assume m'-in-tail: m' ∈ set ms
    and out: Some (x', v') ∈ set-dist (execute m' u')
    have m'-in: m' ∈ set (m # ms)
      using m'-in-tail by simp
    show u' ≤ v'
      using Cons.premis(2)[OF m'-in out] .
  qed
  have tail-ext: u ≤ t
    by (rule Cons.IH[OF tail tail-step])
  show ?case
    using head-ext tail-ext by (rule p.hash-ext-trans)

```

qed

lemma *mfold-hash-extends*:

fixes $s\ t :: ('f, 'more)\ \text{protocol-channel-scheme}$
assumes *outcome*: $\text{Some } (x, t) \in \text{set-dist } (\text{execute } (\text{mfold } a\ ms)\ s)$
and step: $\bigwedge m\ a0\ y\ u\ v. m \in \text{set } ms \implies$
 $\text{Some } (y, v) \in \text{set-dist } (\text{execute } (m\ a0)\ u) \implies$
 $u \leq v$

shows $s \leq t$

using *outcome step*

proof (*induction ms arbitrary: a x s t*)

case *Nil*

have *t-eq*: $t = s$

using *Nil.premis(1)* **by** *simp*

show *?case*

unfolding *t-eq less-eq-hash-ext-def less-eq-fmap-def* **by** *simp*

next

case (*Cons m ms*)

from *Cons.premis(1)* **obtain** $y\ u$ **where**

head: $\text{Some } (y, u) \in \text{set-dist } (\text{execute } (m\ a)\ s)$

and *tail*: $\text{Some } (x, t) \in \text{set-dist } (\text{execute } (\text{mfold } y\ ms)\ u)$

by (*auto elim!*: *p.set-dist-bindE*)

have *m-in*: $m \in \text{set } (m \# ms)$

by *simp*

have *head-ext*: $s \leq u$

using *Cons.premis(2)[OF m-in head]* .

have *tail-step*:

$\bigwedge m'\ a0\ y'\ u'\ v'. m' \in \text{set } ms \implies$

$\text{Some } (y', v') \in \text{set-dist } (\text{execute } (m'\ a0)\ u') \implies$

$u' \leq v'$

proof –

fix $m'\ a0\ y'\ u'\ v'$

assume *m'-in-tail*: $m' \in \text{set } ms$

and *out*: $\text{Some } (y', v') \in \text{set-dist } (\text{execute } (m'\ a0)\ u')$

have *m'-in*: $m' \in \text{set } (m \# ms)$

using *m'-in-tail* **by** *simp*

show $u' \leq v'$

using *Cons.premis(2)[OF m'-in out]* .

qed

have *tail-ext*: $u \leq t$

by (*rule Cons.IH[OF tail tail-step]*)

show *?case*

using *head-ext tail-ext* **by** (*rule p.hash-ext-trans*)

qed

lemma *mmap-preserves-query-counter*:

fixes $s\ t :: ('f, 'more)\ \text{protocol-channel-scheme}$

assumes *outcome*: $\text{Some } (xs, t) \in \text{set-dist } (\text{execute } (\text{mmap } ms)\ s)$

and step: $\bigwedge m\ x\ u\ v. m \in \text{set } ms \implies$


```

    Some (x, v) ∈ set-dist (execute m u) ⇒
    PQueryCounter v = PQueryCounter u
  shows PQueryCounter t = PQueryCounter s
  using outcome step
proof (induction ms arbitrary: xs s t)
  case Nil
  then show ?case by simp
next
  case (Cons m ms)
  from Cons.prem1(1) obtain x u xs' where
    head: Some (x, u) ∈ set-dist (execute m s)
    and tail: Some (xs', t) ∈ set-dist (execute (mmap ms) u)
    by (auto elim!: p.set-dist-bindE)
  have m-in: m ∈ set (m # ms)
    by simp
  have q-head: PQueryCounter u = PQueryCounter s
    by (rule Cons.prem1(2)[OF m-in head])
  have tail-step:
    ∧ m' x' u' v'. m' ∈ set ms ⇒
    Some (x', v') ∈ set-dist (execute m' u') ⇒
    PQueryCounter v' = PQueryCounter u'
  proof -
    fix m' x' u' v'
    assume m'-in-tail: m' ∈ set ms
    and out: Some (x', v') ∈ set-dist (execute m' u')
    have m'-in: m' ∈ set (m # ms)
      using m'-in-tail by simp
    show PQueryCounter v' = PQueryCounter u'
      by (rule Cons.prem1(2)[OF m'-in out])
  qed
  have q-tail: PQueryCounter t = PQueryCounter u
    by (rule Cons.IH[OF tail tail-step])
  show ?case
    using q-head q-tail by simp
qed

lemma mfold-preserves-query-counter:
  fixes s t :: ('f, 'more) protocol-channel-scheme
  assumes outcome: Some (x, t) ∈ set-dist (execute (mfold a ms) s)
  and step: ∧ m a0 y u v. m ∈ set ms ⇒
    Some (y, v) ∈ set-dist (execute (m a0) u) ⇒
    PQueryCounter v = PQueryCounter u
  shows PQueryCounter t = PQueryCounter s
  using outcome step
proof (induction ms arbitrary: a x s t)
  case Nil
  then show ?case by simp
next
  case (Cons m ms)

```

```

from Cons.prem(1) obtain y u where
  head: Some (y, u) ∈ set-dist (execute (m a) s)
  and tail: Some (x, t) ∈ set-dist (execute (mfold y ms) u)
  by (auto elim!: p.set-dist-bindE)
have m-in: m ∈ set (m # ms)
  by simp
have q-head: PQueryCounter u = PQueryCounter s
  by (rule Cons.prem(2)[OF m-in head])
have tail-step:
   $\bigwedge m' a0 y' u' v'. m' \in \text{set } ms \implies$ 
    Some (y', v') ∈ set-dist (execute (m' a0) u')  $\implies$ 
    PQueryCounter v' = PQueryCounter u'
proof -
  fix m' a0 y' u' v'
  assume m'-in-tail: m' ∈ set ms
  and out: Some (y', v') ∈ set-dist (execute (m' a0) u')
  have m'-in: m' ∈ set (m # ms)
  using m'-in-tail by simp
  show PQueryCounter v' = PQueryCounter u'
  by (rule Cons.prem(2)[OF m'-in out])
qed
have q-tail: PQueryCounter t = PQueryCounter u
  by (rule Cons.IH[OF tail tail-step])
show ?case
  using q-head q-tail by simp
qed

lemma mfold2-send-transcript-extends:
  assumes Some ((), t) ∈ set-dist (execute (mfold2 p.send xs) s)
  shows transcript-extends t s
  using p.mfold2-send-outcome[OF assms]
  unfolding transcript-extends-def by auto

lemma mfold2-send-hash-extends:
  assumes Some ((), t) ∈ set-dist (execute (mfold2 p.send xs) s)
  shows s ≤ t
  using p.mfold2-send-outcome[OF assms]
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp

lemma mfold2-send-preserves-query-counter:
  assumes Some ((), t) ∈ set-dist (execute (mfold2 p.send xs) s)
  shows PQueryCounter t = PQueryCounter s
  using p.mfold2-send-outcome[OF assms] by simp

lemma fri-commit-transcript-extends:
  assumes outcome:
    Some ((ps', ds', ls', ms'), t) ∈
      set-dist (execute (p.fri-commit n ps ds ls ms) s)
  shows transcript-extends t s

```

```

using outcome
proof (induction n arbitrary: ps ds ls ms s ps' ds' ls' ms' t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.prems obtain s1 b s2 next-poly next-domain next-layer m s3 where
    send-root: Some ((), s1)  $\in$  set-dist (execute (p.send (value (last ms)))) s
    and rand: Some (b, s2)  $\in$  set-dist (execute p.receive-random-field-element s1)
    and next-layer-eq: p.next-fri-layer (last ps) (last ds) b =
      (next-poly, next-domain, next-layer)
    and create-m: Some (m, s3)  $\in$  set-dist (execute (p.create next-layer) s2)
    and rest: Some ((ps', ds', ls', ms'), t)  $\in$ 
      set-dist (execute
        (p.fri-commit n
          (ps @ [next-poly])
          (ds @ [next-domain])
          (ls @ [next-layer])
          (ms @ [m])) s3)
      by (auto simp: Let-def split: prod.splits elim!: p.set-dist-bindE)
  have s-s1: transcript-extends s1 s
    using send-transcript-extends[OF send-root] .
  have s1-s2: transcript-extends s2 s1
    using receive-random-field-element-transcript-extends[OF rand] .
  have s2-s3: transcript-extends s3 s2
    using create-transcript-extends[OF create-m] .
  have s3-t: transcript-extends t s3
    using Suc.IH[OF rest] .
  show ?case
    using s-s1 s1-s2 s2-s3 s3-t by (meson transcript-extends-trans)
qed

lemma fri-commit-replay-data:
  assumes outcome:
    Some ((ps', ds', ls', ms'), t)  $\in$ 
      set-dist (execute (p.fri-commit n ps ds ls ms) s)
  shows  $\exists$  roots challenges.
    length roots = n  $\wedge$ 
    length challenges = n  $\wedge$ 
    PTranscript t = rev roots @ PTranscript s  $\wedge$ 
    PState t = foldl concat (PState s) roots  $\wedge$ 
    s  $\leq$  t  $\wedge$ 
    ( $\forall i < \text{length roots.}$ 
      fmlookup (HashMap t) (FiatShamirChallenge (foldl concat (PState s) (take (Suc i) roots))))
  =
    Some (challenges ! i)
  using outcome
proof (induction n arbitrary: ps ds ls ms s ps' ds' ls' ms' t)
  case 0

```

```

then show ?case
  by (auto simp: p.hash-ext-refl)
next
case (Suc n)
from Suc.premis obtain s1 b s2 next-poly next-domain next-layer m s3 where
  send-root: Some ((), s1) ∈ set-dist (execute (p.send (value (last ms))) s)
  and rand: Some (b, s2) ∈ set-dist (execute p.receive-random-field-element s1)
  and next-layer-eq: p.next-fri-layer (last ps) (last ds) b =
    (next-poly, next-domain, next-layer)
  and create-m: Some (m, s3) ∈ set-dist (execute (p.create next-layer) s2)
  and rest:
    Some ((ps', ds', ls', ms'), t) ∈
      set-dist (execute
        (p.fri-commit n
          (ps @ [next-poly])
          (ds @ [next-domain])
          (ls @ [next-layer])
          (ms @ [m])) s3)
  by (auto simp: Let-def split: prod.splits elim!: p.set-dist-bindE)
let ?root = value (last ms)
have send-res:
  s1 = s(PState := concat (PState s) ?root, PTranscript := ?root # PTranscript s)
  using p.send-outcome[OF send-root] .
have rand-data:
  s1 ≤ s2 ∧
  PState s2 = PState s1 ∧
  PTranscript s2 = PTranscript s1 ∧
  fmlookup (HashMap s2) (FiatShamirChallenge (PState s1)) = Some b
  using receive-random-field-element-outcome[OF rand] .
have create-state: PState s3 = PState s2
  using create-preserves-state[OF create-m] .
have create-tr: PTranscript s3 = PTranscript s2
  using create-preserves-transcript[OF create-m] .
have s2-s3: s2 ≤ s3
  using create-hash-extends[OF create-m] .
from Suc.IH[OF rest] obtain roots challenges where
  roots-len: length roots = n
  and challenges-len: length challenges = n
  and tr-t: PTranscript t = rev roots @ PTranscript s3
  and st-t: PState t = foldl concat (PState s3) roots
  and s3-t: s3 ≤ t
  and lookups-tail:
    ∀ i < length roots.
      fmlookup (HashMap t) (FiatShamirChallenge (foldl concat (PState s3) (take (Suc i)
roots))) =
        Some (challenges ! i)
  by blast
have s-s1: s ≤ s1
  using send-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp

```

```

have s-t:  $s \leq t$ 
  using s-s1 rand-data s2-s3 s3-t by (meson p.hash-ext-trans)
have s2-t:  $s2 \leq t$ 
  using s2-s3 s3-t by (rule p.hash-ext-trans)
have lookup-head:
  fmllookup (HashMap t) (FiatShamirChallenge (concat (PState s) ?root)) = Some b
proof -
  have lookup-s2: fmllookup (HashMap s2) (FiatShamirChallenge (PState s1)) = Some b
    using rand-data by simp
  have fmllookup (HashMap t) (FiatShamirChallenge (PState s1)) = Some b
    using p.hash-extension-lookup[OF lookup-s2 s2-t] .
  then show ?thesis
    using send-res by simp
qed
have tr-final:
  PTranscript t = rev (?root # roots) @ PTranscript s
  using tr-t create-tr rand-data send-res by simp
have st-final:
  PState t = foldl concat (PState s) (?root # roots)
  using st-t create-state rand-data send-res by simp
have lookups:
   $\forall i < \text{length } (?root \# \text{roots}).$ 
  fmllookup (HashMap t) (FiatShamirChallenge (foldl concat (PState s) (take (Suc i) (?root
# roots)))) =
    Some ((b # challenges) ! i)
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{length } (?root \# \text{roots})$ 
  show fmllookup (HashMap t) (FiatShamirChallenge (foldl concat (PState s) (take (Suc i)
(?root # roots)))) =
    Some ((b # challenges) ! i)
  proof (cases i)
    case 0
    then show ?thesis
      using lookup-head by simp
  next
    case (Suc j)
    then have j-bound:  $j < \text{length roots}$ 
      using i-bound by simp
    have key-eq:
      foldl concat (PState s) (take (Suc (Suc j)) (?root # roots)) =
        foldl concat (PState s3) (take (Suc j) roots)
      using send-res rand-data create-state by simp
    show ?thesis
      using lookups-tail j-bound key-eq Suc by simp
  qed
qed
show ?case
proof (rule exI[where x=?root # roots])

```

```

show  $\exists$  challenges.
  length (?root # roots) = Suc n  $\wedge$ 
  length challenges = Suc n  $\wedge$ 
  PTranscript t = rev (?root # roots) @ PTranscript s  $\wedge$ 
  PState t = foldl concat (PState s) (?root # roots)  $\wedge$ 
  s  $\leq$  t  $\wedge$ 
  ( $\forall i < \text{length } (?root \# roots)$ .
    fmllookup (HashMap t)
      (FiatShamirChallenge (foldl concat (PState s) (take (Suc i) (?root # roots)))) =
        Some (challenges ! i))
proof (rule exI[where x=b # challenges])
show length (?root # roots) = Suc n  $\wedge$ 
  length (b # challenges) = Suc n  $\wedge$ 
  PTranscript t = rev (?root # roots) @ PTranscript s  $\wedge$ 
  PState t = foldl concat (PState s) (?root # roots)  $\wedge$ 
  s  $\leq$  t  $\wedge$ 
  ( $\forall i < \text{length } (?root \# roots)$ .
    fmllookup (HashMap t)
      (FiatShamirChallenge (foldl concat (PState s) (take (Suc i) (?root # roots)))) =
        Some ((b # challenges) ! i))
using roots-len challenges-len tr-final st-final s-t lookups by simp
qed
qed
qed

lemma fri-commit-replay-state:
assumes outcome:
  Some ((ps', ds', ls', ms'), t)  $\in$ 
    set-dist (execute (p.fri-commit n ps ds ls ms) s)
shows  $\exists$  roots.
  length roots = n  $\wedge$ 
  PTranscript t = rev roots @ PTranscript s  $\wedge$ 
  PState t = foldl concat (PState s) roots  $\wedge$ 
  s  $\leq$  t
using outcome
proof (induction n arbitrary: ps ds ls ms s ps' ds' ls' ms' t)
case 0
then show ?case
by (auto simp: p.hash-ext-refl)
next
case (Suc n)
from Suc.premis obtain s1 b s2 next-poly next-domain next-layer m s3 where
  send-root: Some ((), s1)  $\in$  set-dist (execute (p.send (value (last ms))) s)
and rand: Some (b, s2)  $\in$  set-dist (execute p.receive-random-field-element s1)
and next-layer-eq: p.next-fri-layer (last ps) (last ds) b =
  (next-poly, next-domain, next-layer)
and create-m: Some (m, s3)  $\in$  set-dist (execute (p.create next-layer) s2)
and rest:
  Some ((ps', ds', ls', ms'), t)  $\in$ 

```

```

    set-dist (execute
      (p.fri-commit n
        (ps @ [next-poly])
        (ds @ [next-domain])
        (ls @ [next-layer])
        (ms @ [m])) s3)
  by (auto simp: Let-def split: prod.splits elim!: p.set-dist-bindE)
let ?root = value (last ms)
have send-res:
  s1 = s(PState := concat (PState s) ?root, PTranscript := ?root # PTranscript s)
  using p.send-outcome[OF send-root] .
have rand-data:
  s1 ≤ s2 ∧ PState s2 = PState s1 ∧ PTranscript s2 = PTranscript s1
  using receive-random-field-element-outcome[OF rand] by simp
have create-state: PState s3 = PState s2
  using create-preserves-state[OF create-m] .
have create-tr: PTranscript s3 = PTranscript s2
  using create-preserves-transcript[OF create-m] .
have s-s1: s ≤ s1
  using send-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have s2-s3: s2 ≤ s3
  using create-hash-extends[OF create-m] .
from Suc.IH[OF rest] obtain roots where
  roots-len: length roots = n
  and tr-t: PTranscript t = rev roots @ PTranscript s3
  and st-t: PState t = foldl concat (PState s3) roots
  and s3-t: s3 ≤ t
  by blast
have tr-final: PTranscript t = rev (?root # roots) @ PTranscript s
  using tr-t create-tr rand-data send-res by simp
have st-final: PState t = foldl concat (PState s) (?root # roots)
  using st-t create-state rand-data send-res by simp
have s-t: s ≤ t
  using s-s1 rand-data s2-s3 s3-t by (meson p.hash-ext-trans)
show ?case
  by (intro exI[where x=?root # roots])
    (use roots-len tr-final st-final s-t in simp)
qed

```

lemma *fri-commit-successor-layers*:

```

assumes outcome:
  Some ((ps', ds', ls', ms'), t) ∈
  set-dist (execute (p.fri-commit n ps ds ls ms) s)
and lens:
  length ps = length ds
  length ds = length ls
  length ls = length ms
and nonempty: ps ≠ []
shows ∃ challenges.

```

```

length challenges = n ∧
(∀ j < n.
  p.next-fri-layer
    (ps' ! (length ps - 1 + j))
    (ds' ! (length ps - 1 + j))
    (challenges ! j) =
      (ps' ! (length ps + j),
       ds' ! (length ps + j),
       ls' ! (length ps + j)))
using outcome lens nonempty
proof (induction n arbitrary: ps ds ls ms s ps' ds' ls' ms' t)
  case 0
  then show ?case by auto
next
  case (Suc n)
  from Suc.prem1(1) obtain s1 b s2 next-poly next-domain next-layer m s3 where
    send-root:
      Some ((), s1) ∈ set-dist (execute (p.send (value (last ms))) s)
  and rand:
      Some (b, s2) ∈ set-dist (execute p.receive-random-field-element s1)
  and next-layer-eq:
      p.next-fri-layer (last ps) (last ds) b =
        (next-poly, next-domain, next-layer)
  and create-m:
      Some (m, s3) ∈ set-dist (execute (p.create next-layer) s2)
  and rest:
      Some ((ps', ds', ls', ms'), t) ∈
        set-dist (execute
          (p.fri-commit n
            (ps @ [next-poly])
            (ds @ [next-domain])
            (ls @ [next-layer])
            (ms @ [m])) s3)
  by (auto simp: Let-def split: prod.splits elim!: p.set-dist-bindE)
have lens-tail:
  length (ps @ [next-poly]) = length (ds @ [next-domain])
  length (ds @ [next-domain]) = length (ls @ [next-layer])
  length (ls @ [next-layer]) = length (ms @ [m])
  using Suc.prem1(2-4) by simp-all
have nonempty-tail: ps @ [next-poly] ≠ []
  by simp
from Suc.IH[OF rest lens-tail nonempty-tail] obtain challenges where
  challenges-len: length challenges = n
  and tail-layers:
    ∀ j < n.
      p.next-fri-layer
        (ps' ! (length (ps @ [next-poly]) - 1 + j))
        (ds' ! (length (ps @ [next-poly]) - 1 + j))
        (challenges ! j) =

```



```

      (ps' ! (length (ps @ [next-poly]) + j),
       ds' ! (length (ps @ [next-poly]) + j),
       ls' ! (length (ps @ [next-poly]) + j))
    by blast
have prefix:
  take (length (ps @ [next-poly])) ps' = ps @ [next-poly] ∧
  take (length (ds @ [next-domain])) ds' = ds @ [next-domain] ∧
  take (length (ls @ [next-layer])) ls' = ls @ [next-layer]
  using p.fri-commit-preserves-prefix[OF rest] by simp
have head-ps: ps' ! (length ps - 1) = last ps
proof -
  obtain q where len-ps: length ps = Suc q
  using Suc.premis(5) by (cases ps) auto
  have idx-lt: length ps - 1 < length (ps @ [next-poly])
  using Suc.premis(5) by simp
  have ps'-nth: ps' ! (length ps - 1) =
    (ps @ [next-poly]) ! (length ps - 1)
  proof -
    have (take (length (ps @ [next-poly])) ps') ! (length ps - 1) =
      (ps @ [next-poly]) ! (length ps - 1)
    using conjunct1[OF prefix] by simp
    then show ?thesis
    using idx-lt by simp
  qed
  have append-nth:
    (ps @ [next-poly]) ! (length ps - 1) = ps ! (length ps - 1)
  using len-ps by (simp add: nth-append)
  have last-ps: last ps = ps ! (length ps - 1)
  using Suc.premis(5) by (simp add: last-conv-nth)
  show ?thesis
  using ps'-nth append-nth last-ps by simp
qed
have head-ds: ds' ! (length ps - 1) = last ds
proof -
  have ds-ne: ds ≠ []
  using Suc.premis(2,5) by auto
  obtain q where len-ds: length ds = Suc q
  using ds-ne by (cases ds) auto
  have idx-lt: length ps - 1 < length (ds @ [next-domain])
  using Suc.premis(2,5) by simp
  have ds'-nth: ds' ! (length ps - 1) =
    (ds @ [next-domain]) ! (length ps - 1)
  proof -
    have (take (length (ds @ [next-domain])) ds') ! (length ps - 1) =
      (ds @ [next-domain]) ! (length ps - 1)
    using conjunct1[OF conjunct2[OF prefix]] by simp
    then show ?thesis
    using idx-lt by simp
  qed
qed

```

```

have append-nth:
  (ds @ [next-domain]) ! (length ps - 1) = ds ! (length ds - 1)
  using Suc.prem(2) len-ds by (simp add: nth-append)
have last-ds: last ds = ds ! (length ds - 1)
  using ds-ne by (simp add: last-conv-nth)
show ?thesis
  using ds'-nth append-nth last-ds by simp
qed
have head-next-ps: ps' ! length ps = next-poly
proof -
  have pref: take (Suc (length ps)) ps' = ps @ [next-poly]
    using conjunct1[OF prefix] by simp
  have ps' ! length ps = (take (Suc (length ps)) ps') ! length ps
    by simp
  also have ... = next-poly
    using pref by simp
  finally show ?thesis .
qed
have head-next-ds: ds' ! length ps = next-domain
proof -
  have pref: take (Suc (length ds)) ds' = ds @ [next-domain]
    using conjunct1[OF conjunct2[OF prefix]] Suc.prem(2) by simp
  have ds' ! length ps = (take (Suc (length ds)) ds') ! length ps
    using Suc.prem(2) by simp
  also have ... = next-domain
    using pref Suc.prem(2) by simp
  finally show ?thesis .
qed
have head-next-ls: ls' ! length ps = next-layer
proof -
  have pref: take (Suc (length ls)) ls' = ls @ [next-layer]
    using conjunct2[OF conjunct2[OF prefix]] Suc.prem(2,3) by simp
  have ls' ! length ps = (take (Suc (length ls)) ls') ! length ps
    using Suc.prem(2,3) by simp
  also have ... = next-layer
    using pref Suc.prem(2,3) by simp
  finally show ?thesis .
qed
have layers:
  ∀ j < Suc n.
    p.next-fri-layer
      (ps' ! (length ps - 1 + j))
      (ds' ! (length ps - 1 + j))
      ((b # challenges) ! j) =
      (ps' ! (length ps + j),
       ds' ! (length ps + j),
       ls' ! (length ps + j))
proof (intro allI impI)
  fix j

```

```

assume j-bound:  $j < \text{Suc } n$ 
show p.next-fri-layer
  ( $ps' ! (\text{length } ps - 1 + j)$ )
  ( $ds' ! (\text{length } ps - 1 + j)$ )
  ( $(b \# \text{challenges}) ! j$ ) =
    ( $ps' ! (\text{length } ps + j)$ ,
      $ds' ! (\text{length } ps + j)$ ,
      $ls' ! (\text{length } ps + j)$ )
proof (cases j)
  case 0
  then show ?thesis
    using next-layer-eq head-ps head-ds head-next-ps head-next-ds head-next-ls
    by simp
next
  case (Suc k)
  then have k-bound:  $k < n$ 
    using j-bound by simp
  have idx1:
     $\text{length } (ps @ [\text{next-poly}]) - 1 + k = \text{length } ps + k$ 
    by simp
  have idx2:
     $\text{length } (ps @ [\text{next-poly}]) + k = \text{length } ps + \text{Suc } k$ 
    by simp
  have idx-current:
     $\text{length } ps - 1 + \text{Suc } k = \text{length } ps + k$ 
  proof –
    obtain q where  $\text{length } ps = \text{Suc } q$ 
    using Suc.prem5 by (cases ps) auto
    then show ?thesis by simp
  qed
  have idx-left:
     $\text{Suc } (\text{length } ps - 1 + k) = \text{length } ps + k$ 
  proof –
    obtain q where  $\text{length } ps = \text{Suc } q$ 
    using Suc.prem5 by (cases ps) auto
    then show ?thesis by simp
  qed
  have tail-k:
    p.next-fri-layer
    ( $ps' ! (\text{length } ps + k)$ )
    ( $ds' ! (\text{length } ps + k)$ )
    ( $\text{challenges} ! k$ ) =
      ( $ps' ! \text{Suc } (\text{length } ps + k)$ ,
        $ds' ! \text{Suc } (\text{length } ps + k)$ ,
        $ls' ! \text{Suc } (\text{length } ps + k)$ )
    using tail-layers k-bound by simp
show ?thesis
  unfolding Suc
  apply (subst idx-current)

```

```

    apply (subst idx-current)
    using tail-k
    apply simp
    done
  qed
qed
show ?case
  by (intro exI[where x=b # challenges])
    (use challenges-len layers in simp)
qed

lemma fri-commit-replay-successor-data:
  assumes outcome:
    Some ((ps', ds', ls', ms'), t) ∈
      set-dist (execute (p.fri-commit n ps ds ls ms) s)
  and lens:
    length ps = length ds
    length ds = length ls
    length ls = length ms
  and nonempty: ps ≠ []
  shows ∃ roots challenges.
    length roots = n ∧
    length challenges = n ∧
    PTranscript t = rev roots @ PTranscript s ∧
    PState t = foldl concat (PState s) roots ∧
    s ≤ t ∧
    (∀ i < length roots.
      fmlookup (HashMap t) (FiatShamirChallenge (foldl concat (PState s) (take (Suc i) roots)))
    =
      Some (challenges ! i)) ∧
    (∀ j < n.
      roots ! j = value (ms' ! (length ms - 1 + j))) ∧
    (∀ j < n.
      p.next-fri-layer
        (ps' ! (length ps - 1 + j))
        (ds' ! (length ps - 1 + j))
        (challenges ! j) =
        (ps' ! (length ps + j),
         ds' ! (length ps + j),
         ls' ! (length ps + j)))
  using outcome lens nonempty
proof (induction n arbitrary: ps ds ls ms s ps' ds' ls' ms' t)
  case 0
  then show ?case
    by (auto simp: p.hash-ext-refl)
next
  case (Suc n)
  from Suc.premis(1) obtain s1 b s2 next-poly next-domain next-layer m s3 where
    send-root:

```

Some $(((), s1) \in \text{set-dist } (\text{execute } (p.\text{send } (\text{value } (\text{last } ms)))) s)$
and *rand*:
 Some $(b, s2) \in \text{set-dist } (\text{execute } p.\text{receive-random-field-element } s1)$
and *next-layer-eq*:
 $p.\text{next-fri-layer } (\text{last } ps) (\text{last } ds) b =$
 $(\text{next-poly}, \text{next-domain}, \text{next-layer})$
and *create-m*:
 Some $(m, s3) \in \text{set-dist } (\text{execute } (p.\text{create next-layer}) s2)$
and *rest*:
 Some $((ps', ds', ls', ms'), t) \in$
 $\text{set-dist } (\text{execute}$
 $(p.\text{fri-commit } n$
 $(ps @ [\text{next-poly}])$
 $(ds @ [\text{next-domain}])$
 $(ls @ [\text{next-layer}])$
 $(ms @ [m])) s3)$
by (*auto simp: Let-def split: prod.splits elim!: p.set-dist-bindE*)
let $?root = \text{value } (\text{last } ms)$
have *lens-tail*:
 $\text{length } (ps @ [\text{next-poly}]) = \text{length } (ds @ [\text{next-domain}])$
 $\text{length } (ds @ [\text{next-domain}]) = \text{length } (ls @ [\text{next-layer}])$
 $\text{length } (ls @ [\text{next-layer}]) = \text{length } (ms @ [m])$
using *Suc.premis(2-4)* **by** *simp-all*
have *nonempty-tail*: $ps @ [\text{next-poly}] \neq []$
by *simp*
from *Suc.IH[OF rest lens-tail nonempty-tail]* **obtain** *roots challenges* **where**
 $\text{roots-len: length roots} = n$
and *challenges-len*: $\text{length challenges} = n$
and *tr-t*: $P\text{Transcript } t = \text{rev roots} @ P\text{Transcript } s3$
and *st-t*: $P\text{State } t = \text{foldl concat } (P\text{State } s3) \text{ roots}$
and *s3-t*: $s3 \leq t$
and *lookups-tail*:
 $\forall i < \text{length roots.}$
 $\text{fmlookup } (\text{HashMap } t) (\text{FiatShamirChallenge } (\text{foldl concat } (P\text{State } s3) (\text{take } (Suc i)$
 $\text{roots}))) =$
 $\text{Some } (\text{challenges } ! i)$
and *roots-tail*:
 $\forall j < n.$
 $\text{roots } ! j =$
 $\text{value } (ms' ! (\text{length } (ms @ [m]) - 1 + j))$
and *tail-layers*:
 $\forall j < n.$
 $p.\text{next-fri-layer}$
 $(ps' ! (\text{length } (ps @ [\text{next-poly}]) - 1 + j))$
 $(ds' ! (\text{length } (ps @ [\text{next-poly}]) - 1 + j))$
 $(\text{challenges } ! j) =$
 $(ps' ! (\text{length } (ps @ [\text{next-poly}]) + j),$
 $ds' ! (\text{length } (ps @ [\text{next-poly}]) + j),$
 $ls' ! (\text{length } (ps @ [\text{next-poly}]) + j))$

```

  by blast
have send-res:
  s1 = s(PState := concat (PState s) ?root, PTranscript := ?root # PTranscript s)
  using p.send-outcome[OF send-root] .
have rand-data:
  s1 ≤ s2 ∧
  PState s2 = PState s1 ∧
  PTranscript s2 = PTranscript s1 ∧
  fmlookup (HashMap s2) (FiatShamirChallenge (PState s1)) = Some b
  using receive-random-field-element-outcome[OF rand] .
have create-state: PState s3 = PState s2
  using create-preserves-state[OF create-m] .
have create-tr: PTranscript s3 = PTranscript s2
  using create-preserves-transcript[OF create-m] .
have s2-s3: s2 ≤ s3
  using create-hash-extends[OF create-m] .
have s-s1: s ≤ s1
  using send-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have s-t: s ≤ t
  using s-s1 rand-data s2-s3 s3-t by (meson p.hash-ext-trans)
have s2-t: s2 ≤ t
  using s2-s3 s3-t by (rule p.hash-ext-trans)
have lookup-head:
  fmlookup (HashMap t) (FiatShamirChallenge (concat (PState s) ?root)) = Some b
proof -
  have lookup-s2: fmlookup (HashMap s2) (FiatShamirChallenge (PState s1)) = Some b
    using rand-data by simp
  have fmlookup (HashMap t) (FiatShamirChallenge (PState s1)) = Some b
    using p.hash-extension-lookup[OF lookup-s2 s2-t] .
  then show ?thesis
    using send-res by simp
qed
have tr-final:
  PTranscript t = rev (?root # roots) @ PTranscript s
  using tr-t create-tr rand-data send-res by simp
have st-final:
  PState t = foldl concat (PState s) (?root # roots)
  using st-t create-state rand-data send-res by simp
have lookups:
  ∀ i < length (?root # roots).
    fmlookup (HashMap t) (FiatShamirChallenge (foldl concat (PState s) (take (Suc i) (?root
# roots)))) =
      Some ((b # challenges) ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length (?root # roots)
  show fmlookup (HashMap t)
    (FiatShamirChallenge (foldl concat (PState s) (take (Suc i) (?root # roots)))) =
      Some ((b # challenges) ! i)

```

```

proof (cases i)
  case 0
  then show ?thesis
    using lookup-head by simp
next
  case (Suc j)
  then have j-bound:  $j < \text{length roots}$ 
    using i-bound by simp
  have key-eq:
    foldl concat (PState s) (take (Suc (Suc j)) (?root # roots)) =
      foldl concat (PState s3) (take (Suc j) roots)
    using send-res rand-data create-state by simp
  show ?thesis
    using lookups-tail j-bound key-eq Suc by simp
qed
qed
have prefix:
  take (length (ps @ [next-poly])) ps' = ps @ [next-poly] ∧
  take (length (ds @ [next-domain])) ds' = ds @ [next-domain] ∧
  take (length (ls @ [next-layer])) ls' = ls @ [next-layer] ∧
  take (length (ms @ [m])) ms' = ms @ [m]
  using p.fri-commit-preserves-prefix[OF rest] by simp
have head-ms: ms' ! (length ms - 1) = last ms
proof -
  have ms-ne:  $ms \neq []$ 
    using Suc.premis(2-5) by auto
  obtain q where len-ms: length ms = Suc q
    using ms-ne by (cases ms) auto
  have idx-lt: length ms - 1 < length (ms @ [m])
    using ms-ne by simp
  have ms'-nth: ms' ! (length ms - 1) =
    (ms @ [m]) ! (length ms - 1)
  proof -
    have (take (length (ms @ [m])) ms') ! (length ms - 1) =
      (ms @ [m]) ! (length ms - 1)
      using conjunct2[OF conjunct2[OF conjunct2[OF prefix]]] by simp
    then show ?thesis
      using idx-lt by simp
  qed
have append-nth:
  (ms @ [m]) ! (length ms - 1) = ms ! (length ms - 1)
  using len-ms by (simp add: nth-append)
  have last-ms: last ms = ms ! (length ms - 1)
    using ms-ne by (simp add: last-conv-nth)
  show ?thesis
    using ms'-nth append-nth last-ms by simp
qed
have head-ps: ps' ! (length ps - 1) = last ps
proof -

```

```

obtain  $q$  where  $len-ps$ :  $length\ ps = Suc\ q$ 
  using  $Suc.prem5(5)$  by ( $cases\ ps$ )  $auto$ 
have  $idx-lt$ :  $length\ ps - 1 < length\ (ps\ @\ [next-poly])$ 
  using  $Suc.prem5(5)$  by  $simp$ 
have  $ps'-nth$ :  $ps' ! (length\ ps - 1) =$ 
   $(ps\ @\ [next-poly]) ! (length\ ps - 1)$ 
proof –
  have  $(take\ (length\ (ps\ @\ [next-poly]))\ ps') ! (length\ ps - 1) =$ 
   $(ps\ @\ [next-poly]) ! (length\ ps - 1)$ 
  using  $conjunct1[OF\ prefix]$  by  $simp$ 
  then show  $?thesis$ 
    using  $idx-lt$  by  $simp$ 
qed
have  $append-nth$ :
   $(ps\ @\ [next-poly]) ! (length\ ps - 1) = ps ! (length\ ps - 1)$ 
  using  $len-ps$  by ( $simp\ add$ :  $nth-append$ )
have  $last-ps$ :  $last\ ps = ps ! (length\ ps - 1)$ 
  using  $Suc.prem5(5)$  by ( $simp\ add$ :  $last-conv-nth$ )
show  $?thesis$ 
  using  $ps'-nth\ append-nth\ last-ps$  by  $simp$ 
qed
have  $head-ds$ :  $ds' ! (length\ ps - 1) = last\ ds$ 
proof –
  have  $ds-ne$ :  $ds \neq []$ 
    using  $Suc.prem5(2,5)$  by  $auto$ 
obtain  $q$  where  $len-ds$ :  $length\ ds = Suc\ q$ 
  using  $ds-ne$  by ( $cases\ ds$ )  $auto$ 
have  $idx-lt$ :  $length\ ps - 1 < length\ (ds\ @\ [next-domain])$ 
  using  $Suc.prem5(2,5)$  by  $simp$ 
have  $ds'-nth$ :  $ds' ! (length\ ps - 1) =$ 
   $(ds\ @\ [next-domain]) ! (length\ ps - 1)$ 
proof –
  have  $(take\ (length\ (ds\ @\ [next-domain]))\ ds') ! (length\ ps - 1) =$ 
   $(ds\ @\ [next-domain]) ! (length\ ps - 1)$ 
  using  $conjunct1[OF\ conjunct2[OF\ prefix]]$  by  $simp$ 
  then show  $?thesis$ 
    using  $idx-lt$  by  $simp$ 
qed
have  $append-nth$ :
   $(ds\ @\ [next-domain]) ! (length\ ps - 1) = ds ! (length\ ds - 1)$ 
  using  $Suc.prem5(2)\ len-ds$  by ( $simp\ add$ :  $nth-append$ )
have  $last-ds$ :  $last\ ds = ds ! (length\ ds - 1)$ 
  using  $ds-ne$  by ( $simp\ add$ :  $last-conv-nth$ )
show  $?thesis$ 
  using  $ds'-nth\ append-nth\ last-ds$  by  $simp$ 
qed
have  $head-next-ps$ :  $ps' ! length\ ps = next-poly$ 
proof –
  have  $pref$ :  $take\ (Suc\ (length\ ps))\ ps' = ps\ @\ [next-poly]$ 

```



```

    using conjunct1[OF prefix] by simp
  have  $ps' ! \text{length } ps = (\text{take } (\text{Suc } (\text{length } ps)) \text{ } ps') ! \text{length } ps$ 
    by simp
  also have  $\dots = \text{next-poly}$ 
    using pref by simp
  finally show ?thesis .
qed
have head-next-ds:  $ds' ! \text{length } ps = \text{next-domain}$ 
proof -
  have pref:  $\text{take } (\text{Suc } (\text{length } ds)) \text{ } ds' = ds @ [\text{next-domain}]$ 
    using conjunct1[OF conjunct2[OF prefix]] Suc.premis(2) by simp
  have  $ds' ! \text{length } ps = (\text{take } (\text{Suc } (\text{length } ds)) \text{ } ds') ! \text{length } ps$ 
    using Suc.premis(2) by simp
  also have  $\dots = \text{next-domain}$ 
    using pref Suc.premis(2) by simp
  finally show ?thesis .
qed
have head-next-ls:  $ls' ! \text{length } ps = \text{next-layer}$ 
proof -
  have pref:  $\text{take } (\text{Suc } (\text{length } ls)) \text{ } ls' = ls @ [\text{next-layer}]$ 
    using conjunct2[OF conjunct2[OF prefix]] Suc.premis(2,3) by simp
  have  $ls' ! \text{length } ps = (\text{take } (\text{Suc } (\text{length } ls)) \text{ } ls') ! \text{length } ps$ 
    using Suc.premis(2,3) by simp
  also have  $\dots = \text{next-layer}$ 
    using pref Suc.premis(2,3) by simp
  finally show ?thesis .
qed
have layers:
   $\forall j < \text{Suc } n.$ 
   $p.\text{next-fri-layer}$ 
   $(ps' ! (\text{length } ps - 1 + j))$ 
   $(ds' ! (\text{length } ps - 1 + j))$ 
   $((b \# \text{challenges}) ! j) =$ 
   $(ps' ! (\text{length } ps + j),$ 
   $ds' ! (\text{length } ps + j),$ 
   $ls' ! (\text{length } ps + j))$ 
proof (intro allI impI)
  fix j
  assume j-bound:  $j < \text{Suc } n$ 
  show  $p.\text{next-fri-layer}$ 
     $(ps' ! (\text{length } ps - 1 + j))$ 
     $(ds' ! (\text{length } ps - 1 + j))$ 
     $((b \# \text{challenges}) ! j) =$ 
     $(ps' ! (\text{length } ps + j),$ 
     $ds' ! (\text{length } ps + j),$ 
     $ls' ! (\text{length } ps + j))$ 
  proof (cases j)
    case 0
    then show ?thesis

```

```

    using next-layer-eq head-ps head-ds head-next-ps head-next-ds head-next-ls
    by simp
next
case (Suc k)
then have k-bound:  $k < n$ 
    using j-bound by simp
have idx-current:
    length ps - 1 + Suc k = length ps + k
proof -
    obtain q where length ps = Suc q
    using Suc.prem5 by (cases ps) auto
    then show ?thesis by simp
qed
have tail-k:
    p.next-fri-layer
    (ps' ! (length ps + k))
    (ds' ! (length ps + k))
    (challenges ! k) =
    (ps' ! Suc (length ps + k),
     ds' ! Suc (length ps + k),
     ls' ! Suc (length ps + k))
    using tail-layers k-bound by simp
show ?thesis
    unfolding Suc
    apply (subst idx-current)
    apply (subst idx-current)
    using tail-k
    apply simp
done
qed
qed
have roots-aligned:
     $\forall j < \text{Suc } n.$ 
    ( $\text{?root} \# \text{roots}$ ) ! j = value (ms' ! (length ms - 1 + j))
proof (intro allI impI)
    fix j
    assume j-bound:  $j < \text{Suc } n$ 
    show ( $\text{?root} \# \text{roots}$ ) ! j = value (ms' ! (length ms - 1 + j))
    proof (cases j)
    case 0
    then show ?thesis
        using head-ms by simp
    next
    case (Suc k)
    then have k-bound:  $k < n$ 
        using j-bound by simp
    have idx-current:
        length ms - 1 + Suc k = length ms + k
    proof -

```

```

    obtain  $q$  where  $\text{length } ms = \text{Suc } q$ 
    using  $\text{Suc.prem}(2-5)$  by (cases  $ms$ ) auto
    then show ?thesis by simp
qed
have tail-k:
  roots !  $k = \text{value } (ms' ! (\text{length } ms + k))$ 
  using roots-tail  $k$ -bound by simp
show ?thesis
  unfolding Suc
  apply (subst idx-current)
  using tail-k
  apply simp
done
qed
qed
show ?case
  by (intro exI[where  $x=?\text{root} \# \text{roots}$ ] exI[where  $x=b \# \text{challenges}$ ])
    (use roots-len challenges-len tr-final st-final s-t lookups roots-aligned layers in simp)
qed

lemma fri-commit-initial-replay-successor-data:
  assumes outcome:
    Some  $((ps, ds, ls, ms), t) \in$ 
      set-dist (execute
         $(p.\text{fri-commit } n [cp] [d] [l] [m]) s$ )
  shows  $\exists \text{roots challenges.}$ 
    length roots =  $n \wedge$ 
    length challenges =  $n \wedge$ 
    PTranscript  $t = \text{rev roots} @ \text{PTranscript } s \wedge$ 
    PState  $t = \text{foldl concat } (PState s) \text{ roots} \wedge$ 
     $s \leq t \wedge$ 
     $(\forall i < \text{length roots.}$ 
      fmlookup (HashMap  $t$ ) (FiatShamirChallenge (foldl concat (PState  $s$ ) (take (Suc  $i$ ) roots)))
    =
      Some (challenges !  $i$ ))  $\wedge$ 
     $(\forall j < n. \text{roots} ! j = \text{value } (ms ! j)) \wedge$ 
     $(\forall j < n.$ 
       $p.\text{next-fri-layer}$ 
       $(ps ! j)$ 
       $(ds ! j)$ 
       $(challenges ! j) =$ 
       $(ps ! \text{Suc } j, ds ! \text{Suc } j, ls ! \text{Suc } j))$ 
proof -
  have data:
     $\exists \text{roots challenges.}$ 
    length roots =  $n \wedge$ 
    length challenges =  $n \wedge$ 
    PTranscript  $t = \text{rev roots} @ \text{PTranscript } s \wedge$ 
    PState  $t = \text{foldl concat } (PState s) \text{ roots} \wedge$ 

```

```

    s ≤ t ∧
    (∀ i < length roots.
      fmllookup (HashMap t) (FiatShamirChallenge (foldl concat (PState s) (take (Suc i) roots)))
    =
      Some (challenges ! i)) ∧
    (∀ j < n. roots ! j = value (ms ! (length [m] - 1 + j))) ∧
    (∀ j < n.
      p.next-fri-layer
      (ps ! (length [cp] - 1 + j))
      (ds ! (length [cp] - 1 + j))
      (challenges ! j) =
        (ps ! (length [cp] + j),
         ds ! (length [cp] + j),
         ls ! (length [cp] + j)))
    by (rule fri-commit-replay-successor-data[OF outcome]) simp-all
  from data obtain roots challenges where
    len-roots: length roots = n
    and len-challenges: length challenges = n
    and tr: PTranscript t = rev roots @ PTranscript s
    and st: PState t = foldl concat (PState s) roots
    and ext: s ≤ t
    and lookups:
      ∀ i < length roots.
        fmllookup (HashMap t) (FiatShamirChallenge (foldl concat (PState s) (take (Suc i) roots)))
  =
    Some (challenges ! i)
  and roots-aligned:
    ∀ j < n. roots ! j = value (ms ! (length [m] - 1 + j))
  and layers:
    ∀ j < n.
      p.next-fri-layer
      (ps ! (length [cp] - 1 + j))
      (ds ! (length [cp] - 1 + j))
      (challenges ! j) =
        (ps ! (length [cp] + j),
         ds ! (length [cp] + j),
         ls ! (length [cp] + j))
  by blast
  have roots-simple:
    ∀ j < n. roots ! j = value (ms ! j)
  using roots-aligned by simp
  have layers-simple:
    ∀ j < n.
      p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
        (ps ! Suc j, ds ! Suc j, ls ! Suc j)
  using layers by simp
  show ?thesis
  by (intro exI[where x=roots] exI[where x=challenges])
    (use len-roots len-challenges tr st ext lookups roots-simple layers-simple in simp)

```

qed

lemma *fri-commit-with-replay-successor-data:*

assumes *send-counter:*

$\bigwedge x \ s \ t. \text{Some } ((), t) \in \text{set-dist } (\text{execute } (p.\text{send } x) \ s) \implies$
 $\text{counter } t = \text{counter } s$

and *receiver-data:*

$\bigwedge b \ s \ t. \text{Some } (b, t) \in \text{set-dist } (\text{execute } \text{receive-challenge } s) \implies$
 $s \leq t \wedge$

$P\text{State } t = P\text{State } s \wedge$

$P\text{Transcript } t = P\text{Transcript } s \wedge$

$\text{counter } t = \text{Suc } (\text{counter } s) \wedge$

$\text{fmlookup } (\text{HashMap } t) (\text{tag } (\text{counter } s) (P\text{State } s)) = \text{Some } b$

and *create-counter:*

$\bigwedge xs \ m \ s \ t. \text{Some } (m, t) \in \text{set-dist } (\text{execute } (p.\text{create } xs) \ s) \implies$
 $\text{counter } t = \text{counter } s$

and *outcome:*

$\text{Some } ((ps', ds', ls', ms'), t) \in$

$\text{set-dist } (\text{execute } (p.\text{fri-commit-with } \text{receive-challenge } n \ ps \ ds \ ls \ ms) \ s)$

and *lens:*

$\text{length } ps = \text{length } ds$

$\text{length } ds = \text{length } ls$

$\text{length } ls = \text{length } ms$

and *nonempty:* $ps \neq []$

shows $\exists \text{roots challenges.}$

$\text{length roots} = n \wedge$

$\text{length challenges} = n \wedge$

$P\text{Transcript } t = \text{rev roots } @ \ P\text{Transcript } s \wedge$

$P\text{State } t = \text{foldl concat } (P\text{State } s) \ \text{roots} \wedge$

$s \leq t \wedge$

$\text{counter } t = \text{counter } s + n \wedge$

$(\forall i < \text{length roots.}$

$\text{fmlookup } (\text{HashMap } t)$

$(\text{tag } (\text{counter } s + i)$

$(\text{foldl concat } (P\text{State } s) \ (\text{take } (\text{Suc } i) \ \text{roots}))) =$

$\text{Some } (\text{challenges } ! \ i) \wedge$

$(\forall j < n.$

$\text{roots } ! \ j = \text{value } (ms' ! \ (\text{length } ms - 1 + j))) \wedge$

$(\forall j < n.$

$p.\text{next-fri-layer}$

$(ps' ! \ (\text{length } ps - 1 + j))$

$(ds' ! \ (\text{length } ps - 1 + j))$

$(\text{challenges } ! \ j) =$

$(ps' ! \ (\text{length } ps + j),$

$ds' ! \ (\text{length } ps + j),$

$ls' ! \ (\text{length } ps + j)))$

using *outcome lens nonempty*

proof (*induction* n *arbitrary:* $ps \ ds \ ls \ ms \ s \ ps' \ ds' \ ls' \ ms' \ t$)

case 0

```

then show ?case
  by (auto simp: p.hash-ext-refl)
next
case (Suc n)
from Suc.prem1(1) obtain s1 b s2 next-poly next-domain next-layer m s3 where
  send-root:
    Some (((), s1) ∈ set-dist (execute (p.send (value (last ms))) s)
  and rand:
    Some (b, s2) ∈ set-dist (execute receive-challenge s1)
  and next-layer-eq:
    p.next-fri-layer (last ps) (last ds) b =
      (next-poly, next-domain, next-layer)
  and create-m:
    Some (m, s3) ∈ set-dist (execute (p.create next-layer) s2)
  and rest:
    Some ((ps', ds', ls', ms'), t) ∈
      set-dist (execute
        (p.fri-commit-with receive-challenge n
          (ps @ [next-poly])
          (ds @ [next-domain])
          (ls @ [next-layer])
          (ms @ [m])) s3)
    by (auto simp: Let-def split: prod.splits elim!: p.set-dist-bindE)
let ?root = value (last ms)
have lens-tail:
  length (ps @ [next-poly]) = length (ds @ [next-domain])
  length (ds @ [next-domain]) = length (ls @ [next-layer])
  length (ls @ [next-layer]) = length (ms @ [m])
  using Suc.prem1(2-4) by simp-all
have nonempty-tail: ps @ [next-poly] ≠ []
  by simp
from Suc.IH[OF rest lens-tail nonempty-tail] obtain roots challenges where
  roots-len: length roots = n
  and challenges-len: length challenges = n
  and tr-t: PTranscript t = rev roots @ PTranscript s3
  and st-t: PState t = foldl concat (PState s3) roots
  and s3-t: s3 ≤ t
  and counter-t: counter t = counter s3 + n
  and lookups-tail:
    ∀ i < length roots.
      fmllookup (HashMap t)
        (tag (counter s3 + i)
          (foldl concat (PState s3) (take (Suc i) roots))) =
        Some (challenges ! i)
  and roots-tail:
    ∀ j < n.
      roots ! j =
        value (ms' ! (length (ms @ [m]) - 1 + j))
  and tail-layers:

```

```

   $\forall j < n.$ 
  p.next-fri-layer
    (ps' ! (length (ps @ [next-poly]) - 1 + j))
    (ds' ! (length (ps @ [next-poly]) - 1 + j))
    (challenges ! j) =
      (ps' ! (length (ps @ [next-poly]) + j),
       ds' ! (length (ps @ [next-poly]) + j),
       ls' ! (length (ps @ [next-poly]) + j))
  by blast
have send-res:
  s1 = s(PState := concat (PState s) ?root,
    PTranscript := ?root # PTranscript s)
  using p.send-outcome[OF send-root] .
have rand-data:
  s1 ≤ s2 ∧
  PState s2 = PState s1 ∧
  PTranscript s2 = PTranscript s1 ∧
  counter s2 = Suc (counter s1) ∧
  fmlookup (HashMap s2) (tag (counter s1) (PState s1)) = Some b
  using receiver-data[OF rand] .
have create-state: PState s3 = PState s2
  using create-preserves-state[OF create-m] .
have create-tr: PTranscript s3 = PTranscript s2
  using create-preserves-transcript[OF create-m] .
have counter-s1: counter s1 = counter s
  using send-counter[OF send-root] .
have counter-s2: counter s2 = Suc (counter s1)
  using rand-data by simp
have counter-s3: counter s3 = counter s2
  using create-counter[OF create-m] .
have counter-s3-s: counter s3 = Suc (counter s)
  using counter-s1 counter-s2 counter-s3 by simp
have counter-final: counter t = counter s + Suc n
  using counter-t counter-s3-s by simp
have s2-s3: s2 ≤ s3
  using create-hash-extends[OF create-m] .
have s-s1: s ≤ s1
  using send-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have s-t: s ≤ t
  using s-s1 rand-data s2-s3 s3-t by (meson p.hash-ext-trans)
have s2-t: s2 ≤ t
  using s2-s3 s3-t by (rule p.hash-ext-trans)
have lookup-head:
  fmlookup (HashMap t)
    (tag (counter s) (concat (PState s) ?root)) = Some b
proof -
  have lookup-s2:
    fmlookup (HashMap s2) (tag (counter s1) (PState s1)) = Some b
    using rand-data by simp

```

```

have fmlookup (HashMap t) (tag (counter s1) (PState s1)) = Some b
  using p.hash-extension-lookup[OF lookup-s2 s2-t] .
then show ?thesis
  using send-res counter-s1 by simp
qed
have tr-final:
  PTranscript t = rev (?root # roots) @ PTranscript s
  using tr-t create-tr rand-data send-res by simp
have st-final:
  PState t = foldl concat (PState s) (?root # roots)
  using st-t create-state rand-data send-res by simp
have lookups:
   $\forall i < \text{length } (?root \# roots).$ 
  fmlookup (HashMap t)
    (tag (counter s + i)
      (foldl concat (PState s) (take (Suc i) (?root # roots)))) =
    Some ((b # challenges) ! i)
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{length } (?root \# roots)$ 
  show fmlookup (HashMap t)
    (tag (counter s + i)
      (foldl concat (PState s) (take (Suc i) (?root # roots)))) =
    Some ((b # challenges) ! i)
proof (cases i)
  case 0
  then show ?thesis
    using lookup-head by simp
next
  case (Suc j)
  then have j-bound:  $j < \text{length } roots$ 
    using i-bound by simp
  have key-eq:
    foldl concat (PState s) (take (Suc (Suc j)) (?root # roots)) =
    foldl concat (PState s3) (take (Suc j) roots)
    using send-res rand-data create-state by simp
  show ?thesis
    using lookups-tail j-bound key-eq Suc counter-s3-s by simp
qed
qed
have prefix:
  take (length (ps @ [next-poly])) ps' = ps @ [next-poly]  $\wedge$ 
  take (length (ds @ [next-domain])) ds' = ds @ [next-domain]  $\wedge$ 
  take (length (ls @ [next-layer])) ls' = ls @ [next-layer]  $\wedge$ 
  take (length (ms @ [m])) ms' = ms @ [m]
  using p.fri-commit-with-preserves-prefix[OF rest] by simp
have head-ms: ms' ! (length ms - 1) = last ms
proof -
  have ms-ne: ms  $\neq []$ 

```



```

    using Suc.premis(2-5) by auto
  obtain q where len-ms: length ms = Suc q
    using ms-ne by (cases ms) auto
  have idx-lt: length ms - 1 < length (ms @ [m])
    using ms-ne by simp
  have ms'-nth: ms' ! (length ms - 1) =
    (ms @ [m]) ! (length ms - 1)
  proof -
    have (take (length (ms @ [m])) ms') ! (length ms - 1) =
      (ms @ [m]) ! (length ms - 1)
      using conjunct2[OF conjunct2[OF conjunct2[OF prefix]]] by simp
    then show ?thesis
      using idx-lt by simp
  qed
  have append-nth:
    (ms @ [m]) ! (length ms - 1) = ms ! (length ms - 1)
    using len-ms by (simp add: nth-append)
  have last-ms: last ms = ms ! (length ms - 1)
    using ms-ne by (simp add: last-conv-nth)
  show ?thesis
    using ms'-nth append-nth last-ms by simp
  qed
  have head-ps: ps' ! (length ps - 1) = last ps
  proof -
    obtain q where len-ps: length ps = Suc q
      using Suc.premis(5) by (cases ps) auto
    have idx-lt: length ps - 1 < length (ps @ [next-poly])
      using Suc.premis(5) by simp
    have ps'-nth: ps' ! (length ps - 1) =
      (ps @ [next-poly]) ! (length ps - 1)
    proof -
      have (take (length (ps @ [next-poly])) ps') ! (length ps - 1) =
        (ps @ [next-poly]) ! (length ps - 1)
        using conjunct1[OF prefix] by simp
      then show ?thesis
        using idx-lt by simp
    qed
    have append-nth:
      (ps @ [next-poly]) ! (length ps - 1) = ps ! (length ps - 1)
      using len-ps by (simp add: nth-append)
    have last-ps: last ps = ps ! (length ps - 1)
      using Suc.premis(5) by (simp add: last-conv-nth)
    show ?thesis
      using ps'-nth append-nth last-ps by simp
  qed
  have head-ds: ds' ! (length ps - 1) = last ds
  proof -
    have ds-ne: ds ≠ []
      using Suc.premis(2,5) by auto

```

```

obtain  $q$  where  $len\text{-}ds$ :  $length\ ds = Suc\ q$ 
  using  $ds\text{-}ne$  by ( $cases\ ds$ ) auto
have  $idx\text{-}lt$ :  $length\ ps - 1 < length\ (ds\ @\ [next\text{-}domain])$ 
  using  $Suc.prems(2,5)$  by simp
have  $ds'\text{-}nth$ :  $ds'!\ (length\ ps - 1) =$ 
   $(ds\ @\ [next\text{-}domain])!\ (length\ ps - 1)$ 
proof -
  have ( $take\ (length\ (ds\ @\ [next\text{-}domain]))\ ds^\wedge!\ (length\ ps - 1) =$ 
     $(ds\ @\ [next\text{-}domain])!\ (length\ ps - 1)$ )
    using  $conjunct1[OF\ conjunct2[OF\ prefix]]$  by simp
  then show ?thesis
    using  $idx\text{-}lt$  by simp
qed
have  $append\text{-}nth$ :
   $(ds\ @\ [next\text{-}domain])!\ (length\ ps - 1) = ds!\ (length\ ds - 1)$ 
  using  $Suc.prems(2)\ len\text{-}ds$  by (simp add: nth-append)
have  $last\text{-}ds$ :  $last\ ds = ds!\ (length\ ds - 1)$ 
  using  $ds\text{-}ne$  by (simp add: last-conv-nth)
show ?thesis
  using  $ds'\text{-}nth\ append\text{-}nth\ last\text{-}ds$  by simp
qed
have  $head\text{-}next\text{-}ps$ :  $ps'!\ length\ ps = next\text{-}poly$ 
proof -
  have  $pref$ :  $take\ (Suc\ (length\ ps))\ ps' = ps\ @\ [next\text{-}poly]$ 
    using  $conjunct1[OF\ prefix]$  by simp
  have  $ps'!\ length\ ps = (take\ (Suc\ (length\ ps))\ ps')!\ length\ ps$ 
    by simp
  also have  $\dots = next\text{-}poly$ 
    using  $pref$  by simp
  finally show ?thesis .
qed
have  $head\text{-}next\text{-}ds$ :  $ds'!\ length\ ps = next\text{-}domain$ 
proof -
  have  $pref$ :  $take\ (Suc\ (length\ ds))\ ds' = ds\ @\ [next\text{-}domain]$ 
    using  $conjunct1[OF\ conjunct2[OF\ prefix]]\ Suc.prems(2)$  by simp
  have  $ds'!\ length\ ps = (take\ (Suc\ (length\ ds))\ ds')!\ length\ ps$ 
    using  $Suc.prems(2)$  by simp
  also have  $\dots = next\text{-}domain$ 
    using  $pref\ Suc.prems(2)$  by simp
  finally show ?thesis .
qed
have  $head\text{-}next\text{-}ls$ :  $ls'!\ length\ ps = next\text{-}layer$ 
proof -
  have  $pref$ :  $take\ (Suc\ (length\ ls))\ ls' = ls\ @\ [next\text{-}layer]$ 
    using  $conjunct2[OF\ conjunct2[OF\ prefix]]\ Suc.prems(2,3)$  by simp
  have  $ls'!\ length\ ps = (take\ (Suc\ (length\ ls))\ ls')!\ length\ ps$ 
    using  $Suc.prems(2,3)$  by simp
  also have  $\dots = next\text{-}layer$ 
    using  $pref\ Suc.prems(2,3)$  by simp

```

```

    finally show ?thesis .
qed
have layers:
   $\forall j < \text{Suc } n.$ 
    p.next-fri-layer
      ( $ps' ! (\text{length } ps - 1 + j)$ )
      ( $ds' ! (\text{length } ps - 1 + j)$ )
      (( $b \# \text{challenges}$ ) !  $j$ ) =
        ( $ps' ! (\text{length } ps + j)$ ,
          $ds' ! (\text{length } ps + j)$ ,
          $ls' ! (\text{length } ps + j)$ )
proof (intro allI impI)
  fix j
  assume j-bound:  $j < \text{Suc } n$ 
  show p.next-fri-layer
    ( $ps' ! (\text{length } ps - 1 + j)$ )
    ( $ds' ! (\text{length } ps - 1 + j)$ )
    (( $b \# \text{challenges}$ ) !  $j$ ) =
      ( $ps' ! (\text{length } ps + j)$ ,
        $ds' ! (\text{length } ps + j)$ ,
        $ls' ! (\text{length } ps + j)$ )
  proof (cases j)
    case 0
    then show ?thesis
      using next-layer-eq head-ps head-ds head-next-ps head-next-ds head-next-ls
      by simp
  next
    case (Suc k)
    then have k-bound:  $k < n$ 
      using j-bound by simp
    have idx-current:
       $\text{length } ps - 1 + \text{Suc } k = \text{length } ps + k$ 
    proof -
      obtain q where  $\text{length } ps = \text{Suc } q$ 
      using Suc.prem5 by (cases ps) auto
      then show ?thesis by simp
    qed
    have tail-k:
      p.next-fri-layer
        ( $ps' ! (\text{length } ps + k)$ )
        ( $ds' ! (\text{length } ps + k)$ )
        ( $\text{challenges} ! k$ ) =
          ( $ps' ! \text{Suc } (\text{length } ps + k)$ ,
            $ds' ! \text{Suc } (\text{length } ps + k)$ ,
            $ls' ! \text{Suc } (\text{length } ps + k)$ )
      using tail-layers k-bound by simp
    show ?thesis
      unfolding Suc
      apply (subst idx-current)

```

```

    apply (subst idx-current)
    using tail-k
    apply simp
    done
  qed
qed
have roots-aligned:
   $\forall j < \text{Suc } n.$ 
   $(?root \# roots) ! j = \text{value } (ms' ! (\text{length } ms - 1 + j))$ 
proof (intro allI impI)
  fix j
  assume j-bound:  $j < \text{Suc } n$ 
  show  $(?root \# roots) ! j = \text{value } (ms' ! (\text{length } ms - 1 + j))$ 
  proof (cases j)
    case 0
    then show ?thesis
      using head-ms by simp
  next
    case (Suc k)
    then have k-bound:  $k < n$ 
      using j-bound by simp
    have idx-current:
       $\text{length } ms - 1 + \text{Suc } k = \text{length } ms + k$ 
    proof -
      obtain q where  $\text{length } ms = \text{Suc } q$ 
      using Suc.premis(2-5) by (cases ms) auto
      then show ?thesis by simp
    qed
    have tail-k:
       $roots ! k = \text{value } (ms' ! (\text{length } ms + k))$ 
      using roots-tail k-bound by simp
    show ?thesis
      unfolding Suc
      apply (subst idx-current)
      using tail-k
      apply simp
      done
  qed
qed
show ?case
proof (rule exI[where x=?root # roots])
  show  $\exists \text{ challenges.}$ 
     $\text{length } (?root \# roots) = \text{Suc } n \wedge$ 
     $\text{length } \text{challenges} = \text{Suc } n \wedge$ 
     $P\text{Transcript } t = \text{rev } (?root \# roots) @ P\text{Transcript } s \wedge$ 
     $P\text{State } t = \text{foldl concat } (P\text{State } s) (?root \# roots) \wedge$ 
     $s \leq t \wedge$ 
     $\text{counter } t = \text{counter } s + \text{Suc } n \wedge$ 
     $(\forall i < \text{length } (?root \# roots).$ 

```

```

    fmlookup (HashMap t)
      (tag (counter s + i)
        (foldl concat (PState s) (take (Suc i) (?root # roots)))) =
    Some (challenges ! i)) ∧
  (∀ j < Suc n.
    (?root # roots) ! j = value (ms' ! (length ms - 1 + j))) ∧
  (∀ j < Suc n.
    p.next-fri-layer
      (ps' ! (length ps - 1 + j))
      (ds' ! (length ps - 1 + j))
      (challenges ! j) =
      (ps' ! (length ps + j),
       ds' ! (length ps + j),
       ls' ! (length ps + j)))
proof (rule exI[where x=b # challenges])
show length (?root # roots) = Suc n ∧
  length (b # challenges) = Suc n ∧
  PTranscript t = rev (?root # roots) @ PTranscript s ∧
  PState t = foldl concat (PState s) (?root # roots) ∧
  s ≤ t ∧
  counter t = counter s + Suc n ∧
  (∀ i < length (?root # roots).
    fmlookup (HashMap t)
      (tag (counter s + i)
        (foldl concat (PState s) (take (Suc i) (?root # roots)))) =
    Some ((b # challenges) ! i)) ∧
  (∀ j < Suc n.
    (?root # roots) ! j = value (ms' ! (length ms - 1 + j))) ∧
  (∀ j < Suc n.
    p.next-fri-layer
      (ps' ! (length ps - 1 + j))
      (ds' ! (length ps - 1 + j))
      ((b # challenges) ! j) =
      (ps' ! (length ps + j),
       ds' ! (length ps + j),
       ls' ! (length ps + j)))
using roots-len challenges-len tr-final st-final s-t counter-final
  lookups roots-aligned layers
by simp
qed
qed
qed

```

lemma *fri-commit-with-preserves-counter*:

fixes counter :: 'f protocol-channel ⇒ nat

assumes send-counter:

$\bigwedge x s t. \text{Some } ((\cdot), t) \in \text{set-dist } (\text{execute } (p.\text{send } x) s) \implies$

counter t = counter s

and receiver-counter:

$\bigwedge b \ s \ t. \text{Some } (b, t) \in \text{set-dist } (\text{execute receive-challenge } s) \implies$
 $\text{counter } t = \text{counter } s$
and *create-counter*:
 $\bigwedge xs \ m \ s \ t. \text{Some } (m, t) \in \text{set-dist } (\text{execute } (p.\text{create } xs) \ s) \implies$
 $\text{counter } t = \text{counter } s$
and *outcome*:
 $\text{Some } ((ps', ds', ls', ms'), t) \in$
 $\text{set-dist } (\text{execute}$
 $\quad (p.\text{fri-commit-with receive-challenge } n \ ps \ ds \ ls \ ms) \ s)$
shows $\text{counter } t = \text{counter } s$
using *outcome*
proof (*induction* n *arbitrary*: $ps \ ds \ ls \ ms \ s \ ps' \ ds' \ ls' \ ms' \ t$)
case 0
then show *?case* **by** *simp*
next
case (*Suc* n)
from *Suc.prem*s **obtain** $s1 \ b \ s2 \ \text{next-poly} \ \text{next-domain} \ \text{next-layer} \ m \ s3$ **where**
 $\text{send-root}: \text{Some } ((), s1) \in \text{set-dist } (\text{execute } (p.\text{send } (\text{value } (\text{last } ms)))) \ s)$
and $\text{rand}: \text{Some } (b, s2) \in \text{set-dist } (\text{execute receive-challenge } s1)$
and *next-layer-eq*:
 $p.\text{next-fri-layer } (\text{last } ps) \ (\text{last } ds) \ b =$
 $(\text{next-poly}, \text{next-domain}, \text{next-layer})$
and *create-m*:
 $\text{Some } (m, s3) \in \text{set-dist } (\text{execute } (p.\text{create next-layer}) \ s2)$
and *rest*:
 $\text{Some } ((ps', ds', ls', ms'), t) \in$
 $\text{set-dist } (\text{execute}$
 $\quad (p.\text{fri-commit-with receive-challenge } n$
 $\quad (ps \ @ \ [\text{next-poly}])$
 $\quad (ds \ @ \ [\text{next-domain}])$
 $\quad (ls \ @ \ [\text{next-layer}])$
 $\quad (ms \ @ \ [m])) \ s3)$
by (*auto simp*: *Let-def split*: *prod.splits elim!*: *p.set-dist-bindE*)
have $c\text{-}s1: \text{counter } s1 = \text{counter } s$
by (*rule send-counter*[*OF send-root*])
have $c\text{-}s2: \text{counter } s2 = \text{counter } s1$
by (*rule receiver-counter*[*OF rand*])
have $c\text{-}s3: \text{counter } s3 = \text{counter } s2$
by (*rule create-counter*[*OF create-m*])
have $c\text{-}t: \text{counter } t = \text{counter } s3$
by (*rule Suc.IH*[*OF rest*])
show *?case*
using $c\text{-}s1 \ c\text{-}s2 \ c\text{-}s3 \ c\text{-}t$ **by** *simp*
qed

lemma *trace-fri-commit-preserves-alpha-counter*:

assumes *outcome*:

$\text{Some } ((ps, ds, ls, ms), t) \in$

$\text{set-dist } (\text{execute } (p.\text{trace-fri-commit } n \ ps0 \ ds0 \ ls0 \ ms0) \ s)$

shows $P\text{AlphaCounter } t = P\text{AlphaCounter } s$

proof –

have *send-counter*:

$\bigwedge x s t. \text{Some } ((), t) \in \text{set-dist } (\text{execute } (p.\text{send } x) s) \implies$
 $P\text{AlphaCounter } t = P\text{AlphaCounter } s$

using *p.send-outcome* **by** *fastforce*

have *receiver-counter*:

$\bigwedge b s t. \text{Some } (b, t) \in \text{set-dist } (\text{execute } p.\text{receive-trace-fri-challenge } s) \implies$
 $P\text{AlphaCounter } t = P\text{AlphaCounter } s$

using *p.receive-trace-fri-challenge-counter-outcome* **by** *fastforce*

have *create-counter*:

$\bigwedge xs m s t. \text{Some } (m, t) \in \text{set-dist } (\text{execute } (p.\text{create } xs) s) \implies$
 $P\text{AlphaCounter } t = P\text{AlphaCounter } s$

by (rule *p.create-preserves-channel(5)*)

show *?thesis*

unfolding *p.trace-fri-commit-def*

by (rule *fri-commit-with-preserves-counter* [
where *counter*=*PAlphaCounter* **and** *receive-challenge*=*p.receive-trace-fri-challenge*,
OF send-counter receiver-counter create-counter
outcome[unfolded p.trace-fri-commit-def]])

qed

lemma *trace-fri-commit-preserves-composition-counter*:

assumes *outcome*:

$\text{Some } ((ps, ds, ls, ms), t) \in$
 $\text{set-dist } (\text{execute } (p.\text{trace-fri-commit } n ps0 ds0 ls0 ms0) s)$

shows $P\text{CompositionFriCounter } t = P\text{CompositionFriCounter } s$

proof –

have *send-counter*:

$\bigwedge x s t. \text{Some } ((), t) \in \text{set-dist } (\text{execute } (p.\text{send } x) s) \implies$
 $P\text{CompositionFriCounter } t = P\text{CompositionFriCounter } s$

using *p.send-outcome* **by** *fastforce*

have *receiver-counter*:

$\bigwedge b s t. \text{Some } (b, t) \in \text{set-dist } (\text{execute } p.\text{receive-trace-fri-challenge } s) \implies$
 $P\text{CompositionFriCounter } t = P\text{CompositionFriCounter } s$

using *p.receive-trace-fri-challenge-counter-outcome* **by** *fastforce*

have *create-counter*:

$\bigwedge xs m s t. \text{Some } (m, t) \in \text{set-dist } (\text{execute } (p.\text{create } xs) s) \implies$
 $P\text{CompositionFriCounter } t = P\text{CompositionFriCounter } s$

by (rule *p.create-preserves-channel(4)*)

show *?thesis*

unfolding *p.trace-fri-commit-def*

by (rule *fri-commit-with-preserves-counter* [
where *counter*=*PCompositionFriCounter* **and** *receive-challenge*=*p.receive-trace-fri-challenge*,
OF send-counter receiver-counter create-counter
outcome[unfolded p.trace-fri-commit-def]])

qed

lemma *fri-commit-preserves-query-counter*:

assumes *outcome*:
 $\text{Some } ((ps, ds, ls, ms), t) \in \text{set-dist } (\text{execute } (p.\text{fri-commit } n \ ps0 \ ds0 \ ls0 \ ms0) \ s)$
shows $P\text{QueryCounter } t = P\text{QueryCounter } s$
proof –
have *send-counter*:
 $\bigwedge x \ s \ t. \text{Some } ((), t) \in \text{set-dist } (\text{execute } (p.\text{send } x) \ s) \implies$
 $P\text{QueryCounter } t = P\text{QueryCounter } s$
using *p.send-outcome* **by** *fastforce*
have *receiver-counter*:
 $\bigwedge b \ s \ t. \text{Some } (b, t) \in \text{set-dist } (\text{execute } p.\text{receive-random-field-element } s) \implies$
 $P\text{QueryCounter } t = P\text{QueryCounter } s$
using *p.receive-random-field-element-counter-outcome* **by** *fastforce*
have *create-counter*:
 $\bigwedge xs \ m \ s \ t. \text{Some } (m, t) \in \text{set-dist } (\text{execute } (p.\text{create } xs) \ s) \implies$
 $P\text{QueryCounter } t = P\text{QueryCounter } s$
by (rule *p.create-preserves-channel(6)*)
have *outcome'*:
 $\text{Some } ((ps, ds, ls, ms), t) \in \text{set-dist } (\text{execute } (p.\text{fri-commit-with } p.\text{receive-random-field-element } n \ ps0 \ ds0 \ ls0 \ ms0) \ s)$
using *outcome p.fri-commit-with-receive-random-field-element* **by** *simp*
show *?thesis*
by (rule *fri-commit-with-preserves-counter* [
where *counter*=*PQueryCounter* **and** *receive-challenge*=*p.receive-random-field-element*,
OF send-counter receiver-counter create-counter outcome'])
qed

lemma *trace-fri-commit-preserves-query-counter*:

assumes *outcome*:
 $\text{Some } ((ps, ds, ls, ms), t) \in \text{set-dist } (\text{execute } (p.\text{trace-fri-commit } n \ ps0 \ ds0 \ ls0 \ ms0) \ s)$
shows $P\text{QueryCounter } t = P\text{QueryCounter } s$
proof –
have *send-counter*:
 $\bigwedge x \ s \ t. \text{Some } ((), t) \in \text{set-dist } (\text{execute } (p.\text{send } x) \ s) \implies$
 $P\text{QueryCounter } t = P\text{QueryCounter } s$
using *p.send-outcome* **by** *fastforce*
have *receiver-counter*:
 $\bigwedge b \ s \ t. \text{Some } (b, t) \in \text{set-dist } (\text{execute } p.\text{receive-trace-fri-challenge } s) \implies$
 $P\text{QueryCounter } t = P\text{QueryCounter } s$
using *p.receive-trace-fri-challenge-counter-outcome* **by** *fastforce*
have *create-counter*:
 $\bigwedge xs \ m \ s \ t. \text{Some } (m, t) \in \text{set-dist } (\text{execute } (p.\text{create } xs) \ s) \implies$
 $P\text{QueryCounter } t = P\text{QueryCounter } s$
by (rule *p.create-preserves-channel(6)*)
show *?thesis*
unfolding *p.trace-fri-commit-def*
by (rule *fri-commit-with-preserves-counter* [

where $\text{counter} = P\text{QueryCounter}$ **and** $\text{receive-challenge} = p.\text{receive-trace-fri-challenge}$,
 $OF \text{ send-counter receiver-counter create-counter}$
 $\text{outcome}[\text{unfolded } p.\text{trace-fri-commit-def}]]$
qed

lemma *composition-fri-commit-preserves-query-counter*:
assumes outcome :
 $\text{Some } ((ps, ds, ls, ms), t) \in$
 $\text{set-dist } (\text{execute } (p.\text{composition-fri-commit } n \text{ } ps0 \text{ } ds0 \text{ } ls0 \text{ } ms0) \text{ } s)$
shows $P\text{QueryCounter } t = P\text{QueryCounter } s$
proof –
have send-counter :
 $\bigwedge x \text{ } s \text{ } t. \text{Some } ((), t) \in \text{set-dist } (\text{execute } (p.\text{send } x) \text{ } s) \implies$
 $P\text{QueryCounter } t = P\text{QueryCounter } s$
using $p.\text{send-outcome}$ **by** *fastforce*
have receiver-counter :
 $\bigwedge b \text{ } s \text{ } t. \text{Some } (b, t) \in \text{set-dist } (\text{execute } p.\text{receive-composition-fri-challenge } s) \implies$
 $P\text{QueryCounter } t = P\text{QueryCounter } s$
using $p.\text{receive-composition-fri-challenge-counter-outcome}$ **by** *fastforce*
have create-counter :
 $\bigwedge x \text{ } m \text{ } s \text{ } t. \text{Some } (m, t) \in \text{set-dist } (\text{execute } (p.\text{create } xs) \text{ } s) \implies$
 $P\text{QueryCounter } t = P\text{QueryCounter } s$
by (rule $p.\text{create-preserves-channel}(6)$)
show *?thesis*
unfolding $p.\text{composition-fri-commit-def}$
by (rule $\text{fri-commit-with-preserves-counter}$ [
where $\text{counter} = P\text{QueryCounter}$ **and** $\text{receive-challenge} = p.\text{receive-composition-fri-challenge}$,
 $OF \text{ send-counter receiver-counter create-counter}$
 $\text{outcome}[\text{unfolded } p.\text{composition-fri-commit-def}]]$)
qed

lemma *trace-fri-commit-initial-replay-successor-data*:
assumes outcome :
 $\text{Some } ((ps, ds, ls, ms), t) \in$
 $\text{set-dist } (\text{execute}$
 $(p.\text{trace-fri-commit } n \text{ } [cp] \text{ } [d] \text{ } [l] \text{ } [m]) \text{ } s)$
shows $\exists \text{ roots challenges.}$
 $\text{length roots} = n \wedge$
 $\text{length challenges} = n \wedge$
 $P\text{Transcript } t = \text{rev roots } @ \text{ } P\text{Transcript } s \wedge$
 $P\text{State } t = \text{foldl concat } (P\text{State } s) \text{ roots } \wedge$
 $s \leq t \wedge$
 $(\forall i < \text{length roots.}$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{TraceFriChallenge } (P\text{TraceFriCounter } s + i)$
 $(\text{foldl concat } (P\text{State } s) (\text{take } (\text{Suc } i) \text{ roots}))) =$
 $\text{Some } (\text{challenges } ! i)) \wedge$
 $(\forall j < n. \text{roots } ! j = \text{value } (ms ! j)) \wedge$
 $(\forall j < n.$

```

    p.next-fri-layer
      (ps ! j)
      (ds ! j)
      (challenges ! j) =
        (ps ! Suc j, ds ! Suc j, ls ! Suc j))
  proof -
  have send-counter:
     $\bigwedge x s t. \text{Some } ((), t) \in \text{set-dist } (\text{execute } (p.\text{send } x) s) \implies$ 
      PTraceFriCounter t = PTraceFriCounter s
  proof -
  fix x s t
  assume send:  $\text{Some } ((), t) \in \text{set-dist } (\text{execute } (p.\text{send } x) s)$ 
  show PTraceFriCounter t = PTraceFriCounter s
    using p.send-outcome[OF send] by simp
  qed
  have receiver:
     $\bigwedge b s t. \text{Some } (b, t) \in$ 
      set-dist (execute p.receive-trace-fri-challenge s)  $\implies$ 
      s  $\leq$  t  $\wedge$ 
      PState t = PState s  $\wedge$ 
      PTranscript t = PTranscript s  $\wedge$ 
      PTraceFriCounter t = Suc (PTraceFriCounter s)  $\wedge$ 
      fmlookup (HashMap t)
        (TraceFriChallenge (PTraceFriCounter s) (PState s)) = Some b
  proof -
  fix b s t
  assume rand:
     $\text{Some } (b, t) \in \text{set-dist } (\text{execute } p.\text{receive-trace-fri-challenge } s)$ 
  show s  $\leq$  t  $\wedge$ 
    PState t = PState s  $\wedge$ 
    PTranscript t = PTranscript s  $\wedge$ 
    PTraceFriCounter t = Suc (PTraceFriCounter s)  $\wedge$ 
    fmlookup (HashMap t)
      (TraceFriChallenge (PTraceFriCounter s) (PState s)) = Some b
    using p.receive-trace-fri-challenge-outcome[OF rand]
      p.receive-trace-fri-challenge-counter-outcome[OF rand]
      by simp
  qed
  have create-counter:
     $\bigwedge xs m s t. \text{Some } (m, t) \in \text{set-dist } (\text{execute } (p.\text{create } xs) s) \implies$ 
      PTraceFriCounter t = PTraceFriCounter s
    by (rule p.create-preserves-channel(3))
  have data:
     $\exists \text{roots challenges.}$ 
    length roots = n  $\wedge$ 
    length challenges = n  $\wedge$ 
    PTranscript t = rev roots @ PTranscript s  $\wedge$ 
    PState t = foldl concat (PState s) roots  $\wedge$ 
    s  $\leq$  t  $\wedge$ 

```

```

PTraceFriCounter t = PTraceFriCounter s + n ∧
(∀ i < length roots.
  fmlookup (HashMap t)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (PState s) (take (Suc i) roots))) =
    Some (challenges ! i)) ∧
(∀ j < n. roots ! j = value (ms ! (length [m] - 1 + j))) ∧
(∀ j < n.
  p.next-fri-layer
    (ps ! (length [cp] - 1 + j))
    (ds ! (length [cp] - 1 + j))
    (challenges ! j) =
    (ps ! (length [cp] + j),
     ds ! (length [cp] + j),
     ls ! (length [cp] + j)))
unfolding p.trace-fri-commit-def
by (rule fri-commit-with-replay-successor-data
  [where counter=PTraceFriCounter and tag=TraceFriChallenge,
   OF send-counter receiver create-counter
   outcome[unfolded p.trace-fri-commit-def]]) simp-all
then show ?thesis by auto
qed

```

lemma composition-fri-commit-initial-replay-successor-data:

```

assumes outcome:
  Some ((ps, ds, ls, ms), t) ∈
    set-dist (execute
      (p.composition-fri-commit n [cp] [d] [l] [m]) s)
shows ∃ roots challenges.
  length roots = n ∧
  length challenges = n ∧
  PTranscript t = rev roots @ PTranscript s ∧
  PState t = foldl concat (PState s) roots ∧
  s ≤ t ∧
  (∀ i < length roots.
    fmlookup (HashMap t)
      (CompositionFriChallenge (PCompositionFriCounter s + i)
        (foldl concat (PState s) (take (Suc i) roots))) =
        Some (challenges ! i)) ∧
  (∀ j < n. roots ! j = value (ms ! j)) ∧
  (∀ j < n.
    p.next-fri-layer
      (ps ! j)
      (ds ! j)
      (challenges ! j) =
      (ps ! Suc j, ds ! Suc j, ls ! Suc j))

```

proof –

```

have send-counter:
  ∧x s t. Some ((), t) ∈ set-dist (execute (p.send x) s) ⇒

```

```

    PCompositionFriCounter t = PCompositionFriCounter s
proof –
  fix x s t
  assume send: Some ((), t) ∈ set-dist (execute (p.send x) s)
  show PCompositionFriCounter t = PCompositionFriCounter s
    using p.send-outcome[OF send] by simp
qed
have receiver:
   $\bigwedge b\ s\ t.$  Some (b, t) ∈
    set-dist (execute p.receive-composition-fri-challenge s)  $\implies$ 
    s ≤ t ∧
    PState t = PState s ∧
    PTranscript t = PTranscript s ∧
    PCompositionFriCounter t = Suc (PCompositionFriCounter s) ∧
    fmlookup (HashMap t)
      (CompositionFriChallenge (PCompositionFriCounter s) (PState s)) = Some b
proof –
  fix b s t
  assume rand:
    Some (b, t) ∈
      set-dist (execute p.receive-composition-fri-challenge s)
  show s ≤ t ∧
    PState t = PState s ∧
    PTranscript t = PTranscript s ∧
    PCompositionFriCounter t = Suc (PCompositionFriCounter s) ∧
    fmlookup (HashMap t)
      (CompositionFriChallenge (PCompositionFriCounter s) (PState s)) = Some b
  using p.receive-composition-fri-challenge-outcome[OF rand]
    p.receive-composition-fri-challenge-counter-outcome[OF rand]
  by simp
qed
have create-counter:
   $\bigwedge xs\ m\ s\ t.$  Some (m, t) ∈ set-dist (execute (p.create xs) s)  $\implies$ 
    PCompositionFriCounter t = PCompositionFriCounter s
  by (rule p.create-preserves-channel(4))
have data:
   $\exists$  roots challenges.
    length roots = n ∧
    length challenges = n ∧
    PTranscript t = rev roots @ PTranscript s ∧
    PState t = foldl concat (PState s) roots ∧
    s ≤ t ∧
    PCompositionFriCounter t = PCompositionFriCounter s + n ∧
    ( $\forall i < \text{length roots}.$ 
      fmlookup (HashMap t)
        (CompositionFriChallenge (PCompositionFriCounter s + i)
          (foldl concat (PState s) (take (Suc i) roots))) =
        Some (challenges ! i)) ∧
    ( $\forall j < n.$  roots ! j = value (ms ! (length [m] - 1 + j))) ∧

```

```

  (∀ j < n.
    p.next-fri-layer
      (ps ! (length [cp] - 1 + j))
      (ds ! (length [cp] - 1 + j))
      (challenges ! j) =
        (ps ! (length [cp] + j),
         ds ! (length [cp] + j),
         ls ! (length [cp] + j)))
  unfolding p.composition-fri-commit-def
  by (rule fri-commit-with-replay-successor-data
    [where counter=PCompositionFriCounter and tag=CompositionFriChallenge,
      OF send-counter receiver create-counter
      outcome[unfolded p.composition-fri-commit-def]]) simp-all
  then show ?thesis by auto
qed

```

lemma *fri-commit-initial-heads*:

```

  assumes outcome:
    Some ((ps, ds, ls, ms), t) ∈
      set-dist (execute
        (p.fri-commit n [cp] [d] [l] [m]) s)
  shows
    ps ! 0 = cp
    ds ! 0 = d
    ls ! 0 = l
    ms ! 0 = m

```

proof –

```

  have lens:
    length ps = Suc n
    length ds = Suc n
    length ls = Suc n
    length ms = Suc n
  using p.fri-commit-lengths[OF outcome] by simp-all
  have prefixes:
    take (length [cp]) ps = [cp]
    take (length [d]) ds = [d]
    take (length [l]) ls = [l]
    take (length [m]) ms = [m]
  using p.fri-commit-preserves-prefix[OF outcome] by simp-all
  show ps ! 0 = cp
    using lens prefixes by (cases ps) auto
  show ds ! 0 = d
    using lens prefixes by (cases ds) auto
  show ls ! 0 = l
    using lens prefixes by (cases ls) auto
  show ms ! 0 = m
    using lens prefixes by (cases ms) auto
qed

```

lemma *trace-fri-commit-initial-heads*:

assumes *outcome*:

Some $((ps, ds, ls, ms), t) \in$
set-dist (*execute*
 $(p.trace-fri-commit\ n\ [cp]\ [d]\ [l]\ [m])\ s)$

shows

$ps ! 0 = cp$
 $ds ! 0 = d$
 $ls ! 0 = l$
 $ms ! 0 = m$

proof –

have *lens*:

$length\ ps = Suc\ n$
 $length\ ds = Suc\ n$
 $length\ ls = Suc\ n$
 $length\ ms = Suc\ n$

using *p.trace-fri-commit-lengths*[*OF outcome*] **by** *simp-all*

have *prefixes*:

$take\ (length\ [cp])\ ps = [cp]$
 $take\ (length\ [d])\ ds = [d]$
 $take\ (length\ [l])\ ls = [l]$
 $take\ (length\ [m])\ ms = [m]$

using *p.trace-fri-commit-preserves-prefix*[*OF outcome*] **by** *simp-all*

show $ps ! 0 = cp$

using *lens prefixes* **by** (*cases ps*) *auto*

show $ds ! 0 = d$

using *lens prefixes* **by** (*cases ds*) *auto*

show $ls ! 0 = l$

using *lens prefixes* **by** (*cases ls*) *auto*

show $ms ! 0 = m$

using *lens prefixes* **by** (*cases ms*) *auto*

qed

lemma *composition-fri-commit-initial-heads*:

assumes *outcome*:

Some $((ps, ds, ls, ms), t) \in$
set-dist (*execute*
 $(p.composition-fri-commit\ n\ [cp]\ [d]\ [l]\ [m])\ s)$

shows

$ps ! 0 = cp$
 $ds ! 0 = d$
 $ls ! 0 = l$
 $ms ! 0 = m$

proof –

have *lens*:

$length\ ps = Suc\ n$
 $length\ ds = Suc\ n$
 $length\ ls = Suc\ n$
 $length\ ms = Suc\ n$

```

using p.composition-fri-commit-lengths[OF outcome] by simp-all
have prefixes:
  take (length [cp]) ps = [cp]
  take (length [d]) ds = [d]
  take (length [l]) ls = [l]
  take (length [m]) ms = [m]
using p.composition-fri-commit-preserves-prefix[OF outcome] by simp-all
show ps ! 0 = cp
  using lens prefixes by (cases ps) auto
show ds ! 0 = d
  using lens prefixes by (cases ds) auto
show ls ! 0 = l
  using lens prefixes by (cases ls) auto
show ms ! 0 = m
  using lens prefixes by (cases ms) auto
qed

```

definition *fri-commit-outcome* **where**

```

fri-commit-outcome n cp d l m ps ds ls ms s t  $\longleftrightarrow$ 
  Some ((ps, ds, ls, ms), t)  $\in$ 
    set-dist (execute (p.fri-commit n [cp] [d] [l] [m]) s)  $\vee$ 
  Some ((ps, ds, ls, ms), t)  $\in$ 
    set-dist (execute (p.trace-fri-commit n [cp] [d] [l] [m]) s)  $\vee$ 
  Some ((ps, ds, ls, ms), t)  $\in$ 
    set-dist (execute (p.composition-fri-commit n [cp] [d] [l] [m]) s)

```

lemma *fri-commit-outcomeI*:

```

assumes Some ((ps, ds, ls, ms), t)  $\in$ 
  set-dist (execute (p.fri-commit n [cp] [d] [l] [m]) s)
shows fri-commit-outcome n cp d l m ps ds ls ms s t
using assms unfolding fri-commit-outcome-def by blast

```

lemma *trace-fri-commit-outcomeI*:

```

assumes Some ((ps, ds, ls, ms), t)  $\in$ 
  set-dist (execute (p.trace-fri-commit n [cp] [d] [l] [m]) s)
shows fri-commit-outcome n cp d l m ps ds ls ms s t
using assms unfolding fri-commit-outcome-def by blast

```

lemma *composition-fri-commit-outcomeI*:

```

assumes Some ((ps, ds, ls, ms), t)  $\in$ 
  set-dist (execute (p.composition-fri-commit n [cp] [d] [l] [m]) s)
shows fri-commit-outcome n cp d l m ps ds ls ms s t
using assms unfolding fri-commit-outcome-def by blast

```

lemma *fri-commit-outcome-initial-heads*:

```

assumes fri-commit-outcome n cp d l m ps ds ls ms s t
shows
  ps ! 0 = cp
  ds ! 0 = d

```

```

  ls ! 0 = l
  ms ! 0 = m
using assms
unfolding fri-commit-outcome-def
by (auto dest: fri-commit-initial-heads trace-fri-commit-initial-heads
      composition-fri-commit-initial-heads)

lemma decommit-on-query-step-transcript-extends:
  assumes step-in: m ∈ set (p.decommit-on-query idx f-merkle)
  and outcome: Some (x, t) ∈ set-dist (execute m s)
  shows transcript-extends t s
proof –
  from step-in obtain i where
    m-eq:
      m = do {
        p.send (p.f-eval ! i);
        mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle)
      }
    unfolding p.decommit-on-query-def by auto
  from outcome[unfolded m-eq] obtain u where
    send-leaf: Some ((), u) ∈ set-dist (execute (p.send (p.f-eval ! i)) s)
    and send-path:
      Some (x, t) ∈
        set-dist (execute
          (mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle)) u)
    by (auto elim!: p.set-dist-bindE)
  have s-u: transcript-extends u s
    using send-transcript-extends[OF send-leaf] .
  have x-unit: x = ()
    by (cases x) simp
  have path-out:
    Some ((), t) ∈
      set-dist (execute
        (mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle)) u)
    using send-path x-unit by simp
  have u-t: transcript-extends t u
    by (rule mfold2-send-transcript-extends[OF path-out])
  show ?thesis
    using s-u u-t by (rule transcript-extends-trans)
qed

```

```

lemma decommit-on-query-step-hash-extends:
  assumes step-in: m ∈ set (p.decommit-on-query idx f-merkle)
  and outcome: Some (x, t) ∈ set-dist (execute m s)
  shows s ≤ t
proof –
  from step-in obtain i where
    m-eq:
      m = do {

```



```

    p.send (p.f-eval ! i);
    mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle)
  }
  unfolding p.decommit-on-query-def by auto
from outcome[unfolded m-eq] obtain u where
  send-leaf: Some ((), u) ∈ set-dist (execute (p.send (p.f-eval ! i)) s)
and send-path:
  Some (x, t) ∈
    set-dist (execute
      (mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle)) u)
  by (auto elim!: p.set-dist-bindE)
have s-u: s ≤ u
  using send-hash-extends[OF send-leaf] .
have x-unit: x = ()
  by (cases x) simp
have path-out:
  Some ((), t) ∈
    set-dist (execute
      (mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle)) u)
  using send-path x-unit by simp
have u-t: u ≤ t
  using mfold2-send-hash-extends[OF path-out] .
show ?thesis
  using s-u u-t by (rule p.hash-ext-trans)
qed

```

lemma *decommit-on-query-step-preserves-query-counter*:

assumes *step-in*: $m \in \text{set } (p.\text{decommit-on-query } \text{idx } f\text{-merkle})$

and *outcome*: $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s)$

shows $P\text{QueryCounter } t = P\text{QueryCounter } s$

proof –

from *step-in* **obtain** *i* **where**

m-eq:

```

  m = do {
    p.send (p.f-eval ! i);
    mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle)
  }

```

unfolding *p.decommit-on-query-def* **by** *auto*

from *outcome[unfolded m-eq]* **obtain** *u* **where**

send-leaf: $\text{Some } ((), u) \in \text{set-dist } (\text{execute } (p.\text{send } (p.f\text{-eval } ! \ i)) \ s)$

and *send-path*:

```

  Some (x, t) ∈
    set-dist (execute
      (mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle)) u)

```

by *(auto elim!: p.set-dist-bindE)*

have *x-unit*: $x = ()$

by *(cases x) simp*

have *path-out*:

```

  Some ((), t) ∈

```

```

    set-dist (execute
      (mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle)) u)
  using send-path x-unit by simp
have q-u: PQueryCounter u = PQueryCounter s
  using p.send-outcome[OF send-leaf] by simp
have q-t: PQueryCounter t = PQueryCounter u
  by (rule mfold2-send-preserves-query-counter[OF path-out])
show ?thesis
  using q-u q-t by simp
qed

lemma decommit-on-fri-layers-step-transcript-extends:
  assumes step-in:  $m \in \text{set } (p.\text{decommit-on-fri-layers } fs)$ 
    and outcome:  $\text{Some } (x, t) \in \text{set-dist } (\text{execute } (m \text{ idx}) s)$ 
  shows transcript-extends t s
proof -
  from step-in obtain l tree where
    pair-in:  $(l, \text{tree}) \in \text{set } fs$ 
  and m-eq:
     $m = (\lambda \text{idx}. \text{do } \{$ 
      let len = length l;
      let idx' = idx mod len;
      let sidx = (idx' + (len div 2)) mod len;
      p.send (l ! idx');
      mfold2 p.send (get-authentication-path len idx' tree);
      p.send (l ! sidx);
      mfold2 p.send (get-authentication-path len sidx tree);
      return idx'
    })
  unfolding p.decommit-on-fri-layers-def by auto
let ?len = length l
let ?idx' = idx mod ?len
let ?sidx = (?idx' + (?len div 2)) mod ?len
from outcome[unfolded m-eq Let-def] obtain u1 u2 u3 u4 where
  send-xp:  $\text{Some } ((), u1) \in \text{set-dist } (\text{execute } (p.\text{send } (l ! ?idx')) s)$ 
  and path-xp:
     $\text{Some } ((), u2) \in$ 
     $\text{set-dist } (\text{execute } (mfold2 p.\text{send } (get-authentication-path ?len ?idx' \text{tree})) u1)$ 
  and send-xn:  $\text{Some } ((), u3) \in \text{set-dist } (\text{execute } (p.\text{send } (l ! ?sidx)) u2)$ 
  and path-xn:
     $\text{Some } ((), u4) \in$ 
     $\text{set-dist } (\text{execute } (mfold2 p.\text{send } (get-authentication-path ?len ?sidx \text{tree})) u3)$ 
  and ret:  $\text{Some } (x, t) \in \text{set-dist } (\text{execute } (\text{return } ?idx') u4)$ 
  by (auto elim!: p.set-dist-bindE)
have s-u1: transcript-extends u1 s
  using send-transcript-extends[OF send-xp] .
have u1-u2: transcript-extends u2 u1
  using mfold2-send-transcript-extends[OF path-xp] .
have u2-u3: transcript-extends u3 u2

```

```

    using send-transcript-extends[OF send-xn] .
  have u3-u4: transcript-extends u4 u3
    using mfold2-send-transcript-extends[OF path-xn] .
  have u4-t: transcript-extends t u4
    using ret by simp
  show ?thesis
    using s-u1 u1-u2 u2-u3 u3-u4 u4-t by (meson transcript-extends-trans)
qed

```

lemma *decommit-on-fri-layers-step-hash-extends:*

```

  assumes step-in:  $m \in \text{set } (p.\text{decommit-on-fri-layers } fs)$ 
    and outcome:  $\text{Some } (x, t) \in \text{set-dist } (\text{execute } (m \text{ } idx) \text{ } s)$ 
  shows  $s \leq t$ 

```

proof –

from *step-in* **obtain** l *tree* **where**

pair-in: $(l, \text{tree}) \in \text{set } fs$

and *m-eq*:

```

  m = ( $\lambda idx.$  do {
    let len = length l;
    let idx' = idx mod len;
    let sidx = (idx' + (len div 2)) mod len;
    p.send (l ! idx');
    mfold2 p.send (get-authentication-path len idx' tree);
    p.send (l ! sidx);
    mfold2 p.send (get-authentication-path len sidx tree);
    return idx'
  })

```

unfolding *p.decommit-on-fri-layers-def* **by** *auto*

let $?len = \text{length } l$

let $?idx' = idx \text{ mod } ?len$

let $?sidx = (?idx' + (?len \text{ div } 2)) \text{ mod } ?len$

from *outcome[unfolded m-eq Let-def]* **obtain** $u1 \ u2 \ u3 \ u4$ **where**

send-xp: $\text{Some } ((), u1) \in \text{set-dist } (\text{execute } (p.\text{send } (l ! ?idx')) \text{ } s)$

and *path-xp*:

$\text{Some } ((), u2) \in$

$\text{set-dist } (\text{execute } (mfold2 \text{ } p.\text{send } (get\text{-authentication-path } ?len \text{ } ?idx' \text{ tree})) \text{ } u1)$

and *send-xn*: $\text{Some } ((), u3) \in \text{set-dist } (\text{execute } (p.\text{send } (l ! ?sidx)) \text{ } u2)$

and *path-xn*:

$\text{Some } ((), u4) \in$

$\text{set-dist } (\text{execute } (mfold2 \text{ } p.\text{send } (get\text{-authentication-path } ?len \text{ } ?sidx \text{ tree})) \text{ } u3)$

and *ret*: $\text{Some } (x, t) \in \text{set-dist } (\text{execute } (\text{return } ?idx') \text{ } u4)$

by (*auto elim!:* *p.set-dist-bindE*)

have $s-u1: s \leq u1$

using *send-hash-extends*[*OF send-xp*] .

have $u1-u2: u1 \leq u2$

using *mfold2-send-hash-extends*[*OF path-xp*] .

have $u2-u3: u2 \leq u3$

using *send-hash-extends*[*OF send-xn*] .

have $u3-u4: u3 \leq u4$

```

    using mfold2-send-hash-extends[OF path-xn] .
  have u4-t:  $u_4 \leq t$ 
    using ret unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  show ?thesis
    using s-u1 u1-u2 u2-u3 u3-u4 u4-t by (meson p.hash-ext-trans)
qed

```

lemma *decommit-on-fri-layers-step-preserves-query-counter*:

assumes *step-in*: $m \in \text{set } (p.\text{decommit-on-fri-layers } fs)$
and *outcome*: $\text{Some } (x, t) \in \text{set-dist } (\text{execute } (m \text{ idx}) s)$
shows $P\text{QueryCounter } t = P\text{QueryCounter } s$

proof –

from *step-in* **obtain** l **tree** **where**

m-eq:

```

  m = ( $\lambda \text{idx}.$  do {
    let len = length l;
    let idx' = idx mod len;
    let sidx = (idx' + (len div 2)) mod len;
    p.send (l ! idx');
    mfold2 p.send (get-authentication-path len idx' tree);
    p.send (l ! sidx);
    mfold2 p.send (get-authentication-path len sidx tree);
    return idx'
  })

```

unfolding *p.decommit-on-fri-layers-def* **by** *auto*

let $?len = \text{length } l$

let $?idx' = \text{idx mod } ?len$

let $?sidx = (?idx' + (?len \text{ div } 2)) \text{ mod } ?len$

from *outcome[unfolded m-eq Let-def]* **obtain** $u1 \ u2 \ u3 \ u4$ **where**

send-xp: $\text{Some } ((), u1) \in \text{set-dist } (\text{execute } (p.\text{send } (l ! ?idx')) s)$

and *path-xp*:

$\text{Some } ((), u2) \in \text{set-dist } (\text{execute } (mfold2 p.\text{send } (get-authentication-path ?len ?idx' tree)) u1)$

and *send-xn*: $\text{Some } ((), u3) \in \text{set-dist } (\text{execute } (p.\text{send } (l ! ?sidx)) u2)$

and *path-xn*:

$\text{Some } ((), u4) \in \text{set-dist } (\text{execute } (mfold2 p.\text{send } (get-authentication-path ?len ?sidx tree)) u3)$

and *ret*: $\text{Some } (x, t) \in \text{set-dist } (\text{execute } (\text{return } ?idx') u4)$

by (*auto elim!*: *p.set-dist-bindE*)

have $q\text{-}u1: P\text{QueryCounter } u1 = P\text{QueryCounter } s$

using *p.send-outcome[OF send-xp]* **by** *simp*

have $q\text{-}u2: P\text{QueryCounter } u2 = P\text{QueryCounter } u1$

by (*rule mfold2-send-preserves-query-counter[OF path-xp]*)

have $q\text{-}u3: P\text{QueryCounter } u3 = P\text{QueryCounter } u2$

using *p.send-outcome[OF send-xn]* **by** *simp*

have $q\text{-}u4: P\text{QueryCounter } u4 = P\text{QueryCounter } u3$

by (*rule mfold2-send-preserves-query-counter[OF path-xn]*)

have *t-eq*: $t = u4$

using *ret* **by** *simp*

```

show ?thesis
  using q-u1 q-u2 q-u3 q-u4 unfolding t-eq by simp
qed

lemma prover-query-round-hash-extends:
  assumes outcome:
    Some (x, t) ∈ set-dist (execute (prover-query-round f-merkle f-ls f-ms ls ms) s)
  shows s ≤ t
proof –
  from outcome obtain idx s1 qouts s2 f-fri-idx s3 fri-idx where
    rand: Some (idx, s1) ∈ set-dist (execute p.receive-query-index-challenge s)
  and query:
    Some (qouts, s2) ∈
      set-dist (execute (mmap (p.decommit-on-query (p.index (to-nat idx)) f-merkle)) s1)
  and trace-fri:
    Some (f-fri-idx, s3) ∈
      set-dist (execute
        (mfold (p.index (to-nat idx))
          (p.decommit-on-fri-layers (butlast (zip f-ls f-ms)))) s2)
  and fri:
    Some (fri-idx, t) ∈
      set-dist (execute
        (mfold (p.index (to-nat idx))
          (p.decommit-on-fri-layers (butlast (zip ls ms)))) s3)
  unfolding prover-query-round-def
  by (auto elim!: p.set-dist-bindE)
  have s-s1: s ≤ s1
    using p.receive-query-index-challenge-extends[OF rand] .
  have s1-s2: s1 ≤ s2
    by (rule mmap-hash-extends[OF query])
      (rule decommit-on-query-step-hash-extends)
  have s2-s3: s2 ≤ s3
    by (rule mfold-hash-extends[OF trace-fri])
      (rule decommit-on-fri-layers-step-hash-extends)
  have s3-t: s3 ≤ t
    by (rule mfold-hash-extends[OF fri])
      (rule decommit-on-fri-layers-step-hash-extends)
  show ?thesis
    using s-s1 s1-s2 s2-s3 s3-t by (meson p.hash-ext-trans)
qed

```

```

lemma prover-query-round-query-counter:
  assumes outcome:
    Some (x, t) ∈ set-dist (execute (prover-query-round f-merkle f-ls f-ms ls ms) s)
  shows PQueryCounter t = Suc (PQueryCounter s)
proof –
  from outcome obtain idx s1 qouts s2 f-fri-idx s3 fri-idx where
    rand: Some (idx, s1) ∈ set-dist (execute p.receive-query-index-challenge s)
  and query:

```

```

    Some (qouts, s2) ∈
      set-dist (execute (mmap (p.decommit-on-query (p.index (to-nat idx)) f-merkle)) s1)
  and trace-fri:
    Some (f-fri-idx, s3) ∈
      set-dist (execute
        (mfold (p.index (to-nat idx))
          (p.decommit-on-fri-layers (butlast (zip f-ls f-ms))))) s2)
  and fri:
    Some (fri-idx, t) ∈
      set-dist (execute
        (mfold (p.index (to-nat idx))
          (p.decommit-on-fri-layers (butlast (zip ls ms))))) s3)
  unfolding prover-query-round-def
  by (auto elim!: p.set-dist-bindE)
have q-s1: PQueryCounter s1 = Suc (PQueryCounter s)
  using p.receive-query-index-challenge-counter-outcome[OF rand] by simp
have q-s2: PQueryCounter s2 = PQueryCounter s1
  by (rule mmap-preserves-query-counter[OF query])
  (rule decommit-on-query-step-preserves-query-counter)
have q-s3: PQueryCounter s3 = PQueryCounter s2
  by (rule mfold-preserves-query-counter[OF trace-fri])
  (rule decommit-on-fri-layers-step-preserves-query-counter)
have q-t: PQueryCounter t = PQueryCounter s3
  by (rule mfold-preserves-query-counter[OF fri])
  (rule decommit-on-fri-layers-step-preserves-query-counter)
show ?thesis
  using q-s1 q-s2 q-s3 q-t by simp
qed

```

lemma prover-query-round-transcript-extends:

assumes outcome:

Some (x, t) ∈ set-dist (execute (prover-query-round f-merkle f-ls f-ms ls ms) s)

shows transcript-extends t s

proof –

from outcome **obtain** idx s1 qouts s2 f-fri-idx s3 fri-idx **where**

rand: Some (idx, s1) ∈ set-dist (execute p.receive-query-index-challenge s)

and query:

Some (qouts, s2) ∈
 set-dist (execute (mmap (p.decommit-on-query (p.index (to-nat idx)) f-merkle)) s1)

and trace-fri:

Some (f-fri-idx, s3) ∈
 set-dist (execute
 (mfold (p.index (to-nat idx))
 (p.decommit-on-fri-layers (butlast (zip f-ls f-ms))))) s2)

and fri:

Some (fri-idx, t) ∈
 set-dist (execute
 (mfold (p.index (to-nat idx))
 (p.decommit-on-fri-layers (butlast (zip ls ms))))) s3)

```

unfolding prover-query-round-def
  by (auto elim!: p.set-dist-bindE)
have s-s1: transcript-extends s1 s
  using receive-query-index-challenge-transcript-extends[OF rand] .
have s1-s2: transcript-extends s2 s1
  by (rule mmap-transcript-extends[OF query])
    (rule decommit-on-query-step-transcript-extends)
have s2-s3: transcript-extends s3 s2
  by (rule mfold-transcript-extends[OF trace-fri])
    (rule decommit-on-fri-layers-step-transcript-extends)
have s3-t: transcript-extends t s3
  by (rule mfold-transcript-extends[OF fri])
    (rule decommit-on-fri-layers-step-transcript-extends)
show ?thesis
  using s-s1 s1-s2 s2-s3 s3-t by (meson transcript-extends-trans)
qed

lemma prover-query-round-outcomeE:
assumes outcome:
  Some (x, t) ∈ set-dist (execute (prover-query-round f-merkle f-ls f-ms ls ms) s)
obtains idx s1 qouts s2 f-fri-idx s3 fri-idx
where
  Some (idx, s1) ∈ set-dist (execute p.receive-query-index-challenge s)
  Some (qouts, s2) ∈
    set-dist (execute
      (mmap (p.decommit-on-query (p.index (to-nat idx)) f-merkle)) s1)
  Some (f-fri-idx, s3) ∈
    set-dist (execute
      (mfold (p.index (to-nat idx))
        (p.decommit-on-fri-layers (butlast (zip f-ls f-ms)))) s2)
  Some (fri-idx, t) ∈
    set-dist (execute
      (mfold (p.index (to-nat idx))
        (p.decommit-on-fri-layers (butlast (zip ls ms)))) s3)
  x = ()
proof -
from outcome obtain idx s1 qouts s2 f-fri-idx s3 fri-idx s4 where
  rand: Some (idx, s1) ∈ set-dist (execute p.receive-query-index-challenge s)
and query:
  Some (qouts, s2) ∈
    set-dist (execute
      (mmap (p.decommit-on-query (p.index (to-nat idx)) f-merkle)) s1)
and trace-fri:
  Some (f-fri-idx, s3) ∈
    set-dist (execute
      (mfold (p.index (to-nat idx))
        (p.decommit-on-fri-layers (butlast (zip f-ls f-ms)))) s2)
and fri:
  Some (fri-idx, s4) ∈

```

```

    set-dist (execute
      (mfold (p.index (to-nat idx))
        (p.decommit-on-fri-layers (butlast (zip ls ms)))) s3)
  and ret: Some (x, t) ∈ set-dist (execute (return ()) s4)
  unfolding prover-query-round-def
  by (auto elim!: p.set-dist-bindE)
have x-unit: x = ()
  using ret by simp
have t-eq: t = s4
  using ret by simp
show ?thesis
  by (rule that[OF rand query trace-fri - x-unit])
    (use fri t-eq in simp)
qed

lemma random-send-round-transcript:
  assumes outcome:
    Some (a, t) ∈
      set-dist (execute
        (do {
          a ← p.receive-alpha-challenge;
          p.send a;
          return a
        }) s)
  shows PTranscript t = a # PTranscript s
proof -
  from outcome obtain u v where
    rand: Some (a, u) ∈ set-dist (execute p.receive-alpha-challenge s)
    and send-a: Some ((), v) ∈ set-dist (execute (p.send a) u)
    and ret: Some (a, t) ∈ set-dist (execute (return a) v)
    by (auto elim!: p.set-dist-bindE)
  have t-v: t = v
    using ret by simp
  have tr-u: PTranscript u = PTranscript s
    using receive-alpha-challenge-preserves-transcript[OF rand] .
  have v = u(PState := concat (PState u) a, PTranscript := a # PTranscript u)
    using p.send-outcome[OF send-a] .
  then show ?thesis
    using t-v tr-u by simp
qed

lemma random-send-round-parts-outcome:
  assumes rand: Some (a, u) ∈ set-dist (execute p.receive-alpha-challenge s)
    and send-a: Some ((), t) ∈ set-dist (execute (p.send a) u)
  shows
    s ≤ t ∧
    PState t = concat (PState s) a ∧
    PTranscript t = a # PTranscript s ∧
    PTraceFriCounter t = PTraceFriCounter s ∧

```


$PCompositionFriCounter\ t = PCompositionFriCounter\ s \wedge$
 $PAlphaCounter\ t = Suc\ (PAlphaCounter\ s) \wedge$
 $PQueryCounter\ t = PQueryCounter\ s \wedge$
 $fmlookup\ (HashMap\ t)\ (AlphaChallenge\ (PAlphaCounter\ s)\ (PState\ s)) = Some\ a$
proof –
have *rand-out*:
 $s \leq u \wedge$
 $PState\ u = PState\ s \wedge$
 $PTranscript\ u = PTranscript\ s \wedge$
 $fmlookup\ (HashMap\ u)\ (AlphaChallenge\ (PAlphaCounter\ s)\ (PState\ s)) = Some\ a$
by (rule *p.receive-alpha-challenge-outcome*[*OF rand*])
have *s-u*: $s \leq u$
using *rand-out* **by** *simp*
have *lookup-u*: $fmlookup\ (HashMap\ u)\ (AlphaChallenge\ (PAlphaCounter\ s)\ (PState\ s)) = Some\ a$
using *rand-out* **by** *simp*
have *state-u*: $PState\ u = PState\ s$
using *rand-out* **by** *simp*
have *tr-u*: $PTranscript\ u = PTranscript\ s$
using *rand-out* **by** *simp*
have *alpha-counter-u*: $PAlphaCounter\ u = Suc\ (PAlphaCounter\ s)$
using *p.receive-alpha-challenge-counter-outcome*[*OF rand*] **by** *simp*
have *other-counters-u*:
 $PTraceFriCounter\ u = PTraceFriCounter\ s \wedge$
 $PCompositionFriCounter\ u = PCompositionFriCounter\ s \wedge$
 $PQueryCounter\ u = PQueryCounter\ s$
using *p.receive-alpha-challenge-counter-outcome*[*OF rand*] **by** *simp*
have *t-eq*: $t = u[PState := concat\ (PState\ u)\ a,\ PTranscript := a \# PTranscript\ u]$
using *p.send-outcome*[*OF send-a*] .
have *u-t*: $u \leq t$
unfolding *t-eq* *less-eq-hash-ext-def* *less-eq-fmap-def* **by** *simp*
have *s-t*: $s \leq t$
using *s-u* *u-t* **by** (rule *p.hash-ext-trans*)
have *lookup-t*: $fmlookup\ (HashMap\ t)\ (AlphaChallenge\ (PAlphaCounter\ s)\ (PState\ s)) = Some\ a$
using *p.hash-extension-lookup*[*OF lookup-u* *u-t*] .
show *?thesis*
using *s-t* *state-u* *tr-u* *alpha-counter-u* *other-counters-u* *lookup-t*
unfolding *t-eq* **by** *simp*
qed

lemma *random-send-round-outcome*:

assumes *outcome*:

$Some\ (a,\ t) \in$

set-dist (*execute*

(*do* {

$a \leftarrow p.receive-alpha-challenge;$

$p.send\ a;$

return a

```

    }) s)
shows
   $s \leq t \wedge$ 
   $PState\ t = concat\ (PState\ s)\ a \wedge$ 
   $PTranscript\ t = a \# PTranscript\ s \wedge$ 
   $PTraceFriCounter\ t = PTraceFriCounter\ s \wedge$ 
   $PCompositionFriCounter\ t = PCompositionFriCounter\ s \wedge$ 
   $PAlphaCounter\ t = Suc\ (PAlphaCounter\ s) \wedge$ 
   $PQueryCounter\ t = PQueryCounter\ s \wedge$ 
   $fmlookup\ (HashMap\ t)\ (AlphaChallenge\ (PAlphaCounter\ s)\ (PState\ s)) = Some\ a$ 
proof –
from outcome obtain  $u\ v$  where
  rand:  $Some\ (a, u) \in set-dist\ (execute\ p.receive-alpha-challenge\ s)$ 
  and send-a:  $Some\ ((), v) \in set-dist\ (execute\ (p.send\ a)\ u)$ 
  and ret:  $Some\ (a, t) \in set-dist\ (execute\ (return\ a)\ v)$ 
  by (auto elim!: p.set-dist-bindE)
have t-v:  $t = v$ 
  using ret by simp
show ?thesis
  using random-send-round-parts-outcome[OF rand send-a] unfolding t-v .
qed

lemma alpha-mmap-transcript:
assumes outcome:
   $Some\ (as, t) \in$ 
   $set-dist\ (execute$ 
     $(mmap\ (replicate\ n$ 
       $(do\ \{$ 
         $a \leftarrow p.receive-alpha-challenge;$ 
         $p.send\ a;$ 
         $return\ a$ 
       $\})))\ s)$ 
shows  $length\ as = n \wedge PTranscript\ t = rev\ as\ @\ PTranscript\ s$ 
using outcome
proof (induction n arbitrary: as s t)
case 0
  then show ?case by simp
next
case (Suc n)
let ?round =
   $do\ \{$ 
     $a \leftarrow p.receive-alpha-challenge;$ 
     $p.send\ a;$ 
     $return\ a$ 
   $\}$ 
from Suc.prems obtain  $a\ u0\ u\ as'$  where
  rand:  $Some\ (a, u0) \in set-dist\ (execute\ p.receive-alpha-challenge\ s)$ 
  and send-a:  $Some\ ((), u) \in set-dist\ (execute\ (p.send\ a)\ u0)$ 
  and rest:

```

```

    Some (as', t) ∈
      set-dist (execute (mmap (replicate n ?round)) u)
  and as-eq: as = a # as'
  by (auto elim!: p.set-dist-bindE)
have tr-u0: PTranscript u0 = PTranscript s
  using receive-alpha-challenge-preserves-transcript[OF rand] .
have tr-u: PTranscript u = a # PTranscript s
  using p.send-outcome[OF send-a] tr-u0 by simp
have ih: length as' = n ∧ PTranscript t = rev as' @ PTranscript u
  using Suc.IH[OF rest] .
show ?case
  using as-eq tr-u ih by simp
qed

```

lemma *alpha-mmap-outcome*:

assumes *outcome*:

```

  Some (as, t) ∈
    set-dist (execute
      (mmap (replicate n
        (do {
          a ← p.receive-alpha-challenge;
          p.send a;
          return a
        }))) s)

```

shows

```

  length as = n ∧
  s ≤ t ∧
  PState t = foldl concat (PState s) as ∧
  PTranscript t = rev as @ PTranscript s ∧
  PTraceFriCounter t = PTraceFriCounter s ∧
  PCompositionFriCounter t = PCompositionFriCounter s ∧
  PAlphaCounter t = PAlphaCounter s + length as ∧
  PQueryCounter t = PQueryCounter s ∧
  (∀ i < length as.
    fmllookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s + i)
        (foldl concat (PState s) (take i as))) = Some (as ! i))

```

using *outcome*

proof (*induction n arbitrary: as s t*)

case 0

then show ?case

unfolding *less-eq-hash-ext-def less-eq-fmap-def* **by** *simp*

next

case (*Suc n*)

let ?round =

```

  do {
    a ← p.receive-alpha-challenge;
    p.send a;
    return a
  }

```

```

}
from Suc.prems obtain a u0 u as' where
  rand: Some (a, u0) ∈ set-dist (execute p.receive-alpha-challenge s)
  and send-a: Some ((), u) ∈ set-dist (execute (p.send a) u0)
  and tail:
    Some (as', t) ∈
      set-dist (execute (mmap (replicate n ?round)) u)
  and as-eq: as = a # as'
  by (auto elim!: p.set-dist-bindE)
have head-data:
  s ≤ u ∧
    PState u = concat (PState s) a ∧
    PTranscript u = a # PTranscript s ∧
    PTraceFriCounter u = PTraceFriCounter s ∧
    PCompositionFriCounter u = PCompositionFriCounter s ∧
    PAlphaCounter u = Suc (PAlphaCounter s) ∧
    PQueryCounter u = PQueryCounter s ∧
    fmlookup (HashMap u) (AlphaChallenge (PAlphaCounter s) (PState s)) = Some a
  using random-send-round-parts-outcome[OF rand send-a] .
have tail-data:
  length as' = n ∧
    u ≤ t ∧
    PState t = foldl concat (PState u) as' ∧
    PTranscript t = rev as' @ PTranscript u ∧
    PTraceFriCounter t = PTraceFriCounter u ∧
    PCompositionFriCounter t = PCompositionFriCounter u ∧
    PAlphaCounter t = PAlphaCounter u + length as' ∧
    PQueryCounter t = PQueryCounter u ∧
    (∀ i < length as'.
      fmlookup (HashMap t)
        (AlphaChallenge (PAlphaCounter u + i)
          (foldl concat (PState u) (take i as')))) = Some (as' ! i))
  using Suc.IH[OF tail] .
have s-t: s ≤ t
  using head-data tail-data by (meson p.hash-ext-trans)
have lookup-head:
  fmlookup (HashMap t) (AlphaChallenge (PAlphaCounter s) (PState s)) = Some a
  using head-data tail-data by (meson p.hash-extension-lookup)
have lookup-all:
  ∀ i < length (a # as').
    fmlookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s + i)
        (foldl concat (PState s) (take i (a # as'))))) =
      Some ((a # as') ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length (a # as')
  show fmlookup (HashMap t)
    (AlphaChallenge (PAlphaCounter s + i)
      (foldl concat (PState s) (take i (a # as'))))) =
      Some ((a # as') ! i)

```

```

    (foldl concat (PState s) (take i (a # as')))) =
    Some ((a # as') ! i)
proof (cases i)
  case 0
  then show ?thesis
    using lookup-head by simp
next
  case (Suc j)
  then have j-bound:  $j < \text{length } as'$ 
    using i-bound by simp
  have fold-eq:
    foldl concat (PState s) (take (Suc j) (a # as')) =
    foldl concat (PState u) (take j as')
    using head-data by simp
  have alpha-u:  $P\text{AlphaCounter } u = \text{Suc } (P\text{AlphaCounter } s)$ 
    using head-data by simp
  show ?thesis
    using tail-data j-bound fold-eq Suc alpha-u by simp
qed
qed
show ?case
  using as-eq head-data tail-data s-t lookup-all by simp
qed

lemma honest-alpha-mmap-replay-no-failure:
assumes lookups:
   $\forall i < \text{length } as.$ 
  fmlookup (HashMap s)
    (AlphaChallenge (PAlphaCounter s + i)
      (foldl concat (PState s) (take i as))) = Some (as ! i)
shows
  None  $\notin \text{dom } (\text{dist } (\text{execute }
    (\text{mmap } (\text{replicate } (\text{length } as)
      (\text{do } \{
        a0 \leftarrow p.\text{receive-alpha-challenge};
        \text{let } a0' = a0;
        a1 \leftarrow p.\text{read};
        \text{let } a1' = a1;
        \text{assert } (a0' = a1');
        \text{return } a1';
      \})))
    (s(PTranscript := as @ rest))))))
  using lookups
proof (induction as arbitrary: s rest)
  case Nil
  then show ?case by simp
next
  case (Cons a as)
  let ?round =$ 
```

```

do {
  a0 ← p.receive-alpha-challenge;
  let a0' = a0;
  a1 ← p.read;
  let a1' = a1;
  assert (a0' = a1');
  return a1'
}
have lookup-a: fmlookup (HashMap s) (AlphaChallenge (PAlphaCounter s) (PState s)) = Some
a
  using spec[OF Cons.premis, of 0] by simp
have head-nf:
  None ∉ dom (dist (execute ?round
    (s(PTranscript := a # (as @ rest))))))
  using honest-alpha-round-replay-no-failure[OF lookup-a, of as @ rest]
  by simp
have full-eq:
  mmap (replicate (length (a # as)) ?round) =
    (?round ≫ (λx. mmap (replicate (length as) ?round) ≫ (λxs. return (x # xs))))
  by simp
have start-eq:
  s(PTranscript := (a # as) @ rest) =
    s(PTranscript := a # (as @ rest))
  by simp
show ?case
  unfolding full-eq start-eq
proof (rule no-failure-bindI[OF head-nf])
  fix y t
  assume head:
    Some (y, t) ∈
      set-dist (execute ?round (s(PTranscript := a # as @ rest)))
  have head-out:
    y = a ∧
    PState t = concat (PState s) a ∧
    PTranscript t = as @ rest ∧
    PAlphaCounter t = Suc (PAlphaCounter s) ∧
    s ≤ t
  using honest-alpha-round-replay-outcome[OF lookup-a head] by simp
have tail-lookups:
  ∀ i < length as.
    fmlookup (HashMap t)
      (AlphaChallenge (PAlphaCounter t + i)
        (foldl concat (PState t) (take i as))) = Some (as ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length as
  have old-lookup:
    fmlookup (HashMap s)
      (AlphaChallenge (PAlphaCounter s + Suc i)

```

```

      (foldl concat (PState s) (take (Suc i) (a # as)))) =
      Some ((a # as) ! Suc i)
    using spec[OF Cons.premis, of Suc i] i-bound by simp
  have key-eq:
    foldl concat (PState s) (take (Suc i) (a # as)) =
      foldl concat (PState t) (take i as)
    using head-out by simp
  show fmlookup (HashMap t)
    (AlphaChallenge (PAlphaCounter t + i)
      (foldl concat (PState t) (take i as))) =
    Some (as ! i)
    using p.hash-extension-lookup[OF old-lookup, of t] head-out key-eq i-bound
    by simp
qed
have tail-nf-start:
  None  $\notin$  dom (dist (execute
    (mmap (replicate (length as) ?round))
    (t(PTranscript := as @ rest))))
  using Cons.IH[OF tail-lookups, of rest] .
have t-update: t(PTranscript := as @ rest) = t
  using head-out by simp
have tail-nf:
  None  $\notin$  dom (dist (execute
    (mmap (replicate (length as) ?round)) t))
  using tail-nf-start unfolding t-update .
show None  $\notin$  dom (dist (execute
  (mmap (replicate (length as) ?round)  $\gg$  ( $\lambda xs.$  return (y # xs))) t))
  by (rule no-failure-bindI[OF tail-nf]) simp
qed
qed

lemma honest-alpha-mmap-replay-outcome:
  assumes lookups:
     $\forall i < \text{length } as.$ 
    fmlookup (HashMap s)
      (AlphaChallenge (PAlphaCounter s + i)
        (foldl concat (PState s) (take i as))) = Some (as ! i)
  and outcome:
    Some (ys, t)  $\in$  set-dist (execute
      (mmap (replicate (length as)
        (do {
          a0  $\leftarrow$  p.receive-alpha-challenge;
          let a0' = a0;
          a1  $\leftarrow$  p.read;
          let a1' = a1;
          assert (a0' = a1');
          return a1'
        }))))
      (s(PTranscript := as @ rest)))

```

```

shows
   $ys = as \wedge$ 
   $PState\ t = foldl\ concat\ (PState\ s)\ as \wedge$ 
   $PTranscript\ t = rest \wedge$ 
   $PTraceFriCounter\ t = PTraceFriCounter\ s \wedge$ 
   $PCompositionFriCounter\ t = PCompositionFriCounter\ s \wedge$ 
   $PAlphaCounter\ t = PAlphaCounter\ s + length\ as \wedge$ 
   $PQueryCounter\ t = PQueryCounter\ s \wedge$ 
   $s \leq t$ 
using lookups outcome
proof (induction as arbitrary:  $ys\ s\ t\ rest$ )
case Nil
  have t-eq:  $t = s(PTranscript := rest)$ 
    using Nil.premis(2) by simp
  have ys-eq:  $ys = []$ 
    using Nil.premis(2) by simp
  have s-t:  $s \leq t$ 
    unfolding t-eq less-eq-hash-ext-def less-eq-fmap-def by simp
  show ?case
    using ys-eq t-eq s-t by simp
next
case (Cons a as)
let ?round =
  do {
     $a0 \leftarrow p.receive\alpha\ challenge;$ 
     $let\ a0' = a0;$ 
     $a1 \leftarrow p.read;$ 
     $let\ a1' = a1;$ 
     $assert\ (a0' = a1');$ 
     $return\ a1'$ 
  }
from Cons.premis(2) obtain  $a0\ r\text{-}state\ y\ read\text{-}state\ u\ ys'$  where
  rand:
     $Some\ (a0,\ r\text{-}state) \in$ 
     $set\text{-}dist\ (execute\ p.receive\alpha\ challenge$ 
     $(s(PTranscript := a \# (as @ rest))))$ 
  and read:
     $Some\ (y,\ read\text{-}state) \in set\text{-}dist\ (execute\ p.read\ r\text{-}state)$ 
  and assert-ok:
     $Some\ ((),\ u) \in set\text{-}dist\ (execute\ (assert\ (a0 = y))\ read\text{-}state)$ 
  and tail:
     $Some\ (ys',\ t) \in$ 
     $set\text{-}dist\ (execute\ (mmap\ (replicate\ (length\ as)\ ?round))\ u)$ 
  and ys-eq:  $ys = y \# ys'$ 
  by (auto elim!: p.set-dist-bindE)
have lookup-a:  $fmlookup\ (HashMap\ s)\ (AlphaChallenge\ (PAlphaCounter\ s)\ (PState\ s)) = Some$ 
 $a$ 
  using spec[OF Cons.premis(1), of 0] by simp
have head-out:

```



```

y = a ∧
PState u = concat (PState s) a ∧
PTranscript u = as @ rest ∧
PTraceFriCounter u = PTraceFriCounter s ∧
PCompositionFriCounter u = PCompositionFriCounter s ∧
PAlphaCounter u = Suc (PAlphaCounter s) ∧
PQueryCounter u = PQueryCounter s ∧
s ≤ u
using honest-alpha-round-replay-parts-outcome[
  OF lookup-a rand read assert-ok] .
have tail-lookups:
  ∀ i < length as.
    fmlookup (HashMap u)
      (AlphaChallenge (PAlphaCounter u + i)
        (foldl concat (PState u) (take i as))) = Some (as ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length as
  have old-lookup:
    fmlookup (HashMap s)
      (AlphaChallenge (PAlphaCounter s + Suc i)
        (foldl concat (PState s) (take (Suc i) (a # as)))) =
      Some ((a # as) ! Suc i)
    using spec[OF Cons.prem1, of Suc i] i-bound by simp
  have key-eq:
    foldl concat (PState s) (take (Suc i) (a # as)) =
    foldl concat (PState u) (take i as)
    using head-out by simp
  show fmlookup (HashMap u)
    (AlphaChallenge (PAlphaCounter u + i)
      (foldl concat (PState u) (take i as))) =
    Some (as ! i)
    using p.hash-extension-lookup[OF old-lookup, of u] head-out key-eq i-bound
    by simp
qed
have u-update: u(PTranscript := as @ rest) = u
  using head-out by simp
have tail':
  Some (ys', t) ∈
    set-dist (execute (mmap (replicate (length as) ?round))
      (u(PTranscript := as @ rest)))
  using tail unfolding u-update .
have tail-out:
  ys' = as ∧
  PState t = foldl concat (PState u) as ∧
  PTranscript t = rest ∧
  PTraceFriCounter t = PTraceFriCounter u ∧
  PCompositionFriCounter t = PCompositionFriCounter u ∧
  PAlphaCounter t = PAlphaCounter u + length as ∧

```

$PQueryCounter\ t = PQueryCounter\ u \wedge$
 $u \leq t$
using *Cons.IH*[*OF tail-lookups tail*] .
have *s-t*: $s \leq t$
using *head-out tail-out* **by** (*meson p.hash-ext-trans*)
show *?case*
using *ys-eq head-out tail-out s-t* **by** *simp*
qed

lemma *honest-alpha-replay-round-preserves-query-counter*:

assumes *outcome*:

Some (y, t) ∈ set-dist (execute

(do {
 $a0 \leftarrow p.receive-alpha-challenge;$
 $let\ a0' = a0;$
 $a1 \leftarrow p.read;$
 $let\ a1' = a1;$
 $assert\ (a0' = a1');$
 $return\ a1'$
}) s)

shows $PQueryCounter\ t = PQueryCounter\ s$

proof –

from *outcome* **obtain** *a0 r-state read-state* **where**

rand:

Some (a0, r-state) ∈ set-dist (execute p.receive-alpha-challenge s)

and *read*:

Some (y, read-state) ∈ set-dist (execute p.read r-state)

and *assert-ok*: *Some ((), t) ∈ set-dist (execute (assert (a0 = y)) read-state)*

by (*auto simp: Let-def elim!: p.set-dist-bindE*)

have *q-rand*: $PQueryCounter\ r-state = PQueryCounter\ s$

using *p.receive-alpha-challenge-counter-outcome*[*OF rand*] **by** *simp*

have *q-read*: $PQueryCounter\ read-state = PQueryCounter\ r-state$

using *read-preserves-counters*[*OF read*] **by** *simp*

have *t-eq*: $t = read-state$

using *assert-outcomeD*(2)[*OF assert-ok*] .

show *?thesis*

using *q-rand q-read unfolding t-eq* **by** *simp*

qed

lemma *honest-alpha-mmap-replay-preserves-query-counter*:

assumes *outcome*:

Some (ys, t) ∈ set-dist (execute

(mmap (replicate n
(do {
 $a0 \leftarrow p.receive-alpha-challenge;$
 $let\ a0' = a0;$
 $a1 \leftarrow p.read;$
 $let\ a1' = a1;$
 $assert\ (a0' = a1');$

```

      return a1'
    }))) s)
  shows PQueryCounter t = PQueryCounter s
  using outcome
proof (induction n arbitrary: ys s t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  let ?round =
    do {
      a0 ← p.receive-alpha-challenge;
      let a0' = a0;
      a1 ← p.read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    }
  from Suc.premis obtain a0 r-state y read-state u ys' where
    rand:
      Some (a0, r-state) ∈ set-dist (execute p.receive-alpha-challenge s)
  and read:
      Some (y, read-state) ∈ set-dist (execute p.read r-state)
  and assert-ok:
      Some ((), u) ∈ set-dist (execute (assert (a0 = y)) read-state)
  and tail: Some (ys', t) ∈ set-dist (execute (mmap (replicate n ?round)) u)
  by (auto elim!: p.set-dist-bindE)
  have q-rand: PQueryCounter r-state = PQueryCounter s
    using p.receive-alpha-challenge-counter-outcome[OF rand] by simp
  have q-read: PQueryCounter read-state = PQueryCounter r-state
    using read-preserves-counters[OF read] by simp
  have u-eq: u = read-state
    using assert-outcomeD(2)[OF assert-ok] .
  have q-head: PQueryCounter u = PQueryCounter s
    using q-rand q-read unfolding u-eq by simp
  have q-tail: PQueryCounter t = PQueryCounter u
    by (rule Suc.IH[OF tail])
  show ?case
    using q-head q-tail by simp
qed

```

lemma prover-root-alpha-degree-transcript:

```

assumes create-f:
  Some (f-merkle, s1) ∈
    set-dist (execute (p.create p.f-eval) init-state)
and send-f:
  Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and alphas:
  Some (as, s3) ∈

```

```

    set-dist (execute
      (mmap (replicate (length spec)
        (do {
          a ← p.receive-alpha-challenge;
          p.send a;
          return a
        }))) s2)
  and cp'-def: cp' = p.cp as p.f-powers
  and send-degree:
    Some (((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp')))) s3)
  shows
    length as = length spec ∧
    PTranscript s4 = of-nat (degree cp') # rev as @ [value f-merkle]
  proof -
  have tr-s1: PTranscript s1 = []
  using create-preserves-transcript[OF create-f]
  unfolding init-state-def by simp
  have tr-s2: PTranscript s2 = [value f-merkle]
  using p.send-outcome[OF send-f] tr-s1 by simp
  have alpha-tr:
    length as = length spec ∧ PTranscript s3 = rev as @ PTranscript s2
  using alpha-mmap-transcript[OF alphas] .
  have tr-s4: PTranscript s4 = of-nat (degree cp') # PTranscript s3
  using p.send-outcome[OF send-degree] by simp
  show ?thesis
  using alpha-tr tr-s2 tr-s4 by simp
qed

lemma prover-later-phases-transcript-extends:
  assumes create-cp:
    Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
  and fri:
    Some ((ps, ds, ls, ms), s6) ∈
      set-dist (execute
        (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
  and final-send:
    Some (((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
  and random-idx:
    Some (idx, s7) ∈ set-dist (execute p.receive-query-index-challenge s-final)
  and query-decommit:
    Some (qouts, s8) ∈
      set-dist (execute (mmap (p.decommit-on-query idx' f-merkle)) s7)
  and fri-decommit:
    Some (fri-idx, s9) ∈
      set-dist (execute
        (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
  and prover-state-eq: prover-state = s9
  shows transcript-extends prover-state s4
  proof -

```

```

have s4-s5: transcript-extends s5 s4
  using create-transcript-extends[OF create-cp] .
have s5-s6: transcript-extends s6 s5
  using fri-commit-transcript-extends[OF fri] .
have s6-final: transcript-extends s-final s6
  using send-transcript-extends[OF final-send] .
have final-s7: transcript-extends s7 s-final
  using receive-query-index-challenge-transcript-extends[OF random-idx] .
have s7-s8: transcript-extends s8 s7
  by (rule mmap-transcript-extends[OF query-decommit])
    (rule decommit-on-query-step-transcript-extends)
have s8-s9: transcript-extends s9 s8
  by (rule mfold-transcript-extends[OF fri-decommit])
    (rule decommit-on-fri-layers-step-transcript-extends)
have s9-prover: transcript-extends prover-state s9
  using prover-state-eq by simp
show ?thesis
  using s4-s5 s5-s6 s6-final final-s7 s7-s8 s8-s9 s9-prover
  by (meson transcript-extends-trans)
qed

lemma prover-later-phases-hash-extends:
  assumes send-degree:
    Some ((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp'))) s3)
  and create-cp:
    Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
  and fri:
    Some ((ps, ds, ls, ms), s6) ∈
      set-dist (execute
        (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
  and final-send:
    Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
  and random-idx:
    Some (idx, s7) ∈ set-dist (execute p.receive-query-index-challenge s-final)
  and query-decommit:
    Some (qouts, s8) ∈
      set-dist (execute (mmap (p.decommit-on-query idx' f-merkle)) s7)
  and fri-decommit:
    Some (fri-idx, s9) ∈
      set-dist (execute
        (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
  and prover-state-eq: prover-state = s9
  shows s3 ≤ prover-state
proof -
  have s3-s4: s3 ≤ s4
    using send-hash-extends[OF send-degree] .
  have s4-s5: s4 ≤ s5
    using create-hash-extends[OF create-cp] .
  have s5-s6: s5 ≤ s6

```

```

    using honest-fri-commit-extends[OF fri] .
have s6-final: s6 ≤ s-final
    using send-hash-extends[OF final-send] .
have final-s7: s-final ≤ s7
    using p.receive-query-index-challenge-extends[OF random-idx] .
have s7-s8: s7 ≤ s8
    by (rule mmap-hash-extends[OF query-decommit])
       (rule decommit-on-query-step-hash-extends)
have s8-s9: s8 ≤ s9
    by (rule mfold-hash-extends[OF fri-decommit])
       (rule decommit-on-fri-layers-step-hash-extends)
have s9-prover: s9 ≤ prover-state
    using prover-state-eq p.hash-ext-refl by simp
show ?thesis
    using s3-s4 s4-s5 s5-s6 s6-final final-s7 s7-s8 s8-s9 s9-prover
    by (meson p.hash-ext-trans)
qed

lemma prover-after-fri-transcript-extends:
  assumes final-send:
    Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
  and random-idx:
    Some (idx, s7) ∈ set-dist (execute p.receive-query-index-challenge s-final)
  and query-decommit:
    Some (qouts, s8) ∈
      set-dist (execute (mmap (p.decommit-on-query idx' f-merkle)) s7)
  and fri-decommit:
    Some (fri-idx, s9) ∈
      set-dist (execute
        (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
  and prover-state-eq: prover-state = s9
  shows transcript-extends prover-state s6
proof -
  have s6-final: transcript-extends s-final s6
    using send-transcript-extends[OF final-send] .
  have final-s7: transcript-extends s7 s-final
    using receive-query-index-challenge-transcript-extends[OF random-idx] .
  have s7-s8: transcript-extends s8 s7
    by (rule mmap-transcript-extends[OF query-decommit])
       (rule decommit-on-query-step-transcript-extends)
  have s8-s9: transcript-extends s9 s8
    by (rule mfold-transcript-extends[OF fri-decommit])
       (rule decommit-on-fri-layers-step-transcript-extends)
  have s9-prover: transcript-extends prover-state s9
    using prover-state-eq by simp
  show ?thesis
    using s6-final final-s7 s7-s8 s8-s9 s9-prover by (meson transcript-extends-trans)
qed

```

lemma *prover-after-fri-hash-extends*:

assumes *final-send*:

Some $((), s\text{-final}) \in \text{set-dist } (\text{execute } (p.\text{send } (\text{hd } (\text{last } ls)))) s6)$

and *random-idx*:

Some $(idx, s7) \in \text{set-dist } (\text{execute } p.\text{receive-query-index-challenge } s\text{-final})$

and *query-decommit*:

Some $(qouts, s8) \in$

$\text{set-dist } (\text{execute } (\text{mmap } (p.\text{decommit-on-query } idx' \text{ f-merkle}))) s7)$

and *fri-decommit*:

Some $(\text{fri-idx}, s9) \in$

$\text{set-dist } (\text{execute } (\text{mfold } idx' (p.\text{decommit-on-fri-layers } (\text{butlast } (\text{zip } ls \text{ ms})))) s8)$

and *prover-state-eq*: *prover-state* = *s9*

shows $s6 \leq \text{prover-state}$

proof –

have *s6-final*: $s6 \leq s\text{-final}$

using *send-hash-extends*[*OF final-send*] .

have *final-s7*: $s\text{-final} \leq s7$

using *p.receive-query-index-challenge-extends*[*OF random-idx*] .

have *s7-s8*: $s7 \leq s8$

by (rule *mmap-hash-extends*[*OF query-decommit*])

(rule *decommit-on-query-step-hash-extends*)

have *s8-s9*: $s8 \leq s9$

by (rule *mfold-hash-extends*[*OF fri-decommit*])

(rule *decommit-on-fri-layers-step-hash-extends*)

have *s9-prover*: $s9 \leq \text{prover-state}$

using *prover-state-eq* *p.hash-ext-refl* **by** *simp*

show *?thesis*

using *s6-final final-s7 s7-s8 s8-s9 s9-prover* **by** (*meson p.hash-ext-trans*)

qed

lemma *prover-after-final-hash-extends*:

assumes *random-idx*:

Some $(idx, s7) \in \text{set-dist } (\text{execute } p.\text{receive-query-index-challenge } s\text{-final})$

and *query-decommit*:

Some $(qouts, s8) \in$

$\text{set-dist } (\text{execute } (\text{mmap } (p.\text{decommit-on-query } idx' \text{ f-merkle}))) s7)$

and *fri-decommit*:

Some $(\text{fri-idx}, s9) \in$

$\text{set-dist } (\text{execute } (\text{mfold } idx' (p.\text{decommit-on-fri-layers } (\text{butlast } (\text{zip } ls \text{ ms})))) s8)$

and *prover-state-eq*: *prover-state* = *s9*

shows $s\text{-final} \leq \text{prover-state}$

proof –

have *final-s7*: $s\text{-final} \leq s7$

using *p.receive-query-index-challenge-extends*[*OF random-idx*] .

have *s7-s8*: $s7 \leq s8$

by (rule *mmap-hash-extends*[*OF query-decommit*])

(rule *decommit-on-query-step-hash-extends*)

```

have s8-s9:  $s8 \leq s9$ 
  by (rule mfold-hash-extends[OF fri-decommit])
    (rule decommit-on-fri-layers-step-hash-extends)
have s9-prover:  $s9 \leq \text{prover-state}$ 
  using prover-state-eq p.hash-ext-refl by simp
show ?thesis
  using final-s7 s7-s8 s8-s9 s9-prover by (meson p.hash-ext-trans)
qed

lemma prover-query-random-lookup-in-replay-state:
  assumes random-idx:
    Some (idx, s7)  $\in$  set-dist (execute p.receive-query-index-challenge s-final)
  and query-decommit:
    Some (qouts, s8)  $\in$ 
      set-dist (execute (mmap (p.decommit-on-query idx' f-merkle)) s7)
  and fri-decommit:
    Some (fri-idx, s9)  $\in$ 
      set-dist (execute
        (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
  and prover-state-eq: prover-state = s9
  and replay: replay-state = verifier-replay-state prover-state
  shows fmlookup (HashMap replay-state) (QueryIndexChallenge (PQueryCounter s-final) (PState
    s-final)) = Some idx
  proof -
    have lookup-s7: fmlookup (HashMap s7) (QueryIndexChallenge (PQueryCounter s-final) (PState
      s-final)) = Some idx
      using p.receive-query-index-challenge-outcome[OF random-idx] by simp
    have s7-prover:  $s7 \leq \text{prover-state}$ 
    proof -
      have s7-s8:  $s7 \leq s8$ 
        by (rule mmap-hash-extends[OF query-decommit])
          (rule decommit-on-query-step-hash-extends)
      have s8-s9:  $s8 \leq s9$ 
        by (rule mfold-hash-extends[OF fri-decommit])
          (rule decommit-on-fri-layers-step-hash-extends)
      have s9-prover:  $s9 \leq \text{prover-state}$ 
        using prover-state-eq p.hash-ext-refl by simp
      show ?thesis
        using s7-s8 s8-s9 s9-prover by (meson p.hash-ext-trans)
    qed
    have lookup-prover:
      fmlookup (HashMap prover-state) (QueryIndexChallenge (PQueryCounter s-final) (PState
        s-final)) = Some idx
      using p.hash-extension-lookup[OF lookup-s7 s7-prover] .
    show ?thesis
      using lookup-prover replay
      unfolding verifier-replay-state-def by simp
  qed

```


lemma *prover-final-query-counter-in-replay-state:*
assumes *create-f:*
Some (f-merkle, s1) ∈
set-dist (execute (p.create p.f-eval) init-state)
and *send-f:*
Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and *alphas:*
Some (as, s3) ∈
set-dist (execute
(mmap (replicate (length spec)
(do {
a ← p.receive-alpha-challenge;
p.send a;
return a
}))) s2)
and *send-degree:*
Some ((), s4) ∈ set-dist (execute (p.send degree-msg) s3)
and *create-cp:*
Some (cp-merkle, s5) ∈ set-dist (execute (p.create cp-eval) s4)
and *fri:*
Some ((ps, ds, ls, ms), s6) ∈
set-dist (execute (p.fri-commit n ps0 ds0 ls0 ms0) s5)
and *final-send:*
Some ((), s-final) ∈ set-dist (execute (p.send final-msg) s6)
and *replay:* *replay-state = verifier-replay-state prover-state*
shows *PQueryCounter s-final = PQueryCounter replay-state*
proof –
have *q-s1-zero: PQueryCounter s1 = 0*
using *p.create-preserves-channel(6)[OF create-f]*
unfolding *init-state-def* **by** *simp*
have *q-s2-zero: PQueryCounter s2 = 0*
using *p.send-outcome[OF send-f]* *q-s1-zero* **by** *simp*
have *q-s3-zero: PQueryCounter s3 = 0*
using *alpha-mmap-outcome[OF alphas]* *q-s2-zero* **by** *simp*
have *q-s4-zero: PQueryCounter s4 = 0*
using *p.send-outcome[OF send-degree]* *q-s3-zero* **by** *simp*
have *q-s5-zero: PQueryCounter s5 = 0*
using *p.create-preserves-channel(6)[OF create-cp]* *q-s4-zero* **by** *simp*
have *q-s6-zero: PQueryCounter s6 = 0*
using *fri-commit-preserves-query-counter[OF fri]* *q-s5-zero* **by** *simp*
have *q-final-zero: PQueryCounter s-final = 0*
using *p.send-outcome[OF final-send]* *q-s6-zero* **by** *simp*
have *q-replay-zero: PQueryCounter replay-state = 0*
using *replay* **unfolding** *verifier-replay-state-def* **by** *simp*
show *?thesis*
using *q-final-zero q-replay-zero* **by** *simp*
qed

lemma *prover-alpha-lookups-in-replay-state:*

```

assumes create-f:
  Some (f-merkle, s1) ∈
    set-dist (execute (p.create p.f-eval) init-state)
and send-f:
  Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and alphas:
  Some (as, s3) ∈
    set-dist (execute
      (mmap (replicate (length spec)
        (do {
          a ← p.receive-alpha-challenge;
          p.send a;
          return a
        }))) s2)
and later-hash: s3 ≤ prover-state
and replay: replay-state = verifier-replay-state prover-state
shows
  ∀ i < length as.
    fmlookup (HashMap replay-state)
      (AlphaChallenge (PAlphaCounter replay-state + i)
        (foldl concat (concat (PState replay-state) (value f-merkle)) (take i as))) =
      Some (as ! i)
proof –
  have st-s1: PState s1 = 0
    using create-preserves-state[OF create-f]
    unfolding init-state-def by simp
  have send-res:
    s2 = s1 (
      PState := concat (PState s1) (value f-merkle),
      PTranscript := value f-merkle # PTranscript s1)
    using p.send-outcome[OF send-f] .
  have st-s2: PState s2 = concat 0 (value f-merkle)
    using st-s1 send-res by simp
  have alpha-s1-zero: PAlphaCounter s1 = 0
    using p.create-preserves-channel(5)[OF create-f]
    unfolding init-state-def by simp
  have alpha-s2-zero: PAlphaCounter s2 = 0
    using p.send-outcome[OF send-f] alpha-s1-zero by simp
  have alpha-data:
    length as = length spec ∧
    s2 ≤ s3 ∧
    PState s3 = foldl concat (PState s2) as ∧
    PTranscript s3 = rev as @ PTranscript s2 ∧
    (∀ i < length as.
      fmlookup (HashMap s3)
        (AlphaChallenge (PAlphaCounter s2 + i)
          (foldl concat (PState s2) (take i as))) = Some (as ! i))
    using alpha-mmap-outcome[OF alphas] by simp
  show ?thesis

```

```

proof (intro allI impI)
  fix i
    assume i-bound: i < length as
    have lookup-s3:
      fmlookup (HashMap s3)
        (AlphaChallenge (PAlphaCounter s2 + i)
          (foldl concat (PState s2) (take i as))) = Some (as ! i)
    using alpha-data i-bound by simp
    have lookup-final:
      fmlookup (HashMap prover-state)
        (AlphaChallenge (PAlphaCounter s2 + i)
          (foldl concat (PState s2) (take i as))) =
        Some (as ! i)
    using p.hash-extension-lookup[OF lookup-s3 later-hash] .
    show fmlookup (HashMap replay-state)
      (AlphaChallenge (PAlphaCounter replay-state + i)
        (foldl concat (concat (PState replay-state) (value f-merkle)) (take i as))) =
      Some (as ! i)
    using lookup-final st-s2 replay alpha-s2-zero
    unfolding verifier-replay-state-def by simp
  qed
qed

end

end

```

```

theory Completeness-Replay
  imports Completeness-Transcript
begin

```

Verifier replay of honest root, alpha, degree, and FRI-prefix transcript phases.

```

context verification
begin

```

```

lemma verifier-replay-root-alpha-degree-prefix:
  assumes prefix:
    PTranscript s4 = of-nat (degree cp') # rev as @ [value f-merkle]
  and later: transcript-extends prover-state s4
  and replay: replay-state = verifier-replay-state prover-state
  shows  $\exists$  rest.
    PTranscript replay-state = value f-merkle # as @ [of-nat (degree cp')] @ rest
proof –
  from later obtain xs where
    tr-final: PTranscript prover-state = xs @ PTranscript s4
  unfolding transcript-extends-def by auto
  have PTranscript replay-state =
    rev (xs @ (of-nat (degree cp') # rev as @ [value f-merkle]))

```

```

    using replay tr-final prefix
    unfolding verifier-replay-state-def by simp
    also have ... = value f-merkle # as @ [of-nat (degree cp')] @ rev xs
    by simp
    finally show ?thesis by blast
qed

lemma verifier-root-read-from-replay-prefix:
  assumes prefix:
    PTranscript replay-state = value f-merkle # as @ [of-nat (degree cp')] @ rest
  and read-fr:
    Some (fr, s1) ∈ set-dist (execute p.read replay-state)
  shows
    fr = value f-merkle ∧
    PTranscript s1 = as @ [of-nat (degree cp')] @ rest
proof -
  have start-eq:
    replay-state =
      replay-state(PTranscript := value f-merkle # (as @ [of-nat (degree cp')] @ rest))
  using prefix by simp
  have read-start:
    Some (fr, s1) ∈
      set-dist (execute p.read
        (replay-state(
          PTranscript := value f-merkle # (as @ [of-nat (degree cp')] @ rest))))
  by (subst start-eq[symmetric]) (rule read-fr)
  have replay:
    fr = value f-merkle ∧
    s1 = replay-state(
      PState := concat (PState replay-state) (value f-merkle),
      PTranscript := as @ [of-nat (degree cp')] @ rest)
  by (rule p.read-cons-outcome[OF read-start])
  then show ?thesis by simp
qed

```

```

lemma honest-root-alpha-degree-prefix-no-failure:
  assumes htv: honest-trace-valid
  and len-as: length as = length spec
  and prefix:
    PTranscript replay-state =
      value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @ rest
  and lookups:
    ∀ i < length as.
      fmlookup (HashMap replay-state)
        (AlphaChallenge (PAlphaCounter replay-state + i)
          (foldl concat (concat (PState replay-state) (value f-merkle)) (take i as))) =
        Some (as ! i)
  shows
    None ∉ dom (dist (execute

```

```

(do {
  fr ← p.read;
  as' ← mmap (replicate (length spec)
    (do {
      a0 ← p.receive-alpha-challenge;
      let a0' = a0;
      a1 ← p.read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    }));
  dg ← p.read;
  assert (to-nat dg ≤ p.maxDegree);
  return (fr, as', dg)
}) replay-state))
proof –
let ?dg = of-nat (degree (p.cp as p.f-powers))
let ?round =
do {
  a0 ← p.receive-alpha-challenge;
  let a0' = a0;
  a1 ← p.read;
  let a1' = a1;
  assert (a0' = a1');
  return a1'
}
show ?thesis
proof (rule no-failure-bindI)
  show None ∉ dom (dist (execute p.read replay-state))
  by (rule p.read-no-failure) (use prefix in simp)
next
fix fr s1
assume read-fr: Some (fr, s1) ∈ set-dist (execute p.read replay-state)
have read-start:
  Some (fr, s1) ∈
    set-dist (execute p.read
      (replay-state(PTranscript := value f-merkle # (as @ [?dg] @ rest))))
proof –
  have start-eq:
    replay-state =
      replay-state(PTranscript := value f-merkle # (as @ [?dg] @ rest))
  using prefix by simp
show ?thesis
  by (subst start-eq[symmetric]) (rule read-fr)
qed
have read-res:
  fr = value f-merkle ∧
  s1 =
    replay-state(

```

```

    PState := concat (PState replay-state) (value f-merkle),
    PTranscript := as @ [?dg] @ rest)
  using p.read-cons-outcome[OF read-start] by simp
have alpha-lookups:
  ∀ i < length as.
    fmlookup (HashMap s1)
      (AlphaChallenge (PAlphaCounter s1 + i)
        (foldl concat (PState s1) (take i as))) = Some (as ! i)
  using lookups read-res by simp
have alpha-nf:
  None ∉ dom (dist (execute
    (mmap (replicate (length spec) ?round)) s1))
proof -
  have s1-update: s1(PTranscript := as @ [?dg] @ rest) = s1
    using read-res by simp
  have nf:
    None ∉ dom (dist (execute
      (mmap (replicate (length as) ?round))
        (s1(PTranscript := as @ [?dg] @ rest))))
    using honest-alpha-mmap-replay-no-failure[OF alpha-lookups, of ?dg # rest]
    by simp
  show ?thesis
    using nf unfolding len-as[symmetric] s1-update .
qed
show None ∉ dom (dist (execute
  (mmap (replicate (length spec) ?round) >>=
    (λ as'. p.read >>=
      (λ dg. assert (to-nat dg ≤ p.maxDegree) >>=
        (λ -. return (fr, as', dg))))))
  s1))
proof (rule no-failure-bindI[OF alpha-nf])
  fix as' s2
  assume alpha-out:
    Some (as', s2) ∈
      set-dist (execute (mmap (replicate (length spec) ?round)) s1)
  have s1-update: s1(PTranscript := as @ [?dg] @ rest) = s1
    using read-res by simp
  have alpha-out':
    Some (as', s2) ∈
      set-dist (execute (mmap (replicate (length as) ?round))
        (s1(PTranscript := as @ [?dg] @ rest)))
    using alpha-out unfolding len-as[symmetric] s1-update .
  have alpha-res:
    as' = as ∧
    PState s2 = foldl concat (PState s1) as ∧
    PTranscript s2 = ?dg # rest ∧
    s1 ≤ s2
    using honest-alpha-mmap-replay-outcome[OF alpha-lookups alpha-out'] by simp
  show None ∉ dom (dist (execute

```

```

    (p.read  $\gg$ 
      ( $\lambda dg.$  assert ( $to\text{-}nat\ dg \leq p.maxDegree$ )  $\gg$ 
        ( $\lambda -. return\ (fr, as', dg)$ ))) s2))
proof (rule no-failure-bindI)
  show None  $\notin dom\ (dist\ (execute\ p.read\ s2))$ 
    by (rule p.read-no-failure) (use alpha-res in simp)
next
  fix dg s3
  assume read-dg: Some (dg, s3)  $\in set\text{-}dist\ (execute\ p.read\ s2)$ 
  have read-dg-start:
    Some (dg, s3)  $\in$ 
      set-dist (execute p.read (s2(PTranscript := ?dg # rest)))
  proof -
    have s2-update: s2 = s2(PTranscript := ?dg # rest)
      using alpha-res by simp
    show ?thesis
      by (subst s2-update[symmetric]) (rule read-dg)
  qed
have dg-res:
  dg = ?dg  $\wedge$ 
  s3 = s2(PState := concat (PState s2) ?dg, PTranscript := rest)
  using p.read-cons-outcome[OF read-dg-start] by simp
have degree-ok: to-nat dg  $\leq p.maxDegree$ 
  using honest-degree-message-assert-hol[OF htv, of as] dg-res by simp
show None  $\notin dom\ (dist\ (execute\$ 
  (assert ( $to\text{-}nat\ dg \leq p.maxDegree$ )  $\gg$  ( $\lambda -. return\ (fr, as', dg)$ ))) s3))
  using degree-ok by (simp add: assert-def)
qed
qed
qed
qed

lemma honest-root-alpha-degree-prefix-outcome:
assumes htv: honest-trace-valid
and len-as: length as = length spec
and prefix:
  PTranscript replay-state =
    value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @ rest
and lookups:
   $\forall i < length\ as.$ 
    fmlookup (HashMap replay-state)
      (AlphaChallenge (PAlphaCounter replay-state + i)
        (foldl concat (concat (PState replay-state) (value f-merkle)) (take i as))) =
        Some (as ! i)
and outcome:
  Some ((fr, as', dg), t)  $\in set\text{-}dist\ (execute\$ 
    (do {
      fr  $\leftarrow p.read$ ;
      as'  $\leftarrow mmap\ (replicate\ (length\ spec))$ 

```

```

      (do {
        a0 ← p.receive-alpha-challenge;
        let a0' = a0;
        a1 ← p.read;
        let a1' = a1;
        assert (a0' = a1');
        return a1'
      }));
    dg ← p.read;
    assert (to-nat dg ≤ p.maxDegree);
    return (fr, as', dg)
  }) replay-state)
shows
  fr = value f-merkle ∧
  as' = as ∧
  dg = of-nat (degree (p.cp as p.f-powers)) ∧
  PState t =
    concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
      (of-nat (degree (p.cp as p.f-powers))) ∧
  PTranscript t = rest ∧
  replay-state ≤ t
proof –
  let ?dg = of-nat (degree (p.cp as p.f-powers))
  let ?round =
    do {
      a0 ← p.receive-alpha-challenge;
      let a0' = a0;
      a1 ← p.read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    }
  from outcome obtain s1 s2 s3 where
    read-fr: Some (fr, s1) ∈ set-dist (execute p.read replay-state)
    and alphas:
      Some (as', s2) ∈
        set-dist (execute (mmap (replicate (length spec) ?round)) s1)
    and read-dg: Some (dg, s3) ∈ set-dist (execute p.read s2)
    and degree-assert:
      Some ((), t) ∈ set-dist (execute (assert (to-nat dg ≤ p.maxDegree)) s3)
    by (auto elim!: p.set-dist-bindE)
  have read-start:
    Some (fr, s1) ∈
      set-dist (execute p.read
        (replay-state(PTranscript := value f-merkle # (as @ [?dg] @ rest))))
  proof –
    have start-eq:
      replay-state =
        replay-state(PTranscript := value f-merkle # (as @ [?dg] @ rest))

```



```

    using prefix by simp
  show ?thesis
    by (subst start-eq[symmetric]) (rule read-fr)
qed
have read-res:
  fr = value f-merkle ∧
  s1 =
    replay-state(
      PState := concat (PState replay-state) (value f-merkle),
      PTranscript := as @ [?dg] @ rest)
  using p.read-cons-outcome[OF read-start] by simp
have alpha-lookups:
  ∀ i < length as.
    fmlookup (HashMap s1)
      (AlphaChallenge (PAlphaCounter s1 + i)
        (foldl concat (PState s1) (take i as))) = Some (as ! i)
  using lookups read-res by simp
have s1-update: s1(PTranscript := as @ [?dg] @ rest) = s1
  using read-res by simp
have alpha-out:
  Some (as', s2) ∈
    set-dist (execute (mmap (replicate (length as) ?round))
      (s1(PTranscript := as @ [?dg] @ rest)))
  using alphas unfolding len-as[symmetric] s1-update .
have alpha-res:
  as' = as ∧
  PState s2 = foldl concat (PState s1) as ∧
  PTranscript s2 = ?dg # rest ∧
  s1 ≤ s2
  using honest-alpha-mmap-replay-outcome[OF alpha-lookups alpha-out] by simp
have read-dg-start:
  Some (dg, s3) ∈
    set-dist (execute p.read (s2(PTranscript := ?dg # rest)))
proof -
  have s2-update: s2 = s2(PTranscript := ?dg # rest)
    using alpha-res by simp
  show ?thesis
    by (subst s2-update[symmetric]) (rule read-dg)
qed
have dg-res:
  dg = ?dg ∧
  s3 = s2(PState := concat (PState s2) ?dg, PTranscript := rest)
  using p.read-cons-outcome[OF read-dg-start] by simp
have degree-ok: to-nat dg ≤ p.maxDegree
  using honest-degree-message-assert-hol[OF htv, of as] dg-res by simp
have t-eq: t = s3
  using degree-assert degree-ok by (simp add: assert-def)
have replay-s1: replay-state ≤ s1
  using read-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp

```

```

have s2-s3:  $s2 \leq s3$ 
  using dg-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have replay-t:  $\text{replay-state} \leq t$ 
  using replay-s1 alpha-res s2-s3 unfolding t-eq by (meson p.hash-ext-trans)
show ?thesis
  using read-res alpha-res dg-res t-eq replay-t by simp
qed

lemma honest-root-alpha-degree-fri-prefix-no-failure:
  assumes htv: honest-trace-valid
  and len-as:  $\text{length } as = \text{length spec}$ 
  and len-challenges:  $\text{length challenges} = \text{length roots}$ 
  and prefix:
    PTranscript replay-state =
      value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @ roots @ rest
  and alpha-lookups:
     $\forall i < \text{length } as.$ 
      fmlookup (HashMap replay-state)
        (AlphaChallenge (PAlphaCounter replay-state + i)
          (foldl concat (concat (PState replay-state) (value f-merkle)) (take i as))) =
        Some (as ! i)
  and fri-lookups:
     $\forall i < \text{length roots}.$ 
      fmlookup (HashMap replay-state)
        (FiatShamirChallenge (foldl concat
          (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
            (of-nat (degree (p.cp as p.f-powers))))
          (take (Suc i) roots))) =
        Some (challenges ! i)
  shows
    None  $\notin$  dom (dist (execute
      (do {
        fr  $\leftarrow$  p.read;
        as'  $\leftarrow$  mmap (replicate (length spec)
          (do {
            a0  $\leftarrow$  p.receive-alpha-challenge;
            let a0' = a0;
            a1  $\leftarrow$  p.read;
            let a1' = a1;
            assert (a0' = a1');
            return a1'
          })));
        dg  $\leftarrow$  p.read;
        assert (to-nat dg  $\leq$  p.maxDegree);
        fl  $\leftarrow$  ntimes v.receive-fri-commits (length roots);
        return (fr, as', dg, fl)
      }) replay-state))
  proof -
    let ?dg = of-nat (degree (p.cp as p.f-powers))

```

```

let ?round =
  do {
    a0 ← p.receive-alpha-challenge;
    let a0' = a0;
    a1 ← p.read;
    let a1' = a1;
    assert (a0' = a1');
    return a1'
  }
let ?prefix =
  do {
    fr ← p.read;
    as' ← mmap (replicate (length spec) ?round);
    dg ← p.read;
    assert (to-nat dg ≤ p.maxDegree);
    return (fr, as', dg)
  }
let ?full =
  do {
    fr ← p.read;
    as' ← mmap (replicate (length spec) ?round);
    dg ← p.read;
    assert (to-nat dg ≤ p.maxDegree);
    fl ← ntimes v.receive-fri-commits (length roots);
    return (fr, as', dg, fl)
  }
have prefix-nf:
  None ∉ dom (dist (execute ?prefix replay-state))
proof –
  have prefix-root-alpha-degree:
    PTranscript replay-state =
      value f-merkle # as @ [?dg] @ (roots @ rest)
  using prefix by simp
  show ?thesis
  by (rule honest-root-alpha-degree-prefix-no-failure[
    OF htv len-as prefix-root-alpha-degree alpha-lookups])
qed
have full-eq:
  ?full =
    (?prefix ≫=
      (λout.
        ntimes v.receive-fri-commits (length roots) ≫=
          (λfl. return (case out of (fr, as', dg) ⇒ (fr, as', dg, fl)))))
  by (simp add: sm-bind-assoc split: prod.splits)
show ?thesis
  unfolding full-eq
proof (rule no-failure-bindI[OF prefix-nf])
  fix out s-prefix
  assume prefix-out:

```

```

    Some (out, s-prefix) ∈ set-dist (execute ?prefix replay-state)
obtain fr as' dg where out-eq: out = (fr, as', dg)
    by (cases out)
have prefix-out':
    Some ((fr, as', dg), s-prefix) ∈ set-dist (execute ?prefix replay-state)
    using prefix-out unfolding out-eq .
have prefix-res:
    fr = value f-merkle ∧
    as' = as ∧
    dg = ?dg ∧
    PState s-prefix =
      concat (foldl concat (concat (PState replay-state) (value f-merkle)) as) ?dg ∧
    PTranscript s-prefix = roots @ rest ∧
    replay-state ≤ s-prefix
proof –
    have prefix-root-alpha-degree:
      PTranscript replay-state =
        value f-merkle # as @ [?dg] @ (roots @ rest)
      using prefix by simp
    show ?thesis
      by (rule honest-root-alpha-degree-prefix-outcome[
        OF htv len-as prefix-root-alpha-degree alpha-lookups prefix-out])
qed
have fri-lookups-prefix:
  ∀ i < length roots.
    fmlookup (HashMap s-prefix)
      (FiatShamirChallenge (foldl concat (PState s-prefix) (take (Suc i) roots))) =
      Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have lookup-replay:
    fmlookup (HashMap replay-state)
      (FiatShamirChallenge (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          ?dg)
        (take (Suc i) roots))) =
      Some (challenges ! i)
    using fri-lookups i-bound by simp
  show fmlookup (HashMap s-prefix)
    (FiatShamirChallenge (foldl concat (PState s-prefix) (take (Suc i) roots))) =
    Some (challenges ! i)
    using p.hash-extension-lookup[OF lookup-replay, of s-prefix] prefix-res
    by simp
qed
have fri-nf-start:
  None ∉ dom (dist (execute
    (ntimes v.receive-fri-commits (length roots))
    (s-prefix(PTranscript := roots @ rest))))

```

```

using honest-receive-fri-commits-list-no-failure[
  OF fri-lookups-prefix len-challenges, of rest] .
have s-prefix-update: s-prefix( $\backslash$ PTranscript := roots @ rest) = s-prefix
using prefix-res by simp
have fri-nf:
  None  $\notin$  dom (dist (execute
    (ntimes v.receive-fri-commits (length roots)) s-prefix))
using fri-nf-start unfolding s-prefix-update .
show None  $\notin$  dom (dist (execute
  (ntimes v.receive-fri-commits (length roots)  $\gg$ 
    ( $\lambda$ fl. return (case out of (fr, as', dg)  $\Rightarrow$  (fr, as', dg, fl))))
    s-prefix))
by (rule no-failure-bind-returnI[OF fri-nf])
qed
qed

lemma honest-root-alpha-degree-fri-final-prefix-no-failure:
assumes htv: honest-trace-valid
and len-as: length as = length spec
and len-challenges: length challenges = length roots
and prefix:
  PTranscript replay-state =
    value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @
      roots @ [final-msg] @ rest
and alpha-lookups:
   $\forall i < \text{length as}.$ 
    fmllookup (HashMap replay-state)
      (AlphaChallenge (PAlphaCounter replay-state + i)
        (foldl concat (concat (PState replay-state) (value f-merkle)) (take i as))) =
      Some (as ! i)
and fri-lookups:
   $\forall i < \text{length roots}.$ 
    fmllookup (HashMap replay-state)
      (FiatShamirChallenge (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          (of-nat (degree (p.cp as p.f-powers))))
        (take (Suc i) roots))) =
      Some (challenges ! i)
shows
  None  $\notin$  dom (dist (execute
    (do {
      fr  $\leftarrow$  p.read;
      as'  $\leftarrow$  mmap (replicate (length spec)
        (do {
          a0  $\leftarrow$  p.receive-alpha-challenge;
          let a0' = a0;
          a1  $\leftarrow$  p.read;
          let a1' = a1;
          assert (a0' = a1');

```

```

    return a1'
  }));
  dg ← p.read;
  assert (to-nat dg ≤ p.maxDegree);
  fl ← ntimes v.receive-fri-commits (length roots);
  final ← p.read;
  return (fr, as', dg, fl, final)
}) replay-state))
proof –
let ?dg = of-nat (degree (p.cp as p.f-powers))
let ?round =
  do {
    a0 ← p.receive-alpha-challenge;
    let a0' = a0;
    a1 ← p.read;
    let a1' = a1;
    assert (a0' = a1');
    return a1'
  }
let ?prefix =
  do {
    fr ← p.read;
    as' ← mmap (replicate (length spec) ?round);
    dg ← p.read;
    assert (to-nat dg ≤ p.maxDegree);
    return (fr, as', dg)
  }
let ?rad-fri =
  do {
    fr ← p.read;
    as' ← mmap (replicate (length spec) ?round);
    dg ← p.read;
    assert (to-nat dg ≤ p.maxDegree);
    fl ← ntimes v.receive-fri-commits (length roots);
    return (fr, as', dg, fl)
  }
have prefix-nf:
  None ∉ dom (dist (execute ?rad-fri replay-state))
proof –
have prefix': PTranscript replay-state =
  value f-merkle # as @ [?dg] @ roots @ ([final-msg] @ rest)
using prefix by simp
show ?thesis
by (rule honest-root-alpha-degree-fri-prefix-no-failure[
  OF htv len-as len-challenges prefix' alpha-lookups fri-lookups])
qed
have full-eq:
  (do {
    fr ← p.read;

```

```

  as' ← mmap (replicate (length spec) ?round);
  dg ← p.read;
  assert (to-nat dg ≤ p.maxDegree);
  fl ← ntimes v.receive-fri-commits (length roots);
  final ← p.read;
  return (fr, as', dg, fl, final)
}) =
(?rad-fri >>=
  (λout. p.read >>=
    (λfinal. return (case out of (fr, as', dg, fl) =>
      (fr, as', dg, fl, final))))))
by (simp add: sm-bind-assoc split: prod.splits)
show ?thesis
  unfolding full-eq
proof (rule no-failure-bindI[OF prefix-nf])
  fix out s-fri
  assume prefix-out:
    Some (out, s-fri) ∈ set-dist (execute ?rad-fri replay-state)
  obtain fr as' dg fl where out-eq: out = (fr, as', dg, fl)
  by (cases out) auto
  from prefix-out[unfolded out-eq] obtain s1 s2 s3 s-prefix where
    read-fr: Some (fr, s1) ∈ set-dist (execute p.read replay-state)
  and alpha-out:
    Some (as', s2) ∈
      set-dist (execute (mmap (replicate (length spec) ?round)) s1)
  and read-dg: Some (dg, s3) ∈ set-dist (execute p.read s2)
  and degree-assert:
    Some ((), s-prefix) ∈
      set-dist (execute (assert (to-nat dg ≤ p.maxDegree)) s3)
  and fri-out:
    Some (fl, s-fri) ∈
      set-dist (execute (ntimes v.receive-fri-commits (length roots)) s-prefix)
  by (auto elim!: p.set-dist-bindE split: prod.splits)
  have prefix3-out:
    Some ((fr, as', dg), s-prefix) ∈ set-dist (execute ?prefix replay-state)
  apply (rule set-dist-bindI[OF read-fr])
  apply (rule set-dist-bindI[OF alpha-out])
  apply (rule set-dist-bindI[OF read-dg])
  apply (rule set-dist-bindI[OF degree-assert])
  by simp
  have prefix3:
    PTranscript replay-state =
      value f-merkle # as @ [?dg] @ (roots @ [final-msg] @ rest)
  using prefix by simp
  have prefix-res:
    fr = value f-merkle ∧
    as' = as ∧
    dg = ?dg ∧
    PState s-prefix =

```

```

    concat (foldl concat (concat (PState replay-state) (value f-merkle)) as) ?dg ∧
    PTranscript s-prefix = roots @ [final-msg] @ rest ∧
    replay-state ≤ s-prefix
  by (rule honest-root-alpha-degree-prefix-outcome[
    OF htv len-as prefix3 alpha-lookups prefix3-out])
have fri-lookups-prefix:
  ∀ i < length roots.
    fmllookup (HashMap s-prefix)
      (FiatShamirChallenge (foldl concat (PState s-prefix) (take (Suc i) roots))) =
      Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have lookup-replay:
    fmllookup (HashMap replay-state)
      (FiatShamirChallenge (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          ?dg)
        (take (Suc i) roots))) =
      Some (challenges ! i)
  using fri-lookups i-bound by simp
show fmllookup (HashMap s-prefix)
  (FiatShamirChallenge (foldl concat (PState s-prefix) (take (Suc i) roots))) =
  Some (challenges ! i)
  using p.hash-extension-lookup[OF lookup-replay, of s-prefix] prefix-res
  by simp
qed
have fri-out':
  Some (fl, s-fri) ∈
    set-dist (execute (ntimes v.receive-fri-commits (length roots))
      (s-prefix(PTranscript := roots @ ([final-msg] @ rest))))
proof -
  have s-prefix(PTranscript := roots @ ([final-msg] @ rest)) = s-prefix
  using prefix-res by (cases s-prefix) simp
  then show ?thesis
  using fri-out by simp
qed
have tr-fri: PTranscript s-fri = final-msg # rest
  using honest-receive-fri-commits-list-outcome[
    OF fri-lookups-prefix len-challenges fri-out']
  by simp
show None ∉ dom (dist (execute
  (p.read >=>
    (λfinal. return (case out of (fr, as', dg, fl) =>
      (fr, as', dg, fl, final)))) s-fri))
  unfolding out-eq
  by (rule no-failure-bind-returnI)
  (rule p.read-no-failure, simp add: tr-fri)
qed

```


qed

lemma *honest-root-alpha-degree-fri-final-prefix-outcome:*

assumes *htv: honest-trace-valid*

and *len-as: length as = length spec*

and *len-challenges: length challenges = length roots*

and *prefix:*

PTranscript replay-state =
value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @
roots @ [final-msg] @ rest

and *alpha-lookups:*

∀ i < length as.
fmlookup (HashMap replay-state)
(AlphaChallenge (PAlphaCounter replay-state + i)
(foldl concat (concat (PState replay-state) (value f-merkle)) (take i as))) =
Some (as ! i)

and *fri-lookups:*

∀ i < length roots.
fmlookup (HashMap replay-state)
(FiatShamirChallenge (foldl concat
(concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
(of-nat (degree (p.cp as p.f-powers))))
(take (Suc i) roots))) =
Some (challenges ! i)

and *outcome:*

Some ((fr, as', dg, fl, final), t) ∈ set-dist (execute
(do {
fr ← p.read;
as' ← mmap (replicate (length spec)
(do {
a0 ← p.receive-alpha-challenge;
let a0' = a0;
a1 ← p.read;
let a1' = a1;
assert (a0' = a1');
return a1'
}));
dg ← p.read;
assert (to-nat dg ≤ p.maxDegree);
fl ← ntimes v.receive-fri-commits (length roots);
final ← p.read;
return (fr, as', dg, fl, final)
}) replay-state)

shows

fr = value f-merkle ∧
as' = as ∧
dg = of-nat (degree (p.cp as p.f-powers)) ∧
fl = zip challenges roots ∧
final = final-msg ∧

```

PState t =
  concat
    (foldl concat
      (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
        (of-nat (degree (p.cp as p.f-powers))))
      roots)
    final-msg ∧
PTranscript t = rest ∧
replay-state ≤ t
proof –
let ?dg = of-nat (degree (p.cp as p.f-powers))
let ?round =
  do {
    a0 ← p.receive-alpha-challenge;
    let a0' = a0;
    a1 ← p.read;
    let a1' = a1;
    assert (a0' = a1');
    return a1'
  }
let ?rad =
  do {
    fr ← p.read;
    as' ← mmap (replicate (length spec) ?round);
    dg ← p.read;
    assert (to-nat dg ≤ p.maxDegree);
    return (fr, as', dg)
  }
from outcome obtain s1 s2 s3 s-prefix s-fri where
  read-fr: Some (fr, s1) ∈ set-dist (execute p.read replay-state)
  and alpha-out:
    Some (as', s2) ∈
      set-dist (execute (mmap (replicate (length spec) ?round)) s1)
  and read-dg: Some (dg, s3) ∈ set-dist (execute p.read s2)
  and degree-assert:
    Some ((), s-prefix) ∈
      set-dist (execute (assert (to-nat dg ≤ p.maxDegree)) s3)
  and fri-out:
    Some (fl, s-fri) ∈
      set-dist (execute (ntimes v.receive-fri-commits (length roots)) s-prefix)
  and read-final:
    Some (final, t) ∈ set-dist (execute p.read s-fri)
  by (auto elim!: p.set-dist-bindE)
have rad-out:
  Some ((fr, as', dg), s-prefix) ∈ set-dist (execute ?rad replay-state)
apply (rule set-dist-bindI[OF read-fr])
apply (rule set-dist-bindI[OF alpha-out])
apply (rule set-dist-bindI[OF read-dg])
apply (rule set-dist-bindI[OF degree-assert])

```

```

by simp
have rad-prefix:
  PTranscript replay-state =
    value f-merkle # as @ [?dg] @ (roots @ [final-msg] @ rest)
  using prefix by simp
have rad-res:
  fr = value f-merkle ∧
  as' = as ∧
  dg = ?dg ∧
  PState s-prefix =
    concat (foldl concat (concat (PState replay-state) (value f-merkle)) as) ?dg ∧
  PTranscript s-prefix = roots @ [final-msg] @ rest ∧
  replay-state ≤ s-prefix
by (rule honest-root-alpha-degree-prefix-outcome[
  OF htv len-as rad-prefix alpha-lookups rad-out])
have fri-lookups-prefix:
  ∀ i < length roots.
    fmlookup (HashMap s-prefix)
      (FiatShamirChallenge (foldl concat (PState s-prefix) (take (Suc i) roots))) =
      Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have lookup-replay:
    fmlookup (HashMap replay-state)
      (FiatShamirChallenge (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          ?dg)
        (take (Suc i) roots))) =
      Some (challenges ! i)
    using fri-lookups i-bound by simp
  show fmlookup (HashMap s-prefix)
    (FiatShamirChallenge (foldl concat (PState s-prefix) (take (Suc i) roots))) =
    Some (challenges ! i)
    using p.hash-extension-lookup[OF lookup-replay, of s-prefix] rad-res
    by simp
qed
have s-prefix-update:
  s-prefix(PTranscript := roots @ [final-msg] @ rest) = s-prefix
  using rad-res by simp
have fri-out':
  Some (fl, s-fri) ∈
    set-dist (execute (ntimes v.receive-fri-commits (length roots))
      (s-prefix(PTranscript := roots @ ([final-msg] @ rest))))
  using fri-out unfolding s-prefix-update by simp
have fri-res:
  fl = zip challenges roots ∧
  PState s-fri = foldl concat (PState s-prefix) roots ∧
  PTranscript s-fri = [final-msg] @ rest ∧

```

```

     $s\text{-prefix} \leq s\text{-fri}$ 
using honest-receive-fri-commits-list-outcome[
    OF fri-lookups-prefix len-challenges fri-out] .
have read-final:
    Some (final, t) ∈
        set-dist (execute p.read (s-fri(PTranscript := final-msg # rest)))
proof –
    have update: s-fri = s-fri(PTranscript := final-msg # rest)
        using fri-res by simp
    show ?thesis
        by (subst update[symmetric]) (rule read-final)
qed
have final-res:
    final = final-msg ∧
        t = s-fri(PState := concat (PState s-fri) final-msg, PTranscript := rest)
    using p.read-cons-outcome[OF read-final] by simp
have replay-t: replay-state ≤ t
proof –
    have s-fri-t: s-fri ≤ t
        using final-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
    show ?thesis
        using rad-res fri-res s-fri-t by (meson p.hash-ext-trans)
qed
show ?thesis
    using rad-res fri-res final-res replay-t by simp
qed

lemma honest-root-alpha-degree-fri-final-dynamic-prefix-outcome:
assumes htv: honest-trace-valid
and len-as: length as = length spec
and len-challenges: length challenges = length roots
and roots-rounds:
    length roots = ceil-log (degree (p.cp as p.f-powers) + 1)
and prefix:
    PTranscript replay-state =
        value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers)))] @
        roots @ [final-msg] @ rest
and alpha-lookups:
    ∀ i < length as.
        fmlookup (HashMap replay-state)
            (AlphaChallenge (PAlphaCounter replay-state + i)
                (foldl concat (concat (PState replay-state) (value f-merkle)) (take i as))) =
                Some (as ! i)
and fri-lookups:
    ∀ i < length roots.
        fmlookup (HashMap replay-state)
            (FiatShamirChallenge (foldl concat
                (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
                    (of-nat (degree (p.cp as p.f-powers))))))

```

```

      (take (Suc i) roots))) =
    Some (challenges ! i)
and outcome:
    Some ((fr, as', dg, fl, final), t) ∈ set-dist (execute
      (do {
        fr ← p.read;
        as' ← mmap (replicate (length spec)
          (do {
            a0 ← p.receive-alpha-challenge;
            let a0' = a0;
            a1 ← p.read;
            let a1' = a1;
            assert (a0' = a1');
            return a1'
          })));
        dg ← p.read;
        assert (to-nat dg ≤ p.maxDegree);
        fl ← ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1));
        final ← p.read;
        return (fr, as', dg, fl, final)
      }) replay-state)
shows
    fr = value f-merkle ∧
    as' = as ∧
    dg = of-nat (degree (p.cp as p.f-powers)) ∧
    fl = zip challenges roots ∧
    final = final-msg ∧
    PState t =
      concat
        (foldl concat
          (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
            (of-nat (degree (p.cp as p.f-powers)))))
          roots)
        final-msg ∧
    PTranscript t = rest ∧
    replay-state ≤ t
proof –
    let ?dg = of-nat (degree (p.cp as p.f-powers))
    let ?round =
      do {
        a0 ← p.receive-alpha-challenge;
        let a0' = a0;
        a1 ← p.read;
        let a1' = a1;
        assert (a0' = a1');
        return a1'
      }
    let ?rad =
      do {

```

```

    fr ← p.read;
    as' ← mmap (replicate (length spec) ?round);
    dg ← p.read;
    assert (to-nat dg ≤ p.maxDegree);
    return (fr, as', dg)
  }
from outcome obtain s1 s2 s3 s-prefix s-fri where
  read-fr: Some (fr, s1) ∈ set-dist (execute p.read replay-state)
  and alpha-out:
    Some (as', s2) ∈
      set-dist (execute (mmap (replicate (length spec) ?round)) s1)
  and read-dg: Some (dg, s3) ∈ set-dist (execute p.read s2)
  and degree-assert:
    Some ((), s-prefix) ∈
      set-dist (execute (assert (to-nat dg ≤ p.maxDegree)) s3)
  and fri-out:
    Some (fl, s-fri) ∈
      set-dist (execute (ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1))) s-prefix)
  and read-final:
    Some (final, t) ∈ set-dist (execute p.read s-fri)
  by (auto elim!: p.set-dist-bindE)
have rad-out:
  Some ((fr, as', dg), s-prefix) ∈ set-dist (execute ?rad replay-state)
  apply (rule set-dist-bindI[OF read-fr])
  apply (rule set-dist-bindI[OF alpha-out])
  apply (rule set-dist-bindI[OF read-dg])
  apply (rule set-dist-bindI[OF degree-assert])
  by simp
have rad-prefix:
  PTranscript replay-state =
    value f-merkle # as @ [?dg] @ (roots @ [final-msg] @ rest)
  using prefix by simp
have rad-res:
  fr = value f-merkle ∧
  as' = as ∧
  dg = ?dg ∧
  PState s-prefix =
    concat (foldl concat (concat (PState replay-state) (value f-merkle)) as) ?dg ∧
  PTranscript s-prefix = roots @ [final-msg] @ rest ∧
  replay-state ≤ s-prefix
  by (rule honest-root-alpha-degree-prefix-outcome[
    OF htv len-as rad-prefix alpha-lookups rad-out])
have rounds-eq: ceil-log (to-nat dg + 1) = length roots
  using honest-degree-message-rounds[OF htv, of as] rad-res roots-rounds by simp
have fri-lookups-prefix:
  ∀ i < length roots.
    fmlookup (HashMap s-prefix)
      (FiatShamirChallenge (foldl concat (PState s-prefix) (take (Suc i) roots))) =
      Some (challenges ! i)

```

```

proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{length roots}$ 
  have lookup-replay:
    fmlookup (HashMap replay-state)
      (FiatShamirChallenge (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          ?dg)
        (take (Suc i) roots))) =
      Some (challenges ! i)
  using fri-lookups i-bound by simp
  show fmlookup (HashMap s-prefix)
    (FiatShamirChallenge (foldl concat (PState s-prefix) (take (Suc i) roots))) =
    Some (challenges ! i)
  using p.hash-extension-lookup[OF lookup-replay, of s-prefix] rad-res
  by simp
qed
have s-prefix-update:
  s-prefix(|PTranscript := roots @ [final-msg] @ rest|) = s-prefix
  using rad-res by simp
have fri-out':
  Some (fl, s-fri) ∈
    set-dist (execute (ntimes v.receive-fri-commits (length roots))
      (s-prefix(|PTranscript := roots @ ([final-msg] @ rest|))))
  using fri-out rounds-eq unfolding s-prefix-update by simp
have fri-res:
  fl = zip challenges roots ∧
  PState s-fri = foldl concat (PState s-prefix) roots ∧
  PTranscript s-fri = [final-msg] @ rest ∧
  s-prefix ≤ s-fri
  using honest-receive-fri-commits-list-outcome[
    OF fri-lookups-prefix len-challenges fri-out'] .
have read-final':
  Some (final, t) ∈
    set-dist (execute p.read (s-fri(|PTranscript := final-msg # rest|)))
proof –
  have update: s-fri = s-fri(|PTranscript := final-msg # rest|)
  using fri-res by simp
  show ?thesis
  by (subst update[symmetric]) (rule read-final)
qed
have final-res:
  final = final-msg ∧
  t = s-fri(|PState := concat (PState s-fri) final-msg, PTranscript := rest|)
  using p.read-cons-outcome[OF read-final'] by simp
have replay-t: replay-state ≤ t
proof –
  have s-fri-t: s-fri ≤ t
  using final-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp

```

```

show ?thesis
  using rad-res fri-res s-fri-t by (meson p.hash-ext-trans)
qed
show ?thesis
  using rad-res fri-res final-res replay-t by simp
qed

lemma honest-root-alpha-degree-fri-final-prefix-preserves-query-counter:
assumes outcome:
  Some ((fr, as', dg, fl, final), t) ∈ set-dist (execute
    (do {
      fr ← p.read;
      as' ← mmap (replicate (length spec)
        (do {
          a0 ← p.receive-alpha-challenge;
          let a0' = a0;
          a1 ← p.read;
          let a1' = a1;
          assert (a0' = a1');
          return a1'
        })));
      dg ← p.read;
      assert (to-nat dg ≤ p.maxDegree);
      fl ← ntimes v.receive-fri-commits n;
      final ← p.read;
      return (fr, as', dg, fl, final)
    }) s)
shows PQueryCounter t = PQueryCounter s
proof –
let ?round =
  do {
    a0 ← p.receive-alpha-challenge;
    let a0' = a0;
    a1 ← p.read;
    let a1' = a1;
    assert (a0' = a1');
    return a1'
  }
from outcome obtain s-read s-alpha s-dg s-assert s-fri where
  read-fr: Some (fr, s-read) ∈ set-dist (execute p.read s)
and alpha-out:
  Some (as', s-alpha) ∈
    set-dist (execute (mmap (replicate (length spec) ?round)) s-read)
and read-dg: Some (dg, s-dg) ∈ set-dist (execute p.read s-alpha)
and assert-dg:
  Some ((), s-assert) ∈
    set-dist (execute (assert (to-nat dg ≤ p.maxDegree)) s-dg)
and fri-out:
  Some (fl, s-fri) ∈

```



```

    set-dist (execute (ntimes v.receive-fri-commits n) s-assert)
  and read-final: Some (final, t) ∈ set-dist (execute p.read s-fri)
  by (auto elim!: p.set-dist-bindE)
have q-read: PQueryCounter s-read = PQueryCounter s
  using read-preserves-counters[OF read-fr] by simp
have q-alpha: PQueryCounter s-alpha = PQueryCounter s-read
  by (rule honest-alpha-mmap-replay-preserves-query-counter[OF alpha-out])
have q-dg: PQueryCounter s-dg = PQueryCounter s-alpha
  using read-preserves-counters[OF read-dg] by simp
have s-assert-eq: s-assert = s-dg
  using assert-outcomeD(2)[OF assert-dg] .
have q-fri: PQueryCounter s-fri = PQueryCounter s-assert
  by (rule receive-fri-commits-preserves-query-counter[OF fri-out])
have q-final: PQueryCounter t = PQueryCounter s-fri
  using read-preserves-counters[OF read-final] by simp
show ?thesis
  using q-read q-alpha q-dg q-fri q-final unfolding s-assert-eq by simp
qed

```

lemma honest-root-alpha-degree-fri-final-dynamic-prefix-preserves-query-counter:

assumes outcome:

```

Some ((fr, as', dg, fl, final), t) ∈ set-dist (execute
  (do {
    fr ← p.read;
    as' ← mmap (replicate (length spec)
      (do {
        a0 ← p.receive-alpha-challenge;
        let a0' = a0;
        a1 ← p.read;
        let a1' = a1;
        assert (a0' = a1');
        return a1'
      })));
    dg ← p.read;
    assert (to-nat dg ≤ p.maxDegree);
    fl ← ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1));
    final ← p.read;
    return (fr, as', dg, fl, final)
  }) s)

```

shows PQueryCounter t = PQueryCounter s

proof –

let ?round =

```

do {
  a0 ← p.receive-alpha-challenge;
  let a0' = a0;
  a1 ← p.read;
  let a1' = a1;
  assert (a0' = a1');
  return a1'
}

```

```

}
from outcome obtain s-read s-alpha s-dg s-assert s-fri where
  read-fr: Some (fr, s-read) ∈ set-dist (execute p.read s)
and alpha-out:
  Some (as', s-alpha) ∈
    set-dist (execute (mmap (replicate (length spec) ?round)) s-read)
and read-dg: Some (dg, s-dg) ∈ set-dist (execute p.read s-alpha)
and assert-dg:
  Some ((), s-assert) ∈
    set-dist (execute (assert (to-nat dg ≤ p.maxDegree)) s-dg)
and fri-out:
  Some (fl, s-fri) ∈
    set-dist (execute (ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1))) s-assert)
and read-final: Some (final, t) ∈ set-dist (execute p.read s-fri)
by (auto elim!: p.set-dist-bindE)
have q-read: PQueryCounter s-read = PQueryCounter s
  using read-preserves-counters[OF read-fr] by simp
have q-alpha: PQueryCounter s-alpha = PQueryCounter s-read
  by (rule honest-alpha-mmap-replay-preserves-query-counter[OF alpha-out])
have q-dg: PQueryCounter s-dg = PQueryCounter s-alpha
  using read-preserves-counters[OF read-dg] by simp
have s-assert-eq: s-assert = s-dg
  using assert-outcomeD(2)[OF assert-dg] .
have q-fri: PQueryCounter s-fri = PQueryCounter s-assert
  by (rule receive-fri-commits-preserves-query-counter[OF fri-out])
have q-final: PQueryCounter t = PQueryCounter s-fri
  using read-preserves-counters[OF read-final] by simp
show ?thesis
  using q-read q-alpha q-dg q-fri q-final unfolding s-assert-eq by simp
qed

```

```

lemma honest-root-alpha-degree-fri-final-dynamic-prefix-no-failure:
assumes htv: honest-trace-valid
and len-as: length as = length spec
and len-challenges: length challenges = length roots
and roots-rounds:
  length roots = ceil-log (degree (p.cp as p.f-powers) + 1)
and prefix:
  PTranscript replay-state =
    value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @
    roots @ [final-msg] @ rest
and alpha-lookups:
   $\forall i < \text{length } as.$ 
    fmllookup (HashMap replay-state)
      (AlphaChallenge (PAlphaCounter replay-state + i)
        (foldl concat (concat (PState replay-state) (value f-merkle)) (take i as))) =
      Some (as ! i)
and fri-lookups:
   $\forall i < \text{length } roots.$ 

```

```

    fmlookup (HashMap replay-state)
      (FiatShamirChallenge (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          (of-nat (degree (p.cp as p.f-powers))))))
        (take (Suc i) roots))) =
      Some (challenges ! i)
  shows
    None  $\notin$  dom (dist (execute
      (do {
        fr  $\leftarrow$  p.read;
        as'  $\leftarrow$  mmap (replicate (length spec)
          (do {
            a0  $\leftarrow$  p.receive-alpha-challenge;
            let a0' = a0;
            a1  $\leftarrow$  p.read;
            let a1' = a1;
            assert (a0' = a1');
            return a1'
          })));
        dg  $\leftarrow$  p.read;
        assert (to-nat dg  $\leq$  p.maxDegree);
        fl  $\leftarrow$  ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1));
        final  $\leftarrow$  p.read;
        return (fr, as', dg, fl, final)
      }) replay-state))
  proof -
    let ?dg = of-nat (degree (p.cp as p.f-powers))
    let ?round =
      do {
        a0  $\leftarrow$  p.receive-alpha-challenge;
        let a0' = a0;
        a1  $\leftarrow$  p.read;
        let a1' = a1;
        assert (a0' = a1');
        return a1'
      }
    let ?rad =
      do {
        fr  $\leftarrow$  p.read;
        as'  $\leftarrow$  mmap (replicate (length spec) ?round);
        dg  $\leftarrow$  p.read;
        assert (to-nat dg  $\leq$  p.maxDegree);
        return (fr, as', dg)
      }
    have rad-prefix:
      PTranscript replay-state =
        value f-merkle # as @ [?dg] @ (roots @ [final-msg] @ rest)
    using prefix by simp
    have rad-nf:

```

```

None  $\notin$  dom (dist (execute ?rad replay-state))
by (rule honest-root-alpha-degree-prefix-no-failure[
  OF htv len-as rad-prefix alpha-lookups])
have full-eq:
  (do {
    fr  $\leftarrow$  p.read;
    as'  $\leftarrow$  mmap (replicate (length spec) ?round);
    dg  $\leftarrow$  p.read;
    assert (to-nat dg  $\leq$  p.maxDegree);
    fl  $\leftarrow$  ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1));
    final  $\leftarrow$  p.read;
    return (fr, as', dg, fl, final)
  }) =
  (?rad  $\gg$ 
    ( $\lambda$ out. case out of (fr, as', dg)  $\Rightarrow$ 
      ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1))  $\gg$ 
      ( $\lambda$ fl. p.read  $\gg$ 
        ( $\lambda$ final. return (fr, as', dg, fl, final))))))
  by (simp add: sm-bind-assoc split: prod.splits)
show ?thesis
  unfolding full-eq
proof (rule no-failure-bindI[OF rad-nf])
  fix out s-prefix
  assume rad-out: Some (out, s-prefix)  $\in$  set-dist (execute ?rad replay-state)
  obtain fr as' dg where out-eq: out = (fr, as', dg)
  by (cases out) auto
  have rad-out':
    Some ((fr, as', dg), s-prefix)  $\in$  set-dist (execute ?rad replay-state)
  using rad-out unfolding out-eq .
  have rad-res:
    fr = value f-merkle  $\wedge$ 
    as' = as  $\wedge$ 
    dg = ?dg  $\wedge$ 
    PState s-prefix =
      concat (foldl concat (concat (PState replay-state) (value f-merkle)) as) ?dg  $\wedge$ 
    PTranscript s-prefix = roots @ [final-msg] @ rest  $\wedge$ 
    replay-state  $\leq$  s-prefix
  by (rule honest-root-alpha-degree-prefix-outcome[
    OF htv len-as rad-prefix alpha-lookups rad-out'])
  have rounds-eq: ceil-log (to-nat dg + 1) = length roots
  using honest-degree-message-rounds[OF htv, of as] rad-res roots-rounds by simp
  have fri-lookups-prefix:
     $\forall i < \text{length roots.}$ 
    fmllookup (HashMap s-prefix)
      (FiatShamirChallenge (foldl concat (PState s-prefix) (take (Suc i) roots))) =
      Some (challenges ! i)
  proof (intro allI impI)
    fix i
    assume i-bound: i < length roots

```

```

have lookup-replay:
  fmlookup (HashMap replay-state)
    (FiatShamirChallenge (foldl concat
      (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
        ?dg)
      (take (Suc i) roots))) =
    Some (challenges ! i)
  using fri-lookups i-bound by simp
show fmlookup (HashMap s-prefix)
  (FiatShamirChallenge (foldl concat (PState s-prefix) (take (Suc i) roots))) =
    Some (challenges ! i)
  using p.hash-extension-lookup[OF lookup-replay, of s-prefix] rad-res
  by simp
qed
have s-prefix-update:
  s-prefix(PTranscript := roots @ [final-msg] @ rest) = s-prefix
  using rad-res by simp
have fri-nf-start:
  None  $\notin$  dom (dist (execute
    (ntimes v.receive-fri-commits (length roots))
    (s-prefix(PTranscript := roots @ ([final-msg] @ rest)))))
  using honest-receive-fri-commits-list-no-failure[
    OF fri-lookups-prefix len-challenges, of [final-msg] @ rest] .
have fri-nf:
  None  $\notin$  dom (dist (execute
    (ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1))) s-prefix))
  using fri-nf-start unfolding rounds-eq s-prefix-update by simp
have cont-nf:
  None  $\notin$  dom (dist (execute
    (ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1))  $\gg$ 
    ( $\lambda$ fl. p.read  $\gg$  ( $\lambda$ final. return (fr, as', dg, fl, final)))) s-prefix))
proof (rule no-failure-bindI[OF fri-nf])
  fix fl t
  assume fri-out:
    Some (fl, t)  $\in$ 
    set-dist (execute
      (ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1))) s-prefix)
  have fri-out-start:
    Some (fl, t)  $\in$ 
    set-dist (execute
      (ntimes v.receive-fri-commits (length roots))
      (s-prefix(PTranscript := roots @ ([final-msg] @ rest))))
  using fri-out unfolding rounds-eq s-prefix-update by simp
have tr-t: PTranscript t = final-msg # rest
  using honest-receive-fri-commits-list-outcome[
    OF fri-lookups-prefix len-challenges fri-out-start]
  by simp
show None  $\notin$  dom
  (dist (execute (p.read  $\gg$ 

```

```

    (λfinal. return (fr, as', dg, fl, final))) t))
  by (rule no-failure-bind-returnI)
    (rule p.read-no-failure, simp add: tr-t)
qed
show None ∉ dom (dist (execute
  ((case out of (fr, as', dg) ⇒
    ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1)) ≫=
    (λfl. p.read ≫= (λfinal. return (fr, as', dg, fl, final)))))) s-prefix))
  unfolding out-eq
  using cont-nf by simp
qed
qed

lemma honest-alpha-degree-fri-final-dynamic-prefix-outcome:
  assumes htv: honest-trace-valid
  and len-as: length as = length spec
  and len-challenges: length challenges = length roots
  and roots-rounds:
    length roots = ceil-log (degree (p.cp as p.f-powers) + 1)
  and prefix:
    PTranscript replay-state =
      as @ [of-nat (degree (p.cp as p.f-powers))] @ roots @ [final-msg] @ rest
  and alpha-lookups:
    ∀ i < length as.
      fmlookup (HashMap replay-state)
        (AlphaChallenge (PAlphaCounter replay-state + i)
          (foldl concat (PState replay-state) (take i as))) =
        Some (as ! i)
  and fri-lookups:
    ∀ i < length roots.
      fmlookup (HashMap replay-state)
        (CompositionFriChallenge (PCompositionFriCounter replay-state + i) (foldl concat
          (concat (foldl concat (PState replay-state) as)
            (of-nat (degree (p.cp as p.f-powers))))
          (take (Suc i) roots))) =
        Some (challenges ! i)
  and outcome:
    Some ((as', dg, fl, final), t) ∈ set-dist (execute
      (do {
        as' ← mmap (replicate (length spec)
          (do {
            a0 ← p.receive-alpha-challenge;
            let a0' = a0;
            a1 ← p.read;
            let a1' = a1;
            assert (a0' = a1');
            return a1'
          }));
        dg ← p.read;

```

```

    assert (to-nat dg ≤ p.maxDegree);
    fl ← ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1));
    final ← p.read;
    return (as', dg, fl, final)
  }) replay-state)
shows
  as' = as ∧
  dg = of-nat (degree (p.cp as p.f-powers)) ∧
  fl = zip challenges roots ∧
  final = final-msg ∧
  PState t =
    concat
      (foldl concat
        (concat (foldl concat (PState replay-state) as)
          (of-nat (degree (p.cp as p.f-powers)))))
        roots)
      final-msg ∧
  PTranscript t = rest ∧
  replay-state ≤ t
proof –
  let ?dg = of-nat (degree (p.cp as p.f-powers))
  let ?round =
    do {
      a0 ← p.receive-alpha-challenge;
      let a0' = a0;
      a1 ← p.read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    }
  let ?ad =
    do {
      as' ← mmap (replicate (length spec) ?round);
      dg ← p.read;
      assert (to-nat dg ≤ p.maxDegree);
      return (as', dg)
    }
  from outcome obtain s-alpha s-dg s-prefix s-fri where
  alpha-out:
    Some (as', s-alpha) ∈
      set-dist (execute (mmap (replicate (length spec) ?round)) replay-state)
  and read-dg: Some (dg, s-dg) ∈ set-dist (execute p.read s-alpha)
  and degree-assert:
    Some ((), s-prefix) ∈
      set-dist (execute (assert (to-nat dg ≤ p.maxDegree)) s-dg)
  and fri-out:
    Some (fl, s-fri) ∈
      set-dist (execute
        (ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1))))

```

```

      s-prefix)
and read-final:
  Some (final, t) ∈ set-dist (execute p.read s-fri)
by (auto elim!: p.set-dist-bindE)
have alpha-start:
  Some (as', s-alpha) ∈
    set-dist (execute (mmap (replicate (length as) ?round))
      (replay-state(PTranscript := as @ [?dg] @ roots @ [final-msg] @ rest)))
proof -
  have alpha-out-as:
    Some (as', s-alpha) ∈
      set-dist (execute (mmap (replicate (length as) ?round)) replay-state)
    using alpha-out len-as by simp
  have replay-update:
    replay-state =
      replay-state(PTranscript := as @ [?dg] @ roots @ [final-msg] @ rest)
    using prefix by (simp add: append-assoc)
  show ?thesis
    by (subst replay-update[symmetric]) (rule alpha-out-as)
qed
have alpha-res:
  as' = as ∧
    PState s-alpha = foldl concat (PState replay-state) as ∧
    PTranscript s-alpha = ?dg # roots @ [final-msg] @ rest ∧
    PCompositionFriCounter s-alpha = PCompositionFriCounter replay-state ∧
    replay-state ≤ s-alpha
  using honest-alpha-mmap-replay-outcome[OF alpha-lookups alpha-start] by simp
have read-dg':
  Some (dg, s-dg) ∈
    set-dist (execute p.read (s-alpha(PTranscript := ?dg # roots @ [final-msg] @ rest)))
proof -
  have s-alpha-update:
    s-alpha = s-alpha(PTranscript := ?dg # roots @ [final-msg] @ rest)
    using alpha-res by simp
  then show ?thesis
    by (subst s-alpha-update[symmetric]) (rule read-dg)
qed
have dg-res:
  dg = ?dg ∧
    s-dg = s-alpha(PState := concat (PState s-alpha) ?dg,
      PTranscript := roots @ [final-msg] @ rest)
  using p.read-cons-outcome[OF read-dg'] by simp
have degree-ok: to-nat dg ≤ p.maxDegree
  using honest-degree-message-assert-hol[OF htv, of as] dg-res by simp
have prefix-eq: s-prefix = s-dg
  using degree-assert degree-ok by (simp add: assert-def)
have rounds-eq: ceil-log (to-nat dg + 1) = length roots
  using honest-degree-message-rounds[OF htv, of as] dg-res roots-rounds by simp
have fri-lookups-prefix:

```



```

 $\forall i < \text{length roots}.$ 
  fmlookup (HashMap s-prefix)
    (CompositionFriChallenge
      (PCompositionFriCounter s-prefix + i)
      (foldl concat (PState s-prefix) (take (Suc i) roots))) =
    Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{length roots}$ 
  have lookup-replay:
    fmlookup (HashMap replay-state)
      (CompositionFriChallenge (PCompositionFriCounter replay-state + i) (foldl concat
        (concat (foldl concat (PState replay-state) as) ?dg)
        (take (Suc i) roots))) =
      Some (challenges ! i)
  using fri-lookups i-bound by simp
  have lookup-alpha:
    fmlookup (HashMap s-alpha)
      (CompositionFriChallenge (PCompositionFriCounter s-alpha + i) (foldl concat
        (concat (foldl concat (PState replay-state) as) ?dg)
        (take (Suc i) roots))) =
      Some (challenges ! i)
  using p.hash-extension-lookup[OF lookup-replay, of s-alpha] alpha-res by simp
  show fmlookup (HashMap s-prefix)
    (CompositionFriChallenge
      (PCompositionFriCounter s-prefix + i)
      (foldl concat (PState s-prefix) (take (Suc i) roots))) =
    Some (challenges ! i)
  using lookup-alpha alpha-res dg-res prefix-eq by simp
qed
have fri-out':
  Some (fl, s-fri)  $\in$ 
    set-dist (execute (ntimes v.receive-composition-fri-commits (length roots))
      (s-prefix(PTranscript := roots @ ([final-msg] @ rest))))
  using fri-out rounds-eq alpha-res dg-res prefix-eq by simp
have fri-res:
  fl = zip challenges roots  $\wedge$ 
  PState s-fri = foldl concat (PState s-prefix) roots  $\wedge$ 
  PTranscript s-fri = [final-msg] @ rest  $\wedge$ 
  s-prefix  $\leq$  s-fri
  using honest-receive-composition-fri-commits-list-outcome[
    OF fri-lookups-prefix len-challenges fri-out'] .
have read-final':
  Some (final, t)  $\in$ 
    set-dist (execute p.read (s-fri(PTranscript := final-msg # rest)))
proof -
  have s-fri-update: s-fri = s-fri(PTranscript := final-msg # rest)
    using fri-res by simp
  then show ?thesis

```

```

    by (subst s-fri-update[symmetric]) (rule read-final)
qed
have final-res:
  final = final-msg ∧
  t = s-fri(PState := concat (PState s-fri) final-msg, PTranscript := rest)
  using p.read-cons-outcome[OF read-final] by simp
have replay-t: replay-state ≤ t
proof -
  have s-alpha-s-dg: s-alpha ≤ s-dg
    using dg-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  have s-fri-t: s-fri ≤ t
    using final-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  show ?thesis
    using alpha-res s-alpha-s-dg prefix-eq fri-res s-fri-t by (meson p.hash-ext-trans)
qed
show ?thesis
  using alpha-res dg-res prefix-eq fri-res final-res replay-t by simp
qed

lemma honest-alpha-degree-fri-final-dynamic-prefix-no-failure:
  assumes htv: honest-trace-valid
  and len-as: length as = length spec
  and len-challenges: length challenges = length roots
  and roots-rounds:
    length roots = ceil-log (degree (p.cp as p.f-powers) + 1)
  and prefix:
    PTranscript replay-state =
      as @ [of-nat (degree (p.cp as p.f-powers))] @ roots @ [final-msg] @ rest
  and alpha-lookups:
    ∀ i < length as.
      fmlookup (HashMap replay-state)
        (AlphaChallenge (PAlphaCounter replay-state + i)
          (foldl concat (PState replay-state) (take i as))) =
        Some (as ! i)
  and fri-lookups:
    ∀ i < length roots.
      fmlookup (HashMap replay-state)
        (CompositionFriChallenge (PCompositionFriCounter replay-state + i) (foldl concat
          (concat (foldl concat (PState replay-state) as)
            (of-nat (degree (p.cp as p.f-powers))))
          (take (Suc i) roots))) =
        Some (challenges ! i)
  shows
    None ∉ dom (dist (execute
      (do {
        as' ← mmap (replicate (length spec)
          (do {
            a0 ← p.receive-alpha-challenge;
            let a0' = a0;

```

```

    a1 ← p.read;
    let a1' = a1;
    assert (a0' = a1');
    return a1'
  }));
  dg ← p.read;
  assert (to-nat dg ≤ p.maxDegree);
  fl ← ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1));
  final ← p.read;
  return (as', dg, fl, final)
}) replay-state))
proof –
  let ?dg = of-nat (degree (p.cp as p.f-powers))
  let ?round =
    do {
      a0 ← p.receive-alpha-challenge;
      let a0' = a0;
      a1 ← p.read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    }
  let ?ad =
    do {
      as' ← mmap (replicate (length spec) ?round);
      dg ← p.read;
      assert (to-nat dg ≤ p.maxDegree);
      return (as', dg)
    }
  have alpha-nf-start:
    None ∉ dom (dist (execute
      (mmap (replicate (length as) ?round))
      (replay-state(PTranscript := as @ [?dg] @ roots @ [final-msg] @ rest))))
    using honest-alpha-mmap-replay-no-failure[OF alpha-lookups, of ?dg # roots @ [final-msg]
@ rest]
    by (simp add: append-assoc)
  have alpha-nf:
    None ∉ dom (dist (execute
      (mmap (replicate (length spec) ?round)) replay-state))
  proof –
    have replay-update:
      replay-state =
        replay-state(PTranscript := as @ [?dg] @ roots @ [final-msg] @ rest)
      using prefix by (simp add: append-assoc)
    have alpha-nf-as:
      None ∉ dom (dist (execute
        (mmap (replicate (length as) ?round)) replay-state))
      by (subst replay-update) (rule alpha-nf-start)
    show ?thesis

```

```

    using alpha-nf-as len-as by simp
qed
show ?thesis
proof (rule no-failure-bindI[OF alpha-nf])
  fix as' s-alpha
  assume alpha-out:
    Some (as', s-alpha) ∈
      set-dist (execute (mmap (replicate (length spec) ?round)) replay-state)
  have alpha-start:
    Some (as', s-alpha) ∈
      set-dist (execute (mmap (replicate (length as) ?round))
        (replay-state(PTranscript := as @ [?dg] @ roots @ [final-msg] @ rest)))
  proof -
    have alpha-out-as:
      Some (as', s-alpha) ∈
        set-dist (execute (mmap (replicate (length as) ?round)) replay-state)
      using alpha-out len-as by simp
    have replay-update:
      replay-state =
        replay-state(PTranscript := as @ [?dg] @ roots @ [final-msg] @ rest)
      using prefix by (simp add: append-assoc)
    show ?thesis
      by (subst replay-update[symmetric]) (rule alpha-out-as)
  qed
  have alpha-res:
    as' = as ∧
    PState s-alpha = foldl concat (PState replay-state) as ∧
    PTranscript s-alpha = ?dg # roots @ [final-msg] @ rest ∧
    PCompositionFriCounter s-alpha = PCompositionFriCounter replay-state ∧
    replay-state ≤ s-alpha
    using honest-alpha-mmap-replay-outcome[OF alpha-lookups alpha-start] by simp
  show None ∉ dom (dist (execute
    (p.read >>=
      (λdg. assert (to-nat dg ≤ p.maxDegree) >>=
        (λ-. ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1)) >>=
          (λfl. p.read >>= (λfinal. return (as', dg, fl, final)))))) s-alpha))
  proof (rule no-failure-bindI)
    show None ∉ dom (dist (execute p.read s-alpha))
      by (rule p.read-no-failure) (use alpha-res in simp)
  next
    fix dg s-dg
    assume read-dg: Some (dg, s-dg) ∈ set-dist (execute p.read s-alpha)
    have read-dg':
      Some (dg, s-dg) ∈
        set-dist (execute p.read (s-alpha(PTranscript := ?dg # roots @ [final-msg] @ rest)))
    proof -
      have s-alpha-update:
        s-alpha = s-alpha(PTranscript := ?dg # roots @ [final-msg] @ rest)
        using alpha-res by simp

```

```

then show ?thesis
  by (subst s-alpha-update[symmetric]) (rule read-dg)
qed
have dg-res:
  dg = ?dg ∧
  s-dg = s-alpha(|PState := concat (PState s-alpha) ?dg,
    PTranscript := roots @ [final-msg] @ rest|)
  using p.read-cons-outcome[OF read-dg] by simp
have degree-ok: to-nat dg ≤ p.maxDegree
  using honest-degree-message-assert-hol[OF htv, of as] dg-res by simp
have rounds-eq: ceil-log (to-nat dg + 1) = length roots
  using honest-degree-message-rounds[OF htv, of as] dg-res roots-rounds by simp
have fri-lookups-dg:
  ∀ i < length roots.
    fmlookup (HashMap s-dg)
      (CompositionFriChallenge
        (PCompositionFriCounter s-dg + i)
        (foldl concat (PState s-dg) (take (Suc i) roots))) =
      Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have lookup-replay:
    fmlookup (HashMap replay-state)
      (CompositionFriChallenge (PCompositionFriCounter replay-state + i) (foldl concat
        (concat (foldl concat (PState replay-state) as) ?dg)
        (take (Suc i) roots))) =
      Some (challenges ! i)
    using fri-lookups i-bound by simp
  have replay-s-dg: replay-state ≤ s-dg
  proof -
    have s-alpha-s-dg: s-alpha ≤ s-dg
    using dg-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
    show ?thesis
    using alpha-res s-alpha-s-dg by (meson p.hash-ext-trans)
  qed
  show fmlookup (HashMap s-dg)
    (CompositionFriChallenge
      (PCompositionFriCounter s-dg + i)
      (foldl concat (PState s-dg) (take (Suc i) roots))) =
    Some (challenges ! i)
    using p.hash-extension-lookup[OF lookup-replay replay-s-dg] alpha-res dg-res
    by simp
qed
have fri-nf-start:
  None ∉ dom (dist (execute
    (ntimes v.receive-composition-fri-commits (length roots))
    (s-dg(|PTranscript := roots @ ([final-msg] @ rest|))))
  using honest-receive-composition-fri-commits-list-no-failure[

```

```

    OF fri-lookups-dg len-challenges, of [final-msg] @ rest] .
have fri-nf:
  None  $\notin$  dom (dist (execute
    (ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1))) s-dg))
  using fri-nf-start dg-res rounds-eq by simp
have cont-nf:
  None  $\notin$  dom (dist (execute
    (ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1)))  $\gg$ 
    ( $\lambda$ fl. p.read  $\gg$  ( $\lambda$ final. return (as', dg, fl, final)))) s-dg))
proof (rule no-failure-bindI[OF fri-nf])
  fix fl t
  assume fri-out:
    Some (fl, t)  $\in$ 
      set-dist (execute
        (ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1))) s-dg)
  have fri-out-start:
    Some (fl, t)  $\in$ 
      set-dist (execute
        (ntimes v.receive-composition-fri-commits (length roots))
        (s-dg( $\backslash$ PTranscript := roots @ ([final-msg] @ rest))))
  using fri-out dg-res rounds-eq by simp
have tr-t: PTranscript t = final-msg # rest
  using honest-receive-composition-fri-commits-list-outcome[
    OF fri-lookups-dg len-challenges fri-out-start]
  by simp
show None  $\notin$  dom
  (dist (execute (p.read  $\gg$ 
    ( $\lambda$ final. return (as', dg, fl, final))) t))
  by (rule no-failure-bind-returnI)
  (rule p.read-no-failure, simp add: tr-t)
qed
show None  $\notin$  dom (dist (execute
  (assert (to-nat dg  $\leq$  p.maxDegree)  $\gg$ 
    ( $\lambda$ -. ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1)))  $\gg$ 
    ( $\lambda$ fl. p.read  $\gg$  ( $\lambda$ final. return (as', dg, fl, final)))) s-dg))
  using degree-ok cont-nf by (simp add: assert-def)
qed
qed
qed

```

lemma honest-trace-root-alpha-degree-fri-final-dynamic-prefix-outcome:

```

assumes htv: honest-trace-valid
and len-as: length as = length spec
and len-f-challenges: length f-challenges = length f-roots
and f-roots-rounds: length f-roots = ceil-log clength
and len-challenges: length challenges = length roots
and roots-rounds:
  length roots = ceil-log (degree (p.cp as p.f-powers) + 1)
and prefix:

```

```

PTranscript replay-state =
  value f-merkle # f-roots @ [f-final-msg] @
    as @ [of-nat (degree (p.cp as p.f-powers))] @ roots @ [final-msg] @ rest
and f-fri-lookups:
  ∀ i < length f-roots.
    fmlookup (HashMap replay-state)
      (TraceFriChallenge (PTraceFriCounter replay-state + i) (foldl concat
        (concat (PState replay-state) (value f-merkle))
        (take (Suc i) f-roots))) =
      Some (f-challenges ! i)
and alpha-lookups:
  ∀ i < length as.
    fmlookup (HashMap replay-state)
      (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
        f-final-msg)
        (take i as))) =
      Some (as ! i)
and fri-lookups:
  ∀ i < length roots.
    fmlookup (HashMap replay-state)
      (CompositionFriChallenge (PCompositionFriCounter replay-state + i) (foldl concat
        (concat
          (foldl concat
            (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
            f-final-msg)
            as)
            (of-nat (degree (p.cp as p.f-powers))))
          (take (Suc i) roots))) =
      Some (challenges ! i)
and outcome:
  Some ((fr, f-fl, f-final, as', dg, fl, final), t) ∈ set-dist (execute
    (do {
      fr ← p.read;
      f-fl ← ntimes v.receive-trace-fri-commits (ceil-log clength);
      f-final ← p.read;
      as' ← mmap (replicate (length spec)
        (do {
          a0 ← p.receive-alpha-challenge;
          let a0' = a0;
          a1 ← p.read;
          let a1' = a1;
          assert (a0' = a1');
          return a1'
        }));
      dg ← p.read;
      assert (to-nat dg ≤ p.maxDegree);
      fl ← ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1));
      final ← p.read;
    }
  )

```

```

    return (fr, f-fl, f-final, as', dg, fl, final)
  }) replay-state)
shows
  fr = value f-merkle ∧
  f-fl = zip f-challenges f-roots ∧
  f-final = f-final-msg ∧
  as' = as ∧
  dg = of-nat (degree (p.cp as p.f-powers)) ∧
  fl = zip challenges roots ∧
  final = final-msg ∧
  PState t =
    concat
      (foldl concat
        (concat
          (foldl concat
            (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
              f-final-msg)
            as)
            (of-nat (degree (p.cp as p.f-powers))))
          roots)
        final-msg ∧
      PTranscript t = rest ∧
      replay-state ≤ t
proof -
  let ?dg = of-nat (degree (p.cp as p.f-powers))
  let ?round =
    do {
      a0 ← p.receive-alpha-challenge;
      let a0' = a0;
      a1 ← p.read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    }
  let ?tail =
    do {
      as' ← mmap (replicate (length spec) ?round);
      dg ← p.read;
      assert (to-nat dg ≤ p.maxDegree);
      fl ← ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1));
      final ← p.read;
      return (as', dg, fl, final)
    }
  from outcome obtain s-root s-fri s-trace s-alpha s-dg s-assert s-comp where
    read-fr: Some (fr, s-root) ∈ set-dist (execute p.read replay-state)
  and trace-fri-out:
    Some (f-fl, s-fri) ∈
      set-dist (execute
        (ntimes v.receive-trace-fri-commits (ceil-log clength)) s-root)

```



```

and read-f-final: Some (f-final, s-trace)  $\in$  set-dist (execute p.read s-fri)
and alpha-out:
  Some (as', s-alpha)  $\in$ 
    set-dist (execute (mmap (replicate (length spec) ?round)) s-trace)
and read-dg: Some (dg, s-dg)  $\in$  set-dist (execute p.read s-alpha)
and assert-dg:
  Some ((), s-assert)  $\in$  set-dist (execute (assert (to-nat dg  $\leq$  p.maxDegree)) s-dg)
and fri-out:
  Some (fl, s-comp)  $\in$ 
    set-dist (execute
      (ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1)))
      s-assert)
and read-final: Some (final, t)  $\in$  set-dist (execute p.read s-comp)
by (auto elim!: p.set-dist-bindE)
have tail-out:
  Some ((as', dg, fl, final), t)  $\in$  set-dist (execute ?tail s-trace)
apply (rule set-dist-bindI[OF alpha-out])
apply (rule set-dist-bindI[OF read-dg])
apply (rule set-dist-bindI[OF assert-dg])
apply (rule set-dist-bindI[OF fri-out])
apply (rule set-dist-bindI[OF read-final])
by simp
have read-fr':
  Some (fr, s-root)  $\in$ 
    set-dist (execute p.read
      (replay-state(PTranscript :=
        value f-merkle # (f-roots @ [f-final-msg] @ as @ [?dg] @ roots @ [final-msg] @ rest))))
proof -
  have replay-update:
    replay-state =
      replay-state(PTranscript :=
        value f-merkle # (f-roots @ [f-final-msg] @ as @ [?dg] @ roots @ [final-msg] @ rest))
  using prefix by simp
show ?thesis
  by (subst replay-update[symmetric]) (rule read-fr)
qed
have read-fr-res:
  fr = value f-merkle  $\wedge$ 
  s-root =
    replay-state(PState := concat (PState replay-state) (value f-merkle),
      PTranscript := f-roots @ [f-final-msg] @ as @ [?dg] @ roots @ [final-msg] @ rest)
using p.read-cons-outcome[OF read-fr'] by simp
have f-fri-lookups-root:
   $\forall i < \text{length } f\text{-roots}.$ 
  fmlookup (HashMap s-root)
    (TraceFriChallenge
      (PTraceFriCounter s-root + i)
      (foldl concat (PState s-root) (take (Suc i) f-roots))) =
    Some (f-challenges ! i)

```

```

using f-fri-lookups read-fr-res by simp
have trace-fri-out':
  Some (f-fl, s-fri) ∈
    set-dist (execute (ntimes v.receive-trace-fri-commits (length f-roots))
      (s-root(PTranscript := f-roots @ ([f-final-msg] @ as @ [?dg] @ roots @ [final-msg] @
rest))))))
    using trace-fri-out f-roots-rounds read-fr-res by simp
have trace-fri-res:
  f-fl = zip f-challenges f-roots ∧
  PState s-fri = foldl concat (PState s-root) f-roots ∧
  PTranscript s-fri = [f-final-msg] @ as @ [?dg] @ roots @ [final-msg] @ rest ∧
  s-root ≤ s-fri
  using honest-receive-trace-fri-commits-list-outcome[
    OF f-fri-lookups-root len-f-challenges trace-fri-out'] .
have trace-fri-counters:
  PCompositionFriCounter s-fri = PCompositionFriCounter s-root ∧
  PAlphaCounter s-fri = PAlphaCounter s-root ∧
  PQueryCounter s-fri = PQueryCounter s-root
  using receive-trace-fri-commits-preserves-other-counters[OF trace-fri-out']
  by simp
have read-f-final':
  Some (f-final, s-trace) ∈
    set-dist (execute p.read (s-fri(PTranscript :=
      f-final-msg # (as @ [?dg] @ roots @ [final-msg] @ rest))))
proof –
  have s-fri-update:
    s-fri =
      s-fri(PTranscript := f-final-msg # (as @ [?dg] @ roots @ [final-msg] @ rest))
    using trace-fri-res by simp
  show ?thesis
    by (subst s-fri-update[symmetric]) (rule read-f-final)
qed
have read-f-final-res:
  f-final = f-final-msg ∧
  s-trace =
    s-fri(PState := concat (PState s-fri) f-final-msg,
      PTranscript := as @ [?dg] @ roots @ [final-msg] @ rest)
  using p.read-cons-outcome[OF read-f-final'] by simp
have s-trace-counters:
  PCompositionFriCounter s-trace = PCompositionFriCounter replay-state ∧
  PAlphaCounter s-trace = PAlphaCounter replay-state
  using read-fr-res trace-fri-counters read-f-final-res by simp
have replay-s-trace: replay-state ≤ s-trace
proof –
  have replay-root: replay-state ≤ s-root
    using read-fr-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  have s-fri-trace: s-fri ≤ s-trace
    using read-f-final-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  show ?thesis

```

```

    using replay-root trace-fri-res s-fri-trace by (meson p.hash-ext-trans)
  qed
  have alpha-lookups-trace:
     $\forall i < \text{length } as.$ 
    fmlookup (HashMap s-trace)
      (AlphaChallenge (PAlphaCounter s-trace + i)
        (foldl concat (PState s-trace) (take i as))) =
      Some (as ! i)
  proof (intro allI impI)
    fix i
    assume i-bound:  $i < \text{length } as$ 
    have lookup-replay:
      fmlookup (HashMap replay-state)
        (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat
          (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
            f-final-msg)
          (take i as))) =
      Some (as ! i)
    using alpha-lookups i-bound by simp
    show fmlookup (HashMap s-trace)
      (AlphaChallenge (PAlphaCounter s-trace + i)
        (foldl concat (PState s-trace) (take i as))) =
      Some (as ! i)
    using p.hash-extension-lookup[OF lookup-replay replay-s-trace]
      read-fr-res trace-fri-res read-f-final-res s-trace-counters
    by simp
  qed
  have fri-lookups-trace:
     $\forall i < \text{length } roots.$ 
    fmlookup (HashMap s-trace)
      (CompositionFriChallenge (PCompositionFriCounter s-trace + i) (foldl concat
        (concat (foldl concat (PState s-trace) as) ?dg)
        (take (Suc i) roots))) =
      Some (challenges ! i)
  proof (intro allI impI)
    fix i
    assume i-bound:  $i < \text{length } roots$ 
    have lookup-replay:
      fmlookup (HashMap replay-state)
        (CompositionFriChallenge (PCompositionFriCounter replay-state + i) (foldl concat
          (concat
            (foldl concat
              (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
                f-final-msg)
              as)
            ?dg)
          (take (Suc i) roots))) =
      Some (challenges ! i)
    using fri-lookups i-bound by simp

```

```

show fmlookup (HashMap s-trace)
  (CompositionFriChallenge (PCompositionFriCounter s-trace + i) (foldl concat
    (concat (foldl concat (PState s-trace) as) ?dg)
    (take (Suc i) roots))) =
  Some (challenges ! i)
using p.hash-extension-lookup[OF lookup-replay replay-s-trace]
  read-fr-res trace-fri-res read-f-final-res s-trace-counters
by simp
qed
have tail-prefix:
  PTranscript s-trace = as @ [?dg] @ roots @ [final-msg] @ rest
using read-f-final-res by simp
have tail-res:
  as' = as ∧
  dg = ?dg ∧
  fl = zip challenges roots ∧
  final = final-msg ∧
  PState t =
    concat
      (foldl concat
        (concat (foldl concat (PState s-trace) as) ?dg)
        roots)
      final-msg ∧
  PTranscript t = rest ∧
  s-trace ≤ t
by (rule honest-alpha-degree-fri-final-dynamic-prefix-outcome[
  OF htv len-as len-challenges roots-rounds tail-prefix
  alpha-lookups-trace fri-lookups-trace tail-out])
have replay-t: replay-state ≤ t
proof -
  have replay-root: replay-state ≤ s-root
    using read-fr-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  have s-fri-trace: s-fri ≤ s-trace
    using read-f-final-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  show ?thesis
    using replay-root trace-fri-res s-fri-trace tail-res by (meson p.hash-ext-trans)
qed
show ?thesis
  using read-fr-res trace-fri-res read-f-final-res tail-res replay-t by simp
qed

lemma honest-trace-root-alpha-degree-fri-final-dynamic-prefix-preserves-query-counter:
assumes outcome:
  Some ((fr, f-fl, f-final, as', dg, fl, final), t) ∈ set-dist (execute
    (do {
      fr ← p.read;
      f-fl ← ntimes v.receive-trace-fri-commits (ceil-log clength);
      f-final ← p.read;
      as' ← mmap (replicate (length spec)

```

```

    (do {
      a0 ← p.receive-alpha-challenge;
      let a0' = a0;
      a1 ← p.read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    }));
  dg ← p.read;
  assert (to-nat dg ≤ p.maxDegree);
  fl ← ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1));
  final ← p.read;
  return (fr, f-fl, f-final, as', dg, fl, final)
}) s)
shows PQueryCounter t = PQueryCounter s
proof –
  let ?round =
    do {
      a0 ← p.receive-alpha-challenge;
      let a0' = a0;
      a1 ← p.read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    }
  from outcome obtain s-root s-fri s-trace s-alpha s-dg s-assert s-comp where
    read-fr: Some (fr, s-root) ∈ set-dist (execute p.read s)
    and trace-fri-out:
      Some (f-fl, s-fri) ∈
        set-dist (execute (ntimes v.receive-trace-fri-commits (ceil-log clength))) s-root)
    and read-f-final: Some (f-final, s-trace) ∈ set-dist (execute p.read s-fri)
    and alpha-out:
      Some (as', s-alpha) ∈
        set-dist (execute (mmap (replicate (length spec) ?round)) s-trace)
    and read-dg: Some (dg, s-dg) ∈ set-dist (execute p.read s-alpha)
    and assert-dg:
      Some ((), s-assert) ∈ set-dist (execute (assert (to-nat dg ≤ p.maxDegree)) s-dg)
    and comp-out:
      Some (fl, s-comp) ∈
        set-dist (execute
          (ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1)))) s-assert)
    and read-final: Some (final, t) ∈ set-dist (execute p.read s-comp)
    by (auto elim!: p.set-dist-bindE)
  have q-root: PQueryCounter s-root = PQueryCounter s
    using read-preserves-counters[OF read-fr] by simp
  have q-fri: PQueryCounter s-fri = PQueryCounter s-root
    using receive-trace-fri-commits-preserves-other-counters[OF trace-fri-out] by simp
  have q-trace: PQueryCounter s-trace = PQueryCounter s-fri
    using read-preserves-counters[OF read-f-final] by simp

```

```

have q-alpha: PQueryCounter s-alpha = PQueryCounter s-trace
  by (rule honest-alpha-mmap-replay-preserves-query-counter[OF alpha-out])
have q-dg: PQueryCounter s-dg = PQueryCounter s-alpha
  using read-preserves-counters[OF read-dg] by simp
have s-assert-eq: s-assert = s-dg
  using assert-outcomeD(2)[OF assert-dg] .
have q-comp: PQueryCounter s-comp = PQueryCounter s-assert
  using receive-composition-fri-commits-preserves-other-counters[OF comp-out] by simp
have q-final: PQueryCounter t = PQueryCounter s-comp
  using read-preserves-counters[OF read-final] by simp
show ?thesis
  using q-root q-fri q-trace q-alpha q-dg q-comp q-final
  unfolding s-assert-eq by simp
qed

```

lemma *honest-trace-root-alpha-degree-fri-final-dynamic-prefix-no-failure:*

```

assumes htv: honest-trace-valid
and len-as: length as = length spec
and len-f-challenges: length f-challenges = length f-roots
and f-roots-rounds: length f-roots = ceil-log clength
and len-challenges: length challenges = length roots
and roots-rounds:
  length roots = ceil-log (degree (p.cp as p.f-powers) + 1)
and prefix:
  PTranscript replay-state =
    value f-merkle # f-roots @ [f-final-msg] @
    as @ [of-nat (degree (p.cp as p.f-powers))] @ roots @ [final-msg] @ rest
and f-fri-lookups:
  ∀ i < length f-roots.
    fmlookup (HashMap replay-state)
      (TraceFriChallenge (PTraceFriCounter replay-state + i) (foldl concat
        (concat (PState replay-state) (value f-merkle))
        (take (Suc i) f-roots))) =
      Some (f-challenges ! i)
and alpha-lookups:
  ∀ i < length as.
    fmlookup (HashMap replay-state)
      (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
        f-final-msg)
        (take i as))) =
      Some (as ! i)
and fri-lookups:
  ∀ i < length roots.
    fmlookup (HashMap replay-state)
      (CompositionFriChallenge (PCompositionFriCounter replay-state + i) (foldl concat
        (concat
          (foldl concat
            (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)

```

```

      f-final-msg)
    as)
    (of-nat (degree (p.cp as p.f-powers))))
    (take (Suc i) roots))) =
  Some (challenges ! i)
shows
  None  $\notin$  dom (dist (execute
    (do {
      fr  $\leftarrow$  p.read;
      f-fl  $\leftarrow$  ntimes v.receive-trace-fri-commits (ceil-log clength);
      f-final  $\leftarrow$  p.read;
      as'  $\leftarrow$  mmap (replicate (length spec)
        (do {
          a0  $\leftarrow$  p.receive-alpha-challenge;
          let a0' = a0;
          a1  $\leftarrow$  p.read;
          let a1' = a1;
          assert (a0' = a1');
          return a1'
        })));
      dg  $\leftarrow$  p.read;
      assert (to-nat dg  $\leq$  p.maxDegree);
      fl  $\leftarrow$  ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1));
      final  $\leftarrow$  p.read;
      return (fr, f-fl, f-final, as', dg, fl, final)
    }) replay-state))
proof -
  let ?dg = of-nat (degree (p.cp as p.f-powers))
  let ?round =
    do {
      a0  $\leftarrow$  p.receive-alpha-challenge;
      let a0' = a0;
      a1  $\leftarrow$  p.read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    }
  show ?thesis
proof (rule no-failure-bindI)
  show None  $\notin$  dom (dist (execute p.read replay-state))
  by (rule p.read-no-failure) (use prefix in simp)
next
  fix fr s-root
  assume read-fr: Some (fr, s-root)  $\in$  set-dist (execute p.read replay-state)
  have read-fr':
    Some (fr, s-root)  $\in$ 
      set-dist (execute p.read
        (replay-state(PTranscript :=
          value f-merkle # (f-roots @ [f-final-msg] @ as @ [?dg] @ roots @ [final-msg] @ rest))))

```

```

proof –
  have replay-update:
    replay-state =
      replay-state(PTranscript :=
        value f-merkle # (f-roots @ [f-final-msg] @ as @ [?dg] @ roots @ [final-msg] @ rest))
    using prefix by simp
  show ?thesis
    by (subst replay-update[symmetric]) (rule read-fr)
qed
have read-fr-res:
  s-root =
    replay-state(PState := concat (PState replay-state) (value f-merkle),
      PTranscript := f-roots @ [f-final-msg] @ as @ [?dg] @ roots @ [final-msg] @ rest)
    using p.read-cons-outcome[OF read-fr] by simp
have f-fri-lookups-root:
   $\forall i < \text{length } f\text{-roots}.$ 
  fmllookup (HashMap s-root)
    (TraceFriChallenge
      (PTraceFriCounter s-root + i)
      (foldl concat (PState s-root) (take (Suc i) f-roots))) =
    Some (f-challenges ! i)
    using f-fri-lookups read-fr-res by simp
have trace-fri-nf-start:
  None  $\notin \text{dom}$  (dist (execute
    (ntimes v.receive-trace-fri-commits (length f-roots))
    (s-root(PTranscript := f-roots @ ([f-final-msg] @ as @ [?dg] @ roots @ [final-msg] @
rest))))))
    using honest-receive-trace-fri-commits-list-no-failure[
      OF f-fri-lookups-root len-f-challenges, of [f-final-msg] @ as @ [?dg] @ roots @ [final-msg]
@ rest] .
have trace-fri-nf:
  None  $\notin \text{dom}$  (dist (execute
    (ntimes v.receive-trace-fri-commits (ceil-log clength)) s-root))
    using trace-fri-nf-start f-roots-rounds read-fr-res by simp
show None  $\notin \text{dom}$  (dist (execute
  (ntimes v.receive-trace-fri-commits (ceil-log clength)  $\gg=$ 
    ( $\lambda f\text{-fl}.$  p.read  $\gg=$ 
      ( $\lambda f\text{-final}.$  mmap (replicate (length spec) ?round)  $\gg=$ 
        ( $\lambda as'.$  p.read  $\gg=$ 
          ( $\lambda dg.$  assert (to-nat dg  $\leq$  p.maxDegree)  $\gg=$ 
            ( $\lambda -.$  ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1))  $\gg=$ 
              ( $\lambda fl.$  p.read  $\gg=$ 
                ( $\lambda final.$  return (fr, f-fl, f-final, as', dg, fl, final)))))))) s-root))
proof (rule no-failure-bindI[OF trace-fri-nf])
fix f-fl s-fri
assume trace-fri-out:
  Some (f-fl, s-fri)  $\in$ 
    set-dist (execute
      (ntimes v.receive-trace-fri-commits (ceil-log clength)) s-root)

```



```

have trace-fri-out':
  Some (f-fl, s-fri) ∈
    set-dist (execute (ntimes v.receive-trace-fri-commits (length f-roots))
      (s-root(PTranscript := f-roots @ ([f-final-msg] @ as @ [?dg] @ roots @ [final-msg] @
rest))))))
  using trace-fri-out f-roots-rounds read-fr-res by simp
have trace-fri-res:
  PState s-fri = foldl concat (PState s-root) f-roots ∧
  PTranscript s-fri = [f-final-msg] @ as @ [?dg] @ roots @ [final-msg] @ rest ∧
  s-root ≤ s-fri
  using honest-receive-trace-fri-commits-list-outcome[
    OF f-fri-lookups-root len-f-challenges trace-fri-out'] by simp
have trace-fri-counters:
  PCompositionFriCounter s-fri = PCompositionFriCounter s-root ∧
  PAlphaCounter s-fri = PAlphaCounter s-root ∧
  PQueryCounter s-fri = PQueryCounter s-root
  using receive-trace-fri-commits-preserves-other-counters[OF trace-fri-out']
  by simp
show None ∉ dom (dist (execute
  (p.read >>=
    (λf-final. mmap (replicate (length spec) ?round) >>=
      (λas'. p.read >>=
        (λdg. assert (to-nat dg ≤ p.maxDegree) >>=
          (λ-. ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1)) >>=
            (λfl. p.read >>=
              (λfinal. return (fr, f-fl, f-final, as', dg, fl, final)))))))))) s-fri))
proof (rule no-failure-bindI)
  show None ∉ dom (dist (execute p.read s-fri))
  by (rule p.read-no-failure) (use trace-fri-res in simp)
next
  fix f-final s-trace
  assume read-f-final: Some (f-final, s-trace) ∈ set-dist (execute p.read s-fri)
  have read-f-final':
    Some (f-final, s-trace) ∈
      set-dist (execute p.read (s-fri(PTranscript :=
        f-final-msg # (as @ [?dg] @ roots @ [final-msg] @ rest))))))
  proof –
    have s-fri-update:
      s-fri =
        s-fri(PTranscript := f-final-msg # (as @ [?dg] @ roots @ [final-msg] @ rest))
      using trace-fri-res by simp
    show ?thesis
    by (subst s-fri-update[symmetric]) (rule read-f-final)
  qed
have read-f-final-res:
  s-trace =
    s-fri(PState := concat (PState s-fri) f-final-msg,
      PTranscript := as @ [?dg] @ roots @ [final-msg] @ rest)
  using p.read-cons-outcome[OF read-f-final'] by simp

```

```

have s-trace-counters:
  PCompositionFriCounter s-trace = PCompositionFriCounter replay-state  $\wedge$ 
    PAlphaCounter s-trace = PAlphaCounter replay-state
  using read-fr-res trace-fri-counters read-f-final-res by simp
have replay-s-trace: replay-state  $\leq$  s-trace
proof –
  have replay-root: replay-state  $\leq$  s-root
    using read-fr-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  have s-fri-trace: s-fri  $\leq$  s-trace
    using read-f-final-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  show ?thesis
    using replay-root trace-fri-res s-fri-trace by (meson p.hash-ext-trans)
qed
have alpha-lookups-trace:
   $\forall i < \text{length } as.$ 
    fmlookup (HashMap s-trace)
      (AlphaChallenge (PAlphaCounter s-trace + i)
        (foldl concat (PState s-trace) (take i as))) =
      Some (as ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length as
  have lookup-replay:
    fmlookup (HashMap replay-state)
      (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
        f-final-msg)
        (take i as))) =
      Some (as ! i)
    using alpha-lookups i-bound by simp
  show fmlookup (HashMap s-trace)
    (AlphaChallenge (PAlphaCounter s-trace + i)
      (foldl concat (PState s-trace) (take i as))) =
    Some (as ! i)
    using p.hash-extension-lookup[OF lookup-replay replay-s-trace]
      read-fr-res trace-fri-res read-f-final-res s-trace-counters
    by simp
qed
have fri-lookups-trace:
   $\forall i < \text{length roots}.$ 
    fmlookup (HashMap s-trace)
      (CompositionFriChallenge (PCompositionFriCounter s-trace + i) (foldl concat
        (concat (foldl concat (PState s-trace) as) ?dg)
        (take (Suc i) roots))) =
      Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have lookup-replay:

```

```

fmlookup (HashMap replay-state)
  (CompositionFriChallenge (PCompositionFriCounter replay-state + i) (foldl concat
    (concat
      (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
          f-final-msg)
        as)
      ?dg)
    (take (Suc i) roots))) =
  Some (challenges ! i)
using fri-lookups i-bound by simp
show fmlookup (HashMap s-trace)
  (CompositionFriChallenge (PCompositionFriCounter s-trace + i) (foldl concat
    (concat (foldl concat (PState s-trace) as) ?dg)
    (take (Suc i) roots))) =
  Some (challenges ! i)
using p.hash-extension-lookup[OF lookup-replay replay-s-trace]
  read-fr-res trace-fri-res read-f-final-res s-trace-counters
by simp
qed
have tail-prefix:
  PTranscript s-trace = as @ [?dg] @ roots @ [final-msg] @ rest
using read-f-final-res by simp
have tail-nf:
  None  $\notin$  dom (dist (execute
    (do {
      as'  $\leftarrow$  mmap (replicate (length spec) ?round);
      dg  $\leftarrow$  p.read;
      assert (to-nat dg  $\leq$  p.maxDegree);
      fl  $\leftarrow$  ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1));
      final  $\leftarrow$  p.read;
      return (as', dg, fl, final)
    }) s-trace))
by (rule honest-alpha-degree-fri-final-dynamic-prefix-no-failure[
  OF htv len-as len-challenges roots-rounds tail-prefix
  alpha-lookups-trace fri-lookups-trace])
show None  $\notin$  dom (dist (execute
  (mmap (replicate (length spec) ?round)  $\gg$ 
    ( $\lambda$ as'. p.read  $\gg$ 
      ( $\lambda$ dg. assert (to-nat dg  $\leq$  p.maxDegree)  $\gg$ 
        ( $\lambda$ -. ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1))  $\gg$ 
          ( $\lambda$ fl. p.read  $\gg$ 
            ( $\lambda$ final. return (fr, f-fl, f-final, as', dg, fl, final)))))) s-trace))
proof -
let ?tail =
  do {
    as'  $\leftarrow$  mmap (replicate (length spec) ?round);
    dg  $\leftarrow$  p.read;
    assert (to-nat dg  $\leq$  p.maxDegree);

```

```

    fl ← ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1));
    final ← p.read;
    return (as', dg, fl, final)
  }
have tail-full-eq:
  (mmap (replicate (length spec) ?round) >>=
    (λ as'. p.read >>=
      (λ dg. assert (to-nat dg ≤ p.maxDegree) >>=
        (λ -. ntimes v.receive-composition-fri-commits (ceil-log (to-nat dg + 1)) >>=
          (λ fl. p.read >>=
            (λ final. return (fr, f-fl, f-final, as', dg, fl, final))))))) =
    (?tail >>=
      (λ out. return (case out of (as', dg, fl, final) ⇒
        (fr, f-fl, f-final, as', dg, fl, final))))
    by (simp add: sm-bind-assoc split: prod.splits)
  show ?thesis
    unfolding tail-full-eq
    by (rule no-failure-bind-returnI[OF tail-nf])
qed
qed
qed
qed
qed

```

lemma honest-trace-root-alpha-degree-fri-final-dynamic-from-prover-tail-no-failure:

```

assumes htv: honest-trace-valid
and create-f:
  Some (f-merkle, s1) ∈
    set-dist (execute (p.create p.f-eval) init-state)
and send-f:
  Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and f-nrounds-def: f-nrounds = ceil-log clength
and trace-fri:
  Some ((f-ps, f-ds, f-ls, f-ms), s-trace-fri) ∈
    set-dist (execute
      (p.trace-fri-commit f-nrounds [p.f] [p.eval-domain] [p.f-eval] [f-merkle]) s2)
and trace-final-send:
  Some ((), s-trace-final) ∈
    set-dist (execute (p.send (hd (last f-ls))) s-trace-fri)
and alphas:
  Some (as, s3) ∈
    set-dist (execute
      (mmap (replicate (length spec)
        (do {
          a ← p.receive-alpha-challenge;
          p.send a;
          return a
        }))) s-trace-final)
and cp'-def: cp' = p.cp as p.f-powers

```

and *send-degree*:
Some $(((), s_4) \in \text{set-dist } (\text{execute } (p.\text{send } (\text{of-nat } (\text{degree } cp')))) s_3)$
and *create-cp*:
Some $(cp\text{-merkle}, s_5) \in \text{set-dist } (\text{execute } (p.\text{create } (p.\text{cp-eval } as)) s_4)$
and *nrounds-def*: $nrounds = \text{ceil-log } (\text{degree } cp' + 1)$
and *fri*:
Some $((ps, ds, ls, ms), s_6) \in$
 $\text{set-dist } (\text{execute } (p.\text{composition-fri-commit } nrounds [cp'] [p.\text{eval-domain}] [p.\text{cp-eval } as] [cp\text{-merkle}]) s_5)$
and *final-send*:
Some $(((), s\text{-final}) \in \text{set-dist } (\text{execute } (p.\text{send } (\text{hd } (\text{last } ls)))) s_6)$
and *tail*:
Some $(\text{prover-result}, \text{prover-state}) \in$
 $\text{set-dist } (\text{execute } (ntimes (\text{prover-query-round } f\text{-merkle } f\text{-ls } f\text{-ms } ls \text{ } ms) rounds) s\text{-final})$
and *replay*: $\text{replay-state} = \text{verifier-replay-state } \text{prover-state}$
shows
 $None \notin \text{dom } (\text{dist } (\text{execute } (do \{$
 $\text{fr} \leftarrow p.\text{read};$
 $f\text{-fl} \leftarrow ntimes \text{v.receive-trace-fri-commits } (\text{ceil-log } \text{clength});$
 $f\text{-final} \leftarrow p.\text{read};$
 $as' \leftarrow \text{mmap } (\text{replicate } (\text{length } spec)$
 $(do \{$
 $a0 \leftarrow p.\text{receive-alpha-challenge};$
 $\text{let } a0' = a0;$
 $a1 \leftarrow p.\text{read};$
 $\text{let } a1' = a1;$
 $\text{assert } (a0' = a1');$
 $\text{return } a1'$
 $\}));$
 $dg \leftarrow p.\text{read};$
 $\text{assert } (\text{to-nat } dg \leq p.\text{maxDegree});$
 $fl \leftarrow ntimes \text{v.receive-composition-fri-commits } (\text{ceil-log } (\text{to-nat } dg + 1));$
 $\text{final} \leftarrow p.\text{read};$
 $\text{return } (fr, f\text{-fl}, f\text{-final}, as', dg, fl, \text{final})$
 $\}) \text{ replay-state}))$
proof –
from $\text{trace-fri-commit-initial-replay-successor-data}[OF \text{ trace-fri}]$ **obtain** $f\text{-roots } f\text{-challenges}$
where
 $f\text{-len-roots}: \text{length } f\text{-roots} = f\text{-nrounds}$
and $f\text{-len-challenges}: \text{length } f\text{-challenges} = f\text{-nrounds}$
and $tr\text{-trace-fri}: P\text{Transcript } s\text{-trace-fri} = \text{rev } f\text{-roots} @ P\text{Transcript } s_2$
and $st\text{-trace-fri}: P\text{State } s\text{-trace-fri} = \text{foldl concat } (P\text{State } s_2) f\text{-roots}$
and $\text{trace-fri-lookups-state}$:
 $\forall i < \text{length } f\text{-roots}.$
 $\text{fmlookup } (\text{HashMap } s\text{-trace-fri})$
 $(\text{TraceFriChallenge } (P\text{TraceFriCounter } s_2 + i)$
 $(\text{foldl concat } (P\text{State } s_2) (\text{take } (\text{Suc } i) f\text{-roots}))) =$
 $\text{Some } (f\text{-challenges } ! i)$

```

by blast
from composition-fri-commit-initial-replay-successor-data[OF fri] obtain roots challenges where
  len-roots: length roots = nrounds
  and len-challenges: length challenges = nrounds
  and tr-s6: PTranscript s6 = rev roots @ PTranscript s5
  and st-s6: PState s6 = foldl concat (PState s5) roots
  and fri-lookups-s6:
     $\forall i < \text{length roots.}$ 
    fmlookup (HashMap s6)
      (CompositionFriChallenge
        (PCompositionFriCounter s5 + i)
        (foldl concat (PState s5) (take (Suc i) roots))) =
      Some (challenges ! i)
by blast
have alpha-data:
  length as = length spec  $\wedge$ 
  s-trace-final  $\leq$  s3  $\wedge$ 
  PState s3 = foldl concat (PState s-trace-final) as  $\wedge$ 
  PTranscript s3 = rev as @ PTranscript s-trace-final  $\wedge$ 
  PCompositionFriCounter s3 = PCompositionFriCounter s-trace-final  $\wedge$ 
  ( $\forall i < \text{length as.}$ 
    fmlookup (HashMap s3)
      (AlphaChallenge (PAlphaCounter s-trace-final + i)
        (foldl concat (PState s-trace-final) (take i as))) =
    Some (as ! i))
  using alpha-mmap-outcome[OF alphas] by simp
have tail-hash: s-final  $\leq$  prover-state
  by (rule ntimes-hash-extends[OF - tail])
    (rule prover-query-round-hash-extends)
have s2-trace-fri: s2  $\leq$  s-trace-fri
  using p.trace-fri-commit-extends[OF trace-fri] .
have trace-fri-final-hash: s-trace-fri  $\leq$  s-trace-final
  using send-hash-extends[OF trace-final-send] .
have s3-s4: s3  $\leq$  s4
  using send-hash-extends[OF send-degree] .
have s4-s5: s4  $\leq$  s5
  using create-hash-extends[OF create-cp] .
have s5-s6: s5  $\leq$  s6
  using p.composition-fri-commit-extends[OF fri] .
have s6-final-hash: s6  $\leq$  s-final
  using send-hash-extends[OF final-send] .
have trace-fri-prover: s-trace-fri  $\leq$  prover-state
  using trace-fri-final-hash alpha-data s3-s4 s4-s5 s5-s6 s6-final-hash tail-hash
  by (meson p.hash-ext-trans)
have s3-prover: s3  $\leq$  prover-state
  using s3-s4 s4-s5 s5-s6 s6-final-hash tail-hash by (meson p.hash-ext-trans)
have s6-prover: s6  $\leq$  prover-state
  using s6-final-hash tail-hash by (rule p.hash-ext-trans)
have final-tail-tr: transcript-extends prover-state s-final

```

```

    by (rule ntimes-transcript-extends[OF - tail])
      (rule prover-query-round-transcript-extends)
  from final-tail-tr obtain ys where
    tr-prover: PTranscript prover-state = ys @ PTranscript s-final
    unfolding transcript-extends-def by auto
  have tr-s1: PTranscript s1 = []
    using create-preserves-transcript[OF create-f]
    unfolding init-state-def by simp
  have send-f-res:
    s2 = s1 (
      PState := concat (PState s1) (value f-merkle),
      PTranscript := value f-merkle # PTranscript s1 )
    using p.send-outcome[OF send-f] .
  have trace-final-res:
    s-trace-final = s-trace-fri (
      PState := concat (PState s-trace-fri) (hd (last f-ls)),
      PTranscript := hd (last f-ls) # PTranscript s-trace-fri)
    using p.send-outcome[OF trace-final-send] .
  have send-degree-res:
    s4 = s3 (
      PState := concat (PState s3) (of-nat (degree cp')),
      PTranscript := of-nat (degree cp') # PTranscript s3)
    using p.send-outcome[OF send-degree] .
  have tr-s5: PTranscript s5 = PTranscript s4
    using create-preserves-transcript[OF create-cp] .
  have final-send-res:
    s-final = s6 (
      PState := concat (PState s6) (hd (last ls)),
      PTranscript := hd (last ls) # PTranscript s6)
    using p.send-outcome[OF final-send] .
  have replay-prefix:
    PTranscript replay-state =
      value f-merkle # f-roots @ [hd (last f-ls)] @
      as @ [of-nat (degree (p.cp as p.f-powers))] @ roots @ [hd (last ls)] @ rev ys
    using replay tr-prover final-send-res tr-s6 tr-s5 send-degree-res alpha-data
      trace-final-res tr-trace-fri send-f-res tr-s1 cp'-def
    unfolding verifier-replay-state-def by simp
  have st-s1: PState s1 = 0
    using create-preserves-state[OF create-f]
    unfolding init-state-def by simp
  have st-trace-final:
    PState s-trace-final =
      concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
        (hd (last f-ls))
    using st-s1 send-f-res st-trace-fri trace-final-res replay
    unfolding verifier-replay-state-def by simp
  have st-s5:
    PState s5 =
      concat

```

```

    (foldl concat
      (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
        (hd (last f-ls)))
      as)
    (of-nat (degree (p.cp as p.f-powers)))
  using st-trace-final alpha-data send-degree-res create-preserves-state[OF create-cp] cp'-def
  by simp
have trace-counter-s1-zero: PTraceFriCounter s1 = 0
  using p.create-preserves-channel(3)[OF create-f]
  unfolding init-state-def by simp
have trace-counter-s2-zero: PTraceFriCounter s2 = 0
  using send-f-res trace-counter-s1-zero by simp
have alpha-counter-s1-zero: PAlphaCounter s1 = 0
  using p.create-preserves-channel(5)[OF create-f]
  unfolding init-state-def by simp
have alpha-counter-s2-zero: PAlphaCounter s2 = 0
  using send-f-res alpha-counter-s1-zero by simp
have alpha-counter-trace-fri-zero: PAlphaCounter s-trace-fri = 0
  using trace-fri-commit-preserves-alpha-counter[OF trace-fri]
  alpha-counter-s2-zero by simp
have alpha-counter-trace-final-zero: PAlphaCounter s-trace-final = 0
  using trace-final-res alpha-counter-trace-fri-zero by simp
have comp-counter-s1-zero: PCompositionFriCounter s1 = 0
  using p.create-preserves-channel(4)[OF create-f]
  unfolding init-state-def by simp
have comp-counter-s2-zero: PCompositionFriCounter s2 = 0
  using send-f-res comp-counter-s1-zero by simp
have comp-counter-trace-fri-zero: PCompositionFriCounter s-trace-fri = 0
  using trace-fri-commit-preserves-composition-counter[OF trace-fri]
  comp-counter-s2-zero by simp
have comp-counter-trace-final-zero:
  PCompositionFriCounter s-trace-final = 0
  using trace-final-res comp-counter-trace-fri-zero by simp
have comp-counter-s3-zero: PCompositionFriCounter s3 = 0
  using alpha-data comp-counter-trace-final-zero by simp
have comp-counter-s4-zero: PCompositionFriCounter s4 = 0
  using send-degree-res comp-counter-s3-zero by simp
have comp-counter-s5-zero: PCompositionFriCounter s5 = 0
  using p.create-preserves-channel(4)[OF create-cp] comp-counter-s4-zero
  by simp
have f-fri-lookups:
   $\forall i < \text{length } f\text{-roots}.$ 
  fmlookup (HashMap replay-state)
    (TraceFriChallenge (PTraceFriCounter replay-state + i) (foldl concat
      (concat (PState replay-state) (value f-merkle))
      (take (Suc i) f-roots))) =
    Some (f-challenges ! i)
proof (intro allI impI)
  fix i

```



```

assume i-bound:  $i < \text{length } f\text{-roots}$ 
have lookup-trace:
  fmlookup (HashMap s-trace-fri)
    (TraceFriChallenge (PTraceFriCounter s2 + i)
      (foldl concat (PState s2) (take (Suc i) f-roots))) =
      Some (f-challenges ! i)
  using trace-fri-lookups-state i-bound by simp
have lookup-prover:
  fmlookup (HashMap prover-state)
    (TraceFriChallenge (PTraceFriCounter s2 + i)
      (foldl concat (PState s2) (take (Suc i) f-roots))) =
      Some (f-challenges ! i)
  using p.hash-extension-lookup[OF lookup-trace trace-fri-prover] .
show fmlookup (HashMap replay-state)
  (TraceFriChallenge (PTraceFriCounter replay-state + i) (foldl concat
    (concat (PState replay-state) (value f-merkle))
    (take (Suc i) f-roots))) =
    Some (f-challenges ! i)
  using lookup-prover send-f-res st-s1 replay trace-counter-s2-zero
  unfolding verifier-replay-state-def by simp
qed
have alpha-lookups:
   $\forall i < \text{length } as.$ 
  fmlookup (HashMap replay-state)
    (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat
      (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
      (hd (last f-ls)))
      (take i as))) =
      Some (as ! i)
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{length } as$ 
  have alpha-lookups-s3:
     $\forall i < \text{length } as.$ 
    fmlookup (HashMap s3)
      (AlphaChallenge (PAlphaCounter s-trace-final + i)
        (foldl concat (PState s-trace-final) (take i as))) =
        Some (as ! i)
    using alpha-data by simp
  have lookup-s3:
    fmlookup (HashMap s3)
      (AlphaChallenge (PAlphaCounter s-trace-final + i)
        (foldl concat (PState s-trace-final) (take i as))) =
        Some (as ! i)
    using alpha-lookups-s3 i-bound by simp
  have lookup-prover:
    fmlookup (HashMap prover-state)
      (AlphaChallenge (PAlphaCounter s-trace-final + i)
        (foldl concat (PState s-trace-final) (take i as))) =

```

```

    Some (as ! i)
  using p.hash-extension-lookup[OF lookup-s3 s3-prover] .
show fmlookup (HashMap replay-state)
  (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat
    (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
      (hd (last f-ls)))
    (take i as))) =
  Some (as ! i)
  using lookup-prover st-trace-final replay alpha-counter-trace-final-zero
  unfolding verifier-replay-state-def by simp
qed
have fri-lookups:
  ∀ i < length roots.
  fmlookup (HashMap replay-state)
    (CompositionFriChallenge (PCompositionFriCounter replay-state + i) (foldl concat
      (concat
        (foldl concat
          (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
            (hd (last f-ls)))
          as)
        (of-nat (degree (p.cp as p.f-powers))))
      (take (Suc i) roots))) =
    Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have lookup-s6:
    fmlookup (HashMap s6)
      (CompositionFriChallenge
        (PCompositionFriCounter s5 + i)
        (foldl concat (PState s5) (take (Suc i) roots))) =
      Some (challenges ! i)
  using fri-lookups-s6 i-bound by simp
  have lookup-prover:
    fmlookup (HashMap prover-state)
      (CompositionFriChallenge
        (PCompositionFriCounter s5 + i)
        (foldl concat (PState s5) (take (Suc i) roots))) =
      Some (challenges ! i)
  using p.hash-extension-lookup[OF lookup-s6 s6-prover] .
show fmlookup (HashMap replay-state)
  (CompositionFriChallenge (PCompositionFriCounter replay-state + i) (foldl concat
    (concat
      (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
          (hd (last f-ls)))
        as)
        (of-nat (degree (p.cp as p.f-powers))))
      (take (Suc i) roots))) =

```

```

    Some (challenges ! i)
    using lookup-prover st-s5 replay comp-counter-s5-zero
    unfolding verifier-replay-state-def by simp
qed
have f-roots-rounds: length f-roots = ceil-log clength
  using f-len-roots f-nrounds-def by simp
have f-len-challenges-roots: length f-challenges = length f-roots
  using f-len-challenges f-len-roots by simp
have roots-rounds:
  length roots = ceil-log (degree (p.cp as p.f-powers) + 1)
  using len-roots nrounds-def cp'-def by simp
have len-challenges-roots: length challenges = length roots
  using len-challenges len-roots by simp
show ?thesis
  by (rule honest-trace-root-alpha-degree-fri-final-dynamic-prefix-no-failure[
    OF htv conjunct1[OF alpha-data] f-len-challenges-roots f-roots-rounds
    len-challenges-roots roots-rounds replay-prefix f-fri-lookups
    alpha-lookups fri-lookups])
qed

lemma honest-trace-root-alpha-degree-fri-final-dynamic-from-prover-tail-outcome:
  assumes htv: honest-trace-valid
  and create-f:
    Some (f-merkle, s1) ∈
      set-dist (execute (p.create p.f-eval) init-state)
  and send-f:
    Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
  and f-nrounds-def: f-nrounds = ceil-log clength
  and trace-fri:
    Some ((f-ps, f-ds, f-ls, f-ms), s-trace-fri) ∈
      set-dist (execute
        (p.trace-fri-commit f-nrounds [p.f] [p.eval-domain] [p.f-eval] [f-merkle]) s2)
  and trace-final-send:
    Some ((), s-trace-final) ∈
      set-dist (execute (p.send (hd (last f-ls))) s-trace-fri)
  and alphas:
    Some (as, s3) ∈
      set-dist (execute
        (mmap (replicate (length spec)
          (do {
            a ← p.receive-alpha-challenge;
            p.send a;
            return a
          }))) s-trace-final)
  and cp'-def: cp' = p.cp as p.f-powers
  and send-degree:
    Some ((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp'))) s3)
  and create-cp:
    Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)

```

and *nrounds-def*: $nrounds = \text{ceil-log } (\text{degree } cp' + 1)$
and *fri*:
 Some $((ps, ds, ls, ms), s6) \in$
 set-dist (*execute*
 $(p.\text{composition-fri-commit } nrounds [cp'] [p.\text{eval-domain}] [p.\text{cp-eval } as] [cp\text{-merkle}] s5)$)
and *final-send*:
 Some $(((), s\text{-final}) \in \text{set-dist } (\text{execute } (p.\text{send } (hd (last ls)))) s6)$
and *tail*:
 Some $(\text{prover-result}, \text{prover-state}) \in$
 set-dist (*execute* (*ntimes* (*prover-query-round* *f-merkle f-ls f-ms ls ms*) *nrounds*) *s-final*)
and *replay*: *replay-state* = *verifier-replay-state prover-state*
and *outcome*:
 Some $((fr, f\text{-fl}, f\text{-final}, as', dg, fl, final), t) \in \text{set-dist } (\text{execute}$
 (*do* {
 $fr \leftarrow p.\text{read};$
 $f\text{-fl} \leftarrow \text{ntimes } v.\text{receive-trace-fri-commits } (\text{ceil-log } clength);$
 $f\text{-final} \leftarrow p.\text{read};$
 $as' \leftarrow \text{mmap } (\text{replicate } (length \text{ spec}))$
 (*do* {
 $a0 \leftarrow p.\text{receive-alpha-challenge};$
 $\text{let } a0' = a0;$
 $a1 \leftarrow p.\text{read};$
 $\text{let } a1' = a1;$
 $\text{assert } (a0' = a1');$
 $\text{return } a1'$
 });
 $dg \leftarrow p.\text{read};$
 $\text{assert } (to\text{-nat } dg \leq p.\text{maxDegree});$
 $fl \leftarrow \text{ntimes } v.\text{receive-composition-fri-commits } (\text{ceil-log } (to\text{-nat } dg + 1));$
 $final \leftarrow p.\text{read};$
 $\text{return } (fr, f\text{-fl}, f\text{-final}, as', dg, fl, final)$
 }) *replay-state*)
obtains *f-roots f-challenges roots challenges ys where*
 $length \text{ f-roots} = f\text{-nrounds}$
 $length \text{ f-challenges} = f\text{-nrounds}$
 $f\text{-fl} = \text{zip } f\text{-challenges } f\text{-roots}$
 $\bigwedge j. j < f\text{-nrounds} \implies f\text{-roots } ! j = \text{value } (f\text{-ms } ! j)$
 $\bigwedge j. j < f\text{-nrounds} \implies$
 $p.\text{next-fri-layer } (f\text{-ps } ! j) (f\text{-ds } ! j) (f\text{-challenges } ! j) =$
 $(f\text{-ps } ! \text{Suc } j, f\text{-ds } ! \text{Suc } j, f\text{-ls } ! \text{Suc } j)$
 $f\text{-final} = hd (last \text{ f-ls})$
 $length \text{ roots} = nrounds$
 $length \text{ challenges} = nrounds$
 $fl = \text{zip } challenges \text{ roots}$
 $\bigwedge j. j < nrounds \implies roots ! j = \text{value } (ms ! j)$
 $\bigwedge j. j < nrounds \implies$
 $p.\text{next-fri-layer } (ps ! j) (ds ! j) (challenges ! j) =$
 $(ps ! \text{Suc } j, ds ! \text{Suc } j, ls ! \text{Suc } j)$
 $fr = \text{value } f\text{-merkle}$

```

as' = as
dg = of-nat (degree (p.cp as p.f-powers))
final = hd (last ls)
PState t = PState s-final
PTranscript t = rev ys
PTranscript prover-state = ys @ PTranscript s-final
replay-state ≤ t
proof -
  from trace-fri-commit-initial-replay-successor-data[OF trace-fri] obtain f-roots f-challenges
where
  f-len-roots: length f-roots = f-nrounds
  and f-len-challenges: length f-challenges = f-nrounds
  and tr-trace-fri: PTranscript s-trace-fri = rev f-roots @ PTranscript s2
  and st-trace-fri: PState s-trace-fri = foldl concat (PState s2) f-roots
  and trace-fri-lookups-state:
    ∀ i < length f-roots.
      fmllookup (HashMap s-trace-fri)
        (TraceFriChallenge (PTraceFriCounter s2 + i)
          (foldl concat (PState s2) (take (Suc i) f-roots))) =
        Some (f-challenges ! i)
  and f-roots-aligned-all: ∀ j < f-nrounds. f-roots ! j = value (f-ms ! j)
  and f-successors-all:
    ∀ j < f-nrounds.
      p.next-fri-layer (f-ps ! j) (f-ds ! j) (f-challenges ! j) =
      (f-ps ! Suc j, f-ds ! Suc j, f-ls ! Suc j)
  by blast
from composition-fri-commit-initial-replay-successor-data[OF fri] obtain roots challenges where
  len-roots: length roots = nrounds
  and len-challenges: length challenges = nrounds
  and tr-s6: PTranscript s6 = rev roots @ PTranscript s5
  and st-s6: PState s6 = foldl concat (PState s5) roots
  and fri-lookups-s6:
    ∀ i < length roots.
      fmllookup (HashMap s6)
        (CompositionFriChallenge
          (PCompositionFriCounter s5 + i)
          (foldl concat (PState s5) (take (Suc i) roots))) =
        Some (challenges ! i)
  and roots-aligned-all: ∀ j < nrounds. roots ! j = value (ms ! j)
  and successors-all:
    ∀ j < nrounds.
      p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
      (ps ! Suc j, ds ! Suc j, ls ! Suc j)
  by blast
have alpha-data:
  length as = length spec ∧
  s-trace-final ≤ s3 ∧
  PState s3 = foldl concat (PState s-trace-final) as ∧
  PTranscript s3 = rev as @ PTranscript s-trace-final ∧

```

```

    PCompositionFriCounter s3 = PCompositionFriCounter s-trace-final ∧
    (∀ i < length as.
      fmllookup (HashMap s3)
        (AlphaChallenge (PAlphaCounter s-trace-final + i)
          (foldl concat (PState s-trace-final) (take i as))) =
        Some (as ! i))
  using alpha-mmap-outcome[OF alphas] by simp
have tail-hash: s-final ≤ prover-state
  by (rule ntimes-hash-extends[OF - tail])
  (rule prover-query-round-hash-extends)
have trace-fri-final-hash: s-trace-fri ≤ s-trace-final
  using send-hash-extends[OF trace-final-send] .
have s3-s4: s3 ≤ s4
  using send-hash-extends[OF send-degree] .
have s4-s5: s4 ≤ s5
  using create-hash-extends[OF create-cp] .
have s5-s6: s5 ≤ s6
  using p.composition-fri-commit-extends[OF fri] .
have s6-final-hash: s6 ≤ s-final
  using send-hash-extends[OF final-send] .
have trace-fri-prover: s-trace-fri ≤ prover-state
  using trace-fri-final-hash alpha-data s3-s4 s4-s5 s5-s6 s6-final-hash tail-hash
  by (meson p.hash-ext-trans)
have s3-prover: s3 ≤ prover-state
  using s3-s4 s4-s5 s5-s6 s6-final-hash tail-hash by (meson p.hash-ext-trans)
have s6-prover: s6 ≤ prover-state
  using s6-final-hash tail-hash by (rule p.hash-ext-trans)
have final-tail-tr: transcript-extends prover-state s-final
  by (rule ntimes-transcript-extends[OF - tail])
  (rule prover-query-round-transcript-extends)
from final-tail-tr obtain ys where
  tr-prover: PTranscript prover-state = ys @ PTranscript s-final
  unfolding transcript-extends-def by auto
have tr-s1: PTranscript s1 = []
  using create-preserves-transcript[OF create-f]
  unfolding init-state-def by simp
have send-f-res:
  s2 = s1 (|
    PState := concat (PState s1) (value f-merkle),
    PTranscript := value f-merkle # PTranscript s1 |)
  using p.send-outcome[OF send-f] .
have trace-final-res:
  s-trace-final = s-trace-fri (|
    PState := concat (PState s-trace-fri) (hd (last f-ls)),
    PTranscript := hd (last f-ls) # PTranscript s-trace-fri |)
  using p.send-outcome[OF trace-final-send] .
have send-degree-res:
  s4 = s3 (|
    PState := concat (PState s3) (of-nat (degree cp')),

```

```

    PTranscript := of-nat (degree cp') # PTranscript s3)
  using p.send-outcome[OF send-degree] .
have tr-s5: PTranscript s5 = PTranscript s4
  using create-preserves-transcript[OF create-cp] .
have final-send-res:
  s-final = s6()
  PState := concat (PState s6) (hd (last ls)),
  PTranscript := hd (last ls) # PTranscript s6)
  using p.send-outcome[OF final-send] .
have replay-prefix:
  PTranscript replay-state =
    value f-merkle # f-roots @ [hd (last f-ls)] @
    as @ [of-nat (degree (p.cp as p.f-powers))] @ roots @ [hd (last ls)] @ rev ys
  using replay tr-prover final-send-res tr-s6 tr-s5 send-degree-res alpha-data
    trace-final-res tr-trace-fri send-f-res tr-s1 cp'-def
  unfolding verifier-replay-state-def by simp
have st-s1: PState s1 = 0
  using create-preserves-state[OF create-f]
  unfolding init-state-def by simp
have st-trace-final:
  PState s-trace-final =
    concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
    (hd (last f-ls))
  using st-s1 send-f-res st-trace-fri trace-final-res replay
  unfolding verifier-replay-state-def by simp
have st-s5:
  PState s5 =
    concat
    (foldl concat
      (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
        (hd (last f-ls)))
      as)
    (of-nat (degree (p.cp as p.f-powers)))
  using st-trace-final alpha-data send-degree-res create-preserves-state[OF create-cp] cp'-def
  by simp
have trace-counter-s1-zero: PTraceFriCounter s1 = 0
  using p.create-preserves-channel(3)[OF create-f]
  unfolding init-state-def by simp
have trace-counter-s2-zero: PTraceFriCounter s2 = 0
  using send-f-res trace-counter-s1-zero by simp
have alpha-counter-s1-zero: PAlphaCounter s1 = 0
  using p.create-preserves-channel(5)[OF create-f]
  unfolding init-state-def by simp
have alpha-counter-s2-zero: PAlphaCounter s2 = 0
  using send-f-res alpha-counter-s1-zero by simp
have alpha-counter-trace-fri-zero: PAlphaCounter s-trace-fri = 0
  using trace-fri-commit-preserves-alpha-counter[OF trace-fri]
    alpha-counter-s2-zero by simp
have alpha-counter-trace-final-zero: PAlphaCounter s-trace-final = 0

```

```

using trace-final-res alpha-counter-trace-fri-zero by simp
have comp-counter-s1-zero: PCompositionFriCounter s1 = 0
  using p.create-preserves-channel(4)[OF create-f]
  unfolding init-state-def by simp
have comp-counter-s2-zero: PCompositionFriCounter s2 = 0
  using send-f-res comp-counter-s1-zero by simp
have comp-counter-trace-fri-zero: PCompositionFriCounter s-trace-fri = 0
  using trace-fri-commit-preserves-composition-counter[OF trace-fri]
    comp-counter-s2-zero by simp
have comp-counter-trace-final-zero:
  PCompositionFriCounter s-trace-final = 0
  using trace-final-res comp-counter-trace-fri-zero by simp
have comp-counter-s3-zero: PCompositionFriCounter s3 = 0
  using alpha-data comp-counter-trace-final-zero by simp
have comp-counter-s4-zero: PCompositionFriCounter s4 = 0
  using send-degree-res comp-counter-s3-zero by simp
have comp-counter-s5-zero: PCompositionFriCounter s5 = 0
  using p.create-preserves-channel(4)[OF create-cp] comp-counter-s4-zero
  by simp
have f-fri-lookups:
   $\forall i < \text{length } f\text{-roots}.$ 
  fmlookup (HashMap replay-state)
    (TraceFriChallenge (PTraceFriCounter replay-state + i) (foldl concat
      (concat (PState replay-state) (value f-merkle))
      (take (Suc i) f-roots))) =
    Some (f-challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length f-roots
  have lookup-trace:
    fmlookup (HashMap s-trace-fri)
      (TraceFriChallenge (PTraceFriCounter s2 + i)
        (foldl concat (PState s2) (take (Suc i) f-roots))) =
      Some (f-challenges ! i)
    using trace-fri-lookups-state i-bound by simp
  have lookup-prover:
    fmlookup (HashMap prover-state)
      (TraceFriChallenge (PTraceFriCounter s2 + i)
        (foldl concat (PState s2) (take (Suc i) f-roots))) =
      Some (f-challenges ! i)
    using p.hash-extension-lookup[OF lookup-trace trace-fri-prover] .
  show fmlookup (HashMap replay-state)
    (TraceFriChallenge (PTraceFriCounter replay-state + i) (foldl concat
      (concat (PState replay-state) (value f-merkle))
      (take (Suc i) f-roots))) =
    Some (f-challenges ! i)
    using lookup-prover send-f-res st-s1 replay trace-counter-s2-zero
  unfolding verifier-replay-state-def by simp
qed

```



```

have alpha-lookups:
   $\forall i < \text{length } as.$ 
  fmlookup (HashMap replay-state)
    (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat
      (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
        (hd (last f-ls)))
      (take i as))) =
    Some (as ! i)
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{length } as$ 
  have alpha-lookups-s3:
     $\forall i < \text{length } as.$ 
    fmlookup (HashMap s3)
      (AlphaChallenge (PAlphaCounter s-trace-final + i)
        (foldl concat (PState s-trace-final) (take i as))) =
      Some (as ! i)
    using alpha-data by simp
  have lookup-s3:
    fmlookup (HashMap s3)
      (AlphaChallenge (PAlphaCounter s-trace-final + i)
        (foldl concat (PState s-trace-final) (take i as))) =
      Some (as ! i)
    using alpha-lookups-s3 i-bound by simp
  have lookup-prover:
    fmlookup (HashMap prover-state)
      (AlphaChallenge (PAlphaCounter s-trace-final + i)
        (foldl concat (PState s-trace-final) (take i as))) =
      Some (as ! i)
    using p.hash-extension-lookup[OF lookup-s3 s3-prover] .
  show fmlookup (HashMap replay-state)
    (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat
      (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
        (hd (last f-ls)))
      (take i as))) =
    Some (as ! i)
    using lookup-prover st-trace-final replay alpha-counter-trace-final-zero
    unfolding verifier-replay-state-def by simp
qed
have fri-lookups:
   $\forall i < \text{length } roots.$ 
  fmlookup (HashMap replay-state)
    (CompositionFriChallenge (PCompositionFriCounter replay-state + i) (foldl concat
      (concat
        (foldl concat
          (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
            (hd (last f-ls)))
          as)
        (of-nat (degree (p.cp as p.f-powers))))))

```

```

      (take (Suc i) roots))) =
    Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have lookup-s6:
    fmlookup (HashMap s6)
      (CompositionFriChallenge
        (PCompositionFriCounter s5 + i)
        (foldl concat (PState s5) (take (Suc i) roots))) =
      Some (challenges ! i)
  using fri-lookups-s6 i-bound by simp
  have lookup-prover:
    fmlookup (HashMap prover-state)
      (CompositionFriChallenge
        (PCompositionFriCounter s5 + i)
        (foldl concat (PState s5) (take (Suc i) roots))) =
      Some (challenges ! i)
  using p.hash-extension-lookup[OF lookup-s6 s6-prover] .
  show fmlookup (HashMap replay-state)
    (CompositionFriChallenge (PCompositionFriCounter replay-state + i) (foldl concat
      (concat
        (foldl concat
          (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
            (hd (last f-ls)))
          as)
        (of-nat (degree (p.cp as p.f-powers))))
      (take (Suc i) roots))) =
    Some (challenges ! i)
  using lookup-prover st-s5 replay comp-counter-s5-zero
  unfolding verifier-replay-state-def by simp
qed
have f-roots-rounds: length f-roots = ceil-log clength
  using f-len-roots f-nrounds-def by simp
have f-len-challenges-roots: length f-challenges = length f-roots
  using f-len-challenges f-len-roots by simp
have roots-rounds:
  length roots = ceil-log (degree (p.cp as p.f-powers) + 1)
  using len-roots nrounds-def cp'-def by simp
have len-challenges-roots: length challenges = length roots
  using len-challenges len-roots by simp
have prefix-res:
  fr = value f-merkle ∧
  f-fl = zip f-challenges f-roots ∧
  f-final = hd (last f-ls) ∧
  as' = as ∧
  dg = of-nat (degree (p.cp as p.f-powers)) ∧
  fl = zip challenges roots ∧
  final = hd (last ls) ∧

```

```

PState t =
  concat
    (foldl concat
      (concat
        (foldl concat
          (concat (foldl concat (concat (PState replay-state) (value f-merkle)) f-roots)
            (hd (last f-ls)))
          as)
        (of-nat (degree (p.cp as p.f-powers))))
      roots)
    (hd (last ls)) ∧
PTranscript t = rev ys ∧
replay-state ≤ t
by (rule honest-trace-root-alpha-degree-fri-final-dynamic-prefix-outcome[
  OF htv conjunct1[OF alpha-data] f-len-challenges-roots f-roots-rounds
  len-challenges-roots roots-rounds replay-prefix f-fri-lookups
  alpha-lookups fri-lookups outcome])
have st-t-final: PState t = PState s-final
  using prefix-res st-s5 st-s6 final-send-res by simp
show ?thesis
  by (rule that[
    OF f-len-roots f-len-challenges - - - len-roots len-challenges - - -
    - - - st-t-final - tr-prover -])
    (use prefix-res f-roots-aligned-all f-successors-all roots-aligned-all successors-all in simp-all)
qed

```

lemma *honest-root-alpha-degree-from-prover-no-failure:*

```

assumes htv: honest-trace-valid
and create-f:
  Some (f-merkle, s1) ∈
    set-dist (execute (p.create p.f-eval) init-state)
and created-f: p.created-tree p.f-eval f-merkle s1
and send-f:
  Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and alphas:
  Some (as, s3) ∈
    set-dist (execute
      (mmap (replicate (length spec)
        (do {
          a ← p.receive-alpha-challenge;
          p.send a;
          return a
        }))) s2)
and cp'-def: cp' = p.cp as p.f-powers
and send-degree:
  Some ((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp'))) s3)
and create-cp:
  Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
and fri:

```

```

    Some ((ps, ds, ls, ms), s6) ∈
      set-dist (execute
        (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
  and final-send:
    Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
  and random-idx:
    Some (idx, s7) ∈ set-dist (execute p.receive-query-index-challenge s-final)
  and idx-def: idx' = p.index (to-nat idx)
  and query-decommit:
    Some (qouts, s8) ∈
      set-dist (execute (mmap (p.decommit-on-query idx' f-merkle)) s7)
  and fri-decommit:
    Some (fri-idx, s9) ∈
      set-dist (execute
        (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
  and prover-state-eq: prover-state = s9
  and replay: replay-state = verifier-replay-state prover-state
shows
  None ∉ dom (dist (execute
    (do {
      fr ← p.read;
      as' ← mmap (replicate (length spec)
        (do {
          a0 ← p.receive-alpha-challenge;
          let a0' = a0;
          a1 ← p.read;
          let a1' = a1;
          assert (a0' = a1');
          return a1'
        })));
      dg ← p.read;
      assert (to-nat dg ≤ p.maxDegree);
      return (fr, as', dg)
    }) replay-state))
proof –
  have root-alpha-tr:
    length as = length spec ∧
    PTranscript s4 = of-nat (degree cp') # rev as @ [value f-merkle]
  by (rule prover-root-alpha-degree-transcript[
    OF create-f send-f alphas cp'-def send-degree])
  have later-tr: transcript-extends prover-state s4
  by (rule prover-later-phases-transcript-extends[
    OF create-cp fri final-send random-idx query-decommit fri-decommit prover-state-eq])
  from verifier-replay-root-alpha-degree-prefix[
    OF conjunct2[OF root-alpha-tr] later-tr replay]
  obtain rest where prefix:
    PTranscript replay-state =
      value f-merkle # as @ [of-nat (degree cp')] @ rest
  by blast

```

```

have prefix-cp:
  PTranscript replay-state =
    value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @ rest
  using prefix cp'-def by simp
have later-hash: s3 ≤ prover-state
  by (rule prover-later-phases-hash-extends[
    OF send-degree create-cp fri final-send random-idx query-decommit fri-decommit prover-state-eq])
have lookups:
  ∀ i < length as.
    fmllookup (HashMap replay-state)
      (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat (concat (PState re-
        play-state) (value f-merkle)) (take i as))) =
      Some (as ! i)
  by (rule prover-alpha-lookups-in-replay-state[
    OF create-f send-f alphas later-hash replay])
show ?thesis
  by (rule honest-root-alpha-degree-prefix-no-failure[
    OF htv conjunct1[OF root-alpha-tr] prefix-cp lookups])
qed

```

```

lemma honest-root-alpha-degree-from-prover-outcome:
assumes htv: honest-trace-valid
and create-f:
  Some (f-merkle, s1) ∈
    set-dist (execute (p.create p.f-eval) init-state)
and send-f:
  Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and alphas:
  Some (as, s3) ∈
    set-dist (execute
      (mmap (replicate (length spec)
        (do {
          a ← p.receive-alpha-challenge;
          p.send a;
          return a
        }))) s2)
and cp'-def: cp' = p.cp as p.f-powers
and send-degree:
  Some ((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp'))) s3)
and create-cp:
  Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
and fri:
  Some ((ps, ds, ls, ms), s6) ∈
    set-dist (execute
      (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
and final-send:
  Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
and random-idx:
  Some (idx, s7) ∈ set-dist (execute p.receive-query-index-challenge s-final)

```

```

and idx-def: idx' = p.index (to-nat idx)
and query-decommit:
  Some (qouts, s8) ∈
    set-dist (execute (mmap (p.decommit-on-query idx' f-merkle)) s7)
and fri-decommit:
  Some (fri-idx, s9) ∈
    set-dist (execute
      (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
and prover-state-eq: prover-state = s9
and replay: replay-state = verifier-replay-state prover-state
and outcome:
  Some ((fr, as', dg), t) ∈ set-dist (execute
    (do {
      fr ← p.read;
      as' ← mmap (replicate (length spec)
        (do {
          a0 ← p.receive-alpha-challenge;
          let a0' = a0;
          a1 ← p.read;
          let a1' = a1;
          assert (a0' = a1');
          return a1'
        })));
      dg ← p.read;
      assert (to-nat dg ≤ p.maxDegree);
      return (fr, as', dg)
    }) replay-state)
shows
  fr = value f-merkle ∧
  as' = as ∧
  dg = of-nat (degree (p.cp as p.f-powers)) ∧
  replay-state ≤ t
proof –
  have root-alpha-tr:
    length as = length spec ∧
    PTranscript s4 = of-nat (degree cp') # rev as @ [value f-merkle]
  by (rule prover-root-alpha-degree-transcript[
    OF create-f send-f alphas cp'-def send-degree])
  have later-tr: transcript-extends prover-state s4
  by (rule prover-later-phases-transcript-extends[
    OF create-cp fri final-send random-idx query-decommit fri-decommit prover-state-eq])
  from verifier-replay-root-alpha-degree-prefix[
    OF conjunct2[OF root-alpha-tr] later-tr replay]
  obtain rest where prefix:
    PTranscript replay-state =
      value f-merkle # as @ [of-nat (degree cp')] @ rest
  by blast
  have prefix-cp:
    PTranscript replay-state =

```

```

    value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @ rest
using prefix cp'-def by simp
have later-hash: s3 ≤ prover-state
by (rule prover-later-phases-hash-extends[
    OF send-degree create-cp fri final-send random-idx query-decommit fri-decommit prover-state-eq])
have lookups:
  ∀ i < length as.
    fmllookup (HashMap replay-state)
      (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat (concat (PState re-
play-state) (value f-merkle)) (take i as))) =
      Some (as ! i)
by (rule prover-alpha-lookups-in-replay-state[
    OF create-f send-f alphas later-hash replay])
have out:
  fr = value f-merkle ∧
  as' = as ∧
  dg = of-nat (degree (p.cp as p.f-powers)) ∧
  PState t =
    concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
      (of-nat (degree (p.cp as p.f-powers))) ∧
  PTranscript t = rest ∧
  replay-state ≤ t
by (rule honest-root-alpha-degree-prefix-outcome[
    OF htv conjunct1[OF root-alpha-tr] prefix-cp lookups outcome])
then show ?thesis by simp
qed

```

lemma honest-root-alpha-degree-fri-from-prover-no-failure:

```

assumes htv: honest-trace-valid
and create-f:
  Some (f-merkle, s1) ∈
    set-dist (execute (p.create p.f-eval) init-state)
and send-f:
  Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and alphas:
  Some (as, s3) ∈
    set-dist (execute
      (mmap (replicate (length spec)
        (do {
          a ← p.receive-alpha-challenge;
          p.send a;
          return a
        }))) s2)
and cp'-def: cp' = p.cp as p.f-powers
and send-degree:
  Some ((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp'))) s3)
and create-cp:
  Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
and fri:

```

```

    Some ((ps, ds, ls, ms), s6) ∈
      set-dist (execute
        (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
  and final-send:
    Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
  and random-idx:
    Some (idx, s7) ∈ set-dist (execute p.receive-query-index-challenge s-final)
  and idx-def: idx' = p.index (to-nat idx)
  and query-decommit:
    Some (qouts, s8) ∈
      set-dist (execute (mmap (p.decommit-on-query idx' f-merkle)) s7)
  and fri-decommit:
    Some (fri-idx, s9) ∈
      set-dist (execute
        (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
  and prover-state-eq: prover-state = s9
  and replay: replay-state = verifier-replay-state prover-state
shows
  None ∉ dom (dist (execute
    (do {
      fr ← p.read;
      as' ← mmap (replicate (length spec)
        (do {
          a0 ← p.receive-alpha-challenge;
          let a0' = a0;
          a1 ← p.read;
          let a1' = a1;
          assert (a0' = a1');
          return a1'
        })));
      dg ← p.read;
      assert (to-nat dg ≤ p.maxDegree);
      fl ← ntimes v.receive-fri-commits nrounds;
      return (fr, as', dg, fl)
    }) replay-state))
proof -
  have root-alpha-tr:
    length as = length spec ∧
    PTranscript s4 = of-nat (degree cp') # rev as @ [value f-merkle]
  by (rule prover-root-alpha-degree-transcript[
    OF create-f send-f alphas cp'-def send-degree])
  have later-hash: s3 ≤ prover-state
  by (rule prover-later-phases-hash-extends[
    OF send-degree create-cp fri final-send random-idx query-decommit fri-decommit prover-state-eq])
  have alpha-lookups:
    ∀ i < length as.
      fmllookup (HashMap replay-state)
        (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat (concat (PState re-
          play-state) (value f-merkle)) (take i as))) =

```



```

    Some (as ! i)
  by (rule prover-alpha-lookups-in-replay-state[
    OF create-f send-f alphas later-hash replay])
from fri-commit-replay-data[OF fri] obtain roots challenges where
  len-roots: length roots = nrounds
  and len-challenges: length challenges = nrounds
  and tr-s6: PTranscript s6 = rev roots @ PTranscript s5
  and st-s6: PState s6 = foldl concat (PState s5) roots
  and fri-hash: s5 ≤ s6
  and fri-lookups-s6:
    ∀ i < length roots.
      fmllookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i)
roots))) =
        Some (challenges ! i)
  by blast
have after-tr: transcript-extends prover-state s6
  by (rule prover-after-fri-transcript-extends[
    OF final-send random-idx query-decommit fri-decommit prover-state-eq])
from after-tr obtain xs where
  tr-prover: PTranscript prover-state = xs @ PTranscript s6
  unfolding transcript-extends-def by auto
have tr-s5: PTranscript s5 = PTranscript s4
  using create-preserves-transcript[OF create-cp] .
have replay-prefix:
  PTranscript replay-state =
    value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))]] @ roots @ rev xs
  using replay tr-prover tr-s6 tr-s5 root-alpha-tr cp'-def
  unfolding verifier-replay-state-def by simp
have st-s1: PState s1 = 0
  using create-preserves-state[OF create-f]
  unfolding init-state-def by simp
have send-f-res:
  s2 = s1 (|
    PState := concat (PState s1) (value f-merkle),
    PTranscript := value f-merkle # PTranscript s1 |)
  using p.send-outcome[OF send-f] .
have alpha-data:
  PState s3 = foldl concat (PState s2) as
  using alpha-mmap-outcome[OF alphas] by simp
have send-degree-res:
  s4 = s3 (|
    PState := concat (PState s3) (of-nat (degree cp')),
    PTranscript := of-nat (degree cp') # PTranscript s3 |)
  using p.send-outcome[OF send-degree] .
have st-s5:
  PState s5 =
    concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
      (of-nat (degree (p.cp as p.f-powers)))
  using st-s1 send-f-res alpha-data send-degree-res

```

```

    create-preserves-state[OF create-cp] replay cp'-def
  unfolding verifier-replay-state-def by simp
have after-hash: s6 ≤ prover-state
  by (rule prover-after-fri-hash-extends[
    OF final-send random-idx query-decommit fri-decommit prover-state-eq])
have fri-lookups:
  ∀ i < length roots.
    fmlookup (HashMap replay-state)
      (FiatShamirChallenge (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          (of-nat (degree (p.cp as p.f-powers))))
        (take (Suc i) roots))) =
      Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have lookup-s6:
    fmlookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i)
roots))) =
    Some (challenges ! i)
  using fri-lookups-s6 i-bound by simp
  have lookup-prover:
    fmlookup (HashMap prover-state) (FiatShamirChallenge (foldl concat (PState s5) (take
(Suc i) roots))) =
    Some (challenges ! i)
  using p.hash-extension-lookup[OF lookup-s6 after-hash] .
show fmlookup (HashMap replay-state)
  (FiatShamirChallenge (foldl concat
    (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
      (of-nat (degree (p.cp as p.f-powers))))
    (take (Suc i) roots))) =
    Some (challenges ! i)
  using lookup-prover st-s5 replay
  unfolding verifier-replay-state-def by simp
qed
have nf-roots:
  None ∉ dom (dist (execute
    (do {
      fr ← p.read;
      as' ← mmap (replicate (length spec)
        (do {
          a0 ← p.receive-alpha-challenge;
          let a0' = a0;
          a1 ← p.read;
          let a1' = a1;
          assert (a0' = a1');
          return a1'
        })));
      dg ← p.read;

```

```

    assert (to-nat dg ≤ p.maxDegree);
    fl ← ntimes v.receive-fri-commits (length roots);
    return (fr, as', dg, fl)
  }) replay-state))
by (rule honest-root-alpha-degree-fri-prefix-no-failure[
  OF htv conjunct1[OF root-alpha-tr] - replay-prefix alpha-lookups fri-lookups])
  (use len-roots len-challenges in simp-all)
show ?thesis
  using nf-roots len-roots by simp
qed

lemma honest-root-alpha-degree-fri-from-prover-tail-no-failure:
  assumes htv: honest-trace-valid
  and create-f:
    Some (f-merkle, s1) ∈
      set-dist (execute (p.create p.f-eval) init-state)
  and send-f:
    Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
  and alphas:
    Some (as, s3) ∈
      set-dist (execute
        (mmap (replicate (length spec)
          (do {
            a ← p.receive-alpha-challenge;
            p.send a;
            return a
          }))) s2)
  and cp'-def: cp' = p.cp as p.f-powers
  and send-degree:
    Some ((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp'))) s3)
  and create-cp:
    Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
  and fri:
    Some ((ps, ds, ls, ms), s6) ∈
      set-dist (execute
        (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
  and final-send:
    Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
  and tail:
    Some (prover-result, prover-state) ∈
      set-dist (execute (ntimes (prover-query-round f-merkle f-ls f-ms ls ms) rounds) s-final)
  and replay: replay-state = verifier-replay-state prover-state
shows
  None ∉ dom (dist (execute
    (do {
      fr ← p.read;
      as' ← mmap (replicate (length spec)
        (do {
          a0 ← p.receive-alpha-challenge;

```

```

    let a0' = a0;
    a1 ← p.read;
    let a1' = a1;
    assert (a0' = a1');
    return a1'
  }));
  dg ← p.read;
  assert (to-nat dg ≤ p.maxDegree);
  fl ← ntimes v.receive-fri-commits nrounds;
  return (fr, as', dg, fl)
}) replay-state))
proof -
  have root-alpha-tr:
    length as = length spec ∧
    PTranscript s4 = of-nat (degree cp') # rev as @ [value f-merkle]
  by (rule prover-root-alpha-degree-transcript[
    OF create-f send-f alphas cp'-def send-degree])
  have tail-hash: s-final ≤ prover-state
  by (rule ntimes-hash-extends[OF - tail])
    (rule prover-query-round-hash-extends)
  have s3-s4: s3 ≤ s4
  using send-hash-extends[OF send-degree] .
  have s4-s5: s4 ≤ s5
  using create-hash-extends[OF create-cp] .
  have s5-s6: s5 ≤ s6
  using honest-fri-commit-extends[OF fri] .
  have s6-final-hash: s6 ≤ s-final
  using send-hash-extends[OF final-send] .
  have later-hash: s3 ≤ prover-state
  using s3-s4 s4-s5 s5-s6 s6-final-hash tail-hash by (meson p.hash-ext-trans)
  have alpha-lookups:
    ∀ i < length as.
    fmlookup (HashMap replay-state)
      (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat (concat (PState replay-state) (value f-merkle)) (take i as))) =
      Some (as ! i)
  by (rule prover-alpha-lookups-in-replay-state[
    OF create-f send-f alphas later-hash replay])
  from fri-commit-replay-data[OF fri] obtain roots challenges where
    len-roots: length roots = nrounds
  and len-challenges: length challenges = nrounds
  and tr-s6: PTranscript s6 = rev roots @ PTranscript s5
  and st-s6: PState s6 = foldl concat (PState s5) roots
  and fri-hash: s5 ≤ s6
  and fri-lookups-s6:
    ∀ i < length roots.
    fmlookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i) roots))) =
    Some (challenges ! i)

```

```

    by blast
have s6-final-tr: transcript-extends s-final s6
  using send-transcript-extends[OF final-send] .
have final-tail-tr: transcript-extends prover-state s-final
  by (rule ntimes-transcript-extends[OF - tail])
    (rule prover-query-round-transcript-extends)
have after-tr: transcript-extends prover-state s6
  using s6-final-tr final-tail-tr by (meson transcript-extends-trans)
from after-tr obtain xs where
  tr-prover: PTranscript prover-state = xs @ PTranscript s6
  unfolding transcript-extends-def by auto
have tr-s5: PTranscript s5 = PTranscript s4
  using create-preserves-transcript[OF create-cp] .
have replay-prefix:
  PTranscript replay-state =
    value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @ roots @ rev xs
  using replay tr-prover tr-s6 tr-s5 root-alpha-tr cp'-def
  unfolding verifier-replay-state-def by simp
have st-s1: PState s1 = 0
  using create-preserves-state[OF create-f]
  unfolding init-state-def by simp
have send-f-res:
  s2 = s1(
    PState := concat (PState s1) (value f-merkle),
    PTranscript := value f-merkle # PTranscript s1)
  using p.send-outcome[OF send-f] .
have alpha-data:
  PState s3 = foldl concat (PState s2) as
  using alpha-mmap-outcome[OF alphas] by simp
have send-degree-res:
  s4 = s3(
    PState := concat (PState s3) (of-nat (degree cp')),
    PTranscript := of-nat (degree cp') # PTranscript s3)
  using p.send-outcome[OF send-degree] .
have st-s5:
  PState s5 =
    concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
      (of-nat (degree (p.cp as p.f-powers)))
  using st-s1 send-f-res alpha-data send-degree-res
    create-preserves-state[OF create-cp] replay cp'-def
  unfolding verifier-replay-state-def by simp
have after-hash: s6 ≤ prover-state
  using s6-final-hash tail-hash by (rule p.hash-ext-trans)
have fri-lookups:
  ∀ i < length roots.
    fmlookup (HashMap replay-state)
      (FiatShamirChallenge (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          (of-nat (degree (p.cp as p.f-powers))))))

```

```

      (take (Suc i) roots))) =
    Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have lookup-s6:
    fmlookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i)
roots))) =
      Some (challenges ! i)
  using fri-lookups-s6 i-bound by simp
  have lookup-prover:
    fmlookup (HashMap prover-state) (FiatShamirChallenge (foldl concat (PState s5) (take
(Suc i) roots))) =
      Some (challenges ! i)
  using p.hash-extension-lookup[OF lookup-s6 after-hash] .
show fmlookup (HashMap replay-state)
(FiatShamirChallenge (foldl concat
  (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
    (of-nat (degree (p.cp as p.f-powers)))))
  (take (Suc i) roots))) =
  Some (challenges ! i)
  using lookup-prover st-s5 replay
  unfolding verifier-replay-state-def by simp
qed
have nf-roots:
  None  $\notin$  dom (dist (execute
    (do {
      fr  $\leftarrow$  p.read;
      as'  $\leftarrow$  mmap (replicate (length spec)
        (do {
          a0  $\leftarrow$  p.receive-alpha-challenge;
          let a0' = a0;
          a1  $\leftarrow$  p.read;
          let a1' = a1;
          assert (a0' = a1');
          return a1'
        })))
      dg  $\leftarrow$  p.read;
      assert (to-nat dg  $\leq$  p.maxDegree);
      fl  $\leftarrow$  ntimes v.receive-fri-commits (length roots);
      return (fr, as', dg, fl)
    }) replay-state))
  by (rule honest-root-alpha-degree-fri-prefix-no-failure[
    OF htv conjunct1[OF root-alpha-tr] - replay-prefix alpha-lookups fri-lookups])
    (use len-roots len-challenges in simp-all)
show ?thesis
  using nf-roots len-roots by simp
qed

```

lemma *honest-root-alpha-degree-fri-final-dynamic-from-prover-tail-no-failure:*
assumes *htv: honest-trace-valid*
and *create-f:*
 Some (f-merkle, s1) ∈
 set-dist (execute (p.create p.f-eval) init-state)
and *send-f:*
 Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and *alphas:*
 Some (as, s3) ∈
 set-dist (execute
 (mmap (replicate (length spec)
 (do {
 a ← p.receive-alpha-challenge;
 p.send a;
 return a
 }})) s2)
and *cp'-def: cp' = p.cp as p.f-powers*
and *send-degree:*
 Some ((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp'))) s3)
and *create-cp:*
 Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
and *nrounds-def: nrounds = ceil-log (degree cp' + 1)*
and *fri:*
 Some ((ps, ds, ls, ms), s6) ∈
 set-dist (execute
 (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
and *final-send:*
 Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
and *tail:*
 Some (prover-result, prover-state) ∈
 set-dist (execute (ntimes (prover-query-round f-merkle f-ls f-ms ls ms) rounds) s-final)
and *replay: replay-state = verifier-replay-state prover-state*
shows
 None ∉ dom (dist (execute
 (do {
 fr ← p.read;
 as' ← mmap (replicate (length spec)
 (do {
 a0 ← p.receive-alpha-challenge;
 let a0' = a0;
 a1 ← p.read;
 let a1' = a1;
 assert (a0' = a1');
 return a1'
 }}));
 dg ← p.read;
 assert (to-nat dg ≤ p.maxDegree);
 fl ← ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1));
 final ← p.read;

```

    return (fr, as', dg, fl, final)
  }) replay-state))
proof -
  have root-alpha-tr:
    length as = length spec ∧
    PTranscript s4 = of-nat (degree cp') # rev as @ [value f-merkle]
  by (rule prover-root-alpha-degree-transcript[
    OF create-f send-f alphas cp'-def send-degree])
  have tail-hash: s-final ≤ prover-state
  by (rule ntimes-hash-extends[OF - tail])
    (rule prover-query-round-hash-extends)
  have s3-s4: s3 ≤ s4
  using send-hash-extends[OF send-degree] .
  have s4-s5: s4 ≤ s5
  using create-hash-extends[OF create-cp] .
  have s5-s6: s5 ≤ s6
  using honest-fri-commit-extends[OF fri] .
  have s6-final-hash: s6 ≤ s-final
  using send-hash-extends[OF final-send] .
  have later-hash: s3 ≤ prover-state
  using s3-s4 s4-s5 s5-s6 s6-final-hash tail-hash by (meson p.hash-ext-trans)
  have alpha-lookups:
    ∀ i < length as.
    fmlookup (HashMap replay-state)
      (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat (concat (PState re-
        play-state) (value f-merkle)) (take i as))) =
      Some (as ! i)
  by (rule prover-alpha-lookups-in-replay-state[
    OF create-f send-f alphas later-hash replay])
  from fri-commit-replay-data[OF fri] obtain roots challenges where
    len-roots: length roots = nrounds
    and len-challenges: length challenges = nrounds
    and tr-s6: PTranscript s6 = rev roots @ PTranscript s5
    and st-s6: PState s6 = foldl concat (PState s5) roots
    and fri-lookups-s6:
      ∀ i < length roots.
      fmlookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i)
        roots))) =
        Some (challenges ! i)
  by blast
  have final-tail-tr: transcript-extends prover-state s-final
  by (rule ntimes-transcript-extends[OF - tail])
    (rule prover-query-round-transcript-extends)
  from final-tail-tr obtain ys where
    tr-prover: PTranscript prover-state = ys @ PTranscript s-final
  unfolding transcript-extends-def by auto
  have tr-s5: PTranscript s5 = PTranscript s4
  using create-preserves-transcript[OF create-cp] .
  have final-send-res:

```



```

s-final = s6(
  PState := concat (PState s6) (hd (last ls)),
  PTranscript := hd (last ls) # PTranscript s6)
using p.send-outcome[OF final-send] .
have replay-prefix:
  PTranscript replay-state =
    value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @
      roots @ [hd (last ls)] @ rev ys
  using replay tr-prover final-send-res tr-s6 tr-s5 root-alpha-tr cp'-def
  unfolding verifier-replay-state-def by simp
have st-s1: PState s1 = 0
  using create-preserves-state[OF create-f]
  unfolding init-state-def by simp
have send-f-res:
  s2 = s1(
    PState := concat (PState s1) (value f-merkle),
    PTranscript := value f-merkle # PTranscript s1)
  using p.send-outcome[OF send-f] .
have alpha-data:
  PState s3 = foldl concat (PState s2) as
  using alpha-mmap-outcome[OF alphas] by simp
have send-degree-res:
  s4 = s3(
    PState := concat (PState s3) (of-nat (degree cp')),
    PTranscript := of-nat (degree cp') # PTranscript s3)
  using p.send-outcome[OF send-degree] .
have st-s5:
  PState s5 =
    concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
      (of-nat (degree (p.cp as p.f-powers))))
  using st-s1 send-f-res alpha-data send-degree-res
  create-preserves-state[OF create-cp] replay cp'-def
  unfolding verifier-replay-state-def by simp
have after-hash: s6 ≤ prover-state
  using s6-final-hash tail-hash by (rule p.hash-ext-trans)
have fri-lookups:
  ∀ i < length roots.
    fmlookup (HashMap replay-state)
      (FiatShamirChallenge (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          (of-nat (degree (p.cp as p.f-powers))))))
        (take (Suc i) roots))) =
      Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have lookup-s6:
    fmlookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i)
    roots))) =

```

```

    Some (challenges ! i)
  using fri-lookups-s6 i-bound by simp
have lookup-prover:
  fmlookup (HashMap prover-state) (FiatShamirChallenge (foldl concat (PState s5) (take
(Suc i) roots))) =
    Some (challenges ! i)
  using p.hash-extension-lookup[OF lookup-s6 after-hash] .
show fmlookup (HashMap replay-state)
(FiatShamirChallenge (foldl concat
  (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
    (of-nat (degree (p.cp as p.f-powers)))))
  (take (Suc i) roots))) =
  Some (challenges ! i)
  using lookup-prover st-s5 replay
  unfolding verifier-replay-state-def by simp
qed
have roots-rounds:
  length roots = ceil-log (degree (p.cp as p.f-powers) + 1)
  using len-roots nrounds-def cp'-def by simp
have len-challenges-roots: length challenges = length roots
  using len-challenges len-roots by simp
show ?thesis
  by (rule honest-root-alpha-degree-fri-final-dynamic-prefix-no-failure[
    OF htv conjunct1[OF root-alpha-tr] len-challenges-roots roots-rounds
    replay-prefix alpha-lookups fri-lookups])
qed

```

lemma *honest-root-alpha-degree-fri-final-dynamic-from-prover-tail-outcome:*

```

assumes htv: honest-trace-valid
and create-f:
  Some (f-merkle, s1) ∈
    set-dist (execute (p.create p.f-eval) init-state)
and send-f:
  Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and alphas:
  Some (as, s3) ∈
    set-dist (execute
      (mmap (replicate (length spec)
        (do {
          a ← p.receive-alpha-challenge;
          p.send a;
          return a
        }))) s2)
and cp'-def: cp' = p.cp as p.f-powers
and send-degree:
  Some ((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp'))) s3)
and create-cp:
  Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
and nrounds-def: nrounds = ceil-log (degree cp' + 1)

```

```

and fri:
  Some ((ps, ds, ls, ms), s6) ∈
    set-dist (execute
      (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
and final-send:
  Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls)))) s6)
and tail:
  Some (prover-result, prover-state) ∈
    set-dist (execute (ntimes (prover-query-round f-merkle f-ls f-ms ls ms) rounds) s-final)
and replay: replay-state = verifier-replay-state prover-state
and outcome:
  Some ((fr, as', dg, fl, final), t) ∈ set-dist (execute
    (do {
      fr ← p.read;
      as' ← mmap (replicate (length spec)
        (do {
          a0 ← p.receive-alpha-challenge;
          let a0' = a0;
          a1 ← p.read;
          let a1' = a1;
          assert (a0' = a1');
          return a1'
        }));
      dg ← p.read;
      assert (to-nat dg ≤ p.maxDegree);
      fl ← ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1));
      final ← p.read;
      return (fr, as', dg, fl, final)
    }) replay-state)
obtains roots challenges ys where
  length roots = nrounds
  length challenges = nrounds
  fl = zip challenges roots
   $\bigwedge j. j < \text{nrounds} \implies \text{roots } ! j = \text{value } (\text{ms } ! j)$ 
   $\bigwedge j. j < \text{nrounds} \implies$ 
    p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
      (ps ! Suc j, ds ! Suc j, ls ! Suc j)
  fr = value f-merkle
  as' = as
  dg = of-nat (degree (p.cp as p.f-powers))
  final = hd (last ls)
  PState t = PState s-final
  PTranscript t = rev ys
  PTranscript prover-state = ys @ PTranscript s-final
  replay-state ≤ t
proof –
have root-alpha-tr:
  length as = length spec ∧
    PTranscript s4 = of-nat (degree cp') # rev as @ [value f-merkle]

```

```

    by (rule prover-root-alpha-degree-transcript[
      OF create-f send-f alphas cp'-def send-degree])
  have tail-hash:  $s\text{-final} \leq \text{prover-state}$ 
    by (rule ntimes-hash-extends[OF - tail])
      (rule prover-query-round-hash-extends)
  have  $s3\text{-}s4$ :  $s3 \leq s4$ 
    using send-hash-extends[OF send-degree] .
  have  $s4\text{-}s5$ :  $s4 \leq s5$ 
    using create-hash-extends[OF create-cp] .
  have  $s5\text{-}s6$ :  $s5 \leq s6$ 
    using honest-fri-commit-extends[OF fri] .
  have  $s6\text{-final-hash}$ :  $s6 \leq s\text{-final}$ 
    using send-hash-extends[OF final-send] .
  have later-hash:  $s3 \leq \text{prover-state}$ 
    using  $s3\text{-}s4$   $s4\text{-}s5$   $s5\text{-}s6$   $s6\text{-final-hash}$  tail-hash by (meson p.hash-ext-trans)
  have alpha-lookups:
     $\forall i < \text{length as}.$ 
      fmllookup (HashMap replay-state)
        (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat (concat (PState re-
play-state) (value f-merkle)) (take i as))) =
        Some (as ! i)
    by (rule prover-alpha-lookups-in-replay-state[
      OF create-f send-f alphas later-hash replay])
  from fri-commit-initial-replay-successor-data[OF fri] obtain roots challenges where
    len-roots:  $\text{length roots} = \text{nrounds}$ 
    and len-challenges:  $\text{length challenges} = \text{nrounds}$ 
    and tr-s6:  $P\text{Transcript } s6 = \text{rev roots @ } P\text{Transcript } s5$ 
    and st-s6:  $P\text{State } s6 = \text{foldl concat } (P\text{State } s5) \text{ roots}$ 
    and  $s5\text{-}s6'$ :  $s5 \leq s6$ 
    and fri-lookups-s6:
       $\forall i < \text{length roots}.$ 
        fmllookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i)
roots))) =
        Some (challenges ! i)
    and roots-aligned-all:  $\forall j < \text{nrounds}. \text{roots ! } j = \text{value } (ms ! j)$ 
    and successors-all:
       $\forall j < \text{nrounds}.$ 
        p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
        (ps ! Suc j, ds ! Suc j, ls ! Suc j)
    by blast
  have roots-aligned:
     $\bigwedge j. j < \text{nrounds} \implies \text{roots ! } j = \text{value } (ms ! j)$ 
    using roots-aligned-all by simp
  have successors:
     $\bigwedge j. j < \text{nrounds} \implies$ 
      p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
      (ps ! Suc j, ds ! Suc j, ls ! Suc j)
    using successors-all by simp
  have final-tail-tr: transcript-extends prover-state s-final

```

```

    by (rule ntimes-transcript-extends[OF - tail])
      (rule prover-query-round-transcript-extends)
  from final-tail-tr obtain ys where
    tr-prover: PTranscript prover-state = ys @ PTranscript s-final
  unfolding transcript-extends-def by auto
  have tr-s5: PTranscript s5 = PTranscript s4
    using create-preserves-transcript[OF create-cp] .
  have final-send-res:
    s-final = s6⟦
      PState := concat (PState s6) (hd (last ls)),
      PTranscript := hd (last ls) # PTranscript s6⟧
    using p.send-outcome[OF final-send] .
  have replay-prefix:
    PTranscript replay-state =
      value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @
        roots @ [hd (last ls)] @ rev ys
    using replay tr-prover final-send-res tr-s6 tr-s5 root-alpha-tr cp'-def
  unfolding verifier-replay-state-def by simp
  have st-s1: PState s1 = 0
    using create-preserves-state[OF create-f]
  unfolding init-state-def by simp
  have send-f-res:
    s2 = s1⟦
      PState := concat (PState s1) (value f-merkle),
      PTranscript := value f-merkle # PTranscript s1⟧
    using p.send-outcome[OF send-f] .
  have alpha-data:
    PState s3 = foldl concat (PState s2) as
    using alpha-mmap-outcome[OF alphas] by simp
  have send-degree-res:
    s4 = s3⟦
      PState := concat (PState s3) (of-nat (degree cp')),
      PTranscript := of-nat (degree cp') # PTranscript s3⟧
    using p.send-outcome[OF send-degree] .
  have st-s5:
    PState s5 =
      concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
        (of-nat (degree (p.cp as p.f-powers)))
    using st-s1 send-f-res alpha-data send-degree-res
      create-preserves-state[OF create-cp] replay cp'-def
  unfolding verifier-replay-state-def by simp
  have after-hash: s6 ≤ prover-state
    using s6-final-hash tail-hash by (rule p.hash-ext-trans)
  have fri-lookups:
    ∀ i < length roots.
      fmlookup (HashMap replay-state)
        (FiatShamirChallenge (foldl concat
          (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
            (of-nat (degree (p.cp as p.f-powers))))))

```

```

      (take (Suc i) roots))) =
      Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{length roots}$ 
  have lookup-s6:
    fmlookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i)
roots))) =
      Some (challenges ! i)
  using fri-lookups-s6 i-bound by simp
  have lookup-prover:
    fmlookup (HashMap prover-state) (FiatShamirChallenge (foldl concat (PState s5) (take
(Suc i) roots))) =
      Some (challenges ! i)
  using p.hash-extension-lookup[OF lookup-s6 after-hash] .
  show fmlookup (HashMap replay-state)
    (FiatShamirChallenge (foldl concat
      (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
        (of-nat (degree (p.cp as p.f-powers)))))
      (take (Suc i) roots))) =
      Some (challenges ! i)
  using lookup-prover st-s5 replay
  unfolding verifier-replay-state-def by simp
qed
have roots-rounds:
  length roots = ceil-log (degree (p.cp as p.f-powers) + 1)
  using len-roots nrounds-def cp'-def by simp
have len-challenges-roots: length challenges = length roots
  using len-challenges len-roots by simp
have prefix-res:
  fr = value f-merkle  $\wedge$ 
  as' = as  $\wedge$ 
  dg = of-nat (degree (p.cp as p.f-powers))  $\wedge$ 
  fl = zip challenges roots  $\wedge$ 
  final = hd (last ls)  $\wedge$ 
  PState t =
    concat
      (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          (of-nat (degree (p.cp as p.f-powers)))))
        roots)
      (hd (last ls))  $\wedge$ 
  PTranscript t = rev ys  $\wedge$ 
  replay-state  $\leq$  t
by (rule honest-root-alpha-degree-fri-final-dynamic-prefix-outcome[
  OF htv conjunct1[OF root-alpha-tr] len-challenges-roots roots-rounds
  replay-prefix alpha-lookups fri-lookups outcome])
have st-t-final: PState t = PState s-final
  using prefix-res st-s5 st-s6 final-send-res by simp

```

```

show ?thesis
  by (rule that[
    OF len-roots len-challenges - roots-aligned successors - - - -
    st-t-final - tr-prover -])
    (use prefix-res in simp-all)
qed

lemma honest-root-alpha-degree-fri-final-from-prover-no-failure:
assumes htv: honest-trace-valid
and create-f:
  Some (f-merkle, s1) ∈
    set-dist (execute (p.create p.f-eval) init-state)
and send-f:
  Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and alphas:
  Some (as, s3) ∈
    set-dist (execute
      (mmap (replicate (length spec)
        (do {
          a ← p.receive-alpha-challenge;
          p.send a;
          return a
        }))) s2)
and cp'-def: cp' = p.cp as p.f-powers
and send-degree:
  Some ((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp'))) s3)
and create-cp:
  Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
and fri:
  Some ((ps, ds, ls, ms), s6) ∈
    set-dist (execute
      (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
and final-send:
  Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
and random-idx:
  Some (idx, s7) ∈ set-dist (execute p.receive-query-index-challenge s-final)
and idx-def: idx' = p.index (to-nat idx)
and query-decommit:
  Some (qouts, s8) ∈
    set-dist (execute (mmap (p.decommit-on-query idx' f-merkle)) s7)
and fri-decommit:
  Some (fri-idx, s9) ∈
    set-dist (execute
      (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
and prover-state-eq: prover-state = s9
and replay: replay-state = verifier-replay-state prover-state
shows
  None ∉ dom (dist (execute
    (do {

```

```

    fr ← p.read;
    as' ← mmap (replicate (length spec)
      (do {
        a0 ← p.receive-alpha-challenge;
        let a0' = a0;
        a1 ← p.read;
        let a1' = a1;
        assert (a0' = a1');
        return a1'
      }));
    dg ← p.read;
    assert (to-nat dg ≤ p.maxDegree);
    fl ← ntimes v.receive-fri-commits nrounds;
    final ← p.read;
    return (fr, as', dg, fl, final)
  }) replay-state))
proof -
  have root-alpha-tr:
    length as = length spec ∧
    PTranscript s4 = of-nat (degree cp') # rev as @ [value f-merkle]
  by (rule prover-root-alpha-degree-transcript[
    OF create-f send-f alphas cp'-def send-degree])
  have later-hash: s3 ≤ prover-state
  by (rule prover-later-phases-hash-extends[
    OF send-degree create-cp fri final-send random-idx
    query-decommit fri-decommit prover-state-eq])
  have alpha-lookups:
    ∀ i < length as.
    fmlookup (HashMap replay-state)
      (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat (concat (PState re-
play-state) (value f-merkle)) (take i as))) =
    Some (as ! i)
  by (rule prover-alpha-lookups-in-replay-state[
    OF create-f send-f alphas later-hash replay])
  from fri-commit-replay-data[OF fri] obtain roots challenges where
    len-roots: length roots = nrounds
  and len-challenges: length challenges = nrounds
  and tr-s6: PTranscript s6 = rev roots @ PTranscript s5
  and st-s6: PState s6 = foldl concat (PState s5) roots
  and fri-hash: s5 ≤ s6
  and fri-lookups-s6:
    ∀ i < length roots.
    fmlookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i)
roots))) =
    Some (challenges ! i)
  by blast
  have final-tail-tr: transcript-extends prover-state s-final
proof -
  have final-s7: transcript-extends s7 s-final

```



```

    using receive-query-index-challenge-transcript-extends[OF random-idx] .
have s7-s8: transcript-extends s8 s7
  by (rule mmap-transcript-extends[OF query-decommit])
    (rule decommit-on-query-step-transcript-extends)
have s8-s9: transcript-extends s9 s8
  by (rule mfold-transcript-extends[OF fri-decommit])
    (rule decommit-on-fri-layers-step-transcript-extends)
have s9-prover: transcript-extends prover-state s9
  using prover-state-eq by simp
show ?thesis
  using final-s7 s7-s8 s8-s9 s9-prover by (meson transcript-extends-trans)
qed
from final-tail-tr obtain ys where
  tr-prover: PTranscript prover-state = ys @ PTranscript s-final
  unfolding transcript-extends-def by auto
have tr-s5: PTranscript s5 = PTranscript s4
  using create-preserves-transcript[OF create-cp] .
have final-send-res:
  s-final = s6(
    PState := concat (PState s6) (hd (last ls)),
    PTranscript := hd (last ls) # PTranscript s6)
  using p.send-outcome[OF final-send] .
have replay-prefix:
  PTranscript replay-state =
    value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @
    roots @ [hd (last ls)] @ rev ys
  using replay tr-prover final-send-res tr-s6 tr-s5 root-alpha-tr cp'-def
  unfolding verifier-replay-state-def by simp
have st-s1: PState s1 = 0
  using create-preserves-state[OF create-f]
  unfolding init-state-def by simp
have send-f-res:
  s2 = s1(
    PState := concat (PState s1) (value f-merkle),
    PTranscript := value f-merkle # PTranscript s1)
  using p.send-outcome[OF send-f] .
have alpha-data:
  PState s3 = foldl concat (PState s2) as
  using alpha-mmap-outcome[OF alphas] by simp
have send-degree-res:
  s4 = s3(
    PState := concat (PState s3) (of-nat (degree cp')),
    PTranscript := of-nat (degree cp') # PTranscript s3)
  using p.send-outcome[OF send-degree] .
have st-s5:
  PState s5 =
    concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
    (of-nat (degree (p.cp as p.f-powers)))
  using st-s1 send-f-res alpha-data send-degree-res

```

```

    create-preserves-state[OF create-cp] replay cp'-def
  unfolding verifier-replay-state-def by simp
have after-hash: s6 ≤ prover-state
  by (rule prover-after-fri-hash-extends[
    OF final-send random-idx query-decommit fri-decommit prover-state-eq])
have fri-lookups:
  ∀ i < length roots.
    fmlookup (HashMap replay-state)
      (FiatShamirChallenge (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          (of-nat (degree (p.cp as p.f-powers))))
        (take (Suc i) roots))) =
      Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have lookup-s6:
    fmlookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i)
roots))) =
    Some (challenges ! i)
  using fri-lookups-s6 i-bound by simp
  have lookup-prover:
    fmlookup (HashMap prover-state) (FiatShamirChallenge (foldl concat (PState s5) (take
(Suc i) roots))) =
    Some (challenges ! i)
  using p.hash-extension-lookup[OF lookup-s6 after-hash] .
  show fmlookup (HashMap replay-state)
    (FiatShamirChallenge (foldl concat
      (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
        (of-nat (degree (p.cp as p.f-powers))))
      (take (Suc i) roots))) =
    Some (challenges ! i)
  using lookup-prover st-s5 replay
  unfolding verifier-replay-state-def by simp
qed
have len-challenges-roots: length challenges = length roots
  using len-challenges len-roots by simp
have nf-roots:
  None ∉ dom (dist (execute
    (do {
      fr ← p.read;
      as' ← mmap (replicate (length spec)
        (do {
          a0 ← p.receive-alpha-challenge;
          let a0' = a0;
          a1 ← p.read;
          let a1' = a1;
          assert (a0' = a1');
          return a1'
        }
      )
    )
  )

```

```

    ));
    dg ← p.read;
    assert (to-nat dg ≤ p.maxDegree);
    fl ← ntimes v.receive-fri-commits (length roots);
    final ← p.read;
    return (fr, as', dg, fl, final)
  }) replay-state))
by (rule honest-root-alpha-degree-fri-final-prefix-no-failure[
  OF htv conjunct1[OF root-alpha-tr] len-challenges-roots
  replay-prefix alpha-lookups fri-lookups])
show ?thesis
using nf-roots len-roots by simp
qed

```

lemma *honest-root-alpha-degree-fri-final-dynamic-from-prover-no-failure:*
assumes *htv: honest-trace-valid*
and *create-f:*
 Some (*f-merkle*, *s1*) ∈
 set-dist (execute (p.create p.f-eval) init-state)
and *send-f:*
 Some ((), *s2*) ∈ set-dist (execute (p.send (value f-merkle)) *s1*)
and *alphas:*
 Some (*as*, *s3*) ∈
 set-dist (execute
 (mmap (replicate (length spec)
 (do {
 a ← p.receive-alpha-challenge;
 p.send a;
 return a
 }))) *s2*)
and *cp'-def:* *cp' = p.cp as p.f-powers*
and *send-degree:*
 Some ((), *s4*) ∈ set-dist (execute (p.send (of-nat (degree cp'))) *s3*)
and *create-cp:*
 Some (*cp-merkle*, *s5*) ∈ set-dist (execute (p.create (p.cp-eval as)) *s4*)
and *nrounds-def:* *nrounds = ceil-log (degree cp' + 1)*
and *fri:*
 Some ((*ps*, *ds*, *ls*, *ms*), *s6*) ∈
 set-dist (execute
 (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) *s5*)
and *final-send:*
 Some ((), *s-final*) ∈ set-dist (execute (p.send (hd (last ls))) *s6*)
and *random-idx:*
 Some (*idx*, *s7*) ∈ set-dist (execute p.receive-query-index-challenge s-final)
and *idx-def:* *idx' = p.index (to-nat idx)*
and *query-decommit:*
 Some (*qouts*, *s8*) ∈
 set-dist (execute (mmap (p.decommit-on-query idx' f-merkle)) *s7*)
and *fri-decommit:*

```

    Some (fri-idx, s9) ∈
      set-dist (execute
        (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
  and prover-state-eq: prover-state = s9
  and replay: replay-state = verifier-replay-state prover-state
shows
  None ∉ dom (dist (execute
    (do {
      fr ← p.read;
      as' ← mmap (replicate (length spec)
        (do {
          a0 ← p.receive-alpha-challenge;
          let a0' = a0;
          a1 ← p.read;
          let a1' = a1;
          assert (a0' = a1');
          return a1'
        })));
      dg ← p.read;
      assert (to-nat dg ≤ p.maxDegree);
      fl ← ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1));
      final ← p.read;
      return (fr, as', dg, fl, final)
    }) replay-state))
proof -
  have root-alpha-tr:
    length as = length spec ∧
    PTranscript s4 = of-nat (degree cp') # rev as @ [value f-merkle]
  by (rule prover-root-alpha-degree-transcript[
    OF create-f send-f alphas cp'-def send-degree])
  have later-hash: s3 ≤ prover-state
  by (rule prover-later-phases-hash-extends[
    OF send-degree create-cp fri final-send random-idx
    query-decommit fri-decommit prover-state-eq])
  have alpha-lookups:
    ∀ i < length as.
      fmlookup (HashMap replay-state)
        (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat (concat (PState re-
          play-state) (value f-merkle)) (take i as))) =
        Some (as ! i)
  by (rule prover-alpha-lookups-in-replay-state[
    OF create-f send-f alphas later-hash replay])
  from fri-commit-replay-data[OF fri] obtain roots challenges where
    len-roots: length roots = nrounds
  and len-challenges: length challenges = nrounds
  and tr-s6: PTranscript s6 = rev roots @ PTranscript s5
  and st-s6: PState s6 = foldl concat (PState s5) roots
  and fri-hash: s5 ≤ s6
  and fri-lookups-s6:

```

```

     $\forall i < \text{length roots}.$ 
    fmllookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i)
roots))) =
      Some (challenges ! i)
  by blast
have final-tail-tr: transcript-extends prover-state s-final
proof -
  have final-s7: transcript-extends s7 s-final
    using receive-query-index-challenge-transcript-extends[OF random-idx] .
  have s7-s8: transcript-extends s8 s7
    by (rule mmap-transcript-extends[OF query-decommit])
      (rule decommit-on-query-step-transcript-extends)
  have s8-s9: transcript-extends s9 s8
    by (rule mfold-transcript-extends[OF fri-decommit])
      (rule decommit-on-fri-layers-step-transcript-extends)
  have s9-prover: transcript-extends prover-state s9
    using prover-state-eq by simp
  show ?thesis
    using final-s7 s7-s8 s8-s9 s9-prover by (meson transcript-extends-trans)
qed
from final-tail-tr obtain ys where
  tr-prover: PTranscript prover-state = ys @ PTranscript s-final
  unfolding transcript-extends-def by auto
have tr-s5: PTranscript s5 = PTranscript s4
  using create-preserves-transcript[OF create-cp] .
have final-send-res:
  s-final = s6(
    PState := concat (PState s6) (hd (last ls)),
    PTranscript := hd (last ls) # PTranscript s6)
  using p.send-outcome[OF final-send] .
have replay-prefix:
  PTranscript replay-state =
    value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @
    roots @ [hd (last ls)] @ rev ys
  using replay tr-prover final-send-res tr-s6 tr-s5 root-alpha-tr cp'-def
  unfolding verifier-replay-state-def by simp
have st-s1: PState s1 = 0
  using create-preserves-state[OF create-f]
  unfolding init-state-def by simp
have send-f-res:
  s2 = s1(
    PState := concat (PState s1) (value f-merkle),
    PTranscript := value f-merkle # PTranscript s1)
  using p.send-outcome[OF send-f] .
have alpha-data:
  PState s3 = foldl concat (PState s2) as
  using alpha-mmap-outcome[OF alphas] by simp
have send-degree-res:
  s4 = s3(

```

```

    PState := concat (PState s3) (of-nat (degree cp')),
    PTranscript := of-nat (degree cp') # PTranscript s3)
using p.send-outcome[OF send-degree] .
have st-s5:
  PState s5 =
    concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
      (of-nat (degree (p.cp as p.f-powers)))
using st-s1 send-f-res alpha-data send-degree-res
  create-preserves-state[OF create-cp] replay cp'-def
unfolding verifier-replay-state-def by simp
have after-hash: s6 ≤ prover-state
by (rule prover-after-fri-hash-extends[
  OF final-send random-idx query-decommit fri-decommit prover-state-eq])
have fri-lookups:
  ∀ i < length roots.
    fmlookup (HashMap replay-state)
      (FiatShamirChallenge (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          (of-nat (degree (p.cp as p.f-powers))))
        (take (Suc i) roots))) =
      Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have lookup-s6:
    fmlookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i)
roots))) =
      Some (challenges ! i)
    using fri-lookups-s6 i-bound by simp
  have lookup-prover:
    fmlookup (HashMap prover-state) (FiatShamirChallenge (foldl concat (PState s5) (take
(Suc i) roots))) =
      Some (challenges ! i)
    using p.hash-extension-lookup[OF lookup-s6 after-hash] .
  show fmlookup (HashMap replay-state)
    (FiatShamirChallenge (foldl concat
      (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
        (of-nat (degree (p.cp as p.f-powers))))
      (take (Suc i) roots))) =
      Some (challenges ! i)
    using lookup-prover st-s5 replay
    unfolding verifier-replay-state-def by simp
qed
have roots-rounds:
  length roots = ceil-log (degree (p.cp as p.f-powers) + 1)
  using len-roots nrounds-def cp'-def by simp
have len-challenges-roots: length challenges = length roots
  using len-challenges len-roots by simp
show ?thesis

```

```

    by (rule honest-root-alpha-degree-fri-final-dynamic-prefix-no-failure[
      OF htv conjunct1[OF root-alpha-tr] len-challenges-roots roots-rounds
      replay-prefix alpha-lookups fri-lookups])
qed
end
end

```

```

theory Completeness-Query
  imports Completeness-Replay
begin

```

Honest query decommitment values and authenticated query replay.

```

context verification
begin

```

```

lemma transcript-read-cons-wp-error:
  wp-error p.read (s(PTranscript := x # xs)) = 0
  unfolding p.read-def protocol-read-def assert-def by (simp add: wpsimps)

```

```

lemma merkle-authentication-path-outcome:
  assumes p.created-tree xs r s0
  and length xs = 2 ^ n
  and i < length xs
  and v = xs ! i
  and s0 ≤ s
  and Some (h, t) ∈ set-dist (execute
    (p.check-authentication-path (length xs) i v
      (get-authentication-path (length xs) i r)) s)
  shows h = value r
  using p.check-created-tree-outcome[OF assms] by simp

```

```

lemma query-f-eval-nth:
  assumes i < length p.eval-domain
  shows p.f-eval ! i = poly p.f (p.eval-domain ! i)
  using assms unfolding p.f-eval-def by simp

```

```

lemma f-powers-nth:
  assumes k < powers
  shows p.f-powers ! k = p.f ∘p XP (p.g ^ k)
  using assms unfolding p.f-powers-def by simp

```

```

lemma f-powers-length:
  length p.f-powers = powers
  unfolding p.f-powers-def by simp

```

```

lemma f-powers-eval:

```

assumes $k < \text{powers}$
shows $\text{poly } (p.f\text{-powers } ! k) x = \text{poly } p.f (x * p.g \wedge k)$
using $f\text{-powers-nth}[OF \text{ assms}]$
unfolding $XP\text{-def}$
by ($\text{simp add: poly-pcompose poly-monom algebra-simps}$)

lemma query-cp-eval-nth :
assumes $i < \text{length } p.\text{eval-domain}$
shows $p.\text{cp-eval as } ! i = \text{poly } (p.\text{cp as } p.f\text{-powers}) (p.\text{eval-domain } ! i)$
using assms **unfolding** $p.\text{cp-eval-def}$ **by** simp

lemma $\text{powers-scaled-setD}$:
assumes $i \in \text{set } (p.\text{powers-scaled } \text{idx})$
obtains k **where** $k < \text{powers}$ **and** $i = \text{idx} + k * \text{scale}$
using assms **unfolding** $p.\text{powers-scaled-def}$ **by** auto

lemma clength-pos :
 $0 < \text{clength}$
proof –
obtain n **where** $\text{len: clength} * \text{scale} = 2 \wedge n$
using $p.\text{eval-domain-length-power}$ **by** blast
show $?thesis$
using $\text{len } p.\text{scale-pos}$ **by** (cases clength) simp-all
qed

lemma $\text{max-powers-less-clength}$:
 $\text{Max } (\text{set } [0..<\text{powers}]) < \text{clength}$
proof –
have $\text{Max } (\text{set } [0..<\text{powers}]) = \text{powers} - 1$
using $p.\text{powers-pos}$ **by** (intro Max-eqI) auto
also have $\dots < \text{clength}$
using $p.\text{powers-pos } p.\text{powers-le-clength}$ **by** simp
finally show $?thesis$.
qed

lemma index-less-domain :
 $p.\text{index } x < \text{clength} * \text{scale}$
proof –
have $\text{max-le: Max } (\text{set } [0..<\text{powers}]) < \text{clength}$
by ($\text{rule max-powers-less-clength}$)
have $\text{mod-pos: } 0 < \text{clength} * \text{scale} - \text{Max } (\text{set } [0..<\text{powers}]) * \text{scale}$
using $\text{max-le } p.\text{scale-pos}$
by ($\text{simp add: diff-mult-distrib2}$)
have idx-lt:
 $p.\text{index } x < \text{clength} * \text{scale} - \text{Max } (\text{set } [0..<\text{powers}]) * \text{scale}$
unfolding $p.\text{index-def}$
using mod-pos **by** simp
have $\text{clength} * \text{scale} - \text{Max } (\text{set } [0..<\text{powers}]) * \text{scale} \leq \text{clength} * \text{scale}$
by simp


```

then show ?thesis
  using idx-lt by linarith
qed

lemma index-less-domain-tail:
  assumes  $k < \text{powers}$ 
  shows  $p.\text{index } x + k * \text{scale} < \text{clength} * \text{scale}$ 
proof -
  have max-le:  $\text{Max} (\text{set } [0..<\text{powers}]) < \text{clength}$ 
    by (rule max-powers-less-clength)
  have mod-pos:  $0 < \text{clength} * \text{scale} - \text{Max} (\text{set } [0..<\text{powers}]) * \text{scale}$ 
    using max-le p.scale-pos
    by (simp add: diff-mult-distrib2)
  have idx-lt:
     $p.\text{index } x < \text{clength} * \text{scale} - \text{Max} (\text{set } [0..<\text{powers}]) * \text{scale}$ 
    unfolding p.index-def
    using mod-pos by simp
  have k-le-max:  $k * \text{scale} \leq \text{Max} (\text{set } [0..<\text{powers}]) * \text{scale}$ 
proof (cases powers = 0)
  case True
    then show ?thesis using assms by simp
  next
    case False
    then have  $k \leq \text{Max} (\text{set } [0..<\text{powers}])$ 
      using assms by simp
    then show ?thesis by simp
qed
have p.index  $x + k * \text{scale} <$ 
   $(\text{clength} * \text{scale} - \text{Max} (\text{set } [0..<\text{powers}]) * \text{scale}) +$ 
   $\text{Max} (\text{set } [0..<\text{powers}]) * \text{scale}$ 
  using idx-lt k-le-max by linarith
also have  $\dots = \text{clength} * \text{scale}$ 
  using max-le p.scale-pos
  by (simp add: diff-mult-distrib2)
finally show ?thesis .
qed

lemma powers-scaled-index-bound:
  assumes  $i \in \text{set } (p.\text{powers-scaled } (p.\text{index } x))$ 
  shows  $i < \text{clength} * \text{scale}$ 
  using assms
  by (auto elim!: powers-scaled-setD intro!: index-less-domain-tail)

lemma f-eval-length:
   $\text{length } p.f\text{-eval} = \text{clength} * \text{scale}$ 
  unfolding p.f-eval-def p.eval-domain-def p.H-def by simp

lemma trace-polynomial-degree-lt-clength:
   $\text{degree } p.f < \text{clength}$ 

```

```

proof –
  have vals-len:  $\text{length } \text{vals} = \text{clength}$ 
    using p.trace-length by simp
  have G-len:  $\text{length } p.G = \text{clength}$ 
    unfolding p.G-def by simp
  have deg-le:
     $\text{degree } p.f \leq \text{length } (\text{zip } p.G \text{ vals}) - 1$ 
    unfolding p.f-def by (rule degree-lagrange-interpolation-poly)
  have zip-len:  $\text{length } (\text{zip } p.G \text{ vals}) = \text{clength}$ 
    using vals-len G-len by simp
  show ?thesis
    using deg-le zip-len p.clength-pos by simp
qed

lemma honest-initial-trace-fri-accumulator-value:
  assumes fv-eq:  $fv = \text{map } (\lambda i. p.f\text{-eval } ! i) (p.\text{powers-scaled } (p.\text{index } x))$ 
    and l0-eq:  $l0 = p.f\text{-eval}$ 
  shows  $\text{hd } fv = l0 \ ! \ p.\text{index } x$ 
proof –
  have range:  $[0..<\text{powers}] = 0 \ \# \ [1..<\text{powers}]$ 
    using p.powers-pos by (simp add: upt-conv-Cons)
  have ps:
     $p.\text{powers-scaled } (p.\text{index } x) =$ 
     $p.\text{index } x \ \# \ \text{map } (\lambda xa. p.\text{index } x + xa * \text{scale}) [1..<\text{powers}]$ 
    unfolding p.powers-scaled-def using range by simp
  show ?thesis
    using fv-eq l0-eq ps by simp
qed

lemma eval-domain-length:
   $\text{length } p.\text{eval-domain} = \text{clength} * \text{scale}$ 
  unfolding p.eval-domain-def p.H-def by simp

lemma eval-domain-nth:
  assumes  $i < \text{clength} * \text{scale}$ 
  shows  $p.\text{eval-domain } ! i = p.h \ ^i * \text{shift}$ 
  using assms
  unfolding p.eval-domain-def p.H-def
  by (simp add: mult.commute)

lemma query-f-power-eval-nth:
  assumes k-bound:  $k < \text{powers}$ 
    and idx-bound:  $\text{idx} + k * \text{scale} < \text{clength} * \text{scale}$ 
  shows
     $p.f\text{-eval } ! (\text{idx} + k * \text{scale}) =$ 
     $\text{poly } (p.f\text{-powers } ! k) (p.h \ ^{\text{idx} * \text{shift}})$ 
proof –
  have eval:
     $p.f\text{-eval } ! (\text{idx} + k * \text{scale}) =$ 

```

```

    poly p.f (p.h ^ (idx + k * scale) * shift)
using query-f-eval-nth[of idx + k * scale] eval-domain-nth[OF idx-bound]
    idx-bound eval-domain-length
by simp
have arg:
    (p.h ^ idx * shift) * p.g ^ k = p.h ^ (idx + k * scale) * shift
proof -
    have gpow: p.g ^ k = p.h ^ (scale * k)
      using p.domain-alignment by (simp add: power-mult)
    have (p.h ^ idx * shift) * p.g ^ k =
      p.h ^ idx * p.h ^ (scale * k) * shift
      using gpow by (simp add: algebra-simps)
    also have ... = p.h ^ (idx + k * scale) * shift
      by (simp add: power-add algebra-simps mult.commute)
    finally show ?thesis .
qed
show ?thesis
  using eval f-powers-eval[OF k-bound, of p.h ^ idx * shift] arg
  by simp
qed

lemma honest-query-values-f-powers:
  map (λi. p.f-eval ! i) (p.powers-scaled (p.index x)) =
    map (λq. poly q (p.h ^ p.index x * shift)) p.f-powers
proof (rule nth-equalityI)
  show length (map (λi. p.f-eval ! i) (p.powers-scaled (p.index x))) =
    length (map (λq. poly q (p.h ^ p.index x * shift)) p.f-powers)
    unfolding p.powers-scaled-def using f-powers-length by simp
next
  fix k
  assume k-bound:
    k < length (map (λi. p.f-eval ! i) (p.powers-scaled (p.index x)))
  then have k-lt: k < powers
    unfolding p.powers-scaled-def by simp
  have idx-bound: p.index x + k * scale < clength * scale
    by (rule index-less-domain-tail[OF k-lt])
  have left:
    map (λi. p.f-eval ! i) (p.powers-scaled (p.index x)) ! k =
      p.f-eval ! (p.index x + k * scale)
    using k-lt unfolding p.powers-scaled-def by simp
  have right:
    map (λq. poly q (p.h ^ p.index x * shift)) p.f-powers ! k =
      poly (p.f-powers ! k) (p.h ^ p.index x * shift)
    using k-lt f-powers-length by simp
  show
    map (λi. p.f-eval ! i) (p.powers-scaled (p.index x)) ! k =
      map (λq. poly q (p.h ^ p.index x * shift)) p.f-powers ! k
    using query-f-power-eval-nth[OF k-lt idx-bound] left right by simp
qed

```

```

lemma honest-cp-eval-agrees-lists:
  fixes specs :: (('f poly list  $\Rightarrow$  'f poly)  $\times$  nat list  $\times$  nat) list
    and spec2s :: (('f list  $\Rightarrow$  'f)  $\times$  nat list) list
    and pacc :: 'f poly
    and vacc :: 'f
  assumes hta: honest-trace-algebra
    and idx-bound: idx < clength * scale
    and len: length spec2s = length specs
    and zip-subset: set (zip specs spec2s)  $\subseteq$  set (zip spec spec2)
    and specs-subset: set specs  $\subseteq$  set spec
    and acc-eq: vacc = poly pacc (p.h ^ idx * shift)
  shows
    fold
      ( $\lambda(a, (c2, roots2)).$ 
        (+) (a * (c2 (map ( $\lambda q. poly\ q\ (p.h \wedge idx * shift))\ p.f-powers)\ div$ 
          poly (prod (p.g-map roots2)) (p.h ^ idx * shift)))))
        (zip as spec2s) vacc =
      poly
        (fold
          ( $\lambda(a, (c, roots, -)).$ 
            (+) (CP a * ((c p.f-powers) div prod (p.g-map roots))))
          (zip as specs) pacc)
          (p.h ^ idx * shift)
        using len zip-subset specs-subset acc-eq
  proof (induction specs arbitrary: as spec2s pacc vacc)
    case Nil
    then have spec2s = []
    by simp
    then show ?case
    using Nil.premis(4) by simp
  next
    case (Cons s specs')
    obtain c roots d where s-eq: s = (c, roots, d)
    by (cases s) auto
    show ?case
    proof (cases spec2s)
      case Nil
      then show ?thesis
      using Cons.premis(1) by simp
    next
      case spec2-Cons: (Cons s2 spec2s')
      then obtain c2 roots2 where s2-eq: s2 = (c2, roots2)
      by (cases s2) auto
      have len-tail: length spec2s' = length specs'
      using Cons.premis(1) spec2-Cons by simp
      show ?thesis
      proof (cases as)
        case Nil

```

```

then show ?thesis
  using Cons.premis(4) by simp
next
case as-Cons: (Cons a as')
have pair-in:
  ((c, roots, d), (c2, roots2)) ∈ set (zip spec spec2)
proof -
  have (s, s2) ∈ set (zip (s # specs') (s2 # spec2s'))
    by simp
  moreover have set (zip (s # specs') (s2 # spec2s')) ⊆ set (zip spec spec2)
    using Cons.premis(2) spec2-Cons by simp
  ultimately have (s, s2) ∈ set (zip spec spec2)
    by blast
  then show ?thesis
    using s-eq s2-eq by simp
qed
have spec-entry: (c, roots, d) ∈ set spec
proof -
  have s ∈ set (s # specs')
    by simp
  then have s ∈ set spec
    using Cons.premis(3) by blast
  then show ?thesis
    using s-eq by simp
qed
have roots-eq: roots2 = roots
and raw-eq:
  c2 (map (λq. poly q (p.h ^ idx * shift)) p.f-powers) =
    poly (c p.f-powers) (p.h ^ idx * shift)
using v.specs-agree[OF pair-in, of p.f-powers p.h ^ idx * shift]
unfolding p.f-powers-def by simp-all
have quotient-eq:
  poly ((c p.f-powers) div prod (p.g-map roots)) (p.h ^ idx * shift) =
    poly (c p.f-powers) (p.h ^ idx * shift) div
      poly (prod (p.g-map roots)) (p.h ^ idx * shift)
  by (rule honest-trace-algebra-quotient-eval[OF hta spec-entry idx-bound])
have cp-eval: poly (CP a) (p.h ^ idx * shift) = a
  by (simp add: CP-def poly-monom)
have term-eq:
  a * (c2 (map (λq. poly q (p.h ^ idx * shift)) p.f-powers) div
    poly (prod (p.g-map roots2)) (p.h ^ idx * shift)) =
    poly (CP a * ((c p.f-powers) div prod (p.g-map roots)))
      (p.h ^ idx * shift)
  using roots-eq raw-eq quotient-eq cp-eval by simp
have acc-step:
  a * (c2 (map (λq. poly q (p.h ^ idx * shift)) p.f-powers) div
    poly (prod (p.g-map roots2)) (p.h ^ idx * shift)) + vacc =
    poly (CP a * ((c p.f-powers) div prod (p.g-map roots)) + pacc)
      (p.h ^ idx * shift)

```

```

using term-eq Cons.premis(4) by simp
have tail:
  fold
    (λ(a, (c2, roots2)).
      (+) (a * (c2 (map (λq. poly q (p.h ^ idx * shift)) p.f-powers) div
        poly (prod (p.g-map roots2)) (p.h ^ idx * shift))))
    (zip as' spec2s')
    (a * (c2 (map (λq. poly q (p.h ^ idx * shift)) p.f-powers) div
      poly (prod (p.g-map roots2)) (p.h ^ idx * shift)) + vacc) =
  poly
    (fold
      (λ(a, (c, roots, -)).
        (+) (CP a * ((c p.f-powers) div prod (p.g-map roots))))
      (zip as' specs')
      (CP a * ((c p.f-powers) div prod (p.g-map roots)) + pacc))
    (p.h ^ idx * shift)
proof (rule Cons.IH)
show length spec2s' = length specs'
using len-tail .
show set (zip specs' spec2s') ⊆ set (zip spec spec2)
using Cons.premis(2) spec2-Cons by auto
show set specs' ⊆ set spec
using Cons.premis(3) by auto
show a * (c2 (map (λq. poly q (p.h ^ idx * shift)) p.f-powers) /
  poly (prod (p.g-map roots2)) (p.h ^ idx * shift)) + vacc =
  poly (CP a * ((c p.f-powers) div prod (p.g-map roots)) + pacc)
  (p.h ^ idx * shift)
using acc-step by simp
qed
show ?thesis
using as-Cons spec2-Cons s-eq s2-eq tail by simp
qed
qed
qed

```

lemma honest-cp-eval-agrees:

```

assumes hta: honest-trace-algebra
and idx-bound: idx < clength * scale
shows
  v.cp-eval as (map (λq. poly q (p.h ^ idx * shift)) p.f-powers)
    (p.h ^ idx * shift) =
  poly (p.cp as p.f-powers) (p.h ^ idx * shift)
unfolding v.cp-eval-def p.cp-def
by (rule honest-cp-eval-agrees-lists[OF hta idx-bound])
  (use v.spec2-length in auto)

```

lemma honest-query-cp-value:

```

assumes honest-trace-valid
shows

```

```

    v.cp-eval as (map (λi. p.f-eval ! i) (p.powers-scaled (p.index x)))
      (p.h ^ p.index x * shift) =
      poly (p.cp as p.f-powers) (p.h ^ p.index x * shift)
  proof -
    have hta: honest-trace-algebra
      using assms unfolding honest-trace-valid-def by simp
    have idx-bound: p.index x < clength * scale
      by (rule index-less-domain)
    show ?thesis
      using honest-cp-eval-agrees[OF hta idx-bound, of as]
        honest-query-values-f-powers[of x]
      by simp
  qed

lemma cp-eval-index-value:
  p.cp-eval as ! p.index x =
    poly (p.cp as p.f-powers) (p.h ^ p.index x * shift)
proof -
  have idx-bound: p.index x < length p.eval-domain
    using index-less-domain eval-domain-length by simp
  have p.cp-eval as ! p.index x =
    poly (p.cp as p.f-powers) (p.eval-domain ! p.index x)
    by (rule query-cp-eval-nth[OF idx-bound])
  also have ... = poly (p.cp as p.f-powers) (p.h ^ p.index x * shift)
    using eval-domain-nth[OF index-less-domain[of x]] by simp
  finally show ?thesis .
qed

lemma honest-initial-fri-accumulator-value:
  assumes htv: honest-trace-valid
    and as-eq: as' = as
    and fv-eq: fv = map (λi. p.f-eval ! i) (p.powers-scaled (p.index x))
    and l0-eq: l0 = p.cp-eval as
  shows
    v.cp-eval as' fv (p.h ^ p.index x * shift) = l0 ! p.index x
  using honest-query-cp-value[OF htv, of as x] cp-eval-index-value[of as x]
    as-eq fv-eq l0-eq
  by simp

lemma h-nonzero:
  p.h ≠ 0
  unfolding p.h-def using p.omega-nonzero-derived by simp

lemma verifier-domain-point-nonzero:
  ((p.h ^ i) * shift) ^ pw ≠ 0
  using h-nonzero p.shift-nonzero by simp

lemma eval-domain-size-pos:
  0 < clength * scale

```

using *maxDegree-less-eval-domain* **by** *linarith*

lemma *h-power-mod-eval-domain*:

$p.h \wedge (k \bmod (clength * scale)) = p.h \wedge k$

proof –

let $?N = clength * scale$

have *order*: $p.h \wedge ?N = 1$

using *p.h-order* .

have *k-eq*: $k = (k \text{ div } ?N) * ?N + k \bmod ?N$

using *div-mult-mod-eq[of k ?N]* **by** (*simp add: mult.commute*)

have *period*: $p.h \wedge ((k \text{ div } ?N) * ?N) = 1$

proof –

have $p.h \wedge ((k \text{ div } ?N) * ?N) = p.h \wedge (?N * (k \text{ div } ?N))$

by (*simp add: mult.commute*)

also have $\dots = (p.h \wedge ?N) \wedge (k \text{ div } ?N)$

by (*simp only: power-mult*)

also have $\dots = 1$

using *order* **by** *simp*

finally show *?thesis* .

qed

have $p.h \wedge k = p.h \wedge ((k \text{ div } ?N) * ?N + k \bmod ?N)$

using *k-eq* **by** *simp*

also have $\dots = p.h \wedge ((k \text{ div } ?N) * ?N) * p.h \wedge (k \bmod ?N)$

by (*simp add: power-add*)

also have $\dots = p.h \wedge (k \bmod ?N)$

using *period* **by** *simp*

finally show *?thesis*

by *simp*

qed

lemma *h-half-shift*:

$p.h \wedge (k + (clength * scale) \text{ div } 2) = -(p.h \wedge k)$

using *p.h-half-order*

by (*simp add: power-add algebra-simps*)

lemma *mod-mult-right-factor-nat*:

fixes $a \ b \ c :: \text{nat}$

assumes *b-pos*: $0 < b$

and *c-pos*: $0 < c$

shows $((a \bmod b) * c) \bmod (b * c) = (a * c) \bmod (b * c)$

proof –

have *decomp*: $a = a \text{ div } b * b + a \bmod b$

using *div-mult-mod-eq[of a b]* **by** *simp*

have *r-less*: $(a \bmod b) * c < b * c$

using *b-pos c-pos mod-less-divisor[OF b-pos]*

by (*simp add: mult-less-mono2*)

have *r-le*: $(a \bmod b) * c \leq a * c$

proof –

have $a \bmod b \leq a$


```

    using decomp by linarith
  then show ?thesis
    by simp
qed
have dvd-diff:  $b * c \text{ dvd } a * c - (a \text{ mod } b) * c$ 
proof -
  have ac-decomp:  $a * c = (a \text{ div } b) * (b * c) + (a \text{ mod } b) * c$ 
  proof -
    have  $a * c = (a \text{ div } b * b + a \text{ mod } b) * c$ 
    using decomp by simp
    also have  $\dots = (a \text{ div } b * b) * c + (a \text{ mod } b) * c$ 
    by (simp only: distrib-right)
    also have  $\dots = (a \text{ div } b) * (b * c) + (a \text{ mod } b) * c$ 
    by (simp add: algebra-simps)
    finally show ?thesis .
  qed
  have  $a * c - (a \text{ mod } b) * c = (a \text{ div } b) * (b * c)$ 
  using ac-decomp by simp
  then show ?thesis by simp
qed
have  $(a * c) \text{ mod } (b * c) = (a \text{ mod } b) * c$ 
  by (rule mod-nat-eqI[OF r-less r-le dvd-diff])
then show ?thesis by simp
qed

lemma fri-sibling-domain-round:
  assumes len-pos:  $0 < \text{len}$ 
    and even-len:  $2 \text{ dvd } \text{len}$ 
    and round:  $\text{len} * \text{pw} = \text{clength} * \text{scale}$ 
  shows
     $((p.h \wedge ((i + \text{len} \text{ div } 2) \text{ mod } \text{len})) * \text{shift}) \wedge \text{pw} =$ 
     $- (((p.h \wedge i) * \text{shift}) \wedge \text{pw})$ 
proof -
  let ?N =  $\text{clength} * \text{scale}$ 
  have pw-pos:  $0 < \text{pw}$ 
  using len-pos round eval-domain-size-pos by (cases pw) simp-all
  have half-round:  $(\text{len} \text{ div } 2) * \text{pw} = ?N \text{ div } 2$ 
  proof -
    from even-len obtain k where len-eq:  $\text{len} = 2 * k$ 
    by blast
    then have  $?N = 2 * (k * \text{pw})$ 
    using round by simp
    then show ?thesis
      using len-eq by simp
  qed
  have exp-mod:
     $((i + \text{len} \text{ div } 2) \text{ mod } \text{len}) * \text{pw} \text{ mod } ?N =$ 
     $(i * \text{pw} + ?N \text{ div } 2) \text{ mod } ?N$ 
  proof -

```

```

have (((i + len div 2) mod len) * pw) mod ?N =
  ((i + len div 2) * pw) mod ?N
  using mod-mult-right-factor-nat[OF len-pos pw-pos, of i + len div 2]
  round by simp
also have ... = (i * pw + ?N div 2) mod ?N
  using half-round by (simp add: algebra-simps)
finally show ?thesis .
qed
have h-exp:
  p.h ^ (((i + len div 2) mod len) * pw) = -(p.h ^ (i * pw))
proof -
  have p.h ^ (((i + len div 2) mod len) * pw) =
    p.h ^ (((i + len div 2) mod len) * pw) mod ?N
    using h-power-mod-eval-domain[of ((i + len div 2) mod len) * pw]
    by simp
  also have ... = p.h ^ (i * pw + ?N div 2) mod ?N
    using exp-mod by simp
  also have ... = p.h ^ (i * pw + ?N div 2)
    using h-power-mod-eval-domain[of i * pw + ?N div 2] by simp
  also have ... = -(p.h ^ (i * pw))
    by (rule h-half-shift)
  finally show ?thesis .
qed
have lhs:
  ((p.h ^ ((i + len div 2) mod len)) * shift) ^ pw =
  p.h ^ (((i + len div 2) mod len) * pw) * shift ^ pw
  by (simp add: power-mult power-mult-distrib)
have rhs:
  ((p.h ^ i) * shift) ^ pw = p.h ^ (i * pw) * shift ^ pw
  by (simp add: power-mult power-mult-distrib)
show ?thesis
  using h-exp by (simp add: power-mult power-mult-distrib)
qed

lemma fri-sibling-domain-at:
  assumes len-pos: 0 < len
  and even-len: 2 dvd len
  and round: len * pw = clength * scale
  and z-eq: z = ((p.h ^ i) * shift) ^ pw
  and d-sib:
    d ! ((i + len div 2) mod len) =
      ((p.h ^ ((i + len div 2) mod len)) * shift) ^ pw
  shows d ! ((i + len div 2) mod len) = - z
  using fri-sibling-domain-round[OF len-pos even-len round, of i] z-eq d-sib
  by simp

lemma fri-next-domain-round:
  assumes half-pos: 0 < len div 2
  and even-len: 2 dvd len

```

```

    and round: len * pw = clength * scale
  shows
    ((p.h ^ i) * shift) ^ pw * ((p.h ^ i) * shift) ^ pw =
      ((p.h ^ (i mod (len div 2))) * shift) ^ (pw + pw)
  proof -
    let ?N = clength * scale
    have len-pos: 0 < len
      using half-pos by simp
    have pw-pos: 0 < pw
      using len-pos round eval-domain-size-pos by (cases pw) simp-all
    have double-pw-pos: 0 < pw + pw
      using pw-pos by simp
    have double-round: (len div 2) * (pw + pw) = ?N
  proof -
    from even-len obtain k where len-eq: len = 2 * k
      by blast
    have len-half: len div 2 = k
      using len-eq by simp
    have k * (pw + pw) = 2 * k * pw
  proof -
    have k * (pw + pw) = k * pw + k * pw
      by (simp add: distrib-left)
    also have ... = (k + k) * pw
      by (simp add: distrib-right)
    also have ... = 2 * k * pw
      by simp
    finally show ?thesis .
  qed
  moreover have 2 * k * pw = ?N
    using len-half round by simp
  ultimately show ?thesis
    using len-half by simp
  qed
  have exp-mod:
    ((i mod (len div 2)) * (pw + pw)) mod ?N =
      (i * (pw + pw)) mod ?N
  proof -
    have ((i mod (len div 2)) * (pw + pw)) mod ?N =
      ((i mod (len div 2)) * (pw + pw)) mod ((len div 2) * (pw + pw))
      using double-round by simp
    also have ... = (i * (pw + pw)) mod ((len div 2) * (pw + pw))
      using mod-mult-right-factor-nat[OF half-pos double-pw-pos, of i]
      by simp
    also have ... = (i * (pw + pw)) mod ?N
      using double-round by simp
    finally show ?thesis .
  qed
  have h-exp:
    p.h ^ ((i mod (len div 2)) * (pw + pw)) =

```

$p.h \wedge (i * (pw + pw))$
proof –
have $p.h \wedge ((i \bmod (len \div 2)) * (pw + pw)) =$
 $p.h \wedge (((i \bmod (len \div 2)) * (pw + pw)) \bmod ?N)$
using *h-power-mod-eval-domain*[*of* $(i \bmod (len \div 2)) * (pw + pw)$]
by *simp*
also have $\dots = p.h \wedge ((i * (pw + pw)) \bmod ?N)$
using *exp-mod* **by** *simp*
also have $\dots = p.h \wedge (i * (pw + pw))$
using *h-power-mod-eval-domain*[*of* $i * (pw + pw)$] **by** *simp*
finally show *?thesis* .

qed

show *?thesis*

proof –

have *lhs*:

$$((p.h \wedge i) * shift) \wedge pw * ((p.h \wedge i) * shift) \wedge pw =$$

$$p.h \wedge (i * (pw + pw)) * shift \wedge (pw + pw)$$

proof –

$$\mathbf{have} \ ((p.h \wedge i) * shift) \wedge pw * ((p.h \wedge i) * shift) \wedge pw =$$

$$((p.h \wedge i) * shift) \wedge (pw + pw)$$

by (*simp add: power-add*)

$$\mathbf{also \ have} \ \dots = (p.h \wedge i) \wedge (pw + pw) * shift \wedge (pw + pw)$$

by (*simp add: power-mult-distrib*)

$$\mathbf{also \ have} \ \dots = p.h \wedge (i * (pw + pw)) * shift \wedge (pw + pw)$$

by (*simp add: power-mult*)

finally show *?thesis* .

qed

have *rhs*:

$$((p.h \wedge (i \bmod (len \div 2))) * shift) \wedge (pw + pw) =$$

$$p.h \wedge ((i \bmod (len \div 2)) * (pw + pw)) * shift \wedge (pw + pw)$$

by (*simp add: power-mult power-mult-distrib*)

show *?thesis*

using *lhs rhs h-exp* **by** *simp*

qed

qed

lemma *fri-next-domain-at*:

assumes *half-pos*: $0 < len \div 2$

and *even-len*: $2 \text{ dvd } len$

and *round*: $len * pw = clength * scale$

and *d-i*: $d ! i = ((p.h \wedge i) * shift) \wedge pw$

and *d-next*: $d' ! (i \bmod (len \div 2)) = d ! i * d ! i$

shows

$$d' ! (i \bmod (len \div 2)) =$$

$$((p.h \wedge (i \bmod (len \div 2))) * shift) \wedge (pw + pw)$$

using *fri-next-domain-round*[*OF* *half-pos even-len round*, *of i*] *d-i d-next*

by *simp*

lemma *cp-eval-length*:

$length\ (p.cp\text{-}eval\ as) = clength * scale$
unfolding $p.cp\text{-}eval\text{-}def$ **using** $eval\text{-}domain\text{-}length$ **by** $simp$

lemma $f\text{-}eval\text{-}length\text{-}power$:
 $\exists n. length\ p.f\text{-}eval = 2 \wedge n$
using $p.eval\text{-}domain\text{-}length\text{-}power\ f\text{-}eval\text{-}length$ **by** $simp$

lemma $cp\text{-}eval\text{-}length\text{-}power$:
 $\exists n. length\ (p.cp\text{-}eval\ as) = 2 \wedge n$
using $p.eval\text{-}domain\text{-}length\text{-}power\ cp\text{-}eval\text{-}length$ **by** $simp$

lemma $ceil\text{-}log\text{-}Suc\text{-}le\text{-}power$:
assumes $d < 2 \wedge N$
shows $ceil\text{-}log\ (Suc\ d) \leq N$
proof ($cases\ d = 0$)
 case $True$
 then show $?thesis$
 unfolding $ceil\text{-}log\text{-}def$ **by** $simp$
next
 case $False$
 then have $d\text{-}pos: 0 < d$
 by $simp$
 have $2 \wedge floor\text{-}log\ d \leq d$
 by ($rule\ floor\text{-}log\text{-}exp2\text{-}le[OF\ d\text{-}pos]$)
 also have $\dots < 2 \wedge N$
 using $assms$.
 finally have $floor\text{-}log\ d < N$
 by ($subst\ (asm)\ power\text{-}strict\text{-}increasing\text{-}iff$) $simp\text{-}all$
 then show $?thesis$
 using $False$ **unfolding** $ceil\text{-}log\text{-}def$ **by** $simp$
qed

lemma $ceil\text{-}log\text{-}mono$:
assumes $m \leq n$
shows $ceil\text{-}log\ m \leq ceil\text{-}log\ n$
proof ($cases\ m \leq 1$)
 case $True$
 then show $?thesis$
 unfolding $ceil\text{-}log\text{-}def$ **by** $simp$
next
 case $False$
 then have $m\text{-}gt: 1 < m$
 by $simp$
 then have $n\text{-}gt: 1 < n$
 using $assms$ **by** $simp$
 have $m - 1 \leq n - 1$
 using $assms$ **by** $simp$
 then have $floor\text{-}log\ (m - 1) \leq floor\text{-}log\ (n - 1)$
 using $floor\text{-}log\text{-}le\text{-}iff$ **by** $blast$

then show ?thesis
 using m-gt n-gt unfolding ceil-log-def by simp
 qed

lemma *fri-rounds-le-eval-domain-log*:
 assumes htv: honest-trace-valid
 and cp'-def: $cp' = p.cp$ as p.f-powers
 and nrounds-def: $nrounds = \text{ceil-log } (\text{degree } cp' + 1)$
 and init-len: $\text{length } (p.cp\text{-eval } as) = 2^N$
 shows $nrounds \leq N$
proof –
 have $\text{degree } cp' \leq p.\text{maxDegree}$
 using composition-degree-from-honest-trace-valid[OF htv, of as] cp'-def by simp
 also have $\dots < \text{clength} * \text{scale}$
 by (rule maxDegree-less-eval-domain)
 also have $\dots = 2^N$
 using init-len cp-eval-length by simp
 finally have $\text{degree } cp' < 2^N$.
 then show ?thesis
 using ceil-log-Suc-le-power[of degree cp' N]
 unfolding nrounds-def by simp
 qed

lemma *query-authentication-path-length*:
 assumes create-f:
 Some (f-merkle, s1) $\in$ set-dist (execute (p.create p.f-eval) init-state)
 and i-in: $i \in \text{set } (p.\text{powers-scaled } (p.\text{index } x))$
 shows
 $\text{length } (\text{get-authentication-path } (\text{length } p.f\text{-eval}) i \text{ f-merkle}) =$
 $\text{floor-log } (\text{length } p.f\text{-eval})$
proof –
 have i-bound: $i < \text{length } p.f\text{-eval}$
 using i-in unfolding f-eval-length
 by (rule powers-scaled-index-bound)
 then have len-pos: $0 < \text{length } p.f\text{-eval}$
 by (cases p.f-eval) simp-all
 show ?thesis
 by (rule p.create-get-authentication-path-len[OF create-f len-pos])
 simp
 qed

definition *query-decommitment-transcript*
 where
 $\text{query-decommitment-transcript } idx \text{ f-merkle} \equiv$
 $\text{List.concat } (\text{map } (\lambda i. p.f\text{-eval } ! i \#$
 $\quad \text{get-authentication-path } (\text{length } p.f\text{-eval}) i \text{ f-merkle})$
 $\quad (p.\text{powers-scaled } idx))$

lemma *decommit-on-query-step-transcript*:

assumes *step*:

Some (x, t) $\in$
set-dist (*execute*
 (do {
 $p.\text{send } (p.f\text{-eval } ! i);$
 $\text{mfold2 } p.\text{send } (\text{get-authentication-path } (\text{length } p.f\text{-eval}) \ i \ f\text{-merkle})$
 }) s)

shows

$x = () \wedge$
 $P\text{Transcript } t =$
 $\text{rev } (p.f\text{-eval } ! i \ \#$
 $\text{get-authentication-path } (\text{length } p.f\text{-eval}) \ i \ f\text{-merkle}) \ @ \ P\text{Transcript } s$

proof –

from *step* **obtain** u **where**

send-leaf:

Some ($()$, u) $\in$ *set-dist* (*execute* ($p.\text{send } (p.f\text{-eval } ! i)$) s)

and *send-path*:

Some (x, t) $\in$
set-dist (*execute*
 $(\text{mfold2 } p.\text{send } (\text{get-authentication-path } (\text{length } p.f\text{-eval}) \ i \ f\text{-merkle})) \ u$)
by (*auto elim!*: $p.\text{set-dist-bindE}$)

have $x\text{-unit}$: $x = ()$

by (*cases* x) *simp*

have path-unit :

Some ($()$, t) $\in$
set-dist (*execute*
 $(\text{mfold2 } p.\text{send } (\text{get-authentication-path } (\text{length } p.f\text{-eval}) \ i \ f\text{-merkle})) \ u$)
using *send-path* $x\text{-unit}$ **by** *simp*

have tr-u : $P\text{Transcript } u = p.f\text{-eval } ! i \ \# \ P\text{Transcript } s$

using $p.\text{send-outcome}[OF \ \text{send-leaf}]$ **by** *simp*

have tr-t :

$P\text{Transcript } t =$
 $\text{rev } (\text{get-authentication-path } (\text{length } p.f\text{-eval}) \ i \ f\text{-merkle}) \ @ \ P\text{Transcript } u$
using $p.\text{mfold2-send-outcome}[OF \ \text{path-unit}]$ **by** *simp*

show *?thesis*

using $x\text{-unit}$ tr-u tr-t **by** *simp*

qed

lemma *decommit-on-query-mmap-transcript*:

assumes *outcome*:

Some ($qouts, t$) $\in$
set-dist (*execute* ($\text{mmap } (p.\text{decommit-on-query } \text{idx } f\text{-merkle})$) s)

shows

$P\text{Transcript } t =$
 $\text{rev } (\text{query-decommitment-transcript } \text{idx } f\text{-merkle}) \ @ \ P\text{Transcript } s$

proof –

have aux :

$\bigwedge qs \ qouts \ t \ s. \ \text{Some } (qouts, t) \in$

```

    set-dist (execute
      (mmap
        (map
          (λi. do {
            p.send (p.f-eval ! i);
            mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle)
          })
        qs)) s) ⇒
PTranscript t =
  rev (List.concat (map
    (λi. p.f-eval ! i #
      get-authentication-path (length p.f-eval) i f-merkle)
    qs)) @ PTranscript s
proof –
fix qs qouts t s
assume outcome-qs:
  Some (qouts, t) ∈
    set-dist (execute
      (mmap
        (map
          (λi. do {
            p.send (p.f-eval ! i);
            mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle)
          })
        qs)) s)
then show
  PTranscript t =
    rev (List.concat (map
      (λi. p.f-eval ! i #
        get-authentication-path (length p.f-eval) i f-merkle)
      qs)) @ PTranscript s
proof (induction qs arbitrary: qouts t s)
case Nil
then show ?case by simp
next
case (Cons i qs)
from Cons.premis obtain u0 u qouts' where
  send-leaf:
    Some ((), u0) ∈ set-dist (execute (p.send (p.f-eval ! i)) s)
and send-path:
  Some ((), u) ∈
    set-dist (execute
      (mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle))
      u0)
and tail:
  Some (qouts', t) ∈
    set-dist (execute
      (mmap
        (map

```



```

      (λi. do {
        p.send (p.f-eval ! i);
        mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle)
      })
    qs)) u)
  by (auto elim!: p.set-dist-bindE)
have head-tr:
  PTranscript u =
    rev (p.f-eval ! i #
      get-authentication-path (length p.f-eval) i f-merkle) @ PTranscript s
proof -
  have tr-u0: PTranscript u0 = p.f-eval ! i # PTranscript s
    using p.send-outcome[OF send-leaf] by simp
  have tr-u:
    PTranscript u =
      rev (get-authentication-path (length p.f-eval) i f-merkle) @ PTranscript u0
    using p.mfold2-send-outcome[OF send-path] by simp
  show ?thesis
    using tr-u0 tr-u by simp
qed
have tail-tr:
  PTranscript t =
    rev (List.concat (map
      (λi. p.f-eval ! i #
        get-authentication-path (length p.f-eval) i f-merkle)
      qs)) @ PTranscript u
    using Cons.IH[OF tail] .
  show ?case
    using head-tr tail-tr by simp
qed
qed
show ?thesis
  using aux[OF outcome[unfolded p.decommit-on-query-def]]
  unfolding query-decommitment-transcript-def .
qed

lemma decommit-on-query-mmap-state:
  assumes outcome:
    Some (qouts, t) ∈
      set-dist (execute (mmap (p.decommit-on-query idx f-merkle)) s)
  shows PState t = foldl concat (PState s) (query-decommitment-transcript idx f-merkle)
proof -
  have aux:
    ∧qs qouts t s. Some (qouts, t) ∈
      set-dist (execute
        (mmap
          (map
            (λi. do {
              p.send (p.f-eval ! i);

```

```

      mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle)
    })
  qs)) s) ==>
PState t =
  foldl concat (PState s)
    (List.concat (map
      (λi. p.f-eval ! i #
        get-authentication-path (length p.f-eval) i f-merkle)
      qs))
proof -
  fix qs qouts t s
  assume outcome-qs:
    Some (qouts, t) ∈
      set-dist (execute
        (mmap
          (map
            (λi. do {
              p.send (p.f-eval ! i);
              mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle)
            })
          qs)) s)
  then show
    PState t =
      foldl concat (PState s)
        (List.concat (map
          (λi. p.f-eval ! i #
            get-authentication-path (length p.f-eval) i f-merkle)
          qs))
  proof (induction qs arbitrary: qouts t s)
  case Nil
  then show ?case by simp
  next
  case (Cons i qs)
  from Cons.prem obtain u0 u qouts' where
    send-leaf:
      Some ((), u0) ∈ set-dist (execute (p.send (p.f-eval ! i)) s)
  and send-path:
    Some ((), u) ∈
      set-dist (execute
        (mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle))
        u0)
  and tail:
    Some (qouts', t) ∈
      set-dist (execute
        (mmap
          (map
            (λi. do {
              p.send (p.f-eval ! i);
              mfold2 p.send (get-authentication-path (length p.f-eval) i f-merkle)

```

```

    })
    qs)) u)
  by (auto elim!: p.set-dist-bindE)
have head-state:
  PState u =
    foldl concat (PState s)
      (p.f-eval ! i # get-authentication-path (length p.f-eval) i f-merkle)
proof -
  have st-u0: PState u0 = concat (PState s) (p.f-eval ! i)
    using p.send-outcome[OF send-leaf] by simp
  show ?thesis
    using p.mfold2-send-outcome[OF send-path] st-u0 by simp
qed
have tail-state:
  PState t =
    foldl concat (PState u)
      (List.concat (map
        (λi. p.f-eval ! i #
          get-authentication-path (length p.f-eval) i f-merkle)
        qs))
    using Cons.IH[OF tail] .
show ?case
  using head-state tail-state by simp
qed
qed
show ?thesis
  using aux[OF outcome[unfolded p.decommit-on-query-def]]
  unfolding query-decommitment-transcript-def .
qed

lemma prover-query-decommitment-transcript-in-replay-state:
  assumes final-send:
    Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls)))) s6)
  and random-idx:
    Some (idx, s7) ∈ set-dist (execute p.receive-query-index-challenge s-final)
  and query-decommit:
    Some (qouts, s8) ∈
      set-dist (execute (mmap (p.decommit-on-query idx' f-merkle)) s7)
  and fri-decommit:
    Some (fri-idx, s9) ∈
      set-dist (execute
        (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
  and prover-state-eq: prover-state = s9
  and replay: replay-state = verifier-replay-state prover-state
  obtains rest where
    PTranscript replay-state =
      rev (PTranscript s6) @ [hd (last ls)] @
        query-decommitment-transcript idx' f-merkle @ rest
proof -

```

```

have final-tr:
  PTranscript s-final = hd (last ls) # PTranscript s6
  using p.send-outcome[OF final-send] by simp
have s7-tr: PTranscript s7 = PTranscript s-final
  using receive-query-index-challenge-preserves-transcript[OF random-idx] .
have s8-tr:
  PTranscript s8 =
    rev (query-decommitment-transcript idx' f-merkle) @ PTranscript s7
  using decommit-on-query-mmap-transcript[OF query-decommit] .
have s8-s9: transcript-extends s9 s8
  by (rule mfold-transcript-extends[OF fri-decommit])
    (rule decommit-on-fri-layers-step-transcript-extends)
then obtain more where more:
  PTranscript s9 = more @ PTranscript s8
  unfolding transcript-extends-def by blast
have prover-tr:
  PTranscript prover-state =
    more @ rev (query-decommitment-transcript idx' f-merkle) @
    hd (last ls) # PTranscript s6
  using more s8-tr s7-tr final-tr prover-state-eq by simp
have replay-tr:
  PTranscript replay-state =
    rev (PTranscript s6) @ [hd (last ls)] @
    query-decommitment-transcript idx' f-merkle @ rev more
  using prover-tr replay
  unfolding verifier-replay-state-def by simp
show ?thesis
  by (rule that[OF replay-tr])
qed

```

```

lemma honest-query-decommitment-list-no-failure:
assumes created: p.created-tree p.f-eval f-merkle s0
and len-pow: length p.f-eval = 2 ^ n
and ext: s0 ≤ s
and paths:
   $\bigwedge i. i \in \text{set } (p.\text{powers-scaled } (p.\text{index } x)) \implies$ 
    length (get-authentication-path (length p.f-eval) i f-merkle) =
      floor-log (length p.f-eval)
and root: fr = value f-merkle
shows
  None ∉ dom (dist (execute
    (mmap (v.check-decommit-on-query fr (p.index x)))
    (s
      (PTranscript :=
        query-decommitment-transcript (p.index x) f-merkle @ rest
        ))))
proof –
have check-eq:
  v.check-decommit-on-query fr (p.index x) =

```

```

map
  (λi. do {
    gh ← p.read;
    let len = scale * clength;
    gh-path ← ntimes p.read (floor-log len);
    ap ← p.check-authentication-path len i gh gh-path;
    assert (ap = fr);
    return gh
  })
  (p.powers-scaled (p.index x))
unfolding v.check-decommit-on-query-def p.powers-scaled-def by simp
define qs where qs = p.powers-scaled (p.index x)
have list-nf:
  ∧qs rest s.
    set qs ⊆ set (p.powers-scaled (p.index x)) ⇒
    s0 ≤ s ⇒
    None ∉ dom (dist (execute
      (mmap
        (map
          (λi. do {
            gh ← p.read;
            let len = scale * clength;
            gh-path ← ntimes p.read (floor-log len);
            ap ← p.check-authentication-path len i gh gh-path;
            assert (ap = fr);
            return gh
          })
          qs))
        (s0
          PTranscript :=
          List.concat (map
            (λi. p.f-eval ! i #
              get-authentication-path (length p.f-eval) i f-merkle)
            qs) @ rest
          )))
proof –
fix qs rest s
assume qs-subset: set qs ⊆ set (p.powers-scaled (p.index x))
and s0-s: s0 ≤ s
then show
  None ∉ dom (dist (execute
    (mmap
      (map
        (λi. do {
          gh ← p.read;
          let len = scale * clength;
          gh-path ← ntimes p.read (floor-log len);
          ap ← p.check-authentication-path len i gh gh-path;
          assert (ap = fr);

```

```

        return qh
      })
    (qs))
  (s(
    PTranscript :=
      List.concat (map
        (λi. p.f-eval ! i #
          get-authentication-path (length p.f-eval) i f-merkle)
        qs) @ rest
      )))
proof (induction qs arbitrary: rest s)
  case Nil
  then show ?case by simp
next
  case (Cons i qs)
have i-in:  $i \in \text{set } (p.\text{powers-scaled } (p.\text{index } x))$ 
  using Cons.premis(1) by simp
let ?path = get-authentication-path (length p.f-eval) i f-merkle
let ?msg = p.f-eval ! i # ?path
let ?tail =
  List.concat (map
    (λi. p.f-eval ! i #
      get-authentication-path (length p.f-eval) i f-merkle)
    qs) @ rest
have i-bound:  $i < \text{length } p.f\text{-eval}$ 
  using i-in unfolding f-eval-length
  by (rule powers-scaled-index-bound)
have path-len:
  length ?path = floor-log (length p.f-eval)
  using paths[OF i-in] .
have step-nf:
  None  $\notin \text{dom } (\text{dist } (\text{execute }
    (\text{do } \{
      qh \leftarrow p.\text{read};
      \text{let } \text{len} = \text{scale} * \text{clength};
      qh\text{-path} \leftarrow \text{ntimes } p.\text{read } (\text{floor-log } \text{len});
      ap \leftarrow p.\text{check-authentication-path } \text{len } i \text{ } qh \text{ } qh\text{-path};
      \text{assert } (ap = \text{fr});
      \text{return } qh
    \})))$ 
  (s(PTranscript := ?msg @ ?tail))))
using p.honest-query-decommitment-no-failure[
  OF created len-pow i-bound refl Cons.premis(2) path-len root,
  of ?tail]
unfolding f-eval-length by (simp add: mult.commute Let-def)
let ?step =
  do {
    qh  $\leftarrow$  p.read;
    let len = scale * clength;

```

```

     $qh\text{-}path \leftarrow ntimes\ p.read\ (floor\text{-}log\ len);$ 
     $ap \leftarrow p.check\text{-}authentication\text{-}path\ len\ i\ qh\ qh\text{-}path;$ 
     $assert\ (ap = fr);$ 
     $return\ qh$ 
  }
let  $?tail\text{-}m =$ 
   $mmap$ 
  ( $map$ 
    ( $\lambda i.$   $do\ \{$ 
       $qh \leftarrow p.read;$ 
       $let\ len = scale * clength;$ 
       $qh\text{-}path \leftarrow ntimes\ p.read\ (floor\text{-}log\ len);$ 
       $ap \leftarrow p.check\text{-}authentication\text{-}path\ len\ i\ qh\ qh\text{-}path;$ 
       $assert\ (ap = fr);$ 
       $return\ qh$ 
    })
  )
have  $case\text{-}eq:$ 
   $?case =$ 
  ( $None \notin dom\ (dist\ (execute$ 
    ( $?step \gg (\lambda qh. ?tail\text{-}m \gg (\lambda qhs. return\ (qh \# qhs))))$ 
    ( $s(PTranscript := ?msg @ ?tail))))$ 
  )
  by ( $simp\ add: Let\text{-}def$ )
show  $?case$ 
  unfolding  $case\text{-}eq$ 
proof ( $rule\ no\text{-}failure\text{-}bindI[OF\ step\text{-}nf]$ )
fix  $qh\ t$ 
assume  $step:$ 
   $Some\ (qh, t) \in set\text{-}dist\ (execute$ 
    ( $do\ \{$ 
       $qh \leftarrow p.read;$ 
       $let\ len = scale * clength;$ 
       $qh\text{-}path \leftarrow ntimes\ p.read\ (floor\text{-}log\ len);$ 
       $ap \leftarrow p.check\text{-}authentication\text{-}path\ len\ i\ qh\ qh\text{-}path;$ 
       $assert\ (ap = fr);$ 
       $return\ qh$ 
    })
    ( $s(PTranscript := ?msg @ ?tail)))$ 
  )
have  $head\text{-}state: s0 \leq t \wedge PTranscript\ t = ?tail$ 
proof –
  from  $step$  obtain  $r\ u\ path\ u'\ ap\ u''\ u'''$  where
     $read\text{-}leaf:$ 
     $Some\ (r, u) \in set\text{-}dist\ (execute\ p.read\ (s(PTranscript := ?msg @ ?tail)))$ 
  and  $read\text{-}path:$ 
     $Some\ (path, u') \in$ 
     $set\text{-}dist\ (execute\ (ntimes\ p.read\ (floor\text{-}log\ (scale * clength)))\ u)$ 
  and  $check:$ 
     $Some\ (ap, u'') \in$ 
     $set\text{-}dist\ (execute\ (p.check\text{-}authentication\text{-}path\ (scale * clength)\ i\ r\ path)\ u')$ 

```

```

and assert-ok:
  Some ( $()$ ,  $u''$ )  $\in$  set-dist (execute (assert ( $ap = fr$ ))  $u''$ )
and ret:
  Some ( $qh$ ,  $t$ )  $\in$  set-dist (execute (return  $r$ )  $u''$ )
by (auto simp: Let-def elim!: p.set-dist-bindE)
have read-start-eq:
   $s(PTranscript := ?msg @ ?tail) =$ 
   $s(PTranscript := p.f\text{-eval} ! i \# (?path @ ?tail))$ 
by simp
have read-leaf-replay:
   $r = p.f\text{-eval} ! i \wedge$ 
   $u = s($ 
     $PState := \text{concat } (PState\ s) (p.f\text{-eval} ! i),$ 
     $PTranscript := ?path @ ?tail)$ 
using p.read-cons-outcome[OF read-leaf[unfolded read-start-eq]] by simp
have r-eq:  $r = p.f\text{-eval} ! i$ 
using read-leaf-replay by simp
have read-leaf-res:
   $u = s($ 
     $PState := \text{concat } (PState\ s) (p.f\text{-eval} ! i),$ 
     $PTranscript := ?path @ ?tail)$ 
using read-leaf-replay by simp
have u-base:
   $u =$ 
   $(s(PState := \text{concat } (PState\ s) (p.f\text{-eval} ! i)))$ 
   $(PTranscript := ?path @ ?tail)$ 
using read-leaf-res by simp
have scale-len:  $scale * clength = length\ p.f\text{-eval}$ 
using f-eval-length by (simp add: mult.commute)
have path-replay:
   $path = ?path \wedge$ 
   $u' =$ 
   $(s(PState := \text{concat } (PState\ s) (p.f\text{-eval} ! i)))$ 
   $($ 
     $PState :=$ 
     $\text{foldl concat}$ 
     $(\text{concat } (PState\ s) (p.f\text{-eval} ! i))$ 
     $?path,$ 
     $PTranscript := ?tail$ 
   $)$ 
using p.ntimes-read-prefix-outcome[
  OF read-path[unfolded scale-len path-len[symmetric] u-base]]
by simp
have path-eq:  $path = ?path$ 
using path-replay by simp
have path-res:
   $u' =$ 
   $(s(PState := \text{concat } (PState\ s) (p.f\text{-eval} ! i)))$ 
   $($ 

```



```

    PState :=
      foldl concat
        (concat (PState s) (p.f-eval ! i))
        ?path,
    PTranscript := ?tail
  )
  using path-replay by simp
have s0-u': s0 ≤ u'
  using Cons.premis path-res
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have u'-u'': u' ≤ u''
  using p.check-created-tree-outcome[
    OF created len-pow i-bound refl s0-u'
    check[unfolded scale-len r-eq path-eq]]
  unfolding root by simp
have check-transcript: PTranscript u'' = PTranscript u'
  using p.check-authentication-path-preserves-channel(2)[OF check] .
have assert-state: u''' = u''
  using assert-ok unfolding assert-def
  by (cases ap = fr)
    (auto simp: return.rep-eq dist-return-def throw.rep-eq dist-throw-def
      dist-delta-dist delta-map-def set-dist-def)
have ret-state: t = u'''
  using ret by simp
have u''-t: u'' ≤ t
  unfolding assert-state[symmetric] ret-state
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have t-transcript: PTranscript t = ?tail
  using path-res check-transcript assert-state ret-state by simp
show ?thesis
proof
  show s0 ≤ t
    using s0-u' u'-u'' u''-t
    unfolding less-eq-hash-ext-def less-eq-fmap-def
    by metis
  show PTranscript t = ?tail
    using t-transcript .
qed
qed
have s0-t: s0 ≤ t
  using head-state by simp
have t-transcript: PTranscript t = ?tail
  using head-state by simp
have tail-nf-start:
  None ∉ dom (dist (execute ?tail-m (t(PTranscript := ?tail))))
  using Cons.IH[of t rest] Cons.premis(1) s0-t by simp
have t-update: t(PTranscript := ?tail) = t
  using t-transcript by simp
have tail-nf: None ∉ dom (dist (execute ?tail-m t))

```

```

    using tail-nf-start unfolding t-update .
  show None  $\notin$  dom (dist (execute (?tail-m  $\gg$  ( $\lambda$  qhs. return (qh # qhs))) t))
    by (rule no-failure-bindI[OF tail-nf]) simp
qed
qed
qed
show ?thesis
  using list-nf[of p.powers-scaled (p.index x) s rest] ext
  unfolding query-decommitment-transcript-def check-eq qs-def by simp
qed

lemma honest-query-decommitment-replay-no-failure:
  assumes create-f:
    Some (f-merkle, s0)  $\in$ 
      set-dist (execute (p.create p.f-eval) init-state)
  and ext: s0  $\leq$  s
  and root: fr = value f-merkle
  and transcript:
    PTranscript s =
      query-decommitment-transcript (p.index x) f-merkle @ rest
  shows
    None  $\notin$  dom (dist (execute
      (mmap (v.check-decommit-on-query fr (p.index x))) s))
proof -
  have created: p.created-tree p.f-eval f-merkle s0
  using p.create-outcome[OF create-f] by simp
  from f-eval-length-power obtain n where len-pow: length p.f-eval =  $2^n$ 
  by blast
  have paths:
     $\bigwedge i. i \in \text{set } (p.powers-scaled (p.index x)) \implies$ 
      length (get-authentication-path (length p.f-eval) i f-merkle) =
        floor-log (length p.f-eval)
  using query-authentication-path-length[OF create-f] by blast
  have nf:
    None  $\notin$  dom (dist (execute
      (mmap (v.check-decommit-on-query fr (p.index x)))
      (s(
        PTranscript :=
          query-decommitment-transcript (p.index x) f-merkle @ rest
        )))))
  by (rule honest-query-decommitment-list-no-failure[
    OF created len-pow ext paths root])
  have update:
    s(PTranscript :=
      query-decommitment-transcript (p.index x) f-merkle @ rest) = s
  using transcript by simp
  show ?thesis
  using nf unfolding update .
qed

```

lemma *honest-query-decommitment-replay-outcome*:

assumes *paths*:

$\bigwedge i. i \in \text{set } (p.\text{powers-scaled } \text{idx}) \implies$
 $\text{length } (\text{get-authentication-path } (\text{length } p.\text{f-eval}) \ i \ f\text{-merkle}) =$
 $\text{floor-log } (\text{length } p.\text{f-eval})$

and *outcome*:

$\text{Some } (fv, t) \in$
 $\text{set-dist } (\text{execute}$
 $\quad (\text{mmap } (v.\text{check-decommit-on-query } fr \ \text{idx}))$
 $\quad (s(|P\text{Transcript} := \text{query-decommitment-transcript } \text{idx } f\text{-merkle } @ \ \text{rest}|)))$

shows

$fv = \text{map } (\lambda i. p.\text{f-eval } ! \ i) \ (p.\text{powers-scaled } \text{idx}) \wedge$
 $P\text{Transcript } t = \text{rest} \wedge$
 $P\text{State } t = \text{foldl concat } (P\text{State } s) \ (\text{query-decommitment-transcript } \text{idx } f\text{-merkle}) \wedge$
 $s(|P\text{Transcript} := \text{query-decommitment-transcript } \text{idx } f\text{-merkle } @ \ \text{rest}|) \leq t$

proof –

have *check-eq*:

$v.\text{check-decommit-on-query } fr \ \text{idx} =$
 map
 $\quad (\lambda i. \text{do } \{$
 $\quad \quad qh \leftarrow p.\text{read};$
 $\quad \quad \text{let } len = \text{scale} * \text{clength};$
 $\quad \quad qh\text{-path} \leftarrow \text{ntimes } p.\text{read } (\text{floor-log } len);$
 $\quad \quad ap \leftarrow p.\text{check-authentication-path } len \ i \ qh \ qh\text{-path};$
 $\quad \quad \text{assert } (ap = fr);$
 $\quad \quad \text{return } qh$
 $\quad \})$
 $\quad (p.\text{powers-scaled } \text{idx})$

unfolding $v.\text{check-decommit-on-query-def } p.\text{powers-scaled-def}$ **by** *simp*

have *list-out*:

$\bigwedge qs \ fv \ t \ s \ \text{rest}.$
 $\text{set } qs \subseteq \text{set } (p.\text{powers-scaled } \text{idx}) \implies$
 $\text{Some } (fv, t) \in$
 $\text{set-dist } (\text{execute}$
 $\quad (\text{mmap}$
 $\quad \quad (\text{map}$
 $\quad \quad \quad (\lambda i. \text{do } \{$
 $\quad \quad \quad \quad qh \leftarrow p.\text{read};$
 $\quad \quad \quad \quad \text{let } len = \text{scale} * \text{clength};$
 $\quad \quad \quad \quad qh\text{-path} \leftarrow \text{ntimes } p.\text{read } (\text{floor-log } len);$
 $\quad \quad \quad \quad ap \leftarrow p.\text{check-authentication-path } len \ i \ qh \ qh\text{-path};$
 $\quad \quad \quad \quad \text{assert } (ap = fr);$
 $\quad \quad \quad \quad \text{return } qh$
 $\quad \quad \quad \})$
 $\quad \quad qs))$
 $\quad (s(|$
 $\quad \quad P\text{Transcript} :=$
 $\quad \quad \text{List.concat } (\text{map}$

```

      (λi. p.f-eval ! i #
        get-authentication-path (length p.f-eval) i f-merkle)
      qs) @ rest
    ))) ⇒
fv = map (λi. p.f-eval ! i) qs ∧
PTranscript t = rest ∧
PState t =
  foldl concat (PState s)
  (List.concat (map
    (λi. p.f-eval ! i #
      get-authentication-path (length p.f-eval) i f-merkle)
    qs)) ∧
s(
  PTranscript :=
  List.concat (map
    (λi. p.f-eval ! i #
      get-authentication-path (length p.f-eval) i f-merkle)
    qs) @ rest
) ≤ t
proof –
fix qs fv t s rest
assume qs-subset: set qs ⊆ set (p.powers-scaled idx)
and outcome-qs:
  Some (fv, t) ∈
  set-dist (execute
    (mmap
      (map
        (λi. do {
          qh ← p.read;
          let len = scale * clength;
          qh-path ← ntimes p.read (floor-log len);
          ap ← p.check-authentication-path len i qh qh-path;
          assert (ap = fr);
          return qh
        })
      qs))
    (s(
      PTranscript :=
      List.concat (map
        (λi. p.f-eval ! i #
          get-authentication-path (length p.f-eval) i f-merkle)
        qs) @ rest
    )))
then show
fv = map (λi. p.f-eval ! i) qs ∧
PTranscript t = rest ∧
PState t =
  foldl concat (PState s)
  (List.concat (map

```

```

      (λi. p.f-eval ! i #
        get-authentication-path (length p.f-eval) i f-merkle)
      qs)) ∧
s(
  PTranscript :=
    List.concat (map
      (λi. p.f-eval ! i #
        get-authentication-path (length p.f-eval) i f-merkle)
      qs) @ rest
) ≤ t
proof (induction qs arbitrary: fv t s rest)
  case Nil
  then show ?case
    unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
next
  case (Cons i qs)
  let ?path = get-authentication-path (length p.f-eval) i f-merkle
  let ?msg = p.f-eval ! i # ?path
  let ?tail =
    List.concat (map
      (λi. p.f-eval ! i #
        get-authentication-path (length p.f-eval) i f-merkle)
      qs) @ rest
  let ?start = s(PTranscript := ?msg @ ?tail)
  let ?step =
    do {
      qh ← p.read;
      let len = scale * clength;
      qh-path ← ntimes p.read (floor-log len);
      ap ← p.check-authentication-path len i qh qh-path;
      assert (ap = fr);
      return qh
    }
  let ?tail-m =
    mmap
      (map
        (λi. do {
          qh ← p.read;
          let len = scale * clength;
          qh-path ← ntimes p.read (floor-log len);
          ap ← p.check-authentication-path len i qh qh-path;
          assert (ap = fr);
          return qh
        })
      qs)
  have cons-out:
    Some (fv, t) ∈
      set-dist (execute
        (?step ≫ (λqh. ?tail-m ≫ (λfv-tail. return (qh # fv-tail))))))

```

```

    ?start)
  using Cons.premis(2) by simp
from cons-out obtain qh u where
  head:
    Some (qh, u) ∈ set-dist (execute ?step ?start)
  and tail-bind:
    Some (fv, t) ∈
      set-dist (execute (?tail-m ≫ (λfv-tail. return (qh # fv-tail))) u)
proof -
  show ?thesis
  using cons-out
proof (rule p.set-dist-bindE)
  fix qh u
  assume head:
    Some (qh, u) ∈ set-dist (execute ?step ?start)
  and tail-bind:
    Some (fv, t) ∈
      set-dist (execute
        (?tail-m ≫ (λfv-tail. return (qh # fv-tail))) u)
  show ?thesis
  by (rule that[OF head tail-bind])
qed
qed
from tail-bind obtain fv-tail u-tail where
  tail0:
    Some (fv-tail, u-tail) ∈ set-dist (execute ?tail-m u)
  and ret-tail:
    Some (fv, t) ∈ set-dist (execute (return (qh # fv-tail)) u-tail)
proof -
  show ?thesis
  using tail-bind
proof (rule p.set-dist-bindE)
  fix fv-tail u-tail
  assume tail0:
    Some (fv-tail, u-tail) ∈ set-dist (execute ?tail-m u)
  and ret-tail:
    Some (fv, t) ∈
      set-dist (execute (return (qh # fv-tail)) u-tail)
  show ?thesis
  by (rule that[OF tail0 ret-tail])
qed
qed
have ret-tail-res: fv = qh # fv-tail ∧ t = u-tail
  using ret-tail by simp
have tail:
  Some (fv-tail, t) ∈ set-dist (execute ?tail-m u)
  using tail0 ret-tail-res by simp
have fv-eq: fv = qh # fv-tail
  using ret-tail-res by simp

```

```

have i-in:  $i \in \text{set } (p.\text{powers-scaled } \text{idx})$ 
  using Cons.premis(1) by simp
have path-len:
   $\text{length } ?\text{path} = \text{floor-log } (\text{scale} * \text{clength})$ 
  using paths[OF i-in] f-eval-length by (simp add: mult.commute)
from head obtain qh-path ap s1 s2 s3 s4 where
  read-leaf:  $\text{Some } (qh, s1) \in \text{set-dist } (\text{execute } p.\text{read } ?\text{start})$ 
  and read-path:
     $\text{Some } (qh\text{-path}, s2) \in$ 
     $\text{set-dist } (\text{execute } (\text{ntimes } p.\text{read } (\text{floor-log } (\text{scale} * \text{clength})))) s1)$ 
  and check:
     $\text{Some } (ap, s3) \in$ 
     $\text{set-dist } (\text{execute } (p.\text{check-authentication-path } (\text{scale} * \text{clength}) i qh qh\text{-path}) s2)$ 
  and assert-ok:
     $\text{Some } ((), s4) \in \text{set-dist } (\text{execute } (\text{assert } (ap = fr)) s3)$ 
  and ret:  $\text{Some } (qh, u) \in \text{set-dist } (\text{execute } (\text{return } qh) s4)$ 
  by (auto simp: Let-def elim!: p.set-dist-bindE)
have leaf-res:
   $qh = p.f\text{-eval } ! i \wedge$ 
   $s1 = s(|$ 
     $P\text{State} := \text{concat } (P\text{State } s) (p.f\text{-eval } ! i),$ 
     $P\text{Transcript} := ?\text{path} @ ?\text{tail})$ 
proof –
  have start-eq:
     $?\text{start} =$ 
     $s(|P\text{Transcript} := p.f\text{-eval } ! i \# (?\text{path} @ ?\text{tail}))$ 
  by simp
show ?thesis
  using p.read-cons-outcome[OF read-leaf[unfolded start-eq]] by simp
qed
let ?s-leaf =  $s(|P\text{State} := \text{concat } (P\text{State } s) (p.f\text{-eval } ! i))$ 
have s1-eq:  $s1 = ?s\text{-leaf}(|P\text{Transcript} := ?\text{path} @ ?\text{tail})$ 
  using leaf-res by simp
have path-res:
   $qh\text{-path} = ?\text{path} \wedge$ 
   $s2 = ?s\text{-leaf}(|$ 
     $P\text{State} := \text{foldl } \text{concat } (P\text{State } ?s\text{-leaf}) ?\text{path},$ 
     $P\text{Transcript} := ?\text{tail})$ 
  using p.ntimes-read-prefix-outcome[
    OF read-path[unfolded path-len[symmetric] s1-eq]
  by simp
have s3-tr:  $P\text{Transcript } s3 = ?\text{tail}$ 
  using p.check-authentication-path-preserves-channel(2)[OF check] path-res
  by simp
have s4-tr:  $P\text{Transcript } s4 = ?\text{tail}$ 
  using assert-outcomeD(2)[OF assert-ok] s3-tr by simp
have u-eq:  $u = s4$ 
  using ret by simp

```

```

have u-tr: PTranscript u = ?tail
  using s4-tr u-eq by simp
have u-state:
  PState u = foldl concat (PState s) ?msg
proof –
  have s3-state: PState s3 = PState s2
    using p.check-authentication-path-preserves-channel(1)[OF check] .
  have s4-state: PState s4 = PState s3
    using assert-outcomeD(2)[OF assert-ok] by simp
  show ?thesis
    using leaf-res path-res s3-state s4-state u-eq by simp
qed
have start-u: ?start ≤ u
proof –
  have ?start ≤ s1
    using read-leaf by (rule read-hash-extends)
  moreover have s1 ≤ s2
    using read-path by (rule ntimes-read-hash-extends)
  moreover have s2 ≤ s3
    using check by (rule check-authentication-path-hash-extends)
  moreover have s3 ≤ s4
    using assert-ok by (rule assert-hash-extends)
  moreover have s4 ≤ u
    using u-eq p.hash-ext-refl by simp
  ultimately show ?thesis
    by (meson p.hash-ext-trans)
qed
have u-update: u(PTranscript := ?tail) = u
  using u-tr by simp
have tail-start:
  Some (fv-tail, t) ∈
    set-dist (execute ?tail-m (u(PTranscript := ?tail)))
  using tail unfolding u-update .
have tail-out:
  fv-tail = map (λi. p.f-eval ! i) qs ∧
  PTranscript t = rest ∧
  PState t =
    foldl concat (PState u)
      (List.concat (map
        (λi. p.f-eval ! i #
          get-authentication-path (length p.f-eval) i f-merkle
            qs)) ∧
    u(PTranscript := ?tail) ≤ t
proof (rule Cons.IH)
  show set qs ⊆ set (p.powers-scaled idx)
    using Cons.premis(1) by simp
  show Some (fv-tail, t) ∈ set-dist (execute ?tail-m
    (u(PTranscript := ?tail)))
    using tail-start .

```



```

qed
have start-t: ?start ≤ t
  using start-u tail-out unfolding u-update by (meson p.hash-ext-trans)
show ?case
  using fv-eq leaf-res tail-out start-t u-state by simp
qed
qed
have outcome':
  Some (fv, t) ∈
    set-dist (execute
      (mmap
        (map
          (λi. do {
            qh ← p.read;
            let len = scale * clength;
            qh-path ← ntimes p.read (floor-log len);
            ap ← p.check-authentication-path len i qh qh-path;
            assert (ap = fr);
            return qh
          })
        (p.powers-scaled idx)))
    (s(
      PTranscript :=
        List.concat (map
          (λi. p.f-eval ! i #
            get-authentication-path (length p.f-eval) i f-merkle)
          (p.powers-scaled idx)) @ rest
    )))
  using outcome
unfolding check-eq query-decommitment-transcript-def by simp
have result:
  fv = map (λi. p.f-eval ! i) (p.powers-scaled idx) ∧
  PTranscript t = rest ∧
  PState t =
    foldl concat (PState s)
    (List.concat (map
      (λi. p.f-eval ! i #
        get-authentication-path (length p.f-eval) i f-merkle)
      (p.powers-scaled idx))) ∧
  s(
    PTranscript :=
      List.concat (map
        (λi. p.f-eval ! i #
          get-authentication-path (length p.f-eval) i f-merkle)
        (p.powers-scaled idx)) @ rest
  ) ≤ t
by (rule list-out[OF - outcome']) simp
show ?thesis
  using result unfolding query-decommitment-transcript-def by simp

```

qed

lemma *check-decommit-on-query-step-preserves-query-counter:*

assumes *step-in*: $m \in \text{set } (v.\text{check-decommit-on-query } \text{fr } \text{idx})$

and *outcome*: $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s)$

shows $P\text{QueryCounter } t = P\text{QueryCounter } s$

proof –

from *step-in* **obtain** *i* **where**

m-eq:

```

m = do {
  qh ← p.read;
  let len = scale * clength;
  qh-path ← ntimes p.read (floor-log len);
  ap ← p.check-authentication-path len i qh qh-path;
  assert (ap = fr);
  return qh
}

```

unfolding *v.check-decommit-on-query-def* *p.powers-scaled-def* **by** *auto*

from *outcome*[*unfolded m-eq*] **obtain** *qh s1 qh-path s2 ap s3 s4* **where**

read-qh: $\text{Some } (qh, s1) \in \text{set-dist } (\text{execute } p.\text{read } s)$

and *read-path*:

$\text{Some } (qh\text{-path}, s2) \in$
 $\text{set-dist } (\text{execute } (\text{ntimes } p.\text{read } (\text{floor-log } (\text{scale} * \text{clength})))) \ s1)$

and *check*:

$\text{Some } (ap, s3) \in$
 $\text{set-dist } (\text{execute } (p.\text{check-authentication-path } (\text{scale} * \text{clength}) \ i \ qh \ qh\text{-path}) \ s2)$

and *assert-ok*: $\text{Some } ((), s4) \in \text{set-dist } (\text{execute } (\text{assert } (ap = fr)) \ s3)$

and *ret*: $\text{Some } (x, t) \in \text{set-dist } (\text{execute } (\text{return } qh) \ s4)$

by (*auto simp: Let-def elim!: p.set-dist-bindE*)

have *q-s1*: $P\text{QueryCounter } s1 = P\text{QueryCounter } s$

using *read-preserves-counters*[*OF read-qh*] **by** *simp*

have *q-s2*: $P\text{QueryCounter } s2 = P\text{QueryCounter } s1$

by (*rule ntimes-read-preserves-query-counter*[*OF read-path*])

have *q-s3*: $P\text{QueryCounter } s3 = P\text{QueryCounter } s2$

using *p.check-authentication-path-preserves-channel*(6)[*OF check*] .

have *s4-eq*: $s4 = s3$

using *assert-outcomeD*(2)[*OF assert-ok*] .

have *t-eq*: $t = s4$

using *ret* **by** *simp*

show *?thesis*

using *q-s1 q-s2 q-s3* **unfolding** *s4-eq t-eq* **by** *simp*

qed

lemma *honest-root-alpha-degree-fri-final-query-from-prover-no-failure:*

assumes *htv*: *honest-trace-valid*

and *create-f*:

$\text{Some } (f\text{-merkle}, s1) \in$
 $\text{set-dist } (\text{execute } (p.\text{create } p.f\text{-eval}) \ \text{init-state})$

and *send-f*:

```

    Some ( $()$ ,  $s2$ )  $\in$  set-dist (execute ( $p$ .send (value f-merkle))  $s1$ )
and alphas:
    Some ( $as$ ,  $s3$ )  $\in$ 
      set-dist (execute
        (mmap (replicate (length spec)
          (do {
             $a \leftarrow p$ .receive-alpha-challenge;
             $p$ .send  $a$ ;
            return  $a$ 
          })))  $s2$ )
and  $cp'$ -def:  $cp' = p$ .cp as  $p$ .f-powers
and send-degree:
    Some ( $()$ ,  $s4$ )  $\in$  set-dist (execute ( $p$ .send (of-nat (degree  $cp'$ )))  $s3$ )
and create-cp:
    Some ( $cp$ -merkle,  $s5$ )  $\in$  set-dist (execute ( $p$ .create ( $p$ .cp-eval  $as$ ))  $s4$ )
and fri:
    Some ( $(ps, ds, ls, ms)$ ,  $s6$ )  $\in$ 
      set-dist (execute
        ( $p$ .fri-commit nrounds [ $cp'$ ] [ $p$ .eval-domain] [ $p$ .cp-eval  $as$ ] [ $cp$ -merkle])  $s5$ )
and final-send:
    Some ( $()$ ,  $s$ -final)  $\in$  set-dist (execute ( $p$ .send (hd (last  $ls$ )))  $s6$ )
and random-idx:
    Some ( $idx$ ,  $s7$ )  $\in$  set-dist (execute  $p$ .receive-query-index-challenge  $s$ -final)
and  $idx$ -def:  $idx' = p$ .index (to-nat  $idx$ )
and query-decommit:
    Some ( $qouts$ ,  $s8$ )  $\in$ 
      set-dist (execute (mmap ( $p$ .decommit-on-query  $idx'$  f-merkle))  $s7$ )
and fri-decommit:
    Some ( $fri$ - $idx$ ,  $s9$ )  $\in$ 
      set-dist (execute
        (mfold  $idx'$  ( $p$ .decommit-on-fri-layers (butlast (zip  $ls$   $ms$ ))))  $s8$ )
and prover-state-eq: prover-state =  $s9$ 
and replay: replay-state = verifier-replay-state prover-state
shows
    None  $\notin$  dom (dist (execute
      (do {
         $fr \leftarrow p$ .read;
         $as' \leftarrow$  mmap (replicate (length spec)
          (do {
             $a0 \leftarrow p$ .receive-alpha-challenge;
            let  $a0' = a0$ ;
             $a1 \leftarrow p$ .read;
            let  $a1' = a1$ ;
            assert ( $a0' = a1'$ );
            return  $a1'$ 
          })))
         $dg \leftarrow p$ .read;
        assert (to-nat  $dg \leq p$ .maxDegree);
         $fl \leftarrow$  ntimes  $v$ .receive-fri-commits nrounds;

```

```

    final ← p.read;
    idxv ← p.receive-query-index-challenge;
    let idxv' = p.index (to-nat idxv);
    fv ← mmap (v.check-decommit-on-query fr idxv');
    return (fr, as', dg, fl, final, idxv, fv)
  }) replay-state))
proof –
  let ?round =
    do {
      a0 ← p.receive-alpha-challenge;
      let a0' = a0;
      a1 ← p.read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    }
  let ?prefix-final =
    do {
      fr ← p.read;
      as' ← mmap (replicate (length spec) ?round);
      dg ← p.read;
      assert (to-nat dg ≤ p.maxDegree);
      fl ← ntimes v.receive-fri-commits nrounds;
      final ← p.read;
      return (fr, as', dg, fl, final)
    }
  let ?full =
    do {
      fr ← p.read;
      as' ← mmap (replicate (length spec) ?round);
      dg ← p.read;
      assert (to-nat dg ≤ p.maxDegree);
      fl ← ntimes v.receive-fri-commits nrounds;
      final ← p.read;
      idxv ← p.receive-query-index-challenge;
      let idxv' = p.index (to-nat idxv);
      fv ← mmap (v.check-decommit-on-query fr idxv');
      return (fr, as', dg, fl, final, idxv, fv)
    }
  have prefix-nf:
    None ∉ dom (dist (execute ?prefix-final replay-state))
  by (rule honest-root-alpha-degree-fri-final-from-prover-no-failure[
    OF htv create-f send-f alphas cp'-def send-degree create-cp fri
    final-send random-idx idx-def query-decommit fri-decommit prover-state-eq replay])
  have root-alpha-tr:
    length as = length spec ∧
    PTranscript s4 = of-nat (degree cp') # rev as @ [value f-merkle]
  by (rule prover-root-alpha-degree-transcript[
    OF create-f send-f alphas cp'-def send-degree])

```

```

have later-hash:  $s3 \leq \text{prover-state}$ 
  by (rule prover-later-phases-hash-extends[
    OF send-degree create-cp fri final-send random-idx query-decommit fri-decommit prover-state-eq])
have alpha-lookups:
   $\forall i < \text{length as}.$ 
    fmllookup (HashMap replay-state)
      (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat (concat (PState re-
play-state) (value f-merkle)) (take i as))) =
      Some (as ! i)
  by (rule prover-alpha-lookups-in-replay-state[
    OF create-f send-f alphas later-hash replay])
from fri-commit-replay-data[OF fri] obtain roots challenges where
  len-roots: length roots = nrounds
  and len-challenges: length challenges = nrounds
  and tr-s6: PTranscript s6 = rev roots @ PTranscript s5
  and st-s6: PState s6 = foldl concat (PState s5) roots
  and fri-hash:  $s5 \leq s6$ 
  and fri-lookups-s6:
     $\forall i < \text{length roots}.$ 
      fmllookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i)
roots))) =
      Some (challenges ! i)
  by blast
from prover-query-decommitment-transcript-in-replay-state[
  OF final-send random-idx query-decommit fri-decommit prover-state-eq replay]
obtain rest where replay-query-tr:
  PTranscript replay-state =
    rev (PTranscript s6) @ [hd (last ls)] @
    query-decommitment-transcript idx' f-merkle @ rest
  by blast
have tr-s5: PTranscript s5 = PTranscript s4
  using create-preserves-transcript[OF create-cp] .
have replay-prefix:
  PTranscript replay-state =
    value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @
    roots @ [hd (last ls)] @ query-decommitment-transcript idx' f-merkle @ rest
  using replay-query-tr tr-s6 tr-s5 root-alpha-tr cp'-def by simp
have st-s1: PState s1 = 0
  using create-preserves-state[OF create-f]
  unfolding init-state-def by simp
have send-f-res:
  s2 = s1 (
    PState := concat (PState s1) (value f-merkle),
    PTranscript := value f-merkle # PTranscript s1)
  using p.send-outcome[OF send-f] .
have alpha-data:
  s2  $\leq$  s3  $\wedge$  PState s3 = foldl concat (PState s2) as
  using alpha-mmap-outcome[OF alphas] by simp
have send-degree-res:

```

```

s4 = s3(|
  PState := concat (PState s3) (of-nat (degree cp')),
  PTranscript := of-nat (degree cp') # PTranscript s3|)
using p.send-outcome[OF send-degree] .
have st-s5:
  PState s5 =
    concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
      (of-nat (degree (p.cp as p.f-powers)))
  using st-s1 send-f-res alpha-data send-degree-res
    create-preserves-state[OF create-cp] replay cp'-def
  unfolding verifier-replay-state-def by simp
have after-hash: s6 ≤ prover-state
  by (rule prover-after-fri-hash-extends[
    OF final-send random-idx query-decommit fri-decommit prover-state-eq])
have fri-lookups:
  ∀ i < length roots.
    fmlookup (HashMap replay-state)
      (FiatShamirChallenge (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          (of-nat (degree (p.cp as p.f-powers))))
        (take (Suc i) roots))) =
      Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have lookup-s6:
    fmlookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i)
roots))) =
      Some (challenges ! i)
  using fri-lookups-s6 i-bound by simp
  have lookup-prover:
    fmlookup (HashMap prover-state) (FiatShamirChallenge (foldl concat (PState s5) (take
(Suc i) roots))) =
      Some (challenges ! i)
  using p.hash-extension-lookup[OF lookup-s6 after-hash] .
show fmlookup (HashMap replay-state)
  (FiatShamirChallenge (foldl concat
    (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
      (of-nat (degree (p.cp as p.f-powers))))
    (take (Suc i) roots))) =
    Some (challenges ! i)
  using lookup-prover st-s5 replay
  unfolding verifier-replay-state-def by simp
qed
have final-send-res:
  s-final = s6(|
    PState := concat (PState s6) (hd (last ls)),
    PTranscript := hd (last ls) # PTranscript s6|)
  using p.send-outcome[OF final-send] .

```

```

have query-lookup-replay:
  fmlookup (HashMap replay-state) (QueryIndexChallenge (PQueryCounter s-final) (PState
s-final)) = Some idx
  by (rule prover-query-random-lookup-in-replay-state[
    OF random-idx query-decommit fri-decommit prover-state-eq replay])
have s1-s2: s1 ≤ s2
  using p.send-outcome[OF send-f]
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have s1-s3: s1 ≤ s3
  using s1-s2 alpha-data by (meson p.hash-ext-trans)
have s3-prover: s3 ≤ prover-state
  using later-hash .
have s1-prover: s1 ≤ prover-state
  using s1-s3 s3-prover by (meson p.hash-ext-trans)
have s1-replay: s1 ≤ replay-state
  using s1-prover replay
  unfolding verifier-replay-state-def less-eq-hash-ext-def less-eq-fmap-def
  by simp
have full-eq:
  ?full =
    (?prefix-final >>=
      (λout. p.receive-query-index-challenge >>=
        (λidxv.
          let idxv' = p.index (to-nat idxv) in
          mmap (v.check-decommit-on-query (case out of (fr, as', dg, fl, final) ⇒ fr) idxv') >>=
            (λfv. return (case out of (fr, as', dg, fl, final) ⇒
              (fr, as', dg, fl, final, idxv, fv))))))
    by (simp add: sm-bind-assoc split: prod.splits)
show ?thesis
  unfolding full-eq
proof (rule no-failure-bindI[OF prefix-nf])
  fix out s-prefix
  assume prefix-out: Some (out, s-prefix) ∈ set-dist (execute ?prefix-final replay-state)
  obtain fr as' dg fl final where out-eq: out = (fr, as', dg, fl, final)
  by (cases out) auto
  have prefix-out':
    Some ((fr, as', dg, fl, final), s-prefix) ∈
      set-dist (execute ?prefix-final replay-state)
  using prefix-out unfolding out-eq .
  have prefix-out-len:
    Some ((fr, as', dg, fl, final), s-prefix) ∈ set-dist (execute
      (do {
        fr ← p.read;
        as' ← mmap (replicate (length spec) ?round);
        dg ← p.read;
        assert (to-nat dg ≤ p.maxDegree);
        fl ← ntimes v.receive-fri-commits (length roots);
        final ← p.read;
        return (fr, as', dg, fl, final)
      })

```

```

    }) replay-state)
  using prefix-out' len-roots by simp
have prefix-res:
  fr = value f-merkle ∧
  as' = as ∧
  dg = of-nat (degree (p.cp as p.f-powers)) ∧
  fl = zip challenges roots ∧
  final = hd (last ls) ∧
  PState s-prefix =
    concat
      (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          (of-nat (degree (p.cp as p.f-powers))))
        roots)
      (hd (last ls)) ∧
  PTranscript s-prefix = query-decommitment-transcript idx' f-merkle @ rest ∧
  replay-state ≤ s-prefix
by (rule honest-root-alpha-degree-fri-final-prefix-outcome[
  OF htv conjunct1[OF root-alpha-tr] - replay-prefix alpha-lookups fri-lookups
  prefix-out-len])
  (use len-roots len-challenges in simp-all)
have st-prefix-final: PState s-prefix = PState s-final
  using prefix-res st-s5 st-s6 final-send-res by simp
have q-prefix-replay: PQueryCounter s-prefix = PQueryCounter replay-state
  by (rule honest-root-alpha-degree-fri-final-prefix-preserves-query-counter[
    OF prefix-out-len])
have q-final-replay: PQueryCounter s-final = PQueryCounter replay-state
  by (rule prover-final-query-counter-in-replay-state[
    OF create-f send-f alphas send-degree create-cp fri final-send replay])
have lookup-prefix:
  fmlookup (HashMap s-prefix) (QueryIndexChallenge (PQueryCounter s-prefix) (PState
s-prefix)) = Some idx
proof -
  have lookup:
    fmlookup (HashMap s-prefix) (QueryIndexChallenge (PQueryCounter s-final) (PState
s-final)) = Some idx
  using p.hash-extension-lookup[OF query-lookup-replay, of s-prefix] prefix-res
  by simp
  then show ?thesis
    using st-prefix-final q-prefix-replay q-final-replay by simp
qed
show None ∉ dom (dist (execute
  (p.receive-query-index-challenge >>=
    (λidxv.
      let idxv' = p.index (to-nat idxv)
      in mmap
        (v.check-decommit-on-query
          (case out of (fr, as', dg, fl, final) ⇒ fr) idxv') >>=
        (λfv.

```



```

      return
      (case out of (fr, as', dg, fl, final) ⇒
        (fr, as', dg, fl, final, idxv, fv)))) s-prefix))
proof (rule no-failure-bindI)
  show None ∉ dom (dist (execute p.receive-query-index-challenge s-prefix))
    by (rule p.receive-query-index-challenge-no-failure)
next
  fix idxv s-idx
  assume rand-v:
    Some (idxv, s-idx) ∈ set-dist (execute p.receive-query-index-challenge s-prefix)
  have rand-res:
    idxv = idx ∧
    PState s-idx = PState s-prefix ∧
    PTranscript s-idx = PTranscript s-prefix ∧
    s-prefix ≤ s-idx
    using p.receive-query-index-challenge-known-outcome[OF lookup-prefix rand-v]
    by simp
  have s1-s-idx: s1 ≤ s-idx
    using s1-replay prefix-res rand-res by (meson p.hash-ext-trans)
  have tr-idx:
    PTranscript s-idx =
      query-decommitment-transcript (p.index (to-nat idxv)) f-merkle @ rest
    using rand-res prefix-res idx-def by simp
  have query-nf:
    None ∉ dom (dist (execute
      (mmap (v.check-decommit-on-query fr (p.index (to-nat idxv)))) s-idx))
    by (rule honest-query-decommitment-replay-no-failure[
      OF create-f s1-s-idx - tr-idx])
      (use prefix-res in simp)
  show None ∉ dom (dist (execute
    ((let idxv' = p.index (to-nat idxv)
      in mmap
        (v.check-decommit-on-query
          (case out of (fr, as', dg, fl, final) ⇒ fr) idxv') ≫=
            (λfv.
              return
                (case out of (fr, as', dg, fl, final) ⇒
                  (fr, as', dg, fl, final, idxv, fv)))))) s-idx))
    unfolding out-eq
    by (simp add: no-failure-bind-returnI[OF query-nf])
qed
qed
qed

```

lemma honest-root-alpha-degree-fri-final-query-dynamic-from-prover-no-failure:

assumes htv: honest-trace-valid

and create-f:

Some (f-merkle, s1) ∈
 set-dist (execute (p.create p.f-eval) init-state)

```

and send-f:
  Some ( $()$ ,  $s2$ )  $\in$  set-dist (execute ( $p$ .send (value f-merkle))  $s1$ )
and alphas:
  Some ( $as$ ,  $s3$ )  $\in$ 
    set-dist (execute
      (mmap (replicate (length spec)
        (do {
           $a \leftarrow p$ .receive-alpha-challenge;
           $p$ .send  $a$ ;
          return  $a$ 
        })))  $s2$ )
and cp'-def:  $cp' = p$ .cp as  $p$ .f-powers
and send-degree:
  Some ( $()$ ,  $s4$ )  $\in$  set-dist (execute ( $p$ .send (of-nat (degree cp')))  $s3$ )
and create-cp:
  Some (cp-merkle,  $s5$ )  $\in$  set-dist (execute ( $p$ .create ( $p$ .cp-eval as))  $s4$ )
and nrounds-def:  $nrounds = \text{ceil-log} (\text{degree } cp' + 1)$ 
and fri:
  Some ( $(ps, ds, ls, ms)$ ,  $s6$ )  $\in$ 
    set-dist (execute
      ( $p$ .fri-commit  $nrounds$  [ $cp'$ ] [ $p$ .eval-domain] [ $p$ .cp-eval as] [cp-merkle])  $s5$ )
and final-send:
  Some ( $()$ ,  $s$ -final)  $\in$  set-dist (execute ( $p$ .send (hd (last ls)))  $s6$ )
and random-idx:
  Some ( $idx$ ,  $s7$ )  $\in$  set-dist (execute  $p$ .receive-query-index-challenge s-final)
and idx-def:  $idx' = p$ .index (to-nat idx)
and query-decommit:
  Some ( $qouts$ ,  $s8$ )  $\in$ 
    set-dist (execute (mmap ( $p$ .decommit-on-query idx' f-merkle))  $s7$ )
and fri-decommit:
  Some ( $fri$ - $idx$ ,  $s9$ )  $\in$ 
    set-dist (execute
      (mfold  $idx'$  ( $p$ .decommit-on-fri-layers (butlast (zip ls ms))))  $s8$ )
and prover-state-eq:  $prover\text{-}state = s9$ 
and replay:  $replay\text{-}state = \text{verifier-replay-state } prover\text{-}state$ 
shows
  None  $\notin \text{dom}$  (dist (execute
    (do {
       $fr \leftarrow p$ .read;
       $as' \leftarrow \text{mmap}$  (replicate (length spec)
        (do {
           $a0 \leftarrow p$ .receive-alpha-challenge;
          let  $a0' = a0$ ;
           $a1 \leftarrow p$ .read;
          let  $a1' = a1$ ;
          assert ( $a0' = a1'$ );
          return  $a1'$ 
        })))
       $dg \leftarrow p$ .read;

```

```

    assert (to-nat dg ≤ p.maxDegree);
    fl ← ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1));
    final ← p.read;
    idxv ← p.receive-query-index-challenge;
    let idxv' = p.index (to-nat idxv);
    fv ← mmap (v.check-decommit-on-query fr idxv');
    return (fr, as', dg, fl, final, idxv, fv)
  }) replay-state))
proof –
  let ?round =
    do {
      a0 ← p.receive-alpha-challenge;
      let a0' = a0;
      a1 ← p.read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    }
  let ?prefix-final =
    do {
      fr ← p.read;
      as' ← mmap (replicate (length spec) ?round);
      dg ← p.read;
      assert (to-nat dg ≤ p.maxDegree);
      fl ← ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1));
      final ← p.read;
      return (fr, as', dg, fl, final)
    }
  let ?full =
    do {
      fr ← p.read;
      as' ← mmap (replicate (length spec) ?round);
      dg ← p.read;
      assert (to-nat dg ≤ p.maxDegree);
      fl ← ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1));
      final ← p.read;
      idxv ← p.receive-query-index-challenge;
      let idxv' = p.index (to-nat idxv);
      fv ← mmap (v.check-decommit-on-query fr idxv');
      return (fr, as', dg, fl, final, idxv, fv)
    }
  have prefix-nf:
    None ∉ dom (dist (execute ?prefix-final replay-state))
  by (rule honest-root-alpha-degree-fri-final-dynamic-from-prover-no-failure[
    OF htv create-f send-f alphas cp'-def send-degree create-cp nrounds-def fri
    final-send random-idx idx-def query-decommit fri-decommit prover-state-eq replay])
  have root-alpha-tr:
    length as = length spec ∧
    PTranscript s4 = of-nat (degree cp') # rev as @ [value f-merkle]

```

```

    by (rule prover-root-alpha-degree-transcript[
      OF create-f send-f alphas cp'-def send-degree])
  have later-hash:  $s3 \leq \text{prover-state}$ 
  by (rule prover-later-phases-hash-extends[
    OF send-degree create-cp fri final-send random-idx query-decommit fri-decommit prover-state-eq])
  have alpha-lookups:
     $\forall i < \text{length } as.$ 
    fmllookup (HashMap replay-state)
      (AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat (concat (PState re-
play-state) (value f-merkle)) (take i as))) =
      Some (as ! i)
    by (rule prover-alpha-lookups-in-replay-state[
      OF create-f send-f alphas later-hash replay])
  from fri-commit-replay-data[OF fri] obtain roots challenges where
    len-roots:  $\text{length } roots = nrounds$ 
    and len-challenges:  $\text{length } challenges = nrounds$ 
    and tr-s6:  $PTranscript\ s6 = \text{rev } roots @ PTranscript\ s5$ 
    and st-s6:  $PState\ s6 = \text{foldl } concat\ (PState\ s5)\ roots$ 
    and fri-hash:  $s5 \leq s6$ 
    and fri-lookups-s6:
       $\forall i < \text{length } roots.$ 
      fmllookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i)
roots))) =
      Some (challenges ! i)
    by blast
  from prover-query-decommitment-transcript-in-replay-state[
    OF final-send random-idx query-decommit fri-decommit prover-state-eq replay]
  obtain rest where replay-query-tr:
    PTranscript replay-state =
      rev (PTranscript s6) @ [hd (last ls)] @
      query-decommitment-transcript idx' f-merkle @ rest
    by blast
  have tr-s5:  $PTranscript\ s5 = PTranscript\ s4$ 
    using create-preserves-transcript[OF create-cp] .
  have replay-prefix:
    PTranscript replay-state =
      value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @
      roots @ [hd (last ls)] @ query-decommitment-transcript idx' f-merkle @ rest
    using replay-query-tr tr-s6 tr-s5 root-alpha-tr cp'-def by simp
  have st-s1:  $PState\ s1 = 0$ 
    using create-preserves-state[OF create-f]
    unfolding init-state-def by simp
  have send-f-res:
     $s2 = s1 \mid$ 
    PState := concat (PState s1) (value f-merkle),
    PTranscript := value f-merkle # PTranscript s1
    using p.send-outcome[OF send-f] .
  have alpha-data:
     $s2 \leq s3 \wedge PState\ s3 = \text{foldl } concat\ (PState\ s2)\ as$ 

```

```

using alpha-mmap-outcome[OF alphas] by simp
have send-degree-res:
  s4 = s3(
    PState := concat (PState s3) (of-nat (degree cp')),
    PTranscript := of-nat (degree cp') # PTranscript s3)
using p.send-outcome[OF send-degree] .
have st-s5:
  PState s5 =
    concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
      (of-nat (degree (p.cp as p.f-powers)))
using st-s1 send-f-res alpha-data send-degree-res
  create-preserves-state[OF create-cp] replay cp'-def
unfolding verifier-replay-state-def by simp
have after-hash: s6 ≤ prover-state
by (rule prover-after-fri-hash-extends[
  OF final-send random-idx query-decommit fri-decommit prover-state-eq])
have fri-lookups:
  ∀ i < length roots.
    fmlookup (HashMap replay-state)
      (FiatShamirChallenge (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          (of-nat (degree (p.cp as p.f-powers))))))
        (take (Suc i) roots))) =
      Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have lookup-s6:
    fmlookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i)
    roots))) =
      Some (challenges ! i)
  using fri-lookups-s6 i-bound by simp
  have lookup-prover:
    fmlookup (HashMap prover-state) (FiatShamirChallenge (foldl concat (PState s5) (take
    (Suc i) roots))) =
      Some (challenges ! i)
  using p.hash-extension-lookup[OF lookup-s6 after-hash] .
  show fmlookup (HashMap replay-state)
    (FiatShamirChallenge (foldl concat
      (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
        (of-nat (degree (p.cp as p.f-powers))))))
      (take (Suc i) roots))) =
      Some (challenges ! i)
  using lookup-prover st-s5 replay
  unfolding verifier-replay-state-def by simp
qed
have final-send-res:
  s-final = s6(
    PState := concat (PState s6) (hd (last ls)),

```

```

    PTranscript := hd (last ls) # PTranscript s6)
  using p.send-outcome[OF final-send] .
  have query-lookup-replay:
    fmlookup (HashMap replay-state) (QueryIndexChallenge (PQueryCounter s-final) (PState
s-final)) = Some idx
  by (rule prover-query-random-lookup-in-replay-state[
    OF random-idx query-decommit fri-decommit prover-state-eq replay])
  have s1-s2: s1 ≤ s2
  using p.send-outcome[OF send-f]
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  have s1-s3: s1 ≤ s3
  using s1-s2 alpha-data by (meson p.hash-ext-trans)
  have s3-prover: s3 ≤ prover-state
  using later-hash .
  have s1-prover: s1 ≤ prover-state
  using s1-s3 s3-prover by (meson p.hash-ext-trans)
  have s1-replay: s1 ≤ replay-state
  using s1-prover replay
  unfolding verifier-replay-state-def less-eq-hash-ext-def less-eq-fmap-def
  by simp
  have roots-rounds:
    length roots = ceil-log (degree (p.cp as p.f-powers) + 1)
  using len-roots nrounds-def cp'-def by simp
  have full-eq:
    ?full =
      (?prefix-final >>=
        (λout. p.receive-query-index-challenge >>=
          (λidxv.
            let idxv' = p.index (to-nat idxv) in
            mmap (v.check-decommit-on-query (case out of (fr, as', dg, fl, final) ⇒ fr) idxv') >>=
              (λfv. return (case out of (fr, as', dg, fl, final) ⇒
                (fr, as', dg, fl, final, idxv, fv)))))))
      by (simp add: sm-bind-assoc split: prod.splits)
  show ?thesis
  unfolding full-eq
  proof (rule no-failure-bindI[OF prefix-nf])
    fix out s-prefix
    assume prefix-out: Some (out, s-prefix) ∈ set-dist (execute ?prefix-final replay-state)
    obtain fr as' dg fl final where out-eq: out = (fr, as', dg, fl, final)
    by (cases out) auto
    have prefix-out':
      Some ((fr, as', dg, fl, final), s-prefix) ∈
        set-dist (execute ?prefix-final replay-state)
    using prefix-out unfolding out-eq .
    have prefix-res:
      fr = value f-merkle ∧
      as' = as ∧
      dg = of-nat (degree (p.cp as p.f-powers)) ∧
      fl = zip challenges roots ∧

```

```

final = hd (last ls) ∧
PState s-prefix =
  concat
    (foldl concat
      (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
        (of-nat (degree (p.cp as p.f-powers))))
      roots)
    (hd (last ls)) ∧
PTranscript s-prefix = query-decommitment-transcript idx' f-merkle @ rest ∧
replay-state ≤ s-prefix
by (rule honest-root-alpha-degree-fri-final-dynamic-prefix-outcome[
  OF htv conjunct1[OF root-alpha-tr] - roots-rounds replay-prefix
  alpha-lookups fri-lookups prefix-out'])
  (use len-roots len-challenges in simp-all)
have st-prefix-final: PState s-prefix = PState s-final
  using prefix-res st-s5 st-s6 final-send-res by simp
have q-prefix-replay: PQueryCounter s-prefix = PQueryCounter replay-state
  by (rule honest-root-alpha-degree-fri-final-dynamic-prefix-preserves-query-counter[
    OF prefix-out'])
have q-final-replay: PQueryCounter s-final = PQueryCounter replay-state
  by (rule prover-final-query-counter-in-replay-state[
    OF create-f send-f alphas send-degree create-cp fri final-send replay])
have lookup-prefix:
  fmllookup (HashMap s-prefix) (QueryIndexChallenge (PQueryCounter s-prefix) (PState
s-prefix)) = Some idx
  proof -
    have lookup:
      fmllookup (HashMap s-prefix) (QueryIndexChallenge (PQueryCounter s-final) (PState
s-final)) = Some idx
      using p.hash-extension-lookup[OF query-lookup-replay, of s-prefix] prefix-res
      by simp
      then show ?thesis
      using st-prefix-final q-prefix-replay q-final-replay by simp
    qed
show None ∉ dom (dist (execute
  (p.receive-query-index-challenge >>=
    (λidxv.
      let idxv' = p.index (to-nat idxv)
      in mmap
        (v.check-decommit-on-query
          (case out of (fr, as', dg, fl, final) ⇒ fr) idxv') >>=
            (λfv.
              return
                (case out of (fr, as', dg, fl, final) ⇒
                  (fr, as', dg, fl, final, idxv, fv)))))) s-prefix))
proof (rule no-failure-bindI)
  show None ∉ dom (dist (execute p.receive-query-index-challenge s-prefix))
  by (rule p.receive-query-index-challenge-no-failure)
next

```

```

fix idxv s-idx
assume rand-v:
  Some (idxv, s-idx) ∈ set-dist (execute p.receive-query-index-challenge s-prefix)
have rand-res:
  idxv = idx ∧
  PState s-idx = PState s-prefix ∧
  PTranscript s-idx = PTranscript s-prefix ∧
  s-prefix ≤ s-idx
  using p.receive-query-index-challenge-known-outcome[OF lookup-prefix rand-v]
  by simp
have s1-s-idx: s1 ≤ s-idx
  using s1-replay prefix-res rand-res by (meson p.hash-ext-trans)
have tr-idx:
  PTranscript s-idx =
    query-decommitment-transcript (p.index (to-nat idxv)) f-merkle @ rest
  using rand-res prefix-res idx-def by simp
have query-nf:
  None ∉ dom (dist (execute
    (mmap (v.check-decommit-on-query fr (p.index (to-nat idxv)))) s-idx))
  by (rule honest-query-decommitment-replay-no-failure[
    OF create-f s1-s-idx - tr-idx])
    (use prefix-res in simp)
show None ∉ dom (dist (execute
  ((let idxv' = p.index (to-nat idxv)
    in mmap
      (v.check-decommit-on-query
        (case out of (fr, as', dg, fl, final) ⇒ fr) idxv') ≫=
        (λfv.
          return
            (case out of (fr, as', dg, fl, final) ⇒
              (fr, as', dg, fl, final, idxv, fv)))) s-idx))
    unfolding out-eq
    by (simp add: no-failure-bind-returnI[OF query-nf])
qed
qed
qed

```

lemma *honest-root-alpha-degree-fri-final-query-dynamic-outcome-to-fixed:*

```

assumes htv: honest-trace-valid
and create-f:
  Some (f-merkle, s1) ∈
    set-dist (execute (p.create p.f-eval) init-state)
and send-f:
  Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and alphas:
  Some (as, s3) ∈
    set-dist (execute
      (mmap (replicate (length spec)
        (do {

```



```

    a ← p.receive-alpha-challenge;
    p.send a;
    return a
  }))) s2)
and cp'-def: cp' = p.cp as p.f-powers
and send-degree:
  Some ((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp'))) s3)
and create-cp:
  Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
and nrounds-def: nrounds = ceil-log (degree cp' + 1)
and fri:
  Some ((ps, ds, ls, ms), s6) ∈
    set-dist (execute
      (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
and final-send:
  Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
and random-idx:
  Some (idx, s7) ∈ set-dist (execute p.receive-query-index-challenge s-final)
and idx-def: idx' = p.index (to-nat idx)
and query-decommit:
  Some (qouts, s8) ∈
    set-dist (execute (mmap (p.decommit-on-query idx' f-merkle)) s7)
and fri-decommit:
  Some (fri-idx, s9) ∈
    set-dist (execute
      (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
and prover-state-eq: prover-state = s9
and replay: replay-state = verifier-replay-state prover-state
and outcome:
  Some ((fr, as', dg, fl, final, idxv, fv), t) ∈
    set-dist (execute
      (do {
        fr ← p.read;
        as' ← mmap (replicate (length spec)
          (do {
            a0 ← p.receive-alpha-challenge;
            let a0' = a0;
            a1 ← p.read;
            let a1' = a1;
            assert (a0' = a1');
            return a1'
          })));
        dg ← p.read;
        assert (to-nat dg ≤ p.maxDegree);
        fl ← ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1));
        final ← p.read;
        idxv ← p.receive-query-index-challenge;
        let idxv' = p.index (to-nat idxv);
        fv ← mmap (v.check-decommit-on-query fr idxv');

```

```

    return (fr, as', dg, fl, final, idxv, fv)
  }) replay-state)
shows
  Some ((fr, as', dg, fl, final, idxv, fv), t) ∈
    set-dist (execute
      (do {
        fr ← p.read;
        as' ← mmap (replicate (length spec)
          (do {
            a0 ← p.receive-alpha-challenge;
            let a0' = a0;
            a1 ← p.read;
            let a1' = a1;
            assert (a0' = a1');
            return a1'
          })));
        dg ← p.read;
        assert (to-nat dg ≤ p.maxDegree);
        fl ← ntimes v.receive-fri-commits nrounds;
        final ← p.read;
        idxv ← p.receive-query-index-challenge;
        let idxv' = p.index (to-nat idxv);
        fv ← mmap (v.check-decommit-on-query fr idxv');
        return (fr, as', dg, fl, final, idxv, fv)
      }) replay-state)
proof —
  let ?round =
    do {
      a0 ← p.receive-alpha-challenge;
      let a0' = a0;
      a1 ← p.read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    }
  from outcome obtain r1 r2 r3 r4 r5 r6 r7 where
    read-fr: Some (fr, r1) ∈ set-dist (execute p.read replay-state)
    and alpha-out:
      Some (as', r2) ∈
        set-dist (execute (mmap (replicate (length spec) ?round)) r1)
    and read-dg: Some (dg, r3) ∈ set-dist (execute p.read r2)
    and degree-assert:
      Some ((), r4) ∈ set-dist (execute (assert (to-nat dg ≤ p.maxDegree)) r3)
    and fri-reads:
      Some (fl, r5) ∈
        set-dist (execute (ntimes v.receive-fri-commits (ceil-log (to-nat dg + 1))) r4)
    and read-final: Some (final, r6) ∈ set-dist (execute p.read r5)
    and random-v:
      Some (idxv, r7) ∈ set-dist (execute p.receive-query-index-challenge r6)

```

```

and query-reads:
  Some (fv, t) ∈
    set-dist (execute (mmap (v.check-decommit-on-query fr (p.index (to-nat idxv)))) r7)
by (auto simp: Let-def elim! p.set-dist-bindE)
let ?rad =
  do {
    fr ← p.read;
    as' ← mmap (replicate (length spec) ?round);
    dg ← p.read;
    assert (to-nat dg ≤ p.maxDegree);
    return (fr, as', dg)
  }
have rad-out:
  Some ((fr, as', dg), r4) ∈ set-dist (execute ?rad replay-state)
apply (rule set-dist-bindI[OF read-fr])
apply (rule set-dist-bindI[OF alpha-out])
apply (rule set-dist-bindI[OF read-dg])
apply (rule set-dist-bindI[OF degree-assert])
by simp
have dg-eq: dg = of-nat (degree (p.cp as p.f-powers))
using honest-root-alpha-degree-from-prover-outcome[
  OF htv create-f send-f alphas cp'-def send-degree create-cp fri
  final-send random-idx idx-def query-decommit fri-decommit prover-state-eq
  replay rad-out]
by blast
have rounds-eq: ceil-log (to-nat dg + 1) = nrounds
using honest-degree-message-rounds[OF htv, of as] dg-eq cp'-def nrounds-def
by simp
have fri-reads-fixed:
  Some (f1, r5) ∈ set-dist (execute (ntimes v.receive-fri-commits nrounds) r4)
using fri-reads rounds-eq by simp
show ?thesis
apply (rule set-dist-bindI[OF read-fr])
apply (rule set-dist-bindI[OF alpha-out])
apply (rule set-dist-bindI[OF read-dg])
apply (rule set-dist-bindI[OF degree-assert])
apply (rule set-dist-bindI[OF fri-reads-fixed])
apply (rule set-dist-bindI[OF read-final])
apply (rule set-dist-bindI[OF random-v])
apply (simp add: Let-def)
apply (rule set-dist-bindI[OF query-reads])
by simp
qed

lemma honest-root-alpha-degree-fri-final-query-from-prover-outcome:
assumes htv: honest-trace-valid
and create-f:
  Some (f-merkle, s1) ∈
    set-dist (execute (p.create p.f-eval) init-state)

```

```

and send-f:
  Some ( $()$ ,  $s2$ )  $\in$  set-dist (execute ( $p$ .send (value f-merkle))  $s1$ )
and alphas:
  Some ( $as$ ,  $s3$ )  $\in$ 
    set-dist (execute
      (mmap (replicate (length spec)
        (do {
           $a \leftarrow p$ .receive-alpha-challenge;
           $p$ .send  $a$ ;
          return  $a$ 
        })))  $s2$ )
and cp'-def:  $cp' = p$ .cp as p.f-powers
and send-degree:
  Some ( $()$ ,  $s4$ )  $\in$  set-dist (execute ( $p$ .send (of-nat (degree cp')))  $s3$ )
and create-cp:
  Some (cp-merkle,  $s5$ )  $\in$  set-dist (execute ( $p$ .create ( $p$ .cp-eval as))  $s4$ )
and fri:
  Some ( $(ps, ds, ls, ms)$ ,  $s6$ )  $\in$ 
    set-dist (execute
      ( $p$ .fri-commit nrounds [ $cp'$ ] [ $p$ .eval-domain] [ $p$ .cp-eval as] [cp-merkle])  $s5$ )
and final-send:
  Some ( $()$ ,  $s$ -final)  $\in$  set-dist (execute ( $p$ .send (hd (last ls)))  $s6$ )
and random-idx:
  Some ( $idx$ ,  $s7$ )  $\in$  set-dist (execute  $p$ .receive-query-index-challenge s-final)
and idx-def:  $idx' = p$ .index (to-nat idx)
and query-decommit:
  Some ( $qouts$ ,  $s8$ )  $\in$ 
    set-dist (execute (mmap ( $p$ .decommit-on-query idx' f-merkle))  $s7$ )
and fri-decommit:
  Some (fri-idx,  $s9$ )  $\in$ 
    set-dist (execute
      (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))))  $s8$ )
and prover-state-eq: prover-state =  $s9$ 
and replay: replay-state = verifier-replay-state prover-state
and outcome:
  Some ( $(fr, as', dg, fl, final, idxv, fv)$ ,  $t$ )  $\in$ 
    set-dist (execute
      (do {
         $fr \leftarrow p$ .read;
         $as' \leftarrow$  mmap (replicate (length spec)
          (do {
             $a0 \leftarrow p$ .receive-alpha-challenge;
            let  $a0' = a0$ ;
             $a1 \leftarrow p$ .read;
            let  $a1' = a1$ ;
            assert ( $a0' = a1'$ );
            return  $a1'$ 
          }));
         $dg \leftarrow p$ .read;

```

```

    assert (to-nat dg ≤ p.maxDegree);
    fl ← ntimes v.receive-fri-commits nrounds;
    final ← p.read;
    idxv ← p.receive-query-index-challenge;
    let idxv' = p.index (to-nat idxv);
    fv ← mmap (v.check-decommit-on-query fr idxv');
    return (fr, as', dg, fl, final, idxv, fv)
  }) replay-state)
shows
fr = value f-merkle ∧
as' = as ∧
dg = of-nat (degree (p.cp as p.f-powers)) ∧
final = hd (last ls) ∧
idxv = idx ∧
fv = map (λi. p.f-eval ! i) (p.powers-scaled idx') ∧
replay-state ≤ t ∧
(∃ roots challenges rest.
  length roots = nrounds ∧
  length challenges = nrounds ∧
  fl = zip challenges roots ∧
  (∀ j < nrounds. roots ! j = value (ms ! j)) ∧
  (∀ j < nrounds.
    p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
      (ps ! Suc j, ds ! Suc j, ls ! Suc j)) ∧
  PTranscript t = rest ∧
  PTranscript replay-state =
    value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @
      roots @ [hd (last ls)] @
        query-decommitment-transcript idx' f-merkle @ rest)
proof -
let ?round =
do {
  a0 ← p.receive-alpha-challenge;
  let a0' = a0;
  a1 ← p.read;
  let a1' = a1;
  assert (a0' = a1');
  return a1'
}
let ?prefix-final =
do {
  fr ← p.read;
  as' ← mmap (replicate (length spec) ?round);
  dg ← p.read;
  assert (to-nat dg ≤ p.maxDegree);
  fl ← ntimes v.receive-fri-commits nrounds;
  final ← p.read;
  return (fr, as', dg, fl, final)
}

```

```

let ?full =
do {
  fr ← p.read;
  as' ← mmap (replicate (length spec) ?round);
  dg ← p.read;
  assert (to-nat dg ≤ p.maxDegree);
  fl ← ntimes v.receive-fri-commits nrounds;
  final ← p.read;
  idxv ← p.receive-query-index-challenge;
  let idxv' = p.index (to-nat idxv);
  fv ← mmap (v.check-decommit-on-query fr idxv');
  return (fr, as', dg, fl, final, idxv, fv)
}
have root-alpha-tr:
length as = length spec ∧
PTranscript s4 = of-nat (degree cp') # rev as @ [value f-merkle]
by (rule prover-root-alpha-degree-transcript[
  OF create-f send-f alphas cp'-def send-degree])
have later-hash: s3 ≤ prover-state
by (rule prover-later-phases-hash-extends[
  OF send-degree create-cp fri final-send random-idx query-decommit fri-decommit prover-state-eq])
have alpha-lookups:
∀ i < length as.
fmlookup (HashMap replay-state)
(AlphaChallenge (PAlphaCounter replay-state + i) (foldl concat (concat (PState replay-state) (value f-merkle)) (take i as))) =
Some (as ! i)
by (rule prover-alpha-lookups-in-replay-state[
  OF create-f send-f alphas later-hash replay])
from fri-commit-initial-replay-successor-data[OF fri] obtain roots challenges where
len-roots: length roots = nrounds
and len-challenges: length challenges = nrounds
and tr-s6: PTranscript s6 = rev roots @ PTranscript s5
and st-s6: PState s6 = foldl concat (PState s5) roots
and fri-hash: s5 ≤ s6
and fri-lookups-s6:
∀ i < length roots.
fmlookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i) roots))) =
Some (challenges ! i)
and roots-aligned: ∀ j < nrounds. roots ! j = value (ms ! j)
and successors:
∀ j < nrounds.
p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
(ps ! Suc j, ds ! Suc j, ls ! Suc j)
by blast
from prover-query-decommitment-transcript-in-replay-state[
  OF final-send random-idx query-decommit fri-decommit prover-state-eq replay]
obtain rest where replay-query-tr:

```

```

PTranscript replay-state =
  rev (PTranscript s6) @ [hd (last ls)] @
    query-decommitment-transcript idx' f-merkle @ rest
by blast
have tr-s5: PTranscript s5 = PTranscript s4
  using create-preserves-transcript[OF create-cp] .
have replay-prefix:
  PTranscript replay-state =
    value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @
      roots @ [hd (last ls)] @ query-decommitment-transcript idx' f-merkle @ rest
  using replay-query-tr tr-s6 tr-s5 root-alpha-tr cp'-def by simp
have st-s1: PState s1 = 0
  using create-preserves-state[OF create-f]
  unfolding init-state-def by simp
have send-f-res:
  s2 = s1 (
    PState := concat (PState s1) (value f-merkle),
    PTranscript := value f-merkle # PTranscript s1)
  using p.send-outcome[OF send-f] .
have alpha-data:
  s2 ≤ s3 ∧ PState s3 = foldl concat (PState s2) as
  using alpha-mmap-outcome[OF alphas] by simp
have send-degree-res:
  s4 = s3 (
    PState := concat (PState s3) (of-nat (degree cp')),
    PTranscript := of-nat (degree cp') # PTranscript s3)
  using p.send-outcome[OF send-degree] .
have st-s5:
  PState s5 =
    concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
      (of-nat (degree (p.cp as p.f-powers)))
  using st-s1 send-f-res alpha-data send-degree-res
  create-preserves-state[OF create-cp] replay cp'-def
  unfolding verifier-replay-state-def by simp
have after-hash: s6 ≤ prover-state
  by (rule prover-after-fri-hash-extends[
    OF final-send random-idx query-decommit fri-decommit prover-state-eq])
have fri-lookups:
  ∀ i < length roots.
    fmlookup (HashMap replay-state)
      (FiatShamirChallenge (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          (of-nat (degree (p.cp as p.f-powers)))))
        (take (Suc i) roots))) =
      Some (challenges ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < length roots
  have lookup-s6:

```

```

    fmlookup (HashMap s6) (FiatShamirChallenge (foldl concat (PState s5) (take (Suc i)
roots))) =
    Some (challenges ! i)
  using fri-lookups-s6 i-bound by simp
  have lookup-prover:
    fmlookup (HashMap prover-state) (FiatShamirChallenge (foldl concat (PState s5) (take
(Suc i) roots))) =
    Some (challenges ! i)
  using p.hash-extension-lookup[OF lookup-s6 after-hash] .
  show fmlookup (HashMap replay-state)
(FiatShamirChallenge (foldl concat
(concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
(of-nat (degree (p.cp as p.f-powers))))
(take (Suc i) roots))) =
  Some (challenges ! i)
  using lookup-prover st-s5 replay
  unfolding verifier-replay-state-def by simp
qed
  have final-send-res:
    s-final = s6(|
      PState := concat (PState s6) (hd (last ls)),
      PTranscript := hd (last ls) # PTranscript s6|)
  using p.send-outcome[OF final-send] .
  have query-lookup-replay:
    fmlookup (HashMap replay-state) (QueryIndexChallenge (PQueryCounter s-final) (PState
s-final)) = Some idx
  by (rule prover-query-random-lookup-in-replay-state[
    OF random-idx query-decommit fri-decommit prover-state-eq replay])
  have s1-s2: s1 ≤ s2
  using p.send-outcome[OF send-f]
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  have s1-s3: s1 ≤ s3
  using s1-s2 alpha-data by (meson p.hash-ext-trans)
  have s3-prover: s3 ≤ prover-state
  using later-hash .
  have s1-prover: s1 ≤ prover-state
  using s1-s3 s3-prover by (meson p.hash-ext-trans)
  have s1-replay: s1 ≤ replay-state
  using s1-prover replay
  unfolding verifier-replay-state-def less-eq-hash-ext-def less-eq-fmap-def
  by simp
  have full-eq:
    ?full =
    (?prefix-final ≫=
    (λout. p.receive-query-index-challenge ≫=
    (λidxv.
    let idxv' = p.index (to-nat idxv) in
    mmap (v.check-decommit-on-query (case out of (fr, as', dg, fl, final) ⇒ fr) idxv') ≫=
    (λfv. return (case out of (fr, as', dg, fl, final) ⇒

```



```

      (fr, as', dg, fl, final, idxv, fv))))))
  by (simp add: sm-bind-assoc split: prod.splits)
have outcome-full:
  Some ((fr, as', dg, fl, final, idxv, fv), t) ∈
    set-dist (execute ?full replay-state)
  using outcome by simp
from outcome-full[unfolded full-eq] obtain out s-prefix where
  prefix-out: Some (out, s-prefix) ∈ set-dist (execute ?prefix-final replay-state)
  and after-prefix:
    Some ((fr, as', dg, fl, final, idxv, fv), t) ∈
      set-dist (execute
        (p.receive-query-index-challenge >>=
          (λidxv.
            let idxv' = p.index (to-nat idxv) in
            mmap (v.check-decommit-on-query
              (case out of (fr, as', dg, fl, final) ⇒ fr) idxv') >>=
              (λfv. return (case out of (fr, as', dg, fl, final) ⇒
                (fr, as', dg, fl, final, idxv, fv)))))) s-prefix)
proof -
  show ?thesis
    using outcome-full[unfolded full-eq]
  proof (rule p.set-dist-bindE)
    fix out s-prefix
    assume prefix-out:
      Some (out, s-prefix) ∈ set-dist (execute ?prefix-final replay-state)
    and after-prefix:
      Some ((fr, as', dg, fl, final, idxv, fv), t) ∈
        set-dist (execute
          (p.receive-query-index-challenge >>=
            (λidxv.
              let idxv' = p.index (to-nat idxv) in
              mmap (v.check-decommit-on-query
                (case out of (fr, as', dg, fl, final) ⇒ fr) idxv') >>=
                (λfv. return (case out of (fr, as', dg, fl, final) ⇒
                  (fr, as', dg, fl, final, idxv, fv)))))) s-prefix)
    show ?thesis
      by (rule that[OF prefix-out after-prefix])
  qed
qed
obtain fr0 as0 dg0 fl0 final0 where out-eq:
  out = (fr0, as0, dg0, fl0, final0)
  by (cases out) auto
have prefix-out':
  Some ((fr0, as0, dg0, fl0, final0), s-prefix) ∈
    set-dist (execute ?prefix-final replay-state)
  using prefix-out unfolding out-eq .
have prefix-out-len:
  Some ((fr0, as0, dg0, fl0, final0), s-prefix) ∈ set-dist (execute
    (do {

```

```

    fr ← p.read;
    as' ← mmap (replicate (length spec) ?round);
    dg ← p.read;
    assert (to-nat dg ≤ p.maxDegree);
    fl ← ntimes v.receive-fri-commits (length roots);
    final ← p.read;
    return (fr, as', dg, fl, final)
  }) replay-state)
using prefix-out' len-roots by simp
have prefix-res:
  fr0 = value f-merkle ∧
  as0 = as ∧
  dg0 = of-nat (degree (p.cp as p.f-powers)) ∧
  fl0 = zip challenges roots ∧
  final0 = hd (last ls) ∧
  PState s-prefix =
    concat
      (foldl concat
        (concat (foldl concat (concat (PState replay-state) (value f-merkle)) as)
          (of-nat (degree (p.cp as p.f-powers)))))
        roots)
      (hd (last ls)) ∧
  PTranscript s-prefix = query-decommitment-transcript idx' f-merkle @ rest ∧
  replay-state ≤ s-prefix
by (rule honest-root-alpha-degree-fri-final-prefix-outcome[
  OF htv conjunct1[OF root-alpha-tr] - replay-prefix alpha-lookups fri-lookups
  prefix-out-len])
  (use len-roots len-challenges in simp-all)
have st-prefix-final: PState s-prefix = PState s-final
  using prefix-res st-s5 st-s6 final-send-res by simp
have q-prefix-replay: PQueryCounter s-prefix = PQueryCounter replay-state
  by (rule honest-root-alpha-degree-fri-final-prefix-preserves-query-counter[
  OF prefix-out-len])
have q-final-replay: PQueryCounter s-final = PQueryCounter replay-state
  by (rule prover-final-query-counter-in-replay-state[
  OF create-f send-f alphas send-degree create-cp fri final-send replay])
have lookup-prefix:
  fmlookup (HashMap s-prefix) (QueryIndexChallenge (PQueryCounter s-prefix) (PState
  s-prefix)) = Some idx
proof -
  have lookup:
    fmlookup (HashMap s-prefix) (QueryIndexChallenge (PQueryCounter s-final) (PState
    s-final)) = Some idx
  using p.hash-extension-lookup[OF query-lookup-replay, of s-prefix] prefix-res
  by simp
  then show ?thesis
  using st-prefix-final q-prefix-replay q-final-replay by simp
qed
from after-prefix obtain idxv-state s-idx where

```

```

rand-v:
  Some (idxv-state, s-idx) ∈
    set-dist (execute p.receive-query-index-challenge s-prefix)
and after-rand:
  Some ((fr, as', dg, fl, final, idxv, fv), t) ∈
    set-dist (execute
      ((let idxv' = p.index (to-nat idxv-state) in
        mmap (v.check-decommit-on-query
          (case out of (fr, as', dg, fl, final) ⇒ fr) idxv') ≫=
          (λfv. return (case out of (fr, as', dg, fl, final) ⇒
            (fr, as', dg, fl, final, idxv-state, fv)))))) s-idx)
proof –
  show ?thesis
    using after-prefix
  proof (rule p.set-dist-bindE)
    fix idxv-state s-idx
    assume rand-v:
      Some (idxv-state, s-idx) ∈
        set-dist (execute p.receive-query-index-challenge s-prefix)
      and after-rand:
        Some ((fr, as', dg, fl, final, idxv, fv), t) ∈
          set-dist (execute
            ((let idxv' = p.index (to-nat idxv-state) in
              mmap (v.check-decommit-on-query
                (case out of (fr, as', dg, fl, final) ⇒ fr) idxv') ≫=
                (λfv. return (case out of (fr, as', dg, fl, final) ⇒
                  (fr, as', dg, fl, final, idxv-state, fv)))))) s-idx)
          show ?thesis
            by (rule that[OF rand-v after-rand])
    qed
  qed
have rand-res:
  idxv-state = idx ∧
    PState s-idx = PState s-prefix ∧
    PTranscript s-idx = PTranscript s-prefix ∧
    s-prefix ≤ s-idx
  using p.receive-query-index-challenge-known-outcome[OF lookup-prefix rand-v]
  by simp
have s1-s-idx: s1 ≤ s-idx
  using s1-replay prefix-res rand-res by (meson p.hash-ext-trans)
have tr-idx:
  PTranscript s-idx =
    query-decommitment-transcript (p.index (to-nat idxv-state)) f-merkle @ rest
  using rand-res prefix-res idx-def by simp
have paths:
  ∧i. i ∈ set (p.powers-scaled (p.index (to-nat idxv-state))) ⇒
    length (get-authentication-path (length p.f-eval) i f-merkle) =
      floor-log (length p.f-eval)
  using query-authentication-path-length[OF create-f] by blast

```

```

from after-rand obtain fv0 t0 where
  query-out:
    Some (fv0, t0) ∈
      set-dist (execute
        (mmap (v.check-decommit-on-query
          (case out of (fr, as', dg, fl, final) ⇒ fr)
          (p.index (to-nat idrv-state)))) s-idx)
  and ret-out:
    Some ((fr, as', dg, fl, final, idrv, fv), t) ∈
      set-dist (execute
        (return (case out of (fr, as', dg, fl, final) ⇒
          (fr, as', dg, fl, final, idrv-state, fv0))) t0)
  using rand-res by (auto simp: Let-def elim!: p.set-dist-bindE)
have query-out-replay:
  Some (fv0, t0) ∈
    set-dist (execute
      (mmap (v.check-decommit-on-query
        (case out of (fr, as', dg, fl, final) ⇒ fr)
        (p.index (to-nat idrv-state))))
      (s-idx(
        PTranscript :=
          query-decommitment-transcript (p.index (to-nat idrv-state)) f-merkle @ rest
        )))
proof −
  have s-idx-update:
    s-idx(
      PTranscript :=
        query-decommitment-transcript (p.index (to-nat idrv-state)) f-merkle @ rest
    ) = s-idx
    using tr-idx by simp
  show ?thesis
    using query-out unfolding s-idx-update .
qed
have query-res:
  fv0 = map ( $\lambda i. p.f\text{-eval } ! i$ ) (p.powers-scaled (p.index (to-nat idrv-state))) ∧
  PTranscript t0 = rest ∧
  PState t0 =
    foldl concat (PState s-idx)
      (query-decommitment-transcript (p.index (to-nat idrv-state)) f-merkle) ∧
  s-idx(
    PTranscript :=
      query-decommitment-transcript (p.index (to-nat idrv-state)) f-merkle @ rest
    ) ≤ t0
  by (rule honest-query-decommitment-replay-outcome[
    OF paths query-out-replay])
have t-res:
  (fr, as', dg, fl, final, idrv, fv) =
    (fr0, as0, dg0, fl0, final0, idrv-state, fv0) ∧
  t = t0

```

```

    using ret-out unfolding out-eq by simp
have idxv-eq: idxv = idx
    using t-res rand-res by simp
have fv-eq: fv = map (λi. p.f-eval ! i) (p.powers-scaled idx')
    using t-res query-res rand-res idx-def by simp
have replay-t: replay-state ≤ t
proof -
  have s-idx ≤ t0
  proof -
    have s-idx-update:
      s-idx(
        PTranscript :=
          query-decommitment-transcript (p.index (to-nat idxv-state)) f-merkle @ rest
      ) = s-idx
    using tr-idx by simp
  show ?thesis
    using query-res unfolding s-idx-update by simp
qed
then show ?thesis
  using prefix-res rand-res t-res by (meson p.hash-ext-trans)
qed
show ?thesis
  by (intro conjI exI[where x=roots] exI[where x=challenges] exI[where x=rest])
    (use prefix-res len-roots len-challenges query-res t-res idxv-eq fv-eq replay-t
      replay-prefix roots-aligned successors in simp-all)
qed

end

end

```

```

theory Completeness-FRI
  imports Completeness-Query
begin

```

Honest FRI decommitment, fold, degree-halving, and final-value facts.

```

context verification
begin

```

```

definition fri-layer-decommitment-transcript
  where
    fri-layer-decommitment-transcript l m i len ≡
      (let idx = i mod len;
       side = (idx + len div 2) mod len
       in l ! idx # get-authentication-path len idx m @
         l ! side # get-authentication-path len side m)

```

```

fun fri-decommitment-transcript

```

where

```
fri-decommitment-transcript idx [] = []
| fri-decommitment-transcript idx ((l, m) # fs) =
  fri-layer-decommitment-transcript l m idx (length l) @
  fri-decommitment-transcript (idx mod length l) fs
```

lemma *fri-layer-decommitment-transcript-cong*:

assumes $\text{idx mod len} = \text{idx}' \text{ mod len}$

shows

```
fri-layer-decommitment-transcript l m idx len =
  fri-layer-decommitment-transcript l m idx' len
```

using *assms*

unfolding *fri-layer-decommitment-transcript-def* **by** *simp*

lemma *fri-decommitment-transcript-cong-mod-head*:

assumes $\text{idx mod length l} = \text{idx}' \text{ mod length l}$

shows

```
fri-decommitment-transcript idx ((l, m) # fs) =
  fri-decommitment-transcript idx' ((l, m) # fs)
```

proof –

have *layer-eq*:

```
fri-layer-decommitment-transcript l m idx (length l) =
  fri-layer-decommitment-transcript l m idx' (length l)
by (rule fri-layer-decommitment-transcript-cong[OF assms])
```

have *tail-eq*:

```
fri-decommitment-transcript (idx mod length l) fs =
  fri-decommitment-transcript (idx' mod length l) fs
```

using *assms* **by** *simp*

show *?thesis*

by (*simp only: fri-decommitment-transcript.simps layer-eq tail-eq*)

qed

lemma *mod-mod-half-power*:

fixes $\text{len idx} :: \text{nat}$

assumes $\text{len} = 2 \wedge \text{Suc } n$

shows $(\text{idx mod len}) \text{ mod } (\text{len div } 2) = \text{idx mod } (\text{len div } 2)$

proof –

have *half*: $\text{len div } 2 = 2 \wedge n$

using *assms* **by** (*simp add: power-Suc*)

have *dvd-half*: $\text{len div } 2 \text{ dvd len}$

using *assms half* **by** (*simp add: power-Suc*)

show *?thesis*

using *mod-mod-cancel[of len div 2 len idx]* *dvd-half* **by** *simp*

qed

lemma *fri-decommitment-transcript-half-index-tail*:

assumes *len-pow*: $\text{len} = 2 \wedge \text{Suc } n$

and *next-len*: $\text{length l} = \text{len div } 2$

shows

$\text{fri-decommitment-transcript } (idx \text{ mod } len) ((l, m) \# fs) =$
 $\text{fri-decommitment-transcript } (idx \text{ mod } (len \text{ div } 2)) ((l, m) \# fs)$
proof –
have $idx \text{ mod } len \text{ mod length } l = idx \text{ mod } (len \text{ div } 2) \text{ mod length } l$
using $\text{mod-mod-half-power}[OF \text{ len-pow, of } idx] \text{ next-len}$ **by** simp
then show $?thesis$
by $(\text{rule fri-decommitment-transcript-cong-mod-head})$
qed

definition fri-fold-value

where

$\text{fri-fold-value } b \ l \ i \ len \ pw \equiv$
 $((l! \ i) + (l! ((i + len \text{ div } 2) \text{ mod } len))) \text{ div } 2 +$
 $b * (((l! \ i) - (l! ((i + len \text{ div } 2) \text{ mod } len))) \text{ div}$
 $(2 * ((p.h \hat{i}) * \text{shift}) \hat{pw}))$

definition $\text{fri-verifier-layer}$

where

$\text{fri-verifier-layer } b \ fr \ i \ x \ len \ pw \equiv$
 $\text{do } \{$
 $\quad xp \leftarrow p.\text{read};$
 $\quad xp' \leftarrow \text{ntimes } p.\text{read } (\text{floor-log } len);$
 $\quad xn \leftarrow p.\text{read};$
 $\quad xn' \leftarrow \text{ntimes } p.\text{read } (\text{floor-log } len);$
 $\quad \text{assert } (xp = x);$
 $\quad ap \leftarrow p.\text{check-authentication-path } len \ i \ xp \ xp';$
 $\quad \text{assert } (ap = fr);$
 $\quad \text{let } sidx = (i + len \text{ div } 2) \text{ mod } len;$
 $\quad ap \leftarrow p.\text{check-authentication-path } len \ sidx \ xn \ xn';$
 $\quad \text{assert } (ap = fr);$
 $\quad \text{let } gp = (xp + xn) \text{ div } 2;$
 $\quad \text{let } hp = (xp - xn) \text{ div } (2 * ((p.h \hat{i}) * \text{shift}) \hat{pw});$
 $\quad \text{let } x = gp + b * hp;$
 $\quad \text{return } (i \text{ mod } (len \text{ div } 2), x, len \text{ div } 2, pw + pw)$
 $\}$

lemma $\text{receive-query-commits-step-eq}$:

$m \in \text{set } (v.\text{receive-query-commits } fl) \implies$

$\exists b \ fr. (b, fr) \in \text{set } fl \wedge$

$m = (\lambda(i, x, len, pw). \text{fri-verifier-layer } b \ fr \ i \ x \ len \ pw)$

unfolding $v.\text{receive-query-commits-def}$ $\text{fri-verifier-layer-def}$

by auto

lemma $\text{receive-query-commits-Cons}$:

$v.\text{receive-query-commits } ((b, fr) \# fl) =$

$(\lambda(i, x, len, pw). \text{fri-verifier-layer } b \ fr \ i \ x \ len \ pw) \#$

$v.\text{receive-query-commits } fl$

unfolding $v.\text{receive-query-commits-def}$ $\text{fri-verifier-layer-def}$ **by** simp

lemma *receive-query-commits-nth-apply*:
assumes $j < \text{length } fl$
shows
 $((v.\text{receive-query-commits } fl) ! j) (i, x, \text{len}, pw) =$
 $\text{fri-verifier-layer } (fst (fl ! j)) (snd (fl ! j)) i x \text{len } pw$
proof –
obtain $bb \ rr$ **where** $pair: fl ! j = (bb, rr)$
by $(\text{cases } fl ! j)$
have $nth\text{-eq}$:
 $((v.\text{receive-query-commits } fl) ! j) =$
 $(\lambda(i, x, \text{len}, pw). \text{fri-verifier-layer } bb \ rr \ i \ x \ \text{len} \ pw)$
using $assms \ pair$
unfolding $v.\text{receive-query-commits-def} \ \text{fri-verifier-layer-def}$
by simp
show $?thesis$
by $(\text{subst } nth\text{-eq}, \text{simp add: } pair)$
qed

lemma *receive-query-commits-zip-nth-apply*:
assumes $j < \text{length } challenges$
and $j < \text{length } roots$
shows
 $((v.\text{receive-query-commits } (\text{zip } challenges \ roots)) ! j) (i, x, \text{len}, pw) =$
 $\text{fri-verifier-layer } (challenges ! j) (roots ! j) i x \text{len } pw$
using $assms \ \text{receive-query-commits-nth-apply} [$
 $\text{of } j \ \text{zip } challenges \ roots \ i \ x \ \text{len } pw]$
by simp

lemma *butlast-zip-suc-lengths*:
assumes $\text{length } xs = \text{Suc } n$
and $\text{length } ys = \text{Suc } n$
shows $\text{butlast } (\text{zip } xs \ ys) = \text{take } n (\text{zip } xs \ ys)$
using $assms$ **by** $(\text{simp add: butlast-conv-take})$

lemma *butlast-zip-suc-nth*:
assumes $\text{length } xs = \text{Suc } n$
and $\text{length } ys = \text{Suc } n$
and $j < n$
shows $\text{butlast } (\text{zip } xs \ ys) ! j = (xs ! j, ys ! j)$
proof –
have $\text{butlast } (\text{zip } xs \ ys) ! j = \text{zip } xs \ ys ! j$
using $\text{butlast-zip-suc-lengths} [OF \ assms(1,2)] \ assms(3)$ **by** simp
also have $\dots = (xs ! j, ys ! j)$
using $assms$ **by** $(\text{simp add: nth-zip})$
finally show $?thesis$.
qed

lemma *butlast-zip-suc-drop-nth*:
assumes $\text{length } xs = \text{Suc } n$


```

    and length ys = Suc n
    and j < n
  shows
    drop j (butlast (zip xs ys)) =
      (xs ! j, ys ! j) # drop (Suc j) (butlast (zip xs ys))
  proof -
    have bl-len: length (butlast (zip xs ys)) = n
      using butlast-zip-suc-lengths[OF assms(1,2)] assms(1,2) by simp
    have drop-eq:
      drop j (butlast (zip xs ys)) =
        butlast (zip xs ys) ! j # drop (Suc j) (butlast (zip xs ys))
      using Cons-nth-drop-Suc[OF assms(3)[folded bl-len], symmetric] .
    show ?thesis
      using drop-eq butlast-zip-suc-nth[OF assms] by simp
  qed

lemma fri-decommitment-transcript-drop-half-index:
  assumes len-ls: length ls = Suc n
    and len-ms: length ms = Suc n
    and j-bound: j < n
    and len-pow: length (ls ! j) = 2 ^ Suc N
    and next-len: length (ls ! Suc j) = length (ls ! j) div 2
  shows
    fri-decommitment-transcript (idx mod length (ls ! j))
      (drop (Suc j) (butlast (zip ls ms))) =
      fri-decommitment-transcript (idx mod (length (ls ! j) div 2))
      (drop (Suc j) (butlast (zip ls ms)))
  proof (cases Suc j < n)
    case True
      have bl-len: length (butlast (zip ls ms)) = n
        using butlast-zip-suc-lengths[OF len-ls len-ms] len-ls len-ms by simp
      have drop-eq:
        drop (Suc j) (butlast (zip ls ms)) =
          butlast (zip ls ms) ! Suc j #
            drop (Suc (Suc j)) (butlast (zip ls ms))
        using Cons-nth-drop-Suc[OF True[folded bl-len], symmetric] .
      have nth-eq:
        butlast (zip ls ms) ! Suc j = (ls ! Suc j, ms ! Suc j)
        by (rule butlast-zip-suc-nth[OF len-ls len-ms True])
      show ?thesis
        unfolding drop-eq nth-eq
        by (rule fri-decommitment-transcript-half-index-tail[
            OF len-pow next-len])
    next
      case False
        have suc-eq: Suc j = n
          using False j-bound by simp
        have bl-len: length (butlast (zip ls ms)) = n
          using butlast-zip-suc-lengths[OF len-ls len-ms] len-ls len-ms by simp

```

then show *?thesis*
using *suc-eq* **by** *simp*
qed

lemma *decommit-on-fri-layer-transcript*:

assumes *outcome*:

Some (x, t) $\in$
set-dist (*execute*
 (do {
 let $len = \text{length } l$;
 let $idx' = idx \bmod len$;
 let $sidx = (idx' + (len \div 2)) \bmod len$;
p.send ($l ! idx'$);
mfold2 *p.send* (*get-authentication-path* $len\ idx' m$);
p.send ($l ! sidx$);
mfold2 *p.send* (*get-authentication-path* $len\ sidx m$);
 return idx'
 }) *s*)

shows

$x = idx \bmod \text{length } l \wedge$
PTranscript $t =$
rev (*fri-layer-decommitment-transcript* $l\ m\ idx\ (\text{length } l)$) @ *PTranscript* s

proof –

let $?len = \text{length } l$
let $?idx = idx \bmod ?len$
let $?sidx = (?idx + (?len \div 2)) \bmod ?len$
from *outcome*[*unfolded Let-def*] **obtain** $u1\ u2\ u3\ u4$ **where**
send-xp: *Some* ($()$, $u1$) $\in$ *set-dist* (*execute* (*p.send* ($l ! ?idx$)) s)
and *path-xp*:
Some ($()$, $u2$) $\in$
set-dist (*execute* (*mfold2* *p.send* (*get-authentication-path* $?len\ ?idx\ m$)) $u1$)
and *send-xn*: *Some* ($()$, $u3$) $\in$ *set-dist* (*execute* (*p.send* ($l ! ?sidx$)) $u2$)
and *path-xn*:
Some ($()$, $u4$) $\in$
set-dist (*execute* (*mfold2* *p.send* (*get-authentication-path* $?len\ ?sidx\ m$)) $u3$)
and *ret*: *Some* (x, t) $\in$ *set-dist* (*execute* (*return* $?idx$) $u4$)
by (*auto elim!*: *p.set-dist-bindE*)
have *x-eq*: $x = ?idx$
using *ret* **by** *simp*
have *t-eq*: $t = u4$
using *ret* **by** *simp*
have *tr-u1*: *PTranscript* $u1 = l ! ?idx \# PTranscript\ s$
using *p.send-outcome*[*OF send-xp*] **by** *simp*
have *tr-u2*:
PTranscript $u2 =$
rev (*get-authentication-path* $?len\ ?idx\ m$) @ *PTranscript* $u1$
using *p.mfold2-send-outcome*[*OF path-xp*] **by** *simp*
have *tr-u3*: *PTranscript* $u3 = l ! ?sidx \# PTranscript\ u2$
using *p.send-outcome*[*OF send-xn*] **by** *simp*

```

have tr-u4:
  PTranscript u4 =
    rev (get-authentication-path ?len ?sidx m) @ PTranscript u3
  using p.mfold2-send-outcome[OF path-xn] by simp
have tr:
  PTranscript t =
    rev (fri-layer-decommitment-transcript l m idx ?len) @ PTranscript s
  using tr-u1 tr-u2 tr-u3 tr-u4 t-eq
  unfolding fri-layer-decommitment-transcript-def
  by (simp add: Let-def)
show ?thesis
  using x-eq tr by simp
qed

```

lemma *decommit-on-fri-layer-state:*

assumes *outcome:*

```

Some (x, t) ∈
  set-dist (execute
    (do {
      let len = length l;
      let idx' = idx mod len;
      let sidx = (idx' + (len div 2)) mod len;
      p.send (l ! idx');
      mfold2 p.send (get-authentication-path len idx' m);
      p.send (l ! sidx);
      mfold2 p.send (get-authentication-path len sidx m);
      return idx'
    }) s)

```

shows

```

PState t =
  foldl concat (PState s)
    (fri-layer-decommitment-transcript l m idx (length l))

```

proof –

```

let ?len = length l
let ?idx = idx mod ?len
let ?sidx = (?idx + (?len div 2)) mod ?len
from outcome[unfolded Let-def] obtain u1 u2 u3 u4 where
  send-xp: Some ((), u1) ∈ set-dist (execute (p.send (l ! ?idx)) s)
and path-xp:
  Some ((), u2) ∈
    set-dist (execute (mfold2 p.send (get-authentication-path ?len ?idx m)) u1)
and send-xn: Some ((), u3) ∈ set-dist (execute (p.send (l ! ?sidx)) u2)
and path-xn:
  Some ((), u4) ∈
    set-dist (execute (mfold2 p.send (get-authentication-path ?len ?sidx m)) u3)
and ret: Some (x, t) ∈ set-dist (execute (return ?idx) u4)
by (auto elim!: p.set-dist-bindE)
have t-eq: t = u4
using ret by simp

```

```

have st-u1: PState u1 = concat (PState s) (l ! ?idx)
  using p.send-outcome[OF send-xp] by simp
have st-u2:
  PState u2 =
    foldl concat (concat (PState s) (l ! ?idx))
      (get-authentication-path ?len ?idx m)
    using p.mfold2-send-outcome[OF path-xp] st-u1 by simp
have st-u3: PState u3 = concat (PState u2) (l ! ?sidx)
  using p.send-outcome[OF send-xn] by simp
have st-u4:
  PState u4 =
    foldl concat (concat (PState u2) (l ! ?sidx))
      (get-authentication-path ?len ?sidx m)
    using p.mfold2-send-outcome[OF path-xn] st-u3 by simp
show ?thesis
  using st-u2 st-u4 t-eq
  unfolding fri-layer-decommitment-transcript-def
  by (simp add: Let-def)
qed

```

```

lemma decommit-on-fri-layers-mfold-transcript:
  assumes outcome:
    Some (idx', t) ∈
      set-dist (execute
        (mfold idx
          (p.decommit-on-fri-layers fs)) s)
  shows
    PTranscript t =
      rev (fri-decommitment-transcript idx fs) @ PTranscript s
  using outcome
proof (induction fs arbitrary: idx idx' s t)
  case Nil
  then have t = s
    unfolding p.decommit-on-fri-layers-def by simp
  then show ?case
    by simp
next
  case (Cons pair fs)
  obtain l m where pair-eq: pair = (l, m)
    by (cases pair)
  from Cons.prems obtain idx1 u where
    head:
      Some (idx1, u) ∈
        set-dist (execute
          (( $\lambda(l, m) \text{ } idx. \text{ do } \{$ 
            let len = length l;
            let idx' = idx mod len;
            let sid = (idx' + (len div 2)) mod len;
            p.send (l ! idx');

```

```

      mfold2 p.send (get-authentication-path len idx' m);
      p.send (l ! sidx);
      mfold2 p.send (get-authentication-path len sidx m);
      return idx'
    }) pair idx) s)
and tail:
  Some (idx', t) ∈
    set-dist (execute
      (mfold idx1 (p.decommit-on-fri-layers fs)) u)
  unfolding p.decommit-on-fri-layers-def
  by (auto elim!: p.set-dist-bindE)
have head':
  Some (idx1, u) ∈
    set-dist (execute
      (do {
        let len = length l;
        let idx' = idx mod len;
        let sidx = (idx' + (len div 2)) mod len;
        p.send (l ! idx');
        mfold2 p.send (get-authentication-path len idx' m);
        p.send (l ! sidx);
        mfold2 p.send (get-authentication-path len sidx m);
        return idx'
      }) s)
  using head unfolding pair-eq by simp
have head-res:
  idx1 = idx mod length l ∧
  PTranscript u =
    rev (fri-layer-decommitment-transcript l m idx (length l)) @ PTranscript s
  by (rule decommit-on-fri-layer-transcript[OF head'])
have tail-tr:
  PTranscript t =
    rev (fri-decommitment-transcript idx1 fs) @ PTranscript u
  using Cons.IH[OF tail] .
show ?case
  using head-res tail-tr unfolding pair-eq by simp
qed

```

lemma decommit-on-fri-layers-mfold-state:

assumes outcome:

Some (idx', t) ∈
 set-dist (execute
 (mfold idx
 (p.decommit-on-fri-layers fs)) s)

shows

PState t =
 foldl concat (PState s) (fri-decommitment-transcript idx fs)

using outcome

proof (induction fs arbitrary: idx idx' s t)

```

case Nil
then show ?case
  unfolding p.decommit-on-fri-layers-def by simp
next
case (Cons pair fs)
obtain l m where pair-eq: pair = (l, m)
  by (cases pair)
from Cons.prems obtain idx1 u where
  head:
    Some (idx1, u) ∈
      set-dist (execute
        (( $\lambda(l, m) \text{ idx. do } \{$ 
          let len = length l;
          let idx' = idx mod len;
          let sidx = (idx' + (len div 2)) mod len;
          p.send (l ! idx');
          mfold2 p.send (get-authentication-path len idx' m);
          p.send (l ! sidx);
          mfold2 p.send (get-authentication-path len sidx m);
          return idx'
         $\}) \text{ pair idx} \text{ s})$ 
      and tail:
        Some (idx', t) ∈
          set-dist (execute
            (mfold idx1 (p.decommit-on-fri-layers fs) u)
          unfolding p.decommit-on-fri-layers-def
          by (auto elim!: p.set-dist-bindE)
have head':
      Some (idx1, u) ∈
        set-dist (execute
          (do {
            let len = length l;
            let idx' = idx mod len;
            let sidx = (idx' + (len div 2)) mod len;
            p.send (l ! idx');
            mfold2 p.send (get-authentication-path len idx' m);
            p.send (l ! sidx);
            mfold2 p.send (get-authentication-path len sidx m);
            return idx'
           $\}) \text{ s})$ 
        using head unfolding pair-eq by simp
have idx1-eq: idx1 = idx mod length l
        using decommit-on-fri-layer-transcript[OF head'] by simp
have head-state:
      PState u =
        foldl concat (PState s)
          (fri-layer-decommitment-transcript l m idx (length l))
        by (rule decommit-on-fri-layer-state[OF head'])
have tail-state:

```

```

PState t =
  foldl concat (PState u) (fri-decommitment-transcript idx1 fs)
  using Cons.IH[OF tail] .
show ?case
  using idx1-eq head-state tail-state unfolding pair-eq by simp
qed

lemma prover-query-round-transcript:
  assumes outcome:
    Some (x, t) ∈ set-dist (execute (prover-query-round f-merkle f-ls f-ms ls ms) s)
  obtains idx where
    PTranscript t =
      rev (fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip ls ms))) @
      rev (fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip f-ls f-ms))) @
      rev (query-decommitment-transcript (p.index (to-nat idx)) f-merkle) @
      PTranscript s
  proof (rule prover-query-round-outcomeE[OF outcome])
    fix idx s1 qouts s2 f-fri-idx s3 fri-idx
    assume
      rand: Some (idx, s1) ∈ set-dist (execute p.receive-query-index-challenge s)
    and query:
      Some (qouts, s2) ∈
        set-dist (execute
          (mmap (p.decommit-on-query (p.index (to-nat idx)) f-merkle)) s1)
    and trace-fri:
      Some (f-fri-idx, s3) ∈
        set-dist (execute
          (mfold (p.index (to-nat idx))
            (p.decommit-on-fri-layers (butlast (zip f-ls f-ms)))) s2)
    and fri:
      Some (fri-idx, t) ∈
        set-dist (execute
          (mfold (p.index (to-nat idx))
            (p.decommit-on-fri-layers (butlast (zip ls ms)))) s3)
    and unit: x = ()
    have tr-s1: PTranscript s1 = PTranscript s
    using receive-query-index-challenge-preserves-transcript[OF rand] .
    have tr-s2:
      PTranscript s2 =
        rev (query-decommitment-transcript (p.index (to-nat idx)) f-merkle) @
        PTranscript s1
    using decommit-on-query-mmap-transcript[OF query] .
    have tr-s3:
      PTranscript s3 =
        rev (fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip f-ls f-ms))) @
        PTranscript s2
    using decommit-on-fri-layers-mfold-transcript[OF trace-fri] .
    have tr-t:
      PTranscript t =

```

```

    rev (fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip ls ms))) @
    PTranscript s3
  using decommit-on-fri-layers-mfold-transcript[OF fri] .
show ?thesis
  by (rule that[of idx]) (use tr-s1 tr-s2 tr-s3 tr-t in simp)
qed

lemma prover-query-round-lookup-transcript:
  assumes outcome:
    Some (x, t) ∈ set-dist (execute (prover-query-round f-merkle f-ls f-ms ls ms) s)
  obtains idx where
    x = ()
    fmlookup (HashMap t) (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some idx
    PTranscript t =
      rev (fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip ls ms))) @
      rev (fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip f-ls f-ms))) @
      rev (query-decommitment-transcript (p.index (to-nat idx)) f-merkle) @
      PTranscript s
    s ≤ t
proof (rule prover-query-round-outcomeE[OF outcome])
  fix idx s1 qouts s2 f-fri-idx s3 fri-idx
  assume rand: Some (idx, s1) ∈ set-dist (execute p.receive-query-index-challenge s)
  and query:
    Some (qouts, s2) ∈
      set-dist (execute
        (mmap (p.decommit-on-query (p.index (to-nat idx)) f-merkle)) s1)
  and trace-fri:
    Some (f-fri-idx, s3) ∈
      set-dist (execute
        (mfold (p.index (to-nat idx))
          (p.decommit-on-fri-layers (butlast (zip f-ls f-ms)))) s2)
  and fri:
    Some (fri-idx, t) ∈
      set-dist (execute
        (mfold (p.index (to-nat idx))
          (p.decommit-on-fri-layers (butlast (zip ls ms)))) s3)
  and x-unit: x = ()
  have rand-res:
    s ≤ s1 ∧
    PState s1 = PState s ∧
    PTranscript s1 = PTranscript s ∧
    fmlookup (HashMap s1) (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some idx
  using p.receive-query-index-challenge-outcome[OF rand] by simp
  have query-ext: s1 ≤ s2
  by (rule mmap-hash-extends[OF query])
    (rule decommit-on-query-step-hash-extends)
  have trace-fri-ext: s2 ≤ s3
  by (rule mfold-hash-extends[OF trace-fri])
    (rule decommit-on-fri-layers-step-hash-extends)

```



```

have fri-ext:  $s3 \leq t$ 
  by (rule mfold-hash-extends[OF fri])
    (rule decommit-on-fri-layers-step-hash-extends)
have s-t:  $s \leq t$ 
  using rand-res query-ext trace-fri-ext fri-ext by (meson p.hash-ext-trans)
have s1-t:  $s1 \leq t$ 
  using query-ext trace-fri-ext fri-ext by (meson p.hash-ext-trans)
have lookup-s1: fmllookup (HashMap s1) (QueryIndexChallenge (PQueryCounter s) (PState
s)) = Some idx
  using rand-res by simp
have lookup-t: fmllookup (HashMap t) (QueryIndexChallenge (PQueryCounter s) (PState s))
= Some idx
  by (rule p.hash-extension-lookup[OF lookup-s1 s1-t])
have tr-s2:
  PTranscript s2 =
    rev (query-decommitment-transcript (p.index (to-nat idx)) f-merkle) @
    PTranscript s1
  by (rule decommit-on-query-mmap-transcript[OF query])
have tr-s3:
  PTranscript s3 =
    rev (fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip f-ls f-ms))) @
    PTranscript s2
  by (rule decommit-on-fri-layers-mfold-transcript[OF trace-fri])
have tr-t:
  PTranscript t =
    rev (fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip ls ms))) @
    PTranscript s3
  by (rule decommit-on-fri-layers-mfold-transcript[OF fri])
show ?thesis
  by (rule that[OF x-unit lookup-t])
    (use rand-res tr-s2 tr-s3 tr-t s-t in simp-all)
qed

```

lemma prover-query-round-lookup-state-transcript:

assumes outcome:

$\text{Some } (x, t) \in \text{set-dist } (\text{execute } (\text{prover-query-round } f\text{-merkle } f\text{-ls } f\text{-ms } ls \text{ ms}) \text{ } s)$

obtains idx **where**

$x = ()$

$\text{fmllookup } (\text{HashMap } t) (\text{QueryIndexChallenge } (\text{PQueryCounter } s) (\text{PState } s)) = \text{Some idx}$

$\text{PState } t =$

$\text{foldl concat } (\text{PState } s)$

$(\text{query-decommitment-transcript } (p.\text{index } (\text{to-nat idx})) \text{ } f\text{-merkle } @$

$\text{fri-decommitment-transcript } (p.\text{index } (\text{to-nat idx})) (\text{butlast } (\text{zip } f\text{-ls } f\text{-ms})) @$

$\text{fri-decommitment-transcript } (p.\text{index } (\text{to-nat idx})) (\text{butlast } (\text{zip } ls \text{ ms})))$

$\text{PTranscript } t =$

$\text{rev } (\text{fri-decommitment-transcript } (p.\text{index } (\text{to-nat idx})) (\text{butlast } (\text{zip } ls \text{ ms}))) @$

$\text{rev } (\text{fri-decommitment-transcript } (p.\text{index } (\text{to-nat idx})) (\text{butlast } (\text{zip } f\text{-ls } f\text{-ms}))) @$

$\text{rev } (\text{query-decommitment-transcript } (p.\text{index } (\text{to-nat idx})) \text{ } f\text{-merkle } @$

$\text{PTranscript } s$

```

 $s \leq t$ 
proof (rule prover-query-round-outcomeE[OF outcome])
  fix  $idx\ s1\ qouts\ s2\ f\text{-}fri\text{-}idx\ s3\ fri\text{-}idx$ 
  assume  $rand$ :  $Some\ (idx,\ s1) \in set\text{-}dist\ (execute\ p.receive\text{-}query\text{-}index\text{-}challenge\ s)$ 
  and  $query$ :
     $Some\ (qouts,\ s2) \in$ 
     $set\text{-}dist\ (execute$ 
       $(mmap\ (p.decommit\text{-}on\text{-}query\ (p.index\ (to\text{-}nat\ idx))\ f\text{-}merkle))\ s1)$ 
  and  $trace\text{-}fri$ :
     $Some\ (f\text{-}fri\text{-}idx,\ s3) \in$ 
     $set\text{-}dist\ (execute$ 
       $(mfold\ (p.index\ (to\text{-}nat\ idx))$ 
         $(p.decommit\text{-}on\text{-}fri\text{-}layers\ (butlast\ (zip\ f\text{-}ls\ f\text{-}ms))))\ s2)$ 
  and  $fri$ :
     $Some\ (fri\text{-}idx,\ t) \in$ 
     $set\text{-}dist\ (execute$ 
       $(mfold\ (p.index\ (to\text{-}nat\ idx))$ 
         $(p.decommit\text{-}on\text{-}fri\text{-}layers\ (butlast\ (zip\ ls\ ms))))\ s3)$ 
  and  $x\text{-}unit$ :  $x = ()$ 
have  $rand\text{-}res$ :
   $s \leq s1 \wedge$ 
   $PState\ s1 = PState\ s \wedge$ 
   $PTranscript\ s1 = PTranscript\ s \wedge$ 
   $fmlookup\ (HashMap\ s1)\ (QueryIndexChallenge\ (PQueryCounter\ s)\ (PState\ s)) = Some\ idx$ 
using  $p.receive\text{-}query\text{-}index\text{-}challenge\text{-}outcome$ [OF  $rand$ ] by  $simp$ 
have  $query\text{-}ext$ :  $s1 \leq s2$ 
by (rule  $mmap\text{-}hash\text{-}extends$ [OF  $query$ ])
  (rule  $decommit\text{-}on\text{-}query\text{-}step\text{-}hash\text{-}extends$ )
have  $trace\text{-}fri\text{-}ext$ :  $s2 \leq s3$ 
by (rule  $mfold\text{-}hash\text{-}extends$ [OF  $trace\text{-}fri$ ])
  (rule  $decommit\text{-}on\text{-}fri\text{-}layers\text{-}step\text{-}hash\text{-}extends$ )
have  $fri\text{-}ext$ :  $s3 \leq t$ 
by (rule  $mfold\text{-}hash\text{-}extends$ [OF  $fri$ ])
  (rule  $decommit\text{-}on\text{-}fri\text{-}layers\text{-}step\text{-}hash\text{-}extends$ )
have  $s\text{-}t$ :  $s \leq t$ 
using  $rand\text{-}res\ query\text{-}ext\ trace\text{-}fri\text{-}ext\ fri\text{-}ext$  by ( $meson\ p.hash\text{-}ext\text{-}trans$ )
have  $s1\text{-}t$ :  $s1 \leq t$ 
using  $query\text{-}ext\ trace\text{-}fri\text{-}ext\ fri\text{-}ext$  by ( $meson\ p.hash\text{-}ext\text{-}trans$ )
have  $lookup\text{-}s1$ :  $fmlookup\ (HashMap\ s1)\ (QueryIndexChallenge\ (PQueryCounter\ s)\ (PState\ s)) = Some\ idx$ 
using  $rand\text{-}res$  by  $simp$ 
have  $lookup\text{-}t$ :  $fmlookup\ (HashMap\ t)\ (QueryIndexChallenge\ (PQueryCounter\ s)\ (PState\ s)) = Some\ idx$ 
by (rule  $p.hash\text{-}extension\text{-}lookup$ [OF  $lookup\text{-}s1\ s1\text{-}t$ ])
have  $query\text{-}state$ :
   $PState\ s2 =$ 
   $foldl\ concat\ (PState\ s)$ 
   $(query\text{-}decommitment\text{-}transcript\ (p.index\ (to\text{-}nat\ idx))\ f\text{-}merkle)$ 
using  $decommit\text{-}on\text{-}query\text{-}mmap\text{-}state$ [OF  $query$ ]  $rand\text{-}res$  by  $simp$ 

```

```

have trace-fri-state:
  PState s3 =
    foldl concat (PState s2)
      (fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip f-ls f-ms)))
  using decommit-on-fri-layers-mfold-state[OF trace-fri] .
have fri-state:
  PState t =
    foldl concat (PState s3)
      (fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip ls ms)))
  using decommit-on-fri-layers-mfold-state[OF fri] .
have state-t:
  PState t =
    foldl concat (PState s)
      (query-decommitment-transcript (p.index (to-nat idx)) f-merkle @
        fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip f-ls f-ms)) @
        fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip ls ms)))
  using query-state trace-fri-state fri-state by simp
have tr-s2:
  PTranscript s2 =
    rev (query-decommitment-transcript (p.index (to-nat idx)) f-merkle) @
    PTranscript s1
  by (rule decommit-on-query-mmap-transcript[OF query])
have tr-s3:
  PTranscript s3 =
    rev (fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip f-ls f-ms))) @
    PTranscript s2
  by (rule decommit-on-fri-layers-mfold-transcript[OF trace-fri])
have tr-t:
  PTranscript t =
    rev (fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip ls ms))) @
    PTranscript s3
  by (rule decommit-on-fri-layers-mfold-transcript[OF fri])
show ?thesis
  by (rule that[OF x-unit lookup-t state-t])
    (use rand-res tr-s2 tr-s3 tr-t s-t in simp-all)
qed

```

```

lemma prover-fri-decommitment-transcript-in-replay-state:
assumes fri-decommit:
  Some (fri-idx, s9) ∈
    set-dist (execute
      (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
and prover-state-eq: prover-state = s9
and replay: replay-state = verifier-replay-state prover-state
shows
  PTranscript replay-state =
    rev (PTranscript s8) @
      fri-decommitment-transcript idx' (butlast (zip ls ms))
proof —

```

```

have s9-tr:
  PTranscript s9 =
    rev (fri-decommitment-transcript idx' (butlast (zip ls ms))) @
      PTranscript s8
  by (rule decommit-on-fri-layers-mfold-transcript[OF fri-decommit])
show ?thesis
  using s9-tr prover-state-eq replay
  unfolding verifier-replay-state-def by simp
qed

lemma prover-query-fri-decommitment-transcript-in-replay-state:
assumes final-send:
  Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
and random-idx:
  Some (idx, s7) ∈ set-dist (execute p.receive-query-index-challenge s-final)
and query-decommit:
  Some (qouts, s8) ∈
    set-dist (execute (mmap (p.decommit-on-query idx' f-merkle)) s7)
and fri-decommit:
  Some (fri-idx, s9) ∈
    set-dist (execute
      (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
and prover-state-eq: prover-state = s9
and replay: replay-state = verifier-replay-state prover-state
shows
  PTranscript replay-state =
    rev (PTranscript s6) @ [hd (last ls)] @
      query-decommitment-transcript idx' f-merkle @
      fri-decommitment-transcript idx' (butlast (zip ls ms))
proof –
have final-tr:
  PTranscript s-final = hd (last ls) # PTranscript s6
  using p.send-outcome[OF final-send] by simp
have s7-tr: PTranscript s7 = PTranscript s-final
  using receive-query-index-challenge-preserves-transcript[OF random-idx] .
have s8-tr:
  PTranscript s8 =
    rev (query-decommitment-transcript idx' f-merkle) @ PTranscript s7
  using decommit-on-query-mmap-transcript[OF query-decommit] .
have replay-tr:
  PTranscript replay-state =
    rev (PTranscript s8) @
      fri-decommitment-transcript idx' (butlast (zip ls ms))
  by (rule prover-fri-decommitment-transcript-in-replay-state[
    OF fri-decommit prover-state-eq replay])
show ?thesis
  using replay-tr s8-tr s7-tr final-tr by simp
qed

```

lemma *honest-query-prefix-initial-fri-value:*
assumes *htv: honest-trace-valid*
and *create-f:*
 Some (f-merkle, s1) ∈
 set-dist (execute (p.create p.f-eval) init-state)
and *send-f:*
 Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and *alphas:*
 Some (as, s3) ∈
 set-dist (execute
 (mmap (replicate (length spec)
 (do {
 a ← p.receive-alpha-challenge;
 p.send a;
 return a
 }})) s2)
and *cp'-def: cp' = p.cp as p.f-powers*
and *send-degree:*
 Some ((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp'))) s3)
and *create-cp:*
 Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
and *fri:*
 Some ((ps, ds, ls, ms), s6) ∈
 set-dist (execute
 (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
and *final-send:*
 Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
and *random-idx:*
 Some (idx, s7) ∈ set-dist (execute p.receive-query-index-challenge s-final)
and *idx-def: idx' = p.index (to-nat idx)*
and *query-decommit:*
 Some (qouts, s8) ∈
 set-dist (execute (mmap (p.decommit-on-query idx' f-merkle)) s7)
and *fri-decommit:*
 Some (fri-idx, s9) ∈
 set-dist (execute
 (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
and *prover-state-eq: prover-state = s9*
and *replay: replay-state = verifier-replay-state prover-state*
and *outcome:*
 Some ((fr, as', dg, fl, final, idxv, fv), t) ∈
 set-dist (execute
 (do {
 fr ← p.read;
 as' ← mmap (replicate (length spec)
 (do {
 a0 ← p.receive-alpha-challenge;
 let a0' = a0;
 a1 ← p.read;
 })
 })

```

    let  $a1' = a1$ ;
    assert ( $a0' = a1'$ );
    return  $a1'$ 
  }));
   $dg \leftarrow p.read$ ;
  assert ( $to\_nat\ dg \leq p.maxDegree$ );
   $fl \leftarrow ntimes\ v.receive\_fri\_commits\ nrounds$ ;
   $final \leftarrow p.read$ ;
   $idxv \leftarrow p.receive\_query\_index\_challenge$ ;
  let  $idxv' = p.index\ (to\_nat\ idxv)$ ;
   $fv \leftarrow mmap\ (v.check\_decommit\_on\_query\ fr\ idxv')$ ;
  return ( $fr, as', dg, fl, final, idxv, fv$ )
}) replay-state)

shows
   $v.cp\_eval\ as'\ fv\ (p.h \wedge idx' * shift) =$ 
   $poly\ (p.cp\ as\ p.f-powers)\ (p.h \wedge idx' * shift)$ 
proof -
  have replay-values:
     $as' = as \wedge$ 
     $idxv = idx \wedge$ 
     $fv = map\ (\lambda i. p.f\_eval\ !\ i)\ (p.powers-scaled\ idx')$ 
  using honest-root-alpha-degree-fri-final-query-from-prover-outcome[
    OF htv create-f send-f alphas cp'-def send-degree create-cp fri
    final-send random-idx idx-def query-decommit fri-decommit
    prover-state-eq replay outcome]
  by blast
  have cp:
     $v.cp\_eval\ as\ (map\ (\lambda i. p.f\_eval\ !\ i)\ (p.powers-scaled\ (p.index\ (to\_nat\ idx))))$ 
     $(p.h \wedge p.index\ (to\_nat\ idx) * shift) =$ 
     $poly\ (p.cp\ as\ p.f-powers)\ (p.h \wedge p.index\ (to\_nat\ idx) * shift)$ 
  by (rule honest-query-cp-value[OF htv])
  show ?thesis
    using replay-values idx-def cp by simp
qed

```

lemma *honest-query-prefix-leaves-fri-decommitment-transcript:*
assumes *htv: honest-trace-valid*
and *create-f:*
 Some (f-merkle, s1) ∈
 set-dist (execute (p.create p.f-eval) init-state)
and *send-f:*
 Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and *alphas:*
 Some (as, s3) ∈
 set-dist (execute
 (mmap (replicate (length spec)
 (do {
 a ← p.receive-alpha-challenge;
 p.send a;

```

    return a
  }))) s2)
and cp'-def: cp' = p.cp as p.f-powers
and send-degree:
  Some (((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp')))) s3)
and create-cp:
  Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
and fri:
  Some ((ps, ds, ls, ms), s6) ∈
    set-dist (execute
      (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
and final-send:
  Some (((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
and random-idx:
  Some (idx, s7) ∈ set-dist (execute p.receive-query-index-challenge s-final)
and idx-def: idx' = p.index (to-nat idx)
and query-decommit:
  Some (qouts, s8) ∈
    set-dist (execute (mmap (p.decommit-on-query idx' f-merkle)) s7)
and fri-decommit:
  Some (fri-idx, s9) ∈
    set-dist (execute
      (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
and prover-state-eq: prover-state = s9
and replay: replay-state = verifier-replay-state prover-state
and outcome:
  Some ((fr, as', dg, fl, final, idxv, fv), t) ∈
    set-dist (execute
      (do {
        fr ← p.read;
        as' ← mmap (replicate (length spec)
          (do {
            a0 ← p.receive-alpha-challenge;
            let a0' = a0;
            a1 ← p.read;
            let a1' = a1;
            assert (a0' = a1');
            return a1'
          })));
        dg ← p.read;
        assert (to-nat dg ≤ p.maxDegree);
        fl ← ntimes v.receive-fri-commits nrounds;
        final ← p.read;
        idxv ← p.receive-query-index-challenge;
        let idxv' = p.index (to-nat idxv);
        fv ← mmap (v.check-decommit-on-query fr idxv');
        return (fr, as', dg, fl, final, idxv, fv)
      }) replay-state)
shows PTranscript t = fri-decommitment-transcript idx' (butlast (zip ls ms))

```

proof –

from *honest-root-alpha-degree-fri-final-query-from-prover-outcome*[
OF htv create-f send-f alphas cp'-def send-degree create-cp fri
final-send random-idx idx-def query-decommit fri-decommit
prover-state-eq replay outcome]

obtain *roots challenges rest* **where**
len-roots: length roots = nrounds
and *replay-prefix*:
PTranscript replay-state =
value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @
roots @ [hd (last ls)] @
query-decommitment-transcript idx' f-merkle @ rest
and *t-tr: PTranscript t = rest*
by *blast*

have *root-alpha-tr*:
length as = length spec ∧
PTranscript s4 = of-nat (degree cp') # rev as @ [value f-merkle]
by (*rule prover-root-alpha-degree-transcript*[
OF create-f send-f alphas cp'-def send-degree])

from *fri-commit-replay-data*[*OF fri*] **obtain** *roots0 challenges0* **where**
len-roots0: length roots0 = nrounds
and *tr-s6: PTranscript s6 = rev roots0 @ PTranscript s5*
by *blast*

have *tr-s5: PTranscript s5 = PTranscript s4*
using *create-preserves-transcript*[*OF create-cp*] .

have *replay-fri-suffix*:
PTranscript replay-state =
value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))] @
roots0 @ [hd (last ls)] @
query-decommitment-transcript idx' f-merkle @
fri-decommitment-transcript idx' (butlast (zip ls ms))
using *prover-query-fri-decommitment-transcript-in-replay-state*[
OF final-send random-idx query-decommit fri-decommit prover-state-eq replay
tr-s6 tr-s5 root-alpha-tr cp'-def
by *simp*

let *?A = value f-merkle # as @ [of-nat (degree (p.cp as p.f-powers))]*
let *?B = [hd (last ls)] @ query-decommitment-transcript idx' f-merkle*
have *eq*:
?A @ roots @ ?B @ rest =
?A @ roots0 @ ?B @
fri-decommitment-transcript idx' (butlast (zip ls ms))
using *replay-prefix replay-fri-suffix* **by** *simp*

have *after-A*:
roots @ ?B @ rest =
roots0 @ ?B @ fri-decommitment-transcript idx' (butlast (zip ls ms))
using *arg-cong*[*OF eq, of λxs. drop (length ?A) xs*] **by** *simp*

have *after-roots*:
?B @ rest =
?B @ fri-decommitment-transcript idx' (butlast (zip ls ms))


```

    using arg-cong[OF after-A, of  $\lambda xs. \text{drop } n \text{ rounds } xs$ ]
    len-roots len-roots0 by simp
  have rest = fri-decommitment-transcript idx' (butlast (zip ls ms))
    using arg-cong[OF after-roots, of  $\lambda xs. \text{drop } (\text{length } ?B) xs$ ] by simp
  then show ?thesis
    using t-tr by simp
qed

lemma created-tree-get-authentication-path-len:
  assumes created: p.created-tree xs r s0
    and len: length xs =  $2^n$ 
    and i-bound:  $i < \text{length } xs$ 
  shows length (get-authentication-path (length xs) i r) = floor-log (length xs)
    using created len i-bound
proof (induction n arbitrary: xs r i)
  case 0
  then obtain x where xs-eq:  $xs = [x]$ 
    by (cases xs) auto
  then obtain h where r-eq:  $r = \langle \rangle, h, \langle \rangle$ 
    using 0.prem1 by
  by (auto simp: p.created-tree-def protocol-created-tree-def
    protocol-merkle.created-tree.simps)
  show ?case
    unfolding xs-eq r-eq by (simp add: floor-log-Suc-zero)
next
  case (Suc n)
  let ?m =  $2^n$ 
  let ?lxs = take ?m xs
  let ?rxs = drop ?m xs
  have len-xs: length xs =  $2 * ?m$ 
    using Suc.prem2 by simp
  have mid: length xs div 2 = ?m
    using len-xs by simp
  obtain a b cs where xs-cons:  $xs = a \# b \# cs$ 
    using len-xs by (cases xs; cases tl xs) auto
  have m-eq: ?m = Suc (length cs div 2)
    using mid unfolding xs-cons by simp
  have take-eq: ?lxs =  $a \# \text{take } (\text{length } cs \text{ div } 2) (b \# cs)$ 
    unfolding xs-cons m-eq by simp
  have drop-eq: ?rxs =  $\text{drop } (\text{length } cs \text{ div } 2) (b \# cs)$ 
    unfolding xs-cons m-eq by simp
  have created-raw:
    protocol-merkle.created-tree (map MerkleLeaf xs) r s0
    using Suc.prem1
    unfolding p.created-tree-def protocol-created-tree-def .
  have raw-left-eq:
    MerkleLeaf a #
      take (length cs div 2) (MerkleLeaf b # map MerkleLeaf cs) =
      map MerkleLeaf (a # take (length cs div 2) (b # cs))

```

```

  by (metis list.simps(9) take-map)
have raw-right-eq:
  drop (length cs div 2) (MerkleLeaf b # map MerkleLeaf cs) =
    map MerkleLeaf (drop (length cs div 2) (b # cs))
  by (metis list.simps(9) drop-map)
from created-raw obtain l h rr where
  r-eq: r = ⟨l, h, rr⟩
  and l-created-raw0:
    protocol-merkle.created-tree
      (MerkleLeaf a #
        take (length cs div 2) (MerkleLeaf b # map MerkleLeaf cs)) l s0
  and r-created-raw0:
    protocol-merkle.created-tree
      (drop (length cs div 2) (MerkleLeaf b # map MerkleLeaf cs)) rr s0
  unfolding xs-cons
  by (auto simp: protocol-merkle.created-tree.simps Let-def)
have l-created-raw:
  protocol-merkle.created-tree
    (map MerkleLeaf (a # take (length cs div 2) (b # cs))) l s0
  using l-created-raw0 unfolding raw-left-eq .
have r-created-raw:
  protocol-merkle.created-tree
    (map MerkleLeaf (drop (length cs div 2) (b # cs))) rr s0
  using r-created-raw0 unfolding raw-right-eq .
have l-created': p.created-tree (a # take (length cs div 2) (b # cs)) l s0
  using l-created-raw
  unfolding p.created-tree-def protocol-created-tree-def .
have r-created': p.created-tree (drop (length cs div 2) (b # cs)) rr s0
  using r-created-raw
  unfolding p.created-tree-def protocol-created-tree-def .
have l-created: p.created-tree ?lxs l s0
  using l-created' unfolding take-eq .
have r-created: p.created-tree ?rxs rr s0
  using r-created' unfolding drop-eq .
have len-l: length ?lxs = 2 ^ n
  using len-xs by simp
have len-r: length ?rxs = 2 ^ n
  using len-xs by simp
have lgth-ge: 2 ≤ length xs
  using len-xs by simp
have floor-eq: floor-log (length xs) = Suc (floor-log (length xs div 2))
  using floor-log-rec[OF lgth-ge] .
show ?case
proof (cases i < ?m)
  case True
  have i-l: i < length ?lxs
    using True len-l by simp
  have left-len:
    length (get-authentication-path (length ?lxs) i l) =

```

```

    floor-log (length ?lxs)
  by (rule Suc.IH[OF l-created len-l i-l])
have left-len':
  length (get-authentication-path ?m i l) = floor-log ?m
  using left-len len-l by simp
have length (get-authentication-path (length xs) i r) =
  Suc (floor-log ?m)
  using True left-len' unfolding r-eq
  by (simp add: mid len-xs)
then show ?thesis
  using floor-eq mid len-l by simp
next
case False
have i-ge: ?m ≤ i
  using False by simp
have i-r: i - ?m < length ?rxs
  using Suc.prem3 len-xs i-ge by simp
have right-len:
  length (get-authentication-path (length ?rxs) (i - ?m) rr) =
    floor-log (length ?rxs)
  by (rule Suc.IH[OF r-created len-r i-r])
have right-len':
  length (get-authentication-path ?m (i - ?m) rr) = floor-log ?m
  using right-len len-r by simp
have length (get-authentication-path (length xs) i r) =
  Suc (floor-log ?m)
  using False right-len' unfolding r-eq
  by (simp add: mid len-xs)
then show ?thesis
  using floor-eq mid len-r by simp
qed
qed

lemma honest-fri-layer-decommitment-no-failure:
  assumes created: p.created-tree l m s0
  and len-eq: len = length l
  and len-pow: length l = 2 ^ n
  and i-bound: i < len
  and x-eq: x = l ! i
  and ext: s0 ≤ s
  and path-i:
    length (get-authentication-path len i m) = floor-log len
  and path-s:
    length (get-authentication-path len ((i + len div 2) mod len) m) =
      floor-log len
  and root: fr = value m
shows
  None ∉ dom (dist (execute
    (do {

```

```

   $xp \leftarrow p.read;$ 
   $xp' \leftarrow ntimes\ p.read\ (floor\text{-}log\ len);$ 
   $xn \leftarrow p.read;$ 
   $xn' \leftarrow ntimes\ p.read\ (floor\text{-}log\ len);$ 
   $assert\ (xp = x);$ 
   $ap \leftarrow p.check\text{-}authentication\text{-}path\ len\ i\ xp\ xp';$ 
   $assert\ (ap = fr);$ 
   $let\ sidx = (i + len\ div\ 2)\ mod\ len;$ 
   $ap \leftarrow p.check\text{-}authentication\text{-}path\ len\ sidx\ xn\ xn';$ 
   $assert\ (ap = fr);$ 
   $let\ gp = (xp + xn)\ div\ 2;$ 
   $let\ hp = (xp - xn)\ div\ (2 * ((p.h \hat{=} i) * shift) \hat{=} pw);$ 
   $let\ x = gp + b * hp;$ 
   $return\ (i\ mod\ (len\ div\ 2),\ x,\ len\ div\ 2,\ pw + pw)$ 
})
( $s(PTranscript := fri\text{-}layer\text{-}decommitment\text{-}transcript\ l\ m\ i\ len\ @\ rest)$ )))
proof –
  let  $?sidx = (i + len\ div\ 2)\ mod\ len$ 
  let  $?path\text{-}i = get\text{-}authentication\text{-}path\ len\ i\ m$ 
  let  $?path\text{-}s = get\text{-}authentication\text{-}path\ len\ ?sidx\ m$ 
  let  $?prefix =$ 
    do {
       $xp \leftarrow p.read;$ 
       $xp' \leftarrow ntimes\ p.read\ (floor\text{-}log\ len);$ 
       $xn \leftarrow p.read;$ 
       $xn' \leftarrow ntimes\ p.read\ (floor\text{-}log\ len);$ 
       $return\ (xp,\ xp',\ xn,\ xn')$ 
    }
  let  $?start =$ 
     $s(PTranscript := l\ !\ i\ \# \ ?path\text{-}i\ @\ l\ !\ ?sidx\ \# \ ?path\text{-}s\ @\ rest)$ 
  have  $len\text{-}pos: 0 < len$ 
    using  $i\text{-}bound$  by  $simp$ 
  have  $sidx\text{-}bound: ?sidx < length\ l$ 
    using  $len\text{-}pos$  unfolding  $len\text{-}eq$  by  $simp$ 
  have  $start\text{-}eq:$ 
     $s(PTranscript := fri\text{-}layer\text{-}decommitment\text{-}transcript\ l\ m\ i\ len\ @\ rest) =$ 
     $?start$ 
    using  $i\text{-}bound$ 
    unfolding  $fri\text{-}layer\text{-}decommitment\text{-}transcript\text{-}def$ 
    by ( $simp\ add: Let\text{-}def$ )
  have  $prefix\text{-}nf: None \notin dom\ (dist\ (execute\ ?prefix\ ?start))$ 
proof –
  have  $nf\text{-}read: None \notin dom\ (dist\ (execute\ p.read\ ?start))$ 
    by ( $rule\ p.read\text{-}no\text{-}failure$ )  $simp$ 
  have  $nf\text{-}after\text{-}read:$ 
     $\bigwedge xp\ s.xp. Some\ (xp,\ s.xp) \in set\text{-}dist\ (execute\ p.read\ ?start) \implies$ 
     $None \notin dom$ 
     $(dist\ (execute$ 
       $(ntimes\ p.read\ (floor\text{-}log\ len)) \gg=$ 

```

$(\lambda xp'. p.read \gg=$
 $(\lambda xn. ntimes p.read (floor-log len) \gg=$
 $(\lambda xn'. return (xp, xp', xn, xn')))) s-xp))$

proof –

fix xp $s-xp$

assume $read-xp$: $Some (xp, s-xp) \in set-dist (execute p.read ?start)$

have $read-xp-res$:

$s-xp = s(PState := concat (PState s) (l ! i),$
 $PTranscript := ?path-i @ l ! ?sidx \# ?path-s @ rest)$

using $p.read-cons-outcome[OF read-xp]$ **by** $simp$

have $enough-path-i$: $floor-log len \leq length (PTranscript s-xp)$

using $path-i$ **unfolding** $read-xp-res$ **by** $simp$

have $nf-path-i$:

$None \notin dom (dist (execute (ntimes p.read (floor-log len)) s-xp))$
by $(rule p.ntimes-read-no-failure[OF enough-path-i])$

have $nf-after-path-i$:

$\bigwedge xp' s-path. Some (xp', s-path) \in$
 $set-dist (execute (ntimes p.read (floor-log len)) s-xp) \implies$
 $None \notin dom$
 $(dist (execute$
 $(p.read \gg=$
 $(\lambda xn. ntimes p.read (floor-log len) \gg=$
 $(\lambda xn'. return (xp, xp', xn, xn')))) s-path))$

proof –

fix xp' $s-path$

assume $read-path$:

$Some (xp', s-path) \in$
 $set-dist (execute (ntimes p.read (floor-log len)) s-xp)$

let $?s-xp0 = s(PState := concat (PState s) (l ! i))$

have $s-xp-eq$:

$s-xp = ?s-xp0(PTranscript := ?path-i @ (l ! ?sidx \# ?path-s @ rest))$

using $read-xp-res$ **by** $simp$

have $path-res$:

$s-path = ?s-xp0(PState := foldl concat (PState ?s-xp0) ?path-i,$
 $PTranscript := l ! ?sidx \# ?path-s @ rest)$

using $p.ntimes-read-prefix-outcome[$
 $OF read-path[unfolded path-i[symmetric] s-xp-eq]]$
by $simp$

have $nf-read-xn$: $None \notin dom (dist (execute p.read s-path))$
by $(rule p.read-no-failure) (simp add: path-res)$

have $nf-after-read-xn$:

$\bigwedge xn s-xn. Some (xn, s-xn) \in set-dist (execute p.read s-path) \implies$
 $None \notin dom$
 $(dist (execute$
 $(ntimes p.read (floor-log len) \gg=$
 $(\lambda xn'. return (xp, xp', xn, xn')))) s-xn))$

proof –

fix xn $s-xn$

```

assume read-xn: Some (xn, s-xn) ∈ set-dist (execute p.read s-path)
have read-xn-res:
  s-xn = s-path(
    PState := concat (PState s-path) (l ! ?sidx),
    PTranscript := ?path-s @ rest)
  using p.read-nonempty-outcome[OF read-xn] path-res
  by simp
have enough-path-s: floor-log len ≤ length (PTranscript s-xn)
  using path-s unfolding read-xn-res by simp
have nf-path-s:
  None ∉ dom (dist (execute (ntimes p.read (floor-log len)) s-xn))
  by (rule p.ntimes-read-no-failure[OF enough-path-s])
show None ∉ dom
  (dist (execute
    (ntimes p.read (floor-log len) >>=
      (λxn'. return (xp, xp', xn, xn')))) s-xn))
  by (rule no-failure-bind-returnI[OF nf-path-s])
qed
show None ∉ dom
  (dist (execute
    (p.read >>=
      (λxn. ntimes p.read (floor-log len) >>=
        (λxn'. return (xp, xp', xn, xn')))) s-path))
  using no-failure-bindI[OF nf-read-xn nf-after-read-xn] .
qed
show None ∉ dom
  (dist (execute
    (ntimes p.read (floor-log len) >>=
      (λxp'. p.read >>=
        (λxn. ntimes p.read (floor-log len) >>=
          (λxn'. return (xp, xp', xn, xn')))))) s-xp))
  using no-failure-bindI[OF nf-path-i nf-after-path-i] .
qed
show ?thesis
  using no-failure-bindI[OF nf-read nf-after-read] by simp
qed
have full-eq:
  (do {
    xp ← p.read;
    xp' ← ntimes p.read (floor-log len);
    xn ← p.read;
    xn' ← ntimes p.read (floor-log len);
    assert (xp = x);
    ap ← p.check-authentication-path len i xp xp';
    assert (ap = fr);
    let sidx = (i + len div 2) mod len;
    ap ← p.check-authentication-path len sidx xn xn';
    assert (ap = fr);
    let gp = (xp + xn) div 2;

```

```

    let hp = (xp - xn) div (2 * ((p.h ^ i) * shift) ^ pw);
    let x = gp + b * hp;
    return (i mod (len div 2), x, len div 2, pw + pw)
  }) =
  (?prefix >>= (λ(xp, xp', xn, xn').
    do {
      assert (xp = x);
      ap ← p.check-authentication-path len i xp xp';
      assert (ap = fr);
      let idx = (i + len div 2) mod len;
      ap ← p.check-authentication-path len idx xn xn';
      assert (ap = fr);
      let gp = (xp + xn) div 2;
      let hp = (xp - xn) div (2 * ((p.h ^ i) * shift) ^ pw);
      let x = gp + b * hp;
      return (i mod (len div 2), x, len div 2, pw + pw)
    })
  by (simp add: sm-bind-assoc split: prod.splits)
show ?thesis
  unfolding start-eq full-eq
proof (rule no-failure-bindI[OF prefix-nf])
  fix out t
  assume out: Some (out, t) ∈ set-dist (execute ?prefix ?start)
  obtain xp xp' xn xn' where out-eq: out = (xp, xp', xn, xn')
  by (cases out) auto
  from out[unfolded out-eq] obtain s1 s2 s3 where
    read-xp: Some (xp, s1) ∈ set-dist (execute p.read ?start)
  and read-xp-path:
    Some (xp', s2) ∈ set-dist (execute (ntimes p.read (floor-log len)) s1)
  and read-xn: Some (xn, s3) ∈ set-dist (execute p.read s2)
  and read-xn-path:
    Some (xn', t) ∈ set-dist (execute (ntimes p.read (floor-log len)) s3)
  by (auto elim!: p.set-dist-bindE)
  have read-xp-start:
    ?start = s(PTranscript := l ! i # (?path-i @ l ! ?sidx # ?path-s @ rest))
  by simp
  have read-xp':
    Some (xp, s1) ∈
      set-dist (execute p.read
        (s(PTranscript := l ! i # (?path-i @ l ! ?sidx # ?path-s @ rest))))
  by (subst read-xp-start[symmetric]) (rule read-xp)
  have xp-replay:
    xp = l ! i ∧
    s1 = s(
      PState := concat (PState s) (l ! i),
      PTranscript := ?path-i @ l ! ?sidx # ?path-s @ rest)
  by (rule p.read-cons-outcome[OF read-xp'])
  let ?s-xp = s(PState := concat (PState s) (l ! i))
  have s1-eq: s1 = ?s-xp(PTranscript := ?path-i @ l ! ?sidx # ?path-s @ rest)

```

```

using xp-replay by simp
have xp-path-replay:
   $xp' = ?path-i \wedge$ 
   $s2 = ?s-xp \langle$ 
     $PState := \text{foldl concat } (PState \ ?s-xp) \ ?path-i,$ 
     $PTranscript := l ! ?sidx \# ?path-s @ rest \rangle$ 
  using p.ntimes-read-prefix-outcome[
     $OF \text{ read-xp-path}[\text{unfolded path-i}[\text{symmetric}] \ s1-eq]$ 
  ]
  by simp
have read-xn-start:
   $s2 = (s2 \langle PTranscript := l ! ?sidx \# (?path-s @ rest) \rangle)$ 
  using xp-path-replay by simp
have read-xn':
   $Some \ (xn, s3) \in$ 
   $set-dist \ (execute \ p.read$ 
   $\ (s2 \langle PTranscript := l ! ?sidx \# (?path-s @ rest) \rangle))$ 
  by  $(subst \ read-xn-start[\text{symmetric}]) \ (rule \ read-xn)$ 
have xn-replay:
   $xn = l ! ?sidx \wedge$ 
   $s3 = s2 \langle$ 
     $PState := concat \ (PState \ s2) \ (l ! ?sidx),$ 
     $PTranscript := ?path-s @ rest \rangle$ 
  by  $(rule \ p.read-cons-outcome[OF \ read-xn'])$ 
let  $?s-xn = s2 \langle PState := concat \ (PState \ s2) \ (l ! ?sidx) \rangle$ 
have s3-eq:  $s3 = ?s-xn \langle PTranscript := ?path-s @ rest \rangle$ 
  using xn-replay by simp
have xn-path-replay:
   $xn' = ?path-s \wedge$ 
   $t = ?s-xn \langle$ 
     $PState := \text{foldl concat } (PState \ ?s-xn) \ ?path-s,$ 
     $PTranscript := rest \rangle$ 
  using p.ntimes-read-prefix-outcome[
     $OF \text{ read-xn-path}[\text{unfolded path-s}[\text{symmetric}] \ s3-eq]$ 
  ]
  by simp
have s0-t:  $s0 \leq t$ 
  using ext xp-replay xp-path-replay xn-replay xn-path-replay
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have xp-x:  $xp = x$ 
  using xp-replay x-eq by simp
let  $?after =$ 
  do {
     $assert \ (xp = x);$ 
     $ap \leftarrow p.check-authentication-path \ len \ i \ xp \ xp';$ 
     $assert \ (ap = fr);$ 
     $let \ sidx = (i + len \text{ div } 2) \bmod len;$ 
     $ap \leftarrow p.check-authentication-path \ len \ sidx \ xn \ xn';$ 
     $assert \ (ap = fr);$ 
     $let \ gp = (xp + xn) \text{ div } 2;$ 
     $let \ hp = (xp - xn) \text{ div } (2 * ((p.h \hat{i}) * shift) \hat{pw});$ 
  }

```



```

    let x = gp + b * hp;
    return (i mod (len div 2), x, len div 2, pw + pw)
  }
have after-eq:
  ?after =
    (assert (xp = x) >>= (λ-.
      p.check-authentication-path len i xp xp' >>= (λap.
        assert (ap = fr) >>= (λ-.
          p.check-authentication-path len ?sidx xn xn' >>= (λap.
            assert (ap = fr) >>= (λ-.
              let gp = (xp + xn) div 2;
              hp = (xp - xn) div (2 * ((p.h ^ i) * shift) ^ pw);
              x = gp + b * hp
              in return (i mod (len div 2), x, len div 2, pw + pw)))))))))
  by (simp add: Let-def sm-bind-assoc)
have after-nf: None ∉ dom (dist (execute ?after t))
  unfolding after-eq
proof (rule no-failure-assert-bindI[OF xp-x])
  show None ∉ dom
    (dist (execute
      (p.check-authentication-path len i xp xp' >>=
        (λapi.
          assert (api = fr) >>=
            (λ-.
              p.check-authentication-path len ?sidx xn xn' >>=
                (λap.
                  assert (ap = fr) >>=
                    (λ-.
                      let gp = (xp + xn) div 2;
                      hp = (xp - xn) div (2 * ((p.h ^ i) * shift) ^ pw);
                      x = gp + b * hp
                      in return (i mod (len div 2), x, len div 2, pw + pw))))))))) t))
proof (rule no-failure-bindI[OF p.check-authentication-path-no-failure])
fix api u
assume check-i:
  Some (api, u) ∈
    set-dist (execute (p.check-authentication-path len i xp xp') t)
have check-i-honest:
  Some (api, u) ∈
    set-dist (execute
      (p.check-authentication-path (length l) i (l ! i) ?path-i) t)
  using check-i xp-replay xp-path-replay len-eq by simp
have i-bound-l: i < length l
  using i-bound len-eq by simp
have check-i-honest-l:
  Some (api, u) ∈
    set-dist (execute
      (p.check-authentication-path (length l) i (l ! i)
        (get-authentication-path (length l) i m)) t)

```

```

    using check-i-honest len-eq by simp
have ap-i: api = value m  $\wedge$  t  $\leq$  u
    using p.check-created-tree-outcome[
      OF created len-pow i-bound-l refl s0-t check-i-honest-l]
    by simp
then have ap-fr: api = fr
    using root by simp
have tail-nf: None  $\notin$  dom
  (dist (execute
    (p.check-authentication-path len ?sidx xn xn')  $\gg$ 
    ( $\lambda$ ap.
      assert (ap = fr)  $\gg$ 
      ( $\lambda$ -.
        let gp = (xp + xn) div 2;
        hp = (xp - xn) div (2 * ((p.h  $\wedge$  i) * shift)  $\wedge$  pw);
        x = gp + b * hp
        in return (i mod (len div 2), x, len div 2, pw + pw)))) u))
proof (rule no-failure-bindI)
  show None  $\notin$  dom
    (dist (execute (p.check-authentication-path len ?sidx xn xn') u))
    by (rule p.check-authentication-path-no-failure)
  fix ap2 u2
  assume check-s:
    Some (ap2, u2)  $\in$ 
      set-dist (execute (p.check-authentication-path len ?sidx xn xn') u)
  have s0-u: s0  $\leq$  u
    using s0-t ap-i
  unfolding less-eq-hash-ext-def less-eq-fmap-def by metis
  have check-s-honest:
    Some (ap2, u2)  $\in$ 
      set-dist (execute
        (p.check-authentication-path (length l) ?sidx (l ! ?sidx) ?path-s) u)
    using check-s xn-replay xn-path-replay len-eq by simp
  have check-s-honest-l:
    Some (ap2, u2)  $\in$ 
      set-dist (execute
        (p.check-authentication-path (length l) ?sidx (l ! ?sidx)
          (get-authentication-path (length l) ?sidx m)) u)
    using check-s-honest len-eq by simp
  have ap-s: ap2 = value m
    using p.check-created-tree-outcome[
      OF created len-pow sidx-bound refl s0-u check-s-honest-l]
    by simp
  have ap-s-fr: ap2 = fr
    using ap-s root by simp
  have ap-s-fr-true: (ap2 = fr) = True
    using ap-s-fr by simp
  show None  $\notin$  dom
    (dist (execute

```

```

    (assert (ap2 = fr) >>=
      (λ-.
        let gp = (xp + xn) div 2;
        hp = (xp - xn) div (2 * ((p.h ^ i) * shift) ^ pw);
        x = gp + b * hp
        in return (i mod (len div 2), x, len div 2, pw + pw))) u2))
  using ap-s-fr by (simp add: assert-def)
qed
show None ∉ dom
  (dist (execute
    (assert (api = fr) >>=
      (λ-.
        p.check-authentication-path len ?sidx xn xn' >>=
        (λap.
          assert (ap = fr) >>=
          (λ-.
            let gp = (xp + xn) div 2;
            hp = (xp - xn) div (2 * ((p.h ^ i) * shift) ^ pw);
            x = gp + b * hp
            in return
              (i mod (len div 2),
                x,
                len div 2, pw + pw)))))) u))
    by (rule no-failure-assert-bindI[OF ap-fr tail-nf])
  )
qed
qed
show None ∉ dom
  (dist (execute
    ((case out of (xp, xp', xn, xn') ⇒
      do {
        assert (xp = x);
        ap ← p.check-authentication-path len i xp xp';
        assert (ap = fr);
        let sidx = (i + len div 2) mod len;
        ap ← p.check-authentication-path len sidx xn xn';
        assert (ap = fr);
        let gp = (xp + xn) div 2;
        let hp = (xp - xn) div (2 * ((p.h ^ i) * shift) ^ pw);
        let x = gp + b * hp;
        return (i mod (len div 2), x, len div 2, pw + pw)
      }))) t))
    unfolding out-eq
    using after-nf by simp
  )
qed
qed

```

lemma *honest-fri-layer-decommitment-outcome:*

assumes *len-eq:* len = length l

and *i-bound:* i < len

```

and path-i:
  length (get-authentication-path len i m) = floor-log len
and path-s:
  length (get-authentication-path len ((i + len div 2) mod len) m) =
    floor-log len
and outcome:
  Some (res, t') ∈ set-dist (execute
    (do {
      xp ← p.read;
      xp' ← ntimes p.read (floor-log len);
      xn ← p.read;
      xn' ← ntimes p.read (floor-log len);
      assert (xp = x);
      ap ← p.check-authentication-path len i xp xp';
      assert (ap = fr);
      let sidx = (i + len div 2) mod len;
      ap ← p.check-authentication-path len sidx xn xn';
      assert (ap = fr);
      let gp = (xp + xn) div 2;
      let hp = (xp − xn) div (2 * ((p.hi) * shift)pw);
      let x = gp + b * hp;
      return (i mod (len div 2), x, len div 2, pw + pw)
    })
    (s(PTranscript := fri-layer-decommitment-transcript l m i len @ rest)))
shows
  res =
    (i mod (len div 2),
      ((l ! i) + (l ! ((i + len div 2) mod len))) div 2 +
      b * (((l ! i) − (l ! ((i + len div 2) mod len))) div
        (2 * ((p.hi) * shift)pw),
      len div 2,
      pw + pw)
  PTranscript t' = rest
  PState t' =
    foldl concat (PState s) (fri-layer-decommitment-transcript l m i len)
proof −
  let ?sidx = (i + len div 2) mod len
  let ?path-i = get-authentication-path len i m
  let ?path-s = get-authentication-path len ?sidx m
  let ?start =
    s(PTranscript := l ! i # ?path-i @ l ! ?sidx # ?path-s @ rest)
  have len-pos: 0 < len
    using i-bound by simp
  have sidx-bound: ?sidx < length l
    using len-pos unfolding len-eq by simp
  have start-eq:
    s(PTranscript := fri-layer-decommitment-transcript l m i len @ rest) =
      ?start
  using i-bound

```

```

unfolding fri-layer-decommitment-transcript-def
by (simp add: Let-def)
from outcome[unfolded start-eq] obtain xp xp' xn xn' api aps
  s1 s2 s3 s4 s5 s6 s7 s8 where
  read-xp: Some (xp, s1) ∈ set-dist (execute p.read ?start)
and read-xp-path:
  Some (xp', s2) ∈
    set-dist (execute (ntimes p.read (floor-log len)) s1)
and read-xn: Some (xn, s3) ∈ set-dist (execute p.read s2)
and read-xn-path:
  Some (xn', s4) ∈
    set-dist (execute (ntimes p.read (floor-log len)) s3)
and assert-x: Some ((), s5) ∈ set-dist (execute (assert (xp = x)) s4)
and check-i:
  Some (api, s6) ∈
    set-dist (execute (p.check-authentication-path len i xp xp') s5)
and assert-i: Some ((), s7) ∈ set-dist (execute (assert (api = fr)) s6)
and check-s:
  Some (aps, s8) ∈
    set-dist (execute (p.check-authentication-path len ?sidx xn xn') s7)
and assert-s: Some ((), t') ∈ set-dist (execute (assert (aps = fr)) s8)
and res-eq:
  res =
    (i mod (len div 2),
      (xp + xn) div 2 + b * ((xp - xn) div (2 * ((p.h ^ i) * shift) ^ pw)),
      len div 2, pw + pw)
by (auto simp: Let-def elim!: p.set-dist-bindE)
have xp-replay:
  xp = l ! i ∧
  s1 = s⟦
    PState := concat (PState s) (l ! i),
    PTranscript := ?path-i @ l ! ?sidx # ?path-s @ rest⟧
by (rule p.read-cons-outcome[OF read-xp])
let ?s-xp = s⟦PState := concat (PState s) (l ! i)⟧
have s1-eq:
  s1 = ?s-xp⟦PTranscript := ?path-i @ l ! ?sidx # ?path-s @ rest⟧
using xp-replay by simp
have xp-path-replay:
  xp' = ?path-i ∧
  s2 = ?s-xp⟦
    PState := foldl concat (PState ?s-xp) ?path-i,
    PTranscript := l ! ?sidx # ?path-s @ rest⟧
using p.ntimes-read-prefix-outcome[
  OF read-xp-path[unfolded path-i[symmetric] s1-eq]
by simp
have read-xn':
  Some (xn, s3) ∈
    set-dist (execute p.read
      (s2⟦PTranscript := l ! ?sidx # (?path-s @ rest)⟧))

```

```

using read-xn xp-path-replay by simp
have xn-replay:
  xn = l ! ?sidx ∧
  s3 = s2(
    PState := concat (PState s2) (l ! ?sidx),
    PTranscript := ?path-s @ rest)
  by (rule p.read-cons-outcome[OF read-xn])
let ?s-xn = s2(PState := concat (PState s2) (l ! ?sidx))
have s3-eq:
  s3 = ?s-xn(PTranscript := ?path-s @ rest)
  using xn-replay by simp
have xn-path-replay:
  xn' = ?path-s ∧
  s4 = ?s-xn(
    PState := foldl concat (PState ?s-xn) ?path-s,
    PTranscript := rest)
  using p.ntimes-read-prefix-outcome[
    OF read-xn-path[unfolded path-s[symmetric] s3-eq]]
  by simp
have s5-tr: PTranscript s5 = rest
  using assert-outcomeD(2)[OF assert-x] xn-path-replay by simp
have s6-tr: PTranscript s6 = rest
  using p.check-authentication-path-preserves-channel(2)[OF check-i] s5-tr
  by simp
have s7-tr: PTranscript s7 = rest
  using assert-outcomeD(2)[OF assert-i] s6-tr by simp
have s8-tr: PTranscript s8 = rest
  using p.check-authentication-path-preserves-channel(2)[OF check-s] s7-tr
  by simp
have state-t:
  PState t' =
    foldl concat (PState s) (fri-layer-decommitment-transcript l m i len)
proof –
  have i-mod: i mod len = i
  using i-bound by simp
  have s5-state: PState s5 = PState s4
  using assert-outcomeD(2)[OF assert-x] by simp
  have s6-state: PState s6 = PState s5
  using p.check-authentication-path-preserves-channel(1)[OF check-i] .
  have s7-state: PState s7 = PState s6
  using assert-outcomeD(2)[OF assert-i] by simp
  have s8-state: PState s8 = PState s7
  using p.check-authentication-path-preserves-channel(1)[OF check-s] .
  have t-state: PState t' = PState s8
  using assert-outcomeD(2)[OF assert-s] by simp
show ?thesis
  using xp-replay xp-path-replay xn-replay xn-path-replay
    s5-state s6-state s7-state s8-state t-state i-mod
  unfolding fri-layer-decommitment-transcript-def

```

```

    by (simp add: Let-def)
qed
show
  res =
    (i mod (len div 2),
     ((l ! i) + (l ! ?sidx)) div 2 +
     b * (((l ! i) - (l ! ?sidx)) div
      (2 * ((p.h^i) * shift) ^ pw)),
     len div 2,
     pw + pw)
  using res-eq xp-replay xn-replay by simp
show PTranscript t' = rest
  using assert-outcomeD(2)[OF assert-s] s8-tr by simp
show PState t' =
  foldl concat (PState s) (fri-layer-decommitment-transcript l m i len)
  using state-t .
qed

```

lemma *fri-layer-decommitment-hash-extends:*

assumes *outcome:*

Some (res, t') ∈ set-dist (execute
(do {
xp ← p.read;
xp' ← ntimes p.read (floor-log len);
xn ← p.read;
xn' ← ntimes p.read (floor-log len);
assert (xp = x);
ap ← p.check-authentication-path len i xp xp';
assert (ap = fr);
let sidx = (i + len div 2) mod len;
ap ← p.check-authentication-path len sidx xn xn';
assert (ap = fr);
let gp = (xp + xn) div 2;
*let hp = (xp - xn) div (2 * ((p.h^i) * shift) ^ pw);*
*let x = gp + b * hp;*
return (i mod (len div 2), x, len div 2, pw + pw)
}) s)

shows $s \leq t'$

proof –

let $?sidx = (i + len \text{ div } 2) \text{ mod } len$

from *outcome* **obtain** $xp \ xp' \ xn \ xn' \ api \ aps$

$s1 \ s2 \ s3 \ s4 \ s5 \ s6 \ s7 \ s8$ **where**

read-xp: $Some (xp, s1) \in set-dist (execute \ p.read \ s)$

and *read-xp-path:*

$Some (xp', s2) \in$
 $set-dist (execute (ntimes \ p.read \ (floor-log \ len)) \ s1)$

and *read-xn:* $Some (xn, s3) \in set-dist (execute \ p.read \ s2)$

and *read-xn-path:*

$Some (xn', s4) \in$

```

    set-dist (execute (ntimes p.read (floor-log len)) s3)
  and assert-x: Some ((), s5) ∈ set-dist (execute (assert (xp = x)) s4)
  and check-i:
    Some (api, s6) ∈
      set-dist (execute (p.check-authentication-path len i xp xp') s5)
  and assert-i: Some ((), s7) ∈ set-dist (execute (assert (api = fr)) s6)
  and check-s:
    Some (aps, s8) ∈
      set-dist (execute (p.check-authentication-path len ?sidx xn xn') s7)
  and assert-s: Some ((), t') ∈ set-dist (execute (assert (aps = fr)) s8)
  by (auto simp: Let-def elim!: p.set-dist-bindE)
have s ≤ s1
  using read-xp by (rule read-hash-extends)
moreover have s1 ≤ s2
  using read-xp-path by (rule ntimes-read-hash-extends)
moreover have s2 ≤ s3
  using read-xn by (rule read-hash-extends)
moreover have s3 ≤ s4
  using read-xn-path by (rule ntimes-read-hash-extends)
moreover have s4 ≤ s5
  using assert-x by (rule assert-hash-extends)
moreover have s5 ≤ s6
  using check-i by (rule check-authentication-path-hash-extends)
moreover have s6 ≤ s7
  using assert-i by (rule assert-hash-extends)
moreover have s7 ≤ s8
  using check-s by (rule check-authentication-path-hash-extends)
moreover have s8 ≤ t'
  using assert-s by (rule assert-hash-extends)
ultimately show ?thesis
  by (meson p.hash-ext-trans)
qed

```

```

lemma honest-fri-layer-decommitment-step:
  assumes created: p.created-tree l m s0
  and len-eq: len = length l
  and len-pow: length l = 2 ^ n
  and i-bound: i < len
  and x-eq: x = l ! i
  and ext: s0 ≤ s
  and path-i:
    length (get-authentication-path len i m) = floor-log len
  and path-s:
    length (get-authentication-path len ((i + len div 2) mod len) m) =
      floor-log len
  and root: fr = value m
  and tr:
    start =
      s(PTranscript := fri-layer-decommitment-transcript l m i len @ rest)

```


shows

$None \notin dom (dist (execute (fri-verifier-layer\ b\ fr\ i\ x\ len\ pw)\ start))$
 $\wedge res\ t'. Some (res, t') \in$
 $set-dist (execute (fri-verifier-layer\ b\ fr\ i\ x\ len\ pw)\ start) \implies$
 $res =$
 $(i \bmod (len \div 2), fri-fold-value\ b\ l\ i\ len\ pw, len \div 2, pw + pw) \wedge$
 $PTranscript\ t' = rest \wedge$
 $start \leq t'$

proof –

show *nf*:

$None \notin dom (dist (execute (fri-verifier-layer\ b\ fr\ i\ x\ len\ pw)\ start))$

unfolding *fri-verifier-layer-def tr*

by (rule *honest-fri-layer-decommitment-no-failure*[
 $OF\ created\ len-eq\ len-pow\ i-bound\ x-eq\ ext\ path-i\ path-s\ root$])

fix *res t'*

assume *outcome*:

$Some (res, t') \in$
 $set-dist (execute (fri-verifier-layer\ b\ fr\ i\ x\ len\ pw)\ start)$

have *res-tr*:

$res =$
 $(i \bmod (len \div 2), fri-fold-value\ b\ l\ i\ len\ pw, len \div 2, pw + pw)$
 $PTranscript\ t' = rest$
using *honest-fri-layer-decommitment-outcome*[
 $OF\ len-eq\ i-bound\ path-i\ path-s\ outcome[unfolded\ fri-verifier-layer-def\ tr]$]

unfolding *fri-fold-value-def* **by** *simp-all*

have *ext-start*: $start \leq t'$

using *fri-layer-decommitment-hash-extends*[
 $OF\ outcome[unfolded\ fri-verifier-layer-def]$] .

show

$res =$
 $(i \bmod (len \div 2), fri-fold-value\ b\ l\ i\ len\ pw, len \div 2, pw + pw) \wedge$
 $PTranscript\ t' = rest \wedge$
 $start \leq t'$

using *res-tr ext-start* **by** *simp*

qed

lemma *honest-fri-layer-decommitment-step-created*:

assumes *created*: $p.created-tree\ l\ m\ s0$

and *len-eq*: $len = length\ l$

and *len-pow*: $length\ l = 2^n$

and *i-bound*: $i < len$

and *x-eq*: $x = l[i]$

and *ext*: $s0 \leq s$

and *root*: $fr = value\ m$

and *tr*:

$start =$
 $s(PTranscript := fri-layer-decommitment-transcript\ l\ m\ i\ len\ @\ rest)$

shows

$None \notin dom (dist (execute (fri-verifier-layer\ b\ fr\ i\ x\ len\ pw)\ start))$

$\bigwedge res\ t'. \text{Some } (res, t') \in$
 $\text{set-dist } (\text{execute } (\text{fri-verifier-layer } b\ \text{fr } i\ x\ \text{len } pw)\ \text{start}) \implies$
 $res =$
 $(i \bmod (\text{len div } 2), \text{fri-fold-value } b\ l\ i\ \text{len } pw, \text{len div } 2, pw + pw) \wedge$
 $P\text{Transcript } t' = \text{rest} \wedge$
 $\text{start} \leq t'$
proof –
have *path-i*:
 $\text{length } (\text{get-authentication-path } \text{len } i\ m) = \text{floor-log } \text{len}$
using *created-tree-get-authentication-path-len*[
 $OF\ \text{created } \text{len-pow},\ of\ i]$ *i-bound len-eq* **by** *simp*
have *len-pos*: $0 < \text{len}$
using *i-bound* **by** *simp*
have *sidx-bound*:
 $(i + \text{len div } 2) \bmod \text{len} < \text{length } l$
using *len-pos len-eq* **by** *simp*
have *path-s*:
 $\text{length } (\text{get-authentication-path } \text{len } ((i + \text{len div } 2) \bmod \text{len})\ m) =$
 $\text{floor-log } \text{len}$
using *created-tree-get-authentication-path-len*[
 $OF\ \text{created } \text{len-pow},\ of\ (i + \text{len div } 2) \bmod \text{len}]$ *sidx-bound len-eq*
by *simp*
show $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (\text{fri-verifier-layer } b\ \text{fr } i\ x\ \text{len } pw)\ \text{start}))$
by (*rule honest-fri-layer-decommitment-step(1)*[
 $OF\ \text{created } \text{len-eq } \text{len-pow } i\text{-bound } x\text{-eq } \text{ext } \text{path-i } \text{path-s } \text{root } tr]$)
fix *res t'*
assume *outcome*:
 $\text{Some } (res, t') \in$
 $\text{set-dist } (\text{execute } (\text{fri-verifier-layer } b\ \text{fr } i\ x\ \text{len } pw)\ \text{start})$
show
 $res =$
 $(i \bmod (\text{len div } 2), \text{fri-fold-value } b\ l\ i\ \text{len } pw, \text{len div } 2, pw + pw) \wedge$
 $P\text{Transcript } t' = \text{rest} \wedge$
 $\text{start} \leq t'$
by (*rule honest-fri-layer-decommitment-step(2)*[
 $OF\ \text{created } \text{len-eq } \text{len-pow } i\text{-bound } x\text{-eq } \text{ext } \text{path-i } \text{path-s } \text{root } tr\ \text{outcome}]$)
qed

lemma *receive-query-commits-step-hash-extends*:
assumes *step-in*: $m \in \text{set } (v.\text{receive-query-commits } fl)$
and *outcome*: $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (m\ a)\ s)$
shows $s \leq t$
proof –
from *receive-query-commits-step-eq*[*OF step-in*] **obtain** *b fr* **where**
 $m\text{-eq}: m = (\lambda(i, x, \text{len}, pw). \text{fri-verifier-layer } b\ \text{fr } i\ x\ \text{len } pw)$
by *blast*
obtain $i\ x\ \text{len } pw$ **where** $a\text{-eq}: a = (i, x, \text{len}, pw)$
by (*cases a*) *auto*
have *layer-out*:

Some $(y, t) \in$
 $set-dist (execute (fri-verifier-layer\ b\ fr\ i\ x\ len\ pw)\ s)$
 using outcome unfolding $m-eq\ a-eq$ by simp
 show ?thesis
 using fri-layer-decommitment-hash-extends[
 OF layer-out[unfolded fri-verifier-layer-def]] .
 qed

lemma *fri-verifier-layer-preserves-query-counter*:

assumes outcome:

 Some $(res, t) \in set-dist (execute (fri-verifier-layer\ b\ fr\ i\ x\ len\ pw)\ s)$

shows $PQueryCounter\ t = PQueryCounter\ s$

proof –

from outcome **obtain** $xp\ s1\ xp'\ s2\ xn\ s3\ xn'\ s4\ s5\ ap-i\ s6\ s7\ ap-s\ s8\ s9$ **where**

$read-xp$: Some $(xp, s1) \in set-dist (execute\ p.read\ s)$

and $read-xp-path$:

 Some $(xp', s2) \in set-dist (execute\ (ntimes\ p.read\ (floor-log\ len))\ s1)$

and $read-xn$: Some $(xn, s3) \in set-dist (execute\ p.read\ s2)$

and $read-xn-path$:

 Some $(xn', s4) \in set-dist (execute\ (ntimes\ p.read\ (floor-log\ len))\ s3)$

and $assert-x$: Some $((), s5) \in set-dist (execute\ (assert\ (xp = x))\ s4)$

and $check-i$:

 Some $(ap-i, s6) \in$

$set-dist (execute\ (p.check-authentication-path\ len\ i\ xp\ xp')\ s5)$

and $assert-i$: Some $((), s7) \in set-dist (execute\ (assert\ (ap-i = fr))\ s6)$

and $check-s$:

 Some $(ap-s, s8) \in$

$set-dist (execute\ (p.check-authentication-path\ len\ ((i + len\ div\ 2)\ mod\ len)\ xn\ xn')\ s7)$

and $assert-s$: Some $((), s9) \in set-dist (execute\ (assert\ (ap-s = fr))\ s8)$

and ret :

 Some $(res, t) \in$

$set-dist (execute$

$(return$

$(i\ mod\ (len\ div\ 2),$

$(xp + xn)\ div\ 2 + b * ((xp - xn)\ div\ (2 * ((p.h \wedge i) * shift) \wedge pw)),$

$len\ div\ 2, pw + pw))\ s9)$

unfolding *fri-verifier-layer-def*

by (auto simp: Let-def elim!: *p.set-dist-bindE*)

have $q-s1$: $PQueryCounter\ s1 = PQueryCounter\ s$

using *read-preserves-counters*[OF $read-xp$] **by** simp

have $q-s2$: $PQueryCounter\ s2 = PQueryCounter\ s1$

by (rule *ntimes-read-preserves-query-counter*[OF $read-xp-path$])

have $q-s3$: $PQueryCounter\ s3 = PQueryCounter\ s2$

using *read-preserves-counters*[OF $read-xn$] **by** simp

have $q-s4$: $PQueryCounter\ s4 = PQueryCounter\ s3$

by (rule *ntimes-read-preserves-query-counter*[OF $read-xn-path$])

have $s5-eq$: $s5 = s4$

using *assert-outcomeD*(2)[OF $assert-x$] .

have $q-s6$: $PQueryCounter\ s6 = PQueryCounter\ s5$

```

    using p.check-authentication-path-preserves-channel(6)[OF check-i] .
  have s7-eq: s7 = s6
    using assert-outcomeD(2)[OF assert-i] .
  have q-s8: PQueryCounter s8 = PQueryCounter s7
    using p.check-authentication-path-preserves-channel(6)[OF check-s] .
  have s9-eq: s9 = s8
    using assert-outcomeD(2)[OF assert-s] .
  have t-eq: t = s9
    using ret by simp
  show ?thesis
    using q-s1 q-s2 q-s3 q-s4 q-s6 q-s8
    unfolding s5-eq s7-eq s9-eq t-eq by simp
qed

```

lemma *receive-query-commits-step-preserves-query-counter*:

```

  assumes step-in:  $m \in \text{set } (v.\text{receive-query-commits fl})$ 
    and outcome:  $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (m \ a) \ s)$ 
  shows PQueryCounter t = PQueryCounter s
proof -
  from receive-query-commits-step-eq[OF step-in] obtain b fr where
    m-eq:  $m = (\lambda(i, x, \text{len}, pw). \text{fri-verifier-layer } b \text{ fr } i \ x \ \text{len} \ pw)$ 
    by blast
  obtain i x len pw where a-eq:  $a = (i, x, \text{len}, pw)$ 
    by (cases a) auto
  have layer-out:
     $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (\text{fri-verifier-layer } b \text{ fr } i \ x \ \text{len} \ pw) \ s)$ 
    using outcome unfolding m-eq a-eq by simp
  show ?thesis
    by (rule fri-verifier-layer-preserves-query-counter[OF layer-out])
qed

```

lemma *receive-query-commits-mfold-preserves-query-counter*:

```

  assumes outcome:
     $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (\text{mfold } a \ (v.\text{receive-query-commits fl})) \ s)$ 
  shows PQueryCounter t = PQueryCounter s
  by (rule mfold-preserves-query-counter[OF outcome])
    (rule receive-query-commits-step-preserves-query-counter)

```

lemma *receive-query-commits-mfold-hash-extends*:

```

  assumes outcome:
     $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (\text{mfold } a \ (v.\text{receive-query-commits fl})) \ s)$ 
  shows  $s \leq t$ 
  by (rule mfold-hash-extends[OF outcome])
    (rule receive-query-commits-step-hash-extends)

```

lemma *next-fri-domain-length*:

$\text{length } (p.\text{next-fri-domain } d) = \text{length } d \text{ div } 2$
unfolding $p.\text{next-fri-domain-def}$ **by** simp

lemma $\text{next-fri-domain-nth}$:
assumes $i < \text{length } d \text{ div } 2$
shows $p.\text{next-fri-domain } d ! i = d ! i * d ! i$
using assms **unfolding** $p.\text{next-fri-domain-def}$ **by** simp

lemma $\text{next-fri-layer-components}$:
assumes $p.\text{next-fri-layer } q \ d \ b = (q', d', l')$
shows
 $q' = p.\text{next-fri-polynomial } q \ b$
 $d' = p.\text{next-fri-domain } d$
 $l' = \text{map } (\text{poly } q') \ d'$
using assms
unfolding $p.\text{next-fri-layer-def}$
by $(\text{auto } \text{simp: Let-def split: prod.splits})$

lemma $\text{next-fri-layer-length}$:
assumes $p.\text{next-fri-layer } q \ d \ b = (q', d', l')$
shows $\text{length } l' = \text{length } d \text{ div } 2$
using $\text{next-fri-layer-components}[OF \ \text{assms}] \ \text{next-fri-domain-length}$ **by** simp

lemma $\text{next-fri-layer-domain-nth}$:
assumes $\text{layer: } p.\text{next-fri-layer } q \ d \ b = (q', d', l')$
and $i\text{-bound: } i < \text{length } d \text{ div } 2$
shows $d' ! i = d ! i * d ! i$
using $\text{next-fri-layer-components}(2)[OF \ \text{layer}] \ \text{next-fri-domain-nth}[OF \ i\text{-bound}]$
by simp

lemma $\text{fri-successor-domain-points}$:
assumes $ds0: ds ! 0 = p.\text{eval-domain}$
and successors:
 $\bigwedge j. j < n \implies$
 $p.\text{next-fri-layer } (ps ! j) \ (ds ! j) \ (\text{challenges} ! j) =$
 $(ps ! \text{Suc } j, ds ! \text{Suc } j, ls ! \text{Suc } j)$
and $j\text{-le: } j \leq n$
and $k\text{-bound: } k < \text{length } (ds ! j)$
shows $ds ! j ! k = ((p.h \wedge k) * \text{shift}) \wedge (2 \wedge j)$
using $j\text{-le } k\text{-bound}$
proof $(\text{induction } j \text{ arbitrary: } k)$
case 0
have $k\text{-lt: } k < \text{clength} * \text{scale}$
using $0.\text{prems}(2) \ ds0 \ \text{eval-domain-length}$ **by** simp
show $?case$
using $\text{eval-domain-nth}[OF \ k\text{-lt}] \ ds0$ **by** simp
next
case $(\text{Suc } j)$
have $j\text{-lt-}n: j < n$

```

    using Suc.premis(1) by simp
  have ds-suc:
    ds ! Suc j = p.next-fri-domain (ds ! j)
    using next-fri-layer-components(2)[OF successors[OF j-lt-n]]
    by simp
  have k-half: k < length (ds ! j) div 2
    using Suc.premis(2) ds-suc next-fri-domain-length by simp
  have k-prev: k < length (ds ! j)
    using k-half by linarith
  have len-pos: 0 < length (ds ! j)
    using k-prev by (cases ds ! j) auto
  have curr:
    ds ! Suc j ! k = ds ! j ! k * ds ! j ! k
    using ds-suc next-fri-domain-nth[OF k-half] by simp
  have prev:
    ds ! j ! k = ((p.h ^ k) * shift) ^ (2 ^ j)
    by (rule Suc.IH) (use Suc.premis(1) k-prev in simp-all)
  have pow-suc: (2::nat) ^ j + (2::nat) ^ j = (2::nat) ^ Suc j
  proof -
    have (2::nat) ^ Suc j = 2 * 2 ^ j
      by (simp add: power-Suc)
    also have ... = 2 ^ j + 2 ^ j
      by presburger
    finally show ?thesis
      by simp
  qed
  have next':
    ((p.h ^ k) * shift) ^ (2 ^ j) *
    ((p.h ^ k) * shift) ^ (2 ^ j) =
    ((p.h ^ k) * shift) ^ (2 ^ Suc j)
  proof -
    have ((p.h ^ k) * shift) ^ (2 ^ j) *
    ((p.h ^ k) * shift) ^ (2 ^ j) =
    ((p.h ^ k) * shift) ^ (2 ^ j + 2 ^ j)
      by (simp add: power-add[symmetric])
    also have ... = ((p.h ^ k) * shift) ^ (2 ^ Suc j)
      by (simp only: pow-suc)
    finally show ?thesis .
  qed
  show ?case
    using curr prev next' by simp
qed

lemma fri-successor-layer-lengths-power:
  assumes init-len: length (ls ! 0) = 2 ^ N
  and n-le-N: n ≤ N
  and layer-lens:  $\bigwedge j. j < n \implies \text{length } (ds ! j) = \text{length } (ls ! j)$ 
  and successors:
     $\bigwedge j. j < n \implies$ 

```

```

      p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
        (ps ! Suc j, ds ! Suc j, ls ! Suc j)
    shows j ≤ n ⇒ length (ls ! j) = 2 ^ (N - j)
  proof (induction j)
    case 0
    then show ?case
      using init-len by simp
  next
    case (Suc j)
    have j-lt-n: j < n
      using Suc.premis by simp
    have j-le-n: j ≤ n
      using Suc.premis by simp
    have len-j: length (ls ! j) = 2 ^ (N - j)
      using Suc.IH[OF j-le-n] .
    have len-next:
      length (ls ! Suc j) = length (ds ! j) div 2
      using next-fri-layer-length[OF successors[OF j-lt-n]] by simp
    have j-lt-N: j < N
      using j-lt-n n-le-N by linarith
    have diff-suc: N - j = Suc (N - Suc j)
      using j-lt-N by simp
    show ?case
      using len-next layer-lens[OF j-lt-n] len-j diff-suc by simp
  qed

```

lemma *fri-successor-layer-lengths-are-powers:*

```

  assumes init-len: length (ls ! 0) = 2 ^ N
  and n-le-N: n ≤ N
  and layer-lens: ∧j. j < n ⇒ length (ds ! j) = length (ls ! j)
  and successors:
    ∧j. j < n ⇒
      p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
        (ps ! Suc j, ds ! Suc j, ls ! Suc j)
  and j-bound: j < n
  shows ∃k. length (ls ! j) = 2 ^ k
  using fri-successor-layer-lengths-power[
    OF init-len n-le-N layer-lens successors, of j] j-bound
  by auto

```

lemma *fri-commit-initial-decommit-layer-lengths-are-powers:*

```

  assumes outcome:
    Some ((ps, ds, ls, ms), t) ∈
      set-dist (execute
        (p.fri-commit n [cp] [d] [l] [m]) s)
  and layers:
    ∧i. i < length ps ⇒ i < length ds ⇒ i < length ls ⇒
      ls ! i = map (poly (ps ! i)) (ds ! i)
  and init-len: length l = 2 ^ N

```

```

    and n-le-N:  $n \leq N$ 
    and j-bound:  $j < n$ 
  shows  $\exists k. \text{length } (ls \ ! \ j) = 2 \wedge k$ 
proof -
  have lens:
    length ps = Suc n
    length ds = Suc n
    length ls = Suc n
    length ms = Suc n
    using p.fri-commit-lengths[OF outcome] by simp-all
  have ls0:  $ls \ ! \ 0 = l$ 
proof -
  have take (length [l]) ls = [l]
    using p.fri-commit-preserves-prefix[OF outcome] by simp
  then show ?thesis
    using lens by (cases ls) auto
qed
from fri-commit-initial-replay-successor-data[OF outcome]
obtain roots challenges where
  successors:
     $\forall j < n.$ 
     $p.\text{next-fri-layer } (ps \ ! \ j) (ds \ ! \ j) (challenges \ ! \ j) =$ 
     $(ps \ ! \ \text{Suc } j, ds \ ! \ \text{Suc } j, ls \ ! \ \text{Suc } j)$ 
  by blast
have layer-lens:
   $\bigwedge j. j < n \implies \text{length } (ds \ ! \ j) = \text{length } (ls \ ! \ j)$ 
proof -
  fix j
  assume j-lt:  $j < n$ 
  have  $ls \ ! \ j = \text{map } (\text{poly } (ps \ ! \ j)) (ds \ ! \ j)$ 
    by (rule layers) (use j-lt lens in simp-all)
  then show  $\text{length } (ds \ ! \ j) = \text{length } (ls \ ! \ j)$ 
    by simp
qed
have init-len':  $\text{length } (ls \ ! \ 0) = 2 \wedge N$ 
  using ls0 init-len by simp
have successors':
   $\bigwedge j. j < n \implies$ 
   $p.\text{next-fri-layer } (ps \ ! \ j) (ds \ ! \ j) (challenges \ ! \ j) =$ 
   $(ps \ ! \ \text{Suc } j, ds \ ! \ \text{Suc } j, ls \ ! \ \text{Suc } j)$ 
  using successors by simp
show ?thesis
  by (rule fri-successor-layer-lengths-are-powers[
    where ps=ps and ds=ds and ls=ls and challenges=challenges,
    OF init-len' n-le-N layer-lens successors' j-bound])
qed

```

lemma *fri-commit-initial-layer-length-power*:
 assumes *outcome*:


```

Some ((ps, ds, ls, ms), t) ∈
  set-dist (execute
    (p.fri-commit n [cp] [d] [l] [m]) s)
and layers:
  ∧i. i < length ps ⇒ i < length ds ⇒ i < length ls ⇒
    ls ! i = map (poly (ps ! i)) (ds ! i)
and init-len: length l = 2 ^ N
and n-le-N: n ≤ N
and j-le: j ≤ n
shows length (ls ! j) = 2 ^ (N - j)
proof -
  have lens:
    length ps = Suc n
    length ds = Suc n
    length ls = Suc n
    length ms = Suc n
  using p.fri-commit-lengths[OF outcome] by simp-all
  have ls0: ls ! 0 = l
  using fri-commit-initial-heads(3)[OF outcome] .
  from fri-commit-initial-replay-successor-data[OF outcome]
  obtain roots challenges where
    successors:
      ∀j < n.
        p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
          (ps ! Suc j, ds ! Suc j, ls ! Suc j)
  by blast
  have layer-lens:
    ∧j. j < n ⇒ length (ds ! j) = length (ls ! j)
  proof -
    fix j
    assume j-lt: j < n
    have ls ! j = map (poly (ps ! j)) (ds ! j)
      by (rule layers) (use j-lt lens in simp-all)
    then show length (ds ! j) = length (ls ! j)
      by simp
  qed
  have init-len': length (ls ! 0) = 2 ^ N
  using ls0 init-len by simp
  have successors':
    ∧j. j < n ⇒
      p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
        (ps ! Suc j, ds ! Suc j, ls ! Suc j)
  using successors by simp
  show ?thesis
    by (rule fri-successor-layer-lengths-power[
      OF init-len' n-le-N layer-lens successors' j-le])
  qed

```

lemma *fri-commit-initial-domain-points:*

```

assumes outcome:
  Some ((ps, ds, ls, ms), t) ∈
    set-dist (execute
      (p.fri-commit n [cp] [p.eval-domain] [l] [m]) s)
  and j-le: j ≤ n
  and k-bound: k < length (ds ! j)
shows ds ! j ! k = ((p.h ^ k) * shift) ^ (2 ^ j)
proof –
  have ds0: ds ! 0 = p.eval-domain
    using fri-commit-initial-heads(2)[OF outcome] .
  from fri-commit-initial-replay-successor-data[OF outcome]
  obtain roots challenges where
    successors:
       $\bigwedge j. j < n \implies$ 
        p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
          (ps ! Suc j, ds ! Suc j, ls ! Suc j)
    by blast
  show ?thesis
    by (rule fri-successor-domain-points[
      OF ds0 successors j-le k-bound])
qed

lemma trace-fri-commit-initial-layer-length-power:
assumes outcome:
  Some ((ps, ds, ls, ms), t) ∈
    set-dist (execute
      (p.trace-fri-commit n [cp] [d] [l] [m]) s)
  and layers:
     $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$ 
      ls ! i = map (poly (ps ! i)) (ds ! i)
  and init-len: length l = 2 ^ N
  and n-le-N: n ≤ N
  and j-le: j ≤ n
shows length (ls ! j) = 2 ^ (N – j)
proof –
  have lens:
    length ps = Suc n
    length ds = Suc n
    length ls = Suc n
    length ms = Suc n
  using p.trace-fri-commit-lengths[OF outcome] by simp-all
  have ls0: ls ! 0 = l
    using trace-fri-commit-initial-heads(3)[OF outcome] .
  from trace-fri-commit-initial-replay-successor-data[OF outcome]
  obtain roots challenges where
    successors:
       $\forall j < n.$ 
        p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
          (ps ! Suc j, ds ! Suc j, ls ! Suc j)

```

```

    by blast
  have layer-lens:
     $\bigwedge j. j < n \implies \text{length } (ds ! j) = \text{length } (ls ! j)$ 
  proof -
    fix j
    assume j-lt:  $j < n$ 
    have  $ls ! j = \text{map } (\text{poly } (ps ! j)) (ds ! j)$ 
      by (rule layers) (use j-lt lens in simp-all)
    then show  $\text{length } (ds ! j) = \text{length } (ls ! j)$ 
      by simp
  qed
  have init-len':  $\text{length } (ls ! 0) = 2 \wedge N$ 
    using ls0 init-len by simp
  have successors':
     $\bigwedge j. j < n \implies$ 
     $p.\text{next-fri-layer } (ps ! j) (ds ! j) (\text{challenges ! } j) =$ 
     $(ps ! \text{Suc } j, ds ! \text{Suc } j, ls ! \text{Suc } j)$ 
    using successors by simp
  show ?thesis
    by (rule fri-successor-layer-lengths-power[
      OF init-len' n-le-N layer-lens successors' j-le])
  qed

lemma composition-fri-commit-initial-layer-length-power:
  assumes outcome:
    Some  $((ps, ds, ls, ms), t) \in$ 
    set-dist (execute
       $(p.\text{composition-fri-commit } n [cp] [d] [l] [m]) s$ )
  and layers:
     $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$ 
     $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$ 
  and init-len:  $\text{length } l = 2 \wedge N$ 
  and n-le-N:  $n \leq N$ 
  and j-le:  $j \leq n$ 
  shows  $\text{length } (ls ! j) = 2 \wedge (N - j)$ 
  proof -
    have lens:
       $\text{length } ps = \text{Suc } n$ 
       $\text{length } ds = \text{Suc } n$ 
       $\text{length } ls = \text{Suc } n$ 
       $\text{length } ms = \text{Suc } n$ 
    using p.composition-fri-commit-lengths[OF outcome] by simp-all
    have ls0:  $ls ! 0 = l$ 
      using composition-fri-commit-initial-heads(3)[OF outcome] .
    from composition-fri-commit-initial-replay-successor-data[OF outcome]
    obtain roots challenges where
      successors:
         $\forall j < n.$ 
         $p.\text{next-fri-layer } (ps ! j) (ds ! j) (\text{challenges ! } j) =$ 

```

```

      (ps ! Suc j, ds ! Suc j, ls ! Suc j)
    by blast
  have layer-lens:
     $\bigwedge j. j < n \implies \text{length } (ds ! j) = \text{length } (ls ! j)$ 
  proof -
    fix j
    assume j-lt:  $j < n$ 
    have ls ! j = map (poly (ps ! j)) (ds ! j)
      by (rule layers) (use j-lt lens in simp-all)
    then show  $\text{length } (ds ! j) = \text{length } (ls ! j)$ 
      by simp
  qed
  have init-len':  $\text{length } (ls ! 0) = 2 \wedge N$ 
    using ls0 init-len by simp
  have successors':
     $\bigwedge j. j < n \implies$ 
     $p.\text{next-fri-layer } (ps ! j) (ds ! j) (\text{challenges ! } j) =$ 
     $(ps ! \text{Suc } j, ds ! \text{Suc } j, ls ! \text{Suc } j)$ 
    using successors by simp
  show ?thesis
    by (rule fri-successor-layer-lengths-power[
      OF init-len' n-le-N layer-lens successors' j-le])
  qed

lemma fri-commit-outcome-initial-layer-length-power:
  assumes outcome:
    fri-commit-outcome n cp d l m ps ds ls ms s t
  and layers:
     $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$ 
     $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$ 
  and init-len:  $\text{length } l = 2 \wedge N$ 
  and n-le-N:  $n \leq N$ 
  and j-le:  $j \leq n$ 
  shows  $\text{length } (ls ! j) = 2 \wedge (N - j)$ 
  proof -
    from outcome consider
      (old) Some ((ps, ds, ls, ms), t)  $\in$ 
        set-dist (execute (p.fri-commit n [cp] [d] [l] [m]) s)
    | (trace) Some ((ps, ds, ls, ms), t)  $\in$ 
        set-dist (execute (p.trace-fri-commit n [cp] [d] [l] [m]) s)
    | (composition) Some ((ps, ds, ls, ms), t)  $\in$ 
        set-dist (execute (p.composition-fri-commit n [cp] [d] [l] [m]) s)
    unfolding fri-commit-outcome-def by blast
  then show ?thesis
  proof cases
    case old
    show ?thesis
      by (rule fri-commit-initial-layer-length-power[
        OF old layers init-len n-le-N j-le])
  end

```

```

next
  case trace
  show ?thesis
    by (rule trace-fri-commit-initial-layer-length-power[
      OF trace layers init-len n-le-N j-le])
next
  case composition
  show ?thesis
    by (rule composition-fri-commit-initial-layer-length-power[
      OF composition layers init-len n-le-N j-le])
qed
qed

```

lemma *trace-fri-commit-initial-domain-points:*

```

assumes outcome:
  Some ((ps, ds, ls, ms), t) ∈
    set-dist (execute
      (p.trace-fri-commit n [cp] [p.eval-domain] [l] [m]) s)
  and j-le: j ≤ n
  and k-bound: k < length (ds ! j)
shows ds ! j ! k = ((p.h ^ k) * shift) ^ (2 ^ j)
proof –
  have ds0: ds ! 0 = p.eval-domain
    using trace-fri-commit-initial-heads(2)[OF outcome] .
  from trace-fri-commit-initial-replay-successor-data[OF outcome]
  obtain roots challenges where
    successors:
       $\bigwedge j. j < n \implies$ 
       $p.\text{next-fri-layer } (ps ! j) (ds ! j) (challenges ! j) =$ 
       $(ps ! \text{Suc } j, ds ! \text{Suc } j, ls ! \text{Suc } j)$ 
  by blast
  show ?thesis
    by (rule fri-successor-domain-points[
      OF ds0 successors j-le k-bound])
qed

```

lemma *composition-fri-commit-initial-domain-points:*

```

assumes outcome:
  Some ((ps, ds, ls, ms), t) ∈
    set-dist (execute
      (p.composition-fri-commit n [cp] [p.eval-domain] [l] [m]) s)
  and j-le: j ≤ n
  and k-bound: k < length (ds ! j)
shows ds ! j ! k = ((p.h ^ k) * shift) ^ (2 ^ j)
proof –
  have ds0: ds ! 0 = p.eval-domain
    using composition-fri-commit-initial-heads(2)[OF outcome] .
  from composition-fri-commit-initial-replay-successor-data[OF outcome]
  obtain roots challenges where

```

```

successors:
   $\bigwedge j. j < n \implies$ 
     $p.\text{next-fri-layer } (ps ! j) (ds ! j) (\text{challenges} ! j) =$ 
       $(ps ! \text{Suc } j, ds ! \text{Suc } j, ls ! \text{Suc } j)$ 
  by blast
show ?thesis
  by (rule fri-successor-domain-points[
    OF ds0 successors j-le k-bound])
qed

lemma fri-commit-outcome-initial-domain-points:
  assumes outcome:
    fri-commit-outcome  $n$   $cp$   $p.\text{eval-domain } l$   $m$   $ps$   $ds$   $ls$   $ms$   $s$   $t$ 
    and j-le:  $j \leq n$ 
    and k-bound:  $k < \text{length } (ds ! j)$ 
  shows  $ds ! j ! k = ((p.h \wedge k) * \text{shift}) \wedge (2 \wedge j)$ 
proof –
  from outcome consider
    (old) Some  $((ps, ds, ls, ms), t) \in$ 
       $\text{set-dist } (\text{execute } (p.\text{fri-commit } n [cp] [p.\text{eval-domain}] [l] [m]) s)$ 
  | (trace) Some  $((ps, ds, ls, ms), t) \in$ 
       $\text{set-dist } (\text{execute } (p.\text{trace-fri-commit } n [cp] [p.\text{eval-domain}] [l] [m]) s)$ 
  | (composition) Some  $((ps, ds, ls, ms), t) \in$ 
       $\text{set-dist } (\text{execute } (p.\text{composition-fri-commit } n [cp] [p.\text{eval-domain}] [l] [m]) s)$ 
  unfolding fri-commit-outcome-def by blast
then show ?thesis
proof cases
  case old
  show ?thesis
    by (rule fri-commit-initial-domain-points[OF old j-le k-bound])
  next
  case trace
  show ?thesis
    by (rule trace-fri-commit-initial-domain-points[OF trace j-le k-bound])
  next
  case composition
  show ?thesis
    by (rule composition-fri-commit-initial-domain-points[
      OF composition j-le k-bound])
  qed
qed

lemma fri-degree-decreases:
  assumes degree  $q > 0$ 
    and  $p.\text{next-fri-layer } q$   $d$   $b = (q', d', l')$ 
  shows degree  $q' < \text{degree } q$ 
  using assms  $p.\text{degree-less-degree next-fri-layer-components}(1)$  by blast

```

```

lemma degree-poly-of-list-less-length:
  assumes  $xs \neq []$ 
  shows  $\text{degree } (\text{poly-of-list } xs) < \text{length } xs$ 
proof -
  have  $\text{degree } (\text{poly-of-list } xs) = \text{length } (\text{coeffs } (\text{poly-of-list } xs)) - 1$ 
    by (simp add: degree-eq-length-coeffs)
  also have  $\dots \leq \text{length } xs - 1$ 
  proof -
    have  $\text{length } (\text{coeffs } (\text{poly-of-list } xs)) \leq \text{length } xs$ 
      by (simp add: coeffs-Poly length-strip-while-le)
    with assms show ?thesis by linarith
  qed
  also have  $\dots < \text{length } xs$ 
    using assms by simp
  finally show ?thesis .
qed

lemma degree-nths-even-coeffs-le-half:
   $\text{degree } (\text{poly-of-list } (\text{nths-pred } (\text{coeffs } q) \text{ even})) \leq \text{degree } q \text{ div } 2$ 
proof (cases coeffs q = [])
  case True
    then show ?thesis by simp
next
  case False
    let  $?xs = \text{nths-pred } (\text{coeffs } q) \text{ even}$ 
    have  $\text{len: length } ?xs = \text{Suc } (\text{length } (\text{coeffs } q)) \text{ div } 2$ 
      by (rule nths-pred-even-length-eq)
    show ?thesis
    proof (cases ?xs = [])
      case True
        then show ?thesis by simp
    next
      case  $xs\text{-ne: False}$ 
        have  $\text{dlt: degree } (\text{poly-of-list } ?xs) < \text{length } ?xs$ 
          using xs-ne by (rule degree-poly-of-list-less-length)
        have  $\text{degree } (\text{poly-of-list } ?xs) < \text{Suc } (\text{length } (\text{coeffs } q)) \text{ div } 2$ 
          using dlt len by simp
        then have  $\text{degree } (\text{poly-of-list } ?xs) \leq (\text{length } (\text{coeffs } q) - 1) \text{ div } 2$ 
          by linarith
        then show ?thesis
          by (simp add: degree-eq-length-coeffs)
    qed
  qed

lemma degree-nths-odd-coeffs-le-half:
   $\text{degree } (\text{poly-of-list } (\text{nths-pred } (\text{coeffs } q) \text{ odd})) \leq \text{degree } q \text{ div } 2$ 
proof (cases coeffs q = [])
  case True
    then show ?thesis by simp

```

```

next
  case False
  let ?xs = nth-pred (coeffs q) odd
  have len: length ?xs = length (coeffs q) div 2
    by (rule nth-pred-odd-length-eq)
  show ?thesis
  proof (cases ?xs = [])
    case True
    then show ?thesis by simp
  next
    case xs-ne: False
    have dlt: degree (poly-of-list ?xs) < length ?xs
      using xs-ne by (rule degree-poly-of-list-less-length)
    then have degree (poly-of-list ?xs) < Suc (length (coeffs q)) div 2
      using len by linarith
    then have degree (poly-of-list ?xs) ≤ (length (coeffs q) - 1) div 2
      by linarith
    then show ?thesis
      by (simp add: degree-eq-length-coeffs)
  qed
qed

lemma fri-degree-halves-polynomial:
  degree (p.next-fri-polynomial q b) ≤ degree q div 2
proof -
  define odd-coefficients where odd-coefficients = nth-pred (coeffs q) odd
  define even-coefficients where even-coefficients = nth-pred (coeffs q) even
  define oddp where oddp = CP b * poly-of-list odd-coefficients
  define evenp where evenp = poly-of-list even-coefficients
  have odd-le: degree oddp ≤ degree q div 2
  proof -
    have degree oddp ≤ degree (CP b) + degree (poly-of-list odd-coefficients)
      unfolding oddp-def by (rule degree-mult-le)
    also have ... ≤ degree q div 2
      unfolding CP-def odd-coefficients-def
      using degree-nth-odd-coeffs-le-half[of q]
      by (simp add: monom-0)
    finally show ?thesis .
  qed
  have even-le: degree evenp ≤ degree q div 2
    unfolding evenp-def even-coefficients-def
    by (rule degree-nth-even-coeffs-le-half)
  have degree (p.next-fri-polynomial q b) = degree (oddp + evenp)
    unfolding p.next-fri-polynomial-def oddp-def evenp-def
    odd-coefficients-def even-coefficients-def
    by (simp add: Let-def)
  also have ... ≤ degree q div 2
    using odd-le even-le by (rule degree-add-le)
  finally show ?thesis .

```


qed

lemma *fri-degree-halves-layer*:

assumes $p.\text{next-fri-layer } q \ d \ b = (q', d', l')$

shows $\text{degree } q' \leq \text{degree } q \ \text{div } 2$

using *next-fri-layer-components(1)[OF assms]* *fri-degree-halves-polynomial* **by** *simp*

lemma *ceil-log-Suc-power-gt*:

$d < 2 \wedge \text{ceil-log } (\text{Suc } d)$

proof (*cases d*)

case 0

then show *?thesis*

unfolding *ceil-log-def* **by** *simp*

next

case (*Suc d'*)

then show *?thesis*

using *floor-log-exp2-gt[of d]*

unfolding *ceil-log-def*

by *simp*

qed

lemma *trace-fri-rounds-enough*:

$\text{ceil-log } (\text{degree } p.f + 1) \leq \text{ceil-log } \text{clength}$

proof –

have $\text{degree } p.f + 1 \leq \text{clength}$

using *trace-polynomial-degree-lt-clength* **by** *simp*

then show *?thesis*

using *ceil-log-mono* **by** *blast*

qed

lemma *div-power-two-ceil-log-Suc*:

$d \ \text{div } 2 \wedge \text{ceil-log } (\text{Suc } d) = 0$

using *ceil-log-Suc-power-gt[of d]* **by** *simp*

lemma *div-div-two-power*:

fixes $m \ n :: \text{nat}$

shows $(m \ \text{div } 2 \wedge n) \ \text{div } 2 = m \ \text{div } 2 \wedge \text{Suc } n$

proof –

have *div-mult*:

$m \ \text{div } (2 \wedge n * 2) = (m \ \text{div } 2 \wedge n) \ \text{div } 2$

using *div-mult2-eq'[of m 2 ^ n 2]* **by** *simp*

have $(m \ \text{div } 2 \wedge n) \ \text{div } 2 = m \ \text{div } (2 \wedge n * 2)$

using *div-mult* **by** *simp*

also have $\dots = m \ \text{div } 2 \wedge \text{Suc } n$

by (*simp add: power-Suc mult.commute*)

finally show *?thesis* .

qed

lemma *fri-degree-chain-halves*:

```

assumes successors:
   $\bigwedge j. j < n \implies$ 
     $p.\text{next-fri-layer } (ps ! j) (ds ! j) (\text{challenges} ! j) =$ 
       $(ps ! \text{Suc } j, ds ! \text{Suc } j, ls ! \text{Suc } j)$ 
    and  $j\text{-le}: j \leq n$ 
shows  $\text{degree } (ps ! j) \leq \text{degree } (ps ! 0) \text{ div } 2^j$ 
using  $j\text{-le}$ 
proof (induction j)
  case 0
    then show ?case by simp
next
  case (Suc j)
    have  $j\text{-lt}: j < n$ 
      using Suc.premis by simp
    have step:
       $\text{degree } (ps ! \text{Suc } j) \leq \text{degree } (ps ! j) \text{ div } 2$ 
      by (rule fri-degree-halves-layer[OF successors[OF j-lt]])
    also have  $\dots \leq (\text{degree } (ps ! 0) \text{ div } 2^j) \text{ div } 2$ 
      by (rule div-le-mono) (rule Suc.IH, use Suc.premis in simp)
    also have  $\dots = \text{degree } (ps ! 0) \text{ div } 2^{\text{Suc } j}$ 
      by (rule div-div-two-power)
    finally show ?case .
qed

lemma fri-degree-final-zero:
assumes successors:
   $\bigwedge j. j < n \implies$ 
     $p.\text{next-fri-layer } (ps ! j) (ds ! j) (\text{challenges} ! j) =$ 
       $(ps ! \text{Suc } j, ds ! \text{Suc } j, ls ! \text{Suc } j)$ 
    and  $n\text{-def}: n = \text{ceil-log } (\text{degree } (ps ! 0) + 1)$ 
shows  $\text{degree } (ps ! n) = 0$ 
proof -
  have  $\text{degree } (ps ! n) \leq \text{degree } (ps ! 0) \text{ div } 2^n$ 
    by (rule fri-degree-chain-halves[OF successors le-refl])
  also have  $\dots = 0$ 
    using  $n\text{-def}$  div-power-two-ceil-log-Suc[of degree (ps ! 0)] by simp
  finally show ?thesis by simp
qed

lemma fri-degree-final-zero-ge:
assumes successors:
   $\bigwedge j. j < n \implies$ 
     $p.\text{next-fri-layer } (ps ! j) (ds ! j) (\text{challenges} ! j) =$ 
       $(ps ! \text{Suc } j, ds ! \text{Suc } j, ls ! \text{Suc } j)$ 
    and  $\text{enough}: \text{ceil-log } (\text{degree } (ps ! 0) + 1) \leq n$ 
shows  $\text{degree } (ps ! n) = 0$ 
proof -
  have  $\text{degree } (ps ! n) \leq \text{degree } (ps ! 0) \text{ div } 2^n$ 
    by (rule fri-degree-chain-halves[OF successors le-refl])

```

```

also have ... = 0
proof -
  have lt-ceil: degree (ps ! 0) < 2 ^ ceil-log (degree (ps ! 0) + 1)
    using ceil-log-Suc-power-gt[of degree (ps ! 0)] by simp
  also have ... ≤ 2 ^ n
    using enough by simp
  finally show ?thesis
    by simp
qed
finally show ?thesis by simp
qed

```

```

lemma poly-degree-zero-const:
  assumes degree q = 0
  shows poly q x = poly q y
proof -
  have q-eq: q = [: coeff q 0 :]
    using degree-0-id[OF assms] by simp
  have poly q x = coeff q 0
    by (subst q-eq) simp
  also have ... = poly q y
    by (subst q-eq) simp
  finally show ?thesis .
qed

```

```

lemma fri-final-layer-constant-value:
  assumes degree0: degree (ps ! n) = 0
    and layer: ls ! n = map (poly (ps ! n)) (ds ! n)
    and i-bound: i < length (ls ! n)
    and len-pos: 0 < length (ls ! n)
  shows ls ! n ! i = hd (ls ! n)
proof -
  have ds-len: length (ds ! n) = length (ls ! n)
    using layer by simp
  then have ds-ne: ds ! n ≠ []
    using len-pos by auto
  have ls ! n ! i = poly (ps ! n) (ds ! n ! i)
    using layer i-bound by simp
  also have ... = poly (ps ! n) (hd (ds ! n))
    by (rule poly-degree-zero-const[OF degree0])
  also have ... = hd (map (poly (ps ! n)) (ds ! n))
proof -
  have hd (map (poly (ps ! n)) (ds ! n)) =
    poly (ps ! n) (hd (ds ! n))
    by (rule hd-map[OF ds-ne])
  then show ?thesis by simp
qed
also have ... = hd (ls ! n)
  using layer by simp

```

finally show ?thesis .
qed

lemma *honest-fri-final-layer-equals-sent*:

assumes *fri*:

Some $((ps, ds, ls, ms), s6) \in$

set-dist (execute

$(p.fri-commit\ nrounds\ [cp']\ [p.eval-domain]\ [p.cp-eval\ as]\ [cp-merkle])\ s5)$

and *len-ps*: $length\ ps = Suc\ nrounds$

and *len-ds*: $length\ ds = Suc\ nrounds$

and *len-ls*: $length\ ls = Suc\ nrounds$

and *layers*:

$\bigwedge i. i < length\ ps \implies i < length\ ds \implies i < length\ ls \implies$

$ls\ !\ i = map\ (poly\ (ps\ !\ i))\ (ds\ !\ i)$

and *init-len*: $length\ (p.cp-eval\ as) = 2^N$

and *n-le-N*: $nrounds \leq N$

and *nrounds-def*: $nrounds = ceil-log\ (degree\ cp' + 1)$

and *successors*:

$\bigwedge j. j < nrounds \implies$

$p.next-fri-layer\ (ps\ !\ j)\ (ds\ !\ j)\ (challenges\ !\ j) =$

$(ps\ !\ Suc\ j,\ ds\ !\ Suc\ j,\ ls\ !\ Suc\ j)$

and *i-bound*: $i < length\ (ls\ !\ nrounds)$

shows $ls\ !\ nrounds\ !\ i = hd\ (last\ ls)$

proof –

have *ps0*: $ps\ !\ 0 = cp'$

using *fri-commit-initial-heads*(1)[*OF fri*] .

have *n-def*: $nrounds = ceil-log\ (degree\ (ps\ !\ 0) + 1)$

using *nrounds-def ps0* by *simp*

have *degree0*: $degree\ (ps\ !\ nrounds) = 0$

by (rule *fri-degree-final-zero*[*OF successors n-def*])

have *layer-n*: $ls\ !\ nrounds = map\ (poly\ (ps\ !\ nrounds))\ (ds\ !\ nrounds)$

by (rule *layers*) (use *len-ps len-ds len-ls* in *simp-all*)

have *len-pos*: $0 < length\ (ls\ !\ nrounds)$

using *fri-commit-initial-layer-length-power*[*OF fri layers init-len n-le-N, of nrounds*]

by *simp*

have $ls\ !\ nrounds\ !\ i = hd\ (ls\ !\ nrounds)$

by (rule *fri-final-layer-constant-value*[*OF degree0 layer-n i-bound len-pos*])

also have $\dots = hd\ (last\ ls)$

proof –

have *ls-ne*: $ls \neq []$

using *len-ls* by *auto*

have $last\ ls = ls\ !\ nrounds$

using *last-conv-nth*[*OF ls-ne len-ls*] by *simp*

then show ?thesis by *simp*

qed

finally show ?thesis .

qed

lemma *honest-fri-final-layer-equals-sent-initial*:

```

assumes fri:
  Some ((ps, ds, ls, ms), s6) ∈
    set-dist (execute
      (p.fri-commit nrounds [cp] [p.eval-domain] [l0] [m0]) s5)
and len-ps: length ps = Suc nrounds
and len-ds: length ds = Suc nrounds
and len-ls: length ls = Suc nrounds
and layers:
   $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$ 
     $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$ 
and init-len: length l0 =  $2^N$ 
and n-le-N: nrounds ≤ N
and nrounds-enough: ceil-log (degree cp + 1) ≤ nrounds
and successors:
   $\bigwedge j. j < \text{nrounds} \implies$ 
     $p.\text{next-fri-layer } (ps ! j) (ds ! j) (\text{challenges} ! j) =$ 
     $(ps ! \text{Suc } j, ds ! \text{Suc } j, ls ! \text{Suc } j)$ 
and i-bound: i < length (ls ! nrounds)
shows ls ! nrounds ! i = hd (last ls)
proof –
  have ps0: ps ! 0 = cp
    using fri-commit-initial-heads(1)[OF fri] .
  have n-enough: ceil-log (degree (ps ! 0) + 1) ≤ nrounds
    using nrounds-enough ps0 by simp
  have degree0: degree (ps ! nrounds) = 0
    by (rule fri-degree-final-zero-ge[OF successors n-enough])
  have layer-n: ls ! nrounds = map (poly (ps ! nrounds)) (ds ! nrounds)
    by (rule layers) (use len-ps len-ds len-ls in simp-all)
  have len-pos: 0 < length (ls ! nrounds)
    using fri-commit-initial-layer-length-power[OF fri layers init-len n-le-N, of nrounds]
    by simp
  have ls ! nrounds ! i = hd (ls ! nrounds)
    by (rule fri-final-layer-constant-value[OF degree0 layer-n i-bound len-pos])
  also have ... = hd (last ls)
  proof –
    have ls-ne: ls ≠ []
      using len-ls by auto
    have last ls = ls ! nrounds
      using last-conv-nth[OF ls-ne] len-ls by simp
    then show ?thesis by simp
  qed
  finally show ?thesis .
qed

```

lemma *honest-fri-final-layer-equals-sent-shape*:

```

assumes fri:
  fri-commit-outcome nrounds cp p.eval-domain l0 m0 ps ds ls ms s5 s6
and len-ps: length ps = Suc nrounds
and len-ds: length ds = Suc nrounds

```

```

and len-ls: length ls = Suc nrounds
and layers:
   $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$ 
   $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$ 
and init-len: length l0 =  $2^N$ 
and n-le-N: nrounds  $\leq N$ 
and nrounds-def: nrounds = ceil-log (degree cp + 1)
and successors:
   $\bigwedge j. j < \text{nrounds} \implies$ 
   $p.\text{next-fri-layer } (ps ! j) (ds ! j) (\text{challenges} ! j) =$ 
   $(ps ! \text{Suc } j, ds ! \text{Suc } j, ls ! \text{Suc } j)$ 
and i-bound:  $i < \text{length } (ls ! \text{nrounds})$ 
shows  $ls ! \text{nrounds} ! i = \text{hd } (\text{last } ls)$ 
proof –
  have ps0:  $ps ! 0 = cp$ 
  using fri-commit-outcome-initial-heads(1)[OF fri] .
  have n-def: nrounds = ceil-log (degree ( $ps ! 0$ ) + 1)
  using nrounds-def ps0 by simp
  have degree0: degree ( $ps ! \text{nrounds}$ ) = 0
  by (rule fri-degree-final-zero[OF successors n-def])
  have layer-n:  $ls ! \text{nrounds} = \text{map } (\text{poly } (ps ! \text{nrounds})) (ds ! \text{nrounds})$ 
  by (rule layers) (use len-ps len-ds len-ls in simp-all)
  have len-pos:  $0 < \text{length } (ls ! \text{nrounds})$ 
  using fri-commit-outcome-initial-layer-length-power[
    OF fri layers init-len n-le-N, of nrounds]
  by simp
  have  $ls ! \text{nrounds} ! i = \text{hd } (ls ! \text{nrounds})$ 
  by (rule fri-final-layer-constant-value[OF degree0 layer-n i-bound len-pos])
  also have ... = hd (last ls)
  proof –
  have ls-ne:  $ls \neq []$ 
  using len-ls by auto
  have last ls =  $ls ! \text{nrounds}$ 
  using last-conv-nth[OF ls-ne] len-ls by simp
  then show ?thesis by simp
  qed
  finally show ?thesis .
qed

```

lemma *honest-fri-final-layer-equals-sent-initial-shape*:

assumes *fri*:

fri-commit-outcome nrounds cp p.eval-domain l0 m0 ps ds ls ms s5 s6

and *len-ps*: *length ps* = *Suc nrounds*

and *len-ds*: *length ds* = *Suc nrounds*

and *len-ls*: *length ls* = *Suc nrounds*

and *layers*:

$\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$

$ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$

and *init-len*: *length l0* = 2^N

```

and  $n\text{-le-}N$ :  $nrounds \leq N$ 
and  $nrounds\text{-enough}$ :  $\text{ceil-log } (degree\ cp + 1) \leq nrounds$ 
and  $successors$ :
   $\bigwedge j. j < nrounds \implies$ 
     $p.\text{next-fri-layer } (ps ! j) (ds ! j) (\text{challenges} ! j) =$ 
       $(ps ! \text{Suc } j, ds ! \text{Suc } j, ls ! \text{Suc } j)$ 
    and  $i\text{-bound}$ :  $i < \text{length } (ls ! nrounds)$ 
shows  $ls ! nrounds ! i = \text{hd } (\text{last } ls)$ 
proof –
  have  $ps0$ :  $ps ! 0 = cp$ 
    using  $\text{fri-commit-outcome-initial-heads}(1)[OF\ \text{fri}]$  .
  have  $n\text{-enough}$ :  $\text{ceil-log } (degree\ (ps ! 0) + 1) \leq nrounds$ 
    using  $nrounds\text{-enough } ps0$  by  $\text{simp}$ 
  have  $degree0$ :  $degree\ (ps ! nrounds) = 0$ 
    by  $(\text{rule } \text{fri-degree-final-zero-ge}[OF\ \text{successors } n\text{-enough}])$ 
  have  $\text{layer-n}$ :  $ls ! nrounds = \text{map } (\text{poly } (ps ! nrounds)) (ds ! nrounds)$ 
    by  $(\text{rule } \text{layers})\ (\text{use } \text{len-ps } \text{len-ds } \text{len-ls } \text{in } \text{simp-all})$ 
  have  $\text{len-pos}$ :  $0 < \text{length } (ls ! nrounds)$ 
    using  $\text{fri-commit-outcome-initial-layer-length-power}[$ 
       $OF\ \text{fri layers init-len } n\text{-le-}N, \text{ of } nrounds]$ 
    by  $\text{simp}$ 
  have  $ls ! nrounds ! i = \text{hd } (ls ! nrounds)$ 
    by  $(\text{rule } \text{fri-final-layer-constant-value}[OF\ degree0\ \text{layer-n } i\text{-bound } \text{len-pos}])$ 
  also have  $\dots = \text{hd } (\text{last } ls)$ 
proof –
  have  $ls\text{-ne}$ :  $ls \neq []$ 
    using  $\text{len-ls}$  by  $\text{auto}$ 
  have  $\text{last } ls = ls ! nrounds$ 
    using  $\text{last-conv-nth}[OF\ ls\text{-ne}]$   $\text{len-ls}$  by  $\text{simp}$ 
  then show  $?thesis$  by  $\text{simp}$ 
qed
finally show  $?thesis$  .
qed

```

```

lemma  $\text{poly-of-list-even-odd-decompose}$ :
  fixes  $xs :: 'f\ \text{list}$ 
  and  $x :: 'f$ 
  shows
     $\text{poly } (\text{poly-of-list } xs) x =$ 
       $\text{poly } (\text{poly-of-list } (\text{nths-pred } xs\ \text{even})) (x * x) +$ 
       $x * \text{poly } (\text{poly-of-list } (\text{nths-pred } xs\ \text{odd})) (x * x)$ 
proof  $(\text{induction } xs\ \text{rule: } \text{measure-induct-rule}[of\ \text{length}])$ 
  case  $(\text{less } xs)$ 
  show  $?case$ 
proof  $(\text{cases } xs)$ 
  case  $Nil$ 
  then show  $?thesis$  by  $\text{simp}$ 
next
  case  $(\text{Cons } a\ ys)$ 

```

```

note  $xs\text{-}eq = Cons$ 
show  $?thesis$ 
proof ( $cases\ ys$ )
  case  $Nil$ 
  then show  $?thesis$ 
    unfolding  $xs\text{-}eq$  by ( $simp\ add:\ nth\text{-}Cons$ )
next
  case ( $Cons\ b\ zs$ )
  note  $ys\text{-}eq = Cons$ 
  have  $even\text{-}tail$ :
     $nths\text{-}pred\ (a \# b \# zs)\ even = a \# nths\text{-}pred\ zs\ even$ 
    unfolding  $nths\text{-}pred\text{-}def$  by ( $simp\ add:\ nth\text{-}Cons$ )
  have  $odd\text{-}tail$ :
     $nths\text{-}pred\ (a \# b \# zs)\ odd = b \# nths\text{-}pred\ zs\ odd$ 
    unfolding  $nths\text{-}pred\text{-}def$  by ( $simp\ add:\ nth\text{-}Cons$ )
  have  $zs\text{-}less$ :  $length\ zs < length\ xs$ 
    unfolding  $xs\text{-}eq\ ys\text{-}eq$  by  $simp$ 
  have  $poly\text{-}tail$ :
     $poly\ (poly\text{-}of\text{-}list\ zs)\ x =$ 
     $poly\ (poly\text{-}of\text{-}list\ (nths\text{-}pred\ zs\ even))\ (x * x) +$ 
     $x * poly\ (poly\text{-}of\text{-}list\ (nths\text{-}pred\ zs\ odd))\ (x * x)$ 
    by ( $rule\ less.IH[OF\ zs\text{-}less]$ )
  show  $?thesis$ 
    unfolding  $xs\text{-}eq\ ys\text{-}eq\ even\text{-}tail\ odd\text{-}tail$ 
    using  $poly\text{-}tail$ 
    by ( $simp\ add:\ algebra\text{-}simps\ power2\text{-}eq\text{-}square$ )
qed
qed
qed

```

```

lemma  $poly\text{-}even\text{-}part\text{-}reconstruct$ :
  fixes  $q :: 'f\ poly$ 
  and  $x :: 'f$ 
  assumes  $two\text{-}nonzero$ :  $(2 :: 'f) \neq 0$ 
  shows
     $poly\ (poly\text{-}of\text{-}list\ (nths\text{-}pred\ (coeffs\ q)\ even))\ (x * x) =$ 
     $(poly\ q\ x + poly\ q\ (-x))\ div\ 2$ 
proof -
  have  $decomp\text{-}x$ :
     $poly\ q\ x =$ 
     $poly\ (poly\text{-}of\text{-}list\ (nths\text{-}pred\ (coeffs\ q)\ even))\ (x * x) +$ 
     $x * poly\ (poly\text{-}of\text{-}list\ (nths\text{-}pred\ (coeffs\ q)\ odd))\ (x * x)$ 
    using  $poly\text{-}of\text{-}list\text{-}even\text{-}odd\text{-}decompose[of\ coeffs\ q\ x]$ 
    by  $simp$ 
  have  $decomp\text{-}neg$ :
     $poly\ q\ (-x) =$ 
     $poly\ (poly\text{-}of\text{-}list\ (nths\text{-}pred\ (coeffs\ q)\ even))\ (x * x) -$ 
     $x * poly\ (poly\text{-}of\text{-}list\ (nths\text{-}pred\ (coeffs\ q)\ odd))\ (x * x)$ 
    using  $poly\text{-}of\text{-}list\text{-}even\text{-}odd\text{-}decompose[of\ coeffs\ q\ -x]$ 

```



```

    by (simp add: algebra-simps power2-eq-square)
  show ?thesis
    using decomp-x decomp-neg two-nonzero
    by (simp add: field-simps)
qed

lemma poly-odd-part-reconstruct:
  fixes q :: 'f poly
  and x :: 'f
  assumes two-nonzero: (2::'f) ≠ 0
  and x-nonzero: x ≠ 0
  shows poly (poly-of-list (nth-pred (coeffs q) odd)) (x * x) =
    (poly q x - poly q (-x)) div (2 * x)
proof -
  have decomp-x:
    poly q x =
      poly (poly-of-list (nth-pred (coeffs q) even)) (x * x) +
      x * poly (poly-of-list (nth-pred (coeffs q) odd)) (x * x)
    using poly-of-list-even-odd-decompose[of coeffs q x]
    by simp
  have decomp-neg:
    poly q (-x) =
      poly (poly-of-list (nth-pred (coeffs q) even)) (x * x) -
      x * poly (poly-of-list (nth-pred (coeffs q) odd)) (x * x)
    using poly-of-list-even-odd-decompose[of coeffs q -x]
    by (simp add: algebra-simps power2-eq-square)
  show ?thesis
    using decomp-x decomp-neg two-nonzero x-nonzero
    by (simp add: field-simps mult.commute)
qed

```

```

lemma next-fri-polynomial-fold:
  fixes q :: 'f poly
  and b z :: 'f
  assumes two-nonzero: (2::'f) ≠ 0
  and z-nonzero: z ≠ 0
  shows
    ((poly q z + poly q (-z)) div 2 +
     b * ((poly q z - poly q (-z)) div (2 * z))) =
    poly (p.next-fri-polynomial q b) (z * z)
proof -
  let ?E = poly (poly-of-list (nth-pred (coeffs q) even)) (z * z)
  let ?O = poly (poly-of-list (nth-pred (coeffs q) odd)) (z * z)
  have even:
    ?E = (poly q z + poly q (-z)) div 2
    by (rule poly-even-part-reconstruct[OF two-nonzero])
  have odd:
    ?O = (poly q z - poly q (-z)) div (2 * z)
    by (rule poly-odd-part-reconstruct[OF two-nonzero z-nonzero])

```

```

have next-poly-eval:
  poly (p.next-fri-polynomial q b) (z * z) = b * ?O + ?E
  unfolding p.next-fri-polynomial-def CP-def
  by (simp add: Let-def monom-0 algebra-simps)
have lhs:
  ((poly q z + poly q (- z)) div 2 +
    b * ((poly q z - poly q (- z)) div (2 * z))) =
    ?E + b * ?O
  using even odd by simp
also have ... = poly (p.next-fri-polynomial q b) (z * z)
  using next-poly-eval by (simp add: algebra-simps)
finally show ?thesis .
qed

```

```

lemma fri-fold-value-poly-next:
  fixes q :: 'f poly
  and b z :: 'f
  assumes two-nonzero: (2::'f) ≠ 0
  and z-nonzero: z ≠ 0
  and len-eq: len = length l
  and l-eval: l = map (poly q) d
  and d-len: length d = length l
  and i-bound: i < len
  and z-eq: z = ((p.h ^ i) * shift) ^ pw
  and d-i: d ! i = z
  and d-sib: d ! ((i + len div 2) mod len) = - z
  shows fri-fold-value b l i len pw =
    poly (p.next-fri-polynomial q b) (z * z)
proof -
  let ?sidx = (i + len div 2) mod len
  have len-pos: 0 < len
  using i-bound by simp
  have i-d-bound: i < length d
  using i-bound len-eq d-len by simp
  have sib-l-bound: ?sidx < length l
  using len-pos len-eq by simp
  have sib-d-bound: ?sidx < length d
  using sib-l-bound d-len by simp
  have l-i: l ! i = poly q z
  using l-eval d-i i-d-bound by simp
  have l-sib: l ! ?sidx = poly q (- z)
  using l-eval d-sib sib-d-bound by simp
  show ?thesis
  unfolding fri-fold-value-def
  using l-i l-sib z-eq next-fri-polynomial-fold[OF two-nonzero z-nonzero, of q b]
  by simp
qed

```

```

lemma fri-fold-value-next-layer-value:

```

```

fixes  $q :: 'f \text{ poly}$ 
  and  $b \ z :: 'f$ 
assumes  $\text{two-nonzero}: (2 :: 'f) \neq 0$ 
  and  $\text{z-nonzero}: z \neq 0$ 
  and  $\text{layer}: p.\text{next-fri-layer } q \ d \ b = (q', d', l')$ 
  and  $\text{len-eq}: \text{len} = \text{length } l$ 
  and  $\text{l-eval}: l = \text{map } (\text{poly } q) \ d$ 
  and  $\text{d-len}: \text{length } d = \text{length } l$ 
  and  $\text{i-bound}: i < \text{len}$ 
  and  $\text{z-eq}: z = ((p.h \wedge i) * \text{shift}) \wedge pw$ 
  and  $\text{d-i}: d ! i = z$ 
  and  $\text{d-sib}: d ! ((i + \text{len div } 2) \bmod \text{len}) = -z$ 
  and  $\text{next-idx-bound}: i \bmod (\text{len div } 2) < \text{length } d'$ 
  and  $\text{d-next}: d' ! (i \bmod (\text{len div } 2)) = z * z$ 
shows  $\text{fri-fold-value } b \ l \ i \ \text{len} \ pw = l' ! (i \bmod (\text{len div } 2))$ 
proof –
  have  $\text{fold}:$ 
     $\text{fri-fold-value } b \ l \ i \ \text{len} \ pw =$ 
       $\text{poly } (p.\text{next-fri-polynomial } q \ b) \ (z * z)$ 
    by ( $\text{rule } \text{fri-fold-value-poly-next} [$ 
       $OF \ \text{two-nonzero } \text{z-nonzero } \text{len-eq } \text{l-eval } \text{d-len } \text{i-bound } \text{z-eq } \text{d-i } \text{d-sib}]$ )
  have  $q'\text{-eq}: q' = p.\text{next-fri-polynomial } q \ b$ 
    using  $\text{next-fri-layer-components}(1)[OF \ \text{layer}]$  .
  have  $l'\text{-eq}: l' = \text{map } (\text{poly } q') \ d'$ 
    using  $\text{next-fri-layer-components}(3)[OF \ \text{layer}]$  .
  have  $l' ! (i \bmod (\text{len div } 2)) = \text{poly } q' \ (z * z)$ 
    using  $l'\text{-eq } \text{next-idx-bound } \text{d-next}$  by  $\text{simp}$ 
  then show  $?thesis$ 
    using  $\text{fold } q'\text{-eq}$  by  $\text{simp}$ 
qed

```

```

lemma  $\text{honest-fri-layer-decommitment-step-created-next-value}:$ 
  fixes  $q \ q' :: 'f \text{ poly}$ 
    and  $d \ d' \ l \ l' :: 'f \text{ list}$ 
  assumes  $\text{created}: p.\text{created-tree } l \ m \ s0$ 
    and  $\text{len-eq}: \text{len} = \text{length } l$ 
    and  $\text{len-pow}: \text{length } l = 2 \wedge n$ 
    and  $\text{i-bound}: i < \text{len}$ 
    and  $\text{x-eq}: x = l ! i$ 
    and  $\text{ext}: s0 \leq s$ 
    and  $\text{root}: fr = \text{value } m$ 
    and  $\text{tr}:$ 
       $\text{start} =$ 
         $s(P\text{Transcript} := \text{fri-layer-decommitment-transcript } l \ m \ i \ \text{len} \ @ \ \text{rest})$ 
    and  $\text{layer}: p.\text{next-fri-layer } q \ d \ b = (q', d', l')$ 
    and  $\text{l-eval}: l = \text{map } (\text{poly } q) \ d$ 
    and  $\text{d-len}: \text{length } d = \text{length } l$ 
    and  $\text{z-eq}: z = ((p.h \wedge i) * \text{shift}) \wedge pw$ 
    and  $\text{d-i}: d ! i = z$ 

```

and $d\text{-sib}$: $d \neq ((i + \text{len div } 2) \bmod \text{len}) = -z$
and next-idx-bound : $i \bmod (\text{len div } 2) < \text{length } d'$
and $d\text{-next}$: $d' \neq (i \bmod (\text{len div } 2)) = z * z$
shows
 $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (\text{fri-verifier-layer } b \text{ fr } i \text{ x len pw}) \text{ start}))$
 $\wedge \text{res } t'. \text{Some } (\text{res}, t') \in$
 $\text{set-dist } (\text{execute } (\text{fri-verifier-layer } b \text{ fr } i \text{ x len pw}) \text{ start}) \implies$
 $\text{res} =$
 $(i \bmod (\text{len div } 2), l' \neq (i \bmod (\text{len div } 2)), \text{len div } 2, \text{pw} + \text{pw}) \wedge$
 $P\text{Transcript } t' = \text{rest} \wedge$
 $\text{start} \leq t'$
proof –
show $\text{None} \notin \text{dom } (\text{dist } (\text{execute } (\text{fri-verifier-layer } b \text{ fr } i \text{ x len pw}) \text{ start}))$
by (rule honest-fri-layer-decommitment-step-created(1)[
 $OF \text{ created len-eq len-pow i-bound x-eq ext root tr}]$)
fix $\text{res } t'$
assume outcome :
 $\text{Some } (\text{res}, t') \in$
 $\text{set-dist } (\text{execute } (\text{fri-verifier-layer } b \text{ fr } i \text{ x len pw}) \text{ start})$
have step :
 $\text{res} =$
 $(i \bmod (\text{len div } 2), \text{fri-fold-value } b \text{ l } i \text{ len pw}, \text{len div } 2, \text{pw} + \text{pw}) \wedge$
 $P\text{Transcript } t' = \text{rest} \wedge$
 $\text{start} \leq t'$
by (rule honest-fri-layer-decommitment-step-created(2)[
 $OF \text{ created len-eq len-pow i-bound x-eq ext root tr outcome}]$)
have $z\text{-nonzero}$: $z \neq 0$
using $z\text{-eq verifier-domain-point-nonzero}$ **by** simp
have fold-eq :
 $\text{fri-fold-value } b \text{ l } i \text{ len pw} = l' \neq (i \bmod (\text{len div } 2))$
by (rule fri-fold-value-next-layer-value[
 $OF p.\text{two-nonzero } z\text{-nonzero layer len-eq l-eval d-len i-bound z-eq}$
 $d\text{-i } d\text{-sib next-idx-bound } d\text{-next}]$)
show
 $\text{res} =$
 $(i \bmod (\text{len div } 2), l' \neq (i \bmod (\text{len div } 2)), \text{len div } 2, \text{pw} + \text{pw}) \wedge$
 $P\text{Transcript } t' = \text{rest} \wedge$
 $\text{start} \leq t'$
using step fold-eq **by** simp
qed

lemma $\text{honest-fri-layer-decommitment-step-created-next-value-domain}$:
fixes $q \ q' :: 'f \text{ poly}$
and $d \ d' \ l \ l' :: 'f \text{ list}$
assumes created : $p.\text{created-tree } l \ m \ s0$
and len-eq : $\text{len} = \text{length } l$
and len-pow : $\text{length } l = 2^n$
and $i\text{-bound}$: $i < \text{len}$
and $x\text{-eq}$: $x = l \ ! \ i$

```

and ext:  $s0 \leq s$ 
and root:  $fr = \text{value } m$ 
and tr:
  start =
     $s(PTranscript := \text{fri-layer-decommitment-transcript } l \ m \ i \ len \ @ \ rest)$ 
and layer:  $p.\text{next-fri-layer } q \ d \ b = (q', d', l')$ 
and l-eval:  $l = \text{map } (poly \ q) \ d$ 
and d-len:  $\text{length } d = \text{length } l$ 
and even-len:  $2 \ \text{dvd} \ len$ 
and round:  $len * pw = \text{clength} * \text{scale}$ 
and d-i-point:  $d \ ! \ i = ((p.h \wedge i) * \text{shift}) \wedge pw$ 
and d-sib-point:
   $d \ ! \ ((i + len \ \text{div} \ 2) \ \text{mod} \ len) =$ 
   $((p.h \wedge ((i + len \ \text{div} \ 2) \ \text{mod} \ len)) * \text{shift}) \wedge pw$ 
and next-idx-bound:  $i \ \text{mod} \ (len \ \text{div} \ 2) < \text{length } d'$ 
and d-next:
   $d' \ ! \ (i \ \text{mod} \ (len \ \text{div} \ 2)) =$ 
   $((p.h \wedge i) * \text{shift}) \wedge pw * ((p.h \wedge i) * \text{shift}) \wedge pw$ 
shows
   $None \notin \text{dom} (\text{dist } (\text{execute } (\text{fri-verifier-layer } b \ fr \ i \ x \ len \ pw) \ start))$ 
   $\wedge \text{res } t'. \text{Some } (res, t') \in$ 
   $\text{set-dist } (\text{execute } (\text{fri-verifier-layer } b \ fr \ i \ x \ len \ pw) \ start) \implies$ 
   $res =$ 
   $(i \ \text{mod} \ (len \ \text{div} \ 2), l' \ ! \ (i \ \text{mod} \ (len \ \text{div} \ 2)), len \ \text{div} \ 2, pw + pw) \wedge$ 
   $PTranscript \ t' = rest \wedge$ 
   $start \leq t'$ 
proof –
  let  $?z = ((p.h \wedge i) * \text{shift}) \wedge pw$ 
  have len-pos:  $0 < len$ 
  using i-bound by simp
  have d-sib:  $d \ ! \ ((i + len \ \text{div} \ 2) \ \text{mod} \ len) = - \ ?z$ 
  by (rule fri-sibling-domain-at[OF len-pos even-len round refl d-sib-point])
  show  $None \notin \text{dom} (\text{dist } (\text{execute } (\text{fri-verifier-layer } b \ fr \ i \ x \ len \ pw) \ start))$ 
  by (rule honest-fri-layer-decommitment-step-created-next-value(1)[
    OF created len-eq len-pow i-bound x-eq ext root tr layer l-eval d-len
    refl d-i-point d-sib next-idx-bound d-next])
  fix res t'
  assume outcome:
     $\text{Some } (res, t') \in$ 
     $\text{set-dist } (\text{execute } (\text{fri-verifier-layer } b \ fr \ i \ x \ len \ pw) \ start)$ 
  show
     $res =$ 
     $(i \ \text{mod} \ (len \ \text{div} \ 2), l' \ ! \ (i \ \text{mod} \ (len \ \text{div} \ 2)), len \ \text{div} \ 2, pw + pw) \wedge$ 
     $PTranscript \ t' = rest \wedge$ 
     $start \leq t'$ 
    by (rule honest-fri-layer-decommitment-step-created-next-value(2)[
      OF created len-eq len-pow i-bound x-eq ext root tr layer l-eval d-len
      refl d-i-point d-sib next-idx-bound d-next outcome])
qed

```

lemma *honest-fri-layer-decommitment-step-created-next-value-domain-state*:

fixes $q\ q' :: 'f\ poly$
and $d\ d'\ l\ l' :: 'f\ list$
assumes *created*: $p.created\ tree\ l\ m\ s0$
and *len-eq*: $len = length\ l$
and *len-pow*: $length\ l = 2^n$
and *i-bound*: $i < len$
and *x-eq*: $x = l!\ i$
and *ext*: $s0 \leq s$
and *root*: $fr = value\ m$
and *tr*:
 $start =$
 $s(PTranscript := fri\ layer\ decommitment\ transcript\ l\ m\ i\ len\ @\ rest)$
and *layer*: $p.next\ fri\ layer\ q\ d\ b = (q', d', l')$
and *l-eval*: $l = map\ (poly\ q)\ d$
and *d-len*: $length\ d = length\ l$
and *even-len*: $2\ dvd\ len$
and *round*: $len * pw = clength * scale$
and *d-i-point*: $d!\ i = ((p.h \wedge i) * shift) \wedge pw$
and *d-sib-point*:
 $d!\ ((i + len\ div\ 2) \bmod len) =$
 $((p.h \wedge ((i + len\ div\ 2) \bmod len)) * shift) \wedge pw$
and *next-idx-bound*: $i \bmod (len\ div\ 2) < length\ d'$
and *d-next*:
 $d'\ (i \bmod (len\ div\ 2)) =$
 $((p.h \wedge i) * shift) \wedge pw * ((p.h \wedge i) * shift) \wedge pw$
and *outcome*:
 $Some\ (res, t') \in$
 $set\ dist\ (execute\ (fri\ verifier\ layer\ b\ fr\ i\ x\ len\ pw)\ start)$
shows
 $res =$
 $(i \bmod (len\ div\ 2), l'\ (i \bmod (len\ div\ 2)), len\ div\ 2, pw + pw) \wedge$
 $PTranscript\ t' = rest \wedge$
 $start \leq t' \wedge$
 $PState\ t' = foldl\ concat\ (PState\ s)\ (fri\ layer\ decommitment\ transcript\ l\ m\ i\ len)$
proof –
have *old*:
 $res =$
 $(i \bmod (len\ div\ 2), l'\ (i \bmod (len\ div\ 2)), len\ div\ 2, pw + pw) \wedge$
 $PTranscript\ t' = rest \wedge$
 $start \leq t'$
by (rule *honest-fri-layer-decommitment-step-created-next-value-domain*(2)[
 $OF\ created\ len\ eq\ len\ pow\ i\ bound\ x\ eq\ ext\ root\ tr\ layer\ l\ eval\ d\ len$
 $even\ len\ round\ d\ i\ point\ d\ sib\ point\ next\ idx\ bound\ d\ next\ outcome$])
have *path-i*:
 $length\ (get\ authentication\ path\ len\ i\ m) = floor\ log\ len$
using *created-tree-get-authentication-path-len*[
 $OF\ created\ len\ pow,\ of\ i]$ *i-bound len-eq* **by** *simp*

```

have len-pos: 0 < len
  using i-bound by simp
have sidx-bound:
  (i + len div 2) mod len < length l
  using len-pos len-eq by simp
have path-s:
  length (get-authentication-path len ((i + len div 2) mod len) m) =
    floor-log len
  using created-tree-get-authentication-path-len[
    OF created len-pow, of (i + len div 2) mod len] sidx-bound len-eq
  by simp
have state:
  PState t' = foldl concat (PState s) (fri-layer-decommitment-transcript l m i len)
  using honest-fri-layer-decommitment-outcome(3)[
    OF len-eq i-bound path-i path-s outcome[unfolded fri-verifier-layer-def tr]] .
show ?thesis
  using old state by simp
qed

end

end

```

```

theory Completeness-Verifier
  imports Completeness-FRI
begin

```

Final honest verifier query-round replay and completeness theorem.

```

context verification
begin

```

```

lemma honest-fri-decommitments-mfold-no-failure:
  assumes fri:
    fri-commit-outcome n cp p.eval-domain l0 m0 ps ds ls ms s5 s6
  and len-ps: length ps = Suc n
  and len-ds: length ds = Suc n
  and len-ls: length ls = Suc n
  and len-ms: length ms = Suc n
  and layers:
     $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$ 
     $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$ 
  and created:
     $\bigwedge i. i < \text{length } ls \implies i < \text{length } ms \implies$ 
     $p.\text{created-tree } (ls ! i) (ms ! i) s6$ 
  and init-len: length l0 =  $2^N$ 
  and init-domain:  $2^N = \text{clength} * \text{scale}$ 
  and n-le-N:  $n \leq N$ 
  and len-roots: length roots = n

```

```

and len-challenges: length challenges = n
and roots-aligned:  $\bigwedge j. j < n \implies \text{roots} ! j = \text{value} (\text{ms} ! j)$ 
and successors:
   $\bigwedge j. j < n \implies$ 
    p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
      (ps ! Suc j, ds ! Suc j, ls ! Suc j)
and idx-bound: idx < length (ls ! 0)
and x0-eq: x0 = ls ! 0 ! idx
and len0-eq: len0 = length (ls ! 0)
and start-tr:
  PTranscript start = fri-decommitment-transcript idx (butlast (zip ls ms)) @ rest
and start-ext: s6 ≤ start
shows
  None ∉ dom (dist (execute
    (mfold (idx, x0, len0, 1) (v.receive-query-commits (zip challenges roots)))
    start))
proof –
  let ?fl =
    v.receive-query-commits (zip challenges roots) ::
      ((nat × 'f × nat × nat) ⇒
        (nat × 'f × nat × nat, 'f protocol-channel) state-monad) list
  let ?bl = butlast (zip ls ms)
  define I where
    I = ( $\lambda(j::\text{nat}) (a::\text{nat} \times 'f \times \text{nat} \times \text{nat})$ 
      (s::'f protocol-channel).
      (case a of (i, x, len, pw) ⇒
        i = idx mod length (ls ! j) ∧
        x = ls ! j ! i ∧
        len = length (ls ! j) ∧
        pw =  $2^{\wedge j}$  ∧
        PTranscript s = fri-decommitment-transcript i (drop j ?bl) @ rest ∧
        s6 ≤ s))
  have fl-len: length ?fl = n
  using len-roots len-challenges
  unfolding v.receive-query-commits-def by simp
  have init-I: I 0 (idx, x0, len0, 1) start
  unfolding I-def
  using idx-bound x0-eq len0-eq start-tr start-ext by simp
  have step-nf:
     $\bigwedge j m a st. j < \text{length } ?fl \implies$ 
      m = ?fl ! j ⇒ I j a st ⇒
      None ∉ dom (dist (execute (m a st)))
  proof –
    fix j m a st
    assume j-lt-fl: j < length ?fl
    and m-eq: m = ?fl ! j
    and inv: I j a st
    have j-lt: j < n
    using j-lt-fl fl-len by presburger

```



```

obtain  $i\ x\ len\ pw$  where  $a\text{-eq}: a = (i, x, len, pw)$ 
  by ( $cases\ a$ ) auto
have  $invD$ :
   $i = idx\ mod\ length\ (ls\ !\ j)$ 
   $x = ls\ !\ j\ !\ i$ 
   $len = length\ (ls\ !\ j)$ 
   $pw = 2^j$ 
   $PTranscript\ st = fri\text{-}decommitment\text{-}transcript\ i\ (drop\ j\ ?bl)\ @\ rest$ 
   $s6 \leq st$ 
  using  $inv$  unfolding  $I\text{-}def\ a\text{-}eq$  by  $simp\text{-}all$ 
have  $j\text{-}le: j \leq n$ 
  using  $j\text{-}lt$  by  $simp$ 
have  $len\text{-}exact$ :
   $length\ (ls\ !\ j) = 2^{(N - j)}$ 
  by ( $rule\ fri\text{-}commit\text{-}outcome\text{-}initial\text{-}layer\text{-}length\text{-}power[$ 
     $OF\ fri\ layers\ init\text{-}len\ n\text{-}le\text{-}N\ j\text{-}le]$ )
have  $j\text{-}lt\text{-}N: j < N$ 
  using  $j\text{-}lt\ n\text{-}le\text{-}N$  by  $linarith$ 
have  $diff\text{-}suc: N - j = Suc\ (N - Suc\ j)$ 
  using  $j\text{-}lt\text{-}N$  by  $simp$ 
have  $len\text{-}pow\text{-}suc$ :
   $length\ (ls\ !\ j) = 2^{Suc\ (N - Suc\ j)}$ 
  using  $len\text{-}exact\ diff\text{-}suc$  by  $simp$ 
have  $len\text{-}pos: 0 < length\ (ls\ !\ j)$ 
  using  $len\text{-}pow\text{-}suc$  by  $simp$ 
have  $i\text{-}bound: i < len$ 
  using  $invD\ len\text{-}pos$  by  $simp$ 
have  $drop\text{-}eq$ :
   $drop\ j\ ?bl = (ls\ !\ j, ms\ !\ j) \# drop\ (Suc\ j)\ ?bl$ 
  by ( $rule\ butlast\text{-}zip\text{-}suc\text{-}drop\text{-}nth[OF\ len\text{-}ls\ len\text{-}ms\ j\text{-}lt]$ )
have  $tr\text{-}layer$ :
   $st =$ 
   $st(PTranscript :=$ 
     $fri\text{-}layer\text{-}decommitment\text{-}transcript\ (ls\ !\ j)\ (ms\ !\ j)\ i\ len\ @$ 
     $fri\text{-}decommitment\text{-}transcript\ i\ (drop\ (Suc\ j)\ ?bl)\ @\ rest)$ 
  using  $invD\ drop\text{-}eq$  unfolding  $fri\text{-}decommitment\text{-}transcript.simps$ 
  by  $simp$ 
have  $len\text{-}pow: \exists K. length\ (ls\ !\ j) = 2^K$ 
  using  $len\text{-}pow\text{-}suc$  by  $blast$ 
then obtain  $K$  where  $len\text{-}pow\text{-}K: length\ (ls\ !\ j) = 2^K$ 
  by  $blast$ 
have  $l\text{-}eval$ :
   $ls\ !\ j = map\ (poly\ (ps\ !\ j))\ (ds\ !\ j)$ 
  by ( $rule\ layers$ ) ( $use\ j\text{-}lt\ len\text{-}ps\ len\text{-}ds\ len\text{-}ls$  in  $simp\text{-}all$ )
have  $d\text{-}len: length\ (ds\ !\ j) = length\ (ls\ !\ j)$ 
  using  $l\text{-}eval$  by  $simp$ 
have  $i\text{-}d\text{-}bound: i < length\ (ds\ !\ j)$ 
  using  $i\text{-}bound\ invD\ d\text{-}len$  by  $simp$ 
have  $d\text{-}i$ :

```

```

  ds ! j ! i = ((p.h ^ i) * shift) ^ pw
  using fri-commit-outcome-initial-domain-points[OF fri j-le i-d-bound] invD by simp
have sidx-bound:
  (i + len div 2) mod len < length (ds ! j)
  using len-pos invD d-len by simp
have d-sib:
  ds ! j ! ((i + len div 2) mod len) =
    ((p.h ^ ((i + len div 2) mod len)) * shift) ^ pw
  using fri-commit-outcome-initial-domain-points[OF fri j-le sidx-bound] invD
  by simp
have next-len:
  length (ls ! Suc j) = length (ls ! j) div 2
  using next-fri-layer-length[OF successors[OF j-lt]] d-len by simp
have half-pos: 0 < len div 2
  using invD len-pow-suc by simp
have even-len: 2 dvd len
  using invD len-pow-suc by simp
have round: len * pw = clength * scale
proof -
  have len * pw = 2 ^ (N - j) * 2 ^ j
    using invD len-exact by simp
  also have ... = 2 ^ ((N - j) + j)
    by (simp add: power-add)
  also have ... = 2 ^ N
    using j-lt-N by simp
  also have ... = clength * scale
    using init-domain .
  finally show ?thesis .
qed
have next-idx-bound:
  i mod (len div 2) < length (ds ! Suc j)
proof -
  have length (ds ! Suc j) = length (ls ! Suc j)
  proof -
    have ls ! Suc j = map (poly (ps ! Suc j)) (ds ! Suc j)
      by (rule layers) (use j-lt len-ps len-ds len-ls in simp-all)
    then show ?thesis by simp
  qed
  then show ?thesis
    using next-len half-pos invD by simp
qed
have next-domain:
  ds ! Suc j ! (i mod (len div 2)) =
    ((p.h ^ i) * shift) ^ pw * ((p.h ^ i) * shift) ^ pw
proof -
  have suc-le: Suc j ≤ n
    using j-lt by simp
  have pw-next: pw + pw = 2 ^ Suc j
  proof -

```

```

have (2::nat) ^ j + 2 ^ j = 2 ^ Suc j
proof -
  have (2::nat) ^ Suc j = 2 * 2 ^ j
    by (simp add: power-Suc)
  also have ... = 2 ^ j + 2 ^ j
    by presburger
  finally show ?thesis by simp
qed
then show ?thesis
  using invD by simp
qed
have point-next:
  ds ! Suc j ! (i mod (len div 2)) =
    ((p.h ^ (i mod (len div 2))) * shift) ^ (2 ^ Suc j)
  using fri-commit-outcome-initial-domain-points[OF fri suc-le next-idx-bound]
    invD next-len
  by simp
have fold-domain:
  ((p.h ^ i) * shift) ^ pw * ((p.h ^ i) * shift) ^ pw =
    ((p.h ^ (i mod (len div 2))) * shift) ^ (pw + pw)
  by (rule fri-next-domain-round[OF half-pos even-len round])
show ?thesis
  using point-next fold-domain pw-next by simp
qed
have step-eq:
  m a = fri-verifier-layer (challenges ! j) (roots ! j) i x len pw
  using receive-query-commits-zip-nth-apply[
    OF j-lt[folded len-challenges] j-lt[folded len-roots],
    of i x len pw]
  m-eq a-eq
  by simp
have created-j:
  p.created-tree (ls ! j) (ms ! j) s6
  by (rule created) (use j-lt len-ls len-ms in simp-all)
show None ∉ dom (dist (execute (m a) st))
  unfolding step-eq
proof (rule honest-fri-layer-decommitment-step-created-next-value-domain(1)[
  where rest=fri-decommitment-transcript i (drop (Suc j) ?bl) @ rest])
  show p.created-tree (ls ! j) (ms ! j) s6
    using created-j .
  show len = length (ls ! j)
    using invD by simp
  show length (ls ! j) = 2 ^ K
    using len-pow-K .
  show i < len
    using i-bound .
  show x = ls ! j ! i
    using invD by simp
  show s6 ≤ st

```

```

    using invD by simp
show roots ! j = value (ms ! j)
  by (rule roots-aligned[OF j-lt])
show st =
  st(PTranscript :=
    fri-layer-decommitment-transcript (ls ! j) (ms ! j) i len @
    fri-decommitment-transcript i (drop (Suc j) ?bl) @ rest)
  using tr-layer .
show p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
  (ps ! Suc j, ds ! Suc j, ls ! Suc j)
  by (rule successors[OF j-lt])
show ls ! j = map (poly (ps ! j)) (ds ! j)
  using l-eval .
show length (ds ! j) = length (ls ! j)
  using d-len .
show 2 dvd len
  using even-len .
show len * pw = clength * scale
  using round .
show ds ! j ! i = (p.h ^ i * shift) ^ pw
  using d-i .
show ds ! j ! ((i + len div 2) mod len) =
  (p.h ^ ((i + len div 2) mod len) * shift) ^ pw
  using d-sib .
show i mod (len div 2) < length (ds ! Suc j)
  using next-idx-bound .
show ds ! Suc j ! (i mod (len div 2)) =
  (p.h ^ i * shift) ^ pw * (p.h ^ i * shift) ^ pw
  using next-domain .
qed
qed
have nf-map:
  None ∉ dom (dist (execute
    (mfold (idx, x0, len0, 1) (map (λm. m) ?fl)) start))
proof (rule no-failure-mfold-map-indexI[
  where I=I and xs=?fl and F=λm. m, OF init-I])
show ∧j m a st. j < length ?fl ⇒
  m = ?fl ! j ⇒ I j a st ⇒
  None ∉ dom (dist (execute (m a) st))
  by (rule step-nf)
next
fix j m a st y t
assume j-lt-fl: j < length ?fl
  and m-eq: m = ?fl ! j
  and inv: I j a st
  and out: Some (y, t) ∈ set-dist (execute (m a) st)
have j-lt: j < n
  using j-lt-fl fl-len by presburger
obtain i x len pw where a-eq: a = (i, x, len, pw)

```

```

  by (cases a) auto
have invD:
  i = idx mod length (ls ! j)
  x = ls ! j ! i
  len = length (ls ! j)
  pw = 2 ^ j
  PTranscript st = fri-decommitment-transcript i (drop j ?bl) @ rest
  s6 ≤ st
  using inv unfolding I-def a-eq by simp-all
have j-le: j ≤ n
  using j-lt by simp
have len-exact:
  length (ls ! j) = 2 ^ (N - j)
  by (rule fri-commit-outcome-initial-layer-length-power[
    OF fri layers init-len n-le-N j-le])
have j-lt-N: j < N
  using j-lt n-le-N by linarith
have diff-suc: N - j = Suc (N - Suc j)
  using j-lt-N by simp
have len-pow-suc:
  length (ls ! j) = 2 ^ Suc (N - Suc j)
  using len-exact diff-suc by simp
have len-pos: 0 < length (ls ! j)
  using len-pow-suc by simp
have i-bound: i < len
  using invD len-pos by simp
have drop-eq:
  drop j ?bl = (ls ! j, ms ! j) # drop (Suc j) ?bl
  by (rule butlast-zip-suc-drop-nth[OF len-ls len-ms j-lt])
have tr-layer:
  st =
    st(PTranscript :=
      fri-layer-decommitment-transcript (ls ! j) (ms ! j) i len @
      fri-decommitment-transcript i (drop (Suc j) ?bl) @ rest)
  using invD drop-eq unfolding fri-decommitment-transcript.simps
  by simp
obtain K where len-pow-K: length (ls ! j) = 2 ^ K
  using len-pow-suc by blast
have l-eval:
  ls ! j = map (poly (ps ! j)) (ds ! j)
  by (rule layers) (use j-lt len-ps len-ds len-ls in simp-all)
have d-len: length (ds ! j) = length (ls ! j)
  using l-eval by simp
have i-d-bound: i < length (ds ! j)
  using i-bound invD d-len by simp
have d-i:
  ds ! j ! i = ((p.h ^ i) * shift) ^ pw
  using fri-commit-outcome-initial-domain-points[OF fri j-le i-d-bound] invD by simp
have sidx-bound:

```

```

  (i + len div 2) mod len < length (ds ! j)
  using len-pos invD d-len by simp
have d-sib:
  ds ! j ! ((i + len div 2) mod len) =
    ((p.h ^ ((i + len div 2) mod len)) * shift) ^ pw
  using fri-commit-outcome-initial-domain-points[OF fri j-le sidx-bound] invD
  by simp
have next-len:
  length (ls ! Suc j) = length (ls ! j) div 2
  using next-fri-layer-length[OF successors[OF j-lt]] d-len by simp
have half-pos: 0 < len div 2
  using invD len-pow-suc by simp
have even-len: 2 dvd len
  using invD len-pow-suc by simp
have round: len * pw = clength * scale
proof -
  have len * pw = 2 ^ (N - j) * 2 ^ j
    using invD len-exact by simp
  also have ... = 2 ^ ((N - j) + j)
    by (simp add: power-add)
  also have ... = 2 ^ N
    using j-lt-N by simp
  also have ... = clength * scale
    using init-domain .
  finally show ?thesis .
qed
have next-idx-bound:
  i mod (len div 2) < length (ds ! Suc j)
proof -
  have length (ds ! Suc j) = length (ls ! Suc j)
  proof -
    have ls ! Suc j = map (poly (ps ! Suc j)) (ds ! Suc j)
      by (rule layers) (use j-lt len-ps len-ds len-ls in simp-all)
    then show ?thesis by simp
  qed
  then show ?thesis
    using next-len half-pos invD by simp
qed
have next-domain:
  ds ! Suc j ! (i mod (len div 2)) =
    ((p.h ^ i) * shift) ^ pw * ((p.h ^ i) * shift) ^ pw
proof -
  have suc-le: Suc j ≤ n
    using j-lt by simp
  have pw-next: pw + pw = 2 ^ Suc j
  proof -
    have (2::nat) ^ j + 2 ^ j = 2 ^ Suc j
    proof -
      have (2::nat) ^ Suc j = 2 * 2 ^ j

```

```

    by (simp add: power-Suc)
  also have ... =  $2^j + 2^j$ 
    by presburger
  finally show ?thesis by simp
qed
then show ?thesis
  using invD by simp
qed
have point-next:
  ds ! Suc j ! (i mod (len div 2)) =
    ((p.h ^ (i mod (len div 2))) * shift) ^ (2 ^ Suc j)
  using fri-commit-outcome-initial-domain-points[OF fri suc-le next-idx-bound]
    invD next-len
  by simp
have fold-domain:
  ((p.h ^ i) * shift) ^ pw * ((p.h ^ i) * shift) ^ pw =
    ((p.h ^ (i mod (len div 2))) * shift) ^ (pw + pw)
  by (rule fri-next-domain-round[OF half-pos even-len round])
show ?thesis
  using point-next fold-domain pw-next by simp
qed
have step-eq:
  m a = fri-verifier-layer (challenges ! j) (roots ! j) i x len pw
  using receive-query-commits-zip-nth-apply[
    OF j-lt[folded len-challenges] j-lt[folded len-roots],
    of i x len pw]
  m-eq a-eq
  by simp
have created-j:
  p.created-tree (ls ! j) (ms ! j) s6
  by (rule created) (use j-lt len-ls len-ms in simp-all)
have out':
  Some (y, t) ∈
    set-dist (execute
      (fri-verifier-layer (challenges ! j) (roots ! j) i x len pw) st)
  using out unfolding step-eq .
have step-res:
  y =
    (i mod (len div 2), ls ! Suc j ! (i mod (len div 2)),
    len div 2, pw + pw) ∧
  PTranscript t = fri-decommitment-transcript i (drop (Suc j) ?bl) @ rest ∧
  st ≤ t
proof (rule honest-fri-layer-decommitment-step-created-next-value-domain(2)[
  where rest=fri-decommitment-transcript i (drop (Suc j) ?bl) @ rest])
show p.created-tree (ls ! j) (ms ! j) s6
  using created-j .
show len = length (ls ! j)
  using invD by simp
show length (ls ! j) =  $2^K$ 

```

```

    using len-pow-K .
show i < len
    using i-bound .
show x = ls ! j ! i
    using invD by simp
show s6 ≤ st
    using invD by simp
show roots ! j = value (ms ! j)
    by (rule roots-aligned[OF j-lt])
show st =
    st(PTranscript :=
        fri-layer-decommitment-transcript (ls ! j) (ms ! j) i len @
        fri-decommitment-transcript i (drop (Suc j) ?bl) @ rest)
    using tr-layer .
show p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
    (ps ! Suc j, ds ! Suc j, ls ! Suc j)
    by (rule successors[OF j-lt])
show ls ! j = map (poly (ps ! j)) (ds ! j)
    using l-eval .
show length (ds ! j) = length (ls ! j)
    using d-len .
show 2 dvd len
    using even-len .
show len * pw = clength * scale
    using round .
show ds ! j ! i = (p.h ^ i * shift) ^ pw
    using d-i .
show ds ! j ! ((i + len div 2) mod len) =
    (p.h ^ ((i + len div 2) mod len) * shift) ^ pw
    using d-sib .
show i mod (len div 2) < length (ds ! Suc j)
    using next-idx-bound .
show ds ! Suc j ! (i mod (len div 2)) =
    (p.h ^ i * shift) ^ pw * (p.h ^ i * shift) ^ pw
    using next-domain .
show Some (y, t) ∈
    set-dist (execute
        (fri-verifier-layer (challenges ! j) (roots ! j) i x len pw) st)
    using out' .
qed
have i-next:
    i mod (len div 2) = idx mod length (ls ! Suc j)
proof -
    have mod-half:
        (idx mod length (ls ! j)) mod (length (ls ! j) div 2) =
            idx mod (length (ls ! j) div 2)
        by (rule mod-mod-half-power[OF len-pow-suc])
    show ?thesis
        using invD next-len mod-half by simp

```



```

qed
have pw-next: pw + pw = 2 ^ Suc j
proof -
  have (2::nat) ^ j + 2 ^ j = 2 ^ Suc j
  proof -
    have (2::nat) ^ Suc j = 2 * 2 ^ j
      by (simp add: power-Suc)
    also have ... = 2 ^ j + 2 ^ j
      by presburger
    finally show ?thesis by simp
  qed
then show ?thesis
  using invD by simp
qed
have tr-next:
  PTranscript t =
    fri-decommitment-transcript (i mod (len div 2))
    (drop (Suc j) ?bl) @ rest
proof -
  have bridge:
    fri-decommitment-transcript (idx mod length (ls ! j))
      (drop (Suc j) ?bl) =
    fri-decommitment-transcript
      (idx mod (length (ls ! j) div 2))
      (drop (Suc j) ?bl)
  by (rule fri-decommitment-transcript-drop-half-index[
    OF len-ls len-ms j-lt len-pow-suc next-len])
  have mod-half:
    (idx mod length (ls ! j)) mod (length (ls ! j) div 2) =
    idx mod (length (ls ! j) div 2)
  by (rule mod-mod-half-power[OF len-pow-suc])
  have bridge':
    fri-decommitment-transcript (idx mod (length (ls ! j) div 2))
      (drop (Suc j) ?bl) =
    fri-decommitment-transcript
      (idx mod length (ls ! j) mod (length (ls ! j) div 2))
      (drop (Suc j) ?bl)
  using mod-half by simp
show ?thesis
  using step-res bridge bridge' invD by simp
qed
have ext-next: s6 ≤ t
  using invD step-res by (meson p.hash-ext-trans)
have len-half: len div 2 = length (ls ! Suc j)
  using invD next-len by simp
show I (Suc j) y t
  using step-res i-next pw-next tr-next ext-next next-len invD(3) len-half
  unfolding I-def
  by (simp add: split: prod.splits)

```

```

qed
show ?thesis
  using nf-map by simp
qed

lemma honest-fri-decommitments-mfold-outcome:
  assumes fri:
    fri-commit-outcome n cp p.eval-domain l0 m0 ps ds ls ms s5 s6
    and len-ps: length ps = Suc n
    and len-ds: length ds = Suc n
    and len-ls: length ls = Suc n
    and len-ms: length ms = Suc n
    and layers:
       $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$ 
       $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$ 
    and created:
       $\bigwedge i. i < \text{length } ls \implies i < \text{length } ms \implies$ 
       $p.\text{created-tree } (ls ! i) (ms ! i) s6$ 
    and init-len: length l0 =  $2^N$ 
    and init-domain:  $2^N = \text{clength} * \text{scale}$ 
    and n-le-N:  $n \leq N$ 
    and len-roots: length roots = n
    and len-challenges: length challenges = n
    and roots-aligned:  $\bigwedge j. j < n \implies \text{roots} ! j = \text{value } (ms ! j)$ 
    and successors:
       $\bigwedge j. j < n \implies$ 
       $p.\text{next-fri-layer } (ps ! j) (ds ! j) (\text{challenges} ! j) =$ 
       $(ps ! \text{Suc } j, ds ! \text{Suc } j, ls ! \text{Suc } j)$ 
    and idx-bound:  $\text{idx} < \text{length } (ls ! 0)$ 
    and x0-eq:  $x0 = ls ! 0 ! \text{idx}$ 
    and len0-eq:  $\text{len0} = \text{length } (ls ! 0)$ 
    and start-tr:
       $P\text{Transcript start} = \text{fri-decommitment-transcript idx (butlast (zip ls ms)) @ rest}$ 
    and start-ext:  $s6 \leq \text{start}$ 
    and outcome:
      Some ((i, x, len, pw), t)  $\in$ 
      set-dist (execute
        (mfold (idx, x0, len0, 1)
          (v.receive-query-commits (zip challenges roots))) start)
  shows
    i = idx mod length (ls ! n)  $\wedge$ 
    x = ls ! n ! i  $\wedge$ 
    len = length (ls ! n)  $\wedge$ 
    pw =  $2^n$   $\wedge$ 
    PTranscript t =
      fri-decommitment-transcript i (drop n (butlast (zip ls ms))) @ rest  $\wedge$ 
    PState t =
      foldl concat (PState start)
        (fri-decommitment-transcript idx (butlast (zip ls ms)))  $\wedge$ 

```

```

    s6 ≤ t
proof -
  let ?fl =
    v.receive-query-commits (zip challenges roots) ::
      ((nat × 'f × nat × nat) ⇒
        (nat × 'f × nat × nat, 'f protocol-channel) state-monad) list
  let ?bl = butlast (zip ls ms)
  define I where
    I = (λ(j::nat) (a::nat × 'f × nat × nat)
      (s::'f protocol-channel).
      (case a of (i, x, len, pw) ⇒
        i = idx mod length (ls ! j) ∧
        x = ls ! j ! i ∧
        len = length (ls ! j) ∧
        pw = 2 ^ j ∧
        PTranscript s = fri-decommitment-transcript i (drop j ?bl) @ rest ∧
        (∃ consumed.
          fri-decommitment-transcript idx ?bl =
            consumed @ fri-decommitment-transcript i (drop j ?bl) ∧
            PState s = foldl concat (PState start) consumed) ∧
        s6 ≤ s))
  have fl-len: length ?fl = n
    using len-roots len-challenges
    unfolding v.receive-query-commits-def by simp
  have init-I: I 0 (idx, x0, len0, 1) start
    unfolding I-def
    using idx-bound x0-eq len0-eq start-tr start-ext by simp
  have outcome-map:
    Some ((i, x, len, pw), t) ∈
      set-dist (execute
        (mfold (idx, x0, len0, 1) (map (λm. m) ?fl)) start)
    using outcome by simp
  have final-I: I (length ?fl) (i, x, len, pw) t
  proof (rule mfold-map-index-outcome-invariantI[
    where I=I and xs=?fl and F=λm. m, OF init-I - outcome-map])
    fix j m a st y t'
    assume j-lt-fl: j < length ?fl
    and m-eq: m = ?fl ! j
    and inv: I j a st
    and out: Some (y, t') ∈ set-dist (execute (m a) st)
    have j-lt: j < n
      using j-lt-fl fl-len by presburger
    obtain ii xx lenn pww where a-eq: a = (ii, xx, lenn, pww)
      by (cases a) auto
    have invD:
      ii = idx mod length (ls ! j)
      xx = ls ! j ! ii
      lenn = length (ls ! j)
      pww = 2 ^ j

```

```

PTranscript st = fri-decommitment-transcript ii (drop j ?bl) @ rest
s6 ≤ st
using inv unfolding I-def a-eq by simp-all
have consumed-ex:
  ∃ consumed.
    fri-decommitment-transcript idx ?bl =
      consumed @ fri-decommitment-transcript ii (drop j ?bl) ∧
      PState st = foldl concat (PState start) consumed
using inv invD(1) unfolding I-def a-eq by simp
from consumed-ex obtain consumed where
  consumed-total:
    fri-decommitment-transcript idx ?bl =
      consumed @ fri-decommitment-transcript ii (drop j ?bl)
and consumed-state:
  PState st = foldl concat (PState start) consumed
by blast
have j-le: j ≤ n
using j-lt by simp
have len-exact:
  length (ls ! j) =  $2^{(N - j)}$ 
by (rule fri-commit-outcome-initial-layer-length-power[
    OF fri layers init-len n-le-N j-le])
have j-lt-N: j < N
using j-lt n-le-N by linarith
have diff-suc: N - j = Suc (N - Suc j)
using j-lt-N by simp
have len-pow-suc:
  length (ls ! j) =  $2^{Suc\ (N - Suc\ j)}$ 
using len-exact diff-suc by simp
have len-pos:  $0 < \text{length}\ (ls\ !\ j)$ 
using len-pow-suc by simp
have ii-bound: ii < lenn
using invD len-pos by simp
have drop-eq:
  drop j ?bl = (ls ! j, ms ! j) # drop (Suc j) ?bl
by (rule butlast-zip-suc-drop-nth[OF len-ls len-ms j-lt])
have tr-layer:
  st =
    st(PTranscript :=
      fri-layer-decommitment-transcript (ls ! j) (ms ! j) ii lenn @
      fri-decommitment-transcript ii (drop (Suc j) ?bl) @ rest)
using invD drop-eq unfolding fri-decommitment-transcript.simps
by simp
obtain K where len-pow-K: length (ls ! j) =  $2^K$ 
using len-pow-suc by blast
have l-eval:
  ls ! j = map (poly (ps ! j)) (ds ! j)
by (rule layers) (use j-lt len-ps len-ds len-ls in simp-all)
have d-len: length (ds ! j) = length (ls ! j)

```

```

    using l-eval by simp
have ii-d-bound:  $ii < \text{length } (ds ! j)$ 
    using ii-bound invD d-len by simp
have d-i:
     $ds ! j ! ii = ((p.h \wedge ii) * \text{shift}) \wedge pww$ 
    using fri-commit-outcome-initial-domain-points[OF fri j-le ii-d-bound] invD by simp
have sidx-bound:
     $(ii + \text{lenn div } 2) \bmod \text{lenn} < \text{length } (ds ! j)$ 
    using len-pos invD d-len by simp
have d-sib:
     $ds ! j ! ((ii + \text{lenn div } 2) \bmod \text{lenn}) =$ 
     $((p.h \wedge ((ii + \text{lenn div } 2) \bmod \text{lenn})) * \text{shift}) \wedge pww$ 
    using fri-commit-outcome-initial-domain-points[OF fri j-le sidx-bound] invD
    by simp
have next-len:
     $\text{length } (ls ! \text{Suc } j) = \text{length } (ls ! j) \text{ div } 2$ 
    using next-fri-layer-length[OF successors[OF j-lt]] d-len by simp
have half-pos:  $0 < \text{lenn div } 2$ 
    using invD len-pow-suc by simp
have even-len:  $2 \text{ dvd } \text{lenn}$ 
    using invD len-pow-suc by simp
have round:  $\text{lenn} * pww = \text{clength} * \text{scale}$ 
proof -
    have  $\text{lenn} * pww = 2^{(N - j)} * 2^j$ 
        using invD len-exact by simp
    also have  $\dots = 2^{((N - j) + j)}$ 
        by (simp add: power-add)
    also have  $\dots = 2^N$ 
        using j-lt-N by simp
    also have  $\dots = \text{clength} * \text{scale}$ 
        using init-domain .
    finally show ?thesis .
qed
have next-idx-bound:
     $ii \bmod (\text{lenn div } 2) < \text{length } (ds ! \text{Suc } j)$ 
proof -
    have  $\text{length } (ds ! \text{Suc } j) = \text{length } (ls ! \text{Suc } j)$ 
    proof -
        have  $ls ! \text{Suc } j = \text{map } (\text{poly } (ps ! \text{Suc } j)) (ds ! \text{Suc } j)$ 
        by (rule layers) (use j-lt len-ps len-ds len-ls in simp-all)
        then show ?thesis by simp
    qed
    then show ?thesis
        using next-len half-pos invD by simp
qed
have next-domain:
     $ds ! \text{Suc } j ! (ii \bmod (\text{lenn div } 2)) =$ 
     $((p.h \wedge ii) * \text{shift}) \wedge pww * ((p.h \wedge ii) * \text{shift}) \wedge pww$ 
proof -

```

```

have suc-le: Suc j ≤ n
  using j-lt by simp
have pw-next: pww + pww = 2 ^ Suc j
proof -
  have (2::nat) ^ j + 2 ^ j = 2 ^ Suc j
  proof -
    have (2::nat) ^ Suc j = 2 * 2 ^ j
      by (simp add: power-Suc)
    also have ... = 2 ^ j + 2 ^ j
      by presburger
    finally show ?thesis by simp
qed
then show ?thesis
  using invD by simp
qed
have point-next:
  ds ! Suc j ! (ii mod (lenn div 2)) =
    ((p.h ^ (ii mod (lenn div 2))) * shift) ^ (2 ^ Suc j)
  using fri-commit-outcome-initial-domain-points[OF fri suc-le next-idx-bound]
    invD next-len
  by simp
have fold-domain:
  ((p.h ^ ii) * shift) ^ pww * ((p.h ^ ii) * shift) ^ pww =
    ((p.h ^ (ii mod (lenn div 2))) * shift) ^ (pww + pww)
  by (rule fri-next-domain-round[OF half-pos even-len round])
show ?thesis
  using point-next fold-domain pw-next by simp
qed
have step-eq:
  m a = fri-verifier-layer (challenges ! j) (roots ! j) ii xx lenn pww
  using receive-query-commits-zip-nth-apply[
    OF j-lt[folded len-challenges] j-lt[folded len-roots],
    of ii xx lenn pww]
  m-eq a-eq
  by simp
have created-j:
  p.created-tree (ls ! j) (ms ! j) s6
  by (rule created) (use j-lt len-ls len-ms in simp-all)
have out':
  Some (y, t') ∈
    set-dist (execute
      (fri-verifier-layer (challenges ! j) (roots ! j) ii xx lenn pww) st)
  using out unfolding step-eq .
have step-res:
  y =
    (ii mod (lenn div 2), ls ! Suc j ! (ii mod (lenn div 2)),
      lenn div 2, pww + pww) ∧
  PTranscript t' = fri-decommitment-transcript ii (drop (Suc j) ?bl) @ rest ∧
  st ≤ t' ∧

```

```

PState t' =
  foldl concat (PState st)
    (fri-layer-decommitment-transcript (ls ! j) (ms ! j) ii lenn)
proof (rule honest-fri-layer-decommitment-step-created-next-value-domain-state[
  where rest=fri-decommitment-transcript ii (drop (Suc j) ?bl) @ rest])
show p.created-tree (ls ! j) (ms ! j) s6
  using created-j .
show lenn = length (ls ! j)
  using invD by simp
show length (ls ! j) = 2 ^ K
  using len-pow-K .
show ii < lenn
  using ii-bound .
show xx = ls ! j ! ii
  using invD by simp
show s6 ≤ st
  using invD by simp
show roots ! j = value (ms ! j)
  by (rule roots-aligned[OF j-lt])
show st =
  st(PTranscript :=
    fri-layer-decommitment-transcript (ls ! j) (ms ! j) ii lenn @
    fri-decommitment-transcript ii (drop (Suc j) ?bl) @ rest)
  using tr-layer .
show p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
  (ps ! Suc j, ds ! Suc j, ls ! Suc j)
  by (rule successors[OF j-lt])
show ls ! j = map (poly (ps ! j)) (ds ! j)
  using l-eval .
show length (ds ! j) = length (ls ! j)
  using d-len .
show 2 dvd lenn
  using even-len .
show lenn * pww = clength * scale
  using round .
show ds ! j ! ii = (p.h ^ ii * shift) ^ pww
  using d-i .
show ds ! j ! ((ii + lenn div 2) mod lenn) =
  (p.h ^ ((ii + lenn div 2) mod lenn) * shift) ^ pww
  using d-sib .
show ii mod (lenn div 2) < length (ds ! Suc j)
  using next-idx-bound .
show ds ! Suc j ! (ii mod (lenn div 2)) =
  (p.h ^ ii * shift) ^ pww * (p.h ^ ii * shift) ^ pww
  using next-domain .
show Some (y, t') ∈
  set-dist (execute
    (fri-verifier-layer (challenges ! j) (roots ! j) ii xx lenn pww) st)
  using out' .

```

```

qed
have i-next:
  ii mod (lenn div 2) = idx mod length (ls ! Suc j)
proof -
  have mod-half:
    (idx mod length (ls ! j)) mod (length (ls ! j) div 2) =
      idx mod (length (ls ! j) div 2)
    by (rule mod-mod-half-power[OF len-pow-suc])
  show ?thesis
    using invD next-len mod-half by simp
qed
have pw-next: pww + pww = 2 ^ Suc j
proof -
  have (2::nat) ^ j + 2 ^ j = 2 ^ Suc j
  proof -
    have (2::nat) ^ Suc j = 2 * 2 ^ j
      by (simp add: power-Suc)
    also have ... = 2 ^ j + 2 ^ j
      by presburger
    finally show ?thesis by simp
  qed
then show ?thesis
  using invD by simp
qed
have tail-bridge:
  fri-decommitment-transcript ii (drop (Suc j) ?bl) =
    fri-decommitment-transcript (ii mod (lenn div 2))
      (drop (Suc j) ?bl)
proof -
  have bridge:
    fri-decommitment-transcript (idx mod length (ls ! j))
      (drop (Suc j) ?bl) =
      fri-decommitment-transcript
        (idx mod (length (ls ! j) div 2))
          (drop (Suc j) ?bl)
    by (rule fri-decommitment-transcript-drop-half-index[
      OF len-ls len-ms j-lt len-pow-suc next-len])
  have mod-half:
    (idx mod length (ls ! j)) mod (length (ls ! j) div 2) =
      idx mod (length (ls ! j) div 2)
    by (rule mod-mod-half-power[OF len-pow-suc])
  have bridge':
    fri-decommitment-transcript (idx mod (length (ls ! j) div 2))
      (drop (Suc j) ?bl) =
      fri-decommitment-transcript
        (idx mod length (ls ! j) mod (length (ls ! j) div 2))
          (drop (Suc j) ?bl)
    using mod-half by simp
  show ?thesis

```



```

    using bridge bridge' invD by simp
qed
have tr-next:
  PTranscript t' =
    fri-decommitment-transcript (ii mod (lenn div 2))
      (drop (Suc j) ?bl) @ rest
    using step-res tail-bridge by simp
have ext-next: s6 ≤ t'
  using invD step-res by (meson p.hash-ext-trans)
have len-half: lenn div 2 = length (ls ! Suc j)
  using invD next-len by simp
have consumed-next:
  fri-decommitment-transcript idx ?bl =
    (consumed @
      fri-layer-decommitment-transcript (ls ! j) (ms ! j) ii lenn) @
    fri-decommitment-transcript (ii mod (lenn div 2))
      (drop (Suc j) ?bl)
proof -
  have tail-split:
    fri-decommitment-transcript ii (drop j ?bl) =
      fri-layer-decommitment-transcript (ls ! j) (ms ! j) ii lenn @
      fri-decommitment-transcript ii (drop (Suc j) ?bl)
    using drop-eq invD unfolding fri-decommitment-transcript.simps by simp
  show ?thesis
    using consumed-total tail-split tail-bridge by simp
qed
have state-next:
  PState t' =
    foldl concat (PState start)
      (consumed @
        fri-layer-decommitment-transcript (ls ! j) (ms ! j) ii lenn)
    using step-res consumed-state by simp
show I (Suc j) y t'
  using step-res i-next pw-next tr-next ext-next next-len invD(3) len-half
    consumed-next state-next
  unfolding I-def
  by (simp add: split: prod.splits)
qed
show ?thesis
proof -
  have bl-len: length ?bl = n
    using butlast-zip-suc-lengths[OF len-ls len-ms] len-ls len-ms by simp
  obtain consumed where
    consumed-total:
      fri-decommitment-transcript idx ?bl =
        consumed @ fri-decommitment-transcript i (drop (length ?fl) ?bl)
  and consumed-state:
    PState t = foldl concat (PState start) consumed
  using final-I unfolding I-def by auto

```

```

have drop-empty: drop (length ?fl) ?bl = []
using fl-len bl-len by simp
have consumed-eq: consumed = fri-decommitment-transcript idx ?bl
using consumed-total drop-empty by simp
show ?thesis
using final-I fl-len consumed-state consumed-eq unfolding I-def by simp
qed
qed

```

lemma honest-query-prefix-fri-mfold-no-failure:

```

assumes htv: honest-trace-valid
and create-f:
  Some (f-merkle, s1) ∈
    set-dist (execute (p.create p.f-eval) init-state)
and send-f:
  Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and alphas:
  Some (as, s3) ∈
    set-dist (execute
      (mmap (replicate (length spec)
        (do {
          a ← p.receive-alpha-challenge;
          p.send a;
          return a
        }))) s2)
and cp'-def: cp' = p.cp as p.f-powers
and send-degree:
  Some ((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp'))) s3)
and create-cp:
  Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
and fri:
  Some ((ps, ds, ls, ms), s6) ∈
    set-dist (execute
      (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
and len-ps: length ps = Suc nrounds
and len-ds: length ds = Suc nrounds
and len-ls: length ls = Suc nrounds
and len-ms: length ms = Suc nrounds
and layers:
   $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$ 
   $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$ 
and created:
   $\bigwedge i. i < \text{length } ls \implies i < \text{length } ms \implies$ 
   $p.\text{created-tree } (ls ! i) (ms ! i) s6$ 
and init-len: length (p.cp-eval as) =  $2^N$ 
and n-le-N: nrounds  $\leq N$ 
and final-send:
  Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
and random-idx:

```

Some $(idx, s7) \in \text{set-dist } (\text{execute } p.\text{receive-query-index-challenge } s\text{-final})$
and $idx\text{-def}$: $idx' = p.\text{index } (to\text{-nat } idx)$
and $query\text{-decommit}$:
 Some $(qouts, s8) \in$
 $\text{set-dist } (\text{execute } (mmap (p.\text{decommit-on-query } idx' f\text{-merkle})) s7)$
and $fri\text{-decommit}$:
 Some $(fri\text{-idx}, s9) \in$
 $\text{set-dist } (\text{execute } (mfold idx' (p.\text{decommit-on-fri-layers } (butlast (zip ls ms)))) s8)$
and $prover\text{-state-eq}$: $prover\text{-state} = s9$
and $replay$: $replay\text{-state} = \text{verifier-replay-state } prover\text{-state}$
and $prefix\text{-out}$:
 Some $((fr, as', dg, fl, final, idrv, fv), t) \in$
 $\text{set-dist } (\text{execute } (do \{$
 $\quad fr \leftarrow p.\text{read};$
 $\quad as' \leftarrow mmap (\text{replicate } (length \text{ spec}))$
 $\quad (do \{$
 $\quad \quad a0 \leftarrow p.\text{receive-alpha-challenge};$
 $\quad \quad let a0' = a0;$
 $\quad \quad a1 \leftarrow p.\text{read};$
 $\quad \quad let a1' = a1;$
 $\quad \quad assert (a0' = a1');$
 $\quad \quad return a1'$
 $\quad \quad \}));$
 $\quad dg \leftarrow p.\text{read};$
 $\quad assert (to\text{-nat } dg \leq p.\text{maxDegree});$
 $\quad fl \leftarrow ntimes \ v.\text{receive-fri-commits } nrounds;$
 $\quad final \leftarrow p.\text{read};$
 $\quad idrv \leftarrow p.\text{receive-query-index-challenge};$
 $\quad let idrv' = p.\text{index } (to\text{-nat } idrv);$
 $\quad fv \leftarrow mmap (v.\text{check-decommit-on-query } fr \ idrv');$
 $\quad return (fr, as', dg, fl, final, idrv, fv)$
 $\quad \}) \text{ replay-state})$
and $fl\text{-eq}$: $fl = zip \text{ challenges roots}$
and $len\text{-roots}$: $length \text{ roots} = nrounds$
and $len\text{-challenges}$: $length \text{ challenges} = nrounds$
and $roots\text{-aligned}$: $\bigwedge j. j < nrounds \implies roots ! j = value (ms ! j)$
and $successors$:
 $\bigwedge j. j < nrounds \implies$
 $p.\text{next-fri-layer } (ps ! j) (ds ! j) (\text{challenges} ! j) =$
 $(ps ! Suc \ j, ds ! Suc \ j, ls ! Suc \ j)$
shows
 $None \notin dom \ (dist \ (\text{execute } (mfold$
 $\quad (idx', v.\text{cp-eval } as' \ fv \ (p.h \wedge idx' * shift), clength * scale, 1)$
 $\quad (v.\text{receive-query-commits } fl)) \ t))$
proof –
have $ls0$: $ls ! 0 = p.\text{cp-eval } as$

```

    using fri-commit-initial-heads(3)[OF fri] .
have idx-bound:  $idx' < \text{length } (ls ! 0)$ 
    using idx-def index-less-domain cp-eval-length ls0 by simp
have x0-eq:
    v.cp-eval as' fv ( $p.h \wedge idx' * \text{shift}$ ) =  $ls ! 0 ! idx'$ 
proof -
    have value-eq:
        v.cp-eval as' fv ( $p.h \wedge idx' * \text{shift}$ ) =
            poly (p.cp as p.f-powers) ( $p.h \wedge idx' * \text{shift}$ )
        by (rule honest-query-prefix-initial-fri-value[
            OF htv create-f send-f alphas cp'-def send-degree create-cp fri
            final-send random-idx idx-def query-decommit fri-decommit
            prover-state-eq replay prefix-out])
    have ls0-value:
         $ls ! 0 ! idx' =$ 
        poly (p.cp as p.f-powers) ( $p.h \wedge idx' * \text{shift}$ )
        using cp-eval-index-value[of as to-nat idx] idx-def ls0 by simp
    show ?thesis
        using value-eq ls0-value by simp
qed
have len0-eq:  $\text{clength} * \text{scale} = \text{length } (ls ! 0)$ 
    using ls0 cp-eval-length by simp
have init-domain:  $2^N = \text{clength} * \text{scale}$ 
    using init-len cp-eval-length by simp
have fri-shape:
    fri-commit-outcome nrounds cp' p.eval-domain (p.cp-eval as) cp-merkle
    ps ds ls ms s5 s6
    by (rule fri-commit-outcomeI[OF fri])
have tr-t:
    PTranscript t = fri-decommitment-transcript idx' (butlast (zip ls ms))
    by (rule honest-query-prefix-leaves-fri-decommitment-transcript[
        OF htv create-f send-f alphas cp'-def send-degree create-cp fri
        final-send random-idx idx-def query-decommit fri-decommit
        prover-state-eq replay prefix-out])
have replay-t:  $\text{replay-state} \leq t$ 
    using honest-root-alpha-degree-fri-final-query-from-prover-outcome[
        OF htv create-f send-f alphas cp'-def send-degree create-cp fri
        final-send random-idx idx-def query-decommit fri-decommit
        prover-state-eq replay prefix-out]
    by blast
have s6-prover:  $s6 \leq \text{prover-state}$ 
    by (rule prover-after-fri-hash-extends[
        OF final-send random-idx query-decommit fri-decommit prover-state-eq])
have s6-replay:  $s6 \leq \text{replay-state}$ 
    using s6-prover replay
    unfolding verifier-replay-state-def less-eq-hash-ext-def less-eq-fmap-def
    by simp
have s6-t:  $s6 \leq t$ 
    using s6-replay replay-t by (meson p.hash-ext-trans)

```

have *tr-t-empty*:
PTranscript t = fri-decommitment-transcript idx' (butlast (zip ls ms)) @ []
using *tr-t* **by** *simp*
show *?thesis*
unfolding *fl-eq*
by (*rule honest-fri-decommitments-mfold-no-failure*[
OF fri-shape len-ps len-ds len-ls len-ms layers created init-len init-domain n-le-N
len-roots len-challenges roots-aligned successors idx-bound x0-eq
len0-eq tr-t-empty s6-t])
qed

lemma *honest-query-prefix-fri-mfold-outcome*:
assumes *htv: honest-trace-valid*
and *create-f*:
Some (f-merkle, s1) ∈
set-dist (execute (p.create p.f-eval) init-state)
and *send-f*:
Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and *alphas*:
Some (as, s3) ∈
set-dist (execute
(mmap (replicate (length spec)
(do {
a ← p.receive-alpha-challenge;
p.send a;
return a
}})) s2)
and *cp'-def*: *cp' = p.cp as p.f-powers*
and *send-degree*:
Some ((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp'))) s3)
and *create-cp*:
Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
and *fri*:
Some ((ps, ds, ls, ms), s6) ∈
set-dist (execute
(p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
and *len-ps*: *length ps = Suc nrounds*
and *len-ds*: *length ds = Suc nrounds*
and *len-ls*: *length ls = Suc nrounds*
and *len-ms*: *length ms = Suc nrounds*
and *layers*:
 $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$
 $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$
and *created*:
 $\bigwedge i. i < \text{length } ls \implies i < \text{length } ms \implies$
 $p.\text{created-tree } (ls ! i) (ms ! i) s6$
and *init-len*: *length (p.cp-eval as) = 2 ^ N*
and *n-le-N*: *nrounds ≤ N*
and *final-send*:

```

    Some ( $()$ ,  $s\text{-final}$ )  $\in$  set-dist (execute ( $p.\text{send}$  ( $\text{hd}$  ( $\text{last}$   $ls$ ))))  $s6$ )
and random-idx:
    Some ( $idx$ ,  $s7$ )  $\in$  set-dist (execute  $p.\text{receive-query-index-challenge}$   $s\text{-final}$ )
and idx-def:  $idx' = p.\text{index}$  ( $\text{to-nat}$   $idx$ )
and query-decommit:
    Some ( $qouts$ ,  $s8$ )  $\in$ 
      set-dist (execute ( $mmap$  ( $p.\text{decommit-on-query}$   $idx'$   $f\text{-merkle}$ )))  $s7$ )
and fri-decommit:
    Some ( $fri\text{-idx}$ ,  $s9$ )  $\in$ 
      set-dist (execute
        ( $mfold$   $idx'$  ( $p.\text{decommit-on-fri-layers}$  ( $\text{butlast}$  ( $\text{zip}$   $ls$   $ms$ ))))  $s8$ )
and prover-state-eq:  $\text{prover-state} = s9$ 
and replay:  $\text{replay-state} = \text{verifier-replay-state}$   $\text{prover-state}$ 
and prefix-out:
    Some ( $(fr, as', dg, fl, final, idxv, fv)$ ,  $t$ )  $\in$ 
      set-dist (execute
        (do {
           $fr \leftarrow p.\text{read}$ ;
           $as' \leftarrow mmap$  ( $\text{replicate}$  ( $\text{length}$   $spec$ )
            (do {
               $a0 \leftarrow p.\text{receive-alpha-challenge}$ ;
               $\text{let } a0' = a0$ ;
               $a1 \leftarrow p.\text{read}$ ;
               $\text{let } a1' = a1$ ;
               $\text{assert } (a0' = a1')$ ;
               $\text{return } a1'$ 
            })));
           $dg \leftarrow p.\text{read}$ ;
           $\text{assert } (\text{to-nat } dg \leq p.\text{maxDegree})$ ;
           $fl \leftarrow ntimes$   $v.\text{receive-fri-commits}$   $nrounds$ ;
           $final \leftarrow p.\text{read}$ ;
           $idxv \leftarrow p.\text{receive-query-index-challenge}$ ;
           $\text{let } idxv' = p.\text{index}$  ( $\text{to-nat}$   $idxv$ );
           $fv \leftarrow mmap$  ( $v.\text{check-decommit-on-query}$   $fr$   $idxv'$ );
           $\text{return } (fr, as', dg, fl, final, idxv, fv)$ 
        })  $\text{replay-state}$ )
and fl-eq:  $fl = \text{zip}$   $\text{challenges}$   $\text{roots}$ 
and len-roots:  $\text{length}$   $\text{roots} = nrounds$ 
and len-challenges:  $\text{length}$   $\text{challenges} = nrounds$ 
and roots-aligned:  $\bigwedge j. j < nrounds \implies \text{roots} ! j = \text{value } (ms ! j)$ 
and successors:
     $\bigwedge j. j < nrounds \implies$ 
       $p.\text{next-fri-layer } (ps ! j) (ds ! j) (\text{challenges} ! j) =$ 
       $(ps ! \text{Suc } j, ds ! \text{Suc } j, ls ! \text{Suc } j)$ 
and outcome:
    Some ( $(i, x, len, pw)$ ,  $u$ )  $\in$ 
      set-dist (execute
        ( $mfold$ 
          ( $idx'$ ,  $v.\text{cp-eval}$   $as'$   $fv$  ( $p.h \wedge idx' * \text{shift}$ ),  $\text{clength} * \text{scale}$ ,  $1$ )

```

```

      (v.receive-query-commits fl)) t)
shows
  i = idx' mod length (ls ! nrounds) ∧
  x = ls ! nrounds ! i ∧
  len = length (ls ! nrounds) ∧
  pw = 2 ^ nrounds ∧
  PTranscript u =
    fri-decommitment-transcript i (drop nrounds (butlast (zip ls ms))) ∧
    s6 ≤ u
proof -
  have ls0: ls ! 0 = p.cp-eval as
    using fri-commit-initial-heads(3)[OF fri] .
  have idx-bound: idx' < length (ls ! 0)
    using idx-def index-less-domain cp-eval-length ls0 by simp
  have x0-eq:
    v.cp-eval as' fv (p.h ^ idx' * shift) = ls ! 0 ! idx'
  proof -
    have value-eq:
      v.cp-eval as' fv (p.h ^ idx' * shift) =
        poly (p.cp as p.f-powers) (p.h ^ idx' * shift)
    by (rule honest-query-prefix-initial-fri-value[
      OF htv create-f send-f alphas cp'-def send-degree create-cp fri
      final-send random-idx idx-def query-decommit fri-decommit
      prover-state-eq replay prefix-out])
  have ls0-value:
    ls ! 0 ! idx' =
      poly (p.cp as p.f-powers) (p.h ^ idx' * shift)
    using cp-eval-index-value[of as to-nat idx] idx-def ls0 by simp
  show ?thesis
    using value-eq ls0-value by simp
qed
have len0-eq: clength * scale = length (ls ! 0)
  using ls0 cp-eval-length by simp
have init-domain: 2 ^ N = clength * scale
  using init-len cp-eval-length by simp
have fri-shape:
  fri-commit-outcome nrounds cp' p.eval-domain (p.cp-eval as) cp-merkle
  ps ds ls ms s5 s6
  by (rule fri-commit-outcomeI[OF fri])
have tr-t:
  PTranscript t = fri-decommitment-transcript idx' (butlast (zip ls ms))
  by (rule honest-query-prefix-leaves-fri-decommitment-transcript[
    OF htv create-f send-f alphas cp'-def send-degree create-cp fri
    final-send random-idx idx-def query-decommit fri-decommit
    prover-state-eq replay prefix-out])
have replay-t: replay-state ≤ t
  using honest-root-alpha-degree-fri-final-query-from-prover-outcome[
    OF htv create-f send-f alphas cp'-def send-degree create-cp fri
    final-send random-idx idx-def query-decommit fri-decommit

```

```

    prover-state-eq replay prefix-out]
  by blast
have s6-prover: s6 ≤ prover-state
  by (rule prover-after-fri-hash-extends[
    OF final-send random-idx query-decommit fri-decommit prover-state-eq])
have s6-replay: s6 ≤ replay-state
  using s6-prover replay
  unfolding verifier-replay-state-def less-eq-hash-ext-def less-eq-fmap-def
  by simp
have s6-t: s6 ≤ t
  using s6-replay replay-t by (meson p.hash-ext-trans)
have outcome-zip:
  Some ((i, x, len, pw), u) ∈
    set-dist (execute
      (mfold
        (idx', v.cp-eval as' fv (p.h ^ idx' * shift), clength * scale, 1)
        (v.receive-query-commits (zip challenges roots))) t)
  using outcome fl-eq by simp
have tr-t-empty:
  PTranscript t = fri-decommitment-transcript idx' (butlast (zip ls ms)) @ []
  using tr-t by simp
have res:
  i = idx' mod length (ls ! nrounds) ∧
  x = ls ! nrounds ! i ∧
  len = length (ls ! nrounds) ∧
  pw = 2 ^ nrounds ∧
  PTranscript u =
    fri-decommitment-transcript i (drop nrounds (butlast (zip ls ms))) @ [] ∧
  PState u =
    foldl concat (PState t)
      (fri-decommitment-transcript idx' (butlast (zip ls ms))) ∧
  s6 ≤ u
  by (rule honest-fri-decommitments-mfold-outcome[
    OF fri-shape len-ps len-ds len-ls len-ms layers created init-len init-domain n-le-N
    len-roots len-challenges roots-aligned successors idx-bound x0-eq
    len0-eq tr-t-empty s6-t outcome-zip])
show ?thesis
  using res by simp
qed

```

```

lemma honest-query-prefix-final-assert-no-failure:
  assumes htv: honest-trace-valid
  and create-f:
    Some (f-merkle, s1) ∈
      set-dist (execute (p.create p.f-eval) init-state)
  and send-f:
    Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
  and alphas:
    Some (as, s3) ∈

```



```

    set-dist (execute
      (mmap (replicate (length spec)
        (do {
          a ← p.receive-alpha-challenge;
          p.send a;
          return a
        }))) s2)
  and cp'-def: cp' = p.cp as p.f-powers
  and send-degree:
    Some (((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp')))) s3)
  and create-cp:
    Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
  and nrounds-def: nrounds = ceil-log (degree cp' + 1)
  and fri:
    Some ((ps, ds, ls, ms), s6) ∈
      set-dist (execute
        (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
  and len-ps: length ps = Suc nrounds
  and len-ds: length ds = Suc nrounds
  and len-ls: length ls = Suc nrounds
  and len-ms: length ms = Suc nrounds
  and layers:
    ∧ i. i < length ps ⇒ i < length ds ⇒ i < length ls ⇒
      ls ! i = map (poly (ps ! i)) (ds ! i)
  and created:
    ∧ i. i < length ls ⇒ i < length ms ⇒
      p.created-tree (ls ! i) (ms ! i) s6
  and init-len: length (p.cp-eval as) = 2 ^ N
  and n-le-N: nrounds ≤ N
  and final-send:
    Some (((), s-final) ∈ set-dist (execute (p.send (hd (last ls))) s6)
  and random-idx:
    Some (idx, s7) ∈ set-dist (execute p.receive-query-index-challenge s-final)
  and idx-def: idx' = p.index (to-nat idx)
  and query-decommit:
    Some (qouts, s8) ∈
      set-dist (execute (mmap (p.decommit-on-query idx' f-merkle)) s7)
  and fri-decommit:
    Some (fri-idx, s9) ∈
      set-dist (execute
        (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
  and prover-state-eq: prover-state = s9
  and replay: replay-state = verifier-replay-state prover-state
  and prefix-out:
    Some ((fr, as', dg, fl, final, idxv, fv), t) ∈
      set-dist (execute
        (do {
          fr ← p.read;
          as' ← mmap (replicate (length spec)

```

```

    (do {
      a0 ← p.receive-alpha-challenge;
      let a0' = a0;
      a1 ← p.read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    }));
  dg ← p.read;
  assert (to-nat dg ≤ p.maxDegree);
  fl ← ntimes v.receive-fri-commits nrounds;
  final ← p.read;
  idxv ← p.receive-query-index-challenge;
  let idxv' = p.index (to-nat idxv);
  fv ← mmap (v.check-decommit-on-query fr idxv');
  return (fr, as', dg, fl, final, idxv, fv)
}) replay-state)
and fl-eq: fl = zip challenges roots
and len-roots: length roots = nrounds
and len-challenges: length challenges = nrounds
and roots-aligned:  $\bigwedge j. j < nrounds \implies roots ! j = value (ms ! j)$ 
and successors:
   $\bigwedge j. j < nrounds \implies$ 
    p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
      (ps ! Suc j, ds ! Suc j, ls ! Suc j)
and fold-out:
  Some ((i, x, len, pw), u) ∈
    set-dist (execute
      (mfold
        (idx', v.cp-eval as' fv (p.h ^ idx' * shift), clength * scale, 1)
        (v.receive-query-commits fl)) t)
shows None ∉ dom (dist (execute (assert (x = final)) u))
proof –
have fold-res:
  i = idx' mod length (ls ! nrounds) ∧
  x = ls ! nrounds ! i ∧
  len = length (ls ! nrounds) ∧
  pw = 2 ^ nrounds ∧
  PTranscript u =
    fri-decommitment-transcript i (drop nrounds (butlast (zip ls ms))) ∧
    s6 ≤ u
by (rule honest-query-prefix-fri-mfold-outcome[
  OF htv create-f send-f alphas cp'-def send-degree create-cp fri
  len-ps len-ds len-ls len-ms layers created init-len n-le-N
  final-send random-idx idx-def query-decommit fri-decommit prover-state-eq
  replay prefix-out fl-eq len-roots len-challenges roots-aligned successors
  fold-out])
have prefix-res:
  final = hd (last ls)

```

```

using honest-root-alpha-degree-fri-final-query-from-prover-outcome[
  OF htv create-f send-f alphas cp'-def send-degree create-cp fri
  final-send random-idx idx-def query-decommit fri-decommit prover-state-eq
  replay prefix-out]
by blast
have i-bound:  $i < \text{length } (ls \text{ ! } nrounds)$ 
proof -
  have len-pos:  $0 < \text{length } (ls \text{ ! } nrounds)$ 
    using fri-commit-initial-layer-length-power[OF fri layers init-len n-le-N, of nrounds]
    by simp
  then show ?thesis
    using fold-res by simp
qed
have final-layer:
   $ls \text{ ! } nrounds \text{ ! } i = \text{hd } (\text{last } ls)$ 
  by (rule honest-fri-final-layer-equals-sent[
    OF fri len-ps len-ds len-ls layers init-len n-le-N nrounds-def
    successors i-bound])
have x = final
  using fold-res prefix-res final-layer by simp
then show ?thesis
  by (simp add: assert-def)
qed

lemma honest-query-prefix-fri-mfold-final-no-failure:
assumes htv: honest-trace-valid
and create-f:
  Some (f-merkle, s1) ∈
  set-dist (execute (p.create p.f-eval) init-state)
and send-f:
  Some ((), s2) ∈ set-dist (execute (p.send (value f-merkle)) s1)
and alphas:
  Some (as, s3) ∈
  set-dist (execute
    (mmap (replicate (length spec)
      (do {
         $a \leftarrow p.\text{receive-alpha-challenge};$ 
        p.send a;
        return a
      }))) s2)
and cp'-def:  $cp' = p.cp \text{ as } p.f\text{-powers}$ 
and send-degree:
  Some ((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp'))) s3)
and create-cp:
  Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
and nrounds-def:  $nrounds = \text{ceil-log } (\text{degree } cp' + 1)$ 
and fri:
  Some ((ps, ds, ls, ms), s6) ∈
  set-dist (execute

```

```

    (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
and len-ps: length ps = Suc nrounds
and len-ds: length ds = Suc nrounds
and len-ls: length ls = Suc nrounds
and len-ms: length ms = Suc nrounds
and layers:
   $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$ 
     $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$ 
and created:
   $\bigwedge i. i < \text{length } ls \implies i < \text{length } ms \implies$ 
    p.created-tree (ls ! i) (ms ! i) s6
and init-len: length (p.cp-eval as) =  $2^N$ 
and n-le-N: nrounds  $\leq N$ 
and final-send:
  Some ((), s-final)  $\in$  set-dist (execute (p.send (hd (last ls))) s6)
and random-idx:
  Some (idx, s7)  $\in$  set-dist (execute p.receive-query-index-challenge s-final)
and idx-def: idx' = p.index (to-nat idx)
and query-decommit:
  Some (qouts, s8)  $\in$ 
    set-dist (execute (mmap (p.decommit-on-query idx' f-merkle)) s7)
and fri-decommit:
  Some (fri-idx, s9)  $\in$ 
    set-dist (execute
      (mfold idx' (p.decommit-on-fri-layers (butlast (zip ls ms)))) s8)
and prover-state-eq: prover-state = s9
and replay: replay-state = verifier-replay-state prover-state
and prefix-out:
  Some ((fr, as', dg, fl, final, idxv, fv), t)  $\in$ 
    set-dist (execute
      (do {
        fr  $\leftarrow$  p.read;
        as'  $\leftarrow$  mmap (replicate (length spec)
          (do {
            a0  $\leftarrow$  p.receive-alpha-challenge;
            let a0' = a0;
            a1  $\leftarrow$  p.read;
            let a1' = a1;
            assert (a0' = a1');
            return a1'
          })));
        dg  $\leftarrow$  p.read;
        assert (to-nat dg  $\leq$  p.maxDegree);
        fl  $\leftarrow$  ntimes v.receive-fri-commits nrounds;
        final  $\leftarrow$  p.read;
        idxv  $\leftarrow$  p.receive-query-index-challenge;
        let idxv' = p.index (to-nat idxv);
        fv  $\leftarrow$  mmap (v.check-decommit-on-query fr idxv');
        return (fr, as', dg, fl, final, idxv, fv)
      })

```

```

    }) replay-state)
  and fl-eq: fl = zip challenges roots
  and len-roots: length roots = nrounds
  and len-challenges: length challenges = nrounds
  and roots-aligned:  $\bigwedge j. j < nrounds \implies roots ! j = value (ms ! j)$ 
  and successors:
     $\bigwedge j. j < nrounds \implies$ 
      p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
        (ps ! Suc j, ds ! Suc j, ls ! Suc j)
shows
  None  $\notin$  dom (dist (execute
    (do {
      (i, x, len, pw)  $\leftarrow$ 
        mfold
          (idx', v.cp-eval as' fv (p.h  $\wedge$  idx' * shift), clength * scale, 1)
          (v.receive-query-commits fl);
      assert (x = final)
    }) t))
proof (rule no-failure-bindI)
show None  $\notin$  dom (dist (execute
  (mfold
    (idx', v.cp-eval as' fv (p.h  $\wedge$  idx' * shift), clength * scale, 1)
    (v.receive-query-commits fl)) t))
by (rule honest-query-prefix-fri-mfold-no-failure[
  OF htv create-f send-f alphas cp'-def send-degree create-cp fri
  len-ps len-ds len-ls len-ms layers created init-len n-le-N
  final-send random-idx idx-def query-decommit fri-decommit prover-state-eq
  replay prefix-out fl-eq len-roots len-challenges roots-aligned successors])
next
fix fold-result u
assume fold-out:
  Some (fold-result, u)  $\in$ 
    set-dist (execute
      (mfold
        (idx', v.cp-eval as' fv (p.h  $\wedge$  idx' * shift), clength * scale, 1)
        (v.receive-query-commits fl)) t)
obtain i x len pw where fold-result-eq: fold-result = (i, x, len, pw)
by (cases fold-result)
have fold-out':
  Some ((i, x, len, pw), u)  $\in$ 
    set-dist (execute
      (mfold
        (idx', v.cp-eval as' fv (p.h  $\wedge$  idx' * shift), clength * scale, 1)
        (v.receive-query-commits fl)) t)
using fold-out unfolding fold-result-eq .
have assert-nf: None  $\notin$  dom (dist (execute (assert (x = final)) u))
by (rule honest-query-prefix-final-assert-no-failure[
  OF htv create-f send-f alphas cp'-def send-degree create-cp nrounds-def
  fri len-ps len-ds len-ls len-ms layers created init-len n-le-N

```

```

    final-send random-idx idx-def query-decommit fri-decommit prover-state-eq
    replay prefix-out fl-eq len-roots len-challenges roots-aligned successors
    fold-out])
show None  $\notin$  dom (dist (execute
  ((case fold-result of (i, x, len, pw)  $\Rightarrow$  assert (x = final))) u))
using assert-nf unfolding fold-result-eq case-prod-unfold by simp
qed

lemma honest-query-prefix-fri-mfold-final-dynamic-no-failure:
assumes htv: honest-trace-valid
and create-f:
  Some (f-merkle, s1)  $\in$ 
    set-dist (execute (p.create p.f-eval) init-state)
and send-f:
  Some ((), s2)  $\in$  set-dist (execute (p.send (value f-merkle)) s1)
and alphas:
  Some (as, s3)  $\in$ 
    set-dist (execute
      (mmap (replicate (length spec)
        (do {
          a  $\leftarrow$  p.receive-alpha-challenge;
          p.send a;
          return a
        }))) s2)
and cp'-def: cp' = p.cp as p.f-powers
and send-degree:
  Some ((), s4)  $\in$  set-dist (execute (p.send (of-nat (degree cp'))) s3)
and create-cp:
  Some (cp-merkle, s5)  $\in$  set-dist (execute (p.create (p.cp-eval as)) s4)
and nrounds-def: nrounds = ceil-log (degree cp' + 1)
and fri:
  Some ((ps, ds, ls, ms), s6)  $\in$ 
    set-dist (execute
      (p.fri-commit nrounds [cp'] [p.eval-domain] [p.cp-eval as] [cp-merkle]) s5)
and len-ps: length ps = Suc nrounds
and len-ds: length ds = Suc nrounds
and len-ls: length ls = Suc nrounds
and len-ms: length ms = Suc nrounds
and layers:
   $\bigwedge i. i < \text{length } ps \Rightarrow i < \text{length } ds \Rightarrow i < \text{length } ls \Rightarrow$ 
    ls ! i = map (poly (ps ! i)) (ds ! i)
and created:
   $\bigwedge i. i < \text{length } ls \Rightarrow i < \text{length } ms \Rightarrow$ 
    p.created-tree (ls ! i) (ms ! i) s6
and init-len: length (p.cp-eval as) =  $2^N$ 
and n-le-N: nrounds  $\leq N$ 
and final-send:
  Some ((), s-final)  $\in$  set-dist (execute (p.send (hd (last ls))) s6)
and random-idx:

```

Some ($idx, s7$) $\in$ *set-dist* (*execute* $p.receive\text{-}query\text{-}index\text{-}challenge\ s\text{-}final$)
and *idx-def*: $idx' = p.index\ (to\text{-}nat\ idx)$
and *query-decommit*:
Some ($qouts, s8$) $\in$
set-dist (*execute* (*mmap* ($p.decommit\text{-}on\text{-}query\ idx'\ f\text{-}merkle$)) $s7$)
and *fri-decommit*:
Some ($fri\text{-}idx, s9$) $\in$
set-dist (*execute*
(*mfold* $idx'\ (p.decommit\text{-}on\text{-}fri\text{-}layers\ (butlast\ (zip\ ls\ ms))))$ $s8$)
and *prover-state-eq*: $prover\text{-}state = s9$
and *replay*: $replay\text{-}state = verifier\text{-}replay\text{-}state\ prover\text{-}state$
and *prefix-out*:
Some ($(fr, as', dg, fl, final, idrv, fv), t$) $\in$
set-dist (*execute*
(*do* {
 $fr \leftarrow p.read$;
 $as' \leftarrow mmap\ (replicate\ (length\ spec)$
(*do* {
 $a0 \leftarrow p.receive\text{-}\alpha\text{-}challenge$;
 $let\ a0' = a0$;
 $a1 \leftarrow p.read$;
 $let\ a1' = a1$;
 $assert\ (a0' = a1')$;
 $return\ a1'$
});
 $dg \leftarrow p.read$;
 $assert\ (to\text{-}nat\ dg \leq p.maxDegree)$;
 $fl \leftarrow ntimes\ v.receive\text{-}fri\text{-}commits\ (ceil\text{-}log\ (to\text{-}nat\ dg + 1))$;
 $final \leftarrow p.read$;
 $idrv \leftarrow p.receive\text{-}query\text{-}index\text{-}challenge$;
 $let\ idrv' = p.index\ (to\text{-}nat\ idrv)$;
 $fv \leftarrow mmap\ (v.check\text{-}decommit\text{-}on\text{-}query\ fr\ idrv')$;
 $return\ (fr, as', dg, fl, final, idrv, fv)$
}) *replay-state*)
and *fl-eq*: $fl = zip\ challenges\ roots$
and *len-roots*: $length\ roots = nrounds$
and *len-challenges*: $length\ challenges = nrounds$
and *roots-aligned*: $\bigwedge j. j < nrounds \implies roots\ !\ j = value\ (ms\ !\ j)$
and *successors*:
 $\bigwedge j. j < nrounds \implies$
 $p.next\text{-}fri\text{-}layer\ (ps\ !\ j)\ (ds\ !\ j)\ (challenges\ !\ j) =$
 $(ps\ !\ Suc\ j, ds\ !\ Suc\ j, ls\ !\ Suc\ j)$
shows
 $None \notin dom\ (dist\ (execute$
(*do* {
 $(i, x, len, pw) \leftarrow$
mfold
 $(idx', v.cp\text{-}eval\ as'\ fv\ (p.h \wedge idx' * shift), clength * scale, 1)$
 $(v.receive\text{-}query\text{-}commits\ fl)$;

```

    assert (x = final)
  }) t))
proof –
  have prefix-fixed:
    Some ((fr, as', dg, fl, final, idxv, fv), t) ∈
      set-dist (execute
        (do {
          fr ← p.read;
          as' ← mmap (replicate (length spec)
            (do {
              a0 ← p.receive-alpha-challenge;
              let a0' = a0;
              a1 ← p.read;
              let a1' = a1;
              assert (a0' = a1');
              return a1'
            })));
          dg ← p.read;
          assert (to-nat dg ≤ p.maxDegree);
          fl ← ntimes v.receive-fri-commits nrounds;
          final ← p.read;
          idxv ← p.receive-query-index-challenge;
          let idxv' = p.index (to-nat idxv);
          fv ← mmap (v.check-decommit-on-query fr idxv');
          return (fr, as', dg, fl, final, idxv, fv)
        }) replay-state)
  by (rule honest-root-alpha-degree-fri-final-query-dynamic-outcome-to-fixed[
    OF htv create-f send-f alphas cp'-def send-degree create-cp nrounds-def
    fri final-send random-idx idx-def query-decommit fri-decommit prover-state-eq
    replay prefix-out])
  show ?thesis
  by (rule honest-query-prefix-fri-mfold-final-no-failure[
    OF htv create-f send-f alphas cp'-def send-degree create-cp nrounds-def
    fri len-ps len-ds len-ls len-ms layers created init-len n-le-N
    final-send random-idx idx-def query-decommit fri-decommit prover-state-eq
    replay prefix-fixed fl-eq len-roots len-challenges roots-aligned successors])
qed

```

lemma *honest-verifier-query-round-no-failure-from-state*:

```

assumes htv: honest-trace-valid
and create-f:
  Some (f-merkle, s1) ∈
    set-dist (execute (p.create p.f-eval) init-state)
and trace-fri:
  Some ((f-ps, f-ds, f-ls, f-ms), s-trace-fri) ∈
    set-dist (execute
      (p.trace-fri-commit f-nrounds [p.f] [p.eval-domain] [p.f-eval]
        [f-merkle]) s2)
and f-len-ps: length f-ps = Suc f-nrounds

```


and *f-len-ds*: $\text{length } f\text{-ds} = \text{Suc } f\text{-nrounds}$
and *f-len-ls*: $\text{length } f\text{-ls} = \text{Suc } f\text{-nrounds}$
and *f-len-ms*: $\text{length } f\text{-ms} = \text{Suc } f\text{-nrounds}$
and *f-layers*:
 $\bigwedge i. i < \text{length } f\text{-ps} \implies i < \text{length } f\text{-ds} \implies i < \text{length } f\text{-ls} \implies$
 $f\text{-ls} ! i = \text{map } (\text{poly } (f\text{-ps} ! i)) (f\text{-ds} ! i)$
and *f-created*:
 $\bigwedge i. i < \text{length } f\text{-ls} \implies i < \text{length } f\text{-ms} \implies$
 $p.\text{created-tree } (f\text{-ls} ! i) (f\text{-ms} ! i) s\text{-trace-fri}$
and *f-init-len*: $\text{length } p.f\text{-eval} = 2 \wedge FN$
and *f-n-le-N*: $f\text{-nrounds} \leq FN$
and *f-nrounds-def*: $f\text{-nrounds} = \text{ceil-log } clength$
and *f-fl-eq*: $f\text{-fl} = \text{zip } f\text{-challenges } f\text{-roots}$
and *f-final-eq*: $f\text{-final} = \text{hd } (\text{last } f\text{-ls})$
and *f-len-roots*: $\text{length } f\text{-roots} = f\text{-nrounds}$
and *f-len-challenges*: $\text{length } f\text{-challenges} = f\text{-nrounds}$
and *f-roots-aligned*: $\bigwedge j. j < f\text{-nrounds} \implies f\text{-roots} ! j = \text{value } (f\text{-ms} ! j)$
and *f-successors*:
 $\bigwedge j. j < f\text{-nrounds} \implies$
 $p.\text{next-fri-layer } (f\text{-ps} ! j) (f\text{-ds} ! j) (f\text{-challenges} ! j) =$
 $(f\text{-ps} ! \text{Suc } j, f\text{-ds} ! \text{Suc } j, f\text{-ls} ! \text{Suc } j)$
and *fri*:
 $\text{Some } ((ps, ds, ls, ms), s6) \in$
 $\text{set-dist } (\text{execute}$
 $(p.\text{composition-fri-commit } nrounds [cp'] [p.\text{eval-domain}]$
 $[p.\text{cp-eval as}] [cp\text{-merkle}]) s5)$
and *len-ps*: $\text{length } ps = \text{Suc } nrounds$
and *len-ds*: $\text{length } ds = \text{Suc } nrounds$
and *len-ls*: $\text{length } ls = \text{Suc } nrounds$
and *len-ms*: $\text{length } ms = \text{Suc } nrounds$
and *layers*:
 $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$
 $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$
and *created*:
 $\bigwedge i. i < \text{length } ls \implies i < \text{length } ms \implies$
 $p.\text{created-tree } (ls ! i) (ms ! i) s6$
and *init-len*: $\text{length } (p.\text{cp-eval as}) = 2 \wedge N$
and *n-le-N*: $nrounds \leq N$
and *nrounds-def*: $nrounds = \text{ceil-log } (\text{degree } cp' + 1)$
and *fr-eq*: $fr = \text{value } f\text{-merkle}$
and *as-eq*: $as' = as$
and *fl-eq*: $fl = \text{zip } challenges roots$
and *final-eq*: $final = \text{hd } (\text{last } ls)$
and *len-roots*: $\text{length } roots = nrounds$
and *len-challenges*: $\text{length } challenges = nrounds$
and *roots-aligned*: $\bigwedge j. j < nrounds \implies roots ! j = \text{value } (ms ! j)$
and *successors*:
 $\bigwedge j. j < nrounds \implies$
 $p.\text{next-fri-layer } (ps ! j) (ds ! j) (challenges ! j) =$

```

      (ps ! Suc j, ds ! Suc j, ls ! Suc j)
    and lookup: fmlookup (HashMap s) (QueryIndexChallenge (PQueryCounter s) (PState s)) =
Some idx
    and idx-def: idx' = p.index (to-nat idx)
    and query-ext: s1 ≤ s
    and trace-fri-ext: s-trace-fri ≤ s
    and fri-ext: s6 ≤ s
    and transcript:
      PTranscript s =
        query-decommitment-transcript idx' f-merkle @
        fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)) @
        fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest
  shows
    None ∉ dom (dist (execute (verifier-query-round fr f-fl f-final as' fl final) s))
proof -
  have ls0: ls ! 0 = p.cp-eval as
    using composition-fri-commit-initial-heads(3)[OF fri] .
  have idx-bound: idx' < length (ls ! 0)
    using idx-def index-less-domain cp-eval-length ls0 by simp
  have len0-eq: clength * scale = length (ls ! 0)
    using ls0 cp-eval-length by simp
  have init-domain: 2 ^ N = clength * scale
    using init-len cp-eval-length by simp
  have trace-fri-shape:
    fri-commit-outcome f-nrounds p.f p.eval-domain p.f-eval f-merkle
    f-ps f-ds f-ls f-ms s2 s-trace-fri
    by (rule trace-fri-commit-outcomeI[OF trace-fri])
  have fri-shape:
    fri-commit-outcome nrounds cp' p.eval-domain (p.cp-eval as) cp-merkle
    ps ds ls ms s5 s6
    by (rule composition-fri-commit-outcomeI[OF fri])
  have full-eq:
    verifier-query-round fr f-fl f-final as' fl final =
      (p.receive-query-index-challenge >>= (λidxv.
        let idxv' = p.index (to-nat idxv) in
        mmap (v.check-decommit-on-query fr idxv') >>= (λfv.
          mfold
            (idxv', hd fv, clength * scale, 1)
            (v.receive-query-commits f-fl) >>=
            (λ(f-i, f-x, f-len, f-pw).
              assert (f-x = f-final) >>= (λ-.
                mfold
                  (idxv', v.cp-eval as' fv (p.h ^ idxv' * shift), clength * scale, 1)
                  (v.receive-query-commits fl) >>=
                  (λ(i, x, len, pw). assert (x = final))))))))))
    unfolding verifier-query-round-def by (simp add: Let-def sm-bind-assoc split: prod.splits)
  show ?thesis
    unfolding full-eq
  proof (rule no-failure-bindI)

```

```

show  $\text{None} \notin \text{dom} (\text{dist} (\text{execute } p.\text{receive-query-index-challenge } s))$ 
  by (rule p.receive-query-index-challenge-no-failure)
next
fix idxv s-idx
assume rand:
   $\text{Some} (idxv, s-idx) \in \text{set-dist} (\text{execute } p.\text{receive-query-index-challenge } s)$ 
have rand-res:
   $idxv = idx \wedge$ 
   $P\text{State } s-idx = P\text{State } s \wedge$ 
   $P\text{Transcript } s-idx = P\text{Transcript } s \wedge$ 
   $s \leq s-idx$ 
  using p.receive-query-index-challenge-known-outcome[OF lookup rand] by simp
have idxv-def:  $p.\text{index} (\text{to-nat } idxv) = idx'$ 
  using rand-res idx-def by simp
have s1-idx:  $s1 \leq s-idx$ 
  using query-ext rand-res by (meson p.hash-ext-trans)
have tr-query:
   $P\text{Transcript } s-idx =$ 
   $\text{query-decommitment-transcript } (p.\text{index} (\text{to-nat } idxv)) \text{ f-merkle } @$ 
   $\text{fri-decommitment-transcript } idx' (\text{butlast } (\text{zip } f\text{-ls } f\text{-ms})) @$ 
   $\text{fri-decommitment-transcript } idx' (\text{butlast } (\text{zip } ls \text{ ms})) @ \text{rest}$ 
  using transcript rand-res idxv-def by simp
have query-nf:
   $\text{None} \notin \text{dom} (\text{dist} (\text{execute}$ 
     $(\text{mmap } (v.\text{check-decommit-on-query } fr (p.\text{index} (\text{to-nat } idxv)))) s-idx))$ 
  by (rule honest-query-decommitment-replay-no-failure[
    OF create-f s1-idx fr-eq tr-query])
show  $\text{None} \notin \text{dom} (\text{dist} (\text{execute}$ 
   $((\text{let } idxv' = p.\text{index} (\text{to-nat } idxv)$ 
     $\text{in } \text{mmap } (v.\text{check-decommit-on-query } fr \text{ idxv}') \gg=$ 
     $(\lambda fv.$ 
       $\text{mfold}$ 
       $(idxv', \text{hd } fv, \text{clength} * \text{scale}, 1)$ 
       $(v.\text{receive-query-commits } f\text{-fl}) \gg=$ 
       $(\lambda(f-i, f-x, f-len, f-pw).$ 
         $\text{assert } (f-x = f\text{-final}) \gg= (\lambda-.$ 
           $\text{mfold}$ 
           $(idxv', v.\text{cp-eval } as' fv (p.h \wedge idxv' * \text{shift}), \text{clength} * \text{scale}, 1)$ 
           $(v.\text{receive-query-commits } fl) \gg=$ 
           $(\lambda(i, x, len, pw). \text{assert } (x = \text{final})))))) s-idx))$ 
  unfolding Let-def
proof (rule no-failure-bindI[OF query-nf])
fix fv t
assume query-out:
   $\text{Some} (fv, t) \in$ 
   $\text{set-dist} (\text{execute}$ 
     $(\text{mmap } (v.\text{check-decommit-on-query } fr (p.\text{index} (\text{to-nat } idxv)))) s-idx)$ 
have paths:
   $\bigwedge i. i \in \text{set} (p.\text{powers-scaled } (p.\text{index} (\text{to-nat } idxv))) \implies$ 

```

```

length (get-authentication-path (length p.f-eval) i f-merkle) =
  floor-log (length p.f-eval)
using query-authentication-path-length[OF create-f] by blast
have query-out-replay:
  Some (fv, t) ∈
    set-dist (execute
      (mmap (v.check-decommit-on-query fr (p.index (to-nat idrv))))
      (s-idx(PTranscript :=
        query-decommitment-transcript (p.index (to-nat idrv)) f-merkle @
        fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)) @
        fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest)))
proof –
  have upd:
    s-idx(PTranscript :=
      query-decommitment-transcript (p.index (to-nat idrv)) f-merkle @
      fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)) @
      fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest) =
    s-idx
  using tr-query by simp
  show ?thesis
  using query-out unfolding upd .
qed
have query-res:
  fv = map (λi. p.f-eval ! i) (p.powers-scaled (p.index (to-nat idrv))) ∧
  PTranscript t =
    fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)) @
    fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest ∧
  PState t =
    foldl concat (PState s-idx)
      (query-decommitment-transcript (p.index (to-nat idrv)) f-merkle) ∧
  s-idx(PTranscript :=
    query-decommitment-transcript (p.index (to-nat idrv)) f-merkle @
    fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)) @
    fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest) ≤ t
  by (rule honest-query-decommitment-replay-outcome[
    OF paths query-out-replay])
have s-idx-t: s-idx ≤ t
proof –
  have upd:
    s-idx(PTranscript :=
      query-decommitment-transcript (p.index (to-nat idrv)) f-merkle @
      fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)) @
      fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest) =
    s-idx
  using tr-query by simp
  show ?thesis
  using query-res unfolding upd by simp
qed
have f-ls0: f-ls ! 0 = p.f-eval

```

```

    using trace-fri-commit-initial-heads(3)[OF trace-fri] .
have f-idx-bound:  $idx' < \text{length } (f\text{-ls} \ ! \ 0)$ 
    using idx-def index-less-domain f-eval-length f-ls0 by simp
have f-len0-eq:  $\text{clength} * \text{scale} = \text{length } (f\text{-ls} \ ! \ 0)$ 
    using f-ls0 f-eval-length by simp
have f-init-domain:  $2^{\wedge} FN = \text{clength} * \text{scale}$ 
    using f-init-len f-eval-length by simp
have s-trace-fri-t:  $s\text{-trace-fri} \leq t$ 
    using trace-fri-ext rand-res s-idx-t by (meson p.hash-ext-trans)
have s6-t:  $s6 \leq t$ 
    using fri-ext rand-res s-idx-t by (meson p.hash-ext-trans)
have f-tr-t:
  PTranscript t =
    fri-decommitment-transcript  $idx'$  (butlast (zip f-ls f-ms)) @
    fri-decommitment-transcript  $idx'$  (butlast (zip ls ms)) @ rest
    using query-res by simp
have f-x0-eq:  $hd \ f_v = f\text{-ls} \ ! \ 0 \ ! \ idx'$ 
proof -
  have raw:
     $hd \ f_v = p.f\text{-eval} \ ! \ p.index \ (to\text{-nat } idx_v)$ 
    by (rule honest-initial-trace-fri-accumulator-value)
    (use query-res in simp-all)
  show ?thesis
    using raw idxv-def f-ls0 by simp
qed
have f-fold-nf:
  None  $\notin \text{dom } (dist \ (execute \ (mfold \ (idx', \ hd \ f_v, \ \text{clength} * \text{scale}, \ 1) \ (v.receive\text{-query}\text{-commits } f\text{-fl})) \ t)))$ 
  unfolding f-fl-eq
  by (rule honest-fri-decommitments-mfold-no-failure[
    OF trace-fri-shape f-len-ps f-len-ds f-len-ls f-len-ms f-layers f-created
    f-init-len f-init-domain f-n-le-N f-len-roots f-len-challenges
    f-roots-aligned f-successors f-idx-bound f-x0-eq f-len0-eq
    f-tr-t s-trace-fri-t])
show None  $\notin \text{dom } (dist \ (execute \ (mfold \ (p.index \ (to\text{-nat } idx_v), \ hd \ f_v, \ \text{clength} * \text{scale}, \ 1) \ (v.receive\text{-query}\text{-commits } f\text{-fl})) \gg= \ (\lambda(f\text{-i}, \ f\text{-x}, \ f\text{-len}, \ f\text{-pw}). \ \text{assert } (f\text{-x} = f\text{-final})) \gg= \ (\lambda\_. \ mfold \ (p.index \ (to\text{-nat } idx_v), \ v.cp\text{-eval } as' \ f_v \ (p.h \wedge p.index \ (to\text{-nat } idx_v) * \text{shift}), \ \text{clength} * \text{scale}, \ 1) \ (v.receive\text{-query}\text{-commits } fl) \gg= \ (\lambda(i, \ x, \ len, \ pw). \ \text{assert } (x = final)))) \ t)))$ 
  unfolding idxv-def
proof (rule no-failure-bindI[OF f-fold-nf])
  fix f-fold-result t-trace

```

```

assume f-fold-out:
  Some (f-fold-result, t-trace) ∈
    set-dist (execute
      (mfold (idx', hd fv, clength * scale, 1)
        (v.receive-query-commits f-fl)) t)
obtain f-i f-x f-len f-pw where f-fold-eq:
  f-fold-result = (f-i, f-x, f-len, f-pw)
  by (cases f-fold-result)
have f-fold-out':
  Some ((f-i, f-x, f-len, f-pw), t-trace) ∈
    set-dist (execute
      (mfold (idx', hd fv, clength * scale, 1)
        (v.receive-query-commits f-fl)) t)
  using f-fold-out unfolding f-fold-eq .
have f-fold-out-zip:
  Some ((f-i, f-x, f-len, f-pw), t-trace) ∈
    set-dist (execute
      (mfold (idx', hd fv, clength * scale, 1)
        (v.receive-query-commits (zip f-challenges f-roots))) t)
  using f-fold-out' f-fl-eq by simp
have f-fold-res:
  f-i = idx' mod length (f-ls ! f-nrounds) ∧
  f-x = f-ls ! f-nrounds ! f-i ∧
  f-len = length (f-ls ! f-nrounds) ∧
  f-pw = 2 ^ f-nrounds ∧
  PTranscript t-trace =
    fri-decommitment-transcript f-i (drop f-nrounds (butlast (zip f-ls f-ms))) @
    fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest ∧
  PState t-trace =
    foldl concat (PState t)
      (fri-decommitment-transcript idx' (butlast (zip f-ls f-ms))) ∧
  s-trace-fri ≤ t-trace
by (rule honest-fri-decommitments-mfold-outcome[
  OF trace-fri-shape f-len-ps f-len-ds f-len-ls f-len-ms f-layers f-created
  f-init-len f-init-domain f-n-le-N f-len-roots f-len-challenges
  f-roots-aligned f-successors f-idx-bound f-x0-eq f-len0-eq
  f-tr-t s-trace-fri-t f-fold-out-zip])
have f-i-bound: f-i < length (f-ls ! f-nrounds)
proof –
  have len-pos: 0 < length (f-ls ! f-nrounds)
  using fri-commit-outcome-initial-layer-length-power[
    OF trace-fri-shape f-layers f-init-len f-n-le-N, of f-nrounds]
  by simp
  then show ?thesis
  using f-fold-res by simp
qed
have f-rounds-enough:
  ceil-log (degree p.f + 1) ≤ f-nrounds
  using trace-fri-rounds-enough f-nrounds-def by simp

```

```

have f-final-layer:
  f-ls ! f-nrounds ! f-i = hd (last f-ls)
  by (rule honest-fri-final-layer-equals-sent-initial-shape[
    OF trace-fri-shape f-len-ps f-len-ds f-len-ls f-layers f-init-len
    f-n-le-N f-rounds-enough f-successors f-i-bound])
have f-x-final: f-x = f-final
  using f-fold-res f-final-layer f-final-eq by simp
have after-trace-nf:
  None  $\notin$  dom (dist (execute
    (assert (f-x = f-final)  $\gg$ 
    ( $\lambda$ -. mfold
      (idx',
        v.cp-eval as' fv (p.h  $\wedge$  idx' * shift), clength * scale, 1)
        (v.receive-query-commits fl)  $\gg$ 
        ( $\lambda$ (i, x, len, pw). assert (x = final)))) t-trace))
proof (rule no-failure-bindI)
  show None  $\notin$  dom (dist (execute (assert (f-x = f-final)) t-trace))
    using f-x-final by (simp add: assert-def)
next
  fix unit-after-assert t-after-assert
  assume assert-out:
    Some (unit-after-assert, t-after-assert)  $\in$ 
      set-dist (execute (assert (f-x = f-final)) t-trace)
  have t-after-assert-eq: t-after-assert = t-trace
    using assert-out f-x-final by (simp add: assert-def)
  have f-bl-len: length (butlast (zip f-ls f-ms)) = f-nrounds
    using butlast-zip-suc-lengths[OF f-len-ls f-len-ms] f-len-ls f-len-ms
    by simp
  have f-drop-empty:
    drop f-nrounds (butlast (zip f-ls f-ms)) = []
    using f-bl-len by simp
  have tr-t-trace:
    PTranscript t-trace =
      fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest
    using f-fold-res f-drop-empty by simp
  have t-t-trace: t  $\leq$  t-trace
    by (rule receive-query-commits-mfold-hash-extends[OF f-fold-out])
  have s6-t-trace: s6  $\leq$  t-trace
    using s6-t t-t-trace by (meson p.hash-ext-trans)
  have tr-t-after-assert:
    PTranscript t-after-assert =
      fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest
    using t-after-assert-eq tr-t-trace by simp
  have s6-t-after-assert: s6  $\leq$  t-after-assert
    using t-after-assert-eq s6-t-trace by simp
  have x0-eq:
    v.cp-eval as' fv (p.h  $\wedge$  idx' * shift) = ls ! 0 ! idx'
  proof -
    have raw:

```

```

    v.cp-eval as' fv (p.h ^ p.index (to-nat idxv) * shift) =
      ls ! 0 ! p.index (to-nat idxv)
  by (rule honest-initial-fri-accumulator-value[
      OF htv as-eq - ls0])
    (use query-res in simp)
show ?thesis
  using raw idxv-def by simp
qed
have fold-nf:
  None ∉ dom (dist (execute
    (mfold
      (idx', v.cp-eval as' fv (p.h ^ idx' * shift), clength * scale, 1)
      (v.receive-query-commits fl)) t-after-assert))
  unfolding t-after-assert-eq fl-eq
  by (rule honest-fri-decommitments-mfold-no-failure[
      OF fri-shape len-ps len-ds len-ls len-ms layers created init-len init-domain
      n-le-N len-roots len-challenges roots-aligned successors idx-bound
      x0-eq len0-eq tr-t-trace s6-t-trace])
show None ∉ dom (dist (execute
  (mfold
    (idx', v.cp-eval as' fv (p.h ^ idx' * shift), clength * scale, 1)
    (v.receive-query-commits fl) >>=
    (λ(i, x, len, pw). assert (x = final))) t-after-assert))
proof (rule no-failure-bindI[OF fold-nf])
  fix fold-result u
  assume fold-out:
    Some (fold-result, u) ∈
      set-dist (execute
        (mfold
          (idx', v.cp-eval as' fv (p.h ^ idx' * shift), clength * scale, 1)
          (v.receive-query-commits fl)) t-after-assert)
  obtain i x len pw where fold-eq: fold-result = (i, x, len, pw)
  by (cases fold-result)
  have fold-out':
    Some ((i, x, len, pw), u) ∈
      set-dist (execute
        (mfold
          (idx', v.cp-eval as' fv (p.h ^ idx' * shift), clength * scale, 1)
          (v.receive-query-commits fl)) t-after-assert)
  using fold-out unfolding fold-eq .
  have fold-out-zip:
    Some ((i, x, len, pw), u) ∈
      set-dist (execute
        (mfold
          (idx', v.cp-eval as' fv (p.h ^ idx' * shift), clength * scale, 1)
          (v.receive-query-commits (zip challenges roots))) t-after-assert)
  using fold-out' fl-eq by simp
  have fold-res:
    i = idx' mod length (ls ! nrounds) ∧

```



```

    x = ls ! nrounds ! i ∧
    len = length (ls ! nrounds) ∧
    pw = 2 ^ nrounds ∧
    PTranscript u =
      fri-decommitment-transcript i (drop nrounds (butlast (zip ls ms))) @ rest ∧
    PState u =
      foldl concat (PState t-after-assert)
        (fri-decommitment-transcript idx' (butlast (zip ls ms))) ∧
    s6 ≤ u
  by (rule honest-fri-decommitments-mfold-outcome[
    OF fri-shape len-ps len-ds len-ls len-ms layers created init-len init-domain
      n-le-N len-roots len-challenges roots-aligned successors idx-bound
      x0-eq len0-eq tr-t-after-assert s6-t-after-assert fold-out-zip])
  have i-bound: i < length (ls ! nrounds)
  proof -
    have len-pos: 0 < length (ls ! nrounds)
    using fri-commit-outcome-initial-layer-length-power[
      OF fri-shape layers init-len n-le-N, of nrounds]
    by simp
    then show ?thesis
    using fold-res by simp
  qed
  have final-layer:
    ls ! nrounds ! i = hd (last ls)
  by (rule honest-fri-final-layer-equals-sent-shape[
    OF fri-shape len-ps len-ds len-ls layers init-len n-le-N nrounds-def
      successors i-bound])
  have x-final: x = final
  using fold-res final-layer final-eq by simp
  show None ∉ dom (dist (execute
    ((case fold-result of (i, x, len, pw) ⇒ assert (x = final))) u))
  unfolding fold-eq using x-final by (simp add: assert-def)
  qed
  qed
  show None ∉ dom (dist (execute
    ((case f-fold-result of (f-i, f-x, f-len, f-pw) ⇒
      assert (f-x = f-final) >>=
      (λ-. mfold
        (idx',
          v.cp-eval as' fv (p.h ^ idx' * shift), clength * scale, 1)
        (v.receive-query-commits fl) >>=
        (λ(i, x, len, pw). assert (x = final)))))) t-trace))
  unfolding f-fold-eq using after-trace-nf by simp
  qed
  qed
  qed
  qed

```

lemma *honest-verifier-query-round-outcome-from-state:*

assumes *htv*: *honest-trace-valid*
and *create-f*:
 Some (*f-merkle*, *s1*) $\in$
 set-dist (*execute* (*p.create p.f-eval*) *init-state*)
and *trace-fri*:
 Some ((*f-ps*, *f-ds*, *f-ls*, *f-ms*), *s-trace-fri*) $\in$
 set-dist (*execute*
 (*p.trace-fri-commit f-nrounds [p.f] [p.eval-domain] [p.f-eval]*
 [*f-merkle*]) *s2*)
and *f-len-ps*: *length f-ps = Suc f-nrounds*
and *f-len-ds*: *length f-ds = Suc f-nrounds*
and *f-len-ls*: *length f-ls = Suc f-nrounds*
and *f-len-ms*: *length f-ms = Suc f-nrounds*
and *f-layers*:
 $\bigwedge i. i < \text{length } f\text{-ps} \implies i < \text{length } f\text{-ds} \implies i < \text{length } f\text{-ls} \implies$
 $f\text{-ls} ! i = \text{map } (\text{poly } (f\text{-ps} ! i)) (f\text{-ds} ! i)$
and *f-created*:
 $\bigwedge i. i < \text{length } f\text{-ls} \implies i < \text{length } f\text{-ms} \implies$
 $p.\text{created-tree } (f\text{-ls} ! i) (f\text{-ms} ! i) s\text{-trace-fri}$
and *f-init-len*: *length p.f-eval = 2 ^ FN*
and *f-n-le-N*: *f-nrounds ≤ FN*
and *f-nrounds-def*: *f-nrounds = ceil-log clength*
and *f-fl-eq*: *f-fl = zip f-challenges f-roots*
and *f-final-eq*: *f-final = hd (last f-ls)*
and *f-len-roots*: *length f-roots = f-nrounds*
and *f-len-challenges*: *length f-challenges = f-nrounds*
and *f-roots-aligned*: $\bigwedge j. j < f\text{-nrounds} \implies f\text{-roots} ! j = \text{value } (f\text{-ms} ! j)$
and *f-successors*:
 $\bigwedge j. j < f\text{-nrounds} \implies$
 $p.\text{next-fri-layer } (f\text{-ps} ! j) (f\text{-ds} ! j) (f\text{-challenges} ! j) =$
 $(f\text{-ps} ! \text{Suc } j, f\text{-ds} ! \text{Suc } j, f\text{-ls} ! \text{Suc } j)$
and *fri*:
 Some ((*ps*, *ds*, *ls*, *ms*), *s6*) $\in$
 set-dist (*execute*
 (*p.composition-fri-commit nrounds [cp'] [p.eval-domain]*
 [*p.cp-eval as*] [*cp-merkle*]) *s5*)
and *len-ps*: *length ps = Suc nrounds*
and *len-ds*: *length ds = Suc nrounds*
and *len-ls*: *length ls = Suc nrounds*
and *len-ms*: *length ms = Suc nrounds*
and *layers*:
 $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$
 $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$
and *created*:
 $\bigwedge i. i < \text{length } ls \implies i < \text{length } ms \implies$
 $p.\text{created-tree } (ls ! i) (ms ! i) s6$
and *init-len*: *length (p.cp-eval as) = 2 ^ N*
and *n-le-N*: *nrounds ≤ N*
and *nrounds-def*: *nrounds = ceil-log (degree cp' + 1)*

```

and fr-eq: fr = value f-merkle
and as-eq: as' = as
and fl-eq: fl = zip challenges roots
and final-eq: final = hd (last ls)
and len-roots: length roots = nrounds
and len-challenges: length challenges = nrounds
and roots-aligned:  $\bigwedge j. j < nrounds \implies roots ! j = value (ms ! j)$ 
and successors:
   $\bigwedge j. j < nrounds \implies$ 
    p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
      (ps ! Suc j, ds ! Suc j, ls ! Suc j)
and lookup: fmlookup (HashMap s) (QueryIndexChallenge (PQueryCounter s) (PState s)) =
Some idx
and idx-def: idx' = p.index (to-nat idx)
and query-ext: s1 ≤ s
and trace-fri-ext: s-trace-fri ≤ s
and fri-ext: s6 ≤ s
and transcript:
  PTranscript s =
    query-decommitment-transcript idx' f-merkle @
    fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)) @
    fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest
and outcome:
  Some (res, t') ∈ set-dist (execute (verifier-query-round fr f-fl f-final as' fl final) s)
shows
  PState t' =
    foldl concat (PState s)
      (query-decommitment-transcript idx' f-merkle @
        fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)) @
        fri-decommitment-transcript idx' (butlast (zip ls ms)))
  PTranscript t' = rest
  s ≤ t'
proof –
  have f-ls0: f-ls ! 0 = p.f-eval
    using trace-fri-commit-initial-heads(3)[OF trace-fri] .
  have f-idx-bound: idx' < length (f-ls ! 0)
    using idx-def index-less-domain f-eval-length f-ls0 by simp
  have f-len0-eq: clength * scale = length (f-ls ! 0)
    using f-ls0 f-eval-length by simp
  have f-init-domain:  $2 \wedge FN = clength * scale$ 
    using f-init-len f-eval-length by simp
  have ls0: ls ! 0 = p.cp-eval as
    using composition-fri-commit-initial-heads(3)[OF fri] .
  have idx-bound: idx' < length (ls ! 0)
    using idx-def index-less-domain cp-eval-length ls0 by simp
  have len0-eq: clength * scale = length (ls ! 0)
    using ls0 cp-eval-length by simp
  have init-domain:  $2 \wedge N = clength * scale$ 
    using init-len cp-eval-length by simp

```

have *trace-fri-shape*:
fri-commit-outcome f-nrounds p.f p.eval-domain p.f-eval f-merkle
f-ps f-ds f-ls f-ms s2 s-trace-fri
by (*rule trace-fri-commit-outcomeI[OF trace-fri]*)
have *fri-shape*:
fri-commit-outcome nrounds cp' p.eval-domain (p.cp-eval as) cp-merkle
ps ds ls ms s5 s6
by (*rule composition-fri-commit-outcomeI[OF fri]*)
have *full-eq*:
verifier-query-round fr f-fl f-final as' fl final =
(p.receive-query-index-challenge $\gg$ (λidv .
let $idv' = p.index (to-nat idv)$ in
mmap ($v.check-decommit-on-query fr idv'$) $\gg$ (λfv .
mfold
*(idv' , $hd fv$, $clength * scale$, 1)*
($v.receive-query-commits f-fl$) $\gg$
($\lambda(f-i, f-x, f-len, f-pw)$.
assert ($f-x = f-final$) $\gg$ (λ .
mfold
*(idv' , $v.cp-eval as' fv (p.h \wedge idv' * shift)$, $clength * scale$, 1)*
($v.receive-query-commits fl$) $\gg$
($\lambda(i, x, len, pw)$. assert ($x = final$))))))
unfolding *verifier-query-round-def* **by** (*simp add: Let-def sm-bind-assoc split: prod.splits*)
from *outcome* **obtain** *idv s-idv fv t f-fold-result t-trace t-after-trace fold-result u* **where**
rand:
Some ($idv, s-idv$) $\in$ set-dist (execute $p.receive-query-index-challenge s$)
and *query-out:*
Some (fv, t) $\in$
set-dist (execute
(mmap ($v.check-decommit-on-query fr (p.index (to-nat idv))$))) $s-idv$)
and *f-fold-out:*
Some ($f-fold-result, t-trace$) $\in$
set-dist (execute
(mfold
*($p.index (to-nat idv)$, $hd fv$, $clength * scale$, 1)*
($v.receive-query-commits f-fl$) t)
and *f-assert-out:*
Some ($()$, $t-after-trace$) $\in$
set-dist (execute
((case $f-fold-result$ of ($f-i, f-x, f-len, f-pw$) $\Rightarrow$
assert ($f-x = f-final$))) $t-trace$)
and *fold-out:*
Some ($fold-result, u$) $\in$
set-dist (execute
(mfold
($p.index (to-nat idv)$,
 *$v.cp-eval as' fv (p.h \wedge p.index (to-nat idv) * shift)$,*
 *$clength * scale$, 1)*
($v.receive-query-commits fl$) $t-after-trace$)

```

and assert-out:
  Some (res, t') ∈
    set-dist (execute
      ((case fold-result of (i, x, len, pw) ⇒ assert (x = final))) u)
unfolding full-eq
by (auto simp: Let-def elim!: p.set-dist-bindE split: prod.splits)
have rand-res:
  idxv = idx ∧
    PState s-idx = PState s ∧
    PTranscript s-idx = PTranscript s ∧
    s ≤ s-idx
using p.receive-query-index-challenge-known-outcome[OF lookup rand] by simp
have idxv-def: p.index (to-nat idxv) = idx'
using rand-res idx-def by simp
have s1-idx: s1 ≤ s-idx
using query-ext rand-res by (meson p.hash-ext-trans)
have tr-query:
  PTranscript s-idx =
    query-decommitment-transcript (p.index (to-nat idxv)) f-merkle @
    fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)) @
    fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest
using transcript rand-res idxv-def by simp
have paths:
  ∧i. i ∈ set (p.powers-scaled (p.index (to-nat idxv))) ⇒
    length (get-authentication-path (length p.f-eval) i f-merkle) =
      floor-log (length p.f-eval)
using query-authentication-path-length[OF create-f] by blast
have query-out-replay:
  Some (fv, t) ∈
    set-dist (execute
      (mmap (v.check-decommit-on-query fr (p.index (to-nat idxv))))
      (s-idx(PTranscript :=
        query-decommitment-transcript (p.index (to-nat idxv)) f-merkle @
        fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)) @
        fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest)))
proof –
have upd:
  s-idx(PTranscript :=
    query-decommitment-transcript (p.index (to-nat idxv)) f-merkle @
    fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)) @
    fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest)) =
    s-idx
using tr-query by simp
show ?thesis
using query-out unfolding upd .
qed
have query-res:
  fv = map (λi. p.f-eval ! i) (p.powers-scaled (p.index (to-nat idxv))) ∧
    PTranscript t =

```

```

    fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)) @
    fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest ∧
PState t =
  foldl concat (PState s-idx)
    (query-decommitment-transcript (p.index (to-nat idxv)) f-merkle) ∧
s-idx(PTranscript :=
  query-decommitment-transcript (p.index (to-nat idxv)) f-merkle @
  fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)) @
  fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest) ≤ t
by (rule honest-query-decommitment-replay-outcome[
  OF paths query-out-replay])
have s-idx-t: s-idx ≤ t
proof -
  have upd:
    s-idx(PTranscript :=
      query-decommitment-transcript (p.index (to-nat idxv)) f-merkle @
      fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)) @
      fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest) =
      s-idx
    using tr-query by simp
  show ?thesis
    using query-res unfolding upd by simp
qed
have s-trace-fri-t: s-trace-fri ≤ t
  using trace-fri-ext rand-res s-idx-t by (meson p.hash-ext-trans)
have tr-t:
  PTranscript t =
    fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)) @
    fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest
  using query-res by simp
have f-x0-eq: hd fv = f-ls ! 0 ! idx'
proof -
  have raw: hd fv = p.f-eval ! p.index (to-nat idxv)
    by (rule honest-initial-trace-fri-accumulator-value)
    (use query-res in simp-all)
  show ?thesis
    using raw idxv-def f-ls0 by simp
qed
obtain f-i f-x f-len f-pw where f-fold-eq: f-fold-result = (f-i, f-x, f-len, f-pw)
  by (cases f-fold-result)
have f-fold-out':
  Some ((f-i, f-x, f-len, f-pw), t-trace) ∈
    set-dist (execute
      (mfold (idx', hd fv, clength * scale, 1)
        (v.receive-query-commits f-fl)) t)
  using f-fold-out unfolding f-fold-eq idxv-def by simp
have f-fold-out-zip:
  Some ((f-i, f-x, f-len, f-pw), t-trace) ∈
    set-dist (execute

```

```

      (mfold (idx', hd fv, clength * scale, 1)
        (v.receive-query-commits (zip f-challenges f-roots))) t)
    using f-fold-out' f-fl-eq by simp
  have f-fold-res:
    f-i = idx' mod length (f-ls ! f-nrounds) ∧
    f-x = f-ls ! f-nrounds ! f-i ∧
    f-len = length (f-ls ! f-nrounds) ∧
    f-pw = 2 ^ f-nrounds ∧
    PTranscript t-trace =
      fri-decommitment-transcript f-i
        (drop f-nrounds (butlast (zip f-ls f-ms))) @
      fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest ∧
    PState t-trace =
      foldl concat (PState t)
        (fri-decommitment-transcript idx' (butlast (zip f-ls f-ms))) ∧
    s-trace-fri ≤ t-trace
  by (rule honest-fri-decommitments-mfold-outcome[
    OF trace-fri-shape f-len-ps f-len-ds f-len-ls f-len-ms f-layers f-created
    f-init-len f-init-domain f-n-le-N f-len-roots f-len-challenges
    f-roots-aligned f-successors f-idx-bound f-x0-eq f-len0-eq
    tr-t s-trace-fri-t f-fold-out-zip])
  have f-i-bound: f-i < length (f-ls ! f-nrounds)
  proof -
    have len-pos: 0 < length (f-ls ! f-nrounds)
    using fri-commit-outcome-initial-layer-length-power[
      OF trace-fri-shape f-layers f-init-len f-n-le-N, of f-nrounds]
    by simp
    then show ?thesis
    using f-fold-res by simp
  qed
  have f-rounds-enough:
    ceil-log (degree p.f + 1) ≤ f-nrounds
  using trace-fri-rounds-enough f-nrounds-def by simp
  have f-final-layer:
    f-ls ! f-nrounds ! f-i = hd (last f-ls)
  by (rule honest-fri-final-layer-equals-sent-initial-shape[
    OF trace-fri-shape f-len-ps f-len-ds f-len-ls f-layers f-init-len
    f-n-le-N f-rounds-enough f-successors f-i-bound])
  have f-x-final: f-x = f-final
  using f-fold-res f-final-layer f-final-eq by simp
  have t-after-trace-eq: t-after-trace = t-trace
  using f-assert-out unfolding f-fold-eq f-x-final by (simp add: assert-def)
  have f-bl-len: length (butlast (zip f-ls f-ms)) = f-nrounds
  using butlast-zip-suc-lengths[OF f-len-ls f-len-ms] f-len-ls f-len-ms by simp
  have f-drop-empty:
    drop f-nrounds (butlast (zip f-ls f-ms)) = []
  using f-bl-len by simp
  have tr-t-after-trace:
    PTranscript t-after-trace =

```

```

    fri-decommitment-transcript idx' (butlast (zip ls ms)) @ rest
  using f-fold-res f-drop-empty t-after-trace-eq by simp
have state-t-after-trace:
  PState t-after-trace =
    foldl concat (PState t)
      (fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)))
  using f-fold-res t-after-trace-eq by simp
have s6-t-after-trace: s6 ≤ t-after-trace
proof -
  have t-t-trace: t ≤ t-trace
    by (rule receive-query-commits-mfold-hash-extends[OF f-fold-out])
  have t-t-after: t ≤ t-after-trace
    using t-after-trace-eq t-t-trace by simp
  show ?thesis
    using fri-ext rand-res s-idx-t t-t-after by (meson p.hash-ext-trans)
qed
have t-t-trace-ext: t ≤ t-trace
  by (rule receive-query-commits-mfold-hash-extends[OF f-fold-out])
have x0-eq:
  v.cp-eval as' fv (p.h ^ idx' * shift) = ls ! 0 ! idx'
proof -
  have raw:
    v.cp-eval as' fv (p.h ^ p.index (to-nat idxv) * shift) =
      ls ! 0 ! p.index (to-nat idxv)
    by (rule honest-initial-fri-accumulator-value[
      OF htv as-eq - ls0])
      (use query-res in simp)
  show ?thesis
    using raw idxv-def by simp
qed
obtain i x len pw where fold-eq: fold-result = (i, x, len, pw)
  by (cases fold-result)
have fold-out':
  Some ((i, x, len, pw), u) ∈
    set-dist (execute
      (mfold
        (idx', v.cp-eval as' fv (p.h ^ idx' * shift), clength * scale, 1)
        (v.receive-query-commits fl)) t-after-trace)
  using fold-out unfolding fold-eq idxv-def by simp
have fold-out-zip:
  Some ((i, x, len, pw), u) ∈
    set-dist (execute
      (mfold
        (idx', v.cp-eval as' fv (p.h ^ idx' * shift), clength * scale, 1)
        (v.receive-query-commits (zip challenges roots))) t-after-trace)
  using fold-out' fl-eq by simp
have fold-res:
  i = idx' mod length (ls ! nrounds) ∧
  x = ls ! nrounds ! i ∧

```



```

len = length (ls ! nrounds) ∧
pw = 2 ^ nrounds ∧
PTranscript u =
  fri-decommitment-transcript i (drop nrounds (butlast (zip ls ms))) @ rest ∧
PState u =
  foldl concat (PState t-after-trace)
    (fri-decommitment-transcript idx' (butlast (zip ls ms))) ∧
s6 ≤ u
by (rule honest-fri-decommitments-mfold-outcome[
  OF fri-shape len-ps len-ds len-ls len-ms layers created init-len init-domain
  n-le-N len-roots len-challenges roots-aligned successors idx-bound
  x0-eq len0-eq tr-t-after-trace s6-t-after-trace fold-out-zip])
have i-bound: i < length (ls ! nrounds)
proof -
  have len-pos: 0 < length (ls ! nrounds)
  using fri-commit-outcome-initial-layer-length-power[
    OF fri-shape layers init-len n-le-N, of nrounds]
  by simp
  then show ?thesis
  using fold-res by simp
qed
have final-layer:
  ls ! nrounds ! i = hd (last ls)
by (rule honest-fri-final-layer-equals-sent-shape[
  OF fri-shape len-ps len-ds len-ls layers init-len n-le-N nrounds-def
  successors i-bound])
have x-final: x = final
  using fold-res final-layer final-eq by simp
have t'-eq: t' = u
  using assert-out unfolding fold-eq x-final by (simp add: assert-def)
have state-query:
  PState t =
    foldl concat (PState s)
      (query-decommitment-transcript idx' f-merkle)
  using query-res rand-res idxv-def by simp
show PState t' =
  foldl concat (PState s)
    (query-decommitment-transcript idx' f-merkle @
      fri-decommitment-transcript idx' (butlast (zip f-ls f-ms)) @
      fri-decommitment-transcript idx' (butlast (zip ls ms)))
  using fold-res state-query state-t-after-trace t'-eq by simp
show PTranscript t' = rest
proof -
  have bl-len: length (butlast (zip ls ms)) = nrounds
  using butlast-zip-suc-lengths[OF len-ls len-ms] len-ls len-ms by simp
  have drop-empty: drop nrounds (butlast (zip ls ms)) = []
  using bl-len by simp
  show ?thesis
  using fold-res t'-eq drop-empty by simp

```

```

qed
show  $s \leq t'$ 
proof -
  have  $s-t: s \leq t$ 
    using rand-res s-idx-t by (meson p.hash-ext-trans)
  have  $t\text{-after-}u: t\text{-after-trace} \leq u$ 
    by (rule receive-query-commits-mfold-hash-extends[OF fold-out])
  show ?thesis
    using s-t t-t-trace-ext t-after-trace-eq t-after-u t'-eq by (meson p.hash-ext-trans)
qed
qed

```

lemma *verifier-query-round-query-counter*:

assumes *outcome*:

Some (res, t') ∈ set-dist (execute (verifier-query-round fr f-fl f-final as' fl final) s)

shows $PQueryCounter\ t' = Suc\ (PQueryCounter\ s)$

proof -

have *full-eq*:

verifier-query-round fr f-fl f-final as' fl final =
(p.receive-query-index-challenge $\gg$ ($\lambda idrv.$
let $idrv' = p.index\ (to-nat\ idrv)$ in
mmap (v.check-decommit-on-query fr $idrv'$) $\gg$ ($\lambda fv.$

mfold
*($idrv', hd\ fv, clength * scale, 1$)*
*(v.receive-query-commits *f-fl*) $\gg$*
($\lambda(f-i, f-x, f-len, f-pw).$
assert ($f-x = f-final$) $\gg$ ($\lambda-.$

mfold
*($idrv', v.cp-eval\ as'\ fv\ (p.h \wedge idrv' * shift), clength * scale, 1$)*
*(v.receive-query-commits *fl*) $\gg$*
($\lambda(i, x, len, pw). assert\ (x = final)$))))))

unfolding *verifier-query-round-def* **by** (*simp add: Let-def sm-bind-assoc split: prod.splits*)

from *outcome* **obtain** $idrv\ s\text{-}idx\ fv\ t\ f\text{-fold-result}\ t\text{-trace}\ t\text{-after-trace}\ fold\text{-result}\ u$ **where**

rand:

Some (idrv, s-idx) ∈ set-dist (execute p.receive-query-index-challenge s)

and *query-out*:

Some (fv, t) ∈
set-dist (execute
(mmap (v.check-decommit-on-query fr (p.index (to-nat idrv)))) s-idx)

and *f-fold-out*:

Some (f-fold-result, t-trace) ∈
set-dist (execute
(mfold
*(p.index (to-nat idrv), hd fv, clength * scale, 1)*
*(v.receive-query-commits *f-fl*) t)*

and *f-assert-out*:

Some ((), t-after-trace) ∈
set-dist (execute
((case f-fold-result of (f-i, f-x, f-len, f-pw) $\Rightarrow$

```

    assert (f-x = f-final))) t-trace)
and fold-out:
  Some (fold-result, u) ∈
    set-dist (execute
      (mfold
        (p.index (to-nat idxv),
          v.cp-eval as' fv (p.h ^ p.index (to-nat idxv) * shift),
          clength * scale, 1)
        (v.receive-query-commits fl)) t-after-trace)
and assert-out:
  Some (res, t') ∈
    set-dist (execute
      ((case fold-result of (i, x, len, pw) ⇒ assert (x = final))) u)
unfolding full-eq
by (auto simp: Let-def elim!: p.set-dist-bindE split: prod.splits)
have q-idx: PQueryCounter s-idx = Suc (PQueryCounter s)
  using p.receive-query-index-challenge-counter-outcome[OF rand] by simp
have q-query: PQueryCounter t = PQueryCounter s-idx
  by (rule mmap-preserves-query-counter[OF query-out])
  (rule check-decommit-on-query-step-preserves-query-counter)
have q-f-fold: PQueryCounter t-trace = PQueryCounter t
  by (rule receive-query-commits-mfold-preserves-query-counter[OF f-fold-out])
have q-f-assert: PQueryCounter t-after-trace = PQueryCounter t-trace
proof -
  obtain f-i f-x f-len f-pw where f-fold-eq:
    f-fold-result = (f-i, f-x, f-len, f-pw)
  by (cases f-fold-result)
  have assert':
    Some ((), t-after-trace) ∈
      set-dist (execute (assert (f-x = f-final)) t-trace)
  using f-assert-out unfolding f-fold-eq by simp
  show ?thesis
  using assert-outcomeD(2)[OF assert'] by simp
qed
have q-fold: PQueryCounter u = PQueryCounter t-after-trace
  by (rule receive-query-commits-mfold-preserves-query-counter[OF fold-out])
have q-assert: PQueryCounter t' = PQueryCounter u
proof -
  obtain i x len pw where fold-eq: fold-result = (i, x, len, pw)
  by (cases fold-result)
  have assert':
    Some ((), t') ∈ set-dist (execute (assert (x = final)) u)
  using assert-out unfolding fold-eq by simp
  show ?thesis
  using assert-outcomeD(2)[OF assert'] by simp
qed
show ?thesis
  using q-idx q-query q-f-fold q-f-assert q-fold q-assert by simp
qed

```

lemma *honest-verifier-query-tail-from-prover-tail-no-failure:*
assumes *htv: honest-trace-valid*
and *create-f:*
 Some (*f-merkle*, *s1*) ∈
 set-dist (execute (*p.create p.f-eval*) *init-state*)
and *trace-fri:*
 Some ((*f-ps*, *f-ds*, *f-ls*, *f-ms*), *s-trace-fri*) ∈
 set-dist (execute
 (*p.trace-fri-commit f-nrounds [p.f] [p.eval-domain] [p.f-eval]*
 [*f-merkle*]) *s2*)
and *f-len-ps:* length *f-ps* = Suc *f-nrounds*
and *f-len-ds:* length *f-ds* = Suc *f-nrounds*
and *f-len-ls:* length *f-ls* = Suc *f-nrounds*
and *f-len-ms:* length *f-ms* = Suc *f-nrounds*
and *f-layers:*
 $\bigwedge i. i < \text{length } f\text{-ps} \implies i < \text{length } f\text{-ds} \implies i < \text{length } f\text{-ls} \implies$
 $f\text{-ls} ! i = \text{map } (\text{poly } (f\text{-ps} ! i)) (f\text{-ds} ! i)$
and *f-created:*
 $\bigwedge i. i < \text{length } f\text{-ls} \implies i < \text{length } f\text{-ms} \implies$
 $p.\text{created-tree } (f\text{-ls} ! i) (f\text{-ms} ! i) s\text{-trace-fri}$
and *f-init-len:* length *p.f-eval* = $2 \wedge FN$
and *f-n-le-N:* *f-nrounds* ≤ *FN*
and *f-nrounds-def:* *f-nrounds* = ceil-log clength
and *f-fl-eq:* *f-fl* = zip *f-challenges* *f-roots*
and *f-final-eq:* *f-final* = hd (last *f-ls*)
and *f-len-roots:* length *f-roots* = *f-nrounds*
and *f-len-challenges:* length *f-challenges* = *f-nrounds*
and *f-roots-aligned:* $\bigwedge j. j < f\text{-nrounds} \implies f\text{-roots} ! j = \text{value } (f\text{-ms} ! j)$
and *f-successors:*
 $\bigwedge j. j < f\text{-nrounds} \implies$
 $p.\text{next-fri-layer } (f\text{-ps} ! j) (f\text{-ds} ! j) (f\text{-challenges} ! j) =$
 $(f\text{-ps} ! \text{Suc } j, f\text{-ds} ! \text{Suc } j, f\text{-ls} ! \text{Suc } j)$
and *fri:*
 Some ((*ps*, *ds*, *ls*, *ms*), *s6*) ∈
 set-dist (execute
 (*p.composition-fri-commit nrounds [cp'] [p.eval-domain]*
 [*p.cp-eval as*] [*cp-merkle*]) *s5*)
and *len-ps:* length *ps* = Suc *nrounds*
and *len-ds:* length *ds* = Suc *nrounds*
and *len-ls:* length *ls* = Suc *nrounds*
and *len-ms:* length *ms* = Suc *nrounds*
and *layers:*
 $\bigwedge i. i < \text{length } ps \implies i < \text{length } ds \implies i < \text{length } ls \implies$
 $ls ! i = \text{map } (\text{poly } (ps ! i)) (ds ! i)$
and *created:*
 $\bigwedge i. i < \text{length } ls \implies i < \text{length } ms \implies$
 $p.\text{created-tree } (ls ! i) (ms ! i) s6$
and *init-len:* length (*p.cp-eval as*) = $2 \wedge N$

```

and n-le-N: nrounds ≤ N
and nrounds-def: nrounds = ceil-log (degree cp' + 1)
and fr-eq: fr = value f-merkle
and as-eq: as' = as
and fl-eq: fl = zip challenges roots
and final-eq: final = hd (last ls)
and len-roots: length roots = nrounds
and len-challenges: length challenges = nrounds
and roots-aligned:  $\bigwedge j. j < \textit{nrounds} \implies \textit{roots} ! j = \textit{value} (\textit{ms} ! j)$ 
and successors:
   $\bigwedge j. j < \textit{nrounds} \implies$ 
    p.next-fri-layer (ps ! j) (ds ! j) (challenges ! j) =
      (ps ! Suc j, ds ! Suc j, ls ! Suc j)
and tail:
  Some (prover-result, prover-state) ∈
    set-dist (execute (ntimes (prover-query-round f-merkle f-ls f-ms ls ms) n) p-s)
and state-eq: PState v-s = PState p-s
and transcript-v: PTranscript v-s = rev ys
and transcript-p: PTranscript prover-state = ys @ PTranscript p-s
and prover-ext: prover-state ≤ v-s
and query-ext: s1 ≤ v-s
and trace-fri-ext: s-trace-fri ≤ v-s
and fri-ext: s6 ≤ v-s
and query-counter-eq: PQueryCounter v-s = PQueryCounter p-s
shows
  None ∉ dom (dist (execute (ntimes (verifier-query-round fr f-fl f-final as' fl final) n) v-s))
using tail state-eq transcript-v transcript-p prover-ext query-ext trace-fri-ext fri-ext
  query-counter-eq
proof (induction n arbitrary: prover-result prover-state p-s v-s ys)
case 0
then show ?case
by simp
next
case (Suc n)
from ntimes-Suc-outcomeE[OF Suc.premis(1)] obtain x xs p-next where
  step:
    Some (x, p-next) ∈
      set-dist (execute (prover-query-round f-merkle f-ls f-ms ls ms) p-s)
and tail-rest:
  Some (xs, prover-state) ∈
    set-dist (execute (ntimes (prover-query-round f-merkle f-ls f-ms ls ms) n) p-next)
and prover-result-eq: prover-result = x # xs
by blast
from prover-query-round-lookup-state-transcript[OF step] obtain idx where
  x-eq: x = ()
and lookup-next: fmlookup (HashMap p-next) (QueryIndexChallenge (PQueryCounter p-s))
  (PState p-s) = Some idx
and state-next:
  PState p-next =

```

```

    foldl concat (PState p-s)
      (query-decommitment-transcript (p.index (to-nat idx)) f-merkle @
        fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip f-ls f-ms)) @
        fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip ls ms)))
  and tr-next:
    PTranscript p-next =
      rev (fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip ls ms))) @
      rev (fri-decommitment-transcript (p.index (to-nat idx)) (butlast (zip f-ls f-ms))) @
      rev (query-decommitment-transcript (p.index (to-nat idx)) f-merkle) @
      PTranscript p-s
  and p-s-next: p-s ≤ p-next
  by blast
  let ?idx' = p.index (to-nat idx)
  let ?query = query-decommitment-transcript ?idx' f-merkle
  let ?f-fri = fri-decommitment-transcript ?idx' (butlast (zip f-ls f-ms))
  let ?fri = fri-decommitment-transcript ?idx' (butlast (zip ls ms))
  have p-next-prover: p-next ≤ prover-state
    by (rule ntimes-hash-extends[OF - tail-rest])
      (rule prover-query-round-hash-extends)
  have p-next-v: p-next ≤ v-s
    using p-next-prover Suc.prems(5) by (meson p.hash-ext-trans)
  have lookup-v: fmllookup (HashMap v-s) (QueryIndexChallenge (PQueryCounter v-s) (PState
v-s)) = Some idx
    using p.hash-extension-lookup[OF lookup-next p-next-v] Suc.prems(2) Suc.prems(9)
    by simp
  have tail-tr-ext: transcript-extends prover-state p-next
    by (rule ntimes-transcript-extends[OF - tail-rest])
      (rule prover-query-round-transcript-extends)
  from tail-tr-ext obtain ys-tail where
    tr-tail: PTranscript prover-state = ys-tail @ PTranscript p-next
  unfolding transcript-extends-def by auto
  have ys-eq: ys = ys-tail @ rev ?fri @ rev ?f-fri @ rev ?query
    using Suc.prems(4) tr-tail tr-next by simp
  have transcript-round:
    PTranscript v-s = ?query @ ?f-fri @ ?fri @ rev ys-tail
    using Suc.prems(3) ys-eq by simp
  have step-nf:
    None ∉ dom (dist (execute (verifier-query-round fr f-fl f-final as' fl final) v-s))
  by (rule honest-verifier-query-round-no-failure-from-state[
    OF htv create-f trace-fri f-len-ps f-len-ds f-len-ls f-len-ms f-layers
    f-created f-init-len f-n-le-N f-nrounds-def f-fl-eq f-final-eq
    f-len-roots f-len-challenges f-roots-aligned f-successors
    fri len-ps len-ds len-ls len-ms layers created init-len n-le-N
    nrounds-def fr-eq as-eq fl-eq final-eq len-roots len-challenges
    roots-aligned successors lookup-v refl Suc.prems(6) Suc.prems(7) Suc.prems(8)
    transcript-round])
  show ?case
    unfolding ntimes.simps
  proof (rule no-failure-bindI[OF step-nf])

```

```

fix v-res v-next
assume v-step:
  Some (v-res, v-next) ∈
    set-dist (execute (verifier-query-round fr f-fl f-final as' fl final) v-s)
have v-state:
  PState v-next =
    foldl concat (PState v-s) (?query @ ?f-fri @ ?fri)
by (rule honest-verifier-query-round-outcome-from-state(1)[
  OF htv create-f trace-fri f-len-ps f-len-ds f-len-ls f-len-ms f-layers
  f-created f-init-len f-n-le-N f-nrounds-def f-fl-eq f-final-eq
  f-len-roots f-len-challenges f-roots-aligned f-successors
  fri len-ps len-ds len-ls len-ms layers created init-len n-le-N
  nrounds-def fr-eq as-eq fl-eq final-eq len-roots len-challenges
  roots-aligned successors lookup-v refl Suc.premis(6) Suc.premis(7) Suc.premis(8)
  transcript-round v-step])
have v-transcript: PTranscript v-next = rev ys-tail
by (rule honest-verifier-query-round-outcome-from-state(2)[
  OF htv create-f trace-fri f-len-ps f-len-ds f-len-ls f-len-ms f-layers
  f-created f-init-len f-n-le-N f-nrounds-def f-fl-eq f-final-eq
  f-len-roots f-len-challenges f-roots-aligned f-successors
  fri len-ps len-ds len-ls len-ms layers created init-len n-le-N
  nrounds-def fr-eq as-eq fl-eq final-eq len-roots len-challenges
  roots-aligned successors lookup-v refl Suc.premis(6) Suc.premis(7) Suc.premis(8)
  transcript-round v-step])
have v-ext: v-s ≤ v-next
by (rule honest-verifier-query-round-outcome-from-state(3)[
  OF htv create-f trace-fri f-len-ps f-len-ds f-len-ls f-len-ms f-layers
  f-created f-init-len f-n-le-N f-nrounds-def f-fl-eq f-final-eq
  f-len-roots f-len-challenges f-roots-aligned f-successors
  fri len-ps len-ds len-ls len-ms layers created init-len n-le-N
  nrounds-def fr-eq as-eq fl-eq final-eq len-roots len-challenges
  roots-aligned successors lookup-v refl Suc.premis(6) Suc.premis(7) Suc.premis(8)
  transcript-round v-step])
have state-v-next: PState v-next = PState p-next
using v-state state-next Suc.premis(2) by simp
have prover-ext-next: prover-state ≤ v-next
using Suc.premis(5) v-ext by (meson p.hash-ext-trans)
have query-ext-next: s1 ≤ v-next
using Suc.premis(6) v-ext by (meson p.hash-ext-trans)
have trace-fri-ext-next: s-trace-fri ≤ v-next
using Suc.premis(7) v-ext by (meson p.hash-ext-trans)
have fri-ext-next: s6 ≤ v-next
using Suc.premis(8) v-ext by (meson p.hash-ext-trans)
have q-p-next: PQueryCounter p-next = Suc (PQueryCounter p-s)
by (rule prover-query-round-query-counter[OF step])
have q-v-next: PQueryCounter v-next = Suc (PQueryCounter v-s)
by (rule verifier-query-round-query-counter[OF v-step])
have query-counter-next: PQueryCounter v-next = PQueryCounter p-next
using q-p-next q-v-next Suc.premis(9) by simp

```

have *tail-nf*:
None $\notin$ *dom*
 (*dist* (*execute* (*ntimes* (*verifier-query-round* *fr f-fl f-final as' fl final*) *n*) *v-next*))
by (*rule* *Suc.IH*[
OF tail-rest state-v-next v-transcript tr-tail prover-ext-next
query-ext-next trace-fri-ext-next fri-ext-next query-counter-next])
show *None* $\notin$ *dom* (*dist* (*execute*
 (*ntimes* (*verifier-query-round* *fr f-fl f-final as' fl final*) *n* $\gg$
 ($\lambda x s. \text{return } (v\text{-res} \# xs)$) *v-next*))
by (*rule* *no-failure-bind-returnI*[*OF tail-nf*])
qed
qed

lemma *honest-verify-from-prover-no-failure*:
assumes *htv*: *honest-trace-valid*
and *prover*:
Some (*prover-result*, *prover-state*) $\in$
set-dist (*execute* *p.prover-monad init-state*)
and *replay*: *replay-state* = *verifier-replay-state prover-state*
shows *None* $\notin$ *dom* (*dist* (*execute* *v.verify-monad replay-state*))
proof (*rule* *honest-prover-monad-outcome*[*OF prover*])
fix *f-merkle s1 s2 f-nrounds f-ps f-ds f-ls f-ms s-trace-fri*
s-trace-final as s3 cp' s4 cp-merkle s5 nrounds ps ds ls ms s6 s-final
assume *create-f*:
Some (*f-merkle*, *s1*) $\in$
set-dist (*execute* (*p.create p.f-eval*) *init-state*)
and *created-f*: *p.created-tree p.f-eval f-merkle s1*
and *send-f*:
Some ($\emptyset$, *s2*) $\in$ *set-dist* (*execute* (*p.send* (*value f-merkle*)) *s1*)
and *f-nrounds-def*: *f-nrounds* = *ceil-log clength*
and *trace-fri*:
Some (*f-ps*, *f-ds*, *f-ls*, *f-ms*), *s-trace-fri*) $\in$
set-dist (*execute*
 (*p.trace-fri-commit* *f-nrounds* [*p.f*] [*p.eval-domain*] [*p.f-eval*]
 [*f-merkle*]) *s2*)
and *f-len-ps*: *length f-ps* = *Suc f-nrounds*
and *f-len-ds*: *length f-ds* = *Suc f-nrounds*
and *f-len-ls*: *length f-ls* = *Suc f-nrounds*
and *f-len-ms*: *length f-ms* = *Suc f-nrounds*
and *f-layers*:
 $\bigwedge i. i < \text{length } f\text{-ps} \implies i < \text{length } f\text{-ds} \implies i < \text{length } f\text{-ls} \implies$
 $f\text{-ls} ! i = \text{map } (\text{poly } (f\text{-ps} ! i)) (f\text{-ds} ! i)$
and *f-created*:
 $\bigwedge i. i < \text{length } f\text{-ls} \implies i < \text{length } f\text{-ms} \implies$
 $p.\text{created-tree } (f\text{-ls} ! i) (f\text{-ms} ! i) s\text{-trace-fri}$
and *trace-final-send*:
Some ($\emptyset$, *s-trace-final*) $\in$
set-dist (*execute* (*p.send* (*hd* (*last f-ls*))) *s-trace-fri*)
and *alphas*:


```

Some (as, s3) ∈
  set-dist (execute
    (mmap (replicate (length spec)
      (do {
        a ← p.receive-alpha-challenge;
        p.send a;
        return a
      }))) s-trace-final)
and cp'-def: cp' = p.cp as p.f-powers
and send-degree:
  Some ((), s4) ∈ set-dist (execute (p.send (of-nat (degree cp')))) s3)
and create-cp:
  Some (cp-merkle, s5) ∈ set-dist (execute (p.create (p.cp-eval as)) s4)
and created-cp: p.created-tree (p.cp-eval as) cp-merkle s5
and nrounds-def: nrounds = ceil-log (degree cp' + 1)
and fri:
  Some ((ps, ds, ls, ms), s6) ∈
    set-dist (execute
      (p.composition-fri-commit nrounds [cp'] [p.eval-domain]
        [p.cp-eval as] [cp-merkle] s5))
and len-ps: length ps = Suc nrounds
and len-ds: length ds = Suc nrounds
and len-ls: length ls = Suc nrounds
and len-ms: length ms = Suc nrounds
and layers:
  ∧ i. i < length ps ⇒ i < length ds ⇒ i < length ls ⇒
    ls ! i = map (poly (ps ! i)) (ds ! i)
and created:
  ∧ i. i < length ls ⇒ i < length ms ⇒
    p.created-tree (ls ! i) (ms ! i) s6
and final-send:
  Some ((), s-final) ∈ set-dist (execute (p.send (hd (last ls)))) s6)
and tail:
  Some (prover-result, prover-state) ∈
    set-dist (execute
      (ntimes (prover-query-round f-merkle f-ls f-ms ls ms) rounds) s-final)
from cp-eval-length-power obtain N where init-len:
  length (p.cp-eval as) = 2 ^ N
  by blast
have n-le-N: nrounds ≤ N
  by (rule fri-rounds-le-eval-domain-log[OF htv cp'-def nrounds-def init-len])
from f-eval-length-power obtain FN where f-init-len:
  length p.f-eval = 2 ^ FN
  by blast
have f-n-le-N: f-nrounds ≤ FN
proof —
  have cl-le-len: clength ≤ length p.f-eval
    using f-eval-length p.scale-pos by simp
  have f-rounds-le-len: f-nrounds ≤ ceil-log (length p.f-eval)

```

```

    using ceil-log-mono[OF cl-le-len] f-nrounds-def by simp
  have len-pos:  $0 < \text{length } p.f\text{-eval}$ 
    using f-init-len by simp
  have ceil-len-le:  $\text{ceil-log } (\text{length } p.f\text{-eval}) \leq FN$ 
  proof -
    have  $\text{length } p.f\text{-eval} - 1 < 2 \wedge FN$ 
      using f-init-len len-pos by simp
    then have  $\text{ceil-log } (\text{Suc } (\text{length } p.f\text{-eval} - 1)) \leq FN$ 
      by (rule ceil-log-Suc-le-power)
    then show ?thesis
      using len-pos by simp
  qed
  show ?thesis
    using f-rounds-le-len ceil-len-le by simp
  qed
  from trace-fri-commit-initial-replay-successor-data[OF trace-fri]
  obtain f-roots f-challenges where
    f-len-roots:  $\text{length } f\text{-roots} = f\text{-nrounds}$ 
    and f-len-challenges:  $\text{length } f\text{-challenges} = f\text{-nrounds}$ 
    and f-roots-aligned:  $\forall j < f\text{-nrounds}. f\text{-roots } ! j = \text{value } (f\text{-ms } ! j)$ 
    and f-successors:
       $\forall j < f\text{-nrounds}.$ 
         $p.\text{next-fri-layer } (f\text{-ps } ! j) (f\text{-ds } ! j) (f\text{-challenges } ! j) =$ 
         $(f\text{-ps } ! \text{Suc } j, f\text{-ds } ! \text{Suc } j, f\text{-ls } ! \text{Suc } j)$ 
    by blast
  from composition-fri-commit-initial-replay-successor-data[OF fri]
  obtain roots challenges where
    len-roots:  $\text{length } \text{roots} = n\text{rounds}$ 
    and len-challenges:  $\text{length } \text{challenges} = n\text{rounds}$ 
    and roots-aligned:  $\forall j < n\text{rounds}. \text{roots } ! j = \text{value } (\text{ms } ! j)$ 
    and successors:
       $\forall j < n\text{rounds}.$ 
         $p.\text{next-fri-layer } (\text{ps } ! j) (\text{ds } ! j) (\text{challenges } ! j) =$ 
         $(\text{ps } ! \text{Suc } j, \text{ds } ! \text{Suc } j, \text{ls } ! \text{Suc } j)$ 
    by blast
  let ?round =
    do {
       $a0 \leftarrow p.\text{receive-alpha-challenge};$ 
       $\text{let } a0' = a0;$ 
       $a1 \leftarrow p.\text{read};$ 
       $\text{let } a1' = a1;$ 
       $\text{assert } (a0' = a1');$ 
       $\text{return } a1'$ 
    }
  let ?prefix =
    do {
       $fr \leftarrow p.\text{read};$ 
       $f\text{-fl} \leftarrow n\text{times } v.\text{receive-trace-fri-commits } (\text{ceil-log } \text{clength});$ 
       $f\text{-final} \leftarrow p.\text{read};$ 

```

```

    as' ← mmap (replicate (length spec) ?round);
    dg ← p.read;
    assert (to-nat dg ≤ p.maxDegree);
    fl ← ntimes v.receive-composition-fri-commits
      (ceil-log (to-nat dg + 1));
    final ← p.read;
    return (fr, f-fl, f-final, as', dg, fl, final)
  }
have prefix-nf:
  None ∉ dom (dist (execute ?prefix replay-state))
by (rule honest-trace-root-alpha-degree-fri-final-dynamic-from-prover-tail-no-failure[
    OF htv create-f send-f f-nrounds-def trace-fri trace-final-send alphas
    cp'-def send-degree create-cp nrounds-def fri final-send tail replay])
have full-eq:
  v.verify-monad =
    (?prefix >>=
      (λout. case out of (fr, f-fl, f-final, as', dg, fl, final) ⇒
        ntimes (verifier-query-round fr f-fl f-final as' fl final) rounds))
  unfolding v.verify-monad-def
  by (simp add: Let-def verifier-query-round-def sm-bind-assoc split: prod.splits)
show ?thesis
  unfolding full-eq
proof (rule no-failure-bindI[OF prefix-nf])
  fix out t
  assume prefix-out-raw: Some (out, t) ∈ set-dist (execute ?prefix replay-state)
  obtain fr f-fl f-final as' dg fl final where out-eq:
    out = (fr, f-fl, f-final, as', dg, fl, final)
  by (cases out) auto
  show None ∉ dom (dist (execute
    ((case out of (fr, f-fl, f-final, as', dg, fl, final) ⇒
      ntimes (verifier-query-round fr f-fl f-final as' fl final) rounds)) t))
  proof –
  have prefix-out:
    Some ((fr, f-fl, f-final, as', dg, fl, final), t) ∈
      set-dist (execute ?prefix replay-state)
  using prefix-out-raw unfolding out-eq .
  obtain f-roots0 f-challenges0 roots0 challenges0 ys where
    f-len-roots0: length f-roots0 = f-nrounds
    and f-len-challenges0: length f-challenges0 = f-nrounds
    and f-fl-eq0: f-fl = zip f-challenges0 f-roots0
    and f-roots-aligned0:
      ∧j. j < f-nrounds ⇒ f-roots0 ! j = value (f-ms ! j)
    and f-successors0:
      ∧j. j < f-nrounds ⇒
        p.next-fri-layer (f-ps ! j) (f-ds ! j) (f-challenges0 ! j) =
          (f-ps ! Suc j, f-ds ! Suc j, f-ls ! Suc j)
    and f-final-eq: f-final = hd (last f-ls)
    and
    len-roots0: length roots0 = nrounds

```

```

and len-challenges0: length challenges0 = nrounds
and fl-eq0: fl = zip challenges0 roots0
and roots-aligned0:
   $\bigwedge j. j < nrounds \implies roots0 ! j = value (ms ! j)$ 
and successors0:
   $\bigwedge j. j < nrounds \implies$ 
    p.next-fri-layer (ps ! j) (ds ! j) (challenges0 ! j) =
      (ps ! Suc j, ds ! Suc j, ls ! Suc j)
and fr-eq: fr = value f-merkle
and as-eq: as' = as
and dg-eq: dg = of-nat (degree (p.cp as p.f-powers))
and final-eq: final = hd (last ls)
and state-t: PState t = PState s-final
and transcript-t: PTranscript t = rev ys
and transcript-prover:
  PTranscript prover-state = ys @ PTranscript s-final
and replay-t: replay-state ≤ t
using honest-trace-root-alpha-degree-fri-final-dynamic-from-prover-tail-outcome[
  OF htv create-f send-f f-nrounds-def trace-fri trace-final-send alphas
  cp'-def send-degree create-cp nrounds-def fri final-send tail replay prefix-out]
by blast
have tail-hash: s-final ≤ prover-state
  by (rule ntimes-hash-extends[OF - tail])
    (rule prover-query-round-hash-extends)
have prover-replay: prover-state ≤ replay-state
  using replay
  unfolding verifier-replay-state-def less-eq-hash-ext-def less-eq-fmap-def
  by simp
have prover-t: prover-state ≤ t
  using prover-replay replay-t by (meson p.hash-ext-trans)
have alpha-data:
  length as = length spec ∧
  s-trace-final ≤ s3 ∧
  PState s3 = foldl concat (PState s-trace-final) as ∧
  PTranscript s3 = rev as @ PTranscript s-trace-final ∧
  PTraceFriCounter s3 = PTraceFriCounter s-trace-final ∧
  PCompositionFriCounter s3 = PCompositionFriCounter s-trace-final ∧
  PAlphaCounter s3 = PAlphaCounter s-trace-final + length as ∧
  PQueryCounter s3 = PQueryCounter s-trace-final ∧
  ( $\forall i < length\ as.$ 
    fmlookup (HashMap s3)
      (AlphaChallenge (PAlphaCounter s-trace-final + i)
        (foldl concat (PState s-trace-final) (take i as)))) =
      Some (as ! i))
  using alpha-mmap-outcome[OF alphas] by simp
have s1-s2: s1 ≤ s2
  using send-hash-extends[OF send-f] .
have s2-trace-fri: s2 ≤ s-trace-fri
  using p.trace-fri-commit-extends[OF trace-fri] .

```

```

have trace-fri-final:  $s\text{-trace-fri} \leq s\text{-trace-final}$ 
  using send-hash-extends[OF trace-final-send] .
have trace-final-s3:  $s\text{-trace-final} \leq s3$ 
  using alpha-data by simp
have s3-s4:  $s3 \leq s4$ 
  using send-hash-extends[OF send-degree] .
have s4-s5:  $s4 \leq s5$ 
  using create-hash-extends[OF create-cp] .
have s5-s6:  $s5 \leq s6$ 
  using p.composition-fri-commit-extends[OF fri] .
have s6-final-hash:  $s6 \leq s\text{-final}$ 
  using send-hash-extends[OF final-send] .
have s1-prover:  $s1 \leq \text{prover-state}$ 
  using s1-s2 s2-trace-fri trace-fri-final trace-final-s3
    s3-s4 s4-s5 s5-s6 s6-final-hash tail-hash
  by (meson p.hash-ext-trans)
have trace-fri-prover:  $s\text{-trace-fri} \leq \text{prover-state}$ 
  using trace-fri-final trace-final-s3 s3-s4 s4-s5 s5-s6
    s6-final-hash tail-hash
  by (meson p.hash-ext-trans)
have s6-prover:  $s6 \leq \text{prover-state}$ 
  using s6-final-hash tail-hash by (rule p.hash-ext-trans)
have s1-t:  $s1 \leq t$ 
  using s1-prover prover-t by (meson p.hash-ext-trans)
have s-trace-fri-t:  $s\text{-trace-fri} \leq t$ 
  using trace-fri-prover prover-t by (meson p.hash-ext-trans)
have s6-t:  $s6 \leq t$ 
  using s6-prover prover-t by (meson p.hash-ext-trans)
have q-t-replay:  $P\text{QueryCounter } t = P\text{QueryCounter replay-state}$ 
  by (rule honest-trace-root-alpha-degree-fri-final-dynamic-prefix-preserves-query-counter[
    OF prefix-out])
have q-s1-zero:  $P\text{QueryCounter } s1 = 0$ 
  using p.create-preserves-channel(6)[OF create-f]
  unfolding init-state-def by simp
have q-s2-zero:  $P\text{QueryCounter } s2 = 0$ 
  using p.send-outcome[OF send-f] q-s1-zero by simp
have q-trace-fri-zero:  $P\text{QueryCounter } s\text{-trace-fri} = 0$ 
  using trace-fri-commit-preserves-query-counter[OF trace-fri] q-s2-zero
  by simp
have q-trace-final-zero:  $P\text{QueryCounter } s\text{-trace-final} = 0$ 
  using p.send-outcome[OF trace-final-send] q-trace-fri-zero by simp
have q-s3-zero:  $P\text{QueryCounter } s3 = 0$ 
  using alpha-data q-trace-final-zero by simp
have q-s4-zero:  $P\text{QueryCounter } s4 = 0$ 
  using p.send-outcome[OF send-degree] q-s3-zero by simp
have q-s5-zero:  $P\text{QueryCounter } s5 = 0$ 
  using p.create-preserves-channel(6)[OF create-cp] q-s4-zero by simp
have q-s6-zero:  $P\text{QueryCounter } s6 = 0$ 
  using composition-fri-commit-preserves-query-counter[OF fri] q-s5-zero

```

```

    by simp
  have q-s-final-zero: PQueryCounter s-final = 0
    using p.send-outcome[OF final-send] q-s6-zero by simp
  have query-counter-t-final: PQueryCounter t = PQueryCounter s-final
    using q-t-replay q-s-final-zero replay
    unfolding verifier-replay-state-def by simp
  have tail-nf:
    None  $\notin$  dom
      (dist (execute (ntimes (verifier-query-round fr f-fl f-final as' fl final) rounds) t))
  by (rule honest-verifier-query-tail-from-prover-tail-no-failure[
    OF htv create-f trace-fri f-len-ps f-len-ds f-len-ls f-len-ms f-layers
    f-created f-init-len f-n-le-N f-nrounds-def f-fl-eq0 f-final-eq
    f-len-roots0 f-len-challenges0 f-roots-aligned0 f-successors0
    fri len-ps len-ds len-ls len-ms layers created init-len n-le-N
    nrounds-def fr-eq as-eq fl-eq0 final-eq len-roots0 len-challenges0
    roots-aligned0 successors0 tail state-t transcript-t transcript-prover
    prover-t s1-t s-trace-fri-t s6-t query-counter-t-final])
  show ?thesis
    unfolding out-eq using tail-nf by simp
qed
qed
qed

```

lemma *honest-trace-valid-implies-no-failure:*

```

  assumes htv: honest-trace-valid
  shows None  $\notin$  dom (dist (execute exec init-state))
  unfolding exec-def

```

proof (rule no-failure-bindI[OF prover-monad-no-failure])

```

  fix prover-result prover-state
  assume prover:
    Some (prover-result, prover-state)  $\in$ 
      set-dist (execute p.prover-monad init-state)
  show None  $\notin$  dom (dist (execute
    (modify ( $\lambda$ c. c[PState := 0, PTranscript := rev (PTranscript c),
      PTraceFriCounter := 0, PCompositionFriCounter := 0,
      PAlphaCounter := 0, PQueryCounter := 0]))  $\gg$ 
    ( $\lambda$ -. v.verify-monad))
    prover-state))

```

proof (rule no-failure-bindI)

```

  show None  $\notin$  dom (dist (execute
    (modify ( $\lambda$ c. c[PState := 0, PTranscript := rev (PTranscript c),
      PTraceFriCounter := 0, PCompositionFriCounter := 0,
      PAlphaCounter := 0, PQueryCounter := 0]))
    prover-state))
    by simp

```

next

```

  fix u replay-state
  assume reset:
    Some (u, replay-state)  $\in$ 

```

```

    set-dist (execute
      (modify (λc. c(PState := 0, PTranscript := rev (PTranscript c),
        PTraceFriCounter := 0, PCompositionFriCounter := 0,
        PAlphaCounter := 0, PQueryCounter := 0)))
      prover-state)
  have replay: replay-state = verifier-replay-state prover-state
  using modify-verifier-replay-state-outcome[OF reset] by simp
  show None ∉ dom (dist (execute v.verify-monad replay-state))
  by (rule honest-verify-from-prover-no-failure[OF htv prover replay])
qed
qed

lemma honest-query-decommitments-no-fail:
  assumes honest-trace-valid
  shows no-failure exec init-state
  using honest-trace-valid-implies-no-failure[OF assms]
  unfolding no-failure-def .

lemma honest-fri-decommitments-no-fail:
  assumes honest-trace-valid
  shows no-failure exec init-state
  using honest-trace-valid-implies-no-failure[OF assms]
  unfolding no-failure-def .

lemma honest-final-constant-no-fail:
  assumes honest-trace-valid
  shows no-failure exec init-state
  using honest-trace-valid-implies-no-failure[OF assms]
  unfolding no-failure-def .

lemma honest-verify-no-fail:
  assumes honest-trace-valid
  shows no-failure exec init-state
  using honest-trace-valid-implies-no-failure[OF assms]
  unfolding no-failure-def .

lemma wp-event-is-none-eq-wp-error:
  wp-event m (λa. Option.is-none a) s = wp-error m s
  unfolding wp-event-def wp-error-def
  by (rule arg-cong[where f=λQ. wp m Q s]) (auto intro!: ext split: option.splits)

lemma wp-error-zero-if-no-failure:
  assumes None ∉ dom (dist (execute m s))
  shows wp-error m s = 0
proof -
  have failure-prob m s = 0
  using assms
  unfolding failure-prob-def
  by (cases dist (execute m s) None) (auto simp: domIff)

```

```

    then show ?thesis
      by (simp add: wp-error-eq-failure-prob)
qed

lemma completeness-wp-reduction:
  assumes honest-trace-valid
  shows wp-event exec ( $\lambda a. \text{Option.is-none } a$ ) init-state = 0
  using honest-trace-valid-implies-no-failure[OF assms]
  by (simp add: wp-event-is-none-eq-wp-error wp-error-zero-if-no-failure)

lemma completeness:
  assumes honest-trace-valid
  shows wp-event exec ( $\lambda a. \text{Option.is-none } a$ ) init-state  $\leq (0::prob)$ 
  using completeness-wp-reduction[OF assms]
  by simp

end

end

```

```

theory Completeness
  imports Completeness-Verifier
begin

```

Public entry point for the honest-completeness development. The imported theories reduce the probabilistic verifier failure event to deterministic algebraic, transcript, Merkle, query, and FRI facts for the honest prover.

```
end
```

3 Executable Example

```

theory Square-Sequence
  imports
    Stark-Core.Stark
    HOL-Number-Theory.Number-Theory
    Stark-Core.Galois-Field-5
begin

```

3.1 Square Sequence Example

In the following example we generate a proof for a program which computes the square sequence: $x, x^2, x^4, \dots$ Since the computation is probabilistic we keep the executable example small: a sequence of size 2 with scaling factor 1.

We use $\text{GF}(5)$ - the finite field of order 5 - for our example.

- It is a Proth field.
- It is large enough for a trace of length 2 with a scaling factor of 1 and a shifted evaluation domain disjoint from H .
- It is still feasible to execute probabilistically. Larger fields make the executable example substantially more expensive.

3.2 Polynomial Constraints

The first number in the sequence (at position 0) is 2

definition $p0:: gf5\ poly\ list \Rightarrow gf5\ poly$

where

$$p0\ f \equiv f!0 - 2$$

definition $r0:: nat\ list$

where

$$r0 \equiv [0]$$

The last number in the sequence (at position 1) is 4

definition $p1:: gf5\ poly\ list \Rightarrow gf5\ poly$

where

$$p1\ f \equiv f!0 - 4$$

definition $r1:: nat\ list$

where

$$r1 \equiv [1]$$

For all intermediate positions (which is only position 0) the next position is the square of the previous position.

definition $p2:: gf5\ poly\ list \Rightarrow gf5\ poly$

where

$$p2\ ps \equiv ps\ !\ 1 - (ps\ !\ 0 * ps\ !\ 0)$$

definition $r2:: nat\ list$

where

$$r2 \equiv [0..<1]$$

The spec consists of a list containing the above constraints. Note that the third parameter in each entry is the degree of the constraint polynomial.

definition $spec::((gf5\ poly\ list \Rightarrow gf5\ poly) \times nat\ list \times nat)\ list$

where

$$spec = [(p0, r0, 1), (p1, r1, 1), (p2, r2, 2)]$$

We also need the spec in an other format. These need to be equivalent and for the future it should be unified with the other spec.

definition $p0':: gf5\ list \Rightarrow gf5$

where

$p0' f \equiv f!0 - 2$

definition $p1':: gf5\ list \Rightarrow gf5$

where

$p1' f \equiv f!0 - 4$

definition $p2':: gf5\ list \Rightarrow gf5$

where

$p2' ps \equiv ps ! 1 - (ps ! 0 * ps ! 0)$

definition $spec2::(gf5\ list \Rightarrow gf5) \times nat\ list \Rightarrow list$

where

$spec2 = [(p0', r0), (p1', r1), (p2', r2)]$

3.3 Interpreting the stark locale

abbreviation $tlength::nat$ **where** $tlength \equiv 2$

lemma *of-nat-to-nat:*

fixes n

assumes $n \leq Max\ (degrees\ tlength\ spec)$

shows $Galois-Field-5.to-nat\ (Galois-Field-5.of-nat\ n) = n$

proof $-$

have $1 * (tlength - 1) - length\ r0 = 0$ **unfolding** $r0-def$ **by** *simp*

moreover have $1 * (tlength - 1) - length\ r1 = 0$ **unfolding** $r1-def$ **by** *simp*

moreover have $tlength * (tlength - 1) - length\ r2 = 1$ **unfolding** $r2-def$ **by** *simp*

ultimately have $degrees\ tlength\ spec = \{0,1\}$ **unfolding** $degrees-def\ spec-def$

apply $(auto\ simp\ add:r0-def\ r1-def\ r2-def)$

by $(metis\ diff-Suc-1'\ length-Suc-conv\ list.size(3)\ numeral-2-eq-2)$

then have $Max\ (degrees\ tlength\ spec) = 1$ **by** *simp*

then consider $n = 0 \mid n = 1$ **using** *assms* **by** *linarith*

then show *?thesis*

proof *cases*

case 1

then show *?thesis*

by $(metis\ nat-zero-as-int\ of-nat-to-nat\ to-nat.rep-eq\ zero-gf5.rep-eq)$

next

case 2

then show *?thesis*

by $(metis\ nat-one-as-int\ of-nat-to-nat\ one-gf5.rep-eq\ to-nat.rep-eq)$

qed

qed

lemma *exact-order-gf5-2-4:*

exact-order gf5-2 4

unfolding *exact-order-def*

apply $(intro\ conjI)$

apply *simp*

apply *eval*

```

apply (intro allI impI)
apply (subgoal-tac m = 1 ∨ m = 2 ∨ m = 3)
apply (elim disjE; hypsubst; eval)
by linarith

```

lemma *gf5-2-nonzero*:

```

gf5-2 ≠ 0
by eval

```

lemma *numeral-2-gf5-nonzero*:

```

(2::gf5) ≠ 0
by eval

```

lemma *square-sequence-spec-degree-fits*:

```

assumes (c, roots, d) ∈ set spec
shows d * (tlength - 1) - length roots < tlength * 1
using assms
unfolding spec-def r0-def r1-def r2-def
by auto

```

lemma *square-sequence-constraint-degree-wellformed*:

```

assumes deg-f: degree f < tlength
and spec-entry: (c, roots, d) ∈ set spec
shows
  degree
    (c (map (λp. f ∘p XP ((gf5-2 ^ ((5 - 1) div tlength)) ^ p))
      [0..<tlength])) ≤ d * (tlength - 1)

```

proof –

```

let ?fs =
  map (λp. f ∘p XP ((gf5-2 ^ ((5 - 1) div tlength)) ^ p))
    [0..<tlength]
have deg-XP: degree (XP a :: gf5 poly) ≤ 1 for a
unfolding XP-def by (rule degree-monom-le)
have deg-f-le: degree f ≤ 1
using deg-f by simp
have compose-le: degree (f ∘p XP a) ≤ 1 for a
proof –
  have degree (f ∘p XP a) ≤ degree f * degree (XP a)
    by (rule degree-pcompose-le)
  also have ... ≤ 1
    using deg-f-le deg-XP[of a] by (cases degree f; cases degree (XP a :: gf5 poly)) auto
  finally show ?thesis .

```

qed

```

have deg0: degree (?fs ! 0) ≤ 1
using compose-le[of ((gf5-2 ^ ((5 - 1) div tlength)) ^ 0)] by simp
have deg1: degree (?fs ! 1) ≤ 1
using compose-le[of ((gf5-2 ^ ((5 - 1) div tlength)) ^ 1)] by simp
have deg-square: degree (?fs ! 0 * ?fs ! 0) ≤ 2
proof –

```

```

have degree (?fs ! 0 * ?fs ! 0) ≤ degree (?fs ! 0) + degree (?fs ! 0)
  by (rule degree-mult-le)
also have ... ≤ 2
  using deg0 by linarith
finally show ?thesis .
qed
show ?thesis
proof -
  have p0-bound: degree (?fs ! 0 - 2) ≤ 1
    using degree-diff-le[OF deg0, of 2] by simp
  have p1-bound: degree (?fs ! 0 - 4) ≤ 1
    using degree-diff-le[OF deg0, of 4] by simp
  have p2-bound: degree (?fs ! 1 - ?fs ! 0 * ?fs ! 0) ≤ 2
  proof -
    have degree (?fs ! 1 - ?fs ! 0 * ?fs ! 0) ≤ max (degree (?fs ! 1)) (degree (?fs ! 0 * ?fs !
0))
      by (rule degree-diff-le-max)
    also have ... ≤ 2
      using deg1 deg-square by simp
    finally show ?thesis .
  qed
show ?thesis
  using spec-entry p0-bound p1-bound p2-bound
  unfolding spec-def p0-def p1-def p2-def
  by auto
qed
qed

lemma square-sequence-spec-roots-in-range:
  assumes (c, roots, d) ∈ set spec
  and r ∈ set roots
  shows r < tlength
  using assms
  unfolding spec-def r0-def r1-def r2-def
  by auto

lemma square-sequence-spec-roots-distinct:
  assumes (c, roots, d) ∈ set spec
  shows distinct roots
  using assms
  unfolding spec-def r0-def r1-def r2-def
  by auto

lemma gf5-2-notin-scale1-H-raw:
  gf5-2 ∉
  set (map (( $\wedge$ ) (gf5-2  $\wedge$  ((5 - 1) div (tlength * 1)))) [0..<1 * tlength])
  by eval

lemma gf5-to-nat-range:

```

```

    range Galois-Field-5.to-nat = {0..<5}
proof –
  have vals:
    Galois-Field-5.to-nat gf5-0 = 0
    Galois-Field-5.to-nat gf5-1 = 1
    Galois-Field-5.to-nat gf5-2 = 2
    Galois-Field-5.to-nat gf5-3 = 3
    Galois-Field-5.to-nat gf5-4 = 4
  by eval+
  have mems:
    0 ∈ range Galois-Field-5.to-nat
    1 ∈ range Galois-Field-5.to-nat
    2 ∈ range Galois-Field-5.to-nat
    3 ∈ range Galois-Field-5.to-nat
    4 ∈ range Galois-Field-5.to-nat
  using vals by (metis rangeI)+
  have range Galois-Field-5.to-nat ⊆ {0..<5}
proof
  fix n :: nat
  assume n ∈ range Galois-Field-5.to-nat
  then obtain x where n-eq: n = Galois-Field-5.to-nat x
  by blast
  show n ∈ {0..<5}
  unfolding n-eq
  by (cases x; simp add: vals)
qed
moreover have {0..<5} ⊆ range Galois-Field-5.to-nat
proof
  fix n :: nat
  assume n ∈ {0..<5}
  then have n = 0 ∨ n = 1 ∨ n = 2 ∨ n = 3 ∨ n = 4
  by auto
  then show n ∈ range Galois-Field-5.to-nat
  using mems by auto
qed
ultimately show ?thesis
by blast
qed

lemma square-sequence-query-sample-space-size-dvd:
  tlength * 1 – Max (set [0..<tlength]) * 1 dvd 5
by eval

global-interpretation
  stark:
    stark
    concat-gf5 gf5-2 gf5-2 1 tlength 2 Galois-Field-5.to-nat Galois-Field-5.of-nat 5 spec 1
defines stark-send = channel.send
  and stark-send2 = channel.send2

```

```

and stark-read = channel.read
and stark-g = stark.g
and stark-G = stark.G
and stark-h = stark.h
and stark-H = stark.H
and stark-eval-domain = stark.eval-domain
and stark-powers-scaled = stark.powers-scaled
and stark-index = stark.index
and stark-g-map = stark.g-map
and stark-maxDegree = stark.maxDegree
and stark-cp = stark.cp
apply standard
  apply simp
  apply (rule exI[where x=1], simp)
  apply (rule exI[where x=0], simp)
  apply (simp add: exact-order-gf5-2-4)
  apply simp
  apply simp
  apply simp
  apply (rule gf5-to-nat-range)
  apply simp
  apply simp
  apply (rule square-sequence-query-sample-space-size-dvd)
  apply (rule gf5-2-nonzero)
  apply (rule numeral-2-gf5-nonzero)
  apply (erule square-sequence-spec-degree-fits)
  apply (erule (1) square-sequence-constraint-degree-wellformed)
  apply (erule (1) square-sequence-spec-roots-in-range)
  apply (erule square-sequence-spec-roots-distinct)
  apply (rule gf5-2-notin-scale1-H-raw)
using of-nat-to-nat by presburger

```

3.4 Interpretation of prover locale

definition *trace* where $trace = [2,4]$

lemma *length-trace*: $length\ trace = tlength$ **unfolding** *trace-def* **by** *simp*

global-interpretation

prover:

prover

concat-gf5 gf5-2 gf5-2 1 tlength 2 Galois-Field-5.to-nat Galois-Field-5.of-nat 5 spec 1 trace

defines *prover-f* = *prover.f*

and *prover-f-eval* = *prover.f-eval*

and *prover-cp-eval* = *prover.cp-eval*

and *prover-next-fri-domain* = *prover.next-fri-domain*

and *prover-next-fri-polynomial* = *prover.next-fri-polynomial*

and *prover-next-fri-layer* = *prover.next-fri-layer*

and *prover-fri-commit* = *prover.fri-commit*

```

and prover-decommit-on-fri-layers = prover.decommit-on-fri-layers
and prover-decommit-on-query = prover.decommit-on-query
and prover-sproof = prover.sproof
and prover-prover-monad = prover.prover-monad
and prover-f-powers = prover.f-powers
apply (rule prover.intro)
apply standard
using length-trace by metis

```

3.5 Interpretation of verifier locale

```

lemma nth-map-tlength-0[simp]:
  assumes tlength ≤ length xs
  shows map f xs ! 0 = f (xs ! 0)
  by (rule nth-map) (use assms in linarith)

```

```

lemma nth-map-tlength-1[simp]:
  assumes tlength ≤ length xs
  shows map f xs ! 1 = f (xs ! 1)
  by (rule nth-map) (use assms in linarith)

```

global-interpretation

```

verifier:
verifier
  concat-gf5 gf5-2 gf5-2 1 tlength 2 Galois-Field-5.to-nat Galois-Field-5.of-nat 5 spec 1 spec2
defines verifier-receive-fri-commits = verifier.receive-fri-commits
and verifier-receive-query-commits = verifier.receive-query-commits
and verifier-check-decommit-on-query = verifier.check-decommit-on-query
and verifier-verify = verifier.verify
and verifier-cp-eval = verifier.cp-eval
and verifier-verify-monad = verifier.verify-monad
apply (rule verifier.intro)
apply standard
by (auto simp: spec-def spec2-def p0-def p1-def p2-def p0'-def p1'-def p2'-def)

```

3.6 Creating and Verifying Proof

The executable prover state can be inspected with *value prover-sproof*. Concrete verifier transcripts are intentionally not hard-coded here: the transcript shape changes when protocol checks such as trace FRI are added.

end

4 Soundness

```

theory Soundness-Core-Base
  imports Stark-Core.Stark

```

begin

Core definitions and deterministic relations used by the STARK soundness development. Later layers separate Merkle binding, verifier execution, bad events, FRI, random-oracle accounting, and the public reduction theorems.

locale *soundness* =
verifier concat omega shift scale clength powers to-nat of-nat size spec rounds spec2
for *concat* :: '*f*::*proth-field* $\Rightarrow$ '*f* $\Rightarrow$ '*f*
and *omega* :: '*f*
and *shift* :: '*f*
and *scale* :: *nat*
and *clength* :: *nat*
and *powers* :: *nat*
and *to-nat* :: '*f* $\Rightarrow$ *nat*
and *of-nat* :: *nat* $\Rightarrow$ '*f*
and *size* :: *nat*
and *spec* :: ((*f poly list* $\Rightarrow$ '*f poly*) $\times$ *nat list* $\times$ *nat*) *list*
and *rounds* :: *nat*
and *spec2* :: ((*f list* $\Rightarrow$ '*f*) $\times$ *nat list*) *list*
+
fixes *trace-fri-error* :: *prob*
and *composition-fri-error* :: *prob*
and *merkle-binding-error* :: *prob*
assumes *rounds-positive*: $0 < \text{rounds}$
and *spec-query-margin*:
 $\text{Max} (\text{degrees } \text{clength } \text{spec}) +$
 $\text{sum-list } (\text{map } (\lambda(-, \text{roots}, -). \text{length } \text{roots}) \text{ spec})$
 $< \text{clength} * \text{scale} - \text{Max} (\text{set } [0..<\text{powers}]) * \text{scale}$
begin

definition *trace-satisfies-constraint*
:: '*f poly* $\Rightarrow$ ('*f poly list* $\Rightarrow$ '*f poly*) $\Rightarrow$ *nat list* $\Rightarrow$ *bool*
where
trace-satisfies-constraint f c roots $\longleftrightarrow$
 $(\forall r \in \text{set roots.}$
 $\text{poly } (c (\text{trace-powers-of } f)) (g \wedge r) = 0)$

definition *exists-valid-trace* :: *bool*
where
exists-valid-trace $\longleftrightarrow$
 $(\exists f.$
 $\text{degree } f < \text{clength} \wedge$
 $(\forall (c, \text{roots}, d) \in \text{set spec.}$
 $\text{trace-satisfies-constraint } f \text{ c roots}))$

definition *composition-error-bound* :: *prob*
where *composition-error-bound* = $1 / \text{nnreal size}$

definition *query-sample-space-size* :: *nat*

where

query-sample-space-size =
 length * *scale* - *Max* (*set* [*0..<powers*]) * *scale*

definition *query-sample-space* :: *nat set*

where *query-sample-space* = {*idx. idx < query-sample-space-size*}

definition *all-constraint-roots* :: '*f list*

where

all-constraint-roots =
 remdups (*List.concat* (*map* ($\lambda(c, roots, d). g\text{-map } roots$) *spec*))

definition *query-agreement-bound* :: *nat*

where *query-agreement-bound* = *maxDegree* + *length all-constraint-roots*

definition *query-error-bound* :: *prob*

where

query-error-bound =
 nnreal query-agreement-bound / *nnreal query-sample-space-size*

definition *soundness-bound* :: *prob*

where

soundness-bound =
 composition-error-bound +
 trace-fri-error +
 composition-fri-error +
 nnreal rounds * *query-error-bound*

definition *soundness-bound-with-merkle* :: *prob*

where

soundness-bound-with-merkle =
 soundness-bound + *merkle-binding-error*

Soundness should expose a transcript-only verifier interface: the adversary chooses the proof transcript, while the verifier starts from an empty random-oracle/hash map and the initial Fiat-Shamir state.

definition *verifier-initial-state* :: '*f list* $\Rightarrow$ '*f protocol-channel*

where

verifier-initial-state tr =
 ($\langle HashMap = fmempty, PState = 0, PTranscript = tr,$
 PTraceFriCounter = 0, *PCompositionFriCounter* = 0,
 PAlphaCounter = 0, *PQueryCounter* = 0 $\rangle$)

definition *verify-transcript*

where

verify-transcript tr =
 execute verify-monad (*verifier-initial-state tr*)

lemma *verifier-initial-state-simps[simp]*:

$\text{HashMap } (\text{verifier-initial-state } tr) = \text{fmempty}$
 $\text{PState } (\text{verifier-initial-state } tr) = 0$
 $\text{PTranscript } (\text{verifier-initial-state } tr) = tr$
 $\text{PTraceFriCounter } (\text{verifier-initial-state } tr) = 0$
 $\text{PCompositionFriCounter } (\text{verifier-initial-state } tr) = 0$
 $\text{PAlphaCounter } (\text{verifier-initial-state } tr) = 0$
 $\text{PQueryCounter } (\text{verifier-initial-state } tr) = 0$
unfolding *verifier-initial-state-def* **by** *simp-all*

definition *alpha-future-fresh*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{alpha-future-fresh } s \longleftrightarrow$
 $(\forall i x.$
 $\quad \text{PAlphaCounter } s \leq i \longrightarrow$
 $\quad \text{fmlookup } (\text{HashMap } s) (\text{AlphaChallenge } i x) = \text{None})$

definition *trace-fri-future-fresh*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{trace-fri-future-fresh } s \longleftrightarrow$
 $(\forall i x.$
 $\quad \text{PTraceFriCounter } s \leq i \longrightarrow$
 $\quad \text{fmlookup } (\text{HashMap } s) (\text{TraceFriChallenge } i x) = \text{None})$

definition *composition-fri-future-fresh*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{composition-fri-future-fresh } s \longleftrightarrow$
 $(\forall i x.$
 $\quad \text{PCompositionFriCounter } s \leq i \longrightarrow$
 $\quad \text{fmlookup } (\text{HashMap } s) (\text{CompositionFriChallenge } i x) = \text{None})$

definition *query-future-fresh*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{query-future-fresh } s \longleftrightarrow$
 $(\forall i x.$
 $\quad \text{PQueryCounter } s \leq i \longrightarrow$
 $\quad \text{fmlookup } (\text{HashMap } s) (\text{QueryIndexChallenge } i x) = \text{None})$

lemma *alpha-future-fresh-current:*

assumes *alpha-future-fresh* *s*

shows

$\text{fmlookup } (\text{HashMap } s)$
 $(\text{AlphaChallenge } (\text{PAlphaCounter } s) (\text{PState } s)) = \text{None}$

using *assms* **unfolding** *alpha-future-fresh-def* **by** *simp*

lemma *query-future-fresh-current:*

```

assumes query-future-fresh s
shows
  fmlookup (HashMap s)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = None
using assms unfolding query-future-fresh-def by simp

```

```

lemma verifier-initial-alpha-future-fresh[simp]:
  alpha-future-fresh (verifier-initial-state tr)
unfolding alpha-future-fresh-def by simp

```

```

lemma verifier-initial-trace-fri-future-fresh[simp]:
  trace-fri-future-fresh (verifier-initial-state tr)
unfolding trace-fri-future-fresh-def by simp

```

```

lemma verifier-initial-composition-fri-future-fresh[simp]:
  composition-fri-future-fresh (verifier-initial-state tr)
unfolding composition-fri-future-fresh-def by simp

```

```

lemma verifier-initial-query-future-fresh[simp]:
  query-future-fresh (verifier-initial-state tr)
unfolding query-future-fresh-def by simp

```

The current bound has no separate Merkle-binding error term. Merkle binding is represented by the conditional premise *initial-merkle-binding-no-bad* below, because the present hash-map model does not assign a collision-resistance probability to Merkle roots.

4.1 Proof structure for soundness

The proof should decompose an accepting verifier execution for a false statement into four bad events:

- the trace FRI accepts a table that is not the evaluation of a trace polynomial satisfying *degree f < clength*;
- the composition FRI accepts a table that is not low-degree;
- the random linear combination hides a violated constraint;
- the verifier's sampled query indices miss all disagreement points.

The following predicates and lemmas are the intended interface between the probabilistic proof over *verify-monad* and the deterministic algebraic arguments.

```

definition accepted :: ('x × 's) option ⇒ bool
where accepted out ⇔ ¬ Option.is-none out

```

```

definition trace-table-low-degree :: 'f list ⇒ bool
where
  trace-table-low-degree ys ⇔
    (∃ f. degree f < clength ∧ ys = map (poly f) eval-domain)

```

definition *composition-table-low-degree* :: nat $\Rightarrow$ 'f list $\Rightarrow$ bool
where
composition-table-low-degree d ys $\longleftrightarrow$
 $(\exists q. \text{degree } q \leq d \wedge \text{ys} = \text{map } (\text{poly } q) \text{ eval-domain})$

definition *trace-table-valid* :: 'f list $\Rightarrow$ bool
where
trace-table-valid ys $\longleftrightarrow$
 $(\exists f. \text{degree } f < \text{clength} \wedge$
 $\text{ys} = \text{map } (\text{poly } f) \text{ eval-domain} \wedge$
 $(\forall (c, \text{roots}, d) \in \text{set spec.}$
 $\text{trace-satisfies-constraint } f \text{ c roots}))$

definition *query-consistent-at* :: 'f list $\Rightarrow$ 'f list $\Rightarrow$ 'f list $\Rightarrow$ nat $\Rightarrow$ bool
where
query-consistent-at trace-table composition-table as idx $\longleftrightarrow$
 $\text{idx} < \text{clength} * \text{scale} \wedge$
 $\text{length trace-table} = \text{clength} * \text{scale} \wedge$
 $\text{length composition-table} = \text{clength} * \text{scale} \wedge$
 $(\forall i \in \text{set } (\text{powers-scaled idx}). i < \text{length trace-table}) \wedge$
 $\text{composition-table} ! \text{idx} =$
 $\text{cp-eval as } (\text{map } (!) \text{ trace-table}) (\text{powers-scaled idx}) (h \wedge \text{idx} * \text{shift})$

lemma *query-consistent-at-cong-opened-values*:

assumes *trace-len*:
 $\text{length trace-table} = \text{length trace-table}'$
and *composition-len*:
 $\text{length composition-table} = \text{length composition-table}'$
and *trace-values*:
 $\text{map } (!) \text{ trace-table} (\text{powers-scaled idx}) =$
 $\text{map } (!) \text{ trace-table}' (\text{powers-scaled idx})$
and *composition-value*:
 $\text{composition-table} ! \text{idx} = \text{composition-table}' ! \text{idx}$

shows

$\text{query-consistent-at trace-table composition-table as idx} \longleftrightarrow$
 $\text{query-consistent-at trace-table}' \text{ composition-table}' \text{ as idx}$

proof –

have *cp-eq*:
 $\text{cp-eval as } (\text{map } (!) \text{ trace-table}) (\text{powers-scaled idx})$
 $(h \wedge \text{idx} * \text{shift}) =$
 $\text{cp-eval as } (\text{map } (!) \text{ trace-table}') (\text{powers-scaled idx})$
 $(h \wedge \text{idx} * \text{shift})$
by (rule arg-cong[**where**
 $f = \lambda xs. \text{cp-eval as } xs (h \wedge \text{idx} * \text{shift}), \text{ OF trace-values}]$)
show ?thesis
unfolding *query-consistent-at-def*
using *trace-len composition-len composition-value cp-eq* **by** simp
qed

definition *all-queries-consistent* :: 'f list $\Rightarrow$ 'f list $\Rightarrow$ 'f list $\Rightarrow$ bool

where

all-queries-consistent trace-table composition-table as $\longleftrightarrow$
 $(\forall \text{idx} \in \text{query-sample-space}.$
 $\text{query-consistent-at trace-table composition-table as idx})$

definition *bad-query-indices* :: 'f list $\Rightarrow$ 'f list $\Rightarrow$ 'f list $\Rightarrow$ nat set

where

bad-query-indices trace-table composition-table as =
 $\{\text{idx} \in \text{query-sample-space}.$
 $\neg \text{query-consistent-at trace-table composition-table as idx}\}$

definition *query-agreement-indices* :: 'f list $\Rightarrow$ 'f list $\Rightarrow$ 'f list $\Rightarrow$ nat set

where

query-agreement-indices trace-table composition-table as =
 $\{\text{idx} \in \text{query-sample-space}.$
 $\text{query-consistent-at trace-table composition-table as idx}\}$

definition *query-disagreement-indices* :: 'f list $\Rightarrow$ 'f list $\Rightarrow$ 'f list $\Rightarrow$ nat set

where

query-disagreement-indices trace-table composition-table as =
bad-query-indices trace-table composition-table as

definition *query-samples-miss-disagreements*

:: 'f list $\Rightarrow$ 'f list $\Rightarrow$ 'f list $\Rightarrow$ nat list $\Rightarrow$ bool

where

query-samples-miss-disagreements trace-table composition-table as query-idxs $\longleftrightarrow$
 $\text{query-disagreement-indices trace-table composition-table as} \neq \{\}$ $\wedge$
 $\text{set query-idxs} \subseteq \text{query-agreement-indices trace-table composition-table as}$

definition *violated-constraints*

:: 'f poly $\Rightarrow$ (('f poly list $\Rightarrow$ 'f poly) $\times$ nat list $\times$ nat) set

where

violated-constraints f =
 $\{\text{entry} \in \text{set spec}.$
 $\text{case entry of } (c, \text{roots}, d) \Rightarrow$
 $\neg \text{trace-satisfies-constraint f c roots}\}$

definition *constraint-remainder-bad*

:: 'f poly $\Rightarrow$ (('f poly list $\Rightarrow$ 'f poly) $\times$ nat list $\times$ nat) $\Rightarrow$ bool

where

constraint-remainder-bad f entry $\longleftrightarrow$
 $(\text{case entry of } (c, \text{roots}, d) \Rightarrow$
 $(\exists r \in \text{set roots}.$
 $\text{poly } (c \text{ (trace-powers-of f)}) (g \wedge r) \neq 0))$

definition *composition-vanishes-on-constraint-domain* :: 'f poly $\Rightarrow$ 'f list $\Rightarrow$ bool

where

composition-vanishes-on-constraint-domain f as $\longleftrightarrow$
 $(\forall x \in \text{set } G. \text{poly } (cp \text{ as } (\text{trace-powers-of } f)) \ x = 0)$

definition *constraint-root-residual*

$:: 'f \text{ poly} \Rightarrow \text{nat} \Rightarrow$
 $('f \times (('f \text{ poly list} \Rightarrow 'f \text{ poly}) \times \text{nat list} \times \text{nat})) \Rightarrow 'f$

where

constraint-root-residual f r *weighted-entry* =
 $(\text{case weighted-entry of } (a, (c, \text{roots}, d)) \Rightarrow$
 $\text{if } r \in \text{set roots}$
 $\text{then } a * \text{poly } (c \ (\text{trace-powers-of } f)) \ (g \wedge r)$
 $\text{else } 0)$

definition *composition-root-residual* $:: 'f \text{ poly} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f$

where

composition-root-residual f as r =
 $\text{sum-list } (\text{map } (\text{constraint-root-residual } f \ r) \ (\text{zip as spec}))$

definition *random-combination-hides-violations* $:: 'f \text{ poly} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

random-combination-hides-violations f as $\longleftrightarrow$
 $\text{violated-constraints } f \neq \{\}$ $\wedge$
 $(\forall \text{entry} \in \text{violated-constraints } f.$
 $\text{case entry of } (c, \text{roots}, d) \Rightarrow$
 $(\forall r \in \text{set roots. composition-root-residual } f \text{ as } r = 0))$

definition *constraint-root-cofactor* $:: \text{nat list} \Rightarrow 'f \text{ poly}$

where

constraint-root-cofactor roots =
 $\text{prod } (\text{filter } (\lambda x. x \notin \text{set } (g\text{-map roots})) \ \text{all-constraint-roots})$

definition *common-denominator-poly* $:: 'f \text{ poly}$

where *common-denominator-poly* = $\text{prod all-constraint-roots}$

definition *common-denominator-numerator-term*

$:: 'f \text{ poly} \Rightarrow$
 $('f \times (('f \text{ poly list} \Rightarrow 'f \text{ poly}) \times \text{nat list} \times \text{nat})) \Rightarrow$
 $'f \text{ poly}$

where

common-denominator-numerator-term f *weighted-entry* =
 $(\text{case weighted-entry of } (a, (c, \text{roots}, d)) \Rightarrow$
 $CP \ a * c \ (\text{trace-powers-of } f) * \text{constraint-root-cofactor roots})$

definition *common-denominator-numerator* $:: 'f \text{ poly} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ poly}$

where

common-denominator-numerator f as =
 sum-list
 $(\text{map } (\text{common-denominator-numerator-term } f) \ (\text{zip as spec}))$

definition *composition-spec-eval-term*

$:: 'f \text{ poly} \Rightarrow 'f \Rightarrow$
 $(('f \times ((('f \text{ poly list} \Rightarrow 'f \text{ poly}) \times \text{nat list} \times \text{nat})) \Rightarrow$
 $'f$

where

composition-spec-eval-term $f \ x \ \text{weighted-entry} =$
 $(\text{case weighted-entry of } (a, (c, \text{roots}, d)) \Rightarrow$
 $a * (\text{poly } (c \ (\text{trace-powers-of } f)) \ x \ /$
 $\text{poly } (\text{prod } (g\text{-map roots}) \ x))$

definition *composition-spec-eval* $:: 'f \text{ poly} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f$

where

composition-spec-eval $f \ \text{as} \ x =$
 $\text{sum-list } (\text{map } (\text{composition-spec-eval-term } f \ x) \ (\text{zip as spec}))$

definition *common-denominator-constraint-root-residual*

$:: 'f \text{ poly} \Rightarrow \text{nat} \Rightarrow$
 $(('f \times ((('f \text{ poly list} \Rightarrow 'f \text{ poly}) \times \text{nat list} \times \text{nat})) \Rightarrow 'f$

where

common-denominator-constraint-root-residual $f \ r \ \text{weighted-entry} =$
 $(\text{case weighted-entry of } (a, (c, \text{roots}, d)) \Rightarrow$
 $\text{if } r \in \text{set roots}$
 then
 $a * \text{poly } (c \ (\text{trace-powers-of } f)) \ (g \wedge r) *$
 $\text{poly } (\text{constraint-root-cofactor roots}) \ (g \wedge r)$
 $\text{else } 0)$

definition *common-denominator-composition-root-residual*

$:: 'f \text{ poly} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f$

where

common-denominator-composition-root-residual $f \ \text{as} \ r =$
 sum-list
 $(\text{map } (\text{common-denominator-constraint-root-residual } f \ r) \ (\text{zip as spec}))$

definition *random-combination-common-denominator-hides-violations*

$:: 'f \text{ poly} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

random-combination-common-denominator-hides-violations $f \ \text{as} \longleftrightarrow$
 $\text{violated-constraints } f \neq \{\} \wedge$
 $(\forall \text{entry} \in \text{violated-constraints } f.$
 $\text{case entry of } (c, \text{roots}, d) \Rightarrow$
 $(\forall r \in \text{set roots.}$
 $\text{common-denominator-composition-root-residual } f \ \text{as} \ r = 0))$

definition *common-denominator-degree-bounds* $:: 'f \text{ poly} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

common-denominator-degree-bounds $f \ \text{as} \longleftrightarrow$
 $\text{degree } (\text{common-denominator-numerator } f \ \text{as}) < \text{query-sample-space-size} \wedge$
 $(\forall q. \text{degree } q \leq \text{maxDegree} \longrightarrow$

$\text{degree } (\text{common-denominator-poly} * q) < \text{query-sample-space-size}$

definition *query-agreement-margin* :: bool

where

query-agreement-margin $\longleftrightarrow$
query-agreement-bound < *query-sample-space-size*

definition *spec-degree-wellformed* :: bool

where

spec-degree-wellformed $\longleftrightarrow$ *query-agreement-margin*

lemma *spec-degree-wellformedI*:

assumes *query-agreement-margin*

shows *spec-degree-wellformed*

using *assms* **unfolding** *spec-degree-wellformed-def* **by** *simp*

lemma *spec-degree-wellformed-query-agreement-margin*:

assumes *spec-degree-wellformed*

shows *query-agreement-margin*

using *assms* **unfolding** *spec-degree-wellformed-def* **by** *simp*

lemma *spec-degree-wellformed-constraint-degree*:

assumes *wf*: *spec-degree-wellformed*

and *deg-f*: *degree f* < *clength*

and *spec-entry*: $(c, \text{roots}, d) \in \text{set spec}$

shows *degree (c (trace-powers-of f))* $\leq d * (\text{clength} - 1)$

by (rule *constraint-degree-wellformed[OF deg-f spec-entry]*)

lemma *query-agreement-marginD*:

assumes *query-agreement-margin*

shows *query-agreement-bound* < *query-sample-space-size*

using *assms* **unfolding** *query-agreement-margin-def* **by** *simp*

lemma *spec-degree-wellformed-query-margin*:

assumes *spec-degree-wellformed*

shows *query-agreement-bound* < *query-sample-space-size*

by (rule *query-agreement-marginD*

[*OF spec-degree-wellformed-query-agreement-margin[OF assms]*])

lemma *all-constraint-roots-length-le-sum-roots*:

length all-constraint-roots $\leq$

sum-list (map ($\lambda(-, \text{roots}, -). \text{length roots}$) spec)

proof –

have *length all-constraint-roots* $\leq$

length (List.concat (map ($\lambda(c, \text{roots}, d). g\text{-map roots}$) spec))

unfolding *all-constraint-roots-def*

by (rule *length-remdups-leq*)

also have ... =

sum-list (map ($\lambda(-, \text{roots}, -). \text{length roots}$) spec)


```

    unfolding g-map-def
    by (induction spec) auto
    finally show ?thesis .
qed

```

```

lemma spec-degree-wellformed-from-spec-query-margin:
  spec-degree-wellformed
proof -
  have margin: query-agreement-bound < query-sample-space-size
    using spec-query-margin all-constraint-roots-length-le-sum-roots
    unfolding query-agreement-bound-def query-sample-space-size-def
    maxDegree-def
    by linarith
  then show ?thesis
    unfolding spec-degree-wellformed-def query-agreement-margin-def by simp
qed

```

```

lemma exists-valid-traceI:
  assumes degree f < clength
    and  $\bigwedge c \text{ roots } d. (c, \text{roots}, d) \in \text{set spec} \implies$ 
      trace-satisfies-constraint f c roots
  shows exists-valid-trace
  using assms unfolding exists-valid-trace-def by auto

```

```

lemma trace-table-valid-imp-exists-valid-trace:
  assumes trace-table-valid ys
  shows exists-valid-trace
  using assms unfolding trace-table-valid-def
  by (auto intro!: exists-valid-traceI)

```

```

lemma false-statement-violated-constraints:
  assumes false-statement:  $\neg$  exists-valid-trace
    and low-degree: degree f < clength
  shows violated-constraints f  $\neq \{\}$ 
proof
  assume empty: violated-constraints f =  $\{\}$ 
  have  $\bigwedge c \text{ roots } d. (c, \text{roots}, d) \in \text{set spec} \implies$ 
    trace-satisfies-constraint f c roots
    using empty unfolding violated-constraints-def by force
  then have exists-valid-trace
    using low-degree unfolding exists-valid-trace-def by auto
  then show False
    using false-statement by contradiction
qed

```

```

lemma constraint-remainder-bad-iff-not-trace-satisfies:
  constraint-remainder-bad f (c, roots, d)  $\longleftrightarrow$ 
     $\neg$  trace-satisfies-constraint f c roots
  unfolding constraint-remainder-bad-def trace-satisfies-constraint-def

```

by auto

lemma *violated-constraints-has-remainder-bad*:
 assumes *violated-constraints* $f \neq \{\}$
 shows $\exists \text{entry} \in \text{violated-constraints } f. \text{constraint-remainder-bad } f \text{ entry}$
 using *assms*
 unfolding *violated-constraints-def*
 by (auto simp: *constraint-remainder-bad-iff-not-trace-satisfies*)

lemma *random-combination-hides-violations-has-remainder-bad*:
 assumes *random-combination-hides-violations* $f \text{ as}$
 shows $\exists \text{entry} \in \text{violated-constraints } f. \text{constraint-remainder-bad } f \text{ entry}$
 using *assms violated-constraints-has-remainder-bad*
 unfolding *random-combination-hides-violations-def*
 by *blast*

lemma *linear-equation-solution-unique*:
 fixes $c \ b \ x \ y :: 'f$
 assumes *c-nonzero*: $c \neq 0$
 and *x-sol*: $c * x + b = 0$
 and *y-sol*: $c * y + b = 0$
 shows $x = y$
proof –
 have *x-eq*: $c * x = - b$
 proof –
 have $c * x + b + (- b) = 0 + (- b)$
 using *x-sol* **by** *simp*
 then show *?thesis*
 by *simp*
 qed
 have *y-eq*: $c * y = - b$
 proof –
 have $c * y + b + (- b) = 0 + (- b)$
 using *y-sol* **by** *simp*
 then show *?thesis*
 by *simp*
 qed
 have $c * (x - y) = 0$
 using *x-eq y-eq* **by** (*simp add: algebra-simps*)
 then have $x - y = 0$
 using *c-nonzero* **by** *simp*
 then show *?thesis*
 by *simp*
qed

lemma *composition-root-residual-sum-nth*:
 assumes *len*: $\text{length } as = \text{length } spec$
 shows
 $\text{composition-root-residual } f \text{ as } r =$

```

    ( $\sum i \in \{0..<\text{length spec}\}$ .
       $\text{constraint-root-residual } f \ r \ (as ! i, \text{spec} ! i)$ )
  proof -
    have  $\text{composition-root-residual } f \ as \ r =$ 
       $\text{sum-list } (\text{map } (\text{constraint-root-residual } f \ r) \ (\text{zip } as \ \text{spec}))$ 
    by (simp add:  $\text{composition-root-residual-def}$ )
    also have ... =
      ( $\sum i \in \{0..<\text{length } (\text{zip } as \ \text{spec})\}$ .
         $\text{map } (\text{constraint-root-residual } f \ r) \ (\text{zip } as \ \text{spec}) ! i$ )
    by (simp add:  $\text{sum-list-sum-nth}$ )
    also have ... =
      ( $\sum i \in \{0..<\text{length spec}\}$ .
         $\text{constraint-root-residual } f \ r \ (as ! i, \text{spec} ! i)$ )
    using len by (intro  $\text{sum.cong}$ ) auto
    finally show ?thesis .
  qed

lemma  $\text{composition-root-residual-split-index}$ :
  assumes len:  $\text{length } as = \text{length spec}$ 
    and i-bound:  $i < \text{length spec}$ 
  shows
     $\text{composition-root-residual } f \ as \ r =$ 
       $\text{constraint-root-residual } f \ r \ (as ! i, \text{spec} ! i) +$ 
      ( $\sum j \in \{0..<\text{length spec}\} - \{i\}$ .
         $\text{constraint-root-residual } f \ r \ (as ! j, \text{spec} ! j)$ )
  proof -
    let ?F =  $\lambda j. \text{constraint-root-residual } f \ r \ (as ! j, \text{spec} ! j)$ 
    have  $\text{composition-root-residual } f \ as \ r = (\sum j \in \{0..<\text{length spec}\}. ?F \ j)$ 
    by (rule  $\text{composition-root-residual-sum-nth[OF len]}$ )
    also have ... =  $?F \ i + (\sum j \in \{0..<\text{length spec}\} - \{i\}. ?F \ j)$ 
    using i-bound by (simp add:  $\text{sum.remove}$ )
    finally show ?thesis .
  qed

lemma  $\text{composition-root-residual-zero-unique-nth}$ :
  fixes as bs :: 'f list
  assumes len-as:  $\text{length } as = \text{length spec}$ 
    and len-bs:  $\text{length } bs = \text{length spec}$ 
    and i-bound:  $i < \text{length spec}$ 
    and spec-i:  $\text{spec} ! i = (c, \text{roots}, d)$ 
    and r-in:  $r \in \text{set roots}$ 
    and nonzero:
       $\text{poly } (c \ (\text{trace-powers-of } f)) \ (g \wedge r) \neq 0$ 
    and outside:
       $\bigwedge j. j < \text{length spec} \implies j \neq i \implies as ! j = bs ! j$ 
    and zero-as:  $\text{composition-root-residual } f \ as \ r = 0$ 
    and zero-bs:  $\text{composition-root-residual } f \ bs \ r = 0$ 
  shows  $as ! i = bs ! i$ 
  proof -

```

```

let ?C = poly (c (trace-powers-of f)) (g ^ r)
let ?B =
  (∑ j ∈ {0..<length spec} - {i}.
    constraint-root-residual f r (as ! j, spec ! j))
have B-eq:
  (∑ j ∈ {0..<length spec} - {i}.
    constraint-root-residual f r (bs ! j, spec ! j)) = ?B
  by (intro sum.cong) (use outside in auto)
have as-eq:
  composition-root-residual f as r = ?C * (as ! i) + ?B
  using composition-root-residual-split-index[OF len-as i-bound]
    spec-i r-in
  unfolding constraint-root-residual-def
  by (simp add: algebra-simps)
have bs-eq:
  composition-root-residual f bs r = ?C * (bs ! i) + ?B
  using composition-root-residual-split-index[OF len-bs i-bound]
    spec-i r-in B-eq
  unfolding constraint-root-residual-def
  by (simp add: algebra-simps)
show ?thesis
proof (rule linear-equation-solution-unique[OF nonzero])
  show ?C * (as ! i) + ?B = 0
    using as-eq zero-as by simp
  show ?C * (bs ! i) + ?B = 0
    using bs-eq zero-bs by simp
qed
qed

```

lemma *random-combination-hides-violations-unique-nth*:

```

fixes as bs :: 'f list
assumes len-as: length as = length spec
  and len-bs: length bs = length spec
  and i-bound: i < length spec
  and spec-i: spec ! i = (c, roots, d)
  and violated: (c, roots, d) ∈ violated-constraints f
  and r-in: r ∈ set roots
  and nonzero:
    poly (c (trace-powers-of f)) (g ^ r) ≠ 0
  and outside:
    ∧ j. j < length spec ⇒ j ≠ i ⇒ as ! j = bs ! j
  and hides-as: random-combination-hides-violations f as
  and hides-bs: random-combination-hides-violations f bs
shows as ! i = bs ! i

```

```

proof -
  have zero-as: composition-root-residual f as r = 0
    using hides-as violated r-in
  unfolding random-combination-hides-violations-def by auto
  have zero-bs: composition-root-residual f bs r = 0

```

```

    using hides-bs violated r-in
    unfolding random-combination-hides-violations-def by auto
show ?thesis
  by (rule composition-root-residual-zero-unique-nth
      [OF len-as len-bs i-bound spec-i r-in nonzero outside zero-as zero-bs])
qed

lemma bad-query-indices-subset:
  bad-query-indices trace-table composition-table as  $\subseteq \{0..<\text{clength} * \text{scale}\}$ 
  unfolding bad-query-indices-def query-sample-space-def query-sample-space-size-def
  by auto

lemma index-less-domain:
  index  $x < \text{clength} * \text{scale}$ 
proof -
  have max-lt:  $\text{Max} (\text{set } [0..<\text{powers}]) < \text{clength}$ 
  proof -
    have  $\text{Max} (\text{set } [0..<\text{powers}]) = \text{powers} - 1$ 
      using powers-pos by (intro Max-eqI) auto
    also have  $\dots < \text{clength}$ 
      using powers-pos powers-le-clength by simp
    finally show ?thesis .
  qed
  have mod-pos:  $0 < \text{clength} * \text{scale} - \text{Max} (\text{set } [0..<\text{powers}]) * \text{scale}$ 
    using max-lt scale-pos
    by (simp add: diff-mult-distrib2)
  have index  $x < \text{clength} * \text{scale} - \text{Max} (\text{set } [0..<\text{powers}]) * \text{scale}$ 
    unfolding index-def using mod-pos by simp
  moreover have  $\text{clength} * \text{scale} - \text{Max} (\text{set } [0..<\text{powers}]) * \text{scale} \leq \text{clength} * \text{scale}$ 
    by simp
  ultimately show ?thesis
    by linarith
qed

lemma query-sample-space-size-pos:
   $0 < \text{query-sample-space-size}$ 
proof -
  have max-lt:  $\text{Max} (\text{set } [0..<\text{powers}]) < \text{clength}$ 
  proof -
    have  $\text{Max} (\text{set } [0..<\text{powers}]) = \text{powers} - 1$ 
      using powers-pos by (intro Max-eqI) auto
    also have  $\dots < \text{clength}$ 
      using powers-pos powers-le-clength by simp
    finally show ?thesis .
  qed
  show ?thesis
    unfolding query-sample-space-size-def
    using max-lt scale-pos by (simp add: diff-mult-distrib2)
qed

```

lemma *index-less-query-sample-space*:
index $x < \text{query-sample-space-size}$
unfolding *index-def query-sample-space-size-def*
using *query-sample-space-size-pos*[*unfolded query-sample-space-size-def*] **by** *simp*

lemma *query-sample-space-size-le-domain*:
query-sample-space-size $\leq \text{clength} * \text{scale}$
unfolding *query-sample-space-size-def* **by** *simp*

lemma *query-sample-spaceD*:
assumes *idx* $\in \text{query-sample-space}$
shows *idx* $< \text{query-sample-space-size}$
using *assms* **unfolding** *query-sample-space-def* **by** *simp*

lemma *finite-query-sample-space*[*simp*]:
finite query-sample-space
unfolding *query-sample-space-def* **by** *simp*

lemma *card-query-sample-space*[*simp*]:
card query-sample-space $= \text{query-sample-space-size}$
unfolding *query-sample-space-def* **by** *simp*

lemma *query-error-bound-query-sample-space*:
query-error-bound $=$
 $\text{nnreal } \text{query-agreement-bound} / \text{nnreal } (\text{card } \text{query-sample-space})$
unfolding *query-error-bound-def* **by** *simp*

lemma *query-sample-space-less-domain*:
assumes *idx* $\in \text{query-sample-space}$
shows *idx* $< \text{clength} * \text{scale}$
using *query-sample-spaceD*[*OF assms*] *query-sample-space-size-le-domain*
by *linarith*

lemma *query-sample-space-powers-scaled-bound*:
assumes *idx-in*: *idx* $\in \text{query-sample-space}$
and *i-in*: *i* $\in \text{set } (\text{powers-scaled } \text{idx})$
shows *i* $< \text{clength} * \text{scale}$
proof –
obtain *k* **where** *k-bound*: *k* $< \text{powers}$ **and** *i-eq*: *i* $= \text{idx} + k * \text{scale}$
using *i-in* **unfolding** *powers-scaled-def* **by** *auto*
have *max-eq*: $\text{Max } (\text{set } [0..<\text{powers}]) = \text{powers} - 1$
using *powers-pos* **by** (*intro Max-eqI*) *auto*
have *k-le-max*: *k* $\leq \text{Max } (\text{set } [0..<\text{powers}])$
using *k-bound* **unfolding** *max-eq* **by** *simp*
have *idx-bound*:
 $\text{idx} < \text{clength} * \text{scale} - \text{Max } (\text{set } [0..<\text{powers}]) * \text{scale}$
using *query-sample-spaceD*[*OF idx-in*]
unfolding *query-sample-space-size-def* .

```

have  $idx + k * scale \leq idx + \text{Max}(\text{set } [0..<powers]) * scale$ 
  using k-le-max by simp
also have  $\dots < \text{clength} * scale$ 
  using idx-bound by simp
finally show ?thesis
  unfolding i-eq .
qed

definition query-sample-domain :: 'f list
  where query-sample-domain = map ( $\lambda idx. h \wedge idx * \text{shift}$ )  $[0..<query\text{-}sample\text{-}space\text{-}size]$ 

lemma eval-domain-length:
   $\text{length } eval\text{-}domain = \text{clength} * scale$ 
  unfolding eval-domain-def H-def by simp

lemma eval-domain-nth:
  assumes  $i < \text{clength} * scale$ 
  shows  $eval\text{-}domain ! i = h \wedge i * \text{shift}$ 
  using assms
  unfolding eval-domain-def H-def
  by (simp add: mult.commute)

lemma h-nonzero:
   $h \neq 0$ 
  unfolding h-def using omega-nonzero-derived by simp

lemma h-power-inj-on-eval-domain:
  assumes i-bound:  $i < \text{clength} * scale$ 
    and j-bound:  $j < \text{clength} * scale$ 
    and eq:  $h \wedge i = h \wedge j$ 
  shows  $i = j$ 
proof (rule ccontr)
  let ?N =  $\text{clength} * scale$ 
  assume neg:  $i \neq j$ 
  consider  $i < j \mid j < i$ 
    using neg by linarith
  then show False
proof cases
  case 1
  have diff-pos:  $0 < j - i$ 
    using 1 by simp
  have diff-lt:  $j - i < ?N$ 
    using j-bound by simp
  have hi-nonzero:  $h \wedge i \neq 0$ 
    using h-nonzero by simp
  have j-decomp:  $j = i + (j - i)$ 
    using 1 by simp
  have mult-eq:  $h \wedge j = h \wedge i * h \wedge (j - i)$ 
    by (subst j-decomp) (simp add: power-add)

```

```

have cancel-eq:  $h^i * 1 = h^i * h^{(j-i)}$ 
  using eq mult-eq by simp
have  $h^i = 0 \vee 1 = h^{(j-i)}$ 
  using cancel-eq by (simp only: mult-cancel-left)
then have  $h^{(j-i)} = 1$ 
  using hi-nonzero by auto
then show False
  using h-exact-order diff-pos diff-lt unfolding exact-order-def by blast
next
case 2
have diff-pos:  $0 < i - j$ 
  using 2 by simp
have diff-lt:  $i - j < ?N$ 
  using i-bound by simp
have hj-nonzero:  $h^j \neq 0$ 
  using h-nonzero by simp
have i-decomp:  $i = j + (i - j)$ 
  using 2 by simp
have mult-eq:  $h^i = h^j * h^{(i-j)}$ 
  by (subst i-decomp) (simp add: power-add)
have cancel-eq:  $h^j * 1 = h^j * h^{(i-j)}$ 
  using eq mult-eq by simp
have  $h^j = 0 \vee 1 = h^{(i-j)}$ 
  using cancel-eq by (simp only: mult-cancel-left)
then have  $h^{(i-j)} = 1$ 
  using hj-nonzero by auto
then show False
  using h-exact-order diff-pos diff-lt unfolding exact-order-def by blast
qed
qed

lemma H-distinct:
  distinct H
proof -
  have inj: inj-on (( $\wedge$ ) h) (set [0.. $\text{scale} * \text{clength}$ ])
  proof (intro inj-onI)
    fix x y
    assume x-in:  $x \in \text{set } [0.. $\text{scale} * \text{clength}$ ]$ 
    and y-in:  $y \in \text{set } [0.. $\text{scale} * \text{clength}$ ]$ 
    and eq:  $h^x = h^y$ 
    have x-bound:  $x < \text{clength} * \text{scale}$ 
      using x-in by (simp add: mult.commute)
    have y-bound:  $y < \text{clength} * \text{scale}$ 
      using y-in by (simp add: mult.commute)
    show  $x = y$ 
      by (rule h-power-inj-on-eval-domain[OF x-bound y-bound eq])
  qed
show ?thesis
  unfolding H-def using inj by (simp add: distinct-map)

```


qed

lemma *eval-domain-distinct*:

distinct eval-domain

proof –

have *inj*: *inj-on* $((*) \text{ shift})$ (*set H*)

using *shift-nonzero* **by** (*auto intro!*: *inj-onI*)

show *?thesis*

unfolding *eval-domain-def*

using *H-distinct inj* **by** (*simp add: distinct-map*)

qed

lemma *card-set-eval-domain*:

*card (set eval-domain) = clength * scale*

using *eval-domain-distinct eval-domain-length*

by (*simp add: distinct-card*)

lemma *poly-eq-on-eval-domainI*:

assumes *deg-p*: *degree p < clength * scale*

and *deg-q*: *degree q < clength * scale*

and *agree*:

$\bigwedge \text{idx. } \text{idx} < \text{clength} * \text{scale} \implies$

$\text{poly } p (h \wedge \text{idx} * \text{shift}) = \text{poly } q (h \wedge \text{idx} * \text{shift})$

shows $p = q$

proof (*rule poly-eqI-degree*[**where** $A = \text{set eval-domain}$])

fix *x*

assume *x-in*: $x \in \text{set eval-domain}$

then obtain *idx* **where** *idx-bound*: $\text{idx} < \text{clength} * \text{scale}$

and *x-eq*: $x = h \wedge \text{idx} * \text{shift}$

unfolding *eval-domain-def H-def*

by (*auto simp: mult.commute*)

show $\text{poly } p x = \text{poly } q x$

using *agree*[*OF idx-bound*] *x-eq* **by** *simp*

next

show $\text{degree } p < \text{card} (\text{set eval-domain})$

using *deg-p card-set-eval-domain* **by** *simp*

next

show $\text{degree } q < \text{card} (\text{set eval-domain})$

using *deg-q card-set-eval-domain* **by** *simp*

qed

lemma *query-sample-domain-distinct*:

distinct query-sample-domain

proof –

have *inj*: *inj-on* $(\lambda \text{idx. } h \wedge \text{idx} * \text{shift})$ (*set* $[0..<\text{query-sample-space-size}]$)

proof (*intro inj-onI*)

fix *i j*

assume *i-in*: $i \in \text{set } [0..<\text{query-sample-space-size}]$

and *j-in*: $j \in \text{set } [0..<\text{query-sample-space-size}]$

```

    and eq:  $h \wedge i * \text{shift} = h \wedge j * \text{shift}$ 
  have i-bound:  $i < \text{clength} * \text{scale}$ 
    using i-in query-sample-space-size-le-domain by simp
  have j-bound:  $j < \text{clength} * \text{scale}$ 
    using j-in query-sample-space-size-le-domain by simp
  have  $h \wedge i = h \wedge j$ 
    using eq shift-nonzero by simp
  then show  $i = j$ 
    by (rule h-power-inj-on-eval-domain[OF i-bound j-bound])
qed
show ?thesis
  unfolding query-sample-domain-def using inj by (simp add: distinct-map)
qed

lemma card-set-query-sample-domain:
  card (set query-sample-domain) = query-sample-space-size
proof -
  have card (set query-sample-domain) = length query-sample-domain
    using query-sample-domain-distinct by (simp add: distinct-card)
  also have ... = query-sample-space-size
    unfolding query-sample-domain-def by simp
  finally show ?thesis .
qed

lemma poly-eq-on-query-sample-domainI:
  assumes deg-p: degree  $p < \text{query-sample-space-size}$ 
  and deg-q: degree  $q < \text{query-sample-space-size}$ 
  and agree:
     $\bigwedge \text{idx}. \text{idx} \in \text{query-sample-space} \implies$ 
     $\text{poly } p (h \wedge \text{idx} * \text{shift}) = \text{poly } q (h \wedge \text{idx} * \text{shift})$ 
  shows  $p = q$ 
proof (rule poly-eqI-degree[where  $A = \text{set query-sample-domain}$ ])
  fix x
  assume x-in:  $x \in \text{set query-sample-domain}$ 
  then obtain idx where idx-bound:  $\text{idx} < \text{query-sample-space-size}$ 
    and x-eq:  $x = h \wedge \text{idx} * \text{shift}$ 
    unfolding query-sample-domain-def by auto
  have idx-in:  $\text{idx} \in \text{query-sample-space}$ 
    using idx-bound unfolding query-sample-space-def by simp
  show  $\text{poly } p x = \text{poly } q x$ 
    using agree[OF idx-in] x-eq by simp
next
  show degree  $p < \text{card (set query-sample-domain)}$ 
    using deg-p card-set-query-sample-domain by simp
next
  show degree  $q < \text{card (set query-sample-domain)}$ 
    using deg-q card-set-query-sample-domain by simp
qed

```

```

lemma prod-fold-acc:
  fixes roots :: 'f list
  shows  $\text{fold } (\lambda s \text{ acc. } (XP\ 1 - CP\ s) * acc) \text{ roots } acc =$ 
     $\text{fold } (\lambda s \text{ acc. } (XP\ 1 - CP\ s) * acc) \text{ roots } (1 :: 'f\ poly) * acc$ 
proof (induction roots arbitrary: acc)
  case Nil
  then show ?case by simp
next
  case (Cons r roots)
  let ?F =  $\lambda s \text{ acc. } (XP\ 1 - CP\ s) * acc$ 
  let ?fac =  $XP\ 1 - CP\ r$ 
  have lhs:
     $\text{fold } ?F\ (r \# \text{roots})\ acc =$ 
       $\text{fold } ?F\ \text{roots}\ 1 * (?fac * acc)$ 
    using Cons.IH[of ?fac * acc] by simp
  have  $\text{fold } ?F\ (r \# \text{roots})\ acc =$ 
     $(\text{fold } ?F\ \text{roots}\ 1 * ?fac) * acc$ 
    using lhs by (simp only: mult.assoc)
  also have ... =  $\text{fold } ?F\ \text{roots}\ ?fac * acc$ 
    using Cons.IH[of ?fac, symmetric] by simp
  also have ... =  $\text{fold } ?F\ (r \# \text{roots})\ 1 * acc$ 
    by simp
  finally show ?case .
qed

```

```

lemma prod-Cons:
  fixes r :: 'f and roots :: 'f list
  shows  $\text{prod } (r \# \text{roots}) = (XP\ 1 - CP\ r) * (\text{prod } \text{roots} :: 'f\ poly)$ 
proof -
  let ?F =  $\lambda s \text{ acc. } (XP\ 1 - CP\ s) * acc$ 
  have  $\text{prod } (r \# \text{roots}) = \text{fold } ?F\ \text{roots}\ ((XP\ 1 - CP\ r) * 1)$ 
    unfolding prod-def by simp
  also have ... =  $\text{fold } ?F\ \text{roots}\ 1 * ((XP\ 1 - CP\ r) * 1)$ 
    by (rule prod-fold-acc)
  also have ... =  $(\text{prod } \text{roots} :: 'f\ poly) * (XP\ 1 - CP\ r)$ 
    unfolding prod-def by simp
  also have ... =  $(XP\ 1 - CP\ r) * (\text{prod } \text{roots} :: 'f\ poly)$ 
    by (rule mult.commute)
  finally show ?thesis .
qed

```

```

lemma prod-append:
  fixes xs ys :: 'f list
  shows  $\text{prod } (xs @ ys) = (\text{prod } xs :: 'f\ poly) * \text{prod } ys$ 
proof (induction xs)
  case Nil
  then show ?case
    unfolding prod-def by simp
next

```

```

    case (Cons x xs)
  then show ?case
    by (simp add: prod-Cons algebra-simps)
qed

lemma prod-alt:
  fixes roots :: 'f list
  shows prod roots =
    prod-list (map ( $\lambda r. XP\ 1 - CP\ r$ ) roots :: 'f poly list)
proof (induction roots)
  case Nil
  then show ?case
    unfolding prod-def by simp
next
  case (Cons r roots)
  then show ?case
    by (simp add: prod-Cons)
qed

lemma degree-prod-le-length:
  fixes roots :: 'f list
  shows degree (prod roots :: 'f poly)  $\leq$  length roots
proof (induction roots)
  case Nil
  then show ?case
    unfolding prod-def by simp
next
  case (Cons r roots)
  have factor: degree ( $XP\ 1 - CP\ r :: 'f poly$ )  $\leq 1$ 
  proof -
    have deg-x: degree (Polynomial.monom (1::'f) 1)  $\leq 1$ 
      by (rule degree-monom-le)
    have deg-c: degree (Polynomial.monom r 0)  $\leq 1$ 
    proof -
      have degree (Polynomial.monom r 0)  $\leq 0$ 
        by (rule degree-monom-le)
      then show ?thesis by simp
    qed
    show ?thesis
      unfolding XP-def CP-def
      by (rule degree-diff-le[OF deg-x deg-c])
  qed
  have degree (prod (r # roots) :: 'f poly) =
    degree (( $XP\ 1 - CP\ r$ ) * (prod roots :: 'f poly))
    by (simp add: prod-Cons)
  also have ...  $\leq$ 
    degree ( $XP\ 1 - CP\ r :: 'f poly$ ) + degree (prod roots :: 'f poly)
    by (rule degree-mult-le)
  also have ...  $\leq$  Suc (length roots)

```

```

    using factor Cons.IH by simp
  finally show ?case by simp
qed

lemma degree-sum-list-less:
  fixes ps :: 'f poly list
  assumes  $\bigwedge p. p \in \text{set } ps \implies \text{degree } p < n$ 
    and  $0 < n$ 
  shows  $\text{degree } (\text{sum-list } ps) < n$ 
  using assms
proof (induction ps)
  case Nil
  then show ?case by simp
next
  case (Cons p ps)
  have p-lt:  $\text{degree } p < n$ 
    using Cons.prem by simp
  have ps-lt:  $\text{degree } (\text{sum-list } ps) < n$ 
    by (rule Cons.IH) (use Cons.prem in auto)
  show ?case
    using degree-add-less[OF p-lt ps-lt] by simp
qed

```

```

lemma degree-sum-list-le:
  fixes ps :: 'f poly list
  assumes  $\bigwedge p. p \in \text{set } ps \implies \text{degree } p \leq n$ 
  shows  $\text{degree } (\text{sum-list } ps) \leq n$ 
  using assms
proof (induction ps)
  case Nil
  then show ?case by simp
next
  case (Cons p ps)
  have p-le:  $\text{degree } p \leq n$ 
    using Cons.prem by simp
  have ps-le:  $\text{degree } (\text{sum-list } ps) \leq n$ 
    by (rule Cons.IH) (use Cons.prem in auto)
  show ?case
  proof -
    have  $\text{degree } (p + \text{sum-list } ps) \leq \max (\text{degree } p) (\text{degree } (\text{sum-list } ps))$ 
      by (rule degree-add-le-max)
    also have  $\dots \leq n$ 
      using p-le ps-le by simp
    finally show ?thesis by simp
  qed
qed

```

```

lemma prod-mset-eq:
  fixes xs ys :: 'f list

```

```

assumes mset xs = mset ys
shows prod xs = (prod ys :: 'f poly)
using assms
by (simp add: prod-alt flip: prod-mset-prod-list)

```

```

lemma prod-eval-nonzero:
  fixes roots :: 'f list
  assumes x ∉ set roots
  shows poly (prod roots :: 'f poly) x ≠ 0
  using assms
proof (induction roots)
  case Nil
  then show ?case
    unfolding prod-def by simp
next
  case (Cons r roots)
  have x-ne: x ≠ r
    using Cons.prems by simp
  have x-notin: x ∉ set roots
    using Cons.prems by simp
  have poly (prod (r # roots) :: 'f poly) x =
    (x - r) * poly (prod roots :: 'f poly) x
    unfolding prod-Cons XP-def CP-def
    by (simp add: poly-monom)
  then show ?case
    using x-ne Cons.IH[OF x-notin] by simp
qed

```

```

lemma prod-eval-zero:
  fixes roots :: 'f list
  assumes x ∈ set roots
  shows poly (prod roots :: 'f poly) x = 0
  using assms
proof (induction roots)
  case Nil
  then show ?case by simp
next
  case (Cons r roots)
  show ?case
    proof (cases x = r)
      case True
      then show ?thesis
        unfolding prod-Cons XP-def CP-def
        by (simp add: poly-monom)
    next
      case False
      then have x ∈ set roots
        using Cons.prems by simp
      then show ?thesis

```

```

    using Cons.IH
    unfolding prod-Cons XP-def CP-def
    by (simp add: poly-monom)
  qed
qed

lemma query-denominator-nonzero:
  assumes spec-entry:  $(c, \text{roots}, d) \in \text{set spec}$ 
  and idx-bound:  $\text{idx} < \text{clength} * \text{scale}$ 
  shows poly (prod (g-map roots))  $(h \wedge \text{idx} * \text{shift}) \neq 0$ 
proof -
  have  $h \wedge \text{idx} * \text{shift} \notin \text{set (g-map roots)}$ 
  by (rule query-domain-disjoint[OF spec-entry idx-bound])
  then show ?thesis
  by (rule prod-eval-nonzero)
qed

lemma common-denominator-nonzero-on-eval-domain:
  assumes idx-bound:  $\text{idx} < \text{clength} * \text{scale}$ 
  shows poly common-denominator-poly  $(h \wedge \text{idx} * \text{shift}) \neq 0$ 
proof -
  have notin:  $h \wedge \text{idx} * \text{shift} \notin \text{set all-constraint-roots}$ 
  proof
    assume in-roots:  $h \wedge \text{idx} * \text{shift} \in \text{set all-constraint-roots}$ 
    then obtain c roots d where spec-entry:  $(c, \text{roots}, d) \in \text{set spec}$ 
    and in-g-map:  $h \wedge \text{idx} * \text{shift} \in \text{set (g-map roots)}$ 
    unfolding all-constraint-roots-def by (auto split: prod.splits)
    have  $h \wedge \text{idx} * \text{shift} \notin \text{set (g-map roots)}$ 
    by (rule query-domain-disjoint[OF spec-entry idx-bound])
    then show False
    using in-g-map by simp
  qed
  then show ?thesis
  unfolding common-denominator-poly-def
  by (rule prod-eval-nonzero)
qed

lemma g-power-in-g-map:
  assumes  $r \in \text{set roots}$ 
  shows  $g \wedge r \in \text{set (g-map roots)}$ 
  using assms unfolding g-map-def by simp

lemma all-constraint-roots-distinct[simp]:
  distinct all-constraint-roots
  unfolding all-constraint-roots-def by simp

lemma g-map-subset-all-constraint-roots:
  assumes spec-entry:  $(c, \text{roots}, d) \in \text{set spec}$ 
  shows  $\text{set (g-map roots)} \subseteq \text{set all-constraint-roots}$ 

```

```

using spec-entry
unfolding all-constraint-roots-def
by (auto split: prod.splits)

lemma common-denominator-factor:
  assumes spec-entry: (c, roots, d) ∈ set spec
  shows
    common-denominator-poly =
      prod (g-map roots) * constraint-root-cofactor roots
proof –
  let ?rest = filter (λx. x ∉ set (g-map roots)) all-constraint-roots
  have subset: set (g-map roots) ⊆ set all-constraint-roots
    by (rule g-map-subset-all-constraint-roots[OF spec-entry])
  have distinct-roots: distinct (g-map roots)
    by (rule g-map-distinct[OF spec-entry])
  have distinct-all: distinct all-constraint-roots
    by simp
  have distinct-split: distinct (g-map roots @ ?rest)
    using distinct-roots distinct-all subset by auto
  have set-split: set (g-map roots @ ?rest) = set all-constraint-roots
    using subset by auto
  have mset-eq:
    mset all-constraint-roots = mset (g-map roots @ ?rest)
    using distinct-all distinct-split set-split
    by (metis set-eq-iff-mset-eq-distinct)
  have common-denominator-poly = prod all-constraint-roots
    unfolding common-denominator-poly-def by simp
  also have ... = prod (g-map roots @ ?rest)
    by (rule prod-mset-eq[OF mset-eq])
  also have ... = prod (g-map roots) * prod ?rest
    by (rule prod-append)
  finally show ?thesis
    unfolding constraint-root-cofactor-def .
qed

```

```

lemma declared-quotient-degree-le-maxDegree:
  assumes spec-entry: (c, roots, d) ∈ set spec
  shows d * (clength - 1) - length roots ≤ maxDegree
proof –
  have deg-set:
     $\{d * (clength - 1) - length\ rs \mid a\ rs\ d. (a, rs, d) \in set\ spec\} =$ 
     $(\lambda(a, rs, d). d * (clength - 1) - length\ rs)\ `set\ spec$ 
    by force
  have finite-degrees: finite (degrees clength spec)
    unfolding degrees-def deg-set by simp
  have entry-degree:
    d * (clength - 1) - length roots ∈ degrees clength spec
    using spec-entry unfolding degrees-def by blast
  show ?thesis

```


unfolding *maxDegree-def*
by (rule *Max-ge*[*OF finite-degrees entry-degree*])
qed

lemma *constraint-roots-length-le-all-constraint-roots:*

assumes *spec-entry*: $(c, \text{roots}, d) \in \text{set spec}$

shows $\text{length roots} \leq \text{length all-constraint-roots}$

proof –

have *subset*: $\text{set } (g\text{-map roots}) \subseteq \text{set all-constraint-roots}$

by (rule *g-map-subset-all-constraint-roots*[*OF spec-entry*])

have *distinct-roots*: $\text{distinct } (g\text{-map roots})$

by (rule *g-map-distinct*[*OF spec-entry*])

have *card-le*:

$\text{card } (\text{set } (g\text{-map roots})) \leq \text{card } (\text{set all-constraint-roots})$

by (rule *card-mono*) (use *subset* **in** *auto*)

have $\text{length roots} = \text{length } (g\text{-map roots})$

unfolding *g-map-def* **by** *simp*

also have $\dots = \text{card } (\text{set } (g\text{-map roots}))$

using *distinct-roots* **by** (*simp add: distinct-card*)

also have $\dots \leq \text{card } (\text{set all-constraint-roots})$

by (rule *card-le*)

also have $\dots = \text{length all-constraint-roots}$

by (*simp add: distinct-card*)

finally show *?thesis* .

qed

lemma *constraint-root-cofactor-degree-le:*

assumes *spec-entry*: $(c, \text{roots}, d) \in \text{set spec}$

shows

$\text{degree } (\text{constraint-root-cofactor roots}) \leq$
 $\text{length all-constraint-roots} - \text{length roots}$

proof –

let *?rest* = $\text{filter } (\lambda x. x \notin \text{set } (g\text{-map roots})) \text{ all-constraint-roots}$

have *subset*: $\text{set } (g\text{-map roots}) \subseteq \text{set all-constraint-roots}$

by (rule *g-map-subset-all-constraint-roots*[*OF spec-entry*])

have *distinct-roots*: $\text{distinct } (g\text{-map roots})$

by (rule *g-map-distinct*[*OF spec-entry*])

have *distinct-rest*: $\text{distinct } ?\text{rest}$

by *simp*

have *rest-set*: $\text{set } ?\text{rest} = \text{set all-constraint-roots} - \text{set } (g\text{-map roots})$

by *auto*

have *rest-len*:

$\text{length } ?\text{rest} = \text{length all-constraint-roots} - \text{length roots}$

proof –

have $\text{length } ?\text{rest} = \text{card } (\text{set } ?\text{rest})$

using *distinct-card*[*OF distinct-rest*] **by** *simp*

also have $\dots = \text{card } (\text{set all-constraint-roots} - \text{set } (g\text{-map roots}))$

using *rest-set* **by** *simp*

also have $\dots =$

```

      card (set all-constraint-roots) - card (set (g-map roots))
    by (rule card-Diff-subset) (use subset in auto)
  also have ... = length all-constraint-roots - length roots
proof -
  have card (set (g-map roots)) = length (g-map roots)
    using distinct-roots by (simp add: distinct-card)
  also have ... = length roots
    unfolding g-map-def by simp
  finally show ?thesis
    by (simp add: distinct-card)
qed
finally show ?thesis .
qed
have degree (constraint-root-cofactor roots) ≤ length ?rest
  unfolding constraint-root-cofactor-def by (rule degree-prod-le-length)
then show ?thesis
  unfolding rest-len .
qed

lemma common-denominator-poly-degree-le:
  degree common-denominator-poly ≤ length all-constraint-roots
  unfolding common-denominator-poly-def
  by (rule degree-prod-le-length)

lemma common-denominator-numerator-term-degree-lt:
  assumes wf: spec-degree-wellformed
    and deg-f: degree f < clength
    and spec-entry: (c, roots, d) ∈ set spec
  shows
    degree (common-denominator-numerator-term f (a, (c, roots, d))) <
      query-sample-space-size
proof -
  let ?dc = d * (clength - 1)
  let ?all = length all-constraint-roots
  have deg-c:
    degree (c (trace-powers-of f)) ≤ ?dc
    by (rule spec-degree-wellformed-constraint-degree
      [OF wf deg-f spec-entry])
  have deg-cofactor:
    degree (constraint-root-cofactor roots) ≤ ?all - length roots
    by (rule constraint-root-cofactor-degree-le[OF spec-entry])
  have deg-cp: degree (CP a :: 'f poly) ≤ 0
    unfolding CP-def by (rule degree-monom-le)
  have term-le:
    degree (CP a * c (trace-powers-of f) *
      constraint-root-cofactor roots) ≤
      ?dc + (?all - length roots)
proof -
  have degree (CP a * c (trace-powers-of f) *

```

```

      constraint-root-cofactor roots) ≤
degree (CP a * c (trace-powers-of f)) +
degree (constraint-root-cofactor roots)
by (rule degree-mult-le)
also have ... ≤
degree (CP a :: 'f poly) + degree (c (trace-powers-of f)) +
degree (constraint-root-cofactor roots)
using degree-mult-le[of CP a :: 'f poly c (trace-powers-of f)]
by linarith
also have ... ≤ ?dc + (?all - length roots)
using deg-cp deg-c deg-cofactor by linarith
finally show ?thesis .
qed
have roots-le-all: length roots ≤ ?all
by (rule constraint-roots-length-le-all-constraint-roots[OF spec-entry])
have quotient-le:
?dc - length roots ≤ maxDegree
by (rule declared-quotient-degree-le-maxDegree[OF spec-entry])
have arith:
?dc + (?all - length roots) ≤ maxDegree + ?all
using roots-le-all quotient-le by linarith
have margin: maxDegree + ?all < query-sample-space-size
using spec-degree-wellformed-query-margin[OF wf]
unfolding query-agreement-bound-def by simp
show ?thesis
using term-le arith margin
unfolding common-denominator-numerator-term-def
by simp
qed

lemma common-denominator-numerator-degree-lt:
assumes wf: spec-degree-wellformed
and deg-f: degree f < clength
shows degree (common-denominator-numerator f as) < query-sample-space-size
proof -
have term-lt:
 $\bigwedge p. p \in$ 
set (map (common-denominator-numerator-term f) (zip as spec))  $\implies$ 
degree p < query-sample-space-size
proof -
fix p
assume p-in:
p ∈ set (map (common-denominator-numerator-term f) (zip as spec))
then obtain z where
z-in: z ∈ set (zip as spec)
and p-eq: p = common-denominator-numerator-term f z
by auto
obtain a entry where z-eq: z = (a, entry)
by (cases z) auto

```

```

obtain  $c$  roots  $d$  where  $\text{entry-eq}$ :  $\text{entry} = (c, \text{roots}, d)$ 
  by ( $\text{cases entry}$ ) auto
have  $\text{spec-entry}$ :  $(c, \text{roots}, d) \in \text{set spec}$ 
  using  $\text{z-in unfolding z-eq entry-eq}$  by ( $\text{auto simp: set-zip}$ )
show  $\text{degree } p < \text{query-sample-space-size}$ 
  unfolding  $p\text{-eq z-eq entry-eq}$ 
  by ( $\text{rule common-denominator-numerator-term-degree-lt}$ 
    [ $OF \text{ wf deg-f spec-entry}$ ])
qed
show  $?thesis$ 
  unfolding  $\text{common-denominator-numerator-def}$ 
  by ( $\text{rule degree-sum-list-less}[OF \text{ term-lt query-sample-space-size-pos}]$ )
qed

lemma  $\text{common-denominator-product-degree-lt}$ :
  assumes  $\text{wf: spec-degree-wellformed}$ 
  and  $\text{deg-q: degree } q \leq \text{maxDegree}$ 
  shows  $\text{degree } (\text{common-denominator-poly} * q) < \text{query-sample-space-size}$ 
proof –
  have  $\text{degree } (\text{common-denominator-poly} * q) \leq$ 
     $\text{degree common-denominator-poly} + \text{degree } q$ 
  by ( $\text{rule degree-mult-le}$ )
  also have  $\dots \leq \text{length all-constraint-roots} + \text{maxDegree}$ 
  using  $\text{common-denominator-poly-degree-le deg-q}$  by  $\text{linarith}$ 
  also have  $\dots = \text{maxDegree} + \text{length all-constraint-roots}$ 
  by  $\text{simp}$ 
  also have  $\dots < \text{query-sample-space-size}$ 
  using  $\text{spec-degree-wellformed-query-margin}[OF \text{ wf}]$ 
  unfolding  $\text{query-agreement-bound-def}$  by  $\text{simp}$ 
  finally show  $?thesis$  .
qed

lemma  $\text{common-denominator-numerator-term-degree-le-bound}$ :
  assumes  $\text{deg-f: degree } f < \text{clength}$ 
  and  $\text{spec-entry: } (c, \text{roots}, d) \in \text{set spec}$ 
  shows
     $\text{degree } (\text{common-denominator-numerator-term } f \ (a, (c, \text{roots}, d))) \leq$ 
     $\text{query-agreement-bound}$ 
proof –
  let  $?dc = d * (\text{clength} - 1)$ 
  let  $?all = \text{length all-constraint-roots}$ 
  have  $\text{deg-c}$ :
     $\text{degree } (c \ (\text{trace-powers-of } f)) \leq ?dc$ 
  by ( $\text{rule constraint-degree-wellformed}[OF \text{ deg-f spec-entry}]$ )
  have  $\text{deg-cofactor}$ :
     $\text{degree } (\text{constraint-root-cofactor roots}) \leq ?all - \text{length roots}$ 
  by ( $\text{rule constraint-root-cofactor-degree-le}[OF \text{ spec-entry}]$ )
  have  $\text{deg-cp: degree } (CP \ a :: 'f \text{ poly}) \leq 0$ 
  unfolding  $CP\text{-def}$  by ( $\text{rule degree-monom-le}$ )

```

```

have term-le:
  degree (CP a * c (trace-powers-of f) *
    constraint-root-cofactor roots) ≤
    ?dc + (?all - length roots)
proof -
  have degree (CP a * c (trace-powers-of f) *
    constraint-root-cofactor roots) ≤
    degree (CP a * c (trace-powers-of f)) +
    degree (constraint-root-cofactor roots)
  by (rule degree-mult-le)
  also have ... ≤
    degree (CP a :: 'f poly) + degree (c (trace-powers-of f)) +
    degree (constraint-root-cofactor roots)
  using degree-mult-le[of CP a :: 'f poly c (trace-powers-of f)]
  by linarith
  also have ... ≤ ?dc + (?all - length roots)
  using deg-cp deg-c deg-cofactor by linarith
  finally show ?thesis .
qed
have roots-le-all: length roots ≤ ?all
  by (rule constraint-roots-length-le-all-constraint-roots[OF spec-entry])
have quotient-le:
  ?dc - length roots ≤ maxDegree
  by (rule declared-quotient-degree-le-maxDegree[OF spec-entry])
have ?dc + (?all - length roots) ≤ maxDegree + ?all
  using roots-le-all quotient-le by linarith
then show ?thesis
  using term-le unfolding common-denominator-numerator-term-def
  query-agreement-bound-def
  by simp
qed

lemma common-denominator-numerator-degree-le-bound:
  assumes deg-f: degree f < clength
  shows degree (common-denominator-numerator f as) ≤ query-agreement-bound
proof -
  have term-le:
     $\bigwedge p. p \in$ 
    set (map (common-denominator-numerator-term f) (zip as spec))  $\implies$ 
    degree p ≤ query-agreement-bound
  proof -
    fix p
    assume p-in:
      p ∈ set (map (common-denominator-numerator-term f) (zip as spec))
    then obtain z where
      z-in: z ∈ set (zip as spec)
      and p-eq: p = common-denominator-numerator-term f z
      by auto
    obtain a entry where z-eq: z = (a, entry)

```

```

    by (cases z) auto
  obtain c roots d where entry-eq: entry = (c, roots, d)
    by (cases entry) auto
  have spec-entry: (c, roots, d) ∈ set spec
    using z-in unfolding z-eq entry-eq by (auto simp: set-zip)
  show degree p ≤ query-agreement-bound
    unfolding p-eq z-eq entry-eq
    by (rule common-denominator-numerator-term-degree-le-bound
        [OF deg-f spec-entry])
qed
show ?thesis
  unfolding common-denominator-numerator-def
  by (rule degree-sum-list-le[OF term-le])
qed

lemma common-denominator-product-degree-le-bound:
  assumes deg-q: degree q ≤ maxDegree
  shows degree (common-denominator-poly * q) ≤ query-agreement-bound
proof -
  have degree (common-denominator-poly * q) ≤
    degree common-denominator-poly + degree q
    by (rule degree-mult-le)
  also have ... ≤ length all-constraint-roots + maxDegree
    using common-denominator-poly-degree-le deg-q by linarith
  also have ... = query-agreement-bound
    unfolding query-agreement-bound-def by simp
  finally show ?thesis .
qed

lemma common-denominator-residual-degree-le-bound:
  assumes deg-f: degree f < clength
    and deg-q: degree q ≤ maxDegree
  shows
    degree (common-denominator-numerator f as - common-denominator-poly * q)
      ≤ query-agreement-bound
proof -
  have num-le:
    degree (common-denominator-numerator f as) ≤ query-agreement-bound
    by (rule common-denominator-numerator-degree-le-bound[OF deg-f])
  have prod-le:
    degree (common-denominator-poly * q) ≤ query-agreement-bound
    by (rule common-denominator-product-degree-le-bound[OF deg-q])
  show ?thesis
    using degree-diff-le[OF num-le prod-le] .
qed

lemma common-denominator-degree-bounds-from-spec-degree-wellformed:
  assumes wf: spec-degree-wellformed
    and deg-f: degree f < clength

```

shows *common-denominator-degree-bounds f as*
unfolding *common-denominator-degree-bounds-def*
using *common-denominator-numerator-degree-lt[OF wf deg-f]*
common-denominator-product-degree-lt[OF wf]
by *simp*

lemma *common-denominator-vanishes-on-constraint-root:*

assumes *spec-entry: (c, roots, d) ∈ set spec*
and *r-in: r ∈ set roots*
shows *poly common-denominator-poly (g ^ r) = 0*

proof –

have *factor:*
common-denominator-poly =
*prod (g-map roots) * constraint-root-cofactor roots*
by (*rule common-denominator-factor[OF spec-entry]*)
have *root-in: g ^ r ∈ set (g-map roots)*
by (*rule g-power-in-g-map[OF r-in]*)
have *poly (prod (g-map roots)) (g ^ r) = 0*
by (*rule prod-eval-zero[OF root-in]*)
then show *?thesis*
using *factor* **by** *simp*

qed

lemma *g-power-notin-g-map-if-root-notin:*

assumes *spec-entry: (c, roots, d) ∈ set spec*
and *r-bound: r < clength*
and *r-notin: r ∉ set roots*
shows *g ^ r ∉ set (g-map roots)*

proof

assume *g ^ r ∈ set (g-map roots)*
then obtain *r' where r'-in: r' ∈ set roots*
and *eq: g ^ r = g ^ r'*
unfolding *g-map-def* **by** *auto*
have *r'-bound: r' < clength*
by (*rule spec-roots-in-range[OF spec-entry r'-in]*)
have *r = r'*
by (*rule g-power-inj-on-range[OF r-bound r'-bound eq]*)
then show *False*
using *r-notin r'-in* **by** *simp*

qed

lemma *constraint-root-cofactor-eval-nonzero:*

assumes *spec-entry: (c, roots, d) ∈ set spec*
and *r-in: r ∈ set roots*
shows *poly (constraint-root-cofactor roots) (g ^ r) ≠ 0*

proof –

have *root-in: g ^ r ∈ set (g-map roots)*
by (*rule g-power-in-g-map[OF r-in]*)
have *g ^ r ∉*

```

      set (filter (λx. x ∉ set (g-map roots)) all-constraint-roots)
    using root-in by auto
  then show ?thesis
    unfolding constraint-root-cofactor-def
    by (rule prod-eval-nonzero)
qed

```

```

lemma constraint-root-cofactor-kills-other-root:
  assumes root-spec: (c0, roots0, d0) ∈ set spec
    and r-in0: r ∈ set roots0
    and spec-entry: (c, roots, d) ∈ set spec
    and r-notin: r ∉ set roots
  shows poly (constraint-root-cofactor roots) (g ^ r) = 0
proof -
  have r-bound: r < clength
    by (rule spec-roots-in-range[OF root-spec r-in0])
  have root-in-all: g ^ r ∈ set all-constraint-roots
    using g-map-subset-all-constraint-roots[OF root-spec]
      g-power-in-g-map[OF r-in0]
    by auto
  have root-notin-entry: g ^ r ∉ set (g-map roots)
    by (rule g-power-notin-g-map-if-root-notin
      [OF spec-entry r-bound r-notin])
  have g ^ r ∈
    set (filter (λx. x ∉ set (g-map roots)) all-constraint-roots)
    using root-in-all root-notin-entry by simp
  then show ?thesis
    unfolding constraint-root-cofactor-def
    by (rule prod-eval-zero)
qed

```

```

lemma common-denominator-residual-coefficient-nonzero:
  assumes spec-entry: (c, roots, d) ∈ set spec
    and r-in: r ∈ set roots
    and nonzero:
      poly (c (trace-powers-of f)) (g ^ r) ≠ 0
  shows
    poly (c (trace-powers-of f)) (g ^ r) *
      poly (constraint-root-cofactor roots) (g ^ r) ≠ 0
  using nonzero constraint-root-cofactor-eval-nonzero[OF spec-entry r-in]
  by simp

```

```

lemma common-denominator-numerator-root-eval:
  assumes len: length as = length spec
    and root-spec: (c0, roots0, d0) ∈ set spec
    and r-in0: r ∈ set roots0
  shows
    poly (common-denominator-numerator f as) (g ^ r) =
      common-denominator-composition-root-residual f as r

```



```

proof –
  have lhs:
    poly (common-denominator-numerator f as) (g r) =
      (∑ i ∈ {0..length spec}.
        poly
          (case spec ! i of (c, roots, d) ⇒
            CP (as ! i) * c (trace-powers-of f) *
              constraint-root-cofactor roots)
            (g r))
    using len
    unfolding common-denominator-numerator-def
    by (simp add: sum-list-sum-nth poly-sum
      common-denominator-numerator-term-def split-beta' case-prod-unfold
      CP-def poly-monom algebra-simps)
  have rhs:
    common-denominator-composition-root-residual f as r =
      (∑ i ∈ {0..length spec}.
        common-denominator-constraint-root-residual f r
          (as ! i, spec ! i))
    using len
    unfolding common-denominator-composition-root-residual-def
    by (simp add: sum-list-sum-nth split-beta')
  have pointwise:
    ∧i. i ∈ {0..length spec} ⇒
      poly
        (case spec ! i of (c, roots, d) ⇒
          CP (as ! i) * c (trace-powers-of f) *
            constraint-root-cofactor roots)
          (g r) =
        common-denominator-constraint-root-residual f r
          (as ! i, spec ! i)
  proof –
    fix i
    assume i-in: i ∈ {0..length spec}
    then have i-bound: i < length spec
      by simp
    obtain c roots d where spec-i: spec ! i = (c, roots, d)
      by (cases spec ! i) auto
    have spec-entry: (c, roots, d) ∈ set spec
      using nth-mem[OF i-bound] spec-i by simp
    show
      poly
        (case spec ! i of (c, roots, d) ⇒
          CP (as ! i) * c (trace-powers-of f) *
            constraint-root-cofactor roots)
          (g r) =
        common-denominator-constraint-root-residual f r
          (as ! i, spec ! i)
    proof (cases r ∈ set roots)

```

```

    case True
    then show ?thesis
      using spec-i
      unfolding common-denominator-constraint-root-residual-def
      by (simp add: CP-def poly-monom)
  next
  case False
  have kill:
    poly (constraint-root-cofactor roots) (g ^ r) = 0
    by (rule constraint-root-cofactor-kills-other-root
      [OF root-spec r-in0 spec-entry False])
  show ?thesis
    using spec-i False kill
    unfolding common-denominator-constraint-root-residual-def
    by (simp add: CP-def poly-monom)
qed
qed
show ?thesis
  unfolding lhs rhs
  by (intro sum.cong refl pointwise) simp
qed

lemma common-denominator-mult-composition-spec-eval:
  assumes denom-nonzero:
     $\bigwedge c \text{ roots } d. (c, \text{roots}, d) \in \text{set spec} \implies$ 
    poly (prod (g-map roots)) x  $\neq 0$ 
  shows
    poly common-denominator-poly x * composition-spec-eval f as x =
    poly (common-denominator-numerator f as) x
proof -
  let ?D = poly common-denominator-poly x
  have term-eval:
     $\bigwedge z. z \in \text{set (zip as spec)} \implies$ 
    ?D * composition-spec-eval-term f x z =
    poly (common-denominator-numerator-term f z) x
proof -
  fix z
  assume z-in: z  $\in$  set (zip as spec)
  obtain a entry where z-eq: z = (a, entry)
    by (cases z) auto
  obtain c roots d where entry-eq: entry = (c, roots, d)
    by (cases entry) auto
  have entry-in: entry  $\in$  set spec
  proof -
    obtain i where i-bound: i < length as i < length spec
      and z-i: z = (as ! i, spec ! i)
      using z-in by (auto simp: set-zip)
    have entry = spec ! i
      using z-eq z-i by simp

```

```

then show ?thesis
  using nth-mem[OF i-bound(2)] by simp
qed
have spec-entry:  $(c, \text{roots}, d) \in \text{set spec}$ 
  using entry-in entry-eq by simp
have factor:
  ?D =
    poly (prod (g-map roots)) x *
    poly (constraint-root-cofactor roots) x
  using common-denominator-factor[OF spec-entry]
  by simp
have denom:  $\text{poly}(\text{prod}(\text{g-map roots})) x \neq 0$ 
  by (rule denom-nonzero[OF spec-entry])
show
  ?D * composition-spec-eval-term f x z =
    poly (common-denominator-numerator-term f z) x
  using z-eq entry-eq factor denom
  unfolding composition-spec-eval-term-def
    common-denominator-numerator-term-def
  by (simp add: field-simps CP-def poly-monom)
qed
have mult-sum:
  ?D * sum-list (map (composition-spec-eval-term f x) (zip as spec)) =
    sum-list
      (map ( $\lambda z.$  poly (common-denominator-numerator-term f z) x)
        (zip as spec))
proof -
  have ?D * sum-list (map (composition-spec-eval-term f x) (zip as spec)) =
    sum-list
      (map ( $\lambda z.$  ?D * composition-spec-eval-term f x z)
        (zip as spec))
    by (simp add: sum-list-const-mult algebra-simps)
  also have ... =
    sum-list
      (map ( $\lambda z.$  poly (common-denominator-numerator-term f z) x)
        (zip as spec))
    using term-eval by (intro arg-cong[where f=sum-list] map-cong) auto
  finally show ?thesis .
qed
have rhs:
  poly (common-denominator-numerator f as) x =
    sum-list
      (map ( $\lambda z.$  poly (common-denominator-numerator-term f z) x)
        (zip as spec))
  unfolding common-denominator-numerator-def
  apply (simp add: poly-sum-list)
  apply (intro arg-cong[where f=sum-list] map-cong refl)
  apply (auto simp: common-denominator-numerator-term-def
    split-beta' case-prod-unfold algebra-simps CP-def poly-monom)

```

```

      split: prod.splits)
    done
  show ?thesis
    unfolding composition-spec-eval-def
    using mult-sum rhs by simp
qed

lemma common-denominator-composition-root-residual-sum-nth:
  assumes len: length as = length spec
  shows
    common-denominator-composition-root-residual f as r =
      ( $\sum i \in \{0..<\text{length spec}\}.$ 
        common-denominator-constraint-root-residual f r (as ! i, spec ! i))
  proof -
    have common-denominator-composition-root-residual f as r =
      sum-list
        (map (common-denominator-constraint-root-residual f r) (zip as spec))
    by (simp add: common-denominator-composition-root-residual-def)
    also have ... =
      ( $\sum i \in \{0..<\text{length (zip as spec)}\}.$ 
        map (common-denominator-constraint-root-residual f r) (zip as spec) ! i)
    by (simp add: sum-list-sum-nth)
    also have ... =
      ( $\sum i \in \{0..<\text{length spec}\}.$ 
        common-denominator-constraint-root-residual f r (as ! i, spec ! i))
    using len by (intro sum.cong) auto
    finally show ?thesis .
  qed

lemma common-denominator-composition-root-residual-split-index:
  assumes len: length as = length spec
  and i-bound: i < length spec
  shows
    common-denominator-composition-root-residual f as r =
      common-denominator-constraint-root-residual f r (as ! i, spec ! i) +
      ( $\sum j \in \{0..<\text{length spec}\} - \{i\}.$ 
        common-denominator-constraint-root-residual f r (as ! j, spec ! j))
  proof -
    let ?F =
       $\lambda j.$  common-denominator-constraint-root-residual f r (as ! j, spec ! j)
    have common-denominator-composition-root-residual f as r =
      ( $\sum j \in \{0..<\text{length spec}\}.$  ?F j)
    by (rule common-denominator-composition-root-residual-sum-nth[OF len])
    also have ... = ?F i + ( $\sum j \in \{0..<\text{length spec}\} - \{i\}.$  ?F j)
    using i-bound by (simp add: sum.remove)
    finally show ?thesis .
  qed

lemma common-denominator-composition-root-residual-zero-unique-nth:

```

```

fixes as bs :: 'f list
assumes len-as:  $\text{length } as = \text{length } spec$ 
  and len-bs:  $\text{length } bs = \text{length } spec$ 
  and i-bound:  $i < \text{length } spec$ 
  and spec-i:  $spec ! i = (c, roots, d)$ 
  and r-in:  $r \in \text{set } roots$ 
  and nonzero:
     $\text{poly } (c \text{ (trace-powers-of } f)) (g \wedge r) \neq 0$ 
  and outside:
     $\bigwedge j. j < \text{length } spec \implies j \neq i \implies as ! j = bs ! j$ 
  and zero-as:  $\text{common-denominator-composition-root-residual } f \text{ as } r = 0$ 
  and zero-bs:  $\text{common-denominator-composition-root-residual } f \text{ bs } r = 0$ 
shows  $as ! i = bs ! i$ 
proof -
  let ?C =
     $\text{poly } (c \text{ (trace-powers-of } f)) (g \wedge r) * \text{poly } (\text{constraint-root-cofactor } roots) (g \wedge r)$ 
  let ?B =
     $(\sum j \in \{0..<\text{length } spec\} - \{i\}. \text{common-denominator-constraint-root-residual } f \text{ r } (as ! j, spec ! j))$ 
  have spec-entry:  $(c, roots, d) \in \text{set } spec$ 
    using nth-mem[OF i-bound] spec-i by simp
  have C-nonzero:  $?C \neq 0$ 
    by (rule common-denominator-residual-coefficient-nonzero [OF spec-entry r-in nonzero])
  have B-eq:
     $(\sum j \in \{0..<\text{length } spec\} - \{i\}. \text{common-denominator-constraint-root-residual } f \text{ r } (bs ! j, spec ! j)) = ?B$ 
    by (intro sum.cong) (use outside in auto)
  have as-eq:
     $\text{common-denominator-composition-root-residual } f \text{ as } r = ?C * (as ! i) + ?B$ 
    using common-denominator-composition-root-residual-split-index [OF len-as i-bound] spec-i r-in
    unfolding common-denominator-constraint-root-residual-def
    by (simp add: algebra-simps)
  have bs-eq:
     $\text{common-denominator-composition-root-residual } f \text{ bs } r = ?C * (bs ! i) + ?B$ 
    using common-denominator-composition-root-residual-split-index [OF len-bs i-bound] spec-i r-in B-eq
    unfolding common-denominator-constraint-root-residual-def
    by (simp add: algebra-simps)
  show ?thesis
proof (rule linear-equation-solution-unique[OF C-nonzero])
  show  $?C * (as ! i) + ?B = 0$ 
    using as-eq zero-as by simp

```

```

    show ?C * (bs ! i) + ?B = 0
      using bs-eq zero-bs by simp
  qed
qed

lemma random-combination-common-denominator-hides-violations-unique-nth:
  fixes as bs :: 'f list
  assumes len-as: length as = length spec
    and len-bs: length bs = length spec
    and i-bound: i < length spec
    and spec-i: spec ! i = (c, roots, d)
    and violated: (c, roots, d) ∈ violated-constraints f
    and r-in: r ∈ set roots
    and nonzero:
      poly (c (trace-powers-of f)) (g ^ r) ≠ 0
    and outside:
       $\bigwedge j. j < \text{length spec} \implies j \neq i \implies \text{as} ! j = \text{bs} ! j$ 
    and hides-as: random-combination-common-denominator-hides-violations f as
    and hides-bs: random-combination-common-denominator-hides-violations f bs
  shows as ! i = bs ! i
proof -
  have zero-as:
    common-denominator-composition-root-residual f as r = 0
  using hides-as violated r-in
  unfolding random-combination-common-denominator-hides-violations-def
  by auto
  have zero-bs:
    common-denominator-composition-root-residual f bs r = 0
  using hides-bs violated r-in
  unfolding random-combination-common-denominator-hides-violations-def
  by auto
  show ?thesis
  by (rule common-denominator-composition-root-residual-zero-unique-nth
    [OF len-as len-bs i-bound spec-i r-in nonzero outside zero-as zero-bs])
qed

lemma violated-constraints-has-common-denominator-coefficient:
  assumes violated: violated-constraints f ≠ {}
  obtains i c roots d r
  where i < length spec
    and spec ! i = (c, roots, d)
    and (c, roots, d) ∈ violated-constraints f
    and r ∈ set roots
    and
      poly (c (trace-powers-of f)) (g ^ r) *
      poly (constraint-root-cofactor roots) (g ^ r) ≠ 0
proof -
  from violated-constraints-has-remainder-bad[OF violated]
  obtain entry where entry0:

```

```

    entry ∈ violated-constraints f
    constraint-remainder-bad f entry
  by blast
obtain c roots d where entry-eq: entry = (c, roots, d)
  by (cases entry) auto
have entry:
  (c, roots, d) ∈ violated-constraints f
  constraint-remainder-bad f (c, roots, d)
  using entry0 unfolding entry-eq by simp-all
then obtain r where r:
  r ∈ set roots
  poly (c (trace-powers-of f)) (g ^ r) ≠ 0
  unfolding constraint-remainder-bad-def by auto
have spec-entry: (c, roots, d) ∈ set spec
  using entry(1) unfolding violated-constraints-def by auto
then obtain i where i:
  i < length spec
  spec ! i = (c, roots, d)
  by (auto simp: in-set-conv-nth)
have coeff:
  poly (c (trace-powers-of f)) (g ^ r) *
  poly (constraint-root-cofactor roots) (g ^ r) ≠ 0
  by (rule common-denominator-residual-coefficient-nonzero
    [OF spec-entry r(1) r(2)])
show ?thesis
  by (rule that[OF i entry(1) r(1) coeff])
qed

lemma random-combination-common-denominator-hides-has-coefficient:
  assumes hides: random-combination-common-denominator-hides-violations f as
  obtains i c roots d r
  where i < length spec
  and spec ! i = (c, roots, d)
  and (c, roots, d) ∈ violated-constraints f
  and r ∈ set roots
  and
    poly (c (trace-powers-of f)) (g ^ r) *
    poly (constraint-root-cofactor roots) (g ^ r) ≠ 0
proof –
  have violated: violated-constraints f ≠ {}
  using hides
  unfolding random-combination-common-denominator-hides-violations-def
  by simp
show ?thesis
  by (rule violated-constraints-has-common-denominator-coefficient[OF violated])
    (rule that)
qed

definition alpha-space :: 'f list set

```

where $\text{alpha-space} = \{as. \text{length } as = \text{length spec}\}$

definition $\text{common-denominator-hiding-alpha-space} :: 'f \text{ poly} \Rightarrow 'f \text{ list set}$

where

$\text{common-denominator-hiding-alpha-space } f =$

$\{as \in \text{alpha-space}.$

$\text{random-combination-common-denominator-hides-violations } f \text{ as}\}$

lemma $\text{finite-length-lists-UNIV}:$

$\text{finite } \{xs :: 'f \text{ list}. \text{length } xs = n\}$

proof $-$

have $\{xs :: 'f \text{ list}. \text{length } xs = n\} =$

$\{xs. \text{set } xs \subseteq (\text{UNIV} :: 'f \text{ set}) \wedge \text{length } xs = n\}$

by auto

then show $?thesis$

using $\text{finite-lists-length-eq}$ [of $\text{UNIV} :: 'f \text{ set } n$] **by** simp

qed

lemma $\text{card-length-lists-UNIV}:$

$\text{card } \{xs :: 'f \text{ list}. \text{length } xs = n\} = \text{CARD}('f) \wedge n$

proof $-$

have $\{xs :: 'f \text{ list}. \text{length } xs = n\} =$

$\{xs. \text{set } xs \subseteq (\text{UNIV} :: 'f \text{ set}) \wedge \text{length } xs = n\}$

by auto

then show $?thesis$

using $\text{card-lists-length-eq}$ [of $\text{UNIV} :: 'f \text{ set } n$] **by** simp

qed

definition $\text{fri-challenge-space} :: \text{nat} \Rightarrow 'f \text{ list set}$

where $\text{fri-challenge-space } n = \{bs. \text{length } bs = n\}$

lemma $\text{finite-fri-challenge-space}[\text{simp}]:$

$\text{finite } (\text{fri-challenge-space } n)$

unfolding $\text{fri-challenge-space-def}$

by $(\text{rule } \text{finite-length-lists-UNIV})$

lemma $\text{card-fri-challenge-space}:$

$\text{card } (\text{fri-challenge-space } n) = \text{CARD}('f) \wedge n$

unfolding $\text{fri-challenge-space-def}$

by $(\text{rule } \text{card-length-lists-UNIV})$

lemma $\text{card-alpha-space}[\text{simp}]:$

$\text{card } \text{alpha-space} = \text{CARD}('f) \wedge \text{length spec}$

unfolding alpha-space-def **by** $(\text{rule } \text{card-length-lists-UNIV})$

lemma $\text{common-denominator-hiding-alpha-space-card-bound}:$

$\text{card } (\text{common-denominator-hiding-alpha-space } f) \leq$

$(\text{if } \text{length spec} = 0 \text{ then } 0 \text{ else } \text{CARD}('f) \wedge (\text{length spec} - 1))$

proof $(\text{cases } \text{common-denominator-hiding-alpha-space } f = \{\})$


```

case True
then show ?thesis by simp
next
case False
let ?S = common-denominator-hiding-alpha-space f
from False obtain as0 where as0-in: as0 ∈ ?S
  by auto
have hides0: random-combination-common-denominator-hides-violations f as0
  using as0-in unfolding common-denominator-hiding-alpha-space-def by simp
from random-combination-common-denominator-hides-has-coefficient[OF hides0]
obtain i c roots d r where
  i-bound: i < length spec
  and spec-i: spec ! i = (c, roots, d)
  and violated: (c, roots, d) ∈ violated-constraints f
  and r-in: r ∈ set roots
  and coeff-nonzero:
    poly (c (trace-powers-of f)) (g ^ r) *
    poly (constraint-root-cofactor roots) (g ^ r) ≠ 0
  by blast
have nonzero: poly (c (trace-powers-of f)) (g ^ r) ≠ 0
  using coeff-nonzero by auto
let ?key =  $\lambda xs :: 'f\ list. (take\ i\ xs, drop\ (Suc\ i)\ xs)$ 
have inj: inj-on ?key ?S
proof (rule inj-onI)
  fix xs ys
  assume xs-in: xs ∈ ?S
  and ys-in: ys ∈ ?S
  and key-eq: ?key xs = ?key ys
  have len-xs: length xs = length spec
  using xs-in
  unfolding common-denominator-hiding-alpha-space-def alpha-space-def
  by simp
  have len-ys: length ys = length spec
  using ys-in
  unfolding common-denominator-hiding-alpha-space-def alpha-space-def
  by simp
  have hides-xs:
    random-combination-common-denominator-hides-violations f xs
  using xs-in unfolding common-denominator-hiding-alpha-space-def by simp
  have hides-ys:
    random-combination-common-denominator-hides-violations f ys
  using ys-in unfolding common-denominator-hiding-alpha-space-def by simp
  have take-eq: take i xs = take i ys
  using key-eq by simp
  have drop-eq: drop (Suc i) xs = drop (Suc i) ys
  using key-eq by simp
  have outside:
     $\bigwedge j. j < \text{length } spec \implies j \neq i \implies xs ! j = ys ! j$ 
proof –

```

```

fix j
assume j-bound: j < length spec
  and j-ne: j ≠ i
show xs ! j = ys ! j
proof (cases j < i)
  case True
    have xs ! j = take i xs ! j
      using True by simp
    also have ... = take i ys ! j
      using take-eq by simp
    also have ... = ys ! j
      using True by simp
    finally show ?thesis .
  next
    case False
    then have i-lt-j: i < j
      using j-ne by linarith
    have j-decomp: j = Suc i + (j - Suc i)
      using i-lt-j by simp
    have drop-idx-bound-xs: j - Suc i < length (drop (Suc i) xs)
      using j-bound i-lt-j len-xs by simp
    have drop-idx-bound-ys: j - Suc i < length (drop (Suc i) ys)
      using j-bound i-lt-j len-ys by simp
    have xs ! j = drop (Suc i) xs ! (j - Suc i)
      using drop-idx-bound-xs j-decomp by simp
    also have ... = drop (Suc i) ys ! (j - Suc i)
      using drop-eq by simp
    also have ... = ys ! j
      using drop-idx-bound-ys j-decomp by simp
    finally show ?thesis .
qed
qed
have nth-i: xs ! i = ys ! i
  by (rule random-combination-common-denominator-hides-violations-unique-nth
    [OF len-xs len-ys i-bound spec-i violated r-in nonzero outside
      hides-xs hides-ys])
show xs = ys
proof (rule nth-equalityI)
  show length xs = length ys
    using len-xs len-ys by simp
next
  fix j
  assume j-bound: j < length xs
  then have j-spec: j < length spec
    using len-xs by simp
  show xs ! j = ys ! j
  proof (cases j = i)
    case True
      then show ?thesis

```

```

    using nth-i by simp
  next
    case False
    then show ?thesis
      by (rule outside[OF j-spec])
    qed
  qed
qed
let ?A = {xs :: 'f list. length xs = i}
let ?B = {xs :: 'f list. length xs = length spec - Suc i}
have image-subset: ?key ' ?S  $\subseteq$  ?A  $\times$  ?B
proof
  fix x
  assume x  $\in$  ?key ' ?S
  then obtain xs where xs-in: xs  $\in$  ?S and x-eq: x = ?key xs
  by auto
  have len-xs: length xs = length spec
  using xs-in
  unfolding common-denominator-hiding-alpha-space-def alpha-space-def
  by simp
  show x  $\in$  ?A  $\times$  ?B
  using len-xs i-bound unfolding x-eq by simp
qed
have finite-A: finite ?A
  by (rule finite-length-lists-UNIV)
have finite-B: finite ?B
  by (rule finite-length-lists-UNIV)
have finite-prod: finite (?A  $\times$  ?B)
  using finite-A finite-B by simp
have finite-alpha: finite alpha-space
  unfolding alpha-space-def by (rule finite-length-lists-UNIV)
have finite-S: finite ?S
  unfolding common-denominator-hiding-alpha-space-def
  using finite-alpha by simp
have card-image-le: card (?key ' ?S)  $\leq$  card (?A  $\times$  ?B)
  by (rule card-mono[OF finite-prod image-subset])
have card-S: card ?S = card (?key ' ?S)
  using card-image[OF inj] by simp
have card-prod:
  card (?A  $\times$  ?B) =
    CARD('f)  $\wedge$  i * CARD('f)  $\wedge$  (length spec - Suc i)
  using finite-A finite-B by (simp add: card-length-lists-UNIV)
have exp-eq:
  i + (length spec - Suc i) = length spec - 1
  using i-bound by simp
have card ?S  $\leq$  CARD('f)  $\wedge$  i * CARD('f)  $\wedge$  (length spec - Suc i)
  using card-image-le card-S card-prod by simp
also have ... = CARD('f)  $\wedge$  (length spec - 1)
  using exp-eq by (simp add: power-add[symmetric])

```

finally have *bound*:
 $\text{card } ?S \leq \text{CARD}('f) \wedge (\text{length spec} - 1) .$
have $\text{length spec} \neq 0$
using *i-bound* **by** *auto*
then show *?thesis*
using *bound* **by** *simp*
qed

lemma *nnreal-nat-divide-right-mono*:
assumes $a \leq b$
shows $\text{nnreal } a / \text{nnreal } c \leq \text{nnreal } b / \text{nnreal } c$
using *assms*
by *transfer (simp add: divide-right-mono)*

lemma *nnreal-nat-divide-self*:
assumes $k \neq 0$
shows $\text{nnreal } k / \text{nnreal } k = 1$
using *assms* **by** *transfer simp*

lemma *nnreal-power-fraction-cancel*:
assumes $k \neq 0$
shows $\text{nnreal } (k \wedge n) / \text{nnreal } (k \wedge \text{Suc } n) = 1 / \text{nnreal } k$
using *assms*
by *transfer (simp add: power-Suc field-simps)*

lemma *common-denominator-hiding-alpha-space-fraction-bound*:
 $\text{nnreal } (\text{card } (\text{common-denominator-hiding-alpha-space } f)) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{length spec}) \leq \text{composition-error-bound}$
proof *(cases length spec)*
case 0
then have $\text{card } (\text{common-denominator-hiding-alpha-space } f) = 0$
using *common-denominator-hiding-alpha-space-card-bound[of f]* **by** *simp*
then show *?thesis*
unfolding *composition-error-bound-def* **by** *simp*
next
case *(Suc n)*
have *card-le*:
 $\text{card } (\text{common-denominator-hiding-alpha-space } f) \leq \text{CARD}('f) \wedge n$
using *common-denominator-hiding-alpha-space-card-bound[of f]*
unfolding *Suc* **by** *simp*
have *field-nonzero*: $\text{CARD}('f) \neq 0$
by *simp*
have $\text{nnreal } (\text{card } (\text{common-denominator-hiding-alpha-space } f)) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{length spec}) \leq$
 $\text{nnreal } (\text{CARD}('f) \wedge n) / \text{nnreal } (\text{CARD}('f) \wedge \text{Suc } n)$
unfolding *Suc*
using *card-le*
by *(rule nnreal-nat-divide-right-mono)*
also have $\dots = 1 / \text{nnreal } (\text{CARD}('f))$

```

    by (rule nnreal-power-fraction-cancel[OF field-nonzero])
  also have ... = composition-error-bound
    using size-card unfolding composition-error-bound-def by simp
  finally show ?thesis .
qed

lemma low-degree-trace-table-unique:
  assumes deg-f: degree f < clength
    and deg-q: degree q < clength
    and table-eq: map (poly f) eval-domain = map (poly q) eval-domain
  shows f = q
proof -
  have clength-le-domain: clength ≤ clength * scale
    using scale-pos by simp
  have deg-f-domain: degree f < clength * scale
    using deg-f clength-le-domain by linarith
  have deg-q-domain: degree q < clength * scale
    using deg-q clength-le-domain by linarith
  show ?thesis
proof (rule poly-eq-on-eval-domainI[OF deg-f-domain deg-q-domain])
  fix idx
  assume idx-bound: idx < clength * scale
  have idx-len: idx < length eval-domain
    using idx-bound eval-domain-length by simp
  have map (poly f) eval-domain ! idx = map (poly q) eval-domain ! idx
    using table-eq idx-len by simp
  then show poly f (h ^ idx * shift) = poly q (h ^ idx * shift)
    using idx-len idx-bound eval-domain-nth by simp
qed
qed

definition low-degree-trace-witness :: 'f list ⇒ 'f poly
where
  low-degree-trace-witness trace-table =
    (SOME f. degree f < clength ∧
      trace-table = map (poly f) eval-domain)

lemma low-degree-trace-witness-eq:
  assumes deg-f: degree f < clength
    and trace-table: trace-table = map (poly f) eval-domain
  shows low-degree-trace-witness trace-table = f
  unfolding low-degree-trace-witness-def
proof (rule some-equality)
  show degree f < clength ∧ trace-table = map (poly f) eval-domain
    using deg-f trace-table by simp
next
  fix q
  assume q:
    degree q < clength ∧ trace-table = map (poly q) eval-domain

```

```

then have table-eq: map (poly f) eval-domain = map (poly q) eval-domain
using trace-table by simp
show q = f
using low-degree-trace-table-unique[OF deg-f conjunct1[OF q] table-eq]
by simp
qed

```

```

definition composition-trace-bad-alpha-space :: 'f list  $\Rightarrow$  'f list set
where
  composition-trace-bad-alpha-space trace-table =
    (let f = low-degree-trace-witness trace-table in
     if degree f < clength  $\wedge$ 
       trace-table = map (poly f) eval-domain  $\wedge$ 
       violated-constraints f  $\neq$  {}
     then
       {as  $\in$  common-denominator-hiding-alpha-space f.
        common-denominator-degree-bounds f as}
     else {})

```

```

lemma finite-alpha-space:
  finite alpha-space
unfolding alpha-space-def by (rule finite-length-lists-UNIV)

```

```

lemma common-denominator-hiding-alpha-space-subset-alpha-space:
  common-denominator-hiding-alpha-space f  $\subseteq$  alpha-space
unfolding common-denominator-hiding-alpha-space-def by auto

```

```

lemma finite-common-denominator-hiding-alpha-space:
  finite (common-denominator-hiding-alpha-space f)
using finite-alpha-space
unfolding common-denominator-hiding-alpha-space-def by simp

```

```

lemma composition-trace-bad-alpha-space-subset-alpha-space:
  composition-trace-bad-alpha-space trace-table  $\subseteq$  alpha-space
proof -
  let ?f = low-degree-trace-witness trace-table
  have composition-trace-bad-alpha-space trace-table  $\subseteq$ 
    common-denominator-hiding-alpha-space ?f
  unfolding composition-trace-bad-alpha-space-def Let-def by auto
  also have ...  $\subseteq$  alpha-space
  by (rule common-denominator-hiding-alpha-space-subset-alpha-space)
  finally show ?thesis .
qed

```

```

lemma composition-trace-bad-alpha-space-fraction-bound:
  nnreal (card (composition-trace-bad-alpha-space trace-table)) /
  nnreal (CARD('f) ^ length spec)  $\leq$  composition-error-bound
proof -
  let ?f = low-degree-trace-witness trace-table

```

```

have subset:
  composition-trace-bad-alpha-space trace-table  $\subseteq$ 
    common-denominator-hiding-alpha-space ?f
  unfolding composition-trace-bad-alpha-space-def Let-def by auto
have card-le:
  card (composition-trace-bad-alpha-space trace-table)  $\leq$ 
    card (common-denominator-hiding-alpha-space ?f)
  by (rule card-mono[OF finite-common-denominator-hiding-alpha-space subset])
have nnreal (card (composition-trace-bad-alpha-space trace-table)) /
  nnreal (CARD('f) ^ length spec)  $\leq$ 
  nnreal (card (common-denominator-hiding-alpha-space ?f)) /
  nnreal (CARD('f) ^ length spec)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
also have ...  $\leq$  composition-error-bound
  by (rule common-denominator-hiding-alpha-space-fraction-bound)
finally show ?thesis .
qed

```

```

lemma composition-trace-bad-alpha-space-fraction-bound-alpha-space:
  nnreal (card (composition-trace-bad-alpha-space trace-table)) /
  nnreal (card alpha-space)  $\leq$  composition-error-bound
using composition-trace-bad-alpha-space-fraction-bound[of trace-table]
by simp

```

```

lemma composition-trace-bad-alpha-space-card-bound:
  card (composition-trace-bad-alpha-space trace-table)  $\leq$ 
    (if length spec = 0 then 0 else CARD('f) ^ (length spec - 1))
proof -
  let ?f = low-degree-trace-witness trace-table
  have subset:
    composition-trace-bad-alpha-space trace-table  $\subseteq$ 
      common-denominator-hiding-alpha-space ?f
    unfolding composition-trace-bad-alpha-space-def Let-def by auto
  have card-le:
    card (composition-trace-bad-alpha-space trace-table)  $\leq$ 
      card (common-denominator-hiding-alpha-space ?f)
    by (rule card-mono[OF finite-common-denominator-hiding-alpha-space subset])
  also have ...  $\leq$ 
    (if length spec = 0 then 0 else CARD('f) ^ (length spec - 1))
    by (rule common-denominator-hiding-alpha-space-card-bound)
  finally show ?thesis .
qed

```

```

lemma trace-powers-of-eval-nth:
  assumes k < powers
  shows poly (trace-powers-of f ! k) x = poly f (g ^ k * x)
  using assms
  unfolding trace-powers-of-def XP-def
  by (simp add: poly-pcompose poly-monom)

```

```

lemma query-trace-power-eval-nth:
  assumes k-bound:  $k < \text{powers}$ 
    and idx-bound:  $\text{idx} + k * \text{scale} < \text{clength} * \text{scale}$ 
  shows
     $\text{map } (\text{poly } f) \text{ eval-domain } ! (\text{idx} + k * \text{scale}) =$ 
       $\text{poly } (\text{trace-powers-of } f ! k) (h \wedge \text{idx} * \text{shift})$ 
proof -
  have eval:
     $\text{map } (\text{poly } f) \text{ eval-domain } ! (\text{idx} + k * \text{scale}) =$ 
       $\text{poly } f (h \wedge (\text{idx} + k * \text{scale}) * \text{shift})$ 
    using eval-domain-nth[OF idx-bound] idx-bound eval-domain-length by simp
  have arg:
     $g \wedge k * (h \wedge \text{idx} * \text{shift}) = h \wedge (\text{idx} + k * \text{scale}) * \text{shift}$ 
proof -
  have gpow:  $g \wedge k = h \wedge (\text{scale} * k)$ 
    using domain-alignment by (simp add: power-mult)
  have  $g \wedge k * (h \wedge \text{idx} * \text{shift}) =$ 
     $h \wedge (\text{scale} * k) * h \wedge \text{idx} * \text{shift}$ 
    using gpow by (simp add: algebra-simps)
  also have  $\dots = h \wedge (\text{idx} + k * \text{scale}) * \text{shift}$ 
    by (simp add: power-add algebra-simps mult.commute)
  finally show ?thesis .
qed
show ?thesis
  using eval trace-powers-of-eval-nth[OF k-bound, of f h \wedge idx * shift] arg
  by simp
qed

lemma query-values-trace-powers:
  assumes trace-table:  $\text{trace-table} = \text{map } (\text{poly } f) \text{ eval-domain}$ 
    and scaled-bound:
       $\bigwedge i. i \in \text{set } (\text{powers-scaled idx}) \implies i < \text{clength} * \text{scale}$ 
  shows
     $\text{map } (! \text{ trace-table}) (\text{powers-scaled idx}) =$ 
       $\text{map } (\lambda q. \text{poly } q (h \wedge \text{idx} * \text{shift})) (\text{trace-powers-of } f)$ 
proof (rule nth-equalityI)
  show  $\text{length } (\text{map } (! \text{ trace-table}) (\text{powers-scaled idx})) =$ 
     $\text{length } (\text{map } (\lambda q. \text{poly } q (h \wedge \text{idx} * \text{shift})) (\text{trace-powers-of } f))$ 
    unfolding powers-scaled-def by simp
next
  fix k
  assume k-bound:
     $k < \text{length } (\text{map } (! \text{ trace-table}) (\text{powers-scaled idx}))$ 
  then have k-lt:  $k < \text{powers}$ 
    unfolding powers-scaled-def by simp
  have scaled-idx-in:
     $\text{idx} + k * \text{scale} \in \text{set } (\text{powers-scaled idx})$ 
    using k-lt unfolding powers-scaled-def by auto

```



```

have idx-bound:  $idx + k * scale < clength * scale$ 
  by (rule scaled-bound[OF scaled-idx-in])
have left:
  map (!) trace-table (powers-scaled idx) !  $k =$ 
    trace-table ! ( $idx + k * scale$ )
  using k-lt unfolding powers-scaled-def by simp
have right:
  map ( $\lambda q. poly\ q\ (h \wedge idx * shift)$ ) (trace-powers-of f) !  $k =$ 
    poly (trace-powers-of f !  $k$ ) ( $h \wedge idx * shift$ )
  using k-lt by simp
show
  map (!) trace-table (powers-scaled idx) !  $k =$ 
    map ( $\lambda q. poly\ q\ (h \wedge idx * shift)$ ) (trace-powers-of f) !  $k$ 
  using query-trace-power-eval-nth[OF k-lt idx-bound] left right trace-table
  by simp
qed

lemma query-consistent-at-trace-powers:
  assumes qc: query-consistent-at trace-table composition-table as idx
  and trace-table: trace-table = map (poly f) eval-domain
  shows
    composition-table ! idx =
      cp-eval as (map ( $\lambda q. poly\ q\ (h \wedge idx * shift)$ ) (trace-powers-of f))
        ( $h \wedge idx * shift$ )
proof –
  have qc-unfolded:
     $idx < clength * scale \wedge$ 
     $length\ trace-table = clength * scale \wedge$ 
     $length\ composition-table = clength * scale \wedge$ 
     $(\forall i \in set\ (powers-scaled\ idx). i < length\ trace-table) \wedge$ 
    composition-table ! idx =
      cp-eval as (map (!) trace-table (powers-scaled idx)) ( $h \wedge idx * shift$ )
  using qc unfolding query-consistent-at-def .
have trace-len:  $length\ trace-table = clength * scale$ 
  using qc-unfolded by blast
have scaled-bound:
     $\bigwedge i. i \in set\ (powers-scaled\ idx) \implies i < clength * scale$ 
proof –
  fix i
  assume i-in:  $i \in set\ (powers-scaled\ idx)$ 
  have  $i < length\ trace-table$ 
  using qc-unfolded i-in by blast
  then show  $i < clength * scale$ 
  using trace-len by simp
qed
have vals-eq:
  map (!) trace-table (powers-scaled idx) =
    map ( $\lambda q. poly\ q\ (h \wedge idx * shift)$ ) (trace-powers-of f)
  by (rule query-values-trace-powers[OF trace-table scaled-bound])

```

```

have comp:
  composition-table ! idx =
    cp-eval as (map (! trace-table) (powers-scaled idx)) (h ^ idx * shift)
using qc-unfolded by blast
have cp-eq:
  cp-eval as (map (! trace-table) (powers-scaled idx)) (h ^ idx * shift) =
    cp-eval as (map (λq. poly q (h ^ idx * shift)) (trace-powers-of f))
      (h ^ idx * shift)
using vals-eq by (rule arg-cong)
show ?thesis
using comp cp-eq by simp
qed

```

```

lemma fold-add-sum-list:
  fixes f :: 'x ⇒ 'b::comm-monoid-add
  shows fold (λx acc. f x + acc) xs acc = sum-list (map f xs) + acc
proof (induction xs arbitrary: acc)
  case Nil
  then show ?case by simp
next
  case (Cons x xs)
  then show ?case
    by (simp add: ac-simps)
qed

```

```

lemma cp-eval-agrees-with-spec-lists:
  fixes specs :: (('f poly list ⇒ 'f poly) × nat list × nat) list
  and spec2s :: (('f list ⇒ 'f) × nat list) list
  and qs :: 'f poly list
  assumes len: length spec2s = length specs
  and zip-subset: set (zip specs spec2s) ⊆ set (zip spec spec2)
  and qs-len: powers ≤ length qs
  shows
    fold
      (λ(a, (c2, roots2)).
        (+) (a * (c2 (map (λq. poly q x) qs) /
          poly (prod (g-map roots2)) x)))
      (zip as spec2s) vacc =
    sum-list
      (map
        (λ(a, (c, roots, d)).
          a * (poly (c qs) x / poly (prod (g-map roots)) x))
        (zip as specs)) + vacc
proof –
  let ?vterm =
    λz. case z of (a, entry) ⇒
      case entry of (c2, roots2) ⇒
        a * (c2 (map (λq. poly q x) qs) /
          poly (prod (g-map roots2)) x)

```

```

let ?sterm =
  λz. case z of (a, entry) ⇒
    case entry of (c, roots, d) ⇒
      a * (poly (c qs) x / poly (prod (g-map roots)) x)
have terms-eq:
  map ?vterm (zip as spec2s) = map ?sterm (zip as specs)
proof (rule nth-equalityI)
  show length (map ?vterm (zip as spec2s)) =
    length (map ?sterm (zip as specs))
  using len by simp
next
fix i
assume i-bound: i < length (map ?vterm (zip as spec2s))
then have i-as: i < length as
  by simp
have i-spec2: i < length spec2s
  using i-bound by simp
have i-specs: i < length specs
  using i-spec2 len by simp
obtain c roots d where specs-i: specs ! i = (c, roots, d)
  by (cases specs ! i) auto
obtain c2 roots2 where spec2s-i: spec2s ! i = (c2, roots2)
  by (cases spec2s ! i) auto
have pair-in:
  ((c, roots, d), (c2, roots2)) ∈ set (zip spec spec2)
proof -
  have (specs ! i, spec2s ! i) ∈ set (zip specs spec2s)
  proof -
    have i < length (zip specs spec2s)
    using i-specs i-spec2 by simp
    then have zip specs spec2s ! i ∈ set (zip specs spec2s)
    by (rule nth-mem)
    then show ?thesis
    using i-specs i-spec2 by simp
  qed
  then show ?thesis
  using zip-subset specs-i spec2s-i by auto
qed
have roots-eq: roots2 = roots
and raw-eq:
  c2 (map (λq. poly q x) qs) = poly (c qs) x
using specs-agree[OF pair-in qs-len, of x] by simp-all
show map ?vterm (zip as spec2s) ! i =
  map ?sterm (zip as specs) ! i
using i-as i-spec2 i-specs specs-i spec2s-i roots-eq raw-eq
by simp
qed
have fold-eq:
  fold

```

```

    (λ(a, (c2, roots2))).
    (+) (a * (c2 (map (λq. poly q x) qs) /
      poly (prod (g-map roots2)) x)))
    (zip as spec2s) vacc =
  fold (λz acc. ?vterm z + acc) (zip as spec2s) vacc
  by (rule fold-cong) (auto simp: fun-eq-iff split: prod.splits)
have
  fold
    (λ(a, (c2, roots2))).
    (+) (a * (c2 (map (λq. poly q x) qs) /
      poly (prod (g-map roots2)) x)))
    (zip as spec2s) vacc =
  sum-list (map ?vterm (zip as spec2s)) + vacc
  unfolding fold-eq
  by (rule fold-add-sum-list)
also have ... =
  sum-list
    (map
      (λ(a, (c, roots, d)).
        a * (poly (c qs) x / poly (prod (g-map roots)) x))
      (zip as specs)) + vacc
    using terms-eq by simp
  finally show ?thesis .
qed

lemma cp-eval-trace-powers-agrees-with-spec:
  cp-eval as (map (λq. poly q x) (trace-powers-of f)) x =
  composition-spec-eval f as x
proof -
  have aligned:
    fold
      (λ(a, (c2, roots2))).
      (+) (a * (c2
        (map (λq. poly q x) (trace-powers-of f)) /
        poly (prod (g-map roots2)) x)))
      (zip as spec2) 0 =
    sum-list
      (map
        (λ(a, (c, roots, d)).
          a * (poly (c (trace-powers-of f)) x /
            poly (prod (g-map roots)) x))
        (zip as spec)) + 0
      by (rule cp-eval-agrees-with-spec-lists[OF spec2-length subset-refl])
      simp
  have spec-sum:
    sum-list
      (map
        (λ(a, (c, roots, d)).
          a * (poly (c (trace-powers-of f)) x /

```

```

      poly (prod (g-map roots)) x))
    (zip as spec)) =
    sum-list (map (composition-spec-eval-term f x) (zip as spec))
  apply (intro arg-cong[where f=sum-list] map-cong refl)
  apply (auto simp: composition-spec-eval-term-def split-beta'
    case-prod-unfold split: prod.splits)
done
show ?thesis
  unfolding cp-eval-def composition-spec-eval-def
  using aligned-spec-sum by simp
qed

lemma common-denominator-mult-cp-eval-trace-powers:
  assumes idx-bound: idx < clength * scale
  shows
    poly common-denominator-poly (h ^ idx * shift) *
      cp-eval as
      (map (λq. poly q (h ^ idx * shift)) (trace-powers-of f))
      (h ^ idx * shift) =
      poly (common-denominator-numerator f as) (h ^ idx * shift)
  proof -
    have denom:
      ∧ c roots d. (c, roots, d) ∈ set spec ⇒
      poly (prod (g-map roots)) (h ^ idx * shift) ≠ 0
    proof -
      fix c roots d
      assume spec-entry: (c, roots, d) ∈ set spec
      show poly (prod (g-map roots)) (h ^ idx * shift) ≠ 0
        by (rule query-denominator-nonzero[OF spec-entry idx-bound])
    qed
    have mult:
      poly common-denominator-poly (h ^ idx * shift) *
        composition-spec-eval f as (h ^ idx * shift) =
        poly (common-denominator-numerator f as) (h ^ idx * shift)
      by (rule common-denominator-mult-composition-spec-eval[OF denom])
    show ?thesis
      using mult cp-eval-trace-powers-agrees-with-spec[of as h ^ idx * shift f]
      by simp
  qed

lemma common-denominator-numerator-identity-residual-zero:
  assumes len: length as = length spec
  and identity:
    common-denominator-numerator f as = common-denominator-poly * q
  and spec-entry: (c, roots, d) ∈ set spec
  and r-in: r ∈ set roots
  shows common-denominator-composition-root-residual f as r = 0
  proof -
    have numerator-eval:

```

```

poly (common-denominator-numerator f as) (g ^ r) =
  common-denominator-composition-root-residual f as r
by (rule common-denominator-numerator-root-eval[OF len spec-entry r-in])
have root-zero: poly common-denominator-poly (g ^ r) = 0
by (rule common-denominator-vanishes-on-constraint-root
  [OF spec-entry r-in])
have poly (common-denominator-numerator f as) (g ^ r) = 0
  using identity root-zero by simp
then show ?thesis
  using numerator-eval by simp
qed

```

```

lemma common-denominator-numerator-identity-hides-violations:
  assumes len: length as = length spec
  and violated: violated-constraints f ≠ {}
  and identity:
    common-denominator-numerator f as = common-denominator-poly * q
  shows random-combination-common-denominator-hides-violations f as
proof -
  have residual-zero:
     $\bigwedge c \text{ roots } d \text{ r.}$ 
     $(c, \text{roots}, d) \in \text{violated-constraints } f \implies$ 
     $r \in \text{set roots} \implies$ 
     $\text{common-denominator-composition-root-residual } f \text{ as } r = 0$ 
  proof -
    fix c roots d r
    assume entry:  $(c, \text{roots}, d) \in \text{violated-constraints } f$ 
    and r-in:  $r \in \text{set roots}$ 
    have spec-entry:  $(c, \text{roots}, d) \in \text{set spec}$ 
    using entry unfolding violated-constraints-def by simp
    show common-denominator-composition-root-residual f as r = 0
    by (rule common-denominator-numerator-identity-residual-zero
      [OF len identity spec-entry r-in])
  qed
  show ?thesis
    using violated residual-zero
    unfolding random-combination-common-denominator-hides-violations-def
    by auto
qed

```

```

lemma composition-table-agrees-with-rational-expression:
  assumes composition-low:
    composition-table-low-degree maxDegree composition-table
  and trace-table: trace-table = map (poly f) eval-domain
  and all-queries: all-queries-consistent trace-table composition-table as
  obtains q where
    degree q ≤ maxDegree
  and  $\bigwedge \text{idx. idx} \in \text{query-sample-space} \implies$ 
    poly q (h ^ idx * shift) =

```

```

      cp-eval as (map (λp. poly p (h ^ idx * shift)) (trace-powers-of f))
      (h ^ idx * shift)
proof -
  from composition-low obtain q where deg-q: degree q ≤ maxDegree
  and table-q: composition-table = map (poly q) eval-domain
  unfolding composition-table-low-degree-def by blast
  have agree:
    ∧idx. idx ∈ query-sample-space ⇒
      poly q (h ^ idx * shift) =
        cp-eval as (map (λp. poly p (h ^ idx * shift)) (trace-powers-of f))
        (h ^ idx * shift)
  proof -
    fix idx
    assume idx-in: idx ∈ query-sample-space
    have idx-bound: idx < clength * scale
    by (rule query-sample-space-less-domain[OF idx-in])
    have qc: query-consistent-at trace-table composition-table as idx
    using all-queries idx-in unfolding all-queries-consistent-def by simp
    have comp-value:
      composition-table ! idx = poly q (h ^ idx * shift)
    using table-q eval-domain-nth[OF idx-bound] idx-bound eval-domain-length
    by simp
    show poly q (h ^ idx * shift) =
      cp-eval as (map (λp. poly p (h ^ idx * shift)) (trace-powers-of f))
      (h ^ idx * shift)
    using query-consistent-at-trace-powers[OF qc trace-table] comp-value
    by simp
  qed
  show ?thesis
  by (rule that[OF deg-q agree])
qed

lemma common-denominator-numerator-identity-from-queries:
  assumes composition-low:
    composition-table-low-degree maxDegree composition-table
  and trace-table: trace-table = map (poly f) eval-domain
  and all-queries: all-queries-consistent trace-table composition-table as
  and num-deg:
    degree (common-denominator-numerator f as) < query-sample-space-size
  and prod-deg:
    ∧q. degree q ≤ maxDegree ⇒
      degree (common-denominator-poly * q) < query-sample-space-size
  obtains q where
    degree q ≤ maxDegree
  and common-denominator-numerator f as = common-denominator-poly * q
proof -
  from composition-table-agrees-with-rational-expression
  [OF composition-low trace-table all-queries]
  obtain q where deg-q: degree q ≤ maxDegree

```

```

and agree:
   $\bigwedge idx. idx \in \text{query-sample-space} \implies$ 
     $\text{poly } q (h \wedge idx * \text{shift}) =$ 
      cp-eval as
         $(\text{map } (\lambda p. \text{poly } p (h \wedge idx * \text{shift})) (\text{trace-powers-of } f))$ 
         $(h \wedge idx * \text{shift})$ 
    by blast
have eval-agree:
   $\bigwedge idx. idx \in \text{query-sample-space} \implies$ 
     $\text{poly } (\text{common-denominator-numerator } f \text{ as}) (h \wedge idx * \text{shift}) =$ 
     $\text{poly } (\text{common-denominator-poly} * q) (h \wedge idx * \text{shift})$ 
proof –
  fix idx
  assume idx-in:  $idx \in \text{query-sample-space}$ 
  have idx-bound:  $idx < \text{clength} * \text{scale}$ 
    by (rule query-sample-space-less-domain[OF idx-in])
  have mult:
     $\text{poly } \text{common-denominator-poly} (h \wedge idx * \text{shift}) *$ 
    cp-eval as
       $(\text{map } (\lambda p. \text{poly } p (h \wedge idx * \text{shift})) (\text{trace-powers-of } f))$ 
       $(h \wedge idx * \text{shift}) =$ 
       $\text{poly } (\text{common-denominator-numerator } f \text{ as}) (h \wedge idx * \text{shift})$ 
    by (rule common-denominator-mult-cp-eval-trace-powers[OF idx-bound])
  have poly  $(\text{common-denominator-poly} * q) (h \wedge idx * \text{shift}) =$ 
     $\text{poly } \text{common-denominator-poly} (h \wedge idx * \text{shift}) *$ 
     $\text{poly } q (h \wedge idx * \text{shift})$ 
    by simp
  also have ... =
     $\text{poly } \text{common-denominator-poly} (h \wedge idx * \text{shift}) *$ 
    cp-eval as
       $(\text{map } (\lambda p. \text{poly } p (h \wedge idx * \text{shift})) (\text{trace-powers-of } f))$ 
       $(h \wedge idx * \text{shift})$ 
    using agree[OF idx-in] by simp
  also have ... =  $\text{poly } (\text{common-denominator-numerator } f \text{ as}) (h \wedge idx * \text{shift})$ 
    using mult by simp
  finally show
     $\text{poly } (\text{common-denominator-numerator } f \text{ as}) (h \wedge idx * \text{shift}) =$ 
     $\text{poly } (\text{common-denominator-poly} * q) (h \wedge idx * \text{shift})$ 
    by simp
qed
have identity:
   $\text{common-denominator-numerator } f \text{ as} = \text{common-denominator-poly} * q$ 
  by (rule poly-eq-on-query-sample-domainI[OF num-deg prod-deg[OF deg-q]])
  (rule eval-agree)
show ?thesis
  by (rule that[OF deg-q identity])
qed

```

lemma common-denominator-numerator-identity-from-queries-bounds:


```

assumes composition-low:
  composition-table-low-degree maxDegree composition-table
and trace-table: trace-table = map (poly f) eval-domain
and all-queries: all-queries-consistent trace-table composition-table as
and bounds: common-denominator-degree-bounds f as
obtains q where
  degree q ≤ maxDegree
and common-denominator-numerator f as = common-denominator-poly * q
proof –
have num-deg:
  degree (common-denominator-numerator f as) < query-sample-space-size
using bounds unfolding common-denominator-degree-bounds-def by simp
have prod-deg:
   $\wedge q. \text{degree } q \leq \text{maxDegree} \implies$ 
    degree (common-denominator-poly * q) < query-sample-space-size
using bounds unfolding common-denominator-degree-bounds-def by simp
from common-denominator-numerator-identity-from-queries
  [OF composition-low trace-table all-queries num-deg prod-deg]
obtain q where deg-q: degree q ≤ maxDegree
and identity:
  common-denominator-numerator f as = common-denominator-poly * q
by blast
show ?thesis
by (rule that[OF deg-q identity])
qed

```

lemma *random-combination-common-denominator-hides-from-queries*:

```

assumes len-as: length as = length spec
and false-statement:  $\neg$  exists-valid-trace
and deg-f: degree f < clength
and trace-table: trace-table = map (poly f) eval-domain
and composition-low:
  composition-table-low-degree maxDegree composition-table
and all-queries: all-queries-consistent trace-table composition-table as
and bounds: common-denominator-degree-bounds f as
shows random-combination-common-denominator-hides-violations f as

```

```

proof –
have violated: violated-constraints f ≠ {}
by (rule false-statement-violated-constraints[OF false-statement deg-f])
from common-denominator-numerator-identity-from-queries-bounds
  [OF composition-low trace-table all-queries bounds]
obtain q where
  degree q ≤ maxDegree
and identity:
  common-denominator-numerator f as = common-denominator-poly * q
by blast
show ?thesis
by (rule common-denominator-numerator-identity-hides-violations
  [OF len-as violated identity])

```

qed

lemma *all-queries-consistent-iff-no-bad-queries:*

all-queries-consistent trace-table composition-table as $\longleftrightarrow$

bad-query-indices trace-table composition-table as = $\{\}$

unfolding *all-queries-consistent-def bad-query-indices-def* **by** *auto*

lemma *query-disagreement-indices-nonempty-iff:*

query-disagreement-indices trace-table composition-table as $\neq \{\}$ $\longleftrightarrow$

$\neg$ all-queries-consistent trace-table composition-table as

unfolding *query-disagreement-indices-def*

using *all-queries-consistent-iff-no-bad-queries* **by** *simp*

lemma *query-consistent-at-from-common-denominator-residual-zero:*

assumes *trace-table: trace-table = map (poly f) eval-domain*

and *composition-table: composition-table = map (poly q) eval-domain*

and *residual-zero:*

*common-denominator-numerator f as - common-denominator-poly * q = 0*

and *idx-in: idx $\in$ query-sample-space*

shows *query-consistent-at trace-table composition-table as idx*

proof -

have *idx-bound: idx < clength * scale*

by *(rule query-sample-space-less-domain[OF idx-in])*

have *trace-len: length trace-table = clength * scale*

using *trace-table eval-domain-length* **by** *simp*

have *comp-len: length composition-table = clength * scale*

using *composition-table eval-domain-length* **by** *simp*

have *powers-bound:*

$\forall i \in \text{set (powers-scaled idx)}. i < \text{length trace-table}$

proof

fix *i*

assume *i-in: i $\in$ set (powers-scaled idx)*

show *i < length trace-table*

using *query-sample-space-powers-scaled-bound[OF idx-in i-in] trace-len*

by *simp*

qed

have *comp-value:*

*composition-table ! idx = poly q (h $\wedge$ idx * shift)*

using *composition-table eval-domain-nth[OF idx-bound] idx-bound eval-domain-length*

by *simp*

have *trace-powers:*

map ((!) trace-table) (powers-scaled idx) =

*map ($\lambda p. \text{poly } p (h \wedge \text{idx} * \text{shift})$) (trace-powers-of f)*

by *(rule query-values-trace-powers[OF trace-table])*

(rule query-sample-space-powers-scaled-bound[OF idx-in])

have *mult:*

*poly common-denominator-poly (h $\wedge$ idx * shift) **

cp-eval as

*(map ($\lambda p. \text{poly } p (h \wedge \text{idx} * \text{shift})$) (trace-powers-of f))*

```

      (h ^ idx * shift) =
      poly (common-denominator-numerator f as) (h ^ idx * shift)
    by (rule common-denominator-mult-cp-eval-trace-powers[OF idx-bound])
  have denom-nonzero:
    poly common-denominator-poly (h ^ idx * shift) ≠ 0
  by (rule common-denominator-nonzero-on-eval-domain[OF idx-bound])
  have residual-eval:
    poly (common-denominator-numerator f as) (h ^ idx * shift) =
    poly common-denominator-poly (h ^ idx * shift) *
    poly q (h ^ idx * shift)
  using residual-zero by simp
  have cp-eq:
    cp-eval as
      (map (λp. poly p (h ^ idx * shift)) (trace-powers-of f))
      (h ^ idx * shift) =
    poly q (h ^ idx * shift)
  using mult residual-eval denom-nonzero by simp
  show ?thesis
  unfolding query-consistent-at-def
  using idx-bound trace-len comp-len powers-bound comp-value trace-powers cp-eq
  by simp
qed

```

lemma *all-queries-consistent-from-common-denominator-residual-zero:*
assumes *trace-table:* trace-table = map (poly f) eval-domain
and *composition-table:* composition-table = map (poly q) eval-domain
and *residual-zero:*
 common-denominator-numerator f as – *common-denominator-poly * q = 0*
shows *all-queries-consistent* trace-table composition-table as
unfolding *all-queries-consistent-def*
using *query-consistent-at-from-common-denominator-residual-zero*
 [*OF trace-table composition-table residual-zero*]
by *blast*

lemma *common-denominator-residual-nonzero-from-query-disagreement:*
assumes *trace-table:* trace-table = map (poly f) eval-domain
and *composition-table:* composition-table = map (poly q) eval-domain
and *not-all:* ¬ *all-queries-consistent* trace-table composition-table as
shows *common-denominator-numerator f as* – *common-denominator-poly * q ≠ 0*
proof
assume *zero:*
 common-denominator-numerator f as – *common-denominator-poly * q = 0*
have *all-queries-consistent* trace-table composition-table as
by (rule *all-queries-consistent-from-common-denominator-residual-zero*
 [*OF trace-table composition-table zero*])
then show *False*
using *not-all* **by** *simp*
qed

```

lemma query-agreement-indices-card-bound:
  assumes trace-low: trace-table-low-degree trace-table
  and composition-low:
    composition-table-low-degree maxDegree composition-table
  and not-all:
     $\neg$  all-queries-consistent trace-table composition-table as
  shows card (query-agreement-indices trace-table composition-table as)  $\leq$ 
    query-agreement-bound
proof –
  from trace-low obtain f where deg-f: degree f < clength
  and trace-table: trace-table = map (poly f) eval-domain
  unfolding trace-table-low-degree-def by blast
  from composition-low obtain q where deg-q: degree q  $\leq$  maxDegree
  and composition-table: composition-table = map (poly q) eval-domain
  unfolding composition-table-low-degree-def by blast
  let ?A = query-agreement-indices trace-table composition-table as
  let ?R = common-denominator-numerator f as – common-denominator-poly * q
  have A-subset: ?A  $\subseteq$  query-sample-space
  unfolding query-agreement-indices-def by auto
  have finite-A: finite ?A
  unfolding query-agreement-indices-def query-sample-space-def by simp
  have inj: inj-on ( $\lambda$ idx. h  $\wedge$  idx * shift) ?A
  proof (intro inj-onI)
    fix i j
    assume i-in: i  $\in$  ?A
    and j-in: j  $\in$  ?A
    and eq: h  $\wedge$  i * shift = h  $\wedge$  j * shift
    have i-bound: i < clength * scale
    by (rule query-sample-space-less-domain[OF set-mp[OF A-subset i-in]])
    have j-bound: j < clength * scale
    by (rule query-sample-space-less-domain[OF set-mp[OF A-subset j-in]])
    have h  $\wedge$  i = h  $\wedge$  j
    using eq shift-nonzero by simp
    then show i = j
    by (rule h-power-inj-on-eval-domain[OF i-bound j-bound])
  qed
  have residual-nonzero:
    ?R  $\neq$  0
    by (rule common-denominator-residual-nonzero-from-query-disagreement
      [OF trace-table composition-table not-all])
  have image-subset:
    ( $\lambda$ idx. h  $\wedge$  idx * shift) ‘ ?A  $\subseteq$  {x. poly ?R x = 0}
  proof
    fix x
    assume x-in: x  $\in$  ( $\lambda$ idx. h  $\wedge$  idx * shift) ‘ ?A
    then obtain idx where idx-in: idx  $\in$  ?A
    and x-eq: x = h  $\wedge$  idx * shift
    by blast
    have idx-sample: idx  $\in$  query-sample-space

```

```

    by (rule set-mp[OF A-subset idx-in])
  have idx-bound:  $idx < clength * scale$ 
    by (rule query-sample-space-less-domain[OF idx-sample])
  have qc: query-consistent-at trace-table composition-table as idx
    using idx-in unfolding query-agreement-indices-def by simp
  have comp-value:
    composition-table ! idx = poly q ( $h \wedge idx * shift$ )
    using composition-table eval-domain-nth[OF idx-bound] idx-bound
      eval-domain-length by simp
  have trace-powers:
    map (! trace-table) (powers-scaled idx) =
      map ( $\lambda p. \text{poly } p (h \wedge idx * shift)$ ) (trace-powers-of f)
    by (rule query-values-trace-powers[OF trace-table])
      (rule query-sample-space-powers-scaled-bound[OF idx-sample])
  have mult:
    poly common-denominator-poly ( $h \wedge idx * shift$ ) *
      cp-eval as
      (map ( $\lambda p. \text{poly } p (h \wedge idx * shift)$ ) (trace-powers-of f))
      ( $h \wedge idx * shift$ ) =
      poly (common-denominator-numerator f as) ( $h \wedge idx * shift$ )
    by (rule common-denominator-mult-cp-eval-trace-powers[OF idx-bound])
  have cp-eq:
    cp-eval as
      (map ( $\lambda p. \text{poly } p (h \wedge idx * shift)$ ) (trace-powers-of f))
      ( $h \wedge idx * shift$ ) =
      poly q ( $h \wedge idx * shift$ )
    using qc comp-value trace-powers
    unfolding query-consistent-at-def by simp
  have poly (common-denominator-numerator f as) ( $h \wedge idx * shift$ ) =
    poly common-denominator-poly ( $h \wedge idx * shift$ ) *
    poly q ( $h \wedge idx * shift$ )
    using mult cp-eq by simp
  then have poly ?R ( $h \wedge idx * shift$ ) = 0
    by simp
  then show  $x \in \{x. \text{poly } ?R x = 0\}$ 
    unfolding x-eq by simp
qed
have card ?A = card (( $\lambda idx. h \wedge idx * shift$ ) ‘ ?A)
  using card-image[OF inj] by simp
also have  $\dots \leq \text{card } \{x. \text{poly } ?R x = 0\}$ 
  by (rule card-mono[OF poly-roots-finite[OF residual-nonzero] image-subset])
also have  $\dots \leq \text{degree } ?R$ 
  by (rule card-poly-roots-bound[OF residual-nonzero])
also have  $\dots \leq \text{query-agreement-bound}$ 
  by (rule common-denominator-residual-degree-le-bound[OF deg-f deg-q])
finally show ?thesis .
qed
end

```

end

theory *Soundness-Core-WP*
imports *Soundness-Core-Base*
begin

Generic probability and WP combinators used by soundness reductions.

context *soundness*
begin

lemma *wp-event-union-bound*:

fixes $P\ Q :: ('x \times 's)\ \text{option} \Rightarrow \text{bool}$

shows

$$\text{wp-event } m\ (\lambda \text{out. } P\ \text{out} \vee Q\ \text{out})\ s \leq \\ \text{wp-event } m\ P\ s + \text{wp-event } m\ Q\ s$$

proof –

have *pointwise*:

$$\bigwedge \text{out } p. p * (\text{if } P\ \text{out} \vee Q\ \text{out} \text{ then } 1 \text{ else } 0) \leq \\ p * ((\text{if } P\ \text{out} \text{ then } 1 \text{ else } 0) + (\text{if } Q\ \text{out} \text{ then } 1 \text{ else } 0)) :: \text{prob}$$

by (*simp add: mult-left-mono*)

have $\text{wp-event } m\ (\lambda \text{out. } P\ \text{out} \vee Q\ \text{out})\ s \leq$

$$\text{wp } m\ (\lambda \text{out. } (\text{if } P\ \text{out} \text{ then } 1 \text{ else } 0) + (\text{if } Q\ \text{out} \text{ then } 1 \text{ else } 0))\ s$$

unfolding *wp-event-def wp-def dist-expect-def*

by (*intro sum-mono pointwise*)

also have $\dots = \text{wp-event } m\ P\ s + \text{wp-event } m\ Q\ s$

unfolding *wp-event-def wp-def dist-expect-def*

by (*simp add: algebra-simps sum.distrib*)

finally show *?thesis* .

qed

lemma *wp-event-union-bound4*:

fixes $A\ B\ C\ D :: ('x \times 's)\ \text{option} \Rightarrow \text{bool}$

shows

$$\text{wp-event } m\ (\lambda \text{out. } A\ \text{out} \vee B\ \text{out} \vee C\ \text{out} \vee D\ \text{out})\ s \leq \\ \text{wp-event } m\ A\ s + \text{wp-event } m\ B\ s + \text{wp-event } m\ C\ s + \text{wp-event } m\ D\ s$$

proof –

have *pointwise*:

$$\bigwedge \text{out } p. p * (\text{if } A\ \text{out} \vee B\ \text{out} \vee C\ \text{out} \vee D\ \text{out} \text{ then } 1 \text{ else } 0) \leq \\ p * ((\text{if } A\ \text{out} \text{ then } 1 \text{ else } 0) + (\text{if } B\ \text{out} \text{ then } 1 \text{ else } 0) + \\ (\text{if } C\ \text{out} \text{ then } 1 \text{ else } 0) + (\text{if } D\ \text{out} \text{ then } 1 \text{ else } 0)) :: \text{prob}$$

by (*simp add: mult-left-mono*)

have $\text{wp-event } m\ (\lambda \text{out. } A\ \text{out} \vee B\ \text{out} \vee C\ \text{out} \vee D\ \text{out})\ s \leq$

$$\text{wp } m\ (\lambda \text{out. } (\text{if } A\ \text{out} \text{ then } 1 \text{ else } 0) + (\text{if } B\ \text{out} \text{ then } 1 \text{ else } 0) + \\ (\text{if } C\ \text{out} \text{ then } 1 \text{ else } 0) + (\text{if } D\ \text{out} \text{ then } 1 \text{ else } 0))\ s$$

unfolding *wp-event-def wp-def dist-expect-def*

by (*intro sum-mono pointwise*)

```

also have ... =
  wp-event m A s + wp-event m B s + wp-event m C s + wp-event m D s
  unfolding wp-event-def wp-def dist-expect-def
  by (simp add: algebra-simps sum.distrib)
finally show ?thesis
.
qed

lemma wp-event-finite-UN-bound:
  fixes P :: 'a  $\Rightarrow$  ('x  $\times$  's) option  $\Rightarrow$  bool
  and C :: 'a  $\Rightarrow$  prob
  assumes finite-A: finite A
  and bounds:  $\bigwedge a. a \in A \implies wp\text{-event } m (P a) s \leq C a$ 
  shows
    wp-event m ( $\lambda out. \exists a \in A. P a out$ ) s  $\leq$  ( $\sum a \in A. C a$ )
  proof -
    have pointwise:
       $\bigwedge out p. p * (if \exists a \in A. P a out \text{ then } 1 \text{ else } 0) \leq$ 
       $p * (\sum a \in A. if P a out \text{ then } 1 \text{ else } 0 :: prob)$ 
    proof -
      fix out p
      have indicator-le:
         $(if \exists a \in A. P a out \text{ then } 1 \text{ else } 0 :: prob) \leq$ 
         $(\sum a \in A. if P a out \text{ then } 1 \text{ else } 0)$ 
      proof (cases  $\exists a \in A. P a out$ )
        case True
        then obtain a where a:  $a \in A \ P a out$ 
        by blast
        have 1 =  $(if P a out \text{ then } 1 \text{ else } 0 :: prob)$ 
        using a by simp
        also have ...  $\leq (\sum a \in A. if P a out \text{ then } 1 \text{ else } 0)$ 
        by (rule member-le-sum) (use a finite-A in simp-all)
        finally show ?thesis
        using True by simp
      next
        case False
        then show ?thesis by simp
      qed
    show  $p * (if \exists a \in A. P a out \text{ then } 1 \text{ else } 0) \leq$ 
       $p * (\sum a \in A. if P a out \text{ then } 1 \text{ else } 0 :: prob)$ 
    by (simp add: mult-left-mono[OF indicator-le])
  qed
  have wp-event m ( $\lambda out. \exists a \in A. P a out$ ) s  $\leq$ 
    wp m ( $\lambda out. (\sum a \in A. if P a out \text{ then } 1 \text{ else } 0)$ ) s
  unfolding wp-event-def wp-def dist-expect-def
  by (intro sum-mono pointwise)
  also have ... =  $(\sum a \in A. wp\text{-event } m (P a) s)$ 
  using finite-A
  proof (induction A)

```

```

case empty
then show ?case
  unfolding wp-event-def wp-def dist-expect-def by simp
next
case (insert a A)
have wp m (λout. ∑ x∈insert a A. if P x out then 1 else 0) s =
  wp m (λout.
    (if P a out then 1 else 0) +
    (∑ x∈A. if P x out then 1 else 0)) s
  using insert.hyps by simp
also have ... =
  wp-event m (P a) s +
  wp m (λout. ∑ x∈A. if P x out then 1 else 0) s
  unfolding wp-event-def wp-def dist-expect-def
  by (simp add: sum.distrib algebra-simps)
also have ... = wp-event m (P a) s + (∑ x∈A. wp-event m (P x) s)
  using insert.IH by simp
also have ... = (∑ x∈insert a A. wp-event m (P x) s)
  using insert.hyps by simp
finally show ?case .
qed
also have ... ≤ (∑ a∈A. C a)
  by (rule sum-mono) (rule bounds, assumption)
finally show ?thesis .
qed

```

```

lemma wp-event-mono:
  fixes P Q :: ('x × 's) option ⇒ bool
  assumes ∧out. P out ⇒ Q out
  shows wp-event m P s ≤ wp-event m Q s
proof –
  have pointwise:
    ∧out p. p * (if P out then 1 else 0) ≤
    p * (if Q out then 1 else 0 :: prob)
    using assms by (simp add: mult-left-mono)
  show ?thesis
    unfolding wp-event-def wp-def dist-expect-def
    by (intro sum-mono pointwise)
qed

```

```

lemma wp-event-le-1:
  fixes P :: ('x × 's) option ⇒ bool
  shows wp-event m P s ≤ 1
proof –
  have pointwise:
    ∧out p. p * (if P out then 1 else 0 :: prob) ≤ p
    by (simp add: mult-left-le)
  have wp-event m P s ≤
    (∑ out∈dom (dist (execute m s)).

```



```

      the (dist (execute m s) out))
    unfolding wp-event-def wp-def dist-expect-def
    by (intro sum-mono pointwise)
  also have ... = 1
    unfolding sum-map-def[symmetric] by simp
  finally show ?thesis .
qed

lemma wp-le-const-on-support:
  assumes  $\bigwedge out. out \in \text{set-dist } (execute\ m\ s) \implies Q\ out \leq C$ 
  shows  $wp\ m\ Q\ s \leq C$ 
proof -
  have pointwise:
     $\bigwedge out\ p. out \in \text{dom } (dist\ (execute\ m\ s)) \implies$ 
     $p * Q\ out \leq p * C$ 
  using assms unfolding set-dist-def by (simp add: mult-left-mono)
  have  $wp\ m\ Q\ s \leq$ 
     $(\sum out \in \text{dom } (dist\ (execute\ m\ s)).$ 
     $the\ (dist\ (execute\ m\ s)\ out) * C)$ 
  unfolding wp-def dist-expect-def
  by (intro sum-mono pointwise)
  also have ... =
     $(\sum out \in \text{dom } (dist\ (execute\ m\ s)).$ 
     $the\ (dist\ (execute\ m\ s)\ out)) * C$ 
  by (simp add: sum-distrib-right)
  also have ... = C
    unfolding sum-map-def[symmetric] by simp
  finally show ?thesis .
qed

```

```

lemma wp-mono-on-support:
  assumes  $\bigwedge out. out \in \text{set-dist } (execute\ m\ s) \implies Q\ out \leq R\ out$ 
  shows  $wp\ m\ Q\ s \leq wp\ m\ R\ s$ 
proof -
  have pointwise:
     $\bigwedge out\ p. out \in \text{dom } (dist\ (execute\ m\ s)) \implies$ 
     $p * Q\ out \leq p * R\ out$ 
  using assms unfolding set-dist-def by (simp add: mult-left-mono)
  show ?thesis
    unfolding wp-def dist-expect-def
    by (intro sum-mono pointwise) simp
qed

```

```

lemma wp-event-mono-on-support:
  fixes  $P\ Q :: ('x \times 's)\ \text{option} \Rightarrow \text{bool}$ 
  assumes  $\bigwedge out. out \in \text{set-dist } (execute\ m\ s) \implies P\ out \implies Q\ out$ 
  shows  $wp\text{-event}\ m\ P\ s \leq wp\text{-event}\ m\ Q\ s$ 
proof -
  have pointwise:

```

$\bigwedge out\ p. out \in \text{dom } (\text{dist } (\text{execute } m\ s)) \implies$
 $p * (\text{if } P\ out\ \text{then } 1\ \text{else } 0) \leq$
 $p * (\text{if } Q\ out\ \text{then } 1\ \text{else } 0 :: \text{prob})$
using *assms* **unfolding** *set-dist-def* **by** (*simp add: mult-left-mono*)
show *?thesis*
unfolding *wp-event-def wp-def dist-expect-def*
by (*intro sum-mono pointwise*) *simp*
qed

lemma *wp-event-pos-of-support*:
assumes *out*: $out \in \text{set-dist } (\text{execute } m\ s)$
and *event*: $P\ out$
shows $0 < \text{wp-event } m\ P\ s$
proof –
let $?D = \text{dom } (\text{dist } (\text{execute } m\ s))$
have *out-dom*: $out \in ?D$
using *out* **unfolding** *set-dist-def* .
have *out-weight-pos*: $0 < \text{the } (\text{dist } (\text{execute } m\ s)\ out)$
using *dist-nonzero-on-dom[OF out-dom]* **by** *simp*
have $0 < (\sum_{x \in ?D} \text{the } (\text{dist } (\text{execute } m\ s)\ x) * (\text{if } P\ x\ \text{then } 1\ \text{else } 0 :: \text{prob}))$
proof (*rule sum-pos2*)
show *finite ?D*
by *simp*
show $out \in ?D$
by (*rule out-dom*)
show $0 < \text{the } (\text{dist } (\text{execute } m\ s)\ out) * (\text{if } P\ out\ \text{then } 1\ \text{else } 0 :: \text{prob})$
using *event out-weight-pos* **by** *simp*
qed *simp*
then show *?thesis*
unfolding *wp-event-def wp-def dist-expect-def* .
qed

lemma *wp-event-pos-imp-exists-support*:
assumes $0 < \text{wp-event } m\ P\ s$
obtains *out* **where** $out \in \text{set-dist } (\text{execute } m\ s)$ **and** $P\ out$
proof –
have $\exists out \in \text{set-dist } (\text{execute } m\ s). P\ out$
proof (*rule ccontr*)
assume $\neg (\exists out \in \text{set-dist } (\text{execute } m\ s). P\ out)$
then have *no-event*:
 $\bigwedge out. out \in \text{dom } (\text{dist } (\text{execute } m\ s)) \implies \neg P\ out$
unfolding *set-dist-def* **by** *blast*
have $\text{wp-event } m\ P\ s = 0$
unfolding *wp-event-def wp-def dist-expect-def*
by (*intro sum.neutral*) (*simp add: no-event*)
then show *False*
using *assms* **by** *simp*

```

qed
then show ?thesis
  using that by blast
qed

lemma wp-event-bind-bound-by-head-event:
  fixes m :: ('x, 's) state-monad
  and k :: 'x  $\Rightarrow$  ('y, 's) state-monad
  assumes m-bound: wp-event m P s  $\leq$  C
  and none-imp: Q None  $\Rightarrow$  P None
  and cont-imp:
     $\bigwedge x\ t\ out. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m\ s) \Rightarrow$ 
       $out \in \text{set-dist } (\text{execute } (k\ x)\ t) \Rightarrow$ 
       $Q\ out \Rightarrow P\ (\text{Some } (x, t))$ 
  shows wp-event (m  $\gg$  k) Q s  $\leq$  C
proof -
  have bind-eq:
    wp-event (m  $\gg$  k) Q s =
      wp m ( $\lambda out. \text{case out of}$ 
        None  $\Rightarrow$  if Q None then 1 else 0
        | Some (x, t)  $\Rightarrow$  wp-event (k x) Q t) s
    unfolding wp-event-def by (simp add: wpsimps)
  have bind-head:
    wp-event (m  $\gg$  k) Q s  $\leq$  wp-event m P s
proof -
  have wp-event (m  $\gg$  k) Q s =
    wp m ( $\lambda out. \text{case out of}$ 
      None  $\Rightarrow$  if Q None then 1 else 0
      | Some (x, t)  $\Rightarrow$  wp-event (k x) Q t) s
    by (rule bind-eq)
  also have ...  $\leq$  wp-event m P s
    unfolding wp-event-def
  proof (rule wp-mono-on-support)
    fix out
    assume out: out  $\in$  set-dist (execute m s)
    show
      (case out of None  $\Rightarrow$  if Q None then 1 else 0
        | Some (x, t)  $\Rightarrow$ 
          wp (k x) ( $\lambda out. \text{if } Q\ out \text{ then } 1 \text{ else } 0$ ) t)
         $\leq$  (if P out then 1 else 0)
    proof (cases out)
      case None
      then show ?thesis
        using none-imp by (cases Q None) simp-all
    next
      case (Some xt)
      then obtain x t where xt: xt = (x, t)
        by (cases xt) auto
      have out-eq: out = Some (x, t)

```

```

    using Some xt by simp
  have head-out: Some (x, t) ∈ set-dist (execute m s)
    using out out-eq by simp
  have wp-bound:
    wp (k x) (λout. if Q out then 1 else 0) t ≤
      (if P (Some (x, t)) then 1 else 0 :: prob)
  proof (rule wp-le-const-on-support)
    fix out'
    assume out': out' ∈ set-dist (execute (k x) t)
    show (if Q out' then 1 else 0) ≤
      (if P (Some (x, t)) then 1 else 0 :: prob)
    proof (cases Q out')
      case True
      then have P (Some (x, t))
        by (rule cont-imp[OF head-out out'])
      then show ?thesis
        using True by simp
    next
      case False
      then show ?thesis by simp
    qed
  qed
  show ?thesis
    using wp-bound out-eq by simp
  qed
  qed
  finally show ?thesis .
  qed
show ?thesis
  by (rule order-trans[OF bind-head m-bound])
qed

lemma wp-event-bind-cong-cont:
  fixes m :: ('x, 's) state-monad
  and k :: 'x ⇒ ('y, 's) state-monad
  and l :: 'x ⇒ ('z, 's) state-monad
  assumes none: Q None = R None
  and cont:
    ∧x t. wp-event (k x) Q t = wp-event (l x) R t
  shows wp-event (m ≫ k) Q s = wp-event (m ≫ l) R s
proof -
  have left:
    wp-event (m ≫ k) Q s =
    wp m (λout. case out of
      None ⇒ if Q None then 1 else 0
      | Some (x, t) ⇒ wp-event (k x) Q t) s
  unfolding wp-event-def by (simp add: wpsimps)
  have right:
    wp-event (m ≫ l) R s =

```

```

    wp m (λout. case out of
      None ⇒ if R None then 1 else 0
      | Some (x, t) ⇒ wp-event (l x) R t) s
  unfolding wp-event-def by (simp add: wpsimps)
have fun-eq:
  (λout. case out of
    None ⇒ if Q None then 1 else 0
    | Some (x, t) ⇒ wp-event (k x) Q t) =
  (λout. case out of
    None ⇒ if R None then 1 else 0
    | Some (x, t) ⇒ wp-event (l x) R t)
  by (rule ext) (simp add: none cont split: option.splits prod.splits)
show ?thesis
  unfolding left right
  by (simp add: fun-eq)
qed

```

```

lemma wp-event-bind-bound-by-head-and-cont:
  fixes m :: ('x, 's) state-monad
  and k :: 'x ⇒ ('y, 's) state-monad
  assumes none: ¬ Q None
  and cont-zero:
    λx t. Some (x, t) ∈ set-dist (execute m s) ⇒
      ¬ P (Some (x, t)) ⇒ wp-event (k x) Q t = 0
  and cont-bound:
    λx t. Some (x, t) ∈ set-dist (execute m s) ⇒
      P (Some (x, t)) ⇒ wp-event (k x) Q t ≤ D
  shows wp-event (m ≫ k) Q s ≤ wp-event m P s * D
proof -
  have bind-eq:
    wp-event (m ≫ k) Q s =
    wp m (λout. case out of
      None ⇒ if Q None then 1 else 0
      | Some (x, t) ⇒ wp-event (k x) Q t) s
  unfolding wp-event-def by (simp add: wpsimps)
have wp-event (m ≫ k) Q s ≤
  wp m (λout. (if P out then 1 else 0) * D) s
  unfolding bind-eq
proof (rule wp-mono-on-support)
  fix out
  assume out: out ∈ set-dist (execute m s)
  show (case out of None ⇒ if Q None then 1 else 0
    | Some (x, t) ⇒ wp-event (k x) Q t)
    ≤ (if P out then 1 else 0) * D
  proof (cases out)
    case None
    then show ?thesis
      using none by simp
  next

```

```

case (Some xt)
then obtain x t where xt: xt = (x, t)
  by (cases xt) auto
have head: Some (x, t) ∈ set-dist (execute m s)
  using out Some xt by simp
show ?thesis
proof (cases P (Some (x, t)))
  case True
  then show ?thesis
    using cont-bound[OF head True] Some xt by simp
next
  case False
  then show ?thesis
    using cont-zero[OF head False] Some xt by simp
qed
qed
qed
also have ... = D * wp-event m P s
  unfolding wp-event-def wp-def dist-expect-def
  by (simp add: sum-distrib-left algebra-simps)
also have ... = wp-event m P s * D
  by (simp add: mult.commute)
finally show ?thesis .
qed

```

```

lemma wp-event-bind-bound-by-cont:
  fixes m :: ('x, 's) state-monad
  and k :: 'x ⇒ ('y, 's) state-monad
  assumes none: ¬ Q None
  and cont-bound:
     $\bigwedge x t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$ 
     $\text{wp-event } (k \ x) \ Q \ t \leq C$ 
  shows wp-event (m  $\gg$  k) Q s ≤ C
proof -
  have wp-event (m  $\gg$  k) Q s =
    wp m (λout. case out of
      None ⇒ if Q None then 1 else 0
      | Some (x, t) ⇒ wp-event (k x) Q t) s
  unfolding wp-event-def by (simp add: wpsimps)
  also have ... ≤ C
  proof (rule wp-le-const-on-support)
    fix out
    assume out: out ∈ set-dist (execute m s)
    show (case out of None ⇒ if Q None then 1 else 0
      | Some (x, t) ⇒ wp-event (k x) Q t) ≤ C
    proof (cases out)
      case None
      then show ?thesis
        using none by simp
    end
  end

```

```

next
  case (Some xt)
  then obtain x t where xt: xt = (x, t)
    by (cases xt) auto
  show ?thesis
    using cont-bound[of x t] out Some xt by simp
qed
qed
finally show ?thesis .
qed

lemma sm-bind-return-map:
  ((m >>= (λx. return (f x))) >>= k) = (m >>= (λx. k (f x)))
  by (simp add: sm-bind-assoc)

lemma wp-bind-return-map:
  fixes m :: ('x, 's) state-monad
  shows
    wp (m >>= (λx. return (f x))) Q s =
      wp m (λout. case out of None ⇒ Q None
        | Some (x, t) ⇒ Q (Some (f x, t))) s
proof -
  have cont:
    (λr. case r of None ⇒ Q None
      | Some (x, t) ⇒ wp (return (f x)) Q t) =
      (λout. case out of None ⇒ Q None
        | Some (x, t) ⇒ Q (Some (f x, t)))
  by (rule ext) (simp add: wp-return split: option.splits prod.splits)
  show ?thesis
    by (subst wp-bind) (simp add: cont)
qed

lemma wp-event-bind-return-map:
  fixes m :: ('x, 's) state-monad
  shows
    wp-event (m >>= (λx. return (f x))) P s =
      wp-event m (λout. case out of None ⇒ P None
        | Some (x, t) ⇒ P (Some (f x, t))) s
proof -
  have indicator-eq:
    (λout. case out of None ⇒ if P None then 1 else 0
      | Some (x, t) ⇒ if P (Some (f x, t)) then 1 else 0) =
    (λout. if (case out of None ⇒ P None
      | Some (x, t) ⇒ P (Some (f x, t))) then 1 else 0)
  by (rule ext) (simp split: option.splits prod.splits)
  show ?thesis
    unfolding wp-event-def
    by (simp add: wp-bind-return-map indicator-eq)
qed

```

```

lemma set-dist-bindI:
  fixes  $m :: ('x, 's) \text{state-monad}$ 
    and  $k :: 'x \Rightarrow ('y, 's) \text{state-monad}$ 
  assumes head:  $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s)$ 
    and tail:  $\text{out} \in \text{set-dist } (\text{execute } (k \ x) \ t)$ 
  shows  $\text{out} \in \text{set-dist } (\text{execute } (m \gg k) \ s)$ 
proof -
  let  $?M = \text{dist } (\text{execute } m \ s)$ 
  let  $?K = \lambda r. \text{bind-cont-map } (\lambda x \ t. \text{dist } (\text{execute } (k \ x) \ t)) \ r$ 
  let  $?h = \text{Some } (x, t)$ 
  have head-dom:  $?h \in \text{dom } ?M$ 
    using head unfolding set-dist-def .
  have tail-dom:  $\text{out} \in \text{dom } (?K \ ?h)$ 
    using tail unfolding set-dist-def bind-cont-map-def by simp
  have tail-dist-dom:  $\text{out} \in \text{dom } (\text{dist } (\text{execute } (k \ x) \ t))$ 
    using tail unfolding set-dist-def .
  have head-pos:  $0 < \text{the } (?M \ ?h)$ 
    using dist-nonzero-on-dom[OF head-dom] by simp
  have tail-dist-pos:  $0 < \text{the } (\text{dist } (\text{execute } (k \ x) \ t) \ \text{out})$ 
    using dist-nonzero-on-dom[OF tail-dist-dom] by simp
  obtain  $p$  where p-eq:  $\text{dist } (\text{execute } (k \ x) \ t) \ \text{out} = \text{Some } p$ 
    using tail-dist-dom by auto
  have p-pos:  $0 < p$ 
    using tail-dist-pos p-eq by simp
  have tail-pos:  $0 < \text{option-default } (?K \ ?h \ \text{out})$ 
    using p-eq p-pos by (simp add: bind-cont-map-def)
  have term-pos:
     $0 < \text{the } (?M \ ?h) * \text{option-default } (?K \ ?h \ \text{out})$ 
    using head-pos tail-pos by simp
  have sum-pos:
     $0 < (\sum r \in \text{dom } ?M. \text{the } (?M \ r) * \text{option-default } (?K \ r \ \text{out}))$ 
proof (rule sum-pos2)
  show finite ( $\text{dom } ?M$ )
    by simp
  show  $?h \in \text{dom } ?M$ 
    by (rule head-dom)
  show  $0 < \text{the } (?M \ ?h) * \text{option-default } (?K \ ?h \ \text{out})$ 
    by (rule term-pos)
qed simp
let  $?p = (\sum r \in \text{dom } ?M. \text{the } (?M \ r) * \text{option-default } (?K \ r \ \text{out}))$ 
have p-nonzero:  $?p \neq 0$ 
    using sum-pos by simp
have dist-eq:  $\text{dist } (\text{execute } (m \gg k) \ s) \ \text{out} = \text{Some } ?p$ 
    unfolding sm-bind.rep-eq dist-bind.rep-eq map-bind-def
    by (simp add: Let-def o-def map-fun-def p-nonzero)
then show ?thesis
    unfolding set-dist-def by auto
qed

```


end

end

theory *Soundness-Core-Events*
 imports *Soundness-Core-WP*
begin

Transcript-shape, Merkle binding, and alpha/header event definitions.

context *soundness*
begin

definition *verifier-header-messages*

$:: 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list}$

where

verifier-header-messages fr *f-fri-roots* *f-final* as *dg* *composition-fri-roots* *final* =
 [*fr*] @ *f-fri-roots* @ [*f-final*] @ *as* @ [*dg*] @
 composition-fri-roots @ [*final*]

definition *verifier-header-state*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f$

where

verifier-header-state *s* fr *f-fri-roots* *f-final* as *dg* *composition-fri-roots* *final* =
 foldl concat (*PState* *s*)
 (*verifier-header-messages* fr *f-fri-roots* *f-final* as *dg*
 composition-fri-roots *final*)

definition *verifier-header-transcript*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

verifier-header-transcript *s* fr *f-fri-roots* *f-final* as *dg*
 composition-fri-roots *final* *rest* $\longleftrightarrow$
 PTranscript *s* =
 verifier-header-messages fr *f-fri-roots* *f-final* as *dg*
 composition-fri-roots *final* @ *rest* $\wedge$
 length *f-fri-roots* = ceil-log clength $\wedge$
 length *as* = length *spec* $\wedge$
 length *composition-fri-roots* = ceil-log (to-nat *dg* + 1)

definition *merkle-root-binds-table*

$:: 'f \Rightarrow 'f \text{ list} \Rightarrow ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

merkle-root-binds-table *rt* *table* *s* $\longleftrightarrow$
 ($\exists \text{ tree} :: 'f \text{ tree. created-tree table tree } s \wedge \text{rt} = \text{value tree}$)

definition *merkle-root-binding-collision*
 $:: 'f \Rightarrow ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{merkle-root-binding-collision } rt \ s \longleftrightarrow$
 $(\exists \text{ table table'}. \text{merkle-root-binds-table } rt \ \text{table } s \wedge$
 $\text{merkle-root-binds-table } rt \ \text{table'} \ s \wedge$
 $\text{table} \neq \text{table'})$

definition *hash-map-output-collision*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{hash-map-output-collision } s \longleftrightarrow$
 $(\exists (x :: 'f \text{ protocol-hash-input}) (y :: 'f \text{ protocol-hash-input}) \ z. \text{fmlookup } (\text{HashMap } s) \ x = \text{Some } z \wedge$
 $x \neq y \wedge \text{fmlookup } (\text{HashMap } s) \ y = \text{Some } z)$

definition *hash-map-output-values*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ set}$
where
 $\text{hash-map-output-values } s =$
 $\{z. \exists x :: 'f \text{ protocol-hash-input}. \text{fmlookup } (\text{HashMap } s) \ x = \text{Some } z\}$

lemma *hash-map-output-values-verifier-initial-state[simp]:*
 $\text{hash-map-output-values } (\text{verifier-initial-state } tr) = \{\}$
unfolding *hash-map-output-values-def* **by** *simp*

lemma *merkle-root-binding-collisionI:*
assumes *merkle-root-binds-table* $rt \ \text{table } s$
and *merkle-root-binds-table* $rt \ \text{table'} \ s$
and $\text{table} \neq \text{table'}$
shows *merkle-root-binding-collision* $rt \ s$
using *assms* **unfolding** *merkle-root-binding-collision-def* **by** *blast*

lemma *merkle-root-binds-table-unique-if-no-collision:*
assumes $\neg \text{merkle-root-binding-collision } rt \ s$
and *merkle-root-binds-table* $rt \ \text{table } s$
and *merkle-root-binds-table* $rt \ \text{table'} \ s$
shows $\text{table} = \text{table'}$
using *assms* **unfolding** *merkle-root-binding-collision-def* **by** *blast*

lemma *merkle-root-binds-table-mono:*
assumes *bind:* *merkle-root-binds-table* $rt \ \text{table } s$
and *ext:* $s \leq t$
shows *merkle-root-binds-table* $rt \ \text{table } t$
proof –
from *bind* **obtain** *tree* **where**

```

    created: created-tree table tree s
  and root: rt = value tree
  unfolding merkle-root-binds-table-def by blast
have created-tree table tree t
  by (rule created-tree-mono[OF created ext])
then show ?thesis
  unfolding merkle-root-binds-table-def
  using root by blast
qed

definition hash-maps-compatible
:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool
where
  hash-maps-compatible s t  $\longleftrightarrow$ 
    ( $\forall x y z.$ 
      fmlookup (HashMap s) x = Some y  $\longrightarrow$ 
      fmlookup (HashMap t) x = Some z  $\longrightarrow$ 
      y = z)

definition hash-map-value-conflict
:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool
where
  hash-map-value-conflict s t  $\longleftrightarrow$ 
    ( $\exists x y z.$ 
      fmlookup (HashMap s) x = Some y  $\wedge$ 
      fmlookup (HashMap t) x = Some z  $\wedge$ 
      y  $\neq$  z)

lemma hash-maps-compatible-iff-no-value-conflict:
  hash-maps-compatible s t  $\longleftrightarrow$   $\neg$  hash-map-value-conflict s t
  unfolding hash-maps-compatible-def hash-map-value-conflict-def by blast

lemma hash-map-value-conflict-sym:
  assumes hash-map-value-conflict s t
  shows hash-map-value-conflict t s
  using assms unfolding hash-map-value-conflict-def by blast

lemma hash-maps-compatible-sym:
  assumes hash-maps-compatible s t
  shows hash-maps-compatible t s
  using assms
  unfolding hash-maps-compatible-iff-no-value-conflict
  using hash-map-value-conflict-sym by blast

lemma hash-map-value-conflict-no-common-extension:
  assumes conflict: hash-map-value-conflict s t
  and s-ext: s  $\leq$  u

```

```

    and t-ext:  $t \leq u$ 
  shows False
proof -
  from conflict obtain x y z where
    s-lookup: fmlookup (HashMap s) x = Some y
    and t-lookup: fmlookup (HashMap t) x = Some z
    and neg:  $y \neq z$ 
    unfolding hash-map-value-conflict-def by blast
  have u-y: fmlookup (HashMap u) x = Some y
    by (rule hash-extension-lookup[OF s-lookup s-ext])
  have u-z: fmlookup (HashMap u) x = Some z
    by (rule hash-extension-lookup[OF t-lookup t-ext])
  show False
    using u-y u-z neg by simp
qed

lemma common-extension-imp-no-hash-map-value-conflict:
  assumes s-ext:  $s \leq u$ 
    and t-ext:  $t \leq u$ 
  shows  $\neg$  hash-map-value-conflict s t
  using hash-map-value-conflict-no-common-extension[OF s-ext t-ext] by blast

lemma hash-map-value-conflict-blocks-clean-merge-obligation:
  assumes hash-map-value-conflict s t
  shows  $\neg$  (hash-maps-compatible s t  $\wedge$ 
     $\neg$  hash-map-output-collision (hash-state-merge s t))
  using assms unfolding hash-maps-compatible-iff-no-value-conflict by simp

fun merkle-hash-key :: 'f protocol-hash-input  $\Rightarrow$  bool
where
  merkle-hash-key (MerkleLeaf _)  $\longleftrightarrow$  True
| merkle-hash-key (MerkleNode _ _)  $\longleftrightarrow$  True
| merkle-hash-key (FiatShamirChallenge _)  $\longleftrightarrow$  False
| merkle-hash-key (TraceFriChallenge _ _)  $\longleftrightarrow$  False
| merkle-hash-key (CompositionFriChallenge _ _)  $\longleftrightarrow$  False
| merkle-hash-key (AlphaChallenge _ _)  $\longleftrightarrow$  False
| merkle-hash-key (QueryIndexChallenge _ _)  $\longleftrightarrow$  False

definition merkle-hash-maps-compatible
:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool
where
  merkle-hash-maps-compatible s t  $\longleftrightarrow$ 
    ( $\forall$  x y z.
      merkle-hash-key x  $\longrightarrow$ 
      fmlookup (HashMap s) x = Some y  $\longrightarrow$ 
      fmlookup (HashMap t) x = Some z  $\longrightarrow$ 
      y = z)

```

definition *merkle-hash-value-conflict*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{merkle-hash-value-conflict } s \ t \longleftrightarrow$
 $(\exists x \ y \ z.$
 $\text{merkle-hash-key } x \wedge$
 $\text{fmlookup } (\text{HashMap } s) \ x = \text{Some } y \wedge$
 $\text{fmlookup } (\text{HashMap } t) \ x = \text{Some } z \wedge$
 $y \neq z)$

lemma *merkle-hash-maps-compatible-iff-no-value-conflict*:

$\text{merkle-hash-maps-compatible } s \ t \longleftrightarrow$
 $\neg \text{merkle-hash-value-conflict } s \ t$

unfolding *merkle-hash-maps-compatible-def merkle-hash-value-conflict-def*
by *blast*

lemma *merkle-hash-value-conflict-sym*:

assumes *merkle-hash-value-conflict* $s \ t$

shows *merkle-hash-value-conflict* $t \ s$

using *assms* **unfolding** *merkle-hash-value-conflict-def* **by** *blast*

lemma *merkle-hash-maps-compatible-sym*:

assumes *merkle-hash-maps-compatible* $s \ t$

shows *merkle-hash-maps-compatible* $t \ s$

using *assms* **unfolding** *merkle-hash-maps-compatible-def* **by** *blast*

definition *merkle-hash-extends*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{merkle-hash-extends } s \ t \longleftrightarrow$
 $(\forall x \ y.$
 $\text{merkle-hash-key } x \longrightarrow$
 $\text{fmlookup } (\text{HashMap } s) \ x = \text{Some } y \longrightarrow$
 $\text{fmlookup } (\text{HashMap } t) \ x = \text{Some } y)$

lemma *merkle-hash-value-conflict-no-common-extension*:

assumes *conflict*: *merkle-hash-value-conflict* $s \ t$

and *ext-s*: *merkle-hash-extends* $s \ u$

and *ext-t*: *merkle-hash-extends* $t \ u$

shows *False*

proof –

from *conflict* **obtain** $x \ y \ z$ **where**

merkle: *merkle-hash-key* x

and *s-lookup*: *fmlookup* $(\text{HashMap } s) \ x = \text{Some } y$

and *t-lookup*: *fmlookup* $(\text{HashMap } t) \ x = \text{Some } z$

and *neg*: $y \neq z$

unfolding *merkle-hash-value-conflict-def* **by** *blast*

```

have u-y: fmlookup (HashMap u) x = Some y
  using ext-s merkle s-lookup unfolding merkle-hash-extends-def by blast
have u-z: fmlookup (HashMap u) x = Some z
  using ext-t merkle t-lookup unfolding merkle-hash-extends-def by blast
show False
  using u-y u-z neq by simp
qed

```

```

lemma common-merkle-extension-imp-no-merkle-hash-value-conflict:
  assumes ext-s: merkle-hash-extends s u
  and ext-t: merkle-hash-extends t u
  shows  $\neg$  merkle-hash-value-conflict s t
  using merkle-hash-value-conflict-no-common-extension[OF ext-s ext-t]
  by blast

```

```

lemma merkle-hash-maps-compatible-if-common-merkle-extension:
  assumes ext-s: merkle-hash-extends s u
  and ext-t: merkle-hash-extends t u
  shows merkle-hash-maps-compatible s t
  using common-merkle-extension-imp-no-merkle-hash-value-conflict
    [OF ext-s ext-t]
  unfolding merkle-hash-maps-compatible-iff-no-value-conflict .

```

```

definition merkle-hash-state-merge
:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  ('f, 'a) protocol-channel-scheme
where
  merkle-hash-state-merge s t =
    s(HashMap :=
      fmlfilter merkle-hash-key (HashMap s) ++f
      fmlfilter merkle-hash-key (HashMap t))

```

```

lemma merkle-hash-state-merge-extends-right:
  merkle-hash-extends t (merkle-hash-state-merge s t)
  unfolding merkle-hash-extends-def merkle-hash-state-merge-def
  by (simp add: fmllookup-dom-iff)

```

```

lemma merkle-hash-state-merge-extends-left-if-compatible:
  assumes compatible: merkle-hash-maps-compatible s t
  shows merkle-hash-extends s (merkle-hash-state-merge s t)
  unfolding merkle-hash-extends-def
proof (intro allI impI)
  fix x y
  assume merkle: merkle-hash-key x
  and s-lookup: fmlookup (HashMap s) x = Some y
  show fmlookup (HashMap (merkle-hash-state-merge s t)) x = Some y
proof (cases fmlookup (HashMap t) x)
  case None

```

```

then show ?thesis
  using merkle s-lookup
  unfolding merkle-hash-state-merge-def
  by (simp add: fmlookup-dom-iff)
next
case (Some z)
have z = y
  using compatible merkle s-lookup Some
  unfolding merkle-hash-maps-compatible-def by blast
then show ?thesis
  using merkle Some
  unfolding merkle-hash-state-merge-def
  by (simp add: fmlookup-dom-iff)
qed
qed

lemma merkle-hash-state-merge-lookupD:
  assumes lookup:
    fmlookup (HashMap (merkle-hash-state-merge s t)) x = Some z
  obtains
    merkle-hash-key x fmlookup (HashMap s) x = Some z
  | merkle-hash-key x fmlookup (HashMap t) x = Some z
proof -
  have merkle: merkle-hash-key x
    using lookup
    unfolding merkle-hash-state-merge-def
    by (cases merkle-hash-key x) auto
  show ?thesis
proof (cases fmlookup (HashMap t) x)
  case (Some z')
  then have z-eq: z' = z
    using lookup merkle
    unfolding merkle-hash-state-merge-def
    by (simp add: fmlookup-dom-iff)
  show ?thesis
    by (rule that(2)[OF merkle Some[unfolded z-eq]])
next
case None
have s-lookup: fmlookup (HashMap s) x = Some z
  using lookup merkle None
  unfolding merkle-hash-state-merge-def
  by (simp add: fmlookup-dom-iff)
show ?thesis
  by (rule that(1)[OF merkle s-lookup])
qed
qed

lemma merkle-hash-state-merge-clean-if-common-clean-merkle-extension:
  assumes ext-s: merkle-hash-extends s u

```

```

    and ext-t: merkle-hash-extends t u
    and clean:  $\neg$  hash-map-output-collision u
  shows  $\neg$  hash-map-output-collision (merkle-hash-state-merge s t)
proof
  assume collision: hash-map-output-collision (merkle-hash-state-merge s t)
  then obtain x y z where neg:  $x \neq y$ 
  and x-lookup:
    fmllookup (HashMap (merkle-hash-state-merge s t)) x = Some z
  and y-lookup:
    fmllookup (HashMap (merkle-hash-state-merge s t)) y = Some z
  unfolding hash-map-output-collision-def by blast
  have x-lookup-u: fmllookup (HashMap u) x = Some z
  proof -
    from x-lookup show ?thesis
  proof (rule merkle-hash-state-merge-lookupD)
    assume merkle: merkle-hash-key x
    and lookup-s: fmllookup (HashMap s) x = Some z
    show ?thesis
    using ext-s merkle lookup-s
    unfolding merkle-hash-extends-def by blast
  next
    assume merkle: merkle-hash-key x
    and lookup-t: fmllookup (HashMap t) x = Some z
    show ?thesis
    using ext-t merkle lookup-t
    unfolding merkle-hash-extends-def by blast
  qed
  qed
  have y-lookup-u: fmllookup (HashMap u) y = Some z
  proof -
    from y-lookup show ?thesis
  proof (rule merkle-hash-state-merge-lookupD)
    assume merkle: merkle-hash-key y
    and lookup-s: fmllookup (HashMap s) y = Some z
    show ?thesis
    using ext-s merkle lookup-s
    unfolding merkle-hash-extends-def by blast
  next
    assume merkle: merkle-hash-key y
    and lookup-t: fmllookup (HashMap t) y = Some z
    show ?thesis
    using ext-t merkle lookup-t
    unfolding merkle-hash-extends-def by blast
  qed
  qed
  have hash-map-output-collision u
  unfolding hash-map-output-collision-def
  using neg x-lookup-u y-lookup-u by blast
  then show False

```


using *clean* by *contradiction*
qed

lemma *common-clean-merkle-extension-imp-pairwise-merkle-clean*:
assumes *ext-s*: *merkle-hash-extends s u*
and *ext-t*: *merkle-hash-extends t u*
and *clean*: $\neg$ *hash-map-output-collision u*
shows
 $\neg$ *merkle-hash-value-conflict s t* $\wedge$
 $\neg$ *hash-map-output-collision (merkle-hash-state-merge s t)*
using *common-merkle-extension-imp-no-merkle-hash-value-conflict*
[*OF ext-s ext-t*]
merkle-hash-state-merge-clean-if-common-clean-merkle-extension
[*OF ext-s ext-t clean*]
by *simp*

definition *merkle-hash-cross-output-collision*
 $:: (f, 'a)$ *protocol-channel-scheme* $\Rightarrow$
 $(f, 'a)$ *protocol-channel-scheme* $\Rightarrow$ *bool*
where
merkle-hash-cross-output-collision s t $\longleftrightarrow$
 $(\exists x y z.$
merkle-hash-key x $\wedge$ *merkle-hash-key y* $\wedge$ $x \neq y \wedge$
 $((\text{fmlookup } (\text{HashMap } s) x = \text{Some } z \wedge$
 $\text{fmlookup } (\text{HashMap } t) y = \text{Some } z) \vee$
 $(\text{fmlookup } (\text{HashMap } t) x = \text{Some } z \wedge$
 $\text{fmlookup } (\text{HashMap } s) y = \text{Some } z)))$

definition *merkle-hash-pairwise-coupling-bad*
 $:: (f, 'a)$ *protocol-channel-scheme* $\Rightarrow$
 $(f, 'a)$ *protocol-channel-scheme* $\Rightarrow$ *bool*
where
merkle-hash-pairwise-coupling-bad s t $\longleftrightarrow$
merkle-hash-value-conflict s t $\vee$
merkle-hash-cross-output-collision s t

lemma *merkle-hash-cross-output-collision-sym*:
assumes *merkle-hash-cross-output-collision s t*
shows *merkle-hash-cross-output-collision t s*
using *assms* **unfolding** *merkle-hash-cross-output-collision-def* **by** *blast*

lemma *merkle-hash-pairwise-coupling-bad-sym*:
assumes *merkle-hash-pairwise-coupling-bad s t*
shows *merkle-hash-pairwise-coupling-bad t s*
using *assms*
unfolding *merkle-hash-pairwise-coupling-bad-def*
using *merkle-hash-value-conflict-sym*
merkle-hash-cross-output-collision-sym **by** *blast*

```

lemma merkle-hash-state-merge-collision-imp-local-or-cross:
  assumes collision:
    hash-map-output-collision (merkle-hash-state-merge s t)
  shows
    hash-map-output-collision s  $\vee$ 
    hash-map-output-collision t  $\vee$ 
    merkle-hash-cross-output-collision s t
proof -
  from collision obtain x y z where neq: x  $\neq$  y
  and x-lookup:
    fmlookup (HashMap (merkle-hash-state-merge s t)) x = Some z
  and y-lookup:
    fmlookup (HashMap (merkle-hash-state-merge s t)) y = Some z
  unfolding hash-map-output-collision-def by blast
from x-lookup consider
    (xs) merkle-hash-key x fmlookup (HashMap s) x = Some z
  | (xt) merkle-hash-key x fmlookup (HashMap t) x = Some z
  by (rule merkle-hash-state-merge-lookupD)
then show ?thesis
proof cases
  case xs
  from y-lookup consider
    (ys) merkle-hash-key y fmlookup (HashMap s) y = Some z
  | (yt) merkle-hash-key y fmlookup (HashMap t) y = Some z
  by (rule merkle-hash-state-merge-lookupD)
then show ?thesis
proof cases
  case ys
  have hash-map-output-collision s
    unfolding hash-map-output-collision-def
    using neq xs ys by blast
  then show ?thesis by blast
next
  case yt
  have merkle-hash-cross-output-collision s t
    unfolding merkle-hash-cross-output-collision-def
    using neq xs yt by blast
  then show ?thesis by blast
qed
next
  case xt
  from y-lookup consider
    (ys) merkle-hash-key y fmlookup (HashMap s) y = Some z
  | (yt) merkle-hash-key y fmlookup (HashMap t) y = Some z
  by (rule merkle-hash-state-merge-lookupD)
then show ?thesis
proof cases
  case ys
  have merkle-hash-cross-output-collision s t

```

```

    unfolding merkle-hash-cross-output-collision-def
    using neq xt ys by blast
  then show ?thesis by blast
next
case yt
have hash-map-output-collision t
  unfolding hash-map-output-collision-def
  using neq xt yt by blast
then show ?thesis by blast
qed
qed
qed

lemma merkle-hash-pairwise-bad-imp-local-collision-or-coupling-bad:
  assumes pair-bad:
    merkle-hash-value-conflict s t  $\vee$ 
    hash-map-output-collision (merkle-hash-state-merge s t)
  shows
    hash-map-output-collision s  $\vee$ 
    hash-map-output-collision t  $\vee$ 
    merkle-hash-pairwise-coupling-bad s t
proof (cases merkle-hash-value-conflict s t)
case True
  then show ?thesis
    unfolding merkle-hash-pairwise-coupling-bad-def by blast
next
case False
  then have merge-collision:
    hash-map-output-collision (merkle-hash-state-merge s t)
  using pair-bad by blast
from merkle-hash-state-merge-collision-imp-local-or-cross
  [OF merge-collision]
show ?thesis
  unfolding merkle-hash-pairwise-coupling-bad-def by blast
qed

definition hash-state-merge
:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  ('f, 'a) protocol-channel-scheme
where
  hash-state-merge s t =
    s( $\llbracket$ HashMap := HashMap s ++f HashMap t $\rrbracket$ )

lemma hash-state-merge-extends-right:
  t  $\leq$  hash-state-merge s t
  unfolding hash-state-merge-def less-eq-hash-ext-def less-eq-fmap-def
  by auto

```

```

lemma hash-state-merge-extends-left-if-compatible:
  assumes compatible: hash-maps-compatible s t
  shows  $s \leq \text{hash-state-merge } s \ t$ 
proof –
  have lookup:
     $\bigwedge x \ y. \text{fmlookup } (\text{HashMap } s) \ x = \text{Some } y \implies$ 
     $\text{fmlookup } (\text{HashMap } (\text{hash-state-merge } s \ t)) \ x = \text{Some } y$ 
  proof –
    fix  $x \ y$ 
    assume s-lookup:  $\text{fmlookup } (\text{HashMap } s) \ x = \text{Some } y$ 
    show  $\text{fmlookup } (\text{HashMap } (\text{hash-state-merge } s \ t)) \ x = \text{Some } y$ 
    proof (cases  $x \in \text{fmdom } (\text{HashMap } t)$ )
      case True
      then obtain  $z$  where t-lookup:  $\text{fmlookup } (\text{HashMap } t) \ x = \text{Some } z$ 
      unfolding fmlookup-dom-iff by blast
      have  $z = y$ 
      using compatible s-lookup t-lookup
      unfolding hash-maps-compatible-def by blast
      then show ?thesis
      using True t-lookup
      unfolding hash-state-merge-def by simp
    next
      case False
      then show ?thesis
      using s-lookup unfolding hash-state-merge-def by simp
    qed
  qed
show ?thesis
  unfolding less-eq-hash-ext-def less-eq-fmap-def
proof
  fix  $x$ 
  show  $\text{fmlookup } (\text{HashMap } s) \ x = \text{None} \vee$ 
   $\text{fmlookup } (\text{HashMap } s) \ x =$ 
   $\text{fmlookup } (\text{HashMap } (\text{hash-state-merge } s \ t)) \ x$ 
  proof (cases  $\text{fmlookup } (\text{HashMap } s) \ x$ )
    case None
    then show ?thesis by simp
  next
    case (Some  $y$ )
    then have  $\text{fmlookup } (\text{HashMap } (\text{hash-state-merge } s \ t)) \ x = \text{Some } y$ 
    by (rule lookup)
    then show ?thesis
    using Some by simp
  qed
qed
qed

```

```

lemma hash-maps-compatible-if-common-extension:
  assumes s-ext:  $s \leq u$ 

```

```

    and t-ext:  $t \leq u$ 
  shows hash-maps-compatible s t
  unfolding hash-maps-compatible-def
proof (intro allI impI)
  fix x y z
  assume s-lookup: fmlookup (HashMap s) x = Some y
    and t-lookup: fmlookup (HashMap t) x = Some z
  have u-y: fmlookup (HashMap u) x = Some y
    by (rule hash-extension-lookup[OF s-lookup s-ext])
  have u-z: fmlookup (HashMap u) x = Some z
    by (rule hash-extension-lookup[OF t-lookup t-ext])
  show y = z
    using u-y u-z by simp
qed

lemma hash-state-merge-le-common-extension:
  assumes s-ext:  $s \leq u$ 
    and t-ext:  $t \leq u$ 
  shows hash-state-merge s t  $\leq u$ 
  unfolding less-eq-hash-ext-def less-eq-fmap-def
proof
  fix x
  show fmlookup (HashMap (hash-state-merge s t)) x = None  $\vee$ 
    fmlookup (HashMap (hash-state-merge s t)) x = fmlookup (HashMap u) x
proof (cases x | $\in$ | fndom (HashMap t))
  case True
  then obtain z where t-lookup: fmlookup (HashMap t) x = Some z
    unfolding fmlookup-dom-iff by blast
  have merge-lookup:
    fmlookup (HashMap (hash-state-merge s t)) x = Some z
    using True t-lookup unfolding hash-state-merge-def by simp
  have u-lookup: fmlookup (HashMap u) x = Some z
    by (rule hash-extension-lookup[OF t-lookup t-ext])
  show ?thesis
    using merge-lookup u-lookup by simp
next
  case False
  show ?thesis
proof (cases fmlookup (HashMap s) x)
  case None
  then have fmlookup (HashMap (hash-state-merge s t)) x = None
    using False unfolding hash-state-merge-def by simp
  then show ?thesis
    by simp
next
  case (Some y)
  then have merge-lookup:
    fmlookup (HashMap (hash-state-merge s t)) x = Some y
    using False unfolding hash-state-merge-def by simp

```

```

    have u-lookup: fmlookup (HashMap u) x = Some y
      by (rule hash-extension-lookup[OF Some s-ext])
    show ?thesis
      using merge-lookup u-lookup by simp
  qed
qed
qed

lemma hash-state-merge-clean-if-common-clean-extension:
  assumes s-ext:  $s \leq u$ 
    and t-ext:  $t \leq u$ 
    and clean:  $\neg \text{hash-map-output-collision } u$ 
  shows  $\neg \text{hash-map-output-collision } (\text{hash-state-merge } s \ t)$ 
proof
  assume collision: hash-map-output-collision (hash-state-merge s t)
  have merge-ext: hash-state-merge s t  $\leq u$ 
    by (rule hash-state-merge-le-common-extension[OF s-ext t-ext])
  from collision obtain x y z where neg:  $x \neq y$ 
    and x-lookup:
      fmlookup (HashMap (hash-state-merge s t)) x = Some z
    and y-lookup:
      fmlookup (HashMap (hash-state-merge s t)) y = Some z
  unfolding hash-map-output-collision-def by blast
  have x-lookup-u: fmlookup (HashMap u) x = Some z
    by (rule hash-extension-lookup[OF x-lookup merge-ext])
  have y-lookup-u: fmlookup (HashMap u) y = Some z
    by (rule hash-extension-lookup[OF y-lookup merge-ext])
  have hash-map-output-collision u
    unfolding hash-map-output-collision-def
    using neg x-lookup-u y-lookup-u by blast
  then show False
    using clean by contradiction
qed

lemma hash-map-output-collisionI:
  fixes x y :: 'f protocol-hash-input
  assumes  $x \neq y$ 
    and fmlookup (HashMap s) x = Some z
    and fmlookup (HashMap s) y = Some z
  shows hash-map-output-collision s
  using assms unfolding hash-map-output-collision-def by blast

lemma hash-map-output-collision-merkle-leafI:
  assumes  $x \neq y$ 
    and fmlookup (HashMap s) (MerkleLeaf x) = Some z
    and fmlookup (HashMap s) (MerkleLeaf y) = Some z
  shows hash-map-output-collision s
  by (rule hash-map-output-collisionI[OF - assms(2,3)])
    (use assms(1) in simp)

```

lemma *hash-map-output-collision-merkle-nodeI*:
assumes $(x1, x2) \neq (y1, y2)$
and $\text{fmlookup } (\text{HashMap } s) (\text{MerkleNode } x1 \ x2) = \text{Some } z$
and $\text{fmlookup } (\text{HashMap } s) (\text{MerkleNode } y1 \ y2) = \text{Some } z$
shows *hash-map-output-collision s*
by (*rule hash-map-output-collisionI* [*OF* - *assms*(2,3)])
(use assms(1) in auto)

lemma *hash-map-output-collision-merkle-leaf-nodeI*:
assumes $\text{fmlookup } (\text{HashMap } s) (\text{MerkleLeaf } x) = \text{Some } z$
and $\text{fmlookup } (\text{HashMap } s) (\text{MerkleNode } y1 \ y2) = \text{Some } z$
shows *hash-map-output-collision s*
by (*rule hash-map-output-collisionI* [*OF* - *assms*])
simp

lemma *hash-map-output-valuesI*:
fixes $x :: 'f \text{ protocol-hash-input}$
assumes $\text{fmlookup } (\text{HashMap } s) \ x = \text{Some } z$
shows $z \in \text{hash-map-output-values } s$
using *assms* **unfolding** *hash-map-output-values-def* **by** *blast*

lemma *hash-map-output-values-alt-def*:
 $\text{hash-map-output-values } s =$
 $(\lambda x. \text{the } (\text{fmlookup } (\text{HashMap } s) \ x)) \text{ 'fmdom' } (\text{HashMap } s)$
unfolding *hash-map-output-values-def*

proof
show $\{z. \exists x. \text{fmlookup } (\text{HashMap } s) \ x = \text{Some } z\}$
 $\subseteq (\lambda x. \text{the } (\text{fmlookup } (\text{HashMap } s) \ x)) \text{ 'fmdom' } (\text{HashMap } s)$
proof
fix z
assume $z \in \{z. \exists x. \text{fmlookup } (\text{HashMap } s) \ x = \text{Some } z\}$
then obtain x **where** $\text{lookup: fmlookup } (\text{HashMap } s) \ x = \text{Some } z$
by *blast*
have $x \in \text{fmdom' } (\text{HashMap } s)$
using *lookup* **by** (*simp add: fmlookup-dom'-iff*)
moreover have $\text{the } (\text{fmlookup } (\text{HashMap } s) \ x) = z$
using *lookup* **by** *simp*
ultimately show $z \in$
 $(\lambda x. \text{the } (\text{fmlookup } (\text{HashMap } s) \ x)) \text{ 'fmdom' } (\text{HashMap } s)$
by *blast*
qed
show $(\lambda x. \text{the } (\text{fmlookup } (\text{HashMap } s) \ x)) \text{ 'fmdom' } (\text{HashMap } s)$
 $\subseteq \{z. \exists x. \text{fmlookup } (\text{HashMap } s) \ x = \text{Some } z\}$
by (*auto simp: fmlookup-dom'-iff*)
qed

lemma *finite-hash-map-output-values[simp]*:
 $\text{finite } (\text{hash-map-output-values } s)$

unfolding *hash-map-output-values-alt-def* **by** *simp*

lemma *card-hash-map-output-values-le-fmdom*:
 $\text{card } (\text{hash-map-output-values } s) \leq \text{card } (\text{fmdom}' (\text{HashMap } s))$
unfolding *hash-map-output-values-alt-def*
by (*rule card-image-le*) *simp*

lemma *hash-map-output-values-after-update-subset*:
fixes $x :: 'f \text{ protocol-hash-input}$
shows
 $\text{hash-map-output-values } (s \langle \text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s) \rangle) \subseteq$
 $\text{insert } y \ (\text{hash-map-output-values } s)$
unfolding *hash-map-output-values-def*
by *auto*

lemma *card-hash-map-output-values-after-update-le*:
fixes $x :: 'f \text{ protocol-hash-input}$
shows
 $\text{card } (\text{hash-map-output-values } (s \langle \text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s) \rangle)) \leq$
 $\text{Suc } (\text{card } (\text{hash-map-output-values } s))$
proof –
have $\text{card } (\text{hash-map-output-values } (s \langle \text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s) \rangle)) \leq$
 $\text{card } (\text{insert } y \ (\text{hash-map-output-values } s))$
by (*rule card-mono*)
 $(\text{simp-all add: hash-map-output-values-after-update-subset})$
also have $\dots \leq \text{Suc } (\text{card } (\text{hash-map-output-values } s))$
by (*simp add: card-insert-if*)
finally show *?thesis* .
qed

lemma *hash-map-output-collision-after-fresh-updateD*:
fixes $x :: 'f \text{ protocol-hash-input}$
assumes *clean*: $\neg \text{hash-map-output-collision } s$
and *fresh*: $\text{fmlookup } (\text{HashMap } s) \ x = \text{None}$
and *collision*:
 $\text{hash-map-output-collision } (s \langle \text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s) \rangle)$
shows $y \in \text{hash-map-output-values } s$
proof –
obtain $a \ b \ z$ **where** *neq*: $a \neq b$
and *a-lookup*:
 $\text{fmlookup } (\text{fmupd } x \ y \ (\text{HashMap } s)) \ a = \text{Some } z$
and *b-lookup*:
 $\text{fmlookup } (\text{fmupd } x \ y \ (\text{HashMap } s)) \ b = \text{Some } z$
using *collision* **unfolding** *hash-map-output-collision-def* **by** *auto*
show *?thesis*


```

proof (cases a = x)
  case True
  then have z-y: z = y
    using a-lookup by simp
  have b-ne-x: b ≠ x
    using neq True by simp
  have fmlookup (HashMap s) b = Some y
    using b-lookup b-ne-x z-y by simp
  then show ?thesis
    by (rule hash-map-output-valuesI)
next
  case a-ne-x: False
  show ?thesis
  proof (cases b = x)
    case True
    then have z-y: z = y
      using b-lookup by simp
    have fmlookup (HashMap s) a = Some y
      using a-lookup a-ne-x z-y by simp
    then show ?thesis
      by (rule hash-map-output-valuesI)
    next
    case b-ne-x: False
    have a-old: fmlookup (HashMap s) a = Some z
      using a-lookup a-ne-x by simp
    have b-old: fmlookup (HashMap s) b = Some z
      using b-lookup b-ne-x by simp
    have hash-map-output-collision s
      by (rule hash-map-output-collisionI[OF neq a-old b-old])
    then show ?thesis
      using clean by contradiction
  qed
qed
qed

```

lemma no-hash-map-output-collision-after-known-update:

```

fixes x :: 'f protocol-hash-input
assumes clean: ¬ hash-map-output-collision s
  and known: fmlookup (HashMap s) x = Some y
shows ¬ hash-map-output-collision
  (s(HashMap := fmupd x y (HashMap s)))

```

```

proof
assume collision:
  hash-map-output-collision
  (s(HashMap := fmupd x y (HashMap s)))
obtain a b z where neq: a ≠ b
and a-lookup:
  fmlookup (fmupd x y (HashMap s)) a = Some z
and b-lookup:

```

```

    fmlookup (fmupd x y (HashMap s)) b = Some z
  using collision unfolding hash-map-output-collision-def by auto
have a-old: fmlookup (HashMap s) a = Some z
  using a-lookup known by (cases a = x) simp-all
have b-old: fmlookup (HashMap s) b = Some z
  using b-lookup known by (cases b = x) simp-all
have hash-map-output-collision s
  by (rule hash-map-output-collisionI[OF neq a-old b-old])
then show False
  using clean by contradiction
qed

lemma dist-expect-uniform-fUNIV-indicator:
  fixes S :: 'f set
  shows
    dist-expect (dist-uniform (fUNIV :: 'f fset))
      (λx. if x ∈ S then 1 else 0) =
      nnreal (card S) / nnreal size
proof -
  let ?U = (fUNIV :: 'f fset)
  let ?w = 1 / nnreal (CARD('f))
  let ?B = ((λx :: 'f. (x, ?w)) |` ?U)

  have U-nonempty: ?U ≠ {}
  proof -
    have (undefined :: 'f) |∈| ?U
      by simp
    then show ?thesis
      by auto
  qed
  have fcard-U: fcard ?U = CARD('f)
    by (simp add: fcard.rep-eq)
  have total-nonzero:
    fsum snd ((λx :: 'f. (x, 1::prob)) |` ?U) ≠ 0
    using fsum-snd-uniform-ones[of ?U] fcard-U by simp
  have dist-eq:
    dist (dist-uniform ?U) = map-of ?B
    unfolding dist-uniform-def dist-of-fset.rep-eq map-of-fset-def
    using norm-fset-uniform-ones[OF U-nonempty] total-nonzero fcard-U
    by (simp add: Let-def)

  have lookup:
    dist (dist-uniform ?U) x = Some ?w for x :: 'f
  proof -
    have mem: (x, ?w) |∈| ?B
      by simp
    have uniq:
      ∧y z. (x, y) |∈| ?B ⇒ (x, z) |∈| ?B ⇒ y = z
      by auto
  qed

```

```

have val: map-of-val ?B x = ?w
  by (rule map-of-val-unique-key[OF uniq mem])
have x |∈| fimage fst ?B
  using mem by force
then have map-of ?B x = Some (map-of-val ?B x)
  unfolding map-of-def by simp
also have ... = Some ?w
  using val by simp
finally show ?thesis
  using dist-eq by simp
qed

have dom-eq: dom (dist (dist-uniform ?U)) = (UNIV :: 'f set)
  using lookup by auto

have dist-expect (dist-uniform ?U) (λx. if x ∈ S then 1 else 0) =
  (∑ x∈UNIV. ?w * (if x ∈ S then 1 else 0))
  unfolding dist-expect-def dom-eq
  by (intro sum.cong refl) (simp add: lookup)
also have ... = (∑ x∈UNIV. if x ∈ S then ?w else 0)
  by (intro sum.cong refl) (simp split: if-splits)
also have ... = (∑ x∈S. ?w)
proof -
  have (∑ x∈UNIV. if x ∈ S then ?w else 0) =
    (∑ x∈UNIV ∩ S. ?w)
    by (subst sum.inter-restrict[symmetric]) simp-all
  also have ... = (∑ x∈S. ?w)
    by simp
  finally show ?thesis .
qed
also have ... = (∑ x∈S. (1::prob)) / nnreal (CARD('f))
  by (rule sum-divide-nnreal) simp
also have ... = nnreal (card S) / nnreal (CARD('f))
  by (simp add: sum-1-nnreal)
also have ... = nnreal (card S) / nnreal size
  using size-card by simp
finally show ?thesis .
qed

lemma dist-expect-hash-dist-fresh-output-values:
  fixes x :: 'f protocol-hash-input
  assumes fresh: fmllookup (HashMap s) x = None
  shows
    dist-expect (hash-dist x s)
      (λy. if y ∈ hash-map-output-values s then 1 else 0) =
      nnreal (card (hash-map-output-values s)) / nnreal size
  using fresh
  unfolding hash-dist-def option-default-dist-def hash-default-def
  by (simp add: dist-expect-uniform-fUNIV-indicator)

```

lemma *dist-expect-hash-dist-fresh-indicator*:
fixes $x :: 'f$ *protocol-hash-input*
and $s :: ('f, 'a)$ *protocol-channel-scheme*
assumes *fresh*: $\text{fmlookup } (\text{HashMap } s) \ x = \text{None}$
shows
 $\text{dist-expect } (\text{hash-dist } x \ s) \ (\lambda y. \text{if } y \in B \text{ then } 1 \text{ else } 0) =$
 $\text{nnreal } (\text{card } B) / \text{nnreal size}$
using *fresh*
unfolding *hash-dist-def option-default-dist-def hash-default-def*
by (*simp add: dist-expect-uniform-fUNIV-indicator*)

lemma *wp-hash-fresh-set*:
fixes $x :: 'f$ *protocol-hash-input*
and $s :: ('f, 'a)$ *protocol-channel-scheme*
assumes *fresh*: $\text{fmlookup } (\text{HashMap } s) \ x = \text{None}$
shows
 $\text{wp-event } (\text{hash } x) \ (\lambda \text{out. case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (y, -) \Rightarrow y \in B) \ s =$
 $\text{nnreal } (\text{card } B) / \text{nnreal size}$
unfolding *wp-event-def*
by (*simp add: wp-hash dist-expect-hash-dist-fresh-indicator[OF fresh]*)

lemma *wp-hash-fresh-singleton-outcome*:
fixes $x :: 'f$ *protocol-hash-input*
and $s :: ('f, 'a)$ *protocol-channel-scheme*
assumes *fresh*: $\text{fmlookup } (\text{HashMap } s) \ x = \text{None}$
shows
 $\text{wp-event } (\text{hash } x)$
 $(\lambda \text{out. out} =$
 $\text{Some } (y, s(\text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s)))) \ s =$
 $1 / \text{nnreal size}$

proof –
have *indicator*:
 $(\lambda xa. \text{if } xa = y \wedge$
 $s(\text{HashMap} := \text{fmupd } x \ xa \ (\text{HashMap } s)) =$
 $s(\text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s))$
 $\text{then } 1 \text{ else } 0 :: \text{prob}) =$
 $(\lambda xa. \text{if } xa \in \{y\} \text{ then } 1 \text{ else } 0)$
by *auto*
have $\text{wp-event } (\text{hash } x)$
 $(\lambda \text{out. out} =$
 $\text{Some } (y, s(\text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s)))) \ s =$
 $\text{dist-expect } (\text{hash-dist } x \ s) \ (\lambda xa. \text{if } xa \in \{y\} \text{ then } 1 \text{ else } 0)$
unfolding *wp-event-def* **by** (*simp add: wp-hash indicator*)
also have $\dots = \text{nnreal } (\text{card } (\{y\} :: 'f \text{ set})) / \text{nnreal size}$
by (*rule dist-expect-hash-dist-fresh-indicator[OF fresh]*)
also have $\dots = 1 / \text{nnreal size}$

by *simp*
 finally show *?thesis* .
 qed

lemma *hash-fresh-outcome*:

fixes $x :: 'f$ protocol-hash-input
 and $s :: ('f, 'a)$ protocol-channel-scheme
 assumes *fresh*: $\text{fmlookup } (\text{HashMap } s) \ x = \text{None}$
 shows
 $\text{Some } (y, s(\text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s))) \in$
 $\text{set-dist } (\text{execute } (\text{hash } x) \ s)$

proof –

have *positive*:
 $0 < \text{wp-event } (\text{hash } x)$
 $(\lambda \text{out. out} =$
 $\text{Some } (y, s(\text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s)))) \ s$
 using *wp-hash-fresh-singleton-outcome*[*OF fresh, of y*]
 size-card by *simp*
 then obtain *out* where *support*:
 $\text{out} \in \text{set-dist } (\text{execute } (\text{hash } x) \ s)$
 and *out-eq*:
 $\text{out} = \text{Some } (y, s(\text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s)))$
 by (rule *wp-event-pos-imp-exists-support*)
 show *?thesis*
 using *support out-eq* by *simp*

qed

lemma *fresh-merkle-hash-has-pairwise-value-conflict*:

fixes $x :: 'f$ protocol-hash-input
 and $s :: ('f, 'a)$ protocol-channel-scheme
 assumes *merkle*: *merkle-hash-key* x
 and *fresh*: $\text{fmlookup } (\text{HashMap } s) \ x = \text{None}$
 and *neq*: $y \neq z$

shows

$\exists t \ u.$
 $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (\text{hash } x) \ s) \wedge$
 $\text{Some } (z, u) \in \text{set-dist } (\text{execute } (\text{hash } x) \ s) \wedge$
 $\text{merkle-hash-value-conflict } t \ u \wedge$
 $\text{merkle-hash-pairwise-coupling-bad } t \ u$

proof –

let $?t = s(\text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s))$
 let $?u = s(\text{HashMap} := \text{fmupd } x \ z \ (\text{HashMap } s))$
 have *out-y*: $\text{Some } (y, ?t) \in \text{set-dist } (\text{execute } (\text{hash } x) \ s)$
 by (rule *hash-fresh-outcome*[*OF fresh*])
 have *out-z*: $\text{Some } (z, ?u) \in \text{set-dist } (\text{execute } (\text{hash } x) \ s)$
 by (rule *hash-fresh-outcome*[*OF fresh*])
 have *conflict*: *merkle-hash-value-conflict* $?t \ ?u$
 unfolding *merkle-hash-value-conflict-def*
 using *merkle neq* by *auto*

then have *coupling: merkle-hash-pairwise-coupling-bad ?t ?u*
unfolding *merkle-hash-pairwise-coupling-bad-def* **by** *simp*
show *?thesis*
using *out-y out-z conflict coupling* **by** *blast*
qed

lemma *wp-receive-tagged-random-field-element-fresh-set:*
fixes *s :: ('f, 'a) protocol-channel-scheme*
assumes *fresh: fmlookup (HashMap s) (tag (PState s)) = None*
shows
wp-event (receive-tagged-random-field-element tag)
(λout. case out of None ⇒ False | Some (y, -) ⇒ y ∈ B) s =
nnreal (card B) / nnreal size
unfolding *wp-event-def receive-tagged-random-field-element-def*
protocol-receive-tagged-random-field-element-def
by (*simp add: wpsimps dist-expect-hash-dist-fresh-indicator[OF fresh]*)

lemma *wp-receive-counted-tagged-random-field-element-fresh-set:*
fixes *s :: ('f, 'a) protocol-channel-scheme*
assumes *fresh:*
fmlookup (HashMap s) (tag (counter s) (PState s)) = None
shows
wp-event
(protocol-receive-counted-tagged-random-field-element counter bump tag)
(λout. case out of None ⇒ False | Some (y, -) ⇒ y ∈ B) s =
nnreal (card B) / nnreal size
unfolding *wp-event-def protocol-receive-counted-tagged-random-field-element-def*
by (*simp add: wpsimps dist-expect-hash-dist-fresh-indicator[OF fresh]*)

lemma *wp-receive-alpha-challenge-fresh-set:*
fixes *s :: ('f, 'a) protocol-channel-scheme*
assumes *fresh: fmlookup (HashMap s) (AlphaChallenge (PAlphaCounter s) (PState s)) = None*
shows
wp-event receive-alpha-challenge
(λout. case out of None ⇒ False | Some (y, -) ⇒ y ∈ B) s =
nnreal (card B) / nnreal size
unfolding *receive-alpha-challenge-def*
by (*rule wp-receive-counted-tagged-random-field-element-fresh-set*
[where counter=PAlphaCounter
and bump=λs. s(PAlphaCounter := Suc (PAlphaCounter s))
and tag=AlphaChallenge and B=B and s=s, OF fresh])

lemma *wp-receive-query-index-challenge-fresh-set:*
fixes *s :: ('f, 'a) protocol-channel-scheme*
assumes *fresh:*
fmlookup (HashMap s) (QueryIndexChallenge (PQueryCounter s) (PState s)) = None
shows
wp-event receive-query-index-challenge
(λout. case out of None ⇒ False | Some (y, -) ⇒ y ∈ B) s =

```

    nreal (card B) / nreal size
unfolding receive-query-index-challenge-def
by (rule wp-receive-counted-tagged-random-field-element-fresh-set
    [where counter=PQueryCounter
     and bump=λs. s(PQueryCounter := Suc (PQueryCounter s))
     and tag=QueryIndexChallenge and B=B and s=s, OF fresh])

definition query-index-raw-preimage :: nat set ⇒ 'f set
  where
    query-index-raw-preimage B =
      {raw. index (to-nat raw) ∈ B}

definition query-index-nat-preimage :: nat set ⇒ nat set
  where
    query-index-nat-preimage B =
      {n ∈ range to-nat. n mod query-sample-space-size ∈ B}

lemma finite-query-index-raw-preimage[simp]:
  finite (query-index-raw-preimage B)
unfolding query-index-raw-preimage-def by simp

lemma finite-query-index-nat-preimage[simp]:
  finite (query-index-nat-preimage B)
unfolding query-index-nat-preimage-def by simp

lemma query-index-raw-preimage-image-to-nat:
  to-nat ' query-index-raw-preimage B = query-index-nat-preimage B
unfolding query-index-raw-preimage-def query-index-nat-preimage-def
  index-def query-sample-space-size-def
by auto

lemma card-query-index-raw-preimage-eq-nat-preimage:
  card (query-index-raw-preimage B) = card (query-index-nat-preimage B)
proof -
  have inj: inj-on to-nat (query-index-raw-preimage B)
    using to-nat-inj by (rule inj-on-subset) simp
  have card (query-index-raw-preimage B) =
    card (to-nat ' query-index-raw-preimage B)
    by (rule card-image[symmetric, OF inj])
  also have ... = card (query-index-nat-preimage B)
    unfolding query-index-raw-preimage-image-to-nat ..
  finally show ?thesis .
qed

lemma wp-receive-query-index-challenge-fresh-index-set:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes fresh:
    fmllookup (HashMap s) (QueryIndexChallenge (PQueryCounter s) (PState s)) = None
  shows

```

```

wp-event receive-query-index-challenge
  (λout. case out of
    None ⇒ False
    | Some (raw, -) ⇒ index (to-nat raw) ∈ B) s =
  nnreal (card (query-index-raw-preimage B)) / nnreal size
proof -
  have event-eq:
    (λout. case out of
      None ⇒ False
      | Some (raw, -) ⇒ index (to-nat raw) ∈ B) =
    (λout. case out of
      None ⇒ False
      | Some (raw, -) ⇒ raw ∈ query-index-raw-preimage B)
  unfolding query-index-raw-preimage-def
  by (rule ext) (auto split: option.splits prod.splits)
show ?thesis
  unfolding event-eq
  by (rule wp-receive-query-index-challenge-fresh-set[OF fresh])
qed

```

```

lemma card-mod-lessThan-dvd:
  assumes q-pos: 0 < q
  and dvd: q dvd N
  and b-lt: b < q
  shows card {n. n < N ∧ n mod q = b} = N div q
proof -
  let ?A = {0..<N div q}
  let ?B = {n. n < N ∧ n mod q = b}
  have N-eq: N = (N div q) * q
    using dvd q-pos by (simp add: dvd-eq-mod-eq-0)
  have image-eq: (λk. k * q + b) ' ?A = ?B
  proof
    show (λk. k * q + b) ' ?A ⊆ ?B
  proof
    fix n
    assume n ∈ (λk. k * q + b) ' ?A
    then obtain k where k-lt: k < N div q and n-eq: n = k * q + b
    by auto
    have k * q + b < (N div q) * q
  proof -
    have sk-le: Suc k ≤ N div q
      using k-lt by simp
    have skq-le: Suc k * q ≤ (N div q) * q
      by (rule mult-le-mono1[OF sk-le])
    have k * q + b < k * q + q
      using b-lt by simp
    also have ... = Suc k * q
      by simp
    also have ... ≤ (N div q) * q

```



```

    by (rule skq-le)
    finally show ?thesis .
qed
then have  $n < N$ 
    using  $N\text{-eq } n\text{-eq}$  by simp
moreover have  $n \bmod q = b$ 
    using  $n\text{-eq } b\text{-lt}$  by simp
ultimately show  $n \in ?B$ 
    by simp
qed
show  $?B \subseteq (\lambda k. k * q + b) \text{ ` } ?A$ 
proof
  fix  $n$ 
  assume  $n\text{-in}: n \in ?B$ 
  then have  $n\text{-lt}: n < N$  and  $n\text{-mod}: n \bmod q = b$ 
    by simp-all
  have  $n\text{-eq}: n = (n \text{ div } q) * q + b$ 
    using  $\text{div-mult-mod-eq}[of\ n\ q]\ n\text{-mod}$  by simp
  have  $\text{div-lt}: n \text{ div } q < N \text{ div } q$ 
  proof (rule ccontr)
    assume  $\neg n \text{ div } q < N \text{ div } q$ 
    then have  $ge: N \text{ div } q \leq n \text{ div } q$ 
      by simp
    have  $N = (N \text{ div } q) * q$ 
      by (rule  $N\text{-eq}$ )
    also have  $\dots \leq (n \text{ div } q) * q$ 
      using  $ge$  by simp
    also have  $\dots \leq (n \text{ div } q) * q + b$ 
      by simp
    also have  $\dots = n$ 
      using  $n\text{-eq}$  by simp
    finally show  $False$ 
      using  $n\text{-lt}$  by simp
  qed
  then show  $n \in (\lambda k. k * q + b) \text{ ` } ?A$ 
    using  $n\text{-eq}$  by auto
qed
qed
have  $\text{inj}: \text{inj-on } (\lambda k. k * q + b) \text{ ` } ?A$ 
  using  $q\text{-pos}$  unfolding  $\text{inj-on-def}$  by auto
have  $\text{card } ?B = \text{card } ((\lambda k. k * q + b) \text{ ` } ?A)$ 
  using  $\text{image-eq}$  by simp
also have  $\dots = \text{card } ?A$ 
  by (rule  $\text{card-image}[OF\ \text{inj}]$ )
also have  $\dots = N \text{ div } q$ 
  by simp
finally show ?thesis .
qed

```

lemma *card-query-index-nat-preimage-uniform-range*:

assumes *range-eq*: $\text{range to-nat} = \{0..<\text{size}\}$
and *dvd*: $\text{query-sample-space-size} \text{ dvd } \text{size}$
and *subset*: $B \subseteq \text{query-sample-space}$
shows
 $\text{card } (\text{query-index-nat-preimage } B) =$
 $(\text{size div query-sample-space-size}) * \text{card } B$

proof –

let $?q = \text{query-sample-space-size}$
let $?A = \lambda b. \{n. n < \text{size} \wedge n \bmod ?q = b\}$
have *finite-space*: *finite query-sample-space*
unfolding *query-sample-space-def* **by** *simp*
have *finite-B*: *finite B*
by (*rule finite-subset[OF subset finite-space]*)
have *B-lt*: $\bigwedge b. b \in B \implies b < ?q$
using *subset* **unfolding** *query-sample-space-def* **by** *auto*
have *preimage-eq*: $\text{query-index-nat-preimage } B = \bigcup (?A \text{ ‘ } B)$
unfolding *query-index-nat-preimage-def range-eq* **by** *auto*
have *finite-A*: $\forall b \in B. \text{finite } (?A \text{ } b)$
by *simp*
have *disjoint*:
 $\forall b \in B. \forall c \in B. b \neq c \longrightarrow ?A \text{ } b \cap ?A \text{ } c = \{\}$
by *auto*
have $\text{card } (\text{query-index-nat-preimage } B) = \text{card } (\bigcup (?A \text{ ‘ } B))$
unfolding *preimage-eq ..*
also have $\dots = (\sum b \in B. \text{card } (?A \text{ } b))$
by (*rule card-UN-disjoint[OF finite-B finite-A disjoint]*)
also have $\dots = (\sum b \in B. \text{size div } ?q)$
by (*rule sum.cong*)
 $(\text{use } B\text{-lt in}$
 $\text{‹simp-all add: card-mod-lessThan-dvd[OF query-sample-space-size-pos dvd]›})$
also have $\dots = (\text{size div } ?q) * \text{card } B$
by *simp*
finally show *?thesis* .

qed

lemma *wp-receive-trace-fri-challenge-fresh-set*:

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes *fresh*:
 $\text{fmlookup } (\text{HashMap } s) (\text{TraceFriChallenge } (P\text{TraceFriCounter } s) (P\text{State } s)) = \text{None}$
shows
 $\text{wp-event receive-trace-fri-challenge}$
 $(\lambda \text{out. case out of None} \implies \text{False} \mid \text{Some } (y, -) \implies y \in B) \text{ } s =$
 $\text{nnreal } (\text{card } B) / \text{nnreal } \text{size}$
unfolding *receive-trace-fri-challenge-def*
by (*rule wp-receive-counted-tagged-random-field-element-fresh-set*
 $[\text{where counter} = P\text{TraceFriCounter}$
and $\text{bump} = \lambda s. s(P\text{TraceFriCounter} := \text{Suc } (P\text{TraceFriCounter } s))$
and $\text{tag} = \text{TraceFriChallenge}$ **and** $B = B$ **and** $s = s, \text{ OF fresh}]$)

lemma *wp-receive-composition-fri-challenge-fresh-set:*
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes *fresh:*
 $\text{fmlookup } (\text{HashMap } s) (\text{CompositionFriChallenge } (\text{PCompositionFriCounter } s) (\text{PState } s))$
 $= \text{None}$
shows
 $\text{wp-event receive-composition-fri-challenge}$
 $(\lambda \text{out. case out of None} \Rightarrow \text{False} \mid \text{Some } (y, -) \Rightarrow y \in B) \ s =$
 $\text{nnreal } (\text{card } B) / \text{nnreal size}$
unfolding *receive-composition-fri-challenge-def*
by (*rule wp-receive-counted-tagged-random-field-element-fresh-set*
 $[\text{where counter} = \text{PCompositionFriCounter}$
 $\text{and bump} = \lambda s. s \llbracket \text{PCompositionFriCounter} := \text{Suc } (\text{PCompositionFriCounter } s) \rrbracket$
 $\text{and tag} = \text{CompositionFriChallenge and } B = B \text{ and } s = s, \text{ OF fresh}]$)

fun *merkle-path-root-in-map*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $\text{nat} \Rightarrow \text{nat} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ option}$
where
 $\text{merkle-path-root-in-map } s \text{ len idx leaf } [] =$
 $\text{fmlookup } (\text{HashMap } s) (\text{MerkleLeaf leaf})$
 $\mid \text{merkle-path-root-in-map } s \text{ len idx leaf } (a \# \text{path}) =$
 $(\text{if idx} < \text{len div } 2$
 $\text{then case merkle-path-root-in-map } s (\text{len div } 2) \text{ idx leaf path of}$
 $\text{None} \Rightarrow \text{None}$
 $\mid \text{Some } x \Rightarrow \text{fmlookup } (\text{HashMap } s) (\text{MerkleNode } x \ a)$
 $\text{else case merkle-path-root-in-map } s (\text{len div } 2) (\text{idx} - \text{len div } 2)$
 leaf path of
 $\text{None} \Rightarrow \text{None}$
 $\mid \text{Some } x \Rightarrow \text{fmlookup } (\text{HashMap } s) (\text{MerkleNode } a \ x))$

lemma *check-authentication-path-outcome-merkle-path-root:*
fixes $\text{root leaf} :: 'f$
and $s \ t \ u :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes *outcome:*
 $\text{Some } (\text{root}, t) \in$
 $\text{set-dist } (\text{execute } (\text{check-authentication-path len idx leaf path}) \ s)$
and *ext:* $t \leq u$
shows $\text{merkle-path-root-in-map } u \text{ len idx leaf path} = \text{Some root}$
using *outcome ext*
unfolding *check-authentication-path-def protocol-check-authentication-path-def*
proof (*induction path arbitrary: len idx leaf s root t*)
case *Nil*
then have *hash-step:*
 $\text{Some } (\text{root}, t) \in \text{set-dist } (\text{execute } (\text{hash } (\text{MerkleLeaf leaf})) \ s)$
by *simp*
have *lookup-t:*
 $\text{fmlookup } (\text{HashMap } t) (\text{MerkleLeaf leaf}) = \text{Some root}$

```

    by (rule hash-outcome(2)[OF hash-step])
  show ?case
    using hash-extension-lookup[OF lookup-t Nil.prem(2)] by simp
next
case (Cons a path)
show ?case
proof (cases idx < len div 2)
  case True
  from Cons.prem(1) obtain child s1 where
    child-check:
      Some (child, s1) ∈
        set-dist (execute
          (protocol-merkle.check-authentication-path
            (len div 2) idx (MerkleLeaf leaf) path) s)
    and root-hash:
      Some (root, t) ∈
        set-dist (execute (hash (MerkleNode child a)) s1)
    using True by (auto elim!: set-dist-bindE)
  have s1-t: s1 ≤ t
    by (rule hash-outcome(1)[OF root-hash])
  have s1-u: s1 ≤ u
    using s1-t Cons.prem(2) by (meson hash-ext-trans)
  have child-root:
    merkle-path-root-in-map u (len div 2) idx leaf path = Some child
    by (rule Cons.IH[OF child-check s1-u])
  have lookup-t:
    fmlookup (HashMap t) (MerkleNode child a) = Some root
    by (rule hash-outcome(2)[OF root-hash])
  have lookup-u:
    fmlookup (HashMap u) (MerkleNode child a) = Some root
    by (rule hash-extension-lookup[OF lookup-t Cons.prem(2)])
  show ?thesis
    using True child-root lookup-u by simp
next
case False
  from Cons.prem(1) obtain child s1 where
    child-check:
      Some (child, s1) ∈
        set-dist (execute
          (protocol-merkle.check-authentication-path
            (len div 2) (idx - len div 2) (MerkleLeaf leaf) path) s)
    and root-hash:
      Some (root, t) ∈
        set-dist (execute (hash (MerkleNode a child)) s1)
    using False by (auto elim!: set-dist-bindE)
  have s1-t: s1 ≤ t
    by (rule hash-outcome(1)[OF root-hash])
  have s1-u: s1 ≤ u
    using s1-t Cons.prem(2) by (meson hash-ext-trans)

```

```

have child-root:
  merkle-path-root-in-map u (len div 2) (idx - len div 2) leaf path =
    Some child
  by (rule Cons.IH[OF child-check s1-u])
have lookup-t:
  fmlookup (HashMap t) (MerkleNode a child) = Some root
  by (rule hash-outcome(2)[OF root-hash])
have lookup-u:
  fmlookup (HashMap u) (MerkleNode a child) = Some root
  by (rule hash-extension-lookup[OF lookup-t Cons.prem(2)])
show ?thesis
  using False child-root lookup-u by simp
qed
qed

lemma merkle-path-root-same-index-eq-or-hash-collision:
  fixes root leaf leaf' :: 'f
  and s :: ('f, 'a) protocol-channel-scheme
  assumes root1:
    merkle-path-root-in-map s len idx leaf path = Some root
  and root2:
    merkle-path-root-in-map s len idx leaf' path' = Some root
  shows (leaf = leaf'  $\wedge$  path = path')  $\vee$  hash-map-output-collision s
  using root1 root2
proof (induction path arbitrary: len idx leaf path' leaf' root)
  case Nil
  show ?case
  proof (cases path')
  case Nil
  show ?thesis
  proof (cases leaf = leaf')
  case True
  then show ?thesis
    using Nil by simp
  next
  case False
  have lookup-leaf: fmlookup (HashMap s) (MerkleLeaf leaf) = Some root
    using Nil.prem(1) by simp
  have lookup-leaf':
    fmlookup (HashMap s) (MerkleLeaf leaf') = Some root
    using Nil.prem(2) unfolding Nil by simp
  have hash-map-output-collision s
    by (rule hash-map-output-collision-merkle-leafI
      [OF False lookup-leaf lookup-leaf'])
  then show ?thesis by simp
  qed
next
  case (Cons a rest)
  have lookup-leaf: fmlookup (HashMap s) (MerkleLeaf leaf) = Some root

```

```

    using Nil.premis(1) by simp
have hash-map-output-collision s
proof (cases idx < len div 2)
  case True
  from Nil.premis(2)[unfolded Cons] True obtain child where
    lookup-node: fmlookup (HashMap s) (MerkleNode child a) = Some root
    by (auto split: option.splits)
  show ?thesis
    by (rule hash-map-output-collision-merkle-leaf-nodeI
        [OF lookup-leaf lookup-node])
next
  case False
  from Nil.premis(2)[unfolded Cons] False obtain child where
    lookup-node: fmlookup (HashMap s) (MerkleNode a child) = Some root
    by (auto split: option.splits)
  show ?thesis
    by (rule hash-map-output-collision-merkle-leaf-nodeI
        [OF lookup-leaf lookup-node])
qed
then show ?thesis by simp
qed
next
  case (Cons a path)
  show ?case
  proof (cases path')
    case Nil
    have lookup-leaf': fmlookup (HashMap s) (MerkleLeaf leaf') = Some root
      using Cons.premis(2) unfolding Nil by simp
    have hash-map-output-collision s
    proof (cases idx < len div 2)
      case True
      from Cons.premis(1) True obtain child where
        lookup-node: fmlookup (HashMap s) (MerkleNode child a) = Some root
        by (auto split: option.splits)
      show ?thesis
        by (rule hash-map-output-collisionI
            [of MerkleNode child a MerkleLeaf leaf' s root])
            (use lookup-node lookup-leaf' in simp-all)
    next
      case False
      from Cons.premis(1) False obtain child where
        lookup-node: fmlookup (HashMap s) (MerkleNode a child) = Some root
        by (auto split: option.splits)
      show ?thesis
        by (rule hash-map-output-collisionI
            [of MerkleNode a child MerkleLeaf leaf' s root])
            (use lookup-node lookup-leaf' in simp-all)
    qed
  then show ?thesis by simp
qed

```

```

next
case (Cons b path'')
show ?thesis
proof (cases idx < len div 2)
case True
from Cons.premis(1) obtain child where
child1:
merkle-path-root-in-map s (len div 2) idx leaf path =
Some child
and lookup1:
fmlookup (HashMap s) (MerkleNode child a) = Some root
using True by (auto split: option.splits)
from Cons.premis(2)[unfolded Cons] obtain child' where
child2:
merkle-path-root-in-map s (len div 2) idx leaf' path'' =
Some child'
and lookup2:
fmlookup (HashMap s) (MerkleNode child' b) = Some root
using True by (auto split: option.splits)
show ?thesis
proof (cases (child, a) = (child', b))
case False
have hash-map-output-collision s
by (rule hash-map-output-collision-merkle-nodeI
[OF False lookup1 lookup2])
then show ?thesis by simp
next
case pair-eq: True
have child-eq: child = child' and a-eq: a = b
using pair-eq by simp-all
have rec:
(leaf = leaf'  $\wedge$  path = path'')  $\vee$  hash-map-output-collision s
by (rule Cons.IH[OF child1 child2[unfolded child-eq[symmetric]]])
then show ?thesis
using a-eq Cons by auto
qed
next
case False
from Cons.premis(1) obtain child where
child1:
merkle-path-root-in-map s (len div 2) (idx - len div 2)
leaf path = Some child
and lookup1:
fmlookup (HashMap s) (MerkleNode a child) = Some root
using False by (auto split: option.splits)
from Cons.premis(2)[unfolded Cons] obtain child' where
child2:
merkle-path-root-in-map s (len div 2) (idx - len div 2)
leaf' path'' = Some child'

```

```

    and lookup2:
      fmlookup (HashMap s) (MerkleNode b child') = Some root
    using False by (auto split: option.splits)
  show ?thesis
  proof (cases (a, child) = (b, child'))
    case False
    have hash-map-output-collision s
    by (rule hash-map-output-collision-merkle-nodeI
      [OF False lookup1 lookup2])
    then show ?thesis by simp
  next
    case pair-eq: True
    have child-eq: child = child' and a-eq: a = b
    using pair-eq by simp-all
    have rec:
      (leaf = leaf'  $\wedge$  path = path')  $\vee$  hash-map-output-collision s
    by (rule Cons.IH[OF child1 child2[unfolded child-eq[symmetric]]])
    then show ?thesis
  using a-eq Cons by auto
qed
qed
qed
qed

```

lemma *protocol-created-tree-singletonD*:

```

  assumes protocol-created-tree [x] t s
  obtains h where
    t = ⟨⟨⟩, h, ⟨⟩⟩
    fmlookup (HashMap s) (MerkleLeaf x) = Some h
  using assms
  unfolding protocol-created-tree-def
  by (auto simp: protocol-merkle.created-tree.simps)

```

lemma *protocol-created-tree-internalD*:

```

  assumes created: protocol-created-tree xs t s
  and len: 2 ≤ length xs
  obtains l h r i where
    i = length xs div 2
    t = ⟨l, h, r⟩
    protocol-created-tree (take i xs) l s
    protocol-created-tree (drop i xs) r s
    fmlookup (HashMap s) (MerkleNode (value l) (value r)) = Some h
  proof -
    obtain a b cs where xs-eq: xs = a # b # cs
    using len by (cases xs; cases tl xs) auto
    obtain l h r where
      t: t = ⟨l, h, r⟩
    and left:
      protocol-merkle.created-tree

```



```

    (take (length xs div 2) (map MerkleLeaf xs)) l s
  and right:
    protocol-merkle.created-tree
    (drop (length xs div 2) (map MerkleLeaf xs)) r s
  and root:
    fmllookup (HashMap s) (MerkleNode (value l) (value r)) = Some h
  using created len
  unfolding protocol-created-tree-def
  by (auto simp: xs-eq protocol-merkle.created-tree.simps Let-def)
have left':
  protocol-created-tree (take (length xs div 2) xs) l s
  using left
  unfolding protocol-created-tree-def by (simp add: take-map)
have right':
  protocol-created-tree (drop (length xs div 2) xs) r s
  using right
  unfolding protocol-created-tree-def by (simp add: drop-map)
show ?thesis
  by (rule that[OF refl t left' right' root])
qed

```

```

lemma protocol-created-tree-path-root-in-map:
  assumes created: protocol-created-tree xs tree s
  and len-pow: length xs = 2 ^ n
  and i-bound: i < length xs
  shows
    merkle-path-root-in-map s (length xs) i (xs ! i)
    (get-authentication-path (length xs) i tree) = Some (value tree)
  using created len-pow i-bound
proof (induction n arbitrary: xs tree i)
  case 0
  then have len-xs: length xs = 1
    by simp
  then obtain x where xs-eq: xs = [x]
    by (cases xs) auto
  from protocol-created-tree-singletonD[OF 0.premis(1)[unfolded xs-eq]]
  obtain h where tree-eq:
    tree = (⟨⟩, h, ⟨⟩)
    and lookup: fmllookup (HashMap s) (MerkleLeaf x) = Some h
  by blast
  have i-eq: i = 0
    using 0.premis(3) unfolding xs-eq by simp
  show ?case
    using lookup unfolding xs-eq tree-eq i-eq by simp
next
  case (Suc n)
  let ?mid = length xs div 2
  have len-xs: length xs = 2 ^ Suc n
    by (rule Suc.premis(2))

```

```

have len-ge:  $2 \leq \text{length } xs$ 
  using len-xs by simp
have len-not-one:  $\text{length } xs \neq 1$ 
  using len-ge by simp
obtain l h r mid where mid-def:  $mid = ?mid$ 
  and tree-eq:  $tree = \langle l, h, r \rangle$ 
  and left:  $\text{protocol-created-tree } (take\ mid\ xs)\ l\ s$ 
  and right:  $\text{protocol-created-tree } (drop\ mid\ xs)\ r\ s$ 
  and root-lookup:
    fmlookup (HashMap s) (MerkleNode (value l) (value r)) = Some h
  by (rule protocol-created-tree-internalD[OF Suc.prem1 len-ge])
have mid-pow:  $mid = 2^n$ 
  using len-xs unfolding mid-def by simp
have mid-pos:  $0 < mid$ 
  using mid-pow by simp
have mid-lt:  $mid < \text{length } xs$ 
  using len-xs mid-pow by simp
have take-len:  $\text{length } (take\ mid\ xs) = 2^n$ 
  using mid-lt mid-pow by simp
have drop-len:  $\text{length } (drop\ mid\ xs) = 2^n$ 
  using len-xs mid-pow by simp
show ?case
proof (cases  $i < mid$ )
case True
  have idx-take:  $i < \text{length } (take\ mid\ xs)$ 
    using True take-len mid-pow by simp
  have child:
    merkle-path-root-in-map s ( $\text{length } (take\ mid\ xs)$ ) i
      ( $take\ mid\ xs ! i$ )
      ( $\text{get-authentication-path } (\text{length } (take\ mid\ xs))\ i\ l$ ) =
      Some (value l)
    by (rule Suc.IH[OF left take-len idx-take])
  have child-mid:
    merkle-path-root-in-map s mid i ( $xs ! i$ )
      ( $\text{get-authentication-path } mid\ i\ l$ ) = Some (value l)
    using child True take-len mid-pow by simp
  show ?thesis
    using True child-mid root-lookup len-not-one
    unfolding tree-eq mid-def by simp
next
case False
  have idx-drop:  $i - mid < \text{length } (drop\ mid\ xs)$ 
    using False Suc.prem3 drop-len mid-pow mid-def by simp
  have child:
    merkle-path-root-in-map s ( $\text{length } (drop\ mid\ xs)$ ) ( $i - mid$ )
      ( $drop\ mid\ xs ! (i - mid)$ )
      ( $\text{get-authentication-path } (\text{length } (drop\ mid\ xs))\ (i - mid)\ r$ ) =
      Some (value r)
    by (rule Suc.IH[OF right drop-len idx-drop])

```

```

have child-mid:
  merkle-path-root-in-map s mid (i - mid) (xs ! i)
    (get-authentication-path mid (i - mid) r) = Some (value r)
  using child False Suc.premis(3) drop-len mid-pow by simp
show ?thesis
  using False child-mid root-lookup len-not-one
  unfolding tree-eq mid-def by simp
qed
qed

lemma protocol-created-tree-merkle-mono:
  assumes created: protocol-created-tree xs t s
  and ext: merkle-hash-extends s u
  shows protocol-created-tree xs t u
  using created
proof (induction xs arbitrary: t rule: length-induct)
  case (1 xs)
  show ?case
  proof (cases xs = [])
  case True
  then show ?thesis
    using 1.premis
    unfolding protocol-created-tree-def
    by (simp add: protocol-merkle.created-tree.simps)
  next
  case xs-nonempty: False
  show ?thesis
  proof (cases length xs = 1)
  case True
  then obtain x where xs-eq: xs = [x]
    using xs-nonempty by (cases xs) auto
  from protocol-created-tree-singletonD[OF 1.premis[unfolded xs-eq]]
  obtain h where
    t-eq: t = ⟨⟨⟩, h, ⟨⟩⟩
    and lookup-s: fmllookup (HashMap s) (MerkleLeaf x) = Some h
    by blast
  have lookup-u: fmllookup (HashMap u) (MerkleLeaf x) = Some h
    using ext lookup-s unfolding merkle-hash-extends-def by simp
  show ?thesis
    unfolding protocol-created-tree-def xs-eq t-eq
    using lookup-u by (simp add: protocol-merkle.created-tree.simps)
  next
  case not-singleton: False
  have len-ge: 2 ≤ length xs
    using xs-nonempty not-singleton by (cases xs; cases tl xs) auto
  obtain l h r i where
    i-def: i = length xs div 2
    and t-eq: t = ⟨l, h, r⟩
    and left-created: protocol-created-tree (take i xs) l s

```

```

and right-created: protocol-created-tree (drop i xs) r s
and root-lookup:
  fmlookup (HashMap s) (MerkleNode (value l) (value r)) = Some h
  by (rule protocol-created-tree-internalD[OF 1.prems len-ge])
have i-pos:  $0 < i$ 
  using i-def len-ge by simp
have i-lt:  $i < \text{length } xs$ 
  using i-def len-ge by simp
have take-less:  $\text{length } (\text{take } i \text{ } xs) < \text{length } xs$ 
  using i-lt by simp
have drop-less:  $\text{length } (\text{drop } i \text{ } xs) < \text{length } xs$ 
  using i-pos i-lt by simp
have left-u: protocol-created-tree (take i xs) l u
  by (rule 1.IH[rule-format, OF take-less left-created])
have right-u: protocol-created-tree (drop i xs) r u
  by (rule 1.IH[rule-format, OF drop-less right-created])
have root-lookup-u:
  fmlookup (HashMap u) (MerkleNode (value l) (value r)) = Some h
  using ext root-lookup unfolding merkle-hash-extends-def by simp
obtain a b cs where xs-eq:  $xs = a \# b \# cs$ 
  using len-ge by (cases xs; cases tl xs) auto
have take-leaf-eq:
  MerkleLeaf a #
    take ( $\text{length } cs \text{ div } 2$ ) (MerkleLeaf b # map MerkleLeaf cs) =
  MerkleLeaf a #
    map MerkleLeaf (take ( $\text{length } cs \text{ div } 2$ ) (b # cs))
  by (metis list.map(2) take-map)
have drop-leaf-eq:
  drop ( $\text{length } cs \text{ div } 2$ ) (MerkleLeaf b # map MerkleLeaf cs) =
  map MerkleLeaf (drop ( $\text{length } cs \text{ div } 2$ ) (b # cs))
  by (metis list.map(2) drop-map)
have left-u-raw:
  protocol-merkle.created-tree
    (MerkleLeaf a #
      take ( $\text{length } cs \text{ div } 2$ ) (MerkleLeaf b # map MerkleLeaf cs))
    l u
  using left-u
  unfolding protocol-created-tree-def i-def xs-eq
  by (simp add: take-leaf-eq)
have right-u-raw:
  protocol-merkle.created-tree
    (drop ( $\text{length } cs \text{ div } 2$ ) (MerkleLeaf b # map MerkleLeaf cs))
    r u
  using right-u
  unfolding protocol-created-tree-def i-def xs-eq
  by (simp add: drop-leaf-eq)
show ?thesis
  using left-u-raw right-u-raw root-lookup-u
  unfolding protocol-created-tree-def t-eq xs-eq

```

```

      by (auto simp: protocol-merkle.created-tree.simps Let-def)
    qed
  qed
qed

lemma created-tree-merkle-mono:
  assumes created: created-tree xs t s
    and ext: merkle-hash-extends s u
  shows created-tree xs t u
  using protocol-created-tree-merkle-mono
    [OF created[unfolded created-tree-def] ext]
  unfolding created-tree-def .

lemma merkle-root-binds-table-merkle-mono:
  assumes bind: merkle-root-binds-table rt table s
    and ext: merkle-hash-extends s u
  shows merkle-root-binds-table rt table u
proof -
  from bind obtain tree where
    created: created-tree table tree s
    and root: rt = value tree
    unfolding merkle-root-binds-table-def by blast
  have created-tree table tree u
    by (rule created-tree-merkle-mono[OF created ext])
  then show ?thesis
    unfolding merkle-root-binds-table-def
    using root by blast
qed

lemma protocol-created-tree-same-length-root-eq-or-hash-collision:
  assumes left-created: protocol-created-tree xs tx s
    and right-created: protocol-created-tree ys ty s
    and same-len: length xs = length ys
    and nonempty-xs: xs ≠ []
    and nonempty-ys: ys ≠ []
    and same-root: value tx = value ty
  shows xs = ys ∨ hash-map-output-collision s
  using assms
proof (induction xs arbitrary: ys tx ty rule: length-induct)
  case (1 xs)
  show ?case
  proof (cases length xs = 1)
    case True
    then obtain x where xs-eq: xs = [x]
      using 1.premis(4) by (cases xs) auto
    from True 1.premis(3,5) obtain y where ys-eq: ys = [y]
      by (cases ys) auto
    from protocol-created-tree-singletonD[OF 1.premis(1)[unfolded xs-eq]]
    obtain hx where tx: tx = ⟨⟨⟩, hx, ⟨⟩⟩

```

```

    and hx-lookup: fmlookup (HashMap s) (MerkleLeaf x) = Some hx
    by blast
from protocol-created-tree-singletonD[OF 1.premis(2)[unfolded ys-eq]]
obtain hy where ty: ty = ⟨⟨⟩, hy, ⟨⟩⟩
    and hy-lookup: fmlookup (HashMap s) (MerkleLeaf y) = Some hy
    by blast
have h-eq: hx = hy
    using 1.premis(6) unfolding tx ty by simp
show ?thesis
proof (cases x = y)
  case True
  then show ?thesis
    using xs-eq ys-eq by simp
next
  case False
  have hy-lookup': fmlookup (HashMap s) (MerkleLeaf y) = Some hx
    using hy-lookup h-eq by simp
  then have hash-map-output-collision s
    by (rule hash-map-output-collision-merkle-leafI[OF False hx-lookup])
  then show ?thesis by simp
qed
next
case False
have 0 < length xs
  using 1.premis(4) by simp
have len-ge: 2 ≤ length xs
  using False ⟨0 < length xs⟩ by linarith
have len-ge-y: 2 ≤ length ys
  using len-ge 1.premis(3) by simp
obtain lx hx rx i where i-def: i = length xs div 2
  and tx: tx = ⟨lx, hx, rx⟩
  and lx-created: protocol-created-tree (take i xs) lx s
  and rx-created: protocol-created-tree (drop i xs) rx s
  and hx-lookup:
    fmlookup (HashMap s) (MerkleNode (value lx) (value rx)) = Some hx
  by (rule protocol-created-tree-internalD[OF 1.premis(1) len-ge])
obtain ly hy ry j where j-def: j = length ys div 2
  and ty: ty = ⟨ly, hy, ry⟩
  and ly-created: protocol-created-tree (take j ys) ly s
  and ry-created: protocol-created-tree (drop j ys) ry s
  and hy-lookup:
    fmlookup (HashMap s) (MerkleNode (value ly) (value ry)) = Some hy
  by (rule protocol-created-tree-internalD[OF 1.premis(2) len-ge-y])
have j-i: j = i
  using i-def j-def 1.premis(3) by simp
have ly-created-i: protocol-created-tree (take i ys) ly s
  using ly-created j-i by simp
have ry-created-i: protocol-created-tree (drop i ys) ry s
  using ry-created j-i by simp

```

```

have h-eq: hx = hy
  using 1.premis(6) unfolding tx ty by simp
show ?thesis
proof (cases (value lx, value rx) = (value ly, value ry))
  case False
  have hy-lookup':
    fmlookup (HashMap s) (MerkleNode (value ly) (value ry)) = Some hx
    using hy-lookup h-eq by simp
  then have hash-map-output-collision s
    by (rule hash-map-output-collision-merkle-nodeI[OF False hx-lookup])
  then show ?thesis by simp
next
case True
then have left-root: value lx = value ly
  and right-root: value rx = value ry
  by simp-all
have i-pos: 0 < i
  using i-def len-ge by simp
have i-lt: i < length xs
  using i-def len-ge by simp
have i-lt-y: i < length ys
  using i-lt 1.premis(3) by simp
have length-take-xs: length (take i xs) = i
  using i-lt by simp
have length-take-ys: length (take i ys) = i
  using i-lt-y by simp
have left-len: length (take i xs) = length (take i ys)
  using i-def j-i 1.premis(3) by simp
have right-len: length (drop i xs) = length (drop i ys)
  using i-def j-i 1.premis(3) by simp
have left-nonempty-xs: take i xs ≠ []
  using i-pos length-take-xs by auto
have left-nonempty-ys: take i ys ≠ []
  using i-pos length-take-ys by auto
have right-nonempty-xs: drop i xs ≠ []
  using i-lt by simp
have right-nonempty-ys: drop i ys ≠ []
  using i-lt-y by simp
have take-less: length (take i xs) < length xs
  using i-lt by simp
have drop-less: length (drop i xs) < length xs
  using i-pos i-lt by simp
show ?thesis
proof (cases take i xs = take i ys)
  case False
  have take i xs = take i ys ∨ hash-map-output-collision s
    by (rule 1.IH[rule-format, OF take-less lx-created ly-created-i
      left-len left-nonempty-xs left-nonempty-ys left-root])
  then show ?thesis

```

```

    using False by simp
next
case left-eq: True
show ?thesis
proof (cases drop i xs = drop i ys)
case True
then have xs = ys
  using left-eq i-def j-i 1.prems(3)
  by (metis append-take-drop-id)
then show ?thesis by simp
next
case False
have drop i xs = drop i ys  $\vee$  hash-map-output-collision s
  by (rule 1.IH[rule-format, OF drop-less rx-created ry-created-i
    right-len right-nonempty-xs right-nonempty-ys right-root])
then show ?thesis
  using False by simp
qed
qed
qed
qed
qed
qed

lemma merkle-root-binds-same-length-tables-collision-imp-hash-collision:
  assumes bind1: merkle-root-binds-table rt table s
    and bind2: merkle-root-binds-table rt table' s
    and same-len: length table = length table'
    and nonempty: table  $\neq$  []
    and neq: table  $\neq$  table'
  shows hash-map-output-collision s
proof -
  from bind1 obtain tree where tree:
    created-tree table tree s rt = value tree
  unfolding merkle-root-binds-table-def by blast
  from bind2 obtain tree' where tree':
    created-tree table' tree' s rt = value tree'
  unfolding merkle-root-binds-table-def by blast
  have nonempty': table'  $\neq$  []
    using same-len nonempty by auto
  have same-root: value tree = value tree'
    using tree tree' by simp
  have table = table'  $\vee$  hash-map-output-collision s
    by (rule protocol-created-tree-same-length-root-eq-or-hash-collision
      [OF tree(1)[unfolded created-tree-def]
        tree'(1)[unfolded created-tree-def]
          same-len nonempty nonempty' same-root])
  then show ?thesis
    using neq by simp
qed

```


lemma *merkle-root-binds-same-length-tables-unique-if-no-hash-collision:*
assumes *bind1*: merkle-root-binds-table *rt* *table* *s*
and *bind2*: merkle-root-binds-table *rt* *table'* *s*
and *same-len*: *length table* = *length table'*
and *clean*: $\neg$ *hash-map-output-collision s*
shows *table* = *table'*
proof (*cases table* = [])
case *True*
then show *?thesis*
using *same-len* **by** *simp*
next
case *False*
show *?thesis*
proof (*rule ccontr*)
assume *neq*: *table* $\neq$ *table'*
have *hash-map-output-collision s*
by (*rule merkle-root-binds-same-length-tables-collision-imp-hash-collision*
[*OF bind1 bind2 same-len False neq*])
then show *False*
using *clean* **by** *contradiction*
qed
qed

lemma *merkle-root-binds-same-length-tables-common-extension-collision:*
assumes *bind1*: merkle-root-binds-table *rt* *table* *s1*
and *bind2*: merkle-root-binds-table *rt* *table'* *s2*
and *ext1*: *s1* $\leq$ *u*
and *ext2*: *s2* $\leq$ *u*
and *same-len*: *length table* = *length table'*
and *nonempty*: *table* $\neq$ []
and *neq*: *table* $\neq$ *table'*
shows *hash-map-output-collision u*
proof –
have *bind1-u*: merkle-root-binds-table *rt* *table* *u*
by (*rule merkle-root-binds-table-mono*[*OF bind1 ext1*])
have *bind2-u*: merkle-root-binds-table *rt* *table'* *u*
by (*rule merkle-root-binds-table-mono*[*OF bind2 ext2*])
show *?thesis*
by (*rule merkle-root-binds-same-length-tables-collision-imp-hash-collision*
[*OF bind1-u bind2-u same-len nonempty neq*])
qed

lemma *merkle-root-binds-same-length-tables-common-extension-unique-if-clean:*
assumes *clean*: $\neg$ *hash-map-output-collision u*
and *bind1*: merkle-root-binds-table *rt* *table* *s1*
and *bind2*: merkle-root-binds-table *rt* *table'* *s2*
and *ext1*: *s1* $\leq$ *u*
and *ext2*: *s2* $\leq$ *u*

```

    and same-len: length table = length table'
    and nonempty: table ≠ []
  shows table = table'
proof (rule ccontr)
  assume table ≠ table'
  then have hash-map-output-collision u
    by (rule merkle-root-binds-same-length-tables-common-extension-collision
        [OF bind1 bind2 ext1 ext2 same-len nonempty])
  then show False
    using clean by contradiction
qed

lemma merkle-root-binds-same-length-tables-compatible-states-unique-if-clean-merge:
  assumes compatible: hash-maps-compatible s1 s2
    and clean: ¬ hash-map-output-collision (hash-state-merge s1 s2)
    and bind1: merkle-root-binds-table rt table s1
    and bind2: merkle-root-binds-table rt table' s2
    and same-len: length table = length table'
    and nonempty: table ≠ []
  shows table = table'
proof -
  have ext1: s1 ≤ hash-state-merge s1 s2
    by (rule hash-state-merge-extends-left-if-compatible[OF compatible])
  have ext2: s2 ≤ hash-state-merge s1 s2
    by (rule hash-state-merge-extends-right)
  show ?thesis
    by (rule merkle-root-binds-same-length-tables-common-extension-unique-if-clean
        [OF clean bind1 bind2 ext1 ext2 same-len nonempty])
qed

lemma merkle-root-binds-same-length-tables-merkle-compatible-unique-if-clean-merge:
  assumes compatible: merkle-hash-maps-compatible s1 s2
    and clean:
      ¬ hash-map-output-collision (merkle-hash-state-merge s1 s2)
    and bind1: merkle-root-binds-table rt table s1
    and bind2: merkle-root-binds-table rt table' s2
    and same-len: length table = length table'
    and nonempty: table ≠ []
  shows table = table'
proof (rule ccontr)
  let ?u = merkle-hash-state-merge s1 s2
  assume neg: table ≠ table'
  have ext1: merkle-hash-extends s1 ?u
    by (rule merkle-hash-state-merge-extends-left-if-compatible[OF compatible])
  have ext2: merkle-hash-extends s2 ?u
    by (rule merkle-hash-state-merge-extends-right)
  have bind1-u: merkle-root-binds-table rt table ?u
    by (rule merkle-root-binds-table-merkle-mono[OF bind1 ext1])
  have bind2-u: merkle-root-binds-table rt table' ?u

```

```

    by (rule merkle-root-binds-table-merkle-mono[OF bind2 ext2])
  have hash-map-output-collision ?u
    by (rule merkle-root-binds-same-length-tables-collision-imp-hash-collision
        [OF bind1-u bind2-u same-len nonempty neq])
  then show False
    using clean by contradiction
qed

```

```

definition state-after-query-chunks :: 'f  $\Rightarrow$  'f list list  $\Rightarrow$  nat  $\Rightarrow$  'f
  where
    state-after-query-chunks st chunks i =
      foldl ( $\lambda$ acc chunk. foldl concat acc chunk) st (take i chunks)

```

lemma concat-append-eq-concat-same-chunk-lengths:

```

assumes len: length xs = length ys
and lens:
   $\bigwedge i. i < \text{length } xs \implies \text{length } (xs ! i) = \text{length } (ys ! i)$ 
and concat-eq: List.concat xs @ trailing = List.concat ys
shows xs = ys  $\wedge$  trailing = []
using len lens concat-eq
proof (induction xs arbitrary: ys trailing)
  case Nil
  then show ?case
    by (cases ys) simp-all
next
  case (Cons x xs)
  then obtain y ys' where ys-eq: ys = y # ys'
  proof (cases ys)
    case Nil
    then show ?thesis
      using Cons.prem1(1) by simp
  next
    case (Cons y ys')
    then show ?thesis
      by (rule that)
  qed

```

```

have head-len: length x = length y
  using Cons.prem1(2)[of 0] unfolding ys-eq by simp
have append-eq:
  x @ (List.concat xs @ trailing) = y @ List.concat ys'
  using Cons.prem1(3) unfolding ys-eq by simp
have head-eq: x = y
  using append-eq head-len
  by (metis append-eq-conv-conj)
have tail-concat-eq:
  List.concat xs @ trailing = List.concat ys'
  using append-eq head-eq by simp
have tail-len: length xs = length ys'
  using Cons.prem1(1) unfolding ys-eq by simp

```

```

have tail-lens:
   $\bigwedge i. i < \text{length } xs \implies \text{length } (xs ! i) = \text{length } (ys' ! i)$ 
proof –
  fix i
  assume i-bound:  $i < \text{length } xs$ 
  then have  $\text{Suc } i < \text{length } (x \# xs)$ 
    by simp
  from Cons.prems(2)[OF this] show
     $\text{length } (xs ! i) = \text{length } (ys' ! i)$ 
    unfolding ys-eq by simp
qed
have  $xs = ys' \wedge \text{trailing} = []$ 
  by (rule Cons.IH[OF tail-len tail-lens tail-concat-eq])
then show ?case
  using head-eq unfolding ys-eq by simp
qed

fun fri-query-transcript-length ::  $\text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat}$ 
  where
    fri-query-transcript-length 0 len = 0
  | fri-query-transcript-length (Suc n) len =
    2 + 2 * floor-log len + fri-query-transcript-length n (len div 2)

definition query-decommitment-transcript-length ::  $\text{nat} \Rightarrow \text{nat}$ 
  where
    query-decommitment-transcript-length idx =
       $\text{length } (\text{powers-scaled } idx) * (1 + \text{floor-log } (\text{clength} * \text{scale}))$ 

definition query-leaf-path-chunk ::  $\text{nat} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$ 
  where
    query-leaf-path-chunk len leaf path chunk  $\longleftrightarrow$ 
       $\text{length } path = \text{floor-log } len \wedge$ 
       $chunk = [leaf] @ path$ 

definition query-decommitment-transcript
  ::  $\text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$ 
  where
    query-decommitment-transcript idx leaves paths chunk  $\longleftrightarrow$ 
       $\text{length } leaves = \text{length } (\text{powers-scaled } idx) \wedge$ 
       $\text{length } paths = \text{length } (\text{powers-scaled } idx) \wedge$ 
       $chunk = \text{List.concat}$ 
        ( $\text{map } (\lambda(leaf, path). [leaf] @ path) (\text{zip } leaves paths)$ )  $\wedge$ 
       $(\forall path \in \text{set } paths. \text{length } path = \text{floor-log } (\text{clength} * \text{scale}))$ 

lemma query-decommitment-transcript-length:
  assumes query-decommitment-transcript idx leaves paths chunk
  shows  $\text{length } chunk = \text{query-decommitment-transcript-length } idx$ 
proof –

```

```

have leaves-len: length leaves = length (powers-scaled idx)
  using assms unfolding query-decommitment-transcript-def by simp
have paths-len: length paths = length (powers-scaled idx)
  using assms unfolding query-decommitment-transcript-def by simp
have leaves-paths-len: length leaves = length paths
  using leaves-len paths-len by simp
have map2-eq:
  map2 ( $\lambda$ leaf path. [leaf] @ path) leaves paths =
    map2 ( $\lambda$ leaf path. leaf # path) leaves paths
proof (induction leaves arbitrary: paths)
  case Nil
  then show ?case
    by (cases paths) simp-all
next
  case (Cons leaf leaves)
  then show ?case
    by (cases paths) simp-all
qed
have chunk-eq:
  chunk =
    List.concat (map2 ( $\lambda$ leaf path. leaf # path) leaves paths)
  using assms map2-eq unfolding query-decommitment-transcript-def
  by simp
have path-lens:
   $\bigwedge$ path. path  $\in$  set paths  $\implies$ 
    length path = floor-log (clength * scale)
  using assms unfolding query-decommitment-transcript-def by blast
have concat-len:
  length (List.concat (map2 ( $\lambda$ leaf path. leaf # path)
    leaves paths)) =
    length leaves * (1 + floor-log (clength * scale))
  using leaves-paths-len path-lens
proof (induction leaves arbitrary: paths)
  case Nil
  then show ?case
    by (cases paths) simp-all
next
  case (Cons leaf leaves)
  then obtain path paths' where paths-eq: paths = path # paths'
    by (cases paths) auto
  have len-rest: length leaves = length paths'
    using Cons.prem1 unfolding paths-eq by simp
  have path-len: length path = floor-log (clength * scale)
    using Cons.prem2 unfolding paths-eq by simp
  have paths'-len:
     $\bigwedge$ p. p  $\in$  set paths'  $\implies$ 
      length p = floor-log (clength * scale)
    using Cons.prem2 unfolding paths-eq by simp
  have rest:

```

```

    length (List.concat (map2 (λleaf path. leaf # path)
      leaves paths')) =
    length leaves * (1 + floor-log (clength * scale))
  by (rule Cons.IH[OF len-rest paths'-lens])
show ?case
  unfolding paths-eq using path-len rest by simp
qed
have length chunk =
  length leaves * (1 + floor-log (clength * scale))
  unfolding chunk-eq by (rule concat-len)
also have ... =
  length (powers-scaled idx) * (1 + floor-log (clength * scale))
  using leaves-len by simp
finally show ?thesis
  unfolding query-decommitment-transcript-length-def .
qed

definition fri-layer-opening-chunk
  :: nat ⇒ 'f ⇒ 'f list ⇒ 'f ⇒ 'f list ⇒ 'f list ⇒ bool
where
  fri-layer-opening-chunk len xp xp-path xn xn-path chunk ⟷
    length xp-path = floor-log len ∧
    length xn-path = floor-log len ∧
    chunk = [xp] @ xp-path @ [xn] @ xn-path

lemma fri-layer-opening-chunk-length:
  assumes fri-layer-opening-chunk len xp xp-path xn xn-path chunk
  shows length chunk = 2 + 2 * floor-log len
  using assms unfolding fri-layer-opening-chunk-def by simp

fun fri-layers-transcript-length :: nat ⇒ nat ⇒ nat
  where
    fri-layers-transcript-length 0 len = 0
  | fri-layers-transcript-length (Suc n) len =
    2 + 2 * floor-log len + fri-layers-transcript-length n (len div 2)

fun fri-layer-lengths :: nat ⇒ nat ⇒ nat list
  where
    fri-layer-lengths 0 len = []
  | fri-layer-lengths (Suc n) len = len # fri-layer-lengths n (len div 2)

fun fri-layer-indices :: nat ⇒ nat ⇒ nat ⇒ nat list
  where
    fri-layer-indices 0 idx len = []
  | fri-layer-indices (Suc n) idx len =
    idx # fri-layer-indices n (idx mod (len div 2)) (len div 2)

definition fri-sibling-index :: nat ⇒ nat ⇒ nat
  where fri-sibling-index len idx = (idx + len div 2) mod len

```

definition *fri-fold-denominator* :: 'f $\Rightarrow$ nat $\Rightarrow$ 'f
where *fri-fold-denominator* x pw = 2 * x $\wedge$ pw

definition *fri-fold-value* :: 'f $\Rightarrow$ 'f $\Rightarrow$ 'f $\Rightarrow$ 'f $\Rightarrow$ 'f
where
fri-fold-value b xp xn denom =
 (xp + xn) div 2 + b * ((xp - xn) div denom)

lemma *length-fri-layer-lengths[simp]*:
length (*fri-layer-lengths* n len) = n
by (*induction* n *arbitrary*: len) *simp-all*

lemma *length-fri-layer-indices[simp]*:
length (*fri-layer-indices* n idx len) = n
by (*induction* n *arbitrary*: idx len) *simp-all*

definition *fri-layers-transcript*
 :: nat $\Rightarrow$ nat $\Rightarrow$ 'f list list $\Rightarrow$ 'f list $\Rightarrow$ bool
where
fri-layers-transcript n len layer-chunks chunk $\longleftrightarrow$
length layer-chunks = n $\wedge$
 chunk = List.concat layer-chunks $\wedge$
 ($\forall$ (layer-len, layer-chunk) $\in$ set (zip (*fri-layer-lengths* n len) layer-chunks).
 $\exists$ xp xp-path xn xn-path.
fri-layer-opening-chunk layer-len xp xp-path xn xn-path layer-chunk)

lemma *fri-layers-transcript-length*:
assumes *fri-layers-transcript* n len layer-chunks chunk
shows *length* chunk = *fri-layers-transcript-length* n len
using *assms*

proof (*induction* n *arbitrary*: len layer-chunks chunk)
case 0
then show ?case
unfolding *fri-layers-transcript-def* **by** *simp*

next
case (Suc n)
from Suc.premis **have** len-chunks:
length layer-chunks = Suc n
and chunk-eq: chunk = List.concat layer-chunks
and openings:
 $\forall$ (layer-len, layer-chunk) $\in$
 set (zip (*fri-layer-lengths* (Suc n) len) layer-chunks).
 $\exists$ xp xp-path xn xn-path.
fri-layer-opening-chunk layer-len xp xp-path xn xn-path
 layer-chunk
unfolding *fri-layers-transcript-def* **by** *simp-all*

obtain first rest **where** chunk-eq: layer-chunks = first # rest
using len-chunks **by** (cases layer-chunks) *auto*

```

from openings obtain xp xp-path xn xn-path where first-open:
  fri-layer-opening-chunk len xp xp-path xn xn-path first
  unfolding chunks-eq by auto
have rest-layers:
  fri-layers-transcript n (len div 2) rest (List.concat rest)
  unfolding fri-layers-transcript-def
proof (intro conjI)
  show length rest = n
    using len-chunks unfolding chunks-eq by simp
next
  show List.concat rest = List.concat rest
    by simp
next
  show  $\forall (layer-len, layer-chunk) \in$ 
    set (zip (fri-layer-lengths n (len div 2)) rest).
     $\exists xp xp-path xn xn-path.$ 
    fri-layer-opening-chunk layer-len xp xp-path xn xn-path
    layer-chunk
    using openings unfolding chunks-eq by auto
qed
have length chunk = length first + length (List.concat rest)
  unfolding chunk-eq chunks-eq by simp
also have ... =
  (2 + 2 * floor-log len) +
  fri-layers-transcript-length n (len div 2)
  using fri-layer-opening-chunk-length[OF first-open]
  Suc.IH[OF rest-layers] by simp
finally show ?case
  by simp
qed

lemma fri-layers-transcript-nth-opening:
  assumes layers: fri-layers-transcript n len layer-chunks chunk
  and j-bound: j < n
  obtains xp xp-path xn xn-path
  where fri-layer-opening-chunk (fri-layer-lengths n len ! j)
    xp xp-path xn xn-path (layer-chunks ! j)
proof –
  have len-chunks: length layer-chunks = n
    using layers unfolding fri-layers-transcript-def by simp
  have pair-in:
    (fri-layer-lengths n len ! j, layer-chunks ! j)  $\in$ 
    set (zip (fri-layer-lengths n len) layer-chunks)
  proof –
  have j-zip:
    j < length (zip (fri-layer-lengths n len) layer-chunks)
    using j-bound len-chunks by simp
  have zip (fri-layer-lengths n len) layer-chunks ! j =
    (fri-layer-lengths n len ! j, layer-chunks ! j)

```



```

    using j-bound len-chunks by simp
  then show ?thesis
    using nth-mem[OF j-zip] by metis
qed
then obtain xp xp-path xn xn-path where
  fri-layer-opening-chunk (fri-layer-lengths n len ! j)
  xp xp-path xn xn-path (layer-chunks ! j)
  using layers unfolding fri-layers-transcript-def by blast
then show ?thesis
  by (rule that)
qed

definition verifier-query-round-transcript-length
  :: nat  $\Rightarrow$  'f list  $\Rightarrow$  'f list  $\Rightarrow$  nat
where
  verifier-query-round-transcript-length idx f-fri-roots composition-fri-roots =
    query-decommitment-transcript-length idx +
    fri-layers-transcript-length (length f-fri-roots) (clength * scale) +
    fri-layers-transcript-length (length composition-fri-roots) (clength * scale)

definition verifier-query-round-chunk
  :: nat  $\Rightarrow$  'f list  $\Rightarrow$  'f list  $\Rightarrow$  'f list  $\Rightarrow$  bool
where
  verifier-query-round-chunk idx f-fri-roots composition-fri-roots chunk  $\longleftrightarrow$ 
    ( $\exists$  query-chunk trace-fri-chunk composition-fri-chunk
      leaves paths trace-layer-chunks composition-layer-chunks.
      chunk = query-chunk @ trace-fri-chunk @ composition-fri-chunk  $\wedge$ 
      query-decommitment-transcript idx leaves paths query-chunk  $\wedge$ 
      fri-layers-transcript (length f-fri-roots) (clength * scale)
      trace-layer-chunks trace-fri-chunk  $\wedge$ 
      fri-layers-transcript (length composition-fri-roots) (clength * scale)
      composition-layer-chunks composition-fri-chunk)

lemma verifier-query-round-chunk-length:
assumes verifier-query-round-chunk idx f-fri-roots composition-fri-roots chunk
shows
  length chunk =
    verifier-query-round-transcript-length idx f-fri-roots
    composition-fri-roots
proof -
from assms obtain query-chunk trace-fri-chunk composition-fri-chunk
  leaves paths trace-layer-chunks composition-layer-chunks where
  chunk-eq: chunk = query-chunk @ trace-fri-chunk @ composition-fri-chunk
and query:
    query-decommitment-transcript idx leaves paths query-chunk
and trace:
    fri-layers-transcript (length f-fri-roots) (clength * scale)
    trace-layer-chunks trace-fri-chunk
and composition:

```

```

    fri-layers-transcript (length composition-fri-roots) (clength * scale)
    composition-layer-chunks composition-fri-chunk
unfolding verifier-query-round-chunk-def by blast
show ?thesis
unfolding chunk-eq verifier-query-round-transcript-length-def
using query-decommitment-transcript-length[OF query]
    fri-layers-transcript-length[OF trace]
    fri-layers-transcript-length[OF composition]
by simp
qed

definition query-round-fri-layer-transcripts
:: nat  $\Rightarrow$  'f list  $\Rightarrow$  'f list  $\Rightarrow$  'f list  $\Rightarrow$ 
    'f list list  $\Rightarrow$  'f list list  $\Rightarrow$  bool
where
    query-round-fri-layer-transcripts idx trace-roots composition-roots
    chunk trace-layer-chunks composition-layer-chunks  $\longleftrightarrow$ 
    ( $\exists$  query-chunk trace-fri-chunk composition-fri-chunk leaves paths.
    chunk = query-chunk @ trace-fri-chunk @ composition-fri-chunk  $\wedge$ 
    query-decommitment-transcript idx leaves paths query-chunk  $\wedge$ 
    fri-layers-transcript (length trace-roots) (clength * scale)
    trace-layer-chunks trace-fri-chunk  $\wedge$ 
    fri-layers-transcript (length composition-roots) (clength * scale)
    composition-layer-chunks composition-fri-chunk)

lemma verifier-query-round-chunk-fri-layer-transcriptsE:
assumes verifier-query-round-chunk idx trace-roots composition-roots chunk
obtains trace-layer-chunks composition-layer-chunks
where query-round-fri-layer-transcripts idx trace-roots composition-roots
    chunk trace-layer-chunks composition-layer-chunks
using assms
unfolding verifier-query-round-chunk-def query-round-fri-layer-transcripts-def
by blast

definition verifier-query-indices-derived
:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$ 
    'f  $\Rightarrow$  'f list  $\Rightarrow$  'f list  $\Rightarrow$  'f list  $\Rightarrow$  nat list  $\Rightarrow$  bool
where
    verifier-query-indices-derived s out query-start-state rest
    f-fri-roots composition-fri-roots query-idxs  $\longleftrightarrow$ 
    ( $\exists$  result final-state raw-idxs query-chunks trailing.
    out = Some (result, final-state)  $\wedge$ 
    length raw-idxs = rounds  $\wedge$ 
    query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs  $\wedge$ 
    length query-chunks = rounds  $\wedge$ 
    List.concat query-chunks @ trailing = rest  $\wedge$ 
    ( $\forall i < rounds$ .
    verifier-query-round-chunk (query-idxs ! i)

```

$f\text{-fri-roots } \text{composition-fri-roots } (\text{query-chunks } ! i)) \wedge$
 $(\forall i < \text{rounds.}$
 $\text{fmlookup } (\text{HashMap } \text{final-state})$
 $(\text{QueryIndexChallenge } (\text{PQueryCounter } s + i)$
 $(\text{state-after-query-chunks } \text{query-start-state } \text{query-chunks } i)) =$
 $\text{Some } (\text{raw-idxs } ! i)) \wedge$
 $(\forall \text{idx} \in \text{set } \text{query-idxs. } \text{idx} < \text{clength} * \text{scale}))$

definition *accepted-transcript-shape*

$:: (\text{'f}, \text{'a}) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{protocol-channel-scheme}) \text{option} \Rightarrow$
 $\text{'f list} \Rightarrow \text{nat list} \Rightarrow \text{bool}$

where

$\text{accepted-transcript-shape } s \text{ out alphas } \text{query-idxs} \longleftrightarrow$
 $(\exists \text{result } \text{final-state } \text{fr } f\text{-fri-roots } f\text{-final } \text{dg } \text{composition-fri-roots } \text{final } \text{rest.}$
 $\text{out} = \text{Some } (\text{result}, \text{final-state}) \wedge$
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final } \text{alphas } \text{dg}$
 $\text{composition-fri-roots } \text{final } \text{rest} \wedge$
 $\text{verifier-query-indices-derived } s \text{ out}$
 $(\text{verifier-header-state } s \text{ fr } f\text{-fri-roots } f\text{-final } \text{alphas } \text{dg}$
 $\text{composition-fri-roots } \text{final})$
 $\text{rest } f\text{-fri-roots } \text{composition-fri-roots } \text{query-idxs})$

lemma *accepted-transcript-shape-query-chunksE*:

assumes *shape*: $\text{accepted-transcript-shape } s \text{ out alphas } \text{query-idxs}$

obtains $\text{result } \text{final-state } \text{fr } f\text{-fri-roots } f\text{-final } \text{dg}$
 $\text{composition-fri-roots } \text{final } \text{rest } \text{raw-idxs } \text{query-chunks } \text{trailing}$

where

$\text{out} = \text{Some } (\text{result}, \text{final-state})$
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final } \text{alphas } \text{dg}$
 $\text{composition-fri-roots } \text{final } \text{rest}$
 $\text{length } \text{raw-idxs} = \text{rounds}$
 $\text{query-idxs} = \text{map } (\lambda \text{raw. } \text{index } (\text{to-nat } \text{raw})) \text{ raw-idxs}$
 $\text{length } \text{query-chunks} = \text{rounds}$
 $\text{List.concat } \text{query-chunks } @ \text{trailing} = \text{rest}$
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{verifier-query-round-chunk } (\text{query-idxs } ! i)$
 $f\text{-fri-roots } \text{composition-fri-roots } (\text{query-chunks } ! i)$
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{fmlookup } (\text{HashMap } \text{final-state})$
 $(\text{QueryIndexChallenge } (\text{PQueryCounter } s + i)$
 $(\text{state-after-query-chunks}$
 $(\text{verifier-header-state } s \text{ fr } f\text{-fri-roots } f\text{-final } \text{alphas } \text{dg}$
 $\text{composition-fri-roots } \text{final})$
 $\text{query-chunks } i)) =$
 $\text{Some } (\text{raw-idxs } ! i)$
 $\forall \text{idx} \in \text{set } \text{query-idxs. } \text{idx} < \text{clength} * \text{scale}$

proof —

from *shape* **obtain** $\text{result } \text{final-state } \text{fr } f\text{-fri-roots } f\text{-final } \text{dg}$

```

    composition-fri-roots final rest where
out-eq: out = Some (result, final-state)
and header:
    verifier-header-transcript s fr f-fri-roots f-final alphas dg
    composition-fri-roots final rest
and query:
    verifier-query-indices-derived s out
    (verifier-header-state s fr f-fri-roots f-final alphas dg
     composition-fri-roots final)
    rest f-fri-roots composition-fri-roots query-idxs
unfolding accepted-transcript-shape-def by blast
from query obtain result' final-state' raw-idxs query-chunks trailing where
out-eq': out = Some (result', final-state')
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-chunks: length query-chunks = rounds
and concat-chunks: List.concat query-chunks @ trailing = rest
and chunk-shape:
    ∧ i. i < rounds ⇒
        verifier-query-round-chunk (query-idxs ! i)
        f-fri-roots composition-fri-roots (query-chunks ! i)
and lookup:
    ∧ i. i < rounds ⇒
        fmlookup (HashMap final-state')
        (QueryIndexChallenge (PQueryCounter s + i)
         (state-after-query-chunks
          (verifier-header-state s fr f-fri-roots f-final alphas dg
           composition-fri-roots final)
           query-chunks i)) =
        Some (raw-idxs ! i)
and idx-bound:
    ∀ idx ∈ set query-idxs. idx < clength * scale
unfolding verifier-query-indices-derived-def by blast
have final-state-eq: final-state' = final-state
using out-eq out-eq' by simp
show ?thesis
by (rule that[OF out-eq header len-raw query-idxs-eq len-chunks
    concat-chunks chunk-shape])
    (use lookup idx-bound final-state-eq in simp-all)
qed

```

```

lemma accepted-transcript-shape-query-chunk-lengthsE:
assumes shape: accepted-transcript-shape s out alphas query-idxs
obtains result final-state fr f-fri-roots f-final dg
    composition-fri-roots final rest raw-idxs query-chunks trailing
where
    out = Some (result, final-state)
    verifier-header-transcript s fr f-fri-roots f-final alphas dg

```

```

    composition-fri-roots final rest
length raw-idxs = rounds
query-idxs = map (λraw. index (to-nat raw)) raw-idxs
length query-chunks = rounds
List.concat query-chunks @ trailing = rest
Λi. i < rounds ⇒
    length (query-chunks ! i) =
        verifier-query-round-transcript-length (query-idxs ! i)
        f-fri-roots composition-fri-roots
Λi. i < rounds ⇒
    fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter s + i)
    (state-after-query-chunks
    (verifier-header-state s fr f-fri-roots f-final alphas dg
    composition-fri-roots final)
    query-chunks i)) =
    Some (raw-idxs ! i)
    ∀ idx ∈ set query-idxs. idx < clength * scale
proof –
from accepted-transcript-shape-query-chunksE[OF shape]
obtain result final-state fr f-fri-roots f-final dg
    composition-fri-roots final rest raw-idxs query-chunks trailing where
    out-eq: out = Some (result, final-state)
and header:
    verifier-header-transcript s fr f-fri-roots f-final alphas dg
    composition-fri-roots final rest
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-chunks: length query-chunks = rounds
and concat-chunks: List.concat query-chunks @ trailing = rest
and chunk-shape:
    Λi. i < rounds ⇒
        verifier-query-round-chunk (query-idxs ! i)
        f-fri-roots composition-fri-roots (query-chunks ! i)
and lookup:
    Λi. i < rounds ⇒
        fmlookup (HashMap final-state)
        (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks
        (verifier-header-state s fr f-fri-roots f-final alphas dg
        composition-fri-roots final)
        query-chunks i)) =
        Some (raw-idxs ! i)
and idx-bound:
    ∀ idx ∈ set query-idxs. idx < clength * scale
by blast
have chunk-len:
    Λi. i < rounds ⇒

```

$\text{length } (\text{query-chunks } ! i) =$
 $\text{verifier-query-round-transcript-length } (\text{query-idxs } ! i)$
 $\text{f-fri-roots composition-fri-roots}$
using $\text{verifier-query-round-chunk-length}[OF \text{ chunk-shape}]$ **by** blast
show $?thesis$
by $(\text{rule that}[OF \text{ out-eq header len-raw query-idxs-eq len-chunks}$
 $\text{concat-chunks chunk-len lookup idx-bound}])$
qed

definition *accepted-with-tables*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow \text{bool}$

where

$\text{accepted-with-tables } s \text{ out trace-table composition-table alphas query-idxs} \longleftrightarrow$
 $\text{accepted-transcript-shape } s \text{ out alphas query-idxs} \wedge$
 $(\forall \text{ idx} \in \text{set query-idxs.}$
 $\text{query-consistent-at trace-table composition-table alphas idx}) \wedge$
 $\text{length trace-table} = \text{clength} * \text{scale} \wedge$
 $\text{length composition-table} = \text{clength} * \text{scale} \wedge$
 $\text{length alphas} = \text{length spec}$

definition *accepted-with-bound-tables*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow \text{bool}$

where

$\text{accepted-with-bound-tables } s \text{ out trace-table composition-table alphas query-idxs} \longleftrightarrow$
 $\text{accepted-with-tables } s \text{ out trace-table composition-table alphas query-idxs} \wedge$
 $(\exists \text{ result final-state fr f-fri-roots f-final dg composition-fri-roots final rest.}$
 $\text{out} = \text{Some } (\text{result}, \text{final-state}) \wedge$
 $\text{verifier-header-transcript } s \text{ fr f-fri-roots f-final alphas dg}$
 $\text{composition-fri-roots final rest} \wedge$
 $\text{merkle-root-binds-table fr trace-table final-state} \wedge$
 $\text{composition-fri-roots} \neq [] \wedge$
 $\text{merkle-root-binds-table } (\text{hd composition-fri-roots}) \text{ composition-table final-state})$

definition *alpha-list-set-hit*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ list set} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{alpha-list-set-hit } s \text{ B out} \longleftrightarrow$
 $(\exists \text{ as query-idxs.}$
 $\text{accepted-transcript-shape } s \text{ out as query-idxs} \wedge \text{as} \in \text{B})$

definition *alpha-header-list-set-hit*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where
 $\alpha\text{-header-list-set-hit } s \ B \ out \longleftrightarrow$
 $(\exists \ as \ query\text{-idxs} \ result \ final\text{-state} \ fr \ f\text{-fri-roots} \ f\text{-final} \ dg$
 $\quad composition\text{-fri-roots} \ final \ rest.$
 $accepted\text{-transcript-shape } s \ out \ as \ query\text{-idxs} \wedge$
 $out = Some \ (result, \ final\text{-state}) \wedge$
 $verifier\text{-header-transcript } s \ fr \ f\text{-fri-roots} \ f\text{-final} \ as \ dg$
 $\quad composition\text{-fri-roots} \ final \ rest \wedge$
 $as \in B \ fr \ f\text{-fri-roots} \ f\text{-final})$

end

end

theory *Soundness-Core*
imports *Soundness-Core-Events*
begin

Compatibility umbrella for core soundness definitions and lemmas.

end

theory *Soundness-Merkle*
imports
 $\quad Soundness-Core$
begin

context *soundness*
begin

A successful verifier execution only checks the sampled authentication paths. Turning those local checks into full trace and composition tables bound to the initial committed roots is a Merkle binding/collision-resistance property. The current random-oracle hash-map model records enough information to replay checked paths, but it does not by itself imply existence or uniqueness of a complete maliciously committed table. We therefore expose the missing part as a bad event over accepted verifier executions. Binding for intermediate FRI layers is not hidden here; it is part of the FRI bad-challenge-set reductions introduced below.

definition *initial-merkle-binding-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(unit \ list \times (f, 'a) \text{ protocol-channel-scheme}) \ option \Rightarrow bool$

where
 $initial\text{-merkle-binding-bad } s \ out \longleftrightarrow$
 $(\exists \ alphas \ query\text{-idxs}.$
 $\quad accepted\text{-transcript-shape } s \ out \ alphas \ query\text{-idxs} \wedge$
 $\neg (\exists \ trace\text{-table} \ composition\text{-table}.$
 $\quad \quad accepted\text{-with-bound-tables } s \ out \ trace\text{-table} \ composition\text{-table})$

alphas query-idxs))

definition *initial-merkle-binding-no-bad* :: (*'f*, *'a*) *protocol-channel-scheme* $\Rightarrow$ *bool*

where

initial-merkle-binding-no-bad *s* $\longleftrightarrow$
 $(\forall \text{out} \in \text{set-dist } (\text{execute verify-monad } s).$
 $\neg \text{initial-merkle-binding-bad } s \text{ out})$

definition *initial-merkle-binding-collision-bad*

:: (*'f*, *'a*) *protocol-channel-scheme* $\Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

initial-merkle-binding-collision-bad *s out* $\longleftrightarrow$
 $\text{accepted out} \wedge$
 $(\exists \text{result final-state rt.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{merkle-root-binding-collision rt final-state})$

definition *merkle-root-binding-collision-bad*

:: (*'f*, *'a*) *protocol-channel-scheme* $\Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

merkle-root-binding-collision-bad *s out* $\longleftrightarrow$
 $\text{accepted out} \wedge$
 $(\exists \text{result final-state rt.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{merkle-root-binding-collision rt final-state})$

definition *hash-map-output-collision-bad*

:: (*'f*, *'a*) *protocol-channel-scheme* $\Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

hash-map-output-collision-bad *s out* $\longleftrightarrow$
 $\text{accepted out} \wedge$
 $(\exists \text{result final-state.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{hash-map-output-collision final-state})$

lemma *initial-merkle-binding-collision-bad-eq-root-collision-bad*:

initial-merkle-binding-collision-bad *s out* =
merkle-root-binding-collision-bad *s out*

unfolding *initial-merkle-binding-collision-bad-def*

merkle-root-binding-collision-bad-def

by *simp*

lemma *merkle-root-binding-collision-bad-imp-accepted*:

assumes *merkle-root-binding-collision-bad* *s out*

shows *accepted out*

using *assms* **unfolding** *merkle-root-binding-collision-bad-def* **by** *simp*

lemma *hash-map-output-collision-bad-imp-accepted*:
assumes *hash-map-output-collision-bad s out*
shows *accepted out*
using *assms* **unfolding** *hash-map-output-collision-bad-def* **by** *simp*

lemma *merkle-root-binding-collision-bad-mono-accepted*:
 $wp\text{-}event\ verify\text{-}monad\ (merkle\text{-}root\text{-}binding\text{-}collision\text{-}bad\ s)\ s \leq$
 $wp\text{-}event\ verify\text{-}monad\ accepted\ s$
by (rule *wp-event-mono*)
(simp add: merkle-root-binding-collision-bad-imp-accepted)

lemma *hash-map-output-collision-bad-mono-accepted*:
 $wp\text{-}event\ verify\text{-}monad\ (hash\text{-}map\text{-}output\text{-}collision\text{-}bad\ s)\ s \leq$
 $wp\text{-}event\ verify\text{-}monad\ accepted\ s$
by (rule *wp-event-mono*)
(simp add: hash-map-output-collision-bad-imp-accepted)

lemma *accepted-inconsistent-merkle-bindings-imp-collision-bad*:
assumes *out: out = Some (result, final-state)*
and *bind1: merkle-root-binds-table rt table final-state*
and *bind2: merkle-root-binds-table rt table' final-state*
and *neg: table $\neq$ table'*
shows *initial-merkle-binding-collision-bad s out*
proof –
have *collision: merkle-root-binding-collision rt final-state*
by (rule *merkle-root-binding-collisionI[OF bind1 bind2 neg]*)
show *?thesis*
proof (*unfold initial-merkle-binding-collision-bad-def accepted-def, intro conjI*)
show $\neg\ Option.is\ none\ out$
using *out* **by** *simp*
show $\exists\ result\ final\ state\ rt.$
 $out = Some\ (result,\ final\ state) \wedge$
 $merkle\text{-}root\text{-}binding\text{-}collision\ rt\ final\ state$
using *out collision* **by** *blast*
qed
qed

lemma *accepted-inconsistent-merkle-bindings-imp-root-collision-bad*:
assumes *out: out = Some (result, final-state)*
and *bind1: merkle-root-binds-table rt table final-state*
and *bind2: merkle-root-binds-table rt table' final-state*
and *neg: table $\neq$ table'*
shows *merkle-root-binding-collision-bad s out*
using *accepted-inconsistent-merkle-bindings-imp-collision-bad*
 $[OF\ out\ bind1\ bind2\ neg]$
unfolding *initial-merkle-binding-collision-bad-eq-root-collision-bad* .

lemma *hash-map-output-collision-badI*:

```

assumes out: out = Some (result, final-state)
and neg: x ≠ y
and x-lookup: fmlookup (HashMap final-state) x = Some z
and y-lookup: fmlookup (HashMap final-state) y = Some z
shows hash-map-output-collision-bad s out
proof –
  have collision: hash-map-output-collision final-state
    by (rule hash-map-output-collisionI[OF neg x-lookup y-lookup])
  show ?thesis
    unfolding hash-map-output-collision-bad-def accepted-def
    using out collision by auto
qed

```

lemma *accepted-inconsistent-same-length-merkle-bindings-imp-hash-collision-bad*:

```

assumes out: out = Some (result, final-state)
and bind1: merkle-root-binds-table rt table final-state
and bind2: merkle-root-binds-table rt table' final-state
and same-len: length table = length table'
and nonempty: table ≠ []
and neg: table ≠ table'
shows hash-map-output-collision-bad s out
proof –
  have collision: hash-map-output-collision final-state
    by (rule merkle-root-binds-same-length-tables-collision-imp-hash-collision
      [OF bind1 bind2 same-len nonempty neg])
  show ?thesis
    unfolding hash-map-output-collision-bad-def accepted-def
    using out collision by auto
qed

```

lemma *initial-merkle-binding-collision-bad-no-collisionD*:

```

assumes ¬ initial-merkle-binding-collision-bad s (Some (result, final-state))
shows ¬ merkle-root-binding-collision rt final-state
using assms unfolding initial-merkle-binding-collision-bad-def accepted-def
by auto

```

lemma *accepted-with-tables-imp-accepted*:

```

assumes accepted-with-tables s out trace-table composition-table alphas query-idxs
shows accepted out
using assms unfolding accepted-with-tables-def accepted-transcript-shape-def
  verifier-query-indices-derived-def
  accepted-def
by auto

```

lemma *accepted-transcript-shape-imp-accepted*:

```

assumes accepted-transcript-shape s out alphas query-idxs
shows accepted out
using assms unfolding accepted-transcript-shape-def verifier-query-indices-derived-def
  accepted-def

```

by *auto*

lemma *accepted-with-tables-imp-accepted-transcript-shape:*

assumes *accepted-with-tables s out trace-table composition-table alphas query-idxs*

shows *accepted-transcript-shape s out alphas query-idxs*

using *assms* **unfolding** *accepted-with-tables-def* **by** *simp*

lemma *accepted-with-tables-shapes:*

assumes *accepted-with-tables s out trace-table composition-table alphas query-idxs*

shows *length trace-table = clength * scale*

and *length composition-table = clength * scale*

and *length alphas = length spec*

and *length query-idxs = rounds*

and $\bigwedge idx. idx \in \text{set query-idxs} \implies idx < \text{clength} * \text{scale}$

using *assms*

unfolding *accepted-with-tables-def accepted-transcript-shape-def*

verifier-query-indices-derived-def

by *auto*

lemma *accepted-transcript-shape-query-shapes:*

assumes *accepted-transcript-shape s out alphas query-idxs*

shows *length query-idxs = rounds*

and $\bigwedge idx. idx \in \text{set query-idxs} \implies idx < \text{clength} * \text{scale}$

using *assms*

unfolding *accepted-transcript-shape-def verifier-query-indices-derived-def*

by *auto*

lemma *verifier-query-indices-derived-query-sample-space:*

assumes *verifier-query-indices-derived s out query-start-state rest*

f-fri-roots composition-fri-roots query-idxs

shows $\bigwedge idx. idx \in \text{set query-idxs} \implies idx \in \text{query-sample-space}$

using *assms index-less-query-sample-space*

unfolding *verifier-query-indices-derived-def query-sample-space-def*

by *auto*

lemma *accepted-transcript-shape-query-sample-space:*

assumes *accepted-transcript-shape s out alphas query-idxs*

shows $\bigwedge idx. idx \in \text{set query-idxs} \implies idx \in \text{query-sample-space}$

proof –

fix *idx*

assume *idx-in: idx ∈ set query-idxs*

obtain *result final-state fr f-fri-roots f-final dg composition-fri-roots final rest*

where *derived:*

verifier-query-indices-derived s out

(verifier-header-state s fr f-fri-roots f-final alphas dg

composition-fri-roots final)

rest f-fri-roots composition-fri-roots query-idxs

using *assms* **unfolding** *accepted-transcript-shape-def* **by** *blast*

show *idx ∈ query-sample-space*

by (rule verifier-query-indices-derived-query-sample-space[OF derived idx-in])
qed

lemma *accepted-with-tables-query-sample-space*:
assumes *accepted-with-tables s out trace-table composition-table alphas query-idxs*
shows $\bigwedge idx. idx \in \text{set query-idxs} \implies idx \in \text{query-sample-space}$
using *accepted-transcript-shape-query-sample-space*
 [OF *accepted-with-tables-imp-accepted-transcript-shape*[OF *assms*]]
by *blast*

lemma *accepted-with-bound-tables-query-sample-space*:
assumes *accepted-with-bound-tables s out trace-table composition-table alphas query-idxs*
shows $\bigwedge idx. idx \in \text{set query-idxs} \implies idx \in \text{query-sample-space}$
proof –
have *tables: accepted-with-tables s out trace-table composition-table alphas query-idxs*
using *assms unfolding accepted-with-bound-tables-def* **by** *simp*
show $\bigwedge idx. idx \in \text{set query-idxs} \implies idx \in \text{query-sample-space}$
by (rule *accepted-with-tables-query-sample-space*[OF *tables*])
 qed

lemma *accepted-with-tables-authenticated-roots*:
assumes *accepted-with-bound-tables s out trace-table composition-table alphas query-idxs*
obtains *result final-state fr composition-root*
where *out = Some (result, final-state)*
and *merkle-root-binds-table fr trace-table final-state*
and *merkle-root-binds-table composition-root composition-table final-state*
using *assms unfolding accepted-with-bound-tables-def* **by** *auto*

lemma *accepted-with-bound-tables-imp-accepted-with-tables*:
assumes *accepted-with-bound-tables s out trace-table composition-table alphas query-idxs*
shows *accepted-with-tables s out trace-table composition-table alphas query-idxs*
using *assms unfolding accepted-with-bound-tables-def* **by** *simp*

lemma *accepted-with-bound-tables-shapes*:
assumes *accepted-with-bound-tables s out trace-table composition-table alphas query-idxs*
shows *length trace-table = clength * scale*
and *length composition-table = clength * scale*
and *length alphas = length spec*
and *length query-idxs = rounds*
and $\bigwedge idx. idx \in \text{set query-idxs} \implies idx < \text{clength} * \text{scale}$
using *accepted-with-tables-shapes*
 [OF *accepted-with-bound-tables-imp-accepted-with-tables*[OF *assms*]]
by *blast+*

lemma *accepted-with-bound-tables-imp-accepted*:
assumes *accepted-with-bound-tables s out trace-table composition-table alphas query-idxs*
shows *accepted out*
using *accepted-with-bound-tables-imp-accepted-with-tables*[OF *assms*]
by (rule *accepted-with-tables-imp-accepted*)

definition *supported-hash-output-collision-possible*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{supported-hash-output-collision-possible } s \longleftrightarrow$

$(\exists \text{ out} \in \text{set-dist } (\text{execute verify-monad } s).$

$\text{hash-map-output-collision-bad } s \text{ out})$

lemma *accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible:*

assumes *outcome:*

$\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute verify-monad } s)$

and *bound:*

$\text{accepted-with-bound-tables } s \text{ (Some } (\text{result}, \text{final-state}))$

$\text{trace-table composition-table as query-idxs}$

and *collision:* $\text{hash-map-output-collision final-state}$

shows *supported-hash-output-collision-possible* s

proof –

have *accepted:* $\text{accepted (Some (result, final-state))}$

by $(\text{rule accepted-with-bound-tables-imp-accepted[OF bound]})$

have *bad:*

$\text{hash-map-output-collision-bad } s \text{ (Some (result, final-state))}$

unfolding *hash-map-output-collision-bad-def*

using *accepted collision* **by** *blast*

show *?thesis*

unfolding *supported-hash-output-collision-possible-def*

using *outcome bad* **by** *blast*

qed

definition *alpha-bad-set-hit*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$(f \text{ list} \Rightarrow f \text{ list set}) \Rightarrow$

$(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{alpha-bad-set-hit } s \text{ bad-sets out} \longleftrightarrow$

$(\exists \text{ trace-table composition-table as query-idxs.}$

$\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as query-idxs} \wedge$

$\text{as} \in \text{bad-sets trace-table})$

lemma *alpha-bad-set-hit-imp-alpha-list-set-hit:*

assumes *hit:* $\text{alpha-bad-set-hit } s \text{ bad-sets out}$

and *envelope:* $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq B$

shows *alpha-list-set-hit* $s \text{ B out}$

proof –

from *hit* **obtain** $\text{trace-table composition-table as query-idxs}$ **where**

bound:

$\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as query-idxs}$

and *as-bad:* $\text{as} \in \text{bad-sets trace-table}$

unfolding *alpha-bad-set-hit-def* **by** *blast*

have *shape:*

accepted-transcript-shape s out as query-idxs
using *bound unfolding accepted-with-bound-tables-def*
accepted-with-tables-def **by** *simp*
have *as* $\in B$
using *as-bad envelope[of trace-table]* **by** *auto*
then show *?thesis*
unfolding *alpha-list-set-hit-def* **using** *shape* **by** *blast*
qed

definition *alpha-header-trace-table-candidates*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list set}$

where

alpha-header-trace-table-candidates s fr f-fri-roots f-final =
 $\{ \text{trace-table.}$
 $\exists \text{ out composition-table as query-idxs dg composition-fri-roots final rest.}$
 $\text{accepted-with-bound-tables s out trace-table composition-table}$
 $\text{as query-idxs} \wedge$
 $\text{verifier-header-transcript s fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final rest} \}$

definition *alpha-header-union-bad-sets*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list set}$

where

alpha-header-union-bad-sets s bad-sets fr f-fri-roots f-final =
 $\{ \text{as} \in \text{alpha-space.}$
 $\exists \text{ trace-table} \in$
 $\text{alpha-header-trace-table-candidates s fr f-fri-roots f-final.}$
 $\text{as} \in \text{bad-sets trace-table} \}$

definition *alpha-header-supported-trace-table-candidates*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list set}$

where

alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final =
 $\{ \text{trace-table.}$
 $\exists \text{ out composition-table as query-idxs dg composition-fri-roots final rest.}$
 $\text{out} \in \text{set-dist (execute verify-monad s)} \wedge$
 $\text{accepted-with-bound-tables s out trace-table composition-table}$
 $\text{as query-idxs} \wedge$
 $\text{verifier-header-transcript s fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final rest} \}$

definition *alpha-header-supported-union-bad-sets*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list set}$

where

$\alpha\text{-header-supported-union-bad-sets } s \text{ bad-sets fr } f\text{-fri-roots } f\text{-final} =$
 $\{as \in \alpha\text{-space}.$
 $\exists \text{ trace-table} \in$
 $\alpha\text{-header-supported-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final}.$
 $as \in \text{bad-sets trace-table}\}$

lemma $\alpha\text{-header-union-bad-sets-subset-}\alpha\text{-space}:$

$\alpha\text{-header-union-bad-sets } s \text{ bad-sets fr } f\text{-fri-roots } f\text{-final} \subseteq$
 $\alpha\text{-space}$

unfolding $\alpha\text{-header-union-bad-sets-def}$ **by** *auto*

lemma $\alpha\text{-header-supported-union-bad-sets-subset-}\alpha\text{-space}:$

$\alpha\text{-header-supported-union-bad-sets } s \text{ bad-sets fr } f\text{-fri-roots } f\text{-final}$
 $\subseteq \alpha\text{-space}$

unfolding $\alpha\text{-header-supported-union-bad-sets-def}$ **by** *auto*

lemma $\alpha\text{-header-union-bad-sets-fraction-bound-if-unique-candidate}:$

fixes $C :: \text{prob}$

assumes *unique*:

$\exists \text{ trace-table}.$

$\alpha\text{-header-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final} \subseteq$
 $\{\text{trace-table}\}$

and *subset*: $\bigwedge \text{trace-table}.$ $\text{bad-sets trace-table} \subseteq \alpha\text{-space}$

and *bound*:

$\bigwedge \text{trace-table}.$

$\text{nnreal } (\text{card } (\text{bad-sets trace-table})) / \text{nnreal } (\text{card } \alpha\text{-space}) \leq C$

shows

nnreal

$(\text{card } (\alpha\text{-header-union-bad-sets } s \text{ bad-sets fr } f\text{-fri-roots } f\text{-final})) /$
 $\text{nnreal } (\text{card } \alpha\text{-space}) \leq C$

proof –

from *unique* **obtain** *trace-table* **where** *candidates*:

$\alpha\text{-header-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final} \subseteq$
 $\{\text{trace-table}\}$

by *blast*

have *union-subset*:

$\alpha\text{-header-union-bad-sets } s \text{ bad-sets fr } f\text{-fri-roots } f\text{-final} \subseteq$
 $\text{bad-sets trace-table}$

unfolding $\alpha\text{-header-union-bad-sets-def}$

using *candidates* **by** *auto*

have *finite-bad*: *finite* (*bad-sets trace-table*)

by (*rule finite-subset*[*OF subset finite-alpha-space*])

have *card-le*:

$\text{card } (\alpha\text{-header-union-bad-sets } s \text{ bad-sets fr } f\text{-fri-roots } f\text{-final}) \leq$
 $\text{card } (\text{bad-sets trace-table})$

by (*rule card-mono*[*OF finite-bad union-subset*])

have *nnreal*

```

      (card (alpha-header-union-bad-sets s bad-sets fr f-fri-roots f-final)) /
      nnreal (card alpha-space) ≤
      nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)
    by (rule nnreal-nat-divide-right-mono[OF card-le])
  also have ... ≤ C
    by (rule bound)
  finally show ?thesis .
qed

lemma alpha-header-supported-union-bad-sets-fraction-bound-if-unique-candidate:
  fixes C :: prob
  assumes unique:
    ∃ trace-table.
      alpha-header-supported-trace-table-candidates s fr f-fri-roots
      f-final ⊆ {trace-table}
  and subset:  $\bigwedge$  trace-table. bad-sets trace-table ⊆ alpha-space
  and bound:
     $\bigwedge$  trace-table.
      nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space) ≤ C
  shows
    nnreal
      (card
        (alpha-header-supported-union-bad-sets s bad-sets fr f-fri-roots
          f-final)) /
      nnreal (card alpha-space) ≤ C
proof -
  from unique obtain trace-table where candidates:
    alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
    ⊆ {trace-table}
  by blast
  have union-subset:
    alpha-header-supported-union-bad-sets s bad-sets fr f-fri-roots f-final
    ⊆ bad-sets trace-table
  unfolding alpha-header-supported-union-bad-sets-def
  using candidates by auto
  have finite-bad: finite (bad-sets trace-table)
  by (rule finite-subset[OF subset finite-alpha-space])
  have card-le:
    card
      (alpha-header-supported-union-bad-sets s bad-sets fr f-fri-roots
        f-final) ≤ card (bad-sets trace-table)
  by (rule card-mono[OF finite-bad union-subset])
  have nnreal
    (card
      (alpha-header-supported-union-bad-sets s bad-sets fr f-fri-roots
        f-final)) /
    nnreal (card alpha-space) ≤
    nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])

```


also have $\dots \leq C$
 by (rule bound)
 finally show ?thesis .
 qed

lemma *composition-trace-alpha-header-union-fraction-bound-if-unique-candidate:*

assumes *unique:*
 $\exists$ *trace-table.*
 $\alpha\text{-header-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\}$
shows
 $\text{nnreal } (\text{card } (\alpha\text{-header-union-bad-sets } s \text{ composition-trace-bad-alpha-space fr } f\text{-fri-roots } f\text{-final})) / \text{nnreal } (\text{card } \alpha\text{-space}) \leq \text{composition-error-bound}$
by (rule *alpha-header-union-bad-sets-fraction-bound-if-unique-candidate* [OF *unique composition-trace-bad-alpha-space-subset-alpha-space composition-trace-bad-alpha-space-fraction-bound-alpha-space*])

lemma *composition-trace-alpha-header-supported-union-fraction-bound-if-unique-candidate:*

assumes *unique:*
 $\exists$ *trace-table.*
 $\alpha\text{-header-supported-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\}$
shows
 $\text{nnreal } (\text{card } (\alpha\text{-header-supported-union-bad-sets } s \text{ composition-trace-bad-alpha-space fr } f\text{-fri-roots } f\text{-final})) / \text{nnreal } (\text{card } \alpha\text{-space}) \leq \text{composition-error-bound}$
by (rule *alpha-header-supported-union-bad-sets-fraction-bound-if-unique-candidate* [OF *unique composition-trace-bad-alpha-space-subset-alpha-space composition-trace-bad-alpha-space-fraction-bound-alpha-space*])

lemma *alpha-bad-set-hit-imp-alpha-header-union-bad-set-hit:*

assumes *hit:* $\alpha\text{-bad-set-hit } s \text{ bad-sets out}$
and *subset:* $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \alpha\text{-space}$
shows
 $\alpha\text{-header-list-set-hit } s$
 $(\alpha\text{-header-union-bad-sets } s \text{ bad-sets}) \text{ out}$

proof –

from *hit* **obtain** *trace-table composition-table as query-idxs* **where**
bound:
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as query-idxs}$
and *as-bad:* $as \in \text{bad-sets trace-table}$
unfolding *alpha-bad-set-hit-def* **by** *blast*
from *bound* **obtain** *result final-state fr f-fri-roots f-final dg*

```

    composition-fri-roots final rest where
    out-eq: out = Some (result, final-state)
and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
unfolding accepted-with-bound-tables-def by blast
have shape:
    accepted-transcript-shape s out as query-idxs
using accepted-with-bound-tables-imp-accepted-with-tables[OF bound]
    accepted-with-tables-imp-accepted-transcript-shape
by blast
have candidate:
    trace-table ∈
    alpha-header-trace-table-candidates s fr f-fri-roots f-final
unfolding alpha-header-trace-table-candidates-def
using bound header by blast
have as-union:
    as ∈ alpha-header-union-bad-sets s bad-sets fr f-fri-roots f-final
unfolding alpha-header-union-bad-sets-def
using as-bad candidate subset[of trace-table] by auto
show ?thesis
    unfolding alpha-header-list-set-hit-def
    using shape out-eq header as-union by blast
qed

lemma alpha-bad-set-hit-imp-alpha-header-supported-union-bad-set-hit:
assumes outcome: out ∈ set-dist (execute verify-monad s)
and hit: alpha-bad-set-hit s bad-sets out
and subset:  $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$ 
shows
    alpha-header-list-set-hit s
    (alpha-header-supported-union-bad-sets s bad-sets) out
proof –
from hit obtain trace-table composition-table as query-idxs where
    bound:
    accepted-with-bound-tables s out trace-table composition-table
    as query-idxs
and as-bad: as ∈ bad-sets trace-table
unfolding alpha-bad-set-hit-def by blast
from bound obtain result final-state fr f-fri-roots f-final dg
    composition-fri-roots final rest where
    out-eq: out = Some (result, final-state)
and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
unfolding accepted-with-bound-tables-def by blast
have shape:
    accepted-transcript-shape s out as query-idxs
using accepted-with-bound-tables-imp-accepted-with-tables[OF bound]

```

```

    accepted-with-tables-imp-accepted-transcript-shape
  by blast
have candidate:
  trace-table ∈
    alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
  unfolding alpha-header-supported-trace-table-candidates-def
  using outcome bound header by blast
have as-union:
  as ∈
    alpha-header-supported-union-bad-sets s bad-sets fr f-fri-roots f-final
  unfolding alpha-header-supported-union-bad-sets-def
  using as-bad candidate subset[of trace-table] by auto
show ?thesis
  unfolding alpha-header-list-set-hit-def
  using shape out-eq header as-union by blast
qed

```

definition *alpha-challenge-freshness-assumption*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
alpha-challenge-freshness-assumption $s \longleftrightarrow$
 $(\forall \text{ bad-sets.}$
 $(\forall \text{ trace-table. bad-sets trace-table} \subseteq \text{alpha-space}) \longrightarrow$
 $(\forall \text{ trace-table.}$
 $\text{nnreal (card (bad-sets trace-table)) /}$
 $\text{nnreal (card alpha-space)} \leq \text{composition-error-bound}) \longrightarrow$
 $\text{wp-event verify-monad (alpha-bad-set-hit s bad-sets) s} \leq$
 $\text{composition-error-bound})$

The protocol currently represents all Fiat-Shamir challenges through the same random-oracle hash map, without protocol-level domain-separated labels for alpha, query-index, and FRI challenges. The following predicates below state the precise freshness obligations needed for each challenge family. Phase 4E bundles them into one random-oracle model interface rather than adding labels to the protocol in this refactor. Merkle hash uses are not represented as freshness obligations here; their binding role remains the explicit *initial-merkle-binding-no-bad* premise.

lemma *initial-merkle-binding-no-badE*:
assumes *bind*: *initial-merkle-binding-no-bad* s
and *outcome*:
 $\text{Some (result, final-state)} \in \text{set-dist (execute verify-monad s)}$
and *shape*: *accepted-transcript-shape* s $(\text{Some (result, final-state)})$ *alphas query-idxs*
obtains *trace-table composition-table*
where *accepted-with-bound-tables* s $(\text{Some (result, final-state)})$
 $\text{trace-table composition-table alphas query-idxs}$
using *assms*
unfolding *initial-merkle-binding-no-bad-def initial-merkle-binding-bad-def*
by *blast*

lemma *initial-merkle-binding-bad-bound*:
assumes *bind: initial-merkle-binding-no-bad s*
shows *wp-event verify-monad (initial-merkle-binding-bad s) s ≤ 0*
proof –
have *wp-event verify-monad (initial-merkle-binding-bad s) s ≤*
wp-event verify-monad
(λ- :: (unit list × ('f, 'a) protocol-channel-scheme) option. False) s
by *(rule wp-event-mono-on-support)*
(use bind in ‹auto simp: initial-merkle-binding-no-bad-def›)
also have *... = 0*
unfolding *wp-event-def wp-def dist-expect-def* **by** *simp*
finally show *?thesis* .
qed

lemma *accepted-with-bound-tables-query-consistency*:
assumes *bound:*
accepted-with-bound-tables s out trace-table composition-table alphas query-idxs
shows $\bigwedge idx. idx \in \text{set query-idxs} \implies$
query-consistent-at trace-table composition-table alphas idx
using *bound* **unfolding** *accepted-with-bound-tables-def accepted-with-tables-def* **by** *simp*

lemma *accepted-with-bound-tables-bound-roots*:
assumes *bound:*
accepted-with-bound-tables s out trace-table composition-table alphas query-idxs
obtains *result final-state fr composition-root*
where *out = Some (result, final-state)*
and *merkle-root-binds-table fr trace-table final-state*
and *merkle-root-binds-table composition-root composition-table final-state*
using *accepted-with-tables-authenticated-roots[OF bound]* **by** *blast*

lemma *verifier-header-transcript-shapes*:
assumes *verifier-header-transcript s fr f-fri-roots f-final as dg*
composition-fri-roots final rest
shows *PTranscript s =*
verifier-header-messages fr f-fri-roots f-final as dg
composition-fri-roots final @ rest
and *length f-fri-roots = ceil-log clength*
and *length as = length spec*
and *length composition-fri-roots = ceil-log (to-nat dg + 1)*
using *assms* **unfolding** *verifier-header-transcript-def* **by** *simp-all*

lemma *verifier-header-transcript-unique*:
assumes *h1:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *h2:*
verifier-header-transcript s fr' f-fri-roots' f-final' as' dg'
composition-fri-roots' final' rest'
shows

```

fr' = fr ∧
f-fri-roots' = f-fri-roots ∧
f-final' = f-final ∧
as' = as ∧
dg' = dg ∧
composition-fri-roots' = composition-fri-roots ∧
final' = final ∧
rest' = rest
proof –
  have lengths:
    length f-fri-roots = ceil-log clength
    length as = length spec
    length composition-fri-roots = ceil-log (to-nat dg + 1)
    length f-fri-roots' = ceil-log clength
    length as' = length spec
    length composition-fri-roots' = ceil-log (to-nat dg' + 1)
  using h1 h2 unfolding verifier-header-transcript-def by simp-all
  have msg-eq:
    verifier-header-messages fr f-fri-roots f-final as dg
      composition-fri-roots final @ rest =
    verifier-header-messages fr' f-fri-roots' f-final' as' dg'
      composition-fri-roots' final' @ rest'
  using h1 h2 unfolding verifier-header-transcript-def by simp
  then show ?thesis
    using lengths
    unfolding verifier-header-messages-def
    by (auto simp: append-eq-append-conv-if split: if-splits)
qed

```

definition *alpha-header-trace-table-candidate-state*

```

:: ('f, 'a) protocol-channel-scheme ⇒
  'f ⇒ 'f list ⇒ 'f ⇒ 'f list ⇒
  ('f, 'a) protocol-channel-scheme ⇒ bool

```

where

```

alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
  trace-table final-state ⟷
  (∃ result composition-table as query-idxs dg composition-fri-roots final
    rest.
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs ∧
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest ∧
    merkle-root-binds-table fr trace-table final-state)

```

definition *alpha-header-supported-trace-table-candidate-state*

```

:: ('f, 'a) protocol-channel-scheme ⇒
  'f ⇒ 'f list ⇒ 'f ⇒ 'f list ⇒
  ('f, 'a) protocol-channel-scheme ⇒ bool

```

where

alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table final-state $\longleftrightarrow$
 $(\exists \text{result composition-table as query-idxs dg composition-fri-roots final}$
rest.
Some (result, final-state) \in set-dist (execute verify-monad s) \wedge
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs \wedge
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest \wedge
merkle-root-binds-table fr trace-table final-state)

lemma *alpha-header-trace-table-candidates-iff-state:*

trace-table \in
alpha-header-trace-table-candidates s fr f-fri-roots f-final $\longleftrightarrow$
 $(\exists \text{final-state.}$
alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
trace-table final-state)

proof

assume *candidate:*

trace-table \in
alpha-header-trace-table-candidates s fr f-fri-roots f-final

then obtain *out composition-table as query-idxs dg composition-fri-roots*
final rest **where**

bound:

accepted-with-bound-tables s out trace-table composition-table as
query-idxs

and *header:*

verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest

unfolding *alpha-header-trace-table-candidates-def* **by** *blast*
from *bound* **obtain** *result final-state fr' f-fri-roots' f-final' dg'*
composition-fri-roots' final' rest' where

out-eq: out = Some (result, final-state)

and *header':*

verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
composition-fri-roots' final' rest'

and *bind:*

merkle-root-binds-table fr' trace-table final-state

unfolding *accepted-with-bound-tables-def* **by** *blast*

have *fr-eq: fr' = fr*

using *verifier-header-transcript-unique[OF header' header]* **by** *simp*

have *bound-some:*

accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs

using *bound out-eq* **by** *simp*

have *state:*

alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
trace-table final-state

unfolding *alpha-header-trace-table-candidate-state-def*

```

    by (intro exI[of - result] exI[of - composition-table] exI[of - as]
        exI[of - query-idxs] exI[of - dg] exI[of - composition-fri-roots]
        exI[of - final] exI[of - rest])
    (use bound-some header bind fr-eq in simp)
  then show
     $\exists$  final-state.
      alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
      trace-table final-state
    by blast
next
assume
   $\exists$  final-state.
    alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table final-state
then obtain final-state result composition-table as query-idxs dg
  composition-fri-roots final rest where
  bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  unfolding alpha-header-trace-table-candidate-state-def by blast
show
  trace-table  $\in$ 
    alpha-header-trace-table-candidates s fr f-fri-roots f-final
  unfolding alpha-header-trace-table-candidates-def
  using bound header by blast
qed

```

lemma *alpha-header-supported-trace-table-candidates-iff-state:*

```

  trace-table  $\in$ 
    alpha-header-supported-trace-table-candidates s fr f-fri-roots
    f-final  $\longleftrightarrow$ 
  ( $\exists$  final-state.
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table final-state)

```

proof

```

  assume candidate:
    trace-table  $\in$ 
      alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
  then obtain out composition-table as query-idxs dg composition-fri-roots
    final rest where
    outcome: out  $\in$  set-dist (execute verify-monad s)
  and bound:
    accepted-with-bound-tables s out trace-table composition-table as
    query-idxs
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg

```

```

      composition-fri-roots final rest
    unfolding alpha-header-supported-trace-table-candidates-def by blast
from bound obtain result final-state fr' f-fri-roots' f-final' dg'
      composition-fri-roots' final' rest' where
      out-eq: out = Some (result, final-state)
    and header':
      verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
      composition-fri-roots' final' rest'
    and bind:
      merkle-root-binds-table fr' trace-table final-state
    unfolding accepted-with-bound-tables-def by blast
have fr-eq: fr' = fr
    using verifier-header-transcript-unique[OF header' header] by simp
have state:
      alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
      f-final trace-table final-state
    unfolding alpha-header-supported-trace-table-candidate-state-def
    by (intro exI[of - result] exI[of - composition-table] exI[of - as]
      exI[of - query-idxs] exI[of - dg] exI[of - composition-fri-roots]
      exI[of - final] exI[of - rest])
      (use outcome bound header bind fr-eq out-eq in simp)
    then show
       $\exists$  final-state.
      alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
      f-final trace-table final-state
    by blast
next
    assume
       $\exists$  final-state.
      alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
      f-final trace-table final-state
    then obtain final-state result composition-table as query-idxs dg
      composition-fri-roots final rest where
      outcome:
        Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
    and bound:
      accepted-with-bound-tables s (Some (result, final-state))
      trace-table composition-table as query-idxs
    and header:
      verifier-header-transcript s fr f-fri-roots f-final as dg
      composition-fri-roots final rest
    unfolding alpha-header-supported-trace-table-candidate-state-def by blast
show
      trace-table  $\in$ 
      alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
    unfolding alpha-header-supported-trace-table-candidates-def
    using outcome bound header by blast
qed

```


lemma *alpha-header-supported-trace-table-candidate-state-imp-candidate-state:*
assumes
alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table final-state
shows
alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
trace-table final-state
using *assms*
unfolding *alpha-header-supported-trace-table-candidate-state-def*
alpha-header-trace-table-candidate-state-def
by *blast*

lemma *alpha-header-trace-table-candidate-states-common-extension-unique-if-clean:*
assumes *clean: $\neg$ hash-map-output-collision u*
and *candidate1:*
alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
trace-table final-state
and *candidate2:*
alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
trace-table' final-state'
and *ext1: final-state $\leq$ u*
and *ext2: final-state' $\leq$ u*
shows *trace-table = trace-table'*
proof –
from *candidate1* **obtain** *result composition-table as query-idxs dg*
composition-fri-roots final rest **where**
bound1:
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
and *bind1: merkle-root-binds-table fr trace-table final-state*
unfolding *alpha-header-trace-table-candidate-state-def* **by** *blast*
from *candidate2* **obtain** *result' composition-table' as' query-idxs' dg'*
composition-fri-roots' final' rest' **where**
bound2:
accepted-with-bound-tables s (Some (result', final-state'))
trace-table' composition-table' as' query-idxs'
and *bind2: merkle-root-binds-table fr trace-table' final-state'*
unfolding *alpha-header-trace-table-candidate-state-def* **by** *blast*
have *same-len: length trace-table = length trace-table'*
using *accepted-with-bound-tables-shapes(1)[OF bound1]*
accepted-with-bound-tables-shapes(1)[OF bound2]
by *simp*
have *nonempty: trace-table $\neq$ []*
using *accepted-with-bound-tables-shapes(1)[OF bound1]*
eval-domain-nontrivial **by** *auto*
show *?thesis*
by (*rule merkle-root-binds-same-length-tables-common-extension-unique-if-clean*
[OF clean bind1 bind2 ext1 ext2 same-len nonempty])
qed

lemma *alpha-header-trace-table-candidate-states-compatible-unique-if-clean-merge:*

assumes *compatible: hash-maps-compatible final-state final-state'*

and *clean:*

$\neg$ *hash-map-output-collision*

 (*hash-state-merge final-state final-state'*)

and *candidate1:*

alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
 trace-table final-state

and *candidate2:*

alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
 trace-table' final-state'

shows *trace-table = trace-table'*

proof –

from *candidate1* **obtain** *result composition-table as query-idxs dg*

composition-fri-roots final rest **where**

bound1:

accepted-with-bound-tables s (Some (result, final-state))

trace-table composition-table as query-idxs

and *bind1: merkle-root-binds-table fr trace-table final-state*

unfolding *alpha-header-trace-table-candidate-state-def* **by** *blast*

from *candidate2* **obtain** *result' composition-table' as' query-idxs' dg'*

composition-fri-roots' final' rest' **where**

bound2:

accepted-with-bound-tables s (Some (result', final-state'))

trace-table' composition-table' as' query-idxs'

and *bind2: merkle-root-binds-table fr trace-table' final-state'*

unfolding *alpha-header-trace-table-candidate-state-def* **by** *blast*

have *same-len: length trace-table = length trace-table'*

using *accepted-with-bound-tables-shapes(1)[OF bound1]*

accepted-with-bound-tables-shapes(1)[OF bound2]

by *simp*

have *nonempty: trace-table $\neq$ []*

using *accepted-with-bound-tables-shapes(1)[OF bound1]*

eval-domain-nontrivial **by** *auto*

show *?thesis*

by (*rule*

merkle-root-binds-same-length-tables-compatible-states-unique-if-clean-merge

 [*OF compatible clean bind1 bind2 same-len nonempty*])

qed

lemma *alpha-header-trace-table-candidate-states-merkle-compatible-unique-if-clean-merge:*

assumes *compatible:*

merkle-hash-maps-compatible final-state final-state'

and *clean:*

$\neg$ *hash-map-output-collision*

 (*merkle-hash-state-merge final-state final-state'*)

and *candidate1:*

alpha-header-trace-table-candidate-state s fr f-fri-roots f-final

```

    trace-table final-state
  and candidate2:
    alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table' final-state'
  shows trace-table = trace-table'
proof -
  from candidate1 obtain result composition-table as query-idxs dg
    composition-fri-roots final rest where
  bound1:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and bind1: merkle-root-binds-table fr trace-table final-state
  unfolding alpha-header-trace-table-candidate-state-def by blast
  from candidate2 obtain result' composition-table' as' query-idxs' dg'
    composition-fri-roots' final' rest' where
  bound2:
    accepted-with-bound-tables s (Some (result', final-state'))
    trace-table' composition-table' as' query-idxs'
  and bind2: merkle-root-binds-table fr trace-table' final-state'
  unfolding alpha-header-trace-table-candidate-state-def by blast
  have same-len: length trace-table = length trace-table'
  using accepted-with-bound-tables-shapes(1)[OF bound1]
    accepted-with-bound-tables-shapes(1)[OF bound2]
  by simp
  have nonempty: trace-table ≠ []
  using accepted-with-bound-tables-shapes(1)[OF bound1]
    eval-domain-nontrivial by auto
  show ?thesis
  by (rule
    merkle-root-binds-same-length-tables-merkle-compatible-unique-if-clean-merge
    [OF compatible clean bind1 bind2 same-len nonempty])
qed

```

lemma *alpha-header-trace-table-candidates-subset-singleton-if-common-clean-extension:*

```

  assumes clean: ¬ hash-map-output-collision u
  and common-ext:
     $\bigwedge$  trace-table final-state.
    alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table final-state  $\implies$ 
    final-state  $\leq$  u

```

shows

```

 $\exists$  trace-table.
  alpha-header-trace-table-candidates s fr f-fri-roots f-final  $\subseteq$ 
  {trace-table}

```

proof (cases

```

  alpha-header-trace-table-candidates s fr f-fri-roots f-final = {})

```

case True

then show ?thesis

```

  by (intro exI[of - []]) simp

```

```

next
  case False
  then obtain trace-table0 where table0:
    trace-table0 ∈
      alpha-header-trace-table-candidates s fr f-fri-roots f-final
    by blast
  then obtain final-state0 where state0:
    alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table0 final-state0
    using alpha-header-trace-table-candidates-iff-state by blast
  have ext0: final-state0 ≤ u
    by (rule common-ext[OF state0])
  have subset:
    alpha-header-trace-table-candidates s fr f-fri-roots f-final ⊆
      {trace-table0}
  proof
    fix trace-table
    assume table:
      trace-table ∈
        alpha-header-trace-table-candidates s fr f-fri-roots f-final
    then obtain final-state where state:
      alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
      trace-table final-state
      using alpha-header-trace-table-candidates-iff-state by blast
    have ext: final-state ≤ u
      by (rule common-ext[OF state])
    have trace-table = trace-table0
      by (rule
        alpha-header-trace-table-candidate-states-common-extension-unique-if-clean
        [OF clean state state0 ext ext0])
    then show trace-table ∈ {trace-table0}
      by simp
  qed
  then show ?thesis
    by blast
qed

```

lemma *alpha-header-trace-table-candidates-subset-singleton-if-pairwise-clean-merge*:

```

assumes pairwise:
  ∧ trace-table final-state trace-table' final-state'.
    alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table final-state ⇒
    alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table' final-state' ⇒
    hash-maps-compatible final-state final-state' ∧
    ¬ hash-map-output-collision
    (hash-state-merge final-state final-state')

```

shows

```

  ∃ trace-table.

```

```

      alpha-header-trace-table-candidates s fr f-fri-roots f-final  $\subseteq$ 
        {trace-table}
proof (cases
  alpha-header-trace-table-candidates s fr f-fri-roots f-final = {})
case True
then show ?thesis
  by (intro exI[of - []]) simp
next
case False
then obtain trace-table0 where table0:
  trace-table0  $\in$ 
    alpha-header-trace-table-candidates s fr f-fri-roots f-final
by blast
then obtain final-state0 where state0:
  alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table0 final-state0
using alpha-header-trace-table-candidates-iff-state by blast
have subset:
  alpha-header-trace-table-candidates s fr f-fri-roots f-final  $\subseteq$ 
    {trace-table0}
proof
  fix trace-table
  assume table:
    trace-table  $\in$ 
      alpha-header-trace-table-candidates s fr f-fri-roots f-final
  then obtain final-state where state:
    alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
      trace-table final-state
  using alpha-header-trace-table-candidates-iff-state by blast
  have compatible:
    hash-maps-compatible final-state final-state0
  using pairwise[OF state state0] by simp
  have clean:
     $\neg$  hash-map-output-collision
      (hash-state-merge final-state final-state0)
  using pairwise[OF state state0] by simp
  have trace-table = trace-table0
    by (rule
      alpha-header-trace-table-candidate-states-compatible-unique-if-clean-merge
        [OF compatible clean state state0])
  then show trace-table  $\in$  {trace-table0}
    by simp
qed
then show ?thesis
  by blast
qed

lemma alpha-header-supported-trace-table-candidates-subset-singleton-if-pairwise-clean-merge:
  assumes pairwise:

```

```

 $\wedge$  trace-table final-state trace-table' final-state'.
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table final-state  $\implies$ 
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table' final-state'  $\implies$ 
  hash-maps-compatible final-state final-state'  $\wedge$ 
 $\neg$  hash-map-output-collision
  (hash-state-merge final-state final-state')
shows
 $\exists$  trace-table.
  alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
 $\subseteq \{ \text{trace-table} \}$ 
proof (cases
  alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final =
 $\{ \}$ )
case True
then show ?thesis
by (intro exI[of -  $\square$ ]) simp
next
case False
then obtain trace-table0 where table0:
  trace-table0  $\in$ 
  alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
by blast
then obtain final-state0 where state0:
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table0 final-state0
using alpha-header-supported-trace-table-candidates-iff-state by blast
have state0-plain:
  alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
  trace-table0 final-state0
by (rule alpha-header-supported-trace-table-candidate-state-imp-candidate-state
  [OF state0])
have subset:
  alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
 $\subseteq \{ \text{trace-table0} \}$ 
proof
fix trace-table
assume table:
  trace-table  $\in$ 
  alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
then obtain final-state where state:
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table final-state
using alpha-header-supported-trace-table-candidates-iff-state by blast
have state-plain:
  alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
  trace-table final-state
by (rule

```

```

      alpha-header-supported-trace-table-candidate-state-imp-candidate-state
      [OF state])
  have compatible:
    hash-maps-compatible final-state final-state0
    using pairwise[OF state state0] by simp
  have clean:
    ¬ hash-map-output-collision
    (hash-state-merge final-state final-state0)
    using pairwise[OF state state0] by simp
  have trace-table = trace-table0
    by (rule
      alpha-header-trace-table-candidate-states-compatible-unique-if-clean-merge
      [OF compatible clean state-plain state0-plain])
  then show trace-table ∈ {trace-table0}
    by simp
qed
then show ?thesis
  by blast
qed

lemma alpha-header-supported-trace-table-candidates-subset-singleton-if-pairwise-no-value-conflict-clean-merge:
  assumes pairwise:
    ∧ trace-table final-state trace-table' final-state'.
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table final-state ⇒
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table' final-state' ⇒
    ¬ hash-map-value-conflict final-state final-state' ∧
    ¬ hash-map-output-collision
    (hash-state-merge final-state final-state')
  shows
    ∃ trace-table.
    alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
    ⊆ {trace-table}
proof (rule
  alpha-header-supported-trace-table-candidates-subset-singleton-if-pairwise-clean-merge)
fix trace-table final-state trace-table' final-state'
assume candidate:
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table final-state
and candidate':
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table' final-state'
have no-conflict:
  ¬ hash-map-value-conflict final-state final-state'
  using pairwise[OF candidate candidate'] by simp
have compatible:
  hash-maps-compatible final-state final-state'
  using no-conflict

```

```

unfolding hash-maps-compatible-iff-no-value-conflict .
have clean:
   $\neg$  hash-map-output-collision
    (hash-state-merge final-state final-state')
using pairwise[OF candidate candidate'] by simp
show hash-maps-compatible final-state final-state'  $\wedge$ 
   $\neg$  hash-map-output-collision
    (hash-state-merge final-state final-state')
using compatible clean by simp
qed

lemma alpha-header-supported-trace-table-candidates-subset-singleton-if-pairwise-merkle-clean-merge:
assumes pairwise:
   $\wedge$  trace-table final-state trace-table' final-state'.
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table final-state  $\implies$ 
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table' final-state'  $\implies$ 
  merkle-hash-maps-compatible final-state final-state'  $\wedge$ 
   $\neg$  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
shows
   $\exists$  trace-table.
  alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
     $\subseteq$  {trace-table}
proof (cases
  alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final =
    {})
case True
then show ?thesis
  by (intro exI[of - []]) simp
next
case False
then obtain trace-table0 where table0:
  trace-table0  $\in$ 
  alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
by blast
then obtain final-state0 where state0:
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table0 final-state0
using alpha-header-supported-trace-table-candidates-iff-state by blast
have state0-plain:
  alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table0 final-state0
by (rule alpha-header-supported-trace-table-candidate-state-imp-candidate-state
    [OF state0])
have subset:
  alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
     $\subseteq$  {trace-table0}

```


proof
 fix *trace-table*
 assume *table*:
 trace-table ∈
 alpha-header-supported-trace-table-candidates *s* fr *f-fri-roots* *f-final*
 then obtain *final-state* where *state*:
 alpha-header-supported-trace-table-candidate-state *s* fr *f-fri-roots*
 f-final *trace-table* *final-state*
 using *alpha-header-supported-trace-table-candidates-iff-state* by blast
 have *state-plain*:
 alpha-header-trace-table-candidate-state *s* fr *f-fri-roots* *f-final*
 trace-table *final-state*
 by (rule
 alpha-header-supported-trace-table-candidate-state-imp-candidate-state
 [*OF state*])
 have *compatible*:
 merkle-hash-maps-compatible *final-state* *final-state0*
 using *pairwise*[*OF state state0*] by simp
 have *clean*:
 ¬ *hash-map-output-collision*
 (*merkle-hash-state-merge* *final-state* *final-state0*)
 using *pairwise*[*OF state state0*] by simp
 have *trace-table* = *trace-table0*
 by (rule
 alpha-header-trace-table-candidate-states-merkle-compatible-unique-if-clean-merge
 [*OF compatible clean state-plain state0-plain*])
 then show *trace-table* ∈ {*trace-table0*}
 by simp
 qed
 then show ?*thesis*
 by blast
 qed

lemma *alpha-header-supported-trace-table-candidates-subset-singleton-if-pairwise-no-merkle-value-conflict-clean-merge*

assumes *pairwise*:

∧ *trace-table* *final-state* *trace-table'* *final-state'*.
 alpha-header-supported-trace-table-candidate-state *s* fr *f-fri-roots*
 f-final *trace-table* *final-state* ⇒
 alpha-header-supported-trace-table-candidate-state *s* fr *f-fri-roots*
 f-final *trace-table'* *final-state'* ⇒
 ¬ *merkle-hash-value-conflict* *final-state* *final-state'* ∧
 ¬ *hash-map-output-collision*
 (*merkle-hash-state-merge* *final-state* *final-state'*)

shows

∃ *trace-table*.
 alpha-header-supported-trace-table-candidates *s* fr *f-fri-roots* *f-final*
 ⊆ {*trace-table*}

proof (rule

alpha-header-supported-trace-table-candidates-subset-singleton-if-pairwise-merkle-clean-merge)

```

fix trace-table final-state trace-table' final-state'
assume candidate:
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table final-state
and candidate':
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table' final-state'
have no-conflict:
  ¬ merkle-hash-value-conflict final-state final-state'
  using pairwise[OF candidate candidate'] by simp
have compatible:
  merkle-hash-maps-compatible final-state final-state'
  using no-conflict
  unfolding merkle-hash-maps-compatible-iff-no-value-conflict .
have clean:
  ¬ hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')
  using pairwise[OF candidate candidate'] by simp
show merkle-hash-maps-compatible final-state final-state' ∧
  ¬ hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')
  using compatible clean by simp
qed

lemma alpha-header-supported-trace-table-candidates-subset-singleton-if-common-clean-merkle-extension:
assumes clean: ¬ hash-map-output-collision u
and common-ext:
  ∧ trace-table final-state.
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table final-state ⇒
  merkle-hash-extends final-state u
shows
  ∃ trace-table.
  alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
  ⊆ {trace-table}
proof (rule
  alpha-header-supported-trace-table-candidates-subset-singleton-if-pairwise-no-merkle-value-conflict-clean-merge)
fix trace-table final-state trace-table' final-state'
assume candidate:
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table final-state
and candidate':
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table' final-state'
have ext: merkle-hash-extends final-state u
  by (rule common-ext[OF candidate])
have ext': merkle-hash-extends final-state' u
  by (rule common-ext[OF candidate'])
show ¬ merkle-hash-value-conflict final-state final-state' ∧

```

```

    ¬ hash-map-output-collision
      (merkle-hash-state-merge final-state final-state')
  by (rule common-clean-merkle-extension-imp-pairwise-merkle-clean
      [OF ext ext' clean])
qed

lemma alpha-header-supported-trace-table-candidate-states-distinct-imp-merkle-conflict-or-merge-collision:
  assumes candidate:
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table final-state
  and candidate':
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table' final-state'
  and distinct-tables: trace-table ≠ trace-table'
  shows
    merkle-hash-value-conflict final-state final-state' ∨
    hash-map-output-collision
      (merkle-hash-state-merge final-state final-state')
proof (rule ccontr)
  assume no-bad:
    ¬ (merkle-hash-value-conflict final-state final-state' ∨
        hash-map-output-collision
          (merkle-hash-state-merge final-state final-state'))
  have compatible:
    merkle-hash-maps-compatible final-state final-state'
  using no-bad unfolding merkle-hash-maps-compatible-iff-no-value-conflict
  by simp
  have clean:
    ¬ hash-map-output-collision
      (merkle-hash-state-merge final-state final-state')
  using no-bad by simp
  have candidate-plain:
    alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table final-state
  by (rule alpha-header-supported-trace-table-candidate-state-imp-candidate-state
      [OF candidate])
  have candidate-plain':
    alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table' final-state'
  by (rule alpha-header-supported-trace-table-candidate-state-imp-candidate-state
      [OF candidate'])
  have trace-table = trace-table'
  by (rule alpha-header-trace-table-candidate-states-merkle-compatible-unique-if-clean-merge
      [OF compatible clean candidate-plain candidate-plain'])
  then show False
  using distinct-tables by contradiction
qed

```

lemma alpha-header-supported-trace-table-candidates-not-singleton-imp-pairwise-merkle-bad:

assumes *not-unique*:

$\neg (\exists \text{ trace-table.}$
 $\text{alpha-header-supported-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final} \subseteq \{\text{trace-table}\})$

shows

$\exists \text{ trace-table final-state trace-table' final-state'.$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final trace-table final-state} \wedge$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final trace-table' final-state'} \wedge$
 $\text{trace-table} \neq \text{trace-table'} \wedge$
 $(\text{merkle-hash-value-conflict final-state final-state'} \vee$
 $\text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge final-state final-state'}))$

proof —

let $?C =$
 $\text{alpha-header-supported-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final}$
have *nonempty*: $?C \neq \{\}$
 using *not-unique* **by** *auto*
then obtain trace-table **where** $\text{table: trace-table} \in ?C$
 by *blast*
have $\neg ?C \subseteq \{\text{trace-table}\}$
 using *not-unique* **by** *blast*
then obtain trace-table' **where** table':
 $\text{trace-table'} \in ?C$
 and *distinct-tables*: $\text{trace-table} \neq \text{trace-table'}$
 by *auto*
from table **obtain** final-state **where** *candidate*:
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final trace-table final-state}$
 using *alpha-header-supported-trace-table-candidates-iff-state* **by** *blast*
from table' **obtain** final-state' **where** *candidate'*:
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final trace-table' final-state'}$
 using *alpha-header-supported-trace-table-candidates-iff-state* **by** *blast*
have *bad*:
 $\text{merkle-hash-value-conflict final-state final-state'} \vee$
 $\text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge final-state final-state'})$
 by (*rule*
 $\text{alpha-header-supported-trace-table-candidate-states-distinct-imp-merkle-conflict-or-merge-collision}$
 $[\text{OF candidate candidate' distinct-tables}]$)
show *?thesis*
 using $\text{candidate candidate' distinct-tables bad}$ **by** *blast*
qed

definition *alpha-header-supported-pairwise-merkle-bad*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow \text{bool}$

where

alpha-header-supported-pairwise-merkle-bad *s* *fr* *f-fri-roots* *f-final*
 $\longleftrightarrow$
 $(\exists \text{ trace-table } \text{final-state } \text{trace-table}' \text{ final-state}'.$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final } \text{trace-table } \text{final-state} \wedge$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final } \text{trace-table}' \text{ final-state}' \wedge$
 $\text{trace-table} \neq \text{trace-table}' \wedge$
 $(\text{merkle-hash-value-conflict } \text{final-state } \text{final-state}' \vee$
 $\text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge } \text{final-state } \text{final-state}'))))$

definition *alpha-supported-pairwise-merkle-bad*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

alpha-supported-pairwise-merkle-bad *s* $\longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final}.$
 $\text{alpha-header-supported-pairwise-merkle-bad } s \text{ fr } f\text{-fri-roots } f\text{-final})$

lemma *alpha-header-supported-candidate-unique-if-no-pairwise-merkle-bad*:

assumes *no-bad*:

$\neg \text{alpha-header-supported-pairwise-merkle-bad } s \text{ fr } f\text{-fri-roots } f\text{-final}$

shows

$\exists \text{ trace-table}.$
 $\text{alpha-header-supported-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final} \subseteq \{\text{trace-table}\}$

proof (rule *ccontr*)

assume *not-unique*:

$\neg (\exists \text{ trace-table}.$
 $\text{alpha-header-supported-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final} \subseteq \{\text{trace-table}\})$

have *alpha-header-supported-pairwise-merkle-bad* *s* *fr* *f-fri-roots* *f-final*

using

alpha-header-supported-trace-table-candidates-not-singleton-imp-pairwise-merkle-bad
 $[OF \text{ not-unique}]$

unfolding *alpha-header-supported-pairwise-merkle-bad-def*

by *blast*

then show *False*

using *no-bad* **by** *contradiction*

qed

lemma *alpha-header-supported-candidate-unique-if-no-global-pairwise-merkle-bad*:

assumes *no-bad*: $\neg \text{alpha-supported-pairwise-merkle-bad } s$

shows

$\exists \text{ trace-table}.$
 $\text{alpha-header-supported-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final} \subseteq \{\text{trace-table}\}$

proof (rule *alpha-header-supported-candidate-unique-if-no-pairwise-merkle-bad*)

show $\neg$ *alpha-header-supported-pairwise-merkle-bad* *s* *fr* *f-fri-roots* *f-final*
using *no-bad* **unfolding** *alpha-supported-pairwise-merkle-bad-def* **by** *blast*
qed

lemma *accepted-with-bound-tables-common-roots*:

assumes *bound1*:

accepted-with-bound-tables *s* *out* *trace-table* *composition-table*

alphas *query-idxs*

and *bound2*:

accepted-with-bound-tables *s* *out* *trace-table'* *composition-table'*

alphas' *query-idxs'*

obtains *result* *final-state* *fr* *composition-root*

where *out* = *Some* (*result*, *final-state*)

and *merkle-root-binds-table* *fr* *trace-table* *final-state*

and *merkle-root-binds-table* *fr* *trace-table'* *final-state*

and *merkle-root-binds-table* *composition-root* *composition-table* *final-state*

and *merkle-root-binds-table* *composition-root* *composition-table'* *final-state*

proof –

from *bound1* **obtain** *result1* *final-state1* *fr* *f-fri-roots* *f-final* *dg*

composition-fri-roots *final* *rest* **where**

out1: *out* = *Some* (*result1*, *final-state1*)

and *header1*:

verifier-header-transcript *s* *fr* *f-fri-roots* *f-final* *alphas* *dg*

composition-fri-roots *final* *rest*

and *trace-bind1*:

merkle-root-binds-table *fr* *trace-table* *final-state1*

and *comp-bind1*:

merkle-root-binds-table (*hd* *composition-fri-roots*)

composition-table *final-state1*

unfolding *accepted-with-bound-tables-def* **by** *blast*

from *bound2* **obtain** *result2* *final-state2* *fr'* *f-fri-roots'* *f-final'* *dg'*

composition-fri-roots' *final'* *rest'* **where**

out2: *out* = *Some* (*result2*, *final-state2*)

and *header2*:

verifier-header-transcript *s* *fr'* *f-fri-roots'* *f-final'* *alphas'* *dg'*

composition-fri-roots' *final'* *rest'*

and *trace-bind2*:

merkle-root-binds-table *fr'* *trace-table'* *final-state2*

and *comp-bind2*:

merkle-root-binds-table (*hd* *composition-fri-roots'*)

composition-table' *final-state2*

unfolding *accepted-with-bound-tables-def* **by** *blast*

have *final-state-eq*: *final-state2* = *final-state1*

using *out1* *out2* **by** *simp*

have *roots-eq*:

fr' = *fr* $\wedge$ *composition-fri-roots'* = *composition-fri-roots*

using *verifier-header-transcript-unique*[*OF* *header1* *header2*] **by** *simp*

have *trace-bind2'*:

merkle-root-binds-table *fr* *trace-table'* *final-state1*

```

    using trace-bind2 roots-eq final-state-eq by simp
  have comp-bind2':
    merkle-root-binds-table (hd composition-fri-roots)
      composition-table' final-state1
    using comp-bind2 roots-eq final-state-eq by simp
  show ?thesis
    by (rule that[OF out1 trace-bind1 trace-bind2' comp-bind1 comp-bind2'])
qed

lemma accepted-with-bound-tables-trace-table-unique-or-hash-collision-bad:
  assumes bound1:
    accepted-with-bound-tables s out trace-table composition-table
    alphas query-idxs
  and bound2:
    accepted-with-bound-tables s out trace-table' composition-table'
    alphas' query-idxs'
  shows trace-table = trace-table'  $\vee$  hash-map-output-collision-bad s out
proof (cases trace-table = trace-table')
  case True
  then show ?thesis by simp
next
  case neg: False
  from accepted-with-bound-tables-common-roots[OF bound1 bound2]
  obtain result final-state fr composition-root where
    out: out = Some (result, final-state)
    and bind1: merkle-root-binds-table fr trace-table final-state
    and bind2: merkle-root-binds-table fr trace-table' final-state
    by blast
  have same-len: length trace-table = length trace-table'
    using accepted-with-bound-tables-shapes(1)[OF bound1]
      accepted-with-bound-tables-shapes(1)[OF bound2]
    by simp
  have nonempty: trace-table  $\neq$  []
    using accepted-with-bound-tables-shapes(1)[OF bound1]
      eval-domain-nontrivial
    by auto
  have hash-map-output-collision-bad s out
    by (rule accepted-inconsistent-same-length-merkle-bindings-imp-hash-collision-bad
      [OF out bind1 bind2 same-len nonempty neg])
  then show ?thesis by simp
qed

```

```

lemma accepted-with-bound-tables-composition-table-unique-or-hash-collision-bad:
  assumes bound1:
    accepted-with-bound-tables s out trace-table composition-table
    alphas query-idxs
  and bound2:
    accepted-with-bound-tables s out trace-table' composition-table'
    alphas' query-idxs'

```

```

shows composition-table = composition-table'  $\vee$ 
  hash-map-output-collision-bad s out
proof (cases composition-table = composition-table')
  case True
  then show ?thesis by simp
next
  case neg: False
  from accepted-with-bound-tables-common-roots[OF bound1 bound2]
  obtain result final-state fr composition-root where
    out: out = Some (result, final-state)
  and bind1:
    merkle-root-binds-table composition-root composition-table final-state
  and bind2:
    merkle-root-binds-table composition-root composition-table' final-state
  by blast
  have same-len: length composition-table = length composition-table'
  using accepted-with-bound-tables-shapes(2)[OF bound1]
    accepted-with-bound-tables-shapes(2)[OF bound2]
  by simp
  have nonempty: composition-table  $\neq$  []
  using accepted-with-bound-tables-shapes(2)[OF bound1]
    eval-domain-nontrivial
  by auto
  have hash-map-output-collision-bad s out
  by (rule accepted-inconsistent-same-length-merkle-bindings-imp-hash-collision-bad
    [OF out bind1 bind2 same-len nonempty neg])
  then show ?thesis by simp
qed

lemma accepted-with-bound-tables-tables-unique-or-hash-collision-bad:
  assumes bound1:
    accepted-with-bound-tables s out trace-table composition-table
    alphas query-idxs
  and bound2:
    accepted-with-bound-tables s out trace-table' composition-table'
    alphas' query-idxs'
  shows
    (trace-table = trace-table'  $\wedge$  composition-table = composition-table')  $\vee$ 
    hash-map-output-collision-bad s out
proof –
  have trace-unique:
    trace-table = trace-table'  $\vee$  hash-map-output-collision-bad s out
  by (rule accepted-with-bound-tables-trace-table-unique-or-hash-collision-bad
    [OF bound1 bound2])
  have comp-unique:
    composition-table = composition-table'  $\vee$ 
    hash-map-output-collision-bad s out
  by (rule accepted-with-bound-tables-composition-table-unique-or-hash-collision-bad
    [OF bound1 bound2])

```



```

show ?thesis
  using trace-unique comp-unique by blast
qed

lemma accepted-with-bound-tables-trace-table-unique-if-no-hash-collision-bad:
assumes no-collision:  $\neg$  hash-map-output-collision-bad  $s$  out
and bound1:
  accepted-with-bound-tables  $s$  out trace-table composition-table
  alphas query-idxs
and bound2:
  accepted-with-bound-tables  $s$  out trace-table' composition-table'
  alphas' query-idxs'
shows trace-table = trace-table'
using accepted-with-bound-tables-trace-table-unique-or-hash-collision-bad
  [OF bound1 bound2] no-collision
by blast

lemma accepted-with-bound-tables-composition-table-unique-if-no-hash-collision-bad:
assumes no-collision:  $\neg$  hash-map-output-collision-bad  $s$  out
and bound1:
  accepted-with-bound-tables  $s$  out trace-table composition-table
  alphas query-idxs
and bound2:
  accepted-with-bound-tables  $s$  out trace-table' composition-table'
  alphas' query-idxs'
shows composition-table = composition-table'
using accepted-with-bound-tables-composition-table-unique-or-hash-collision-bad
  [OF bound1 bound2] no-collision
by blast

definition accepted-bound-table-ambiguity-bad
:: ( $f$ ,  $a$ ) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ( $f$ ,  $a$ ) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  accepted-bound-table-ambiguity-bad  $s$  out  $\longleftrightarrow$ 
  ( $\exists$  trace-table composition-table alphas query-idxs
    trace-table' composition-table' alphas' query-idxs'.
    accepted-with-bound-tables  $s$  out trace-table composition-table
    alphas query-idxs  $\wedge$ 
    accepted-with-bound-tables  $s$  out trace-table' composition-table'
    alphas' query-idxs'  $\wedge$ 
    (trace-table  $\neq$  trace-table'  $\vee$ 
     composition-table  $\neq$  composition-table'))

lemma accepted-bound-table-ambiguity-bad-imp-hash-map-output-collision-bad:
assumes ambiguity: accepted-bound-table-ambiguity-bad  $s$  out
shows hash-map-output-collision-bad  $s$  out
proof –
  from ambiguity obtain trace-table composition-table alphas query-idxs

```

```

    trace-table' composition-table' alphas' query-idxs' where
bound:
    accepted-with-bound-tables s out trace-table composition-table
    alphas query-idxs
and bound':
    accepted-with-bound-tables s out trace-table' composition-table'
    alphas' query-idxs'
and distinct:
    trace-table  $\neq$  trace-table'  $\vee$ 
    composition-table  $\neq$  composition-table'
unfolding accepted-bound-table-ambiguity-bad-def by blast
have
    (trace-table = trace-table'  $\wedge$ 
     composition-table = composition-table')  $\vee$ 
    hash-map-output-collision-bad s out
by (rule accepted-with-bound-tables-tables-unique-or-hash-collision-bad
     [OF bound bound'])
then show ?thesis
    using distinct by blast
qed

lemma accepted-bound-table-ambiguity-bad-mono-hash-map-output-collision-bad:
    wp-event verify-monad (accepted-bound-table-ambiguity-bad s) s  $\leq$ 
    wp-event verify-monad (hash-map-output-collision-bad s) s
by (rule wp-event-mono)
    (rule
     accepted-bound-table-ambiguity-bad-imp-hash-map-output-collision-bad)

end

end

theory Soundness-Execution-Query
    imports Soundness-Merkle
begin

Verifier query-round and authenticated-opening execution lemmas.

context soundness
begin

lemma receive-random-field-element-outcome:
    assumes outcome: Some (a, t)  $\in$  set-dist (execute receive-random-field-element s)
    shows
        s  $\leq$  t  $\wedge$ 
        PState t = PState s  $\wedge$ 
        PTranscript t = PTranscript s  $\wedge$ 
        fmlookup (HashMap t) (FiatShamirChallenge (PState s)) = Some a
    using protocol-receive-random-field-element-outcome

```

$[OF\ outcome[unfolding\ receive-random-field-element-def]]$
by *simp*

lemma *throw-no-outcome*:
Some $(x, t) \notin set-dist\ (execute\ throw\ s)$
unfolding *set-dist-def throw.rep-eq dist-throw-def dist-delta-dist delta-map-def*
by *simp*

lemma *read-outcome*:
assumes *outcome*: *Some* $(x, t) \in set-dist\ (execute\ read\ s)$
shows
 $\exists rest.$
 $PTranscript\ s = x \# rest \wedge$
 $PState\ t = concat\ (PState\ s)\ x \wedge$
 $PTranscript\ t = rest \wedge$
 $s \leq t \wedge$
 $PTraceFriCounter\ t = PTraceFriCounter\ s \wedge$
 $PCompositionFriCounter\ t = PCompositionFriCounter\ s \wedge$
 $PAlphaCounter\ t = PAlphaCounter\ s \wedge$
 $PQueryCounter\ t = PQueryCounter\ s$

proof –
obtain *rest* **where** *tr-s*: $PTranscript\ s = x \# rest$
proof –
have $PTranscript\ s \neq []$
using *outcome* **unfolding** *read-def protocol-read-def assert-def*
by $(cases\ PTranscript\ s)\ (auto\ simp:\ throw-no-outcome\ elim!:\ set-dist-bindE)$
then obtain *y* *ys* **where** *ys*: $PTranscript\ s = y \# ys$
by $(cases\ PTranscript\ s)\ auto$
have *x-y*: $x = y$
using *read-nonempty-outcome*[*OF outcome* *ys*] **by** *simp*
show *?thesis*
using *that* *ys* *x-y* **by** *auto*

qed
have *t-res*:
 $t = s(PState := concat\ (PState\ s)\ x,\ PTranscript := rest)$
using *read-nonempty-outcome*[*OF outcome* *tr-s*] **by** *simp*
have $s \leq t$
unfolding *t-res less-eq-hash-ext-def less-eq-fmap-def* **by** *simp*
then show *?thesis*
using *tr-s* *t-res* **by** *auto*

qed

lemma *read-preserves-hash-map*:
assumes *outcome*: *Some* $(x, t) \in set-dist\ (execute\ read\ s)$
shows $HashMap\ t = HashMap\ s$

proof –
obtain *rest* **where** *tr-s*: $PTranscript\ s = x \# rest$
proof –
have $PTranscript\ s \neq []$

```

    using outcome unfolding read-def protocol-read-def assert-def
    by (cases PTranscript s) (auto simp: throw-no-outcome elim!: set-dist-bindE)
then obtain y ys where ys: PTranscript s = y # ys
    by (cases PTranscript s) auto
have x-y: x = y
    using read-nonempty-outcome[OF outcome ys] by simp
show ?thesis
    using that ys x-y by auto
qed
show ?thesis
    using read-nonempty-outcome[OF outcome tr-s] by simp
qed

```

```

lemma read-preserves-alpha-future-fresh:
  assumes future: alpha-future-fresh s
    and outcome: Some (x, t) ∈ set-dist (execute read s)
  shows alpha-future-fresh t
proof -
  have hash-eq: HashMap t = HashMap s
    by (rule read-preserves-hash-map[OF outcome])
  from read-outcome[OF outcome] have counter-eq:
    PAlphaCounter t = PAlphaCounter s
    by blast
  show ?thesis
    using future hash-eq counter-eq
    unfolding alpha-future-fresh-def by simp
qed

```

```

lemma read-preserves-trace-fri-future-fresh:
  assumes future: trace-fri-future-fresh s
    and outcome: Some (x, t) ∈ set-dist (execute read s)
  shows trace-fri-future-fresh t
proof -
  have hash-eq: HashMap t = HashMap s
    by (rule read-preserves-hash-map[OF outcome])
  from read-outcome[OF outcome] have counter-eq:
    PTraceFriCounter t = PTraceFriCounter s
    by blast
  show ?thesis
    using future hash-eq counter-eq
    unfolding trace-fri-future-fresh-def by simp
qed

```

```

lemma read-preserves-composition-fri-future-fresh:
  assumes future: composition-fri-future-fresh s
    and outcome: Some (x, t) ∈ set-dist (execute read s)
  shows composition-fri-future-fresh t
proof -
  have hash-eq: HashMap t = HashMap s

```

```

    by (rule read-preserves-hash-map[OF outcome])
  from read-outcome[OF outcome] have counter-eq:
    PCompositionFriCounter t = PCompositionFriCounter s
  by blast
show ?thesis
  using future hash-eq counter-eq
  unfolding composition-fri-future-fresh-def by simp
qed

lemma read-preserves-query-future-fresh:
  assumes future: query-future-fresh s
    and outcome: Some (x, t) ∈ set-dist (execute read s)
  shows query-future-fresh t
proof -
  have hash-eq: HashMap t = HashMap s
  by (rule read-preserves-hash-map[OF outcome])
  from read-outcome[OF outcome] have counter-eq:
    PQueryCounter t = PQueryCounter s
  by blast
show ?thesis
  using future hash-eq counter-eq
  unfolding query-future-fresh-def by simp
qed

lemma ntimes-read-any-outcome:
  assumes outcome: Some (xs, t) ∈ set-dist (execute (ntimes read n) s)
  shows
    length xs = n ∧
    PTranscript s = xs @ PTranscript t ∧
    PState t = foldl concat (PState s) xs ∧
    s ≤ t ∧
    PTraceFriCounter t = PTraceFriCounter s ∧
    PCompositionFriCounter t = PCompositionFriCounter s ∧
    PAlphaCounter t = PAlphaCounter s ∧
    PQueryCounter t = PQueryCounter s
  using outcome
proof (induction n arbitrary: s xs t)
  case 0
  then show ?case
    by (simp add: hash-ext-refl)
next
  case (Suc n)
  from Suc.prem1 obtain x xs' s1 where
    read-x: Some (x, s1) ∈ set-dist (execute read s)
  and reads:
    Some (xs', t) ∈ set-dist (execute (ntimes read n) s1)
  and xs-eq: xs = x # xs'
  by (auto elim!: set-dist-bindE)
  from read-outcome[OF read-x] obtain rest1 where

```

```
|s: PTranscript s = x # rest1
and st-s1: PState s1 = concat (PState s) x
and tr-s1: PTranscript s1 = rest1
and ext-s-s1: s ≤ s1
and trace-count-s1: PTraceFriCounter s1 = PTraceFriCounter s
and comp-count-s1: PCompositionFriCounter s1 = PCompositionFriCounter s
and alpha-count-s1: PAlphaCounter s1 = PAlphaCounter s
and query-count-s1: PQueryCounter s1 = PQueryCounter s
by blast
have tail:
length xs' = n ∧
PTranscript s1 = xs' @ PTranscript t ∧
PState t = foldl concat (PState s1) xs' ∧
s1 ≤ t ∧
PTraceFriCounter t = PTraceFriCounter s1 ∧
PCompositionFriCounter t = PCompositionFriCounter s1 ∧
PAlphaCounter t = PAlphaCounter s1 ∧
PQueryCounter t = PQueryCounter s1
using Suc.IH[OF reads] .
have s ≤ t
using ext-s-s1 tail by (meson hash-ext-trans)
then show ?case
using tr-s st-s1 tr-s1 tail xs-eq trace-count-s1 comp-count-s1
alpha-count-s1 query-count-s1
by simp
qed

|  |

```

lemma *ntimes-read-preserves-lookup*:

```

assumes outcome: Some (xs, t) ∈ set-dist (execute (ntimes read n) s)
shows fmlookup (HashMap t) key = fmlookup (HashMap s) key
proof (rule ntimes-preserves-lookup[OF - outcome])
fix x s t
assume read-x: Some (x, t) ∈ set-dist (execute read s)
show fmlookup (HashMap t) key = fmlookup (HashMap s) key
using read-preserves-hash-map[OF read-x] by simp
qed

```

lemma *check-authentication-path-hash-extends*:

```

assumes outcome:
Some (hh, t) ∈
set-dist (execute (check-authentication-path len i v path) s)
shows s ≤ t
using outcome
unfolding check-authentication-path-def protocol-check-authentication-path-def
proof (induction path arbitrary: len i v s hh t)
case Nil
then have hash-step:
Some (hh, t) ∈ set-dist (execute (hash (MerkleLeaf v)) s)
by simp

```

```

show ?case
  by (rule hash-outcome(1)[OF hash-step])
next
case (Cons a path)
show ?case
proof (cases i < len div 2)
  case True
  from Cons.premis obtain x u where
    rec:
      Some (x, u) ∈
        set-dist (execute
          (protocol-merkle.check-authentication-path
            (len div 2) i (MerkleLeaf v) path) s)
    and hash:
      Some (hh, t) ∈ set-dist (execute (hash (MerkleNode x a)) u)
    using True by (auto elim!: set-dist-bindE)
  have s ≤ u
    by (rule Cons.IH[OF rec])
  moreover have u ≤ t
    using hash-outcome(1)[OF hash] .
  ultimately show ?thesis
    by (rule hash-ext-trans)
next
case False
  from Cons.premis obtain x u where
    rec:
      Some (x, u) ∈
        set-dist (execute
          (protocol-merkle.check-authentication-path
            (len div 2) (i - len div 2) (MerkleLeaf v) path) s)
    and hash:
      Some (hh, t) ∈ set-dist (execute (hash (MerkleNode a x)) u)
    using False by (auto elim!: set-dist-bindE)
  have s ≤ u
    by (rule Cons.IH[OF rec])
  moreover have u ≤ t
    using hash-outcome(1)[OF hash] .
  ultimately show ?thesis
    by (rule hash-ext-trans)
qed
qed

```

lemma check-decommit-on-query-step-outcome:

assumes outcome:

```

Some (qh, t) ∈ set-dist (execute
  (do {
    qh ← read;
    let len = scale * clength;
    qh-path ← ntimes read (floor-log len);

```

```

    ap ← check-authentication-path len i qh qh-path;
    assert (ap = fr);
    return qh
  }) s)
shows
  ∃ path.
    length path = floor-log (scale * clength) ∧
    PTranscript s = ([qh] @ path) @ PTranscript t ∧
    PState t = foldl concat (PState s) ([qh] @ path) ∧
    s ≤ t ∧
    PQueryCounter t = PQueryCounter s
proof –
  let ?len = scale * clength
  from outcome obtain s1 path s2 ap s3 s4 where
    read-gh: Some (qh, s1) ∈ set-dist (execute read s)
    and read-path:
      Some (path, s2) ∈
        set-dist (execute (ntimes read (floor-log ?len)) s1)
    and check:
      Some (ap, s3) ∈
        set-dist (execute (check-authentication-path ?len i qh path) s2)
    and assert-ap: Some ((), s4) ∈ set-dist (execute (assert (ap = fr)) s3)
    and ret-gh: Some (qh, t) ∈ set-dist (execute (return qh) s4)
    by (auto simp: Let-def elim!: set-dist-bindE)
  from read-outcome[OF read-gh] obtain rest1 where
    tr-s: PTranscript s = qh # rest1
    and st-s1: PState s1 = concat (PState s) qh
    and tr-s1: PTranscript s1 = rest1
    and ext-s-s1: s ≤ s1
    and query-count-s1: PQueryCounter s1 = PQueryCounter s
    by blast
  have path-res:
    length path = floor-log ?len ∧
    PTranscript s1 = path @ PTranscript s2 ∧
    PState s2 = foldl concat (PState s1) path ∧
    s1 ≤ s2 ∧
    PQueryCounter s2 = PQueryCounter s1
    using ntimes-read-any-outcome[OF read-path] by simp
  have s2-s3: s2 ≤ s3
    by (rule check-authentication-path-hash-extends[OF check])
  have st-s3: PState s3 = PState s2
    using check-authentication-path-preserves-channel(1)[OF check] .
  have tr-s3: PTranscript s3 = PTranscript s2
    using check-authentication-path-preserves-channel(2)[OF check] .
  have query-count-s3: PQueryCounter s3 = PQueryCounter s2
    using check-authentication-path-preserves-channel(6)[OF check] .
  have ap-eq: ap = fr
    using assert-ap unfolding assert-def
    by (cases ap = fr) (auto simp: throw-no-outcome)

```



```

have s4-eq: s4 = s3
  using assert-ap ap-eq unfolding assert-def by simp
have t-eq: t = s4
  using ret-gh by simp
have s ≤ t
  using ext-s-s1 path-res s2-s3 unfolding t-eq s4-eq
  by (meson hash-ext-trans)
then show ?thesis
  using tr-s st-s1 tr-s1 path-res st-s3 tr-s3 query-count-s1
    query-count-s3 s4-eq t-eq
  by auto
qed

```

definition *query-decommitment-step*

where

```

query-decommitment-step fr i =
  (do {
    gh ← read;
    let len = scale * clength;
    gh-path ← ntimes read (floor-log len);
    ap ← check-authentication-path len i gh gh-path;
    assert (ap = fr);
    return gh
  })

```

lemma *query-decommitment-step-outcome:*

assumes *outcome:*

Some (gh, t) ∈ set-dist (execute (query-decommitment-step fr i) s)

shows

```

∃ path.
  length path = floor-log (scale * clength) ∧
  PTranscript s = ([gh] @ path) @ PTranscript t ∧
  PState t = foldl concat (PState s) ([gh] @ path) ∧
  s ≤ t ∧
  PQueryCounter t = PQueryCounter s

```

using *check-decommit-on-query-step-outcome[OF outcome[unfolded query-decommitment-step-def]]*

.

lemma *mmap-query-decommitment-steps-outcome:*

assumes *outcome:*

*Some (leaves, t) ∈
set-dist (execute (mmap (map (query-decommitment-step fr) idxs)) s)*

shows

```

∃ paths chunk.
  length leaves = length idxs ∧
  length paths = length idxs ∧
  chunk = List.concat (map (λ(leaf, path). [leaf] @ path) (zip leaves paths)) ∧
  (∀ path ∈ set paths. length path = floor-log (scale * clength)) ∧
  PTranscript s = chunk @ PTranscript t ∧

```

```

    PState t = foldl concat (PState s) chunk ∧
    s ≤ t ∧
    PQueryCounter t = PQueryCounter s
  using outcome
proof (induction idxs arbitrary: s leaves t)
  case Nil
  then show ?case
    by (intro exI[of - []]) (simp add: hash-ext-refl)
next
  case (Cons i idxs)
  from Cons.prem obtain leaf leaves' s1 where
    head:
      Some (leaf, s1) ∈
        set-dist (execute (query-decommitment-step fr i) s)
    and tail:
      Some (leaves', t) ∈
        set-dist (execute (mmap (map (query-decommitment-step fr) idxs)) s1)
    and leaves-eq: leaves = leaf # leaves'
    by (auto elim!: set-dist-bindE)
  from query-decommitment-step-outcome[OF head] obtain path where
    path-len: length path = floor-log (scale * clength)
    and tr-s: PTranscript s = ([leaf] @ path) @ PTranscript s1
    and st-s1: PState s1 = foldl concat (PState s) ([leaf] @ path)
    and ext-s-s1: s ≤ s1
    and query-count-s1: PQueryCounter s1 = PQueryCounter s
    by blast
  from Cons.IH[OF tail] obtain paths chunk-tail where
    len-leaves': length leaves' = length idxs
    and len-paths: length paths = length idxs
    and chunk-tail:
      chunk-tail =
        List.concat (map (λ(leaf, path). [leaf] @ path) (zip leaves' paths))
    and paths-len:
      ∀ path ∈ set paths. length path = floor-log (scale * clength)
    and tr-s1: PTranscript s1 = chunk-tail @ PTranscript t
    and st-t: PState t = foldl concat (PState s1) chunk-tail
    and ext-s1-t: s1 ≤ t
    and query-count-t: PQueryCounter t = PQueryCounter s1
    by blast
  let ?paths = path # paths
  let ?chunk = ([leaf] @ path) @ chunk-tail
  have s ≤ t
    using ext-s-s1 ext-s1-t by (rule hash-ext-trans)
  then show ?case
    apply (intro exI[of - ?paths] exI[of - ?chunk])
    using leaves-eq path-len tr-s tr-s1 st-s1 st-t len-leaves' len-paths
      chunk-tail paths-len query-count-s1 query-count-t
    by simp
qed

```

lemma *check-decommit-on-query-outcome*:
assumes *outcome*:
Some (leaves, t) ∈ set-dist (execute (mmap (check-decommit-on-query fr idx)) s)
shows
 $\exists$ *paths chunk.*
query-decommitment-transcript idx leaves paths chunk $\wedge$
PTranscript s = chunk @ PTranscript t $\wedge$
PState t = foldl concat (PState s) chunk $\wedge$
s ≤ t $\wedge$
PQueryCounter t = PQueryCounter s
proof –
have *map-eq*:
check-decommit-on-query fr idx =
map (query-decommitment-step fr) (powers-scaled idx)
unfolding *check-decommit-on-query-def query-decommitment-step-def* **by** *simp*
from *mmap-query-decommitment-steps-outcome[OF outcome[unfolded map-eq]]*
obtain *paths chunk* **where**
len-leaves: length leaves = length (powers-scaled idx)
and *len-paths: length paths = length (powers-scaled idx)*
and *chunk*:
chunk = List.concat
(map (λ(leaf, path). [leaf] @ path) (zip leaves paths))
and *paths-len*:
 $\forall$ *path ∈ set paths. length path = floor-log (scale * clength)*
and *tr-s: PTranscript s = chunk @ PTranscript t*
and *st-t: PState t = foldl concat (PState s) chunk*
and *ext: s ≤ t*
and *query-count-t: PQueryCounter t = PQueryCounter s*
by *blast*
have *qtr: query-decommitment-transcript idx leaves paths chunk*
using *len-leaves len-paths chunk paths-len*
unfolding *query-decommitment-transcript-def*
by (*simp add: mult.commute case-prod-unfold*)
show *?thesis*
using *qtr tr-s st-t ext query-count-t* **by** *blast*
qed

lemma *query-decommitment-step-preserves-query-lookup*:
assumes *outcome*:
Some (qh, t) ∈ set-dist (execute (query-decommitment-step fr i) s)
shows
fmlookup (HashMap t) (QueryIndexChallenge j x) =
fmlookup (HashMap s) (QueryIndexChallenge j x)
proof –
let *?len = scale * clength*
from *outcome* **obtain** *s1 path s2 ap s3 s4* **where**
read-qh: Some (qh, s1) ∈ set-dist (execute read s)

```

and read-path:
  Some (path, s2) ∈
    set-dist (execute (ntimes read (floor-log ?len)) s1)
and check:
  Some (ap, s3) ∈
    set-dist (execute (check-authentication-path ?len i qh path) s2)
and assert-ap: Some ((), s4) ∈ set-dist (execute (assert (ap = fr)) s3)
and ret-qh: Some (qh, t) ∈ set-dist (execute (return qh) s4)
unfolding query-decommitment-step-def
by (auto simp: Let-def elim!: set-dist-bindE)
have lookup-s1:
  fmlookup (HashMap s1) (QueryIndexChallenge j x) =
    fmlookup (HashMap s) (QueryIndexChallenge j x)
  using read-preserves-hash-map[OF read-qh] by simp
have lookup-s2:
  fmlookup (HashMap s2) (QueryIndexChallenge j x) =
    fmlookup (HashMap s1) (QueryIndexChallenge j x)
  by (rule ntimes-read-preserves-lookup[OF read-path])
have lookup-s3:
  fmlookup (HashMap s3) (QueryIndexChallenge j x) =
    fmlookup (HashMap s2) (QueryIndexChallenge j x)
  by (rule check-authentication-path-preserves-challenge-lookups(2)
    [OF check])
have s4-eq: s4 = s3
  using assert-ap unfolding assert-def
  by (cases ap = fr) (auto simp: throw-no-outcome)
have t-eq: t = s4
  using ret-qh by simp
show ?thesis
  using lookup-s1 lookup-s2 lookup-s3 s4-eq t-eq by simp
qed

```

```

lemma mmap-query-decommitment-steps-preserves-query-lookup:
assumes outcome:
  Some (leaves, t) ∈
    set-dist (execute (mmap (map (query-decommitment-step fr) idxs)) s)
shows
  fmlookup (HashMap t) (QueryIndexChallenge j x) =
    fmlookup (HashMap s) (QueryIndexChallenge j x)
using outcome
proof (induction idxs arbitrary: s leaves t)
case Nil
then show ?case
  by simp
next
case (Cons i idxs)
from Cons.prems obtain leaf leaves' s1 where
  head:
    Some (leaf, s1) ∈

```

```

    set-dist (execute (query-decommitment-step fr i) s)
  and tail:
    Some (leaves', t) ∈
      set-dist (execute (mmap (map (query-decommitment-step fr) idxs)) s1)
  by (auto elim!: set-dist-bindE)
  have head-pres:
    fmlookup (HashMap s1) (QueryIndexChallenge j x) =
      fmlookup (HashMap s) (QueryIndexChallenge j x)
  by (rule query-decommitment-step-preserves-query-lookup[OF head])
  have tail-pres:
    fmlookup (HashMap t) (QueryIndexChallenge j x) =
      fmlookup (HashMap s1) (QueryIndexChallenge j x)
  by (rule Cons.IH[OF tail])
  show ?case
    using head-pres tail-pres by simp
qed

lemma check-decommit-on-query-preserves-query-lookup:
  assumes outcome:
    Some (leaves, t) ∈
      set-dist (execute (mmap (check-decommit-on-query fr idx)) s)
  shows
    fmlookup (HashMap t) (QueryIndexChallenge j x) =
      fmlookup (HashMap s) (QueryIndexChallenge j x)
proof -
  have map-eq:
    check-decommit-on-query fr idx =
      map (query-decommitment-step fr) (powers-scaled idx)
  unfolding check-decommit-on-query-def query-decommitment-step-def by simp
  show ?thesis
    by (rule mmap-query-decommitment-steps-preserves-query-lookup
        [OF outcome[unfolded map-eq]])
qed

```

```

definition fri-layer-opening-step
  where
    fri-layer-opening-step bf st =
      (case bf of (b, f) ⇒
        case st of (i, x, len, pw) ⇒
          do {
            xp ← read;
            xp' ← ntimes read (floor-log len);
            xn ← read;
            xn' ← ntimes read (floor-log len);
            assert (xp = x);
            ap ← check-authentication-path len i xp xp';
            assert (ap = f);
            let sidx = (i + len div 2) mod len;
            ap ← check-authentication-path len sidx xn xn';

```

```

    assert (ap = f);
    let gp = (xp + xn) div 2;
    let hp = (xp - xn) div (2*((h ^ i) * shift) ^ pw);
    let x = gp + b * hp;
    return (i mod (len div 2), x, len div 2, pw+pw)
  })

```

definition *fri-layer-opening-finish*

where

```

fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path =
  do {
    assert (xp = x);
    ap ← check-authentication-path len i xp xp-path;
    assert (ap = f);
    let idx = (i + len div 2) mod len;
    ap ← check-authentication-path len idx xn xn-path;
    assert (ap = f);
    let gp = (xp + xn) div 2;
    let hp = (xp - xn) div (2*((h ^ i) * shift) ^ pw);
    let x = gp + b * hp;
    return (i mod (len div 2), x, len div 2, pw+pw)
  }

```

lemma *fri-layer-opening-step-unfold-finish*:

```

fri-layer-opening-step (b, f) (i, x, len, pw) =
  do {
    xp ← read;
    xp-path ← ntimes read (floor-log len);
    xn ← read;
    xn-path ← ntimes read (floor-log len);
    fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path
  }

```

unfolding *fri-layer-opening-step-def* *fri-layer-opening-finish-def*
by *simp*

lemma *fri-layer-opening-step-outcome*:

assumes *outcome*:

Some (out, t) ∈
set-dist (*execute* (*fri-layer-opening-step* (b, f) (i, x, len, pw)) s)

shows

```

∃ xp xp-path xn xn-path x'.
  out = (i mod (len div 2), x', len div 2, pw + pw) ∧
  xp = x ∧
  x' = fri-fold-value b xp xn
    (fri-fold-denominator ((h ^ i) * shift) pw) ∧
  fri-layer-opening-chunk len xp xp-path xn xn-path
    ([xp] @ xp-path @ [xn] @ xn-path) ∧
  PTranscript s = ([xp] @ xp-path @ [xn] @ xn-path) @ PTranscript t ∧
  PState t = foldl concat (PState s) ([xp] @ xp-path @ [xn] @ xn-path) ∧

```

$s \leq t \wedge$
 $PQueryCounter\ t = PQueryCounter\ s$
proof –
from *outcome* **obtain** $xp\ s1\ xp\text{-}path\ s2\ xn\ s3\ xn\text{-}path\ s4\ s5\ ap1\ s6\ s7$
 $ap2\ s8\ s9$ **where**
read-xp: $Some\ (xp,\ s1) \in set\text{-}dist\ (execute\ read\ s)$
and *read-xp-path*:
 $Some\ (xp\text{-}path,\ s2) \in$
 $set\text{-}dist\ (execute\ (ntimes\ read\ (floor\text{-}log\ len))\ s1)$
and *read-xn*: $Some\ (xn,\ s3) \in set\text{-}dist\ (execute\ read\ s2)$
and *read-xn-path*:
 $Some\ (xn\text{-}path,\ s4) \in$
 $set\text{-}dist\ (execute\ (ntimes\ read\ (floor\text{-}log\ len))\ s3)$
and *assert-xp*: $Some\ ((),\ s5) \in set\text{-}dist\ (execute\ (assert\ (xp = x))\ s4)$
and *check1*:
 $Some\ (ap1,\ s6) \in$
 $set\text{-}dist\ (execute\ (check\text{-}authentication\text{-}path\ len\ i\ xp\ xp\text{-}path)\ s5)$
and *assert-ap1*: $Some\ ((),\ s7) \in set\text{-}dist\ (execute\ (assert\ (ap1 = f))\ s6)$
and *check2*:
 $Some\ (ap2,\ s8) \in$
 $set\text{-}dist\ (execute$
 $(check\text{-}authentication\text{-}path\ len\ ((i + len\ div\ 2)\ mod\ len)\ xn\ xn\text{-}path)\ s7)$
and *assert-ap2*: $Some\ ((),\ s9) \in set\text{-}dist\ (execute\ (assert\ (ap2 = f))\ s8)$
and *ret*:
 $Some\ (out,\ t) \in$
 $set\text{-}dist\ (execute$
 $(return$
 $(i\ mod\ (len\ div\ 2),$
 $(xp + xn)\ div\ 2 +$
 $b * ((xp - xn)\ div\ (2 * ((h \wedge i) * shift) \wedge pw)),$
 $len\ div\ 2,\ pw + pw))\ s9)$
unfolding *fri-layer-opening-step-def*
by (*auto simp: Let-def elim!: set-dist-bindE*)
from *read-outcome*[*OF read-xp*] **obtain** *rest1* **where**
tr-s: $PTranscript\ s = xp \# rest1$
and *st-s1*: $PState\ s1 = concat\ (PState\ s)\ xp$
and *tr-s1*: $PTranscript\ s1 = rest1$
and *ext-s-s1*: $s \leq s1$
and *query-count-s1*: $PQueryCounter\ s1 = PQueryCounter\ s$
by *blast*
have *xp-path-res*:
 $length\ xp\text{-}path = floor\text{-}log\ len \wedge$
 $PTranscript\ s1 = xp\text{-}path\ @\ PTranscript\ s2 \wedge$
 $PState\ s2 = foldl\ concat\ (PState\ s1)\ xp\text{-}path \wedge$
 $s1 \leq s2 \wedge$
 $PQueryCounter\ s2 = PQueryCounter\ s1$
using *ntimes-read-any-outcome*[*OF read-xp-path*] **by** *simp*
from *read-outcome*[*OF read-xn*] **obtain** *rest3* **where**
tr-s2: $PTranscript\ s2 = xn \# rest3$

```

and st-s3: PState s3 = concat (PState s2) xn
and tr-s3: PTranscript s3 = rest3
and ext-s2-s3: s2 ≤ s3
and query-count-s3: PQueryCounter s3 = PQueryCounter s2
by blast
have xn-path-res:
  length xn-path = floor-log len ∧
    PTranscript s3 = xn-path @ PTranscript s4 ∧
    PState s4 = foldl concat (PState s3) xn-path ∧
    s3 ≤ s4 ∧
    PQueryCounter s4 = PQueryCounter s3
  using ntimes-read-any-outcome[OF read-xn-path] by simp
have xp-eq: xp = x
  using assert-xp unfolding assert-def
  by (cases xp = x) (auto simp: throw-no-outcome)
have s5-eq: s5 = s4
  using assert-xp xp-eq unfolding assert-def by simp
have s5-s6: s5 ≤ s6
  by (rule check-authentication-path-hash-extends[OF check1])
have st-s6: PState s6 = PState s5
  using check-authentication-path-preserves-channel(1)[OF check1] .
have tr-s6: PTranscript s6 = PTranscript s5
  using check-authentication-path-preserves-channel(2)[OF check1] .
have query-count-s6: PQueryCounter s6 = PQueryCounter s5
  using check-authentication-path-preserves-channel(6)[OF check1] .
have ap1-eq: ap1 = f
  using assert-ap1 unfolding assert-def
  by (cases ap1 = f) (auto simp: throw-no-outcome)
have s7-eq: s7 = s6
  using assert-ap1 ap1-eq unfolding assert-def by simp
have s7-s8: s7 ≤ s8
  by (rule check-authentication-path-hash-extends[OF check2])
have st-s8: PState s8 = PState s7
  using check-authentication-path-preserves-channel(1)[OF check2] .
have tr-s8: PTranscript s8 = PTranscript s7
  using check-authentication-path-preserves-channel(2)[OF check2] .
have query-count-s8: PQueryCounter s8 = PQueryCounter s7
  using check-authentication-path-preserves-channel(6)[OF check2] .
have ap2-eq: ap2 = f
  using assert-ap2 unfolding assert-def
  by (cases ap2 = f) (auto simp: throw-no-outcome)
have s9-eq: s9 = s8
  using assert-ap2 ap2-eq unfolding assert-def by simp
let ?x' =
  (xp + xn) div 2 +
  b * ((xp - xn) div (2 * ((h ^ i) * shift) ^ pw))
have out-t: out = (i mod (len div 2), ?x', len div 2, pw + pw) ∧ t = s9
  using ret by simp
have x'-fold:

```



```

?x' = fri-fold-value b xp xn
(fri-fold-denominator ((h ^ i) * shift) pw)
unfolding fri-fold-value-def fri-fold-denominator-def by simp
have chunk-shape:
  fri-layer-opening-chunk len xp xp-path xn xn-path
  ([xp] @ xp-path @ [xn] @ xn-path)
using xp-path-res xn-path-res unfolding fri-layer-opening-chunk-def by simp
have tr-t:
  PTranscript s = ([xp] @ xp-path @ [xn] @ xn-path) @ PTranscript t
using tr-s tr-s1 xp-path-res tr-s2 tr-s3 xn-path-res s5-eq tr-s6
  s7-eq tr-s8 s9-eq out-t
by simp
have st-t:
  PState t = foldl concat (PState s) ([xp] @ xp-path @ [xn] @ xn-path)
using st-s1 xp-path-res st-s3 xn-path-res s5-eq st-s6 s7-eq st-s8
  s9-eq out-t
by simp
have s-t: s ≤ t
using ext-s-s1 xp-path-res ext-s2-s3 xn-path-res s5-s6 s7-s8 out-t
unfolding s5-eq s7-eq s9-eq
by (meson hash-ext-trans)
have query-count-t: PQueryCounter t = PQueryCounter s
using out-t query-count-s1
  xp-path-res query-count-s3 xn-path-res query-count-s6 query-count-s8
  s5-eq s7-eq s9-eq
by simp
show ?thesis
apply (rule exI[where x=xp])
apply (rule exI[where x=xp-path])
apply (rule exI[where x=xn])
apply (rule exI[where x=xn-path])
apply (rule exI[where x=?x'])
using out-t xp-eq x'-fold chunk-shape tr-t st-t s-t query-count-t
by simp
qed

lemma mfold-fri-layer-openings-outcome:
assumes outcome:
  Some (out, t) ∈
    set-dist (execute (mfold (i, x, len, pw) (map fri-layer-opening-step bfs)) s)
shows
  ∃ layer-chunks chunk.
    fri-layers-transcript (length bfs) len layer-chunks chunk ∧
    PTranscript s = chunk @ PTranscript t ∧
    PState t = foldl concat (PState s) chunk ∧
    s ≤ t ∧
    PQueryCounter t = PQueryCounter s
using outcome
proof (induction bfs arbitrary: i x len pw s out t)

```

```

case Nil
then show ?case
  by (intro exI[of - []]) (simp add: fri-layers-transcript-def hash-ext-refl)
next
case (Cons bf bfs)
obtain b f where bf-eq: bf = (b, f)
  by (cases bf)
from Cons.prems obtain out1 s1 where
  head:
    Some (out1, s1) ∈
      set-dist (execute (fri-layer-opening-step bf (i, x, len, pw)) s)
  and tail:
    Some (out, t) ∈
      set-dist (execute (mfold out1 (map fri-layer-opening-step bfs)) s1)
  by (auto elim!: set-dist-bindE)
from fri-layer-opening-step-outcome[OF head[unfolded bf-eq]]
obtain xp xp-path xn xn-path x' where
  out1-eq: out1 = (i mod (len div 2), x', len div 2, pw + pw)
  and head-chunk:
    fri-layer-opening-chunk len xp xp-path xn xn-path
      ([xp] @ xp-path @ [xn] @ xn-path)
  and tr-s: PTranscript s = ([xp] @ xp-path @ [xn] @ xn-path) @ PTranscript s1
  and st-s1:
    PState s1 = foldl concat (PState s) ([xp] @ xp-path @ [xn] @ xn-path)
  and ext-s-s1: s ≤ s1
  and query-count-s1: PQueryCounter s1 = PQueryCounter s
  by blast
from Cons.IH[OF tail[unfolded out1-eq]]
obtain layer-chunks chunk-tail where
  tail-layers:
    fri-layers-transcript (length bfs) (len div 2) layer-chunks chunk-tail
  and tr-s1: PTranscript s1 = chunk-tail @ PTranscript t
  and st-t: PState t = foldl concat (PState s1) chunk-tail
  and ext-s1-t: s1 ≤ t
  and query-count-t: PQueryCounter t = PQueryCounter s1
  by blast
let ?head-chunk = [xp] @ xp-path @ [xn] @ xn-path
let ?chunks = ?head-chunk # layer-chunks
let ?chunk = ?head-chunk @ chunk-tail
have layers:
  fri-layers-transcript (length (bf # bfs)) len ?chunks ?chunk
  using head-chunk tail-layers
  unfolding fri-layers-transcript-def
  by auto
have s ≤ t
  using ext-s-s1 ext-s1-t by (rule hash-ext-trans)
then show ?case
  apply (intro exI[of - ?chunks] exI[of - ?chunk])
  using layers tr-s tr-s1 st-s1 st-t query-count-s1 query-count-t

```

by simp
qed

lemma *receive-query-commits-layers-outcome:*

assumes *outcome:*

Some (out, t) ∈

set-dist (execute (mfold (i, x, len, pw) (receive-query-commits bfs)) s)

shows

∃ layer-chunks chunk.

fri-layers-transcript (length bfs) len layer-chunks chunk ∧

PTranscript s = chunk @ PTranscript t ∧

PState t = foldl concat (PState s) chunk ∧

s ≤ t ∧

PQueryCounter t = PQueryCounter s

proof –

have *map-eq: receive-query-commits bfs = map fri-layer-opening-step bfs*

unfolding *receive-query-commits-def fri-layer-opening-step-def*

by (*simp add: fun-eq-iff split: prod.splits*)

show *?thesis*

using *mfold-fri-layer-openings-outcome[OF outcome[unfolded map-eq]] .*

qed

lemma *fri-layer-opening-step-preserves-query-lookup:*

assumes *outcome:*

Some (out, t) ∈ set-dist (execute (fri-layer-opening-step bf st) s)

shows

fmllookup (HashMap t) (QueryIndexChallenge j x) =

fmllookup (HashMap s) (QueryIndexChallenge j x)

proof –

obtain *b f where bf-eq: bf = (b, f)*

by (*cases bf*)

obtain *i0 x0 len pw where st-eq: st = (i0, x0, len, pw)*

by (*cases st*)

from *outcome[unfolded bf-eq st-eq fri-layer-opening-step-def]*

obtain *xp s1 xp-path s2 xn s3 xn-path s4 s5 ap1 s6 s7 ap2 s8 s9 where*

read-xp: Some (xp, s1) ∈ set-dist (execute read s)

and *read-xp-path:*

Some (xp-path, s2) ∈

set-dist (execute (ntimes read (floor-log len)) s1)

and *read-xn: Some (xn, s3) ∈ set-dist (execute read s2)*

and *read-xn-path:*

Some (xn-path, s4) ∈

set-dist (execute (ntimes read (floor-log len)) s3)

and *assert-xp:*

Some ((), s5) ∈ set-dist (execute (assert (xp = x0)) s4)

and *check1:*

Some (ap1, s6) ∈

set-dist (execute (check-authentication-path len i0 xp xp-path) s5)

and *assert-ap1:*

```

    Some ((), s7) ∈ set-dist (execute (assert (ap1 = f)) s6)
and check2:
    Some (ap2, s8) ∈
      set-dist (execute
        (check-authentication-path len ((i0 + len div 2) mod len) xn
          xn-path) s7)
and assert-ap2:
    Some ((), s9) ∈ set-dist (execute (assert (ap2 = f)) s8)
and ret:
    Some (out, t) ∈
      set-dist
        (execute
          (return (i0 mod (len div 2),
            (xp + xn) div 2 +
              b * ((xp - xn) div (2 * ((h ^ i0) * shift) ^ pw)),
            len div 2, pw + pw)) s9)
by (auto elim!: set-dist-bindE)
have lookup-s1:
  fmlookup (HashMap s1) (QueryIndexChallenge j x) =
    fmlookup (HashMap s) (QueryIndexChallenge j x)
using read-preserves-hash-map[OF read-xp] by simp
have lookup-s2:
  fmlookup (HashMap s2) (QueryIndexChallenge j x) =
    fmlookup (HashMap s1) (QueryIndexChallenge j x)
by (rule ntimes-read-preserves-lookup[OF read-xp-path])
have lookup-s3:
  fmlookup (HashMap s3) (QueryIndexChallenge j x) =
    fmlookup (HashMap s2) (QueryIndexChallenge j x)
using read-preserves-hash-map[OF read-xn] by simp
have lookup-s4:
  fmlookup (HashMap s4) (QueryIndexChallenge j x) =
    fmlookup (HashMap s3) (QueryIndexChallenge j x)
by (rule ntimes-read-preserves-lookup[OF read-xn-path])
have s5-eq: s5 = s4
using assert-xp unfolding assert-def
by (cases xp = x0) (auto simp: throw-no-outcome)
have lookup-s6:
  fmlookup (HashMap s6) (QueryIndexChallenge j x) =
    fmlookup (HashMap s5) (QueryIndexChallenge j x)
by (rule check-authentication-path-preserves-challenge-lookups(2)
  [OF check1])
have s7-eq: s7 = s6
using assert-ap1 unfolding assert-def
by (cases ap1 = f) (auto simp: throw-no-outcome)
have lookup-s8:
  fmlookup (HashMap s8) (QueryIndexChallenge j x) =
    fmlookup (HashMap s7) (QueryIndexChallenge j x)
by (rule check-authentication-path-preserves-challenge-lookups(2)
  [OF check2])

```

```

have s9-eq: s9 = s8
  using assert-ap2 unfolding assert-def
  by (cases ap2 = f) (auto simp: throw-no-outcome)
have t-eq: t = s9
  using ret by simp
show ?thesis
  using lookup-s1 lookup-s2 lookup-s3 lookup-s4 lookup-s6 lookup-s8
    s5-eq s7-eq s9-eq t-eq
  by simp
qed

lemma mfold-fri-layer-openings-preserves-query-lookup:
  assumes outcome:
    Some (out, t) ∈
      set-dist (execute (mfold st (map fri-layer-opening-step bfs)) s)
  shows
    fmlookup (HashMap t) (QueryIndexChallenge j x) =
      fmlookup (HashMap s) (QueryIndexChallenge j x)
  using outcome
proof (induction bfs arbitrary: st s out t)
  case Nil
  then show ?case
    by simp
next
  case (Cons bf bfs)
  from Cons.prem obtain out1 s1 where
    head:
      Some (out1, s1) ∈
        set-dist (execute (fri-layer-opening-step bf st) s)
  and tail:
    Some (out, t) ∈
      set-dist (execute (mfold out1 (map fri-layer-opening-step bfs)) s1)
  by (auto elim!: set-dist-bindE)
  have head-pres:
    fmlookup (HashMap s1) (QueryIndexChallenge j x) =
      fmlookup (HashMap s) (QueryIndexChallenge j x)
  by (rule fri-layer-opening-step-preserves-query-lookup[OF head])
  have tail-pres:
    fmlookup (HashMap t) (QueryIndexChallenge j x) =
      fmlookup (HashMap s1) (QueryIndexChallenge j x)
  by (rule Cons.IH[OF tail])
  show ?case
    using head-pres tail-pres by simp
qed

lemma receive-query-commits-layers-preserves-query-lookup:
  assumes outcome:
    Some (out, t) ∈
      set-dist (execute (mfold st (receive-query-commits bfs)) s)

```

shows
 $\text{fmlookup } (\text{HashMap } t) \text{ } (\text{QueryIndexChallenge } j \ x) =$
 $\text{fmlookup } (\text{HashMap } s) \text{ } (\text{QueryIndexChallenge } j \ x)$
proof –
have $\text{map-eq} : \text{receive-query-commits bfs} = \text{map fri-layer-opening-step bfs}$
unfolding $\text{receive-query-commits-def fri-layer-opening-step-def}$
by $(\text{simp add: fun-eq-iff split: prod.splits})$
show $?thesis$
by $(\text{rule mfold-fri-layer-openings-preserves-query-lookup}$
 $\text{[OF outcome[unfolded map-eq]])}$
qed

definition $\text{verifier-query-round-after-index-program}$
where
 $\text{verifier-query-round-after-index-program fr f-fl f-final as fl final raw} =$
 $(\text{let idx}' = \text{index } (\text{to-nat raw}) \text{ in}$
 $\text{do } \{$
 $\text{fv} \leftarrow \text{mmap } (\text{check-decommit-on-query fr idx}')$

 $(\text{f-i, f-x, f-len, f-pow}) \leftarrow$
 $\text{mfold } (\text{idx}', \text{hd fv, clength} * \text{scale, 1})$
 $(\text{receive-query-commits f-fl});$
 $\text{assert } (\text{f-x} = \text{f-final});$

 $(\text{i, x, len, pow}) \leftarrow$
 $\text{mfold } (\text{idx}', \text{cp-eval as fv } (h^{\wedge} \text{idx}' * \text{shift}), \text{clength} * \text{scale, 1})$
 $(\text{receive-query-commits fl});$

 $\text{assert } (\text{x} = \text{final})$
 $\})$

definition $\text{verifier-query-round-program}$
where
 $\text{verifier-query-round-program fr f-fl f-final as fl final} =$
 $(\text{do } \{$
 $\text{idx} \leftarrow \text{receive-query-index-challenge};$
 $\text{let idx}' = \text{index } (\text{to-nat idx});$
 $\text{fv} \leftarrow \text{mmap } (\text{check-decommit-on-query fr idx}')$

 $(\text{f-i, f-x, f-len, f-pow}) \leftarrow$
 $\text{mfold } (\text{idx}', \text{hd fv, clength} * \text{scale, 1}) (\text{receive-query-commits f-fl});$
 $\text{assert } (\text{f-x} = \text{f-final});$

 $(\text{i, x, len, pow}) \leftarrow$
 $\text{mfold } (\text{idx}', \text{cp-eval as fv } (h^{\wedge} \text{idx}' * \text{shift}), \text{clength} * \text{scale, 1})$
 $(\text{receive-query-commits fl});$

 $\text{assert } (\text{x} = \text{final})$
 $\})$

lemma *verifier-query-round-program-alt-def*:
verifier-query-round-program fr f-fl f-final as fl final =
 (receive-query-index-challenge $\gg$
 verifier-query-round-after-index-program fr f-fl f-final as fl final)
unfolding *verifier-query-round-program-def*
 verifier-query-round-after-index-program-def
by (*simp* add: *Let-def*)

lemma *verifier-query-round-after-index-program-outcome*:
assumes *outcome*:
 Some ($\langle \rangle$, t) $\in$
 set-dist (*execute*
 (*verifier-query-round-after-index-program* fr f-fl f-final as fl final raw) s)
shows $s \leq t \wedge P\text{QueryCounter } t = P\text{QueryCounter } s$
proof –
let $?idx = \text{index } (to\text{-nat } raw)$
from *outcome* **obtain** $fv\ s1\ f\text{-out}\ s2\ s3\ c\text{-out}\ s4$ **where**
 query-decommit:
 Some ($fv, s1$) $\in$
 set-dist (*execute* (*mmap* (*check-decommit-on-query* fr $?idx$)) s)
and *trace-fri*:
 Some ($f\text{-out}, s2$) $\in$
 set-dist (*execute*
 (*mfold* ($?idx, hd\ fv, clength * scale, 1$)
 (*receive-query-commits* f-fl)) $s1$)
and *assert-trace*:
 Some ($\langle \rangle, s3$) $\in$
 set-dist
 (*execute*
 (*assert* (*case* $f\text{-out}$ of $(-, f\text{-}x, -, -) \Rightarrow f\text{-}x = f\text{-final}$)) $s2$)
and *comp-fri*:
 Some ($c\text{-out}, s4$) $\in$
 set-dist (*execute*
 (*mfold* ($?idx, cp\text{-eval as } fv\ (h \wedge ?idx * shift),$
 $clength * scale, 1$)
 (*receive-query-commits* fl)) $s3$)
and *assert-comp*:
 Some ($\langle \rangle, t$) $\in$
 set-dist
 (*execute*
 (*assert* (*case* $c\text{-out}$ of $(-, x, -, -) \Rightarrow x = final$)) $s4$)
unfolding *verifier-query-round-after-index-program-def*
by (*auto simp: Let-def split: prod.splits elim!: set-dist-bindE*)
from *check-decommit-on-query-outcome*[*OF* *query-decommit*]
obtain *query-paths query-chunk* **where**
 $ext\text{-}s\text{-}s1: s \leq s1$
and *query-count-s1*: $P\text{QueryCounter } s1 = P\text{QueryCounter } s$
by *blast*

```

from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  ext-s1-s2:  $s1 \leq s2$ 
  and query-count-s2: PQueryCounter  $s2 = PQueryCounter\ s1$ 
  by blast
have s3-eq:  $s3 = s2$ 
  using assert-trace unfolding assert-def
  by (cases case f-out of  $(-, f-x, -, -) \Rightarrow f-x = f-final$ )
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  ext-s3-s4:  $s3 \leq s4$ 
  and query-count-s4: PQueryCounter  $s4 = PQueryCounter\ s3$ 
  by blast
have t-eq:  $t = s4$ 
  using assert-comp unfolding assert-def
  by (cases case c-out of  $(-, x, -, -) \Rightarrow x = final$ )
    (auto simp: throw-no-outcome)
have  $s \leq t$ 
  using ext-s-s1 ext-s1-s2 ext-s3-s4 t-eq unfolding s3-eq
  by (meson hash-ext-trans)
moreover have PQueryCounter  $t = PQueryCounter\ s$ 
  using query-count-s1 query-count-s2 query-count-s4 s3-eq t-eq by simp
ultimately show ?thesis
  by simp
qed

```

```

lemma verifier-query-round-after-index-program-preserves-query-lookup:
  assumes outcome:
    Some  $(((), t) \in$ 
      set-dist (execute
        (verifier-query-round-after-index-program fr f-fl f-final as fl final raw) s)
  shows
    fmlookup (HashMap t) (QueryIndexChallenge j x) =
      fmlookup (HashMap s) (QueryIndexChallenge j x)
proof -
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some  $(fv, s1) \in$ 
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
  and trace-fri:
    Some  $(f-out, s2) \in$ 
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some  $(((), s3) \in$ 
      set-dist

```



```

      (execute
        (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
    and comp-fri:
      Some (c-out, s4) ∈
        set-dist (execute
          (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
            clength * scale, 1)
            (receive-query-commits fl)) s3)
    and assert-comp:
      Some (( ), t) ∈
        set-dist
          (execute
            (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
    unfolding verifier-query-round-after-index-program-def
    by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
  have lookup-s1:
    fmlookup (HashMap s1) (QueryIndexChallenge j x) =
      fmlookup (HashMap s) (QueryIndexChallenge j x)
  by (rule check-decommit-on-query-preserves-query-lookup
    [OF query-decommit])
  have lookup-s2:
    fmlookup (HashMap s2) (QueryIndexChallenge j x) =
      fmlookup (HashMap s1) (QueryIndexChallenge j x)
  by (rule receive-query-commits-layers-preserves-query-lookup
    [OF trace-fri])
  have s3-eq: s3 = s2
    using assert-trace unfolding assert-def
    by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
      (auto simp: throw-no-outcome)
  have lookup-s4:
    fmlookup (HashMap s4) (QueryIndexChallenge j x) =
      fmlookup (HashMap s3) (QueryIndexChallenge j x)
  by (rule receive-query-commits-layers-preserves-query-lookup
    [OF comp-fri])
  have t-eq: t = s4
    using assert-comp unfolding assert-def
    by (cases case c-out of (-, x, -, -) ⇒ x = final)
      (auto simp: throw-no-outcome)
  show ?thesis
    using lookup-s1 lookup-s2 lookup-s4 s3-eq t-eq by simp
qed

lemma receive-query-index-challenge-preserves-query-future-fresh:
  assumes future: query-future-fresh s
  and outcome:
    Some (raw, t) ∈ set-dist (execute receive-query-index-challenge s)
  shows query-future-fresh t
proof -
  have counter-t: PQueryCounter t = Suc (PQueryCounter s)

```

```

    using receive-query-index-challenge-counter-outcome[OF outcome] by simp
show ?thesis
  unfolding query-future-fresh-def
proof (intro allI impI)
  fix i x
  assume i-ge-t: PQueryCounter t ≤ i
  have i-ge-s: PQueryCounter s ≤ i
    using i-ge-t counter-t by simp
  have neg:
    QueryIndexChallenge i x ≠
      QueryIndexChallenge (PQueryCounter s) (PState s)
    using i-ge-t counter-t by auto
  have fmllookup (HashMap t) (QueryIndexChallenge i x) =
    fmllookup (HashMap s) (QueryIndexChallenge i x)
    by (rule receive-query-index-challenge-preserves-other-lookup
      [OF outcome neg])
  also have ... = None
    using future i-ge-s unfolding query-future-fresh-def by blast
  finally show fmllookup (HashMap t) (QueryIndexChallenge i x) = None .
qed
qed

lemma verifier-query-round-program-preserves-query-future-fresh:
  assumes future: query-future-fresh s
  and outcome:
    Some ((), t) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  shows query-future-fresh t
proof -
  from outcome[unfolded verifier-query-round-program-alt-def]
  obtain raw s0 where
    rand:
      Some (raw, s0) ∈
        set-dist (execute receive-query-index-challenge s)
  and tail:
    Some ((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program fr f-fl f-final as fl
            final raw) s0)
  by (auto elim!: set-dist-bindE)
  have future-s0: query-future-fresh s0
    by (rule receive-query-index-challenge-preserves-query-future-fresh
      [OF future rand])
  have counter-t: PQueryCounter t = PQueryCounter s0
    using verifier-query-round-after-index-program-outcome[OF tail] by simp
  show ?thesis

```

```

unfolding query-future-fresh-def
proof (intro allI impI)
  fix i x
  assume i-ge-t: PQueryCounter t ≤ i
  have i-ge-s0: PQueryCounter s0 ≤ i
    using i-ge-t counter-t by simp
  have fmlookup (HashMap t) (QueryIndexChallenge i x) =
    fmlookup (HashMap s0) (QueryIndexChallenge i x)
    by (rule verifier-query-round-after-index-program-preserves-query-lookup
      [OF tail])
  also have ... = None
    using future-s0 i-ge-s0 unfolding query-future-fresh-def by blast
  finally show fmlookup (HashMap t) (QueryIndexChallenge i x) = None .
qed
qed

lemma ntimes-verifier-query-round-program-preserves-query-future-fresh:
assumes outcome:
  Some (results, t) ∈
    set-dist
      (execute
        (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
          n) s)
  and future: query-future-fresh s
shows query-future-fresh t
using outcome future
proof (induction n arbitrary: s results t)
case 0
then show ?case
  by simp
next
case (Suc n)
from Suc.premis obtain u results' s1 where
  head:
    Some (u, s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results', t) ∈
      set-dist
        (execute
          (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
            n) s1)
  by (auto elim!: set-dist-bindE)
have u-eq: u = ()
  by (cases u) simp
have future-s1: query-future-fresh s1
  by (rule verifier-query-round-program-preserves-query-future-fresh

```

```

    [OF Suc.premis(2) head[unfolded u-eq]])
  show ?case
  by (rule Suc.IH[OF tail future-s1])
qed

lemma verifier-query-round-program-outcome:
  assumes outcome:
    Some ((), t) ∈
      set-dist (execute (verifier-query-round-program fr f-fl f-final as fl final) s)
  obtains raw-idx idx chunk
  where
    idx = index (to-nat raw-idx)
  and verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
  and PTranscript s = chunk @ PTranscript t
  and PState t = foldl concat (PState s) chunk
  and s ≤ t
  and PQueryCounter t = Suc (PQueryCounter s)
  and fmlookup (HashMap t) (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some
raw-idx
proof -
  from outcome obtain raw-idx s0 idx fv s1 f-out s2 s3 c-out s4 where
  rand:
    Some (raw-idx, s0) ∈
      set-dist (execute receive-query-index-challenge s)
  and idx-def: idx = index (to-nat raw-idx)
  and query-decommit:
    Some (fv, s1) ∈
      set-dist (execute (mmap (check-decommit-on-query fr idx)) s0)
  and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (idx, hd fv, clength * scale, 1) (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some ((), s3) ∈ set-dist (execute (assert (case f-out of (_, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold
          (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some ((), t) ∈ set-dist (execute (assert (case c-out of (_, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have rand-res:
  s ≤ s0 ∧
  PState s0 = PState s ∧
  PTranscript s0 = PTranscript s ∧
  fmlookup (HashMap s0) (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some

```

raw-idx

```

using receive-query-index-challenge-outcome[OF rand] .
have rand-counter:
  PQueryCounter s0 = Suc (PQueryCounter s)
  using receive-query-index-challenge-counter-outcome[OF rand] by simp
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript idx fv query-paths query-chunk
  and tr-s0: PTranscript s0 = query-chunk @ PTranscript s1
  and st-s1: PState s1 = foldl concat (PState s0) query-chunk
  and ext-s0-s1: s0 ≤ s1
  and query-count-s1: PQueryCounter s1 = PQueryCounter s0
  by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  trace-layers:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
  and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
  and st-s2: PState s2 = foldl concat (PState s1) trace-fri-chunk
  and ext-s1-s2: s1 ≤ s2
  and query-count-s2: PQueryCounter s2 = PQueryCounter s1
  by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk
  and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
  and st-s4: PState s4 = foldl concat (PState s3) comp-fri-chunk
  and ext-s3-s4: s3 ≤ s4
  and query-count-s4: PQueryCounter s4 = PQueryCounter s3
  by blast
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
let ?chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have round-chunk:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl) ?chunk
  unfolding verifier-query-round-chunk-def
  apply (rule exI[where x=query-chunk])
  apply (rule exI[where x=trace-fri-chunk])
  apply (rule exI[where x=comp-fri-chunk])

```

```

apply (rule exI[where  $x=fv$ ])
apply (rule exI[where  $x=query-paths$ ])
apply (rule exI[where  $x=trace-layer-chunks$ ])
apply (rule exI[where  $x=comp-layer-chunks$ ])
using query-chunk-shape trace-layers comp-layers
by simp
have tr-t: PTranscript  $s = ?chunk @ PTranscript t$ 
  using rand-res tr-s0 tr-s1 tr-s3 s3-eq t-eq by simp
have st-t: PState  $t = foldl concat (PState s) ?chunk$ 
  using rand-res st-s1 st-s2 st-s4 s3-eq t-eq by simp
have ext-s0-t:  $s0 \leq t$ 
  using ext-s0-s1 ext-s1-s2 ext-s3-s4 t-eq unfolding s3-eq
  by (meson hash-ext-trans)
have ext-s-t:  $s \leq t$ 
  using rand-res ext-s0-t by (meson hash-ext-trans)
have lookup-t: fmlookup (HashMap  $t$ ) (QueryIndexChallenge (PQueryCounter  $s$ ) (PState  $s$ ))
= Some raw-idx
  using rand-res hash-extension-lookup[OF conjunct2[OF conjunct2[OF conjunct2[OF rand-res]]]
ext-s0-t]
  by simp
have query-count-t: PQueryCounter  $t = Suc (PQueryCounter s)$ 
  using rand-counter query-count-s1 query-count-s2 query-count-s4 s3-eq t-eq
  by simp
show ?thesis
  by (rule that[OF idx-def round-chunk tr-t st-t ext-s-t query-count-t lookup-t])
qed

```

lemma *ntimes-verifier-query-rounds-outcome:*

assumes *outcome:*

Some (results, t) ∈

set-dist (execute

(ntimes (verifier-query-round-program fr f-fl f-final as fl final) n) s)

shows

$\exists$ raw-idxs query-idxs query-chunks.

$length\ raw-idxs = n \wedge$

$query-idxs = map (\lambda raw. index (to-nat raw))\ raw-idxs \wedge$

$length\ query-chunks = n \wedge$

$PTranscript\ s = List.concat\ query-chunks @ PTranscript\ t \wedge$

$PState\ t = state-after-query-chunks (PState\ s)\ query-chunks\ n \wedge$

$s \leq t \wedge$

$PQueryCounter\ t = PQueryCounter\ s + n \wedge$

$(\forall i < n.$

$verifier-query-round-chunk\ (query-idxs\ !\ i)$

$(map\ snd\ f-fl)\ (map\ snd\ fl)\ (query-chunks\ !\ i)) \wedge$

$(\forall i < n.$

$fmlookup\ (HashMap\ t)$

$(QueryIndexChallenge\ (PQueryCounter\ s + i)$

$(state-after-query-chunks\ (PState\ s)\ query-chunks\ i)) =$

$Some\ (raw-idxs\ !\ i)) \wedge$

```

    (∀ idx ∈ set query-idxs. idx < clength * scale)
  using outcome
proof (induction n arbitrary: s results t)
  case 0
  then show ?case
    by (intro exI[of - []]) (simp add: state-after-query-chunks-def hash-ext-refl)
next
  case (Suc n)
  from Suc.premis obtain u results' s1 where
    head:
      Some (u, s1) ∈
        set-dist (execute (verifier-query-round-program fr f-fl f-final as fl final) s)
    and tail:
      Some (results', t) ∈
        set-dist (execute
          (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n) s1)
    and results-eq: results = u # results'
  by (auto elim!: set-dist-bindE)
  have u-unit: u = ()
  by (cases u) simp
  from verifier-query-round-program-outcome[OF head[unfolded u-unit]]
  obtain raw idx chunk where
    idx-def: idx = index (to-nat raw)
    and round-chunk:
      verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
    and tr-s: PTranscript s = chunk @ PTranscript s1
    and st-s1: PState s1 = foldl concat (PState s) chunk
    and ext-s-s1: s ≤ s1
    and query-count-s1: PQueryCounter s1 = Suc (PQueryCounter s)
    and lookup-s1: fmlookup (HashMap s1) (QueryIndexChallenge (PQueryCounter s) (PState
s)) = Some raw
  by this
  from Suc.IH[OF tail] obtain raw-tail idx-tail chunk-tail where
    len-raw-tail: length raw-tail = n
    and idx-tail-def: idx-tail = map (λraw. index (to-nat raw)) raw-tail
    and len-chunk-tail: length chunk-tail = n
    and tr-s1: PTranscript s1 = List.concat chunk-tail @ PTranscript t
    and st-t-tail:
      PState t = state-after-query-chunks (PState s1) chunk-tail n
    and ext-s1-t: s1 ≤ t
    and query-count-t: PQueryCounter t = PQueryCounter s1 + n
    and round-tail:
      ∧ i. i < n ⇒
        verifier-query-round-chunk (idx-tail ! i)
          (map snd f-fl) (map snd fl) (chunk-tail ! i)
    and lookup-tail:
      ∧ i. i < n ⇒
        fmlookup (HashMap t)
          (QueryIndexChallenge (PQueryCounter s1 + i)

```

```

      (state-after-query-chunks (PState s1) chunk-tail i)) =
    Some (raw-tail ! i)
  and idx-tail-bound:
     $\forall idx \in \text{set } \text{idx-tail}. \text{idx} < \text{clength} * \text{scale}$ 
  by blast
  let ?raws = raw # raw-tail
  let ?idxs = idx # idx-tail
  let ?chunks = chunk # chunk-tail
  have idxs-def: ?idxs = map ( $\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})$ ) ?raws
    using idx-def idx-tail-def by simp
  have tr-all: PTranscript s = List.concat ?chunks @ PTranscript t
    using tr-s tr-s1 by simp
  have state-shift:
     $\bigwedge i. \text{state-after-query-chunks } (PState s) \text{ ?chunks } (Suc i) =$ 
      state-after-query-chunks (PState s1) chunk-tail i
    using st-s1 unfolding state-after-query-chunks-def by simp
  have st-all:
    PState t = state-after-query-chunks (PState s) ?chunks (Suc n)
    using st-t-tail state-shift[of n] by simp
  have ext-s-t:  $s \leq t$ 
    using ext-s-s1 ext-s1-t by (rule hash-ext-trans)
  have query-count-all: PQueryCounter t = PQueryCounter s + Suc n
    using query-count-s1 query-count-t by simp
  have round-all:
     $\bigwedge i. i < Suc n \implies$ 
      verifier-query-round-chunk (?idxs ! i)
      (map snd f-fl) (map snd fl) (?chunks ! i)
  proof -
    fix i
    assume i-bound:  $i < Suc n$ 
    show verifier-query-round-chunk (?idxs ! i)
      (map snd f-fl) (map snd fl) (?chunks ! i)
    proof (cases i)
      case 0
      then show ?thesis
        using round-chunk by simp
    next
      case (Suc j)
      then show ?thesis
        using round-tail[of j] i-bound by simp
    qed
  qed
  have lookup-head-t:
    fmlookup (HashMap t) (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
    using hash-extension-lookup[OF lookup-s1 ext-s1-t] .
  have lookup-all:
     $\bigwedge i. i < Suc n \implies$ 
      fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)

```



```

      (state-after-query-chunks (PState s) ?chunks i)) =
      Some (?rows ! i)
proof -
  fix i
  assume i-bound: i < Suc n
  show fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + i)
      (state-after-query-chunks (PState s) ?chunks i)) =
      Some (?rows ! i)
  proof (cases i)
    case 0
    then show ?thesis
      using lookup-head-t unfolding state-after-query-chunks-def by simp
  next
    case (Suc j)
    then show ?thesis
      using lookup-tail[of j] i-bound state-shift[of j] query-count-s1 by simp
  qed
qed
have idx-bound-all:
   $\forall idx \in \text{set } ?idxs. \text{idx} < \text{clength} * \text{scale}$ 
  using idx-tail-bound idx-def index-less-domain by auto
show ?case
  apply (intro exI[of - ?rows] exI[of - ?idxs] exI[of - ?chunks] conjI)
    apply (simp add: len-raw-tail)
    apply (rule idxs-def)
    apply (simp add: len-chunk-tail)
    apply (rule tr-all)
    apply (rule st-all)
    apply (rule ext-s-t)
    apply (rule query-count-all)
    apply (intro allI impI, rule round-all, assumption)
    apply (intro allI impI, rule lookup-all, assumption)
  using idx-bound-all by simp
qed

lemma verifier-query-round-program-preserves-past-query-lookup:
  assumes past: j < PQueryCounter s
  and outcome:
    Some (( ), t)  $\in$ 
      set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
  shows
    fmlookup (HashMap t) (QueryIndexChallenge j x) =
    fmlookup (HashMap s) (QueryIndexChallenge j x)
proof -
  from outcome[unfolded verifier-query-round-program-alt-def]
  obtain raw s0 where

```

```

  rand:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
  and tail:
    Some ((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program fr f-fl f-final as fl
            final raw) s0)
  by (auto elim!: set-dist-bindE)
  have neg:
    QueryIndexChallenge j x ≠
      QueryIndexChallenge (PQueryCounter s) (PState s)
  using past by auto
  have lookup-s0:
    fmlookup (HashMap s0) (QueryIndexChallenge j x) =
      fmlookup (HashMap s) (QueryIndexChallenge j x)
  by (rule receive-query-index-challenge-preserves-other-lookup
    [OF rand neg])
  have lookup-t:
    fmlookup (HashMap t) (QueryIndexChallenge j x) =
      fmlookup (HashMap s0) (QueryIndexChallenge j x)
  by (rule verifier-query-round-after-index-program-preserves-query-lookup
    [OF tail])
  show ?thesis
    using lookup-s0 lookup-t by simp
qed

lemma ntimes-verifier-query-round-program-preserves-past-query-lookup:
  assumes past: j < PQueryCounter s
  and outcome:
    Some (results, t) ∈
      set-dist
        (execute
          (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
            n) s)
  shows
    fmlookup (HashMap t) (QueryIndexChallenge j x) =
      fmlookup (HashMap s) (QueryIndexChallenge j x)
  using outcome past
  proof (induction n arbitrary: s results t)
    case 0
    then show ?case
      by simp
  next
    case (Suc n)
    from Suc.premis obtain u results' s1 where
      head:
        Some (u, s1) ∈

```

```

    set-dist
    (execute
      (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results', t) ∈
      set-dist
      (execute
        (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
          n) s1)
  by (auto elim!: set-dist-bindE)
have u-eq: u = ()
  by (cases u) simp
have head-pres:
  fmlookup (HashMap s1) (QueryIndexChallenge j x) =
  fmlookup (HashMap s) (QueryIndexChallenge j x)
  by (rule verifier-query-round-program-preserves-past-query-lookup
    [OF Suc.premis(2) head[unfolded u-eq]])
have counter-s1:
  PQueryCounter s1 = Suc (PQueryCounter s)
  using verifier-query-round-program-outcome[OF head[unfolded u-eq]]
  by blast
have past-s1: j < PQueryCounter s1
  using Suc.premis(2) counter-s1 by simp
have tail-pres:
  fmlookup (HashMap t) (QueryIndexChallenge j x) =
  fmlookup (HashMap s1) (QueryIndexChallenge j x)
  by (rule Suc.IH[OF tail past-s1])
show ?case
  using head-pres tail-pres by simp
qed

lemma receive-fri-commits-outcome:
  assumes outcome: Some ((b, r), t) ∈ set-dist (execute receive-fri-commits s)
  shows
    ∃ rest.
      PTranscript s = r # rest ∧
      PState t = concat (PState s) r ∧
      PTranscript t = rest ∧
      s ≤ t ∧
      fmlookup (HashMap t) (FiatShamirChallenge (concat (PState s) r)) = Some b
proof -
  from outcome obtain s-read where
    read-r: Some (r, s-read) ∈ set-dist (execute read s)
  and rand-b:
    Some (b, t) ∈ set-dist (execute receive-random-field-element s-read)
  unfolding receive-fri-commits-def
  by (auto elim!: set-dist-bindE)
obtain rest where tr-s: PTranscript s = r # rest
proof -

```

```

have PTranscript s ≠ []
  using read-r unfolding read-def protocol-read-def assert-def
  by (cases PTranscript s) (auto simp: throw-no-outcome elim!: set-dist-bindE)
then obtain x xs where xs: PTranscript s = x # xs
  by (cases PTranscript s) auto
have r-x: r = x
  using read-nonempty-outcome[OF read-r xs] by simp
show ?thesis
  using that xs r-x by auto
qed
have read-res:
  r = r ∧
  s-read = s(PState := concat (PState s) r, PTranscript := rest)
  using read-nonempty-outcome[OF read-r tr-s] by simp
have rand-res:
  s-read ≤ t ∧
  PState t = PState s-read ∧
  PTranscript t = PTranscript s-read ∧
  fmlookup (HashMap t) (FiatShamirChallenge (PState s-read)) = Some b
  using receive-random-field-element-outcome[OF rand-b] .
have s-read-ext: s ≤ s-read
  using read-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have s-t: s ≤ t
  using s-read-ext rand-res by (meson hash-ext-trans)
show ?thesis
  using tr-s read-res rand-res s-t by auto
qed

lemma ntimes-receive-fri-commits-outcome:
  assumes outcome:
    Some (brs, t) ∈
      set-dist (execute (ntimes receive-fri-commits n) s)
  shows
    length brs = n ∧
    PTranscript s = map snd brs @ PTranscript t ∧
    PState t = foldl concat (PState s) (map snd brs) ∧
    s ≤ t ∧
    (∀ i < n.
      fmlookup (HashMap t)
        (FiatShamirChallenge (foldl concat (PState s) (take (Suc i) (map snd brs)))) =
        Some (fst (brs ! i)))
  using outcome
proof (induction n arbitrary: s brs t)
  case 0
  then show ?case
    by (simp add: hash-ext-refl)
next
  case (Suc n)
  from Suc.premis obtain br brs' s1 where

```

```

  head: Some (br, s1) ∈ set-dist (execute receive-fri-commits s)
and tail:
  Some (brs', t) ∈
    set-dist (execute (ntimes receive-fri-commits n) s1)
and brs-eq: brs = br # brs'
by (auto elim!: set-dist-bindE)
obtain b r where br-eq: br = (b, r)
by (cases br)
from receive-fri-commits-outcome[OF head[unfolded br-eq]]
obtain rest1 where
  tr-s: PTranscript s = r # rest1
and st-s1: PState s1 = concat (PState s) r
and tr-s1: PTranscript s1 = rest1
and ext-head: s ≤ s1
and lookup-head:
  fmlookup (HashMap s1) (FiatShamirChallenge (concat (PState s) r)) = Some b
by blast
have tail-res:
  length brs' = n ∧
  PTranscript s1 = map snd brs' @ PTranscript t ∧
  PState t = foldl concat (PState s1) (map snd brs') ∧
  s1 ≤ t ∧
  (∀ i < n.
    fmlookup (HashMap t)
      (FiatShamirChallenge (foldl concat (PState s1) (take (Suc i) (map snd brs')))) =
      Some (fst (brs' ! i)))
  using Suc.IH[OF tail] .
have ext-s1-t: s1 ≤ t
  using tail-res by simp
have lookup-head-t:
  fmlookup (HashMap t) (FiatShamirChallenge (concat (PState s) r)) = Some b
  using hash-extension-lookup[OF lookup-head ext-s1-t] .
have ext-combined: s ≤ t
  using ext-head ext-s1-t by (meson hash-ext-trans)
have lookups:
  ∀ i < Suc n.
    fmlookup (HashMap t)
      (FiatShamirChallenge (foldl concat (PState s) (take (Suc i) (map snd brs')))) =
      Some (fst (brs ! i))
proof (intro allI impI)
  fix i
  assume i-bound: i < Suc n
  show fmlookup (HashMap t)
    (FiatShamirChallenge (foldl concat (PState s) (take (Suc i) (map snd brs')))) =
    Some (fst (brs ! i))
  proof (cases i)
    case 0
    then show ?thesis
      using lookup-head-t brs-eq br-eq by simp

```

```

next
  case (Suc j)
  then have j-bound:  $j < n$ 
    using i-bound by simp
  have key-eq:
    foldl concat (PState s) (take (Suc i) (map snd brs)) =
      foldl concat (PState s1) (take (Suc j) (map snd brs'))
    using Suc brs-eq br-eq st-s1 by simp
  show ?thesis
    using tail-res j-bound Suc brs-eq key-eq by simp
qed
qed
show ?case
  using tail-res tr-s tr-s1 st-s1 ext-combined lookups brs-eq br-eq
  by simp
qed

end

end

```

```

theory Soundness-Execution-Prefix
imports Soundness-Execution-Query
begin

```

Verifier alpha/FRI prefix execution and challenge-space bounds.

```

context soundness
begin

```

```

definition alpha-round
where
  alpha-round =
    (do {
      a0 ← receive-alpha-challenge;
      let a0' = a0;
      a1 ← read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    })

```

```

definition verifier-alpha-prefix
:: ('f × ('f × 'f) list × 'f × 'f list,
   ('f, 'a) protocol-channel-scheme) state-monad
where
  verifier-alpha-prefix =
    (do {
      fr ← read;

```

```

    f-fl ← ntimes receive-trace-fri-commits (ceil-log clength);
    f-final ← read;
    as ← mmap (replicate (length spec) alpha-round);
    return (fr, f-fl, f-final, as)
  })

```

definition *verifier-trace-fri-prefix*

```

:: ('f × ('f × 'f) list,
   ('f, 'a) protocol-channel-scheme) state-monad

```

where

```

verifier-trace-fri-prefix =
  (do {
    fr ← read;
    f-fl ← ntimes receive-trace-fri-commits (ceil-log clength);
    return (fr, f-fl)
  })

```

definition *verifier-after-trace-fri*

```

:: 'f × ('f × 'f) list ⇒
   (unit list, ('f, 'a) protocol-channel-scheme) state-monad

```

where

```

verifier-after-trace-fri header =
  (case header of (fr, f-fl) ⇒
    do {
      f-final ← read;
      as ← mmap (replicate (length spec) alpha-round);
      dg ← read;
      assert (to-nat dg ≤ maxDegree);
      fl ← ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1));
      final ← read;
      ntimes (verifier-query-round-program fr f-fl f-final as fl final)
        rounds
    })

```

definition *verifier-composition-fri-prefix*

```

:: ('f × ('f × 'f) list × 'f × 'f list ×
   'f × ('f × 'f) list,
   ('f, 'a) protocol-channel-scheme) state-monad

```

where

```

verifier-composition-fri-prefix =
  (do {
    fr ← read;
    f-fl ← ntimes receive-trace-fri-commits (ceil-log clength);
    f-final ← read;
    as ← mmap (replicate (length spec) alpha-round);
    dg ← read;
    assert (to-nat dg ≤ maxDegree);
    fl ← ntimes receive-composition-fri-commits

```

```

      (ceil-log (to-nat dg + 1));
    return (fr, f-fl, f-final, as, dg, fl)
  })

```

definition *verifier-after-composition-fri*

```

:: 'f × ('f × 'f) list × 'f × 'f list ×
   'f × ('f × 'f) list ⇒
   (unit list, ('f, 'a) protocol-channel-scheme) state-monad

```

where

```

verifier-after-composition-fri header =
  (case header of (fr, f-fl, f-final, as, dg, fl) ⇒
    do {
      final ← read;
      ntimes (verifier-query-round-program fr f-fl f-final as fl final)
        rounds
    })

```

definition *verifier-after-alpha*

```

:: 'f × ('f × 'f) list × 'f × 'f list ⇒
   (unit list, ('f, 'a) protocol-channel-scheme) state-monad

```

where

```

verifier-after-alpha header =
  (case header of (fr, f-fl, f-final, as) ⇒
    do {
      dg ← read;
      assert (to-nat dg ≤ maxDegree);
      fl ← ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1));
      final ← read;
      ntimes (verifier-query-round-program fr f-fl f-final as fl final)
        rounds
    })

```

lemma *verify-monad-alpha-decomposition:*

```

verify-monad = (verifier-alpha-prefix ≫= verifier-after-alpha)
unfolding verify-monad-def verifier-alpha-prefix-def verifier-after-alpha-def
  alpha-round-def verifier-query-round-program-def
by (simp add: Let-def sm-bind-assoc sm-bind-return-map split: prod.splits)

```

lemma *verify-monad-trace-fri-decomposition:*

```

verify-monad = (verifier-trace-fri-prefix ≫= verifier-after-trace-fri)
unfolding verify-monad-def verifier-trace-fri-prefix-def
  verifier-after-trace-fri-def alpha-round-def verifier-query-round-program-def
by (simp add: Let-def sm-bind-assoc sm-bind-return-map split: prod.splits)

```

lemma *verify-monad-composition-fri-decomposition:*

```

verify-monad =
  (verifier-composition-fri-prefix ≫= verifier-after-composition-fri)
unfolding verify-monad-def verifier-composition-fri-prefix-def

```



```

verifier-after-composition-fri-def alpha-round-def
verifier-query-round-program-def
by (simp add: Let-def sm-bind-assoc sm-bind-return-map split: prod.splits)

lemma alpha-round-outcome:
  assumes outcome: Some (a, t) ∈ set-dist (execute alpha-round s)
  shows
    ∃ rest.
      PTranscript s = a # rest ∧
      PState t = concat (PState s) a ∧
      PTranscript t = rest ∧
      s ≤ t ∧
      PTraceFriCounter t = PTraceFriCounter s ∧
      PCompositionFriCounter t = PCompositionFriCounter s ∧
      PAlphaCounter t = Suc (PAlphaCounter s) ∧
      PQueryCounter t = PQueryCounter s ∧
      fmlookup (HashMap t) (AlphaChallenge (PAlphaCounter s) (PState s)) = Some a
  proof -
    from outcome obtain a0 s0 a1 s1 s2 where
      rand: Some (a0, s0) ∈ set-dist (execute receive-alpha-challenge s)
      and read-a: Some (a1, s1) ∈ set-dist (execute read s0)
      and assert-a: Some ((), s2) ∈ set-dist (execute (assert (a0 = a1)) s1)
      and ret-a: Some (a, t) ∈ set-dist (execute (return a1) s2)
      unfolding alpha-round-def
      by (auto simp: Let-def elim!: set-dist-bindE)
    have rand-res:
      s ≤ s0 ∧
      PState s0 = PState s ∧
      PTranscript s0 = PTranscript s ∧
      fmlookup (HashMap s0) (AlphaChallenge (PAlphaCounter s) (PState s)) = Some a0
      using receive-alpha-challenge-outcome[OF rand] .
    have rand-counter:
      PTraceFriCounter s0 = PTraceFriCounter s ∧
      PCompositionFriCounter s0 = PCompositionFriCounter s ∧
      PAlphaCounter s0 = Suc (PAlphaCounter s) ∧
      PQueryCounter s0 = PQueryCounter s
      using receive-alpha-challenge-counter-outcome[OF rand] by simp
    obtain rest where tr-s0: PTranscript s0 = a1 # rest
    proof -
      have PTranscript s0 ≠ []
        using read-a unfolding read-def protocol-read-def assert-def
        by (cases PTranscript s0) (auto simp: throw-no-outcome elim!: set-dist-bindE)
      then obtain x xs where xs: PTranscript s0 = x # xs
        by (cases PTranscript s0) auto
      have a1-x: a1 = x
        using read-nonempty-outcome[OF read-a xs] by simp
      show ?thesis
        using that xs a1-x by auto
    qed
  qed

```

```

have read-res:
  s1 = s0(PState := concat (PState s0) a1, PTranscript := rest)
  using read-nonempty-outcome[OF read-a tr-s0] by simp
have a-eq: a0 = a1
  using assert-a unfolding assert-def
  by (cases a0 = a1) (auto simp: throw-no-outcome)
have s2-eq: s2 = s1
  using assert-a a-eq unfolding assert-def by simp
have t-eq: t = s1 and a-ret: a = a1
  using ret-a s2-eq by simp-all
have lookup-t: fmllookup (HashMap t) (AlphaChallenge (PAlphaCounter s) (PState s)) = Some
a
  using rand-res read-res t-eq a-ret a-eq
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have ext: s ≤ t
  using rand-res read-res t-eq a-ret a-eq
  unfolding less-eq-hash-ext-def less-eq-fmap-def by auto
show ?thesis
  using rand-res rand-counter tr-s0 read-res t-eq a-ret a-eq lookup-t ext
  by auto
qed

lemma alpha-round-preserves-alpha-future-fresh:
  assumes future: alpha-future-fresh s
  and outcome: Some (a, t) ∈ set-dist (execute alpha-round s)
  shows alpha-future-fresh t
proof -
  from outcome obtain a0 s0 a1 s1 s2 where
    rand: Some (a0, s0) ∈ set-dist (execute receive-alpha-challenge s)
    and read-a: Some (a1, s1) ∈ set-dist (execute read s0)
    and assert-a: Some ((), s2) ∈ set-dist (execute (assert (a0 = a1))) s1
    and ret-a: Some (a, t) ∈ set-dist (execute (return a1) s2)
    unfolding alpha-round-def
    by (auto simp: Let-def elim!: set-dist-bindE)
  have counter-t: PAlphaCounter t = Suc (PAlphaCounter s)
    using alpha-round-outcome[OF outcome] by blast
  have future-s0:
    ∧ i x. Suc (PAlphaCounter s) ≤ i ⇒
      fmllookup (HashMap s0) (AlphaChallenge i x) = None
  proof -
    fix i x
    assume i-ge: Suc (PAlphaCounter s) ≤ i
    have neg:
      AlphaChallenge i x ≠
        AlphaChallenge (PAlphaCounter s) (PState s)
      using i-ge by auto
    have fmllookup (HashMap s0) (AlphaChallenge i x) =
      fmllookup (HashMap s) (AlphaChallenge i x)
      by (rule receive-alpha-challenge-preserves-other-lookup[OF rand neg])
  end

```

```

also have ... = None
  using future i-ge unfolding alpha-future-fresh-def by auto
finally show
  fmllookup (HashMap s0) (AlphaChallenge i x) = None .
qed
have hash-s1: HashMap s1 = HashMap s0
  by (rule read-preserves-hash-map[OF read-a])
have s2-eq: s2 = s1
  using assert-a unfolding assert-def
  by (cases a0 = a1) (auto simp: throw-no-outcome)
have t-eq: t = s2
  using ret-a by simp
show ?thesis
  unfolding alpha-future-fresh-def
proof (intro allI impI)
  fix i x
  assume PAlphaCounter t ≤ i
  then have Suc (PAlphaCounter s) ≤ i
    using counter-t by simp
  then show fmllookup (HashMap t) (AlphaChallenge i x) = None
    using future-s0[of i x] hash-s1 unfolding t-eq s2-eq by simp
qed
qed

lemma alpha-round-preserves-composition-fri-future-fresh:
  assumes future: composition-fri-future-fresh s
  and outcome: Some (a, t) ∈ set-dist (execute alpha-round s)
  shows composition-fri-future-fresh t
proof -
  from outcome obtain a0 s0 a1 s1 s2 where
    rand: Some (a0, s0) ∈ set-dist (execute receive-alpha-challenge s)
  and read-a: Some (a1, s1) ∈ set-dist (execute read s0)
  and assert-a: Some ((), s2) ∈ set-dist (execute (assert (a0 = a1)) s1)
  and ret-a: Some (a, t) ∈ set-dist (execute (return a1) s2)
  unfolding alpha-round-def
  by (auto simp: Let-def elim!: set-dist-bindE)
have counter-s0:
  PCompositionFriCounter s0 = PCompositionFriCounter s
  using receive-alpha-challenge-counter-outcome[OF rand] by simp
have future-s0:
  composition-fri-future-fresh s0
  unfolding composition-fri-future-fresh-def
proof (intro allI impI)
  fix i x
  assume i-ge-s0: PCompositionFriCounter s0 ≤ i
  have i-ge-s: PCompositionFriCounter s ≤ i
    using i-ge-s0 counter-s0 by simp
  have neg:
    CompositionFriChallenge i x ≠

```

```

    AlphaChallenge (PAlphaCounter s) (PState s)
  by simp
  have fmlookup (HashMap s0) (CompositionFriChallenge i x) =
    fmlookup (HashMap s) (CompositionFriChallenge i x)
  by (rule receive-alpha-challenge-preserves-other-lookup[OF rand neg])
  also have ... = None
  using future i-ge-s
  unfolding composition-fri-future-fresh-def by blast
  finally show
    fmlookup (HashMap s0) (CompositionFriChallenge i x) = None .
qed
have future-s1:
  composition-fri-future-fresh s1
  by (rule read-preserves-composition-fri-future-fresh
    [OF future-s0 read-a])
have s2-eq: s2 = s1
  using assert-a unfolding assert-def
  by (cases a0 = a1) (auto simp: throw-no-outcome)
have t-eq: t = s2
  using ret-a by simp
show ?thesis
  using future-s1 unfolding t-eq s2-eq .
qed

lemma alpha-round-preserves-query-future-fresh:
  assumes future: query-future-fresh s
  and outcome: Some (a, t) ∈ set-dist (execute alpha-round s)
  shows query-future-fresh t
proof -
  from outcome obtain a0 s0 a1 s1 s2 where
    rand: Some (a0, s0) ∈ set-dist (execute receive-alpha-challenge s)
    and read-a: Some (a1, s1) ∈ set-dist (execute read s0)
    and assert-a: Some ((), s2) ∈ set-dist (execute (assert (a0 = a1)) s1)
    and ret-a: Some (a, t) ∈ set-dist (execute (return a1) s2)
  unfolding alpha-round-def
  by (auto simp: Let-def elim!: set-dist-bindE)
have counter-s0:
  PQueryCounter s0 = PQueryCounter s
  using receive-alpha-challenge-counter-outcome[OF rand] by simp
have future-s0:
  query-future-fresh s0
  unfolding query-future-fresh-def
proof (intro allI impI)
  fix i x
  assume i-ge-s0: PQueryCounter s0 ≤ i
  have i-ge-s: PQueryCounter s ≤ i
    using i-ge-s0 counter-s0 by simp
  have neg:
    QueryIndexChallenge i x ≠

```

```

    AlphaChallenge (PAlphaCounter s) (PState s)
  by simp
  have fmllookup (HashMap s0) (QueryIndexChallenge i x) =
    fmllookup (HashMap s) (QueryIndexChallenge i x)
  by (rule receive-alpha-challenge-preserves-other-lookup[OF rand neq])
  also have ... = None
  using future i-ge-s
  unfolding query-future-fresh-def by blast
  finally show
    fmllookup (HashMap s0) (QueryIndexChallenge i x) = None .
qed
have future-s1:
  query-future-fresh s1
  by (rule read-preserves-query-future-fresh[OF future-s0 read-a])
have s2-eq: s2 = s1
  using assert-a unfolding assert-def
  by (cases a0 = a1) (auto simp: throw-no-outcome)
have t-eq: t = s2
  using ret-a by simp
show ?thesis
  using future-s1 unfolding t-eq s2-eq .
qed

lemma wp-alpha-round-fresh-set:
  assumes future: alpha-future-fresh s
  shows
    wp-event alpha-round
      ( $\lambda out. \text{case out of None} \Rightarrow \text{False} \mid \text{Some } (a, -) \Rightarrow a \in B$ ) s
       $\leq$  nnreal (card B) / nnreal size
  proof -
    let ?P =  $\lambda out. \text{case out of None} \Rightarrow \text{False} \mid \text{Some } (a, -) \Rightarrow a \in B$ 
    let ?k =
       $\lambda a0. \text{read} \gg (\lambda a1. \text{assert } (a0 = a1) \gg (\lambda -. \text{return } a1))$ 
    have head-bound:
      wp-event receive-alpha-challenge ?P s  $\leq$ 
        nnreal (card B) / nnreal size
    using wp-receive-alpha-challenge-fresh-set
      [OF alpha-future-fresh-current[OF future], of B]
    by simp
    have round-eq: alpha-round = (receive-alpha-challenge  $\gg$  ?k)
    unfolding alpha-round-def by (simp add: Let-def)
    show ?thesis
      unfolding round-eq
    proof (rule wp-event-bind-bound-by-head-event[OF head-bound])
      show ?P None  $\implies$  ?P None
      by simp
    next
      fix a0 s0 out

```

```

assume cont: out ∈ set-dist (execute (?k a0) s0)
  and hit: ?P out
show ?P (Some (a0, s0))
proof (cases out)
  case None
  then show ?thesis
    using hit by simp
next
  case (Some pair)
  obtain a t where pair-eq: pair = (a, t)
    by (cases pair)
  from cont[unfolded Some pair-eq]
  obtain a1 s1 s2 where
    read-a: Some (a1, s1) ∈ set-dist (execute read s0)
    and assert-a:
      Some ((), s2) ∈ set-dist (execute (assert (a0 = a1)) s1)
    and ret-a: Some (a, t) ∈ set-dist (execute (return a1) s2)
    by (auto elim!: set-dist-bindE)
  have a0-eq: a0 = a1
    using assert-a unfolding assert-def
    by (cases a0 = a1) (auto simp: throw-no-outcome)
  have a-eq: a = a1
    using ret-a by simp
  have a ∈ B
    using hit Some pair-eq by simp
  then show ?thesis
    using a0-eq a-eq by simp
qed
qed
qed

lemma wp-alpha-round-success-bound:
  assumes future: alpha-future-fresh s
  shows
    wp-event alpha-round
    ( $\lambda out. \text{case out of } None \Rightarrow False \mid \text{Some } - \Rightarrow True$ ) s
    ≤ 1 / nnreal size
proof (cases PTranscript s)
  case Nil
  have no-success:
     $\bigwedge out. out \in \text{set-dist } (\text{execute } \alpha\text{-round } s) \implies$ 
    ( $\text{case out of } None \Rightarrow False \mid \text{Some } - \Rightarrow True$ )  $\implies False$ 
  proof –
    fix out
    assume out: out ∈ set-dist (execute alpha-round s)
    and success: ( $\text{case out of } None \Rightarrow False \mid \text{Some } - \Rightarrow True$ )
    from success obtain a t where out-eq: out = Some (a, t)
      by (cases out) auto
    from alpha-round-outcome[OF out[unfolded out-eq]]

```

```

obtain rest where PTranscript s = a # rest
  by blast
then show False
  using Nil by simp
qed
have wp-event alpha-round
  ( $\lambda out. \text{case out of None} \Rightarrow \text{False} \mid \text{Some } - \Rightarrow \text{True}$ ) s
   $\leq$  wp-event alpha-round
  ( $\lambda - :: ('f \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option. False}$ ) s
  by (rule wp-event-mono-on-support) (rule no-success)
also have  $\dots = 0$ 
  unfolding wp-event-def wp-def dist-expect-def by simp
also have  $\dots \leq 1 / \text{nnreal size}$ 
  by simp
finally show ?thesis .
next
case (Cons x rest)
let ?Q =  $\lambda out. \text{case out of None} \Rightarrow \text{False} \mid \text{Some } - \Rightarrow \text{True}$ 
let ?P =  $\lambda out. \text{case out of None} \Rightarrow \text{False} \mid \text{Some } (a, -) \Rightarrow a \in \{x\}$ 
let ?k =
   $\lambda a0. \text{read} \gg (\lambda a1. \text{assert } (a0 = a1) \gg (\lambda -. \text{return } a1))$ 
have head-bound:
  wp-event receive-alpha-challenge ?P s  $\leq 1 / \text{nnreal size}$ 
proof –
  have wp-event receive-alpha-challenge ?P s =
  nnreal (card ({x} :: 'f set)) / nnreal size
  using wp-receive-alpha-challenge-fresh-set
  [OF alpha-future-fresh-current[OF future], of {x}]
  by simp
  then show ?thesis
  by simp
qed
have round-eq: alpha-round = (receive-alpha-challenge  $\gg$  ?k)
  unfolding alpha-round-def by (simp add: Let-def)
show ?thesis
  unfolding round-eq
proof (rule wp-event-bind-bound-by-head-event[OF head-bound])
  show ?Q None  $\implies$  ?P None
  by simp
next
fix a0 s0 out
assume rand:
  Some (a0, s0)  $\in$  set-dist (execute receive-alpha-challenge s)
  and cont: out  $\in$  set-dist (execute (?k a0) s0)
  and success: ?Q out
from success obtain a t where out-eq: out = Some (a, t)
  by (cases out) auto
from cont[unfolded out-eq]

```

```

obtain  $a1\ s1\ s2$  where
   $read\text{-}a:$   $Some\ (a1,\ s1) \in set\text{-}dist\ (execute\ read\ s0)$ 
  and  $assert\text{-}a:$ 
     $Some\ ((),\ s2) \in set\text{-}dist\ (execute\ (assert\ (a0 = a1))\ s1)$ 
  and  $ret\text{-}a:$   $Some\ (a,\ t) \in set\text{-}dist\ (execute\ (return\ a1)\ s2)$ 
  by  $(auto\ elim!:\ set\text{-}dist\ bindE)$ 
have  $tr\text{-}s0:$   $PTranscript\ s0 = x \# rest$ 
  using  $receive\text{-}\alpha\text{-}challenge\text{-}outcome[OF\ rand]$  Cons by  $simp$ 
have  $a1\text{-}eq:$   $a1 = x$ 
  using  $read\text{-}nonempty\text{-}outcome[OF\ read\text{-}a\ tr\text{-}s0]$  by  $simp$ 
have  $a0\text{-}eq:$   $a0 = a1$ 
  using  $assert\text{-}a$  unfolding  $assert\text{-}def$ 
  by  $(cases\ a0 = a1)\ (auto\ simp:\ throw\text{-}no\text{-}outcome)$ 
show  $?P\ (Some\ (a0,\ s0))$ 
  using  $a0\text{-}eq\ a1\text{-}eq$  by  $simp$ 
qed
qed

lemma  $mmap\text{-}\alpha\text{-}round\text{-}outcome:$ 
assumes  $outcome:$ 
   $Some\ (as,\ t) \in set\text{-}dist\ (execute\ (mmap\ (replicate\ n\ \alpha\text{-}round))\ s)$ 
shows
   $length\ as = n \wedge$ 
   $PTranscript\ s = as\ @\ PTranscript\ t \wedge$ 
   $PState\ t = foldl\ concat\ (PState\ s)\ as \wedge$ 
   $s \leq t \wedge$ 
   $PTraceFriCounter\ t = PTraceFriCounter\ s \wedge$ 
   $PCompositionFriCounter\ t = PCompositionFriCounter\ s \wedge$ 
   $PAlphaCounter\ t = PAlphaCounter\ s + n \wedge$ 
   $PQueryCounter\ t = PQueryCounter\ s \wedge$ 
   $(\forall i < n.$ 
     $fmlookup\ (HashMap\ t)$ 
     $(AlphaChallenge\ (PAlphaCounter\ s + i)$ 
       $(foldl\ concat\ (PState\ s)\ (take\ i\ as))) =$ 
       $Some\ (as\ !\ i))$ 
  using  $outcome$ 
proof  $(induction\ n\ arbitrary:\ s\ as\ t)$ 
case  $0$ 
then show  $?case$ 
  by  $(simp\ add:\ hash\text{-}ext\text{-}refl)$ 
next
case  $(Suc\ n)$ 
from  $Suc.premis$  obtain  $a\ as'\ s1$  where
   $head:$   $Some\ (a,\ s1) \in set\text{-}dist\ (execute\ \alpha\text{-}round\ s)$ 
  and  $tail:$ 
     $Some\ (as',\ t) \in set\text{-}dist\ (execute\ (mmap\ (replicate\ n\ \alpha\text{-}round))\ s1)$ 
  and  $as\text{-}eq:$   $as = a \# as'$ 
  by  $(auto\ elim!:\ set\text{-}dist\ bindE)$ 
from  $\alpha\text{-}round\text{-}outcome[OF\ head]$  obtain  $rest1$  where

```



```
|s: PTranscript s = a # rest1
and st-s1: PState s1 = concat (PState s) a
and tr-s1: PTranscript s1 = rest1
and ext-head: s ≤ s1
and trace-count-s1: PTraceFriCounter s1 = PTraceFriCounter s
and comp-count-s1: PCompositionFriCounter s1 = PCompositionFriCounter s
and alpha-count-s1: PAlphaCounter s1 = Suc (PAlphaCounter s)
and query-count-s1: PQueryCounter s1 = PQueryCounter s
and lookup-head: fmlookup (HashMap s1) (AlphaChallenge (PAlphaCounter s) (PState s))
= Some a
  by blast
have tail-res:
  length as' = n ∧
  PTranscript s1 = as' @ PTranscript t ∧
  PState t = foldl concat (PState s1) as' ∧
  s1 ≤ t ∧
  PTraceFriCounter t = PTraceFriCounter s1 ∧
  PCompositionFriCounter t = PCompositionFriCounter s1 ∧
  PAlphaCounter t = PAlphaCounter s1 + n ∧
  PQueryCounter t = PQueryCounter s1 ∧
  (∀ i < n.
    fmlookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s1 + i)
        (foldl concat (PState s1) (take i as')))) =
    Some (as' ! i))
  using Suc.IH[OF tail] .
have ext-s1-t: s1 ≤ t
  using tail-res by simp
have lookup-head-t:
  fmlookup (HashMap t) (AlphaChallenge (PAlphaCounter s) (PState s)) = Some a
  using hash-extension-lookup[OF lookup-head ext-s1-t] .
have ext-combined: s ≤ t
  using ext-head ext-s1-t by (meson hash-ext-trans)
have counters-t:
  PTraceFriCounter t = PTraceFriCounter s ∧
  PCompositionFriCounter t = PCompositionFriCounter s ∧
  PAlphaCounter t = PAlphaCounter s + Suc n ∧
  PQueryCounter t = PQueryCounter s
  using tail-res trace-count-s1 comp-count-s1 alpha-count-s1 query-count-s1
  by simp
have lookups:
  ∀ i < Suc n.
    fmlookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s + i)
        (foldl concat (PState s) (take i as')))) =
    Some (as ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < Suc n

|  |

```

```

show fmlookup (HashMap t)
  (AlphaChallenge (PAlphaCounter s + i)
    (foldl concat (PState s) (take i as))) =
  Some (as ! i)
proof (cases i)
  case 0
  then show ?thesis
    using lookup-head-t as-eq by simp
next
  case (Suc j)
  then have j-bound: j < n
    using i-bound by simp
  have key-eq:
    foldl concat (PState s) (take i as) =
      foldl concat (PState s1) (take j as')
    using Suc as-eq st-s1 by simp
  show ?thesis
    using tail-res j-bound Suc as-eq key-eq alpha-count-s1 by simp
qed
qed
show ?case
  using tail-res tr-s tr-s1 st-s1 ext-combined lookups as-eq
    counters-t
  by simp
qed

lemma mmap-alpha-round-preserves-alpha-future-fresh:
  assumes future: alpha-future-fresh s
  and outcome:
    Some (as, t) ∈
      set-dist (execute (mmap (replicate n alpha-round))) s)
  shows alpha-future-fresh t
  using outcome future
proof (induction n arbitrary: s as t)
  case 0
  then show ?case
    by simp
next
  case (Suc n)
  from Suc.prems obtain a as' s1 where
    head: Some (a, s1) ∈ set-dist (execute alpha-round s)
    and tail:
      Some (as', t) ∈
        set-dist (execute (mmap (replicate n alpha-round))) s1)
    by (auto elim!: set-dist-bindE)
  have future-s1: alpha-future-fresh s1
    by (rule alpha-round-preserves-alpha-future-fresh[OF Suc.prems(2) head])
  show ?case
    by (rule Suc.IH[OF tail future-s1])

```

qed

lemma *mmap-alpha-round-preserves-composition-fri-future-fresh:*

assumes *future: composition-fri-future-fresh s*

and *outcome:*

Some (as, t) ∈
set-dist (execute (mmap (replicate n alpha-round)) s)

shows *composition-fri-future-fresh t*

using *outcome future*

proof (*induction n arbitrary: s as t*)

case 0

then show ?*case*

by *simp*

next

case (*Suc n*)

from *Suc.prem*s **obtain** *a as' s1* **where**

head: Some (a, s1) ∈ set-dist (execute alpha-round s)

and *tail:*

Some (as', t) ∈
set-dist (execute (mmap (replicate n alpha-round)) s1)

by (*auto elim!: set-dist-bindE*)

have *future-s1: composition-fri-future-fresh s1*

by (*rule alpha-round-preserves-composition-fri-future-fresh*
*[OF Suc.prem*s(2) *head]*)

show ?*case*

by (*rule Suc.IH[OF tail future-s1]*)

qed

lemma *mmap-alpha-round-preserves-query-future-fresh:*

assumes *future: query-future-fresh s*

and *outcome:*

Some (as, t) ∈
set-dist (execute (mmap (replicate n alpha-round)) s)

shows *query-future-fresh t*

using *outcome future*

proof (*induction n arbitrary: s as t*)

case 0

then show ?*case*

by *simp*

next

case (*Suc n*)

from *Suc.prem*s **obtain** *a as' s1* **where**

head: Some (a, s1) ∈ set-dist (execute alpha-round s)

and *tail:*

Some (as', t) ∈
set-dist (execute (mmap (replicate n alpha-round)) s1)

by (*auto elim!: set-dist-bindE*)

have *future-s1: query-future-fresh s1*

by (*rule alpha-round-preserves-query-future-fresh*)

```

    [OF Suc.premis(2) head])
  show ?case
  by (rule Suc.IH[OF tail future-s1])
qed

lemma wp-mmap-alpha-round-success-bound:
  assumes future: alpha-future-fresh s
  shows
    wp-event (mmap (replicate n alpha-round))
      (λout. case out of None ⇒ False | Some - ⇒ True) s
      ≤ (1 / nnreal size) ^ n
  using future
proof (induction n arbitrary: s)
  case 0
  then show ?case
  by (simp add: wp-event-def wpsimps)
next
  case (Suc n)
  let ?ListSuccess =
    λout :: ('f list × ('f, 'a) protocol-channel-scheme) option.
      case out of None ⇒ False | Some - ⇒ True
  let ?HeadSuccess =
    λout :: ('f × ('f, 'a) protocol-channel-scheme) option.
      case out of None ⇒ False | Some - ⇒ True
  let ?tail =
    (mmap (replicate n alpha-round) ::
      ('f list, ('f, 'a) protocol-channel-scheme) state-monad)
  let ?D = (1 / nnreal size) ^ n
  have bind-bound:
    wp-event
      (alpha-round >>=
        (λa :: 'f. ?tail >>= (λas :: 'f list. return (a # as))))
      ?ListSuccess s
      ≤ wp-event alpha-round ?HeadSuccess s * ?D
  proof (rule wp-event-bind-bound-by-head-and-cont
    [where P = ?HeadSuccess])
    show ¬ ?ListSuccess None
    by simp
  next
    fix a t
    assume ¬ ?HeadSuccess (Some (a, t))
    then show
      wp-event (?tail >>= (λas :: 'f list. return (a # as)))
        ?ListSuccess t = 0
    by simp
  next
    fix a t
    assume head: Some (a, t) ∈ set-dist (execute alpha-round s)
    have future-t: alpha-future-fresh t

```

```

  by (rule alpha-round-preserves-alpha-future-fresh[OF Suc.prem head])
have tail-bound: wp-event ?tail ?ListSuccess t ≤ ?D
  by (rule Suc.IH[OF future-t])
have cont-eq:
  (λout. case out of
    None ⇒ (0 :: prob)
  | Some (as, u) ⇒
    wp (return (a # as))
    (λout. if ?ListSuccess out then 1 else 0) u) =
  (λout. if ?ListSuccess out then 1 else 0)
  by (rule ext) (auto simp: wpsimps split: option.splits prod.splits)
have wp-event (?tail ≫ (λas :: 'f list. return (a # as)))
  ?ListSuccess t =
  wp-event ?tail ?ListSuccess t
  unfolding wp-event-def
  by (simp add: wpsimps cont-eq)
also have ... ≤ ?D
  by (rule tail-bound)
finally show
  wp-event (?tail ≫ (λas :: 'f list. return (a # as)))
  ?ListSuccess t ≤ ?D .
qed
have wp-event (mmap (replicate (Suc n) alpha-round)) ?ListSuccess s
  ≤ wp-event alpha-round ?HeadSuccess s * ?D
  using bind-bound by simp
also have ... ≤ (1 / nnreal size) * ?D
  by (rule mult-right-mono[OF wp-alpha-round-success-bound[OF Suc.prem]])
  simp
also have ... = (1 / nnreal size) ^ Suc n
  by simp
finally show ?case .
qed

lemma wp-mmap-alpha-round-finite-set-bound:
  assumes future: alpha-future-fresh s
  and finite-B: finite B
  shows
    wp-event (mmap (replicate n alpha-round))
      (λout. case out of None ⇒ False | Some (as, -) ⇒ as ∈ B) s
      ≤ nnreal (card B) / (nnreal size) ^ n
proof (cases B = {})
case True
have wp-event (mmap (replicate n alpha-round))
  (λout. case out of None ⇒ False | Some (as, -) ⇒ as ∈ B) s
  ≤ wp-event (mmap (replicate n alpha-round))
    (λ- :: ('f list × ('f, 'a) protocol-channel-scheme) option. False) s
  unfolding True
  by (rule wp-event-mono) (auto split: option.splits prod.splits)
also have ... = 0

```

```

  unfolding wp-event-def wp-def dist-expect-def by simp
  finally show ?thesis
  unfolding True by simp
next
case False
have card-pos:  $0 < \text{card } B$ 
  using finite-B False by (simp add: card-gt-0-iff)
have event-success:
  wp-event (mmap (replicate n alpha-round))
    ( $\lambda \text{out. case out of None} \Rightarrow \text{False} \mid \text{Some } (as, -) \Rightarrow as \in B$ ) s
     $\leq$  wp-event (mmap (replicate n alpha-round))
    ( $\lambda \text{out. case out of None} \Rightarrow \text{False} \mid \text{Some } - \Rightarrow \text{True}$ ) s
  by (rule wp-event-mono) (auto split: option.splits prod.splits)
also have ...  $\leq (1 / \text{nnreal size}) ^ n$ 
  by (rule wp-mmap-alpha-round-success-bound[OF future])
also have ...  $= 1 / (\text{nnreal size}) ^ n$ 
  using size-card by transfer (simp add: power-divide)
also have ...  $= \text{nnreal } (1::\text{nat}) / \text{nnreal } (\text{size} ^ n)$ 
  by simp
also have ...  $\leq \text{nnreal } (\text{card } B) / \text{nnreal } (\text{size} ^ n)$ 
  by (rule nnreal-nat-divide-right-mono) (use card-pos in simp)
also have ...  $\leq \text{nnreal } (\text{card } B) / (\text{nnreal size}) ^ n$ 
  by simp
finally show ?thesis .
qed

```

```

lemma wp-mmap-alpha-round-alpha-space-set-bound:
  assumes future: alpha-future-fresh s
  and subset:  $B \subseteq \text{alpha-space}$ 
  shows
    wp-event (mmap (replicate (length spec) alpha-round))
      ( $\lambda \text{out. case out of None} \Rightarrow \text{False} \mid \text{Some } (as, -) \Rightarrow as \in B$ ) s
       $\leq \text{nnreal } (\text{card } B) / \text{nnreal } (\text{card alpha-space})$ 
proof -
  have finite-B: finite B
  by (rule finite-subset[OF subset finite-alpha-space])
  have wp-event (mmap (replicate (length spec) alpha-round))
    ( $\lambda \text{out. case out of None} \Rightarrow \text{False} \mid \text{Some } (as, -) \Rightarrow as \in B$ ) s
     $\leq \text{nnreal } (\text{card } B) / (\text{nnreal size}) ^ \text{length spec}$ 
  by (rule wp-mmap-alpha-round-finite-set-bound[OF future finite-B])
  also have ...  $= \text{nnreal } (\text{card } B) / \text{nnreal } (\text{card alpha-space})$ 
  using size-card by simp
  finally show ?thesis .
qed

```

```

lemma receive-fri-commits-with-outcome:
  assumes receiver-outcome:
     $\bigwedge b \ s\text{-read } t.$ 
    Some  $(b, t) \in \text{set-dist } (\text{execute receive-challenge } s\text{-read}) \Rightarrow$ 

```

```

    s-read ≤ t ∧
    PState t = PState s-read ∧
    PTranscript t = PTranscript s-read ∧
    fmlookup (HashMap t) (tag (PState s-read)) = Some b
  and outcome:
    Some ((b, r), t) ∈
      set-dist (execute (receive-fri-commits-with receive-challenge) s)
  shows
    ∃ rest.
      PTranscript s = r # rest ∧
      PState t = concat (PState s) r ∧
      PTranscript t = rest ∧
      s ≤ t ∧
      fmlookup (HashMap t) (tag (concat (PState s) r)) = Some b
  proof -
    from outcome obtain s-read where
      read-r: Some (r, s-read) ∈ set-dist (execute read s)
    and rand-b:
      Some (b, t) ∈ set-dist (execute receive-challenge s-read)
    unfolding receive-fri-commits-with-def
    by (auto elim!: set-dist-bindE)
  obtain rest where tr-s: PTranscript s = r # rest
  proof -
    have PTranscript s ≠ []
      using read-r unfolding read-def protocol-read-def assert-def
      by (cases PTranscript s) (auto simp: throw-no-outcome elim!: set-dist-bindE)
    then obtain x xs where xs: PTranscript s = x # xs
      by (cases PTranscript s) auto
    have r-x: r = x
      using read-nonempty-outcome[OF read-r xs] by simp
    show ?thesis
      using that xs r-x by auto
  qed
  have read-res:
    r = r ∧
    s-read = s(PState := concat (PState s) r, PTranscript := rest)
  using read-nonempty-outcome[OF read-r tr-s] by simp
  have rand-res:
    s-read ≤ t ∧
    PState t = PState s-read ∧
    PTranscript t = PTranscript s-read ∧
    fmlookup (HashMap t) (tag (PState s-read)) = Some b
  using receiver-outcome[OF rand-b] .
  have s-read-ext: s ≤ s-read
    using read-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  have s-t: s ≤ t
    using s-read-ext rand-res by (meson hash-ext-trans)
  show ?thesis
    using tr-s read-res rand-res s-t by auto

```

qed

lemma *receive-trace-fri-commits-outcome*:

assumes *outcome*:

Some ((b, r), t) ∈
set-dist (execute receive-trace-fri-commits s)

shows

$\exists \text{rest.}$
 $P\text{Transcript } s = r \# \text{rest} \wedge$
 $P\text{State } t = \text{concat } (P\text{State } s) \ r \wedge$
 $P\text{Transcript } t = \text{rest} \wedge$
 $s \leq t \wedge$
 $P\text{TraceFriCounter } t = \text{Suc } (P\text{TraceFriCounter } s) \wedge$
 $P\text{CompositionFriCounter } t = P\text{CompositionFriCounter } s \wedge$
 $P\text{AlphaCounter } t = P\text{AlphaCounter } s \wedge$
 $P\text{QueryCounter } t = P\text{QueryCounter } s \wedge$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{TraceFriChallenge } (P\text{TraceFriCounter } s) (\text{concat } (P\text{State } s) \ r)) =$
 $\text{Some } b$

proof –

from *outcome* **obtain** *s-read* **where**

read-r: *Some (r, s-read) ∈ set-dist (execute read s)*

and *rand-b*:

Some (b, t) ∈ set-dist (execute receive-trace-fri-challenge s-read)

unfolding *receive-trace-fri-commits-def* *receive-fri-commits-with-def*

by (*auto elim!:* *set-dist-bindE*)

obtain *rest* **where** *tr-s*: $P\text{Transcript } s = r \# \text{rest}$

proof –

have $P\text{Transcript } s \neq []$

using *read-r* **unfolding** *read-def* *protocol-read-def* *assert-def*

by (*cases* $P\text{Transcript } s$) (*auto simp:* *throw-no-outcome elim!:* *set-dist-bindE*)

then obtain *x xs* **where** *xs*: $P\text{Transcript } s = x \# \text{xs}$

by (*cases* $P\text{Transcript } s$) *auto*

have *r-x*: $r = x$

using *read-nonempty-outcome*[*OF* *read-r xs*] **by** *simp*

show *?thesis*

using *that xs r-x* **by** *auto*

qed

have *read-res*:

$s\text{-read} = s(P\text{State} := \text{concat } (P\text{State } s) \ r, P\text{Transcript} := \text{rest})$

using *read-nonempty-outcome*[*OF* *read-r tr-s*] **by** *simp*

have *rand-res*:

$s\text{-read} \leq t \wedge$

$P\text{State } t = P\text{State } s\text{-read} \wedge$

$P\text{Transcript } t = P\text{Transcript } s\text{-read} \wedge$

$\text{fmlookup } (\text{HashMap } t)$

$(\text{TraceFriChallenge } (P\text{TraceFriCounter } s\text{-read}) (P\text{State } s\text{-read})) =$

$\text{Some } b$

using *receive-trace-fri-challenge-outcome*[*OF* *rand-b*] .


```

have rand-counter:
  PTraceFriCounter t = Suc (PTraceFriCounter s-read) ∧
  PCompositionFriCounter t = PCompositionFriCounter s-read ∧
  PAlphaCounter t = PAlphaCounter s-read ∧
  PQueryCounter t = PQueryCounter s-read
  using receive-trace-fri-challenge-counter-outcome[OF rand-b] by simp
have s-read-ext: s ≤ s-read
  using read-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have s-t: s ≤ t
  using s-read-ext rand-res by (meson hash-ext-trans)
show ?thesis
  using tr-s read-res rand-res rand-counter s-t by auto
qed

lemma receive-trace-fri-commits-preserves-trace-fri-future-fresh:
  assumes future: trace-fri-future-fresh s
  and outcome:
    Some ((b, r), t) ∈
      set-dist (execute receive-trace-fri-commits s)
  shows trace-fri-future-fresh t
proof -
  from outcome obtain s-read where
    read-r: Some (r, s-read) ∈ set-dist (execute read s)
  and rand-b:
    Some (b, t) ∈ set-dist (execute receive-trace-fri-challenge s-read)
  unfolding receive-trace-fri-commits-def receive-fri-commits-with-def
  by (auto elim!: set-dist-bindE)
from read-outcome[OF read-r] have read-counts:
  PTraceFriCounter s-read = PTraceFriCounter s
  by blast
have read-hash: HashMap s-read = HashMap s
  by (rule read-preserves-hash-map[OF read-r])
have counter-t:
  PTraceFriCounter t = Suc (PTraceFriCounter s)
  using receive-trace-fri-challenge-counter-outcome[OF rand-b] read-counts
  by simp
show ?thesis
  unfolding trace-fri-future-fresh-def
proof (intro allI impI)
  fix i x
  assume i-ge-t: PTraceFriCounter t ≤ i
  have i-ge-s: PTraceFriCounter s ≤ i
  using i-ge-t counter-t by simp
  have neg:
    TraceFriChallenge i x ≠
      TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)
  using i-ge-t counter-t read-counts by auto
  have fmlookup (HashMap t) (TraceFriChallenge i x) =
    fmlookup (HashMap s-read) (TraceFriChallenge i x)

```

```

    by (rule receive-trace-fri-challenge-preserves-other-lookup
        [OF rand-b neq])
  also have ... = fmlookup (HashMap s) (TraceFriChallenge i x)
    using read-hash by simp
  also have ... = None
    using future i-ge-s unfolding trace-fri-future-fresh-def by blast
  finally show fmlookup (HashMap t) (TraceFriChallenge i x) = None .
qed
qed

```

```

lemma receive-trace-fri-commits-preserves-alpha-future-fresh:
  assumes future: alpha-future-fresh s
  and outcome:
    Some ((b, r), t) ∈
      set-dist (execute receive-trace-fri-commits s)
  shows alpha-future-fresh t
proof -
  from outcome obtain s-read where
    read-r: Some (r, s-read) ∈ set-dist (execute read s)
  and rand-b:
    Some (b, t) ∈ set-dist (execute receive-trace-fri-challenge s-read)
  unfolding receive-trace-fri-commits-def receive-fri-commits-with-def
  by (auto elim!: set-dist-bindE)
  from read-outcome[OF read-r] have read-counts:
    PAlphaCounter s-read = PAlphaCounter s
  by blast
  have read-hash: HashMap s-read = HashMap s
  by (rule read-preserves-hash-map[OF read-r])
  have counter-t:
    PAlphaCounter t = PAlphaCounter s
  using receive-trace-fri-challenge-counter-outcome[OF rand-b]
  read-counts
  by simp
  show ?thesis
  unfolding alpha-future-fresh-def
proof (intro allI impI)
  fix i x
  assume i-ge-t: PAlphaCounter t ≤ i
  have i-ge-s: PAlphaCounter s ≤ i
    using i-ge-t counter-t by simp
  have neg:
    AlphaChallenge i x ≠
      TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)
  by simp
  have fmlookup (HashMap t) (AlphaChallenge i x) =
    fmlookup (HashMap s-read) (AlphaChallenge i x)
  by (rule receive-trace-fri-challenge-preserves-other-lookup
      [OF rand-b neq])
  also have ... = fmlookup (HashMap s) (AlphaChallenge i x)

```

```

    using read-hash by simp
  also have ... = None
    using future i-ge-s unfolding alpha-future-fresh-def by blast
  finally show fmlookup (HashMap t) (AlphaChallenge i x) = None .
qed
qed

lemma receive-trace-fri-commits-preserves-composition-fri-future-fresh:
  assumes future: composition-fri-future-fresh s
  and outcome:
    Some ((b, r), t) ∈
      set-dist (execute receive-trace-fri-commits s)
  shows composition-fri-future-fresh t
proof -
  from outcome obtain s-read where
    read-r: Some (r, s-read) ∈ set-dist (execute read s)
  and rand-b:
    Some (b, t) ∈ set-dist (execute receive-trace-fri-challenge s-read)
  unfolding receive-trace-fri-commits-def receive-fri-commits-with-def
  by (auto elim!: set-dist-bindE)
  from read-outcome[OF read-r] have read-counts:
    PCompositionFriCounter s-read = PCompositionFriCounter s
  by blast
  have read-hash: HashMap s-read = HashMap s
  by (rule read-preserves-hash-map[OF read-r])
  have counter-t:
    PCompositionFriCounter t = PCompositionFriCounter s
  using receive-trace-fri-challenge-counter-outcome[OF rand-b]
  read-counts
  by simp
  show ?thesis
  unfolding composition-fri-future-fresh-def
proof (intro allI impI)
  fix i x
  assume i-ge-t: PCompositionFriCounter t ≤ i
  have i-ge-s: PCompositionFriCounter s ≤ i
    using i-ge-t counter-t by simp
  have neg:
    CompositionFriChallenge i x ≠
      TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)
  by simp
  have fmlookup (HashMap t) (CompositionFriChallenge i x) =
    fmlookup (HashMap s-read) (CompositionFriChallenge i x)
  by (rule receive-trace-fri-challenge-preserves-other-lookup
    [OF rand-b neg])
  also have ... = fmlookup (HashMap s) (CompositionFriChallenge i x)
  using read-hash by simp
  also have ... = None
  using future i-ge-s

```

```

    unfolding composition-fri-future-fresh-def by blast
  finally show
    fmlookup (HashMap t) (CompositionFriChallenge i x) = None .
qed
qed

lemma receive-trace-fri-commits-preserves-query-future-fresh:
  assumes future: query-future-fresh s
  and outcome:
    Some ((b, r), t) ∈
      set-dist (execute receive-trace-fri-commits s)
  shows query-future-fresh t
proof -
  from outcome obtain s-read where
    read-r: Some (r, s-read) ∈ set-dist (execute read s)
  and rand-b:
    Some (b, t) ∈ set-dist (execute receive-trace-fri-challenge s-read)
  unfolding receive-trace-fri-commits-def receive-fri-commits-with-def
  by (auto elim!: set-dist-bindE)
  from read-outcome[OF read-r] have read-counts:
    PQueryCounter s-read = PQueryCounter s
  by blast
  have read-hash: HashMap s-read = HashMap s
  by (rule read-preserves-hash-map[OF read-r])
  have counter-t:
    PQueryCounter t = PQueryCounter s
  using receive-trace-fri-challenge-counter-outcome[OF rand-b]
  read-counts
  by simp
  show ?thesis
  unfolding query-future-fresh-def
proof (intro allI impI)
  fix i x
  assume i-ge-t: PQueryCounter t ≤ i
  have i-ge-s: PQueryCounter s ≤ i
  using i-ge-t counter-t by simp
  have neg:
    QueryIndexChallenge i x ≠
      TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)
  by simp
  have fmlookup (HashMap t) (QueryIndexChallenge i x) =
    fmlookup (HashMap s-read) (QueryIndexChallenge i x)
  by (rule receive-trace-fri-challenge-preserves-other-lookup
    [OF rand-b neg])
  also have ... = fmlookup (HashMap s) (QueryIndexChallenge i x)
  using read-hash by simp
  also have ... = None
  using future i-ge-s unfolding query-future-fresh-def by blast
  finally show fmlookup (HashMap t) (QueryIndexChallenge i x) = None .

```

```

qed
qed

lemma wp-receive-trace-fri-commits-fresh-set:
  assumes future: trace-fri-future-fresh s
  shows
    wp-event receive-trace-fri-commits
      (λout. case out of
        None ⇒ False
        | Some ((b, r), -) ⇒ b ∈ B) s
      ≤ nnreal (card B) / nnreal size
  unfolding receive-trace-fri-commits-def receive-fri-commits-with-def
proof (rule wp-event-bind-bound-by-cont)
  show ¬ (case None of
    None ⇒ False | Some ((b, r), -) ⇒ b ∈ B)
  by simp
next
  fix r s-read
  assume read-r: Some (r, s-read) ∈ set-dist (execute read s)
  have read-counts:
    PTraceFriCounter s-read = PTraceFriCounter s
  using read-outcome[OF read-r] by blast
  have read-hash: HashMap s-read = HashMap s
  by (rule read-preserves-hash-map[OF read-r])
  have fresh:
    fmlookup (HashMap s-read)
      (TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)) = None
  using future read-counts read-hash
  unfolding trace-fri-future-fresh-def by simp
  let ?Q =
    λout :: (('f × 'f) × ('f, 'a) protocol-channel-scheme) option.
      case out of None ⇒ False | Some ((b, r), -) ⇒ b ∈ B
  have cont-eq:
    (λout. case out of
      None ⇒ (0 :: prob)
      | Some (b, u) ⇒
        wp (return (b, r)) (λout. if ?Q out then 1 else 0) u) =
    (λout :: ('f × ('f, 'a) protocol-channel-scheme) option.
      if (case out of None ⇒ False | Some (b, -) ⇒ b ∈ B)
      then 1 else 0)
  by (rule ext) (auto simp: wpsimps split: option.splits prod.splits)
  have wp-event
    (receive-trace-fri-challenge ≫ (λb :: 'f. return (b, r)))
    ?Q s-read =
    wp-event receive-trace-fri-challenge
      (λout. case out of None ⇒ False | Some (b, -) ⇒ b ∈ B)
      s-read
  unfolding wp-event-def by (simp add: wpsimps cont-eq)
  also have ... = nnreal (card B) / nnreal size

```

```

    by (rule wp-receive-trace-fri-challenge-fresh-set[OF fresh])
  finally show
    wp-event
      (receive-trace-fri-challenge  $\gg$  ( $\lambda b :: 'f. \text{return } (b, r)$ ))
      ( $\lambda \text{out. case out of None} \Rightarrow \text{False} \mid \text{Some } ((b, r), -) \Rightarrow b \in B$ )
      s-read
       $\leq \text{nnreal } (\text{card } B) / \text{nnreal size}$ 
    by simp
qed

lemma ntimes-receive-trace-fri-commits-preserves-trace-fri-future-fresh:
  assumes future: trace-fri-future-fresh s
  and outcome:
    Some (brs, t)  $\in$ 
      set-dist (execute (ntimes receive-trace-fri-commits n) s)
  shows trace-fri-future-fresh t
  using outcome future
proof (induction n arbitrary: s brs t)
  case 0
  then show ?case
    by simp
next
  case (Suc n)
  from Suc.prem1 obtain br brs' s1 where
    head:
      Some (br, s1)  $\in$  set-dist (execute receive-trace-fri-commits s)
  and tail:
    Some (brs', t)  $\in$ 
      set-dist (execute (ntimes receive-trace-fri-commits n) s1)
  by (auto elim!: set-dist-bindE)
  obtain b r where br-eq: br = (b, r)
  by (cases br)
  have future-s1:
    trace-fri-future-fresh s1
  by (rule receive-trace-fri-commits-preserves-trace-fri-future-fresh
      [OF Suc.prem2(2) head[unfolded br-eq]])
  show ?case
    by (rule Suc.IH[OF tail future-s1])
qed

lemma ntimes-receive-trace-fri-commits-preserves-alpha-future-fresh:
  assumes outcome:
    Some (brs, t)  $\in$ 
      set-dist (execute (ntimes receive-trace-fri-commits n) s)
  and future: alpha-future-fresh s
  shows alpha-future-fresh t
  using outcome future
proof (induction n arbitrary: s brs t)
  case 0

```

```

then show ?case
  by simp
next
case (Suc n)
from Suc.premis(1) obtain br brs' s1 where
  head:
    Some (br, s1) ∈ set-dist (execute receive-trace-fri-commits s)
  and tail:
    Some (brs', t) ∈
      set-dist (execute (ntimes receive-trace-fri-commits n) s1)
  by (auto elim!: set-dist-bindE)
obtain b r where br-eq: br = (b, r)
  by (cases br)
have future-s1:
  alpha-future-fresh s1
  by (rule receive-trace-fri-commits-preserves-alpha-future-fresh
    [OF Suc.premis(2) head[unfolded br-eq]])
show ?case
  by (rule Suc.IH[OF tail future-s1])
qed

lemma ntimes-receive-trace-fri-commits-preserves-composition-fri-future-fresh:
assumes outcome:
  Some (brs, t) ∈
    set-dist (execute (ntimes receive-trace-fri-commits n) s)
and future: composition-fri-future-fresh s
shows composition-fri-future-fresh t
using outcome future
proof (induction n arbitrary: s brs t)
case 0
then show ?case
  by simp
next
case (Suc n)
from Suc.premis(1) obtain br brs' s1 where
  head:
    Some (br, s1) ∈ set-dist (execute receive-trace-fri-commits s)
  and tail:
    Some (brs', t) ∈
      set-dist (execute (ntimes receive-trace-fri-commits n) s1)
  by (auto elim!: set-dist-bindE)
obtain b r where br-eq: br = (b, r)
  by (cases br)
have future-s1:
  composition-fri-future-fresh s1
  by (rule receive-trace-fri-commits-preserves-composition-fri-future-fresh
    [OF Suc.premis(2) head[unfolded br-eq]])
show ?case
  by (rule Suc.IH[OF tail future-s1])

```

qed

lemma *ntimes-receive-trace-fri-commits-preserves-query-future-fresh:*

assumes *outcome:*

Some (brs, t) ∈

set-dist (execute (ntimes receive-trace-fri-commits n) s)

and *future: query-future-fresh s*

shows *query-future-fresh t*

using *outcome future*

proof (*induction n arbitrary: s brs t*)

case 0

then show ?case

by *simp*

next

case (*Suc n*)

from *Suc.prem1* **obtain** *br brs' s1* **where**

head:

Some (br, s1) ∈ set-dist (execute receive-trace-fri-commits s)

and *tail:*

Some (brs', t) ∈

set-dist (execute (ntimes receive-trace-fri-commits n) s1)

by (*auto elim!: set-dist-bindE*)

obtain *b r* **where** *br-eq: br = (b, r)*

by (*cases br*)

have *future-s1:*

query-future-fresh s1

by (*rule receive-trace-fri-commits-preserves-query-future-fresh*
[*OF Suc.prem2*] *head[unfolded br-eq]*)

show ?case

by (*rule Suc.IH[OF tail future-s1]*)

qed

lemma *verifier-alpha-prefix-preserves-query-future-fresh:*

assumes *future: query-future-fresh s*

and *outcome:*

Some (header, t) ∈ set-dist (execute verifier-alpha-prefix s)

shows *query-future-fresh t*

proof –

from *outcome* **obtain** *fr s1 f-fl s2 f-final s3 as* **where**

read-fr: Some (fr, s1) ∈ set-dist (execute read s)

and *trace-fri:*

Some (f-fl, s2) ∈

set-dist (execute (ntimes receive-trace-fri-commits (ceil-log clength)) s1)

and *read-final: Some (f-final, s3) ∈ set-dist (execute read s2)*

and *alpha:*

Some (as, t) ∈

set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)

unfolding *verifier-alpha-prefix-def*

by (*auto elim!: set-dist-bindE*)


```

have future-s1: query-future-fresh s1
  by (rule read-preserves-query-future-fresh[OF future read-fr])
have future-s2: query-future-fresh s2
  by (rule ntimes-receive-trace-fri-commits-preserves-query-future-fresh
      [OF trace-fri future-s1])
have future-s3: query-future-fresh s3
  by (rule read-preserves-query-future-fresh[OF future-s2 read-final])
show ?thesis
  by (rule mmap-alpha-round-preserves-query-future-fresh
      [OF future-s3 alpha])
qed

```

lemma *wp-verifier-alpha-prefix-alpha-space-set-bound:*

assumes *future: alpha-future-fresh s*

and *subset: B ⊆ alpha-space*

shows

wp-event verifier-alpha-prefix
 $(\lambda out. \text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } ((-, -, -, as'), -) \Rightarrow as' \in B) \ s$
 $\leq \text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{alpha-space})$

proof –

let *?Q =*

$\lambda out. \text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } ((-, -, -, as'), -) \Rightarrow as' \in B$

let *?C = nnreal (card B) / nnreal (card alpha-space)*

show *?thesis*

unfolding *verifier-alpha-prefix-def*

proof (rule *wp-event-bind-bound-by-cont*)

show $\neg ?Q \text{ None}$

by *simp*

next

fix *fr s1*

assume *read-fr: Some (fr, s1) ∈ set-dist (execute read s)*

have *future-s1: alpha-future-fresh s1*

by (rule *read-preserves-alpha-future-fresh*[OF *future read-fr*])

show *wp-event*

$(\text{ntimes receive-trace-fri-commits } (\text{ceil-log clength}) \gg=$
 $(\lambda f.fl. \text{read } \gg=$
 $(\lambda f.final.$
 $\quad \text{mmap } (\text{replicate } (\text{length spec}) \text{ alpha-round}) \gg=$
 $\quad (\lambda as. \text{return } (fr, f.fl, f.final, as))))$
 $?Q \ s1 \leq ?C$

proof (rule *wp-event-bind-bound-by-cont*)

show $\neg ?Q \text{ None}$

by *simp*

next

fix *f-fl s2*

```

assume trace-fri:
  Some (f-fl, s2) ∈
    set-dist
    (execute (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
have future-s2: alpha-future-fresh s2
  by (rule ntimes-receive-trace-fri-commits-preserves-alpha-future-fresh
    [OF trace-fri future-s1])
show wp-event
  (read  $\gg$ 
    (λf-final.
      mmap (replicate (length spec) alpha-round)  $\gg$ 
        (λas. return (fr, f-fl, f-final, as))))
  ?Q s2 ≤ ?C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?Q None
  by simp
next
  fix f-final s3
  assume read-final:
    Some (f-final, s3) ∈ set-dist (execute read s2)
  have future-s3: alpha-future-fresh s3
  by (rule read-preserves-alpha-future-fresh[OF future-s2 read-final])
  let ?A =
    λout. case out of
      None ⇒ False
    | Some (as, -) ⇒ as ∈ B
  have alpha-bound:
    wp-event (mmap (replicate (length spec) alpha-round)) ?A s3 ≤ ?C
  by (rule wp-mmap-alpha-round-alpha-space-set-bound
    [OF future-s3 subset])
  show wp-event
    (mmap (replicate (length spec) alpha-round)  $\gg$ 
      (λas. return (fr, f-fl, f-final, as)))
    ?Q s3 ≤ ?C
  proof (rule wp-event-bind-bound-by-head-event[OF alpha-bound])
  show ?Q None  $\implies$  ?A None
  by simp
next
  fix as t out
  assume cont:
    out ∈
      set-dist
      (execute (return (fr, f-fl, f-final, as)) t)
  and hit: ?Q out
  have out-eq: out = Some ((fr, f-fl, f-final, as), t)
  using cont
  unfolding set-dist-def return.rep-eq dist-return-def
    dist-delta-dist delta-map-def
  by simp

```

```

      show ?A (Some (as, t))
      using hit unfolding out-eq by simp
    qed
  qed
  qed
  qed
  qed

lemma wp-verifier-alpha-prefix-dependent-alpha-space-set-bound:
  fixes C :: prob
  assumes future: alpha-future-fresh s
  and subset:
     $\bigwedge fr\ f-fl\ f-final. B\ fr\ f-fl\ f-final \subseteq \text{alpha-space}$ 
  and bound:
     $\bigwedge fr\ f-fl\ f-final. \text{nnreal} (\text{card} (B\ fr\ f-fl\ f-final)) / \text{nnreal} (\text{card} \text{alpha-space}) \leq C$ 
  shows
    wp-event verifier-alpha-prefix
      ( $\lambda out. \text{case out of}$ 
        None  $\Rightarrow$  False
        | Some ((fr, f-fl, f-final, as'), -)  $\Rightarrow$ 
           $as' \in B\ fr\ f-fl\ f-final$ ) s
       $\leq C$ 
  proof -
    let ?Q =
       $\lambda out. \text{case out of}$ 
        None  $\Rightarrow$  False
        | Some ((fr, f-fl, f-final, as'), -)  $\Rightarrow$ 
           $as' \in B\ fr\ f-fl\ f-final$ 
    show ?thesis
      unfolding verifier-alpha-prefix-def
    proof (rule wp-event-bind-bound-by-cont)
      show  $\neg ?Q\ None$ 
        by simp
    next
      fix fr s1
      assume read-fr: Some (fr, s1)  $\in$  set-dist (execute read s)
      have future-s1: alpha-future-fresh s1
        by (rule read-preserves-alpha-future-fresh[OF future read-fr])
      show wp-event
        ( $\text{ntimes receive-trace-fri-commits} (\text{ceil-log clength}) \gg=$ 
          ( $\lambda f-fl. \text{read} \gg=$ 
            ( $\lambda f-final. \text{mmap} (\text{replicate} (\text{length spec}) \text{alpha-round}) \gg=$ 
              ( $\lambda as. \text{return} (fr, f-fl, f-final, as))))$ 
           $?Q\ s1 \leq C$ 
        )
    proof (rule wp-event-bind-bound-by-cont)
      show  $\neg ?Q\ None$ 
        by simp
    end
  end

```

```

next
  fix f-fl s2
  assume trace-fri:
    Some (f-fl, s2) ∈
      set-dist
      (execute (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
  have future-s2: alpha-future-fresh s2
  by (rule ntimes-receive-trace-fri-commits-preserves-alpha-future-fresh
      [OF trace-fri future-s1])
  show wp-event
    (read ≫=
      (λf-final.
        mmap (replicate (length spec) alpha-round) ≫=
          (λas. return (fr, f-fl, f-final, as))))
    ?Q s2 ≤ C
  proof (rule wp-event-bind-bound-by-cont)
    show ¬ ?Q None
    by simp
  next
    fix f-final s3
    assume read-final:
      Some (f-final, s3) ∈ set-dist (execute read s2)
    have future-s3: alpha-future-fresh s3
    by (rule read-preserves-alpha-future-fresh[OF future-s2 read-final])
    let ?B = B fr f-fl f-final
    let ?A =
      λout. case out of
        None ⇒ False
        | Some (as, -) ⇒ as ∈ ?B
    have alpha-bound:
      wp-event (mmap (replicate (length spec) alpha-round)) ?A s3 ≤
        nnreal (card ?B) / nnreal (card alpha-space)
    by (rule wp-mmap-alpha-round-alpha-space-set-bound
        [OF future-s3 subset[of fr f-fl f-final]])
    have alpha-bound-C:
      wp-event (mmap (replicate (length spec) alpha-round)) ?A s3 ≤ C
    by (rule order-trans[OF alpha-bound bound[of fr f-fl f-final]])
    show wp-event
      (mmap (replicate (length spec) alpha-round) ≫=
        (λas. return (fr, f-fl, f-final, as)))
      ?Q s3 ≤ C
    proof (rule wp-event-bind-bound-by-head-event[OF alpha-bound-C])
      show ?Q None ⇒ ?A None
      by simp
    next
      fix as t out
      assume cont:
        out ∈
          set-dist

```

```

      (execute (return (fr, f-fl, f-final, as)) t)
    and hit: ?Q out
  have out-eq: out = Some ((fr, f-fl, f-final, as), t)
    using cont
    unfolding set-dist-def return.rep-eq dist-return-def
      dist-delta-dist delta-map-def
    by simp
  show ?A (Some (as, t))
    using hit unfolding out-eq by simp
qed
qed
qed
qed
qed

lemma wp-ntimes-receive-trace-fri-commits-exact-challenges-bound:
  assumes future: trace-fri-future-fresh s
  and len: length bs = n
  shows
    wp-event (ntimes receive-trace-fri-commits n)
      (λout. case out of
        None ⇒ False
        | Some (brs, -) ⇒ map fst brs = bs) s
      ≤ (1 / nnreal size) ^ n
  using len future
proof (induction n arbitrary: s bs)
  case 0
  then have bs-empty: bs = []
    by simp
  show ?case
    unfolding bs-empty by (simp add: wp-event-def wpsimps)
next
  case (Suc n)
  obtain b bs' where bs-eq: bs = b # bs'
    using Suc.prem1(1) by (cases bs) auto
  have len-bs': length bs' = n
    using Suc.prem1(1) bs-eq by simp
  let ?tail =
    (ntimes receive-trace-fri-commits n ::
      (('f × 'f) list, ('f, 'a) protocol-channel-scheme) state-monad)
  let ?Q =
    λout :: (('f × 'f) list) ×
      ('f, 'a) protocol-channel-scheme) option.
      case out of None ⇒ False | Some (brs, -) ⇒ map fst brs = bs
  let ?Head =
    λout :: (('f × 'f) × ('f, 'a) protocol-channel-scheme) option.
      case out of None ⇒ False | Some ((b0, r0), -) ⇒ b0 ∈ {b}
  have head-bound:
    wp-event receive-trace-fri-commits ?Head s ≤ 1 / nnreal size

```

```

proof –
  have wp-event receive-trace-fri-commits ?Head s ≤
    nnreal (card ({b} :: 'f set)) / nnreal size
    by (rule wp-receive-trace-fri-commits-fresh-set[OF Suc.premss(2)])
  then show ?thesis by simp
qed
have bind-bound:
  wp-event
    (receive-trace-fri-commits  $\gg$ 
      ( $\lambda br :: 'f \times 'f.$ 
         $?tail \gg (\lambda brs :: ('f \times 'f) \text{ list}. \text{return } (br \# brs))$ )))
   $?Q \ s$ 
   $\leq wp\text{-event receive-trace-fri-commits } ?Head \ s \ *$ 
   $(1 / nnreal \ size) \wedge^n$ 
proof (rule wp-event-bind-bound-by-head-and-cont[where P = ?Head])
  show  $\neg ?Q \ None$ 
  by simp
next
  fix br t
  assume not-head:  $\neg ?Head \ (Some \ (br, \ t))$ 
  obtain b0 r0 where br-eq:  $br = (b0, \ r0)$ 
  by (cases br)
  have b0-ne:  $b0 \neq b$ 
  using not-head unfolding br-eq by simp
  have cont-false:
    wp-event ( $?tail \gg (\lambda brs :: ('f \times 'f) \text{ list}. \text{return } (br \# brs))$ )
     $?Q \ t$ 
     $\leq wp\text{-event } (?tail \gg (\lambda brs :: ('f \times 'f) \text{ list}. \text{return } (br \# brs)))$ 
     $(\lambda - :: ((('f \times 'f) \text{ list}) \times$ 
       $(('f, 'a) \text{ protocol-channel-scheme}) \text{ option}. \text{False}) \ t$ 
    by (rule wp-event-mono-on-support)
     $(\text{use } b0\text{-ne } bs\text{-eq } br\text{-eq} \text{ in}$ 
       $\langle \text{auto simp: wpsimps elim!: set-dist-bindE split: option.splits prod.splits} \rangle)$ 
  also have  $\dots = 0$ 
  unfolding wp-event-def wp-def dist-expect-def by simp
  finally have le-zero:
    wp-event ( $?tail \gg (\lambda brs :: ('f \times 'f) \text{ list}. \text{return } (br \# brs))$ )
     $?Q \ t \leq 0$  .
  show
    wp-event ( $?tail \gg (\lambda brs :: ('f \times 'f) \text{ list}. \text{return } (br \# brs))$ )
     $?Q \ t = 0$ 
    by (rule antisym[OF le-zero]) simp
next
  fix br t
  assume head:
     $Some \ (br, \ t) \in \text{set-dist } (\text{execute receive-trace-fri-commits } s)$ 
    and head-hit:  $?Head \ (Some \ (br, \ t))$ 
  obtain b0 r0 where br-eq:  $br = (b0, \ r0)$ 
  by (cases br)

```

```

have b0-eq: b0 = b
  using head-hit unfolding br-eq by simp
have future-t: trace-fri-future-fresh t
  by (rule receive-trace-fri-commits-preserves-trace-fri-future-fresh
    [OF Suc.premis(2) head[unfolded br-eq]])
have tail-bound:
  wp-event ?tail
    ( $\lambda out. \text{case out of } None \Rightarrow False$ 
      |  $\text{Some } (brs, -) \Rightarrow \text{map fst } brs = bs'$ ) t
     $\leq (1 / \text{nnreal size}) ^ n$ 
  by (rule Suc.IH[OF len-bs' future-t])
have cont-eq:
  ( $\lambda out. \text{case out of}$ 
    None  $\Rightarrow (0 :: \text{prob})$ 
    | Some (brs, u)  $\Rightarrow$ 
      wp (return (br # brs)) ( $\lambda out. \text{if } ?Q \text{ out then } 1 \text{ else } 0$ ) u) =
  ( $\lambda out :: ((f \times 'f) \text{ list}) \times$ 
    (f, 'a) protocol-channel-scheme) option.
    if (case out of None  $\Rightarrow False$ 
      | Some (brs, -)  $\Rightarrow \text{map fst } brs = bs'$ )
      then 1 else 0)
  by (rule ext)
  (auto simp: wpsimps b0-eq bs-eq br-eq
    split: option.splits prod.splits)
have wp-event (?tail  $\gg= (\lambda brs :: (f \times 'f) \text{ list. return } (br \# brs))$ )
  ?Q t =
  wp-event ?tail
    ( $\lambda out. \text{case out of } None \Rightarrow False$ 
      | Some (brs, -)  $\Rightarrow \text{map fst } brs = bs'$ ) t
  unfolding wp-event-def by (simp add: wpsimps cont-eq)
also have ...  $\leq (1 / \text{nnreal size}) ^ n$ 
  by (rule tail-bound)
finally show
  wp-event (?tail  $\gg= (\lambda brs :: (f \times 'f) \text{ list. return } (br \# brs))$ )
  ?Q t  $\leq (1 / \text{nnreal size}) ^ n$  .
qed
have wp-event (ntimes receive-trace-fri-commits (Suc n)) ?Q s
   $\leq \text{wp-event receive-trace-fri-commits } ?Head \text{ s } *$ 
  ( $1 / \text{nnreal size}) ^ n$ 
  using bind-bound by simp
also have ...  $\leq (1 / \text{nnreal size}) * (1 / \text{nnreal size}) ^ n$ 
  by (rule mult-right-mono[OF head-bound]) simp
also have ...  $= (1 / \text{nnreal size}) ^ {Suc n}$ 
  by simp
finally show ?case .
qed

lemma wp-ntimes-receive-trace-fri-commits-finite-set-bound:
  assumes future: trace-fri-future-fresh s

```

and *finite-B*: *finite B*
and *lengths*: $\bigwedge bs. bs \in B \implies \text{length } bs = n$
shows
wp-event (ntimes receive-trace-fri-commits n)
 $(\lambda out. \text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } (brs, -) \Rightarrow \text{map fst } brs \in B) \ s$
 $\leq \text{nnreal } (\text{card } B) / (\text{nnreal size}) ^ n$
proof –
let *?P* =
 $\lambda bs \text{ out. case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } (brs, -) \Rightarrow \text{map fst } brs = bs$
have *event-mono*:
wp-event (ntimes receive-trace-fri-commits n)
 $(\lambda out. \text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } (brs, -) \Rightarrow \text{map fst } brs \in B) \ s$
 $\leq \text{wp-event } (ntimes \text{ receive-trace-fri-commits } n)$
 $(\lambda out. \exists bs \in B. ?P \ bs \ out) \ s$
by (*rule wp-event-mono*) (*auto split: option.splits prod.splits*)
also have $\dots \leq (\sum bs \in B. (1 / \text{nnreal size}) ^ n)$
by (*rule wp-event-finite-UN-bound[OF finite-B]*)
 $(\text{rule wp-ntimes-receive-trace-fri-commits-exact-challenges-bound}$
 $\quad [\text{OF future lengths}])$
also have $\dots = \text{nnreal } (\text{card } B) * ((1 / \text{nnreal size}) ^ n)$
using *finite-B* **by** *simp*
also have $\dots = \text{nnreal } (\text{card } B) / (\text{nnreal size}) ^ n$
using *size-card* **by** *transfer (simp add: power-divide)*
finally show *?thesis* .
qed

lemma *wp-ntimes-receive-trace-fri-commits-challenge-space-set-bound*:

assumes *future: trace-fri-future-fresh s*

and *subset*: $B \subseteq \text{fri-challenge-space } n$

shows

wp-event (ntimes receive-trace-fri-commits n)
 $(\lambda out. \text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } (brs, -) \Rightarrow \text{map fst } brs \in B) \ s$
 $\leq \text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}(f) ^ n)$

proof –

have *finite-B*: *finite B*

by (*rule finite-subset[OF subset finite-fri-challenge-space]*)

have *lengths*: $\bigwedge bs. bs \in B \implies \text{length } bs = n$

using *subset* **unfolding** *fri-challenge-space-def* **by** *auto*

have *wp-event (ntimes receive-trace-fri-commits n)*

$(\lambda out. \text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$


```

    | Some (brs, -)  $\Rightarrow$  map fst brs  $\in B$  s
   $\leq$  nnreal (card B) / (nnreal size)  $^n$ 
  by (rule wp-ntimes-receive-trace-fri-commits-finite-set-bound
    [OF future finite-B lengths])
  also have ... = nnreal (card B) / nnreal (CARD('f)  $^n$ )
    using size-card by simp
  finally show ?thesis .
qed

lemma wp-verifier-trace-fri-prefix-challenge-space-set-bound:
  assumes future: trace-fri-future-fresh s
  and subset:  $B \subseteq \text{fri-challenge-space } (\text{ceil-log length})$ 
  shows
    wp-event verifier-trace-fri-prefix
      ( $\lambda \text{out. case out of}$ 
        None  $\Rightarrow$  False
        | Some ((-, f-fl), -)  $\Rightarrow$  map fst f-fl  $\in B$  s
         $\leq$  nnreal (card B) / nnreal (CARD('f)  $^{\text{ceil-log length}}$ )
      )
  proof -
    let ?Q =
       $\lambda \text{out. case out of}$ 
        None  $\Rightarrow$  False
        | Some ((-, f-fl), -)  $\Rightarrow$  map fst f-fl  $\in B$ 
    let ?C = nnreal (card B) / nnreal (CARD('f)  $^{\text{ceil-log length}}$ )
    show ?thesis
      unfolding verifier-trace-fri-prefix-def
    proof (rule wp-event-bind-bound-by-cont)
      show  $\neg ?Q \text{ None}$ 
        by simp
    next
      fix fr s1
      assume read-fr: Some (fr, s1)  $\in \text{set-dist } (\text{execute read } s)$ 
      have future-s1: trace-fri-future-fresh s1
        by (rule read-preserves-trace-fri-future-fresh[OF future read-fr])
      let ?Head =
         $\lambda \text{out. case out of}$ 
          None  $\Rightarrow$  False
          | Some (f-fl, -)  $\Rightarrow$  map fst f-fl  $\in B$ 
      have trace-bound:
        wp-event (ntimes receive-trace-fri-commits (ceil-log length))
          ?Head s1  $\leq$  ?C
      by (rule wp-ntimes-receive-trace-fri-commits-challenge-space-set-bound
        [OF future-s1 subset])
      show wp-event
        (ntimes receive-trace-fri-commits (ceil-log length)  $\gg$ 
          ( $\lambda f\text{-fl. return } (\text{fr}, f\text{-fl})$ ))
        ?Q s1  $\leq$  ?C
    proof (rule wp-event-bind-bound-by-head-event[OF trace-bound])
      show ?Q None  $\Rightarrow$  ?Head None

```

```

    by simp
next
  fix f-fl t out
  assume cont:
    out ∈ set-dist (execute (return (fr, f-fl)) t)
  and hit: ?Q out
  have out-eq: out = Some ((fr, f-fl), t)
  using cont
  unfolding set-dist-def return.rep-eq dist-return-def
    dist-delta-dist delta-map-def
  by simp
  show ?Head (Some (f-fl, t))
    using hit unfolding out-eq by simp
qed
qed
qed

lemma wp-verifier-trace-fri-prefix-dependent-challenge-space-set-bound:
  fixes C :: prob
  assumes future: trace-fri-future-fresh s
  and subset:  $\bigwedge fr. B \text{ fr} \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$ 
  and bound:
     $\bigwedge fr. \text{nnreal } (\text{card } (B \text{ fr})) / \text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } \text{clength}) \leq C$ 
  shows
    wp-event verifier-trace-fri-prefix
      ( $\lambda out. \text{case out of}$ 
        None  $\Rightarrow$  False
        | Some ((fr, f-fl), -)  $\Rightarrow \text{map fst f-fl} \in B \text{ fr}$ ) s
       $\leq C$ 
  proof -
    let ?Q =
       $\lambda out. \text{case out of}$ 
        None  $\Rightarrow$  False
        | Some ((fr, f-fl), -)  $\Rightarrow \text{map fst f-fl} \in B \text{ fr}$ 
    show ?thesis
      unfolding verifier-trace-fri-prefix-def
    proof (rule wp-event-bind-bound-by-cont)
      show  $\neg ?Q \text{ None}$ 
        by simp
    next
      fix fr s1
      assume read-fr: Some (fr, s1) ∈ set-dist (execute read s)
      have future-s1: trace-fri-future-fresh s1
        by (rule read-preserves-trace-fri-future-fresh[OF future read-fr])
      let ?Head =
         $\lambda out. \text{case out of}$ 
          None  $\Rightarrow$  False
          | Some (f-fl, -)  $\Rightarrow \text{map fst f-fl} \in B \text{ fr}$ 

```

```

have trace-bound:
  wp-event (ntimes receive-trace-fri-commits (ceil-log clength))
    ?Head s1 ≤
      nnreal (card (B fr)) / nnreal (CARD('f) ^ ceil-log clength)
  by (rule wp-ntimes-receive-trace-fri-commits-challenge-space-set-bound
    [OF future-s1 subset[of fr]])
have trace-bound-C:
  wp-event (ntimes receive-trace-fri-commits (ceil-log clength))
    ?Head s1 ≤ C
  by (rule order-trans[OF trace-bound bound[of fr]])
show wp-event
  (ntimes receive-trace-fri-commits (ceil-log clength) ≫=
    (λf-fl. return (fr, f-fl)))
  ?Q s1 ≤ C
proof (rule wp-event-bind-bound-by-head-event[OF trace-bound-C])
  show ?Q None ⇒ ?Head None
  by simp
next
  fix f-fl t out
  assume cont:
    out ∈ set-dist (execute (return (fr, f-fl)) t)
  and hit: ?Q out
  have out-eq: out = Some ((fr, f-fl), t)
  using cont
  unfolding set-dist-def return.rep-eq dist-return-def
    dist-delta-dist delta-map-def
  by simp
  show ?Head (Some (f-fl, t))
  using hit unfolding out-eq by simp
qed
qed
qed

lemma receive-composition-fri-commits-outcome:
assumes outcome:
  Some ((b, r), t) ∈
    set-dist (execute receive-composition-fri-commits s)
shows
  ∃ rest.
    PTranscript s = r # rest ∧
    PState t = concat (PState s) r ∧
    PTranscript t = rest ∧
    s ≤ t ∧
    PTraceFriCounter t = PTraceFriCounter s ∧
    PCompositionFriCounter t = Suc (PCompositionFriCounter s) ∧
    PAlphaCounter t = PAlphaCounter s ∧
    PQueryCounter t = PQueryCounter s ∧
    fmlookup (HashMap t)
      (CompositionFriChallenge (PCompositionFriCounter s))

```

```

      (concat (PState s) r)) =
    Some b
proof -
  from outcome obtain s-read where
    read-r: Some (r, s-read) ∈ set-dist (execute read s)
  and rand-b:
    Some (b, t) ∈ set-dist (execute receive-composition-fri-challenge s-read)
  unfolding receive-composition-fri-commits-def receive-fri-commits-with-def
  by (auto elim!: set-dist-bindE)
  obtain rest where tr-s: PTranscript s = r # rest
proof -
  have PTranscript s ≠ []
    using read-r unfolding read-def protocol-read-def assert-def
    by (cases PTranscript s) (auto simp: throw-no-outcome elim!: set-dist-bindE)
  then obtain x xs where xs: PTranscript s = x # xs
    by (cases PTranscript s) auto
  have r-x: r = x
    using read-nonempty-outcome[OF read-r xs] by simp
  show ?thesis
    using that xs r-x by auto
qed
have read-res:
  s(PState := concat (PState s) r, PTranscript := rest)
  using read-nonempty-outcome[OF read-r tr-s] by simp
have rand-res:
  s-read ≤ t ∧
  PState t = PState s-read ∧
  PTranscript t = PTranscript s-read ∧
  fmlookup (HashMap t)
  (CompositionFriChallenge (PCompositionFriCounter s-read)
    (PState s-read)) =
  Some b
  using receive-composition-fri-challenge-outcome[OF rand-b] .
have rand-counter:
  PTraceFriCounter t = PTraceFriCounter s-read ∧
  PCompositionFriCounter t = Suc (PCompositionFriCounter s-read) ∧
  PAlphaCounter t = PAlphaCounter s-read ∧
  PQueryCounter t = PQueryCounter s-read
  using receive-composition-fri-challenge-counter-outcome[OF rand-b] by simp
have s-read-ext: s ≤ s-read
  using read-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have s-t: s ≤ t
  using s-read-ext rand-res by (meson hash-ext-trans)
show ?thesis
  using tr-s read-res rand-res rand-counter s-t by auto
qed

lemma receive-composition-fri-commits-preserves-composition-fri-future-fresh:
  assumes future: composition-fri-future-fresh s

```

```

and outcome:
  Some ((b, r), t) ∈
    set-dist (execute receive-composition-fri-commits s)
shows composition-fri-future-fresh t
proof –
from outcome obtain s-read where
  read-r: Some (r, s-read) ∈ set-dist (execute read s)
and rand-b:
  Some (b, t) ∈
    set-dist (execute receive-composition-fri-challenge s-read)
unfolding receive-composition-fri-commits-def receive-fri-commits-with-def
by (auto elim!: set-dist-bindE)
from read-outcome[OF read-r] have read-counts:
  PCompositionFriCounter s-read = PCompositionFriCounter s
by blast
have read-hash: HashMap s-read = HashMap s
by (rule read-preserves-hash-map[OF read-r])
have counter-t:
  PCompositionFriCounter t = Suc (PCompositionFriCounter s)
using receive-composition-fri-challenge-counter-outcome[OF rand-b]
  read-counts
by simp
show ?thesis
unfolding composition-fri-future-fresh-def
proof (intro allI impI)
fix i x
assume i-ge-t: PCompositionFriCounter t ≤ i
have i-ge-s: PCompositionFriCounter s ≤ i
using i-ge-t counter-t by simp
have neg:
  CompositionFriChallenge i x ≠
    CompositionFriChallenge
      (PCompositionFriCounter s-read) (PState s-read)
using i-ge-t counter-t read-counts by auto
have fmlookup (HashMap t) (CompositionFriChallenge i x) =
  fmlookup (HashMap s-read) (CompositionFriChallenge i x)
by (rule receive-composition-fri-challenge-preserves-other-lookup
  [OF rand-b neg])
also have ... = fmlookup (HashMap s) (CompositionFriChallenge i x)
using read-hash by simp
also have ... = None
using future i-ge-s unfolding composition-fri-future-fresh-def by blast
finally show fmlookup (HashMap t) (CompositionFriChallenge i x) = None .
qed
qed

```

```

lemma receive-composition-fri-commits-preserves-query-future-fresh:
assumes future: query-future-fresh s
and outcome:

```

```

    Some ((b, r), t) ∈
      set-dist (execute receive-composition-fri-commits s)
  shows query-future-fresh t
proof -
  from outcome obtain s-read where
    read-r: Some (r, s-read) ∈ set-dist (execute read s)
  and rand-b:
    Some (b, t) ∈
      set-dist (execute receive-composition-fri-challenge s-read)
  unfolding receive-composition-fri-commits-def receive-fri-commits-with-def
  by (auto elim!: set-dist-bindE)
  from read-outcome[OF read-r] have read-counts:
    PQueryCounter s-read = PQueryCounter s
  by blast
  have read-hash: HashMap s-read = HashMap s
  by (rule read-preserves-hash-map[OF read-r])
  have counter-t:
    PQueryCounter t = PQueryCounter s
  using receive-composition-fri-challenge-counter-outcome[OF rand-b]
    read-counts
  by simp
  show ?thesis
  unfolding query-future-fresh-def
  proof (intro allI impI)
    fix i x
    assume i-ge-t: PQueryCounter t ≤ i
    have i-ge-s: PQueryCounter s ≤ i
      using i-ge-t counter-t by simp
    have neg:
      QueryIndexChallenge i x ≠
        CompositionFriChallenge
          (PCompositionFriCounter s-read) (PState s-read)
    by simp
    have fmllookup (HashMap t) (QueryIndexChallenge i x) =
      fmllookup (HashMap s-read) (QueryIndexChallenge i x)
    by (rule receive-composition-fri-challenge-preserves-other-lookup
      [OF rand-b neg])
    also have ... = fmllookup (HashMap s) (QueryIndexChallenge i x)
      using read-hash by simp
    also have ... = None
      using future i-ge-s unfolding query-future-fresh-def by blast
    finally show fmllookup (HashMap t) (QueryIndexChallenge i x) = None .
  qed
qed

```

```

lemma wp-receive-composition-fri-commits-fresh-set:
  assumes future: composition-fri-future-fresh s
  shows
    wp-event receive-composition-fri-commits

```

```

    (λout. case out of
      None ⇒ False
      | Some ((b, r), -) ⇒ b ∈ B) s
    ≤ nnreal (card B) / nnreal size
  unfolding receive-composition-fri-commits-def receive-fri-commits-with-def
proof (rule wp-event-bind-bound-by-cont)
  show ¬ (case None of
    None ⇒ False | Some ((b, r), -) ⇒ b ∈ B)
  by simp
next
fix r s-read
assume read-r: Some (r, s-read) ∈ set-dist (execute read s)
have read-counts:
  PCompositionFriCounter s-read = PCompositionFriCounter s
  using read-outcome[OF read-r] by blast
have read-hash: HashMap s-read = HashMap s
  by (rule read-preserves-hash-map[OF read-r])
have fresh:
  fmlookup (HashMap s-read)
    (CompositionFriChallenge
      (PCompositionFriCounter s-read) (PState s-read)) = None
  using future read-counts read-hash
  unfolding composition-fri-future-fresh-def by simp
let ?Q =
  λout :: (('f × 'f) × ('f, 'a) protocol-channel-scheme) option.
    case out of None ⇒ False | Some ((b, r), -) ⇒ b ∈ B
have cont-eq:
  (λout. case out of
    None ⇒ (0 :: prob)
    | Some (b, u) ⇒
      wp (return (b, r)) (λout. if ?Q out then 1 else 0) u) =
  (λout :: (('f × ('f, 'a) protocol-channel-scheme) option.
    if (case out of None ⇒ False | Some (b, -) ⇒ b ∈ B)
    then 1 else 0)
  by (rule ext) (auto simp: wpsimps split: option.splits prod.splits)
have wp-event
  (receive-composition-fri-challenge ≫= (λb :: 'f. return (b, r)))
  ?Q s-read =
  wp-event receive-composition-fri-challenge
    (λout. case out of None ⇒ False | Some (b, -) ⇒ b ∈ B)
    s-read
  unfolding wp-event-def by (simp add: wpsimps cont-eq)
also have ... = nnreal (card B) / nnreal size
  by (rule wp-receive-composition-fri-challenge-fresh-set[OF fresh])
finally show
  wp-event
    (receive-composition-fri-challenge ≫= (λb :: 'f. return (b, r)))
    (λout. case out of None ⇒ False | Some ((b, r), -) ⇒ b ∈ B)
    s-read

```

$\leq \text{nnreal } (\text{card } B) / \text{nnreal size}$
by *simp*
qed

lemma *ntimes-receive-composition-fri-commits-preserves-composition-fri-future-fresh*:
assumes *outcome*:
 $\text{Some } (brs, t) \in$
 $\text{set-dist } (\text{execute } (ntimes \text{ receive-composition-fri-commits } n) s)$
and *future*: *composition-fri-future-fresh s*
shows *composition-fri-future-fresh t*
using *outcome future*
proof (*induction n arbitrary: s brs t*)
case 0
then show ?*case* **by** *simp*
next
case (*Suc n*)
from *Suc.prem1* **obtain** *br brs' s1* **where**
head:
 $\text{Some } (br, s1) \in$
 $\text{set-dist } (\text{execute } \text{receive-composition-fri-commits } s)$
and *tail*:
 $\text{Some } (brs', t) \in$
 $\text{set-dist } (\text{execute } (ntimes \text{ receive-composition-fri-commits } n) s1)$
and *brs-eq*: $brs = br \# brs'$
by (*auto elim!: set-dist-bindE*)
obtain *b r* **where** *br-eq*: $br = (b, r)$
by (*cases br*)
have *future-s1*:
composition-fri-future-fresh s1
by (*rule receive-composition-fri-commits-preserves-composition-fri-future-fresh*
 $[OF \text{ Suc.prem2 } \text{head}[\text{unfolded } br\text{-eq}]]$)
show ?*case*
by (*rule Suc.IH* [*OF tail future-s1*])
qed

lemma *ntimes-receive-composition-fri-commits-preserves-query-future-fresh*:
assumes *outcome*:
 $\text{Some } (brs, t) \in$
 $\text{set-dist } (\text{execute } (ntimes \text{ receive-composition-fri-commits } n) s)$
and *future*: *query-future-fresh s*
shows *query-future-fresh t*
using *outcome future*
proof (*induction n arbitrary: s brs t*)
case 0
then show ?*case* **by** *simp*
next
case (*Suc n*)
from *Suc.prem1* **obtain** *br brs' s1* **where**
head:


```

    Some (br, s1) ∈
      set-dist (execute receive-composition-fri-commits s)
  and tail:
    Some (brs', t) ∈
      set-dist (execute (ntimes receive-composition-fri-commits n) s1)
  by (auto elim!: set-dist-bindE)
  obtain b r where br-eq: br = (b, r)
  by (cases br)
  have future-s1:
    query-future-fresh s1
  by (rule receive-composition-fri-commits-preserves-query-future-fresh
      [OF Suc.premis(2) head[unfolded br-eq]])
  show ?case
  by (rule Suc.IH[OF tail future-s1])
qed

lemma wp-ntimes-receive-composition-fri-commits-exact-challenges-bound:
  assumes future: composition-fri-future-fresh s
  and len: length bs = n
  shows
    wp-event (ntimes receive-composition-fri-commits n)
      (λout. case out of
        None ⇒ False
        | Some (brs, -) ⇒ map fst brs = bs) s
      ≤ (1 / nnreal size) ^ n
  using len future
proof (induction n arbitrary: s bs)
  case 0
  then have bs-empty: bs = []
  by simp
  show ?case
  unfolding bs-empty by (simp add: wp-event-def wpsimps)
next
  case (Suc n)
  obtain b bs' where bs-eq: bs = b # bs'
  using Suc.premis(1) by (cases bs) auto
  have len-bs': length bs' = n
  using Suc.premis(1) bs-eq by simp
  let ?tail =
    (ntimes receive-composition-fri-commits n ::
      (('f × 'f) list, ('f, 'a) protocol-channel-scheme) state-monad)
  let ?Q =
    λout :: (((('f × 'f) list) ×
      ('f, 'a) protocol-channel-scheme) option.
      case out of None ⇒ False | Some (brs, -) ⇒ map fst brs = bs
  let ?Head =
    λout :: (((('f × 'f) × ('f, 'a) protocol-channel-scheme) option.
      case out of None ⇒ False | Some ((b0, r0), -) ⇒ b0 ∈ {b}
  have head-bound:

```

```

wp-event receive-composition-fri-commits ?Head s ≤ 1 / nnreal size
proof -
  have wp-event receive-composition-fri-commits ?Head s ≤
    nnreal (card ({b} :: 'f set)) / nnreal size
    by (rule wp-receive-composition-fri-commits-fresh-set[OF Suc.premis(2)])
  then show ?thesis by simp
qed
have bind-bound:
  wp-event
    (receive-composition-fri-commits ≫
      (λbr :: 'f × 'f.
        ?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs))))
  ?Q s
  ≤ wp-event receive-composition-fri-commits ?Head s *
    (1 / nnreal size) ^ n
proof (rule wp-event-bind-bound-by-head-and-cont[where P = ?Head])
  show ¬ ?Q None
    by simp
next
  fix br t
  assume not-head: ¬ ?Head (Some (br, t))
  obtain b0 r0 where br-eq: br = (b0, r0)
    by (cases br)
  have b0-ne: b0 ≠ b
    using not-head unfolding br-eq by simp
  have cont-false:
    wp-event (?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs)))
    ?Q t
    ≤ wp-event (?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs)))
      (λ- :: (((('f × 'f) list) ×
        ('f, 'a) protocol-channel-scheme) option. False) t
    by (rule wp-event-mono-on-support)
      (use b0-ne bs-eq br-eq in
        ‹auto simp: wpsimps elim!: set-dist-bindE split: option.splits prod.splits›)
  also have ... = 0
    unfolding wp-event-def wp-def dist-expect-def by simp
  finally have le-zero:
    wp-event (?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs)))
    ?Q t ≤ 0 .
  show
    wp-event (?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs)))
    ?Q t = 0
    by (rule antisym[OF le-zero]) simp
next
  fix br t
  assume head:
    Some (br, t) ∈
      set-dist (execute receive-composition-fri-commits s)
    and head-hit: ?Head (Some (br, t))

```

```

obtain  $b0\ r0$  where  $br\text{-}eq$ :  $br = (b0, r0)$ 
  by (cases  $br$ )
have  $b0\text{-}eq$ :  $b0 = b$ 
  using head-hit unfolding  $br\text{-}eq$  by simp
have  $future\text{-}t$ : composition-fri-future-fresh  $t$ 
  by (rule receive-composition-fri-commits-preserves-composition-fri-future-fresh
    [OF Suc.prems(2) head[unfolded  $br\text{-}eq$ ]])
have tail-bound:
  wp-event  $?tail$ 
    ( $\lambda out.$  case out of None  $\Rightarrow$  False
      | Some ( $brs, -$ )  $\Rightarrow$  map fst  $brs = bs'$ )  $t$ 
       $\leq (1 / nnreal\ size) ^ n$ 
  by (rule Suc.IH[OF len-bs'  $future\text{-}t$ ])
have cont-eq:
  ( $\lambda out.$  case out of
    None  $\Rightarrow (0 :: prob)$ 
    | Some ( $brs, u$ )  $\Rightarrow$ 
      wp (return ( $br \# brs$ )) ( $\lambda out.$  if  $?Q\ out$  then 1 else 0)  $u$ ) =
  ( $\lambda out :: ((f \times 'f)\ list) \times$ 
    ( $'f, 'a$ ) protocol-channel-scheme) option.
    if (case out of None  $\Rightarrow$  False
      | Some ( $brs, -$ )  $\Rightarrow$  map fst  $brs = bs'$ )
      then 1 else 0)
  by (rule ext)
  (auto simp: wpsimps  $b0\text{-}eq\ bs\text{-}eq\ br\text{-}eq$ 
    split: option.splits prod.splits)
have wp-event ( $?tail \gg= (\lambda brs :: (f \times 'f)\ list.$  return ( $br \# brs$ )))
   $?Q\ t =$ 
  wp-event  $?tail$ 
    ( $\lambda out.$  case out of None  $\Rightarrow$  False
      | Some ( $brs, -$ )  $\Rightarrow$  map fst  $brs = bs'$ )  $t$ 
  unfolding wp-event-def by (simp add: wpsimps cont-eq)
also have  $\dots \leq (1 / nnreal\ size) ^ n$ 
  by (rule tail-bound)
finally show
  wp-event ( $?tail \gg= (\lambda brs :: (f \times 'f)\ list.$  return ( $br \# brs$ )))
   $?Q\ t \leq (1 / nnreal\ size) ^ n$  .
qed
have wp-event (ntimes receive-composition-fri-commits (Suc  $n$ ))  $?Q\ s$ 
   $\leq$  wp-event receive-composition-fri-commits  $?Head\ s \ast$ 
   $(1 / nnreal\ size) ^ n$ 
  using bind-bound by simp
also have  $\dots \leq (1 / nnreal\ size) \ast (1 / nnreal\ size) ^ n$ 
  by (rule mult-right-mono[OF head-bound]) simp
also have  $\dots = (1 / nnreal\ size) ^ Suc\ n$ 
  by simp
finally show  $?case$  .
qed

```

lemma *wp-ntimes-receive-composition-fri-commits-finite-set-bound:*
assumes *future: composition-fri-future-fresh s*
and *finite-B: finite B*
and *lengths: $\bigwedge bs. bs \in B \implies \text{length } bs = n$*
shows

$$\text{wp-event } (ntimes \text{ receive-composition-fri-commits } n)$$

$$(\lambda out. \text{case out of}$$

$$\quad \text{None} \Rightarrow \text{False}$$

$$\quad | \text{Some } (brs, -) \Rightarrow \text{map fst } brs \in B) \ s$$

$$\leq \text{nnreal } (\text{card } B) / (\text{nnreal size}) ^ n$$
proof –
let *?P* =

$$\lambda bs \text{ out. case out of}$$

$$\quad \text{None} \Rightarrow \text{False}$$

$$\quad | \text{Some } (brs, -) \Rightarrow \text{map fst } brs = bs$$
have *event-mono:*

$$\text{wp-event } (ntimes \text{ receive-composition-fri-commits } n)$$

$$(\lambda out. \text{case out of}$$

$$\quad \text{None} \Rightarrow \text{False}$$

$$\quad | \text{Some } (brs, -) \Rightarrow \text{map fst } brs \in B) \ s$$

$$\leq \text{wp-event } (ntimes \text{ receive-composition-fri-commits } n)$$

$$(\lambda out. \exists bs \in B. ?P \ bs \ out) \ s$$
by (*rule wp-event-mono*) (*auto split: option.splits prod.splits*)
also have $\dots \leq (\sum bs \in B. (1 / \text{nnreal size}) ^ n)$
by (*rule wp-event-finite-UN-bound[OF finite-B]*)

$$(\text{rule wp-ntimes-receive-composition-fri-commits-exact-challenges-bound}$$

$$[OF \text{ future lengths}])$$
also have $\dots = \text{nnreal } (\text{card } B) * ((1 / \text{nnreal size}) ^ n)$
using *finite-B* **by** *simp*
also have $\dots = \text{nnreal } (\text{card } B) / (\text{nnreal size}) ^ n$
using *size-card* **by** *transfer (simp add: power-divide)*
finally show *?thesis* .
qed

lemma *wp-ntimes-receive-composition-fri-commits-challenge-space-set-bound:*
assumes *future: composition-fri-future-fresh s*
and *subset: $B \subseteq \text{fri-challenge-space } n$*
shows

$$\text{wp-event } (ntimes \text{ receive-composition-fri-commits } n)$$

$$(\lambda out. \text{case out of}$$

$$\quad \text{None} \Rightarrow \text{False}$$

$$\quad | \text{Some } (brs, -) \Rightarrow \text{map fst } brs \in B) \ s$$

$$\leq \text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}(f) ^ n)$$
proof –
have *finite-B: finite B*
by (*rule finite-subset[OF subset finite-fri-challenge-space]*)
have *lengths: $\bigwedge bs. bs \in B \implies \text{length } bs = n$*
using *subset unfolding fri-challenge-space-def* **by** *auto*
have $\text{wp-event } (ntimes \text{ receive-composition-fri-commits } n)$

```

  (λout. case out of
    None ⇒ False
    | Some (brs, -) ⇒ map fst brs ∈ B) s
  ≤ nnreal (card B) / (nnreal size) ^ n
  by (rule wp-ntimes-receive-composition-fri-commits-finite-set-bound
    [OF future finite-B lengths])
  also have ... = nnreal (card B) / nnreal (CARD('f) ^ n)
  using size-card by simp
  finally show ?thesis .
qed

```

lemma *wp-verifier-composition-fri-prefix-challenge-space-set-bound:*

```

assumes future: composition-fri-future-fresh s
and subset:
  ∧ dg. B dg ⊆ fri-challenge-space (ceil-log (to-nat dg + 1))
and bound:
  ∧ dg. nnreal (card (B dg)) /
    nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C

```

shows

```

wp-event verifier-composition-fri-prefix
  (λout. case out of
    None ⇒ False
    | Some ((-, -, -, dg, fl), -) ⇒ map fst fl ∈ B dg) s
  ≤ C

```

proof –

```

let ?Q =
  λout. case out of
    None ⇒ False
    | Some ((-, -, -, dg, fl), -) ⇒ map fst fl ∈ B dg

```

show ?thesis

unfolding verifier-composition-fri-prefix-def

proof (rule wp-event-bind-bound-by-cont)

show ¬ ?Q None

by simp

next

fix fr s1

assume read-fr: Some (fr, s1) ∈ set-dist (execute read s)

have future-s1: composition-fri-future-fresh s1

by (rule read-preserves-composition-fri-future-fresh
[OF future read-fr])

show wp-event

(ntimes receive-trace-fri-commits (ceil-log clength) ≫=

(λf.fl. read ≫=

(λf.final.

mmap (replicate (length spec) alpha-round) ≫=

(λas. read ≫=

(λdg.

assert (to-nat dg ≤ maxDegree) ≫=

(λ-. ntimes receive-composition-fri-commits

```

      (ceil-log (to-nat dg + 1)) >=>
      (λfl.
        return (fr, f-fl, f-final, as, dg, fl))))))
    ?Q s1 ≤ C
  proof (rule wp-event-bind-bound-by-cont)
    show ¬ ?Q None
      by simp
  next
    fix f-fl s2
    assume trace-fri:
      Some (f-fl, s2) ∈
        set-dist
          (execute (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
    have future-s2: composition-fri-future-fresh s2
      by (rule ntimes-receive-trace-fri-commits-preserves-composition-fri-future-fresh
        [OF trace-fri future-s1])
    show wp-event
      (read >=>
        (λf-final.
          mmap (replicate (length spec) alpha-round) >=>
            (λas. read >=>
              (λdg.
                assert (to-nat dg ≤ maxDegree) >=>
                (λ-. ntimes receive-composition-fri-commits
                  (ceil-log (to-nat dg + 1)) >=>
                  (λfl.
                    return (fr, f-fl, f-final, as, dg, fl)))))))
        ?Q s2 ≤ C
    proof (rule wp-event-bind-bound-by-cont)
      show ¬ ?Q None
        by simp
    next
      fix f-final s3
      assume read-final: Some (f-final, s3) ∈ set-dist (execute read s2)
      have future-s3: composition-fri-future-fresh s3
        by (rule read-preserves-composition-fri-future-fresh
          [OF future-s2 read-final])
      show wp-event
        (mmap (replicate (length spec) alpha-round) >=>
          (λas. read >=>
            (λdg.
              assert (to-nat dg ≤ maxDegree) >=>
              (λ-. ntimes receive-composition-fri-commits
                (ceil-log (to-nat dg + 1)) >=>
                (λfl.
                  return (fr, f-fl, f-final, as, dg, fl)))))))
        ?Q s3 ≤ C
    proof (rule wp-event-bind-bound-by-cont)
      show ¬ ?Q None

```

```

    by simp
next
  fix as s4
  assume alpha:
    Some (as, s4) ∈
      set-dist
        (execute (mmap (replicate (length spec) alpha-round)) s3)
  have future-s4: composition-fri-future-fresh s4
    by (rule mmap-alpha-round-preserves-composition-fri-future-fresh
        [OF future-s3 alpha])
  show wp-event
    (read ≫=
      (λdg.
        assert (to-nat dg ≤ maxDegree) ≫=
          (λ-. ntimes receive-composition-fri-commits
            (ceil-log (to-nat dg + 1)) ≫=
              (λfl.
                return (fr, f-fl, f-final, as, dg, fl))))))
    ?Q s4 ≤ C
  proof (rule wp-event-bind-bound-by-cont)
    show ¬ ?Q None
      by simp
  next
    fix dg s5
    assume read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
    have future-s5: composition-fri-future-fresh s5
      by (rule read-preserves-composition-fri-future-fresh
          [OF future-s4 read-dg])
    show wp-event
      (assert (to-nat dg ≤ maxDegree) ≫=
        (λ-. ntimes receive-composition-fri-commits
          (ceil-log (to-nat dg + 1)) ≫=
            (λfl.
              return (fr, f-fl, f-final, as, dg, fl))))
      ?Q s5 ≤ C
    proof (rule wp-event-bind-bound-by-cont)
      show ¬ ?Q None
        by simp
    next
      fix u s6
      assume assert-out:
        Some (u, s6) ∈
          set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
      have s6-eq: s6 = s5
        using assert-out unfolding assert-def
        by (cases to-nat dg ≤ maxDegree)
          (auto simp: throw-no-outcome)
      have future-s6: composition-fri-future-fresh s6
        using future-s5 unfolding s6-eq .

```

```

let ?Head =
  λout. case out of
    None ⇒ False
  | Some (fl, -) ⇒ map fst fl ∈ B dg
have comp-bound:
  wp-event
    (ntimes receive-composition-fri-commits
      (ceil-log (to-nat dg + 1)))
  ?Head s6 ≤
    nnreal (card (B dg)) /
    nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
by (rule
  wp-ntimes-receive-composition-fri-commits-challenge-space-set-bound
  [OF future-s6 subset])
have comp-bound-C:
  wp-event
    (ntimes receive-composition-fri-commits
      (ceil-log (to-nat dg + 1)))
  ?Head s6 ≤ C
by (rule order-trans[OF comp-bound bound])
show wp-event
  (ntimes receive-composition-fri-commits
    (ceil-log (to-nat dg + 1)) >>=
    (λfl.
      return (fr, f-fl, f-final, as, dg, fl)))
  ?Q s6 ≤ C
proof (rule wp-event-bind-bound-by-head-event[OF comp-bound-C])
  show ?Q None ⇒ ?Head None
  by simp
next
  fix fl t out
  assume cont:
    out ∈
      set-dist
        (execute
          (return (fr, f-fl, f-final, as, dg, fl)) t)
  and hit: ?Q out
  have out-eq:
    out = Some ((fr, f-fl, f-final, as, dg, fl), t)
  using cont
  unfolding set-dist-def return.rep-eq dist-return-def
    dist-delta-dist delta-map-def
  by simp
  show ?Head (Some (fl, t))
  using hit unfolding out-eq by simp
qed
qed
qed
qed

```


qed
 qed
 qed
 qed

lemma *ntimes-receive-fri-commits-with-outcome:*

fixes *receive-commits* :: ('f, 'f × 'f, 'a) protocol-c-monad
and *counter* :: ('f, 'a) protocol-channel-scheme ⇒ nat
and *preserve1* :: ('f, 'a) protocol-channel-scheme ⇒ nat
and *preserve2* :: ('f, 'a) protocol-channel-scheme ⇒ nat
and *preserve3* :: ('f, 'a) protocol-channel-scheme ⇒ nat
and *tag* :: nat ⇒ 'f ⇒ 'f protocol-hash-input

assumes *step-outcome:*

$\bigwedge s\ b\ r\ t. \text{Some } ((b, r), t) \in \text{set-dist } (\text{execute receive-commits } s) \implies$
 $\exists \text{rest.}$
 $P\text{Transcript } s = r \# \text{rest} \wedge$
 $P\text{State } t = \text{concat } (P\text{State } s) \ r \wedge$
 $P\text{Transcript } t = \text{rest} \wedge$
 $s \leq t \wedge$
 $\text{counter } t = \text{Suc } (\text{counter } s) \wedge$
 $\text{preserve1 } t = \text{preserve1 } s \wedge$
 $\text{preserve2 } t = \text{preserve2 } s \wedge$
 $\text{preserve3 } t = \text{preserve3 } s \wedge$
 $\text{fmlookup } (\text{HashMap } t) (\text{tag } (\text{counter } s) (\text{concat } (P\text{State } s) \ r)) = \text{Some } b$

and *outcome:*

$\text{Some } (\text{brs}, t) \in \text{set-dist } (\text{execute } (\text{ntimes receive-commits } n) \ s)$

shows

$\text{length brs} = n \wedge$
 $P\text{Transcript } s = \text{map snd brs} \ @ \ P\text{Transcript } t \wedge$
 $P\text{State } t = \text{foldl concat } (P\text{State } s) (\text{map snd brs}) \wedge$
 $s \leq t \wedge$
 $\text{counter } t = \text{counter } s + n \wedge$
 $\text{preserve1 } t = \text{preserve1 } s \wedge$
 $\text{preserve2 } t = \text{preserve2 } s \wedge$
 $\text{preserve3 } t = \text{preserve3 } s \wedge$
 $(\forall i < n.$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{tag } (\text{counter } s + i)$
 $(\text{foldl concat } (P\text{State } s) (\text{take } (\text{Suc } i) (\text{map snd brs})))) =$
 $\text{Some } (\text{fst } (\text{brs } ! \ i)))$

using *outcome*

proof (*induction n arbitrary: s brs t*)

case 0

then show ?case

by (*simp add: hash-ext-refl*)

next

case (*Suc n*)

from *Suc.prem*s **obtain** *br brs' s1* **where**

head: Some (br, s1) ∈ set-dist (execute receive-commits s)

```

and tail:
  Some (brs', t) ∈
    set-dist (execute (ntimes receive-commits n) s1)
and brs-eq: brs = br # brs'
by (auto elim!: set-dist-bindE)
obtain b r where br-eq: br = (b, r)
by (cases br)
from step-outcome[OF head[unfolded br-eq]]
obtain rest1 where
  tr-s: PTranscript s = r # rest1
and st-s1: PState s1 = concat (PState s) r
and tr-s1: PTranscript s1 = rest1
and ext-head: s ≤ s1
and counter-s1: counter s1 = Suc (counter s)
and preserve1-s1: preserve1 s1 = preserve1 s
and preserve2-s1: preserve2 s1 = preserve2 s
and preserve3-s1: preserve3 s1 = preserve3 s
and lookup-head:
  fmlookup (HashMap s1) (tag (counter s) (concat (PState s) r)) = Some b
by blast
have tail-res:
  length brs' = n ∧
    PTranscript s1 = map snd brs' @ PTranscript t ∧
    PState t = foldl concat (PState s1) (map snd brs') ∧
    s1 ≤ t ∧
    counter t = counter s1 + n ∧
    preserve1 t = preserve1 s1 ∧
    preserve2 t = preserve2 s1 ∧
    preserve3 t = preserve3 s1 ∧
    (∀ i < n.
      fmlookup (HashMap t)
        (tag (counter s1 + i)
          (foldl concat (PState s1) (take (Suc i) (map snd brs')))) =
          Some (fst (brs' ! i)))
    using Suc.IH[OF tail] .
have ext-s1-t: s1 ≤ t
using tail-res by simp
have lookup-head-t:
  fmlookup (HashMap t) (tag (counter s) (concat (PState s) r)) = Some b
using hash-extension-lookup[OF lookup-head ext-s1-t] .
have ext-combined: s ≤ t
using ext-head ext-s1-t by (meson hash-ext-trans)
have counter-t: counter t = counter s + Suc n
using tail-res counter-s1 by simp
have preserves-t:
  preserve1 t = preserve1 s ∧
  preserve2 t = preserve2 s ∧
  preserve3 t = preserve3 s
using tail-res preserve1-s1 preserve2-s1 preserve3-s1 by simp

```

```

have lookups:
   $\forall i < \text{Suc } n.$ 
    fmlookup (HashMap t)
      (tag (counter s + i)
        (foldl concat (PState s) (take (Suc i) (map snd brs)))) =
      Some (fst (brs ! i))
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{Suc } n$ 
  show fmlookup (HashMap t)
    (tag (counter s + i)
      (foldl concat (PState s) (take (Suc i) (map snd brs)))) =
    Some (fst (brs ! i))
proof (cases i)
  case 0
  then show ?thesis
    using brs-eq br-eq lookup-head-t by simp
next
  case (Suc j)
  have j-bound:  $j < n$ 
    using i-bound Suc by simp
  have state-shift:
    foldl concat (PState s) (take (Suc (Suc j)) (map snd brs)) =
    foldl concat (PState s1) (take (Suc j) (map snd brs'))
    using brs-eq br-eq st-s1 by simp
  have brs-nth:  $\text{brs} ! \text{Suc } j = \text{brs}' ! j$ 
    using brs-eq j-bound by simp
  show ?thesis
    using tail-res j-bound Suc state-shift brs-nth counter-s1 by simp
qed
qed
have as-eq:
  PTranscript s = map snd brs @ PTranscript t  $\wedge$ 
  PState t = foldl concat (PState s) (map snd brs)
  using tail-res tr-s tr-s1 st-s1 brs-eq br-eq by simp
show ?case
  using tail-res tr-s tr-s1 st-s1 ext-combined lookups as-eq brs-eq br-eq
  counter-t preserves-t
  by simp
qed

```

```

lemma ntimes-receive-trace-fri-commits-outcome:
assumes outcome:
  Some (brs, t)  $\in$ 
    set-dist (execute (ntimes receive-trace-fri-commits n) s)
shows
  length brs = n  $\wedge$ 
  PTranscript s = map snd brs @ PTranscript t  $\wedge$ 
  PState t = foldl concat (PState s) (map snd brs)  $\wedge$ 

```

```

s ≤ t ∧
PTraceFriCounter t = PTraceFriCounter s + n ∧
PCompositionFriCounter t = PCompositionFriCounter s ∧
PAlphaCounter t = PAlphaCounter s ∧
PQueryCounter t = PQueryCounter s ∧
(∀ i < n.
  fmllookup (HashMap t)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (PState s) (take (Suc i) (map snd brs))))) =
    Some (fst (brs ! i)))
proof –
have res:
length brs = n ∧
PTranscript s = map snd brs @ PTranscript t ∧
PState t = foldl concat (PState s) (map snd brs) ∧
s ≤ t ∧
PTraceFriCounter t = PTraceFriCounter s + n ∧
PCompositionFriCounter t = PCompositionFriCounter s ∧
PAlphaCounter t = PAlphaCounter s ∧
PQueryCounter t = PQueryCounter s ∧
(∀ i < n.
  fmllookup (HashMap t)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (PState s) (take (Suc i) (map snd brs))))) =
    Some (fst (brs ! i)))
by (rule ntimes-receive-fri-commits-with-outcome
  [of receive-trace-fri-commits PTraceFriCounter PCompositionFriCounter
    PAlphaCounter PQueryCounter TraceFriChallenge,
    OF receive-trace-fri-commits-outcome outcome])
then show ?thesis .
qed

```

lemma *ntimes-receive-composition-fri-commits-outcome*:

assumes *outcome*:

Some (brs, t) ∈
set-dist (execute (ntimes receive-composition-fri-commits n) s)

shows

length brs = n ∧
PTranscript s = map snd brs @ PTranscript t ∧
PState t = foldl concat (PState s) (map snd brs) ∧
s ≤ t ∧
PCompositionFriCounter t = PCompositionFriCounter s + n ∧
PTraceFriCounter t = PTraceFriCounter s ∧
PAlphaCounter t = PAlphaCounter s ∧
PQueryCounter t = PQueryCounter s ∧
(∀ i < n.
 fmllookup (HashMap t)
 (CompositionFriChallenge (PCompositionFriCounter s + i)
 (foldl concat (PState s) (take (Suc i) (map snd brs))))) =

Some (fst (brs ! i)))

proof –

have *composition-step-outcome*:

$\bigwedge s \ b \ r \ t. \text{Some } ((b, r), t) \in$
 $\text{set-dist } (\text{execute receive-composition-fri-commits } s) \implies$
 $\exists \text{rest.}$
 $P\text{Transcript } s = r \# \text{rest} \wedge$
 $P\text{State } t = \text{concat } (P\text{State } s) \ r \wedge$
 $P\text{Transcript } t = \text{rest} \wedge$
 $s \leq t \wedge$
 $P\text{CompositionFriCounter } t = \text{Suc } (P\text{CompositionFriCounter } s) \wedge$
 $P\text{TraceFriCounter } t = P\text{TraceFriCounter } s \wedge$
 $P\text{AlphaCounter } t = P\text{AlphaCounter } s \wedge$
 $P\text{QueryCounter } t = P\text{QueryCounter } s \wedge$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{CompositionFriChallenge } (P\text{CompositionFriCounter } s)$
 $(\text{concat } (P\text{State } s) \ r)) =$
 $\text{Some } b$

using *receive-composition-fri-commits-outcome* **by** *blast*

have *res*:

$\text{length } \text{brs} = n \wedge$
 $P\text{Transcript } s = \text{map snd } \text{brs} \ @ \ P\text{Transcript } t \wedge$
 $P\text{State } t = \text{foldl concat } (P\text{State } s) \ (\text{map snd } \text{brs}) \wedge$
 $s \leq t \wedge$
 $P\text{CompositionFriCounter } t = P\text{CompositionFriCounter } s + n \wedge$
 $P\text{TraceFriCounter } t = P\text{TraceFriCounter } s \wedge$
 $P\text{AlphaCounter } t = P\text{AlphaCounter } s \wedge$
 $P\text{QueryCounter } t = P\text{QueryCounter } s \wedge$
 $(\forall i < n.$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{CompositionFriChallenge } (P\text{CompositionFriCounter } s + i)$
 $(\text{foldl concat } (P\text{State } s) \ (\text{take } (\text{Suc } i) \ (\text{map snd } \text{brs})))) =$
 $\text{Some } (\text{fst } (\text{brs } ! \ i)))$

using *composition-step-outcome* *outcome*

by (rule *ntimes-receive-fri-commits-with-outcome*
[*of* *receive-composition-fri-commits* *PCompositionFriCounter*
PTraceFriCounter *PAlphaCounter* *PQueryCounter* *CompositionFriChallenge*])

then show *?thesis* .

qed

lemma *ntimes-receive-composition-fri-commits-roots-outcome*:

assumes *outcome*:

$\text{Some } (\text{brs}, t) \in$
 $\text{set-dist } (\text{execute } (\text{ntimes receive-composition-fri-commits } n) \ s)$

shows $\text{map snd } \text{brs} = \text{take } n \ (P\text{Transcript } s)$

proof –

have *res*:

$\text{length } \text{brs} = n \wedge P\text{Transcript } s = \text{map snd } \text{brs} \ @ \ P\text{Transcript } t$

using *ntimes-receive-composition-fri-commits-outcome*[*OF outcome*] **by** *simp*

```

have len: length (map snd brs) = n
  using res by simp
have take n (PTranscript s) = take n (map snd brs @ PTranscript t)
  using res by simp
also have ... = map snd brs
  using len by simp
finally show ?thesis
  by simp
qed

lemma wp-ntimes-receive-composition-fri-commits-root-dependent-challenge-space-set-bound:
  fixes B :: 'f list  $\Rightarrow$  'f list set
  assumes future: composition-fri-future-fresh s
  and subset:  $\bigwedge$  roots. B roots  $\subseteq$  fri-challenge-space n
  shows
    wp-event (ntimes receive-composition-fri-commits n)
      ( $\lambda$ out. case out of
        None  $\Rightarrow$  False
        | Some (brs, -)  $\Rightarrow$  map fst brs  $\in$  B (map snd brs)) s
       $\leq$  nnreal (card (B (take n (PTranscript s)))) /
        nnreal (CARD('f) ^ n)
  proof -
    let ?roots = take n (PTranscript s)
    let ?Q =
       $\lambda$ out. case out of
        None  $\Rightarrow$  False
        | Some (brs, -)  $\Rightarrow$  map fst brs  $\in$  B (map snd brs)
    let ?Q' =
       $\lambda$ out. case out of
        None  $\Rightarrow$  False
        | Some (brs, -)  $\Rightarrow$  map fst brs  $\in$  B ?roots
    have fixed-bound:
      wp-event (ntimes receive-composition-fri-commits n) ?Q' s
       $\leq$  nnreal (card (B ?roots)) / nnreal (CARD('f) ^ n)
    by (rule wp-ntimes-receive-composition-fri-commits-challenge-space-set-bound
      [OF future subset])
    have event-le:
      wp-event (ntimes receive-composition-fri-commits n) ?Q s
       $\leq$  wp-event (ntimes receive-composition-fri-commits n) ?Q' s
    proof (rule wp-event-mono-on-support)
      fix out
      assume out-supp:
        out  $\in$  set-dist (execute (ntimes receive-composition-fri-commits n) s)
      and hit: ?Q out
      show ?Q' out
    proof (cases out)
      case None
      then show ?thesis
        using hit by simp
    end
  end

```

```

next
  case (Some pair)
  then obtain brs t where out-eq: out = Some (brs, t)
    by (cases pair) auto
  have roots-eq: map snd brs = ?roots
    by (rule ntimes-receive-composition-fri-commits-roots-outcome)
      (use out-supp out-eq in simp)
  show ?thesis
    using hit roots-eq out-eq by simp
qed
qed
show ?thesis
  by (rule order-trans[OF event-le fixed-bound])
qed

lemma wp-ntimes-receive-composition-fri-commits-root-dependent-bound:
  fixes C :: prob
  assumes future: composition-fri-future-fresh s
    and subset:  $\bigwedge \text{roots}. B \text{ roots} \subseteq \text{fri-challenge-space } n$ 
    and bound:
      
$$\bigwedge \text{roots}. \text{nnreal} (\text{card } (B \text{ roots})) / \text{nnreal} (\text{CARD}(f) ^ n) \leq C$$

  shows
    wp-event (ntimes receive-composition-fri-commits n)
      ( $\lambda \text{out}. \text{case out of}$ 
        None  $\Rightarrow$  False
        | Some (brs, -)  $\Rightarrow$  map fst brs  $\in B$  (map snd brs)) s
       $\leq C$ 
  proof -
    have root-bound:
      wp-event (ntimes receive-composition-fri-commits n)
        ( $\lambda \text{out}. \text{case out of}$ 
          None  $\Rightarrow$  False
          | Some (brs, -)  $\Rightarrow$  map fst brs  $\in B$  (map snd brs)) s
         $\leq \text{nnreal} (\text{card } (B (\text{take } n (\text{PTranscript } s)))) / \text{nnreal} (\text{CARD}(f) ^ n)$ 
      by (rule
        wp-ntimes-receive-composition-fri-commits-root-dependent-challenge-space-set-bound
        [OF future subset])
    also have ...  $\leq C$ 
      by (rule bound)
    finally show ?thesis .
  qed

```

```

lemma wp-verifier-composition-fri-prefix-root-dependent-challenge-space-set-bound:
  fixes C :: prob
  assumes future: composition-fri-future-fresh s
    and subset:
       $\bigwedge \text{fr } f\text{-fl } f\text{-final as dg roots}.$ 

```

```

    B fr f-fl f-final as dg roots  $\subseteq$ 
      fri-challenge-space (ceil-log (to-nat dg + 1))
and bound:
   $\bigwedge$  fr f-fl f-final as dg roots.
    nreal (card (B fr f-fl f-final as dg roots)) /
      nreal (CARD('f)  $\wedge$  ceil-log (to-nat dg + 1))  $\leq C$ 
shows
  wp-event verifier-composition-fri-prefix
    ( $\lambda$ out. case out of
      None  $\Rightarrow$  False
      | Some ((fr, f-fl, f-final, as, dg, fl), -)  $\Rightarrow$ 
        map fst fl  $\in$  B fr f-fl f-final as dg (map snd fl)) s
     $\leq C$ 
proof -
  let ?Q =
     $\lambda$ out. case out of
      None  $\Rightarrow$  False
      | Some ((fr, f-fl, f-final, as, dg, fl), -)  $\Rightarrow$ 
        map fst fl  $\in$  B fr f-fl f-final as dg (map snd fl)
  show ?thesis
    unfolding verifier-composition-fri-prefix-def
  proof (rule wp-event-bind-bound-by-cont)
    show  $\neg$  ?Q None
      by simp
  next
    fix fr s1
    assume read-fr: Some (fr, s1)  $\in$  set-dist (execute read s)
    have future-s1: composition-fri-future-fresh s1
      by (rule read-preserves-composition-fri-future-fresh
        [OF future read-fr])
    show wp-event
      (ntimes receive-trace-fri-commits (ceil-log clength)  $\gg$ 
        ( $\lambda$ f-fl. read  $\gg$ 
          ( $\lambda$ f-final.
            mmap (replicate (length spec) alpha-round)  $\gg$ 
              ( $\lambda$ as. read  $\gg$ 
                ( $\lambda$ dg.
                  assert (to-nat dg  $\leq$  maxDegree)  $\gg$ 
                    ( $\lambda$ -. ntimes receive-composition-fri-commits
                      (ceil-log (to-nat dg + 1))  $\gg$ 
                        ( $\lambda$ fl.
                          return (fr, f-fl, f-final, as, dg, fl))))))))
      ?Q s1  $\leq C$ 
    proof (rule wp-event-bind-bound-by-cont)
      show  $\neg$  ?Q None
        by simp
  next
    fix f-fl s2
    assume trace-fri:

```



```

    Some (f-fl, s2) ∈
      set-dist
        (execute (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
  have future-s2: composition-fri-future-fresh s2
  by (rule ntimes-receive-trace-fri-commits-preserves-composition-fri-future-fresh
      [OF trace-fri future-s1])
  show wp-event
    (read ≫=
      (λf-final.
        mmap (replicate (length spec) alpha-round) ≫=
          (λas. read ≫=
            (λdg.
              assert (to-nat dg ≤ maxDegree) ≫=
                (λ-. ntimes receive-composition-fri-commits
                  (ceil-log (to-nat dg + 1)) ≫=
                    (λfl.
                      return (fr, f-fl, f-final, as, dg, fl)))))))
    ?Q s2 ≤ C
  proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?Q None
  by simp
  next
  fix f-final s3
  assume read-final: Some (f-final, s3) ∈ set-dist (execute read s2)
  have future-s3: composition-fri-future-fresh s3
  by (rule read-preserves-composition-fri-future-fresh
      [OF future-s2 read-final])
  show wp-event
    (mmap (replicate (length spec) alpha-round) ≫=
      (λas. read ≫=
        (λdg.
          assert (to-nat dg ≤ maxDegree) ≫=
            (λ-. ntimes receive-composition-fri-commits
              (ceil-log (to-nat dg + 1)) ≫=
                (λfl.
                  return (fr, f-fl, f-final, as, dg, fl)))))))
    ?Q s3 ≤ C
  proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?Q None
  by simp
  next
  fix as s4
  assume alpha:
    Some (as, s4) ∈
      set-dist
        (execute (mmap (replicate (length spec) alpha-round)) s3)
  have future-s4: composition-fri-future-fresh s4
  by (rule mmap-alpha-round-preserves-composition-fri-future-fresh
      [OF future-s3 alpha])

```

```

show wp-event
  (read  $\gg$ 
    ( $\lambda dg.$ 
      assert (to-nat dg  $\leq$  maxDegree)  $\gg$ 
        ( $\lambda.$  ntimes receive-composition-fri-commits
          (ceil-log (to-nat dg + 1))  $\gg$ 
            ( $\lambda fl.$ 
              return (fr, f-fl, f-final, as, dg, fl))))))
  ?Q s4  $\leq$  C
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg$  ?Q None
    by simp
next
  fix dg s5
  assume read-dg: Some (dg, s5)  $\in$  set-dist (execute read s4)
  have future-s5: composition-fri-future-fresh s5
    by (rule read-preserves-composition-fri-future-fresh
      [OF future-s4 read-dg])
  show wp-event
    (assert (to-nat dg  $\leq$  maxDegree)  $\gg$ 
      ( $\lambda.$  ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1))  $\gg$ 
          ( $\lambda fl.$ 
            return (fr, f-fl, f-final, as, dg, fl))))))
    ?Q s5  $\leq$  C
  proof (rule wp-event-bind-bound-by-cont)
    show  $\neg$  ?Q None
      by simp
  next
    fix u s6
    assume assert-out:
      Some (u, s6)  $\in$ 
        set-dist (execute (assert (to-nat dg  $\leq$  maxDegree)) s5)
    have s6-eq: s6 = s5
      using assert-out unfolding assert-def
      by (cases to-nat dg  $\leq$  maxDegree)
        (auto simp: throw-no-outcome)
    have future-s6: composition-fri-future-fresh s6
      using future-s5 unfolding s6-eq .
    let ?Head =
       $\lambda out.$  case out of
        None  $\Rightarrow$  False
        | Some (fl,  $-$ )  $\Rightarrow$ 
          map fst fl  $\in$  B fr f-fl f-final as dg (map snd fl)
    have comp-bound-C:
      wp-event
        (ntimes receive-composition-fri-commits
          (ceil-log (to-nat dg + 1))))
      ?Head s6  $\leq$  C

```

```

    by (rule
      wp-ntimes-receive-composition-fri-commits-root-dependent-bound
      [where B = B fr f-fl f-final as dg,
        OF future-s6])
      (use subset bound in simp-all)
  show wp-event
    (ntimes receive-composition-fri-commits
      (ceil-log (to-nat dg + 1)) ≥
      (λfl.
        return (fr, f-fl, f-final, as, dg, fl)))
    ?Q s6 ≤ C
  proof (rule wp-event-bind-bound-by-head-event[OF comp-bound-C])
    show ?Q None ⇒ ?Head None
      by simp
  next
    fix fl t out
    assume cont:
      out ∈
        set-dist
          (execute
            (return (fr, f-fl, f-final, as, dg, fl)) t)
      and hit: ?Q out
    have out-eq:
      out = Some ((fr, f-fl, f-final, as, dg, fl), t)
    using cont
    unfolding set-dist-def return.rep-eq dist-return-def
      dist-delta-dist delta-map-def
    by simp
    show ?Head (Some (fl, t))
      using hit unfolding out-eq by simp
  qed
qed
qed
qed
qed
qed
qed
qed
qed

```

lemma *verifier-alpha-prefix-outcome:*

assumes *outcome:*

Some ((fr, f-fl, f-final, as), t) ∈
set-dist (execute verifier-alpha-prefix s)

shows

length f-fl = ceil-log clength ∧
length as = length spec ∧
PTranscript s =
[fr] @ map snd f-fl @ [f-final] @ as @ PTranscript t ∧
PState t =

```

    foldl concat
      (concat (foldl concat (concat (PState s) fr) (map snd f-fl))
        f-final)
      as ∧
    PQueryCounter t = PQueryCounter s
  proof -
  from outcome obtain s1 s2 s3 s4 where
    read-fr: Some (fr, s1) ∈ set-dist (execute read s)
  and trace-fri:
    Some (f-fl, s2) ∈
      set-dist (execute
        (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
  and read-final: Some (f-final, s3) ∈ set-dist (execute read s2)
  and alpha-out:
    Some (as, s4) ∈
      set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
  and ret:
    Some ((fr, f-fl, f-final, as), t) ∈
      set-dist (execute (return (fr, f-fl, f-final, as)) s4)
  unfolding verifier-alpha-prefix-def
  by (auto elim!: set-dist-bindE)
  from read-outcome[OF read-fr] obtain rest1 where
    tr-s: PTranscript s = fr # rest1
  and st-s1: PState s1 = concat (PState s) fr
  and tr-s1: PTranscript s1 = rest1
  and query-count-s1: PQueryCounter s1 = PQueryCounter s
  by blast
  have trace-res:
    length f-fl = ceil-log clength ∧
    PTranscript s1 = map snd f-fl @ PTranscript s2 ∧
    PState s2 = foldl concat (PState s1) (map snd f-fl) ∧
    PQueryCounter s2 = PQueryCounter s1
  using ntimes-receive-trace-fri-commits-outcome[OF trace-fri] by simp
  from read-outcome[OF read-final] obtain rest3 where
    tr-s2: PTranscript s2 = f-final # rest3
  and st-s3: PState s3 = concat (PState s2) f-final
  and tr-s3: PTranscript s3 = rest3
  and query-count-s3: PQueryCounter s3 = PQueryCounter s2
  by blast
  have alpha-res:
    length as = length spec ∧
    PTranscript s3 = as @ PTranscript s4 ∧
    PState s4 = foldl concat (PState s3) as ∧
    PQueryCounter s4 = PQueryCounter s3
  using mmap-alpha-round-outcome[OF alpha-out] by simp
  have t-eq: t = s4
  using ret by simp
  show ?thesis
  using tr-s tr-s1 trace-res tr-s2 tr-s3 alpha-res st-s1 st-s3

```

query-count-s1 query-count-s3 t-eq
 by simp
 qed

lemma *verifier-trace-fri-prefix-outcome*:

assumes *outcome*:

Some ((fr, f-fl), t) ∈
 set-dist (execute verifier-trace-fri-prefix s)

shows

length f-fl = ceil-log clength ∧
 PTranscript s = [fr] @ map snd f-fl @ PTranscript t ∧
 PState t = foldl concat (concat (PState s) fr) (map snd f-fl) ∧
 s ≤ t ∧
 PTraceFriCounter t = PTraceFriCounter s + ceil-log clength ∧
 PCompositionFriCounter t = PCompositionFriCounter s ∧
 PAlphaCounter t = PAlphaCounter s ∧
 PQueryCounter t = PQueryCounter s ∧
 (∀ i < length f-fl.
 fmllookup (HashMap t)
 (TraceFriChallenge (PTraceFriCounter s + i)
 (foldl concat (concat (PState s) fr)
 (take (Suc i) (map snd f-fl)))) =
 Some (fst (f-fl ! i)))

proof –

from *outcome* **obtain** s1 s2 **where**

read-fr: Some (fr, s1) ∈ set-dist (execute read s)

and trace-fri:

Some (f-fl, s2) ∈
 set-dist (execute
 (ntimes receive-trace-fri-commits (ceil-log clength)) s1)

and ret:

Some ((fr, f-fl), t) ∈ set-dist (execute (return (fr, f-fl)) s2)

unfolding *verifier-trace-fri-prefix-def*

by (auto elim!: set-dist-bindE)

from read-outcome[OF read-fr] **obtain** rest1 **where**

tr-s: PTranscript s = fr # rest1

and st-s1: PState s1 = concat (PState s) fr

and tr-s1: PTranscript s1 = rest1

and ext-s-s1: s ≤ s1

and trace-count-s1: PTraceFriCounter s1 = PTraceFriCounter s

and comp-count-s1: PCompositionFriCounter s1 = PCompositionFriCounter s

and alpha-count-s1: PAlphaCounter s1 = PAlphaCounter s

and query-count-s1: PQueryCounter s1 = PQueryCounter s

by blast

have trace-res:

length f-fl = ceil-log clength ∧
 PTranscript s1 = map snd f-fl @ PTranscript s2 ∧
 PState s2 = foldl concat (PState s1) (map snd f-fl) ∧
 s1 ≤ s2 ∧

```

PTraceFriCounter s2 = PTraceFriCounter s1 + ceil-log clength ∧
PCompositionFriCounter s2 = PCompositionFriCounter s1 ∧
PAlphaCounter s2 = PAlphaCounter s1 ∧
PQueryCounter s2 = PQueryCounter s1 ∧
(∀ i < ceil-log clength.
  fmlookup (HashMap s2)
    (TraceFriChallenge (PTraceFriCounter s1 + i)
      (foldl concat (PState s1) (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i)))
using ntimes-receive-trace-fri-commits-outcome[OF trace-fri] by simp
have t-eq: t = s2
using ret by simp
have ext-s-t: s ≤ t
using ext-s-s1 trace-res t-eq by (meson hash-ext-trans)
have lookups:
  ∀ i < length f-fl.
    fmlookup (HashMap t)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr)
          (take (Suc i) (map snd f-fl)))) =
        Some (fst (f-fl ! i))
using trace-res st-s1 trace-count-s1 t-eq by simp
show ?thesis
using tr-s tr-s1 st-s1 trace-res t-eq ext-s-t trace-count-s1
  comp-count-s1 alpha-count-s1 query-count-s1 lookups
by simp
qed

```

lemma verifier-composition-fri-prefix-outcome:

assumes outcome:

Some ((fr, f-fl, f-final, as, dg, fl), t) ∈
 set-dist (execute verifier-composition-fri-prefix s)

shows

length f-fl = ceil-log clength ∧
 length as = length spec ∧
 length fl = ceil-log (to-nat dg + 1) ∧
 PTranscript s =
 [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
 map snd fl @ PTranscript t ∧
 PState t =
 foldl concat
 (concat
 (foldl concat
 (concat
 (foldl concat (concat (PState s) fr) (map snd f-fl))
 f-final)
 as)
 dg)
 (map snd fl) ∧

```

s ≤ t ∧
PQueryCounter t = PQueryCounter s ∧
(∀ i < length f-fl.
  fmllookup (HashMap t)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr)
        (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))) ∧
(∀ i < length fl.
  fmllookup (HashMap t)
    (CompositionFriChallenge (PCompositionFriCounter s + i)
      (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) (map snd f-fl))
                f-final)
            as)
          dg)
        (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i)))
proof –
from outcome obtain s1 s2 s3 s4 s5 s6 s7 where
  read-fr: Some (fr, s1) ∈ set-dist (execute read s)
  and trace-fri:
    Some (f-fl, s2) ∈
      set-dist (execute
        (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
  and read-f-final: Some (f-final, s3) ∈ set-dist (execute read s2)
  and alpha-out:
    Some (as, s4) ∈
      set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
  and read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
  and degree-assert:
    Some ((), s6) ∈
      set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
  and comp-fri:
    Some (fl, s7) ∈
      set-dist (execute
        (ntimes receive-composition-fri-commits
          (ceil-log (to-nat dg + 1)))) s6)
  and ret:
    Some ((fr, f-fl, f-final, as, dg, fl), t) ∈
      set-dist (execute (return (fr, f-fl, f-final, as, dg, fl)) s7)
  unfolding verifier-composition-fri-prefix-def
  by (auto elim!: set-dist-bindE)
from read-outcome[OF read-fr] obtain rest1 where
  tr-s: PTranscript s = fr # rest1
  and st-s1: PState s1 = concat (PState s) fr

```

```

and tr-s1: PTranscript s1 = rest1
and ext-s-s1: s ≤ s1
and trace-count-s1: PTraceFriCounter s1 = PTraceFriCounter s
and comp-count-s1: PCompositionFriCounter s1 = PCompositionFriCounter s
and alpha-count-s1: PAlphaCounter s1 = PAlphaCounter s
and query-count-s1: PQueryCounter s1 = PQueryCounter s
by blast
have trace-res:
  length f-fl = ceil-log clength ∧
    PTranscript s1 = map snd f-fl @ PTranscript s2 ∧
    PState s2 = foldl concat (PState s1) (map snd f-fl) ∧
    s1 ≤ s2 ∧
    PTraceFriCounter s2 = PTraceFriCounter s1 + ceil-log clength ∧
    PCompositionFriCounter s2 = PCompositionFriCounter s1 ∧
    PAlphaCounter s2 = PAlphaCounter s1 ∧
    PQueryCounter s2 = PQueryCounter s1 ∧
    (∀ i < ceil-log clength.
      fmlookup (HashMap s2)
        (TraceFriChallenge (PTraceFriCounter s1 + i)
          (foldl concat (PState s1) (take (Suc i) (map snd f-fl)))) =
          Some (fst (f-fl ! i)))
    using ntimes-receive-trace-fri-commits-outcome[OF trace-fri] by simp
from read-outcome[OF read-f-final] obtain rest3 where
  tr-s2: PTranscript s2 = f-final # rest3
and st-s3: PState s3 = concat (PState s2) f-final
and tr-s3: PTranscript s3 = rest3
and ext-s2-s3: s2 ≤ s3
and comp-count-s3: PCompositionFriCounter s3 = PCompositionFriCounter s2
and query-count-s3: PQueryCounter s3 = PQueryCounter s2
by blast
have alpha-res:
  length as = length spec ∧
    PTranscript s3 = as @ PTranscript s4 ∧
    PState s4 = foldl concat (PState s3) as ∧
    s3 ≤ s4 ∧
    PCompositionFriCounter s4 = PCompositionFriCounter s3 ∧
    PQueryCounter s4 = PQueryCounter s3
    using mmap-alpha-round-outcome[OF alpha-out] by simp
from read-outcome[OF read-dg] obtain rest5 where
  tr-s4: PTranscript s4 = dg # rest5
and st-s5: PState s5 = concat (PState s4) dg
and tr-s5: PTranscript s5 = rest5
and ext-s4-s5: s4 ≤ s5
and comp-count-s5: PCompositionFriCounter s5 = PCompositionFriCounter s4
and query-count-s5: PQueryCounter s5 = PQueryCounter s4
by blast
have s6-eq: s6 = s5
using degree-assert unfolding assert-def
by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)

```



```

have ext-s5-s6:  $s5 \leq s6$ 
  unfolding s6-eq by (rule hash-ext-refl)
have comp-res:
  length fl = ceil-log (to-nat dg + 1)  $\wedge$ 
  PTranscript s6 = map snd fl @ PTranscript s7  $\wedge$ 
  PState s7 = foldl concat (PState s6) (map snd fl)  $\wedge$ 
   $s6 \leq s7 \wedge$ 
  PCompositionFriCounter s7 =
    PCompositionFriCounter s6 + ceil-log (to-nat dg + 1)  $\wedge$ 
  PQueryCounter s7 = PQueryCounter s6  $\wedge$ 
  ( $\forall i < \text{ceil-log } (\text{to-nat } dg + 1).$ 
    fmlookup (HashMap s7)
      (CompositionFriChallenge (PCompositionFriCounter s6 + i)
        (foldl concat (PState s6) (take (Suc i) (map snd fl)))) =
        Some (fst (fl ! i)))
  using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
  by simp
have t-eq:  $t = s7$ 
  using ret by simp
have ext-s-t:  $s \leq t$ 
  using ext-s-s1 trace-res ext-s2-s3 alpha-res ext-s4-s5 ext-s5-s6
    comp-res t-eq
  by (meson hash-ext-trans)
have ext-s2-t:  $s2 \leq t$ 
  using ext-s2-s3 alpha-res ext-s4-s5 ext-s5-s6 comp-res t-eq
  by (meson hash-ext-trans)
have trace-lookup:
   $\forall i < \text{length } f\text{-}fl.$ 
  fmlookup (HashMap t)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr)
        (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i)))
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{length } f\text{-}fl$ 
  have lookup-s2:
    fmlookup (HashMap s2)
      (TraceFriChallenge (PTraceFriCounter s1 + i)
        (foldl concat (PState s1) (take (Suc i) (map snd f-fl)))) =
        Some (fst (f-fl ! i)))
    using trace-res i-bound by simp
  show fmlookup (HashMap t)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr)
        (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i)))
  using hash-extension-lookup[OF lookup-s2 ext-s2-t] st-s1
    trace-count-s1

```

```

    by simp
qed
have comp-lookup:
  ∀ i < length fl.
    fmlookup (HashMap t)
      (CompositionFriChallenge (PCompositionFriCounter s + i)
        (foldl concat
          (concat
            (foldl concat
              (concat
                (foldl concat (concat (PState s) fr) (map snd f-fl))
                f-final)
              as)
            dg)
          (take (Suc i) (map snd fl)))))) =
    Some (fst (fl ! i))
proof (intro allI impI)
  fix i
  assume i-bound: i < length fl
  have lookup-s7:
    fmlookup (HashMap s7)
      (CompositionFriChallenge (PCompositionFriCounter s6 + i)
        (foldl concat (PState s6) (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
  using comp-res i-bound by simp
show fmlookup (HashMap t)
  (CompositionFriChallenge (PCompositionFriCounter s + i)
    (foldl concat
      (concat
        (foldl concat
          (concat
            (foldl concat (concat (PState s) fr) (map snd f-fl))
            f-final)
          as)
        dg)
      (take (Suc i) (map snd fl)))) =
  Some (fst (fl ! i))
  using lookup-s7 t-eq st-s1 trace-res st-s3 alpha-res st-s5 s6-eq
    comp-count-s1 comp-count-s3 comp-count-s5
  by simp
qed
have query-count-t: PQueryCounter t = PQueryCounter s
  using query-count-s1 trace-res query-count-s3 alpha-res query-count-s5
    s6-eq comp-res t-eq
  by simp
show ?thesis
  using tr-s tr-s1 trace-res tr-s2 tr-s3 alpha-res tr-s4 tr-s5 s6-eq
    comp-res st-s1 st-s3 st-s5 t-eq ext-s-t query-count-t trace-lookup
    comp-lookup

```

by simp
qed

end

end

theory *Soundness-Execution-Extraction*
 imports *Soundness-Execution-Prefix*
begin

Accepted verifier execution transcript extraction and witness lemmas.

context *soundness*
begin

lemma *verify-monad-hash-extends*:

assumes *outcome*:

Some (result, final-state) ∈ set-dist (execute verify-monad s)

shows $s \leq \text{final-state}$

proof –

from *outcome[unfolded verify-monad-composition-fri-decomposition]*

obtain *header prefix-state* **where**

prefix:

Some (header, prefix-state) ∈
 set-dist (execute verifier-composition-fri-prefix s)

and *suffix*:

Some (result, final-state) ∈
 set-dist
 (execute (verifier-after-composition-fri header) prefix-state)

by *(auto elim!: set-dist-bindE)*

obtain *fr f-fl f-final as dg fl* **where** *header-eq*:

header = (fr, f-fl, f-final, as, dg, fl)

by *(cases header) auto*

have *ext-prefix: s ≤ prefix-state*

using *verifier-composition-fri-prefix-outcome*
 [OF prefix[unfolded header-eq]]

by *simp*

from *suffix[unfolded header-eq verifier-after-composition-fri-def]*

obtain *final query-state* **where**

read-final:

Some (final, query-state) ∈ set-dist (execute read prefix-state)

and *query-out*:

Some (result, final-state) ∈
 set-dist
 (execute
 (ntimes
 (verifier-query-round-program fr f-fl f-final as fl final)
 rounds)

```

      query-state)
    by (auto elim!: set-dist-bindE)
  have ext-query: prefix-state  $\leq$  query-state
    using read-outcome[OF read-final] by blast
  have ext-final: query-state  $\leq$  final-state
    using ntimes-verifier-query-rounds-outcome[OF query-out] by blast
  show ?thesis
    using ext-prefix ext-query ext-final by (meson hash-ext-trans)
qed

```

lemma *alpha-header-supported-trace-table-candidate-state-hash-extends:*

```

  assumes
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table final-state
  shows  $s \leq$  final-state
proof -
  from assms obtain result composition-table as query-idxs dg
    composition-fri-roots final rest where
    outcome:
      Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
  unfolding alpha-header-supported-trace-table-candidate-state-def
  by blast
  show ?thesis
    by (rule verify-monad-hash-extends[OF outcome])
qed

```

lemma *alpha-header-supported-pairwise-merkle-bad-extends-initial:*

```

  assumes bad:
    alpha-header-supported-pairwise-merkle-bad s fr f-fri-roots f-final
  obtains trace-table final-state trace-table' final-state'
  where
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table final-state
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table' final-state'
    trace-table  $\neq$  trace-table'
    merkle-hash-value-conflict final-state final-state'  $\vee$ 
    hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
     $s \leq$  final-state
     $s \leq$  final-state'

```

```

proof -
  from bad obtain trace-table final-state trace-table' final-state' where
    candidate:
      alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
      f-final trace-table final-state
  and candidate':
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table' final-state'

```

and *distinct*: $\text{trace-table} \neq \text{trace-table}'$
and *pair-bad*:
 $\text{merkle-hash-value-conflict } \text{final-state } \text{final-state}' \vee$
 $\text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge } \text{final-state } \text{final-state}')$
unfolding *alpha-header-supported-pairwise-merkle-bad-def* **by** *blast*
have *ext*: $s \leq \text{final-state}$
by (rule *alpha-header-supported-trace-table-candidate-state-hash-extends*
 $[\text{OF candidate}]$)
have *ext'*: $s \leq \text{final-state}'$
by (rule *alpha-header-supported-trace-table-candidate-state-hash-extends*
 $[\text{OF candidate}]$)
show *?thesis*
by (rule *that*[*OF candidate candidate' distinct pair-bad ext ext'*])
qed

definition *alpha-header-supported-pairwise-coupling-bad*

$:: (\text{'f}, \text{'a}) \text{protocol-channel-scheme} \Rightarrow$
 $\text{'f} \Rightarrow \text{'f list} \Rightarrow \text{'f} \Rightarrow \text{bool}$

where

$\text{alpha-header-supported-pairwise-coupling-bad } s \text{ fr } \text{f-fri-roots } \text{f-final}$
 $\longleftrightarrow$
 $(\exists \text{trace-table } \text{final-state } \text{trace-table}' \text{ final-state}'.$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } \text{f-fri-roots}$
 $\text{f-final } \text{trace-table } \text{final-state} \wedge$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } \text{f-fri-roots}$
 $\text{f-final } \text{trace-table}' \text{ final-state}' \wedge$
 $\text{trace-table} \neq \text{trace-table}' \wedge$
 $\text{merkle-hash-pairwise-coupling-bad } \text{final-state } \text{final-state}')$

definition *alpha-supported-pairwise-coupling-bad*

$:: (\text{'f}, \text{'a}) \text{protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{alpha-supported-pairwise-coupling-bad } s \longleftrightarrow$
 $(\exists \text{fr } \text{f-fri-roots } \text{f-final}.$
 $\text{alpha-header-supported-pairwise-coupling-bad } s \text{ fr } \text{f-fri-roots}$
 $\text{f-final})$

lemma *alpha-header-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad*:

assumes *bad*:

$\text{alpha-header-supported-pairwise-merkle-bad } s \text{ fr } \text{f-fri-roots } \text{f-final}$

shows

$(\exists \text{trace-table } \text{final-state } \text{trace-table}' \text{ final-state}'.$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } \text{f-fri-roots}$
 $\text{f-final } \text{trace-table } \text{final-state} \wedge$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } \text{f-fri-roots}$
 $\text{f-final } \text{trace-table}' \text{ final-state}' \wedge$
 $\text{trace-table} \neq \text{trace-table}' \wedge$
 $(\text{hash-map-output-collision } \text{final-state} \vee$

$\text{hash-map-output-collision } \text{final-state}') \vee$
 $\text{alpha-header-supported-pairwise-coupling-bad } s \text{ fr } f\text{-fri-roots } f\text{-final}$
proof –
from *bad* **obtain** $\text{trace-table } \text{final-state } \text{trace-table}' \text{ final-state}'$
where *candidate*:
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final } \text{trace-table } \text{final-state}$
and *candidate'*:
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final } \text{trace-table}' \text{ final-state}'$
and *distinct*: $\text{trace-table} \neq \text{trace-table}'$
and *pair-bad*:
 $\text{merkle-hash-value-conflict } \text{final-state } \text{final-state}' \vee$
 $\text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge } \text{final-state } \text{final-state}')$
unfolding $\text{alpha-header-supported-pairwise-merkle-bad-def}$ **by** *blast*
have *decomp*:
 $\text{hash-map-output-collision } \text{final-state} \vee$
 $\text{hash-map-output-collision } \text{final-state}' \vee$
 $\text{merkle-hash-pairwise-coupling-bad } \text{final-state } \text{final-state}'$
by (*rule merkle-hash-pairwise-bad-imp-local-collision-or-coupling-bad*
 $[\text{OF } \text{pair-bad}]$)
show *?thesis*
using *candidate candidate' distinct decomp*
unfolding $\text{alpha-header-supported-pairwise-coupling-bad-def}$ **by** *blast*
qed

lemma $\text{alpha-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad}$:

assumes *bad*: $\text{alpha-supported-pairwise-merkle-bad } s$

shows

$(\exists \text{ fr } f\text{-fri-roots } f\text{-final } \text{trace-table } \text{final-state } \text{trace-table}'$
 $\text{final-state}')$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final } \text{trace-table } \text{final-state} \wedge$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final } \text{trace-table}' \text{ final-state}' \wedge$
 $\text{trace-table} \neq \text{trace-table}' \wedge$
 $(\text{hash-map-output-collision } \text{final-state} \vee$
 $\text{hash-map-output-collision } \text{final-state}') \vee$
 $\text{alpha-supported-pairwise-coupling-bad } s$

proof –

from *bad* **obtain** $\text{fr } f\text{-fri-roots } f\text{-final}$ **where** *header-bad*:

$\text{alpha-header-supported-pairwise-merkle-bad } s \text{ fr } f\text{-fri-roots } f\text{-final}$

unfolding $\text{alpha-supported-pairwise-merkle-bad-def}$ **by** *blast*

from

$\text{alpha-header-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad}$
 $[\text{OF } \text{header-bad}]$

show *?thesis*

unfolding $\text{alpha-supported-pairwise-coupling-bad-def}$ **by** *blast*

qed

lemma *alpha-supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad:*

assumes *bad: alpha-supported-pairwise-merkle-bad s*

shows

supported-hash-output-collision-possible s $\vee$

alpha-supported-pairwise-coupling-bad s

proof –

have *decomp:*

$(\exists \text{fr } f\text{-fri-roots } f\text{-final trace-table final-state trace-table'}$
 $\text{final-state'.$

alpha-header-supported-trace-table-candidate-state s fr f-fri-roots

f-final trace-table final-state $\wedge$

alpha-header-supported-trace-table-candidate-state s fr f-fri-roots

f-final trace-table' final-state' $\wedge$

trace-table $\neq$ *trace-table'* $\wedge$

$(\text{hash-map-output-collision final-state} \vee$

$\text{hash-map-output-collision final-state'}) \vee$

alpha-supported-pairwise-coupling-bad s

by (*rule*

alpha-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad

[*OF bad*])

then show *?thesis*

proof

assume *local:*

$\exists \text{fr } f\text{-fri-roots } f\text{-final trace-table final-state trace-table'}$
 $\text{final-state'.$

alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table final-state $\wedge$

alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table' final-state' $\wedge$

trace-table $\neq$ *trace-table'* $\wedge$

$(\text{hash-map-output-collision final-state} \vee$

$\text{hash-map-output-collision final-state'})$

then obtain *fr f-fri-roots f-final trace-table final-state trace-table'*
final-state' **where**

candidate:

alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table final-state

and *candidate':*

alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table' final-state'

and *collision:*

hash-map-output-collision final-state $\vee$

hash-map-output-collision final-state'

by *blast*

from *collision* **show** *?thesis*

proof

assume *collision: hash-map-output-collision final-state*

```

from candidate obtain result composition-table as query-idxs dg
  composition-fri-roots final rest where
  outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
and bound:
  accepted-with-bound-tables s (Some (result, final-state))
  trace-table composition-table as query-idxs
unfolding alpha-header-supported-trace-table-candidate-state-def
by blast
have supported-hash-output-collision-possible s
by (rule
  accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible
  [OF outcome bound collision])
then show ?thesis by simp
next
assume collision': hash-map-output-collision final-state'
from candidate' obtain result' composition-table' as' query-idxs' dg'
  composition-fri-roots' final' rest' where
  outcome':
    Some (result', final-state') ∈ set-dist (execute verify-monad s)
and bound':
  accepted-with-bound-tables s (Some (result', final-state'))
  trace-table' composition-table' as' query-idxs'
unfolding alpha-header-supported-trace-table-candidate-state-def
by blast
have supported-hash-output-collision-possible s
by (rule
  accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible
  [OF outcome' bound' collision'])
then show ?thesis by simp
qed
next
assume alpha-supported-pairwise-coupling-bad s
then show ?thesis by simp
qed
qed

lemma verifier-composition-fri-prefix-preserves-query-future-fresh:
assumes future: query-future-fresh s
and outcome:
  Some (header, t) ∈
  set-dist (execute verifier-composition-fri-prefix s)
shows query-future-fresh t
proof –
obtain fr f-fl f-final as dg fl where header-eq:
  header = (fr, f-fl, f-final, as, dg, fl)
by (cases header) auto
from outcome[unfolded header-eq] obtain s1 s2 s3 s4 s5 s6 where
  read-fr: Some (fr, s1) ∈ set-dist (execute read s)

```



```

and trace-fri:
  Some (f-fl, s2) ∈
    set-dist (execute
      (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
and read-f-final:
  Some (f-final, s3) ∈ set-dist (execute read s2)
and alpha-out:
  Some (as, s4) ∈
    set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
and read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
and degree-assert:
  Some ((), s6) ∈
    set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
and comp-fri:
  Some (fl, t) ∈
    set-dist (execute
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1))) s6)
unfolding verifier-composition-fri-prefix-def
by (auto elim!: set-dist-bindE)
have future-s1: query-future-fresh s1
by (rule read-preserves-query-future-fresh[OF future read-fr])
have future-s2: query-future-fresh s2
by (rule ntimes-receive-trace-fri-commits-preserves-query-future-fresh
  [OF trace-fri future-s1])
have future-s3: query-future-fresh s3
by (rule read-preserves-query-future-fresh[OF future-s2 read-f-final])
have future-s4: query-future-fresh s4
by (rule mmap-alpha-round-preserves-query-future-fresh
  [OF future-s3 alpha-out])
have future-s5: query-future-fresh s5
by (rule read-preserves-query-future-fresh[OF future-s4 read-dg])
have s6-eq: s6 = s5
using degree-assert unfolding assert-def
by (cases to-nat dg ≤ maxDegree)
  (auto simp: throw-no-outcome)
have future-s6: query-future-fresh s6
using future-s5 unfolding s6-eq .
show ?thesis
by (rule ntimes-receive-composition-fri-commits-preserves-query-future-fresh
  [OF comp-fri future-s6])
qed

lemma verify-monad-header-extraction:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
obtains fr f-fl f-final as dg fl final query-state
where

```

```

verifier-header-transcript s fr (map snd f-fl) f-final as dg
  (map snd fl) final (PTranscript query-state)
and PState query-state =
  verifier-header-state s fr (map snd f-fl) f-final as dg (map snd fl) final
and Some (result, final-state) ∈
  set-dist (execute
    (ntimes (verifier-query-round-program fr f-fl f-final as fl final) rounds)
    query-state)
and PQueryCounter query-state = PQueryCounter s
and length f-fl = ceil-log clength
and length as = length spec
and length fl = ceil-log (to-nat dg + 1)
proof –
  let ?alpha =
    do {
      a0 ← receive-alpha-challenge;
      let a0' = a0;
      a1 ← read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    }
from outcome obtain fr s1 f-fl s2 f-final s3 as s4 dg s5 s6 fl s7 final query-state
where read-fr: Some (fr, s1) ∈ set-dist (execute read s)
and trace-fri:
  Some (f-fl, s2) ∈
    set-dist (execute
      (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
and read-f-final: Some (f-final, s3) ∈ set-dist (execute read s2)
and alpha-out:
  Some (as, s4) ∈
    set-dist (execute (mmap (replicate (length spec) ?alpha)) s3)
and read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
and degree-assert:
  Some ((), s6) ∈ set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
and comp-fri:
  Some (fl, s7) ∈
    set-dist (execute
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1))) s6)
and read-final: Some (final, query-state) ∈ set-dist (execute read s7)
and query-out:
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) rounds)
      query-state)
unfolding verify-monad-def verifier-query-round-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
from read-outcome[OF read-fr] obtain rest1 where

```

```



```

```

and st-query: PState query-state = concat (PState s7) final
and tr-query: PTranscript query-state = rest-query
and query-count-query: PQueryCounter query-state = PQueryCounter s7
by blast
have header-tr:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
using tr-s tr-s1 trace-fri-res tr-s2 tr-s3 alpha-res tr-s4 tr-s5
  s6-eq comp-fri-res tr-s7 tr-query
unfolding verifier-header-transcript-def verifier-header-messages-def
by simp
have header-state:
  PState query-state =
    verifier-header-state s fr (map snd f-fl) f-final as dg (map snd fl) final
using st-s1 trace-fri-res st-s3 alpha-res st-s5 s6-eq comp-fri-res st-query
unfolding verifier-header-state-def verifier-header-messages-def
by simp
have query-count-header:
  PQueryCounter query-state = PQueryCounter s
using query-count-s1 query-count-s2 query-count-s3 query-count-s4
  query-count-s5 s6-eq query-count-s7 query-count-query
by simp
show ?thesis
by (rule that[OF header-tr header-state query-out query-count-header])
  (use trace-fri-res alpha-res comp-fri-res in simp-all)
qed

```

```

lemma verify-monad-header-query-extraction:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  obtains fr f-fl f-final as dg fl final query-state
  where
    verifier-header-transcript s fr (map snd f-fl) f-final as dg
      (map snd fl) final (PTranscript query-state)
    and Some (result, final-state) ∈
      set-dist (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) rounds)
        query-state)
proof (rule verify-monad-header-extraction[OF outcome])
fix fr f-fl f-final as dg fl final query-state
assume header:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
and query-out:
  Some (result, final-state) ∈
    set-dist
    (execute

```

```

      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      query-state)
show ?thesis
  by (rule that[OF header query-out])
qed

lemma verify-monad-header-random-oracle-replay:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  obtains fr f-fl f-final as dg fl final query-state
  where
    verifier-header-transcript s fr (map snd f-fl) f-final as dg
      (map snd fl) final (PTranscript query-state)
  and PState query-state =
    verifier-header-state s fr (map snd f-fl) f-final as dg (map snd fl) final
  and Some (result, final-state) ∈
    set-dist (execute
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) rounds)
      query-state)
  and PQueryCounter query-state = PQueryCounter s
  and s ≤ query-state
  and  $\bigwedge i. i < \text{length } f\text{-fl} \implies$ 
    fmlookup (HashMap query-state)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr) (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i))
  and  $\bigwedge i. i < \text{length } as \implies$ 
    fmlookup (HashMap query-state)
      (AlphaChallenge (PAlphaCounter s + i) (foldl concat
        (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
        (take i as))) =
      Some (as ! i)
  and  $\bigwedge i. i < \text{length } fl \implies$ 
    fmlookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
        (concat
          (foldl concat
            (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
            as)
          dg)
        (take (Suc i) (map snd fl)))) =
      Some (fst (fl ! i))
  and to-nat dg ≤ maxDegree
proof -
  let ?alpha =
  do {

```

```

    a0 ← receive-alpha-challenge;
    let a0' = a0;
    a1 ← read;
    let a1' = a1;
    assert (a0' = a1');
    return a1'
}
from outcome obtain fr s1 f-fl s2 f-final s3 as s4 dg s5 s6 fl s7 final query-state
where read-fr: Some (fr, s1) ∈ set-dist (execute read s)
and trace-fri:
    Some (f-fl, s2) ∈
    set-dist (execute
        (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
and read-f-final: Some (f-final, s3) ∈ set-dist (execute read s2)
and alpha-out:
    Some (as, s4) ∈
    set-dist (execute (mmap (replicate (length spec) ?alpha)) s3)
and read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
and degree-assert:
    Some ((), s6) ∈ set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
and comp-fri:
    Some (fl, s7) ∈
    set-dist (execute
        (ntimes receive-composition-fri-commits
            (ceil-log (to-nat dg + 1))) s6)
and read-final: Some (final, query-state) ∈ set-dist (execute read s7)
and query-out:
    Some (result, final-state) ∈
    set-dist (execute
        (ntimes (verifier-query-round-program fr f-fl f-final as fl final) rounds)
        query-state)
unfolding verify-monad-def verifier-query-round-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
from read-outcome[OF read-fr] obtain rest1 where
    tr-s: PTranscript s = fr # rest1
and st-s1: PState s1 = concat (PState s) fr
and tr-s1: PTranscript s1 = rest1
and ext-s-s1: s ≤ s1
and trace-count-s1: PTraceFriCounter s1 = PTraceFriCounter s
and comp-count-s1: PCompositionFriCounter s1 = PCompositionFriCounter s
and alpha-count-s1: PAlphaCounter s1 = PAlphaCounter s
and query-count-s1: PQueryCounter s1 = PQueryCounter s
by blast
have trace-fri-res:
    length f-fl = ceil-log clength ∧
    PTranscript s1 = map snd f-fl @ PTranscript s2 ∧
    PState s2 = foldl concat (PState s1) (map snd f-fl) ∧
    s1 ≤ s2 ∧
    PTraceFriCounter s2 = PTraceFriCounter s1 + ceil-log clength ∧

```

```

    PCompositionFriCounter s2 = PCompositionFriCounter s1 ∧
    PAlphaCounter s2 = PAlphaCounter s1 ∧
    PQueryCounter s2 = PQueryCounter s1 ∧
    (∀ i < ceil-log clength.
      fmllookup (HashMap s2)
        (TraceFriChallenge (PTraceFriCounter s1 + i)
          (foldl concat (PState s1) (take (Suc i) (map snd f-fl)))) =
        Some (fst (f-fl ! i)))
  using ntimes-receive-trace-fri-commits-outcome[OF trace-fri] by simp
from read-outcome[OF read-f-final] obtain rest3 where
  tr-s2: PTranscript s2 = f-final # rest3
  and st-s3: PState s3 = concat (PState s2) f-final
  and tr-s3: PTranscript s3 = rest3
  and ext-s2-s3: s2 ≤ s3
  and trace-count-s3: PTraceFriCounter s3 = PTraceFriCounter s2
  and comp-count-s3: PCompositionFriCounter s3 = PCompositionFriCounter s2
  and alpha-count-s3: PAlphaCounter s3 = PAlphaCounter s2
  and query-count-s3: PQueryCounter s3 = PQueryCounter s2
  by blast
have alpha-out':
  Some (as, s4) ∈
    set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
  using alpha-out by (simp add: alpha-round-def)
have alpha-res:
  length as = length spec ∧
  PTranscript s3 = as @ PTranscript s4 ∧
  PState s4 = foldl concat (PState s3) as ∧
  s3 ≤ s4 ∧
  PTraceFriCounter s4 = PTraceFriCounter s3 ∧
  PCompositionFriCounter s4 = PCompositionFriCounter s3 ∧
  PAlphaCounter s4 = PAlphaCounter s3 + length spec ∧
  PQueryCounter s4 = PQueryCounter s3 ∧
  (∀ i < length spec.
    fmllookup (HashMap s4)
      (AlphaChallenge (PAlphaCounter s3 + i)
        (foldl concat (PState s3) (take i as)))) =
    Some (as ! i))
  using mmap-alpha-round-outcome[OF alpha-out'] by simp
from read-outcome[OF read-dg] obtain rest5 where
  tr-s4: PTranscript s4 = dg # rest5
  and st-s5: PState s5 = concat (PState s4) dg
  and tr-s5: PTranscript s5 = rest5
  and ext-s4-s5: s4 ≤ s5
  and trace-count-s5: PTraceFriCounter s5 = PTraceFriCounter s4
  and comp-count-s5: PCompositionFriCounter s5 = PCompositionFriCounter s4
  and alpha-count-s5: PAlphaCounter s5 = PAlphaCounter s4
  and query-count-s5: PQueryCounter s5 = PQueryCounter s4
  by blast
have s6-eq: s6 = s5

```

```

using degree-assert unfolding assert-def
by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
have degree-bound: to-nat dg ≤ maxDegree
using degree-assert unfolding assert-def
by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
have ext-s5-s6: s5 ≤ s6
unfolding s6-eq by (rule hash-ext-refl)
have comp-fri-res:
  length fl = ceil-log (to-nat dg + 1) ∧
  PTranscript s6 = map snd fl @ PTranscript s7 ∧
  PState s7 = foldl concat (PState s6) (map snd fl) ∧
  s6 ≤ s7 ∧
  PTraceFriCounter s7 = PTraceFriCounter s6 ∧
  PCompositionFriCounter s7 = PCompositionFriCounter s6 + ceil-log (to-nat dg + 1) ∧
  PAlphaCounter s7 = PAlphaCounter s6 ∧
  PQueryCounter s7 = PQueryCounter s6 ∧
  (∀ i < ceil-log (to-nat dg + 1).
    fmllookup (HashMap s7)
      (CompositionFriChallenge (PCompositionFriCounter s6 + i)
        (foldl concat (PState s6) (take (Suc i) (map snd fl)))) =
      Some (fst (fl ! i)))
using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
by simp
from read-outcome[OF read-final] obtain rest-query where
  tr-s7: PTranscript s7 = final # rest-query
and st-query: PState query-state = concat (PState s7) final
and tr-query: PTranscript query-state = rest-query
and ext-s7-query: s7 ≤ query-state
and query-count-query: PQueryCounter query-state = PQueryCounter s7
by blast
have ext-s2-query: s2 ≤ query-state
using ext-s2-s3 alpha-res ext-s4-s5 ext-s5-s6 comp-fri-res ext-s7-query
by (meson hash-ext-trans)
have ext-s4-query: s4 ≤ query-state
using ext-s4-s5 ext-s5-s6 comp-fri-res ext-s7-query
by (meson hash-ext-trans)
have ext-s7-query': s7 ≤ query-state
using ext-s7-query .
have ext-s1-s2: s1 ≤ s2
using trace-fri-res by simp
have ext-s3-s4: s3 ≤ s4
using alpha-res by simp
have ext-s6-s7: s6 ≤ s7
using comp-fri-res by simp
have ext-s-s2: s ≤ s2
by (rule hash-ext-trans[OF ext-s-s1 ext-s1-s2])
have ext-s-s3: s ≤ s3
by (rule hash-ext-trans[OF ext-s-s2 ext-s2-s3])
have ext-s-s4: s ≤ s4

```



```

  by (rule hash-ext-trans[OF ext-s-s3 ext-s3-s4])
have ext-s-s5:  $s \leq s5$ 
  by (rule hash-ext-trans[OF ext-s-s4 ext-s4-s5])
have ext-s-s6:  $s \leq s6$ 
  by (rule hash-ext-trans[OF ext-s-s5 ext-s5-s6])
have ext-s-s7:  $s \leq s7$ 
  by (rule hash-ext-trans[OF ext-s-s6 ext-s6-s7])
have ext-s-query:  $s \leq \text{query-state}$ 
  by (rule hash-ext-trans[OF ext-s-s7 ext-s7-query])
have header-tr:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
  using tr-s tr-s1 trace-fri-res tr-s2 tr-s3 alpha-res tr-s4 tr-s5
    s6-eq comp-fri-res tr-s7 tr-query
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
have header-state:
  PState query-state =
    verifier-header-state s fr (map snd f-fl) f-final as dg (map snd fl) final
  using st-s1 trace-fri-res st-s3 alpha-res st-s5 s6-eq comp-fri-res st-query
  unfolding verifier-header-state-def verifier-header-messages-def
  by simp
have trace-lookup:
 $\bigwedge i. i < \text{length } f\text{-fl} \implies$ 
  fmlookup (HashMap query-state)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr) (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
proof -
  fix i
  assume i-bound:  $i < \text{length } f\text{-fl}$ 
  have lookup-s2:
    fmlookup (HashMap s2)
      (TraceFriChallenge (PTraceFriCounter s1 + i)
        (foldl concat (PState s1) (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
  using trace-fri-res i-bound by simp
  show fmlookup (HashMap query-state)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr) (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
  using hash-extension-lookup[OF lookup-s2 ext-s2-query] st-s1
    trace-count-s1
  by simp
qed
have alpha-lookup:
 $\bigwedge i. i < \text{length } as \implies$ 
  fmlookup (HashMap query-state)
    (AlphaChallenge (PAlphaCounter s + i) (foldl concat

```

```

      (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
      (take i as))) =
    Some (as ! i)
proof -
  fix i
  assume i-bound: i < length as
  have lookup-s4:
    fmlookup (HashMap s4)
      (AlphaChallenge (PAlphaCounter s3 + i)
        (foldl concat (PState s3) (take i as))) =
      Some (as ! i)
  using alpha-res i-bound by simp
  show fmlookup (HashMap query-state)
    (AlphaChallenge (PAlphaCounter s + i) (foldl concat
      (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
      (take i as))) =
    Some (as ! i)
  using hash-extension-lookup[OF lookup-s4 ext-s4-query]
    st-s1 trace-fri-res st-s3 alpha-count-s1 alpha-count-s3
  by simp
qed
have comp-lookup:
   $\bigwedge i. i < \text{length } fl \implies$ 
  fmlookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
      (concat
        (foldl concat
          (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
          as)
        dg)
      (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
proof -
  fix i
  assume i-bound: i < length fl
  have lookup-s7:
    fmlookup (HashMap s7)
      (CompositionFriChallenge (PCompositionFriCounter s6 + i)
        (foldl concat (PState s6) (take (Suc i) (map snd fl)))) =
      Some (fst (fl ! i))
  using comp-fri-res i-bound by simp
  show fmlookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
      (concat
        (foldl concat
          (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
          as)
        dg)
      (take (Suc i) (map snd fl)))) =

```

```

    Some (fst (fl ! i))
  using hash-extension-lookup[OF lookup-s7 ext-s7-query]
    st-s1 trace-fri-res st-s3 alpha-res st-s5 s6-eq
    comp-count-s1 comp-count-s3 comp-count-s5
  by simp
qed
have query-count-header:
  PQueryCounter query-state = PQueryCounter s
  using query-count-s1 trace-fri-res query-count-s3 alpha-res query-count-s5
    s6-eq comp-fri-res query-count-query
  by simp
show ?thesis
  by (rule that[OF header-tr header-state query-out query-count-header
    ext-s-query trace-lookup alpha-lookup comp-lookup degree-bound])
qed

lemma verify-monad-random-oracle-key-extraction:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  obtains fr :: 'f
    and f-fl :: ('f × 'f) list
    and f-final :: 'f
    and as :: 'f list
    and dg :: 'f
    and fl :: ('f × 'f) list
    and final :: 'f
    and query-state :: ('f, 'a) protocol-channel-scheme
    and raw-idxs :: 'f list
    and query-idxs :: nat list
    and query-chunks :: 'f list list
  where
    verifier-header-transcript s fr (map snd f-fl) f-final as dg
      (map snd fl) final (PTranscript query-state)
    and PState query-state =
      verifier-header-state s fr (map snd f-fl) f-final as dg (map snd fl) final
    and length raw-idxs = rounds
    and query-idxs = map (λraw. index (to-nat raw)) raw-idxs
    and length query-chunks = rounds
    and PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state
    and ∧i. i < length f-fl ⇒
      fmlookup (HashMap query-state)
        (TraceFriChallenge
          (PTraceFriCounter s + i)
          (foldl concat (concat (PState s) fr)
            (take (Suc i) (map snd f-fl)))) =
        Some (fst (f-fl ! i))
    and ∧i. i < length as ⇒

```

```

    fmlookup (HashMap query-state)
      (AlphaChallenge (PAlphaCounter s + i) (foldl concat
        (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
        (take i as))) =
      Some (as ! i)
  and  $\bigwedge i. i < \text{length } fl \implies$ 
    fmlookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
        (concat
          (foldl concat
            (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
            as)
          dg)
        (take (Suc i) (map snd fl)))) =
      Some (fst (fl ! i))
  and  $\bigwedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
  and  $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge
        (PQueryCounter query-state + i) (state-after-query-chunks (PState query-state) query-chunks
          i)) =
      Some (raw-idxs ! i)
  and  $\bigwedge idx. idx \in \text{set query-idxs} \implies idx < \text{clength} * \text{scale}$ 
proof –
from verify-monad-header-random-oracle-replay[OF outcome]
obtain fr f-fl f-final as dg fl final query-state where
  header:
    verifier-header-transcript s fr (map snd f-fl) f-final as dg
      (map snd fl) final (PTranscript query-state)
  and header-state:
    PState query-state =
      verifier-header-state s fr (map snd f-fl) f-final as dg
        (map snd fl) final
  and query-out:
    Some (result, final-state)  $\in$ 
      set-dist (execute
        (ntimes (verifier-query-round-program fr f-fl f-final as fl final) rounds)
        query-state)
  and query-count-header:
    PQueryCounter query-state = PQueryCounter s
  and ext-s-query-header:  $s \leq \text{query-state}$ 
  and trace-lookup:
     $\bigwedge i. i < \text{length } f-fl \implies$ 
      fmlookup (HashMap query-state)
        (TraceFriChallenge
          (PTraceFriCounter s + i) (foldl concat (concat (PState s) fr)
            (take (Suc i) (map snd f-fl)))) =

```

```

    Some (fst (f-fl ! i))
and alpha-lookup:
   $\bigwedge i. i < \text{length } as \implies$ 
    fmlookup (HashMap query-state)
      (AlphaChallenge (PAlphaCounter s + i) (foldl concat
        (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
        (take i as))) =
    Some (as ! i)
and comp-lookup:
   $\bigwedge i. i < \text{length } fl \implies$ 
    fmlookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) (map snd f-fl))
              f-final)
            as)
          dg)
        (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
and degree-bound: to-nat dg  $\leq$  maxDegree
by blast
from ntimes-verifier-query-rounds-outcome[OF query-out]
obtain raw-idxs query-idxs query-chunks where
  len-raw: length raw-idxs = rounds
and query-idxs-def: query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and len-chunks: length query-chunks = rounds
and tr-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and round-chunks:
   $\forall i < \text{rounds}.$ 
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
and query-lookup:
   $\forall i < \text{rounds}.$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge
        (PQueryCounter query-state + i) (state-after-query-chunks (PState query-state)
          query-chunks i)) =
    Some (raw-idxs ! i)
and idx-bounds:  $\forall idx \in \text{set query-idxs}. idx < \text{clength} * \text{scale}$ 
by blast
show ?thesis
proof (rule that[of fr f-fl f-final as dg fl final query-state
  raw-idxs query-idxs query-chunks])
show verifier-header-transcript s fr (map snd f-fl) f-final as dg
  (map snd fl) final (PTranscript query-state)

```

```

    by (rule header)
next
show PState query-state =
  verifier-header-state s fr (map snd f-fl) f-final as dg
  (map snd fl) final
  by (rule header-state)
next
show length raw-idxs = rounds
  by (rule len-raw)
next
show query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  by (rule query-idxs-def)
next
show length query-chunks = rounds
  by (rule len-chunks)
next
show PTranscript query-state =
  List.concat query-chunks @ PTranscript final-state
  by (rule tr-query)
next
fix i
assume i < length f-fl
then show fmlookup (HashMap query-state)
  (TraceFriChallenge
    (PTraceFriCounter s + i)
    (foldl concat (concat (PState s) fr)
      (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
  by (rule trace-lookup)
next
fix i
assume i < length as
then show fmlookup (HashMap query-state)
  (AlphaChallenge (PAlphaCounter s + i) (foldl concat
    (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
    (take i as))) =
    Some (as ! i)
  by (rule alpha-lookup)
next
fix i
assume i < length fl
then show fmlookup (HashMap query-state)
  (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
    (concat
      (foldl concat
        (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
        as)
      dg)
    (take (Suc i) (map snd fl)))) =

```

```

      Some (fst (fl ! i))
    by (rule comp-lookup)
next
  fix i
  assume i < rounds
  then show verifier-query-round-chunk (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
    using round-chunks by blast
next
  fix i
  assume i < rounds
  then show fmlookup (HashMap final-state)
    (QueryIndexChallenge
      (PQueryCounter query-state + i) (state-after-query-chunks (PState query-state) query-chunks
i)) =
    Some (raw-idxs ! i)
    using query-lookup by blast
next
  fix idx
  assume idx ∈ set query-idxs
  then show idx < clength * scale
    using idx-bounds by blast
qed
qed

```

lemma *verify-monad-accepted-transcript-shape:*

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes *outcome:*

$\text{Some (result, final-state)} \in \text{set-dist (execute verify-monad s)}$

shows $\exists \text{ alphas query-idxs.}$

$\text{accepted-transcript-shape s (Some (result, final-state)) alphas query-idxs}$

proof –

from *verify-monad-header-extraction[OF outcome]*

obtain $\text{fr f-fl f-final as dg fl final query-state}$ **where**

header:

$\text{verifier-header-transcript s fr (map snd f-fl) f-final as dg}$

$(\text{map snd fl}) \text{ final (PTranscript query-state)}$

and *header-state:*

$\text{PState query-state} =$

$\text{verifier-header-state s fr (map snd f-fl) f-final as dg (map snd fl) final}$

and *query-out:*

$\text{Some (result, final-state)} \in$

set-dist (execute

$(\text{ntimes (verifier-query-round-program fr f-fl f-final as fl final) rounds})$
 $\text{query-state})$

and *query-count-header:*

$\text{PQueryCounter query-state} = \text{PQueryCounter s}$

and *len-f-fl:* $\text{length f-fl} = \text{ceil-log clength}$

and *len-as:* $\text{length as} = \text{length spec}$

```

and len-fl: length fl = ceil-log (to-nat dg + 1)
by blast
from ntimes-verifier-query-rounds-outcome[OF query-out]
obtain raw-idxs query-idxs query-chunks where
  len-raw: length raw-idxs = rounds
  and query-idxs-def: query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
  and len-chunks: length query-chunks = rounds
  and tr-query:
    PTranscript query-state = List.concat query-chunks @ PTranscript final-state
  and round-chunks:
     $\forall i < \text{rounds}.$ 
      verifier-query-round-chunk (query-idxs ! i)
        (map snd f-fl) (map snd fl) (query-chunks ! i)
  and lookups:
     $\forall i < \text{rounds}.$ 
      fmllookup (HashMap final-state)
        (QueryIndexChallenge (PQueryCounter query-state + i) (state-after-query-chunks (PState
query-state) query-chunks i)) =
        Some (raw-idxs ! i)
  and idx-bounds:  $\forall \text{idx} \in \text{set query-idxs}. \text{idx} < \text{clength} * \text{scale}$ 
by auto
have derived:
  verifier-query-indices-derived s (Some (result, final-state))
    (verifier-header-state s fr (map snd f-fl) f-final as dg (map snd fl) final)
    (PTranscript query-state) (map snd f-fl) (map snd fl) query-idxs
proof –
  have tr-query':
    List.concat query-chunks @ PTranscript final-state = PTranscript query-state
  using tr-query by simp
  have lookups':
     $\forall i < \text{rounds}.$ 
      fmllookup (HashMap final-state)
        (QueryIndexChallenge (PQueryCounter s + i) (state-after-query-chunks
(verifier-header-state s fr (map snd f-fl) f-final as dg (map snd fl) final)
query-chunks i)) =
        Some (raw-idxs ! i)
  using lookups header-state query-count-header by simp
show ?thesis
unfolding verifier-query-indices-derived-def
apply (rule exI[where x=result])
apply (rule exI[where x=final-state])
apply (rule exI[where x=raw-idxs])
apply (rule exI[where x=query-chunks])
apply (rule exI[where x=PTranscript final-state])
apply (intro conjI)
  apply simp
  apply (rule len-raw)
  apply (rule query-idxs-def)
  apply (rule len-chunks)

```



```

    apply (rule tr-query')
    apply (rule round-chunks)
    apply (rule lookups')
    apply (rule idx-bounds)
  done
qed
have shape:
  accepted-transcript-shape s (Some (result, final-state)) as query-idxs
  unfolding accepted-transcript-shape-def
  apply (rule exI[where x=result])
  apply (rule exI[where x=final-state])
  apply (rule exI[where x=fr])
  apply (rule exI[where x=map snd f-fl])
  apply (rule exI[where x=f-final])
  apply (rule exI[where x=dg])
  apply (rule exI[where x=map snd fl])
  apply (rule exI[where x=final])
  apply (rule exI[where x=PTranscript query-state])
  apply (intro conjI)
    apply simp
    apply (rule header)
    apply (rule derived)
  done
show ?thesis
  using shape by blast
qed

```

```

lemma verify-monad-supplied-header-transcript-shape:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and header0:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  obtains query-idxs raw-idxs query-chunks where
    accepted-transcript-shape s (Some (result, final-state)) as query-idxs
    length raw-idxs = rounds
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
    length query-chunks = rounds
    List.concat query-chunks @ PTranscript final-state = rest
    ∧ i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
      f-fri-roots composition-fri-roots (query-chunks ! i)
    ∧ i. i < rounds ⇒
      fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter s + i)
      (state-after-query-chunks
      (verifier-header-state s fr f-fri-roots f-final as dg
      composition-fri-roots final)

```

```

      query-chunks i)) =
      Some (raw-idxs ! i)
proof –
from verify-monad-header-extraction[OF outcome]
obtain fr' f-fl f-final' as' dg' fl final' query-state where
  header:
    verifier-header-transcript s fr' (map snd f-fl) f-final' as' dg'
      (map snd fl) final' (PTranscript query-state)
and header-state:
    PState query-state =
      verifier-header-state s fr' (map snd f-fl) f-final' as' dg'
        (map snd fl) final'
and query-out:
    Some (result, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr' f-fl f-final' as' fl
              final') rounds)
            query-state)
and query-count-header:
    PQueryCounter query-state = PQueryCounter s
by metis
have eqs:
  fr' = fr ∧
    map snd f-fl = f-fri-roots ∧
    f-final' = f-final ∧
    as' = as ∧
    dg' = dg ∧
    map snd fl = composition-fri-roots ∧
    final' = final ∧
    PTranscript query-state = rest
using verifier-header-transcript-unique[OF header header0] by simp
from ntimes-verifier-query-rounds-outcome[OF query-out]
obtain raw-idxs query-idxs query-chunks where
  len-raw: length raw-idxs = rounds
and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-chunks: length query-chunks = rounds
and tr-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state
and chunks:
  ∀ i < rounds.
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
and lookups:
  ∀ i < rounds.
    fmlookup (HashMap final-state)

```

```

      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
    and idx-bounds:  $\forall \text{idx} \in \text{set query-idxs. idx} < \text{clength} * \text{scale}$ 
    by blast
  have tr-query':
    List.concat query-chunks @ PTranscript final-state = rest
  using tr-query eqs by simp
  have chunks':
     $\wedge i. i < \text{rounds} \implies$ 
      verifier-query-round-chunk (query-idxs ! i)
      f-fri-roots composition-fri-roots (query-chunks ! i)
  using chunks eqs by simp
  have lookups':
     $\wedge i. i < \text{rounds} \implies$ 
      fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks
          (verifier-header-state s fr f-fri-roots f-final as dg
            composition-fri-roots final)
          query-chunks i)) =
      Some (raw-idxs ! i)
  using lookups header-state query-count-header eqs by simp
  have derived:
    verifier-query-indices-derived s (Some (result, final-state))
    (verifier-header-state s fr f-fri-roots f-final as dg
      composition-fri-roots final)
    rest f-fri-roots composition-fri-roots query-idxs
  unfolding verifier-query-indices-derived-def
  by (intro exI[of - result] exI[of - final-state]
    exI[of - raw-idxs] exI[of - query-chunks]
    exI[of - PTranscript final-state] conjI)
    (use len-raw query-idxs-eq len-chunks tr-query' chunks' lookups'
      idx-bounds in auto)
  have shape:
    accepted-transcript-shape s (Some (result, final-state)) as query-idxs
  unfolding accepted-transcript-shape-def
  by (intro exI[of - result] exI[of - final-state] exI[of - fr]
    exI[of - f-fri-roots] exI[of - f-final] exI[of - dg]
    exI[of - composition-fri-roots] exI[of - final]
    exI[of - rest] conjI)
    (use header0 derived in simp-all)
  show ?thesis
  by (rule that[OF shape len-raw query-idxs-eq len-chunks tr-query'
    chunks' lookups'])
qed

```

lemma *verify-monad-supplied-header-query-rounds:*

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes *outcome*:
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *header0*:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
obtains *f-fl fl query-state* **where**
map snd f-fl = f-fri-roots
map snd fl = composition-fri-roots
PTranscript query-state = rest
PState query-state =
verifier-header-state s fr f-fri-roots f-final as dg
composition-fri-roots final
PQueryCounter query-state = PQueryCounter s
Some (result, final-state) ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr f-fl f-final as fl final)
rounds)
query-state)
proof –
from *verify-monad-header-extraction[OF outcome]*
obtain *fr' f-fl f-final' as' dg' fl final' query-state* **where**
header:
verifier-header-transcript s fr' (map snd f-fl) f-final' as' dg'
(map snd fl) final' (PTranscript query-state)
and *header-state*:
PState query-state =
verifier-header-state s fr' (map snd f-fl) f-final' as' dg'
(map snd fl) final'
and *query-out*:
Some (result, final-state) ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr' f-fl f-final' as' fl
final') rounds)
query-state)
and *query-count-header*:
PQueryCounter query-state = PQueryCounter s
by *metis*
have *eqs*:
fr' = fr ∧
map snd f-fl = f-fri-roots ∧
f-final' = f-final ∧
as' = as ∧
dg' = dg ∧
map snd fl = composition-fri-roots ∧
final' = final ∧

```

    PTranscript query-state = rest
  using verifier-header-transcript-unique[OF header header0] by simp
show ?thesis
  by (rule that[of f-fl fl query-state])
    (use eqs header-state query-count-header query-out in simp-all)
qed

```

```

lemma verify-monad-supplied-empty-header-query-rounds:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and header0:
    verifier-header-transcript s fr f-fri-roots f-final as dg []
    final rest
  obtains f-fl query-state where
    map snd f-fl = f-fri-roots
    PTranscript query-state = rest
    PState query-state =
      verifier-header-state s fr f-fri-roots f-final as dg [] final
    PQueryCounter query-state = PQueryCounter s
    Some (result, final-state) ∈
      set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as [] final)
          rounds)
        query-state)
proof –
  from verify-monad-supplied-header-query-rounds[OF outcome header0]
  obtain f-fl fl query-state where f-roots:
    map snd f-fl = f-fri-roots
  and fl-roots: map snd fl = []
  and tr: PTranscript query-state = rest
  and st:
    PState query-state =
      verifier-header-state s fr f-fri-roots f-final as dg [] final
  and qc: PQueryCounter query-state = PQueryCounter s
  and out:
    Some (result, final-state) ∈
      set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
        query-state)
  by blast
  have fl-empty: fl = []
  using fl-roots by (cases fl) simp-all
  show ?thesis

```

```

    by (rule that[of f-fl query-state])
      (use f-roots tr st qc out fl-empty in simp-all)
qed

lemma verifier-after-alpha-accepted-transcript-shape:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes prefix:
    Some ((fr, f-fl, f-final, as), prefix-state) ∈
      set-dist (execute verifier-alpha-prefix s)
  and suffix:
    Some (result, final-state) ∈
      set-dist
        (execute (verifier-after-alpha (fr, f-fl, f-final, as))
          prefix-state)
  shows ∃ query-idxs.
    accepted-transcript-shape s (Some (result, final-state)) as query-idxs
proof -
  have prefix-res:
    length f-fl = ceil-log clength ∧
    length as = length spec ∧
    PTranscript s =
      [fr] @ map snd f-fl @ [f-final] @ as @ PTranscript prefix-state ∧
    PState prefix-state =
      foldl concat
        (concat (foldl concat (concat (PState s) fr) (map snd f-fl))
          f-final)
        as ∧
    PQueryCounter prefix-state = PQueryCounter s
  by (rule verifier-alpha-prefix-outcome[OF prefix])
  from suffix obtain dg s5 s6 fl s7 final query-state where
    read-dg: Some (dg, s5) ∈ set-dist (execute read prefix-state)
  and degree-assert:
    Some ((), s6) ∈
      set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
  and comp-fri:
    Some (fl, s7) ∈
      set-dist (execute
        (ntimes receive-composition-fri-commits
          (ceil-log (to-nat dg + 1))) s6)
  and read-final: Some (final, query-state) ∈ set-dist (execute read s7)
  and query-out:
    Some (result, final-state) ∈
      set-dist (execute
        (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
        query-state)
  unfolding verifier-after-alpha-def
  by (auto elim!: set-dist-bindE)
  from read-outcome[OF read-dg] obtain rest5 where

```

```

tr-prefix: PTranscript prefix-state = dg # rest5
and st-s5: PState s5 = concat (PState prefix-state) dg
and tr-s5: PTranscript s5 = rest5
and query-count-s5: PQueryCounter s5 = PQueryCounter prefix-state
by blast
have s6-eq: s6 = s5
using degree-assert unfolding assert-def
by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
have comp-res:
  length fl = ceil-log (to-nat dg + 1) ∧
  PTranscript s6 = map snd fl @ PTranscript s7 ∧
  PState s7 = foldl concat (PState s6) (map snd fl) ∧
  PQueryCounter s7 = PQueryCounter s6
using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
by simp
from read-outcome[OF read-final] obtain rest-query where
  tr-s7: PTranscript s7 = final # rest-query
and st-query: PState query-state = concat (PState s7) final
and tr-query: PTranscript query-state = rest-query
and query-count-query: PQueryCounter query-state = PQueryCounter s7
by blast
have header:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
  (map snd fl) final (PTranscript query-state)
using prefix-res tr-prefix tr-s5 s6-eq comp-res tr-s7 tr-query
unfolding verifier-header-transcript-def verifier-header-messages-def
by simp
have header-state:
  PState query-state =
    verifier-header-state s fr (map snd f-fl) f-final as dg
    (map snd fl) final
using prefix-res st-s5 s6-eq comp-res st-query
unfolding verifier-header-state-def verifier-header-messages-def
by simp
have query-count-header:
  PQueryCounter query-state = PQueryCounter s
using prefix-res query-count-s5 s6-eq comp-res query-count-query
by simp
from ntimes-verifier-query-rounds-outcome[OF query-out]
obtain raw-idxs query-idxs query-chunks where
  len-raw: length raw-idxs = rounds
and query-idxs-def: query-idxs = map ( $\lambda \text{raw. index (to-nat raw)}$ ) raw-idxs
and len-chunks: length query-chunks = rounds
and tr-query-rounds:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and round-chunks:
   $\forall i < \text{rounds.}$ 
  verifier-query-round-chunk (query-idxs ! i)

```

```

      (map snd f-fl) (map snd fl) (query-chunks ! i)
and lookups:
  ∀ i < rounds.
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
and idx-bounds: ∀ idx ∈ set query-idxs. idx < clength * scale
by blast
have derived:
  verifier-query-indices-derived s (Some (result, final-state))
    (verifier-header-state s fr (map snd f-fl) f-final as dg
      (map snd fl) final)
    (PTranscript query-state) (map snd f-fl) (map snd fl) query-idxs
proof -
have tr-query-rounds':
  List.concat query-chunks @ PTranscript final-state =
    PTranscript query-state
using tr-query-rounds by simp
have lookups':
  ∀ i < rounds.
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks
          (verifier-header-state s fr (map snd f-fl) f-final as dg
            (map snd fl) final)
          query-chunks i)) =
      Some (raw-idxs ! i)
using lookups header-state query-count-header by simp
show ?thesis
unfolding verifier-query-indices-derived-def
apply (rule exI[where x=result])
apply (rule exI[where x=final-state])
apply (rule exI[where x=raw-idxs])
apply (rule exI[where x=query-chunks])
apply (rule exI[where x=PTranscript final-state])
apply (intro conjI)
  apply simp
  apply (rule len-raw)
  apply (rule query-idxs-def)
  apply (rule len-chunks)
  apply (rule tr-query-rounds')
  apply (rule round-chunks)
  apply (rule lookups')
  apply (rule idx-bounds)
done
qed
have shape:
  accepted-transcript-shape s (Some (result, final-state)) as query-idxs

```



```

unfolding accepted-transcript-shape-def
apply (rule exI[where  $x=result$ ])
apply (rule exI[where  $x=final-state$ ])
apply (rule exI[where  $x=fr$ ])
apply (rule exI[where  $x=map\ snd\ f-fl$ ])
apply (rule exI[where  $x=f-final$ ])
apply (rule exI[where  $x=dg$ ])
apply (rule exI[where  $x=map\ snd\ fl$ ])
apply (rule exI[where  $x=final$ ])
apply (rule exI[where  $x=PTranscript\ query-state$ ])
apply (intro conjI)
  apply simp
  apply (rule header)
apply (rule derived)
done
show ?thesis
  using shape by blast
qed

lemma verifier-after-alpha-accepted-transcript-header:
fixes  $s :: ('f, 'a)\ protocol-channel-scheme$ 
assumes prefix:
   $Some\ ((fr, f-fl, f-final, as), prefix-state) \in$ 
     $set-dist\ (execute\ verifier-alpha-prefix\ s)$ 
and suffix:
   $Some\ (result, final-state) \in$ 
     $set-dist$ 
     $(execute\ (verifier-after-alpha\ (fr, f-fl, f-final, as))$ 
       $prefix-state)$ 
obtains dg composition-fri-roots final rest query-idxs
where
  verifier-header-transcript  $s\ fr\ (map\ snd\ f-fl)\ f-final\ as\ dg$ 
    composition-fri-roots final rest
and accepted-transcript-shape  $s\ (Some\ (result, final-state))$ 
    as query-idxs
proof –
have prefix-res:
   $length\ f-fl = ceil-log\ clength \wedge$ 
   $length\ as = length\ spec \wedge$ 
   $PTranscript\ s =$ 
     $[fr] @ map\ snd\ f-fl @ [f-final] @ as @ PTranscript\ prefix-state$ 
using verifier-alpha-prefix-outcome[OF prefix] by simp
from suffix obtain  $dg\ s5\ s6\ fl\ s7\ final\ query-state$  where
  read-dg:  $Some\ (dg, s5) \in set-dist\ (execute\ read\ prefix-state)$ 
and degree-assert:
   $Some\ ((), s6) \in$ 
     $set-dist\ (execute\ (assert\ (to-nat\ dg \leq maxDegree))\ s5)$ 
and comp-fri:
   $Some\ (fl, s7) \in$ 

```

```

    set-dist (execute
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1))) s6)
  and read-final: Some (final, query-state) ∈ set-dist (execute read s7)
  unfolding verifier-after-alpha-def
  by (auto elim!: set-dist-bindE)
from read-outcome[OF read-dg] obtain rest5 where
  tr-prefix: PTranscript prefix-state = dg # rest5
  and tr-s5: PTranscript s5 = rest5
  by blast
have s6-eq: s6 = s5
  using degree-assert unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
have comp-res:
  length fl = ceil-log (to-nat dg + 1) ∧
  PTranscript s6 = map snd fl @ PTranscript s7
  using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
  by simp
from read-outcome[OF read-final] obtain rest-query where
  tr-s7: PTranscript s7 = final # rest-query
  and tr-query: PTranscript query-state = rest-query
  by blast
have header:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
  (map snd fl) final (PTranscript query-state)
  using prefix-res tr-prefix tr-s5 s6-eq comp-res tr-s7 tr-query
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
from verifier-after-alpha-accepted-transcript-shape[OF prefix suffix]
obtain query-idxs where shape:
  accepted-transcript-shape s (Some (result, final-state)) as query-idxs
  by blast
show ?thesis
  by (rule that[of dg map snd fl final PTranscript query-state query-idxs,
    OF header shape])
qed

```

```

lemma verify-monad-merkle-bound-witnesses:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes bind: initial-merkle-binding-no-bad s
  and outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  shows ∃ trace-table composition-table alphas query-idxs.
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table alphas query-idxs
proof -
  from verify-monad-accepted-transcript-shape[OF outcome]
  obtain alphas query-idxs where
    shape:

```

```

    accepted-transcript-shape s (Some (result, final-state)) alphas query-idxs
  by blast
from initial-merkle-binding-no-badE[OF bind outcome shape]
obtain trace-table composition-table where
  bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table alphas query-idxs
  by blast
show ?thesis
  using bound by blast
qed

```

```

lemma verify-monad-merkle-bound-tables:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes bind: initial-merkle-binding-no-bad s
  and outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  obtains trace-table composition-table alphas query-idxs
  where
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table alphas query-idxs
  and accepted-with-tables s (Some (result, final-state))
    trace-table composition-table alphas query-idxs
  and  $\bigwedge idx. idx \in \text{set query-idxs} \implies$ 
    query-consistent-at trace-table composition-table alphas idx
  and  $\exists fr \text{ composition-root.}$ 
    merkle-root-binds-table fr trace-table final-state  $\wedge$ 
    merkle-root-binds-table composition-root composition-table final-state
proof -
  from verify-monad-merkle-bound-witnesses[OF bind outcome]
  obtain trace-table composition-table alphas query-idxs where
    bound:
      accepted-with-bound-tables s (Some (result, final-state))
      trace-table composition-table alphas query-idxs
    by blast
  have tables:
    accepted-with-tables s (Some (result, final-state))
    trace-table composition-table alphas query-idxs
  by (rule accepted-with-bound-tables-imp-accepted-with-tables[OF bound])
  have query-cons:
     $\bigwedge idx. idx \in \text{set query-idxs} \implies$ 
    query-consistent-at trace-table composition-table alphas idx
  by (rule accepted-with-bound-tables-query-consistency[OF bound])
  have roots:
     $\exists fr \text{ composition-root.}$ 
    merkle-root-binds-table fr trace-table final-state  $\wedge$ 
    merkle-root-binds-table composition-root composition-table final-state
  using accepted-with-bound-tables-bound-roots[OF bound] by blast
  show ?thesis

```

by (rule that[OF bound tables query-cons roots])
 qed

lemma *verifier-query-indices-derivedE*:
assumes *verifier-query-indices-derived s out query-start-state rest*
f-fri-roots composition-fri-roots query-idxs
obtains *result final-state raw-idxs query-chunks trailing*
where *out = Some (result, final-state)*
and *length raw-idxs = rounds*
and *query-idxs = map (λraw. index (to-nat raw)) raw-idxs*
and *length query-chunks = rounds*
and *List.concat query-chunks @ trailing = rest*
and $\bigwedge i. i < \text{rounds} \implies$
verifier-query-round-chunk (query-idxs ! i)
f-fri-roots composition-fri-roots (query-chunks ! i)
and $\bigwedge i. i < \text{rounds} \implies$
fmlookup (HashMap final-state)
(QueryIndexChallenge (PQueryCounter s + i)
(state-after-query-chunks query-start-state query-chunks i)) =
Some (raw-idxs ! i)
and $\bigwedge \text{idx}. \text{idx} \in \text{set query-idxs} \implies \text{idx} < \text{clength} * \text{scale}$
using *assms unfolding verifier-query-indices-derived-def* **by** *blast*

lemma *accepted-transcript-shape-header-query-extraction*:
assumes *accepted-transcript-shape s out alphas query-idxs*
obtains *result final-state fr f-fri-roots f-final dg composition-fri-roots final rest*
where *out = Some (result, final-state)*
and *verifier-header-transcript s fr f-fri-roots f-final alphas dg*
composition-fri-roots final rest
and *verifier-query-indices-derived s out*
(verifier-header-state s fr f-fri-roots f-final alphas dg composition-fri-roots final)
rest f-fri-roots composition-fri-roots query-idxs
using *assms unfolding accepted-transcript-shape-def* **by** *blast*

lemma *accepted-transcript-shape-alphas-unique*:
assumes *shape1: accepted-transcript-shape s out alphas query-idxs*
and *shape2: accepted-transcript-shape s out alphas' query-idxs'*
shows *alphas' = alphas*
proof –
from *shape1* **obtain** *result final-state fr f-fri-roots f-final dg*
composition-fri-roots final rest **where**
out1: out = Some (result, final-state)
and *header1:*
verifier-header-transcript s fr f-fri-roots f-final alphas dg
composition-fri-roots final rest
by (*elim accepted-transcript-shape-header-query-extraction*)
from *shape2* **obtain** *result' final-state' fr' f-fri-roots' f-final' dg'*
composition-fri-roots' final' rest' **where**
out2: out = Some (result', final-state')

```

and header2:
  verifier-header-transcript s fr' f-fri-roots' f-final' alphas' dg'
    composition-fri-roots' final' rest'
  by (elim accepted-transcript-shape-header-query-extraction)
show ?thesis
  using verifier-header-transcript-unique[OF header1 header2] by simp
qed

lemma accepted-with-bound-tables-alphas-unique:
  assumes bound1:
    accepted-with-bound-tables s out trace-table composition-table
      alphas query-idxs
  and bound2:
    accepted-with-bound-tables s out trace-table' composition-table'
      alphas' query-idxs'
  shows alphas' = alphas
proof -
  have shape1: accepted-transcript-shape s out alphas query-idxs
    using bound1 unfolding accepted-with-bound-tables-def
      accepted-with-tables-def by simp
  have shape2: accepted-transcript-shape s out alphas' query-idxs'
    using bound2 unfolding accepted-with-bound-tables-def
      accepted-with-tables-def by simp
  show ?thesis
    by (rule accepted-transcript-shape-alphas-unique[OF shape1 shape2])
qed

lemma wp-verify-monad-alpha-list-set-bound:
  assumes future: alpha-future-fresh s
  and subset:  $B \subseteq \text{alpha-space}$ 
  shows
    wp-event verify-monad (alpha-list-set-hit s B) s ≤
      nnreal (card B) / nnreal (card alpha-space)
proof -
  let ?Head =
    λout. case out of
      None ⇒ False
      | Some ((-, -, -, as), -) ⇒ as ∈ B
  have prefix-bound:
    wp-event verifier-alpha-prefix ?Head s ≤
      nnreal (card B) / nnreal (card alpha-space)
    by (rule wp-verifier-alpha-prefix-alpha-space-set-bound
      [OF future subset])
  show ?thesis
    unfolding verify-monad-alpha-decomposition
proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
  show alpha-list-set-hit s B None ⇒ ?Head None
    unfolding alpha-list-set-hit-def accepted-transcript-shape-def by blast
next

```

```

fix header prefix-state out
assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-alpha-prefix s)
and suffix:
  out ∈
    set-dist (execute (verifier-after-alpha header) prefix-state)
and hit: alpha-list-set-hit s B out
obtain fr f-fl f-final as where header-eq:
  header = (fr, f-fl, f-final, as)
  by (cases header) auto
from hit obtain as' query-idxs where
  shape-hit: accepted-transcript-shape s out as' query-idxs
  and as'-B: as' ∈ B
  unfolding alpha-list-set-hit-def by blast
from shape-hit obtain result final-state where out-eq:
  out = Some (result, final-state)
  unfolding accepted-transcript-shape-def by blast
have prefix':
  Some ((fr, f-fl, f-final, as), prefix-state) ∈
    set-dist (execute verifier-alpha-prefix s)
  using prefix header-eq by simp
have suffix':
  Some (result, final-state) ∈
    set-dist
      (execute (verifier-after-alpha (fr, f-fl, f-final, as))
        prefix-state)
  using suffix header-eq out-eq by simp
from verifier-after-alpha-accepted-transcript-shape[OF prefix' suffix']
obtain query-idxs' where
  shape-prefix:
    accepted-transcript-shape s (Some (result, final-state))
    as query-idxs'
  by blast
have as' = as
  by (rule accepted-transcript-shape-alphas-unique
    [OF shape-prefix shape-hit[unfolded out-eq]])
then show ?Head (Some (header, prefix-state))
  using as'-B header-eq by simp
qed
qed

```

```

lemma wp-verify-monad-alpha-header-list-set-bound:
  fixes C :: prob
  assumes future: alpha-future-fresh s
  and subset:
     $\bigwedge$  fr f-fri-roots f-final.
    B fr f-fri-roots f-final  $\subseteq$  alpha-space
  and bound:

```

```

 $\bigwedge fr \text{ } f\text{-}fri\text{-}roots \text{ } f\text{-}final.$ 
  nnreal (card (B fr f-fri-roots f-final)) /
  nnreal (card alpha-space)  $\leq C$ 
shows
  wp-event verify-monad (alpha-header-list-set-hit s B) s  $\leq C$ 
proof -
  let ?Head =
     $\lambda out.$  case out of
      None  $\Rightarrow$  False
    | Some ((fr, f-fl, f-final, as), -)  $\Rightarrow$ 
      as  $\in B$  fr (map snd f-fl) f-final
  have prefix-bound:
    wp-event verifier-alpha-prefix ?Head s  $\leq C$ 
  by (rule wp-verifier-alpha-prefix-dependent-alpha-space-set-bound
      [OF future])
    (use subset bound in simp-all)
  show ?thesis
    unfolding verify-monad-alpha-decomposition
  proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
    show alpha-header-list-set-hit s B None  $\implies$  ?Head None
      unfolding alpha-header-list-set-hit-def
        accepted-transcript-shape-def by blast
  next
    fix header prefix-state out
    assume prefix:
      Some (header, prefix-state)  $\in$ 
        set-dist (execute verifier-alpha-prefix s)
    and suffix:
      out  $\in$ 
        set-dist (execute (verifier-after-alpha header) prefix-state)
    and hit: alpha-header-list-set-hit s B out
    obtain fr f-fl f-final as where header-eq:
      header = (fr, f-fl, f-final, as)
    by (cases header) auto
    from hit obtain as' query-idxs result final-state fr' f-fri-roots'
      f-final' dg' composition-fri-roots' final' rest' where
      shape-hit:
        accepted-transcript-shape s out as' query-idxs
    and out-eq: out = Some (result, final-state)
    and header-hit:
      verifier-header-transcript s fr' f-fri-roots' f-final' as' dg'
        composition-fri-roots' final' rest'
    and as'-B: as'  $\in B$  fr' f-fri-roots' f-final'
    unfolding alpha-header-list-set-hit-def by blast
  have prefix':
    Some ((fr, f-fl, f-final, as), prefix-state)  $\in$ 
      set-dist (execute verifier-alpha-prefix s)
    using prefix header-eq by simp
  have suffix':

```

Some (result, final-state) ∈
 set-dist
 (execute (verifier-after-alpha (fr, f-fl, f-final, as))
 prefix-state)
 using suffix header-eq out-eq by simp
 from verifier-after-alpha-accepted-transcript-header[OF prefix' suffix']
 obtain dg composition-fri-roots final rest query-idxs' where
 header-prefix:
 verifier-header-transcript s fr (map snd f-fl) f-final as dg
 composition-fri-roots final rest
 and shape-prefix:
 accepted-transcript-shape s (Some (result, final-state))
 as query-idxs'
 by metis
 have unique-header:
 fr' = fr ∧
 f-fri-roots' = map snd f-fl ∧
 f-final' = f-final ∧
 as' = as
 using verifier-header-transcript-unique[OF header-prefix header-hit]
 by simp
 then show ?Head (Some (header, prefix-state))
 using as'-B header-eq by simp
 qed
 qed

lemma wp-alpha-bad-set-hit-bound-via-header-union:
 fixes C :: prob
 assumes future: alpha-future-fresh s
 and subset: $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$
 and union-bound:
 $\bigwedge \text{fr f-fri-roots f-final.}$
 nnreal
 $(\text{card}$
 $(\text{alpha-header-union-bad-sets s bad-sets fr f-fri-roots f-final})) /$
 $\text{nnreal} (\text{card alpha-space}) \leq C$
 shows wp-event verify-monad (alpha-bad-set-hit s bad-sets) s ≤ C
proof –
 have header-bound:
 wp-event verify-monad
 (alpha-header-list-set-hit s
 (alpha-header-union-bad-sets s bad-sets)) s ≤ C
 by (rule wp-verify-monad-alpha-header-list-set-bound[OF future])
 (use alpha-header-union-bad-sets-subset-alpha-space union-bound in
 simp-all)
 have wp-event verify-monad (alpha-bad-set-hit s bad-sets) s ≤
 wp-event verify-monad
 (alpha-header-list-set-hit s
 (alpha-header-union-bad-sets s bad-sets)) s

by (rule wp-event-mono)
 (rule alpha-bad-set-hit-imp-alpha-header-union-bad-set-hit[OF - subset])
 also have ... $\leq C$
 by (rule header-bound)
 finally show ?thesis .
 qed

lemma wp-alpha-bad-set-hit-bound-via-supported-header-union:

fixes $C :: \text{prob}$
 assumes future: alpha-future-fresh s
 and subset: $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$
 and union-bound:
 $\bigwedge \text{fr f-fri-roots f-final.}$
 nnreal
 $(\text{card}$
 $(\text{alpha-header-supported-union-bad-sets } s \text{ bad-sets fr f-fri-roots}$
 $\text{f-final})) /$
 $\text{nnreal } (\text{card } \text{alpha-space}) \leq C$

shows wp-event verify-monad (alpha-bad-set-hit s bad-sets) $s \leq C$

proof –

have header-bound:
 wp-event verify-monad
 (alpha-header-list-set-hit s
 (alpha-header-supported-union-bad-sets s bad-sets)) $s \leq C$
 by (rule wp-verify-monad-alpha-header-list-set-bound[OF future])
 (use alpha-header-supported-union-bad-sets-subset-alpha-space
 union-bound in simp-all)
 have wp-event verify-monad (alpha-bad-set-hit s bad-sets) $s \leq$
 wp-event verify-monad
 (alpha-header-list-set-hit s
 (alpha-header-supported-union-bad-sets s bad-sets)) s
 by (rule wp-event-mono-on-support)
 (rule alpha-bad-set-hit-imp-alpha-header-supported-union-bad-set-hit
 [OF - - subset])
 also have ... $\leq C$
 by (rule header-bound)
 finally show ?thesis .

qed

lemma accepted-with-tables-header-query-extraction:

assumes accepted-with-tables s out trace-table composition-table alphas query-idxs
 obtains result final-state fr f-fri-roots f-final dg composition-fri-roots final rest
 where out = Some (result, final-state)
 and verifier-header-transcript s fr f-fri-roots f-final alphas dg
 composition-fri-roots final rest
 and verifier-query-indices-derived s out
 (verifier-header-state s fr f-fri-roots f-final alphas dg composition-fri-roots final)
 rest f-fri-roots composition-fri-roots query-idxs
 and $\bigwedge \text{idx. idx} \in \text{set query-idxs} \implies$

```

    query-consistent-at trace-table composition-table alphas idx
  and length trace-table = clength * scale
  and length composition-table = clength * scale
  and length alphas = length spec
using assms
unfolding accepted-with-tables-def accepted-transcript-shape-def
by blast

```

lemma *accepted-with-tables-existential-witnesses:*
assumes *accepted-with-tables s out trace-table composition-table alphas query-idxs*
shows $\exists$ *trace-table composition-table as query-idxs.*
accepted-with-tables s out trace-table composition-table as query-idxs
using *assms* **by** *blast*

lemma *accepted-transcript-shape-header-extraction:*
assumes *shape: accepted-transcript-shape s (Some (result, final-state)) alphas query-idxs*
shows $\exists$ *fr f-fri-roots f-final as dg composition-fri-roots final rest.*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
using *shape* **unfolding** *accepted-transcript-shape-def* **by** *blast*

lemma *accepted-transcript-shape-query-indices-extraction:*
assumes *shape: accepted-transcript-shape s (Some (result, final-state)) alphas query-idxs*
shows $\exists$ *fr f-fri-roots f-final dg composition-fri-roots final rest.*
verifier-query-indices-derived s (Some (result, final-state))
(verifier-header-state s fr f-fri-roots f-final alphas dg
composition-fri-roots final)
rest f-fri-roots composition-fri-roots query-idxs
using *shape* **unfolding** *accepted-transcript-shape-def* **by** *blast*

lemma *accepted-with-tables-has-witnesses:*
assumes *tables: accepted-with-tables s (Some (result, final-state))*
trace-table composition-table alphas query-idxs
shows $\exists$ *trace-table composition-table as query-idxs.*
accepted-with-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
using *tables* **by** *blast*

end

end

theory *Soundness-Execution*
imports *Soundness-Execution-Extraction*
begin

Compatibility umbrella for verifier execution soundness lemmas.

end

```

theory Soundness-Bad-Events
  imports
    Soundness-Execution
begin

```

```

context soundness
begin

```

At this stage *accepted-with-tables* is the boundary between the verifier execution and the algebraic proof. It records the trace table, composition table, and verifier challenges associated with an accepted execution. It now also records that the initial verifier transcript has the expected header shape and that the sampled query positions are locally consistent with the associated tables. Merkle binding is separated into *accepted-with-bound-tables*, because malicious binding requires additional Merkle/collision-resistance reasoning.

definition *composition-bad-context*

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$ 
  'f list  $\Rightarrow$  'f list  $\Rightarrow$  'f list  $\Rightarrow$  nat list  $\Rightarrow$ 
  'f poly  $\Rightarrow$  bool

```

where

```

composition-bad-context s out trace-table composition-table as query-idxs f  $\longleftrightarrow$ 
  accepted-with-bound-tables s out trace-table composition-table as query-idxs f  $\wedge$ 
  degree f < clength  $\wedge$ 
  trace-table = map (poly f) eval-domain  $\wedge$ 
  violated-constraints f  $\neq$  {}  $\wedge$ 
  composition-table-low-degree maxDegree composition-table  $\wedge$ 
  all-queries-consistent trace-table composition-table as

```

definition *composition-degree-bad*

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool

```

where

```

composition-degree-bad s out  $\longleftrightarrow$ 
  ( $\exists$  trace-table composition-table as query-idxs f.
    composition-bad-context s out trace-table composition-table as query-idxs f  $\wedge$ 
     $\neg$  common-denominator-degree-bounds f as)

```

definition *composition-randomization-bad*

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool

```

where

```

composition-randomization-bad s out  $\longleftrightarrow$ 
  ( $\exists$  trace-table composition-table as query-idxs f.
    composition-bad-context s out trace-table composition-table as query-idxs f  $\wedge$ 
    common-denominator-degree-bounds f as  $\wedge$ 
    random-combination-common-denominator-hides-violations f as)

```

definition *composition-bad*

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  composition-bad s out  $\longleftrightarrow$ 
    composition-degree-bad s out  $\vee$  composition-randomization-bad s out

```

lemma *composition-bad-has-common-denominator-witness:*

```

assumes composition-bad s out
shows  $\exists$  trace-table composition-table as query-idxs f.
  accepted-with-bound-tables s out trace-table composition-table as query-idxs  $\wedge$ 
  degree f < clength  $\wedge$ 
  trace-table = map (poly f) eval-domain  $\wedge$ 
  violated-constraints f  $\neq$  {}  $\wedge$ 
  composition-table-low-degree maxDegree composition-table  $\wedge$ 
  all-queries-consistent trace-table composition-table as  $\wedge$ 
  ( $\neg$  common-denominator-degree-bounds f as  $\vee$ 
    random-combination-common-denominator-hides-violations f as)
using assms
unfolding composition-bad-def composition-degree-bad-def
  composition-randomization-bad-def composition-bad-context-def
by blast

```

lemma *composition-bad-imp-spec-nonempty:*

```

assumes composition-bad s out
shows spec  $\neq$  []
proof -
  obtain trace-table composition-table as query-idxs f
  where violated-constraints f  $\neq$  {}
  using composition-bad-has-common-denominator-witness[OF assms] by blast
  then obtain entry where entry  $\in$  violated-constraints f
  by auto
  then have entry  $\in$  set spec
  unfolding violated-constraints-def by simp
  then show ?thesis
  by auto
qed

```

definition *trace-fri-bad*

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  trace-fri-bad s out  $\longleftrightarrow$ 
    ( $\exists$  trace-table composition-table as query-idxs.
      accepted-with-bound-tables s out trace-table composition-table as query-idxs  $\wedge$ 
       $\neg$  trace-table-low-degree trace-table)

```

definition *composition-fri-bad*

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 

```

$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-fri-bad } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table as query-idxs.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as query-idxs} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table})$

definition *query-bad*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{query-bad } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table as query-idxs.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as query-idxs} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\neg \text{all-queries-consistent trace-table composition-table as})$

definition *soundness-bad-event*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{soundness-bad-event } s \text{ out} \longleftrightarrow$
 $\text{composition-bad } s \text{ out} \vee$
 $\text{trace-fri-bad } s \text{ out} \vee$
 $\text{composition-fri-bad } s \text{ out} \vee$
 $\text{query-bad } s \text{ out}$

definition *query-sampling-success-space*
 $:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat set}$
where
 $\text{query-sampling-success-space trace-table composition-table as} =$
 $(\text{if trace-table-low-degree trace-table} \wedge$
 $\text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\neg \text{all-queries-consistent trace-table composition-table as}$
 $\text{then query-agreement-indices trace-table composition-table as}$
 $\text{else } \{\})$

definition *query-index-set-hit*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{query-index-set-hit } s \text{ good-sets out} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table as query-idxs.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as query-idxs} \wedge$
 $\text{set query-idxs} \subseteq \text{good-sets trace-table composition-table as})$

definition *query-index-round-set-hit-at*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat set}) \Rightarrow$
 $\text{nat} \Rightarrow (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{query-index-round-set-hit-at } s \text{ good-sets } i \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table as query-idxs.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as}$
 $\text{query-idxs} \wedge$
 $i < \text{rounds} \wedge$
 $\text{query-idxs} ! i \in \text{good-sets trace-table composition-table as})$

definition *query-index-round-set-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{query-index-round-set-hit } s \text{ good-sets out} \longleftrightarrow$
 $(\exists i < \text{rounds. query-index-round-set-hit-at } s \text{ good-sets } i \text{ out})$

definition *query-round-index-set-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{nat set} \Rightarrow$
 $(\text{unit} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{query-round-index-set-hit } s \text{ B out} \longleftrightarrow$
 $(\exists \text{ raw } t.$
 $\text{out} = \text{Some } ((), t) \wedge$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some raw} \wedge$
 $\text{index } (\text{to-nat raw}) \in B)$

definition *query-round-any-index-set-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{nat set} \Rightarrow$
 $(\text{unit} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{query-round-any-index-set-hit } s \text{ B out} \longleftrightarrow$
 $(\exists \text{ raw } t \ x.$
 $\text{out} = \text{Some } ((), t) \wedge$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s) x) = \text{Some raw} \wedge$
 $\text{index } (\text{to-nat raw}) \in B)$

definition *query-rounds-any-index-set-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{nat set} \Rightarrow \text{nat} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{query-rounds-any-index-set-hit } s \text{ B n out} \longleftrightarrow$
 $(\exists \text{ results } t \ i \ x \ \text{raw.}$
 $\text{out} = \text{Some } (\text{results}, t) \wedge$

$i < n \wedge$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{QueryIndexChallenge } (\text{PQueryCounter } s + i) \ x) = \text{Some } \text{raw} \wedge$
 $\text{index } (\text{to-nat } \text{raw}) \in B$

lemma *accepted-with-bound-tables-query-rounds-any-index-set-hit:*

assumes *bound:*

accepted-with-bound-tables s *out* *trace-table composition-table* *as*
query-idxs

and *i-bound:* $i < \text{rounds}$

and *hit:* $\text{query-idxs} \ ! \ i \in B$

shows *query-rounds-any-index-set-hit* $s \ B$ *rounds* *out*

proof –

have *shape:* *accepted-transcript-shape* s *out* *as* *query-idxs*

using *accepted-with-bound-tables-imp-accepted-with-tables*[*OF* *bound*]

accepted-with-tables-imp-accepted-transcript-shape **by** *blast*

then obtain *result final-state* *fr* *f-fri-roots* *f-final* *dg*

composition-fri-roots *final* *rest* *raw-idxs* *query-chunks* *trailing* **where**

out-eq: $\text{out} = \text{Some } (\text{result}, \text{final-state})$

and *len-raw:* $\text{length } \text{raw-idxs} = \text{rounds}$

and *query-idxs-eq:* $\text{query-idxs} = \text{map } (\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})) \ \text{raw-idxs}$

and *lookup:*

$\forall i < \text{rounds}.$

$\text{fmlookup } (\text{HashMap } \text{final-state})$

$(\text{QueryIndexChallenge } (\text{PQueryCounter } s + i)$

$(\text{state-after-query-chunks}$

$(\text{verifier-header-state } s \ \text{fr } \text{f-fri-roots } \text{f-final} \ \text{as } \text{dg}$

$\text{composition-fri-roots } \text{final})$

$\text{query-chunks } i)) =$

$\text{Some } (\text{raw-idxs} \ ! \ i)$

unfolding *accepted-transcript-shape-def* *verifier-query-indices-derived-def*

by *blast*

have *raw-hit:* $\text{index } (\text{to-nat } (\text{raw-idxs} \ ! \ i)) \in B$

using *hit* *query-idxs-eq* *len-raw* *i-bound* **by** *simp*

have *lookup-i:*

$\text{fmlookup } (\text{HashMap } \text{final-state})$

$(\text{QueryIndexChallenge } (\text{PQueryCounter } s + i)$

$(\text{state-after-query-chunks}$

$(\text{verifier-header-state } s \ \text{fr } \text{f-fri-roots } \text{f-final} \ \text{as } \text{dg}$

$\text{composition-fri-roots } \text{final})$

$\text{query-chunks } i)) =$

$\text{Some } (\text{raw-idxs} \ ! \ i)$

using *lookup* *i-bound* **by** *simp*

show *?thesis*

unfolding *query-rounds-any-index-set-hit-def*

apply $(\text{rule } \text{exI}[\text{where } x=\text{result}])$

apply $(\text{rule } \text{exI}[\text{where } x=\text{final-state}])$

apply $(\text{rule } \text{exI}[\text{where } x=i])$

apply $(\text{rule } \text{exI}[$

where $x = \text{state-after-query-chunks}$
 (verifier-header-state s fr f -fri-roots f -final as dg
 composition-fri-roots final)
 query-chunks i)
apply (rule exI [**where** $x = \text{raw-idxs} ! i$])
using out-eq i -bound lookup- i raw-hit **by** simp
qed

lemma query-index-round-set-hit-at-imp-query-rounds-any-index-set-hit:
assumes hit: query-index-round-set-hit-at s good-sets i out
shows
 $\exists$ trace-table composition-table as.
 query-rounds-any-index-set-hit s
 (good-sets trace-table composition-table as) rounds out
proof –
from hit **obtain** trace-table composition-table as query-idxs **where**
 bound:
 accepted-with-bound-tables s out trace-table composition-table as
 query-idxs
and i -bound: $i < \text{rounds}$
and query-hit:
 query-idxs $! i \in \text{good-sets trace-table composition-table as}$
unfolding query-index-round-set-hit-at-def **by** blast
have round-hit:
 query-rounds-any-index-set-hit s
 (good-sets trace-table composition-table as) rounds out
by (rule accepted-with-bound-tables-query-rounds-any-index-set-hit
 [OF bound i -bound query-hit])
show ?thesis
using round-hit **by** blast
qed

definition query-header-supported-table-candidates
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow ('f \text{ list} \times 'f \text{ list}) \text{ set}$
where
 query-header-supported-table-candidates s fr f -fri-roots f -final as dg
 composition-fri-roots final =
 $\{(\text{trace-table}, \text{composition-table}).$
 $\exists \text{ out query-idxs rest.}$
 $\text{out} \in \text{set-dist (execute verify-monad } s) \wedge$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as}$
 $\text{query-idxs} \wedge$
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final as } dg$
 $\text{composition-fri-roots final rest} \}$

definition query-header-supported-table-candidate-state
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

query-header-supported-table-candidate-state *s* *fr* *f-fri-roots* *f-final*
as dg *composition-fri-roots* *final* *trace-table* *composition-table*
final-state $\longleftrightarrow$
 $(\exists \text{ result } \text{query-idxs } \text{rest}.$
 $\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute } \text{verify-monad } s) \wedge$
 $\text{accepted-with-bound-tables } s \text{ (Some } (\text{result}, \text{final-state}))$
 $\text{trace-table } \text{composition-table} \text{ as } \text{query-idxs} \wedge$
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final} \text{ as } dg$
 $\text{composition-fri-roots } \text{final} \text{ rest} \wedge$
 $\text{merkle-root-binds-table } \text{fr } \text{trace-table } \text{final-state} \wedge$
 $\text{composition-fri-roots} \neq [] \wedge$
 $\text{merkle-root-binds-table } (\text{hd } \text{composition-fri-roots}) \text{ composition-table}$
 $\text{final-state})$

lemma *query-header-supported-table-candidates-iff-state:*

$(\text{trace-table}, \text{composition-table}) \in$
 $\text{query-header-supported-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final} \text{ as } dg$
 $\text{composition-fri-roots } \text{final} \longleftrightarrow$
 $(\exists \text{ final-state}.$
 $\text{query-header-supported-table-candidate-state } s \text{ fr } f\text{-fri-roots } f\text{-final}$
 $\text{as } dg \text{ composition-fri-roots } \text{final} \text{ trace-table } \text{composition-table}$
 $\text{final-state})$

proof

assume *candidate:*

$(\text{trace-table}, \text{composition-table}) \in$
 $\text{query-header-supported-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final} \text{ as } dg$
 $\text{composition-fri-roots } \text{final}$

then obtain *out* *query-idxs* *rest* **where**

outcome: *out* $\in \text{set-dist } (\text{execute } \text{verify-monad } s)$

and *bound:*

$\text{accepted-with-bound-tables } s \text{ out } \text{trace-table} \text{ composition-table} \text{ as}$
 query-idxs

and *header:*

$\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final} \text{ as } dg$
 $\text{composition-fri-roots } \text{final} \text{ rest}$

unfolding *query-header-supported-table-candidates-def* **by** *blast*

from *bound* **obtain** *result* *final-state* *fr'* *f-fri-roots'* *f-final'* *dg'*

composition-fri-roots' *final'* *rest'* **where**

out-eq: *out* = *Some* (*result*, *final-state*)

and *header':*

$\text{verifier-header-transcript } s \text{ fr' } f\text{-fri-roots' } f\text{-final' as } dg'$
 $\text{composition-fri-roots' } \text{final' } \text{rest'}$

and *trace-bind:*

$\text{merkle-root-binds-table } \text{fr' } \text{trace-table } \text{final-state}$

and *comp-nonempty:* *composition-fri-roots'* $\neq []$

```

and comp-bind:
  merkle-root-binds-table (hd composition-fri-roots')
  composition-table final-state
unfolding accepted-with-bound-tables-def by blast
have eqs:
  fr' = fr ∧
  f-fri-roots' = f-fri-roots ∧
  f-final' = f-final ∧
  dg' = dg ∧
  composition-fri-roots' = composition-fri-roots ∧
  final' = final ∧
  rest' = rest
using verifier-header-transcript-unique[OF header' header] by simp
have state:
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
  as dg composition-fri-roots final trace-table composition-table
  final-state
unfolding query-header-supported-table-candidate-state-def
by (intro exI[of - result] exI[of - query-idxs] exI[of - rest])
  (use outcome out-eq bound header trace-bind comp-nonempty comp-bind eqs
   in simp)
then show
  ∃ final-state.
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
  as dg composition-fri-roots final trace-table composition-table
  final-state
by blast
next
assume
  ∃ final-state.
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
  as dg composition-fri-roots final trace-table composition-table
  final-state
then obtain final-state result query-idxs rest where
  outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
and bound:
  accepted-with-bound-tables s (Some (result, final-state))
  trace-table composition-table as query-idxs
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
unfolding query-header-supported-table-candidate-state-def by blast
show (trace-table, composition-table) ∈
  query-header-supported-table-candidates s fr f-fri-roots f-final as dg
  composition-fri-roots final
unfolding query-header-supported-table-candidates-def
using outcome bound header by blast
qed

```

lemma *query-header-supported-table-candidate-state-hash-extends:*
assumes
query-header-supported-table-candidate-state s fr f-fri-roots f-final
as dg composition-fri-roots final trace-table composition-table
final-state
shows $s \leq \text{final-state}$
proof –
from *assms* **obtain** *result query-idxs rest* **where**
outcome:
Some (result, final-state) ∈ set-dist (execute verify-monad s)
unfolding *query-header-supported-table-candidate-state-def* **by** *blast*
show *?thesis*
by *(rule verify-monad-hash-extends[OF outcome])*
qed

lemma *query-header-supported-table-candidate-states-merkle-compatible-unique-if-clean-merge:*
assumes *compatible:*
merkle-hash-maps-compatible final-state final-state'
and *clean:*
 $\neg \text{hash-map-output-collision}$
(merkle-hash-state-merge final-state final-state')
and *candidate:*
query-header-supported-table-candidate-state s fr f-fri-roots f-final
as dg composition-fri-roots final trace-table composition-table
final-state
and *candidate':*
query-header-supported-table-candidate-state s fr f-fri-roots f-final
as dg composition-fri-roots final trace-table' composition-table'
final-state'
shows
 $\text{trace-table} = \text{trace-table}' \wedge$
 $\text{composition-table} = \text{composition-table}'$
proof –
from *candidate* **obtain** *result query-idxs rest* **where**
bound:
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
and *trace-bind:*
merkle-root-binds-table fr trace-table final-state
and *comp-nonempty:* $\text{composition-fri-roots} \neq []$
and *comp-bind:*
merkle-root-binds-table (hd composition-fri-roots) composition-table
final-state
unfolding *query-header-supported-table-candidate-state-def* **by** *blast*
from *candidate'* **obtain** *result' query-idxs' rest'* **where**
bound':
accepted-with-bound-tables s (Some (result', final-state'))
trace-table' composition-table' as query-idxs'

```

and trace-bind':
  merkle-root-binds-table fr trace-table' final-state'
and comp-bind':
  merkle-root-binds-table (hd composition-fri-roots) composition-table'
    final-state'
unfolding query-header-supported-table-candidate-state-def by blast
have same-trace-len: length trace-table = length trace-table'
using accepted-with-bound-tables-shapes(1)[OF bound]
  accepted-with-bound-tables-shapes(1)[OF bound']
by simp
have trace-nonempty: trace-table ≠ []
using accepted-with-bound-tables-shapes(1)[OF bound]
  eval-domain-nontrivial by auto
have trace-eq: trace-table = trace-table'
by (rule
  merkle-root-binds-same-length-tables-merkle-compatible-unique-if-clean-merge
  [OF compatible clean trace-bind trace-bind' same-trace-len
    trace-nonempty'])
have same-comp-len: length composition-table = length composition-table'
using accepted-with-bound-tables-shapes(2)[OF bound]
  accepted-with-bound-tables-shapes(2)[OF bound']
by simp
have comp-nonempty': composition-table ≠ []
using accepted-with-bound-tables-shapes(2)[OF bound]
  eval-domain-nontrivial by auto
have comp-eq: composition-table = composition-table'
by (rule
  merkle-root-binds-same-length-tables-merkle-compatible-unique-if-clean-merge
  [OF compatible clean comp-bind comp-bind' same-comp-len
    comp-nonempty'])
show ?thesis
using trace-eq comp-eq by simp
qed

```

lemma *query-header-supported-table-candidates-subset-singleton-if-pairwise-merkle-clean-merge*:
assumes *pairwise*:

```

  ∧ trace-table composition-table final-state trace-table'
    composition-table' final-state'.
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table composition-table
    final-state ⇒
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table' composition-table'
    final-state' ⇒
  merkle-hash-maps-compatible final-state final-state' ∧
  ¬ hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')

```

shows

∃ *trace-table composition-table*.

```

    query-header-supported-table-candidates s fr f-fri-roots f-final as dg
    composition-fri-roots final  $\subseteq$ 
    {(trace-table, composition-table)}
proof (cases
    query-header-supported-table-candidates s fr f-fri-roots f-final as dg
    composition-fri-roots final = {})
case True
then show ?thesis
    by (intro exI[of -] exI[of -]) simp
next
case False
then obtain table-pair0 where table-pair0:
    table-pair0  $\in$ 
    query-header-supported-table-candidates s fr f-fri-roots f-final as dg
    composition-fri-roots final
by blast
obtain trace-table0 composition-table0 where table-pair0-eq:
    table-pair0 = (trace-table0, composition-table0)
by (cases table-pair0)
have table0:
    (trace-table0, composition-table0)  $\in$ 
    query-header-supported-table-candidates s fr f-fri-roots f-final as dg
    composition-fri-roots final
using table-pair0 unfolding table-pair0-eq .
then obtain final-state0 where state0:
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table0 composition-table0
    final-state0
using query-header-supported-table-candidates-iff-state by blast
have subset:
    query-header-supported-table-candidates s fr f-fri-roots f-final as dg
    composition-fri-roots final  $\subseteq$ 
    {(trace-table0, composition-table0)}
proof
fix table-pair
assume pair-in:
    table-pair  $\in$ 
    query-header-supported-table-candidates s fr f-fri-roots f-final as
    dg composition-fri-roots final
obtain trace-table composition-table where pair-eq:
    table-pair = (trace-table, composition-table)
by (cases table-pair)
from pair-in obtain final-state where state:
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table composition-table
    final-state
unfolding pair-eq
using query-header-supported-table-candidates-iff-state by blast
have compatible:

```

```

merkle-hash-maps-compatible final-state final-state0
using pairwise[OF state state0] by simp
have clean:
  ¬ hash-map-output-collision
    (merkle-hash-state-merge final-state final-state0)
using pairwise[OF state state0] by simp
have eqs:
  trace-table = trace-table0 ∧
  composition-table = composition-table0
by (rule
  query-header-supported-table-candidate-states-merkle-compatible-unique-if-clean-merge
  [OF compatible clean state state0])
show table-pair ∈ {(trace-table0, composition-table0)}
using pair-eq eqs by simp
qed
then show ?thesis
by blast
qed

lemma query-header-supported-table-candidates-subset-singleton-if-pairwise-no-merkle-value-conflict-clean-merge:
assumes pairwise:
  ∧ trace-table composition-table final-state trace-table'
    composition-table' final-state'.
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
  as dg composition-fri-roots final trace-table composition-table
  final-state ⇒
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
  as dg composition-fri-roots final trace-table' composition-table'
  final-state' ⇒
  ¬ merkle-hash-value-conflict final-state final-state' ∧
  ¬ hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
shows
  ∃ trace-table composition-table.
  query-header-supported-table-candidates s fr f-fri-roots f-final as dg
  composition-fri-roots final ⊆
  {(trace-table, composition-table)}
proof (rule
  query-header-supported-table-candidates-subset-singleton-if-pairwise-merkle-clean-merge)
fix trace-table composition-table final-state trace-table'
  composition-table' final-state'
assume candidate:
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
  as dg composition-fri-roots final trace-table composition-table
  final-state
and candidate':
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
  as dg composition-fri-roots final trace-table' composition-table'
  final-state'

```

```

have no-conflict:
   $\neg$  merkle-hash-value-conflict final-state final-state'
  using pairwise[OF candidate candidate'] by simp
have compatible:
  merkle-hash-maps-compatible final-state final-state'
  using no-conflict
  unfolding merkle-hash-maps-compatible-iff-no-value-conflict .
have clean:
   $\neg$  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
  using pairwise[OF candidate candidate'] by simp
show merkle-hash-maps-compatible final-state final-state'  $\wedge$ 
   $\neg$  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
  using compatible clean by simp
qed

lemma query-header-supported-table-candidates-subset-singleton-if-common-clean-merkle-extension:
assumes clean:  $\neg$  hash-map-output-collision u
and common-ext:
   $\bigwedge$  trace-table composition-table final-state.
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
      as dg composition-fri-roots final trace-table composition-table
        final-state  $\implies$ 
          merkle-hash-extends final-state u
shows
   $\exists$  trace-table composition-table.
    query-header-supported-table-candidates s fr f-fri-roots f-final as dg
      composition-fri-roots final  $\subseteq$ 
         $\{(trace-table, composition-table)\}$ 
proof (rule
  query-header-supported-table-candidates-subset-singleton-if-pairwise-no-merkle-value-conflict-clean-merge)
fix trace-table composition-table final-state trace-table'
  composition-table' final-state'
assume candidate:
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table composition-table
      final-state
and candidate':
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table' composition-table'
      final-state'
have ext: merkle-hash-extends final-state u
  by (rule common-ext[OF candidate])
have ext': merkle-hash-extends final-state' u
  by (rule common-ext[OF candidate'])
show  $\neg$  merkle-hash-value-conflict final-state final-state'  $\wedge$ 
   $\neg$  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')

```

by (rule common-clean-merkle-extension-imp-pairwise-merkle-clean
[OF ext ext' clean])

qed

lemma *query-header-supported-table-candidate-states-distinct-imp-merkle-conflict-or-merge-collision*:

assumes *candidate*:

query-header-supported-table-candidate-state s fr f-fri-roots f-final
 as dg composition-fri-roots final trace-table composition-table
 final-state

and *candidate'*:

query-header-supported-table-candidate-state s fr f-fri-roots f-final
 as dg composition-fri-roots final trace-table' composition-table'
 final-state'

and *distinct-tables*:

trace-table $\neq$ *trace-table'* $\vee$
 composition-table $\neq$ *composition-table'*

shows

merkle-hash-value-conflict final-state final-state' $\vee$
 hash-map-output-collision
 (*merkle-hash-state-merge final-state final-state'*)

proof (rule ccontr)

assume *no-bad*:

$\neg$ (*merkle-hash-value-conflict final-state final-state'* $\vee$
 hash-map-output-collision
 (*merkle-hash-state-merge final-state final-state'*))

have *compatible*:

merkle-hash-maps-compatible final-state final-state'

using *no-bad* **unfolding** *merkle-hash-maps-compatible-iff-no-value-conflict*

by *simp*

have *clean*:

$\neg$ *hash-map-output-collision*
 (*merkle-hash-state-merge final-state final-state'*)

using *no-bad* **by** *simp*

have *eqs*:

trace-table = *trace-table'* $\wedge$
 composition-table = *composition-table'*

by (rule
 query-header-supported-table-candidate-states-merkle-compatible-unique-if-clean-merge
 [OF *compatible clean candidate candidate'*])

then show *False*

using *distinct-tables* **by** *simp*

qed

definition *query-header-supported-pairwise-merkle-bad*

$\because$ (*f*, *a*) *protocol-channel-scheme* $\Rightarrow$
 f $\Rightarrow$ *f list* $\Rightarrow$ *f* $\Rightarrow$ *f list* $\Rightarrow$ *f* $\Rightarrow$
 f list $\Rightarrow$ *f* $\Rightarrow$ *bool*

where

query-header-supported-pairwise-merkle-bad s fr f-fri-roots f-final as

$dg \text{ composition-fri-roots final} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table final-state trace-table'}$
 $\quad \text{composition-table' final-state'.$
 $\text{query-header-supported-table-candidate-state } s \text{ fr f-fri-roots f-final}$
 $\quad \text{as } dg \text{ composition-fri-roots final trace-table composition-table}$
 $\quad \text{final-state} \wedge$
 $\text{query-header-supported-table-candidate-state } s \text{ fr f-fri-roots f-final}$
 $\quad \text{as } dg \text{ composition-fri-roots final trace-table' composition-table'}$
 $\quad \text{final-state' } \wedge$
 $(\text{trace-table} \neq \text{trace-table'} \vee$
 $\quad \text{composition-table} \neq \text{composition-table'}) \wedge$
 $(\text{merkle-hash-value-conflict final-state final-state'} \vee$
 $\quad \text{hash-map-output-collision}$
 $\quad (\text{merkle-hash-state-merge final-state final-state'})))$

definition *query-supported-pairwise-merkle-bad*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{query-supported-pairwise-merkle-bad } s \longleftrightarrow$
 $(\exists \text{ fr f-fri-roots f-final as } dg \text{ composition-fri-roots final.}$
 $\quad \text{query-header-supported-pairwise-merkle-bad } s \text{ fr f-fri-roots f-final}$
 $\quad \text{as } dg \text{ composition-fri-roots final})$

lemma *query-header-supported-table-candidates-not-singleton-imp-pairwise-merkle-bad:*

assumes *not-unique:*

$\neg (\exists \text{ trace-table composition-table.}$
 $\quad \text{query-header-supported-table-candidates } s \text{ fr f-fri-roots f-final as } dg$
 $\quad \text{composition-fri-roots final} \subseteq$
 $\quad \{(\text{trace-table}, \text{composition-table} \})$

shows

$\exists \text{ trace-table composition-table final-state trace-table'}$
 $\quad \text{composition-table' final-state'.$
 $\text{query-header-supported-table-candidate-state } s \text{ fr f-fri-roots f-final}$
 $\quad \text{as } dg \text{ composition-fri-roots final trace-table composition-table}$
 $\quad \text{final-state} \wedge$
 $\text{query-header-supported-table-candidate-state } s \text{ fr f-fri-roots f-final}$
 $\quad \text{as } dg \text{ composition-fri-roots final trace-table' composition-table'}$
 $\quad \text{final-state' } \wedge$
 $(\text{trace-table} \neq \text{trace-table'} \vee$
 $\quad \text{composition-table} \neq \text{composition-table'}) \wedge$
 $(\text{merkle-hash-value-conflict final-state final-state'} \vee$
 $\quad \text{hash-map-output-collision}$
 $\quad (\text{merkle-hash-state-merge final-state final-state'}))$

proof —

let $?C =$

$\text{query-header-supported-table-candidates } s \text{ fr f-fri-roots f-final as } dg$
 $\quad \text{composition-fri-roots final}$

have *nonempty:* $?C \neq \{\}$

using *not-unique* **by** *auto*

```

then obtain table-pair where table-pair:
  table-pair  $\in ?C$ 
  by blast
obtain trace-table composition-table where table-pair-eq:
  table-pair = (trace-table, composition-table)
  by (cases table-pair)
have table:
  (trace-table, composition-table)  $\in ?C$ 
  using table-pair unfolding table-pair-eq .
have  $\neg ?C \subseteq \{(trace-table, composition-table)\}$ 
  using not-unique by blast
then obtain table-pair' where table-pair':
  table-pair'  $\in ?C$ 
  and table-pair'-not:
  table-pair'  $\notin \{(trace-table, composition-table)\}$ 
  by auto
obtain trace-table' composition-table' where table-pair'-eq:
  table-pair' = (trace-table', composition-table')
  by (cases table-pair')
have table':
  (trace-table', composition-table')  $\in ?C$ 
  using table-pair' unfolding table-pair'-eq .
have distinct-tables:
  trace-table  $\neq trace-table'$   $\vee$ 
  composition-table  $\neq composition-table'$ 
  using table-pair'-not unfolding table-pair'-eq by auto
from table obtain final-state where candidate:
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
  as dg composition-fri-roots final trace-table composition-table
  final-state
  using query-header-supported-table-candidates-iff-state by blast
from table' obtain final-state' where candidate':
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
  as dg composition-fri-roots final trace-table' composition-table'
  final-state'
  using query-header-supported-table-candidates-iff-state by blast
have bad:
  merkle-hash-value-conflict final-state final-state'  $\vee$ 
  hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')
  by (rule
  query-header-supported-table-candidate-states-distinct-imp-merkle-conflict-or-merge-collision
  [OF candidate candidate' distinct-tables])
show ?thesis
  using candidate candidate' distinct-tables bad by blast
qed

lemma query-header-supported-pairwise-merkle-bad-extends-initial:
  assumes bad:

```

query-header-supported-pairwise-merkle-bad s fr f -fri-roots f -final as
 dg *composition-fri-roots* final
obtains *trace-table* *composition-table* *final-state* *trace-table'*
composition-table' *final-state'*
where
query-header-supported-table-candidate-state s fr f -fri-roots f -final
 as dg *composition-fri-roots* final *trace-table* *composition-table*
final-state
query-header-supported-table-candidate-state s fr f -fri-roots f -final
 as dg *composition-fri-roots* final *trace-table'* *composition-table'*
final-state'
trace-table $\neq$ *trace-table'* $\vee$
composition-table $\neq$ *composition-table'*
merkle-hash-value-conflict *final-state* *final-state'* $\vee$
hash-map-output-collision
 (*merkle-hash-state-merge* *final-state* *final-state'*)
 $s \leq$ *final-state*
 $s \leq$ *final-state'*
proof –
from *bad* **obtain** *trace-table* *composition-table* *final-state* *trace-table'*
composition-table' *final-state'* **where**
candidate:
query-header-supported-table-candidate-state s fr f -fri-roots f -final
 as dg *composition-fri-roots* final *trace-table* *composition-table*
final-state
and *candidate'*:
query-header-supported-table-candidate-state s fr f -fri-roots f -final
 as dg *composition-fri-roots* final *trace-table'* *composition-table'*
final-state'
and *distinct*:
trace-table $\neq$ *trace-table'* $\vee$
composition-table $\neq$ *composition-table'*
and *pair-bad*:
merkle-hash-value-conflict *final-state* *final-state'* $\vee$
hash-map-output-collision
 (*merkle-hash-state-merge* *final-state* *final-state'*)
unfolding *query-header-supported-pairwise-merkle-bad-def* **by** *blast*
have *ext*: $s \leq$ *final-state*
by (rule *query-header-supported-table-candidate-state-hash-extends*
 [OF *candidate*])
have *ext'*: $s \leq$ *final-state'*
by (rule *query-header-supported-table-candidate-state-hash-extends*
 [OF *candidate'*])
show *?thesis*
by (rule *that*[OF *candidate* *candidate'* *distinct* *pair-bad* *ext* *ext'*])
qed

definition *query-header-supported-pairwise-coupling-bad*
 $:: (f, 'a)$ *protocol-channel-scheme* $\Rightarrow$

'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$
'f list $\Rightarrow$ 'f $\Rightarrow$ bool

where

query-header-supported-pairwise-coupling-bad s fr f-fri-roots f-final as
dg composition-fri-roots final $\longleftrightarrow$
($\exists$ trace-table composition-table final-state trace-table'
composition-table' final-state'.
query-header-supported-table-candidate-state s fr f-fri-roots f-final
as dg composition-fri-roots final trace-table composition-table
final-state $\wedge$
query-header-supported-table-candidate-state s fr f-fri-roots f-final
as dg composition-fri-roots final trace-table' composition-table'
final-state' $\wedge$
(trace-table $\neq$ trace-table' $\vee$
composition-table $\neq$ composition-table') $\wedge$
merkle-hash-pairwise-coupling-bad final-state final-state')

definition query-supported-pairwise-coupling-bad

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$ bool

where

query-supported-pairwise-coupling-bad s $\longleftrightarrow$
($\exists$ fr f-fri-roots f-final as dg composition-fri-roots final.
query-header-supported-pairwise-coupling-bad s fr f-fri-roots f-final
as dg composition-fri-roots final)

lemma query-header-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad:

assumes bad:

query-header-supported-pairwise-merkle-bad s fr f-fri-roots f-final as
dg composition-fri-roots final

shows

($\exists$ trace-table composition-table final-state trace-table'
composition-table' final-state'.
query-header-supported-table-candidate-state s fr f-fri-roots f-final
as dg composition-fri-roots final trace-table composition-table
final-state $\wedge$
query-header-supported-table-candidate-state s fr f-fri-roots f-final
as dg composition-fri-roots final trace-table' composition-table'
final-state' $\wedge$
(trace-table $\neq$ trace-table' $\vee$
composition-table $\neq$ composition-table') $\wedge$
(hash-map-output-collision final-state $\vee$
hash-map-output-collision final-state')) $\vee$
query-header-supported-pairwise-coupling-bad s fr f-fri-roots f-final as
dg composition-fri-roots final

proof —

from bad **obtain** trace-table composition-table final-state trace-table'
composition-table' final-state'

where candidate:

query-header-supported-table-candidate-state s fr f-fri-roots f-final

```

    as dg composition-fri-roots final trace-table composition-table
    final-state
  and candidate':
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table' composition-table'
    final-state'
  and distinct:
    trace-table  $\neq$  trace-table'  $\vee$ 
    composition-table  $\neq$  composition-table'
  and pair-bad:
    merkle-hash-value-conflict final-state final-state'  $\vee$ 
    hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
  unfolding query-header-supported-pairwise-merkle-bad-def by blast
have decomp:
  hash-map-output-collision final-state  $\vee$ 
  hash-map-output-collision final-state'  $\vee$ 
  merkle-hash-pairwise-coupling-bad final-state final-state'
by (rule merkle-hash-pairwise-bad-imp-local-collision-or-coupling-bad
  [OF pair-bad])
show ?thesis
  using candidate candidate' distinct decomp
  unfolding query-header-supported-pairwise-coupling-bad-def by blast
qed

```

lemma *query-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad:*

assumes *bad: query-supported-pairwise-merkle-bad s*

shows

```

  ( $\exists$  fr f-fri-roots f-final as dg composition-fri-roots final
    trace-table composition-table final-state trace-table'
    composition-table' final-state'.
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table composition-table
    final-state  $\wedge$ 
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table' composition-table'
    final-state'  $\wedge$ 
    (trace-table  $\neq$  trace-table'  $\vee$ 
     composition-table  $\neq$  composition-table')  $\wedge$ 
    (hash-map-output-collision final-state  $\vee$ 
     hash-map-output-collision final-state'))  $\vee$ 
    query-supported-pairwise-coupling-bad s

```

proof —

from *bad* **obtain** *fr f-fri-roots f-final as dg composition-fri-roots final*

where *header-bad:*

```

  query-header-supported-pairwise-merkle-bad s fr f-fri-roots f-final
  as dg composition-fri-roots final

```

unfolding *query-supported-pairwise-merkle-bad-def* **by** *blast*

from

```

    query-header-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad
      [OF header-bad]
  show ?thesis
    unfolding query-supported-pairwise-coupling-bad-def by blast
qed

lemma query-supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad:
  assumes bad: query-supported-pairwise-merkle-bad s
  shows
    supported-hash-output-collision-possible s ∨
    query-supported-pairwise-coupling-bad s
proof -
  have decomp:
    (∃ fr f-fri-roots f-final as dg composition-fri-roots final
      trace-table composition-table final-state trace-table'
      composition-table' final-state'.
      query-header-supported-table-candidate-state s fr f-fri-roots f-final
        as dg composition-fri-roots final trace-table composition-table
        final-state ∧
      query-header-supported-table-candidate-state s fr f-fri-roots f-final
        as dg composition-fri-roots final trace-table' composition-table'
        final-state' ∧
      (trace-table ≠ trace-table' ∨
        composition-table ≠ composition-table') ∧
      (hash-map-output-collision final-state ∨
        hash-map-output-collision final-state')) ∨
      query-supported-pairwise-coupling-bad s
  by (rule query-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad
      [OF bad])
  then show ?thesis
proof
  assume local:
    ∃ fr f-fri-roots f-final as dg composition-fri-roots final
      trace-table composition-table final-state trace-table'
      composition-table' final-state'.
      query-header-supported-table-candidate-state s fr f-fri-roots f-final
        as dg composition-fri-roots final trace-table composition-table
        final-state ∧
      query-header-supported-table-candidate-state s fr f-fri-roots f-final
        as dg composition-fri-roots final trace-table' composition-table'
        final-state' ∧
      (trace-table ≠ trace-table' ∨
        composition-table ≠ composition-table') ∧
      (hash-map-output-collision final-state ∨
        hash-map-output-collision final-state')
  then obtain fr f-fri-roots f-final as dg composition-fri-roots final
    trace-table composition-table final-state trace-table'
    composition-table' final-state' where
    candidate:

```

```

    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table composition-table
    final-state
  and candidate':
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table' composition-table'
    final-state'
  and collision:
    hash-map-output-collision final-state  $\vee$ 
    hash-map-output-collision final-state'
  by blast
from collision show ?thesis
proof
  assume collision-final: hash-map-output-collision final-state
  from candidate obtain result query-idxs rest where
    outcome:
      Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
  and bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
  unfolding query-header-supported-table-candidate-state-def by blast
have supported-hash-output-collision-possible s
  by (rule
    accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible
    [OF outcome bound collision-final])
  then show ?thesis by simp
next
  assume collision-final': hash-map-output-collision final-state'
  from candidate' obtain result' query-idxs' rest' where
    outcome':
      Some (result', final-state')  $\in$  set-dist (execute verify-monad s)
  and bound':
    accepted-with-bound-tables s (Some (result', final-state'))
    trace-table' composition-table' as query-idxs'
  unfolding query-header-supported-table-candidate-state-def by blast
have supported-hash-output-collision-possible s
  by (rule
    accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible
    [OF outcome' bound' collision-final'])
  then show ?thesis by simp
qed
next
  assume query-supported-pairwise-coupling-bad s
  then show ?thesis by simp
qed
qed

lemma query-header-supported-candidate-unique-if-no-pairwise-merkle-bad:
  assumes no-bad:

```

$\neg$ *query-header-supported-pairwise-merkle-bad* *s* *fr* *f-fri-roots* *f-final*
as dg composition-fri-roots final
shows
 $\exists$ *trace-table* *composition-table*.
query-header-supported-table-candidates *s* *fr* *f-fri-roots* *f-final* *as dg*
composition-fri-roots final $\subseteq$
 $\{(trace-table, composition-table)\}$
proof (*rule ccontr*)
assume *not-unique*:
 $\neg$ ($\exists$ *trace-table* *composition-table*.
query-header-supported-table-candidates *s* *fr* *f-fri-roots* *f-final* *as dg*
composition-fri-roots final $\subseteq$
 $\{(trace-table, composition-table)\}$)
have *query-header-supported-pairwise-merkle-bad* *s* *fr* *f-fri-roots* *f-final*
as dg composition-fri-roots final
using
query-header-supported-table-candidates-not-singleton-imp-pairwise-merkle-bad
[*OF not-unique*]
unfolding *query-header-supported-pairwise-merkle-bad-def*
by *blast*
then show *False*
using *no-bad* **by** *contradiction*
qed

lemma *query-header-supported-candidate-unique-if-no-global-pairwise-merkle-bad*:
assumes *no-bad*: $\neg$ *query-supported-pairwise-merkle-bad* *s*
shows
 $\exists$ *trace-table* *composition-table*.
query-header-supported-table-candidates *s* *fr* *f-fri-roots* *f-final* *as dg*
composition-fri-roots final $\subseteq$
 $\{(trace-table, composition-table)\}$
proof (*rule query-header-supported-candidate-unique-if-no-pairwise-merkle-bad*)
show $\neg$ *query-header-supported-pairwise-merkle-bad* *s* *fr* *f-fri-roots*
f-final *as dg composition-fri-roots final*
using *no-bad* **unfolding** *query-supported-pairwise-merkle-bad-def* **by** *blast*
qed

definition *query-header-supported-union-good-sets*
 $:: (f, 'a)$ *protocol-channel-scheme* $\Rightarrow$
 $(f\ list \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow nat\ set) \Rightarrow$
 $'f \Rightarrow 'f\ list \Rightarrow 'f \Rightarrow 'f\ list \Rightarrow 'f \Rightarrow$
 $'f\ list \Rightarrow 'f \Rightarrow nat\ set$
where
query-header-supported-union-good-sets *s* *good-sets* *fr* *f-fri-roots*
f-final *as dg composition-fri-roots final* =
 $\{idx \in query-sample-space.$
 $\exists$ *trace-table* *composition-table*.
 $(trace-table, composition-table) \in$
query-header-supported-table-candidates *s* *fr* *f-fri-roots* *f-final*

$as \text{ dg composition-fri-roots final} \wedge$
 $idx \in \text{good-sets trace-table composition-table as}\}$

lemma *query-header-supported-union-good-sets-subset:*
query-header-supported-union-good-sets s good-sets fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$ query-sample-space
unfolding *query-header-supported-union-good-sets-def* **by** *auto*

definition *query-header-rounds-any-index-set-hit*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat set}) \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

query-header-rounds-any-index-set-hit s B out $\longleftrightarrow$
 $(\exists \text{ fr f-fri-roots f-final as dg composition-fri-roots final rest.}$
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest $\wedge$
query-rounds-any-index-set-hit s
 $(B \text{ fr f-fri-roots f-final as dg composition-fri-roots final})$
rounds out)

lemma *query-header-supported-union-good-sets-fraction-bound-if-unique-candidate:*

assumes *unique:*

$\exists \text{ trace-table composition-table.}$
query-header-supported-table-candidates s fr f-fri-roots f-final as
dg composition-fri-roots final $\subseteq$
 $\{(\text{trace-table, composition-table})\}$

and *subset:*

$\bigwedge \text{ trace-table composition-table as.}$
good-sets trace-table composition-table as $\subseteq$ query-sample-space

and *bound:*

$\bigwedge \text{ trace-table composition-table as.}$
 $\text{nnreal} (\text{card} (\text{good-sets trace-table composition-table as})) /$
 $\text{nnreal} (\text{card query-sample-space}) \leq \text{query-error-bound}$

shows

nnreal
 $(\text{card}$
 $(\text{query-header-supported-union-good-sets s good-sets fr f-fri-roots}$
 $\text{f-final as dg composition-fri-roots final})) /$
 $\text{nnreal} (\text{card query-sample-space}) \leq \text{query-error-bound}$

proof –

from *unique* **obtain** *trace-table composition-table* **where** *candidates:*

query-header-supported-table-candidates s fr f-fri-roots f-final as dg
composition-fri-roots final $\subseteq$
 $\{(\text{trace-table, composition-table})\}$

by *blast*

have *union-subset:*

query-header-supported-union-good-sets s good-sets fr f-fri-roots

```

    f-final as dg composition-fri-roots final  $\subseteq$ 
    good-sets trace-table composition-table as
unfolding query-header-supported-union-good-sets-def
using candidates by auto
have finite-good: finite (good-sets trace-table composition-table as)
by (rule finite-subset[OF subset finite-query-sample-space])
have card-le:
  card
    (query-header-supported-union-good-sets s good-sets fr f-fri-roots
     f-final as dg composition-fri-roots final)  $\leq$ 
    card (good-sets trace-table composition-table as)
by (rule card-mono[OF finite-good union-subset])
have nnreal
  (card
    (query-header-supported-union-good-sets s good-sets fr f-fri-roots
     f-final as dg composition-fri-roots final)) /
  nnreal (card query-sample-space)  $\leq$ 
  nnreal (card (good-sets trace-table composition-table as)) /
  nnreal (card query-sample-space)
by (rule nnreal-nat-divide-right-mono[OF card-le])
also have ...  $\leq$  query-error-bound
by (rule bound)
finally show ?thesis .
qed

lemma query-header-supported-union-good-sets-fraction-bound-if-no-pairwise-merkle-bad:
assumes no-bad:
   $\neg$  query-header-supported-pairwise-merkle-bad s fr f-fri-roots
  f-final as dg composition-fri-roots final
and subset:
   $\bigwedge$  trace-table composition-table as.
  good-sets trace-table composition-table as  $\subseteq$  query-sample-space
and bound:
   $\bigwedge$  trace-table composition-table as.
  nnreal (card (good-sets trace-table composition-table as)) /
  nnreal (card query-sample-space)  $\leq$  query-error-bound
shows
  nnreal
    (card
      (query-header-supported-union-good-sets s good-sets fr f-fri-roots
       f-final as dg composition-fri-roots final)) /
    nnreal (card query-sample-space)  $\leq$  query-error-bound
proof –
have unique:
   $\exists$  trace-table composition-table.
  query-header-supported-table-candidates s fr f-fri-roots f-final as dg
  composition-fri-roots final  $\subseteq$ 
  {(trace-table, composition-table)}
by (rule query-header-supported-candidate-unique-if-no-pairwise-merkle-bad

```

```

    [OF no-bad])
  show ?thesis
  by (rule query-header-supported-union-good-sets-fraction-bound-if-unique-candidate
      [OF unique subset bound])
qed

lemma query-header-supported-union-good-sets-fraction-bound-if-no-global-pairwise-merkle-bad:
  assumes no-bad:  $\neg$  query-supported-pairwise-merkle-bad s
  and subset:
     $\bigwedge$  trace-table composition-table as.
      good-sets trace-table composition-table as  $\subseteq$  query-sample-space
  and bound:
     $\bigwedge$  trace-table composition-table as.
      nnreal (card (good-sets trace-table composition-table as)) /
        nnreal (card query-sample-space)  $\leq$  query-error-bound
  shows
    nnreal
      (card
        (query-header-supported-union-good-sets s good-sets fr f-fri-roots
          f-final as dg composition-fri-roots final)) /
        nnreal (card query-sample-space)  $\leq$  query-error-bound
  proof -
  have no-header-bad:
     $\neg$  query-header-supported-pairwise-merkle-bad s fr f-fri-roots
      f-final as dg composition-fri-roots final
  using no-bad unfolding query-supported-pairwise-merkle-bad-def by blast
  show ?thesis
  by (rule
      query-header-supported-union-good-sets-fraction-bound-if-no-pairwise-merkle-bad
      [OF no-header-bad subset bound])
qed

definition query-header-committed-table-candidates
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    'f  $\Rightarrow$  'f list  $\Rightarrow$  ('f list  $\times$  'f list) set
  where
    query-header-committed-table-candidates s fr composition-fri-roots =
      {(trace-table, composition-table).
        merkle-root-binds-table fr trace-table s  $\wedge$ 
        composition-fri-roots  $\neq$  []  $\wedge$ 
        merkle-root-binds-table (hd composition-fri-roots) composition-table
        s}

definition query-header-committed-union-good-sets
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    ('f list  $\Rightarrow$  'f list  $\Rightarrow$  'f list  $\Rightarrow$  nat set)  $\Rightarrow$ 
    'f  $\Rightarrow$  'f list  $\Rightarrow$  'f list  $\Rightarrow$  nat set
  where
    query-header-committed-union-good-sets s good-sets fr as

```

$\text{composition-fri-roots} =$
 $\{ \text{idx} \in \text{query-sample-space}.$
 $\quad \exists \text{ trace-table composition-table}.$
 $\quad (\text{trace-table}, \text{composition-table}) \in$
 $\quad \text{query-header-committed-table-candidates } s \text{ fr}$
 $\quad \text{composition-fri-roots} \wedge$
 $\quad \text{idx} \in \text{good-sets trace-table composition-table as} \}$

lemma *query-header-committed-union-good-sets-subset:*
query-header-committed-union-good-sets s good-sets fr as
composition-fri-roots $\subseteq$ query-sample-space
unfolding *query-header-committed-union-good-sets-def* **by** *auto*

lemma *query-header-committed-union-good-sets-fraction-bound-if-unique-candidate:*
assumes *unique:*
 $\quad \exists \text{ trace-table composition-table}.$
 $\quad \text{query-header-committed-table-candidates } s \text{ fr composition-fri-roots}$
 $\quad \subseteq \{ (\text{trace-table}, \text{composition-table}) \}$
and *subset:*
 $\quad \bigwedge \text{ trace-table composition-table as}.$
 $\quad \text{good-sets trace-table composition-table as} \subseteq \text{query-sample-space}$
and *bound:*
 $\quad \bigwedge \text{ trace-table composition-table as}.$
 $\quad \text{nnreal (card (good-sets trace-table composition-table as)) /}$
 $\quad \text{nnreal (card query-sample-space)} \leq \text{query-error-bound}$
shows
 $\quad \text{nnreal}$
 $\quad (\text{card}$
 $\quad (\text{query-header-committed-union-good-sets } s \text{ good-sets fr as}$
 $\quad \text{composition-fri-roots})) /$
 $\quad \text{nnreal (card query-sample-space)} \leq \text{query-error-bound}$

proof –
from *unique* **obtain** *trace-table composition-table* **where** *candidates:*
 $\quad \text{query-header-committed-table-candidates } s \text{ fr composition-fri-roots}$
 $\quad \subseteq \{ (\text{trace-table}, \text{composition-table}) \}$
by *blast*
have *union-subset:*
 $\quad \text{query-header-committed-union-good-sets } s \text{ good-sets fr as}$
 $\quad \text{composition-fri-roots} \subseteq \text{good-sets trace-table composition-table as}$
unfolding *query-header-committed-union-good-sets-def*
using *candidates* **by** *auto*
have *finite-good:* *finite (good-sets trace-table composition-table as)*
by *(rule finite-subset[OF subset finite-query-sample-space])*
have *card-le:*
 $\quad \text{card}$
 $\quad (\text{query-header-committed-union-good-sets } s \text{ good-sets fr as}$
 $\quad \text{composition-fri-roots}) \leq$
 $\quad \text{card (good-sets trace-table composition-table as)}$
by *(rule card-mono[OF finite-good union-subset])*

```

have nnreal
  (card
    (query-header-committed-union-good-sets s good-sets fr as
      composition-fri-roots)) /
  nnreal (card query-sample-space) ≤
  nnreal (card (good-sets trace-table composition-table as)) /
  nnreal (card query-sample-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
also have ...  $\leq$  query-error-bound
  by (rule bound)
finally show ?thesis .
qed

lemma query-header-committed-table-candidates-subset-singleton-if-no-root-collisions:
assumes trace-clean:  $\neg$  merkle-root-binding-collision fr s
and comp-clean:
   $\bigwedge rt. \text{composition-fri-roots} \neq [] \implies$ 
   $rt = \text{hd composition-fri-roots} \implies$ 
   $\neg \text{merkle-root-binding-collision } rt \ s$ 
shows
   $\exists \text{trace-table composition-table.}$ 
   $\text{query-header-committed-table-candidates } s \text{ fr composition-fri-roots}$ 
   $\subseteq \{(\text{trace-table}, \text{composition-table})\}$ 
proof (cases
   $\text{query-header-committed-table-candidates } s \text{ fr composition-fri-roots} = \{\}$ )
case True
then show ?thesis
  by (intro exI[of - []] exI[of - []] simp)
next
case False
then obtain pair where pair-in:
   $\text{pair} \in$ 
   $\text{query-header-committed-table-candidates } s \text{ fr composition-fri-roots}$ 
by blast
then obtain trace-table composition-table where pair-eq:
   $\text{pair} = (\text{trace-table}, \text{composition-table})$ 
by (cases pair)
have candidate:
   $(\text{trace-table}, \text{composition-table}) \in$ 
   $\text{query-header-committed-table-candidates } s \text{ fr composition-fri-roots}$ 
using pair-in unfolding pair-eq .
have trace-bind:
  merkle-root-binds-table fr trace-table s
using candidate unfolding query-header-committed-table-candidates-def
by simp
have comp-nonempty: composition-fri-roots  $\neq []$ 
using candidate unfolding query-header-committed-table-candidates-def
by simp
have comp-bind:

```

```

merkle-root-binds-table (hd composition-fri-roots) composition-table s
using candidate unfolding query-header-committed-table-candidates-def
by simp
have candidates:
  query-header-committed-table-candidates s fr composition-fri-roots
     $\subseteq \{(trace-table, composition-table)\}$ 
proof
  fix pair
  assume pair-in:
    pair  $\in$ 
      query-header-committed-table-candidates s fr composition-fri-roots
  obtain trace-table' composition-table' where pair-eq:
    pair = (trace-table', composition-table')
    by (cases pair)
  have trace-bind':
    merkle-root-binds-table fr trace-table' s
    using pair-in unfolding pair-eq
      query-header-committed-table-candidates-def
    by simp
  have comp-bind':
    merkle-root-binds-table (hd composition-fri-roots) composition-table'
      s
    using pair-in unfolding pair-eq
      query-header-committed-table-candidates-def
    by simp
  have trace-eq: trace-table' = trace-table
    using merkle-root-binds-table-unique-if-no-collision
      [OF trace-clean trace-bind' trace-bind] by simp
  have comp-eq: composition-table' = composition-table
    using merkle-root-binds-table-unique-if-no-collision
      [OF comp-clean [OF comp-nonempty refl] comp-bind' comp-bind]
    by simp
  show pair  $\in \{(trace-table, composition-table)\}$ 
    unfolding pair-eq trace-eq comp-eq by simp
qed
show ?thesis
  using candidates by blast
qed

lemma query-index-round-set-hit-at-imp-query-header-supported-union-hit:
  assumes outcome: out  $\in$  set-dist (execute verify-monad s)
  and hit: query-index-round-set-hit-at s good-sets i out
  shows
     $\exists$  fr f-fri-roots f-final as dg composition-fri-roots final.
      query-rounds-any-index-set-hit s
        (query-header-supported-union-good-sets s good-sets fr f-fri-roots
          f-final as dg composition-fri-roots final)
        rounds out
proof –

```

```

from hit obtain trace-table composition-table as query-idxs where
  bound:
    accepted-with-bound-tables s out trace-table composition-table as
      query-idxs
  and i-bound: i < rounds
  and query-hit:
    query-idxs ! i ∈ good-sets trace-table composition-table as
  unfolding query-index-round-set-hit-at-def by blast
from bound obtain result final-state fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  out-eq: out = Some (result, final-state)
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  unfolding accepted-with-bound-tables-def by blast
have candidate:
  (trace-table, composition-table) ∈
    query-header-supported-table-candidates s fr f-fri-roots f-final as dg
    composition-fri-roots final
  unfolding query-header-supported-table-candidates-def
  using outcome bound header by blast
have len: length query-idxs = rounds
  by (rule accepted-with-bound-tables-shapes(4)[OF bound])
have idx-mem: query-idxs ! i ∈ set query-idxs
  by (rule nth-mem) (use i-bound len in simp)
have idx-sample: query-idxs ! i ∈ query-sample-space
  by (rule accepted-with-bound-tables-query-sample-space
    [OF bound idx-mem])
have union-hit:
  query-idxs ! i ∈
    query-header-supported-union-good-sets s good-sets fr f-fri-roots
    f-final as dg composition-fri-roots final
  unfolding query-header-supported-union-good-sets-def
  using idx-sample candidate query-hit by blast
have round-hit:
  query-rounds-any-index-set-hit s
  (query-header-supported-union-good-sets s good-sets fr f-fri-roots
    f-final as dg composition-fri-roots final)
  rounds out
  by (rule accepted-with-bound-tables-query-rounds-any-index-set-hit
    [OF bound i-bound union-hit])
show ?thesis
  using round-hit by blast
qed

lemma query-index-round-set-hit-at-imp-query-header-supported-rounds-hit:
assumes outcome: out ∈ set-dist (execute verify-monad s)
and hit: query-index-round-set-hit-at s good-sets i out
shows

```

```

    query-header-rounds-any-index-set-hit s
    (query-header-supported-union-good-sets s good-sets) out
proof –
from hit obtain trace-table composition-table as query-idxs where
    bound:
        accepted-with-bound-tables s out trace-table composition-table as
            query-idxs
    and i-bound: i < rounds
    and query-hit:
        query-idxs ! i ∈ good-sets trace-table composition-table as
    unfolding query-index-round-set-hit-at-def by blast
from bound obtain result final-state fr f-fri-roots f-final dg
    composition-fri-roots final rest where
    header:
        verifier-header-transcript s fr f-fri-roots f-final as dg
        composition-fri-roots final rest
    unfolding accepted-with-bound-tables-def by blast
from query-index-round-set-hit-at-imp-query-header-supported-union-hit
    [OF outcome hit]
obtain fr' f-fri-roots' f-final' as' dg' composition-fri-roots' final'
where round-hit':
    query-rounds-any-index-set-hit s
    (query-header-supported-union-good-sets s good-sets fr' f-fri-roots'
        f-final' as' dg' composition-fri-roots' final')
    rounds out
by blast
have candidate:
    (trace-table, composition-table) ∈
        query-header-supported-table-candidates s fr f-fri-roots f-final as dg
        composition-fri-roots final
    unfolding query-header-supported-table-candidates-def
    using outcome bound header by blast
have len: length query-idxs = rounds
    by (rule accepted-with-bound-tables-shapes(4)) [OF bound])
have idx-mem: query-idxs ! i ∈ set query-idxs
    by (rule nth-mem) (use i-bound len in simp)
have idx-sample: query-idxs ! i ∈ query-sample-space
    by (rule accepted-with-bound-tables-query-sample-space
        [OF bound idx-mem])
have union-hit:
    query-idxs ! i ∈
        query-header-supported-union-good-sets s good-sets fr f-fri-roots
        f-final as dg composition-fri-roots final
    unfolding query-header-supported-union-good-sets-def
    using idx-sample candidate query-hit by blast
have round-hit:
    query-rounds-any-index-set-hit s
    (query-header-supported-union-good-sets s good-sets fr f-fri-roots
        f-final as dg composition-fri-roots final)

```



```

    rounds out
  by (rule accepted-with-bound-tables-query-rounds-any-index-set-hit
      [OF bound i-bound union-hit])
  show ?thesis
    unfolding query-header-rounds-any-index-set-hit-def
    using header round-hit by blast
qed

lemma query-index-round-set-hit-imp-query-header-supported-rounds-hit:
  assumes outcome: out ∈ set-dist (execute verify-monad s)
  and hit: query-index-round-set-hit s good-sets out
  shows
    query-header-rounds-any-index-set-hit s
      (query-header-supported-union-good-sets s good-sets) out
  proof -
    from hit obtain i where
      hit-at: query-index-round-set-hit-at s good-sets i out
    unfolding query-index-round-set-hit-def by blast
  show ?thesis
    by (rule query-index-round-set-hit-at-imp-query-header-supported-rounds-hit
        [OF outcome hit-at])
qed

lemma query-index-set-hit-empty-queries:
  assumes bound:
    accepted-with-bound-tables s out trace-table composition-table as []
  shows query-index-set-hit s good-sets out
  unfolding query-index-set-hit-def
  apply (rule exI[where x=trace-table])
  apply (rule exI[where x=composition-table])
  apply (rule exI[where x=as])
  apply (rule exI[where x=[]])
  using bound by simp

lemma query-index-set-hit-vacuous-if-rounds-zero:
  assumes rounds-zero: rounds = 0
  and bound:
    accepted-with-bound-tables s out trace-table composition-table as query-idxs
  shows query-index-set-hit s good-sets out
  proof -
    have length query-idxs = 0
      using accepted-with-bound-tables-shapes(4)[OF bound] rounds-zero by simp
    then have query-idxs-empty: query-idxs = []
      by simp
    show ?thesis
      using bound unfolding query-idxs-empty
      by (rule query-index-set-hit-empty-queries)
  qed

```

lemma *query-index-set-hit-imp-round-set-hit*:
assumes *hit*: *query-index-set-hit s good-sets out*
shows *query-index-round-set-hit s good-sets out*
proof –
from *hit* **obtain** *trace-table composition-table as query-idxs* **where**
bound:
accepted-with-bound-tables s out trace-table composition-table as
query-idxs
and *subset*:
set query-idxs $\subseteq$ good-sets trace-table composition-table as
unfolding *query-index-set-hit-def* **by** *blast*
have *len*: *length query-idxs = rounds*
by (rule *accepted-with-bound-tables-shapes*(4)[OF *bound*])
have *zero-bound*: *0 < length query-idxs*
using *len rounds-positive* **by** *simp*
have *nth-in*: *query-idxs ! 0 $\in$ set query-idxs*
by (rule *nth-mem*[OF *zero-bound*])
have *round-hit*:
query-index-round-set-hit-at s good-sets 0 out
unfolding *query-index-round-set-hit-at-def*
apply (rule *exI*[**where** *x=trace-table*])
apply (rule *exI*[**where** *x=composition-table*])
apply (rule *exI*[**where** *x=as*])
apply (rule *exI*[**where** *x=query-idxs*])
using *bound subset nth-in rounds-positive* **by** *auto*
show *?thesis*
unfolding *query-index-round-set-hit-def*
using *round-hit rounds-positive* **by** *blast*
qed

lemma *accepted-with-bound-tables-query-samples-miss-if-not-all-consistent*:
assumes *bound*:
accepted-with-bound-tables s out trace-table composition-table as query-idxs
and *not-all*:
 $\neg$ *all-queries-consistent trace-table composition-table as*
shows *query-samples-miss-disagreements trace-table composition-table as query-idxs*
proof –
have *disagreements*:
query-disagreement-indices trace-table composition-table as $\neq \{\}$
using *not-all query-disagreement-indices-nonempty-iff* **by** *blast*
have *samples-agree*:
set query-idxs $\subseteq$ query-agreement-indices trace-table composition-table as
proof
fix *idx*
assume *idx-in*: *idx $\in$ set query-idxs*
have *sample*: *idx $\in$ query-sample-space*
by (rule *accepted-with-bound-tables-query-sample-space*[OF *bound idx-in*])
have *consistent*:
query-consistent-at trace-table composition-table as idx

```

    by (rule accepted-with-bound-tables-query-consistency[OF bound idx-in])
  show  $idx \in \text{query-agreement-indices trace-table composition-table as}$ 
    using sample consistent unfolding query-agreement-indices-def by simp
qed
show ?thesis
  using disagreements samples-agree
  unfolding query-samples-miss-disagreements-def by simp
qed

lemma query-sampling-success-space-subset:
  query-sampling-success-space trace-table composition-table as  $\subseteq$  query-sample-space
  unfolding query-sampling-success-space-def query-agreement-indices-def by auto

lemma query-sampling-success-space-fraction-bound:
  shows
    nnreal (card (query-sampling-success-space trace-table composition-table as)) /
      nnreal query-sample-space-size  $\leq$  query-error-bound
  proof (cases
    trace-table-low-degree trace-table  $\wedge$ 
    composition-table-low-degree maxDegree composition-table  $\wedge$ 
     $\neg$  all-queries-consistent trace-table composition-table as)
  case True
  then have space-eq:
    query-sampling-success-space trace-table composition-table as =
      query-agreement-indices trace-table composition-table as
    unfolding query-sampling-success-space-def by simp
  have card-le:
    card (query-sampling-success-space trace-table composition-table as)  $\leq$ 
      query-agreement-bound
    unfolding space-eq
    by (rule query-agreement-indices-card-bound)
      (use True in auto)
  have nnreal (card (query-sampling-success-space trace-table composition-table as)) /
    nnreal query-sample-space-size  $\leq$ 
    nnreal query-agreement-bound / nnreal query-sample-space-size
    by (rule nnreal-nat-divide-right-mono[OF card-le])
  then show ?thesis
    unfolding query-error-bound-def .
next
  case False
  then have space-eq:
    query-sampling-success-space trace-table composition-table as = {}
    unfolding query-sampling-success-space-def by auto
  show ?thesis
    unfolding space-eq by simp
qed

lemma query-sampling-success-space-fraction-bound-query-sample-space:
  shows

```

$\text{nnreal } (\text{card } (\text{query-sampling-success-space trace-table composition-table as})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$
using *query-sampling-success-space-fraction-bound*[*of trace-table composition-table as*]
by *simp*

lemma *query-sampling-success-header-supported-union-fraction-bound-if-no-global-pairwise-merkle-bad*:
assumes *no-bad*: $\neg \text{query-supported-pairwise-merkle-bad } s$
shows
 $\text{nnreal } (\text{card } (\text{query-header-supported-union-good-sets } s \text{ query-sampling-success-space}$
 $\text{fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots final})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$
by (*rule*
query-header-supported-union-good-sets-fraction-bound-if-no-global-pairwise-merkle-bad
 $[OF \text{ no-bad query-sampling-success-space-subset}$
 $\text{query-sampling-success-space-fraction-bound-query-sample-space}]$)

definition *query-index-raw-preimage-bound* :: *bool*
where
 $\text{query-index-raw-preimage-bound} \longleftrightarrow$
 $(\forall B.$
 $B \subseteq \text{query-sample-space} \longrightarrow$
 $\text{nnreal } (\text{card } (\text{query-index-raw-preimage } B)) / \text{nnreal size} \leq$
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card query-sample-space}))$

lemma *query-index-raw-preimage-bound-if-uniform-range*:
assumes *range-eq*: $\text{range to-nat} = \{0..<\text{size}\}$
and *dvd*: $\text{query-sample-space-size} \text{ dvd } \text{size}$
shows *query-index-raw-preimage-bound*
unfolding *query-index-raw-preimage-bound-def*
proof (*intro allI impI*)
fix *B* :: *nat set*
assume *subset*: $B \subseteq \text{query-sample-space}$
let *?q* = *query-sample-space-size*
have *card-raw*:
 $\text{card } (\text{query-index-raw-preimage } B) =$
 $(\text{size div } ?q) * \text{card } B$
unfolding *card-query-index-raw-preimage-eq-nat-preimage*
by (*rule card-query-index-nat-preimage-uniform-range*
 $[OF \text{ range-eq dvd subset}]$)
have *size-eq*: $\text{size} = (\text{size div } ?q) * ?q$
using *dvd query-sample-space-size-pos* **by** (*simp add: dvd-eq-mod-eq-0*)
have *size-pos*: $0 < \text{size}$
using *size-card* **by** *simp*
have *div-pos*: $0 < \text{size div } ?q$
proof (*cases size div ?q*)
case 0
then show *?thesis*

```

    using size-eq size-pos by simp
next
case (Suc n)
then show ?thesis
  by simp
qed
have frac-eq:
  nnreal ((size div ?q) * card B) / nnreal ((size div ?q) * ?q) =
    nnreal (card B) / nnreal ?q
  using div-pos query-sample-space-size-pos
  by transfer (simp add: field-simps)
have lhs-eq:
  nnreal (card (query-index-raw-preimage B)) / nnreal size =
    nnreal ((size div ?q) * card B) / nnreal ((size div ?q) * ?q)
  using card-raw size-eq by simp
show
  nnreal (card (query-index-raw-preimage B)) / nnreal size ≤
    nnreal (card B) / nnreal (card query-sample-space)
  using lhs-eq frac-eq by simp
qed

lemma query-index-raw-preimage-bound-from-sampler-wellformed:
  query-index-raw-preimage-bound
proof (rule query-index-raw-preimage-bound-if-uniform-range)
  show range to-nat = {0..

```

assumes *raw-bound: query-index-raw-preimage-bound*
and *subset: $B \subseteq \text{query-sample-space}$*
and *frac:*
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space}) \leq$
 query-error-bound
shows
 $\text{nnreal } (\text{card } (\text{query-index-raw-preimage } B)) / \text{nnreal size} \leq$
 query-error-bound
proof –
have $\text{nnreal } (\text{card } (\text{query-index-raw-preimage } B)) / \text{nnreal size} \leq$
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space})$
using *raw-bound subset* **unfolding** *query-index-raw-preimage-bound-def* **by** *blast*
also have $\dots \leq \text{query-error-bound}$
by (*rule frac*)
finally show *?thesis* .
qed

lemma *wp-receive-query-index-challenge-fresh-index-set-raw-bound:*

assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *subset: $B \subseteq \text{query-sample-space}$*

shows

wp-event receive-query-index-challenge
 $(\lambda \text{out. case out of None} \Rightarrow \text{False}$
 $\mid \text{Some } (\text{raw}, -) \Rightarrow \text{index } (\text{to-nat raw}) \in B) \text{ } s \leq$
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space})$

proof –

have *fresh:*
 $\text{fmlookup } (\text{HashMap } s)$
 $(\text{QueryIndexChallenge } (\text{PQueryCounter } s) (\text{PState } s)) = \text{None}$
by (*rule query-future-fresh-current[OF future]*)

have *exact:*

wp-event receive-query-index-challenge
 $(\lambda \text{out. case out of None} \Rightarrow \text{False}$
 $\mid \text{Some } (\text{raw}, -) \Rightarrow \text{index } (\text{to-nat raw}) \in B) \text{ } s =$
 $\text{nnreal } (\text{card } (\text{query-index-raw-preimage } B)) / \text{nnreal size}$
by (*rule wp-receive-query-index-challenge-fresh-index-set[OF fresh]*)

have *raw:*

$\text{nnreal } (\text{card } (\text{query-index-raw-preimage } B)) / \text{nnreal size} \leq$
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space})$

using *raw-bound subset*

unfolding *query-index-raw-preimage-bound-def* **by** *blast*

show *?thesis*

unfolding *exact* **by** (*rule raw*)

qed

lemma *wp-receive-query-index-challenge-fresh-index-set-query-error-bound:*

assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*

```

and subset:  $B \subseteq \text{query-sample-space}$ 
and frac:
   $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space}) \leq$ 
   $\text{query-error-bound}$ 
shows
   $\text{wp-event receive-query-index-challenge}$ 
   $(\lambda \text{out. case out of None} \Rightarrow \text{False}$ 
     $| \text{Some } (\text{raw}, -) \Rightarrow \text{index } (\text{to-nat raw}) \in B) \ s \leq$ 
     $\text{query-error-bound}$ 
by (rule order-trans
  [OF wp-receive-query-index-challenge-fresh-index-set-row-bound
  [OF future raw-bound subset] frac])

lemma wp-verifier-query-round-program-index-set-bound:
assumes future: query-future-fresh  $s$ 
and raw-bound: query-index-raw-preimage-bound
and subset:  $B \subseteq \text{query-sample-space}$ 
shows
   $\text{wp-event}$ 
   $(\text{verifier-query-round-program fr f-fl f-final as fl final})$ 
   $(\text{query-round-index-set-hit } s \ B) \ s \leq$ 
   $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space})$ 
proof -
let ?P =
   $\lambda \text{out. case out of None} \Rightarrow \text{False}$ 
   $| \text{Some } (\text{raw}, -) \Rightarrow \text{index } (\text{to-nat raw}) \in B$ 
have head-bound:
   $\text{wp-event receive-query-index-challenge } ?P \ s \leq$ 
   $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space})$ 
by (rule wp-receive-query-index-challenge-fresh-index-set-row-bound
  [OF future raw-bound subset])
show ?thesis
unfolding verifier-query-round-program-alt-def
proof (rule wp-event-bind-bound-by-head-event[OF head-bound])
show query-round-index-set-hit  $s \ B \ \text{None} \Longrightarrow ?P \ \text{None}$ 
unfolding query-round-index-set-hit-def by simp
next
fix raw s0 out
assume head:
   $\text{Some } (\text{raw}, s0) \in$ 
   $\text{set-dist } (\text{execute receive-query-index-challenge } s)$ 
and tail:
   $\text{out} \in$ 
   $\text{set-dist}$ 
   $(\text{execute}$ 
     $(\text{verifier-query-round-after-index-program fr f-fl f-final as fl}$ 
       $\text{final raw}) \ s0)$ 
and hit: query-round-index-set-hit  $s \ B \ \text{out}$ 
from hit obtain raw' t where out-eq: out = Some (( ), t)

```

```

and lookup-t:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw'
and raw'-in: index (to-nat raw') ∈ B
unfolding query-round-index-set-hit-def by blast
have tail-out:
  Some ((), t) ∈
    set-dist
    (execute
      (verifier-query-round-after-index-program fr f-fl f-final as fl
        final raw) s0)
  using tail out-eq by simp
have ext-s0-t: s0 ≤ t
  using verifier-query-round-after-index-program-outcome[OF tail-out]
  by simp
have lookup-s0:
  fmlookup (HashMap s0)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using receive-query-index-challenge-outcome[OF head] by blast
have lookup-t-row:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  by (rule hash-extension-lookup[OF lookup-s0 ext-s0-t])
have raw = raw'
  using lookup-t-row lookup-t by simp
then show ?P (Some (raw, s0))
  using raw'-in by simp
qed
qed

lemma wp-verifier-query-round-program-index-set-query-error-bound:
assumes future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and subset: B ⊆ query-sample-space
and frac:
  nnreal (card B) / nnreal (card query-sample-space) ≤
    query-error-bound
shows
  wp-event
    (verifier-query-round-program fr f-fl f-final as fl final)
    (query-round-index-set-hit s B) s ≤ query-error-bound
by (rule order-trans
  [OF wp-verifier-query-round-program-index-set-bound
    [OF future raw-bound subset] frac])

lemma wp-verifier-query-round-program-any-index-set-bound:
assumes future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and subset: B ⊆ query-sample-space

```



```

shows
  wp-event
    (verifier-query-round-program fr f-fl f-final as fl final)
    (query-round-any-index-set-hit s B) s ≤
    nnreal (card B) / nnreal (card query-sample-space)
proof -
  let ?P =
    λout. case out of None ⇒ False
    | Some (raw, -) ⇒ index (to-nat raw) ∈ B
  have head-bound:
    wp-event receive-query-index-challenge ?P s ≤
    nnreal (card B) / nnreal (card query-sample-space)
  by (rule wp-receive-query-index-challenge-fresh-index-set-row-bound
    [OF future raw-bound subset])
  show ?thesis
    unfolding verifier-query-round-program-alt-def
  proof (rule wp-event-bind-bound-by-head-event[OF head-bound])
  show query-round-any-index-set-hit s B None ⇒ ?P None
    unfolding query-round-any-index-set-hit-def by simp
  next
  fix raw s0 out
  assume head:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
  and tail:
    out ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program fr f-fl f-final as fl
            final raw) s0)
  and hit: query-round-any-index-set-hit s B out
  from hit obtain raw' t x where out-eq: out = Some ((), t)
  and lookup-t:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) x) = Some raw'
  and raw'-in: index (to-nat raw') ∈ B
  unfolding query-round-any-index-set-hit-def by blast
  have tail-out:
    Some ((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program fr f-fl f-final as fl
            final raw) s0)
  using tail out-eq by simp
  have lookup-s0-x:
    fmlookup (HashMap s0)
      (QueryIndexChallenge (PQueryCounter s) x) = Some raw'
  using verifier-query-round-after-index-program-preserves-query-lookup
    [OF tail-out, of PQueryCounter s x]

```

```

    lookup-t
  by simp
have lookup-current:
  fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using receive-query-index-challenge-outcome[OF head] by blast
have key-eq:
  QueryIndexChallenge (PQueryCounter s) x =
    QueryIndexChallenge (PQueryCounter s) (PState s)
proof (rule ccontr)
  assume neg:
    QueryIndexChallenge (PQueryCounter s) x ≠
      QueryIndexChallenge (PQueryCounter s) (PState s)
  have lookup-s0-x':
    fmlookup (HashMap s0)
      (QueryIndexChallenge (PQueryCounter s) x) =
        fmlookup (HashMap s)
          (QueryIndexChallenge (PQueryCounter s) x)
    by (rule receive-query-index-challenge-preserves-other-lookup
        [OF head neg])
  have fmlookup (HashMap s)
    (QueryIndexChallenge (PQueryCounter s) x) = None
    using future unfolding query-future-fresh-def by simp
  then show False
    using lookup-s0-x lookup-s0-x' by simp
qed
have raw' = raw
  using lookup-s0-x lookup-current key-eq by simp
then show ?P (Some (raw, s0))
  using raw'-in by simp
qed
qed

lemma wp-ntimes-verifier-query-round-program-any-index-set-bound:
  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and subset:  $B \subseteq \text{query-sample-space}$ 
  shows
    wp-event
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n)
      (query-rounds-any-index-set-hit s B n)  $s \leq$ 
      nnreal n *
        (nnreal (card B) / nnreal (card query-sample-space))
  using future
proof (induction n arbitrary: s)
  case 0
  show ?case
    unfolding query-rounds-any-index-set-hit-def
    by (simp add: wp-event-def wpsimps)

```

```

next
  case (Suc n)
  let ?prog = verifier-query-round-program fr f-fl f-final as fl final
  let ?C = nnreal (card B) / nnreal (card query-sample-space)
  let ?Q = query-rounds-any-index-set-hit s B (Suc n)
  let ?Head =
    λout :: (unit list × ('f, 'a) protocol-channel-scheme) option.
      case out of None ⇒ False
      | Some (-, t) ⇒
        (∃ raw x.
          fmlookup (HashMap t)
            (QueryIndexChallenge (PQueryCounter s) x) = Some raw ∧
            index (to-nat raw) ∈ B)
  let ?Tail =
    λout :: (unit list × ('f, 'a) protocol-channel-scheme) option.
      case out of None ⇒ False
      | Some (-, t) ⇒
        (∃ i < n. ∃ x raw.
          fmlookup (HashMap t)
            (QueryIndexChallenge (PQueryCounter s + Suc i) x) =
              Some raw ∧
              index (to-nat raw) ∈ B)
  have split-hit:
    ∧out. ?Q out ⇒ ?Head out ∨ ?Tail out
  proof -
    fix out
    assume ?Q out
    then obtain results t i x raw where out-eq: out = Some (results, t)
      and i-bound: i < Suc n
      and lookup:
        fmlookup (HashMap t)
          (QueryIndexChallenge (PQueryCounter s + i) x) = Some raw
      and raw-in: index (to-nat raw) ∈ B
    unfolding query-rounds-any-index-set-hit-def by blast
    show ?Head out ∨ ?Tail out
  proof (cases i)
    case 0
    then have ?Head out
      using out-eq lookup raw-in by simp blast
    then show ?thesis by simp
  next
    case (Suc j)
    then have j-bound: j < n
      using i-bound by simp
    have ?Tail out
      using out-eq lookup raw-in j-bound Suc by simp blast
    then show ?thesis by simp
  qed
qed

```

```

have head-bound:
  wp-event (ntimes ?prog (Suc n)) ?Head s ≤ ?C
proof -
  have one-bound:
    wp-event ?prog (query-round-any-index-set-hit s B) s ≤ ?C
    by (rule wp-verifier-query-round-program-any-index-set-bound
        [OF Suc.premis raw-bound subset])
  show ?thesis
    unfolding ntimes.simps
  proof (rule wp-event-bind-bound-by-head-event[OF one-bound])
    show ?Head None ⇒ query-round-any-index-set-hit s B None
      by simp
  next
    fix u s1 out
    assume head:
      Some (u, s1) ∈ set-dist (execute ?prog s)
    and tail:
      out ∈
        set-dist
          (execute
            (ntimes ?prog n ≫ (λxs. return (u # xs)))) s1)
    and hit: ?Head out
  have u-eq: u = ()
    by (cases u) simp
  from hit obtain ys0 t0 where out-some: out = Some (ys0, t0)
    by (cases out) auto
  from tail[unfolded out-some] obtain results t where
    tail-rounds:
      Some (results, t) ∈ set-dist (execute (ntimes ?prog n) s1)
    and out-ret:
      Some (ys0, t0) ∈ set-dist (execute (return (u # results)) t)
    by (rule set-dist-bindE)
  have out-eq: out = Some (u # results, t)
    using out-ret out-some
    by simp
  from hit obtain raw x where lookup-t:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) x) = Some raw
    and raw-in: index (to-nat raw) ∈ B
    using out-eq by auto
  have counter-s1: PQueryCounter s1 = Suc (PQueryCounter s)
    using verifier-query-round-program-outcome[OF head[unfolded u-eq]]
    by blast
  have past: PQueryCounter s < PQueryCounter s1
    using counter-s1 by simp
  have lookup-s1:
    fmlookup (HashMap s1)
      (QueryIndexChallenge (PQueryCounter s) x) = Some raw
    using ntimes-verifier-query-round-program-preserves-past-query-lookup

```

```

      [OF past tail-rounds, of x]
      lookup-t
    by simp
  show query-round-any-index-set-hit s B (Some (u, s1))
    unfolding query-round-any-index-set-hit-def
    using lookup-s1 raw-in u-eq by blast
qed
qed
have tail-bound:
  wp-event (ntimes ?prog (Suc n)) ?Tail s ≤ nnreal n * ?C
proof -
  show ?thesis
    unfolding ntimes.simps
  proof (rule wp-event-bind-bound-by-cont)
    show ¬ ?Tail None
      by simp
  next
    fix u s1
    assume head:
      Some (u, s1) ∈ set-dist (execute ?prog s)
    have u-eq: u = ()
      by (cases u) simp
    have future-s1: query-future-fresh s1
      by (rule verifier-query-round-program-preserves-query-future-fresh
        [OF Suc.prem head[unfolded u-eq]])
    have counter-s1: PQueryCounter s1 = Suc (PQueryCounter s)
      using verifier-query-round-program-outcome[OF head[unfolded u-eq]]
      by blast
    have tail-induct:
      wp-event (ntimes ?prog n)
        (query-rounds-any-index-set-hit s1 B n) s1 ≤
        nnreal n * ?C
      by (rule Suc.IH[OF future-s1])
    show wp-event
      (ntimes ?prog n ≫ (λxs. return (u # xs))) ?Tail s1
      ≤ nnreal n * ?C
    proof (rule wp-event-bind-bound-by-head-event[OF tail-induct])
      show ?Tail None ⇒ query-rounds-any-index-set-hit s1 B n None
        by simp
    next
      fix results t out
      assume tail-rounds:
        Some (results, t) ∈
          set-dist (execute (ntimes ?prog n) s1)
      and ret:
        out ∈ set-dist (execute (return (u # results)) t)
      and hit: ?Tail out
      have out-eq: out = Some (u # results, t)
        using ret

```

```

unfolding set-dist-def return.rep-eq dist-return-def
  dist-delta-dist delta-map-def
by simp
from hit obtain  $i$   $x$   $raw$  where  $i$ -bound:  $i < n$ 
and lookup:
  fmlookup (HashMap  $t$ )
  (QueryIndexChallenge (PQueryCounter  $s + Suc\ i$ )  $x$ ) =
    Some  $raw$ 
and raw-in:  $index\ (to\ nat\ raw) \in B$ 
using out-eq by auto
have key-eq:
   $Suc\ (PQueryCounter\ s + i) = PQueryCounter\ s1 + i$ 
using counter-s1 by presburger
have lookup':
  fmlookup (HashMap  $t$ )
  (QueryIndexChallenge (PQueryCounter  $s1 + i$ )  $x$ ) = Some  $raw$ 
using lookup key-eq by simp
show query-rounds-any-index-set-hit  $s1\ B\ n\ (Some\ (results,\ t))$ 
  unfolding query-rounds-any-index-set-hit-def
  apply (rule exI[where  $x=results$ ])
  apply (rule exI[where  $x=t$ ])
  apply (rule exI[where  $x=i$ ])
  apply (rule exI[where  $x=x$ ])
  apply (rule exI[where  $x=raw$ ])
  using  $i$ -bound lookup' raw-in by simp
qed
qed
qed
have wp-event ( $ntimes\ ?prog\ (Suc\ n)$ )  $?Q\ s \leq$ 
  wp-event ( $ntimes\ ?prog\ (Suc\ n)$ )
  ( $\lambda out.\ ?Head\ out \vee ?Tail\ out$ )  $s$ 
by (rule wp-event-mono) (use split-hit in blast)
also have  $\dots \leq wp\text{-event}\ (ntimes\ ?prog\ (Suc\ n))\ ?Head\ s +$ 
  wp-event ( $ntimes\ ?prog\ (Suc\ n)$ )  $?Tail\ s$ 
by (rule wp-event-union-bound)
also have  $\dots \leq ?C + nnreal\ n * ?C$ 
by (intro add-mono head-bound tail-bound)
also have  $\dots = nnreal\ (Suc\ n) * ?C$ 
by (simp add: algebra-simps)
finally show  $?case$  .
qed

lemma wp-ntimes-verifier-query-round-program-any-index-set-query-error-bound:
assumes future: query-future-fresh  $s$ 
and raw-bound: query-index-raw-preimage-bound
and subset:  $B \subseteq query\text{-sample-space}$ 
and frac:
   $nnreal\ (card\ B) / nnreal\ (card\ query\text{-sample-space}) \leq$ 
  query-error-bound

```

```

shows
  wp-event
    (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n)
    (query-rounds-any-index-set-hit s B n) s ≤
    nnreal n * query-error-bound
proof –
  have base:
    wp-event
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n)
      (query-rounds-any-index-set-hit s B n) s ≤
      nnreal n *
        (nnreal (card B) / nnreal (card query-sample-space))
  by (rule wp-ntimes-verifier-query-round-program-any-index-set-bound
    [OF future raw-bound subset])
  have mono:
    nnreal n *
      (nnreal (card B) / nnreal (card query-sample-space)) ≤
      nnreal n * query-error-bound
  by (rule mult-left-mono) (use frac in simp-all)
  show ?thesis
    by (rule order-trans[OF base mono])
qed

lemma wp-verifier-after-composition-fri-query-rounds-any-index-set-bound:
assumes future: query-future-fresh t
and counter: PQueryCounter t = PQueryCounter s
and raw-bound: query-index-raw-preimage-bound
and subset: B ⊆ query-sample-space
shows
  wp-event (verifier-after-composition-fri header)
    (query-rounds-any-index-set-hit s B rounds) t ≤
    nnreal rounds *
      (nnreal (card B) / nnreal (card query-sample-space))
proof –
  obtain fr f-fl f-final as dg fl where header-eq:
    header = (fr, f-fl, f-final, as, dg, fl)
  by (cases header) auto
  have after-eq:
    verifier-after-composition-fri header =
      (read >>=
        (λfinal.
          ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds))
  unfolding header-eq verifier-after-composition-fri-def by simp
  show ?thesis
    unfolding after-eq
  proof (rule wp-event-bind-bound-by-cont)
    show ¬ query-rounds-any-index-set-hit s B rounds None

```

```

    unfolding query-rounds-any-index-set-hit-def by simp
next
fix final query-state
assume read-final:
  Some (final, query-state) ∈ set-dist (execute read t)
have future-query: query-future-fresh query-state
  by (rule read-preserves-query-future-fresh[OF future read-final])
from read-outcome[OF read-final] obtain rest where
  counter-query-t: PQueryCounter query-state = PQueryCounter t
  by blast
have counter-query: PQueryCounter query-state = PQueryCounter s
  using counter-query-t counter by simp
have event-imp:
   $\bigwedge \text{out. query-rounds-any-index-set-hit } s \text{ } B \text{ rounds out} \implies$ 
    query-rounds-any-index-set-hit query-state  $B$  rounds out
  using counter-query
  unfolding query-rounds-any-index-set-hit-def by auto
have wp-event
  (ntimes
    (verifier-query-round-program fr f-fl f-final as fl final)
    rounds)
  (query-rounds-any-index-set-hit s  $B$  rounds) query-state  $\leq$ 
wp-event
  (ntimes
    (verifier-query-round-program fr f-fl f-final as fl final)
    rounds)
  (query-rounds-any-index-set-hit query-state  $B$  rounds) query-state
  by (rule wp-event-mono) (rule event-imp)
also have ...  $\leq$ 
  nnreal rounds *
  (nnreal (card  $B$ ) / nnreal (card query-sample-space))
  by (rule wp-ntimes-verifier-query-round-program-any-index-set-bound
    [OF future-query raw-bound subset])
finally show
  wp-event
  (ntimes
    (verifier-query-round-program fr f-fl f-final as fl final)
    rounds)
  (query-rounds-any-index-set-hit s  $B$  rounds) query-state  $\leq$ 
  nnreal rounds *
  (nnreal (card  $B$ ) / nnreal (card query-sample-space)) .
qed
qed

```

lemma *wp-verify-monad-query-rounds-any-index-set-bound:*
assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *subset: $B \subseteq \text{query-sample-space}$*
shows


```

wp-event verify-monad (query-rounds-any-index-set-hit s B rounds) s ≤
  nnreal rounds *
    (nnreal (card B) / nnreal (card query-sample-space))
unfolding verify-monad-composition-fri-decomposition
proof (rule wp-event-bind-bound-by-cont)
  show ¬ query-rounds-any-index-set-hit s B rounds None
    unfolding query-rounds-any-index-set-hit-def by simp
next
fix header t
assume prefix:
  Some (header, t) ∈
    set-dist (execute verifier-composition-fri-prefix s)
obtain fr f-fl f-final as dg fl where header-eq:
  header = (fr, f-fl, f-final, as, dg, fl)
by (cases header) auto
have future-t: query-future-fresh t
  by (rule verifier-composition-fri-prefix-preserves-query-future-fresh
    [OF future prefix])
have counter-t: PQueryCounter t = PQueryCounter s
  using verifier-composition-fri-prefix-outcome[OF prefix[unfolded header-eq]]
  by simp
show
  wp-event (verifier-after-composition-fri header)
    (query-rounds-any-index-set-hit s B rounds) t ≤
    nnreal rounds *
      (nnreal (card B) / nnreal (card query-sample-space))
  by (rule
    wp-verifier-after-composition-fri-query-rounds-any-index-set-bound
    [OF future-t counter-t raw-bound subset])
qed

lemma wp-verify-monad-query-rounds-any-index-set-query-error-bound:
assumes future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and subset: B ⊆ query-sample-space
and frac:
  nnreal (card B) / nnreal (card query-sample-space) ≤
    query-error-bound
shows
  wp-event verify-monad (query-rounds-any-index-set-hit s B rounds) s ≤
    nnreal rounds * query-error-bound
proof –
have base:
  wp-event verify-monad (query-rounds-any-index-set-hit s B rounds) s ≤
    nnreal rounds *
      (nnreal (card B) / nnreal (card query-sample-space))
  by (rule wp-verify-monad-query-rounds-any-index-set-bound
    [OF future raw-bound subset])
have mono:

```

```

nnreal rounds *
  (nnreal (card B) / nnreal (card query-sample-space)) ≤
  nnreal rounds * query-error-bound
by (rule mult-left-mono) (use frac in simp-all)
show ?thesis
by (rule order-trans[OF base mono])
qed

lemma wp-verify-monad-query-header-rounds-any-index-set-bound:
  fixes C :: prob
  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and subset:
    ∧fr f-fri-roots f-final as dg composition-fri-roots final.
    B fr f-fri-roots f-final as dg composition-fri-roots final
    ⊆ query-sample-space
  and bound:
    ∧fr f-fri-roots f-final as dg composition-fri-roots final.
    nnreal
    (card
      (B fr f-fri-roots f-final as dg composition-fri-roots final)) /
    nnreal (card query-sample-space) ≤ C
  shows
    wp-event verify-monad (query-header-rounds-any-index-set-hit s B) s ≤
    nnreal rounds * C
  unfolding verify-monad-composition-fri-decomposition
proof (rule wp-event-bind-bound-by-cont)
  show ¬ query-header-rounds-any-index-set-hit s B None
  unfolding query-header-rounds-any-index-set-hit-def
    query-rounds-any-index-set-hit-def
  by auto
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state) ∈
  set-dist (execute verifier-composition-fri-prefix s)
obtain fr f-fl f-final as dg fl where header-eq:
  header = (fr, f-fl, f-final, as, dg, fl)
by (cases header) auto
have prefix-res:
  length f-fl = ceil-log clength ∧
  length as = length spec ∧
  length fl = ceil-log (to-nat dg + 1) ∧
  PTranscript s =
    [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
    map snd fl @ PTranscript prefix-state ∧
  PQueryCounter prefix-state = PQueryCounter s
using verifier-composition-fri-prefix-outcome
  [OF prefix[unfolded header-eq]]

```

```

  by simp
have future-prefix: query-future-fresh prefix-state
  by (rule verifier-composition-fri-prefix-preserves-query-future-fresh
    [OF future prefix])
have after-eq:
  verifier-after-composition-fri header =
    (read  $\gg$ 
      ( $\lambda$ final.
        ntimes (verifier-query-round-program fr f-fl f-final as fl final)
          rounds))
  unfolding header-eq verifier-after-composition-fri-def by simp
show
  wp-event (verifier-after-composition-fri header)
    (query-header-rounds-any-index-set-hit s B) prefix-state  $\leq$ 
    nnreal rounds * C
  unfolding after-eq
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg$  query-header-rounds-any-index-set-hit s B None
    unfolding query-header-rounds-any-index-set-hit-def
      query-rounds-any-index-set-hit-def
    by auto
next
  fix final query-state
  assume read-final:
    Some (final, query-state)  $\in$  set-dist (execute read prefix-state)
  from read-outcome[OF read-final] obtain rest where
    tr-prefix: PTranscript prefix-state = final # rest
    and tr-query: PTranscript query-state = rest
    and query-counter:
      PQueryCounter query-state = PQueryCounter prefix-state
    by blast
  have future-query: query-future-fresh query-state
    by (rule read-preserves-query-future-fresh[OF future-prefix read-final])
  have counter-query: PQueryCounter query-state = PQueryCounter s
    using prefix-res query-counter by simp
  have header-prefix:
    verifier-header-transcript s fr (map snd f-fl) f-final as dg
      (map snd fl) final (PTranscript query-state)
    using prefix-res tr-prefix tr-query
    unfolding verifier-header-transcript-def verifier-header-messages-def
    by simp
  let ?B = B fr (map snd f-fl) f-final as dg (map snd fl) final
  let ?prog = verifier-query-round-program fr f-fl f-final as fl final
  have base:
    wp-event (ntimes ?prog rounds)
      (query-rounds-any-index-set-hit query-state ?B rounds)
      query-state  $\leq$ 
      nnreal rounds *
        (nnreal (card ?B) / nnreal (card query-sample-space))

```

```

  by (rule wp-ntimes-verifier-query-round-program-any-index-set-bound
      [OF future-query raw-bound subset])
have base-C:
  wp-event (ntimes ?prog rounds)
    (query-rounds-any-index-set-hit query-state ?B rounds)
    query-state ≤ nnreal rounds * C
proof -
  have nnreal rounds *
    (nnreal (card ?B) / nnreal (card query-sample-space)) ≤
    nnreal rounds * C
  by (rule mult-left-mono) (use bound in simp-all)
then show ?thesis
  by (rule order-trans[OF base])
qed
show
  wp-event
    (ntimes ?prog rounds)
    (query-header-rounds-any-index-set-hit s B) query-state ≤
    nnreal rounds * C
proof (rule order-trans[OF - base-C])
  show
    wp-event
      (ntimes ?prog rounds)
      (query-header-rounds-any-index-set-hit s B) query-state ≤
    wp-event
      (ntimes ?prog rounds)
      (query-rounds-any-index-set-hit query-state ?B rounds) query-state
proof (rule wp-event-mono-on-support)
  fix out
  assume out-support:
    out ∈ set-dist (execute (ntimes ?prog rounds) query-state)
  and hit: query-header-rounds-any-index-set-hit s B out
  from hit obtain fr' f-fri-roots' f-final' as' dg'
    composition-fri-roots' final' rest' where
  header-hit:
    verifier-header-transcript s fr' f-fri-roots' f-final' as' dg'
    composition-fri-roots' final' rest'
  and round-hit:
    query-rounds-any-index-set-hit s
      (B fr' f-fri-roots' f-final' as' dg'
        composition-fri-roots' final')
    rounds out
  unfolding query-header-rounds-any-index-set-hit-def by blast
have eqs:
  fr' = fr ∧
  f-fri-roots' = map snd f-fl ∧
  f-final' = f-final ∧
  as' = as ∧
  dg' = dg ∧

```

```

      composition-fri-roots' = map snd fl ∧
      final' = final
    using verifier-header-transcript-unique[OF header-prefix header-hit]
    by simp
  have round-hit-s:
    query-rounds-any-index-set-hit s ?B rounds out
    using round-hit eqs by simp
  show
    query-rounds-any-index-set-hit query-state ?B rounds out
    using round-hit-s counter-query
    unfolding query-rounds-any-index-set-hit-def by auto
qed
qed
qed
qed

```

lemma *wp-query-index-round-set-hit-bound-via-supported-header-union:*

```

  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and union-bound:
    ∧fr f-fri-roots f-final as dg composition-fri-roots final.
    nnreal
    (card
      (query-header-supported-union-good-sets s good-sets fr
        f-fri-roots f-final as dg composition-fri-roots final)) /
    nnreal (card query-sample-space) ≤ query-error-bound

```

shows

```

  wp-event verify-monad (query-index-round-set-hit s good-sets) s ≤
  nnreal rounds * query-error-bound

```

proof –

```

  have header-bound:
    wp-event verify-monad
      (query-header-rounds-any-index-set-hit s
        (query-header-supported-union-good-sets s good-sets)) s ≤
    nnreal rounds * query-error-bound
  by (rule wp-verify-monad-query-header-rounds-any-index-set-bound
    [OF future raw-bound])
    (use query-header-supported-union-good-sets-subset union-bound in
      simp-all)
  have wp-event verify-monad (query-index-round-set-hit s good-sets) s ≤
    wp-event verify-monad
      (query-header-rounds-any-index-set-hit s
        (query-header-supported-union-good-sets s good-sets)) s
  by (rule wp-event-mono-on-support)
    (rule query-index-round-set-hit-imp-query-header-supported-rounds-hit)
  also have ... ≤ nnreal rounds * query-error-bound
  by (rule header-bound)
  finally show ?thesis .

```

qed

lemma *verifier-initial-query-rounds-any-index-set-error-bound:*

assumes *raw-bound: query-index-raw-preimage-bound*

and *subset: $B \subseteq \text{query-sample-space}$*

and *frac:*

*nnreal (card B) / nnreal (card $\text{query-sample-space}$) $\leq$
*query-error-bound**

shows

wp-event verify-monad

(query-rounds-any-index-set-hit (verifier-initial-state tr) B rounds)

(verifier-initial-state tr) $\leq$

*nnreal rounds * query-error-bound*

by (*rule wp-verify-monad-query-rounds-any-index-set-query-error-bound*
[OF verifier-initial-query-future-fresh raw-bound subset frac])

definition *query-index-freshness-assumption*

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$ bool

where

query-index-freshness-assumption s $\longleftrightarrow$

($\forall$ good-sets.

($\forall$ trace-table composition-table as.

good-sets trace-table composition-table as $\subseteq$ query-sample-space) $\longrightarrow$

($\forall$ trace-table composition-table as.

nnreal (card (good-sets trace-table composition-table as)) /

nnreal (card query-sample-space) $\leq$ query-error-bound) $\longrightarrow$

wp-event verify-monad (query-index-set-hit s good-sets) s $\leq$

*nnreal rounds * query-error-bound)*

definition *query-index-round-freshness-assumption*

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$ bool

where

query-index-round-freshness-assumption s $\longleftrightarrow$

($\forall$ good-sets.

($\forall$ trace-table composition-table as.

good-sets trace-table composition-table as $\subseteq$ query-sample-space) $\longrightarrow$

($\forall$ trace-table composition-table as.

nnreal (card (good-sets trace-table composition-table as)) /

nnreal (card query-sample-space) $\leq$ query-error-bound) $\longrightarrow$

wp-event verify-monad (query-index-round-set-hit s good-sets) s $\leq$

*nnreal rounds * query-error-bound)*

lemma *query-index-freshness-from-round-freshness:*

assumes *fresh-round: query-index-round-freshness-assumption s*

shows *query-index-freshness-assumption s*

unfolding *query-index-freshness-assumption-def*

proof (*intro allI impI*)

fix *good-sets :: 'f list $\Rightarrow$ 'f list $\Rightarrow$ 'f list $\Rightarrow$ nat set*

assume *subset:*

$\forall$ trace-table composition-table as.

$\text{good-sets trace-table composition-table as} \subseteq \text{query-sample-space}$
and frac:
 $\forall \text{ trace-table composition-table as.}$
 $\text{nnreal (card (good-sets trace-table composition-table as)) /}$
 $\text{nnreal (card query-sample-space)} \leq \text{query-error-bound}$
have $\text{wp-event verify-monad (query-index-set-hit s good-sets) s} \leq$
 $\text{wp-event verify-monad (query-index-round-set-hit s good-sets) s}$
by $(\text{rule wp-event-mono}) (\text{rule query-index-set-hit-imp-round-set-hit})$
also have $\dots \leq \text{nnreal rounds} * \text{query-error-bound}$
using $\text{fresh-round subset frac}$
unfolding $\text{query-index-round-freshness-assumption-def}$ **by** blast
finally show $\text{wp-event verify-monad (query-index-set-hit s good-sets) s} \leq$
 $\text{nnreal rounds} * \text{query-error-bound} .$
qed

lemma $\text{query-index-round-freshness-if-no-query-supported-pairwise-merkle-bad:}$
assumes $\text{future: query-future-fresh s}$
and $\text{raw-bound: query-index-raw-preimage-bound}$
and $\text{no-bad: } \neg \text{query-supported-pairwise-merkle-bad s}$
shows $\text{query-index-round-freshness-assumption s}$
unfolding $\text{query-index-round-freshness-assumption-def}$
proof (intro allI impI)
fix $\text{good-sets :: 'f list} \Rightarrow \text{'f list} \Rightarrow \text{'f list} \Rightarrow \text{nat set}$
assume subset:
 $\forall \text{ trace-table composition-table as.}$
 $\text{good-sets trace-table composition-table as} \subseteq \text{query-sample-space}$
and bounded:
 $\forall \text{ trace-table composition-table as.}$
 $\text{nnreal (card (good-sets trace-table composition-table as)) /}$
 $\text{nnreal (card query-sample-space)} \leq \text{query-error-bound}$
have union-bound:
 $\bigwedge \text{fr f-fri-roots f-final as dg composition-fri-roots final.}$
 nnreal
 $(\text{card}$
 $(\text{query-header-supported-union-good-sets s good-sets fr f-fri-roots}$
 $\text{f-final as dg composition-fri-roots final})) /$
 $\text{nnreal (card query-sample-space)} \leq \text{query-error-bound}$
by $(\text{rule}$
 $\text{query-header-supported-union-good-sets-fraction-bound-if-no-global-pairwise-merkle-bad}$
 $[\text{OF no-bad}])$
 $(\text{use subset bounded in blast})+$
show $\text{wp-event verify-monad (query-index-round-set-hit s good-sets) s} \leq$
 $\text{nnreal rounds} * \text{query-error-bound}$
by $(\text{rule wp-query-index-round-set-hit-bound-via-supported-header-union}$
 $[\text{OF future raw-bound union-bound}])$
qed

lemma $\text{query-index-freshness-if-no-query-supported-pairwise-merkle-bad:}$
assumes $\text{future: query-future-fresh s}$

and *raw-bound*: *query-index-raw-preimage-bound*
and *no-bad*: $\neg$ *query-supported-pairwise-merkle-bad* *s*
shows *query-index-freshness-assumption* *s*
by (rule *query-index-freshness-from-round-freshness*)
(rule *query-index-round-freshness-if-no-query-supported-pairwise-merkle-bad*
[OF *future raw-bound no-bad*])

lemma *verifier-initial-query-index-freshness-if-no-query-supported-pairwise-merkle-bad*:
assumes *raw-bound*: *query-index-raw-preimage-bound*
and *no-bad*:
 $\neg$ *query-supported-pairwise-merkle-bad* (*verifier-initial-state tr*)
shows *query-index-freshness-assumption* (*verifier-initial-state tr*)
by (rule *query-index-freshness-if-no-query-supported-pairwise-merkle-bad*)
(use *raw-bound no-bad* **in** *simp-all*)

lemma *verifier-initial-query-index-freshness-if-uniform-sampler-and-no-query-supported-pairwise-merkle-bad*:
assumes *range-eq*: *range to-nat* = $\{0..<size\}$
and *dvd*: *query-sample-space-size dvd size*
and *no-bad*:
 $\neg$ *query-supported-pairwise-merkle-bad* (*verifier-initial-state tr*)
shows *query-index-freshness-assumption* (*verifier-initial-state tr*)
proof –
have *raw-bound*: *query-index-raw-preimage-bound*
by (rule *query-index-raw-preimage-bound-if-uniform-range*
[OF *range-eq dvd*])
show *?thesis*
by (rule
verifier-initial-query-index-freshness-if-no-query-supported-pairwise-merkle-bad
[OF *raw-bound no-bad*])

qed

lemma *query-index-freshness-rounds-zero-obstruction*:
assumes *rounds-zero*: *rounds* = 0
and *outcome*: *out* $\in$ *set-dist* (*execute verify-monad s*)
and *bound*:
accepted-with-bound-tables s out trace-table composition-table as query-idxs
shows $\neg$ *query-index-freshness-assumption s*
proof
assume *fresh*: *query-index-freshness-assumption s*
let *?good-sets* = λ - - . $\{\}$
have *hit*: *query-index-set-hit s ?good-sets out*
by (rule *query-index-set-hit-vacuous-if-rounds-zero*
[OF *rounds-zero bound*])
have *positive*:
0 < *wp-event verify-monad* (*query-index-set-hit s ?good-sets*) *s*
by (rule *wp-event-pos-of-support*
[**where** *out*=*out* **and** *m*=*verify-monad* **and** *s*=*s*
and *P*=*query-index-set-hit s ?good-sets*, OF *outcome hit*])
have *subset*:


```

 $\wedge$  trace-table composition-table as.
  ?good-sets trace-table composition-table as  $\subseteq$  query-sample-space
by simp
have frac:
 $\wedge$  trace-table composition-table as.
  nnreal (card (?good-sets trace-table composition-table as)) /
    nnreal (card query-sample-space)  $\leq$  query-error-bound
by simp
have zero-bound:
  wp-event verify-monad (query-index-set-hit s ?good-sets) s  $\leq$  0
  using fresh subset frac rounds-zero
  unfolding query-index-freshness-assumption-def by simp
have zero:
  wp-event verify-monad (query-index-set-hit s ?good-sets) s = 0
  by (rule antisym[OF zero-bound]) simp
show False
  using positive zero by simp
qed

lemma query-bad-imp-query-index-set-hit:
  assumes query-bad s out
  shows query-index-set-hit s query-sampling-success-space out
proof -
  from assms obtain trace-table composition-table as query-idxs
  where bound:
    accepted-with-bound-tables s out trace-table composition-table as query-idxs
  and trace-low: trace-table-low-degree trace-table
  and composition-low:
    composition-table-low-degree maxDegree composition-table
  and not-all:
     $\neg$  all-queries-consistent trace-table composition-table as
  unfolding query-bad-def by blast
have miss:
  query-samples-miss-disagreements trace-table composition-table as query-idxs
  by (rule accepted-with-bound-tables-query-samples-miss-if-not-all-consistent
    [OF bound not-all])
have success-space:
  query-sampling-success-space trace-table composition-table as =
    query-agreement-indices trace-table composition-table as
  using trace-low composition-low not-all
  unfolding query-sampling-success-space-def by simp
have samples:
  set query-idxs  $\subseteq$ 
    query-sampling-success-space trace-table composition-table as
  using miss unfolding query-samples-miss-disagreements-def success-space by simp
show ?thesis
  unfolding query-index-set-hit-def
  using bound samples by blast
qed

```

lemma *query-bad-imp-query-index-round-set-hit*:
assumes *query-bad s out*
shows *query-index-round-set-hit s query-sampling-success-space out*
by (*rule query-index-set-hit-imp-round-set-hit*)
(rule query-bad-imp-query-index-set-hit[OF assms])

lemma *query-bad-bound-from-phase4B*:
assumes *fresh: query-index-freshness-assumption s*
shows *wp-event verify-monad (query-bad s) s ≤*
*nnreal rounds * query-error-bound*

proof –
have *hit-bound*:
wp-event verify-monad
(query-index-set-hit s query-sampling-success-space) s ≤
*nnreal rounds * query-error-bound*
using *fresh query-sampling-success-space-subset*
query-sampling-success-space-fraction-bound-query-sample-space
unfolding *query-index-freshness-assumption-def*
by *blast*
have *wp-event verify-monad (query-bad s) s ≤*
wp-event verify-monad
(query-index-set-hit s query-sampling-success-space) s
by (*rule wp-event-mono*) (*rule query-bad-imp-query-index-set-hit*)
also have *... ≤ nnreal rounds * query-error-bound*
by (*rule hit-bound*)
finally show *?thesis* .
qed

lemma *accepted-bound-trace-low-degree-or-trace-fri-bad*:
assumes *bound*:
accepted-with-bound-tables s out trace-table composition-table as query-idxs
shows *trace-table-low-degree trace-table ∨ trace-fri-bad s out*
proof (*cases trace-table-low-degree trace-table*)
case *True*
then show *?thesis* **by** *simp*
next
case *False*
then have *trace-fri-bad s out*
unfolding *trace-fri-bad-def* **using** *bound* **by** *blast*
then show *?thesis* **by** *simp*
qed

lemma *accepted-bound-composition-low-degree-or-composition-fri-bad*:
assumes *bound*:
accepted-with-bound-tables s out trace-table composition-table as query-idxs
and *trace-low: trace-table-low-degree trace-table*
shows *composition-table-low-degree maxDegree composition-table ∨*
composition-fri-bad s out

```

proof (cases composition-table-low-degree maxDegree composition-table)
  case True
    then show ?thesis by simp
next
  case False
    then have composition-fri-bad s out
      unfolding composition-fri-bad-def using bound trace-low by blast
    then show ?thesis by simp
qed

lemma accepted-bound-all-queries-consistent-or-query-bad:
  assumes bound:
    accepted-with-bound-tables s out trace-table composition-table as query-idxs
    and trace-low: trace-table-low-degree trace-table
    and composition-low: composition-table-low-degree maxDegree composition-table
  shows all-queries-consistent trace-table composition-table as  $\vee$ 
    query-bad s out
proof (cases all-queries-consistent trace-table composition-table as)
  case True
    then show ?thesis by simp
next
  case False
    then have query-bad s out
      unfolding query-bad-def using bound trace-low composition-low by blast
    then show ?thesis by simp
qed

lemma accepted-bound-composition-bad-if-globally-consistent:
  assumes false-statement:  $\neg$  exists-valid-trace
    and bound:
      accepted-with-bound-tables s out trace-table composition-table as query-idxs
      and trace-low: trace-table-low-degree trace-table
      and composition-low: composition-table-low-degree maxDegree composition-table
      and all-queries: all-queries-consistent trace-table composition-table as
  shows composition-bad s out
proof –
  from trace-low obtain f where
    deg-f: degree f < clength
    and trace-table: trace-table = map (poly f) eval-domain
    unfolding trace-table-low-degree-def by blast
  have violated: violated-constraints f  $\neq$  {}
    by (rule false-statement-violated-constraints[OF false-statement deg-f])
  have len-as: length as = length spec
    using accepted-with-tables-shapes(3)
    [OF accepted-with-bound-tables-imp-accepted-with-tables[OF bound]]
    by simp
  have hide-if-bounds:
    common-denominator-degree-bounds f as  $\implies$ 
      random-combination-common-denominator-hides-violations f as

```

```

    by (rule random-combination-common-denominator-hides-from-queries
        [OF len-as false-statement deg-f trace-table composition-low all-queries])
  show ?thesis
    unfolding composition-bad-def composition-degree-bad-def
      composition-randomization-bad-def composition-bad-context-def
    using bound deg-f trace-table violated composition-low all-queries hide-if-bounds
    by blast
qed

```

```

lemma accepted-bound-table-partition:
  assumes false-statement:  $\neg$  exists-valid-trace
  and bound:
    accepted-with-bound-tables s out trace-table composition-table as query-idxs
  shows
    composition-bad s out  $\vee$  trace-fri-bad s out  $\vee$ 
    composition-fri-bad s out  $\vee$  query-bad s out
proof -
  from accepted-bound-trace-low-degree-or-trace-fri-bad[OF bound]
  consider
    (trace-low) trace-table-low-degree trace-table
  | (trace-bad) trace-fri-bad s out
  by blast
  then show ?thesis
proof cases
  case trace-bad
  then show ?thesis by blast
next
  case trace-low
  from accepted-bound-composition-low-degree-or-composition-fri-bad
    [OF bound trace-low]
  consider
    (composition-low) composition-table-low-degree maxDegree composition-table
  | (composition-bad) composition-fri-bad s out
  by blast
  then show ?thesis
proof cases
  case composition-bad
  then show ?thesis by blast
next
  case composition-low
  from accepted-bound-all-queries-consistent-or-query-bad
    [OF bound trace-low composition-low]
  consider
    (all-queries) all-queries-consistent trace-table composition-table as
  | (query-bad) query-bad s out
  by blast
  then show ?thesis
proof cases
  case query-bad

```

```

    then show ?thesis by blast
  next
    case all-queries
    have composition-bad s out
      by (rule accepted-bound-composition-bad-if-globally-consistent
        [OF false-statement bound trace-low composition-low all-queries])
    then show ?thesis by blast
  qed
qed
qed
qed
qed

lemma composition-randomization-bad-imp-alpha-bad-set-hit:
  assumes bad: composition-randomization-bad s out
  shows alpha-bad-set-hit s composition-trace-bad-alpha-space out
proof -
  from bad obtain trace-table composition-table as query-idxs f where
    ctx:
      composition-bad-context s out trace-table composition-table as query-idxs f
    and bounds: common-denominator-degree-bounds f as
    and hides: random-combination-common-denominator-hides-violations f as
    unfolding composition-randomization-bad-def by blast
  have bound:
    accepted-with-bound-tables s out trace-table composition-table as query-idxs
    using ctx unfolding composition-bad-context-def by blast
  have deg-f: degree f < clength
    using ctx unfolding composition-bad-context-def by blast
  have trace-table: trace-table = map (poly f) eval-domain
    using ctx unfolding composition-bad-context-def by blast
  have violated: violated-constraints f ≠ {}
    using ctx unfolding composition-bad-context-def by blast
  have len-as: length as = length spec
    using accepted-with-tables-shapes(3)
    [OF accepted-with-bound-tables-imp-accepted-with-tables[OF bound]] .
  have as-in:
    as ∈ common-denominator-hiding-alpha-space f
    using len-as hides
    unfolding common-denominator-hiding-alpha-space-def alpha-space-def
    by simp
  have witness-eq: low-degree-trace-witness trace-table = f
    by (rule low-degree-trace-witness-eq[OF deg-f trace-table])
  have as ∈ composition-trace-bad-alpha-space trace-table
    using deg-f trace-table violated bounds as-in
    unfolding composition-trace-bad-alpha-space-def witness-eq
    by simp
  then show ?thesis
    using bound unfolding alpha-bad-set-hit-def by blast
qed

```

lemma *composition-randomization-bad-bound-from-alpha-freshness:*
assumes *fresh:* *alpha-challenge-freshness-assumption s*
shows *wp-event verify-monad (composition-randomization-bad s) s ≤ composition-error-bound*
proof –
have *hit-bound:*
wp-event verify-monad
(alpha-bad-set-hit s composition-trace-bad-alpha-space) s ≤ composition-error-bound
using *fresh composition-trace-bad-alpha-space-subset-alpha-space composition-trace-bad-alpha-space-fraction-bound-alpha-space*
unfolding *alpha-challenge-freshness-assumption-def*
by *blast*
have *wp-event verify-monad (composition-randomization-bad s) s ≤ wp-event verify-monad (alpha-bad-set-hit s composition-trace-bad-alpha-space) s*
by *(rule wp-event-mono)*
(rule composition-randomization-bad-imp-alpha-bad-set-hit)
also have *... ≤ composition-error-bound*
by *(rule hit-bound)*
finally show *?thesis .*
qed

lemma *composition-randomization-bad-bound-via-header-union:*
assumes *future:* *alpha-future-fresh s*
and *union-bound:*
 $\bigwedge_{fr} f\text{-fri-roots } f\text{-final.}$
 $nnreal$
 $(card$
 $(alpha\text{-header-union-bad-sets } s$
 $composition\text{-trace-bad-alpha-space } fr\ f\text{-fri-roots } f\text{-final})) /$
 $nnreal (card\ alpha\text{-space}) \leq composition\text{-error-bound}$
shows *wp-event verify-monad (composition-randomization-bad s) s ≤ composition-error-bound*
proof –
have *hit-bound:*
wp-event verify-monad
(alpha-bad-set-hit s composition-trace-bad-alpha-space) s ≤ composition-error-bound
by *(rule wp-alpha-bad-set-hit-bound-via-header-union[OF future])*
(use composition-trace-bad-alpha-space-subset-alpha-space union-bound in simp-all)
have *wp-event verify-monad (composition-randomization-bad s) s ≤ wp-event verify-monad (alpha-bad-set-hit s composition-trace-bad-alpha-space) s*
by *(rule wp-event-mono)*
(rule composition-randomization-bad-imp-alpha-bad-set-hit)
also have *... ≤ composition-error-bound*
by *(rule hit-bound)*

finally show ?thesis .
qed

lemma *composition-randomization-bad-bound-via-supported-header-union:*

assumes *future: alpha-future-fresh s*

and *union-bound:*

$\bigwedge \text{fr } f\text{-fri-roots } f\text{-final.}$

nnreal

(*card*

(*alpha-header-supported-union-bad-sets s*

composition-trace-bad-alpha-space fr f-fri-roots f-final)) /

nnreal (card alpha-space) ≤ composition-error-bound

shows *wp-event verify-monad (composition-randomization-bad s) s ≤ composition-error-bound*

proof –

have *hit-bound:*

wp-event verify-monad

(*alpha-bad-set-hit s composition-trace-bad-alpha-space*) *s ≤*

composition-error-bound

by (rule *wp-alpha-bad-set-hit-bound-via-supported-header-union[OF future]*)

(use *composition-trace-bad-alpha-space-subset-alpha-space union-bound in simp-all*)

have *wp-event verify-monad (composition-randomization-bad s) s ≤*

wp-event verify-monad

(*alpha-bad-set-hit s composition-trace-bad-alpha-space*) *s*

by (rule *wp-event-mono*)

(rule *composition-randomization-bad-imp-alpha-bad-set-hit*)

also have ... *≤ composition-error-bound*

by (rule *hit-bound*)

finally show ?thesis .

qed

lemma *composition-randomization-bad-bound-if-alpha-header-candidate-unique:*

assumes *future: alpha-future-fresh s*

and *unique:*

$\bigwedge \text{fr } f\text{-fri-roots } f\text{-final.}$

$\exists \text{ trace-table.}$

alpha-header-trace-table-candidates s fr f-fri-roots f-final $\subseteq$

{*trace-table*}

shows *wp-event verify-monad (composition-randomization-bad s) s ≤*

composition-error-bound

by (rule *composition-randomization-bad-bound-via-header-union[OF future]*)

(rule *composition-trace-alpha-header-union-fraction-bound-if-unique-candidate [OF unique]*)

lemma *composition-randomization-bad-bound-if-alpha-header-supported-candidate-unique:*

assumes *future: alpha-future-fresh s*

and *unique:*

$\bigwedge \text{fr } f\text{-fri-roots } f\text{-final.}$

$\exists$ *trace-table*.
 α -header-supported-trace-table-candidates *s* fr *f*-fri-roots
 f -final $\subseteq \{\text{trace-table}\}$
shows *wp-event verify-monad* (*composition-randomization-bad s*) $s \leq$
composition-error-bound
by (rule *composition-randomization-bad-bound-via-supported-header-union*
[*OF future*])
(rule
composition-trace-alpha-header-supported-union-fraction-bound-if-unique-candidate
[*OF unique*])

lemma *composition-randomization-bad-bound-if-no-alpha-supported-pairwise-merkle-bad*:
assumes *future*: α -future-fresh *s*
and *no-bad*: $\neg$ α -supported-pairwise-merkle-bad *s*
shows *wp-event verify-monad* (*composition-randomization-bad s*) $s \leq$
composition-error-bound
proof (rule *composition-randomization-bad-bound-if-alpha-header-supported-candidate-unique*
[*OF future*])
fix fr *f*-fri-roots *f*-final
show
 $\exists$ *trace-table*.
 α -header-supported-trace-table-candidates *s* fr *f*-fri-roots *f*-final
 $\subseteq \{\text{trace-table}\}$
by (rule *alpha-header-supported-candidate-unique-if-no-global-pairwise-merkle-bad*
[*OF no-bad*])

qed

lemma *composition-degree-bad-false*:
assumes *wf*: *spec-degree-wellformed*
shows $\neg$ *composition-degree-bad s out*
using *common-denominator-degree-bounds-from-spec-degree-wellformed* [*OF wf*]
unfolding *composition-degree-bad-def composition-bad-context-def*
by *blast*

lemma *composition-degree-bad-bound*:
assumes *wf*: *spec-degree-wellformed*
shows *wp-event verify-monad* (*composition-degree-bad s*) $s \leq 0$
proof –
have *composition-degree-bad s* =
 $(\lambda \text{out} :: (\text{unit list} \times ('f, 'a) \text{protocol-channel-scheme}) \text{option}. \text{False})$
by (rule *ext*) (*simp add: composition-degree-bad-false* [*OF wf*])
then show ?thesis
unfolding *wp-event-def wp-def dist-expect-def* **by** *simp*

qed

lemma *composition-bad-bound-from-phase4A*:
assumes *wf*: *spec-degree-wellformed*
and α -fresh: α -challenge-freshness-assumption *s*
shows *wp-event verify-monad* (*composition-bad s*) $s \leq$ *composition-error-bound*


```

proof –
  have degree-bound:
    wp-event verify-monad (composition-degree-bad s) s ≤ 0
    by (rule composition-degree-bad-bound [OF wf])
  have randomization-bound:
    wp-event verify-monad (composition-randomization-bad s) s ≤
      composition-error-bound
    by (rule composition-randomization-bad-bound-from-alpha-freshness [OF alpha-fresh])
  have wp-event verify-monad (composition-bad s) s =
    wp-event verify-monad
      ( $\lambda out. composition-degree-bad\ s\ out \vee$ 
       composition-randomization-bad s out) s
    unfolding composition-bad-def by simp
  also have ...  $\leq$ 
    wp-event verify-monad (composition-degree-bad s) s +
    wp-event verify-monad (composition-randomization-bad s) s
    by (rule wp-event-union-bound)
  also have ...  $\leq 0 + composition-error-bound$ 
    by (intro add-mono degree-bound randomization-bound)
  finally show ?thesis by simp
qed

end

end

```

```

theory Soundness-FRI
  imports
    Soundness-Bad-Events
begin

```

```

context soundness
begin

```

4.2 Algebraic FRI Soundness Interface

The next definitions isolate the algebraic content of the verifier's FRI fold equation from transcript extraction and Merkle authentication. They are intentionally protocol-independent except for using the same affine fold as *receive-query-commits*.

definition *fri-table-fold-value*

$:: 'f \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow nat \Rightarrow nat \Rightarrow nat \Rightarrow 'f$

where

fri-table-fold-value *b layer fri-dom len pw i* =

fri-fold-value *b (layer ! i) (layer ! fri-sibling-index len i)*

(fri-fold-denominator (fri-dom ! i) 1)

definition *fri-algebraic-layer-relation*

$:: nat \Rightarrow nat \Rightarrow 'f \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow$

```

    'f list  $\Rightarrow$  'f list  $\Rightarrow$  bool
  where
    fri-algebraic-layer-relation len pw b fri-dom layer next-dom
      next-layer  $\longleftrightarrow$ 
      0 < len  $\wedge$ 
      len  $\leq$  length fri-dom  $\wedge$ 
      len  $\leq$  length layer  $\wedge$ 
      len div 2  $\leq$  length next-dom  $\wedge$ 
      len div 2  $\leq$  length next-layer  $\wedge$ 
      ( $\forall i < \text{len div } 2.$ 
        next-dom ! i = (fri-dom ! i)2  $\wedge$ 
        next-layer ! i = fri-table-fold-value b layer fri-dom len pw i)

  lemma fri-algebraic-layer-relationD:
    assumes fri-algebraic-layer-relation len pw b fri-dom layer next-dom
      next-layer
    and i < len div 2
  shows next-dom ! i = (fri-dom ! i)2
    and next-layer ! i = fri-table-fold-value b layer fri-dom len pw i
  using assms unfolding fri-algebraic-layer-relation-def by auto

  definition fri-opening-matches-table
    :: nat  $\Rightarrow$  nat  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$  'f  $\Rightarrow$  bool
  where
    fri-opening-matches-table len idx layer xp xn  $\longleftrightarrow$ 
      idx < len  $\wedge$ 
      len  $\leq$  length layer  $\wedge$ 
      layer ! idx = xp  $\wedge$ 
      layer ! fri-sibling-index len idx = xn

  lemma fri-opening-matches-tableD:
    assumes fri-opening-matches-table len idx layer xp xn
  shows idx < len
    and len  $\leq$  length layer
    and layer ! idx = xp
    and layer ! fri-sibling-index len idx = xn
  using assms unfolding fri-opening-matches-table-def by auto

  lemma fri-fold-value-eq-table-fold-value:
    assumes dom-idx: fri-dom ! idx = (h  $\wedge$  idx * shift)  $\wedge$  pw
    and match: fri-opening-matches-table len idx layer xp xn
  shows
    fri-fold-value b xp xn (fri-fold-denominator (h  $\wedge$  idx * shift) pw) =
      fri-table-fold-value b layer fri-dom len pw idx
  using dom-idx fri-opening-matches-tableD(3,4)[OF match]
  unfolding fri-table-fold-value-def fri-fold-denominator-def by simp

  lemma fri-layer-opening-step-outcome-fold-table-if-opening-matches:
    assumes outcome:

```

Some $(out, t) \in$
 $set-dist \ (execute \ (fri-layer-opening-step \ (b, f) \ (i, x, len, pw)) \ s)$
 and $dom-i: fri-dom \ ! \ i = (h \wedge i * shift) \wedge pw$
obtains $xp \ xp-path \ xn \ xn-path \ x'$
where
 $out = (i \bmod (len \div 2), x', len \div 2, pw + pw)$
 and $xp = x$
 and $fri-layer-opening-chunk \ len \ xp \ xp-path \ xn \ xn-path$
 $([xp] @ xp-path @ [xn] @ xn-path)$
 and $PTranscript \ s = ([xp] @ xp-path @ [xn] @ xn-path) @ PTranscript \ t$
 and $PState \ t = foldl \ concat \ (PState \ s) \ ([xp] @ xp-path @ [xn] @ xn-path)$
 and $s \leq t$
 and $fri-opening-matches-table \ len \ i \ layer \ xp \ xn \implies$
 $x' = fri-table-fold-value \ b \ layer \ fri-dom \ len \ pw \ i$
proof –
from $fri-layer-opening-step-outcome[OF \ outcome]$
obtain $xp \ xp-path \ xn \ xn-path \ x'$ **where**
 $out-eq: out = (i \bmod (len \div 2), x', len \div 2, pw + pw)$
 and $xp-eq: xp = x$
 and $x'-eq:$
 $x' = fri-fold-value \ b \ xp \ xn$
 $(fri-fold-denominator \ ((h \wedge i) * shift) \ pw)$
 and $chunk:$
 $fri-layer-opening-chunk \ len \ xp \ xp-path \ xn \ xn-path$
 $([xp] @ xp-path @ [xn] @ xn-path)$
 and $tr:$
 $PTranscript \ s = ([xp] @ xp-path @ [xn] @ xn-path) @ PTranscript \ t$
 and $st:$
 $PState \ t = foldl \ concat \ (PState \ s) \ ([xp] @ xp-path @ [xn] @ xn-path)$
 and $ext: s \leq t$
by $blast$
have $table-fold:$
 $fri-opening-matches-table \ len \ i \ layer \ xp \ xn \implies$
 $x' = fri-table-fold-value \ b \ layer \ fri-dom \ len \ pw \ i$
proof –
assume $match: fri-opening-matches-table \ len \ i \ layer \ xp \ xn$
have $fri-fold-value \ b \ xp \ xn$
 $(fri-fold-denominator \ ((h \wedge i) * shift) \ pw) =$
 $fri-table-fold-value \ b \ layer \ fri-dom \ len \ pw \ i$
 by $(rule \ fri-fold-value-eq-table-fold-value[OF \ dom-i \ match])$
then show $x' = fri-table-fold-value \ b \ layer \ fri-dom \ len \ pw \ i$
 using $x'-eq$ **by** $simp$
qed
show $?thesis$
 by $(rule \ that[OF \ out-eq \ xp-eq \ chunk \ tr \ st \ ext \ table-fold])$
qed

definition $fri-fold-coeff-pair :: 'f \Rightarrow 'f \Rightarrow 'f \Rightarrow 'f \times 'f$
where

fri-fold-coeff-pair $xp\ xn\ denom =$
 $((xp + xn) \text{ div } 2, (xp - xn) \text{ div } denom)$

definition *fri-local-hiding-challenges*

$:: 'f \Rightarrow 'f \Rightarrow 'f \Rightarrow 'f \Rightarrow 'f \Rightarrow 'f \text{ set}$

where

fri-local-hiding-challenges $xp\ xn\ yp\ yn\ denom =$
 $\{b. \text{fri-fold-value } b\ xp\ xn\ denom = \text{fri-fold-value } b\ yp\ yn\ denom\}$

lemma *fri-affine-collision-challenges-card-le-one*:

fixes $a\ c\ a'\ c' :: 'f$

assumes *pair-diff*: $(a, c) \neq (a', c')$

shows $\text{card } \{b. a + b * c = a' + b * c'\} \leq (1::\text{nat})$

proof –

show *?thesis*

proof (*cases* $c = c'$)

case *True*

then have *a-diff*: $a \neq a'$

using *pair-diff* **by** *auto*

have $\{b. a + b * c = a' + b * c'\} = \{\}$

proof (*rule ccontr*)

assume $\{b. a + b * c = a' + b * c'\} \neq \{\}$

then obtain b **where**

b-in: $b \in \{b. a + b * c = a' + b * c'\}$

by *blast*

then have *eq*: $a + b * c = a' + b * c'$

by *simp*

then have $a + b * c' = a' + b * c'$

using *True* **by** *simp*

then have $(a + b * c') - b * c' = (a' + b * c') - b * c'$

by *simp*

then have $a = a'$

by *simp*

then show *False*

using *a-diff* **by** *contradiction*

qed

then show *?thesis* **by** *simp*

next

case *False*

have *subset*:

$\{b. a + b * c = a' + b * c'\} \subseteq$

$\{(a' - a) \text{ div } (c - c')\}$

proof

fix b

assume *b-in*: $b \in \{b. a + b * c = a' + b * c'\}$

then have *eq*: $a + b * c = a' + b * c'$

by *simp*

have $b * (c - c') = a' - a$

using *eq* **by** (*simp add: algebra-simps*)

```

    then have  $b = (a' - a) \text{ div } (c - c')$ 
      using False by (simp add: field-simps)
    then show  $b \in \{(a' - a) \text{ div } (c - c')\}$ 
      by simp
  qed
  have card  $\{b. a + b * c = a' + b * c'\} \leq$ 
    card  $\{(a' - a) \text{ div } (c - c')\}$ 
    by (rule card-mono) (use subset in auto)
  then show ?thesis by simp
qed
qed

lemma fri-local-hiding-challenges-card-le-one:
  assumes pair-diff:
    fri-fold-coeff-pair  $xp\ xn\ denom \neq$  fri-fold-coeff-pair  $yp\ yn\ denom$ 
  shows card (fri-local-hiding-challenges  $xp\ xn\ yp\ yn\ denom$ )  $\leq 1$ 
proof -
  let ?a =  $(xp + xn) \text{ div } 2$ 
  let ?c =  $(xp - xn) \text{ div } denom$ 
  let ?a' =  $(yp + yn) \text{ div } 2$ 
  let ?c' =  $(yp - yn) \text{ div } denom$ 
  have pair-diff':  $(?a, ?c) \neq (?a', ?c')$ 
    using pair-diff unfolding fri-fold-coeff-pair-def by simp
  have fri-local-hiding-challenges  $xp\ xn\ yp\ yn\ denom =$ 
     $\{b. ?a + b * ?c = ?a' + b * ?c'\}$ 
    unfolding fri-local-hiding-challenges-def fri-fold-value-def by simp
  then show ?thesis
    using fri-affine-collision-challenges-card-le-one[OF pair-diff'] by simp
qed

```

```

lemma fri-one-step-disagreement-hiding-bound:
  assumes
    fri-fold-coeff-pair  $xp\ xn\ denom \neq$  fri-fold-coeff-pair  $yp\ yn\ denom$ 
  shows card (fri-local-hiding-challenges  $xp\ xn\ yp\ yn\ denom$ )  $\leq 1$ 
  by (rule fri-local-hiding-challenges-card-le-one[OF assms])

```

```

lemma fri-one-step-disagreement-outside-hiding:
  assumes  $b \notin$  fri-local-hiding-challenges  $xp\ xn\ yp\ yn\ denom$ 
  shows fri-fold-value  $b\ xp\ xn\ denom \neq$  fri-fold-value  $b\ yp\ yn\ denom$ 
  using assms unfolding fri-local-hiding-challenges-def by simp

```

```

definition fri-table-local-hiding-challenges
  :: 'f list  $\Rightarrow$  'f list  $\Rightarrow$  'f list  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$ 
    'f set

```

```

where
  fri-table-local-hiding-challenges layer claimed fri-dom len pw i =
    fri-local-hiding-challenges
      (layer ! i) (layer ! fri-sibling-index len i)
      (claimed ! i) (claimed ! fri-sibling-index len i)

```

$(\text{fri-fold-denominator } (\text{fri-dom } ! \ i) \ pw)$

lemma *fri-table-local-hiding-challenges-card-le-one*:

assumes

fri-fold-coeff-pair
 $(\text{layer } ! \ i) \ (\text{layer } ! \ \text{fri-sibling-index } \text{len } i)$
 $(\text{fri-fold-denominator } (\text{fri-dom } ! \ i) \ pw) \neq$
fri-fold-coeff-pair
 $(\text{claimed } ! \ i) \ (\text{claimed } ! \ \text{fri-sibling-index } \text{len } i)$
 $(\text{fri-fold-denominator } (\text{fri-dom } ! \ i) \ pw)$

shows $\text{card } (\text{fri-table-local-hiding-challenges } \text{layer } \text{claimed } \text{fri-dom } \text{len } pw \ i)$
 ≤ 1

unfolding *fri-table-local-hiding-challenges-def*

by $(\text{rule } \text{fri-local-hiding-challenges-card-le-one}[\text{OF } \text{assms}])$

definition *fri-one-step-disagreement-challenges*

$:: \text{nat} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}$

where

fri-one-step-disagreement-challenges $\text{len } pw \ \text{layer } \text{claimed } \text{fri-dom} =$
 $\{b. \exists i < \text{len div } 2.$
fri-fold-coeff-pair
 $(\text{layer } ! \ i) \ (\text{layer } ! \ \text{fri-sibling-index } \text{len } i)$
 $(\text{fri-fold-denominator } (\text{fri-dom } ! \ i) \ pw) \neq$
fri-fold-coeff-pair
 $(\text{claimed } ! \ i) \ (\text{claimed } ! \ \text{fri-sibling-index } \text{len } i)$
 $(\text{fri-fold-denominator } (\text{fri-dom } ! \ i) \ pw) \wedge$
 $b \in \text{fri-table-local-hiding-challenges } \text{layer } \text{claimed } \text{fri-dom } \text{len } pw \ i\}$

lemma *fri-one-step-disagreement-challenges-card-bound*:

$\text{card } (\text{fri-one-step-disagreement-challenges } \text{len } pw \ \text{layer } \text{claimed } \text{fri-dom})$
 $\leq \text{CARD}('f)$

proof –

have *finite-univ*: $\text{finite } (\text{UNIV} :: 'f \text{ set})$

by *simp*

have $\text{card } (\text{fri-one-step-disagreement-challenges } \text{len } pw \ \text{layer } \text{claimed } \text{fri-dom})$
 $\leq \text{card } (\text{UNIV} :: 'f \text{ set})$

by $(\text{rule } \text{card-mono}[\text{OF } \text{finite-univ}]) \text{ auto}$

then show *?thesis* **by** *simp*

qed

lemma *fri-one-step-disagreement-challenges-card-bound-local*:

$\text{card } (\text{fri-one-step-disagreement-challenges } \text{len } pw \ \text{layer } \text{claimed } \text{fri-dom})$
 $\leq \text{len div } 2$

proof –

let *?S* =

$\lambda i. \text{if}$
fri-fold-coeff-pair
 $(\text{layer } ! \ i) \ (\text{layer } ! \ \text{fri-sibling-index } \text{len } i)$
 $(\text{fri-fold-denominator } (\text{fri-dom } ! \ i) \ pw) \neq$

```

    fri-fold-coeff-pair
      (claimed ! i) (claimed ! fri-sibling-index len i)
      (fri-fold-denominator (fri-dom ! i) pw)
    then fri-table-local-hiding-challenges layer claimed fri-dom len pw i
    else {}
  have set-eq:
    fri-one-step-disagreement-challenges len pw layer claimed fri-dom =
      ( $\bigcup i \in \{..<len \text{ div } 2\}. ?S i$ )
    unfolding fri-one-step-disagreement-challenges-def by auto
  have local-bound:
     $\bigwedge i. i < len \text{ div } 2 \implies \text{card } (?S i) \leq 1$ 
  proof -
    fix i
    assume i < len div 2
    show card (?S i) ≤ 1
    proof (cases
      fri-fold-coeff-pair
        (layer ! i) (layer ! fri-sibling-index len i)
        (fri-fold-denominator (fri-dom ! i) pw) ≠
      fri-fold-coeff-pair
        (claimed ! i) (claimed ! fri-sibling-index len i)
        (fri-fold-denominator (fri-dom ! i) pw))
      case True
      then have card
        (fri-table-local-hiding-challenges layer claimed fri-dom len pw i)
        ≤ 1
        by (rule fri-table-local-hiding-challenges-card-le-one)
      then show ?thesis
        using True by simp
    next
      case False
      then show ?thesis by simp
    qed
  qed
  have card (fri-one-step-disagreement-challenges len pw layer claimed fri-dom) =
    card ( $\bigcup i \in \{..<len \text{ div } 2\}. ?S i$ )
    using set-eq by simp
  also have ... ≤ ( $\sum i \in \{..<len \text{ div } 2\}. \text{card } (?S i)$ )
    by (rule card-UN-le) simp
  also have ... ≤ ( $\sum i \in \{..<len \text{ div } 2\}. 1$ )
    by (rule sum-mono) (rule local-bound, simp)
  also have ... = len div 2
    by simp
  finally show ?thesis .
qed

definition fri-multiround-bad-challenge-lists
  :: nat ⇒ (nat ⇒ 'f list ⇒ 'f set) ⇒ 'f list set
  where

```

fri-multiround-bad-challenge-lists *n* *bad* =
 $\{bs \in \text{fri-challenge-space } n. \\ \exists i < n. bs ! i \in \text{bad } i \text{ (take } i \text{ bs)}\}$

definition *fri-bad-challenge-lists-bounded-by*
 $:: \text{nat} \Rightarrow 'f \text{ list set} \Rightarrow \text{nat} \Rightarrow \text{bool}$
where
fri-bad-challenge-lists-bounded-by *n* *bad* *bd* $\longleftrightarrow$
 $\text{bad} \subseteq \text{fri-challenge-space } n \wedge \text{card bad} \leq \text{bd}$

lemma *fri-multiround-bad-challenge-lists-subset*:
fri-multiround-bad-challenge-lists *n* *bad* $\subseteq \text{fri-challenge-space } n$
unfolding *fri-multiround-bad-challenge-lists-def* **by** *auto*

lemma *fri-multiround-bad-challenge-lists-card-bound*:
 $\text{card (fri-multiround-bad-challenge-lists } n \text{ bad)} \leq \text{CARD('f)} \wedge n$
proof –
have $\text{card (fri-multiround-bad-challenge-lists } n \text{ bad)} \leq$
 $\text{card (fri-challenge-space } n)$
by (rule *card-mono[OF finite-fri-challenge-space]*)
(auto simp: *fri-multiround-bad-challenge-lists-subset*)
also have $\dots = \text{CARD('f)} \wedge n$
by (rule *card-fri-challenge-space*)
finally show *?thesis* .
qed

lemma *fri-multiround-bad-challenge-lists-bounded-by*:
fri-bad-challenge-lists-bounded-by *n*
 $(\text{fri-multiround-bad-challenge-lists } n \text{ bad}) (\text{CARD('f)} \wedge n)$
unfolding *fri-bad-challenge-lists-bounded-by-def*
using *fri-multiround-bad-challenge-lists-subset*
fri-multiround-bad-challenge-lists-card-bound
by *simp*

definition *fri-final-constant-consistent* $:: 'f \text{ list} \Rightarrow 'f \Rightarrow \text{bool}$
where
fri-final-constant-consistent *layer* *final* $\longleftrightarrow$
 $(\forall x \in \text{set layer. } x = \text{final})$

lemma *fri-final-constant-consistent-nth*:
assumes *fri-final-constant-consistent* *layer* *final*
and $i < \text{length layer}$
shows $\text{layer} ! i = \text{final}$
using *assms* **unfolding** *fri-final-constant-consistent-def*
by (simp add: *nth-mem*)

lemma *fri-final-constant-consistent-hd*:
assumes *fri-final-constant-consistent* *layer* *final*
and $\text{layer} \neq []$

shows $hd\ layer = final$
using $assms\ fri-final-constant-consistent-nth[of\ layer\ final\ 0]$
by $(cases\ layer)\ auto$

definition $fri-table-low-degree-on :: nat \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow bool$
where
 $fri-table-low-degree-on\ d\ fri-dom\ table \longleftrightarrow$
 $(\exists p. degree\ p \leq d \wedge table = map\ (poly\ p)\ fri-dom)$

fun $fri-degree-after :: nat \Rightarrow nat \Rightarrow nat$
where
 $fri-degree-after\ 0\ d = d$
 $| fri-degree-after\ (Suc\ n)\ d = fri-degree-after\ n\ d\ div\ 2$

lemma $fri-degree-after-Suc[simp]$:
 $fri-degree-after\ (Suc\ n)\ d = fri-degree-after\ n\ d\ div\ 2$
by $simp$

definition $fri-one-round-proximity-step$
 $:: nat \Rightarrow nat \Rightarrow nat \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow$
 $'f\ list \Rightarrow 'f\ list \Rightarrow 'f\ set \Rightarrow bool$
where
 $fri-one-round-proximity-step\ d\ len\ pw\ fri-dom\ layer\ next-dom\ next-layer\ bad \longleftrightarrow$
 $(\forall b. b \notin bad \longrightarrow$
 $fri-algebraic-layer-relation\ len\ pw\ b\ fri-dom\ layer\ next-dom\ next-layer \longrightarrow$
 $fri-table-low-degree-on\ (d\ div\ 2)\ next-dom\ next-layer \longrightarrow$
 $fri-table-low-degree-on\ d\ fri-dom\ layer)$

lemma $fri-one-round-proximity-step-bad-or-next-not-low$:
assumes $step$:
 $fri-one-round-proximity-step\ d\ len\ pw\ fri-dom\ layer\ next-dom\ next-layer\ bad$
and $relation$:
 $fri-algebraic-layer-relation\ len\ pw\ b\ fri-dom\ layer\ next-dom\ next-layer$
and $current-not-low$: $\neg fri-table-low-degree-on\ d\ fri-dom\ layer$
and $challenge-not-bad$: $b \notin bad$
shows $\neg fri-table-low-degree-on\ (d\ div\ 2)\ next-dom\ next-layer$
using $assms\ unfolding\ fri-one-round-proximity-step-def\ by\ blast$

definition $fri-multround-proximity-steps$
 $:: nat \Rightarrow nat \Rightarrow nat\ list \Rightarrow nat\ list \Rightarrow$
 $'f\ list\ list \Rightarrow 'f\ list\ list \Rightarrow$
 $(nat \Rightarrow 'f\ list \Rightarrow 'f\ set) \Rightarrow bool$
where
 $fri-multround-proximity-steps\ n\ d\ lens\ pws\ doms\ layers\ bad \longleftrightarrow$
 $length\ lens = n \wedge$
 $length\ pws = n \wedge$
 $length\ doms = Suc\ n \wedge$
 $length\ layers = Suc\ n \wedge$
 $(\forall bs \in fri-challenge-space\ n.$

$\forall i < n.$
 $\text{fri-one-round-proximity-step } (\text{fri-degree-after } i \ d)$
 $(\text{lens} ! i) (\text{pws} ! i) (\text{doms} ! i) (\text{layers} ! i)$
 $(\text{doms} ! \text{Suc } i) (\text{layers} ! \text{Suc } i) (\text{bad } i (\text{take } i \ bs)))$

lemma *fri-multi-round-bad-challenge-lists-notinD*:
assumes *challenges*: $bs \in \text{fri-challenge-space } n$
and *not-bad*: $bs \notin \text{fri-multi-round-bad-challenge-lists } n \ \text{bad}$
and *i-bound*: $i < n$
shows $bs ! i \notin \text{bad } i (\text{take } i \ bs)$
using *assms* **unfolding** *fri-multi-round-bad-challenge-lists-def* **by** *blast*

lemma *fri-multi-round-proximity-skeleton*:
assumes *steps*: *fri-multi-round-proximity-steps* $n \ d \ \text{lens} \ \text{pws} \ \text{doms} \ \text{layers} \ \text{bad}$
and *challenges*: $bs \in \text{fri-challenge-space } n$
and *relations*:
 $\bigwedge i. i < n \implies$
 $\text{fri-algebraic-layer-relation } (\text{lens} ! i) (\text{pws} ! i) (\text{bs} ! i)$
 $(\text{doms} ! i) (\text{layers} ! i) (\text{doms} ! \text{Suc } i) (\text{layers} ! \text{Suc } i)$
and *start-not-low*:
 $\neg \text{fri-table-low-degree-on } d (\text{doms} ! 0) (\text{layers} ! 0)$
and *final-low*:
 $\text{fri-table-low-degree-on } (\text{fri-degree-after } n \ d) (\text{doms} ! n) (\text{layers} ! n)$
shows $bs \in \text{fri-multi-round-bad-challenge-lists } n \ \text{bad}$
proof (*rule ccontr*)
assume *not-bad*: $bs \notin \text{fri-multi-round-bad-challenge-lists } n \ \text{bad}$
have *step-at*:
 $\bigwedge i. i < n \implies$
 $\text{fri-one-round-proximity-step } (\text{fri-degree-after } i \ d)$
 $(\text{lens} ! i) (\text{pws} ! i) (\text{doms} ! i) (\text{layers} ! i)$
 $(\text{doms} ! \text{Suc } i) (\text{layers} ! \text{Suc } i) (\text{bad } i (\text{take } i \ bs))$
using *steps* *challenges* **unfolding** *fri-multi-round-proximity-steps-def* **by** *blast*
have *not-low*:
 $\bigwedge i. i \leq n \implies$
 $\neg \text{fri-table-low-degree-on } (\text{fri-degree-after } i \ d) (\text{doms} ! i) (\text{layers} ! i)$
proof –
fix *i*
assume $i \leq n$
then show
 $\neg \text{fri-table-low-degree-on } (\text{fri-degree-after } i \ d) (\text{doms} ! i) (\text{layers} ! i)$
proof (*induction i*)
case 0
then show ?*case*
using *start-not-low* **by** *simp*
next
case (*Suc i*)
have *i-bound*: $i < n$
using *Suc.premis* **by** *simp*
have *prev-not-low*:

```

    ¬ fri-table-low-degree-on (fri-degree-after i d)
      (doms ! i) (layers ! i)
    using Suc.IH Suc.prem by simp
  have challenge-not-bad: bs ! i ∉ bad i (take i bs)
    by (rule fri-multiround-bad-challenge-lists-notinD
        [OF challenges not-bad i-bound])
  have next-not-low:
    ¬ fri-table-low-degree-on (fri-degree-after i d div 2)
      (doms ! Suc i) (layers ! Suc i)
    by (rule fri-one-round-proximity-step-bad-or-next-not-low
        [OF step-at[OF i-bound] relations[OF i-bound] prev-not-low
         challenge-not-bad])
  then show ?case
    by simp
qed
qed
have ¬ fri-table-low-degree-on (fri-degree-after n d) (doms ! n) (layers ! n)
  by (rule not-low) simp
then show False
  using final-low by contradiction
qed

definition fri-round-count-for-degree-bound :: nat ⇒ nat
  where fri-round-count-for-degree-bound d = ceil-log (d + 1)

definition trace-fri-algebraic-round-count :: nat
  where
    trace-fri-algebraic-round-count =
      fri-round-count-for-degree-bound (clength - 1)

definition composition-fri-algebraic-round-count :: 'f ⇒ nat
  where
    composition-fri-algebraic-round-count dg =
      fri-round-count-for-degree-bound (to-nat dg)

lemma trace-fri-algebraic-round-count-eq:
  trace-fri-algebraic-round-count = ceil-log clength
unfolding trace-fri-algebraic-round-count-def
  fri-round-count-for-degree-bound-def
using clength-pos by simp

lemma composition-fri-algebraic-round-count-eq:
  composition-fri-algebraic-round-count dg =
    ceil-log (to-nat dg + 1)
unfolding composition-fri-algebraic-round-count-def
  fri-round-count-for-degree-bound-def
by simp

lemma trace-fri-algebraic-challenge-space:

```

fri-challenge-space trace-fri-algebraic-round-count =
fri-challenge-space (*ceil-log clength*)
by (*simp add: trace-fri-algebraic-round-count-eq*)

lemma *composition-fri-algebraic-challenge-space*:
fri-challenge-space (*composition-fri-algebraic-round-count dg*) =
fri-challenge-space (*ceil-log (to-nat dg + 1)*)
by (*simp add: composition-fri-algebraic-round-count-eq*)

definition *trace-fri-multiround-bad-sets*
 $:: ('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list set}$
where
trace-fri-multiround-bad-sets bad trace-table =
fri-multiround-bad-challenge-lists trace-fri-algebraic-round-count
(*bad trace-table*)

definition *composition-fri-multiround-bad-sets*
 $:: ('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list set}$
where
composition-fri-multiround-bad-sets bad dg composition-table =
fri-multiround-bad-challenge-lists
(*composition-fri-algebraic-round-count dg*)
(*bad dg composition-table*)

lemma *trace-fri-multiround-bad-sets-subset*:
trace-fri-multiround-bad-sets bad trace-table $\subseteq$
fri-challenge-space (*ceil-log clength*)
proof –
have *trace-fri-multiround-bad-sets bad trace-table* $\subseteq$
fri-challenge-space trace-fri-algebraic-round-count
unfolding *trace-fri-multiround-bad-sets-def*
by (*rule fri-multiround-bad-challenge-lists-subset*)
also have ... = *fri-challenge-space* (*ceil-log clength*)
by (*rule trace-fri-algebraic-challenge-space*)
finally show ?thesis .
qed

lemma *composition-fri-multiround-bad-sets-subset*:
composition-fri-multiround-bad-sets bad dg composition-table $\subseteq$
fri-challenge-space (*ceil-log (to-nat dg + 1)*)
proof –
have *composition-fri-multiround-bad-sets bad dg composition-table* $\subseteq$
fri-challenge-space (*composition-fri-algebraic-round-count dg*)
unfolding *composition-fri-multiround-bad-sets-def*
by (*rule fri-multiround-bad-challenge-lists-subset*)
also have ... = *fri-challenge-space* (*ceil-log (to-nat dg + 1)*)
by (*rule composition-fri-algebraic-challenge-space*)

finally show *?thesis* .
qed

lemma *trace-fri-multiround-bad-sets-card-bound*:
 $\text{card } (\text{trace-fri-multiround-bad-sets bad trace-table}) \leq$
 $\text{CARD('f)} \wedge \text{ceil-log clength}$
unfolding *trace-fri-multiround-bad-sets-def*
using *fri-multiround-bad-challenge-lists-card-bound*
by (*simp add: trace-fri-algebraic-round-count-eq*)

lemma *composition-fri-multiround-bad-sets-card-bound*:
 $\text{card } (\text{composition-fri-multiround-bad-sets bad dg composition-table}) \leq$
 $\text{CARD('f)} \wedge \text{ceil-log (to-nat dg + 1)}$
unfolding *composition-fri-multiround-bad-sets-def*
using *fri-multiround-bad-challenge-lists-card-bound*
by (*simp add: composition-fri-algebraic-round-count-eq*)

definition *accepted-fri-challenges*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

accepted-fri-challenges s out trace-bs dg comp-bs $\longleftrightarrow$
 $(\exists \text{ result final-state fr f-fl f-final as fl final query-state.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{verifier-header-transcript s fr (map snd f-fl) f-final as dg}$
 $(\text{map snd fl}) \text{ final (PTranscript query-state)} \wedge$
 $\text{PState query-state} =$
 $\text{verifier-header-state s fr (map snd f-fl) f-final as dg}$
 $(\text{map snd fl}) \text{ final} \wedge$
 $\text{Some (result, final-state)} \in$
 set-dist (execute
 $(\text{ntimes}$
 $(\text{verifier-query-round-program fr f-fl f-final as fl final})$
 $\text{rounds})}$
 $\text{query-state}) \wedge$
 $\text{PQueryCounter query-state} = \text{PQueryCounter s} \wedge$
 $\text{trace-bs} = \text{map fst f-fl} \wedge$
 $\text{comp-bs} = \text{map fst fl} \wedge$
 $(\forall i < \text{length f-fl.}$
 $\text{fmlookup (HashMap query-state)}$
 $(\text{TraceFriChallenge (PTraceFriCounter s + i) (foldl concat (concat (PState s) fr)}$
 $(\text{take (Suc i) (map snd f-fl))})) =$
 $\text{Some (fst (f-fl ! i))}) \wedge$
 $(\forall i < \text{length fl.}$
 $\text{fmlookup (HashMap query-state)}$
 $(\text{CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat}$
 $(\text{concat}$
 $(\text{foldl concat}$

```

      (concat
        (foldl concat (concat (PState s) fr) (map snd f-fl))
        f-final)
      as)
    dg)
  (take (Suc i) (map snd fl)))) =
  Some (fst (fl ! i)))

```

lemma *verify-monad-accepted-fri-challenges*:

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes *outcome*:

$\text{Some (result, final-state)} \in \text{set-dist (execute verify-monad s)}$

obtains *trace-bs dg comp-bs*

where *accepted-fri-challenges s (Some (result, final-state))*

trace-bs dg comp-bs

proof –

from *verify-monad-header-random-oracle-replay[OF outcome]*

obtain *fr f-fl f-final as dg fl final query-state* **where**

header:

verifier-header-transcript s fr (map snd f-fl) f-final as dg
(map snd fl) final (PTranscript query-state)

and *query-state-eq*:

PState query-state =
verifier-header-state s fr (map snd f-fl) f-final as dg
(map snd fl) final

and *query-out*:

Some (result, final-state) ∈
set-dist (execute
(ntimes
(verifier-query-round-program fr f-fl f-final as fl final)
rounds)
query-state)

and *query-count-header*:

PQueryCounter query-state = PQueryCounter s

and *ext-s-query-header: s ≤ query-state*

and *trace-lookup*:

$\bigwedge i. i < \text{length } f\text{-fl} \implies$
fmlookup (HashMap query-state)
(TraceFriChallenge (PTraceFriCounter s + i) (foldl concat (concat (PState s) fr)
(take (Suc i) (map snd f-fl)))) =
Some (fst (f-fl ! i))

and *alpha-lookup*:

$\bigwedge i. i < \text{length } as \implies$
fmlookup (HashMap query-state)
(AlphaChallenge (PAlphaCounter s + i) (foldl concat
(concat (foldl concat (concat (PState s) fr) (map snd f-fl))
f-final)
(take i as)))) =
Some (as ! i)

```

and comp-lookup:
   $\bigwedge i. i < \text{length } fl \implies$ 
    fmlookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) (map snd f-fl))
              f-final)
            as)
          dg)
        (take (Suc i) (map snd fl)))))) =
      Some (fst (fl ! i))
and degree-bound: to-nat dg  $\leq$  maxDegree
by blast
have accepted-fri-challenges s (Some (result, final-state))
  (map fst f-fl) dg (map fst fl)
proof -
have trace-lookup-all:
   $\forall i < \text{length } f\text{-}fl.$ 
    fmlookup (HashMap query-state)
      (TraceFriChallenge (PTraceFriCounter s + i) (foldl concat (concat (PState s) fr)
        (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i))
using trace-lookup by blast
have comp-lookup-all:
   $\forall i < \text{length } fl.$ 
    fmlookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) (map snd f-fl))
              f-final)
            as)
          dg)
        (take (Suc i) (map snd fl)))))) =
      Some (fst (fl ! i))
using comp-lookup by blast
show ?thesis
unfolding accepted-fri-challenges-def
by (intro exI[of - result] exI[of - final-state] exI[of - fr]
  exI[of - f-fl] exI[of - f-final] exI[of - as] exI[of - fl]
  exI[of - final] exI[of - query-state])
  (use header query-state-eq query-out query-count-header trace-lookup-all
  comp-lookup-all in simp)
qed
then show ?thesis
by (rule that)

```

qed

lemma *accepted-fri-challenges-trace-space*:
 assumes *accepted-fri-challenges s out trace-bs dg comp-bs*
 shows *trace-bs* $\in$ *fri-challenge-space* (*ceil-log clength*)
 using *assms*
 unfolding *accepted-fri-challenges-def verifier-header-transcript-def*
 fri-challenge-space-def
 by *auto*

lemma *accepted-fri-challenges-composition-space*:
 assumes *accepted-fri-challenges s out trace-bs dg comp-bs*
 shows *comp-bs* $\in$ *fri-challenge-space* (*ceil-log (to-nat dg + 1)*)
 using *assms*
 unfolding *accepted-fri-challenges-def verifier-header-transcript-def*
 fri-challenge-space-def
 by *auto*

lemma *accepted-fri-challenges-unique*:
 assumes *c1*:
 accepted-fri-challenges s (Some (result, final-state))
 trace-bs dg comp-bs
 and *c2*:
 accepted-fri-challenges s (Some (result, final-state))
 trace-bs' dg' comp-bs'
 shows *trace-bs' = trace-bs* $\wedge$ *dg' = dg* $\wedge$ *comp-bs' = comp-bs*
proof –
 from *c1* **obtain** *fr f-fl f-final as fl final query-state* **where**
 header1:
 verifier-header-transcript s fr (map snd f-fl) f-final as dg
 (*map snd fl*) *final (PTranscript query-state)*
 and *query-state1*:
 PState query-state =
 verifier-header-state s fr (map snd f-fl) f-final as dg
 (*map snd fl*) *final*
 and *query-out1*:
 Some (result, final-state) $\in$
 set-dist (execute
 (*ntimes*
 (*verifier-query-round-program fr f-fl f-final as fl final*)
 rounds)
 query-state)
 and *trace-bs-eq*: *trace-bs = map fst f-fl*
 and *comp-bs-eq*: *comp-bs = map fst fl*
 and *trace-lookup1*:
 $\bigwedge i. i < \text{length } f\text{-}fl \implies$
 fmlookup (HashMap query-state)
 (*TraceFriChallenge (PTraceFriCounter s + i)*
 (*foldl concat (concat (PState s) fr)*)


```

      (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
and comp-lookup1:
   $\bigwedge i. i < \text{length } fl \implies$ 
    fmllookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i)
        (foldl concat
          (concat
            (foldl concat
              (concat
                (foldl concat (concat (PState s) fr) (map snd f-fl))
                f-final)
              as)
            dg)
          (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
unfolding accepted-fri-challenges-def by blast
from c2 obtain fr' f-fl' f-final' as' fl' final' query-state' where
  header2:
    verifier-header-transcript s fr' (map snd f-fl') f-final' as' dg'
    (map snd fl') final' (PTranscript query-state')
and query-state2:
  PState query-state' =
    verifier-header-state s fr' (map snd f-fl') f-final' as' dg'
    (map snd fl') final'
and query-out2:
  Some (result, final-state)  $\in$ 
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr' f-fl' f-final' as' fl' final')
        rounds)
      query-state')
and trace-bs'-eq: trace-bs' = map fst f-fl'
and comp-bs'-eq: comp-bs' = map fst fl'
and trace-lookup2:
   $\bigwedge i. i < \text{length } f-fl' \implies$ 
    fmllookup (HashMap query-state')
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr')
          (take (Suc i) (map snd f-fl')))) =
    Some (fst (f-fl' ! i))
and comp-lookup2:
   $\bigwedge i. i < \text{length } fl' \implies$ 
    fmllookup (HashMap query-state')
      (CompositionFriChallenge (PCompositionFriCounter s + i)
        (foldl concat
          (concat
            (foldl concat
              (concat

```

```

      (foldl concat (concat (PState s) fr') (map snd f-fl'))
      f-final')
    as')
  dg')
  (take (Suc i) (map snd fl')))) =
  Some (fst (fl' ! i))
unfolding accepted-fri-challenges-def by blast
have header-unique:
  fr' = fr ∧
  map snd f-fl' = map snd f-fl ∧
  f-final' = f-final ∧
  as' = as ∧
  dg' = dg ∧
  map snd fl' = map snd fl ∧
  final' = final ∧
  PTranscript query-state' = PTranscript query-state
by (rule verifier-header-transcript-unique[OF header1 header2])
have ext1: query-state ≤ final-state
  using ntimes-verifier-query-rounds-outcome[OF query-out1] by blast
have ext2: query-state' ≤ final-state
  using ntimes-verifier-query-rounds-outcome[OF query-out2] by blast
have len-f-fl': length f-fl' = length f-fl
  using header-unique by (metis length-map)
have f-fl-fst-eq: map fst f-fl' = map fst f-fl
proof (rule nth-equalityI)
  show length (map fst f-fl') = length (map fst f-fl)
  using len-f-fl' by simp
next
fix i
assume i-bound: i < length (map fst f-fl')
then have i-bound': i < length f-fl'
  by simp
have i-bound-old: i < length f-fl
  using i-bound' len-f-fl' by simp
let ?key =
  TraceFriChallenge (PTraceFriCounter s + i)
  (foldl concat (concat (PState s) fr)
  (take (Suc i) (map snd f-fl)))
have lookup1-final:
  fmlookup (HashMap final-state) ?key =
  Some (fst (f-fl ! i))
  by (rule hash-extension-lookup[OF trace-lookup1[OF i-bound-old] ext1])
have key-eq:
  TraceFriChallenge (PTraceFriCounter s + i)
  (foldl concat (concat (PState s) fr')
  (take (Suc i) (map snd f-fl')))) = ?key
  using header-unique by simp
have lookup2-final:
  fmlookup (HashMap final-state) ?key =

```

```

    Some (fst (f-fl' ! i))
  using hash-extension-lookup[OF trace-lookup2[OF i-bound'] ext2]
  unfolding key-eq .
show map fst f-fl' ! i = map fst f-fl ! i
  using lookup1-final lookup2-final i-bound' i-bound-old by simp
qed
have len-fl': length fl' = length fl
  using header-unique by (metis length-map)
have fl-fst-eq: map fst fl' = map fst fl
proof (rule nth-equalityI)
  show length (map fst fl') = length (map fst fl)
    using len-fl' by simp
next
fix i
assume i-bound: i < length (map fst fl')
then have i-bound': i < length fl'
  by simp
have i-bound-old: i < length fl
  using i-bound' len-fl' by simp
let ?key =
  CompositionFriChallenge (PCompositionFriCounter s + i)
  (foldl concat
    (concat
      (foldl concat
        (concat
          (foldl concat (concat (PState s) fr) (map snd f-fl))
          f-final)
        as)
      dg)
    (take (Suc i) (map snd fl)))
have lookup1-final:
  fmllookup (HashMap final-state) ?key =
    Some (fst (fl ! i))
  by (rule hash-extension-lookup[OF comp-lookup1[OF i-bound-old] ext1])
have key-eq:
  CompositionFriChallenge (PCompositionFriCounter s + i)
  (foldl concat
    (concat
      (foldl concat
        (concat
          (foldl concat (concat (PState s) fr') (map snd f-fl'))
          f-final')
        as')
      dg')
    (take (Suc i) (map snd fl'))) = ?key
  using header-unique by simp
have lookup2-final:
  fmllookup (HashMap final-state) ?key =
    Some (fst (fl' ! i))

```

```

    using hash-extension-lookup[OF comp-lookup2[OF i-bound'] ext2]
    unfolding key-eq .
show map fst fl' ! i = map fst fl ! i
    using lookup1-final lookup2-final i-bound' i-bound-old by simp
qed
show ?thesis
    using header-unique trace-bs-eq trace-bs'-eq comp-bs-eq comp-bs'-eq
    f-fl-fst-eq fl-fst-eq
    by simp
qed

lemma verify-monad-trace-fri-prequery-lookup:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and challenges:
    accepted-fri-challenges s (Some (result, final-state))
    trace-bs dg comp-bs
  and header:
    verifier-header-transcript s fr roots trace-final as dg
    composition-roots final rest
  and i-len: i < length roots
  and lookup-s:
    fmlookup (HashMap s)
    (TraceFriChallenge (PTraceFriCounter s + i)
    (foldl concat (concat (PState s) fr) (take (Suc i) roots))) =
    Some y
  shows i < length trace-bs ∧ y = trace-bs ! i
proof (rule verify-monad-header-random-oracle-replay[OF outcome])
  fix fr' f-fl trace-final' as' dg' fl' final' query-state
  assume header-replay:
    verifier-header-transcript s fr' (map snd f-fl) trace-final' as' dg'
    (map snd fl') final' (PTranscript query-state)
  and query-state-eq:
    PState query-state =
    verifier-header-state s fr' (map snd f-fl) trace-final' as' dg'
    (map snd fl') final'
  and query-out:
    Some (result, final-state) ∈
    set-dist
    (execute
    (ntimes
    (verifier-query-round-program fr' f-fl trace-final' as' fl'
    final')
    rounds)
    query-state)
  and query-count-header: PQueryCounter query-state = PQueryCounter s
  and ext-s-query: s ≤ query-state
  and trace-lookup:

```

```

 $\bigwedge j. j < \text{length } f\text{-}fl \implies$ 
   $\text{fmlookup } (\text{HashMap } \text{query-state})$ 
     $(\text{TraceFriChallenge } (P\text{TraceFriCounter } s + j)$ 
       $(\text{foldl } \text{concat } (\text{concat } (P\text{State } s) \text{ fr}')$ 
         $(\text{take } (\text{Suc } j) (\text{map } \text{snd } f\text{-}fl)))) =$ 
       $\text{Some } (\text{fst } (f\text{-}fl ! j))$ 
and comp-lookup:
 $\bigwedge j. j < \text{length } fl' \implies$ 
   $\text{fmlookup } (\text{HashMap } \text{query-state})$ 
     $(\text{CompositionFriChallenge } (P\text{CompositionFriCounter } s + j)$ 
       $(\text{foldl } \text{concat}$ 
         $(\text{concat}$ 
           $(\text{foldl } \text{concat}$ 
             $(\text{concat}$ 
               $(\text{foldl } \text{concat } (\text{concat } (P\text{State } s) \text{ fr}') (\text{map } \text{snd } f\text{-}fl))$ 
               $\text{trace-final'})$ 
             $\text{as}')$ 
           $\text{dg}')$ 
         $(\text{take } (\text{Suc } j) (\text{map } \text{snd } fl')))) =$ 
       $\text{Some } (\text{fst } (fl' ! j))$ 
and degree-bound:  $\text{to-nat } dg' \leq \text{maxDegree}$ 
have replay-challenges:
   $\text{accepted-fri-challenges } s (\text{Some } (\text{result}, \text{final-state}))$ 
     $(\text{map } \text{fst } f\text{-}fl) \text{ dg' } (\text{map } \text{fst } fl')$ 
unfolding accepted-fri-challenges-def
by  $(\text{intro } \text{exI}[\text{of - result}] \text{ exI}[\text{of - final-state}] \text{ exI}[\text{of - fr'}]$ 
   $\text{exI}[\text{of - f-fl}] \text{ exI}[\text{of - trace-final'}] \text{ exI}[\text{of - as'}]$ 
   $\text{exI}[\text{of - fl'}] \text{ exI}[\text{of - final'}] \text{ exI}[\text{of - query-state}])$ 
   $(\text{use } \text{header-replay } \text{query-state-eq } \text{query-out } \text{query-count-header}$ 
     $\text{trace-lookup } \text{comp-lookup } \text{in } \text{auto})$ 
have trace-bs-eq:  $\text{trace-bs} = \text{map } \text{fst } f\text{-}fl$ 
using accepted-fri-challenges-unique[OF replay-challenges challenges]
by simp
have header-unique:
   $\text{fr} = \text{fr}' \wedge$ 
   $\text{roots} = \text{map } \text{snd } f\text{-}fl \wedge$ 
   $\text{trace-final} = \text{trace-final}' \wedge$ 
   $\text{as} = \text{as}' \wedge$ 
   $\text{dg} = \text{dg}' \wedge$ 
   $\text{composition-roots} = \text{map } \text{snd } fl' \wedge$ 
   $\text{final} = \text{final}' \wedge$ 
   $\text{rest} = P\text{Transcript } \text{query-state}$ 
using verifier-header-transcript-unique[OF header header-replay]
by simp
have i-f-fl:  $i < \text{length } f\text{-}fl$ 
using i-len header-unique by simp
have lookup-query-from-s:
   $\text{fmlookup } (\text{HashMap } \text{query-state})$ 
     $(\text{TraceFriChallenge } (P\text{TraceFriCounter } s + i)$ 

```

```

      (foldl concat (concat (PState s) fr) (take (Suc i) roots))) =
    Some y
  by (rule hash-extension-lookup[OF lookup-s ext-s-query])
have lookup-query-from-trace:
  fmlookup (HashMap query-state)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr) (take (Suc i) roots))) =
    Some (trace-bs ! i)
  using trace-lookup[OF i-f-fl] header-unique trace-bs-eq i-f-fl
  by simp
then have y-eq: y = trace-bs ! i
  using lookup-query-from-s by simp
have i < length trace-bs
  using i-f-fl trace-bs-eq by simp
then show ?thesis
  using y-eq by simp
qed

lemma verify-monad-composition-fri-prequery-lookup:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and challenges:
  accepted-fri-challenges s (Some (result, final-state))
  trace-bs dg comp-bs
and header:
  verifier-header-transcript s fr trace-roots trace-final as dg
  composition-roots final rest
and i-len: i < length composition-roots
and lookup-s:
  fmlookup (HashMap s)
    (CompositionFriChallenge (PCompositionFriCounter s + i)
      (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) trace-roots)
              trace-final)
            as)
          dg)
        (take (Suc i) composition-roots))) =
    Some y
  shows i < length comp-bs ∧ y = comp-bs ! i
proof (rule verify-monad-header-random-oracle-replay[OF outcome])
fix fr' f-fl trace-final' as' dg' fl' final' query-state
assume header-replay:
  verifier-header-transcript s fr' (map snd f-fl) trace-final' as' dg'
  (map snd fl') final' (PTranscript query-state)
and query-state-eq:

```

```

PState query-state =
  verifier-header-state s fr' (map snd f-fl) trace-final' as' dg'
  (map snd fl') final'
and query-out:
  Some (result, final-state) ∈
    set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr' f-fl trace-final' as' fl'
          final')
        rounds)
      query-state)
and query-count-header: PQueryCounter query-state = PQueryCounter s
and ext-s-query: s ≤ query-state
and trace-lookup:
   $\bigwedge j. j < \text{length } f\text{-}fl \implies$ 
    fmlookup (HashMap query-state)
      (TraceFriChallenge (PTraceFriCounter s + j)
        (foldl concat (concat (PState s) fr')
          (take (Suc j) (map snd f-fl)))) =
      Some (fst (f-fl ! j))
and comp-lookup:
   $\bigwedge j. j < \text{length } fl' \implies$ 
    fmlookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + j)
        (foldl concat
          (concat
            (foldl concat
              (concat
                (foldl concat (concat (PState s) fr') (map snd f-fl))
                trace-final')
              as')
            dg')
          (take (Suc j) (map snd fl')))) =
      Some (fst (fl' ! j))
and degree-bound: to-nat dg' ≤ maxDegree
have replay-challenges:
  accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg' (map fst fl')
unfolding accepted-fri-challenges-def
by (intro exI[of - result] exI[of - final-state] exI[of - fr']
  exI[of - f-fl] exI[of - trace-final'] exI[of - as']
  exI[of - fl'] exI[of - final'] exI[of - query-state])
  (use header-replay query-state-eq query-out query-count-header
    trace-lookup comp-lookup in auto)
have comp-bs-eq: comp-bs = map fst fl'
using accepted-fri-challenges-unique[OF replay-challenges challenges]
by simp
have header-unique:

```

```

fr = fr' ∧
trace-roots = map snd f-fl ∧
trace-final = trace-final' ∧
as = as' ∧
dg = dg' ∧
composition-roots = map snd fl' ∧
final = final' ∧
rest = PTranscript query-state
using verifier-header-transcript-unique[OF header header-replay]
by simp
have i-fl: i < length fl'
  using i-len header-unique by simp
have lookup-query-from-s:
  fmlookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter s + i)
      (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) trace-roots)
              trace-final)
            as)
          dg)
        (take (Suc i) composition-roots)))) =
    Some y
  by (rule hash-extension-lookup[OF lookup-s ext-s-query])
have lookup-query-from-comp:
  fmlookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter s + i)
      (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) trace-roots)
              trace-final)
            as)
          dg)
        (take (Suc i) composition-roots)))) =
    Some (comp-bs ! i)
  using comp-lookup[OF i-fl] header-unique comp-bs-eq i-fl
  by simp
then have y-eq: y = comp-bs ! i
  using lookup-query-from-s by simp
have i < length comp-bs
  using i-fl comp-bs-eq by simp
then show ?thesis
  using y-eq by simp
qed

```


lemma *verify-monad-accepted-fri-challenges-degree-bound:*
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes *outcome:*
 $\text{Some (result, final-state)} \in \text{set-dist (execute verify-monad s)}$
and *challenges:*
 $\text{accepted-fri-challenges s (Some (result, final-state))}$
 $\text{trace-bs dg comp-bs}$
shows $\text{to-nat dg} \leq \text{maxDegree}$
proof –
from *verify-monad-header-random-oracle-replay*[$OF \text{ outcome}$]
obtain $\text{fr f-fl f-final as dg' fl final query-state}$ **where**
header:
 $\text{verifier-header-transcript s fr (map snd f-fl) f-final as dg'}$
 $\text{(map snd fl) final (PTranscript query-state)}$
and *query-state-eq:*
 $\text{PState query-state} =$
 $\text{verifier-header-state s fr (map snd f-fl) f-final as dg'}$
 $\text{(map snd fl) final}$
and *query-out:*
 $\text{Some (result, final-state)} \in$
 set-dist (execute
 (ntimes
 $\text{(verifier-query-round-program fr f-fl f-final as fl final)}$
 rounds)
 query-state)
and *query-count-header:*
 $\text{PQueryCounter query-state} = \text{PQueryCounter s}$
and *trace-lookup:*
 $\bigwedge i. i < \text{length f-fl} \implies$
 $\text{fmlookup (HashMap query-state)}$
 $\text{(TraceFriChallenge (PTraceFriCounter s + i) (foldl concat}$
 $\text{(concat (PState s) fr) (take (Suc i) (map snd f-fl))))} =$
 $\text{Some (fst (f-fl ! i))}$
and *comp-lookup:*
 $\bigwedge i. i < \text{length fl} \implies$
 $\text{fmlookup (HashMap query-state)}$
 $\text{(CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat}$
 (concat
 (foldl concat
 (concat
 $\text{(foldl concat (concat (PState s) fr) (map snd f-fl))}$
 f-final)
 as)
 $\text{dg'})}$
 $\text{(take (Suc i) (map snd fl))))} =$
 $\text{Some (fst (fl ! i))}$
and *degree-bound:* $\text{to-nat dg'} \leq \text{maxDegree}$
by *metis*
have *trace-lookup-all:*

```

   $\forall i < \text{length } f\text{-fl}.$ 
  fmlookup (HashMap query-state)
    (TraceFriChallenge (PTraceFriCounter s + i) (foldl concat
      (concat (PState s) fr) (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
  using trace-lookup by blast
have comp-lookup-all:
   $\forall i < \text{length } fl.$ 
  fmlookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
      (concat
        (foldl concat
          (concat
            (foldl concat (concat (PState s) fr) (map snd f-fl))
            f-final)
          as)
        dg')
      (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
  using comp-lookup by blast
have challenges':
  accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg' (map fst fl)
  unfolding accepted-fri-challenges-def
  by (intro exI[of - result] exI[of - final-state] exI[of - fr]
    exI[of - f-fl] exI[of - f-final] exI[of - as] exI[of - fl]
    exI[of - final] exI[of - query-state])
    (use header query-state-eq query-out query-count-header
      trace-lookup-all comp-lookup-all in simp)
have dg' = dg
  using accepted-fri-challenges-unique[OF challenges challenges'] by simp
then show ?thesis
  using degree-bound by simp
qed

lemma verifier-after-trace-fri-accepted-fri-challenges:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes prefix:
    Some ((fr, f-fl), prefix-state)  $\in$ 
    set-dist (execute verifier-trace-fri-prefix s)
  and suffix:
    Some (result, final-state)  $\in$ 
    set-dist
      (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
  shows  $\exists dg \text{ comp-bs}.$ 
    accepted-fri-challenges s (Some (result, final-state))
      (map fst f-fl) dg comp-bs
proof -
  have prefix-res:

```

```

length f-fl = ceil-log clength ∧
PTranscript s = [fr] @ map snd f-fl @ PTranscript prefix-state ∧
PState prefix-state =
  foldl concat (concat (PState s) fr) (map snd f-fl) ∧
s ≤ prefix-state ∧
PTraceFriCounter prefix-state = PTraceFriCounter s + ceil-log clength ∧
PCompositionFriCounter prefix-state = PCompositionFriCounter s ∧
PAlphaCounter prefix-state = PAlphaCounter s ∧
PQueryCounter prefix-state = PQueryCounter s ∧
(∀ i < length f-fl.
  fmllookup (HashMap prefix-state)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr)
        (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i)))
by (rule verifier-trace-fri-prefix-outcome[OF prefix])
from suffix obtain f-final s3 as s4 dg s5 s6 fl s7 final query-state where
read-trace-final:
  Some (f-final, s3) ∈ set-dist (execute read prefix-state)
and alpha-out:
  Some (as, s4) ∈
    set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
and read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
and degree-assert:
  Some ((), s6) ∈
    set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
and comp-fri:
  Some (fl, s7) ∈
    set-dist (execute
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1))) s6)
and read-final: Some (final, query-state) ∈ set-dist (execute read s7)
and query-out:
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      query-state)
unfolding verifier-after-trace-fri-def
by (auto elim!: set-dist-bindE)
from read-outcome[OF read-trace-final] obtain rest3 where
tr-prefix: PTranscript prefix-state = f-final # rest3
and st-s3: PState s3 = concat (PState prefix-state) f-final
and tr-s3: PTranscript s3 = rest3
and ext-prefix-s3: prefix-state ≤ s3
and trace-count-s3: PTraceFriCounter s3 = PTraceFriCounter prefix-state
and comp-count-s3: PCompositionFriCounter s3 = PCompositionFriCounter prefix-state
and alpha-count-s3: PAlphaCounter s3 = PAlphaCounter prefix-state
and query-count-s3: PQueryCounter s3 = PQueryCounter prefix-state

```

```

    by blast
have alpha-res:
  length as = length spec ∧
  PTranscript s3 = as @ PTranscript s4 ∧
  PState s4 = foldl concat (PState s3) as ∧
  s3 ≤ s4 ∧
  PTraceFriCounter s4 = PTraceFriCounter s3 ∧
  PCompositionFriCounter s4 = PCompositionFriCounter s3 ∧
  PAlphaCounter s4 = PAlphaCounter s3 + length spec ∧
  PQueryCounter s4 = PQueryCounter s3
  using mmap-alpha-round-outcome[OF alpha-out] by simp
from read-outcome[OF read-dg] obtain rest5 where
  tr-s4: PTranscript s4 = dg # rest5
  and st-s5: PState s5 = concat (PState s4) dg
  and tr-s5: PTranscript s5 = rest5
  and ext-s4-s5: s4 ≤ s5
  and trace-count-s5: PTraceFriCounter s5 = PTraceFriCounter s4
  and comp-count-s5: PCompositionFriCounter s5 = PCompositionFriCounter s4
  and alpha-count-s5: PAlphaCounter s5 = PAlphaCounter s4
  and query-count-s5: PQueryCounter s5 = PQueryCounter s4
  by blast
have s6-eq: s6 = s5
  using degree-assert unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
have ext-s5-s6: s5 ≤ s6
  unfolding s6-eq by (rule hash-ext-refl)
have comp-res:
  length fl = ceil-log (to-nat dg + 1) ∧
  PTranscript s6 = map snd fl @ PTranscript s7 ∧
  PState s7 = foldl concat (PState s6) (map snd fl) ∧
  s6 ≤ s7 ∧
  PTraceFriCounter s7 = PTraceFriCounter s6 ∧
  PCompositionFriCounter s7 =
    PCompositionFriCounter s6 + ceil-log (to-nat dg + 1) ∧
  PAlphaCounter s7 = PAlphaCounter s6 ∧
  PQueryCounter s7 = PQueryCounter s6 ∧
  (∀ i < ceil-log (to-nat dg + 1).
    fmllookup (HashMap s7)
      (CompositionFriChallenge (PCompositionFriCounter s6 + i)
        (foldl concat (PState s6) (take (Suc i) (map snd fl)))) =
      Some (fst (fl ! i)))
  using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
  by simp
from read-outcome[OF read-final] obtain rest-query where
  tr-s7: PTranscript s7 = final # rest-query
  and st-query: PState query-state = concat (PState s7) final
  and tr-query: PTranscript query-state = rest-query
  and ext-s7-query: s7 ≤ query-state
  and query-count-query: PQueryCounter query-state = PQueryCounter s7

```

```

  by blast
have ext-prefix-query: prefix-state ≤ query-state
  using ext-prefix-s3 alpha-res ext-s4-s5 ext-s5-s6 comp-res ext-s7-query
  by (meson hash-ext-trans)
have header:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
  using prefix-res tr-prefix tr-s3 alpha-res tr-s4 tr-s5 s6-eq comp-res
    tr-s7 tr-query
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
have header-state:
  PState query-state =
    verifier-header-state s fr (map snd f-fl) f-final as dg
      (map snd fl) final
  using prefix-res st-s3 alpha-res st-s5 s6-eq comp-res st-query
  unfolding verifier-header-state-def verifier-header-messages-def
  by simp
have query-count-header:
  PQueryCounter query-state = PQueryCounter s
  using prefix-res query-count-s3 alpha-res query-count-s5 s6-eq comp-res
    query-count-query
  by simp
have trace-lookup:
  ∧i. i < length f-fl ⇒
    fmlookup (HashMap query-state)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr)
          (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i))
proof -
  fix i
  assume i-bound: i < length f-fl
  have lookup-prefix:
    fmlookup (HashMap prefix-state)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr)
          (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
  using prefix-res i-bound by simp
show fmlookup (HashMap query-state)
  (TraceFriChallenge (PTraceFriCounter s + i)
    (foldl concat (concat (PState s) fr)
      (take (Suc i) (map snd f-fl)))) =
  Some (fst (f-fl ! i))
  by (rule hash-extension-lookup[OF lookup-prefix ext-prefix-query])
qed
have comp-lookup:
  ∧i. i < length fl ⇒

```

```

fmlookup (HashMap query-state)
  (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
    (concat
      (foldl concat
        (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
        as)
      dg)
    (take (Suc i) (map snd fl)))) =
  Some (fst (fl ! i))
proof -
  fix i
  assume i-bound: i < length fl
  have lookup-s7:
    fmlookup (HashMap s7)
      (CompositionFriChallenge (PCompositionFriCounter s6 + i)
        (foldl concat (PState s6) (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
  using comp-res i-bound by simp
show fmlookup (HashMap query-state)
  (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
    (concat
      (foldl concat
        (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
        as)
      dg)
    (take (Suc i) (map snd fl)))) =
  Some (fst (fl ! i))
  using hash-extension-lookup[OF lookup-s7 ext-s7-query]
    prefix-res st-s3 alpha-res st-s5 s6-eq comp-count-s3 comp-count-s5
  by simp
qed
have challenges:
  accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg (map fst fl)
  unfolding accepted-fri-challenges-def
  apply (rule exI[where x=result])
  apply (rule exI[where x=final-state])
  apply (rule exI[where x=fr])
  apply (rule exI[where x=f-fl])
  apply (rule exI[where x=f-final])
  apply (rule exI[where x=as])
  apply (rule exI[where x=fl])
  apply (rule exI[where x=final])
  apply (rule exI[where x=query-state])
  apply (intro conjI)
    apply simp
    apply (rule header)
    apply (rule header-state)
    apply (rule query-out)

```

```

    apply (rule query-count-header)
  apply simp
  apply simp
  apply (intro allI impI, rule trace-lookup, assumption)
  apply (intro allI impI, rule comp-lookup, assumption)
done
show ?thesis
using challenges by blast
qed

```

lemma *verifier-after-composition-fri-accepted-fri-challenges:*

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes *prefix:*

Some ((fr, f-fl, f-final, as, dg, fl), prefix-state) ∈
set-dist (execute verifier-composition-fri-prefix s)

and *suffix:*

Some (result, final-state) ∈
set-dist
(execute
(verifier-after-composition-fri
(fr, f-fl, f-final, as, dg, fl))
prefix-state)

shows *accepted-fri-challenges s (Some (result, final-state))*
(map fst f-fl) dg (map fst fl)

proof –

have *prefix-res:*

length f-fl = ceil-log clength ∧
length as = length spec ∧
length fl = ceil-log (to-nat dg + 1) ∧
PTranscript s =
[fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
map snd fl @ PTranscript prefix-state ∧
PState prefix-state =
foldl concat
(concat
(foldl concat
(concat
(foldl concat (concat (PState s) fr) (map snd f-fl))
f-final)
as)
dg)
(map snd fl) ∧
s ≤ prefix-state ∧
PQueryCounter prefix-state = PQueryCounter s ∧
(∀ i < length f-fl.
fmlookup (HashMap prefix-state)
(TraceFriChallenge (PTraceFriCounter s + i)
(foldl concat (concat (PState s) fr)
(take (Suc i) (map snd f-fl)))) =

```

    Some (fst (f-fl ! i))) ∧
  (∀ i < length fl.
    fmllookup (HashMap prefix-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i)
        (foldl concat
          (concat
            (foldl concat
              (concat
                (foldl concat (concat (PState s) fr) (map snd f-fl))
                f-final)
              as)
            dg)
          (take (Suc i) (map snd fl)))))) =
    Some (fst (fl ! i)))
  by (rule verifier-composition-fri-prefix-outcome[OF prefix])
from suffix obtain final query-state where
  read-final:
    Some (final, query-state) ∈
      set-dist (execute read prefix-state)
  and query-out:
    Some (result, final-state) ∈
      set-dist (execute
        (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
        query-state)
  unfolding verifier-after-composition-fri-def
  by (auto elim!: set-dist-bindE)
from read-outcome[OF read-final] obtain rest-query where
  tr-prefix: PTranscript prefix-state = final # rest-query
  and st-query: PState query-state = concat (PState prefix-state) final
  and tr-query: PTranscript query-state = rest-query
  and ext-prefix-query: prefix-state ≤ query-state
  and query-count-query:
    PQueryCounter query-state = PQueryCounter prefix-state
  by blast
have header:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
  using prefix-res tr-prefix tr-query
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
have header-state:
  PState query-state =
    verifier-header-state s fr (map snd f-fl) f-final as dg
      (map snd fl) final
  using prefix-res st-query
  unfolding verifier-header-state-def verifier-header-messages-def
  by simp
have query-count-header:

```



```

PQueryCounter query-state = PQueryCounter s
using prefix-res query-count-query by simp
have trace-lookup:
   $\bigwedge i. i < \text{length } f\text{-fl} \implies$ 
    fmlookup (HashMap query-state)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr)
          (take (Suc i) (map snd f-fl))))) =
      Some (fst (f-fl ! i))
proof –
  fix i
  assume i-bound: i < length f-fl
  have lookup-prefix:
    fmlookup (HashMap prefix-state)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr)
          (take (Suc i) (map snd f-fl))))) =
      Some (fst (f-fl ! i))
    using prefix-res i-bound by simp
  show fmlookup (HashMap query-state)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr)
        (take (Suc i) (map snd f-fl))))) =
    Some (fst (f-fl ! i))
    by (rule hash-extension-lookup[OF lookup-prefix ext-prefix-query])
qed
have comp-lookup:
   $\bigwedge i. i < \text{length } fl \implies$ 
    fmlookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i)
        (foldl concat
          (concat
            (foldl concat
              (concat
                (foldl concat (concat (PState s) fr) (map snd f-fl))
                f-final)
              as)
            dg)
          (take (Suc i) (map snd fl))))) =
      Some (fst (fl ! i))
proof –
  fix i
  assume i-bound: i < length fl
  have lookup-prefix:
    fmlookup (HashMap prefix-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i)
        (foldl concat
          (concat
            (foldl concat
              (concat
                (foldl concat (concat (PState s) fr) (map snd f-fl))
                f-final)
              as)
            dg)
          (take (Suc i) (map snd fl))))) =
      Some (fst (fl ! i))
    using prefix-res i-bound by simp
  show fmlookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter s + i)
      (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) (map snd f-fl))
              f-final)
            as)
          dg)
        (take (Suc i) (map snd fl))))) =
    Some (fst (fl ! i))
    by (rule hash-extension-lookup[OF lookup-prefix ext-prefix-query])
qed

```

```

      (concat
        (foldl concat (concat (PState s) fr) (map snd f-fl))
        f-final)
      as)
    dg)
  (take (Suc i) (map snd fl)))) =
  Some (fst (fl ! i))
using prefix-res i-bound by simp
show fmlookup (HashMap query-state)
  (CompositionFriChallenge (PCompositionFriCounter s + i)
    (foldl concat
      (concat
        (foldl concat
          (concat
            (foldl concat (concat (PState s) fr) (map snd f-fl))
            f-final)
          as)
        dg)
      (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
    by (rule hash-extension-lookup[OF lookup-prefix ext-prefix-query])
qed
show ?thesis
  unfolding accepted-fri-challenges-def
  apply (rule exI[where x=result])
  apply (rule exI[where x=final-state])
  apply (rule exI[where x=fr])
  apply (rule exI[where x=f-fl])
  apply (rule exI[where x=f-final])
  apply (rule exI[where x=as])
  apply (rule exI[where x=fl])
  apply (rule exI[where x=final])
  apply (rule exI[where x=query-state])
  apply (intro conjI)
  apply simp
  apply (rule header)
  apply (rule header-state)
  apply (rule query-out)
  apply (rule query-count-header)
  apply simp
  apply simp
  apply (intro allI impI, rule trace-lookup, assumption)
  apply (intro allI impI, rule comp-lookup, assumption)
done
qed

```

definition trace-fri-challenge-list-set-hit

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  'f list set  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool

```

where

$\text{trace-fri-challenge-list-set-hit } s \ B \ \text{out} \longleftrightarrow$
 $(\exists \text{ trace-bs } dg \ \text{comp-bs}.$
 $\text{accepted-fri-challenges } s \ \text{out } \text{trace-bs } dg \ \text{comp-bs} \wedge$
 $\text{trace-bs} \in B)$

definition *trace-fri-root-list-set-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-root-list-set-hit } s \ B \ \text{out} \longleftrightarrow$
 $(\exists \text{ trace-bs } dg \ \text{comp-bs} \ \text{fr } f\text{-fri-roots } f\text{-final as } \text{composition-fri-roots}$
 $\text{final rest}.$
 $\text{accepted-fri-challenges } s \ \text{out } \text{trace-bs } dg \ \text{comp-bs} \wedge$
 $\text{verifier-header-transcript } s \ \text{fr } f\text{-fri-roots } f\text{-final as } dg$
 $\text{composition-fri-roots final rest} \wedge$
 $\text{trace-bs} \in B \ \text{fr})$

definition *composition-fri-challenge-list-set-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-challenge-list-set-hit } s \ B \ \text{out} \longleftrightarrow$
 $(\exists \text{ trace-bs } dg \ \text{comp-bs}.$
 $\text{accepted-fri-challenges } s \ \text{out } \text{trace-bs } dg \ \text{comp-bs} \wedge$
 $\text{comp-bs} \in B \ dg)$

definition *composition-fri-root-list-set-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-root-list-set-hit } s \ B \ \text{out} \longleftrightarrow$
 $(\exists \text{ trace-bs } dg \ \text{comp-bs} \ \text{fr } f\text{-fri-roots } f\text{-final as}$
 $\text{composition-fri-roots final rest}.$
 $\text{accepted-fri-challenges } s \ \text{out } \text{trace-bs } dg \ \text{comp-bs} \wedge$
 $\text{verifier-header-transcript } s \ \text{fr } f\text{-fri-roots } f\text{-final as } dg$
 $\text{composition-fri-roots final rest} \wedge$
 $\text{comp-bs} \in B \ \text{fr } f\text{-fri-roots } f\text{-final as } dg \ \text{composition-fri-roots})$

lemma *wp-verify-monad-trace-fri-challenge-list-set-bound:*

assumes *future:* $\text{trace-fri-future-fresh } s$

and *subset:* $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$

shows

$\text{wp-event verify-monad } (\text{trace-fri-challenge-list-set-hit } s \ B) \ s \leq$
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } \text{clength})$

proof –

```

let ?Head =
  λout. case out of
    None ⇒ False
    | Some ((-, f-fl), -) ⇒ map fst f-fl ∈ B
have prefix-bound:
  wp-event verifier-trace-fri-prefix ?Head s ≤
    nnreal (card B) / nnreal (CARD('f) ^ ceil-log clength)
by (rule wp-verifier-trace-fri-prefix-challenge-space-set-bound
    [OF future subset])
show ?thesis
  unfolding verify-monad-trace-fri-decomposition
proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
  show trace-fri-challenge-list-set-hit s B None ⇒ ?Head None
    unfolding trace-fri-challenge-list-set-hit-def
      accepted-fri-challenges-def by blast
next
  fix header prefix-state out
  assume prefix:
    Some (header, prefix-state) ∈
      set-dist (execute verifier-trace-fri-prefix s)
  and suffix:
    out ∈
      set-dist (execute (verifier-after-trace-fri header) prefix-state)
  and hit: trace-fri-challenge-list-set-hit s B out
  obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
  from hit obtain trace-bs dg comp-bs where
    challenges-hit: accepted-fri-challenges s out trace-bs dg comp-bs
  and trace-bs-B: trace-bs ∈ B
  unfolding trace-fri-challenge-list-set-hit-def by blast
  from challenges-hit obtain result final-state where out-eq:
    out = Some (result, final-state)
  unfolding accepted-fri-challenges-def by blast
  have prefix':
    Some ((fr, f-fl), prefix-state) ∈
      set-dist (execute verifier-trace-fri-prefix s)
  using prefix header-eq by simp
  have suffix':
    Some (result, final-state) ∈
      set-dist
        (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
  using suffix header-eq out-eq by simp
  from verifier-after-trace-fri-accepted-fri-challenges[OF prefix' suffix']
  obtain dg' comp-bs' where
    challenges-prefix:
      accepted-fri-challenges s (Some (result, final-state))
      (map fst f-fl) dg' comp-bs'
  by blast
  have trace-bs = map fst f-fl

```

```

    using accepted-fri-challenges-unique
      [OF challenges-prefix challenges-hit[unfolded out-eq]]
    by simp
  then show ?Head (Some (header, prefix-state))
    using trace-bs-B header-eq by simp
qed
qed

lemma wp-verify-monad-trace-fri-root-list-set-bound:
  fixes C :: prob
  assumes future: trace-fri-future-fresh s
  and subset:  $\bigwedge fr. B \text{ fr} \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$ 
  and bound:
     $\bigwedge fr. \text{nnreal } (\text{card } (B \text{ fr})) /$ 
     $\text{nnreal } (\text{CARD}'f) \wedge \text{ceil-log } \text{clength}) \leq C$ 
  shows
    wp-event verify-monad (trace-fri-root-list-set-hit s B) s  $\leq C$ 
proof -
  let ?Head =
     $\lambda out. \text{case out of}$ 
     $\text{None} \Rightarrow \text{False}$ 
     $| \text{Some } ((fr, f-fl), -) \Rightarrow \text{map fst } f-fl \in B \text{ fr}$ 
  have prefix-bound:
    wp-event verifier-trace-fri-prefix ?Head s  $\leq C$ 
  by (rule wp-verifier-trace-fri-prefix-dependent-challenge-space-set-bound
    [OF future subset bound])
  show ?thesis
    unfolding verify-monad-trace-fri-decomposition
  proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
    show trace-fri-root-list-set-hit s B None  $\implies$  ?Head None
      unfolding trace-fri-root-list-set-hit-def
      accepted-fri-challenges-def by blast
  next
    fix header prefix-state out
    assume prefix:
      Some (header, prefix-state)  $\in$ 
      set-dist (execute verifier-trace-fri-prefix s)
    and suffix:
      out  $\in$ 
      set-dist (execute (verifier-after-trace-fri header) prefix-state)
    and hit: trace-fri-root-list-set-hit s B out
    obtain fr f-fl where header-eq: header = (fr, f-fl)
      by (cases header) auto
    from hit obtain trace-bs dg comp-bs fr' f-fri-roots' f-final' as'
      composition-fri-roots' final' rest' where
      challenges-hit: accepted-fri-challenges s out trace-bs dg comp-bs
    and header-hit:
      verifier-header-transcript s fr' f-fri-roots' f-final' as' dg
      composition-fri-roots' final' rest'

```

```

    and trace-bs-B: trace-bs ∈ B fr'
    unfolding trace-fri-root-list-set-hit-def by blast
  from challenges-hit obtain result final-state where out-eq:
    out = Some (result, final-state)
    unfolding accepted-fri-challenges-def by blast
  have prefix':
    Some ((fr, f-fl), prefix-state) ∈
      set-dist (execute verifier-trace-fri-prefix s)
    using prefix header-eq by simp
  have suffix':
    Some (result, final-state) ∈
      set-dist
        (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
    using suffix header-eq out-eq by simp
  have prefix-res:
    PTranscript s = [fr] @ map snd f-fl @ PTranscript prefix-state
    using verifier-trace-fri-prefix-outcome[OF prefix'] by simp
  have fr-eq: fr' = fr
    using prefix-res header-hit
    unfolding verifier-header-transcript-def verifier-header-messages-def
    by simp
  from verifier-after-trace-fri-accepted-fri-challenges[OF prefix' suffix']
  obtain dg' comp-bs' where
    challenges-prefix:
      accepted-fri-challenges s (Some (result, final-state))
      (map fst f-fl) dg' comp-bs'
    by blast
  have trace-bs-eq: trace-bs = map fst f-fl
    using accepted-fri-challenges-unique
      [OF challenges-prefix challenges-hit[unfolded out-eq]]
    by simp
  then show ?Head (Some (header, prefix-state))
    using trace-bs-B fr-eq header-eq by simp
qed
qed

```

lemma *wp-verify-monad-composition-fri-root-list-set-bound:*
fixes $C :: \text{prob}$
assumes *future: composition-fri-future-fresh s*
and *subset:*
 $\bigwedge fr \text{ f-fri-roots f-final as dg composition-fri-roots.}$
 $B \text{ fr f-fri-roots f-final as dg composition-fri-roots} \subseteq$
 $\text{fri-challenge-space (ceil-log (to-nat dg + 1))}$
and *bound:*
 $\bigwedge fr \text{ f-fri-roots f-final as dg composition-fri-roots.}$
 nnreal
 $(\text{card}$
 $(B \text{ fr f-fri-roots f-final as dg composition-fri-roots})) /$
 $\text{nnreal (CARD('f) } \wedge \text{ ceil-log (to-nat dg + 1))} \leq C$

```

shows
  wp-event verify-monad (composition-fri-root-list-set-hit s B) s ≤ C
proof –
  let ?Head =
    λout. case out of
      None ⇒ False
    | Some ((fr, f-fl, f-final, as, dg, fl), -) ⇒
      map fst fl ∈ B fr (map snd f-fl) f-final as dg (map snd fl)
  have prefix-bound:
    wp-event verifier-composition-fri-prefix ?Head s ≤ C
  by (rule wp-verifier-composition-fri-prefix-root-dependent-challenge-space-set-bound
    [OF future subset bound])
  show ?thesis
    unfolding verify-monad-composition-fri-decomposition
  proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
    show composition-fri-root-list-set-hit s B None ⇒ ?Head None
      unfolding composition-fri-root-list-set-hit-def
        accepted-fri-challenges-def by blast
  next
    fix header prefix-state out
    assume prefix:
      Some (header, prefix-state) ∈
        set-dist (execute verifier-composition-fri-prefix s)
    and suffix:
      out ∈
        set-dist (execute
          (verifier-after-composition-fri header) prefix-state)
    and hit: composition-fri-root-list-set-hit s B out
  obtain fr f-fl f-final as dg fl where header-eq:
    header = (fr, f-fl, f-final, as, dg, fl)
  by (cases header) auto
  from hit obtain trace-bs dg' comp-bs fr' f-fri-roots' f-final' as'
    composition-fri-roots' final' rest' where
    challenges-hit:
      accepted-fri-challenges s out trace-bs dg' comp-bs
  and header-hit:
    verifier-header-transcript s fr' f-fri-roots' f-final' as' dg'
      composition-fri-roots' final' rest'
  and comp-bs-B:
    comp-bs ∈
      B fr' f-fri-roots' f-final' as' dg' composition-fri-roots'
  unfolding composition-fri-root-list-set-hit-def by blast
  from challenges-hit obtain result final-state where out-eq:
    out = Some (result, final-state)
  unfolding accepted-fri-challenges-def by blast
  have prefix':
    Some ((fr, f-fl, f-final, as, dg, fl), prefix-state) ∈
      set-dist (execute verifier-composition-fri-prefix s)
  using prefix header-eq by simp

```

```

have suffix':
  Some (result, final-state) ∈
    set-dist
      (execute
        (verifier-after-composition-fri
          (fr, f-fl, f-final, as, dg, fl))
        prefix-state)
  using suffix header-eq out-eq by simp
from suffix'[unfolded verifier-after-composition-fri-def]
obtain final query-state where
  read-final:
    Some (final, query-state) ∈ set-dist (execute read prefix-state)
  and query-out:
    Some (result, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          query-state)
    by (auto elim! set-dist-bindE)
have prefix-res:
  length f-fl = ceil-log clength ∧
  length as = length spec ∧
  length fl = ceil-log (to-nat dg + 1) ∧
  PTranscript s =
    [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
    map snd fl @ PTranscript prefix-state
  using verifier-composition-fri-prefix-outcome[OF prefix'] by simp
from read-outcome[OF read-final] obtain rest-query where
  tr-prefix: PTranscript prefix-state = final # rest-query
  and tr-query: PTranscript query-state = rest-query
  by blast
have header-prefix:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
  using prefix-res tr-prefix tr-query
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
have header-egs:
  fr' = fr ∧
  f-fri-roots' = map snd f-fl ∧
  f-final' = f-final ∧
  as' = as ∧
  dg' = dg ∧
  composition-fri-roots' = map snd fl
  using verifier-header-transcript-unique[OF header-prefix header-hit]
  by simp
have challenges-prefix:

```



```

    accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg (map fst fl)
  by (rule verifier-after-composition-fri-accepted-fri-challenges
    [OF prefix' suffix'])
  have comp-bs-eq: comp-bs = map fst fl
    using accepted-fri-challenges-unique
    [OF challenges-prefix challenges-hit[unfolded out-eq]]
    by simp
  show ?Head (Some (header, prefix-state))
    using comp-bs-B comp-bs-eq header-eqs header-eq by simp
qed
qed

lemma wp-verify-monad-composition-fri-challenge-list-set-bound:
  fixes C :: prob
  assumes future: composition-fri-future-fresh s
  and subset:
     $\bigwedge dg. B dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$ 
  and bound:
     $\bigwedge dg. \text{nnreal } (\text{card } (B dg)) / \text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq C$ 
  shows
    wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s  $\leq C$ 
proof -
  let ?Head =
     $\lambda out. \text{case out of}$ 
     $\text{None} \Rightarrow \text{False}$ 
     $| \text{Some } ((-, -, -, dg, fl), -) \Rightarrow \text{map fst fl} \in B dg$ 
  have prefix-bound:
    wp-event verifier-composition-fri-prefix ?Head s  $\leq C$ 
  by (rule wp-verifier-composition-fri-prefix-challenge-space-set-bound
    [OF future subset bound])
  show ?thesis
    unfolding verify-monad-composition-fri-decomposition
  proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
    show composition-fri-challenge-list-set-hit s B None  $\implies$  ?Head None
      unfolding composition-fri-challenge-list-set-hit-def
        accepted-fri-challenges-def by blast
  next
    fix header prefix-state out
    assume prefix:
      Some (header, prefix-state)  $\in$ 
      set-dist (execute verifier-composition-fri-prefix s)
    and suffix:
      out  $\in$ 
      set-dist (execute
        (verifier-after-composition-fri header) prefix-state)
    and hit: composition-fri-challenge-list-set-hit s B out
    obtain fr f-fl f-final as dg fl where header-eq:

```

```

    header = (fr, f-fl, f-final, as, dg, fl)
    by (cases header) auto
from hit obtain trace-bs dg' comp-bs where
    challenges-hit: accepted-fri-challenges s out trace-bs dg' comp-bs
    and comp-bs-B: comp-bs ∈ B dg'
    unfolding composition-fri-challenge-list-set-hit-def by blast
from challenges-hit obtain result final-state where out-eq:
    out = Some (result, final-state)
    unfolding accepted-fri-challenges-def by blast
have prefix':
    Some ((fr, f-fl, f-final, as, dg, fl), prefix-state) ∈
      set-dist (execute verifier-composition-fri-prefix s)
    using prefix header-eq by simp
have suffix':
    Some (result, final-state) ∈
      set-dist
        (execute
          (verifier-after-composition-fri
            (fr, f-fl, f-final, as, dg, fl))
          prefix-state)
    using suffix header-eq out-eq by simp
have challenges-prefix:
    accepted-fri-challenges s (Some (result, final-state))
      (map fst f-fl) dg (map fst fl)
    by (rule verifier-after-composition-fri-accepted-fri-challenges
      [OF prefix' suffix'])
have eqs:
    trace-bs = map fst f-fl ∧ dg' = dg ∧ comp-bs = map fst fl
    using accepted-fri-challenges-unique
      [OF challenges-prefix challenges-hit[unfolded out-eq]]
    by simp
then show ?Head (Some (header, prefix-state))
    using comp-bs-B header-eq by simp
qed
qed

lemma verifier-initial-alpha-list-set-error-bound:
  assumes subset: B ⊆ alpha-space
  and bounded:
    nnreal (card B) / nnreal (card alpha-space) ≤
      composition-error-bound
  shows
    wp-event verify-monad
      (alpha-list-set-hit (verifier-initial-state tr) B)
      (verifier-initial-state tr) ≤ composition-error-bound
proof –
  have wp-event verify-monad
    (alpha-list-set-hit (verifier-initial-state tr) B)
    (verifier-initial-state tr) ≤

```

$\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \alpha\text{-space})$
by (rule *wp-verify-monad-alpha-list-set-bound*) (use *subset in simp-all*)
also have $\dots \leq \text{composition-error-bound}$
by (rule *bounded*)
finally show *?thesis* .
qed

lemma *verifier-initial-trace-fri-challenge-list-set-error-bound*:
assumes *subset*: $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$
and *bounded*:
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } \text{clength}) \leq$
 trace-fri-error
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-challenge-list-set-hit } (\text{verifier-initial-state } tr) B)$
 $(\text{verifier-initial-state } tr) \leq \text{trace-fri-error}$

proof –
have $\text{wp-event verify-monad}$
 $(\text{trace-fri-challenge-list-set-hit } (\text{verifier-initial-state } tr) B)$
 $(\text{verifier-initial-state } tr) \leq$
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } \text{clength})$
by (rule *wp-verify-monad-trace-fri-challenge-list-set-bound*)
 (use *subset in simp-all*)
also have $\dots \leq \text{trace-fri-error}$
by (rule *bounded*)
finally show *?thesis* .
qed

lemma *verifier-initial-composition-fri-challenge-list-set-error-bound*:
assumes *subset*:
 $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$
and *bounded*:
 $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq$
 $\text{composition-fri-error}$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-challenge-list-set-hit } (\text{verifier-initial-state } tr) B)$
 $(\text{verifier-initial-state } tr) \leq \text{composition-fri-error}$
by (rule *wp-verify-monad-composition-fri-challenge-list-set-bound*)
 (use *subset bounded in simp-all*)

definition *accepted-fri-opening-transcript*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $\text{nat list} \Rightarrow 'f \text{ list list list} \Rightarrow 'f \text{ list list list} \Rightarrow \text{bool}$
where

$\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs trace-final}$
 $\text{dg composition-roots composition-bs composition-final query-idxs}$
 $\text{trace-round-layers composition-round-layers} \longleftrightarrow$
 $(\exists \text{result final-state fr f-fl as fl query-state raw-idxs query-chunks.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{trace-roots} = \text{map snd f-fl} \wedge$
 $\text{trace-bs} = \text{map fst f-fl} \wedge$
 $\text{composition-roots} = \text{map snd fl} \wedge$
 $\text{composition-bs} = \text{map fst fl} \wedge$
 $\text{accepted-fri-challenges } s \text{ out trace-bs dg composition-bs} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final (PTranscript query-state)} \wedge$
 $\text{PState query-state} =$
 $\text{verifier-header-state } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final} \wedge$
 $\text{PQueryCounter query-state} = \text{PQueryCounter } s \wedge$
 $\text{Some (result, final-state)} \in$
 set-dist (execute
 (ntimes
 $\text{(verifier-query-round-program fr f-fl trace-final as fl}$
 $\text{composition-final)}$
 rounds)
 $\text{query-state)} \wedge$
 $\text{verifier-query-indices-derived } s \text{ out}$
 $\text{(verifier-header-state } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final)}$
 $\text{(PTranscript query-state) trace-roots composition-roots query-idxs} \wedge$
 $\text{length trace-round-layers} = \text{rounds} \wedge$
 $\text{length composition-round-layers} = \text{rounds} \wedge$
 $\text{length raw-idxs} = \text{rounds} \wedge$
 $\text{query-idxs} = \text{map } (\lambda \text{raw. index (to-nat raw)}) \text{ raw-idxs} \wedge$
 $\text{length query-chunks} = \text{rounds} \wedge$
 $\text{PTranscript query-state} =$
 $\text{List.concat query-chunks @ PTranscript final-state} \wedge$
 $(\forall i < \text{rounds.}$
 $\text{verifier-query-round-chunk (query-idxs ! } i)$
 $\text{trace-roots composition-roots (query-chunks ! } i)) \wedge$
 $(\forall i < \text{rounds.}$
 $\text{query-round-fri-layer-transcripts (query-idxs ! } i)$
 $\text{trace-roots composition-roots (query-chunks ! } i)$
 $\text{(trace-round-layers ! } i) \text{ (composition-round-layers ! } i)) \wedge$
 $(\forall i < \text{rounds.}$
 $\text{fmlookup (HashMap final-state)}$
 $\text{(QueryIndexChallenge (PQueryCounter query-state + } i) \text{ (state-after-query-chunks}$
 $\text{(PState query-state) query-chunks } i)) =$
 $\text{Some (raw-idxs ! } i)) \wedge$
 $(\forall \text{idx} \in \text{set query-idxs. idx} < \text{clength * scale}))$

lemma *verify-monad-accepted-fri-opening-transcript:*

```

fixes  $s :: ('f, 'a) \text{protocol-channel-scheme}$ 
assumes  $\text{outcome}:$ 
   $\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute } \text{verify-monad } s)$ 
obtains  $\text{trace-roots trace-bs trace-final dg composition-roots composition-bs}$ 
   $\text{composition-final query-idxs trace-round-layers composition-round-layers}$ 
where  $\text{accepted-fri-opening-transcript } s (\text{Some } (\text{result}, \text{final-state}))$ 
   $\text{trace-roots trace-bs trace-final dg composition-roots composition-bs}$ 
   $\text{composition-final query-idxs trace-round-layers composition-round-layers}$ 
proof –
from  $\text{verify-monad-accepted-fri-challenges}[OF \text{ outcome}]$ 
obtain  $\text{trace-bs dg composition-bs}$  where  $\text{challenges}:$ 
   $\text{accepted-fri-challenges } s (\text{Some } (\text{result}, \text{final-state}))$ 
   $\text{trace-bs dg composition-bs}$ 
by  $\text{blast}$ 
then obtain  $\text{result}' \text{ final-state}'$   $\text{fr } f\text{-fl } \text{trace-final}$   $\text{as } fl \text{ composition-final}$ 
   $\text{query-state}$  where
   $\text{out-eq}: \text{Some } (\text{result}, \text{final-state}) = \text{Some } (\text{result}', \text{final-state}')$ 
and  $\text{trace-bs-eq}: \text{trace-bs} = \text{map fst } f\text{-fl}$ 
and  $\text{composition-bs-eq}: \text{composition-bs} = \text{map fst } fl$ 
and  $\text{header}:$ 
   $\text{verifier-header-transcript } s \text{ fr } (\text{map snd } f\text{-fl}) \text{ trace-final as dg}$ 
   $(\text{map snd } fl) \text{ composition-final } (P\text{Transcript } \text{query-state})$ 
and  $\text{query-state-eq}:$ 
   $P\text{State } \text{query-state} =$ 
   $\text{verifier-header-state } s \text{ fr } (\text{map snd } f\text{-fl}) \text{ trace-final as dg}$ 
   $(\text{map snd } fl) \text{ composition-final}$ 
and  $\text{query-out}:$ 
   $\text{Some } (\text{result}', \text{final-state}') \in$ 
   $\text{set-dist } (\text{execute}$ 
     $(\text{ntimes}$ 
       $(\text{verifier-query-round-program } \text{fr } f\text{-fl } \text{trace-final as } fl$ 
         $\text{composition-final})$ 
       $\text{rounds})$ 
       $\text{query-state})$ 
  and  $\text{query-count-header}:$ 
   $P\text{QueryCounter } \text{query-state} = P\text{QueryCounter } s$ 
unfolding  $\text{accepted-fri-challenges-def}$  by  $\text{blast}$ 
have  $\text{result}'\text{-eq}: \text{result}' = \text{result}$ 
using  $\text{out-eq}$  by  $\text{simp}$ 
have  $\text{final-state}'\text{-eq}: \text{final-state}' = \text{final-state}$ 
using  $\text{out-eq}$  by  $\text{simp}$ 
have  $\text{query-out}':$ 
   $\text{Some } (\text{result}, \text{final-state}) \in$ 
   $\text{set-dist } (\text{execute}$ 
     $(\text{ntimes}$ 
       $(\text{verifier-query-round-program } \text{fr } f\text{-fl } \text{trace-final as } fl$ 
         $\text{composition-final})$ 
       $\text{rounds})$ 
       $\text{query-state})$ 

```

```

using query-out result'-eq final-state'-eq by simp
from ntimes-verifier-query-rounds-outcome[OF query-out']
obtain raw-idxs query-idxs query-chunks where
  len-raw: length raw-idxs = rounds
  and query-idxs-def: query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
  and len-chunks: length query-chunks = rounds
  and tr-query:
    PTranscript query-state = List.concat query-chunks @ PTranscript final-state
  and round-chunks:
     $\bigwedge i. i < \text{rounds} \implies$ 
      verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
  and lookups:
     $\bigwedge i. i < \text{rounds} \implies$ 
      fmllookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i) (state-after-query-chunks (PState
query-state) query-chunks i)) =
      Some (raw-idxs ! i)
  and idx-bounds:
     $\forall \text{idx} \in \text{set query-idxs}. \text{idx} < \text{clength} * \text{scale}$ 
by auto
have layer-exists:
   $\forall i < \text{rounds}.$ 
     $\exists \text{trace-layers composition-layers}.$ 
      query-round-fri-layer-transcripts (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
      trace-layers composition-layers
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  from verifier-query-round-chunk-fri-layer-transcriptsE
    [OF round-chunks[OF i-bound]]
  show  $\exists \text{trace-layers composition-layers}.$ 
    query-round-fri-layer-transcripts (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
    trace-layers composition-layers
  by blast
qed
then obtain trace-layers-of composition-layers-of where layer-of:
   $\bigwedge i. i < \text{rounds} \implies$ 
    query-round-fri-layer-transcripts (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
    (trace-layers-of i) (composition-layers-of i)
  by metis
let ?trace-round-layers = map trace-layers-of [0.. $\text{rounds}$ ]
let ?composition-round-layers = map composition-layers-of [0.. $\text{rounds}$ ]
have layers:
   $\forall i < \text{rounds}.$ 
    query-round-fri-layer-transcripts (query-idxs ! i)

```

```

    (map snd f-fl) (map snd fl) (query-chunks ! i)
    (?trace-round-layers ! i) (?composition-round-layers ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < rounds
  then show query-round-fri-layer-transcripts (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
    (?trace-round-layers ! i) (?composition-round-layers ! i)
    using layer-of[OF i-bound] by simp
qed
have derived:
  verifier-query-indices-derived s (Some (result, final-state))
    (verifier-header-state s fr (map snd f-fl) trace-final as dg
      (map snd fl) composition-final)
    (PTranscript query-state) (map snd f-fl) (map snd fl) query-idxs
proof -
  have tr-query':
    List.concat query-chunks @ PTranscript final-state = PTranscript query-state
    using tr-query by simp
  have lookups':
    ∀ i < rounds.
      fmlookup (HashMap final-state)
        (QueryIndexChallenge (PQueryCounter s + i) (state-after-query-chunks
          (verifier-header-state s fr (map snd f-fl) trace-final as dg
            (map snd fl) composition-final)
            query-chunks i)) =
        Some (raw-idxs ! i)
    using lookups query-state-eq query-count-header by simp
  show ?thesis
    unfolding verifier-query-indices-derived-def
    apply (rule exI[where x=result])
    apply (rule exI[where x=final-state])
    apply (rule exI[where x=raw-idxs])
    apply (rule exI[where x=query-chunks])
    apply (rule exI[where x=PTranscript final-state])
    apply (intro conjI)
    apply simp
    apply (rule len-raw)
    apply (rule query-idxs-def)
    apply (rule len-chunks)
    apply (rule tr-query')
    apply (intro allI impI, rule round-chunks, assumption)
    apply (rule lookups')
    using idx-bounds by simp
qed
have tr-query-tail:
  List.concat query-chunks @ PTranscript final-state = PTranscript query-state
  using tr-query by simp
have lookups-all:

```

```

   $\forall i < \text{rounds}.$ 
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i) (state-after-query-chunks (PState
query-state) query-chunks i)) =
    Some (raw-idxs ! i)
  using lookups by blast
  have lookups-all-header:
   $\forall i < \text{rounds}.$ 
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks
        (verifier-header-state s fr (map snd f-fl) trace-final as dg
          (map snd fl) composition-final)
        query-chunks i)) =
    Some (raw-idxs ! i)
  using lookups-all query-state-eq by simp
  have challenges-maps:
  accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg (map fst fl)
  using challenges trace-bs-eq composition-bs-eq by simp
  have derived-maps:
  verifier-query-indices-derived s (Some (result, final-state))
    (verifier-header-state s fr (map snd f-fl) trace-final as dg
      (map snd fl) composition-final)
    (PTranscript query-state) (map snd f-fl) (map snd fl)
    (map ( $\lambda \text{raw}.$  index (to-nat raw)) raw-idxs)
  using derived query-idxs-def by simp
  have layers-maps:
   $\forall i < \text{rounds}.$ 
  query-round-fri-layer-transcripts (index (to-nat (raw-idxs ! i)))
    (map snd f-fl) (map snd fl) (query-chunks ! i)
    (trace-layers-of i) (composition-layers-of i)
  using layers query-idxs-def len-raw by auto
  have round-chunks-raw:
   $\forall i < \text{rounds}.$ 
  verifier-query-round-chunk (index (to-nat (raw-idxs ! i)))
    (map snd f-fl) (map snd fl) (query-chunks ! i)
  using round-chunks query-idxs-def len-raw by auto
  have lookups-header:
   $\forall i < \text{rounds}.$ 
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter s + i) (state-after-query-chunks
      (verifier-header-state s fr (map snd f-fl) trace-final as dg
        (map snd fl) composition-final)
      query-chunks i)) =
    Some (raw-idxs ! i)
  using lookups-all query-state-eq query-count-header by simp
  have accepted:
  accepted-fri-opening-transcript s (Some (result, final-state))

```



```

    (map snd f-fl) trace-bs trace-final dg (map snd fl) composition-bs
    composition-final query-idxs ?trace-round-layers ?composition-round-layers
unfolding accepted-fri-opening-transcript-def
apply (rule exI[where x=result])
apply (rule exI[where x=final-state])
apply (rule exI[where x=fr])
apply (rule exI[where x=f-fl])
apply (rule exI[where x=as])
apply (rule exI[where x=fl])
apply (rule exI[where x=query-state])
apply (rule exI[where x=raw-idxs])
apply (rule exI[where x=query-chunks])
apply (simp add: challenges header query-state-eq query-out' derived len-raw
    query-count-header query-idxs-def len-chunks tr-query-tail layers
    lookups-all idx-bounds trace-bs-eq composition-bs-eq challenges-maps
    derived-maps layers-maps lookups-header lookups-all-header round-chunks)
apply (rule round-chunks-raw)
done
then show ?thesis
  by (rule that)
qed

```

```

lemma accepted-fri-opening-transcript-challenges:
assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers
shows accepted-fri-challenges s out trace-bs dg composition-bs
using fri-openings
unfolding accepted-fri-opening-transcript-def
apply (elim exE conjE)
apply assumption
done

```

```

lemma accepted-fri-opening-transcript-trace-challenge-list-set-hit:
assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers
and member: trace-bs ∈ B
shows trace-fri-challenge-list-set-hit s B out
using fri-openings
unfolding accepted-fri-opening-transcript-def trace-fri-challenge-list-set-hit-def
apply (elim exE conjE)
apply (intro exI conjI)
apply assumption
apply (rule member)
done

```

lemma *accepted-fri-opening-transcript-composition-challenge-list-set-hit:*

assumes *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers

and *member: composition-bs ∈ B dg*

shows *composition-fri-challenge-list-set-hit s B out*

using *fri-openings*

unfolding *accepted-fri-opening-transcript-def*
composition-fri-challenge-list-set-hit-def

apply (*elim exE conjE*)

apply (*intro exI conjI*)

apply *assumption*

apply (*rule member*)

done

lemma *accepted-fri-opening-transcript-shapes:*

assumes *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers

shows *length trace-roots = ceil-log clength*

and *length trace-bs = length trace-roots*

and *length composition-roots = ceil-log (to-nat dg + 1)*

and *length composition-bs = length composition-roots*

and *length trace-round-layers = rounds*

and *length composition-round-layers = rounds*

and *length query-idxs = rounds*

using *fri-openings*

unfolding *accepted-fri-opening-transcript-def verifier-header-transcript-def*
by *auto*

lemma *accepted-fri-opening-transcript-algebraic-round-counts:*

assumes *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers

shows *length trace-roots = trace-fri-algebraic-round-count*

and *length trace-bs = trace-fri-algebraic-round-count*

and *length composition-roots = composition-fri-algebraic-round-count dg*

and *length composition-bs = composition-fri-algebraic-round-count dg*

and *length trace-round-layers = rounds*

and *length composition-round-layers = rounds*

and *length query-idxs = rounds*

proof –

have *shapes:*

length trace-roots = ceil-log clength

length trace-bs = length trace-roots

length composition-roots = ceil-log (to-nat dg + 1)

```

length composition-bs = length composition-roots
length trace-round-layers = rounds
length composition-round-layers = rounds
length query-idxs = rounds
using accepted-fri-opening-transcript-shapes[OF fri-openings] by blast+
show length trace-roots = trace-fri-algebraic-round-count
  using shapes(1) by (simp add: trace-fri-algebraic-round-count-eq)
show length trace-bs = trace-fri-algebraic-round-count
  using shapes(1,2) by (simp add: trace-fri-algebraic-round-count-eq)
show length composition-roots = composition-fri-algebraic-round-count dg
  using shapes(3) by (simp add: composition-fri-algebraic-round-count-eq)
show length composition-bs = composition-fri-algebraic-round-count dg
  using shapes(3,4) by (simp add: composition-fri-algebraic-round-count-eq)
show length trace-round-layers = rounds
  using shapes(5) .
show length composition-round-layers = rounds
  using shapes(6) .
show length query-idxs = rounds
  using shapes(7) .
qed

```

lemma *accepted-fri-opening-transcript-trace-round-layer-length:*
assumes *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers
and *round-bound: round-idx < rounds*
shows *length (trace-round-layers ! round-idx) = length trace-roots*
proof –
obtain *result final-state fr f-fl as fl query-state raw-idxs query-chunks*
where *layers:*
 $\bigwedge i. i < rounds \implies$
query-round-fri-layer-transcripts (query-idxs ! i)
trace-roots composition-roots (query-chunks ! i)
(trace-round-layers ! i) (composition-round-layers ! i)
using *fri-openings* **unfolding** *accepted-fri-opening-transcript-def* **by** *blast*
from *layers[OF round-bound]*
obtain *query-chunk trace-fri-chunk composition-fri-chunk leaves paths* **where**
trace-layers:
*fri-layers-transcript (length trace-roots) (clength * scale)*
(trace-round-layers ! round-idx) trace-fri-chunk
unfolding *query-round-fri-layer-transcripts-def* **by** *blast*
then show *?thesis*
unfolding *fri-layers-transcript-def* **by** *simp*
qed

lemma *accepted-fri-opening-transcript-composition-round-layer-length:*
assumes *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final

dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers
and *round-bound: round-idx < rounds*
shows *length (composition-round-layers ! round-idx) = length composition-roots*
proof –
obtain *result final-state fr f-fl as fl query-state raw-idxs query-chunks*
where *layers:*
 $\bigwedge i. i < rounds \implies$
query-round-fri-layer-transcripts (query-idxs ! i)
trace-roots composition-roots (query-chunks ! i)
(trace-round-layers ! i) (composition-round-layers ! i)
using *fri-openings* **unfolding** *accepted-fri-opening-transcript-def* **by** *blast*
from *layers[OF round-bound]*
obtain *query-chunk trace-fri-chunk composition-fri-chunk leaves paths* **where**
composition-layers:
*fri-layers-transcript (length composition-roots) (clength * scale)*
(composition-round-layers ! round-idx) composition-fri-chunk
unfolding *query-round-fri-layer-transcripts-def* **by** *blast*
then show *?thesis*
unfolding *fri-layers-transcript-def* **by** *simp*
qed

lemma *accepted-fri-opening-transcript-trace-layer-opening:*
assumes *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers
and *round-bound: round-idx < rounds*
and *layer-bound: layer-idx < length trace-roots*
obtains *root challenge len idx sibling-idx xp xp-path xn xn-path*
where *root = trace-roots ! layer-idx*
and *challenge = trace-bs ! layer-idx*
and *len = fri-layer-lengths (length trace-roots) (clength * scale) ! layer-idx*
and *idx =*
fri-layer-indices (length trace-roots) (query-idxs ! round-idx)
*(clength * scale) ! layer-idx*
and *sibling-idx = fri-sibling-index len idx*
and *fri-layer-opening-chunk len xp xp-path xn xn-path*
(trace-round-layers ! round-idx ! layer-idx)
proof –
obtain *result final-state fr f-fl as fl query-state raw-idxs query-chunks*
where *layers:*
 $\bigwedge i. i < rounds \implies$
query-round-fri-layer-transcripts (query-idxs ! i)
trace-roots composition-roots (query-chunks ! i)
(trace-round-layers ! i) (composition-round-layers ! i)
using *fri-openings* **unfolding** *accepted-fri-opening-transcript-def* **by** *blast*
from *layers[OF round-bound]*
obtain *query-chunk trace-fri-chunk composition-fri-chunk leaves paths* **where**

```

trace-layers:
  fri-layers-transcript (length trace-roots) (clength * scale)
    (trace-round-layers ! round-idx) trace-fri-chunk
  unfolding query-round-fri-layer-transcripts-def by blast
from fri-layers-transcript-nth-opening[OF trace-layers layer-bound]
obtain xp xp-path xn xn-path where layer-opening:
  fri-layer-opening-chunk
    (fri-layer-lengths (length trace-roots) (clength * scale) ! layer-idx)
    xp xp-path xn xn-path (trace-round-layers ! round-idx ! layer-idx)
  by blast
let ?len = fri-layer-lengths (length trace-roots) (clength * scale) ! layer-idx
let ?idx =
  fri-layer-indices (length trace-roots) (query-idxs ! round-idx)
    (clength * scale) ! layer-idx
let ?sidx = fri-sibling-index ?len ?idx
show ?thesis
  by (rule that[of trace-roots ! layer-idx trace-bs ! layer-idx
    ?len ?idx ?sidx xp xp-path xn xn-path])
    (use layer-opening in simp-all)
qed

lemma accepted-fri-opening-transcript-composition-layer-opening:
assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers
and round-bound: round-idx < rounds
and layer-bound: layer-idx < length composition-roots
obtains root challenge len idx sibling-idx xp xp-path xn xn-path
where root = composition-roots ! layer-idx
and challenge = composition-bs ! layer-idx
and len =
  fri-layer-lengths (length composition-roots) (clength * scale) ! layer-idx
and idx =
  fri-layer-indices (length composition-roots) (query-idxs ! round-idx)
    (clength * scale) ! layer-idx
and sibling-idx = fri-sibling-index len idx
and fri-layer-opening-chunk len xp xp-path xn xn-path
  (composition-round-layers ! round-idx ! layer-idx)
proof –
obtain result final-state fr f-fl as fl query-state raw-idxs query-chunks
where layers:
   $\bigwedge i. i < rounds \implies$ 
    query-round-fri-layer-transcripts (query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
    (trace-round-layers ! i) (composition-round-layers ! i)
  using fri-openings unfolding accepted-fri-opening-transcript-def by blast
from layers[OF round-bound]
obtain query-chunk trace-fri-chunk composition-fri-chunk leaves paths where

```

```

composition-layers:
  fri-layers-transcript (length composition-roots) (clength * scale)
    (composition-round-layers ! round-idx) composition-fri-chunk
  unfolding query-round-fri-layer-transcripts-def by blast
from fri-layers-transcript-nth-opening[OF composition-layers layer-bound]
obtain xp xp-path xn xn-path where layer-opening:
  fri-layer-opening-chunk
    (fri-layer-lengths (length composition-roots) (clength * scale) ! layer-idx)
    xp xp-path xn xn-path
    (composition-round-layers ! round-idx ! layer-idx)
  by blast
let ?len =
  fri-layer-lengths (length composition-roots) (clength * scale) ! layer-idx
let ?idx =
  fri-layer-indices (length composition-roots) (query-idxs ! round-idx)
    (clength * scale) ! layer-idx
let ?sidx = fri-sibling-index ?len ?idx
show ?thesis
  by (rule that[of composition-roots ! layer-idx composition-bs ! layer-idx
    ?len ?idx ?sidx xp xp-path xn xn-path])
    (use layer-opening in simp-all)
qed

```

4.3 Global FRI table bindings

The verifier sees authenticated local openings. The following predicates describe the stronger deterministic object needed by the algebraic FRI argument: complete layer tables bound to the Merkle roots, with all local openings matching the corresponding table entries.

```

definition fri-layer-tables-shape
  :: nat  $\Rightarrow$  'f list list  $\Rightarrow$  bool
  where
    fri-layer-tables-shape n tables  $\longleftrightarrow$ 
      length tables = n  $\wedge$ 
      ( $\forall j < n.$ 
        length (tables ! j) =
          fri-layer-lengths n (clength * scale) ! j)

definition fri-roots-bind-tables
  :: 'f list  $\Rightarrow$  'f list list  $\Rightarrow$ 
    (f, 'a) protocol-channel-scheme  $\Rightarrow$  bool
  where
    fri-roots-bind-tables roots tables final-state  $\longleftrightarrow$ 
      length tables = length roots  $\wedge$ 
      ( $\forall j < \text{length roots}.$ 
        merkle-root-binds-table (roots ! j) (tables ! j) final-state)

```

```

definition fri-bound-layer-tables

```

$:: 'f \text{ list} \Rightarrow 'f \text{ list list} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{fri-bound-layer-tables roots tables final-state} \longleftrightarrow$
 $\text{fri-layer-tables-shape (length roots) tables} \wedge$
 $\text{fri-roots-bind-tables roots tables final-state}$

definition *fri-layer-opening-matches-bound-table*

$:: \text{nat} \Rightarrow \text{nat} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

$\text{fri-layer-opening-matches-bound-table query-idx layer-idx tables}$
 $\text{layer-chunks} \longleftrightarrow$
 $\text{layer-idx} < \text{length tables} \wedge$
 $\text{length layer-chunks} = \text{length tables} \wedge$
 $(\text{let len} =$
 $\quad \text{fri-layer-lengths (length tables) (clength * scale) ! layer-idx};$
 $\quad \text{idx} =$
 $\quad \text{fri-layer-indices (length tables) query-idx (clength * scale) !}$
 $\quad \text{layer-idx}$
 $\text{in } \exists \text{xp xp-path xn xn-path.}$
 $\text{fri-layer-opening-chunk len xp xp-path xn xn-path}$
 $(\text{layer-chunks ! layer-idx}) \wedge$
 $\text{fri-opening-matches-table len idx (tables ! layer-idx) xp xn})$

definition *fri-round-openings-match-tables*

$:: 'f \text{ list list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

$\text{fri-round-openings-match-tables tables query-idx layer-chunks} \longleftrightarrow$
 $\text{length layer-chunks} = \text{length tables} \wedge$
 $(\forall j < \text{length tables.}$
 $\quad \text{fri-layer-opening-matches-bound-table query-idx j tables}$
 $\quad \text{layer-chunks})$

definition *fri-all-round-openings-match-tables*

$:: 'f \text{ list list} \Rightarrow \text{nat list} \Rightarrow 'f \text{ list list list} \Rightarrow \text{bool}$

where

$\text{fri-all-round-openings-match-tables tables query-idxs round-layers} \longleftrightarrow$
 $\text{length round-layers} = \text{length query-idxs} \wedge$
 $(\forall i < \text{length query-idxs.}$
 $\quad \text{fri-round-openings-match-tables tables (query-idxs ! i)}$
 $\quad (\text{round-layers ! i}))$

definition *fri-final-table-matches*

$:: 'f \text{ list} \Rightarrow 'f \Rightarrow \text{bool}$

where

$\text{fri-final-table-matches final-table final} \longleftrightarrow$
 $\text{final-table} \neq [] \wedge$
 $\text{fri-final-constant-consistent final-table final}$

definition *trace-fri-tables-bound*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

trace-fri-tables-bound s out trace-roots trace-bs trace-final
query-idxs trace-round-layers trace-tables trace-final-table $\longleftrightarrow$
 $(\exists \text{result final-state dg composition-roots composition-bs}$
 $\text{composition-final composition-round-layers.}$
 $\text{out} = \text{Some}(\text{result}, \text{final-state}) \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{fri-bound-layer-tables trace-roots trace-tables final-state} \wedge$
 $\text{fri-all-round-openings-match-tables trace-tables query-idxs}$
 $\text{trace-round-layers} \wedge$
 $\text{fri-final-table-matches trace-final-table trace-final})$

definition *composition-fri-tables-bound*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

composition-fri-tables-bound s out dg composition-roots
composition-bs composition-final query-idxs composition-round-layers
composition-tables composition-final-table $\longleftrightarrow$
 $(\exists \text{result final-state trace-roots trace-bs trace-final}$
 $\text{trace-round-layers.}$
 $\text{out} = \text{Some}(\text{result}, \text{final-state}) \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{fri-bound-layer-tables composition-roots composition-tables}$
 $\text{final-state} \wedge$
 $\text{fri-all-round-openings-match-tables composition-tables query-idxs}$
 $\text{composition-round-layers} \wedge$
 $\text{fri-final-table-matches composition-final-table composition-final})$

definition *fri-initial-table-agrees*

$:: 'f \text{ list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

fri-initial-table-agrees initial-table tables $\longleftrightarrow$
 $\text{tables} = [] \vee \text{hd tables} = \text{initial-table}$

definition *fri-merkle-binding-good*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

fri-merkle-binding-good *s out* $\longleftrightarrow$
accepted out $\longrightarrow$
 $(\exists$ *trace-table composition-table as query-idxs*
trace-roots trace-bs trace-final dg composition-roots
composition-bs composition-final trace-round-layers
composition-round-layers trace-tables composition-tables
trace-final-table composition-final-table.
accepted-with-bound-tables s out trace-table composition-table as
query-idxs $\wedge$
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs
composition-final query-idxs trace-round-layers
composition-round-layers $\wedge$
trace-fri-tables-bound s out trace-roots trace-bs trace-final
query-idxs trace-round-layers trace-tables trace-final-table $\wedge$
composition-fri-tables-bound s out dg composition-roots
composition-bs composition-final query-idxs
composition-round-layers composition-tables
composition-final-table $\wedge$
fri-initial-table-agrees trace-table trace-tables $\wedge$
fri-initial-table-agrees composition-table composition-tables)

definition *fri-merkle-binding-bad*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

fri-merkle-binding-bad *s out* $\longleftrightarrow$
accepted out $\wedge \neg$ *fri-merkle-binding-good s out*

definition *fri-merkle-binding-bound*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

fri-merkle-binding-bound *s* $\longleftrightarrow$
 $\text{wp-event verify-monad (fri-merkle-binding-bad s) s} \leq$
 $\text{merkle-binding-error}$

end

end

theory *Soundness-Oracle-Target*

imports *Soundness-FRI*

begin

Random-oracle target-hit events and target-budget calculus.

context *soundness*

begin

definition *hash-map-output-collision-bound*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{hash-map-output-collision-bound } s \longleftrightarrow$
 $\text{wp-event verify-monad } (\text{hash-map-output-collision-bad } s) \ s \leq$
 $\text{merkle-binding-error}$

definition *hash-map-new-output-collision*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{hash-map-new-output-collision initial final} \longleftrightarrow$
 $\neg \text{hash-map-output-collision initial} \wedge$
 $\text{hash-map-output-collision final}$

definition *hash-new-collision-event*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('r \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{hash-new-collision-event } s \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (-, t) \Rightarrow \text{hash-map-new-output-collision } s \ t)$

definition *hash-map-new-output-hit*
 $:: 'f \text{ set} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{hash-map-new-output-hit } B \text{ initial final} \longleftrightarrow$
 $(\exists x \ y.$
 $\text{fmlookup } (\text{HashMap initial}) \ x = \text{None} \wedge$
 $\text{fmlookup } (\text{HashMap final}) \ x = \text{Some } y \wedge$
 $y \in B)$

definition *hash-new-output-hit-event*
 $:: 'f \text{ set} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('r \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{hash-new-output-hit-event } B \ s \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (-, t) \Rightarrow \text{hash-map-new-output-hit } B \ s \ t)$

lemma *hash-map-new-output-hit-subset:*
assumes *subset:* $A \subseteq B$
and hit: *hash-map-new-output-hit* $A \ s \ t$

```

shows hash-map-new-output-hit B s t
using subset hit unfolding hash-map-new-output-hit-def by blast

lemma protocol-merkle-created-tree-pullback-or-new-output-hit:
  assumes created: protocol-merkle.created-tree xs tree t
    and ext: s ≤ t
  shows
    protocol-merkle.created-tree xs tree s ∨
      hash-map-new-output-hit (set-tree tree) s t
  using created ext
proof (induction length xs arbitrary: xs tree s t rule: less-induct)
  case less
  show ?case
  proof (cases xs)
    case Nil
    then show ?thesis
      using less.prem1
      by (simp add: protocol-merkle.created-tree.simps)
  next
    case (Cons x rest)
    have xs-cons: xs = x # rest
      using Cons by simp
    show ?thesis
    proof (cases rest)
      case Nil
      then obtain h where
        tree-eq: tree = ⟨⟨, h, ⟨⟩⟩
        and lookup-t: fmlookup (HashMap t) x = Some h
        using less.prem1 Cons
        by (auto simp: protocol-merkle.created-tree.simps)
      show ?thesis
      proof (cases fmlookup (HashMap s) x)
        case None
        then have hash-map-new-output-hit (set-tree tree) s t
          unfolding hash-map-new-output-hit-def tree-eq
          using lookup-t by auto
        then show ?thesis by simp
      next
        case (Some h')
        have lookup-t': fmlookup (HashMap t) x = Some h'
          by (rule hash-extension-lookup[OF Some less.prem2])
        then have h'-eq: h' = h
          using lookup-t by simp
        have protocol-merkle.created-tree [x] tree s
          unfolding tree-eq
          using Some h'-eq
          by (simp add: protocol-merkle.created-tree.simps)
        then show ?thesis
          using Cons Nil by simp
      end
    end
  end
end

```

```

qed
next
case (Cons y ys)
let ?xs = x # y # ys
let ?i = length ?xs div 2
have xs-eq: xs = ?xs
  using xs-cons Cons by simp
from less.premis(1) obtain l h r where
  tree-eq: tree = ⟨l, h, r⟩
and left-t:
  protocol-merkle.created-tree (take ?i ?xs) l t
and right-t:
  protocol-merkle.created-tree (drop ?i ?xs) r t
and lookup-t:
  fmlookup (HashMap t) (MerkleNode (value l) (value r)) =
    Some h
unfolding xs-eq
by (auto simp: protocol-merkle.created-tree.simps Let-def)
have i-pos: 0 < ?i
  by simp
have i-lt: ?i < length ?xs
  by simp
have len-left: length (take ?i ?xs) < length xs
  using i-lt unfolding xs-eq by simp
have len-right: length (drop ?i ?xs) < length xs
  using i-pos unfolding xs-eq by simp
have left-split:
  protocol-merkle.created-tree (take ?i ?xs) l s ∨
    hash-map-new-output-hit (set-tree l) s t
  by (rule less.hyps[OF len-left])
  (use left-t less.premis(2) in simp-all)
from left-split show ?thesis
proof
  assume left-s:
    protocol-merkle.created-tree (take ?i ?xs) l s
  have right-split:
    protocol-merkle.created-tree (drop ?i ?xs) r s ∨
      hash-map-new-output-hit (set-tree r) s t
    by (rule less.hyps[OF len-right])
    (use right-t less.premis(2) in simp-all)
  from right-split show ?thesis
proof
  assume right-s:
    protocol-merkle.created-tree (drop ?i ?xs) r s
  show ?thesis
proof (cases
  fmlookup (HashMap s)
    (MerkleNode (value l) (value r)))
  case None

```

```

    then have hash-map-new-output-hit (set-tree tree) s t
      unfolding hash-map-new-output-hit-def tree-eq
      using lookup-t by auto
    then show ?thesis by simp
  next
    case (Some h')
    have lookup-t':
      fmllookup (HashMap t) (MerkleNode (value l) (value r)) =
        Some h'
      by (rule hash-extension-lookup[OF Some less.premis(2)])
    then have h'-eq: h' = h
      using lookup-t by simp
    have protocol-merkle.created-tree ?xs tree s
      unfolding tree-eq
      using left-s right-s Some h'-eq
      by (auto simp add: protocol-merkle.created-tree.simps Let-def)
    then have protocol-merkle.created-tree xs tree s
      unfolding xs-eq .
    then show ?thesis by blast
  qed
next
  assume right-hit: hash-map-new-output-hit (set-tree r) s t
  have hash-map-new-output-hit (set-tree tree) s t
    by (rule hash-map-new-output-hit-subset[OF - right-hit])
    (auto simp add: tree-eq)
  then show ?thesis by simp
qed
next
  assume left-hit: hash-map-new-output-hit (set-tree l) s t
  have hash-map-new-output-hit (set-tree tree) s t
    by (rule hash-map-new-output-hit-subset[OF - left-hit])
    (auto simp add: tree-eq)
  then show ?thesis by simp
qed
qed
qed
qed

```

lemma *protocol-created-tree-pullback-or-new-output-hit:*
assumes *created: protocol-created-tree xs tree t*
and *ext: s ≤ t*
shows
protocol-created-tree xs tree s ∨
hash-map-new-output-hit (set-tree tree) s t
using *protocol-merkle-created-tree-pullback-or-new-output-hit*
[OF created[unfolded protocol-created-tree-def] ext]
unfolding *protocol-created-tree-def* .

lemma *protocol-created-tree-nonempty-root-lookup:*

```

assumes created: protocol-created-tree xs tree s
and nonempty:  $xs \neq []$ 
obtains input where
  fmlookup (HashMap s) input = Some (value tree)
proof (cases length xs = 1)
case True
then obtain x where xs-eq:  $xs = [x]$ 
using nonempty by (cases xs) auto
from protocol-created-tree-singletonD[OF created[unfolded xs-eq]]
obtain h where
  tree-eq:  $tree = \langle \rangle, h, \langle \rangle$ 
and lookup: fmlookup (HashMap s) (MerkleLeaf x) = Some h
by blast
show ?thesis
by (rule that[of MerkleLeaf x]) (use lookup in  $\langle \text{simp add: tree-eq} \rangle$ )
next
case False
have len:  $2 \leq \text{length } xs$ 
using nonempty False by (cases xs; cases tl xs) auto
from protocol-created-tree-internalD[OF created len]
obtain l h r i where
  tree-eq:  $tree = \langle l, h, r \rangle$ 
and lookup:
  fmlookup (HashMap s) (MerkleNode (value l) (value r)) = Some h
by blast
show ?thesis
by (rule that[of MerkleNode (value l) (value r)])
  (use lookup in  $\langle \text{simp add: tree-eq} \rangle$ )
qed

```

```

lemma merkle-root-binds-nonempty-table-output-lookup:
fixes rt :: 'f
assumes bind: merkle-root-binds-table rt table s
and nonempty:  $table \neq []$ 
obtains input where
  fmlookup (HashMap s) input = Some rt
proof –
from bind obtain tree where
  created: created-tree table tree s
and root-eq:  $rt = \text{value tree}$ 
unfolding merkle-root-binds-table-def by blast
from protocol-created-tree-nonempty-root-lookup
  [OF created[unfolded created-tree-def] nonempty]
obtain input where
  fmlookup (HashMap s) input = Some (value tree)
by blast
then have lookup: fmlookup (HashMap s) input = Some rt
using root-eq by simp
show ?thesis

```

```

    by (rule that[OF lookup])
qed

lemma merkle-root-binds-table-pullback-or-new-tree-output-hit:
  assumes ext: prefix-state  $\leq$  final-state
    and bind: merkle-root-binds-table rt table final-state
  obtains tree where
    created-tree table tree final-state
    rt = value tree
    merkle-root-binds-table rt table prefix-state  $\vee$ 
    hash-map-new-output-hit (set-tree tree) prefix-state final-state
proof -
  from bind obtain tree where
    created: created-tree table tree final-state
    and rt-eq: rt = value tree
    unfolding merkle-root-binds-table-def by blast
  have pullback:
    created-tree table tree prefix-state  $\vee$ 
    hash-map-new-output-hit (set-tree tree) prefix-state final-state
  using protocol-created-tree-pullback-or-new-output-hit
    [OF created[unfolded created-tree-def] ext]
  unfolding created-tree-def .
  have
    merkle-root-binds-table rt table prefix-state  $\vee$ 
    hash-map-new-output-hit (set-tree tree) prefix-state final-state
  proof -
    from pullback show ?thesis
  proof
    assume created-tree table tree prefix-state
    then have merkle-root-binds-table rt table prefix-state
    unfolding merkle-root-binds-table-def
    using rt-eq by blast
    then show ?thesis by simp
  next
    assume hash-map-new-output-hit (set-tree tree) prefix-state final-state
    then show ?thesis by simp
  qed
qed
then show ?thesis
  by (rule that[OF created rt-eq])
qed

lemma merkle-root-binds-nonempty-table-late-root-output-preexisting-or-new-hit:
  fixes rt :: 'f
  assumes ext: s  $\leq$  t
    and bind: merkle-root-binds-table rt table t
    and nonempty: table  $\neq$  []
  shows
    ( $\exists$  input. fmllookup (HashMap s) input = Some rt)  $\vee$ 

```

```

      hash-map-new-output-hit {rt} s t
proof -
  from merkle-root-binds-nonempty-table-output-lookup[OF bind nonempty]
  obtain input where lookup-t:
    fmlookup (HashMap t) input = Some rt
  by blast
  show ?thesis
proof (cases fmlookup (HashMap s) input)
  case None
  then have hash-map-new-output-hit {rt} s t
    unfolding hash-map-new-output-hit-def
    using lookup-t by blast
  then show ?thesis by simp
next
  case (Some old)
  have lookup-t-old: fmlookup (HashMap t) input = Some old
    by (rule hash-extension-lookup[OF Some ext])
  then have old-eq: old = rt
    using lookup-t by simp
  then show ?thesis
    using Some by blast
qed
qed

lemma accepted-with-bound-tables-trace-root-late-output-preexisting-or-new-hit:
  assumes ext: prefix-state  $\leq$  final-state
  and bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
  obtains fr f-fri-roots f-final dg composition-fri-roots final rest
  where
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
    merkle-root-binds-table fr trace-table final-state
    ( $\exists$  input. fmlookup (HashMap prefix-state) input = Some fr)  $\vee$ 
    hash-map-new-output-hit {fr} prefix-state final-state
proof -
  from bound obtain fr f-fri-roots f-final dg composition-fri-roots final
  rest where
    header:
      verifier-header-transcript s fr f-fri-roots f-final as dg
      composition-fri-roots final rest
  and bind: merkle-root-binds-table fr trace-table final-state
  unfolding accepted-with-bound-tables-def by blast
  have nonempty: trace-table  $\neq$  []
  using accepted-with-bound-tables-shapes(1)[OF bound]
  eval-domain-nontrivial by auto
  have late:
    ( $\exists$  input. fmlookup (HashMap prefix-state) input = Some fr)  $\vee$ 

```



```

    hash-map-new-output-hit {fr} prefix-state final-state
  by (rule
    merkle-root-binds-nonempty-table-late-root-output-preexisting-or-new-hit
    [OF ext bind nonempty])
  show ?thesis
    by (rule that[OF header bind late])
qed

lemma accepted-with-bound-tables-composition-root-late-output-preexisting-or-new-hit:
assumes ext: prefix-state  $\leq$  final-state
and bound:
  accepted-with-bound-tables s (Some (result, final-state))
  trace-table composition-table as query-idxs
obtains fr f-fri-roots f-final dg composition-fri-roots final rest
where
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
  composition-fri-roots  $\neq$  []
  merkle-root-binds-table (hd composition-fri-roots) composition-table
  final-state
  ( $\exists$  input.
    fmlookup (HashMap prefix-state) input =
      Some (hd composition-fri-roots))  $\vee$ 
    hash-map-new-output-hit {hd composition-fri-roots} prefix-state
    final-state
proof –
from bound obtain fr f-fri-roots f-final dg composition-fri-roots final
  rest where
    header:
      verifier-header-transcript s fr f-fri-roots f-final as dg
      composition-fri-roots final rest
    and comp-nonempty: composition-fri-roots  $\neq$  []
    and bind:
      merkle-root-binds-table (hd composition-fri-roots)
      composition-table final-state
    unfolding accepted-with-bound-tables-def by blast
have nonempty: composition-table  $\neq$  []
using accepted-with-bound-tables-shapes(2)[OF bound]
  eval-domain-nontrivial by auto
have late:
  ( $\exists$  input.
    fmlookup (HashMap prefix-state) input =
      Some (hd composition-fri-roots))  $\vee$ 
    hash-map-new-output-hit {hd composition-fri-roots} prefix-state
    final-state
  by (rule
    merkle-root-binds-nonempty-table-late-root-output-preexisting-or-new-hit
    [OF ext bind nonempty])
show ?thesis

```

by (rule that[OF header comp-nonempty bind late])
qed

lemma *accepted-with-bound-tables-initial-roots-late-output-preexisting-or-new-hit*:
assumes *ext*: $\text{prefix-state} \leq \text{final-state}$
and *bound*:
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
obtains *fr f-fri-roots f-final dg composition-fri-roots final rest*
where
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
composition-fri-roots $\neq []$
merkle-root-binds-table fr trace-table final-state
merkle-root-binds-table (hd composition-fri-roots) composition-table
final-state
 $(\exists \text{input. fmlookup (HashMap prefix-state) input = Some fr}) \vee$
hash-map-new-output-hit {fr} prefix-state final-state
 $(\exists \text{input.}$
fmlookup (HashMap prefix-state) input =
Some (hd composition-fri-roots)) $\vee$
hash-map-new-output-hit {hd composition-fri-roots} prefix-state
final-state
proof –
from *accepted-with-bound-tables-trace-root-late-output-preexisting-or-new-hit*
[*OF ext bound*]
obtain *fr f-fri-roots f-final dg composition-fri-roots final rest* **where**
header:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *trace-bind*:
merkle-root-binds-table fr trace-table final-state
and *trace-late*:
 $(\exists \text{input. fmlookup (HashMap prefix-state) input = Some fr}) \vee$
hash-map-new-output-hit {fr} prefix-state final-state
by *blast*
from *accepted-with-bound-tables-composition-root-late-output-preexisting-or-new-hit*
[*OF ext bound*]
obtain *fr' f-fri-roots' f-final' dg' composition-fri-roots' final' rest'*
where *header'*:
verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
composition-fri-roots' final' rest'
and *comp-nonempty'*: $\text{composition-fri-roots}' \neq []$
and *comp-bind'*:
merkle-root-binds-table (hd composition-fri-roots') composition-table final-state
and *comp-late'*:
 $(\exists \text{input.}$
fmlookup (HashMap prefix-state) input =

```

    Some (hd composition-fri-roots')) ∨
    hash-map-new-output-hit {hd composition-fri-roots'} prefix-state
    final-state
  by blast
have eqs:
  fr' = fr ∧
  f-fri-roots' = f-fri-roots ∧
  f-final' = f-final ∧
  dg' = dg ∧
  composition-fri-roots' = composition-fri-roots ∧
  final' = final ∧
  rest' = rest
using verifier-header-transcript-unique[OF header header'] by simp
show ?thesis
by (rule that[OF header])
  (use comp-nonempty' comp-bind' comp-late' trace-bind trace-late eqs
   in simp-all)
qed

lemma accepted-with-bound-tables-initial-roots-late-output-preexisting-or-union-hit:
assumes ext: prefix-state ≤ final-state
and bound:
  accepted-with-bound-tables s (Some (result, final-state))
  trace-table composition-table as query-idxs
obtains fr f-fri-roots f-final dg composition-fri-roots final rest
where
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
  composition-fri-roots ≠ []
  merkle-root-binds-table fr trace-table final-state
  merkle-root-binds-table (hd composition-fri-roots) composition-table
  final-state
  ((∃ input. fmlookup (HashMap prefix-state) input = Some fr) ∧
   (∃ input.
    fmlookup (HashMap prefix-state) input =
    Some (hd composition-fri-roots))) ∨
  hash-map-new-output-hit {fr, hd composition-fri-roots} prefix-state
  final-state
proof –
from accepted-with-bound-tables-initial-roots-late-output-preexisting-or-new-hit
  [OF ext bound]
obtain fr f-fri-roots f-final dg composition-fri-roots final rest where
  header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and comp-nonempty: composition-fri-roots ≠ []
and trace-bind:
  merkle-root-binds-table fr trace-table final-state
and comp-bind:

```

```

merkle-root-binds-table (hd composition-fri-roots)
composition-table final-state
and trace-late:
  ( $\exists$  input. fmlookup (HashMap prefix-state) input = Some fr)  $\vee$ 
  hash-map-new-output-hit {fr} prefix-state final-state
and comp-late:
  ( $\exists$  input.
    fmlookup (HashMap prefix-state) input =
      Some (hd composition-fri-roots))  $\vee$ 
    hash-map-new-output-hit {hd composition-fri-roots} prefix-state
      final-state
by blast
have late-union:
  (( $\exists$  input. fmlookup (HashMap prefix-state) input = Some fr)  $\wedge$ 
    ( $\exists$  input.
      fmlookup (HashMap prefix-state) input =
        Some (hd composition-fri-roots)))  $\vee$ 
    hash-map-new-output-hit {fr, hd composition-fri-roots} prefix-state
      final-state
proof (cases
  ( $\exists$  input. fmlookup (HashMap prefix-state) input = Some fr)  $\wedge$ 
  ( $\exists$  input.
    fmlookup (HashMap prefix-state) input =
      Some (hd composition-fri-roots)))
case True
then show ?thesis by simp
next
case False
then have
  hash-map-new-output-hit {fr} prefix-state final-state  $\vee$ 
  hash-map-new-output-hit {hd composition-fri-roots} prefix-state
    final-state
using trace-late comp-late by blast
then show ?thesis
proof
assume hash-map-new-output-hit {fr} prefix-state final-state
then have
  hash-map-new-output-hit {fr, hd composition-fri-roots}
    prefix-state final-state
using hash-map-new-output-hit-subset[of {fr}
  {fr, hd composition-fri-roots} prefix-state final-state]
by auto
then show ?thesis by simp
next
assume
  hash-map-new-output-hit {hd composition-fri-roots} prefix-state
    final-state
then have
  hash-map-new-output-hit {fr, hd composition-fri-roots}

```

```

    prefix-state final-state
using hash-map-new-output-hit-subset
    [of {hd composition-fri-roots}
     {fr, hd composition-fri-roots} prefix-state final-state]
by auto
then show ?thesis by simp
qed
qed
show ?thesis
    by (rule that[OF header comp-nonempty trace-bind comp-bind late-union])
qed

lemma accepted-with-bound-tables-initial-roots-pullback-or-new-tree-output-hit:
assumes ext: prefix-state  $\leq$  final-state
and bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
obtains fr f-fri-roots f-final dg composition-fri-roots final rest
    trace-tree composition-tree
where
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
    composition-fri-roots  $\neq$  []
    created-tree trace-table trace-tree final-state
    fr = value trace-tree
    created-tree composition-table composition-tree final-state
    hd composition-fri-roots = value composition-tree
    (merkle-root-binds-table fr trace-table prefix-state  $\wedge$ 
     merkle-root-binds-table (hd composition-fri-roots) composition-table
     prefix-state)  $\vee$ 
    hash-map-new-output-hit
    (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
proof –
from bound obtain fr f-fri-roots f-final dg composition-fri-roots final
    rest where
    header:
        verifier-header-transcript s fr f-fri-roots f-final as dg
        composition-fri-roots final rest
and comp-nonempty: composition-fri-roots  $\neq$  []
and trace-bind:
        merkle-root-binds-table fr trace-table final-state
and comp-bind:
        merkle-root-binds-table (hd composition-fri-roots)
        composition-table final-state
unfolding accepted-with-bound-tables-def by blast
from merkle-root-binds-table-pullback-or-new-tree-output-hit
    [OF ext trace-bind]
obtain trace-tree where

```

```

    trace-created: created-tree trace-table trace-tree final-state
  and trace-root: fr = value trace-tree
  and trace-split:
    merkle-root-binds-table fr trace-table prefix-state ∨
    hash-map-new-output-hit (set-tree trace-tree) prefix-state
    final-state
  by blast
from merkle-root-binds-table-pullback-or-new-tree-output-hit
  [OF ext comp-bind]
obtain composition-tree where
  comp-created:
    created-tree composition-table composition-tree final-state
  and comp-root:
    hd composition-fri-roots = value composition-tree
  and comp-split:
    merkle-root-binds-table (hd composition-fri-roots)
    composition-table prefix-state ∨
    hash-map-new-output-hit (set-tree composition-tree) prefix-state
    final-state
  by blast
have split:
  (merkle-root-binds-table fr trace-table prefix-state ∧
   merkle-root-binds-table (hd composition-fri-roots) composition-table
   prefix-state) ∨
  hash-map-new-output-hit
  (set-tree trace-tree ∪ set-tree composition-tree)
  prefix-state final-state
proof —
  from trace-split show ?thesis
proof
  assume trace-prefix:
    merkle-root-binds-table fr trace-table prefix-state
  from comp-split show ?thesis
proof
  assume comp-prefix:
    merkle-root-binds-table (hd composition-fri-roots)
    composition-table prefix-state
  then show ?thesis
  using trace-prefix by simp
next
  assume comp-hit:
    hash-map-new-output-hit (set-tree composition-tree)
    prefix-state final-state
  have hash-map-new-output-hit
    (set-tree trace-tree ∪ set-tree composition-tree)
    prefix-state final-state
  by (rule hash-map-new-output-hit-subset[OF - comp-hit])
  simp
  then show ?thesis by simp

```

```

    qed
  next
    assume trace-hit:
      hash-map-new-output-hit (set-tree trace-tree) prefix-state
      final-state
    have hash-map-new-output-hit
      (set-tree trace-tree  $\cup$  set-tree composition-tree)
      prefix-state final-state
    by (rule hash-map-new-output-hit-subset[OF - trace-hit])
      simp
    then show ?thesis by simp
  qed
qed
show ?thesis
  by (rule that[OF header comp-nonempty trace-created trace-root
    comp-created comp-root split])
qed

lemma hash-map-new-output-hit-after-update:
  hash-map-new-output-hit B s
    (s(HashMap := fmupd x y (HashMap s)))  $\longleftrightarrow$ 
    fmlookup (HashMap s) x = None  $\wedge$  y  $\in$  B
proof
  assume hit:
    hash-map-new-output-hit B s
    (s(HashMap := fmupd x y (HashMap s)))
  then obtain k z where fresh:
    fmlookup (HashMap s) k = None
  and lookup:
    fmlookup (fmupd x y (HashMap s)) k = Some z
  and z-in: z  $\in$  B
  unfolding hash-map-new-output-hit-def by auto
  have k = x
  proof (rule ccontr)
    assume k  $\neq$  x
    then have fmlookup (HashMap s) k = Some z
      using lookup by simp
    then show False
      using fresh by simp
  qed
  then show fmlookup (HashMap s) x = None  $\wedge$  y  $\in$  B
    using fresh lookup z-in by simp
next
  assume fmlookup (HashMap s) x = None  $\wedge$  y  $\in$  B
  then show
    hash-map-new-output-hit B s
    (s(HashMap := fmupd x y (HashMap s)))
  unfolding hash-map-new-output-hit-def by auto
qed

```

```

lemma hash-extension-none:
  assumes ext:  $s \leq t$ 
    and none:  $\text{fmlookup } (\text{HashMap } t) \ x = \text{None}$ 
  shows  $\text{fmlookup } (\text{HashMap } s) \ x = \text{None}$ 
proof (cases fmlookup (HashMap s) x)
  case None
    then show ?thesis .
next
  case (Some y)
  have  $\text{fmlookup } (\text{HashMap } t) \ x = \text{Some } y$ 
    by (rule hash-extension-lookup[OF Some ext])
  then show ?thesis
    using none by simp
qed

lemma hash-map-new-output-hit-extend-initial:
  assumes ext:  $s \leq t$ 
    and hit:  $\text{hash-map-new-output-hit } B \ t \ u$ 
  shows  $\text{hash-map-new-output-hit } B \ s \ u$ 
proof –
  from hit obtain  $x \ y$  where fresh-t:
     $\text{fmlookup } (\text{HashMap } t) \ x = \text{None}$ 
    and lookup-u:  $\text{fmlookup } (\text{HashMap } u) \ x = \text{Some } y$ 
    and y-in:  $y \in B$ 
    unfolding hash-map-new-output-hit-def by blast
  have fresh-s:  $\text{fmlookup } (\text{HashMap } s) \ x = \text{None}$ 
    by (rule hash-extension-none[OF ext fresh-t])
  show ?thesis
    unfolding hash-map-new-output-hit-def
    using fresh-s lookup-u y-in by blast
qed

lemma hash-map-new-output-hit-trans-decomp:
  assumes ext-st:  $s \leq t$ 
    and ext-tu:  $t \leq u$ 
    and hit:  $\text{hash-map-new-output-hit } B \ s \ u$ 
  shows
     $\text{hash-map-new-output-hit } B \ s \ t \ \vee$ 
     $\text{hash-map-new-output-hit } B \ t \ u$ 
proof –
  from hit obtain  $x \ y$  where fresh-s:
     $\text{fmlookup } (\text{HashMap } s) \ x = \text{None}$ 
    and lookup-u:  $\text{fmlookup } (\text{HashMap } u) \ x = \text{Some } y$ 
    and y-in:  $y \in B$ 
    unfolding hash-map-new-output-hit-def by blast
  show ?thesis
proof (cases fmlookup (HashMap t) x)
  case None

```



```

then have hash-map-new-output-hit  $B\ t\ u$ 
  unfolding hash-map-new-output-hit-def
  using lookup-u y-in by blast
then show ?thesis by simp
next
case (Some z)
have lookup-u-z: fmlookup (HashMap u) x = Some z
  by (rule hash-extension-lookup[OF Some ext-tu])
have  $z = y$ 
  using lookup-u lookup-u-z by simp
then have hash-map-new-output-hit B s t
  unfolding hash-map-new-output-hit-def
  using fresh-s Some y-in by blast
then show ?thesis by simp
qed
qed

definition hash-extension-preserving
:: ('r, ('f, 'a) protocol-channel-scheme) state-monad  $\Rightarrow$  bool
where
  hash-extension-preserving m  $\longleftrightarrow$ 
    ( $\forall\ s\ x\ t.$ 
       $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m\ s) \longrightarrow s \leq t$ )

definition hash-map-preserving
:: ('r, ('f, 'a) protocol-channel-scheme) state-monad  $\Rightarrow$  bool
where
  hash-map-preserving m  $\longleftrightarrow$ 
    ( $\forall\ s\ x\ t.$ 
       $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m\ s) \longrightarrow$ 
       $\text{HashMap } t = \text{HashMap } s$ )

lemma hash-map-preserving-imp-hash-extension-preserving:
assumes hash-map-preserving m
shows hash-extension-preserving m
unfolding hash-extension-preserving-def
proof (intro allI impI)
fix  $s\ x\ t$ 
assume outcome: Some (x, t)  $\in$  set-dist (execute m s)
have HashMap t = HashMap s
  using assms outcome unfolding hash-map-preserving-def by blast
then show  $s \leq t$ 
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
qed

lemma hash-extension-preserving-return:
hash-extension-preserving (return x)
unfolding hash-extension-preserving-def
by (intro allI impI) (simp add: hash-ext-refl)

```

lemma *hash-extension-preserving-hash*:
fixes $x :: 'f \text{ protocol-hash-input}$
shows
hash-extension-preserving
 $(\text{hash } x :: ('f, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad})$
unfolding *hash-extension-preserving-def*
by (*intro allI impI*) (*rule hash-outcome(1)*)

lemma *hash-extension-preserving-bind*:
fixes $m :: ('x, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$
and $k :: 'x \Rightarrow ('y, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$
assumes *m-ext*: *hash-extension-preserving* m
and *k-ext*: $\bigwedge x. \text{hash-extension-preserving } (k \ x)$
shows *hash-extension-preserving* $(m \gg k)$
unfolding *hash-extension-preserving-def*
proof (*intro allI impI*)
fix $s \ y \ u$
assume *outcome*: *Some* $(y, u) \in \text{set-dist } (\text{execute } (m \gg k) \ s)$
from *outcome* **obtain** $x \ t$ **where** *m-out*:
Some $(x, t) \in \text{set-dist } (\text{execute } m \ s)$
and *k-out*: *Some* $(y, u) \in \text{set-dist } (\text{execute } (k \ x) \ t)$
by (*auto elim!: set-dist-bindE*)
have *ext-st*: $s \leq t$
using *m-ext m-out* **unfolding** *hash-extension-preserving-def* **by** *blast*
have *ext-tu*: $t \leq u$
using *k-ext[of x] k-out* **unfolding** *hash-extension-preserving-def* **by** *blast*
show $s \leq u$
by (*rule hash-ext-trans[OF ext-st ext-tu]*)
qed

lemma *wp-hash-new-output-hit*:
fixes $x :: 'f \text{ protocol-hash-input}$
and $s :: ('f, 'a) \text{ protocol-channel-scheme}$
shows
 $\text{wp-event } (\text{hash } x) (\text{hash-new-output-hit-event } B \ s) \ s =$
 $(\text{if } \text{fmlookup } (\text{HashMap } s) \ x = \text{None}$
 $\text{then } \text{nnreal } (\text{card } B) / \text{nnreal size}$
 $\text{else } 0)$
proof (*cases fmlookup (HashMap s) x*)
case *None*
have *indicator*:
 $(\lambda y. \text{if } \text{hash-map-new-output-hit } B \ s$
 $(s(\text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s)))$
 $\text{then } 1 \text{ else } 0 :: \text{prob}) =$
 $(\lambda y. \text{if } y \in B \text{ then } 1 \text{ else } 0)$
by (*rule ext*)
 $(\text{simp add: hash-map-new-output-hit-after-update None})$
have $\text{wp-event } (\text{hash } x) (\text{hash-new-output-hit-event } B \ s) \ s =$

```

    dist-expect (hash-dist x s) (λy. if y ∈ B then 1 else 0)
  unfolding wp-event-def hash-new-output-hit-event-def
  by (simp add: wp-hash indicator)
also have ... = nnreal (card B) / nnreal size
  by (rule dist-expect-hash-dist-fresh-indicator[OF None])
finally show ?thesis
  using None by simp
next
case (Some y)
have no-hit:
  ¬ hash-map-new-output-hit B s
  (s(HashMap := fmupd x y (HashMap s)))
  by (simp add: hash-map-new-output-hit-after-update Some)
show ?thesis
  unfolding wp-event-def hash-new-output-hit-event-def
  by (simp add: wp-hash Some hash-dist-def option-default-dist-def no-hit)
qed

```

lemma *wp-hash-new-output-hit-bound*:

```

  fixes x :: 'f protocol-hash-input
  and s :: ('f, 'a) protocol-channel-scheme
  shows
    wp-event (hash x) (hash-new-output-hit-event B s) s ≤
      nnreal (card B) / nnreal size
  unfolding wp-hash-new-output-hit
  by simp

```

definition *hash-target-budget-value*

```

:: 'f set ⇒ nat ⇒ prob
where
  hash-target-budget-value B n =
    nnreal (n * card B) / nnreal size

```

definition *hash-target-budget*

```

:: 'f set ⇒ nat ⇒
  ('r, ('f, 'a) protocol-channel-scheme) state-monad ⇒ bool
where
  hash-target-budget B n m ⟷
    (∀ s.
      wp-event m (hash-new-output-hit-event B s) s ≤
        hash-target-budget-value B n)

```

lemma *hash-target-budget-value-add*:

```

  hash-target-budget-value B n + hash-target-budget-value B m =
    hash-target-budget-value B (n + m)
  unfolding hash-target-budget-value-def
  by (simp add: add-divide-nnreal algebra-simps)

```

lemma *hash-new-output-hit-event-step-bound*:

```

fixes  $m :: ('x, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$ 
assumes  $\text{ext-st}: s \leq t$ 
and  $\text{ext-m}: \text{hash-extension-preserving } m$ 
shows
   $\text{wp-event } m (\text{hash-new-output-hit-event } B \ s) \ t \leq$ 
     $(\text{if } \text{hash-map-new-output-hit } B \ s \ t \text{ then } 1$ 
       $\text{else } \text{wp-event } m (\text{hash-new-output-hit-event } B \ t) \ t)$ 
proof  $(\text{cases } \text{hash-map-new-output-hit } B \ s \ t)$ 
  case True
then show ?thesis
  using  $\text{wp-event-le-1}[\text{of } m \ \text{hash-new-output-hit-event } B \ s \ t]$ 
  by simp
next
  case False
have mono:
   $\text{wp-event } m (\text{hash-new-output-hit-event } B \ s) \ t \leq$ 
     $\text{wp-event } m (\text{hash-new-output-hit-event } B \ t) \ t$ 
proof  $(\text{rule } \text{wp-event-mono-on-support})$ 
  fix out
assume  $\text{support}: \text{out} \in \text{set-dist } (\text{execute } m \ t)$ 
and  $\text{hit-s}: \text{hash-new-output-hit-event } B \ s \ \text{out}$ 
show  $\text{hash-new-output-hit-event } B \ t \ \text{out}$ 
proof  $(\text{cases } \text{out})$ 
  case None
then show ?thesis
  using hit-s unfolding  $\text{hash-new-output-hit-event-def}$  by simp
next
  case  $(\text{Some } xu)$ 
then obtain  $x \ u$  where  $xu: xu = (x, u)$ 
  by  $(\text{cases } xu) \ \text{simp}$ 
have  $\text{out}: \text{Some } (x, u) \in \text{set-dist } (\text{execute } m \ t)$ 
  using support  $\text{Some } xu$  by simp
have  $\text{ext-tu}: t \leq u$ 
  using ext-m out unfolding  $\text{hash-extension-preserving-def}$  by blast
have  $\text{hit-su}: \text{hash-map-new-output-hit } B \ s \ u$ 
  using hit-s  $\text{Some } xu$  unfolding  $\text{hash-new-output-hit-event-def}$  by simp
have  $\text{hash-map-new-output-hit } B \ s \ t \vee$ 
   $\text{hash-map-new-output-hit } B \ t \ u$ 
  by  $(\text{rule } \text{hash-map-new-output-hit-trans-decomp}$ 
     $[\text{OF } \text{ext-st } \text{ext-tu } \text{hit-su}])$ 
then have  $\text{hash-map-new-output-hit } B \ t \ u$ 
  using False by blast
then show ?thesis
  using  $\text{Some } xu$  unfolding  $\text{hash-new-output-hit-event-def}$  by simp
qed
qed
show ?thesis
using False mono by simp
qed

```

```

lemma hash-map-new-output-hit-refl[simp]:
   $\neg$  hash-map-new-output-hit B s s
  unfolding hash-map-new-output-hit-def by auto

lemma hash-target-budget-return:
  hash-target-budget B 0 (return x)
  unfolding hash-target-budget-def hash-target-budget-value-def
    hash-new-output-hit-event-def
  by (simp add: wp-event-def wpsimps)

lemma hash-map-preserving-no-new-output-hit-on-support:
  assumes preserving: hash-map-preserving m
    and support: out  $\in$  set-dist (execute m s)
  shows  $\neg$  hash-new-output-hit-event B s out
proof (cases out)
  case None
  then show ?thesis
    unfolding hash-new-output-hit-event-def by simp
next
  case (Some xt)
  then obtain x t where xt: xt = (x, t)
    by (cases xt) simp
  have out: Some (x, t)  $\in$  set-dist (execute m s)
    using support Some xt by simp
  have map-eq: HashMap t = HashMap s
    using preserving out unfolding hash-map-preserving-def by blast
  have  $\neg$  hash-map-new-output-hit B s t
    using map-eq unfolding hash-map-new-output-hit-def by auto
  then show ?thesis
    using Some xt unfolding hash-new-output-hit-event-def by simp
qed

lemma hash-map-preserving-imp-hash-target-budget-zero:
  assumes preserving: hash-map-preserving m
  shows hash-target-budget B 0 m
  unfolding hash-target-budget-def hash-target-budget-value-def
  apply (intro allI)
  apply (rule order-trans [
    where y=wp-event m ( $\lambda$ -. False) -])
  apply (rule wp-event-mono-on-support)
  apply (meson hash-map-preserving-no-new-output-hit-on-support preserving)
  unfolding wp-event-def wp-def dist-expect-def
  by simp

lemma hash-target-budget-hash:
  fixes x :: 'f protocol-hash-input
  shows
    hash-target-budget B 1

```

```

    (hash x :: ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding hash-target-budget-def
proof (intro allI)
  fix s :: ('f, 'a) protocol-channel-scheme
  show
    wp-event (hash x) (hash-new-output-hit-event B s) s ≤
      hash-target-budget-value B 1
  using wp-hash-new-output-hit-bound[where x=x and s=s and B=B]
  unfolding hash-target-budget-value-def by simp
qed

```

```

lemma hash-target-budget-bind:
  fixes m :: ('x, ('f, 'a) protocol-channel-scheme) state-monad
    and k :: 'x ⇒ ('y, ('f, 'a) protocol-channel-scheme) state-monad
  assumes m-ext: hash-extension-preserving m
    and m-budget: hash-target-budget B n m
    and k-ext:  $\bigwedge x. \text{hash-extension-preserving } (k\ x)$ 
    and k-budget:  $\bigwedge x. \text{hash-target-budget } B\ n'\ (k\ x)$ 
  shows hash-target-budget B (n + n') (m  $\ggg$  k)
  unfolding hash-target-budget-def
proof (intro allI)
  fix s :: ('f, 'a) protocol-channel-scheme
  let ?E = hash-new-output-hit-event B s
  let ?Tail =
     $\lambda out. \text{case out of}$ 
     $\text{None} \Rightarrow 0$ 
     $| \text{Some } (x, t) \Rightarrow$ 
     $\text{if hash-map-new-output-hit } B\ s\ t \text{ then } 0$ 
     $\text{else wp-event } (k\ x) (\text{hash-new-output-hit-event } B\ t)\ t$ 
  let ?R =  $\lambda out. (\text{if } ?E\ out \text{ then } 1 \text{ else } 0) + ?Tail\ out$ 
  have split:
    wp-event (m  $\ggg$  k) ?E s ≤ wp m ?R s
  proof -
    have exact:
      wp-event (m  $\ggg$  k) ?E s =
        wp m
        ( $\lambda out. \text{case out of}$ 
         $\text{None} \Rightarrow 0$ 
         $| \text{Some } (x, t) \Rightarrow \text{wp-event } (k\ x) ?E\ t)\ s$ 
    unfolding wp-event-def hash-new-output-hit-event-def
    by (simp add: wpsimps)
  have wp m
    ( $\lambda out. \text{case out of}$ 
     $\text{None} \Rightarrow 0$ 
     $| \text{Some } (x, t) \Rightarrow \text{wp-event } (k\ x) ?E\ t)\ s$ 
    ≤ wp m ?R s
  proof (rule wp-mono-on-support)
    fix out
    assume support: out ∈ set-dist (execute m s)

```

```

show (case out of
  None  $\Rightarrow 0$ 
  | Some (x, t)  $\Rightarrow$  wp-event (k x) ?E t  $\leq$  ?R out
proof (cases out)
  case None
  then show ?thesis
    unfolding hash-new-output-hit-event-def by simp
next
  case (Some xt)
  then obtain x t where xt: xt = (x, t)
    by (cases xt) simp
  have m-out: Some (x, t)  $\in$  set-dist (execute m s)
    using support Some xt by simp
  have ext-st: s  $\leq$  t
    using m-ext m-out unfolding hash-extension-preserving-def by blast
  have step:
    wp-event (k x) ?E t  $\leq$ 
      (if hash-map-new-output-hit B s t then 1
       else wp-event (k x) (hash-new-output-hit-event B t) t)
    by (rule hash-new-output-hit-event-step-bound
      [OF ext-st k-ext[of x]])
  show ?thesis
proof (cases hash-map-new-output-hit B s t)
  case True
  then show ?thesis
    using Some xt step
    unfolding hash-new-output-hit-event-def by simp
next
  case False
  then show ?thesis
    using Some xt step
    unfolding hash-new-output-hit-event-def by simp
qed
qed
qed
then show ?thesis
  unfolding exact .
qed
have head-bound:
  wp-event m ?E s  $\leq$  hash-target-budget-value B n
  using m-budget unfolding hash-target-budget-def by blast
have tail-bound:
  wp m ?Tail s  $\leq$  hash-target-budget-value B n'
proof (rule wp-le-const-on-support)
  fix out
  assume support: out  $\in$  set-dist (execute m s)
  show ?Tail out  $\leq$  hash-target-budget-value B n'
proof (cases out)
  case None

```

```

    then show ?thesis
      unfolding hash-target-budget-value-def by simp
next
  case (Some xt)
  then obtain x t where xt: xt = (x, t)
    by (cases xt) simp
  have tail:
    wp-event (k x) (hash-new-output-hit-event B t) t ≤
      hash-target-budget-value B n'
    using k-budget[of x] unfolding hash-target-budget-def by blast
  show ?thesis
    using Some xt tail by simp
qed
qed
have wp-event (m ≫ k) ?E s ≤ wp m ?R s
  by (rule split)
also have ... = wp-event m ?E s + wp m ?Tail s
  unfolding wp-event-def wp-def dist-expect-def
  by (simp add: sum.distrib algebra-simps
      hash-new-output-hit-event-def)
also have ... ≤
  hash-target-budget-value B n + hash-target-budget-value B n'
  by (intro add-mono head-bound tail-bound)
also have ... = hash-target-budget-value B (n + n')
  by (rule hash-target-budget-value-add)
finally show
  wp-event (m ≫ k) (hash-new-output-hit-event B s) s ≤
    hash-target-budget-value B (n + n') .
qed

lemma hash-target-budget-value-mono:
  assumes m ≤ n
  shows hash-target-budget-value B m ≤ hash-target-budget-value B n
  unfolding hash-target-budget-value-def
  apply (subst nn2real-le-iff[symmetric])
  using assms by (simp add: divide-right-mono mult-right-mono)

lemma hash-target-budget-mono:
  assumes m ≤ n
  and budget: hash-target-budget B m p
  shows hash-target-budget B n p
  unfolding hash-target-budget-def
proof (intro allI)
  fix s
  have wp-event p (hash-new-output-hit-event B s) s ≤
    hash-target-budget-value B m
    using budget unfolding hash-target-budget-def by blast
  also have ... ≤ hash-target-budget-value B n
    by (rule hash-target-budget-value-mono[OF assms(1)])

```


finally show
 $wp\text{-}event\ p\ (hash\text{-}new\text{-}output\text{-}hit\text{-}event\ B\ s)\ s \leq$
 $hash\text{-}target\text{-}budget\text{-}value\ B\ n\ .$
qed

definition *hash-target-program*
 $:: 'f\ set \Rightarrow nat \Rightarrow$
 $('r, ('f, 'a)\ protocol\text{-}channel\text{-}scheme)\ state\text{-}monad \Rightarrow bool$
where
 $hash\text{-}target\text{-}program\ B\ n\ m \longleftrightarrow$
 $hash\text{-}extension\text{-}preserving\ m \wedge hash\text{-}target\text{-}budget\ B\ n\ m$

lemma *hash-target-program-extension:*
assumes *hash-target-program* $B\ n\ m$
shows *hash-extension-preserving* m
using *assms* **unfolding** *hash-target-program-def* **by** *simp*

lemma *hash-target-program-budget:*
assumes *hash-target-program* $B\ n\ m$
shows *hash-target-budget* $B\ n\ m$
using *assms* **unfolding** *hash-target-program-def* **by** *simp*

lemma *hash-target-program-mono:*
assumes $m \leq n$
and *program:* *hash-target-program* $B\ m\ p$
shows *hash-target-program* $B\ n\ p$
unfolding *hash-target-program-def*
proof
show *hash-extension-preserving* p
using *program* **unfolding** *hash-target-program-def* **by** *simp*
have *hash-target-budget* $B\ m\ p$
using *program* **unfolding** *hash-target-program-def* **by** *simp*
then show *hash-target-budget* $B\ n\ p$
by (*rule* *hash-target-budget-mono*[*OF* *assms*(1)])
qed

lemma *hash-target-program-return:*
 $hash\text{-}target\text{-}program\ B\ 0\ (return\ x)$
unfolding *hash-target-program-def*
by (*intro* *conjI* *hash-extension-preserving-return* *hash-target-budget-return*)

lemma *hash-target-program-hash:*
fixes $x :: 'f\ protocol\text{-}hash\text{-}input$
shows
 $hash\text{-}target\text{-}program\ B\ 1$
 $(hash\ x :: ('f, ('f, 'a)\ protocol\text{-}channel\text{-}scheme)\ state\text{-}monad)$
unfolding *hash-target-program-def*
by (*intro* *conjI* *hash-extension-preserving-hash* *hash-target-budget-hash*)

lemma *hash-map-preserving-imp-hash-target-program-zero*:
assumes *hash-map-preserving m*
shows *hash-target-program B 0 m*
unfolding *hash-target-program-def*
using *assms hash-map-preserving-imp-hash-extension-preserving*
hash-map-preserving-imp-hash-target-budget-zero
by *blast*

lemma *hash-target-program-bind*:
fixes $m :: ('x, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$
and $k :: 'x \Rightarrow ('y, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$
assumes $m: \text{hash-target-program } B \ n \ m$
and $k: \bigwedge x. \text{hash-target-program } B \ n' \ (k \ x)$
shows *hash-target-program B (n + n') (m $\gg$ k)*
unfolding *hash-target-program-def*
proof (*intro conjI*)
show *hash-extension-preserving (m $\gg$ k)*
by (*rule hash-extension-preserving-bind*)
(use m k in <simp-all add: hash-target-program-def>)
show *hash-target-budget B (n + n') (m $\gg$ k)*
by (*rule hash-target-budget-bind*)
(use m k in <simp-all add: hash-target-program-def>)
qed

lemma *hash-extension-preserving-bind-on-outcomes*:
fixes $m :: ('x, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$
and $k :: 'x \Rightarrow ('y, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$
assumes *m-ext: hash-extension-preserving m*
and *k-ext:*
 $\bigwedge s \ x \ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$
 $\text{hash-extension-preserving } (k \ x)$
shows *hash-extension-preserving (m $\gg$ k)*
unfolding *hash-extension-preserving-def*
proof (*intro allI impI*)
fix $s \ y \ u$
assume *outcome: Some (y, u) $\in$ set-dist (execute (m $\gg$ k) s)*
from *outcome* **obtain** $x \ t$ **where** *m-out:*
 $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s)$
and *k-out: Some (y, u) $\in$ set-dist (execute (k x) t)*
by (*auto elim!: set-dist-bindE*)
have *ext-st: s $\leq$ t*
using *m-ext m-out* **unfolding** *hash-extension-preserving-def* **by** *blast*
have *ext-tu: t $\leq$ u*
using *k-ext[OF m-out] k-out*
unfolding *hash-extension-preserving-def* **by** *blast*
show $s \leq u$
by (*rule hash-ext-trans[OF ext-st ext-tu]*)
qed

lemma *hash-target-budget-bind-on-outcomes*:

fixes $m :: ('x, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$
and $k :: 'x \Rightarrow ('y, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$

assumes *m-ext*: *hash-extension-preserving* m
and *m-budget*: *hash-target-budget* $B \ n \ m$
and *k-ext*:
 $\bigwedge s \ x \ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$
hash-extension-preserving $(k \ x)$
and *k-budget*:
 $\bigwedge s \ x \ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$
hash-target-budget $B \ n' \ (k \ x)$

shows *hash-target-budget* $B \ (n + n') \ (m \ggg k)$
unfolding *hash-target-budget-def*

proof (*intro allI*)
fix $s :: ('f, 'a) \text{ protocol-channel-scheme}$
let $?E = \text{hash-new-output-hit-event } B \ s$
let $?Tail =$
 $\lambda out. \text{case out of}$
 $\quad \text{None} \Rightarrow 0$
 $\quad | \text{Some } (x, t) \Rightarrow$
 $\quad \quad \text{if } \text{hash-map-new-output-hit } B \ s \ t \text{ then } 0$
 $\quad \quad \text{else } \text{wp-event } (k \ x) \ (\text{hash-new-output-hit-event } B \ t) \ t$
let $?R = \lambda out. (\text{if } ?E \text{ out then } 1 \text{ else } 0) + ?Tail \text{ out}$
have *split*:
 $\text{wp-event } (m \ggg k) \ ?E \ s \leq \text{wp } m \ ?R \ s$

proof –
have *exact*:
 $\text{wp-event } (m \ggg k) \ ?E \ s =$
 $\text{wp } m$
 $(\lambda out. \text{case out of}$
 $\quad \text{None} \Rightarrow 0$
 $\quad | \text{Some } (x, t) \Rightarrow \text{wp-event } (k \ x) \ ?E \ t) \ s$
unfolding *wp-event-def hash-new-output-hit-event-def*
by (*simp add: wpsimps*)

have $\text{wp } m$
 $(\lambda out. \text{case out of}$
 $\quad \text{None} \Rightarrow 0$
 $\quad | \text{Some } (x, t) \Rightarrow \text{wp-event } (k \ x) \ ?E \ t) \ s$
 $\leq \text{wp } m \ ?R \ s$

proof (*rule wp-mono-on-support*)
fix out
assume *support*: $out \in \text{set-dist } (\text{execute } m \ s)$
show (*case out of*
 $\quad \text{None} \Rightarrow 0$
 $\quad | \text{Some } (x, t) \Rightarrow \text{wp-event } (k \ x) \ ?E \ t) \leq ?R \ out$

proof (*cases out*)
case *None*
then show *?thesis*
unfolding *hash-new-output-hit-event-def* **by** *simp*

```

next
  case (Some xt)
  then obtain x t where xt: xt = (x, t)
    by (cases xt) simp
  have m-out: Some (x, t) ∈ set-dist (execute m s)
    using support Some xt by simp
  have ext-st: s ≤ t
    using m-ext m-out unfolding hash-extension-preserving-def by blast
  have step:
    wp-event (k x) ?E t ≤
      (if hash-map-new-output-hit B s t then 1
       else wp-event (k x) (hash-new-output-hit-event B t) t)
    by (rule hash-new-output-hit-event-step-bound
        [OF ext-st k-ext[OF m-out]])
  show ?thesis
    using Some xt step
    unfolding hash-new-output-hit-event-def
    by (cases hash-map-new-output-hit B s t) simp-all
qed
qed
then show ?thesis
  unfolding exact .
qed
have head-bound:
  wp-event m ?E s ≤ hash-target-budget-value B n
  using m-budget unfolding hash-target-budget-def by blast
have tail-bound:
  wp m ?Tail s ≤ hash-target-budget-value B n'
proof (rule wp-le-const-on-support)
  fix out
  assume support: out ∈ set-dist (execute m s)
  show ?Tail out ≤ hash-target-budget-value B n'
  proof (cases out)
    case None
    then show ?thesis
      unfolding hash-target-budget-value-def by simp
  next
    case (Some xt)
    then obtain x t where xt: xt = (x, t)
      by (cases xt) simp
    have m-out: Some (x, t) ∈ set-dist (execute m s)
      using support Some xt by simp
    have tail:
      wp-event (k x) (hash-new-output-hit-event B t) t ≤
        hash-target-budget-value B n'
      using k-budget[OF m-out] unfolding hash-target-budget-def by blast
    show ?thesis
      using Some xt tail by simp
  qed
qed

```

```

qed
have wp-event (m  $\gg$  k) ?E s  $\leq$  wp m ?R s
  by (rule split)
also have ... = wp-event m ?E s + wp m ?Tail s
  unfolding wp-event-def wp-def dist-expect-def
  by (simp add: sum.distrib algebra-simps
      hash-new-output-hit-event-def)
also have ...  $\leq$ 
  hash-target-budget-value B n + hash-target-budget-value B n'
  by (intro add-mono head-bound tail-bound)
also have ... = hash-target-budget-value B (n + n')
  by (rule hash-target-budget-value-add)
finally show
  wp-event (m  $\gg$  k) (hash-new-output-hit-event B s) s  $\leq$ 
  hash-target-budget-value B (n + n') .
qed

lemma hash-target-program-bind-on-outcomes:
  fixes m :: ('x, ('f, 'a) protocol-channel-scheme) state-monad
  and k :: 'x  $\Rightarrow$  ('y, ('f, 'a) protocol-channel-scheme) state-monad
  assumes m: hash-target-program B n m
  and k:
     $\bigwedge s x t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \Rightarrow$ 
    hash-target-program B n' (k x)
  shows hash-target-program B (n + n') (m  $\gg$  k)
  unfolding hash-target-program-def
proof (intro conjI)
  show hash-extension-preserving (m  $\gg$  k)
    by (rule hash-extension-preserving-bind-on-outcomes)
    (use m k in  $\langle$ simp-all add: hash-target-program-def $\rangle$ )
  show hash-target-budget B (n + n') (m  $\gg$  k)
    by (rule hash-target-budget-bind-on-outcomes)
    (use m k in  $\langle$ simp-all add: hash-target-program-def $\rangle$ )
qed

lemma hash-target-program-ntimes:
  fixes m :: ('x, ('f, 'a) protocol-channel-scheme) state-monad
  assumes m: hash-target-program B n m
  shows hash-target-program B (k * n) (ntimes m k)
  using m
proof (induction k)
  case 0
  then show ?case
    by (simp add: hash-target-program-return)
next
  case (Suc k)
  have tail: hash-target-program B (k * n) (ntimes m k)
    using Suc.IH Suc.premis by simp
  have cont:

```

```

  hash-target-program B (k * n + 0)
    (ntimes m k >>= (λxs. return (x # xs))) for x
  by (rule hash-target-program-bind[OF tail hash-target-program-return])
have hash-target-program B (n + (k * n + 0))
  (m >>= (λx. ntimes m k >>= (λxs. return (x # xs))))
  by (rule hash-target-program-bind[OF Suc.premis cont])
then show ?case by simp
qed

```

```

lemma hash-target-program-mmap:
  fixes ms :: ('x, ('f, 'a) protocol-channel-scheme) state-monad list
  assumes ms:  $\bigwedge m. m \in \text{set } ms \implies \text{hash-target-program } B \ n \ m$ 
  shows hash-target-program B (length ms * n) (mmap ms)
  using ms
proof (induction ms)
  case Nil
  then show ?case
    by (simp add: hash-target-program-return)
next
  case (Cons m ms)
  have head: hash-target-program B n m
    using Cons.premis by simp
  have tail: hash-target-program B (length ms * n) (mmap ms)
    using Cons.IH Cons.premis by simp
  have cont:
    hash-target-program B (length ms * n + 0)
      (mmap ms >>= (λxs. return (x # xs))) for x
    by (rule hash-target-program-bind[OF tail hash-target-program-return])
  have hash-target-program B (n + (length ms * n + 0))
    (m >>= (λx. mmap ms >>= (λxs. return (x # xs))))
    by (rule hash-target-program-bind[OF head cont])
  then show ?case by simp
qed

```

```

lemma hash-target-program-mfold:
  fixes steps ::
    ('x  $\Rightarrow$  ('x, ('f, 'a) protocol-channel-scheme) state-monad) list
  assumes steps:
     $\bigwedge \text{step } x. \text{step} \in \text{set steps} \implies$ 
      hash-target-program B n (step x)
  shows hash-target-program B (length steps * n) (mfold x steps)
  using steps
proof (induction steps arbitrary: x)
  case Nil
  then show ?case
    by (simp add: hash-target-program-return)
next
  case (Cons step steps)
  have head: hash-target-program B n (step x)

```

```

    using Cons.premis by simp
  have tail:
     $\bigwedge y. \text{hash-target-program } B \text{ (length steps * n) (mfold y steps)}$ 
    using Cons.IH Cons.premis by simp
  have hash-target-program  $B \text{ (n + length steps * n)}$ 
    (step  $x \gg (\lambda y. \text{mfold y steps})$ )
    by (rule hash-target-program-bind[OF head tail])
  then show ?case by simp
qed

end

end

```

```

theory Soundness-Oracle-Budgets
  imports Soundness-Oracle-Target
begin

```

Random-oracle range and collision budget calculus for protocol programs.

```

context soundness
begin

```

```

definition hash-collision-budget-value :: nat  $\Rightarrow$  nat  $\Rightarrow$  prob
where
  hash-collision-budget-value c n =
    nnreal (n * (c + n)) / nnreal size

```

```

definition hash-range-budget
:: nat  $\Rightarrow$  ('r, ('f, 'a) protocol-channel-scheme) state-monad  $\Rightarrow$  bool
where
  hash-range-budget n m  $\longleftrightarrow$ 
    ( $\forall s \ x \ t. \text{Some } (x, t) \in \text{set-dist (execute m s)} \longrightarrow$ 
      card (hash-map-output-values t)  $\leq$ 
      card (hash-map-output-values s) + n)

```

```

definition hash-collision-budget
:: nat  $\Rightarrow$  ('r, ('f, 'a) protocol-channel-scheme) state-monad  $\Rightarrow$  bool
where
  hash-collision-budget n m  $\longleftrightarrow$ 
    ( $\forall s. \neg \text{hash-map-output-collision s} \longrightarrow$ 
      wp-event m (hash-new-collision-event s) s  $\leq$ 
      hash-collision-budget-value (card (hash-map-output-values s)) n)

```

```

lemma hash-collision-budget-value-mono-left:
  assumes c  $\leq$  d
  shows hash-collision-budget-value c n  $\leq$ 
    hash-collision-budget-value d n

```

unfolding *hash-collision-budget-value-def*
apply (*subst nn2real-le-iff[symmetric]*)
using *assms* **by** (*simp add: divide-right-mono add-mono mult-left-mono*)

lemma *hash-collision-budget-value-mono-right*:
assumes $m \leq n$
shows *hash-collision-budget-value* $c\ m \leq$
hash-collision-budget-value $c\ n$
unfolding *hash-collision-budget-value-def*
apply (*subst nn2real-le-iff[symmetric]*)
using *assms* **by** (*simp add: divide-right-mono add-mono mult-mono*)

lemma *hash-range-budget-mono*:
fixes $p :: ('r, ('f, 'a)\ \text{protocol-channel-scheme})\ \text{state-monad}$
assumes $m \leq n$
and *budget*: *hash-range-budget* $m\ p$
shows *hash-range-budget* $n\ p$
using *assms* **unfolding** *hash-range-budget-def* **by** *force*

lemma *hash-collision-budget-mono*:
fixes $p :: ('r, ('f, 'a)\ \text{protocol-channel-scheme})\ \text{state-monad}$
assumes $m \leq n$
and *budget*: *hash-collision-budget* $m\ p$
shows *hash-collision-budget* $n\ p$
unfolding *hash-collision-budget-def*
proof (*intro allI impI*)
fix $s :: ('f, 'a)\ \text{protocol-channel-scheme}$
assume *clean*: $\neg\ \text{hash-map-output-collision}\ s$
have *wp-event* $p\ (\text{hash-new-collision-event}\ s)\ s \leq$
hash-collision-budget-value (*card* (*hash-map-output-values* s)) m
using *budget clean* **unfolding** *hash-collision-budget-def* **by** *blast*
also have $\dots \leq$
hash-collision-budget-value (*card* (*hash-map-output-values* s)) n
by (*rule hash-collision-budget-value-mono-right[OF assms(1)]*)
finally show *wp-event* $p\ (\text{hash-new-collision-event}\ s)\ s \leq$
hash-collision-budget-value (*card* (*hash-map-output-values* s)) n .
qed

lemma *hash-collision-budget-value-bind-le*:
hash-collision-budget-value $c\ n +$
hash-collision-budget-value $(c + n)\ n' \leq$
hash-collision-budget-value $c\ (n + n')$
unfolding *hash-collision-budget-value-def*
by (*simp add: add-divide-nnreal algebra-simps*)

lemma *hash-range-budget-bind*:
fixes $m :: ('x, ('f, 'a)\ \text{protocol-channel-scheme})\ \text{state-monad}$
and $k :: 'x \Rightarrow ('y, ('f, 'a)\ \text{protocol-channel-scheme})\ \text{state-monad}$
assumes *m*: *hash-range-budget* $n\ m$

and k : $\bigwedge x. \text{hash-range-budget } n' (k x)$
 shows $\text{hash-range-budget } (n + n') (m \ggg k)$
 unfolding $\text{hash-range-budget-def}$
 proof (intro allI impI)
 fix $s y t$
 assume outcome : $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (m \ggg k) s)$
 from outcome obtain $x s1$ where
 $m\text{-out}$: $\text{Some } (x, s1) \in \text{set-dist } (\text{execute } m s)$
 and $k\text{-out}$: $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (k x) s1)$
 by (auto elim!: set-dist-bindE)
 have $m\text{-range}$:
 $\text{card } (\text{hash-map-output-values } s1) \leq$
 $\text{card } (\text{hash-map-output-values } s) + n$
 using $m m\text{-out}$ unfolding $\text{hash-range-budget-def}$ by blast
 have $k\text{-range}$:
 $\text{card } (\text{hash-map-output-values } t) \leq$
 $\text{card } (\text{hash-map-output-values } s1) + n'$
 using $k k\text{-out}$ unfolding $\text{hash-range-budget-def}$ by blast
 show $\text{card } (\text{hash-map-output-values } t)$
 $\leq \text{card } (\text{hash-map-output-values } s) + (n + n')$
 using $m\text{-range } k\text{-range}$ by linarith
 qed

lemma $\text{hash-range-budget-bind-on-outcomes}$:
 fixes $m :: ('x, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$
 and $k :: 'x \Rightarrow ('y, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$
 assumes m : $\text{hash-range-budget } n m$
 and k :
 $\bigwedge s x t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m s) \implies$
 $\text{hash-range-budget } n' (k x)$
 shows $\text{hash-range-budget } (n + n') (m \ggg k)$
 unfolding $\text{hash-range-budget-def}$
 proof (intro allI impI)
 fix $s y t$
 assume outcome : $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (m \ggg k) s)$
 from outcome obtain $x s1$ where
 $m\text{-out}$: $\text{Some } (x, s1) \in \text{set-dist } (\text{execute } m s)$
 and $k\text{-out}$: $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (k x) s1)$
 by (auto elim!: set-dist-bindE)
 have $m\text{-range}$:
 $\text{card } (\text{hash-map-output-values } s1) \leq$
 $\text{card } (\text{hash-map-output-values } s) + n$
 using $m m\text{-out}$ unfolding $\text{hash-range-budget-def}$ by blast
 have $k\text{-range}$:
 $\text{card } (\text{hash-map-output-values } t) \leq$
 $\text{card } (\text{hash-map-output-values } s1) + n'$
 using $k[OF m\text{-out}] k\text{-out}$ unfolding $\text{hash-range-budget-def}$ by blast
 show $\text{card } (\text{hash-map-output-values } t)$
 $\leq \text{card } (\text{hash-map-output-values } s) + (n + n')$

```

    using m-range k-range by linarith
qed

lemma hash-new-collision-event-step-bound:
  fixes m :: ('x, ('f, 'a) protocol-channel-scheme) state-monad
    and s t :: ('f, 'a) protocol-channel-scheme
  assumes clean:  $\neg$  hash-map-output-collision s
  shows
    wp-event m (hash-new-collision-event s) t  $\leq$ 
      (if hash-map-new-output-collision s t then 1
       else wp-event m (hash-new-collision-event t) t)
proof (cases hash-map-new-output-collision s t)
  case True
  then show ?thesis
    using wp-event-le-1[of m hash-new-collision-event s t] by simp
next
  case False
  then have clean-t:  $\neg$  hash-map-output-collision t
    using clean unfolding hash-map-new-output-collision-def by simp
  have same:
    hash-new-collision-event s out = hash-new-collision-event t out
  for out :: ('x  $\times$  ('f, 'a) protocol-channel-scheme) option
  using clean clean-t
  unfolding hash-new-collision-event-def hash-map-new-output-collision-def
  by (cases out) auto
  have wp-event m (hash-new-collision-event s) t =
    wp-event m (hash-new-collision-event t) t
  unfolding wp-event-def by (simp add: same)
  show ?thesis
    using False  $\langle$ wp-event m (hash-new-collision-event s) t =
      wp-event m (hash-new-collision-event t) t $\rangle$  by simp
qed

lemma hash-collision-budget-bind:
  fixes m :: ('x, ('f, 'a) protocol-channel-scheme) state-monad
    and k :: 'x  $\Rightarrow$  ('y, ('f, 'a) protocol-channel-scheme) state-monad
  assumes range-m: hash-range-budget n m
    and coll-m: hash-collision-budget n m
    and range-k:  $\bigwedge x. \text{hash-range-budget } n' (k\ x)$ 
    and coll-k:  $\bigwedge x. \text{hash-collision-budget } n' (k\ x)$ 
  shows hash-collision-budget (n + n') (m  $\ggg$  k)
  unfolding hash-collision-budget-def
proof (intro allI impI)
  fix s :: ('f, 'a) protocol-channel-scheme
  assume clean:  $\neg$  hash-map-output-collision s
  let ?c = card (hash-map-output-values s)
  let ?B1 = hash-collision-budget-value ?c n
  let ?B2 = hash-collision-budget-value (?c + n) n'
  let ?Em = hash-new-collision-event s

```

```

let ?S =
   $\lambda out. \text{case out of}$ 
    None  $\Rightarrow 0$ 
  | Some (x, s1)  $\Rightarrow$ 
    if hash-map-new-output-collision s s1 then 0
    else wp-event (k x) (hash-new-collision-event s1) s1
let ?R =
   $\lambda out. (if ?Em \text{ out then } 1 \text{ else } 0) + ?S \text{ out}$ 
have split:
  wp-event (m  $\ggg$  k) (hash-new-collision-event s) s  $\leq$  wp m ?R s
proof –
  have exact:
    wp-event (m  $\ggg$  k) (hash-new-collision-event s) s =
      wp m
      ( $\lambda out. \text{case out of}$ 
        None  $\Rightarrow 0$ 
      | Some (x, s1)  $\Rightarrow$ 
        wp-event (k x) (hash-new-collision-event s) s1) s
    unfolding wp-event-def hash-new-collision-event-def
    by (simp add: wpsimps)
  have wp m
    ( $\lambda out. \text{case out of}$ 
      None  $\Rightarrow 0$ 
    | Some (x, s1)  $\Rightarrow$ 
      wp-event (k x) (hash-new-collision-event s) s1) s
     $\leq$  wp m ?R s
  proof (rule wp-mono-on-support)
  fix out
  assume out  $\in$  set-dist (execute m s)
  show (case out of
    None  $\Rightarrow 0$ 
  | Some (x, s1)  $\Rightarrow$ 
    wp-event (k x) (hash-new-collision-event s) s1)  $\leq$  ?R out
  proof (cases out)
  case None
  then show ?thesis
    unfolding hash-new-collision-event-def by simp
  next
  case (Some xs)
  then obtain x s1 where xs: xs = (x, s1)
  by (cases xs) simp
  have step:
    wp-event (k x) (hash-new-collision-event s) s1  $\leq$ 
      (if hash-map-new-output-collision s s1 then 1
      else wp-event (k x) (hash-new-collision-event s1) s1)
    by (rule hash-new-collision-event-step-bound[OF clean])
  have R-eq:
    ?R out =
      (if hash-map-new-output-collision s s1 then 1

```

```

      else wp-event (k x) (hash-new-collision-event s1) s1)
    using Some xs unfolding hash-new-collision-event-def by simp
  have L-eq:
    (case out of
      None  $\Rightarrow$  0
    | Some (x, s1)  $\Rightarrow$ 
      wp-event (k x) (hash-new-collision-event s) s1) =
      wp-event (k x) (hash-new-collision-event s) s1
    using Some xs by simp
  show ?thesis
    using step unfolding L-eq R-eq .
qed
qed
then show ?thesis
  unfolding exact .
qed
have m-bound:
  wp-event m ?Em s  $\leq$  ?B1
  using coll-m clean unfolding hash-collision-budget-def by blast
have S-bound: wp m ?S s  $\leq$  ?B2
proof (rule wp-le-const-on-support)
  fix out
  assume out-support: out  $\in$  set-dist (execute m s)
  show ?S out  $\leq$  ?B2
  proof (cases out)
    case None
    then show ?thesis by simp
  next
    case (Some xs)
    then obtain x s1 where xs: xs = (x, s1)
      by (cases xs) simp
    show ?thesis
    proof (cases hash-map-new-output-collision s s1)
      case True
      then show ?thesis
        using Some xs by simp
    next
      case False
      then have clean-s1:  $\neg$  hash-map-output-collision s1
        using clean unfolding hash-map-new-output-collision-def by simp
      have k-bound:
        wp-event (k x) (hash-new-collision-event s1) s1  $\leq$ 
        hash-collision-budget-value
        (card (hash-map-output-values s1)) n'
        using coll-k[of x] clean-s1 unfolding hash-collision-budget-def
        by blast
      have m-out:
        Some (x, s1)  $\in$  set-dist (execute m s)
        using out-support Some xs by simp

```

```

have range:
  card (hash-map-output-values s1) ≤ ?c + n
  using range-m m-out unfolding hash-range-budget-def by blast
have hash-collision-budget-value
  (card (hash-map-output-values s1)) n' ≤ ?B2
  by (rule hash-collision-budget-value-mono-left[OF range])
then show ?thesis
  using Some xs False k-bound by simp
qed
qed
qed
have wp-event (m ≫ k) (hash-new-collision-event s) s
  ≤ wp m ?R s
  by (rule split)
also have ... =
  wp-event m ?Em s + wp m ?S s
  unfolding wp-event-def wp-def dist-expect-def
  by (simp add: sum.distrib algebra-simps hash-new-collision-event-def)
also have ... ≤ ?B1 + ?B2
  using m-bound S-bound by (intro add-mono)
also have ... ≤ hash-collision-budget-value ?c (n + n')
  by (rule hash-collision-budget-value-bind-le)
finally show
  wp-event (m ≫ k) (hash-new-collision-event s) s
  ≤ hash-collision-budget-value ?c (n + n') .
qed

```

```

lemma hash-collision-budget-bind-on-outcomes:
  fixes m :: ('x, ('f, 'a) protocol-channel-scheme) state-monad
  and k :: 'x ⇒ ('y, ('f, 'a) protocol-channel-scheme) state-monad
  assumes range-m: hash-range-budget n m
  and coll-m: hash-collision-budget n m
  and range-k:
    ∧ s x t. Some (x, t) ∈ set-dist (execute m s) ⇒
      hash-range-budget n' (k x)
  and coll-k:
    ∧ s x t. Some (x, t) ∈ set-dist (execute m s) ⇒
      hash-collision-budget n' (k x)
  shows hash-collision-budget (n + n') (m ≫ k)
  unfolding hash-collision-budget-def
proof (intro allI impI)
  fix s :: ('f, 'a) protocol-channel-scheme
  assume clean: ¬ hash-map-output-collision s
  let ?c = card (hash-map-output-values s)
  let ?B1 = hash-collision-budget-value ?c n
  let ?B2 = hash-collision-budget-value (?c + n) n'
  let ?Em = hash-new-collision-event s
  let ?S =
    λ out. case out of

```

```

  None  $\Rightarrow$  0
| Some (x, s1)  $\Rightarrow$ 
  if hash-map-new-output-collision s s1 then 0
  else wp-event (k x) (hash-new-collision-event s1) s1
let ?R =
   $\lambda$ out. (if ?Em out then 1 else 0) + ?S out
have split:
  wp-event (m  $\gg$  k) (hash-new-collision-event s) s  $\leq$  wp m ?R s
proof -
  have exact:
    wp-event (m  $\gg$  k) (hash-new-collision-event s) s =
      wp m
      ( $\lambda$ out. case out of
        None  $\Rightarrow$  0
        | Some (x, s1)  $\Rightarrow$ 
          wp-event (k x) (hash-new-collision-event s) s1) s
  unfolding wp-event-def hash-new-collision-event-def
  by (simp add: wpsimps)
have wp m
  ( $\lambda$ out. case out of
    None  $\Rightarrow$  0
    | Some (x, s1)  $\Rightarrow$ 
      wp-event (k x) (hash-new-collision-event s) s1) s
   $\leq$  wp m ?R s
proof (rule wp-mono-on-support)
  fix out
  assume out  $\in$  set-dist (execute m s)
  show (case out of
    None  $\Rightarrow$  0
    | Some (x, s1)  $\Rightarrow$ 
      wp-event (k x) (hash-new-collision-event s) s1)  $\leq$  ?R out
proof (cases out)
  case None
  then show ?thesis
    unfolding hash-new-collision-event-def by simp
next
  case (Some xs)
  then obtain x s1 where xs: xs = (x, s1)
  by (cases xs) simp
  have step:
    wp-event (k x) (hash-new-collision-event s) s1  $\leq$ 
      (if hash-map-new-output-collision s s1 then 1
       else wp-event (k x) (hash-new-collision-event s1) s1)
  by (rule hash-new-collision-event-step-bound[OF clean])
  have R-eq:
    ?R out =
      (if hash-map-new-output-collision s s1 then 1
       else wp-event (k x) (hash-new-collision-event s1) s1)
  using Some xs unfolding hash-new-collision-event-def by simp

```

```

have L-eq:
  (case out of
    None  $\Rightarrow 0$ 
  | Some (x, s1)  $\Rightarrow$ 
    wp-event (k x) (hash-new-collision-event s) s1) =
    wp-event (k x) (hash-new-collision-event s) s1
  using Some xs by simp
show ?thesis
  using step unfolding L-eq R-eq .
qed
qed
then show ?thesis
  unfolding exact .
qed
have m-bound:
  wp-event m ?Em s  $\leq ?B1$ 
  using coll-m clean unfolding hash-collision-budget-def by blast
have S-bound: wp m ?S s  $\leq ?B2$ 
proof (rule wp-le-const-on-support)
  fix out
  assume out-support: out  $\in$  set-dist (execute m s)
  show ?S out  $\leq ?B2$ 
  proof (cases out)
    case None
    then show ?thesis by simp
  next
    case (Some xs)
    then obtain x s1 where xs: xs = (x, s1)
    by (cases xs) simp
    have m-out:
      Some (x, s1)  $\in$  set-dist (execute m s)
      using out-support Some xs by simp
    show ?thesis
    proof (cases hash-map-new-output-collision s s1)
      case True
      then show ?thesis
        using Some xs by simp
    next
      case False
      then have clean-s1:  $\neg$  hash-map-output-collision s1
        using clean unfolding hash-map-new-output-collision-def by simp
      have k-bound:
        wp-event (k x) (hash-new-collision-event s1) s1  $\leq$ 
          hash-collision-budget-value
            (card (hash-map-output-values s1)) n'
        using coll-k[OF m-out] clean-s1
        unfolding hash-collision-budget-def by blast
      have range:
        card (hash-map-output-values s1)  $\leq ?c + n$ 

```

```

    using range-m m-out unfolding hash-range-budget-def by blast
  have hash-collision-budget-value
    (card (hash-map-output-values s1))  $n' \leq ?B2$ 
    by (rule hash-collision-budget-value-mono-left[OF range])
  then show ?thesis
    using Some xs False k-bound by simp
qed
qed
qed
have wp-event (m  $\ggg$  k) (hash-new-collision-event s) s
   $\leq$  wp m ?R s
  by (rule split)
also have ... =
  wp-event m ?Em s + wp m ?S s
  unfolding wp-event-def wp-def dist-expect-def
  by (simp add: sum.distrib algebra-simps hash-new-collision-event-def)
also have ...  $\leq$  ?B1 + ?B2
  using m-bound S-bound by (intro add-mono)
also have ...  $\leq$  hash-collision-budget-value ?c (n + n')
  by (rule hash-collision-budget-value-bind-le)
finally show
  wp-event (m  $\ggg$  k) (hash-new-collision-event s) s
   $\leq$  hash-collision-budget-value ?c (n + n') .
qed

lemma wp-hash-new-output-collision-bound:
  fixes x :: 'f protocol-hash-input
  and s :: ('f, 'a) protocol-channel-scheme
  assumes clean:  $\neg$  hash-map-output-collision s
  shows
    wp-event (hash x)
      (hash-new-collision-event s) s
       $\leq$  nnreal (card (hash-map-output-values s)) / nnreal size
  proof (cases fmllookup (HashMap s) x)
  case None
  let ?P =
     $\lambda y.$  hash-map-new-output-collision s
      (s[HashMap := fmutpd x y (HashMap s)])
  let ?Q =  $\lambda y.$  y  $\in$  hash-map-output-values s
  have mono:
    dist-expect (hash-dist x s) ( $\lambda y.$  if ?P y then 1 else 0)  $\leq$ 
    dist-expect (hash-dist x s) ( $\lambda y.$  if ?Q y then 1 else 0)
  unfolding dist-expect-def
  proof (intro sum-mono)
  fix y
  assume y  $\in$  dom (dist (hash-dist x s))
  show the (dist (hash-dist x s) y) * (if ?P y then 1 else 0)
     $\leq$  the (dist (hash-dist x s) y) * (if ?Q y then 1 else 0)
  proof (cases ?P y)

```



```

    case True
    then have ?Q y
      using hash-map-output-collision-after-fresh-updateD
        [OF clean None]
      unfolding hash-map-new-output-collision-def by simp
    then show ?thesis
      using True by simp
  next
    case False
    then show ?thesis by simp
qed
qed
show ?thesis
proof -
  have wp-event (hash x)
    (hash-new-collision-event s) s =
    dist-expect (hash-dist x s) ( $\lambda y. \text{if } ?P y \text{ then } 1 \text{ else } 0$ )
  unfolding wp-event-def hash-new-collision-event-def
  by (simp add: wp-hash)
  also have ...  $\leq$ 
    dist-expect (hash-dist x s) ( $\lambda y. \text{if } ?Q y \text{ then } 1 \text{ else } 0$ )
  by (rule mono)
  also have ... =
    nnreal (card (hash-map-output-values s)) / nnreal size
  by (rule dist-expect-hash-dist-fresh-output-values[OF None])
  finally show ?thesis .
qed
next
  case (Some y)
  have no-new:
     $\neg$  hash-map-new-output-collision s
    (s[HashMap := fmupd x y (HashMap s)])
  using no-hash-map-output-collision-after-known-update[OF clean Some]
  unfolding hash-map-new-output-collision-def by simp
  show ?thesis
    unfolding wp-event-def hash-new-collision-event-def
    by (simp add: wp-hash Some hash-dist-def option-default-dist-def no-new)
qed

lemma hash-range-budget-return:
  hash-range-budget 0 (return x)
  unfolding hash-range-budget-def by simp

lemma hash-collision-budget-return:
  hash-collision-budget 0 (return x)
  unfolding hash-collision-budget-def hash-new-collision-event-def
    hash-collision-budget-value-def hash-map-new-output-collision-def
  by (simp add: wp-event-def wpsimps)

```

```

lemma hash-range-budget-hash:
  fixes x :: 'f protocol-hash-input
  shows
    hash-range-budget 1
    (hash x :: ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding hash-range-budget-def
proof (intro allI impI)
  fix s y t
  assume outcome:
    Some (y, t) ∈
      set-dist (execute
        (hash x :: ('f, ('f, 'a) protocol-channel-scheme) state-monad) s)
  have t-eq: t = s(HashMap := fmupd x y (HashMap s))
proof –
  from outcome obtain a s1 where
    apply-step:
      Some (a, s1) ∈ set-dist (execute (apply-hash x) s)
  and rest:
    Some (y, t) ∈
      set-dist (execute
        (modify-HashMap x a ≫ (λ-. return a)) s1)
proof –
  have Some (y, t) ∈
    set-dist (execute
      (apply-hash x ≫
        (λa. modify-HashMap x a ≫ (λ-. return a)))) s)
  using outcome unfolding hash-def .
  then show ?thesis
    by (rule set-dist-bindE) (rule that)
qed
have apply-img:
  Some (a, s1) ∈ (λa. Some (a, s)) ‘ set-dist (hash-dist x s)
  using apply-step
  unfolding apply-hash-def lift.rep-eq lift-dist-def
  by (simp add: set-dist-dist-map)
then have s1-eq: s1 = s
  by auto
from rest obtain u s2 where
  mod:
    Some (u, s2) ∈ set-dist (execute (modify-HashMap x a) s1)
  and ret:
    Some (y, t) ∈ set-dist (execute (return a) s2)
  by (elim set-dist-bindE)
have s2-eq: s2 = s1(HashMap := fmupd x a (HashMap s1))
  using mod unfolding modify-HashMap-def modify-def
  by (auto elim!: set-dist-bindE)
have y-eq: y = a and t-eq-s2: t = s2
  using ret by auto
show ?thesis

```

```

    using s1-eq s2-eq y-eq t-eq-s2 by simp
  qed
show card (hash-map-output-values t)
  ≤ card (hash-map-output-values s) + 1
  unfolding t-eq
  using card-hash-map-output-values-after-update-le[of x y s]
  by simp
qed

lemma hash-collision-budget-hash:
  fixes x :: 'f protocol-hash-input
  shows
    hash-collision-budget 1
    (hash x :: ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding hash-collision-budget-def
proof (intro allI impI)
  fix s :: ('f, 'a) protocol-channel-scheme
  assume clean: ¬ hash-map-output-collision s
  have one-step:
    wp-event (hash x) (hash-new-collision-event s) s ≤
      nnreal (card (hash-map-output-values s)) / nnreal size
  by (rule wp-hash-new-output-collision-bound[OF clean])
  also have ... ≤
    hash-collision-budget-value (card (hash-map-output-values s)) 1
  unfolding hash-collision-budget-value-def
  apply (subst nn2real-le-iff[symmetric])
  by (simp add: divide-right-mono)
  finally show
    wp-event (hash x) (hash-new-collision-event s) s ≤
      hash-collision-budget-value (card (hash-map-output-values s)) 1 .
qed

lemma hash-range-budget-get:
  hash-range-budget 0
  (get :: (('f, 'a) protocol-channel-scheme,
    ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding hash-range-budget-def by simp

lemma hash-collision-budget-get:
  hash-collision-budget 0
  (get :: (('f, 'a) protocol-channel-scheme,
    ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding hash-collision-budget-def hash-new-collision-event-def
    hash-map-new-output-collision-def hash-collision-budget-value-def
  by (simp add: wp-event-def wpsimps)

lemma hash-range-budget-assert:
  hash-range-budget 0
  (assert b :: (unit, ('f, 'a) protocol-channel-scheme) state-monad)

```

```

unfolding hash-range-budget-def assert-def
by (auto simp: throw-no-outcome)

lemma hash-collision-budget-assert:
  hash-collision-budget 0
  (assert b :: (unit, ('f, 'a) protocol-channel-scheme) state-monad)
unfolding hash-collision-budget-def hash-new-collision-event-def
  hash-map-new-output-collision-def hash-collision-budget-value-def
  assert-def
by (simp add: wp-event-def wpsimps)

lemma hash-range-budget-send:
  hash-range-budget 0 (send x)
unfolding hash-range-budget-def send-def protocol-send-def modify-def
by (auto simp: hash-map-output-values-def elim!: set-dist-bindE)

lemma hash-collision-budget-send:
  hash-collision-budget 0 (send x)
unfolding hash-collision-budget-def hash-new-collision-event-def
  hash-map-new-output-collision-def hash-collision-budget-value-def
  hash-map-output-collision-def
  send-def protocol-send-def
by (simp add: wp-event-def wpsimps)

lemma hash-range-budget-read:
  hash-range-budget 0 (read :: ('f, 'f, 'a) protocol-c-monad)
unfolding hash-range-budget-def
proof (intro allI impI)
  fix s x t
  assume out: Some (x, t) ∈
    set-dist (execute (read :: ('f, 'f, 'a) protocol-c-monad) s)
  obtain y ys where tr: PTranscript s = y # ys
  proof –
    have PTranscript s ≠ []
    using out unfolding read-def protocol-read-def assert-def
    by (cases PTranscript s)
    (auto simp: throw-no-outcome elim!: set-dist-bindE)
    then show ?thesis
    by (cases PTranscript s) (auto intro: that)
  qed
  have t-eq:
    t = s(PState := concat (PState s) y, PTranscript := ys)
    using read-nonempty-outcome[OF out tr] by simp
  show card (hash-map-output-values t)
    ≤ card (hash-map-output-values s) + 0
    unfolding t-eq hash-map-output-values-def by simp
qed

lemma hash-collision-budget-read:

```

hash-collision-budget 0
(read :: ('f, 'f, 'a) protocol-c-monad)
unfolding *hash-collision-budget-def hash-new-collision-event-def*
hash-map-new-output-collision-def hash-collision-budget-value-def
hash-map-output-collision-def
read-def protocol-read-def assert-def
by (*simp add: wp-event-def wpsimps*)

lemma *hash-range-budget-modify-preserves-hash-map:*
assumes $\bigwedge s. \text{HashMap } (f\ s) = \text{HashMap } s$
shows *hash-range-budget 0 (modify f)*
using *assms*
unfolding *hash-range-budget-def modify-def hash-map-output-values-def*
by (*auto elim!: set-dist-bindE*)

lemma *hash-collision-budget-modify-preserves-hash-map:*
assumes $\bigwedge s. \text{HashMap } (f\ s) = \text{HashMap } s$
shows *hash-collision-budget 0 (modify f)*
using *assms*
unfolding *hash-collision-budget-def hash-new-collision-event-def*
hash-map-new-output-collision-def hash-collision-budget-value-def
hash-map-output-collision-def modify-def
by (*simp add: wp-event-def wpsimps*)

lemma *hash-map-preserving-get:*
hash-map-preserving
(get :: (('f, 'a) protocol-channel-scheme,
('f, 'a) protocol-channel-scheme) state-monad)
unfolding *hash-map-preserving-def* **by** *simp*

lemma *hash-map-preserving-assert:*
hash-map-preserving
(assert b :: (unit, ('f, 'a) protocol-channel-scheme) state-monad)
unfolding *hash-map-preserving-def assert-def*
by (*auto simp: throw-no-outcome elim!: set-dist-bindE*)

lemma *hash-map-preserving-send:*
hash-map-preserving (send x)
unfolding *hash-map-preserving-def send-def protocol-send-def modify-def*
by (*auto elim!: set-dist-bindE*)

lemma *hash-map-preserving-read:*
hash-map-preserving (read :: ('f, 'f, 'a) protocol-c-monad)
unfolding *hash-map-preserving-def*
proof (*intro allI impI*)
fix *s x t*
assume *out:*
Some (x, t) ∈
set-dist (execute (read :: ('f, 'f, 'a) protocol-c-monad) s)

```

obtain  $y$   $ys$  where  $tr$ :  $PTranscript\ s = y \# ys$ 
proof –
  have  $PTranscript\ s \neq []$ 
    using  $out$  unfolding  $read-def\ protocol-read-def\ assert-def$ 
    by ( $cases\ PTranscript\ s$ )
      ( $auto\ simp$ :  $throw-no-outcome\ elim!$ :  $set-dist-bindE$ )
  then show  $?thesis$ 
    by ( $cases\ PTranscript\ s$ ) ( $auto\ intro$ :  $that$ )
qed
have  $t-eq$ :
   $t = s(PState := concat\ (PState\ s)\ y,\ PTranscript := ys)$ 
  using  $read-nonempty-outcome[OF\ out\ tr]$  by  $simp$ 
show  $HashMap\ t = HashMap\ s$ 
  unfolding  $t-eq$  by  $simp$ 
qed

```

```

lemma  $hash-map-preserving-modify$ :
  assumes  $\bigwedge s. HashMap\ (f\ s) = HashMap\ s$ 
  shows  $hash-map-preserving\ (modify\ f)$ 
  using  $assms$  unfolding  $hash-map-preserving-def\ modify-def$ 
  by ( $auto\ elim!$ :  $set-dist-bindE$ )

```

```

lemma  $hash-target-program-get$ :
   $hash-target-program\ B\ 0$ 
  ( $get :: (('f, 'a)\ protocol-channel-scheme,$ 
    ( $'f, 'a)\ protocol-channel-scheme)\ state-monad$ )
  by ( $rule\ hash-map-preserving-imp-hash-target-program-zero$ )
    ( $rule\ hash-map-preserving-get$ )

```

```

lemma  $hash-target-program-assert$ :
   $hash-target-program\ B\ 0$ 
  ( $assert\ b :: (unit, ('f, 'a)\ protocol-channel-scheme)\ state-monad$ )
  by ( $rule\ hash-map-preserving-imp-hash-target-program-zero$ )
    ( $rule\ hash-map-preserving-assert$ )

```

```

lemma  $hash-target-program-send$ :
   $hash-target-program\ B\ 0\ (send\ x)$ 
  by ( $rule\ hash-map-preserving-imp-hash-target-program-zero$ )
    ( $rule\ hash-map-preserving-send$ )

```

```

lemma  $hash-target-program-read$ :
   $hash-target-program\ B\ 0\ (read :: ('f, 'f, 'a)\ protocol-c-monad)$ 
  by ( $rule\ hash-map-preserving-imp-hash-target-program-zero$ )
    ( $rule\ hash-map-preserving-read$ )

```

```

lemma  $hash-target-program-ntimes-read-bind$ :
  fixes  $k :: 'f\ list \Rightarrow$ 
    ( $'y, ('f, 'a)\ protocol-channel-scheme)\ state-monad$ 
  assumes  $k$ :

```

```

 $\bigwedge xs. \text{length } xs = n \implies \text{hash-target-program } B \ b \ (k \ xs)$ 
shows
   $\text{hash-target-program } B \ b$ 
   $((\text{ntimes read } n ::$ 
     $(\text{'f list}, (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ state-monad}) \ggg k)$ 
proof –
  have reads:
     $\text{hash-target-program } B \ (n * 0)$ 
     $(\text{ntimes read } n ::$ 
       $(\text{'f list}, (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ state-monad})$ 
    by (rule hash-target-program-ntimes[OF hash-target-program-read])
  have  $\text{hash-target-program } B \ (n * 0 + b)$ 
     $((\text{ntimes read } n ::$ 
       $(\text{'f list}, (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ state-monad}) \ggg k)$ 
proof (rule hash-target-program-bind-on-outcomes[OF reads])
  fix  $s \ xs \ t$ 
  assume out:
     $\text{Some } (xs, t) \in$ 
     $\text{set-dist } (\text{execute}$ 
       $(\text{ntimes read } n ::$ 
         $(\text{'f list}, (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ state-monad}) \ s)$ 
    show  $\text{hash-target-program } B \ b \ (k \ xs)$ 
    by (rule k) (use ntimes-read-any-outcome[OF out] in simp)
  qed
  then show ?thesis by simp
qed

```

```

lemma hash-target-program-modify:
  assumes  $\bigwedge s. \text{HashMap } (f \ s) = \text{HashMap } s$ 
  shows  $\text{hash-target-program } B \ 0 \ (\text{modify } f)$ 
  by (rule hash-map-preserving-imp-hash-target-program-zero)
  (rule hash-map-preserving-modify[OF assms])

```

```

lemma hash-target-program-receive-random-field-element:
   $\text{hash-target-program } B \ 1$ 
   $(\text{receive-random-field-element} :: (\text{'f}, \text{'f}, \text{'a}) \text{ protocol-c-monad})$ 
proof –
  have tail:
     $\text{hash-target-program } B \ (1 + 0)$ 
     $(\text{hash } (\text{FiatShamirChallenge } (PState \ s)) \ggg \text{return} ::$ 
       $(\text{'f}, (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ state-monad})$ 
    for  $s :: (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}$ 
    by (rule hash-target-program-bind)
    (rule hash-target-program-hash, rule hash-target-program-return)
  have  $\text{hash-target-program } B \ (0 + (1 + 0))$ 
     $(\text{get} \ggg$ 
       $(\lambda s :: (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}.$ 
         $\text{hash } (\text{FiatShamirChallenge } (PState \ s)) \ggg \text{return}))$ 
    by (rule hash-target-program-bind[OF hash-target-program-get tail])

```

```

then show ?thesis
  unfolding receive-random-field-element-def
    protocol-receive-random-field-element-def
    protocol-receive-tagged-random-field-element-def
  by simp
qed

lemma hash-target-program-receive-tagged-random-field-element:
  hash-target-program B 1
  (receive-tagged-random-field-element tag ::
    ('f, 'f, 'a) protocol-c-monad)
proof -
  have tail:
    hash-target-program B (1 + 0)
    (hash (tag (PState s))  $\gg=$  return ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  for s :: ('f, 'a) protocol-channel-scheme
  by (rule hash-target-program-bind)
    (rule hash-target-program-hash, rule hash-target-program-return)
  have hash-target-program B (0 + (1 + 0))
    (get  $\gg=$ 
      ( $\lambda$ s :: ('f, 'a) protocol-channel-scheme.
        hash (tag (PState s))  $\gg=$  return))
  by (rule hash-target-program-bind[OF hash-target-program-get tail])
  then show ?thesis
    unfolding receive-tagged-random-field-element-def
      protocol-receive-tagged-random-field-element-def
    by simp
qed

lemma hash-target-program-receive-counted-tagged-random-field-element:
  assumes bump:  $\bigwedge s. \text{HashMap } (\text{bump } s) = \text{HashMap } s$ 
  shows hash-target-program B 1
  (protocol-receive-counted-tagged-random-field-element counter bump tag ::
    ('f, 'f, 'a) protocol-c-monad)
proof -
  have modify-return:
    hash-target-program B (0 + 0)
    (modify bump  $\gg=$  ( $\lambda$ -. return r) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  for r :: 'f
  by (rule hash-target-program-bind)
    (rule hash-target-program-modify[OF bump],
      rule hash-target-program-return)
  have hash-tail:
    hash-target-program B (1 + (0 + 0))
    (hash (tag (counter s) (PState s))  $\gg=$ 
      ( $\lambda$ r. modify bump  $\gg=$  ( $\lambda$ -. return r)) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)

```



```

for  $s :: ('f, 'a) \text{ protocol-channel-scheme}$ 
by (rule hash-target-program-bind)
    (rule hash-target-program-hash, rule modify-return)
have hash-target-program B (0 + (1 + (0 + 0)))
    (get  $\gg$ 
      ( $\lambda s :: ('f, 'a) \text{ protocol-channel-scheme.}$ 
        hash (tag (counter s) (PState s))  $\gg$ 
        ( $\lambda r. \text{ modify bump } \gg (\lambda -. \text{ return } r)$ )))
    by (rule hash-target-program-bind[OF hash-target-program-get hash-tail])
then show ?thesis
    unfolding protocol-receive-counted-tagged-random-field-element-def
    by simp
qed

```

```

lemma hash-target-program-receive-alpha-challenge:
  hash-target-program B 1
  (receive-alpha-challenge  $:: ('f, 'f, 'a) \text{ protocol-c-monad}$ )
unfolding receive-alpha-challenge-def
by (rule hash-target-program-receive-counted-tagged-random-field-element)
    simp

```

```

lemma hash-target-program-receive-query-index-challenge:
  hash-target-program B 1
  (receive-query-index-challenge  $:: ('f, 'f, 'a) \text{ protocol-c-monad}$ )
unfolding receive-query-index-challenge-def
by (rule hash-target-program-receive-counted-tagged-random-field-element)
    simp

```

```

lemma hash-target-program-receive-trace-fri-challenge:
  hash-target-program B 1
  (receive-trace-fri-challenge  $:: ('f, 'f, 'a) \text{ protocol-c-monad}$ )
unfolding receive-trace-fri-challenge-def
by (rule hash-target-program-receive-counted-tagged-random-field-element)
    simp

```

```

lemma hash-target-program-receive-composition-fri-challenge:
  hash-target-program B 1
  (receive-composition-fri-challenge  $:: ('f, 'f, 'a) \text{ protocol-c-monad}$ )
unfolding receive-composition-fri-challenge-def
by (rule hash-target-program-receive-counted-tagged-random-field-element)
    simp

```

```

lemma hash-range-budget-receive-random-field-element:
  hash-range-budget 1
  (receive-random-field-element  $:: ('f, 'f, 'a) \text{ protocol-c-monad}$ )
proof –
  have get-step:
    hash-range-budget 0
    (get  $:: (('f, 'a) \text{ protocol-channel-scheme},$ 

```

```

    ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-get)
have hash-then-return:
  hash-range-budget (1 + 0)
  (hash (FiatShamirChallenge (PState s))  $\gg$  return ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  for s :: ('f, 'a) protocol-channel-scheme
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-hash, rule hash-range-budget-return)
have hash-range-budget (0 + (1 + 0))
  (get  $\gg$ 
    ( $\lambda s :: ('f, 'a) \text{ protocol-channel-scheme.}$ 
      hash (FiatShamirChallenge (PState s))  $\gg$  return))
  by (rule hash-range-budget-bind[OF get-step hash-then-return])
then show ?thesis
  unfolding receive-random-field-element-def
    protocol-receive-random-field-element-def
    protocol-receive-tagged-random-field-element-def
  by simp
qed

lemma hash-range-budget-receive-tagged-random-field-element:
  hash-range-budget 1
  (receive-tagged-random-field-element tag :: ('f, 'f, 'a) protocol-c-monad)
proof -
  have get-step:
    hash-range-budget 0
    (get :: (('f, 'a) protocol-channel-scheme,
      ('f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-range-budget-get)
  have hash-then-return:
    hash-range-budget (1 + 0)
    (hash (tag (PState s))  $\gg$  return ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    for s :: ('f, 'a) protocol-channel-scheme
    by (rule hash-range-budget-bind)
      (rule hash-range-budget-hash, rule hash-range-budget-return)
  have hash-range-budget (0 + (1 + 0))
    (get  $\gg$ 
      ( $\lambda s :: ('f, 'a) \text{ protocol-channel-scheme.}$ 
        hash (tag (PState s))  $\gg$  return))
    by (rule hash-range-budget-bind[OF get-step hash-then-return])
  then show ?thesis
    unfolding receive-tagged-random-field-element-def
      protocol-receive-tagged-random-field-element-def
    by simp
qed

lemma hash-range-budget-receive-counted-tagged-random-field-element:

```

```

assumes bump:  $\bigwedge s. \text{HashMap } (\text{bump } s) = \text{HashMap } s$ 
shows hash-range-budget 1
  (protocol-receive-counted-tagged-random-field-element counter bump tag ::
    ('f, 'f, 'a) protocol-c-monad)
proof –
  have get-step:
    hash-range-budget 0
    (get :: (('f, 'a) protocol-channel-scheme,
      ('f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-range-budget-get)
  have modify-return:
    hash-range-budget (0 + 0)
    (modify bump  $\gg=$  ( $\lambda$ -. return r) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    for r :: 'f
    by (rule hash-range-budget-bind)
    (rule hash-range-budget-modify-preserves-hash-map[OF bump],
      rule hash-range-budget-return)
  have hash-then-tail:
    hash-range-budget (1 + (0 + 0))
    (hash (tag (counter s) (PState s))  $\gg=$ 
      ( $\lambda$ r. modify bump  $\gg=$  ( $\lambda$ -. return r)) ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    for s :: ('f, 'a) protocol-channel-scheme
    by (rule hash-range-budget-bind)
    (rule hash-range-budget-hash, rule modify-return)
  have hash-range-budget (0 + (1 + (0 + 0)))
    (get  $\gg=$ 
      ( $\lambda$ s :: ('f, 'a) protocol-channel-scheme.
        hash (tag (counter s) (PState s))  $\gg=$ 
          ( $\lambda$ r. modify bump  $\gg=$  ( $\lambda$ -. return r))))
    by (rule hash-range-budget-bind[OF get-step hash-then-tail])
  then show ?thesis
    unfolding protocol-receive-counted-tagged-random-field-element-def
    by simp
qed

```

```

lemma hash-range-budget-receive-alpha-challenge:
  hash-range-budget 1
  (receive-alpha-challenge :: ('f, 'f, 'a) protocol-c-monad)
  unfolding receive-alpha-challenge-def
  by (rule hash-range-budget-receive-counted-tagged-random-field-element) simp

```

```

lemma hash-range-budget-receive-query-index-challenge:
  hash-range-budget 1
  (receive-query-index-challenge :: ('f, 'f, 'a) protocol-c-monad)
  unfolding receive-query-index-challenge-def
  by (rule hash-range-budget-receive-counted-tagged-random-field-element) simp

```

lemma *hash-range-budget-receive-trace-fri-challenge*:
hash-range-budget 1
(receive-trace-fri-challenge :: ('f, 'f, 'a) protocol-c-monad)
unfolding *receive-trace-fri-challenge-def*
by (rule *hash-range-budget-receive-counted-tagged-random-field-element*) *simp*

lemma *hash-range-budget-receive-composition-fri-challenge*:
hash-range-budget 1
(receive-composition-fri-challenge :: ('f, 'f, 'a) protocol-c-monad)
unfolding *receive-composition-fri-challenge-def*
by (rule *hash-range-budget-receive-counted-tagged-random-field-element*) *simp*

lemma *hash-collision-budget-receive-random-field-element*:
hash-collision-budget 1
(receive-random-field-element :: ('f, 'f, 'a) protocol-c-monad)
proof –
have *get-range*:
hash-range-budget 0
(get :: (('f, 'a) protocol-channel-scheme,
'f, 'a) protocol-channel-scheme) state-monad)
by (rule *hash-range-budget-get*)
have *get-coll*:
hash-collision-budget 0
(get :: (('f, 'a) protocol-channel-scheme,
'f, 'a) protocol-channel-scheme) state-monad)
by (rule *hash-collision-budget-get*)
have *hash-then-return-range*:
hash-range-budget (1 + 0)
(hash (FiatShamirChallenge (PState s)) $\gg=$ return ::
'f, ('f, 'a) protocol-channel-scheme) state-monad)
for *s :: ('f, 'a) protocol-channel-scheme*
by (rule *hash-range-budget-bind*)
(rule hash-range-budget-hash, rule hash-range-budget-return)
have *hash-then-return-coll*:
hash-collision-budget (1 + 0)
(hash (FiatShamirChallenge (PState s)) $\gg=$ return ::
'f, ('f, 'a) protocol-channel-scheme) state-monad)
for *s :: ('f, 'a) protocol-channel-scheme*
by (rule *hash-collision-budget-bind*)
(rule hash-range-budget-hash, rule hash-collision-budget-hash,
rule hash-range-budget-return, rule hash-collision-budget-return)
have *hash-collision-budget (0 + (1 + 0))*
(get $\gg=$
($\lambda s :: ('f, 'a) protocol-channel-scheme.$
hash (FiatShamirChallenge (PState s)) $\gg=$ return))
by (rule *hash-collision-budget-bind*)
[OF get-range get-coll hash-then-return-range
hash-then-return-coll])
then show *?thesis*

```

unfolding receive-random-field-element-def
  protocol-receive-random-field-element-def
  protocol-receive-tagged-random-field-element-def
by simp
qed

lemma hash-collision-budget-receive-tagged-random-field-element:
  hash-collision-budget 1
  (receive-tagged-random-field-element tag :: ('f, 'a) protocol-c-monad)
proof –
  have get-range:
    hash-range-budget 0
    (get :: (('f, 'a) protocol-channel-scheme,
      ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-get)
  have get-coll:
    hash-collision-budget 0
    (get :: (('f, 'a) protocol-channel-scheme,
      ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-collision-budget-get)
  have hash-then-return-range:
    hash-range-budget (1 + 0)
    (hash (tag (PState s))  $\gg$  return ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  for s :: ('f, 'a) protocol-channel-scheme
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-hash, rule hash-range-budget-return)
  have hash-then-return-coll:
    hash-collision-budget (1 + 0)
    (hash (tag (PState s))  $\gg$  return ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  for s :: ('f, 'a) protocol-channel-scheme
  by (rule hash-collision-budget-bind)
    (rule hash-range-budget-hash, rule hash-collision-budget-hash,
      rule hash-range-budget-return, rule hash-collision-budget-return)
  have hash-collision-budget (0 + (1 + 0))
    (get  $\gg$ 
      ( $\lambda$ s :: ('f, 'a) protocol-channel-scheme.
        hash (tag (PState s))  $\gg$  return))
  by (rule hash-collision-budget-bind
    [OF get-range get-coll hash-then-return-range
      hash-then-return-coll])
  then show ?thesis
  unfolding receive-tagged-random-field-element-def
  protocol-receive-tagged-random-field-element-def
  by simp
qed

lemma hash-collision-budget-receive-counted-tagged-random-field-element:

```

```

assumes bump:  $\bigwedge s. \text{HashMap } (\text{bump } s) = \text{HashMap } s$ 
shows hash-collision-budget 1
  (protocol-receive-counted-tagged-random-field-element counter bump tag ::
    ('f, 'f, 'a) protocol-c-monad)
proof –
  have get-range:
    hash-range-budget 0
    (get :: (('f, 'a) protocol-channel-scheme,
      ('f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-range-budget-get)
  have get-coll:
    hash-collision-budget 0
    (get :: (('f, 'a) protocol-channel-scheme,
      ('f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-collision-budget-get)
  have modify-return-range:
    hash-range-budget (0 + 0)
    (modify bump  $\gg=$  ( $\lambda$ -. return r) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    for r :: 'f
    by (rule hash-range-budget-bind)
    (rule hash-range-budget-modify-preserves-hash-map[OF bump],
      rule hash-range-budget-return)
  have modify-return-coll:
    hash-collision-budget (0 + 0)
    (modify bump  $\gg=$  ( $\lambda$ -. return r) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    for r :: 'f
    by (rule hash-collision-budget-bind)
    (rule hash-range-budget-modify-preserves-hash-map[OF bump],
      rule hash-collision-budget-modify-preserves-hash-map[OF bump],
      rule hash-range-budget-return, rule hash-collision-budget-return)
  have hash-then-tail-range:
    hash-range-budget (1 + (0 + 0))
    (hash (tag (counter s) (PState s))  $\gg=$ 
      ( $\lambda$ r. modify bump  $\gg=$  ( $\lambda$ -. return r)) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    for s :: ('f, 'a) protocol-channel-scheme
    by (rule hash-range-budget-bind)
    (rule hash-range-budget-hash, rule modify-return-range)
  have hash-then-tail-coll:
    hash-collision-budget (1 + (0 + 0))
    (hash (tag (counter s) (PState s))  $\gg=$ 
      ( $\lambda$ r. modify bump  $\gg=$  ( $\lambda$ -. return r)) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    for s :: ('f, 'a) protocol-channel-scheme
    by (rule hash-collision-budget-bind)
    (rule hash-range-budget-hash, rule hash-collision-budget-hash,
      rule modify-return-range, rule modify-return-coll)

```

```

have hash-collision-budget (0 + (1 + (0 + 0)))
  (get  $\gg$ 
    ( $\lambda s :: ('f, 'a)$  protocol-channel-scheme.
      hash (tag (counter s) (PState s))  $\gg$ 
      ( $\lambda r.$  modify bump  $\gg$  ( $\lambda -. return r$ ))))
  by (rule hash-collision-budget-bind
    [OF get-range get-coll hash-then-tail-range hash-then-tail-coll])
then show ?thesis
  unfolding protocol-receive-counted-tagged-random-field-element-def
  by simp
qed

lemma hash-collision-budget-receive-alpha-challenge:
  hash-collision-budget 1
  (receive-alpha-challenge :: ('f, 'f, 'a) protocol-c-monad)
  unfolding receive-alpha-challenge-def
  by (rule hash-collision-budget-receive-counted-tagged-random-field-element) simp

lemma hash-collision-budget-receive-query-index-challenge:
  hash-collision-budget 1
  (receive-query-index-challenge :: ('f, 'f, 'a) protocol-c-monad)
  unfolding receive-query-index-challenge-def
  by (rule hash-collision-budget-receive-counted-tagged-random-field-element) simp

lemma hash-collision-budget-receive-trace-fri-challenge:
  hash-collision-budget 1
  (receive-trace-fri-challenge :: ('f, 'f, 'a) protocol-c-monad)
  unfolding receive-trace-fri-challenge-def
  by (rule hash-collision-budget-receive-counted-tagged-random-field-element) simp

lemma hash-collision-budget-receive-composition-fri-challenge:
  hash-collision-budget 1
  (receive-composition-fri-challenge :: ('f, 'f, 'a) protocol-c-monad)
  unfolding receive-composition-fri-challenge-def
  by (rule hash-collision-budget-receive-counted-tagged-random-field-element) simp

lemma hash-range-budget-ntimes:
  fixes m :: ('x, ('f, 'a) protocol-channel-scheme) state-monad
  assumes m: hash-range-budget n m
  shows hash-range-budget (k * n) (ntimes m k)
  using m
proof (induction k)
  case 0
  then show ?case
    by (simp add: hash-range-budget-return)
next
  case (Suc k)
  have tail: hash-range-budget (k * n) (ntimes m k)
    using Suc.IH Suc.premis by simp

```

```

have cont:
  hash-range-budget ( $k * n + 0$ )
    ( $ntimes\ m\ k \gg (\lambda xs. return\ (x \# xs))$ ) for  $x$ 
  by (rule hash-range-budget-bind[OF tail hash-range-budget-return])
have hash-range-budget ( $n + (k * n + 0)$ )
  ( $m \gg (\lambda x. ntimes\ m\ k \gg (\lambda xs. return\ (x \# xs)))$ )
  by (rule hash-range-budget-bind[OF Suc.prems cont])
then show ?case
  by simp
qed

lemma hash-collision-budget-ntimes:
  fixes  $m :: ('x, ('f, 'a)\ protocol-channel-scheme)\ state-monad$ 
  assumes range-m: hash-range-budget  $n\ m$ 
  and coll-m: hash-collision-budget  $n\ m$ 
  shows hash-collision-budget ( $k * n$ ) ( $ntimes\ m\ k$ )
  using range-m coll-m
proof (induction k)
  case 0
  then show ?case
    by (simp add: hash-collision-budget-return)
next
  case (Suc k)
  have tail-range: hash-range-budget ( $k * n$ ) ( $ntimes\ m\ k$ )
    by (rule hash-range-budget-ntimes[OF Suc.prems(1)])
  have tail-coll: hash-collision-budget ( $k * n$ ) ( $ntimes\ m\ k$ )
    using Suc.IH Suc.prems by simp
  have cont-range:
    hash-range-budget ( $k * n + 0$ )
      ( $ntimes\ m\ k \gg (\lambda xs. return\ (x \# xs))$ ) for  $x$ 
    by (rule hash-range-budget-bind[OF tail-range hash-range-budget-return])
  have cont-coll:
    hash-collision-budget ( $k * n + 0$ )
      ( $ntimes\ m\ k \gg (\lambda xs. return\ (x \# xs))$ ) for  $x$ 
    by (rule hash-collision-budget-bind
      [OF tail-range tail-coll hash-range-budget-return
        hash-collision-budget-return])
  have hash-collision-budget ( $n + (k * n + 0)$ )
    ( $m \gg (\lambda x. ntimes\ m\ k \gg (\lambda xs. return\ (x \# xs)))$ )
    by (rule hash-collision-budget-bind
      [OF Suc.prems(1) Suc.prems(2) cont-range cont-coll])
  then show ?case
    by simp
qed

lemma hash-range-budget-ntimes-read-bind:
  fixes  $k :: 'f\ list \Rightarrow$ 
     $('y, ('f, 'a)\ protocol-channel-scheme)\ state-monad$ 
  assumes  $k: \bigwedge xs. length\ xs = n \implies hash-range-budget\ b\ (k\ xs)$ 

```



```

shows
  hash-range-budget b
    ((ntimes read n :: ('f list, ('f, 'a) protocol-channel-scheme) state-monad)
       $\gg$  k)
proof –
  have read-range:
    hash-range-budget (n * 0)
      (ntimes read n :: ('f list, ('f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-range-budget-ntimes[OF hash-range-budget-read])
  have hash-range-budget (n * 0 + b)
    ((ntimes read n :: ('f list, ('f, 'a) protocol-channel-scheme) state-monad)
       $\gg$  k)
  proof (rule hash-range-budget-bind-on-outcomes[OF read-range])
    fix s xs t
    assume out:
      Some (xs, t)  $\in$ 
        set-dist (execute
          (ntimes read n ::
            ('f list, ('f, 'a) protocol-channel-scheme) state-monad) s)
    show hash-range-budget b (k xs)
      by (rule k) (use ntimes-read-any-outcome[OF out] in simp)
  qed
  then show ?thesis by simp
qed

```

```

lemma hash-collision-budget-ntimes-read-bind:
  fixes k :: 'f list  $\Rightarrow$ 
    ('y, ('f, 'a) protocol-channel-scheme) state-monad
  assumes range-k:  $\bigwedge xs. \text{length } xs = n \implies \text{hash-range-budget } b (k \text{ } xs)$ 
    and coll-k:  $\bigwedge xs. \text{length } xs = n \implies \text{hash-collision-budget } b (k \text{ } xs)$ 
  shows
    hash-collision-budget b
      ((ntimes read n :: ('f list, ('f, 'a) protocol-channel-scheme) state-monad)
         $\gg$  k)
  proof –
  have read-range:
    hash-range-budget (n * 0)
      (ntimes read n :: ('f list, ('f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-range-budget-ntimes[OF hash-range-budget-read])
  have read-coll:
    hash-collision-budget (n * 0)
      (ntimes read n :: ('f list, ('f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-collision-budget-ntimes
      [OF hash-range-budget-read hash-collision-budget-read])
  have hash-collision-budget (n * 0 + b)
    ((ntimes read n :: ('f list, ('f, 'a) protocol-channel-scheme) state-monad)
       $\gg$  k)
  proof (rule hash-collision-budget-bind-on-outcomes
    [OF read-range read-coll])

```

```

fix  $s \ xs \ t$ 
assume  $out$ :
   $Some \ (xs, t) \in$ 
     $set-dist \ (execute$ 
       $(ntimes \ read \ n ::$ 
         $(\lambda f \ list, (\lambda f, 'a) \ protocol-channel-scheme) \ state-monad) \ s)$ 
     $)$ 
have  $len$ :  $length \ xs = n$ 
  using  $ntimes-read-any-outcome[OF \ out]$  by  $simp$ 
show  $hash-range-budget \ b \ (k \ xs)$ 
  by  $(rule \ range-k[OF \ len])$ 
show  $hash-collision-budget \ b \ (k \ xs)$ 
  by  $(rule \ coll-k[OF \ len])$ 
qed
then show  $?thesis$  by  $simp$ 
qed

lemma  $hash-range-budget-mmap$ :
  fixes  $ms :: (\lambda x, (\lambda f, 'a) \ protocol-channel-scheme) \ state-monad \ list$ 
  assumes  $ms$ :  $\bigwedge m. m \in set \ ms \implies hash-range-budget \ n \ m$ 
  shows  $hash-range-budget \ (length \ ms * n) \ (mmap \ ms)$ 
  using  $ms$ 
proof  $(induction \ ms)$ 
  case  $Nil$ 
  then show  $?case$ 
    by  $(simp \ add: \ hash-range-budget-return)$ 
next
  case  $(Cons \ m \ ms)$ 
  have  $head$ :  $hash-range-budget \ n \ m$ 
  using  $Cons.prems$  by  $simp$ 
  have  $tail$ :  $hash-range-budget \ (length \ ms * n) \ (mmap \ ms)$ 
  using  $Cons.IH \ Cons.prems$  by  $simp$ 
  have  $cont$ :
     $hash-range-budget \ (length \ ms * n + 0)$ 
     $(mmap \ ms \gg (\lambda xs. return \ (x \# \ xs)))$  for  $x$ 
    by  $(rule \ hash-range-budget-bind[OF \ tail \ hash-range-budget-return])$ 
  have  $hash-range-budget \ (n + (length \ ms * n + 0))$ 
     $(m \gg (\lambda x. mmap \ ms \gg (\lambda xs. return \ (x \# \ xs))))$ 
    by  $(rule \ hash-range-budget-bind[OF \ head \ cont])$ 
  then show  $?case$ 
    by  $simp$ 
qed

lemma  $hash-collision-budget-mmap$ :
  fixes  $ms :: (\lambda x, (\lambda f, 'a) \ protocol-channel-scheme) \ state-monad \ list$ 
  assumes  $range-ms$ :  $\bigwedge m. m \in set \ ms \implies hash-range-budget \ n \ m$ 
  and  $coll-ms$ :  $\bigwedge m. m \in set \ ms \implies hash-collision-budget \ n \ m$ 
  shows  $hash-collision-budget \ (length \ ms * n) \ (mmap \ ms)$ 
  using  $range-ms \ coll-ms$ 
proof  $(induction \ ms)$ 

```

```

case Nil
then show ?case
  by (simp add: hash-collision-budget-return)
next
case (Cons m ms)
have head-range: hash-range-budget n m
  using Cons.prem by simp
have head-coll: hash-collision-budget n m
  using Cons.prem by simp
have tail-range: hash-range-budget (length ms * n) (mmap ms)
  by (rule hash-range-budget-mmap) (use Cons.prem in auto)
have tail-coll: hash-collision-budget (length ms * n) (mmap ms)
  using Cons.IH Cons.prem by simp
have cont-range:
  hash-range-budget (length ms * n + 0)
  (mmap ms  $\gg$  ( $\lambda x$ . return (x # xs))) for x
  by (rule hash-range-budget-bind[OF tail-range hash-range-budget-return])
have cont-coll:
  hash-collision-budget (length ms * n + 0)
  (mmap ms  $\gg$  ( $\lambda x$ . return (x # xs))) for x
  by (rule hash-collision-budget-bind
    [OF tail-range tail-coll hash-range-budget-return
      hash-collision-budget-return])
have hash-collision-budget (n + (length ms * n + 0))
  (m  $\gg$  ( $\lambda x$ . mmap ms  $\gg$  ( $\lambda x$ . return (x # xs))))
  by (rule hash-collision-budget-bind
    [OF head-range head-coll cont-range cont-coll])
then show ?case
  by simp
qed

```

```

lemma hash-range-budget-mfold:
  fixes steps :: ('x  $\Rightarrow$  ('f, 'a) protocol-channel-scheme) state-monad) list
  assumes steps:  $\bigwedge \text{step } x. \text{step} \in \text{set steps} \implies$ 
    hash-range-budget n (step x)
  shows hash-range-budget (length steps * n) (mfold x steps)
  using steps
proof (induction steps arbitrary: x)
  case Nil
  then show ?case
    by (simp add: hash-range-budget-return)
next
case (Cons step steps)
have head: hash-range-budget n (step x)
  using Cons.prem by simp
have tail:  $\bigwedge y. \text{hash-range-budget (length steps * n) (mfold y steps)}$ 
  using Cons.IH Cons.prem by simp
have hash-range-budget (n + length steps * n)
  (step x  $\gg$  ( $\lambda y. \text{mfold } y \text{ steps}$ ))

```

```

    by (rule hash-range-budget-bind[OF head tail])
  then show ?case
    by simp
qed

lemma hash-collision-budget-mfold:
  fixes steps :: ('x  $\Rightarrow$  ('x, ('f, 'a) protocol-channel-scheme) state-monad) list
  assumes range-steps:  $\bigwedge \text{step } x. \text{step} \in \text{set steps} \Rightarrow$ 
    hash-range-budget n (step x)
  and coll-steps:  $\bigwedge \text{step } x. \text{step} \in \text{set steps} \Rightarrow$ 
    hash-collision-budget n (step x)
  shows hash-collision-budget (length steps * n) (mfold x steps)
  using range-steps coll-steps
proof (induction steps arbitrary: x)
  case Nil
  then show ?case
    by (simp add: hash-collision-budget-return)
next
  case (Cons step steps)
  have head-range: hash-range-budget n (step x)
    using Cons.prem1 by simp
  have head-coll: hash-collision-budget n (step x)
    using Cons.prem2 by simp
  have tail-range:
     $\bigwedge y. \text{hash-range-budget (length steps * n) (mfold y steps)}$ 
    by (rule hash-range-budget-mfold) (use Cons.prem1 in auto)
  have tail-coll:
     $\bigwedge y. \text{hash-collision-budget (length steps * n) (mfold y steps)}$ 
    using Cons.IH Cons.prem2 by simp
  have hash-collision-budget (n + length steps * n)
    (step x  $\gg$  ( $\lambda y. \text{mfold y steps}$ ))
    by (rule hash-collision-budget-bind
      [OF head-range head-coll tail-range tail-coll])
  then show ?case
    by simp
qed

```

```

lemma hash-range-budget-mfold-invariant:
  fixes steps :: ('x  $\Rightarrow$  ('x, ('f, 'a) protocol-channel-scheme) state-monad) list
  assumes init: I x
  and step-budget:
     $\bigwedge \text{step } y. \text{step} \in \text{set steps} \Rightarrow I y \Rightarrow$ 
    hash-range-budget n (step y)
  and step-inv:
     $\bigwedge \text{step } y \ z \ s \ t. \text{step} \in \text{set steps} \Rightarrow I y \Rightarrow$ 
    Some (z, t)  $\in$  set-dist (execute (step y) s)  $\Rightarrow I z$ 
  shows hash-range-budget (length steps * n) (mfold x steps)
  using init step-budget step-inv
proof (induction steps arbitrary: x)

```

```

case Nil
then show ?case
  by (simp add: hash-range-budget-return)
next
case (Cons step steps)
have Ix: I x
  using Cons.prem by simp
have head: hash-range-budget n (step x)
  by (rule Cons.prem(2)) (use Ix in auto)
have tail:
  hash-range-budget (length steps * n) (mfold y steps)
  if out: Some (y, t) ∈ set-dist (execute (step x) s)
  for s y t
proof (rule Cons.IH)
  show I y
  proof –
    have mem: step ∈ set (step # steps)
    by simp
    have inv:  $\bigwedge z s t.$ 
      Some (z, t) ∈ set-dist (execute (step x) s)  $\implies$  I z
    by (rule Cons.prem(3)[OF mem Ix])
    show ?thesis
    by (rule inv[OF out])
  qed
show  $\bigwedge \text{step } y. \text{step} \in \text{set steps} \implies I y \implies$ 
  hash-range-budget n (step y)
  by (rule Cons.prem(2)) simp-all
show  $\bigwedge \text{step } y z s t.$ 
  step ∈ set steps  $\implies$ 
  I y  $\implies$  Some (z, t) ∈ set-dist (execute (step y) s)  $\implies$  I z
  by (rule Cons.prem(3)) simp-all
qed
have hash-range-budget (n + length steps * n)
  (step x  $\gg$  (λy. mfold y steps))
  by (rule hash-range-budget-bind-on-outcomes[OF head]) (rule tail)
then show ?case
  by simp
qed

```

lemma hash-collision-budget-mfold-invariant:

```

fixes steps :: ('x  $\Rightarrow$  ('f, 'a) protocol-channel-scheme) state-monad) list
assumes init: I x
and range-step:
   $\bigwedge \text{step } y. \text{step} \in \text{set steps} \implies I y \implies$ 
  hash-range-budget n (step y)
and coll-step:
   $\bigwedge \text{step } y. \text{step} \in \text{set steps} \implies I y \implies$ 
  hash-collision-budget n (step y)
and step-inv:

```

```

 $\bigwedge \text{step } y \ z \ s \ t. \text{ step} \in \text{set steps} \implies I \ y \implies$ 
  Some  $(z, t) \in \text{set-dist } (\text{execute } (\text{step } y) \ s) \implies I \ z$ 
shows hash-collision-budget (length steps * n) (mfold x steps)
using init range-step coll-step step-inv
proof (induction steps arbitrary: x)
  case Nil
  then show ?case
    by (simp add: hash-collision-budget-return)
next
  case (Cons step steps)
  have Ix: I x
    using Cons.prem1 by simp
  have head-range: hash-range-budget n (step x)
    by (rule Cons.prem2) (use Ix in auto)
  have head-coll: hash-collision-budget n (step x)
    by (rule Cons.prem3) (use Ix in auto)
  have tail-range:
    hash-range-budget (length steps * n) (mfold y steps)
    if out: Some  $(y, t) \in \text{set-dist } (\text{execute } (\text{step } x) \ s)$ 
    for s y t
  proof (rule hash-range-budget-mfold-invariant)
    show I y
    proof –
      have mem: step  $\in$  set (step # steps)
        by simp
      have inv:  $\bigwedge z \ s \ t. \text{ Some } (z, t) \in \text{set-dist } (\text{execute } (\text{step } x) \ s) \implies I \ z$ 
        by (rule Cons.prem4)[OF mem Ix]
      show ?thesis
        by (rule inv[OF out])
    qed
  show  $\bigwedge \text{step } y. \text{ step} \in \text{set steps} \implies I \ y \implies$ 
    hash-range-budget n (step y)
    by (rule Cons.prem2) simp-all
  show  $\bigwedge \text{step } y \ z \ s \ t. \text{ step} \in \text{set steps} \implies$ 
    I y  $\implies \text{Some } (z, t) \in \text{set-dist } (\text{execute } (\text{step } y) \ s) \implies I \ z$ 
    by (rule Cons.prem4) simp-all
  qed
  have tail-coll:
    hash-collision-budget (length steps * n) (mfold y steps)
    if out: Some  $(y, t) \in \text{set-dist } (\text{execute } (\text{step } x) \ s)$ 
    for s y t
  proof (rule Cons.IH)
    show I y
    proof –
      have mem: step  $\in$  set (step # steps)
        by simp
      have inv:  $\bigwedge z \ s \ t. \text{ Some } (z, t) \in \text{set-dist } (\text{execute } (\text{step } x) \ s) \implies I \ z$ 

```

```

    Some (z, t) ∈ set-dist (execute (step x) s) ⇒ I z
  by (rule Cons.prem(4)[OF mem Ix])
show ?thesis
  by (rule inv[OF out])
qed
show ∧ step y. step ∈ set steps ⇒ I y ⇒
  hash-range-budget n (step y)
  by (rule Cons.prem(2)) simp-all
show ∧ step y. step ∈ set steps ⇒ I y ⇒
  hash-collision-budget n (step y)
  by (rule Cons.prem(3)) simp-all
show ∧ step y z s t.
  step ∈ set steps ⇒
  I y ⇒ Some (z, t) ∈ set-dist (execute (step y) s) ⇒ I z
  by (rule Cons.prem(4)) simp-all
qed
have hash-collision-budget (n + length steps * n)
  (step x ≫ (λy. mfold y steps))
  by (rule hash-collision-budget-bind-on-outcomes
    [OF head-range head-coll tail-range tail-coll])
then show ?case
  by simp
qed

```

lemma *hash-range-budget-fri-state-mfold-invariant:*

```

fixes st :: nat × 'f × nat × nat
and steps ::
  (nat × 'f × nat × nat ⇒
   (nat × 'f × nat × nat,
    ('f, 'a) protocol-channel-scheme) state-monad) list
and I :: nat × 'f × nat × nat ⇒ bool
assumes init: I st
and step-budget:
  ∧ step y. step ∈ set steps ⇒ I y ⇒
  hash-range-budget n (step y)
and step-inv:
  ∧ step y z s t. step ∈ set steps ⇒ I y ⇒
  Some (z, t) ∈ set-dist (execute (step y) s) ⇒ I z
shows hash-range-budget (length steps * n) (mfold st steps)
using assms by (rule hash-range-budget-mfold-invariant)

```

lemma *hash-collision-budget-fri-state-mfold-invariant:*

```

fixes st :: nat × 'f × nat × nat
and steps ::
  (nat × 'f × nat × nat ⇒
   (nat × 'f × nat × nat,
    ('f, 'a) protocol-channel-scheme) state-monad) list
and I :: nat × 'f × nat × nat ⇒ bool
assumes init: I st

```

and *range-step*:
 $\bigwedge \text{step } y. \text{ step} \in \text{set steps} \implies I y \implies$
 $\text{hash-range-budget } n (\text{step } y)$
and *coll-step*:
 $\bigwedge \text{step } y. \text{ step} \in \text{set steps} \implies I y \implies$
 $\text{hash-collision-budget } n (\text{step } y)$
and *step-inv*:
 $\bigwedge \text{step } y \ z \ s \ t. \text{ step} \in \text{set steps} \implies I y \implies$
 $\text{Some } (z, t) \in \text{set-dist } (\text{execute } (\text{step } y) \ s) \implies I z$
shows *hash-collision-budget* (*length steps * n*) (*mfold st steps*)
using *assms* **by** (*rule hash-collision-budget-mfold-invariant*)

lemma *hash-target-program-receive-fri-commits*:
hash-target-program B 1 receive-fri-commits
proof –
have *tail*:
hash-target-program B (1 + 0)
(receive-random-field-element $\gg=$ ($\lambda b. \text{return } (b, r)$))
for *r*
by (*rule hash-target-program-bind*)
(rule hash-target-program-receive-random-field-element,
rule hash-target-program-return)
have *hash-target-program B (0 + (1 + 0))*
(read $\gg=$
($\lambda r. \text{receive-random-field-element } \gg=$
($\lambda b. \text{return } (b, r)$))
by (*rule hash-target-program-bind[OF hash-target-program-read tail]*)
then show *?thesis*
unfolding *receive-fri-commits-def* **by** *simp*
qed

lemma *hash-target-program-receive-fri-commits-with*:
assumes *challenge: hash-target-program B 1 receive-challenge*
shows *hash-target-program B 1*
(receive-fri-commits-with receive-challenge)
proof –
have *tail*:
hash-target-program B (1 + 0)
(receive-challenge $\gg=$ ($\lambda b. \text{return } (b, r)$))
for *r*
by (*rule hash-target-program-bind[OF challenge hash-target-program-return]*)
have *hash-target-program B (0 + (1 + 0))*
(read $\gg=$
($\lambda r. \text{receive-challenge } \gg=$
($\lambda b. \text{return } (b, r)$))
by (*rule hash-target-program-bind[OF hash-target-program-read tail]*)
then show *?thesis*

unfolding *receive-fri-commits-with-def* **by** *simp*
qed

lemma *hash-target-program-receive-trace-fri-commits:*
hash-target-program B 1 receive-trace-fri-commits
unfolding *receive-trace-fri-commits-def*
by (*rule hash-target-program-receive-fri-commits-with*)
(*rule hash-target-program-receive-trace-fri-challenge*)

lemma *hash-target-program-receive-composition-fri-commits:*
hash-target-program B 1 receive-composition-fri-commits
unfolding *receive-composition-fri-commits-def*
by (*rule hash-target-program-receive-fri-commits-with*)
(*rule hash-target-program-receive-composition-fri-challenge*)

lemma *hash-range-budget-receive-fri-commits:*
hash-range-budget 1 receive-fri-commits

proof –

have *read-then-random:*
hash-range-budget (0 + (1 + 0))
(*read* $\gg$
($\lambda r.$ *receive-random-field-element* $\gg$
($\lambda b.$ *return (b, r)*)))
by (*rule hash-range-budget-bind*)
(*rule hash-range-budget-read*,
rule hash-range-budget-bind
[*OF hash-range-budget-receive-random-field-element*
hash-range-budget-return])

then show *?thesis*

unfolding *receive-fri-commits-def* **by** *simp*

qed

lemma *hash-collision-budget-receive-fri-commits:*
hash-collision-budget 1 receive-fri-commits

proof –

have *random-then-return-range:*
hash-range-budget (1 + 0)
(*receive-random-field-element* $\gg$ ($\lambda b.$ *return (b, r)*)) **for** *r*
by (*rule hash-range-budget-bind*
[*OF hash-range-budget-receive-random-field-element*
hash-range-budget-return])

have *random-then-return-coll:*
hash-collision-budget (1 + 0)
(*receive-random-field-element* $\gg$ ($\lambda b.$ *return (b, r)*)) **for** *r*
by (*rule hash-collision-budget-bind*
[*OF hash-range-budget-receive-random-field-element*
hash-collision-budget-receive-random-field-element
hash-range-budget-return hash-collision-budget-return])

have *hash-collision-budget (0 + (1 + 0))*

(read $\gg$
 (λr . receive-random-field-element $\gg$
 (λb . return (b, r))))
 by (rule hash-collision-budget-bind
 [OF hash-range-budget-read hash-collision-budget-read
 random-then-return-range random-then-return-coll])
 then show ?thesis
 unfolding receive-fri-commits-def by simp
 qed

lemma hash-range-budget-receive-fri-commits-with:
 assumes challenge: hash-range-budget 1 receive-challenge
 shows hash-range-budget 1 (receive-fri-commits-with receive-challenge)
proof –
 have read-then-challenge:
 hash-range-budget (0 + (1 + 0))
 (read $\gg$
 (λr . receive-challenge $\gg$
 (λb . return (b, r))))
 by (rule hash-range-budget-bind
 (rule hash-range-budget-read,
 rule hash-range-budget-bind [OF challenge hash-range-budget-return])
 then show ?thesis
 unfolding receive-fri-commits-with-def by simp
 qed

lemma hash-collision-budget-receive-fri-commits-with:
 assumes challenge-range: hash-range-budget 1 receive-challenge
 and challenge-collision: hash-collision-budget 1 receive-challenge
 shows hash-collision-budget 1 (receive-fri-commits-with receive-challenge)
proof –
 have challenge-then-return-range:
 hash-range-budget (1 + 0)
 (receive-challenge $\gg$ (λb . return (b, r))) **for** r
 by (rule hash-range-budget-bind
 [OF challenge-range hash-range-budget-return])
 have challenge-then-return-coll:
 hash-collision-budget (1 + 0)
 (receive-challenge $\gg$ (λb . return (b, r))) **for** r
 by (rule hash-collision-budget-bind
 [OF challenge-range challenge-collision
 hash-range-budget-return hash-collision-budget-return])
 have hash-collision-budget (0 + (1 + 0))
 (read $\gg$
 (λr . receive-challenge $\gg$
 (λb . return (b, r))))
 by (rule hash-collision-budget-bind
 [OF hash-range-budget-read hash-collision-budget-read
 challenge-then-return-range challenge-then-return-coll])

```

then show ?thesis
  unfolding receive-fri-commits-with-def by simp
qed

lemma hash-range-budget-receive-trace-fri-commits:
  hash-range-budget 1 receive-trace-fri-commits
  unfolding receive-trace-fri-commits-def
  by (rule hash-range-budget-receive-fri-commits-with)
    (rule hash-range-budget-receive-trace-fri-challenge)

lemma hash-collision-budget-receive-trace-fri-commits:
  hash-collision-budget 1 receive-trace-fri-commits
  unfolding receive-trace-fri-commits-def
  by (rule hash-collision-budget-receive-fri-commits-with)
    (rule hash-range-budget-receive-trace-fri-challenge,
     rule hash-collision-budget-receive-trace-fri-challenge)

lemma hash-range-budget-receive-composition-fri-commits:
  hash-range-budget 1 receive-composition-fri-commits
  unfolding receive-composition-fri-commits-def
  by (rule hash-range-budget-receive-fri-commits-with)
    (rule hash-range-budget-receive-composition-fri-challenge)

lemma hash-collision-budget-receive-composition-fri-commits:
  hash-collision-budget 1 receive-composition-fri-commits
  unfolding receive-composition-fri-commits-def
  by (rule hash-collision-budget-receive-fri-commits-with)
    (rule hash-range-budget-receive-composition-fri-challenge,
     rule hash-collision-budget-receive-composition-fri-challenge)

lemma hash-target-program-protocol-check-authentication-path-raw:
  fixes leaf :: 'f protocol-hash-input
  shows hash-target-program B (Suc (length path))
    (protocol-merkle.check-authentication-path len i leaf path ::
     ('f, ('f, 'a) protocol-channel-scheme) state-monad)
proof (induction path arbitrary: len i)
  case Nil
  have hash-target-program B 1
    (hash leaf :: ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-target-program-hash)
  then show ?case
    by (simp add: protocol-merkle.check-authentication-path.simps)
next
  case (Cons a path)
  show ?case
  proof (cases i < len div 2)
  case True
  have rec:
    hash-target-program B (Suc (length path))

```

```

      (protocol-merkle.check-authentication-path (len div 2) i leaf path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    using Cons.IH[of len div 2 i] by simp
  have bound:
    hash-target-program B (Suc (length path) + 1)
      ((protocol-merkle.check-authentication-path (len div 2) i leaf path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda x$ . hash (MerkleNode x a)))
    by (rule hash-target-program-bind[OF rec hash-target-program-hash])
  show ?thesis
    using True bound
    by (simp add: protocol-merkle.check-authentication-path.simps)
next
case False
have rec:
  hash-target-program B (Suc (length path))
    (protocol-merkle.check-authentication-path
      (len div 2) (i - len div 2) leaf path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  using Cons.IH[of len div 2 i - len div 2] by simp
have bound:
  hash-target-program B (Suc (length path) + 1)
    ((protocol-merkle.check-authentication-path
      (len div 2) (i - len div 2) leaf path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
      ( $\lambda x$ . hash (MerkleNode a x)))
  by (rule hash-target-program-bind[OF rec hash-target-program-hash])
show ?thesis
  using False bound
  by (simp add: protocol-merkle.check-authentication-path.simps)
qed
qed

lemma hash-target-program-check-authentication-path:
  hash-target-program B (Suc (length path))
    (check-authentication-path len i v path)
unfolding check-authentication-path-def protocol-check-authentication-path-def
by (rule hash-target-program-protocol-check-authentication-path-raw)

lemma hash-target-program-check-authentication-path-assert-return:
  fixes path :: 'f list
  and v root :: 'f
  and r :: 'r
shows
  hash-target-program B (Suc (length path))
    ((check-authentication-path len i v path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
      ( $\lambda ap$ . assert (ap = root)  $\gg$  ( $\lambda$ -. return r)))
proof -

```

```

have tail:
  hash-target-program B (0 + 0)
  (assert (ap = root)  $\gg$  (λ-. return r) ::
    ('r, ('f, 'a) protocol-channel-scheme) state-monad)
  for ap
  by (rule hash-target-program-bind)
  (rule hash-target-program-assert, rule hash-target-program-return)
have hash-target-program B (Suc (length path) + (0 + 0))
  ((check-authentication-path len i v path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    (λap. assert (ap = root)  $\gg$  (λ-. return r)))
  by (rule hash-target-program-bind)
  (rule hash-target-program-check-authentication-path, rule tail)
then show ?thesis by simp
qed

```

lemma hash-target-program-check-authentication-path-assert-bind:

```

fixes path :: 'f list
and v root :: 'f
and k :: unit  $\Rightarrow$ 
  ('r, ('f, 'a) protocol-channel-scheme) state-monad
assumes k:  $\bigwedge u.$  hash-target-program B n (k u)
shows
  hash-target-program B (Suc (length path) + n)
  ((check-authentication-path len i v path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    (λap. assert (ap = root)  $\gg$  k))

```

proof –

```

have tail:
  hash-target-program B (0 + n)
  (assert (ap = root)  $\gg$  k ::
    ('r, ('f, 'a) protocol-channel-scheme) state-monad)
  for ap
  by (rule hash-target-program-bind[OF hash-target-program-assert k])
have hash-target-program B (Suc (length path) + (0 + n))
  ((check-authentication-path len i v path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    (λap. assert (ap = root)  $\gg$  k))
  by (rule hash-target-program-bind)
  (rule hash-target-program-check-authentication-path, rule tail)
then show ?thesis by simp
qed

```

lemma hash-target-program-fri-layer-opening-finish:

```

assumes xp-path-len: length xp-path = floor-log len
and xn-path-len: length xn-path = floor-log len
shows
  hash-target-program B
  (Suc (floor-log len) + Suc (floor-log len))

```

```

    (fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)
proof -
  let ?H = Suc (floor-log len)
  let ?sidx = (i + len div 2) mod len
  let ?out =
    (i mod (len div 2),
     (xp + xn) div 2 +
     b * ((xp - xn) div (2 * ((h ^ i) * shift) ^ pw)),
     len div 2, pw + pw)
  have second:
    hash-target-program B ?H
    ((check-authentication-path len ?sidx xn xn-path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = f) >>= (λ-. return ?out)))
  by (subst xn-path-len[symmetric])
    (rule hash-target-program-check-authentication-path-assert-return)
  have first0:
    hash-target-program B (Suc (length xp-path) + ?H)
    ((check-authentication-path len i xp xp-path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = f) >>=
        (λ-.
          (check-authentication-path len ?sidx xn xn-path ::
            ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
            (λap. assert (ap = f) >>= (λ-. return ?out))))))
  by (rule hash-target-program-check-authentication-path-assert-bind)
    (use second in simp)
  have first:
    hash-target-program B (?H + ?H)
    ((check-authentication-path len i xp xp-path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = f) >>=
        (λ-.
          (check-authentication-path len ?sidx xn xn-path ::
            ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
            (λap. assert (ap = f) >>= (λ-. return ?out))))))
  using first0 xp-path-len by simp
  have raw:
    hash-target-program B (0 + (?H + ?H))
    ((assert (xp = x) ::
      (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ-.
        (check-authentication-path len i xp xp-path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λap. assert (ap = f) >>=
            (λ-.
              (check-authentication-path len ?sidx xn xn-path ::
                ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
                (λap. assert (ap = f) >>= (λ-. return ?out))))))))

```

```

    by (rule hash-target-program-bind[OF hash-target-program-assert first])
  show ?thesis
    using raw unfolding fri-layer-opening-finish-def
    by (simp add: Let-def)
qed

lemma hash-target-program-fri-layer-opening-step:
  hash-target-program B
  (Suc (floor-log len) + Suc (floor-log len))
  (fri-layer-opening-step (b, f) (i, x, len, pw))
proof -
  let ?N = Suc (floor-log len) + Suc (floor-log len)
  have after-xn-path:
    hash-target-program B ?N
    ((ntimes read (floor-log len) ::
      ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λxn-path.
        fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
  if xp-path-len: length xp-path = floor-log len
  for xp xp-path xn
proof (rule hash-target-program-ntimes-read-bind)
  fix xn-path :: 'f list
  assume xn-path-len: length xn-path = floor-log len
  show hash-target-program B ?N
    (fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)
  by (rule hash-target-program-fri-layer-opening-finish
      [OF xp-path-len xn-path-len])
qed
have after-xn:
  hash-target-program B ?N
  (read >>=
    (λxn.
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λxn-path.
          fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))
  if xp-path-len: length xp-path = floor-log len
  for xp xp-path
proof -
  have hash-target-program B (0 + ?N)
    (read >>=
      (λxn.
        (ntimes read (floor-log len) ::
          ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λxn-path.
            fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))
  by (rule hash-target-program-bind
      [OF hash-target-program-read after-xn-path[OF xp-path-len]])
  then show ?thesis by simp

```

```

qed
have after-xp-path:
  hash-target-program B ?N
  ((ntimes read (floor-log len) ::
    ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda xp$ .  

      read  $\gg$ 
      ( $\lambda xn$ .  

        (ntimes read (floor-log len) ::
          ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
          ( $\lambda xn$ .path.  

            fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))))
for xp
proof (rule hash-target-program-ntimes-read-bind)
fix xp-path :: 'f list
assume xp-path-len: length xp-path = floor-log len
show hash-target-program B ?N
  (read  $\gg$ 
    ( $\lambda xn$ .  

      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda xn$ .path.  

          fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))
by (rule after-xn[OF xp-path-len])
qed
have hash-target-program B (0 + ?N)
  (read  $\gg$ 
    ( $\lambda xp$ .  

      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda xp$ .path.  

          read  $\gg$ 
          ( $\lambda xn$ .  

            (ntimes read (floor-log len) ::
              ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
              ( $\lambda xn$ .path.  

                fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))))
by (rule hash-target-program-bind
  [OF hash-target-program-read after-xp-path])
then show ?thesis
  unfolding fri-layer-opening-step-unfold-finish by simp
qed

lemma hash-range-budget-protocol-check-authentication-path-raw:
  fixes leaf :: 'f protocol-hash-input
  shows hash-range-budget (Suc (length path))
  (protocol-merkle.check-authentication-path len i leaf path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)
proof (induction path arbitrary: len i)

```



```

case Nil
have hash-budget:
  hash-range-budget 1
  (hash leaf :: ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-hash)
then show ?case
  by (simp add: protocol-merkle.check-authentication-path.simps)
next
case (Cons a path)
show ?case
proof (cases i < len div 2)
  case True
  have rec:
    hash-range-budget (Suc (length path))
    (protocol-merkle.check-authentication-path (len div 2) i leaf path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    using Cons.IH[of len div 2 i] by simp
  have bound:
    hash-range-budget (Suc (length path) + 1)
    ((protocol-merkle.check-authentication-path (len div 2) i leaf path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
      ( $\lambda x$ . hash (MerkleNode x a)))
    by (rule hash-range-budget-bind[OF rec hash-range-budget-hash])
  show ?thesis
  using True bound
  by (simp add: protocol-merkle.check-authentication-path.simps)
next
case False
  have rec:
    hash-range-budget (Suc (length path))
    (protocol-merkle.check-authentication-path
      (len div 2) (i - len div 2) leaf path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    using Cons.IH[of len div 2 i - len div 2] by simp
  have bound:
    hash-range-budget (Suc (length path) + 1)
    ((protocol-merkle.check-authentication-path
      (len div 2) (i - len div 2) leaf path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
      ( $\lambda x$ . hash (MerkleNode a x)))
    by (rule hash-range-budget-bind[OF rec hash-range-budget-hash])
  show ?thesis
  using False bound
  by (simp add: protocol-merkle.check-authentication-path.simps)
qed
qed

```

lemma hash-range-budget-check-authentication-path:
 hash-range-budget (Suc (length path))

```

    (check-authentication-path len i v path)
unfolding check-authentication-path-def protocol-check-authentication-path-def
by (rule hash-range-budget-protocol-check-authentication-path-raw)

lemma hash-collision-budget-protocol-check-authentication-path-raw:
  fixes leaf :: 'f protocol-hash-input
  shows hash-collision-budget (Suc (length path))
    (protocol-merkle.check-authentication-path len i leaf path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
proof (induction path arbitrary: len i)
  case Nil
  have hash-budget:
    hash-collision-budget 1
    (hash leaf :: ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-collision-budget-hash)
  then show ?case
    by (simp add: protocol-merkle.check-authentication-path.simps)
next
  case (Cons a path)
  show ?case
  proof (cases i < len div 2)
  case True
  have rec-range:
    hash-range-budget (Suc (length path))
    (protocol-merkle.check-authentication-path (len div 2) i leaf path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-protocol-check-authentication-path-raw)
  have rec-coll:
    hash-collision-budget (Suc (length path))
    (protocol-merkle.check-authentication-path (len div 2) i leaf path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  using Cons.IH[of len div 2 i] by simp
  have bound:
    hash-collision-budget (Suc (length path) + 1)
    ((protocol-merkle.check-authentication-path (len div 2) i leaf path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
      ( $\lambda x$ . hash (MerkleNode x a)))
  by (rule hash-collision-budget-bind
    [OF rec-range rec-coll hash-range-budget-hash
      hash-collision-budget-hash])
  show ?thesis
    using True bound
    by (simp add: protocol-merkle.check-authentication-path.simps)
next
  case False
  have rec-range:
    hash-range-budget (Suc (length path))
    (protocol-merkle.check-authentication-path
      (len div 2) (i - len div 2) leaf path ::

```

```

      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-protocol-check-authentication-path-raw)
  have rec-coll:
    hash-collision-budget (Suc (length path))
      (protocol-merkle.check-authentication-path
        (len div 2) (i - len div 2) leaf path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    using Cons.IH[of len div 2 i - len div 2] by simp
  have bound:
    hash-collision-budget (Suc (length path) + 1)
      ((protocol-merkle.check-authentication-path
        (len div 2) (i - len div 2) leaf path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda x$ . hash (MerkleNode a x)))
    by (rule hash-collision-budget-bind
      [OF rec-range rec-coll hash-range-budget-hash
        hash-collision-budget-hash])
  show ?thesis
    using False bound
    by (simp add: protocol-merkle.check-authentication-path.simps)
qed
qed

lemma hash-collision-budget-check-authentication-path:
  hash-collision-budget (Suc (length path))
    (check-authentication-path len i v path)
unfolding check-authentication-path-def protocol-check-authentication-path-def
by (rule hash-collision-budget-protocol-check-authentication-path-raw)

lemma hash-range-budget-check-authentication-path-assert-return:
  fixes path :: 'f list
  and v root :: 'f
  and r :: 'r
  shows
    hash-range-budget (Suc (length path))
      (((check-authentication-path len i v path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda ap$ . assert (ap = root)  $\gg$  ( $\lambda$ -. return r))))
proof -
  have check-budget:
    hash-range-budget (Suc (length path))
      (check-authentication-path len i v path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-range-budget-check-authentication-path)
  have assert-return:
    hash-range-budget (0 + 0)
      (assert (ap = root)  $\gg$  ( $\lambda$ -. return r) ::
        ('r, ('f, 'a) protocol-channel-scheme) state-monad) for ap
    by (rule hash-range-budget-bind

```

```

    [OF hash-range-budget-assert hash-range-budget-return])
  have hash-range-budget (Suc (length path) + (0 + 0))
    (((check-authentication-path len i v path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = root) >>= (λ-. return r))))
    by (rule hash-range-budget-bind[OF check-budget assert-return])
  then show ?thesis by simp
qed

lemma hash-collision-budget-check-authentication-path-assert-return:
  fixes path :: 'f list
  and v root :: 'f
  and r :: 'r
  shows
    hash-collision-budget (Suc (length path))
      (((check-authentication-path len i v path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = root) >>= (λ-. return r))))
  proof -
    have check-range:
      hash-range-budget (Suc (length path))
        (check-authentication-path len i v path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad)
      by (rule hash-range-budget-check-authentication-path)
    have check-coll:
      hash-collision-budget (Suc (length path))
        (check-authentication-path len i v path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad)
      by (rule hash-collision-budget-check-authentication-path)
    have assert-return-range:
      hash-range-budget (0 + 0)
        (assert (ap = root) >>= (λ-. return r) ::
          ('r, ('f, 'a) protocol-channel-scheme) state-monad) for ap
      by (rule hash-range-budget-bind
        [OF hash-range-budget-assert hash-range-budget-return])
    have assert-return-coll:
      hash-collision-budget (0 + 0)
        (assert (ap = root) >>= (λ-. return r) ::
          ('r, ('f, 'a) protocol-channel-scheme) state-monad) for ap
      by (rule hash-collision-budget-bind
        [OF hash-range-budget-assert hash-collision-budget-assert
          hash-range-budget-return hash-collision-budget-return])
    have hash-collision-budget (Suc (length path) + (0 + 0))
      (((check-authentication-path len i v path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = root) >>= (λ-. return r))))
      by (rule hash-collision-budget-bind
        [OF check-range check-coll assert-return-range assert-return-coll])
    then show ?thesis by simp
  
```

qed

lemma *hash-range-budget-check-authentication-path-assert-bind*:
fixes *path* :: 'f list
and *v root* :: 'f
and *k* :: unit $\Rightarrow$ ('r, ('f, 'a) protocol-channel-scheme) state-monad
assumes *k*: $\bigwedge u. \text{hash-range-budget } n \ (k \ u)$
shows
hash-range-budget (Suc (length *path*) + *n*)
 (((*check-authentication-path* len *i v path* ::
 ('f, ('f, 'a) protocol-channel-scheme) state-monad) $\gg=$
 ($\lambda ap. \text{assert } (ap = \text{root}) \gg= k$)))
proof –
have *check-budget*:
hash-range-budget (Suc (length *path*))
 (*check-authentication-path* len *i v path* ::
 ('f, ('f, 'a) protocol-channel-scheme) state-monad)
by (rule *hash-range-budget-check-authentication-path*)
have *assert-cont*:
hash-range-budget (0 + *n*)
 ($\text{assert } (ap = \text{root}) \gg= k$::
 ('r, ('f, 'a) protocol-channel-scheme) state-monad) **for** *ap*
by (rule *hash-range-budget-bind*[OF *hash-range-budget-assert k*])
have *hash-range-budget* (Suc (length *path*) + (0 + *n*))
 (((*check-authentication-path* len *i v path* ::
 ('f, ('f, 'a) protocol-channel-scheme) state-monad) $\gg=$
 ($\lambda ap. \text{assert } (ap = \text{root}) \gg= k$)))
by (rule *hash-range-budget-bind*[OF *check-budget assert-cont*])
then show ?thesis **by** simp
qed

lemma *hash-collision-budget-check-authentication-path-assert-bind*:
fixes *path* :: 'f list
and *v root* :: 'f
and *k* :: unit $\Rightarrow$ ('r, ('f, 'a) protocol-channel-scheme) state-monad
assumes *range-k*: $\bigwedge u. \text{hash-range-budget } n \ (k \ u)$
and *coll-k*: $\bigwedge u. \text{hash-collision-budget } n \ (k \ u)$
shows
hash-collision-budget (Suc (length *path*) + *n*)
 (((*check-authentication-path* len *i v path* ::
 ('f, ('f, 'a) protocol-channel-scheme) state-monad) $\gg=$
 ($\lambda ap. \text{assert } (ap = \text{root}) \gg= k$)))
proof –
have *check-range*:
hash-range-budget (Suc (length *path*))
 (*check-authentication-path* len *i v path* ::
 ('f, ('f, 'a) protocol-channel-scheme) state-monad)
by (rule *hash-range-budget-check-authentication-path*)
have *check-coll*:

```

    hash-collision-budget (Suc (length path))
      (check-authentication-path len i v path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-collision-budget-check-authentication-path)
have assert-range:
  hash-range-budget (0 + n)
    (assert (ap = root)  $\gg$  k ::
      ('r, ('f, 'a) protocol-channel-scheme) state-monad) for ap
  by (rule hash-range-budget-bind[OF hash-range-budget-assert range-k])
have assert-coll:
  hash-collision-budget (0 + n)
    (assert (ap = root)  $\gg$  k ::
      ('r, ('f, 'a) protocol-channel-scheme) state-monad) for ap
  by (rule hash-collision-budget-bind
    [OF hash-range-budget-assert hash-collision-budget-assert
      range-k coll-k])
have hash-collision-budget (Suc (length path) + (0 + n))
  (((check-authentication-path len i v path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda$ ap. assert (ap = root)  $\gg$  k)))
  by (rule hash-collision-budget-bind
    [OF check-range check-coll assert-range assert-coll])
then show ?thesis by simp
qed

```

lemma *hash-range-budget-fri-layer-opening-finish:*
assumes *xp-path-len:* $\text{length } xp\text{-path} = \text{floor-log } len$
and *xn-path-len:* $\text{length } xn\text{-path} = \text{floor-log } len$
shows
 $\text{hash-range-budget } (\text{Suc } (\text{floor-log } len) + \text{Suc } (\text{floor-log } len))$
 $(\text{fri-layer-opening-finish } b \ f \ i \ x \ len \ pw \ xp \ xp\text{-path } xn \ xn\text{-path})$
proof –
let $?b = \text{Suc } (\text{floor-log } len)$
let $?sidx = (i + len \text{ div } 2) \bmod len$
let $?out =$
 $(i \bmod (len \text{ div } 2),$
 $(xp + xn) \text{ div } 2 +$
 $b * ((xp - xn) \text{ div } (2 * ((h \wedge i) * shift) \wedge pw)),$
 $len \text{ div } 2, pw + pw)$
have *second:*
 $\text{hash-range-budget } ?b$
 $((\text{check-authentication-path } len \ ?sidx \ xn \ xn\text{-path} ::$
 $(\text{'f}, (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ state-monad}) \gg$
 $(\lambda ap. \text{assert } (ap = f) \gg (\lambda _. \text{return } ?out))))$
by (*subst xn-path-len[symmetric]*)
 (*rule hash-range-budget-check-authentication-path-assert-return*)
have *second-cont:*
 $\bigwedge u. \text{hash-range-budget } ?b$
 $((\text{check-authentication-path } len \ ?sidx \ xn \ xn\text{-path} ::$

```

      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = f) >>= (λ-. return ?out))))
using second by simp
have first0:
  hash-range-budget (Suc (length xp-path) + ?b)
  (((check-authentication-path len i xp xp-path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λap. assert (ap = f) >>=
      (λ-.
        (check-authentication-path len ?sidx xn xn-path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λap. assert (ap = f) >>= (λ-. return ?out)))))))
  by (rule hash-range-budget-check-authentication-path-assert-bind
    [OF second-cont])
have first:
  hash-range-budget (?b + ?b)
  (((check-authentication-path len i xp xp-path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λap. assert (ap = f) >>=
      (λ-.
        (check-authentication-path len ?sidx xn xn-path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λap. assert (ap = f) >>= (λ-. return ?out)))))))
  using first0 xp-path-len by simp
have raw:
  hash-range-budget (0 + (?b + ?b))
  ((assert (xp = x) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ-.
      (check-authentication-path len i xp xp-path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = f) >>=
          (λ-.
            (check-authentication-path len ?sidx xn xn-path ::
              ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
              (λap. assert (ap = f) >>= (λ-. return ?out)))))))
  by (rule hash-range-budget-bind[OF hash-range-budget-assert first])
have all:
  hash-range-budget (?b + ?b)
  ((assert (xp = x) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ-.
      (check-authentication-path len i xp xp-path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = f) >>=
          (λ-.
            (check-authentication-path len ?sidx xn xn-path ::
              ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
              (λap. assert (ap = f) >>= (λ-. return ?out)))))))

```

```

    using raw by simp
  from all show ?thesis
    unfolding fri-layer-opening-finish-def
    by (simp add: Let-def)
qed

lemma hash-collision-budget-fri-layer-opening-finish:
  assumes xp-path-len: length xp-path = floor-log len
    and xn-path-len: length xn-path = floor-log len
  shows
    hash-collision-budget (Suc (floor-log len) + Suc (floor-log len))
      (fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)
proof -
  let ?b = Suc (floor-log len)
  let ?sidx = (i + len div 2) mod len
  let ?out =
    (i mod (len div 2),
     (xp + xn) div 2 +
     b * ((xp - xn) div (2 * ((h ^ i) * shift) ^ pw)),
     len div 2, pw + pw)
  have second-range:
    hash-range-budget ?b
      (((check-authentication-path len ?sidx xn xn-path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = f) >>= (λ-. return ?out))))
  by (subst xn-path-len[symmetric])
    (rule hash-range-budget-check-authentication-path-assert-return)
  have second-coll:
    hash-collision-budget ?b
      (((check-authentication-path len ?sidx xn xn-path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = f) >>= (λ-. return ?out))))
  by (subst xn-path-len[symmetric])
    (rule hash-collision-budget-check-authentication-path-assert-return)
  have second-cont-range:
    ∧u. hash-range-budget ?b
      (((check-authentication-path len ?sidx xn xn-path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = f) >>= (λ-. return ?out))))
  using second-range by simp
  have second-cont-coll:
    ∧u. hash-collision-budget ?b
      (((check-authentication-path len ?sidx xn xn-path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = f) >>= (λ-. return ?out))))
  using second-coll by simp
  have first-range0:
    hash-range-budget (Suc (length xp-path) + ?b)
      (((check-authentication-path len i xp xp-path ::

```



```

    ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
  (λap. assert (ap = f) >>=
    (λ-.
      (check-authentication-path len ?sidx xn xn-path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = f) >>= (λ-. return ?out))))))
  by (rule hash-range-budget-check-authentication-path-assert-bind
    [OF second-cont-range])
have first-coll0:
  hash-collision-budget (Suc (length xp-path) + ?b)
  (((check-authentication-path len i xp xp-path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λap. assert (ap = f) >>=
      (λ-.
        (check-authentication-path len ?sidx xn xn-path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λap. assert (ap = f) >>= (λ-. return ?out))))))
    by (rule hash-collision-budget-check-authentication-path-assert-bind
      [OF second-cont-range second-cont-coll])
have first-range:
  hash-range-budget (?b + ?b)
  (((check-authentication-path len i xp xp-path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λap. assert (ap = f) >>=
      (λ-.
        (check-authentication-path len ?sidx xn xn-path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λap. assert (ap = f) >>= (λ-. return ?out))))))
    using first-range0 xp-path-len by simp
have first-coll:
  hash-collision-budget (?b + ?b)
  (((check-authentication-path len i xp xp-path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λap. assert (ap = f) >>=
      (λ-.
        (check-authentication-path len ?sidx xn xn-path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λap. assert (ap = f) >>= (λ-. return ?out))))))
    using first-coll0 xp-path-len by simp
have raw:
  hash-collision-budget (0 + (?b + ?b))
  ((assert (xp = x) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ-.
      (check-authentication-path len i xp xp-path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = f) >>=
          (λ-.
            (check-authentication-path len ?sidx xn xn-path ::

```

```

      (f, (f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = f) >>= (λ-. return ?out))))))
  by (rule hash-collision-budget-bind
      [OF hash-range-budget-assert hash-collision-budget-assert
        first-range first-coll])
  then show ?thesis
    unfolding fri-layer-opening-finish-def
    by (simp add: Let-def)
qed

lemma hash-range-budget-fri-layer-opening-step:
  hash-range-budget (Suc (floor-log len) + Suc (floor-log len))
  (fri-layer-opening-step (b, f) (i, x, len, pw))
proof -
  let ?B = Suc (floor-log len) + Suc (floor-log len)
  have after-xn-path:
    hash-range-budget ?B
    ((ntimes read (floor-log len) ::
      (f list, (f, 'a) protocol-channel-scheme) state-monad) >>=
      (λxn-path.
        fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
  if xp-path-len: length xp-path = floor-log len
  for xp xp-path xn
proof (rule hash-range-budget-ntimes-read-bind)
  fix xn-path :: f list
  assume xn-path-len: length xn-path = floor-log len
  show hash-range-budget ?B
    (fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)
  by (rule hash-range-budget-fri-layer-opening-finish
      [OF xp-path-len xn-path-len])
qed
have after-xn:
  hash-range-budget ?B
  (read >>=
    (λxn.
      (ntimes read (floor-log len) ::
        (f list, (f, 'a) protocol-channel-scheme) state-monad) >>=
        (λxn-path.
          fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))
  if xp-path-len: length xp-path = floor-log len
  for xp xp-path
proof -
  have hash-range-budget (0 + ?B)
    (read >>=
      (λxn.
        (ntimes read (floor-log len) ::
          (f list, (f, 'a) protocol-channel-scheme) state-monad) >>=
          (λxn-path.
            fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))

```

```

    by (rule hash-range-budget-bind
        [OF hash-range-budget-read after-xn-path[OF xp-path-len]])
  then show ?thesis by simp
qed
have after-xp-path:
  hash-range-budget ?B
  ((ntimes read (floor-log len) ::
    ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λxp-path.
      read >>=
      (λxn.
        (ntimes read (floor-log len) ::
          ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λxn-path.
            fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))))
  for xp
proof (rule hash-range-budget-ntimes-read-bind)
  fix xp-path :: 'f list
  assume xp-path-len: length xp-path = floor-log len
  show hash-range-budget ?B
    (read >>=
      (λxn.
        (ntimes read (floor-log len) ::
          ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λxn-path.
            fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))
  by (rule after-xn[OF xp-path-len])
qed
have hash-range-budget (0 + ?B)
  (read >>=
    (λxp.
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λxp-path.
          read >>=
          (λxn.
            (ntimes read (floor-log len) ::
              ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
              (λxn-path.
                fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))))))
  by (rule hash-range-budget-bind
      [OF hash-range-budget-read after-xp-path])
  then show ?thesis
    unfolding fri-layer-opening-step-unfold-finish by simp
qed

```

lemma hash-collision-budget-fri-layer-opening-step:
 hash-collision-budget (Suc (floor-log len) + Suc (floor-log len))
 (fri-layer-opening-step (b, f) (i, x, len, pw))

```

proof –
  let ?B = Suc (floor-log len) + Suc (floor-log len)
  have after-xn-path-range:
    hash-range-budget ?B
    ((ntimes read (floor-log len) ::
      ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λxn-path.
        fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))
  if xp-path-len: length xp-path = floor-log len
  for xp xp-path xn
proof (rule hash-range-budget-ntimes-read-bind)
  fix xn-path :: 'f list
  assume xn-path-len: length xn-path = floor-log len
  show hash-range-budget ?B
    (fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)
  by (rule hash-range-budget-fri-layer-opening-finish
    [OF xp-path-len xn-path-len])
qed
have after-xn-path-coll:
  hash-collision-budget ?B
  ((ntimes read (floor-log len) ::
    ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λxn-path.
      fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))
  if xp-path-len: length xp-path = floor-log len
  for xp xp-path xn
proof (rule hash-collision-budget-ntimes-read-bind)
  fix xn-path :: 'f list
  assume xn-path-len: length xn-path = floor-log len
  show hash-range-budget ?B
    (fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)
  by (rule hash-range-budget-fri-layer-opening-finish
    [OF xp-path-len xn-path-len])
  show hash-collision-budget ?B
    (fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)
  by (rule hash-collision-budget-fri-layer-opening-finish
    [OF xp-path-len xn-path-len])
qed
have after-xn-range:
  hash-range-budget ?B
  (read >>=
    (λxn.
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λxn-path.
          fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))
  if xp-path-len: length xp-path = floor-log len
  for xp xp-path
proof –

```

```

have hash-range-budget ( $0 + ?B$ )
  (read  $\gg$ 
    ( $\lambda xn.$ 
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
      ( $\lambda xn$ -path.
        fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
  by (rule hash-range-budget-bind
    [OF hash-range-budget-read after-xn-path-range[OF xp-path-len]])
then show ?thesis by simp
qed
have after-xn-coll:
  hash-collision-budget ?B
  (read  $\gg$ 
    ( $\lambda xn.$ 
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
      ( $\lambda xn$ -path.
        fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
  if xp-path-len: length xp-path = floor-log len
  for xp xp-path
proof –
  have hash-collision-budget ( $0 + ?B$ )
    (read  $\gg$ 
      ( $\lambda xn.$ 
        (ntimes read (floor-log len) ::
          ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda xn$ -path.
          fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
    by (rule hash-collision-budget-bind
      [OF hash-range-budget-read hash-collision-budget-read
        after-xn-path-range[OF xp-path-len]
        after-xn-path-coll[OF xp-path-len]])
    then show ?thesis by simp
  qed
have after-xp-path-range:
  hash-range-budget ?B
  ((ntimes read (floor-log len) ::
    ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda xp$ -path.
      read  $\gg$ 
      ( $\lambda xn.$ 
        (ntimes read (floor-log len) ::
          ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda xn$ -path.
          fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))
  for xp
proof (rule hash-range-budget-ntimes-read-bind)
  fix xp-path :: 'f list

```

```

assume xp-path-len: length xp-path = floor-log len
show hash-range-budget ?B
  (read  $\gg$ 
    ( $\lambda x n.$ 
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda x n\text{-path}.$ 
          fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
    by (rule after-xn-range[OF xp-path-len])
qed
have after-xp-path-coll:
  hash-collision-budget ?B
  ((ntimes read (floor-log len) ::
    ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda x p\text{-path}.$ 
      (read  $\gg$ 
        ( $\lambda x n.$ 
          (ntimes read (floor-log len) ::
            ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
            ( $\lambda x n\text{-path}.$ 
              fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))))
    for xp
proof (rule hash-collision-budget-ntimes-read-bind)
fix xp-path :: 'f list
assume xp-path-len: length xp-path = floor-log len
show hash-range-budget ?B
  (read  $\gg$ 
    ( $\lambda x n.$ 
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda x n\text{-path}.$ 
          fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
    by (rule after-xn-range[OF xp-path-len])
show hash-collision-budget ?B
  (read  $\gg$ 
    ( $\lambda x n.$ 
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda x n\text{-path}.$ 
          fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
    by (rule after-xn-coll[OF xp-path-len])
qed
have hash-collision-budget (0 + ?B)
  (read  $\gg$ 
    ( $\lambda x p.$ 
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda x p\text{-path}.$ 
          (read  $\gg$ 
            ( $\lambda x n.$ 
              (ntimes read (floor-log len) ::
                ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
                ( $\lambda x n\text{-path}.$ 
                  fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))))
            for xp
proof (rule hash-collision-budget-ntimes-read-bind)
fix xp-path :: 'f list
assume xp-path-len: length xp-path = floor-log len
show hash-range-budget ?B
  (read  $\gg$ 
    ( $\lambda x n.$ 
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda x n\text{-path}.$ 
          fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
    by (rule after-xn-range[OF xp-path-len])
show hash-collision-budget ?B
  (read  $\gg$ 
    ( $\lambda x n.$ 
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda x n\text{-path}.$ 
          fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
    by (rule after-xn-coll[OF xp-path-len])
qed
have hash-collision-budget (0 + ?B)
  (read  $\gg$ 
    ( $\lambda x p.$ 
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda x p\text{-path}.$ 
          (read  $\gg$ 
            ( $\lambda x n.$ 
              (ntimes read (floor-log len) ::
                ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
                ( $\lambda x n\text{-path}.$ 
                  fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))))
            for xp
proof (rule hash-collision-budget-ntimes-read-bind)
fix xp-path :: 'f list
assume xp-path-len: length xp-path = floor-log len
show hash-range-budget ?B
  (read  $\gg$ 
    ( $\lambda x n.$ 
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda x n\text{-path}.$ 
          fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
    by (rule after-xn-range[OF xp-path-len])
show hash-collision-budget ?B
  (read  $\gg$ 
    ( $\lambda x n.$ 
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda x n\text{-path}.$ 
          fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
    by (rule after-xn-coll[OF xp-path-len])
qed

```

```

    (λxn.
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λxn-path.
        fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))))
  by (rule hash-collision-budget-bind
    [OF hash-range-budget-read hash-collision-budget-read
      after-xp-path-range after-xp-path-coll])
  then show ?thesis
  unfolding fri-layer-opening-step-unfold-finish by simp
qed

```

lemma *floor-log-div2-le-self*:
 $\text{floor-log } (n \text{ div } 2) \leq \text{floor-log } n$
 by (rule *floor-log-le-iff*) simp

lemma *receive-query-commits-eq-fri-layer-opening-steps*:
 $\text{receive-query-commits fl} = \text{map fri-layer-opening-step fl}$
 unfolding *receive-query-commits-def* *fri-layer-opening-step-def*
 by (simp add: *fun-eq-iff* *split-def*)

lemma *hash-target-program-fri-layer-opening-steps-mfold-state*:

fixes $st :: \text{nat} \times 'f \times \text{nat} \times \text{nat}$
assumes *init*:
 $\text{case } st \text{ of } (i, x, \text{len}, pw) \Rightarrow \text{floor-log len} \leq L$

shows

$\text{hash-target-program } B$
 $(\text{length fl} * (\text{Suc } L + \text{Suc } L))$
 $(\text{mfold st } (\text{map fri-layer-opening-step fl}) ::$
 $(\text{nat} \times 'f \times \text{nat} \times \text{nat},$
 $('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad})$

using *init*

proof (*induction fl arbitrary: st*)

case *Nil*

then show ?*case*

by (*simp add: hash-target-program-return*)

next

case (*Cons bf fl*)

obtain $i \ x \ \text{len} \ pw$ **where** *st-eq*: $st = (i, x, \text{len}, pw)$

by (*cases st*)

obtain $b \ f$ **where** *bf-eq*: $bf = (b, f)$

by (*cases bf*)

let $?N = \text{Suc } L + \text{Suc } L$

have *head-exact*:

$\text{hash-target-program } B$
 $(\text{Suc } (\text{floor-log len}) + \text{Suc } (\text{floor-log len}))$
 $(\text{fri-layer-opening-step } (b, f) (i, x, \text{len}, pw))$
 by (*rule hash-target-program-fri-layer-opening-step*)

have *head-le*:

```

  Suc (floor-log len) + Suc (floor-log len) ≤ ?N
  using Cons.premis unfolding st-eq by simp
have head:
  hash-target-program B ?N
  (fri-layer-opening-step bf st ::
    (nat × 'f × nat × nat,
     ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding bf-eq st-eq
  by (rule hash-target-program-mono[OF head-le head-exact])
have tail:
  hash-target-program B (length fl * ?N)
  (mfold z (map fri-layer-opening-step fl) ::
    (nat × 'f × nat × nat,
     ('f, 'a) protocol-channel-scheme) state-monad)
  if out:
    Some (z, t) ∈
      set-dist (execute (fri-layer-opening-step bf st) s)
  for z and s :: ('f, 'a) protocol-channel-scheme
  and t :: ('f, 'a) protocol-channel-scheme
proof -
  from fri-layer-opening-step-outcome[OF out[unfolded bf-eq st-eq]]
  obtain xp xp-path xn xn-path x' where
    z-eq: z = (i mod (len div 2), x', len div 2, pw + pw)
  by blast
  have len-le: floor-log len ≤ L
    using Cons.premis unfolding st-eq by simp
  have next-len: floor-log (len div 2) ≤ L
    using floor-log-div2-le-self[of len] len-le by linarith
  have z-inv:
    case z of (i, x, len, pw) ⇒ floor-log len ≤ L
    unfolding z-eq using next-len by simp
  show ?thesis
    by (rule Cons.IH[OF z-inv])
qed
have hash-target-program B (?N + length fl * ?N)
  ((fri-layer-opening-step bf st ::
    (nat × 'f × nat × nat,
     ('f, 'a) protocol-channel-scheme) state-monad) ≫=
    (λz. mfold z (map fri-layer-opening-step fl)))
  by (rule hash-target-program-bind-on-outcomes[OF head]) (rule tail)
then show ?case
  by (simp add: add.assoc add.commute add.left-commute)
qed

```

lemma hash-target-program-receive-query-commits-mfold:

assumes init: floor-log len ≤ L

shows

hash-target-program B
 (length fl * (Suc L + Suc L))


```

    (mfold (i, x, len, pw) (receive-query-commits fl))
  unfolding receive-query-commits-eq-fri-layer-opening-steps
proof -
  have st-inv:
    case (i, x, len, pw) of
      (i, x, len, pw)  $\Rightarrow$  floor-log len  $\leq$  L
  using init by simp
  show hash-target-program B (length fl * (Suc L + Suc L))
    (mfold (i, x, len, pw) (map fri-layer-opening-step fl)) ::
      (nat  $\times$  'f  $\times$  nat  $\times$  nat,
        ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-target-program-fri-layer-opening-steps-mfold-state
      [OF st-inv])
qed

```

lemma hash-range-budget-fri-layer-opening-steps-mfold-state:

```

  fixes st :: nat  $\times$  'f  $\times$  nat  $\times$  nat
  assumes init:
    case st of (i, x, len, pw)  $\Rightarrow$  floor-log len  $\leq$  L
  shows
    hash-range-budget
      (length fl * (Suc L + Suc L))
      (mfold st (map fri-layer-opening-step fl)) ::
        (nat  $\times$  'f  $\times$  nat  $\times$  nat,
          ('f, 'a) protocol-channel-scheme) state-monad)
  using init
proof (induction fl arbitrary: st)
  case Nil
  then show ?case
    by (simp add: hash-range-budget-return)
next
  case (Cons bf fl)
  obtain i x len pw where st-eq: st = (i, x, len, pw)
  by (cases st)
  obtain b f where bf-eq: bf = (b, f)
  by (cases bf)
  let ?B = Suc L + Suc L
  have head-exact:
    hash-range-budget (Suc (floor-log len) + Suc (floor-log len))
      (fri-layer-opening-step (b, f) (i, x, len, pw))
  by (rule hash-range-budget-fri-layer-opening-step)
  have head-le:
    Suc (floor-log len) + Suc (floor-log len)  $\leq$  ?B
  using Cons.prem1 unfolding st-eq by simp
  have head:
    hash-range-budget ?B
      (fri-layer-opening-step bf st ::
        (nat  $\times$  'f  $\times$  nat  $\times$  nat,
          ('f, 'a) protocol-channel-scheme) state-monad)

```

```

unfolding bf-eq st-eq
by (rule hash-range-budget-mono[OF head-le head-exact])
have tail:
  hash-range-budget (length fl * ?B)
    (mfold z (map fri-layer-opening-step fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad)
if out:
  Some (z, t) ∈
    set-dist (execute (fri-layer-opening-step bf st) s)
for z and s :: ('f, 'a) protocol-channel-scheme
and t :: ('f, 'a) protocol-channel-scheme
proof –
from fri-layer-opening-step-outcome[OF out[unfolded bf-eq st-eq]]
obtain xp xp-path xn xn-path x' where
  z-eq: z = (i mod (len div 2), x', len div 2, pw + pw)
by blast
have len-le: floor-log len ≤ L
  using Cons.prems unfolding st-eq by simp
have next-len: floor-log (len div 2) ≤ L
  using floor-log-div2-le-self[of len] len-le by linarith
have z-inv:
  case z of (i, x, len, pw) ⇒ floor-log len ≤ L
  unfolding z-eq using next-len by simp
have ih-z:
  hash-range-budget (length fl * ?B)
    (mfold z (map fri-layer-opening-step fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad)
  using Cons.IH[OF z-inv] by assumption
show ?thesis
by (rule ih-z)
qed
have hash-range-budget (?B + length fl * ?B)
  ((fri-layer-opening-step bf st ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad) ≫=
    (λz. mfold z (map fri-layer-opening-step fl)))
  by (rule hash-range-budget-bind-on-outcomes[OF head]) (rule tail)
then show ?case
  by (simp add: add.assoc add.commute add.left-commute)
qed

lemma hash-range-budget-receive-query-commits-mfold:
assumes init: floor-log len ≤ L
shows
  hash-range-budget
    (length fl * (Suc L + Suc L))
    (mfold (i, x, len, pw) (receive-query-commits fl))

```

unfolding *receive-query-commits-eq-fri-layer-opening-steps*
proof –
have *st-inv*:
 case (*i*, *x*, *len*, *pw*) *of*
 (*i*, *x*, *len*, *pw*) $\Rightarrow$ *floor-log len* $\leq$ *L*
using *init* **by** *simp*
show *hash-range-budget* (*length fl* * (*Suc L* + *Suc L*))
 (*mfold* (*i*, *x*, *len*, *pw*) (*map fri-layer-opening-step fl*) ::
 (*nat* $\times$ '*f* $\times$ *nat* $\times$ *nat*,
 ('f, 'a) *protocol-channel-scheme*) *state-monad*)
by (*rule hash-range-budget-fri-layer-opening-steps-mfold-state*
 [*OF st-inv*])
qed

lemma *hash-collision-budget-fri-layer-opening-steps-mfold-state*:
fixes *st* :: *nat* $\times$ '*f* $\times$ *nat* $\times$ *nat*
assumes *init*:
 case st of (*i*, *x*, *len*, *pw*) $\Rightarrow$ *floor-log len* $\leq$ *L*
shows
 hash-collision-budget
 (*length fl* * (*Suc L* + *Suc L*))
 (*mfold st* (*map fri-layer-opening-step fl*) ::
 (*nat* $\times$ '*f* $\times$ *nat* $\times$ *nat*,
 ('f, 'a) *protocol-channel-scheme*) *state-monad*)
using *init*
proof (*induction fl arbitrary: st*)
case *Nil*
then show ?*case*
 by (*simp add: hash-collision-budget-return*)
next
case (*Cons bf fl*)
obtain *i x len pw* **where** *st-eq*: *st* = (*i*, *x*, *len*, *pw*)
by (*cases st*)
obtain *b f* **where** *bf-eq*: *bf* = (*b*, *f*)
by (*cases bf*)
let ?*B* = *Suc L* + *Suc L*
have *head-range-exact*:
 hash-range-budget (*Suc* (*floor-log len*) + *Suc* (*floor-log len*))
 (*fri-layer-opening-step* (*b*, *f*) (*i*, *x*, *len*, *pw*))
by (*rule hash-range-budget-fri-layer-opening-step*)
have *head-coll-exact*:
 hash-collision-budget (*Suc* (*floor-log len*) + *Suc* (*floor-log len*))
 (*fri-layer-opening-step* (*b*, *f*) (*i*, *x*, *len*, *pw*))
by (*rule hash-collision-budget-fri-layer-opening-step*)
have *head-le*:
 Suc (*floor-log len*) + *Suc* (*floor-log len*) $\leq$?*B*
using *Cons.prem*s **unfolding** *st-eq* **by** *simp*
have *head-range*:
 hash-range-budget ?*B*

```

    (fri-layer-opening-step bf st ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding bf-eq st-eq
  by (rule hash-range-budget-mono[OF head-le head-range-exact])
have head-coll:
  hash-collision-budget ?B
  (fri-layer-opening-step bf st ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding bf-eq st-eq
  by (rule hash-collision-budget-mono[OF head-le head-coll-exact])
have tail-range:
  hash-range-budget (length fl * ?B)
  (mfold z (map fri-layer-opening-step fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad)
  if out:
    Some (z, t) ∈
      set-dist (execute (fri-layer-opening-step bf st) s)
  for z and s :: ('f, 'a) protocol-channel-scheme
  and t :: ('f, 'a) protocol-channel-scheme
proof -
  from fri-layer-opening-step-outcome[OF out[unfolded bf-eq st-eq]]
  obtain xp xp-path xn xn-path x' where
    z-eq: z = (i mod (len div 2), x', len div 2, pw + pw)
  by blast
  have len-le: floor-log len ≤ L
    using Cons.premis unfolding st-eq by simp
  have next-len: floor-log (len div 2) ≤ L
    using floor-log-div2-le-self[of len] len-le by linarith
  have z-inv:
    case z of (i, x, len, pw) ⇒ floor-log len ≤ L
  unfolding z-eq using next-len by simp
  show ?thesis
    by (rule hash-range-budget-fri-layer-opening-steps-mfold-state
      [OF z-inv])
qed
have tail-coll:
  hash-collision-budget (length fl * ?B)
  (mfold z (map fri-layer-opening-step fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad)
  if out:
    Some (z, t) ∈
      set-dist (execute (fri-layer-opening-step bf st) s)
  for z and s :: ('f, 'a) protocol-channel-scheme
  and t :: ('f, 'a) protocol-channel-scheme
proof -

```

```

from fri-layer-opening-step-outcome[OF out[unfolded bf-eq st-eq]]
obtain xp xp-path xn xn-path x' where
  z-eq:  $z = (i \bmod (\text{len} \div 2), x', \text{len} \div 2, pw + pw)$ 
  by blast
have len-le: floor-log len  $\leq L$ 
  using Cons.premis unfolding st-eq by simp
have next-len: floor-log (len div 2)  $\leq L$ 
  using floor-log-div2-le-self[of len] len-le by linarith
have z-inv:
  case z of (i, x, len, pw)  $\Rightarrow$  floor-log len  $\leq L$ 
  unfolding z-eq using next-len by simp
have ih-z:
  hash-collision-budget (length fl * ?B)
  (mfold z (map fri-layer-opening-step fl) ::
    (nat  $\times$  'f  $\times$  nat  $\times$  nat,
      ('f, 'a) protocol-channel-scheme) state-monad)
  using Cons.IH[OF z-inv] by assumption
show ?thesis
  by (rule ih-z)
qed
have hash-collision-budget (?B + length fl * ?B)
  ((fri-layer-opening-step bf st ::
    (nat  $\times$  'f  $\times$  nat  $\times$  nat,
      ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda z.$  mfold z (map fri-layer-opening-step fl))))
  by (rule hash-collision-budget-bind-on-outcomes
    [OF head-range head-coll tail-range tail-coll])
then show ?case
  by (simp add: add.assoc add.commute add.left-commute)
qed

lemma hash-collision-budget-receive-query-commits-mfold:
assumes init: floor-log len  $\leq L$ 
shows
  hash-collision-budget
    (length fl * (Suc L + Suc L))
    (mfold (i, x, len, pw) (receive-query-commits fl))
unfolding receive-query-commits-eq-fri-layer-opening-steps
proof –
have st-inv:
  case (i, x, len, pw) of
    (i, x, len, pw)  $\Rightarrow$  floor-log len  $\leq L$ 
  using init by simp
show hash-collision-budget (length fl * (Suc L + Suc L))
  (mfold (i, x, len, pw) (map fri-layer-opening-step fl) ::
    (nat  $\times$  'f  $\times$  nat  $\times$  nat,
      ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-collision-budget-fri-layer-opening-steps-mfold-state
    [OF st-inv])

```

qed

lemma *hash-target-program-query-decommitment-step:*

*hash-target-program B (Suc (floor-log (scale * clength)))*
(query-decommitment-step fr i)

proof –

let ?len = scale * clength

have after-path:

hash-target-program B (Suc (floor-log ?len))
 ((*ntimes read (floor-log ?len) ::*
 (*'f list, ('f, 'a) protocol-channel-scheme) state-monad*) $\gg$
 (λ path.
 (*check-authentication-path ?len i qh path ::*
 (*'f, ('f, 'a) protocol-channel-scheme) state-monad*) $\gg$
 (λ ap. assert (ap = fr) $\gg$ (λ -. return qh))))

for qh

proof (rule *hash-target-program-ntimes-read-bind*)

fix path :: 'f list

assume path-len: length path = floor-log ?len

have *hash-target-program B (Suc (length path))*

((*check-authentication-path ?len i qh path ::*
 (*'f, ('f, 'a) protocol-channel-scheme) state-monad*) $\gg$
 (λ ap. assert (ap = fr) $\gg$ (λ -. return qh)))

by (rule *hash-target-program-check-authentication-path-assert-return*)

then show *hash-target-program B (Suc (floor-log ?len))*

((*check-authentication-path ?len i qh path ::*
 (*'f, ('f, 'a) protocol-channel-scheme) state-monad*) $\gg$
 (λ ap. assert (ap = fr) $\gg$ (λ -. return qh)))

by (subst path-len[symmetric])

qed

have *hash-target-program B (0 + Suc (floor-log ?len))*

(*read* $\gg$
 (λ qh. (*ntimes read (floor-log ?len) ::*
 (*'f list, ('f, 'a) protocol-channel-scheme) state-monad*) $\gg$
 (λ path.
 (*check-authentication-path ?len i qh path ::*
 (*'f, ('f, 'a) protocol-channel-scheme) state-monad*) $\gg$
 (λ ap. assert (ap = fr) $\gg$ (λ -. return qh))))))

by (rule *hash-target-program-bind*

[OF *hash-target-program-read after-path*])

then show ?thesis

unfolding *query-decommitment-step-def* by (simp add: Let-def)

qed

lemma *hash-target-program-check-decommit-on-query:*

hash-target-program B
*(powers * Suc (floor-log (scale * clength)))*
(mmap (check-decommit-on-query fr idx))

proof –

```

let ?b = Suc (floor-log (scale * clength))
let ?steps =
  (map (query-decommitment-step fr) (powers-scaled idx) ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad list)
have step-budget:
   $\bigwedge (m :: ('f, ('f, 'a) protocol-channel-scheme) state-monad).$ 
   $m \in \text{set } ?steps \implies$ 
  hash-target-program B ?b m
proof -
  fix m :: ('f, ('f, 'a) protocol-channel-scheme) state-monad
  assume m  $\in$  set ?steps
  then obtain i where i  $\in$  set (powers-scaled idx)
  and m-eq: m = query-decommitment-step fr i
  by auto
  show hash-target-program B ?b m
  unfolding m-eq by (rule hash-target-program-query-decommitment-step)
qed
have map-budget:
  hash-target-program B (length ?steps * ?b) (mmap ?steps)
  by (rule hash-target-program-mmap) (rule step-budget)
then have hash-target-program B
  (length (powers-scaled idx) * ?b) (mmap ?steps)
  by simp
then show ?thesis
  unfolding check-decommit-on-query-def query-decommitment-step-def
  powers-scaled-def
  by simp
qed

lemma hash-range-budget-query-decommitment-step:
  hash-range-budget (Suc (floor-log (scale * clength)))
  (query-decommitment-step fr i)
proof -
  let ?len = scale * clength
  have after-path:
  hash-range-budget (Suc (floor-log ?len))
  ((ntimes read (floor-log ?len) ::
    ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
    ( $\lambda$ path.
      (check-authentication-path ?len i qh path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
        ( $\lambda$ ap. assert (ap = fr)  $\gg=$  ( $\lambda$ -. return qh)))) for qh
proof (rule hash-range-budget-ntimes-read-bind)
  fix path :: 'f list
  assume path-len: length path = floor-log ?len
  have hash-range-budget (Suc (length path))
  (((check-authentication-path ?len i qh path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
    ( $\lambda$ ap. assert (ap = fr)  $\gg=$  ( $\lambda$ -. return qh))))

```

```

    by (rule hash-range-budget-check-authentication-path-assert-return)
  note path-budget = this
  show hash-range-budget (Suc (floor-log ?len))
    ((check-authentication-path ?len i qh path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = fr) >>= (λ-. return qh)))
    by (subst path-len[symmetric]) (rule path-budget)
  qed
  have hash-range-budget (0 + Suc (floor-log ?len))
    (read >>=
      (λqh. (ntimes read (floor-log ?len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λpath.
          (check-authentication-path ?len i qh path ::
            ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
            (λap. assert (ap = fr) >>= (λ-. return qh))))))
    by (rule hash-range-budget-bind[OF hash-range-budget-read after-path])
  then show ?thesis
    unfolding query-decommitment-step-def by (simp add: Let-def)
  qed

```

lemma hash-collision-budget-query-decommitment-step:
 hash-collision-budget (Suc (floor-log (scale * clength)))
 (query-decommitment-step fr i)

```

proof -
  let ?len = scale * clength
  have after-path-range:
    hash-range-budget (Suc (floor-log ?len))
      ((ntimes read (floor-log ?len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λpath.
          (check-authentication-path ?len i qh path ::
            ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
            (λap. assert (ap = fr) >>= (λ-. return qh)))))) for qh
  proof (rule hash-range-budget-ntimes-read-bind)
    fix path :: 'f list
    assume path-len: length path = floor-log ?len
    have hash-range-budget (Suc (length path))
      (((check-authentication-path ?len i qh path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = fr) >>= (λ-. return qh))))
      by (rule hash-range-budget-check-authentication-path-assert-return)
    note path-budget = this
    show hash-range-budget (Suc (floor-log ?len))
      ((check-authentication-path ?len i qh path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = fr) >>= (λ-. return qh)))
      by (subst path-len[symmetric]) (rule path-budget)
  qed

```



```

have after-path-coll:
  hash-collision-budget (Suc (floor-log ?len))
    ((ntimes read (floor-log ?len) ::
      ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λpath.
        (check-authentication-path ?len i qh path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λap. assert (ap = fr) >>= (λ-. return qh)))) for qh
proof (rule hash-collision-budget-ntimes-read-bind)
  fix path :: 'f list
  assume path-len: length path = floor-log ?len
  have range-bind:
    hash-range-budget (Suc (length path))
      (((check-authentication-path ?len i qh path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = fr) >>= (λ-. return qh))))
      by (rule hash-range-budget-check-authentication-path-assert-return)
  show hash-range-budget (Suc (floor-log ?len))
    ((check-authentication-path ?len i qh path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = fr) >>= (λ-. return qh)))
    by (subst path-len[symmetric]) (rule range-bind)
  have hash-collision-budget (Suc (length path))
    (((check-authentication-path ?len i qh path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = fr) >>= (λ-. return qh))))
    by (rule hash-collision-budget-check-authentication-path-assert-return)
  note path-budget = this
  show hash-collision-budget (Suc (floor-log ?len))
    ((check-authentication-path ?len i qh path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = fr) >>= (λ-. return qh)))
    by (subst path-len[symmetric]) (rule path-budget)
qed
have hash-collision-budget (0 + Suc (floor-log ?len))
  (read >>=
    (λqh. (ntimes read (floor-log ?len) ::
      ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λpath.
        (check-authentication-path ?len i qh path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λap. assert (ap = fr) >>= (λ-. return qh))))))
  by (rule hash-collision-budget-bind
    [OF hash-range-budget-read hash-collision-budget-read
      after-path-range after-path-coll])
then show ?thesis
  unfolding query-decommitment-step-def by (simp add: Let-def)
qed

```

lemma *hash-range-budget-check-decommit-on-query:*
hash-range-budget
 (*powers* * *Suc* (*floor-log* (*scale* * *clength*)))
 (*mmap* (*check-decommit-on-query* *fr* *idx*))
proof –
let *?b* = *Suc* (*floor-log* (*scale* * *clength*))
let *?steps* =
 (*map* (*query-decommitment-step* *fr*) (*powers-scaled idx*) ::
 (*'f*, (*'f*, *'a*) *protocol-channel-scheme*) *state-monad list*)
have *step-budget:*
 $\bigwedge (m :: ('f, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}).$
 $m \in \text{set } ?steps \implies$
hash-range-budget *?b* *m*
proof –
fix *m* :: (*'f*, (*'f*, *'a*) *protocol-channel-scheme*) *state-monad*
assume $m \in \text{set } ?steps$
then obtain *i* **where** $i \in \text{set } (\text{powers-scaled } idx)$
and *m-eq*: $m = \text{query-decommitment-step } fr \ i$
by *auto*
show *hash-range-budget* *?b* *m*
unfolding *m-eq* **by** (*rule* *hash-range-budget-query-decommitment-step*)
qed
have *map-budget:*
hash-range-budget
 (*length* *?steps* * *?b*) (*mmap* *?steps*)
by (*rule* *hash-range-budget-mmap*) (*rule* *step-budget*)
then have *hash-range-budget* (*length* (*powers-scaled idx*) * *?b*)
 (*mmap* *?steps*)
by *simp*
then show *?thesis*
unfolding *check-decommit-on-query-def* *query-decommitment-step-def*
powers-scaled-def
by *simp*
qed

lemma *hash-collision-budget-check-decommit-on-query:*
hash-collision-budget
 (*powers* * *Suc* (*floor-log* (*scale* * *clength*)))
 (*mmap* (*check-decommit-on-query* *fr* *idx*))
proof –
let *?b* = *Suc* (*floor-log* (*scale* * *clength*))
let *?steps* =
 (*map* (*query-decommitment-step* *fr*) (*powers-scaled idx*) ::
 (*'f*, (*'f*, *'a*) *protocol-channel-scheme*) *state-monad list*)
have *step-range:*
 $\bigwedge (m :: ('f, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}).$
 $m \in \text{set } ?steps \implies$
hash-range-budget *?b* *m*
proof –

```

fix  $m :: ('f, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$ 
assume  $m \in \text{set } ?\text{steps}$ 
then obtain  $i$  where  $i \in \text{set } (\text{powers-scaled idx})$ 
  and  $m\text{-eq}: m = \text{query-decommitment-step fr } i$ 
  by auto
show  $\text{hash-range-budget } ?b \ m$ 
  unfolding  $m\text{-eq}$  by  $(\text{rule hash-range-budget-query-decommitment-step})$ 
qed
have  $\text{step-coll}:$ 
 $\bigwedge (m :: ('f, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}).$ 
 $m \in \text{set } ?\text{steps} \implies$ 
 $\text{hash-collision-budget } ?b \ m$ 
proof –
  fix  $m :: ('f, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$ 
assume  $m \in \text{set } ?\text{steps}$ 
then obtain  $i$  where  $i \in \text{set } (\text{powers-scaled idx})$ 
  and  $m\text{-eq}: m = \text{query-decommitment-step fr } i$ 
  by auto
show  $\text{hash-collision-budget } ?b \ m$ 
  unfolding  $m\text{-eq}$  by  $(\text{rule hash-collision-budget-query-decommitment-step})$ 
qed
have  $\text{map-budget}:$ 
 $\text{hash-collision-budget}$ 
 $(\text{length } ?\text{steps} * ?b) (\text{mmap } ?\text{steps})$ 
apply  $(\text{rule hash-collision-budget-mmap})$ 
apply  $(\text{rule step-range, assumption})$ 
apply  $(\text{rule step-coll, assumption})$ 
done
then have  $\text{hash-collision-budget } (\text{length } (\text{powers-scaled idx}) * ?b)$ 
 $(\text{mmap } ?\text{steps})$ 
by simp
then show  $?thesis$ 
unfolding  $\text{check-decommit-on-query-def query-decommitment-step-def}$ 
 $\text{powers-scaled-def}$ 
by simp
qed

end

end

```

```

theory Soundness-Oracle-Verifier
imports Soundness-Oracle-Budgets
begin

```

Concrete verifier hash-query budget bounds.

```

context soundness
begin

```

```

lemma ceil-log-mono:
  assumes  $m \leq n$ 
  shows  $\text{ceil-log } m \leq \text{ceil-log } n$ 
proof (cases  $m \leq 1$ )
  case True
  then show ?thesis
    unfolding ceil-log-def by simp
next
  case False
  then have m-pos:  $0 < m$  and m-minus:  $m - 1 \leq n - 1$ 
    using assms by simp-all
  have n-gt:  $1 < n$ 
    using False assms by linarith
  have  $\text{floor-log } (m - 1) \leq \text{floor-log } (n - 1)$ 
    using floor-log-le-iff m-minus by blast
  then show ?thesis
    using False n-gt unfolding ceil-log-def by simp
qed

definition verifier-fri-layer-hash-budget :: nat
  where verifier-fri-layer-hash-budget =
     $\text{Suc } (\text{floor-log } (\text{clength} * \text{scale})) + \text{Suc } (\text{floor-log } (\text{clength} * \text{scale}))$ 

definition verifier-query-decommit-hash-budget :: nat
  where verifier-query-decommit-hash-budget =
     $\text{powers} * \text{Suc } (\text{floor-log } (\text{clength} * \text{scale}))$ 

definition verifier-query-round-hash-budget :: nat
  where verifier-query-round-hash-budget =
     $1 + \text{verifier-query-decommit-hash-budget} +$ 
     $(\text{ceil-log } \text{clength} + \text{ceil-log } (\text{maxDegree} + 1)) * \text{verifier-fri-layer-hash-budget}$ 

definition verifier-header-hash-budget :: nat
  where verifier-header-hash-budget =
     $\text{ceil-log } \text{clength} + \text{length spec} + \text{ceil-log } (\text{maxDegree} + 1)$ 

definition verifier-hash-query-budget :: nat
  where verifier-hash-query-budget =
     $\text{verifier-header-hash-budget} + \text{rounds} * \text{verifier-query-round-hash-budget}$ 

definition concrete-merkle-binding-error :: prob
  where
    concrete-merkle-binding-error =
       $\text{hash-collision-budget-value } 0 \text{ verifier-hash-query-budget}$ 

definition concrete-transcript-target-error :: 'f list  $\Rightarrow$  prob'
  where

```

concrete-transcript-target-error $tr =$
 $nnreal \ (card \ (set \ tr) * verifier\text{-}hash\text{-}query\text{-}budget) / nnreal \ size$

lemma *hash-target-program-alpha-challenge-step:*

hash-target-program $B \ 1$

```
(do {
  a0 ← receive-alpha-challenge;
  let a0' = a0;
  a1 ← read;
  let a1' = a1;
  assert (a0' = a1');
  return a1'
})
```

proof –

have *assert-return:*

```
hash-target-program  $B \ (0 + 0)$ 
(assert (a0 = a1) >>= (λ-. return a1)) ::
('f, ('f, 'a) protocol-channel-scheme) state-monad
for a0 a1
```

```
by (rule hash-target-program-bind
[OF hash-target-program-assert hash-target-program-return])
```

have *after-read:*

```
hash-target-program  $B \ (0 + 0 + 0)$ 
(read >>= (λa1. assert (a0 = a1) >>= (λ-. return a1))) ::
('f, ('f, 'a) protocol-channel-scheme) state-monad
for a0
```

proof –

```
have hash-target-program  $B \ (0 + (0 + 0))$ 
(read >>= (λa1. assert (a0 = a1) >>= (λ-. return a1))) ::
('f, ('f, 'a) protocol-channel-scheme) state-monad
by (rule hash-target-program-bind
[OF hash-target-program-read assert-return])
```

then show *?thesis* **by** *simp*

qed

have *hash-target-program* $B \ (1 + (0 + 0 + 0))$

```
(receive-alpha-challenge >>=
(λa0. read >>= (λa1.
  assert (a0 = a1) >>= (λ-. return a1)))) ::
('f, ('f, 'a) protocol-channel-scheme) state-monad
```

```
by (rule hash-target-program-bind
[OF hash-target-program-receive-alpha-challenge after-read])
```

then show *?thesis*

by *simp*

qed

lemma *hash-range-budget-alpha-challenge-step:*

hash-range-budget 1

```
(do {
  a0 ← receive-alpha-challenge;
```

```

    let a0' = a0;
    a1 ← read;
    let a1' = a1;
    assert (a0' = a1');
    return a1'
  })
proof –
  have assert-return:
    hash-range-budget (0 + 0)
    (assert (a0 = a1) >>= (λ-. return a1)) ::
      (f, (f, 'a) protocol-channel-scheme) state-monad for a0 a1
  by (rule hash-range-budget-bind
    [OF hash-range-budget-assert hash-range-budget-return])
  have after-read:
    hash-range-budget (0 + 0 + 0)
    (read >>= (λa1. assert (a0 = a1) >>= (λ-. return a1))) ::
      (f, (f, 'a) protocol-channel-scheme) state-monad for a0
  proof –
    have hash-range-budget (0 + (0 + 0)):
      (read >>= (λa1. assert (a0 = a1) >>= (λ-. return a1))) ::
        (f, (f, 'a) protocol-channel-scheme) state-monad
    by (rule hash-range-budget-bind
      [OF hash-range-budget-read assert-return])
    then show ?thesis by simp
  qed
  have hash-range-budget (1 + (0 + 0 + 0)):
    (receive-alpha-challenge >>=
      (λa0. read >>= (λa1.
        assert (a0 = a1) >>= (λ-. return a1)))) ::
      (f, (f, 'a) protocol-channel-scheme) state-monad
  by (rule hash-range-budget-bind
    [OF hash-range-budget-receive-alpha-challenge after-read])
  then show ?thesis
  by simp
qed

lemma hash-collision-budget-alpha-challenge-step:
  hash-collision-budget 1
  (do {
    a0 ← receive-alpha-challenge;
    let a0' = a0;
    a1 ← read;
    let a1' = a1;
    assert (a0' = a1');
    return a1'
  })
proof –
  have assert-return-range:
    hash-range-budget (0 + 0)

```

```

    (assert (a0 = a1) >>= (λ-. return a1)) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) for a0 a1
  by (rule hash-range-budget-bind
      [OF hash-range-budget-assert hash-range-budget-return])
  have assert-return-coll:
    hash-collision-budget (0 + 0)
    (assert (a0 = a1) >>= (λ-. return a1)) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) for a0 a1
  by (rule hash-collision-budget-bind
      [OF hash-range-budget-assert hash-collision-budget-assert
          hash-range-budget-return hash-collision-budget-return])
  have after-read-range:
    hash-range-budget (0 + 0 + 0)
    (read >>= (λa1. assert (a0 = a1) >>= (λ-. return a1))) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) for a0
  proof -
    have hash-range-budget (0 + (0 + 0))
      (read >>= (λa1. assert (a0 = a1) >>= (λ-. return a1))) ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-range-budget-bind
        [OF hash-range-budget-read assert-return-range])
    then show ?thesis by simp
  qed
  have after-read-coll:
    hash-collision-budget (0 + 0 + 0)
    (read >>= (λa1. assert (a0 = a1) >>= (λ-. return a1))) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) for a0
  proof -
    have hash-collision-budget (0 + (0 + 0))
      (read >>= (λa1. assert (a0 = a1) >>= (λ-. return a1))) ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-collision-budget-bind
        [OF hash-range-budget-read hash-collision-budget-read
            assert-return-range assert-return-coll])
    then show ?thesis by simp
  qed
  have hash-collision-budget (1 + (0 + 0 + 0))
    (receive-alpha-challenge >>=
      (λa0. read >>= (λa1.
        assert (a0 = a1) >>= (λ-. return a1)))) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-collision-budget-bind
      [OF hash-range-budget-receive-alpha-challenge
          hash-collision-budget-receive-alpha-challenge
          after-read-range after-read-coll])
  then show ?thesis
  by simp
qed

```

```

lemma hash-target-program-verifier-query-round-program-exact:
  hash-target-program B
    (1 + verifier-query-decommit-hash-budget +
     (length f-fl + length fl) * verifier-fri-layer-hash-budget)
    (verifier-query-round-program fr f-fl f-final as fl final)
proof -
  let ?L = floor-log (clength * scale)
  let ?B = verifier-fri-layer-hash-budget
  have B-eq: ?B = Suc ?L + Suc ?L
    unfolding verifier-fri-layer-hash-budget-def by simp
  have after-comp:
    hash-target-program B (length fl * ?B + 0)
      ((mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
         ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final))))
    for idx fv
  proof (rule hash-target-program-bind)
  show hash-target-program B (length fl * ?B)
    (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
       ('f, 'a) protocol-channel-scheme) state-monad)
    unfolding B-eq
    by (rule hash-target-program-receive-query-commits-mfold) simp
  show ∧x. hash-target-program B 0
    ((case x of (i, x, len, pw) ⇒ assert (x = final)) ::
     (unit, ('f, 'a) protocol-channel-scheme) state-monad)
    by (simp add: hash-target-program-assert split: prod.splits)
  qed
  have after-trace-assert:
    hash-target-program B (0 + (length fl * ?B + 0))
      ((assert (f-x = f-final) ::
        (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ-. (mfold
          (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
          (receive-query-commits fl) ::
          (nat × 'f × nat × nat,
           ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λ(i, x, len, pw). assert (x = final)))))
    for idx fv f-i f-x f-len f-pow
  by (rule hash-target-program-bind
    [OF hash-target-program-assert after-comp])
  have after-trace:
    hash-target-program B
      (length f-fl * ?B + (0 + (length fl * ?B + 0)))

```



```

((mfold (idx, hd fv, clength * scale, 1)
  (receive-query-commits f-fl) ::
  (nat × 'f × nat × nat,
  ('f, 'a) protocol-channel-scheme) state-monad) >>=
(λ(f-i, f-x, f-len, f-pow).
  assert (f-x = f-final) >>=
  (λ-. (mfold
    (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat × 'f × nat × nat,
    ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ(i, x, len, pw). assert (x = final))))))
for idx fv
proof (rule hash-target-program-bind)
show hash-target-program B (length f-fl * ?B)
  (mfold (idx, hd fv, clength * scale, 1)
    (receive-query-commits f-fl) ::
    (nat × 'f × nat × nat,
    ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding B-eq
  by (rule hash-target-program-receive-query-commits-mfold) simp
fix x :: nat × 'f × nat × nat
obtain f-i f-x f-len f-pow where x-eq:
  x = (f-i, f-x, f-len, f-pow)
  by (cases x) auto
have case-budget:
  hash-target-program B (0 + (length fl * ?B + 0))
  ((assert (f-x = f-final) ::
  (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
  (λ-. (mfold
    (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat × 'f × nat × nat,
    ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ(i, x, len, pw). assert (x = final))))))
  by (rule hash-target-program-bind
    [OF hash-target-program-assert after-comp])
show hash-target-program B (0 + (length fl * ?B + 0))
  ((case x of (f-i, f-x, f-len, f-pow) ⇒
  assert (f-x = f-final) >>=
  (λ-. (mfold
    (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat × 'f × nat × nat,
    ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ(i, x, len, pw). assert (x = final)))))) ::
  (unit, ('f, 'a) protocol-channel-scheme) state-monad)
using case-budget
unfolding x-eq

```



```

    (λ-. (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) ≫=
      (λ(i, x, len, pw). assert (x = final))))))
  for idx
  using after-query[of idx] by simp
have after-random:
  hash-target-program B
  (verifier-query-decommit-hash-budget +
    (length f-fl * ?B + length fl * ?B))
  ((let idx' = index (to-nat idx) in
    mmap (check-decommit-on-query fr idx') ≫=
    (λfv. (mfold (idx', hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) ≫=
      (λ(f-i, f-x, f-len, f-pow).
        assert (f-x = f-final) ≫=
        (λ-. (mfold
          (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
          (receive-query-commits fl) ::
          (nat × 'f × nat × nat,
            ('f, 'a) protocol-channel-scheme) state-monad) ≫=
          (λ(i, x, len, pw). assert (x = final)))))))
    for idx
    using after-query-simple[of index (to-nat idx)]
    by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
  have hash-target-program B
    (1 + (verifier-query-decommit-hash-budget +
      (length f-fl * ?B + length fl * ?B)))
    (receive-query-index-challenge ≫=
      (λidx.
        let idx' = index (to-nat idx) in
        mmap (check-decommit-on-query fr idx') ≫=
        (λfv. (mfold (idx', hd fv, clength * scale, 1)
          (receive-query-commits f-fl) ::
          (nat × 'f × nat × nat,
            ('f, 'a) protocol-channel-scheme) state-monad) ≫=
          (λ(f-i, f-x, f-len, f-pow).
            assert (f-x = f-final) ≫=
            (λ-. (mfold
              (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
              (receive-query-commits fl) ::
              (nat × 'f × nat × nat,
                ('f, 'a) protocol-channel-scheme) state-monad) ≫=
              (λ(i, x, len, pw). assert (x = final)))))))
        by (rule hash-target-program-bind

```

```

    [OF hash-target-program-receive-query-index-challenge])
  (rule after-random)
then show ?thesis
  unfolding verifier-query-round-program-def
  by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
qed

```

lemma *hash-range-budget-verifier-query-round-program-exact:*

```

hash-range-budget
  (1 + verifier-query-decommit-hash-budget +
    (length f-fl + length fl) * verifier-fri-layer-hash-budget)
  (verifier-query-round-program fr f-fl f-final as fl final)

```

proof –

```

let ?L = floor-log (clength * scale)
let ?B = verifier-fri-layer-hash-budget
have B-eq: ?B = Suc ?L + Suc ?L
  unfolding verifier-fri-layer-hash-budget-def by simp
have after-comp:
  hash-range-budget (length fl * ?B + 0)
    ((mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) ≫=
      (λ(i, x, len, pw). assert (x = final))))

```

for *idx fv*

```

proof (rule hash-range-budget-bind)
show hash-range-budget (length fl * ?B)
  (mfold
    (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding B-eq
  by (rule hash-range-budget-receive-query-commits-mfold) simp
show ∧x. hash-range-budget 0
  ((case x of (i, x, len, pw) ⇒ assert (x = final)) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  by (simp add: hash-range-budget-assert split: prod.splits)

```

qed

have *after-trace-assert:*

```

hash-range-budget (0 + (length fl * ?B + 0))
  ((assert (f-x = f-final) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad) ≫=
  (λ-. (mfold
    (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat × 'f × nat × nat,

```

```

      ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(i, x, len, pw). assert (x = final))))
for idx fv f-i f-x f-len f-pow
by (rule hash-range-budget-bind
    [OF hash-range-budget-assert after-comp])
have after-trace:
  hash-range-budget
    (length f-fl * ?B + (0 + (length fl * ?B + 0)))
    ((mfold (idx, hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(f-i, f-x, f-len, f-pow).
      assert (f-x = f-final) >>=
      (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final))))))
for idx fv
proof (rule hash-range-budget-bind)
show hash-range-budget (length f-fl * ?B)
  (mfold (idx, hd fv, clength * scale, 1)
    (receive-query-commits f-fl) ::
    (nat × 'f × nat × nat,
    ('f, 'a) protocol-channel-scheme) state-monad)
unfolding B-eq
by (rule hash-range-budget-receive-query-commits-mfold) simp
fix x :: nat × 'f × nat × nat
obtain f-i f-x f-len f-pow where x-eq:
  x = (f-i, f-x, f-len, f-pow)
by (cases x) auto
have case-budget:
  hash-range-budget (0 + (length fl * ?B + 0))
    ((assert (f-x = f-final) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ-. (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(i, x, len, pw). assert (x = final))))))
by (rule hash-range-budget-bind
    [OF hash-range-budget-assert after-comp])
show hash-range-budget (0 + (length fl * ?B + 0))
  ((case x of (f-i, f-x, f-len, f-pow) ⇒
    assert (f-x = f-final) >>=
    (λ-. (mfold

```

```

      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(i, x, len, pw). assert (x = final)))) ::
      (unit, ('f, 'a) protocol-channel-scheme) state-monad)
using case-budget
unfolding x-eq
by (simp split: prod.splits)
qed
have after-query:
  hash-range-budget
    (verifier-query-decommit-hash-budget +
      (length f-fl * ?B + (0 + (length fl * ?B + 0))))
    (mmap (check-decommit-on-query fr idx) >>=
      (λfv. (mfold (idx, hd fv, clength * scale, 1)
        (receive-query-commits f-fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(f-i, f-x, f-len, f-pow).
          assert (f-x = f-final) >>=
            (λ-. (mfold
              (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
              (receive-query-commits fl) ::
              (nat × 'f × nat × nat,
                ('f, 'a) protocol-channel-scheme) state-monad) >>=
              (λ(i, x, len, pw). assert (x = final)))))))
    for idx
proof -
  have decommit:
    hash-range-budget verifier-query-decommit-hash-budget
      (mmap (check-decommit-on-query fr idx))
  proof -
    have exact:
      hash-range-budget
        (powers * Suc (floor-log (scale * clength)))
        (mmap (check-decommit-on-query fr idx))
      by (rule hash-range-budget-check-decommit-on-query)
    show ?thesis
      using exact
      unfolding verifier-query-decommit-hash-budget-def
      by (simp add: mult.commute)
  qed
  show ?thesis
    by (rule hash-range-budget-bind[OF decommit after-trace])
qed
have after-query-simple:
  hash-range-budget
    (verifier-query-decommit-hash-budget +

```



```

    assert (f-x = f-final) >>=
    (λ-. (mfold
      (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(i, x, len, pw). assert (x = final))))))
  by (rule hash-range-budget-bind
    [OF hash-range-budget-receive-query-index-challenge])
    (rule after-random)
  then show ?thesis
    unfolding verifier-query-round-program-def
    by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
qed

```

lemma *hash-target-program-verifier-query-round-program:*

assumes *len-f: length f-fl ≤ ceil-log clength*
and *len-c: length fl ≤ ceil-log (maxDegree + 1)*

shows

hash-target-program B verifier-query-round-hash-budget
(verifier-query-round-program fr f-fl f-final as fl final)

proof –

let *?N = verifier-fri-layer-hash-budget*

have *exact:*

hash-target-program B
(1 + verifier-query-decommit-hash-budget +
*(length f-fl + length fl) * ?N)*
(verifier-query-round-program fr f-fl f-final as fl final)

by (rule *hash-target-program-verifier-query-round-program-exact*)

have *le:*

1 + verifier-query-decommit-hash-budget +
*(length f-fl + length fl) * ?N*
≤ verifier-query-round-hash-budget

using *len-f len-c*

unfolding *verifier-query-round-hash-budget-def*

by *simp*

show *?thesis*

by (rule *hash-target-program-mono*[OF *le exact*])

qed

lemma *hash-range-budget-verifier-query-round-program:*

assumes *len-f: length f-fl ≤ ceil-log clength*
and *len-c: length fl ≤ ceil-log (maxDegree + 1)*

shows

hash-range-budget verifier-query-round-hash-budget
(verifier-query-round-program fr f-fl f-final as fl final)

proof –

let *?B = verifier-fri-layer-hash-budget*

have *exact:*


```

hash-range-budget
  (1 + verifier-query-decommit-hash-budget +
   (length f-fl + length fl) * ?B)
  (verifier-query-round-program fr f-fl f-final as fl final)
by (rule hash-range-budget-verifier-query-round-program-exact)
have le:
  1 + verifier-query-decommit-hash-budget +
    (length f-fl + length fl) * ?B
  ≤ verifier-query-round-hash-budget
using len-f len-c
unfolding verifier-query-round-hash-budget-def
by simp
show ?thesis
  by (rule hash-range-budget-mono[OF le exact])
qed

```

lemma hash-collision-budget-verifier-query-round-program-exact:

```

hash-collision-budget
  (1 + verifier-query-decommit-hash-budget +
   (length f-fl + length fl) * verifier-fri-layer-hash-budget)
  (verifier-query-round-program fr f-fl f-final as fl final)
proof -
  let ?L = floor-log (clength * scale)
  let ?B = verifier-fri-layer-hash-budget
  have B-eq: ?B = Suc ?L + Suc ?L
    unfolding verifier-fri-layer-hash-budget-def by simp
  have after-comp-range:
    hash-range-budget (length fl * ?B + 0)
    ((mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
       ('f, 'a) protocol-channel-scheme) state-monad) ≫=
      (λ(i, x, len, pw). assert (x = final))))
    for idx fv
  proof (rule hash-range-budget-bind)
    show hash-range-budget (length fl * ?B)
      (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
         ('f, 'a) protocol-channel-scheme) state-monad)
      unfolding B-eq
      by (rule hash-range-budget-receive-query-commits-mfold) simp
    show ∧x. hash-range-budget 0
      ((case x of (i, x, len, pw) ⇒ assert (x = final)) ::
       (unit, ('f, 'a) protocol-channel-scheme) state-monad)
      by (simp add: hash-range-budget-assert split: prod.splits)
  qed

```

```

have after-comp-coll:
  hash-collision-budget (length fl * ?B + 0)
    ((mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) ≫=
      (λ(i, x, len, pw). assert (x = final))))
  for idx fv
proof (rule hash-collision-budget-bind)
show hash-range-budget (length fl * ?B)
  (mfold
    (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding B-eq
  by (rule hash-range-budget-receive-query-commits-mfold) simp
show hash-collision-budget (length fl * ?B)
  (mfold
    (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding B-eq
  by (rule hash-collision-budget-receive-query-commits-mfold) simp
show  $\bigwedge x.$  hash-range-budget 0
  ((case x of (i, x, len, pw) ⇒ assert (x = final)) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  by (simp add: hash-range-budget-assert split: prod.splits)
show  $\bigwedge x.$  hash-collision-budget 0
  ((case x of (i, x, len, pw) ⇒ assert (x = final)) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  by (simp add: hash-collision-budget-assert split: prod.splits)
qed
have trace-cont-range:
  hash-range-budget (0 + (length fl * ?B + 0))
    ((assert (f-x = f-final) ::
      (unit, ('f, 'a) protocol-channel-scheme) state-monad) ≫=
      (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad) ≫=
        (λ(i, x, len, pw). assert (x = final)))))
  for idx fv f-x
  by (rule hash-range-budget-bind
    [OF hash-range-budget-assert after-comp-range])
have trace-cont-coll:

```

```

hash-collision-budget (0 + (length fl * ?B + 0))
((assert (f-x = f-final) ::
  (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
  (λ-. (mfold
    (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ(i, x, len, pw). assert (x = final))))
for idx fv f-x
by (rule hash-collision-budget-bind
  [OF hash-range-budget-assert hash-collision-budget-assert
    after-comp-range after-comp-coll])
have after-trace-range:
  hash-range-budget
  (length f-fl * ?B + (0 + (length fl * ?B + 0)))
  ((mfold (idx, hd fv, clength * scale, 1)
    (receive-query-commits f-fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ(f-i, f-x, f-len, fpw).
      assert (f-x = f-final) >>=
      (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final))))))
for idx fv
proof (rule hash-range-budget-bind)
show hash-range-budget (length f-fl * ?B)
  (mfold (idx, hd fv, clength * scale, 1)
    (receive-query-commits f-fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad)
unfolding B-eq
by (rule hash-range-budget-receive-query-commits-mfold) simp
fix x :: nat × 'f × nat × nat
obtain f-i f-x f-len fpw where x-eq: x = (f-i, f-x, f-len, fpw)
by (cases x) auto
have case-budget:
  hash-range-budget (0 + (length fl * ?B + 0))
  ((assert (f-x = f-final) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ-. (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(i, x, len, pw). assert (x = final))))

```

```

      (λ(i, x, len, pw). assert (x = final))))
  by (rule trace-cont-range)
show hash-range-budget (0 + (length fl * ?B + 0))
  ((case x of (f-i, f-x, f-len, fpw) ⇒
    assert (f-x = f-final) >>=
    (λ-. (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(i, x, len, pw). assert (x = final)))) ::
      (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  using case-budget unfolding x-eq by (simp split: prod.splits)
qed
have after-trace-coll:
  hash-collision-budget
    (length f-fl * ?B + (0 + (length fl * ?B + 0)))
    ((mfold (idx, hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(f-i, f-x, f-len, fpw).
        assert (f-x = f-final) >>=
        (λ-. (mfold
          (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
          (receive-query-commits fl) ::
          (nat × 'f × nat × nat,
            ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λ(i, x, len, pw). assert (x = final)))))))
  for idx fv
proof (rule hash-collision-budget-bind)
show hash-range-budget (length f-fl * ?B)
  (mfold (idx, hd fv, clength * scale, 1)
    (receive-query-commits f-fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding B-eq
  by (rule hash-range-budget-receive-query-commits-mfold) simp
show hash-collision-budget (length f-fl * ?B)
  (mfold (idx, hd fv, clength * scale, 1)
    (receive-query-commits f-fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding B-eq
  by (rule hash-collision-budget-receive-query-commits-mfold) simp
fix x :: nat × 'f × nat × nat
obtain f-i f-x f-len fpw where x-eq: x = (f-i, f-x, f-len, fpw)
  by (cases x) auto
have range-case:

```

```

hash-range-budget (0 + (length fl * ?B + 0))
  ((assert (f-x = f-final) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ-. (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(i, x, len, pw). assert (x = final))))))
  by (rule trace-cont-range)
show hash-range-budget (0 + (length fl * ?B + 0))
  ((case x of (f-i, f-x, f-len, fpw) ⇒
    assert (f-x = f-final) >>=
    (λ-. (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(i, x, len, pw). assert (x = final)))))) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  using range-case unfolding x-eq by (simp split: prod.splits)
have coll-case:
  hash-collision-budget (0 + (length fl * ?B + 0))
    ((assert (f-x = f-final) ::
      (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final))))))
    by (rule trace-cont-coll)
  show hash-collision-budget (0 + (length fl * ?B + 0))
    ((case x of (f-i, f-x, f-len, fpw) ⇒
      assert (f-x = f-final) >>=
      (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final)))))) ::
      (unit, ('f, 'a) protocol-channel-scheme) state-monad)
    using coll-case unfolding x-eq by (simp split: prod.splits)
qed
have after-query-range:
  hash-range-budget
    (verifier-query-decommit-hash-budget +
      (length f-fl * ?B + (0 + (length fl * ?B + 0))))
    (mmap (check-decommit-on-query fr idx) >>=

```

```

    (λfv. (mfold (idx, hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) ≫=
      (λ(f-i, f-x, f-len, fpw).
        assert (f-x = f-final) ≫=
        (λ-. (mfold
          (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
          (receive-query-commits fl) ::
          (nat × 'f × nat × nat,
            ('f, 'a) protocol-channel-scheme) state-monad) ≫=
          (λ(i, x, len, pw). assert (x = final))))))
  for idx
proof -
  have decommit:
    hash-range-budget verifier-query-decommit-hash-budget
    (mmap (check-decommit-on-query fr idx))
  proof -
    have exact:
      hash-range-budget
      (powers * Suc (floor-log (scale * clength)))
      (mmap (check-decommit-on-query fr idx))
    by (rule hash-range-budget-check-decommit-on-query)
  show ?thesis
    using exact
    unfolding verifier-query-decommit-hash-budget-def
    by (simp add: mult.commute)
qed
show ?thesis
  by (rule hash-range-budget-bind[OF decommit after-trace-range])
qed
have after-query-coll:
  hash-collision-budget
  (verifier-query-decommit-hash-budget +
    (length f-fl * ?B + (0 + (length fl * ?B + 0))))
  (mmap (check-decommit-on-query fr idx) ≫=
    (λfv. (mfold (idx, hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) ≫=
      (λ(f-i, f-x, f-len, fpw).
        assert (f-x = f-final) ≫=
        (λ-. (mfold
          (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
          (receive-query-commits fl) ::
          (nat × 'f × nat × nat,
            ('f, 'a) protocol-channel-scheme) state-monad) ≫=
          (λ(i, x, len, pw). assert (x = final))))))
  for idx

```

```

proof –
  have decommit-range:
    hash-range-budget verifier-query-decommit-hash-budget
      (mmap (check-decommit-on-query fr idx))
  proof –
    have exact:
      hash-range-budget
        (powers * Suc (floor-log (scale * clength)))
        (mmap (check-decommit-on-query fr idx))
      by (rule hash-range-budget-check-decommit-on-query)
    show ?thesis
      using exact
      unfolding verifier-query-decommit-hash-budget-def
      by (simp add: mult.commute)
  qed
have decommit-coll:
  hash-collision-budget verifier-query-decommit-hash-budget
    (mmap (check-decommit-on-query fr idx))
proof –
  have exact:
    hash-collision-budget
      (powers * Suc (floor-log (scale * clength)))
      (mmap (check-decommit-on-query fr idx))
    by (rule hash-collision-budget-check-decommit-on-query)
  show ?thesis
    using exact
    unfolding verifier-query-decommit-hash-budget-def
    by (simp add: mult.commute)
  qed
show ?thesis
  by (rule hash-collision-budget-bind
    [OF decommit-range decommit-coll after-trace-range after-trace-coll])
qed
have after-query-range-simple:
  hash-range-budget
    (verifier-query-decommit-hash-budget +
      (length f-fl * ?B + length fl * ?B))
    (mmap (check-decommit-on-query fr idx)  $\gg=$ 
      ( $\lambda fv. (mfold (idx, hd fv, clength * scale, 1)$ 
        (receive-query-commits f-fl) ::
        (nat  $\times$  'f  $\times$  nat  $\times$  nat,
          ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
        ( $\lambda(f-i, f-x, f-len, fpw).$ 
          assert (f-x = f-final)  $\gg=$ 
          ( $\lambda-. (mfold$ 
            (idx, cp-eval as fv (h  $\wedge$  idx * shift), clength * scale, 1)
            (receive-query-commits fl) ::
            (nat  $\times$  'f  $\times$  nat  $\times$  nat,
              ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 

```

```

      (λ(i, x, len, pw). assert (x = final))))))
for idx
  using after-query-range[of idx] by simp
have after-query-coll-simple:
  hash-collision-budget
    (verifier-query-decommit-hash-budget +
     (length f-fl * ?B + length fl * ?B))
    (mmap (check-decommit-on-query fr idx) >>=
     (λfv. (mfold (idx, hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat,
       ('f, 'a) protocol-channel-scheme) state-monad) >>=
     (λ(f-i, f-x, f-len, fpw).
      assert (f-x = f-final) >>=
      (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
         ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final)))))))
  for idx
    using after-query-coll[of idx] by simp
have after-random-range:
  hash-range-budget
    (verifier-query-decommit-hash-budget +
     (length f-fl * ?B + length fl * ?B))
    ((let idx' = index (to-nat idx) in
     mmap (check-decommit-on-query fr idx') >>=
     (λfv. (mfold (idx', hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat,
       ('f, 'a) protocol-channel-scheme) state-monad) >>=
     (λ(f-i, f-x, f-len, fpw).
      assert (f-x = f-final) >>=
      (λ-. (mfold
        (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
         ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final)))))))
  for idx
    using after-query-range-simple[of index (to-nat idx)]
    by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
have after-random-coll:
  hash-collision-budget
    (verifier-query-decommit-hash-budget +
     (length f-fl * ?B + length fl * ?B))
    ((let idx' = index (to-nat idx) in
     mmap (check-decommit-on-query fr idx') >>=

```



```

    (λfv. (mfold (idx', hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(f-i, f-x, f-len, fpw).
        assert (f-x = f-final) >>=
        (λ-. (mfold
          (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
          (receive-query-commits fl) ::
          (nat × 'f × nat × nat,
            ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λ(i, x, len, pw). assert (x = final)))))))
  for idx
  using after-query-coll-simple[of index (to-nat idx)]
  by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
have hash-collision-budget
  (1 + (verifier-query-decommit-hash-budget +
    (length f-fl * ?B + length fl * ?B)))
  (receive-query-index-challenge >>=
    (λidx.
      let idx' = index (to-nat idx) in
      mmap (check-decommit-on-query fr idx') >>=
      (λfv. (mfold (idx', hd fv, clength * scale, 1)
        (receive-query-commits f-fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(f-i, f-x, f-len, fpw).
          assert (f-x = f-final) >>=
          (λ-. (mfold
            (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
            (receive-query-commits fl) ::
            (nat × 'f × nat × nat,
              ('f, 'a) protocol-channel-scheme) state-monad) >>=
            (λ(i, x, len, pw). assert (x = final)))))))
    by (rule hash-collision-budget-bind
      [OF hash-range-budget-receive-query-index-challenge
        hash-collision-budget-receive-query-index-challenge])
    (rule after-random-range, rule after-random-coll)
  then show ?thesis
  unfolding verifier-query-round-program-def
  by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
qed

```

```

lemma hash-collision-budget-verifier-query-round-program:
  assumes len-f: length f-fl ≤ ceil-log clength
  and len-c: length fl ≤ ceil-log (maxDegree + 1)
  shows
    hash-collision-budget verifier-query-round-hash-budget
      (verifier-query-round-program fr f-fl f-final as fl final)

```

proof –
 let $?B = \text{verifier-fri-layer-hash-budget}$
 have *exact*:
 hash-collision-budget
 $(1 + \text{verifier-query-decommit-hash-budget} +$
 $(\text{length } f\text{-fl} + \text{length } fl) * ?B)$
 $(\text{verifier-query-round-program } fr\ f\text{-fl } f\text{-final } \text{as } fl\ final)$
 by (rule *hash-collision-budget-verifier-query-round-program-exact*)
 have *le*:
 $1 + \text{verifier-query-decommit-hash-budget} +$
 $(\text{length } f\text{-fl} + \text{length } fl) * ?B$
 $\leq \text{verifier-query-round-hash-budget}$
 using *len-f len-c*
 unfolding *verifier-query-round-hash-budget-def*
 by *simp*
 show *?thesis*
 by (rule *hash-collision-budget-mono*[*OF le exact*])
qed

lemma *ceil-log-to-nat-degree-bound*:
 assumes $to\text{-nat } dg \leq \text{maxDegree}$
 shows $\text{ceil-log } (to\text{-nat } dg + 1) \leq \text{ceil-log } (\text{maxDegree} + 1)$
 by (rule *ceil-log-mono*) (use *assms in simp*)

lemma *hash-range-budget-ntimes-receive-fri-commits-bound*:
 assumes $n \leq N$
 shows *hash-range-budget* N (*ntimes receive-fri-commits* n)
proof –
 have *exact*: *hash-range-budget* $(n * 1)$ (*ntimes receive-fri-commits* n)
 by (rule *hash-range-budget-ntimes*[*OF hash-range-budget-receive-fri-commits*])
 show *?thesis*
 by (rule *hash-range-budget-mono*[*OF - exact*]) (use *assms in simp*)
qed

lemma *hash-target-program-ntimes-receive-trace-fri-commits-bound*:
 assumes $n \leq N$
 shows *hash-target-program* B N (*ntimes receive-trace-fri-commits* n)
proof –
 have *exact*:
 hash-target-program B $(n * 1)$ (*ntimes receive-trace-fri-commits* n)
 by (rule *hash-target-program-ntimes*
 [*OF hash-target-program-receive-trace-fri-commits*])
 show *?thesis*
 by (rule *hash-target-program-mono*[*OF - exact*]) (use *assms in simp*)
qed

lemma *hash-target-program-ntimes-receive-composition-fri-commits-bound*:
 assumes $n \leq N$
 shows *hash-target-program* B N (*ntimes receive-composition-fri-commits* n)

```

proof –
  have exact:
    hash-target-program B (n * 1) (ntimes receive-composition-fri-commits n)
  by (rule hash-target-program-ntimes
    [OF hash-target-program-receive-composition-fri-commits])
  show ?thesis
    by (rule hash-target-program-mono[OF - exact]) (use assms in simp)
qed

```

```

lemma hash-target-program-alpha-round:
  hash-target-program B 1 alpha-round
  unfolding alpha-round-def
  by (rule hash-target-program-alpha-challenge-step)

```

```

lemma hash-target-program-verify-monad:
  hash-target-program B verifier-hash-query-budget verify-monad

```

```

proof –
  let ?Q = rounds * verifier-query-round-hash-budget
  have alpha-map-range:
    hash-target-program B (length spec)
    (mmap (replicate (length spec)
      (alpha-round ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)))
  proof –
    let ?steps =
      replicate (length spec)
      (alpha-round ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    have step-range:
       $\bigwedge m. m \in \text{set } ?steps \implies \text{hash-target-program } B \ 1 \ m$ 
    proof –
      fix m
      assume  $m \in \text{set } ?steps$ 
      then have m-eq:
         $m =$ 
        (alpha-round ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad)
      by simp
      show hash-target-program B 1 m
      unfolding m-eq
      by (rule hash-target-program-alpha-round)
    qed
    have hash-target-program B (length ?steps * 1) (mmap ?steps)
      by (rule hash-target-program-mmap)
      (rule step-range)
    then show ?thesis by simp
  qed
  have after-final:
    hash-target-program B (0 + ?Q)

```

```

(read  $\gg$ 
  ( $\lambda$ final. ntimes
    (verifier-query-round-program fr f-fl f-final as fl final)
    rounds))
if len-f: length f-fl  $\leq$  ceil-log clength
  and len-fl: length fl  $\leq$  ceil-log (maxDegree + 1)
for fr f-fl f-final as fl
by (rule hash-target-program-bind
  [OF hash-target-program-read
    hash-target-program-ntimes
    [OF hash-target-program-verifier-query-round-program
      [OF len-f len-fl]]])
have after-comp-fri:
  hash-target-program B (ceil-log (maxDegree + 1) + (0 + ?Q))
  ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
    (((f  $\times$  'f) list),
      (f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda$ fl. read  $\gg$ 
      ( $\lambda$ final. ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds))))
if len-f: length f-fl  $\leq$  ceil-log clength
  and dg-bound: to-nat dg  $\leq$  maxDegree
for fr f-fl f-final as dg
proof (rule hash-target-program-bind-on-outcomes)
show hash-target-program B (ceil-log (maxDegree + 1))
  (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
    (((f  $\times$  'f) list),
      (f, 'a) protocol-channel-scheme) state-monad)
by (rule hash-target-program-ntimes-receive-composition-fri-commits-bound)
  (rule ceil-log-to-nat-degree-bound[OF dg-bound])
fix s fl t
assume out:
  Some (fl, t)  $\in$ 
    set-dist (execute
      (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f  $\times$  'f) list),
          (f, 'a) protocol-channel-scheme) state-monad) s)
have len-fl: length fl  $\leq$  ceil-log (maxDegree + 1)
using ntimes-receive-composition-fri-commits-outcome[OF out]
  ceil-log-to-nat-degree-bound[OF dg-bound]
by simp
show hash-target-program B (0 + ?Q)
  (read  $\gg$ 
    ( $\lambda$ final. ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds))
by (rule after-final[OF len-f len-fl])
qed

```

```

have after-degree-assert:
  hash-target-program B (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))
  (assert (to-nat dg ≤ maxDegree) >>=
    (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
      (((f × 'f) list),
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λfl. read >>=
        (λfinal. ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds))))))
  if len-f: length f-fl ≤ ceil-log clength
  for fr f-fl f-final as dg
proof (rule hash-target-program-bind-on-outcomes)
show hash-target-program B 0
  (assert (to-nat dg ≤ maxDegree) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-target-program-assert)
fix s u t
assume out:
  Some (u, t) ∈
    set-dist (execute
      (assert (to-nat dg ≤ maxDegree) ::
        (unit, ('f, 'a) protocol-channel-scheme) state-monad) s)
have dg-bound: to-nat dg ≤ maxDegree
  using out unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
show hash-target-program B (ceil-log (maxDegree + 1) + (0 + ?Q))
  ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
    (((f × 'f) list),
      ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λfl. read >>=
      (λfinal. ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds))))
  by (rule after-comp-fri[OF len-f dg-bound])
qed
have after-read-dg:
  hash-target-program B
  (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))
  (read >>=
    (λdg. assert (to-nat dg ≤ maxDegree) >>=
      (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f × 'f) list),
          ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λfl. read >>=
            (λfinal. ntimes
              (verifier-query-round-program fr f-fl f-final as fl final)
              rounds))))))
  if len-f: length f-fl ≤ ceil-log clength

```

```

for fr f-fl f-final as
by (rule hash-target-program-bind
  [OF hash-target-program-read after-degree-assert[OF len-f]])
have after-alpha:
  hash-target-program B
    (length spec +
      (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))))
    (mmap (replicate (length spec) alpha-round) >>=
      (λas. read >>=
        (λdg. assert (to-nat dg ≤ maxDegree) >>=
          (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
            (((f × 'f) list),
              ('f, 'a) protocol-channel-scheme) state-monad) >>=
            (λfl. read >>=
              (λfinal. ntimes
                (verifier-query-round-program fr f-fl f-final as fl final)
                rounds))))))
    if len-f: length f-fl ≤ ceil-log clength
    for fr f-fl f-final
    by (rule hash-target-program-bind
      [OF alpha-map-range after-read-dg[OF len-f]])
    have after-read-f-final:
      hash-target-program B
        (0 + (length spec +
          (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
        (read >>=
          (λf-final. mmap (replicate (length spec) alpha-round) >>=
            (λas. read >>=
              (λdg. assert (to-nat dg ≤ maxDegree) >>=
                (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
                  (((f × 'f) list),
                    ('f, 'a) protocol-channel-scheme) state-monad) >>=
                  (λfl. read >>=
                    (λfinal. ntimes
                      (verifier-query-round-program fr f-fl f-final as fl final)
                      rounds))))))
              if len-f: length f-fl ≤ ceil-log clength
              for fr f-fl
              by (rule hash-target-program-bind
                [OF hash-target-program-read after-alpha[OF len-f]])
              have after-trace-fri:
                hash-target-program B
                  (ceil-log clength +
                    (0 + (length spec +
                      (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
                  ((ntimes receive-trace-fri-commits (ceil-log clength) ::
                    (((f × 'f) list),
                      ('f, 'a) protocol-channel-scheme) state-monad) >>=
                    (λf-fl. read >>=

```

```

    (λf-final. mmap (replicate (length spec) alpha-round) >>=
      (λas. read >>=
        (λdg. assert (to-nat dg ≤ maxDegree) >>=
          (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
            (((f × 'f) list),
              (f, 'a) protocol-channel-scheme) state-monad) >>=
            (λfl. read >>=
              (λfinal. ntimes
                (verifier-query-round-program fr f-fl f-final as fl final)
                rounds))))))))))
  for fr
proof (rule hash-target-program-bind-on-outcomes)
show hash-target-program B (ceil-log clength)
  (ntimes receive-trace-fri-commits (ceil-log clength) ::
    (((f × 'f) list),
      (f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-target-program-ntimes-receive-trace-fri-commits-bound) simp
fix s f-fl t
assume out:
  Some (f-fl, t) ∈
    set-dist (execute
      (ntimes receive-trace-fri-commits (ceil-log clength) ::
        (((f × 'f) list),
          (f, 'a) protocol-channel-scheme) state-monad) s)
have len-f: length f-fl ≤ ceil-log clength
  using ntimes-receive-trace-fri-commits-outcome[OF out] by simp
show hash-target-program B
  (0 + (length spec +
    (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
  (read >>=
    (λf-final. mmap (replicate (length spec) alpha-round) >>=
      (λas. read >>=
        (λdg. assert (to-nat dg ≤ maxDegree) >>=
          (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
            (((f × 'f) list),
              (f, 'a) protocol-channel-scheme) state-monad) >>=
            (λfl. read >>=
              (λfinal. ntimes
                (verifier-query-round-program fr f-fl f-final as fl final)
                rounds))))))))))
  by (rule after-read-f-final[OF len-f])
qed
have top:
  hash-target-program B
  (0 + (ceil-log clength +
    (0 + (length spec +
      (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))))))
  (read >>=
    (λfr. (ntimes receive-trace-fri-commits (ceil-log clength) ::

```

```

    (((f × 'f) list),
     (f, 'a) protocol-channel-scheme) state-monad) >>=
  (λf-fl. read >>=
   (λf-final. mmap (replicate (length spec) alpha-round) >>=
    (λas. read >>=
     (λdg. assert (to-nat dg ≤ maxDegree) >>=
      (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f × 'f) list),
         (f, 'a) protocol-channel-scheme) state-monad) >>=
        (λfl. read >>=
         (λfinal. ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds))))))))))
  by (rule hash-target-program-bind
      [OF hash-target-program-read after-trace-fri])
show ?thesis
using top
unfolding verify-monad-def verifier-query-round-program-def alpha-round-def
  verifier-hash-query-budget-def verifier-header-hash-budget-def
by (simp add: algebra-simps)
qed

```

lemma hash-target-budget-verify-monad:
 hash-target-budget B verifier-hash-query-budget verify-monad
 by (rule hash-target-program-budget[OF hash-target-program-verify-monad])

lemma wp-verify-monad-hash-new-output-hit-bound:
 wp-event verify-monad (hash-new-output-hit-event B s) $s \leq$
 hash-target-budget-value B verifier-hash-query-budget
 using hash-target-budget-verify-monad
 unfolding hash-target-budget-def by blast

lemma wp-verify-monad-transcript-hash-target-hit-bound:
 wp-event verify-monad
 (hash-new-output-hit-event (set tr) (verifier-initial-state tr))
 (verifier-initial-state tr)
 $\leq$ nnreal (card (set tr) * verifier-hash-query-budget) / nnreal size
 using wp-verify-monad-hash-new-output-hit-bound
 [of set tr verifier-initial-state tr]
 unfolding hash-target-budget-value-def
 by (simp add: mult.commute)

lemma hash-range-budget-ntimes-receive-trace-fri-commits-bound:
 assumes $n \leq N$
 shows hash-range-budget N (ntimes receive-trace-fri-commits n)
 proof –
 have exact:
 hash-range-budget ($n * 1$) (ntimes receive-trace-fri-commits n)


```

    by (rule hash-range-budget-ntimes
        [OF hash-range-budget-receive-trace-fri-commits])
  show ?thesis
    by (rule hash-range-budget-mono[OF - exact]) (use assms in simp)
qed

lemma hash-range-budget-ntimes-receive-composition-fri-commits-bound:
  assumes  $n \leq N$ 
  shows hash-range-budget N (ntimes receive-composition-fri-commits n)
proof -
  have exact:
    hash-range-budget (n * 1) (ntimes receive-composition-fri-commits n)
  by (rule hash-range-budget-ntimes
      [OF hash-range-budget-receive-composition-fri-commits])
  show ?thesis
    by (rule hash-range-budget-mono[OF - exact]) (use assms in simp)
qed

lemma hash-range-budget-alpha-round:
  hash-range-budget 1 alpha-round
  unfolding alpha-round-def
  by (rule hash-range-budget-alpha-challenge-step)

lemma hash-range-budget-verify-monad:
  hash-range-budget verifier-hash-query-budget verify-monad
proof -
  let ?Q = rounds * verifier-query-round-hash-budget
  have alpha-map-range:
    hash-range-budget (length spec)
      (mmap (replicate (length spec)
        (alpha-round ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad)))
  proof -
    let ?steps =
      replicate (length spec)
        (alpha-round ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    have step-range:
       $\bigwedge m. m \in \text{set } ?steps \implies \text{hash-range-budget } 1 \ m$ 
  proof -
    fix m
    assume  $m \in \text{set } ?steps$ 
    then have m-eq:
      m =
        (alpha-round ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    by simp
    show hash-range-budget 1 m
      unfolding m-eq

```

```

    by (rule hash-range-budget-alpha-round)
qed
have hash-range-budget (length ?steps * 1) (mmap ?steps)
  by (rule hash-range-budget-mmap)
    (rule step-range)
then show ?thesis by simp
qed
have after-final:
  hash-range-budget (0 + ?Q)
    (read  $\gg$ 
      ( $\lambda$ final. ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds))
  if len-f: length f-fl  $\leq$  ceil-log clength
    and len-fl: length fl  $\leq$  ceil-log (maxDegree + 1)
  for fr f-fl f-final as fl
  by (rule hash-range-budget-bind
    [OF hash-range-budget-read
      hash-range-budget-ntimes
      [OF hash-range-budget-verifier-query-round-program
        [OF len-f len-fl]]])
have after-comp-fri:
  hash-range-budget (ceil-log (maxDegree + 1) + (0 + ?Q))
    ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
      (((f  $\times$  'f) list),
        (f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
      ( $\lambda$ fl. read  $\gg$ 
        ( $\lambda$ final. ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds))))
  if len-f: length f-fl  $\leq$  ceil-log clength
    and dg-bound: to-nat dg  $\leq$  maxDegree
  for fr f-fl f-final as dg
proof (rule hash-range-budget-bind-on-outcomes)
show hash-range-budget (ceil-log (maxDegree + 1))
  (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
    (((f  $\times$  'f) list),
      (f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-ntimes-receive-composition-fri-commits-bound)
    (rule ceil-log-to-nat-degree-bound[OF dg-bound])
fix s fl t
assume out:
  Some (fl, t)  $\in$ 
    set-dist (execute
      (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f  $\times$  'f) list),
          (f, 'a) protocol-channel-scheme) state-monad) s)
have len-fl: length fl  $\leq$  ceil-log (maxDegree + 1)
  using ntimes-receive-composition-fri-commits-outcome[OF out]

```

```

    ceil-log-to-nat-degree-bound[OF dg-bound]
  by simp
show hash-range-budget (0 + ?Q)
  (read  $\gg$ 
    ( $\lambda$ final. ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds))
  by (rule after-final[OF len-f len-fl])
qed
have after-degree-assert:
  hash-range-budget (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))
  (assert (to-nat dg  $\leq$  maxDegree)  $\gg$ 
    ( $\lambda$ -. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
      (((f  $\times$  'f) list),
        ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
      ( $\lambda$ fl. read  $\gg$ 
        ( $\lambda$ final. ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds))))
  if len-f: length f-fl  $\leq$  ceil-log clength
  for fr f-fl f-final as dg
proof (rule hash-range-budget-bind-on-outcomes)
show hash-range-budget 0
  (assert (to-nat dg  $\leq$  maxDegree) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-assert)
fix s u t
assume out:
  Some (u, t)  $\in$ 
    set-dist (execute
      (assert (to-nat dg  $\leq$  maxDegree) ::
        (unit, ('f, 'a) protocol-channel-scheme) state-monad) s)
have dg-bound: to-nat dg  $\leq$  maxDegree
  using out unfolding assert-def
  by (cases to-nat dg  $\leq$  maxDegree) (auto simp: throw-no-outcome)
show hash-range-budget (ceil-log (maxDegree + 1) + (0 + ?Q))
  ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
    (((f  $\times$  'f) list),
      ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda$ fl. read  $\gg$ 
      ( $\lambda$ final. ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds))))
  by (rule after-comp-fri[OF len-f dg-bound])
qed
have after-read-dg:
  hash-range-budget
    (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))
  (read  $\gg$ 

```

```

( $\lambda dg.$  assert ( $to\text{-}nat\ dg \leq maxDegree$ )  $\gg$ 
( $\lambda -.$  ( $ntimes$  receive-composition-fri-commits ( $ceil\text{-}log\ (to\text{-}nat\ dg + 1)$ ) ::
((( $f \times f$ ) list),
( $f$ ,  $a$ ) protocol-channel-scheme) state-monad)  $\gg$ 
( $\lambda fl.$  read  $\gg$ 
( $\lambda final.$   $ntimes$ 
(verifier-query-round-program fr  $f\text{-}fl\ f\text{-}final$  as  $fl\ final$ )
rounds))))))
if  $len\text{-}f$ :  $length\ f\text{-}fl \leq ceil\text{-}log\ clength$ 
for fr  $f\text{-}fl\ f\text{-}final$  as
by (rule hash-range-budget-bind
[OF hash-range-budget-read after-degree-assert[OF  $len\text{-}f$ ]])
have after-alpha:
hash-range-budget
(length spec +
(0 + (0 + ( $ceil\text{-}log\ (maxDegree + 1)$ ) + (0 +  $?Q$ ))))))
(mmap (replicate (length spec) alpha-round)  $\gg$ 
( $\lambda as.$  read  $\gg$ 
( $\lambda dg.$  assert ( $to\text{-}nat\ dg \leq maxDegree$ )  $\gg$ 
( $\lambda -.$  ( $ntimes$  receive-composition-fri-commits ( $ceil\text{-}log\ (to\text{-}nat\ dg + 1)$ ) ::
((( $f \times f$ ) list),
( $f$ ,  $a$ ) protocol-channel-scheme) state-monad)  $\gg$ 
( $\lambda fl.$  read  $\gg$ 
( $\lambda final.$   $ntimes$ 
(verifier-query-round-program fr  $f\text{-}fl\ f\text{-}final$  as  $fl\ final$ )
rounds)))))))))
if  $len\text{-}f$ :  $length\ f\text{-}fl \leq ceil\text{-}log\ clength$ 
for fr  $f\text{-}fl\ f\text{-}final$ 
by (rule hash-range-budget-bind
[OF alpha-map-range after-read-dg[OF  $len\text{-}f$ ]])
have after-read-f-final:
hash-range-budget
(0 + (length spec +
(0 + (0 + ( $ceil\text{-}log\ (maxDegree + 1)$ ) + (0 +  $?Q$ ))))))
(read  $\gg$ 
( $\lambda f\text{-}final.$  mmap (replicate (length spec) alpha-round)  $\gg$ 
( $\lambda as.$  read  $\gg$ 
( $\lambda dg.$  assert ( $to\text{-}nat\ dg \leq maxDegree$ )  $\gg$ 
( $\lambda -.$  ( $ntimes$  receive-composition-fri-commits ( $ceil\text{-}log\ (to\text{-}nat\ dg + 1)$ ) ::
((( $f \times f$ ) list),
( $f$ ,  $a$ ) protocol-channel-scheme) state-monad)  $\gg$ 
( $\lambda fl.$  read  $\gg$ 
( $\lambda final.$   $ntimes$ 
(verifier-query-round-program fr  $f\text{-}fl\ f\text{-}final$  as  $fl\ final$ )
rounds)))))))))
if  $len\text{-}f$ :  $length\ f\text{-}fl \leq ceil\text{-}log\ clength$ 
for fr  $f\text{-}fl$ 
by (rule hash-range-budget-bind
[OF hash-range-budget-read after-alpha[OF  $len\text{-}f$ ]])

```

```

have after-trace-fri:
  hash-range-budget
    (ceil-log clength +
      (0 + (length spec +
        (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))))))
    ((ntimes receive-trace-fri-commits (ceil-log clength) ::
      (((f × 'f) list),
        (f, 'a) protocol-channel-scheme) state-monad) >>=
      (λf-fl. read >>=
        (λf-final. mmap (replicate (length spec) alpha-round) >>=
          (λas. read >>=
            (λdg. assert (to-nat dg ≤ maxDegree) >>=
              (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
                (((f × 'f) list),
                  (f, 'a) protocol-channel-scheme) state-monad) >>=
                (λfl. read >>=
                  (λfinal. ntimes
                    (verifier-query-round-program fr f-fl f-final as fl final)
                    rounds))))))))))

for fr
proof (rule hash-range-budget-bind-on-outcomes)
show hash-range-budget (ceil-log clength)
  (ntimes receive-trace-fri-commits (ceil-log clength) ::
    (((f × 'f) list),
      (f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-ntimes-receive-trace-fri-commits-bound) simp
fix s f-fl t
assume out:
  Some (f-fl, t) ∈
    set-dist (execute
      (ntimes receive-trace-fri-commits (ceil-log clength) ::
        (((f × 'f) list),
          (f, 'a) protocol-channel-scheme) state-monad) s)
have len-f: length f-fl ≤ ceil-log clength
  using ntimes-receive-trace-fri-commits-outcome[OF out] by simp
show hash-range-budget
  (0 + (length spec +
    (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
  (read >>=
    (λf-final. mmap (replicate (length spec) alpha-round) >>=
      (λas. read >>=
        (λdg. assert (to-nat dg ≤ maxDegree) >>=
          (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
            (((f × 'f) list),
              (f, 'a) protocol-channel-scheme) state-monad) >>=
            (λfl. read >>=
              (λfinal. ntimes
                (verifier-query-round-program fr f-fl f-final as fl final)
                rounds))))))))))

```

```

    by (rule after-read-f-final[OF len-f])
qed
have top:
  hash-range-budget
    (0 + (ceil-log clength +
      (0 + (length spec +
        (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))))
    (read  $\gg$ 
      ( $\lambda$ fr. (ntimes receive-trace-fri-commits (ceil-log clength) ::
        (((f  $\times$  'f) list),
          (f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda$ f-fl. read  $\gg$ 
          ( $\lambda$ f-final. mmap (replicate (length spec) alpha-round)  $\gg$ 
            ( $\lambda$ as. read  $\gg$ 
              ( $\lambda$ dg. assert (to-nat dg  $\leq$  maxDegree)  $\gg$ 
                ( $\lambda$ -. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
                  (((f  $\times$  'f) list),
                    (f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
                    ( $\lambda$ fl. read  $\gg$ 
                      ( $\lambda$ final. ntimes
                        (verifier-query-round-program fr f-fl f-final as fl final)
                        rounds))))))))))
    by (rule hash-range-budget-bind
      [OF hash-range-budget-read after-trace-fri])
show ?thesis
  using top
  unfolding verify-monad-def verifier-query-round-program-def alpha-round-def
    verifier-hash-query-budget-def verifier-header-hash-budget-def
  by (simp add: algebra-simps)
qed

lemma hash-collision-budget-ntimes-receive-fri-commits-bound:
  assumes  $n \leq N$ 
  shows hash-collision-budget N (ntimes receive-fri-commits n)
proof -
  have exact: hash-collision-budget (n * 1) (ntimes receive-fri-commits n)
  by (rule hash-collision-budget-ntimes
    [OF hash-range-budget-receive-fri-commits
      hash-collision-budget-receive-fri-commits])
  show ?thesis
  by (rule hash-collision-budget-mono[OF - exact]) (use assms in simp)
qed

lemma hash-collision-budget-ntimes-receive-trace-fri-commits-bound:
  assumes  $n \leq N$ 
  shows hash-collision-budget N (ntimes receive-trace-fri-commits n)
proof -
  have exact:
    hash-collision-budget (n * 1) (ntimes receive-trace-fri-commits n)

```

```

    by (rule hash-collision-budget-ntimes
        [OF hash-range-budget-receive-trace-fri-commits
         hash-collision-budget-receive-trace-fri-commits])
  show ?thesis
    by (rule hash-collision-budget-mono[OF - exact]) (use assms in simp)
qed

lemma hash-collision-budget-ntimes-receive-composition-fri-commits-bound:
  assumes  $n \leq N$ 
  shows hash-collision-budget  $N$  (ntimes receive-composition-fri-commits  $n$ )
proof -
  have exact:
    hash-collision-budget ( $n * 1$ )
      (ntimes receive-composition-fri-commits  $n$ )
  by (rule hash-collision-budget-ntimes
      [OF hash-range-budget-receive-composition-fri-commits
       hash-collision-budget-receive-composition-fri-commits])
  show ?thesis
    by (rule hash-collision-budget-mono[OF - exact]) (use assms in simp)
qed

lemma hash-collision-budget-alpha-round:
  hash-collision-budget 1 alpha-round
  unfolding alpha-round-def
  by (rule hash-collision-budget-alpha-challenge-step)

lemma hash-collision-budget-verify-monad:
  hash-collision-budget verifier-hash-query-budget verify-monad
proof -
  let ?Q = rounds * verifier-query-round-hash-budget
  let ?alpha-steps =
    replicate (length spec)
      (alpha-round ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  have alpha-step-range:
     $\bigwedge m. m \in \text{set } ?\text{alpha-steps} \implies \text{hash-range-budget } 1 \ m$ 
  proof -
    fix  $m$ 
    assume  $m \in \text{set } ?\text{alpha-steps}$ 
    then have m-eq:
       $m =$ 
      (alpha-round ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    by simp
    show hash-range-budget 1  $m$ 
      unfolding m-eq by (rule hash-range-budget-alpha-round)
  qed
  have alpha-step-coll:
     $\bigwedge m. m \in \text{set } ?\text{alpha-steps} \implies \text{hash-collision-budget } 1 \ m$ 

```

```

proof –
  fix m
  assume  $m \in \text{set } ?\text{alpha-steps}$ 
  then have m-eq:
    m =
      (alpha-round ::
        (f, (f, 'a') protocol-channel-scheme) state-monad)
    by simp
  show hash-collision-budget 1 m
    unfolding m-eq by (rule hash-collision-budget-alpha-round)
qed
have alpha-map-range:
  hash-range-budget (length spec) (mmap ?alpha-steps)
proof –
  have hash-range-budget (length ?alpha-steps * 1) (mmap ?alpha-steps)
    by (rule hash-range-budget-mmap) (rule alpha-step-range)
  then show ?thesis by simp
qed
have alpha-map-coll:
  hash-collision-budget (length spec) (mmap ?alpha-steps)
proof –
  have hash-collision-budget (length ?alpha-steps * 1) (mmap ?alpha-steps)
  proof (rule hash-collision-budget-mmap)
    fix m
    assume m-in:  $m \in \text{set } ?\text{alpha-steps}$ 
    show hash-range-budget 1 m
      by (rule alpha-step-range[OF m-in])
    show hash-collision-budget 1 m
      by (rule alpha-step-coll[OF m-in])
    qed
  then show ?thesis by simp
qed
have after-final-range:
  hash-range-budget (0 + ?Q)
  (read  $\gg$ 
    ( $\lambda \text{final. ntimes}$ 
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds))
  if len-f: length f-fl  $\leq \text{ceil-log } \text{clength}$ 
    and len-fl: length fl  $\leq \text{ceil-log } (\text{maxDegree} + 1)$ 
  for fr f-fl f-final as fl
  by (rule hash-range-budget-bind
    [OF hash-range-budget-read
      hash-range-budget-ntimes
      [OF hash-range-budget-verifier-query-round-program
        [OF len-f len-fl]]])
have after-final-coll:
  hash-collision-budget (0 + ?Q)
  (read  $\gg$ 

```



```

    (λfinal. ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds))
  if len-f: length f-fl ≤ ceil-log clength
    and len-fl: length fl ≤ ceil-log (maxDegree + 1)
  for fr f-fl f-final as fl
proof -
  have query-range:
    ∧final. hash-range-budget verifier-query-round-hash-budget
      (verifier-query-round-program fr f-fl f-final as fl final)
  by (rule hash-range-budget-verifier-query-round-program
      [OF len-f len-fl])
  have query-coll:
    ∧final. hash-collision-budget verifier-query-round-hash-budget
      (verifier-query-round-program fr f-fl f-final as fl final)
  by (rule hash-collision-budget-verifier-query-round-program
      [OF len-f len-fl])
  have rounds-range:
    ∧final. hash-range-budget ?Q
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
  by (rule hash-range-budget-ntimes) (rule query-range)
  have rounds-coll:
    ∧final. hash-collision-budget ?Q
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
  by (rule hash-collision-budget-ntimes)
    (rule query-range, rule query-coll)
  show ?thesis
  by (rule hash-collision-budget-bind
      [OF hash-range-budget-read hash-collision-budget-read
        rounds-range rounds-coll])
qed
have after-comp-fri-range:
  hash-range-budget (ceil-log (maxDegree + 1) + (0 + ?Q))
    ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
      (((f × 'f) list),
        ('f, 'a) protocol-channel-scheme) state-monad) ≫=
      (λfl. read ≫=
        (λfinal. ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds))))
  if len-f: length f-fl ≤ ceil-log clength
    and dg-bound: to-nat dg ≤ maxDegree
  for fr f-fl f-final as dg
proof (rule hash-range-budget-bind-on-outcomes)
  show hash-range-budget (ceil-log (maxDegree + 1))
    (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
      (((f × 'f) list),

```

```

    ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-ntimes-receive-composition-fri-commits-bound)
    (rule ceil-log-to-nat-degree-bound[OF dg-bound])
fix s fl t
assume out:
  Some (fl, t) ∈
    set-dist (execute
      (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f × 'f) list),
          ('f, 'a) protocol-channel-scheme) state-monad) s)
have len-fl: length fl ≤ ceil-log (maxDegree + 1)
  using ntimes-receive-composition-fri-commits-outcome[OF out]
    ceil-log-to-nat-degree-bound[OF dg-bound]
  by simp
show hash-range-budget (0 + ?Q)
  (read >>=
    (λfinal. ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds))
  by (rule after-final-range[OF len-f len-fl])
qed
have after-comp-fri-coll:
  hash-collision-budget (ceil-log (maxDegree + 1) + (0 + ?Q))
    ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
      (((f × 'f) list),
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λfl. read >>=
        (λfinal. ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds))))
  if len-f: length f-fl ≤ ceil-log clength
    and dg-bound: to-nat dg ≤ maxDegree
  for fr f-fl f-final as dg
proof (rule hash-collision-budget-bind-on-outcomes)
  show hash-range-budget (ceil-log (maxDegree + 1))
    (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
      (((f × 'f) list),
        ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-ntimes-receive-composition-fri-commits-bound)
    (rule ceil-log-to-nat-degree-bound[OF dg-bound])
  show hash-collision-budget (ceil-log (maxDegree + 1))
    (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
      (((f × 'f) list),
        ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-collision-budget-ntimes-receive-composition-fri-commits-bound)
    (rule ceil-log-to-nat-degree-bound[OF dg-bound])
fix s fl t
assume out:
  Some (fl, t) ∈

```

```

    set-dist (execute
      (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f × f) list),
          (f, 'a) protocol-channel-scheme) state-monad) s)
  have len-fl: length fl ≤ ceil-log (maxDegree + 1)
  using ntimes-receive-composition-fri-commits-outcome[OF out]
    ceil-log-to-nat-degree-bound[OF dg-bound]
  by simp
show hash-range-budget (0 + ?Q)
  (read >>=
    (λfinal. ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds))
  by (rule after-final-range[OF len-f len-fl])
show hash-collision-budget (0 + ?Q)
  (read >>=
    (λfinal. ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds))
  by (rule after-final-coll[OF len-f len-fl])
qed
have after-degree-assert-range:
  hash-range-budget (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))
  (assert (to-nat dg ≤ maxDegree) >>=
    (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
      (((f × f) list),
        (f, 'a) protocol-channel-scheme) state-monad) >>=
      (λfl. read >>=
        (λfinal. ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds))))))
  if len-f: length f-fl ≤ ceil-log clength
  for fr f-fl f-final as dg
proof (rule hash-range-budget-bind-on-outcomes)
show hash-range-budget 0
  (assert (to-nat dg ≤ maxDegree) ::
    (unit, (f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-assert)
fix s u t
assume out:
  Some (u, t) ∈
    set-dist (execute
      (assert (to-nat dg ≤ maxDegree) ::
        (unit, (f, 'a) protocol-channel-scheme) state-monad) s)
have dg-bound: to-nat dg ≤ maxDegree
  using out unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
show hash-range-budget (ceil-log (maxDegree + 1) + (0 + ?Q))
  ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::

```

```

      (((f × 'f) list),
       (f, 'a) protocol-channel-scheme) state-monad) >>=
    (λfl. read >>=
     (λfinal. ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds))))
  by (rule after-comp-fri-range[OF len-f dg-bound])
qed
have after-degree-assert-coll:
  hash-collision-budget (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))
  (assert (to-nat dg ≤ maxDegree) >>=
   (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
    (((f × 'f) list),
     (f, 'a) protocol-channel-scheme) state-monad) >>=
    (λfl. read >>=
     (λfinal. ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds))))))
  if len-f: length f-fl ≤ ceil-log clength
  for fr f-fl f-final as dg
proof (rule hash-collision-budget-bind-on-outcomes)
  show hash-range-budget 0
    (assert (to-nat dg ≤ maxDegree) ::
     (unit, (f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-assert)
  show hash-collision-budget 0
    (assert (to-nat dg ≤ maxDegree) ::
     (unit, (f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-collision-budget-assert)
  fix s u t
  assume out:
    Some (u, t) ∈
      set-dist (execute
        (assert (to-nat dg ≤ maxDegree) ::
         (unit, (f, 'a) protocol-channel-scheme) state-monad) s)
  have dg-bound: to-nat dg ≤ maxDegree
    using out unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
  show hash-range-budget (ceil-log (maxDegree + 1) + (0 + ?Q))
    ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
     (((f × 'f) list),
      (f, 'a) protocol-channel-scheme) state-monad) >>=
     (λfl. read >>=
      (λfinal. ntimes
       (verifier-query-round-program fr f-fl f-final as fl final)
       rounds))))
  by (rule after-comp-fri-range[OF len-f dg-bound])
  show hash-collision-budget (ceil-log (maxDegree + 1) + (0 + ?Q))
    ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::

```

```

      (((f × 'f) list),
      (f, 'a) protocol-channel-scheme) state-monad) >>=
    (λfl. read >>=
      (λfinal. ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds))))
  by (rule after-comp-fri-coll[OF len-f dg-bound])
qed
have after-read-dg-range:
  hash-range-budget
  (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))
  (read >>=
    (λdg. assert (to-nat dg ≤ maxDegree) >>=
      (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f × 'f) list),
        (f, 'a) protocol-channel-scheme) state-monad) >>=
        (λfl. read >>=
          (λfinal. ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds))))))
  if len-f: length f-fl ≤ ceil-log clength
  for fr f-fl f-final as
  by (rule hash-range-budget-bind
    [OF hash-range-budget-read after-degree-assert-range[OF len-f]])
have after-read-dg-coll:
  hash-collision-budget
  (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))
  (read >>=
    (λdg. assert (to-nat dg ≤ maxDegree) >>=
      (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f × 'f) list),
        (f, 'a) protocol-channel-scheme) state-monad) >>=
        (λfl. read >>=
          (λfinal. ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds))))))
  if len-f: length f-fl ≤ ceil-log clength
  for fr f-fl f-final as
  by (rule hash-collision-budget-bind
    [OF hash-range-budget-read hash-collision-budget-read
      after-degree-assert-range[OF len-f]
      after-degree-assert-coll[OF len-f]])
have after-alpha-range:
  hash-range-budget
  (length spec +
    (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))
  (mmap ?alpha-steps >>=
    (λas. read >>=
      (λdg. assert (to-nat dg ≤ maxDegree) >>=

```

```

    (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
      (((f × 'f) list),
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λfl. read >>=
      (λfinal. ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds))))))
  if len-f: length f-fl ≤ ceil-log clength
  for fr f-fl f-final
  by (rule hash-range-budget-bind
    [OF alpha-map-range after-read-dg-range[OF len-f]])
  have after-alpha-coll:
    hash-collision-budget
      (length spec +
        (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))))
      (mmap ?alpha-steps >>=
        (λas. read >>=
          (λdg. assert (to-nat dg ≤ maxDegree) >>=
            (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
              (((f × 'f) list),
                ('f, 'a) protocol-channel-scheme) state-monad) >>=
              (λfl. read >>=
                (λfinal. ntimes
                  (verifier-query-round-program fr f-fl f-final as fl final)
                  rounds)))))))
          if len-f: length f-fl ≤ ceil-log clength
          for fr f-fl f-final
          by (rule hash-collision-budget-bind
            [OF alpha-map-range alpha-map-coll
              after-read-dg-range[OF len-f] after-read-dg-coll[OF len-f]])
          have after-read-f-final-range:
            hash-range-budget
              (0 + (length spec +
                (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))))
              (read >>=
                (λf-final. mmap ?alpha-steps >>=
                  (λas. read >>=
                    (λdg. assert (to-nat dg ≤ maxDegree) >>=
                      (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
                        (((f × 'f) list),
                          ('f, 'a) protocol-channel-scheme) state-monad) >>=
                        (λfl. read >>=
                          (λfinal. ntimes
                            (verifier-query-round-program fr f-fl f-final as fl final)
                            rounds)))))))
                    if len-f: length f-fl ≤ ceil-log clength
                    for fr f-fl
                    by (rule hash-range-budget-bind
                      [OF hash-range-budget-read after-alpha-range[OF len-f]])

```

```

have after-read-f-final-coll:
  hash-collision-budget
    (0 + (length spec +
      (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
  (read >=>
    (λf-final. mmap ?alpha-steps >=>
      (λas. read >=>
        (λdg. assert (to-nat dg ≤ maxDegree) >=>
          (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
            (((f × 'f) list),
              (f, 'a) protocol-channel-scheme) state-monad) >=>
            (λfl. read >=>
              (λfinal. ntimes
                (verifier-query-round-program fr f-fl f-final as fl final)
                rounds))))))))
  if len-f: length f-fl ≤ ceil-log clength
  for fr f-fl
  by (rule hash-collision-budget-bind
    [OF hash-range-budget-read hash-collision-budget-read
      after-alpha-range[OF len-f] after-alpha-coll[OF len-f]])
have after-trace-fri-range:
  hash-range-budget
    (ceil-log clength +
      (0 + (length spec +
        (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
    ((ntimes receive-trace-fri-commits (ceil-log clength) ::
      (((f × 'f) list),
        (f, 'a) protocol-channel-scheme) state-monad) >=>
      (λf-fl. read >=>
        (λf-final. mmap ?alpha-steps >=>
          (λas. read >=>
            (λdg. assert (to-nat dg ≤ maxDegree) >=>
              (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
                (((f × 'f) list),
                  (f, 'a) protocol-channel-scheme) state-monad) >=>
                (λfl. read >=>
                  (λfinal. ntimes
                    (verifier-query-round-program fr f-fl f-final as fl final)
                    rounds))))))))
          for fr
proof (rule hash-range-budget-bind-on-outcomes)
show hash-range-budget (ceil-log clength)
  (ntimes receive-trace-fri-commits (ceil-log clength) ::
    (((f × 'f) list),
      (f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-ntimes-receive-trace-fri-commits-bound) simp
fix s f-fl t
assume out:
  Some (f-fl, t) ∈

```

```

    set-dist (execute
      (ntimes receive-trace-fri-commits (ceil-log clength) ::
        (((f × 'f) list),
          (f, 'a) protocol-channel-scheme) state-monad) s)
  have len-f: length f-fl ≤ ceil-log clength
  using ntimes-receive-trace-fri-commits-outcome[OF out] by simp
show hash-range-budget
  (0 + (length spec +
    (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
  (read ≥
    (λf-final. mmap ?alpha-steps ≥
      (λas. read ≥
        (λdg. assert (to-nat dg ≤ maxDegree) ≥
          (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
            (((f × 'f) list),
              (f, 'a) protocol-channel-scheme) state-monad) ≥
            (λfl. read ≥
              (λfinal. ntimes
                (verifier-query-round-program fr f-fl f-final as fl final)
                rounds)))))))
    by (rule after-read-f-final-range[OF len-f])
qed
have after-trace-fri-coll:
  hash-collision-budget
    (ceil-log clength +
      (0 + (length spec +
        (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
    ((ntimes receive-trace-fri-commits (ceil-log clength) ::
      (((f × 'f) list),
        (f, 'a) protocol-channel-scheme) state-monad) ≥
      (λf-fl. read ≥
        (λf-final. mmap ?alpha-steps ≥
          (λas. read ≥
            (λdg. assert (to-nat dg ≤ maxDegree) ≥
              (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
                (((f × 'f) list),
                  (f, 'a) protocol-channel-scheme) state-monad) ≥
                  (λfl. read ≥
                    (λfinal. ntimes
                      (verifier-query-round-program fr f-fl f-final as fl final)
                      rounds)))))))
            by (rule after-read-f-final-range[OF len-f])
            for fr
  proof (rule hash-collision-budget-bind-on-outcomes)
  show hash-range-budget (ceil-log clength)
    (ntimes receive-trace-fri-commits (ceil-log clength) ::
      (((f × 'f) list),
        (f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-range-budget-ntimes-receive-trace-fri-commits-bound) simp
  show hash-collision-budget (ceil-log clength)

```



```

  (ntimes receive-trace-fri-commits (ceil-log clength) ::
    (((f × 'f) list),
     (f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-collision-budget-ntimes-receive-trace-fri-commits-bound) simp
fix s f-fl t
assume out:
  Some (f-fl, t) ∈
    set-dist (execute
      (ntimes receive-trace-fri-commits (ceil-log clength) ::
        (((f × 'f) list),
         (f, 'a) protocol-channel-scheme) state-monad) s)
have len-f: length f-fl ≤ ceil-log clength
  using ntimes-receive-trace-fri-commits-outcome[OF out] by simp
show hash-range-budget
  (0 + (length spec +
    (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
  (read >=
    (λf-final. mmap ?alpha-steps >=
      (λas. read >=
        (λdg. assert (to-nat dg ≤ maxDegree) >=
          (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
            (((f × 'f) list),
             (f, 'a) protocol-channel-scheme) state-monad) >=
            (λfl. read >=
              (λfinal. ntimes
                (verifier-query-round-program fr f-fl f-final as fl final)
                rounds)))))))
    by (rule after-read-f-final-range[OF len-f])
show hash-collision-budget
  (0 + (length spec +
    (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
  (read >=
    (λf-final. mmap ?alpha-steps >=
      (λas. read >=
        (λdg. assert (to-nat dg ≤ maxDegree) >=
          (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
            (((f × 'f) list),
             (f, 'a) protocol-channel-scheme) state-monad) >=
            (λfl. read >=
              (λfinal. ntimes
                (verifier-query-round-program fr f-fl f-final as fl final)
                rounds)))))))
    by (rule after-read-f-final-coll[OF len-f])
qed
have top-range:
  hash-range-budget
  (0 + (ceil-log clength +
    (0 + (length spec +
      (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))))))

```

```

(read >=
  (λfr. (ntimes receive-trace-fri-commits (ceil-log clength) ::
    (((f × f) list),
      (f, 'a) protocol-channel-scheme) state-monad) >=
  (λf-fl. read >=
    (λf-final. mmap ?alpha-steps >=
      (λas. read >=
        (λdg. assert (to-nat dg ≤ maxDegree) >=
          (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
            (((f × f) list),
              (f, 'a) protocol-channel-scheme) state-monad) >=
            (λfl. read >=
              (λfinal. ntimes
                (verifier-query-round-program fr f-fl f-final as fl final)
                rounds))))))))))
  by (rule hash-range-budget-bind
    [OF hash-range-budget-read after-trace-fri-range])
have top-coll:
  hash-collision-budget
  (0 + (ceil-log clength +
    (0 + (length spec +
      (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))))
  (read >=
    (λfr. (ntimes receive-trace-fri-commits (ceil-log clength) ::
      (((f × f) list),
        (f, 'a) protocol-channel-scheme) state-monad) >=
    (λf-fl. read >=
      (λf-final. mmap ?alpha-steps >=
        (λas. read >=
          (λdg. assert (to-nat dg ≤ maxDegree) >=
            (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
              (((f × f) list),
                (f, 'a) protocol-channel-scheme) state-monad) >=
              (λfl. read >=
                (λfinal. ntimes
                  (verifier-query-round-program fr f-fl f-final as fl final)
                  rounds))))))))))
  by (rule hash-collision-budget-bind
    [OF hash-range-budget-read hash-collision-budget-read
      after-trace-fri-range after-trace-fri-coll])
show ?thesis
  using top-coll
  unfolding verify-monad-def verifier-query-round-program-def alpha-round-def
    verifier-hash-query-budget-def verifier-header-hash-budget-def
  by (simp add: algebra-simps)
qed

end

```

end

theory *Soundness-Oracle-Merkle*
imports *Soundness-Oracle-Verifier*
begin

Merkle-binding collision covers and concrete binding interfaces.

context *soundness*
begin

definition *hash-map-new-output-collision-bad*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{hash-map-new-output-collision-bad } s \text{ out} \longleftrightarrow$
 $\text{accepted out} \wedge$
 $(\exists \text{ result final-state.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{hash-map-new-output-collision } s \text{ final-state})$

definition *hash-map-new-output-collision-bound*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{hash-map-new-output-collision-bound } s \longleftrightarrow$
 $\text{wp-event verify-monad (hash-map-new-output-collision-bad } s) s \leq$
 $\text{merkle-binding-error}$

lemma *hash-map-new-output-collision-bound-if-error-ge-one:*
assumes $1 \leq \text{merkle-binding-error}$
shows $\text{hash-map-new-output-collision-bound } s$
unfolding *hash-map-new-output-collision-bound-def*
using *order-trans[OF wp-event-le-1 assms]* .

lemma *hash-map-new-output-collision-bad-imp-hash-new-collision-event:*
assumes $\text{hash-map-new-output-collision-bad } s \text{ out}$
shows $\text{hash-new-collision-event } s \text{ out}$
using *assms*
unfolding *hash-map-new-output-collision-bad-def hash-new-collision-event-def*
by *auto*

lemma *hash-map-new-output-collision-event-bound-from-verify-hash-budget:*
assumes $\text{clean: } \neg \text{hash-map-output-collision } s$
shows
 $\text{wp-event verify-monad (hash-map-new-output-collision-bad } s) s \leq$
 $\text{hash-collision-budget-value (card (hash-map-output-values } s))$
 $\text{verifier-hash-query-budget}$

proof —
have $\text{wp-event verify-monad (hash-map-new-output-collision-bad } s) s \leq$

wp-event verify-monad (hash-new-collision-event s) s
 by (rule wp-event-mono)
 (rule hash-map-new-output-collision-bad-imp-hash-new-collision-event)
 also have ... $\leq$
 hash-collision-budget-value (card (hash-map-output-values s))
 verifier-hash-query-budget
 using hash-collision-budget-verify-monad clean
 unfolding hash-collision-budget-def by blast
 finally show ?thesis .
 qed

lemma hash-map-new-output-collision-bound-from-verify-hash-budget:
 assumes clean: $\neg$ hash-map-output-collision s
 and error:
 hash-collision-budget-value (card (hash-map-output-values s))
 verifier-hash-query-budget $\leq$ merkle-binding-error
 shows hash-map-new-output-collision-bound s
proof –
 have wp-event verify-monad (hash-map-new-output-collision-bad s) s $\leq$
 hash-collision-budget-value (card (hash-map-output-values s))
 verifier-hash-query-budget
 by (rule hash-map-new-output-collision-event-bound-from-verify-hash-budget
 [OF clean])
 also have ... $\leq$ merkle-binding-error
 by (rule error)
 finally show ?thesis
 unfolding hash-map-new-output-collision-bound-def .
 qed

lemma hash-map-new-output-collision-bound-from-verify-hash-budget-empty:
 assumes empty-hashmap: $\text{HashMap } s = \text{fmempty}$
 and error:
 hash-collision-budget-value (card (hash-map-output-values s))
 verifier-hash-query-budget $\leq$ merkle-binding-error
 shows hash-map-new-output-collision-bound s
proof –
 have clean: $\neg$ hash-map-output-collision s
 using empty-hashmap unfolding hash-map-output-collision-def by simp
 show ?thesis
 by (rule hash-map-new-output-collision-bound-from-verify-hash-budget
 [OF clean error])
 qed

lemma verifier-initial-hash-map-new-output-collision-bound:
 assumes error:
 hash-collision-budget-value 0 verifier-hash-query-budget $\leq$
 merkle-binding-error
 shows
 hash-map-new-output-collision-bound (verifier-initial-state tr)

by (rule hash-map-new-output-collision-bound-from-verify-hash-budget-empty)
 (use error in simp-all)

lemma verifier-initial-hash-map-new-output-collision-bad-bound-concrete:

wp-event verify-monad
 (hash-map-new-output-collision-bad (verifier-initial-state tr))
 (verifier-initial-state tr) $\leq$ concrete-merkle-binding-error

proof –

have clean:

$\neg$ hash-map-output-collision (verifier-initial-state tr)
unfolding verifier-initial-state-def hash-map-output-collision-def
by simp

have bound:

wp-event verify-monad
 (hash-map-new-output-collision-bad (verifier-initial-state tr))
 (verifier-initial-state tr) $\leq$
 hash-collision-budget-value
 (card (hash-map-output-values (verifier-initial-state tr)))
 verifier-hash-query-budget
by (rule hash-map-new-output-collision-event-bound-from-verify-hash-budget
 [OF clean])

show ?thesis

using bound **unfolding** concrete-merkle-binding-error-def **by** simp

qed

lemma hash-map-output-collision-bad-imp-new-collision-bad-if-initial-clean:

assumes clean: $\neg$ hash-map-output-collision s
and bad: hash-map-output-collision-bad s out
shows hash-map-new-output-collision-bad s out
using assms
unfolding hash-map-output-collision-bad-def
 hash-map-new-output-collision-bad-def
 hash-map-new-output-collision-def
by auto

lemma hash-map-output-collision-bound-from-new-collision-bound:

assumes clean: $\neg$ hash-map-output-collision s
and new-bound: hash-map-new-output-collision-bound s
shows hash-map-output-collision-bound s

proof –

have wp-event verify-monad (hash-map-output-collision-bad s) s $\leq$
 wp-event verify-monad (hash-map-new-output-collision-bad s) s
by (rule wp-event-mono)
 (rule hash-map-output-collision-bad-imp-new-collision-bad-if-initial-clean
 [OF clean])

also have ... $\leq$ merkle-binding-error

using new-bound **unfolding** hash-map-new-output-collision-bound-def .

finally show ?thesis

unfolding hash-map-output-collision-bound-def .

qed

definition *fri-merkle-binding-bad-covered-by-hash-collision*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

fri-merkle-binding-bad-covered-by-hash-collision $s \longleftrightarrow$
 $(\forall \text{ out} \in \text{set-dist } (\text{execute verify-monad } s).$
 $\text{fri-merkle-binding-bad } s \text{ out} \longrightarrow$
 $\text{hash-map-output-collision-bad } s \text{ out})$

lemma *fri-merkle-binding-bound-from-hash-collision-cover:*

assumes *cover*: *fri-merkle-binding-bad-covered-by-hash-collision* s

and *collision-bound*: *hash-map-output-collision-bound* s

shows *fri-merkle-binding-bound* s

proof –

have *wp-event verify-monad* (*fri-merkle-binding-bad* s) $s \leq$
wp-event verify-monad (*hash-map-output-collision-bad* s) s

by (*rule wp-event-mono-on-support*)

(*use cover in*

$\langle \text{auto simp: fri-merkle-binding-bad-covered-by-hash-collision-def} \rangle$)

also have $\dots \leq \text{merkle-binding-error}$

using *collision-bound* **unfolding** *hash-map-output-collision-bound-def* .

finally show *?thesis*

unfolding *fri-merkle-binding-bound-def* .

qed

lemma *fri-merkle-binding-bound-from-new-hash-collision-cover:*

assumes *clean*: $\neg \text{hash-map-output-collision } s$

and *cover*: *fri-merkle-binding-bad-covered-by-hash-collision* s

and *new-bound*: *hash-map-new-output-collision-bound* s

shows *fri-merkle-binding-bound* s

proof –

have *collision-bound*: *hash-map-output-collision-bound* s

by (*rule hash-map-output-collision-bound-from-new-collision-bound*
 $[\text{OF clean new-bound}]$)

show *?thesis*

by (*rule fri-merkle-binding-bound-from-hash-collision-cover*
 $[\text{OF cover collision-bound}]$)

qed

lemma *hash-map-output-collision-empty-hashmap[simp]:*

assumes *HashMap* $s = \text{fmempty}$

shows $\neg \text{hash-map-output-collision } s$

using *assms* **unfolding** *hash-map-output-collision-def* **by** *simp*

lemma *verifier-initial-no-hash-map-output-collision[simp]:*

$\neg \text{hash-map-output-collision } (\text{verifier-initial-state } tr)$

by *simp*

lemma *verifier-initial-fri-merkle-binding-bound-concrete:*
assumes *cover:*
fri-merkle-binding-bad-covered-by-hash-collision
(verifier-initial-state tr)
shows
wp-event verify-monad
(fri-merkle-binding-bad (verifier-initial-state tr))
(verifier-initial-state tr) ≤ concrete-merkle-binding-error
proof –
let *?s = verifier-initial-state tr*
have *wp-event verify-monad (fri-merkle-binding-bad ?s) ?s ≤*
wp-event verify-monad (hash-map-output-collision-bad ?s) ?s
by *(rule wp-event-mono-on-support)*
(use cover in
⟨auto simp: fri-merkle-binding-bad-covered-by-hash-collision-def⟩)
also have *... ≤*
wp-event verify-monad (hash-map-new-output-collision-bad ?s) ?s
by *(rule wp-event-mono)*
(rule hash-map-output-collision-bad-imp-new-collision-bad-if-initial-clean,
simp, assumption)
also have *... ≤ concrete-merkle-binding-error*
by *(rule verifier-initial-hash-map-new-output-collision-bad-bound-concrete)*
finally show *?thesis .*
qed

lemma *fri-merkle-binding-bound-from-new-hash-collision-cover-empty:*
assumes *HashMap s = fmempty*
and *cover: fri-merkle-binding-bad-covered-by-hash-collision s*
and *new-bound: hash-map-new-output-collision-bound s*
shows *fri-merkle-binding-bound s*
by *(rule fri-merkle-binding-bound-from-new-hash-collision-cover*
[OF - cover new-bound])
(use assms(1) in simp)

lemma *fri-bound-layer-tables-length:*
assumes *fri-bound-layer-tables roots tables final-state*
shows *length tables = length roots*
using *assms*
unfolding *fri-bound-layer-tables-def fri-layer-tables-shape-def* **by** *simp*

lemma *fri-bound-layer-tables-root:*
assumes *fri-bound-layer-tables roots tables final-state*
and *j < length roots*
shows *merkle-root-binds-table (roots ! j) (tables ! j) final-state*
using *assms*
unfolding *fri-bound-layer-tables-def fri-roots-bind-tables-def* **by** *simp*

lemma *fri-bound-layer-tables-table-length:*
assumes *fri-bound-layer-tables roots tables final-state*

```

    and  $j < \text{length roots}$ 
  shows
     $\text{length (tables ! } j) =$ 
       $\text{fri-layer-lengths (length roots) (clength * scale) ! } j$ 
  using assms
  unfolding fri-bound-layer-tables-def fri-layer-tables-shape-def
  by simp

lemma fri-bound-layer-tables-same-root-collision-imp-hash-collision-bad:
  assumes out:  $\text{out} = \text{Some (result, final-state)}$ 
    and bound1: fri-bound-layer-tables roots tables final-state
    and bound2: fri-bound-layer-tables roots tables' final-state
    and j-bound:  $j < \text{length roots}$ 
    and nonempty:  $\text{tables ! } j \neq []$ 
    and neq:  $\text{tables ! } j \neq \text{tables' ! } j$ 
  shows hash-map-output-collision-bad s out
proof -
  have bind1:
    merkle-root-binds-table (roots ! j) (tables ! j) final-state
    by (rule fri-bound-layer-tables-root[OF bound1 j-bound])
  have bind2:
    merkle-root-binds-table (roots ! j) (tables' ! j) final-state
    by (rule fri-bound-layer-tables-root[OF bound2 j-bound])
  have same-len:  $\text{length (tables ! } j) = \text{length (tables' ! } j)$ 
    using fri-bound-layer-tables-table-length[OF bound1 j-bound]
      fri-bound-layer-tables-table-length[OF bound2 j-bound]
    by simp
  show ?thesis
    by (rule accepted-inconsistent-same-length-merkle-bindings-imp-hash-collision-bad
      [OF out bind1 bind2 same-len nonempty neq])
qed

lemma fri-layer-opening-matches-bound-tableD:
  assumes match:
    fri-layer-opening-matches-bound-table query-idx layer-idx tables
    layer-chunks
  obtains len idx xp xp-path xn xn-path
  where  $\text{layer-idx} < \text{length tables}$ 
    and  $\text{length layer-chunks} = \text{length tables}$ 
    and  $\text{len} =$ 
       $\text{fri-layer-lengths (length tables) (clength * scale) ! layer-idx}$ 
    and  $\text{idx} =$ 
       $\text{fri-layer-indices (length tables) query-idx (clength * scale) !}$ 
       $\text{layer-idx}$ 
    and fri-layer-opening-chunk len xp xp-path xn xn-path
      (layer-chunks ! layer-idx)
    and fri-opening-matches-table len idx (tables ! layer-idx) xp xn
  using match
  unfolding fri-layer-opening-matches-bound-table-def Let-def

```


by blast

lemma *fri-round-openings-match-tablesD*:

assumes *fri-round-openings-match-tables tables query-idx layer-chunks*
and $j < \text{length } \text{tables}$

shows

fri-layer-opening-matches-bound-table query-idx j tables layer-chunks

using *assms* **unfolding** *fri-round-openings-match-tables-def* **by** *simp*

lemma *fri-all-round-openings-match-tablesD*:

assumes

fri-all-round-openings-match-tables tables query-idxs round-layers

and $i < \text{length } \text{query-idxs}$

shows

fri-round-openings-match-tables tables (query-idxs ! i)

(round-layers ! i)

using *assms* **unfolding** *fri-all-round-openings-match-tables-def* **by** *simp*

lemma *trace-fri-tables-bound-opening-matches*:

assumes *bound*:

trace-fri-tables-bound s out trace-roots trace-bs trace-final

query-idxs trace-round-layers trace-tables trace-final-table

and *i-bound*: $i < \text{length } \text{query-idxs}$

and *j-bound*: $j < \text{length } \text{trace-tables}$

shows

fri-layer-opening-matches-bound-table (query-idxs ! i) j

trace-tables (trace-round-layers ! i)

proof —

have *rounds*:

fri-all-round-openings-match-tables trace-tables query-idxs

trace-round-layers

using *bound* **unfolding** *trace-fri-tables-bound-def* **by** *blast*

from *fri-all-round-openings-match-tablesD*[*OF rounds i-bound*]

have *fri-round-openings-match-tables trace-tables (query-idxs ! i)*

(trace-round-layers ! i) .

then show *?thesis*

using *j-bound* **by** *(rule fri-round-openings-match-tablesD)*

qed

lemma *composition-fri-tables-bound-opening-matches*:

assumes *bound*:

composition-fri-tables-bound s out dg composition-roots composition-bs

composition-final query-idxs composition-round-layers

composition-tables composition-final-table

and *i-bound*: $i < \text{length } \text{query-idxs}$

and *j-bound*: $j < \text{length } \text{composition-tables}$

shows

fri-layer-opening-matches-bound-table (query-idxs ! i) j

composition-tables (composition-round-layers ! i)

proof –
have *rounds*:
 fri-all-round-openings-match-tables composition-tables query-idxs
 composition-round-layers
 using *bound unfolding composition-fri-tables-bound-def* **by** *blast*
from *fri-all-round-openings-match-tablesD*[*OF rounds i-bound*]
have *fri-round-openings-match-tables composition-tables (query-idxs ! i)*
 (*composition-round-layers ! i*) .
then show *?thesis*
 using *j-bound* **by** (*rule fri-round-openings-match-tablesD*)
qed

lemma *fri-merkle-binding-bad-imp-accepted*:
assumes *fri-merkle-binding-bad s out*
shows *accepted out*
using *assms unfolding fri-merkle-binding-bad-def* **by** *simp*

lemma *fri-merkle-binding-bad-mono-accepted*:
wp-event verify-monad (fri-merkle-binding-bad s) s ≤
wp-event verify-monad accepted s
by (*rule wp-event-mono*)
 (*simp add: fri-merkle-binding-bad-imp-accepted*)

lemma *accepted-good-or-fri-merkle-binding-bad*:
assumes *accepted out*
shows *fri-merkle-binding-good s out ∨ fri-merkle-binding-bad s out*
using *assms unfolding fri-merkle-binding-bad-def* **by** *blast*

lemma *fri-merkle-binding-goodE*:
assumes *good: fri-merkle-binding-good s out*
and *acc: accepted out*
obtains *trace-table composition-table as query-idxs*
 trace-roots trace-bs trace-final dg composition-roots composition-bs
 composition-final trace-round-layers composition-round-layers
 trace-tables composition-tables trace-final-table composition-final-table
where
 accepted-with-bound-tables s out trace-table composition-table as
 query-idxs
and *accepted-fri-opening-transcript s out trace-roots trace-bs*
 trace-final dg composition-roots composition-bs composition-final
 query-idxs trace-round-layers composition-round-layers
and *trace-fri-tables-bound s out trace-roots trace-bs trace-final*
 query-idxs trace-round-layers trace-tables trace-final-table
and *composition-fri-tables-bound s out dg composition-roots*
 composition-bs composition-final query-idxs composition-round-layers
 composition-tables composition-final-table
and *fri-initial-table-agrees trace-table trace-tables*
and *fri-initial-table-agrees composition-table composition-tables*
using *good acc unfolding fri-merkle-binding-good-def* **by** *blast*

end

end

theory *Soundness-Oracle*
 imports *Soundness-Oracle-Merkle*
begin

Compatibility umbrella for random-oracle soundness infrastructure.

end

theory *Controlled-RO*
 imports
 Soundness-Oracle
begin

context *soundness*
begin

Controlled random-oracle programs provide a capability-safe interface for attacker stages. The constructors expose ordinary probabilistic computation and oracle queries, but no operation that can directly modify the oracle map or protocol-local channel fields.

inductive *controlled-ro-program*
 $:: \text{nat} \Rightarrow$
 $(\text{'r}, \text{'f protocol-channel}) \text{ state-monad} \Rightarrow \text{bool}$
where
 Return[intro]:
 $\text{controlled-ro-program } 0 \text{ (return } x \text{)}$
 Fail[intro]:
 $\text{controlled-ro-program } 0 \text{ throw}$
 Sample[intro]:
 $\text{controlled-ro-program } 0 \text{ (lift } (\lambda \cdot. d))$
 Query[intro]:
 $(\forall y. \text{controlled-ro-program } q \text{ (k y)}) \Rightarrow$
 $\text{controlled-ro-program (Suc } q \text{)}$
 $((\text{hash } x :: (\text{'f}, \text{'f protocol-channel}) \text{ state-monad})$
 $\ggg k)$
 Bind[intro]:
 $\text{controlled-ro-program } q \text{ } m \Rightarrow$
 $(\forall x. \text{controlled-ro-program } r \text{ (k } x)) \Rightarrow$
 $\text{controlled-ro-program (q + r) (m } \ggg k)$
 Weaken:
 $\text{controlled-ro-program } q \text{ } m \Rightarrow q \leq r \Rightarrow$
 $\text{controlled-ro-program } r \text{ } m$

lemma *hash-map-preserving-throw:*

```

hash-map-preserving
  (throw :: ('r, 'f protocol-channel) state-monad)
unfolding hash-map-preserving-def
by (simp add: throw-no-outcome)

lemma hash-map-preserving-return:
  hash-map-preserving
  (return x :: ('r, 'f protocol-channel) state-monad)
unfolding hash-map-preserving-def by simp

lemma hash-map-preserving-state-independent-sample:
  hash-map-preserving
  (lift (λ-. d) ::
    ('r, 'f protocol-channel) state-monad)
unfolding hash-map-preserving-def lift.rep-eq lift-dist-def
by (auto simp: set-dist-dist-map)

definition protocol-fields-preserving
  :: ('r, 'f protocol-channel) state-monad ⇒ bool
where
  protocol-fields-preserving m ⟷
    (∀ s x t.
      Some (x, t) ∈ set-dist (execute m s) ⟶
      PState t = PState s ∧
      PTranscript t = PTranscript s ∧
      PTraceFriCounter t = PTraceFriCounter s ∧
      PCompositionFriCounter t = PCompositionFriCounter s ∧
      PAlphaCounter t = PAlphaCounter s ∧
      PQueryCounter t = PQueryCounter s)

lemma controlled-ro-program-preserves-protocol-fields:
  assumes controlled-ro-program q m
  shows protocol-fields-preserving m
  using assms
proof (induction rule: controlled-ro-program.induct)
  case (Return x)
  then show ?case
    unfolding protocol-fields-preserving-def by simp
next
  case Fail
  then show ?case
    unfolding protocol-fields-preserving-def
    by (simp add: throw-no-outcome)
next
  case (Sample d)
  then show ?case
    unfolding protocol-fields-preserving-def lift.rep-eq lift-dist-def
    by (auto simp: set-dist-dist-map)
next

```

```

case (Query q k x)
show ?case
  unfolding protocol-fields-preserving-def
proof (intro allI impI)
  fix s result t
  assume outcome:
    Some (result, t) ∈
      set-dist
      (execute
        ((hash x :: ('f, 'f protocol-channel) state-monad)  $\gg$  k) s)
  then obtain h u where
    hash-out:
      Some (h, u) ∈
        set-dist
        (execute (hash x :: ('f, 'f protocol-channel) state-monad) s)
  and rest: Some (result, t) ∈ set-dist (execute (k h) u)
  by (auto elim!: set-dist-bindE)
have fields-u:
  PState u = PState s ∧
  PTranscript u = PTranscript s ∧
  PTraceFriCounter u = PTraceFriCounter s ∧
  PCompositionFriCounter u = PCompositionFriCounter s ∧
  PAlphaCounter u = PAlphaCounter s ∧
  PQueryCounter u = PQueryCounter s
  using hash-channel-preserves[OF hash-out] by simp
have preserving-k: protocol-fields-preserving (k h)
  using Query.IH by blast
have fields-t:
  PState t = PState u ∧
  PTranscript t = PTranscript u ∧
  PTraceFriCounter t = PTraceFriCounter u ∧
  PCompositionFriCounter t = PCompositionFriCounter u ∧
  PAlphaCounter t = PAlphaCounter u ∧
  PQueryCounter t = PQueryCounter u
  using preserving-k rest
  unfolding protocol-fields-preserving-def by blast
show
  PState t = PState s ∧
  PTranscript t = PTranscript s ∧
  PTraceFriCounter t = PTraceFriCounter s ∧
  PCompositionFriCounter t = PCompositionFriCounter s ∧
  PAlphaCounter t = PAlphaCounter s ∧
  PQueryCounter t = PQueryCounter s
  using fields-u fields-t by simp
qed
next
case (Bind q m r k)
show ?case
  unfolding protocol-fields-preserving-def

```

```

proof (intro allI impI)
  fix s result t
  assume outcome:
    Some (result, t) ∈ set-dist (execute (m ≫ k) s)
  then obtain x u where
    first: Some (x, u) ∈ set-dist (execute m s)
    and rest: Some (result, t) ∈ set-dist (execute (k x) u)
    by (auto elim!: set-dist-bindE)
  have fields-u:
    PState u = PState s ∧
    PTranscript u = PTranscript s ∧
    PTraceFriCounter u = PTraceFriCounter s ∧
    PCompositionFriCounter u = PCompositionFriCounter s ∧
    PAlphaCounter u = PAlphaCounter s ∧
    PQueryCounter u = PQueryCounter s
  using Bind.IH(1) first
  unfolding protocol-fields-preserving-def by blast
  have preserving-k: protocol-fields-preserving (k x)
  using Bind.IH(2) by blast
  have fields-t:
    PState t = PState u ∧
    PTranscript t = PTranscript u ∧
    PTraceFriCounter t = PTraceFriCounter u ∧
    PCompositionFriCounter t = PCompositionFriCounter u ∧
    PAlphaCounter t = PAlphaCounter u ∧
    PQueryCounter t = PQueryCounter u
  using preserving-k rest
  unfolding protocol-fields-preserving-def by blast
  show
    PState t = PState s ∧
    PTranscript t = PTranscript s ∧
    PTraceFriCounter t = PTraceFriCounter s ∧
    PCompositionFriCounter t = PCompositionFriCounter s ∧
    PAlphaCounter t = PAlphaCounter s ∧
    PQueryCounter t = PQueryCounter s
  using fields-u fields-t by simp
  qed
next
  case (Weaken q m r)
  then show ?case by simp
qed

lemma hash-range-budget-throw:
  hash-range-budget 0
  (throw :: ('r, 'f protocol-channel) state-monad)
  unfolding hash-range-budget-def
  by (simp add: throw-no-outcome)

lemma hash-collision-budget-throw:

```

```

hash-collision-budget 0
  (throw :: ('r, 'f protocol-channel) state-monad)
unfolding hash-collision-budget-def wp-event-def wp-def
  throw.rep-eq dist-throw-def dist-delta-dist delta-map-def
  hash-collision-budget-value-def hash-new-collision-event-def
by simp

lemma hash-target-program-throw:
  hash-target-program B 0
  (throw :: ('r, 'f protocol-channel) state-monad)
by (rule hash-map-preserving-imp-hash-target-program-zero)
  (rule hash-map-preserving-throw)

lemma hash-range-budget-state-independent-sample:
  hash-range-budget 0
  (lift (λ-. d) ::
    ('r, 'f protocol-channel) state-monad)
unfolding hash-range-budget-def lift.rep-eq lift-dist-def
by (auto simp: set-dist-dist-map hash-map-output-values-def)

lemma hash-map-preserving-imp-hash-collision-budget-zero:
  fixes m :: ('r, 'f protocol-channel) state-monad
  assumes preserving: hash-map-preserving m
  shows hash-collision-budget 0 m
  unfolding hash-collision-budget-def
proof (intro allI impI)
  fix s :: 'f protocol-channel
  assume clean: ¬ hash-map-output-collision s
  have no-event:
    ∧ out. out ∈ set-dist (execute m s) ⇒
      ¬ hash-new-collision-event s out
proof -
  fix out
  assume support: out ∈ set-dist (execute m s)
  show ¬ hash-new-collision-event s out
proof (cases out)
  case None
  then show ?thesis
    unfolding hash-new-collision-event-def by simp
next
  case (Some xt)
  then obtain x t where xt: xt = (x, t)
  by (cases xt) simp
  have same: HashMap t = HashMap s
  using preserving support Some xt
  unfolding hash-map-preserving-def by blast
  show ?thesis
    using clean same Some xt
    unfolding hash-new-collision-event-def

```

```

      hash-map-new-output-collision-def hash-map-output-collision-def
    by simp
  qed
  qed
  have no-event-dom:
     $\bigwedge out. out \in \text{dom} (\text{dist} (\text{execute } m \ s)) \implies$ 
     $\neg \text{hash-new-collision-event } s \ out$ 
    using no-event unfolding set-dist-def by blast
  have wp-event m (hash-new-collision-event s) s = 0
    unfolding wp-event-def wp-def dist-expect-def
    by (intro sum.neutral) (simp add: no-event-dom)
  then show
    wp-event m (hash-new-collision-event s) s  $\leq$ 
    hash-collision-budget-value
    (card (hash-map-output-values s)) 0
    unfolding hash-collision-budget-value-def by simp
  qed

lemma hash-collision-budget-state-independent-sample:
  hash-collision-budget 0
  (lift ( $\lambda \cdot. d$ ) ::
    ('r, 'f protocol-channel) state-monad)
  by (rule hash-map-preserving-imp-hash-collision-budget-zero)
  (rule hash-map-preserving-state-independent-sample)

lemma hash-target-program-state-independent-sample:
  hash-target-program B 0
  (lift ( $\lambda \cdot. d$ ) ::
    ('r, 'f protocol-channel) state-monad)
  by (rule hash-map-preserving-imp-hash-target-program-zero)
  (rule hash-map-preserving-state-independent-sample)

lemma controlled-ro-program-extension:
  assumes controlled-ro-program q m
  shows hash-extension-preserving m
  using assms
proof (induction rule: controlled-ro-program.induct)
  case (Return x)
  then show ?case by (rule hash-extension-preserving-return)
next
  case Fail
  then show ?case
    by (rule hash-map-preserving-imp-hash-extension-preserving)
    (rule hash-map-preserving-throw)
next
  case (Sample d)
  then show ?case
    by (rule hash-map-preserving-imp-hash-extension-preserving)
    (rule hash-map-preserving-state-independent-sample)

```



```

next
  case (Query q k x)
  have ext-k:  $\bigwedge y. \text{hash-extension-preserving } (k y)$ 
    using Query.IH by blast
  show ?case
    by (rule hash-extension-preserving-bind)
      (rule hash-extension-preserving-hash, rule ext-k)
next
  case (Bind q m r k)
  have ext-k:  $\bigwedge x. \text{hash-extension-preserving } (k x)$ 
    using Bind.IH(2) by blast
  show ?case
    by (rule hash-extension-preserving-bind[OF Bind.IH(1) ext-k])
next
  case (Weaken q m r)
  then show ?case by simp
qed

lemma controlled-ro-program-range:
  assumes controlled-ro-program q m
  shows hash-range-budget q m
  using assms
proof (induction rule: controlled-ro-program.induct)
  case (Return x)
  then show ?case by (rule hash-range-budget-return)
next
  case Fail
  then show ?case by (rule hash-range-budget-throw)
next
  case (Sample d)
  then show ?case by (rule hash-range-budget-state-independent-sample)
next
  case (Query q k x)
  have range-k:  $\bigwedge y. \text{hash-range-budget } q (k y)$ 
    using Query.IH by blast
  have hash-range-budget (1 + q)
    ((hash x :: ('f, 'f protocol-channel) state-monad)
      $\gg= k$ )
    by (rule hash-range-budget-bind)
      (rule hash-range-budget-hash, rule range-k)
  then show ?case by simp
next
  case (Bind q m r k)
  have range-k:  $\bigwedge x. \text{hash-range-budget } r (k x)$ 
    using Bind.IH(2) by blast
  show ?case
    by (rule hash-range-budget-bind[OF Bind.IH(1) range-k])
next
  case (Weaken q m r)

```

```

  show ?case
  by (rule hash-range-budget-mono[OF Weaken.hyps(2) Weaken.IH])
qed

lemma controlled-ro-program-collision:
  assumes controlled-ro-program q m
  shows hash-collision-budget q m
  using assms
proof (induction rule: controlled-ro-program.induct)
  case (Return x)
  then show ?case by (rule hash-collision-budget-return)
next
  case Fail
  then show ?case by (rule hash-collision-budget-throw)
next
  case (Sample d)
  then show ?case by (rule hash-collision-budget-state-independent-sample)
next
  case (Query q k x)
  have range-k:  $\bigwedge y. \text{hash-range-budget } q \ (k \ y)$ 
    using Query(1) by (blast intro: controlled-ro-program-range)
  have collision-k:  $\bigwedge y. \text{hash-collision-budget } q \ (k \ y)$ 
    using Query.IH by blast
  have hash-collision-budget (1 + q)
    ((hash x :: ('f, 'f protocol-channel) state-monad)
      $\gg=$  k)
  by (rule hash-collision-budget-bind)
    (rule hash-range-budget-hash, rule hash-collision-budget-hash,
     rule range-k, rule collision-k)
  then show ?case by simp
next
  case (Bind q m r k)
  have range-m: hash-range-budget q m
    by (rule controlled-ro-program-range[OF Bind(1)])
  have range-k:  $\bigwedge x. \text{hash-range-budget } r \ (k \ x)$ 
    using Bind(3) by (blast intro: controlled-ro-program-range)
  have collision-k:  $\bigwedge x. \text{hash-collision-budget } r \ (k \ x)$ 
    using Bind.IH(2) by blast
  show ?case
    by (rule hash-collision-budget-bind)
      (rule range-m, rule Bind.IH(1), rule range-k, rule collision-k)
next
  case (Weaken q m r)
  show ?case
    by (rule hash-collision-budget-mono[OF Weaken.hyps(2) Weaken.IH])
qed

```

```

lemma controlled-ro-program-target:
  assumes controlled-ro-program q m

```

```

  shows hash-target-program B q m
  using assms
proof (induction rule: controlled-ro-program.induct)
  case (Return x)
  then show ?case by (rule hash-target-program-return)
next
  case Fail
  then show ?case by (rule hash-target-program-throw)
next
  case (Sample d)
  then show ?case by (rule hash-target-program-state-independent-sample)
next
  case (Query q k x)
  have target-k:  $\bigwedge y. \text{hash-target-program } B \ q \ (k \ y)$ 
  using Query.IH by blast
  have hash-target-program B (1 + q)
    ((hash x :: ('f, 'f protocol-channel) state-monad)
      $\gg k$ )
  by (rule hash-target-program-bind)
    (rule hash-target-program-hash, rule target-k)
  then show ?case by simp
next
  case (Bind q m r k)
  have target-k:  $\bigwedge x. \text{hash-target-program } B \ r \ (k \ x)$ 
  using Bind.IH(2) by blast
  show ?case
  by (rule hash-target-program-bind[OF Bind.IH(1) target-k])
next
  case (Weaken q m r)
  show ?case
  by (rule hash-target-program-mono[OF Weaken.hyps(2) Weaken.IH])
qed

```

definition *controlled-ro-admissible*

```

:: nat  $\Rightarrow$ 
  ('r, 'f protocol-channel) state-monad  $\Rightarrow$  bool
where
  controlled-ro-admissible q m  $\longleftrightarrow$ 
    hash-extension-preserving m  $\wedge$ 
    hash-range-budget q m  $\wedge$ 
    hash-collision-budget q m  $\wedge$ 
    ( $\forall B. \text{hash-target-program } B \ q \ m$ )

```

lemma *controlled-ro-program-admissible:*

```

assumes controlled-ro-program q m
shows controlled-ro-admissible q m
unfolding controlled-ro-admissible-def
using controlled-ro-program-extension[OF assms]
  controlled-ro-program-range[OF assms]

```

controlled-ro-program-collision[*OF assms*]
controlled-ro-program-target[*OF assms*]
by *blast*

definition *hash-relation-hit*

$:: ('f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}) \Rightarrow$
 $'f \text{ protocol-channel} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{bool}$

where

$\text{hash-relation-hit } R \text{ initial final} \longleftrightarrow$
 $(\exists x y.$
 $\text{fmlookup } (\text{HashMap initial}) x = \text{None} \wedge$
 $\text{fmlookup } (\text{HashMap final}) x = \text{Some } y \wedge$
 $R x y)$

definition *hash-relation-hit-event*

$:: ('f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}) \Rightarrow$
 $'f \text{ protocol-channel} \Rightarrow$
 $('r \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{hash-relation-hit-event } R \text{ initial out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (-, \text{final}) \Rightarrow \text{hash-relation-hit } R \text{ initial final})$

lemma *hash-relation-hit-after-update:*

$\text{hash-relation-hit } R s$
 $(s \downarrow \text{HashMap} := \text{fmupd } x y (\text{HashMap } s)) \longleftrightarrow$
 $\text{fmlookup } (\text{HashMap } s) x = \text{None} \wedge R x y$

proof

assume *hit:*

$\text{hash-relation-hit } R s$
 $(s \downarrow \text{HashMap} := \text{fmupd } x y (\text{HashMap } s))$

then obtain *k z* **where** *fresh:*

$\text{fmlookup } (\text{HashMap } s) k = \text{None}$

and *lookup:*

$\text{fmlookup } (\text{fmupd } x y (\text{HashMap } s)) k = \text{Some } z$

and *rel:* $R k z$

by (*auto simp: hash-relation-hit-def*)

have $k = x$

proof (*rule ccontr*)

assume $k \neq x$

then have $\text{fmlookup } (\text{HashMap } s) k = \text{Some } z$

using *lookup* **by** *simp*

then show *False* **using** *fresh* **by** *simp*

qed

then show $\text{fmlookup } (\text{HashMap } s) x = \text{None} \wedge R x y$

using *fresh lookup rel* **by** *simp*

next

assume $\text{fmlookup } (\text{HashMap } s) x = \text{None} \wedge R x y$

```

then show
  hash-relation-hit R s
  (s(HashMap := fmupd x y (HashMap s)))
unfolding hash-relation-hit-def by auto
qed

lemma wp-hash-relation-hit:
fixes x :: 'f protocol-hash-input
and s :: 'f protocol-channel
shows
  wp-event (hash x) (hash-relation-hit-event R s) s =
    (if fmllookup (HashMap s) x = None
     then nnreal (card {y. R x y}) / nnreal size
     else 0)
proof (cases fmllookup (HashMap s) x)
case None
have indicator:
  (λy. if hash-relation-hit R s
    (s(HashMap := fmupd x y (HashMap s)))
    then 1 else 0 :: prob) =
  (λy. if y ∈ {y. R x y} then 1 else 0)
by (rule ext)
  (simp add: hash-relation-hit-after-update None)
have wp-event (hash x) (hash-relation-hit-event R s) s =
  dist-expect (hash-dist x s)
  (λy. if y ∈ {y. R x y} then 1 else 0)
unfolding wp-event-def hash-relation-hit-event-def
by (simp add: wp-hash indicator)
also have ... = nnreal (card {y. R x y}) / nnreal size
by (rule dist-expect-hash-dist-fresh-indicator[OF None])
finally show ?thesis using None by simp
next
case (Some y)
have no-hit:
  ¬ hash-relation-hit R s
  (s(HashMap := fmupd x y (HashMap s)))
by (simp add: hash-relation-hit-after-update Some)
show ?thesis
unfolding wp-event-def hash-relation-hit-event-def
by (simp add: wp-hash Some hash-dist-def option-default-dist-def no-hit)
qed

definition hash-relation-budget-value :: nat ⇒ nat ⇒ prob
where
  hash-relation-budget-value b q =
    nnreal (q * b) / nnreal size

definition hash-relation-budget
  :: ('f protocol-hash-input ⇒ 'f ⇒ bool) ⇒

```

```

    nat  $\Rightarrow$  nat  $\Rightarrow$ 
    ('r, 'f protocol-channel) state-monad  $\Rightarrow$  bool
where
    hash-relation-budget R b q m  $\longleftrightarrow$ 
    ( $\forall$  s.
      wp-event m (hash-relation-hit-event R s) s  $\leq$ 
      hash-relation-budget-value b q)

definition hash-relation-program
:: ('f protocol-hash-input  $\Rightarrow$  'f  $\Rightarrow$  bool)  $\Rightarrow$ 
  nat  $\Rightarrow$  nat  $\Rightarrow$ 
  ('r, 'f protocol-channel) state-monad  $\Rightarrow$  bool
where
    hash-relation-program R b q m  $\longleftrightarrow$ 
    hash-extension-preserving m  $\wedge$  hash-relation-budget R b q m

lemma hash-relation-budget-value-add:
  hash-relation-budget-value b q + hash-relation-budget-value b r =
  hash-relation-budget-value b (q + r)
unfolding hash-relation-budget-value-def
by (simp add: add-divide-nnreal algebra-simps)

lemma hash-relation-budget-value-mono:
assumes q  $\leq$  r
shows hash-relation-budget-value b q  $\leq$  hash-relation-budget-value b r
unfolding hash-relation-budget-value-def
apply (subst nn2real-le-iff[symmetric])
using assms by (simp add: divide-right-mono mult-right-mono)

lemma hash-relation-budget-value-mono-left:
assumes b  $\leq$  c
shows hash-relation-budget-value b q  $\leq$  hash-relation-budget-value c q
unfolding hash-relation-budget-value-def
apply (subst nn2real-le-iff[symmetric])
using assms by (simp add: divide-right-mono mult-left-mono)

lemma hash-relation-hit-event-step-bound:
fixes m :: ('r, 'f protocol-channel) state-monad
assumes ext-st: s  $\leq$  t
and ext-m: hash-extension-preserving m
shows
  wp-event m (hash-relation-hit-event R s) t  $\leq$ 
  (if hash-relation-hit R s t then 1
   else wp-event m (hash-relation-hit-event R t) t)
proof (cases hash-relation-hit R s t)
case True
then show ?thesis
  using wp-event-le-1[of m hash-relation-hit-event R s t] by simp
next

```

```

case False
have mono:
  wp-event m (hash-relation-hit-event R s) t ≤
    wp-event m (hash-relation-hit-event R t) t
proof (rule wp-event-mono-on-support)
  fix out
  assume support: out ∈ set-dist (execute m t)
    and hit-s: hash-relation-hit-event R s out
  show hash-relation-hit-event R t out
proof (cases out)
  case None
  then show ?thesis
    using hit-s unfolding hash-relation-hit-event-def by simp
next
  case (Some xu)
  then obtain x u where xu: xu = (x, u)
    by (cases xu) simp
  have out: Some (x, u) ∈ set-dist (execute m t)
    using support Some xu by simp
  have ext-tu: t ≤ u
    using ext-m out unfolding hash-extension-preserving-def by blast
  from hit-s obtain k y where fresh-s:
    fmlookup (HashMap s) k = None
    and lookup-u: fmlookup (HashMap u) k = Some y
    and rel: R k y
    unfolding hash-relation-hit-event-def hash-relation-hit-def
    using Some xu by auto
  show ?thesis
  proof (cases fmlookup (HashMap t) k)
    case None
    then show ?thesis
      using Some xu lookup-u rel
      unfolding hash-relation-hit-event-def hash-relation-hit-def
      by auto
  next
    case (Some z)
    have lookup-u-z: fmlookup (HashMap u) k = Some z
      by (rule hash-extension-lookup[OF Some ext-tu])
    have z = y using lookup-u lookup-u-z by simp
    then have hash-relation-hit R s t
      unfolding hash-relation-hit-def
      using fresh-s Some rel by blast
    then show ?thesis using False by contradiction
  qed
qed
qed
then show ?thesis using False by simp
qed

```

```

lemma hash-relation-budget-bind:
  fixes  $m :: ('x, 'f \text{ protocol-channel}) \text{ state-monad}$ 
    and  $k :: 'x \Rightarrow ('y, 'f \text{ protocol-channel}) \text{ state-monad}$ 
  assumes  $m\text{-ext}: \text{hash-extension-preserving } m$ 
    and  $m\text{-budget}: \text{hash-relation-budget } R \ b \ q \ m$ 
    and  $k\text{-ext}: \bigwedge x. \text{hash-extension-preserving } (k \ x)$ 
    and  $k\text{-budget}: \bigwedge x. \text{hash-relation-budget } R \ b \ r \ (k \ x)$ 
  shows  $\text{hash-relation-budget } R \ b \ (q + r) \ (m \ggg k)$ 
  unfolding hash-relation-budget-def
proof (intro allI)
  fix  $s :: 'f \text{ protocol-channel}$ 
  let  $?E = \text{hash-relation-hit-event } R \ s$ 
  let  $?Tail =$ 
     $\lambda out. \text{case out of}$ 
       $\text{None} \Rightarrow 0$ 
     $| \text{Some } (x, t) \Rightarrow$ 
       $\text{if hash-relation-hit } R \ s \ t \text{ then } 0$ 
       $\text{else wp-event } (k \ x) \ (\text{hash-relation-hit-event } R \ t) \ t$ 
  let  $?R = \lambda out. (\text{if } ?E \text{ out then } 1 \text{ else } 0) + ?Tail \text{ out}$ 
  have split:
     $\text{wp-event } (m \ggg k) \ ?E \ s \leq \text{wp } m \ ?R \ s$ 
proof –
  have exact:
     $\text{wp-event } (m \ggg k) \ ?E \ s =$ 
       $\text{wp } m$ 
       $(\lambda out. \text{case out of}$ 
         $\text{None} \Rightarrow 0$ 
         $| \text{Some } (x, t) \Rightarrow \text{wp-event } (k \ x) \ ?E \ t) \ s$ 
      unfolding wp-event-def hash-relation-hit-event-def
      by (simp add: wpsimps)
  have  $\text{wp } m$ 
     $(\lambda out. \text{case out of}$ 
       $\text{None} \Rightarrow 0$ 
       $| \text{Some } (x, t) \Rightarrow \text{wp-event } (k \ x) \ ?E \ t) \ s$ 
     $\leq \text{wp } m \ ?R \ s$ 
  proof (rule wp-mono-on-support)
  fix  $out$ 
  assume  $\text{support}: out \in \text{set-dist } (\text{execute } m \ s)$ 
  show ( $\text{case out of}$ 
     $\text{None} \Rightarrow 0$ 
     $| \text{Some } (x, t) \Rightarrow \text{wp-event } (k \ x) \ ?E \ t) \leq ?R \ out$ 
  proof (cases out)
  case None
  then show ?thesis
    unfolding hash-relation-hit-event-def by simp
  next
  case (Some xt)
  then obtain  $x \ t$  where  $xt: xt = (x, t)$ 
  by (cases xt) simp

```



```

have m-out: Some (x, t) ∈ set-dist (execute m s)
  using support Some xt by simp
have ext-st: s ≤ t
  using m-ext m-out unfolding hash-extension-preserving-def by blast
have step:
  wp-event (k x) ?E t ≤
    (if hash-relation-hit R s t then 1
     else wp-event (k x) (hash-relation-hit-event R t) t)
  by (rule hash-relation-hit-event-step-bound
    [OF ext-st k-ext[of x]])
show ?thesis
  using Some xt step
  unfolding hash-relation-hit-event-def
  by (cases hash-relation-hit R s t) simp-all
qed
qed
then show ?thesis unfolding exact .
qed
have head-bound:
  wp-event m ?E s ≤ hash-relation-budget-value b q
  using m-budget unfolding hash-relation-budget-def by blast
have tail-bound:
  wp m ?Tail s ≤ hash-relation-budget-value b r
proof (rule wp-le-const-on-support)
  fix out
  assume support: out ∈ set-dist (execute m s)
  show ?Tail out ≤ hash-relation-budget-value b r
  proof (cases out)
    case None
    then show ?thesis
      unfolding hash-relation-budget-value-def by simp
  next
  case (Some xt)
  then obtain x t where xt: xt = (x, t)
  by (cases xt) simp
  have tail:
    wp-event (k x) (hash-relation-hit-event R t) t ≤
      hash-relation-budget-value b r
    using k-budget[of x] unfolding hash-relation-budget-def by blast
  show ?thesis using Some xt tail by simp
qed
qed
have wp-event (m ≫ k) ?E s ≤ wp m ?R s
  by (rule split)
also have ... = wp-event m ?E s + wp m ?Tail s
  unfolding wp-event-def wp-def dist-expect-def
  by (simp add: sum.distrib algebra-simps
    hash-relation-hit-event-def)
also have ... ≤

```

$\text{hash-relation-budget-value } b \ q + \text{hash-relation-budget-value } b \ r$
 by (intro add-mono head-bound tail-bound)
 also have ... = $\text{hash-relation-budget-value } b \ (q + r)$
 by (rule hash-relation-budget-value-add)
 finally show
 $\text{wp-event } (m \ggg k) (\text{hash-relation-hit-event } R \ s) \ s \leq$
 $\text{hash-relation-budget-value } b \ (q + r) .$
 qed

lemma *hash-relation-budget-bind-on-outcomes:*
fixes $m :: ('x, 'f \text{ protocol-channel}) \text{ state-monad}$
and $k :: 'x \Rightarrow ('y, 'f \text{ protocol-channel}) \text{ state-monad}$
assumes *m-ext:* $\text{hash-extension-preserving } m$
and *m-budget:* $\text{hash-relation-budget } R \ b \ q \ m$
and *k-ext:*
 $\bigwedge s \ x \ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \Rightarrow$
 $\text{hash-extension-preserving } (k \ x)$
and *k-budget:*
 $\bigwedge s \ x \ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \Rightarrow$
 $\text{hash-relation-budget } R \ b \ r \ (k \ x)$
shows $\text{hash-relation-budget } R \ b \ (q + r) \ (m \ggg k)$
unfolding *hash-relation-budget-def*
proof (intro allI)
fix $s :: 'f \text{ protocol-channel}$
let $?E = \text{hash-relation-hit-event } R \ s$
let $?Tail =$
 $\lambda \text{out}. \text{case out of}$
 $\text{None} \Rightarrow 0$
 $| \text{Some } (x, t) \Rightarrow$
 $\text{if } \text{hash-relation-hit } R \ s \ t \text{ then } 0$
 $\text{else } \text{wp-event } (k \ x) (\text{hash-relation-hit-event } R \ t) \ t$
let $?R = \lambda \text{out}. (\text{if } ?E \ \text{out then } 1 \text{ else } 0) + ?Tail \ \text{out}$
have *split:*
 $\text{wp-event } (m \ggg k) \ ?E \ s \leq \text{wp } m \ ?R \ s$
proof –
have *exact:*
 $\text{wp-event } (m \ggg k) \ ?E \ s =$
 $\text{wp } m$
 $(\lambda \text{out}. \text{case out of}$
 $\text{None} \Rightarrow 0$
 $| \text{Some } (x, t) \Rightarrow \text{wp-event } (k \ x) \ ?E \ t) \ s$
unfolding *wp-event-def hash-relation-hit-event-def*
by (*simp add: wpsimps*)
have $\text{wp } m$
 $(\lambda \text{out}. \text{case out of}$
 $\text{None} \Rightarrow 0$
 $| \text{Some } (x, t) \Rightarrow \text{wp-event } (k \ x) \ ?E \ t) \ s$
 $\leq \text{wp } m \ ?R \ s$
proof (rule *wp-mono-on-support*)

```

fix out
assume support: out  $\in$  set-dist (execute m s)
show (case out of
  None  $\Rightarrow$  0
  | Some (x, t)  $\Rightarrow$  wp-event (k x) ?E t  $\leq$  ?R out
proof (cases out)
  case None
  then show ?thesis
    unfolding hash-relation-hit-event-def by simp
next
  case (Some xt)
  then obtain x t where xt: xt = (x, t)
    by (cases xt) simp
  have m-out: Some (x, t)  $\in$  set-dist (execute m s)
    using support Some xt by simp
  have ext-st: s  $\leq$  t
    using m-ext m-out unfolding hash-extension-preserving-def by blast
  have step:
    wp-event (k x) ?E t  $\leq$ 
      (if hash-relation-hit R s t then 1
       else wp-event (k x) (hash-relation-hit-event R t) t)
    by (rule hash-relation-hit-event-step-bound
      [OF ext-st k-ext[OF m-out]])
  show ?thesis
    using Some xt step
    unfolding hash-relation-hit-event-def
    by (cases hash-relation-hit R s t) simp-all
  qed
qed
then show ?thesis unfolding exact .
qed
have head-bound:
  wp-event m ?E s  $\leq$  hash-relation-budget-value b q
  using m-budget unfolding hash-relation-budget-def by blast
have tail-bound:
  wp m ?Tail s  $\leq$  hash-relation-budget-value b r
proof (rule wp-le-const-on-support)
  fix out
  assume support: out  $\in$  set-dist (execute m s)
  show ?Tail out  $\leq$  hash-relation-budget-value b r
  proof (cases out)
  case None
  then show ?thesis
    unfolding hash-relation-budget-value-def by simp
next
  case (Some xt)
  then obtain x t where xt: xt = (x, t)
    by (cases xt) simp
  have m-out: Some (x, t)  $\in$  set-dist (execute m s)

```

```

    using support Some xt by simp
  have tail:
    wp-event (k x) (hash-relation-hit-event R t) t ≤
      hash-relation-budget-value b r
    using k-budget[OF m-out] unfolding hash-relation-budget-def
    by blast
  show ?thesis using Some xt tail by simp
qed
qed
have wp-event (m ≫ k) ?E s ≤ wp m ?R s
  by (rule split)
also have ... = wp-event m ?E s + wp m ?Tail s
  unfolding wp-event-def wp-def dist-expect-def
  by (simp add: sum.distrib algebra-simps
      hash-relation-hit-event-def)
also have ... ≤
  hash-relation-budget-value b q + hash-relation-budget-value b r
  by (intro add-mono head-bound tail-bound)
also have ... = hash-relation-budget-value b (q + r)
  by (rule hash-relation-budget-value-add)
finally show
  wp-event (m ≫ k) (hash-relation-hit-event R s) s ≤
    hash-relation-budget-value b (q + r) .
qed

lemma hash-relation-program-bind:
  assumes m: hash-relation-program R b q m
  and k:  $\bigwedge x. \text{hash-relation-program } R \text{ b r } (k \ x)$ 
  shows hash-relation-program R b (q + r) (m ≫ k)
  unfolding hash-relation-program-def
proof
  show hash-extension-preserving (m ≫ k)
    by (rule hash-extension-preserving-bind)
    (use m k in ⟨simp-all add: hash-relation-program-def⟩)
  show hash-relation-budget R b (q + r) (m ≫ k)
    by (rule hash-relation-budget-bind)
    (use m k in ⟨simp-all add: hash-relation-program-def⟩)
qed

lemma hash-relation-program-bind-on-outcomes:
  fixes m :: ('x, 'f protocol-channel) state-monad
  and k :: 'x ⇒ ('y, 'f protocol-channel) state-monad
  assumes m: hash-relation-program R b q m
  and k:
     $\bigwedge s \ x \ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$ 
    hash-relation-program R b r (k x)
  shows hash-relation-program R b (q + r) (m ≫ k)
  unfolding hash-relation-program-def
proof

```

```

show hash-extension-preserving ( $m \gg k$ )
  by (rule hash-extension-preserving-bind-on-outcomes)
    (use  $m$   $k$  in  $\langle \text{simp-all add: hash-relation-program-def} \rangle$ )
show hash-relation-budget  $R$   $b$  ( $q + r$ ) ( $m \gg k$ )
  by (rule hash-relation-budget-bind-on-outcomes)
    (use  $m$   $k$  in  $\langle \text{simp-all add: hash-relation-program-def} \rangle$ )
qed

lemma hash-relation-program-zero:
  assumes preserving: hash-map-preserving  $m$ 
  shows hash-relation-program  $R$   $b$   $0$   $m$ 
  unfolding hash-relation-program-def hash-relation-budget-def
proof
  show hash-extension-preserving  $m$ 
    by (rule hash-map-preserving-imp-hash-extension-preserving[OF preserving])
  show  $\forall s.$ 
    wp-event  $m$  (hash-relation-hit-event  $R$   $s$ )  $s \leq$ 
      hash-relation-budget-value  $b$   $0$ 
  proof
    fix  $s$ 
    have no-event-support:
       $\bigwedge \text{out. out} \in \text{set-dist (execute } m \text{ } s) \implies$ 
         $\neg \text{hash-relation-hit-event } R \text{ } s \text{ out}$ 
    proof –
      fix  $\text{out}$ 
      assume support:  $\text{out} \in \text{set-dist (execute } m \text{ } s)$ 
      show  $\neg \text{hash-relation-hit-event } R \text{ } s \text{ out}$ 
      proof (cases  $\text{out}$ )
        case None
        then show ?thesis
          unfolding hash-relation-hit-event-def by simp
        next
          case (Some  $xt$ )
          then obtain  $x$   $t$  where  $xt: xt = (x, t)$ 
            by (cases  $xt$ ) simp
          have same:  $\text{HashMap } t = \text{HashMap } s$ 
            using preserving support Some  $xt$ 
            unfolding hash-map-preserving-def by blast
          show ?thesis
            using same Some  $xt$ 
            unfolding hash-relation-hit-event-def hash-relation-hit-def
            by auto
          qed
        qed
      have no-event:
         $\bigwedge \text{out. out} \in \text{dom (dist (execute } m \text{ } s)) \implies$ 
           $\neg \text{hash-relation-hit-event } R \text{ } s \text{ out}$ 
        using no-event-support unfolding set-dist-def by blast
      have wp-event  $m$  (hash-relation-hit-event  $R$   $s$ )  $s = 0$ 

```

```

    unfolding wp-event-def wp-def dist-expect-def
    by (intro sum.neutral) (simp add: no-event)
  then show
    wp-event m (hash-relation-hit-event R s) s ≤
      hash-relation-budget-value b 0
    unfolding hash-relation-budget-value-def by simp
  qed
qed

```

```

lemma hash-relation-program-hash:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
    hash-relation-program R b 1
    (hash x :: ('f, 'f protocol-channel) state-monad)
  unfolding hash-relation-program-def hash-relation-budget-def
proof
  show hash-extension-preserving
    (hash x :: ('f, 'f protocol-channel) state-monad)
  by (rule hash-extension-preserving-hash)
  show  $\forall s.$ 
    wp-event (hash x) (hash-relation-hit-event R s) s ≤
      hash-relation-budget-value b 1
proof
  fix s :: 'f protocol-channel
  have
    wp-event (hash x) (hash-relation-hit-event R s) s ≤
      nnreal (card {y. R x y}) / nnreal size
    unfolding wp-hash-relation-hit by simp
  also have ... ≤ nnreal b / nnreal size
  by (rule nnreal-nat-divide-right-mono)
    (rule fibers)
  also have ... = hash-relation-budget-value b 1
  unfolding hash-relation-budget-value-def by simp
  finally show
    wp-event (hash x) (hash-relation-hit-event R s) s ≤
      hash-relation-budget-value b 1 .
  qed
qed

```

```

lemma hash-relation-program-mono:
  assumes qr:  $q \leq r$ 
  and program: hash-relation-program R b q m
  shows hash-relation-program R b r m
  unfolding hash-relation-program-def hash-relation-budget-def
proof
  show hash-extension-preserving m
  using program unfolding hash-relation-program-def by simp
  show  $\forall s.$ 
    wp-event m (hash-relation-hit-event R s) s ≤

```

```

      hash-relation-budget-value b r
proof
  fix s
  have
    wp-event m (hash-relation-hit-event R s) s ≤
      hash-relation-budget-value b q
  using program
  unfolding hash-relation-program-def hash-relation-budget-def by blast
  also have ... ≤ hash-relation-budget-value b r
  by (rule hash-relation-budget-value-mono[OF qr])
  finally show
    wp-event m (hash-relation-hit-event R s) s ≤
      hash-relation-budget-value b r .
qed
qed

lemma controlled-ro-program-relation:
  assumes controlled: controlled-ro-program q m
  and fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows hash-relation-program R b q m
  using controlled
proof (induction rule: controlled-ro-program.induct)
  case (Return x)
  show ?case
  by (rule hash-relation-program-zero)
    (rule hash-map-preserving-return)
next
  case Fail
  show ?case
  by (rule hash-relation-program-zero)
    (rule hash-map-preserving-throw)
next
  case (Sample d)
  show ?case
  by (rule hash-relation-program-zero)
    (rule hash-map-preserving-state-independent-sample)
next
  case (Query q k x)
  have tail:  $\bigwedge y. \text{hash-relation-program } R \ b \ q \ (k \ y)$ 
  using Query.IH fibers by blast
  have hash-relation-program R b (1 + q)
    ((hash x :: ('f, 'f protocol-channel) state-monad)  $\gg=$  k)
  by (rule hash-relation-program-bind)
    (rule hash-relation-program-hash[OF fibers], rule tail)
  then show ?case by simp
next
  case (Bind q m r k)
  have tail:  $\bigwedge x. \text{hash-relation-program } R \ b \ r \ (k \ x)$ 
  using Bind.IH(2) by blast

```

```

show ?case
  by (rule hash-relation-program-bind[OF Bind.IH(1) tail])
next
  case (Weaken q m r)
  show ?case
    by (rule hash-relation-program-mono[OF Weaken.hyps(2)
      Weaken.IH])
qed

theorem controlled-ro-program-relation-bound:
  assumes controlled: controlled-ro-program q m
  and fibers:  $\bigwedge x. \text{card } \{y. R\ x\ y\} \leq b$ 
  shows
    wp-event m (hash-relation-hit-event R s)  $s \leq$ 
      nnreal (q * b) / nnreal size
proof –
  have program: hash-relation-program R b q m
    by (rule controlled-ro-program-relation[OF controlled fibers])
  have budget: hash-relation-budget R b q m
    using program unfolding hash-relation-program-def by simp
  show ?thesis
    using budget unfolding hash-relation-budget-def
      hash-relation-budget-value-def by blast
qed

end

end

```

```

theory Security-Experiment
  imports
    Soundness-Oracle
begin

```

```

type-synonym 'f semantic-adversary =
  ('f list, 'f protocol-channel) state-monad

```

```

context soundness
begin

```

A semantic adversary is an arbitrary probabilistic state-monad program that returns a complete proof transcript. Admissibility states the random-oracle laws needed by the security proof: oracle consistency, bounded range growth, bounded collisions, and bounded hits on every fixed target set.

This interface deliberately does not prescribe an attacker programming language. A future capability-safe command language may expose only probabilistic computation and random-oracle queries, with a refinement theorem showing that every query-bounded

command program satisfies *admissible-adversary*.

definition *adversary-initial-state* :: 'f protocol-channel

where

adversary-initial-state =

($\langle \text{HashMap} = \text{fmempty}, \text{PState} = 0, \text{PTranscript} = [],$
 $\text{PTraceFriCounter} = 0, \text{PCompositionFriCounter} = 0,$
 $\text{PAlphaCounter} = 0, \text{PQueryCounter} = 0 \rangle$)

definition *admissible-adversary*

:: nat $\Rightarrow$ 'f semantic-adversary $\Rightarrow$ bool

where

admissible-adversary q A $\longleftrightarrow$

hash-extension-preserving A $\wedge$

hash-range-budget q A $\wedge$

hash-collision-budget q A $\wedge$

($\forall B. \text{hash-target-program } B \text{ q } A$)

lemma *adversary-initial-state-simps*[simp]:

HashMap adversary-initial-state = *fmempty*

PState adversary-initial-state = 0

PTranscript adversary-initial-state = []

PTraceFriCounter adversary-initial-state = 0

PCompositionFriCounter adversary-initial-state = 0

PAlphaCounter adversary-initial-state = 0

PQueryCounter adversary-initial-state = 0

unfolding *adversary-initial-state-def* **by** *simp-all*

lemma *adversary-initial-state-eq-verifier-initial-state*:

adversary-initial-state = *verifier-initial-state* []

unfolding *adversary-initial-state-def* *verifier-initial-state-def* **by** *simp*

lemma *admissible-adversary-extension*:

assumes *admissible-adversary* q A

shows *hash-extension-preserving* A

using *assms* **unfolding** *admissible-adversary-def* **by** *simp*

lemma *admissible-adversary-range*:

assumes *admissible-adversary* q A

shows *hash-range-budget* q A

using *assms* **unfolding** *admissible-adversary-def* **by** *simp*

lemma *admissible-adversary-collision*:

assumes *admissible-adversary* q A

shows *hash-collision-budget* q A

using *assms* **unfolding** *admissible-adversary-def* **by** *simp*

lemma *admissible-adversary-target*:

assumes *admissible-adversary* q A

shows *hash-target-program* B q A

using *assms* **unfolding** *admissible-adversary-def* **by** *simp*

definition *verifier-state-from-adversary*

$:: 'f \text{ protocol-channel} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ protocol-channel}$

where

verifier-state-from-adversary *s tr* =
 $(\text{HashMap} = \text{HashMap } s, \text{PState} = 0, \text{PTranscript} = tr,$
 $\text{PTraceFriCounter} = 0, \text{PCompositionFriCounter} = 0,$
 $\text{PAlphaCounter} = 0, \text{PQueryCounter} = 0)$

lemma *verifier-state-from-adversary-simps*[*simp*]:

HashMap (*verifier-state-from-adversary* *s tr*) = *HashMap* *s*
PState (*verifier-state-from-adversary* *s tr*) = 0
PTranscript (*verifier-state-from-adversary* *s tr*) = *tr*
PTraceFriCounter (*verifier-state-from-adversary* *s tr*) = 0
PCompositionFriCounter (*verifier-state-from-adversary* *s tr*) = 0
PAlphaCounter (*verifier-state-from-adversary* *s tr*) = 0
PQueryCounter (*verifier-state-from-adversary* *s tr*) = 0
unfolding *verifier-state-from-adversary-def* **by** *simp-all*

lemma *verifier-state-from-adversary-empty*:

verifier-state-from-adversary *adversary-initial-state* *tr* =
verifier-initial-state *tr*
unfolding *verifier-state-from-adversary-def* *adversary-initial-state-def*
verifier-initial-state-def
by *simp*

lemma *verifier-state-from-adversary-hash-extends*:

assumes $s \leq t$
shows *verifier-state-from-adversary* *s tr* $\leq$
verifier-state-from-adversary *t tr*
using *assms*
unfolding *verifier-state-from-adversary-def* *less-eq-hash-ext-def*
less-eq-fmap-def
by *simp*

definition *verifier-state-transfer*

$:: 'f \text{ list} \Rightarrow (\text{unit}, 'f \text{ protocol-channel}) \text{ state-monad}$

where

verifier-state-transfer *tr* =
 $\text{get} \gg (\lambda s. \text{put } (\text{verifier-state-from-adversary } s \text{ tr}))$

lemma *verifier-state-transfer-outcomeE*:

assumes
 $\text{Some } ((), t) \in \text{set-dist } (\text{execute } (\text{verifier-state-transfer } tr) \text{ } s)$
shows $t = \text{verifier-state-from-adversary } s \text{ tr}$
using *assms* **unfolding** *verifier-state-transfer-def*
by (*auto elim!*: *set-dist-bindE*)

```

lemma hash-map-preserving-verifier-state-transfer:
  hash-map-preserving (verifier-state-transfer tr)
  unfolding hash-map-preserving-def
  using verifier-state-transfer-outcomeE by fastforce

lemma hash-map-preserving-imp-hash-range-budget-zero-semantic:
  fixes m :: ('r, 'f protocol-channel) state-monad
  assumes preserving: hash-map-preserving m
  shows hash-range-budget 0 m
  using preserving
  unfolding hash-range-budget-def hash-map-preserving-def
    hash-map-output-values-def
  by auto

lemma hash-map-preserving-imp-hash-collision-budget-zero-semantic:
  fixes m :: ('r, 'f protocol-channel) state-monad
  assumes preserving: hash-map-preserving m
  shows hash-collision-budget 0 m
  unfolding hash-collision-budget-def
proof (intro allI impI)
  fix s :: 'f protocol-channel
  assume clean:  $\neg$  hash-map-output-collision s
  have no-event:
     $\bigwedge out. out \in \text{set-dist } (\text{execute } m \ s) \implies$ 
       $\neg \text{hash-new-collision-event } s \ out$ 
  proof –
    fix out
    assume support:  $out \in \text{set-dist } (\text{execute } m \ s)$ 
    show  $\neg \text{hash-new-collision-event } s \ out$ 
    proof (cases out)
      case None
      then show ?thesis
        unfolding hash-new-collision-event-def by simp
    next
      case (Some xt)
      then obtain x t where xt:  $xt = (x, t)$ 
      by (cases xt) simp
      have same:  $\text{HashMap } t = \text{HashMap } s$ 
      using preserving support Some xt
      unfolding hash-map-preserving-def by blast
      show ?thesis
        using clean same Some xt
        unfolding hash-new-collision-event-def
          hash-map-new-output-collision-def hash-map-output-collision-def
        by simp
    qed
  qed
  have no-event-dom:
     $\bigwedge out. out \in \text{dom } (\text{dist } (\text{execute } m \ s)) \implies$ 

```

```

    ¬ hash-new-collision-event s out
  using no-event unfolding set-dist-def by blast
have wp-event m (hash-new-collision-event s) s = 0
  unfolding wp-event-def wp-def dist-expect-def
  by (intro sum.neutral) (simp add: no-event-dom)
then show
  wp-event m (hash-new-collision-event s) s ≤
    hash-collision-budget-value
      (card (hash-map-output-values s)) 0
  unfolding hash-collision-budget-value-def by simp
qed

lemma hash-range-budget-verifier-state-transfer:
  hash-range-budget 0 (verifier-state-transfer tr)
by (rule hash-map-preserving-imp-hash-range-budget-zero-semantic)
   (rule hash-map-preserving-verifier-state-transfer)

lemma hash-collision-budget-verifier-state-transfer:
  hash-collision-budget 0 (verifier-state-transfer tr)
by (rule hash-map-preserving-imp-hash-collision-budget-zero-semantic)
   (rule hash-map-preserving-verifier-state-transfer)

lemma hash-range-budget-verifier-after-adversary:
  hash-range-budget verifier-hash-query-budget
    (verifier-state-transfer tr ≫ (λ-. verify-monad))
proof -
  have range:
    hash-range-budget (0 + verifier-hash-query-budget)
      (verifier-state-transfer tr ≫ (λ-. verify-monad))
  by (rule hash-range-budget-bind)
     (rule hash-range-budget-verifier-state-transfer,
        rule hash-range-budget-verify-monad)
  then show ?thesis by simp
qed

lemma hash-collision-budget-verifier-after-adversary:
  hash-collision-budget verifier-hash-query-budget
    (verifier-state-transfer tr ≫ (λ-. verify-monad))
proof -
  have collision:
    hash-collision-budget (0 + verifier-hash-query-budget)
      (verifier-state-transfer tr ≫ (λ-. verify-monad))
  by (rule hash-collision-budget-bind)
     (rule hash-range-budget-verifier-state-transfer,
        rule hash-collision-budget-verifier-state-transfer,
        rule hash-range-budget-verify-monad,
        rule hash-collision-budget-verify-monad)
  then show ?thesis by simp
qed

```

definition *security-experiment*
 $:: 'f \text{ semantic-adversary} \Rightarrow$
 $(\text{unit list}, 'f \text{ protocol-channel}) \text{ state-monad}$
where
 $\text{security-experiment } A =$
 $A \gg= (\lambda tr.$
 $\text{verifier-state-transfer } tr \gg= (\lambda -. \text{verify-monad}))$

definition *adversary-acceptance-probability*
 $:: 'f \text{ semantic-adversary} \Rightarrow \text{prob}$
where
 $\text{adversary-acceptance-probability } A =$
 $\text{wp-event } (\text{security-experiment } A)$
 $(\lambda out. \neg \text{Option.is-none out}) \text{ adversary-initial-state}$

lemma *security-experiment-outcomeE*:
assumes *outcome*:
 $\text{Some } (\text{result}, \text{final-state}) \in$
 $\text{set-dist } (\text{execute } (\text{security-experiment } A) \text{ initial-state})$
obtains *tr attacker-state* **where**
 $\text{Some } (tr, \text{attacker-state}) \in \text{set-dist } (\text{execute } A \text{ initial-state})$
 $\text{Some } (\text{result}, \text{final-state}) \in$
 set-dist
 $(\text{execute } \text{verify-monad}$
 $(\text{verifier-state-from-adversary } \text{attacker-state } tr))$
using *outcome*
unfolding *security-experiment-def* *verifier-state-transfer-def*
by (*auto elim!*: *set-dist-bindE* *intro*: *that*)

lemma *accepted-security-experimentE*:
assumes *outcome*:
 $out \in \text{set-dist}$
 $(\text{execute } (\text{security-experiment } A) \text{ adversary-initial-state})$
and *accepted*: $\neg \text{Option.is-none out}$
obtains *result final-state tr attacker-state* **where**
 $out = \text{Some } (\text{result}, \text{final-state})$
 $\text{Some } (tr, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } A \text{ adversary-initial-state})$
 $\text{Some } (\text{result}, \text{final-state}) \in$
 set-dist
 $(\text{execute } \text{verify-monad}$
 $(\text{verifier-state-from-adversary } \text{attacker-state } tr))$
proof –
from *accepted* **obtain** *result final-state* **where**
 $out: out = \text{Some } (\text{result}, \text{final-state})$
by (*cases out*) *auto*
from *security-experiment-outcomeE*[*OF outcome[unfolded out]*]
obtain *tr attacker-state* **where**

```

attacker:
  Some (tr, attacker-state) ∈
    set-dist (execute A adversary-initial-state)
and verifier:
  Some (result, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state tr))
  by blast
show ?thesis
  by (rule that[OF out attacker verifier])
qed

lemma attacker-failure-not-accepted[simp]:
  ¬ (¬ Option.is-none
    (None :: (unit list × 'f protocol-channel) option))
  by simp

lemma security-experiment-success-requires-attacker-success:
  assumes no-success:
    ∧ tr attacker-state.
    Some (tr, attacker-state) ∉ set-dist (execute A initial-state)
  shows
    Some (result, final-state) ∉
      set-dist (execute (security-experiment A) initial-state)
proof
  assume outcome:
    Some (result, final-state) ∈
      set-dist (execute (security-experiment A) initial-state)
  from security-experiment-outcomeE[OF outcome]
  obtain tr attacker-state where
    Some (tr, attacker-state) ∈ set-dist (execute A initial-state)
  by blast
  then show False
    using no-success[of tr attacker-state] by blast
qed

lemma admissible-adversary-outcome-extends:
  assumes admissible: admissible-adversary q A
  and outcome:
    Some (tr, attacker-state) ∈
      set-dist (execute A adversary-initial-state)
  shows adversary-initial-state ≤ attacker-state
  using admissible-adversary-extension[OF admissible] outcome
  unfolding hash-extension-preserving-def by blast

lemma hash-map-output-values-adversary-initial-state[simp]:
  hash-map-output-values adversary-initial-state = {}
  unfolding adversary-initial-state-eq-verifier-initial-state by simp

```

lemma *adversary-initial-state-no-output-collision*[simp]:
 $\neg$ *hash-map-output-collision adversary-initial-state*
unfolding *hash-map-output-collision-def* **by** *simp*

lemma *admissible-adversary-output-range-bound*:
assumes *admissible*: *admissible-adversary* q A
and *outcome*:
Some (tr , *attacker-state*) $\in$
set-dist (*execute* A *adversary-initial-state*)
shows *card* (*hash-map-output-values* *attacker-state*) $\leq q$
proof –
have
card (*hash-map-output-values* *attacker-state*) $\leq$
card (*hash-map-output-values* *adversary-initial-state*) + q
using *admissible-adversary-range*[*OF* *admissible*] *outcome*
unfolding *hash-range-budget-def* **by** *blast*
then show ?thesis **by** *simp*
qed

lemma *admissible-adversary-collision-bound*:
assumes *admissible*: *admissible-adversary* q A
shows
wp-event A (*hash-new-collision-event* *adversary-initial-state*)
adversary-initial-state $\leq$ *hash-collision-budget-value* 0 q
proof –
have *collision-budget*: *hash-collision-budget* q A
by (*rule* *admissible-adversary-collision*[*OF* *admissible*])
have
wp-event A (*hash-new-collision-event* *adversary-initial-state*)
adversary-initial-state $\leq$
hash-collision-budget-value
(*card* (*hash-map-output-values* *adversary-initial-state*)) q
using *collision-budget*
adversary-initial-state-no-output-collision
unfolding *hash-collision-budget-def* **by** *blast*
then show ?thesis **by** *simp*
qed

lemma *admissible-adversary-target-bound*:
assumes *admissible*: *admissible-adversary* q A
shows
wp-event A
(*hash-new-output-hit-event* B *adversary-initial-state*)
adversary-initial-state $\leq$ *hash-target-budget-value* B q
proof –
have *hash-target-budget* B q A
by (*rule* *hash-target-program-budget*)
(*rule* *admissible-adversary-target*[*OF* *admissible*])

then show ?thesis
 unfolding hash-target-budget-def by blast
 qed

lemma hash-range-budget-security-experiment:
 assumes admissible: admissible-adversary q A
 shows
 hash-range-budget ($q + \text{verifier-hash-query-budget}$)
 (security-experiment A)
 unfolding security-experiment-def
 by (rule hash-range-budget-bind)
 (rule admissible-adversary-range[OF admissible],
 rule hash-range-budget-verifier-after-adversary)

lemma hash-collision-budget-security-experiment:
 assumes admissible: admissible-adversary q A
 shows
 hash-collision-budget ($q + \text{verifier-hash-query-budget}$)
 (security-experiment A)
 unfolding security-experiment-def
 by (rule hash-collision-budget-bind)
 (rule admissible-adversary-range[OF admissible],
 rule admissible-adversary-collision[OF admissible],
 rule hash-range-budget-verifier-after-adversary,
 rule hash-collision-budget-verifier-after-adversary)

lemma security-experiment-hash-new-collision-bound:
 assumes admissible: admissible-adversary q A
 shows
 wp-event (security-experiment A)
 (hash-new-collision-event adversary-initial-state)
 adversary-initial-state $\leq$
 hash-collision-budget-value 0 ($q + \text{verifier-hash-query-budget}$)

proof –
 have collision-budget:
 hash-collision-budget ($q + \text{verifier-hash-query-budget}$)
 (security-experiment A)
 by (rule hash-collision-budget-security-experiment[OF admissible])
 have
 wp-event (security-experiment A)
 (hash-new-collision-event adversary-initial-state)
 adversary-initial-state $\leq$
 hash-collision-budget-value
 (card (hash-map-output-values adversary-initial-state))
 ($q + \text{verifier-hash-query-budget}$)
 using collision-budget adversary-initial-state-no-output-collision
 unfolding hash-collision-budget-def by blast
 then show ?thesis by simp
 qed


```

lemma hash-map-output-collision-bad-imp-security-new-collision:
  assumes bad: hash-map-output-collision-bad adversary-initial-state out
  shows hash-new-collision-event adversary-initial-state out
proof –
  have new-bad: hash-map-new-output-collision-bad adversary-initial-state out
    by (rule hash-map-output-collision-bad-imp-new-collision-bad-if-initial-clean)
      (use bad in simp-all)
  show ?thesis
    by (rule hash-map-new-output-collision-bad-imp-hash-new-collision-event
      [OF new-bad])
qed

```

```

lemma security-experiment-hash-map-output-collision-bad-bound:
  assumes admissible: admissible-adversary q A
  shows
    wp-event (security-experiment A)
      (hash-map-output-collision-bad adversary-initial-state)
      adversary-initial-state  $\leq$ 
      hash-collision-budget-value 0 (q + verifier-hash-query-budget)
proof –
  have wp-event (security-experiment A)
    (hash-map-output-collision-bad adversary-initial-state)
    adversary-initial-state  $\leq$ 
    wp-event (security-experiment A)
      (hash-new-collision-event adversary-initial-state)
      adversary-initial-state
    by (rule wp-event-mono)
      (rule hash-map-output-collision-bad-imp-security-new-collision)
  also have ...  $\leq$ 
    hash-collision-budget-value 0 (q + verifier-hash-query-budget)
    by (rule security-experiment-hash-new-collision-bound[OF admissible])
  finally show ?thesis .
qed

```

end

end

```

theory Staged-Security-Experiment-Core
  imports Controlled-RO Security-Experiment
begin

```

Core staged-adversary records, transcript builders, and experiment projections.

```

record staged-budgets =
  trace-root-budget :: nat
  trace-fri-budgets :: nat list
  trace-final-budget :: nat

```

```

degree-budget :: nat
composition-fri-budgets :: nat list
composition-final-budget :: nat
query-opening-budgets :: nat list

record 'f staged-adversary =
  trace-root-stage :: ('f, 'f protocol-channel) state-monad
  trace-fri-root-stage ::
    nat ⇒ 'f list ⇒ ('f, 'f protocol-channel) state-monad
  trace-final-stage ::
    'f list ⇒ ('f, 'f protocol-channel) state-monad
  degree-stage ::
    'f list ⇒ ('f, 'f protocol-channel) state-monad
  composition-fri-root-stage ::
    'f ⇒ nat ⇒ 'f list ⇒
      ('f, 'f protocol-channel) state-monad
  composition-final-stage ::
    'f ⇒ 'f list ⇒ ('f, 'f protocol-channel) state-monad
  query-opening-stage ::
    nat ⇒ 'f ⇒ ('f list, 'f protocol-channel) state-monad

record 'f staged-proof-data =
  staged-trace-root :: 'f
  staged-trace-fri-roots :: 'f list
  staged-trace-fri-challenges :: 'f list
  staged-trace-final :: 'f
  staged-alphas :: 'f list
  staged-degree :: 'f
  staged-composition-fri-roots :: 'f list
  staged-composition-fri-challenges :: 'f list
  staged-composition-final :: 'f
  staged-query-chunks :: 'f list list

record 'f staged-composition-prefix-data =
  scp-trace-root :: 'f
  scp-trace-fri-roots :: 'f list
  scp-trace-fri-challenges :: 'f list
  scp-trace-final :: 'f
  scp-alphas :: 'f list
  scp-degree :: 'f
  scp-composition-fri-roots :: 'f list
  scp-composition-fri-challenges :: 'f list

record 'f staged-query-prefix-data =
  sqp-trace-root :: 'f
  sqp-trace-fri-roots :: 'f list
  sqp-trace-fri-challenges :: 'f list
  sqp-trace-final :: 'f
  sqp-alphas :: 'f list

```

```

sqp-degree :: 'f
sqp-composition-fri-roots :: 'f list
sqp-composition-fri-challenges :: 'f list
sqp-composition-final :: 'f
sqp-query-chunks :: 'f list list

context soundness
begin

lemma wp-event-bind-return-map:
  fixes m :: ('x, 'f protocol-channel) state-monad
  shows
    wp-event (m  $\gg$  (λx. return (f x))) P s =
      wp-event m (λout. case out of None  $\Rightarrow$  P None
        | Some (x, t)  $\Rightarrow$  P (Some (f x, t))) s
  proof -
    have indicator-eq:
      (λout. case out of None  $\Rightarrow$  if P None then 1 else 0
        | Some (x, t)  $\Rightarrow$  if P (Some (f x, t)) then 1 else 0) =
      (λout. if (case out of None  $\Rightarrow$  P None
        | Some (x, t)  $\Rightarrow$  P (Some (f x, t))) then 1 else 0)
    by (rule ext) (simp split: option.splits prod.splits)
  show ?thesis
    unfolding wp-event-def
    apply (subst wp-bind)
    apply (rule arg-cong[where f=λQ. wp m Q s])
    apply (rule ext)
    apply (simp add: wp-return indicator-eq
      split: option.splits prod.splits)
  done
qed

lemma wp-event-bind-mono-cont:
  fixes m :: ('x, 'f protocol-channel) state-monad
  and k :: 'x  $\Rightarrow$  ('y, 'f protocol-channel) state-monad
  and l :: 'x  $\Rightarrow$  ('z, 'f protocol-channel) state-monad
  assumes none: P None  $\Longrightarrow$  Q None
  and cont:
     $\bigwedge x t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \Longrightarrow$ 
      wp-event (k x) P t  $\leq$  wp-event (l x) Q t
  shows wp-event (m  $\gg$  k) P s  $\leq$  wp-event (m  $\gg$  l) Q s
  proof -
    have left:
      wp-event (m  $\gg$  k) P s =
      wp m (λout. case out of
        None  $\Rightarrow$  if P None then 1 else 0
        | Some (x, t)  $\Rightarrow$  wp-event (k x) P t) s
    unfolding wp-event-def by (simp add: wpsimps)
    have right:

```

```

wp-event (m  $\gg$  l) Q s =
  wp m ( $\lambda$ out. case out of
    None  $\Rightarrow$  if Q None then 1 else 0
    | Some (x, t)  $\Rightarrow$  wp-event (l x) Q t) s
  unfolding wp-event-def by (simp add: wpsimps)
show ?thesis
  unfolding left right
proof (rule wp-mono-on-support)
  fix out
  assume out: out  $\in$  set-dist (execute m s)
  show (case out of None  $\Rightarrow$  if P None then 1 else 0
    | Some (x, t)  $\Rightarrow$  wp-event (k x) P t)
     $\leq$  (case out of None  $\Rightarrow$  if Q None then 1 else 0
    | Some (x, t)  $\Rightarrow$  wp-event (l x) Q t)
  proof (cases out)
    case None
    then show ?thesis
      using none by (cases P None) simp-all
  next
    case (Some xt)
    then obtain x t where xt: xt = (x, t)
      by (cases xt) auto
    show ?thesis
      using cont[of x t] out Some xt by simp
  qed
qed
qed

lemma wp-event-assert-bind-le:
  fixes k :: unit  $\Rightarrow$  ('x, 'f protocol-channel) state-monad
  assumes none:  $\neg$  P None
  shows wp-event (assert b  $\gg$  k) P s  $\leq$  wp-event (k ()) P s
proof (cases b)
  case True
  then show ?thesis
    unfolding assert-def by simp
next
  case False
  then show ?thesis
    using none unfolding assert-def
    by (simp add: wp-event-def wpsimps throw-no-outcome)
qed

definition staged-budget-wellformed :: staged-budgets  $\Rightarrow$  bool
where
  staged-budget-wellformed budgets  $\longleftrightarrow$ 
    length (trace-fri-budgets budgets) = ceil-log clength  $\wedge$ 
    length (composition-fri-budgets budgets) =
      ceil-log (maxDegree + 1)  $\wedge$ 

```

$\text{length } (\text{query-opening-budgets } \text{budgets}) = \text{rounds}$

definition *staged-adversary-controlled*

$:: \text{staged-budgets} \Rightarrow 'f \text{ staged-adversary} \Rightarrow \text{bool}$

where

$\text{staged-adversary-controlled } \text{budgets } A \longleftrightarrow$
 $\text{controlled-ro-program } (\text{trace-root-budget } \text{budgets})$
 $(\text{trace-root-stage } A) \wedge$
 $(\forall i \text{ bs.}$
 $i < \text{length } (\text{trace-fri-budgets } \text{budgets}) \longrightarrow$
 $\text{controlled-ro-program } (\text{trace-fri-budgets } \text{budgets } ! i)$
 $(\text{trace-fri-root-stage } A i \text{ bs})) \wedge$
 $(\forall \text{ bs.}$
 $\text{controlled-ro-program } (\text{trace-final-budget } \text{budgets})$
 $(\text{trace-final-stage } A \text{ bs})) \wedge$
 $(\forall \text{ as.}$
 $\text{controlled-ro-program } (\text{degree-budget } \text{budgets})$
 $(\text{degree-stage } A \text{ as})) \wedge$
 $(\forall \text{ dg } i \text{ bs.}$
 $i < \text{length } (\text{composition-fri-budgets } \text{budgets}) \longrightarrow$
 $\text{controlled-ro-program } (\text{composition-fri-budgets } \text{budgets } ! i)$
 $(\text{composition-fri-root-stage } A \text{ dg } i \text{ bs})) \wedge$
 $(\forall \text{ dg } \text{ bs.}$
 $\text{controlled-ro-program } (\text{composition-final-budget } \text{budgets})$
 $(\text{composition-final-stage } A \text{ dg } \text{ bs})) \wedge$
 $(\forall i \text{ raw.}$
 $i < \text{length } (\text{query-opening-budgets } \text{budgets}) \longrightarrow$
 $\text{controlled-ro-program } (\text{query-opening-budgets } \text{budgets } ! i)$
 $(\text{query-opening-stage } A i \text{ raw}))$

definition *staged-attacker-query-budget* $:: \text{staged-budgets} \Rightarrow \text{nat}$

where

$\text{staged-attacker-query-budget } \text{budgets} =$
 $\text{trace-root-budget } \text{budgets} +$
 $\text{sum-list } (\text{trace-fri-budgets } \text{budgets}) +$
 $\text{trace-final-budget } \text{budgets} +$
 $\text{degree-budget } \text{budgets} +$
 $\text{sum-list } (\text{composition-fri-budgets } \text{budgets}) +$
 $\text{composition-final-budget } \text{budgets} +$
 $\text{sum-list } (\text{query-opening-budgets } \text{budgets})$

definition *staged-challenge-query-budget* $:: \text{nat}$

where

$\text{staged-challenge-query-budget} =$
 $\text{ceil-log } \text{clength} + \text{length } \text{spec} +$
 $\text{ceil-log } (\text{maxDegree} + 1) + \text{rounds}$

definition *staged-trace-fri-search-queries*

$:: \text{staged-budgets} \Rightarrow \text{nat} \Rightarrow \text{nat}$

where

staged-trace-fri-search-queries budgets i =
 trace-root-budget budgets +
 sum-list (take (Suc i) (trace-fri-budgets budgets)) + i

definition *staged-alpha-search-queries*

:: staged-budgets $\Rightarrow$ nat $\Rightarrow$ nat

where

staged-alpha-search-queries budgets i =
 trace-root-budget budgets +
 sum-list (trace-fri-budgets budgets) +
 trace-final-budget budgets +
 ceil-log clength + i

definition *staged-composition-fri-search-queries*

:: staged-budgets $\Rightarrow$ nat $\Rightarrow$ nat

where

staged-composition-fri-search-queries budgets i =
 trace-root-budget budgets +
 sum-list (trace-fri-budgets budgets) +
 trace-final-budget budgets +
 degree-budget budgets +
 sum-list (take (Suc i) (composition-fri-budgets budgets)) +
 ceil-log clength + length spec + i

definition *staged-composition-fri-vector-search-queries*

:: staged-budgets $\Rightarrow$ nat

where

staged-composition-fri-vector-search-queries budgets =
 staged-alpha-search-queries budgets 0 +
 length spec +
 degree-budget budgets +
 sum-list (composition-fri-budgets budgets) +
 ceil-log (maxDegree + 1)

definition *staged-query-search-queries*

:: staged-budgets $\Rightarrow$ nat $\Rightarrow$ nat

where

staged-query-search-queries budgets i =
 trace-root-budget budgets +
 sum-list (trace-fri-budgets budgets) +
 trace-final-budget budgets +
 degree-budget budgets +
 sum-list (composition-fri-budgets budgets) +
 composition-final-budget budgets +
 sum-list (take i (query-opening-budgets budgets)) +
 ceil-log clength + length spec +
 ceil-log (maxDegree + 1) + i

definition *staged-phase-relation-error*

$:: \text{nat} \Rightarrow \text{nat} \Rightarrow \text{prob}$

where

staged-phase-relation-error *fiber-bound* *query-bound* =
hash-relation-budget-value *fiber-bound* *query-bound*

definition *staged-phase-target-error*

$:: 'f \text{ set} \Rightarrow \text{nat} \Rightarrow \text{prob}$

where

staged-phase-target-error *B* *query-bound* =
hash-target-budget-value *B* *query-bound*

lemma *staged-phase-query-index-target-error-fraction-bound*:

assumes *raw-bound*: *query-index-raw-preimage-bound*

and *subset*: $B \subseteq \text{query-sample-space}$

shows

staged-phase-target-error (*query-index-raw-preimage* *B*) *q* $\leq$
 $\text{nnreal } q * (\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space}))$

proof –

have *raw*:

$\text{nnreal } (\text{card } (\text{query-index-raw-preimage } B)) / \text{nnreal size} \leq$
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space})$

using *raw-bound* *subset* **unfolding** *query-index-raw-preimage-bound-def*
by *blast*

have *eq*:

staged-phase-target-error (*query-index-raw-preimage* *B*) *q* =
 $\text{nnreal } q * (\text{nnreal } (\text{card } (\text{query-index-raw-preimage } B)) / \text{nnreal size})$

unfolding *staged-phase-target-error-def* *hash-target-budget-value-def*

by *transfer* (*simp add: field-simps*)

show *?thesis*

unfolding *eq* **by** (*rule mult-left-mono*) (*use raw in simp-all*)

qed

lemma *staged-phase-query-index-target-error-query-bound*:

assumes *raw-bound*: *query-index-raw-preimage-bound*

and *subset*: $B \subseteq \text{query-sample-space}$

and *frac*:

$\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space}) \leq C$

shows

staged-phase-target-error (*query-index-raw-preimage* *B*) *q* $\leq$
 $\text{nnreal } q * C$

proof –

have

staged-phase-target-error (*query-index-raw-preimage* *B*) *q* $\leq$
 $\text{nnreal } q * (\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space}))$

by (*rule staged-phase-query-index-target-error-fraction-bound*
 $[OF \text{ raw-bound subset}]$)

also have $\dots \leq \text{nnreal } q * C$

```

    by (rule mult-left-mono) (use frac in simp-all)
  finally show ?thesis .
qed

definition record-staged-message
  :: 'f  $\Rightarrow$  (unit, 'f protocol-channel) state-monad
  where
    record-staged-message x =
      modify
        ( $\lambda s. s \langle$ 
          PState := concat (PState s) x,
          PTranscript := PTranscript s @ [x]  $\rangle$ )

definition record-staged-messages
  :: 'f list  $\Rightarrow$  (unit, 'f protocol-channel) state-monad
  where
    record-staged-messages xs = mfold2 record-staged-message xs

primrec staged-trace-fri-program
  :: 'f staged-adversary  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  'f list  $\Rightarrow$ 
    ('f list  $\times$  'f list, 'f protocol-channel) state-monad
  where
    staged-trace-fri-program A i 0 bs = return ([], bs)
  | staged-trace-fri-program A i (Suc n) bs =
    do {
      root  $\leftarrow$  trace-fri-root-stage A i bs;
      record-staged-message root;
      b  $\leftarrow$  receive-trace-fri-challenge;
      (roots, bs')  $\leftarrow$  staged-trace-fri-program A (Suc i) n (bs @ [b]);
      return (root # roots, bs')
    }

primrec staged-alpha-program
  :: nat  $\Rightarrow$  ('f list, 'f protocol-channel) state-monad
  where
    staged-alpha-program 0 = return []
  | staged-alpha-program (Suc n) =
    do {
      a  $\leftarrow$  receive-alpha-challenge;
      record-staged-message a;
      as  $\leftarrow$  staged-alpha-program n;
      return (a # as)
    }

primrec staged-composition-fri-program
  :: 'f staged-adversary  $\Rightarrow$  'f  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$ 
    'f list  $\Rightarrow$ 
    ('f list  $\times$  'f list, 'f protocol-channel) state-monad
  where

```



```

    staged-composition-fri-program A dg i 0 bs = return ([], bs)
| staged-composition-fri-program A dg i (Suc n) bs =
  do {
    root ← composition-fri-root-stage A dg i bs;
    record-staged-message root;
    b ← receive-composition-fri-challenge;
    (roots, bs') ←
      staged-composition-fri-program A dg (Suc i) n (bs @ [b]);
    return (root # roots, bs')
  }

```

primrec *staged-query-program*
 :: 'f staged-adversary ⇒ nat ⇒ nat ⇒
 ('f list list, 'f protocol-channel) state-monad

where

```

    staged-query-program A i 0 = return []
| staged-query-program A i (Suc n) =
  do {
    raw ← receive-query-index-challenge;
    chunk ← query-opening-stage A i raw;
    record-staged-messages chunk;
    chunks ← staged-query-program A (Suc i) n;
    return (chunk # chunks)
  }

```

primrec *checked-staged-query-program*
 :: 'f staged-adversary ⇒ 'f list ⇒ 'f list ⇒
 nat ⇒ nat ⇒ ('f list list, 'f protocol-channel) state-monad

where

```

    checked-staged-query-program A trace-roots composition-roots i 0 =
      return []
| checked-staged-query-program A trace-roots composition-roots i (Suc n) =
  do {
    raw ← receive-query-index-challenge;
    let idx = index (to-nat raw);
    chunk ← query-opening-stage A i raw;
    assert
      (verifier-query-round-chunk idx trace-roots composition-roots chunk);
    record-staged-messages chunk;
    chunks ←
      checked-staged-query-program A trace-roots composition-roots
        (Suc i) n;
    return (chunk # chunks)
  }

```

definition *staged-transcript-program*

```

  :: 'f staged-adversary ⇒
    ('f staged-proof-data, 'f protocol-channel) state-monad
where

```

```

staged-transcript-program  $A =$ 
do {
   $fr \leftarrow \text{trace-root-stage } A;$ 
   $\text{record-staged-message } fr;$ 
   $(\text{trace-roots}, \text{trace-bs}) \leftarrow$ 
     $\text{staged-trace-fri-program } A \ 0 \ (\text{ceil-log } \text{clength}) \ [];$ 
   $\text{trace-final} \leftarrow \text{trace-final-stage } A \ \text{trace-bs};$ 
   $\text{record-staged-message } \text{trace-final};$ 
   $as \leftarrow \text{staged-alpha-program } (\text{length } \text{spec});$ 
   $dg \leftarrow \text{degree-stage } A \ as;$ 
   $\text{record-staged-message } dg;$ 
   $\text{let } \text{composition-rounds} = \text{ceil-log } (\text{to-nat } dg + 1);$ 
   $\text{assert } (\text{composition-rounds} \leq \text{ceil-log } (\text{maxDegree} + 1));$ 
   $(\text{composition-roots}, \text{composition-bs}) \leftarrow$ 
     $\text{staged-composition-fri-program } A \ dg \ 0 \ \text{composition-rounds} \ [];$ 
   $\text{composition-final} \leftarrow$ 
     $\text{composition-final-stage } A \ dg \ \text{composition-bs};$ 
   $\text{record-staged-message } \text{composition-final};$ 
   $\text{query-chunks} \leftarrow \text{staged-query-program } A \ 0 \ \text{rounds};$ 
  return
  ( $\text{staged-trace-root} = fr,$ 
    $\text{staged-trace-fri-roots} = \text{trace-roots},$ 
    $\text{staged-trace-fri-challenges} = \text{trace-bs},$ 
    $\text{staged-trace-final} = \text{trace-final},$ 
    $\text{staged-alphas} = as,$ 
    $\text{staged-degree} = dg,$ 
    $\text{staged-composition-fri-roots} = \text{composition-roots},$ 
    $\text{staged-composition-fri-challenges} = \text{composition-bs},$ 
    $\text{staged-composition-final} = \text{composition-final},$ 
    $\text{staged-query-chunks} = \text{query-chunks}$ )
}

```

definition *checked-staged-transcript-program*

```

:: 'f staged-adversary  $\Rightarrow$ 
  ('f staged-proof-data, 'f protocol-channel) state-monad

```

where

```

checked-staged-transcript-program  $A =$ 
do {
   $fr \leftarrow \text{trace-root-stage } A;$ 
   $\text{record-staged-message } fr;$ 
   $(\text{trace-roots}, \text{trace-bs}) \leftarrow$ 
     $\text{staged-trace-fri-program } A \ 0 \ (\text{ceil-log } \text{clength}) \ [];$ 
   $\text{trace-final} \leftarrow \text{trace-final-stage } A \ \text{trace-bs};$ 
   $\text{record-staged-message } \text{trace-final};$ 
   $as \leftarrow \text{staged-alpha-program } (\text{length } \text{spec});$ 
   $dg \leftarrow \text{degree-stage } A \ as;$ 
   $\text{record-staged-message } dg;$ 
   $\text{let } \text{composition-rounds} = \text{ceil-log } (\text{to-nat } dg + 1);$ 
   $\text{assert } (\text{composition-rounds} \leq \text{ceil-log } (\text{maxDegree} + 1));$ 

```

```

    (composition-roots, composition-bs) ←
      staged-composition-fri-program A dg 0 composition-rounds [];
    composition-final ←
      composition-final-stage A dg composition-bs;
    record-staged-message composition-final;
    query-chunks ←
      checked-staged-query-program A trace-roots composition-roots
        0 rounds;
    return
      (|staged-trace-root = fr,
        staged-trace-fri-roots = trace-roots,
        staged-trace-fri-challenges = trace-bs,
        staged-trace-final = trace-final,
        staged-alphas = as,
        staged-degree = dg,
        staged-composition-fri-roots = composition-roots,
        staged-composition-fri-challenges = composition-bs,
        staged-composition-final = composition-final,
        staged-query-chunks = query-chunks)
  }

```

definition *staged-alpha-prefix-program*

```

:: 'f staged-adversary ⇒
  ('f × 'f list × 'f list × 'f, 'f protocol-channel)
  state-monad

```

where

```

staged-alpha-prefix-program A =
  do {
    fr ← trace-root-stage A;
    record-staged-message fr;
    (trace-roots, trace-bs) ←
      staged-trace-fri-program A 0 (ceil-log clength) [];
    trace-final ← trace-final-stage A trace-bs;
    record-staged-message trace-final;
    return (fr, trace-roots, trace-bs, trace-final)
  }

```

definition *staged-trace-fri-challenge-prefix-program*

```

:: 'f staged-adversary ⇒ nat ⇒
  ('f × 'f list × 'f, 'f protocol-channel) state-monad

```

where

```

staged-trace-fri-challenge-prefix-program A i =
  do {
    fr ← trace-root-stage A;
    record-staged-message fr;
    (-, trace-bs) ← staged-trace-fri-program A 0 i [];
    root ← trace-fri-root-stage A i trace-bs;
    record-staged-message root;
    return (fr, trace-bs, root)
  }

```

}

definition *staged-alpha-challenge-prefix-program*

$:: 'f \text{ staged-adversary} \Rightarrow \text{nat} \Rightarrow$
 $(('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times 'f \text{ list},$
 $'f \text{ protocol-channel}) \text{ state-monad}$

where

staged-alpha-challenge-prefix-program $A \ i =$
 $\text{do } \{$
 $\text{prefix} \leftarrow \text{staged-alpha-prefix-program } A;$
 $\text{as-prefix} \leftarrow \text{staged-alpha-program } i;$
 $\text{return } (\text{prefix}, \text{as-prefix})$
 $\}$

definition *staged-composition-fri-challenge-prefix-program*

$:: 'f \text{ staged-adversary} \Rightarrow \text{nat} \Rightarrow$
 $('f \times 'f \text{ list} \times 'f, 'f \text{ protocol-channel}) \text{ state-monad}$

where

staged-composition-fri-challenge-prefix-program $A \ i =$
 $\text{do } \{$
 $(-, -, \text{trace-bs}, -) \leftarrow \text{staged-alpha-prefix-program } A;$
 $\text{as} \leftarrow \text{staged-alpha-program } (\text{length spec});$
 $\text{dg} \leftarrow \text{degree-stage } A \ \text{as};$
 $\text{record-staged-message } \text{dg};$
 $\text{let composition-rounds} = \text{ceil-log } (\text{to-nat } \text{dg} + 1);$
 $\text{assert } (\text{Suc } i \leq \text{composition-rounds});$
 $\text{assert } (\text{composition-rounds} \leq \text{ceil-log } (\text{maxDegree} + 1));$
 $(-, \text{composition-bs}) \leftarrow \text{staged-composition-fri-program } A \ \text{dg} \ 0 \ i \ [];$
 $\text{root} \leftarrow \text{composition-fri-root-stage } A \ \text{dg} \ i \ \text{composition-bs};$
 $\text{record-staged-message } \text{root};$
 $\text{return } (\text{dg}, \text{composition-bs}, \text{root})$
 $\}$

definition *staged-composition-fri-vector-program*

$:: 'f \text{ staged-adversary} \Rightarrow$
 $('f \times 'f \text{ list}, 'f \text{ protocol-channel}) \text{ state-monad}$

where

staged-composition-fri-vector-program $A =$
 $\text{do } \{$
 $(-, -, \text{trace-bs}, -) \leftarrow \text{staged-alpha-prefix-program } A;$
 $\text{as} \leftarrow \text{staged-alpha-program } (\text{length spec});$
 $\text{dg} \leftarrow \text{degree-stage } A \ \text{as};$
 $\text{record-staged-message } \text{dg};$
 $\text{let composition-rounds} = \text{ceil-log } (\text{to-nat } \text{dg} + 1);$
 $\text{assert } (\text{composition-rounds} \leq \text{ceil-log } (\text{maxDegree} + 1));$
 $(-, \text{composition-bs}) \leftarrow$
 $\text{staged-composition-fri-program } A \ \text{dg} \ 0 \ \text{composition-rounds} \ [];$
 $\text{return } (\text{dg}, \text{composition-bs})$
 $\}$

definition *staged-composition-fri-full-vector-program*
 $:: 'f \text{ staged-adversary} \Rightarrow$
 $('f \text{ staged-composition-prefix-data}, 'f \text{ protocol-channel}) \text{ state-monad}$
where
staged-composition-fri-full-vector-program $A =$
do {
 $(fr, \text{trace-roots}, \text{trace-bs}, \text{trace-final}) \leftarrow$
 $\text{staged-alpha-prefix-program } A;$
 $as \leftarrow \text{staged-alpha-program } (\text{length spec});$
 $dg \leftarrow \text{degree-stage } A \text{ as};$
 $\text{record-staged-message } dg;$
 $\text{let composition-rounds} = \text{ceil-log } (\text{to-nat } dg + 1);$
 $\text{assert } (\text{composition-rounds} \leq \text{ceil-log } (\text{maxDegree} + 1));$
 $(\text{composition-roots}, \text{composition-bs}) \leftarrow$
 $\text{staged-composition-fri-program } A \text{ dg } 0 \text{ composition-rounds } [];$
return
 $(\backslash \text{scp-trace-root} = fr,$
 $\text{scp-trace-fri-roots} = \text{trace-roots},$
 $\text{scp-trace-fri-challenges} = \text{trace-bs},$
 $\text{scp-trace-final} = \text{trace-final},$
 $\text{scp-alphas} = as,$
 $\text{scp-degree} = dg,$
 $\text{scp-composition-fri-roots} = \text{composition-roots},$
 $\text{scp-composition-fri-challenges} = \text{composition-bs})$
}

definition *staged-query-challenge-prefix-program*
 $:: 'f \text{ staged-adversary} \Rightarrow \text{nat} \Rightarrow$
 $('f \times 'f \times 'f \text{ list list}, 'f \text{ protocol-channel}) \text{ state-monad}$
where
staged-query-challenge-prefix-program $A \text{ } i =$
do {
 $(-, -, \text{trace-bs}, -) \leftarrow \text{staged-alpha-prefix-program } A;$
 $as \leftarrow \text{staged-alpha-program } (\text{length spec});$
 $dg \leftarrow \text{degree-stage } A \text{ as};$
 $\text{record-staged-message } dg;$
 $\text{let composition-rounds} = \text{ceil-log } (\text{to-nat } dg + 1);$
 $\text{assert } (\text{composition-rounds} \leq \text{ceil-log } (\text{maxDegree} + 1));$
 $(-, \text{composition-bs}) \leftarrow$
 $\text{staged-composition-fri-program } A \text{ dg } 0 \text{ composition-rounds } [];$
 $\text{composition-final} \leftarrow$
 $\text{composition-final-stage } A \text{ dg } \text{composition-bs};$
 $\text{record-staged-message } \text{composition-final};$
 $\text{query-chunks} \leftarrow \text{staged-query-program } A \text{ } 0 \text{ } i;$
return $(dg, \text{composition-final}, \text{query-chunks})$
}

definition *checked-staged-query-challenge-prefix-program*

```

:: 'f staged-adversary  $\Rightarrow$  nat  $\Rightarrow$ 
  ('f staged-query-prefix-data, 'f protocol-channel) state-monad
where
checked-staged-query-challenge-prefix-program A i =
do {
  (fr, trace-roots, trace-bs, trace-final)  $\leftarrow$ 
    staged-alpha-prefix-program A;
  as  $\leftarrow$  staged-alpha-program (length spec);
  dg  $\leftarrow$  degree-stage A as;
  record-staged-message dg;
  let composition-rounds = ceil-log (to-nat dg + 1);
  assert (composition-rounds  $\leq$  ceil-log (maxDegree + 1));
  (composition-roots, composition-bs)  $\leftarrow$ 
    staged-composition-fri-program A dg 0 composition-rounds [];
  composition-final  $\leftarrow$ 
    composition-final-stage A dg composition-bs;
  record-staged-message composition-final;
  query-chunks  $\leftarrow$ 
    checked-staged-query-program A trace-roots composition-roots 0 i;
  return
    (\sqp-trace-root = fr,
     sqp-trace-fri-roots = trace-roots,
     sqp-trace-fri-challenges = trace-bs,
     sqp-trace-final = trace-final,
     sqp-alphas = as,
     sqp-degree = dg,
     sqp-composition-fri-roots = composition-roots,
     sqp-composition-fri-challenges = composition-bs,
     sqp-composition-final = composition-final,
     sqp-query-chunks = query-chunks)
}

```

definition staged-proof-transcript :: 'f staged-proof-data $\Rightarrow$ 'f list

where

```

staged-proof-transcript data =
  verifier-header-messages
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data) @
  List.concat (staged-query-chunks data)

```

definition staged-trace-fri-start-hash :: 'f staged-proof-data $\Rightarrow$ 'f

where

```

staged-trace-fri-start-hash data =
  concat 0 (staged-trace-root data)

```

definition *staged-composition-fri-start-hash*

$:: 'f \text{ staged-proof-data} \Rightarrow 'f$

where

staged-composition-fri-start-hash data =
concat
 (*foldl concat*
 (*concat*
 (*foldl concat*
 (*staged-trace-fri-start-hash data*)
 (*staged-trace-fri-roots data*))
 (*staged-trace-final data*))
 (*staged-alphas data*))
 (*staged-degree data*)

definition *staged-query-start-hash* $:: 'f \text{ staged-proof-data} \Rightarrow 'f$

where

staged-query-start-hash data =
concat
 (*foldl concat*
 (*staged-composition-fri-start-hash data*)
 (*staged-composition-fri-roots data*))
 (*staged-composition-final data*)

definition *staged-query-chunks-match-verifier-lengths*

$:: 'f \text{ staged-proof-data} \Rightarrow \text{nat list} \Rightarrow \text{bool}$

where

staged-query-chunks-match-verifier-lengths data query-idxs $\longleftrightarrow$
length (staged-query-chunks data) = rounds $\wedge$
 $(\forall i < \text{rounds}.$
length (staged-query-chunks data ! i) =
verifier-query-round-transcript-length (query-idxs ! i)
(staged-trace-fri-roots data)
(staged-composition-fri-roots data))

lemma *query-chunks-eq-staged-if-matching-lengths:*

assumes *parser-len: length query-chunks = rounds*

and *concat-eq:*

List.concat query-chunks @ trailing =
List.concat (staged-query-chunks data)

and *parser-chunk-len:*

$\bigwedge i. i < \text{rounds} \implies$
length (query-chunks ! i) =
verifier-query-round-transcript-length (query-idxs ! i)
(staged-trace-fri-roots data)
(staged-composition-fri-roots data)

and *staged-len:*

staged-query-chunks-match-verifier-lengths data query-idxs

shows *query-chunks = staged-query-chunks data* $\wedge$ *trailing = []*

```

proof –
  have staged-chunks-len:
    length (staged-query-chunks data) = rounds
  using staged-len
  unfolding staged-query-chunks-match-verifier-lengths-def by simp
  have same-len:
    length query-chunks = length (staged-query-chunks data)
  using parser-len staged-chunks-len by simp
  have same-chunk-lens:
     $\bigwedge i. i < \text{length query-chunks} \implies$ 
      length (query-chunks ! i) =
        length (staged-query-chunks data ! i)
  proof –
    fix i
    assume i-bound:  $i < \text{length query-chunks}$ 
    then have i-round:  $i < \text{rounds}$ 
      using parser-len by simp
    show
      length (query-chunks ! i) =
        length (staged-query-chunks data ! i)
      using parser-chunk-len[OF i-round] staged-len i-round
      unfolding staged-query-chunks-match-verifier-lengths-def by simp
  qed
  show ?thesis
    by (rule concat-append-eq-concat-same-chunk-lengths
      [OF same-len same-chunk-lens concat-eq])
qed

lemma verifier-query-round-transcript-length-index-irrelevant:
  verifier-query-round-transcript-length idx trace-roots composition-roots =
  verifier-query-round-transcript-length idx' trace-roots composition-roots
unfolding verifier-query-round-transcript-length-def
  query-decommitment-transcript-length-def powers-scaled-def
by simp

lemma staged-query-chunks-match-verifier-lengths-transfer:
  assumes match: staged-query-chunks-match-verifier-lengths data query-idxs
    and len: length query-idxs' = rounds
  shows staged-query-chunks-match-verifier-lengths data query-idxs'
proof –
  have chunks-len: length (staged-query-chunks data) = rounds
    using match unfolding staged-query-chunks-match-verifier-lengths-def
    by simp
  have chunk-lens:
     $\forall i < \text{rounds}.$ 
      length (staged-query-chunks data ! i) =
        verifier-query-round-transcript-length (query-idxs' ! i)
          (staged-trace-fri-roots data)
          (staged-composition-fri-roots data)

```



```

proof (intro allI impI)
  fix i
  assume i-bound: i < rounds
  have old:
    length (staged-query-chunks data ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)
  using match i-bound
  unfolding staged-query-chunks-match-verifier-lengths-def by simp
show
  length (staged-query-chunks data ! i) =
    verifier-query-round-transcript-length (query-idxs' ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
  using old
    verifier-query-round-transcript-length-index-irrelevant
    [of query-idxs ! i
      staged-trace-fri-roots data
      staged-composition-fri-roots data query-idxs' ! i]
  by simp
qed
show ?thesis
  unfolding staged-query-chunks-match-verifier-lengths-def
  using chunks-len chunk-lens by simp
qed

```

definition *staged-transcript-trace-fri-vector-hit*
 $:: 'f \text{ list set} \Rightarrow$
 $('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$
where
staged-transcript-trace-fri-vector-hit B out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (data, -) $\Rightarrow$ staged-trace-fri-challenges data $\in$ B)

definition *staged-transcript-alpha-list-hit*
 $:: 'f \text{ list set} \Rightarrow$
 $('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$
where
staged-transcript-alpha-list-hit B out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (data, -) $\Rightarrow$ staged-alphas data $\in$ B)

definition *staged-transcript-composition-fri-vector-hit*
 $:: ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$
where

staged-transcript-composition-fri-vector-hit B out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (data, -) $\Rightarrow$
 staged-composition-fri-challenges data $\in$
 B (*staged-degree* data))

definition *staged-transcript-composition-fri-vector-guarded-hit*

:: ('f $\Rightarrow$ 'f list set) $\Rightarrow$
 ('f staged-proof-data $\times$ 'f protocol-channel) option $\Rightarrow$ bool

where

staged-transcript-composition-fri-vector-guarded-hit B out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (data, -) $\Rightarrow$
 staged-composition-fri-challenges data $\in$
 B (*staged-degree* data) $\wedge$
 $0 < \text{ceil-log } (\text{to-nat } (\text{staged-degree } \text{data}) + 1) \wedge$
 $\text{ceil-log } (\text{to-nat } (\text{staged-degree } \text{data}) + 1) \leq$
 $\text{ceil-log } (\text{maxDegree} + 1)$)

definition *staged-transcript-query-index-set-hit*

:: nat set $\Rightarrow$
 ('f staged-proof-data $\times$ 'f protocol-channel) option $\Rightarrow$ bool

where

staged-transcript-query-index-set-hit B out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (data, t) $\Rightarrow$
 $(\exists i \text{ raw. } i < \text{rounds} \wedge$
 fmlookup (HashMap t)
 (QueryIndexChallenge i
 (state-after-query-chunks
 (staged-query-start-hash data)
 (staged-query-chunks data) i)) =
 Some raw $\wedge$
 index (to-nat raw) $\in B$)

definition *staged-security-with-data-trace-fri-vector-hit*

:: 'f list set $\Rightarrow$
 (('f staged-proof-data $\times$ unit list) $\times$ 'f protocol-channel)
 option $\Rightarrow$ bool

where

staged-security-with-data-trace-fri-vector-hit B out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some ((data, -), -) $\Rightarrow$ *staged-trace-fri-challenges* data $\in B$)

definition *staged-security-with-data-alpha-list-hit*

$:: 'f \text{ list set} \Rightarrow$
 $(('f \text{ staged-proof-data} \times \text{unit list}) \times 'f \text{ protocol-channel})$
 $\text{option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-alpha-list-hit } B \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } ((\text{data}, -), -) \Rightarrow \text{staged-alphas data} \in B)$

definition *staged-security-with-data-alpha-bad-set-hit*

$:: ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(('f \text{ staged-proof-data} \times \text{unit list}) \times 'f \text{ protocol-channel})$
 $\text{option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-alpha-bad-set-hit bad-sets out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } ((\text{data}, -), -) \Rightarrow$
 $\quad (\exists \text{ trace-table. staged-alphas data} \in \text{bad-sets trace-table}))$

definition *staged-security-with-data-composition-fri-vector-guarded-hit*

$:: ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(('f \text{ staged-proof-data} \times \text{unit list}) \times 'f \text{ protocol-channel})$
 $\text{option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-composition-fri-vector-guarded-hit } B \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } ((\text{data}, -), -) \Rightarrow$
 $\quad \text{staged-composition-fri-challenges data} \in$
 $\quad B (\text{staged-degree data}) \wedge$
 $\quad 0 < \text{ceil-log } (\text{to-nat } (\text{staged-degree data}) + 1) \wedge$
 $\quad \text{ceil-log } (\text{to-nat } (\text{staged-degree data}) + 1) \leq$
 $\quad \text{ceil-log } (\text{maxDegree} + 1))$

definition *staged-security-with-data-query-index-set-hit*

$:: \text{nat set} \Rightarrow$
 $(('f \text{ staged-proof-data} \times \text{unit list}) \times 'f \text{ protocol-channel})$
 $\text{option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-query-index-set-hit } B \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } ((\text{data}, -), t) \Rightarrow$
 $\quad (\exists i \text{ raw. } i < \text{rounds} \wedge$
 $\quad \text{fmlookup } (\text{HashMap } t)$
 $\quad (\text{QueryIndexChallenge } i$
 $\quad \quad (\text{state-after-query-chunks}$
 $\quad \quad (\text{staged-query-start-hash data}))$

$(\text{staged-query-chunks data } i)) =$
 $\text{Some raw} \wedge$
 $\text{index (to-nat raw)} \in B))$

definition *staged-security-with-data-query-bad-hit*

$:: ((\text{'f staged-proof-data} \times \text{unit list}) \times \text{'f protocol-channel})$
 $\text{option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-query-bad-hit out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } ((-, \text{result}), \text{final-state}) \Rightarrow$
 $(\exists s.$
 $\text{Some (result, final-state)} \in \text{set-dist (execute verify-monad s)} \wedge$
 $\text{query-bad s (Some (result, final-state))}))$

definition *staged-security-with-data-query-sampling-hit*

$:: ((\text{'f staged-proof-data} \times \text{unit list}) \times \text{'f protocol-channel})$
 $\text{option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-query-sampling-hit out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } ((-, \text{result}), \text{final-state}) \Rightarrow$
 $(\exists s \text{ trace-table composition-table as.}$
 $\text{query-rounds-any-index-set-hit s}$
 $(\text{query-sampling-success-space trace-table composition-table as})$
 rounds
 $(\text{Some (result, final-state)})))$

definition *staged-security-with-data-query-header-sampling-hit*

$:: ((\text{'f staged-proof-data} \times \text{unit list}) \times \text{'f protocol-channel})$
 $\text{option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-query-header-sampling-hit out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } ((-, \text{result}), \text{final-state}) \Rightarrow$
 $(\exists s.$
 $\text{query-header-rounds-any-index-set-hit s}$
 $(\text{query-header-supported-union-good-sets s}$
 $\text{query-sampling-success-space})$
 $(\text{Some (result, final-state)})))$

definition *staged-security-with-data-state-query-bad-hit*

$:: (((\text{'f staged-proof-data} \times \text{'f protocol-channel}) \times \text{unit list}) \times$
 $\text{'f protocol-channel}) \text{option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-state-query-bad-hit out} \longleftrightarrow$

(case out of
 None $\Rightarrow$ False
 | Some (((data, attacker-state), result), final-state) $\Rightarrow$
 (let s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in
 Some (result, final-state) $\in$ set-dist (execute verify-monad s) $\wedge$
 query-bad s (Some (result, final-state))))

definition staged-security-with-data-state-query-header-sampling-hit

:: (((('f staged-proof-data $\times$ 'f protocol-channel) $\times$ unit list) $\times$
 'f protocol-channel) option $\Rightarrow$ bool

where

staged-security-with-data-state-query-header-sampling-hit out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((data, attacker-state), result), final-state) $\Rightarrow$
 (let s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in
 query-header-rounds-any-index-set-hit s
 (query-header-supported-union-good-sets s
 query-sampling-success-space)
 (Some (result, final-state))))

definition staged-security-with-data-state-query-header-key-hit

:: (((('f staged-proof-data $\times$ 'f protocol-channel) $\times$ unit list) $\times$
 'f protocol-channel) option $\Rightarrow$ bool

where

staged-security-with-data-state-query-header-key-hit out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((data, attacker-state), -), final-state) $\Rightarrow$
 (let s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data);
 B =
 query-header-supported-union-good-sets s
 query-sampling-success-space
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
 in

$(\exists i \text{ raw. } i < \text{rounds} \wedge$
 $\text{fmlookup } (\text{HashMap } \text{final-state})$
 $(\text{QueryIndexChallenge } i$
 $(\text{state-after-query-chunks}$
 $(\text{staged-query-start-hash } \text{data})$
 $(\text{staged-query-chunks } \text{data}) \ i)) =$
 $\text{Some } \text{raw} \wedge$
 $\text{index } (\text{to-nat } \text{raw}) \in B)))$

definition *staged-security-with-data-state-verifier-event*

$:: ('f \text{ protocol-channel} \Rightarrow$
 $(\text{unit list} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}) \Rightarrow$
 $((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-state-verifier-event } P \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state}) \Rightarrow$
 P
 $(\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data}))$
 $(\text{Some } (\text{result}, \text{final-state})))$

definition *staged-security-with-data-state-transcript-pre-hit*

$:: (((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-state-transcript-pre-hit } \text{out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((\text{data}, \text{attacker-state}), -), -) \Rightarrow$
 $\text{hash-map-output-values}$
 $(\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data})) \cap$
 $\text{set } (\text{staged-proof-transcript } \text{data}) \neq \{\})$

definition *staged-security-with-data-state-transcript-new-hit*

$:: (((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-state-transcript-new-hit } \text{out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((\text{data}, \text{attacker-state}), -), \text{final-state}) \Rightarrow$
 $\text{hash-map-new-output-hit } (\text{set } (\text{staged-proof-transcript } \text{data}))$
 $(\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data}))$
 $\text{final-state})$

definition *staged-composition-prefix-fri-vector-hit*
 $:: ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $('f \text{ staged-composition-prefix-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow$
 bool
where
 $\text{staged-composition-prefix-fri-vector-hit } B \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some (prefix, -)} \Rightarrow$
 $\text{scp-composition-fri-challenges prefix} \in$
 $B (\text{scp-degree prefix}) \wedge$
 $0 < \text{ceil-log (to-nat (scp-degree prefix) + 1)} \wedge$
 $\text{ceil-log (to-nat (scp-degree prefix) + 1)} \leq$
 $\text{ceil-log (maxDegree + 1)})$

definition *staged-after-alpha-prefix-program*
 $:: 'f \text{ staged-adversary} \Rightarrow$
 $'f \times 'f \text{ list} \times 'f \text{ list} \times 'f \Rightarrow$
 $('f \text{ staged-proof-data}, 'f \text{ protocol-channel}) \text{ state-monad}$
where
 $\text{staged-after-alpha-prefix-program } A \text{ prefix} =$
 $(\text{case prefix of (fr, trace-roots, trace-bs, trace-final)} \Rightarrow$
 $\text{do } \{$
 $\text{as} \leftarrow \text{staged-alpha-program (length spec);}$
 $\text{dg} \leftarrow \text{degree-stage } A \text{ as;}$
 $\text{record-staged-message dg;}$
 $\text{let composition-rounds} = \text{ceil-log (to-nat dg + 1);}$
 $\text{assert (composition-rounds} \leq \text{ceil-log (maxDegree + 1));}$
 $(\text{composition-roots, composition-bs}) \leftarrow$
 $\text{staged-composition-fri-program } A \text{ dg } 0 \text{ composition-rounds } [];$
 $\text{composition-final} \leftarrow$
 $\text{composition-final-stage } A \text{ dg composition-bs;}$
 $\text{record-staged-message composition-final;}$
 $\text{query-chunks} \leftarrow \text{staged-query-program } A \text{ } 0 \text{ rounds;}$
 return
 $(\text{staged-trace-root} = \text{fr,}$
 $\text{staged-trace-fri-roots} = \text{trace-roots,}$
 $\text{staged-trace-fri-challenges} = \text{trace-bs,}$
 $\text{staged-trace-final} = \text{trace-final,}$
 $\text{staged-alphas} = \text{as,}$
 $\text{staged-degree} = \text{dg,}$
 $\text{staged-composition-fri-roots} = \text{composition-roots,}$
 $\text{staged-composition-fri-challenges} = \text{composition-bs,}$
 $\text{staged-composition-final} = \text{composition-final,}$
 $\text{staged-query-chunks} = \text{query-chunks})$
 $\})$

definition *staged-after-composition-fri-full-vector-program*

```

:: 'f staged-adversary ⇒ 'f staged-composition-prefix-data ⇒
  ('f staged-proof-data, 'f protocol-channel) state-monad
where
  staged-after-composition-fri-full-vector-program A prefix =
    do {
      composition-final ←
        composition-final-stage A
          (scp-degree prefix)
          (scp-composition-fri-challenges prefix);
      record-staged-message composition-final;
      query-chunks ← staged-query-program A 0 rounds;
      return
        (|staged-trace-root = scp-trace-root prefix,
          staged-trace-fri-roots = scp-trace-fri-roots prefix,
          staged-trace-fri-challenges = scp-trace-fri-challenges prefix,
          staged-trace-final = scp-trace-final prefix,
          staged-alphas = scp-alphas prefix,
          staged-degree = scp-degree prefix,
          staged-composition-fri-roots =
            scp-composition-fri-roots prefix,
          staged-composition-fri-challenges =
            scp-composition-fri-challenges prefix,
          staged-composition-final = composition-final,
          staged-query-chunks = query-chunks|)
    }

```

lemma *staged-transcript-program-alpha-prefix-decomp:*

```

  staged-transcript-program A =
    staged-alpha-prefix-program A ≍
    staged-after-alpha-prefix-program A
unfolding staged-transcript-program-def staged-alpha-prefix-program-def
  staged-after-alpha-prefix-program-def Let-def
by (simp add: sm-bind-assoc split-def split: prod.splits)

```

lemma *staged-transcript-program-composition-fri-full-vector-decomp:*

```

  staged-transcript-program A =
    staged-composition-fri-full-vector-program A ≍
    staged-after-composition-fri-full-vector-program A
unfolding staged-transcript-program-def
  staged-composition-fri-full-vector-program-def
  staged-after-composition-fri-full-vector-program-def
  staged-alpha-prefix-program-def Let-def
by (simp add: sm-bind-assoc split-def split: prod.splits)

```

lemma *staged-composition-fri-vector-program-projection:*

```

  staged-composition-fri-vector-program A =
    staged-composition-fri-full-vector-program A ≍
    (λprefix. return
      (scp-degree prefix, scp-composition-fri-challenges prefix))

```



```

unfolding staged-composition-fri-vector-program-def
  staged-composition-fri-full-vector-program-def
  staged-alpha-prefix-program-def Let-def
by (simp add: sm-bind-assoc split-def split: prod.splits)

lemma staged-after-alpha-prefix-program-trace-fri-vector-hitD:
assumes outcome:
  out ∈
    set-dist
    (execute
      (staged-after-alpha-prefix-program A
        (fr, trace-roots, trace-bs, trace-final)) s)
and hit: staged-transcript-trace-fri-vector-hit B out
shows trace-bs ∈ B
proof (cases out)
case None
then show ?thesis
  using hit unfolding staged-transcript-trace-fri-vector-hit-def by simp
next
case (Some result)
then obtain data u where out-eq: out = Some (data, u)
  by (cases result) simp
from outcome[unfolded out-eq]
obtain as s1 dg s2 s3 composition-roots composition-bs s4
  composition-final s5 s6 query-chunks where
  ret:
    Some (data, u) ∈
      set-dist
      (execute
        (return
          (|staged-trace-root = fr,
            staged-trace-fri-roots = trace-roots,
            staged-trace-fri-challenges = trace-bs,
            staged-trace-final = trace-final,
            staged-alphas = as,
            staged-degree = dg,
            staged-composition-fri-roots = composition-roots,
            staged-composition-fri-challenges = composition-bs,
            staged-composition-final = composition-final,
            staged-query-chunks = query-chunks|)
          s6))
  unfolding staged-after-alpha-prefix-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have data-eq:
  data =
    (|staged-trace-root = fr,
      staged-trace-fri-roots = trace-roots,
      staged-trace-fri-challenges = trace-bs,
      staged-trace-final = trace-final,

```

```

    staged-alphas = as,
    staged-degree = dg,
    staged-composition-fri-roots = composition-roots,
    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks)
  using ret by simp
show ?thesis
  using hit unfolding out-eq data-eq
    staged-transcript-trace-fri-vector-hit-def
  by simp
qed

lemma staged-after-alpha-prefix-program-alpha-list-hitD:
  assumes outcome:
    out ∈
      set-dist
        (execute
          (staged-after-alpha-prefix-program A
            (fr, trace-roots, trace-bs, trace-final)) s)
  and hit: staged-transcript-alpha-list-hit B out
  shows
    ∃ as s1.
      Some (as, s1) ∈
        set-dist (execute (staged-alpha-program (length spec)) s) ∧
        as ∈ B
  proof (cases out)
  case None
  then show ?thesis
    using hit unfolding staged-transcript-alpha-list-hit-def by simp
  next
  case (Some result)
  then obtain data u where out-eq: out = Some (data, u)
    by (cases result) simp
  from outcome[unfolded out-eq]
  obtain as s1 dg s2 s3 composition-roots composition-bs s4
    composition-final s5 s6 query-chunks where
    alpha-out:
      Some (as, s1) ∈
        set-dist (execute (staged-alpha-program (length spec)) s)
  and ret:
    Some (data, u) ∈
      set-dist
        (execute
          (return
            (|staged-trace-root = fr,
              staged-trace-fri-roots = trace-roots,
              staged-trace-fri-challenges = trace-bs,
              staged-trace-final = trace-final,

```

```

    staged-alphas = as,
    staged-degree = dg,
    staged-composition-fri-roots = composition-roots,
    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks))
s6)
unfolding staged-after-alpha-prefix-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have data-eq:
  data =
    (|staged-trace-root = fr,
     staged-trace-fri-roots = trace-roots,
     staged-trace-fri-challenges = trace-bs,
     staged-trace-final = trace-final,
     staged-alphas = as,
     staged-degree = dg,
     staged-composition-fri-roots = composition-roots,
     staged-composition-fri-challenges = composition-bs,
     staged-composition-final = composition-final,
     staged-query-chunks = query-chunks|)
using ret by simp
have as-in:  $as \in B$ 
using hit unfolding out-eq data-eq
  staged-transcript-alpha-list-hit-def
by simp
show ?thesis
using alpha-out as-in by blast
qed

lemma staged-after-alpha-prefix-program-alpha-list-bound-from-alpha-program:
assumes alpha-bound:
  wp-event (staged-alpha-program (length spec))
  ( $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
   |  $\text{Some } (as, -) \Rightarrow as \in B$ )  $s \leq C$ 
shows
  wp-event
  (staged-after-alpha-prefix-program A
   (fr, trace-roots, trace-bs, trace-final))
  (staged-transcript-alpha-list-hit B)  $s \leq C$ 
proof –
have decomp:
  staged-after-alpha-prefix-program A
  (fr, trace-roots, trace-bs, trace-final) =
  (staged-alpha-program (length spec)  $\gg=$ 
   ( $\lambda as. \text{degree-stage } A \text{ as} \gg=$ 
    ( $\lambda dg. \text{record-staged-message } dg \gg=$ 
     ( $\lambda -. \text{let composition-rounds} =$ 
       $\text{ceil-log (to-nat } dg + 1)$ 

```

```

in assert
  (composition-rounds ≤ ceil-log (maxDegree + 1)) ≫=
  (λ-. staged-composition-fri-program A dg 0
    composition-rounds [] ≫=
    (λ(composition-roots, composition-bs).
      composition-final-stage A dg composition-bs ≫=
      (λcomposition-final.
        record-staged-message composition-final ≫=
        (λ-. staged-query-program A 0 rounds ≫=
        (λquery-chunks.
          return
            (|staged-trace-root = fr,
              staged-trace-fri-roots = trace-roots,
              staged-trace-fri-challenges = trace-bs,
              staged-trace-final = trace-final,
              staged-alphas = as,
              staged-degree = dg,
              staged-composition-fri-roots =
                composition-roots,
              staged-composition-fri-challenges =
                composition-bs,
              staged-composition-final =
                composition-final,
              staged-query-chunks =
                query-chunks|))))))))))
unfolding staged-after-alpha-prefix-program-def Let-def
by simp
show ?thesis
  unfolding decomp
proof (rule wp-event-bind-bound-by-head-event[OF alpha-bound])
  show staged-transcript-alpha-list-hit B None ⇒
    (case None of None ⇒ False | Some (as, -) ⇒ as ∈ B)
  unfolding staged-transcript-alpha-list-hit-def by simp
next
  fix as s1 out
  assume alpha-out:
    Some (as, s1) ∈
      set-dist (execute (staged-alpha-program (length spec)) s)
  and cont:
    out ∈
      set-dist
        (execute
          (degree-stage A as ≫=
            (λdg. record-staged-message dg ≫=
              (λ-. let composition-rounds =
                ceil-log (to-nat dg + 1)
                in assert
                  (composition-rounds ≤ ceil-log (maxDegree + 1)) ≫=
                  (λ-. staged-composition-fri-program A dg 0

```

```

composition-rounds [] >>=
(λ(composition-roots, composition-bs).
  composition-final-stage A dg composition-bs >>=
  (λcomposition-final.
    record-staged-message composition-final >>=
    (λ-. staged-query-program A 0 rounds >>=
      (λquery-chunks.
        return
          (λ(staged-trace-root = fr,
            staged-trace-fri-roots = trace-roots,
            staged-trace-fri-challenges = trace-bs,
            staged-trace-final = trace-final,
            staged-alphas = as,
            staged-degree = dg,
            staged-composition-fri-roots =
              composition-roots,
            staged-composition-fri-challenges =
              composition-bs,
            staged-composition-final =
              composition-final,
            staged-query-chunks =
              query-chunks))))))))))
s1)
and hit: staged-transcript-alpha-list-hit B out
show (case Some (as, s1) of None ⇒ False
      | Some (as, -) ⇒ as ∈ B)
proof (cases out)
case None
then show ?thesis
  using hit unfolding staged-transcript-alpha-list-hit-def by simp
next
case (Some result)
then obtain data u where out-eq: out = Some (data, u)
  by (cases result) simp
from cont[unfolded out-eq]
obtain dg s2 s3 composition-roots composition-bs s4
  composition-final s5 s6 query-chunks where
ret:
Some (data, u) ∈
set-dist
(execute
(return
(λ(staged-trace-root = fr,
  staged-trace-fri-roots = trace-roots,
  staged-trace-fri-challenges = trace-bs,
  staged-trace-final = trace-final,
  staged-alphas = as,
  staged-degree = dg,
  staged-composition-fri-roots = composition-roots,

```

```

      staged-composition-fri-challenges = composition-bs,
      staged-composition-final = composition-final,
      staged-query-chunks = query-chunks))
    s6)
  by (auto simp: Let-def assert-def
      elim!: set-dist-bindE split: prod.splits)
have data-eq:
  data =
    (|staged-trace-root = fr,
      staged-trace-fri-roots = trace-roots,
      staged-trace-fri-challenges = trace-bs,
      staged-trace-final = trace-final,
      staged-alphas = as,
      staged-degree = dg,
      staged-composition-fri-roots = composition-roots,
      staged-composition-fri-challenges = composition-bs,
      staged-composition-final = composition-final,
      staged-query-chunks = query-chunks|)
  using ret by simp
show ?thesis
  using hit unfolding out-eq data-eq
    staged-transcript-alpha-list-hit-def by simp
qed
qed
qed

lemma staged-after-composition-fri-full-vector-program-hitD:
  assumes outcome:
    out ∈
      set-dist
        (execute
          (staged-after-composition-fri-full-vector-program A prefix) s)
  and hit:
    staged-transcript-composition-fri-vector-guarded-hit B out
  shows staged-composition-prefix-fri-vector-hit B (Some (prefix, s))
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding staged-transcript-composition-fri-vector-guarded-hit-def
  by simp
next
case (Some result)
then obtain data u where out-eq: out = Some (data, u)
  by (cases result) simp
from outcome[unfolded out-eq]
obtain composition-final s1 s2 query-chunks where
  ret:
    Some (data, u) ∈

```

```

    set-dist
    (execute
      (return
        (staged-trace-root = scp-trace-root prefix,
         staged-trace-fri-roots = scp-trace-fri-roots prefix,
         staged-trace-fri-challenges =
           scp-trace-fri-challenges prefix,
         staged-trace-final = scp-trace-final prefix,
         staged-alphas = scp-alphas prefix,
         staged-degree = scp-degree prefix,
         staged-composition-fri-roots =
           scp-composition-fri-roots prefix,
         staged-composition-fri-challenges =
           scp-composition-fri-challenges prefix,
         staged-composition-final = composition-final,
         staged-query-chunks = query-chunks))
      s2)
  unfolding staged-after-composition-fri-full-vector-program-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have data-eq:
  data =
    (staged-trace-root = scp-trace-root prefix,
     staged-trace-fri-roots = scp-trace-fri-roots prefix,
     staged-trace-fri-challenges = scp-trace-fri-challenges prefix,
     staged-trace-final = scp-trace-final prefix,
     staged-alphas = scp-alphas prefix,
     staged-degree = scp-degree prefix,
     staged-composition-fri-roots =
       scp-composition-fri-roots prefix,
     staged-composition-fri-challenges =
       scp-composition-fri-challenges prefix,
     staged-composition-final = composition-final,
     staged-query-chunks = query-chunks)
  using ret by simp
show ?thesis
  using hit
  unfolding out-eq data-eq
    staged-transcript-composition-fri-vector-guarded-hit-def
    staged-composition-prefix-fri-vector-hit-def
  by simp
qed

lemma staged-proof-transcript-verifier-header-transcript:
assumes trace-len:
  length (staged-trace-fri-roots data) = ceil-log clength
and alpha-len:
  length (staged-alphas data) = length spec
and composition-len:
  length (staged-composition-fri-roots data) =

```

```

      ceil-log (to-nat (staged-degree data) + 1)
shows
  verifier-header-transcript
    (verifier-state-from-adversary s (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
using trace-len alpha-len composition-len
unfolding verifier-header-transcript-def staged-proof-transcript-def
by simp

```

definition *staged-security-experiment*

```

:: 'f staged-adversary ⇒
  (unit list, 'f protocol-channel) state-monad
where
  staged-security-experiment A =
    staged-transcript-program A ≫= (λdata.
      get ≫= (λs.
        put
          (verifier-state-from-adversary s
            (staged-proof-transcript data)) ≫=
            (λ-. verify-monad)))

```

definition *staged-security-experiment-with-data*

```

:: 'f staged-adversary ⇒
  ('f staged-proof-data × unit list, 'f protocol-channel) state-monad
where
  staged-security-experiment-with-data A =
    staged-transcript-program A ≫= (λdata.
      get ≫= (λs.
        put
          (verifier-state-from-adversary s
            (staged-proof-transcript data)) ≫=
            (λ-. verify-monad ≫= (λresult. return (data, result))))))

```

definition *staged-security-experiment-with-data-state*

```

:: 'f staged-adversary ⇒
  (('f staged-proof-data × 'f protocol-channel) × unit list,
   'f protocol-channel) state-monad
where
  staged-security-experiment-with-data-state A =
    staged-transcript-program A ≫= (λdata.
      get ≫= (λs.
        put

```


$(\text{verifier-state-from-adversary } s$
 $(\text{staged-proof-transcript data})) \gg=$
 $(\lambda-.$
 $\text{verify-monad} \gg=$
 $(\lambda \text{result. return } ((\text{data}, s), \text{result}))))$

definition *checked-staged-security-experiment*
 $:: 'f \text{ staged-adversary} \Rightarrow$
 $(\text{unit list}, 'f \text{ protocol-channel}) \text{ state-monad}$
where

$\text{checked-staged-security-experiment } A =$
 $\text{checked-staged-transcript-program } A \gg= (\lambda \text{data.}$
 $\text{get} \gg= (\lambda s.$
 put
 $(\text{verifier-state-from-adversary } s$
 $(\text{staged-proof-transcript data})) \gg=$
 $(\lambda-. \text{verify-monad}))$

definition *checked-staged-security-experiment-with-data*
 $:: 'f \text{ staged-adversary} \Rightarrow$
 $('f \text{ staged-proof-data} \times \text{unit list}, 'f \text{ protocol-channel}) \text{ state-monad}$
where

$\text{checked-staged-security-experiment-with-data } A =$
 $\text{checked-staged-transcript-program } A \gg= (\lambda \text{data.}$
 $\text{get} \gg= (\lambda s.$
 put
 $(\text{verifier-state-from-adversary } s$
 $(\text{staged-proof-transcript data})) \gg=$
 $(\lambda-. \text{verify-monad} \gg= (\lambda \text{result. return } (\text{data}, \text{result}))))$

definition *checked-staged-security-experiment-with-data-state*
 $:: 'f \text{ staged-adversary} \Rightarrow$
 $(('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list},$
 $'f \text{ protocol-channel}) \text{ state-monad}$
where

$\text{checked-staged-security-experiment-with-data-state } A =$
 $\text{checked-staged-transcript-program } A \gg= (\lambda \text{data.}$
 $\text{get} \gg= (\lambda s.$
 put
 $(\text{verifier-state-from-adversary } s$
 $(\text{staged-proof-transcript data})) \gg=$
 $(\lambda-.$
 $\text{verify-monad} \gg=$
 $(\lambda \text{result. return } ((\text{data}, s), \text{result}))))$

lemma *staged-security-experiment-projection:*
 $\text{staged-security-experiment-with-data } A \gg=$
 $(\lambda x. \text{return } (\text{snd } x)) =$
 $\text{staged-security-experiment } A$

unfolding *staged-security-experiment-with-data-def*
staged-security-experiment-def
by (*simp add: sm-bind-assoc split: prod.splits*)

lemma *staged-security-experiment-with-data-state-projection:*
staged-security-experiment-with-data-state A $\gg=$
 $(\lambda x. \text{return } (\text{snd } x)) =$
staged-security-experiment A
unfolding *staged-security-experiment-with-data-state-def*
staged-security-experiment-def
by (*simp add: sm-bind-assoc split: prod.splits*)

lemma *staged-security-experiment-with-data-state-projection-to-data:*
staged-security-experiment-with-data-state A $\gg=$
 $(\lambda x. \text{case } x \text{ of } ((\text{data}, -), \text{result}) \Rightarrow$
 $\text{return } (\text{data}, \text{result})) =$
staged-security-experiment-with-data A
unfolding *staged-security-experiment-with-data-state-def*
staged-security-experiment-with-data-def
by (*simp add: sm-bind-assoc split: prod.splits*)

lemma *checked-staged-security-experiment-projection:*
checked-staged-security-experiment-with-data A $\gg=$
 $(\lambda x. \text{return } (\text{snd } x)) =$
checked-staged-security-experiment A
unfolding *checked-staged-security-experiment-with-data-def*
checked-staged-security-experiment-def
by (*simp add: sm-bind-assoc split: prod.splits*)

lemma *checked-staged-security-experiment-with-data-state-projection:*
checked-staged-security-experiment-with-data-state A $\gg=$
 $(\lambda x. \text{return } (\text{snd } x)) =$
checked-staged-security-experiment A
unfolding *checked-staged-security-experiment-with-data-state-def*
checked-staged-security-experiment-def
by (*simp add: sm-bind-assoc split: prod.splits*)

lemma *checked-staged-security-experiment-with-data-state-projection-to-data:*
checked-staged-security-experiment-with-data-state A $\gg=$
 $(\lambda x. \text{case } x \text{ of } ((\text{data}, -), \text{result}) \Rightarrow$
 $\text{return } (\text{data}, \text{result})) =$
checked-staged-security-experiment-with-data A
unfolding *checked-staged-security-experiment-with-data-state-def*
checked-staged-security-experiment-with-data-def
by (*simp add: sm-bind-assoc split: prod.splits*)

lemma *staged-security-experiment-acceptance-with-data:*
 $\text{wp-event } (\text{staged-security-experiment } A) \text{ accepted adversary-initial-state} =$
 $\text{wp-event } (\text{staged-security-experiment-with-data } A) \text{ accepted}$

```

    adversary-initial-state
proof -
  have proj:
    staged-security-experiment A =
      staged-security-experiment-with-data A  $\gg$ 
      ( $\lambda x.$  return (snd x))
  using staged-security-experiment-projection by simp
show ?thesis
  unfolding proj
  by (subst wp-event-bind-return-map[where f=snd])
    (unfold wp-event-def accepted-def,
     rule arg-cong[where
      f= $\lambda Q.$  wp (staged-security-experiment-with-data A) Q
      adversary-initial-state],
     rule ext, simp split: option.splits prod.splits)
qed

lemma staged-security-experiment-acceptance-with-data-state:
  wp-event (staged-security-experiment A) accepted adversary-initial-state =
  wp-event (staged-security-experiment-with-data-state A) accepted
  adversary-initial-state
proof -
  have proj:
    staged-security-experiment A =
      staged-security-experiment-with-data-state A  $\gg$ 
      ( $\lambda x.$  return (snd x))
  using staged-security-experiment-with-data-state-projection by simp
show ?thesis
  unfolding proj
  by (subst wp-event-bind-return-map[where f=snd])
    (unfold wp-event-def accepted-def,
     rule arg-cong[where
      f= $\lambda Q.$  wp (staged-security-experiment-with-data-state A) Q
      adversary-initial-state],
     rule ext, simp split: option.splits prod.splits)
qed

lemma checked-staged-security-experiment-acceptance-with-data:
  wp-event (checked-staged-security-experiment A) accepted
  adversary-initial-state =
  wp-event (checked-staged-security-experiment-with-data A) accepted
  adversary-initial-state
proof -
  have proj:
    checked-staged-security-experiment A =
      checked-staged-security-experiment-with-data A  $\gg$ 
      ( $\lambda x.$  return (snd x))
  using checked-staged-security-experiment-projection by simp
show ?thesis

```

```

unfolding proj
by (subst wp-event-bind-return-map[where f=snd])
  (unfold wp-event-def accepted-def,
   rule arg-cong[where
    f= $\lambda Q. wp \text{ (checked-staged-security-experiment-with-data } A) \ Q$ 
    adversary-initial-state],
   rule ext, simp split: option.splits prod.splits)
qed

lemma checked-staged-security-experiment-acceptance-with-data-state:
  wp-event (checked-staged-security-experiment A) accepted
    adversary-initial-state =
  wp-event (checked-staged-security-experiment-with-data-state A) accepted
    adversary-initial-state
proof –
  have proj:
    checked-staged-security-experiment A =
    checked-staged-security-experiment-with-data-state A  $\gg$ 
    ( $\lambda x. \text{return (snd } x)$ )
  using checked-staged-security-experiment-with-data-state-projection
  by simp
show ?thesis
  unfolding proj
  by (subst wp-event-bind-return-map[where f=snd])
    (unfold wp-event-def accepted-def,
     rule arg-cong[where
      f= $\lambda Q. wp \text{ (checked-staged-security-experiment-with-data-state } A) \ Q$ 
      adversary-initial-state],
     rule ext, simp split: option.splits prod.splits)
qed

lemma staged-security-experiment-with-data-event-from-data-state:
  wp-event (staged-security-experiment-with-data A) P adversary-initial-state =
  wp-event (staged-security-experiment-with-data-state A)
    ( $\lambda out. \text{case out of None} \Rightarrow P \text{ None}$ 
     |  $\text{Some } (((data, -), result), t) \Rightarrow P \text{ (Some } ((data, result), t))$ )
    adversary-initial-state
proof –
  have proj:
    staged-security-experiment-with-data A =
    staged-security-experiment-with-data-state A  $\gg$ 
    ( $\lambda x. \text{case } x \text{ of } ((data, -), result) \Rightarrow$ 
     return (data, result))
  using staged-security-experiment-with-data-state-projection-to-data
  by simp
show ?thesis
  unfolding proj wp-event-def
  apply (subst wp-bind)
  apply (rule arg-cong[where

```

```

    f = λQ. wp (staged-security-experiment-with-data-state A) Q
      adversary-initial-state])
  apply (rule ext)
  apply (simp add: wp-return split: option.splits prod.splits)
done
qed

lemma checked-staged-security-experiment-with-data-event-from-data-state:
  wp-event (checked-staged-security-experiment-with-data A) P
    adversary-initial-state =
  wp-event (checked-staged-security-experiment-with-data-state A)
    (λout. case out of None ⇒ P None
      | Some (((data, -), result), t) ⇒ P (Some ((data, result), t)))
    adversary-initial-state
proof -
  have proj:
    checked-staged-security-experiment-with-data A =
    checked-staged-security-experiment-with-data-state A >>=
      (λx. case x of ((data, -), result) ⇒
        return (data, result))
  using checked-staged-security-experiment-with-data-state-projection-to-data
  by simp
  show ?thesis
  unfolding proj wp-event-def
  apply (subst wp-bind)
  apply (rule arg-cong[where
    f = λQ. wp (checked-staged-security-experiment-with-data-state A) Q
      adversary-initial-state])
  apply (rule ext)
  apply (simp add: wp-return split: option.splits prod.splits)
done
qed

lemma staged-security-experiment-with-data-state-outcomeE:
  assumes
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
      (execute (staged-security-experiment-with-data-state A) initial-state)
  obtains
    Some (data, attacker-state) ∈
      set-dist (execute (staged-transcript-program A) initial-state)
    Some (result, final-state) ∈
      set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
  using assms unfolding staged-security-experiment-with-data-state-def
  by (auto elim!: set-dist-bindE intro: that)

```

lemma *checked-staged-security-experiment-with-data-state-outcomeE*:

assumes

Some (((*data*, *attacker-state*), *result*), *final-state*) ∈
set-dist
 (*execute*
 (*checked-staged-security-experiment-with-data-state A*)
initial-state)

obtains

Some (*data*, *attacker-state*) ∈
set-dist
 (*execute* (*checked-staged-transcript-program A*) *initial-state*)
Some (*result*, *final-state*) ∈
set-dist
 (*execute verify-monad*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))))

using *assms*

unfolding *checked-staged-security-experiment-with-data-state-def*

by (*auto elim!*: *set-dist-bindE* *intro*: *that*)

lemma *checked-staged-security-with-data-state-verifier-event-bound-from-cont*:

assumes *no-none*: $\bigwedge s. \neg P \ s \ \text{None}$

and *cont-bound*:

$\bigwedge \text{data attacker-state.}$

Some (*data*, *attacker-state*) ∈
set-dist
 (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) $\implies$
wp-event verify-monad
 (*P*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq C$

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event P*)
adversary-initial-state $\leq C$

unfolding *checked-staged-security-experiment-with-data-state-def*

proof (*rule wp-event-bind-bound-by-cont*)

show $\neg \text{staged-security-with-data-state-verifier-event } P \ \text{None}$

unfolding *staged-security-with-data-state-verifier-event-def* **by** *simp*

next

fix *data attacker-state*

assume *builder*:

Some (*data*, *attacker-state*) ∈
set-dist
 (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state)

```

let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have cont-eq:
  wp-event
    (get  $\gg$ 
      ( $\lambda s$ . put
        (verifier-state-from-adversary s
          (staged-proof-transcript data))  $\gg$ 
        ( $\lambda$ -. verify-monad  $\gg$ 
          ( $\lambda$ result. return ((data, s), result))))))
    (staged-security-with-data-state-verifier-event P) attacker-state =
    wp-event verify-monad (P ?s) ?s
unfolding wp-event-def staged-security-with-data-state-verifier-event-def
apply (simp add: wpsimps)
apply (rule arg-cong[where  $f = \lambda Q$ . wp verify-monad Q ?s])
apply (rule ext)
apply (simp add: no-none wp-return split: option.splits prod.splits)
done
show
  wp-event
    (get  $\gg$ 
      ( $\lambda s$ . put
        (verifier-state-from-adversary s
          (staged-proof-transcript data))  $\gg$ 
        ( $\lambda$ -. verify-monad  $\gg$ 
          ( $\lambda$ result. return ((data, s), result))))))
    (staged-security-with-data-state-verifier-event P) attacker-state  $\leq C$ 
unfolding cont-eq by (rule cont-bound[OF builder])
qed

lemma checked-staged-security-with-data-state-bound-from-data-cont:
assumes none:  $\neg P \text{ None}$ 
and cont-bound:
   $\bigwedge$  data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      ( $\lambda$ out.
        P (case out of
          None  $\implies$  None
          | Some (result, final-state)  $\implies$ 
            Some (((data, attacker-state), result), final-state)))
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))  $\leq C$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A) P

```

```

    adversary-initial-state  $\leq C$ 
unfolding checked-staged-security-experiment-with-data-state-def
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg P \text{ None}$ 
    by (rule none)
next
  fix data attacker-state
  assume builder:
    Some (data, attacker-state)  $\in$ 
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have cont-eq:
    wp-event
      (get  $\gg=$ 
        ( $\lambda s$ . put
          (verifier-state-from-adversary s
            (staged-proof-transcript data))  $\gg=$ 
            ( $\lambda$ -. verify-monad  $\gg=$ 
              ( $\lambda$ result. return ((data, s), result))))))
    P attacker-state =
    wp-event verify-monad
      ( $\lambda$ out.
        P (case out of
          None  $\Rightarrow$  None
          | Some (result, final-state)  $\Rightarrow$ 
            Some (((data, attacker-state), result), final-state)))
      ?s
  unfolding wp-event-def
  apply (simp add: none wpsimps)
  apply (rule arg-cong[where f= $\lambda Q$ . wp verify-monad Q ?s])
  apply (rule ext)
  apply (rename-tac out)
  apply (case-tac out)
  apply (simp add: none)
  apply (rename-tac result-state)
  apply (case-tac result-state)
  by (simp add: wp-return)
show
  wp-event
    (get  $\gg=$ 
      ( $\lambda s$ . put
        (verifier-state-from-adversary s
          (staged-proof-transcript data))  $\gg=$ 
          ( $\lambda$ -. verify-monad  $\gg=$ 
            ( $\lambda$ result. return ((data, s), result))))))

```


$P \text{ attacker-state} \leq C$
unfolding *cont-eq* **by** (rule *cont-bound*[*OF builder*])
qed

lemma *checked-staged-security-with-data-state-query-bad-verifier-event-bound*:
assumes *query-bound*:
 $wp\text{-event} \text{ (checked-staged-security-experiment-with-data-state } A)$
 $staged\text{-security-with-data-state-query-bad-hit}$
 $adversary\text{-initial-state} \leq C$
shows
 $wp\text{-event} \text{ (checked-staged-security-experiment-with-data-state } A)$
 $(staged\text{-security-with-data-state-verifier-event } query\text{-bad})$
 $adversary\text{-initial-state} \leq C$
proof –
have $wp\text{-event} \text{ (checked-staged-security-experiment-with-data-state } A)$
 $(staged\text{-security-with-data-state-verifier-event } query\text{-bad})$
 $adversary\text{-initial-state} \leq$
 $wp\text{-event} \text{ (checked-staged-security-experiment-with-data-state } A)$
 $staged\text{-security-with-data-state-query-bad-hit}$
 $adversary\text{-initial-state}$
proof (rule *wp-event-mono-on-support*)
fix *out*
assume *support*:
 $out \in$
 $set\text{-dist}$
 $(execute$
 $(checked\text{-staged-security-experiment-with-data-state } A)$
 $adversary\text{-initial-state})$
and *event*:
 $staged\text{-security-with-data-state-verifier-event } query\text{-bad } out$
show $staged\text{-security-with-data-state-query-bad-hit } out$
proof (*cases out*)
case *None*
then show *?thesis*
using *event unfolding staged-security-with-data-state-verifier-event-def*
by *simp*
next
case (*Some packed*)
then obtain *data attacker-state result final-state* **where** *out-eq*:
 $out = Some \text{ (((data, attacker-state), result), final-state)}$
by (*cases packed, auto split: prod.splits*)
let *?s* =
 $verifier\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)$
have *verifier*:
 $Some (result, final-state) \in set\text{-dist } (execute \text{ verify-monad } ?s)$
using *checked-staged-security-experiment-with-data-state-outcomeE*
 $[OF \text{ support}[unfolded \text{ out-eq}]]$
by *blast*

```

have bad: query-bad ?s (Some (result, final-state))
  using event
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
show ?thesis
  unfolding out-eq staged-security-with-data-state-query-bad-hit-def
  Let-def
  using verifier bad by simp
qed
qed
also have ...  $\leq C$ 
  by (rule query-bound)
finally show ?thesis .
qed

```

definition *staged-semantic-adversary*
 $:: 'f \text{ staged-adversary} \Rightarrow 'f \text{ semantic-adversary}$
where
staged-semantic-adversary $A =$
staged-transcript-program $A \gg=$
 $(\lambda \text{data. return (staged-proof-transcript data)})$

lemma *staged-security-experiment-eq-security-experiment:*
staged-security-experiment $A =$
security-experiment (*staged-semantic-adversary* A)
unfolding *staged-security-experiment-def* *security-experiment-def*
staged-semantic-adversary-def *verifier-state-transfer-def*
by (*simp add: sm-bind-assoc*)

definition *staged-adversary-acceptance-probability*
 $:: 'f \text{ staged-adversary} \Rightarrow \text{prob}$
where
staged-adversary-acceptance-probability $A =$
wp-event (*staged-security-experiment* A)
 $(\lambda \text{out. } \neg \text{Option.is-none out}) \text{ adversary-initial-state}$

definition *checked-staged-adversary-acceptance-probability*
 $:: 'f \text{ staged-adversary} \Rightarrow \text{prob}$
where
checked-staged-adversary-acceptance-probability $A =$
wp-event (*checked-staged-security-experiment* A)
 $(\lambda \text{out. } \neg \text{Option.is-none out}) \text{ adversary-initial-state}$

end

end

theory *Staged-Security-Experiment-Budgets*

```

imports Staged-Security-Experiment-Core
begin

```

Hash range, collision, relation, and target-budget calculus for staged programs.

```

context soundness
begin

```

lemma *record-staged-message-outcome*:

assumes

Some $((\cdot), t) \in \text{set-dist } (\text{execute } (\text{record-staged-message } x) \ s)$

shows

$t = s \langle \mid$

$\text{PState} := \text{concat } (\text{PState } s) \ x,$

$\text{PTranscript} := \text{PTranscript } s \ @ \ [x] \rangle$

using *assms* **unfolding** *record-staged-message-def modify-def*
by *(auto elim!: set-dist-bindE)*

lemma *record-staged-message-simps[simp]*:

HashMap

$(s \langle \mid \text{PState} := \text{concat } (\text{PState } s) \ x,$

$\text{PTranscript} := \text{PTranscript } s \ @ \ [x] \rangle) = \text{HashMap } s$

PState

$(s \langle \mid \text{PState} := \text{concat } (\text{PState } s) \ x,$

$\text{PTranscript} := \text{PTranscript } s \ @ \ [x] \rangle) =$

$\text{concat } (\text{PState } s) \ x$

PTranscript

$(s \langle \mid \text{PState} := \text{concat } (\text{PState } s) \ x,$

$\text{PTranscript} := \text{PTranscript } s \ @ \ [x] \rangle) =$

$\text{PTranscript } s \ @ \ [x]$

PTraceFriCounter

$(s \langle \mid \text{PState} := \text{concat } (\text{PState } s) \ x,$

$\text{PTranscript} := \text{PTranscript } s \ @ \ [x] \rangle) =$

$\text{PTraceFriCounter } s$

PCompositionFriCounter

$(s \langle \mid \text{PState} := \text{concat } (\text{PState } s) \ x,$

$\text{PTranscript} := \text{PTranscript } s \ @ \ [x] \rangle) =$

$\text{PCompositionFriCounter } s$

PAlphaCounter

$(s \langle \mid \text{PState} := \text{concat } (\text{PState } s) \ x,$

$\text{PTranscript} := \text{PTranscript } s \ @ \ [x] \rangle) =$

$\text{PAlphaCounter } s$

PQueryCounter

$(s \langle \mid \text{PState} := \text{concat } (\text{PState } s) \ x,$

$\text{PTranscript} := \text{PTranscript } s \ @ \ [x] \rangle) =$

$\text{PQueryCounter } s$

by *simp-all*

lemma *record-staged-message-preserves-hash-map*:

hash-map-preserving (record-staged-message x)

unfolding *hash-map-preserving-def record-staged-message-def modify-def*
by (*auto elim!: set-dist-bindE*)

lemma *hash-range-budget-record-staged-message:*
hash-range-budget 0 (record-staged-message x)
unfolding *record-staged-message-def*
by (*rule hash-range-budget-modify-preserves-hash-map*) *simp*

lemma *hash-collision-budget-record-staged-message:*
hash-collision-budget 0 (record-staged-message x)
unfolding *record-staged-message-def*
by (*rule hash-collision-budget-modify-preserves-hash-map*) *simp*

lemma *hash-target-program-record-staged-message:*
hash-target-program B 0 (record-staged-message x)
by (*rule hash-map-preserving-imp-hash-target-program-zero*)
(rule record-staged-message-preserves-hash-map)

lemma *hash-relation-program-get:*
hash-relation-program R b 0
(get :: ('f protocol-channel, 'f protocol-channel) state-monad)
by (*rule hash-relation-program-zero*)
(rule hash-map-preserving-get)

lemma *hash-relation-program-assert:*
hash-relation-program R b 0
(assert x :: (unit, 'f protocol-channel) state-monad)
by (*rule hash-relation-program-zero*)
(rule hash-map-preserving-assert)

lemma *hash-relation-program-modify:*
assumes $\bigwedge s. \text{HashMap } (f\ s) = \text{HashMap } s$
shows *hash-relation-program R b 0 (modify f)*
by (*rule hash-relation-program-zero*)
(rule hash-map-preserving-modify[OF assms])

lemma *hash-relation-program-record-staged-message:*
hash-relation-program R b 0 (record-staged-message x)
by (*rule hash-relation-program-zero*)
(rule record-staged-message-preserves-hash-map)

lemma *record-staged-messages-outcome:*
assumes
Some ((), t) ∈
set-dist (execute (record-staged-messages xs) s)
shows
t = s⟦
PState := foldl concat (PState s) xs,
PTranscript := PTranscript s @ xs⟧

```

using assms
proof (induction xs arbitrary: s)
  case Nil
  then show ?case
    unfolding record-staged-messages-def by simp
next
  case (Cons x xs)
  from Cons.prems obtain u where
    head:
      Some ((), u)  $\in$ 
        set-dist (execute (record-staged-message x) s)
    and tail:
      Some ((), t)  $\in$ 
        set-dist (execute (record-staged-messages xs) u)
    unfolding record-staged-messages-def
    by (auto elim!: set-dist-bindE)
  have u:
    u = s⟦
      PState := concat (PState s) x,
      PTranscript := PTranscript s @ [x]⟧
    by (rule record-staged-message-outcome[OF head])
  have t:
    t = u⟦
      PState := foldl concat (PState u) xs,
      PTranscript := PTranscript u @ xs⟧
    by (rule Cons.IH[OF tail])
  show ?case
    unfolding t u by simp
qed

```

```

lemma record-staged-messages-preserves-hash-map:
  hash-map-preserving (record-staged-messages xs)
  unfolding hash-map-preserving-def
  using record-staged-messages-outcome by fastforce

```

```

lemma hash-range-budget-record-staged-messages:
  hash-range-budget 0 (record-staged-messages xs)
  using record-staged-messages-preserves-hash-map
  unfolding hash-map-preserving-def hash-range-budget-def
    hash-map-output-values-def
  by auto

```

```

lemma hash-collision-budget-record-staged-messages:
  hash-collision-budget 0 (record-staged-messages xs)
  by (rule hash-map-preserving-imp-hash-collision-budget-zero-semantic)
    (rule record-staged-messages-preserves-hash-map)

```

```

lemma hash-target-program-record-staged-messages:
  hash-target-program B 0 (record-staged-messages xs)

```

by (rule hash-map-preserving-imp-hash-target-program-zero)
 (rule record-staged-messages-preserves-hash-map)

lemma hash-relation-program-record-staged-messages:
 hash-relation-program R b 0 (record-staged-messages xs)
 by (rule hash-relation-program-zero)
 (rule record-staged-messages-preserves-hash-map)

lemma hash-relation-program-receive-counted-tagged-random-field-element:
 assumes bump: $\bigwedge s. \text{HashMap } (\text{bump } s) = \text{HashMap } s$
 and fibers: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$
 shows hash-relation-program R b 1
 (protocol-receive-counted-tagged-random-field-element counter bump tag ::
 ('f, 'f, unit) protocol-c-monad)

proof –

have modify-return:
 hash-relation-program R b $(0 + 0)$
 (modify bump $\gg=$ (λ -. return r) ::
 ('f, 'f protocol-channel) state-monad)
 for $r :: 'f$
 by (rule hash-relation-program-bind)
 (rule hash-relation-program-modify[OF bump],
 rule hash-relation-program-zero[OF hash-map-preserving-return])
 have hash-tail:
 hash-relation-program R b $(1 + (0 + 0))$
 (hash (tag (counter s) (PState s)) $\gg=$
 (λr . modify bump $\gg=$ (λ -. return r)) ::
 ('f, 'f protocol-channel) state-monad)
 for $s :: 'f$ protocol-channel
 by (rule hash-relation-program-bind)
 (rule hash-relation-program-hash[OF fibers], rule modify-return)
 have hash-relation-program R b $(0 + (1 + (0 + 0)))$
 (get $\gg=$
 ($\lambda s :: 'f$ protocol-channel.
 hash (tag (counter s) (PState s)) $\gg=$
 (λr . modify bump $\gg=$ (λ -. return r))))
 by (rule hash-relation-program-bind
 [OF hash-relation-program-get hash-tail])
 then show ?thesis
 unfolding protocol-receive-counted-tagged-random-field-element-def
 by simp
 qed

lemma hash-relation-program-receive-query-index-challenge:
 assumes fibers: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$
 shows hash-relation-program R b 1
 (receive-query-index-challenge :: ('f, 'f, unit) protocol-c-monad)
 unfolding receive-query-index-challenge-def
 by (rule hash-relation-program-receive-counted-tagged-random-field-element)

(simp-all add: fibers)

lemma hash-relation-program-receive-trace-fri-challenge:

assumes fibers: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$

shows hash-relation-program $R \ b \ 1$

(receive-trace-fri-challenge :: ('f, 'f, unit) protocol-c-monad)

unfolding receive-trace-fri-challenge-def

by (rule hash-relation-program-receive-counted-tagged-random-field-element)

(simp-all add: fibers)

lemma hash-relation-program-receive-composition-fri-challenge:

assumes fibers: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$

shows hash-relation-program $R \ b \ 1$

(receive-composition-fri-challenge :: ('f, 'f, unit) protocol-c-monad)

unfolding receive-composition-fri-challenge-def

by (rule hash-relation-program-receive-counted-tagged-random-field-element)

(simp-all add: fibers)

lemma hash-relation-program-receive-alpha-challenge:

assumes fibers: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$

shows hash-relation-program $R \ b \ 1$

(receive-alpha-challenge :: ('f, 'f, unit) protocol-c-monad)

unfolding receive-alpha-challenge-def

by (rule hash-relation-program-receive-counted-tagged-random-field-element)

(simp-all add: fibers)

lemma controlled-stage-outcome-fields:

assumes controlled: controlled-ro-program $q \ m$

and outcome: $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s)$

shows

$P\text{State } t = P\text{State } s$

$P\text{Transcript } t = P\text{Transcript } s$

$P\text{TraceFriCounter } t = P\text{TraceFriCounter } s$

$P\text{CompositionFriCounter } t = P\text{CompositionFriCounter } s$

$P\text{AlphaCounter } t = P\text{AlphaCounter } s$

$P\text{QueryCounter } t = P\text{QueryCounter } s$

using controlled-ro-program-preserves-protocol-fields[OF controlled]
outcome

unfolding protocol-fields-preserving-def

by blast+

lemma sum-list-take-Suc-drop:

assumes $i < \text{length } xs$

shows $\text{sum-list } (\text{take } (\text{Suc } n) (\text{drop } i \ xs)) =$

$xs \ ! \ i + \text{sum-list } (\text{take } n (\text{drop } (\text{Suc } i) \ xs))$

proof –

have drop-i:

$\text{drop } i \ xs = xs \ ! \ i \ \# \ \text{drop } (\text{Suc } i) \ xs$

using Cons-nth-drop-Suc[OF assms] **by** simp

```

  show ?thesis
    unfolding drop-i by simp
qed

lemma sum-list-take-le:
  fixes xs :: nat list
  shows sum-list (take n xs) ≤ sum-list xs
proof (induction n arbitrary: xs)
  case 0
  then show ?case by simp
next
  case (Suc n)
  then show ?case
    by (cases xs) simp-all
qed

lemma hash-range-budget-staged-trace-fri-program:
  assumes controlled: staged-adversary-controlled budgets A
  and bound: i + n ≤ length (trace-fri-budgets budgets)
  shows
    hash-range-budget
      (sum-list (take n (drop i (trace-fri-budgets budgets))) + n)
      (staged-trace-fri-program A i n bs)
  using bound
proof (induction n arbitrary: i bs)
  case 0
  then show ?case by (simp add: hash-range-budget-return)
next
  case (Suc n)
  have i-bound: i < length (trace-fri-budgets budgets)
    using Suc.prem1 by simp
  have stage:
    hash-range-budget (trace-fri-budgets budgets ! i)
      (trace-fri-root-stage A i bs)
    using controlled i-bound
  unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
  have record-budget:
     $\wedge$  root. hash-range-budget 0 (record-staged-message root)
    by (rule hash-range-budget-record-staged-message)
  have challenge: hash-range-budget 1 receive-trace-fri-challenge
    by (rule hash-range-budget-receive-trace-fri-challenge)
  have tail:
     $\wedge$  b. hash-range-budget
      (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) + n)
      (staged-trace-fri-program A (Suc i) n (bs @ [b]))
    by (rule Suc.IH) (use Suc.prem1 in simp)
  have tail-return:
     $\wedge$  b root. hash-range-budget

```



```

    (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) + n + 0)
    (staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ =
      ( $\lambda$ (roots, bs'). return (root # roots, bs'))))
  by (rule hash-range-budget-bind)
    (rule tail, simp add: hash-range-budget-return split: prod.splits)
have after-challenge:
 $\wedge$ root. hash-range-budget
  (1 +
    (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
      n + 0))
  (receive-trace-fri-challenge  $\gg$ =
    ( $\lambda$ b. staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ =
      ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
  by (rule hash-range-budget-bind)
    (rule challenge, rule tail-return)
have after-record:
 $\wedge$ root. hash-range-budget
  (0 +
    (1 +
      (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
        n + 0))))
  (record-staged-message root  $\gg$ =
    ( $\lambda$ -. receive-trace-fri-challenge  $\gg$ =
      ( $\lambda$ b. staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ =
        ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
  by (rule hash-range-budget-bind)
    (rule record-budget, rule after-challenge)
have whole:
  hash-range-budget
    (trace-fri-budgets budgets ! i +
      (0 +
        (1 +
          (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
            n + 0))))
    (staged-trace-fri-program A i (Suc n) bs)
  unfolding staged-trace-fri-program.simps
  by (rule hash-range-budget-bind)
    (rule stage, rule after-record)
have budget-eq:
  trace-fri-budgets budgets ! i +
    (0 +
      (1 +
        (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
          n + 0))) =
    sum-list (take (Suc n) (drop i (trace-fri-budgets budgets))) + Suc n
  using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
  using whole unfolding budget-eq .
qed

```

lemma *hash-collision-budget-staged-trace-fri-program:*
assumes *controlled: staged-adversary-controlled budgets A*
and *bound: $i + n \leq \text{length } (\text{trace-fri-budgets budgets})$*
shows
hash-collision-budget
 $(\text{sum-list } (\text{take } n \ (\text{drop } i \ (\text{trace-fri-budgets budgets}))) + n)$
 $(\text{staged-trace-fri-program } A \ i \ n \ bs)$
using *bound*
proof (*induction n arbitrary: i bs*)
case 0
then show ?*case* **by** (*simp add: hash-collision-budget-return*)
next
case (*Suc n*)
have *i-bound: $i < \text{length } (\text{trace-fri-budgets budgets})$*
using *Suc.prem* **by** *simp*
have *stage-range:*
hash-range-budget (*trace-fri-budgets budgets ! i*)
 $(\text{trace-fri-root-stage } A \ i \ bs)$
using *controlled i-bound*
unfolding *staged-adversary-controlled-def*
by (*blast intro: controlled-ro-program-range*)
have *stage-coll:*
hash-collision-budget (*trace-fri-budgets budgets ! i*)
 $(\text{trace-fri-root-stage } A \ i \ bs)$
using *controlled i-bound*
unfolding *staged-adversary-controlled-def*
by (*blast intro: controlled-ro-program-collision*)
have *record-range:*
 $\bigwedge \text{root. hash-range-budget } 0 \ (\text{record-staged-message root})$
by (*rule hash-range-budget-record-staged-message*)
have *record-coll:*
 $\bigwedge \text{root. hash-collision-budget } 0 \ (\text{record-staged-message root})$
by (*rule hash-collision-budget-record-staged-message*)
have *challenge-range: hash-range-budget 1 receive-trace-fri-challenge*
by (*rule hash-range-budget-receive-trace-fri-challenge*)
have *challenge-coll: hash-collision-budget 1 receive-trace-fri-challenge*
by (*rule hash-collision-budget-receive-trace-fri-challenge*)
have *tail-range:*
 $\bigwedge b. \text{hash-range-budget}$
 $(\text{sum-list } (\text{take } n \ (\text{drop } (\text{Suc } i) \ (\text{trace-fri-budgets budgets}))) + n)$
 $(\text{staged-trace-fri-program } A \ (\text{Suc } i) \ n \ (bs @ [b]))$
by (*rule hash-range-budget-staged-trace-fri-program[OF controlled]*)
(use Suc.prem in simp)
have *tail-coll:*
 $\bigwedge b. \text{hash-collision-budget}$
 $(\text{sum-list } (\text{take } n \ (\text{drop } (\text{Suc } i) \ (\text{trace-fri-budgets budgets}))) + n)$
 $(\text{staged-trace-fri-program } A \ (\text{Suc } i) \ n \ (bs @ [b]))$
by (*rule Suc.IH*) *(use Suc.prem in simp)*

have *tail-return-range*:
 $\wedge b$ *root*. *hash-range-budget*
 $(\text{sum-list } (\text{take } n \ (\text{drop } (\text{Suc } i) \ (\text{trace-fri-budgets } \text{budgets}))) + n + 0)$
 $(\text{staged-trace-fri-program } A \ (\text{Suc } i) \ n \ (\text{bs } @ \ [b]) \gg=$
 $(\lambda(\text{roots}, \text{bs}'). \text{return } (\text{root } \# \ \text{roots}, \text{bs}')))$
by (*rule hash-range-budget-bind*)
 $(\text{rule } \text{tail-range}, \text{simp add: hash-range-budget-return split: prod.splits})$
have *tail-return-coll*:
 $\wedge b$ *root*. *hash-collision-budget*
 $(\text{sum-list } (\text{take } n \ (\text{drop } (\text{Suc } i) \ (\text{trace-fri-budgets } \text{budgets}))) + n + 0)$
 $(\text{staged-trace-fri-program } A \ (\text{Suc } i) \ n \ (\text{bs } @ \ [b]) \gg=$
 $(\lambda(\text{roots}, \text{bs}'). \text{return } (\text{root } \# \ \text{roots}, \text{bs}')))$
by (*rule hash-collision-budget-bind*)
 $(\text{rule } \text{tail-range}, \text{rule } \text{tail-coll},$
 $\text{simp add: hash-range-budget-return split: prod.splits},$
 $\text{simp add: hash-collision-budget-return split: prod.splits})$
have *after-challenge-range*:
 $\wedge \text{root}$. *hash-range-budget*
 $(1 +$
 $(\text{sum-list } (\text{take } n \ (\text{drop } (\text{Suc } i) \ (\text{trace-fri-budgets } \text{budgets}))) +$
 $n + 0))$
 $(\text{receive-trace-fri-challenge } \gg=$
 $(\lambda b. \text{staged-trace-fri-program } A \ (\text{Suc } i) \ n \ (\text{bs } @ \ [b]) \gg=$
 $(\lambda(\text{roots}, \text{bs}'). \text{return } (\text{root } \# \ \text{roots}, \text{bs}'))))$
by (*rule hash-range-budget-bind*)
 $(\text{rule } \text{challenge-range}, \text{rule } \text{tail-return-range})$
have *after-challenge-coll*:
 $\wedge \text{root}$. *hash-collision-budget*
 $(1 +$
 $(\text{sum-list } (\text{take } n \ (\text{drop } (\text{Suc } i) \ (\text{trace-fri-budgets } \text{budgets}))) +$
 $n + 0))$
 $(\text{receive-trace-fri-challenge } \gg=$
 $(\lambda b. \text{staged-trace-fri-program } A \ (\text{Suc } i) \ n \ (\text{bs } @ \ [b]) \gg=$
 $(\lambda(\text{roots}, \text{bs}'). \text{return } (\text{root } \# \ \text{roots}, \text{bs}'))))$
by (*rule hash-collision-budget-bind*)
 $(\text{rule } \text{challenge-range}, \text{rule } \text{challenge-coll}, \text{rule } \text{tail-return-range},$
 $\text{rule } \text{tail-return-coll})$
have *after-record-range*:
 $\wedge \text{root}$. *hash-range-budget*
 $(0 +$
 $(1 +$
 $(\text{sum-list } (\text{take } n \ (\text{drop } (\text{Suc } i) \ (\text{trace-fri-budgets } \text{budgets}))) +$
 $n + 0)))$
 $(\text{record-staged-message } \text{root } \gg=$
 $(\lambda-. \text{receive-trace-fri-challenge } \gg=$
 $(\lambda b. \text{staged-trace-fri-program } A \ (\text{Suc } i) \ n \ (\text{bs } @ \ [b]) \gg=$
 $(\lambda(\text{roots}, \text{bs}'). \text{return } (\text{root } \# \ \text{roots}, \text{bs}'))))$
by (*rule hash-range-budget-bind*)
 $(\text{rule } \text{record-range}, \text{rule } \text{after-challenge-range})$

```

have after-record-coll:
   $\wedge$  root. hash-collision-budget
    (0 +
      (1 +
        (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
          n + 0)))
    (record-staged-message root  $\gg$ 
      ( $\lambda$ -. receive-trace-fri-challenge  $\gg$ 
        ( $\lambda$ b. staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ 
          ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
  by (rule hash-collision-budget-bind)
    (rule record-range, rule record-coll, rule after-challenge-range,
      rule after-challenge-coll)
have whole:
  hash-collision-budget
    (trace-fri-budgets budgets ! i +
      (0 +
        (1 +
          (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
            n + 0))))
    (staged-trace-fri-program A i (Suc n) bs)
unfolding staged-trace-fri-program.simps
by (rule hash-collision-budget-bind)
    (rule stage-range, rule stage-coll, rule after-record-range,
      rule after-record-coll)
have budget-eq:
  trace-fri-budgets budgets ! i +
    (0 +
      (1 +
        (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
          n + 0))) =
    sum-list (take (Suc n) (drop i (trace-fri-budgets budgets))) + Suc n
using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
using whole unfolding budget-eq .
qed

lemma hash-range-budget-staged-alpha-program:
  hash-range-budget n (staged-alpha-program n)
proof (induction n)
  case 0
  then show ?case by (simp add: hash-range-budget-return)
next
  case (Suc n)
  have challenge: hash-range-budget 1 receive-alpha-challenge
    by (rule hash-range-budget-receive-alpha-challenge)
  have record-budget:
     $\wedge$  a. hash-range-budget 0 (record-staged-message a)
    by (rule hash-range-budget-record-staged-message)

```

```

have tail-return:
   $\wedge a.$  hash-range-budget ( $n + 0$ )
    (staged-alpha-program  $n \gg= (\lambda as. \text{return } (a \# as))$ )
  by (rule hash-range-budget-bind)
    (rule Suc.IH, rule hash-range-budget-return)
have after-record:
   $\wedge a.$  hash-range-budget ( $0 + (n + 0)$ )
    (record-staged-message  $a \gg=$ 
      ( $\lambda-. \text{staged-alpha-program } n \gg= (\lambda as. \text{return } (a \# as))$ )))
  by (rule hash-range-budget-bind)
    (rule record-budget, rule tail-return)
have hash-range-budget ( $1 + (0 + (n + 0))$ )
  (staged-alpha-program (Suc  $n$ ))
  unfolding staged-alpha-program.simps
  by (rule hash-range-budget-bind)
    (rule challenge, rule after-record)
then show ?case by simp
qed

lemma hash-collision-budget-staged-alpha-program:
  hash-collision-budget  $n$  (staged-alpha-program  $n$ )
proof (induction  $n$ )
  case 0
  then show ?case by (simp add: hash-collision-budget-return)
next
  case (Suc  $n$ )
  have challenge-range: hash-range-budget 1 receive-alpha-challenge
    by (rule hash-range-budget-receive-alpha-challenge)
  have challenge-coll: hash-collision-budget 1 receive-alpha-challenge
    by (rule hash-collision-budget-receive-alpha-challenge)
  have record-range:
     $\wedge a.$  hash-range-budget 0 (record-staged-message  $a$ )
    by (rule hash-range-budget-record-staged-message)
  have record-coll:
     $\wedge a.$  hash-collision-budget 0 (record-staged-message  $a$ )
    by (rule hash-collision-budget-record-staged-message)
  have tail-return-range:
     $\wedge a.$  hash-range-budget ( $n + 0$ )
      (staged-alpha-program  $n \gg= (\lambda as. \text{return } (a \# as))$ )
    by (rule hash-range-budget-bind)
      (rule hash-range-budget-staged-alpha-program,
        rule hash-range-budget-return)
  have tail-return-coll:
     $\wedge a.$  hash-collision-budget ( $n + 0$ )
      (staged-alpha-program  $n \gg= (\lambda as. \text{return } (a \# as))$ )
    by (rule hash-collision-budget-bind)
      (rule hash-range-budget-staged-alpha-program, rule Suc.IH,
        rule hash-range-budget-return, rule hash-collision-budget-return)
  have after-record-range:

```

```

 $\wedge a.$  hash-range-budget  $(0 + (n + 0))$ 
  (record-staged-message  $a \gg=$ 
    ( $\lambda -. \text{staged-alpha-program } n \gg= (\lambda as. \text{return } (a \# as))$ )))
by (rule hash-range-budget-bind)
  (rule record-range, rule tail-return-range)
have after-record-coll:
 $\wedge a.$  hash-collision-budget  $(0 + (n + 0))$ 
  (record-staged-message  $a \gg=$ 
    ( $\lambda -. \text{staged-alpha-program } n \gg= (\lambda as. \text{return } (a \# as))$ )))
proof (rule hash-collision-budget-bind)
  fix a
  show hash-range-budget 0 (record-staged-message a)
    by (rule record-range)
  show hash-collision-budget 0 (record-staged-message a)
    by (rule record-coll)
  show  $\wedge x.$  hash-range-budget  $(n + 0)$ 
    (staged-alpha-program  $n \gg= (\lambda as. \text{return } (a \# as))$ )
    by (rule tail-return-range)
  show  $\wedge x.$  hash-collision-budget  $(n + 0)$ 
    (staged-alpha-program  $n \gg= (\lambda as. \text{return } (a \# as))$ )
    by (rule tail-return-coll)
qed
have hash-collision-budget  $(1 + (0 + (n + 0)))$ 
  (staged-alpha-program (Suc n))
  unfolding staged-alpha-program.simps
  by (rule hash-collision-budget-bind)
    (rule challenge-range, rule challenge-coll, rule after-record-range,
      rule after-record-coll)
then show ?case by simp
qed

lemma hash-range-budget-staged-composition-fri-program:
  assumes controlled: staged-adversary-controlled budgets A
  and bound:  $i + n \leq \text{length } (\text{composition-fri-budgets budgets})$ 
  shows
    hash-range-budget
      (sum-list (take n (drop i (composition-fri-budgets budgets))) + n)
      (staged-composition-fri-program A dg i n bs)
  using bound
proof (induction n arbitrary: i bs)
  case 0
  then show ?case by (simp add: hash-range-budget-return)
next
  case (Suc n)
  have i-bound:  $i < \text{length } (\text{composition-fri-budgets budgets})$ 
  using Suc.prem1 by simp
  have stage:
    hash-range-budget (composition-fri-budgets budgets ! i)
      (composition-fri-root-stage A dg i bs)

```

```

using controlled i-bound
unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-range)
have record-budget:
   $\wedge$  root. hash-range-budget 0 (record-staged-message root)
  by (rule hash-range-budget-record-staged-message)
have challenge:
  hash-range-budget 1 receive-composition-fri-challenge
  by (rule hash-range-budget-receive-composition-fri-challenge)
have tail:
   $\wedge$  b. hash-range-budget
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) + n)
      (staged-composition-fri-program A dg (Suc i) n (bs @ [b]))
    )
  by (rule Suc.IH) (use Suc.premis in simp)
have tail-return:
   $\wedge$  b root. hash-range-budget
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) + n + 0)
      (staged-composition-fri-program A dg (Suc i) n (bs @ [b])  $\gg$ 
        ( $\lambda$ (roots, bs'). return (root # roots, bs')))
    )
  by (rule hash-range-budget-bind)
    (rule tail, simp add: hash-range-budget-return split: prod.splits)
have after-challenge:
   $\wedge$  root. hash-range-budget
    (1 +
      (sum-list
        (take n (drop (Suc i) (composition-fri-budgets budgets))) +
          n + 0))
      (receive-composition-fri-challenge  $\gg$ 
        ( $\lambda$ b. staged-composition-fri-program A dg (Suc i) n
          (bs @ [b])  $\gg$ 
            ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
    )
  by (rule hash-range-budget-bind)
    (rule challenge, rule tail-return)
have after-record:
   $\wedge$  root. hash-range-budget
    (0 +
      (1 +
        (sum-list
          (take n (drop (Suc i) (composition-fri-budgets budgets))) +
            n + 0)))
      (record-staged-message root  $\gg$ 
        ( $\lambda$ -. receive-composition-fri-challenge  $\gg$ 
          ( $\lambda$ b. staged-composition-fri-program A dg (Suc i) n
            (bs @ [b])  $\gg$ 
              ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
        )
    )
  by (rule hash-range-budget-bind)
    (rule record-budget, rule after-challenge)

```

```

have whole:
  hash-range-budget
    (composition-fri-budgets budgets ! i +
      (0 +
        (1 +
          (sum-list
            (take n (drop (Suc i) (composition-fri-budgets budgets))) +
              n + 0))))
    (staged-composition-fri-program A dg i (Suc n) bs)
unfolding staged-composition-fri-program.simps
by (rule hash-range-budget-bind)
    (rule stage, rule after-record)
have budget-eq:
  composition-fri-budgets budgets ! i +
    (0 +
      (1 +
        (sum-list
          (take n (drop (Suc i) (composition-fri-budgets budgets))) +
            n + 0))) =
    sum-list (take (Suc n) (drop i (composition-fri-budgets budgets))) +
      Suc n
using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
using whole unfolding budget-eq .
qed

lemma hash-collision-budget-staged-composition-fri-program:
assumes controlled: staged-adversary-controlled budgets A
and bound:  $i + n \leq \text{length } (\text{composition-fri-budgets budgets})$ 
shows
  hash-collision-budget
    (sum-list (take n (drop i (composition-fri-budgets budgets))) + n)
    (staged-composition-fri-program A dg i n bs)
using bound
proof (induction n arbitrary: i bs)
case 0
then show ?case by (simp add: hash-collision-budget-return)
next
case (Suc n)
have i-bound:  $i < \text{length } (\text{composition-fri-budgets budgets})$ 
using Suc.prem1 by simp
have stage-range:
  hash-range-budget (composition-fri-budgets budgets ! i)
    (composition-fri-root-stage A dg i bs)
using controlled i-bound
unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-range)
have stage-coll:
  hash-collision-budget (composition-fri-budgets budgets ! i)

```



```

    (composition-fri-root-stage A dg i bs)
using controlled i-bound
unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-collision)
have record-range:
   $\wedge$  root. hash-range-budget 0 (record-staged-message root)
  by (rule hash-range-budget-record-staged-message)
have record-coll:
   $\wedge$  root. hash-collision-budget 0 (record-staged-message root)
  by (rule hash-collision-budget-record-staged-message)
have challenge-range:
  hash-range-budget 1 receive-composition-fri-challenge
  by (rule hash-range-budget-receive-composition-fri-challenge)
have challenge-coll:
  hash-collision-budget 1 receive-composition-fri-challenge
  by (rule hash-collision-budget-receive-composition-fri-challenge)
have tail-range:
   $\wedge$  b. hash-range-budget
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) + n)
      (staged-composition-fri-program A dg (Suc i) n (bs @ [b])))
  by (rule hash-range-budget-staged-composition-fri-program
    [OF controlled])
    (use Suc.premis in simp)
have tail-coll:
   $\wedge$  b. hash-collision-budget
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) + n)
      (staged-composition-fri-program A dg (Suc i) n (bs @ [b])))
  by (rule Suc.IH) (use Suc.premis in simp)
have tail-return-range:
   $\wedge$  b root. hash-range-budget
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) + n + 0)
      (staged-composition-fri-program A dg (Suc i) n (bs @ [b])  $\gg$ 
        ( $\lambda$ (roots, bs'). return (root # roots, bs'))))
  by (rule hash-range-budget-bind)
    (rule tail-range, simp add: hash-range-budget-return split: prod.splits)
have tail-return-coll:
   $\wedge$  b root. hash-collision-budget
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) + n + 0)
      (staged-composition-fri-program A dg (Suc i) n (bs @ [b])  $\gg$ 
        ( $\lambda$ (roots, bs'). return (root # roots, bs'))))
  by (rule hash-collision-budget-bind)
    (rule tail-range, rule tail-coll,
      simp add: hash-range-budget-return split: prod.splits,
      simp add: hash-collision-budget-return split: prod.splits)
have after-challenge-range:

```

```

 $\wedge$ root. hash-range-budget
  (1 +
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) +
      n + 0))
    (receive-composition-fri-challenge  $\gg$ 
      ( $\lambda$ b. staged-composition-fri-program A dg (Suc i) n
        (bs @ [b])  $\gg$ 
          ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
  by (rule hash-range-budget-bind)
    (rule challenge-range, rule tail-return-range)
have after-challenge-coll:
 $\wedge$ root. hash-collision-budget
  (1 +
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) +
      n + 0))
    (receive-composition-fri-challenge  $\gg$ 
      ( $\lambda$ b. staged-composition-fri-program A dg (Suc i) n
        (bs @ [b])  $\gg$ 
          ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
  by (rule hash-collision-budget-bind)
    (rule challenge-range, rule challenge-coll, rule tail-return-range,
      rule tail-return-coll)
have after-record-range:
 $\wedge$ root. hash-range-budget
  (0 +
    (1 +
      (sum-list
        (take n (drop (Suc i) (composition-fri-budgets budgets))) +
        n + 0))))
    (record-staged-message root  $\gg$ 
      ( $\lambda$ -. receive-composition-fri-challenge  $\gg$ 
        ( $\lambda$ b. staged-composition-fri-program A dg (Suc i) n
          (bs @ [b])  $\gg$ 
            ( $\lambda$ (roots, bs'). return (root # roots, bs'))))))
  by (rule hash-range-budget-bind)
    (rule record-range, rule after-challenge-range)
have after-record-coll:
 $\wedge$ root. hash-collision-budget
  (0 +
    (1 +
      (sum-list
        (take n (drop (Suc i) (composition-fri-budgets budgets))) +
        n + 0))))
    (record-staged-message root  $\gg$ 
      ( $\lambda$ -. receive-composition-fri-challenge  $\gg$ 
        ( $\lambda$ b. staged-composition-fri-program A dg (Suc i) n
          (bs @ [b])  $\gg$ 
            ( $\lambda$ (roots, bs'). return (root # roots, bs'))))))

```

```

      (λ(roots, bs'). return (root # roots, bs')))))
  by (rule hash-collision-budget-bind)
    (rule record-range, rule record-coll, rule after-challenge-range,
     rule after-challenge-coll)
  have whole:
    hash-collision-budget
      (composition-fri-budgets budgets ! i +
       (0 +
        (1 +
         (sum-list
          (take n (drop (Suc i) (composition-fri-budgets budgets))) +
           n + 0))))
      (staged-composition-fri-program A dg i (Suc n) bs)
  unfolding staged-composition-fri-program.simps
  by (rule hash-collision-budget-bind)
    (rule stage-range, rule stage-coll, rule after-record-range,
     rule after-record-coll)
  have budget-eq:
    composition-fri-budgets budgets ! i +
      (0 +
       (1 +
        (sum-list
         (take n (drop (Suc i) (composition-fri-budgets budgets))) +
          n + 0))) =
    sum-list (take (Suc n) (drop i (composition-fri-budgets budgets))) +
      Suc n
  using sum-list-take-Suc-drop[OF i-bound, of n] by simp
  show ?case
    using whole unfolding budget-eq .
qed

lemma hash-range-budget-staged-query-program:
  assumes controlled: staged-adversary-controlled budgets A
    and bound: i + n ≤ length (query-opening-budgets budgets)
  shows
    hash-range-budget
      (sum-list (take n (drop i (query-opening-budgets budgets))) + n)
      (staged-query-program A i n)
  using bound
proof (induction n arbitrary: i)
  case 0
  then show ?case by (simp add: hash-range-budget-return)
next
  case (Suc n)
  have i-bound: i < length (query-opening-budgets budgets)
  using Suc.prem1 by simp
  have challenge: hash-range-budget 1 receive-query-index-challenge
  by (rule hash-range-budget-receive-query-index-challenge)
  have stage:

```

```

 $\wedge$ raw. hash-range-budget (query-opening-budgets budgets ! i)
  (query-opening-stage A i raw)
using controlled i-bound
unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-range)
have record-budget:
 $\wedge$ chunk. hash-range-budget 0 (record-staged-messages chunk)
by (rule hash-range-budget-record-staged-messages)
have tail:
  hash-range-budget
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n)
    (staged-query-program A (Suc i) n)
by (rule Suc.IH) (use Suc.prem in simp)
have tail-return:
 $\wedge$ chunk. hash-range-budget
  (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
    n + 0)
  (staged-query-program A (Suc i) n  $\gg$ 
    ( $\lambda$ chunks. return (chunk # chunks)))
by (rule hash-range-budget-bind)
  (rule tail, rule hash-range-budget-return)
have after-record:
 $\wedge$ chunk. hash-range-budget
  (0 +
    (sum-list
      (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n + 0))
  (record-staged-messages chunk  $\gg$ 
    ( $\lambda$ -. staged-query-program A (Suc i) n  $\gg$ 
      ( $\lambda$ chunks. return (chunk # chunks)))))
by (rule hash-range-budget-bind)
  (rule record-budget, rule tail-return)
have after-stage:
 $\wedge$ raw. hash-range-budget
  (query-opening-budgets budgets ! i +
    (0 +
      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0)))
  (query-opening-stage A i raw  $\gg$ 
    ( $\lambda$ chunk. record-staged-messages chunk  $\gg$ 
      ( $\lambda$ -. staged-query-program A (Suc i) n  $\gg$ 
        ( $\lambda$ chunks. return (chunk # chunks)))))
by (rule hash-range-budget-bind)
  (rule stage, rule after-record)
have whole:
  hash-range-budget
    (1 +

```

```

    (query-opening-budgets budgets ! i +
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0))))
    (staged-query-program A i (Suc n))
  unfolding staged-query-program.simps
  by (rule hash-range-budget-bind)
    (rule challenge, rule after-stage)
  have budget-eq:
    1 +
      (query-opening-budgets budgets ! i +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
              n + 0))) =
      sum-list (take (Suc n) (drop i (query-opening-budgets budgets))) +
        Suc n
    using sum-list-take-Suc-drop[OF i-bound, of n] by simp
  show ?case
    using whole unfolding budget-eq .
qed

lemma hash-collision-budget-staged-query-program:
  assumes controlled: staged-adversary-controlled budgets A
    and bound:  $i + n \leq \text{length } (\text{query-opening-budgets budgets})$ 
  shows
    hash-collision-budget
      (sum-list (take n (drop i (query-opening-budgets budgets))) + n)
      (staged-query-program A i n)
    using bound
  proof (induction n arbitrary: i)
    case 0
    then show ?case by (simp add: hash-collision-budget-return)
  next
    case (Suc n)
    have i-bound:  $i < \text{length } (\text{query-opening-budgets budgets})$ 
      using Suc.premis by simp
    have challenge-range: hash-range-budget 1 receive-query-index-challenge
      by (rule hash-range-budget-receive-query-index-challenge)
    have challenge-coll: hash-collision-budget 1 receive-query-index-challenge
      by (rule hash-collision-budget-receive-query-index-challenge)
    have stage-range:
       $\bigwedge \text{raw. hash-range-budget } (\text{query-opening-budgets budgets ! } i)
        (\text{query-opening-stage } A \ i \ \text{raw})$ 
      using controlled i-bound
    unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-range)
    have stage-coll:

```

```

 $\wedge$ raw. hash-collision-budget (query-opening-budgets budgets ! i)
  (query-opening-stage A i raw)
using controlled i-bound
unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-collision)
have record-range:
 $\wedge$ chunk. hash-range-budget 0 (record-staged-messages chunk)
by (rule hash-range-budget-record-staged-messages)
have record-coll:
 $\wedge$ chunk. hash-collision-budget 0 (record-staged-messages chunk)
by (rule hash-collision-budget-record-staged-messages)
have tail-range:
  hash-range-budget
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n)
    (staged-query-program A (Suc i) n)
by (rule hash-range-budget-staged-query-program[OF controlled])
    (use Suc.premis in simp)
have tail-coll:
  hash-collision-budget
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n)
    (staged-query-program A (Suc i) n)
by (rule Suc.IH) (use Suc.premis in simp)
have tail-return-range:
 $\wedge$ chunk. hash-range-budget
  (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
    n + 0)
  (staged-query-program A (Suc i) n  $\gg=$ 
    ( $\lambda$ chunks. return (chunk # chunks)))
by (rule hash-range-budget-bind)
    (rule tail-range, rule hash-range-budget-return)
have tail-return-coll:
 $\wedge$ chunk. hash-collision-budget
  (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
    n + 0)
  (staged-query-program A (Suc i) n  $\gg=$ 
    ( $\lambda$ chunks. return (chunk # chunks)))
by (rule hash-collision-budget-bind)
    (rule tail-range, rule tail-coll, rule hash-range-budget-return,
      rule hash-collision-budget-return)
have after-record-range:
 $\wedge$ chunk. hash-range-budget
  (0 +
    (sum-list
      (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n + 0))
  (record-staged-messages chunk  $\gg=$ 
    ( $\lambda$ -. staged-query-program A (Suc i) n  $\gg=$ 

```

```

    (λchunks. return (chunk # chunks))))
  by (rule hash-range-budget-bind)
    (rule record-range, rule tail-return-range)
have after-record-coll:
  ∧chunk. hash-collision-budget
    (0 +
      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0))
    (record-staged-messages chunk >=>
      (λ-. staged-query-program A (Suc i) n >=>
        (λchunks. return (chunk # chunks))))
  by (rule hash-collision-budget-bind)
    (rule record-range, rule record-coll, rule tail-return-range,
      rule tail-return-coll)
have after-stage-range:
  ∧raw. hash-range-budget
    (query-opening-budgets budgets ! i +
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
          n + 0)))
    (query-opening-stage A i raw >=>
      (λchunk. record-staged-messages chunk >=>
        (λ-. staged-query-program A (Suc i) n >=>
          (λchunks. return (chunk # chunks))))))
  by (rule hash-range-budget-bind)
    (rule stage-range, rule after-record-range)
have after-stage-coll:
  ∧raw. hash-collision-budget
    (query-opening-budgets budgets ! i +
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
          n + 0)))
    (query-opening-stage A i raw >=>
      (λchunk. record-staged-messages chunk >=>
        (λ-. staged-query-program A (Suc i) n >=>
          (λchunks. return (chunk # chunks))))))
  by (rule hash-collision-budget-bind)
    (rule stage-range, rule stage-coll, rule after-record-range,
      rule after-record-coll)
have whole:
  hash-collision-budget
    (1 +
      (query-opening-budgets budgets ! i +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +

```

```

      n + 0))))
    (staged-query-program A i (Suc n))
  unfolding staged-query-program.simps
  by (rule hash-collision-budget-bind)
    (rule challenge-range, rule challenge-coll, rule after-stage-range,
      rule after-stage-coll)
  have budget-eq:
    1 +
      (query-opening-budgets budgets ! i +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
              n + 0))) =
      sum-list (take (Suc n) (drop i (query-opening-budgets budgets))) +
        Suc n
    using sum-list-take-Suc-drop[OF i-bound, of n] by simp
  show ?case
    using whole unfolding budget-eq .
qed

```

```

lemma hash-target-program-staged-trace-fri-program:
  assumes controlled: staged-adversary-controlled budgets A
    and bound:  $i + n \leq \text{length } (\text{trace-fri-budgets budgets})$ 
  shows
    hash-target-program B
      (sum-list (take n (drop i (trace-fri-budgets budgets))) + n)
      (staged-trace-fri-program A i n bs)
    using bound
  proof (induction n arbitrary: i bs)
    case 0
    then show ?case by (simp add: hash-target-program-return)
  next
    case (Suc n)
    have i-bound:  $i < \text{length } (\text{trace-fri-budgets budgets})$ 
      using Suc.premis by simp
    have stage:
      hash-target-program B (trace-fri-budgets budgets ! i)
        (trace-fri-root-stage A i bs)
      using controlled i-bound
    unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-target)
    have record-budget:
       $\wedge \text{root. hash-target-program B } 0 \text{ (record-staged-message root)}$ 
      by (rule hash-target-program-record-staged-message)
    have challenge: hash-target-program B 1 receive-trace-fri-challenge
      by (rule hash-target-program-receive-trace-fri-challenge)
    have tail:
       $\wedge b. \text{hash-target-program B}$ 
        (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) + n)

```



```

    (staged-trace-fri-program A (Suc i) n (bs @ [b]))
  by (rule Suc.IH) (use Suc.premis in simp)
have tail-return:
   $\wedge b$  root. hash-target-program B
  (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) + n + 0)
  (staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ 
    ( $\lambda$ (roots, bs'). return (root # roots, bs')))
  by (rule hash-target-program-bind)
  (rule tail, simp add: hash-target-program-return split: prod.splits)
have after-challenge:
   $\wedge$ root. hash-target-program B
  (1 +
    (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
      n + 0))
  (receive-trace-fri-challenge  $\gg$ 
    ( $\lambda b$ . staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ 
      ( $\lambda$ (roots, bs'). return (root # roots, bs'))))
  by (rule hash-target-program-bind)
  (rule challenge, rule tail-return)
have after-record:
   $\wedge$ root. hash-target-program B
  (0 +
    (1 +
      (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
        n + 0))))
  (record-staged-message root  $\gg$ 
    ( $\lambda$ -. receive-trace-fri-challenge  $\gg$ 
      ( $\lambda b$ . staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ 
        ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
  by (rule hash-target-program-bind)
  (rule record-budget, rule after-challenge)
have whole:
  hash-target-program B
  (trace-fri-budgets budgets ! i +
    (0 +
      (1 +
        (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
          n + 0))))
  (staged-trace-fri-program A i (Suc n) bs)
unfolding staged-trace-fri-program.simps
by (rule hash-target-program-bind)
  (rule stage, rule after-record)
have budget-eq:
  trace-fri-budgets budgets ! i +
  (0 +
    (1 +
      (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
        n + 0))) =
  sum-list (take (Suc n) (drop i (trace-fri-budgets budgets))) + Suc n

```

```

    using sum-list-take-Suc-drop[OF i-bound, of n] by simp
  show ?case
    using whole unfolding budget-eq .
qed

lemma hash-target-program-staged-alpha-program:
  hash-target-program B n (staged-alpha-program n)
proof (induction n)
  case 0
  then show ?case by (simp add: hash-target-program-return)
next
  case (Suc n)
  have challenge: hash-target-program B 1 receive-alpha-challenge
    by (rule hash-target-program-receive-alpha-challenge)
  have record-budget:
     $\wedge a.$  hash-target-program B 0 (record-staged-message a)
    by (rule hash-target-program-record-staged-message)
  have tail-return:
     $\wedge a.$  hash-target-program B (n + 0)
      (staged-alpha-program n  $\gg$  ( $\lambda as.$  return (a # as)))
    by (rule hash-target-program-bind)
      (rule Suc.IH, rule hash-target-program-return)
  have after-record:
     $\wedge a.$  hash-target-program B (0 + (n + 0))
      (record-staged-message a  $\gg$ 
        ( $\lambda -. staged-alpha-program n \gg (\lambda as.$  return (a # as))))
    by (rule hash-target-program-bind)
      (rule record-budget, rule tail-return)
  have hash-target-program B (1 + (0 + (n + 0)))
    (staged-alpha-program (Suc n))
    unfolding staged-alpha-program.simps
    by (rule hash-target-program-bind)
      (rule challenge, rule after-record)
  then show ?case by simp
qed

```

```

lemma hash-target-program-staged-composition-fri-program:
  assumes controlled: staged-adversary-controlled budgets A
    and bound:  $i + n \leq \text{length } (\text{composition-fri-budgets budgets})$ 
  shows
    hash-target-program B
      (sum-list (take n (drop i (composition-fri-budgets budgets))) + n)
      (staged-composition-fri-program A dg i n bs)
  using bound
proof (induction n arbitrary: i bs)
  case 0
  then show ?case by (simp add: hash-target-program-return)
next
  case (Suc n)

```

```

have i-bound:  $i < \text{length } (\text{composition-fri-budgets } \text{budgets})$ 
  using Suc.prems by simp
have stage:
  hash-target-program B (composition-fri-budgets budgets ! i)
    (composition-fri-root-stage A dg i bs)
  using controlled i-bound
  unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have record-budget:
   $\wedge \text{root. } \text{hash-target-program } B \ 0 \ (\text{record-staged-message } \text{root})$ 
  by (rule hash-target-program-record-staged-message)
have challenge:
  hash-target-program B 1 receive-composition-fri-challenge
  by (rule hash-target-program-receive-composition-fri-challenge)
have tail:
   $\wedge b. \text{hash-target-program } B$ 
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) + n)
    (staged-composition-fri-program A dg (Suc i) n (bs @ [b]))
  by (rule Suc.IH) (use Suc.prems in simp)
have tail-return:
   $\wedge b \text{ root. } \text{hash-target-program } B$ 
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) + n + 0)
    (staged-composition-fri-program A dg (Suc i) n (bs @ [b]))  $\gg=$ 
    ( $\lambda(\text{roots}, \text{bs}'). \text{return } (\text{root} \# \text{roots}, \text{bs}')$ )
  by (rule hash-target-program-bind)
    (rule tail, simp add: hash-target-program-return split: prod.splits)
have after-challenge:
   $\wedge \text{root. } \text{hash-target-program } B$ 
    ( $1 +$ 
      (sum-list
        (take n (drop (Suc i) (composition-fri-budgets budgets))) +
        n + 0))
    (receive-composition-fri-challenge)  $\gg=$ 
    ( $\lambda b. \text{staged-composition-fri-program } A \ dg \ (Suc \ i) \ n$ 
      (bs @ [b])  $\gg=$ 
      ( $\lambda(\text{roots}, \text{bs}'). \text{return } (\text{root} \# \text{roots}, \text{bs}')$ )))
  by (rule hash-target-program-bind)
    (rule challenge, rule tail-return)
have after-record:
   $\wedge \text{root. } \text{hash-target-program } B$ 
    ( $0 +$ 
      ( $1 +$ 
        (sum-list
          (take n (drop (Suc i) (composition-fri-budgets budgets))) +
          n + 0)))
    (record-staged-message root)  $\gg=$ 
    ( $\lambda-. \text{receive-composition-fri-challenge}$ )  $\gg=$ 

```

```

    (λb. staged-composition-fri-program A dg (Suc i) n
      (bs @ [b]) >>=
      (λ(roots, bs'). return (root # roots, bs')))))
  by (rule hash-target-program-bind)
    (rule record-budget, rule after-challenge)
have whole:
  hash-target-program B
    (composition-fri-budgets budgets ! i +
      (0 +
        (1 +
          (sum-list
            (take n (drop (Suc i) (composition-fri-budgets budgets))) +
              n + 0))))
      (staged-composition-fri-program A dg i (Suc n) bs)
  unfolding staged-composition-fri-program.simps
  by (rule hash-target-program-bind)
    (rule stage, rule after-record)
have budget-eq:
  composition-fri-budgets budgets ! i +
    (0 +
      (1 +
        (sum-list
          (take n (drop (Suc i) (composition-fri-budgets budgets))) +
            n + 0))) =
    sum-list (take (Suc n) (drop i (composition-fri-budgets budgets))) +
      Suc n
  using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
  using whole unfolding budget-eq .
qed

lemma hash-target-program-staged-query-program:
  assumes controlled: staged-adversary-controlled budgets A
    and bound: i + n ≤ length (query-opening-budgets budgets)
  shows
    hash-target-program B
      (sum-list (take n (drop i (query-opening-budgets budgets))) + n)
      (staged-query-program A i n)
  using bound
proof (induction n arbitrary: i)
  case 0
  then show ?case by (simp add: hash-target-program-return)
next
  case (Suc n)
  have i-bound: i < length (query-opening-budgets budgets)
  using Suc.prem1 by simp
  have challenge: hash-target-program B 1 receive-query-index-challenge
  by (rule hash-target-program-receive-query-index-challenge)
  have stage:

```

```

 $\wedge$ raw. hash-target-program  $B$  (query-opening-budgets budgets !  $i$ )
  (query-opening-stage  $A$   $i$  raw)
using controlled  $i$ -bound
unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-target)
have record-budget:
 $\wedge$ chunk. hash-target-program  $B$  0 (record-staged-messages chunk)
by (rule hash-target-program-record-staged-messages)
have tail:
  hash-target-program  $B$ 
    (sum-list (take  $n$  (drop (Suc  $i$ ) (query-opening-budgets budgets))) +
       $n$ )
    (staged-query-program  $A$  (Suc  $i$ )  $n$ )
by (rule Suc.IH) (use Suc.premis in simp)
have tail-return:
 $\wedge$ chunk. hash-target-program  $B$ 
  (sum-list (take  $n$  (drop (Suc  $i$ ) (query-opening-budgets budgets))) +
     $n + 0$ )
  (staged-query-program  $A$  (Suc  $i$ )  $n$   $\gg=$ 
    ( $\lambda$ chunks. return (chunk # chunks)))
by (rule hash-target-program-bind)
  (rule tail, rule hash-target-program-return)
have after-record:
 $\wedge$ chunk. hash-target-program  $B$ 
  (0 +
    (sum-list
      (take  $n$  (drop (Suc  $i$ ) (query-opening-budgets budgets))) +
       $n + 0$ ))
    (record-staged-messages chunk  $\gg=$ 
      ( $\lambda$ -. staged-query-program  $A$  (Suc  $i$ )  $n$   $\gg=$ 
        ( $\lambda$ chunks. return (chunk # chunks)))))
by (rule hash-target-program-bind)
  (rule record-budget, rule tail-return)
have after-stage:
 $\wedge$ raw. hash-target-program  $B$ 
  (query-opening-budgets budgets !  $i$  +
    (0 +
      (sum-list
        (take  $n$  (drop (Suc  $i$ ) (query-opening-budgets budgets))) +
         $n + 0$ )))
    (query-opening-stage  $A$   $i$  raw  $\gg=$ 
      ( $\lambda$ chunk. record-staged-messages chunk  $\gg=$ 
        ( $\lambda$ -. staged-query-program  $A$  (Suc  $i$ )  $n$   $\gg=$ 
          ( $\lambda$ chunks. return (chunk # chunks))))))
by (rule hash-target-program-bind)
  (rule stage, rule after-record)
have whole:
  hash-target-program  $B$ 
    (1 +

```

```

    (query-opening-budgets budgets ! i +
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0))))
    (staged-query-program A i (Suc n))
  unfolding staged-query-program.simps
  by (rule hash-target-program-bind)
    (rule challenge, rule after-stage)
have budget-eq:
  1 +
    (query-opening-budgets budgets ! i +
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0))) =
    sum-list (take (Suc n) (drop i (query-opening-budgets budgets))) +
      Suc n
  using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
  using whole unfolding budget-eq .
qed

lemma hash-target-program-checked-staged-query-program:
  assumes controlled: staged-adversary-controlled budgets A
    and bound:  $i + n \leq \text{length } (\text{query-opening-budgets budgets})$ 
  shows
    hash-target-program B
      (sum-list (take n (drop i (query-opening-budgets budgets))) + n)
      (checked-staged-query-program A trace-roots composition-roots i n)
  using bound
proof (induction n arbitrary: i)
  case 0
  then show ?case by (simp add: hash-target-program-return)
next
  case (Suc n)
  have i-bound:  $i < \text{length } (\text{query-opening-budgets budgets})$ 
  using Suc.premis by simp
  have challenge: hash-target-program B 1 receive-query-index-challenge
  by (rule hash-target-program-receive-query-index-challenge)
  have stage:
     $\bigwedge \text{raw. hash-target-program B } (\text{query-opening-budgets budgets ! } i)$ 
     $(\text{query-opening-stage A } i \text{ raw})$ 
  using controlled i-bound
  unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
  have check:
     $\bigwedge \text{raw chunk. hash-target-program B } 0$ 
    (assert

```

```

      (verifier-query-round-chunk (index (to-nat raw))
        trace-roots composition-roots chunk))
    by (rule hash-target-program-assert)
  have record-budget:
     $\bigwedge$  chunk. hash-target-program B 0 (record-staged-messages chunk)
    by (rule hash-target-program-record-staged-messages)
  have tail:
    hash-target-program B
      (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n)
      (checked-staged-query-program A trace-roots composition-roots
        (Suc i) n)
    by (rule Suc.IH) (use Suc.premis in simp)
  have tail-return:
     $\bigwedge$  chunk. hash-target-program B
      (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0)
      (checked-staged-query-program A trace-roots composition-roots
        (Suc i) n  $\gg$ 
        ( $\lambda$  chunks. return (chunk # chunks)))
    by (rule hash-target-program-bind)
      (rule tail, rule hash-target-program-return)
  have after-record:
     $\bigwedge$  chunk. hash-target-program B
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
          n + 0))
      (record-staged-messages chunk  $\gg$ 
        ( $\lambda$ -. checked-staged-query-program A trace-roots composition-roots
          (Suc i) n  $\gg$ 
            ( $\lambda$  chunks. return (chunk # chunks)))))
    by (rule hash-target-program-bind)
      (rule record-budget, rule tail-return)
  have after-check:
     $\bigwedge$  raw chunk. hash-target-program B
      (0 +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0)))
      (assert
        (verifier-query-round-chunk (index (to-nat raw))
          trace-roots composition-roots chunk)  $\gg$ 
        ( $\lambda$ -. record-staged-messages chunk  $\gg$ 
          ( $\lambda$ -. checked-staged-query-program A trace-roots
            composition-roots (Suc i) n  $\gg$ 
              ( $\lambda$  chunks. return (chunk # chunks)))))
    by (rule hash-target-program-bind)

```

```

    (rule check, rule after-record)
have after-stage:
   $\wedge$ raw. hash-target-program B
    (query-opening-budgets budgets ! i +
      (0 +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
              n + 0))))
    (query-opening-stage A i raw  $\gg$ 
      ( $\lambda$ chunk. assert
        (verifier-query-round-chunk (index (to-nat raw))
          trace-roots composition-roots chunk)  $\gg$ 
        ( $\lambda$ -. record-staged-messages chunk  $\gg$ 
          ( $\lambda$ -. checked-staged-query-program A trace-roots
            composition-roots (Suc i) n  $\gg$ 
              ( $\lambda$ chunks. return (chunk # chunks))))))
by (rule hash-target-program-bind)
    (rule stage, rule after-check)
have whole:
  hash-target-program B
    (1 +
      (query-opening-budgets budgets ! i +
        (0 +
          (0 +
            (sum-list
              (take n (drop (Suc i) (query-opening-budgets budgets))) +
                n + 0))))
      (checked-staged-query-program A trace-roots composition-roots
        i (Suc n))
    unfolding checked-staged-query-program.simps Let-def
by (rule hash-target-program-bind)
    (rule challenge, rule after-stage)
have budget-eq:
  1 +
    (query-opening-budgets budgets ! i +
      (0 +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
              n + 0)))) =
    sum-list (take (Suc n) (drop i (query-opening-budgets budgets))) +
      Suc n
    using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
    using whole unfolding budget-eq .
qed

```

lemma hash-relation-program-checked-staged-query-program:


```

assumes controlled: staged-adversary-controlled budgets A
and bound:  $i + n \leq \text{length } (\text{query-opening-budgets budgets})$ 
and fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
shows
  hash-relation-program R b
  (sum-list (take n (drop i (query-opening-budgets budgets))) + n)
  (checked-staged-query-program A trace-roots composition-roots i n)
using bound
proof (induction n arbitrary: i)
  case 0
  then show ?case
    by (simp add:
      hash-relation-program-zero[OF hash-map-preserving-return])
next
  case (Suc n)
  have i-bound:  $i < \text{length } (\text{query-opening-budgets budgets})$ 
    using Suc.prem by simp
  have challenge:
    hash-relation-program R b 1 receive-query-index-challenge
    by (rule hash-relation-program-receive-query-index-challenge[OF fibers])
  have stage:
     $\bigwedge \text{raw. } \text{hash-relation-program } R \ b$ 
    (query-opening-budgets budgets ! i)
    (query-opening-stage A i raw)
  proof –
    fix raw
    have stage-controlled:
      controlled-ro-program (query-opening-budgets budgets ! i)
      (query-opening-stage A i raw)
      using controlled i-bound
      unfolding staged-adversary-controlled-def by blast
    show hash-relation-program R b
      (query-opening-budgets budgets ! i)
      (query-opening-stage A i raw)
      by (rule controlled-ro-program-relation
        [OF stage-controlled fibers])
  qed
  have check:
     $\bigwedge \text{raw chunk. } \text{hash-relation-program } R \ b \ 0$ 
    (assert
      (verifier-query-round-chunk (index (to-nat raw)))
      (trace-roots composition-roots chunk))
    by (rule hash-relation-program-assert)
  have record-budget:
     $\bigwedge \text{chunk. } \text{hash-relation-program } R \ b \ 0$ 
    (record-staged-messages chunk)
    by (rule hash-relation-program-record-staged-messages)
  have tail:
    hash-relation-program R b

```

```

    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n)
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n)
  by (rule Suc.IH) (use Suc.premis in simp)
have tail-return:
 $\wedge$  chunk. hash-relation-program R b
  (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
    n + 0)
  (checked-staged-query-program A trace-roots composition-roots
    (Suc i) n  $\gg$ 
    ( $\lambda$  chunks. return (chunk # chunks)))
by (rule hash-relation-program-bind)
  (rule tail,
    rule hash-relation-program-zero[OF hash-map-preserving-return])
have after-record:
 $\wedge$  chunk. hash-relation-program R b
  (0 +
    (sum-list
      (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n + 0))
  (record-staged-messages chunk  $\gg$ 
    ( $\lambda$ -. checked-staged-query-program A trace-roots composition-roots
      (Suc i) n  $\gg$ 
      ( $\lambda$  chunks. return (chunk # chunks)))))
by (rule hash-relation-program-bind)
  (rule record-budget, rule tail-return)
have after-check:
 $\wedge$  raw chunk. hash-relation-program R b
  (0 +
    (0 +
      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0))))
  (assert
    (verifier-query-round-chunk (index (to-nat raw))
      trace-roots composition-roots chunk)  $\gg$ 
    ( $\lambda$ -. record-staged-messages chunk  $\gg$ 
      ( $\lambda$ -. checked-staged-query-program A trace-roots
        composition-roots (Suc i) n  $\gg$ 
        ( $\lambda$  chunks. return (chunk # chunks)))))
by (rule hash-relation-program-bind)
  (rule check, rule after-record)
have after-stage:
 $\wedge$  raw. hash-relation-program R b
  (query-opening-budgets budgets ! i +
    (0 +
      (0 +
        (sum-list

```

```

      (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n + 0))))
(query-opening-stage A i raw >>=
  (λchunk. assert
    (verifier-query-round-chunk (index (to-nat raw))
      trace-roots composition-roots chunk) >>=
    (λ-. record-staged-messages chunk >>=
      (λ-. checked-staged-query-program A trace-roots
        composition-roots (Suc i) n >>=
          (λchunks. return (chunk # chunks))))))
by (rule hash-relation-program-bind)
  (rule stage, rule after-check)
have whole:
  hash-relation-program R b
  (1 +
    (query-opening-budgets budgets ! i +
      (0 +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0))))))
  (checked-staged-query-program A trace-roots composition-roots
    i (Suc n))
unfolding checked-staged-query-program.simps Let-def
by (rule hash-relation-program-bind)
  (rule challenge, rule after-stage)
have budget-eq:
  1 +
    (query-opening-budgets budgets ! i +
      (0 +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0)))) =
    sum-list (take (Suc n) (drop i (query-opening-budgets budgets))) +
    Suc n
using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
using whole unfolding budget-eq .
qed

```

lemma hash-relation-program-staged-trace-fri-program:
assumes controlled: staged-adversary-controlled budgets A
and bound: $i + n \leq \text{length } (\text{trace-fri-budgets budgets})$
and fibers: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$
shows
 hash-relation-program R b
 (sum-list (take n (drop i (trace-fri-budgets budgets))) + n)
 (staged-trace-fri-program A i n bs)

```

using bound
proof (induction n arbitrary: i bs)
  case 0
  then show ?case
    by (simp add:
      hash-relation-program-zero[OF hash-map-preserving-return])
next
  case (Suc n)
  have i-bound: i < length (trace-fri-budgets budgets)
    using Suc.premis by simp
  have stage:
    hash-relation-program R b (trace-fri-budgets budgets ! i)
    (trace-fri-root-stage A i bs)
  proof -
    have stage-controlled:
      controlled-ro-program (trace-fri-budgets budgets ! i)
      (trace-fri-root-stage A i bs)
    using controlled i-bound
    unfolding staged-adversary-controlled-def by blast
  show ?thesis
    by (rule controlled-ro-program-relation
      [OF stage-controlled fibers])
  qed
  have record-budget:
     $\bigwedge \text{root. hash-relation-program } R \text{ } b \text{ } 0 \text{ (record-staged-message root)}$ 
    by (rule hash-relation-program-record-staged-message)
  have challenge:
    hash-relation-program R b 1 receive-trace-fri-challenge
    by (rule hash-relation-program-receive-trace-fri-challenge[OF fibers])
  have tail:
     $\bigwedge \text{root } b'. \text{ hash-relation-program } R \text{ } b$ 
    (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) + n)
    (staged-trace-fri-program A (Suc i) n (bs @ [b']))
    by (rule Suc.IH) (use Suc.premis in simp)
  have tail-return:
     $\bigwedge \text{root } b'. \text{ hash-relation-program } R \text{ } b$ 
    (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
      n + 0)
    (staged-trace-fri-program A (Suc i) n (bs @ [b']))  $\gg=$ 
    ( $\lambda(\text{roots}, \text{bs}'). \text{return (root \# roots, bs')}$ )
    by (rule hash-relation-program-bind)
    (rule tail,
      simp add: hash-relation-program-zero[OF hash-map-preserving-return]
      split: prod.splits)
  have after-challenge:
     $\bigwedge \text{root. hash-relation-program } R \text{ } b$ 
    (1 +
      (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
        n + 0))

```

```

    (receive-trace-fri-challenge  $\gg$ 
      ( $\lambda b'$ . staged-trace-fri-program  $A$  ( $Suc$   $i$ )  $n$  ( $bs$  @ [ $b'$ ])  $\gg$ 
        ( $\lambda$ (roots,  $bs'$ ). return (root # roots,  $bs'$ ))))
  by (rule hash-relation-program-bind)
    (rule challenge, rule tail-return)
have after-record:
 $\wedge$ root. hash-relation-program  $R$   $b$ 
  (0 +
    (1 +
      (sum-list (take  $n$  (drop ( $Suc$   $i$ ) (trace-fri-budgets budgets))) +
         $n$  + 0))))
  (record-staged-message root  $\gg$ 
    ( $\lambda$ -. receive-trace-fri-challenge  $\gg$ 
      ( $\lambda b'$ . staged-trace-fri-program  $A$  ( $Suc$   $i$ )  $n$  ( $bs$  @ [ $b'$ ])  $\gg$ 
        ( $\lambda$ (roots,  $bs'$ ). return (root # roots,  $bs'$ ))))
  by (rule hash-relation-program-bind)
    (rule record-budget, rule after-challenge)
have whole:
  hash-relation-program  $R$   $b$ 
    (trace-fri-budgets budgets !  $i$  +
      (0 +
        (1 +
          (sum-list (take  $n$  (drop ( $Suc$   $i$ ) (trace-fri-budgets budgets))) +
             $n$  + 0))))
      (staged-trace-fri-program  $A$   $i$  ( $Suc$   $n$ )  $bs$ )
  unfolding staged-trace-fri-program.simps
  by (rule hash-relation-program-bind)
    (rule stage, rule after-record)
have budget-eq:
  trace-fri-budgets budgets !  $i$  +
    (0 +
      (1 +
        (sum-list (take  $n$  (drop ( $Suc$   $i$ ) (trace-fri-budgets budgets))) +
           $n$  + 0))) =
    sum-list (take ( $Suc$   $n$ ) (drop  $i$  (trace-fri-budgets budgets))) +
       $Suc$   $n$ 
  using sum-list-take-Suc-drop[OF  $i$ -bound, of  $n$ ] by simp
show ?case
  using whole unfolding budget-eq .
qed

```

```

lemma hash-relation-program-staged-alpha-program:
  assumes fibers:  $\wedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows hash-relation-program  $R$   $b$   $n$  (staged-alpha-program  $n$ )
proof (induction  $n$ )
  case 0
  then show ?case
  by (simp add:
    hash-relation-program-zero[OF hash-map-preserving-return])

```

```

next
  case (Suc n)
  have challenge: hash-relation-program R b 1 receive-alpha-challenge
    by (rule hash-relation-program-receive-alpha-challenge[OF fibers])
  have record-budget:
     $\wedge a.$  hash-relation-program R b 0 (record-staged-message a)
    by (rule hash-relation-program-record-staged-message)
  have tail-return:
     $\wedge a.$  hash-relation-program R b (n + 0)
      (staged-alpha-program n  $\gg$  ( $\lambda as.$  return (a # as)))
    by (rule hash-relation-program-bind)
      (rule Suc.IH,
        rule hash-relation-program-zero[OF hash-map-preserving-return])
  have after-record:
     $\wedge a.$  hash-relation-program R b (0 + (n + 0))
      (record-staged-message a  $\gg$ 
        ( $\lambda -. staged-alpha-program n \gg (\lambda as. return (a \# as))$ )))
    by (rule hash-relation-program-bind)
      (rule record-budget, rule tail-return)
  have hash-relation-program R b (1 + (0 + (n + 0)))
    (staged-alpha-program (Suc n))
    unfolding staged-alpha-program.simps
    by (rule hash-relation-program-bind)
      (rule challenge, rule after-record)
  then show ?case by simp
qed

```

```

lemma hash-relation-program-staged-composition-fri-program:
  assumes controlled: staged-adversary-controlled budgets A
    and bound:  $i + n \leq \text{length } (\text{composition-fri-budgets budgets})$ 
    and fibers:  $\wedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
    hash-relation-program R b
      (sum-list (take n (drop i (composition-fri-budgets budgets))) + n)
      (staged-composition-fri-program A dg i n bs)
  using bound
proof (induction n arbitrary: i bs)
  case 0
  then show ?case
    by (simp add:
      hash-relation-program-zero[OF hash-map-preserving-return])
next
  case (Suc n)
  have i-bound:  $i < \text{length } (\text{composition-fri-budgets budgets})$ 
    using Suc.premis by simp
  have stage:
    hash-relation-program R b (composition-fri-budgets budgets ! i)
    (composition-fri-root-stage A dg i bs)
  proof -

```

```

have stage-controlled:
  controlled-ro-program (composition-fri-budgets budgets ! i)
    (composition-fri-root-stage A dg i bs)
  using controlled i-bound
  unfolding staged-adversary-controlled-def by blast
show ?thesis
  by (rule controlled-ro-program-relation
    [OF stage-controlled fibers])
qed
have record-budget:
   $\wedge$ root. hash-relation-program R b 0 (record-staged-message root)
  by (rule hash-relation-program-record-staged-message)
have challenge:
  hash-relation-program R b 1 receive-composition-fri-challenge
  by (rule
    hash-relation-program-receive-composition-fri-challenge[OF fibers])
have tail:
   $\wedge$ root b'. hash-relation-program R b
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) + n)
    (staged-composition-fri-program A dg (Suc i) n (bs @ [b'])))
  by (rule Suc.IH) (use Suc.premis in simp)
have tail-return:
   $\wedge$ root b'. hash-relation-program R b
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) + n + 0)
    (staged-composition-fri-program A dg (Suc i) n (bs @ [b'])  $\gg$ 
      ( $\lambda$ (roots, bs'). return (root # roots, bs'))))
  by (rule hash-relation-program-bind)
    (rule tail,
      simp add: hash-relation-program-zero[OF hash-map-preserving-return]
        split: prod.splits)
have after-challenge:
   $\wedge$ root. hash-relation-program R b
    (1 +
      (sum-list
        (take n (drop (Suc i) (composition-fri-budgets budgets))) +
          n + 0))
    (receive-composition-fri-challenge  $\gg$ 
      ( $\lambda$ b'. staged-composition-fri-program A dg (Suc i) n
        (bs @ [b'])  $\gg$ 
          ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
  by (rule hash-relation-program-bind)
    (rule challenge, rule tail-return)
have after-record:
   $\wedge$ root. hash-relation-program R b
    (0 +
      (1 +
        (sum-list

```

```

      (take n (drop (Suc i) (composition-fri-budgets budgets))) +
      n + 0)))
  (record-staged-message root  $\gg$ 
    ( $\lambda$ -. receive-composition-fri-challenge  $\gg$ 
      ( $\lambda$ b'. staged-composition-fri-program A dg (Suc i) n
        (bs @ [b])  $\gg$ 
          ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
  by (rule hash-relation-program-bind)
    (rule record-budget, rule after-challenge)
have whole:
  hash-relation-program R b
    (composition-fri-budgets budgets ! i +
      (0 +
        (1 +
          (sum-list
            (take n (drop (Suc i) (composition-fri-budgets budgets))) +
            n + 0))))
      (staged-composition-fri-program A dg i (Suc n) bs)
unfolding staged-composition-fri-program.simps
by (rule hash-relation-program-bind)
    (rule stage, rule after-record)
have budget-eq:
  composition-fri-budgets budgets ! i +
    (0 +
      (1 +
        (sum-list
          (take n (drop (Suc i) (composition-fri-budgets budgets))) +
          n + 0))) =
    sum-list (take (Suc n) (drop i (composition-fri-budgets budgets))) +
    Suc n
using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
using whole unfolding budget-eq .
qed

```

lemma hash-relation-program-guarded-staged-composition-fri-program:

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and fibers: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$

shows

hash-relation-program R b
 (sum-list (composition-fri-budgets budgets) +
 ceil-log (maxDegree + 1))
 (assert (ceil-log (to-nat dg + 1) $\leq$ ceil-log (maxDegree + 1)) $\gg$
 (λ -. staged-composition-fri-program A dg 0
 (ceil-log (to-nat dg + 1)) []))

proof —

let ?B =

sum-list (composition-fri-budgets budgets) +


```

    ceil-log (maxDegree + 1)
have hash-relation-program R b (0 + ?B)
  (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
    (λ-. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) []))
proof (rule hash-relation-program-bind-on-outcomes
  [where q=0 and r=?B])
show hash-relation-program R b 0
  (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)))
  by (rule hash-relation-program-assert)
next
fix s x t
assume out:
  Some (x, t) ∈
    set-dist
    (execute
      (assert
        (ceil-log (to-nat dg + 1) ≤
          ceil-log (maxDegree + 1))) s)
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
using out unfolding assert-def
by (cases ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
  (auto simp: throw-no-outcome)
have len:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
using round-bound wf unfolding staged-budget-wellformed-def by simp
have exact:
  hash-relation-program R b
  (sum-list
    (take (ceil-log (to-nat dg + 1))
      (drop 0 (composition-fri-budgets budgets))) +
    ceil-log (to-nat dg + 1))
  (staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) [])
by (rule hash-relation-program-staged-composition-fri-program
  [OF controlled len fibers])
have le:
  sum-list
    (take (ceil-log (to-nat dg + 1))
      (drop 0 (composition-fri-budgets budgets))) +
    ceil-log (to-nat dg + 1) ≤
    sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
using round-bound sum-list-take-le[of ceil-log (to-nat dg + 1)
  composition-fri-budgets budgets]
by simp
show hash-relation-program R b ?B

```

```

    (staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) [])
  by (rule hash-relation-program-mono[OF le exact])
qed
then show ?thesis by simp
qed

lemma hash-relation-program-staged-alpha-prefix-program:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
    hash-relation-program R b
      (staged-alpha-search-queries budgets 0)
      (staged-alpha-prefix-program A)
proof -
  let ?trace-root = trace-root-budget budgets
  let ?trace-fri =
    sum-list (trace-fri-budgets budgets) + ceil-log clength
  let ?trace-final = trace-final-budget budgets
  have trace-root:
    hash-relation-program R b ?trace-root (trace-root-stage A)
proof -
  have controlled-root:
    controlled-ro-program ?trace-root (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def by blast
  show ?thesis
  by (rule controlled-ro-program-relation
      [OF controlled-root fibers])
qed
have record-root:
   $\bigwedge fr. \text{hash-relation-program } R \ b \ 0 \ (\text{record-staged-message } fr)$ 
  by (rule hash-relation-program-record-staged-message)
have trace-fri:
  hash-relation-program R b ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
proof -
  have len:
     $0 + \text{ceil-log clength} \leq \text{length } (\text{trace-fri-budgets budgets})$ 
  using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-relation-program R b
      (sum-list
        (take (ceil-log clength) (drop 0 (trace-fri-budgets budgets))) +
        ceil-log clength)
      (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule hash-relation-program-staged-trace-fri-program
      [OF controlled len fibers])
  have le:

```

```

    sum-list
      (take (ceil-log clength) (drop 0 (trace-fri-budgets budgets))) +
      ceil-log clength ≤
    sum-list (trace-fri-budgets budgets) + ceil-log clength
  using sum-list-take-le[of ceil-log clength
    trace-fri-budgets budgets]
  by simp
show ?thesis
  by (rule hash-relation-program-mono[OF le exact])
qed
have trace-final:
  ∧ trace-roots trace-bs. hash-relation-program R b ?trace-final
  (trace-final-stage A trace-bs)
proof -
  fix trace-roots trace-bs
  have controlled-final:
    controlled-ro-program ?trace-final
    (trace-final-stage A trace-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
show hash-relation-program R b ?trace-final
  (trace-final-stage A trace-bs)
  by (rule controlled-ro-program-relation
    [OF controlled-final fibers])
qed
have record-final:
  ∧ trace-final. hash-relation-program R b 0
  (record-staged-message trace-final)
  by (rule hash-relation-program-record-staged-message)
have after-final-record:
  ∧ fr trace-roots trace-bs trace-final.
  hash-relation-program R b 0
  (record-staged-message trace-final >=>
    (λ-. return (fr, trace-roots, trace-bs, trace-final)))
proof -
  fix fr trace-roots trace-bs trace-final
  have hash-relation-program R b (0 + 0)
    (record-staged-message trace-final >=>
      (λ-. return (fr, trace-roots, trace-bs, trace-final)))
  by (rule hash-relation-program-bind)
  (rule record-final,
    rule hash-relation-program-zero[OF hash-map-preserving-return])
  then show hash-relation-program R b 0
    (record-staged-message trace-final >=>
      (λ-. return (fr, trace-roots, trace-bs, trace-final)))
  by simp
qed
have after-final:
  ∧ fr trace-roots trace-bs. hash-relation-program R b
  (?trace-final + 0)

```

```

    (trace-final-stage A trace-bs >=>
      (λtrace-final. record-staged-message trace-final >=>
        (λ-. return (fr, trace-roots, trace-bs, trace-final))))
  by (rule hash-relation-program-bind)
    (rule trace-final, rule after-final-record)
have after-trace-fri:
  ∧fr. hash-relation-program R b
    (?trace-fri + (?trace-final + 0))
    (staged-trace-fri-program A 0 (ceil-log clength) [] >=>
      (λ(trace-roots, trace-bs).
        trace-final-stage A trace-bs >=>
          (λtrace-final. record-staged-message trace-final >=>
            (λ-. return (fr, trace-roots, trace-bs, trace-final))))))
proof -
  fix fr
  have cont:
    ∧x. hash-relation-program R b (?trace-final + 0)
      (case x of (trace-roots, trace-bs) =>
        trace-final-stage A trace-bs >=>
          (λtrace-final. record-staged-message trace-final >=>
            (λ-. return (fr, trace-roots, trace-bs, trace-final))))
  proof -
    fix x
    show hash-relation-program R b (?trace-final + 0)
      (case x of (trace-roots, trace-bs) =>
        trace-final-stage A trace-bs >=>
          (λtrace-final. record-staged-message trace-final >=>
            (λ-. return (fr, trace-roots, trace-bs, trace-final))))
  proof (cases x)
    case (Pair trace-roots trace-bs)
    have step:
      hash-relation-program R b (?trace-final + 0)
        (trace-final-stage A trace-bs >=>
          (λtrace-final. record-staged-message trace-final >=>
            (λ-. return (fr, trace-roots, trace-bs, trace-final))))
    by (rule after-final)
    show ?thesis
      unfolding Pair using step by simp
  qed
qed
have hash-relation-program R b (?trace-fri + (?trace-final + 0))
  (staged-trace-fri-program A 0 (ceil-log clength) [] >=>
    (λ(trace-roots, trace-bs).
      trace-final-stage A trace-bs >=>
        (λtrace-final. record-staged-message trace-final >=>
          (λ-. return (fr, trace-roots, trace-bs, trace-final))))))
  by (rule hash-relation-program-bind)
    (rule trace-fri, rule cont)
then show hash-relation-program R b

```

```

    (?trace-fri + (?trace-final + 0))
    (staged-trace-fri-program A 0 (ceil-log clength) [] >>=
      (λ(trace-roots, trace-bs).
        trace-final-stage A trace-bs >>=
          (λtrace-final. record-staged-message trace-final >>=
            (λ-. return (fr, trace-roots, trace-bs, trace-final)))))) .
qed
have after-root-record:
  ∧fr. hash-relation-program R b
    (0 + (?trace-fri + (?trace-final + 0)))
    (record-staged-message fr >>=
      (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
        (λ(trace-roots, trace-bs).
          trace-final-stage A trace-bs >>=
            (λtrace-final. record-staged-message trace-final >>=
              (λ-. return (fr, trace-roots, trace-bs, trace-final))))))
  by (rule hash-relation-program-bind)
    (rule record-root, rule after-trace-fri)
have whole:
  hash-relation-program R b
    (?trace-root + (0 + (?trace-fri + (?trace-final + 0))))
    (staged-alpha-prefix-program A)
  unfolding staged-alpha-prefix-program-def
  by (rule hash-relation-program-bind)
    (rule trace-root, rule after-root-record)
show ?thesis
  using whole
  unfolding staged-alpha-search-queries-def
  by (simp add: add.assoc add.commute add.left-commute)
qed

lemma hash-range-budget-staged-alpha-prefix-program:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    hash-range-budget
      (staged-alpha-search-queries budgets 0)
      (staged-alpha-prefix-program A)
proof -
  let ?trace-root = trace-root-budget budgets
  let ?trace-fri =
    sum-list (trace-fri-budgets budgets) + ceil-log clength
  let ?trace-final = trace-final-budget budgets
  have trace-root:
    hash-range-budget ?trace-root (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
  have trace-fri:
    hash-range-budget ?trace-fri

```

```

    (staged-trace-fri-program A 0 (ceil-log clength) [])
  proof -
    have len:
      0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
      using wf unfolding staged-budget-wellformed-def by simp
    have exact:
      hash-range-budget
        (sum-list
          (take (ceil-log clength)
            (drop 0 (trace-fri-budgets budgets))) +
          ceil-log clength)
        (staged-trace-fri-program A 0 (ceil-log clength) [])
      by (rule hash-range-budget-staged-trace-fri-program
        [OF controlled len])
    then show ?thesis
      using wf unfolding staged-budget-wellformed-def by simp
  qed
  have trace-final:
    ∧bs. hash-range-budget ?trace-final (trace-final-stage A bs)
    using controlled unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-range)
  have after-trace-final-record:
    ∧fr trace-roots trace-bs trace-final.
    hash-range-budget 0
      (record-staged-message trace-final ≫=
        (λ-. return (fr, trace-roots, trace-bs, trace-final)))
  proof -
    fix fr trace-roots trace-bs trace-final
    have hash-range-budget (0 + 0)
      (record-staged-message trace-final ≫=
        (λ-. return (fr, trace-roots, trace-bs, trace-final)))
    by (rule hash-range-budget-bind)
      (rule hash-range-budget-record-staged-message,
        rule hash-range-budget-return)
    then show
      hash-range-budget 0
        (record-staged-message trace-final ≫=
          (λ-. return (fr, trace-roots, trace-bs, trace-final)))
      by simp
  qed
  have after-trace-final:
    ∧fr trace-roots trace-bs.
    hash-range-budget (?trace-final + 0)
      (trace-final-stage A trace-bs ≫=
        (λtrace-final.
          record-staged-message trace-final ≫=
            (λ-. return (fr, trace-roots, trace-bs, trace-final))))
    by (rule hash-range-budget-bind)
      (rule trace-final, rule after-trace-final-record)

```

```

have after-trace-fri:
   $\wedge$ fr. hash-range-budget (?trace-fri + (?trace-final + 0))
    (staged-trace-fri-program A 0 (ceil-log clength) []  $\gg$ 
      ( $\lambda$ (trace-roots, trace-bs).
        trace-final-stage A trace-bs  $\gg$ 
          ( $\lambda$ trace-final.
            record-staged-message trace-final  $\gg$ 
              ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))))
proof (rule hash-range-budget-bind)
  fix fr
  show hash-range-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
    by (rule trace-fri)
next
  fix fr x
  show hash-range-budget (?trace-final + 0)
    (case x of (trace-roots, trace-bs)  $\Rightarrow$ 
      trace-final-stage A trace-bs  $\gg$ 
        ( $\lambda$ trace-final.
          record-staged-message trace-final  $\gg$ 
            ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
  proof (cases x)
  case (Pair trace-roots trace-bs)
  have step:
    hash-range-budget (?trace-final + 0)
      (trace-final-stage A trace-bs  $\gg$ 
        ( $\lambda$ trace-final.
          record-staged-message trace-final  $\gg$ 
            ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
    by (rule after-trace-final)
  show ?thesis
    unfolding Pair using step by simp
  qed
qed
have after-trace-root-record:
   $\wedge$ fr. hash-range-budget (0 + (?trace-fri + (?trace-final + 0)))
    (record-staged-message fr  $\gg$ 
      ( $\lambda$ -. staged-trace-fri-program A 0 (ceil-log clength) []  $\gg$ 
        ( $\lambda$ (trace-roots, trace-bs).
          trace-final-stage A trace-bs  $\gg$ 
            ( $\lambda$ trace-final.
              record-staged-message trace-final  $\gg$ 
                ( $\lambda$ -. return
                  (fr, trace-roots, trace-bs, trace-final))))))
    by (rule hash-range-budget-bind)
      (rule hash-range-budget-record-staged-message, rule after-trace-fri)
have whole:
  hash-range-budget
    (?trace-root + (0 + (?trace-fri + (?trace-final + 0))))

```

```

    (staged-alpha-prefix-program A)
  unfolding staged-alpha-prefix-program-def
  by (rule hash-range-budget-bind)
    (rule trace-root, rule after-trace-root-record)
show ?thesis
  using whole
  unfolding staged-alpha-search-queries-def
  by (simp add: add.assoc add.commute add.left-commute)
qed

lemma hash-collision-budget-staged-alpha-prefix-program:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    hash-collision-budget
      (staged-alpha-search-queries budgets 0)
      (staged-alpha-prefix-program A)
proof -
  let ?trace-root = trace-root-budget budgets
  let ?trace-fri =
    sum-list (trace-fri-budgets budgets) + ceil-log clength
  let ?trace-final = trace-final-budget budgets
  have trace-root-range:
    hash-range-budget ?trace-root (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
  have trace-root-coll:
    hash-collision-budget ?trace-root (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-collision)
  have trace-fri-range':
    hash-range-budget ?trace-fri
      (staged-trace-fri-program A 0 (ceil-log clength) [])
proof -
  have len:
    0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
  using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-range-budget
      (sum-list
        (take (ceil-log clength)
          (drop 0 (trace-fri-budgets budgets)))) +
      ceil-log clength
      (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule hash-range-budget-staged-trace-fri-program
    [OF controlled len])
  then show ?thesis
    using wf unfolding staged-budget-wellformed-def by simp
qed

```



```

have trace-fri-coll:
  hash-collision-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
proof –
  have len:
    0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-collision-budget
      (sum-list
        (take (ceil-log clength)
          (drop 0 (trace-fri-budgets budgets))) +
        ceil-log clength)
      (staged-trace-fri-program A 0 (ceil-log clength) [])
    by (rule hash-collision-budget-staged-trace-fri-program
      [OF controlled len])
  then show ?thesis
    using wf unfolding staged-budget-wellformed-def by simp
qed
have trace-final-range:
  ∧bs. hash-range-budget ?trace-final (trace-final-stage A bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
have trace-final-coll:
  ∧bs. hash-collision-budget ?trace-final (trace-final-stage A bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-collision)
have after-trace-final-record-range:
  ∧fr trace-roots trace-bs trace-final.
  hash-range-budget 0
    (record-staged-message trace-final ≫=
      (λ-. return (fr, trace-roots, trace-bs, trace-final)))
proof –
  fix fr trace-roots trace-bs trace-final
  have hash-range-budget (0 + 0)
    (record-staged-message trace-final ≫=
      (λ-. return (fr, trace-roots, trace-bs, trace-final)))
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-record-staged-message,
      rule hash-range-budget-return)
  then show hash-range-budget 0
    (record-staged-message trace-final ≫=
      (λ-. return (fr, trace-roots, trace-bs, trace-final)))
  by simp
qed
have after-trace-final-record-coll:
  ∧fr trace-roots trace-bs trace-final.
  hash-collision-budget 0
    (record-staged-message trace-final ≫=

```

```

      (λ-. return (fr, trace-roots, trace-bs, trace-final)))
proof -
  fix fr trace-roots trace-bs trace-final
  have hash-collision-budget (0 + 0)
    (record-staged-message trace-final >>=
      (λ-. return (fr, trace-roots, trace-bs, trace-final)))
  by (rule hash-collision-budget-bind)
    (rule hash-range-budget-record-staged-message,
      rule hash-collision-budget-record-staged-message,
      rule hash-range-budget-return, rule hash-collision-budget-return)
  then show hash-collision-budget 0
    (record-staged-message trace-final >>=
      (λ-. return (fr, trace-roots, trace-bs, trace-final)))
  by simp
qed
have after-trace-final-range:
  ∧fr trace-roots trace-bs.
  hash-range-budget (?trace-final + 0)
    (trace-final-stage A trace-bs >>=
      (λtrace-final.
        record-staged-message trace-final >>=
          (λ-. return (fr, trace-roots, trace-bs, trace-final))))
  by (rule hash-range-budget-bind)
    (rule trace-final-range, rule after-trace-final-record-range)
have after-trace-final-coll:
  ∧fr trace-roots trace-bs.
  hash-collision-budget (?trace-final + 0)
    (trace-final-stage A trace-bs >>=
      (λtrace-final.
        record-staged-message trace-final >>=
          (λ-. return (fr, trace-roots, trace-bs, trace-final))))
  by (rule hash-collision-budget-bind)
    (rule trace-final-range, rule trace-final-coll,
      rule after-trace-final-record-range,
      rule after-trace-final-record-coll)
have after-trace-fri-range:
  ∧fr. hash-range-budget (?trace-fri + (?trace-final + 0))
    (staged-trace-fri-program A 0 (ceil-log clength) [] >>=
      (λ(trace-roots, trace-bs).
        trace-final-stage A trace-bs >>=
          (λtrace-final.
            record-staged-message trace-final >>=
              (λ-. return (fr, trace-roots, trace-bs, trace-final))))))
proof (rule hash-range-budget-bind)
  fix fr
  show hash-range-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule trace-fri-range)
next

```

```

fix fr x
show hash-range-budget (?trace-final + 0)
  (case x of (trace-roots, trace-bs) ⇒
    trace-final-stage A trace-bs ≫=
    (λtrace-final.
      record-staged-message trace-final ≫=
      (λ-. return (fr, trace-roots, trace-bs, trace-final))))
proof (cases x)
  case (Pair trace-roots trace-bs)
  have step:
    hash-range-budget (?trace-final + 0)
    (trace-final-stage A trace-bs ≫=
    (λtrace-final.
      record-staged-message trace-final ≫=
      (λ-. return (fr, trace-roots, trace-bs, trace-final))))
  by (rule after-trace-final-range)
show ?thesis
  unfolding Pair using step by simp
qed
qed
have after-trace-fri-coll:
  ∧fr. hash-collision-budget (?trace-fri + (?trace-final + 0))
  (staged-trace-fri-program A 0 (ceil-log clength) []) ≫=
  (λ(trace-roots, trace-bs).
    trace-final-stage A trace-bs ≫=
    (λtrace-final.
      record-staged-message trace-final ≫=
      (λ-. return (fr, trace-roots, trace-bs, trace-final))))
proof (rule hash-collision-budget-bind)
  fix fr
  show hash-range-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule trace-fri-range')
  show hash-collision-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule trace-fri-coll)
next
  fix fr x
  show hash-range-budget (?trace-final + 0)
    (case x of (trace-roots, trace-bs) ⇒
      trace-final-stage A trace-bs ≫=
      (λtrace-final.
        record-staged-message trace-final ≫=
        (λ-. return (fr, trace-roots, trace-bs, trace-final))))
  proof (cases x)
  case (Pair trace-roots trace-bs)
  have step:
    hash-range-budget (?trace-final + 0)
    (trace-final-stage A trace-bs ≫=

```

```

      (λtrace-final.
        record-staged-message trace-final >>=
        (λ-. return (fr, trace-roots, trace-bs, trace-final))))
    by (rule after-trace-final-range)
  show ?thesis
    unfolding Pair using step by simp
qed
show hash-collision-budget (?trace-final + 0)
  (case x of (trace-roots, trace-bs) ⇒
    trace-final-stage A trace-bs >>=
    (λtrace-final.
      record-staged-message trace-final >>=
      (λ-. return (fr, trace-roots, trace-bs, trace-final))))
proof (cases x)
case (Pair trace-roots trace-bs)
have step:
  hash-collision-budget (?trace-final + 0)
  (trace-final-stage A trace-bs >>=
  (λtrace-final.
    record-staged-message trace-final >>=
    (λ-. return (fr, trace-roots, trace-bs, trace-final))))
  by (rule after-trace-final-coll)
show ?thesis
  unfolding Pair using step by simp
qed
qed
have after-trace-root-record-range:
  ∧fr. hash-range-budget (0 + (?trace-fri + (?trace-final + 0)))
  (record-staged-message fr >>=
  (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
  (λ(trace-roots, trace-bs).
    trace-final-stage A trace-bs >>=
    (λtrace-final.
      record-staged-message trace-final >>=
      (λ-. return
        (fr, trace-roots, trace-bs, trace-final)))))))
  by (rule hash-range-budget-bind)
  (rule hash-range-budget-record-staged-message,
  rule after-trace-fri-range)
have after-trace-root-record-coll:
  ∧fr. hash-collision-budget
  (0 + (?trace-fri + (?trace-final + 0)))
  (record-staged-message fr >>=
  (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
  (λ(trace-roots, trace-bs).
    trace-final-stage A trace-bs >>=
    (λtrace-final.
      record-staged-message trace-final >>=
      (λ-. return

```

```

      (fr, trace-roots, trace-bs, trace-final))))))
by (rule hash-collision-budget-bind)
  (rule hash-range-budget-record-staged-message,
   rule hash-collision-budget-record-staged-message,
   rule after-trace-fri-range, rule after-trace-fri-coll)
have whole:
  hash-collision-budget
    (?trace-root + (0 + (?trace-fri + (?trace-final + 0))))
    (staged-alpha-prefix-program A)
  unfolding staged-alpha-prefix-program-def
by (rule hash-collision-budget-bind)
  (rule trace-root-range, rule trace-root-coll,
   rule after-trace-root-record-range,
   rule after-trace-root-record-coll)
show ?thesis
  using whole
  unfolding staged-alpha-search-queries-def
  by (simp add: add.assoc add.commute add.left-commute)
qed

lemma hash-range-budget-checked-staged-query-program:
  assumes controlled: staged-adversary-controlled budgets A
  and bound:  $i + n \leq \text{length } (\text{query-opening-budgets } \text{budgets})$ 
  shows
    hash-range-budget
      (sum-list (take n (drop i (query-opening-budgets budgets))) + n)
      (checked-staged-query-program A trace-roots composition-roots i n)
  using bound
proof (induction n arbitrary: i)
  case 0
  then show ?case by (simp add: hash-range-budget-return)
next
  case (Suc n)
  have i-bound:  $i < \text{length } (\text{query-opening-budgets } \text{budgets})$ 
  using Suc.prem1 by simp
  have challenge: hash-range-budget 1 receive-query-index-challenge
  by (rule hash-range-budget-receive-query-index-challenge)
  have stage:
     $\bigwedge \text{raw. hash-range-budget } (\text{query-opening-budgets } \text{budgets } ! i)$ 
    (query-opening-stage A i raw)
  using controlled i-bound
  unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
  have check:
     $\bigwedge \text{raw chunk. hash-range-budget } 0$ 
    (assert
      (verifier-query-round-chunk (index (to-nat raw))
        trace-roots composition-roots chunk))
  by (rule hash-range-budget-assert)

```

```

have record-budget:
   $\wedge$  chunk. hash-range-budget 0 (record-staged-messages chunk)
  by (rule hash-range-budget-record-staged-messages)
have tail:
  hash-range-budget
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n)
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n)
  by (rule Suc.IH) (use Suc.prem in simp)
have tail-return:
   $\wedge$  chunk. hash-range-budget
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n + 0)
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n  $\gg$ 
      ( $\lambda$  chunks. return (chunk # chunks)))
  by (rule hash-range-budget-bind)
    (rule tail, rule hash-range-budget-return)
have after-record:
   $\wedge$  chunk. hash-range-budget
    (0 +
      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0))
    (record-staged-messages chunk  $\gg$ 
      ( $\lambda$ -. checked-staged-query-program A trace-roots composition-roots
        (Suc i) n  $\gg$ 
        ( $\lambda$  chunks. return (chunk # chunks)))))
  by (rule hash-range-budget-bind)
    (rule record-budget, rule tail-return)
have after-check:
   $\wedge$  raw chunk. hash-range-budget
    (0 +
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
          n + 0)))
    (assert
      (verifier-query-round-chunk (index (to-nat raw))
        trace-roots composition-roots chunk)  $\gg$ 
      ( $\lambda$ -. record-staged-messages chunk  $\gg$ 
        ( $\lambda$ -. checked-staged-query-program A trace-roots
          composition-roots (Suc i) n  $\gg$ 
          ( $\lambda$  chunks. return (chunk # chunks)))))
  by (rule hash-range-budget-bind)
    (rule check, rule after-record)
have after-stage:
   $\wedge$  raw. hash-range-budget

```

```

(query-opening-budgets budgets ! i +
  (0 +
    (0 +
      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0))))
(query-opening-stage A i raw >>=
  (λchunk. assert
    (verifier-query-round-chunk (index (to-nat raw))
      trace-roots composition-roots chunk) >>=
    (λ-. record-staged-messages chunk >>=
      (λ-. checked-staged-query-program A trace-roots
        composition-roots (Suc i) n >>=
        (λchunks. return (chunk # chunks))))))
by (rule hash-range-budget-bind)
  (rule stage, rule after-check)
have whole:
  hash-range-budget
  (1 +
    (query-opening-budgets budgets ! i +
      (0 +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0))))))
  (checked-staged-query-program A trace-roots composition-roots
    i (Suc n))
unfolding checked-staged-query-program.simps Let-def
by (rule hash-range-budget-bind)
  (rule challenge, rule after-stage)
have budget-eq:
  1 +
    (query-opening-budgets budgets ! i +
      (0 +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0)))) =
    sum-list (take (Suc n) (drop i (query-opening-budgets budgets))) +
    Suc n
using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
using whole unfolding budget-eq .
qed

```

lemma hash-collision-budget-checked-staged-query-program:
assumes controlled: staged-adversary-controlled budgets A
and bound: $i + n \leq \text{length } (\text{query-opening-budgets budgets})$
shows

```

    hash-collision-budget
      (sum-list (take n (drop i (query-opening-budgets budgets))) + n)
      (checked-staged-query-program A trace-roots composition-roots i n)
  using bound
proof (induction n arbitrary: i)
  case 0
  then show ?case by (simp add: hash-collision-budget-return)
next
  case (Suc n)
  have i-bound: i < length (query-opening-budgets budgets)
    using Suc.prem by simp
  have challenge-range: hash-range-budget 1 receive-query-index-challenge
    by (rule hash-range-budget-receive-query-index-challenge)
  have challenge-coll: hash-collision-budget 1 receive-query-index-challenge
    by (rule hash-collision-budget-receive-query-index-challenge)
  have stage-range:
     $\wedge$ raw. hash-range-budget (query-opening-budgets budgets ! i)
      (query-opening-stage A i raw)
    using controlled i-bound
    unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-range)
  have stage-coll:
     $\wedge$ raw. hash-collision-budget (query-opening-budgets budgets ! i)
      (query-opening-stage A i raw)
    using controlled i-bound
    unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-collision)
  have check-range:
     $\wedge$ raw chunk. hash-range-budget 0
      (assert
        (verifier-query-round-chunk (index (to-nat raw))
          trace-roots composition-roots chunk))
    by (rule hash-range-budget-assert)
  have check-coll:
     $\wedge$ raw chunk. hash-collision-budget 0
      (assert
        (verifier-query-round-chunk (index (to-nat raw))
          trace-roots composition-roots chunk))
    by (rule hash-collision-budget-assert)
  have record-range:
     $\wedge$ chunk. hash-range-budget 0 (record-staged-messages chunk)
    by (rule hash-range-budget-record-staged-messages)
  have record-coll:
     $\wedge$ chunk. hash-collision-budget 0 (record-staged-messages chunk)
    by (rule hash-collision-budget-record-staged-messages)
  have tail-range:
    hash-range-budget
      (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n)

```



```

    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n)
  by (rule hash-range-budget-checked-staged-query-program[OF controlled])
    (use Suc.premis in simp)
have tail-coll:
  hash-collision-budget
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n)
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n)
  by (rule Suc.IH) (use Suc.premis in simp)
have tail-return-range:
   $\bigwedge$ chunk. hash-range-budget
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n + 0)
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n  $\gg=$ 
      ( $\lambda$ chunks. return (chunk # chunks)))
  by (rule hash-range-budget-bind)
    (rule tail-range, rule hash-range-budget-return)
have tail-return-coll:
   $\bigwedge$ chunk. hash-collision-budget
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n + 0)
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n  $\gg=$ 
      ( $\lambda$ chunks. return (chunk # chunks)))
  by (rule hash-collision-budget-bind)
    (rule tail-range, rule tail-coll, rule hash-range-budget-return,
      rule hash-collision-budget-return)
have after-record-range:
   $\bigwedge$ chunk. hash-range-budget
    (0 +
      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0))
    (record-staged-messages chunk  $\gg=$ 
      ( $\lambda$ -. checked-staged-query-program A trace-roots composition-roots
        (Suc i) n  $\gg=$ 
        ( $\lambda$ chunks. return (chunk # chunks)))))
  by (rule hash-range-budget-bind)
    (rule record-range, rule tail-return-range)
have after-record-coll:
   $\bigwedge$ chunk. hash-collision-budget
    (0 +
      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0))
    (record-staged-messages chunk  $\gg=$ 

```

```

    (λ-. checked-staged-query-program A trace-roots composition-roots
      (Suc i) n >>=
        (λchunks. return (chunk # chunks))))
  by (rule hash-collision-budget-bind)
    (rule record-range, rule record-coll, rule tail-return-range,
      rule tail-return-coll)
have after-check-range:
  ∧raw chunk. hash-range-budget
    (0 +
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0)))
    (assert
      (verifier-query-round-chunk (index (to-nat raw))
        trace-roots composition-roots chunk) >>=
        (λ-. record-staged-messages chunk >>=
          (λ-. checked-staged-query-program A trace-roots
            composition-roots (Suc i) n >>=
              (λchunks. return (chunk # chunks))))))
  by (rule hash-range-budget-bind)
    (rule check-range, rule after-record-range)
have after-check-coll:
  ∧raw chunk. hash-collision-budget
    (0 +
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0)))
    (assert
      (verifier-query-round-chunk (index (to-nat raw))
        trace-roots composition-roots chunk) >>=
        (λ-. record-staged-messages chunk >>=
          (λ-. checked-staged-query-program A trace-roots
            composition-roots (Suc i) n >>=
              (λchunks. return (chunk # chunks))))))
  by (rule hash-collision-budget-bind)
    (rule check-range, rule check-coll, rule after-record-range,
      rule after-record-coll)
have after-stage-range:
  ∧raw. hash-range-budget
    (query-opening-budgets budgets ! i +
      (0 +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
              n + 0))))
    (query-opening-stage A i raw >>=
      (λchunk. assert

```

```

    (verifier-query-round-chunk (index (to-nat raw))
      trace-roots composition-roots chunk) >>=
    (λ-. record-staged-messages chunk >>=
      (λ-. checked-staged-query-program A trace-roots
        composition-roots (Suc i) n >>=
          (λchunks. return (chunk # chunks))))))
  by (rule hash-range-budget-bind)
    (rule stage-range, rule after-check-range)
have after-stage-coll:
  ∧ raw. hash-collision-budget
    (query-opening-budgets budgets ! i +
      (0 +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
              n + 0))))))
    (query-opening-stage A i raw >>=
      (λchunk. assert
        (verifier-query-round-chunk (index (to-nat raw))
          trace-roots composition-roots chunk) >>=
        (λ-. record-staged-messages chunk >>=
          (λ-. checked-staged-query-program A trace-roots
            composition-roots (Suc i) n >>=
              (λchunks. return (chunk # chunks))))))
    by (rule hash-collision-budget-bind)
      (rule stage-range, rule stage-coll, rule after-check-range,
        rule after-check-coll)
have whole:
  hash-collision-budget
    (1 +
      (query-opening-budgets budgets ! i +
        (0 +
          (0 +
            (sum-list
              (take n (drop (Suc i) (query-opening-budgets budgets))) +
                n + 0))))))
    (checked-staged-query-program A trace-roots composition-roots
      i (Suc n))
unfolding checked-staged-query-program.simps Let-def
by (rule hash-collision-budget-bind)
    (rule challenge-range, rule challenge-coll, rule after-stage-range,
      rule after-stage-coll)
have budget-eq:
  1 +
    (query-opening-budgets budgets ! i +
      (0 +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +

```

```

      n + 0)))) =
    sum-list (take (Suc n) (drop i (query-opening-budgets budgets))) +
      Suc n
  using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
  using whole unfolding budget-eq .
qed

lemma hash-range-budget-guarded-staged-composition-fri-program:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    hash-range-budget
      (sum-list (composition-fri-budgets budgets) +
        ceil-log (maxDegree + 1))
      (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) ≫=
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []))
  proof -
    let ?B =
      sum-list (composition-fri-budgets budgets) +
        ceil-log (maxDegree + 1)
    have hash-range-budget (0 + ?B)
      (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) ≫=
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []))
    proof (rule hash-range-budget-bind-on-outcomes
      [where n=0 and n'=?B])
      show hash-range-budget 0
        (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)))
      unfolding hash-range-budget-def assert-def
      by (auto simp: throw-no-outcome)
    next
      fix s x t
      assume out:
        Some (x, t) ∈
          set-dist
            (execute
              (assert
                (ceil-log (to-nat dg + 1) ≤
                  ceil-log (maxDegree + 1)))) s)
      have round-bound:
        ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
      using out unfolding assert-def
      by (cases ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
        (auto simp: throw-no-outcome)
      have len:
        0 + ceil-log (to-nat dg + 1) ≤
          length (composition-fri-budgets budgets)

```

```

using round-bound wf unfolding staged-budget-wellformed-def by simp
have exact:
  hash-range-budget
    (sum-list
      (take (ceil-log (to-nat dg + 1))
        (drop 0 (composition-fri-budgets budgets))) +
      ceil-log (to-nat dg + 1))
    (staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) [])
  by (rule hash-range-budget-staged-composition-fri-program
    [OF controlled len])
have le:
  sum-list
    (take (ceil-log (to-nat dg + 1))
      (drop 0 (composition-fri-budgets budgets))) +
    ceil-log (to-nat dg + 1) ≤
  sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
  using round-bound sum-list-take-le[of ceil-log (to-nat dg + 1)
    composition-fri-budgets budgets]
  by simp
show hash-range-budget
  (sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1))
  (staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) [])
  by (rule hash-range-budget-mono[OF le exact])
qed
then show ?thesis by simp
qed

```

```

lemma hash-collision-budget-guarded-staged-composition-fri-program:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  hash-collision-budget
    (sum-list (composition-fri-budgets budgets) +
      ceil-log (maxDegree + 1))
    (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))  $\gg$ 
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
proof -
let ?B =
  sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
have hash-collision-budget (0 + ?B)
  (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))  $\gg$ 
    (λ-. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) []))

```

```

proof (rule hash-collision-budget-bind-on-outcomes
  [where  $n=0$  and  $n'=?B$ ])
show hash-range-budget 0
  (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)))
  unfolding hash-range-budget-def assert-def
  by (auto simp: throw-no-outcome)
show hash-collision-budget 0
  (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)))
  unfolding hash-collision-budget-def hash-new-collision-event-def
    hash-map-new-output-collision-def hash-collision-budget-value-def
    assert-def
  by (simp add: wp-event-def upsimps)
next
fix s x t
assume out:
  Some (x, t) ∈
    set-dist
    (execute
      (assert
        (ceil-log (to-nat dg + 1) ≤
          ceil-log (maxDegree + 1))) s)
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
using out unfolding assert-def
by (cases ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
  (auto simp: throw-no-outcome)
have len:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
using round-bound wf unfolding staged-budget-wellformed-def by simp
have exact:
  hash-range-budget
    (sum-list
      (take (ceil-log (to-nat dg + 1))
        (drop 0 (composition-fri-budgets budgets))) +
      ceil-log (to-nat dg + 1))
    (staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) [])
by (rule hash-range-budget-staged-composition-fri-program
  [OF controlled len])
have le:
  sum-list
    (take (ceil-log (to-nat dg + 1))
      (drop 0 (composition-fri-budgets budgets))) +
    ceil-log (to-nat dg + 1) ≤
  sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
using round-bound sum-list-take-le[of ceil-log (to-nat dg + 1)
  composition-fri-budgets budgets]

```

```

  by simp
show hash-range-budget
  (sum-list (composition-fri-budgets budgets) +
   ceil-log (maxDegree + 1))
  (staged-composition-fri-program A dg 0
   (ceil-log (to-nat dg + 1)) [])
  by (rule hash-range-budget-mono[OF le exact])
next
fix s x t
assume out:
  Some (x, t) ∈
    set-dist
      (execute
        (assert
          (ceil-log (to-nat dg + 1) ≤
            ceil-log (maxDegree + 1))) s)
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
  using out unfolding assert-def
  by (cases ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have len:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
  using round-bound wf unfolding staged-budget-wellformed-def by simp
have exact:
  hash-collision-budget
    (sum-list
      (take (ceil-log (to-nat dg + 1))
        (drop 0 (composition-fri-budgets budgets))) +
      ceil-log (to-nat dg + 1))
    (staged-composition-fri-program A dg 0
     (ceil-log (to-nat dg + 1)) [])
  by (rule hash-collision-budget-staged-composition-fri-program
      [OF controlled len])
have le:
  sum-list
    (take (ceil-log (to-nat dg + 1))
      (drop 0 (composition-fri-budgets budgets))) +
    ceil-log (to-nat dg + 1) ≤
    sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
  using round-bound sum-list-take-le[of ceil-log (to-nat dg + 1)
    composition-fri-budgets budgets]
  by simp
show hash-collision-budget
  (sum-list (composition-fri-budgets budgets) +
   ceil-log (maxDegree + 1))
  (staged-composition-fri-program A dg 0

```

```

    (ceil-log (to-nat dg + 1)) [])
  by (rule hash-collision-budget-mono[OF le exact])
qed
then show ?thesis by simp
qed

lemma hash-target-program-guarded-staged-composition-fri-program:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    hash-target-program B
      (sum-list (composition-fri-budgets budgets) +
        ceil-log (maxDegree + 1))
      (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []))
proof -
  let ?B =
    sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
  have hash-target-program B (0 + ?B)
    (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
  proof (rule hash-target-program-bind-on-outcomes
    [where n=0 and n'=?B])
    show hash-target-program B 0
      (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)))
    by (rule hash-target-program-assert)
  next
    fix s x t
    assume out:
      Some (x, t) ∈
        set-dist
          (execute
            (assert
              (ceil-log (to-nat dg + 1) ≤
                ceil-log (maxDegree + 1)))) s)
    have round-bound:
      ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
    using out unfolding assert-def
    by (cases ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
      (auto simp: throw-no-outcome)
    have len:
      0 + ceil-log (to-nat dg + 1) ≤
        length (composition-fri-budgets budgets)
    using round-bound wf unfolding staged-budget-wellformed-def by simp
    have exact:
      hash-target-program B

```



```

    (sum-list
      (take (ceil-log (to-nat dg + 1))
        (drop 0 (composition-fri-budgets budgets))) +
      ceil-log (to-nat dg + 1))
    (staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) [])
  by (rule hash-target-program-staged-composition-fri-program
    [OF controlled len])
have le:
  sum-list
    (take (ceil-log (to-nat dg + 1))
      (drop 0 (composition-fri-budgets budgets))) +
    ceil-log (to-nat dg + 1) ≤
  sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
  using round-bound sum-list-take-le[of ceil-log (to-nat dg + 1)
    composition-fri-budgets budgets]
  by simp
show hash-target-program B
  (sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1))
  (staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) [])
  by (rule hash-target-program-mono[OF le exact])
qed
then show ?thesis by simp
qed

```

```

lemma hash-target-program-staged-alpha-prefix-program:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    hash-target-program B
      (staged-alpha-search-queries budgets 0)
      (staged-alpha-prefix-program A)
proof -
  let ?trace-root = trace-root-budget budgets
  let ?trace-fri =
    sum-list (trace-fri-budgets budgets) + ceil-log clength
  let ?trace-final = trace-final-budget budgets
  have trace-root-target:
    hash-target-program B ?trace-root (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
  have trace-fri-target:
    hash-target-program B ?trace-fri
      (staged-trace-fri-program A 0 (ceil-log clength)) []
proof -
  have len:

```

```

0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
using wf unfolding staged-budget-wellformed-def by simp
have exact:
  hash-target-program B
    (sum-list
      (take (ceil-log clength)
        (drop 0 (trace-fri-budgets budgets))) +
      ceil-log clength)
    (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule hash-target-program-staged-trace-fri-program
    [OF controlled len])
then show ?thesis
  using wf unfolding staged-budget-wellformed-def by simp
qed
have trace-final-target:
  ∧bs. hash-target-program B ?trace-final (trace-final-stage A bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have after-trace-final-record:
  ∧fr trace-roots trace-bs trace-final.
  hash-target-program B 0
    (record-staged-message trace-final ≫=
      (λ-. return (fr, trace-roots, trace-bs, trace-final)))
proof –
  fix fr trace-roots trace-bs trace-final
  have hash-target-program B (0 + 0)
    (record-staged-message trace-final ≫=
      (λ-. return (fr, trace-roots, trace-bs, trace-final)))
  by (rule hash-target-program-bind)
    (rule hash-target-program-record-staged-message,
      rule hash-target-program-return)
  then show
    hash-target-program B 0
      (record-staged-message trace-final ≫=
        (λ-. return (fr, trace-roots, trace-bs, trace-final)))
    by simp
qed
have after-trace-final:
  ∧fr trace-roots trace-bs.
  hash-target-program B (?trace-final + 0)
    (trace-final-stage A trace-bs ≫=
      (λtrace-final.
        record-staged-message trace-final ≫=
          (λ-. return (fr, trace-roots, trace-bs, trace-final))))
  by (rule hash-target-program-bind)
    (rule trace-final-target, rule after-trace-final-record)
have after-trace-fri:
  ∧fr. hash-target-program B (?trace-fri + (?trace-final + 0))
    (staged-trace-fri-program A 0 (ceil-log clength) []) ≫=

```

```

    (λ(trace-roots, trace-bs).
      trace-final-stage A trace-bs >>=
      (λtrace-final.
        record-staged-message trace-final >>=
        (λ-. return (fr, trace-roots, trace-bs, trace-final))))))
proof (rule hash-target-program-bind)
  fix fr
  show hash-target-program B ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule trace-fri-target)
next
  fix fr x
  show hash-target-program B (?trace-final + 0)
    (case x of (trace-roots, trace-bs) ⇒
      trace-final-stage A trace-bs >>=
      (λtrace-final.
        record-staged-message trace-final >>=
        (λ-. return (fr, trace-roots, trace-bs, trace-final))))
  proof (cases x)
  case (Pair trace-roots trace-bs)
  have target:
    hash-target-program B (?trace-final + 0)
    (trace-final-stage A trace-bs >>=
    (λtrace-final.
      record-staged-message trace-final >>=
      (λ-. return
        (fr, trace-roots, trace-bs, trace-final))))
  by (rule hash-target-program-bind)
    (rule trace-final-target, rule after-trace-final-record)
  show ?thesis
    unfolding Pair using target by simp
  qed
qed
have after-trace-root-record:
  ∧fr. hash-target-program B (0 + (?trace-fri + (?trace-final + 0)))
    (record-staged-message fr >>=
    (λ-. staged-trace-fri-program A 0 (ceil-log clength) []) >>=
    (λ(trace-roots, trace-bs).
      trace-final-stage A trace-bs >>=
      (λtrace-final.
        record-staged-message trace-final >>=
        (λ-. return
          (fr, trace-roots, trace-bs, trace-final))))))
  by (rule hash-target-program-bind)
    (rule hash-target-program-record-staged-message, rule after-trace-fri)
have whole:
  hash-target-program B
    (?trace-root + (0 + (?trace-fri + (?trace-final + 0))))
    (staged-alpha-prefix-program A)

```

```

unfolding staged-alpha-prefix-program-def
by (rule hash-target-program-bind)
    (rule trace-root-target, rule after-trace-root-record)
show ?thesis
using whole
unfolding staged-alpha-search-queries-def
by (simp add: add.assoc add.commute add.left-commute)
qed

lemma staged-alpha-prefix-target-hit-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event (staged-alpha-prefix-program A)
    (hash-new-output-hit-event B s) s ≤
    staged-phase-target-error B
    (staged-alpha-search-queries budgets 0)
proof –
have budget:
  hash-target-budget B (staged-alpha-search-queries budgets 0)
    (staged-alpha-prefix-program A)
using hash-target-program-staged-alpha-prefix-program[OF wf controlled]
unfolding hash-target-program-def by simp
show ?thesis
using budget unfolding hash-target-budget-def
    staged-phase-target-error-def by blast
qed

lemma hash-target-program-staged-alpha-challenge-prefix-program:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i ≤ length spec
shows
  hash-target-program B (staged-alpha-search-queries budgets i)
    (staged-alpha-challenge-prefix-program A i)
proof –
let ?alpha-prefix = staged-alpha-search-queries budgets 0
have prefix-target:
  hash-target-program B ?alpha-prefix (staged-alpha-prefix-program A)
by (rule hash-target-program-staged-alpha-prefix-program[OF wf controlled])
have alpha-target:
   $\bigwedge$  prefix. hash-target-program B i
    (staged-alpha-program i  $\gg=$ 
      ( $\lambda$  as-prefix. return (prefix, as-prefix)))
proof –
fix prefix
have hash-target-program B (i + 0)
  (staged-alpha-program i  $\gg=$ 
    ( $\lambda$  as-prefix. return (prefix, as-prefix)))

```

```

    by (rule hash-target-program-bind)
      (rule hash-target-program-staged-alpha-program,
        rule hash-target-program-return)
  then show hash-target-program B i
    (staged-alpha-program i  $\gg$ 
      ( $\lambda$ as-prefix. return (prefix, as-prefix)))
  by simp
qed
have whole:
  hash-target-program B (?alpha-prefix + i)
    (staged-alpha-challenge-prefix-program A i)
  unfolding staged-alpha-challenge-prefix-program-def
  by (rule hash-target-program-bind[OF prefix-target alpha-target])
show ?thesis
  using whole i-bound
  unfolding staged-alpha-search-queries-def
  by simp
qed

lemma staged-alpha-prefix-target-hit-bound-i:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound:  $i \leq \text{length spec}$ 
  shows
    wp-event (staged-alpha-challenge-prefix-program A i)
      (hash-new-output-hit-event B s) s  $\leq$ 
      staged-phase-target-error B
      (staged-alpha-search-queries budgets i)
  proof -
    have budget:
      hash-target-budget B (staged-alpha-search-queries budgets i)
        (staged-alpha-challenge-prefix-program A i)
    using hash-target-program-staged-alpha-challenge-prefix-program
      [OF wf controlled i-bound]
    unfolding hash-target-program-def by simp
  show ?thesis
    using budget unfolding hash-target-budget-def
      staged-phase-target-error-def by blast
qed

lemma hash-target-program-staged-composition-fri-vector-program:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    hash-target-program B
      (staged-composition-fri-vector-search-queries budgets)
      (staged-composition-fri-vector-program A)
  proof -
    let ?alpha-prefix = staged-alpha-search-queries budgets 0

```

```

let ?alpha = length spec
let ?degree = degree-budget budgets
let ?composition =
  sum-list (composition-fri-budgets budgets) +
  ceil-log (maxDegree + 1)
have alpha-prefix-target:
  hash-target-program B ?alpha-prefix (staged-alpha-prefix-program A)
  by (rule hash-target-program-staged-alpha-prefix-program[OF wf controlled])
have alpha-target:
  hash-target-program B ?alpha (staged-alpha-program (length spec))
  by (rule hash-target-program-staged-alpha-program)
have degree-target:
   $\wedge$ as. hash-target-program B ?degree (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have guarded-return:
   $\wedge$ dg. hash-target-program B (?composition + 0)
  ((assert
    (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ =
    ( $\lambda$ -. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) []))  $\gg$ =
    ( $\lambda$ x. case x of (-, composition-bs)  $\Rightarrow$ 
      return (dg, composition-bs)))
  by (rule hash-target-program-bind)
  (rule hash-target-program-guarded-staged-composition-fri-program
    [OF wf controlled], simp add: hash-target-program-return
    split: prod.splits)
have after-record:
   $\wedge$ dg. hash-target-program B (0 + (?composition + 0))
  (record-staged-message dg  $\gg$ =
    ( $\lambda$ -. (assert
      (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ =
      ( $\lambda$ -. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))  $\gg$ =
      ( $\lambda$ x. case x of (-, composition-bs)  $\Rightarrow$ 
        return (dg, composition-bs))))
  by (rule hash-target-program-bind)
  (rule hash-target-program-record-staged-message, rule guarded-return)
have after-degree:
   $\wedge$ as. hash-target-program B
  (?degree + (0 + (?composition + 0)))
  (degree-stage A as  $\gg$ =
    ( $\lambda$ dg. record-staged-message dg  $\gg$ =
      ( $\lambda$ -. let composition-rounds = ceil-log (to-nat dg + 1) in
        assert (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg$ =
        ( $\lambda$ -. staged-composition-fri-program A dg 0
          composition-rounds []  $\gg$ =
            ( $\lambda$ (-, composition-bs). return (dg, composition-bs))))))
  unfolding Let-def

```

```

proof (rule hash-target-program-bind)
  fix as
  show hash-target-program B ?degree (degree-stage A as)
    by (rule degree-target)
next
  fix as dg
  show hash-target-program B (0 + (?composition + 0))
    (record-staged-message dg  $\gg$ 
      ( $\lambda$ -. assert
        (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
        ( $\lambda$ -. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1))  $\square$   $\gg$ 
            ( $\lambda$ (-, composition-bs). return (dg, composition-bs))))))
    using after-record[of dg] by (simp add: sm-bind-assoc)
qed
have after-alpha:
   $\wedge$ x. hash-target-program B
    (?alpha + (?degree + (0 + (?composition + 0))))
    (staged-alpha-program (length spec)  $\gg$ 
      ( $\lambda$ as. degree-stage A as  $\gg$ 
        ( $\lambda$ dg. record-staged-message dg  $\gg$ 
          ( $\lambda$ -. let composition-rounds = ceil-log (to-nat dg + 1) in
            assert (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
              ( $\lambda$ -. staged-composition-fri-program A dg 0
                composition-rounds  $\square$   $\gg$ 
                  ( $\lambda$ (-, composition-bs). return (dg, composition-bs))))))
proof (rule hash-target-program-bind)
  fix x
  show hash-target-program B ?alpha
    (staged-alpha-program (length spec))
    by (rule alpha-target)
next
  fix x as
  show hash-target-program B (?degree + (0 + (?composition + 0)))
    (degree-stage A as  $\gg$ 
      ( $\lambda$ dg. record-staged-message dg  $\gg$ 
        ( $\lambda$ -. let composition-rounds = ceil-log (to-nat dg + 1)
          in assert
            (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
              ( $\lambda$ -. staged-composition-fri-program A dg 0
                composition-rounds  $\square$   $\gg$ 
                  ( $\lambda$ (-, composition-bs). return (dg, composition-bs))))))
    by (rule after-degree)
qed
have whole:
  hash-target-program B
    (?alpha-prefix +
      (?alpha + (?degree + (0 + (?composition + 0))))
      (staged-composition-fri-vector-program A)

```

```

unfolding staged-composition-fri-vector-program-def
proof (rule hash-target-program-bind)
  show hash-target-program B ?alpha-prefix
    (staged-alpha-prefix-program A)
    by (rule alpha-prefix-target)
next
  fix x
  show hash-target-program B
    (?alpha + (?degree + (0 + (?composition + 0))))
    (case x of
      (a, b, trace-bs, c) =>
        staged-alpha-program (length spec) >>=
        (λas. degree-stage A as >>=
          (λdg. record-staged-message dg >>=
            (λ-. let composition-rounds =
                  ceil-log (to-nat dg + 1)
                in assert
                  (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
                  (λ-. staged-composition-fri-program A dg 0
                    composition-rounds [] >>=
                    (λ(r, composition-bs).
                      return (dg, composition-bs)))))))
    using after-alpha by (cases x) simp
qed
show ?thesis
  using whole
  unfolding staged-composition-fri-vector-search-queries-def
  by (simp add: add.assoc add.commute add.left-commute)
qed

lemma staged-composition-fri-vector-target-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (staged-composition-fri-vector-program A)
      (hash-new-output-hit-event B s) s ≤
      staged-phase-target-error B
      (staged-composition-fri-vector-search-queries budgets)
proof -
  have budget:
    hash-target-budget B
      (staged-composition-fri-vector-search-queries budgets)
      (staged-composition-fri-vector-program A)
  using hash-target-program-staged-composition-fri-vector-program
    [OF wf controlled]
  unfolding hash-target-program-def by simp
show ?thesis
  using budget unfolding hash-target-budget-def
    staged-phase-target-error-def by blast

```


qed

lemma *hash-target-program-staged-trace-fri-challenge-prefix-program:*

assumes *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < length (trace-fri-budgets budgets)*

shows

hash-target-program B
(staged-trace-fri-search-queries budgets i)
(staged-trace-fri-challenge-prefix-program A i)

proof –

let *?trace-root = trace-root-budget budgets*

let *?completed = sum-list (take i (trace-fri-budgets budgets)) + i*

let *?current = trace-fri-budgets budgets ! i*

have *trace-root-target:*

hash-target-program B ?trace-root (trace-root-stage A)
using *controlled unfolding staged-adversary-controlled-def*
by *(blast intro: controlled-ro-program-target)*

have *completed-target:*

hash-target-program B ?completed
(staged-trace-fri-program A 0 i [])

proof –

have *len: 0 + i ≤ length (trace-fri-budgets budgets)*

using *i-bound by simp*

have *exact:*

hash-target-program B
(sum-list (take i (drop 0 (trace-fri-budgets budgets))) + i)
(staged-trace-fri-program A 0 i [])
by *(rule hash-target-program-staged-trace-fri-program*
[OF controlled len])

then show *?thesis by simp*

qed

have *current-target:*

∧ trace-bs. hash-target-program B ?current
(trace-fri-root-stage A i trace-bs)
using *controlled i-bound unfolding staged-adversary-controlled-def*
by *(blast intro: controlled-ro-program-target)*

have *after-current-record:*

∧ fr trace-bs root. hash-target-program B 0
(record-staged-message root ≫=
(λ-. return (fr, trace-bs, root)))

proof –

fix *fr trace-bs root*

have *hash-target-program B (0 + 0)*

(record-staged-message root ≫=
(λ-. return (fr, trace-bs, root)))

by *(rule hash-target-program-bind)*
(rule hash-target-program-record-staged-message,
rule hash-target-program-return)

then show *hash-target-program B 0*

```

    (record-staged-message root  $\gg$ 
      ( $\lambda$ -. return (fr, trace-bs, root)))
  by simp
qed
have after-current:
 $\wedge$ fr trace-bs. hash-target-program B (?current + 0)
  (trace-fri-root-stage A i trace-bs  $\gg$ 
    ( $\lambda$ root. record-staged-message root  $\gg$ 
      ( $\lambda$ -. return (fr, trace-bs, root))))
  by (rule hash-target-program-bind)
    (rule current-target, rule after-current-record)
have after-completed:
 $\wedge$ fr. hash-target-program B (?completed + (?current + 0))
  (staged-trace-fri-program A 0 i []  $\gg$ 
    ( $\lambda$ (-, trace-bs).
      trace-fri-root-stage A i trace-bs  $\gg$ 
        ( $\lambda$ root. record-staged-message root  $\gg$ 
          ( $\lambda$ -. return (fr, trace-bs, root))))))
proof (rule hash-target-program-bind)
  fix fr
  show hash-target-program B ?completed
    (staged-trace-fri-program A 0 i [])
  by (rule completed-target)
next
  fix fr x
  show hash-target-program B (?current + 0)
    (case x of (-, trace-bs)  $\Rightarrow$ 
      trace-fri-root-stage A i trace-bs  $\gg$ 
        ( $\lambda$ root. record-staged-message root  $\gg$ 
          ( $\lambda$ -. return (fr, trace-bs, root))))
  proof (cases x)
    case (Pair roots trace-bs)
    have target:
      hash-target-program B (?current + 0)
        (trace-fri-root-stage A i trace-bs  $\gg$ 
          ( $\lambda$ root. record-staged-message root  $\gg$ 
            ( $\lambda$ -. return (fr, trace-bs, root))))
    by (rule after-current)
    show ?thesis
      unfolding Pair using target by simp
  qed
qed
have after-trace-root-record:
 $\wedge$ fr. hash-target-program B (0 + (?completed + (?current + 0)))
  (record-staged-message fr  $\gg$ 
    ( $\lambda$ -. staged-trace-fri-program A 0 i []  $\gg$ 
      ( $\lambda$ (-, trace-bs).
        trace-fri-root-stage A i trace-bs  $\gg$ 
          ( $\lambda$ root. record-staged-message root  $\gg$ 
            ( $\lambda$ -. return (fr, trace-bs, root))))))

```

```

      (λ-. return (fr, trace-bs, root))))))
  by (rule hash-target-program-bind)
    (rule hash-target-program-record-staged-message, rule after-completed)
  have whole:
    hash-target-program B
      (?trace-root + (0 + (?completed + (?current + 0))))
      (staged-trace-fri-challenge-prefix-program A i)
  unfolding staged-trace-fri-challenge-prefix-program-def
  by (rule hash-target-program-bind)
    (rule trace-root-target, rule after-trace-root-record)
  have sum-eq:
    ?current + sum-list (take i (trace-fri-budgets budgets)) =
      sum-list (take (Suc i) (trace-fri-budgets budgets))
  proof -
    have take (Suc i) (trace-fri-budgets budgets) =
      take i (trace-fri-budgets budgets) @
      [trace-fri-budgets budgets ! i]
    by (rule take-Suc-conv-app-nth[OF i-bound])
    then show ?thesis by simp
  qed
  show ?thesis
    using whole
    unfolding staged-trace-fri-search-queries-def
    by (simp add: sum-eq add.assoc add.commute add.left-commute)
  qed

lemma staged-trace-fri-prefix-target-hit-bound:
  assumes controlled: staged-adversary-controlled budgets A
  and i-bound: i < length (trace-fri-budgets budgets)
  shows
    wp-event (staged-trace-fri-challenge-prefix-program A i)
      (hash-new-output-hit-event B s) s ≤
      staged-phase-target-error B
      (staged-trace-fri-search-queries budgets i)
  proof -
    have budget:
      hash-target-budget B (staged-trace-fri-search-queries budgets i)
      (staged-trace-fri-challenge-prefix-program A i)
    using hash-target-program-staged-trace-fri-challenge-prefix-program
      [OF controlled i-bound]
    unfolding hash-target-program-def by simp
    show ?thesis
      using budget unfolding hash-target-budget-def
      staged-phase-target-error-def by blast
  qed

lemma hash-target-program-staged-composition-fri-challenge-prefix-program:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A

```

```

    and i-bound:  $i < \text{ceil-log} (\text{maxDegree} + 1)$ 
  shows
    hash-target-program B
      (staged-composition-fri-search-queries budgets i)
      (staged-composition-fri-challenge-prefix-program A i)
  proof -
    let ?alpha-prefix = staged-alpha-search-queries budgets 0
    let ?alpha = length spec
    let ?degree = degree-budget budgets
    let ?completed =
      sum-list (take i (composition-fri-budgets budgets)) + i
    let ?current = composition-fri-budgets budgets ! i
    have comp-i-bound:  $i < \text{length} (\text{composition-fri-budgets budgets})$ 
      using wf i-bound unfolding staged-budget-wellformed-def by simp
    have alpha-prefix-target:
      hash-target-program B ?alpha-prefix (staged-alpha-prefix-program A)
      by (rule hash-target-program-staged-alpha-prefix-program[OF wf controlled])
    have alpha-target:
      hash-target-program B ?alpha (staged-alpha-program (length spec))
      by (rule hash-target-program-staged-alpha-program)
    have degree-target:
       $\wedge$  as. hash-target-program B ?degree (degree-stage A as)
      using controlled unfolding staged-adversary-controlled-def
      by (blast intro: controlled-ro-program-target)
    have completed-target:
       $\wedge$  dg. hash-target-program B ?completed
      (staged-composition-fri-program A dg 0 i [])
  proof -
    fix dg
    have len:  $0 + i \leq \text{length} (\text{composition-fri-budgets budgets})$ 
      using comp-i-bound by simp
    have exact:
      hash-target-program B
        (sum-list (take i (drop 0 (composition-fri-budgets budgets)))) + i
        (staged-composition-fri-program A dg 0 i [])
      by (rule hash-target-program-staged-composition-fri-program
        [OF controlled len])
    then show
      hash-target-program B ?completed
        (staged-composition-fri-program A dg 0 i [])
      by simp
  qed
  have current-target:
     $\wedge$  dg composition-bs. hash-target-program B ?current
      (composition-fri-root-stage A dg i composition-bs)
    using controlled comp-i-bound
    unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-target)
  have after-current-record:

```

```

 $\wedge dg$  composition-bs root. hash-target-program B 0
  (record-staged-message root  $\gg=$ 
    ( $\lambda$ -. return (dg, composition-bs, root)))
proof –
  fix dg composition-bs root
  have hash-target-program B (0 + 0)
    (record-staged-message root  $\gg=$ 
      ( $\lambda$ -. return (dg, composition-bs, root)))
  by (rule hash-target-program-bind)
    (rule hash-target-program-record-staged-message,
      rule hash-target-program-return)
  then show
    hash-target-program B 0
    (record-staged-message root  $\gg=$ 
      ( $\lambda$ -. return (dg, composition-bs, root)))
  by simp
qed
have after-current:
   $\wedge dg$  composition-bs. hash-target-program B (?current + 0)
    (composition-fri-root-stage A dg i composition-bs  $\gg=$ 
      ( $\lambda$ root. record-staged-message root  $\gg=$ 
        ( $\lambda$ -. return (dg, composition-bs, root)))))
  by (rule hash-target-program-bind)
    (rule current-target, rule after-current-record)
have after-completed:
   $\wedge dg$ . hash-target-program B (?completed + (?current + 0))
    (staged-composition-fri-program A dg 0 i []  $\gg=$ 
      ( $\lambda$ (-, composition-bs).
        composition-fri-root-stage A dg i composition-bs  $\gg=$ 
          ( $\lambda$ root. record-staged-message root  $\gg=$ 
            ( $\lambda$ -. return (dg, composition-bs, root)))))
proof (rule hash-target-program-bind)
  fix dg
  show hash-target-program B ?completed
    (staged-composition-fri-program A dg 0 i [])
  by (rule completed-target)
next
  fix dg x
  show hash-target-program B (?current + 0)
    (case x of (-, composition-bs)  $\Rightarrow$ 
      composition-fri-root-stage A dg i composition-bs  $\gg=$ 
        ( $\lambda$ root. record-staged-message root  $\gg=$ 
          ( $\lambda$ -. return (dg, composition-bs, root)))))
  proof (cases x)
  case (Pair roots composition-bs)
  have target:
    hash-target-program B (?current + 0)
    (composition-fri-root-stage A dg i composition-bs  $\gg=$ 
      ( $\lambda$ root. record-staged-message root  $\gg=$ 

```

```

      (λ-. return (dg, composition-bs, root))))
    by (rule after-current)
  show ?thesis
    unfolding Pair using target by simp
qed
qed
have after-second-assert:
  ∧ dg. hash-target-program B (0 + (?completed + (?current + 0)))
    (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) ≫=
      (λ-. staged-composition-fri-program A dg 0 i [] ≫=
        (λ(-, composition-bs).
          composition-fri-root-stage A dg i composition-bs ≫=
            (λroot. record-staged-message root ≫=
              (λ-. return (dg, composition-bs, root)))))))
    by (rule hash-target-program-bind)
      (rule hash-target-program-assert, rule after-completed)
have after-first-assert:
  ∧ dg. hash-target-program B
    (0 + (0 + (?completed + (?current + 0))))
    (assert (Suc i ≤ ceil-log (to-nat dg + 1)) ≫=
      (λ-. assert
        (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) ≫=
          (λ-. staged-composition-fri-program A dg 0 i [] ≫=
            (λ(-, composition-bs).
              composition-fri-root-stage A dg i composition-bs ≫=
                (λroot. record-staged-message root ≫=
                  (λ-. return (dg, composition-bs, root)))))))
    by (rule hash-target-program-bind)
      (rule hash-target-program-assert, rule after-second-assert)
have after-degree-record:
  ∧ dg. hash-target-program B
    (0 + (0 + (0 + (?completed + (?current + 0)))))
    (record-staged-message dg ≫=
      (λ-. assert (Suc i ≤ ceil-log (to-nat dg + 1)) ≫=
        (λ-. assert
          (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) ≫=
            (λ-. staged-composition-fri-program A dg 0 i [] ≫=
              (λ(-, composition-bs).
                composition-fri-root-stage A dg i composition-bs ≫=
                  (λroot. record-staged-message root ≫=
                    (λ-. return (dg, composition-bs, root)))))))
    by (rule hash-target-program-bind)
      (rule hash-target-program-record-staged-message, rule after-first-assert)
have after-degree:
  ∧ as. hash-target-program B
    (?degree + (0 + (0 + (0 + (?completed + (?current + 0)))))
    (degree-stage A as ≫=
      (λdg. record-staged-message dg ≫=
        (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in

```

```

    assert (Suc i ≤ composition-rounds) >>=
    (λ-. assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
    (λ-. staged-composition-fri-program A dg 0 i [] >>=
    (λ(-, composition-bs).
    composition-fri-root-stage A dg i composition-bs >>=
    (λroot. record-staged-message root >>=
    (λ-. return (dg, composition-bs, root))))))))))
unfolding Let-def
by (rule hash-target-program-bind)
    (rule degree-target, rule after-degree-record)
have after-alpha:
  ∧x. hash-target-program B
    (?alpha + (?degree + (0 + (0 + (0 + (?completed + (?current + 0)))))))
    (staged-alpha-program (length spec) >>=
    (λas. degree-stage A as >>=
    (λdg. record-staged-message dg >>=
    (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
    assert (Suc i ≤ composition-rounds) >>=
    (λ-. assert
    (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
    (λ-. staged-composition-fri-program A dg 0 i [] >>=
    (λ(-, composition-bs).
    composition-fri-root-stage A dg i composition-bs >>=
    (λroot. record-staged-message root >>=
    (λ-. return
    (dg, composition-bs, root)))))))))))
by (rule hash-target-program-bind)
    (rule alpha-target, rule after-degree)
have after-alpha-prefix:
  hash-target-program B
    (?alpha-prefix +
    (?alpha + (?degree +
    (0 + (0 + (0 + (?completed + (?current + 0)))))))
    (staged-composition-fri-challenge-prefix-program A i)
unfolding staged-composition-fri-challenge-prefix-program-def
proof (rule hash-target-program-bind)
show hash-target-program B ?alpha-prefix
    (staged-alpha-prefix-program A)
by (rule alpha-prefix-target)
next
fix x
show hash-target-program B
    (?alpha + (?degree +
    (0 + (0 + (0 + (?completed + (?current + 0)))))))
    (case x of (-, -, trace-bs, -) ⇒
    staged-alpha-program (length spec) >>=
    (λas. degree-stage A as >>=
    (λdg. record-staged-message dg >>=
    (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in

```

```

    assert (Suc i ≤ composition-rounds) >>=
    (λ-. assert
      (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0 i [] >>=
        (λ(-, composition-bs).
          composition-fri-root-stage A dg i composition-bs >>=
            (λroot. record-staged-message root >>=
              (λ-. return
                (dg, composition-bs, root))))))))))
  proof (cases x)
    case (fields a b c d)
    have target:
      hash-target-program B
      (?alpha + (?degree +
        (0 + (0 + (0 + (?completed + (?current + 0)))))))
      (staged-alpha-program (length spec) >>=
        (λas. degree-stage A as >>=
          (λdg. record-staged-message dg >>=
            (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
              assert (Suc i ≤ composition-rounds) >>=
              (λ-. assert
                (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
                (λ-. staged-composition-fri-program A dg 0 i [] >>=
                  (λ(-, composition-bs).
                    composition-fri-root-stage A dg i composition-bs >>=
                      (λroot. record-staged-message root >>=
                        (λ-. return
                          (dg, composition-bs, root))))))))))
            by (rule after-alpha)
          show ?thesis
            unfolding fields
            using target
            by (simp add: add.assoc add.commute add.left-commute)
        qed
      qed
    have sum-eq:
      ?current + sum-list (take i (composition-fri-budgets budgets)) =
      sum-list (take (Suc i) (composition-fri-budgets budgets))
    proof -
      have take (Suc i) (composition-fri-budgets budgets) =
        take i (composition-fri-budgets budgets) @
        [composition-fri-budgets budgets ! i]
      by (rule take-Suc-conv-app-nth[OF comp-i-bound])
      then show ?thesis by simp
    qed
  show ?thesis
    using after-alpha-prefix
    unfolding staged-composition-fri-search-queries-def
    staged-alpha-search-queries-def

```


by (simp add: sum-eq add.assoc add.commute add.left-commute)
qed

lemma staged-composition-fri-prefix-target-hit-bound:

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i < \text{ceil-log } (\text{maxDegree} + 1)$

shows

wp-event (staged-composition-fri-challenge-prefix-program A i)
(hash-new-output-hit-event B s) $s \leq$
staged-phase-target-error B
(staged-composition-fri-search-queries budgets i)

proof –

have budget:

hash-target-budget B (staged-composition-fri-search-queries budgets i)
(staged-composition-fri-challenge-prefix-program A i)

using

hash-target-program-staged-composition-fri-challenge-prefix-program
[OF wf controlled i-bound]

unfolding hash-target-program-def by simp

show ?thesis

using budget unfolding hash-target-budget-def
staged-phase-target-error-def by blast

qed

lemma hash-target-program-staged-query-challenge-prefix-program:

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i \leq \text{rounds}$

shows

hash-target-program B
(staged-query-search-queries budgets i)
(staged-query-challenge-prefix-program A i)

proof –

let ?alpha-prefix = staged-alpha-search-queries budgets 0

let ?alpha = length spec

let ?degree = degree-budget budgets

let ?composition-fri =

sum-list (composition-fri-budgets budgets) +
ceil-log (maxDegree + 1)

let ?composition-final = composition-final-budget budgets

let ?query = sum-list (take i (query-opening-budgets budgets)) + i

have alpha-prefix-target:

hash-target-program B ?alpha-prefix (staged-alpha-prefix-program A)
by (rule hash-target-program-staged-alpha-prefix-program[OF wf controlled])

have alpha-target:

hash-target-program B ?alpha (staged-alpha-program (length spec))
by (rule hash-target-program-staged-alpha-program)

have degree-target:

```

 $\wedge$ as. hash-target-program B ?degree (degree-stage A as)
using controlled unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-target)
have composition-fri-target:
 $\wedge$ dg. hash-target-program B ?composition-fri
  (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
    ( $\lambda$ -. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) []))
by (rule hash-target-program-guarded-staged-composition-fri-program
  [OF wf controlled])
have composition-final-target:
 $\wedge$ dg bs. hash-target-program B ?composition-final
  (composition-final-stage A dg bs)
using controlled unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-target)
have query-target:
  hash-target-program B ?query (staged-query-program A 0 i)
proof –
  have len: 0 + i  $\leq$  length (query-opening-budgets budgets)
    using wf i-bound unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-target-program B
      (sum-list (take i (drop 0 (query-opening-budgets budgets))) + i)
      (staged-query-program A 0 i)
    by (rule hash-target-program-staged-query-program
      [OF controlled len])
  then show ?thesis by simp
qed
have after-query:
 $\wedge$ dg composition-final. hash-target-program B (?query + 0)
  (staged-query-program A 0 i  $\gg$ 
    ( $\lambda$ query-chunks.
      return (dg, composition-final, query-chunks)))
by (rule hash-target-program-bind)
  (rule query-target, rule hash-target-program-return)
have after-composition-final-record:
 $\wedge$ dg composition-final. hash-target-program B (0 + (?query + 0))
  (record-staged-message composition-final  $\gg$ 
    ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
      ( $\lambda$ query-chunks.
        return (dg, composition-final, query-chunks))))
by (rule hash-target-program-bind)
  (rule hash-target-program-record-staged-message, rule after-query)
have after-composition-final:
 $\wedge$ dg composition-bs. hash-target-program B
  (?composition-final + (0 + (?query + 0)))
  (composition-final-stage A dg composition-bs  $\gg$ 
    ( $\lambda$ composition-final.
      record-staged-message composition-final  $\gg$ 

```

```

    (λ-. staged-query-program A 0 i >>=
      (λquery-chunks.
        return (dg, composition-final, query-chunks))))))
  by (rule hash-target-program-bind)
    (rule composition-final-target, rule after-composition-final-record)
have after-composition-fri:
  ∧ dg. hash-target-program B
    (?composition-fri + (?composition-final + (0 + (?query + 0))))
    ((assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [])) >>=
      (λ(-, composition-bs).
        composition-final-stage A dg composition-bs >>=
          (λcomposition-final.
            record-staged-message composition-final >>=
              (λ-. staged-query-program A 0 i >>=
                (λquery-chunks.
                  return (dg, composition-final, query-chunks)))))))
proof (rule hash-target-program-bind)
  fix dg
  show hash-target-program B ?composition-fri
    (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
    by (rule composition-fri-target)
next
  fix dg x
  show hash-target-program B
    (?composition-final + (0 + (?query + 0)))
    (case x of (-, composition-bs) ⇒
      composition-final-stage A dg composition-bs >>=
        (λcomposition-final.
          record-staged-message composition-final >>=
            (λ-. staged-query-program A 0 i >>=
              (λquery-chunks.
                return (dg, composition-final, query-chunks))))))
proof (cases x)
  case (Pair composition-roots composition-bs)
  have target:
    hash-target-program B
      (?composition-final + (0 + (?query + 0)))
      (composition-final-stage A dg composition-bs >>=
        (λcomposition-final.
          record-staged-message composition-final >>=
            (λ-. staged-query-program A 0 i >>=
              (λquery-chunks.
                return (dg, composition-final, query-chunks))))))
    by (rule after-composition-final)
  show ?thesis

```

```

    unfolding Pair using target by simp
  qed
  qed
  have after-degree-record:
   $\wedge dg.$  hash-target-program  $B$ 
     $(0 + (?composition-fri + (?composition-final + (0 + (?query + 0))))))$ 
    (record-staged-message  $dg \gg=$ 
       $(\lambda-. (assert$ 
         $(ceil-log (to-nat dg + 1) \leq ceil-log (maxDegree + 1)) \gg=$ 
         $(\lambda-. staged-composition-fri-program A dg 0$ 
           $(ceil-log (to-nat dg + 1)) [])) \gg=$ 
         $(\lambda(-, composition-bs).$ 
          composition-final-stage  $A dg composition-bs \gg=$ 
           $(\lambda composition-final.$ 
            record-staged-message composition-final  $\gg=$ 
             $(\lambda-. staged-query-program A 0 i \gg=$ 
               $(\lambda query-chunks.$ 
                return  $(dg, composition-final, query-chunks))))))$ 
      by (rule hash-target-program-bind)
      (rule hash-target-program-record-staged-message,
        rule after-composition-fri)
  have after-degree-record-expanded:
   $\wedge dg.$  hash-target-program  $B$ 
     $(0 + (?composition-fri + (?composition-final + (0 + (?query + 0))))))$ 
    (record-staged-message  $dg \gg=$ 
       $(\lambda-. assert$ 
         $(ceil-log (to-nat dg + 1) \leq ceil-log (maxDegree + 1)) \gg=$ 
         $(\lambda-. staged-composition-fri-program A dg 0$ 
           $(ceil-log (to-nat dg + 1)) [] \gg=$ 
           $(\lambda(-, composition-bs).$ 
            composition-final-stage  $A dg composition-bs \gg=$ 
             $(\lambda composition-final.$ 
              record-staged-message composition-final  $\gg=$ 
               $(\lambda-. staged-query-program A 0 i \gg=$ 
                 $(\lambda query-chunks.$ 
                  return  $(dg, composition-final, query-chunks))))))$ 
      using after-degree-record
      by (simp add: sm-bind-assoc split: prod.splits)
  have after-degree:
   $\wedge as.$  hash-target-program  $B$ 
     $(?degree +$ 
       $(0 + (?composition-fri + (?composition-final + (0 + (?query + 0))))))$ 
    (degree-stage  $A as \gg=$ 
       $(\lambda dg.$  record-staged-message  $dg \gg=$ 
         $(\lambda-. let composition-rounds = ceil-log (to-nat dg + 1) in$ 
          assert  $(composition-rounds \leq ceil-log (maxDegree + 1)) \gg=$ 
           $(\lambda-. staged-composition-fri-program A dg 0$ 
            composition-rounds  $[] \gg=$ 
             $(\lambda(-, composition-bs).$ 

```

```

      composition-final-stage A dg composition-bs >>=
      (λcomposition-final.
        record-staged-message composition-final >>=
        (λ-. staged-query-program A 0 i >>=
          (λquery-chunks.
            return
            (dg, composition-final, query-chunks)))))))))
unfolding Let-def
by (rule hash-target-program-bind)
    (rule degree-target, rule after-degree-record-expanded)
have after-alpha:
  ∧x. hash-target-program B
    (?alpha +
      (?degree +
        (0 + (?composition-fri +
          (?composition-final + (0 + (?query + 0)))))))
    (staged-alpha-program (length spec) >>=
      (λas. degree-stage A as >>=
        (λdg. record-staged-message dg >>=
          (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
            assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
            (λ-. staged-composition-fri-program A dg 0
              composition-rounds [] >>=
              (λ(-, composition-bs).
                composition-final-stage A dg composition-bs >>=
                (λcomposition-final.
                  record-staged-message composition-final >>=
                  (λ-. staged-query-program A 0 i >>=
                    (λquery-chunks.
                      return
                      (dg, composition-final, query-chunks)))))))))
        by (rule hash-target-program-bind)
          (rule alpha-target, rule after-degree)
have after-alpha-prefix:
  hash-target-program B
    (?alpha-prefix +
      (?alpha +
        (?degree +
          (0 + (?composition-fri +
            (?composition-final + (0 + (?query + 0)))))))
    (staged-query-challenge-prefix-program A i)
unfolding staged-query-challenge-prefix-program-def
proof (rule hash-target-program-bind)
show hash-target-program B ?alpha-prefix
  (staged-alpha-prefix-program A)
  by (rule alpha-prefix-target)
next
fix x
show hash-target-program B

```

```

(?alpha +
  (?degree +
    (0 + (?composition-fri +
      (?composition-final + (0 + (?query + 0)))))))
(case x of (-, -, trace-bs, -) =>
  staged-alpha-program (length spec) >=>
    (λas. degree-stage A as >=>
      (λdg. record-staged-message dg >=>
        (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
          assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >=>
            (λ-. staged-composition-fri-program A dg 0
              composition-rounds [] >=>
                (λ(-, composition-bs).
                  composition-final-stage A dg composition-bs >=>
                    (λcomposition-final.
                      record-staged-message composition-final >=>
                        (λ-. staged-query-program A 0 i >=>
                          (λquery-chunks.
                            return
                              (dg, composition-final,
                                query-chunks))))))))))
proof (cases x)
case (fields a b c d)
have target:
  hash-target-program B
  (?alpha +
    (?degree +
      (0 + (?composition-fri +
        (?composition-final + (0 + (?query + 0)))))))
  (staged-alpha-program (length spec) >=>
    (λas. degree-stage A as >=>
      (λdg. record-staged-message dg >=>
        (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
          assert
            (composition-rounds ≤ ceil-log (maxDegree + 1)) >=>
              (λ-. staged-composition-fri-program A dg 0
                composition-rounds [] >=>
                  (λ(-, composition-bs).
                    composition-final-stage A dg composition-bs >=>
                      (λcomposition-final.
                        record-staged-message composition-final >=>
                          (λ-. staged-query-program A 0 i >=>
                            (λquery-chunks.
                              return
                                (dg, composition-final,
                                  query-chunks))))))))))
  by (rule after-alpha)
show ?thesis
unfolding fields

```

```

    using target
    by (simp add: add.assoc add.commute add.left-commute)
qed
qed
show ?thesis
  using after-alpha-prefix
  unfolding staged-query-search-queries-def staged-alpha-search-queries-def
  by (simp add: add.assoc add.commute add.left-commute)
qed

lemma hash-range-budget-staged-query-challenge-prefix-program:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i \leq \text{rounds}$ 
  shows
    hash-range-budget
      (staged-query-search-queries budgets i)
      (staged-query-challenge-prefix-program A i)
proof -
  let ?alpha-prefix = staged-alpha-search-queries budgets 0
  let ?alpha = length spec
  let ?degree = degree-budget budgets
  let ?composition-fri =
    sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
  let ?composition-final = composition-final-budget budgets
  let ?query = sum-list (take i (query-opening-budgets budgets)) + i
  have alpha-prefix:
    hash-range-budget ?alpha-prefix (staged-alpha-prefix-program A)
  by (rule hash-range-budget-staged-alpha-prefix-program
    [OF wf controlled])
  have alpha: hash-range-budget ?alpha (staged-alpha-program (length spec))
  by (rule hash-range-budget-staged-alpha-program)
  have degree:
     $\wedge as. \text{hash-range-budget } ?degree (\text{degree-stage } A \text{ as})$ 
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
  have composition-fri:
     $\wedge dg. \text{hash-range-budget } ?composition-fri$ 
    (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
    ( $\lambda-. \text{staged-composition-fri-program } A \text{ dg } 0$ 
    (ceil-log (to-nat dg + 1)) []))
  by (rule hash-range-budget-guarded-staged-composition-fri-program
    [OF wf controlled])
  have composition-final:
     $\wedge dg \text{ bs. hash-range-budget } ?composition-final$ 
    (composition-final-stage A dg bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)

```

```

have query: hash-range-budget ?query (staged-query-program A 0 i)
proof –
  have len: 0 + i ≤ length (query-opening-budgets budgets)
    using wf i-bound unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-range-budget
      (sum-list (take i (drop 0 (query-opening-budgets budgets))) + i)
      (staged-query-program A 0 i)
    by (rule hash-range-budget-staged-query-program[OF controlled len])
  then show ?thesis by simp
qed
have after-query:
  ∧ dg composition-final. hash-range-budget (?query + 0)
    (staged-query-program A 0 i >>=
      (λ query-chunks. return (dg, composition-final, query-chunks)))
  by (rule hash-range-budget-bind)
    (rule query, rule hash-range-budget-return)
have after-composition-final-record:
  ∧ dg composition-final. hash-range-budget (0 + (?query + 0))
    (record-staged-message composition-final >>=
      (λ-. staged-query-program A 0 i >>=
        (λ query-chunks. return
          (dg, composition-final, query-chunks))))
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-record-staged-message, rule after-query)
have after-composition-final:
  ∧ dg bs. hash-range-budget
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg bs >>=
      (λ composition-final.
        record-staged-message composition-final >>=
          (λ-. staged-query-program A 0 i >>=
            (λ query-chunks. return
              (dg, composition-final, query-chunks))))))
  by (rule hash-range-budget-bind)
    (rule composition-final, rule after-composition-final-record)
have after-composition-fri:
  ∧ dg. hash-range-budget
    (?composition-fri + (?composition-final + (0 + (?query + 0))))
    ((assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [])) >>=
      (λ(-, composition-bs).
        composition-final-stage A dg composition-bs >>=
          (λ composition-final.
            record-staged-message composition-final >>=
              (λ-. staged-query-program A 0 i >>=
                (λ query-chunks. return
                  (dg, composition-final, query-chunks))))))

```



```

proof (rule hash-range-budget-bind)
  fix dg
  show hash-range-budget ?composition-fri
    (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
    by (rule composition-fri)
next
  fix dg x
  show hash-range-budget
    (?composition-final + (0 + (?query + 0)))
    (case x of (·, composition-bs) ⇒
      composition-final-stage A dg composition-bs >>=
      (λcomposition-final.
        record-staged-message composition-final >>=
        (λ-. staged-query-program A 0 i >>=
          (λquery-chunks. return
            (dg, composition-final, query-chunks)))))
proof (cases x)
  case (Pair composition-roots composition-bs)
  have step:
    hash-range-budget
      (?composition-final + (0 + (?query + 0)))
      (composition-final-stage A dg composition-bs >>=
        (λcomposition-final.
          record-staged-message composition-final >>=
          (λ-. staged-query-program A 0 i >>=
            (λquery-chunks. return
              (dg, composition-final, query-chunks)))))
    by (rule after-composition-final)
  show ?thesis
    unfolding Pair using step by simp
qed
qed
have after-degree-record:
  ∧ dg. hash-range-budget
    (0 + (?composition-fri + (?composition-final + (0 + (?query + 0)))))
    (record-staged-message dg >>=
      (λ-. (assert
        (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) [])) >>=
        (λ(·, composition-bs).
          composition-final-stage A dg composition-bs >>=
          (λcomposition-final.
            record-staged-message composition-final >>=
            (λ-. staged-query-program A 0 i >>=
              (λquery-chunks. return
                (dg, composition-final, query-chunks)))))))

```

```

by (rule hash-range-budget-bind)
  (rule hash-range-budget-record-staged-message,
    rule after-composition-fri)
have after-degree-record-expanded:
 $\wedge$  dg. hash-range-budget
  (0 + (?composition-fri + (?composition-final + (0 + (?query + 0)))))
  (record-staged-message dg  $\gg$ 
    ( $\lambda$ -. assert
      (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda$ -. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1))  $\square$   $\gg$ 
        ( $\lambda$ (-, composition-bs).
          composition-final-stage A dg composition-bs  $\gg$ 
          ( $\lambda$ composition-final.
            record-staged-message composition-final  $\gg$ 
            ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
              ( $\lambda$ query-chunks. return
                (dg, composition-final, query-chunks))))))))))
using after-degree-record
by (simp add: sm-bind-assoc split: prod.splits)
have after-degree:
 $\wedge$  as. hash-range-budget
  (?degree +
    (0 + (?composition-fri +
      (?composition-final + (0 + (?query + 0)))))
  (degree-stage A as  $\gg$ 
    ( $\lambda$  dg. record-staged-message dg  $\gg$ 
      ( $\lambda$ -. let composition-rounds = ceil-log (to-nat dg + 1) in
        assert (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
        ( $\lambda$ -. staged-composition-fri-program A dg 0
          composition-rounds  $\square$   $\gg$ 
          ( $\lambda$ (-, composition-bs).
            composition-final-stage A dg composition-bs  $\gg$ 
            ( $\lambda$ composition-final.
              record-staged-message composition-final  $\gg$ 
              ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
                ( $\lambda$ query-chunks. return
                  (dg, composition-final, query-chunks))))))))))
unfolding Let-def
by (rule hash-range-budget-bind)
  (rule degree, rule after-degree-record-expanded)
have after-alpha:
 $\wedge$  x. hash-range-budget
  (?alpha +
    (?degree +
      (0 + (?composition-fri +
        (?composition-final + (0 + (?query + 0)))))
  (staged-alpha-program (length spec)  $\gg$ 
    ( $\lambda$  as. degree-stage A as  $\gg$ 

```

```

(λdg. record-staged-message dg ≫=
  (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
    assert (composition-rounds ≤ ceil-log (maxDegree + 1)) ≫=
    (λ-. staged-composition-fri-program A dg 0
      composition-rounds [] ≫=
      (λ(-, composition-bs).
        composition-final-stage A dg composition-bs ≫=
        (λcomposition-final.
          record-staged-message composition-final ≫=
          (λ-. staged-query-program A 0 i ≫=
            (λquery-chunks. return
              (dg, composition-final, query-chunks))))))))))
by (rule hash-range-budget-bind)
  (rule alpha, rule after-degree)
have after-alpha-prefix:
  hash-range-budget
  (?alpha-prefix +
    (?alpha +
      (?degree +
        (0 + (?composition-fri +
          (?composition-final + (0 + (?query + 0))))))))
  (staged-query-challenge-prefix-program A i)
unfolding staged-query-challenge-prefix-program-def
proof (rule hash-range-budget-bind)
show hash-range-budget ?alpha-prefix
  (staged-alpha-prefix-program A)
  by (rule alpha-prefix)
next
fix x
show hash-range-budget
  (?alpha +
    (?degree +
      (0 + (?composition-fri +
        (?composition-final + (0 + (?query + 0)))))))
  (case x of (-, -, trace-bs, -) ⇒
    staged-alpha-program (length spec) ≫=
    (λas. degree-stage A as ≫=
      (λdg. record-staged-message dg ≫=
        (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
          assert (composition-rounds ≤ ceil-log (maxDegree + 1)) ≫=
          (λ-. staged-composition-fri-program A dg 0
            composition-rounds [] ≫=
            (λ(-, composition-bs).
              composition-final-stage A dg composition-bs ≫=
              (λcomposition-final.
                record-staged-message composition-final ≫=
                (λ-. staged-query-program A 0 i ≫=
                  (λquery-chunks. return
                    (dg, composition-final,

```

```

query-chunks))))))))))
proof (cases x)
  case (fields a b c d)
  have step:
    hash-range-budget
      (?alpha +
        (?degree +
          (0 + (?composition-fri +
            (?composition-final + (0 + (?query + 0)))))))
      (staged-alpha-program (length spec) >=>
        (λas. degree-stage A as >=>
          (λdg. record-staged-message dg >=>
            (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
              assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >=>
                (λ-. staged-composition-fri-program A dg 0
                  composition-rounds [] >=>
                    (λ(-, composition-bs).
                      composition-final-stage A dg composition-bs >=>
                        (λcomposition-final.
                          record-staged-message composition-final >=>
                            (λ-. staged-query-program A 0 i >=>
                              (λquery-chunks. return
                                (dg, composition-final,
                                  query-chunks))))))))))
          by (rule after-alpha)
        show ?thesis
          unfolding fields using step
          by (simp add: add.assoc add.commute add.left-commute)
      qed
    qed
  show ?thesis
    using after-alpha-prefix
    unfolding staged-query-search-queries-def staged-alpha-search-queries-def
    by (simp add: add.assoc add.commute add.left-commute)
qed

lemma hash-collision-budget-staged-query-challenge-prefix-program:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i ≤ rounds
  shows
    hash-collision-budget
      (staged-query-search-queries budgets i)
      (staged-query-challenge-prefix-program A i)
proof -
  let ?alpha-prefix = staged-alpha-search-queries budgets 0
  let ?alpha = length spec
  let ?degree = degree-budget budgets
  let ?composition-fri =

```

```

    sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
  let ?composition-final = composition-final-budget budgets
  let ?query = sum-list (take i (query-opening-budgets budgets)) + i
  have alpha-prefix-range:
    hash-range-budget ?alpha-prefix (staged-alpha-prefix-program A)
    by (rule hash-range-budget-staged-alpha-prefix-program
        [OF wf controlled])
  have alpha-prefix-coll:
    hash-collision-budget ?alpha-prefix (staged-alpha-prefix-program A)
    by (rule hash-collision-budget-staged-alpha-prefix-program
        [OF wf controlled])
  have alpha-range:
    hash-range-budget ?alpha (staged-alpha-program (length spec))
    by (rule hash-range-budget-staged-alpha-program)
  have alpha-coll:
    hash-collision-budget ?alpha (staged-alpha-program (length spec))
    by (rule hash-collision-budget-staged-alpha-program)
  have degree-range:
     $\wedge$  as. hash-range-budget ?degree (degree-stage A as)
    using controlled unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-range)
  have degree-coll:
     $\wedge$  as. hash-collision-budget ?degree (degree-stage A as)
    using controlled unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-collision)
  have composition-fri-range:
     $\wedge$  dg. hash-range-budget ?composition-fri
    (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
    ( $\lambda$ -. staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) []))
    by (rule hash-range-budget-guarded-staged-composition-fri-program
        [OF wf controlled])
  have composition-fri-coll:
     $\wedge$  dg. hash-collision-budget ?composition-fri
    (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
    ( $\lambda$ -. staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) []))
    by (rule hash-collision-budget-guarded-staged-composition-fri-program
        [OF wf controlled])
  have composition-final-range:
     $\wedge$  dg bs. hash-range-budget ?composition-final
    (composition-final-stage A dg bs)
    using controlled unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-range)
  have composition-final-coll:
     $\wedge$  dg bs. hash-collision-budget ?composition-final
    (composition-final-stage A dg bs)
    using controlled unfolding staged-adversary-controlled-def

```

```

  by (blast intro: controlled-ro-program-collision)
have query-range: hash-range-budget ?query (staged-query-program A 0 i)
proof -
  have len: 0 + i ≤ length (query-opening-budgets budgets)
    using wf i-bound unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-range-budget
      (sum-list (take i (drop 0 (query-opening-budgets budgets))) + i)
      (staged-query-program A 0 i)
    by (rule hash-range-budget-staged-query-program[OF controlled len])
  then show ?thesis by simp
qed
have query-coll: hash-collision-budget ?query (staged-query-program A 0 i)
proof -
  have len: 0 + i ≤ length (query-opening-budgets budgets)
    using wf i-bound unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-collision-budget
      (sum-list (take i (drop 0 (query-opening-budgets budgets))) + i)
      (staged-query-program A 0 i)
    by (rule hash-collision-budget-staged-query-program[OF controlled len])
  then show ?thesis by simp
qed
have after-query-range:
  ∧ dg composition-final. hash-range-budget (?query + 0)
    (staged-query-program A 0 i ≫=
      (λ query-chunks. return (dg, composition-final, query-chunks)))
  by (rule hash-range-budget-bind)
    (rule query-range, rule hash-range-budget-return)
have after-query-coll:
  ∧ dg composition-final. hash-collision-budget (?query + 0)
    (staged-query-program A 0 i ≫=
      (λ query-chunks. return (dg, composition-final, query-chunks)))
  by (rule hash-collision-budget-bind)
    (rule query-range, rule query-coll,
      rule hash-range-budget-return, rule hash-collision-budget-return)
have after-composition-final-record-range:
  ∧ dg composition-final. hash-range-budget (0 + (?query + 0))
    (record-staged-message composition-final ≫=
      (λ -. staged-query-program A 0 i ≫=
        (λ query-chunks. return
          (dg, composition-final, query-chunks))))
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-record-staged-message, rule after-query-range)
have after-composition-final-record-coll:
  ∧ dg composition-final. hash-collision-budget (0 + (?query + 0))
    (record-staged-message composition-final ≫=
      (λ -. staged-query-program A 0 i ≫=
        (λ query-chunks. return

```

```

      (dg, composition-final, query-chunks))))
  by (rule hash-collision-budget-bind)
    (rule hash-range-budget-record-staged-message,
      rule hash-collision-budget-record-staged-message,
      rule after-query-range, rule after-query-coll)
  have after-composition-final-range:
     $\wedge dg$  bs. hash-range-budget
      (?composition-final + (0 + (?query + 0)))
      (composition-final-stage A dg bs  $\gg$ 
        ( $\lambda$  composition-final.
          record-staged-message composition-final  $\gg$ 
            ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
              ( $\lambda$  query-chunks. return
                (dg, composition-final, query-chunks))))))
  by (rule hash-range-budget-bind)
    (rule composition-final-range,
      rule after-composition-final-record-range)
  have after-composition-final-coll:
     $\wedge dg$  bs. hash-collision-budget
      (?composition-final + (0 + (?query + 0)))
      (composition-final-stage A dg bs  $\gg$ 
        ( $\lambda$  composition-final.
          record-staged-message composition-final  $\gg$ 
            ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
              ( $\lambda$  query-chunks. return
                (dg, composition-final, query-chunks))))))
  by (rule hash-collision-budget-bind)
    (rule composition-final-range, rule composition-final-coll,
      rule after-composition-final-record-range,
      rule after-composition-final-record-coll)
  have after-composition-fri-range:
     $\wedge dg$ . hash-range-budget
      (?composition-fri + (?composition-final + (0 + (?query + 0))))
      ((assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
        ( $\lambda$ -. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []))  $\gg$ 
        ( $\lambda$ (-, composition-bs).
          composition-final-stage A dg composition-bs  $\gg$ 
            ( $\lambda$  composition-final.
              record-staged-message composition-final  $\gg$ 
                ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
                  ( $\lambda$  query-chunks. return
                    (dg, composition-final, query-chunks))))))
  proof (rule hash-range-budget-bind)
    fix dg
    show hash-range-budget ?composition-fri
      (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
        ( $\lambda$ -. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []))

```

```

    by (rule composition-fri-range)
next
fix dg x
show hash-range-budget
  (?composition-final + (0 + (?query + 0)))
  (case x of (_, composition-bs) ⇒
    composition-final-stage A dg composition-bs ≫=
    (λcomposition-final.
      record-staged-message composition-final ≫=
      (λ-. staged-query-program A 0 i ≫=
        (λquery-chunks. return
          (dg, composition-final, query-chunks))))))
proof (cases x)
case (Pair composition-roots composition-bs)
have step:
  hash-range-budget
  (?composition-final + (0 + (?query + 0)))
  (composition-final-stage A dg composition-bs ≫=
  (λcomposition-final.
    record-staged-message composition-final ≫=
    (λ-. staged-query-program A 0 i ≫=
      (λquery-chunks. return
        (dg, composition-final, query-chunks))))))
  by (rule after-composition-final-range)
show ?thesis
  unfolding Pair using step by simp
qed
qed
have after-composition-fri-coll:
  ∧ dg. hash-collision-budget
  (?composition-fri + (?composition-final + (0 + (?query + 0))))
  ((assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) ≫=
  (λ-. staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) [])) ≫=
  (λ(-, composition-bs).
    composition-final-stage A dg composition-bs ≫=
    (λcomposition-final.
      record-staged-message composition-final ≫=
      (λ-. staged-query-program A 0 i ≫=
        (λquery-chunks. return
          (dg, composition-final, query-chunks))))))
proof (rule hash-collision-budget-bind)
fix dg
show hash-range-budget ?composition-fri
  (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) ≫=
  (λ-. staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) []))
  by (rule composition-fri-range)
show hash-collision-budget ?composition-fri

```



```

(assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
  (λ-. staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) []))
by (rule composition-fri-coll)
next
fix dg x
show hash-range-budget
  (?composition-final + (0 + (?query + 0)))
  (case x of (-, composition-bs) ⇒
    composition-final-stage A dg composition-bs >>=
      (λcomposition-final.
        record-staged-message composition-final >>=
          (λ-. staged-query-program A 0 i >>=
            (λquery-chunks. return
              (dg, composition-final, query-chunks)))))
proof (cases x)
case (Pair composition-roots composition-bs)
have step:
  hash-range-budget
  (?composition-final + (0 + (?query + 0)))
  (composition-final-stage A dg composition-bs >>=
    (λcomposition-final.
      record-staged-message composition-final >>=
        (λ-. staged-query-program A 0 i >>=
          (λquery-chunks. return
            (dg, composition-final, query-chunks)))))
  by (rule after-composition-final-range)
show ?thesis
  unfolding Pair using step by simp
qed
show hash-collision-budget
  (?composition-final + (0 + (?query + 0)))
  (case x of (-, composition-bs) ⇒
    composition-final-stage A dg composition-bs >>=
      (λcomposition-final.
        record-staged-message composition-final >>=
          (λ-. staged-query-program A 0 i >>=
            (λquery-chunks. return
              (dg, composition-final, query-chunks)))))
proof (cases x)
case (Pair composition-roots composition-bs)
have step:
  hash-collision-budget
  (?composition-final + (0 + (?query + 0)))
  (composition-final-stage A dg composition-bs >>=
    (λcomposition-final.
      record-staged-message composition-final >>=
        (λ-. staged-query-program A 0 i >>=
          (λquery-chunks. return
            (dg, composition-final, query-chunks)))))

```

```

      (dg, composition-final, query-chunks))))))
    by (rule after-composition-final-coll)
  show ?thesis
    unfolding Pair using step by simp
qed
qed
have after-composition-fri-range-expanded:
   $\wedge dg.$  hash-range-budget
  ( $?composition-fri + (?composition-final + (0 + (?query + 0))))$ )
  (assert ( $\text{ceil-log } (to\text{-nat } dg + 1) \leq \text{ceil-log } (maxDegree + 1)$ )  $\gg=$ 
    ( $\lambda-. \text{ staged-composition-fri-program } A \ dg \ 0$ 
      ( $\text{ceil-log } (to\text{-nat } dg + 1)$ )  $\square \gg=$ 
      ( $\lambda(-, composition-bs).$ 
        composition-final-stage  $A \ dg \ composition-bs \gg=$ 
        ( $\lambda composition-final.$ 
          record-staged-message composition-final  $\gg=$ 
          ( $\lambda-. \text{ staged-query-program } A \ 0 \ i \gg=$ 
            ( $\lambda query-chunks. \text{ return }$ 
              (dg, composition-final, query-chunks))))))))
  using after-composition-fri-range
  by (simp add: sm-bind-assoc split: prod.splits)
have after-composition-fri-coll-expanded:
   $\wedge dg.$  hash-collision-budget
  ( $?composition-fri + (?composition-final + (0 + (?query + 0))))$ )
  (assert ( $\text{ceil-log } (to\text{-nat } dg + 1) \leq \text{ceil-log } (maxDegree + 1)$ )  $\gg=$ 
    ( $\lambda-. \text{ staged-composition-fri-program } A \ dg \ 0$ 
      ( $\text{ceil-log } (to\text{-nat } dg + 1)$ )  $\square \gg=$ 
      ( $\lambda(-, composition-bs).$ 
        composition-final-stage  $A \ dg \ composition-bs \gg=$ 
        ( $\lambda composition-final.$ 
          record-staged-message composition-final  $\gg=$ 
          ( $\lambda-. \text{ staged-query-program } A \ 0 \ i \gg=$ 
            ( $\lambda query-chunks. \text{ return }$ 
              (dg, composition-final, query-chunks))))))))
  using after-composition-fri-coll
  by (simp add: sm-bind-assoc split: prod.splits)
have after-degree-record-range:
   $\wedge dg.$  hash-range-budget
  ( $0 + (?composition-fri + (?composition-final + (0 + (?query + 0))))$ )
  (record-staged-message dg  $\gg=$ 
    ( $\lambda-. \text{ assert }$ 
      ( $\text{ceil-log } (to\text{-nat } dg + 1) \leq \text{ceil-log } (maxDegree + 1)$ )  $\gg=$ 
      ( $\lambda-. \text{ staged-composition-fri-program } A \ dg \ 0$ 
        ( $\text{ceil-log } (to\text{-nat } dg + 1)$ )  $\square \gg=$ 
        ( $\lambda(-, composition-bs).$ 
          composition-final-stage  $A \ dg \ composition-bs \gg=$ 
          ( $\lambda composition-final.$ 
            record-staged-message composition-final  $\gg=$ 
            ( $\lambda-. \text{ staged-query-program } A \ 0 \ i \gg=$ 
              (dg, composition-final, query-chunks))))))))

```

```

      (λquery-chunks. return
        (dg, composition-final, query-chunks)))))))))
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-record-staged-message,
      rule after-composition-fri-range-expanded)
  have after-degree-record-coll:
    ∧dg. hash-collision-budget
      (0 + (?composition-fri + (?composition-final + (0 + (?query + 0)))))
      (record-staged-message dg ≫=
        (λ-. assert
          (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) ≫=
            (λ-. staged-composition-fri-program A dg 0
              (ceil-log (to-nat dg + 1)) [] ≫=
                (λ(-, composition-bs).
                  composition-final-stage A dg composition-bs ≫=
                    (λcomposition-final.
                      record-staged-message composition-final ≫=
                        (λ-. staged-query-program A 0 i ≫=
                          (λquery-chunks. return
                            (dg, composition-final, query-chunks)))))))))
  by (rule hash-collision-budget-bind)
    (rule hash-range-budget-record-staged-message,
      rule hash-collision-budget-record-staged-message,
      rule after-composition-fri-range-expanded,
      rule after-composition-fri-coll-expanded)
  have after-degree-range:
    ∧as. hash-range-budget
      (?degree +
        (0 + (?composition-fri +
          (?composition-final + (0 + (?query + 0)))))
      (degree-stage A as ≫=
        (λdg. record-staged-message dg ≫=
          (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
            assert (composition-rounds ≤ ceil-log (maxDegree + 1)) ≫=
              (λ-. staged-composition-fri-program A dg 0
                composition-rounds [] ≫=
                  (λ(-, composition-bs).
                    composition-final-stage A dg composition-bs ≫=
                      (λcomposition-final.
                        record-staged-message composition-final ≫=
                          (λ-. staged-query-program A 0 i ≫=
                            (λquery-chunks. return
                              (dg, composition-final, query-chunks)))))))))
  unfolding Let-def
  by (rule hash-range-budget-bind)
    (rule degree-range, rule after-degree-record-range)
  have after-degree-coll:
    ∧as. hash-collision-budget
      (?degree +

```

```

(0 + (?composition-fri +
  (?composition-final + (0 + (?query + 0))))))
(degree-stage A as >>=
  (λdg. record-staged-message dg >>=
    (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
      assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
        (λ-. staged-composition-fri-program A dg 0
          composition-rounds [] >>=
            (λ(-, composition-bs).
              composition-final-stage A dg composition-bs >>=
                (λcomposition-final.
                  record-staged-message composition-final >>=
                    (λ-. staged-query-program A 0 i >>=
                      (λquery-chunks. return
                        (dg, composition-final, query-chunks))))))))))
unfolding Let-def
by (rule hash-collision-budget-bind)
  (rule degree-range, rule degree-coll,
    rule after-degree-record-range, rule after-degree-record-coll)
have after-alpha-range:
  ∧x. hash-range-budget
    (?alpha +
      (?degree +
        (0 + (?composition-fri +
          (?composition-final + (0 + (?query + 0)))))))
    (staged-alpha-program (length spec) >>=
      (λas. degree-stage A as >>=
        (λdg. record-staged-message dg >>=
          (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
            assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                composition-rounds [] >>=
                  (λ(-, composition-bs).
                    composition-final-stage A dg composition-bs >>=
                      (λcomposition-final.
                        record-staged-message composition-final >>=
                          (λ-. staged-query-program A 0 i >>=
                            (λquery-chunks. return
                              (dg, composition-final, query-chunks))))))))))
      by (rule hash-range-budget-bind)
        (rule alpha-range, rule after-degree-range)
have after-alpha-coll:
  ∧x. hash-collision-budget
    (?alpha +
      (?degree +
        (0 + (?composition-fri +
          (?composition-final + (0 + (?query + 0)))))))
    (staged-alpha-program (length spec) >>=
      (λas. degree-stage A as >>=

```

```

(λdg. record-staged-message dg ≫=
  (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
    assert (composition-rounds ≤ ceil-log (maxDegree + 1)) ≫=
    (λ-. staged-composition-fri-program A dg 0
      composition-rounds [] ≫=
      (λ(-, composition-bs).
        composition-final-stage A dg composition-bs ≫=
        (λcomposition-final.
          record-staged-message composition-final ≫=
          (λ-. staged-query-program A 0 i ≫=
            (λquery-chunks. return
              (dg, composition-final, query-chunks))))))))))
by (rule hash-collision-budget-bind)
  (rule alpha-range, rule alpha-coll,
    rule after-degree-range, rule after-degree-coll)
have after-alpha-prefix:
  hash-collision-budget
  (?alpha-prefix +
    (?alpha +
      (?degree +
        (0 + (?composition-fri +
          (?composition-final + (0 + (?query + 0)))))))
    (staged-query-challenge-prefix-program A i)
  unfolding staged-query-challenge-prefix-program-def
proof (rule hash-collision-budget-bind)
show hash-range-budget ?alpha-prefix
  (staged-alpha-prefix-program A)
by (rule alpha-prefix-range)
show hash-collision-budget ?alpha-prefix
  (staged-alpha-prefix-program A)
by (rule alpha-prefix-coll)
next
fix x
show hash-range-budget
  (?alpha +
    (?degree +
      (0 + (?composition-fri +
        (?composition-final + (0 + (?query + 0)))))))
  (case x of (-, -, trace-bs, -) ⇒
    staged-alpha-program (length spec) ≫=
    (λas. degree-stage A as ≫=
      (λdg. record-staged-message dg ≫=
        (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
          assert (composition-rounds ≤ ceil-log (maxDegree + 1)) ≫=
          (λ-. staged-composition-fri-program A dg 0
            composition-rounds [] ≫=
            (λ(-, composition-bs).
              composition-final-stage A dg composition-bs ≫=
              (λcomposition-final.

```

```

      record-staged-message composition-final >>=
      (λ-. staged-query-program A 0 i >>=
      (λquery-chunks. return
      (dg, composition-final,
      query-chunks)))))))))
proof (cases x)
  case (fields a b c d)
  show ?thesis
    unfolding fields using after-alpha-range
    by (simp add: add.assoc add.commute add.left-commute)
qed
show hash-collision-budget
  (?alpha +
  (?degree +
  (0 + (?composition-fri +
  (?composition-final + (0 + (?query + 0)))))))
(case x of (-, -, trace-bs, -) =>
  staged-alpha-program (length spec) >>=
  (λas. degree-stage A as >>=
  (λdg. record-staged-message dg >>=
  (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
  assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
  (λ-. staged-composition-fri-program A dg 0
  composition-rounds [] >>=
  (λ(-, composition-bs).
  composition-final-stage A dg composition-bs >>=
  (λcomposition-final.
  record-staged-message composition-final >>=
  (λ-. staged-query-program A 0 i >>=
  (λquery-chunks. return
  (dg, composition-final,
  query-chunks)))))))))
proof (cases x)
  case (fields a b c d)
  show ?thesis
    unfolding fields using after-alpha-coll
    by (simp add: add.assoc add.commute add.left-commute)
qed
qed
show ?thesis
  using after-alpha-prefix
  unfolding staged-query-search-queries-def staged-alpha-search-queries-def
  by (simp add: add.assoc add.commute add.left-commute)
qed

lemma staged-query-prefix-target-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i ≤ rounds

```

shows
 $wp\text{-}event\ (staged\text{-}query\text{-}challenge\text{-}prefix\text{-}program\ A\ i)$
 $(hash\text{-}new\text{-}output\text{-}hit\text{-}event\ B\ s)\ s \leq$
 $staged\text{-}phase\text{-}target\text{-}error\ B$
 $(staged\text{-}query\text{-}search\text{-}queries\ budgets\ i)$
proof –
have $budget:$
 $hash\text{-}target\text{-}budget\ B\ (staged\text{-}query\text{-}search\text{-}queries\ budgets\ i)$
 $(staged\text{-}query\text{-}challenge\text{-}prefix\text{-}program\ A\ i)$
using $hash\text{-}target\text{-}program\text{-}staged\text{-}query\text{-}challenge\text{-}prefix\text{-}program$
 $[OF\ wf\ controlled\ i\text{-}bound]$
unfolding $hash\text{-}target\text{-}program\text{-}def$ **by** $simp$
show $?thesis$
using $budget$ **unfolding** $hash\text{-}target\text{-}budget\text{-}def$
 $staged\text{-}phase\text{-}target\text{-}error\text{-}def$ **by** $blast$
qed

end

end

theory *Staged-Security-Experiment-Targets*
imports *Staged-Security-Experiment-Budgets*
begin

Bad-value target events and checked-prefix probability bounds.

context $soundness$
begin

definition $staged\text{-}query\text{-}prefix\text{-}bad\text{-}index\text{-}error$
 $:: staged\text{-}budgets \Rightarrow nat\ set \Rightarrow nat \Rightarrow prob$
where
 $staged\text{-}query\text{-}prefix\text{-}bad\text{-}index\text{-}error\ budgets\ B\ i =$
 $staged\text{-}phase\text{-}target\text{-}error\ (query\text{-}index\text{-}raw\text{-}preimage\ B)$
 $(staged\text{-}query\text{-}search\text{-}queries\ budgets\ i)$

definition $fri\text{-}vector\text{-}position\text{-}values :: 'f\ list\ set \Rightarrow nat \Rightarrow 'f\ set$
where
 $fri\text{-}vector\text{-}position\text{-}values\ B\ i =$
 $\{b. \exists bs \in B. i < length\ bs \wedge bs ! i = b\}$

lemma $fri\text{-}vector\text{-}position\text{-}valueI:$
assumes $bs \in B$
and $i < length\ bs$
shows $bs ! i \in fri\text{-}vector\text{-}position\text{-}values\ B\ i$
using $assms$ **unfolding** $fri\text{-}vector\text{-}position\text{-}values\text{-}def$ **by** $blast$

lemma $fri\text{-}vector\text{-}member\text{-}imp\text{-}position\text{-}value:$

assumes *subset*: $B \subseteq \text{fri-challenge-space } n$
and *member*: $bs \in B$
and *i-bound*: $i < n$
shows $bs ! i \in \text{fri-vector-position-values } B \ i$
proof –
have $\text{length } bs = n$
using *subset member unfolding fri-challenge-space-def* **by** *auto*
then have $i < \text{length } bs$
using *i-bound* **by** *simp*
then show *?thesis*
by (rule *fri-vector-position-valueI*[*OF member*])
qed

lemma *fri-vector-member-imp-first-position-value*:
assumes *subset*: $B \subseteq \text{fri-challenge-space } n$
and *member*: $bs \in B$
and *nonempty*: $0 < n$
shows $bs ! 0 \in \text{fri-vector-position-values } B \ 0$
by (rule *fri-vector-member-imp-position-value*[*OF subset member*])
(use nonempty in simp)

lemma *fri-vector-extend-member-imp-position-value*:
assumes *member*: $bs @ [b] \in B$
and *length*: $\text{length } bs = i$
shows $b \in \text{fri-vector-position-values } B \ i$
using *assms unfolding fri-vector-position-values-def* **by** *force*

lemma *fri-vector-position-values-card-le*:
assumes *subset*: $B \subseteq \text{fri-challenge-space } n$
and *i-bound*: $i < n$
shows $\text{card } (\text{fri-vector-position-values } B \ i) \leq \text{card } B$
proof –
have *finite-B*: *finite* B
by (rule *finite-subset*[*OF subset finite-fri-challenge-space*])
have *values-eq*:
 $\text{fri-vector-position-values } B \ i = (\lambda bs. bs ! i) \text{ ` } B$
using *subset i-bound*
unfolding *fri-vector-position-values-def fri-challenge-space-def*
by *auto*
show *?thesis*
unfolding *values-eq* **by** (rule *card-image-le*[*OF finite-B*])
qed

lemma *fri-vector-member-coordinate-cover*:
assumes *subset*: $B \subseteq \text{fri-challenge-space } n$
and *member*: $bs \in B$
and *nonempty*: $0 < n$
shows $\exists i < n. bs ! i \in \text{fri-vector-position-values } B \ i$
proof –

have $bs ! 0 \in \text{fri-vector-position-values } B \ 0$
by (rule *fri-vector-member-imp-first-position-value*
 $[OF \text{ subset member nonempty}]$)
then show *?thesis*
using *nonempty* **by** *blast*
qed

definition *trace-fri-challenge-values-absent*
 $:: 'f \text{ set} \Rightarrow ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{trace-fri-challenge-values-absent } B \ s \longleftrightarrow$
 $(\forall i \ x \ y.$
 $\text{fmlookup } (\text{HashMap } s) \ (\text{TraceFriChallenge } i \ x) = \text{Some } y \longrightarrow$
 $y \notin B)$

definition *composition-fri-challenge-values-absent*
 $:: 'f \text{ set} \Rightarrow ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{composition-fri-challenge-values-absent } B \ s \longleftrightarrow$
 $(\forall i \ x \ y.$
 $\text{fmlookup } (\text{HashMap } s) \ (\text{CompositionFriChallenge } i \ x) = \text{Some } y \longrightarrow$
 $y \notin B)$

definition *alpha-challenge-values-absent*
 $:: 'f \text{ set} \Rightarrow ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{alpha-challenge-values-absent } B \ s \longleftrightarrow$
 $(\forall i \ x \ y.$
 $\text{fmlookup } (\text{HashMap } s) \ (\text{AlphaChallenge } i \ x) = \text{Some } y \longrightarrow$
 $y \notin B)$

definition *query-index-raw-preimage-absent*
 $:: \text{nat set} \Rightarrow ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{query-index-raw-preimage-absent } B \ s \longleftrightarrow$
 $(\forall i \ x \ \text{raw}.$
 $\text{fmlookup } (\text{HashMap } s) \ (\text{QueryIndexChallenge } i \ x) = \text{Some } \text{raw} \longrightarrow$
 $\text{raw} \notin \text{query-index-raw-preimage } B)$

lemma *staged-query-prefix-bad-index-error-eq*:
 $\text{staged-query-prefix-bad-index-error budgets } B \ i =$
 $\text{hash-target-budget-value } (\text{query-index-raw-preimage } B)$
 $(\text{staged-query-search-queries budgets } i)$
unfolding *staged-query-prefix-bad-index-error-def*
 $\text{staged-phase-target-error-def}$ **by** *simp*

lemma *trace-fri-challenge-values-absent-empty[simp]*:
 $\text{trace-fri-challenge-values-absent } B$
 $(s[\![\text{HashMap} := \text{fmempty}]\!])$

unfolding *trace-fri-challenge-values-absent-def* **by** *simp*

lemma *composition-fri-challenge-values-absent-empty[simp]*:

composition-fri-challenge-values-absent *B*

(*s*(*HashMap* := *fmempty*))

unfolding *composition-fri-challenge-values-absent-def* **by** *simp*

lemma *alpha-challenge-values-absent-empty[simp]*:

alpha-challenge-values-absent *B*

(*s*(*HashMap* := *fmempty*))

unfolding *alpha-challenge-values-absent-def* **by** *simp*

lemma *staged-query-prefix-bad-index-target-bound*:

assumes *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets* *A*

and *i-bound*: $i \leq \text{rounds}$

shows

wp-event (*staged-query-challenge-prefix-program* *A* *i*)

(*hash-new-output-hit-event* (*query-index-raw-preimage* *B*) *s*) $s \leq$

staged-query-prefix-bad-index-error *budgets* *B* *i*

unfolding *staged-query-prefix-bad-index-error-def*

by (*rule* *staged-query-prefix-target-hit-bound*[*OF wf controlled i-bound*])

lemma *query-index-raw-preimage-absent-empty[simp]*:

query-index-raw-preimage-absent *B*

(*s*(*HashMap* := *fmempty*))

unfolding *query-index-raw-preimage-absent-def* **by** *simp*

lemma *staged-trace-fri-prefix-recv-bad-value-imp-new-output-hit*:

assumes *controlled*: *staged-adversary-controlled* *budgets* *A*

and *i-bound*: $i < \text{length}(\text{trace-fri-budgets } \text{budgets})$

and *absent*: *trace-fri-challenge-values-absent* *B* *s*

and *out*:

Some (*b*, *t*) $\in$

set-dist

(*execute*

(*staged-trace-fri-challenge-prefix-program* *A* *i* $\gg$

($\lambda\cdot. \text{receive-trace-fri-challenge}$) *s*)

and *bad*: $b \in B$

shows *hash-new-output-hit-event* *B* *s* (*Some* (*b*, *t*))

proof –

obtain *prefix-out* *u* **where** *prefix*:

Some (*prefix-out*, *u*) $\in$

set-dist

(*execute* (*staged-trace-fri-challenge-prefix-program* *A* *i*) *s*)

and *recv*:

Some (*b*, *t*) $\in \text{set-dist}(\text{execute } \text{receive-trace-fri-challenge } u)$

using *out* **by** (*auto elim!*: *set-dist-bindE*)

have *prefix-ext*: $s \leq u$

```

using hash-target-program-staged-trace-fri-challenge-prefix-program
  [OF controlled i-bound, of B] prefix
unfolding hash-target-program-def hash-extension-preserving-def
by blast
have recv-out:
   $u \leq t \wedge$ 
  fmlookup (HashMap t)
    (TraceFriChallenge (PTraceFriCounter u) (PState u)) = Some b
using receive-trace-fri-challenge-outcome[OF recv] by blast
show ?thesis
proof (cases
  fmlookup (HashMap s)
    (TraceFriChallenge (PTraceFriCounter u) (PState u)))
case None
then have hash-map-new-output-hit B s t
  unfolding hash-map-new-output-hit-def
  using recv-out bad by blast
then show ?thesis
  unfolding hash-new-output-hit-event-def by simp
next
case (Some old-b)
have old-lookup-t:
  fmlookup (HashMap t)
    (TraceFriChallenge (PTraceFriCounter u) (PState u)) = Some old-b
by (rule hash-extension-lookup[OF Some])
  (rule hash-ext-trans[OF prefix-ext conjunct1[OF recv-out]])
then have old-eq: old-b = b
  using recv-out by simp
have old-b  $\notin$  B
  using absent Some
  unfolding trace-fri-challenge-values-absent-def by blast
then show ?thesis
  using old-eq bad by simp
qed
qed

lemma staged-trace-fri-prefix-receive-bad-value-bound:
assumes controlled: staged-adversary-controlled budgets A
and i-bound:  $i < \text{length } (\text{trace-fri-budgets budgets})$ 
and absent: trace-fri-challenge-values-absent B s
shows
  wp-event
    (staged-trace-fri-challenge-prefix-program A i  $\gg$ 
      ( $\lambda$ -. receive-trace-fri-challenge))
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (b, -)  $\Rightarrow$   $b \in B$ ) s  $\leq$ 
    staged-phase-target-error B
      (Suc (staged-trace-fri-search-queries budgets i))
proof –

```

```

let ?m =
  staged-trace-fri-challenge-prefix-program A i >>=
    (λ-. receive-trace-fri-challenge)
let ?bad =
  λout. case out of None ⇒ False
    | Some (b, -) ⇒ b ∈ B
let ?hit = hash-new-output-hit-event B s
have bad-imp-hit:
  wp-event ?m ?bad s ≤ wp-event ?m ?hit s
proof (rule wp-event-mono-on-support)
  fix out
  assume support: out ∈ set-dist (execute ?m s)
  and bad: ?bad out
  show ?hit out
  proof (cases out)
    case None
    then show ?thesis using bad by simp
  next
    case (Some bt)
    then obtain b t where out-eq: out = Some (b, t)
      by (cases bt) simp
    have out':
      Some (b, t) ∈ set-dist (execute ?m s)
      using support out-eq by simp
    have b-in: b ∈ B
      using bad out-eq by simp
    show ?thesis
      unfolding out-eq
      by (rule staged-trace-fri-prefix-receive-bad-value-imp-new-output-hit
        [OF controlled i-bound absent out' b-in])
  qed
qed
have prefix-target:
  hash-target-program B
    (staged-trace-fri-search-queries budgets i)
    (staged-trace-fri-challenge-prefix-program A i)
  by (rule hash-target-program-staged-trace-fri-challenge-prefix-program
    [OF controlled i-bound])
have target:
  hash-target-program B
    (staged-trace-fri-search-queries budgets i + 1) ?m
  by (rule hash-target-program-bind[OF prefix-target])
    (rule hash-target-program-receive-trace-fri-challenge)
have hit-bound:
  wp-event ?m ?hit s ≤
    staged-phase-target-error B
      (staged-trace-fri-search-queries budgets i + 1)
  using target unfolding hash-target-program-def hash-target-budget-def
    staged-phase-target-error-def by blast

```

```

show ?thesis
  using order-trans[OF bad-imp-hit hit-bound] by simp
qed

lemma staged-composition-fri-prefix-recv-bad-value-imp-new-output-hit:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{ceil-log } (\text{maxDegree} + 1)$ 
  and absent: composition-fri-challenge-values-absent B s
  and out:
    Some (b, t) ∈
      set-dist
        (execute
          (staged-composition-fri-challenge-prefix-program A i  $\gg$ 
            ( $\lambda$ -. receive-composition-fri-challenge)) s)
  and bad:  $b \in B$ 
  shows hash-new-output-hit-event B s (Some (b, t))
proof –
  obtain prefix-out u where prefix:
    Some (prefix-out, u) ∈
      set-dist
        (execute (staged-composition-fri-challenge-prefix-program A i) s)
  and recv:
    Some (b, t) ∈
      set-dist (execute receive-composition-fri-challenge u)
  using out by (auto elim!: set-dist-bindE)
  have prefix-ext:  $s \leq u$ 
  using hash-target-program-staged-composition-fri-challenge-prefix-program
    [OF wf controlled i-bound, of B] prefix
  unfolding hash-target-program-def hash-extension-preserving-def
  by blast
  have recv-out:
     $u \leq t \wedge$ 
    fmlookup (HashMap t)
      (CompositionFriChallenge (PCompositionFriCounter u) (PState u)) =
      Some b
  using receive-composition-fri-challenge-outcome[OF recv] by blast
  show ?thesis
  proof (cases
    fmlookup (HashMap s)
      (CompositionFriChallenge (PCompositionFriCounter u) (PState u)))
  case None
  then have hash-map-new-output-hit B s t
  unfolding hash-map-new-output-hit-def
  using recv-out bad by blast
  then show ?thesis
  unfolding hash-new-output-hit-event-def by simp
next
  case (Some old-b)

```

```

have old-lookup-t:
  fmlookup (HashMap t)
    (CompositionFriChallenge (PCompositionFriCounter u) (PState u)) =
      Some old-b
  by (rule hash-extension-lookup[OF Some])
    (rule hash-ext-trans[OF prefix-ext conjunct1[OF recv-out]])
then have old-eq: old-b = b
  using recv-out by simp
have old-b  $\notin$  B
  using absent Some
  unfolding composition-fri-challenge-values-absent-def by blast
then show ?thesis
  using old-eq bad by simp
qed
qed

lemma staged-composition-fri-prefix-recv-bad-value-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < ceil-log (maxDegree + 1)
  and absent: composition-fri-challenge-values-absent B s
shows
  wp-event
    (staged-composition-fri-challenge-prefix-program A i  $\gg$ 
      ( $\lambda$ -. receive-composition-fri-challenge))
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (b, -)  $\Rightarrow$  b  $\in$  B) s  $\leq$ 
    staged-phase-target-error B
    (Suc (staged-composition-fri-search-queries budgets i))
proof -
  let ?m =
    staged-composition-fri-challenge-prefix-program A i  $\gg$ 
      ( $\lambda$ -. receive-composition-fri-challenge)
  let ?bad =
     $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (b, -)  $\Rightarrow$  b  $\in$  B
  let ?hit = hash-new-output-hit-event B s
  have bad-imp-hit:
    wp-event ?m ?bad s  $\leq$  wp-event ?m ?hit s
proof (rule wp-event-mono-on-support)
  fix out
  assume support: out  $\in$  set-dist (execute ?m s)
  and bad: ?bad out
  show ?hit out
  proof (cases out)
  case None
  then show ?thesis using bad by simp
next
  case (Some bt)

```

```

then obtain  $b\ t$  where  $out\text{-}eq: out = Some\ (b, t)$ 
  by ( $cases\ bt$ ) simp
have  $out'$ :
   $Some\ (b, t) \in set\text{-}dist\ (execute\ ?m\ s)$ 
  using  $support\ out\text{-}eq$  by simp
have  $b\text{-}in: b \in B$ 
  using  $bad\ out\text{-}eq$  by simp
show  $?thesis$ 
  unfolding  $out\text{-}eq$ 
  by (rule
     $staged\text{-}composition\text{-}fri\text{-}prefix\text{-}receive\text{-}bad\text{-}value\text{-}imp\text{-}new\text{-}output\text{-}hit$ 
     $[OF\ wf\ controlled\ i\text{-}bound\ absent\ out'\ b\text{-}in]$ )
qed
qed
have  $prefix\text{-}target$ :
   $hash\text{-}target\text{-}program\ B$ 
  ( $staged\text{-}composition\text{-}fri\text{-}search\text{-}queries\ budgets\ i$ )
  ( $staged\text{-}composition\text{-}fri\text{-}challenge\text{-}prefix\text{-}program\ A\ i$ )
  by (rule
     $hash\text{-}target\text{-}program\text{-}staged\text{-}composition\text{-}fri\text{-}challenge\text{-}prefix\text{-}program$ 
     $[OF\ wf\ controlled\ i\text{-}bound]$ )
have  $target$ :
   $hash\text{-}target\text{-}program\ B$ 
  ( $staged\text{-}composition\text{-}fri\text{-}search\text{-}queries\ budgets\ i + 1$ )  $?m$ 
  by (rule  $hash\text{-}target\text{-}program\text{-}bind\ [OF\ prefix\text{-}target]$ )
  (rule  $hash\text{-}target\text{-}program\text{-}receive\text{-}composition\text{-}fri\text{-}challenge$ )
have  $hit\text{-}bound$ :
   $wp\text{-}event\ ?m\ ?hit\ s \leq$ 
   $staged\text{-}phase\text{-}target\text{-}error\ B$ 
  ( $staged\text{-}composition\text{-}fri\text{-}search\text{-}queries\ budgets\ i + 1$ )
  using  $target$  unfolding  $hash\text{-}target\text{-}program\text{-}def\ hash\text{-}target\text{-}budget\text{-}def$ 
   $staged\text{-}phase\text{-}target\text{-}error\text{-}def$  by blast
show  $?thesis$ 
  using  $order\text{-}trans\ [OF\ bad\text{-}imp\text{-}hit\ hit\text{-}bound]$  by simp
qed

lemma  $staged\text{-}query\text{-}prefix\text{-}receive\text{-}bad\text{-}index\text{-}imp\text{-}new\text{-}output\text{-}hit$ :
assumes  $wf: staged\text{-}budget\text{-}wellformed\ budgets$ 
and  $controlled: staged\text{-}adversary\text{-}controlled\ budgets\ A$ 
and  $i\text{-}bound: i \leq rounds$ 
and  $absent: query\text{-}index\text{-}raw\text{-}preimage\text{-}absent\ B\ s$ 
and  $out$ :
   $Some\ (raw, t) \in$ 
   $set\text{-}dist$ 
  ( $execute$ 
    ( $staged\text{-}query\text{-}challenge\text{-}prefix\text{-}program\ A\ i \gg=$ 
      ( $\lambda\cdot. receive\text{-}query\text{-}index\text{-}challenge$ ))  $s$ )
and  $bad: index\ (to\text{-}nat\ raw) \in B$ 
shows

```

```

    hash-new-output-hit-event (query-index-raw-preimage B) s
      (Some (raw, t))
  proof -
    obtain prefix-out u where prefix:
      Some (prefix-out, u) ∈
        set-dist (execute (staged-query-challenge-prefix-program A i) s)
    and recv:
      Some (raw, t) ∈
        set-dist (execute receive-query-index-challenge u)
    using out by (auto elim!: set-dist-bindE)
  have prefix-ext:
    s ≤ u
  using hash-target-program-staged-query-challenge-prefix-program
    [OF wf controlled i-bound, of query-index-raw-preimage B]
    prefix
  unfolding hash-target-program-def hash-extension-preserving-def
  by blast
  have recv-out:
    u ≤ t ∧
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter u) (PState u)) = Some raw
  using receive-query-index-challenge-outcome[OF recv] by blast
  have raw-in: raw ∈ query-index-raw-preimage B
  using bad unfolding query-index-raw-preimage-def by simp
  show ?thesis
  proof (cases
    fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter u) (PState u)))
    case None
    then have hash-map-new-output-hit (query-index-raw-preimage B) s t
      unfolding hash-map-new-output-hit-def
      using recv-out raw-in by blast
    then show ?thesis
      unfolding hash-new-output-hit-event-def by simp
  next
    case (Some old-raw)
    have old-lookup-t:
      fmlookup (HashMap t)
        (QueryIndexChallenge (PQueryCounter u) (PState u)) =
        Some old-raw
    by (rule hash-extension-lookup[OF Some])
      (rule hash-ext-trans[OF prefix-ext conjunct1[OF recv-out]])
    then have old-eq: old-raw = raw
      using recv-out by simp
    have old-raw ∉ query-index-raw-preimage B
      using absent Some
      unfolding query-index-raw-preimage-absent-def by blast
    then show ?thesis
      using old-eq raw-in by simp
  end

```


qed
qed

lemma *staged-query-prefix-receive-bad-index-bound*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i \leq \text{rounds}$
and *absent*: *query-index-raw-preimage-absent B s*
shows
 wp-event
 $(\text{staged-query-challenge-prefix-program } A \ i \gg=$
 $(\lambda-. \text{receive-query-index-challenge}))$
 $(\lambda \text{out. case out of None} \Rightarrow \text{False}$
 $\mid \text{Some (raw, -)} \Rightarrow \text{index (to-nat raw)} \in B) \ s \leq$
 $\text{staged-phase-target-error (query-index-raw-preimage B)}$
 $(\text{Suc (staged-query-search-queries budgets i)})$

proof –
let *?m* =
 $\text{staged-query-challenge-prefix-program } A \ i \gg=$
 $(\lambda-. \text{receive-query-index-challenge})$
let *?B* = *query-index-raw-preimage B*
let *?bad* =
 $\lambda \text{out. case out of None} \Rightarrow \text{False}$
 $\mid \text{Some (raw, -)} \Rightarrow \text{index (to-nat raw)} \in B$
let *?hit* = *hash-new-output-hit-event ?B s*
have *bad-imp-hit*:
 $\text{wp-event } ?m \ ?bad \ s \leq \text{wp-event } ?m \ ?hit \ s$
proof (*rule wp-event-mono-on-support*)
fix *out*
assume *support*: $\text{out} \in \text{set-dist (execute ?m s)}$
and *bad*: *?bad out*
show *?hit out*
proof (*cases out*)
case *None*
then show *?thesis* **using** *bad* **by** *simp*
next
case (*Some rt*)
then obtain *raw t* **where** *out-eq*: $\text{out} = \text{Some (raw, t)}$
by (*cases rt*) *simp*
have *out'*:
 $\text{Some (raw, t)} \in \text{set-dist (execute ?m s)}$
using *support out-eq* **by** *simp*
have *bad-raw*: $\text{index (to-nat raw)} \in B$
using *bad out-eq* **by** *simp*
show *?thesis*
unfolding *out-eq*
by (*rule staged-query-prefix-receive-bad-index-imp-new-output-hit*
 $[\text{OF wf controlled i-bound absent out' bad-raw}]$)

qed

```

qed
have prefix-target:
  hash-target-program ?B
    (staged-query-search-queries budgets i)
    (staged-query-challenge-prefix-program A i)
  by (rule hash-target-program-staged-query-challenge-prefix-program
      [OF wf controlled i-bound])
have target:
  hash-target-program ?B
    (staged-query-search-queries budgets i + 1) ?m
  by (rule hash-target-program-bind[OF prefix-target])
    (rule hash-target-program-receive-query-index-challenge)
have hit-bound:
  wp-event ?m ?hit s ≤
    staged-phase-target-error ?B
    (staged-query-search-queries budgets i + 1)
  using target unfolding hash-target-program-def hash-target-budget-def
    staged-phase-target-error-def by blast
show ?thesis
  using order-trans[OF bad-imp-hit hit-bound] by simp
qed

lemma receive-alpha-challenge-bad-value-imp-new-output-hit:
  assumes absent: alpha-challenge-values-absent B s
  and out:
    Some (a, t) ∈ set-dist (execute receive-alpha-challenge s)
  and bad: a ∈ B
  shows hash-new-output-hit-event B s (Some (a, t))
proof -
  have recv-out:
    s ≤ t ∧
    fmlookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s) (PState s)) = Some a
  using receive-alpha-challenge-outcome[OF out] by blast
  show ?thesis
  proof (cases
    fmlookup (HashMap s)
      (AlphaChallenge (PAlphaCounter s) (PState s)))
  case None
  then have hash-map-new-output-hit B s t
    unfolding hash-map-new-output-hit-def
    using recv-out bad by blast
  then show ?thesis
    unfolding hash-new-output-hit-event-def by simp
  next
  case (Some old-a)
  have old-lookup-t:
    fmlookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s) (PState s)) = Some old-a

```

```

    by (rule hash-extension-lookup[OF Some conjunct1[OF recv-out]])
  then have old-eq: old-a = a
    using recv-out by simp
  have old-a  $\notin$  B
    using absent Some
    unfolding alpha-challenge-values-absent-def by blast
  then show ?thesis
    using old-eq bad by simp
qed
qed

lemma receive-alpha-challenge-bad-value-bound:
  assumes absent: alpha-challenge-values-absent B s
  shows
    wp-event receive-alpha-challenge
      ( $\lambda$ out. case out of None  $\Rightarrow$  False
        | Some (a, -)  $\Rightarrow$  a  $\in$  B) s  $\leq$ 
      staged-phase-target-error B 1
proof -
  let ?bad =
     $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (a, -)  $\Rightarrow$  a  $\in$  B
  let ?hit = hash-new-output-hit-event B s
  have bad-imp-hit:
    wp-event receive-alpha-challenge ?bad s  $\leq$ 
    wp-event receive-alpha-challenge ?hit s
proof (rule wp-event-mono-on-support)
  fix out
  assume support: out  $\in$  set-dist (execute receive-alpha-challenge s)
  and bad: ?bad out
  show ?hit out
proof (cases out)
  case None
  then show ?thesis using bad by simp
next
  case (Some pair)
  obtain a t where pair-eq: pair = (a, t)
    by (cases pair)
  have out-eq: out = Some (a, t)
    using Some pair-eq by simp
  have out':
    Some (a, t)  $\in$  set-dist (execute receive-alpha-challenge s)
    using support out-eq by simp
  have a-in: a  $\in$  B
    using bad out-eq by simp
  show ?thesis
    unfolding out-eq
    by (rule receive-alpha-challenge-bad-value-imp-new-output-hit
      [OF absent out' a-in])

```

```

qed
qed
have target:
  hash-target-program B 1 receive-alpha-challenge
  by (rule hash-target-program-receive-alpha-challenge)
have hit-bound:
  wp-event receive-alpha-challenge ?hit s ≤
    staged-phase-target-error B 1
  using target unfolding hash-target-program-def hash-target-budget-def
    staged-phase-target-error-def by blast
show ?thesis
  using order-trans[OF bad-imp-hit hit-bound] .
qed

```

lemma *staged-alpha-prefix-receive-bad-value-imp-new-output-hit*:

```

assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i ≤ length spec
  and absent: alpha-challenge-values-absent B s
  and out:
    Some (a, t) ∈
      set-dist
        (execute
          (staged-alpha-challenge-prefix-program A i >>=
            (λ-. receive-alpha-challenge)) s)
  and bad: a ∈ B

```

shows *hash-new-output-hit-event B s (Some (a, t))*

proof –

```

obtain prefix-out u where prefix:
  Some (prefix-out, u) ∈
    set-dist (execute (staged-alpha-challenge-prefix-program A i) s)
  and recv:
    Some (a, t) ∈ set-dist (execute receive-alpha-challenge u)
  using out by (auto elim!: set-dist-bindE)
have prefix-ext:
  s ≤ u
  using hash-target-program-staged-alpha-challenge-prefix-program
    [OF wf controlled i-bound, of B] prefix
  unfolding hash-target-program-def hash-extension-preserving-def
  by blast

```

have *recv-out*:

```

u ≤ t ∧
  fmlookup (HashMap t)
    (AlphaChallenge (PAlphaCounter u) (PState u)) = Some a
  using receive-alpha-challenge-outcome[OF recv] by blast

```

show *?thesis*

proof (cases

```

  fmlookup (HashMap s)
    (AlphaChallenge (PAlphaCounter u) (PState u)))

```

```

case None
then have hash-map-new-output-hit B s t
  unfolding hash-map-new-output-hit-def
  using recv-out bad by blast
then show ?thesis
  unfolding hash-new-output-hit-event-def by simp
next
case (Some old-a)
have old-lookup-t:
  fmlookup (HashMap t)
  (AlphaChallenge (PAlphaCounter u) (PState u)) = Some old-a
by (rule hash-extension-lookup[OF Some])
  (rule hash-ext-trans[OF prefix-ext conjunct1[OF recv-out]])
then have old-eq: old-a = a
  using recv-out by simp
have old-a  $\notin$  B
  using absent Some
  unfolding alpha-challenge-values-absent-def by blast
then show ?thesis
  using old-eq bad by simp
qed
qed

lemma staged-alpha-prefix-receive-bad-value-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i \leq \text{length spec}$ 
and absent: alpha-challenge-values-absent B s
shows
  wp-event
  (staged-alpha-challenge-prefix-program A i  $\gg$ 
    ( $\lambda\cdot$ . receive-alpha-challenge))
  ( $\lambda\text{out}$ . case out of None  $\Rightarrow$  False
    | Some (a,  $-$ )  $\Rightarrow$   $a \in B$ ) s  $\leq$ 
  staged-phase-target-error B
  (Suc (staged-alpha-search-queries budgets i))
proof -
let ?m =
  staged-alpha-challenge-prefix-program A i  $\gg$ 
  ( $\lambda\cdot$ . receive-alpha-challenge)
let ?bad =
   $\lambda\text{out}$ . case out of None  $\Rightarrow$  False
  | Some (a,  $-$ )  $\Rightarrow$   $a \in B$ 
let ?hit = hash-new-output-hit-event B s
have bad-imp-hit:
  wp-event ?m ?bad s  $\leq$  wp-event ?m ?hit s
proof (rule wp-event-mono-on-support)
fix out
assume support: out  $\in$  set-dist (execute ?m s)

```

```

    and bad: ?bad out
show ?hit out
proof (cases out)
  case None
  then show ?thesis using bad by simp
next
case (Some pair)
obtain a t where pair-eq: pair = (a, t)
  by (cases pair)
have out-eq: out = Some (a, t)
  using Some pair-eq by simp
have out':
  Some (a, t) ∈ set-dist (execute ?m s)
  using support out-eq by simp
have a-in: a ∈ B
  using bad out-eq by simp
show ?thesis
  unfolding out-eq
  by (rule staged-alpha-prefix-receive-bad-value-imp-new-output-hit
    [OF wf controlled i-bound absent out' a-in])
qed
qed
have prefix-target:
  hash-target-program B (staged-alpha-search-queries budgets i)
  (staged-alpha-challenge-prefix-program A i)
  by (rule hash-target-program-staged-alpha-challenge-prefix-program
    [OF wf controlled i-bound])
have target:
  hash-target-program B
  (staged-alpha-search-queries budgets i + 1) ?m
  by (rule hash-target-program-bind [OF prefix-target])
  (rule hash-target-program-receive-alpha-challenge)
have hit-bound:
  wp-event ?m ?hit s ≤
  staged-phase-target-error B
  (staged-alpha-search-queries budgets i + 1)
  using target unfolding hash-target-program-def hash-target-budget-def
  staged-phase-target-error-def by blast
show ?thesis
  using order-trans [OF bad-imp-hit hit-bound] by simp
qed

lemma hash-range-budget-staged-transcript-program:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    hash-range-budget
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (staged-transcript-program A)

```

```

proof –
  let ?trace-root = trace-root-budget budgets
  let ?trace-fri =
    sum-list (trace-fri-budgets budgets) + ceil-log clength
  let ?trace-final = trace-final-budget budgets
  let ?alpha = length spec
  let ?degree = degree-budget budgets
  let ?composition-fri =
    sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
  let ?composition-final = composition-final-budget budgets
  let ?query =
    sum-list (query-opening-budgets budgets) + rounds
  have trace-root-range:
    hash-range-budget ?trace-root (trace-root-stage A)
    using controlled unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-range)
  have trace-fri-range:
    hash-range-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
  proof –
    have len:
      0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
      using wf unfolding staged-budget-wellformed-def by simp
    have exact:
      hash-range-budget
        (sum-list
          (take (ceil-log clength)
            (drop 0 (trace-fri-budgets budgets)))) +
          ceil-log clength)
      (staged-trace-fri-program A 0 (ceil-log clength) [])
      by (rule hash-range-budget-staged-trace-fri-program
        [OF controlled len])
    then show ?thesis
      using wf unfolding staged-budget-wellformed-def by simp
  qed
  have trace-final-range:
    ∧bs. hash-range-budget ?trace-final (trace-final-stage A bs)
    using controlled unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-range)
  have alpha-range:
    hash-range-budget ?alpha (staged-alpha-program (length spec))
    by (rule hash-range-budget-staged-alpha-program)
  have degree-range:
    ∧as. hash-range-budget ?degree (degree-stage A as)
    using controlled unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-range)
  have composition-fri-range:
    ∧dg. hash-range-budget ?composition-fri

```

```

    (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
  by (rule hash-range-budget-guarded-staged-composition-fri-program
    [OF wf controlled])
have composition-final-range:
  ∧ dg bs. hash-range-budget ?composition-final
    (composition-final-stage A dg bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
have query-range:
  hash-range-budget ?query (staged-query-program A 0 rounds)
proof -
  have len:
    0 + rounds ≤ length (query-opening-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-range-budget
      (sum-list
        (take rounds (drop 0 (query-opening-budgets budgets))) +
        rounds)
      (staged-query-program A 0 rounds)
    by (rule hash-range-budget-staged-query-program[OF controlled len])
  then show ?thesis
    using wf unfolding staged-budget-wellformed-def by simp
qed
have query-tail:
  ∧ F. hash-range-budget (?query + 0)
    (staged-query-program A 0 rounds >>=
      (λ query-chunks. return (F query-chunks)))
  by (rule hash-range-budget-bind)
    (rule query-range, rule hash-range-budget-return)
have after-composition-final-record:
  ∧ composition-final F. hash-range-budget (0 + (?query + 0))
    (record-staged-message composition-final >>=
      (λ-. staged-query-program A 0 rounds >>=
        (λ query-chunks. return (F query-chunks))))
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-record-staged-message, rule query-tail)
have after-composition-final:
  ∧ dg bs F. hash-range-budget
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg bs >>=
      (λ composition-final.
        record-staged-message composition-final >>=
          (λ-. staged-query-program A 0 rounds >>=
            (λ query-chunks.
              return (F composition-final query-chunks))))))
  by (rule hash-range-budget-bind)

```



```

(rule composition-final-range, rule after-composition-final-record)
have after-composition-fri:
 $\wedge dg \ F. \text{hash-range-budget}$ 
  ( $?composition-fri +$ 
    ( $?composition-final + (0 + (?query + 0))$ ))
  ((assert ( $\text{ceil-log } (to-nat \ dg + 1) \leq$ 
     $\text{ceil-log } (maxDegree + 1)$ )  $\gg=$ 
    ( $\lambda-. \text{staged-composition-fri-program } A \ dg \ 0$ 
      ( $\text{ceil-log } (to-nat \ dg + 1)$ )  $\square$ ))  $\gg=$ 
    ( $\lambda(\text{composition-roots}, \text{composition-bs}).$ 
       $\text{composition-final-stage } A \ dg \ \text{composition-bs} \gg=$ 
      ( $\lambda\text{composition-final}.$ 
         $\text{record-staged-message } \text{composition-final} \gg=$ 
        ( $\lambda-. \text{staged-query-program } A \ 0 \ \text{rounds} \gg=$ 
          ( $\lambda\text{query-chunks}.$ 
            return
              ( $F \ \text{composition-roots} \ \text{composition-bs} \ \text{composition-final}$ 
                 $\text{query-chunks}$ ))))))
proof (rule hash-range-budget-bind)
  fix  $dg \ F$ 
  show hash-range-budget  $?composition-fri$ 
    (assert ( $\text{ceil-log } (to-nat \ dg + 1) \leq$ 
       $\text{ceil-log } (maxDegree + 1)$ )  $\gg=$ 
      ( $\lambda-. \text{staged-composition-fri-program } A \ dg \ 0$ 
        ( $\text{ceil-log } (to-nat \ dg + 1)$ )  $\square$ ))
    by (rule composition-fri-range)
  fix  $dg \ F \ x$ 
  show hash-range-budget
    ( $?composition-final + (0 + (?query + 0))$ )
    (case  $x$  of ( $\text{composition-roots}, \text{composition-bs}$ )  $\Rightarrow$ 
       $\text{composition-final-stage } A \ dg \ \text{composition-bs} \gg=$ 
      ( $\lambda\text{composition-final}.$ 
         $\text{record-staged-message } \text{composition-final} \gg=$ 
        ( $\lambda-. \text{staged-query-program } A \ 0 \ \text{rounds} \gg=$ 
          ( $\lambda\text{query-chunks}.$ 
            return
              ( $F \ \text{composition-roots} \ \text{composition-bs} \ \text{composition-final}$ 
                 $\text{query-chunks}$ ))))))
  proof (cases  $x$ )
  case ( $\text{Pair } \text{composition-roots} \ \text{composition-bs}$ )
  have hash-range-budget
    ( $?composition-final + (0 + (?query + 0))$ )
    ( $\text{composition-final-stage } A \ dg \ \text{composition-bs} \gg=$ 
      ( $\lambda\text{composition-final}.$ 
         $\text{record-staged-message } \text{composition-final} \gg=$ 
        ( $\lambda-. \text{staged-query-program } A \ 0 \ \text{rounds} \gg=$ 
          ( $\lambda\text{query-chunks}.$ 
            return
              ( $F \ \text{composition-roots} \ \text{composition-bs} \ \text{composition-final}$ 
                 $\text{query-chunks}$ ))))))

```

```

      query-chunks))))))
    by (rule after-composition-final)
  then show ?thesis
    unfolding Pair by simp
  qed
qed
have after-degree-record:
   $\bigwedge dg F.$  hash-range-budget
    (0 +
      (?composition-fri +
        (?composition-final + (0 + (?query + 0)))))
    (record-staged-message dg  $\gg$ 
      ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
        ceil-log (maxDegree + 1))  $\gg$ 
        ( $\lambda$ -. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []))  $\gg$ 
        ( $\lambda$ (composition-roots, composition-bs).
          composition-final-stage A dg composition-bs  $\gg$ 
            ( $\lambda$ composition-final.
              record-staged-message composition-final  $\gg$ 
                ( $\lambda$ -. staged-query-program A 0 rounds  $\gg$ 
                  ( $\lambda$ query-chunks.
                    return
                      (F composition-roots composition-bs composition-final
                        query-chunks)))))))
    by (rule hash-range-budget-bind)
      (rule hash-range-budget-record-staged-message, rule after-composition-fri)
  have after-degree:
     $\bigwedge as F.$  hash-range-budget
      (?degree +
        (0 +
          (?composition-fri +
            (?composition-final + (0 + (?query + 0)))))
        (degree-stage A as  $\gg$ 
          ( $\lambda dg.$  record-staged-message dg  $\gg$ 
            ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
              ceil-log (maxDegree + 1))  $\gg$ 
                ( $\lambda$ -. staged-composition-fri-program A dg 0
                  (ceil-log (to-nat dg + 1)) []))  $\gg$ 
                  ( $\lambda$ (composition-roots, composition-bs).
                    composition-final-stage A dg composition-bs  $\gg$ 
                      ( $\lambda$ composition-final.
                        record-staged-message composition-final  $\gg$ 
                          ( $\lambda$ -. staged-query-program A 0 rounds  $\gg$ 
                            ( $\lambda$ query-chunks.
                              return
                                (F dg composition-roots composition-bs
                                  composition-final query-chunks)))))))
          by (rule hash-range-budget-bind)

```

```

(rule degree-range, rule after-degree-record)
have after-alpha:
 $\wedge F$ . hash-range-budget
  (?alpha +
    (?degree +
      (0 +
        (?composition-fri +
          (?composition-final + (0 + (?query + 0)))))))
    (staged-alpha-program (length spec)  $\gg$ 
      ( $\lambda$ as. degree-stage A as  $\gg$ 
        ( $\lambda$ dg. record-staged-message dg  $\gg$ 
          ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
            ceil-log (maxDegree + 1))  $\gg$ 
            ( $\lambda$ -. staged-composition-fri-program A dg 0
              (ceil-log (to-nat dg + 1)) []))  $\gg$ 
            ( $\lambda$ (composition-roots, composition-bs).
              composition-final-stage A dg composition-bs  $\gg$ 
                ( $\lambda$ composition-final.
                  record-staged-message composition-final  $\gg$ 
                    ( $\lambda$ -. staged-query-program A 0 rounds  $\gg$ 
                      ( $\lambda$ query-chunks.
                        return
                          (F as dg composition-roots composition-bs
                            composition-final query-chunks))))))))
by (rule hash-range-budget-bind)
      (rule alpha-range, rule after-degree)
have after-trace-final-record:
 $\wedge$ trace-final F. hash-range-budget
  (0 +
    (?alpha +
      (?degree +
        (0 +
          (?composition-fri +
            (?composition-final + (0 + (?query + 0)))))))
    (record-staged-message trace-final  $\gg$ 
      ( $\lambda$ -. staged-alpha-program (length spec)  $\gg$ 
        ( $\lambda$ as. degree-stage A as  $\gg$ 
          ( $\lambda$ dg. record-staged-message dg  $\gg$ 
            ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
              ceil-log (maxDegree + 1))  $\gg$ 
              ( $\lambda$ -. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) []))  $\gg$ 
              ( $\lambda$ (composition-roots, composition-bs).
                composition-final-stage A dg composition-bs  $\gg$ 
                  ( $\lambda$ composition-final.
                    record-staged-message composition-final  $\gg$ 
                      ( $\lambda$ -. staged-query-program A 0 rounds  $\gg$ 
                        ( $\lambda$ query-chunks.
                          return

```

```

      (F as dg composition-roots composition-bs
        composition-final query-chunks)))))))))
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-record-staged-message, rule after-alpha)
  have after-trace-final:
     $\wedge$  trace-bs F. hash-range-budget
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))))
      (trace-final-stage A trace-bs  $\gg$ 
        ( $\lambda$  trace-final. record-staged-message trace-final  $\gg$ 
          ( $\lambda$ -. staged-alpha-program (length spec)  $\gg$ 
            ( $\lambda$  as. degree-stage A as  $\gg$ 
              ( $\lambda$  dg. record-staged-message dg  $\gg$ 
                ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
                  ceil-log (maxDegree + 1))  $\gg$ 
                  ( $\lambda$ -. staged-composition-fri-program A dg 0
                    (ceil-log (to-nat dg + 1)) []))  $\gg$ 
                  ( $\lambda$  (composition-roots, composition-bs).
                    composition-final-stage A dg composition-bs  $\gg$ 
                      ( $\lambda$  composition-final.
                        record-staged-message composition-final  $\gg$ 
                          ( $\lambda$ -. staged-query-program A 0 rounds  $\gg$ 
                            ( $\lambda$  query-chunks.
                              return
                                (F trace-final as dg composition-roots
                                  composition-bs composition-final
                                    query-chunks)))))))))
      by (rule hash-range-budget-bind)
        (rule trace-final-range, rule after-trace-final-record)
  have after-trace-fri:
     $\wedge$  F. hash-range-budget
      (?trace-fri +
        (?trace-final +
          (0 +
            (?alpha +
              (?degree +
                (0 +
                  (?composition-fri +
                    (?composition-final + (0 + (?query + 0))))))))))
      (staged-trace-fri-program A 0 (ceil-log clength) []  $\gg$ 
        ( $\lambda$  (trace-roots, trace-bs).
          trace-final-stage A trace-bs  $\gg$ 
            ( $\lambda$  trace-final. record-staged-message trace-final  $\gg$ 
              ( $\lambda$ -. staged-alpha-program (length spec)  $\gg$ 

```

```

(λas. degree-stage A as >>=
  (λdg. record-staged-message dg >>=
    (λ-. (assert (ceil-log (to-nat dg + 1) ≤
      ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [])) >>=
      (λ(composition-roots, composition-bs).
        composition-final-stage A dg composition-bs >>=
        (λcomposition-final.
          record-staged-message composition-final >>=
          (λ-. staged-query-program A 0 rounds >>=
            (λquery-chunks.
              return
                (F trace-roots trace-bs trace-final as dg
                  composition-roots composition-bs
                  composition-final query-chunks))))))))))
proof (rule hash-range-budget-bind)
fix F
show hash-range-budget ?trace-fri
  (staged-trace-fri-program A 0 (ceil-log clength) [])
by (rule trace-fri-range)
fix F x
show hash-range-budget
  (?trace-final +
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0))))))))))
  (case x of (trace-roots, trace-bs) ⇒
    trace-final-stage A trace-bs >>=
    (λtrace-final. record-staged-message trace-final >>=
      (λ-. staged-alpha-program (length spec) >>=
        (λas. degree-stage A as >>=
          (λdg. record-staged-message dg >>=
            (λ-. (assert (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) [])) >>=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs >>=
                (λcomposition-final.
                  record-staged-message composition-final >>=
                  (λ-. staged-query-program A 0 rounds >>=
                    (λquery-chunks.
                      return
                        (F trace-roots trace-bs trace-final as dg
                          composition-roots composition-bs

```

```

                                composition-final query-chunks))))))))))
proof (cases x)
  case (Pair trace-roots trace-bs)
  have hash-range-budget
    (?trace-final +
      (0 +
        (?alpha +
          (?degree +
            (0 +
              (?composition-fri +
                (composition-final + (0 + (?query + 0))))))))))
    (trace-final-stage A trace-bs >=>
      (λtrace-final. record-staged-message trace-final >=>
        (λ-. staged-alpha-program (length spec) >=>
          (λas. degree-stage A as >=>
            (λdg. record-staged-message dg >=>
              (λ-. (assert (ceil-log (to-nat dg + 1) ≤
                ceil-log (maxDegree + 1)) >=>
                (λ-. staged-composition-fri-program A dg 0
                  (ceil-log (to-nat dg + 1)) [])) >=>
                (λ(composition-roots, composition-bs).
                  composition-final-stage A dg composition-bs >=>
                    (λcomposition-final.
                      record-staged-message composition-final >=>
                        (λ-. staged-query-program A 0 rounds >=>
                          (λquery-chunks.
                            return
                              (F trace-roots trace-bs trace-final as dg
                                composition-roots composition-bs
                                composition-final query-chunks))))))))))
          by (rule after-trace-final)
        then show ?thesis
        unfolding Pair by simp
      qed
    qed
  have after-trace-root-record:
    ∧fr F. hash-range-budget
      (0 +
        (?trace-fri +
          (?trace-final +
            (0 +
              (?alpha +
                (?degree +
                  (0 +
                    (?composition-fri +
                      (composition-final + (0 + (?query + 0))))))))))
      (record-staged-message fr >=>
        (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >=>
          (λ(trace-roots, trace-bs).

```

```

    trace-final-stage A trace-bs >>=
    (λtrace-final. record-staged-message trace-final >>=
      (λ-. staged-alpha-program (length spec) >>=
        (λas. degree-stage A as >>=
          (λdg. record-staged-message dg >>=
            (λ-. (assert (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1))) >>=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) [])) >>=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs >>=
                (λcomposition-final.
                  record-staged-message composition-final >>=
                  (λ-. staged-query-program A 0 rounds >>=
                    (λquery-chunks.
                      return
                      (F trace-roots trace-bs trace-final as dg
                        composition-roots composition-bs
                        composition-final query-chunks))))))))))
  by (rule hash-range-budget-bind)
  (rule hash-range-budget-record-staged-message, rule after-trace-fri)
have whole:
  hash-range-budget
  (?trace-root +
    (0 +
      (?trace-fri +
        (?trace-final +
          (0 +
            (?alpha +
              (?degree +
                (0 +
                  (?composition-fri +
                    (?composition-final + (0 + (?query + 0))))))))))
    (staged-transcript-program A)
  unfolding staged-transcript-program-def Let-def
proof (rule hash-range-budget-bind)
show hash-range-budget ?trace-root (trace-root-stage A)
  by (rule trace-root-range)
fix fr
show hash-range-budget
  (0 +
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))))

```

```

(record-staged-message fr >>=
  (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
    (λ(trace-roots, trace-bs).
      trace-final-stage A trace-bs >>=
        (λtrace-final.
          record-staged-message trace-final >>=
            (λ-. staged-alpha-program (length spec) >>=
              (λas. degree-stage A as >>=
                (λdg. record-staged-message dg >>=
                  (λ-. assert (ceil-log (to-nat dg + 1) ≤
                    ceil-log (maxDegree + 1)) >>=
                    (λ-. staged-composition-fri-program A dg 0
                      (ceil-log (to-nat dg + 1)) [] >>=
                        (λ(composition-roots, composition-bs).
                          composition-final-stage A dg composition-bs >>=
                            (λcomposition-final.
                              record-staged-message
                                composition-final >>=
                                  (λ-. staged-query-program A 0
                                    rounds >>=
                                      (λquery-chunks.
                                        return
                                          (λ(staged-trace-root = fr,
                                            staged-trace-fri-roots =
                                              trace-roots,
                                            staged-trace-fri-challenges =
                                              trace-bs,
                                            staged-trace-final =
                                              trace-final,
                                            staged-alphas = as,
                                            staged-degree = dg,
                                            staged-composition-fri-roots =
                                              composition-roots,
                                            staged-composition-fri-challenges =
                                              composition-bs,
                                            staged-composition-final =
                                              composition-final,
                                            staged-query-chunks =
                                              query-chunks))))))))))))))
using after-trace-root-record
[ of fr
  λtrace-roots trace-bs trace-final as dg composition-roots
    composition-bs composition-final query-chunks.
    (λ(staged-trace-root = fr,
      staged-trace-fri-roots = trace-roots,
      staged-trace-fri-challenges = trace-bs,
      staged-trace-final = trace-final,
      staged-alphas = as,
      staged-degree = dg,

```



```

    staged-composition-fri-roots = composition-roots,
    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks))]]
  by (simp add: sm-bind-assoc)
qed
have budget-eq:
  ?trace-root +
  (0 +
  (?trace-fri +
  (?trace-final +
  (0 +
  (?alpha +
  (?degree +
  (0 +
  (?composition-fri +
  (?composition-final + (0 + (?query + 0)))))))))) =
  staged-attacker-query-budget budgets + staged-challenge-query-budget
unfolding staged-attacker-query-budget-def
  staged-challenge-query-budget-def
  by (simp add: add.assoc add.commute add.left-commute)
show ?thesis
  using whole unfolding budget-eq .
qed

lemma hash-range-budget-staged-semantic-adversary:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    hash-range-budget
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
      (staged-semantic-adversary A)
proof -
  have hash-range-budget
    ((staged-attacker-query-budget budgets + staged-challenge-query-budget) +
    0)
    (staged-semantic-adversary A)
  unfolding staged-semantic-adversary-def
proof (rule hash-range-budget-bind
  [where n=staged-attacker-query-budget budgets +
    staged-challenge-query-budget and n'=0])
  show hash-range-budget
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (staged-transcript-program A)
  by (rule hash-range-budget-staged-transcript-program[OF wf controlled])
  show  $\bigwedge x.$  hash-range-budget 0 (return (staged-proof-transcript x))
  by (rule hash-range-budget-return)
qed
then show ?thesis by simp

```

qed

lemma *hash-collision-budget-staged-transcript-program:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

hash-collision-budget

(staged-attacker-query-budget budgets + staged-challenge-query-budget)

(staged-transcript-program A)

proof –

let *?trace-root = trace-root-budget budgets*

let *?trace-fri =*

sum-list (trace-fri-budgets budgets) + ceil-log clength

let *?trace-final = trace-final-budget budgets*

let *?alpha = length spec*

let *?degree = degree-budget budgets*

let *?composition-fri =*

sum-list (composition-fri-budgets budgets) +

ceil-log (maxDegree + 1)

let *?composition-final = composition-final-budget budgets*

let *?query =*

sum-list (query-opening-budgets budgets) + rounds

have *trace-root-range:*

hash-range-budget ?trace-root (trace-root-stage A)

using *controlled unfolding staged-adversary-controlled-def*

by *(blast intro: controlled-ro-program-range)*

have *trace-root-coll:*

hash-collision-budget ?trace-root (trace-root-stage A)

using *controlled unfolding staged-adversary-controlled-def*

by *(blast intro: controlled-ro-program-collision)*

have *trace-fri-range:*

hash-range-budget ?trace-fri

(staged-trace-fri-program A 0 (ceil-log clength) [])

proof –

have *len:*

0 + ceil-log clength ≤ length (trace-fri-budgets budgets)

using *wf unfolding staged-budget-wellformed-def by simp*

have *exact:*

hash-range-budget

(sum-list

(take (ceil-log clength)

(drop 0 (trace-fri-budgets budgets))) +

ceil-log clength)

(staged-trace-fri-program A 0 (ceil-log clength) [])

by *(rule hash-range-budget-staged-trace-fri-program*

[OF controlled len])

then show *?thesis*

using *wf unfolding staged-budget-wellformed-def by simp*

qed

```

have trace-fri-coll:
  hash-collision-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
proof –
  have len:
    0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-collision-budget
      (sum-list
        (take (ceil-log clength)
          (drop 0 (trace-fri-budgets budgets))) +
        ceil-log clength)
      (staged-trace-fri-program A 0 (ceil-log clength) [])
    by (rule hash-collision-budget-staged-trace-fri-program
      [OF controlled len])
  then show ?thesis
    using wf unfolding staged-budget-wellformed-def by simp
qed
have trace-final-range:
  ∧bs. hash-range-budget ?trace-final (trace-final-stage A bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
have trace-final-coll:
  ∧bs. hash-collision-budget ?trace-final (trace-final-stage A bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-collision)
have alpha-range:
  hash-range-budget ?alpha (staged-alpha-program (length spec))
  by (rule hash-range-budget-staged-alpha-program)
have alpha-coll:
  hash-collision-budget ?alpha (staged-alpha-program (length spec))
  by (rule hash-collision-budget-staged-alpha-program)
have degree-range:
  ∧as. hash-range-budget ?degree (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
have degree-coll:
  ∧as. hash-collision-budget ?degree (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-collision)
have composition-fri-range:
  ∧dg. hash-range-budget ?composition-fri
    (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
  by (rule hash-range-budget-guarded-staged-composition-fri-program
    [OF wf controlled])
have composition-fri-coll:

```

```

 $\wedge dg.$  hash-collision-budget ?composition-fri
  (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
    ( $\lambda$ -. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) []))
  by (rule hash-collision-budget-guarded-staged-composition-fri-program
    [OF wf controlled])
have composition-final-range:
 $\wedge dg$  bs. hash-range-budget ?composition-final
  (composition-final-stage A dg bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
have composition-final-coll:
 $\wedge dg$  bs. hash-collision-budget ?composition-final
  (composition-final-stage A dg bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-collision)
have query-range:
  hash-range-budget ?query (staged-query-program A 0 rounds)
proof -
  have len:
    0 + rounds  $\leq$  length (query-opening-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-range-budget
      (sum-list
        (take rounds (drop 0 (query-opening-budgets budgets))) +
        rounds)
      (staged-query-program A 0 rounds)
    by (rule hash-range-budget-staged-query-program[OF controlled len])
  then show ?thesis
    using wf unfolding staged-budget-wellformed-def by simp
qed
have query-coll:
  hash-collision-budget ?query (staged-query-program A 0 rounds)
proof -
  have len:
    0 + rounds  $\leq$  length (query-opening-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-collision-budget
      (sum-list
        (take rounds (drop 0 (query-opening-budgets budgets))) +
        rounds)
      (staged-query-program A 0 rounds)
    by (rule hash-collision-budget-staged-query-program[OF controlled len])
  then show ?thesis
    using wf unfolding staged-budget-wellformed-def by simp
qed
have query-tail-range:

```

```

 $\wedge F.$  hash-range-budget ( $?query + 0$ )
  (staged-query-program  $A$  0 rounds  $\gg$ 
    ( $\lambda query-chunks.$  return ( $F$  query-chunks)))
  by (rule hash-range-budget-bind)
    (rule query-range, rule hash-range-budget-return)
have query-tail-coll:
 $\wedge F.$  hash-collision-budget ( $?query + 0$ )
  (staged-query-program  $A$  0 rounds  $\gg$ 
    ( $\lambda query-chunks.$  return ( $F$  query-chunks)))
  by (rule hash-collision-budget-bind)
    (rule query-range, rule query-coll, rule hash-range-budget-return,
      rule hash-collision-budget-return)
have after-composition-final-record-range:
 $\wedge$  composition-final  $F.$  hash-range-budget ( $0 + (?query + 0)$ )
  (record-staged-message composition-final  $\gg$ 
    ( $\lambda.$  staged-query-program  $A$  0 rounds  $\gg$ 
      ( $\lambda query-chunks.$  return ( $F$  query-chunks))))
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-record-staged-message, rule query-tail-range)
have after-composition-final-record-coll:
 $\wedge$  composition-final  $F.$  hash-collision-budget ( $0 + (?query + 0)$ )
  (record-staged-message composition-final  $\gg$ 
    ( $\lambda.$  staged-query-program  $A$  0 rounds  $\gg$ 
      ( $\lambda query-chunks.$  return ( $F$  query-chunks))))
  by (rule hash-collision-budget-bind)
    (rule hash-range-budget-record-staged-message,
      rule hash-collision-budget-record-staged-message,
      rule query-tail-range, rule query-tail-coll)
have after-composition-final-range:
 $\wedge dg$  bs  $F.$  hash-range-budget
  ( $?composition-final + (0 + (?query + 0))$ )
  (composition-final-stage  $A$  dg bs  $\gg$ 
    ( $\lambda$  composition-final.
      record-staged-message composition-final  $\gg$ 
        ( $\lambda.$  staged-query-program  $A$  0 rounds  $\gg$ 
          ( $\lambda query-chunks.$ 
            return ( $F$  composition-final query-chunks))))))
  by (rule hash-range-budget-bind)
    (rule composition-final-range,
      rule after-composition-final-record-range)
have after-composition-final-coll:
 $\wedge dg$  bs  $F.$  hash-collision-budget
  ( $?composition-final + (0 + (?query + 0))$ )
  (composition-final-stage  $A$  dg bs  $\gg$ 
    ( $\lambda$  composition-final.
      record-staged-message composition-final  $\gg$ 
        ( $\lambda.$  staged-query-program  $A$  0 rounds  $\gg$ 
          ( $\lambda query-chunks.$ 
            return ( $F$  composition-final query-chunks))))))

```

```

by (rule hash-collision-budget-bind)
  (rule composition-final-range, rule composition-final-coll,
   rule after-composition-final-record-range,
   rule after-composition-final-record-coll)
have after-composition-fri-range:
 $\wedge dg F$ . hash-range-budget
  (?composition-fri +
   (?composition-final + (0 + (?query + 0))))
  ((assert (ceil-log (to-nat dg + 1)  $\leq$ 
    ceil-log (maxDegree + 1))  $\gg=$ 
   ( $\lambda$ -. staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) []))  $\gg=$ 
   ( $\lambda$ (composition-roots, composition-bs).
    composition-final-stage A dg composition-bs  $\gg=$ 
    ( $\lambda$ composition-final.
     record-staged-message composition-final  $\gg=$ 
     ( $\lambda$ -. staged-query-program A 0 rounds  $\gg=$ 
      ( $\lambda$ query-chunks.
       return
        (F composition-roots composition-bs composition-final
         query-chunks))))))
proof (rule hash-range-budget-bind)
fix dg F
show hash-range-budget ?composition-fri
  (assert (ceil-log (to-nat dg + 1)  $\leq$ 
    ceil-log (maxDegree + 1))  $\gg=$ 
   ( $\lambda$ -. staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) []))
by (rule composition-fri-range)
fix dg F x
show hash-range-budget
  (?composition-final + (0 + (?query + 0)))
  (case x of (composition-roots, composition-bs)  $\Rightarrow$ 
   composition-final-stage A dg composition-bs  $\gg=$ 
   ( $\lambda$ composition-final.
    record-staged-message composition-final  $\gg=$ 
    ( $\lambda$ -. staged-query-program A 0 rounds  $\gg=$ 
     ( $\lambda$ query-chunks.
      return
       (F composition-roots composition-bs composition-final
        query-chunks))))))
proof (cases x)
case (Pair composition-roots composition-bs)
have hash-range-budget
  (?composition-final + (0 + (?query + 0)))
  (composition-final-stage A dg composition-bs  $\gg=$ 
   ( $\lambda$ composition-final.
    record-staged-message composition-final  $\gg=$ 
    ( $\lambda$ -. staged-query-program A 0 rounds  $\gg=$ 
     ( $\lambda$ query-chunks.
      return
       (F composition-roots composition-bs composition-final
        query-chunks))))))

```

```

      (λquery-chunks.
        return
          (F composition-roots composition-bs composition-final
            query-chunks))))))
    by (rule after-composition-final-range)
  then show ?thesis
    unfolding Pair by simp
qed
qed
have after-composition-fri-coll:
  ∧dg F. hash-collision-budget
    (?composition-fri +
      (?composition-final + (0 + (?query + 0))))
    ((assert (ceil-log (to-nat dg + 1) ≤
      ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [])) >>=
      (λ(composition-roots, composition-bs).
        composition-final-stage A dg composition-bs >>=
          (λcomposition-final.
            record-staged-message composition-final >>=
              (λ-. staged-query-program A 0 rounds >>=
                (λquery-chunks.
                  return
                    (F composition-roots composition-bs composition-final
                      query-chunks)))))))
proof (rule hash-collision-budget-bind)
  fix dg F
  show hash-range-budget ?composition-fri
    (assert (ceil-log (to-nat dg + 1) ≤
      ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
    by (rule composition-fri-range)
  fix dg F
  show hash-collision-budget ?composition-fri
    (assert (ceil-log (to-nat dg + 1) ≤
      ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
    by (rule composition-fri-coll)
  fix dg F x
  show hash-range-budget
    (?composition-final + (0 + (?query + 0)))
    (case x of (composition-roots, composition-bs) ⇒
      composition-final-stage A dg composition-bs >>=
        (λcomposition-final.
          record-staged-message composition-final >>=
            (λ-. staged-query-program A 0 rounds >>=
              (λquery-chunks.
                return
                  (F composition-roots composition-bs composition-final
                    query-chunks))))))

```

```

      (λquery-chunks.
        return
          (F composition-roots composition-bs composition-final
            query-chunks))))))
proof (cases x)
  case (Pair composition-roots composition-bs)
  have hash-range-budget
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg composition-bs ≫=
      (λcomposition-final.
        record-staged-message composition-final ≫=
          (λ-. staged-query-program A 0 rounds ≫=
            (λquery-chunks.
              return
                (F composition-roots composition-bs composition-final
                  query-chunks))))))
  by (rule after-composition-final-range)
  then show ?thesis
    unfolding Pair by simp
  qed
fix dg F x
show hash-collision-budget
  (?composition-final + (0 + (?query + 0)))
  (case x of (composition-roots, composition-bs) ⇒
    composition-final-stage A dg composition-bs ≫=
      (λcomposition-final.
        record-staged-message composition-final ≫=
          (λ-. staged-query-program A 0 rounds ≫=
            (λquery-chunks.
              return
                (F composition-roots composition-bs composition-final
                  query-chunks))))))
proof (cases x)
  case (Pair composition-roots composition-bs)
  have hash-collision-budget
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg composition-bs ≫=
      (λcomposition-final.
        record-staged-message composition-final ≫=
          (λ-. staged-query-program A 0 rounds ≫=
            (λquery-chunks.
              return
                (F composition-roots composition-bs composition-final
                  query-chunks))))))
  by (rule after-composition-final-coll)
  then show ?thesis
    unfolding Pair by simp
  qed
qed

```



```

have after-degree-record-range:
   $\wedge dg F.$  hash-range-budget
    (0 +
      (?composition-fri +
        (?composition-final + (0 + (?query + 0))))))
    (record-staged-message dg  $\gg$ 
      ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
        ceil-log (maxDegree + 1))  $\gg$ 
        ( $\lambda$ -. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []))  $\gg$ 
        ( $\lambda$ (composition-roots, composition-bs).
          composition-final-stage A dg composition-bs  $\gg$ 
            ( $\lambda$ composition-final.
              record-staged-message composition-final  $\gg$ 
                ( $\lambda$ -. staged-query-program A 0 rounds  $\gg$ 
                  ( $\lambda$ query-chunks.
                    return
                      (F composition-roots composition-bs composition-final
                        query-chunks)))))))
    by (rule hash-range-budget-bind)
      (rule hash-range-budget-record-staged-message,
        rule after-composition-fri-range)
have after-degree-record-coll:
   $\wedge dg F.$  hash-collision-budget
    (0 +
      (?composition-fri +
        (?composition-final + (0 + (?query + 0))))))
    (record-staged-message dg  $\gg$ 
      ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
        ceil-log (maxDegree + 1))  $\gg$ 
        ( $\lambda$ -. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []))  $\gg$ 
        ( $\lambda$ (composition-roots, composition-bs).
          composition-final-stage A dg composition-bs  $\gg$ 
            ( $\lambda$ composition-final.
              record-staged-message composition-final  $\gg$ 
                ( $\lambda$ -. staged-query-program A 0 rounds  $\gg$ 
                  ( $\lambda$ query-chunks.
                    return
                      (F composition-roots composition-bs composition-final
                        query-chunks)))))))
    by (rule hash-collision-budget-bind)
      (rule hash-range-budget-record-staged-message,
        rule hash-collision-budget-record-staged-message,
        rule after-composition-fri-range, rule after-composition-fri-coll)
have after-degree-range:
   $\wedge as F.$  hash-range-budget
    (?degree +
      (0 +

```

```

    (?composition-fri +
      (?composition-final + (0 + (?query + 0))))))
  (degree-stage A as >>=
    (λdg. record-staged-message dg >>=
      (λ-. (assert (ceil-log (to-nat dg + 1) ≤
        ceil-log (maxDegree + 1)) >>=
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) [])) >>=
        (λ(composition-roots, composition-bs).
          composition-final-stage A dg composition-bs >>=
            (λcomposition-final.
              record-staged-message composition-final >>=
                (λ-. staged-query-program A 0 rounds >>=
                  (λquery-chunks.
                    return
                      (F dg composition-roots composition-bs
                        composition-final query-chunks))))))))
  by (rule hash-range-budget-bind)
    (rule degree-range, rule after-degree-record-range)
have after-degree-coll:
  ∧as F. hash-collision-budget
    (?degree +
      (0 +
        (?composition-fri +
          (?composition-final + (0 + (?query + 0))))))
    (degree-stage A as >>=
      (λdg. record-staged-message dg >>=
        (λ-. (assert (ceil-log (to-nat dg + 1) ≤
          ceil-log (maxDegree + 1)) >>=
            (λ-. staged-composition-fri-program A dg 0
              (ceil-log (to-nat dg + 1)) [])) >>=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs >>=
                  (λcomposition-final.
                    record-staged-message composition-final >>=
                      (λ-. staged-query-program A 0 rounds >>=
                        (λquery-chunks.
                          return
                            (F dg composition-roots composition-bs
                              composition-final query-chunks))))))))
    by (rule hash-collision-budget-bind)
      (rule degree-range, rule degree-coll, rule after-degree-record-range,
        rule after-degree-record-coll)
have after-alpha-range:
  ∧F. hash-range-budget
    (?alpha +
      (?degree +
        (0 +
          (?composition-fri +
            (?composition-final + (0 + (?query + 0))))))
        (degree-stage A as >>=
          (λdg. record-staged-message dg >>=
            (λ-. (assert (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1)) >>=
                (λ-. staged-composition-fri-program A dg 0
                  (ceil-log (to-nat dg + 1)) [])) >>=
                  (λ(composition-roots, composition-bs).
                    composition-final-stage A dg composition-bs >>=
                      (λcomposition-final.
                        record-staged-message composition-final >>=
                          (λ-. staged-query-program A 0 rounds >>=
                            (λquery-chunks.
                              return
                                (F dg composition-roots composition-bs
                                  composition-final query-chunks))))))))
            by (rule hash-collision-budget-bind)
              (rule degree-range, rule degree-coll, rule after-degree-record-range,
                rule after-degree-record-coll)
          )
        )
      )
    )
  )

```

```

      (?composition-final + (0 + (?query + 0))))))
(staged-alpha-program (length spec) >>=
  (λas. degree-stage A as >>=
    (λdg. record-staged-message dg >>=
      (λ-. (assert (ceil-log (to-nat dg + 1) ≤
        ceil-log (maxDegree + 1)) >>=
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) [])) >>=
        (λ(composition-roots, composition-bs).
          composition-final-stage A dg composition-bs >>=
            (λcomposition-final.
              record-staged-message composition-final >>=
                (λ-. staged-query-program A 0 rounds >>=
                  (λquery-chunks.
                    return
                      (F as dg composition-roots composition-bs
                        composition-final query-chunks))))))))))
by (rule hash-range-budget-bind)
  (rule alpha-range, rule after-degree-range)
have after-alpha-coll:
  ∧F. hash-collision-budget
    (?alpha +
      (?degree +
        (0 +
          (?composition-fri +
            (?composition-final + (0 + (?query + 0))))))
      (staged-alpha-program (length spec) >>=
        (λas. degree-stage A as >>=
          (λdg. record-staged-message dg >>=
            (λ-. (assert (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) [])) >>=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs >>=
                  (λcomposition-final.
                    record-staged-message composition-final >>=
                      (λ-. staged-query-program A 0 rounds >>=
                        (λquery-chunks.
                          return
                            (F as dg composition-roots composition-bs
                              composition-final query-chunks))))))))))
        by (rule hash-collision-budget-bind)
          (rule alpha-range, rule alpha-coll, rule after-degree-range,
            rule after-degree-coll)
have after-trace-final-record-range:
  ∧trace-final F. hash-range-budget
    (0 +
      (?alpha +

```

```

    (?degree +
      (0 +
        (?composition-fri +
          (?composition-final + (0 + (?query + 0)))))))
  (record-staged-message trace-final >>=
    (λ-. staged-alpha-program (length spec) >>=
      (λas. degree-stage A as >>=
        (λdg. record-staged-message dg >>=
          (λ-. (assert (ceil-log (to-nat dg + 1) ≤
            ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) [])) >>=
                (λ(composition-roots, composition-bs).
                  composition-final-stage A dg composition-bs >>=
                    (λcomposition-final.
                      record-staged-message composition-final >>=
                        (λ-. staged-query-program A 0 rounds >>=
                          (λquery-chunks.
                            return
                              (F as dg composition-roots composition-bs
                                composition-final query-chunks))))))))))
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-record-staged-message,
      rule after-alpha-range)
have after-trace-final-record-coll:
  ∧ trace-final F. hash-collision-budget
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0)))))))
      (record-staged-message trace-final >>=
        (λ-. staged-alpha-program (length spec) >>=
          (λas. degree-stage A as >>=
            (λdg. record-staged-message dg >>=
              (λ-. (assert (ceil-log (to-nat dg + 1) ≤
                ceil-log (maxDegree + 1)) >>=
                  (λ-. staged-composition-fri-program A dg 0
                    (ceil-log (to-nat dg + 1)) [])) >>=
                    (λ(composition-roots, composition-bs).
                      composition-final-stage A dg composition-bs >>=
                        (λcomposition-final.
                          record-staged-message composition-final >>=
                            (λ-. staged-query-program A 0 rounds >>=
                              (λquery-chunks.
                                return
                                  (F as dg composition-roots composition-bs
                                    composition-final query-chunks))))))))))

```

```

by (rule hash-collision-budget-bind)
  (rule hash-range-budget-record-staged-message,
   rule hash-collision-budget-record-staged-message,
   rule after-alpha-range, rule after-alpha-coll)
have after-trace-final-range:
 $\wedge$  trace-bs  $F$ . hash-range-budget
  (?trace-final +
   (0 +
    (?alpha +
     (?degree +
      (0 +
       (?composition-fri +
        (?composition-final + (0 + (?query + 0))))))))))
  (trace-final-stage  $A$  trace-bs  $\gg=$ 
   ( $\lambda$  trace-final. record-staged-message trace-final  $\gg=$ 
    ( $\lambda$ -. staged-alpha-program (length spec)  $\gg=$ 
     ( $\lambda$  as. degree-stage  $A$  as  $\gg=$ 
      ( $\lambda$  dg. record-staged-message dg  $\gg=$ 
       ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
        ceil-log (maxDegree + 1))  $\gg=$ 
        ( $\lambda$ -. staged-composition-fri-program  $A$  dg 0
         (ceil-log (to-nat dg + 1)) []))  $\gg=$ 
        ( $\lambda$ (composition-roots, composition-bs).
         composition-final-stage  $A$  dg composition-bs  $\gg=$ 
         ( $\lambda$  composition-final.
          record-staged-message composition-final  $\gg=$ 
          ( $\lambda$ -. staged-query-program  $A$  0 rounds  $\gg=$ 
           ( $\lambda$  query-chunks.
            return
             ( $F$  trace-final as dg composition-roots
              composition-bs composition-final
              query-chunks))))))))))
by (rule hash-range-budget-bind)
  (rule trace-final-range, rule after-trace-final-record-range)
have after-trace-final-coll:
 $\wedge$  trace-bs  $F$ . hash-collision-budget
  (?trace-final +
   (0 +
    (?alpha +
     (?degree +
      (0 +
       (?composition-fri +
        (?composition-final + (0 + (?query + 0))))))))))
  (trace-final-stage  $A$  trace-bs  $\gg=$ 
   ( $\lambda$  trace-final. record-staged-message trace-final  $\gg=$ 
    ( $\lambda$ -. staged-alpha-program (length spec)  $\gg=$ 
     ( $\lambda$  as. degree-stage  $A$  as  $\gg=$ 
      ( $\lambda$  dg. record-staged-message dg  $\gg=$ 
       ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 

```

```

    ceil-log (maxDegree + 1)) >>=
    (λ-. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) []) >>=
    (λ(composition-roots, composition-bs).
      composition-final-stage A dg composition-bs >>=
      (λcomposition-final.
        record-staged-message composition-final >>=
        (λ-. staged-query-program A 0 rounds >>=
          (λquery-chunks.
            return
              (F trace-final as dg composition-roots
                composition-bs composition-final
                query-chunks))))))))))
  by (rule hash-collision-budget-bind)
    (rule trace-final-range, rule trace-final-coll,
      rule after-trace-final-record-range,
      rule after-trace-final-record-coll)
  have after-trace-fri-range:
    ∧F. hash-range-budget
      (?trace-fri +
        (?trace-final +
          (0 +
            (?alpha +
              (?degree +
                (0 +
                  (?composition-fri +
                    (?composition-final + (0 + (?query + 0))))))))))
      (staged-trace-fri-program A 0 (ceil-log clength) []) >>=
      (λ(trace-roots, trace-bs).
        trace-final-stage A trace-bs >>=
        (λtrace-final. record-staged-message trace-final >>=
          (λ-. staged-alpha-program (length spec) >>=
            (λas. degree-stage A as >>=
              (λdg. record-staged-message dg >>=
                (λ-. (assert (ceil-log (to-nat dg + 1) ≤
                  ceil-log (maxDegree + 1)) >>=
                    (λ-. staged-composition-fri-program A dg 0
                      (ceil-log (to-nat dg + 1)) [])) >>=
                    (λ(composition-roots, composition-bs).
                      composition-final-stage A dg composition-bs >>=
                      (λcomposition-final.
                        record-staged-message composition-final >>=
                        (λ-. staged-query-program A 0 rounds >>=
                          (λquery-chunks.
                            return
                              (F trace-roots trace-bs trace-final as dg
                                composition-roots composition-bs
                                composition-final query-chunks))))))))))
      )
  proof (rule hash-range-budget-bind)

```

```

fix F
show hash-range-budget ?trace-fri
  (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule trace-fri-range)
fix F x
show hash-range-budget
  (?trace-final +
   (0 +
    (?alpha +
     (?degree +
      (0 +
       (?composition-fri +
        (?composition-final + (0 + (?query + 0))))))))))
  (case x of (trace-roots, trace-bs)  $\Rightarrow$ 
   trace-final-stage A trace-bs  $\gg$ 
   ( $\lambda$ trace-final. record-staged-message trace-final  $\gg$ 
    ( $\lambda$ -. staged-alpha-program (length spec)  $\gg$ 
     ( $\lambda$ as. degree-stage A as  $\gg$ 
      ( $\lambda$ dg. record-staged-message dg  $\gg$ 
       ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1))  $\leq$ 
        ceil-log (maxDegree + 1))  $\gg$ 
        ( $\lambda$ -. staged-composition-fri-program A dg 0
         (ceil-log (to-nat dg + 1)) [])  $\gg$ 
         ( $\lambda$ (composition-roots, composition-bs).
          composition-final-stage A dg composition-bs  $\gg$ 
          ( $\lambda$ composition-final.
           record-staged-message composition-final  $\gg$ 
           ( $\lambda$ -. staged-query-program A 0 rounds  $\gg$ 
            ( $\lambda$ query-chunks.
             return
              (F trace-roots trace-bs trace-final as dg
               composition-roots composition-bs
               composition-final query-chunks))))))))))
proof (cases x)
case (Pair trace-roots trace-bs)
have hash-range-budget
  (?trace-final +
   (0 +
    (?alpha +
     (?degree +
      (0 +
       (?composition-fri +
        (?composition-final + (0 + (?query + 0))))))))))
  (trace-final-stage A trace-bs  $\gg$ 
   ( $\lambda$ trace-final. record-staged-message trace-final  $\gg$ 
    ( $\lambda$ -. staged-alpha-program (length spec)  $\gg$ 
     ( $\lambda$ as. degree-stage A as  $\gg$ 
      ( $\lambda$ dg. record-staged-message dg  $\gg$ 
       ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1))  $\leq$ 

```

```

      ceil-log (maxDegree + 1)) >>=
    (λ-. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) []) >>=
    (λ(composition-roots, composition-bs).
      composition-final-stage A dg composition-bs >>=
      (λcomposition-final.
        record-staged-message composition-final >>=
        (λ-. staged-query-program A 0 rounds >>=
          (λquery-chunks.
            return
              (F trace-roots trace-bs trace-final as dg
                composition-roots composition-bs
                composition-final query-chunks)))))))))
  by (rule after-trace-final-range)
then show ?thesis
  unfolding Pair by simp
qed
qed
have after-trace-fri-coll:
  ∧F. hash-collision-budget
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))))
    (staged-trace-fri-program A 0 (ceil-log clength) []) >>=
    (λ(trace-roots, trace-bs).
      trace-final-stage A trace-bs >>=
      (λtrace-final. record-staged-message trace-final >>=
        (λ-. staged-alpha-program (length spec) >>=
          (λas. degree-stage A as >>=
            (λdg. record-staged-message dg >>=
              (λ-. (assert (ceil-log (to-nat dg + 1)) ≤
                ceil-log (maxDegree + 1)) >>=
                (λ-. staged-composition-fri-program A dg 0
                  (ceil-log (to-nat dg + 1)) []) >>=
                  (λ(composition-roots, composition-bs).
                    composition-final-stage A dg composition-bs >>=
                    (λcomposition-final.
                      record-staged-message composition-final >>=
                      (λ-. staged-query-program A 0 rounds >>=
                        (λquery-chunks.
                          return
                            (F trace-roots trace-bs trace-final as dg
                              composition-roots composition-bs
                              composition-final query-chunks)))))))))))))

```



```

proof (rule hash-collision-budget-bind)
  fix F
  show hash-range-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
    by (rule trace-fri-range)
  fix F
  show hash-collision-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
    by (rule trace-fri-coll)
  fix F x
  show hash-range-budget
    (?trace-final +
      (0 +
        (?alpha +
          (?degree +
            (0 +
              (?composition-fri +
                (?composition-final + (0 + (?query + 0))))))))))
    (case x of (trace-roots, trace-bs) =>
      trace-final-stage A trace-bs >=>
      (λtrace-final. record-staged-message trace-final >=>
        (λ-. staged-alpha-program (length spec) >=>
          (λas. degree-stage A as >=>
            (λdg. record-staged-message dg >=>
              (λ-. (assert (ceil-log (to-nat dg + 1) ≤
                ceil-log (maxDegree + 1)) >=>
                (λ-. staged-composition-fri-program A dg 0
                  (ceil-log (to-nat dg + 1) [])) >=>
                (λ(composition-roots, composition-bs).
                  composition-final-stage A dg composition-bs >=>
                    (λcomposition-final.
                      record-staged-message composition-final >=>
                        (λ-. staged-query-program A 0 rounds >=>
                          (λquery-chunks.
                            return
                              (F trace-roots trace-bs trace-final as dg
                                composition-roots composition-bs
                                composition-final query-chunks))))))))))
  proof (cases x)
  case (Pair trace-roots trace-bs)
  have hash-range-budget
    (?trace-final +
      (0 +
        (?alpha +
          (?degree +
            (0 +
              (?composition-fri +
                (?composition-final + (0 + (?query + 0))))))))))
    (trace-final-stage A trace-bs >=>

```



```

                                composition-roots composition-bs
                                composition-final query-chunks))))))))))
proof (cases x)
  case (Pair trace-roots trace-bs)
  have hash-collision-budget
    (?trace-final +
      (0 +
        (?alpha +
          (?degree +
            (0 +
              (?composition-fri +
                (?composition-final + (0 + (?query + 0))))))))))
    (trace-final-stage A trace-bs >=>
      (λtrace-final. record-staged-message trace-final >=>
        (λ-. staged-alpha-program (length spec) >=>
          (λas. degree-stage A as >=>
            (λdg. record-staged-message dg >=>
              (λ-. (assert (ceil-log (to-nat dg + 1) ≤
                ceil-log (maxDegree + 1)) >=>
                (λ-. staged-composition-fri-program A dg 0
                  (ceil-log (to-nat dg + 1)) [])) >=>
                (λ(composition-roots, composition-bs).
                  composition-final-stage A dg composition-bs >=>
                    (λcomposition-final.
                      record-staged-message composition-final >=>
                        (λ-. staged-query-program A 0 rounds >=>
                          (λquery-chunks.
                            return
                              (F trace-roots trace-bs trace-final as dg
                                composition-roots composition-bs
                                composition-final query-chunks))))))))))
  by (rule after-trace-final-coll)
  then show ?thesis
  unfolding Pair by simp
qed
qed
have after-trace-root-record-range:
  ∧fr F. hash-range-budget
    (0 +
      (?trace-fri +
        (?trace-final +
          (0 +
            (?alpha +
              (?degree +
                (0 +
                  (?composition-fri +
                    (?composition-final + (0 + (?query + 0))))))))))
    (record-staged-message fr >=>
      (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >=>

```

```

(λ(trace-roots, trace-bs).
  trace-final-stage A trace-bs >>=
  (λtrace-final. record-staged-message trace-final >>=
    (λ-. staged-alpha-program (length spec) >>=
      (λas. degree-stage A as >>=
        (λdg. record-staged-message dg >>=
          (λ-. (assert (ceil-log (to-nat dg + 1) ≤
            ceil-log (maxDegree + 1)) >>=
            (λ-. staged-composition-fri-program A dg 0
              (ceil-log (to-nat dg + 1)) [])) >>=
            (λ(composition-roots, composition-bs).
              composition-final-stage A dg composition-bs >>=
              (λcomposition-final.
                record-staged-message composition-final >>=
                (λ-. staged-query-program A 0 rounds >>=
                  (λquery-chunks.
                    return
                    (F trace-roots trace-bs trace-final as dg
                      composition-roots composition-bs
                      composition-final query-chunks))))))))))
  by (rule hash-range-budget-bind)
  (rule hash-range-budget-record-staged-message,
    rule after-trace-fri-range)
have after-trace-root-record-coll:
  ∧fr F. hash-collision-budget
  (0 +
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))))
  (record-staged-message fr >>=
    (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
      (λ(trace-roots, trace-bs).
        trace-final-stage A trace-bs >>=
        (λtrace-final. record-staged-message trace-final >>=
          (λ-. staged-alpha-program (length spec) >>=
            (λas. degree-stage A as >>=
              (λdg. record-staged-message dg >>=
                (λ-. (assert (ceil-log (to-nat dg + 1) ≤
                  ceil-log (maxDegree + 1)) >>=
                  (λ-. staged-composition-fri-program A dg 0
                    (ceil-log (to-nat dg + 1)) [])) >>=
                  (λ(composition-roots, composition-bs).
                    composition-final-stage A dg composition-bs >>=
                    (λcomposition-final.

```

```

      record-staged-message composition-final >>=
      (λ-. staged-query-program A 0 rounds >>=
      (λquery-chunks.
        return
          (F trace-roots trace-bs trace-final as dg
            composition-roots composition-bs
            composition-final query-chunks)))))))))
by (rule hash-collision-budget-bind)
  (rule hash-range-budget-record-staged-message,
    rule hash-collision-budget-record-staged-message,
    rule after-trace-fri-range, rule after-trace-fri-coll)
have whole:
  hash-collision-budget
    (?trace-root +
      (0 +
        (?trace-fri +
          (?trace-final +
            (0 +
              (?alpha +
                (?degree +
                  (0 +
                    (?composition-fri +
                      (?composition-final + (0 + (?query + 0)))))))))))
      (staged-transcript-program A)
    unfolding staged-transcript-program-def Let-def
  proof (rule hash-collision-budget-bind)
  show hash-range-budget ?trace-root (trace-root-stage A)
    by (rule trace-root-range)
  show hash-collision-budget ?trace-root (trace-root-stage A)
    by (rule trace-root-coll)
  fix fr
  show hash-range-budget
    (0 +
      (?trace-fri +
        (?trace-final +
          (0 +
            (?alpha +
              (?degree +
                (0 +
                  (?composition-fri +
                    (?composition-final + (0 + (?query + 0)))))))))))
    (record-staged-message fr >>=
      (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
        (λ(trace-roots, trace-bs).
          trace-final-stage A trace-bs >>=
            (λtrace-final.
              record-staged-message trace-final >>=
                (λ-. staged-alpha-program (length spec) >>=
                  (λas. degree-stage A as >>=

```

```

( $\lambda$ dg. record-staged-message dg  $\gg$ 
( $\lambda$ -. assert (ceil-log (to-nat dg + 1)  $\leq$ 
  ceil-log (maxDegree + 1))  $\gg$ 
( $\lambda$ -. staged-composition-fri-program A dg 0
  (ceil-log (to-nat dg + 1))  $\square$   $\gg$ 
( $\lambda$ (composition-roots, composition-bs).
  composition-final-stage A dg composition-bs  $\gg$ 
( $\lambda$ composition-final.
  record-staged-message
  composition-final  $\gg$ 
( $\lambda$ -. staged-query-program A 0
  rounds  $\gg$ 
( $\lambda$ query-chunks.
  return
  ( $\square$ staged-trace-root = fr,
    staged-trace-fri-roots =
    trace-roots,
    staged-trace-fri-challenges =
    trace-bs,
    staged-trace-final =
    trace-final,
    staged-alphas = as,
    staged-degree = dg,
    staged-composition-fri-roots =
    composition-roots,
    staged-composition-fri-challenges =
    composition-bs,
    staged-composition-final =
    composition-final,
    staged-query-chunks =
    query-chunks)))))))))))))
using after-trace-root-record-range
[of fr
 $\lambda$ trace-roots trace-bs trace-final as dg composition-roots
  composition-bs composition-final query-chunks.
( $\square$ staged-trace-root = fr,
  staged-trace-fri-roots = trace-roots,
  staged-trace-fri-challenges = trace-bs,
  staged-trace-final = trace-final,
  staged-alphas = as,
  staged-degree = dg,
  staged-composition-fri-roots = composition-roots,
  staged-composition-fri-challenges = composition-bs,
  staged-composition-final = composition-final,
  staged-query-chunks = query-chunks)]
by (simp add: sm-bind-assoc)
fix fr
show hash-collision-budget
(0 +

```

```

(?trace-fri +
  (?trace-final +
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0))))))))))
(record-staged-message fr >>=
  (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
    (λ(trace-roots, trace-bs).
      trace-final-stage A trace-bs >>=
        (λtrace-final.
          record-staged-message trace-final >>=
            (λ-. staged-alpha-program (length spec) >>=
              (λas. degree-stage A as >>=
                (λdg. record-staged-message dg >>=
                  (λ-. assert (ceil-log (to-nat dg + 1) ≤
                    ceil-log (maxDegree + 1)) >>=
                    (λ-. staged-composition-fri-program A dg 0
                      (ceil-log (to-nat dg + 1)) [] >>=
                        (λ(composition-roots, composition-bs).
                          composition-final-stage A dg composition-bs >>=
                            (λcomposition-final.
                              record-staged-message
                                composition-final >>=
                                  (λ-. staged-query-program A 0
                                    rounds >>=
                                      (λquery-chunks.
                                        return
                                          (|staged-trace-root = fr,
                                            staged-trace-fri-roots =
                                              trace-roots,
                                            staged-trace-fri-challenges =
                                              trace-bs,
                                            staged-trace-final =
                                              trace-final,
                                            staged-alphas = as,
                                            staged-degree = dg,
                                            staged-composition-fri-roots =
                                              composition-roots,
                                            staged-composition-fri-challenges =
                                              composition-bs,
                                            staged-composition-final =
                                              composition-final,
                                            staged-query-chunks =
                                              query-chunks|)))))))))))))
using after-trace-root-record-coll
  [of fr

```

```

    λtrace-roots trace-bs trace-final as dg composition-roots
      composition-bs composition-final query-chunks.
    (|staged-trace-root = fr,
     staged-trace-fri-roots = trace-roots,
     staged-trace-fri-challenges = trace-bs,
     staged-trace-final = trace-final,
     staged-alphas = as,
     staged-degree = dg,
     staged-composition-fri-roots = composition-roots,
     staged-composition-fri-challenges = composition-bs,
     staged-composition-final = composition-final,
     staged-query-chunks = query-chunks|)]
  by (simp add: sm-bind-assoc)
qed
have budget-eq:
  ?trace-root +
  (0 +
   (?trace-fri +
    (?trace-final +
     (0 +
      (?alpha +
       (?degree +
        (0 +
         (?composition-fri +
          (?composition-final + (0 + (?query + 0)))))))))) =
    staged-attacker-query-budget budgets + staged-challenge-query-budget
  unfolding staged-attacker-query-budget-def
    staged-challenge-query-budget-def
  by (simp add: add.assoc add.commute add.left-commute)
show ?thesis
  using whole unfolding budget-eq .
qed

lemma hash-collision-budget-staged-semantic-adversary:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    hash-collision-budget
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
      (staged-semantic-adversary A)
proof -
  have hash-collision-budget
    ((staged-attacker-query-budget budgets + staged-challenge-query-budget) +
     0)
    (staged-semantic-adversary A)
  unfolding staged-semantic-adversary-def
proof (rule hash-collision-budget-bind
  [where n=staged-attacker-query-budget budgets +
    staged-challenge-query-budget and n'=0])

```



```

show hash-range-budget
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  (staged-transcript-program A)
  by (rule hash-range-budget-staged-transcript-program[OF wf controlled])
show hash-collision-budget
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  (staged-transcript-program A)
  by (rule hash-collision-budget-staged-transcript-program
    [OF wf controlled])
show  $\bigwedge x.$  hash-range-budget 0 (return (staged-proof-transcript x))
  by (rule hash-range-budget-return)
show  $\bigwedge x.$  hash-collision-budget 0
  (return (staged-proof-transcript x))
  by (rule hash-collision-budget-return)
qed
then show ?thesis by simp
qed

lemma hash-target-program-staged-transcript-program:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    hash-target-program B
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (staged-transcript-program A)
proof –
  let ?trace-root = trace-root-budget budgets
  let ?trace-fri =
    sum-list (trace-fri-budgets budgets) + ceil-log clength
  let ?trace-final = trace-final-budget budgets
  let ?alpha = length spec
  let ?degree = degree-budget budgets
  let ?composition-fri =
    sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
  let ?composition-final = composition-final-budget budgets
  let ?query =
    sum-list (query-opening-budgets budgets) + rounds
  have trace-root-target:
    hash-target-program B ?trace-root (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
  have trace-fri-target:
    hash-target-program B ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
proof –
  have len:
    0 + ceil-log clength  $\leq$  length (trace-fri-budgets budgets)
  using wf unfolding staged-budget-wellformed-def by simp

```

```

have exact:
  hash-target-program B
    (sum-list
      (take (ceil-log clength)
        (drop 0 (trace-fri-budgets budgets))) +
      ceil-log clength)
    (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule hash-target-program-staged-trace-fri-program
    [OF controlled len])
then show ?thesis
  using wf unfolding staged-budget-wellformed-def by simp
qed
have trace-final-target:
   $\wedge bs.$  hash-target-program B ?trace-final (trace-final-stage A bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have alpha-target:
  hash-target-program B ?alpha (staged-alpha-program (length spec))
  by (rule hash-target-program-staged-alpha-program)
have degree-target:
   $\wedge as.$  hash-target-program B ?degree (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have composition-fri-target:
   $\wedge dg.$  hash-target-program B ?composition-fri
    (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda.$  staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
  by (rule hash-target-program-guarded-staged-composition-fri-program
    [OF wf controlled])
have composition-final-target:
   $\wedge dg$  bs. hash-target-program B ?composition-final
    (composition-final-stage A dg bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have query-target:
  hash-target-program B ?query (staged-query-program A 0 rounds)
proof –
  have len:
    0 + rounds  $\leq$  length (query-opening-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-target-program B
      (sum-list
        (take rounds (drop 0 (query-opening-budgets budgets))) +
        rounds)
        (staged-query-program A 0 rounds)
      by (rule hash-target-program-staged-query-program[OF controlled len])
  then show ?thesis

```

```

    using wf unfolding staged-budget-wellformed-def by simp
qed
have query-tail:
   $\wedge F.$  hash-target-program  $B$  ( $?query + 0$ )
    (staged-query-program  $A$  0 rounds  $\gg=$ 
      ( $\lambda$ query-chunks. return ( $F$  query-chunks)))
  by (rule hash-target-program-bind)
    (rule query-target, rule hash-target-program-return)
have after-composition-final-record:
   $\wedge$ composition-final  $F.$  hash-target-program  $B$  ( $0 + (?query + 0)$ )
    (record-staged-message composition-final  $\gg=$ 
      ( $\lambda$ -. staged-query-program  $A$  0 rounds  $\gg=$ 
        ( $\lambda$ query-chunks. return ( $F$  query-chunks))))
  by (rule hash-target-program-bind)
    (rule hash-target-program-record-staged-message, rule query-tail)
have after-composition-final:
   $\wedge$ dg bs  $F.$  hash-target-program  $B$ 
    ( $?composition$ -final + ( $0 + (?query + 0)$ ))
    (composition-final-stage  $A$  dg bs  $\gg=$ 
      ( $\lambda$ composition-final.
        record-staged-message composition-final  $\gg=$ 
          ( $\lambda$ -. staged-query-program  $A$  0 rounds  $\gg=$ 
            ( $\lambda$ query-chunks.
              return ( $F$  composition-final query-chunks))))))
  by (rule hash-target-program-bind)
    (rule composition-final-target, rule after-composition-final-record)
have after-composition-fri:
   $\wedge$ dg  $F.$  hash-target-program  $B$ 
    ( $?composition$ -fri +
      ( $?composition$ -final + ( $0 + (?query + 0)$ )))
    ((assert (ceil-log (to-nat dg + 1)  $\leq$ 
      ceil-log (maxDegree + 1))  $\gg=$ 
      ( $\lambda$ -. staged-composition-fri-program  $A$  dg 0
        (ceil-log (to-nat dg + 1)) []))  $\gg=$ 
      ( $\lambda$ (composition-roots, composition-bs).
        composition-final-stage  $A$  dg composition-bs  $\gg=$ 
          ( $\lambda$ composition-final.
            record-staged-message composition-final  $\gg=$ 
              ( $\lambda$ -. staged-query-program  $A$  0 rounds  $\gg=$ 
                ( $\lambda$ query-chunks.
                  return
                    ( $F$  composition-roots composition-bs composition-final
                      query-chunks)))))))
proof (rule hash-target-program-bind)
fix dg  $F$ 
show hash-target-program  $B$   $?composition$ -fri
  (assert (ceil-log (to-nat dg + 1)  $\leq$ 
    ceil-log (maxDegree + 1))  $\gg=$ 
    ( $\lambda$ -. staged-composition-fri-program  $A$  dg 0
      (ceil-log (to-nat dg + 1)) []))

```

```

      (ceil-log (to-nat dg + 1)) [])
    by (rule composition-fri-target)
  fix dg F x
  show hash-target-program B
    (?composition-final + (0 + (?query + 0)))
    (case x of (composition-roots, composition-bs) =>
      composition-final-stage A dg composition-bs >=>
      (λcomposition-final.
        record-staged-message composition-final >=>
        (λ-. staged-query-program A 0 rounds >=>
          (λquery-chunks.
            return
              (F composition-roots composition-bs composition-final
                query-chunks))))))
  proof (cases x)
  case (Pair composition-roots composition-bs)
  have hash-target-program B
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg composition-bs >=>
      (λcomposition-final.
        record-staged-message composition-final >=>
        (λ-. staged-query-program A 0 rounds >=>
          (λquery-chunks.
            return
              (F composition-roots composition-bs composition-final
                query-chunks))))))
  by (rule after-composition-final)
  then show ?thesis
    unfolding Pair by simp
  qed
qed
have after-degree-record:
  ∧dg F. hash-target-program B
    (0 +
      (?composition-fri +
        (?composition-final + (0 + (?query + 0)))))
  (record-staged-message dg >=>
    (λ-. (assert (ceil-log (to-nat dg + 1) ≤
      ceil-log (maxDegree + 1)) >=>
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [])) >=>
      (λ(composition-roots, composition-bs).
        composition-final-stage A dg composition-bs >=>
        (λcomposition-final.
          record-staged-message composition-final >=>
          (λ-. staged-query-program A 0 rounds >=>
            (λquery-chunks.
              return
                (F composition-roots composition-bs composition-final
                  query-chunks))))))

```

```

                                query-chunks))))))
  by (rule hash-target-program-bind)
    (rule hash-target-program-record-staged-message, rule after-composition-fri)
have after-degree:
 $\wedge$  as  $F$ . hash-target-program  $B$ 
  (?degree +
    (0 +
      (?composition-fri +
        (?composition-final + (0 + (?query + 0))))))
    (degree-stage  $A$  as  $\gg$ 
      ( $\lambda$  dg. record-staged-message dg  $\gg$ 
        ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
          ceil-log (maxDegree + 1))  $\gg$ 
          ( $\lambda$ -. staged-composition-fri-program  $A$  dg 0
            (ceil-log (to-nat dg + 1)) []))  $\gg$ 
          ( $\lambda$ (composition-roots, composition-bs).
            composition-final-stage  $A$  dg composition-bs  $\gg$ 
              ( $\lambda$ composition-final.
                record-staged-message composition-final  $\gg$ 
                  ( $\lambda$ -. staged-query-program  $A$  0 rounds  $\gg$ 
                    ( $\lambda$ query-chunks.
                      return
                        (F dg composition-roots composition-bs
                          composition-final query-chunks))))))))
  by (rule hash-target-program-bind)
    (rule degree-target, rule after-degree-record)
have after-alpha:
 $\wedge$   $F$ . hash-target-program  $B$ 
  (?alpha +
    (?degree +
      (0 +
        (?composition-fri +
          (?composition-final + (0 + (?query + 0))))))
    (staged-alpha-program (length spec)  $\gg$ 
      ( $\lambda$  as. degree-stage  $A$  as  $\gg$ 
        ( $\lambda$  dg. record-staged-message dg  $\gg$ 
          ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
            ceil-log (maxDegree + 1))  $\gg$ 
            ( $\lambda$ -. staged-composition-fri-program  $A$  dg 0
              (ceil-log (to-nat dg + 1)) []))  $\gg$ 
              ( $\lambda$ (composition-roots, composition-bs).
                composition-final-stage  $A$  dg composition-bs  $\gg$ 
                  ( $\lambda$ composition-final.
                    record-staged-message composition-final  $\gg$ 
                      ( $\lambda$ -. staged-query-program  $A$  0 rounds  $\gg$ 
                        ( $\lambda$ query-chunks.
                          return
                            (F as dg composition-roots composition-bs
                              composition-final query-chunks))))))))

```

```

by (rule hash-target-program-bind)
  (rule alpha-target, rule after-degree)
have after-trace-final-record:
 $\wedge$ trace-final  $F$ . hash-target-program  $B$ 
  (0 +
    (?alpha +
      (?degree +
        (0 +
          (?composition-fri +
            (?composition-final + (0 + (?query + 0))))))))
    (record-staged-message trace-final  $\gg$ 
      ( $\lambda$ -. staged-alpha-program (length spec)  $\gg$ 
        ( $\lambda$ as. degree-stage  $A$  as  $\gg$ 
          ( $\lambda$ dg. record-staged-message dg  $\gg$ 
            ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
              ceil-log (maxDegree + 1))  $\gg$ 
              ( $\lambda$ -. staged-composition-fri-program  $A$  dg 0
                (ceil-log (to-nat dg + 1)) []))  $\gg$ 
              ( $\lambda$ (composition-roots, composition-bs).
                composition-final-stage  $A$  dg composition-bs  $\gg$ 
                ( $\lambda$ composition-final.
                  record-staged-message composition-final  $\gg$ 
                  ( $\lambda$ -. staged-query-program  $A$  0 rounds  $\gg$ 
                    ( $\lambda$ query-chunks.
                      return
                      (F as dg composition-roots composition-bs
                        composition-final query-chunks))))))))
    )
by (rule hash-target-program-bind)
  (rule hash-target-program-record-staged-message, rule after-alpha)
have after-trace-final:
 $\wedge$ trace-bs  $F$ . hash-target-program  $B$ 
  (?trace-final +
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0))))))))
    (trace-final-stage  $A$  trace-bs  $\gg$ 
      ( $\lambda$ trace-final. record-staged-message trace-final  $\gg$ 
        ( $\lambda$ -. staged-alpha-program (length spec)  $\gg$ 
          ( $\lambda$ as. degree-stage  $A$  as  $\gg$ 
            ( $\lambda$ dg. record-staged-message dg  $\gg$ 
              ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
                ceil-log (maxDegree + 1))  $\gg$ 
                ( $\lambda$ -. staged-composition-fri-program  $A$  dg 0
                  (ceil-log (to-nat dg + 1)) []))  $\gg$ 
                ( $\lambda$ (composition-roots, composition-bs).
                  composition-final-stage  $A$  dg composition-bs  $\gg$ 

```

```

      (λcomposition-final.
        record-staged-message composition-final >>=
        (λ-. staged-query-program A 0 rounds >>=
          (λquery-chunks.
            return
            (F trace-final as dg composition-roots
              composition-bs composition-final
              query-chunks)))))))))
  by (rule hash-target-program-bind)
    (rule trace-final-target, rule after-trace-final-record)
have after-trace-fri:
  ∧F. hash-target-program B
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))))
    (staged-trace-fri-program A 0 (ceil-log clength) []) >>=
    (λ(trace-roots, trace-bs).
      trace-final-stage A trace-bs >>=
      (λtrace-final. record-staged-message trace-final >>=
        (λ-. staged-alpha-program (length spec) >>=
          (λas. degree-stage A as >>=
            (λdg. record-staged-message dg >>=
              (λ-. (assert (ceil-log (to-nat dg + 1) ≤
                ceil-log (maxDegree + 1)) >>=
                (λ-. staged-composition-fri-program A dg 0
                  (ceil-log (to-nat dg + 1)) [])) >>=
                (λ(composition-roots, composition-bs).
                  composition-final-stage A dg composition-bs >>=
                  (λcomposition-final.
                    record-staged-message composition-final >>=
                    (λ-. staged-query-program A 0 rounds >>=
                      (λquery-chunks.
                        return
                        (F trace-roots trace-bs trace-final as dg
                          composition-roots composition-bs
                          composition-final query-chunks)))))))))
  proof (rule hash-target-program-bind)
    fix F
  show hash-target-program B ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
    by (rule trace-fri-target)
  fix F x
  show hash-target-program B
    (?trace-final +

```

```

(0 +
  (?alpha +
    (?degree +
      (0 +
        (?composition-fri +
          (?composition-final + (0 + (?query + 0))))))))))
(case x of (trace-roots, trace-bs) =>
  trace-final-stage A trace-bs >=>
    (λtrace-final. record-staged-message trace-final >=>
      (λ-. staged-alpha-program (length spec) >=>
        (λas. degree-stage A as >=>
          (λdg. record-staged-message dg >=>
            (λ-. (assert (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1)) >=>
                (λ-. staged-composition-fri-program A dg 0
                  (ceil-log (to-nat dg + 1)) [])) >=>
                  (λ(composition-roots, composition-bs).
                    composition-final-stage A dg composition-bs >=>
                      (λcomposition-final.
                        record-staged-message composition-final >=>
                          (λ-. staged-query-program A 0 rounds >=>
                            (λquery-chunks.
                              return
                                (F trace-roots trace-bs trace-final as dg
                                  composition-roots composition-bs
                                  composition-final query-chunks))))))))))
proof (cases x)
case (Pair trace-roots trace-bs)
have hash-target-program B
  (?trace-final +
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0))))))))))
  (trace-final-stage A trace-bs >=>
    (λtrace-final. record-staged-message trace-final >=>
      (λ-. staged-alpha-program (length spec) >=>
        (λas. degree-stage A as >=>
          (λdg. record-staged-message dg >=>
            (λ-. (assert (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1)) >=>
                (λ-. staged-composition-fri-program A dg 0
                  (ceil-log (to-nat dg + 1)) [])) >=>
                  (λ(composition-roots, composition-bs).
                    composition-final-stage A dg composition-bs >=>
                      (λcomposition-final.
                        record-staged-message composition-final >=>

```



```

      (λ-. staged-query-program A 0 rounds >>=
        (λquery-chunks.
          return
            (F trace-roots trace-bs trace-final as dg
              composition-roots composition-bs
              composition-final query-chunks)))))))))
    by (rule after-trace-final)
  then show ?thesis
    unfolding Pair by simp
qed
qed
have after-trace-root-record:
  ∧fr F. hash-target-program B
    (0 +
      (?trace-fri +
        (?trace-final +
          (0 +
            (?alpha +
              (?degree +
                (0 +
                  (?composition-fri +
                    (?composition-final + (0 + (?query + 0))))))))))
      (record-staged-message fr >>=
        (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
          (λ(trace-roots, trace-bs).
            trace-final-stage A trace-bs >>=
              (λtrace-final. record-staged-message trace-final >>=
                (λ-. staged-alpha-program (length spec) >>=
                  (λas. degree-stage A as >>=
                    (λdg. record-staged-message dg >>=
                      (λ-. (assert (ceil-log (to-nat dg + 1) ≤
                        ceil-log (maxDegree + 1)) >>=
                          (λ-. staged-composition-fri-program A dg 0
                            (ceil-log (to-nat dg + 1)) [])) >>=
                            (λ(composition-roots, composition-bs).
                              composition-final-stage A dg composition-bs >>=
                                (λcomposition-final.
                                  record-staged-message composition-final >>=
                                    (λ-. staged-query-program A 0 rounds >>=
                                      (λquery-chunks.
                                        return
                                          (F trace-roots trace-bs trace-final as dg
                                            composition-roots composition-bs
                                            composition-final query-chunks))))))))))
                    by (rule hash-target-program-bind)
                  (rule hash-target-program-record-staged-message, rule after-trace-fri)
  have whole:
    hash-target-program B
      (?trace-root +

```

```

(0 +
  (?trace-fri +
    (?trace-final +
      (0 +
        (?alpha +
          (?degree +
            (0 +
              (?composition-fri +
                (?composition-final + (0 + (?query + 0))))))))))
  (staged-transcript-program A)
unfolding staged-transcript-program-def Let-def
proof (rule hash-target-program-bind)
show hash-target-program B ?trace-root (trace-root-stage A)
  by (rule trace-root-target)
fix fr
show hash-target-program B
  (0 +
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))))
  (record-staged-message fr >=>
    (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >=>
      (λ(trace-roots, trace-bs).
        trace-final-stage A trace-bs >=>
          (λtrace-final.
            record-staged-message trace-final >=>
              (λ-. staged-alpha-program (length spec) >=>
                (λas. degree-stage A as >=>
                  (λdg. record-staged-message dg >=>
                    (λ-. assert (ceil-log (to-nat dg + 1) ≤
                      ceil-log (maxDegree + 1)) >=>
                      (λ-. staged-composition-fri-program A dg 0
                        (ceil-log (to-nat dg + 1)) [] >=>
                          (λ(composition-roots, composition-bs).
                            composition-final-stage A dg composition-bs >=>
                              (λcomposition-final.
                                record-staged-message
                                  composition-final >=>
                                    (λ-. staged-query-program A 0
                                      rounds >=>
                                        (λquery-chunks.
                                          return
                                            (|staged-trace-root = fr,
                                              staged-trace-fri-roots =

```

```

    trace-roots,
    staged-trace-fri-challenges =
    trace-bs,
    staged-trace-final =
    trace-final,
    staged-alphas = as,
    staged-degree = dg,
    staged-composition-fri-roots =
    composition-roots,
    staged-composition-fri-challenges =
    composition-bs,
    staged-composition-final =
    composition-final,
    staged-query-chunks =
    query-chunks)))))))))))))
using after-trace-root-record
  [of fr
    λtrace-roots trace-bs trace-final as dg composition-roots
      composition-bs composition-final query-chunks.
    (|staged-trace-root = fr,
      staged-trace-fri-roots = trace-roots,
      staged-trace-fri-challenges = trace-bs,
      staged-trace-final = trace-final,
      staged-alphas = as,
      staged-degree = dg,
      staged-composition-fri-roots = composition-roots,
      staged-composition-fri-challenges = composition-bs,
      staged-composition-final = composition-final,
      staged-query-chunks = query-chunks|)]
  by (simp add: sm-bind-assoc)
qed
have budget-eq:
  ?trace-root +
  (0 +
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0)))))))))) =
    staged-attacker-query-budget budgets + staged-challenge-query-budget
unfolding staged-attacker-query-budget-def
  staged-challenge-query-budget-def
by (simp add: add.assoc add.commute add.left-commute)
show ?thesis
using whole unfolding budget-eq .
qed

```

lemma *hash-target-program-checked-staged-transcript-program*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
shows
hash-target-program B
(staged-attacker-query-budget budgets + staged-challenge-query-budget)
(checked-staged-transcript-program A)

proof –
let *?trace-root* = *trace-root-budget budgets*
let *?trace-fri* =
sum-list (trace-fri-budgets budgets) + ceil-log clength
let *?trace-final* = *trace-final-budget budgets*
let *?alpha* = *length spec*
let *?degree* = *degree-budget budgets*
let *?composition-fri* =
sum-list (composition-fri-budgets budgets) +
ceil-log (maxDegree + 1)
let *?composition-final* = *composition-final-budget budgets*
let *?query* =
sum-list (query-opening-budgets budgets) + rounds
have *trace-root-target*:
hash-target-program B ?trace-root (trace-root-stage A)
using *controlled unfolding staged-adversary-controlled-def*
by (*blast intro: controlled-ro-program-target*)
have *trace-fri-target*:
hash-target-program B ?trace-fri
(staged-trace-fri-program A 0 (ceil-log clength) [])
proof –
have *len*:
 $0 + \text{ceil-log clength} \leq \text{length (trace-fri-budgets budgets)}$
using *wf unfolding staged-budget-wellformed-def* **by** *simp*
have *exact*:
hash-target-program B
(sum-list
(take (ceil-log clength)
(drop 0 (trace-fri-budgets budgets))) +
ceil-log clength)
(staged-trace-fri-program A 0 (ceil-log clength) [])
by (*rule hash-target-program-staged-trace-fri-program*
 $[OF \text{ controlled len}]$)
then show *?thesis*
using *wf unfolding staged-budget-wellformed-def* **by** *simp*
qed
have *trace-final-target*:
 $\bigwedge bs. \text{hash-target-program } B \text{ ?trace-final (trace-final-stage } A \text{ bs)}$
using *controlled unfolding staged-adversary-controlled-def*
by (*blast intro: controlled-ro-program-target*)
have *alpha-target*:

```

hash-target-program B ?alpha (staged-alpha-program (length spec))
by (rule hash-target-program-staged-alpha-program)
have degree-target:
   $\wedge$  as. hash-target-program B ?degree (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have composition-fri-target:
   $\wedge$  dg. hash-target-program B ?composition-fri
    (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda$ -. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
  by (rule hash-target-program-guarded-staged-composition-fri-program
    [OF wf controlled])
have composition-final-target:
   $\wedge$  dg bs. hash-target-program B ?composition-final
    (composition-final-stage A dg bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have query-target:
   $\wedge$  trace-roots composition-roots.
    hash-target-program B ?query
    (checked-staged-query-program A trace-roots composition-roots
      0 rounds)
proof -
  fix trace-roots composition-roots
  have len:
    0 + rounds  $\leq$  length (query-opening-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-target-program B
      (sum-list
        (take rounds (drop 0 (query-opening-budgets budgets))) +
        rounds)
      (checked-staged-query-program A trace-roots composition-roots
        0 rounds)
    by (rule hash-target-program-checked-staged-query-program
      [OF controlled len])
  then show
    hash-target-program B ?query
      (checked-staged-query-program A trace-roots composition-roots
        0 rounds)
    using wf unfolding staged-budget-wellformed-def by simp
qed
have query-tail:
   $\wedge$  trace-roots composition-roots F. hash-target-program B (?query + 0)
    (checked-staged-query-program A trace-roots composition-roots
      0 rounds  $\gg$ 
      ( $\lambda$ query-chunks. return (F query-chunks)))
  by (rule hash-target-program-bind)

```

```

    (rule query-target, rule hash-target-program-return)
have after-composition-final-record:
   $\wedge$  trace-roots composition-roots composition-final  $F$ .
    hash-target-program  $B$  ( $0 + (?query + 0)$ )
    (record-staged-message composition-final  $\gg=$ 
      ( $\lambda$ -. checked-staged-query-program  $A$  trace-roots
        composition-roots  $0$  rounds  $\gg=$ 
          ( $\lambda$ query-chunks. return ( $F$  query-chunks))))
by (rule hash-target-program-bind)
    (rule hash-target-program-record-staged-message, rule query-tail)
have after-composition-final:
   $\wedge$  trace-roots  $dg$  composition-roots  $bs$   $F$ . hash-target-program  $B$ 
    (?composition-final + ( $0 + (?query + 0)$ ))
    (composition-final-stage  $A$   $dg$   $bs$   $\gg=$ 
      ( $\lambda$ composition-final.
        record-staged-message composition-final  $\gg=$ 
          ( $\lambda$ -. checked-staged-query-program  $A$  trace-roots
            composition-roots  $0$  rounds  $\gg=$ 
              ( $\lambda$ query-chunks.
                return ( $F$  composition-final query-chunks))))))
by (rule hash-target-program-bind)
    (rule composition-final-target, rule after-composition-final-record)
have after-composition-fri:
   $\wedge$  trace-roots  $dg$   $F$ . hash-target-program  $B$ 
    (?composition-fri +
      (?composition-final + ( $0 + (?query + 0)$ ))))
    ((assert (ceil-log (to-nat  $dg + 1$ )  $\leq$ 
      ceil-log (maxDegree + 1))  $\gg=$ 
      ( $\lambda$ -. staged-composition-fri-program  $A$   $dg$   $0$ 
        (ceil-log (to-nat  $dg + 1$ )) []))  $\gg=$ 
      ( $\lambda$ (composition-roots, composition-bs).
        composition-final-stage  $A$   $dg$  composition-bs  $\gg=$ 
          ( $\lambda$ composition-final.
            record-staged-message composition-final  $\gg=$ 
              ( $\lambda$ -. checked-staged-query-program  $A$  trace-roots
                composition-roots  $0$  rounds  $\gg=$ 
                  ( $\lambda$ query-chunks.
                    return
                      ( $F$  composition-roots composition-bs composition-final
                        query-chunks)))))))
proof (rule hash-target-program-bind)
fix trace-roots  $dg$   $F$ 
show hash-target-program  $B$  ?composition-fri
  (assert (ceil-log (to-nat  $dg + 1$ )  $\leq$ 
    ceil-log (maxDegree + 1))  $\gg=$ 
    ( $\lambda$ -. staged-composition-fri-program  $A$   $dg$   $0$ 
      (ceil-log (to-nat  $dg + 1$ )) []))
by (rule composition-fri-target)
next

```

```

fix trace-roots dg F x
show hash-target-program B
  (?composition-final + (0 + (?query + 0)))
  (case x of (composition-roots, composition-bs) =>
    composition-final-stage A dg composition-bs >>=
    (λcomposition-final.
      record-staged-message composition-final >>=
      (λ-. checked-staged-query-program A trace-roots
        composition-roots 0 rounds >>=
        (λquery-chunks.
          return
            (F composition-roots composition-bs composition-final
              query-chunks))))))
proof (cases x)
  case (Pair composition-roots composition-bs)
  have hash-target-program B
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg composition-bs >>=
    (λcomposition-final.
      record-staged-message composition-final >>=
      (λ-. checked-staged-query-program A trace-roots
        composition-roots 0 rounds >>=
        (λquery-chunks.
          return
            (F composition-roots composition-bs composition-final
              query-chunks))))))
  by (rule after-composition-final)
  then show ?thesis
    unfolding Pair by simp
  qed
qed
have after-degree-record:
  ∧ trace-roots dg F. hash-target-program B
  (0 +
    (?composition-fri +
      (?composition-final + (0 + (?query + 0)))))
  (record-staged-message dg >>=
  (λ-. (assert (ceil-log (to-nat dg + 1) ≤
    ceil-log (maxDegree + 1)) >>=
  (λ-. staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) [])) >>=
  (λ(composition-roots, composition-bs).
    composition-final-stage A dg composition-bs >>=
    (λcomposition-final.
      record-staged-message composition-final >>=
      (λ-. checked-staged-query-program A trace-roots
        composition-roots 0 rounds >>=
        (λquery-chunks.
          return

```

```

      (F composition-roots composition-bs composition-final
        query-chunks))))))
  by (rule hash-target-program-bind)
    (rule hash-target-program-record-staged-message,
      rule after-composition-fri)
  have after-degree:
     $\wedge$  trace-roots as F. hash-target-program B
      (?degree +
        (0 +
          (?composition-fri +
            (?composition-final + (0 + (?query + 0)))))))
      (degree-stage A as  $\gg$ 
        ( $\lambda$  dg. record-staged-message dg  $\gg$ 
          ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
            ceil-log (maxDegree + 1))  $\gg$ 
            ( $\lambda$ -. staged-composition-fri-program A dg 0
              (ceil-log (to-nat dg + 1)) []))  $\gg$ 
            ( $\lambda$ (composition-roots, composition-bs).
              composition-final-stage A dg composition-bs  $\gg$ 
                ( $\lambda$ composition-final.
                  record-staged-message composition-final  $\gg$ 
                    ( $\lambda$ -. checked-staged-query-program A trace-roots
                      composition-roots 0 rounds  $\gg$ 
                        ( $\lambda$ query-chunks.
                          return
                            (F dg composition-roots composition-bs
                              composition-final query-chunks)))))))
        by (rule hash-target-program-bind)
          (rule degree-target, rule after-degree-record)
      have after-alpha:
         $\wedge$  trace-roots F. hash-target-program B
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0)))))))
          (staged-alpha-program (length spec)  $\gg$ 
            ( $\lambda$ as. degree-stage A as  $\gg$ 
              ( $\lambda$  dg. record-staged-message dg  $\gg$ 
                ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
                  ceil-log (maxDegree + 1))  $\gg$ 
                    ( $\lambda$ -. staged-composition-fri-program A dg 0
                      (ceil-log (to-nat dg + 1)) []))  $\gg$ 
                      ( $\lambda$ (composition-roots, composition-bs).
                        composition-final-stage A dg composition-bs  $\gg$ 
                          ( $\lambda$ composition-final.
                            record-staged-message composition-final  $\gg$ 
                              ( $\lambda$ -. checked-staged-query-program A trace-roots
                                composition-roots 0 rounds  $\gg$ 
                                  composition-roots 0 rounds  $\gg$ 

```



```

        (λquery-chunks.
          return
            (F as dg composition-roots composition-bs
              composition-final query-chunks)))))))))
  by (rule hash-target-program-bind)
    (rule alpha-target, rule after-degree)
have after-trace-final-record:
  ∧ trace-roots trace-final F. hash-target-program B
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0))))))))))
    (record-staged-message trace-final >=
      (λ-. staged-alpha-program (length spec) >=
        (λas. degree-stage A as >=
          (λdg. record-staged-message dg >=
            (λ-. (assert (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1)) >=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) [])) >=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs >=
                  (λcomposition-final.
                    record-staged-message composition-final >=
                      (λ-. checked-staged-query-program A trace-roots
                        composition-roots 0 rounds >=
                          (λquery-chunks.
                            return
                              (F as dg composition-roots composition-bs
                                composition-final query-chunks))))))))))
        by (rule hash-target-program-bind)
          (rule hash-target-program-record-staged-message, rule after-alpha)
have after-trace-final:
  ∧ trace-roots trace-bs F. hash-target-program B
    (?trace-final +
      (0 +
        (?alpha +
          (?degree +
            (0 +
              (?composition-fri +
                (?composition-final + (0 + (?query + 0))))))))))
    (trace-final-stage A trace-bs >=
      (λtrace-final. record-staged-message trace-final >=
        (λ-. staged-alpha-program (length spec) >=
          (λas. degree-stage A as >=
            (λdg. record-staged-message dg >=
              (λ-. (assert (ceil-log (to-nat dg + 1) ≤

```

```

    ceil-log (maxDegree + 1)) >>=
    (λ-. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) []) >>=
    (λ(composition-roots, composition-bs).
      composition-final-stage A dg composition-bs >>=
      (λcomposition-final.
        record-staged-message composition-final >>=
        (λ-. checked-staged-query-program A trace-roots
          composition-roots 0 rounds >>=
          (λquery-chunks.
            return
            (F trace-final as dg composition-roots
              composition-bs composition-final
              query-chunks)))))))))
  by (rule hash-target-program-bind)
  (rule trace-final-target, rule after-trace-final-record)
have after-trace-fri:
  ∧F. hash-target-program B
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))))
    (staged-trace-fri-program A 0 (ceil-log clength) []) >>=
    (λ(trace-roots, trace-bs).
      trace-final-stage A trace-bs >>=
      (λtrace-final. record-staged-message trace-final >>=
        (λ-. staged-alpha-program (length spec) >>=
          (λas. degree-stage A as >>=
            (λdg. record-staged-message dg >>=
              (λ-. (assert (ceil-log (to-nat dg + 1) ≤
                ceil-log (maxDegree + 1)) >>=
                (λ-. staged-composition-fri-program A dg 0
                  (ceil-log (to-nat dg + 1)) [])) >>=
                (λ(composition-roots, composition-bs).
                  composition-final-stage A dg composition-bs >>=
                  (λcomposition-final.
                    record-staged-message composition-final >>=
                    (λ-. checked-staged-query-program A
                      trace-roots composition-roots 0 rounds >>=
                      (λquery-chunks.
                        return
                        (F trace-roots trace-bs trace-final as dg
                          composition-roots composition-bs
                          composition-final query-chunks)))))))))
  proof (rule hash-target-program-bind)

```

```

fix F
show hash-target-program B ?trace-fri
  (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule trace-fri-target)
next
fix F x
show hash-target-program B
  (?trace-final +
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0))))))))))
  (case x of (trace-roots, trace-bs) ⇒
    trace-final-stage A trace-bs ≫=
    (λtrace-final. record-staged-message trace-final ≫=
      (λ-. staged-alpha-program (length spec) ≫=
        (λas. degree-stage A as ≫=
          (λdg. record-staged-message dg ≫=
            (λ-. (assert (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1)) ≫=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1) []) ≫=
                (λ(composition-roots, composition-bs).
                  composition-final-stage A dg composition-bs ≫=
                  (λcomposition-final.
                    record-staged-message composition-final ≫=
                    (λ-. checked-staged-query-program A
                      trace-roots composition-roots 0 rounds ≫=
                      (λquery-chunks.
                        return
                        (F trace-roots trace-bs trace-final as dg
                          composition-roots composition-bs
                          composition-final query-chunks))))))))))
  )
proof (cases x)
case (Pair trace-roots trace-bs)
have hash-target-program B
  (?trace-final +
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0))))))))))
  (trace-final-stage A trace-bs ≫=
    (λtrace-final. record-staged-message trace-final ≫=
      (λ-. staged-alpha-program (length spec) ≫=
        (λas. degree-stage A as ≫=

```

```

(λdg. record-staged-message dg ≫=
  (λ-. (assert (ceil-log (to-nat dg + 1) ≤
    ceil-log (maxDegree + 1)) ≫=
    (λ-. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) [])) ≫=
    (λ(composition-roots, composition-bs).
      composition-final-stage A dg composition-bs ≫=
      (λcomposition-final.
        record-staged-message composition-final ≫=
        (λ-. checked-staged-query-program A
          trace-roots composition-roots 0 rounds ≫=
          (λquery-chunks.
            return
            (F trace-roots trace-bs trace-final as dg
              composition-roots composition-bs
              composition-final query-chunks))))))))))
  by (rule after-trace-final)
  then show ?thesis
  unfolding Pair by simp
qed
qed
have after-trace-root-record:
  ∧fr F. hash-target-program B
  (0 +
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))))
  (record-staged-message fr ≫=
    (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] ≫=
      (λ(trace-roots, trace-bs).
        trace-final-stage A trace-bs ≫=
        (λtrace-final. record-staged-message trace-final ≫=
          (λ-. staged-alpha-program (length spec) ≫=
            (λas. degree-stage A as ≫=
              (λdg. record-staged-message dg ≫=
                (λ-. (assert (ceil-log (to-nat dg + 1) ≤
                  ceil-log (maxDegree + 1)) ≫=
                  (λ-. staged-composition-fri-program A dg 0
                    (ceil-log (to-nat dg + 1)) [])) ≫=
                  (λ(composition-roots, composition-bs).
                    composition-final-stage A dg composition-bs ≫=
                    (λcomposition-final.
                      record-staged-message composition-final ≫=
                      (λ-. checked-staged-query-program A

```

```

      trace-roots composition-roots 0 rounds >>=
      (λquery-chunks.
        return
          (F trace-roots trace-bs trace-final as dg
            composition-roots composition-bs
            composition-final query-chunks)))))))))
  by (rule hash-target-program-bind)
    (rule hash-target-program-record-staged-message, rule after-trace-fri)
have whole:
  hash-target-program B
    (?trace-root +
      (0 +
        (?trace-fri +
          (?trace-final +
            (0 +
              (?alpha +
                (?degree +
                  (0 +
                    (?composition-fri +
                      (?composition-final + (0 + (?query + 0))))))))))
            (checked-staged-transcript-program A)
  unfolding checked-staged-transcript-program-def Let-def
proof (rule hash-target-program-bind)
  show hash-target-program B ?trace-root (trace-root-stage A)
    by (rule trace-root-target)
next
fix fr
show hash-target-program B
  (0 +
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))
    (record-staged-message fr >>=
      (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
        (λ(trace-roots, trace-bs).
          trace-final-stage A trace-bs >>=
            (λtrace-final.
              record-staged-message trace-final >>=
                (λ-. staged-alpha-program (length spec) >>=
                  (λas. degree-stage A as >>=
                    (λdg. record-staged-message dg >>=
                      (λ-. assert (ceil-log (to-nat dg + 1) ≤
                        ceil-log (maxDegree + 1)) >>=
                        (λ-. staged-composition-fri-program A dg 0

```

```

(ceil-log (to-nat dg + 1)) [] >=>
(λ(composition-roots, composition-bs).
  composition-final-stage A dg composition-bs >=>
  (λcomposition-final.
    record-staged-message
      composition-final >=>
      (λ-. checked-staged-query-program A
        trace-roots composition-roots 0
        rounds >=>
        (λquery-chunks.
          return
            (|staged-trace-root = fr,
              staged-trace-fri-roots =
                trace-roots,
              staged-trace-fri-challenges =
                trace-bs,
              staged-trace-final =
                trace-final,
              staged-alphas = as,
              staged-degree = dg,
              staged-composition-fri-roots =
                composition-roots,
              staged-composition-fri-challenges =
                composition-bs,
              staged-composition-final =
                composition-final,
              staged-query-chunks =
                query-chunks|)))))))))))))
using after-trace-root-record
[of fr
  λtrace-roots trace-bs trace-final as dg composition-roots
    composition-bs composition-final query-chunks.
    (|staged-trace-root = fr,
      staged-trace-fri-roots = trace-roots,
      staged-trace-fri-challenges = trace-bs,
      staged-trace-final = trace-final,
      staged-alphas = as,
      staged-degree = dg,
      staged-composition-fri-roots = composition-roots,
      staged-composition-fri-challenges = composition-bs,
      staged-composition-final = composition-final,
      staged-query-chunks = query-chunks|)]
by (simp add: sm-bind-assoc)
qed
have budget-eg:
  ?trace-root +
  (0 +
    (?trace-fri +
      (?trace-final +

```

```

      (0 +
        (?alpha +
          (?degree +
            (0 +
              (?composition-fri +
                (?composition-final + (0 + (?query + 0)))))))))) =
      staged-attacker-query-budget budgets + staged-challenge-query-budget
unfolding staged-attacker-query-budget-def
      staged-challenge-query-budget-def
by (simp add: add.assoc add.commute add.left-commute)
show ?thesis
      using whole unfolding budget-eq .
qed

lemma hash-target-program-staged-semantic-adversary:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    hash-target-program B
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
      (staged-semantic-adversary A)
proof -
  have hash-target-program B
    ((staged-attacker-query-budget budgets + staged-challenge-query-budget) +
     0)
    (staged-semantic-adversary A)
  unfolding staged-semantic-adversary-def
proof (rule hash-target-program-bind
  [where n=staged-attacker-query-budget budgets +
    staged-challenge-query-budget and n'=0])
  show hash-target-program B
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (staged-transcript-program A)
    by (rule hash-target-program-staged-transcript-program[OF wf controlled])
  show  $\bigwedge x.$  hash-target-program B 0
    (return (staged-proof-transcript x))
    by (rule hash-target-program-return)
qed
then show ?thesis by simp
qed

lemma hash-extension-preserving-staged-semantic-adversary:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows hash-extension-preserving (staged-semantic-adversary A)
by (rule hash-target-program-extension
  [OF hash-target-program-staged-semantic-adversary
    [OF wf controlled, of {}]])

```

lemma *admissible-staged-semantic-adversary*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
shows
admissible-adversary
(staged-attacker-query-budget budgets + staged-challenge-query-budget)
(staged-semantic-adversary A)
unfolding *admissible-adversary-def*
proof (*intro conjI allI*)
show *hash-extension-preserving (staged-semantic-adversary A)*
by (*rule hash-extension-preserving-staged-semantic-adversary*
[OF wf controlled])
show *hash-range-budget*
(staged-attacker-query-budget budgets + staged-challenge-query-budget)
(staged-semantic-adversary A)
by (*rule hash-range-budget-staged-semantic-adversary**[OF wf controlled]*)
show *hash-collision-budget*
(staged-attacker-query-budget budgets + staged-challenge-query-budget)
(staged-semantic-adversary A)
by (*rule hash-collision-budget-staged-semantic-adversary*
[OF wf controlled])
fix *B*
show *hash-target-program B*
(staged-attacker-query-budget budgets + staged-challenge-query-budget)
(staged-semantic-adversary A)
by (*rule hash-target-program-staged-semantic-adversary**[OF wf controlled]*)
qed

lemma *staged-trace-fri-program-Suc-outcomeE*:
assumes *outcome*:
Some ((roots, bs'), t) ∈
set-dist (execute (staged-trace-fri-program A i (Suc n) bs) s)
obtains *root b roots' s1 s2 s3* **where**
Some (root, s1) ∈
set-dist (execute (trace-fri-root-stage A i bs) s)
Some ((), s2) ∈
set-dist (execute (record-staged-message root) s1)
Some (b, s3) ∈
set-dist (execute receive-trace-fri-challenge s2)
Some ((roots', bs'), t) ∈
set-dist
(execute
(staged-trace-fri-program A (Suc i) n (bs @ [b])) s3)
roots = root # roots'
using *outcome*
by (*auto elim!: set-dist-bindE intro: that*)

lemma *staged-composition-fri-program-Suc-outcomeE*:
assumes *outcome*:

Some ((roots, bs'), t) ∈
set-dist
(execute
(staged-composition-fri-program A dg i (Suc n) bs) s)
obtains *root b roots' s1 s2 s3 where*
Some (root, s1) ∈
set-dist (execute (composition-fri-root-stage A dg i bs) s)
Some ((), s2) ∈
set-dist (execute (record-staged-message root) s1)
Some (b, s3) ∈
set-dist (execute receive-composition-fri-challenge s2)
Some ((roots', bs'), t) ∈
set-dist
(execute
(staged-composition-fri-program
A dg (Suc i) n (bs @ [b])) s3)
roots = root # roots'
using *outcome*
by *(auto elim!: set-dist-bindE intro: that)*

lemma *staged-alpha-program-Suc-outcomeE:*

assumes *outcome:*
Some (as, t) ∈
set-dist (execute (staged-alpha-program (Suc n)) s)
obtains *a as' s1 s2 where*
Some (a, s1) ∈
set-dist (execute receive-alpha-challenge s)
Some ((), s2) ∈
set-dist (execute (record-staged-message a) s1)
Some (as', t) ∈
set-dist (execute (staged-alpha-program n) s2)
as = a # as'
using *outcome*
by *(auto elim!: set-dist-bindE intro: that)*

lemma *staged-query-program-Suc-outcomeE:*

assumes *outcome:*
Some (chunks, t) ∈
set-dist (execute (staged-query-program A i (Suc n)) s)
obtains *raw chunk chunks' s1 s2 s3 where*
Some (raw, s1) ∈
set-dist (execute receive-query-index-challenge s)
Some (chunk, s2) ∈
set-dist (execute (query-opening-stage A i raw) s1)
Some ((), s3) ∈
set-dist (execute (record-staged-messages chunk) s2)
Some (chunks', t) ∈
set-dist (execute (staged-query-program A (Suc i) n) s3)
chunks = chunk # chunks'

using *outcome*
by (*auto elim!:* *set-dist-bindE* *intro:* *that*)

lemma *checked-staged-query-program-Suc-outcomeE:*

assumes *outcome:*

Some (chunks, t) ∈
set-dist
(execute
(checked-staged-query-program A trace-roots composition-roots
i (Suc n)) s)

obtains *raw chunk chunks' s1 s2 s3* **where**

Some (raw, s1) ∈
set-dist (execute receive-query-index-challenge s)
Some (chunk, s2) ∈
set-dist (execute (query-opening-stage A i raw) s1)
verifier-query-round-chunk (index (to-nat raw))
trace-roots composition-roots chunk

Some ((), s3) ∈
set-dist (execute (record-staged-messages chunk) s2)

Some (chunks', t) ∈
set-dist
(execute
(checked-staged-query-program A trace-roots composition-roots
(Suc i) n) s3)

chunks = chunk # chunks'

using *outcome*

unfolding *checked-staged-query-program.simps assert-def Let-def*

by (*auto simp:* *throw-no-outcome elim!:* *set-dist-bindE* *intro:* *that*
split: *if-splits*)

lemma *checked-staged-query-program-outcome-imp-staged-query-program:*

assumes *outcome:*

Some (chunks, t) ∈
set-dist
(execute
(checked-staged-query-program A trace-roots composition-roots
i n) s)

shows

Some (chunks, t) ∈
set-dist (execute (staged-query-program A i n) s)

using *outcome*

proof (*induction n arbitrary:* *i chunks s t*)

case *0*

then show *?case*

by *simp*

next

case *(Suc n)*

obtain *raw chunk chunks' s1 s2 s3* **where**

challenge-out:

Some (raw, s1) ∈
 set-dist (execute receive-query-index-challenge s)
and stage-out:
 Some (chunk, s2) ∈
 set-dist (execute (query-opening-stage A i raw) s1)
and record-out:
 Some ((), s3) ∈
 set-dist (execute (record-staged-messages chunk) s2)
and rest-out:
 Some (chunks', t) ∈
 set-dist
 (execute
 (checked-staged-query-program A trace-roots composition-roots
 (Suc i) n) s3)
and chunks-eq: chunks = chunk # chunks'
proof (rule checked-staged-query-program-Suc-outcomeE[OF Suc.prem])
fix raw chunk chunks' s1 s2 s3
assume challenge:
 Some (raw, s1) ∈
 set-dist (execute receive-query-index-challenge s)
and stage:
 Some (chunk, s2) ∈
 set-dist (execute (query-opening-stage A i raw) s1)
and recorded:
 Some ((), s3) ∈
 set-dist (execute (record-staged-messages chunk) s2)
and rest:
 Some (chunks', t) ∈
 set-dist
 (execute
 (checked-staged-query-program A trace-roots composition-roots
 (Suc i) n) s3)
and chunks: chunks = chunk # chunks'
show ?thesis
by (rule that[OF challenge stage recorded rest chunks])
qed
have rest-unchecked:
 Some (chunks', t) ∈
 set-dist (execute (staged-query-program A (Suc i) n) s3)
by (rule Suc.IH[OF rest-out])
show ?case
unfolding chunks-eq staged-query-program.simps
by (intro set-dist-bindI[OF challenge-out]
 set-dist-bindI[OF stage-out]
 set-dist-bindI[OF record-out]
 set-dist-bindI[OF rest-unchecked])
 simp
qed

lemma *checked-staged-transcript-program-outcome-imp-staged-transcript-program:*

assumes *outcome:*

Some (data, t) ∈
set-dist
(execute (checked-staged-transcript-program A) s)

shows

Some (data, t) ∈
set-dist (execute (staged-transcript-program A) s)

using *outcome*

unfolding *checked-staged-transcript-program-def staged-transcript-program-def*

Let-def

by (*auto elim!: set-dist-bindE*

intro!: set-dist-bindI

intro: checked-staged-query-program-outcome-imp-staged-query-program

split: prod.splits)

lemma

checked-staged-query-challenge-prefix-outcome-imp-staged-query-prefix:

assumes *outcome:*

Some (prefix, t) ∈
set-dist
(execute (checked-staged-query-challenge-prefix-program A i) s)

shows

Some ((sqp-degree prefix, sqp-composition-final prefix,
sqp-query-chunks prefix), t) ∈
set-dist (execute (staged-query-challenge-prefix-program A i) s)

using *outcome*

unfolding *checked-staged-query-challenge-prefix-program-def*

staged-query-challenge-prefix-program-def Let-def

by (*auto elim!: set-dist-bindE*

intro!: set-dist-bindI

intro: checked-staged-query-program-outcome-imp-staged-query-program

split: prod.splits)

lemma *checked-staged-query-program-wp-event-le-staged:*

assumes *none: ¬ P None*

shows

wp-event
(checked-staged-query-program A trace-roots composition-roots i n)
P s ≤
wp-event (staged-query-program A i n) P s

using *none*

proof (*induction n arbitrary: i s P*)

case *0*

then show *?case by simp*

next

case (*Suc n*)

let *?checked =*

checked-staged-query-program A trace-roots composition-roots i (Suc n)

```

let ?plain = staged-query-program A i (Suc n)
show ?case
  unfolding checked-staged-query-program.simps
    staged-query-program.simps Let-def
proof (rule wp-event-bind-mono-cont)
  show P None  $\implies$  P None .
next
  fix raw s1
  assume raw-out:
    Some (raw, s1)  $\in$ 
      set-dist (execute receive-query-index-challenge s)
  show
    wp-event
      (query-opening-stage A i raw  $\gg$ 
        ( $\lambda$ chunk.
          assert
            (verifier-query-round-chunk (index (to-nat raw))
              trace-roots composition-roots chunk)  $\gg$ 
            ( $\lambda$ -.
              record-staged-messages chunk  $\gg$ 
              ( $\lambda$ -.
                checked-staged-query-program A trace-roots
                  composition-roots (Suc i) n  $\gg$ 
                  ( $\lambda$ chunks. return (chunk # chunks)))))))
      P s1
     $\leq$ 
    wp-event
      (query-opening-stage A i raw  $\gg$ 
        ( $\lambda$ chunk.
          record-staged-messages chunk  $\gg$ 
          ( $\lambda$ -.
            staged-query-program A (Suc i) n  $\gg$ 
            ( $\lambda$ chunks. return (chunk # chunks))))))
      P s1
proof (rule wp-event-bind-mono-cont)
  show P None  $\implies$  P None .
next
  fix chunk s2
  assume chunk-out:
    Some (chunk, s2)  $\in$ 
      set-dist (execute (query-opening-stage A i raw) s1)
  let ?tailP =
     $\lambda$ out. case out of None  $\Rightarrow$  P None
      | Some (chunks, t)  $\Rightarrow$  P (Some (chunk # chunks, t))
  have tail-none:  $\neg$  ?tailP None
  using Suc.premis by simp
  have checked-tail-le:
     $\bigwedge s'$ .
    wp-event

```

```

    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n)
    ?tailP s'
  ≤
  wp-event (staged-query-program A (Suc i) n) ?tailP s'
by (rule Suc.IH) (rule tail-none)
have after-record:
  wp-event
    (record-staged-messages chunk ≫=
      (λ-.
        checked-staged-query-program A trace-roots
          composition-roots (Suc i) n ≫=
            (λchunks. return (chunk # chunks))))
    P s2
  ≤
  wp-event
    (record-staged-messages chunk ≫=
      (λ-.
        staged-query-program A (Suc i) n ≫=
          (λchunks. return (chunk # chunks))))
    P s2
proof (rule wp-event-bind-mono-cont)
  show P None ⇒ P None .
next
  fix u s3
  assume Some (u, s3) ∈
    set-dist (execute (record-staged-messages chunk) s2)
  show
    wp-event
      (checked-staged-query-program A trace-roots composition-roots
        (Suc i) n ≫=
          (λchunks. return (chunk # chunks)))
    P s3
  ≤
  wp-event
    (staged-query-program A (Suc i) n ≫=
      (λchunks. return (chunk # chunks)))
    P s3
  unfolding wp-event-bind-return-map
  using checked-tail-le[of s3]
  by simp
qed
have checked-assert:
  wp-event
    (assert
      (verifier-query-round-chunk (index (to-nat raw))
        trace-roots composition-roots chunk) ≫=
      (λ-.
        record-staged-messages chunk ≫=

```

```

      (λ-.
        checked-staged-query-program A trace-roots
        composition-roots (Suc i) n  $\gg$ =
        (λchunks. return (chunk # chunks))))))
    P s2
  ≤
  wp-event
  (record-staged-messages chunk  $\gg$ =
    (λ-.
      checked-staged-query-program A trace-roots
      composition-roots (Suc i) n  $\gg$ =
      (λchunks. return (chunk # chunks))))))
    P s2
  by (rule wp-event-assert-bind-le) (rule Suc.prem)
show
  wp-event
  (assert
    (verifier-query-round-chunk (index (to-nat raw))
      trace-roots composition-roots chunk)  $\gg$ =
    (λ-.
      record-staged-messages chunk  $\gg$ =
      (λ-.
        checked-staged-query-program A trace-roots
        composition-roots (Suc i) n  $\gg$ =
        (λchunks. return (chunk # chunks))))))
    P s2
  ≤
  wp-event
  (record-staged-messages chunk  $\gg$ =
    (λ-.
      staged-query-program A (Suc i) n  $\gg$ =
      (λchunks. return (chunk # chunks))))))
    P s2
  by (rule order-trans[OF checked-assert after-record])
qed
qed
qed

lemma checked-staged-transcript-program-wp-event-le-staged:
assumes none:  $\neg P \text{ None}$ 
shows
  wp-event (checked-staged-transcript-program A) P s ≤
  wp-event (staged-transcript-program A) P s
unfolding checked-staged-transcript-program-def staged-transcript-program-def
  Let-def
apply (rule wp-event-bind-mono-cont)
apply simp
apply (rename-tac fr s1)
apply (rule wp-event-bind-mono-cont)

```

```

  apply simp
  apply (rename-tac u1 s2)
  apply (rule wp-event-bind-mono-cont)
  apply simp
  apply (rename-tac trace-pair s3)
  apply (case-tac trace-pair)
  apply (rename-tac trace-roots trace-bs)
  apply simp
  apply (rule wp-event-bind-mono-cont)
  apply simp
  apply (rename-tac trace-final s4)
  apply (rule wp-event-bind-mono-cont)
  apply simp
  apply (rename-tac u2 s5)
  apply (rule wp-event-bind-mono-cont)
  apply simp
  apply (rename-tac as s6)
  apply (rule wp-event-bind-mono-cont)
  apply simp
  apply (rename-tac dg s7)
  apply (rule wp-event-bind-mono-cont)
  apply simp
  apply (rename-tac u3 s8)
  apply (rule wp-event-bind-mono-cont)
  apply simp
  apply (rename-tac u4 s9)
  apply (rule wp-event-bind-mono-cont)
  apply simp
  apply (rename-tac composition-pair s10)
  apply (case-tac composition-pair)
  apply (rename-tac composition-roots composition-bs)
  apply simp
  apply (rule wp-event-bind-mono-cont)
  apply simp
  apply (rename-tac composition-final s11)
  apply (rule wp-event-bind-mono-cont)
  apply simp
  apply (rename-tac u5 s12)
  apply (subst wp-event-bind-return-map)
  apply (subst wp-event-bind-return-map)
  apply (rule checked-staged-query-program-wp-event-le-staged)
  apply (simp add: none)
done

```

lemma *hash-range-budget-checked-staged-query-challenge-prefix-program:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i \leq \text{rounds}$*
shows


```

    hash-range-budget
      (staged-query-search-queries budgets i)
      (checked-staged-query-challenge-prefix-program A i)
  unfolding hash-range-budget-def
proof (intro allI impI)
fix s prefix t
assume outcome:
  Some (prefix, t) ∈
    set-dist
      (execute (checked-staged-query-challenge-prefix-program A i) s)
have unchecked-out:
  Some ((sqp-degree prefix, sqp-composition-final prefix,
    sqp-query-chunks prefix), t) ∈
    set-dist (execute (staged-query-challenge-prefix-program A i) s)
by (rule checked-staged-query-challenge-prefix-outcome-imp-staged-query-prefix
  [OF outcome])
have budget:
  hash-range-budget
    (staged-query-search-queries budgets i)
    (staged-query-challenge-prefix-program A i)
  by (rule hash-range-budget-staged-query-challenge-prefix-program
    [OF wf controlled i-bound])
show card (hash-map-output-values t)
  ≤ card (hash-map-output-values s) +
    staged-query-search-queries budgets i
using budget unchecked-out unfolding hash-range-budget-def by blast
qed

```

lemma *hash-collision-budget-checked-staged-query-challenge-prefix-program:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i ≤ rounds*

shows

hash-collision-budget
 (staged-query-search-queries budgets i)
 (checked-staged-query-challenge-prefix-program A i)

unfolding *hash-collision-budget-def*

proof (intro allI impI)

fix *s :: 'f protocol-channel*

assume *clean: ¬ hash-map-output-collision s*

have *checked-le:*

wp-event (checked-staged-query-challenge-prefix-program A i)
 (hash-new-collision-event s) *s* ≤
wp-event (staged-query-challenge-prefix-program A i)
 (hash-new-collision-event s) *s*

unfolding *checked-staged-query-challenge-prefix-program-def*
staged-query-challenge-prefix-program-def Let-def

apply (rule *wp-event-bind-mono-cont*)

apply (*simp add: hash-new-collision-event-def*)

```

apply (rename-tac alpha-prefix s1)
apply (case-tac alpha-prefix)
apply (rename-tac fr trace-roots trace-bs trace-final)
apply simp
apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-collision-event-def)
apply (rename-tac as s2)
apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-collision-event-def)
apply (rename-tac dg s3)
apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-collision-event-def)
apply (rename-tac u s4)
apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-collision-event-def)
apply (rename-tac u2 s5)
apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-collision-event-def)
apply (rename-tac composition-pair s6)
apply (case-tac composition-pair)
apply (rename-tac composition-roots composition-bs)
apply simp
apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-collision-event-def)
apply (rename-tac composition-final s7)
apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-collision-event-def)
apply (rename-tac u3 s8)
apply (subst wp-event-bind-return-map)
apply (subst wp-event-bind-return-map)
apply (simp add: hash-new-collision-event-def)
apply (rule checked-staged-query-program-wp-event-le-staged)
apply (simp add: hash-new-collision-event-def)
done
have unchecked-bound:
  wp-event (staged-query-challenge-prefix-program A i)
    (hash-new-collision-event s)  $s \leq$ 
    hash-collision-budget-value (card (hash-map-output-values s))
    (staged-query-search-queries budgets i)
using hash-collision-budget-staged-query-challenge-prefix-program
  [OF wf controlled i-bound] clean
unfolding hash-collision-budget-def by blast
show wp-event (checked-staged-query-challenge-prefix-program A i)
  (hash-new-collision-event s)  $s$ 
   $\leq$  hash-collision-budget-value (card (hash-map-output-values s))
  (staged-query-search-queries budgets i)
by (rule order-trans[OF checked-le unchecked-bound])
qed

```

lemma *hash-range-budget-checked-staged-transcript-program:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
hash-range-budget
(staged-attacker-query-budget budgets + staged-challenge-query-budget)
(checked-staged-transcript-program A)
unfolding *hash-range-budget-def*
proof (*intro allI impI*)
fix *s data t*
assume *outcome:*
Some (data, t) ∈
set-dist (execute (checked-staged-transcript-program A) s)
have *unchecked-out:*
Some (data, t) ∈ set-dist (execute (staged-transcript-program A) s)
by (*rule checked-staged-transcript-program-outcome-imp-staged-transcript-program*
[OF outcome])
have *budget:*
hash-range-budget
(staged-attacker-query-budget budgets + staged-challenge-query-budget)
(staged-transcript-program A)
by (*rule hash-range-budget-staged-transcript-program [OF wf controlled]*)
show *card (hash-map-output-values t)*
 $\leq$ *card (hash-map-output-values s) +*
(staged-attacker-query-budget budgets + staged-challenge-query-budget)
using *budget unchecked-out unfolding hash-range-budget-def by blast*
qed

lemma *hash-collision-budget-checked-staged-transcript-program:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
hash-collision-budget
(staged-attacker-query-budget budgets + staged-challenge-query-budget)
(checked-staged-transcript-program A)
unfolding *hash-collision-budget-def*
proof (*intro allI impI*)
fix *s :: 'f protocol-channel*
assume *clean: ¬ hash-map-output-collision s*
let *?budget =*
staged-attacker-query-budget budgets + staged-challenge-query-budget
have *checked-le:*
wp-event (checked-staged-transcript-program A)
(hash-new-collision-event s) s ≤
wp-event (staged-transcript-program A)
(hash-new-collision-event s) s
by (*rule checked-staged-transcript-program-wp-event-le-staged*)
(simp add: hash-new-collision-event-def)
have *unchecked-bound:*

$wp\text{-}event$ ($staged\text{-}transcript\text{-}program$ A)
 $(hash\text{-}new\text{-}collision\text{-}event$ $s)$ $s \leq$
 $hash\text{-}collision\text{-}budget\text{-}value$ ($card$ ($hash\text{-}map\text{-}output\text{-}values$ s)) $?budget$
using $hash\text{-}collision\text{-}budget\text{-}staged\text{-}transcript\text{-}program$ [OF wf $controlled$]
 $clean$
unfolding $hash\text{-}collision\text{-}budget\text{-}def$ **by** $blast$
show $wp\text{-}event$ ($checked\text{-}staged\text{-}transcript\text{-}program$ A)
 $(hash\text{-}new\text{-}collision\text{-}event$ $s)$ $s \leq$
 $hash\text{-}collision\text{-}budget\text{-}value$ ($card$ ($hash\text{-}map\text{-}output\text{-}values$ s)) $?budget$
by ($rule$ $order\text{-}trans$ [OF $checked\text{-}le$ $unchecked\text{-}bound$])
qed

lemma $checked\text{-}staged\text{-}query\text{-}prefix\text{-}target\text{-}hit\text{-}bound$:

assumes wf : $staged\text{-}budget\text{-}wellformed$ $budgets$
and $controlled$: $staged\text{-}adversary\text{-}controlled$ $budgets$ A
and $i\text{-}bound$: $i \leq rounds$

shows

$wp\text{-}event$ ($checked\text{-}staged\text{-}query\text{-}challenge\text{-}prefix\text{-}program$ A i)
 $(hash\text{-}new\text{-}output\text{-}hit\text{-}event$ B $s)$ $s \leq$
 $staged\text{-}phase\text{-}target\text{-}error$ B
 $(staged\text{-}query\text{-}search\text{-}queries$ $budgets$ $i)$

proof —

have $checked\text{-}le\text{-}unchecked$:

$wp\text{-}event$ ($checked\text{-}staged\text{-}query\text{-}challenge\text{-}prefix\text{-}program$ A i)
 $(hash\text{-}new\text{-}output\text{-}hit\text{-}event$ B $s)$ $s \leq$
 $wp\text{-}event$ ($staged\text{-}query\text{-}challenge\text{-}prefix\text{-}program$ A i)
 $(hash\text{-}new\text{-}output\text{-}hit\text{-}event$ B $s)$ s

unfolding $checked\text{-}staged\text{-}query\text{-}challenge\text{-}prefix\text{-}program\text{-}def$
 $staged\text{-}query\text{-}challenge\text{-}prefix\text{-}program\text{-}def$ $Let\text{-}def$

apply ($rule$ $wp\text{-}event\text{-}bind\text{-}mono\text{-}cont$)
apply ($simp$ add : $hash\text{-}new\text{-}output\text{-}hit\text{-}event\text{-}def$)
apply ($rename\text{-}tac$ $alpha\text{-}prefix$ $s1$)
apply ($case\text{-}tac$ $alpha\text{-}prefix$)
apply ($rename\text{-}tac$ fr $trace\text{-}roots$ $trace\text{-}bs$ $trace\text{-}final$)
apply $simp$
apply ($rule$ $wp\text{-}event\text{-}bind\text{-}mono\text{-}cont$)
apply ($simp$ add : $hash\text{-}new\text{-}output\text{-}hit\text{-}event\text{-}def$)
apply ($rename\text{-}tac$ as $s2$)
apply ($rule$ $wp\text{-}event\text{-}bind\text{-}mono\text{-}cont$)
apply ($simp$ add : $hash\text{-}new\text{-}output\text{-}hit\text{-}event\text{-}def$)
apply ($rename\text{-}tac$ dg $s3$)
apply ($rule$ $wp\text{-}event\text{-}bind\text{-}mono\text{-}cont$)
apply ($simp$ add : $hash\text{-}new\text{-}output\text{-}hit\text{-}event\text{-}def$)
apply ($rename\text{-}tac$ u $s4$)
apply ($rule$ $wp\text{-}event\text{-}bind\text{-}mono\text{-}cont$)
apply ($simp$ add : $hash\text{-}new\text{-}output\text{-}hit\text{-}event\text{-}def$)
apply ($rename\text{-}tac$ $u2$ $s5$)
apply ($rule$ $wp\text{-}event\text{-}bind\text{-}mono\text{-}cont$)
apply ($simp$ add : $hash\text{-}new\text{-}output\text{-}hit\text{-}event\text{-}def$)

```

apply (rename-tac composition-pair s6)
apply (case-tac composition-pair)
apply (rename-tac composition-roots composition-bs)
apply simp
apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-output-hit-event-def)
apply (rename-tac composition-final s7)
apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-output-hit-event-def)
apply (rename-tac u3 s8)
apply (subst wp-event-bind-return-map)
apply (subst wp-event-bind-return-map)
apply (simp add: hash-new-output-hit-event-def)
apply (rule checked-staged-query-program-wp-event-le-staged)
apply simp
done
also have ...  $\leq$ 
  staged-phase-target-error B
  (staged-query-search-queries budgets i)
  by (rule staged-query-prefix-target-hit-bound[OF wf controlled i-bound])
finally show ?thesis .
qed

```

```

lemma checked-staged-query-prefix-bad-index-target-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i  $\leq$  rounds
  shows
    wp-event (checked-staged-query-challenge-prefix-program A i)
    (hash-new-output-hit-event (query-index-raw-preimage B) s) s  $\leq$ 
    staged-query-prefix-bad-index-error budgets B i
  unfolding staged-query-prefix-bad-index-error-def
  by (rule checked-staged-query-prefix-target-hit-bound
    [OF wf controlled i-bound])

```

end

end

```

theory Staged-Security-Experiment-Checked-Prefix
  imports Staged-Security-Experiment-Targets
begin

```

Checked staged-prefix replay, alignment, and transcript extraction lemmas.

```

context soundness
begin

```

```

lemma hash-relation-program-checked-staged-query-challenge-prefix-program:

```

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i \leq \text{rounds}$ 
and fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
shows
  hash-relation-program R b
    (staged-query-search-queries budgets i)
    (checked-staged-query-challenge-prefix-program A i)
proof –
  let ?alpha-prefix = staged-alpha-search-queries budgets 0
  let ?alpha = length spec
  let ?degree = degree-budget budgets
  let ?composition-fri =
    sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
  let ?composition-final = composition-final-budget budgets
  let ?query = sum-list (take i (query-opening-budgets budgets)) + i
  have alpha-prefix:
    hash-relation-program R b ?alpha-prefix
      (staged-alpha-prefix-program A)
    by (rule hash-relation-program-staged-alpha-prefix-program
      [OF wf controlled fibers])
  have alpha:
    hash-relation-program R b ?alpha
      (staged-alpha-program (length spec))
    by (rule hash-relation-program-staged-alpha-program [OF fibers])
  have degree:
     $\bigwedge as. \text{hash-relation-program } R \ b \ ?degree \ (\text{degree-stage } A \ as)$ 
  proof –
    fix as
    have controlled-degree:
      controlled-ro-program ?degree (degree-stage A as)
      using controlled unfolding staged-adversary-controlled-def by blast
    show hash-relation-program R b ?degree (degree-stage A as)
      by (rule controlled-ro-program-relation
        [OF controlled-degree fibers])
  qed
  have composition-fri:
     $\bigwedge dg. \text{hash-relation-program } R \ b \ ?composition-fri$ 
    (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda -. \text{staged-composition-fri-program } A \ dg \ 0$ 
        (ceil-log (to-nat dg + 1)) []))
    by (rule hash-relation-program-guarded-staged-composition-fri-program
      [OF wf controlled fibers])
  have composition-final:
     $\bigwedge dg \text{ composition-bs. hash-relation-program } R \ b \ ?composition-final$ 
    (composition-final-stage A dg composition-bs)
  proof –
    fix dg composition-bs

```

```

have controlled-final:
  controlled-ro-program ?composition-final
    (composition-final-stage A dg composition-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
show hash-relation-program R b ?composition-final
  (composition-final-stage A dg composition-bs)
  by (rule controlled-ro-program-relation
    [OF controlled-final fibers])
qed
have query:
   $\bigwedge$ trace-roots composition-roots.
    hash-relation-program R b ?query
      (checked-staged-query-program A trace-roots composition-roots 0 i)
proof –
  fix trace-roots composition-roots
  have len: 0 + i ≤ length (query-opening-budgets budgets)
    using wf i-bound unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-relation-program R b
      (sum-list (take i (drop 0 (query-opening-budgets budgets))) + i)
      (checked-staged-query-program A trace-roots composition-roots 0 i)
    by (rule hash-relation-program-checked-staged-query-program
      [OF controlled len fibers])
  then show hash-relation-program R b ?query
    (checked-staged-query-program A trace-roots composition-roots 0 i)
    by simp
qed
have after-query:
   $\bigwedge$ fr trace-roots trace-bs trace-final as dg composition-roots
    composition-bs composition-final.
    hash-relation-program R b (?query + 0)
      (checked-staged-query-program A trace-roots composition-roots 0 i  $\gg$ 
        ( $\lambda$ query-chunks. return
          ( $\downarrow$ sqp-trace-root = fr,
            sqp-trace-fri-roots = trace-roots,
            sqp-trace-fri-challenges = trace-bs,
            sqp-trace-final = trace-final,
            sqp-alphas = as,
            sqp-degree = dg,
            sqp-composition-fri-roots = composition-roots,
            sqp-composition-fri-challenges = composition-bs,
            sqp-composition-final = composition-final,
            sqp-query-chunks = query-chunks)))
    by (rule hash-relation-program-bind)
      (rule query,
        rule hash-relation-program-zero[OF hash-map-preserving-return])
have after-composition-final-record:
   $\bigwedge$ fr trace-roots trace-bs trace-final as dg composition-roots
    composition-bs composition-final.

```

```

hash-relation-program R b (0 + (?query + 0))
(record-staged-message composition-final  $\gg$ 
  ( $\lambda$ -. checked-staged-query-program A trace-roots
    composition-roots 0 i  $\gg$ 
    ( $\lambda$ query-chunks. return
      (sqp-trace-root = fr,
        sqp-trace-fri-roots = trace-roots,
        sqp-trace-fri-challenges = trace-bs,
        sqp-trace-final = trace-final,
        sqp-alphas = as,
        sqp-degree = dg,
        sqp-composition-fri-roots = composition-roots,
        sqp-composition-fri-challenges = composition-bs,
        sqp-composition-final = composition-final,
        sqp-query-chunks = query-chunks))))))
by (rule hash-relation-program-bind)
(rule hash-relation-program-record-staged-message, rule after-query)
have after-composition-final:
 $\wedge$ fr trace-roots trace-bs trace-final as dg composition-roots
  composition-bs.
hash-relation-program R b
  (?composition-final + (0 + (?query + 0)))
  (composition-final-stage A dg composition-bs  $\gg$ 
    ( $\lambda$ composition-final.
      record-staged-message composition-final  $\gg$ 
      ( $\lambda$ -. checked-staged-query-program A trace-roots
        composition-roots 0 i  $\gg$ 
        ( $\lambda$ query-chunks. return
          (sqp-trace-root = fr,
            sqp-trace-fri-roots = trace-roots,
            sqp-trace-fri-challenges = trace-bs,
            sqp-trace-final = trace-final,
            sqp-alphas = as,
            sqp-degree = dg,
            sqp-composition-fri-roots = composition-roots,
            sqp-composition-fri-challenges = composition-bs,
            sqp-composition-final = composition-final,
            sqp-query-chunks = query-chunks))))))
by (rule hash-relation-program-bind)
(rule composition-final, rule after-composition-final-record)
have after-composition-fri:
 $\wedge$ fr trace-roots trace-bs trace-final as dg.
hash-relation-program R b
  (?composition-fri +
    (?composition-final + (0 + (?query + 0))))
  ((assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
    ( $\lambda$ -. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) []))  $\gg$ 
    ( $\lambda$ (composition-roots, composition-bs).
```



```

composition-final-stage A dg composition-bs ≫=
(λcomposition-final.
  record-staged-message composition-final ≫=
  (λ-. checked-staged-query-program A trace-roots
    composition-roots 0 i ≫=
    (λquery-chunks. return
      (λsqp-trace-root = fr,
        sqp-trace-fri-roots = trace-roots,
        sqp-trace-fri-challenges = trace-bs,
        sqp-trace-final = trace-final,
        sqp-alphas = as,
        sqp-degree = dg,
        sqp-composition-fri-roots = composition-roots,
        sqp-composition-fri-challenges = composition-bs,
        sqp-composition-final = composition-final,
        sqp-query-chunks = query-chunks))))))

proof –
fix fr trace-roots trace-bs trace-final as dg
have cont:
  ∧x. hash-relation-program R b
    (?composition-final + (0 + (?query + 0)))
    (case x of (composition-roots, composition-bs) ⇒
      composition-final-stage A dg composition-bs ≫=
      (λcomposition-final.
        record-staged-message composition-final ≫=
        (λ-. checked-staged-query-program A trace-roots
          composition-roots 0 i ≫=
          (λquery-chunks. return
            (λsqp-trace-root = fr,
              sqp-trace-fri-roots = trace-roots,
              sqp-trace-fri-challenges = trace-bs,
              sqp-trace-final = trace-final,
              sqp-alphas = as,
              sqp-degree = dg,
              sqp-composition-fri-roots = composition-roots,
              sqp-composition-fri-challenges = composition-bs,
              sqp-composition-final = composition-final,
              sqp-query-chunks = query-chunks))))))

proof –
fix x
show hash-relation-program R b
  (?composition-final + (0 + (?query + 0)))
  (case x of (composition-roots, composition-bs) ⇒
    composition-final-stage A dg composition-bs ≫=
    (λcomposition-final.
      record-staged-message composition-final ≫=
      (λ-. checked-staged-query-program A trace-roots
        composition-roots 0 i ≫=
        (λquery-chunks. return

```

```

    (|spp-trace-root = fr,
     spp-trace-fri-roots = trace-roots,
     spp-trace-fri-challenges = trace-bs,
     spp-trace-final = trace-final,
     spp-alphas = as,
     spp-degree = dg,
     spp-composition-fri-roots = composition-roots,
     spp-composition-fri-challenges = composition-bs,
     spp-composition-final = composition-final,
     spp-query-chunks = query-chunks|))))
proof (cases x)
  case (Pair composition-roots composition-bs)
  have step:
    hash-relation-program R b
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg composition-bs >>=
     (λcomposition-final.
      record-staged-message composition-final >>=
      (λ-. checked-staged-query-program A trace-roots
       composition-roots 0 i >>=
       (λquery-chunks. return
        (|spp-trace-root = fr,
         spp-trace-fri-roots = trace-roots,
         spp-trace-fri-challenges = trace-bs,
         spp-trace-final = trace-final,
         spp-alphas = as,
         spp-degree = dg,
         spp-composition-fri-roots = composition-roots,
         spp-composition-fri-challenges = composition-bs,
         spp-composition-final = composition-final,
         spp-query-chunks = query-chunks|))))))
    by (rule after-composition-final)
  show ?thesis
    unfolding Pair using step by simp
  qed
qed
show hash-relation-program R b
  (?composition-fri +
   (?composition-final + (0 + (?query + 0))))
  ((assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
   (λ-. staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) [])) >>=
   (λ(composition-roots, composition-bs).
    composition-final-stage A dg composition-bs >>=
    (λcomposition-final.
     record-staged-message composition-final >>=
     (λ-. checked-staged-query-program A trace-roots
      composition-roots 0 i >>=
      (λquery-chunks. return

```

```

      (|sqp-trace-root = fr,
       sqp-trace-fri-roots = trace-roots,
       sqp-trace-fri-challenges = trace-bs,
       sqp-trace-final = trace-final,
       sqp-alphas = as,
       sqp-degree = dg,
       sqp-composition-fri-roots = composition-roots,
       sqp-composition-fri-challenges = composition-bs,
       sqp-composition-final = composition-final,
       sqp-query-chunks = query-chunks|))))))
  by (rule hash-relation-program-bind)
    (rule composition-fri, rule cont)
qed
have after-degree-record:
   $\wedge$ fr trace-roots trace-bs trace-final as dg.
  hash-relation-program R b
  (0 + (?composition-fri +
    (?composition-final + (0 + (?query + 0)))))
  (record-staged-message dg  $\gg$ 
    ( $\lambda$ -. (assert
      (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda$ -. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))  $\gg$ 
      ( $\lambda$ (composition-roots, composition-bs).
        composition-final-stage A dg composition-bs  $\gg$ 
        ( $\lambda$ composition-final.
          record-staged-message composition-final  $\gg$ 
          ( $\lambda$ -. checked-staged-query-program A trace-roots
            composition-roots 0 i  $\gg$ 
            ( $\lambda$ query-chunks. return
              (|sqp-trace-root = fr,
               sqp-trace-fri-roots = trace-roots,
               sqp-trace-fri-challenges = trace-bs,
               sqp-trace-final = trace-final,
               sqp-alphas = as,
               sqp-degree = dg,
               sqp-composition-fri-roots = composition-roots,
               sqp-composition-fri-challenges = composition-bs,
               sqp-composition-final = composition-final,
               sqp-query-chunks = query-chunks|)))))))
  by (rule hash-relation-program-bind)
    (rule hash-relation-program-record-staged-message,
     rule after-composition-fri)
have after-degree:
   $\wedge$ fr trace-roots trace-bs trace-final as.
  hash-relation-program R b
  (?degree + (0 + (?composition-fri +
    (?composition-final + (0 + (?query + 0)))))
  (degree-stage A as  $\gg$ 

```

```

( $\lambda$ dg. record-staged-message dg  $\gg$ 
( $\lambda$ -. let composition-rounds = ceil-log (to-nat dg + 1) in
  assert (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
( $\lambda$ -. staged-composition-fri-program A dg 0
  composition-rounds  $\square$   $\gg$ 
( $\lambda$ (composition-roots, composition-bs).
  composition-final-stage A dg composition-bs  $\gg$ 
( $\lambda$ composition-final.
  record-staged-message composition-final  $\gg$ 
( $\lambda$ -. checked-staged-query-program A trace-roots
  composition-roots 0 i  $\gg$ 
( $\lambda$ query-chunks. return
  ( $\lambda$ sqp-trace-root = fr,
    sqp-trace-fri-roots = trace-roots,
    sqp-trace-fri-challenges = trace-bs,
    sqp-trace-final = trace-final,
    sqp-alphas = as,
    sqp-degree = dg,
    sqp-composition-fri-roots = composition-roots,
    sqp-composition-fri-challenges = composition-bs,
    sqp-composition-final = composition-final,
    sqp-query-chunks = query-chunks)))))))))

```

proof –

fix fr trace-roots trace-bs trace-final as

have cont:

```

 $\wedge$ dg. hash-relation-program R b
(0 + (?composition-fri +
  (?composition-final + (0 + (?query + 0))))))
(record-staged-message dg  $\gg$ 
( $\lambda$ -. assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
( $\lambda$ -. staged-composition-fri-program A dg 0
  (ceil-log (to-nat dg + 1))  $\square$   $\gg$ 
( $\lambda$ (composition-roots, composition-bs).
  composition-final-stage A dg composition-bs  $\gg$ 
( $\lambda$ composition-final.
  record-staged-message composition-final  $\gg$ 
( $\lambda$ -. checked-staged-query-program A trace-roots
  composition-roots 0 i  $\gg$ 
( $\lambda$ query-chunks. return
  ( $\lambda$ sqp-trace-root = fr,
    sqp-trace-fri-roots = trace-roots,
    sqp-trace-fri-challenges = trace-bs,
    sqp-trace-final = trace-final,
    sqp-alphas = as,
    sqp-degree = dg,
    sqp-composition-fri-roots = composition-roots,
    sqp-composition-fri-challenges = composition-bs,
    sqp-composition-final = composition-final,
    sqp-query-chunks = query-chunks)))))))))

```

```

proof –
  fix dg
  have step:
    hash-relation-program R b
    (0 + (?composition-fri +
      (?composition-final + (0 + (?query + 0)))))
    (record-staged-message dg  $\gg$ 
      (λ-. (assert
        (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))  $\gg$ 
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []))  $\gg$ 
        (λ(composition-roots, composition-bs).
          composition-final-stage A dg composition-bs  $\gg$ 
          (λcomposition-final.
            record-staged-message composition-final  $\gg$ 
            (λ-. checked-staged-query-program A trace-roots
              composition-roots 0 i  $\gg$ 
              (λquery-chunks. return
                (|spp-trace-root = fr,
                  spp-trace-fri-roots = trace-roots,
                  spp-trace-fri-challenges = trace-bs,
                  spp-trace-final = trace-final,
                  spp-alphas = as,
                  spp-degree = dg,
                  spp-composition-fri-roots = composition-roots,
                  spp-composition-fri-challenges = composition-bs,
                  spp-composition-final = composition-final,
                  spp-query-chunks = query-chunks|)))))))
    by (rule after-degree-record)
  show hash-relation-program R b
  (0 + (?composition-fri +
    (?composition-final + (0 + (?query + 0)))))
  (record-staged-message dg  $\gg$ 
    (λ-. assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))  $\gg$ 
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []  $\gg$ 
        (λ(composition-roots, composition-bs).
          composition-final-stage A dg composition-bs  $\gg$ 
          (λcomposition-final.
            record-staged-message composition-final  $\gg$ 
            (λ-. checked-staged-query-program A trace-roots
              composition-roots 0 i  $\gg$ 
              (λquery-chunks. return
                (|spp-trace-root = fr,
                  spp-trace-fri-roots = trace-roots,
                  spp-trace-fri-challenges = trace-bs,
                  spp-trace-final = trace-final,
                  spp-alphas = as,
                  spp-degree = dg,

```

```

      sqp-composition-fri-roots = composition-roots,
      sqp-composition-fri-challenges = composition-bs,
      sqp-composition-final = composition-final,
      sqp-query-chunks = query-chunks)))))))))
  using step by (simp add: sm-bind-assoc)
qed
show hash-relation-program R b
  (?degree + (0 + (?composition-fri +
    (?composition-final + (0 + (?query + 0)))))))
  (degree-stage A as >=
    (λdg. record-staged-message dg >=
      (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
        assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >=
          (λ-. staged-composition-fri-program A dg 0
            composition-rounds [] >=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs >=
                  (λcomposition-final.
                    record-staged-message composition-final >=
                      (λ-. checked-staged-query-program A trace-roots
                        composition-roots 0 i >=
                          (λquery-chunks. return
                            (sqp-trace-root = fr,
                              sqp-trace-fri-roots = trace-roots,
                              sqp-trace-fri-challenges = trace-bs,
                              sqp-trace-final = trace-final,
                              sqp-alphas = as,
                              sqp-degree = dg,
                              sqp-composition-fri-roots = composition-roots,
                              sqp-composition-fri-challenges = composition-bs,
                              sqp-composition-final = composition-final,
                              sqp-query-chunks = query-chunks))))))))))
  unfolding Let-def
  by (rule hash-relation-program-bind)
    (rule degree, rule cont)
qed
have after-alpha:
  ∧fr trace-roots trace-bs trace-final.
  hash-relation-program R b
    (?alpha + (?degree + (0 + (?composition-fri +
      (?composition-final + (0 + (?query + 0)))))))
    (staged-alpha-program (length spec) >=
      (λas. degree-stage A as >=
        (λdg. record-staged-message dg >=
          (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
            assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >=
              (λ-. staged-composition-fri-program A dg 0
                composition-rounds [] >=
                  (λ(composition-roots, composition-bs).

```

```

composition-final-stage A dg composition-bs >>=
  (λcomposition-final.
    record-staged-message composition-final >>=
    (λ-. checked-staged-query-program A trace-roots
      composition-roots 0 i >>=
      (λquery-chunks. return
        (λspp-trace-root = fr,
          spp-trace-fri-roots = trace-roots,
          spp-trace-fri-challenges = trace-bs,
          spp-trace-final = trace-final,
          spp-alphas = as,
          spp-degree = dg,
          spp-composition-fri-roots = composition-roots,
          spp-composition-fri-challenges = composition-bs,
          spp-composition-final = composition-final,
          spp-query-chunks = query-chunks)))))))))
by (rule hash-relation-program-bind)
  (rule alpha, rule after-degree)
have after-alpha-prefix:
  hash-relation-program R b
    (?alpha-prefix +
      (?alpha + (?degree + (0 + (?composition-fri +
        (?composition-final + (0 + (?query + 0))))))))))
    (checked-staged-query-challenge-prefix-program A i)
  unfolding checked-staged-query-challenge-prefix-program-def
proof (rule hash-relation-program-bind)
show hash-relation-program R b ?alpha-prefix
  (staged-alpha-prefix-program A)
  by (rule alpha-prefix)
next
fix x
show hash-relation-program R b
  (?alpha + (?degree + (0 + (?composition-fri +
    (?composition-final + (0 + (?query + 0))))))))
  (case x of (fr, trace-roots, trace-bs, trace-final) =>
    staged-alpha-program (length spec) >>=
    (λas. degree-stage A as >>=
      (λdg. record-staged-message dg >>=
        (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
          assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
          (λ-. staged-composition-fri-program A dg 0
            composition-rounds [] >>=
            (λ(composition-roots, composition-bs).
              composition-final-stage A dg composition-bs >>=
              (λcomposition-final.
                record-staged-message composition-final >>=
                (λ-. checked-staged-query-program A trace-roots
                  composition-roots 0 i >>=
                  (λquery-chunks. return

```

```

    (|spp-trace-root = fr,
     spp-trace-fri-roots = trace-roots,
     spp-trace-fri-challenges = trace-bs,
     spp-trace-final = trace-final,
     spp-alphas = as,
     spp-degree = dg,
     spp-composition-fri-roots = composition-roots,
     spp-composition-fri-challenges = composition-bs,
     spp-composition-final = composition-final,
     spp-query-chunks = query-chunks|)))))))))
proof (cases x)
  case (fields fr trace-roots trace-bs trace-final)
  have step:
    hash-relation-program R b
    (?alpha + (?degree + (0 + (?composition-fri +
      (?composition-final + (0 + (?query + 0)))))))
    (staged-alpha-program (length spec) >>=
      (λas. degree-stage A as >>=
        (λdg. record-staged-message dg >>=
          (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
            assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                composition-rounds [] >>=
                  (λ(composition-roots, composition-bs).
                    composition-final-stage A dg composition-bs >>=
                      (λcomposition-final.
                        record-staged-message composition-final >>=
                          (λ-. checked-staged-query-program A trace-roots
                            composition-roots 0 i >>=
                              (λquery-chunks. return
                                (|spp-trace-root = fr,
                                 spp-trace-fri-roots = trace-roots,
                                 spp-trace-fri-challenges = trace-bs,
                                 spp-trace-final = trace-final,
                                 spp-alphas = as,
                                 spp-degree = dg,
                                 spp-composition-fri-roots = composition-roots,
                                 spp-composition-fri-challenges = composition-bs,
                                 spp-composition-final = composition-final,
                                 spp-query-chunks = query-chunks|)))))))))
          by (rule after-alpha)
        show ?thesis
        unfolding fields using step by simp
      qed
    qed
  show ?thesis
  using after-alpha-prefix
  unfolding staged-query-search-queries-def
    staged-alpha-search-queries-def

```



```

    by (simp add: add.assoc add.commute add.left-commute)
qed

lemma staged-trace-fri-program-challenge-prefix-pre-alignment:
  assumes outcome:
    Some ((roots, bs'), t) ∈
      set-dist (execute (staged-trace-fri-program A i n bs) s)
  shows take (length bs) bs' = bs
  using outcome
proof (induction n arbitrary: i bs roots bs' s t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.premis obtain root b roots' s1 s2 s3 where
    rest-out:
      Some ((roots', bs'), t) ∈
        set-dist
          (execute
            (staged-trace-fri-program A (Suc i) n (bs @ [b])) s3)
  unfolding staged-trace-fri-program.simps
  by (auto elim!: set-dist-bindE)
have prefix-suc:
  take (length (bs @ [b])) bs' = bs @ [b]
  by (rule Suc.IH[OF rest-out])
have take (length bs) bs' =
  take (length bs) (take (length (bs @ [b])) bs')
  by simp
also have ... = bs
  using prefix-suc by simp
finally show ?case .
qed

```

```

lemma staged-trace-fri-program-alignment:
  assumes controlled: staged-adversary-controlled budgets A
  and bound: i + n ≤ length (trace-fri-budgets budgets)
  and outcome:
    Some ((roots, bs'), t) ∈
      set-dist (execute (staged-trace-fri-program A i n bs) s)
  shows
    length roots = n ∧
    length bs' = length bs + n ∧
    PTranscript t = PTranscript s @ roots ∧
    PState t = foldl concat (PState s) roots ∧
    s ≤ t ∧
    PTraceFriCounter t = PTraceFriCounter s + n ∧
    PCompositionFriCounter t = PCompositionFriCounter s ∧
    PAlphaCounter t = PAlphaCounter s ∧
    PQueryCounter t = PQueryCounter s ∧

```

```

  (∀ j < n.
    fmllookup (HashMap t)
      (TraceFriChallenge (PTraceFriCounter s + j)
        (foldl concat (PState s) (take (Suc j) roots))) =
        Some (bs' ! (length bs + j)))
  using bound outcome
proof (induction n arbitrary: i bs roots bs' s t)
  case 0
  then show ?case by (simp add: hash-ext-refl)
next
  case (Suc n)
  from staged-trace-fri-program-Suc-outcomeE[OF Suc.prem1(2)]
  obtain root b roots' s1 s2 s3 where
    stage-out:
      Some (root, s1) ∈
        set-dist (execute (trace-fri-root-stage A i bs) s)
  and record-out:
      Some ((), s2) ∈
        set-dist (execute (record-staged-message root) s1)
  and challenge-out:
      Some (b, s3) ∈
        set-dist (execute receive-trace-fri-challenge s2)
  and rest-out:
      Some ((roots', bs'), t) ∈
        set-dist
          (execute
            (staged-trace-fri-program A (Suc i) n (bs @ [b])) s3)
  and roots-eq: roots = root # roots'
  .
have i-bound: i < length (trace-fri-budgets budgets)
  using Suc.prem1(1) by simp
have stage-controlled:
  controlled-ro-program (trace-fri-budgets budgets ! i)
    (trace-fri-root-stage A i bs)
  using controlled i-bound
  unfolding staged-adversary-controlled-def by blast
have stage-fields:
  PState s1 = PState s ∧
  PTranscript s1 = PTranscript s ∧
  PTraceFriCounter s1 = PTraceFriCounter s ∧
  PCompositionFriCounter s1 = PCompositionFriCounter s ∧
  PAlphaCounter s1 = PAlphaCounter s ∧
  PQueryCounter s1 = PQueryCounter s
  using controlled-stage-outcome-fields[OF stage-controlled stage-out]
  by simp
have ext-s-s1: s ≤ s1
  using controlled-ro-program-extension[OF stage-controlled] stage-out
  unfolding hash-extension-preserving-def by blast
have s2:

```

```

s2 = s1(|
  PState := concat (PState s1) root,
  PTranscript := PTranscript s1 @ [root])|)
by (rule record-staged-message-outcome[OF record-out])
have ext-s1-s2: s1 ≤ s2
  unfolding s2 less-eq-hash-ext-def less-eq-fmap-def by simp
have s2-fields:
  PState s2 = concat (PState s) root ∧
  PTranscript s2 = PTranscript s @ [root] ∧
  PTraceFriCounter s2 = PTraceFriCounter s ∧
  PCompositionFriCounter s2 = PCompositionFriCounter s ∧
  PAlphaCounter s2 = PAlphaCounter s ∧
  PQueryCounter s2 = PQueryCounter s
  unfolding s2 using stage-fields by simp
have challenge-fields:
  PState s3 = PState s2 ∧
  PTranscript s3 = PTranscript s2
  using receive-trace-fri-challenge-outcome[OF challenge-out] by simp
have ext-s2-s3: s2 ≤ s3
  using receive-trace-fri-challenge-outcome[OF challenge-out] by simp
have challenge-counters:
  PTraceFriCounter s3 = Suc (PTraceFriCounter s2) ∧
  PCompositionFriCounter s3 = PCompositionFriCounter s2 ∧
  PAlphaCounter s3 = PAlphaCounter s2 ∧
  PQueryCounter s3 = PQueryCounter s2
  using receive-trace-fri-challenge-counter-outcome[OF challenge-out] by simp
have rest-bound:
  Suc i + n ≤ length (trace-fri-budgets budgets)
  using Suc.premis(1) by simp
have rest-alignment:
  length roots' = n ∧
  length bs' = length (bs @ [b]) + n ∧
  PTranscript t = PTranscript s3 @ roots' ∧
  PState t = foldl concat (PState s3) roots' ∧
  s3 ≤ t ∧
  PTraceFriCounter t = PTraceFriCounter s3 + n ∧
  PCompositionFriCounter t = PCompositionFriCounter s3 ∧
  PAlphaCounter t = PAlphaCounter s3 ∧
  PQueryCounter t = PQueryCounter s3 ∧
  (∀ j < n.
    fmllookup (HashMap t)
      (TraceFriChallenge (PTraceFriCounter s3 + j)
        (foldl concat (PState s3) (take (Suc j) roots')))) =
    Some (bs' ! (length (bs @ [b]) + j)))
  by (rule Suc.IH[OF rest-bound rest-out])
have ext-s3-t: s3 ≤ t
  using rest-alignment by simp
have ext-s-t: s ≤ t
  by (rule hash-ext-trans[OF ext-s-s1])

```

```

    (rule hash-ext-trans[OF ext-s1-s2],
     rule hash-ext-trans[OF ext-s2-s3 ext-s3-t])
  have challenge-lookup-s3:
    fmlookup (HashMap s3)
      (TraceFriChallenge (PTraceFriCounter s)
        (concat (PState s) root)) = Some b
  using receive-trace-fri-challenge-outcome[OF challenge-out]
    s2-fields by simp
  have challenge-lookup-t:
    fmlookup (HashMap t)
      (TraceFriChallenge (PTraceFriCounter s)
        (concat (PState s) root)) = Some b
  by (rule hash-extension-lookup[OF challenge-lookup-s3 ext-s3-t])
  have prefix-suc:
    take (length (bs @ [b])) bs' = bs @ [b]
  by (rule staged-trace-fri-program-challenge-prefix-pre-alignment
      [OF rest-out])
  have lookup-all:
     $\forall j < \text{Suc } n.$ 
    fmlookup (HashMap t)
      (TraceFriChallenge (PTraceFriCounter s + j)
        (foldl concat (PState s) (take (Suc j) (root # roots')))) =
      Some (bs' ! (length bs + j))
  proof (intro allI impI)
    fix j
    assume j-bound:  $j < \text{Suc } n$ 
    show
      fmlookup (HashMap t)
        (TraceFriChallenge (PTraceFriCounter s + j)
          (foldl concat (PState s) (take (Suc j) (root # roots')))) =
        Some (bs' ! (length bs + j))
  proof (cases j)
    case 0
    have bs' ! length bs = b
    proof -
      have idx-lt:  $\text{length } bs < \text{length } (bs @ [b])$ 
      by simp
      have bs' ! length bs =
        take (length (bs @ [b])) bs' ! length bs
      using idx-lt by simp
      also have ... = (bs @ [b]) ! length bs
      using prefix-suc by simp
      also have ... = b
      by simp
      finally show ?thesis .
    qed
    then show ?thesis
      using challenge-lookup-t unfolding 0 by simp
  next

```

```

    case (Suc k)
    then have k-bound:  $k < n$ 
      using j-bound by simp
    show ?thesis
      using rest-alignment k-bound s2-fields challenge-fields
        challenge-counters
      unfolding Suc
      by simp
  qed
qed
show ?case
  unfolding roots-eq
  using s2-fields challenge-fields challenge-counters rest-alignment
    ext-s-t lookup-all
  by simp
qed

```

```

lemma staged-trace-fri-program-challenge-prefix:
  assumes outcome:
    Some ((roots, bs'), t) ∈
      set-dist (execute (staged-trace-fri-program A i n bs) s)
  shows take (length bs) bs' = bs
  using outcome
proof (induction n arbitrary: i bs roots bs' s t)
  case 0
  then show ?case by (simp add: hash-ext-refl)
next
  case (Suc n)
  from Suc.premis obtain root b roots' s1 s2 s3 where
    rest-out:
      Some ((roots', bs'), t) ∈
        set-dist
          (execute
            (staged-trace-fri-program A (Suc i) n (bs @ [b])) s3)
  unfolding staged-trace-fri-program.simps
  by (auto elim!: set-dist-bindE)
  have prefix-suc:
    take (length (bs @ [b])) bs' = bs @ [b]
  by (rule Suc.IH[OF rest-out])
  have take (length bs) bs' =
    take (length bs) (take (length (bs @ [b])) bs')
  by simp
  also have ... = bs
  using prefix-suc by simp
  finally show ?case .
qed

```

```

lemma staged-trace-fri-program-challenge-length:
  assumes outcome:

```

$\text{Some } ((\text{roots}, \text{bs}'), t) \in$
 $\text{set-dist } (\text{execute } (\text{staged-trace-fri-program } A \ i \ n \ \text{bs}) \ s)$
shows $\text{length } \text{bs}' = \text{length } \text{bs} + n$
using *outcome*
proof (*induction n arbitrary: i bs roots bs' s t*)
case 0
then show ?case **by** (*simp add: hash-ext-refl*)
next
case (*Suc n*)
from *Suc.prem*s **obtain** *root b roots' s1 s2 s3* **where**
rest-out:
 $\text{Some } ((\text{roots}', \text{bs}'), t) \in$
 set-dist
 $(\text{execute}$
 $(\text{staged-trace-fri-program } A \ (\text{Suc } i) \ n \ (\text{bs} @ [b])) \ s3)$
unfolding *staged-trace-fri-program.simp*s
by (*auto elim!: set-dist-bindE*)
have $\text{length } \text{bs}' = \text{length } (\text{bs} @ [b]) + n$
by (*rule Suc.IH[OF rest-out]*)
then show ?case **by** *simp*
qed

lemma *staged-alpha-prefix-program-trace-challenge-length:*
assumes *outcome:*
 $\text{Some } ((\text{fr}, \text{trace-roots}, \text{trace-bs}, \text{trace-final}), t) \in$
 $\text{set-dist } (\text{execute } (\text{staged-alpha-prefix-program } A) \ s)$
shows $\text{length } \text{trace-bs} = \text{ceil-log } \text{clength}$
proof –
from *outcome* **obtain** *s1 s2 s3* **where**
fri-out:
 $\text{Some } ((\text{trace-roots}, \text{trace-bs}), s2) \in$
 set-dist
 $(\text{execute}$
 $(\text{staged-trace-fri-program } A \ 0 \ (\text{ceil-log } \text{clength}) \ []) \ s1)$
unfolding *staged-alpha-prefix-program-def*
by (*auto elim!: set-dist-bindE*)
have $\text{length } \text{trace-bs} = \text{length } ([] :: 'f \text{ list}) + \text{ceil-log } \text{clength}$
by (*rule staged-trace-fri-program-challenge-length[OF fri-out]*)
then show ?thesis **by** *simp*
qed

lemma *staged-alpha-prefix-trace-vector-member-imp-position-value:*
assumes *outcome:*
 $\text{Some } ((\text{fr}, \text{trace-roots}, \text{trace-bs}, \text{trace-final}), t) \in$
 $\text{set-dist } (\text{execute } (\text{staged-alpha-prefix-program } A) \ s)$
and *subset:* $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$
and *member:* $\text{trace-bs} \in B$
and *i-bound:* $i < \text{ceil-log } \text{clength}$
shows $\text{trace-bs} ! i \in \text{fri-vector-position-values } B \ i$

by (rule fri-vector-member-imp-position-value
 [OF subset member i-bound])

lemma staged-alpha-prefix-bad-trace-vector-coordinate-cover-bound:

assumes subset: $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$

and nonempty: $0 < \text{ceil-log } \text{clength}$

shows

$\text{wp-event } (\text{staged-alpha-prefix-program } A)$
 $(\lambda \text{out. case out of None} \Rightarrow \text{False}$
 $\quad | \text{Some } ((-, -, \text{trace-bs}, -), -) \Rightarrow \text{trace-bs} \in B) s \leq$
 $(\sum i < \text{ceil-log } \text{clength.}$
 $\quad \text{wp-event } (\text{staged-alpha-prefix-program } A)$
 $\quad (\lambda \text{out. case out of None} \Rightarrow \text{False}$
 $\quad \quad | \text{Some } ((-, -, \text{trace-bs}, -), -) \Rightarrow$
 $\quad \quad \text{trace-bs ! } i \in \text{fri-vector-position-values } B \ i) s)$

proof –

let ?bad =

$\lambda \text{out. case out of None} \Rightarrow \text{False}$
 $\quad | \text{Some } ((-, -, \text{trace-bs}, -), -) \Rightarrow \text{trace-bs} \in B$

let ?coord =

$\lambda i \text{ out. case out of None} \Rightarrow \text{False}$
 $\quad | \text{Some } ((-, -, \text{trace-bs}, -), -) \Rightarrow$
 $\quad \text{trace-bs ! } i \in \text{fri-vector-position-values } B \ i$

have bad-to-coord:

$\text{wp-event } (\text{staged-alpha-prefix-program } A) \ ?\text{bad } s \leq$
 $\text{wp-event } (\text{staged-alpha-prefix-program } A)$
 $(\lambda \text{out. } \exists i \in \{..<\text{ceil-log } \text{clength}\}. \ ?\text{coord } i \ \text{out}) \ s$

proof (rule wp-event-mono)

fix out

assume bad: ?bad out

show $\exists i \in \{..<\text{ceil-log } \text{clength}\}. \ ?\text{coord } i \ \text{out}$

proof (cases out)

case None

then show ?thesis **using** bad **by** simp

next

case (Some result)

then obtain fr trace-roots trace-bs trace-final t **where**

out-eq:

$\text{out} = \text{Some } ((\text{fr}, \text{trace-roots}, \text{trace-bs}, \text{trace-final}), t)$

by (cases result, auto split: prod.splits)

have member: trace-bs $\in B$

using bad **unfolding** out-eq **by** simp

from fri-vector-member-coordinate-cover[OF subset member nonempty]

obtain i **where** i-bound: $i < \text{ceil-log } \text{clength}$

and coord: trace-bs ! $i \in \text{fri-vector-position-values } B \ i$

by blast

show ?thesis

by (rule bexI[of - i]) (use i-bound coord out-eq **in** simp-all)

qed

qed
also have ... $\leq$
 $(\sum i < \text{ceil-log } \text{clength}.$
 $\text{wp-event } (\text{staged-alpha-prefix-program } A) \text{ } (?coord \ i) \ s)$
by (rule *wp-event-finite-UN-bound*) *simp-all*
finally show ?thesis .
qed

lemma *staged-alpha-prefix-trace-coordinate-hit-imp-new-output-hit*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < \text{ceil-log } \text{clength}$
and *absent*:
 $\text{trace-fri-challenge-values-absent}$
 $(\text{fri-vector-position-values } B \ i) \ s$
and *outcome*:
 $\text{Some } ((fr, \text{trace-roots}, \text{trace-bs}, \text{trace-final}), t) \in$
 $\text{set-dist } (\text{execute } (\text{staged-alpha-prefix-program } A) \ s)$
and *hit*: $\text{trace-bs} \ ! \ i \in \text{fri-vector-position-values } B \ i$
shows
 $\text{hash-new-output-hit-event } (\text{fri-vector-position-values } B \ i) \ s$
 $(\text{Some } ((fr, \text{trace-roots}, \text{trace-bs}, \text{trace-final}), t))$
proof –
from *outcome* **obtain** $s1 \ s2 \ s3 \ s4$ **where**
 root-out:
 $\text{Some } (fr, s1) \in \text{set-dist } (\text{execute } (\text{trace-root-stage } A) \ s)$
and *record-root*:
 $\text{Some } ((), s2) \in \text{set-dist } (\text{execute } (\text{record-staged-message } fr) \ s1)$
and *fri-out*:
 $\text{Some } ((\text{trace-roots}, \text{trace-bs}), s3) \in$
 set-dist
 $(\text{execute}$
 $(\text{staged-trace-fri-program } A \ 0 \ (\text{ceil-log } \text{clength}) \ []) \ s2)$
and *final-out*:
 $\text{Some } (\text{trace-final}, s4) \in$
 $\text{set-dist } (\text{execute } (\text{trace-final-stage } A \ \text{trace-bs}) \ s3)$
and *record-final*:
 $\text{Some } ((), t) \in$
 $\text{set-dist } (\text{execute } (\text{record-staged-message } \text{trace-final}) \ s4)$
unfolding *staged-alpha-prefix-program-def*
by (auto elim!: *set-dist-bindE*)
have *root-controlled*:
 $\text{controlled-ro-program } (\text{trace-root-budget } \text{budgets})$
 $(\text{trace-root-stage } A)$
using *controlled* **unfolding** *staged-adversary-controlled-def* **by** *simp*
have *final-controlled*:
 $\text{controlled-ro-program } (\text{trace-final-budget } \text{budgets})$
 $(\text{trace-final-stage } A \ \text{trace-bs})$
using *controlled* **unfolding** *staged-adversary-controlled-def* **by** *blast*


```

have ext-s-s1:  $s \leq s1$ 
  using controlled-ro-program-extension[OF root-controlled] root-out
  unfolding hash-extension-preserving-def by blast
have s2-eq:
   $s2 = s1 \langle$ 
     $PState := \text{concat } (PState \ s1) \ fr,$ 
     $PTranscript := PTranscript \ s1 \ @ \ [fr] \rangle$ 
  by (rule record-staged-message-outcome[OF record-root])
have ext-s1-s2:  $s1 \leq s2$ 
  unfolding s2-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have trace-bound:
   $0 + \text{ceil-log } \text{length} \leq \text{length } (\text{trace-fri-budgets } \text{budgets})$ 
  using wf unfolding staged-budget-wellformed-def by simp
have trace-alignment:
   $\text{length } \text{trace-roots} = \text{ceil-log } \text{length} \wedge$ 
   $\text{length } \text{trace-bs} = \text{length } ([\ ] :: 'f \text{ list}) + \text{ceil-log } \text{length} \wedge$ 
   $PTranscript \ s3 = PTranscript \ s2 \ @ \ \text{trace-roots} \wedge$ 
   $PState \ s3 = \text{foldl } \text{concat } (PState \ s2) \ \text{trace-roots} \wedge$ 
   $s2 \leq s3 \wedge$ 
   $PTraceFriCounter \ s3 = PTraceFriCounter \ s2 + \text{ceil-log } \text{length} \wedge$ 
   $PCompositionFriCounter \ s3 = PCompositionFriCounter \ s2 \wedge$ 
   $PAlphaCounter \ s3 = PAlphaCounter \ s2 \wedge$ 
   $PQueryCounter \ s3 = PQueryCounter \ s2 \wedge$ 
   $(\forall j < \text{ceil-log } \text{length}.$ 
     $\text{fmlookup } (\text{HashMap } s3)$ 
     $(\text{TraceFriChallenge } (PTraceFriCounter \ s2 + j)$ 
       $(\text{foldl } \text{concat } (PState \ s2) (\text{take } (\text{Suc } j) \ \text{trace-roots}))) =$ 
       $\text{Some } (\text{trace-bs } ! (\text{length } ([\ ] :: 'f \text{ list}) + j)))$ 
     $)$ 
  by (rule staged-trace-fri-program-alignment
    [OF controlled trace-bound fri-out])
let ?key =
   $\text{TraceFriChallenge } (PTraceFriCounter \ s2 + i)$ 
   $(\text{foldl } \text{concat } (PState \ s2) (\text{take } (\text{Suc } i) \ \text{trace-roots}))$ 
have lookup-s3:
   $\text{fmlookup } (\text{HashMap } s3) \ ?key = \text{Some } (\text{trace-bs } ! \ i)$ 
  using trace-alignment i-bound by simp
have ext-s-s3:  $s \leq s3$ 
  by (rule hash-ext-trans[OF ext-s-s1])
  (rule hash-ext-trans[OF ext-s1-s2], use trace-alignment in simp)
have ext-s3-s4:  $s3 \leq s4$ 
  using controlled-ro-program-extension[OF final-controlled] final-out
  unfolding hash-extension-preserving-def by blast
have t-eq:
   $t = s4 \langle$ 
     $PState := \text{concat } (PState \ s4) \ \text{trace-final},$ 
     $PTranscript := PTranscript \ s4 \ @ \ [\text{trace-final}] \rangle$ 
  by (rule record-staged-message-outcome[OF record-final])
have ext-s4-t:  $s4 \leq t$ 
  unfolding t-eq less-eq-hash-ext-def less-eq-fmap-def by simp

```

```

have ext-s3-t:  $s3 \leq t$ 
  by (rule hash-ext-trans[OF ext-s3-s4 ext-s4-t])
have lookup-t:
  fmlookup (HashMap t) ?key = Some (trace-bs ! i)
  by (rule hash-extension-lookup[OF lookup-s3 ext-s3-t])
have none-s: fmlookup (HashMap s) ?key = None
proof (cases fmlookup (HashMap s) ?key)
  case None
  then show ?thesis .
next
  case (Some old)
  have lookup-s3-old: fmlookup (HashMap s3) ?key = Some old
    by (rule hash-extension-lookup[OF Some ext-s-s3])
  then have old-eq: old = trace-bs ! i
    using lookup-s3 by simp
  have old  $\notin$  fri-vector-position-values B i
    using absent Some
    unfolding trace-fri-challenge-values-absent-def by blast
  then show ?thesis
    using old-eq hit by simp
qed
have hash-map-new-output-hit (fri-vector-position-values B i) s t
  unfolding hash-map-new-output-hit-def
  using none-s lookup-t hit by blast
then show ?thesis
  unfolding hash-new-output-hit-event-def by simp
qed

```

```

lemma staged-alpha-prefix-trace-coordinate-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{ceil-log } \text{clength}$ 
  and absent:
    trace-fri-challenge-values-absent
    (fri-vector-position-values B i) s
  shows
    wp-event (staged-alpha-prefix-program A)
    ( $\lambda \text{out. case out of None} \Rightarrow \text{False}$ 
       $\mid \text{Some } ((-, -, \text{trace-bs}, -), -) \Rightarrow$ 
         $\text{trace-bs ! } i \in \text{fri-vector-position-values } B \text{ } i$ )  $s \leq$ 
    staged-phase-target-error (fri-vector-position-values B i)
    (staged-alpha-search-queries budgets 0)
proof –
  let ?P =
     $\lambda \text{out. case out of None} \Rightarrow \text{False}$ 
     $\mid \text{Some } ((-, -, \text{trace-bs}, -), -) \Rightarrow$ 
       $\text{trace-bs ! } i \in \text{fri-vector-position-values } B \text{ } i$ 
  let ?H =
    hash-new-output-hit-event (fri-vector-position-values B i) s

```

```

have event-imp-hit:
  wp-event (staged-alpha-prefix-program A) ?P s ≤
    wp-event (staged-alpha-prefix-program A) ?H s
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈ set-dist (execute (staged-alpha-prefix-program A) s)
    and event: ?P out
  show ?H out
  proof (cases out)
    case None
    then show ?thesis using event by simp
  next
    case (Some result)
    then obtain fr trace-roots trace-bs trace-final t where
      out-eq:
        out = Some ((fr, trace-roots, trace-bs, trace-final), t)
      by (cases result, auto split: prod.splits)
    have outcome:
      Some ((fr, trace-roots, trace-bs, trace-final), t) ∈
        set-dist (execute (staged-alpha-prefix-program A) s)
      using support unfolding out-eq .
    have hit: trace-bs ! i ∈ fri-vector-position-values B i
      using event unfolding out-eq by simp
    show ?thesis
      unfolding out-eq
      by (rule
        staged-alpha-prefix-trace-coordinate-hit-imp-new-output-hit
        [OF wf controlled i-bound absent outcome hit])
    qed
  qed
also have
  wp-event (staged-alpha-prefix-program A) ?H s ≤
    staged-phase-target-error (fri-vector-position-values B i)
    (staged-alpha-search-queries budgets 0)
  by (rule staged-alpha-prefix-target-hit-bound[OF wf controlled])
  finally show ?thesis .
qed

```

```

lemma staged-trace-fri-challenge-prefix-program-length:
  assumes outcome:
    Some ((fr, trace-bs, rt), t) ∈
      set-dist
        (execute (staged-trace-fri-challenge-prefix-program A i) s)
  shows length trace-bs = i
proof –
  from outcome obtain roots pre-state post-state where
    fri-out:
      Some ((roots, trace-bs), post-state) ∈

```

set-dist
 (execute (staged-trace-fri-program A 0 i []) pre-state)
unfolding staged-trace-fri-challenge-prefix-program-def
by (auto elim!: set-dist-bindE)
have length trace-bs = length ([] :: 'f list) + i
by (rule staged-trace-fri-program-challenge-length[OF fri-out])
then show ?thesis **by** simp
qed

lemma staged-trace-fri-prefix-vector-member-imp-position-value:
assumes outcome:
 Some ((fr, trace-bs, rt), t) ∈
set-dist
 (execute (staged-trace-fri-challenge-prefix-program A i) s)
and subset: $B \subseteq \text{fri-challenge-space } i$
and member: trace-bs ∈ B
and j-bound: $j < i$
shows trace-bs ! j ∈ fri-vector-position-values B j
by (rule fri-vector-member-imp-position-value
 [OF subset member j-bound])

lemma staged-trace-fri-prefix-head-challenge-imp-position-value:
assumes outcome:
 Some (((fr, trace-bs, rt), b), t) ∈
set-dist
 (execute
 (staged-trace-fri-challenge-prefix-program A i >>=
 (λprefix. receive-trace-fri-challenge >>=
 (λb. return (prefix, b)))) s)
and member: trace-bs @ [b] ∈ B
shows b ∈ fri-vector-position-values B i

proof –
from outcome **obtain** s1 **where**
 prefix-out:
 Some ((fr, trace-bs, rt), s1) ∈
set-dist
 (execute (staged-trace-fri-challenge-prefix-program A i) s)
by (auto elim!: set-dist-bindE)
have len: length trace-bs = i
by (rule staged-trace-fri-challenge-prefix-program-length
 [OF prefix-out])
show ?thesis
by (rule fri-vector-extend-member-imp-position-value
 [OF member len])
qed

lemma staged-alpha-program-alignment:
assumes outcome:
 Some (as, t) ∈ set-dist (execute (staged-alpha-program n) s)

shows
 $length\ as = n \wedge$
 $PTranscript\ t = PTranscript\ s\ @\ as \wedge$
 $PState\ t = foldl\ concat\ (PState\ s)\ as \wedge$
 $PTraceFriCounter\ t = PTraceFriCounter\ s \wedge$
 $PCompositionFriCounter\ t = PCompositionFriCounter\ s \wedge$
 $PAlphaCounter\ t = PAlphaCounter\ s + n \wedge$
 $PQueryCounter\ t = PQueryCounter\ s$
using *outcome*
proof (*induction n arbitrary: as s t*)
case 0
then show ?*case* **by** *simp*
next
case (*Suc n*)
from *staged-alpha-program-Suc-outcomeE[OF Suc.premis]*
obtain *a as' s1 s2* **where**
challenge-out:
 $Some\ (a,\ s1) \in$
 $set-dist\ (execute\ receive-alpha-challenge\ s)$
and *record-out:*
 $Some\ (\(),\ s2) \in$
 $set-dist\ (execute\ (record-staged-message\ a)\ s1)$
and *rest-out:*
 $Some\ (as',\ t) \in$
 $set-dist\ (execute\ (staged-alpha-program\ n)\ s2)$
and *as-eq:* $as = a \ \# \ as'$
 $\cdot$
have *challenge-fields:*
 $PState\ s1 = PState\ s \wedge$
 $PTranscript\ s1 = PTranscript\ s$
using *receive-alpha-challenge-outcome[OF challenge-out]* **by** *simp*
have *challenge-counters:*
 $PTraceFriCounter\ s1 = PTraceFriCounter\ s \wedge$
 $PCompositionFriCounter\ s1 = PCompositionFriCounter\ s \wedge$
 $PAlphaCounter\ s1 = Suc\ (PAlphaCounter\ s) \wedge$
 $PQueryCounter\ s1 = PQueryCounter\ s$
using *receive-alpha-challenge-counter-outcome[OF challenge-out]* **by** *simp*
have *s2:*
 $s2 = s1\ |\$
 $PState := concat\ (PState\ s1)\ a,$
 $PTranscript := PTranscript\ s1\ @\ [a]\)$
by (*rule record-staged-message-outcome[OF record-out]*)
have *s2-fields:*
 $PState\ s2 = concat\ (PState\ s)\ a \wedge$
 $PTranscript\ s2 = PTranscript\ s\ @\ [a] \wedge$
 $PTraceFriCounter\ s2 = PTraceFriCounter\ s \wedge$
 $PCompositionFriCounter\ s2 = PCompositionFriCounter\ s \wedge$
 $PAlphaCounter\ s2 = Suc\ (PAlphaCounter\ s) \wedge$
 $PQueryCounter\ s2 = PQueryCounter\ s$

```

unfolding s2 using challenge-fields challenge-counters by simp
have rest-alignment:
  length as' = n ∧
  PTranscript t = PTranscript s2 @ as' ∧
  PState t = foldl concat (PState s2) as' ∧
  PTraceFriCounter t = PTraceFriCounter s2 ∧
  PCompositionFriCounter t = PCompositionFriCounter s2 ∧
  PAlphaCounter t = PAlphaCounter s2 + n ∧
  PQueryCounter t = PQueryCounter s2
by (rule Suc.IH[OF rest-out])
show ?case
  unfolding as-eq
  using s2-fields rest-alignment
  by simp
qed

lemma staged-receive-alpha-challenge-preserves-alpha-future-fresh:
  assumes future: alpha-future-fresh s
  and outcome: Some (a, t) ∈ set-dist (execute receive-alpha-challenge s)
  shows alpha-future-fresh t
proof -
  have counter-t: PAlphaCounter t = Suc (PAlphaCounter s)
  using receive-alpha-challenge-counter-outcome[OF outcome] by simp
  show ?thesis
  unfolding alpha-future-fresh-def
proof (intro allI impI)
  fix i x
  assume i-future: PAlphaCounter t ≤ i
  have i-ge-s: PAlphaCounter s ≤ i
  using counter-t i-future by simp
  have neg:
    AlphaChallenge i x ≠
    AlphaChallenge (PAlphaCounter s) (PState s)
  using counter-t i-future by auto
  have fmlookup (HashMap t) (AlphaChallenge i x) =
    fmlookup (HashMap s) (AlphaChallenge i x)
  by (rule receive-alpha-challenge-preserves-other-lookup
    [OF outcome neg])
  also have ... = None
  using future i-ge-s unfolding alpha-future-fresh-def by blast
  finally show fmlookup (HashMap t) (AlphaChallenge i x) = None .
qed
qed

lemma record-staged-message-preserves-alpha-future-fresh:
  assumes future: alpha-future-fresh s
  and outcome:
    Some ((), t) ∈ set-dist (execute (record-staged-message x) s)
  shows alpha-future-fresh t

```

```

proof –
  have t-eq:
     $t = s \parallel$ 
       $PState := \text{concat } (PState\ s)\ x,$ 
       $PTranscript := PTranscript\ s\ @\ [x]$ 
    by (rule record-staged-message-outcome[OF outcome])
  show ?thesis
    using future unfolding t-eq alpha-future-fresh-def by simp
qed

lemma staged-alpha-program-preserves-alpha-future-fresh:
  assumes future: alpha-future-fresh s
  and outcome:
     $\text{Some } (as, t) \in \text{set-dist } (\text{execute } (\text{staged-alpha-program } n)\ s)$ 
  shows alpha-future-fresh t
  using future outcome
proof (induction n arbitrary: as s t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  show ?case
  proof (cases rule:
    staged-alpha-program-Suc-outcomeE[OF Suc.prems(2)])
  case ( $1\ a\ as'\ s1\ s2$ )
  have future-s1:
    alpha-future-fresh s1
  by (rule staged-receive-alpha-challenge-preserves-alpha-future-fresh
    [OF Suc.prems(1) 1(1)])
  have future-s2:
    alpha-future-fresh s2
  by (rule record-staged-message-preserves-alpha-future-fresh
    [OF future-s1 1(2)])
  show ?thesis
    by (rule Suc.IH[OF future-s2 1(3)])
  qed
qed

lemma wp-staged-alpha-program-exact-challenges-bound:
  assumes len:  $\text{length } as = n$ 
  and future: alpha-future-fresh s
  shows
     $\text{wp-event } (\text{staged-alpha-program } n)$ 
     $(\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
       $\mid \text{Some } (as', -) \Rightarrow as' = as)\ s \leq$ 
     $(1 / \text{nnreal size})^n$ 
  using len future
proof (induction n arbitrary: as s)
  case 0

```

```

then have as-empty: as = []
  by simp
show ?case
  unfolding as-empty by (simp add: wp-event-def wpsimps)
next
case (Suc n)
obtain a as-tail where as-eq: as = a # as-tail
  using Suc.premis(1) by (cases as) auto
have len-tail: length as-tail = n
  using Suc.premis(1) as-eq by simp
let ?tail =
  staged-alpha-program n ::
    ('f list, 'f protocol-channel) state-monad
let ?Q =
  λout :: ('f list ×
    'f protocol-channel) option.
    case out of None ⇒ False | Some (as', -) ⇒ as' = as
let ?Head =
  λout :: ('f ×
    'f protocol-channel) option.
    case out of None ⇒ False | Some (a0, -) ⇒ a0 ∈ {a}
have head-bound:
  wp-event receive-alpha-challenge ?Head s ≤ 1 / nnreal size
proof -
  have current-fresh:
    fmlookup (HashMap s)
      (AlphaChallenge (PAlphaCounter s) (PState s)) = None
  by (rule alpha-future-fresh-current[OF Suc.premis(2)])
  have wp-event receive-alpha-challenge ?Head s =
    nnreal (card ({a} :: 'f set)) / nnreal size
  by (rule wp-receive-alpha-challenge-fresh-set[OF current-fresh])
  then show ?thesis by simp
qed
have bind-bound:
  wp-event
    (receive-alpha-challenge >>=
      (λa0. record-staged-message a0 >>=
        (λ-. ?tail >>=
          (λas'. return (a0 # as'))))))
  ?Q s ≤
  wp-event receive-alpha-challenge ?Head s *
  (1 / nnreal size) ^ n
proof (rule wp-event-bind-bound-by-head-and-cont[where P = ?Head])
  show ¬ ?Q None
  by simp
next
fix a0 t
assume recv:
  Some (a0, t) ∈ set-dist (execute receive-alpha-challenge s)

```



```

and not-head:  $\neg ?\text{Head } (\text{Some } (a0, t))$ 
have a0-ne:  $a0 \neq a$ 
using not-head by simp
have cont-false:
  wp-event
  (record-staged-message  $a0 \gg=$ 
    ( $\lambda-. ?\text{tail} \gg= (\lambda as'. \text{return } (a0 \# as'))$ )))
   $?Q \ t \leq$ 
  wp-event
  (record-staged-message  $a0 \gg=$ 
    ( $\lambda-. ?\text{tail} \gg= (\lambda as'. \text{return } (a0 \# as'))$ )))
  ( $\lambda- :: ('f \text{ list } \times$ 
     $'f \text{ protocol-channel}) \text{ option. False} \ t$ 
  by (rule wp-event-mono-on-support)
  (use a0-ne as-eq in
     $\langle \text{auto simp: wpsimps elim!: set-dist-bindE}$ 
       $\text{split: option.splits prod.splits} \rangle$ )
also have  $\dots = 0$ 
unfolding wp-event-def wp-def dist-expect-def by simp
finally have le-zero:
  wp-event
  (record-staged-message  $a0 \gg=$ 
    ( $\lambda-. ?\text{tail} \gg= (\lambda as'. \text{return } (a0 \# as'))$ )))
   $?Q \ t \leq 0$  .
show
  wp-event
  (record-staged-message  $a0 \gg=$ 
    ( $\lambda-. ?\text{tail} \gg= (\lambda as'. \text{return } (a0 \# as'))$ )))
   $?Q \ t = 0$ 
  by (rule antisym[OF le-zero]) simp
next
fix  $a0 \ t$ 
assume recv:
   $\text{Some } (a0, t) \in \text{set-dist } (\text{execute receive-alpha-challenge } s)$ 
and head:  $?\text{Head } (\text{Some } (a0, t))$ 
have a0-eq:  $a0 = a$ 
using head by simp
have future-t: alpha-future-fresh  $t$ 
by (rule staged-receive-alpha-challenge-preserves-alpha-future-fresh
   $[\text{OF Suc.prems}(2) \text{ recv}]$ )
have record-cont-bound:
  wp-event
  (record-staged-message  $a0 \gg=$ 
    ( $\lambda-. ?\text{tail} \gg= (\lambda as'. \text{return } (a0 \# as'))$ )))
   $?Q \ t \leq (1 / \text{nnreal size}) ^ n$ 
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg ?Q \ \text{None}$ 
  by simp
next

```

```

fix uu u
assume record-out:
  Some (uu, u) ∈
    set-dist (execute (record-staged-message a0) t)
have record-unit:
  Some ((), u) ∈
    set-dist (execute (record-staged-message a0) t)
  using record-out by (cases uu) simp
have future-u: alpha-future-fresh u
  by (rule record-staged-message-preserves-alpha-future-fresh
    [OF future-t record-unit])
have tail-bound:
  wp-event ?tail
    ( $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
      |  $\text{Some } (as', -) \Rightarrow as' = as\text{-tail}$ ) u
     $\leq (1 / \text{nnreal size}) ^ n$ 
  by (rule Suc.IH[OF len-tail future-u])
have cont-eq:
  wp-event
    ( $?tail \gg (\lambda as'. \text{return } (a0 \# as'))$ )  $?Q u =$ 
    wp-event ?tail
      ( $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
        |  $\text{Some } (as', -) \Rightarrow as' = as\text{-tail}$ ) u
  by (simp add: wp-event-bind-return-map a0-eq as-eq
    split: option.splits prod.splits)
show
  wp-event (?tail  $\gg (\lambda as'. \text{return } (a0 \# as'))$ ) ?Q u
     $\leq (1 / \text{nnreal size}) ^ n$ 
  unfolding cont-eq by (rule tail-bound)
qed
show
  wp-event
    (record-staged-message a0  $\gg$ 
      ( $\lambda -. ?tail \gg (\lambda as'. \text{return } (a0 \# as'))$ ))
     $?Q t \leq (1 / \text{nnreal size}) ^ n$ 
  by (rule record-cont-bound)
qed
have wp-event (staged-alpha-program (Suc n)) ?Q s
   $\leq \text{wp-event receive-alpha-challenge ?Head s} *$ 
     $(1 / \text{nnreal size}) ^ n$ 
  unfolding staged-alpha-program.simps by (rule bind-bound)
also have  $\dots \leq (1 / \text{nnreal size}) * (1 / \text{nnreal size}) ^ n$ 
  by (rule mult-right-mono[OF head-bound]) simp
also have  $\dots = (1 / \text{nnreal size}) ^ {Suc\ n}$ 
  by simp
finally show ?case .
qed

```

lemma *wp-staged-alpha-program-finite-set-bound*:

```

assumes future: alpha-future-fresh s
and finite-B: finite B
and lengths:  $\bigwedge as. as \in B \implies \text{length } as = n$ 
shows
  wp-event (staged-alpha-program n)
    ( $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
      |  $\text{Some } (as, -) \Rightarrow as \in B$ )  $s \leq$ 
      nnreal (card B) / (nnreal size)  $^n$ 
proof -
  let ?P =
     $\lambda as out. \text{case out of None} \Rightarrow \text{False}$ 
      |  $\text{Some } (as', -) \Rightarrow as' = as$ 
  have event-mono:
    wp-event (staged-alpha-program n)
      ( $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
        |  $\text{Some } (as, -) \Rightarrow as \in B$ )  $s \leq$ 
      wp-event (staged-alpha-program n)
        ( $\lambda out. \exists as \in B. ?P as out$ ) s
  by (rule wp-event-mono) (auto split: option.splits prod.splits)
also have ...  $\leq (\sum as \in B. (1 / \text{nnreal size})^n)$ 
  by (rule wp-event-finite-UN-bound[OF finite-B])
    (rule wp-staged-alpha-program-exact-challenges-bound
      [OF lengths future])
also have ... = nnreal (card B) * ((1 / nnreal size)  $^n$ )
  using finite-B by simp
also have ... = nnreal (card B) / (nnreal size)  $^n$ 
  using size-card by transfer (simp add: power-divide)
finally show ?thesis .
qed

```

lemma wp-staged-alpha-program-alpha-space-set-bound:

assumes future: alpha-future-fresh s
and subset: $B \subseteq \text{alpha-space}$

shows

wp-event (staged-alpha-program (length spec))
 ($\lambda out. \text{case out of None} \Rightarrow \text{False}$
 | $\text{Some } (as, -) \Rightarrow as \in B$) $s \leq$
 nnreal (card B) / nnreal (CARD('f) $^{\text{length spec}}$)

proof -

have finite-B: finite B

by (rule finite-subset[OF subset])
 (simp add: alpha-space-def finite-length-lists-UNIV)

have lengths: $\bigwedge as. as \in B \implies \text{length } as = \text{length spec}$

using subset **unfolding** alpha-space-def **by** auto

have wp-event (staged-alpha-program (length spec))

($\lambda out. \text{case out of None} \Rightarrow \text{False}$
 | $\text{Some } (as, -) \Rightarrow as \in B$) $s \leq$
 nnreal (card B) / (nnreal size) $^{\text{length spec}}$

by (rule wp-staged-alpha-program-finite-set-bound

```

    [OF future finite-B lengths])
  also have ... =
    nnreal (card B) / nnreal (CARD('f) ^ length spec)
  using size-card by simp
  finally show ?thesis .
qed

lemma staged-after-alpha-prefix-program-alpha-space-list-bound-from-fresh:
  assumes future: alpha-future-fresh s
  and subset: B ⊆ alpha-space
  shows
    wp-event
      (staged-after-alpha-prefix-program A
        (fr, trace-roots, trace-bs, trace-final))
      (staged-transcript-alpha-list-hit B) s ≤
      nnreal (card B) / nnreal (CARD('f) ^ length spec)
  proof -
    have alpha-bound:
      wp-event (staged-alpha-program (length spec))
        (λout. case out of None ⇒ False
          | Some (as, -) ⇒ as ∈ B) s ≤
        nnreal (card B) / nnreal (CARD('f) ^ length spec)
    by (rule wp-staged-alpha-program-alpha-space-set-bound
        [OF future subset])
    show ?thesis
      by (rule staged-after-alpha-prefix-program-alpha-list-bound-from-alpha-program
          [OF alpha-bound])
  qed

lemma staged-transcript-program-alpha-space-list-bound-from-fresh-prefix:
  assumes prefix-future:
    ∧ prefix t.
    Some (prefix, t) ∈
      set-dist (execute (staged-alpha-prefix-program A)
        adversary-initial-state) ⇒
      alpha-future-fresh t
  and subset: B ⊆ alpha-space
  shows
    wp-event (staged-transcript-program A)
      (staged-transcript-alpha-list-hit B) adversary-initial-state ≤
      nnreal (card B) / nnreal (CARD('f) ^ length spec)
  unfolding staged-transcript-program-alpha-prefix-decomp
  proof (rule wp-event-bind-bound-by-cont)
    show ¬ staged-transcript-alpha-list-hit B None
      unfolding staged-transcript-alpha-list-hit-def by simp
  next
    fix prefix t
    assume prefix-out:
      Some (prefix, t) ∈

```

```

      set-dist (execute (staged-alpha-prefix-program A)
        adversary-initial-state)
obtain fr trace-roots trace-bs trace-final
  where prefix-eq: prefix = (fr, trace-roots, trace-bs, trace-final)
  by (cases prefix) auto
have future-t: alpha-future-fresh t
  by (rule prefix-future[OF prefix-out])
show
  wp-event (staged-after-alpha-prefix-program A prefix)
    (staged-transcript-alpha-list-hit B) t ≤
    nnreal (card B) / nnreal (CARD('f) ^ length spec)
  unfolding prefix-eq
  by (rule staged-after-alpha-prefix-program-alpha-space-list-bound-from-fresh
    [OF future-t subset])
qed

```

lemma staged-composition-fri-program-challenge-prefix-pre-alignment:

```

assumes outcome:
  Some ((roots, bs'), t) ∈
    set-dist
      (execute (staged-composition-fri-program A dg i n bs) s)
shows take (length bs) bs' = bs
using outcome
proof (induction n arbitrary: i bs roots bs' s t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.prem obtain root b roots' s1 s2 s3 where
    rest-out:
      Some ((roots', bs'), t) ∈
        set-dist
          (execute
            (staged-composition-fri-program
              A dg (Suc i) n (bs @ [b])) s3)
  unfolding staged-composition-fri-program.simps
  by (auto elim!: set-dist-bindE)
have prefix-suc:
  take (length (bs @ [b])) bs' = bs @ [b]
  by (rule Suc.IH[OF rest-out])
have take (length bs) bs' =
  take (length bs) (take (length (bs @ [b])) bs')
  by simp
also have ... = bs
  using prefix-suc by simp
finally show ?case .
qed

```

lemma staged-composition-fri-program-alignment:

```

assumes controlled: staged-adversary-controlled budgets A
and bound:
   $i + n \leq \text{length } (\text{composition-fri-budgets } \text{budgets})$ 
and outcome:
   $\text{Some } ((\text{roots}, \text{bs}'), t) \in$ 
    set-dist
     $(\text{execute}$ 
       $(\text{staged-composition-fri-program } A \text{ dg } i \text{ n } \text{bs}) \text{ s})$ 
shows
   $\text{length roots} = n \wedge$ 
   $\text{length bs}' = \text{length bs} + n \wedge$ 
   $P\text{Transcript } t = P\text{Transcript } s @ \text{roots} \wedge$ 
   $P\text{State } t = \text{foldl concat } (P\text{State } s) \text{ roots} \wedge$ 
   $s \leq t \wedge$ 
   $P\text{TraceFriCounter } t = P\text{TraceFriCounter } s \wedge$ 
   $P\text{CompositionFriCounter } t = P\text{CompositionFriCounter } s + n \wedge$ 
   $P\text{AlphaCounter } t = P\text{AlphaCounter } s \wedge$ 
   $P\text{QueryCounter } t = P\text{QueryCounter } s \wedge$ 
   $(\forall j < n.$ 
     $\text{fmlookup } (\text{HashMap } t)$ 
     $(\text{CompositionFriChallenge } (P\text{CompositionFriCounter } s + j)$ 
       $(\text{foldl concat } (P\text{State } s) (\text{take } (\text{Suc } j) \text{ roots}))) =$ 
       $\text{Some } (\text{bs}' ! (\text{length bs} + j)))$ 
  using bound outcome
proof  $(\text{induction } n \text{ arbitrary: } i \text{ bs roots bs}' s t)$ 
  case 0
  then show ?case by (simp add: hash-ext-refl)
next
  case  $(\text{Suc } n)$ 
from staged-composition-fri-program-Suc-outcomeE[OF Suc.premis(2)]
obtain root b roots' s1 s2 s3 where
  stage-out:
   $\text{Some } (\text{root}, s1) \in$ 
    set-dist
     $(\text{execute } (\text{composition-fri-root-stage } A \text{ dg } i \text{ bs}) s)$ 
and record-out:
   $\text{Some } ((), s2) \in$ 
    set-dist  $(\text{execute } (\text{record-staged-message } \text{root}) s1)$ 
and challenge-out:
   $\text{Some } (b, s3) \in$ 
    set-dist  $(\text{execute } \text{receive-composition-fri-challenge } s2)$ 
and rest-out:
   $\text{Some } ((\text{roots}', \text{bs}'), t) \in$ 
    set-dist
     $(\text{execute}$ 
       $(\text{staged-composition-fri-program}$ 
         $A \text{ dg } (\text{Suc } i) \text{ n } (\text{bs} @ [b]) s3)$ 
and roots-eq:  $\text{roots} = \text{root} \# \text{roots}'$ 
  .

```

```

have i-bound:  $i < \text{length } (\text{composition-fri-budgets } \text{budgets})$ 
  using Suc.premis(1) by simp
have stage-controlled:
  controlled-ro-program (composition-fri-budgets budgets ! i)
    (composition-fri-root-stage A dg i bs)
  using controlled i-bound
  unfolding staged-adversary-controlled-def by blast
have stage-fields:
  PState s1 = PState s  $\wedge$ 
  PTranscript s1 = PTranscript s  $\wedge$ 
  PTraceFriCounter s1 = PTraceFriCounter s  $\wedge$ 
  PCompositionFriCounter s1 = PCompositionFriCounter s  $\wedge$ 
  PAlphaCounter s1 = PAlphaCounter s  $\wedge$ 
  PQueryCounter s1 = PQueryCounter s
  using controlled-stage-outcome-fields[OF stage-controlled stage-out]
  by simp
have ext-s-s1:  $s \leq s1$ 
  using controlled-ro-program-extension[OF stage-controlled] stage-out
  unfolding hash-extension-preserving-def by blast
have s2:
  s2 = s1 (
    PState := concat (PState s1) root,
    PTranscript := PTranscript s1 @ [root])
  by (rule record-staged-message-outcome[OF record-out])
have ext-s1-s2:  $s1 \leq s2$ 
  unfolding s2 less-eq-hash-ext-def less-eq-fmap-def by simp
have s2-fields:
  PState s2 = concat (PState s) root  $\wedge$ 
  PTranscript s2 = PTranscript s @ [root]  $\wedge$ 
  PTraceFriCounter s2 = PTraceFriCounter s  $\wedge$ 
  PCompositionFriCounter s2 = PCompositionFriCounter s  $\wedge$ 
  PAlphaCounter s2 = PAlphaCounter s  $\wedge$ 
  PQueryCounter s2 = PQueryCounter s
  unfolding s2 using stage-fields by simp
have challenge-fields:
  PState s3 = PState s2  $\wedge$ 
  PTranscript s3 = PTranscript s2
  using receive-composition-fri-challenge-outcome[OF challenge-out]
  by simp
have ext-s2-s3:  $s2 \leq s3$ 
  using receive-composition-fri-challenge-outcome[OF challenge-out]
  by simp
have challenge-counters:
  PTraceFriCounter s3 = PTraceFriCounter s2  $\wedge$ 
  PCompositionFriCounter s3 =
    Suc (PCompositionFriCounter s2)  $\wedge$ 
  PAlphaCounter s3 = PAlphaCounter s2  $\wedge$ 
  PQueryCounter s3 = PQueryCounter s2
  using

```

```

    receive-composition-fri-challenge-counter-outcome[OF challenge-out]
  by simp
have rest-bound:
  Suc i + n ≤ length (composition-fri-budgets budgets)
  using Suc.prem1 by simp
have rest-alignment:
  length roots' = n ∧
  length bs' = length (bs @ [b]) + n ∧
  PTranscript t = PTranscript s3 @ roots' ∧
  PState t = foldl concat (PState s3) roots' ∧
  s3 ≤ t ∧
  PTraceFriCounter t = PTraceFriCounter s3 ∧
  PCompositionFriCounter t = PCompositionFriCounter s3 + n ∧
  PAlphaCounter t = PAlphaCounter s3 ∧
  PQueryCounter t = PQueryCounter s3 ∧
  (∀ j < n.
    fmllookup (HashMap t)
      (CompositionFriChallenge (PCompositionFriCounter s3 + j)
        (foldl concat (PState s3) (take (Suc j) roots')))) =
    Some (bs' ! (length (bs @ [b]) + j)))
  by (rule Suc.IH[OF rest-bound rest-out])
have ext-s3-t: s3 ≤ t
  using rest-alignment by simp
have ext-s-t: s ≤ t
  by (rule hash-ext-trans[OF ext-s-s1])
  (rule hash-ext-trans[OF ext-s1-s2],
   rule hash-ext-trans[OF ext-s2-s3 ext-s3-t])
have challenge-lookup-s3:
  fmllookup (HashMap s3)
    (CompositionFriChallenge (PCompositionFriCounter s)
      (concat (PState s) root)) = Some b
  using receive-composition-fri-challenge-counter-outcome[OF challenge-out]
  s2-fields by simp
have challenge-lookup-t:
  fmllookup (HashMap t)
    (CompositionFriChallenge (PCompositionFriCounter s)
      (concat (PState s) root)) = Some b
  by (rule hash-extension-lookup[OF challenge-lookup-s3 ext-s3-t])
have prefix-suc:
  take (length (bs @ [b])) bs' = bs @ [b]
  by (rule staged-composition-fri-program-challenge-prefix-pre-alignment
    [OF rest-out])
have lookup-all:
  ∀ j < Suc n.
    fmllookup (HashMap t)
      (CompositionFriChallenge (PCompositionFriCounter s + j)
        (foldl concat (PState s) (take (Suc j) (root # roots')))) =
    Some (bs' ! (length bs + j))
proof (intro allI impI)

```



```

fix j
assume j-bound:  $j < \text{Suc } n$ 
show
  fmlookup (HashMap t)
    (CompositionFriChallenge (PCompositionFriCounter s + j)
      (foldl concat (PState s) (take (Suc j) (root # roots')))) =
    Some (bs' ! (length bs + j))
proof (cases j)
  case 0
  have bs' ! length bs = b
  proof -
    have idx-lt: length bs < length (bs @ [b])
    by simp
    have bs' ! length bs =
      take (length (bs @ [b])) bs' ! length bs
    using idx-lt by simp
    also have ... = (bs @ [b]) ! length bs
    using prefix-suc by simp
    also have ... = b
    by simp
    finally show ?thesis .
  qed
then show ?thesis
  using challenge-lookup-t unfolding 0 by simp
next
  case (Suc k)
  then have k-bound:  $k < n$ 
  using j-bound by simp
  show ?thesis
  using rest-alignment k-bound s2-fields challenge-fields
    challenge-counters
  unfolding Suc
  by simp
  qed
qed
show ?case
  unfolding roots-eq
  using s2-fields challenge-fields challenge-counters rest-alignment
    ext-s-t lookup-all
  by simp
qed

lemma staged-composition-fri-program-challenge-prefix:
assumes outcome:
  Some ((roots, bs'), t) ∈
    set-dist
    (execute (staged-composition-fri-program A dg i n bs) s)
shows take (length bs) bs' = bs
using outcome

```

```

proof (induction n arbitrary: i bs roots bs' s t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.premis obtain root b roots' s1 s2 s3 where
    rest-out:
      Some ((roots', bs'), t) ∈
        set-dist
          (execute
            (staged-composition-fri-program
              A dg (Suc i) n (bs @ [b])) s3)
    unfolding staged-composition-fri-program.simps
    by (auto elim!: set-dist-bindE)
  have prefix-suc:
    take (length (bs @ [b])) bs' = bs @ [b]
    by (rule Suc.IH[OF rest-out])
  have take (length bs) bs' =
    take (length bs) (take (length (bs @ [b])) bs')
    by simp
  also have ... = bs
    using prefix-suc by simp
  finally show ?case .
qed

lemma staged-composition-fri-program-challenge-length:
  assumes outcome:
    Some ((roots, bs'), t) ∈
      set-dist
        (execute (staged-composition-fri-program A dg i n bs) s)
  shows length bs' = length bs + n
  using outcome
proof (induction n arbitrary: i bs roots bs' s t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.premis obtain root b roots' s1 s2 s3 where
    rest-out:
      Some ((roots', bs'), t) ∈
        set-dist
          (execute
            (staged-composition-fri-program
              A dg (Suc i) n (bs @ [b])) s3)
    unfolding staged-composition-fri-program.simps
    by (auto elim!: set-dist-bindE)
  have length bs' = length (bs @ [b]) + n
    by (rule Suc.IH[OF rest-out])
  then show ?case by simp

```

qed

lemma *staged-composition-fri-challenge-prefix-program-length:*

assumes *outcome:*

Some ((dg, composition-bs, rt), t) ∈
set-dist
(execute (staged-composition-fri-challenge-prefix-program A i) s)

shows *length composition-bs = i*

proof –

from *outcome* **obtain** *as s1 dg0 s2 s3* **where**

inner:

Some ((dg, composition-bs, rt), t) ∈
set-dist
(execute
(let composition-rounds = ceil-log (to-nat dg0 + 1)
in assert (Suc i ≤ composition-rounds) >>=
(λ-. assert
(composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
(λ-. staged-composition-fri-program A dg0 0 i [] >>=
(λ(composition-roots, composition-bs).
composition-fri-root-stage A dg0 i
composition-bs >>=
(λrt.
record-staged-message rt >>=
(λ-. return
(dg0, composition-bs, rt))))))

s3)

unfolding *staged-composition-fri-challenge-prefix-program-def*

by *(auto elim!: set-dist-bindE)*

let *?rounds = ceil-log (to-nat dg0 + 1)*

have *fri-out:*

∃ composition-roots s4.
Some ((composition-roots, composition-bs), s4) ∈
set-dist
(execute (staged-composition-fri-program A dg0 0 i []) s3)

proof *(cases Suc i ≤ ?rounds)*

case *False*

then show *?thesis*

using *inner*

by *(auto simp: assert-def throw-no-outcome elim!: set-dist-bindE)*

next

case *first-guard: True*

show *?thesis*

proof *(cases ?rounds ≤ ceil-log (maxDegree + 1))*

case *False*

then show *?thesis*

using *inner first-guard*

by *(auto simp: assert-def throw-no-outcome elim!: set-dist-bindE)*

```

next
  case second-guard: True
  from inner obtain composition-roots s4 where
    Some ((composition-roots, composition-bs), s4) ∈
      set-dist
        (execute (staged-composition-fri-program A dg0 0 i []) s3)
    using first-guard second-guard
    by (auto simp: assert-def elim!: set-dist-bindE split: prod.splits)
    then show ?thesis by blast
qed
qed
then obtain composition-roots s4 where fri-out:
  Some ((composition-roots, composition-bs), s4) ∈
    set-dist
      (execute (staged-composition-fri-program A dg0 0 i []) s3)
  by blast
have length composition-bs = length ([]) :: 'f list) + i
  by (rule staged-composition-fri-program-challenge-length[OF fri-out])
then show ?thesis by simp
qed

lemma staged-composition-fri-prefix-vector-member-imp-position-value:
assumes outcome:
  Some ((dg, composition-bs, rt), t) ∈
    set-dist
      (execute (staged-composition-fri-challenge-prefix-program A i) s)
and subset: B ⊆ fri-challenge-space i
and member: composition-bs ∈ B
and j-bound: j < i
shows composition-bs ! j ∈ fri-vector-position-values B j
by (rule fri-vector-member-imp-position-value
  [OF subset member j-bound])

lemma staged-composition-fri-prefix-head-challenge-imp-position-value:
assumes outcome:
  Some (((dg, composition-bs, rt), b), t) ∈
    set-dist
      (execute
        (staged-composition-fri-challenge-prefix-program A i >>=
          (λprefix. receive-composition-fri-challenge >>=
            (λb. return (prefix, b)))) s)
and member: composition-bs @ [b] ∈ B
shows b ∈ fri-vector-position-values B i
proof –
from outcome obtain s1 where
  prefix-out:
    Some ((dg, composition-bs, rt), s1) ∈
      set-dist
        (execute (staged-composition-fri-challenge-prefix-program A i) s)

```

```

    by (auto elim!: set-dist-bindE)
  have len: length composition-bs = i
    by (rule staged-composition-fri-challenge-prefix-program-length
        [OF prefix-out])
  show ?thesis
    by (rule fri-vector-extend-member-imp-position-value
        [OF member len])
qed

lemma staged-composition-fri-vector-bad-coordinate-cover-bound:
  assumes subset:
     $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$ 
  shows
    wp-event (staged-composition-fri-vector-program A)
      ( $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
        |  $\text{Some } ((dg, \text{composition-bs}), -) \Rightarrow$ 
           $\text{composition-bs} \in B \ dg \wedge$ 
           $0 < \text{ceil-log } (\text{to-nat } dg + 1) \wedge$ 
           $\text{ceil-log } (\text{to-nat } dg + 1) \leq \text{ceil-log } (\text{maxDegree} + 1)) \ s \leq$ 
           $(\sum i < \text{ceil-log } (\text{maxDegree} + 1).$ 
            wp-event (staged-composition-fri-vector-program A)
              ( $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
                |  $\text{Some } ((dg, \text{composition-bs}), -) \Rightarrow$ 
                   $i < \text{ceil-log } (\text{to-nat } dg + 1) \wedge$ 
                   $\text{composition-bs} ! i \in$ 
                    fri-vector-position-values (B dg) i) s)
      )
  proof -
    let ?bad =
       $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
        |  $\text{Some } ((dg, \text{composition-bs}), -) \Rightarrow$ 
           $\text{composition-bs} \in B \ dg \wedge$ 
           $0 < \text{ceil-log } (\text{to-nat } dg + 1) \wedge$ 
           $\text{ceil-log } (\text{to-nat } dg + 1) \leq \text{ceil-log } (\text{maxDegree} + 1)$ 
    let ?coord =
       $\lambda i \ out. \text{case out of None} \Rightarrow \text{False}$ 
        |  $\text{Some } ((dg, \text{composition-bs}), -) \Rightarrow$ 
           $i < \text{ceil-log } (\text{to-nat } dg + 1) \wedge$ 
           $\text{composition-bs} ! i \in \text{fri-vector-position-values } (B \ dg) \ i$ 
    have bad-to-coord:
      wp-event (staged-composition-fri-vector-program A) ?bad s  $\leq$ 
      wp-event (staged-composition-fri-vector-program A)
        ( $\lambda out. \exists i \in \{.. < \text{ceil-log } (\text{maxDegree} + 1)\}. ?coord \ i \ out) \ s$ 
    proof (rule wp-event-mono)
      fix out
      assume bad: ?bad out
      show  $\exists i \in \{.. < \text{ceil-log } (\text{maxDegree} + 1)\}. ?coord \ i \ out$ 
      proof (cases out)
        case None
        then show ?thesis using bad by simp
      end
    end
  end

```

```

next
  case (Some result)
  then obtain dg composition-bs t where
    out-eq: out = Some ((dg, composition-bs), t)
    by (cases result, auto split: prod.splits)
  let ?n = ceil-log (to-nat dg + 1)
  have member: composition-bs ∈ B dg
    and nonempty: 0 < ?n
    and rounds-bound: ?n ≤ ceil-log (maxDegree + 1)
    using bad unfolding out-eq by simp-all
  from fri-vector-member-coordinate-cover
    [OF subset[of dg] member nonempty]
  obtain i where i-bound: i < ?n
    and coord:
      composition-bs ! i ∈ fri-vector-position-values (B dg) i
    by blast
  have i-max: i < ceil-log (maxDegree + 1)
    using i-bound rounds-bound by simp
  show ?thesis
    by (rule bexI[of - i])
      (use i-bound i-max coord out-eq in simp-all)
qed
qed
also have ... ≤
  (∑ i < ceil-log (maxDegree + 1).
    wp-event (staged-composition-fri-vector-program A) (?coord i) s)
  by (rule wp-event-finite-UN-bound) simp-all
finally show ?thesis .
qed

lemma
  staged-composition-fri-vector-coordinate-hit-imp-new-output-hit:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound: i < ceil-log (to-nat dg + 1)
    and absent:
      composition-fri-challenge-values-absent
      (fri-vector-position-values (B dg) i) s
    and outcome:
      Some ((dg, composition-bs), t) ∈
      set-dist (execute (staged-composition-fri-vector-program A) s)
    and hit:
      composition-bs ! i ∈ fri-vector-position-values (B dg) i
  shows
    hash-new-output-hit-event (fri-vector-position-values (B dg) i) s
      (Some ((dg, composition-bs), t))
proof -
  from outcome obtain alpha-prefix s1 as s2 s3 s4 s5 roots where
    alpha-prefix-out:

```

```

    Some (alpha-prefix, s1) ∈
      set-dist (execute (staged-alpha-prefix-program A) s)
and alpha-out:
    Some (as, s2) ∈
      set-dist (execute (staged-alpha-program (length spec)) s1)
and degree-out:
    Some (dg, s3) ∈ set-dist (execute (degree-stage A as) s2)
and record-degree:
    Some ((), s4) ∈
      set-dist (execute (record-staged-message dg) s3)
and assert-out:
    Some ((), s5) ∈
      set-dist
        (execute
          (assert
            (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1))) s4)
and fri-out:
    Some ((roots, composition-bs), t) ∈
      set-dist
        (execute
          (staged-composition-fri-program A dg 0
            (ceil-log (to-nat dg + 1)) []) s5)
unfolding staged-composition-fri-vector-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
using assert-out unfolding assert-def
by (cases
    ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
  (auto simp: throw-no-outcome)
have len:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
using round-bound wf unfolding staged-budget-wellformed-def by simp
have alignment:
  length roots = ceil-log (to-nat dg + 1) ∧
  length composition-bs =
    length ([] :: 'f list) + ceil-log (to-nat dg + 1) ∧
  PTranscript t = PTranscript s5 @ roots ∧
  PState t = foldl concat (PState s5) roots ∧
  s5 ≤ t ∧
  PTraceFriCounter t = PTraceFriCounter s5 ∧
  PCompositionFriCounter t =
    PCompositionFriCounter s5 + ceil-log (to-nat dg + 1) ∧
  PAlphaCounter t = PAlphaCounter s5 ∧
  PQueryCounter t = PQueryCounter s5 ∧
  (∀ j < ceil-log (to-nat dg + 1).
    fmllookup (HashMap t)

```

```

      (CompositionFriChallenge (PCompositionFriCounter s5 + j)
        (foldl concat (PState s5) (take (Suc j) roots))) =
      Some (composition-bs ! (length [] :: 'f list) + j)))
  by (rule staged-composition-fri-program-alignment
    [OF controlled len fri-out])
let ?key =
  CompositionFriChallenge (PCompositionFriCounter s5 + i)
    (foldl concat (PState s5) (take (Suc i) roots))
have lookup-t:
  fmlookup (HashMap t) ?key = Some (composition-bs ! i)
  using alignment i-bound by simp
have ext-s-t: s ≤ t
  using hash-target-program-staged-composition-fri-vector-program
    [OF wf controlled,
      of fri-vector-position-values (B dg) i] outcome
  unfolding hash-target-program-def hash-extension-preserving-def
  by blast
have none-s: fmlookup (HashMap s) ?key = None
proof (cases fmlookup (HashMap s) ?key)
  case None
  then show ?thesis .
next
  case (Some old)
  have old-lookup-t:
    fmlookup (HashMap t) ?key = Some old
    by (rule hash-extension-lookup[OF Some ext-s-t])
  then have old-eq: old = composition-bs ! i
    using lookup-t by simp
  have old ∉ fri-vector-position-values (B dg) i
    using absent Some
    unfolding composition-fri-challenge-values-absent-def by blast
  then show ?thesis
    using old-eq hit by simp
qed
have hash-map-new-output-hit
  (fri-vector-position-values (B dg) i) s t
  unfolding hash-map-new-output-hit-def
  using none-s lookup-t hit by blast
then show ?thesis
  unfolding hash-new-output-hit-event-def by simp
qed

lemma staged-composition-fri-vector-coordinate-bound-fixed-degree:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < ceil-log (to-nat dg + 1)
  and absent:
    composition-fri-challenge-values-absent
      (fri-vector-position-values (B dg) i) s

```



```

shows
  wp-event (staged-composition-fri-vector-program A)
    (λout. case out of None ⇒ False
      | Some ((dg', composition-bs), -) ⇒
        dg' = dg ∧
        composition-bs ! i ∈ fri-vector-position-values (B dg) i) s ≤
    staged-phase-target-error (fri-vector-position-values (B dg) i)
      (staged-composition-fri-vector-search-queries budgets)
proof -
  let ?P =
    λout. case out of None ⇒ False
      | Some ((dg', composition-bs), -) ⇒
        dg' = dg ∧
        composition-bs ! i ∈ fri-vector-position-values (B dg) i
  let ?H =
    hash-new-output-hit-event (fri-vector-position-values (B dg) i) s
  have event-imp-hit:
    wp-event (staged-composition-fri-vector-program A) ?P s ≤
    wp-event (staged-composition-fri-vector-program A) ?H s
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out ∈ set-dist
      (execute (staged-composition-fri-vector-program A) s)
    and event: ?P out
  show ?H out
  proof (cases out)
    case None
    then show ?thesis using event by simp
  next
    case (Some result)
    then obtain dg' composition-bs t where
      out-eq: out = Some ((dg', composition-bs), t)
      by (cases result, auto split: prod.splits)
    have dg-eq: dg' = dg
      using event unfolding out-eq by simp
    have outcome:
      Some ((dg, composition-bs), t) ∈
        set-dist
        (execute (staged-composition-fri-vector-program A) s)
      using support unfolding out-eq dg-eq .
    have hit:
      composition-bs ! i ∈ fri-vector-position-values (B dg) i
      using event unfolding out-eq by simp
    show ?thesis
      unfolding out-eq dg-eq
      by (rule
        staged-composition-fri-vector-coordinate-hit-imp-new-output-hit
        [where B=B, OF wf controlled i-bound absent outcome hit])

```

```

    qed
  qed
  also have
    wp-event (staged-composition-fri-vector-program A) ?H s ≤
      staged-phase-target-error (fri-vector-position-values (B dg) i)
        (staged-composition-fri-vector-search-queries budgets)
    by (rule staged-composition-fri-vector-target-hit-bound
        [OF wf controlled])
  finally show ?thesis .
qed

lemma composition-fri-challenge-values-absent-subset:
  assumes subset:  $A \subseteq B$ 
    and absent: composition-fri-challenge-values-absent B s
  shows composition-fri-challenge-values-absent A s
  using subset absent
  unfolding composition-fri-challenge-values-absent-def by blast

lemma hash-map-new-output-hit-mono-set:
  assumes subset:  $A \subseteq B$ 
    and hit: hash-map-new-output-hit A s t
  shows hash-map-new-output-hit B s t
  using subset hit unfolding hash-map-new-output-hit-def by blast

lemma hash-new-output-hit-event-mono-set:
  assumes subset:  $A \subseteq B$ 
    and hit: hash-new-output-hit-event A s out
  shows hash-new-output-hit-event B s out
proof (cases out)
  case None
  then show ?thesis
    using hit unfolding hash-new-output-hit-event-def by simp
next
  case (Some result)
  then obtain x t where out-eq: out = Some (x, t)
    by (cases result) simp
  have map-hit: hash-map-new-output-hit A s t
    using hit unfolding out-eq hash-new-output-hit-event-def by simp
  have hash-map-new-output-hit B s t
    by (rule hash-map-new-output-hit-mono-set[OF subset map-hit])
  show ?thesis
    using ⟨hash-map-new-output-hit B s t⟩
    unfolding out-eq hash-new-output-hit-event-def by simp
qed

lemma staged-composition-fri-vector-coordinate-bound-union:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and absent:

```

```

    composition-fri-challenge-values-absent
    (⋃ dg. fri-vector-position-values (B dg) i) s
shows
  wp-event (staged-composition-fri-vector-program A)
    (λout. case out of None ⇒ False
      | Some ((dg, composition-bs), -) ⇒
        i < ceil-log (to-nat dg + 1) ∧
        composition-bs ! i ∈
        fri-vector-position-values (B dg) i) s ≤
    staged-phase-target-error
    (⋃ dg. fri-vector-position-values (B dg) i)
    (staged-composition-fri-vector-search-queries budgets)
proof -
  let ?U = (⋃ dg. fri-vector-position-values (B dg) i)
  let ?P =
    λout. case out of None ⇒ False
      | Some ((dg, composition-bs), -) ⇒
        i < ceil-log (to-nat dg + 1) ∧
        composition-bs ! i ∈ fri-vector-position-values (B dg) i
  let ?H = hash-new-output-hit-event ?U s
  have event-imp-hit:
    wp-event (staged-composition-fri-vector-program A) ?P s ≤
    wp-event (staged-composition-fri-vector-program A) ?H s
  proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈ set-dist
    (execute (staged-composition-fri-vector-program A) s)
  and event: ?P out
  show ?H out
  proof (cases out)
  case None
  then show ?thesis using event by simp
  next
  case (Some result)
  then obtain dg composition-bs t where
    out-eq: out = Some ((dg, composition-bs), t)
  by (cases result, auto split: prod.splits)
  have outcome:
    Some ((dg, composition-bs), t) ∈
    set-dist
    (execute (staged-composition-fri-vector-program A) s)
  using support unfolding out-eq .
  have i-bound: i < ceil-log (to-nat dg + 1)
  using event unfolding out-eq by simp
  have hit:
    composition-bs ! i ∈ fri-vector-position-values (B dg) i
  using event unfolding out-eq by simp
  have small-absent:

```

```

    composition-fri-challenge-values-absent
      (fri-vector-position-values (B dg) i) s
  by (rule composition-fri-challenge-values-absent-subset
      [OF - absent]) blast
have small-hit:
  hash-new-output-hit-event
    (fri-vector-position-values (B dg) i) s
    (Some ((dg, composition-bs), t))
  by (rule
      staged-composition-fri-vector-coordinate-hit-imp-new-output-hit
      [where B=B,
        OF wf controlled i-bound small-absent outcome hit])
have subset:
  fri-vector-position-values (B dg) i  $\subseteq$  ?U
  by blast
show ?thesis
  unfolding out-eq
  by (rule hash-new-output-hit-event-mono-set
      [OF subset small-hit])
qed
qed
also have
  wp-event (staged-composition-fri-vector-program A) ?H s  $\leq$ 
    staged-phase-target-error ?U
    (staged-composition-fri-vector-search-queries budgets)
  by (rule staged-composition-fri-vector-target-hit-bound
      [OF wf controlled])
finally show ?thesis .
qed

lemma staged-composition-fri-vector-bad-bound-union:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and subset:
     $\bigwedge dg. B dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$ 
  and absent:
     $\bigwedge i. i < \text{ceil-log } (\text{maxDegree} + 1) \implies$ 
      composition-fri-challenge-values-absent
      ( $\bigcup dg. \text{fri-vector-position-values } (B dg) i$ ) s
  shows
    wp-event (staged-composition-fri-vector-program A)
      ( $\lambda out. \text{case out of None} \implies \text{False}$ 
        | Some ((dg, composition-bs), -)  $\implies$ 
          composition-bs  $\in B dg \wedge$ 
           $0 < \text{ceil-log } (\text{to-nat } dg + 1) \wedge$ 
           $\text{ceil-log } (\text{to-nat } dg + 1) \leq \text{ceil-log } (\text{maxDegree} + 1)$ ) s  $\leq$ 
      ( $\sum i < \text{ceil-log } (\text{maxDegree} + 1).$ 
        staged-phase-target-error
        ( $\bigcup dg. \text{fri-vector-position-values } (B dg) i$ ))

```

```

      (staged-composition-fri-vector-search-queries budgets))
proof -
  let ?coord =
     $\lambda i$  out. case out of None  $\Rightarrow$  False
    | Some ((dg, composition-bs), -)  $\Rightarrow$ 
       $i < \text{ceil-log } (\text{to-nat } dg + 1) \wedge$ 
      composition-bs !  $i \in \text{fri-vector-position-values } (B \text{ dg}) \ i$ 
  have cover:
    wp-event (staged-composition-fri-vector-program A)
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
    | Some ((dg, composition-bs), -)  $\Rightarrow$ 
      composition-bs  $\in B \text{ dg} \wedge$ 
       $0 < \text{ceil-log } (\text{to-nat } dg + 1) \wedge$ 
       $\text{ceil-log } (\text{to-nat } dg + 1) \leq$ 
       $\text{ceil-log } (\text{maxDegree} + 1)) \ s \leq$ 
       $(\sum i < \text{ceil-log } (\text{maxDegree} + 1)).$ 
      wp-event (staged-composition-fri-vector-program A)
      (?coord i) s)
  by (rule staged-composition-fri-vector-bad-coordinate-cover-bound
    [OF subset])
  also have ...  $\leq$ 
     $(\sum i < \text{ceil-log } (\text{maxDegree} + 1)).$ 
    staged-phase-target-error
    ( $\bigcup dg. \text{fri-vector-position-values } (B \text{ dg}) \ i$ )
    (staged-composition-fri-vector-search-queries budgets))
  proof (rule sum-mono)
    fix i
    assume i-in:  $i \in \{.. < \text{ceil-log } (\text{maxDegree} + 1)\}$ 
    then have i-bound:  $i < \text{ceil-log } (\text{maxDegree} + 1)$ 
    by simp
    show
      wp-event (staged-composition-fri-vector-program A)
      (?coord i) s  $\leq$ 
      staged-phase-target-error
      ( $\bigcup dg. \text{fri-vector-position-values } (B \text{ dg}) \ i$ )
      (staged-composition-fri-vector-search-queries budgets)
    by (rule staged-composition-fri-vector-coordinate-bound-union
      [OF wf controlled absent[OF i-bound]])
  qed
  finally show ?thesis .
qed

```

```

lemma staged-composition-fri-full-vector-bad-bound-union:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and subset:
     $\bigwedge dg. B \text{ dg} \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$ 
  and absent:
     $\bigwedge i. i < \text{ceil-log } (\text{maxDegree} + 1) \Rightarrow$ 

```

composition-fri-challenge-values-absent
 $(\bigcup dg. \text{fri-vector-position-values } (B \text{ dg}) \ i)$
adversary-initial-state
shows
 $\text{wp-event } (\text{staged-composition-fri-full-vector-program } A)$
 $(\text{staged-composition-prefix-fri-vector-hit } B)$
adversary-initial-state $\leq$
 $(\sum i < \text{ceil-log } (\text{maxDegree} + 1).$
staged-phase-target-error
 $(\bigcup dg. \text{fri-vector-position-values } (B \text{ dg}) \ i)$
 $(\text{staged-composition-fri-vector-search-queries budgets}))$
proof –
let $?Q =$
 $\lambda out. \text{case out of None} \Rightarrow \text{False}$
 $| \text{Some } ((dg, \text{composition-bs}), -) \Rightarrow$
 $\text{composition-bs} \in B \text{ dg} \wedge$
 $0 < \text{ceil-log } (\text{to-nat } dg + 1) \wedge$
 $\text{ceil-log } (\text{to-nat } dg + 1) \leq \text{ceil-log } (\text{maxDegree} + 1)$
have *projected*:
 $\text{wp-event } (\text{staged-composition-fri-vector-program } A) \ ?Q$
adversary-initial-state $\leq$
 $(\sum i < \text{ceil-log } (\text{maxDegree} + 1).$
staged-phase-target-error
 $(\bigcup dg. \text{fri-vector-position-values } (B \text{ dg}) \ i)$
 $(\text{staged-composition-fri-vector-search-queries budgets}))$
by (*rule staged-composition-fri-vector-bad-bound-union*
 $[OF \text{ wf controlled subset absent}])$
have *eq*:
 $\text{wp-event } (\text{staged-composition-fri-vector-program } A) \ ?Q$
adversary-initial-state $=$
 $\text{wp-event } (\text{staged-composition-fri-full-vector-program } A)$
 $(\text{staged-composition-prefix-fri-vector-hit } B)$
adversary-initial-state
unfolding *staged-composition-fri-vector-program-projection*
wp-event-bind-return-map
staged-composition-prefix-fri-vector-hit-def
by (*simp split: option.splits prod.splits*)
show *?thesis*
using *projected unfolding eq* .
qed

lemma *staged-transcript-program-composition-fri-vector-bound*:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *subset*:
 $\bigwedge dg. B \text{ dg} \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$
and *absent*:
 $\bigwedge i. i < \text{ceil-log } (\text{maxDegree} + 1) \implies$
 $\text{composition-fri-challenge-values-absent}$

$(\bigcup dg. \text{fri-vector-position-values } (B \text{ dg}) \ i)$
adversary-initial-state

shows

wp-event (*staged-transcript-program* *A*)
(staged-transcript-composition-fri-vector-guarded-hit *B*)
adversary-initial-state $\leq$
 $(\sum i < \text{ceil-log } (\text{maxDegree} + 1).$
staged-phase-target-error
 $(\bigcup dg. \text{fri-vector-position-values } (B \text{ dg}) \ i)$
(staged-composition-fri-vector-search-queries budgets))

proof –

have *prefix-bound*:

wp-event (*staged-composition-fri-full-vector-program* *A*)
(staged-composition-prefix-fri-vector-hit *B*)
adversary-initial-state $\leq$
 $(\sum i < \text{ceil-log } (\text{maxDegree} + 1).$
staged-phase-target-error
 $(\bigcup dg. \text{fri-vector-position-values } (B \text{ dg}) \ i)$
(staged-composition-fri-vector-search-queries budgets))

by (*rule staged-composition-fri-full-vector-bad-bound-union*
 $[OF \text{ wf controlled subset absent}]$)

show *?thesis*

unfolding *staged-transcript-program-composition-fri-full-vector-decomp*

proof (*rule wp-event-bind-bound-by-head-event* $[OF \text{ prefix-bound}]$)

show

staged-transcript-composition-fri-vector-guarded-hit *B* *None* $\implies$
staged-composition-prefix-fri-vector-hit *B* *None*

unfolding *staged-transcript-composition-fri-vector-guarded-hit-def*
staged-composition-prefix-fri-vector-hit-def

by *simp*

next

fix *prefix t out*

assume *cont*:

out $\in$
set-dist
(execute
(staged-after-composition-fri-full-vector-program *A* *prefix*) *t*)

and *hit*:

staged-transcript-composition-fri-vector-guarded-hit *B* *out*

show *staged-composition-prefix-fri-vector-hit* *B* (*Some* (*prefix*, *t*))

by (*rule staged-after-composition-fri-full-vector-program-hitD*
 $[OF \text{ cont hit}]$)

qed

qed

lemma *staged-query-program-alignment*:

assumes *controlled*: *staged-adversary-controlled budgets* *A*

and *bound*: $i + n \leq \text{length } (\text{query-opening-budgets budgets})$

and *outcome*:

Some (chunks, t) ∈
 set-dist (execute (staged-query-program A i n) s)
shows
 length chunks = n ∧
 PTranscript t = PTranscript s @ List.concat chunks ∧
 PState t = foldl concat (PState s) (List.concat chunks) ∧
 PTraceFriCounter t = PTraceFriCounter s ∧
 PCompositionFriCounter t = PCompositionFriCounter s ∧
 PAlphaCounter t = PAlphaCounter s ∧
 PQueryCounter t = PQueryCounter s + n
using bound outcome
proof (induction n arbitrary: i chunks s t)
case 0
then show ?case **by** simp
next
case (Suc n)
from staged-query-program-Suc-outcomeE[OF Suc.premis(2)]
obtain raw chunk chunks' s1 s2 s3 **where**
 challenge-out:
 Some (raw, s1) ∈
 set-dist (execute receive-query-index-challenge s)
and stage-out:
 Some (chunk, s2) ∈
 set-dist (execute (query-opening-stage A i raw) s1)
and record-out:
 Some ((), s3) ∈
 set-dist (execute (record-staged-messages chunk) s2)
and rest-out:
 Some (chunks', t) ∈
 set-dist (execute (staged-query-program A (Suc i) n) s3)
and chunks-eq: chunks = chunk # chunks'
 .
have challenge-fields:
 PState s1 = PState s ∧
 PTranscript s1 = PTranscript s
using receive-query-index-challenge-outcome[OF challenge-out] **by** simp
have challenge-counters:
 PTraceFriCounter s1 = PTraceFriCounter s ∧
 PCompositionFriCounter s1 = PCompositionFriCounter s ∧
 PAlphaCounter s1 = PAlphaCounter s ∧
 PQueryCounter s1 = Suc (PQueryCounter s)
using receive-query-index-challenge-counter-outcome[OF challenge-out]
by simp
have i-bound: i < length (query-opening-budgets budgets)
using Suc.premis(1) **by** simp
have stage-controlled:
 controlled-ro-program (query-opening-budgets budgets ! i)
 (query-opening-stage A i raw)
using controlled i-bound


```

unfolding staged-adversary-controlled-def by blast
have stage-fields:
  PState s2 = PState s1 ∧
  PTranscript s2 = PTranscript s1 ∧
  PTraceFriCounter s2 = PTraceFriCounter s1 ∧
  PCompositionFriCounter s2 = PCompositionFriCounter s1 ∧
  PAlphaCounter s2 = PAlphaCounter s1 ∧
  PQueryCounter s2 = PQueryCounter s1
using controlled-stage-outcome-fields[OF stage-controlled stage-out]
by simp
have s3:
  s3 = s2(|
    PState := foldl concat (PState s2) chunk,
    PTranscript := PTranscript s2 @ chunk|)
by (rule record-staged-messages-outcome[OF record-out])
have s3-fields:
  PState s3 = foldl concat (PState s) chunk ∧
  PTranscript s3 = PTranscript s @ chunk ∧
  PTraceFriCounter s3 = PTraceFriCounter s ∧
  PCompositionFriCounter s3 = PCompositionFriCounter s ∧
  PAlphaCounter s3 = PAlphaCounter s ∧
  PQueryCounter s3 = Suc (PQueryCounter s)
unfolding s3
using challenge-fields challenge-counters stage-fields by simp
have rest-bound:
  Suc i + n ≤ length (query-opening-budgets budgets)
using Suc.prem(1) by simp
have rest-alignment:
  length chunks' = n ∧
  PTranscript t = PTranscript s3 @ List.concat chunks' ∧
  PState t =
    foldl concat (PState s3) (List.concat chunks') ∧
  PTraceFriCounter t = PTraceFriCounter s3 ∧
  PCompositionFriCounter t = PCompositionFriCounter s3 ∧
  PAlphaCounter t = PAlphaCounter s3 ∧
  PQueryCounter t = PQueryCounter s3 + n
by (rule Suc.IH[OF rest-bound rest-out])
show ?case
unfolding chunks-eq
using s3-fields rest-alignment
by (simp add: foldl-append)
qed

```

```

lemma staged-query-program-outcome-with-raws:
assumes controlled: staged-adversary-controlled budgets A
and bound: i + n ≤ length (query-opening-budgets budgets)
and outcome:
  Some (chunks, t) ∈
    set-dist (execute (staged-query-program A i n) s)

```

shows

```

 $\exists$  raw-idxs query-idxs.
  length raw-idxs = n  $\wedge$ 
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs  $\wedge$ 
  length chunks = n  $\wedge$ 
  PTranscript t = PTranscript s @ List.concat chunks  $\wedge$ 
  PState t = state-after-query-chunks (PState s) chunks n  $\wedge$ 
  s  $\leq$  t  $\wedge$ 
  PQueryCounter t = PQueryCounter s + n  $\wedge$ 
  ( $\forall j < n$ .
    fmllookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + j)
        (state-after-query-chunks (PState s) chunks j)) =
        Some (raw-idxs ! j))  $\wedge$ 
    ( $\forall idx \in$  set query-idxs. idx < length * scale)
  using bound outcome
proof (induction n arbitrary: i chunks s t)
case 0
then show ?case
  by (intro exI[of - []])
    (simp add: state-after-query-chunks-def hash-ext-refl)
next
case (Suc n)
from staged-query-program-Suc-outcomeE[OF Suc.prem1(2)]
obtain raw chunk chunks' s1 s2 s3 where
  challenge-out:
    Some (raw, s1)  $\in$ 
    set-dist (execute receive-query-index-challenge s)
and stage-out:
    Some (chunk, s2)  $\in$ 
    set-dist (execute (query-opening-stage A i raw) s1)
and record-out:
    Some ((), s3)  $\in$ 
    set-dist (execute (record-staged-messages chunk) s2)
and rest-out:
    Some (chunks', t)  $\in$ 
    set-dist (execute (staged-query-program A (Suc i) n) s3)
and chunks-eq: chunks = chunk # chunks'
  .
have i-bound: i < length (query-opening-budgets budgets)
  using Suc.prem1(1) by simp
have stage-controlled:
    controlled-ro-program (query-opening-budgets budgets ! i)
      (query-opening-stage A i raw)
  using controlled i-bound
  unfolding staged-adversary-controlled-def by blast
have challenge:
  s  $\leq$  s1  $\wedge$ 
  PState s1 = PState s  $\wedge$ 

```

```

  PTranscript s1 = PTranscript s ∧
  fmlookup (HashMap s1)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  by (rule receive-query-index-challenge-outcome[OF challenge-out])
have challenge-counters:
  PQueryCounter s1 = Suc (PQueryCounter s)
  using receive-query-index-challenge-counter-outcome[OF challenge-out]
  by simp
have ext-s1-s2: s1 ≤ s2
  using controlled-ro-program-extension[OF stage-controlled] stage-out
  unfolding hash-extension-preserving-def by blast
have stage-fields:
  PState s2 = PState s1 ∧
  PTranscript s2 = PTranscript s1 ∧
  PQueryCounter s2 = PQueryCounter s1
  using controlled-stage-outcome-fields[OF stage-controlled stage-out]
  by simp
have s3-eq:
  s3 = s2(|
    PState := foldl concat (PState s2) chunk,
    PTranscript := PTranscript s2 @ chunk|)
  by (rule record-staged-messages-outcome[OF record-out])
have ext-s2-s3: s2 ≤ s3
  unfolding s3-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have s3-fields:
  PState s3 = foldl concat (PState s) chunk ∧
  PTranscript s3 = PTranscript s @ chunk ∧
  PQueryCounter s3 = Suc (PQueryCounter s)
  unfolding s3-eq using challenge stage-fields challenge-counters by simp
have rest-bound:
  Suc i + n ≤ length (query-opening-budgets budgets)
  using Suc.premis(1) by simp
from Suc.IH[OF rest-bound rest-out]
obtain raw-tail idx-tail where
  len-raw-tail: length raw-tail = n
  and idx-tail-def:
    idx-tail = map (λraw. index (to-nat raw)) raw-tail
  and len-chunks-tail: length chunks' = n
  and tr-tail:
    PTranscript t = PTranscript s3 @ List.concat chunks'
  and st-tail:
    PState t = state-after-query-chunks (PState s3) chunks' n
  and ext-s3-t: s3 ≤ t
  and query-count-tail:
    PQueryCounter t = PQueryCounter s3 + n
  and lookup-tail:
    ∧j. j < n ⇒
      fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s3 + j)

```

```

      (state-after-query-chunks (PState s3) chunks' j)) =
      Some (raw-tail ! j)
and idx-tail-bound:
   $\forall \text{idx} \in \text{set } \text{idx-tail}. \text{idx} < \text{clength} * \text{scale}$ 
by blast
let ?raws = raw # raw-tail
let ?idxs = index (to-nat raw) # idx-tail
let ?chunks = chunk # chunks'
have idxs-def: ?idxs = map ( $\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})$ ) ?raws
  using idx-tail-def by simp
have tr-all:
  PTranscript t = PTranscript s @ List.concat ?chunks
  using tr-tail s3-fields by simp
have state-shift:
   $\bigwedge j. \text{state-after-query-chunks } (PState s) \text{ ?chunks } (Suc j) =$ 
    state-after-query-chunks (PState s3) chunks' j
  using s3-fields unfolding state-after-query-chunks-def by simp
have st-all:
  PState t = state-after-query-chunks (PState s) ?chunks (Suc n)
  using st-tail state-shift[of n] by simp
have ext-s-s2:  $s \leq s2$ 
  by (rule hash-ext-trans[OF conjunct1[OF challenge] ext-s1-s2])
have ext-s-s3:  $s \leq s3$ 
  by (rule hash-ext-trans[OF ext-s-s2 ext-s2-s3])
have ext-s-t:  $s \leq t$ 
  by (rule hash-ext-trans[OF ext-s-s3 ext-s3-t])
have query-count-all: PQueryCounter t = PQueryCounter s + Suc n
  using query-count-tail s3-fields by simp
have ext-s1-t:  $s1 \leq t$ 
  by (rule hash-ext-trans[OF ext-s1-s2])
  (rule hash-ext-trans[OF ext-s2-s3 ext-s3-t])
have lookup-head-t:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) =
  Some raw
  by (rule hash-extension-lookup[OF conjunct2[OF conjunct2[OF conjunct2[OF challenge]]]
ext-s1-t])
have lookup-all:
   $\bigwedge j. j < Suc n \implies$ 
    fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + j)
      (state-after-query-chunks (PState s) ?chunks j)) =
    Some (?raws ! j)
proof -
  fix j
  assume j-bound:  $j < Suc n$ 
  show
    fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + j)

```

```

      (state-after-query-chunks (PState s) ?chunks j)) =
      Some (?rows ! j)
proof (cases j)
  case 0
  then show ?thesis
    using lookup-head-t unfolding state-after-query-chunks-def by simp
next
  case (Suc k)
  then show ?thesis
    using lookup-tail[of k] j-bound state-shift[of k] s3-fields by simp
qed
qed
have idx-bound-all:
   $\forall idx \in \text{set } ?idxs. \text{idx} < \text{clength} * \text{scale}$ 
  using idx-tail-bound index-less-domain by auto
show ?case
  unfolding chunks-eq
  apply (intro exI[of - ?rows] exI[of - ?idxs] conjI)
    apply (simp add: len-raw-tail)
    apply (rule idxs-def)
    apply (simp add: len-chunks-tail)
    apply (rule tr-all)
    apply (rule st-all)
    apply (rule ext-s-t)
    apply (rule query-count-all)
    apply (intro allI impI, rule lookup-all, assumption)
  using idx-bound-all by simp
qed

lemma checked-staged-query-program-chunks-match-verifier:
assumes outcome:
  Some (chunks, t)  $\in$ 
  set-dist
  (execute
    (checked-staged-query-program A trace-roots composition-roots
      i n) s)
shows
   $\exists \text{raw-idxs query-idxs}.$ 
  length raw-idxs = n  $\wedge$ 
  query-idxs = map ( $\lambda \text{raw}. \text{index } (\text{to-nat raw})$ ) raw-idxs  $\wedge$ 
  length chunks = n  $\wedge$ 
  ( $\forall j < n.$ 
    verifier-query-round-chunk (query-idxs ! j)
    trace-roots composition-roots (chunks ! j))  $\wedge$ 
  ( $\forall idx \in \text{set query-idxs}. \text{idx} < \text{clength} * \text{scale}$ )
using outcome
proof (induction n arbitrary: i chunks s t)
  case 0
  then show ?case

```

```

    by (intro exI[of - []] exI[of - []]) simp
next
case (Suc n)
obtain raw chunk chunks' s1 s2 s3 where
  chunk-shape:
    verifier-query-round-chunk (index (to-nat raw))
    trace-roots composition-roots chunk
  and rest-out:
    Some (chunks', t) ∈
      set-dist
      (execute
        (checked-staged-query-program A trace-roots composition-roots
          (Suc i) n) s3)
  and chunks-eq: chunks = chunk # chunks'
proof (rule checked-staged-query-program-Suc-outcomeE[OF Suc.prem])
fix raw chunk chunks' s1 s2 s3
assume shape:
  verifier-query-round-chunk (index (to-nat raw))
  trace-roots composition-roots chunk
  and rest:
    Some (chunks', t) ∈
      set-dist
      (execute
        (checked-staged-query-program A trace-roots composition-roots
          (Suc i) n) s3)
  and chunks: chunks = chunk # chunks'
show ?thesis
  by (rule that[OF shape rest chunks])
qed
from Suc.IH[OF rest-out]
obtain raw-tail idx-tail where
  len-raw-tail: length raw-tail = n
  and idx-tail-def:
    idx-tail = map (λraw. index (to-nat raw)) raw-tail
  and len-chunks-tail: length chunks' = n
  and tail-shape:
    ∀ j < n.
      verifier-query-round-chunk (idx-tail ! j)
      trace-roots composition-roots (chunks' ! j)
  and idx-tail-bound:
    ∀ idx ∈ set idx-tail. idx < clength * scale
  by blast
let ?raws = raw # raw-tail
let ?idxs = index (to-nat raw) # idx-tail
have idxs-def: ?idxs = map (λraw. index (to-nat raw)) ?raws
  using idx-tail-def by simp
have shape-all:
  ∀ j < Suc n.
    verifier-query-round-chunk (?idxs ! j)

```

```

      trace-roots composition-roots ((chunk # chunks') ! j)
proof (intro allI impI)
  fix j
  assume j-bound:  $j < \text{Suc } n$ 
  show verifier-query-round-chunk (?idxs ! j)
    trace-roots composition-roots ((chunk # chunks') ! j)
proof (cases j)
  case 0
  then show ?thesis
    using chunk-shape by simp
next
  case (Suc k)
  then have k-bound:  $k < n$ 
    using j-bound by simp
  then show ?thesis
    using tail-shape Suc by simp
qed
qed
have idx-bound-all:
   $\forall \text{idx} \in \text{set } ?\text{idxs}. \text{idx} < \text{clength} * \text{scale}$ 
  using idx-tail-bound index-less-domain by auto
show ?case
  unfolding chunks-eq
  by (intro exI[of - ?raws] exI[of - ?idxs] conjI)
    (use len-raw-tail idxs-def len-chunks-tail shape-all idx-bound-all
      in simp-all)
qed

lemma checked-staged-query-program-outcome-with-raws:
assumes controlled: staged-adversary-controlled budgets A
and bound:  $i + n \leq \text{length } (\text{query-opening-budgets } \text{budgets})$ 
and outcome:
  Some (chunks, t)  $\in$ 
    set-dist
      (execute
        (checked-staged-query-program A trace-roots composition-roots
          i n) s)
shows
   $\exists \text{raw-idxs } \text{query-idxs}.$ 
   $\text{length } \text{raw-idxs} = n \wedge$ 
   $\text{query-idxs} = \text{map } (\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})) \text{ raw-idxs} \wedge$ 
   $\text{length } \text{chunks} = n \wedge$ 
   $P\text{Transcript } t = P\text{Transcript } s @ \text{List.concat } \text{chunks} \wedge$ 
   $P\text{State } t = \text{state-after-query-chunks } (P\text{State } s) \text{ chunks } n \wedge$ 
   $s \leq t \wedge$ 
   $P\text{QueryCounter } t = P\text{QueryCounter } s + n \wedge$ 
   $(\forall j < n.$ 
    verifier-query-round-chunk (query-idxs ! j)
    trace-roots composition-roots (chunks ! j))  $\wedge$ 

```

```

  (∀ j < n.
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + j)
        (state-after-query-chunks (PState s) chunks j)) =
        Some (raw-idxs ! j)) ∧
  (∀ idx ∈ set query-idxs. idx < clength * scale)
using bound outcome
proof (induction n arbitrary: i chunks s t)
  case 0
  then show ?case
    by (intro exI[of - []] exI[of - []])
      (simp add: state-after-query-chunks-def hash-ext-refl)
next
  case (Suc n)
from checked-staged-query-program-Suc-outcomeE[OF Suc.premis(2)]
obtain raw chunk chunks' s1 s2 s3 where
  challenge-out:
    Some (raw, s1) ∈
      set-dist (execute receive-query-index-challenge s)
and stage-out:
    Some (chunk, s2) ∈
      set-dist (execute (query-opening-stage A i raw) s1)
and chunk-shape:
    verifier-query-round-chunk (index (to-nat raw))
      trace-roots composition-roots chunk
and record-out:
    Some ((), s3) ∈
      set-dist (execute (record-staged-messages chunk) s2)
and rest-out:
    Some (chunks', t) ∈
      set-dist
        (execute
          (checked-staged-query-program A trace-roots composition-roots
            (Suc i) n) s3)
and chunks-eq: chunks = chunk # chunks'
  .
have i-bound: i < length (query-opening-budgets budgets)
using Suc.premis(1) by simp
have stage-controlled:
  controlled-ro-program (query-opening-budgets budgets ! i)
    (query-opening-stage A i raw)
using controlled i-bound
unfolding staged-adversary-controlled-def by blast
have challenge:
  s ≤ s1 ∧
  PState s1 = PState s ∧
  PTranscript s1 = PTranscript s ∧
  fmlookup (HashMap s1)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw

```



```

  by (rule receive-query-index-challenge-outcome[OF challenge-out])
have challenge-counters:
  PQueryCounter s1 = Suc (PQueryCounter s)
  using receive-query-index-challenge-counter-outcome[OF challenge-out]
  by simp
have ext-s1-s2: s1 ≤ s2
  using controlled-ro-program-extension[OF stage-controlled] stage-out
  unfolding hash-extension-preserving-def by blast
have stage-fields:
  PState s2 = PState s1 ∧
  PTranscript s2 = PTranscript s1 ∧
  PQueryCounter s2 = PQueryCounter s1
  using controlled-stage-outcome-fields[OF stage-controlled stage-out]
  by simp
have s3-eq:
  s3 = s2(
    PState := foldl concat (PState s2) chunk,
    PTranscript := PTranscript s2 @ chunk)
  by (rule record-staged-messages-outcome[OF record-out])
have ext-s2-s3: s2 ≤ s3
  unfolding s3-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have s3-fields:
  PState s3 = foldl concat (PState s) chunk ∧
  PTranscript s3 = PTranscript s @ chunk ∧
  PQueryCounter s3 = Suc (PQueryCounter s)
  unfolding s3-eq using challenge stage-fields challenge-counters by simp
have rest-bound:
  Suc i + n ≤ length (query-opening-budgets budgets)
  using Suc.prem(1) by simp
from Suc.IH[OF rest-bound rest-out]
obtain raw-tail idx-tail where
  len-raw-tail: length raw-tail = n
  and idx-tail-def:
    idx-tail = map (λraw. index (to-nat raw)) raw-tail
  and len-chunks-tail: length chunks' = n
  and tr-tail:
    PTranscript t = PTranscript s3 @ List.concat chunks'
  and st-tail:
    PState t = state-after-query-chunks (PState s3) chunks' n
  and ext-s3-t: s3 ≤ t
  and query-count-tail:
    PQueryCounter t = PQueryCounter s3 + n
  and chunk-shape-tail:
    ∧j. j < n ⇒
      verifier-query-round-chunk (idx-tail ! j)
      trace-roots composition-roots (chunks' ! j)
  and lookup-tail:
    ∧j. j < n ⇒
      fmllookup (HashMap t)

```

```

    (QueryIndexChallenge (PQueryCounter s3 + j)
      (state-after-query-chunks (PState s3) chunks' j)) =
    Some (raw-tail ! j)
  and idx-tail-bound:
     $\forall idx \in \text{set } \text{idx-tail}. \text{idx} < \text{clength} * \text{scale}$ 
  by blast
  let ?raws = raw # raw-tail
  let ?idxs = index (to-nat raw) # idx-tail
  let ?chunks = chunk # chunks'
  have idxs-def: ?idxs = map ( $\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})$ ) ?raws
    using idx-tail-def by simp
  have tr-all:
    PTranscript t = PTranscript s @ List.concat ?chunks
    using tr-tail s3-fields by simp
  have state-shift:
     $\bigwedge j. \text{state-after-query-chunks } (PState s) \text{ ?chunks } (Suc j) =$ 
      state-after-query-chunks (PState s3) chunks' j
    using s3-fields unfolding state-after-query-chunks-def by simp
  have st-all:
    PState t = state-after-query-chunks (PState s) ?chunks (Suc n)
    using st-tail state-shift[of n] by simp
  have ext-s-s2:  $s \leq s2$ 
    by (rule hash-ext-trans[OF conjunct1[OF challenge] ext-s1-s2])
  have ext-s-s3:  $s \leq s3$ 
    by (rule hash-ext-trans[OF ext-s-s2 ext-s2-s3])
  have ext-s-t:  $s \leq t$ 
    by (rule hash-ext-trans[OF ext-s-s3 ext-s3-t])
  have query-count-all: PQueryCounter t = PQueryCounter s + Suc n
    using query-count-tail s3-fields by simp
  have ext-s1-t:  $s1 \leq t$ 
    by (rule hash-ext-trans[OF ext-s1-s2])
      (rule hash-ext-trans[OF ext-s2-s3 ext-s3-t])
  have lookup-head-t:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) =
    Some raw
    by (rule hash-extension-lookup
      [OF conjunct2[OF conjunct2[OF conjunct2[OF challenge]]] ext-s1-t])
  have chunk-shape-all:
     $\bigwedge j. j < Suc n \implies$ 
      verifier-query-round-chunk (?idxs ! j)
      trace-roots composition-roots (?chunks ! j)
  proof -
    fix j
    assume j-bound:  $j < Suc n$ 
    show
      verifier-query-round-chunk (?idxs ! j)
      trace-roots composition-roots (?chunks ! j)
  proof (cases j)

```

```

    case 0
    then show ?thesis
      using chunk-shape by simp
  next
    case (Suc k)
    then show ?thesis
      using chunk-shape-tail[of k] j-bound by simp
  qed
qed
have lookup-all:
   $\bigwedge j. j < \text{Suc } n \implies$ 
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + j)
      (state-after-query-chunks (PState s) ?chunks j)) =
    Some (?rows ! j)
proof -
  fix j
  assume j-bound: j < Suc n
  show
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + j)
        (state-after-query-chunks (PState s) ?chunks j)) =
        Some (?rows ! j)
  proof (cases j)
    case 0
    then show ?thesis
      using lookup-head-t unfolding state-after-query-chunks-def by simp
  next
    case (Suc k)
    then show ?thesis
      using lookup-tail[of k] j-bound state-shift[of k] s3-fields by simp
  qed
qed
have idx-bound-all:
   $\forall \text{idx} \in \text{set } ?\text{idxs}. \text{idx} < \text{clength} * \text{scale}$ 
  using idx-tail-bound index-less-domain by auto
show ?case
  unfolding chunks-eq
  by (intro exI[of - ?rows] exI[of - ?idxs] conjI)
    (use len-raw-tail idxs-def len-chunks-tail tr-all st-all ext-s-t
      query-count-all chunk-shape-all lookup-all idx-bound-all in simp-all)
qed

lemma checked-staged-query-program-chunks-match-verifier-lengths:
  assumes outcome:
    Some (chunks, t)  $\in$ 
    set-dist
    (execute
      (checked-staged-query-program A trace-roots composition-roots

```

$i \text{ rounds}) \ s)$

shows

$\exists \text{ query-ids}.$
 $\text{staged-query-chunks-match-verifier-lengths}$
 $(\text{staged-trace-root} = \text{fr},$
 $\text{staged-trace-fri-roots} = \text{trace-roots},$
 $\text{staged-trace-fri-challenges} = \text{trace-bs},$
 $\text{staged-trace-final} = \text{trace-final},$
 $\text{staged-alphas} = \text{as},$
 $\text{staged-degree} = \text{dg},$
 $\text{staged-composition-fri-roots} = \text{composition-roots},$
 $\text{staged-composition-fri-challenges} = \text{composition-bs},$
 $\text{staged-composition-final} = \text{composition-final},$
 $\text{staged-query-chunks} = \text{chunks})$
 query-ids

proof –

from $\text{checked-staged-query-program-chunks-match-verifier}[\text{OF outcome}]$
obtain $\text{raw-ids} \ \text{query-ids}$ **where**
 $\text{len-chunks: length chunks} = \text{rounds}$
and chunk-shape:
 $\forall j < \text{rounds}.$
 $\text{verifier-query-round-chunk} (\text{query-ids} \ ! \ j)$
 $\text{trace-roots composition-roots} (\text{chunks} \ ! \ j)$
by blast
have chunk-lens:
 $\forall j < \text{rounds}.$
 $\text{length} (\text{chunks} \ ! \ j) =$
 $\text{verifier-query-round-transcript-length} (\text{query-ids} \ ! \ j)$
 $\text{trace-roots composition-roots}$
using $\text{chunk-shape verifier-query-round-chunk-length}$ **by** blast
show $?thesis$
by $(\text{intro exI}[\text{of - query-ids}])$
 $(\text{use len-chunks chunk-lens in}$
 $\langle \text{simp add: staged-query-chunks-match-verifier-lengths-def} \rangle)$

qed

lemma $\text{checked-staged-transcript-program-query-chunks-match-verifier-lengths:}$
assumes outcome:
 $\text{Some} (\text{data}, t) \in$
 set-dist
 $(\text{execute} (\text{checked-staged-transcript-program } A) \ s)$
shows $\exists \text{ query-ids}.$
 $\text{staged-query-chunks-match-verifier-lengths data query-ids}$

proof –

from outcome **obtain** $\text{fr s1 s2 trace-roots trace-bs s3 trace-final s4 s5}$
 $\text{as s6 dg s7 s8 s9 composition-roots composition-bs s10}$
 $\text{composition-final s11 s12 query-chunks}$ **where**
 query-out:
 $\text{Some} (\text{query-chunks}, s12) \in$

```

    set-dist
      (execute
        (checked-staged-query-program A trace-roots composition-roots
          0 rounds)
        s11)
  and data-eq:
    data =
      (|staged-trace-root = fr,
        staged-trace-fri-roots = trace-roots,
        staged-trace-fri-challenges = trace-bs,
        staged-trace-final = trace-final,
        staged-alphas = as,
        staged-degree = dg,
        staged-composition-fri-roots = composition-roots,
        staged-composition-fri-challenges = composition-bs,
        staged-composition-final = composition-final,
        staged-query-chunks = query-chunks)
    unfolding checked-staged-transcript-program-def Let-def
    by (auto elim!: set-dist-bindE split: prod.splits)
  show ?thesis
    unfolding data-eq
    by (rule checked-staged-query-program-chunks-match-verifier-lengths
      [OF query-out])
qed

lemma checked-staged-security-experiment-with-data-state-query-lengthsE:
  assumes outcome:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
          initial-state)
  obtains query-idxs where
    staged-query-chunks-match-verifier-lengths data query-idxs
proof -
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF outcome]
  have builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A) initial-state)
  by blast
  from checked-staged-transcript-program-query-chunks-match-verifier-lengths
    [OF builder]
  obtain query-idxs where
    staged-query-chunks-match-verifier-lengths data query-idxs
  by blast
  then show ?thesis
    by (rule that)

```

qed

lemma *verifier-query-round-program-replays-shaped-chunk:*

assumes *outcome:*

Some $((\cdot), t) \in$

set-dist

(execute

(verifier-query-round-program fr f-fl f-final as fl final) s)

and *lookup:*

fmlookup $(\text{HashMap } s)$

$(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some } \text{raw}$

and *chunk-shape:*

verifier-query-round-chunk $(\text{index } (\text{to-nat } \text{raw}))$

$(\text{map } \text{snd } f\text{-fl}) (\text{map } \text{snd } fl) \text{ chunk}$

and *transcript:* $P\text{Transcript } s = \text{chunk} @ \text{rest}$

shows

$P\text{Transcript } t = \text{rest} \wedge$

$P\text{State } t = \text{foldl concat } (P\text{State } s) \text{ chunk} \wedge$

$P\text{QueryCounter } t = \text{Suc } (P\text{QueryCounter } s) \wedge$

$s \leq t$

proof –

from *verifier-query-round-program-outcome* $[OF \text{ outcome}]$

obtain *raw' idx' chunk' where*

idx'-def: $\text{idx}' = \text{index } (\text{to-nat } \text{raw}')$

and *chunk'-shape:*

verifier-query-round-chunk $\text{idx}' (\text{map } \text{snd } f\text{-fl}) (\text{map } \text{snd } fl)$

chunk'

and *transcript':* $P\text{Transcript } s = \text{chunk}' @ P\text{Transcript } t$

and *state-t:* $P\text{State } t = \text{foldl concat } (P\text{State } s) \text{ chunk}'$

and *ext:* $s \leq t$

and *counter-t:* $P\text{QueryCounter } t = \text{Suc } (P\text{QueryCounter } s)$

and *lookup-t:*

fmlookup $(\text{HashMap } t)$

$(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some } \text{raw}'$

by *blast*

have *lookup-t-from-s:*

fmlookup $(\text{HashMap } t)$

$(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some } \text{raw}$

by $(\text{rule hash-extension-lookup}[OF \text{ lookup ext}])$

have *raw'-eq:* $\text{raw}' = \text{raw}$

using *lookup-t lookup-t-from-s* **by** *simp*

have *idx'-eq:* $\text{idx}' = \text{index } (\text{to-nat } \text{raw})$

unfolding *idx'-def raw'-eq* **by** *simp*

have *len-eq:* $\text{length } \text{chunk}' = \text{length } \text{chunk}$

using *verifier-query-round-chunk-length* $[OF \text{ chunk'-shape}]$

verifier-query-round-chunk-length $[OF \text{ chunk-shape}]$

unfolding *idx'-eq* **by** *simp*

have *chunk'-eq:* $\text{chunk}' = \text{chunk}$

proof –

```

have chunk' = take (length chunk') (PTranscript s)
  using transcript' by simp
also have ... = chunk
  using transcript len-eq by simp
finally show ?thesis .
qed
have transcript-t: PTranscript t = rest
  using transcript' transcript unfolding chunk'-eq by simp
show ?thesis
  using transcript-t state-t counter-t ext unfolding chunk'-eq by simp
qed

lemma ntimes-verifier-query-round-program-replays-shaped-chunks:
  assumes outcome:
    Some (results, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            n) s)
  and len-raw: length raw-idxs = n
  and len-chunks: length chunks = n
  and chunk-shape:
     $\bigwedge i. i < n \implies$ 
    verifier-query-round-chunk (index (to-nat (raw-idxs ! i)))
      (map snd f-fl) (map snd fl) (chunks ! i)
  and lookup:
     $\bigwedge i. i < n \implies$ 
    fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) chunks i)) =
    Some (raw-idxs ! i)
  and transcript: PTranscript s = List.concat chunks @ rest
  shows
    PTranscript t = rest  $\wedge$ 
    PState t = state-after-query-chunks (PState s) chunks n  $\wedge$ 
    PQueryCounter t = PQueryCounter s + n  $\wedge$ 
    s  $\leq$  t
  using outcome len-raw len-chunks chunk-shape lookup transcript
proof (induction n arbitrary: s results t raw-idxs chunks rest)
  case 0
  then show ?case
    by (simp add: state-after-query-chunks-def hash-ext-refl)
next
  case (Suc n)
  from Suc.prem1(2) obtain raw raw-tail where raw-eq:
    raw-idxs = raw # raw-tail
  by (cases raw-idxs) simp-all
  from Suc.prem1(3) obtain chunk chunk-tail where chunks-eq:

```

```

  chunks = chunk # chunk-tail
  by (cases chunks) simp-all
  from Suc.premis(1) obtain u s1 results' where
  head:
    Some (u, s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results', t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            n) s1)
  and results-eq: results = u # results'
  by (auto elim!: set-dist-bindE)
  have u-eq: u = ()
  by (cases u) simp
  have len-raw-tail: length raw-tail = n
  using Suc.premis(2) unfolding raw-eq by simp
  have len-chunk-tail: length chunk-tail = n
  using Suc.premis(3) unfolding chunks-eq by simp
  have head-shape:
    verifier-query-round-chunk (index (to-nat raw))
      (map snd f-fl) (map snd fl) chunk
  using Suc.premis(4)[of 0] unfolding raw-eq chunks-eq by simp
  have head-lookup:
    fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) =
      Some raw
  using Suc.premis(5)[of 0]
  unfolding raw-eq chunks-eq state-after-query-chunks-def by simp
  have head-transcript:
    PTranscript s = chunk @ (List.concat chunk-tail @ rest)
  using Suc.premis(6) unfolding chunks-eq by simp
  from verifier-query-round-program-replays-shaped-chunk
    [OF head[unfolded u-eq] head-lookup head-shape head-transcript]
  have replay-head:
    PTranscript s1 = List.concat chunk-tail @ rest ∧
    PState s1 = foldl concat (PState s) chunk ∧
    PQueryCounter s1 = Suc (PQueryCounter s) ∧
    s ≤ s1 .
  have tail-shape:
    ∧ i. i < n ⇒
      verifier-query-round-chunk (index (to-nat (raw-tail ! i)))
        (map snd f-fl) (map snd fl) (chunk-tail ! i)
  proof -
    fix i

```



```

assume i-bound:  $i < n$ 
then show
  verifier-query-round-chunk (index (to-nat (raw-tail ! i)))
    (map snd f-fl) (map snd fl) (chunk-tail ! i)
  using Suc.prem(4)[of Suc i]
  unfolding raw-eq chunks-eq by simp
qed
have state-shift:
   $\wedge i. \text{state-after-query-chunks } (PState\ s) \ (chunk\ \# \ chunk\text{-tail})$ 
    (Suc i) =
    state-after-query-chunks (PState s1) chunk-tail i
  using replay-head unfolding state-after-query-chunks-def by simp
have tail-lookup:
   $\wedge i. i < n \implies$ 
    fmlookup (HashMap s1)
      (QueryIndexChallenge (PQueryCounter s1 + i)
        (state-after-query-chunks (PState s1) chunk-tail i)) =
      Some (raw-tail ! i)
proof –
  fix i
  assume i-bound:  $i < n$ 
  have lookup-s:
    fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s + Suc i)
        (state-after-query-chunks (PState s) (chunk # chunk-tail)
          (Suc i))) =
      Some (raw-tail ! i)
  using Suc.prem(5)[of Suc i] i-bound
  unfolding raw-eq chunks-eq by simp
  have key-eq:
    QueryIndexChallenge (PQueryCounter s + Suc i)
      (state-after-query-chunks (PState s) (chunk # chunk-tail)
        (Suc i)) =
    QueryIndexChallenge (PQueryCounter s1 + i)
      (state-after-query-chunks (PState s1) chunk-tail i)
  using replay-head state-shift[of i] by simp
  show
    fmlookup (HashMap s1)
      (QueryIndexChallenge (PQueryCounter s1 + i)
        (state-after-query-chunks (PState s1) chunk-tail i)) =
      Some (raw-tail ! i)
    using hash-extension-lookup[OF lookup-s conjunct2[OF conjunct2[OF conjunct2[OF replay-head]]]]
    unfolding key-eq .
qed
from Suc.IH[OF tail len-raw-tail len-chunk-tail tail-shape tail-lookup
  conjunct1[OF replay-head]]
have replay-tail:
  PTranscript t = rest  $\wedge$ 

```

```

    PState t = state-after-query-chunks (PState s1) chunk-tail n ∧
    PQueryCounter t = PQueryCounter s1 + n ∧
    s1 ≤ t .
have state-all:
    PState t =
      state-after-query-chunks (PState s) (chunk # chunk-tail) (Suc n)
    using replay-tail state-shift[of n] by simp
have counter-all: PQueryCounter t = PQueryCounter s + Suc n
    using replay-head replay-tail by simp
have ext-all: s ≤ t
    using replay-head replay-tail by (blast intro: hash-ext-trans)
show ?case
    unfolding raw-eq chunks-eq
    using replay-tail state-all counter-all ext-all by simp
qed

lemma checked-staged-query-program-ntimes-replay:
assumes controlled: staged-adversary-controlled budgets A
and bound: i + n ≤ length (query-opening-budgets budgets)
and checked:
    Some (chunks, attacker-t) ∈
      set-dist
        (execute
          (checked-staged-query-program A (map snd f-fl) (map snd fl)
            i n) attacker-s)
and verifier:
    Some (results, verifier-t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            n) verifier-s)
and hash-ext: attacker-t ≤ verifier-s
and state-eq: PState verifier-s = PState attacker-s
and counter-eq: PQueryCounter verifier-s = PQueryCounter attacker-s
and transcript-eq:
    PTranscript verifier-s = List.concat chunks @ rest
shows
    PTranscript verifier-t = rest ∧
    PState verifier-t =
      state-after-query-chunks (PState verifier-s) chunks n ∧
    PQueryCounter verifier-t = PQueryCounter verifier-s + n ∧
    verifier-s ≤ verifier-t
proof —
from checked-staged-query-program-outcome-with-raws
  [OF controlled bound checked]
obtain raw-idxs query-idxs where
  len-raw: length raw-idxs = n
and query-idxs-def:

```

```

    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-chunks: length chunks = n
and chunk-shape:
  ∧j. j < n ⇒
    verifier-query-round-chunk (query-idxs ! j)
      (map snd f-fl) (map snd fl) (chunks ! j)
and lookup:
  ∧j. j < n ⇒
    fmlookup (HashMap attacker-t)
      (QueryIndexChallenge (PQueryCounter attacker-s + j)
        (state-after-query-chunks (PState attacker-s) chunks j)) =
      Some (raw-idxs ! j)
by blast
have chunk-shape-raw:
  ∧j. j < n ⇒
    verifier-query-round-chunk (index (to-nat (raw-idxs ! j)))
      (map snd f-fl) (map snd fl) (chunks ! j)
proof –
  fix j
  assume j-bound: j < n
  have query-idxs ! j = index (to-nat (raw-idxs ! j))
    using query-idxs-def len-raw j-bound by simp
  then show
    verifier-query-round-chunk (index (to-nat (raw-idxs ! j)))
      (map snd f-fl) (map snd fl) (chunks ! j)
    using chunk-shape[OF j-bound] by simp
qed
have lookup-verifier:
  ∧j. j < n ⇒
    fmlookup (HashMap verifier-s)
      (QueryIndexChallenge (PQueryCounter verifier-s + j)
        (state-after-query-chunks (PState verifier-s) chunks j)) =
      Some (raw-idxs ! j)
proof –
  fix j
  assume j-bound: j < n
  have lookup-attacker:
    fmlookup (HashMap attacker-t)
      (QueryIndexChallenge (PQueryCounter attacker-s + j)
        (state-after-query-chunks (PState attacker-s) chunks j)) =
      Some (raw-idxs ! j)
    by (rule lookup[OF j-bound])
  have lookup-verifier-s:
    fmlookup (HashMap verifier-s)
      (QueryIndexChallenge (PQueryCounter attacker-s + j)
        (state-after-query-chunks (PState attacker-s) chunks j)) =
      Some (raw-idxs ! j)
    by (rule hash-extension-lookup[OF lookup-attacker hash-ext])
show

```

```

    fmlookup (HashMap verifier-s)
      (QueryIndexChallenge (PQueryCounter verifier-s + j)
        (state-after-query-chunks (PState verifier-s) chunks j)) =
    Some (raw-idxs ! j)
  using lookup-verifier-s state-eq counter-eq by simp
qed
show ?thesis
by (rule ntimes-verifier-query-round-program-replays-shaped-chunks
  [OF verifier len-raw len-chunks chunk-shape-raw lookup-verifier
    transcript-eq])
qed

lemma checked-staged-transcript-program-outcome-shape:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, t) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  shows
    length (staged-trace-fri-roots data) = ceil-log clength ∧
    length (staged-trace-fri-challenges data) = ceil-log clength ∧
    length (staged-alphas data) = length spec ∧
    length (staged-composition-fri-roots data) =
      ceil-log (to-nat (staged-degree data) + 1) ∧
    length (staged-composition-fri-challenges data) =
      ceil-log (to-nat (staged-degree data) + 1) ∧
    ceil-log (to-nat (staged-degree data) + 1) ≤
      ceil-log (maxDegree + 1) ∧
    length (staged-query-chunks data) = rounds
proof -
  from outcome obtain fr s1 s2 trace-roots trace-bs s3 trace-final s4 s5
    as s6 dg s7 s8 s9 composition-roots composition-bs s10
    composition-final s11 s12 query-chunks where
  trace-out:
    Some ((trace-roots, trace-bs), s3) ∈
      set-dist
        (execute
          (staged-trace-fri-program A 0 (ceil-log clength) []) s2)
  and alpha-out:
    Some (as, s6) ∈
      set-dist (execute (staged-alpha-program (length spec)) s5)
  and degree-eq: staged-degree data = dg
  and assert-out:
    Some ((), s9) ∈
      set-dist
        (execute
          (assert

```

```

      (ceil-log (to-nat dg + 1) ≤
        ceil-log (maxDegree + 1))) s8)
and composition-out:
  Some ((composition-roots, composition-bs), s10) ∈
    set-dist
      (execute
        (staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) [] s9)
and query-out:
  Some (query-chunks, s12) ∈
    set-dist
      (execute
        (checked-staged-query-program A trace-roots composition-roots
          0 rounds)
          s11)
and data-eq:
  data =
    (|staged-trace-root = fr,
     staged-trace-fri-roots = trace-roots,
     staged-trace-fri-challenges = trace-bs,
     staged-trace-final = trace-final,
     staged-alphas = as,
     staged-degree = dg,
     staged-composition-fri-roots = composition-roots,
     staged-composition-fri-challenges = composition-bs,
     staged-composition-final = composition-final,
     staged-query-chunks = query-chunks|)
unfolding checked-staged-transcript-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have trace-len:
  length trace-roots = ceil-log clength ∧
  length trace-bs = ceil-log clength
proof –
have len:
  0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
using wf unfolding staged-budget-wellformed-def by simp
have alignment:
  length trace-roots = ceil-log clength ∧
  length trace-bs = length (| :: 'f list) + ceil-log clength ∧
  PTranscript s3 = PTranscript s2 @ trace-roots ∧
  PState s3 = foldl concat (PState s2) trace-roots ∧
  s2 ≤ s3 ∧
  PTraceFriCounter s3 = PTraceFriCounter s2 + ceil-log clength ∧
  PCompositionFriCounter s3 = PCompositionFriCounter s2 ∧
  PAlphaCounter s3 = PAlphaCounter s2 ∧
  PQueryCounter s3 = PQueryCounter s2 ∧
  (∀ j < ceil-log clength.
    fmllookup (HashMap s3)
      (TraceFriChallenge (PTraceFriCounter s2 + j))

```

```

      (foldl concat (PState s2) (take (Suc j) trace-roots))) =
      Some (trace-bs ! (length ([] :: 'f list) + j)))
    by (rule staged-trace-fri-program-alignment
        [OF controlled len trace-out])
  then show ?thesis by simp
qed
have alpha-len: length as = length spec
  using staged-alpha-program-alignment[OF alpha-out] by simp
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
  using assert-out unfolding assert-def
  by (cases
      ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
      (auto simp: throw-no-outcome)
have composition-len:
  length composition-roots = ceil-log (to-nat dg + 1) ∧
  length composition-bs = ceil-log (to-nat dg + 1)
proof -
  have len:
    0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
    using wf round-bound unfolding staged-budget-wellformed-def by simp
  have alignment:
    length composition-roots = ceil-log (to-nat dg + 1) ∧
    length composition-bs =
      length ([] :: 'f list) + ceil-log (to-nat dg + 1) ∧
    PTranscript s10 = PTranscript s9 @ composition-roots ∧
    PState s10 = foldl concat (PState s9) composition-roots ∧
    s9 ≤ s10 ∧
    PTraceFriCounter s10 = PTraceFriCounter s9 ∧
    PCompositionFriCounter s10 =
      PCompositionFriCounter s9 + ceil-log (to-nat dg + 1) ∧
    PAlphaCounter s10 = PAlphaCounter s9 ∧
    PQueryCounter s10 = PQueryCounter s9 ∧
    (∀ j < ceil-log (to-nat dg + 1).
      fmlookup (HashMap s10)
        (CompositionFriChallenge (PCompositionFriCounter s9 + j)
          (foldl concat (PState s9) (take (Suc j) composition-roots))) =
          Some (composition-bs ! (length ([] :: 'f list) + j)))
    by (rule staged-composition-fri-program-alignment
        [OF controlled len composition-out])
  then show ?thesis by simp
qed
have query-len: length query-chunks = rounds
  using checked-staged-query-program-chunks-match-verifier[OF query-out]
  by blast
show ?thesis
  unfolding data-eq degree-eq
  using trace-len alpha-len composition-len round-bound query-len by simp

```

qed

lemma *checked-staged-transcript-program-outcome-header-transcript:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *outcome:*

Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)

shows

verifier-header-transcript
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
(List.concat (staged-query-chunks data))

proof —

have *shape:*

length (staged-trace-fri-roots data) = ceil-log clength ∧
length (staged-trace-fri-challenges data) = ceil-log clength ∧
length (staged-alphas data) = length spec ∧
length (staged-composition-fri-roots data) =
ceil-log (to-nat (staged-degree data) + 1) ∧
length (staged-composition-fri-challenges data) =
ceil-log (to-nat (staged-degree data) + 1) ∧
ceil-log (to-nat (staged-degree data) + 1) ≤
ceil-log (maxDegree + 1) ∧
length (staged-query-chunks data) = rounds

by *(rule checked-staged-transcript-program-outcome-shape*
[OF wf controlled outcome])

show *?thesis*

by *(rule staged-proof-transcript-verifier-header-transcript)*
(use shape in simp-all)

qed

lemma *staged-transcript-program-outcome-shape:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *outcome:*

Some (data, t) ∈
set-dist
(execute (staged-transcript-program A)
adversary-initial-state)

shows

$\text{length } (\text{staged-trace-fri-roots } \text{data}) = \text{ceil-log } \text{clength} \wedge$
 $\text{length } (\text{staged-trace-fri-challenges } \text{data}) = \text{ceil-log } \text{clength} \wedge$
 $\text{length } (\text{staged-alphas } \text{data}) = \text{length } \text{spec} \wedge$
 $\text{length } (\text{staged-composition-fri-roots } \text{data}) =$
 $\quad \text{ceil-log } (\text{to-nat } (\text{staged-degree } \text{data}) + 1) \wedge$
 $\text{length } (\text{staged-composition-fri-challenges } \text{data}) =$
 $\quad \text{ceil-log } (\text{to-nat } (\text{staged-degree } \text{data}) + 1) \wedge$
 $\text{ceil-log } (\text{to-nat } (\text{staged-degree } \text{data}) + 1) \leq$
 $\quad \text{ceil-log } (\text{maxDegree} + 1) \wedge$
 $\text{length } (\text{staged-query-chunks } \text{data}) = \text{rounds}$

proof –

from *outcome* **obtain** *fr s1 s2 trace-roots trace-bs s3 trace-final s4 s5*
as s6 dg s7 s8 s9 composition-roots composition-bs s10
composition-final s11 s12 query-chunks **where**

trace-out:

Some $((\text{trace-roots}, \text{trace-bs}), s3) \in$
set-dist
 $(\text{execute}$
 $\quad (\text{staged-trace-fri-program } A \ 0 \ (\text{ceil-log } \text{clength}) \ \square) \ s2)$

and *alpha-out:*

Some $(\text{as}, s6) \in$
set-dist $(\text{execute } (\text{staged-alpha-program } (\text{length } \text{spec})) \ s5)$

and *degree-eq:* $\text{staged-degree } \text{data} = \text{dg}$

and *assert-out:*

Some $((), s9) \in$
set-dist
 $(\text{execute}$
 $\quad (\text{assert}$
 $\quad \quad (\text{ceil-log } (\text{to-nat } \text{dg} + 1) \leq$
 $\quad \quad \quad \text{ceil-log } (\text{maxDegree} + 1))) \ s8)$

and *composition-out:*

Some $((\text{composition-roots}, \text{composition-bs}), s10) \in$
set-dist
 $(\text{execute}$
 $\quad (\text{staged-composition-fri-program } A \ \text{dg} \ 0$
 $\quad \quad (\text{ceil-log } (\text{to-nat } \text{dg} + 1)) \ \square) \ s9)$

and *query-out:*

Some $(\text{query-chunks}, s12) \in$
set-dist $(\text{execute } (\text{staged-query-program } A \ 0 \ \text{rounds}) \ s11)$

and *data-eq:*

$\text{data} =$
 $(\text{staged-trace-root} = \text{fr},$
 $\text{staged-trace-fri-roots} = \text{trace-roots},$
 $\text{staged-trace-fri-challenges} = \text{trace-bs},$
 $\text{staged-trace-final} = \text{trace-final},$
 $\text{staged-alphas} = \text{as},$
 $\text{staged-degree} = \text{dg},$
 $\text{staged-composition-fri-roots} = \text{composition-roots},$


```

    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks)
unfolding staged-transcript-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have trace-len:
  length trace-roots = ceil-log clength ∧
  length trace-bs = ceil-log clength
proof –
  have len:
    0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have alignment:
    length trace-roots = ceil-log clength ∧
    length trace-bs = length ([] :: 'f list) + ceil-log clength ∧
    PTranscript s3 = PTranscript s2 @ trace-roots ∧
    PState s3 = foldl concat (PState s2) trace-roots ∧
    s2 ≤ s3 ∧
    PTraceFriCounter s3 = PTraceFriCounter s2 + ceil-log clength ∧
    PCompositionFriCounter s3 = PCompositionFriCounter s2 ∧
    PAlphaCounter s3 = PAlphaCounter s2 ∧
    PQueryCounter s3 = PQueryCounter s2 ∧
    (∀ j < ceil-log clength.
      fmlookup (HashMap s3)
        (TraceFriChallenge (PTraceFriCounter s2 + j)
          (foldl concat (PState s2) (take (Suc j) trace-roots)))) =
      Some (trace-bs ! (length ([] :: 'f list) + j)))
    by (rule staged-trace-fri-program-alignment
      [OF controlled len trace-out])
  then show ?thesis by simp
qed
have alpha-len: length as = length spec
  using staged-alpha-program-alignment[OF alpha-out] by simp
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
  using assert-out unfolding assert-def
  by (cases
    ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have composition-len:
  length composition-roots = ceil-log (to-nat dg + 1) ∧
  length composition-bs = ceil-log (to-nat dg + 1)
proof –
  have len:
    0 + ceil-log (to-nat dg + 1) ≤
      length (composition-fri-budgets budgets)
    using wf round-bound unfolding staged-budget-wellformed-def by simp
  have alignment:
    length composition-roots = ceil-log (to-nat dg + 1) ∧

```

```

length composition-bs =
  length ([] :: 'f list) + ceil-log (to-nat dg + 1) ∧
  PTranscript s10 = PTranscript s9 @ composition-roots ∧
  PState s10 = foldl concat (PState s9) composition-roots ∧
  s9 ≤ s10 ∧
  PTraceFriCounter s10 = PTraceFriCounter s9 ∧
  PCompositionFriCounter s10 =
    PCompositionFriCounter s9 + ceil-log (to-nat dg + 1) ∧
  PAlphaCounter s10 = PAlphaCounter s9 ∧
  PQueryCounter s10 = PQueryCounter s9 ∧
  (∀ j < ceil-log (to-nat dg + 1).
    fmlookup (HashMap s10)
      (CompositionFriChallenge (PCompositionFriCounter s9 + j)
        (foldl concat (PState s9) (take (Suc j) composition-roots))) =
      Some (composition-bs ! (length ([] :: 'f list) + j)))
  by (rule staged-composition-fri-program-alignment
    [OF controlled len composition-out])
  then show ?thesis by simp
qed
have query-len:
  length query-chunks = rounds
proof -
  have len: 0 + rounds ≤ length (query-opening-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have alignment:
    length query-chunks = rounds ∧
    PTranscript s12 = PTranscript s11 @ List.concat query-chunks ∧
    PState s12 =
      foldl concat (PState s11) (List.concat query-chunks) ∧
    PTraceFriCounter s12 = PTraceFriCounter s11 ∧
    PCompositionFriCounter s12 = PCompositionFriCounter s11 ∧
    PAlphaCounter s12 = PAlphaCounter s11 ∧
    PQueryCounter s12 = PQueryCounter s11 + rounds
    by (rule staged-query-program-alignment[OF controlled len query-out])
  then show ?thesis by simp
qed
show ?thesis
  unfolding data-eq degree-eq
  using trace-len alpha-len composition-len round-bound query-len by simp
qed

lemma hash-target-program-outcome-extension:
  assumes target: hash-target-program B q m
    and outcome: Some (x, t) ∈ set-dist (execute m s)
  shows s ≤ t
  using hash-target-program-extension[OF target] outcome
  unfolding hash-extension-preserving-def by blast

lemma staged-transcript-program-outcome-query-lookups:

```

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *outcome*:
Some (*data*, *attacker-state*) $\in$
set-dist
(execute (staged-transcript-program A)
adversary-initial-state)
shows
 $\exists$ *raw-idxs query-idxs*.
length raw-idxs = rounds $\wedge$
query-idxs = map (λ *raw. index (to-nat raw)*) *raw-idxs* $\wedge$
length (staged-query-chunks data) = rounds $\wedge$
PState attacker-state =
state-after-query-chunks
(staged-query-start-hash data)
(staged-query-chunks data) rounds $\wedge$
PQueryCounter attacker-state = rounds $\wedge$
 $(\forall j < \text{rounds}.$
fmlookup (HashMap attacker-state)
(QueryIndexChallenge j
(state-after-query-chunks
(staged-query-start-hash data)
(staged-query-chunks data) j)) =
Some (raw-idxs ! j)) $\wedge$
 $(\forall \text{idx} \in \text{set query-idxs. idx} < \text{clength} * \text{scale})$
proof –
from *outcome* **obtain** *fr s2 trace-roots trace-bs s3 trace-final s4 s5*
as s6 dg s7 s8 s9 composition-roots composition-bs s10
composition-final s11 s12 query-chunks **where**
root-out:
Some (fr, s1) $\in$
set-dist (execute (trace-root-stage A) adversary-initial-state)
and *record-root*:
Some ((), s2) $\in$ set-dist (execute (record-staged-message fr) s1)
and *trace-out*:
Some ((trace-roots, trace-bs), s3) $\in$
set-dist
(execute
(staged-trace-fri-program A 0 (ceil-log clength) []) s2)
and *final-out*:
Some (trace-final, s4) $\in$
set-dist (execute (trace-final-stage A trace-bs) s3)
and *record-final*:
Some ((), s5) $\in$
set-dist (execute (record-staged-message trace-final) s4)
and *alpha-out*:
Some (as, s6) $\in$
set-dist (execute (staged-alpha-program (length spec)) s5)
and *degree-out*:

```

    Some (dg, s7) ∈ set-dist (execute (degree-stage A as) s6)
and record-degree:
    Some ((), s8) ∈ set-dist (execute (record-staged-message dg) s7)
and assert-out:
    Some ((), s9) ∈
      set-dist
        (execute
          (assert
            (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1))) s8)
and composition-out:
    Some ((composition-roots, composition-bs), s10) ∈
      set-dist
        (execute
          (staged-composition-fri-program A dg 0
            (ceil-log (to-nat dg + 1)) []) s9)
and composition-final-out:
    Some (composition-final, s11) ∈
      set-dist
        (execute (composition-final-stage A dg composition-bs) s10)
and record-composition-final:
    Some ((), s12) ∈
      set-dist (execute (record-staged-message composition-final) s11)
and query-out:
    Some (query-chunks, attacker-state) ∈
      set-dist (execute (staged-query-program A 0 rounds) s12)
and data-eq:
    data =
      (|staged-trace-root = fr,
        staged-trace-fri-roots = trace-roots,
        staged-trace-fri-challenges = trace-bs,
        staged-trace-final = trace-final,
        staged-alphas = as,
        staged-degree = dg,
        staged-composition-fri-roots = composition-roots,
        staged-composition-fri-challenges = composition-bs,
        staged-composition-final = composition-final,
        staged-query-chunks = query-chunks|)
unfolding staged-transcript-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have root-controlled:
  controlled-ro-program (trace-root-budget budgets)
    (trace-root-stage A)
using controlled unfolding staged-adversary-controlled-def by simp
have trace-final-controlled:
  controlled-ro-program (trace-final-budget budgets)
    (trace-final-stage A trace-bs)
using controlled unfolding staged-adversary-controlled-def by blast
have degree-controlled:

```

```

controlled-ro-program (degree-budget budgets) (degree-stage A as)
using controlled unfolding staged-adversary-controlled-def by blast
have composition-final-controlled:
  controlled-ro-program (composition-final-budget budgets)
    (composition-final-stage A dg composition-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have root-state: PState s1 = 0
  using controlled-stage-outcome-fields[OF root-controlled root-out]
  by simp
have s2-eq:
  s2 = s1(
    PState := concat (PState s1) fr,
    PTranscript := PTranscript s1 @ [fr])
  by (rule record-staged-message-outcome[OF record-root])
have trace-bound:
   $0 + \text{ceil-log } \text{clength} \leq \text{length } (\text{trace-fri-budgets } \text{budgets})$ 
  using wf unfolding staged-budget-wellformed-def by simp
have trace-alignment:
  PState s3 = foldl concat (PState s2) trace-roots
  using staged-trace-fri-program-alignment
    [OF controlled trace-bound trace-out]
  by simp
have state-s3:
  PState s3 = foldl concat (concat 0 fr) trace-roots
  using root-state trace-alignment unfolding s2-eq by simp
have state-s4: PState s4 = PState s3
  using controlled-stage-outcome-fields
    [OF trace-final-controlled final-out]
  by simp
have s5-eq:
  s5 = s4(
    PState := concat (PState s4) trace-final,
    PTranscript := PTranscript s4 @ [trace-final])
  by (rule record-staged-message-outcome[OF record-final])
have alpha-alignment:
  PState s6 = foldl concat (PState s5) as
  using staged-alpha-program-alignment[OF alpha-out] by simp
have state-s6:
  PState s6 =
    foldl concat
      (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
    as
  using state-s3 state-s4 alpha-alignment unfolding s5-eq by simp
have state-s7: PState s7 = PState s6
  using controlled-stage-outcome-fields[OF degree-controlled degree-out]
  by simp
have s8-eq:
  s8 = s7(
    PState := concat (PState s7) dg,

```

```

    PTranscript := PTranscript s7 @ [dg])
  by (rule record-staged-message-outcome[OF record-degree])
have s9-eq: s9 = s8
  using assert-out unfolding assert-def
  by (cases
      ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
      (auto simp: throw-no-outcome)
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
  using assert-out unfolding assert-def
  by (cases
      ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
      (auto simp: throw-no-outcome)
have composition-bound:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
  using wf round-bound unfolding staged-budget-wellformed-def by simp
have composition-alignment:
  PState s10 = foldl concat (PState s9) composition-roots
  using staged-composition-fri-program-alignment
    [OF controlled composition-bound composition-out]
  by simp
have state-s10:
  PState s10 =
    foldl concat
      (concat
        (foldl concat
          (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
          as)
        dg)
      composition-roots
  using state-s6 state-s7 composition-alignment
  unfolding s8-eq s9-eq by simp
have state-s11: PState s11 = PState s10
  using controlled-stage-outcome-fields
    [OF composition-final-controlled composition-final-out]
  by simp
have s12-eq:
  s12 = s11 (
    PState := concat (PState s11) composition-final,
    PTranscript := PTranscript s11 @ [composition-final])
  by (rule record-staged-message-outcome[OF record-composition-final])
have state-s12:
  PState s12 =
    concat
      (foldl concat
        (concat
          (foldl concat
            (concat (foldl concat (concat 0 fr) trace-roots) trace-final)

```

```

      as)
    dg)
  composition-roots)
  composition-final
  using state-s10 state-s11 unfolding s12-eq by simp
have query-counter-s12: PQueryCounter s12 = 0
proof -
  have root-fields:
    PQueryCounter s1 = PQueryCounter adversary-initial-state
    using controlled-stage-outcome-fields[OF root-controlled root-out]
    by simp
  have trace-counters:
    PQueryCounter s3 = PQueryCounter s2
    using staged-trace-fri-program-alignment
      [OF controlled trace-bound trace-out]
    by simp
  have final-counter: PQueryCounter s4 = PQueryCounter s3
    using controlled-stage-outcome-fields
      [OF trace-final-controlled final-out]
    by simp
  have alpha-counter: PQueryCounter s6 = PQueryCounter s5
    using staged-alpha-program-alignment[OF alpha-out] by simp
  have degree-counter: PQueryCounter s7 = PQueryCounter s6
    using controlled-stage-outcome-fields[OF degree-controlled degree-out]
    by simp
  have comp-counter: PQueryCounter s10 = PQueryCounter s9
    using staged-composition-fri-program-alignment
      [OF controlled composition-bound composition-out]
    by simp
  have comp-final-counter: PQueryCounter s11 = PQueryCounter s10
    using controlled-stage-outcome-fields
      [OF composition-final-controlled composition-final-out]
    by simp
  show ?thesis
    using root-fields trace-counters final-counter alpha-counter
      degree-counter comp-counter comp-final-counter
    unfolding s2-eq s5-eq s8-eq s9-eq s12-eq by simp
qed
have query-bound: 0 + rounds ≤ length (query-opening-budgets budgets)
  using wf unfolding staged-budget-wellformed-def by simp
from staged-query-program-outcome-with-rows
  [OF controlled query-bound query-out]
obtain raw-idxs query-idxs where
  len-raw: length raw-idxs = rounds
  and query-idxs-def:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-chunks: length query-chunks = rounds
  and state-attacker:
    PState attacker-state =

```

```

    state-after-query-chunks (PState s12) query-chunks rounds
  and query-counter-attacker:
    PQueryCounter attacker-state = PQueryCounter s12 + rounds
  and lookup:
     $\bigwedge j. j < rounds \implies$ 
      fmlookup (HashMap attacker-state)
        (QueryIndexChallenge (PQueryCounter s12 + j)
          (state-after-query-chunks (PState s12) query-chunks j)) =
        Some (raw-idxs ! j)
  and idx-bound:
     $\forall idx \in \text{set query-idxs}. idx < \text{clength} * \text{scale}$ 
  by blast
  have start-eq:
    PState s12 = staged-query-start-hash data
  unfolding data-eq staged-query-start-hash-def
    staged-composition-fri-start-hash-def staged-trace-fri-start-hash-def
  using state-s12 by simp
  have chunks-eq: staged-query-chunks data = query-chunks
  unfolding data-eq by simp
  have state-attacker':
    PState attacker-state =
      state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) rounds
  using state-attacker start-eq chunks-eq by simp
  have query-counter-attacker':
    PQueryCounter attacker-state = rounds
  using query-counter-attacker query-counter-s12 by simp
  have lookup':
     $\bigwedge j. j < rounds \implies$ 
      fmlookup (HashMap attacker-state)
        (QueryIndexChallenge j
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) j)) =
        Some (raw-idxs ! j)
  using lookup start-eq query-counter-s12 chunks-eq by simp
  show ?thesis
  by (intro exI[of - raw-idxs] exI[of - query-idxs] conjI)
    (use len-raw query-idxs-def chunks-eq len-chunks state-attacker'
      query-counter-attacker' lookup' idx-bound
      in simp-all)
qed

```

lemma *checked-staged-transcript-program-outcome-query-lookups:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *outcome:*
Some (data, attacker-state) $\in$


```

    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  shows
    ∃ raw-idxs query-idxs.
      length raw-idxs = rounds ∧
      query-idxs = map (λ raw. index (to-nat raw)) raw-idxs ∧
      length (staged-query-chunks data) = rounds ∧
      PState attacker-state =
        state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) rounds ∧
      PQueryCounter attacker-state = rounds ∧
      (∀ j < rounds.
        fmllookup (HashMap attacker-state)
          (QueryIndexChallenge j
            (state-after-query-chunks
              (staged-query-start-hash data)
              (staged-query-chunks data) j)) =
          Some (raw-idxs ! j)) ∧
      (∀ idx ∈ set query-idxs. idx < clength * scale)
  proof -
    have unchecked-out:
      Some (data, attacker-state) ∈
        set-dist
          (execute (staged-transcript-program A) adversary-initial-state)
    by (rule checked-staged-transcript-program-outcome-imp-staged-transcript-program
      [OF outcome])
    show ?thesis
      by (rule staged-transcript-program-outcome-query-lookups
        [OF wf controlled unchecked-out])
  qed

lemma checked-staged-transcript-program-query-phaseE:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  obtains fr trace-roots trace-bs trace-final as dg composition-roots
    composition-bs composition-final query-start query-chunks
  where
    Some (query-chunks, attacker-state) ∈
      set-dist
        (execute
          (checked-staged-query-program A trace-roots composition-roots
            0 rounds)

```

```

    query-start)
data =
  (|staged-trace-root = fr,
   staged-trace-fri-roots = trace-roots,
   staged-trace-fri-challenges = trace-bs,
   staged-trace-final = trace-final,
   staged-alphas = as,
   staged-degree = dg,
   staged-composition-fri-roots = composition-roots,
   staged-composition-fri-challenges = composition-bs,
   staged-composition-final = composition-final,
   staged-query-chunks = query-chunks|)
PState query-start = staged-query-start-hash data
PQueryCounter query-start = 0
proof -
from outcome obtain fr s1 s2 trace-roots trace-bs s3 trace-final s4 s5
  as s6 dg s7 s8 s9 composition-roots composition-bs s10
  composition-final s11 query-start query-chunks where
root-out:
  Some (fr, s1) ∈
    set-dist (execute (trace-root-stage A) adversary-initial-state)
and record-root:
  Some ((), s2) ∈ set-dist (execute (record-staged-message fr) s1)
and trace-out:
  Some ((trace-roots, trace-bs), s3) ∈
    set-dist
      (execute
        (staged-trace-fri-program A 0 (ceil-log clength) []) s2)
and final-out:
  Some (trace-final, s4) ∈
    set-dist (execute (trace-final-stage A trace-bs) s3)
and record-final:
  Some ((), s5) ∈
    set-dist (execute (record-staged-message trace-final) s4)
and alpha-out:
  Some (as, s6) ∈
    set-dist (execute (staged-alpha-program (length spec)) s5)
and degree-out:
  Some (dg, s7) ∈ set-dist (execute (degree-stage A as) s6)
and record-degree:
  Some ((), s8) ∈ set-dist (execute (record-staged-message dg) s7)
and assert-out:
  Some ((), s9) ∈
    set-dist
      (execute
        (assert
          (ceil-log (to-nat dg + 1) ≤
            ceil-log (maxDegree + 1))) s8)
and composition-out:

```

```

Some ((composition-roots, composition-bs), s10) ∈
  set-dist
    (execute
      (staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [] s9)
    and composition-final-out:
      Some (composition-final, s11) ∈
        set-dist
          (execute (composition-final-stage A dg composition-bs) s10)
    and record-composition-final:
      Some ((), query-start) ∈
        set-dist (execute (record-staged-message composition-final) s11)
    and query-out:
      Some (query-chunks, attacker-state) ∈
        set-dist
          (execute
            (checked-staged-query-program A trace-roots composition-roots
              0 rounds)
            query-start)
    and data-eq:
      data =
        (|staged-trace-root = fr,
          staged-trace-fri-roots = trace-roots,
          staged-trace-fri-challenges = trace-bs,
          staged-trace-final = trace-final,
          staged-alphas = as,
          staged-degree = dg,
          staged-composition-fri-roots = composition-roots,
          staged-composition-fri-challenges = composition-bs,
          staged-composition-final = composition-final,
          staged-query-chunks = query-chunks|)
    unfolding checked-staged-transcript-program-def Let-def
    by (auto elim!: set-dist-bindE split: prod.splits)
have root-controlled:
  controlled-ro-program (trace-root-budget budgets)
    (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def by simp
have trace-final-controlled:
  controlled-ro-program (trace-final-budget budgets)
    (trace-final-stage A trace-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have degree-controlled:
  controlled-ro-program (degree-budget budgets) (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def by blast
have composition-final-controlled:
  controlled-ro-program (composition-final-budget budgets)
    (composition-final-stage A dg composition-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have root-state: PState s1 = 0

```

```

using controlled-stage-outcome-fields[OF root-controlled root-out]
by simp
have s2-eq:
  s2 = s1(
    PState := concat (PState s1) fr,
    PTranscript := PTranscript s1 @ [fr])
  by (rule record-staged-message-outcome[OF record-root])
have trace-bound:
  0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
  using wf unfolding staged-budget-wellformed-def by simp
have trace-alignment:
  PState s3 = foldl concat (PState s2) trace-roots
  using staged-trace-fri-program-alignment
    [OF controlled trace-bound trace-out]
  by simp
have state-s3:
  PState s3 = foldl concat (concat 0 fr) trace-roots
  using root-state trace-alignment unfolding s2-eq by simp
have state-s4: PState s4 = PState s3
  using controlled-stage-outcome-fields
    [OF trace-final-controlled final-out]
  by simp
have s5-eq:
  s5 = s4(
    PState := concat (PState s4) trace-final,
    PTranscript := PTranscript s4 @ [trace-final])
  by (rule record-staged-message-outcome[OF record-final])
have alpha-alignment:
  PState s6 = foldl concat (PState s5) as
  using staged-alpha-program-alignment[OF alpha-out] by simp
have state-s6:
  PState s6 =
    foldl concat
      (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
    as
  using state-s3 state-s4 alpha-alignment unfolding s5-eq by simp
have state-s7: PState s7 = PState s6
  using controlled-stage-outcome-fields[OF degree-controlled degree-out]
  by simp
have s8-eq:
  s8 = s7(
    PState := concat (PState s7) dg,
    PTranscript := PTranscript s7 @ [dg])
  by (rule record-staged-message-outcome[OF record-degree])
have s9-eq: s9 = s8
  using assert-out unfolding assert-def
  by (cases
    ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)

```

```

have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
using assert-out unfolding assert-def
by (cases
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
  (auto simp: throw-no-outcome)
have composition-bound:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
using wf round-bound unfolding staged-budget-wellformed-def by simp
have composition-alignment:
  PState s10 = foldl concat (PState s9) composition-roots
using staged-composition-fri-program-alignment
  [OF controlled composition-bound composition-out]
by simp
have state-s10:
  PState s10 =
    foldl concat
      (concat
        (foldl concat
          (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
          as)
        dg)
      composition-roots
using state-s6 state-s7 composition-alignment
unfolding s8-eq s9-eq by simp
have state-s11: PState s11 = PState s10
using controlled-stage-outcome-fields
  [OF composition-final-controlled composition-final-out]
by simp
have query-start-eq:
  query-start = s11(|
    PState := concat (PState s11) composition-final,
    PTranscript := PTranscript s11 @ [composition-final]|)
by (rule record-staged-message-outcome[OF record-composition-final])
have state-query-start:
  PState query-start =
    concat
      (foldl concat
        (concat
          (foldl concat
            (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
            as)
          dg)
        composition-roots)
      composition-final
using state-s10 state-s11 unfolding query-start-eq by simp
have query-counter-start: PQueryCounter query-start = 0
proof –

```

```

have root-fields:
  PQueryCounter s1 = PQueryCounter adversary-initial-state
  using controlled-stage-outcome-fields[OF root-controlled root-out]
  by simp
have trace-counters:
  PQueryCounter s3 = PQueryCounter s2
  using staged-trace-fri-program-alignment
    [OF controlled trace-bound trace-out]
  by simp
have final-counter: PQueryCounter s4 = PQueryCounter s3
  using controlled-stage-outcome-fields
    [OF trace-final-controlled final-out]
  by simp
have alpha-counter: PQueryCounter s6 = PQueryCounter s5
  using staged-alpha-program-alignment[OF alpha-out] by simp
have degree-counter: PQueryCounter s7 = PQueryCounter s6
  using controlled-stage-outcome-fields[OF degree-controlled degree-out]
  by simp
have comp-counter: PQueryCounter s10 = PQueryCounter s9
  using staged-composition-fri-program-alignment
    [OF controlled composition-bound composition-out]
  by simp
have comp-final-counter: PQueryCounter s11 = PQueryCounter s10
  using controlled-stage-outcome-fields
    [OF composition-final-controlled composition-final-out]
  by simp
show ?thesis
  using root-fields trace-counters final-counter alpha-counter
    degree-counter comp-counter comp-final-counter
  unfolding s2-eq s5-eq s8-eq s9-eq query-start-eq by simp
qed
have start-eq:
  PState query-start = staged-query-start-hash data
  unfolding data-eq staged-query-start-hash-def
    staged-composition-fri-start-hash-def staged-trace-fri-start-hash-def
  using state-query-start by simp
show ?thesis
  by (rule that[OF query-out data-eq start-eq query-counter-start])
qed

lemma checked-staged-security-with-data-state-accepted-shape-query-keys:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist (execute
      (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
and shape:

```

```

    accepted-transcript-shape
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state)) as query-idxs
  shows
    as = staged-alphas data ∧
    (∃ raw-idxs.
      length raw-idxs = rounds ∧
      query-idxs = map (λraw. index (to-nat raw)) raw-idxs ∧
      (∀ i < rounds.
        fmllookup (HashMap final-state)
          (QueryIndexChallenge i
            (state-after-query-chunks
              (staged-query-start-hash data)
              (staged-query-chunks data) i)) =
            Some (raw-idxs ! i)) ∧
        (∀ idx ∈ set query-idxs. idx < clength * scale)))
  proof -
    let ?s = verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF outcome]
  have builder:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈
      set-dist (execute verify-monad ?s)
  by blast+
  have staged-header:
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
  from accepted-transcript-shape-query-chunksE[OF shape]
  obtain result' final-state' fr f-fri-roots f-final dg
    composition-fri-roots final rest raw-idxs query-chunks trailing
  where out-eq:
    Some (result, final-state) = Some (result', final-state')
  and header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg

```

```

    composition-fri-roots final rest
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-chunks: length query-chunks = rounds
and concat-chunks: List.concat query-chunks @ trailing = rest
and chunk-shape:
    ∧ i. i < rounds ⇒
        verifier-query-round-chunk (query-idxs ! i)
        f-fri-roots composition-fri-roots (query-chunks ! i)
and lookup:
    ∧ i. i < rounds ⇒
        fmllookup (HashMap final-state')
        (QueryIndexChallenge (PQueryCounter ?s + i)
        (state-after-query-chunks
        (verifier-header-state ?s fr f-fri-roots f-final as dg
        composition-fri-roots final)
        query-chunks i)) =
        Some (raw-idxs ! i)
and idx-bound:
    ∀ idx ∈ set query-idxs. idx < clength * scale
by blast
have header-eq:
    fr = staged-trace-root data ∧
    f-fri-roots = staged-trace-fri-roots data ∧
    f-final = staged-trace-final data ∧
    as = staged-alphas data ∧
    dg = staged-degree data ∧
    composition-fri-roots = staged-composition-fri-roots data ∧
    final = staged-composition-final data ∧
    rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF staged-header header]
by simp
have final-state-eq: final-state' = final-state
using out-eq by simp
have header-state-eq:
    verifier-header-state ?s fr f-fri-roots f-final as dg
    composition-fri-roots final =
    staged-query-start-hash data
using header-eq
unfolding verifier-header-state-def verifier-header-messages-def
    staged-query-start-hash-def staged-composition-fri-start-hash-def
    staged-trace-fri-start-hash-def
by simp
from checked-staged-transcript-program-query-chunks-match-verifier-lengths
    [OF builder]
obtain staged-query-idxs where
    staged-match:
        staged-query-chunks-match-verifier-lengths data staged-query-idxs

```



```

    by blast
  have query-idxs-len: length query-idxs = rounds
    using query-idxs-eq len-raw by simp
  have staged-match-query-idxs:
    staged-query-chunks-match-verifier-lengths data query-idxs
  by (rule staged-query-chunks-match-verifier-lengths-transfer
    [OF staged-match query-idxs-len])
  have parser-chunk-len:
     $\wedge i. i < rounds \implies$ 
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)
  proof -
    fix i
    assume i-bound: i < rounds
    show
      length (query-chunks ! i) =
        verifier-query-round-transcript-length (query-idxs ! i)
          (staged-trace-fri-roots data)
          (staged-composition-fri-roots data)
    using verifier-query-round-chunk-length[OF chunk-shape[OF i-bound]]
      header-eq by simp
  qed
  have concat-staged:
    List.concat query-chunks @ trailing =
      List.concat (staged-query-chunks data)
  using concat-chunks header-eq by simp
  have query-chunks-eq:
    query-chunks = staged-query-chunks data  $\wedge$  trailing = []
  by (rule query-chunks-eq-staged-if-matching-lengths
    [OF len-chunks concat-staged parser-chunk-len
      staged-match-query-idxs])
  have lookup-staged:
     $\wedge i. i < rounds \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
  proof -
    fix i
    assume i-bound: i < rounds
    have lookup-i:
      fmlookup (HashMap final-state')
        (QueryIndexChallenge (PQueryCounter ?s + i)
          (state-after-query-chunks
            (verifier-header-state ?s fr f-fri-roots f-final as dg

```

```

      composition-fri-roots final)
    query-chunks i)) =
  Some (raw-idxs ! i)
  by (rule lookup[OF i-bound])
show
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  using lookup-i final-state-eq header-state-eq query-chunks-eq by simp
qed
show ?thesis
  using header-eq len-raw query-idxs-eq lookup-staged idx-bound by blast
qed

lemma checked-staged-security-with-data-state-accepted-shape-query-set-hit:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist (execute
        (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  and shape:
    accepted-transcript-shape
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state)) as query-idxs
  and hit: ∃ i < rounds. query-idxs ! i ∈ B
  shows staged-transcript-query-index-set-hit B (Some (data, attacker-state))
proof -
  let ?s = verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF outcome]
  have builder:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈
      set-dist (execute verify-monad ?s)
  by blast+
  from checked-staged-transcript-program-outcome-query-lookups
    [OF wf controlled builder]
  obtain attacker-raw-idxs where
    lookup-attacker:

```

```

   $\forall j < \text{rounds}.$ 
     $\text{fmlookup } (\text{HashMap } \text{attacker-state})$ 
       $(\text{QueryIndexChallenge } j$ 
         $(\text{state-after-query-chunks } (\text{staged-query-start-hash } \text{data})$ 
           $(\text{staged-query-chunks } \text{data}) \ j)) =$ 
         $\text{Some } (\text{attacker-raw-idxs } ! \ j)$ 
      by blast
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled outcome shape]
obtain raw-idxs where
  len-raw:  $\text{length } \text{raw-idxs} = \text{rounds}$ 
and query-idxs-eq:
   $\text{query-idxs} = \text{map } (\lambda \text{raw. index } (\text{to-nat } \text{raw})) \ \text{raw-idxs}$ 
and lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
     $\text{fmlookup } (\text{HashMap } \text{final-state})$ 
       $(\text{QueryIndexChallenge } i$ 
         $(\text{state-after-query-chunks } (\text{staged-query-start-hash } \text{data})$ 
           $(\text{staged-query-chunks } \text{data}) \ i)) =$ 
         $\text{Some } (\text{raw-idxs } ! \ i)$ 
    by blast
from hit obtain i where i-bound:  $i < \text{rounds}$ 
and query-hit:  $\text{query-idxs } ! \ i \in B$ 
by blast
have raw-i:  $i < \text{length } \text{raw-idxs}$ 
using len-raw i-bound by simp
have raw-hit:  $\text{index } (\text{to-nat } (\text{raw-idxs } ! \ i)) \in B$ 
using query-hit query-idxs-eq raw-i by (simp add: nth-map)
have ext:  $?s \leq \text{final-state}$ 
by (rule verify-monad-hash-extends[OF verifier])
have lookup-initial:
   $\text{fmlookup } (\text{HashMap } ?s)$ 
     $(\text{QueryIndexChallenge } i$ 
       $(\text{state-after-query-chunks } (\text{staged-query-start-hash } \text{data})$ 
         $(\text{staged-query-chunks } \text{data}) \ i)) =$ 
       $\text{Some } (\text{attacker-raw-idxs } ! \ i)$ 
    using lookup-attacker i-bound by simp
have lookup-final-from-initial:
   $\text{fmlookup } (\text{HashMap } \text{final-state})$ 
     $(\text{QueryIndexChallenge } i$ 
       $(\text{state-after-query-chunks } (\text{staged-query-start-hash } \text{data})$ 
         $(\text{staged-query-chunks } \text{data}) \ i)) =$ 
       $\text{Some } (\text{attacker-raw-idxs } ! \ i)$ 
    by (rule hash-extension-lookup[OF lookup-initial ext])
have raw-eq:  $\text{attacker-raw-idxs } ! \ i = \text{raw-idxs } ! \ i$ 
using lookup-final-from-initial lookup[OF i-bound] by simp
have attacker-raw-hit:  $\text{index } (\text{to-nat } (\text{attacker-raw-idxs } ! \ i)) \in B$ 
using raw-hit raw-eq by simp
have lookup-attacker-i:

```

```

fmlookup (HashMap attacker-state)
  (QueryIndexChallenge i
    (state-after-query-chunks (staged-query-start-hash data)
      (staged-query-chunks data) i)) =
  Some (attacker-raw-idxs ! i)
using lookup-attacker i-bound by simp
show ?thesis
  unfolding staged-transcript-query-index-set-hit-def
  using i-bound lookup-attacker-i attacker-raw-hit by auto
qed

lemma staged-security-with-data-query-index-set-bound-from-transcript:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and transcript-bound:
    wp-event (staged-transcript-program A)
      (staged-transcript-query-index-set-hit B)
    adversary-initial-state  $\leq$  C
  shows
    wp-event (staged-security-experiment-with-data A)
      (staged-security-with-data-query-index-set-hit B)
    adversary-initial-state  $\leq$  C
  unfolding staged-security-experiment-with-data-def
proof (rule wp-event-bind-bound-by-head-event[OF transcript-bound])
  show staged-security-with-data-query-index-set-hit B None  $\implies$ 
    staged-transcript-query-index-set-hit B None
  unfolding staged-security-with-data-query-index-set-hit-def
    staged-transcript-query-index-set-hit-def
  by simp
next
fix data attacker-state out
assume builder:
  Some (data, attacker-state)  $\in$ 
    set-dist
      (execute (staged-transcript-program A)
        adversary-initial-state)
and cont:
  out  $\in$ 
    set-dist
      (execute
        (get  $\gg=$ 
          ( $\lambda s$ . put
            (verifier-state-from-adversary s
              (staged-proof-transcript data))  $\gg=$ 
            ( $\lambda$ -. verify-monad  $\gg=$ 
              ( $\lambda$ result. return (data, result))))))
        attacker-state)
  and hit: staged-security-with-data-query-index-set-hit B out
from staged-transcript-program-outcome-query-lookups

```

```

    [OF wf controlled builder]
obtain raw-idxs query-idxs where
  len-raw: length raw-idxs = rounds
and lookup-attacker:
  ∀ j < rounds.
    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge j
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) j)) =
        Some (raw-idxs ! j)
by blast
from hit obtain data' result final-state i raw where
  out-eq: out = Some ((data', result), final-state)
and i-bound: i < rounds
and lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data')
        (staged-query-chunks data') i)) =
      Some raw
and raw-hit: index (to-nat raw) ∈ B
unfolding staged-security-with-data-query-index-set-hit-def
by (auto split: option.splits prod.splits)
from cont[unfolded out-eq] obtain verifier-state verifier-result where
  verifier:
    Some (verifier-result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
and data'-eq: data' = data
by (auto elim!: set-dist-bindE)
have ext:
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data) ≤ final-state
by (rule verify-monad-hash-extends[OF verifier])
have lookup-initial:
  fmlookup
    (HashMap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
using lookup-attacker i-bound by simp

```

```

have lookup-final-from-initial:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  by (rule hash-extension-lookup[OF lookup-initial ext])
have raw-eq: raw = raw-idxs ! i
  using lookup-final lookup-final-from-initial data'-eq by simp
have lookup-attacker-i:
  fmlookup (HashMap attacker-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  using lookup-attacker i-bound by simp
have raw-idxs-hit: index (to-nat (raw-idxs ! i)) ∈ B
  using raw-hit raw-eq by simp
show staged-transcript-query-index-set-hit B
  (Some (data, attacker-state))
  unfolding staged-transcript-query-index-set-hit-def
  using i-bound lookup-attacker-i raw-idxs-hit by auto
qed

lemma checked-staged-security-with-data-query-index-set-bound-from-transcript:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and transcript-bound:
    wp-event (checked-staged-transcript-program A)
      (staged-transcript-query-index-set-hit B)
      adversary-initial-state ≤ C
  shows
    wp-event (checked-staged-security-experiment-with-data A)
      (staged-security-with-data-query-index-set-hit B)
      adversary-initial-state ≤ C
  unfolding checked-staged-security-experiment-with-data-def
proof (rule wp-event-bind-bound-by-head-event[OF transcript-bound])
  show staged-security-with-data-query-index-set-hit B None ⇒
    staged-transcript-query-index-set-hit B None
  unfolding staged-security-with-data-query-index-set-hit-def
    staged-transcript-query-index-set-hit-def
  by simp
next
fix data attacker-state out
assume builder:
  Some (data, attacker-state) ∈
    set-dist

```

```

    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
and cont:
  out ∈
    set-dist
      (execute
        (get >>=
          (λs. put
            (verifier-state-from-adversary s
              (staged-proof-transcript data)) >>=
            (λ-. verify-monad >>=
              (λresult. return (data, result))))))
          attacker-state)
    and hit: staged-security-with-data-query-index-set-hit B out
from checked-staged-transcript-program-outcome-query-lookups
  [OF wf controlled builder]
obtain raw-idxs query-idxs where
  len-raw: length raw-idxs = rounds
and lookup-attacker:
  ∀ j < rounds.
    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge j
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) j)) =
      Some (raw-idxs ! j)
by blast
from hit obtain data' result final-state i raw where
  out-eq: out = Some ((data', result), final-state)
and i-bound: i < rounds
and lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data')
        (staged-query-chunks data') i)) =
    Some raw
and raw-hit: index (to-nat raw) ∈ B
unfolding staged-security-with-data-query-index-set-hit-def
by (auto split: option.splits prod.splits)
from cont[unfolded out-eq] obtain verifier-state verifier-result where
  verifier:
    Some (verifier-result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
and data'-eq: data' = data
by (auto elim!: set-dist-bindE)

```

```

have ext:
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data) ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
have lookup-initial:
  fmlookup
    (HashMap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  using lookup-attacker i-bound by simp
have lookup-final-from-initial:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  by (rule hash-extension-lookup[OF lookup-initial ext])
have raw-eq: raw = raw-idxs ! i
  using lookup-final lookup-final-from-initial data'-eq by simp
have lookup-attacker-i:
  fmlookup (HashMap attacker-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  using lookup-attacker i-bound by simp
have raw-idxs-hit: index (to-nat (raw-idxs ! i)) ∈ B
  using raw-hit raw-eq by simp
show staged-transcript-query-index-set-hit B
  (Some (data, attacker-state))
  unfolding staged-transcript-query-index-set-hit-def
  using i-bound lookup-attacker-i raw-idxs-hit by auto
qed

lemma staged-transcript-query-index-set-hit-imp-target-hit:
  assumes hit: staged-transcript-query-index-set-hit B out
  shows
    hash-new-output-hit-event (query-index-raw-preimage B)
    adversary-initial-state out
proof (cases out)
  case None
  then show ?thesis

```



```

    using hit unfolding staged-transcript-query-index-set-hit-def by simp
next
case (Some result)
then obtain data t where out-eq: out = Some (data, t)
  by (cases result) simp
from hit[unfolded out-eq staged-transcript-query-index-set-hit-def]
obtain i raw where lookup:
  fmlookup (HashMap t)
  (QueryIndexChallenge i
   (state-after-query-chunks
    (staged-query-start-hash data)
    (staged-query-chunks data) i)) =
    Some raw
and raw-in: raw ∈ query-index-raw-preimage B
unfolding query-index-raw-preimage-def by auto
have new-hit:
  hash-map-new-output-hit (query-index-raw-preimage B)
  adversary-initial-state t
  unfolding hash-map-new-output-hit-def
  using lookup raw-in by auto
show ?thesis
  unfolding out-eq hash-new-output-hit-event-def
  using new-hit by simp
qed

```

```

lemma staged-transcript-query-index-set-hit-bound-from-target-program:
  assumes target:
    hash-target-program (query-index-raw-preimage B) q m
  shows
    wp-event m (staged-transcript-query-index-set-hit B)
    adversary-initial-state ≤
    staged-phase-target-error (query-index-raw-preimage B) q
proof -
  have event-mono:
    wp-event m (staged-transcript-query-index-set-hit B)
    adversary-initial-state ≤
    wp-event m
      (hash-new-output-hit-event (query-index-raw-preimage B)
       adversary-initial-state)
    adversary-initial-state
  by (rule wp-event-mono)
  (rule staged-transcript-query-index-set-hit-imp-target-hit)
  have target-bound:
    wp-event m
      (hash-new-output-hit-event (query-index-raw-preimage B)
       adversary-initial-state)
    adversary-initial-state ≤
    staged-phase-target-error (query-index-raw-preimage B) q
  using target unfolding hash-target-program-def hash-target-budget-def

```

staged-phase-target-error-def **by** blast
show ?thesis
by (rule order-trans[OF event-mono target-bound])
qed

lemma staged-transcript-query-index-set-hit-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
 wp-event (staged-transcript-program A)
 (staged-transcript-query-index-set-hit B)
 adversary-initial-state $\leq$
 staged-phase-target-error (query-index-raw-preimage B)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
by (rule staged-transcript-query-index-set-hit-bound-from-target-program)
 (rule hash-target-program-staged-transcript-program[OF wf controlled])

lemma checked-staged-transcript-query-index-set-hit-bound-from-target-program:
assumes target:
 hash-target-program (query-index-raw-preimage B) q
 (checked-staged-transcript-program A)
shows
 wp-event (checked-staged-transcript-program A)
 (staged-transcript-query-index-set-hit B)
 adversary-initial-state $\leq$
 staged-phase-target-error (query-index-raw-preimage B) q
by (rule staged-transcript-query-index-set-hit-bound-from-target-program
 [OF target])

lemma checked-staged-transcript-query-index-set-hit-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
 wp-event (checked-staged-transcript-program A)
 (staged-transcript-query-index-set-hit B)
 adversary-initial-state $\leq$
 staged-phase-target-error (query-index-raw-preimage B)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
by (rule checked-staged-transcript-query-index-set-hit-bound-from-target-program)
 (rule hash-target-program-checked-staged-transcript-program
 [OF wf controlled])

lemma staged-security-with-data-query-index-set-hit-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
 wp-event (staged-security-experiment-with-data A)

(staged-security-with-data-query-index-set-hit B)
 adversary-initial-state $\leq$
 staged-phase-target-error (query-index-raw-preimage B)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
by (rule staged-security-with-data-query-index-set-bound-from-transcript
 [OF wf controlled
 staged-transcript-query-index-set-hit-bound[OF wf controlled]])

lemma *checked-staged-security-with-data-query-index-set-hit-bound-from-target-program:*

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and target:
 hash-target-program (query-index-raw-preimage B) q
 (checked-staged-transcript-program A)

shows

wp-event (checked-staged-security-experiment-with-data A)
 (staged-security-with-data-query-index-set-hit B)
 adversary-initial-state $\leq$
 staged-phase-target-error (query-index-raw-preimage B) q
by (rule checked-staged-security-with-data-query-index-set-bound-from-transcript
 [OF wf controlled
 checked-staged-transcript-query-index-set-hit-bound-from-target-program
 [OF target]])

lemma *checked-staged-security-with-data-query-index-set-hit-bound:*

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows

wp-event (checked-staged-security-experiment-with-data A)
 (staged-security-with-data-query-index-set-hit B)
 adversary-initial-state $\leq$
 staged-phase-target-error (query-index-raw-preimage B)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
by (rule checked-staged-security-with-data-query-index-set-hit-bound-from-target-program
 [OF wf controlled
 hash-target-program-checked-staged-transcript-program
 [OF wf controlled]])

end

end

theory *Staged-Security-Experiment-Checked*

imports *Staged-Security-Experiment-Checked-Prefix*

begin

Final checked staged query-bad and collision bounds.

context *soundness*
begin

lemma *query-bad-imp-query-rounds-sampling-success-hit:*

assumes *bad: query-bad s out*

shows

$\exists$ *trace-table composition-table as.*
query-rounds-any-index-set-hit s
(query-sampling-success-space trace-table composition-table as)
rounds out

proof –

have *round-hit:*

query-index-round-set-hit s query-sampling-success-space out
by (rule *query-bad-imp-query-index-round-set-hit*[*OF bad*])

then obtain *i* **where** *hit-at:*

query-index-round-set-hit-at s query-sampling-success-space i out
unfolding *query-index-round-set-hit-def* **by** *blast*

show *?thesis*

by (rule *query-index-round-set-hit-at-imp-query-rounds-any-index-set-hit*
[*OF hit-at*])

qed

lemma *query-bad-imp-query-header-sampling-success-hit:*

assumes *outcome: out $\in$ set-dist (execute verify-monad s)*

and *bad: query-bad s out*

shows

query-header-rounds-any-index-set-hit s
(query-header-supported-union-good-sets s query-sampling-success-space)
out

proof –

have *round-hit:*

query-index-round-set-hit s query-sampling-success-space out
by (rule *query-bad-imp-query-index-round-set-hit*[*OF bad*])

show *?thesis*

by (rule *query-index-round-set-hit-imp-query-header-supported-rounds-hit*
[*OF outcome round-hit*])

qed

lemma *staged-security-with-data-query-bad-imp-sampling-hit:*

assumes *bad: staged-security-with-data-query-bad-hit out*

shows *staged-security-with-data-query-sampling-hit out*

proof (*cases out*)

case *None*

then show *?thesis*

using *bad* **unfolding** *staged-security-with-data-query-bad-hit-def* **by** *simp*

next

case (*Some result*)

then obtain *data verifier-result final-state* **where** *out-eq:*

out = Some ((data, verifier-result), final-state)

```

    by (cases result, auto split: prod.splits)
  from bad[unfolded out-eq staged-security-with-data-query-bad-hit-def]
  obtain s where query-bad:
    query-bad s (Some (verifier-result, final-state))
  by auto
  from query-bad-imp-query-rounds-sampling-success-hit[OF query-bad]
  obtain trace-table composition-table as where hit:
    query-rounds-any-index-set-hit s
    (query-sampling-success-space trace-table composition-table as)
    rounds (Some (verifier-result, final-state))
  by blast
  have witness:
    ∃ s trace-table composition-table as.
      query-rounds-any-index-set-hit s
      (query-sampling-success-space trace-table composition-table as)
      rounds (Some (verifier-result, final-state))
  by (intro exI[of - s] exI[of - trace-table]
      exI[of - composition-table] exI[of - as])
    (rule hit)
  show ?thesis
  unfolding out-eq staged-security-with-data-query-sampling-hit-def
  using witness by simp
qed

lemma staged-security-with-data-query-bad-imp-header-sampling-hit:
  assumes bad: staged-security-with-data-query-bad-hit out
  shows staged-security-with-data-query-header-sampling-hit out
proof (cases out)
  case None
  then show ?thesis
  using bad unfolding staged-security-with-data-query-bad-hit-def by simp
next
  case (Some result)
  then obtain data verifier-result final-state where out-eq:
    out = Some ((data, verifier-result), final-state)
  by (cases result, auto split: prod.splits)
  from bad[unfolded out-eq staged-security-with-data-query-bad-hit-def]
  obtain s where outcome:
    Some (verifier-result, final-state) ∈
    set-dist (execute verify-monad s)
  and query-bad:
    query-bad s (Some (verifier-result, final-state))
  by auto
  have hit:
    query-header-rounds-any-index-set-hit s
    (query-header-supported-union-good-sets s query-sampling-success-space)
    (Some (verifier-result, final-state))
  by (rule query-bad-imp-query-header-sampling-success-hit
      [OF outcome query-bad])

```

```

then show ?thesis
  unfolding out-eq staged-security-with-data-query-header-sampling-hit-def
  by auto
qed

lemma staged-security-with-data-state-query-bad-imp-header-sampling-hit:
  assumes bad: staged-security-with-data-state-query-bad-hit out
  shows staged-security-with-data-state-query-header-sampling-hit out
proof (cases out)
  case None
  then show ?thesis
    using bad
    unfolding staged-security-with-data-state-query-bad-hit-def by simp
  next
  case (Some result)
  then obtain data attacker-state verifier-result final-state where out-eq:
    out = Some (((data, attacker-state), verifier-result), final-state)
    by (cases result, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from bad[unfolded out-eq staged-security-with-data-state-query-bad-hit-def]
  have outcome:
    Some (verifier-result, final-state) ∈
      set-dist (execute verify-monad ?s)
  and query-bad:
    query-bad ?s (Some (verifier-result, final-state))
  by (simp-all add: Let-def)
  have hit:
    query-header-rounds-any-index-set-hit ?s
    (query-header-supported-union-good-sets ?s query-sampling-success-space)
    (Some (verifier-result, final-state))
  by (rule query-bad-imp-query-header-sampling-success-hit
    [OF outcome query-bad])
  then show ?thesis
    unfolding out-eq
    staged-security-with-data-state-query-header-sampling-hit-def
    by simp
qed

lemma checked-staged-security-with-data-state-query-bad-imp-header-key-hit-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    out ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)

```

```

    and bad: staged-security-with-data-state-query-bad-hit out
  shows staged-security-with-data-state-query-header-key-hit out
proof (cases out)
case None
then show ?thesis
  using bad unfolding staged-security-with-data-state-query-bad-hit-def
  by simp
next
case (Some result-pack)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have support-some:
  Some (((data, attacker-state), result), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
  using support unfolding out-eq .
from checked-staged-security-experiment-with-data-state-outcomeE
[OF support-some]
have builder:
  Some (data, attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-program A)
    adversary-initial-state)
and verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast+
have staged-header:
  verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
from bad[unfolded out-eq
  staged-security-with-data-state-query-bad-hit-def Let-def]
have query-bad: query-bad ?s (Some (result, final-state))
  by simp
from query-bad-imp-query-index-round-set-hit[OF query-bad]

```

```

obtain i trace-table composition-table as query-idxs where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
and i-bound: i < rounds
and query-hit:
  query-idxs ! i ∈
    query-sampling-success-space trace-table composition-table as
unfolding query-index-round-set-hit-def
  query-index-round-set-hit-at-def
by blast
have shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  as query-idxs
using bound
unfolding accepted-with-bound-tables-def accepted-with-tables-def
by simp
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support-some shape]
obtain raw-idxs where
  as-eq: as = staged-alphas data
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup:
   $\bigwedge j. j < rounds \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge j
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) j)) =
      Some (raw-idxs ! j)
and idx-bound:
   $\forall idx \in \text{set query-idxs}. idx < \text{clength} * \text{scale}$ 
by blast
have idx-sample:
  query-idxs ! i ∈ query-sample-space
proof –
  have len-query: length query-idxs = rounds
    using query-idxs-eq len-raw by simp
  have query-idxs ! i ∈ set query-idxs
    by (rule nth-mem) (use i-bound len-query in simp)
  then show ?thesis
    by (rule accepted-with-bound-tables-query-sample-space[OF bound])
qed
have candidate:
  (trace-table, composition-table) ∈
    query-header-supported-table-candidates ?s
    (staged-trace-root data)

```



```

    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  proof —
    have bound-staged:
      accepted-with-bound-tables ?s (Some (result, final-state))
      trace-table composition-table (staged-alphas data) query-idxs
    using bound as-eq by simp
    have witness:
      ∃ out query-idxs rest.
        out ∈ set-dist (execute verify-monad ?s) ∧
        accepted-with-bound-tables ?s out trace-table composition-table
          (staged-alphas data) query-idxs ∧
        verifier-header-transcript ?s
          (staged-trace-root data)
          (staged-trace-fri-roots data)
          (staged-trace-final data)
          (staged-alphas data)
          (staged-degree data)
          (staged-composition-fri-roots data)
          (staged-composition-final data) rest
    by (intro exI[where x=Some (result, final-state)]
        exI[where x=query-idxs]
        exI[where x=List.concat (staged-query-chunks data)] conjI)
      (use verifier bound-staged staged-header in simp-all)
    show ?thesis
      using witness
      unfolding query-header-supported-table-candidates-def by simp
  qed
  have raw-hit:
    index (to-nat (raw-idxs ! i)) ∈
      query-header-supported-union-good-sets ?s
        query-sampling-success-space
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)
        (staged-composition-fri-roots data)
        (staged-composition-final data)
  proof —
    have raw-idx-eq: index (to-nat (raw-idxs ! i)) = query-idxs ! i
      using query-idxs-eq len-raw i-bound by simp
    show ?thesis
      unfolding query-header-supported-union-good-sets-def raw-idx-eq
      using idx-sample candidate query-hit as-eq by blast

```

```

qed
have lookup-i:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data i))) =
    Some (raw-idxs ! i)
by (rule lookup[OF i-bound])
show ?thesis
  unfolding out-eq staged-security-with-data-state-query-header-key-hit-def
    Let-def
  using i-bound lookup-i raw-hit by auto
qed

lemma staged-security-with-data-state-query-bad-bound-from-header-sampling-hit:
assumes header-bound:
  wp-event (staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-header-sampling-hit
  adversary-initial-state ≤ C
shows
  wp-event (staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-bad-hit
  adversary-initial-state ≤ C
proof –
have wp-event (staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-bad-hit
  adversary-initial-state ≤
  wp-event (staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-header-sampling-hit
  adversary-initial-state
by (rule wp-event-mono)
  (rule staged-security-with-data-state-query-bad-imp-header-sampling-hit)
also have ... ≤ C
by (rule header-bound)
finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-query-bad-bound-from-header-key-hit:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and header-key-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-header-key-hit
  adversary-initial-state ≤ C
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-bad-hit
  adversary-initial-state ≤ C

```

proof –
have *wp-event (checked-staged-security-experiment-with-data-state A)*
 staged-security-with-data-state-query-bad-hit
 adversary-initial-state $\leq$
wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-query-header-key-hit
 adversary-initial-state
proof (*rule wp-event-mono-on-support*)
fix *out*
assume *support*:
 out $\in$
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-data-state A*)
 adversary-initial-state)
and *bad*: *staged-security-with-data-state-query-bad-hit out*
show *staged-security-with-data-state-query-header-key-hit out*
by (*rule*
 checked-staged-security-with-data-state-query-bad-imp-header-key-hit-on-support
 [*OF wf controlled support bad*])
qed
also have $\dots \leq C$
by (*rule header-key-bound*)
finally show *?thesis* .
qed

lemma
checked-staged-security-with-data-state-header-key-hit-bound-from-index-set:
assumes *cover*:
 $\bigwedge \text{data attacker-state.}$
 query-header-supported-union-good-sets
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 query-sampling-success-space
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)
 (*staged-composition-final data*) $\subseteq B$
and *index-bound*:
 wp-event (checked-staged-security-experiment-with-data A)
 (*staged-security-with-data-query-index-set-hit B*)
 adversary-initial-state $\leq C$
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-query-header-key-hit
 adversary-initial-state $\leq C$

```

proof –
  let ?projected =
     $\lambda out. \text{case out of None} \Rightarrow$ 
      staged-security-with-data-query-index-set-hit  $B$   $\text{None}$ 
    |  $\text{Some } (((data, -), result), t) \Rightarrow$ 
      staged-security-with-data-query-index-set-hit  $B$ 
      ( $\text{Some } ((data, result), t)$ )
  have event-le:
    wp-event (checked-staged-security-experiment-with-data-state  $A$ )
      staged-security-with-data-state-query-header-key-hit
      adversary-initial-state  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state  $A$ )
      ?projected adversary-initial-state
proof (rule wp-event-mono)
  fix out
  assume hit:
    staged-security-with-data-state-query-header-key-hit out
  show ?projected out
proof (cases out)
  case  $\text{None}$ 
  then show ?thesis
    using hit
    unfolding staged-security-with-data-state-query-header-key-hit-def
    by simp
next
  case ( $\text{Some packed}$ )
  then obtain data attacker-state result final-state where out-eq:
    out =  $\text{Some } (((data, attacker-state), result), final-state)$ 
    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  let ?B =
    query-header-supported-union-good-sets ?s
    query-sampling-success-space
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  from hit[unfolded out-eq
    staged-security-with-data-state-query-header-key-hit-def Let-def]
  obtain  $i$  raw where i-bound:  $i < \text{rounds}$ 
  and lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge  $i$ 
        (state-after-query-chunks

```

```

      (staged-query-start-hash data)
      (staged-query-chunks data) i)) =
    Some raw
  and raw-hit: index (to-nat raw) ∈ ?B
  by auto
  have raw-in-B: index (to-nat raw) ∈ B
  proof -
    have ?B ⊆ B
    by (rule cover)
    then show ?thesis
    using raw-hit by blast
  qed
  show ?thesis
    unfolding out-eq staged-security-with-data-query-index-set-hit-def
    using i-bound lookup raw-in-B by auto
  qed
  qed
  also have
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?projected adversary-initial-state =
    wp-event (checked-staged-security-experiment-with-data A)
      (staged-security-with-data-query-index-set-hit B)
      adversary-initial-state
    using checked-staged-security-experiment-with-data-event-from-data-state
      [of A staged-security-with-data-query-index-set-hit B]
    by simp
  also have ... ≤ C
  by (rule index-bound)
  finally show ?thesis .
  qed

```

lemma

checked-staged-security-with-data-state-header-key-hit-bound-from-index-set-controlled:

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and cover:

$\bigwedge$ data attacker-state.
 query-header-supported-union-good-sets
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 query-sampling-success-space
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) ⊆ B

shows

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-query-header-key-hit
adversary-initial-state $\leq$
staged-phase-target-error (query-index-raw-preimage B)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)
by (rule
checked-staged-security-with-data-state-header-key-hit-bound-from-index-set
[OF cover])
 (rule *checked-staged-security-with-data-query-index-set-hit-bound*
[OF wf controlled])

lemma

checked-staged-security-with-data-state-query-bad-bound-from-index-set-controlled:

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *cover:*

$\bigwedge$ *data attacker-state.*
query-header-supported-union-good-sets
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
query-sampling-success-space
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data) $\subseteq B$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-query-bad-hit
adversary-initial-state $\leq$
staged-phase-target-error (query-index-raw-preimage B)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)
by (rule *checked-staged-security-with-data-state-query-bad-bound-from-header-key-hit*
[OF wf controlled])
 (rule
checked-staged-security-with-data-state-header-key-hit-bound-from-index-set-controlled
[OF wf controlled cover])

lemma

checked-staged-security-with-data-state-query-bad-bound-from-fixed-query-error-cover:

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *cover:*

$\bigwedge$ *data attacker-state.*
query-header-supported-union-good-sets

```

    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    query-sampling-success-space
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)  $\subseteq B$ 
  and raw-bound: query-index-raw-preimage-bound
  and subset:  $B \subseteq \text{query-sample-space}$ 
  and frac:
    nnreal (card B) / nnreal (card query-sample-space)  $\leq$ 
    query-error-bound
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-bad-hit
    adversary-initial-state  $\leq$ 
    nnreal
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget) * query-error-bound
  proof -
  have target:
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-bad-hit
    adversary-initial-state  $\leq$ 
    staged-phase-target-error (query-index-raw-preimage B)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
  by (rule
    checked-staged-security-with-data-state-query-bad-bound-from-index-set-controlled
    [OF wf controlled cover])
  also have ...  $\leq$ 
    nnreal
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget) * query-error-bound
  by (rule staged-phase-query-index-target-error-query-bound
    [OF raw-bound subset frac])
  finally show ?thesis .
qed

lemma
  checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-index-set-controlled:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and cover:
     $\bigwedge \text{data attacker-state.}$ 
    query-header-supported-union-good-sets

```

*(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
query-sampling-success-space
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data) $\subseteq B$*

shows

*wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event query-bad)
adversary-initial-state $\leq$
staged-phase-target-error (query-index-raw-preimage B)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)*

by (rule

checked-staged-security-with-data-state-query-bad-verifier-event-bound)
(rule
*checked-staged-security-with-data-state-query-bad-bound-from-index-set-controlled
[OF wf controlled cover])*

lemma

checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-fixed-query-error-cover:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *cover:*
 $\bigwedge$ *data attacker-state.
query-header-supported-union-good-sets
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
query-sampling-success-space
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data) $\subseteq B$*
and *raw-bound: query-index-raw-preimage-bound*
and *subset: $B \subseteq \text{query-sample-space}$*
and *frac:*
 $\text{nnreal (card } B) / \text{nnreal (card query-sample-space)} \leq$
query-error-bound

shows

*wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event query-bad)
adversary-initial-state $\leq$*

$nnreal$
 $(staged-attacker-query-budget \text{ budgets} +$
 $staged-challenge-query-budget) * query-error-bound$
by (rule
 $checked-staged-security-with-data-state-query-bad-verifier-event-bound)$
(rule
 $checked-staged-security-with-data-state-query-bad-bound-from-fixed-query-error-cover$
 $[OF \text{ wf controlled cover raw-bound subset frac}]$)

lemma *staged-security-with-data-query-bad-bound-from-sampling-hit*:
assumes *sampling-bound*:
 $wp-event \text{ (staged-security-experiment-with-data } A)$
 $staged-security-with-data-query-sampling-hit$
 $adversary-initial-state \leq C$
shows
 $wp-event \text{ (staged-security-experiment-with-data } A)$
 $staged-security-with-data-query-bad-hit$
 $adversary-initial-state \leq C$
proof –
have $wp-event \text{ (staged-security-experiment-with-data } A)$
 $staged-security-with-data-query-bad-hit$
 $adversary-initial-state \leq$
 $wp-event \text{ (staged-security-experiment-with-data } A)$
 $staged-security-with-data-query-sampling-hit$
 $adversary-initial-state$
by (rule *wp-event-mono*)
(rule *staged-security-with-data-query-bad-imp-sampling-hit*)
also have $\dots \leq C$
by (rule *sampling-bound*)
finally show *?thesis* .
qed

lemma *staged-security-with-data-query-bad-bound-from-header-sampling-hit*:
assumes *header-bound*:
 $wp-event \text{ (staged-security-experiment-with-data } A)$
 $staged-security-with-data-query-header-sampling-hit$
 $adversary-initial-state \leq C$
shows
 $wp-event \text{ (staged-security-experiment-with-data } A)$
 $staged-security-with-data-query-bad-hit$
 $adversary-initial-state \leq C$
proof –
have $wp-event \text{ (staged-security-experiment-with-data } A)$
 $staged-security-with-data-query-bad-hit$
 $adversary-initial-state \leq$
 $wp-event \text{ (staged-security-experiment-with-data } A)$
 $staged-security-with-data-query-header-sampling-hit$
 $adversary-initial-state$
by (rule *wp-event-mono*)

(rule staged-security-with-data-query-bad-imp-header-sampling-hit)
 also have ... $\leq C$
 by (rule header-bound)
 finally show ?thesis .
 qed

lemma *staged-transcript-program-outcome-trace-challenge-lookup:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *outcome:*
Some (data, attacker-state) ∈
set-dist
(execute (staged-transcript-program A)
adversary-initial-state)
and *i-bound: i < ceil-log clength*

shows

fmlookup (HashMap attacker-state)
(TraceFriChallenge i
(foldl concat (staged-trace-fri-start-hash data)
(take (Suc i) (staged-trace-fri-roots data)))) =
Some (staged-trace-fri-challenges data ! i)

proof –

from *outcome* **obtain** *fr s1 s2 trace-roots trace-bs s3 trace-final s4 s5*
as s6 dg s7 s8 s9 composition-roots composition-bs s10
composition-final s11 s12 query-chunks **where**

root-out:

Some (fr, s1) ∈
set-dist (execute (trace-root-stage A) adversary-initial-state)

and *record-root:*

Some ((), s2) ∈ set-dist (execute (record-staged-message fr) s1)

and *trace-out:*

Some ((trace-roots, trace-bs), s3) ∈
set-dist
(execute
(staged-trace-fri-program A 0 (ceil-log clength) []) s2)

and *final-out:*

Some (trace-final, s4) ∈
set-dist (execute (trace-final-stage A trace-bs) s3)

and *record-final:*

Some ((), s5) ∈
set-dist (execute (record-staged-message trace-final) s4)

and *alpha-out:*

Some (as, s6) ∈
set-dist (execute (staged-alpha-program (length spec)) s5)

and *degree-out:*

Some (dg, s7) ∈ set-dist (execute (degree-stage A as) s6)

and *record-degree:*

Some ((), s8) ∈ set-dist (execute (record-staged-message dg) s7)

and *assert-out:*

```

Some (((), s9) ∈
  set-dist
    (execute
      (assert
        (ceil-log (to-nat dg + 1) ≤
          ceil-log (maxDegree + 1))) s8)
and composition-out:
Some ((composition-roots, composition-bs), s10) ∈
  set-dist
    (execute
      (staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []) s9)
and composition-final-out:
Some (composition-final, s11) ∈
  set-dist
    (execute (composition-final-stage A dg composition-bs) s10)
and record-composition-final:
Some (((), s12) ∈
  set-dist (execute (record-staged-message composition-final) s11)
and query-out:
Some (query-chunks, attacker-state) ∈
  set-dist (execute (staged-query-program A 0 rounds) s12)
and data-eq:
data =
  (|staged-trace-root = fr,
    staged-trace-fri-roots = trace-roots,
    staged-trace-fri-challenges = trace-bs,
    staged-trace-final = trace-final,
    staged-alphas = as,
    staged-degree = dg,
    staged-composition-fri-roots = composition-roots,
    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks|)
unfolding staged-transcript-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have root-controlled:
  controlled-ro-program (trace-root-budget budgets)
    (trace-root-stage A)
using controlled unfolding staged-adversary-controlled-def by simp
have trace-final-controlled:
  controlled-ro-program (trace-final-budget budgets)
    (trace-final-stage A trace-bs)
using controlled unfolding staged-adversary-controlled-def by blast
have degree-controlled:
  controlled-ro-program (degree-budget budgets) (degree-stage A as)
using controlled unfolding staged-adversary-controlled-def by blast
have composition-final-controlled:
  controlled-ro-program (composition-final-budget budgets)

```

```

    (composition-final-stage A dg composition-bs)
using controlled unfolding staged-adversary-controlled-def by blast
have root-fields:
  PState s1 = 0 ∧
  PTraceFriCounter s1 = 0
using controlled-stage-outcome-fields[OF root-controlled root-out]
by simp
have s2-eq:
  s2 = s1 (
    PState := concat (PState s1) fr,
    PTranscript := PTranscript s1 @ [fr])
by (rule record-staged-message-outcome[OF record-root])
have trace-bound:
  0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
using wf unfolding staged-budget-wellformed-def by simp
have trace-alignment:
  length trace-roots = ceil-log clength ∧
  length trace-bs = length ([] :: 'f list) + ceil-log clength ∧
  PTranscript s3 = PTranscript s2 @ trace-roots ∧
  PState s3 = foldl concat (PState s2) trace-roots ∧
  s2 ≤ s3 ∧
  PTraceFriCounter s3 = PTraceFriCounter s2 + ceil-log clength ∧
  PCompositionFriCounter s3 = PCompositionFriCounter s2 ∧
  PAlphaCounter s3 = PAlphaCounter s2 ∧
  PQueryCounter s3 = PQueryCounter s2 ∧
  (∀ j < ceil-log clength.
    fmllookup (HashMap s3)
      (TraceFriChallenge (PTraceFriCounter s2 + j)
        (foldl concat (PState s2) (take (Suc j) trace-roots))) =
        Some (trace-bs ! (length ([] :: 'f list) + j)))
by (rule staged-trace-fri-program-alignment
  [OF controlled trace-bound trace-out])
let ?key =
  TraceFriChallenge i
    (foldl concat (concat 0 fr) (take (Suc i) trace-roots))
have lookup-s3: fmllookup (HashMap s3) ?key = Some (trace-bs ! i)
using trace-alignment i-bound root-fields
unfolding s2-eq by simp
have ext-s3-s4: s3 ≤ s4
using controlled-ro-program-extension[OF trace-final-controlled]
  final-out
unfolding hash-extension-preserving-def by blast
have s5-eq:
  s5 = s4 (
    PState := concat (PState s4) trace-final,
    PTranscript := PTranscript s4 @ [trace-final])
by (rule record-staged-message-outcome[OF record-final])
have ext-s4-s5: s4 ≤ s5
unfolding s5-eq less-eq-hash-ext-def less-eq-fmap-def by simp

```

```

have ext-s5-s6:  $s5 \leq s6$ 
  by (rule hash-target-program-outcome-extension
      [OF hash-target-program-staged-alpha-program alpha-out])
have ext-s6-s7:  $s6 \leq s7$ 
  using controlled-ro-program-extension[OF degree-controlled] degree-out
  unfolding hash-extension-preserving-def by blast
have s8-eq:
  s8 = s7(
    PState := concat (PState s7) dg,
    PTranscript := PTranscript s7 @ [dg])
  by (rule record-staged-message-outcome[OF record-degree])
have ext-s7-s8:  $s7 \leq s8$ 
  unfolding s8-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have s9-eq:  $s9 = s8$ 
  using assert-out unfolding assert-def
  by (cases
      ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
      (auto simp: throw-no-outcome)
have ext-s8-s9:  $s8 \leq s9$ 
  unfolding s9-eq by (simp add: hash-ext-refl)
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
  using assert-out unfolding assert-def
  by (cases
      ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
      (auto simp: throw-no-outcome)
have composition-bound:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
  using wf round-bound unfolding staged-budget-wellformed-def by simp
have ext-s9-s10:  $s9 \leq s10$ 
  using staged-composition-fri-program-alignment
      [OF controlled composition-bound composition-out]
  by simp
have ext-s10-s11:  $s10 \leq s11$ 
  using controlled-ro-program-extension[OF composition-final-controlled]
      composition-final-out
  unfolding hash-extension-preserving-def by blast
have s12-eq:
  s12 = s11(
    PState := concat (PState s11) composition-final,
    PTranscript := PTranscript s11 @ [composition-final])
  by (rule record-staged-message-outcome[OF record-composition-final])
have ext-s11-s12:  $s11 \leq s12$ 
  unfolding s12-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have query-bound:  $0 + \text{rounds} \leq \text{length (query-opening-budgets budgets)}$ 
  using wf unfolding staged-budget-wellformed-def by simp
have ext-s12-attacker:  $s12 \leq \text{attacker-state}$ 
  by (rule hash-target-program-outcome-extension

```

```

    [OF hash-target-program-staged-query-program[OF controlled query-bound]
      query-out])
  have ext-s3-s5:  $s3 \leq s5$ 
    by (rule hash-ext-trans[OF ext-s3-s4 ext-s4-s5])
  have ext-s3-s6:  $s3 \leq s6$ 
    by (rule hash-ext-trans[OF ext-s3-s5 ext-s5-s6])
  have ext-s3-s7:  $s3 \leq s7$ 
    by (rule hash-ext-trans[OF ext-s3-s6 ext-s6-s7])
  have ext-s3-s8:  $s3 \leq s8$ 
    by (rule hash-ext-trans[OF ext-s3-s7 ext-s7-s8])
  have ext-s3-s9:  $s3 \leq s9$ 
    by (rule hash-ext-trans[OF ext-s3-s8 ext-s8-s9])
  have ext-s3-s10:  $s3 \leq s10$ 
    by (rule hash-ext-trans[OF ext-s3-s9 ext-s9-s10])
  have ext-s3-s11:  $s3 \leq s11$ 
    by (rule hash-ext-trans[OF ext-s3-s10 ext-s10-s11])
  have ext-s3-s12:  $s3 \leq s12$ 
    by (rule hash-ext-trans[OF ext-s3-s11 ext-s11-s12])
  have ext-s3-attacker:  $s3 \leq \text{attacker-state}$ 
    by (rule hash-ext-trans[OF ext-s3-s12 ext-s12-attacker])
  have lookup-attacker:
    fmlookup (HashMap attacker-state) ?key = Some (trace-bs ! i)
    by (rule hash-extension-lookup[OF lookup-s3 ext-s3-attacker])
  show ?thesis
    unfolding data-eq staged-trace-fri-start-hash-def
    using lookup-attacker by simp
qed

```

lemma staged-transcript-program-outcome-composition-challenge-lookup:

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
 Some (data, attacker-state) $\in$
 set-dist
 (execute (staged-transcript-program A)
 adversary-initial-state)
and i-bound:
 $i < \text{ceil-log } (\text{to-nat } (\text{staged-degree data}) + 1)$

shows

fmlookup (HashMap attacker-state)
 (CompositionFriChallenge i
 (foldl concat (staged-composition-fri-start-hash data)
 (take (Suc i) (staged-composition-fri-roots data)))) =
 Some (staged-composition-fri-challenges data ! i)

proof –

from outcome **obtain** fr s1 s2 trace-roots trace-bs s3 trace-final s4 s5
 as s6 dg s7 s8 s9 composition-roots composition-bs s10
 composition-final s11 s12 query-chunks **where**
 root-out:

Some (*fr*, *s1*) ∈
set-dist (*execute* (*trace-root-stage* *A*) *adversary-initial-state*)
and *record-root*:
Some (*()*, *s2*) ∈ *set-dist* (*execute* (*record-staged-message* *fr*) *s1*)
and *trace-out*:
Some (*(trace-roots, trace-bs)*, *s3*) ∈
set-dist
(*execute*
(*staged-trace-fri-program* *A* 0 (*ceil-log* *clength*) []) *s2*)
and *final-out*:
Some (*trace-final*, *s4*) ∈
set-dist (*execute* (*trace-final-stage* *A* *trace-bs*) *s3*)
and *record-final*:
Some (*()*, *s5*) ∈
set-dist (*execute* (*record-staged-message* *trace-final*) *s4*)
and *alpha-out*:
Some (*as*, *s6*) ∈
set-dist (*execute* (*staged-alpha-program* (*length spec*)) *s5*)
and *degree-out*:
Some (*dg*, *s7*) ∈ *set-dist* (*execute* (*degree-stage* *A* *as*) *s6*)
and *record-degree*:
Some (*()*, *s8*) ∈ *set-dist* (*execute* (*record-staged-message* *dg*) *s7*)
and *assert-out*:
Some (*()*, *s9*) ∈
set-dist
(*execute*
(*assert*
(*ceil-log* (*to-nat* *dg* + 1) ≤
ceil-log (*maxDegree* + 1))) *s8*)
and *composition-out*:
Some (*(composition-roots, composition-bs)*, *s10*) ∈
set-dist
(*execute*
(*staged-composition-fri-program* *A* *dg* 0
(*ceil-log* (*to-nat* *dg* + 1)) []) *s9*)
and *composition-final-out*:
Some (*composition-final*, *s11*) ∈
set-dist
(*execute* (*composition-final-stage* *A* *dg* *composition-bs*) *s10*)
and *record-composition-final*:
Some (*()*, *s12*) ∈
set-dist (*execute* (*record-staged-message* *composition-final*) *s11*)
and *query-out*:
Some (*query-chunks, attacker-state*) ∈
set-dist (*execute* (*staged-query-program* *A* 0 *rounds*) *s12*)
and *data-eq*:
data =
(*staged-trace-root* = *fr*,
staged-trace-fri-roots = *trace-roots*,

```

    staged-trace-fri-challenges = trace-bs,
    staged-trace-final = trace-final,
    staged-alphas = as,
    staged-degree = dg,
    staged-composition-fri-roots = composition-roots,
    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks)
unfolding staged-transcript-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have root-controlled:
  controlled-ro-program (trace-root-budget budgets)
    (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def by simp
have trace-final-controlled:
  controlled-ro-program (trace-final-budget budgets)
    (trace-final-stage A trace-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have degree-controlled:
  controlled-ro-program (degree-budget budgets) (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def by blast
have composition-final-controlled:
  controlled-ro-program (composition-final-budget budgets)
    (composition-final-stage A dg composition-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have root-fields:
  PState s1 = 0 ∧
  PCompositionFriCounter s1 = 0
  using controlled-stage-outcome-fields[OF root-controlled root-out]
  by simp
have s2-eq:
  s2 = s1 (
    PState := concat (PState s1) fr,
    PTranscript := PTranscript s1 @ [fr])
  by (rule record-staged-message-outcome[OF record-root])
have s2-fields:
  PState s2 = concat 0 fr ∧
  PCompositionFriCounter s2 = 0
  unfolding s2-eq using root-fields by simp
have trace-bound:
  0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
  using wf unfolding staged-budget-wellformed-def by simp
have trace-alignment:
  length trace-roots = ceil-log clength ∧
  length trace-bs = length ([] :: 'f list) + ceil-log clength ∧
  PTranscript s3 = PTranscript s2 @ trace-roots ∧
  PState s3 = foldl concat (PState s2) trace-roots ∧
  s2 ≤ s3 ∧
  PTraceFriCounter s3 = PTraceFriCounter s2 + ceil-log clength ∧

```



```

PCompositionFriCounter s3 = PCompositionFriCounter s2 ∧
PAlphaCounter s3 = PAlphaCounter s2 ∧
PQueryCounter s3 = PQueryCounter s2 ∧
(∀ j < ceil-log clength.
  fmlookup (HashMap s3)
    (TraceFriChallenge (PTraceFriCounter s2 + j)
      (foldl concat (PState s2) (take (Suc j) trace-roots))) =
      Some (trace-bs ! (length ([]) :: 'f list) + j)))
by (rule staged-trace-fri-program-alignment
  [OF controlled trace-bound trace-out])
have final-fields:
  PState s4 = PState s3 ∧
  PCompositionFriCounter s4 = PCompositionFriCounter s3
using controlled-stage-outcome-fields
  [OF trace-final-controlled final-out]
by simp
have s5-eq:
  s5 = s4(|
    PState := concat (PState s4) trace-final,
    PTranscript := PTranscript s4 @ [trace-final])
by (rule record-staged-message-outcome[OF record-final])
have s5-fields:
  PState s5 =
    concat (foldl concat (concat 0 fr) trace-roots) trace-final ∧
  PCompositionFriCounter s5 = 0
unfolding s5-eq
using s2-fields trace-alignment final-fields by simp
have alpha-alignment:
  length as = length spec ∧
  PTranscript s6 = PTranscript s5 @ as ∧
  PState s6 = foldl concat (PState s5) as ∧
  PTraceFriCounter s6 = PTraceFriCounter s5 ∧
  PCompositionFriCounter s6 = PCompositionFriCounter s5 ∧
  PAlphaCounter s6 = PAlphaCounter s5 + length spec ∧
  PQueryCounter s6 = PQueryCounter s5
by (rule staged-alpha-program-alignment[OF alpha-out])
have degree-fields:
  PState s7 = PState s6 ∧
  PCompositionFriCounter s7 = PCompositionFriCounter s6
using controlled-stage-outcome-fields[OF degree-controlled degree-out]
by simp
have s8-eq:
  s8 = s7(|
    PState := concat (PState s7) dg,
    PTranscript := PTranscript s7 @ [dg])
by (rule record-staged-message-outcome[OF record-degree])
have s9-eq: s9 = s8
using assert-out unfolding assert-def
by (cases

```

```

      ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have start-fields:
  PState s9 =
    concat
      (foldl concat
        (concat
          (foldl concat (concat 0 fr) trace-roots)
          trace-final)
        as)
      dg ∧
    PCompositionFriCounter s9 = 0
unfolding s9-eq s8-eq
using s5-fields alpha-alignment degree-fields
by simp
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
using assert-out unfolding assert-def
by (cases
      ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have composition-bound:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
using wf round-bound unfolding staged-budget-wellformed-def by simp
have composition-alignment:
  length composition-roots = ceil-log (to-nat dg + 1) ∧
  length composition-bs =
    length ([] :: 'f list) + ceil-log (to-nat dg + 1) ∧
  PTranscript s10 = PTranscript s9 @ composition-roots ∧
  PState s10 = foldl concat (PState s9) composition-roots ∧
  s9 ≤ s10 ∧
  PTraceFriCounter s10 = PTraceFriCounter s9 ∧
  PCompositionFriCounter s10 =
    PCompositionFriCounter s9 + ceil-log (to-nat dg + 1) ∧
  PAlphaCounter s10 = PAlphaCounter s9 ∧
  PQueryCounter s10 = PQueryCounter s9 ∧
  (∀ j < ceil-log (to-nat dg + 1).
    fmlookup (HashMap s10)
      (CompositionFriChallenge (PCompositionFriCounter s9 + j)
        (foldl concat (PState s9) (take (Suc j) composition-roots))) =
        Some (composition-bs ! (length ([] :: 'f list) + j)))
by (rule staged-composition-fri-program-alignment
    [OF controlled composition-bound composition-out])
let ?key =
  CompositionFriChallenge i
    (foldl concat
      (concat
        (foldl concat

```

```

      (concat
        (foldl concat (concat 0 fr) trace-roots)
        trace-final)
      as)
    dg)
  (take (Suc i) composition-roots))
have i-bound':  $i < \text{ceil-log } (\text{to-nat } dg + 1)$ 
  using i-bound unfolding data-eq by simp
have lookup-s10:
  fmlookup (HashMap s10) ?key = Some (composition-bs ! i)
  using composition-alignment i-bound' start-fields by simp
have ext-s10-s11:  $s10 \leq s11$ 
  using controlled-ro-program-extension[OF composition-final-controlled]
    composition-final-out
  unfolding hash-extension-preserving-def by blast
have s12-eq:
  s12 = s11()
  PState := concat (PState s11) composition-final,
  PTranscript := PTranscript s11 @ [composition-final])
  by (rule record-staged-message-outcome[OF record-composition-final])
have ext-s11-s12:  $s11 \leq s12$ 
  unfolding s12-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have query-bound:  $0 + \text{rounds} \leq \text{length } (\text{query-opening-budgets budgets})$ 
  using wf unfolding staged-budget-wellformed-def by simp
have ext-s12-attacker:  $s12 \leq \text{attacker-state}$ 
  by (rule hash-target-program-outcome-extension
    [OF hash-target-program-staged-query-program[OF controlled query-bound]
      query-out])
have ext-s10-s12:  $s10 \leq s12$ 
  by (rule hash-ext-trans[OF ext-s10-s11 ext-s11-s12])
have ext-s10-attacker:  $s10 \leq \text{attacker-state}$ 
  by (rule hash-ext-trans[OF ext-s10-s12 ext-s12-attacker])
have lookup-attacker:
  fmlookup (HashMap attacker-state) ?key = Some (composition-bs ! i)
  by (rule hash-extension-lookup[OF lookup-s10 ext-s10-attacker])
show ?thesis
  unfolding data-eq staged-composition-fri-start-hash-def
    staged-trace-fri-start-hash-def
  using lookup-attacker by simp
qed

```

lemma staged-transcript-program-outcome-header-transcript:

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and outcome:

Some (data, attacker-state) $\in$

set-dist

(execute (staged-transcript-program A)

adversary-initial-state)

shows

verifier-header-transcript
(*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
(*staged-trace-root data*)
(*staged-trace-fri-roots data*)
(*staged-trace-final data*)
(*staged-alphas data*)
(*staged-degree data*)
(*staged-composition-fri-roots data*)
(*staged-composition-final data*)
(*List.concat (staged-query-chunks data)*))

proof –

have *shape*:

length (staged-trace-fri-roots data) = ceil-log clength $\wedge$
length (staged-trace-fri-challenges data) = ceil-log clength $\wedge$
length (staged-alphas data) = length spec $\wedge$
length (staged-composition-fri-roots data) =
 ceil-log (to-nat (staged-degree data) + 1) $\wedge$
length (staged-composition-fri-challenges data) =
 ceil-log (to-nat (staged-degree data) + 1) $\wedge$
ceil-log (to-nat (staged-degree data) + 1) $\leq$
 ceil-log (maxDegree + 1) $\wedge$
length (staged-query-chunks data) = rounds

by (*rule staged-transcript-program-outcome-shape*
 [*OF wf controlled outcome*])

show *?thesis*

by (*rule staged-proof-transcript-verifier-header-transcript*)
 (*use shape in simp-all*)

qed

lemma *staged-security-experiment-outcomeE*:

assumes

Some (result, final-state) $\in$
 set-dist (execute (staged-security-experiment A) initial-state)

obtains *data attacker-state* **where**

Some (data, attacker-state) $\in$
 set-dist (execute (staged-transcript-program A) initial-state)

Some (result, final-state) $\in$

set-dist

(*execute verify-monad*

 (*verifier-state-from-adversary attacker-state*

 (*staged-proof-transcript data*)))

using *assms unfolding staged-security-experiment-def*

by (*auto elim!: set-dist-bindE intro: that*)

lemma *accepted-staged-security-experimentE*:

assumes *outcome*:

out $\in$ set-dist

```

    (execute (staged-security-experiment A) adversary-initial-state)
  and accepted:  $\neg$  Option.is-none out
obtains result final-state data attacker-state where
  out = Some (result, final-state)
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (staged-transcript-program A) adversary-initial-state)
  Some (result, final-state)  $\in$ 
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
proof –
from accepted obtain result final-state where
  out: out = Some (result, final-state)
by (cases out) auto
from staged-security-experiment-outcomeE[OF outcome[unfolded out]]
obtain data attacker-state where
  builder:
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (staged-transcript-program A)
        adversary-initial-state)
  and verifier:
    Some (result, final-state)  $\in$ 
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
by blast
show ?thesis by (rule that[OF out builder verifier])
qed

lemma accepted-staged-security-experiment-headerE:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
  out  $\in$  set-dist
    (execute (staged-security-experiment A) adversary-initial-state)
and accepted:  $\neg$  Option.is-none out
obtains result final-state data attacker-state where
  out = Some (result, final-state)
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (staged-transcript-program A)
      adversary-initial-state)
  Some (result, final-state)  $\in$ 
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
  verifier-header-transcript

```

```

    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  proof -
    from accepted-staged-security-experimentE[OF outcome accepted]
    obtain result final-state data attacker-state where
      out-eq: out = Some (result, final-state)
    and builder:
      Some (data, attacker-state) ∈
        set-dist (execute (staged-transcript-program A)
          adversary-initial-state)
    and verifier:
      Some (result, final-state) ∈
        set-dist
          (execute verify-monad
            (verifier-state-from-adversary attacker-state
              (staged-proof-transcript data)))
    by blast
  have header:
    verifier-header-transcript
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      (List.concat (staged-query-chunks data))
    by (rule staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
  show ?thesis
    by (rule that[OF out-eq builder verifier header])
qed

lemma accepted-staged-security-experiment-transcript-shapeE:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    out ∈ set-dist
      (execute (staged-security-experiment A) adversary-initial-state)

```

```

and accepted:  $\neg$  Option.is-none out
obtains result final-state data attacker-state query-idxs where
  out = Some (result, final-state)
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (staged-transcript-program A)
      adversary-initial-state)
  Some (result, final-state)  $\in$ 
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
  accepted-transcript-shape
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
  out
    (staged-alphas data)
  query-idxs
proof –
from accepted-staged-security-experiment-headerE
  [OF wf controlled outcome accepted]
obtain result final-state data attacker-state where
  out-eq: out = Some (result, final-state)
and builder:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (staged-transcript-program A)
      adversary-initial-state)
and verifier:
  Some (result, final-state)  $\in$ 
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
and staged-header:
  verifier-header-transcript
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by blast
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from verify-monad-accepted-transcript-shape[OF verifier]

```

```

obtain as query-idxs where shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  as query-idxs
  by blast
from shape obtain result' final-state' fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  shape-out: Some (result, final-state) = Some (result', final-state')
  and shape-header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  by (elim accepted-transcript-shape-header-query-extraction)
have as-eq: as = staged-alphas data
  using verifier-header-transcript-unique[OF staged-header shape-header]
  by simp
have shape-staged:
  accepted-transcript-shape ?s out (staged-alphas data) query-idxs
  using shape unfolding out-eq as-eq .
show ?thesis
  by (rule that[OF out-eq builder verifier shape-staged])
qed

```

```

lemma accepted-staged-security-experiment-query-chunksE:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
  out ∈ set-dist
  (execute (staged-security-experiment A) adversary-initial-state)
and accepted: ¬ Option.is-none out
obtains result final-state data attacker-state raw-idxs query-idxs
  query-chunks trailing
where
  out = Some (result, final-state)
  Some (data, attacker-state) ∈
    set-dist (execute (staged-transcript-program A)
      adversary-initial-state)
  Some (result, final-state) ∈
    set-dist
    (execute verify-monad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
  length raw-idxs = rounds
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  length query-chunks = rounds
  List.concat query-chunks @ trailing =
    List.concat (staged-query-chunks data)
  ∧ i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
    (staged-trace-fri-roots data)
    (staged-composition-fri-roots data)

```



```

    (query-chunks ! i)
   $\wedge i. i < \text{rounds} \implies$ 
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)
   $\wedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) query-chunks i)) =
      Some (raw-idxs ! i)
   $\forall \text{idx} \in \text{set query-idxs}. \text{idx} < \text{clength} * \text{scale}$ 
proof –
from accepted-staged-security-experiment-headerE
  [OF wf controlled outcome accepted]
obtain result final-state data attacker-state where
  out-eq: out = Some (result, final-state)
and builder:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (staged-transcript-program A)
      adversary-initial-state)
and verifier:
  Some (result, final-state)  $\in$ 
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))))
and staged-header:
  verifier-header-transcript
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by blast
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from verify-monad-accepted-transcript-shape[OF verifier]
obtain as accepted-query-idxs where shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  as accepted-query-idxs
by blast

```

```

from accepted-transcript-shape-query-chunksE[OF shape]
obtain result' final-state' fr f-fri-roots f-final dg
    composition-fri-roots final rest raw-idxs query-chunks trailing
where shape-out:
    Some (result, final-state) = Some (result', final-state')
and header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
    accepted-query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-chunks: length query-chunks = rounds
and concat-chunks: List.concat query-chunks @ trailing = rest
and chunk-shape:
     $\bigwedge i. i < rounds \implies$ 
    verifier-query-round-chunk (accepted-query-idxs ! i)
    f-fri-roots composition-fri-roots (query-chunks ! i)
and lookup:
     $\bigwedge i. i < rounds \implies$ 
    fmlookup (HashMap final-state')
    (QueryIndexChallenge (PQueryCounter ?s + i)
    (state-after-query-chunks
    (verifier-header-state ?s fr f-fri-roots f-final as dg
    composition-fri-roots final
    query-chunks i))) =
    Some (raw-idxs ! i)
and idx-bound:
     $\forall idx \in \text{set } \text{accepted-query-idxs}. idx < \text{clength} * \text{scale}$ 
by blast
have header-eq:
    fr = staged-trace-root data  $\wedge$ 
    f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
    f-final = staged-trace-final data  $\wedge$ 
    as = staged-alphas data  $\wedge$ 
    dg = staged-degree data  $\wedge$ 
    composition-fri-roots = staged-composition-fri-roots data  $\wedge$ 
    final = staged-composition-final data  $\wedge$ 
    rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF staged-header header]
by simp
have final-state-eq: final-state' = final-state
using shape-out by simp
have header-state-eq:
    verifier-header-state ?s fr f-fri-roots f-final as dg
    composition-fri-roots final =
    staged-query-start-hash data
using header-eq
unfolding verifier-header-state-def verifier-header-messages-def
    staged-query-start-hash-def staged-composition-fri-start-hash-def

```

```

    staged-trace-fri-start-hash-def
  by simp
have lookup':
   $\wedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) query-chunks i)) =
        Some (raw-idxs ! i)
  using lookup final-state-eq header-state-eq by simp
have chunk-shape':
   $\wedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (accepted-query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
      (query-chunks ! i)
  using chunk-shape header-eq by simp
have chunk-len:
   $\wedge i. i < \text{rounds} \implies$ 
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (accepted-query-idxs ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)
  by (rule verifier-query-round-chunk-length[OF chunk-shape'])
show ?thesis
  by (rule that[OF out-eq builder verifier len-raw query-idxs-eq
    len-chunks])
    (use concat-chunks header-eq chunk-shape' chunk-len lookup' idx-bound
      in simp-all)
qed

lemma staged-transcript-shape-alpha-badD:
  assumes shape:
    accepted-transcript-shape s out staged-as staged-query-idxs
  and bad: alpha-bad-set-hit s B out
  shows  $\exists \text{trace-table}. \text{staged-as} \in B \text{ trace-table}$ 
proof -
  from bad obtain trace-table composition-table as query-idxs where
    bound:
      accepted-with-bound-tables s out trace-table composition-table
      as query-idxs
  and as-bad: as  $\in B$  trace-table
  unfolding alpha-bad-set-hit-def by blast
  have shape-bad: accepted-transcript-shape s out as query-idxs
  using bound unfolding accepted-with-bound-tables-def
    accepted-with-tables-def by simp
  have as = staged-as
  by (rule accepted-transcript-shape-alphas-unique[OF shape shape-bad])
  then show ?thesis

```

```

    using as-bad by blast
qed

lemma staged-security-with-data-alpha-bad-set-hit-imp-alpha-list-hit:
  assumes envelope:  $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq B$ 
    and hit: staged-security-with-data-alpha-bad-set-hit bad-sets out
  shows staged-security-with-data-alpha-list-hit B out
  using assms
  unfolding staged-security-with-data-alpha-bad-set-hit-def
    staged-security-with-data-alpha-list-hit-def
  by (auto split: option.splits prod.splits)

lemma staged-security-with-data-alpha-list-bound-from-transcript:
  assumes transcript-bound:
    wp-event (staged-transcript-program A)
      (staged-transcript-alpha-list-hit B) adversary-initial-state  $\leq C$ 
  shows
    wp-event (staged-security-experiment-with-data A)
      (staged-security-with-data-alpha-list-hit B)
      adversary-initial-state  $\leq C$ 
  unfolding staged-security-experiment-with-data-def
proof (rule wp-event-bind-bound-by-head-event[OF transcript-bound])
  show staged-security-with-data-alpha-list-hit B None  $\implies$ 
    staged-transcript-alpha-list-hit B None
  unfolding staged-security-with-data-alpha-list-hit-def
    staged-transcript-alpha-list-hit-def
  by simp
next
fix data attacker-state out
assume cont:
  out  $\in$ 
    set-dist
      (execute
        (get  $\gg=$ 
          ( $\lambda s.$  put
            (verifier-state-from-adversary s
              (staged-proof-transcript data))  $\gg=$ 
            ( $\lambda -. \text{verify-monad}$   $\gg=$ 
              ( $\lambda \text{result. return (data, result)}$ ))))))
        attacker-state)
    and hit: staged-security-with-data-alpha-list-hit B out
have data-bad: staged-alphas data  $\in B$ 
proof (cases out)
  case None
  then show ?thesis
    using hit unfolding staged-security-with-data-alpha-list-hit-def
    by simp
next
  case (Some result)

```

```

then obtain data' result' final-state where out-eq:
  out = Some ((data', result'), final-state)
  by (cases result, auto split: prod.splits)
from cont[unfolded out-eq]
obtain s3 where
  ret:
    Some ((data', result'), final-state) ∈
      set-dist (execute (return (data, result')) s3)
  by (auto elim!: set-dist-bindE)
have data'-eq: data' = data
  using ret by simp
show ?thesis
  using hit unfolding out-eq data'-eq
    staged-security-with-data-alpha-list-hit-def
  by simp
qed
show staged-transcript-alpha-list-hit B
  (Some (data, attacker-state))
  using data-bad unfolding staged-transcript-alpha-list-hit-def by simp
qed

lemma staged-security-with-data-alpha-bad-set-bound-from-alpha-list:
assumes envelope:  $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq B$ 
and list-bound:
  wp-event (staged-security-experiment-with-data A)
    (staged-security-with-data-alpha-list-hit B)
    adversary-initial-state ≤ C
shows
  wp-event (staged-security-experiment-with-data A)
    (staged-security-with-data-alpha-bad-set-hit bad-sets)
    adversary-initial-state ≤ C
proof –
have wp-event (staged-security-experiment-with-data A)
  (staged-security-with-data-alpha-bad-set-hit bad-sets)
  adversary-initial-state ≤
  wp-event (staged-security-experiment-with-data A)
  (staged-security-with-data-alpha-list-hit B)
  adversary-initial-state
by (rule wp-event-mono)
  (rule staged-security-with-data-alpha-bad-set-hit-imp-alpha-list-hit
    [OF envelope])
also have ... ≤ C
by (rule list-bound)
finally show ?thesis .
qed

definition checked-staged-security-with-data-state-alpha-list-hit
  :: 'f list set ⇒
    ((('f staged-proof-data × 'f protocol-channel) × unit list) ×

```

```

    'f protocol-channel) option  $\Rightarrow$  bool
  where
    checked-staged-security-with-data-state-alpha-list-hit B out  $\longleftrightarrow$ 
      (case out of
        None  $\Rightarrow$  False
      | Some (((data, -), -), -)  $\Rightarrow$  staged-alphas data  $\in$  B)

definition checked-staged-security-with-data-state-alpha-bad-set-hit
  :: ('f list  $\Rightarrow$  'f list set)  $\Rightarrow$ 
    (((('f staged-proof-data  $\times$  'f protocol-channel)  $\times$  unit list)  $\times$ 
      'f protocol-channel) option  $\Rightarrow$  bool
  where
    checked-staged-security-with-data-state-alpha-bad-set-hit bad-sets out
       $\longleftrightarrow$ 
      (case out of
        None  $\Rightarrow$  False
      | Some (((data, -), -), -)  $\Rightarrow$ 
          ( $\exists$  trace-table. staged-alphas data  $\in$  bad-sets trace-table))

definition checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
  :: ('f list  $\Rightarrow$  'f list set)  $\Rightarrow$ 
    (((('f staged-proof-data  $\times$  'f protocol-channel)  $\times$  unit list)  $\times$ 
      'f protocol-channel) option  $\Rightarrow$  bool
  where
    checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
      bad-sets out  $\longleftrightarrow$ 
      (case out of
        None  $\Rightarrow$  False
      | Some (((data, attacker-state), -), -)  $\Rightarrow$ 
          (let s =
              verifier-state-from-adversary attacker-state
              (staged-proof-transcript data)
            in staged-alphas data  $\in$ 
              alpha-header-supported-union-bad-sets s bad-sets
              (staged-trace-root data)
              (staged-trace-fri-roots data)
              (staged-trace-final data))))

lemma checked-staged-security-with-data-state-alpha-bad-set-hit-imp-alpha-list-hit:
  assumes envelope:  $\bigwedge$  trace-table. bad-sets trace-table  $\subseteq$  B
  and hit:
    checked-staged-security-with-data-state-alpha-bad-set-hit bad-sets out
  shows checked-staged-security-with-data-state-alpha-list-hit B out
  using assms
  unfolding checked-staged-security-with-data-state-alpha-bad-set-hit-def
    checked-staged-security-with-data-state-alpha-list-hit-def
  by (auto split: option.splits prod.splits)

lemma checked-staged-security-with-data-state-alpha-list-bound-from-transcript:

```

```

assumes transcript-bound:
  wp-event (checked-staged-transcript-program A)
    (staged-transcript-alpha-list-hit B) adversary-initial-state  $\leq C$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (checked-staged-security-with-data-state-alpha-list-hit B)
    adversary-initial-state  $\leq C$ 
unfolding checked-staged-security-experiment-with-data-state-def
proof (rule wp-event-bind-bound-by-head-event[OF transcript-bound])
show
  checked-staged-security-with-data-state-alpha-list-hit B None  $\implies$ 
    staged-transcript-alpha-list-hit B None
unfolding checked-staged-security-with-data-state-alpha-list-hit-def
  staged-transcript-alpha-list-hit-def
by simp
next
fix data attacker-state out
assume cont:
  out  $\in$ 
    set-dist
      (execute
        (get  $\gg$ 
          ( $\lambda s$ . put
            (verifier-state-from-adversary s
              (staged-proof-transcript data))  $\gg$ 
            ( $\lambda$ -. verify-monad  $\gg$ 
              ( $\lambda$ result. return ((data, s), result))))))
          attacker-state)
and hit: checked-staged-security-with-data-state-alpha-list-hit B out
have data-bad: staged-alphas data  $\in B$ 
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding checked-staged-security-with-data-state-alpha-list-hit-def
  by simp
next
case (Some result-pack)
then obtain data' attacker-state' result final-state where out-eq:
  out = Some (((data', attacker-state'), result), final-state)
  by (cases result-pack, auto split: prod.splits)
from cont[unfolded out-eq]
obtain saved-state state-after-get where
  ret:
    Some (((data', attacker-state'), result), final-state)  $\in$ 
      set-dist (execute (return ((data, saved-state), result))
        final-state)
  by (auto elim!: set-dist-bindE)
have data'-eq: data' = data

```

```

    using ret by simp
show ?thesis
    using hit
    unfolding out-eq data'-eq
      checked-staged-security-with-data-state-alpha-list-hit-def
    by simp
qed
show staged-transcript-alpha-list-hit B
  (Some (data, attacker-state))
    using data-bad unfolding staged-transcript-alpha-list-hit-def by simp
qed

```

lemma *checked-staged-security-with-data-state-alpha-bad-set-bound-from-alpha-list:*

```

assumes envelope:  $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq B$ 
and list-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (checked-staged-security-with-data-state-alpha-list-hit B)
  adversary-initial-state  $\leq C$ 

```

shows

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (checked-staged-security-with-data-state-alpha-bad-set-hit bad-sets)
  adversary-initial-state  $\leq C$ 

```

proof –

```

have wp-event (checked-staged-security-experiment-with-data-state A)
  (checked-staged-security-with-data-state-alpha-bad-set-hit bad-sets)
  adversary-initial-state  $\leq$ 
wp-event (checked-staged-security-experiment-with-data-state A)
  (checked-staged-security-with-data-state-alpha-list-hit B)
  adversary-initial-state
by (rule wp-event-mono)
  (rule
    checked-staged-security-with-data-state-alpha-bad-set-hit-imp-alpha-list-hit
    [OF envelope])
also have ...  $\leq C$ 
by (rule list-bound)
finally show ?thesis .

```

qed

lemma *checked-staged-security-with-data-state-composition-randomization-bad-imp-alpha-header-supported-bad-set-h*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out  $\in$ 
  set-dist
  (execute (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
and bad:
  staged-security-with-data-state-verifier-event
  composition-randomization-bad out

```



```

shows
  checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
  composition-trace-bad-alpha-space out
proof (cases out)
case None
then show ?thesis
  using bad
  unfolding staged-security-with-data-state-verifier-event-def
  by simp
next
case (Some result-pack)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have support-some:
  Some (((data, attacker-state), result), final-state) ∈
  set-dist
  (execute (checked-staged-security-experiment-with-data-state A)
  adversary-initial-state)
  using support unfolding out-eq .
from checked-staged-security-experiment-with-data-state-outcomeE
[OF support-some]
have builder:
  Some (data, attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-program A)
  adversary-initial-state)
and verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast+
have staged-header:
  verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
from bad[unfolded out-eq
  staged-security-with-data-state-verifier-event-def]
have random-bad:
  composition-randomization-bad ?s (Some (result, final-state))

```

```

by simp
have alpha-hit:
  alpha-bad-set-hit ?s composition-trace-bad-alpha-space
    (Some (result, final-state))
  by (rule composition-randomization-bad-imp-alpha-bad-set-hit
      [OF random-bad])
from alpha-hit obtain trace-table composition-table as query-idxs where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and as-bad: as ∈ composition-trace-bad-alpha-space trace-table
  unfolding alpha-bad-set-hit-def by blast
have shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  as query-idxs
  using bound
  unfolding accepted-with-bound-tables-def accepted-with-tables-def
  by simp
from shape obtain result' final-state' fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  by (elim accepted-transcript-shape-header-query-extraction)
have as-eq: as = staged-alphas data
  using verifier-header-transcript-unique[OF staged-header header]
  by simp
have bound-staged:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table (staged-alphas data) query-idxs
  using bound as-eq by simp
have staged-bad:
  staged-alphas data ∈ composition-trace-bad-alpha-space trace-table
  using as-bad as-eq by simp
have candidate:
  trace-table ∈
    alpha-header-supported-trace-table-candidates ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
  unfolding alpha-header-supported-trace-table-candidates-def
  using verifier bound-staged staged-header by blast
have union-bad:
  staged-alphas data ∈
    alpha-header-supported-union-bad-sets ?s
      composition-trace-bad-alpha-space
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)

```

```

unfolding alpha-header-supported-union-bad-sets-def
using candidate staged-bad
      composition-trace-bad-alpha-space-subset-alpha-space[of trace-table]
by auto
show ?thesis
using union-bad
unfolding out-eq
      checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-def
by simp
qed

```

```

lemma checked-staged-security-with-data-state-composition-randomization-bad-bound-from-alpha-header-supported-bad-set-hit-on-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and alpha-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
    composition-trace-bad-alpha-space)
  adversary-initial-state ≤ C
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-randomization-bad)
  adversary-initial-state ≤ C
proof –
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-randomization-bad)
  adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
    composition-trace-bad-alpha-space)
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (rule
    checked-staged-security-with-data-state-composition-randomization-bad-imp-alpha-header-supported-bad-set-hit-on-support
      [OF wf controlled])
also have ... ≤ C
by (rule alpha-bound)
finally show ?thesis .
qed

```

```

lemma checked-staged-security-with-data-state-composition-randomization-bad-imp-alpha-bad-set-hit-on-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out ∈
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A))

```

```

      adversary-initial-state)
and bad:
  staged-security-with-data-state-verifier-event
  composition-randomization-bad out
shows
  checked-staged-security-with-data-state-alpha-bad-set-hit
  composition-trace-bad-alpha-space out
proof (cases out)
case None
then show ?thesis
  using bad
  unfolding staged-security-with-data-state-verifier-event-def
  by simp
next
case (Some result-pack)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have support-some:
  Some (((data, attacker-state), result), final-state) ∈
  set-dist
  (execute (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
  using support unfolding out-eq .
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF support-some]
have builder:
  Some (data, attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-program A)
    adversary-initial-state)
  by blast
have staged-header:
  verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
from bad[unfolded out-eq
  staged-security-with-data-state-verifier-event-def]

```

```

have random-bad:
  composition-randomization-bad ?s (Some (result, final-state))
  by simp
have alpha-hit:
  alpha-bad-set-hit ?s composition-trace-bad-alpha-space
    (Some (result, final-state))
  by (rule composition-randomization-bad-imp-alpha-bad-set-hit
    [OF random-bad])
from alpha-hit obtain trace-table composition-table as query-idxs where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and as-bad: as ∈ composition-trace-bad-alpha-space trace-table
  unfolding alpha-bad-set-hit-def by blast
have shape:
  accepted-transcript-shape ?s (Some (result, final-state))
    as query-idxs
  using bound
  unfolding accepted-with-bound-tables-def accepted-with-tables-def
  by simp
from shape obtain result' final-state' fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  by (elim accepted-transcript-shape-header-query-extraction)
have as-eq: as = staged-alphas data
  using verifier-header-transcript-unique[OF staged-header header]
  by simp
have staged-bad:
  staged-alphas data ∈ composition-trace-bad-alpha-space trace-table
  using as-bad as-eq by simp
show ?thesis
  using staged-bad
  unfolding out-eq
    checked-staged-security-with-data-state-alpha-bad-set-hit-def
  by auto
qed

lemma checked-staged-security-with-data-state-composition-randomization-bad-bound-from-alpha-bad-set:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and alpha-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (checked-staged-security-with-data-state-alpha-bad-set-hit
      composition-trace-bad-alpha-space)
    adversary-initial-state ≤ C
shows
  wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (staged-security-with-data-state-verifier-event
     composition-randomization-bad)
    adversary-initial-state ≤ C
proof –
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-randomization-bad)
    adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (checked-staged-security-with-data-state-alpha-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (rule
     checked-staged-security-with-data-state-composition-randomization-bad-imp-alpha-bad-set-hit-on-support
     [OF wf controlled])
  also have ... ≤ C
    by (rule alpha-bound)
  finally show ?thesis .
qed

```

lemma *checked-staged-security-with-data-state-composition-degree-bad-bound:*

shows

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-degree-bad)
    adversary-initial-state ≤ 0

```

proof –

have event-eq:

```

  staged-security-with-data-state-verifier-event composition-degree-bad =
    (λout :: (((f staged-proof-data × f protocol-channel) ×
               unit list) × f protocol-channel) option. False)
  by (rule ext)
    (auto simp: staged-security-with-data-state-verifier-event-def
               composition-degree-bad-false[OF spec-degree-wellformed-from-spec-query-margin]
               split: option.splits prod.splits)

```

show ?thesis

unfolding event-eq wp-event-def wp-def dist-expect-def **by** simp

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-alpha-bad-set:*

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and alpha-bound:

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (checked-staged-security-with-data-state-alpha-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ composition-error-bound

```

shows

```

  wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state ≤ composition-error-bound
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?degree =
    staged-security-with-data-state-verifier-event composition-degree-bad
  let ?random =
    staged-security-with-data-state-verifier-event
    composition-randomization-bad
  have event-eq:
    staged-security-with-data-state-verifier-event composition-bad =
      (λout. ?degree out ∨ ?random out)
  by (rule ext)
    (auto simp: staged-security-with-data-state-verifier-event-def
      composition-bad-def split: option.splits prod.splits)
  have degree-bound:
    wp-event ?M ?degree adversary-initial-state ≤ 0
  by (rule
    checked-staged-security-with-data-state-composition-degree-bad-bound)
  have random-bound:
    wp-event ?M ?random adversary-initial-state ≤
      composition-error-bound
  by (rule
    checked-staged-security-with-data-state-composition-randomization-bad-bound-from-alpha-bad-set
      [OF wf controlled alpha-bound])
  have wp-event ?M
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state ≤
    wp-event ?M ?degree adversary-initial-state +
    wp-event ?M ?random adversary-initial-state
  unfolding event-eq by (rule wp-event-union-bound)
  also have ... ≤ 0 + composition-error-bound
  by (intro add-mono degree-bound random-bound)
  finally show ?thesis by simp
qed

```

```

lemma checked-staged-security-with-data-state-composition-bad-bound-from-alpha-header-supported-bad-set:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and alpha-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
        composition-trace-bad-alpha-space)
      adversary-initial-state ≤ composition-error-bound
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event composition-bad)
      adversary-initial-state ≤ composition-error-bound
proof –

```

```

let ?M = checked-staged-security-experiment-with-data-state A
let ?degree =
  staged-security-with-data-state-verifier-event composition-degree-bad
let ?random =
  staged-security-with-data-state-verifier-event
  composition-randomization-bad
have event-eq:
  staged-security-with-data-state-verifier-event composition-bad =
    ( $\lambda$ out. ?degree out  $\vee$  ?random out)
by (rule ext)
  (auto simp: staged-security-with-data-state-verifier-event-def
    composition-bad-def split: option.splits prod.splits)
have degree-bound:
  wp-event ?M ?degree adversary-initial-state  $\leq 0$ 
by (rule
  checked-staged-security-with-data-state-composition-degree-bad-bound)
have random-bound:
  wp-event ?M ?random adversary-initial-state  $\leq$ 
    composition-error-bound
by (rule
  checked-staged-security-with-data-state-composition-randomization-bad-bound-from-alpha-header-supported-bad-s
    [OF wf controlled alpha-bound])
have wp-event ?M
  (staged-security-with-data-state-verifier-event composition-bad)
  adversary-initial-state  $\leq$ 
  wp-event ?M ?degree adversary-initial-state +
  wp-event ?M ?random adversary-initial-state
  unfolding event-eq by (rule wp-event-union-bound)
also have ...  $\leq 0 + \text{composition-error-bound}$ 
  by (intro add-mono degree-bound random-bound)
finally show ?thesis by simp
qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-alpha-header-supported-bad-set-general:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and alpha-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
    composition-trace-bad-alpha-space)
  adversary-initial-state  $\leq C$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event composition-bad)
  adversary-initial-state  $\leq C$ 
proof –
let ?M = checked-staged-security-experiment-with-data-state A
let ?degree =
  staged-security-with-data-state-verifier-event composition-degree-bad

```



```

let ?random =
  staged-security-with-data-state-verifier-event
  composition-randomization-bad
have event-eq:
  staged-security-with-data-state-verifier-event composition-bad =
    ( $\lambda$ out. ?degree out  $\vee$  ?random out)
by (rule ext)
    (auto simp: staged-security-with-data-state-verifier-event-def
      composition-bad-def split: option.splits prod.splits)
have degree-bound:
  wp-event ?M ?degree adversary-initial-state  $\leq 0$ 
by (rule
  checked-staged-security-with-data-state-composition-degree-bad-bound)
have random-bound:
  wp-event ?M ?random adversary-initial-state  $\leq C$ 
by (rule
  checked-staged-security-with-data-state-composition-randomization-bad-bound-from-alpha-header-supported-bad-s
    [OF wf controlled alpha-bound])
have wp-event ?M
  (staged-security-with-data-state-verifier-event composition-bad)
  adversary-initial-state  $\leq$ 
  wp-event ?M ?degree adversary-initial-state +
  wp-event ?M ?random adversary-initial-state
  unfolding event-eq by (rule wp-event-union-bound)
also have ...  $\leq 0 + C$ 
by (intro add-mono degree-bound random-bound)
finally show ?thesis by simp
qed

lemma staged-composition-alpha-header-supported-union-fraction-bound-if-no-pairwise-merkle-bad:
assumes no-bad:
   $\neg$  alpha-supported-pairwise-merkle-bad
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
shows
  nnreal
    (card
      (alpha-header-supported-union-bad-sets
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
        composition-trace-bad-alpha-space
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data))) /
  nnreal (card alpha-space)  $\leq$  composition-error-bound
by (rule
  composition-trace-alpha-header-supported-union-fraction-bound-if-unique-candidate)
  (rule
  alpha-header-supported-candidate-unique-if-no-global-pairwise-merkle-bad

```

[OF no-bad])

lemma *accepted-staged-security-experiment-fri-challengesE*:

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *outcome*:

out $\in$ *set-dist*

(*execute* (*staged-security-experiment* *A*) *adversary-initial-state*)

and *accepted*: $\neg$ *Option.is-none* *out*

obtains *result final-state data attacker-state* **where**

out = *Some* (*result*, *final-state*)

Some (*data*, *attacker-state*) $\in$

set-dist (*execute* (*staged-transcript-program* *A*)

adversary-initial-state)

Some (*result*, *final-state*) $\in$

set-dist

(*execute* *verify-monad*

(*verifier-state-from-adversary* *attacker-state*

(*staged-proof-transcript* *data*)))

accepted-fri-challenges

(*verifier-state-from-adversary* *attacker-state*

(*staged-proof-transcript* *data*))

out

(*staged-trace-fri-challenges* *data*)

(*staged-degree* *data*)

(*staged-composition-fri-challenges* *data*)

proof –

from *accepted-staged-security-experiment-headerE*

[OF *wf* *controlled* *outcome* *accepted*]

obtain *result final-state data attacker-state* **where**

out-eq: *out* = *Some* (*result*, *final-state*)

and *builder*:

Some (*data*, *attacker-state*) $\in$

set-dist (*execute* (*staged-transcript-program* *A*)

adversary-initial-state)

and *verifier*:

Some (*result*, *final-state*) $\in$

set-dist

(*execute* *verify-monad*

(*verifier-state-from-adversary* *attacker-state*

(*staged-proof-transcript* *data*)))

and *staged-header*:

verifier-header-transcript

(*verifier-state-from-adversary* *attacker-state*

(*staged-proof-transcript* *data*))

(*staged-trace-root* *data*)

(*staged-trace-fri-roots* *data*)

(*staged-trace-final* *data*)

(*staged-alphas* *data*)

```

    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by blast
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from verify-monad-header-random-oracle-replay[OF verifier]
obtain fr f-fl f-final as dg fl final query-state where
  header:
    verifier-header-transcript ?s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
and query-state-eq:
  PState query-state =
    verifier-header-state ?s fr (map snd f-fl) f-final as dg
    (map snd fl) final
and query-out:
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      query-state)
and query-count-header: PQueryCounter query-state = PQueryCounter ?s
and ext-s-query: ?s ≤ query-state
and trace-lookup:
  ∧i. i < length f-fl ⇒
    fmlookup (HashMap query-state)
      (TraceFriChallenge (PTraceFriCounter ?s + i)
        (foldl concat (concat (PState ?s) fr)
          (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i))
and alpha-lookup:
  ∧i. i < length as ⇒
    fmlookup (HashMap query-state)
      (AlphaChallenge (PAlphaCounter ?s + i) (foldl concat
        (concat (foldl concat (concat (PState ?s) fr) (map snd f-fl))
          f-final)
        (take i as))) =
      Some (as ! i)
and comp-lookup:
  ∧i. i < length fl ⇒
    fmlookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter ?s + i)
        (foldl concat
          (concat
            (foldl concat
              (concat

```

```

      (foldl concat (concat (PState ?s) fr) (map snd f-fl))
      f-final)
    as)
  dg)
  (take (Suc i) (map snd fl)))) =
  Some (fst (fl ! i))
and degree-bound: to-nat dg ≤ maxDegree
by blast
have header-unique:
  fr = staged-trace-root data ∧
  map snd f-fl = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  map snd fl = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  PTranscript query-state = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF staged-header header] by simp
have shape:
  length (staged-trace-fri-roots data) = ceil-log clength ∧
  length (staged-trace-fri-challenges data) = ceil-log clength ∧
  length (staged-alphas data) = length spec ∧
  length (staged-composition-fri-roots data) =
    ceil-log (to-nat (staged-degree data) + 1) ∧
  length (staged-composition-fri-challenges data) =
    ceil-log (to-nat (staged-degree data) + 1) ∧
  ceil-log (to-nat (staged-degree data) + 1) ≤
    ceil-log (maxDegree + 1) ∧
  length (staged-query-chunks data) = rounds
by (rule staged-transcript-program-outcome-shape
  [OF wf controlled builder])
have trace-bs-eq:
  map fst f-fl = staged-trace-fri-challenges data
proof (rule nth-equalityI)
  show length (map fst f-fl) =
    length (staged-trace-fri-challenges data)
    using header-unique shape by (metis length-map)
next
  fix i
  assume i-bound: i < length (map fst f-fl)
  then have i-f-fl: i < length f-fl
    by simp
  have i-round: i < ceil-log clength
    using i-f-fl header-unique shape by (metis length-map)
  have staged-lookup-attacker:
    fmlookup (HashMap attacker-state)
      (TraceFriChallenge i
        (foldl concat (staged-trace-fri-start-hash data)
          (take (Suc i) (staged-trace-fri-roots data)))) =

```

```

    Some (staged-trace-fri-challenges data ! i)
  by (rule staged-transcript-program-outcome-trace-challenge-lookup
      [OF wf controlled builder i-round])
have staged-lookup-s:
  fmlookup (HashMap ?s)
    (TraceFriChallenge i
      (foldl concat (staged-trace-fri-start-hash data)
        (take (Suc i) (staged-trace-fri-roots data)))) =
    Some (staged-trace-fri-challenges data ! i)
  using staged-lookup-attacker by simp
have staged-lookup-query:
  fmlookup (HashMap query-state)
    (TraceFriChallenge i
      (foldl concat (staged-trace-fri-start-hash data)
        (take (Suc i) (staged-trace-fri-roots data)))) =
    Some (staged-trace-fri-challenges data ! i)
  by (rule hash-extension-lookup[OF staged-lookup-s ext-s-query])
have verifier-lookup-query:
  fmlookup (HashMap query-state)
    (TraceFriChallenge i
      (foldl concat (staged-trace-fri-start-hash data)
        (take (Suc i) (staged-trace-fri-roots data)))) =
    Some (fst (f-fl ! i))
  using trace-lookup[OF i-f-fl] header-unique
  unfolding staged-trace-fri-start-hash-def
  by simp
show map fst f-fl ! i = staged-trace-fri-challenges data ! i
  using verifier-lookup-query staged-lookup-query i-f-fl
  by simp
qed
have comp-bs-eq:
  map fst fl = staged-composition-fri-challenges data
proof (rule nth-equalityI)
  show length (map fst fl) =
    length (staged-composition-fri-challenges data)
    using header-unique shape by (metis length-map)
next
fix i
assume i-bound: i < length (map fst fl)
then have i-fl: i < length fl
  by simp
have i-round:
  i < ceil-log (to-nat (staged-degree data) + 1)
  using i-fl header-unique shape by (metis length-map)
have staged-lookup-attacker:
  fmlookup (HashMap attacker-state)
    (CompositionFriChallenge i
      (foldl concat (staged-composition-fri-start-hash data)
        (take (Suc i) (staged-composition-fri-roots data)))) =

```

```

    Some (staged-composition-fri-challenges data ! i)
  by (rule staged-transcript-program-outcome-composition-challenge-lookup
      [OF wf controlled builder i-round])
have staged-lookup-s:
  fmlookup (HashMap ?s)
    (CompositionFriChallenge i
      (foldl concat (staged-composition-fri-start-hash data)
        (take (Suc i) (staged-composition-fri-roots data)))) =
    Some (staged-composition-fri-challenges data ! i)
  using staged-lookup-attacker by simp
have staged-lookup-query:
  fmlookup (HashMap query-state)
    (CompositionFriChallenge i
      (foldl concat (staged-composition-fri-start-hash data)
        (take (Suc i) (staged-composition-fri-roots data)))) =
    Some (staged-composition-fri-challenges data ! i)
  by (rule hash-extension-lookup[OF staged-lookup-s ext-s-query])
have verifier-lookup-query:
  fmlookup (HashMap query-state)
    (CompositionFriChallenge i
      (foldl concat (staged-composition-fri-start-hash data)
        (take (Suc i) (staged-composition-fri-roots data)))) =
    Some (fst (fl ! i))
  using comp-lookup[OF i-fl] header-unique
  unfolding staged-composition-fri-start-hash-def
    staged-trace-fri-start-hash-def
  by simp
show map fst fl ! i =
  staged-composition-fri-challenges data ! i
  using verifier-lookup-query staged-lookup-query i-fl
  by simp
qed
have trace-lookup-all:
   $\forall i < \text{length } f\text{-}fl.$ 
  fmlookup (HashMap query-state)
    (TraceFriChallenge (PTraceFriCounter ?s + i)
      (foldl concat (concat (PState ?s) fr)
        (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
  using trace-lookup by blast
have comp-lookup-all:
   $\forall i < \text{length } fl.$ 
  fmlookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter ?s + i)
      (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState ?s) fr) (map snd f-fl))

```

```

      f-final)
    as)
  dg)
  (take (Suc i) (map snd fl)))) =
  Some (fst (fl ! i))
using comp-lookup by blast
have challenges:
  accepted-fri-challenges ?s (Some (result, final-state))
  (staged-trace-fri-challenges data)
  (staged-degree data)
  (staged-composition-fri-challenges data)
unfolding accepted-fri-challenges-def
by (intro exI[of - result] exI[of - final-state] exI[of - fr]
    exI[of - f-fl] exI[of - f-final] exI[of - as] exI[of - fl]
    exI[of - final] exI[of - query-state])
  (use header query-state-eq query-out query-count-header
    header-unique trace-bs-eq comp-bs-eq trace-lookup-all comp-lookup-all
    in simp)
have challenges-out:
  accepted-fri-challenges ?s out
  (staged-trace-fri-challenges data)
  (staged-degree data)
  (staged-composition-fri-challenges data)
using challenges out-eq by simp
show ?thesis
  by (rule that[OF out-eq builder verifier challenges-out])
qed

```

lemma staged-budget-trace-length:
assumes staged-budget-wellformed budgets
shows length (trace-fri-budgets budgets) = ceil-log clength
using asms **unfolding** staged-budget-wellformed-def **by** simp

lemma staged-budget-composition-length:
assumes staged-budget-wellformed budgets
shows
 length (composition-fri-budgets budgets) =
 ceil-log (maxDegree + 1)
using asms **unfolding** staged-budget-wellformed-def **by** simp

lemma staged-budget-query-length:
assumes staged-budget-wellformed budgets
shows length (query-opening-budgets budgets) = rounds
using asms **unfolding** staged-budget-wellformed-def **by** simp

lemma staged-trace-fri-search-queries-le-total:
assumes wf: staged-budget-wellformed budgets
 and i-bound: $i < \text{ceil-log clength}$
shows

$staged\text{-}trace\text{-}fri\text{-}search\text{-}queries\ budgets\ i \leq$
 $staged\text{-}attacker\text{-}query\text{-}budget\ budgets + staged\text{-}challenge\text{-}query\text{-}budget$
proof –
have $take\text{-}le$:
 $sum\text{-}list\ (take\ (Suc\ i)\ (trace\text{-}fri\text{-}budgets\ budgets)) \leq$
 $sum\text{-}list\ (trace\text{-}fri\text{-}budgets\ budgets)$
by $(rule\ sum\text{-}list\text{-}take\text{-}le)$
have $i\text{-}le$: $i \leq ceil\text{-}log\ clength$
using $i\text{-}bound$ **by** $simp$
show $?thesis$
unfolding $staged\text{-}trace\text{-}fri\text{-}search\text{-}queries\text{-}def$
 $staged\text{-}attacker\text{-}query\text{-}budget\text{-}def\ staged\text{-}challenge\text{-}query\text{-}budget\text{-}def$
using $take\text{-}le\ i\text{-}le$ **by** $simp$
qed

lemma $staged\text{-}\alpha\text{-}search\text{-}queries\text{-}le\text{-}total$:
assumes $i \leq length\ spec$
shows
 $staged\text{-}\alpha\text{-}search\text{-}queries\ budgets\ i \leq$
 $staged\text{-}attacker\text{-}query\text{-}budget\ budgets + staged\text{-}challenge\text{-}query\text{-}budget$
unfolding $staged\text{-}\alpha\text{-}search\text{-}queries\text{-}def$
 $staged\text{-}attacker\text{-}query\text{-}budget\text{-}def\ staged\text{-}challenge\text{-}query\text{-}budget\text{-}def$
using $assms$ **by** $simp$

lemma $staged\text{-}composition\text{-}fri\text{-}search\text{-}queries\text{-}le\text{-}total$:
assumes wf : $staged\text{-}budget\text{-}wellformed\ budgets$
and $i\text{-}bound$: $i < ceil\text{-}log\ (maxDegree + 1)$
shows
 $staged\text{-}composition\text{-}fri\text{-}search\text{-}queries\ budgets\ i \leq$
 $staged\text{-}attacker\text{-}query\text{-}budget\ budgets + staged\text{-}challenge\text{-}query\text{-}budget$
proof –
have $take\text{-}le$:
 $sum\text{-}list\ (take\ (Suc\ i)\ (composition\text{-}fri\text{-}budgets\ budgets)) \leq$
 $sum\text{-}list\ (composition\text{-}fri\text{-}budgets\ budgets)$
by $(rule\ sum\text{-}list\text{-}take\text{-}le)$
have $i\text{-}le$: $i \leq ceil\text{-}log\ (maxDegree + 1)$
using $i\text{-}bound$ **by** $simp$
show $?thesis$
unfolding $staged\text{-}composition\text{-}fri\text{-}search\text{-}queries\text{-}def$
 $staged\text{-}attacker\text{-}query\text{-}budget\text{-}def\ staged\text{-}challenge\text{-}query\text{-}budget\text{-}def$
using $take\text{-}le\ i\text{-}le$ **by** $simp$
qed

lemma $staged\text{-}query\text{-}search\text{-}queries\text{-}le\text{-}total$:
assumes wf : $staged\text{-}budget\text{-}wellformed\ budgets$
and $i\text{-}bound$: $i < rounds$
shows
 $staged\text{-}query\text{-}search\text{-}queries\ budgets\ i \leq$
 $staged\text{-}attacker\text{-}query\text{-}budget\ budgets + staged\text{-}challenge\text{-}query\text{-}budget$

proof –
have *take-le*:
 $\text{sum-list } (\text{take } i \text{ (query-opening-budgets budgets)}) \leq$
 $\text{sum-list } (\text{query-opening-budgets budgets})$
by (rule *sum-list-take-le*)
have *i-le*: $i \leq \text{rounds}$
using *i-bound* **by** *simp*
show ?thesis
unfolding *staged-query-search-queries-def*
staged-attacker-query-budget-def *staged-challenge-query-budget-def*
using *take-le i-le* **by** *simp*
qed

lemma *staged-phase-relation-error-mono*:
assumes $q \leq r$
shows
 $\text{staged-phase-relation-error fiber-bound } q \leq$
 $\text{staged-phase-relation-error fiber-bound } r$
unfolding *staged-phase-relation-error-def*
by (rule *hash-relation-budget-value-mono*[OF *assms*])

lemma *staged-phase-target-error-mono*:
assumes $q \leq r$
shows $\text{staged-phase-target-error } B \ q \leq \text{staged-phase-target-error } B \ r$
unfolding *staged-phase-target-error-def*
by (rule *hash-target-budget-value-mono*[OF *assms*])

lemma *staged-phase-target-error-card-mono*:
assumes $\text{card } B \leq b$
shows $\text{staged-phase-target-error } B \ q \leq$
 $\text{staged-phase-relation-error } b \ q$
unfolding *staged-phase-target-error-def* *staged-phase-relation-error-def*
hash-target-budget-value-def *hash-relation-budget-value-def*
apply (subst *nn2real-le-iff*[symmetric])
using *assms* **by** (*simp add: divide-right-mono mult-left-mono*)

lemma *staged-dynamic-query-set-subset*:
 $\text{query-header-supported-union-good-sets}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $\text{query-sampling-success-space}$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $(\text{staged-alphas data})$
 $(\text{staged-degree data})$
 $(\text{staged-composition-fri-roots data})$
 $(\text{staged-composition-final data}) \subseteq \text{query-sample-space}$
by (rule *query-header-supported-union-good-sets-subset*)

lemma *staged-dynamic-query-set-fraction-bound-if-no-pairwise-merkle-bad*:
assumes *no-bad*:
 $\neg$ *query-supported-pairwise-merkle-bad*
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
shows
nnreal
(card
(query-header-supported-union-good-sets
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
query-sampling-success-space
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data))) /
nnreal (card query-sample-space) $\leq$ query-error-bound
by *(rule*
query-header-supported-union-good-sets-fraction-bound-if-no-global-pairwise-merkle-bad
[OF no-bad query-sampling-success-space-subset
query-sampling-success-space-fraction-bound-query-sample-space])

lemma *staged-dynamic-query-set-target-error-bound-if-no-pairwise-merkle-bad*:
assumes *raw-bound*: *query-index-raw-preimage-bound*
and *no-bad*:
 $\neg$ *query-supported-pairwise-merkle-bad*
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
shows
staged-phase-target-error
(query-index-raw-preimage
(query-header-supported-union-good-sets
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
query-sampling-success-space
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data))) $q \leq$
*nnreal q * query-error-bound*
by *(rule staged-phase-query-index-target-error-query-bound*
[OF raw-bound staged-dynamic-query-set-subset

staged-dynamic-query-set-fraction-bound-if-no-pairwise-merkle-bad
[OF no-bad]]])

lemma *staged-alpha-prefix-receive-bad-vector-position-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < length spec*

and *absent:*

alpha-challenge-values-absent

(fri-vector-position-values B i) s

shows

wp-event

(staged-alpha-challenge-prefix-program A i $\gg$
(λ -. receive-alpha-challenge))

(λ out. case out of None $\Rightarrow$ False

| Some (a, -) $\Rightarrow$ a $\in$ fri-vector-position-values B i) s $\leq$

staged-phase-target-error (fri-vector-position-values B i)

(Suc (staged-alpha-search-queries budgets i))

by *(rule staged-alpha-prefix-receive-bad-value-bound*

[OF wf controlled - absent])

(use i-bound in simp)

lemma *staged-alpha-prefix-receive-bad-vector-position-relation-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *subset: B $\subseteq$ fri-challenge-space (length spec)*

and *i-bound: i < length spec*

and *absent:*

alpha-challenge-values-absent

(fri-vector-position-values B i) s

shows

wp-event

(staged-alpha-challenge-prefix-program A i $\gg$
(λ -. receive-alpha-challenge))

(λ out. case out of None $\Rightarrow$ False

| Some (a, -) $\Rightarrow$ a $\in$ fri-vector-position-values B i) s $\leq$

staged-phase-relation-error (card B)

(Suc (staged-alpha-search-queries budgets i))

proof –

have *target:*

wp-event

(staged-alpha-challenge-prefix-program A i $\gg$
(λ -. receive-alpha-challenge))

(λ out. case out of None $\Rightarrow$ False

| Some (a, -) $\Rightarrow$ a $\in$ fri-vector-position-values B i) s $\leq$

staged-phase-target-error (fri-vector-position-values B i)

(Suc (staged-alpha-search-queries budgets i))

by *(rule staged-alpha-prefix-receive-bad-vector-position-bound*

[OF wf controlled i-bound absent])

also have ... $\leq$
 staged-phase-relation-error (*card B*)
 (*Suc* (*staged-alpha-search-queries budgets i*))
 by (*rule staged-phase-target-error-card-mono*)
 (*rule fri-vector-position-values-card-le*[*OF subset i-bound*])
 finally show ?thesis .
 qed

lemma *staged-alpha-prefix-receive-bad-vector-position-relation-bound-total*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *subset*: $B \subseteq \text{fri-challenge-space}$ (*length spec*)
and *i-bound*: $i < \text{length spec}$
and *absent*:
 alpha-challenge-values-absent
 (*fri-vector-position-values B i*) *s*
shows
 wp-event
 (*staged-alpha-challenge-prefix-program A i* $\gg$
 (λ -. *receive-alpha-challenge*))
 (λ *out*. *case out of None* $\Rightarrow$ *False*
 | *Some* (*a*, -) $\Rightarrow a \in \text{fri-vector-position-values B i}$) *s* $\leq$
 staged-phase-relation-error (*card B*)
 (*Suc*
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget))

proof –
have *local*:
 wp-event
 (*staged-alpha-challenge-prefix-program A i* $\gg$
 (λ -. *receive-alpha-challenge*))
 (λ *out*. *case out of None* $\Rightarrow$ *False*
 | *Some* (*a*, -) $\Rightarrow a \in \text{fri-vector-position-values B i}$) *s* $\leq$
 staged-phase-relation-error (*card B*)
 (*Suc* (*staged-alpha-search-queries budgets i*))
by (*rule staged-alpha-prefix-receive-bad-vector-position-relation-bound*
 [*OF wf controlled subset i-bound absent*])
also have
 staged-phase-relation-error (*card B*)
 (*Suc* (*staged-alpha-search-queries budgets i*)) $\leq$
 staged-phase-relation-error (*card B*)
 (*Suc*
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget))
by (*rule staged-phase-relation-error-mono*)
 (*simp add: staged-alpha-search-queries-le-total*[*OF less-imp-le*[*OF i-bound*]])
finally show ?thesis .
 qed

lemma *staged-alpha-prefix-receive-keep-prefix-bad-vector-position-relation-bound-total:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *subset: $B \subseteq \text{fri-challenge-space}$ (length spec)*

and *i-bound: $i < \text{length spec}$*

and *absent:*

alpha-challenge-values-absent

(fri-vector-position-values B i) s

shows

wp-event

(staged-alpha-challenge-prefix-program A i $\gg$

(λ prefix. receive-alpha-challenge $\gg$

(λa . return (prefix, a))))

(λ out. case out of None $\Rightarrow$ False

| Some ((-, a), -) $\Rightarrow a \in \text{fri-vector-position-values B i}$) s $\leq$

staged-phase-relation-error (card B)

(Suc

(staged-attacker-query-budget budgets +

staged-challenge-query-budget)))

proof –

let *?P =*

λ out. case out of None $\Rightarrow$ False

| Some (a, -) $\Rightarrow a \in \text{fri-vector-position-values B i}$

let *?Q =*

λ out. case out of None $\Rightarrow$ False

| Some ((-, a), -) $\Rightarrow a \in \text{fri-vector-position-values B i}$

have *cont-eq:*

$\bigwedge$ prefix t. wp-event

(receive-alpha-challenge $\gg$ (λa . return (prefix, a))) ?Q t =

wp-event receive-alpha-challenge ?P t

proof –

fix *prefix t*

show *wp-event*

(receive-alpha-challenge $\gg$ (λa . return (prefix, a))) ?Q t =

wp-event receive-alpha-challenge ?P t

unfolding *wp-event-def*

apply (*subst wp-bind*)

apply (*rule arg-cong[where*

f= λQ . wp receive-alpha-challenge Q t])

apply (*rule ext*)

apply (*simp add: wp-return split: option.splits prod.splits*)

done

qed

have *eq:*

wp-event

(staged-alpha-challenge-prefix-program A i $\gg$

(λ prefix. receive-alpha-challenge $\gg$

(λa . return (prefix, a)))) ?Q s =

wp-event

```

    (staged-alpha-challenge-prefix-program A i  $\gg$ 
      ( $\lambda$ -. receive-alpha-challenge)) ?P s
  by (rule wp-event-bind-cong-cont) (simp-all add: cont-eq)
show ?thesis
  unfolding eq
  by (rule staged-alpha-prefix-receive-bad-vector-position-relation-bound-total
      [OF wf controlled subset i-bound absent])
qed

```

lemma *staged-alpha-prefix-bad-trace-vector-bound:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *subset: $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$*
and *nonempty: $0 < \text{ceil-log } \text{clength}$*
and *absent:*
 $\bigwedge i. i < \text{ceil-log } \text{clength} \implies$
 $\text{trace-fri-challenge-values-absent}$
 $(\text{fri-vector-position-values } B \ i) \ s$

shows

```

wp-event (staged-alpha-prefix-program A)
  ( $\lambda$ out. case out of None  $\Rightarrow$  False
    | Some ((-, -, trace-bs, -), -)  $\Rightarrow$  trace-bs  $\in$  B) s  $\leq$ 
nnreal (ceil-log clength) *
  staged-phase-relation-error (card B)
  (staged-alpha-search-queries budgets 0)

```

proof –

```

let ?coord =
 $\lambda i$  out. case out of None  $\Rightarrow$  False
  | Some ((-, -, trace-bs, -), -)  $\Rightarrow$ 
    trace-bs !  $i \in \text{fri-vector-position-values } B \ i$ 

```

have *cover:*

```

wp-event (staged-alpha-prefix-program A)
  ( $\lambda$ out. case out of None  $\Rightarrow$  False
    | Some ((-, -, trace-bs, -), -)  $\Rightarrow$  trace-bs  $\in$  B) s  $\leq$ 
  ( $\sum i < \text{ceil-log } \text{clength}.$ 
    wp-event (staged-alpha-prefix-program A) (?coord i) s)
by (rule staged-alpha-prefix-bad-trace-vector-coordinate-cover-bound
    [OF subset nonempty])

```

also have $\dots \leq$

```

  ( $\sum i < \text{ceil-log } \text{clength}.$ 
    staged-phase-relation-error (card B)
    (staged-alpha-search-queries budgets 0))

```

proof (rule sum-mono)

fix i

assume $i\text{-in}: i \in \{.. < \text{ceil-log } \text{clength}\}$

then have $i\text{-bound}: i < \text{ceil-log } \text{clength}$

by *simp*

have *target:*

```

wp-event (staged-alpha-prefix-program A) (?coord i) s  $\leq$ 

```

```

    staged-phase-target-error (fri-vector-position-values B i)
      (staged-alpha-search-queries budgets 0)
  by (rule staged-alpha-prefix-trace-coordinate-bound
      [OF wf controlled i-bound absent[OF i-bound]])
  also have ... ≤
    staged-phase-relation-error (card B)
      (staged-alpha-search-queries budgets 0)
  by (rule staged-phase-target-error-card-mono)
      (rule fri-vector-position-values-card-le[OF subset i-bound])
  finally show
    wp-event (staged-alpha-prefix-program A) (?coord i) s ≤
      staged-phase-relation-error (card B)
        (staged-alpha-search-queries budgets 0) .
qed
also have ... =
  nnreal (ceil-log clength) *
    staged-phase-relation-error (card B)
      (staged-alpha-search-queries budgets 0)
  by simp
  finally show ?thesis .
qed

lemma staged-transcript-program-trace-fri-vector-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and subset: B ⊆ fri-challenge-space (ceil-log clength)
    and nonempty: 0 < ceil-log clength
    and absent:
      ∧ i. i < ceil-log clength ⇒
        trace-fri-challenge-values-absent
          (fri-vector-position-values B i) adversary-initial-state
  shows
    wp-event (staged-transcript-program A)
      (staged-transcript-trace-fri-vector-hit B)
      adversary-initial-state ≤
        nnreal (ceil-log clength) *
          staged-phase-relation-error (card B)
            (staged-alpha-search-queries budgets 0)
proof -
  let ?Head =
    λout. case out of
      None ⇒ False
    | Some ((-, -, trace-bs, -), -) ⇒ trace-bs ∈ B
  have prefix-bound:
    wp-event (staged-alpha-prefix-program A) ?Head
      adversary-initial-state ≤
        nnreal (ceil-log clength) *
          staged-phase-relation-error (card B)
            (staged-alpha-search-queries budgets 0)

```

```

    by (rule staged-alpha-prefix-bad-trace-vector-bound
        [OF wf controlled subset nonempty absent])
  show ?thesis
    unfolding staged-transcript-program-alpha-prefix-decomp
  proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
    show staged-transcript-trace-fri-vector-hit B None  $\implies$ 
      ?Head None
    unfolding staged-transcript-trace-fri-vector-hit-def by simp
  next
    fix x t out
    assume cont:
      out  $\in$ 
        set-dist
          (execute (staged-after-alpha-prefix-program A x) t)
    and hit: staged-transcript-trace-fri-vector-hit B out
    obtain fr trace-roots trace-bs trace-final where x-eq:
      x = (fr, trace-roots, trace-bs, trace-final)
    by (cases x) auto
    have trace-bs-in: trace-bs  $\in$  B
    by (rule staged-after-alpha-prefix-program-trace-fri-vector-hitD
        [OF cont[unfolded x-eq] hit])
    show ?Head (Some (x, t))
    unfolding x-eq
    using trace-bs-in by simp
  qed
qed

```

lemma *staged-security-with-data-trace-fri-vector-bound:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *subset: $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$*
and *nonempty: $0 < \text{ceil-log } \text{clength}$*
and *absent:*
 $\bigwedge i. i < \text{ceil-log } \text{clength} \implies$
 $\text{trace-fri-challenge-values-absent}$
 $(\text{fri-vector-position-values } B \ i) \text{ adversary-initial-state}$

shows

wp-event (staged-security-experiment-with-data A)
(staged-security-with-data-trace-fri-vector-hit B)
adversary-initial-state $\leq$
*nnreal (ceil-log clength) **
staged-phase-relation-error (card B)
(staged-alpha-search-queries budgets 0)

proof –

have *transcript-bound:*

wp-event (staged-transcript-program A)
(staged-transcript-trace-fri-vector-hit B)
adversary-initial-state $\leq$
*nnreal (ceil-log clength) **


```

    staged-phase-relation-error (card B)
    (staged-alpha-search-queries budgets 0)
  by (rule staged-transcript-program-trace-fri-vector-bound
    [OF wf controlled subset nonempty absent])
show ?thesis
  unfolding staged-security-experiment-with-data-def
proof (rule wp-event-bind-bound-by-head-event[OF transcript-bound])
  show staged-security-with-data-trace-fri-vector-hit B None ==>
    staged-transcript-trace-fri-vector-hit B None
  unfolding staged-security-with-data-trace-fri-vector-hit-def
    staged-transcript-trace-fri-vector-hit-def
  by simp
next
fix data attacker-state out
assume cont:
  out ∈
    set-dist
    (execute
      (get >>=
        (λs. put
          (verifier-state-from-adversary s
            (staged-proof-transcript data)) >>=
            (λ-. verify-monad >>=
              (λresult. return (data, result))))))
      attacker-state)
  and hit: staged-security-with-data-trace-fri-vector-hit B out
have data-bad: staged-trace-fri-challenges data ∈ B
proof (cases out)
  case None
  then show ?thesis
    using hit unfolding staged-security-with-data-trace-fri-vector-hit-def
    by simp
next
case (Some result)
then obtain data' result' final-state where out-eq:
  out = Some ((data', result'), final-state)
  by (cases result, auto split: prod.splits)
from cont[unfolded out-eq]
obtain s1 s2 s3 where
  ret:
    Some ((data', result'), final-state) ∈
      set-dist (execute (return (data, result')) s3)
  by (auto elim!: set-dist-bindE)
have data'-eq: data' = data
  using ret by simp
show ?thesis
  using hit unfolding out-eq data'-eq
    staged-security-with-data-trace-fri-vector-hit-def
  by simp

```

```

qed
show staged-transcript-trace-fri-vector-hit B
  (Some (data, attacker-state))
  using data-bad unfolding staged-transcript-trace-fri-vector-hit-def
  by simp
qed
qed

lemma staged-security-with-data-composition-fri-vector-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and subset:
     $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$ 
  and absent:
     $\bigwedge i. i < \text{ceil-log } (\text{maxDegree} + 1) \implies$ 
      composition-fri-challenge-values-absent
       $(\bigcup dg. \text{fri-vector-position-values } (B \ dg) \ i)$ 
      adversary-initial-state
  shows
    wp-event (staged-security-experiment-with-data A)
      (staged-security-with-data-composition-fri-vector-guarded-hit B)
      adversary-initial-state  $\leq$ 
       $(\sum i < \text{ceil-log } (\text{maxDegree} + 1).$ 
        staged-phase-target-error
         $(\bigcup dg. \text{fri-vector-position-values } (B \ dg) \ i)$ 
        (staged-composition-fri-vector-search-queries budgets))
  proof -
    have transcript-bound:
      wp-event (staged-transcript-program A)
        (staged-transcript-composition-fri-vector-guarded-hit B)
        adversary-initial-state  $\leq$ 
         $(\sum i < \text{ceil-log } (\text{maxDegree} + 1).$ 
          staged-phase-target-error
           $(\bigcup dg. \text{fri-vector-position-values } (B \ dg) \ i)$ 
          (staged-composition-fri-vector-search-queries budgets))
      by (rule staged-transcript-program-composition-fri-vector-bound
        [OF wf controlled subset absent])
    show ?thesis
      unfolding staged-security-experiment-with-data-def
    proof (rule wp-event-bind-bound-by-head-event[OF transcript-bound])
      show
        staged-security-with-data-composition-fri-vector-guarded-hit B None
           $\implies$  staged-transcript-composition-fri-vector-guarded-hit B None
      unfolding
        staged-security-with-data-composition-fri-vector-guarded-hit-def
        staged-transcript-composition-fri-vector-guarded-hit-def
      by simp
    next
      fix data attacker-state out

```

```

assume cont:
  out ∈
    set-dist
      (execute
        (get  $\gg$ 
          ( $\lambda s.$  put
            (verifier-state-from-adversary s
              (staged-proof-transcript data))  $\gg$ 
            ( $\lambda -.$  verify-monad  $\gg$ 
              ( $\lambda result.$  return (data, result))))))
          attacker-state)
    and hit:
      staged-security-with-data-composition-fri-vector-guarded-hit B out
have data-bad:
  staged-composition-fri-challenges data ∈
    B (staged-degree data) ∧
     $0 < \text{ceil-log } (\text{to-nat } (\text{staged-degree } \text{data}) + 1) \wedge$ 
     $\text{ceil-log } (\text{to-nat } (\text{staged-degree } \text{data}) + 1) \leq$ 
     $\text{ceil-log } (\text{maxDegree} + 1)$ 
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      staged-security-with-data-composition-fri-vector-guarded-hit-def
    by simp
next
  case (Some result)
  then obtain data' result' final-state where out-eq:
    out = Some ((data', result'), final-state)
    by (cases result, auto split: prod.splits)
  from cont[unfolded out-eq]
  obtain s1 s2 s3 where
    ret:
      Some ((data', result'), final-state) ∈
        set-dist (execute (return (data, result')) s3)
    by (auto elim!: set-dist-bindE)
  have data'-eq: data' = data
    using ret by simp
  show ?thesis
    using hit unfolding out-eq data'-eq
      staged-security-with-data-composition-fri-vector-guarded-hit-def
    by simp
qed
show staged-transcript-composition-fri-vector-guarded-hit B
  (Some (data, attacker-state))
using data-bad
unfolding staged-transcript-composition-fri-vector-guarded-hit-def
by simp

```

qed
qed

lemma *staged-trace-fri-prefix-receive-bad-value-bound-total:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < ceil-log clength*
and *absent: trace-fri-challenge-values-absent B s*

shows

wp-event
 (*staged-trace-fri-challenge-prefix-program A i* $\gg$
 (λ -. *receive-trace-fri-challenge*))
 (λ *out. case out of None* $\Rightarrow$ *False*
 | *Some (b, -)* $\Rightarrow$ *b* $\in$ *B*) *s* $\leq$
staged-phase-target-error B
 (*Suc*
 (*staged-attacker-query-budget budgets* +
staged-challenge-query-budget))

proof –

have *i-len: i < length (trace-fri-budgets budgets)*
using *wf i-bound* **unfolding** *staged-budget-wellformed-def* **by** *simp*

have *local:*

wp-event
 (*staged-trace-fri-challenge-prefix-program A i* $\gg$
 (λ -. *receive-trace-fri-challenge*))
 (λ *out. case out of None* $\Rightarrow$ *False*
 | *Some (b, -)* $\Rightarrow$ *b* $\in$ *B*) *s* $\leq$
staged-phase-target-error B
 (*Suc* (*staged-trace-fri-search-queries budgets i*))
by (*rule staged-trace-fri-prefix-receive-bad-value-bound*
 [*OF controlled i-len absent*])

also have

staged-phase-target-error B
 (*Suc* (*staged-trace-fri-search-queries budgets i*)) $\leq$
staged-phase-target-error B
 (*Suc*
 (*staged-attacker-query-budget budgets* +
staged-challenge-query-budget))
by (*rule staged-phase-target-error-mono*)
 (*simp add: staged-trace-fri-search-queries-le-total [OF wf i-bound]*)

finally show *?thesis* .

qed

lemma *staged-trace-fri-prefix-receive-bad-vector-position-bound-total:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < ceil-log clength*
and *absent:*
trace-fri-challenge-values-absent

$(\text{fri-vector-position-values } B \ i) \ s$
shows
 wp-event
 $(\text{staged-trace-fri-challenge-prefix-program } A \ i \gg=$
 $(\lambda-. \text{receive-trace-fri-challenge}))$
 $(\lambda \text{out. case out of None} \Rightarrow \text{False}$
 $\mid \text{Some } (b, -) \Rightarrow b \in \text{fri-vector-position-values } B \ i) \ s \leq$
 $\text{staged-phase-target-error } (\text{fri-vector-position-values } B \ i)$
 $(\text{Suc}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}))$
by $(\text{rule staged-trace-fri-prefix-receive-bad-value-bound-total}$
 $[\text{OF wf controlled i-bound absent}])$

lemma *staged-trace-fri-prefix-receive-bad-vector-position-relation-bound:*

assumes $\text{wf: staged-budget-wellformed budgets}$
and $\text{controlled: staged-adversary-controlled budgets } A$
and $\text{subset: } B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$
and $\text{i-bound: } i < \text{ceil-log clength}$
and absent:
 $\text{trace-fri-challenge-values-absent}$
 $(\text{fri-vector-position-values } B \ i) \ s$

shows
 wp-event
 $(\text{staged-trace-fri-challenge-prefix-program } A \ i \gg=$
 $(\lambda-. \text{receive-trace-fri-challenge}))$
 $(\lambda \text{out. case out of None} \Rightarrow \text{False}$
 $\mid \text{Some } (b, -) \Rightarrow b \in \text{fri-vector-position-values } B \ i) \ s \leq$
 $\text{staged-phase-relation-error } (\text{card } B)$
 $(\text{Suc}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}))$

proof –

have target:
 wp-event
 $(\text{staged-trace-fri-challenge-prefix-program } A \ i \gg=$
 $(\lambda-. \text{receive-trace-fri-challenge}))$
 $(\lambda \text{out. case out of None} \Rightarrow \text{False}$
 $\mid \text{Some } (b, -) \Rightarrow b \in \text{fri-vector-position-values } B \ i) \ s \leq$
 $\text{staged-phase-target-error } (\text{fri-vector-position-values } B \ i)$
 $(\text{Suc}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}))$
by $(\text{rule staged-trace-fri-prefix-receive-bad-vector-position-bound-total}$
 $[\text{OF wf controlled i-bound absent}])$
also have $\dots \leq$
 $\text{staged-phase-relation-error } (\text{card } B)$
 $(\text{Suc}$
 $(\text{staged-attacker-query-budget budgets} +$

```

    staged-challenge-query-budget))
  by (rule staged-phase-target-error-card-mono)
    (rule fri-vector-position-values-card-le[OF subset i-bound])
  finally show ?thesis .
qed

lemma staged-trace-fri-prefix-receive-keep-prefix-bad-vector-position-relation-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and subset:  $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$ 
    and i-bound:  $i < \text{ceil-log } \text{clength}$ 
    and absent:
      trace-fri-challenge-values-absent
      (fri-vector-position-values B i) s
  shows
    wp-event
      (staged-trace-fri-challenge-prefix-program A i  $\gg$ 
        ( $\lambda \text{prefix. receive-trace-fri-challenge } \gg$ 
          ( $\lambda b. \text{return } (\text{prefix}, b)$ ))))
      ( $\lambda \text{out. case out of None } \Rightarrow \text{False}$ 
        |  $\text{Some } ((-, b), -) \Rightarrow b \in \text{fri-vector-position-values } B \text{ i}$ ) s  $\leq$ 
        staged-phase-relation-error (card B)
        (Suc
          (staged-attacker-query-budget budgets +
            staged-challenge-query-budget)))
  proof -
    let ?P =
       $\lambda \text{out. case out of None } \Rightarrow \text{False}$ 
      |  $\text{Some } (b, -) \Rightarrow b \in \text{fri-vector-position-values } B \text{ i}$ 
    let ?Q =
       $\lambda \text{out. case out of None } \Rightarrow \text{False}$ 
      |  $\text{Some } ((-, b), -) \Rightarrow b \in \text{fri-vector-position-values } B \text{ i}$ 
    have cont-eq:
       $\bigwedge \text{prefix } t. \text{wp-event}$ 
      (receive-trace-fri-challenge  $\gg$  ( $\lambda b. \text{return } (\text{prefix}, b)$ )) ?Q t =
      wp-event receive-trace-fri-challenge ?P t
    proof -
      fix prefix t
      show wp-event
        (receive-trace-fri-challenge  $\gg$  ( $\lambda b. \text{return } (\text{prefix}, b)$ )) ?Q t =
        wp-event receive-trace-fri-challenge ?P t
      unfolding wp-event-def
      apply (subst wp-bind)
      apply (rule arg-cong[where
        f= $\lambda Q. \text{wp receive-trace-fri-challenge } Q \text{ t}$ ])
      apply (rule ext)
      apply (simp add: wp-return split: option.splits prod.splits)
      done
    qed
  qed

```

have *eq*:

$$\begin{aligned} & \text{wp-event} \\ & \quad (\text{staged-trace-fri-challenge-prefix-program } A \ i \gg= \\ & \quad \quad (\lambda \text{prefix. receive-trace-fri-challenge } \gg= \\ & \quad \quad \quad (\lambda b. \text{return } (\text{prefix}, b)))) \ ?Q \ s = \\ & \text{wp-event} \\ & \quad (\text{staged-trace-fri-challenge-prefix-program } A \ i \gg= \\ & \quad \quad (\lambda \cdot. \text{receive-trace-fri-challenge})) \ ?P \ s \\ & \text{by (rule wp-event-bind-cong-cont) (simp-all add: cont-eq)} \\ & \text{show ?thesis} \\ & \text{unfolding eq} \\ & \text{by (rule staged-trace-fri-prefix-receive-bad-vector-position-relation-bound} \\ & \quad [OF wf controlled subset i-bound absent])} \\ & \text{qed}
\end{aligned}$$

lemma *staged-trace-fri-bad-vector-coordinate-sum-bound*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *subset*: $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$
and *absent*:

$$\bigwedge i. i < \text{ceil-log } \text{clength} \implies$$

$$\text{trace-fri-challenge-values-absent}$$

$$(\text{fri-vector-position-values } B \ i) \ s$$
shows

$$(\sum i < \text{ceil-log } \text{clength.}$$

$$\text{wp-event}$$

$$\quad (\text{staged-trace-fri-challenge-prefix-program } A \ i \gg=$$

$$\quad \quad (\lambda \cdot. \text{receive-trace-fri-challenge}))$$

$$\quad (\lambda \text{out. case out of None } \Rightarrow \text{False}$$

$$\quad \quad | \text{Some } (b, -) \Rightarrow b \in \text{fri-vector-position-values } B \ i) \ s) \leq$$

$$\text{nnreal } (\text{ceil-log } \text{clength}) \ *$$

$$\text{staged-phase-relation-error } (\text{card } B)$$

$$\quad (\text{Suc}$$

$$\quad \quad (\text{staged-attacker-query-budget budgets } +$$

$$\quad \quad \text{staged-challenge-query-budget}))$$

proof –
have

$$(\sum i < \text{ceil-log } \text{clength.}$$

$$\text{wp-event}$$

$$\quad (\text{staged-trace-fri-challenge-prefix-program } A \ i \gg=$$

$$\quad \quad (\lambda \cdot. \text{receive-trace-fri-challenge}))$$

$$\quad (\lambda \text{out. case out of None } \Rightarrow \text{False}$$

$$\quad \quad | \text{Some } (b, -) \Rightarrow b \in \text{fri-vector-position-values } B \ i) \ s) \leq$$

$$(\sum i < \text{ceil-log } \text{clength.}$$

$$\quad \text{staged-phase-relation-error } (\text{card } B)$$

$$\quad \quad (\text{Suc}$$

$$\quad \quad \quad (\text{staged-attacker-query-budget budgets } +$$

$$\quad \quad \quad \text{staged-challenge-query-budget})))$$
by (rule sum-mono)

```

(rule staged-trace-fri-prefix-receive-bad-vector-position-relation-bound
  [OF wf controlled subset], simp-all add: absent)
also have ... =
  nnreal (ceil-log clength) *
    staged-phase-relation-error (card B)
    (Suc
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget))
  by simp
finally show ?thesis .
qed

```

lemma *staged-trace-fri-bad-vector-coordinate-sum-keep-prefix-bound:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *subset: $B \subseteq \text{fri-challenge-space (ceil-log clength)}$*
and *absent:*

$\bigwedge i. i < \text{ceil-log clength} \implies$
trace-fri-challenge-values-absent
(fri-vector-position-values B i) s

shows

$(\sum i < \text{ceil-log clength}.$
wp-event
(staged-trace-fri-challenge-prefix-program A i $\gg$
($\lambda \text{prefix. receive-trace-fri-challenge $\gg$$
($\lambda b. \text{return (prefix, b)}$))))
($\lambda \text{out. case out of None} \Rightarrow \text{False}$
| $\text{Some ((-, b), -)} \Rightarrow$
 $b \in \text{fri-vector-position-values B i} \text{) } s \leq$
*nnreal (ceil-log clength) **
staged-phase-relation-error (card B)
(Suc
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)))

proof –

have

$(\sum i < \text{ceil-log clength}.$
wp-event
(staged-trace-fri-challenge-prefix-program A i $\gg$
($\lambda \text{prefix. receive-trace-fri-challenge $\gg$$
($\lambda b. \text{return (prefix, b)}$))))
($\lambda \text{out. case out of None} \Rightarrow \text{False}$
| $\text{Some ((-, b), -)} \Rightarrow$
 $b \in \text{fri-vector-position-values B i} \text{) } s \leq$
 $(\sum i < \text{ceil-log clength}.$
staged-phase-relation-error (card B)
(Suc
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)))


```

by (rule sum-mono)
  (rule
    staged-trace-fri-prefix-keep-prefix-bad-vector-position-relation-bound
    [OF wf controlled subset], simp-all add: absent)
also have ... =
  nnreal (ceil-log clength) *
  staged-phase-relation-error (card B)
  (Suc
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget))
by simp
finally show ?thesis .
qed

```

lemma *staged-query-prefix-bad-index-error-mono*:

assumes *i-bound*: $i < \text{rounds}$
and *wf*: *staged-budget-wellformed budgets*
shows
 $\text{staged-query-prefix-bad-index-error budgets } B \ i \leq$
 $\text{staged-phase-target-error (query-index-raw-preimage } B)$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
unfolding *staged-query-prefix-bad-index-error-def*
by (rule *staged-phase-target-error-mono*)
(rule *staged-query-search-queries-le-total*[OF *wf i-bound*])

lemma *staged-query-prefix-bad-index-target-bound-total*:

assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < \text{rounds}$
shows
 $\text{wp-event (staged-query-challenge-prefix-program } A \ i)$
 $(\text{hash-new-output-hit-event (query-index-raw-preimage } B) \ s) \ s \leq$
 $\text{staged-phase-target-error (query-index-raw-preimage } B)$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$

proof –
have *local*:
 $\text{wp-event (staged-query-challenge-prefix-program } A \ i)$
 $(\text{hash-new-output-hit-event (query-index-raw-preimage } B) \ s) \ s \leq$
 $\text{staged-query-prefix-bad-index-error budgets } B \ i$
by (rule *staged-query-prefix-bad-index-target-bound*)
(use *wf controlled i-bound* **in** *simp-all*)
also have
 $\text{staged-query-prefix-bad-index-error budgets } B \ i \leq$
 $\text{staged-phase-target-error (query-index-raw-preimage } B)$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
by (rule *staged-query-prefix-bad-index-error-mono*[OF *i-bound wf*])

finally show ?thesis .
qed

lemma *checked-staged-query-prefix-bad-index-target-bound-total:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

shows

wp-event (checked-staged-query-challenge-prefix-program A i)
(hash-new-output-hit-event (query-index-raw-preimage B) s) s ≤
staged-phase-target-error (query-index-raw-preimage B)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)

proof –

have *local:*

wp-event (checked-staged-query-challenge-prefix-program A i)
(hash-new-output-hit-event (query-index-raw-preimage B) s) s ≤
staged-query-prefix-bad-index-error budgets B i
by (rule *checked-staged-query-prefix-bad-index-target-bound*)
 (use *wf controlled i-bound* **in** *simp-all*)

also have

staged-query-prefix-bad-index-error budgets B i ≤
staged-phase-target-error (query-index-raw-preimage B)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)
by (rule *staged-query-prefix-bad-index-error-mono[OF i-bound wf]*)

finally show ?thesis .

qed

lemma *staged-query-prefix-receive-bad-index-bound-total:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *absent: query-index-raw-preimage-absent B s*

shows

wp-event
(staged-query-challenge-prefix-program A i ≫=
(λ-. receive-query-index-challenge))
(λout. case out of None ⇒ False
| Some (raw, -) ⇒ index (to-nat raw) ∈ B) s ≤
staged-phase-target-error (query-index-raw-preimage B)
(Suc
(staged-attacker-query-budget budgets +
staged-challenge-query-budget))

proof –

have *local:*

wp-event
(staged-query-challenge-prefix-program A i ≫=
(λ-. receive-query-index-challenge))

($\lambda out. case out of None \Rightarrow False$
 | $Some (raw, -) \Rightarrow index (to-nat\ raw) \in B$) $s \leq$
 staged-phase-target-error (query-index-raw-preimage B)
 (Suc (staged-query-search-queries budgets i))
by (rule staged-query-prefix-receive-bad-index-bound)
 (use wf controlled i-bound absent **in** simp-all)
also have
 staged-phase-target-error (query-index-raw-preimage B)
 (Suc (staged-query-search-queries budgets i)) $\leq$
 staged-phase-target-error (query-index-raw-preimage B)
 (Suc
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget))
by (rule staged-phase-target-error-mono)
 (simp add: staged-query-search-queries-le-total[OF wf i-bound])
finally show ?thesis .
qed

lemma staged-query-prefix-receive-bad-index-query-bound-total:

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and i-bound: $i < rounds$

and absent: query-index-raw-preimage-absent $B\ s$

and raw-bound: query-index-raw-preimage-bound

and subset: $B \subseteq query-sample-space$

and frac:

$nnreal (card\ B) / nnreal (card\ query-sample-space) \leq C$

shows

wp-event

(staged-query-challenge-prefix-program $A\ i \gg$
 ($\lambda -. receive-query-index-challenge$))

($\lambda out. case out of None \Rightarrow False$
 | $Some (raw, -) \Rightarrow index (to-nat\ raw) \in B$) $s \leq$

nnreal

(Suc
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) * C

proof –

have local:

wp-event

(staged-query-challenge-prefix-program $A\ i \gg$
 ($\lambda -. receive-query-index-challenge$))

($\lambda out. case out of None \Rightarrow False$
 | $Some (raw, -) \Rightarrow index (to-nat\ raw) \in B$) $s \leq$

staged-phase-target-error (query-index-raw-preimage B)
 (Suc

(staged-attacker-query-budget budgets +
 staged-challenge-query-budget))

by (rule staged-query-prefix-receive-bad-index-bound-total

```

    [OF wf controlled i-bound absent])
also have ... ≤
  nnreal
  (Suc
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)) * C
  by (rule staged-phase-query-index-target-error-query-bound
    [OF raw-bound subset frac])
  finally show ?thesis .
qed

lemma staged-query-prefix-recv-header-supported-sampling-bound-total:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and absent:
  query-index-raw-preimage-absent
  (query-header-supported-union-good-sets verifier-state
   query-sampling-success-space fr f-fri-roots f-final as dg
   composition-fri-roots final)
  prefix-state
and raw-bound: query-index-raw-preimage-bound
and no-bad: ¬ query-supported-pairwise-merkle-bad verifier-state
shows
  wp-event
  (staged-query-challenge-prefix-program A i ≫
   (λ-. receive-query-index-challenge))
  (λout. case out of None ⇒ False
   | Some (raw, -) ⇒
     index (to-nat raw) ∈
     query-header-supported-union-good-sets verifier-state
     query-sampling-success-space fr f-fri-roots f-final as dg
     composition-fri-roots final)
  prefix-state ≤
  nnreal
  (Suc
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)) * query-error-bound

proof –
let ?B =
  query-header-supported-union-good-sets verifier-state
  query-sampling-success-space fr f-fri-roots f-final as dg
  composition-fri-roots final
have subset: ?B ⊆ query-sample-space
by (rule query-header-supported-union-good-sets-subset)
have frac:
  nnreal (card ?B) / nnreal (card query-sample-space) ≤
  query-error-bound
by (rule

```

```

    query-header-supported-union-good-sets-fraction-bound-if-no-global-pairwise-merkle-bad
    [OF no-bad query-sampling-success-space-subset
     query-sampling-success-space-fraction-bound-query-sample-space])
  show ?thesis
  by (rule staged-query-prefix-receive-bad-index-query-bound-total
      [OF wf controlled i-bound absent raw-bound subset frac])
qed

lemma staged-composition-fri-prefix-receive-bad-value-bound-total:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{ceil-log } (\text{maxDegree} + 1)$ 
  and absent: composition-fri-challenge-values-absent B s
  shows
    wp-event
    (staged-composition-fri-challenge-prefix-program A i  $\gg=$ 
     ( $\lambda$ -. receive-composition-fri-challenge))
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
     | Some (b, -)  $\Rightarrow$   $b \in B$ ) s  $\leq$ 
    staged-phase-target-error B
    (Suc
     (staged-attacker-query-budget budgets +
      staged-challenge-query-budget))
  proof -
    have local:
      wp-event
      (staged-composition-fri-challenge-prefix-program A i  $\gg=$ 
       ( $\lambda$ -. receive-composition-fri-challenge))
      ( $\lambda$ out. case out of None  $\Rightarrow$  False
       | Some (b, -)  $\Rightarrow$   $b \in B$ ) s  $\leq$ 
      staged-phase-target-error B
      (Suc (staged-composition-fri-search-queries budgets i))
    by (rule staged-composition-fri-prefix-receive-bad-value-bound
        [OF wf controlled i-bound absent])
    also have
      staged-phase-target-error B
      (Suc (staged-composition-fri-search-queries budgets i))  $\leq$ 
      staged-phase-target-error B
      (Suc
       (staged-attacker-query-budget budgets +
        staged-challenge-query-budget))
    by (rule staged-phase-target-error-mono)
      (simp add: staged-composition-fri-search-queries-le-total
        [OF wf i-bound])
    finally show ?thesis .
  qed

lemma staged-composition-fri-prefix-receive-bad-vector-position-bound-total:
  assumes wf: staged-budget-wellformed budgets

```

and *controlled*: staged-adversary-controlled budgets A
and *i-bound*: $i < \text{ceil-log } (\text{maxDegree} + 1)$
and *absent*:
 composition-fri-challenge-values-absent
 (*fri-vector-position-values* B i) s
shows
 wp-event
 (*staged-composition-fri-challenge-prefix-program* A $i \gg=$
 ($\lambda\text{-}$. *receive-composition-fri-challenge*))
 ($\lambda\text{out. case out of None} \Rightarrow \text{False}$
 | $\text{Some } (b, -) \Rightarrow b \in \text{fri-vector-position-values } B$ i) $s \leq$
 staged-phase-target-error (*fri-vector-position-values* B i)
 (*Suc*
 (*staged-attacker-query-budget* budgets +
 staged-challenge-query-budget))
by (*rule staged-composition-fri-prefix-receive-bad-value-bound-total*
 [*OF wf controlled i-bound absent*])

lemma

staged-composition-fri-prefix-receive-bad-vector-position-relation-bound:

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets A

and *subset*:

$B \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$

and *i-bound*: $i < \text{ceil-log } (\text{to-nat } dg + 1)$

and *dg-bound*: $\text{ceil-log } (\text{to-nat } dg + 1) \leq \text{ceil-log } (\text{maxDegree} + 1)$

and *absent*:

composition-fri-challenge-values-absent

 (*fri-vector-position-values* B i) s

shows

wp-event
 (*staged-composition-fri-challenge-prefix-program* A $i \gg=$
 ($\lambda\text{-}$. *receive-composition-fri-challenge*))
 ($\lambda\text{out. case out of None} \Rightarrow \text{False}$
 | $\text{Some } (b, -) \Rightarrow b \in \text{fri-vector-position-values } B$ i) $s \leq$
staged-phase-relation-error (*card* B)
 (*Suc*
 (*staged-attacker-query-budget* budgets +
staged-challenge-query-budget))

proof –

have *i-max*: $i < \text{ceil-log } (\text{maxDegree} + 1)$

using *i-bound dg-bound* **by** *simp*

have *target*:

wp-event
 (*staged-composition-fri-challenge-prefix-program* A $i \gg=$
 ($\lambda\text{-}$. *receive-composition-fri-challenge*))
 ($\lambda\text{out. case out of None} \Rightarrow \text{False}$
 | $\text{Some } (b, -) \Rightarrow b \in \text{fri-vector-position-values } B$ i) $s \leq$
staged-phase-target-error (*fri-vector-position-values* B i)

```

    (Suc
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget))
  by (rule
      staged-composition-fri-prefix-receive-bad-vector-position-bound-total
      [OF wf controlled i-max absent])
also have ... ≤
  staged-phase-relation-error (card B)
  (Suc
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget))
  by (rule staged-phase-target-error-card-mono)
      (rule fri-vector-position-values-card-le[OF subset i-bound])
finally show ?thesis .
qed

lemma
  staged-composition-fri-prefix-receive-keep-prefix-bad-vector-position-relation-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and subset:
    B ⊆ fri-challenge-space (ceil-log (to-nat dg + 1))
  and i-bound: i < ceil-log (to-nat dg + 1)
  and dg-bound: ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
  and absent:
    composition-fri-challenge-values-absent
    (fri-vector-position-values B i) s
  shows
    wp-event
    (staged-composition-fri-challenge-prefix-program A i ≫=
     (λprefix. receive-composition-fri-challenge ≫=
      (λb. return (prefix, b))))
    (λout. case out of None ⇒ False
      | Some ((-, b), -) ⇒ b ∈ fri-vector-position-values B i) s ≤
    staged-phase-relation-error (card B)
    (Suc
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget))

proof -
  let ?P =
    λout. case out of None ⇒ False
    | Some (b, -) ⇒ b ∈ fri-vector-position-values B i
  let ?Q =
    λout. case out of None ⇒ False
    | Some ((-, b), -) ⇒ b ∈ fri-vector-position-values B i
  have cont-eq:
    ∧prefix t. wp-event
    (receive-composition-fri-challenge ≫=
     (λb. return (prefix, b))) ?Q t =

```

```

    wp-event receive-composition-fri-challenge ?P t
proof -
  fix prefix t
  show wp-event
    (receive-composition-fri-challenge  $\gg$ 
      ( $\lambda b.$  return (prefix, b))) ?Q t =
    wp-event receive-composition-fri-challenge ?P t
  unfolding wp-event-def
  apply (subst wp-bind)
  apply (rule arg-cong[where
    f= $\lambda Q.$  wp receive-composition-fri-challenge Q t])
  apply (rule ext)
  apply (simp add: wp-return split: option.splits prod.splits)
  done
qed
have eq:
  wp-event
    (staged-composition-fri-challenge-prefix-program A i  $\gg$ 
      ( $\lambda$ prefix. receive-composition-fri-challenge  $\gg$ 
        ( $\lambda b.$  return (prefix, b)))) ?Q s =
  wp-event
    (staged-composition-fri-challenge-prefix-program A i  $\gg$ 
      ( $\lambda.$  receive-composition-fri-challenge)) ?P s
  by (rule wp-event-bind-cong-cont) (simp-all add: cont-eq)
show ?thesis
  unfolding eq
  by (rule
    staged-composition-fri-prefix-receive-bad-vector-position-relation-bound
    [OF wf controlled subset i-bound dg-bound absent])
qed

lemma staged-composition-fri-bad-vector-coordinate-sum-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and subset:
  B  $\subseteq$  fri-challenge-space (ceil-log (to-nat dg + 1))
and dg-bound:
  ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1)
and absent:
   $\bigwedge i.$  i < ceil-log (to-nat dg + 1)  $\implies$ 
    composition-fri-challenge-values-absent
      (fri-vector-position-values B i) s
shows
  ( $\sum i$  < ceil-log (to-nat dg + 1)).
  wp-event
    (staged-composition-fri-challenge-prefix-program A i  $\gg$ 
      ( $\lambda.$  receive-composition-fri-challenge))
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (b, -)  $\Rightarrow$  b  $\in$  fri-vector-position-values B i) s  $\leq$ 

```



```

    nnreal (ceil-log (to-nat dg + 1)) *
    staged-phase-relation-error (card B)
    (Suc
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget))
  proof -
  have
    (∑ i < ceil-log (to-nat dg + 1).
     wp-event
      (staged-composition-fri-challenge-prefix-program A i >>=
       (λ-. receive-composition-fri-challenge))
      (λout. case out of None ⇒ False
        | Some (b, -) ⇒ b ∈ fri-vector-position-values B i) s) ≤
    (∑ i < ceil-log (to-nat dg + 1).
     staged-phase-relation-error (card B)
     (Suc
       (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)))
  proof (rule sum-mono)
  fix i
  assume i-bound: i ∈ {.. $\text{ceil-log (to-nat dg + 1)}$ }
  then have i-lt: i < ceil-log (to-nat dg + 1)
  by simp
  show
    wp-event
      (staged-composition-fri-challenge-prefix-program A i >>=
       (λ-. receive-composition-fri-challenge))
      (λout. case out of None ⇒ False
        | Some (b, -) ⇒ b ∈ fri-vector-position-values B i) s ≤
    staged-phase-relation-error (card B)
    (Suc
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget))
  by (rule
      staged-composition-fri-prefix-receive-bad-vector-position-relation-bound
      [OF wf controlled subset i-lt dg-bound absent[OF i-lt]])
  qed
  also have ... =
    nnreal (ceil-log (to-nat dg + 1)) *
    staged-phase-relation-error (card B)
    (Suc
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget))
  by simp
  finally show ?thesis .
  qed

```

lemma *staged-composition-fri-bad-vector-coordinate-sum-keep-prefix-bound:*
assumes *wf: staged-budget-wellformed budgets*

and *controlled*: staged-adversary-controlled budgets A
and *subset*:
 $B \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$
and *dg-bound*:
 $\text{ceil-log } (\text{to-nat } dg + 1) \leq \text{ceil-log } (\text{maxDegree} + 1)$
and *absent*:
 $\bigwedge i. i < \text{ceil-log } (\text{to-nat } dg + 1) \implies$
 $\text{composition-fri-challenge-values-absent}$
 $(\text{fri-vector-position-values } B \ i) \ s$
shows
 $(\sum i < \text{ceil-log } (\text{to-nat } dg + 1)).$
 wp-event
 $(\text{staged-composition-fri-challenge-prefix-program } A \ i \gg=$
 $(\lambda \text{prefix}. \text{receive-composition-fri-challenge } \gg=$
 $(\lambda b. \text{return } (\text{prefix}, b))))$
 $(\lambda \text{out}. \text{case out of None} \Rightarrow \text{False}$
 $| \text{Some } ((-, b), -) \Rightarrow$
 $b \in \text{fri-vector-position-values } B \ i) \ s) \leq$
 $\text{nnreal } (\text{ceil-log } (\text{to-nat } dg + 1)) *$
 $\text{staged-phase-relation-error } (\text{card } B)$
 $(\text{Suc}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}))$

proof –
have
 $(\sum i < \text{ceil-log } (\text{to-nat } dg + 1)).$
 wp-event
 $(\text{staged-composition-fri-challenge-prefix-program } A \ i \gg=$
 $(\lambda \text{prefix}. \text{receive-composition-fri-challenge } \gg=$
 $(\lambda b. \text{return } (\text{prefix}, b))))$
 $(\lambda \text{out}. \text{case out of None} \Rightarrow \text{False}$
 $| \text{Some } ((-, b), -) \Rightarrow$
 $b \in \text{fri-vector-position-values } B \ i) \ s) \leq$
 $(\sum i < \text{ceil-log } (\text{to-nat } dg + 1)).$
 $\text{staged-phase-relation-error } (\text{card } B)$
 $(\text{Suc}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}))$

proof (rule *sum-mono*)
fix i
assume $i\text{-bound}: i \in \{.. < \text{ceil-log } (\text{to-nat } dg + 1)\}$
then have $i\text{-lt}: i < \text{ceil-log } (\text{to-nat } dg + 1)$
by *simp*
show
 wp-event
 $(\text{staged-composition-fri-challenge-prefix-program } A \ i \gg=$
 $(\lambda \text{prefix}. \text{receive-composition-fri-challenge } \gg=$
 $(\lambda b. \text{return } (\text{prefix}, b))))$
 $(\lambda \text{out}. \text{case out of None} \Rightarrow \text{False}$

```

| Some ((-, b), -) ⇒
  b ∈ fri-vector-position-values B i) s ≤
staged-phase-relation-error (card B)
(Suc
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget))
by (rule
  staged-composition-fri-prefix-recv-keep-prefix-bad-vector-position-relation-bound
  [OF wf controlled subset i-lt dg-bound absent[OF i-lt]])
qed
also have ... =
  nnreal (ceil-log (to-nat dg + 1)) *
  staged-phase-relation-error (card B)
  (Suc
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget))
  by simp
finally show ?thesis .
qed

lemma staged-controlled-trace-root:
  assumes staged-adversary-controlled budgets A
  shows
    controlled-ro-program (trace-root-budget budgets)
    (trace-root-stage A)
  using assms unfolding staged-adversary-controlled-def by simp

lemma staged-controlled-trace-fri:
  assumes controlled: staged-adversary-controlled budgets A
  and i < length (trace-fri-budgets budgets)
  shows
    controlled-ro-program (trace-fri-budgets budgets ! i)
    (trace-fri-root-stage A i bs)
  using assms unfolding staged-adversary-controlled-def by blast

lemma staged-controlled-trace-final:
  assumes staged-adversary-controlled budgets A
  shows
    controlled-ro-program (trace-final-budget budgets)
    (trace-final-stage A bs)
  using assms unfolding staged-adversary-controlled-def by blast

lemma staged-controlled-degree:
  assumes staged-adversary-controlled budgets A
  shows
    controlled-ro-program (degree-budget budgets)
    (degree-stage A as)
  using assms unfolding staged-adversary-controlled-def by blast

```

lemma *staged-controlled-composition-fri*:
assumes *controlled: staged-adversary-controlled budgets A*
and $i < \text{length } (\text{composition-fri-budgets budgets})$
shows
controlled-ro-program (composition-fri-budgets budgets ! i)
(composition-fri-root-stage A dg i bs)
using *assms unfolding staged-adversary-controlled-def* **by** *blast*

lemma *staged-controlled-composition-final*:
assumes *staged-adversary-controlled budgets A*
shows
controlled-ro-program (composition-final-budget budgets)
(composition-final-stage A dg bs)
using *assms unfolding staged-adversary-controlled-def* **by** *blast*

lemma *staged-controlled-query*:
assumes *controlled: staged-adversary-controlled budgets A*
and $i < \text{length } (\text{query-opening-budgets budgets})$
shows
controlled-ro-program (query-opening-budgets budgets ! i)
(query-opening-stage A i raw)
using *assms unfolding staged-adversary-controlled-def* **by** *blast*

lemma *staged-security-experiment-hash-map-output-collision-bad-bound*:
assumes *admissible*:
admissible-adversary q (staged-semantic-adversary A)
shows
wp-event (staged-security-experiment A)
(hash-map-output-collision-bad adversary-initial-state)
adversary-initial-state $\leq$
hash-collision-budget-value 0 (q + verifier-hash-query-budget)
unfolding *staged-security-experiment-eq-security-experiment*
by *(rule security-experiment-hash-map-output-collision-bad-bound*
[OF admissible])

lemma *staged-security-experiment-hash-map-output-collision-bad-bound-controlled*:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
wp-event (staged-security-experiment A)
(hash-map-output-collision-bad adversary-initial-state)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by *(rule staged-security-experiment-hash-map-output-collision-bad-bound*
[OF admissible-staged-semantic-adversary[OF wf controlled]])

lemma *hash-range-budget-checked-staged-security-experiment*:

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  hash-range-budget
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget)
    (checked-staged-security-experiment A)
proof –
  have cont:
     $\wedge$ data. hash-range-budget verifier-hash-query-budget
      (get  $\gg=$ 
        ( $\lambda$ s. put
          (verifier-state-from-adversary s
            (staged-proof-transcript data))  $\gg=$ 
            ( $\lambda$ -. verify-monad)))
  using hash-range-budget-verifier-after-adversary
    [of staged-proof-transcript data for data]
  unfolding verifier-state-transfer-def sm-bind-assoc .
  have budget:
    hash-range-budget
      ((staged-attacker-query-budget budgets +
        staged-challenge-query-budget) +
       verifier-hash-query-budget)
      (checked-staged-security-experiment A)
  unfolding checked-staged-security-experiment-def
  by (rule hash-range-budget-bind)
      (rule hash-range-budget-checked-staged-transcript-program
        [OF wf controlled], rule cont)
  then show ?thesis by (simp add: add.assoc)
qed

```

```

lemma hash-collision-budget-checked-staged-security-experiment:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  hash-collision-budget
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget)
    (checked-staged-security-experiment A)
proof –
  have cont-range:
     $\wedge$ data. hash-range-budget verifier-hash-query-budget
      (get  $\gg=$ 
        ( $\lambda$ s. put
          (verifier-state-from-adversary s
            (staged-proof-transcript data))  $\gg=$ 
            ( $\lambda$ -. verify-monad)))
  using hash-range-budget-verifier-after-adversary
    [of staged-proof-transcript data for data]

```

```

unfolding verifier-state-transfer-def sm-bind-assoc .
have cont-collision:
   $\wedge$ data. hash-collision-budget verifier-hash-query-budget
    (get  $\gg$ 
      ( $\lambda$ s. put
        (verifier-state-from-adversary s
          (staged-proof-transcript data))  $\gg$ 
          ( $\lambda$ -. verify-monad)))
using hash-collision-budget-verifier-after-adversary
  [of staged-proof-transcript data for data]
unfolding verifier-state-transfer-def sm-bind-assoc .
have budget:
  hash-collision-budget
    ((staged-attacker-query-budget budgets +
      staged-challenge-query-budget) +
      verifier-hash-query-budget)
    (checked-staged-security-experiment A)
unfolding checked-staged-security-experiment-def
by (rule hash-collision-budget-bind)
  (rule hash-range-budget-checked-staged-transcript-program
    [OF wf controlled],
    rule hash-collision-budget-checked-staged-transcript-program
    [OF wf controlled],
    rule cont-range,
    rule cont-collision)
then show ?thesis by (simp add: add.assoc)
qed

lemma checked-staged-security-experiment-hash-new-collision-bound-controlled:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event (checked-staged-security-experiment A)
    (hash-new-collision-event adversary-initial-state)
    adversary-initial-state  $\leq$ 
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget + verifier-hash-query-budget)
proof -
let ?q =
  staged-attacker-query-budget budgets +
  staged-challenge-query-budget + verifier-hash-query-budget
have collision-budget:
  hash-collision-budget ?q (checked-staged-security-experiment A)
by (rule hash-collision-budget-checked-staged-security-experiment
    [OF wf controlled])
have wp-event (checked-staged-security-experiment A)
  (hash-new-collision-event adversary-initial-state)
  adversary-initial-state  $\leq$ 

```

```

    hash-collision-budget-value
    (card (hash-map-output-values adversary-initial-state)) ?q
using collision-budget adversary-initial-state-no-output-collision
unfolding hash-collision-budget-def by blast
then show ?thesis by simp
qed

lemma checked-staged-security-experiment-hash-map-output-collision-bad-bound-controlled:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event (checked-staged-security-experiment A)
    (hash-map-output-collision-bad adversary-initial-state)
    adversary-initial-state  $\leq$ 
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget)
proof –
have wp-event (checked-staged-security-experiment A)
  (hash-map-output-collision-bad adversary-initial-state)
  adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment A)
    (hash-new-collision-event adversary-initial-state)
    adversary-initial-state
by (rule wp-event-mono)
    (rule hash-map-output-collision-bad-imp-security-new-collision)
also have ...  $\leq$ 
  hash-collision-budget-value 0
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget + verifier-hash-query-budget)
by (rule checked-staged-security-experiment-hash-new-collision-bound-controlled
    [OF wf controlled])
finally show ?thesis .
qed

```

```

lemma checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     hash-map-output-collision-bad)
    adversary-initial-state  $\leq$ 
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget)
proof –
let ?M = checked-staged-security-experiment-with-data-state A
let ?P = hash-map-output-collision-bad adversary-initial-state

```

```

let ?projected =
  λout. case out of None ⇒ ?P None
    | Some (x, t) ⇒ ?P (Some (snd x, t))
have event-le-projected:
  wp-event ?M
    (staged-security-with-data-state-verifier-event
     hash-map-output-collision-bad)
  adversary-initial-state ≤
  wp-event ?M ?projected adversary-initial-state
proof (rule wp-event-mono)
  fix out
  assume bad:
    staged-security-with-data-state-verifier-event
    hash-map-output-collision-bad out
  show ?projected out
  proof (cases out)
    case None
    then show ?thesis
      using bad unfolding staged-security-with-data-state-verifier-event-def
      by simp
  next
    case (Some packed)
    then obtain data attacker-state result final-state where out-eq:
      out = Some (((data, attacker-state), result), final-state)
      by (cases packed, auto split: prod.splits)
    have local-bad:
      hash-map-output-collision-bad
        (verifier-state-from-adversary attacker-state
         (staged-proof-transcript data))
        (Some (result, final-state))
      using bad unfolding out-eq
        staged-security-with-data-state-verifier-event-def by simp
    then have global-bad:
      hash-map-output-collision-bad adversary-initial-state
        (Some (result, final-state))
      unfolding hash-map-output-collision-bad-def by simp
    show ?thesis
      unfolding out-eq using global-bad by simp
  qed
qed
have projected-eq:
  wp-event ?M ?projected adversary-initial-state =
  wp-event (checked-staged-security-experiment A) ?P
  adversary-initial-state
proof -
  have ?M ≧ (λx. return (snd x)) =
    checked-staged-security-experiment A
    by (rule checked-staged-security-experiment-with-data-state-projection)
  then have wp-event (checked-staged-security-experiment A) ?P

```



```

    adversary-initial-state =
wp-event (?M  $\gg$  ( $\lambda x.$  return (snd x))) ?P
    adversary-initial-state
  by simp
also have ... = wp-event ?M ?projected adversary-initial-state
  by (subst wp-event-bind-return-map[where f=snd]) simp
finally show ?thesis by simp
qed
have checked-bound:
wp-event (checked-staged-security-experiment A)
  (hash-map-output-collision-bad adversary-initial-state)
  adversary-initial-state  $\leq$ 
  hash-collision-budget-value 0
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule
    checked-staged-security-experiment-hash-map-output-collision-bad-bound-controlled
    [OF wf controlled])
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    hash-map-output-collision-bad)
  adversary-initial-state  $\leq$ 
  wp-event ?M ?projected adversary-initial-state
  by (rule event-le-projected)
also have ... =
  wp-event (checked-staged-security-experiment A) ?P
  adversary-initial-state
  by (rule projected-eq)
also have ...  $\leq$ 
  hash-collision-budget-value 0
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule checked-bound)
finally show ?thesis .
qed

end

end

```

```

theory Soundness-Oracle-Dynamic-Target
  imports Soundness-Oracle-Budgets
begin

```

State-dependent target-hit combinators.

These lemmas are deliberately generic: the selected target set may depend on an intermediate oracle state, but all probability bounds are still discharged through the existing random-oracle target-budget interface.

context *soundness*
begin

definition *state-dependent-output-hit*
 $:: ((f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{state-dependent-output-hit } B \text{ s out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } (y, -) \Rightarrow y \in B \text{ s})$

definition *state-dependent-new-output-hit*
 $:: ((f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{state-dependent-new-output-hit } B \text{ s out} \longleftrightarrow$
 $\text{hash-new-output-hit-event } (B \text{ s}) \text{ s out}$

definition *state-dependent-prequeried-hash-hit*
 $:: ((f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ set}) \Rightarrow$
 $'f \text{ protocol-hash-input} \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{state-dependent-prequeried-hash-hit } B \text{ x s} \longleftrightarrow$
 $(\exists y. \text{fmlookup } (\text{HashMap } s) \text{ x} = \text{Some } y \wedge y \in B \text{ s})$

lemma *state-dependent-output-hit-None[simp]*:
 $\neg \text{state-dependent-output-hit } B \text{ s None}$
unfolding *state-dependent-output-hit-def* **by** *simp*

lemma *state-dependent-new-output-hit-None[simp]*:
 $\neg \text{state-dependent-new-output-hit } B \text{ s None}$
unfolding *state-dependent-new-output-hit-def* *hash-new-output-hit-event-def*
by *simp*

lemma *state-dependent-prequeried-hash-hitI*:
assumes $\text{fmlookup } (\text{HashMap } s) \text{ x} = \text{Some } y$
and $y \in B \text{ s}$
shows $\text{state-dependent-prequeried-hash-hit } B \text{ x s}$
using *assms* **unfolding** *state-dependent-prequeried-hash-hit-def* **by** *blast*

lemma *state-dependent-output-hit-mono*:
assumes $\text{subset: } \bigwedge s. B \text{ s} \subseteq C \text{ s}$
and *hit*: $\text{state-dependent-output-hit } B \text{ s out}$
shows $\text{state-dependent-output-hit } C \text{ s out}$
using *subset hit*

unfolding *state-dependent-output-hit-def*
by (*cases out*) *auto*

lemma *state-dependent-new-output-hit-mono*:
assumes *subset*: $\bigwedge s. B\ s \subseteq C\ s$
and *hit*: *state-dependent-new-output-hit* *B s out*
shows *state-dependent-new-output-hit* *C s out*
using *subset hit*
unfolding *state-dependent-new-output-hit-def hash-new-output-hit-event-def*
by (*cases out*) (*auto intro: hash-map-new-output-hit-subset*)

lemma *state-dependent-output-hit-union*:
state-dependent-output-hit $(\lambda s. B\ s \cup C\ s)\ s\ out \longleftrightarrow$
state-dependent-output-hit *B s out* $\vee$
state-dependent-output-hit *C s out*
unfolding *state-dependent-output-hit-def*
by (*cases out*) *auto*

lemma *state-dependent-new-output-hit-union-imp*:
assumes *hit*: *state-dependent-new-output-hit* $(\lambda s. B\ s \cup C\ s)\ s\ out$
shows
state-dependent-new-output-hit *B s out* $\vee$
state-dependent-new-output-hit *C s out*
proof (*cases out*)
case *None*
then show *?thesis*
using *hit* **by** *simp*
next
case (*Some rt*)
then obtain *r t* **where** *rt*: *rt* = (*r*, *t*)
by (*cases rt*) *simp*
obtain *x y* **where**
none: *fmlookup* (*HashMap s*) *x* = *None*
and *some*: *fmlookup* (*HashMap t*) *x* = *Some y*
and *y*: *y* $\in B\ s \cup C\ s$
using *hit Some rt*
unfolding *state-dependent-new-output-hit-def hash-new-output-hit-event-def*
hash-map-new-output-hit-def
by *auto*
show *?thesis*
proof (*cases y* $\in B\ s$)
case *True*
then show *?thesis*
using *Some rt none some*
unfolding *state-dependent-new-output-hit-def hash-new-output-hit-event-def*
hash-map-new-output-hit-def
by *auto*
next
case *False*

```

then have  $y \in C\ s$ 
  using  $y$  by simp
then show ?thesis
  using Some rt none some
  unfolding state-dependent-new-output-hit-def hash-new-output-hit-event-def
    hash-map-new-output-hit-def
  by auto
qed
qed

```

lemma *wp-event-state-dependent-output-union-bound*:

```

wp-event  $m$ 
  (state-dependent-output-hit ( $\lambda s. B\ s \cup C\ s$ )  $s$ )  $s \leq$ 
  wp-event  $m$  (state-dependent-output-hit  $B\ s$ )  $s$  +
  wp-event  $m$  (state-dependent-output-hit  $C\ s$ )  $s$ 
unfolding state-dependent-output-hit-union
by (rule wp-event-union-bound)

```

lemma *wp-event-state-dependent-new-output-union-bound*:

```

wp-event  $m$ 
  (state-dependent-new-output-hit ( $\lambda s. B\ s \cup C\ s$ )  $s$ )  $s \leq$ 
  wp-event  $m$  (state-dependent-new-output-hit  $B\ s$ )  $s$  +
  wp-event  $m$  (state-dependent-new-output-hit  $C\ s$ )  $s$ 

```

proof –

```

have wp-event  $m$ 
  (state-dependent-new-output-hit ( $\lambda s. B\ s \cup C\ s$ )  $s$ )  $s \leq$ 
  wp-event  $m$ 
    ( $\lambda out. \text{state-dependent-new-output-hit } B\ s\ out \vee$ 
      state-dependent-new-output-hit  $C\ s\ out$ )  $s$ 
  by (rule wp-event-mono) (rule state-dependent-new-output-hit-union-imp)
also have ...  $\leq$ 
  wp-event  $m$  (state-dependent-new-output-hit  $B\ s$ )  $s$  +
  wp-event  $m$  (state-dependent-new-output-hit  $C\ s$ )  $s$ 
  by (rule wp-event-union-bound)

```

finally show ?thesis .

qed

lemma *wp-hash-state-dependent-new-output-hit*:

```

fixes  $x :: 'f\ \text{protocol-hash-input}$ 
shows
  wp-event (hash  $x$ ) (state-dependent-new-output-hit  $B\ s$ )  $s =$ 
    (if fmlookup (HashMap  $s$ )  $x = \text{None}$ 
      then nnreal (card ( $B\ s$ )) / nnreal size
      else 0)
unfolding state-dependent-new-output-hit-def
by (rule wp-hash-new-output-hit)

```

lemma *wp-hash-state-dependent-new-output-hit-bound*:

```

fixes  $x :: 'f\ \text{protocol-hash-input}$ 

```

shows
 $wp\text{-}event\ (hash\ x)\ (state\text{-}dependent\text{-}new\text{-}output\text{-}hit\ B\ s)\ s \leq$
 $nnreal\ (card\ (B\ s))\ /\ nnreal\ size$
unfolding $state\text{-}dependent\text{-}new\text{-}output\text{-}hit\text{-}def$
by $(rule\ wp\text{-}hash\text{-}new\text{-}output\text{-}hit\text{-}bound)$

lemma $wp\text{-}hash\text{-}state\text{-}dependent\text{-}new\text{-}output\text{-}hit\text{-}bound\text{-}card$:
fixes $x :: 'f\ protocol\text{-}hash\text{-}input$
assumes $card\text{-}bound$: $card\ (B\ s) \leq N$
shows
 $wp\text{-}event\ (hash\ x)\ (state\text{-}dependent\text{-}new\text{-}output\text{-}hit\ B\ s)\ s \leq$
 $nnreal\ N\ /\ nnreal\ size$
proof –
have $wp\text{-}event\ (hash\ x)\ (state\text{-}dependent\text{-}new\text{-}output\text{-}hit\ B\ s)\ s \leq$
 $nnreal\ (card\ (B\ s))\ /\ nnreal\ size$
by $(rule\ wp\text{-}hash\text{-}state\text{-}dependent\text{-}new\text{-}output\text{-}hit\text{-}bound)$
also have $\dots \leq nnreal\ N\ /\ nnreal\ size$
by $(rule\ nnreal\text{-}nat\text{-}divide\text{-}right\text{-}mono[OF\ card\text{-}bound])$
finally show $?thesis$.
qed

lemma $hash\text{-}state\text{-}dependent\text{-}output\text{-}hit\text{-}fresh\text{-}or\text{-}prequeried$:
fixes $x :: 'f\ protocol\text{-}hash\text{-}input$
assumes out : $out \in set\text{-}dist\ (execute\ (hash\ x)\ s)$
and hit : $state\text{-}dependent\text{-}output\text{-}hit\ B\ s\ out$
shows
 $state\text{-}dependent\text{-}new\text{-}output\text{-}hit\ B\ s\ out \vee$
 $state\text{-}dependent\text{-}prequeried\text{-}hash\text{-}hit\ B\ x\ s$
proof $(cases\ out)$
case $None$
then show $?thesis$
using hit **by** $simp$
next
case $(Some\ ht)$
then obtain $h\ t$ **where** ht : $ht = (h, t)$
by $(cases\ ht)\ simp$
have $hash\text{-}out$: $Some\ (h, t) \in set\text{-}dist\ (execute\ (hash\ x)\ s)$
using $out\ Some\ ht$ **by** $simp$
have $h\text{-}in$: $h \in B\ s$
using $hit\ Some\ ht$ **unfolding** $state\text{-}dependent\text{-}output\text{-}hit\text{-}def$ **by** $simp$
have $lookup\text{-}t$: $fmlookup\ (HashMap\ t)\ x = Some\ h$
using $protocol\text{-}merkle.hash\text{-}outcome(2)[OF\ hash\text{-}out]$.
show $?thesis$
proof $(cases\ fmlookup\ (HashMap\ s)\ x)$
case $None$
then have $hash\text{-}map\text{-}new\text{-}output\text{-}hit\ (B\ s)\ s\ t$
unfolding $hash\text{-}map\text{-}new\text{-}output\text{-}hit\text{-}def$
using $lookup\text{-}t\ h\text{-}in$ **by** $blast$
then show $?thesis$

```

    using Some ht
    unfolding state-dependent-new-output-hit-def hash-new-output-hit-event-def
    by simp
next
case (Some y)
then have y = h
using lookup-t hash-extension-lookup[OF Some protocol-merkle.hash-outcome(1)[OF hash-out]]
by simp
then show ?thesis
using Some h-in
unfolding state-dependent-prequeried-hash-hit-def
by blast
qed
qed

lemma receive-alpha-challenge-state-dependent-hit-fresh-or-prequeried:
  assumes out: Some (a, t) ∈ set-dist (execute receive-alpha-challenge s)
  and hit: a ∈ B s
  shows
    state-dependent-new-output-hit B s (Some (a, t)) ∨
    state-dependent-prequeried-hash-hit B
    (AlphaChallenge (PAlphaCounter s) (PState s)) s
proof -
  have recv:
    s ≤ t ∧
    fmlookup (HashMap t)
    (AlphaChallenge (PAlphaCounter s) (PState s)) = Some a
  using receive-alpha-challenge-outcome[OF out] by blast
  show ?thesis
proof (cases
  fmlookup (HashMap s)
  (AlphaChallenge (PAlphaCounter s) (PState s)))
case None
then have hash-map-new-output-hit (B s) s t
  unfolding hash-map-new-output-hit-def
  using recv hit by blast
then show ?thesis
  unfolding state-dependent-new-output-hit-def
  hash-new-output-hit-event-def
  by simp
next
case (Some old-a)
have old-lookup-t:
  fmlookup (HashMap t)
  (AlphaChallenge (PAlphaCounter s) (PState s)) = Some old-a
  by (rule hash-extension-lookup[OF Some conjunct1[OF recv]])
then have old-a = a
  using recv by simp
then show ?thesis

```

```

    using Some hit
    unfolding state-dependent-prequeried-hash-hit-def
    by blast
qed
qed

lemma wp-receive-alpha-challenge-state-dependent-hit-split:
  wp-event receive-alpha-challenge
    ( $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
     |  $\text{Some } (a, -) \Rightarrow a \in B \ s$ )  $s \leq$ 
  wp-event receive-alpha-challenge
    ( $\text{state-dependent-new-output-hit } B \ s$ )  $s +$ 
    ( $\text{if state-dependent-prequeried-hash-hit } B$ 
     ( $\text{AlphaChallenge } (P\text{AlphaCounter } s) (P\text{State } s)$ )  $s$ 
     then 1 else 0)
proof -
  let ?bad =
     $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
    |  $\text{Some } (a, -) \Rightarrow a \in B \ s$ 
  let ?fresh = state-dependent-new-output-hit  $B \ s$ 
  let ?pre =
    state-dependent-prequeried-hash-hit  $B$ 
    ( $\text{AlphaChallenge } (P\text{AlphaCounter } s) (P\text{State } s)$ )  $s$ 
  have wp-event receive-alpha-challenge ?bad  $s \leq$ 
    wp-event receive-alpha-challenge
      ( $\lambda out. ?fresh \text{ out} \vee ?pre$ )  $s$ 
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:  $out \in \text{set-dist } (\text{execute receive-alpha-challenge } s)$ 
    and bad: ?bad out
    show ?fresh out  $\vee$  ?pre
    proof (cases out)
      case None
      then show ?thesis
        using bad by simp
    next
      case (Some pair)
      then obtain  $a \ t$  where out-eq:  $out = \text{Some } (a, t)$ 
        by (cases pair) auto
      have a-in:  $a \in B \ s$ 
        using bad unfolding out-eq by simp
      show ?thesis
        unfolding out-eq
        by (rule
            receive-alpha-challenge-state-dependent-hit-fresh-or-prequeried
            [ $\text{where } B=B \text{ and } s=s \text{ and } a=a \text{ and } t=t, \text{ OF } a\text{-in}$ ])
            (use support out-eq in simp)
    qed
  qed

```

```

also have ...  $\leq$ 
  wp-event receive-alpha-challenge ?fresh s +
  wp-event receive-alpha-challenge ( $\lambda$ -. ?pre) s
by (rule wp-event-union-bound)
also have ...  $\leq$ 
  wp-event receive-alpha-challenge ?fresh s +
  (if ?pre then 1 else 0)
proof (cases ?pre)
  case True
  have wp-event receive-alpha-challenge ( $\lambda$ -. ?pre) s  $\leq$  1
    by (rule wp-event-le-1)
  then show ?thesis
    using True by simp
  next
  case False
  then show ?thesis
    by (simp add: wp-event-def wp-def)
qed
finally show ?thesis .
qed

lemma wp-event-bind-state-dependent-output-bound-by-cont:
fixes m :: ('x, ('f, 'a) protocol-channel-scheme) state-monad
  and k :: 'x  $\Rightarrow$  ('f, ('f, 'a) protocol-channel-scheme) state-monad
assumes none:  $\neg E \text{ None}$ 
and event-imp:
   $\bigwedge x \ t \ out. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$ 
   $out \in \text{set-dist } (\text{execute } (k \ x) \ t) \implies$ 
   $E \ out \implies \text{state-dependent-output-hit } B \ t \ out$ 
and cont-bound:
   $\bigwedge x \ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$ 
   $wp\text{-event } (k \ x) \ (\text{state-dependent-output-hit } B \ t) \ t \leq C$ 
shows wp-event (m  $\gg$  k) E s  $\leq$  C
proof (rule wp-event-bind-bound-by-cont
  [where Q=E and m=m and k=k and s=s and C=C])
  show  $\neg E \text{ None}$ 
    by (rule none)
next
fix x t
assume head:  $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s)$ 
have wp-event (k x) E t  $\leq$ 
  wp-event (k x) (state-dependent-output-hit B t) t
    by (rule wp-event-mono-on-support)
    (use head event-imp in blast)
also have ...  $\leq$  C
    by (rule cont-bound[OF head])
finally show wp-event (k x) E t  $\leq$  C .
qed

```


lemma *wp-event-bind-state-dependent-new-output-bound-by-cont:*
fixes $m :: ('x, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$
and $k :: 'x \Rightarrow ('y, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$
assumes *none:* $\neg E \text{ None}$
and *event-imp:*
 $\bigwedge x \ t \ \text{out}. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$
 $\text{out} \in \text{set-dist } (\text{execute } (k \ x) \ t) \implies$
 $E \ \text{out} \implies \text{state-dependent-new-output-hit } B \ t \ \text{out}$
and *cont-bound:*
 $\bigwedge x \ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$
 $\text{wp-event } (k \ x) \ (\text{state-dependent-new-output-hit } B \ t) \ t \leq C$
shows $\text{wp-event } (m \gg k) \ E \ s \leq C$
proof (*rule wp-event-bind-bound-by-cont*
[**where** $Q=E$ **and** $m=m$ **and** $k=k$ **and** $s=s$ **and** $C=C$])
show $\neg E \text{ None}$
by (*rule none*)
next
fix $x \ t$
assume *head:* $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s)$
have $\text{wp-event } (k \ x) \ E \ t \leq$
 $\text{wp-event } (k \ x) \ (\text{state-dependent-new-output-hit } B \ t) \ t$
by (*rule wp-event-mono-on-support*)
(use head event-imp in blast)
also have $\dots \leq C$
by (*rule cont-bound[OF head]*)
finally show $\text{wp-event } (k \ x) \ E \ t \leq C$.
qed

lemma *wp-event-bind-state-dependent-new-output-bound-by-target-budget:*
fixes $m :: ('x, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$
and $k :: 'x \Rightarrow ('y, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$
assumes *none:* $\neg E \text{ None}$
and *event-imp:*
 $\bigwedge x \ t \ \text{out}. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$
 $\text{out} \in \text{set-dist } (\text{execute } (k \ x) \ t) \implies$
 $E \ \text{out} \implies \text{state-dependent-new-output-hit } B \ t \ \text{out}$
and *budget:*
 $\bigwedge x \ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$
 $\text{hash-target-budget } (B \ t) \ n \ (k \ x)$
and *value-bound:*
 $\bigwedge x \ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$
 $\text{hash-target-budget-value } (B \ t) \ n \leq C$
shows
 $\text{wp-event } (m \gg k) \ E \ s \leq C$
proof (*rule wp-event-bind-state-dependent-new-output-bound-by-cont*
[**where** $E=E$ **and** $m=m$ **and** $k=k$ **and** $s=s$ **and** $B=B$ **and** $C=C$])
show $\neg E \text{ None}$
by (*rule none*)
next

```

show  $\bigwedge x t$  out.
  Some ( $x, t$ )  $\in$  set-dist (execute  $m$   $s$ )  $\implies$ 
  out  $\in$  set-dist (execute ( $k$   $x$ )  $t$ )  $\implies$ 
   $E$  out  $\implies$  state-dependent-new-output-hit  $B$   $t$  out
  by (rule event-imp)
next
  fix  $x t$ 
  assume head: Some ( $x, t$ )  $\in$  set-dist (execute  $m$   $s$ )
  have wp-event ( $k$   $x$ ) (state-dependent-new-output-hit  $B$   $t$ )  $t \leq$ 
    hash-target-budget-value ( $B$   $t$ )  $n$ 
  using budget[OF head]
  unfolding hash-target-budget-def state-dependent-new-output-hit-def
  by blast
  also have  $\dots \leq C$ 
  by (rule value-bound[OF head])
  finally show
    wp-event ( $k$   $x$ ) (state-dependent-new-output-hit  $B$   $t$ )  $t \leq C$  .
qed

end

end

```

```

theory Staged-Security-Experiment-Alpha-Prequery
imports
  Staged-Security-Experiment-Checked-Prefix
  Soundness-Oracle-Dynamic-Target
begin

```

Alpha-key prequery support for staged soundness.

This layer records deterministic facts about the alpha challenge key path. It is proof infrastructure for accounting adversarial prequeries to future alpha keys; it does not add assumptions or change the oracle model.

```

context soundness
begin

```

```

definition alpha-challenge-key-at
  :: 'f protocol-channel  $\Rightarrow$  nat  $\Rightarrow$  'f list  $\Rightarrow$ 
    'f protocol-hash-input
  where
    alpha-challenge-key-at  $s$   $i$   $as$  =
      AlphaChallenge (PAlphaCounter  $s$  +  $i$ )
      (foldl concat (PState  $s$ ) (take  $i$   $as$ ))

```

```

definition alpha-vector-key-relation
  :: 'f protocol-channel  $\Rightarrow$  'f list set  $\Rightarrow$  nat  $\Rightarrow$ 
    'f protocol-hash-input  $\Rightarrow$  'f  $\Rightarrow$  bool
  where

```

$\text{alpha-vector-key-relation } s \ B \ n \ x \ y \longleftrightarrow$
 $(\exists as \in B. \exists i < n.$
 $i < \text{length } as \wedge$
 $x = \text{alpha-challenge-key-at } s \ i \ as \wedge$
 $y = as \ ! \ i)$

definition *alpha-vector-prequery-hit-event*

$:: 'f \text{ protocol-channel} \Rightarrow 'f \text{ list set} \Rightarrow \text{nat} \Rightarrow$
 $('r \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{alpha-vector-prequery-hit-event } s \ B \ n =$
 $\text{hash-relation-hit-event } (\text{alpha-vector-key-relation } s \ B \ n) \ s$

definition *alpha-vector-prequeried-in-state*

$:: 'f \text{ protocol-channel} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow \text{bool}$

where

$\text{alpha-vector-prequeried-in-state } s \ as \ n \longleftrightarrow$
 $(\exists i < n.$
 $i < \text{length } as \wedge$
 $\text{fmlookup } (\text{HashMap } s) (\text{alpha-challenge-key-at } s \ i \ as) =$
 $\text{Some } (as \ ! \ i))$

definition *alpha-vector-path-fresh*

$:: 'f \text{ protocol-channel} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow \text{bool}$

where

$\text{alpha-vector-path-fresh } s \ as \ n \longleftrightarrow$
 $(\forall i < n.$
 $i < \text{length } as \longrightarrow$
 $\text{fmlookup } (\text{HashMap } s) (\text{alpha-challenge-key-at } s \ i \ as) = \text{None})$

lemma *alpha-vector-key-relation-fiber-card-bound:*

assumes *finite-B: finite B*

shows $\text{card } \{y. \text{alpha-vector-key-relation } s \ B \ n \ x \ y\} \leq \text{card } B * n$

proof –

let $?pairs = B \times \{..<n\}$

let $?values = (\lambda(as, i). as \ ! \ i) \ ` \ ?pairs$

have *subset:*

$\{y. \text{alpha-vector-key-relation } s \ B \ n \ x \ y\} \subseteq ?values$

unfolding *alpha-vector-key-relation-def*

by *force*

have *finite-values: finite ?values*

using *finite-B* **by** *simp*

have $\text{card } \{y. \text{alpha-vector-key-relation } s \ B \ n \ x \ y\} \leq \text{card } ?values$

by $(\text{rule card-mono}[\text{OF } \text{finite-values subset}])$

also have $\dots \leq \text{card } ?pairs$

by $(\text{rule card-image-le}) \ (use \ \text{finite-B} \ \text{in} \ \text{simp})$

also have $\dots = \text{card } B * n$

using *finite-B* **by** *simp*

finally show *?thesis* .

qed

lemma *hash-relation-program-staged-alpha-prefix-alpha-vector-prequery:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *finite-B: finite B*

shows

*hash-relation-program (alpha-vector-key-relation s B n) (card B * n)*
(staged-alpha-search-queries budgets 0)
(staged-alpha-prefix-program A)

by (*rule hash-relation-program-staged-alpha-prefix-program*
[OF wf controlled])

(*rule alpha-vector-key-relation-fiber-card-bound [OF finite-B]*)

lemma *staged-alpha-prefix-alpha-vector-prequery-hit-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *finite-B: finite B*

shows

wp-event (staged-alpha-prefix-program A)
(alpha-vector-prequery-hit-event s B n) s ≤
*hash-relation-budget-value (card B * n)*
(staged-alpha-search-queries budgets 0)

proof –

have *program:*

*hash-relation-program (alpha-vector-key-relation s B n) (card B * n)*
(staged-alpha-search-queries budgets 0)
(staged-alpha-prefix-program A)

by (*rule hash-relation-program-staged-alpha-prefix-alpha-vector-prequery*
[OF wf controlled finite-B])

then have *budget:*

*hash-relation-budget (alpha-vector-key-relation s B n) (card B * n)*
(staged-alpha-search-queries budgets 0)
(staged-alpha-prefix-program A)

unfolding *hash-relation-program-def* **by** *simp*

show *?thesis*

using *budget*

unfolding *alpha-vector-prequery-hit-event-def*

hash-relation-budget-def **by** *blast*

qed

lemma *staged-alpha-prefix-event-bound-from-hash-relation:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *fibers: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$*

and *event-imp:*

$\bigwedge x \ t. \text{Some } (x, t) \in$
 $\text{set-dist } (\text{execute } (\text{staged-alpha-prefix-program } A) \ s) \implies$
 $E \ (\text{Some } (x, t)) \implies \text{hash-relation-hit } R \ s \ t$

and *none*: $\neg E \text{ None}$
shows
 $wp\text{-event } (staged\text{-alpha}\text{-prefix}\text{-program } A) E s \leq$
 $hash\text{-relation}\text{-budget}\text{-value } b \text{ (staged}\text{-alpha}\text{-search}\text{-queries budgets } 0)$
proof –
have *program*:
 $hash\text{-relation}\text{-program } R b \text{ (staged}\text{-alpha}\text{-search}\text{-queries budgets } 0)$
 $(staged\text{-alpha}\text{-prefix}\text{-program } A)$
by (rule *hash-relation-program-staged-alpha-prefix-program*
 $[OF \text{ wf controlled fibers}]$)
have
 $wp\text{-event } (staged\text{-alpha}\text{-prefix}\text{-program } A) E s \leq$
 $wp\text{-event } (staged\text{-alpha}\text{-prefix}\text{-program } A)$
 $(hash\text{-relation}\text{-hit}\text{-event } R s) s$
proof (rule *wp-event-mono-on-support*)
fix *out*
assume *support*:
 $out \in set\text{-dist } (execute \text{ (staged}\text{-alpha}\text{-prefix}\text{-program } A) s)$
and *event*: $E \text{ out}$
show $hash\text{-relation}\text{-hit}\text{-event } R s \text{ out}$
proof (*cases out*)
case *None*
then show *?thesis* **using** *none event* **by** *simp*
next
case (*Some xt*)
then obtain $x \ t$ **where** $xt: xt = (x, t)$
by (*cases xt*) *simp*
have $hash\text{-relation}\text{-hit } R s \ t$
by (rule *event-imp[of x t]*) (*use support event Some xt in simp-all*)
then show *?thesis*
unfolding *Some xt hash-relation-hit-event-def* **by** *simp*
qed
qed
also have $\dots \leq$
 $hash\text{-relation}\text{-budget}\text{-value } b \text{ (staged}\text{-alpha}\text{-search}\text{-queries budgets } 0)$
using *program unfolding hash-relation-program-def hash-relation-budget-def*
by *blast*
finally show *?thesis* .
qed

lemma *alpha-vector-path-fresh-Cons-tail*:
assumes *fresh*: $alpha\text{-vector}\text{-path}\text{-fresh } s \ (a \# xs) \ (Suc \ n)$
and *recv*:
 $Some \ (a, t) \in set\text{-dist } (execute \ receive\text{-alpha}\text{-challenge } s)$
and *record-out*:
 $Some \ ((), u) \in set\text{-dist } (execute \ (record\text{-staged}\text{-message } a) \ t)$
shows $alpha\text{-vector}\text{-path}\text{-fresh } u \ xs \ n$
proof –
have *recv-fields*:

```

PAlphaCounter t = Suc (PAlphaCounter s)
PState t = PState s
fmlookup (HashMap t)
  (AlphaChallenge (PAlphaCounter s) (PState s)) = Some a
using receive-alpha-challenge-outcome[OF recv]
  receive-alpha-challenge-counter-outcome[OF recv]
by simp-all
have u-eq:
  u = t[
    PState := concat (PState t) a,
    PTranscript := PTranscript t @ [a]]
  by (rule record-staged-message-outcome[OF record-out])
show ?thesis
  unfolding alpha-vector-path-fresh-def
proof (intro allI impI)
  fix i
  assume i-bound: i < n
  and i-len: i < length xs
  let ?tail-key = alpha-challenge-key-at u i xs
  let ?full-key = alpha-challenge-key-at s (Suc i) (a # xs)
  have key-eq: ?tail-key = ?full-key
    unfolding alpha-challenge-key-at-def u-eq
    using recv-fields
    by simp
  have full-none:
    fmlookup (HashMap s) ?full-key = None
    using fresh i-bound i-len
    unfolding alpha-vector-path-fresh-def
    by simp
  have key-ne:
    ?full-key ≠ AlphaChallenge (PAlphaCounter s) (PState s)
    unfolding alpha-challenge-key-at-def by simp
  have lookup-t:
    fmlookup (HashMap t) ?full-key = None
    using receive-alpha-challenge-preserves-other-lookup[OF recv key-ne]
    full-none
    by simp
  show fmlookup (HashMap u) ?tail-key = None
    using lookup-t key-eq unfolding u-eq by simp
qed
qed

lemma wp-staged-alpha-program-exact-challenges-bound-from-path-fresh:
assumes len: length as = n
and fresh: alpha-vector-path-fresh s as n
shows
  wp-event (staged-alpha-program n)
  (λout. case out of None ⇒ False
    | Some (as', -) ⇒ as' = as) s ≤

```

```

      (1 / nnreal size) ^ n
    using len fresh
  proof (induction n arbitrary: as s)
    case 0
    then have as-empty: as = []
      by simp
    show ?case
      unfolding as-empty by (simp add: wp-event-def wpsimps)
  next
    case (Suc n)
    obtain a as-tail where as-eq: as = a # as-tail
      using Suc.prem1 by (cases as) auto
    have len-tail: length as-tail = n
      using Suc.prem1 as-eq by simp
    let ?tail =
      staged-alpha-program n ::
        ('f list, 'f protocol-channel) state-monad
    let ?Q =
      λout :: ('f list ×
        'f protocol-channel) option.
        case out of None ⇒ False | Some (as', -) ⇒ as' = as
    let ?Head =
      λout :: ('f ×
        'f protocol-channel) option.
        case out of None ⇒ False | Some (a0, -) ⇒ a0 ∈ {a}
    have head-fresh:
      fmllookup (HashMap s)
        (AlphaChallenge (PAlphaCounter s) (PState s)) = None
    proof -
      have fmllookup (HashMap s)
        (alpha-challenge-key-at s 0 (a # as-tail)) = None
        using Suc.prem2
      unfolding as-eq alpha-vector-path-fresh-def by simp
      then show ?thesis
        unfolding alpha-challenge-key-at-def by simp
    qed
    have head-bound:
      wp-event receive-alpha-challenge ?Head s ≤ 1 / nnreal size
    proof -
      have wp-event receive-alpha-challenge ?Head s =
        nnreal (card ({a} :: 'f set)) / nnreal size
        by (rule wp-receive-alpha-challenge-fresh-set[OF head-fresh])
      then show ?thesis by simp
    qed
    have bind-bound:
      wp-event
        (receive-alpha-challenge >=>
          (λa0. record-staged-message a0 >=>
            (λ-. ?tail >=>

```

```

      (λas'. return (a0 # as')))))
    ?Q s ≤
    wp-event receive-alpha-challenge ?Head s *
    (1 / nreal size) ^ n
  proof (rule wp-event-bind-bound-by-head-and-cont[where P = ?Head])
    show ¬ ?Q None
      by simp
  next
  fix a0 t
  assume recv:
    Some (a0, t) ∈ set-dist (execute receive-alpha-challenge s)
    and not-head: ¬ ?Head (Some (a0, t))
  have a0-ne: a0 ≠ a
    using not-head by simp
  have cont-false:
    wp-event
      (record-staged-message a0 ≫=
        (λ-. ?tail ≫= (λas'. return (a0 # as')))))
    ?Q t ≤
    wp-event
      (record-staged-message a0 ≫=
        (λ-. ?tail ≫= (λas'. return (a0 # as')))))
      (λ- :: ('f list × 'f protocol-channel) option. False) t
    by (rule wp-event-mono-on-support)
      (use a0-ne as-eq in
        ‹auto simp: wpsimps elim!: set-dist-bindE
          split: option.splits prod.splits›)
  also have ... = 0
    unfolding wp-event-def wp-def dist-expect-def by simp
  finally have le-zero:
    wp-event
      (record-staged-message a0 ≫=
        (λ-. ?tail ≫= (λas'. return (a0 # as')))))
    ?Q t ≤ 0 .
  show
    wp-event
      (record-staged-message a0 ≫=
        (λ-. ?tail ≫= (λas'. return (a0 # as')))))
    ?Q t = 0
    by (rule antisym[OF le-zero]) simp
  next
  fix a0 t
  assume recv:
    Some (a0, t) ∈ set-dist (execute receive-alpha-challenge s)
    and head: ?Head (Some (a0, t))
  have a0-eq: a0 = a
    using head by simp
  have record-cont-bound:
    wp-event

```



```

    (record-staged-message a0  $\gg$ 
      ( $\lambda$ -. ?tail  $\gg$  ( $\lambda$ as'. return (a0 # as')))))
    ?Q t  $\leq$  (1 / nnreal size) ^ n
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg$  ?Q None
    by simp
next
  fix uu u
  assume record-out:
    Some (uu, u)  $\in$ 
      set-dist (execute (record-staged-message a0) t)
  have record-unit:
    Some ((), u)  $\in$ 
      set-dist (execute (record-staged-message a0) t)
  using record-out by (cases uu) simp
  have tail-fresh: alpha-vector-path-fresh u as-tail n
proof -
  have fresh-cons:
    alpha-vector-path-fresh s (a # as-tail) (Suc n)
    using Suc.premis(2) unfolding as-eq .
  have recv-a:
    Some (a, t)  $\in$  set-dist (execute receive-alpha-challenge s)
    using recv a0-eq by simp
  have record-a:
    Some ((), u)  $\in$  set-dist (execute (record-staged-message a) t)
    using record-unit a0-eq by simp
  show ?thesis
    by (rule alpha-vector-path-fresh-Cons-tail
      [OF fresh-cons recv-a record-a])
qed
have tail-bound:
  wp-event ?tail
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (as', -)  $\Rightarrow$  as' = as-tail) u
     $\leq$  (1 / nnreal size) ^ n
  by (rule Suc.IH[OF len-tail tail-fresh])
have cont-eq:
  wp-event
    (?tail  $\gg$  ( $\lambda$ as'. return (a0 # as'))) ?Q u =
  wp-event ?tail
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (as', -)  $\Rightarrow$  as' = as-tail) u
  by (simp add: wp-event-bind-return-map a0-eq as-eq
    split: option.splits prod.splits)
show
  wp-event (?tail  $\gg$  ( $\lambda$ as'. return (a0 # as'))) ?Q u
     $\leq$  (1 / nnreal size) ^ n
  unfolding cont-eq by (rule tail-bound)
qed

```

```

show
  wp-event
    (record-staged-message a0  $\gg$ 
      ( $\lambda$ -. ?tail  $\gg$  ( $\lambda$ as'. return (a0 # as'))))
    ?Q t  $\leq$  (1 / nnreal size)  $^n$ 
  by (rule record-cont-bound)
qed
have wp-event (staged-alpha-program (Suc n)) ?Q s
   $\leq$  wp-event receive-alpha-challenge ?Head s *
    (1 / nnreal size)  $^n$ 
  unfolding staged-alpha-program.simps by (rule bind-bound)
also have ...  $\leq$  (1 / nnreal size) * (1 / nnreal size)  $^n$ 
  by (rule mult-right-mono[OF head-bound]) simp
also have ... = (1 / nnreal size)  $^{Suc\ n}$ 
  by simp
finally show ?case .
qed

lemma wp-staged-alpha-program-finite-set-bound-from-path-fresh:
  assumes finite-B: finite B
  and lengths:  $\bigwedge as. as \in B \implies length\ as = n$ 
  and fresh:  $\bigwedge as. as \in B \implies alpha\text{-vector-path-fresh}\ s\ as\ n$ 
shows
  wp-event (staged-alpha-program n)
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (as, -)  $\Rightarrow as \in B$ ) s  $\leq$ 
    nnreal (card B) / (nnreal size)  $^n$ 
proof -
  let ?P =
     $\lambda a$  out. case out of None  $\Rightarrow$  False
      | Some (as', -)  $\Rightarrow as' = a$ 
  have event-mono:
    wp-event (staged-alpha-program n)
      ( $\lambda$ out. case out of None  $\Rightarrow$  False
        | Some (as, -)  $\Rightarrow as \in B$ ) s  $\leq$ 
    wp-event (staged-alpha-program n)
      ( $\lambda$ out.  $\exists as \in B. ?P\ as\ out$ ) s
  by (rule wp-event-mono) (auto split: option.splits prod.splits)
also have ...  $\leq$ 
  ( $\sum a \in B. wp\text{-event}\ (staged\text{-alpha-program}\ n)\ ( ?P\ a)\ s$ )
  by (rule wp-event-finite-UN-bound[where A = B and P = ?P])
  (use finite-B in simp-all)
also have ...  $\leq$  ( $\sum as \in B. (1 / nnreal\ size)^n$ )
proof (rule sum-mono)
  fix as
  assume as-in: as  $\in B$ 
  show
    wp-event (staged-alpha-program n) (?P as) s  $\leq$ 
    (1 / nnreal size)  $^n$ 

```

by (rule wp-staged-alpha-program-exact-challenges-bound-from-path-fresh
 [OF lengths[OF as-in] fresh[OF as-in]])
 qed
 also have ... = nnreal (card B) * ((1 / nnreal size) ^ n)
 using finite-B by simp
 also have ... = nnreal (card B) / (nnreal size) ^ n
 using size-card by transfer (simp add: power-divide)
 finally show ?thesis .
 qed

lemma alpha-vector-prequeried-in-state-imp-hash-relation-hit:
 assumes prequeried: alpha-vector-prequeried-in-state s as n
 and member: as ∈ B
 and initial-absent:
 ∧ i. i < n ⇒ i < length as ⇒
 fmlookup (HashMap initial) (alpha-challenge-key-at s i as) = None
 shows hash-relation-hit (alpha-vector-key-relation s B n) initial s
proof –
 from prequeried obtain i where
 i-n: i < n
 and i-len: i < length as
 and lookup:
 fmlookup (HashMap s) (alpha-challenge-key-at s i as) =
 Some (as ! i)
 unfolding alpha-vector-prequeried-in-state-def by blast
 have relation:
 alpha-vector-key-relation s B n (alpha-challenge-key-at s i as)
 (as ! i)
 unfolding alpha-vector-key-relation-def
 using member i-n i-len by blast
 show ?thesis
 unfolding hash-relation-hit-def
 using initial-absent[OF i-n i-len] lookup relation by blast
 qed

lemma alpha-vector-prequeried-in-state-imp-hash-relation-hit-empty-initial:
 assumes prequeried: alpha-vector-prequeried-in-state s as n
 and member: as ∈ B
 and empty: HashMap initial = fmempty
 shows hash-relation-hit (alpha-vector-key-relation s B n) initial s
 by (rule alpha-vector-prequeried-in-state-imp-hash-relation-hit
 [OF prequeried member])
 (simp add: empty)

lemma staged-alpha-program-outcome-lookup:
 assumes outcome:
 Some (as, t) ∈ set-dist (execute (staged-alpha-program n) s)
 and i-bound: i < n
 shows

```

    fmlookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s + i)
        (foldl concat (PState s) (take i as))) = Some (as ! i)
  using outcome i-bound
proof (induction n arbitrary: as s t i)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from staged-alpha-program-Suc-outcomeE[OF Suc.premis(1)]
  obtain a as' s1 s2 where
    challenge-out:
      Some (a, s1) ∈ set-dist (execute receive-alpha-challenge s)
    and record-out:
      Some ((), s2) ∈ set-dist (execute (record-staged-message a) s1)
    and rest-out:
      Some (as', t) ∈
        set-dist (execute (staged-alpha-program n) s2)
    and as-eq: as = a # as'
  .
  have challenge-lookup:
    fmlookup (HashMap s1)
      (AlphaChallenge (PAlphaCounter s) (PState s)) = Some a
  using receive-alpha-challenge-outcome[OF challenge-out] by blast
  have challenge-counter:
    PAlphaCounter s1 = Suc (PAlphaCounter s)
  using receive-alpha-challenge-counter-outcome[OF challenge-out] by simp
  have record-eq:
    s2 = s1 (|
      PState := concat (PState s1) a,
      PTranscript := PTranscript s1 @ [a])
  by (rule record-staged-message-outcome[OF record-out])
  have s2-fields:
    HashMap s2 = HashMap s1
    PState s2 = concat (PState s) a
    PAlphaCounter s2 = Suc (PAlphaCounter s)
  unfolding record-eq
  using receive-alpha-challenge-outcome[OF challenge-out]
    challenge-counter
  by simp-all
  have rest-ext: s2 ≤ t
proof -
  have hash-target-program {} n (staged-alpha-program n)
  by (rule hash-target-program-staged-alpha-program)
  then show ?thesis
  using rest-out
  unfolding hash-target-program-def hash-extension-preserving-def
  by blast
qed

```

```

show ?case
proof (cases i)
  case 0
  have lookup-s2:
    fmlookup (HashMap s2)
      (AlphaChallenge (PAlphaCounter s) (PState s)) = Some a
    using challenge-lookup unfolding s2-fields by simp
  have lookup-t:
    fmlookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s) (PState s)) = Some a
    by (rule hash-extension-lookup[OF lookup-s2 rest-ext])
  then show ?thesis
    using 0 as-eq by simp
next
  case (Suc j)
  then have j-bound:  $j < n$ 
    using Suc.premis(2) by simp
  have rest-lookup:
    fmlookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s2 + j)
        (foldl concat (PState s2) (take j as'))) = Some (as' ! j)
    by (rule Suc.IH[OF rest-out j-bound])
  show ?thesis
    using rest-lookup
    unfolding Suc as-eq s2-fields
    by simp
qed
qed

lemma staged-alpha-program-outcome-not-path-fresh-imp-prequeried:
  assumes outcome:
    Some (as, t)  $\in$  set-dist (execute (staged-alpha-program n) s)
    and not-fresh:  $\neg$  alpha-vector-path-fresh s as n
  shows alpha-vector-prequeried-in-state s as n
proof -
  from not-fresh obtain i old where
    i-bound:  $i < n$ 
    and i-len:  $i < \text{length } as$ 
    and lookup-s:
      fmlookup (HashMap s) (alpha-challenge-key-at s i as) = Some old
    unfolding alpha-vector-path-fresh-def
    by (metis option.exhaust)
  have lookup-t:
    fmlookup (HashMap t) (alpha-challenge-key-at s i as) =
      Some (as ! i)
    using staged-alpha-program-outcome-lookup[OF outcome i-bound]
    unfolding alpha-challenge-key-at-def .
  have ext:  $s \leq t$ 
proof -

```

```

have hash-target-program {} n (staged-alpha-program n)
  by (rule hash-target-program-staged-alpha-program)
then show ?thesis
  using outcome
  unfolding hash-target-program-def hash-extension-preserving-def
  by blast
qed
have lookup-t-old:
  fmlookup (HashMap t) (alpha-challenge-key-at s i as) = Some old
  by (rule hash-extension-lookup[OF lookup-s ext])
have old-eq: old = as ! i
  using lookup-t lookup-t-old by simp
show ?thesis
  unfolding alpha-vector-prequeried-in-state-def
  using i-bound i-len lookup-s old-eq by blast
qed

lemma wp-staged-alpha-program-set-bound-from-path-fresh-or-prequery:
  assumes subset:  $B \subseteq \text{alpha-space}$ 
  shows
    wp-event (staged-alpha-program (length spec))
      ( $\lambda \text{out. case out of None} \Rightarrow \text{False}$ 
        |  $\text{Some (as, -)} \Rightarrow \text{as} \in B$ ) s  $\leq$ 
        nnreal (card B) / nnreal (CARD('f) ^ length spec) +
        (if  $\exists \text{as} \in B.$ 
          alpha-vector-prequeried-in-state s as (length spec)
        then 1 else 0)
  proof (cases  $\exists \text{as} \in B.$ 
    alpha-vector-prequeried-in-state s as (length spec))
  case True
  have wp-event (staged-alpha-program (length spec))
    ( $\lambda \text{out. case out of None} \Rightarrow \text{False}$ 
      |  $\text{Some (as, -)} \Rightarrow \text{as} \in B$ ) s  $\leq$  1
    by (rule wp-event-le-1)
  also have ...  $\leq$ 
    nnreal (card B) / nnreal (CARD('f) ^ length spec) + 1
    by simp
  finally show ?thesis
    using True by simp
next
  case False
  let ?BF =
    {as  $\in$  B. alpha-vector-path-fresh s as (length spec)}
  have finite-B: finite B
    by (rule finite-subset[OF subset])
    (simp add: alpha-space-def finite-length-lists-UNIV)
  have finite-BF: finite ?BF
    using finite-B by simp
  have lengths:

```

```

 $\wedge as. as \in ?BF \implies length\ as = length\ spec$ 
using subset unfolding alpha-space-def by auto
have fresh:
 $\wedge as. as \in ?BF \implies$ 
   $alpha\text{-}vector\text{-}path\text{-}fresh\ s\ as\ (length\ spec)$ 
by simp
have event-mono:
  wp-event (staged-alpha-program (length spec))
    ( $\lambda out. case\ out\ of\ None \Rightarrow False$ 
      |  $Some\ (as, -) \Rightarrow as \in B$ ) s  $\leq$ 
    wp-event (staged-alpha-program (length spec))
      ( $\lambda out. case\ out\ of\ None \Rightarrow False$ 
        |  $Some\ (as, -) \Rightarrow as \in ?BF$ ) s
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out  $\in set\text{-}dist$ 
  (execute (staged-alpha-program (length spec)) s)
and hit:
  ( $case\ out\ of\ None \Rightarrow False$ 
    |  $Some\ (as, -) \Rightarrow as \in B$ )
show
  ( $case\ out\ of\ None \Rightarrow False$ 
    |  $Some\ (as, -) \Rightarrow as \in ?BF$ )
proof (cases out)
  case None
  then show ?thesis using hit by simp
next
  case (Some result)
  then obtain as t where out-eq: out = Some (as, t)
  by (cases result) simp
  have as-in: as  $\in B$ 
  using hit unfolding out-eq by simp
  have outcome:
     $Some\ (as, t) \in$ 
     $set\text{-}dist\ (execute\ (staged\text{-}alpha\text{-}program\ (length\ spec))\ s)$ 
    using support unfolding out-eq .
  have path-fresh:
     $alpha\text{-}vector\text{-}path\text{-}fresh\ s\ as\ (length\ spec)$ 
  proof (rule ccontr)
  assume  $\neg\ alpha\text{-}vector\text{-}path\text{-}fresh\ s\ as\ (length\ spec)$ 
  then have alpha-vector-prequeried-in-state s as (length spec)
  by (rule
    staged-alpha-program-outcome-not-path-fresh-imp-prequeried
    [OF outcome])
  then show False
  using False as-in by blast
qed
show ?thesis

```

```

      unfolding out-eq using as-in path-fresh by simp
    qed
  qed
  also have ... ≤
    nnreal (card ?BF) / (nnreal size) ^ length spec
  by (rule wp-staged-alpha-program-finite-set-bound-from-path-fresh
    [OF finite-BF lengths fresh])
  also have ... ≤ nnreal (card B) / (nnreal size) ^ length spec
  proof -
    have card ?BF ≤ card B
      by (rule card-mono[OF finite-B]) blast
    then show ?thesis
      apply (subst nn2real-le-iff[symmetric])
      by (simp add: divide-right-mono)
    qed
  also have ... =
    nnreal (card B) / nnreal (CARD('f) ^ length spec)
  using size-card by simp
  finally show ?thesis
    using False by simp
  qed

lemma wp-staged-alpha-program-set-bound-excluding-prequeried:
  assumes subset: B ⊆ alpha-space
  shows
    wp-event (staged-alpha-program (length spec))
      (λout. case out of None ⇒ False
        | Some (as, -) ⇒
          as ∈ B ∧
            ¬ alpha-vector-prequeried-in-state s as (length spec))
      s ≤ nnreal (card B) / nnreal (CARD('f) ^ length spec)
  proof -
    let ?BF =
      {as ∈ B. alpha-vector-path-fresh s as (length spec)}
    have finite-B: finite B
      by (rule finite-subset[OF subset])
      (simp add: alpha-space-def finite-length-lists-UNIV)
    have finite-BF: finite ?BF
      using finite-B by simp
    have lengths:
      ∧ as. as ∈ ?BF ⇒ length as = length spec
    using subset unfolding alpha-space-def by auto
    have fresh:
      ∧ as. as ∈ ?BF ⇒
        alpha-vector-path-fresh s as (length spec)
    by simp
    have event-mono:
      wp-event (staged-alpha-program (length spec))
        (λout. case out of None ⇒ False

```



```

| Some (as, -)  $\Rightarrow$ 
  as  $\in B \wedge$ 
   $\neg$  alpha-vector-prequeried-in-state s as (length spec))
s  $\leq$ 
wp-event (staged-alpha-program (length spec))
( $\lambda$ out. case out of None  $\Rightarrow$  False
| Some (as, -)  $\Rightarrow$  as  $\in$  ?BF) s
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out  $\in$  set-dist
  (execute (staged-alpha-program (length spec)) s)
and hit:
  (case out of None  $\Rightarrow$  False
  | Some (as, -)  $\Rightarrow$ 
    as  $\in B \wedge$ 
     $\neg$  alpha-vector-prequeried-in-state s as (length spec))
show
  (case out of None  $\Rightarrow$  False
  | Some (as, -)  $\Rightarrow$  as  $\in$  ?BF)
proof (cases out)
  case None
  then show ?thesis using hit by simp
next
  case (Some result)
  then obtain as t where out-eq: out = Some (as, t)
  by (cases result) simp
  have as-in: as  $\in B$ 
  and not-prequeried:
     $\neg$  alpha-vector-prequeried-in-state s as (length spec)
  using hit unfolding out-eq by simp-all
  have outcome:
    Some (as, t)  $\in$ 
    set-dist (execute (staged-alpha-program (length spec)) s)
  using support unfolding out-eq .
  have path-fresh:
    alpha-vector-path-fresh s as (length spec)
  proof (rule ccontr)
    assume  $\neg$  alpha-vector-path-fresh s as (length spec)
    then have alpha-vector-prequeried-in-state s as (length spec)
    by (rule
      staged-alpha-program-outcome-not-path-fresh-imp-prequeried
      [OF outcome])
    then show False using not-prequeried by contradiction
  qed
show ?thesis
  unfolding out-eq using as-in path-fresh by simp
qed
qed

```

```

also have ...  $\leq$ 
   $\text{nnreal } (\text{card } ?BF) / (\text{nnreal size}) \wedge \text{length spec}$ 
by (rule wp-staged-alpha-program-finite-set-bound-from-path-fresh
  [OF finite-BF lengths fresh])
also have ...  $\leq \text{nnreal } (\text{card } B) / (\text{nnreal size}) \wedge \text{length spec}$ 
proof -
  have  $\text{card } ?BF \leq \text{card } B$ 
    by (rule card-mono[OF finite-B]) blast
  then show ?thesis
    apply (subst nn2real-le-iff[symmetric])
    by (simp add: divide-right-mono)
qed
also have ... =
   $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}('f) \wedge \text{length spec})$ 
  using size-card by simp
finally show ?thesis .
qed

lemma staged-after-alpha-prefix-program-alpha-list-bound-from-path-fresh-or-prequery:
assumes subset:  $B \subseteq \text{alpha-space}$ 
shows
  wp-event
    (staged-after-alpha-prefix-program A
      (fr, trace-roots, trace-bs, trace-final))
    (staged-transcript-alpha-list-hit B) s  $\leq$ 
     $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}('f) \wedge \text{length spec}) +$ 
    (if  $\exists as \in B.$ 
      alpha-vector-prequeried-in-state s as (length spec)
      then 1 else 0)
proof -
  have alpha-bound:
    wp-event (staged-alpha-program (length spec))
    ( $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
      |  $\text{Some } (as, -) \Rightarrow as \in B$ ) s  $\leq$ 
     $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}('f) \wedge \text{length spec}) +$ 
    (if  $\exists as \in B.$ 
      alpha-vector-prequeried-in-state s as (length spec)
      then 1 else 0)
  by (rule wp-staged-alpha-program-set-bound-from-path-fresh-or-prequery
    [OF subset])
show ?thesis
  by (rule staged-after-alpha-prefix-program-alpha-list-bound-from-alpha-program
    [OF alpha-bound])
qed

end

end

```

```

theory Staged-Security-Experiment-Composition
imports
  Staged-Security-Experiment-Checked
  Staged-Security-Experiment-Alpha-Prequery
begin

```

Prefix-fixed composition alpha targets for staged soundness.

```

context soundness
begin

```

```

definition checked-staged-after-alpha-prefix-program
:: 'f staged-adversary  $\Rightarrow$ 
  ('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\Rightarrow$ 
  ('f staged-proof-data, 'f protocol-channel) state-monad
where
  checked-staged-after-alpha-prefix-program A prefix =
    (case prefix of (fr, trace-roots, trace-bs, trace-final)  $\Rightarrow$ 
      do {
        as  $\leftarrow$  staged-alpha-program (length spec);
        dg  $\leftarrow$  degree-stage A as;
        record-staged-message dg;
        let composition-rounds = ceil-log (to-nat dg + 1);
        assert (composition-rounds  $\leq$  ceil-log (maxDegree + 1));
        (composition-roots, composition-bs)  $\leftarrow$ 
          staged-composition-fri-program A dg 0 composition-rounds [];
        composition-final  $\leftarrow$ 
          composition-final-stage A dg composition-bs;
        record-staged-message composition-final;
        query-chunks  $\leftarrow$ 
          checked-staged-query-program A trace-roots composition-roots
            0 rounds;
        return
          (|staged-trace-root = fr,
            staged-trace-fri-roots = trace-roots,
            staged-trace-fri-challenges = trace-bs,
            staged-trace-final = trace-final,
            staged-alphas = as,
            staged-degree = dg,
            staged-composition-fri-roots = composition-roots,
            staged-composition-fri-challenges = composition-bs,
            staged-composition-final = composition-final,
            staged-query-chunks = query-chunks|)
      })

```

```

lemma checked-staged-transcript-program-alpha-prefix-decomp:
checked-staged-transcript-program A =
  staged-alpha-prefix-program A  $\gg$ 
  checked-staged-after-alpha-prefix-program A

```

```

unfolding checked-staged-transcript-program-def
  staged-alpha-prefix-program-def
  checked-staged-after-alpha-prefix-program-def Let-def
by (simp add: sm-bind-assoc split-def split: prod.splits)

```

lemma *sm-bind-get-ignore:*

```

(get  $\gg=$  ( $\lambda\cdot. m$ )) = m
apply transfer
apply (rule ext)
apply (rule dist-inject[THEN iffD1])
unfolding dist-bind.rep-eq dist-get-def
apply (simp add: o-def dist-delta-dist)
apply (subst map-bind-delta-left-state[where f= $\lambda s. \text{Some } (s, s)$ ])
apply (auto simp: bind-cont-map-def split: option.splits prod.splits)
done

```

lemma *checked-staged-after-alpha-prefix-program-alpha-list-bound-from-alpha-program:*

assumes *alpha-bound:*

```

wp-event (staged-alpha-program (length spec))
  ( $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
    |  $\text{Some } (as, -) \Rightarrow as \in B$ )  $s \leq C$ 

```

shows

```

wp-event (checked-staged-after-alpha-prefix-program A prefix)
  (staged-transcript-alpha-list-hit B)  $s \leq C$ 

```

proof –

obtain *fr trace-roots trace-bs trace-final* **where** *prefix-eq:*

```

prefix = (fr, trace-roots, trace-bs, trace-final)

```

by (*cases prefix*) *auto*

let *?P* =

```

 $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
  |  $\text{Some } (as, -) \Rightarrow as \in B$ 

```

have *after-eq:*

```

checked-staged-after-alpha-prefix-program A prefix =
  staged-alpha-program (length spec)  $\gg=$ 
  ( $\lambda as. \text{degree-stage } A \text{ as} \gg=$ 
    ( $\lambda dg. \text{record-staged-message } dg \gg=$ 
      ( $\lambda\cdot. \text{let composition-rounds} = \text{ceil-log } (to\text{-nat } dg + 1)$ 
        in assert
          ( $\text{composition-rounds} \leq \text{ceil-log } (maxDegree + 1)$ )  $\gg=$ 
            ( $\lambda\cdot. \text{staged-composition-fri-program } A \text{ dg } 0$ 
              composition-rounds  $\square \gg=$ 
                ( $\lambda(\text{composition-roots}, \text{composition-bs}).$ 
                  composition-final-stage A dg composition-bs  $\gg=$ 
                    ( $\lambda \text{composition-final.}$ 
                      record-staged-message composition-final  $\gg=$ 
                        ( $\lambda\cdot. \text{checked-staged-query-program } A \text{ trace-roots}$ 
                          composition-roots 0 rounds  $\gg=$ 
                            ( $\lambda \text{query-chunks.}$ 
                              return

```

```

    (|staged-trace-root = fr,
     staged-trace-fri-roots = trace-roots,
     staged-trace-fri-challenges = trace-bs,
     staged-trace-final = trace-final,
     staged-alphas = as,
     staged-degree = dg,
     staged-composition-fri-roots =
       composition-roots,
     staged-composition-fri-challenges =
       composition-bs,
     staged-composition-final = composition-final,
     staged-query-chunks =
       query-chunks|)))))))))
  unfolding checked-staged-after-alpha-prefix-program-def prefix-eq Let-def
  by simp
show ?thesis
  unfolding after-eq staged-transcript-alpha-list-hit-def
proof (rule wp-event-bind-bound-by-head-event[OF alpha-bound])
  show (case None of None  $\Rightarrow$  False
        | Some (data, -)  $\Rightarrow$  staged-alphas data  $\in$  B)  $\Rightarrow$ 
    ?P None
  by simp
next
fix as t out
assume cont:
  out  $\in$ 
  set-dist
  (execute
    (degree-stage A as  $\gg$ 
     ( $\lambda$ dg. record-staged-message dg  $\gg$ 
      ( $\lambda$ -. let composition-rounds =
        ceil-log (to-nat dg + 1)
        in assert
          (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
          ( $\lambda$ -. staged-composition-fri-program A dg 0
            composition-rounds []  $\gg$ 
            ( $\lambda$ (composition-roots, composition-bs).
              composition-final-stage A dg composition-bs  $\gg$ 
              ( $\lambda$ composition-final.
                record-staged-message composition-final  $\gg$ 
                ( $\lambda$ -. checked-staged-query-program A trace-roots
                  composition-roots 0 rounds  $\gg$ 
                  ( $\lambda$ query-chunks.
                    return
                      (|staged-trace-root = fr,
                       staged-trace-fri-roots = trace-roots,
                       staged-trace-fri-challenges = trace-bs,
                       staged-trace-final = trace-final,
                       staged-alphas = as,

```

```

    staged-degree = dg,
    staged-composition-fri-roots =
      composition-roots,
    staged-composition-fri-challenges =
      composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks =
      query-chunks)))))))))
  t)
and hit:
  (case out of None  $\Rightarrow$  False
   | Some (data, -)  $\Rightarrow$  staged-alphas data  $\in$  B)
show ?P (Some (as, t))
proof (cases out)
  case None
  then show ?thesis using hit by simp
next
  case (Some result)
  then obtain data u where out-eq: out = Some (data, u)
  by (cases result) simp
have data-as: staged-alphas data = as
  using cont unfolding out-eq Let-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have staged-alphas data  $\in$  B
  using hit unfolding out-eq by simp
then show ?thesis
  using data-as by simp
qed
qed
qed

lemma checked-staged-after-alpha-prefix-program-alpha-list-bound-from-path-fresh-or-prequery:
  assumes subset:  $B \subseteq \text{alpha-space}$ 
  shows
    wp-event (checked-staged-after-alpha-prefix-program A prefix)
      (staged-transcript-alpha-list-hit B)  $s \leq$ 
      nnreal (card B) / nnreal (CARD('f)  $\wedge$  length spec) +
      (if  $\exists as \in B.$ 
        alpha-vector-prequeried-in-state s as (length spec)
      then 1 else 0)
proof -
  have alpha-bound:
    wp-event (staged-alpha-program (length spec))
      ( $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
       | Some (as, -)  $\Rightarrow as \in B$ )  $s \leq$ 
      nnreal (card B) / nnreal (CARD('f)  $\wedge$  length spec) +
      (if  $\exists as \in B.$ 
        alpha-vector-prequeried-in-state s as (length spec)
      then 1 else 0)

```

```

    by (rule wp-staged-alpha-program-set-bound-from-path-fresh-or-prequery
        [OF subset])
  show ?thesis
    by (rule
        checked-staged-after-alpha-prefix-program-alpha-list-bound-from-alpha-program
        [OF alpha-bound])
qed

lemma checked-staged-after-alpha-prefix-program-alpha-list-bound-excluding-prequeried:
  assumes subset:  $B \subseteq \text{alpha-space}$ 
  shows
    wp-event (checked-staged-after-alpha-prefix-program A prefix)
      ( $\lambda \text{out. case out of None} \Rightarrow \text{False}$ 
        |  $\text{Some (data, -)} \Rightarrow$ 
          staged-alphas data  $\in B \wedge$ 
             $\neg \text{alpha-vector-prequeried-in-state } s$ 
            (staged-alphas data) (length spec))
       $s \leq \text{nnreal (card } B) / \text{nnreal (CARD('f) } \wedge \text{length spec)}$ 
  proof -
    obtain fr trace-roots trace-bs trace-final where prefix-eq:
      prefix = (fr, trace-roots, trace-bs, trace-final)
    by (cases prefix) auto
    let ?P =
       $\lambda \text{out. case out of None} \Rightarrow \text{False}$ 
      |  $\text{Some (as, -)} \Rightarrow$ 
        as  $\in B \wedge$ 
           $\neg \text{alpha-vector-prequeried-in-state } s \text{ as (length spec)}$ 
    have alpha-bound:
      wp-event (staged-alpha-program (length spec))
        ?P  $s \leq \text{nnreal (card } B) / \text{nnreal (CARD('f) } \wedge \text{length spec)}$ 
    by (rule wp-staged-alpha-program-set-bound-excluding-prequeried
        [OF subset])
    have after-eq:
      checked-staged-after-alpha-prefix-program A prefix =
        staged-alpha-program (length spec)  $\gg=$ 
        ( $\lambda \text{as. degree-stage } A \text{ as} \gg=$ 
          ( $\lambda \text{dg. record-staged-message dg} \gg=$ 
            ( $\lambda -. \text{let composition-rounds} = \text{ceil-log (to-nat dg + 1)}$ 
              in assert
                (composition-rounds  $\leq \text{ceil-log (maxDegree + 1)}$ )  $\gg=$ 
                ( $\lambda -. \text{staged-composition-fri-program } A \text{ dg } 0$ 
                  composition-rounds  $\square \gg=$ 
                  ( $\lambda (\text{composition-roots, composition-bs}).$ 
                    composition-final-stage A dg composition-bs  $\gg=$ 
                    ( $\lambda \text{composition-final.}$ 
                      record-staged-message composition-final  $\gg=$ 
                      ( $\lambda -. \text{checked-staged-query-program } A \text{ trace-roots}$ 
                        composition-roots 0 rounds  $\gg=$ 
                        ( $\lambda \text{query-chunks.}$ 

```

```

    return
    (|staged-trace-root = fr,
     staged-trace-fri-roots = trace-roots,
     staged-trace-fri-challenges = trace-bs,
     staged-trace-final = trace-final,
     staged-alphas = as,
     staged-degree = dg,
     staged-composition-fri-roots =
       composition-roots,
     staged-composition-fri-challenges =
       composition-bs,
     staged-composition-final = composition-final,
     staged-query-chunks =
       query-chunks|)))))))))
  unfolding checked-staged-after-alpha-prefix-program-def prefix-eq Let-def
  by simp
show ?thesis
  unfolding after-eq
proof (rule wp-event-bind-bound-by-head-event[OF alpha-bound])
  show (case None of None  $\Rightarrow$  False
        | Some (data, -)  $\Rightarrow$ 
          staged-alphas data  $\in B \wedge$ 
           $\neg$  alpha-vector-prequeried-in-state s
          (staged-alphas data) (length spec))  $\implies$ 
    ?P None
  by simp
next
fix as t out
assume cont:
  out  $\in$ 
  set-dist
  (execute
   (degree-stage A as  $\gg$ 
    ( $\lambda$ dg. record-staged-message dg  $\gg$ 
     ( $\lambda$ -. let composition-rounds =
          ceil-log (to-nat dg + 1)
        in assert
          (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
          ( $\lambda$ -. staged-composition-fri-program A dg 0
            composition-rounds []  $\gg$ 
             ( $\lambda$ (composition-roots, composition-bs).
              composition-final-stage A dg composition-bs  $\gg$ 
              ( $\lambda$ composition-final.
               record-staged-message composition-final  $\gg$ 
               ( $\lambda$ -. checked-staged-query-program A trace-roots
                 composition-roots 0 rounds  $\gg$ 
                 ( $\lambda$ query-chunks.
                  return
                   (|staged-trace-root = fr,

```



```

    staged-trace-fri-roots = trace-roots,
    staged-trace-fri-challenges = trace-bs,
    staged-trace-final = trace-final,
    staged-alphas = as,
    staged-degree = dg,
    staged-composition-fri-roots =
      composition-roots,
    staged-composition-fri-challenges =
      composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks =
      query-chunks)))))))))
  t)
and hit:
  (case out of None  $\Rightarrow$  False
   | Some (data, -)  $\Rightarrow$ 
     staged-alphas data  $\in B \wedge$ 
      $\neg$  alpha-vector-prequeried-in-state s
     (staged-alphas data) (length spec))
show ?P (Some (as, t))
proof (cases out)
  case None
  then show ?thesis using hit by simp
next
  case (Some result)
  then obtain data u where out-eq: out = Some (data, u)
  by (cases result) simp
have data-as: staged-alphas data = as
  using cont unfolding out-eq Let-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have staged-alphas data  $\in B$ 
   $\neg$  alpha-vector-prequeried-in-state s
  (staged-alphas data) (length spec)
  using hit unfolding out-eq by simp-all
then show ?thesis
  using data-as by simp
qed
qed
qed

definition checked-staged-transcript-with-alpha-prefix-program
:: 'f staged-adversary  $\Rightarrow$ 
  (((('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
   'f protocol-channel)  $\times$  'f staged-proof-data,
   'f protocol-channel) state-monad
where
checked-staged-transcript-with-alpha-prefix-program A =
  staged-alpha-prefix-program A  $\gg=$ 
  ( $\lambda$ prefix. get  $\gg=$ 

```

(λ prefix-state.
 checked-staged-after-alpha-prefix-program A prefix $\gg=$
 (λ data. return ((prefix, prefix-state), data))))

definition checked-staged-security-experiment-with-actual-alpha-prefix-data-state

:: 'f staged-adversary $\Rightarrow$
 (((('f $\times$ 'f list $\times$ 'f list $\times$ 'f) $\times$
 'f protocol-channel) $\times$ 'f staged-proof-data) $\times$
 'f protocol-channel) $\times$ unit list,
 'f protocol-channel) state-monad

where

checked-staged-security-experiment-with-actual-alpha-prefix-data-state $A =$
 checked-staged-transcript-with-alpha-prefix-program $A \gg=$
 (λ packed. get $\gg=$
 (λ s.
 put
 (verifier-state-from-adversary s
 (staged-proof-transcript (snd packed))) $\gg=$
 (λ -.
 verify-monad $\gg=$
 (λ result. return ((packed, s), result))))))

lemma checked-staged-transcript-with-alpha-prefix-program-projection:

checked-staged-transcript-with-alpha-prefix-program $A \gg=$
 (λ packed. return (snd packed)) =
 checked-staged-transcript-program A

unfolding checked-staged-transcript-with-alpha-prefix-program-def

checked-staged-transcript-program-alpha-prefix-decomp

by (simp add: sm-bind-assoc sm-bind-get-ignore)

lemma checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection:

checked-staged-security-experiment-with-actual-alpha-prefix-data-state $A \gg=$
 (λ x. return ((snd (fst (fst x)), snd (fst x)), snd x)) =
 checked-staged-security-experiment-with-data-state A

proof –

let ?K =
 λ data. get $\gg=$
 (λ s. put
 (verifier-state-from-adversary s
 (staged-proof-transcript data)) $\gg=$
 (λ -. verify-monad $\gg=$
 (λ result. return ((data, s), result))))

have

checked-staged-security-experiment-with-actual-alpha-prefix-data-state $A \gg=$
 (λ x. return ((snd (fst (fst x)), snd (fst x)), snd x)) =
 checked-staged-transcript-with-alpha-prefix-program $A \gg=$
 (λ packed. ?K (snd packed))

unfolding

checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def

```

    by (simp add: sm-bind-assoc split-def)
  also have ... =
    (checked-staged-transcript-with-alpha-prefix-program A  $\gg$ 
      ( $\lambda$ packed. return (snd packed)))  $\gg$  ?K
    by (simp add: sm-bind-assoc)
  also have ... =
    checked-staged-transcript-program A  $\gg$  ?K
  unfolding checked-staged-transcript-with-alpha-prefix-program-projection
  by simp
  also have ... =
    checked-staged-security-experiment-with-data-state A
  unfolding checked-staged-security-experiment-with-data-state-def by simp
  finally show ?thesis .
qed

```

lemma *alpha-prefix-hash-extends-verifier-state-from-adversary-right:*

```

  assumes ext:  $s \leq t$ 
  shows  $s \leq \text{verifier-state-from-adversary } t \text{ tr}$ 
  using ext
  unfolding verifier-state-from-adversary-def less-eq-hash-ext-def
    less-eq-fmap-def
  by simp

```

definition *alpha-prefix-trace-table-candidates*

```

:: 'f protocol-channel  $\Rightarrow$  'f  $\Rightarrow$  'f list set
where
  alpha-prefix-trace-table-candidates prefix-state fr =
    {trace-table.
      length trace-table = clength * scale  $\wedge$ 
      merkle-root-binds-table fr trace-table prefix-state}

```

lemma *alpha-prefix-trace-table-candidateI:*

```

  assumes len: length trace-table = clength * scale
  and bind: merkle-root-binds-table fr trace-table prefix-state
  shows trace-table  $\in$  alpha-prefix-trace-table-candidates prefix-state fr
  using assms unfolding alpha-prefix-trace-table-candidates-def by simp

```

definition *alpha-prefix-union-bad-sets*

```

:: 'f protocol-channel  $\Rightarrow$  ('f list  $\Rightarrow$  'f list set)  $\Rightarrow$ 
  'f  $\Rightarrow$  'f list set
where
  alpha-prefix-union-bad-sets prefix-state bad-sets fr =
    {as  $\in$  alpha-space.
       $\exists$  trace-table  $\in$ 
        alpha-prefix-trace-table-candidates prefix-state fr.
        as  $\in$  bad-sets trace-table}

```

definition *checked-staged-transcript-actual-alpha-prefix-bad-set-hit*

```

:: ('f list  $\Rightarrow$  'f list set)  $\Rightarrow$ 

```

$(((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{checked-staged-transcript-actual-alpha-prefix-bad-set-hit bad-sets out}$
 $\longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\mid \text{Some } (((fr, -, -, -), \text{prefix-state}), \text{data}), -) \Rightarrow$
 $\quad \text{staged-alphas data} \in$
 $\quad \text{alpha-prefix-union-bad-sets prefix-state bad-sets fr})$

definition *checked-staged-transcript-actual-alpha-prefix-prequery-hit*

$:: ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{checked-staged-transcript-actual-alpha-prefix-prequery-hit bad-sets out}$
 $\longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\mid \text{Some } (((fr, -, -, -), \text{prefix-state}), \text{data}), -) \Rightarrow$
 $\quad \text{staged-alphas data} \in$
 $\quad \text{alpha-prefix-union-bad-sets prefix-state bad-sets fr} \wedge$
 $\quad \text{alpha-vector-prequeried-in-state prefix-state}$
 $\quad (\text{staged-alphas data}) (\text{length spec}))$

lemma *checked-staged-transcript-actual-alpha-prefix-prequery-hit-imp-bad-set-hit:*

assumes

$\text{checked-staged-transcript-actual-alpha-prefix-prequery-hit bad-sets out}$

shows $\text{checked-staged-transcript-actual-alpha-prefix-bad-set-hit bad-sets out}$

using *assms*

unfolding *checked-staged-transcript-actual-alpha-prefix-prequery-hit-def*

checked-staged-transcript-actual-alpha-prefix-bad-set-hit-def

by (*auto split: option.splits prod.splits*)

definition *checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit*

$:: ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit}$
 $\text{bad-sets out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\mid \text{Some } ((\text{prefix}, \text{prefix-state}), \text{data}), -) \Rightarrow$
 $\quad \neg \text{hash-map-output-collision prefix-state} \wedge$

staged-alphas data $\in$
alpha-prefix-union-bad-sets prefix-state bad-sets (*fst prefix*) $\wedge$
 $\neg$ *alpha-vector-prequeried-in-state prefix-state*
(staged-alphas data) (length spec)

lemma *checked-staged-transcript-actual-alpha-prefix-bad-set-hit-split:*

assumes

checked-staged-transcript-actual-alpha-prefix-bad-set-hit bad-sets out

shows

checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit
bad-sets out $\vee$

checked-staged-transcript-actual-alpha-prefix-prequery-hit bad-sets out $\vee$
(case out of

None $\Rightarrow$ *False*

| *Some (((-, prefix-state), -), -)* $\Rightarrow$

hash-map-output-collision prefix-state)

using *assms*

unfolding *checked-staged-transcript-actual-alpha-prefix-bad-set-hit-def*

checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit-def

checked-staged-transcript-actual-alpha-prefix-prequery-hit-def

by (*auto split: option.splits prod.splits*)

lemma *alpha-prefix-union-bad-sets-subset-alpha-space:*

alpha-prefix-union-bad-sets prefix-state bad-sets fr $\subseteq$ *alpha-space*

unfolding *alpha-prefix-union-bad-sets-def* **by** *auto*

lemma *alpha-prefix-union-bad-sets-finite:*

finite (alpha-prefix-union-bad-sets prefix-state bad-sets fr)

by (*rule finite-subset[OF alpha-prefix-union-bad-sets-subset-alpha-space*
finite-alpha-space])

lemma *alpha-prefix-union-bad-sets-fraction-bound-if-unique-candidate:*

fixes *C :: prob*

assumes *unique:*

$\exists$ *trace-table.*

alpha-prefix-trace-table-candidates prefix-state fr $\subseteq$
 $\{trace-table\}$

and *subset:* $\bigwedge trace-table. bad-sets trace-table \subseteq alpha-space$

and *bound:*

$\bigwedge trace-table.$

$nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space) \leq C$

shows

$nnreal (card (alpha-prefix-union-bad-sets prefix-state bad-sets fr)) /$
 $nnreal (card alpha-space) \leq C$

proof –

from *unique* **obtain** *trace-table* **where** *candidates:*

alpha-prefix-trace-table-candidates prefix-state fr $\subseteq$
 $\{trace-table\}$

by *blast*

```

have union-subset:
  alpha-prefix-union-bad-sets prefix-state bad-sets fr  $\subseteq$ 
    bad-sets trace-table
  unfolding alpha-prefix-union-bad-sets-def
  using candidates by auto
have finite-bad: finite (bad-sets trace-table)
  by (rule finite-subset[OF subset finite-alpha-space])
have card-le:
  card (alpha-prefix-union-bad-sets prefix-state bad-sets fr)  $\leq$ 
    card (bad-sets trace-table)
  by (rule card-mono[OF finite-bad union-subset])
have nnreal (card (alpha-prefix-union-bad-sets prefix-state bad-sets fr)) /
  nnreal (card alpha-space)  $\leq$ 
  nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
also have ...  $\leq C$ 
  by (rule bound)
finally show ?thesis .
qed

lemma alpha-prefix-trace-table-candidates-unique-if-no-hash-collision:
  assumes clean:  $\neg$  hash-map-output-collision prefix-state
  shows
     $\exists$  trace-table.
      alpha-prefix-trace-table-candidates prefix-state fr  $\subseteq$ 
        {trace-table}
proof (cases alpha-prefix-trace-table-candidates prefix-state fr = {})
  case True
  then show ?thesis
  by blast
next
  case False
  then obtain trace-table where table-in:
    trace-table  $\in$  alpha-prefix-trace-table-candidates prefix-state fr
  by blast
have table-len: length trace-table = clength * scale
  using table-in unfolding alpha-prefix-trace-table-candidates-def by simp
have table-bind: merkle-root-binds-table fr trace-table prefix-state
  using table-in unfolding alpha-prefix-trace-table-candidates-def by simp
have subset-single:
  alpha-prefix-trace-table-candidates prefix-state fr  $\subseteq$ 
    {trace-table}
proof
  fix trace-table'
  assume table'-in:
    trace-table'  $\in$  alpha-prefix-trace-table-candidates prefix-state fr
  have table'-len: length trace-table' = clength * scale
  using table'-in unfolding alpha-prefix-trace-table-candidates-def
  by simp

```

```

have table'-bind:
  merkle-root-binds-table fr trace-table' prefix-state
  using table'-in unfolding alpha-prefix-trace-table-candidates-def
  by simp
have trace-table' = trace-table
  by (rule merkle-root-binds-same-length-tables-unique-if-no-hash-collision
    [OF table'-bind table-bind])
    (use table'-len table-len clean in simp-all)
  then show trace-table'  $\in \{trace-table\}$ 
    by simp
qed
then show ?thesis by blast
qed

```

```

lemma composition-alpha-prefix-union-fraction-bound-if-no-hash-collision:
  assumes clean:  $\neg$  hash-map-output-collision prefix-state
  shows
    nnreal
      (card
        (alpha-prefix-union-bad-sets prefix-state
          composition-trace-bad-alpha-space fr)) /
      nnreal (card alpha-space)  $\leq$  composition-error-bound
  by (rule alpha-prefix-union-bad-sets-fraction-bound-if-unique-candidate)
    (rule alpha-prefix-trace-table-candidates-unique-if-no-hash-collision
      [OF clean],
      rule composition-trace-bad-alpha-space-subset-alpha-space,
      rule composition-trace-bad-alpha-space-fraction-bound-alpha-space)

```

```

lemma alpha-prefix-union-bad-sets-card-bound-if-unique-candidate:
  assumes unique:
     $\exists$  trace-table.
      alpha-prefix-trace-table-candidates prefix-state fr  $\subseteq$ 
        {trace-table}
  and subset:  $\bigwedge$  trace-table. bad-sets trace-table  $\subseteq$  alpha-space
  and bound:
     $\bigwedge$  trace-table. card (bad-sets trace-table)  $\leq C$ 
  shows
    card (alpha-prefix-union-bad-sets prefix-state bad-sets fr)  $\leq C$ 
proof –
  from unique obtain trace-table where candidates:
    alpha-prefix-trace-table-candidates prefix-state fr  $\subseteq$ 
      {trace-table}
  by blast
  have union-subset:
    alpha-prefix-union-bad-sets prefix-state bad-sets fr  $\subseteq$ 
      bad-sets trace-table
  unfolding alpha-prefix-union-bad-sets-def
  using candidates by auto
  have finite-bad: finite (bad-sets trace-table)

```

```

  by (rule finite-subset[OF subset finite-alpha-space])
have card (alpha-prefix-union-bad-sets prefix-state bad-sets fr) ≤
  card (bad-sets trace-table)
  by (rule card-mono[OF finite-bad union-subset])
also have ... ≤ C
  by (rule bound)
finally show ?thesis .
qed

```

lemma *composition-alpha-prefix-union-card-bound-if-no-hash-collision:*

assumes *clean*: $\neg$ hash-map-output-collision prefix-state

shows

```

  card
    (alpha-prefix-union-bad-sets prefix-state
      composition-trace-bad-alpha-space fr) ≤
    (if length spec = 0 then 0 else CARD('f) ^ (length spec - 1))
by (rule alpha-prefix-union-bad-sets-card-bound-if-unique-candidate)
    (rule alpha-prefix-trace-table-candidates-unique-if-no-hash-collision
      [OF clean],
      rule composition-trace-bad-alpha-space-subset-alpha-space,
      rule composition-trace-bad-alpha-space-card-bound)

```

lemma *checked-staged-transcript-with-alpha-prefix-clean-fresh-composition-bound:*

shows

```

  wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit
      composition-trace-bad-alpha-space)
  adversary-initial-state ≤ composition-error-bound

```

unfolding *checked-staged-transcript-with-alpha-prefix-program-def*

proof (rule wp-event-bind-bound-by-cont)

show

```

  ¬ checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit
    composition-trace-bad-alpha-space None

```

unfolding

```

  checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit-def

```

by *simp*

next

fix *prefix prefix-state*

assume *prefix-out*:

```

  Some (prefix, prefix-state) ∈
    set-dist
      (execute (staged-alpha-prefix-program A) adversary-initial-state)

```

obtain *fr trace-roots trace-bs trace-final* **where** *prefix-eq*:

```

  prefix = (fr, trace-roots, trace-bs, trace-final)

```

by (cases *prefix*) *auto*

let *?B* =

```

  alpha-prefix-union-bad-sets prefix-state
  composition-trace-bad-alpha-space fr

```

let *?Q* =


```

checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit
composition-trace-bad-alpha-space
let ?E =
  λout. case out of
    None ⇒ False
  | Some (data, -) ⇒
    ¬ hash-map-output-collision prefix-state ∧
    staged-alphas data ∈ ?B ∧
    ¬ alpha-vector-prequeried-in-state prefix-state
      (staged-alphas data) (length spec)
have get-eq:
  wp-event
  (get >=>
    (λprefix-state'.
      checked-staged-after-alpha-prefix-program A prefix >=>
        (λdata. return ((prefix, prefix-state'), data))))
  ?Q prefix-state =
  wp-event
  (checked-staged-after-alpha-prefix-program A prefix >=>
    (λdata. return ((prefix, prefix-state), data)))
  ?Q
  prefix-state
  unfolding wp-event-def by (simp add: wpsimps)
have return-eq:
  wp-event
  (checked-staged-after-alpha-prefix-program A prefix >=>
    (λdata. return ((prefix, prefix-state), data)))
  ?Q prefix-state =
  wp-event (checked-staged-after-alpha-prefix-program A prefix) ?E
  prefix-state
  apply (subst wp-event-bind-return-map)
  unfolding
    checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit-def
    prefix-eq
  by (simp split: option.splits prod.splits)
have cont-eq:
  wp-event
  (get >=>
    (λprefix-state'.
      checked-staged-after-alpha-prefix-program A prefix >=>
        (λdata. return ((prefix, prefix-state'), data))))
  ?Q prefix-state =
  wp-event (checked-staged-after-alpha-prefix-program A prefix) ?E
  prefix-state
  using get-eq return-eq by simp
show
  wp-event
  (get >=>
    (λprefix-state.

```

```

    checked-staged-after-alpha-prefix-program A prefix  $\gg$ 
      ( $\lambda$ data. return ((prefix, prefix-state), data)))
  ?Q prefix-state  $\leq$  composition-error-bound
proof (cases hash-map-output-collision prefix-state)
case True
have zero:
  wp-event (checked-staged-after-alpha-prefix-program A prefix) ?E
    prefix-state = 0
proof -
  have indicator-zero:
    ( $\lambda$ x. if ?E x then 1 else 0 :: prob) = ( $\lambda$ -. 0)
  using True
  by (auto simp: fun-eq-iff split: option.splits prod.splits)
show ?thesis
  unfolding wp-event-def indicator-zero wp-def by simp
qed
show ?thesis
  unfolding cont-eq zero by simp
next
case False
have subset: ?B  $\subseteq$  alpha-space
  by (rule alpha-prefix-union-bad-sets-subset-alpha-space)
have suffix-bound:
  wp-event (checked-staged-after-alpha-prefix-program A prefix)
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (data, -)  $\Rightarrow$ 
        staged-alphas data  $\in$  ?B  $\wedge$ 
         $\neg$  alpha-vector-prequeried-in-state prefix-state
          (staged-alphas data) (length spec))
    prefix-state  $\leq$ 
    nnreal (card ?B) / nnreal (CARD('f)  $\wedge$  length spec)
  by (rule
    checked-staged-after-alpha-prefix-program-alpha-list-bound-excluding-prequeried
    [OF subset])
have clean-bound:
  nnreal (card ?B) / nnreal (CARD('f)  $\wedge$  length spec)  $\leq$ 
    composition-error-bound
  using composition-alpha-prefix-union-fraction-bound-if-no-hash-collision
    [OF False, of fr]
  by (simp add: card-alpha-space)
have event-eq:
  ?E =
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (data, -)  $\Rightarrow$ 
        staged-alphas data  $\in$  ?B  $\wedge$ 
         $\neg$  alpha-vector-prequeried-in-state prefix-state
          (staged-alphas data) (length spec))
  using False
  by (auto simp: fun-eq-iff split: option.splits prod.splits)

```

```

show ?thesis
  unfolding cont-eq event-eq
  by (rule order-trans[OF suffix-bound clean-bound])
qed
qed

lemma alpha-prefix-composition-vector-key-relation-fiber-bound-if-no-hash-collision:
  assumes clean:  $\neg$  hash-map-output-collision prefix-state
  shows
    card
      {y. alpha-vector-key-relation prefix-state
        (alpha-prefix-union-bad-sets prefix-state
          composition-trace-bad-alpha-space fr)
        (length spec) x y}
       $\leq$  (if length spec = 0 then 0 else CARD('f)  $^{\wedge}$  (length spec - 1)) *
        length spec
proof -
  have finite-bad:
    finite
      (alpha-prefix-union-bad-sets prefix-state
        composition-trace-bad-alpha-space fr)
  by (rule alpha-prefix-union-bad-sets-finite)
have relation-bound:
  card
    {y. alpha-vector-key-relation prefix-state
      (alpha-prefix-union-bad-sets prefix-state
        composition-trace-bad-alpha-space fr)
      (length spec) x y}
     $\leq$ 
    card
      (alpha-prefix-union-bad-sets prefix-state
        composition-trace-bad-alpha-space fr) *
      length spec
  by (rule alpha-vector-key-relation-fiber-card-bound[OF finite-bad])
also have ...  $\leq$ 
  (if length spec = 0 then 0 else CARD('f)  $^{\wedge}$  (length spec - 1)) *
    length spec
proof (cases length spec = 0)
  case True
    then show ?thesis by simp
  next
    case False
      have card
        (alpha-prefix-union-bad-sets prefix-state
          composition-trace-bad-alpha-space fr)
         $\leq$  CARD('f)  $^{\wedge}$  (length spec - 1)
      using composition-alpha-prefix-union-card-bound-if-no-hash-collision
        [OF clean] False
      by simp

```

```

then show ?thesis
using False
by (intro mult-right-mono) simp-all
qed
finally show ?thesis .
qed

definition checked-staged-security-with-data-state-alpha-prefix-bad-set-hit
:: 'f staged-adversary  $\Rightarrow$  ('f list  $\Rightarrow$  'f list set)  $\Rightarrow$ 
  (((('f staged-proof-data  $\times$  'f protocol-channel)  $\times$  unit list)  $\times$ 
   'f protocol-channel) option  $\Rightarrow$  bool
where
checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
bad-sets out  $\longleftrightarrow$ 
(case out of
  None  $\Rightarrow$  False
| Some (((data, -), -), -)  $\Rightarrow$ 
  ( $\exists$  prefix prefix-state.
   Some (prefix, prefix-state)  $\in$ 
    set-dist
      (execute (staged-alpha-prefix-program A)
        adversary-initial-state)  $\wedge$ 
      staged-trace-root data = fst prefix  $\wedge$ 
      staged-trace-fri-roots data = fst (snd prefix)  $\wedge$ 
      staged-trace-fri-challenges data = fst (snd (snd prefix))  $\wedge$ 
      staged-trace-final data = snd (snd (snd prefix))  $\wedge$ 
      staged-alphas data  $\in$ 
        alpha-prefix-union-bad-sets prefix-state bad-sets
        (staged-trace-root data)))

definition checked-staged-transcript-alpha-prefix-prequery-hit
:: 'f staged-adversary  $\Rightarrow$  ('f list  $\Rightarrow$  'f list set)  $\Rightarrow$ 
  (('f staged-proof-data  $\times$  'f protocol-channel) option  $\Rightarrow$  bool
where
checked-staged-transcript-alpha-prefix-prequery-hit A bad-sets out
 $\longleftrightarrow$ 
(case out of
  None  $\Rightarrow$  False
| Some (data, -)  $\Rightarrow$ 
  ( $\exists$  prefix prefix-state.
   Some (prefix, prefix-state)  $\in$ 
    set-dist
      (execute (staged-alpha-prefix-program A)
        adversary-initial-state)  $\wedge$ 
      staged-trace-root data = fst prefix  $\wedge$ 
      staged-trace-fri-roots data = fst (snd prefix)  $\wedge$ 
      staged-trace-fri-challenges data = fst (snd (snd prefix))  $\wedge$ 
      staged-trace-final data = snd (snd (snd prefix))  $\wedge$ 
      staged-alphas data  $\in$ 

```

$\alpha\text{-prefix-union-bad-sets } \text{prefix-state } \text{bad-sets}$
 $(\text{staged-trace-root } \text{data}) \wedge$
 $\alpha\text{-vector-prequeried-in-state } \text{prefix-state}$
 $(\text{staged-alphas } \text{data}) (\text{length spec}))$

definition *checked-staged-transcript-alpha-prefix-bad-set-hit*

$:: 'f \text{ staged-adversary} \Rightarrow ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{checked-staged-transcript-alpha-prefix-bad-set-hit } A \text{ bad-sets out}$
 $\longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\mid \text{Some } (\text{data}, -) \Rightarrow$
 $\quad (\exists \text{ prefix prefix-state.}$
 $\quad \quad \text{Some } (\text{prefix}, \text{prefix-state}) \in$
 $\quad \quad \text{set-dist}$
 $\quad \quad (\text{execute } (\text{staged-alpha-prefix-program } A)$
 $\quad \quad \quad \text{adversary-initial-state}) \wedge$
 $\quad \quad \text{staged-trace-root data} = \text{fst prefix} \wedge$
 $\quad \quad \text{staged-trace-fri-roots data} = \text{fst } (\text{snd prefix}) \wedge$
 $\quad \quad \text{staged-trace-fri-challenges data} = \text{fst } (\text{snd } (\text{snd prefix})) \wedge$
 $\quad \quad \text{staged-trace-final data} = \text{snd } (\text{snd } (\text{snd prefix})) \wedge$
 $\quad \quad \text{staged-alphas data} \in$
 $\quad \quad \alpha\text{-prefix-union-bad-sets } \text{prefix-state } \text{bad-sets}$
 $\quad \quad (\text{staged-trace-root data})))$

lemma *checked-staged-transcript-alpha-prefix-prequery-hit-imp-bad-set-hit:*

assumes *hit:*

$\text{checked-staged-transcript-alpha-prefix-prequery-hit } A \text{ bad-sets out}$

shows $\text{checked-staged-transcript-alpha-prefix-bad-set-hit } A \text{ bad-sets out}$

using *hit*

unfolding *checked-staged-transcript-alpha-prefix-prequery-hit-def*

$\text{checked-staged-transcript-alpha-prefix-bad-set-hit-def}$

by (*auto split: option.splits prod.splits*)

lemma *checked-staged-transcript-alpha-prefix-prequery-hitE:*

assumes *hit:*

$\text{checked-staged-transcript-alpha-prefix-prequery-hit } A \text{ bad-sets}$
 $(\text{Some } (\text{data}, \text{attacker-state}))$

obtains $\text{prefix prefix-state}$ **where**

$\text{Some } (\text{prefix}, \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{staged-alpha-prefix-program } A) \text{ adversary-initial-state})$
 $\text{staged-trace-root data} = \text{fst prefix}$
 $\text{staged-trace-fri-roots data} = \text{fst } (\text{snd prefix})$
 $\text{staged-trace-fri-challenges data} = \text{fst } (\text{snd } (\text{snd prefix}))$
 $\text{staged-trace-final data} = \text{snd } (\text{snd } (\text{snd prefix}))$
 $\text{staged-alphas data} \in$

```

    alpha-prefix-union-bad-sets prefix-state bad-sets
      (staged-trace-root data)
  alpha-vector-prequeried-in-state prefix-state
    (staged-alphas data) (length spec)
  hash-relation-hit
    (alpha-vector-key-relation prefix-state
      (alpha-prefix-union-bad-sets prefix-state bad-sets
        (staged-trace-root data))
      (length spec))
  adversary-initial-state prefix-state
proof –
from hit obtain prefix prefix-state where
  prefix-out:
    Some (prefix, prefix-state) ∈
      set-dist
        (execute (staged-alpha-prefix-program A)
          adversary-initial-state)
  and root-eq: staged-trace-root data = fst prefix
  and trace-roots-eq:
    staged-trace-fri-roots data = fst (snd prefix)
  and trace-bs-eq:
    staged-trace-fri-challenges data = fst (snd (snd prefix))
  and trace-final-eq:
    staged-trace-final data = snd (snd (snd prefix))
  and member:
    staged-alphas data ∈
      alpha-prefix-union-bad-sets prefix-state bad-sets
        (staged-trace-root data)
  and prequeried:
    alpha-vector-prequeried-in-state prefix-state
      (staged-alphas data) (length spec)
  unfolding checked-staged-transcript-alpha-prefix-prequery-hit-def
  by auto
have relation-hit:
  hash-relation-hit
    (alpha-vector-key-relation prefix-state
      (alpha-prefix-union-bad-sets prefix-state bad-sets
        (staged-trace-root data))
      (length spec))
    adversary-initial-state prefix-state
  by (rule alpha-vector-prequeried-in-state-imp-hash-relation-hit-empty-initial
    [OF prequeried member])
    simp
show ?thesis
  by (rule that[OF prefix-out root-eq trace-roots-eq trace-bs-eq
    trace-final-eq member prequeried relation-hit])
qed

definition staged-alpha-prefix-dynamic-prequery-hit

```

$:: ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool})$
where
 $\text{staged-alpha-prefix-dynamic-prequery-hit bad-sets out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } ((fr, -), \text{prefix-state}) \Rightarrow$
 $(\exists as \in \text{alpha-prefix-union-bad-sets prefix-state bad-sets fr.}$
 $\text{alpha-vector-prequeried-in-state prefix-state as}$
 $(\text{length spec})))$

definition *staged-alpha-prefix-dynamic-prequery-relation*

$:: 'f \text{ staged-adversary} \Rightarrow 'f \text{ protocol-channel} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$

where

$\text{staged-alpha-prefix-dynamic-prequery-relation } A \text{ s bad-sets } x \text{ y}$
 $\longleftrightarrow$
 $(\exists fr \text{ trace-roots trace-bs trace-final prefix-state as } i.$
 $\text{Some } ((fr, \text{trace-roots}, \text{trace-bs}, \text{trace-final}), \text{prefix-state}) \in$
 $\text{set-dist } (\text{execute } (\text{staged-alpha-prefix-program } A) \text{ s}) \wedge$
 $as \in \text{alpha-prefix-union-bad-sets prefix-state bad-sets fr} \wedge$
 $i < \text{length spec} \wedge$
 $i < \text{length as} \wedge$
 $x = \text{alpha-challenge-key-at prefix-state } i \text{ as} \wedge$
 $y = as ! i)$

lemma *staged-alpha-prefix-dynamic-prequery-hit-imp-relation-hit:*

assumes *empty*: $\text{HashMap } s = \text{fmempty}$

and support:

$\text{Some } ((fr, \text{trace-roots}, \text{trace-bs}, \text{trace-final}), \text{prefix-state}) \in$
 $\text{set-dist } (\text{execute } (\text{staged-alpha-prefix-program } A) \text{ s})$

and hit:

$\text{staged-alpha-prefix-dynamic-prequery-hit bad-sets}$
 $(\text{Some } ((fr, \text{trace-roots}, \text{trace-bs}, \text{trace-final}), \text{prefix-state}))$

shows

$\text{hash-relation-hit-event}$
 $(\text{staged-alpha-prefix-dynamic-prequery-relation } A \text{ s bad-sets})$
 $s (\text{Some } ((fr, \text{trace-roots}, \text{trace-bs}, \text{trace-final}), \text{prefix-state}))$

proof –

from hit obtain *as i* **where member:**

$as \in \text{alpha-prefix-union-bad-sets prefix-state bad-sets fr}$

and *i-bound*: $i < \text{length spec}$

and *i-len*: $i < \text{length as}$

and lookup:

$\text{fmlookup } (\text{HashMap } \text{prefix-state})$
 $(\text{alpha-challenge-key-at prefix-state } i \text{ as}) = \text{Some } (as ! i)$

unfolding *staged-alpha-prefix-dynamic-prequery-hit-def*

```

    alpha-vector-prequeried-in-state-def
  by auto
  have initial-none:
    fmlookup (HashMap s) (alpha-challenge-key-at prefix-state i as) =
      None
  using empty by simp
  have relation:
    staged-alpha-prefix-dynamic-prequery-relation A s bad-sets
      (alpha-challenge-key-at prefix-state i as) (as ! i)
  unfolding staged-alpha-prefix-dynamic-prequery-relation-def
  using support member i-bound i-len by blast
  have hit-relation:
    hash-relation-hit
      (staged-alpha-prefix-dynamic-prequery-relation A s bad-sets)
      s prefix-state
  unfolding hash-relation-hit-def
  by (intro exI[of - alpha-challenge-key-at prefix-state i as]
      exI[of - as ! i] conjI)
      (use initial-none lookup relation in simp-all)
  show ?thesis
    unfolding hash-relation-hit-event-def using hit-relation by simp
qed

lemma staged-alpha-prefix-dynamic-prequery-hit-bound-by-relation-hit:
  assumes empty: HashMap s = fmempty
  shows
    wp-event (staged-alpha-prefix-program A)
      (staged-alpha-prefix-dynamic-prequery-hit bad-sets) s ≤
    wp-event (staged-alpha-prefix-program A)
      (hash-relation-hit-event
        (staged-alpha-prefix-dynamic-prequery-relation A s bad-sets) s) s
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out ∈ set-dist (execute (staged-alpha-prefix-program A) s)
    and prehit: staged-alpha-prefix-dynamic-prequery-hit bad-sets out
    show
      hash-relation-hit-event
        (staged-alpha-prefix-dynamic-prequery-relation A s bad-sets)
        s out
    proof (cases out)
    case None
    then show ?thesis
      using prehit unfolding staged-alpha-prefix-dynamic-prequery-hit-def
      by simp
    next
    case (Some packed)
    then obtain fr trace-roots trace-bs trace-final prefix-state
      where out-eq:

```



```

      out =
        Some ((fr, trace-roots, trace-bs, trace-final), prefix-state)
    by (cases packed, auto split: prod.splits)
  show ?thesis
    unfolding out-eq
    by (rule staged-alpha-prefix-dynamic-prequery-hit-imp-relation-hit
        [OF empty])
      (use support prehit out-eq in simp-all)
qed
qed

```

```

lemma staged-alpha-prefix-dynamic-prequery-relation-fiber-bound-size:
  card
    {y. staged-alpha-prefix-dynamic-prequery-relation A s bad-sets x y}
    ≤ size
proof —
  have subset:
    {y. staged-alpha-prefix-dynamic-prequery-relation A s bad-sets x y}
    ⊆ (UNIV :: 'f set)
  by simp
  have finite-field: finite (UNIV :: 'f set)
  by simp
  have card
    {y. staged-alpha-prefix-dynamic-prequery-relation A s bad-sets x y}
    ≤ card (UNIV :: 'f set)
  by (rule card-mono[OF finite-field subset])
  also have ... = size
  using size-card by simp
  finally show ?thesis .
qed

```

```

lemma staged-alpha-prefix-dynamic-prequery-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (staged-alpha-prefix-program A)
      (staged-alpha-prefix-dynamic-prequery-hit bad-sets)
      adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0)
proof —
  have prehit-le-relation:
    wp-event (staged-alpha-prefix-program A)
      (staged-alpha-prefix-dynamic-prequery-hit bad-sets)
      adversary-initial-state ≤
    wp-event (staged-alpha-prefix-program A)
      (hash-relation-hit-event
        (staged-alpha-prefix-dynamic-prequery-relation A
          adversary-initial-state bad-sets))

```

```

      adversary-initial-state)
    adversary-initial-state
  by (rule staged-alpha-prefix-dynamic-prequery-hit-bound-by-relation-hit)
    (simp add: adversary-initial-state-def)
also have ... ≤
  staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
proof -
  have program:
    hash-relation-program
      (staged-alpha-prefix-dynamic-prequery-relation A
        adversary-initial-state bad-sets)
      size (staged-alpha-search-queries budgets 0)
      (staged-alpha-prefix-program A)
  by (rule hash-relation-program-staged-alpha-prefix-program
    [OF wf controlled])
    (rule staged-alpha-prefix-dynamic-prequery-relation-fiber-bound-size)
  show ?thesis
    using program
    unfolding hash-relation-program-def hash-relation-budget-def
      staged-phase-relation-error-def
    by blast
qed
finally show ?thesis .
qed

```

definition *checked-staged-transcript-actual-alpha-prefix-collision-hit*

```

:: (((('f × 'f list × 'f list × 'f) ×
      'f protocol-channel) × 'f staged-proof-data) ×
      'f protocol-channel) option ⇒ bool

```

where

```

checked-staged-transcript-actual-alpha-prefix-collision-hit out
  ⇔
  (case out of
    None ⇒ False
  | Some (((-, prefix-state), -), -) ⇒
    hash-map-output-collision prefix-state)

```

lemma *checked-staged-transcript-actual-alpha-prefix-bad-set-hit-split-events:*

assumes

```

checked-staged-transcript-actual-alpha-prefix-bad-set-hit bad-sets out

```

shows

```

checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit
  bad-sets out ∨
checked-staged-transcript-actual-alpha-prefix-prequery-hit bad-sets out ∨
checked-staged-transcript-actual-alpha-prefix-collision-hit out

```

using *checked-staged-transcript-actual-alpha-prefix-bad-set-hit-split*
 [OF *assms*]

unfolding *checked-staged-transcript-actual-alpha-prefix-collision-hit-def*

```

by (auto split: option.splits prod.splits)

lemma checked-staged-transcript-with-alpha-prefix-prequery-bound:
  shows
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
      (checked-staged-transcript-actual-alpha-prefix-prequery-hit bad-sets)
      adversary-initial-state ≤
    wp-event (staged-alpha-prefix-program A)
      (staged-alpha-prefix-dynamic-prequery-hit bad-sets)
      adversary-initial-state
  unfolding checked-staged-transcript-with-alpha-prefix-program-def
proof (rule wp-event-bind-bound-by-head-event)
  show
    wp-event (staged-alpha-prefix-program A)
      (staged-alpha-prefix-dynamic-prequery-hit bad-sets)
      adversary-initial-state ≤
    wp-event (staged-alpha-prefix-program A)
      (staged-alpha-prefix-dynamic-prequery-hit bad-sets)
      adversary-initial-state
  by simp
next
  show
    checked-staged-transcript-actual-alpha-prefix-prequery-hit
      bad-sets None  $\implies$ 
    staged-alpha-prefix-dynamic-prequery-hit bad-sets None
  unfolding checked-staged-transcript-actual-alpha-prefix-prequery-hit-def
    staged-alpha-prefix-dynamic-prequery-hit-def
  by simp
next
  fix prefix prefix-state out
  assume prefix-out:
    Some (prefix, prefix-state) ∈
      set-dist
        (execute (staged-alpha-prefix-program A) adversary-initial-state)
  and cont:
    out ∈
      set-dist
        (execute
          (get  $\gg$ 
            ( $\lambda$ prefix-state.
              checked-staged-after-alpha-prefix-program A prefix  $\gg$ 
                ( $\lambda$ data. return ((prefix, prefix-state), data))))
          prefix-state)
  and hit:
    checked-staged-transcript-actual-alpha-prefix-prequery-hit
      bad-sets out
  obtain fr trace-roots trace-bs trace-final where prefix-eq:
    prefix = (fr, trace-roots, trace-bs, trace-final)
  by (cases prefix) auto

```

```

from hit cont show
  staged-alpha-prefix-dynamic-prequery-hit bad-sets
  (Some (prefix, prefix-state))
unfolding checked-staged-transcript-actual-alpha-prefix-prequery-hit-def
  staged-alpha-prefix-dynamic-prequery-hit-def prefix-eq
by (auto elim!: set-dist-bindE split: option.splits prod.splits)
qed

lemma checked-staged-transcript-with-alpha-prefix-collision-bound:
shows
  wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    checked-staged-transcript-actual-alpha-prefix-collision-hit
    adversary-initial-state  $\leq$ 
  wp-event (staged-alpha-prefix-program A)
    (hash-new-collision-event adversary-initial-state)
    adversary-initial-state
unfolding checked-staged-transcript-with-alpha-prefix-program-def
proof (rule wp-event-bind-bound-by-head-event)
show
  wp-event (staged-alpha-prefix-program A)
    (hash-new-collision-event adversary-initial-state)
    adversary-initial-state  $\leq$ 
  wp-event (staged-alpha-prefix-program A)
    (hash-new-collision-event adversary-initial-state)
    adversary-initial-state
by simp
next
show
  checked-staged-transcript-actual-alpha-prefix-collision-hit None  $\implies$ 
  hash-new-collision-event adversary-initial-state None
unfolding checked-staged-transcript-actual-alpha-prefix-collision-hit-def
  hash-new-collision-event-def
by simp
next
fix prefix prefix-state out
assume prefix-out:
  Some (prefix, prefix-state)  $\in$ 
    set-dist
    (execute (staged-alpha-prefix-program A) adversary-initial-state)
and cont:
  out  $\in$ 
    set-dist
    (execute
      (get  $\gg$ 
        ( $\lambda$ prefix-state.
          checked-staged-after-alpha-prefix-program A prefix  $\gg$ 
            ( $\lambda$ data. return ((prefix, prefix-state), data))))
      prefix-state)
and hit:

```

```

    checked-staged-transcript-actual-alpha-prefix-collision-hit out
  have collision: hash-map-output-collision prefix-state
  using hit cont
  unfolding checked-staged-transcript-actual-alpha-prefix-collision-hit-def
  by (auto elim!: set-dist-bindE split: option.splits prod.splits)
show
  hash-new-collision-event adversary-initial-state
    (Some (prefix, prefix-state))
  using collision adversary-initial-state-no-output-collision
  unfolding hash-new-collision-event-def hash-map-new-output-collision-def
  by simp
qed

```

lemma *checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound:*
shows

```

  wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
      composition-trace-bad-alpha-space)
  adversary-initial-state ≤
  composition-error-bound +
  wp-event (staged-alpha-prefix-program A)
    (staged-alpha-prefix-dynamic-prequery-hit
      composition-trace-bad-alpha-space)
  adversary-initial-state +
  wp-event (staged-alpha-prefix-program A)
    (hash-new-collision-event adversary-initial-state)
  adversary-initial-state

```

proof —

```

  let ?m = checked-staged-transcript-with-alpha-prefix-program A
  let ?bad =
    checked-staged-transcript-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space
  let ?clean =
    checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit
    composition-trace-bad-alpha-space
  let ?pre =
    checked-staged-transcript-actual-alpha-prefix-prequery-hit
    composition-trace-bad-alpha-space
  let ?coll = checked-staged-transcript-actual-alpha-prefix-collision-hit
  have bad-le-split:
    wp-event ?m ?bad adversary-initial-state ≤
      wp-event ?m (λout. ?clean out ∨ ?pre out ∨ ?coll out)
      adversary-initial-state
  by (rule wp-event-mono)
  (rule
    checked-staged-transcript-actual-alpha-prefix-bad-set-hit-split-events)
  also have ... ≤
    wp-event ?m ?clean adversary-initial-state +
    wp-event ?m ?pre adversary-initial-state +

```

```

    wp-event ?m ?coll adversary-initial-state
proof –
  have ... ≤
    wp-event ?m ?clean adversary-initial-state +
    wp-event ?m (λout. ?pre out ∨ ?coll out)
      adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤
    wp-event ?m ?clean adversary-initial-state +
    (wp-event ?m ?pre adversary-initial-state +
     wp-event ?m ?coll adversary-initial-state)
  by (intro add-mono order.refl wp-event-union-bound)
  finally show ?thesis
    by (simp add: add.assoc)
qed
also have ... ≤
  composition-error-bound +
  wp-event (staged-alpha-prefix-program A)
    (staged-alpha-prefix-dynamic-prequery-hit
     composition-trace-bad-alpha-space)
  adversary-initial-state +
  wp-event (staged-alpha-prefix-program A)
    (hash-new-collision-event adversary-initial-state)
  adversary-initial-state
  by (intro add-mono
    checked-staged-transcript-with-alpha-prefix-clean-fresh-composition-bound
    checked-staged-transcript-with-alpha-prefix-prequery-bound
    checked-staged-transcript-with-alpha-prefix-collision-bound)
  finally show ?thesis .
qed

lemma checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
      (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
       composition-trace-bad-alpha-space)
    adversary-initial-state ≤
    composition-error-bound +
    staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0)
proof –
  have prequery-bound:
    wp-event (staged-alpha-prefix-program A)
      (staged-alpha-prefix-dynamic-prequery-hit
       composition-trace-bad-alpha-space)

```

```

    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  by (rule staged-alpha-prefix-dynamic-prequery-hit-bound
    [OF wf controlled])
  have collision-budget:
    hash-collision-budget
    (staged-alpha-search-queries budgets 0)
    (staged-alpha-prefix-program A)
  by (rule hash-collision-budget-staged-alpha-prefix-program
    [OF wf controlled])
  have collision-bound:
    wp-event (staged-alpha-prefix-program A)
    (hash-new-collision-event adversary-initial-state)
    adversary-initial-state ≤
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0)
  proof —
  have
    wp-event (staged-alpha-prefix-program A)
    (hash-new-collision-event adversary-initial-state)
    adversary-initial-state ≤
    hash-collision-budget-value
    (card (hash-map-output-values adversary-initial-state))
    (staged-alpha-search-queries budgets 0)
  using collision-budget adversary-initial-state-no-output-collision
  unfolding hash-collision-budget-def by blast
  also have ... =
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0)
  by simp
  finally show ?thesis .
qed
have
  wp-event (checked-staged-transcript-with-alpha-prefix-program A)
  (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space)
  adversary-initial-state ≤
  composition-error-bound +
  wp-event (staged-alpha-prefix-program A)
  (staged-alpha-prefix-dynamic-prequery-hit
    composition-trace-bad-alpha-space)
  adversary-initial-state +
  wp-event (staged-alpha-prefix-program A)
  (hash-new-collision-event adversary-initial-state)
  adversary-initial-state
  by (rule checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound)
  also have ... ≤
    composition-error-bound +

```

```

    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0)
  by (intro add-mono order.refl prequery-bound collision-bound)
  finally show ?thesis .
qed

```

definition *checked-staged-security-with-actual-alpha-prefix-bad-set-hit*

```

:: ('f list  $\Rightarrow$  'f list set)  $\Rightarrow$ 
  ((((((('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
    'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
    'f protocol-channel)  $\times$  unit list)  $\times$ 
    'f protocol-channel) option  $\Rightarrow$  bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-bad-set-hit
bad-sets out  $\longleftrightarrow$ 
(case out of
  None  $\Rightarrow$  False
| Some (x, -)  $\Rightarrow$ 
  (let packed = fst (fst x);
    prefix = fst (fst packed);
    prefix-state = snd (fst packed);
    data = snd packed
  in staged-alphas data  $\in$ 
    alpha-prefix-union-bad-sets prefix-state bad-sets (fst prefix)))

```

definition *checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit*

```

:: ('f list  $\Rightarrow$  'f list set)  $\Rightarrow$ 
  ((((((('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
    'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
    'f protocol-channel)  $\times$  unit list)  $\times$ 
    'f protocol-channel) option  $\Rightarrow$  bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit
bad-sets out  $\longleftrightarrow$ 
(case out of
  None  $\Rightarrow$  False
| Some (x, -)  $\Rightarrow$ 
  (let packed = fst (fst x);
    data = snd packed;
    attacker-state = snd (fst x)
  in staged-alphas data  $\in$ 
    alpha-header-supported-union-bad-sets
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    bad-sets
    (staged-trace-root data)
    (staged-trace-fri-roots data))

```


(staged-trace-final data)))

definition *checked-staged-security-with-actual-alpha-prefix-tree-output-hit*

$:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-security-with-actual-alpha-prefix-tree-output-hit out

$\longleftrightarrow$

(case out of

None $\Rightarrow$ False

| Some (x, -) $\Rightarrow$

(let packed = fst (fst x);

prefix-state = snd (fst packed);

data = snd packed;

attacker-state = snd (fst x);

s =

verifier-state-from-adversary attacker-state

(staged-proof-transcript data)

in $\exists$ result final-state trace-table composition-table as

query-idxs dg composition-fri-roots final rest trace-tree

composition-tree.

Some (result, final-state) $\in$

set-dist (execute verify-monad s) $\wedge$

accepted-with-bound-tables s (Some (result, final-state))

trace-table composition-table as query-idxs $\wedge$

verifier-header-transcript s

(staged-trace-root data)

(staged-trace-fri-roots data)

(staged-trace-final data)

as dg composition-fri-roots final rest $\wedge$

created-tree trace-table trace-tree final-state $\wedge$

created-tree composition-table composition-tree final-state $\wedge$

hash-map-new-output-hit

(set-tree trace-tree $\cup$ set-tree composition-tree)

prefix-state final-state))

lemma *checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript:*

assumes transcript-bound:

wp-event (checked-staged-transcript-with-alpha-prefix-program A)

(checked-staged-transcript-actual-alpha-prefix-bad-set-hit bad-sets)

adversary-initial-state $\leq C$

shows

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

(checked-staged-security-with-actual-alpha-prefix-bad-set-hit bad-sets)

adversary-initial-state $\leq C$

unfolding

```

checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
proof (rule wp-event-bind-bound-by-head-event[OF transcript-bound])
  show
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
      bad-sets None  $\implies$ 
      checked-staged-transcript-actual-alpha-prefix-bad-set-hit bad-sets None
    unfolding checked-staged-security-with-actual-alpha-prefix-bad-set-hit-def
      checked-staged-transcript-actual-alpha-prefix-bad-set-hit-def
    by simp
  next
    fix packed attacker-state out
    assume cont:
      out  $\in$ 
        set-dist
          (execute
            (get  $\gg$ 
              ( $\lambda s.$  put
                (verifier-state-from-adversary s
                  (staged-proof-transcript (snd packed))))  $\gg$ 
                ( $\lambda -. \text{verify-monad } \gg$ 
                  ( $\lambda \text{result. return } ((\text{packed}, s), \text{result})$ ))))
              attacker-state)
          and hit:
            checked-staged-security-with-actual-alpha-prefix-bad-set-hit
              bad-sets out
          show
            checked-staged-transcript-actual-alpha-prefix-bad-set-hit bad-sets
              (Some (packed, attacker-state))
          using cont hit
          unfolding checked-staged-security-with-actual-alpha-prefix-bad-set-hit-def
            checked-staged-transcript-actual-alpha-prefix-bad-set-hit-def
          by (auto elim!: set-dist-bindE split: option.splits prod.splits)
    qed

```

```

lemma checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-actual-projection:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
      bad-sets)
    adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit
      bad-sets)
    adversary-initial-state
proof –
  let ?project =
     $\lambda x. ((\text{snd } (\text{fst } (\text{fst } x))), \text{snd } (\text{fst } x)), \text{snd } x$ 
  have
    wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
      bad-sets)
  adversary-initial-state =
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
     $\gg$  ( $\lambda x.$  return (?project x)))
  (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
    bad-sets)
  adversary-initial-state
using
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
  [of A]
by simp
also have ... =
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ( $\lambda$ out. case out of
    None  $\Rightarrow$ 
      checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
      bad-sets None
    | Some (x, t)  $\Rightarrow$ 
      checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
      bad-sets (Some (?project x, t)))
  adversary-initial-state
by (rule wp-event-bind-return-map)
also have ... =
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit
    bad-sets)
  adversary-initial-state
unfolding
  checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-def
  checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit-def
apply (rule arg-cong[where
  f= $\lambda P.$  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  P adversary-initial-state])
apply (rule ext)
apply (simp add: Let-def split: option.splits prod.splits)
done
finally show ?thesis .
qed

```

```

lemma checked-staged-security-with-actual-alpha-prefix-tree-output-hitI:
assumes verify-out:
  Some (result, final-state)  $\in$ 
    set-dist
    (execute verify-monad

```

(verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
and bound:
 accepted-with-bound-tables
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result, final-state)) trace-table composition-table as
 query-idxs
and header:
 verifier-header-transcript
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 as dg composition-fri-roots final rest
and trace-created: created-tree trace-table trace-tree final-state
and composition-created:
 created-tree composition-table composition-tree final-state
and tree-hit:
 hash-map-new-output-hit
 (set-tree trace-tree $\cup$ set-tree composition-tree)
 prefix-state final-state
shows
 checked-staged-security-with-actual-alpha-prefix-tree-output-hit
 (Some (((((prefix, prefix-state), data), attacker-state),
 checked-result), checked-final-state)))
proof –
have witness:
 $\exists$ result final-state trace-table composition-table as query-idxs dg
 composition-fri-roots final rest trace-tree composition-tree.
 Some (result, final-state) $\in$
 set-dist
 (execute verify-monad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))) $\wedge$
 accepted-with-bound-tables
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result, final-state)) trace-table composition-table as
 query-idxs $\wedge$
 verifier-header-transcript
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 as dg composition-fri-roots final rest $\wedge$
 created-tree trace-table trace-tree final-state $\wedge$

```

    created-tree composition-table composition-tree final-state  $\wedge$ 
    hash-map-new-output-hit
    (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
using verify-out bound header trace-created composition-created tree-hit
by blast
show ?thesis
unfolding
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit-def
    Let-def
using witness by simp
qed

```

definition *checked-staged-security-with-data-state-alpha-prefix-tree-output-hit*

```

:: 'f staged-adversary  $\Rightarrow$ 
  (((('f staged-proof-data  $\times$  'f protocol-channel)  $\times$  unit list)  $\times$ 
   'f protocol-channel) option  $\Rightarrow$  bool

```

where

```

checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A
  out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (((data, attacker-state), -), -)  $\Rightarrow$ 
    (let s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    in  $\exists$  prefix prefix-state result final-state trace-table
      composition-table as query-idxs dg composition-fri-roots final
      rest trace-tree composition-tree.
      Some (prefix, prefix-state)  $\in$ 
        set-dist
        (execute (staged-alpha-prefix-program A)
          adversary-initial-state)  $\wedge$ 
        staged-trace-root data = fst prefix  $\wedge$ 
        staged-trace-fri-roots data = fst (snd prefix)  $\wedge$ 
        staged-trace-fri-challenges data = fst (snd (snd prefix))  $\wedge$ 
        staged-trace-final data = snd (snd (snd prefix))  $\wedge$ 
        Some (result, final-state)  $\in$ 
          set-dist (execute verify-monad s)  $\wedge$ 
        accepted-with-bound-tables s (Some (result, final-state))
        trace-table composition-table as query-idxs  $\wedge$ 
        verifier-header-transcript s
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        as dg composition-fri-roots final rest  $\wedge$ 
        created-tree trace-table trace-tree final-state  $\wedge$ 
        created-tree composition-table composition-tree final-state  $\wedge$ 
        hash-map-new-output-hit

```

(*set-tree trace-tree* $\cup$ *set-tree composition-tree*)
prefix-state final-state))

lemma *checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-alphas-in-space:*

assumes *hit:*

checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A

bad-sets out

shows

case out of

None $\Rightarrow$ False

| *Some (((data, -), -), -) $\Rightarrow$ staged-alphas data $\in$ alpha-space*

using *hit*

unfolding *checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-def*

alpha-prefix-union-bad-sets-def

by (*auto split: option.splits prod.splits*)

lemma *checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-bound-from-transcript:*

assumes *transcript-bound:*

wp-event (checked-staged-transcript-program A)

(checked-staged-transcript-alpha-prefix-bad-set-hit A bad-sets)

adversary-initial-state $\leq$ C

shows

wp-event (checked-staged-security-experiment-with-data-state A)

*(checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
bad-sets)*

adversary-initial-state $\leq$ C

unfolding *checked-staged-security-experiment-with-data-state-def*

proof (*rule wp-event-bind-bound-by-head-event[OF transcript-bound]*)

show

checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A

bad-sets None $\implies$

checked-staged-transcript-alpha-prefix-bad-set-hit A bad-sets None

unfolding

checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-def

checked-staged-transcript-alpha-prefix-bad-set-hit-def

by *simp*

next

fix *data attacker-state out*

assume *cont:*

out $\in$

set-dist

(execute

(get $\gg=$

($\lambda s.$ put

(verifier-state-from-adversary s

(staged-proof-transcript data)) $\gg=$

($\lambda -. verify-monad \gg=$

($\lambda result.$ return ((data, s), result))))))

attacker-state)

```

and hit:
  checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
  bad-sets out
show
  checked-staged-transcript-alpha-prefix-bad-set-hit A bad-sets
  (Some (data, attacker-state))
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-def
    by simp
next
  case (Some result-pack)
  then obtain data' attacker-state' result final-state where out-eq:
    out = Some (((data', attacker-state'), result), final-state)
    by (cases result-pack, auto split: prod.splits)
  have data-eq: data' = data
    and attacker-eq: attacker-state' = attacker-state
    using cont unfolding out-eq
    by (auto elim!: set-dist-bindE)
  show ?thesis
    using hit unfolding out-eq data-eq attacker-eq
      checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-def
      checked-staged-transcript-alpha-prefix-bad-set-hit-def
    by simp
qed
qed

lemma staged-after-alpha-prefix-program-outcome-extension:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, t) ∈
      set-dist
        (execute (staged-after-alpha-prefix-program A prefix) s)
  shows s ≤ t
proof -
  obtain fr trace-roots trace-bs trace-final where prefix-eq:
    prefix = (fr, trace-roots, trace-bs, trace-final)
    by (cases prefix) simp
  from outcome[unfolded prefix-eq]
  obtain as s1 dg s2 s3 s4 composition-roots composition-bs s5
    composition-final s6 s7 query-chunks where
    alpha-out:
      Some (as, s1) ∈
        set-dist (execute (staged-alpha-program (length spec)) s)
  and degree-out:

```

```

    Some (dg, s2) ∈ set-dist (execute (degree-stage A as) s1)
and record-degree-out:
    Some ((), s3) ∈ set-dist (execute (record-staged-message dg) s2)
and assert-out:
    Some ((), s4) ∈
      set-dist
        (execute
          (assert
            (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)))
          s3)
and composition-out:
    Some ((composition-roots, composition-bs), s5) ∈
      set-dist
        (execute
          (staged-composition-fri-program A dg 0
            (ceil-log (to-nat dg + 1)) []))
          s4)
and composition-final-out:
    Some (composition-final, s6) ∈
      set-dist (execute (composition-final-stage A dg composition-bs) s5)
and record-composition-final-out:
    Some ((), s7) ∈
      set-dist (execute (record-staged-message composition-final) s6)
and query-out:
    Some (query-chunks, t) ∈
      set-dist (execute (staged-query-program A 0 rounds) s7)
unfolding staged-after-alpha-prefix-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have degree-controlled:
  controlled-ro-program (degree-budget budgets) (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def by blast
have composition-final-controlled:
  controlled-ro-program (composition-final-budget budgets)
    (composition-final-stage A dg composition-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have ext-s-s1: s ≤ s1
  by (rule hash-target-program-outcome-extension
    [OF hash-target-program-staged-alpha-program alpha-out])
have ext-s1-s2: s1 ≤ s2
  using controlled-ro-program-extension[OF degree-controlled] degree-out
  unfolding hash-extension-preserving-def by blast
have s3-eq:
  s3 = s2⟦
    PState := concat (PState s2) dg,
    PTranscript := PTranscript s2 @ [dg]⟧
  by (rule record-staged-message-outcome[OF record-degree-out])
have ext-s2-s3: s2 ≤ s3
  unfolding s3-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have s4-eq: s4 = s3

```



```

using assert-out unfolding assert-def
by (cases
   $\text{ceil-log } (to\text{-nat } dg + 1) \leq \text{ceil-log } (maxDegree + 1)$ )
  (auto simp: throw-no-outcome)
have ext-s3-s4:  $s3 \leq s4$ 
  unfolding s4-eq by (simp add: hash-ext-refl)
have round-bound:
   $\text{ceil-log } (to\text{-nat } dg + 1) \leq \text{ceil-log } (maxDegree + 1)$ 
  using assert-out unfolding assert-def
  by (cases
     $\text{ceil-log } (to\text{-nat } dg + 1) \leq \text{ceil-log } (maxDegree + 1)$ )
    (auto simp: throw-no-outcome)
have composition-bound:
   $0 + \text{ceil-log } (to\text{-nat } dg + 1) \leq$ 
   $\text{length } (composition\text{-fri}\text{-budgets } budgets)$ 
  using wf round-bound unfolding staged-budget-wellformed-def by simp
have ext-s4-s5:  $s4 \leq s5$ 
  using staged-composition-fri-program-alignment
  [OF controlled composition-bound composition-out]
  by simp
have ext-s5-s6:  $s5 \leq s6$ 
  using controlled-ro-program-extension[OF composition-final-controlled]
  composition-final-out
  unfolding hash-extension-preserving-def by blast
have s7-eq:
   $s7 = s6 \langle$ 
   $PState := \text{concat } (PState\ s6)\ \text{composition-final},$ 
   $PTranscript := PTranscript\ s6\ @\ [composition\text{-final}] \rangle$ 
  by (rule record-staged-message-outcome[OF record-composition-final-out])
have ext-s6-s7:  $s6 \leq s7$ 
  unfolding s7-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have query-bound:  $0 + \text{rounds} \leq \text{length } (query\text{-opening}\text{-budgets } budgets)$ 
  using wf unfolding staged-budget-wellformed-def by simp
have ext-s7-t:  $s7 \leq t$ 
  by (rule hash-target-program-outcome-extension
    [OF hash-target-program-staged-query-program[OF controlled query-bound]
    query-out])
show ?thesis
  by (rule hash-ext-trans[OF ext-s-s1])
    (rule hash-ext-trans[OF ext-s1-s2],
     rule hash-ext-trans[OF ext-s2-s3],
     rule hash-ext-trans[OF ext-s3-s4],
     rule hash-ext-trans[OF ext-s4-s5],
     rule hash-ext-trans[OF ext-s5-s6],
     rule hash-ext-trans[OF ext-s6-s7 ext-s7-t])
qed

```

lemma *checked-staged-after-alpha-prefix-program-outcome-extension*:
assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: staged-adversary-controlled budgets A
and *outcome*:
Some $(data, t) \in$
set-dist
(execute (checked-staged-after-alpha-prefix-program A prefix) s)
shows $s \leq t$
proof –
obtain fr *trace-roots* *trace-bs* *trace-final* **where** *prefix-eq*:
prefix = (fr, trace-roots, trace-bs, trace-final)
by *(cases prefix) simp*
from *outcome[unfolded prefix-eq]*
obtain as $s1$ dg $s2$ $s3$ $s4$ *composition-roots* *composition-bs* $s5$
composition-final $s6$ $s7$ *query-chunks* **where**
alpha-out:
Some $(as, s1) \in$
set-dist (execute (staged-alpha-program (length spec)) s)
and *degree-out*:
Some $(dg, s2) \in$ *set-dist (execute (degree-stage A as) $s1$)*
and *record-degree-out*:
Some $((), s3) \in$ *set-dist (execute (record-staged-message dg) $s2$)*
and *assert-out*:
Some $((), s4) \in$
set-dist
(execute
(assert
(ceil-log (to-nat dg + 1) $\leq$ ceil-log (maxDegree + 1)))
 $s3$)
and *composition-out*:
Some $((composition-roots, composition-bs), s5) \in$
set-dist
(execute
(staged-composition-fri-program A dg 0
(ceil-log (to-nat dg + 1)) $\square$)
 $s4$)
and *composition-final-out*:
Some $(composition-final, s6) \in$
set-dist (execute (composition-final-stage A dg composition-bs) $s5$)
and *record-composition-final-out*:
Some $((), s7) \in$
set-dist (execute (record-staged-message composition-final) $s6$)
and *query-out*:
Some $(query-chunks, t) \in$
set-dist
(execute
(checked-staged-query-program A trace-roots composition-roots
0 rounds)
 $s7$)
unfolding *checked-staged-after-alpha-prefix-program-def Let-def*
by *(auto elim!: set-dist-bindE split: prod.splits)*

```

have degree-controlled:
  controlled-ro-program (degree-budget budgets) (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def by blast
have composition-final-controlled:
  controlled-ro-program (composition-final-budget budgets)
    (composition-final-stage A dg composition-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have ext-s-s1:  $s \leq s1$ 
  by (rule hash-target-program-outcome-extension
    [OF hash-target-program-staged-alpha-program alpha-out])
have ext-s1-s2:  $s1 \leq s2$ 
  using controlled-ro-program-extension[OF degree-controlled] degree-out
  unfolding hash-extension-preserving-def by blast
have s3-eq:
   $s3 = s2 \mid$ 
     $PState := \text{concat } (PState\ s2)\ dg,$ 
     $PTranscript := PTranscript\ s2\ @\ [dg]$ 
  by (rule record-staged-message-outcome[OF record-degree-out])
have ext-s2-s3:  $s2 \leq s3$ 
  unfolding s3-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have s4-eq:  $s4 = s3$ 
  using assert-out unfolding assert-def
  by (cases
     $\text{ceil-log } (to\text{-nat } dg + 1) \leq \text{ceil-log } (maxDegree + 1))$ 
    (auto simp: throw-no-outcome)
have ext-s3-s4:  $s3 \leq s4$ 
  unfolding s4-eq by (simp add: hash-ext-refl)
have round-bound:
   $\text{ceil-log } (to\text{-nat } dg + 1) \leq \text{ceil-log } (maxDegree + 1)$ 
  using assert-out unfolding assert-def
  by (cases
     $\text{ceil-log } (to\text{-nat } dg + 1) \leq \text{ceil-log } (maxDegree + 1))$ 
    (auto simp: throw-no-outcome)
have composition-bound:
   $0 + \text{ceil-log } (to\text{-nat } dg + 1) \leq$ 
     $\text{length } (composition\text{-fri}\text{-budgets } budgets)$ 
  using wf round-bound unfolding staged-budget-wellformed-def by simp
have ext-s4-s5:  $s4 \leq s5$ 
  using staged-composition-fri-program-alignment
    [OF controlled composition-bound composition-out]
  by simp
have ext-s5-s6:  $s5 \leq s6$ 
  using controlled-ro-program-extension[OF composition-final-controlled]
    composition-final-out
  unfolding hash-extension-preserving-def by blast
have s7-eq:
   $s7 = s6 \mid$ 
     $PState := \text{concat } (PState\ s6)\ composition\text{-final},$ 
     $PTranscript := PTranscript\ s6\ @\ [composition\text{-final}]$ 

```

```

  by (rule record-staged-message-outcome[OF record-composition-final-out])
have ext-s6-s7:  $s6 \leq s7$ 
  unfolding s7-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have query-bound:  $0 + \text{rounds} \leq \text{length } (\text{query-opening-budgets } \text{budgets})$ 
  using wf unfolding staged-budget-wellformed-def by simp
have query-program:
  hash-relation-program ( $\lambda - . \text{False}$ ) 0
    (sum-list (take rounds (drop 0 (query-opening-budgets budgets))) +
      rounds)
    (checked-staged-query-program A trace-roots composition-roots 0
      rounds)
  by (rule hash-relation-program-checked-staged-query-program
    [OF controlled query-bound])
    simp
have ext-s7-t:  $s7 \leq t$ 
  using query-program query-out
  unfolding hash-relation-program-def hash-extension-preserving-def
  by blast
show ?thesis
  by (rule hash-ext-trans[OF ext-s1])
    (rule hash-ext-trans[OF ext-s1-s2],
      rule hash-ext-trans[OF ext-s2-s3],
      rule hash-ext-trans[OF ext-s3-s4],
      rule hash-ext-trans[OF ext-s4-s5],
      rule hash-ext-trans[OF ext-s5-s6],
      rule hash-ext-trans[OF ext-s6-s7 ext-s7-t])
qed

```

lemma *checked-staged-transcript-with-alpha-prefix-program-support:*
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
 Some (((prefix, prefix-state), data), attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-with-alpha-prefix-program A)
 adversary-initial-state)
obtains
 Some (prefix, prefix-state) $\in$
 set-dist
 (execute (staged-alpha-prefix-program A) adversary-initial-state)
 staged-trace-root data = fst prefix
 staged-trace-fri-roots data = fst (snd prefix)
 staged-trace-fri-challenges data = fst (snd (snd prefix))
 staged-trace-final data = snd (snd (snd prefix))
 prefix-state $\leq$ attacker-state
proof –
from outcome **obtain** after-out **where** prefix-out:
 Some (prefix, prefix-state) $\in$
 set-dist

```

    (execute (staged-alpha-prefix-program A) adversary-initial-state)
and after-out:
Some (data, attacker-state) ∈
  set-dist
    (execute (checked-staged-after-alpha-prefix-program A prefix)
      prefix-state)
unfolding checked-staged-transcript-with-alpha-prefix-program-def
by (auto elim!: set-dist-bindE)
have prefix-ext: prefix-state ≤ attacker-state
by (rule checked-staged-after-alpha-prefix-program-outcome-extension
    [OF - - after-out])
    (fact wf, fact controlled)
have root-eq:
  staged-trace-root data = fst prefix
and trace-roots-eq:
  staged-trace-fri-roots data = fst (snd prefix)
and trace-bs-eq:
  staged-trace-fri-challenges data = fst (snd (snd prefix))
and trace-final-eq:
  staged-trace-final data = snd (snd (snd prefix))
proof -
obtain fr trace-roots trace-bs trace-final where prefix-eq:
  prefix = (fr, trace-roots, trace-bs, trace-final)
by (cases prefix) simp
from after-out[unfolded prefix-eq]
obtain as s1 dg s2 s3 composition-roots composition-bs s4
  composition-final s5 s6 query-chunks where ret:
Some (data, attacker-state) ∈
  set-dist
    (execute
      (return
        (|staged-trace-root = fr,
          staged-trace-fri-roots = trace-roots,
          staged-trace-fri-challenges = trace-bs,
          staged-trace-final = trace-final,
          staged-alphas = as,
          staged-degree = dg,
          staged-composition-fri-roots = composition-roots,
          staged-composition-fri-challenges = composition-bs,
          staged-composition-final = composition-final,
          staged-query-chunks = query-chunks|))
        s6))
unfolding checked-staged-after-alpha-prefix-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have data-eq:
  data =
    (|staged-trace-root = fr,
      staged-trace-fri-roots = trace-roots,
      staged-trace-fri-challenges = trace-bs,

```

```

    staged-trace-final = trace-final,
    staged-alphas = as,
    staged-degree = dg,
    staged-composition-fri-roots = composition-roots,
    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks)
  using ret by simp
show staged-trace-root data = fst prefix
  unfolding prefix-eq data-eq by simp
show staged-trace-fri-roots data = fst (snd prefix)
  unfolding prefix-eq data-eq by simp
show staged-trace-fri-challenges data = fst (snd (snd prefix))
  unfolding prefix-eq data-eq by simp
show staged-trace-final data = snd (snd (snd prefix))
  unfolding prefix-eq data-eq by simp
qed
show ?thesis
  by (rule that[OF prefix-out root-eq trace-roots-eq trace-bs-eq
    trace-final-eq prefix-ext])
qed

lemma checked-staged-transcript-program-alpha-prefix-support:
  assumes outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  obtains prefix prefix-state
  where
    Some (prefix, prefix-state) ∈
      set-dist
        (execute (staged-alpha-prefix-program A)
          adversary-initial-state)
    staged-trace-root data = fst prefix
    staged-trace-fri-roots data = fst (snd prefix)
    staged-trace-fri-challenges data = fst (snd (snd prefix))
    staged-trace-final data = snd (snd (snd prefix))
proof -
  have staged-out:
    Some (data, attacker-state) ∈
      set-dist
        (execute (staged-transcript-program A) adversary-initial-state)
  by (rule checked-staged-transcript-program-outcome-imp-staged-transcript-program
    [OF outcome])
  then obtain prefix prefix-state where prefix-out:
    Some (prefix, prefix-state) ∈
      set-dist
        (execute (staged-alpha-prefix-program A)

```

```

    adversary-initial-state)
and after-out:
Some (data, attacker-state) ∈
  set-dist
    (execute (staged-after-alpha-prefix-program A prefix)
      prefix-state)
unfolding staged-transcript-program-alpha-prefix-decomp
by (auto elim!: set-dist-bindE)
have root-eq:
  staged-trace-root data = fst prefix
and trace-roots-eq:
  staged-trace-fri-roots data = fst (snd prefix)
and trace-bs-eq:
  staged-trace-fri-challenges data = fst (snd (snd prefix))
and trace-final-eq:
  staged-trace-final data = snd (snd (snd prefix))
proof –
obtain fr trace-roots trace-bs trace-final where prefix-eq:
  prefix = (fr, trace-roots, trace-bs, trace-final)
  by (cases prefix) simp
from after-out[unfolded prefix-eq]
obtain as s1 dg s2 s3 composition-roots composition-bs s4
  composition-final s5 s6 query-chunks where ret:
Some (data, attacker-state) ∈
  set-dist
    (execute
      (return
        (|staged-trace-root = fr,
          staged-trace-fri-roots = trace-roots,
          staged-trace-fri-challenges = trace-bs,
          staged-trace-final = trace-final,
          staged-alphas = as,
          staged-degree = dg,
          staged-composition-fri-roots = composition-roots,
          staged-composition-fri-challenges = composition-bs,
          staged-composition-final = composition-final,
          staged-query-chunks = query-chunks|))
        s6)
unfolding staged-after-alpha-prefix-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have data-eq:
  data =
    (|staged-trace-root = fr,
      staged-trace-fri-roots = trace-roots,
      staged-trace-fri-challenges = trace-bs,
      staged-trace-final = trace-final,
      staged-alphas = as,
      staged-degree = dg,
      staged-composition-fri-roots = composition-roots,

```

```

    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks)
  using ret by simp
show staged-trace-root data = fst prefix
  unfolding prefix-eq data-eq by simp
show staged-trace-fri-roots data = fst (snd prefix)
  unfolding prefix-eq data-eq by simp
show staged-trace-fri-challenges data = fst (snd (snd prefix))
  unfolding prefix-eq data-eq by simp-all
show staged-trace-final data = snd (snd (snd prefix))
  unfolding prefix-eq data-eq by simp
qed
show ?thesis
  by (rule that[OF prefix-out root-eq trace-roots-eq trace-bs-eq
    trace-final-eq])
qed

lemma checked-staged-transcript-program-alpha-prefix-match-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  obtains prefix prefix-state
  where
    Some (prefix, prefix-state) ∈
      set-dist
        (execute (staged-alpha-prefix-program A)
          adversary-initial-state)
    staged-trace-root data = fst prefix
    staged-trace-fri-roots data = fst (snd prefix)
    staged-trace-fri-challenges data = fst (snd (snd prefix))
    staged-trace-final data = snd (snd (snd prefix))
    prefix-state ≤ attacker-state
proof -
  have staged-out:
    Some (data, attacker-state) ∈
      set-dist
        (execute (staged-transcript-program A) adversary-initial-state)
  by (rule checked-staged-transcript-program-outcome-imp-staged-transcript-program
    [OF outcome])
  then obtain prefix prefix-state where prefix-out:
    Some (prefix, prefix-state) ∈
      set-dist
        (execute (staged-alpha-prefix-program A)
          adversary-initial-state)

```



```

and after-out:
  Some (data, attacker-state) ∈
    set-dist
      (execute (staged-after-alpha-prefix-program A prefix)
        prefix-state)
  unfolding staged-transcript-program-alpha-prefix-decomp
  by (auto elim!: set-dist-bindE)
have prefix-ext: prefix-state ≤ attacker-state
  by (rule staged-after-alpha-prefix-program-outcome-extension
    [OF wf controlled after-out])
have root-eq:
  staged-trace-root data = fst prefix
  and trace-roots-eq:
  staged-trace-fri-roots data = fst (snd prefix)
  and trace-bs-eq:
  staged-trace-fri-challenges data = fst (snd (snd prefix))
  and trace-final-eq:
  staged-trace-final data = snd (snd (snd prefix))
proof –
  obtain fr trace-roots trace-bs trace-final where prefix-eq:
    prefix = (fr, trace-roots, trace-bs, trace-final)
  by (cases prefix) simp
  from after-out[unfolded prefix-eq]
  obtain as s1 dg s2 s3 composition-roots composition-bs s4
    composition-final s5 s6 query-chunks where ret:
    Some (data, attacker-state) ∈
      set-dist
        (execute
          (return
            (|staged-trace-root = fr,
              staged-trace-fri-roots = trace-roots,
              staged-trace-fri-challenges = trace-bs,
              staged-trace-final = trace-final,
              staged-alphas = as,
              staged-degree = dg,
              staged-composition-fri-roots = composition-roots,
              staged-composition-fri-challenges = composition-bs,
              staged-composition-final = composition-final,
              staged-query-chunks = query-chunks|))
            s6))
  unfolding staged-after-alpha-prefix-program-def Let-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have data-eq:
  data =
    (|staged-trace-root = fr,
      staged-trace-fri-roots = trace-roots,
      staged-trace-fri-challenges = trace-bs,
      staged-trace-final = trace-final,
      staged-alphas = as,

```

```

    staged-degree = dg,
    staged-composition-fri-roots = composition-roots,
    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks)
  using ret by simp
show staged-trace-root data = fst prefix
  unfolding prefix-eq data-eq by simp
show staged-trace-fri-roots data = fst (snd prefix)
  unfolding prefix-eq data-eq by simp
show staged-trace-fri-challenges data = fst (snd (snd prefix))
  unfolding prefix-eq data-eq by simp
show staged-trace-final data = snd (snd (snd prefix))
  unfolding prefix-eq data-eq by simp
qed
show ?thesis
  by (rule that[OF prefix-out root-eq trace-roots-eq trace-bs-eq
    trace-final-eq prefix-ext])
qed

lemma checked-staged-security-with-data-state-alpha-prefix-tree-output-hitI:
  assumes prefix-out:
    Some (prefix, prefix-state) ∈
      set-dist
        (execute (staged-alpha-prefix-program A)
          adversary-initial-state)
  and root-eq: staged-trace-root data = fst prefix
  and trace-roots-eq:
    staged-trace-fri-roots data = fst (snd prefix)
  and trace-bs-eq:
    staged-trace-fri-challenges data = fst (snd (snd prefix))
  and trace-final-eq:
    staged-trace-final data = snd (snd (snd prefix))
  and verify-out:
    Some (result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))))
  and bound:
    accepted-with-bound-tables
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
    (Some (result, final-state)) trace-table composition-table as
    query-idxs
  and header:
    verifier-header-transcript
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))

```

```

    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    as dg composition-fri-roots final rest
and trace-created: created-tree trace-table trace-tree final-state
and composition-created:
    created-tree composition-table composition-tree final-state
and tree-hit:
    hash-map-new-output-hit
    (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
shows
    checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A
    (Some (((data, attacker-state), checked-result), checked-final-state))
proof -
  have prefix-tuple:
    prefix =
      (staged-trace-root data, staged-trace-fri-roots data,
       staged-trace-fri-challenges data, staged-trace-final data)
  by (cases prefix; cases snd prefix; cases snd (snd prefix))
    (use root-eq trace-roots-eq trace-bs-eq trace-final-eq in auto)
show ?thesis
  unfolding checked-staged-security-with-data-state-alpha-prefix-tree-output-hit-def
    Let-def
  apply (simp add: prefix-tuple)
  apply (rule exI[where x=prefix-state])
  apply (rule conjI)
  using prefix-out prefix-tuple
  apply simp
  apply (rule exI[where x=result])
  apply (rule exI[where x=final-state])
  apply (rule conjI)
  using verify-out
  apply simp
  apply (rule exI[where x=trace-table])
  apply (rule exI[where x=composition-table])
  apply (rule exI[where x=as])
  apply (rule conjI)
  apply (rule exI[where x=query-idxs])
  using bound
  apply simp
  apply (rule conjI)
  apply (rule exI[where x=dg])
  apply (rule exI[where x=composition-fri-roots])
  apply (rule exI[where x=final])
  apply (rule exI[where x=rest])
  using header
  apply simp
  apply (rule exI[where x=trace-tree])

```

```

apply (rule conjI)
  using trace-created
  apply simp
apply (rule exI[where x=composition-tree])
using composition-created tree-hit
apply simp
done
qed

lemma checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-imp-prefix-or-tree-output-hit-on-su
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
and hit:
  checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
    bad-sets out
shows
  checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
    bad-sets out ∨
  checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A
    out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-def
  by simp
next
case (Some result-pack)
then obtain data attacker-state checked-result checked-final-state where
  out-eq:
    out =
      Some (((data, attacker-state), checked-result),
        checked-final-state)
  by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have support-some:
  Some (((data, attacker-state), checked-result),
    checked-final-state) ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)

```

```

using support unfolding out-eq .
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF support-some]
have builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and checked-verify:
    Some (checked-result, checked-final-state) ∈
      set-dist (execute verify-monad ?s)
  by blast+
from checked-staged-transcript-program-alpha-prefix-match-support
  [OF wf controlled builder]
obtain prefix prefix-state where prefix-out:
  Some (prefix, prefix-state) ∈
    set-dist
      (execute (staged-alpha-prefix-program A)
        adversary-initial-state)
  and root-eq: staged-trace-root data = fst prefix
  and trace-roots-eq:
    staged-trace-fri-roots data = fst (snd prefix)
  and trace-bs-eq:
    staged-trace-fri-challenges data = fst (snd (snd prefix))
  and trace-final-eq:
    staged-trace-final data = snd (snd (snd prefix))
  and prefix-ext: prefix-state ≤ attacker-state
  by blast
have prefix-tuple:
  prefix =
    (staged-trace-root data, staged-trace-fri-roots data,
      staged-trace-fri-challenges data, staged-trace-final data)
  by (cases prefix; cases snd prefix; cases snd (snd prefix))
    (use root-eq trace-roots-eq trace-bs-eq trace-final-eq in auto)
have union-bad:
  staged-alphas data ∈
    alpha-header-supported-union-bad-sets ?s bad-sets
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
  using hit unfolding out-eq
    checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-def
  by simp
then have alphas-space: staged-alphas data ∈ alpha-space
  unfolding alpha-header-supported-union-bad-sets-def by auto
from union-bad obtain trace-table where header-candidate:
  trace-table ∈
    alpha-header-supported-trace-table-candidates ?s
    (staged-trace-root data)

```

```

    (staged-trace-fri-roots data)
    (staged-trace-final data)
  and bad:
    staged-alphas data  $\in$  bad-sets trace-table
  unfolding alpha-header-supported-union-bad-sets-def by blast
from header-candidate obtain candidate-out composition-table as query-idxs
  dg composition-fri-roots final rest where candidate-out:
  candidate-out  $\in$  set-dist (execute verify-monad ?s)
and bound:
  accepted-with-bound-tables ?s candidate-out trace-table
  composition-table as query-idxs
and candidate-header:
  verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  as dg composition-fri-roots final rest
  unfolding alpha-header-supported-trace-table-candidates-def by blast
from bound obtain result final-state where candidate-out-eq:
  candidate-out = Some (result, final-state)
  unfolding accepted-with-bound-tables-def by blast
have verifier-out:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)
  using candidate-out candidate-out-eq by simp
have bound-some:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs
  using bound candidate-out-eq by simp
have prefix-s: prefix-state  $\leq$  ?s
  by (rule
    alpha-prefix-hash-extends-verifier-state-from-adversary-right
    [OF prefix-ext])
have s-final: ?s  $\leq$  final-state
  by (rule verify-monad-hash-extends[OF verifier-out])
have prefix-final: prefix-state  $\leq$  final-state
  by (rule hash-ext-trans[OF prefix-s s-final])
show ?thesis
proof (rule
  accepted-with-bound-tables-initial-roots-pullback-or-new-tree-output-hit
  [OF prefix-final bound-some])
  fix fr f-fri-roots f-final dg' composition-fri-roots' final'
  rest' trace-tree composition-tree
  assume pull-header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg'
    composition-fri-roots' final' rest'
  and trace-created:
    created-tree trace-table trace-tree final-state
  and composition-created:
    created-tree composition-table composition-tree final-state

```

```

and split:
  (merkle-root-binds-table fr trace-table prefix-state  $\wedge$ 
   merkle-root-binds-table (hd composition-fri-roots')
   composition-table prefix-state)  $\vee$ 
  hash-map-new-output-hit
  (set-tree trace-tree  $\cup$  set-tree composition-tree)
  prefix-state final-state
have pull-header-eq:
  fr = staged-trace-root data  $\wedge$ 
  f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
  f-final = staged-trace-final data  $\wedge$ 
  dg' = dg  $\wedge$ 
  composition-fri-roots' = composition-fri-roots  $\wedge$ 
  final' = final  $\wedge$ 
  rest' = rest
using verifier-header-transcript-unique[OF candidate-header pull-header]
by simp
then have fr-eq: fr = staged-trace-root data
by simp
from split show ?thesis
proof
assume binds:
  merkle-root-binds-table fr trace-table prefix-state  $\wedge$ 
  merkle-root-binds-table (hd composition-fri-roots')
  composition-table prefix-state
have trace-len: length trace-table = clength * scale
using bound-some unfolding accepted-with-bound-tables-def
  accepted-with-tables-def by simp
have prefix-candidate:
  trace-table  $\in$ 
    alpha-prefix-trace-table-candidates prefix-state
    (staged-trace-root data)
by (rule alpha-prefix-trace-table-candidateI[OF trace-len])
  (use binds fr-eq in simp)
have prefix-bad:
  staged-alphas data  $\in$ 
    alpha-prefix-union-bad-sets prefix-state bad-sets
    (staged-trace-root data)
unfolding alpha-prefix-union-bad-sets-def
using alphas-space prefix-candidate bad by auto
have
  checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
  bad-sets out
unfolding out-eq
  checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-def
apply simp
apply (rule exI[where x=prefix-state])
using prefix-out prefix-tuple prefix-bad
apply simp

```

```

    done
  then show ?thesis by simp
next
  assume tree-hit:
    hash-map-new-output-hit
      (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
  have
    checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A
      out
  unfolding out-eq
  by (rule
    checked-staged-security-with-data-state-alpha-prefix-tree-output-hitI
    [OF prefix-out root-eq trace-roots-eq trace-bs-eq trace-final-eq
      verifier-out bound-some candidate-header trace-created
      composition-created tree-hit])
  then show ?thesis by simp
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit-imp-prefix-or-tree-output-hit-on-
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    out  $\in$ 
      set-dist
        (execute
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
            A)
          adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit
      bad-sets out
  shows
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
      bad-sets out  $\vee$ 
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit out
proof (cases out)
case None
  then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit-def
  by simp
next
case (Some result-pack)
  then obtain prefix prefix-state data attacker-state checked-result
    checked-final-state where out-eq:

```



```

    out =
      Some (((((prefix, prefix-state), data), attacker-state),
        checked-result), checked-final-state)
  by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have support-some:
  Some (((((prefix, prefix-state), data), attacker-state),
    checked-result), checked-final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)
  using support unfolding out-eq .
from support-some obtain trans-out where trans-out:
  Some (((prefix, prefix-state), data), attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-with-alpha-prefix-program A)
    adversary-initial-state)
and checked-verify:
  Some (checked-result, checked-final-state) ∈
  set-dist (execute verify-monad ?s)
unfolding
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
by (auto elim!: set-dist-bindE split: prod.splits)
from checked-staged-transcript-with-alpha-prefix-program-support
  [OF wf controlled trans-out]
have prefix-out:
  Some (prefix, prefix-state) ∈
  set-dist
  (execute (staged-alpha-prefix-program A)
    adversary-initial-state)
and root-eq: staged-trace-root data = fst prefix
and trace-roots-eq:
  staged-trace-fri-roots data = fst (snd prefix)
and trace-bs-eq:
  staged-trace-fri-challenges data = fst (snd (snd prefix))
and trace-final-eq:
  staged-trace-final data = snd (snd (snd prefix))
and prefix-ext: prefix-state ≤ attacker-state
by blast+
have union-bad:
  staged-alphas data ∈
  alpha-header-supported-union-bad-sets ?s bad-sets
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)

```

```

using hit unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit-def
  Let-def
by simp
then have alphas-space: staged-alphas data ∈ alpha-space
  unfolding alpha-header-supported-union-bad-sets-def by auto
from union-bad obtain trace-table where header-candidate:
  trace-table ∈
    alpha-header-supported-trace-table-candidates ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
and bad:
  staged-alphas data ∈ bad-sets trace-table
unfolding alpha-header-supported-union-bad-sets-def by blast
from header-candidate obtain candidate-out composition-table as query-idxs
  dg composition-fri-roots final rest where candidate-out:
  candidate-out ∈ set-dist (execute verify-monad ?s)
and bound:
  accepted-with-bound-tables ?s candidate-out trace-table
  composition-table as query-idxs
and candidate-header:
  verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  as dg composition-fri-roots final rest
unfolding alpha-header-supported-trace-table-candidates-def by blast
from bound obtain result final-state where candidate-out-eq:
  candidate-out = Some (result, final-state)
unfolding accepted-with-bound-tables-def by blast
have verifier-out:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
using candidate-out candidate-out-eq by simp
have bound-some:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs
using bound candidate-out-eq by simp
have prefix-s: prefix-state ≤ ?s
by (rule
  alpha-prefix-hash-extends-verifier-state-from-adversary-right
  [OF prefix-ext])
have s-final: ?s ≤ final-state
by (rule verify-monad-hash-extends [OF verifier-out])
have prefix-final: prefix-state ≤ final-state
by (rule hash-ext-trans [OF prefix-s s-final])
show ?thesis
proof (rule
  accepted-with-bound-tables-initial-roots-pullback-or-new-tree-output-hit

```

```

    [OF prefix-final bound-some])
fix fr f-fri-roots f-final dg' composition-fri-roots' final'
    rest' trace-tree composition-tree
assume pull-header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg'
    composition-fri-roots' final' rest'
and trace-created:
    created-tree trace-table trace-tree final-state
and composition-created:
    created-tree composition-table composition-tree final-state
and split:
    (merkle-root-binds-table fr trace-table prefix-state ∧
     merkle-root-binds-table (hd composition-fri-roots')
     composition-table prefix-state) ∨
    hash-map-new-output-hit
    (set-tree trace-tree ∪ set-tree composition-tree)
    prefix-state final-state
have pull-header-eq:
    fr = staged-trace-root data ∧
    f-fri-roots = staged-trace-fri-roots data ∧
    f-final = staged-trace-final data ∧
    dg' = dg ∧
    composition-fri-roots' = composition-fri-roots ∧
    final' = final ∧
    rest' = rest
using verifier-header-transcript-unique[OF candidate-header pull-header]
by simp
then have fr-eq: fr = staged-trace-root data
by simp
from split show ?thesis
proof
assume binds:
    merkle-root-binds-table fr trace-table prefix-state ∧
    merkle-root-binds-table (hd composition-fri-roots')
    composition-table prefix-state
have trace-len: length trace-table = clength * scale
using bound-some unfolding accepted-with-bound-tables-def
    accepted-with-tables-def by simp
have prefix-candidate:
    trace-table ∈
    alpha-prefix-trace-table-candidates prefix-state (fst prefix)
by (rule alpha-prefix-trace-table-candidateI[OF trace-len])
    (use binds fr-eq root-eq in simp)
have prefix-bad:
    staged-alphas data ∈
    alpha-prefix-union-bad-sets prefix-state bad-sets (fst prefix)
unfolding alpha-prefix-union-bad-sets-def
using alphas-space prefix-candidate bad by auto
have

```

```

    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
      bad-sets out
    unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-bad-set-hit-def
      Let-def
    using prefix-bad
    by simp
  then show ?thesis by simp
next
  assume tree-hit:
    hash-map-new-output-hit
      (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
  have
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit
      out
    unfolding out-eq
    by (rule
      checked-staged-security-with-actual-alpha-prefix-tree-output-hitI
      [OF verifier-out bound-some candidate-header trace-created
        composition-created tree-hit])
    then show ?thesis by simp
qed
qed
qed

lemma checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-bound-from-actual-prefix-and-tree:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and prefix-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
        bad-sets)
      adversary-initial-state  $\leq P$ 
  and tree-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-tree-output-hit
      adversary-initial-state  $\leq T$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
        bad-sets)
      adversary-initial-state  $\leq P + T$ 
proof -
  let ?M =

```

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let *?Prefix* =
checked-staged-security-with-actual-alpha-prefix-bad-set-hit bad-sets
let *?Tree* = *checked-staged-security-with-actual-alpha-prefix-tree-output-hit*
let *?Header* =
checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit
bad-sets
have *wp-event (checked-staged-security-experiment-with-data-state A)*
(checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
bad-sets)
adversary-initial-state =
wp-event ?M ?Header adversary-initial-state
by (rule
checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-actual-projection)
also have ... $\leq$ *wp-event ?M (λ out. ?Prefix out $\vee$?Tree out)*
adversary-initial-state
proof (rule *wp-event-mono-on-support*)
fix *out*
assume *support: out $\in$ set-dist (execute ?M adversary-initial-state)*
and *hit: ?Header out*
show *?Prefix out $\vee$?Tree out*
by (rule
checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit-imp-prefix-or-tree-output-hit-on-su
[OF wf controlled support hit])
qed
also have ... $\leq$
wp-event ?M ?Prefix adversary-initial-state +
wp-event ?M ?Tree adversary-initial-state
by (rule *wp-event-union-bound*)
also have ... $\leq P + T$
by (rule *add-mono[OF prefix-bound tree-bound]*)
finally show *?thesis .*
qed

lemma *checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-bound-from-prefix-and-tree:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *prefix-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
bad-sets)
adversary-initial-state $\leq P$
and *tree-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A)
adversary-initial-state $\leq T$
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit

```

      bad-sets)
    adversary-initial-state  $\leq P + T$ 
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?Prefix =
    checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
    bad-sets
  let ?Tree =
    checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A
  let ?Header =
    checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
    bad-sets
  have wp-event ?M ?Header adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda$ out. ?Prefix out  $\vee$  ?Tree out)
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out  $\in$  set-dist (execute ?M adversary-initial-state)
      and hit: ?Header out
    show ?Prefix out  $\vee$  ?Tree out
      by (rule
        checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-imp-prefix-or-tree-output-hit-on-suppp
        [OF wf controlled support hit])
  qed
also have ...  $\leq$ 
  wp-event ?M ?Prefix adversary-initial-state +
  wp-event ?M ?Tree adversary-initial-state
  by (rule wp-event-union-bound)
also have ...  $\leq P + T$ 
  by (rule add-mono[OF prefix-bound tree-bound])
finally show ?thesis .
qed

```

lemma checked-staged-security-with-data-state-composition-bad-bound-from-alpha-prefix-and-tree:

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and prefix-bound:

wp-event (checked-staged-security-experiment-with-data-state A)

(checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A

composition-trace-bad-alpha-space)

adversary-initial-state $\leq P$

and tree-bound:

wp-event (checked-staged-security-experiment-with-data-state A)

(checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A)

adversary-initial-state $\leq T$

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event composition-bad)

$adversary-initial-state \leq P + T$
proof –
have *alpha-bound*:
 $wp-event$ (*checked-staged-security-experiment-with-data-state A*)
(*checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit*
composition-trace-bad-alpha-space)
 $adversary-initial-state \leq P + T$
by (*rule*
checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-bound-from-prefix-and-tree
[*OF wf controlled prefix-bound tree-bound*])
show *?thesis*
by (*rule*
checked-staged-security-with-data-state-composition-bad-bound-from-alpha-header-supported-bad-set-general
[*OF wf controlled alpha-bound*])
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-and-tree*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *prefix-bound*:

$wp-event$
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
composition-trace-bad-alpha-space)
 $adversary-initial-state \leq P$

and *tree-bound*:

$wp-event$
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-tree-output-hit
 $adversary-initial-state \leq T$

shows

$wp-event$ (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event composition-bad*)
 $adversary-initial-state \leq P + T$

proof –

have *alpha-bound*:

$wp-event$ (*checked-staged-security-experiment-with-data-state A*)
(*checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit*
composition-trace-bad-alpha-space)
 $adversary-initial-state \leq P + T$

by (*rule*

checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-bound-from-actual-prefix-and-tree
[*OF wf controlled prefix-bound tree-bound*])

show *?thesis*

by (*rule*

checked-staged-security-with-data-state-composition-bad-bound-from-alpha-header-supported-bad-set-general
[*OF wf controlled alpha-bound*])

qed

end

end

theory *Staged-Security-Experiment-Composition-Tree-Output*
 imports *Staged-Security-Experiment-Composition*
begin

Bridges for the composition tree-output side event.

The actual-alpha-prefix experiment carries the concrete alpha-prefix state through the verifier run. This layer relates that path-specific event back to the ordinary checked data-state experiment, so later bounds can be proved once for the data-state event.

context *soundness*
begin

lemma *hash-map-new-output-hit-Un-iff*:
 $\text{hash-map-new-output-hit } (A \cup B) \ s \ t \longleftrightarrow$
 $\text{hash-map-new-output-hit } A \ s \ t \vee \text{hash-map-new-output-hit } B \ s \ t$
unfolding *hash-map-new-output-hit-def* **by** *blast*

lemma *hash-new-output-hit-event-Un-iff*:
 $\text{hash-new-output-hit-event } (A \cup B) \ s \ \text{out} \longleftrightarrow$
 $\text{hash-new-output-hit-event } A \ s \ \text{out} \vee$
 $\text{hash-new-output-hit-event } B \ s \ \text{out}$
unfolding *hash-new-output-hit-event-def*
by (*cases out*) (*auto simp: hash-map-new-output-hit-Un-iff*)

lemma *hash-map-new-output-hit-tree-root-or-subtree*:
 assumes *hit*: $\text{hash-map-new-output-hit } (\text{set-tree } \text{tree}) \ s \ t$
 shows
 $\text{hash-map-new-output-hit } \{\text{value } \text{tree}\} \ s \ t \vee$
 $(\exists l \ v \ r. \ \text{tree} = \langle l, v, r \rangle \wedge$
 $\text{hash-map-new-output-hit } (\text{set-tree } l \cup \text{set-tree } r) \ s \ t)$
 using *hit*
 unfolding *hash-map-new-output-hit-def*
 by (*cases tree*) *auto*

lemma *hash-map-new-output-hit-two-trees-root-or-subtree*:
 assumes *hit*:
 $\text{hash-map-new-output-hit } (\text{set-tree } \text{trace-tree} \cup \text{set-tree } \text{composition-tree})$
 $s \ t$
 shows
 $\text{hash-map-new-output-hit } \{\text{value } \text{trace-tree}, \text{value } \text{composition-tree}\} \ s \ t \vee$
 $(\exists l \ v \ r. \ \text{trace-tree} = \langle l, v, r \rangle \wedge$
 $\text{hash-map-new-output-hit } (\text{set-tree } l \cup \text{set-tree } r) \ s \ t) \vee$
 $(\exists l \ v \ r. \ \text{composition-tree} = \langle l, v, r \rangle \wedge$


```

      hash-map-new-output-hit (set-tree l  $\cup$  set-tree r) s t)
proof -
  from hit have split:
    hash-map-new-output-hit (set-tree trace-tree) s t  $\vee$ 
    hash-map-new-output-hit (set-tree composition-tree) s t
  unfolding hash-map-new-output-hit-Un-iff .
  from split show ?thesis
proof
  assume trace-hit: hash-map-new-output-hit (set-tree trace-tree) s t
  from hash-map-new-output-hit-tree-root-or-subtree[OF trace-hit]
  show ?thesis
proof
  assume root-hit: hash-map-new-output-hit {value trace-tree} s t
  have hash-map-new-output-hit
    {value trace-tree, value composition-tree} s t
    by (rule hash-map-new-output-hit-subset[OF - root-hit]) auto
  then show ?thesis by simp
next
  assume  $\exists l v r. \text{trace-tree} = \langle l, v, r \rangle \wedge$ 
    hash-map-new-output-hit (set-tree l  $\cup$  set-tree r) s t
  then show ?thesis by blast
qed
next
  assume composition-hit:
    hash-map-new-output-hit (set-tree composition-tree) s t
  from hash-map-new-output-hit-tree-root-or-subtree[OF composition-hit]
  show ?thesis
proof
  assume root-hit:
    hash-map-new-output-hit {value composition-tree} s t
  have hash-map-new-output-hit
    {value trace-tree, value composition-tree} s t
    by (rule hash-map-new-output-hit-subset[OF - root-hit]) auto
  then show ?thesis by simp
next
  assume  $\exists l v r. \text{composition-tree} = \langle l, v, r \rangle \wedge$ 
    hash-map-new-output-hit (set-tree l  $\cup$  set-tree r) s t
  then show ?thesis by blast
qed
qed
qed

lemma card-set-tree-le-inorder-length:
  card (set-tree t)  $\leq$  length (inorder t)
  using card-length[of inorder t]
  by simp

lemma protocol-created-tree-inorder-length-bound:
  assumes created: protocol-created-tree xs t s

```

```

shows length (inorder t) ≤ 2 * length xs - 1
using created
proof (induction xs arbitrary: t rule: length-induct)
case (1 xs)
show ?case
proof (cases length xs = 0)
case True
then have xs-eq: xs = []
by simp
then show ?thesis
using 1.prem
unfolding protocol-created-tree-def
by (simp add: protocol-merkle.created-tree.simps)
next
case False
show ?thesis
proof (cases length xs = 1)
case True
then obtain x where xs-eq: xs = [x]
by (cases xs) auto
from protocol-created-tree-singletonD[OF 1.prem[unfolded xs-eq]]
obtain h where t-eq: t = ⟨⟨⟩, h, ⟨⟩⟩
by blast
then show ?thesis
using xs-eq by simp
next
case False
then have len-ge: 2 ≤ length xs
using ⟨length xs ≠ 0⟩ by linarith
let ?i = length xs div 2
have i-pos: 0 < ?i
using len-ge by simp
have i-lt: ?i < length xs
using len-ge by simp
from 1.prem obtain l h r where
i-def: ?i = length xs div 2
and
t-eq: t = ⟨l, h, r⟩
and left: protocol-created-tree (take ?i xs) l s
and right: protocol-created-tree (drop ?i xs) r s
using protocol-created-tree-internalD[OF 1.prem len-ge]
by blast
have drop-lt: length (drop ?i xs) < length xs
using i-pos by simp
have right-bound: length (inorder r) ≤
2 * length (drop ?i xs) - 1
using 1.IH[rule-format, OF drop-lt, of r] right by simp
have take-lt: length (take ?i xs) < length xs
using i-lt by simp

```

```

have left-bound: length (inorder l) ≤
  2 * length (take ?i xs) - 1
  using 1.IH[rule-format, OF take-lt, of l] left by simp
have take-pos: 0 < length (take ?i xs)
  using i-pos i-lt by simp
have drop-pos: 0 < length (drop ?i xs)
  using i-lt by simp
have length (inorder t) =
  Suc (length (inorder l) + length (inorder r))
  using t-eq by simp
also have ... ≤
  Suc ((2 * length (take ?i xs) - 1) +
    (2 * length (drop ?i xs) - 1))
  using left-bound right-bound by simp
also have ... = 2 * length xs - 1
  using take-pos drop-pos i-lt by simp
finally show ?thesis .
qed
qed
qed

lemma created-tree-inorder-length-bound:
  assumes created: created-tree xs t s
  shows length (inorder t) ≤ 2 * length xs - 1
  using protocol-created-tree-inorder-length-bound[
    OF created[unfolded created-tree-def]]
  .

lemma created-tree-card-set-tree-bound:
  assumes created: created-tree xs t s
  shows card (set-tree t) ≤ 2 * length xs - 1
  by (rule order.trans[OF card-set-tree-le-inorder-length
    created-tree-inorder-length-bound[OF created]])

lemma protocol-merkle-created-tree-same-inputs-value-eq:
  assumes left: protocol-merkle.created-tree xs left-tree s
  and right: protocol-merkle.created-tree xs right-tree s
  shows value left-tree = value right-tree
  using assms
proof (induction xs arbitrary: left-tree right-tree rule: length-induct)
  case (1 xs)
  show ?case
  proof (cases xs)
    case Nil
    then show ?thesis
      using 1.prem1
      by (simp add: protocol-merkle.created-tree.simps)
  next
    case (Cons x rest)

```

```

have xs-cons: xs = x # rest
  using Cons by simp
show ?thesis
proof (cases rest)
case Nil
then obtain left-hash right-hash where
  left-tree:
    left-tree = ⟨⟨⟩, left-hash, ⟨⟩⟩
    fmllookup (HashMap s) x = Some left-hash
  and right-tree:
    right-tree = ⟨⟨⟩, right-hash, ⟨⟩⟩
    fmllookup (HashMap s) x = Some right-hash
  using 1.prem1 Cons Nil
  by (auto simp: protocol-merkle.created-tree.simps)
then show ?thesis by simp
next
case (Cons y ys)
have xs-eq: xs = x # y # ys
  using xs-cons Cons by simp
let ?i = length xs div 2
have len: 2 ≤ length xs
  using xs-eq by simp
obtain ll lh lr i where i-def: i = ?i
  and left-tree-eq: left-tree = ⟨ll, lh, lr⟩
  and ll-created: protocol-merkle.created-tree (take i xs) ll s
  and lr-created: protocol-merkle.created-tree (drop i xs) lr s
  and lh-lookup:
    fmllookup (HashMap s) (MerkleNode (value ll) (value lr)) =
      Some lh
  using 1.prem1(1) xs-eq
  by (auto simp: protocol-merkle.created-tree.simps Let-def)
obtain rl rh rr j where j-def: j = ?i
  and right-tree-eq: right-tree = ⟨rl, rh, rr⟩
  and rl-created: protocol-merkle.created-tree (take j xs) rl s
  and rr-created: protocol-merkle.created-tree (drop j xs) rr s
  and rh-lookup:
    fmllookup (HashMap s) (MerkleNode (value rl) (value rr)) =
      Some rh
  using 1.prem1(2) xs-eq
  by (auto simp: protocol-merkle.created-tree.simps Let-def)
have j-i: j = i
  using i-def j-def by simp
have i-pos: 0 < i
  using i-def len by simp
have i-lt: i < length xs
  using i-def len by simp
have take-less: length (take i xs) < length xs
  using i-lt by simp
have drop-less: length (drop i xs) < length xs

```

```

    using i-pos i-lt by simp
  have left-value: value ll = value rl
    by (rule 1.IH[rule-format, OF take-less ll-created])
      (use rl-created j-i in simp)
  have right-value: value lr = value rr
    by (rule 1.IH[rule-format, OF drop-less lr-created])
      (use rr-created j-i in simp)
  have lh = rh
    using lh-lookup rh-lookup left-value right-value by simp
  then show ?thesis
    using left-tree-eq right-tree-eq by simp
qed
qed
qed

lemma protocol-created-tree-same-table-value-eq:
  assumes left: protocol-created-tree xs left-tree s
    and right: protocol-created-tree xs right-tree s
  shows value left-tree = value right-tree
  by (rule protocol-merkle-created-tree-same-inputs-value-eq
      [OF left[unfolded protocol-created-tree-def]
        right[unfolded protocol-created-tree-def]])

lemma created-tree-same-table-value-eq:
  assumes left: created-tree xs left-tree s
    and right: created-tree xs right-tree s
  shows value left-tree = value right-tree
  by (rule protocol-created-tree-same-table-value-eq
      [OF left[unfolded created-tree-def] right[unfolded created-tree-def]])

definition merkle-node-child-output-relation
  :: 'f protocol-hash-input  $\Rightarrow$  'f  $\Rightarrow$  bool
where
  merkle-node-child-output-relation x y  $\longleftrightarrow$ 
    ( $\exists z. x = \text{MerkleNode } y \ z \vee x = \text{MerkleNode } z \ y$ )

lemma merkle-node-child-output-relation-values-subset:
  {y. merkle-node-child-output-relation x y}  $\subseteq$ 
    (case x of
      MerkleNode l r  $\Rightarrow$  {l, r}
    | -  $\Rightarrow$  {})
  unfolding merkle-node-child-output-relation-def
  by (cases x) auto

lemma merkle-node-child-output-relation-fiber-card-bound:
  card {y. merkle-node-child-output-relation x y}  $\leq 2$ 
proof -
  have finite-values:
    finite

```

```

    (case x of
      MerkleNode l r  $\Rightarrow$  {l, r}
    | -  $\Rightarrow$  {})
  by (cases x) simp-all
have card {y. merkle-node-child-output-relation x y}  $\leq$ 
  card
    (case x of
      MerkleNode l r  $\Rightarrow$  {l, r}
    | -  $\Rightarrow$  {})
  by (rule card-mono[OF finite-values])
    (rule merkle-node-child-output-relation-values-subset)
also have ...  $\leq$  2
proof (cases x)
  case (MerkleNode l r)
  then show ?thesis
    by (simp add: card-insert-if)
qed simp-all
finally show ?thesis .
qed

```

definition *merkle-root-or-child-output-relation*
 $:: 'f \text{ set} \Rightarrow 'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$
where
 $\text{merkle-root-or-child-output-relation roots } x \ y \longleftrightarrow$
 $y \in \text{roots} \vee \text{merkle-node-child-output-relation } x \ y$

lemma *merkle-root-or-child-output-relation-values-subset*:
 $\{y. \text{merkle-root-or-child-output-relation roots } x \ y\} \subseteq$
 $\text{roots} \cup$
 $(\text{case } x \text{ of}$
 $\text{MerkleNode } l \ r \Rightarrow \{l, r\}$
 $| - \Rightarrow \{\})$
unfolding *merkle-root-or-child-output-relation-def*
using *merkle-node-child-output-relation-values-subset*
by *blast*

lemma *merkle-root-or-child-output-relation-fiber-card-bound*:
assumes *finite-roots*: *finite roots*
shows $\text{card } \{y. \text{merkle-root-or-child-output-relation roots } x \ y\} \leq$
 $\text{card roots} + 2$
proof –
have *finite-values*:
finite
 $(\text{roots} \cup$
 $(\text{case } x \text{ of}$
 $\text{MerkleNode } l \ r \Rightarrow \{l, r\}$
 $| - \Rightarrow \{\}))$
using *finite-roots* **by** (cases x) simp-all
have $\text{card } \{y. \text{merkle-root-or-child-output-relation roots } x \ y\} \leq$

```

    card
    (roots  $\cup$ 
      (case  $x$  of
        MerkleNode  $l\ r \Rightarrow \{l, r\}$ 
        |  $- \Rightarrow \{\}$ )))
  by (rule card-mono[OF finite-values])
  (rule merkle-root-or-child-output-relation-values-subset)
also have ...  $\leq$ 
  card roots +
  card
  (case  $x$  of
    MerkleNode  $l\ r \Rightarrow \{l, r\}$ 
    |  $- \Rightarrow \{\}$ )
proof -
  show ?thesis
  by (rule card-Un-le)
qed
also have ...  $\leq$  card roots + 2
proof (cases  $x$ )
  case (MerkleNode  $l\ r$ )
  then show ?thesis
  by (simp add: card-insert-if)
qed simp-all
finally show ?thesis .
qed

lemma hash-relation-program-protocol-check-authentication-path-raw:
  fixes leaf :: 'f protocol-hash-input
  assumes fibers:  $\bigwedge x. \text{card } \{y. R\ x\ y\} \leq b$ 
  shows hash-relation-program  $R\ b\ (\text{Suc } (\text{length } \text{path}))$ 
    (protocol-merkle.check-authentication-path len  $i$  leaf path ::
      ('f, 'f protocol-channel) state-monad)
proof (induction path arbitrary: len  $i$ )
  case Nil
  have hash-relation-program  $R\ b\ 1$ 
    (hash leaf :: ('f, 'f protocol-channel) state-monad)
  by (rule hash-relation-program-hash[OF fibers])
  then show ?case
  by (simp add: protocol-merkle.check-authentication-path.simps)
next
  case (Cons  $a\ \text{path}$ )
  show ?case
proof (cases  $i < \text{len } \text{div } 2$ )
  case True
  have rec:
    hash-relation-program  $R\ b\ (\text{Suc } (\text{length } \text{path}))$ 
    (protocol-merkle.check-authentication-path (len div 2)  $i$  leaf path ::
      ('f, 'f protocol-channel) state-monad)
  using Cons.IH[of len div 2  $i$ ] by simp

```

```

have bound:
  hash-relation-program R b (Suc (length path) + 1)
    ((protocol-merkle.check-authentication-path (len div 2) i leaf path ::
      ('f, 'f protocol-channel) state-monad)  $\gg$ 
      ( $\lambda x$ . hash (MerkleNode x a)))
  by (rule hash-relation-program-bind)
    (rule rec, rule hash-relation-program-hash[OF fibers])
show ?thesis
  using True bound
  by (simp add: protocol-merkle.check-authentication-path.simps)
next
case False
have rec:
  hash-relation-program R b (Suc (length path))
    (protocol-merkle.check-authentication-path
      (len div 2) (i - len div 2) leaf path ::
      ('f, 'f protocol-channel) state-monad)
  using Cons.IH[of len div 2 i - len div 2] by simp
have bound:
  hash-relation-program R b (Suc (length path) + 1)
    ((protocol-merkle.check-authentication-path
      (len div 2) (i - len div 2) leaf path ::
      ('f, 'f protocol-channel) state-monad)  $\gg$ 
      ( $\lambda x$ . hash (MerkleNode a x)))
  by (rule hash-relation-program-bind)
    (rule rec, rule hash-relation-program-hash[OF fibers])
show ?thesis
  using False bound
  by (simp add: protocol-merkle.check-authentication-path.simps)
qed
qed

lemma hash-relation-program-check-authentication-path:
  assumes fibers:  $\bigwedge x$ . card {y. R x y}  $\leq$  b
  shows hash-relation-program R b (Suc (length path))
    (check-authentication-path len i v path)
  unfolding check-authentication-path-def protocol-check-authentication-path-def
  by (rule hash-relation-program-protocol-check-authentication-path-raw
    [OF fibers])

lemma hash-relation-program-check-authentication-path-assert-return:
  fixes path :: 'f list
  and v root :: 'f
  and r :: 'r
  assumes fibers:  $\bigwedge x$ . card {y. R x y}  $\leq$  b
  shows
    hash-relation-program R b (Suc (length path))
      ((check-authentication-path len i v path ::
        ('f, 'f protocol-channel) state-monad)  $\gg$ 
        ('f, 'f protocol-channel) state-monad)  $\gg$ 

```



```

      (λap. assert (ap = root) ≫= (λ-. return r)))
proof -
  have tail:
    hash-relation-program R b (0 + 0)
    (assert (ap = root) ≫= (λ-. return r) ::
      ('r, 'f protocol-channel) state-monad)
  for ap
  by (rule hash-relation-program-bind)
    (rule hash-relation-program-assert, rule hash-relation-program-zero
      [OF hash-map-preserving-return])
  have hash-relation-program R b (Suc (length path) + (0 + 0))
    ((check-authentication-path len i v path ::
      ('f, 'f protocol-channel) state-monad) ≫=
      (λap. assert (ap = root) ≫= (λ-. return r)))
  by (rule hash-relation-program-bind)
    (rule hash-relation-program-check-authentication-path[OF fibers],
      rule tail)
  then show ?thesis by simp
qed

```

lemma hash-relation-program-check-authentication-path-assert-bind:

```

fixes path :: 'f list
and v root :: 'f
and k :: unit ⇒ ('r, 'f protocol-channel) state-monad
assumes fibers: ∧x. card {y. R x y} ≤ b
and k: ∧u. hash-relation-program R b n (k u)
shows
  hash-relation-program R b (Suc (length path) + n)
  ((check-authentication-path len i v path ::
    ('f, 'f protocol-channel) state-monad) ≫=
    (λap. assert (ap = root) ≫= k))
proof -
  have tail:
    hash-relation-program R b (0 + n)
    (assert (ap = root) ≫= k ::
      ('r, 'f protocol-channel) state-monad)
  for ap
  by (rule hash-relation-program-bind)
    (rule hash-relation-program-assert, rule k)
  have hash-relation-program R b (Suc (length path) + (0 + n))
    ((check-authentication-path len i v path ::
      ('f, 'f protocol-channel) state-monad) ≫=
      (λap. assert (ap = root) ≫= k))
  by (rule hash-relation-program-bind)
    (rule hash-relation-program-check-authentication-path[OF fibers],
      rule tail)
  then show ?thesis by simp
qed

```

```

lemma hash-relation-program-read:
  hash-relation-program R b 0
  (read :: ('f, 'f, unit) protocol-c-monad)
by (rule hash-relation-program-zero)
  (rule hash-map-preserving-read)

lemma hash-relation-program-ntimes:
  fixes m :: ('x, 'f protocol-channel) state-monad
  assumes m: hash-relation-program R b n m
  shows hash-relation-program R b (k * n) (ntimes m k)
  using m
proof (induction k)
  case 0
  then show ?case
    by (simp add: hash-relation-program-zero[OF hash-map-preserving-return])
next
  case (Suc k)
  have tail: hash-relation-program R b (k * n) (ntimes m k)
    using Suc.IH Suc.premis by simp
  have cont:
    hash-relation-program R b (k * n + 0)
    (ntimes m k  $\gg$  (λxs. return (x # xs))) for x
  by (rule hash-relation-program-bind)
  (rule tail, rule hash-relation-program-zero[OF hash-map-preserving-return])
  have hash-relation-program R b (n + (k * n + 0))
    (m  $\gg$  (λx. ntimes m k  $\gg$  (λxs. return (x # xs))))
  by (rule hash-relation-program-bind[OF Suc.premis cont])
  then show ?case by simp
qed

lemma hash-relation-program-mmap:
  fixes ms :: ('x, 'f protocol-channel) state-monad list
  assumes ms:  $\bigwedge m. m \in \text{set } ms \implies \text{hash-relation-program } R \text{ } b \text{ } n \text{ } m$ 
  shows hash-relation-program R b (length ms * n) (mmap ms)
  using ms
proof (induction ms)
  case Nil
  then show ?case
    by (simp add: hash-relation-program-zero[OF hash-map-preserving-return])
next
  case (Cons m ms)
  have head: hash-relation-program R b n m
    using Cons.premis by simp
  have tail: hash-relation-program R b (length ms * n) (mmap ms)
    using Cons.IH Cons.premis by simp
  have cont:
    hash-relation-program R b (length ms * n + 0)
    (mmap m  $\gg$  (λxs. return (x # xs))) for x
  by (rule hash-relation-program-bind)

```

```

    (rule tail, rule hash-relation-program-zero[OF hash-map-preserving-return])
  have hash-relation-program R b (n + (length ms * n + 0))
    (m  $\gg$  (λx. mmap ms  $\gg$  (λxs. return (x # xs))))
    by (rule hash-relation-program-bind[OF head cont])
  then show ?case by simp
qed

```

```

lemma hash-relation-program-mfold:
  fixes steps :: ('x  $\Rightarrow$  ('x, 'f protocol-channel) state-monad) list
  assumes steps:
     $\bigwedge$  step x. step  $\in$  set steps  $\implies$ 
      hash-relation-program R b n (step x)
  shows hash-relation-program R b (length steps * n) (mfold x steps)
  using steps
proof (induction steps arbitrary: x)
  case Nil
  then show ?case
    by (simp add: hash-relation-program-zero[OF hash-map-preserving-return])
next
  case (Cons step steps)
  have head: hash-relation-program R b n (step x)
    using Cons.premis by simp
  have tail:
     $\bigwedge$  y. hash-relation-program R b (length steps * n) (mfold y steps)
    using Cons.IH Cons.premis by simp
  have hash-relation-program R b (n + length steps * n)
    (step x  $\gg$  (λy. mfold y steps))
    by (rule hash-relation-program-bind[OF head tail])
  then show ?case by simp
qed

```

```

lemma hash-relation-program-ntimes-read-bind:
  fixes k :: 'f list  $\Rightarrow$  ('y, 'f protocol-channel) state-monad
  assumes k:
     $\bigwedge$  xs. length xs = n  $\implies$  hash-relation-program R b q (k xs)
  shows
    hash-relation-program R b q
    ((ntimes read n :: ('f list, 'f protocol-channel) state-monad)  $\gg$  k)
proof -
  have reads:
    hash-relation-program R b (n * 0)
    (ntimes read n :: ('f list, 'f protocol-channel) state-monad)
    by (rule hash-relation-program-ntimes[OF hash-relation-program-read])
  have hash-relation-program R b (n * 0 + q)
    ((ntimes read n :: ('f list, 'f protocol-channel) state-monad)  $\gg$  k)
proof (rule hash-relation-program-bind-on-outcomes[OF reads])
  fix s xs t
  assume out:
    Some (xs, t)  $\in$ 

```

```

    set-dist
    (execute
      (ntimes read n :: ('f list, 'f protocol-channel) state-monad)
      s)
  then have length xs = n
    using ntimes-read-any-outcome by blast
  then show hash-relation-program R b q (k xs)
    by (rule k)
qed
then show ?thesis by simp
qed

lemma hash-relation-program-check-decommit-on-query-step:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
    hash-relation-program R b (Suc (floor-log (scale * clength)))
    (do {
      qh ← read;
      let len = scale * clength;
      qh-path ← ntimes read (floor-log len);
      ap ← check-authentication-path len i qh qh-path;
      assert (ap = fr);
      return qh
    })
  proof -
    let ?len = scale * clength
    have after-path:
      hash-relation-program R b (Suc (floor-log ?len))
      ((check-authentication-path ?len i qh qh-path ::
        ('f, 'f protocol-channel) state-monad)  $\gg=$ 
        ( $\lambda$ ap. assert (ap = fr)  $\gg=$  ( $\lambda$ -. return qh)))
    if path-len: length qh-path = floor-log ?len
    for qh qh-path
    using path-len
    by (subst path-len[symmetric])
      (rule hash-relation-program-check-authentication-path-assert-return
        [OF fibers])
    have after-reads:
      hash-relation-program R b (Suc (floor-log ?len))
      ((ntimes read (floor-log ?len) ::
        ('f list, 'f protocol-channel) state-monad)  $\gg=$ 
        ( $\lambda$ qh-path.
          (check-authentication-path ?len i qh qh-path ::
            ('f, 'f protocol-channel) state-monad)  $\gg=$ 
            ( $\lambda$ ap. assert (ap = fr)  $\gg=$  ( $\lambda$ -. return qh))))
    for qh
    by (rule hash-relation-program-ntimes-read-bind)
      (rule after-path)
  have hash-relation-program R b (0 + Suc (floor-log ?len))

```

```

(read >>=
  (λqh.
    (ntimes read (floor-log ?len) ::
      ('f list, 'f protocol-channel) state-monad) >>=
    (λqh-path.
      (check-authentication-path ?len i qh qh-path ::
        ('f, 'f protocol-channel) state-monad) >>=
      (λap. assert (ap = fr) >>= (λ-. return qh))))))
by (rule hash-relation-program-bind)
   (rule hash-relation-program-read, rule after-reads)
then show ?thesis
  by (simp add: Let-def)
qed

lemma hash-relation-program-check-decommit-on-query:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
    hash-relation-program R b
      (length (powers-scaled idx) * Suc (floor-log (scale * clength)))
      (mmap (check-decommit-on-query fr idx))
proof -
  let ?step =
    λi. do {
      qh ← read;
      let len = scale * clength;
      qh-path ← ntimes read (floor-log len);
      ap ← check-authentication-path len i qh qh-path;
      assert (ap = fr);
      return qh
    }
  have mapped-general:
     $\bigwedge xs. \text{hash-relation-program } R \ b$ 
      (length xs + length xs * floor-log (scale * clength))
      (mmap (map ?step xs))
proof -
  fix xs
  show hash-relation-program R b
    (length xs + length xs * floor-log (scale * clength))
    (mmap (map ?step xs))
proof (induction xs)
case Nil
then show ?case
  by (simp add: hash-relation-program-zero[OF hash-map-preserving-return])
next
case (Cons i rest)
have head:
  hash-relation-program R b (Suc (floor-log (scale * clength)))
    (?step i)
  by (rule hash-relation-program-check-decommit-on-query-step[OF fibers])

```

```

have tail:
  hash-relation-program R b
    (length rest + length rest * floor-log (scale * clength))
    (mmap (map ?step rest))
  by (rule Cons.IH)
have cont:
  hash-relation-program R b
    ((length rest + length rest * floor-log (scale * clength)) + 0)
    (mmap (map ?step rest)  $\gg$  ( $\lambda x s$ . return (x # xs)))
  for x
  by (rule hash-relation-program-bind)
    (rule tail, rule hash-relation-program-zero[OF hash-map-preserving-return])
have hash-relation-program R b
  (Suc (floor-log (scale * clength)) +
    ((length rest + length rest * floor-log (scale * clength)) + 0))
  (?step i  $\gg$  ( $\lambda x$ . mmap (map ?step rest)  $\gg$ 
    ( $\lambda x s$ . return (x # xs)))))
  by (rule hash-relation-program-bind[OF head cont])
then show ?case
  by (simp add: algebra-simps)
qed
qed
have mapped:
  hash-relation-program R b
    (length (powers-scaled idx) +
      length (powers-scaled idx) * floor-log (scale * clength))
    (mmap (map ?step (powers-scaled idx)))
  by (rule mapped-general)
show ?thesis
  using mapped
  unfolding check-decommit-on-query-def by simp
qed

lemma hash-relation-program-fri-layer-opening-finish:
  assumes fibers:  $\bigwedge x$ . card {y. R x y}  $\leq$  b
  and xp-path-len: length xp-path = floor-log len
  and xn-path-len: length xn-path = floor-log len
  shows
    hash-relation-program R b
      (Suc (floor-log len) + Suc (floor-log len))
      (fri-layer-opening-finish b' f i x len pw xp xp-path xn xn-path)
proof -
  let ?H = Suc (floor-log len)
  let ?sidx = (i + len div 2) mod len
  let ?out =
    (i mod (len div 2),
      (xp + xn) div 2 +
        b' * ((xp - xn) div (2 * ((h ^ i) * shift) ^ pw)),
      len div 2, pw + pw)

```

```

have second:
  hash-relation-program R b ?H
  ((check-authentication-path len ?sidx xn xn-path ::
    ('f, 'f protocol-channel) state-monad) >>=
    (λap. assert (ap = f) >>= (λ-. return ?out)))
  by (subst xn-path-len[symmetric])
    (rule hash-relation-program-check-authentication-path-assert-return
      [OF fibers])
have first0:
  hash-relation-program R b (Suc (length xp-path) + ?H)
  ((check-authentication-path len i xp xp-path ::
    ('f, 'f protocol-channel) state-monad) >>=
    (λap. assert (ap = f) >>=
      (λ-.
        (check-authentication-path len ?sidx xn xn-path ::
          ('f, 'f protocol-channel) state-monad) >>=
          (λap. assert (ap = f) >>= (λ-. return ?out))))))
  by (rule hash-relation-program-check-authentication-path-assert-bind
    [OF fibers])
    (use second in simp)
have first:
  hash-relation-program R b (?H + ?H)
  ((check-authentication-path len i xp xp-path ::
    ('f, 'f protocol-channel) state-monad) >>=
    (λap. assert (ap = f) >>=
      (λ-.
        (check-authentication-path len ?sidx xn xn-path ::
          ('f, 'f protocol-channel) state-monad) >>=
          (λap. assert (ap = f) >>= (λ-. return ?out))))))
  using first0 xp-path-len by simp
have raw:
  hash-relation-program R b (0 + (?H + ?H))
  ((assert (xp = x) ::
    (unit, 'f protocol-channel) state-monad) >>=
    (λ-.
      (check-authentication-path len i xp xp-path ::
        ('f, 'f protocol-channel) state-monad) >>=
        (λap. assert (ap = f) >>=
          (λ-.
            (check-authentication-path len ?sidx xn xn-path ::
              ('f, 'f protocol-channel) state-monad) >>=
              (λap. assert (ap = f) >>= (λ-. return ?out)))))))
  by (rule hash-relation-program-bind)
    (rule hash-relation-program-assert, rule first)
show ?thesis
  using raw unfolding fri-layer-opening-finish-def
  by (simp add: Let-def)
qed

```

lemma *hash-relation-program-fri-layer-opening-step*:
assumes *fibers*: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$
shows
hash-relation-program $R \ b$
 $(\text{Suc } (\text{floor-log } \text{len}) + \text{Suc } (\text{floor-log } \text{len}))$
 $(\text{fri-layer-opening-step } (b', f) \ (i, x, \text{len}, \text{pw}))$
proof –
let $?N = \text{Suc } (\text{floor-log } \text{len}) + \text{Suc } (\text{floor-log } \text{len})$
have *after-xn-path*:
hash-relation-program $R \ b \ ?N$
 $((\text{ntimes read } (\text{floor-log } \text{len}) ::$
 $(f \ \text{list}, f \ \text{protocol-channel}) \ \text{state-monad}) \gg=$
 $(\lambda xn\text{-path}.$
 $\text{fri-layer-opening-finish } b' \ f \ i \ x \ \text{len} \ \text{pw} \ \text{xp} \ \text{xp-path} \ \text{xn} \ \text{xn-path}))$
if *xp-path-len*: $\text{length } \text{xp-path} = \text{floor-log } \text{len}$
for $\text{xp} \ \text{xp-path} \ \text{xn}$
proof (rule *hash-relation-program-ntimes-read-bind*)
fix $\text{xn-path} :: f \ \text{list}$
assume *xn-path-len*: $\text{length } \text{xn-path} = \text{floor-log } \text{len}$
show *hash-relation-program* $R \ b \ ?N$
 $(\text{fri-layer-opening-finish } b' \ f \ i \ x \ \text{len} \ \text{pw} \ \text{xp} \ \text{xp-path} \ \text{xn} \ \text{xn-path})$
by (rule *hash-relation-program-fri-layer-opening-finish*
 $[OF \ \text{fibers} \ \text{xp-path-len} \ \text{xn-path-len}])$
qed
have *after-xn*:
hash-relation-program $R \ b \ ?N$
 $(\text{read } \gg=$
 $(\lambda xn.$
 $(\text{ntimes read } (\text{floor-log } \text{len}) ::$
 $(f \ \text{list}, f \ \text{protocol-channel}) \ \text{state-monad}) \gg=$
 $(\lambda xn\text{-path}.$
 $\text{fri-layer-opening-finish } b' \ f \ i \ x \ \text{len} \ \text{pw} \ \text{xp} \ \text{xp-path} \ \text{xn} \ \text{xn-path})))$
if *xp-path-len*: $\text{length } \text{xp-path} = \text{floor-log } \text{len}$
for $\text{xp} \ \text{xp-path}$
proof –
have *hash-relation-program* $R \ b \ (0 + ?N)$
 $(\text{read } \gg=$
 $(\lambda xn.$
 $(\text{ntimes read } (\text{floor-log } \text{len}) ::$
 $(f \ \text{list}, f \ \text{protocol-channel}) \ \text{state-monad}) \gg=$
 $(\lambda xn\text{-path}.$
 $\text{fri-layer-opening-finish } b' \ f \ i \ x \ \text{len} \ \text{pw} \ \text{xp} \ \text{xp-path} \ \text{xn} \ \text{xn-path})))$
by (rule *hash-relation-program-bind*)
 $(\text{rule } \text{hash-relation-program-read}, \text{rule } \text{after-xn-path}[OF \ \text{xp-path-len}])$
then show *?thesis* **by** *simp*
qed
have *after-xp-path*:
hash-relation-program $R \ b \ ?N$
 $((\text{ntimes read } (\text{floor-log } \text{len}) ::$


```

      ('f list, 'f protocol-channel) state-monad) >>=
    (λxp-path.
      read >>=
      (λxn.
        (ntimes read (floor-log len) ::
          ('f list, 'f protocol-channel) state-monad) >>=
          (λxn-path.
            fri-layer-opening-finish b' f i x len pw xp xp-path xn xn-path))))
  for xp
proof (rule hash-relation-program-ntimes-read-bind)
fix xp-path :: 'f list
assume xp-path-len: length xp-path = floor-log len
show hash-relation-program R b ?N
  (read >>=
    (λxn.
      (ntimes read (floor-log len) ::
        ('f list, 'f protocol-channel) state-monad) >>=
        (λxn-path.
          fri-layer-opening-finish b' f i x len pw xp xp-path xn xn-path))))
  by (rule after-xn[OF xp-path-len])
qed
have hash-relation-program R b (0 + ?N)
  (read >>=
    (λxp.
      (ntimes read (floor-log len) ::
        ('f list, 'f protocol-channel) state-monad) >>=
        (λxp-path.
          read >>=
          (λxn.
            (ntimes read (floor-log len) ::
              ('f list, 'f protocol-channel) state-monad) >>=
              (λxn-path.
                fri-layer-opening-finish b' f i x len pw xp xp-path xn xn-path))))))
    by (rule hash-relation-program-bind)
      (rule hash-relation-program-read, rule after-xp-path)
  then show ?thesis
  unfolding fri-layer-opening-step-unfold-finish by simp
qed

```

lemma *hash-relation-program-fri-layer-opening-steps-mfold-state:*

fixes *st* :: *nat* × *'f* × *nat* × *nat*

assumes *fibers*: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$

and *init*:

case st of (*i, x, len, pw*) $\Rightarrow$ *floor-log len* ≤ *L*

shows

hash-relation-program R b

(*length fl* * (*Suc L* + *Suc L*))

(*mfold st* (*map fri-layer-opening-step fl*) ::

(*nat* × *'f* × *nat* × *nat*, 'f protocol-channel) state-monad)

```

using init
proof (induction fl arbitrary: st)
  case Nil
  then show ?case
    by (simp add: hash-relation-program-zero[OF hash-map-preserving-return])
next
  case (Cons bf fl)
  obtain i x len pw where st-eq: st = (i, x, len, pw)
    by (cases st)
  obtain b' f where bf-eq: bf = (b', f)
    by (cases bf)
  let ?N = Suc L + Suc L
  have head-exact:
    hash-relation-program R b
      (Suc (floor-log len) + Suc (floor-log len))
      (fri-layer-opening-step (b', f) (i, x, len, pw))
    by (rule hash-relation-program-fri-layer-opening-step[OF fibers])
  have head-le:
    Suc (floor-log len) + Suc (floor-log len) ≤ ?N
    using Cons.prems unfolding st-eq by simp
  have head:
    hash-relation-program R b ?N
      (fri-layer-opening-step bf st ::
        (nat × 'f × nat × nat, 'f protocol-channel) state-monad)
    unfolding bf-eq st-eq
    by (rule hash-relation-program-mono[OF head-le head-exact])
  have tail:
    hash-relation-program R b (length fl * ?N)
      (mfold z (map fri-layer-opening-step fl) ::
        (nat × 'f × nat × nat, 'f protocol-channel) state-monad)
    if out:
      Some (z, t) ∈
        set-dist (execute (fri-layer-opening-step bf st) s)
    for z and s :: 'f protocol-channel and t :: 'f protocol-channel
  proof –
    from fri-layer-opening-step-outcome[OF out[unfolded bf-eq st-eq]]
    obtain xp xp-path xn xn-path x' where
      z-eq: z = (i mod (len div 2), x', len div 2, pw + pw)
      by blast
    have len-le: floor-log len ≤ L
      using Cons.prems unfolding st-eq by simp
    have next-len: floor-log (len div 2) ≤ L
      using floor-log-div2-le-self[of len] len-le by linarith
    have z-inv:
      case z of (i, x, len, pw) ⇒ floor-log len ≤ L
      unfolding z-eq using next-len by simp
    show ?thesis
      by (rule Cons.IH[OF z-inv])
  qed

```

have *hash-relation-program* R b ($?N + \text{length } fl * ?N$)
 ((*fri-layer-opening-step* bf st ::
 ($\text{nat} \times 'f \times \text{nat} \times \text{nat}, 'f \text{ protocol-channel}$) *state-monad*) $\gg$
 ($\lambda z. \text{mfold } z (\text{map } \text{fri-layer-opening-step } fl)$))
by (*rule hash-relation-program-bind-on-outcomes*[*OF head*]) (*rule tail*)
then show *?case*
by (*simp add: add.assoc add.commute add.left-commute*)
qed

lemma *hash-relation-program-receive-query-commits-mfold:*

assumes *fibers*: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$

and *init*: $\text{floor-log } \text{len} \leq L$

shows

hash-relation-program R b
 ($\text{length } fl * (\text{Suc } L + \text{Suc } L)$)
 ($\text{mfold } (i, x, \text{len}, pw) (\text{receive-query-commits } fl)$)

unfolding *receive-query-commits-eq-fri-layer-opening-steps*

proof –

have *st-inv*:

case (i, x, len, pw) *of*
 (i, x, len, pw) $\Rightarrow \text{floor-log } \text{len} \leq L$

using *init* **by** *simp*

show *hash-relation-program* R b ($\text{length } fl * (\text{Suc } L + \text{Suc } L)$)

($\text{mfold } (i, x, \text{len}, pw) (\text{map } \text{fri-layer-opening-step } fl) ::$
 ($\text{nat} \times 'f \times \text{nat} \times \text{nat}, 'f \text{ protocol-channel}$) *state-monad*)

by (*rule hash-relation-program-fri-layer-opening-steps-mfold-state*
 [*OF fibers st-inv*])

qed

lemma *hash-relation-program-check-decommit-on-query-exact:*

assumes *fibers*: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$

shows

hash-relation-program R b *verifier-query-decommit-hash-budget*
 ($\text{mmap } (\text{check-decommit-on-query } fr \text{ } idx)$)

proof –

have *exact*:

hash-relation-program R b
 ($\text{length } (\text{powers-scaled } idx) * \text{Suc } (\text{floor-log } (\text{scale} * \text{clength})))$
 ($\text{mmap } (\text{check-decommit-on-query } fr \text{ } idx)$)

by (*rule hash-relation-program-check-decommit-on-query*[*OF fibers*])

show *?thesis*

using *exact*

unfolding *verifier-query-decommit-hash-budget-def powers-scaled-def*

by (*simp add: mult.commute*)

qed

lemma *hash-relation-program-verifier-query-round-program-exact:*

assumes *fibers*: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$

shows

```

hash-relation-program R b
  (1 + verifier-query-decommit-hash-budget +
   (length f-fl + length fl) * verifier-fri-layer-hash-budget)
  (verifier-query-round-program fr f-fl f-final as fl final)
proof -
  let ?L = floor-log (clength * scale)
  let ?B = verifier-fri-layer-hash-budget
  have B-eq: ?B = Suc ?L + Suc ?L
  unfolding verifier-fri-layer-hash-budget-def by simp
  have after-comp:
    hash-relation-program R b (length fl * ?B + 0)
      ((mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final))))
  for idx fv
proof (rule hash-relation-program-bind)
  show hash-relation-program R b (length fl * ?B)
    (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat, 'f protocol-channel) state-monad)
    unfolding B-eq
    by (rule hash-relation-program-receive-query-commits-mfold)
      (rule fibers, simp)
  show ∧x. hash-relation-program R b 0
    ((case x of (i, x, len, pw) ⇒ assert (x = final)) ::
     (unit, 'f protocol-channel) state-monad)
    by (simp add: hash-relation-program-assert split: prod.splits)
qed
have after-trace:
  hash-relation-program R b
    (length f-fl * ?B + (0 + (length fl * ?B + 0)))
    ((mfold (idx, hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
      (λ(f-i, f-x, f-len, f-pow).
        assert (f-x = f-final) >>=
        (λ-. (mfold
          (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
          (receive-query-commits fl) ::
          (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
          (λ(i, x, len, pw). assert (x = final)))))))
  for idx fv
proof (rule hash-relation-program-bind)
  show hash-relation-program R b (length f-fl * ?B)
    (mfold (idx, hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::

```

```

    (nat × 'f × nat × nat, 'f protocol-channel) state-monad)
unfolding B-eq
by (rule hash-relation-program-receive-query-commits-mfold)
    (rule fibers, simp)
fix x :: nat × 'f × nat × nat
obtain f-i f-x f-len f-pow where x-eq:
    x = (f-i, f-x, f-len, f-pow)
by (cases x) auto
have case-budget:
    hash-relation-program R b (0 + (length fl * ?B + 0))
    ((assert (f-x = f-final) ::
      (unit, 'f protocol-channel) state-monad) >>=
      (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final))))))
by (rule hash-relation-program-bind)
    (rule hash-relation-program-assert, rule after-comp)
show hash-relation-program R b (0 + (length fl * ?B + 0))
    ((case x of (f-i, f-x, f-len, f-pow) ⇒
      assert (f-x = f-final) >>=
      (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final)))))) ::
      (unit, 'f protocol-channel) state-monad)
using case-budget
unfolding x-eq
by (simp split: prod.splits)
qed
have after-query:
    hash-relation-program R b
    (verifier-query-decommit-hash-budget +
      (length f-fl * ?B + (0 + (length fl * ?B + 0))))
    (mmap (check-decommit-on-query fr idx) >>=
      (λfv. (mfold (idx, hd fv, clength * scale, 1)
        (receive-query-commits f-fl) ::
        (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
        (λ(f-i, f-x, f-len, f-pow).
          assert (f-x = f-final) >>=
          (λ-. (mfold
            (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
            (receive-query-commits fl) ::
            (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
            (λ(i, x, len, pw). assert (x = final)))))))
for idx
by (rule hash-relation-program-bind)

```

```

(rule hash-relation-program-check-decommit-on-query-exact[OF fibers],
 rule after-trace)
have after-query-simple:
  hash-relation-program R b
    (verifier-query-decommit-hash-budget +
     (length f-fl * ?B + length fl * ?B))
    (mmap (check-decommit-on-query fr idx) >>=
     (λfv. (mfold (idx, hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
      (λ(f-i, f-x, f-len, f-pow).
       assert (f-x = f-final) >>=
       (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final)))))))
    for idx
    using after-query[of idx] by simp
have after-random:
  hash-relation-program R b
    (verifier-query-decommit-hash-budget +
     (length f-fl * ?B + length fl * ?B))
    ((let idx' = index (to-nat idx) in
     mmap (check-decommit-on-query fr idx') >>=
     (λfv. (mfold (idx', hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
      (λ(f-i, f-x, f-len, f-pow).
       assert (f-x = f-final) >>=
       (λ-. (mfold
        (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final)))))))
    for idx
    using after-query-simple[of index (to-nat idx)]
    by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
have hash-relation-program R b
  (1 + (verifier-query-decommit-hash-budget +
   (length f-fl * ?B + length fl * ?B)))
  (receive-query-index-challenge >>=
   (λidx.
    let idx' = index (to-nat idx) in
    mmap (check-decommit-on-query fr idx') >>=
    (λfv. (mfold (idx', hd fv, clength * scale, 1)
     (receive-query-commits f-fl) ::
     (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
     (λ(f-i, f-x, f-len, f-pow).
      (λ(f-i, f-x, f-len, f-pow).

```

```

    assert (f-x = f-final) >>=
    (λ-. (mfold
      (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
      (λ(i, x, len, pw). assert (x = final))))))
  by (rule hash-relation-program-bind)
  (rule hash-relation-program-receive-query-index-challenge[OF fibers],
    rule after-random)
then show ?thesis
  unfolding verifier-query-round-program-def
  by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
qed

```

lemma *hash-relation-program-verifier-query-round-program:*

```

  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  and len-f:  $\text{length } f\text{-fl} \leq \text{ceil-log } \text{clength}$ 
  and len-c:  $\text{length } fl \leq \text{ceil-log } (\text{maxDegree} + 1)$ 
  shows
    hash-relation-program  $R \ b$  verifier-query-round-hash-budget
      (verifier-query-round-program fr f-fl f-final as fl final)
  proof -
    let ?N = verifier-fri-layer-hash-budget
    have exact:
      hash-relation-program  $R \ b$ 
      (1 + verifier-query-decommit-hash-budget +
        (length f-fl + length fl) * ?N)
      (verifier-query-round-program fr f-fl f-final as fl final)
    by (rule hash-relation-program-verifier-query-round-program-exact
      [OF fibers])
    have le:
      1 + verifier-query-decommit-hash-budget +
        (length f-fl + length fl) * ?N
        ≤ verifier-query-round-hash-budget
    using len-f len-c
    unfolding verifier-query-round-hash-budget-def
    by simp
    show ?thesis
      by (rule hash-relation-program-mono[OF le exact])
  qed

```

lemma *wp-verifier-query-round-program-hash-relation-hit-bound:*

```

  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  and len-f:  $\text{length } f\text{-fl} \leq \text{ceil-log } \text{clength}$ 
  and len-c:  $\text{length } fl \leq \text{ceil-log } (\text{maxDegree} + 1)$ 
  shows
    wp-event
      (verifier-query-round-program fr f-fl f-final as fl final)
      (hash-relation-hit-event  $R \ s$ )  $s \leq$ 

```

$\text{hash-relation-budget-value } b \text{ verifier-query-round-hash-budget}$
proof –
have *program*:
 $\text{hash-relation-program } R \text{ } b \text{ verifier-query-round-hash-budget}$
 $(\text{verifier-query-round-program } fr \text{ } f\text{-}fl \text{ } f\text{-}final \text{ as } fl \text{ } final)$
by (rule *hash-relation-program-verifier-query-round-program*
 $[OF \text{ fibers } len\text{-}f \text{ } len\text{-}c]$)
then show *?thesis*
unfolding *hash-relation-program-def hash-relation-budget-def* **by** *blast*
qed

lemma *wp-verifier-query-round-program-merkle-root-or-child-relation-bound*:
assumes *finite-roots*: *finite roots*
and *len-f*: $\text{length } f\text{-}fl \leq \text{ceil-log } clength$
and *len-c*: $\text{length } fl \leq \text{ceil-log } (\text{maxDegree} + 1)$
shows
 wp-event
 $(\text{verifier-query-round-program } fr \text{ } f\text{-}fl \text{ } f\text{-}final \text{ as } fl \text{ } final)$
 $(\text{hash-relation-hit-event}$
 $(\text{merkle-root-or-child-output-relation roots}) \text{ } s) \text{ } s \leq$
 $\text{hash-relation-budget-value } (\text{card roots} + 2)$
 $\text{verifier-query-round-hash-budget}$
by (rule *wp-verifier-query-round-program-hash-relation-hit-bound*)
(rule *merkle-root-or-child-output-relation-fiber-card-bound*
 $[OF \text{ finite-roots}]$, rule *len-f*, rule *len-c*)

lemma *hash-relation-program-ntimes-verifier-query-round-program*:
assumes *fibers*: $\bigwedge x. \text{card } \{y. R \text{ } x \text{ } y\} \leq b$
and *len-f*: $\text{length } f\text{-}fl \leq \text{ceil-log } clength$
and *len-c*: $\text{length } fl \leq \text{ceil-log } (\text{maxDegree} + 1)$
shows
 $\text{hash-relation-program } R \text{ } b \text{ } (n * \text{verifier-query-round-hash-budget})$
 $(ntimes$
 $(\text{verifier-query-round-program } fr \text{ } f\text{-}fl \text{ } f\text{-}final \text{ as } fl \text{ } final) \text{ } n)$
proof (rule *hash-relation-program-ntimes*)
show $\bigwedge x.$
 $\text{hash-relation-program } R \text{ } b \text{ verifier-query-round-hash-budget}$
 $(\text{verifier-query-round-program } fr \text{ } f\text{-}fl \text{ } f\text{-}final \text{ as } fl \text{ } final)$
by (rule *hash-relation-program-verifier-query-round-program*
 $[OF \text{ fibers } len\text{-}f \text{ } len\text{-}c]$)
qed

lemma *wp-ntimes-verifier-query-round-program-hash-relation-hit-bound*:
assumes *fibers*: $\bigwedge x. \text{card } \{y. R \text{ } x \text{ } y\} \leq b$
and *len-f*: $\text{length } f\text{-}fl \leq \text{ceil-log } clength$
and *len-c*: $\text{length } fl \leq \text{ceil-log } (\text{maxDegree} + 1)$
shows
 wp-event
 $(ntimes$


```

      (verifier-query-round-program fr f-fl f-final as fl final) n)
    (hash-relation-hit-event R s) s ≤
    hash-relation-budget-value b
    (n * verifier-query-round-hash-budget)
  proof -
  have program:
    hash-relation-program R b (n * verifier-query-round-hash-budget)
    (ntimes
      (verifier-query-round-program fr f-fl f-final as fl final) n)
  by (rule hash-relation-program-ntimes-verifier-query-round-program
      [OF fibers len-f len-c])
  then show ?thesis
    unfolding hash-relation-program-def hash-relation-budget-def by blast
qed

```

lemma *wp-ntimes-verifier-query-round-program-merkle-root-or-child-relation-bound*:

assumes *finite-roots*: finite roots
and *len-f*: length f-fl ≤ ceil-log clength
and *len-c*: length fl ≤ ceil-log (maxDegree + 1)
shows
wp-event
 (ntimes
 (verifier-query-round-program fr f-fl f-final as fl final) n)
 (hash-relation-hit-event
 (merkle-root-or-child-output-relation roots) s) s ≤
 hash-relation-budget-value (card roots + 2)
 (n * verifier-query-round-hash-budget)
by (rule wp-ntimes-verifier-query-round-program-hash-relation-hit-bound)
 (rule merkle-root-or-child-output-relation-fiber-card-bound
 [OF finite-roots], rule len-f, rule len-c)

definition *checked-staged-security-with-actual-alpha-prefix-tree-output-case*
where
checked-staged-security-with-actual-alpha-prefix-tree-output-case P out
 $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (x, -) $\Rightarrow$
 (let packed = fst (fst x);
 prefix-state = snd (fst packed);
 data = snd packed;
 attacker-state = snd (fst x);
 s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in $\exists$ result final-state trace-table composition-table as
 query-idxs dg composition-fri-roots final rest trace-tree
 composition-tree.
 Some (result, final-state) $\in$

$set-dist \ (execute \ verify-monad \ s) \wedge$
 $accepted-with-bound-tables \ s \ (Some \ (result, \ final-state))$
 $trace-table \ composition-table \ as \ query-idxs \wedge$
 $verifier-header-transcript \ s$
 $(staged-trace-root \ data)$
 $(staged-trace-fri-roots \ data)$
 $(staged-trace-final \ data)$
 $as \ dg \ composition-fri-roots \ final \ rest \wedge$
 $created-tree \ trace-table \ trace-tree \ final-state \wedge$
 $created-tree \ composition-table \ composition-tree \ final-state \wedge$
 $P \ trace-tree \ composition-tree \ prefix-state \ final-state))$

definition *checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit*
where

$checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit =$
 $checked-staged-security-with-actual-alpha-prefix-tree-output-case$
 $(\lambda trace-tree \ composition-tree \ prefix-state \ final-state.$
 $hash-map-new-output-hit$
 $\{value \ trace-tree, \ value \ composition-tree\}$
 $prefix-state \ final-state)$

definition

checked-staged-security-with-actual-alpha-prefix-tree-subtree-output-hit
where

$checked-staged-security-with-actual-alpha-prefix-tree-subtree-output-hit =$
 $checked-staged-security-with-actual-alpha-prefix-tree-output-case$
 $(\lambda trace-tree \ composition-tree \ prefix-state \ final-state.$
 $(\exists l \ v \ r. \ trace-tree = \langle l, \ v, \ r \rangle \wedge$
 $hash-map-new-output-hit \ (set-tree \ l \cup \ set-tree \ r)$
 $prefix-state \ final-state) \vee$
 $(\exists l \ v \ r. \ composition-tree = \langle l, \ v, \ r \rangle \wedge$
 $hash-map-new-output-hit \ (set-tree \ l \cup \ set-tree \ r)$
 $prefix-state \ final-state))$

lemma *checked-staged-security-with-actual-alpha-prefix-tree-output-hit-imp-root-or-subtree:*
assumes *hit:*

$checked-staged-security-with-actual-alpha-prefix-tree-output-hit \ out$

shows

$checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit \ out \vee$
 $checked-staged-security-with-actual-alpha-prefix-tree-subtree-output-hit$
 out

proof (*cases out*)

case *None*

then show *?thesis*

using *hit*

unfolding

$checked-staged-security-with-actual-alpha-prefix-tree-output-hit-def$

by *simp*

next

```

case (Some result-pack)
then obtain x checked-final-state where out-eq:
  out = Some (x, checked-final-state)
  by (cases result-pack) simp
let ?packed = fst (fst x)
let ?prefix-state = snd (fst ?packed)
let ?data = snd ?packed
let ?attacker-state = snd (fst x)
let ?s =
  verifier-state-from-adversary ?attacker-state
  (staged-proof-transcript ?data)
from hit obtain result final-state trace-table composition-table as
  query-idxs dg composition-fri-roots final rest trace-tree
  composition-tree where
verify-out:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
and bound:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs
and header:
  verifier-header-transcript ?s
  (staged-trace-root ?data)
  (staged-trace-fri-roots ?data)
  (staged-trace-final ?data)
  as dg composition-fri-roots final rest
and trace-created:
  created-tree trace-table trace-tree final-state
and composition-created:
  created-tree composition-table composition-tree final-state
and tree-hit:
  hash-map-new-output-hit
  (set-tree trace-tree ∪ set-tree composition-tree)
  ?prefix-state final-state
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-tree-output-hit-def
  Let-def
apply simp
by blast
have split:
  hash-map-new-output-hit
  {value trace-tree, value composition-tree}
  ?prefix-state final-state ∨
  ( $\exists l\ v\ r. \text{trace-tree} = \langle l, v, r \rangle \wedge$ 
    hash-map-new-output-hit (set-tree l ∪ set-tree r)
    ?prefix-state final-state)  $\vee$ 
  ( $\exists l\ v\ r. \text{composition-tree} = \langle l, v, r \rangle \wedge$ 
    hash-map-new-output-hit (set-tree l ∪ set-tree r)
    ?prefix-state final-state)
by (rule hash-map-new-output-hit-two-trees-root-or-subtree

```

```

    [OF tree-hit])
  from split show ?thesis
proof
  assume root-hit:
    hash-map-new-output-hit
      {value trace-tree, value composition-tree}
      ?prefix-state final-state
  have checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit
    out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit-def
    checked-staged-security-with-actual-alpha-prefix-tree-output-case-def
    Let-def
  using verify-out bound header trace-created composition-created root-hit
  apply simp
  by blast
  then show ?thesis by simp
next
  assume subtree-hit:
    ( $\exists l \ v \ r. \text{trace-tree} = \langle l, v, r \rangle \wedge$ 
      hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
      ?prefix-state final-state)  $\vee$ 
    ( $\exists l \ v \ r. \text{composition-tree} = \langle l, v, r \rangle \wedge$ 
      hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
      ?prefix-state final-state)
  have
    checked-staged-security-with-actual-alpha-prefix-tree-subtree-output-hit
      out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-tree-subtree-output-hit-def
    checked-staged-security-with-actual-alpha-prefix-tree-output-case-def
    Let-def
  using verify-out bound header trace-created composition-created
    subtree-hit
  apply simp
  by blast
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-tree-output-hit-bound-from-root-and-subtree:
  assumes root-bound:
    wp-event m
    checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit s
     $\leq R$ 
  and subtree-bound:
    wp-event m
    checked-staged-security-with-actual-alpha-prefix-tree-subtree-output-hit s
     $\leq S$ 

```

shows
 $wp\text{-event } m \text{ checked-staged-security-with-actual-alpha-prefix-tree-output-hit}$
 $s \leq R + S$
proof –
have
 $wp\text{-event } m \text{ checked-staged-security-with-actual-alpha-prefix-tree-output-hit}$
 $s \leq$
 $wp\text{-event } m$
 $(\lambda out.$
 $\text{checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit}$
 $\text{out} \vee$
 $\text{checked-staged-security-with-actual-alpha-prefix-tree-subtree-output-hit}$
 $\text{out})$
 s
by (rule $wp\text{-event-mono}$)
(rule
 $\text{checked-staged-security-with-actual-alpha-prefix-tree-output-hit-imp-root-or-subtree}$)
also have $\dots \leq$
 $wp\text{-event } m$
 $\text{checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit } s +$
 $wp\text{-event } m$
 $\text{checked-staged-security-with-actual-alpha-prefix-tree-subtree-output-hit}$
 s
by (rule $wp\text{-event-union-bound}$)
also have $\dots \leq R + S$
by (rule $\text{add-mono}[OF \text{ root-bound subtree-bound}]$)
finally show $?thesis$.
qed

definition $\text{checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit}$
where

$\text{checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit}$
 $out \longleftrightarrow$
(case out of
 $None \Rightarrow False$
| $Some (x, -) \Rightarrow$
 $(let \text{ packed} = fst (fst x);$
 $\text{prefix-state} = snd (fst \text{ packed});$
 $\text{data} = snd \text{ packed};$
 $\text{attacker-state} = snd (fst x);$
 $s =$
 $\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})$
in $\exists \text{ result final-state trace-table composition-table as}$
 $\text{query-idxs dg composition-fri-roots final rest.}$
 $Some (\text{result}, \text{final-state}) \in$
 $\text{set-dist (execute verify-monad } s) \wedge$
 $\text{accepted-with-bound-tables } s (Some (\text{result}, \text{final-state}))$
 $\text{trace-table composition-table as query-idxs} \wedge$

```

    verifier-header-transcript s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      as dg composition-fri-roots final rest  $\wedge$ 
hash-map-new-output-hit
  {staged-trace-root data, hd composition-fri-roots}
  prefix-state final-state))

```

lemma *checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit-imp-transcript-root-output-hit:*
assumes *root-hit:*

checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit out

shows

checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit out

proof (*cases out*)

case *None*

then show *?thesis*

using *root-hit*

unfolding

checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit-def

checked-staged-security-with-actual-alpha-prefix-tree-output-case-def

by *simp*

next

case (*Some result-pack*)

then obtain *x checked-final-state* **where** *out-eq:*

out = Some (x, checked-final-state)

by (*cases result-pack*) *simp*

let *?packed = fst (fst x)*

let *?prefix-state = snd (fst ?packed)*

let *?data = snd ?packed*

let *?attacker-state = snd (fst x)*

let *?s =*

verifier-state-from-adversary ?attacker-state

(staged-proof-transcript ?data)

from *root-hit* **obtain** *result final-state trace-table composition-table as*

query-idxs dg composition-fri-roots final rest trace-tree

composition-tree **where**

verify-out:

Some (result, final-state) $\in$ set-dist (execute verify-monad ?s)

and *bound:*

accepted-with-bound-tables ?s (Some (result, final-state))

trace-table composition-table as query-idxs

and *header:*

verifier-header-transcript ?s

(staged-trace-root ?data)

(staged-trace-fri-roots ?data)

(staged-trace-final ?data)

as dg composition-fri-roots final rest

```

and trace-created:
  created-tree trace-table trace-tree final-state
and composition-created:
  created-tree composition-table composition-tree final-state
and tree-root-hit:
  hash-map-new-output-hit
  { value trace-tree, value composition-tree }
  ?prefix-state final-state
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit-def
  checked-staged-security-with-actual-alpha-prefix-tree-output-case-def
  Let-def
apply simp
by blast
from bound obtain fr' f-fri-roots' f-final' dg'
  composition-fri-roots' final' rest' where
  header':
    verifier-header-transcript ?s fr' f-fri-roots' f-final' as dg'
    composition-fri-roots' final' rest'
and trace-bind:
  merkle-root-binds-table fr' trace-table final-state
and composition-bind:
  merkle-root-binds-table (hd composition-fri-roots')
  composition-table final-state
unfolding accepted-with-bound-tables-def by auto
have header-eq:
  fr' = staged-trace-root ?data ∧
  f-fri-roots' = staged-trace-fri-roots ?data ∧
  f-final' = staged-trace-final ?data ∧
  dg' = dg ∧
  composition-fri-roots' = composition-fri-roots ∧
  final' = final ∧
  rest' = rest
using verifier-header-transcript-unique[OF header' header]
by simp
then have fr-eq: fr' = staged-trace-root ?data
and comp-roots-eq:
  composition-fri-roots' = composition-fri-roots
by simp-all
from trace-bind obtain trace-root-tree where
  trace-root-created:
    created-tree trace-table trace-root-tree final-state
and trace-root-eq: fr' = value trace-root-tree
unfolding merkle-root-binds-table-def by blast
have trace-value-eq:
  value trace-tree = staged-trace-root ?data
using created-tree-same-table-value-eq
  [OF trace-created trace-root-created]
  trace-root-eq fr-eq

```

```

    by simp
  from composition-bind obtain composition-root-tree where
    composition-root-created:
      created-tree composition-table composition-root-tree final-state
  and composition-root-eq:
    hd composition-fri-roots' = value composition-root-tree
  unfolding merkle-root-binds-table-def by blast
  have composition-value-eq:
    value composition-tree = hd composition-fri-roots
  using created-tree-same-table-value-eq
    [OF composition-created composition-root-created]
    composition-root-eq comp-roots-eq
  by simp
  have transcript-root-hit:
    hash-map-new-output-hit
      {staged-trace-root ?data, hd composition-fri-roots}
      ?prefix-state final-state
  using tree-root-hit trace-value-eq composition-value-eq by simp
  show ?thesis
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit-def
    Let-def
  using verify-out bound header transcript-root-hit
  apply simp
  by blast
qed

lemma checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit-bound-from-transcript-roots:
  assumes root-bound:
    wp-event m
    checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit
    s ≤ R
  shows
    wp-event m
    checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit s
    ≤ R
  by (rule order.trans[OF - root-bound])
  (rule wp-event-mono,
   rule
    checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit-imp-transcript-root-output-hit)

definition
  checked-staged-security-with-actual-alpha-prefix-transcript-root-preverifier-output-hit
  where
    checked-staged-security-with-actual-alpha-prefix-transcript-root-preverifier-output-hit
      out ↔
      (case out of
        None ⇒ False
      | Some (x, -) ⇒

```


s final-state))

lemma *checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit-imp-preverifier-or-verifier-on-support*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

out $\in$

set-dist

(*execute*

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

adversary-initial-state)

and *hit:*

checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit

out

shows

checked-staged-security-with-actual-alpha-prefix-transcript-root-preverifier-output-hit

out $\vee$

checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit

out

proof (*cases out*)

case *None*

then show *?thesis*

using *hit*

unfolding

checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit-def

by *simp*

next

case (*Some result-pack*)

then obtain *prefix prefix-state data attacker-state checked-result*

checked-final-state **where** *out-eq:*

out =

Some (((((*prefix, prefix-state*), *data*), *attacker-state*),

checked-result), *checked-final-state*)

by (*cases result-pack, auto split: prod.splits*)

let *?s* =

verifier-state-from-adversary attacker-state

(*staged-proof-transcript data*)

have *support-some:*

Some (((((*prefix, prefix-state*), *data*), *attacker-state*),

checked-result), *checked-final-state*) $\in$

set-dist

(*execute*

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

adversary-initial-state)

using *support* **unfolding** *out-eq* .

from *support-some* **obtain** *trans-out* **where** *trans-out:*

Some (((*prefix, prefix-state*), *data*), *attacker-state*) $\in$

```

    set-dist
      (execute (checked-staged-transcript-with-alpha-prefix-program A)
        adversary-initial-state)
  unfolding
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
from checked-staged-transcript-with-alpha-prefix-program-support
  [OF wf controlled trans-out]
have prefix-ext: prefix-state  $\leq$  attacker-state
  by blast
have prefix-s: prefix-state  $\leq$  ?s
  by (rule
    alpha-prefix-hash-extends-verifier-state-from-adversary-right
    [OF prefix-ext])
from hit obtain result final-state trace-table composition-table as
  query-idxs dg composition-fri-roots final rest where
  verify-out:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)
and bound:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs
and header:
  verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  as dg composition-fri-roots final rest
and root-hit:
  hash-map-new-output-hit
  {staged-trace-root data, hd composition-fri-roots}
  prefix-state final-state
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit-def
  Let-def
  by simp blast
have s-final: ?s  $\leq$  final-state
  by (rule verify-monad-hash-extends[OF verify-out])
from hash-map-new-output-hit-trans-decomp
  [OF prefix-s s-final root-hit]
show ?thesis
proof
  assume pre:
    hash-map-new-output-hit
    {staged-trace-root data, hd composition-fri-roots}
    prefix-state ?s
  have
    checked-staged-security-with-actual-alpha-prefix-transcript-root-preverifier-output-hit
    out
  unfolding out-eq

```

```

    checked-staged-security-with-actual-alpha-prefix-transcript-root-preverifier-output-hit-def
    Let-def
    using verify-out bound header pre
    by simp blast
    then show ?thesis by simp
next
assume verifier:
  hash-map-new-output-hit
  {staged-trace-root data, hd composition-fri-roots}
  ?s final-state
have
  checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit
  out
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit-def
  Let-def
  using verify-out bound header verifier
  by simp blast
then show ?thesis by simp
qed
qed

lemma verifier-header-transcript-initial-roots-subset-transcript:
  assumes header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and nonempty: composition-fri-roots  $\neq$  []
  shows {fr, hd composition-fri-roots}  $\subseteq$  set (PTranscript s)
  using header nonempty
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by (cases composition-fri-roots) auto

definition
  checked-staged-security-with-actual-alpha-prefix-verifier-transcript-output-hit
  where
    checked-staged-security-with-actual-alpha-prefix-verifier-transcript-output-hit
    out  $\longleftrightarrow$ 
    (case out of
      None  $\Rightarrow$  False
    | Some (x, -)  $\Rightarrow$ 
      (let packed = fst (fst x);
        data = snd packed;
        attacker-state = snd (fst x);
        s =
          verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)
        in  $\exists$  result final-state trace-table composition-table as
          query-idxs.
          Some (result, final-state)  $\in$ 

```

set-dist (execute verify-monad s) $\wedge$
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs $\wedge$
hash-map-new-output-hit (set (PTranscript s)) s final-state))

lemma *checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit-imp-verifier-transcript-outp*

assumes *hit:*

checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit
out

shows

checked-staged-security-with-actual-alpha-prefix-verifier-transcript-output-hit
out

proof (*cases out*)

case *None*

then show *?thesis*

using *hit*

unfolding

checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit-def

by *simp*

next

case (*Some result-pack*)

then obtain *x checked-final-state* **where** *out-eq:*

out = Some (x, checked-final-state)

by (*cases result-pack*) *simp*

let *?packed = fst (fst x)*

let *?data = snd ?packed*

let *?attacker-state = snd (fst x)*

let *?s =*

verifier-state-from-adversary ?attacker-state

(staged-proof-transcript ?data)

from *hit* **obtain** *result final-state trace-table composition-table as*

query-idxs dg composition-fri-roots final rest **where**

verify-out:

Some (result, final-state) $\in$ set-dist (execute verify-monad ?s)

and *bound:*

accepted-with-bound-tables ?s (Some (result, final-state))

trace-table composition-table as query-idxs

and *header:*

verifier-header-transcript ?s

(staged-trace-root ?data)

(staged-trace-fri-roots ?data)

(staged-trace-final ?data)

as dg composition-fri-roots final rest

and *root-hit:*

hash-map-new-output-hit

{staged-trace-root ?data, hd composition-fri-roots}

?s final-state

unfolding *out-eq*

checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit-def

```

    Let-def
  by simp blast
from bound obtain result' final-state' fr' f-fri-roots' f-final' dg'
  composition-fri-roots' final' rest' where
  out-eq':
    Some (result, final-state) = Some (result', final-state')
  and header':
    verifier-header-transcript ?s fr' f-fri-roots' f-final' as dg'
    composition-fri-roots' final' rest'
  and nonempty': composition-fri-roots' ≠ []
  unfolding accepted-with-bound-tables-def by blast
have header-eq:
  fr' = staged-trace-root ?data ∧
  f-fri-roots' = staged-trace-fri-roots ?data ∧
  f-final' = staged-trace-final ?data ∧
  dg' = dg ∧
  composition-fri-roots' = composition-fri-roots ∧
  final' = final ∧
  rest' = rest
  using verifier-header-transcript-unique[OF header' header]
  by simp
have nonempty: composition-fri-roots ≠ []
  using nonempty' header-eq by simp
have subset:
  {staged-trace-root ?data, hd composition-fri-roots}
  ⊆ set (PTranscript ?s)
  by (rule verifier-header-transcript-initial-roots-subset-transcript
    [OF header nonempty])
have transcript-hit:
  hash-map-new-output-hit (set (PTranscript ?s)) ?s final-state
  by (rule hash-map-new-output-hit-subset[OF subset root-hit])
show ?thesis
  unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-verifier-transcript-output-hit-def
  Let-def
  using verify-out bound transcript-hit
  by simp blast
qed

```

```

lemma checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit-bound-from-verifier-transcript
assumes transcript-bound:
  wp-event m
  checked-staged-security-with-actual-alpha-prefix-verifier-transcript-output-hit
  s ≤ C
shows
  wp-event m
  checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit
  s ≤ C
by (rule order.trans[OF - transcript-bound])

```

```

(rule wp-event-mono,
  rule
    checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit-imp-verifier-transcript-output-on-
lemma checked-staged-security-with-actual-alpha-prefix-tree-output-hit-imp-data-state-alpha-prefix-tree-output-hit-on-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
and hit: checked-staged-security-with-actual-alpha-prefix-tree-output-hit
  out
shows
  case out of
    None ⇒ False
  | Some (((((prefix, prefix-state), data), attacker-state),
    checked-result), checked-final-state) ⇒
    checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A
      (Some (((data, attacker-state), checked-result),
        checked-final-state)))
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding checked-staged-security-with-actual-alpha-prefix-tree-output-hit-def
  by simp
next
case (Some result-pack)
then obtain prefix prefix-state data attacker-state checked-result
  checked-final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state),
        checked-result), checked-final-state)
  by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from support[unfolded out-eq] obtain trans-out where trans-out:
  Some (((prefix, prefix-state), data), attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-with-alpha-prefix-program A)
        adversary-initial-state)
unfolding
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
by (auto elim!: set-dist-bindE split: prod.splits)

```

```

from checked-staged-transcript-with-alpha-prefix-program-support
  [OF wf controlled trans-out]
have prefix-out:
  Some (prefix, prefix-state) ∈
    set-dist
      (execute (staged-alpha-prefix-program A)
        adversary-initial-state)
and root-eq: staged-trace-root data = fst prefix
and trace-roots-eq:
  staged-trace-fri-roots data = fst (snd prefix)
and trace-bs-eq:
  staged-trace-fri-challenges data = fst (snd (snd prefix))
and trace-final-eq:
  staged-trace-final data = snd (snd (snd prefix))
by blast+
have actual-body:
   $\exists$  result final-state trace-table composition-table as query-idxs dg
    composition-fri-roots final rest trace-tree composition-tree.
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s) ∧
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs ∧
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
    as dg composition-fri-roots final rest ∧
    created-tree trace-table trace-tree final-state ∧
    created-tree composition-table composition-tree final-state ∧
    hash-map-new-output-hit
      (set-tree trace-tree ∪ set-tree composition-tree)
    prefix-state final-state
using hit
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-tree-output-hit-def
  Let-def
by simp
from actual-body obtain result final-state trace-table composition-table as
  query-idxs dg composition-fri-roots final rest trace-tree
  composition-tree where
verify-out:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
and bound:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs
and header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)

```



```

    as dg composition-fri-roots final rest
  and trace-created: created-tree trace-table trace-tree final-state
  and composition-created:
    created-tree composition-table composition-tree final-state
  and tree-hit:
    hash-map-new-output-hit
    (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
  by blast
  have data-hit:
    checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A
    (Some (((data, attacker-state), checked-result),
      checked-final-state))
  by (rule
    checked-staged-security-with-data-state-alpha-prefix-tree-output-hitI
    [OF prefix-out root-eq trace-roots-eq trace-bs-eq trace-final-eq
      verify-out bound header trace-created composition-created
      tree-hit])
  show ?thesis
    unfolding out-eq
    using data-hit by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-tree-output-hit-bound-from-data-state:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and data-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A)
    adversary-initial-state  $\leq T$ 
  shows
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit
    adversary-initial-state  $\leq T$ 
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?project =
     $\lambda x. ((snd (fst (fst x)), snd (fst x)), snd x)$ 
  let ?DataTree =
    checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A
  have wp-event ?M checked-staged-security-with-actual-alpha-prefix-tree-output-hit
    adversary-initial-state  $\leq$ 
  wp-event ?M
    ( $\lambda out. case out of$ 
      None  $\Rightarrow$  False
      | Some (x, t)  $\Rightarrow$  ?DataTree (Some (?project x, t)))
    adversary-initial-state

```

```

proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈ set-dist (execute ?M adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit out
  show
    (case out of None ⇒ False
     | Some (x, t) ⇒ ?DataTree (Some (?project x, t)))
  using
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit-imp-data-state-alpha-prefix-tree-output-hit-on-sup
    [OF wf controlled support hit]
  by (cases out) (auto split: prod.splits)
qed
also have ... =
  wp-event (?M ≫= (λx. return (?project x))) ?DataTree
  adversary-initial-state
unfolding
  checked-staged-security-with-data-state-alpha-prefix-tree-output-hit-def
by (subst wp-event-bind-return-map)
  (simp split: option.splits prod.splits)
also have ... =
  wp-event (checked-staged-security-experiment-with-data-state A)
  ?DataTree adversary-initial-state
using
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
  [of A]
by simp
also have ... ≤ T
  by (rule data-bound)
finally show ?thesis .
qed

```

Branch-specific tree-output events.

The actual-prefix tree-output events are existential over verifier witnesses for a fixed prefix. They are useful for deterministic reductions, but too coarse for direct verifier-local WP bounds. The following auxiliary events tie the witness to the current verifier branch of the checked experiment, so target-budget lemmas can be applied without changing the public theorem interface.

definition

```

checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
where
checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
  out ↔
  (case out of
   None ⇒ False
   | Some (x, final-state) ⇒
     (let packed = fst (fst x);

```

```

    data = snd packed;
    attacker-state = snd (fst x);
    result = snd x;
    s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  in  $\exists$  trace-table composition-table as query-idxs.
  Some (result, final-state)  $\in$ 
    set-dist (execute verify-monad s)  $\wedge$ 
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs  $\wedge$ 
    hash-map-new-output-hit (set (PTranscript s)) s final-state))

```

definition

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit

where

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit

out $\longleftrightarrow$

(case out of

None $\Rightarrow$ False

| Some (x, final-state) $\Rightarrow$

(let packed = fst (fst x);

data = snd packed;

attacker-state = snd (fst x);

result = snd x;

s =

verifier-state-from-adversary attacker-state

(staged-proof-transcript data)

in $\exists$ trace-table composition-table as query-idxs dg

composition-fri-roots final rest.

Some (result, final-state) $\in$

set-dist (execute verify-monad s) $\wedge$

accepted-with-bound-tables s (Some (result, final-state))

trace-table composition-table as query-idxs $\wedge$

verifier-header-transcript s

(staged-trace-root data)

(staged-trace-fri-roots data)

(staged-trace-final data)

as dg composition-fri-roots final rest $\wedge$

hash-map-new-output-hit

{staged-trace-root data, hd composition-fri-roots}

s final-state))

lemma *branch-verifier-transcript-output-hit-imp-verifier-transcript-output-hit:*

assumes *hit:*

checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit

out

shows

checked-staged-security-with-actual-alpha-prefix-verifier-transcript-output-hit

```

    out
using hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit-def
  checked-staged-security-with-actual-alpha-prefix-verifier-transcript-output-hit-def
  Let-def
by (cases out) (fastforce split: prod.splits)+

lemma branch-transcript-root-verifier-output-hit-imp-branch-verifier-transcript-output-hit:
assumes hit:
  checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
  out
shows
  checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
  out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-def
  by simp
next
case (Some result-pack)
then obtain x final-state where out-eq: out = Some (x, final-state)
  by (cases result-pack) simp
let ?packed = fst (fst x)
let ?data = snd ?packed
let ?attacker-state = snd (fst x)
let ?result = snd x
let ?s =
  verifier-state-from-adversary ?attacker-state
  (staged-proof-transcript ?data)
from hit obtain trace-table composition-table as query-idxs dg
  composition-fri-roots final rest where
  verify-out:
    Some (?result, final-state) ∈ set-dist (execute verify-monad ?s)
and bound:
  accepted-with-bound-tables ?s (Some (?result, final-state))
  trace-table composition-table as query-idxs
and header:
  verifier-header-transcript ?s
  (staged-trace-root ?data)
  (staged-trace-fri-roots ?data)
  (staged-trace-final ?data)
  as dg composition-fri-roots final rest
and root-hit:
  hash-map-new-output-hit
  {staged-trace-root ?data, hd composition-fri-roots}

```

```

    ?s final-state
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-def
  Let-def
  by simp blast
from bound obtain result' final-state' fr' f-fri-roots' f-final' dg'
  composition-fri-roots' final' rest' where
  out-eq':
    Some (?result, final-state) = Some (result', final-state')
  and header':
    verifier-header-transcript ?s fr' f-fri-roots' f-final' as dg'
    composition-fri-roots' final' rest'
  and nonempty': composition-fri-roots' ≠ []
  unfolding accepted-with-bound-tables-def by blast
have header-eq:
  fr' = staged-trace-root ?data ∧
  f-fri-roots' = staged-trace-fri-roots ?data ∧
  f-final' = staged-trace-final ?data ∧
  dg' = dg ∧
  composition-fri-roots' = composition-fri-roots ∧
  final' = final ∧
  rest' = rest
  using verifier-header-transcript-unique[OF header' header]
  by simp
have nonempty: composition-fri-roots ≠ []
  using nonempty' header-eq by simp
have subset:
  {staged-trace-root ?data, hd composition-fri-roots}
  ⊆ set (PTranscript ?s)
  by (rule verifier-header-transcript-initial-roots-subset-transcript
    [OF header nonempty])
have transcript-hit:
  hash-map-new-output-hit (set (PTranscript ?s)) ?s final-state
  by (rule hash-map-new-output-hit-subset[OF subset root-hit])
show ?thesis
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit-def
    Let-def
  using verify-out bound transcript-hit
  by simp blast
qed

```

lemma branch-transcript-root-verifier-output-hit-bound-from-branch-verifier-transcript-output:

assumes transcript-bound:

wp-event m

checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit

$s \leq C$

shows

wp-event m

```

    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
     $s \leq C$ 
  by (rule order.trans[OF - transcript-bound])
    (rule wp-event-mono,
     rule
       branch-transcript-root-verifier-output-hit-imp-branch-verifier-transcript-output-hit)

lemma actual-alpha-prefix-branch-verifier-transcript-output-cont-imp-hash-new-output-hit:
  fixes packed and attacker-state
  defines  $s \equiv$ 
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript (snd packed))
  assumes hit:
    (case out of
      None  $\Rightarrow$ 
        checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
        None
      | Some (result, t)  $\Rightarrow$ 
        checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
        (Some (((packed, attacker-state), result), t)))
  shows hash-new-output-hit-event (set (PTranscript s)) s out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit-def
    by simp
next
  case (Some result-state)
  then obtain result t where result-state: result-state = (result, t)
    by (cases result-state) simp
  have hash-map-new-output-hit (set (PTranscript s)) s t
    using hit Some result-state
    unfolding
      checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit-def
      s-def Let-def
    by (auto split: prod.splits)
  then show ?thesis
    using Some result-state
    unfolding hash-new-output-hit-event-def by simp
qed

lemma wp-verify-monad-bind-return-actual-alpha-prefix-branch-verifier-transcript-output-hit-bound:
  fixes packed and attacker-state
  defines  $s \equiv$ 
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript (snd packed))
  shows

```

```

wp-event
  (verify-monad  $\gg$ 
    ( $\lambda$ result. return ((packed, attacker-state), result)))
checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
s  $\leq$ 
hash-target-budget-value (set (staged-proof-transcript (snd packed)))
  verifier-hash-query-budget
proof –
have mono:
wp-event verify-monad
  ( $\lambda$ out. case out of
    None  $\Rightarrow$ 
      checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
        None
    | Some (x, t)  $\Rightarrow$ 
      checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
        (Some (((packed, attacker-state), x), t))) s  $\leq$ 
wp-event verify-monad
  (hash-new-output-hit-event (set (PTranscript s)) s) s
proof (rule wp-event-mono)
fix out
assume hit:
  (case out of
    None  $\Rightarrow$ 
      checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
        None
    | Some (x, t)  $\Rightarrow$ 
      checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
        (Some (((packed, attacker-state), x), t)))
show hash-new-output-hit-event (set (PTranscript s)) s out
using hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit-def
  hash-new-output-hit-event-def s-def Let-def
by (cases out) (auto split: prod.splits)
qed
have target:
wp-event verify-monad
  (hash-new-output-hit-event (set (PTranscript s)) s) s  $\leq$ 
  hash-target-budget-value (set (PTranscript s))
    verifier-hash-query-budget
by (rule wp-verify-monad-hash-new-output-hit-bound)
have cont-bound:
wp-event verify-monad
  ( $\lambda$ out. case out of
    None  $\Rightarrow$ 
      checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
        None
    | Some (x, t)  $\Rightarrow$ 

```

```

    checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
      (Some (((packed, attacker-state), x), t))) s ≤
    hash-target-budget-value (set (staged-proof-transcript (snd packed)))
      verifier-hash-query-budget
using order.trans[OF mono target]
unfolding s-def by simp
show ?thesis
apply (subst wp-event-bind-return-map)
using cont-bound
apply (simp split: option.splits prod.splits)
done
qed

lemma wp-verify-monad-bind-return-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-bound:
fixes packed and attacker-state
defines s ≡
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript (snd packed))
shows
  wp-event
    (verify-monad >>=
      (λresult. return ((packed, attacker-state), result)))
    checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-root-verifier-output-hit
    s ≤
    hash-target-budget-value (set (staged-proof-transcript (snd packed)))
      verifier-hash-query-budget
proof –
have transcript-bound:
  wp-event
    (verify-monad >>=
      (λresult. return ((packed, attacker-state), result)))
    checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
    s ≤
    hash-target-budget-value (set (staged-proof-transcript (snd packed)))
      verifier-hash-query-budget
unfolding s-def
by (rule
  wp-verify-monad-bind-return-actual-alpha-prefix-branch-verifier-transcript-output-hit-bound)
show ?thesis
by (rule
  branch-transcript-root-verifier-output-hit-bound-from-branch-verifier-transcript-output
  [OF transcript-bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-bound-from-transcript-root-verifier-output-hit-bound:
assumes target-bound:
  ∧packed attacker-state.
  Some (packed, attacker-state) ∈
    set-dist

```



```

      (execute (checked-staged-transcript-with-alpha-prefix-program A)
        adversary-initial-state)  $\implies$ 
      hash-target-budget-value (set (staged-proof-transcript (snd packed)))
        verifier-hash-query-budget  $\leq C$ 
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
    adversary-initial-state  $\leq C$ 
unfolding checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
proof (rule wp-event-bind-bound-by-cont)
show
   $\neg$ 
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
    None
unfolding
  checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-def
by simp
next
fix packed attacker-state
assume support:
  Some (packed, attacker-state)  $\in$ 
    set-dist
      (execute (checked-staged-transcript-with-alpha-prefix-program A)
        adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript (snd packed))
have cont-eq:
  wp-event
    (get  $\gg=$ 
      ( $\lambda s$ . put
        (verifier-state-from-adversary s
          (staged-proof-transcript (snd packed)))  $\gg=$ 
        ( $\lambda$ -. verify-monad  $\gg=$ 
          ( $\lambda$ result. return ((packed, s), result)))))
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
    attacker-state =
  wp-event
    (verify-monad  $\gg=$ 
      ( $\lambda$ result. return ((packed, attacker-state), result)))
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
    ?s
unfolding wp-event-def by (simp add: wpsimps)
have verifier-bound:
  wp-event
    (verify-monad  $\gg=$ 
      ( $\lambda$ result. return ((packed, attacker-state), result)))
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit

```

```

    ?s ≤
    hash-target-budget-value (set (staged-proof-transcript (snd packed)))
    verifier-hash-query-budget
  by (rule
    wp-verify-monad-bind-return-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-bound)
  show
    wp-event
    (get ≥
      (λs. put
        (verifier-state-from-adversary s
          (staged-proof-transcript (snd packed))) ≥
        (λ-. verify-monad ≥
          (λresult. return ((packed, s), result))))))
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
    attacker-state ≤ C
  unfolding cont-eq
  by (rule order.trans[OF verifier-bound target-bound[OF support]])
qed

lemma checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-bound-from-concrete
  assumes target-bound:
    ∧packed attacker-state.
    Some (packed, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-with-alpha-prefix-program A)
          adversary-initial-state) ⇒
    concrete-transcript-target-error (staged-proof-transcript (snd packed))
      ≤ C
  shows
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
    adversary-initial-state ≤ C
  proof (rule
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-bound-from-transcript-target)
  fix packed attacker-state
  assume support:
    Some (packed, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-with-alpha-prefix-program A)
          adversary-initial-state)
  show
    hash-target-budget-value (set (staged-proof-transcript (snd packed)))
    verifier-hash-query-budget ≤ C
  using target-bound[OF support]
  unfolding concrete-transcript-target-error-def hash-target-budget-value-def
  by (simp add: mult.commute)
qed

```

end

end

theory *Staged-Security-Experiment-Composition-Bounds*
 imports *Staged-Security-Experiment-Composition-Tree-Output*
begin

Budget-level composition corollaries for the staged security experiment.

This layer consumes the actual-alpha-prefix prequery accounting theorem and exposes the remaining composition side condition as the tree-output event only. The tree-output event is still Merkle/candidate proof work; the alpha-key prequery accounting itself is derived from the current staged oracle-budget interface.

context *soundness*
begin

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-tree-and-budgets:*

assumes *wf: staged-budget-wellformed budgets*
 and *controlled: staged-adversary-controlled budgets A*
 and *tree-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-tree-output-hit
 adversary-initial-state $\leq T$

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event composition-bad*)
 adversary-initial-state $\leq$
 composition-error-bound +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 T

proof –

let *?P* =
 composition-error-bound +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*)

have *transcript-bound:*

wp-event (*checked-staged-transcript-with-alpha-prefix-program A*)
 (*checked-staged-transcript-actual-alpha-prefix-bad-set-hit*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq ?P$

by (*rule*)

```

      checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
      [OF wf controlled])
have prefix-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq$  ?P
  by (rule
      checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
      [OF transcript-bound])
have
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state  $\leq$  ?P + T
  by (rule
      checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-and-tree
      [OF wf controlled prefix-bound tree-bound])
then show ?thesis
  by (simp add: add.assoc)
qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-data-alpha-prefix-tree-and-budgets:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and data-tree-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A)
    adversary-initial-state  $\leq$  T
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state  $\leq$ 
      composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      T
proof –
have actual-tree-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit
    adversary-initial-state  $\leq$  T
  by (rule
      checked-staged-security-with-actual-alpha-prefix-tree-output-hit-bound-from-data-state
      [OF wf controlled data-tree-bound])

```

```

show ?thesis
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-tree-and-budgets
    [OF wf controlled actual-tree-bound])
qed

end

end

```

```

theory Staged-Security-Experiment-Composition-Branch
  imports Staged-Security-Experiment-Composition-Bounds
begin

```

Branch-specific composition reductions.

This layer introduces branch-tied events for the verifier result actually sampled by the checked experiment. These events are suitable for verifier-local WP bounds and avoid broad existential side events.

```

context soundness
begin

```

```

definition staged-proof-data-of-query-prefix
  :: 'f staged-query-prefix-data  $\Rightarrow$  'f staged-proof-data
  where
    staged-proof-data-of-query-prefix prefix =
      (|staged-trace-root = sqp-trace-root prefix,
        staged-trace-fri-roots = sqp-trace-fri-roots prefix,
        staged-trace-fri-challenges = sqp-trace-fri-challenges prefix,
        staged-trace-final = sqp-trace-final prefix,
        staged-alphas = sqp-alphas prefix,
        staged-degree = sqp-degree prefix,
        staged-composition-fri-roots = sqp-composition-fri-roots prefix,
        staged-composition-fri-challenges =
          sqp-composition-fri-challenges prefix,
        staged-composition-final = sqp-composition-final prefix,
        staged-query-chunks = sqp-query-chunks prefix|)

```

```

lemma checked-staged-transcript-program-query-prefix-map:
  checked-staged-transcript-program A =
    checked-staged-query-challenge-prefix-program A rounds  $\gg$ 
      ( $\lambda$ prefix. return (staged-proof-data-of-query-prefix prefix))
unfolding checked-staged-transcript-program-def
  checked-staged-query-challenge-prefix-program-def
  staged-alpha-prefix-program-def
  staged-proof-data-of-query-prefix-def Let-def
by (simp add: sm-bind-assoc split-def split: prod.splits)

```

```

lemma staged-query-search-queries-rounds-eq-total:

```

```

assumes wf: staged-budget-wellformed budgets
shows
  staged-query-search-queries budgets rounds =
    staged-attacker-query-budget budgets + staged-challenge-query-budget
using wf
unfolding staged-query-search-queries-def
  staged-attacker-query-budget-def staged-challenge-query-budget-def
  staged-budget-wellformed-def
by simp

lemma hash-relation-program-checked-staged-transcript-program:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
shows
  hash-relation-program R b
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (checked-staged-transcript-program A)
proof –
have prefix:
  hash-relation-program R b
    (staged-query-search-queries budgets rounds)
    (checked-staged-query-challenge-prefix-program A rounds)
by (rule hash-relation-program-checked-staged-query-challenge-prefix-program
    [OF wf controlled - fibers])
  simp
have mapped:
  hash-relation-program R b
    (staged-query-search-queries budgets rounds + 0)
    (checked-staged-query-challenge-prefix-program A rounds  $\gg=$ 
      ( $\lambda$ prefix. return (staged-proof-data-of-query-prefix prefix)))
by (rule hash-relation-program-bind)
    (rule prefix,
      simp add: hash-relation-program-zero[OF hash-map-preserving-return])
have budget-eq:
  staged-query-search-queries budgets rounds =
    staged-attacker-query-budget budgets + staged-challenge-query-budget
by (rule staged-query-search-queries-rounds-eq-total[OF wf])
show ?thesis
unfolding checked-staged-transcript-program-query-prefix-map
using mapped budget-eq by simp
qed

```

```

definition checked-staged-transcript-initial-roots-relation
  :: 'f staged-adversary  $\Rightarrow$  'f protocol-hash-input  $\Rightarrow$  'f  $\Rightarrow$  bool
where
  checked-staged-transcript-initial-roots-relation A x y  $\longleftrightarrow$ 
    ( $\exists$  data attacker-state.
      Some (data, attacker-state)  $\in$ 

```

set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\wedge$
 $y \in \{\text{staged-trace-root data},$
 hd (staged-composition-fri-roots data))}

lemma checked-staged-transcript-initial-roots-relation-fiber-bound-size:
 card {y. checked-staged-transcript-initial-roots-relation A x y}
 $\leq$ size

proof –
 have subset:
 {y. checked-staged-transcript-initial-roots-relation A x y}
 $\subseteq$ (UNIV :: 'f set)
 by simp
 have finite-univ: finite (UNIV :: 'f set)
 by simp
 have card {y. checked-staged-transcript-initial-roots-relation A x y}
 $\leq$ card (UNIV :: 'f set)
 by (rule card-mono[OF finite-univ subset])
 also have ... = size
 using size-card by simp
 finally show ?thesis .
qed

lemma checked-staged-transcript-initial-roots-relation-hit-bound:
 assumes wf: staged-budget-wellformed budgets
 and controlled: staged-adversary-controlled budgets A
 shows
 wp-event (checked-staged-transcript-program A)
 (hash-relation-hit-event
 (checked-staged-transcript-initial-roots-relation A)
 adversary-initial-state)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-attacker-query-budget budgets + staged-challenge-query-budget)

proof –
 have program:
 hash-relation-program
 (checked-staged-transcript-initial-roots-relation A)
 size
 (staged-attacker-query-budget budgets + staged-challenge-query-budget)
 (checked-staged-transcript-program A)
 by (rule hash-relation-program-checked-staged-transcript-program
 [OF wf controlled])
 (rule checked-staged-transcript-initial-roots-relation-fiber-bound-size)
 show ?thesis
 using program
 unfolding hash-relation-program-def hash-relation-budget-def
 staged-phase-relation-error-def

by blast
qed

lemma *checked-staged-transcript-with-alpha-prefix-initial-roots-relation-hit-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event (checked-staged-transcript-with-alpha-prefix-program A)
 $(\lambda out. \text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some (packed, attacker-state)} \Rightarrow$
 $\quad \quad \text{hash-relation-hit}$
 $\quad \quad (\text{checked-staged-transcript-initial-roots-relation } A)$
 $\quad \quad \text{adversary-initial-state attacker-state})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget})$

proof –

let *?R = checked-staged-transcript-initial-roots-relation A*

let *?H =*

$\lambda out. \text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some (packed, attacker-state)} \Rightarrow$
 $\quad \quad \text{hash-relation-hit ?R adversary-initial-state attacker-state}$

have *map-eq:*

wp-event (checked-staged-transcript-with-alpha-prefix-program A) ?H
 $\text{adversary-initial-state} =$
 wp-event
 $(\text{checked-staged-transcript-with-alpha-prefix-program } A \gg=$
 $\quad (\lambda \text{packed. return (snd packed)}))$
 $(\text{hash-relation-hit-event ?R adversary-initial-state})$
 $\text{adversary-initial-state}$

by *(subst wp-event-bind-return-map)*

(simp add: hash-relation-hit-event-def split: option.splits prod.splits)

also have *... =*

wp-event (checked-staged-transcript-program A)
 $(\text{hash-relation-hit-event ?R adversary-initial-state})$
 $\text{adversary-initial-state}$

unfolding *checked-staged-transcript-with-alpha-prefix-program-projection*

by *simp*

also have *... ≤*

$\text{staged-phase-relation-error size}$
 $(\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget})$

by *(rule checked-staged-transcript-initial-roots-relation-hit-bound*
 $[OF \text{ wf controlled}]$ *)*

finally show *?thesis .*

qed

definition *staged-proof-transcript-length-bound :: nat*

where

staged-proof-transcript-length-bound =
 $4 + \text{ceil-log } \text{clength} + \text{length spec} + \text{ceil-log } (\text{maxDegree} + 1) +$
 $\text{rounds} * (\text{query-decommitment-transcript-length } 0 +$
 $\text{fri-layers-transcript-length } (\text{ceil-log } \text{clength})$
 $(\text{clength} * \text{scale}) +$
 $\text{fri-layers-transcript-length } (\text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{clength} * \text{scale}))$

definition *staged-concrete-transcript-target-error-bound* :: *prob*

where

staged-concrete-transcript-target-error-bound =
 nnreal
 $(\text{staged-proof-transcript-length-bound} * \text{verifier-hash-query-budget}) /$
 nnreal size

lemma *fri-layers-transcript-length-mono-layers*:

assumes $m \leq n$

shows *fri-layers-transcript-length* $m \text{ len} \leq$
fri-layers-transcript-length $n \text{ len}$

using *assms*

proof (*induction* m *arbitrary*: $n \text{ len}$)

case 0

then show ?*case* **by** *simp*

next

case (*Suc* m)

then obtain n' **where** $n\text{-eq}$: $n = \text{Suc } n'$

by (*cases* n) *auto*

have $m\text{-le}$: $m \leq n'$

using *Suc.prem*s $n\text{-eq}$ **by** *simp*

have *rec*:

fri-layers-transcript-length $m (\text{len div } 2) \leq$

fri-layers-transcript-length $n' (\text{len div } 2)$

by (*rule* *Suc.IH*[*OF* $m\text{-le}$])

show ?*case*

unfolding $n\text{-eq}$ **by** *simp* (*rule* *rec*)

qed

lemma *length-concat-eq-sum-list-map-length*:

length (*List.concat* xss) = *sum-list* (*map* *length* xss)

by (*induction* xss) *simp-all*

lemma *sum-list-map-const*:

$(\sum x \leftarrow xs. c) = \text{length } xs * c$

by (*induction* xs) *simp-all*

lemma *staged-query-chunks-total-length-le*:

assumes *match*: *staged-query-chunks-match-verifier-lengths* $data \text{ query-idxs}$

```

and trace-len:
  length (staged-trace-fri-roots data) = ceil-log clength
and composition-len:
  length (staged-composition-fri-roots data) ≤
    ceil-log (maxDegree + 1)
shows
  length (List.concat (staged-query-chunks data)) ≤
    rounds *
      (query-decommitment-transcript-length 0 +
        fri-layers-transcript-length (ceil-log clength)
          (clength * scale) +
        fri-layers-transcript-length (ceil-log (maxDegree + 1))
          (clength * scale))
proof -
  let ?Q =
    query-decommitment-transcript-length 0 +
    fri-layers-transcript-length (ceil-log clength) (clength * scale) +
    fri-layers-transcript-length (ceil-log (maxDegree + 1))
      (clength * scale)
  have chunks-len: length (staged-query-chunks data) = rounds
    using match unfolding staged-query-chunks-match-verifier-lengths-def
    by simp
  have chunk-bound:
     $\bigwedge \text{chunk}. \text{chunk} \in \text{set } (\text{staged-query-chunks data}) \implies$ 
      length chunk ≤ ?Q
  proof -
    fix chunk
    assume chunk-in: chunk ∈ set (staged-query-chunks data)
    then obtain i where i-bound: i < rounds
      and chunk-eq: chunk = staged-query-chunks data ! i
      using chunks-len
      by (metis in-set-conv-nth)
    have chunk-len:
      length chunk =
        verifier-query-round-transcript-length (query-idxs ! i)
          (staged-trace-fri-roots data)
          (staged-composition-fri-roots data)
      using match i-bound chunk-eq
      unfolding staged-query-chunks-match-verifier-lengths-def by simp
    have comp-mono:
      fri-layers-transcript-length
        (length (staged-composition-fri-roots data))
        (clength * scale)
      ≤
        fri-layers-transcript-length (ceil-log (maxDegree + 1))
          (clength * scale)
      by (rule fri-layers-transcript-length-mono-layers[OF composition-len])
    show length chunk ≤ ?Q
      unfolding chunk-len verifier-query-round-transcript-length-def

```

```

    trace-len query-decommitment-transcript-length-def powers-scaled-def
  using comp-mono by simp
qed
have sum-bound:
  ( $\sum \text{chunk} \leftarrow \text{staged-query-chunks data. length chunk} \leq$ 
   ( $\sum \text{chunk} \leftarrow \text{staged-query-chunks data. ?Q}$ )
  by (rule sum-list-mono) (rule chunk-bound)
have concat-len:
  length (List.concat (staged-query-chunks data)) =
    ( $\sum \text{chunk} \leftarrow \text{staged-query-chunks data. length chunk}$ )
  by (rule length-concat-eq-sum-list-map-length)
also have ...  $\leq$  ( $\sum \text{chunk} \leftarrow \text{staged-query-chunks data. ?Q}$ )
  by (rule sum-bound)
also have ... = length (staged-query-chunks data) * ?Q
  by (rule sum-list-map-const)
also have ... = rounds * ?Q
  using chunks-len by simp
finally show ?thesis .
qed

lemma checked-staged-transcript-program-staged-proof-transcript-length-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state)  $\in$ 
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  shows length (staged-proof-transcript data)  $\leq$ 
    staged-proof-transcript-length-bound
proof -
  have shape:
    length (staged-trace-fri-roots data) = ceil-log clength  $\wedge$ 
    length (staged-alphas data) = length spec  $\wedge$ 
    length (staged-composition-fri-roots data) =
      ceil-log (to-nat (staged-degree data) + 1)  $\wedge$ 
    ceil-log (to-nat (staged-degree data) + 1)  $\leq$ 
      ceil-log (maxDegree + 1)
  using checked-staged-transcript-program-outcome-shape
    [OF wf controlled outcome]
  by blast
from checked-staged-transcript-program-query-chunks-match-verifier-lengths
  [OF outcome]
obtain query-idxs where match:
  staged-query-chunks-match-verifier-lengths data query-idxs
  by blast
have query-len:
  length (List.concat (staged-query-chunks data))  $\leq$ 
    rounds *

```

```

    (query-decommitment-transcript-length 0 +
     fri-layers-transcript-length (ceil-log clength)
     (clength * scale) +
     fri-layers-transcript-length (ceil-log (maxDegree + 1))
     (clength * scale))
  by (rule staged-query-chunks-total-length-le[OF match])
  (use shape in simp-all)
have header-len:
  length
    (verifier-header-messages
     (staged-trace-root data)
     (staged-trace-fri-roots data)
     (staged-trace-final data)
     (staged-alphas data)
     (staged-degree data)
     (staged-composition-fri-roots data)
     (staged-composition-final data))
  ≤ 4 + ceil-log clength + length spec + ceil-log (maxDegree + 1)
using shape unfolding verifier-header-messages-def by simp
show ?thesis
  unfolding staged-proof-transcript-def
    staged-proof-transcript-length-bound-def
  using header-len query-len by simp
qed

```

lemma concrete-transcript-target-error-bound-from-length:

assumes length tr ≤ n
shows concrete-transcript-target-error tr ≤
 nnreal (n * verifier-hash-query-budget) / nnreal size

proof –

have card (set tr) ≤ length tr
by (rule card-length)
then have card-le: card (set tr) ≤ n
using assms **by** linarith
have mult-le:
 card (set tr) * verifier-hash-query-budget ≤
 n * verifier-hash-query-budget
by (rule mult-right-mono[OF card-le]) simp
show ?thesis
 unfolding concrete-transcript-target-error-def
by (rule nnreal-nat-divide-right-mono[OF mult-le])

qed

lemma checked-staged-transcript-with-alpha-prefix-program-data-outcome:

assumes outcome:
 Some (packed, attacker-state) ∈
 set-dist
 (execute (checked-staged-transcript-with-alpha-prefix-program A)
 adversary-initial-state)

shows

Some (snd packed, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)

using *outcome*

unfolding *checked-staged-transcript-with-alpha-prefix-program-projection*
[symmetric]

by (rule-tac *x=packed and t=attacker-state in set-dist-bindI*) *simp-all*

lemma *checked-staged-transcript-with-alpha-prefix-program-concrete-transcript-target-error-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *outcome:*

Some (packed, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-with-alpha-prefix-program A)
adversary-initial-state)

shows

concrete-transcript-target-error (staged-proof-transcript (snd packed))
 $\leq$ *staged-concrete-transcript-target-error-bound*

proof –

have *data-out:*

Some (snd packed, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)

by (rule *checked-staged-transcript-with-alpha-prefix-program-data-outcome*
[OF outcome])

have *len:*

length (staged-proof-transcript (snd packed)) ≤
staged-proof-transcript-length-bound
by (rule *checked-staged-transcript-program-staged-proof-transcript-length-bound*
[OF wf controlled data-out])

show *?thesis*

unfolding *staged-concrete-transcript-target-error-bound-def*

by (rule *concrete-transcript-target-error-bound-from-length* *[OF len]*)

qed

definition *checked-staged-security-with-actual-alpha-prefix-verifier-event*

$:: ('f \text{ protocol-channel} \Rightarrow$
 $(\text{unit list} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool})$
 $\Rightarrow$
 $(((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-security-with-actual-alpha-prefix-verifier-event P out

$\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (x, final-state) $\Rightarrow$
 (let packed = fst (fst x);
 data = snd packed;
 attacker-state = snd (fst x);
 result = snd x;
 s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in P s (Some (result, final-state))))

definition checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit

:: ((((((f $\times$ 'f list $\times$ 'f list $\times$ 'f) $\times$
 'f protocol-channel) $\times$ 'f staged-proof-data) $\times$
 'f protocol-channel) $\times$ unit list) $\times$
 'f protocol-channel) option $\Rightarrow$ bool

where

checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
 out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (x, final-state) $\Rightarrow$
 (let packed = fst (fst x);
 prefix-state = snd (fst packed);
 data = snd packed;
 attacker-state = snd (fst x);
 result = snd x;
 s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in $\exists$ trace-table composition-table as query-idxs dg
 composition-fri-roots final rest trace-tree composition-tree.
 Some (result, final-state) $\in$
 set-dist (execute verify-monad s) $\wedge$
 accepted-with-bound-tables s (Some (result, final-state))
 trace-table composition-table as query-idxs $\wedge$
 verifier-header-transcript s
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 as dg composition-fri-roots final rest $\wedge$
 created-tree trace-table trace-tree final-state $\wedge$
 created-tree composition-table composition-tree final-state $\wedge$
 hash-map-new-output-hit
 (set-tree trace-tree $\cup$ set-tree composition-tree)
 prefix-state final-state))

definition *checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case*

where

checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case
 $P \text{ out} \longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (x, final-state) $\Rightarrow$
 (let packed = fst (fst x);
 prefix-state = snd (fst packed);
 data = snd packed;
 attacker-state = snd (fst x);
 result = snd x;
 s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in $\exists$ trace-table composition-table as query-idxs dg
 composition-fri-roots final rest trace-tree composition-tree.
 Some (result, final-state) $\in$
 set-dist (execute verify-monad s) $\wedge$
 accepted-with-bound-tables s (Some (result, final-state))
 trace-table composition-table as query-idxs $\wedge$
 verifier-header-transcript s
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 as dg composition-fri-roots final rest $\wedge$
 created-tree trace-table trace-tree final-state $\wedge$
 created-tree composition-table composition-tree final-state $\wedge$
 P trace-tree composition-tree prefix-state final-state))

definition *checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit*

where

checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit =
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case
 (λ trace-tree composition-tree prefix-state final-state.
 hash-map-new-output-hit
 {value trace-tree, value composition-tree}
 prefix-state final-state)

definition

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit

where

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit =
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case
 (λ trace-tree composition-tree prefix-state final-state.
 ($\exists l v r$. trace-tree = $\langle l, v, r \rangle \wedge$
 hash-map-new-output-hit (set-tree l $\cup$ set-tree r)
 prefix-state final-state) $\vee$
 ($\exists l v r$. composition-tree = $\langle l, v, r \rangle \wedge$

hash-map-new-output-hit (*set-tree* *l* $\cup$ *set-tree* *r*)
prefix-state *final-state*))

definition

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit

where

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
out $\longleftrightarrow$
(case *out* of
 None $\Rightarrow$ *False*
| *Some* (*x*, *final-state*) $\Rightarrow$
 (let *packed* = *fst* (*fst* *x*);
 prefix-state = *snd* (*fst* *packed*);
 data = *snd* *packed*;
 attacker-state = *snd* (*fst* *x*);
 result = *snd* *x*;
 s =
 verifier-state-from-adversary *attacker-state*
 (*staged-proof-transcript* *data*)
 in $\exists$ *trace-table* *composition-table* as *query-idxs* *dg*
 composition-fri-roots *final* *rest* *trace-tree* *composition-tree*.
 Some (*result*, *final-state*) $\in$
 set-dist (*execute* *verify-monad* *s*) $\wedge$
 accepted-with-bound-tables *s* (*Some* (*result*, *final-state*))
 trace-table *composition-table* as *query-idxs* $\wedge$
 verifier-header-transcript *s*
 (*staged-trace-root* *data*)
 (*staged-trace-fri-roots* *data*)
 (*staged-trace-final* *data*)
 as *dg* *composition-fri-roots* *final* *rest* $\wedge$
 created-tree *trace-table* *trace-tree* *final-state* $\wedge$
 created-tree *composition-table* *composition-tree* *final-state* $\wedge$
 ($(\exists l\ v\ r. \text{trace-tree} = \langle l, v, r \rangle \wedge$
 hash-map-new-output-hit (*set-tree* *l* $\cup$ *set-tree* *r*)
 prefix-state *s*) $\vee$
 ($\exists l\ v\ r. \text{composition-tree} = \langle l, v, r \rangle \wedge$
 hash-map-new-output-hit (*set-tree* *l* $\cup$ *set-tree* *r*)
 prefix-state *s*))))))

definition

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit

where

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
out $\longleftrightarrow$
(case *out* of
 None $\Rightarrow$ *False*
| *Some* (*x*, *final-state*) $\Rightarrow$
 (let *packed* = *fst* (*fst* *x*);
 data = *snd* *packed*;


```

    attacker-state = snd (fst x);
    result = snd x;
    s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  in  $\exists$  trace-table composition-table as query-idxs dg
    composition-fri-roots final rest trace-tree composition-tree.
  Some (result, final-state)  $\in$ 
    set-dist (execute verify-monad s)  $\wedge$ 
  accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs  $\wedge$ 
  verifier-header-transcript s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    as dg composition-fri-roots final rest  $\wedge$ 
  created-tree trace-table trace-tree final-state  $\wedge$ 
  created-tree composition-table composition-tree final-state  $\wedge$ 
  (( $\exists l v r$ . trace-tree =  $\langle l, v, r \rangle$   $\wedge$ 
    hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
      s final-state)  $\vee$ 
    ( $\exists l v r$ . composition-tree =  $\langle l, v, r \rangle$   $\wedge$ 
      hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
        s final-state))))

```

definition

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit

where

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit

out $\longleftrightarrow$

(case out of

None $\Rightarrow$ False

| Some (x, final-state) $\Rightarrow$

(let packed = fst (fst x);

prefix-state = snd (fst packed);

data = snd packed;

attacker-state = snd (fst x);

result = snd x;

s =

verifier-state-from-adversary attacker-state

(staged-proof-transcript data)

in $\exists$ trace-table composition-table as query-idxs dg

composition-fri-roots final rest.

Some (result, final-state) $\in$

set-dist (execute verify-monad s) $\wedge$

accepted-with-bound-tables s (Some (result, final-state))

trace-table composition-table as query-idxs $\wedge$

verifier-header-transcript s

(staged-trace-root data)

(staged-trace-fri-roots data)
 (staged-trace-final data)
 as dg composition-fri-roots final rest $\wedge$
 hash-map-new-output-hit
 {staged-trace-root data, hd composition-fri-roots}
 prefix-state final-state))

definition

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit

where

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit

out $\longleftrightarrow$

(case *out* of

None $\Rightarrow$ False

| Some (*x*, *final-state*) $\Rightarrow$

(let *packed* = fst (fst *x*);

prefix-state = snd (fst *packed*);

data = snd *packed*;

attacker-state = snd (fst *x*);

result = snd *x*;

s =

verifier-state-from-adversary *attacker-state*

(staged-proof-transcript *data*)

in $\exists$ *trace-table* *composition-table* as *query-idxs* dg

composition-fri-roots *final* *rest*.

Some (*result*, *final-state*) $\in$

set-dist (execute *verify-monad* *s*) $\wedge$

accepted-with-bound-tables *s* (Some (*result*, *final-state*))

trace-table *composition-table* as *query-idxs* $\wedge$

verifier-header-transcript *s*

(staged-trace-root *data*)

(staged-trace-fri-roots *data*)

(staged-trace-final *data*)

as dg *composition-fri-roots* *final* *rest* $\wedge$

hash-map-new-output-hit

{staged-trace-root *data*, hd *composition-fri-roots*}

prefix-state *s*))

lemma *checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit-imp-initial-roots*

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *support*:

out $\in$

set-dist

(execute

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

A)

adversary-initial-state)

and *hit*:

```

    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
    out
  shows
    case out of
      None  $\Rightarrow$  False
    | Some (((((prefix, prefix-state), data), attacker-state),
      result), final-state)  $\Rightarrow$ 
      hash-relation-hit
      (checked-staged-transcript-initial-roots-relation A)
      adversary-initial-state attacker-state
  proof (cases out)
  case None
  then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit-def
  by simp
next
case (Some result-pack)
then obtain prefix prefix-state data attacker-state result final-state
  where out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state),
        result), final-state)
  by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have support-some:
  Some (((((prefix, prefix-state), data), attacker-state),
    result), final-state)  $\in$ 
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)
  using support unfolding out-eq .
from support-some obtain trans-out where trans-out:
  Some (((prefix, prefix-state), data), attacker-state)  $\in$ 
  set-dist
  (execute (checked-staged-transcript-with-alpha-prefix-program A)
    adversary-initial-state)
  unfolding
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have data-out:
  Some (data, attacker-state)  $\in$ 
  set-dist
  (execute (checked-staged-transcript-program A)

```

```

    adversary-initial-state)
using checked-staged-transcript-with-alpha-prefix-program-data-outcome
  [OF trans-out]
by simp
from checked-staged-transcript-with-alpha-prefix-program-support
  [OF wf controlled trans-out]
have prefix-out:
  Some (prefix, prefix-state) ∈
    set-dist
      (execute (staged-alpha-prefix-program A)
        adversary-initial-state)
by blast
have initial-prefix: adversary-initial-state ≤ prefix-state
proof –
  have program:
    hash-target-program {} (staged-alpha-search-queries budgets 0)
    (staged-alpha-prefix-program A)
  by (rule hash-target-program-staged-alpha-prefix-program
    [OF wf controlled])
  then show ?thesis
    using prefix-out
    unfolding hash-target-program-def hash-extension-preserving-def
    by blast
qed
from hit obtain trace-table composition-table as query-idxs dg
  composition-fri-roots final rest where
  verify-out:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  and bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and header:
    verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    as dg composition-fri-roots final rest
  and root-hit:
    hash-map-new-output-hit
      {staged-trace-root data, hd composition-fri-roots}
    prefix-state ?s
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit-def
    Let-def
  by simp blast
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)

```

```

    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled data-out])
  have header-eq:
    as = staged-alphas data ∧
    dg = staged-degree data ∧
    composition-fri-roots = staged-composition-fri-roots data ∧
    final = staged-composition-final data ∧
    rest = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF header staged-header]
  by simp
  from root-hit obtain x y where
    y-in: y ∈ {staged-trace-root data, hd composition-fri-roots}
    and lookup-prefix-none: fmlookup (HashMap prefix-state) x = None
    and lookup-s: fmlookup (HashMap ?s) x = Some y
    unfolding hash-map-new-output-hit-def by blast
  have lookup-initial-none: fmlookup (HashMap adversary-initial-state) x = None
  proof (cases fmlookup (HashMap adversary-initial-state) x)
    case None
    then show ?thesis .
  next
    case (Some old)
    have fmlookup (HashMap prefix-state) x = Some old
    by (rule hash-extension-lookup[OF Some initial-prefix])
    then show ?thesis
    using lookup-prefix-none by simp
  qed
  have lookup-attacker: fmlookup (HashMap attacker-state) x = Some y
  using lookup-s by simp
  have relation:
    checked-staged-transcript-initial-roots-relation A x y
  proof -
    have y ∈
      {staged-trace-root data, hd (staged-composition-fri-roots data)}
    using y-in header-eq by auto
    then show ?thesis
    unfolding checked-staged-transcript-initial-roots-relation-def
    using data-out by blast
  qed
  have
    hash-relation-hit
    (checked-staged-transcript-initial-roots-relation A)
    adversary-initial-state attacker-state
  unfolding hash-relation-hit-def

```

```

    by (intro exI[of - x] exI[of - y] conjI)
      (use lookup-initial-none lookup-attacker relation in simp-all)
  then show ?thesis
    unfolding out-eq by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
      adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  unfolding checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
proof (rule wp-event-bind-bound-by-head-event)
  show
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
      (λout. case out of
        None ⇒ False
      | Some (packed, attacker-state) ⇒
        hash-relation-hit
          (checked-staged-transcript-initial-roots-relation A)
          adversary-initial-state attacker-state)
      adversary-initial-state
      ≤ staged-phase-relation-error size
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
  by (rule
    checked-staged-transcript-with-alpha-prefix-initial-roots-relation-hit-bound
    [OF wf controlled])
next
  show
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
      None ⇒
      (case None of None ⇒ False
      | Some (packed, attacker-state) ⇒
        hash-relation-hit
          (checked-staged-transcript-initial-roots-relation A)
          adversary-initial-state attacker-state)
  unfolding
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit-def
  by simp
next
  fix packed attacker-state out
  assume trans-out:
    Some (packed, attacker-state) ∈

```

```

    set-dist
    (execute (checked-staged-transcript-with-alpha-prefix-program A)
      adversary-initial-state)
and cont:
  out ∈
    set-dist
    (execute
      (get ≫=
        (λs. put
          (verifier-state-from-adversary s
            (staged-proof-transcript (snd packed)))) ≫=
          (λ-. verify-monad ≫=
            (λresult. return ((packed, s), result))))))
      attacker-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
  out
have support:
  out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
unfolding
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
by (rule-tac x=packed and t=attacker-state in set-dist-bindI)
  (use trans-out cont in simp-all)
have rel:
  case out of
    None ⇒ False
  | Some (((prefix, prefix-state), data), attacker-state),
    result), final-state) ⇒
    hash-relation-hit
    (checked-staged-transcript-initial-roots-relation A)
    adversary-initial-state attacker-state
by (rule
  checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit-imp-initial-roots-rel
  [OF wf controlled support hit])
show
  (case Some (packed, attacker-state) of
    None ⇒ False
  | Some (packed, attacker-state) ⇒
    hash-relation-hit
    (checked-staged-transcript-initial-roots-relation A)
    adversary-initial-state attacker-state)
using cont rel
by (cases out)
  (auto elim!: set-dist-bindE split: option.splits prod.splits)

```

qed

lemma *checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-imp-tree-output-hit:*

assumes *hit:*

checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
out

shows

checked-staged-security-with-actual-alpha-prefix-tree-output-hit out

using *hit*

unfolding

checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-def
checked-staged-security-with-actual-alpha-prefix-tree-output-hit-def
Let-def

by (*cases out*) (*fastforce split: prod.splits*)+

lemma *checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-imp-root-or-subtree:*

assumes *hit:*

checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
out

shows

checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
out $\vee$
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
out

proof (*cases out*)

case *None*

then show *?thesis*

using *hit*

unfolding

checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-def

by *simp*

next

case (*Some result-pack*)

then obtain *x final-state* **where** *out-eq:*

out = Some (x, final-state)

by (*cases result-pack*) *simp*

from *hit* **obtain** *trace-table composition-table as query-idxs dg*

composition-fri-roots final rest trace-tree composition-tree **where**

body:

Some (snd x, final-state) ∈
set-dist
(execute verify-monad
(verifier-state-from-adversary (snd (fst x))
(staged-proof-transcript (snd (fst (fst x))))))

accepted-with-bound-tables

(verifier-state-from-adversary (snd (fst x))
(staged-proof-transcript (snd (fst (fst x))))

(Some (snd x, final-state)) trace-table composition-table as
query-idxs


```

verifier-header-transcript
  (verifier-state-from-adversary (snd (fst x))
    (staged-proof-transcript (snd (fst (fst x)))))
  (staged-trace-root (snd (fst (fst x))))
  (staged-trace-fri-roots (snd (fst (fst x))))
  (staged-trace-final (snd (fst (fst x))))
  as dg composition-fri-roots final rest
created-tree trace-table trace-tree final-state
created-tree composition-table composition-tree final-state
hash-map-new-output-hit
  (set-tree trace-tree  $\cup$  set-tree composition-tree)
  (snd (fst (fst (fst x)))) final-state
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-def
  Let-def
by (auto split: prod.splits)
from hash-map-new-output-hit-two-trees-root-or-subtree[OF body(6)]
show ?thesis
proof
  assume root-hit:
    hash-map-new-output-hit {value trace-tree, value composition-tree}
    (snd (fst (fst (fst x)))) final-state
  have
    checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
    out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-def
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case-def
    Let-def
  using body(1-5) root-hit
  by (cases x) (fastforce split: prod.splits)+
  then show ?thesis by simp
next
  assume subtree:
    ( $\exists l \ v \ r. \text{trace-tree} = \langle l, v, r \rangle \wedge$ 
      hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
      (snd (fst (fst (fst x)))) final-state)  $\vee$ 
    ( $\exists l \ v \ r. \text{composition-tree} = \langle l, v, r \rangle \wedge$ 
      hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
      (snd (fst (fst (fst x)))) final-state)
  have
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
    out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit-def
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case-def
    Let-def
  using body(1-5) subtree
  by (cases x) (fastforce split: prod.splits)+

```

then show *?thesis* by simp
 qed
 qed

lemma *checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-bound-from-root-and-subtree:*

assumes *root-bound:*

wp-event m

checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit

$s \leq R$

and *subtree-bound:*

wp-event m

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit

$s \leq S$

shows

wp-event m

checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit

$s \leq R + S$

proof —

have *wp-event m*

checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit

$s \leq$

wp-event m

($\lambda out.$

checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit

out $\vee$

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit

out) s

by (rule *wp-event-mono*)

(rule

checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-imp-root-or-subtree)

also have $\dots \leq$

wp-event m

checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit

$s +$

wp-event m

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit

s

by (rule *wp-event-union-bound*)

also have $\dots \leq R + S$

by (*intro add-mono root-bound subtree-bound*)

finally show *?thesis* .

qed

lemma *checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit-imp-preverifier-or-verifier-on-s*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

out $\in$

set-dist

```

      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
    and hit:
      checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
      out
  shows
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
    out ∨
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
    out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit-def
      checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case-def
    by simp
next
  case (Some result-pack)
  then obtain prefix prefix-state data attacker-state result final-state where
    out-eq:
      out =
        Some (((((prefix, prefix-state), data), attacker-state),
          result), final-state)
    by (cases result-pack, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have support-some:
    Some (((((prefix, prefix-state), data), attacker-state),
      result), final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
    using support unfolding out-eq .
  from support-some obtain trans-out where trans-out:
    Some (((prefix, prefix-state), data), attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-with-alpha-prefix-program A)
      adversary-initial-state)
  unfolding
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
from checked-staged-transcript-with-alpha-prefix-program-support

```

$[OF \text{ wf controlled trans-out}]$
have $prefix\text{-}ext: prefix\text{-}state \leq attacker\text{-}state$
by *blast*
have $prefix\text{-}s: prefix\text{-}state \leq ?s$
by (rule
 $alpha\text{-}prefix\text{-}hash\text{-}extends\text{-}verifier\text{-}state\text{-}from\text{-}adversary\text{-}right$
 $[OF \text{ prefix-ext}]$)
from *hit* **obtain** $trace\text{-}table \text{ composition-table as query-idxs dg}$
 $composition\text{-}fri\text{-}roots \text{ final rest trace-tree composition-tree}$ **where**
 $verify\text{-}out:$
 $Some (result, final\text{-}state) \in set\text{-}dist (execute \text{ verify-monad } ?s)$
and $bound:$
 $accepted\text{-}with\text{-}bound\text{-}tables ?s (Some (result, final\text{-}state))$
 $trace\text{-}table \text{ composition-table as query-idxs}$
and $header:$
 $verifier\text{-}header\text{-}transcript ?s$
 $(staged\text{-}trace\text{-}root \text{ data})$
 $(staged\text{-}trace\text{-}fri\text{-}roots \text{ data})$
 $(staged\text{-}trace\text{-}final \text{ data})$
 $as dg \text{ composition-fri-roots final rest}$
and $trace\text{-}created:$
 $created\text{-}tree \text{ trace-table trace-tree final-state}$
and $composition\text{-}created:$
 $created\text{-}tree \text{ composition-table composition-tree final-state}$
and $subtree:$
 $(\exists l \ v \ r. \text{ trace-tree} = \langle l, v, r \rangle \wedge$
 $hash\text{-}map\text{-}new\text{-}output\text{-}hit (set\text{-}tree \ l \cup \ set\text{-}tree \ r)$
 $prefix\text{-}state \ final\text{-}state) \vee$
 $(\exists l \ v \ r. \text{ composition-tree} = \langle l, v, r \rangle \wedge$
 $hash\text{-}map\text{-}new\text{-}output\text{-}hit (set\text{-}tree \ l \cup \ set\text{-}tree \ r)$
 $prefix\text{-}state \ final\text{-}state)$
unfolding $out\text{-}eq$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}branch\text{-}tree\text{-}subtree\text{-}output\text{-}hit\text{-}def$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}branch\text{-}tree\text{-}output\text{-}case\text{-}def$
 $Let\text{-}def$
by *simp blast*
have $s\text{-}final: ?s \leq final\text{-}state$
by (rule $verify\text{-}monad\text{-}hash\text{-}extends[OF \text{ verify-out}]$)
from $subtree$ **show** $?thesis$
proof
assume $trace\text{-}subtree:$
 $\exists l \ v \ r. \text{ trace-tree} = \langle l, v, r \rangle \wedge$
 $hash\text{-}map\text{-}new\text{-}output\text{-}hit (set\text{-}tree \ l \cup \ set\text{-}tree \ r)$
 $prefix\text{-}state \ final\text{-}state$
then obtain $l \ v \ r$ **where** $trace\text{-}tree\text{-}eq: \text{ trace-tree} = \langle l, v, r \rangle$
and $tree\text{-}hit:$
 $hash\text{-}map\text{-}new\text{-}output\text{-}hit (set\text{-}tree \ l \cup \ set\text{-}tree \ r)$
 $prefix\text{-}state \ final\text{-}state$
by *blast*

```

from hash-map-new-output-hit-trans-decomp
  [OF prefix-s s-final tree-hit]
show ?thesis
proof
  assume pre:
    hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
    prefix-state ?s
  have
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
      out
    unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit-def
      Let-def
    using verify-out bound header trace-created composition-created
      trace-tree-eq pre
    by simp blast
  then show ?thesis by simp
next
  assume verifier:
    hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
    ?s final-state
  have
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
      out
    unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit-def
      Let-def
    using verify-out bound header trace-created composition-created
      trace-tree-eq verifier
    by simp blast
  then show ?thesis by simp
qed
next
  assume composition-subtree:
     $\exists l \ v \ r. \text{composition-tree} = \langle l, v, r \rangle \wedge$ 
    hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
    prefix-state final-state
  then obtain l v r where
    composition-tree-eq: composition-tree =  $\langle l, v, r \rangle$ 
  and tree-hit:
    hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
    prefix-state final-state
  by blast
from hash-map-new-output-hit-trans-decomp
  [OF prefix-s s-final tree-hit]
show ?thesis
proof
  assume pre:
    hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)

```

```

    prefix-state ?s
  have
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
      out
    unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit-def
      Let-def
    using verify-out bound header trace-created composition-created
      composition-tree-eq pre
    by simp blast
  then show ?thesis by simp
next
  assume verifier:
    hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
    ?s final-state
  have
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
      out
    unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit-def
      Let-def
    using verify-out bound header trace-created composition-created
      composition-tree-eq verifier
    by simp blast
  then show ?thesis by simp
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit-bound-from-preverifier-and-verifier
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and pre-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
    adversary-initial-state  $\leq P$ 
  and verifier-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
    adversary-initial-state  $\leq V$ 
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
    adversary-initial-state  $\leq P + V$ 

```

```

proof –
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?Sub =
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
  let ?Pre =
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
  let ?Verifier =
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
  have wp-event ?M ?Sub adversary-initial-state ≤
    wp-event ?M (λout. ?Pre out ∨ ?Verifier out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (rule
      checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit-imp-preverifier-or-verifier-on-supp
      [OF wf controlled])
  also have ... ≤
    wp-event ?M ?Pre adversary-initial-state +
    wp-event ?M ?Verifier adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤ P + V
  by (intro add-mono pre-bound verifier-bound)
  finally show ?thesis .
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-imp-transcript-root-output-hit:*
assumes *root-hit:*

checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
out

shows

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit
out

proof (*cases out*)

case *None*

then show ?thesis

using *root-hit*

unfolding

checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-def

checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case-def

by *simp*

next

case (*Some result-pack*)

then obtain *x final-state* **where** *out-eq:*

out = Some (x, final-state)

by (*cases result-pack*) *simp*

let ?packed = *fst (fst x)*

let ?prefix-state = *snd (fst ?packed)*

let ?data = *snd ?packed*

let ?attacker-state = *snd (fst x)*

```

let ?result = snd x
let ?s =
  verifier-state-from-adversary ?attacker-state
  (staged-proof-transcript ?data)
from root-hit obtain trace-table composition-table as query-idxs dg
  composition-fri-roots final rest trace-tree composition-tree where
  verify-out:
    Some (?result, final-state) ∈ set-dist (execute verify-monad ?s)
and bound:
  accepted-with-bound-tables ?s (Some (?result, final-state))
  trace-table composition-table as query-idxs
and header:
  verifier-header-transcript ?s
  (staged-trace-root ?data)
  (staged-trace-fri-roots ?data)
  (staged-trace-final ?data)
  as dg composition-fri-roots final rest
and trace-created:
  created-tree trace-table trace-tree final-state
and composition-created:
  created-tree composition-table composition-tree final-state
and tree-root-hit:
  hash-map-new-output-hit
  {value trace-tree, value composition-tree}
  ?prefix-state final-state
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-def
  checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case-def
  Let-def
by simp blast
from bound obtain fr' f-fri-roots' f-final' dg'
  composition-fri-roots' final' rest' where
  header':
    verifier-header-transcript ?s fr' f-fri-roots' f-final' as dg'
    composition-fri-roots' final' rest'
and trace-bind:
  merkle-root-binds-table fr' trace-table final-state
and composition-bind:
  merkle-root-binds-table (hd composition-fri-roots')
  composition-table final-state
unfolding accepted-with-bound-tables-def by auto
have header-eq:
  fr' = staged-trace-root ?data ∧
  f-fri-roots' = staged-trace-fri-roots ?data ∧
  f-final' = staged-trace-final ?data ∧
  dg' = dg ∧
  composition-fri-roots' = composition-fri-roots ∧
  final' = final ∧
  rest' = rest

```



```

using verifier-header-transcript-unique[OF header' header]
by simp
then have fr-eq: fr' = staged-trace-root ?data
and comp-roots-eq:
  composition-fri-roots' = composition-fri-roots
by simp-all
from trace-bind obtain trace-root-tree where
  trace-root-created:
    created-tree trace-table trace-root-tree final-state
and trace-root-eq: fr' = value trace-root-tree
unfolding merkle-root-binds-table-def by blast
have trace-value-eq:
  value trace-tree = staged-trace-root ?data
using created-tree-same-table-value-eq
  [OF trace-created trace-root-created]
  trace-root-eq fr-eq
by simp
from composition-bind obtain composition-root-tree where
  composition-root-created:
    created-tree composition-table composition-root-tree final-state
and composition-root-eq:
  hd composition-fri-roots' = value composition-root-tree
unfolding merkle-root-binds-table-def by blast
have composition-value-eq:
  value composition-tree = hd composition-fri-roots
using created-tree-same-table-value-eq
  [OF composition-created composition-root-created]
  composition-root-eq comp-roots-eq
by simp
have transcript-root-hit:
  hash-map-new-output-hit
  {staged-trace-root ?data, hd composition-fri-roots}
  ?prefix-state final-state
using tree-root-hit trace-value-eq composition-value-eq by simp
show ?thesis
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit-def
  Let-def
using verify-out bound header transcript-root-hit
by simp blast
qed

lemma checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-bound-from-transcript-roots:
assumes root-bound:
  wp-event m
  checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit
   $s \leq R$ 
shows
  wp-event m

```

checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
 $s \leq R$
by (*rule order.trans[OF - root-bound]*)
(rule wp-event-mono,
rule
checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-imp-transcript-root-output-hit)

lemma *checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit-imp-preverifier-or-verifier-o*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
out $\in$
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)
and *hit:*
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit
out
shows
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
out $\vee$
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
out
proof (*cases out*)
case *None*
then show *?thesis*
using *hit*
unfolding
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit-def
by *simp*
next
case (*Some result-pack*)
then obtain *prefix prefix-state data attacker-state result final-state* **where**
out-eq:
out =
Some (((((prefix, prefix-state), data), attacker-state),
result), final-state)
by (*cases result-pack, auto split: prod.splits*)
let *?s* =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
have *support-some:*
Some (((((prefix, prefix-state), data), attacker-state),
result), final-state) $\in$
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

```

    A)
    adversary-initial-state)
  using support unfolding out-eq .
from support-some obtain trans-out where trans-out:
  Some (((prefix, prefix-state), data), attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-with-alpha-prefix-program A)
    adversary-initial-state)
  unfolding
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
from checked-staged-transcript-with-alpha-prefix-program-support
  [OF wf controlled trans-out]
have prefix-ext: prefix-state ≤ attacker-state
  by blast
have prefix-s: prefix-state ≤ ?s
  by (rule
    alpha-prefix-hash-extends-verifier-state-from-adversary-right
    [OF prefix-ext])
from hit obtain trace-table composition-table as query-idxs dg
  composition-fri-roots final rest where
  verify-out:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  and bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and header:
    verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    as dg composition-fri-roots final rest
  and root-hit:
    hash-map-new-output-hit
    {staged-trace-root data, hd composition-fri-roots}
    prefix-state final-state
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit-def
    Let-def
  by simp blast
have s-final: ?s ≤ final-state
  by (rule verify-monad-hash-extends[OF verify-out])
from hash-map-new-output-hit-trans-decomp
  [OF prefix-s s-final root-hit]
show ?thesis
proof
  assume pre:
    hash-map-new-output-hit
    {staged-trace-root data, hd composition-fri-roots}

```

```

    prefix-state ?s
  have
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
      out
    unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit-def
      Let-def
    using verify-out bound header pre
    by simp blast
  then show ?thesis by simp
next
  assume verifier:
    hash-map-new-output-hit
      {staged-trace-root data, hd composition-fri-roots}
      ?s final-state
  have
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
      out
    unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-def
      Let-def
    using verify-out bound header verifier
    by simp blast
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit-bound-from-preverifier-and-
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and pre-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
      adversary-initial-state  $\leq P$ 
  and verifier-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
      adversary-initial-state  $\leq V$ 
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit
      adversary-initial-state  $\leq P + V$ 
proof -
  let ?M =

```

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let ?Root =
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit
let ?Pre =
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
let ?Verifier =
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
have wp-event ?M ?Root adversary-initial-state ≤
wp-event ?M (λout. ?Pre out ∨ ?Verifier out)
adversary-initial-state
by (rule wp-event-mono-on-support)
(rule
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit-imp-preverifier-or-verifier-on-s
[OF wf controlled])
also have ... ≤
wp-event ?M ?Pre adversary-initial-state +
wp-event ?M ?Verifier adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤ P + V
by (intro add-mono pre-bound verifier-bound)
finally show ?thesis .
qed

lemma *checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-bound-from-preverifier-and-verifier*
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and pre-bound:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
adversary-initial-state ≤ P
and verifier-bound:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
adversary-initial-state ≤ V
shows
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
adversary-initial-state ≤ P + V
proof –
have root-bound:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit
adversary-initial-state ≤ P + V

by (rule
 checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit-bound-from-preverifier-and-verifier
 [*OF wf controlled pre-bound verifier-bound*])
 show ?thesis
 by (rule
 checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-bound-from-transcript-roots
 [*OF root-bound*])
 qed

lemma *checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-bound-from-preverifier-and-concrete-transcript*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *pre-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
 adversary-initial-state $\leq P$
and *target-bound:*
 $\bigwedge$ *packed attacker-state.*
 Some (packed, attacker-state) ∈
 set-dist
 (*execute (checked-staged-transcript-with-alpha-prefix-program A)*
 adversary-initial-state) $\implies$
 concrete-transcript-target-error (staged-proof-transcript (snd packed))
 $\leq V$
shows
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
 adversary-initial-state $\leq P + V$
proof –
have *verifier-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
 adversary-initial-state $\leq V$
by (rule
 checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-bound-from-concrete-transcript
 [*OF target-bound*])
show ?thesis
by (rule
 checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-bound-from-preverifier-and-verifier
 [*OF wf controlled pre-bound verifier-bound*])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-bound-from-preverifier-concrete-transcript*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*

and *pre-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
adversary-initial-state $\leq P$
and *target-bound*:
 $\bigwedge$ *packed attacker-state*.
Some (packed, attacker-state) $\in$
set-dist
(execute (checked-staged-transcript-with-alpha-prefix-program A)
adversary-initial-state) $\implies$
concrete-transcript-target-error (staged-proof-transcript (snd packed))
 $\leq V$
and *subtree-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
adversary-initial-state $\leq S$
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
adversary-initial-state $\leq P + V + S$
proof –
have *root-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
adversary-initial-state $\leq P + V$
by (rule
checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-bound-from-preverifier-and-concrete-transcript
 $[OF\ wf\ controlled\ pre-bound\ target-bound]$)
have
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
adversary-initial-state $\leq (P + V) + S$
by (rule
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-bound-from-root-and-subtree
 $[OF\ root-bound\ subtree-bound]$)
then show *?thesis*
by (*simp add: add.assoc*)
qed

lemma *checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-bound-from-preverifier-concrete-transcript*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*

and *root-pre-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
adversary-initial-state $\leq P$
and *target-bound*:
 $\bigwedge$ *packed attacker-state.*
Some (packed, attacker-state) $\in$
set-dist
(execute (checked-staged-transcript-with-alpha-prefix-program A)
adversary-initial-state) $\implies$
concrete-transcript-target-error (staged-proof-transcript (snd packed))
 $\leq V$
and *subtree-pre-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
adversary-initial-state $\leq SP$
and *subtree-verifier-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
adversary-initial-state $\leq SV$
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
adversary-initial-state $\leq P + V + SP + SV$
proof –
have *subtree-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
adversary-initial-state $\leq SP + SV$
by (rule
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit-bound-from-preverifier-and-verifier
[*OF wf controlled subtree-pre-bound subtree-verifier-bound*])
have
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
adversary-initial-state $\leq P + V + (SP + SV)$
by (rule
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-bound-from-preverifier-concrete-transcript
[*OF wf controlled root-pre-bound target-bound subtree-bound*])
then show ?thesis


```

    by (simp add: add.assoc)
qed

lemma checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event P)
    adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event P)
    adversary-initial-state
proof -
  let ?project =
    λx. ((snd (fst (fst x)), snd (fst x)), snd x)
  have
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event P)
      adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
        ≫ (λx. return (?project x))
      (staged-security-with-data-state-verifier-event P)
      adversary-initial-state
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
also have ... =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (λout. case out of
      None ⇒ staged-security-with-data-state-verifier-event P None
    | Some (x, t) ⇒
      staged-security-with-data-state-verifier-event P
        (Some (?project x, t)))
    adversary-initial-state
  by (rule wp-event-bind-return-map)
also have ... =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event P)
    adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  apply (rule arg-cong[where
    f = λQ. wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      Q adversary-initial-state])
  apply (rule ext)

```

```

    apply (simp add: Let-def split: option.splits prod.splits)
  done
  finally show ?thesis .
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-prefix-or-branch-tree-on-
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    out ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
            A)
          adversary-initial-state)
  and bad:
    checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-randomization-bad out
  shows
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
      composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
      out
  proof (cases out)
  case None
  then show ?thesis
    using bad
    unfolding
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    by simp
  next
  case (Some result-pack)
  then obtain prefix prefix-state data attacker-state result final-state where
    out-eq:
      out =
        Some (((((prefix, prefix-state), data), attacker-state),
          result), final-state)
    by (cases result-pack, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have support-some:
    Some (((((prefix, prefix-state), data), attacker-state),
      result), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)

```

```

using support unfolding out-eq .
from support-some obtain trans-out where trans-out:
  Some (((prefix, prefix-state), data), attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-with-alpha-prefix-program A)
      adversary-initial-state)
and verify-out:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
unfolding
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
by (auto elim! set-dist-bindE split prod.splits)
from checked-staged-transcript-with-alpha-prefix-program-support
  [OF wf controlled trans-out]
have prefix-out:
  Some (prefix, prefix-state) ∈
    set-dist
    (execute (staged-alpha-prefix-program A)
      adversary-initial-state)
and root-eq: staged-trace-root data = fst prefix
and trace-roots-eq:
  staged-trace-fri-roots data = fst (snd prefix)
and trace-bs-eq:
  staged-trace-fri-challenges data = fst (snd (snd prefix))
and trace-final-eq:
  staged-trace-final data = snd (snd (snd prefix))
and prefix-ext: prefix-state ≤ attacker-state
by blast+
have prefix-tuple:
  prefix =
    (staged-trace-root data, staged-trace-fri-roots data,
      staged-trace-fri-challenges data, staged-trace-final data)
  by (cases prefix; cases snd prefix; cases snd (snd prefix))
    (use root-eq trace-roots-eq trace-bs-eq trace-final-eq in auto)
have random-bad:
  composition-randomization-bad ?s (Some (result, final-state))
using bad unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  Let-def
by simp
have alpha-hit:
  alpha-bad-set-hit ?s composition-trace-bad-alpha-space
    (Some (result, final-state))
  by (rule composition-randomization-bad-imp-alpha-bad-set-hit
    [OF random-bad])
from alpha-hit obtain trace-table composition-table as query-idxs where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
and as-bad: as ∈ composition-trace-bad-alpha-space trace-table

```

```

unfolding alpha-bad-set-hit-def by blast
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
proof –
  have projected:
    Some (data, attacker-state) ∈
      set-dist
        (execute
          (checked-staged-transcript-with-alpha-prefix-program A  $\gg$ 
            ( $\lambda$ packed. return (snd packed))))
          adversary-initial-state)
    by (rule set-dist-bindI[OF trans-out]) simp
  have checked-builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
    using projected
    unfolding checked-staged-transcript-with-alpha-prefix-program-projection
    by simp
  show ?thesis
    by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled checked-builder])
qed
from bound obtain fr f-fri-roots f-final dg composition-fri-roots final
  rest where
    header:
      verifier-header-transcript ?s fr f-fri-roots f-final as dg
      composition-fri-roots final rest
    unfolding accepted-with-bound-tables-def by blast
  have header-eq:
    fr = staged-trace-root data  $\wedge$ 
    f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
    f-final = staged-trace-final data  $\wedge$ 
    as = staged-alphas data  $\wedge$ 
    dg = staged-degree data  $\wedge$ 
    composition-fri-roots = staged-composition-fri-roots data  $\wedge$ 
    final = staged-composition-final data  $\wedge$ 
    rest = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF header staged-header]
  by simp

```

```

have staged-bad:
  staged-alphas data  $\in$  composition-trace-bad-alpha-space trace-table
  using as-bad header-eq by simp
have alphas-space: staged-alphas data  $\in$  alpha-space
  using staged-bad composition-trace-bad-alpha-space-subset-alpha-space
  by blast
have prefix-s: prefix-state  $\leq$  ?s
  by (rule
    alpha-prefix-hash-extends-verifier-state-from-adversary-right
    [OF prefix-ext])
have s-final: ?s  $\leq$  final-state
  by (rule verify-monad-hash-extends[OF verify-out])
have prefix-final: prefix-state  $\leq$  final-state
  by (rule hash-ext-trans[OF prefix-s s-final])
show ?thesis
proof (rule
  accepted-with-bound-tables-initial-roots-pullback-or-new-tree-output-hit
  [OF prefix-final bound])
  fix fr' f-fri-roots' f-final' dg' composition-fri-roots' final'
  rest' trace-tree composition-tree
  assume pull-header:
    verifier-header-transcript ?s fr' f-fri-roots' f-final' as dg'
    composition-fri-roots' final' rest'
  and trace-created:
    created-tree trace-table trace-tree final-state
  and composition-created:
    created-tree composition-table composition-tree final-state
  and split:
    (merkle-root-binds-table fr' trace-table prefix-state  $\wedge$ 
    merkle-root-binds-table (hd composition-fri-roots')
    composition-table prefix-state)  $\vee$ 
    hash-map-new-output-hit
    (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
  have pull-eq:
    fr' = staged-trace-root data  $\wedge$ 
    f-fri-roots' = staged-trace-fri-roots data  $\wedge$ 
    f-final' = staged-trace-final data  $\wedge$ 
    dg' = staged-degree data  $\wedge$ 
    composition-fri-roots' = staged-composition-fri-roots data  $\wedge$ 
    final' = staged-composition-final data  $\wedge$ 
    rest' = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF pull-header staged-header]
  header-eq
  by simp
from split show ?thesis
proof
  assume binds:
    merkle-root-binds-table fr' trace-table prefix-state  $\wedge$ 

```

```

    merkle-root-binds-table (hd composition-fri-roots')
    composition-table prefix-state
have trace-len: length trace-table = clength * scale
    using bound unfolding accepted-with-bound-tables-def
    accepted-with-tables-def by simp
have prefix-candidate:
    trace-table ∈
    alpha-prefix-trace-table-candidates prefix-state (fst prefix)
    by (rule alpha-prefix-trace-table-candidateI[OF trace-len])
    (use binds pull-eq root-eq in simp)
have prefix-bad:
    staged-alphas data ∈
    alpha-prefix-union-bad-sets prefix-state
    composition-trace-bad-alpha-space (fst prefix)
    unfolding alpha-prefix-union-bad-sets-def
    using alphas-space prefix-candidate staged-bad by auto
have
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out
    unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-def
    Let-def
    using prefix-bad by simp
then show ?thesis by simp
next
assume tree-hit:
    hash-map-new-output-hit
    (set-tree trace-tree ∪ set-tree composition-tree)
    prefix-state final-state
have
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
    out
    unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-def
    Let-def
    using verify-out bound pull-header trace-created composition-created
    tree-hit pull-eq
    by simp blast
then show ?thesis by simp
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-bound-from-prefix-and-branch
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and prefix-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state

```

A)
 (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq P$
and branch-tree-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
 adversary-initial-state $\leq T$
shows
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-randomization-bad)
 adversary-initial-state $\leq P + T$
proof –
 let ?M =
 checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
 let ?random =
 checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-randomization-bad
 let ?prefix =
 checked-staged-security-with-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space
 let ?tree =
 checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
have wp-event ?M ?random adversary-initial-state $\leq$
 wp-event ?M ($\lambda out. ?prefix\ out \vee ?tree\ out$)
 adversary-initial-state
by (rule wp-event-mono-on-support)
 (rule
 checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-prefix-or-branch-tree-on-sup
 [OF wf controlled])
also have ... $\leq$
 wp-event ?M ?prefix adversary-initial-state +
 wp-event ?M ?tree adversary-initial-state
by (rule wp-event-union-bound)
also have ... $\leq P + T$
by (intro add-mono prefix-bound branch-tree-bound)
finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-branch-tree:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and prefix-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state

```

    A)
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq P$ 
and branch-tree-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
    adversary-initial-state  $\leq T$ 
shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state  $\leq P + T$ 
proof –
    let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
    let ?degree =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-degree-bad
    let ?random =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-randomization-bad
    let ?bad =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-bad
    have projected:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state =
    wp-event ?M ?bad adversary-initial-state
    by (rule
    checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
    have event-eq: ?bad = ( $\lambda$ out. ?degree out  $\vee$  ?random out)
    by (rule ext)
    (auto simp:
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    composition-bad-def split: option.splits prod.splits)
    have degree-false: ?degree = ( $\lambda$ out. False)
    by (rule ext)
    (auto simp:
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    composition-degree-bad-false[OF spec-degree-wellformed-from-spec-query-margin]
    split: option.splits prod.splits)
    have degree-bound:
    wp-event ?M ?degree adversary-initial-state  $\leq 0$ 
    unfolding degree-false wp-event-def wp-def dist-expect-def by simp
    have random-bound:
    wp-event ?M ?random adversary-initial-state  $\leq P + T$ 

```


by (rule
 checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-bound-from-prefix-and-branch-tree
 [*OF wf controlled prefix-bound branch-tree-bound*])
 have *wp-event* ?*M* ?*bad* *adversary-initial-state* ≤
 wp-event ?*M* ?*degree* *adversary-initial-state* +
 wp-event ?*M* ?*random* *adversary-initial-state*
 unfolding *event-eq* by (rule *wp-event-union-bound*)
 also have ... ≤ 0 + (*P* + *T*)
 by (intro *add-mono degree-bound random-bound*)
 finally show ?*thesis*
 unfolding *projected* by *simp*
 qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-branch-tree-and-budget*
assumes *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *branch-tree-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
 adversary-initial-state ≤ *T*

shows

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event composition-bad*)
adversary-initial-state ≤
composition-error-bound +
staged-phase-relation-error size
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
T

proof –

let ?*P* =

composition-error-bound +
staged-phase-relation-error size
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0)

have *transcript-bound*:

wp-event (*checked-staged-transcript-with-alpha-prefix-program* *A*)
 (*checked-staged-transcript-actual-alpha-prefix-bad-set-hit*
 composition-trace-bad-alpha-space)
adversary-initial-state ≤ ?*P*

by (rule

checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
 [*OF wf controlled*])

have *prefix-bound*:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq ?P$
 by (rule
 checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
 [OF transcript-bound])
 have
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event composition-bad)
 adversary-initial-state $\leq ?P + T$
 by (rule
 checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-branch-tree
 [OF wf controlled prefix-bound branch-tree-bound])
 then show ?thesis
 by (simp add: add.assoc)
 qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-side-events:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and root-pre-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
 adversary-initial-state $\leq RP$
and target-bound:
 $\bigwedge$ packed attacker-state.
 Some (packed, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-with-alpha-prefix-program A)
 adversary-initial-state) $\implies$
 concrete-transcript-target-error (staged-proof-transcript (snd packed))
 $\leq RV$
and subtree-pre-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
 adversary-initial-state $\leq SP$
and subtree-verifier-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
 adversary-initial-state $\leq SV$
shows

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event composition-bad*)
 $adversary\text{-}initial\text{-}state \leq$
 $composition\text{-}error\text{-}bound +$
 $staged\text{-}phase\text{-}relation\text{-}error\ size$
 (*staged-alpha-search-queries budgets 0*) +
 $hash\text{-}collision\text{-}budget\text{-}value\ 0$
 (*staged-alpha-search-queries budgets 0*) +
 $(RP + RV + SP + SV)$

proof –

have *branch-tree-bound*:

$wp\text{-}event$
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
 $adversary\text{-}initial\text{-}state \leq RP + RV + SP + SV$

by (*rule*
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-bound-from-preverifier-concrete-transcript-b
 [*OF wf controlled root-pre-bound target-bound subtree-pre-bound*
subtree-verifier-bound])

show *?thesis*

by (*rule*
checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-branch-tree-and-budgets
 [*OF wf controlled branch-tree-bound*])

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-side-events-and-transcript-b*

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *root-pre-bound*:

$wp\text{-}event$
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
 $adversary\text{-}initial\text{-}state \leq RP$

and *subtree-pre-bound*:

$wp\text{-}event$
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
 $adversary\text{-}initial\text{-}state \leq SP$

and *subtree-verifier-bound*:

$wp\text{-}event$
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
 $adversary\text{-}initial\text{-}state \leq SV$

shows

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event composition-bad*)

$adversary-initial-state \leq$
 $composition-error-bound +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $(RP + staged-concrete-transcript-target-error-bound + SP + SV)$

proof –

have *target-bound*:

$\wedge packed\ attacker-state.$

$Some\ (packed,\ attacker-state) \in$
 $set-dist$
 $(execute\ (checked-staged-transcript-with-alpha-prefix-program\ A)$
 $adversary-initial-state) \implies$
 $concrete-transcript-target-error\ (staged-proof-transcript\ (snd\ packed))$
 $\leq\ staged-concrete-transcript-target-error-bound$

by (rule
 $checked-staged-transcript-with-alpha-prefix-program-concrete-transcript-target-error-bound$
 $[OF\ wf\ controlled])$

show *?thesis*

by (rule
 $checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-side-events$
 $[OF\ wf\ controlled\ root-pre-bound\ target-bound\ subtree-pre-bound$
 $subtree-verifier-bound])$

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-side-events-and-transcript-b*

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *subtree-pre-bound*:

wp-event

$(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$

$checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit$
 $adversary-initial-state \leq SP$

and *subtree-verifier-bound*:

wp-event

$(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$

$checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit$
 $adversary-initial-state \leq SV$

shows

wp-event $(checked-staged-security-experiment-with-data-state\ A)$

$(staged-security-with-data-state-verifier-event\ composition-bad)$

$adversary-initial-state \leq$

$composition-error-bound +$

$staged-phase-relation-error\ size$

$(staged-alpha-search-queries\ budgets\ 0) +$

$hash-collision-budget-value\ 0$

```

      (staged-alpha-search-queries budgets 0) +
      (staged-phase-relation-error size
      (staged-attacker-query-budget budgets + staged-challenge-query-budget) +
      staged-concrete-transcript-target-error-bound + SP + SV)
proof -
  have root-pre-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit-bound
    [OF wf controlled])
  show ?thesis
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-side-events-and-transcript-budg
    [OF wf controlled root-pre-bound subtree-pre-bound
     subtree-verifier-bound])
qed

end

end

```

```

theory Staged-Security-Experiment
  imports Staged-Security-Experiment-Composition-Branch
begin

```

Compatibility umbrella for the staged-adversary security experiment.

```

end

```

```

theory Soundness-Partial-Merkle
  imports
    Staged-Security-Experiment
begin

```

4.4 Partial Merkle Openings

This theory records the authenticated openings that the verifier actually observes. It provides partial table candidates, consistency predicates, and reductions from inconsistent authenticated openings to hash-collision events.

```

record 'f authenticated-opening =
  opening-root :: 'f

```

```

opening-length :: nat
opening-index  :: nat
opening-value  :: 'f
opening-path   :: 'f list

fun merkle-path-bound
  :: 'f ⇒ nat ⇒ nat ⇒ 'f ⇒ 'f list ⇒
    ('f, 'a) protocol-channel-scheme ⇒ bool
where
  merkle-path-bound rt len idx v [] s ⟷
    fmlookup (HashMap s) (MerkleLeaf v) = Some rt
| merkle-path-bound rt len idx v (sibling # path) s ⟷
    (if idx < len div 2 then
      (∃ child.
        merkle-path-bound child (len div 2) idx v path s ∧
        fmlookup (HashMap s) (MerkleNode child sibling) = Some rt)
    else
      (∃ child.
        merkle-path-bound child (len div 2) (idx - len div 2)
          v path s ∧
        fmlookup (HashMap s) (MerkleNode sibling child) = Some rt))

context soundness
begin

definition authenticated-opening-in
  :: ('f, 'a) protocol-channel-scheme ⇒
    'f authenticated-opening ⇒ bool
where
  authenticated-opening-in s opening ⟷
    opening-index opening < opening-length opening ∧
    length (opening-path opening) =
      floor-log (opening-length opening) ∧
    merkle-path-bound
      (opening-root opening)
      (opening-length opening)
      (opening-index opening)
      (opening-value opening)
      (opening-path opening)
    s

definition partial-authenticated-table
  :: 'f ⇒ nat ⇒ 'f authenticated-opening list ⇒
    ('f, 'a) protocol-channel-scheme ⇒ bool
where
  partial-authenticated-table rt len openings s ⟷
    (∀ opening ∈ set openings.
      opening-root opening = rt ∧
      opening-length opening = len ∧

```

authenticated-opening-in s opening)

lemma *merkle-path-bound-mono*:
 assumes *bound*: *merkle-path-bound* *rt len idx v path s*
 and *ext*: $s \leq t$
 shows *merkle-path-bound* *rt len idx v path t*
 using *bound*
proof (*induction path arbitrary: rt len idx*)
 case *Nil*
 then show *?case*
 using *hash-extension-lookup*[*OF - ext*] **by** *simp*
next
 case (*Cons sibling path*)
 show *?case*
 proof (*cases idx < len div 2*)
 case *True*
 then obtain child where
 child:
 merkle-path-bound *child (len div 2) idx v path s*
 and *lookup*:
 fmlookup (*HashMap s*) (*MerkleNode child sibling*) = *Some rt*
 using *Cons.premis* **by** *auto*
 have *child-t*:
 merkle-path-bound *child (len div 2) idx v path t*
 by (*rule Cons.IH*[*OF child*])
 have *lookup-t*:
 fmlookup (*HashMap t*) (*MerkleNode child sibling*) = *Some rt*
 by (*rule hash-extension-lookup*[*OF lookup ext*])
 show *?thesis*
 using *True child-t lookup-t*
 by (*auto intro!: exI*[*of - child*])
next
 case *False*
 then obtain child where
 child:
 merkle-path-bound *child (len div 2) (idx - len div 2)*
 v path s
 and *lookup*:
 fmlookup (*HashMap s*) (*MerkleNode sibling child*) = *Some rt*
 using *Cons.premis* **by** *auto*
 have *child-t*:
 merkle-path-bound *child (len div 2) (idx - len div 2)*
 v path t
 by (*rule Cons.IH*[*OF child*])
 have *lookup-t*:
 fmlookup (*HashMap t*) (*MerkleNode sibling child*) = *Some rt*
 by (*rule hash-extension-lookup*[*OF lookup ext*])
 show *?thesis*
 using *False child-t lookup-t*

```

      by (auto intro!: exI[of - child])
    qed
  qed

```

```

lemma authenticated-opening-in-mono:
  assumes opn-auth: authenticated-opening-in s opn
    and ext:  $s \leq t$ 
  shows authenticated-opening-in t opn
  using opn-auth merkle-path-bound-mono[OF - ext]
  unfolding authenticated-opening-in-def by blast

```

```

lemma partial-authenticated-table-mono:
  assumes table: partial-authenticated-table rt len openings s
    and ext:  $s \leq t$ 
  shows partial-authenticated-table rt len openings t
  using table authenticated-opening-in-mono[OF - ext]
  unfolding partial-authenticated-table-def by blast

```

```

lemma merkle-path-bound-root-lookup:
  assumes merkle-path-bound rt len idx v path s
  shows  $\exists \text{input. fmlookup (HashMap s) input = Some rt}$ 
  using assms
proof (induction path arbitrary: rt len idx)
  case Nil
  then show ?case
    by (intro exI[of - MerkleLeaf v]) simp
next
  case (Cons sibling path)
  then show ?case
    by (cases idx < len div 2) auto
qed

```

```

lemma authenticated-opening-root-lookup:
  assumes authenticated-opening-in s opening
  shows  $\exists \text{input. fmlookup (HashMap s) input = Some (opening-root opening)}$ 
  using assms
  unfolding authenticated-opening-in-def
  by (blast intro: merkle-path-bound-root-lookup)

```

```

lemma authenticated-opening-root-late-output-preexisting-or-new-hit:
  assumes ext:  $s \leq t$ 
    and auth: authenticated-opening-in t opening
  shows
     $(\exists \text{input. fmlookup (HashMap s) input = Some (opening-root opening)}) \vee$ 
    hash-map-new-output-hit {opening-root opening} s t
proof –
  from authenticated-opening-root-lookup[OF auth]
  obtain input where lookup-t:
    fmlookup (HashMap t) input = Some (opening-root opening)

```



```

    by blast
show ?thesis
proof (cases fmlookup (HashMap s) input)
  case None
  then have hash-map-new-output-hit {opening-root opening} s t
    unfolding hash-map-new-output-hit-def
    using lookup-t by blast
  then show ?thesis by simp
next
  case (Some old)
  have lookup-t-old: fmlookup (HashMap t) input = Some old
    by (rule hash-extension-lookup[OF Some ext])
  then have old = opening-root opening
    using lookup-t by simp
  then show ?thesis
    using Some by blast
qed
qed

lemma partial-authenticated-table-root-late-output-preexisting-or-new-hit:
  assumes ext:  $s \leq t$ 
  and table: partial-authenticated-table rt len openings t
  and opn-in: opening  $\in$  set openings
  shows
    ( $\exists$  input. fmlookup (HashMap s) input = Some rt)  $\vee$ 
    hash-map-new-output-hit {rt} s t
proof -
  have auth: authenticated-opening-in t opening
    using table opn-in unfolding partial-authenticated-table-def by blast
  have root-eq: opening-root opening = rt
    using table opn-in unfolding partial-authenticated-table-def by blast
  show ?thesis
    using authenticated-opening-root-late-output-preexisting-or-new-hit
      [OF ext auth]
    unfolding root-eq .
qed

lemma merkle-path-bound-root-in-map:
  assumes merkle-path-bound rt len idx v path s
  shows merkle-path-root-in-map s len idx v path = Some rt
  using assms
proof (induction path arbitrary: rt len idx)
  case Nil
  then show ?case by simp
next
  case (Cons sibling path)
  then show ?case
    by (cases idx < len div 2) auto
qed

```

```

lemma merkle-path-root-in-map-some-imp-bound:
  assumes merkle-path-root-in-map s len idx v path = Some rt
  shows merkle-path-bound rt len idx v path s
  using assms
proof (induction path arbitrary: rt len idx)
  case Nil
  then show ?case by simp
next
  case (Cons sibling path)
  show ?case
  proof (cases idx < len div 2)
    case True
    from Cons.prem1 obtain child where
      child-root:
        merkle-path-root-in-map s (len div 2) idx v path =
          Some child
    and lookup:
      fmlookup (HashMap s) (MerkleNode child sibling) = Some rt
    using True by (auto split: option.splits)
    have child-bound:
      merkle-path-bound child (len div 2) idx v path s
    by (rule Cons.IH[OF child-root])
    show ?thesis
    using True child-bound lookup by auto
  next
  case False
  from Cons.prem1 obtain child where
    child-root:
      merkle-path-root-in-map s (len div 2) (idx - len div 2)
        v path = Some child
    and lookup:
      fmlookup (HashMap s) (MerkleNode sibling child) = Some rt
    using False by (auto split: option.splits)
    have child-bound:
      merkle-path-bound child (len div 2) (idx - len div 2) v path s
    by (rule Cons.IH[OF child-root])
    show ?thesis
    using False child-bound lookup by auto
  qed
qed

fun merkle-path-target-roots
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  'f  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$ 
    'f  $\Rightarrow$  'f list  $\Rightarrow$  'f set
where
  merkle-path-target-roots t rt len idx v [] = {rt}
  | merkle-path-target-roots t rt len idx v (sibling # path) =
    {rt}  $\cup$ 

```

```

(if idx < len div 2 then
  (case merkle-path-root-in-map t (len div 2) idx v path of
    None  $\Rightarrow$  {}
  | Some child  $\Rightarrow$ 
    merkle-path-target-roots t child (len div 2) idx v path)
else
  (case merkle-path-root-in-map t (len div 2) (idx - len div 2)
    v path of
    None  $\Rightarrow$  {}
  | Some child  $\Rightarrow$ 
    merkle-path-target-roots t child (len div 2)
      (idx - len div 2) v path))

```

lemma *finite-merkle-path-target-roots[simp]:*
finite (merkle-path-target-roots t rt len idx v path)
by (*induction path arbitrary: rt len idx*)
(auto split: option.splits if-splits)

lemma *card-merkle-path-target-roots-le:*
card (merkle-path-target-roots t rt len idx v path) $\leq$ Suc (length path)

proof (*induction path arbitrary: rt len idx*)
case *Nil*
then show *?case by simp*
next
case (*Cons sibling path*)
show *?case*
proof (*cases idx < len div 2*)
case *True*
then show *?thesis*
proof (*cases merkle-path-root-in-map t (len div 2) idx v path*)
case *None*
then show *?thesis*
using *True by simp*
next
case (*Some child*)
have *finite-child:*
finite (merkle-path-target-roots t child (len div 2) idx v path)
by *simp*
have *card-le:*
card ({rt} $\cup$
merkle-path-target-roots t child (len div 2) idx v path) $\leq$
Suc (card
(merkle-path-target-roots t child (len div 2) idx v path))
using *finite-child by (simp add: card-insert-if)*
also have *... $\leq$ Suc (Suc (length path))*
using *Cons.IH[of child len div 2 idx] by simp*
finally show *?thesis*
using *True Some by simp*
qed

```

next
case False
then show ?thesis
proof (cases
  merkle-path-root-in-map t (len div 2) (idx - len div 2) v path)
case None
then show ?thesis
  using False by simp
next
case (Some child)
have finite-child:
  finite (merkle-path-target-roots t child (len div 2)
    (idx - len div 2) v path)
  by simp
have card-le:
  card ({rt} ∪
    merkle-path-target-roots t child (len div 2)
    (idx - len div 2) v path) ≤
  Suc (card
    (merkle-path-target-roots t child (len div 2)
    (idx - len div 2) v path))
  using finite-child by (simp add: card-insert-if)
also have ... ≤ Suc (Suc (length path))
  using Cons.IH[of child len div 2 idx - len div 2] by simp
finally show ?thesis
  using False Some by simp
qed
qed
qed

lemma merkle-path-bound-pullback-or-new-output-hit:
  assumes ext:  $s \leq t$ 
  and bound: merkle-path-bound rt len idx v path t
  shows
    merkle-path-bound rt len idx v path s ∨
    hash-map-new-output-hit
    (merkle-path-target-roots t rt len idx v path) s t
  using bound
proof (induction path arbitrary: rt len idx)
case Nil
have lookup-t: fmlookup (HashMap t) (MerkleLeaf v) = Some rt
  using Nil.prem by simp
show ?case
proof (cases fmlookup (HashMap s) (MerkleLeaf v))
case None
then have
  hash-map-new-output-hit
    (merkle-path-target-roots t rt len idx v []) s t
  unfolding hash-map-new-output-hit-def

```

```

    using lookup-t by simp blast
  then show ?thesis by simp
next
case (Some old)
have lookup-t-old: fmlookup (HashMap t) (MerkleLeaf v) = Some old
  by (rule hash-extension-lookup[OF Some ext])
then have old = rt
  using lookup-t by simp
then have merkle-path-bound rt len idx v [] s
  using Some by simp
then show ?thesis by simp
qed
next
case (Cons sibling path)
show ?case
proof (cases idx < len div 2)
case True
from Cons.premis obtain child where child-bound:
  merkle-path-bound child (len div 2) idx v path t
and lookup-t:
  fmlookup (HashMap t) (MerkleNode child sibling) = Some rt
using True by auto
have child-root:
  merkle-path-root-in-map t (len div 2) idx v path = Some child
  by (rule merkle-path-bound-root-in-map[OF child-bound])
show ?thesis
proof (cases fmlookup (HashMap s) (MerkleNode child sibling))
case None
then have root-hit:
  hash-map-new-output-hit {rt} s t
  unfolding hash-map-new-output-hit-def
  using lookup-t by blast
have subset:
  {rt} ⊆
    merkle-path-target-roots t rt len idx v (sibling # path)
  using True child-root by simp
have
  hash-map-new-output-hit
    (merkle-path-target-roots t rt len idx v (sibling # path)) s t
  by (rule hash-map-new-output-hit-subset[OF subset root-hit])
then show ?thesis by simp
next
case (Some old)
have lookup-t-old:
  fmlookup (HashMap t) (MerkleNode child sibling) = Some old
  by (rule hash-extension-lookup[OF Some ext])
then have old-eq: old = rt
  using lookup-t by simp
have lookup-s:

```

```

    fmlookup (HashMap s) (MerkleNode child sibling) = Some rt
    using Some old-eq by simp
from Cons.IH[OF child-bound]
have child-pullback:
  merkle-path-bound child (len div 2) idx v path s ∨
  hash-map-new-output-hit
  (merkle-path-target-roots t child (len div 2) idx v path) s t
.
then show ?thesis
proof
  assume child-s:
    merkle-path-bound child (len div 2) idx v path s
  then have merkle-path-bound rt len idx v (sibling # path) s
    using True lookup-s by auto
  then show ?thesis by simp
next
  assume child-hit:
    hash-map-new-output-hit
    (merkle-path-target-roots t child (len div 2) idx v path) s t
  have subset:
    merkle-path-target-roots t child (len div 2) idx v path
    ⊆
    merkle-path-target-roots t rt len idx v (sibling # path)
  using True child-root by auto
  have
    hash-map-new-output-hit
    (merkle-path-target-roots t rt len idx v (sibling # path)) s t
  by (rule hash-map-new-output-hit-subset[OF subset child-hit])
  then show ?thesis by simp
qed
qed
next
case False
from Cons.premis obtain child where child-bound:
  merkle-path-bound child (len div 2) (idx - len div 2) v path t
and lookup-t:
  fmlookup (HashMap t) (MerkleNode sibling child) = Some rt
using False by auto
have child-root:
  merkle-path-root-in-map t (len div 2) (idx - len div 2) v path =
  Some child
by (rule merkle-path-bound-root-in-map[OF child-bound])
show ?thesis
proof (cases fmlookup (HashMap s) (MerkleNode sibling child))
case None
  then have root-hit:
    hash-map-new-output-hit {rt} s t
  unfolding hash-map-new-output-hit-def
  using lookup-t by blast

```

```

have subset:
  {rt}  $\subseteq$ 
    merkle-path-target-roots t rt len idx v (sibling # path)
  using False child-root by simp
have
  hash-map-new-output-hit
    (merkle-path-target-roots t rt len idx v (sibling # path)) s t
  by (rule hash-map-new-output-hit-subset[OF subset root-hit])
then show ?thesis by simp
next
case (Some old)
have lookup-t-old:
  fmllookup (HashMap t) (MerkleNode sibling child) = Some old
  by (rule hash-extension-lookup[OF Some ext])
then have old-eq: old = rt
  using lookup-t by simp
have lookup-s:
  fmllookup (HashMap s) (MerkleNode sibling child) = Some rt
  using Some old-eq by simp
from Cons.IH[OF child-bound]
have child-pullback:
  merkle-path-bound child (len div 2) (idx - len div 2) v path s  $\vee$ 
    hash-map-new-output-hit
      (merkle-path-target-roots t child (len div 2)
        (idx - len div 2) v path) s t
  .
then show ?thesis
proof
  assume child-s:
    merkle-path-bound child (len div 2) (idx - len div 2) v path s
  then have merkle-path-bound rt len idx v (sibling # path) s
    using False lookup-s by auto
  then show ?thesis by simp
next
  assume child-hit:
    hash-map-new-output-hit
      (merkle-path-target-roots t child (len div 2)
        (idx - len div 2) v path) s t
  have subset:
    merkle-path-target-roots t child (len div 2)
      (idx - len div 2) v path
     $\subseteq$ 
    merkle-path-target-roots t rt len idx v (sibling # path)
  using False child-root by auto
have
  hash-map-new-output-hit
    (merkle-path-target-roots t rt len idx v (sibling # path)) s t
  by (rule hash-map-new-output-hit-subset[OF subset child-hit])
then show ?thesis by simp

```

qed
 qed
 qed
 qed

lemma *authenticated-opening-pullback-or-new-output-hit*:

assumes *ext*: $s \leq t$

and *auth*: *authenticated-opening-in* *t* *opening*

shows

authenticated-opening-in *s* *opening* $\vee$
hash-map-new-output-hit
 (*merkle-path-target-roots* *t*
 (*opening-root* *opening*)
 (*opening-length* *opening*)
 (*opening-index* *opening*)
 (*opening-value* *opening*)
 (*opening-path* *opening*)) *s* *t*

proof —

have *idx-bound*: *opening-index* *opening* < *opening-length* *opening*

using *auth* **unfolding** *authenticated-opening-in-def* **by** *simp*

have *path-len*:

length (*opening-path* *opening*) =
floor-log (*opening-length* *opening*)

using *auth* **unfolding** *authenticated-opening-in-def* **by** *simp*

have *bound-t*:

merkle-path-bound
 (*opening-root* *opening*)
 (*opening-length* *opening*)
 (*opening-index* *opening*)
 (*opening-value* *opening*)
 (*opening-path* *opening*) *t*

using *auth* **unfolding** *authenticated-opening-in-def* **by** *simp*

from *merkle-path-bound-pullback-or-new-output-hit*[*OF ext bound-t*]

show *?thesis*

proof

assume *bound-s*:

merkle-path-bound
 (*opening-root* *opening*)
 (*opening-length* *opening*)
 (*opening-index* *opening*)
 (*opening-value* *opening*)
 (*opening-path* *opening*) *s*

have *authenticated-opening-in* *s* *opening*

unfolding *authenticated-opening-in-def*

using *idx-bound path-len bound-s* **by** *simp*

then show *?thesis* **by** *simp*

next

assume

hash-map-new-output-hit


```

      (merkle-path-target-roots t
       (opening-root opening)
       (opening-length opening)
       (opening-index opening)
       (opening-value opening)
       (opening-path opening)) s t
    then show ?thesis by simp
  qed
qed

lemma partial-authenticated-table-opening-pullback-or-new-output-hit:
  assumes ext:  $s \leq t$ 
  and table: partial-authenticated-table rt len openings t
  and opn-in: opening  $\in$  set openings
  shows
    authenticated-opening-in s opening  $\vee$ 
    hash-map-new-output-hit
      (merkle-path-target-roots t rt len
       (opening-index opening)
       (opening-value opening)
       (opening-path opening)) s t
  proof -
    have auth: authenticated-opening-in t opening
      using table opn-in unfolding partial-authenticated-table-def by blast
    have root-eq: opening-root opening = rt
      using table opn-in unfolding partial-authenticated-table-def by blast
    have len-eq: opening-length opening = len
      using table opn-in unfolding partial-authenticated-table-def by blast
    show ?thesis
      using authenticated-opening-pullback-or-new-output-hit[OF ext auth]
      unfolding root-eq len-eq .
  qed

lemma partial-authenticated-table-pullback-or-new-output-hit:
  assumes ext:  $s \leq t$ 
  and table: partial-authenticated-table rt len openings t
  shows
    partial-authenticated-table rt len openings s  $\vee$ 
    ( $\exists$  opening  $\in$  set openings.
     hash-map-new-output-hit
       (merkle-path-target-roots t rt len
        (opening-index opening)
        (opening-value opening)
        (opening-path opening)) s t)
  proof (cases
     $\exists$  opening  $\in$  set openings.
    hash-map-new-output-hit
      (merkle-path-target-roots t rt len
       (opening-index opening)

```

```

      (opening-value opening)
      (opening-path opening)) s t)
case True
then show ?thesis by simp
next
case False
have table-s:
  partial-authenticated-table rt len openings s
  unfolding partial-authenticated-table-def
proof
  fix opn
  assume opn-in: opn ∈ set openings
  have root-eq: opening-root opn = rt
    using table opn-in unfolding partial-authenticated-table-def by blast
  have len-eq: opening-length opn = len
    using table opn-in unfolding partial-authenticated-table-def by blast
  have auth-s: authenticated-opening-in s opn
  proof -
    have pullback:
      authenticated-opening-in s opn ∨
      hash-map-new-output-hit
      (merkle-path-target-roots t rt len
       (opening-index opn)
       (opening-value opn)
       (opening-path opn)) s t
    by (rule partial-authenticated-table-opening-pullback-or-new-output-hit
        [OF ext table opn-in])
  then show ?thesis
    using False opn-in by blast
qed
show
  opening-root opn = rt ∧
  opening-length opn = len ∧
  authenticated-opening-in s opn
  using root-eq len-eq auth-s by simp
qed
then show ?thesis by simp
qed

lemma protocol-created-tree-get-authentication-path-len:
  assumes created: protocol-created-tree xs tree s
  and pos: 0 < lgth
  and le: lgth ≤ length xs
  shows length (get-authentication-path lgth idx tree) = floor-log lgth
  using assms
proof (induction xs arbitrary: tree lgth idx rule: measure-induct-rule[of length])
  case (less xs)
  show ?case
  proof (cases xs)

```

```

case Nil
then show ?thesis
  using less.prem1 by simp
next
case (Cons x xs)
show ?thesis
proof (cases xs)
  case Nil
  then have lgth-eq: lgth = 1
    using less.prem1 Cons by simp
  obtain h where tree-eq: tree = ⟨⟨⟩, h, ⟨⟩⟩
    using protocol-created-tree-singletonD[OF less.prem1][unfolded Cons Nil]
    by blast
  show ?thesis
    unfolding tree-eq lgth-eq by (simp add: floor-log-Suc-zero)
next
case (Cons y zs)
let ?mid = length xs div 2
have xs-eq: xs = x # y # zs
  using ⟨xs = x # ys⟩ ⟨ys = y # zs⟩ by simp
have len-ge: 2 ≤ length xs
  unfolding xs-eq by simp
obtain l h r i where i-eq: i = ?mid
  and tree-eq: tree = ⟨l, h, r⟩
  and left-i: protocol-created-tree (take i xs) l s
  and right-i: protocol-created-tree (drop i xs) r s
  and root-lookup:
    fmlookup (HashMap s) (MerkleNode (value l) (value r)) =
      Some h
  by (rule protocol-created-tree-internalD[OF less.prem1 len-ge])
have left: protocol-created-tree (take ?mid xs) l s
  using left-i i-eq by simp
have right: protocol-created-tree (drop ?mid xs) r s
  using right-i i-eq by simp
show ?thesis
proof (cases lgth = 1)
  case True
  then show ?thesis
    unfolding tree-eq by (simp add: floor-log-Suc-zero)
next
case False
note lgth-ne-one = False
have lgth-ge: 2 ≤ lgth
  using lgth-ne-one less.prem1 by simp
have half-pos: 0 < lgth div 2
  using lgth-ge by simp
have take-less: length (take ?mid xs) < length xs
  using len-ge by simp
have drop-less: length (drop ?mid xs) < length xs

```

```

    using len-ge by simp
  have half-le-take:  $\text{lgth div } 2 \leq \text{length (take ?mid xs)}$ 
    using less.premis(3) by simp
  have half-le-drop:  $\text{lgth div } 2 \leq \text{length (drop ?mid xs)}$ 
    using less.premis(3) len-ge by simp
  have left-len:
     $\text{length (get-authentication-path (lgth div 2) idx l)} =$ 
     $\text{floor-log (lgth div 2)}$ 
    by (rule less.IH[OF take-less left half-pos half-le-take])
  have right-len:
     $\text{length (get-authentication-path (lgth div 2)}$ 
     $(\text{idx} - \text{lgth div } 2) \text{ r}) =$ 
     $\text{floor-log (lgth div 2)}$ 
    by (rule less.IH[OF drop-less right half-pos half-le-drop])
  have floor-eq:  $\text{floor-log lgth} = \text{Suc (floor-log (lgth div 2))}$ 
    using floor-log-rec[OF lgth-ge] .
  show ?thesis
  proof (cases  $\text{idx} < \text{lgth div } 2$ )
    case True
    then have  $\text{length (get-authentication-path lgth idx tree)} =$ 
       $\text{Suc (floor-log (lgth div 2))}$ 
      using lgth-ne-one left-len unfolding tree-eq by simp
    then show ?thesis
      by (simp only: floor-eq)
    next
    case False
    then have  $\text{length (get-authentication-path lgth idx tree)} =$ 
       $\text{Suc (floor-log (lgth div 2))}$ 
      using lgth-ne-one right-len unfolding tree-eq by simp
    then show ?thesis
      by (simp only: floor-eq)
  qed
qed
qed
qed
qed
qed

```

lemma *created-tree-get-authentication-path-len:*
assumes *created: created-tree xs tree s*
and *xs-nonempty: xs $\neq$ []*
shows $\text{length (get-authentication-path (length xs) idx tree)} =$
 $\text{floor-log (length xs)}$
by (rule *protocol-created-tree-get-authentication-path-len*
 [OF *created[unfolded created-tree-def]*])
 (use *xs-nonempty* in *simp-all*)

lemma *merkle-root-binds-table-authenticated-openings:*
assumes *bind: merkle-root-binds-table rt table final-state*
and *len-table: length table = scale * clength*

```

    and idxs-bound:  $\bigwedge idx. idx \in \text{set } idxs \implies idx < \text{scale} * \text{clength}$ 
  obtains openings where
    map opening-index openings = idxs
    map opening-value openings = map (! table) idxs
    partial-authenticated-table rt (scale * clength) openings final-state
proof -
  obtain tree where created:
    created-tree table tree final-state
  and rt-eq: rt = value tree
  using bind unfolding merkle-root-binds-table-def by blast
  let ?opening =  $\lambda idx.$ 
    ( $\mid$  opening-root = rt,
     opening-length = scale * clength,
     opening-index = idx,
     opening-value = table ! idx,
     opening-path = get-authentication-path (length table) idx tree  $\mid$ )
  let ?openings = map ?opening idxs
  have auth-all:
     $\forall \text{opn} \in \text{set } ?\text{openings}.$ 
      opening-root opn = rt  $\wedge$ 
      opening-length opn = scale * clength  $\wedge$ 
      authenticated-opening-in final-state opn
  proof
    fix opn
    assume opn-in: opn  $\in$  set ?openings
    then obtain idx where idx-in: idx  $\in$  set idxs
      and opn-eq: opn = ?opening idx
      by auto
    have idx-bound-len: idx < length table
      using idxs-bound[OF idx-in] len-table by simp
    have idx-bound-domain: idx < scale * clength
      by (rule idxs-bound[OF idx-in])
    obtain n where len-pow: length table =  $2^n$ 
      using len-table eval-domain-length-power
      by (metis mult.commute)
    have root-in-map:
      merkle-path-root-in-map final-state (length table) idx
        (table ! idx) (get-authentication-path (length table) idx tree) =
        Some rt
      using protocol-created-tree-path-root-in-map
        [OF created[unfolded created-tree-def] len-pow idx-bound-len]
      unfolding rt-eq .
    have path-bound:
      merkle-path-bound rt (length table) idx (table ! idx)
        (get-authentication-path (length table) idx tree) final-state
      by (rule merkle-path-root-in-map-some-imp-bound[OF root-in-map])
    have path-len:
      length (get-authentication-path (length table) idx tree) =
      floor-log (scale * clength)

```

```

proof –
  have table-nonempty: table ≠ []
    using idx-bound-len by auto
  show ?thesis
    using created-tree-get-authentication-path-len[OF created table-nonempty,
      of idx] len-table
    by simp
qed
have auth: authenticated-opening-in final-state opn
  unfolding authenticated-opening-in-def opn-eq
  using idx-bound-domain path-len path-bound len-table by simp
show opening-root opn = rt ∧
  opening-length opn = scale * clength ∧
  authenticated-opening-in final-state opn
  using auth unfolding opn-eq by simp
qed
have partial-table:
  partial-authenticated-table rt (scale * clength) ?openings
  final-state
  using auth-all unfolding partial-authenticated-table-def by blast
have indices: map opening-index ?openings = idxs
  by (induction idxs) simp-all
have value-map: map opening-value ?openings = map (!) table idxs
  by (induction idxs) simp-all
show ?thesis
  by (rule that[OF indices value-map partial-table])
qed

lemma authenticated-opening-root-in-map:
  assumes authenticated-opening-in s opening
  shows
    merkle-path-root-in-map s
    (opening-length opening)
    (opening-index opening)
    (opening-value opening)
    (opening-path opening) =
    Some (opening-root opening)

proof –
  have bound:
    merkle-path-bound
    (opening-root opening)
    (opening-length opening)
    (opening-index opening)
    (opening-value opening)
    (opening-path opening) s
  using assms unfolding authenticated-opening-in-def by simp
  show ?thesis
  by (rule merkle-path-bound-root-in-map[OF bound])
qed

```

```

lemma check-authentication-path-outcome-bound:
  fixes  $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$ 
  assumes outcome:
     $\text{Some}\ (rt, t) \in$ 
       $\text{set-dist}\ (\text{execute}\ (\text{check-authentication-path}\ \text{len}\ \text{idx}\ v\ \text{path})\ s)$ 
  shows merkle-path-bound  $rt\ \text{len}\ \text{idx}\ v\ \text{path}\ t$ 
  using outcome
proof (induction path arbitrary: rt len idx s t)
  case Nil
  have hash-out:
     $\text{Some}\ (rt, t) \in$ 
       $\text{set-dist}$ 
       $(\text{execute}$ 
         $(\text{hash}\ (\text{MerkleLeaf}\ v) ::$ 
           $('f, ('f, 'a)\ \text{protocol-channel-scheme})\ \text{state-monad})\ s)$ 
       $)$ 
  using Nil.prems
  unfolding check-authentication-path-def
    protocol-check-authentication-path-def
  by simp
  show  $?case$  using hash-outcome(2)[OF hash-out] by simp
next
  case (Cons sibling path)
  show  $?case$ 
  proof (cases idx < len div 2)
  case True
  from Cons.prems obtain child u where
    rec-out:
       $\text{Some}\ (\text{child}, u) \in$ 
         $\text{set-dist}$ 
         $(\text{execute}$ 
           $(\text{check-authentication-path}\ (\text{len}\ \text{div}\ 2)\ \text{idx}\ v\ \text{path})\ s)$ 
         $)$ 
  and hash-out:
     $\text{Some}\ (rt, t) \in$ 
       $\text{set-dist}$ 
       $(\text{execute}$ 
         $(\text{hash}\ (\text{MerkleNode}\ \text{child}\ \text{sibling}) ::$ 
           $('f, ('f, 'a)\ \text{protocol-channel-scheme})\ \text{state-monad})\ u)$ 
       $)$ 
  unfolding check-authentication-path-def
    protocol-check-authentication-path-def
  using True
  by (auto simp: protocol-merkle.check-authentication-path.simps
    elim!: set-dist-bindE)
  have child-u:
    merkle-path-bound child (len div 2) idx v path u
    by (rule Cons.IH[OF rec-out])
  have ext:  $u \leq t$ 
    by (rule hash-outcome(1)[OF hash-out])
  have child-t:

```

```

    merkle-path-bound child (len div 2) idx v path t
  by (rule merkle-path-bound-mono[OF child-u ext])
have lookup:
  fmlookup (HashMap t) (MerkleNode child sibling) = Some rt
  by (rule hash-outcome(2)[OF hash-out])
show ?thesis
  using True child-t lookup
  by (auto intro!: exI[of - child])
next
case False
from Cons.premis obtain child u where
  rec-out:
    Some (child, u) ∈
      set-dist
        (execute
          (check-authentication-path
            (len div 2) (idx - len div 2) v path) s)
  and hash-out:
    Some (rt, t) ∈
      set-dist
        (execute
          (hash (MerkleNode sibling child) ::
            ('f, ('f, 'a) protocol-channel-scheme) state-monad) u)
  unfolding check-authentication-path-def
    protocol-check-authentication-path-def
  using False
  by (auto simp: protocol-merkle.check-authentication-path.simps
    elim!: set-dist-bindE)
have child-u:
  merkle-path-bound child (len div 2) (idx - len div 2)
    v path u
  by (rule Cons.IH[OF rec-out])
have ext:  $u \leq t$ 
  by (rule hash-outcome(1)[OF hash-out])
have child-t:
  merkle-path-bound child (len div 2) (idx - len div 2)
    v path t
  by (rule merkle-path-bound-mono[OF child-u ext])
have lookup:
  fmlookup (HashMap t) (MerkleNode sibling child) = Some rt
  by (rule hash-outcome(2)[OF hash-out])
show ?thesis
  using False child-t lookup
  by (auto intro!: exI[of - child])
qed
qed

```

lemma *query-decommitment-step-authenticated-opening:*
fixes $s\ t :: ('f, 'a)\text{ protocol-channel-scheme}$


```

assumes idx-bound:  $idx < scale * clength$ 
and outcome:
  Some (qh, t)  $\in$ 
    set-dist (execute (query-decommitment-step rt idx) s)
obtains path opn where
  opn =
    ( $\langle$ opening-root = rt,
     opening-length =  $scale * clength$ ,
     opening-index = idx,
     opening-value = qh,
     opening-path = path $\rangle$ )
  authenticated-opening-in t opn
proof –
  let ?len =  $scale * clength$ 
  from outcome obtain s1 path s2 ap s3 s4 where
    read-gh:
      Some (qh, s1)  $\in$  set-dist (execute read s)
    and read-path:
      Some (path, s2)  $\in$ 
        set-dist (execute (ntimes read (floor-log ?len)) s1)
    and check:
      Some (ap, s3)  $\in$ 
        set-dist
          (execute (check-authentication-path ?len idx qh path) s2)
    and assert-ap:
      Some ( $\langle$ , s4)  $\in$  set-dist (execute (assert (ap = rt)) s3)
    and ret:
      Some (qh, t)  $\in$  set-dist (execute (return qh) s4)
    unfolding query-decommitment-step-def
    by (auto simp: Let-def elim!: set-dist-bindE)
  have path-length:  $length\ path = floor-log\ ?len$ 
    using ntimes-read-any-outcome[OF read-path] by simp
  have ap-eq: ap = rt
    using assert-ap unfolding assert-def
    by (cases ap = rt) (auto simp: throw-no-outcome)
  have s4: s4 = s3
    using assert-ap ap-eq unfolding assert-def by simp
  have t: t = s3
    using ret unfolding s4 by simp
  have path-bound:
    merkle-path-bound rt ?len idx qh path t
  proof –
    have merkle-path-bound ap ?len idx qh path s3
      by (rule check-authentication-path-outcome-bound[OF check])
    then show ?thesis unfolding ap-eq t .
  qed
  let ?opn =
    ( $\langle$ opening-root = rt,
     opening-length = ?len,

```

```

    opening-index = idx,
    opening-value = qh,
    opening-path = path\}
have authenticated: authenticated-opening-in t ?opn
unfolding authenticated-opening-in-def
using idx-bound path-length path-bound by simp
show ?thesis
by (rule that[of ?opn path]) (simp-all add: authenticated)
qed

lemma mmap-query-decommitment-steps-authenticated-openings:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes idxs-bound:  $\bigwedge idx. idx \in \text{set } idxs \implies idx < \text{scale} * \text{clength}$ 
and outcome:
  Some (leaves, t)  $\in$ 
    set-dist (execute (mmap (map (query-decommitment-step rt) idxs)) s)
shows
   $\exists$  openings.
    length openings = length idxs  $\wedge$ 
    map opening-value openings = leaves  $\wedge$ 
    map opening-index openings = idxs  $\wedge$ 
    partial-authenticated-table rt (scale * clength) openings t
using outcome idxs-bound
proof (induction idxs arbitrary: s leaves t)
case Nil
then have leaves = [] and t = s
by simp-all
moreover have
  partial-authenticated-table rt (scale * clength) [] t
unfolding partial-authenticated-table-def by simp
ultimately show ?case
by (intro exI[of - []]) simp
next
case (Cons idx idxs)
from Cons.prem1 obtain leaf leaves' s1 where
  head:
    Some (leaf, s1)  $\in$ 
      set-dist (execute (query-decommitment-step rt idx) s)
and tail:
    Some (leaves', t)  $\in$ 
      set-dist (execute (mmap (map (query-decommitment-step rt) idxs)) s1)
and leaves-eq: leaves = leaf # leaves'
by (auto elim!: set-dist-bindE)
have idx-bound: idx < scale * clength
by (rule Cons.prem2) simp
from query-decommitment-step-authenticated-opening[OF idx-bound head]
obtain path opn where
  opn-eq:
    opn =

```

```

    (opening-root = rt,
     opening-length = scale * clength,
     opening-index = idx,
     opening-value = leaf,
     opening-path = path)
  and opn-auth-s1: authenticated-opening-in s1 opn
  by blast
have tail-bound:
   $\bigwedge idx. idx \in \text{set } idxs \implies idx < \text{scale} * \text{clength}$ 
  by (rule Cons.premis(2)) simp
from Cons.IH[OF tail tail-bound] obtain openings where
  len-openings: length openings = length idxs
  and values-openings: map opening-value openings = leaves'
  and idx-openings: map opening-index openings = idxs
  and table-tail:
    partial-authenticated-table rt (scale * clength) openings t
  by blast
have s1-t: s1  $\leq$  t
  using mmap-query-decommitment-steps-outcome[OF tail] by blast
have opn-auth-t: authenticated-opening-in t opn
  by (rule authenticated-opening-in-mono[OF opn-auth-s1 s1-t])
let ?openings = opn # openings
have table-all:
  partial-authenticated-table rt (scale * clength) ?openings t
  using opn-auth-t table-tail opn-eq
  unfolding partial-authenticated-table-def by simp
show ?case
  using len-openings values-openings idx-openings leaves-eq opn-eq table-all
  by (intro exI[of - ?openings]) simp
qed

```

```

lemma check-decommit-on-query-authenticated-openings:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes idx-in: idx  $\in$  query-sample-space
  and outcome:
    Some (leaves, t)  $\in$ 
      set-dist (execute (mmap (check-decommit-on-query rt idx)) s)
  obtains openings where
    length openings = length (powers-scaled idx)
    map opening-value openings = leaves
    map opening-index openings = powers-scaled idx
    partial-authenticated-table rt (scale * clength) openings t
proof -
  have map-eq:
    check-decommit-on-query rt idx =
      map (query-decommitment-step rt) (powers-scaled idx)
  unfolding check-decommit-on-query-def query-decommitment-step-def by simp
  have idxs-bound:
     $\bigwedge i. i \in \text{set } (\text{powers-scaled } idx) \implies i < \text{scale} * \text{clength}$ 

```

```

using query-sample-space-powers-scaled-bound[OF idx-in]
by (simp add: mult.commute)
from mmap-query-decommitment-steps-authenticated-openings
  [OF idxs-bound outcome[unfolded map-eq]]
obtain openings where
  length openings = length (powers-scaled idx)
  map opening-value openings = leaves
  map opening-index openings = powers-scaled idx
  partial-authenticated-table rt (scale * clength) openings t
by blast
then show ?thesis by (rule that)
qed

lemma fri-layer-opening-finish-authenticated-openings:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes idx-bound: idx < len
and sibling-bound: (idx + len div 2) mod len < len
and xp-path-length: length xp-path = floor-log len
and xn-path-length: length xn-path = floor-log len
and outcome:
  Some (out, t) ∈
    set-dist
      (execute
        (fri-layer-opening-finish b rt idx x len pw
          xp xp-path xn xn-path) s)
obtains xp-opening xn-opening where
  xp-opening =
    (|opening-root = rt,
      opening-length = len,
      opening-index = idx,
      opening-value = xp,
      opening-path = xp-path|)
  xn-opening =
    (|opening-root = rt,
      opening-length = len,
      opening-index = (idx + len div 2) mod len,
      opening-value = xn,
      opening-path = xn-path|)
  authenticated-opening-in t xp-opening
  authenticated-opening-in t xn-opening
proof –
from outcome obtain s1 ap1 s2 s3 ap2 s4 s5 where
  assert-xp:
    Some ((), s1) ∈ set-dist (execute (assert (xp = x)) s)
and check-xp:
  Some (ap1, s2) ∈
    set-dist
      (execute (check-authentication-path len idx xp xp-path) s1)
and assert-ap1:

```

```

    Some ( $()$ ,  $s3$ )  $\in$  set-dist (execute (assert ( $ap1 = rt$ ))  $s2$ )
and check-xn:
    Some ( $ap2$ ,  $s4$ )  $\in$ 
      set-dist
      (execute
        (check-authentication-path len
          (( $idx + len \text{ div } 2$ ) mod len)  $xn$   $xn\text{-path}$ )  $s3$ )
and assert-ap2:
    Some ( $()$ ,  $s5$ )  $\in$  set-dist (execute (assert ( $ap2 = rt$ ))  $s4$ )
and ret:
    Some ( $out$ ,  $t$ )  $\in$ 
      set-dist
      (execute
        (return
          ( $idx \text{ mod } (len \text{ div } 2)$ ,
            ( $xp + xn$ ) div 2 +
               $b * ((xp - xn) \text{ div } (2 * ((h \wedge idx) * shift) \wedge pw))$ ,
             $len \text{ div } 2$ ,  $pw + pw$ )  $s5$ )
      unfolding fri-layer-opening-finish-def
      by (auto simp: Let-def elim!: set-dist-bindE)
have ap1-eq:  $ap1 = rt$ 
  using assert-ap1 unfolding assert-def
  by (cases  $ap1 = rt$ ) (auto simp: throw-no-outcome)
have ap2-eq:  $ap2 = rt$ 
  using assert-ap2 unfolding assert-def
  by (cases  $ap2 = rt$ ) (auto simp: throw-no-outcome)
have xp-eq:  $xp = x$ 
  using assert-xp unfolding assert-def
  by (cases  $xp = x$ ) (auto simp: throw-no-outcome)
have s1:  $s1 = s$ 
  using assert-xp xp-eq unfolding assert-def by simp
have s3:  $s3 = s2$ 
  using assert-ap1 ap1-eq unfolding assert-def by simp
have s5:  $s5 = s4$ 
  using assert-ap2 ap2-eq unfolding assert-def by simp
have t:  $t = s4$ 
  using ret unfolding s5 by simp
have xp-bound-s2:
  merkle-path-bound  $rt$  len  $idx$   $xp$   $xp\text{-path}$   $s2$ 
proof -
  have merkle-path-bound  $ap1$  len  $idx$   $xp$   $xp\text{-path}$   $s2$ 
    by (rule check-authentication-path-outcome-bound[OF check-xp])
  then show ?thesis unfolding ap1-eq .
qed
have s2-s4:  $s2 \leq s4$ 
  unfolding s3[symmetric]
  by (rule check-authentication-path-hash-extends[OF check-xn])
have xp-bound:

```

```

merkle-path-bound rt len idx xp xp-path t
unfolding t
by (rule merkle-path-bound-mono[OF xp-bound-s2 s2-s4])
have xn-bound:
merkle-path-bound rt len ((idx + len div 2) mod len)
  xn xn-path t
proof -
  have merkle-path-bound ap2 len ((idx + len div 2) mod len)
    xn xn-path s4
  by (rule check-authentication-path-outcome-bound[OF check-xn])
  then show ?thesis unfolding ap2-eq t .
qed
let ?xp-opening =
  (opening-root = rt,
   opening-length = len,
   opening-index = idx,
   opening-value = xp,
   opening-path = xp-path)
let ?xn-opening =
  (opening-root = rt,
   opening-length = len,
   opening-index = (idx + len div 2) mod len,
   opening-value = xn,
   opening-path = xn-path)
have xp-authenticated: authenticated-opening-in t ?xp-opening
unfolding authenticated-opening-in-def
using idx-bound xp-path-length xp-bound by simp
have xn-authenticated: authenticated-opening-in t ?xn-opening
unfolding authenticated-opening-in-def
using sibling-bound xn-path-length xn-bound by simp
show ?thesis
by (rule that[of ?xp-opening ?xn-opening])
  (simp-all add: xp-authenticated xn-authenticated)
qed

lemma fri-layer-opening-step-authenticated-openings:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes len-pos: 0 < len
and idx-bound: idx < len
and outcome:
  Some (out, t) ∈
    set-dist
    (execute (fri-layer-opening-step (b, rt) (idx, x, len, pw)) s)
obtains xp-opening xn-opening where
  opening-root xp-opening = rt
  opening-length xp-opening = len
  opening-index xp-opening = idx
  opening-root xn-opening = rt
  opening-length xn-opening = len

```

$opening-index\ xn-opening = (idx + len \div 2) \bmod len$
 $authenticated-opening-in\ t\ xp-opening$
 $authenticated-opening-in\ t\ xn-opening$

proof –

from *outcome[unfolding fri-layer-opening-step-unfold-finish]*

obtain $xp\ s1\ xp-path\ s2\ xn\ s3\ xn-path\ s4$ **where**

$read-xp: Some\ (xp, s1) \in set-dist\ (execute\ read\ s)$

and $read-xp-path:$

$Some\ (xp-path, s2) \in$

$set-dist\ (execute\ (ntimes\ read\ (floor-log\ len))\ s1)$

and $read-xn: Some\ (xn, s3) \in set-dist\ (execute\ read\ s2)$

and $read-xn-path:$

$Some\ (xn-path, s4) \in$

$set-dist\ (execute\ (ntimes\ read\ (floor-log\ len))\ s3)$

and *finish:*

$Some\ (out, t) \in$

$set-dist$

$(execute$

$(fri-layer-opening-finish\ b\ rt\ idx\ x\ len\ pw$

$xp\ xp-path\ xn\ xn-path)\ s4)$

by *(auto elim!: set-dist-bindE)*

have $xp-path-length: length\ xp-path = floor-log\ len$

using $ntimes-read-any-outcome[OF\ read-xp-path]$ **by** *simp*

have $xn-path-length: length\ xn-path = floor-log\ len$

using $ntimes-read-any-outcome[OF\ read-xn-path]$ **by** *simp*

have $sibling-bound: (idx + len \div 2) \bmod len < len$

using $len-pos$ **by** *simp*

from *fri-layer-opening-finish-authenticated-openings*

$[OF\ idx-bound\ sibling-bound\ xp-path-length\ xn-path-length\ finish]$

obtain $xp-opening\ xn-opening$ **where**

$xp-eq:$

$xp-opening =$

$(\parallel opening-root = rt,$

$opening-length = len,$

$opening-index = idx,$

$opening-value = xp,$

$opening-path = xp-path\parallel)$

and $xn-eq:$

$xn-opening =$

$(\parallel opening-root = rt,$

$opening-length = len,$

$opening-index = (idx + len \div 2) \bmod len,$

$opening-value = xn,$

$opening-path = xn-path\parallel)$

and $xp-auth: authenticated-opening-in\ t\ xp-opening$

and $xn-auth: authenticated-opening-in\ t\ xn-opening$

by *blast*

show *?thesis*

by *(rule that[of xp-opening xn-opening])*

(use *xp-eq xn-eq xp-auth xn-auth* in *simp-all*)

qed

lemma *fri-layer-opening-step-partial-authenticated-table*:

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes *len-pos*: $0 < \text{len}$

and *idx-bound*: $\text{idx} < \text{len}$

and *outcome*:

Some (*out*, *t*) $\in$

set-dist

(*execute* (*fri-layer-opening-step* (*b*, *rt*) (*idx*, *x*, *len*, *pw*)) *s*)

obtains *openings* **where**

partial-authenticated-table *rt len openings t*

map opening-index openings = $[\text{idx}, (\text{idx} + \text{len} \text{ div } 2) \bmod \text{len}]$

length openings = 2

proof –

from *fri-layer-opening-step-authenticated-openings*

[*OF len-pos idx-bound outcome*]

obtain *xp-opening xn-opening* **where**

xp-root: *opening-root xp-opening* = *rt*

and *xp-len*: *opening-length xp-opening* = *len*

and *xp-idx*: *opening-index xp-opening* = *idx*

and *xn-root*: *opening-root xn-opening* = *rt*

and *xn-len*: *opening-length xn-opening* = *len*

and *xn-idx*:

opening-index xn-opening = $(\text{idx} + \text{len} \text{ div } 2) \bmod \text{len}$

and *xp-auth*: *authenticated-opening-in t xp-opening*

and *xn-auth*: *authenticated-opening-in t xn-opening*

by *blast*

let *?openings* = [*xp-opening*, *xn-opening*]

have *table*: *partial-authenticated-table rt len ?openings t*

unfolding *partial-authenticated-table-def*

using *xp-root xp-len xp-auth xn-root xn-len xn-auth* **by** *simp*

have *indices*:

map opening-index ?openings = $[\text{idx}, (\text{idx} + \text{len} \text{ div } 2) \bmod \text{len}]$

using *xp-idx xn-idx* **by** *simp*

show *?thesis*

by (*rule that*[*OF table indices*]) *simp*

qed

lemma *verifier-query-round-after-index-authenticated-trace-openings*:

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes *outcome*:

Some (*()*, *t*) $\in$

set-dist

(*execute*

verifier-query-round-after-index-program

fr f-fl f-final as fl final raw) *s*)

obtains *idx leaves openings* **where**


```

  idx = index (to-nat raw)
  length openings = length (powers-scaled idx)
  map opening-value openings = leaves
  map opening-index openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) openings t
proof –
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1) ∈
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
  and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some ((), s3) ∈
      set-dist
        (execute
          (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
          clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some ((), t) ∈
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have idx-in: ?idx ∈ query-sample-space
  using index-less-query-sample-space
  unfolding query-sample-space-def by simp
from check-decommit-on-query-authenticated-openings
  [OF idx-in query-decommit]
obtain openings where
  len-openings: length openings = length (powers-scaled ?idx)
  and values-openings: map opening-value openings = fv
  and idx-openings: map opening-index openings = powers-scaled ?idx
  and table-s1:
    partial-authenticated-table fr (scale * clength) openings s1
  by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  s1-s2: s1 ≤ s2

```

```

    by blast
  have s3-eq: s3 = s2
    using assert-trace unfolding assert-def
    by (cases case f-out of (-, f-x, -, -)  $\Rightarrow$  f-x = f-final)
      (auto simp: throw-no-outcome)
  from receive-query-commits-layers-outcome[OF comp-fri]
  obtain comp-layer-chunks comp-fri-chunk where
    s3-s4: s3  $\leq$  s4
    by blast
  have t-eq: t = s4
    using assert-comp unfolding assert-def
    by (cases case c-out of (-, x, -, -)  $\Rightarrow$  x = final)
      (auto simp: throw-no-outcome)
  have s1-t: s1  $\leq$  t
    using s1-s2 s3-s4 unfolding s3-eq t-eq
    by (meson hash-ext-trans)
  have table-t:
    partial-authenticated-table fr (scale * clength) openings t
    by (rule partial-authenticated-table-mono[OF table-s1 s1-t])
  show ?thesis
    by (rule that[OF refl len-openings values-openings idx-openings table-t])
qed

```

lemma verifier-query-round-after-index-authenticated-composition-openings:

```

  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-eq: fl = (b, composition-root) # fl-tail
  and outcome:
    Some (( ), t)  $\in$ 
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s)
  obtains idx openings where
    idx = index (to-nat raw)
    partial-authenticated-table composition-root (scale * clength)
      openings t
    map opening-index openings =
      [idx, fri-sibling-index (scale * clength) idx]
    length openings = 2

```

proof –

```

  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1)  $\in$ 
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
  and trace-fri:
    Some (f-out, s2)  $\in$ 
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)

```

```

    (receive-query-commits f-fl)) s1)
and assert-trace:
  Some ((), s3) ∈
    set-dist
      (execute
        (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
and comp-fri:
  Some (c-out, s4) ∈
    set-dist (execute
      (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
        clength * scale, 1)
        (receive-query-commits fl)) s3)
and assert-comp:
  Some ((), t) ∈
    set-dist
      (execute
        (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
unfolding verifier-query-round-after-index-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have comp-steps:
  receive-query-commits fl =
    fri-layer-opening-step (b, composition-root) #
      receive-query-commits fl-tail
unfolding fl-eq receive-query-commits-def fri-layer-opening-step-def
by simp
from comp-fri[unfolded comp-steps]
obtain out1 s-head where
  head:
    Some (out1, s-head) ∈
      set-dist
        (execute
          (fri-layer-opening-step (b, composition-root)
            (?idx, cp-eval as fv (h ^ ?idx * shift),
              clength * scale, 1)) s3)
and tail:
  Some (c-out, s4) ∈
    set-dist (execute (mfold out1 (receive-query-commits fl-tail))
      s-head)
by (auto elim!: set-dist-bindE)
have len-pos: 0 < clength * scale
using eval-domain-nontrivial by linarith
have idx-bound: ?idx < clength * scale
by (rule index-less-domain)
from fri-layer-opening-step-partial-authenticated-table
  [OF len-pos idx-bound head]
obtain openings where
  table-head:
    partial-authenticated-table composition-root (clength * scale)
    openings s-head

```

```

and indices:
  map opening-index openings =
    [?idx, (?idx + (clength * scale) div 2) mod (clength * scale)]
and len-openings: length openings = 2
by (rule fri-layer-opening-step-partial-authenticated-table
    [OF len-pos idx-bound head])
obtain i1 x1 len1 pw1 where out1-eq: out1 = (i1, x1, len1, pw1)
  by (cases out1)
from receive-query-commits-layers-outcome [OF tail[unfolded out1-eq]]
obtain tail-layer-chunks tail-chunk where s-head-s4: s-head ≤ s4
  by blast
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
have table-t:
  partial-authenticated-table composition-root (scale * clength)
    openings t
  using partial-authenticated-table-mono [OF table-head s-head-s4]
  unfolding t-eq by (simp add: mult.commute)
have indices':
  map opening-index openings =
    [?idx, fri-sibling-index (scale * clength) ?idx]
  using indices
  unfolding fri-sibling-index-def
  by (simp add: mult.commute)
show ?thesis
  by (rule that[OF refl table-t indices' len-openings])
qed

```

lemma *verifier-query-round-program-authenticated-trace-openings*:

```

fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
obtains raw idx leaves openings where
  idx = index (to-nat raw)
  length openings = length (powers-scaled idx)
  map opening-value openings = leaves
  map opening-index openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) openings t
proof –
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where
  rand:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)

```

```

and tail:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s0)
  by (auto elim!: set-dist-bindE)
from verifier-query-round-after-index-authenticated-trace-openings[OF tail]
obtain idx leaves openings where
  idx-eq: idx = index (to-nat raw)
  and len-openings: length openings = length (powers-scaled idx)
  and values-openings: map opening-value openings = leaves
  and idx-openings: map opening-index openings = powers-scaled idx
  and table-t:
    partial-authenticated-table fr (scale * clength) openings t
  by blast
show ?thesis
  by (rule that[OF idx-eq len-openings values-openings idx-openings table-t])
qed

```

lemma *verifier-query-round-program-authenticated-composition-openings*:

```

fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-eq: fl = (b, composition-root) # fl-tail
and outcome:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
obtains raw idx openings where
  idx = index (to-nat raw)
  partial-authenticated-table composition-root (scale * clength)
    openings t
  map opening-index openings =
    [idx, fri-sibling-index (scale * clength) idx]
  length openings = 2
proof –
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where
  rand:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
  and tail:
    Some ((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s0)
  by (auto elim!: set-dist-bindE)
from verifier-query-round-after-index-authenticated-composition-openings

```

```

    [OF fl-eq tail]
obtain idx openings where
  idx-eq: idx = index (to-nat raw)
and table-t:
  partial-authenticated-table composition-root (scale * clength)
    openings t
and idx-openings:
  map opening-index openings =
    [idx, fri-sibling-index (scale * clength) idx]
and len-openings: length openings = 2
by blast
show ?thesis
by (rule that[OF idx-eq table-t idx-openings len-openings])
qed

lemma ntimes-verifier-query-rounds-authenticated-trace-openings:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (results, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s)
obtains raw-idxs query-idxs trace-openings where
  length raw-idxs = n
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  length trace-openings = n
  ∧ i. i < n ⇒
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
  ∧ i. i < n ⇒
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) t
proof –
have
  ∃ raw-idxs query-idxs trace-openings.
    length raw-idxs = n ∧
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs ∧
    length trace-openings = n ∧
    (∀ i < n.
      map opening-index (trace-openings ! i) =
        powers-scaled (query-idxs ! i) ∧
      (∀ i < n.
        partial-authenticated-table fr (scale * clength)
          (trace-openings ! i) t)
    )
using outcome
proof (induction n arbitrary: s results t)
case 0

```

```

then show ?case
  by (intro exI[of - []]) simp
next
case (Suc n)
from Suc.premis obtain u results' s1 where
  head:
    Some (u, s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results', t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            n) s1)
  and results-eq: results = u # results'
  by (auto elim!: set-dist-bindE)
have u-eq: u = ()
  by (cases u) simp
from verifier-query-round-program-authenticated-trace-openings
  [OF head[unfolded u-eq]]
obtain raw idx leaves openings where
  idx-eq: idx = index (to-nat raw)
  and idx-openings: map opening-index openings = powers-scaled idx
  and table-s1:
    partial-authenticated-table fr (scale * clength) openings s1
  by blast
from Suc.IH[OF tail] obtain raw-tail idx-tail openings-tail where
  len-raw-tail: length raw-tail = n
  and idx-tail-eq:
    idx-tail = map (λraw. index (to-nat raw)) raw-tail
  and len-openings-tail: length openings-tail = n
  and tail-indices:
    ∀ i < n.
      map opening-index (openings-tail ! i) =
        powers-scaled (idx-tail ! i)
  and tail-tables:
    ∀ i < n.
      partial-authenticated-table fr (scale * clength)
        (openings-tail ! i) t
  by blast
have s1-t: s1 ≤ t
  using ntimes-verifier-query-rounds-outcome[OF tail] by blast
have table-t:
  partial-authenticated-table fr (scale * clength) openings t
  by (rule partial-authenticated-table-mono[OF table-s1 s1-t])
let ?raws = raw # raw-tail

```

```

let ?idxs = idx # idx-tail
let ?openings = openings # openings-tail
have idxs-eq:
  ?idxs = map (λraw. index (to-nat raw)) ?raws
  using idx-eq idx-tail-eq by simp
have indices-all:
  ∀ i < Suc n.
    map opening-index (?openings ! i) =
      powers-scaled (?idxs ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < Suc n
  show map opening-index (?openings ! i) =
    powers-scaled (?idxs ! i)
  proof (cases i)
    case 0
    then show ?thesis
      using idx-openings by simp
  next
    case (Suc j)
    then show ?thesis
      using i-bound tail-indices by simp
  qed
qed
have tables-all:
  ∀ i < Suc n.
    partial-authenticated-table fr (scale * clength)
    (?openings ! i) t
proof (intro allI impI)
  fix i
  assume i-bound: i < Suc n
  show partial-authenticated-table fr (scale * clength)
    (?openings ! i) t
  proof (cases i)
    case 0
    then show ?thesis
      using table-t by simp
  next
    case (Suc j)
    then show ?thesis
      using i-bound tail-tables by simp
  qed
qed
show ?case
  apply (intro exI[of - ?raws] exI[of - ?idxs] exI[of - ?openings]
    conjI)
  apply (simp add: len-raw-tail)
  apply (rule idxs-eq)
  apply (simp add: len-openings-tail)

```



```

    apply (rule indices-all)
    apply (rule tables-all)
  done
qed
then obtain raw-idxs query-idxs trace-openings where
  length raw-idxs = n
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
  length trace-openings = n
   $\bigwedge i. i < n \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
   $\bigwedge i. i < n \implies$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) t
  by blast
then show ?thesis by (rule that)
qed

lemma ntimes-verifier-query-rounds-authenticated-composition-openings:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-eq: fl = (b, composition-root) # fl-tail
and outcome:
  Some (results, t)  $\in$ 
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s)
obtains raw-idxs query-idxs composition-openings where
  length raw-idxs = n
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
  length composition-openings = n
   $\bigwedge i. i < n \implies$ 
    map opening-index (composition-openings ! i) =
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]
   $\bigwedge i. i < n \implies$ 
    partial-authenticated-table composition-root (scale * clength)
      (composition-openings ! i) t
proof –
have
   $\exists$  raw-idxs query-idxs composition-openings.
    length raw-idxs = n  $\wedge$ 
    query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs  $\wedge$ 
    length composition-openings = n  $\wedge$ 
    ( $\forall i < n.$ 
      map opening-index (composition-openings ! i) =
        [query-idxs ! i,
         fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 

```

```

  (∀ i < n.
    partial-authenticated-table composition-root (scale * clength)
      (composition-openings ! i) t)
using outcome
proof (induction n arbitrary: s results t)
  case 0
  then show ?case
    by (intro exI[of - []]) simp
next
  case (Suc n)
  from Suc.premis obtain u results' s1 where
    head:
      Some (u, s1) ∈
        set-dist
          (execute
            (verifier-query-round-program fr f-fl f-final as fl final) s)
    and tail:
      Some (results', t) ∈
        set-dist
          (execute
            (ntimes
              (verifier-query-round-program fr f-fl f-final as fl final)
            n) s1)
    by (auto elim!: set-dist-bindE)
  have u-eq: u = ()
    by (cases u) simp
  from verifier-query-round-program-authenticated-composition-openings
    [OF fl-eq head[unfolded u-eq]]
  obtain raw idx openings where
    idx-eq: idx = index (to-nat raw)
    and table-s1:
      partial-authenticated-table composition-root (scale * clength)
        openings s1
    and idx-openings:
      map opening-index openings =
        [idx, fri-sibling-index (scale * clength) idx]
    and len-openings: length openings = 2
    by (rule verifier-query-round-program-authenticated-composition-openings
      [OF fl-eq head[unfolded u-eq]])
  from Suc.IH[OF tail] obtain raw-tail idx-tail openings-tail where
    len-raw-tail: length raw-tail = n
    and idx-tail-eq:
      idx-tail = map (λraw. index (to-nat raw)) raw-tail
    and len-openings-tail: length openings-tail = n
    and tail-indices:
      ∀ i < n.
        map opening-index (openings-tail ! i) =
          [idx-tail ! i,
            fri-sibling-index (scale * clength) (idx-tail ! i)]

```

```

and tail-tables:
   $\forall i < n.$ 
    partial-authenticated-table composition-root (scale * clength)
      (openings-tail ! i) t
  by blast
have s1-t: s1 ≤ t
  using ntimes-verifier-query-rounds-outcome[OF tail] by blast
have table-t:
  partial-authenticated-table composition-root (scale * clength)
    openings t
  by (rule partial-authenticated-table-mono[OF table-s1 s1-t])
let ?raws = raw # raw-tail
let ?idxs = idx # idx-tail
let ?openings = openings # openings-tail
have idxs-eq:
  ?idxs = map ( $\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})$ ) ?raws
  using idx-eq idx-tail-eq by simp
have indices-all:
   $\forall i < \text{Suc } n.$ 
    map opening-index (?openings ! i) =
      [?idxs ! i, fri-sibling-index (scale * clength) (?idxs ! i)]
proof (intro allI impI)
  fix i
  assume i-bound: i < Suc n
  show map opening-index (?openings ! i) =
    [?idxs ! i, fri-sibling-index (scale * clength) (?idxs ! i)]
  proof (cases i)
    case 0
    then show ?thesis
    using idx-openings by simp
  next
    case (Suc j)
    then show ?thesis
    using i-bound tail-indices by simp
  qed
qed
have tables-all:
   $\forall i < \text{Suc } n.$ 
    partial-authenticated-table composition-root (scale * clength)
      (?openings ! i) t
proof (intro allI impI)
  fix i
  assume i-bound: i < Suc n
  show partial-authenticated-table composition-root (scale * clength)
    (?openings ! i) t
  proof (cases i)
    case 0
    then show ?thesis
    using table-t by simp

```

```

next
  case (Suc j)
  then show ?thesis
    using i-bound tail-tables by simp
qed
qed
show ?case
  apply (intro exI[of - ?rows] exI[of - ?idxs]
    exI[of - ?openings] conjI)
    apply (simp add: len-row-tail)
    apply (rule idxs-eq)
    apply (simp add: len-openings-tail)
    apply (rule indices-all)
    apply (rule tables-all)
  done
qed
then obtain raw-idxs query-idxs composition-openings where
  len-row: length raw-idxs = n
  and idxs-eq: query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
  and len-openings: length composition-openings = n
  and indices:
     $\forall i < n.$ 
      map opening-index (composition-openings ! i) =
        [query-idxs ! i,
          fri-sibling-index (scale * clength) (query-idxs ! i)]
  and tables:
     $\forall i < n.$ 
      partial-authenticated-table composition-root (scale * clength)
        (composition-openings ! i) t
  by (elim exE conjE, blast)
show ?thesis
  by (rule that[OF len-row idxs-eq len-openings])
    (use indices tables in auto)
qed

lemma ntimes-verifier-query-rounds-authenticated-trace-openings-no-rows:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (results, t)  $\in$ 
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) n)
          s)
  obtains query-idxs trace-openings where
    length query-idxs = n
    length trace-openings = n
     $\bigwedge i. i < n \implies$ 
      map opening-index (trace-openings ! i) =

```

```

    powers-scaled (query-idxs ! i)
   $\wedge i. i < n \implies$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) t
proof (rule ntimes-verifier-query-rounds-authenticated-trace-openings
  [OF outcome])
fix raw-idxs query-idxs trace-openings
assume len-raw: length raw-idxs = n
and idxs-eq: query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and len-openings: length trace-openings = n
and indices:
   $\wedge i. i < n \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
and tables:
   $\wedge i. i < n \implies$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) t
have len-query-idxs: length query-idxs = n
using len-raw idxs-eq by simp
show ?thesis
by (rule that[OF len-query-idxs len-openings indices tables])
qed

```

lemma verify-monad-authenticated-trace-openings:

fixes $s :: ('f, 'a)$ protocol-channel-scheme

assumes outcome:

Some (result, final-state) $\in$ set-dist (execute verify-monad s)

obtains fr query-idxs trace-openings **where**

length query-idxs = rounds

length trace-openings = rounds

$\wedge i. i < \text{rounds} \implies$

map opening-index (trace-openings ! i) =

powers-scaled (query-idxs ! i)

$\wedge i. i < \text{rounds} \implies$

partial-authenticated-table fr (scale * clength)

(trace-openings ! i) final-state

proof –

from verify-monad-header-extraction[OF outcome]

obtain fr f-fl f-final as dg fl final query-state **where**

query-out:

Some (result, final-state) $\in$

set-dist (execute

(ntimes

(verifier-query-round-program fr f-fl f-final as fl final)

rounds)

query-state)

by metis

from ntimes-verifier-query-rounds-authenticated-trace-openings

$[OF \text{ query-out}]$
obtain $raw\text{-}idxs \text{ query-}idxs \text{ trace-openings}$ **where**
 $len\text{-}raw: \text{length } raw\text{-}idxs = \text{rounds}$
and $query\text{-}idxs\text{-}def:$
 $query\text{-}idxs = \text{map } (\lambda raw. \text{index } (to\text{-}nat \text{ raw})) \text{ raw-}idxs$
and $len\text{-}openings: \text{length } trace\text{-}openings = \text{rounds}$
and $indices:$
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{map } opening\text{-}index \text{ (trace-openings ! } i) =$
 $\text{powers-scaled } (query\text{-}idxs ! i)$
and $tables:$
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{partial-authenticated-table fr (scale * clength)}$
 $\text{(trace-openings ! } i) \text{ final-state}$
by $blast$
have $len\text{-}query\text{-}idxs: \text{length } query\text{-}idxs = \text{rounds}$
using $len\text{-}raw \text{ query-}idxs\text{-}def$ **by** $simp$
show $?thesis$
by $(\text{rule that}[OF \text{ len-query-}idxs \text{ len-openings indices tables}])$
qed

definition $accepted\text{-}with\text{-}partial\text{-}trace\text{-}openings$

$:: (f, 'a) \text{ protocol-channel-scheme} \implies$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \implies$
 $f \implies \text{nat list} \implies f \text{ authenticated-opening list list} \implies \text{bool}$

where

$accepted\text{-}with\text{-}partial\text{-}trace\text{-}openings \text{ s out fr query-}idxs \text{ trace-openings} \longleftrightarrow$
 $accepted \text{ out} \wedge$
 $(\exists \text{ result final-state.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{length } query\text{-}idxs = \text{rounds} \wedge$
 $\text{length } trace\text{-}openings = \text{rounds} \wedge$
 $(\forall i < \text{rounds.}$
 $\text{map } opening\text{-}index \text{ (trace-openings ! } i) =$
 $\text{powers-scaled } (query\text{-}idxs ! i)) \wedge$
 $(\forall i < \text{rounds.}$
 $\text{partial-authenticated-table fr (scale * clength)}$
 $\text{(trace-openings ! } i) \text{ final-state}))$

lemma $accepted\text{-}with\text{-}partial\text{-}trace\text{-}openings\text{-}imp\text{-}accepted:$

assumes $accepted\text{-}with\text{-}partial\text{-}trace\text{-}openings \text{ s out fr query-}idxs \text{ trace-openings}$
shows $accepted \text{ out}$
using $assms \text{ unfolding } accepted\text{-}with\text{-}partial\text{-}trace\text{-}openings\text{-}def$ **by** $simp$

lemma $accepted\text{-}with\text{-}partial\text{-}trace\text{-}openings\text{-}shapes:$

assumes $accepted\text{-}with\text{-}partial\text{-}trace\text{-}openings \text{ s out fr query-}idxs \text{ trace-openings}$
shows $\text{length } query\text{-}idxs = \text{rounds}$
and $\text{length } trace\text{-}openings = \text{rounds}$
and $\bigwedge i. i < \text{rounds} \implies$

$\text{map opening-index (trace-openings ! i) =}$
 $\text{powers-scaled (query-idxs ! i)}$
using *assms* **unfolding** *accepted-with-partial-trace-openings-def* **by** *blast+*

definition *table-agrees-with-authenticated-openings*
 $:: 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ authenticated-opening list} \Rightarrow \text{bool}$

where

$\text{table-agrees-with-authenticated-openings table len openings} \longleftrightarrow$
 $\text{length table} = \text{len} \wedge$
 $(\forall \text{ opening} \in \text{set openings.}$
 $\text{opening-index opening} < \text{len} \wedge$
 $\text{table ! opening-index opening} = \text{opening-value opening})$

lemma *table-agrees-with-authenticated-openings-subset:*

assumes *agrees:*

$\text{table-agrees-with-authenticated-openings table len openings}$

and *subset:* $\text{set openings}' \subseteq \text{set openings}$

shows $\text{table-agrees-with-authenticated-openings table len openings}'$

using *assms*

unfolding *table-agrees-with-authenticated-openings-def* **by** *blast*

lemma *table-agrees-with-authenticated-openings-consistent-exists:*

assumes *idx-bound:*

$\bigwedge \text{opn. opn} \in \text{set openings} \implies \text{opening-index opn} < \text{len}$

and *consistent:*

$\bigwedge \text{opn opn'. opn} \in \text{set openings} \implies$

$\text{opn}' \in \text{set openings} \implies$

$\text{opening-index opn} = \text{opening-index opn}' \implies$

$\text{opening-value opn} = \text{opening-value opn}'$

shows $\exists \text{table. table-agrees-with-authenticated-openings table len openings}$

proof –

let *?value* =

$\lambda j. \text{if } \exists \text{opn} \in \text{set openings. opening-index opn} = j$

$\text{then opening-value}$

$(\text{SOME opn. opn} \in \text{set openings} \wedge \text{opening-index opn} = j)$

$\text{else } 0$

let *?table* = $\text{map ?value } [0..<\text{len}]$

have *agrees:*

$\bigwedge \text{opn. opn} \in \text{set openings} \implies$

$\text{?table ! opening-index opn} = \text{opening-value opn}$

proof –

fix *opn*

assume *opn-in:* $\text{opn} \in \text{set openings}$

let *?chosen* =

$\text{SOME opn'. opn}' \in \text{set openings} \wedge$

$\text{opening-index opn}' = \text{opening-index opn}$

have *ex-chosen:*

$\exists \text{opn'. opn}' \in \text{set openings} \wedge$

```

    opening-index  $opn'$  = opening-index  $opn$ 
  using  $opn\text{-}in$  by blast
have chosen:
  ?chosen  $\in$  set openings  $\wedge$ 
  opening-index ?chosen = opening-index  $opn$ 
  by (rule someI-ex[OF ex-chosen])
have value-eq:
  opening-value ?chosen = opening-value  $opn$ 
  using consistent[OF conjunct1[OF chosen]  $opn\text{-}in$  conjunct2[OF chosen]]
  by simp
have ?table ! opening-index  $opn$  = ?value (opening-index  $opn$ )
  using idx-bound[OF  $opn\text{-}in$ ] by simp
also have ... = opening-value ?chosen
proof -
  have  $\exists opn' \in$  set openings. opening-index  $opn' =$ 
    opening-index  $opn$ 
  using  $opn\text{-}in$  by blast
  then show ?thesis by simp
qed
also have ... = opening-value  $opn$ 
  by (rule value-eq)
finally show ?table ! opening-index  $opn$  = opening-value  $opn$  .
qed
have table-agrees-with-authenticated-openings ?table len openings
  unfolding table-agrees-with-authenticated-openings-def
  using idx-bound agrees by simp
then show ?thesis by blast
qed

```

definition *partial-trace-table-candidate*
 $:: 'f \text{ list} \Rightarrow 'f \text{ authenticated-opening list list} \Rightarrow \text{bool}$
where
partial-trace-table-candidate $\text{trace-table trace-openings} \longleftrightarrow$
 $\text{length trace-table} = \text{scale} * \text{clength} \wedge$
 $\text{length trace-openings} = \text{rounds} \wedge$
 $(\forall i < \text{rounds}.$
 $\text{table-agrees-with-authenticated-openings trace-table}$
 $(\text{scale} * \text{clength}) (\text{trace-openings} ! i))$

definition *partial-trace-table-candidates*
 $:: 'f \text{ authenticated-opening list list} \Rightarrow 'f \text{ list set}$
where
partial-trace-table-candidates $\text{trace-openings} =$
 $\{\text{trace-table. partial-trace-table-candidate trace-table trace-openings}\}$

definition *accepted-with-partial-composition-openings*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \Rightarrow \text{nat list} \Rightarrow 'f \text{ authenticated-opening list list} \Rightarrow \text{bool}$

where

accepted-with-partial-composition-openings *s out composition-root*
query-idxs composition-openings $\longleftrightarrow$
accepted out $\wedge$
 $(\exists \text{ result final-state.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{length query-idxs} = \text{rounds} \wedge$
 $\text{length composition-openings} = \text{rounds} \wedge$
 $(\forall i < \text{rounds.}$
 $\text{map opening-index (composition-openings ! i)} =$
 $[\text{query-idxs ! i,}$
 $\text{fri-sibling-index (scale * clength) (query-idxs ! i)}]) \wedge$
 $(\forall i < \text{rounds.}$
 $\text{partial-authenticated-table composition-root (scale * clength)}$
 $(\text{composition-openings ! i final-state}))$

definition *partial-composition-table-candidate*

$:: 'f \text{ list} \Rightarrow 'f \text{ authenticated-opening list list} \Rightarrow \text{bool}$

where

partial-composition-table-candidate composition-table composition-openings
 $\longleftrightarrow$
 $\text{length composition-table} = \text{scale * clength} \wedge$
 $\text{length composition-openings} = \text{rounds} \wedge$
 $(\forall i < \text{rounds.}$
 $\text{table-agrees-with-authenticated-openings composition-table}$
 $(\text{scale * clength) (composition-openings ! i)})$

definition *partial-composition-table-candidates*

$:: 'f \text{ authenticated-opening list list} \Rightarrow 'f \text{ list set}$

where

partial-composition-table-candidates composition-openings =
 $\{\text{composition-table.}$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings}\}$

lemma *partial-trace-table-candidateD:*

assumes *cand:* *partial-trace-table-candidate trace-table trace-openings*
and *i-bound:* $i < \text{rounds}$
and *opn-in:* $\text{opn} \in \text{set (trace-openings ! i)}$
shows $\text{length trace-table} = \text{scale * clength}$
and $\text{opening-index opn} < \text{scale * clength}$
and $\text{trace-table ! opening-index opn} = \text{opening-value opn}$
using *assms*
unfolding *partial-trace-table-candidate-def*
table-agrees-with-authenticated-openings-def
by *blast+*

lemma *table-agrees-with-authenticated-openings-map-values:*

assumes *agrees:*

```

    table-agrees-with-authenticated-openings table len openings
  and idxs: map opening-index openings = idxs
  shows map (!) table idxs = map opening-value openings
proof (rule nth-equalityI)
  show length (map (!) table) idxs =
    length (map opening-value openings)
proof -
  have length idxs = length openings
    using arg-cong[OF idxs, of length] by simp
  then show ?thesis by simp
qed
next
fix k
assume k-bound: k < length (map (!) table) idxs
have len-idxs: length idxs = length openings
  using arg-cong[OF idxs, of length] by simp
then have k-openings: k < length openings
  using k-bound len-idxs by simp
have idx-eq: idxs ! k = opening-index (openings ! k)
  using idxs k-openings by (metis length-map nth-map)
have opening-in: openings ! k ∈ set openings
  by (rule nth-mem[OF k-openings])
have table ! opening-index (openings ! k) =
  opening-value (openings ! k)
  using agrees opening-in
  unfolding table-agrees-with-authenticated-openings-def by blast
then show map (!) table idxs ! k =
  map opening-value openings ! k
  using k-bound k-openings idx-eq by simp
qed

lemma partial-trace-table-candidate-query-values:
  assumes cand: partial-trace-table-candidate trace-table trace-openings
  and i-bound: i < rounds
  and idxs:
    map opening-index (trace-openings ! i) = powers-scaled idx
  shows
    map (!) trace-table (powers-scaled idx) =
      map opening-value (trace-openings ! i)
proof -
  have agrees:
    table-agrees-with-authenticated-openings trace-table
      (scale * clength) (trace-openings ! i)
  using cand i-bound unfolding partial-trace-table-candidate-def by blast
  show ?thesis
    by (rule table-agrees-with-authenticated-openings-map-values
        [OF agrees idxs])
qed

```

lemma *partial-trace-table-candidates-iff*:
 $\text{trace-table} \in \text{partial-trace-table-candidates } \text{trace-openings} \longleftrightarrow$
 $\text{partial-trace-table-candidate } \text{trace-table } \text{trace-openings}$
unfolding *partial-trace-table-candidates-def* **by** *simp*

lemma *accepted-with-partial-composition-openings-imp-accepted*:
assumes
 $\text{accepted-with-partial-composition-openings } s \text{ out } \text{composition-root}$
 $\text{query-idxs } \text{composition-openings}$
shows accepted out
using *assms*
unfolding *accepted-with-partial-composition-openings-def* **by** *simp*

lemma *accepted-with-partial-composition-openings-shapes*:
assumes
 $\text{accepted-with-partial-composition-openings } s \text{ out } \text{composition-root}$
 $\text{query-idxs } \text{composition-openings}$
shows $\text{length query-idxs} = \text{rounds}$
and $\text{length composition-openings} = \text{rounds}$
and $\bigwedge i. i < \text{rounds} \implies$
 $\text{map opening-index } (\text{composition-openings } ! i) =$
 $[\text{query-idxs } ! i,$
 $\text{fri-sibling-index } (\text{scale} * \text{clength}) (\text{query-idxs } ! i)]$
using *assms*
unfolding *accepted-with-partial-composition-openings-def* **by** *blast+*

definition *accepted-with-partial-initial-openings*
 $:: ('f, 'a) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{protocol-channel-scheme}) \text{option} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow \text{bool}$

where

$\text{accepted-with-partial-initial-openings } s \text{ out fr } f\text{-fri-roots } f\text{-final as } dg$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings} \longleftrightarrow$
 $\text{accepted out} \wedge$
 $\text{composition-fri-roots} \neq [] \wedge$
 $(\exists \text{ rest.}$
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final as } dg$
 $\text{composition-fri-roots final rest}) \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr } \text{trace-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{accepted-with-partial-composition-openings } s \text{ out}$
 $(\text{hd } \text{composition-fri-roots}) \text{ composition-query-idxs composition-openings}$

lemma *accepted-with-partial-initial-openings-imp-accepted*:
assumes

accepted-with-partial-initial-openings s out fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings
shows *accepted out*
using *assms unfolding accepted-with-partial-initial-openings-def* **by** *simp*

lemma *accepted-with-partial-initial-openings-shapes:*

assumes

accepted-with-partial-initial-openings s out fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings

shows *composition-fri-roots* $\neq []$

and $\exists$ *rest. verifier-header-transcript s fr f-fri-roots f-final as dg*
composition-fri-roots final rest

and *accepted-with-partial-trace-openings s out fr trace-query-idxs*
trace-openings

and *accepted-with-partial-composition-openings s out*
(hd composition-fri-roots) composition-query-idxs composition-openings

using *assms unfolding accepted-with-partial-initial-openings-def* **by** *blast+*

lemma *query-rounds-accepted-with-partial-composition-openings:*

fixes *s query-state final-state :: ('f, 'a) protocol-channel-scheme*

assumes *fl-eq: fl = (b, composition-root) # fl-tail*

and *outcome:*

Some (results, final-state) ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr f-fl f-final as fl final)
rounds)
query-state)

obtains *query-idxs composition-openings* **where**

accepted-with-partial-composition-openings s
(Some (results, final-state)) composition-root query-idxs
composition-openings

proof –

from *ntimes-verifier-query-rounds-authenticated-composition-openings*
[OF fl-eq outcome]

obtain *raw-idxs query-idxs composition-openings* **where**

len-raw: length raw-idxs = rounds

and *query-idxs-def:*

query-idxs = map (λraw. index (to-nat raw)) raw-idxs

and *len-openings: length composition-openings = rounds*

and *indices:*

$\bigwedge i. i < rounds \implies$
 $map\ opening-index\ (composition-openings\ !\ i) =$
 $[query-idxs\ !\ i,$
 $fri-sibling-index\ (scale * clength)\ (query-idxs\ !\ i)]$

and *tables:*

```

 $\bigwedge i. i < \text{rounds} \implies$ 
  partial-authenticated-table composition-root (scale * clength)
    (composition-openings ! i) final-state
  by blast
have len-query-idxs: length query-idxs = rounds
  using len-raw query-idxs-def by simp
have partial:
  accepted-with-partial-composition-openings s
    (Some (results, final-state)) composition-root query-idxs
    composition-openings
  unfolding accepted-with-partial-composition-openings-def accepted-def
  apply (intro conjI exI)
    apply simp
    apply simp
    apply (rule len-query-idxs)
    apply (rule len-openings)
  using indices tables by simp-all
show ?thesis
  by (rule that[OF partial])
qed

lemma verify-monad-accepted-with-partial-composition-openings-if-bound-tables:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
  and bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs-bound
  obtains composition-root query-idxs composition-openings where
    accepted-with-partial-composition-openings s
      (Some (result, final-state)) composition-root query-idxs
      composition-openings
    merkle-root-binds-table composition-root composition-table final-state
proof -
  from verify-monad-header-extraction[OF outcome]
  obtain fr f-fl f-final as' dg fl final query-state where
    header:
      verifier-header-transcript s fr (map snd f-fl) f-final as' dg
      (map snd fl) final (PTranscript query-state)
  and query-out:
    Some (result, final-state)  $\in$ 
      set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as' fl final)
          rounds)
        query-state)
  by metis
from bound obtain fr' f-fri-roots' f-final' dg'

```

```

    composition-fri-roots' final' rest' where
header':
    verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
    composition-fri-roots' final' rest'
and comp-nonempty: composition-fri-roots' ≠ []
and comp-bind:
    merkle-root-binds-table (hd composition-fri-roots')
    composition-table final-state
unfolding accepted-with-bound-tables-def by blast
have eqs:
    fr' = fr ∧
    f-fri-roots' = map snd f-fl ∧
    f-final' = f-final ∧
    as = as' ∧
    dg' = dg ∧
    composition-fri-roots' = map snd fl ∧
    final' = final ∧
    rest' = PTranscript query-state
using verifier-header-transcript-unique[OF header' header] by simp
then have fl-nonempty: fl ≠ []
    using comp-nonempty by (cases fl) simp-all
then obtain b composition-root fl-tail where fl-eq:
    fl = (b, composition-root) # fl-tail
    by (cases fl) auto
have comp-root-eq: hd composition-fri-roots' = composition-root
    using eqs fl-eq comp-nonempty by simp
from query-rounds-accepted-with-partial-composition-openings
    [OF fl-eq query-out]
obtain query-idxs composition-openings where partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) composition-root query-idxs
    composition-openings
    by blast
have bind:
    merkle-root-binds-table composition-root composition-table final-state
    using comp-bind unfolding comp-root-eq .
show ?thesis
    by (rule that[OF partial bind])
qed

lemma verify-monad-accepted-with-partial-composition-openings-if-header-nonempty:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
and header0:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and comp-nonempty: composition-fri-roots ≠ []
obtains query-idxs composition-openings where

```

```

    accepted-with-partial-composition-openings s
      (Some (result, final-state)) (hd composition-fri-roots) query-idxs
      composition-openings
  proof -
    from verify-monad-header-extraction[OF outcome]
    obtain fr' f-fl f-final' as' dg' fl final' query-state where
      header:
        verifier-header-transcript s fr' (map snd f-fl) f-final' as' dg'
          (map snd fl) final' (PTranscript query-state)
    and query-out:
      Some (result, final-state) ∈
        set-dist
          (execute
            (ntimes
              (verifier-query-round-program fr' f-fl f-final' as' fl
                final') rounds)
            query-state)
    by metis
  have eqs:
    fr' = fr ∧
    map snd f-fl = f-fri-roots ∧
    f-final' = f-final ∧
    as' = as ∧
    dg' = dg ∧
    map snd fl = composition-fri-roots ∧
    final' = final ∧
    PTranscript query-state = rest
  using verifier-header-transcript-unique[OF header header0] by simp
  have fl-nonempty: fl ≠ []
    using comp-nonempty eqs by (cases fl) simp-all
  then obtain b composition-root fl-tail where fl-eq:
    fl = (b, composition-root) # fl-tail
    by (cases fl) auto
  have root-eq: composition-root = hd composition-fri-roots
  proof -
    have map-eq: map snd fl = composition-fri-roots
      using eqs by simp
    then have composition-root # map snd fl-tail = composition-fri-roots
      unfolding fl-eq by simp
    then have hd composition-fri-roots = composition-root
      by (metis list.sel(1))
    then show ?thesis
      by simp
  qed
  from query-rounds-accepted-with-partial-composition-openings
    [OF fl-eq query-out]
  obtain query-idxs composition-openings where partial:
    accepted-with-partial-composition-openings s
      (Some (result, final-state)) composition-root query-idxs

```

```

      composition-openings
    by blast
  show ?thesis
    by (rule that[of query-idxs composition-openings])
      (use partial-root-eq in simp)
qed

lemma partial-composition-table-candidateD:
  assumes
    cand:
      partial-composition-table-candidate composition-table
      composition-openings
    and i-bound:  $i < \text{rounds}$ 
    and opn-in:  $\text{opn} \in \text{set } (\text{composition-openings } ! i)$ 
  shows  $\text{length } \text{composition-table} = \text{scale} * \text{clength}$ 
    and  $\text{opening-index } \text{opn} < \text{scale} * \text{clength}$ 
    and  $\text{composition-table } ! \text{opening-index } \text{opn} = \text{opening-value } \text{opn}$ 
  using assms
  unfolding partial-composition-table-candidate-def
    table-agrees-with-authenticated-openings-def
  by blast+

lemma partial-composition-table-candidate-query-values:
  assumes cand:
    partial-composition-table-candidate composition-table
    composition-openings
  and i-bound:  $i < \text{rounds}$ 
  and idxs:
     $\text{map } \text{opening-index } (\text{composition-openings } ! i) =$ 
     $[\text{idx}, \text{fri-sibling-index } (\text{scale} * \text{clength}) \text{idx}]$ 
  shows
     $\text{map } (!) \text{ composition-table}$ 
     $[\text{idx}, \text{fri-sibling-index } (\text{scale} * \text{clength}) \text{idx}] =$ 
     $\text{map } \text{opening-value } (\text{composition-openings } ! i)$ 
  proof -
    have agrees:
      table-agrees-with-authenticated-openings composition-table
         $(\text{scale} * \text{clength}) (\text{composition-openings } ! i)$ 
    using cand i-bound
    unfolding partial-composition-table-candidate-def by blast
  show ?thesis
    by (rule table-agrees-with-authenticated-openings-map-values
        [OF agrees idxs])
qed

```

```

lemma partial-composition-table-candidate-query-value:
  assumes cand:
    partial-composition-table-candidate composition-table
    composition-openings

```



```

and i-bound:  $i < \text{rounds}$ 
and idxs:
   $\text{map } \text{opening-index } (\text{composition-openings } ! i) =$ 
     $[\text{idx}, \text{fri-sibling-index } (\text{scale} * \text{clength}) \text{idx}]$ 
shows
   $\text{composition-table } ! \text{idx} =$ 
     $\text{opening-value } ((\text{composition-openings } ! i) ! 0)$ 
proof –
  have vals:
     $\text{map } (!) \text{composition-table}$ 
       $[\text{idx}, \text{fri-sibling-index } (\text{scale} * \text{clength}) \text{idx}] =$ 
       $\text{map } \text{opening-value } (\text{composition-openings } ! i)$ 
    by (rule partial-composition-table-candidate-query-values
      [OF cand i-bound idxs])
  have len-openings:  $\text{length } (\text{composition-openings } ! i) = 2$ 
proof –
  have  $\text{length } (\text{map } \text{opening-index } (\text{composition-openings } ! i)) =$ 
     $\text{length } [\text{idx}, \text{fri-sibling-index } (\text{scale} * \text{clength}) \text{idx}]$ 
    using arg-cong[OF idxs, of length] by simp
    then show ?thesis by simp
qed
have value0:
   $\text{map } (!) \text{composition-table}$ 
     $[\text{idx}, \text{fri-sibling-index } (\text{scale} * \text{clength}) \text{idx}] ! 0 =$ 
     $\text{map } \text{opening-value } (\text{composition-openings } ! i) ! 0$ 
  using vals by simp
show ?thesis
  using value0 len-openings by (cases composition-openings ! i) auto
qed

lemma partial-composition-table-candidates-iff:
   $\text{composition-table} \in$ 
     $\text{partial-composition-table-candidates } \text{composition-openings} \longleftrightarrow$ 
     $\text{partial-composition-table-candidate } \text{composition-table } \text{composition-openings}$ 
  unfolding partial-composition-table-candidates-def by simp

lemma verify-monad-accepted-with-partial-trace-openings:
  fixes  $s :: ('f, 'a) \text{protocol-channel-scheme}$ 
  assumes outcome:
     $\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute } \text{verify-monad } s)$ 
  obtains fr query-idxs trace-openings where
    accepted-with-partial-trace-openings s
     $(\text{Some } (\text{result}, \text{final-state})) \text{ fr } \text{query-idxs } \text{trace-openings}$ 
proof –
  show ?thesis
proof (rule verify-monad-authenticated-trace-openings[OF outcome])
  fix fr query-idxs trace-openings
  assume len-query-idxs:  $\text{length } \text{query-idxs} = \text{rounds}$ 
    and len-openings:  $\text{length } \text{trace-openings} = \text{rounds}$ 

```

```

and indices:
   $\bigwedge i. i < \text{rounds} \implies$ 
     $\text{map } \text{opening-index } (\text{trace-openings } ! i) =$ 
       $\text{powers-scaled } (\text{query-idxs } ! i)$ 
and tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
     $\text{partial-authenticated-table } \text{fr } (\text{scale} * \text{clength})$ 
     $(\text{trace-openings } ! i) \text{ final-state}$ 
have partial:
   $\text{accepted-with-partial-trace-openings } s$ 
   $(\text{Some } (\text{result}, \text{final-state})) \text{ fr } \text{query-idxs } \text{trace-openings}$ 
unfolding accepted-with-partial-trace-openings-def accepted-def
apply (intro conjI exI)
  apply simp
  apply simp
  apply (rule len-query-idxs)
  apply (rule len-openings)
using indices tables by simp-all
show ?thesis
  by (rule that[OF partial])
qed
qed

lemma verify-monad-accepted-with-partial-trace-openings-for-header:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes outcome:
   $\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute } \text{verify-monad } s)$ 
and header0:
   $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final } \text{as } dg$ 
   $\text{composition-fri-roots } \text{final } \text{rest}$ 
obtains query-idxs trace-openings where
   $\text{accepted-with-partial-trace-openings } s$ 
   $(\text{Some } (\text{result}, \text{final-state})) \text{ fr } \text{query-idxs } \text{trace-openings}$ 
proof –
from verify-monad-header-extraction[OF outcome]
obtain fr' f-fl f-final' as' dg' fl final' query-state where
  header:
     $\text{verifier-header-transcript } s \text{ fr' } (\text{map } \text{snd } f\text{-fl}) f\text{-final' } \text{as' } dg'$ 
     $(\text{map } \text{snd } fl) \text{ final' } (P\text{Transcript } \text{query-state})$ 
and query-out:
   $\text{Some } (\text{result}, \text{final-state}) \in$ 
     $\text{set-dist}$ 
     $(\text{execute}$ 
       $(\text{ntimes}$ 
         $(\text{verifier-query-round-program } \text{fr' } f\text{-fl } f\text{-final' } \text{as' } fl$ 
           $\text{final'}) \text{ rounds})$ 
       $\text{query-state})$ 
    by metis
have fr-eq:  $\text{fr}' = \text{fr}$ 

```

```

using verifier-header-transcript-unique[OF header header0] by simp
obtain query-idxs trace-openings where
  len-query-idxs: length query-idxs = rounds
and len-openings: length trace-openings = rounds
and indices:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
and tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr' (scale * clength)
      (trace-openings ! i) final-state
proof (rule ntimes-verifier-query-rounds-authenticated-trace-openings-no-raws
  [OF query-out])
fix query-idxs trace-openings
assume len-query-idxs: length query-idxs = rounds
and len-openings: length trace-openings = rounds
and indices:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
and tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr' (scale * clength)
      (trace-openings ! i) final-state
show ?thesis
  by (rule that[OF len-query-idxs len-openings indices tables])
qed
have partial:
  accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
proof -
have all-indices:
   $\forall i < \text{rounds}.$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
using indices by blast
have all-tables:
   $\forall i < \text{rounds}.$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
using tables fr-eq by simp
show ?thesis
unfolding accepted-with-partial-trace-openings-def accepted-def
by (intro conjI exI[of - result] exI[of - final-state])
  (use len-query-idxs len-openings all-indices all-tables in simp-all)
qed
show ?thesis
by (rule that[OF partial])

```

qed

lemma *verify-monad-accepted-with-partial-initial-openings-if-header-nonempty:*

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes *outcome:*

$\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute verify-monad } s)$

and *header:*

$\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final as } dg$

$\text{composition-fri-roots final rest}$

and *comp-nonempty:* $\text{composition-fri-roots} \neq []$

obtains $\text{trace-query-idxs trace-openings composition-query-idxs}$

$\text{composition-openings}$ **where**

$\text{accepted-with-partial-trace-openings } s$

$(\text{Some } (\text{result}, \text{final-state})) \text{ fr } \text{trace-query-idxs trace-openings}$

$\text{accepted-with-partial-composition-openings } s$

$(\text{Some } (\text{result}, \text{final-state})) (\text{hd } \text{composition-fri-roots})$

$\text{composition-query-idxs composition-openings}$

proof –

from *verify-monad-accepted-with-partial-trace-openings-for-header*

$[OF \text{ outcome header}]$

obtain $\text{trace-query-idxs trace-openings}$ **where** *trace-partial:*

$\text{accepted-with-partial-trace-openings } s$

$(\text{Some } (\text{result}, \text{final-state})) \text{ fr } \text{trace-query-idxs trace-openings}$

by *blast*

from *verify-monad-accepted-with-partial-composition-openings-if-header-nonempty*

$[OF \text{ outcome header comp-nonempty}]$

obtain $\text{composition-query-idxs composition-openings}$ **where** *composition-partial:*

$\text{accepted-with-partial-composition-openings } s$

$(\text{Some } (\text{result}, \text{final-state})) (\text{hd } \text{composition-fri-roots})$

$\text{composition-query-idxs composition-openings}$

by *blast*

show *?thesis*

by $(\text{rule that}[OF \text{ trace-partial composition-partial}])$

qed

lemma *verify-monad-accepted-with-partial-initial-openings:*

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes *outcome:*

$\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute verify-monad } s)$

and *header:*

$\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final as } dg$

$\text{composition-fri-roots final rest}$

and *comp-nonempty:* $\text{composition-fri-roots} \neq []$

obtains $\text{trace-query-idxs trace-openings composition-query-idxs}$

$\text{composition-openings}$ **where**

$\text{accepted-with-partial-initial-openings } s$

$(\text{Some } (\text{result}, \text{final-state})) \text{ fr } f\text{-fri-roots } f\text{-final as } dg$

$\text{composition-fri-roots final trace-query-idxs trace-openings}$

$\text{composition-query-idxs composition-openings}$

proof –
from *verify-monad-accepted-with-partial-initial-openings-if-header-nonempty*
 [*OF outcome header comp-nonempty*]
obtain *trace-query-idxs trace-openings composition-query-idxs*
composition-openings **where**
trace-partial:
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *composition-partial:*
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
by *blast*
have *accepted-out: accepted (Some (result, final-state))*
by (*rule accepted-with-partial-trace-openings-imp-accepted*
 [*OF trace-partial*])
have *evidence:*
accepted-with-partial-initial-openings s
(Some (result, final-state)) fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings
unfolding *accepted-with-partial-initial-openings-def*
by (*intro conjI exI[of - rest]*)
(use accepted-out comp-nonempty header trace-partial
composition-partial in simp-all)
show *?thesis*
by (*rule that[OF evidence]*)
qed

lemma *merkle-path-bound-unique-value-if-clean:*
assumes *clean: ¬ hash-map-output-collision s*
and *first: merkle-path-bound rt len idx v path s*
and *second: merkle-path-bound rt len idx v' path' s*
and *same-length: length path = length path'*
shows *v = v'*
using *first second same-length*
proof (*induction path arbitrary: path' rt len idx*)
case *Nil*
then have *path': path' = []* **by** *simp*
have *first-lookup:*
fmlookup (HashMap s) (MerkleLeaf v) = Some rt
using *Nil.premis(1)* **by** *simp*
have *second-lookup:*
fmlookup (HashMap s) (MerkleLeaf v') = Some rt
using *Nil.premis(2) path'* **by** *simp*
show *?case*
proof (*rule ccontr*)
assume *v ≠ v'*
then have *keys: MerkleLeaf v ≠ MerkleLeaf v'* **by** *simp*

```

have hash-map-output-collision s
  by (rule hash-map-output-collisionI
    [OF keys first-lookup second-lookup])
then show False using clean by contradiction
qed
next
case (Cons sibling path)
from Cons.premis(3) obtain sibling' path'' where
  path': path' = sibling' # path''
  and lengths: length path = length path''
  by (cases path') auto
show ?case
proof (cases idx < len div 2)
  case True
  obtain child where
    child:
      merkle-path-bound child (len div 2) idx v path s
  and first-lookup:
    fmlookup (HashMap s) (MerkleNode child sibling) = Some rt
  using Cons.premis(1) True by auto
  obtain child' where
    child':
      merkle-path-bound child' (len div 2) idx v' path'' s
  and second-lookup:
    fmlookup (HashMap s) (MerkleNode child' sibling') = Some rt
  using Cons.premis(2) True path' by auto
  have node-eq:
    MerkleNode child sibling = MerkleNode child' sibling'
  proof (rule ccontr)
  assume neg:
    MerkleNode child sibling ≠ MerkleNode child' sibling'
  have hash-map-output-collision s
    by (rule hash-map-output-collisionI
      [OF neg first-lookup second-lookup])
  then show False using clean by contradiction
  qed
  have child-eq: child = child' using node-eq by simp
  show ?thesis
    by (rule Cons.IH[OF child - lengths])
      (use child' child-eq in simp)
next
case False
obtain child where
  child:
    merkle-path-bound child (len div 2) (idx - len div 2)
      v path s
  and first-lookup:
    fmlookup (HashMap s) (MerkleNode sibling child) = Some rt
  using Cons.premis(1) False by auto

```

```

obtain child' where
  child':
    merkle-path-bound child' (len div 2) (idx - len div 2)
    v' path'' s
  and second-lookup:
    fmlookup (HashMap s) (MerkleNode sibling' child') = Some rt
  using Cons.premis(2) False path' by auto
have node-eq:
  MerkleNode sibling child = MerkleNode sibling' child'
proof (rule ccontr)
  assume neg:
    MerkleNode sibling child ≠ MerkleNode sibling' child'
  have hash-map-output-collision s
  by (rule hash-map-output-collisionI
    [OF neg first-lookup second-lookup])
  then show False using clean by contradiction
qed
have child-eq: child = child' using node-eq by simp
show ?thesis
  by (rule Cons.IH[OF child - lengths])
  (use child' child-eq in simp)
qed
qed

lemma inconsistent-authenticated-openings-imp-hash-collision:
  assumes first: authenticated-opening-in s opening
  and second: authenticated-opening-in s opening'
  and same-root: opening-root opening = opening-root opening'
  and same-length: opening-length opening = opening-length opening'
  and same-index: opening-index opening = opening-index opening'
  and different-value: opening-value opening ≠ opening-value opening'
  shows hash-map-output-collision s
proof (rule ccontr)
  assume clean: ¬ hash-map-output-collision s
  have path-lengths:
    length (opening-path opening) = length (opening-path opening')
  using first second same-length
  unfolding authenticated-opening-in-def by simp
  have first-bound:
    merkle-path-bound
      (opening-root opening)
      (opening-length opening)
      (opening-index opening)
      (opening-value opening)
      (opening-path opening)
    s
  using first unfolding authenticated-opening-in-def by blast
  have second-bound:
    merkle-path-bound

```

```

    (opening-root opening')
    (opening-length opening')
    (opening-index opening')
    (opening-value opening')
    (opening-path opening')
  s
  using second unfolding authenticated-opening-in-def by blast
have values-eq:
  opening-value opening = opening-value opening'
proof (rule merkle-path-bound-unique-value-if-clean[
  OF clean first-bound - path-lengths])
show
  merkle-path-bound
  (opening-root opening)
  (opening-length opening)
  (opening-index opening)
  (opening-value opening')
  (opening-path opening')
  s
  using second-bound same-root same-length same-index by simp
qed
show False using different-value values-eq by contradiction
qed

lemma partial-authenticated-table-values-unique-if-clean:
  assumes table: partial-authenticated-table rt len openings s
  and clean:  $\neg$  hash-map-output-collision s
  and first: opening  $\in$  set openings
  and second: opening'  $\in$  set openings
  and same-index: opening-index opening = opening-index opening'
  shows opening-value opening = opening-value opening'
proof (rule ccontr)
  assume different: opening-value opening  $\neq$  opening-value opening'
  have first-auth: authenticated-opening-in s opening
  using table first unfolding partial-authenticated-table-def by blast
  have second-auth: authenticated-opening-in s opening'
  using table second unfolding partial-authenticated-table-def by blast
  have roots: opening-root opening = opening-root opening'
  proof -
    have opening-root opening = rt
    using table first unfolding partial-authenticated-table-def by blast
    moreover have opening-root opening' = rt
    using table second unfolding partial-authenticated-table-def by blast
    ultimately show ?thesis by simp
  qed
  have lengths: opening-length opening = opening-length opening'
  proof -
    have opening-length opening = len
    using table first unfolding partial-authenticated-table-def by blast

```



```

moreover have opening-length opening' = len
  using table second unfolding partial-authenticated-table-def by blast
ultimately show ?thesis by simp
qed
have hash-map-output-collision s
  by (rule inconsistent-authenticated-openings-imp-hash-collision[
    OF first-auth second-auth roots lengths same-index different])
then show False using clean by contradiction
qed

lemma accepted-with-partial-trace-openings-values-unique-if-clean:
assumes partial:
  accepted-with-partial-trace-openings s (Some (result, final-state))
  fr query-idxs trace-openings
and clean:  $\neg$  hash-map-output-collision final-state
and i-bound:  $i < \text{rounds}$ 
and opn-in:  $\text{opn} \in \text{set } (\text{trace-openings } ! i)$ 
and opn'-in:  $\text{opn}' \in \text{set } (\text{trace-openings } ! i)$ 
and same-index:  $\text{opening-index } \text{opn} = \text{opening-index } \text{opn}'$ 
shows opening-value opn = opening-value opn'
proof -
have table:
  partial-authenticated-table fr (scale * clength)
  (trace-openings ! i) final-state
using partial i-bound unfolding accepted-with-partial-trace-openings-def
by blast
show ?thesis
  by (rule partial-authenticated-table-values-unique-if-clean
    [OF table clean opn-in opn'-in same-index])
qed

definition partial-trace-openings-inconsistent-bad
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  partial-trace-openings-inconsistent-bad s out  $\longleftrightarrow$ 
    ( $\exists$  fr query-idxs trace-openings result final-state i opn opn'.
      out = Some (result, final-state)  $\wedge$ 
      accepted-with-partial-trace-openings s out fr query-idxs trace-openings  $\wedge$ 
       $i < \text{rounds}$   $\wedge$ 
       $\text{opn} \in \text{set } (\text{trace-openings } ! i)$   $\wedge$ 
       $\text{opn}' \in \text{set } (\text{trace-openings } ! i)$   $\wedge$ 
       $\text{opening-index } \text{opn} = \text{opening-index } \text{opn}'$   $\wedge$ 
      opening-value opn  $\neq$  opening-value opn')

lemma accepted-with-partial-trace-openings-values-unique-if-no-bad:
assumes partial:
  accepted-with-partial-trace-openings s (Some (result, final-state))
  fr query-idxs trace-openings

```

and *no-bad*:
 $\neg$ *partial-trace-openings-inconsistent-bad* *s*
(Some (result, final-state))
and *i-bound*: *i* < *rounds*
and *opn-in*: *opn* $\in$ *set (trace-openings ! i)*
and *opn'-in*: *opn'* $\in$ *set (trace-openings ! i)*
and *same-index*: *opening-index opn* = *opening-index opn'*
shows *opening-value opn* = *opening-value opn'*
proof (*rule ccontr*)
assume *neg*: *opening-value opn* $\neq$ *opening-value opn'*
have *partial-trace-openings-inconsistent-bad* *s*
(Some (result, final-state))
unfolding *partial-trace-openings-inconsistent-bad-def*
by (*intro exI[of - fr] exI[of - query-idxs]*
exI[of - trace-openings] exI[of - result]
exI[of - final-state] exI[of - i] exI[of - opn]
exI[of - opn'])
(use partial i-bound opn-in opn'-in same-index neg in simp)
then show *False* **using** *no-bad* **by** *contradiction*
qed

lemma *merkle-bound-table-agrees-with-authenticated-opening-if-clean*:
assumes *bind*: *merkle-root-binds-table* *rt* *table* *final-state*
and *len-pow*: *length table* = 2^n
and *auth*: *authenticated-opening-in* *final-state* *opening*
and *root*: *opening-root* *opening* = *rt*
and *len*: *opening-length* *opening* = *length table*
and *clean*: $\neg$ *hash-map-output-collision* *final-state*
shows
opening-index *opening* < *length table* $\wedge$
table ! opening-index *opening* = *opening-value* *opening*
proof –
from *bind* **obtain** *tree* **where** *created*:
created-tree *table* *tree* *final-state*
and *rt-eq*: *rt* = *value* *tree*
unfolding *merkle-root-binds-table-def* **by** *blast*
have *idx-bound*: *opening-index* *opening* < *length table*
using *auth* *len* **unfolding** *authenticated-opening-in-def* **by** *simp*
have *table-root*:
merkle-path-root-in-map *final-state* (*length table*)
(*opening-index* *opening*) (*table ! opening-index* *opening*)
(*get-authentication-path* (*length table*) (*opening-index* *opening*) *tree*) =
Some *rt*
using *protocol-created-tree-path-root-in-map*
OF *created[unfolded created-tree-def] len-pow idx-bound*
unfolding *rt-eq* .
have *opening-root-in-map*:
merkle-path-root-in-map *final-state* (*length table*)
(*opening-index* *opening*) (*opening-value* *opening*)

```

    (opening-path opening) = Some rt
  using authenticated-opening-root-in-map[OF auth] root len by simp
have same:
  (table ! opening-index opening = opening-value opening ∧
   get-authentication-path (length table) (opening-index opening) tree =
   opening-path opening) ∨
  hash-map-output-collision final-state
by (rule merkle-path-root-same-index-eq-or-hash-collision[
  OF table-root opening-root-in-map])
then have value-eq:
  table ! opening-index opening = opening-value opening
  using clean by blast
show ?thesis
  using idx-bound value-eq by simp
qed

lemma accepted-bound-trace-table-partial-candidate-if-clean:
  assumes bind: merkle-root-binds-table fr trace-table final-state
  and len-table: length trace-table = scale * clength
  and partial:
    accepted-with-partial-trace-openings s (Some (result, final-state))
    fr query-idxs trace-openings
  and clean: ¬ hash-map-output-collision final-state
  shows partial-trace-table-candidate trace-table trace-openings
proof -
  obtain n where len-pow: clength * scale = 2 ^ n
  using eval-domain-length-power by blast
  have table-pow: length trace-table = 2 ^ n
  using len-table len-pow by (simp add: mult.commute)
  have len-openings: length trace-openings = rounds
  using partial unfolding accepted-with-partial-trace-openings-def by blast
  have agrees:
    ∧ i. i < rounds ⇒
      table-agrees-with-authenticated-openings trace-table
      (scale * clength) (trace-openings ! i)
  proof -
    fix i
    assume i-bound: i < rounds
    have table-i:
      partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
    using partial i-bound
    unfolding accepted-with-partial-trace-openings-def by blast
  show
    table-agrees-with-authenticated-openings trace-table
    (scale * clength) (trace-openings ! i)
  proof -
    have opening-agrees:
      ∀ opn ∈ set (trace-openings ! i).

```

```

    opening-index opn < scale * clength ∧
    trace-table ! opening-index opn = opening-value opn
proof
fix opn
assume opn-in: opn ∈ set (trace-openings ! i)
have root: opening-root opn = fr
    using table-i opn-in unfolding partial-authenticated-table-def
    by blast
have len: opening-length opn = length trace-table
    using table-i opn-in len-table
    unfolding partial-authenticated-table-def by simp
have auth: authenticated-opening-in final-state opn
    using table-i opn-in unfolding partial-authenticated-table-def
    by blast
have agreement:
    opening-index opn < length trace-table ∧
    trace-table ! opening-index opn = opening-value opn
    by (rule merkle-bound-table-agrees-with-authenticated-opening-if-clean
        [OF bind table-pow auth root len clean])
then show
    opening-index opn < scale * clength ∧
    trace-table ! opening-index opn = opening-value opn
    using len-table by simp
qed
show ?thesis
    unfolding table-agrees-with-authenticated-openings-def
    using len-table opening-agrees by simp
qed
qed
show ?thesis
    unfolding partial-trace-table-candidate-def
    using len-table len-openings agrees by simp
qed

lemma accepted-bound-composition-table-partial-candidate-if-clean:
assumes bind:
    merkle-root-binds-table composition-root composition-table
    final-state
and len-table: length composition-table = scale * clength
and partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) composition-root query-idxs
    composition-openings
and clean: ¬ hash-map-output-collision final-state
shows
    partial-composition-table-candidate composition-table
    composition-openings
proof –
    obtain n where len-pow: clength * scale = 2 ^ n

```

```

using eval-domain-length-power by blast
have table-pow: length composition-table =  $2^n$ 
using len-table len-pow by (simp add: mult.commute)
have len-openings: length composition-openings = rounds
using partial
unfolding accepted-with-partial-composition-openings-def by blast
have agrees:
   $\bigwedge i. i < \text{rounds} \implies$ 
    table-agrees-with-authenticated-openings composition-table
      (scale * clength) (composition-openings ! i)
proof -
  fix i
  assume i-bound: i < rounds
  have table-i:
    partial-authenticated-table composition-root (scale * clength)
      (composition-openings ! i) final-state
  using partial i-bound
  unfolding accepted-with-partial-composition-openings-def by blast
show
  table-agrees-with-authenticated-openings composition-table
    (scale * clength) (composition-openings ! i)
proof -
  have opening-agrees:
     $\forall \text{opn} \in \text{set } (\text{composition-openings} ! i).$ 
      opening-index opn < scale * clength  $\wedge$ 
      composition-table ! opening-index opn = opening-value opn
  proof
    fix opn
    assume opn-in: opn  $\in$  set (composition-openings ! i)
    have root: opening-root opn = composition-root
      using table-i opn-in unfolding partial-authenticated-table-def
      by blast
    have len: opening-length opn = length composition-table
      using table-i opn-in len-table
      unfolding partial-authenticated-table-def by simp
    have auth: authenticated-opening-in final-state opn
      using table-i opn-in unfolding partial-authenticated-table-def
      by blast
    have agreement:
      opening-index opn < length composition-table  $\wedge$ 
      composition-table ! opening-index opn = opening-value opn
    by (rule merkle-bound-table-agrees-with-authenticated-opening-if-clean
      [OF bind table-pow auth root len clean])
    then show
      opening-index opn < scale * clength  $\wedge$ 
      composition-table ! opening-index opn = opening-value opn
    using len-table by simp
  qed
show ?thesis

```

```

    unfolding table-agrees-with-authenticated-openings-def
    using len-table opening-agrees by simp
qed
qed
show ?thesis
    unfolding partial-composition-table-candidate-def
    using len-table len-openings agrees by simp
qed

definition partial-merkle-inconsistency-bad
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  partial-merkle-inconsistency-bad s out  $\longleftrightarrow$ 
    accepted out  $\wedge$ 
    ( $\exists$  result final-state opn opn'.
      out = Some (result, final-state)  $\wedge$ 
      authenticated-opening-in final-state opn  $\wedge$ 
      authenticated-opening-in final-state opn'  $\wedge$ 
      opening-root opn = opening-root opn'  $\wedge$ 
      opening-length opn = opening-length opn'  $\wedge$ 
      opening-index opn = opening-index opn'  $\wedge$ 
      opening-value opn  $\neq$  opening-value opn')

lemma partial-trace-openings-inconsistent-bad-imp-partial-merkle-inconsistency-bad:
  assumes bad: partial-trace-openings-inconsistent-bad s out
  shows partial-merkle-inconsistency-bad s out
proof -
  obtain fr query-idxs trace-openings result final-state i opn opn' where
    out: out = Some (result, final-state)
  and partial:
    accepted-with-partial-trace-openings s out fr query-idxs trace-openings
  and i-bound:  $i < \text{rounds}$ 
  and opn-in:  $\text{opn} \in \text{set } (\text{trace-openings } ! i)$ 
  and opn'-in:  $\text{opn}' \in \text{set } (\text{trace-openings } ! i)$ 
  and same-index:  $\text{opening-index } \text{opn} = \text{opening-index } \text{opn}'$ 
  and diff-value:  $\text{opening-value } \text{opn} \neq \text{opening-value } \text{opn}'$ 
  using bad unfolding partial-trace-openings-inconsistent-bad-def by blast
have table:
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) final-state
proof -
  obtain result' final-state' where
    partial-out: out = Some (result', final-state')
  and tables:
     $\forall i < \text{rounds}.$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state'
  using partial unfolding accepted-with-partial-trace-openings-def

```

```

    by auto
  have final-state-eq: final-state' = final-state
    using out partial-out by simp
  show ?thesis
    using tables i-bound unfolding final-state-eq by simp
qed
have opn-auth: authenticated-opening-in final-state opn
  using table opn-in unfolding partial-authenticated-table-def by blast
have opn'-auth: authenticated-opening-in final-state opn'
  using table opn'-in unfolding partial-authenticated-table-def by blast
have same-root: opening-root opn = opening-root opn'
proof -
  have root-opn: opening-root opn = fr
    using table opn-in unfolding partial-authenticated-table-def by simp
  have root-opn': opening-root opn' = fr
    using table opn'-in unfolding partial-authenticated-table-def by simp
  show ?thesis
    using root-opn root-opn' by simp
qed
have same-length: opening-length opn = opening-length opn'
proof -
  have length-opn: opening-length opn = scale * clength
    using table opn-in unfolding partial-authenticated-table-def by simp
  have length-opn': opening-length opn' = scale * clength
    using table opn'-in unfolding partial-authenticated-table-def by simp
  show ?thesis
    using length-opn length-opn' by simp
qed
show ?thesis
  unfolding partial-merkle-inconsistency-bad-def
  using partial out opn-auth opn'-auth same-root same-length same-index
    diff-value
  unfolding accepted-with-partial-trace-openings-def
  by blast
qed

lemma partial-merkle-inconsistency-bad-imp-hash-map-output-collision-bad:
  assumes partial-merkle-inconsistency-bad s out
  shows hash-map-output-collision-bad s out
proof -
  obtain result final-state opn opn' where
    out: out = Some (result, final-state)
    and first: authenticated-opening-in final-state opn
    and second: authenticated-opening-in final-state opn'
    and same-root: opening-root opn = opening-root opn'
    and same-length: opening-length opn = opening-length opn'
    and same-index: opening-index opn = opening-index opn'
    and different-value:
      opening-value opn ≠ opening-value opn'

```

using *assms* **unfolding** *partial-merkle-inconsistency-bad-def* **by** *blast*
have *collision*: *hash-map-output-collision final-state*
by (rule *inconsistent-authenticated-openings-imp-hash-collision*[
OF first second same-root same-length same-index different-value])
show *?thesis*
unfolding *hash-map-output-collision-bad-def accepted-def*
using *out collision* **by** *simp*
qed

lemma *partial-trace-openings-inconsistent-bad-imp-hash-map-output-collision-bad*:
assumes *partial-trace-openings-inconsistent-bad s out*
shows *hash-map-output-collision-bad s out*
by (rule *partial-merkle-inconsistency-bad-imp-hash-map-output-collision-bad*)
(rule *partial-trace-openings-inconsistent-bad-imp-partial-merkle-inconsistency-bad*
[*OF assms*])

lemma *partial-trace-openings-inconsistent-bad-mono-hash-map-output-collision-bad*:
wp-event verify-monad (partial-trace-openings-inconsistent-bad s) s ≤
wp-event verify-monad (hash-map-output-collision-bad s) s
by (rule *wp-event-mono*)
(rule *partial-trace-openings-inconsistent-bad-imp-hash-map-output-collision-bad*)

lemma *partial-merkle-inconsistency-bad-mono-hash-map-output-collision-bad*:
wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤
wp-event verify-monad (hash-map-output-collision-bad s) s
by (rule *wp-event-mono*)
(rule *partial-merkle-inconsistency-bad-imp-hash-map-output-collision-bad*)

lemma *partial-trace-openings-inconsistent-bad-bound-from-hash-map-output-collision*:
fixes *H :: prob*
assumes *wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H*
shows *wp-event verify-monad (partial-trace-openings-inconsistent-bad s) s ≤*
H
by (rule *order-trans*
[*OF partial-trace-openings-inconsistent-bad-mono-hash-map-output-collision-bad*
assms])

lemma *partial-merkle-inconsistency-bad-bound-from-hash-map-output-collision*:
fixes *H :: prob*
assumes *wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H*
shows *wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ H*
by (rule *order-trans*
[*OF partial-merkle-inconsistency-bad-mono-hash-map-output-collision-bad*
assms])

lemma *partial-merkle-inconsistency-bad-imp-accepted*:
assumes *partial-merkle-inconsistency-bad s out*
shows *accepted out*
using *assms* **unfolding** *partial-merkle-inconsistency-bad-def* **by** *simp*

lemma *accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad:*

assumes *partial:*

accepted-with-partial-trace-openings s (Some (result, final-state))
fr query-idxs trace-openings

and *no-bad:*

$\neg$ *partial-merkle-inconsistency-bad s*
(Some (result, final-state))

shows $\exists$ *trace-table.*

partial-trace-table-candidate trace-table trace-openings

proof –

have *len-openings: length trace-openings = rounds*

using *partial unfolding accepted-with-partial-trace-openings-def* **by** *blast*

have *table:*

$\bigwedge i. i < rounds \implies$

*partial-authenticated-table fr (scale * clength)*
(trace-openings ! i) final-state

using *partial unfolding accepted-with-partial-trace-openings-def* **by** *blast*

have *accepted-out: accepted (Some (result, final-state))*

by *(rule accepted-with-partial-trace-openings-imp-accepted[OF partial])*

have *idx-bound:*

$\bigwedge opn. opn \in set (List.concat trace-openings) \implies$
*opening-index opn < scale * clength*

proof –

fix *opn*

assume *opn-in: opn ∈ set (List.concat trace-openings)*

then obtain *openings where openings-in: openings ∈ set trace-openings*

and *opn-openings: opn ∈ set openings*

by *auto*

then obtain *i where i-bound: i < length trace-openings*

and *openings-eq: trace-openings ! i = openings*

by *(meson in-set-conv-nth)*

have *table-i:*

*partial-authenticated-table fr (scale * clength)*
(trace-openings ! i) final-state

using *table i-bound len-openings* **by** *simp*

have *auth: authenticated-opening-in final-state opn*

using *table-i opn-openings unfolding openings-eq*

partial-authenticated-table-def **by** *blast*

have *len: opening-length opn = scale * clength*

using *table-i opn-openings unfolding openings-eq*

partial-authenticated-table-def **by** *blast*

show *opening-index opn < scale * clength*

using *auth len unfolding authenticated-opening-in-def* **by** *simp*

qed

have *consistent:*

$\bigwedge opn opn'. opn \in set (List.concat trace-openings) \implies$

$opn' \in set (List.concat trace-openings) \implies$

$opening-index opn = opening-index opn' \implies$

```

    opening-value opn = opening-value opn'
proof (rule ccontr)
  fix opn opn'
  assume opn-in: opn ∈ set (List.concat trace-openings)
    and opn'-in: opn' ∈ set (List.concat trace-openings)
    and same-index: opening-index opn = opening-index opn'
    and neg: opening-value opn ≠ opening-value opn'
  obtain openings where openings-in: openings ∈ set trace-openings
    and opn-openings: opn ∈ set openings
    using opn-in by auto
  then obtain i where i-bound: i < length trace-openings
    and openings-eq: trace-openings ! i = openings
    by (meson in-set-conv-nth)
  obtain openings' where openings'-in: openings' ∈ set trace-openings
    and opn'-openings: opn' ∈ set openings'
    using opn'-in by auto
  then obtain j where j-bound: j < length trace-openings
    and openings'-eq: trace-openings ! j = openings'
    by (meson in-set-conv-nth)
  have table-i:
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
    using table i-bound len-openings by simp
  have table-j:
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! j) final-state
    using table j-bound len-openings by simp
  have auth: authenticated-opening-in final-state opn
    using table-i opn-openings unfolding openings-eq
      partial-authenticated-table-def by blast
  have auth': authenticated-opening-in final-state opn'
    using table-j opn'-openings unfolding openings'-eq
      partial-authenticated-table-def by blast
  have root: opening-root opn = opening-root opn'
    using table-i table-j opn-openings opn'-openings
    unfolding openings-eq openings'-eq partial-authenticated-table-def
    by auto
  have len: opening-length opn = opening-length opn'
    using table-i table-j opn-openings opn'-openings
    unfolding openings-eq openings'-eq partial-authenticated-table-def
    by auto
  have bad:
    partial-merkle-inconsistency-bad s (Some (result, final-state))
    unfolding partial-merkle-inconsistency-bad-def
    by (intro conjI exI[of - result] exI[of - final-state]
      exI[of - opn] exI[of - opn'])
      (use accepted-out auth auth' root len same-index neg in simp-all)
  then show False using no-bad by contradiction
qed

```

```

obtain trace-table where agrees-all:
  table-agrees-with-authenticated-openings trace-table (scale * clength)
    (List.concat trace-openings)
using table-agrees-with-authenticated-openings-consistent-exists
  [OF idx-bound consistent]
by blast
have agrees:
   $\bigwedge i. i < \text{rounds} \implies$ 
    table-agrees-with-authenticated-openings trace-table
      (scale * clength) (trace-openings ! i)
proof –
  fix i
  assume i-bound: i < rounds
  have set (trace-openings ! i)  $\subseteq$ 
    set (List.concat trace-openings)
  proof
    fix opn
    assume opn-in: opn  $\in$  set (trace-openings ! i)
    have trace-openings ! i  $\in$  set trace-openings
      using i-bound len-openings by (intro nth-mem) simp
    then show opn  $\in$  set (List.concat trace-openings)
      using opn-in by auto
  qed
  then show
    table-agrees-with-authenticated-openings trace-table
      (scale * clength) (trace-openings ! i)
    by (rule table-agrees-with-authenticated-openings-subset
      [OF agrees-all])
  qed
have len-table: length trace-table = scale * clength
  using agrees-all
  unfolding table-agrees-with-authenticated-openings-def by simp
have partial-trace-table-candidate trace-table trace-openings
  unfolding partial-trace-table-candidate-def
  using len-table len-openings agrees by simp
then show ?thesis by blast
qed

lemma accepted-with-partial-composition-openings-candidate-if-no-partial-merkle-bad:
assumes partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) composition-root query-idxs
  composition-openings
and no-bad:
   $\neg$  partial-merkle-inconsistency-bad s
  (Some (result, final-state))
shows  $\exists$  composition-table.
  partial-composition-table-candidate composition-table composition-openings
proof –

```

```

have len-openings: length composition-openings = rounds
  using partial
  unfolding accepted-with-partial-composition-openings-def by blast
have table:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table composition-root (scale * clength)
    (composition-openings ! i) final-state
  using partial
  unfolding accepted-with-partial-composition-openings-def by blast
have accepted-out: accepted (Some (result, final-state))
  by (rule accepted-with-partial-composition-openings-imp-accepted
    [OF partial])
have idx-bound:
   $\bigwedge \text{opn}. \text{opn} \in \text{set (List.concat composition-openings)} \implies$ 
    opening-index opn < scale * clength
proof -
  fix opn
  assume opn-in: opn  $\in$  set (List.concat composition-openings)
  then obtain openings where openings-in:
    openings  $\in$  set composition-openings
  and opn-openings: opn  $\in$  set openings
  by auto
  then obtain i where i-bound: i < length composition-openings
  and openings-eq: composition-openings ! i = openings
  by (meson in-set-conv-nth)
  have table-i:
    partial-authenticated-table composition-root (scale * clength)
    (composition-openings ! i) final-state
  using table i-bound len-openings by simp
  have auth: authenticated-opening-in final-state opn
  using table-i opn-openings unfolding openings-eq
    partial-authenticated-table-def by blast
  have len: opening-length opn = scale * clength
  using table-i opn-openings unfolding openings-eq
    partial-authenticated-table-def by blast
  show opening-index opn < scale * clength
  using auth len unfolding authenticated-opening-in-def by simp
qed
have consistent:
   $\bigwedge \text{opn opn}'. \text{opn} \in \text{set (List.concat composition-openings)} \implies$ 
    opn'  $\in$  set (List.concat composition-openings)  $\implies$ 
    opening-index opn = opening-index opn'  $\implies$ 
    opening-value opn = opening-value opn'
proof (rule ccontr)
  fix opn opn'
  assume opn-in: opn  $\in$  set (List.concat composition-openings)
  and opn'-in: opn'  $\in$  set (List.concat composition-openings)
  and same-index: opening-index opn = opening-index opn'
  and neq: opening-value opn  $\neq$  opening-value opn'

```

```

obtain openings where openings-in: openings  $\in$  set composition-openings
  and opn-openings: opn  $\in$  set openings
  using opn-in by auto
then obtain i where i-bound: i < length composition-openings
  and openings-eq: composition-openings ! i = openings
  by (meson in-set-conv-nth)
obtain openings' where openings'-in:
  openings'  $\in$  set composition-openings
  and opn'-openings: opn'  $\in$  set openings'
  using opn'-in by auto
then obtain j where j-bound: j < length composition-openings
  and openings'-eq: composition-openings ! j = openings'
  by (meson in-set-conv-nth)
have table-i:
  partial-authenticated-table composition-root (scale * clength)
  (composition-openings ! i) final-state
  using table i-bound len-openings by simp
have table-j:
  partial-authenticated-table composition-root (scale * clength)
  (composition-openings ! j) final-state
  using table j-bound len-openings by simp
have auth: authenticated-opening-in final-state opn
  using table-i opn-openings unfolding openings-eq
  partial-authenticated-table-def by blast
have auth': authenticated-opening-in final-state opn'
  using table-j opn'-openings unfolding openings'-eq
  partial-authenticated-table-def by blast
have root: opening-root opn = opening-root opn'
  using table-i table-j opn-openings opn'-openings
  unfolding openings-eq openings'-eq partial-authenticated-table-def
  by auto
have len: opening-length opn = opening-length opn'
  using table-i table-j opn-openings opn'-openings
  unfolding openings-eq openings'-eq partial-authenticated-table-def
  by auto
have bad:
  partial-merkle-inconsistency-bad s (Some (result, final-state))
  unfolding partial-merkle-inconsistency-bad-def
  by (intro conjI exI[of - result] exI[of - final-state]
    exI[of - opn] exI[of - opn'])
    (use accepted-out auth auth' root len same-index neq in simp-all)
then show False using no-bad by contradiction
qed
obtain composition-table where agrees-all:
  table-agrees-with-authenticated-openings composition-table
  (scale * clength) (List.concat composition-openings)
using table-agrees-with-authenticated-openings-consistent-exists
  [OF idx-bound consistent]
by blast

```

```

have agrees:
   $\bigwedge i. i < \text{rounds} \implies$ 
    table-agrees-with-authenticated-openings composition-table
      (scale * clength) (composition-openings ! i)
proof –
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  have set (composition-openings ! i)  $\subseteq$ 
    set (List.concat composition-openings)
  proof
    fix opn
    assume opn-in:  $\text{opn} \in \text{set (composition-openings ! i)}$ 
    have composition-openings ! i  $\in \text{set composition-openings}$ 
      using i-bound len-openings by (intro nth-mem) simp
    then show  $\text{opn} \in \text{set (List.concat composition-openings)}$ 
      using opn-in by auto
  qed
  then show
    table-agrees-with-authenticated-openings composition-table
      (scale * clength) (composition-openings ! i)
    by (rule table-agrees-with-authenticated-openings-subset
      [OF agrees-all])
  qed
have len-table: length composition-table = scale * clength
  using agrees-all
  unfolding table-agrees-with-authenticated-openings-def by simp
have partial-composition-table-candidate composition-table
  composition-openings
  unfolding partial-composition-table-candidate-def
  using len-table len-openings agrees by simp
then show ?thesis by blast
qed

lemma accepted-with-partial-initial-openings-candidates-if-no-partial-merkle-bad:
  assumes partial:
    accepted-with-partial-initial-openings s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings
  and no-bad:
     $\neg$  partial-merkle-inconsistency-bad s
    (Some (result, final-state))
  shows  $\exists \text{trace-table composition-table.}$ 
    partial-trace-table-candidate trace-table trace-openings  $\wedge$ 
    partial-composition-table-candidate composition-table
      composition-openings
proof –
  have trace-partial:
    accepted-with-partial-trace-openings s

```

```

    (Some (result, final-state)) fr trace-query-idxs trace-openings
using accepted-with-partial-initial-openings-shapes(3)[OF partial] .
have comp-partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) (hd composition-fri-roots)
  composition-query-idxs composition-openings
using accepted-with-partial-initial-openings-shapes(4)[OF partial] .
obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
  [OF trace-partial no-bad]
by blast
obtain composition-table where comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
using
  accepted-with-partial-composition-openings-candidate-if-no-partial-merkle-bad
  [OF comp-partial no-bad]
by blast
show ?thesis
  using trace-candidate comp-candidate by blast
qed

lemma verify-monad-partial-initial-candidates-or-partial-merkle-bad:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and comp-nonempty: composition-fri-roots ≠ []
shows
  partial-merkle-inconsistency-bad s (Some (result, final-state)) ∨
  (∃ trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table.
    accepted-with-partial-initial-openings s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings ∧
    partial-trace-table-candidate trace-table trace-openings ∧
    partial-composition-table-candidate composition-table
    composition-openings)
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then show ?thesis by simp
next
case False
from verify-monad-accepted-with-partial-initial-openings

```

```

    [OF outcome header comp-nonempty]
obtain trace-query-idxs trace-openings composition-query-idxs
    composition-openings where partial:
    accepted-with-partial-initial-openings s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings
by blast
obtain trace-table composition-table where candidates:
    partial-trace-table-candidate trace-table trace-openings
    partial-composition-table-candidate composition-table
    composition-openings
using accepted-with-partial-initial-openings-candidates-if-no-partial-merkle-bad
    [OF partial False]
by blast
show ?thesis
using partial candidates by blast
qed

lemma partial-merkle-inconsistency-bad-mono-accepted:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤
    wp-event verify-monad accepted s
by (rule wp-event-mono)
    (rule partial-merkle-inconsistency-bad-imp-accepted)

end

end

theory Soundness-Reductions-Trace
imports Soundness-Partial-Merkle
begin

FRI-table acceptance, transcript-target, and trace-table candidate reductions.

context soundness
begin

definition accepted-with-fri-tables
:: ('f, 'a) protocol-channel-scheme ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
    accepted-with-fri-tables s out ⟷
    accepted out ∧ fri-merkle-binding-good s out

lemma accepted-with-fri-tables-imp-accepted:
assumes accepted-with-fri-tables s out
shows accepted out
using assms unfolding accepted-with-fri-tables-def by simp

```


lemma *accepted-with-fri-tablesE*:
assumes *tables: accepted-with-fri-tables s out*
obtains *trace-table composition-table as query-idxs*
trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final trace-round-layers composition-round-layers
trace-tables composition-tables trace-final-table composition-final-table
where
accepted-with-bound-tables s out trace-table composition-table as
query-idxs
and *accepted-fri-opening-transcript s out trace-roots trace-bs*
trace-final dg composition-roots composition-bs composition-final
query-idxs trace-round-layers composition-round-layers
and *trace-fri-tables-bound s out trace-roots trace-bs trace-final*
query-idxs trace-round-layers trace-tables trace-final-table
and *composition-fri-tables-bound s out dg composition-roots*
composition-bs composition-final query-idxs composition-round-layers
composition-tables composition-final-table
and *fri-initial-table-agrees trace-table trace-tables*
and *fri-initial-table-agrees composition-table composition-tables*
using *tables*
unfolding *accepted-with-fri-tables-def fri-merkle-binding-good-def*
by (*blast intro: that*)

lemma *accepted-with-fri-tables-obtain-bound-tables*:
assumes *accepted-with-fri-tables s out*
obtains *trace-table composition-table as query-idxs*
where *accepted-with-bound-tables s out trace-table composition-table as*
query-idxs
using *assms* **by** (*elim accepted-with-fri-tablesE*) *blast*

lemma *accepted-partition-fri-merkle-binding*:
assumes *accepted out*
shows *fri-merkle-binding-bad s out* $\vee$ *accepted-with-fri-tables s out*
using *assms*
unfolding *fri-merkle-binding-bad-def accepted-with-fri-tables-def*
by *blast*

lemma *accepted-with-bound-tables-trace-root-target-hit*:
assumes *bound*:
accepted-with-bound-tables (verifier-initial-state tr)
(Some (result, final-state)) trace-table composition-table as
query-idxs
shows
hash-map-new-output-hit (set tr) (verifier-initial-state tr) final-state
proof –
from *bound* **obtain** *fr f-fri-roots f-final dg composition-fri-roots final rest*
where *header*:
verifier-header-transcript (verifier-initial-state tr) fr f-fri-roots

f-final as dg composition-fri-roots final rest
and *bind:*
merkle-root-binds-table fr trace-table final-state
unfolding *accepted-with-bound-tables-def* **by** *blast*
have *table-nonempty: trace-table $\neq \square$*
using *accepted-with-bound-tables-shapes(1)[OF bound]*
eval-domain-nontrivial **by** *auto*
from *merkle-root-binds-nonempty-table-output-lookup[OF bind table-nonempty]*
obtain *input* **where** *lookup:*
fmlookup (HashMap final-state) input = Some fr
by *blast*
have *root-in: fr $\in$ set tr*
using *header*
unfolding *verifier-header-transcript-def verifier-header-messages-def*
by *simp*
have *fresh:*
fmlookup (HashMap (verifier-initial-state tr)) input = None
by *simp*
show *?thesis*
unfolding *hash-map-new-output-hit-def*
using *fresh lookup root-in* **by** *blast*
qed

lemma *accepted-with-fri-tables-imp-transcript-target-hit:*
assumes *tables:*
accepted-with-fri-tables (verifier-initial-state tr) out
shows
hash-new-output-hit-event (set tr) (verifier-initial-state tr) out
proof –
from *accepted-with-fri-tables-obtain-bound-tables[OF tables]*
obtain *trace-table composition-table as query-idxs* **where** *bound:*
accepted-with-bound-tables (verifier-initial-state tr) out trace-table
composition-table as query-idxs
by *blast*
from *accepted-with-bound-tables-imp-accepted[OF bound]*
obtain *result final-state* **where** *out-eq: out = Some (result, final-state)*
unfolding *accepted-def* **by** *(cases out) auto*
have *hit:*
hash-map-new-output-hit (set tr) (verifier-initial-state tr) final-state
by *(rule accepted-with-bound-tables-trace-root-target-hit)*
(use bound out-eq in simp)
show *?thesis*
unfolding *hash-new-output-hit-event-def out-eq* **using** *hit* **by** *simp*
qed

lemma *accepted-imp-fri-merkle-binding-bad-or-transcript-target-hit:*
assumes *acc: accepted out*
shows
fri-merkle-binding-bad (verifier-initial-state tr) out $\vee$

```

      hash-new-output-hit-event (set tr) (verifier-initial-state tr) out
proof –
  from accepted-partition-fri-merkle-binding
    [of out verifier-initial-state tr, OF acc]
  show ?thesis
proof
  assume fri-merkle-binding-bad (verifier-initial-state tr) out
  then show ?thesis by simp
next
  assume tables:
    accepted-with-fri-tables (verifier-initial-state tr) out
  have hash-new-output-hit-event (set tr) (verifier-initial-state tr) out
    by (rule accepted-with-fri-tables-imp-transcript-target-hit[OF tables])
  then show ?thesis by simp
qed
qed

```

```

lemma verify-monad-verifier-initial-trace-root-target-hit:
  assumes outcome:
    Some (result, final-state) ∈
      set-dist (execute verify-monad (verifier-initial-state tr))
  shows
    hash-map-new-output-hit (set tr) (verifier-initial-state tr)
      final-state
proof –
  let ?s = verifier-initial-state tr
  from verify-monad-header-extraction[OF outcome]
  obtain fr f-fl f-final as dg fl final query-state where
    header:
      verifier-header-transcript ?s fr (map snd f-fl) f-final as dg
        (map snd fl) final (PTranscript query-state)
  and query-out:
    Some (result, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          query-state)
  by (rule verify-monad-header-query-extraction[OF outcome])
  obtain query-idxs trace-openings where
    len-query-idxs: length query-idxs = rounds
  and len-openings: length trace-openings = rounds
  and indices:
     $\bigwedge i. i < \text{rounds} \implies$ 
      map opening-index (trace-openings ! i) =
        powers-scaled (query-idxs ! i)
  and tables:
     $\bigwedge i. i < \text{rounds} \implies$ 

```

```

    partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) final-state
proof (rule ntimes-verifier-query-rounds-authenticated-trace-openings-no-rows
  [OF query-out])
fix query-idxs trace-openings
assume len-query-idxs: length query-idxs = rounds
and len-openings: length trace-openings = rounds
and indices:
   $\bigwedge i. i < rounds \implies$ 
    map opening-index (trace-openings ! i) =
    powers-scaled (query-idxs ! i)
and tables:
   $\bigwedge i. i < rounds \implies$ 
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) final-state
show ?thesis
  by (rule that[OF len-query-idxs len-openings indices tables])
qed
have round0: 0 < rounds
  by (rule rounds-positive)
have openings0-nonempty:
  trace-openings ! 0  $\neq$  []
proof -
  have map opening-index (trace-openings ! 0) =
    powers-scaled (query-idxs ! 0)
  by (rule indices[OF round0])
moreover have powers-scaled (query-idxs ! 0)  $\neq$  []
  using powers-pos unfolding powers-scaled-def by simp
ultimately show ?thesis
  by auto
qed
then obtain opn where opn-in:
  opn  $\in$  set (trace-openings ! 0)
  by (cases trace-openings ! 0) auto
have table0:
  partial-authenticated-table fr (scale * clength)
  (trace-openings ! 0) final-state
  using tables[OF round0] by simp
have opn-root: opening-root opn = fr
  using table0 opn-in unfolding partial-authenticated-table-def by blast
have opn-auth: authenticated-opening-in final-state opn
  using table0 opn-in unfolding partial-authenticated-table-def by blast
from authenticated-opening-root-lookup[OF opn-auth]
obtain input where lookup:
  fmlookup (HashMap final-state) input = Some fr
  using opn-root by auto
have root-in: fr  $\in$  set tr
  using header
  unfolding verifier-header-transcript-def verifier-header-messages-def

```

```

    by simp
  have fresh: fmlookup (HashMap ?s) input = None
    by simp
  show ?thesis
    unfolding hash-map-new-output-hit-def
    using fresh lookup root-in by blast
qed

lemma verify-monad-trace-root-target-preexisting-or-new:
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and tr-eq: PTranscript s = tr
  shows
    hash-map-output-values s ∩ set tr ≠ {} ∨
    hash-map-new-output-hit (set tr) s final-state
proof -
  from verify-monad-header-extraction[OF outcome]
  obtain fr f-fl f-final as dg fl final query-state where
    header:
      verifier-header-transcript s fr (map snd f-fl) f-final as dg
      (map snd fl) final (PTranscript query-state)
  and query-out:
    Some (result, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          query-state)
  by (rule verify-monad-header-query-extraction[OF outcome])
  obtain query-idxs trace-openings where
    indices:
       $\bigwedge i. i < \text{rounds} \implies$ 
      map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
  and tables:
       $\bigwedge i. i < \text{rounds} \implies$ 
      partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
proof (rule ntimes-verifier-query-rounds-authenticated-trace-openings-no-raws
  [OF query-out])
  fix query-idxs trace-openings
  assume indices:
     $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (trace-openings ! i) =
    powers-scaled (query-idxs ! i)
  and tables:
     $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr (scale * clength)

```

```

      (trace-openings ! i) final-state
show ?thesis
  by (rule that[OF indices tables])
qed
have round0: 0 < rounds
  by (rule rounds-positive)
have openings0-nonempty:
  trace-openings ! 0 ≠ []
proof -
  have map opening-index (trace-openings ! 0) =
    powers-scaled (query-idxs ! 0)
  by (rule indices[OF round0])
  moreover have powers-scaled (query-idxs ! 0) ≠ []
    using powers-pos unfolding powers-scaled-def by simp
  ultimately show ?thesis
    by auto
qed
then obtain opn where opn-in:
  opn ∈ set (trace-openings ! 0)
  by (cases trace-openings ! 0) auto
have table0:
  partial-authenticated-table fr (scale * clength)
  (trace-openings ! 0) final-state
  using tables[OF round0] by simp
have opn-root: opening-root opn = fr
  using table0 opn-in unfolding partial-authenticated-table-def by blast
have opn-auth: authenticated-opening-in final-state opn
  using table0 opn-in unfolding partial-authenticated-table-def by blast
from authenticated-opening-root-lookup[OF opn-auth]
obtain input where lookup-final:
  fmlookup (HashMap final-state) input = Some fr
  using opn-root by auto
have root-in: fr ∈ set tr
  using header tr-eq
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
have ext: s ≤ final-state
  by (rule verify-monad-hash-extends[OF outcome])
show ?thesis
proof (cases fmlookup (HashMap s) input)
case None
  then have hash-map-new-output-hit (set tr) s final-state
    unfolding hash-map-new-output-hit-def
    using lookup-final root-in by blast
  then show ?thesis by simp
next
case (Some z)
  have lookup-final-z: fmlookup (HashMap final-state) input = Some z
    by (rule hash-extension-lookup[OF Some ext])

```

```

    then have  $z = fr$ 
      using lookup-final by simp
    then have  $fr \in \text{hash-map-output-values } s$ 
      using Some by (auto intro: hash-map-output-valuesI)
    then show ?thesis
      using root-in by blast
  qed
qed

definition soundness-bad-event-with-merkle
:: ( $f, 'a$ ) protocol-channel-scheme  $\Rightarrow$ 
  ( $\text{unit list} \times (\mathbf{f}, 'a)$  protocol-channel-scheme) option  $\Rightarrow$  bool
where
  soundness-bad-event-with-merkle s out  $\longleftrightarrow$ 
    fri-merkle-binding-bad s out  $\vee$  soundness-bad-event s out

lemma soundness-bad-event-imp-soundness-bad-event-with-merkle:
  assumes soundness-bad-event s out
  shows soundness-bad-event-with-merkle s out
  using assms unfolding soundness-bad-event-with-merkle-def by simp

lemma fri-merkle-binding-bad-imp-soundness-bad-event-with-merkle:
  assumes fri-merkle-binding-bad s out
  shows soundness-bad-event-with-merkle s out
  using assms unfolding soundness-bad-event-with-merkle-def by simp

lemma accepted-with-fri-tables-table-partition:
  assumes false-statement:  $\neg$  exists-valid-trace
  and tables: accepted-with-fri-tables s out
  shows soundness-bad-event s out
proof –
  from accepted-with-fri-tables-obtain-bound-tables[OF tables]
  obtain trace-table composition-table as query-idxs where bound:
    accepted-with-bound-tables s out trace-table composition-table as
    query-idxs
  by blast
  have partition:
    composition-bad s out  $\vee$  trace-fri-bad s out  $\vee$ 
    composition-fri-bad s out  $\vee$  query-bad s out
  by (rule accepted-bound-table-partition[OF false-statement bound])
  then show ?thesis
    unfolding soundness-bad-event-def by blast
qed

lemma accepted-partition-soundness-bad-event-with-merkle:
  assumes false-statement:  $\neg$  exists-valid-trace
  and acc: accepted out
  shows soundness-bad-event-with-merkle s out
proof –

```

```

from accepted-partition-fri-merkle-binding[OF acc]
consider
  (merkle-bad) fri-merkle-binding-bad s out
  | (with-tables) accepted-with-fri-tables s out
by blast
then show ?thesis
proof cases
  case merkle-bad
  then show ?thesis
    by (rule fri-merkle-binding-bad-imp-soundness-bad-event-with-merkle)
  next
  case with-tables
  have soundness-bad-event s out
    by (rule accepted-with-fri-tables-table-partition
      [OF false-statement with-tables])
  then show ?thesis
    by (rule soundness-bad-event-imp-soundness-bad-event-with-merkle)
qed
qed

definition trace-fri-challenge-set-hit
  :: (f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (f list  $\Rightarrow$  'f list set)  $\Rightarrow$ 
    (unit list  $\times$  (f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  trace-fri-challenge-set-hit s bad-sets out  $\longleftrightarrow$ 
    ( $\exists$  trace-table composition-table as query-idxs trace-bs dg comp-bs.
      accepted-with-bound-tables s out trace-table composition-table as query-idxs  $\wedge$ 
      accepted-fri-challenges s out trace-bs dg comp-bs  $\wedge$ 
       $\neg$  trace-table-low-degree trace-table  $\wedge$ 
      trace-bs  $\in$  bad-sets trace-table)

definition composition-fri-challenge-set-hit
  :: (f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (f  $\Rightarrow$  'f list  $\Rightarrow$  'f list set)  $\Rightarrow$ 
    (unit list  $\times$  (f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  composition-fri-challenge-set-hit s bad-sets out  $\longleftrightarrow$ 
    ( $\exists$  trace-table composition-table as query-idxs trace-bs dg comp-bs.
      accepted-with-bound-tables s out trace-table composition-table as query-idxs  $\wedge$ 
      accepted-fri-challenges s out trace-bs dg comp-bs  $\wedge$ 
      trace-table-low-degree trace-table  $\wedge$ 
       $\neg$  composition-table-low-degree maxDegree composition-table  $\wedge$ 
      comp-bs  $\in$  bad-sets dg composition-table)

lemma trace-fri-challenge-set-hit-imp-list-set-hit:
  assumes hit: trace-fri-challenge-set-hit s bad-sets out
  and envelope:  $\bigwedge$  trace-table. bad-sets trace-table  $\subseteq B$ 
  shows trace-fri-challenge-list-set-hit s B out

```


proof –

from *hit* **obtain** *trace-table composition-table as query-idxs trace-bs dg comp-bs*
where *challenges*:
 accepted-fri-challenges s out trace-bs dg comp-bs
and *trace-bad*: *trace-bs ∈ bad-sets trace-table*
unfolding *trace-fri-challenge-set-hit-def* **by** *blast*
have *trace-bs ∈ B*
 using *trace-bad envelope[of trace-table]* **by** *auto*
then show *?thesis*
 unfolding *trace-fri-challenge-list-set-hit-def*
 using *challenges* **by** *blast*
qed

definition *trace-fri-root-trace-table-candidates*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list set}$

where

trace-fri-root-trace-table-candidates s fr =
 {*trace-table*.
 $\exists$ *out composition-table as query-idxs trace-bs dg comp-bs*
 f-fri-roots f-final header-as composition-fri-roots final rest.
 accepted-with-bound-tables s out trace-table composition-table
 as query-idxs $\wedge$
 accepted-fri-challenges s out trace-bs dg comp-bs $\wedge$
 verifier-header-transcript s fr f-fri-roots f-final header-as dg
 composition-fri-roots final rest}

definition *trace-fri-supported-root-trace-table-candidates*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list set}$

where

trace-fri-supported-root-trace-table-candidates s fr =
 {*trace-table*.
 $\exists$ *out composition-table as query-idxs trace-bs dg comp-bs*
 f-fri-roots f-final header-as composition-fri-roots final rest.
 out ∈ set-dist (execute verify-monad s) $\wedge$
 accepted-with-bound-tables s out trace-table composition-table
 as query-idxs $\wedge$
 accepted-fri-challenges s out trace-bs dg comp-bs $\wedge$
 verifier-header-transcript s fr f-fri-roots f-final header-as dg
 composition-fri-roots final rest}

definition *trace-fri-supported-root-partial-trace-table-candidates*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list set}$

where

trace-fri-supported-root-partial-trace-table-candidates s fr =
 {*trace-table*.
 $\exists$ *out query-idxs trace-openings trace-bs dg comp-bs*
 f-fri-roots f-final header-as composition-fri-roots final rest.
 out ∈ set-dist (execute verify-monad s) $\wedge$
 accepted-with-partial-trace-openings s out fr query-idxs

$\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate } \text{trace-table } \text{trace-openings} \wedge$
 $\text{accepted-fri-challenges } s \text{ out } \text{trace-bs } dg \text{ comp-bs} \wedge$
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final } \text{header-as } dg$
 $\text{composition-fri-roots } \text{final } \text{rest}\}$

definition *trace-fri-supported-root-partial-trace-table-candidate-state*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow f \Rightarrow f \text{ list} \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{trace-fri-supported-root-partial-trace-table-candidate-state } s \text{ fr}$
 $\text{trace-table } \text{final-state} \longleftrightarrow$
 $(\exists \text{result } \text{query-idxs } \text{trace-openings } \text{trace-bs } dg \text{ comp-bs}$
 $f\text{-fri-roots } f\text{-final } \text{header-as } \text{composition-fri-roots } \text{final } \text{rest}.$
 $\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute } \text{verify-monad } s) \wedge$
 $\text{accepted-with-partial-trace-openings } s$
 $(\text{Some } (\text{result}, \text{final-state})) \text{ fr } \text{query-idxs } \text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate } \text{trace-table } \text{trace-openings} \wedge$
 $\text{accepted-fri-challenges } s (\text{Some } (\text{result}, \text{final-state})) \text{ trace-bs } dg$
 $\text{comp-bs} \wedge$
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final } \text{header-as } dg$
 $\text{composition-fri-roots } \text{final } \text{rest})$

definition *alpha-header-supported-partial-trace-table-candidates*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow f \text{ list } \text{set}$

where

$\text{alpha-header-supported-partial-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final} =$
 $\{\text{trace-table}.$
 $\exists \text{out } \text{query-idxs } \text{trace-openings } \text{as } dg \text{ composition-fri-roots } \text{final } \text{rest}.$
 $\text{out} \in \text{set-dist } (\text{execute } \text{verify-monad } s) \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr } \text{query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate } \text{trace-table } \text{trace-openings} \wedge$
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final } \text{as } dg$
 $\text{composition-fri-roots } \text{final } \text{rest}\}$

lemma *alpha-header-supported-partial-trace-table-candidatesE:*

assumes

$\text{trace-table} \in$
 $\text{alpha-header-supported-partial-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final}$

obtains $\text{out } \text{query-idxs } \text{trace-openings } \text{as } dg \text{ composition-fri-roots } \text{final } \text{rest}$

where

$\text{out} \in \text{set-dist } (\text{execute } \text{verify-monad } s)$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr } \text{query-idxs}$
 trace-openings
 $\text{partial-trace-table-candidate } \text{trace-table } \text{trace-openings}$

verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
using *assms*
unfolding *alpha-header-supported-partial-trace-table-candidates-def*
by *blast*

definition *alpha-header-supported-partial-union-bad-sets*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list set}$

where

alpha-header-supported-partial-union-bad-sets s bad-sets fr
f-fri-roots f-final =
 $\{as \in \text{alpha-space}.$
 $\exists \text{ trace-table} \in$
 $\text{alpha-header-supported-partial-trace-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final}.$
 $as \in \text{bad-sets trace-table}\}$

definition *trace-fri-root-trace-table-candidate-state*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

trace-fri-root-trace-table-candidate-state s fr trace-table
final-state $\longleftrightarrow$
 $(\exists \text{ result composition-table as query-idxs trace-bs dg comp-bs}$
 $f\text{-fri-roots } f\text{-final header-as composition-fri-roots final rest}.$
 $\text{accepted-with-bound-tables } s (\text{Some } (\text{result}, \text{final-state}))$
 $\text{trace-table composition-table as query-idxs} \wedge$
 $\text{accepted-fri-challenges } s (\text{Some } (\text{result}, \text{final-state})) \text{ trace-bs dg}$
 $\text{comp-bs} \wedge$
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final header-as dg}$
 $\text{composition-fri-roots final rest} \wedge$
 $\text{merkle-root-binds-table fr trace-table final-state})$

definition *trace-fri-supported-root-trace-table-candidate-state*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state $\longleftrightarrow$
 $(\exists \text{ result composition-table as query-idxs trace-bs dg comp-bs}$
 $f\text{-fri-roots } f\text{-final header-as composition-fri-roots final rest}.$
 $\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute verify-monad } s) \wedge$
 $\text{accepted-with-bound-tables } s (\text{Some } (\text{result}, \text{final-state}))$
 $\text{trace-table composition-table as query-idxs} \wedge$
 $\text{accepted-fri-challenges } s (\text{Some } (\text{result}, \text{final-state})) \text{ trace-bs dg}$
 $\text{comp-bs} \wedge$
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final header-as dg}$

composition-fri-roots final rest $\wedge$
merkle-root-binds-table fr trace-table final-state)

lemma *trace-fri-root-trace-table-candidates-iff-state*:
trace-table $\in$ *trace-fri-root-trace-table-candidates s fr* $\longleftrightarrow$
 $(\exists$ *final-state*.
trace-fri-root-trace-table-candidate-state s fr trace-table
final-state)

proof

assume *candidate*:

trace-table $\in$ *trace-fri-root-trace-table-candidates s fr*

then obtain *out composition-table as query-idxs trace-bs dg comp-bs*
f-fri-roots f-final header-as composition-fri-roots final rest **where**
bound:

accepted-with-bound-tables s out trace-table composition-table as
query-idxs

and *challenges*:

accepted-fri-challenges s out trace-bs dg comp-bs

and *header*:

verifier-header-transcript s fr f-fri-roots f-final header-as dg
composition-fri-roots final rest

unfolding *trace-fri-root-trace-table-candidates-def* **by** *blast*
from *bound* **obtain** *result final-state fr' f-fri-roots' f-final' dg'*
composition-fri-roots' final' rest' **where**
out-eq: out = Some (result, final-state)

and *header'*:

verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
composition-fri-roots' final' rest'

and *bind*:

merkle-root-binds-table fr' trace-table final-state

unfolding *accepted-with-bound-tables-def* **by** *blast*

have *fr-eq: fr' = fr*

using *verifier-header-transcript-unique[OF header' header]* **by** *simp*

have *bound-some*:

accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs

using *bound out-eq* **by** *simp*

have *challenges-some*:

accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
comp-bs

using *challenges out-eq* **by** *simp*

have *bind-fr: merkle-root-binds-table fr trace-table final-state*

using *bind fr-eq* **by** *simp*

have *state*:

trace-fri-root-trace-table-candidate-state s fr trace-table
final-state

unfolding *trace-fri-root-trace-table-candidate-state-def*

apply (rule *exI*[**where** *x=result*])

apply (rule *exI*[**where** *x=composition-table*])

```

apply (rule exI[where x=as])
apply (rule exI[where x=query-idxs])
apply (rule exI[where x=trace-bs])
apply (rule exI[where x=dg])
apply (rule exI[where x=comp-bs])
apply (rule exI[where x=f-fri-roots])
apply (rule exI[where x=f-final])
apply (rule exI[where x=header-as])
apply (rule exI[where x=composition-fri-roots])
apply (rule exI[where x=final])
apply (rule exI[where x=rest])
apply (intro conjI)
  apply (rule bound-some)
  apply (rule challenges-some)
  apply (rule header)
apply (rule bind-fr)
done
then show
   $\exists$  final-state.
    trace-fri-root-trace-table-candidate-state s fr trace-table
    final-state
  by blast
next
assume
   $\exists$  final-state.
    trace-fri-root-trace-table-candidate-state s fr trace-table
    final-state
then obtain final-state result composition-table as query-idxs trace-bs dg
  comp-bs f-fri-roots f-final header-as composition-fri-roots final rest
where
  bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
and challenges:
    accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
    comp-bs
and header:
    verifier-header-transcript s fr f-fri-roots f-final header-as dg
    composition-fri-roots final rest
unfolding trace-fri-root-trace-table-candidate-state-def by blast
show trace-table  $\in$  trace-fri-root-trace-table-candidates s fr
unfolding trace-fri-root-trace-table-candidates-def
using bound challenges header by blast
qed

```

lemma *trace-fri-supported-root-trace-table-candidates-iff-state*:

trace-table $\in$ *trace-fri-supported-root-trace-table-candidates s fr* $\longleftrightarrow$
 $(\exists$ *final-state*.
trace-fri-supported-root-trace-table-candidate-state s fr trace-table

final-state)
proof
assume *candidate*:
trace-table $\in$ *trace-fri-supported-root-trace-table-candidates* *s* *fr*
then obtain *out* *composition-table* *as* *query-idxs* *trace-bs* *dg* *comp-bs*
f-fri-roots *f-final* *header-as* *composition-fri-roots* *final* *rest* **where**
outcome: *out* $\in$ *set-dist* (*execute* *verify-monad* *s*)
and *bound*:
accepted-with-bound-tables *s* *out* *trace-table* *composition-table* *as*
query-idxs
and *challenges*:
accepted-fri-challenges *s* *out* *trace-bs* *dg* *comp-bs*
and *header*:
verifier-header-transcript *s* *fr* *f-fri-roots* *f-final* *header-as* *dg*
composition-fri-roots *final* *rest*
unfolding *trace-fri-supported-root-trace-table-candidates-def* **by** *blast*
from *bound* **obtain** *result* *final-state* *fr'* *f-fri-roots'* *f-final'* *dg'*
composition-fri-roots' *final'* *rest'* **where**
out-eq: *out* = *Some* (*result*, *final-state*)
and *header'*:
verifier-header-transcript *s* *fr'* *f-fri-roots'* *f-final'* *as* *dg'*
composition-fri-roots' *final'* *rest'*
and *bind*:
merkle-root-binds-table *fr'* *trace-table* *final-state*
unfolding *accepted-with-bound-tables-def* **by** *blast*
have *fr-eq*: *fr'* = *fr*
using *verifier-header-transcript-unique*[*OF* *header'* *header*] **by** *simp*
have *state*:
trace-fri-supported-root-trace-table-candidate-state *s* *fr* *trace-table*
final-state
unfolding *trace-fri-supported-root-trace-table-candidate-state-def*
apply (*rule* *exI*[**where** *x=result*])
apply (*rule* *exI*[**where** *x=composition-table*])
apply (*rule* *exI*[**where** *x=as*])
apply (*rule* *exI*[**where** *x=query-idxs*])
apply (*rule* *exI*[**where** *x=trace-bs*])
apply (*rule* *exI*[**where** *x=dg*])
apply (*rule* *exI*[**where** *x=comp-bs*])
apply (*rule* *exI*[**where** *x=f-fri-roots*])
apply (*rule* *exI*[**where** *x=f-final*])
apply (*rule* *exI*[**where** *x=header-as*])
apply (*rule* *exI*[**where** *x=composition-fri-roots*])
apply (*rule* *exI*[**where** *x=final*])
apply (*rule* *exI*[**where** *x=rest*])
apply (*intro* *conjI*)
apply (*use* *outcome* *out-eq* **in** *simp*)
apply (*use* *bound* *out-eq* **in** *simp*)
apply (*use* *challenges* *out-eq* **in** *simp*)
apply (*rule* *header*)

```

    apply (use bind fr-eq in simp)
  done
then show
   $\exists$  final-state.
    trace-fri-supported-root-trace-table-candidate-state s fr trace-table
    final-state
  by blast
next
assume
   $\exists$  final-state.
    trace-fri-supported-root-trace-table-candidate-state s fr trace-table
    final-state
then obtain final-state result composition-table as query-idxs trace-bs dg
  comp-bs f-fri-roots f-final header-as composition-fri-roots final rest
where
  outcome:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
  and bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and challenges:
    accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
    comp-bs
  and header:
    verifier-header-transcript s fr f-fri-roots f-final header-as dg
    composition-fri-roots final rest
  unfolding trace-fri-supported-root-trace-table-candidate-state-def by blast
show trace-table  $\in$  trace-fri-supported-root-trace-table-candidates s fr
  unfolding trace-fri-supported-root-trace-table-candidates-def
  using outcome bound challenges header by blast
qed

```

lemma trace-fri-supported-root-partial-trace-table-candidates-iff-state:

```

  trace-table  $\in$ 
    trace-fri-supported-root-partial-trace-table-candidates s fr  $\longleftrightarrow$ 
    ( $\exists$  final-state.
      trace-fri-supported-root-partial-trace-table-candidate-state s fr
      trace-table final-state)

```

proof

```

  assume candidate:
    trace-table  $\in$ 
      trace-fri-supported-root-partial-trace-table-candidates s fr
  then obtain out query-idxs trace-openings trace-bs dg comp-bs
    f-fri-roots f-final header-as composition-fri-roots final rest where
    outcome: out  $\in$  set-dist (execute verify-monad s)
  and partial:
    accepted-with-partial-trace-openings s out fr query-idxs
    trace-openings
  and table:

```

```

    partial-trace-table-candidate trace-table trace-openings
and challenges:
    accepted-fri-challenges s out trace-bs dg comp-bs
and header:
    verifier-header-transcript s fr f-fri-roots f-final header-as dg
    composition-fri-roots final rest
unfolding trace-fri-supported-root-partial-trace-table-candidates-def
by blast
from partial obtain result final-state where out-eq:
    out = Some (result, final-state)
    unfolding accepted-with-partial-trace-openings-def by blast
have state:
    trace-fri-supported-root-partial-trace-table-candidate-state s fr
    trace-table final-state
proof –
    have outcome-some:
        Some (result, final-state) ∈ set-dist (execute verify-monad s)
        using outcome out-eq by simp
    have partial-some:
        accepted-with-partial-trace-openings s
        (Some (result, final-state)) fr query-idxs trace-openings
        using partial out-eq by simp
    have challenges-some:
        accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
        comp-bs
        using challenges out-eq by simp
    show ?thesis
        unfolding
            trace-fri-supported-root-partial-trace-table-candidate-state-def
        using outcome-some partial-some table challenges-some header by blast
qed
then show
    ∃ final-state.
        trace-fri-supported-root-partial-trace-table-candidate-state s fr
        trace-table final-state
    by blast
next
assume
    ∃ final-state.
        trace-fri-supported-root-partial-trace-table-candidate-state s fr
        trace-table final-state
then obtain final-state result query-idxs trace-openings trace-bs dg
    comp-bs f-fri-roots f-final header-as composition-fri-roots final rest
where
    outcome:
        Some (result, final-state) ∈ set-dist (execute verify-monad s)
and partial:
        accepted-with-partial-trace-openings s
        (Some (result, final-state)) fr query-idxs trace-openings

```


and *table*:
partial-trace-table-candidate trace-table trace-openings
and *challenges*:
accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
comp-bs
and *header*:
verifier-header-transcript s fr f-fri-roots f-final header-as dg
composition-fri-roots final rest
unfolding
trace-fri-supported-root-partial-trace-table-candidate-state-def
by *blast*
show
trace-table ∈
trace-fri-supported-root-partial-trace-table-candidates s fr
unfolding *trace-fri-supported-root-partial-trace-table-candidates-def*
using *outcome partial table challenges header* **by** *blast*
qed

lemma *trace-fri-supported-root-partial-trace-table-candidate-length*:
assumes
trace-fri-supported-root-partial-trace-table-candidate-state s fr
trace-table final-state
shows *length trace-table = scale * clength*
using *assms*
unfolding *trace-fri-supported-root-partial-trace-table-candidate-state-def*
partial-trace-table-candidate-def
by *blast*

lemma *alpha-header-supported-partial-union-bad-sets-subset-alpha-space*:
alpha-header-supported-partial-union-bad-sets s bad-sets fr f-fri-roots
f-final ⊆ alpha-space
unfolding *alpha-header-supported-partial-union-bad-sets-def* **by** *auto*

lemma *trace-fri-supported-root-trace-table-candidate-state-imp-candidate-state*:
assumes
trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state
shows
trace-fri-root-trace-table-candidate-state s fr trace-table final-state
using *assms*
unfolding *trace-fri-supported-root-trace-table-candidate-state-def*
trace-fri-root-trace-table-candidate-state-def
by *blast*

lemma *trace-fri-supported-root-trace-table-candidate-state-hash-extends*:
assumes
trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state
shows *s ≤ final-state*

proof –

from *assms* **obtain** *result composition-table as query-idxs trace-bs dg comp-bs*
 f-fri-roots f-final header-as composition-fri-roots final rest **where**
 outcome:
 Some (result, final-state) ∈ set-dist (execute verify-monad s)
 unfolding *trace-fri-supported-root-trace-table-candidate-state-def*
 by *blast*
show *?thesis*
 by *(rule verify-monad-hash-extends[OF outcome])*
qed

lemma *trace-fri-supported-root-trace-table-candidate-state-imp-partial-candidate:*

assumes *state:*
 trace-fri-supported-root-trace-table-candidate-state s fr trace-table
 final-state
and *clean:* $\neg$ *hash-map-output-collision final-state*
shows
 trace-table ∈
 trace-fri-supported-root-partial-trace-table-candidates s fr

proof –

from *state* **obtain** *result composition-table as query-idxs trace-bs dg comp-bs*
 f-fri-roots f-final header-as composition-fri-roots final rest **where**
 outcome:
 Some (result, final-state) ∈ set-dist (execute verify-monad s)
 and *bound:*
 accepted-with-bound-tables s (Some (result, final-state))
 trace-table composition-table as query-idxs
 and *challenges:*
 accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
 comp-bs
 and *header:*
 verifier-header-transcript s fr f-fri-roots f-final header-as dg
 composition-fri-roots final rest
 and *bind:*
 merkle-root-binds-table fr trace-table final-state
 unfolding *trace-fri-supported-root-trace-table-candidate-state-def*
 by *blast*
from *verify-monad-header-extraction[OF outcome]*
obtain *fr' f-fl f-final' as' dg' fl final' query-state* **where**
 header':
 verifier-header-transcript s fr' (map snd f-fl) f-final' as' dg'
 (map snd fl) final' (PTranscript query-state)
 and *query-out:*
 Some (result, final-state) ∈
 set-dist
 (execute
 (ntimes
 (verifier-query-round-program fr' f-fl f-final' as' fl final')
 rounds)

```

      query-state)
    by (rule verify-monad-header-query-extraction[OF outcome])
  have fr-eq: fr' = fr
    using verifier-header-transcript-unique[OF header' header] by simp
  obtain query-idxs' trace-openings where
    len-query-idxs: length query-idxs' = rounds
    and len-openings: length trace-openings = rounds
    and indices:
       $\bigwedge i. i < \text{rounds} \implies$ 
        map opening-index (trace-openings ! i) =
          powers-scaled (query-idxs' ! i)
    and tables:
       $\bigwedge i. i < \text{rounds} \implies$ 
        partial-authenticated-table fr'
          (scale * clength) (trace-openings ! i) final-state
  proof (rule ntimes-verifier-query-rounds-authenticated-trace-openings-no-raws
    [OF query-out])
    fix query-idxs' trace-openings
    assume len-query-idxs: length query-idxs' = rounds
    and len-openings: length trace-openings = rounds
    and indices:
       $\bigwedge i. i < \text{rounds} \implies$ 
        map opening-index (trace-openings ! i) =
          powers-scaled (query-idxs' ! i)
    and tables:
       $\bigwedge i. i < \text{rounds} \implies$ 
        partial-authenticated-table fr'
          (scale * clength) (trace-openings ! i) final-state
    show ?thesis
      by (rule that[OF len-query-idxs len-openings indices tables])
  qed
  have partial:
    accepted-with-partial-trace-openings s
      (Some (result, final-state)) fr query-idxs' trace-openings
  proof -
    have all-indices:
       $\forall i < \text{rounds}.$ 
        map opening-index (trace-openings ! i) =
          powers-scaled (query-idxs' ! i)
      using indices by blast
    have all-tables:
       $\forall i < \text{rounds}.$ 
        partial-authenticated-table fr (scale * clength)
          (trace-openings ! i) final-state
      using tables fr-eq by simp
    show ?thesis
      unfolding accepted-with-partial-trace-openings-def accepted-def
      by (intro conjI exI[of - result] exI[of - final-state])
        (use len-query-idxs len-openings all-indices all-tables in simp-all)

```

```

qed
have len-table: length trace-table = scale * clength
  using accepted-with-bound-tables-shapes(1)[OF bound] by simp
have partial-candidate:
  partial-trace-table-candidate trace-table trace-openings
  by (rule accepted-bound-trace-table-partial-candidate-if-clean
    [OF bind len-table partial clean])
show ?thesis
proof -
  have witness:
    ∃ out query-idxs trace-openings trace-bs dg comp-bs
      f-fri-roots f-final header-as composition-fri-roots final rest.
    out ∈ set-dist (execute verify-monad s) ∧
    accepted-with-partial-trace-openings s out fr query-idxs
      trace-openings ∧
    partial-trace-table-candidate trace-table trace-openings ∧
    accepted-fri-challenges s out trace-bs dg comp-bs ∧
    verifier-header-transcript s fr f-fri-roots f-final header-as dg
      composition-fri-roots final rest
  using outcome partial partial-candidate challenges header by blast
show ?thesis
  using witness
  unfolding trace-fri-supported-root-partial-trace-table-candidates-def
  by simp
qed
qed

lemma alpha-header-supported-trace-table-candidate-state-imp-partial-candidate:
  assumes state:
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table final-state
  and clean: ¬ hash-map-output-collision final-state
  shows
    trace-table ∈
      alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
      f-final
proof -
  from state obtain result composition-table as query-idxs dg
    composition-fri-roots final rest where
    outcome:
      Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and bind:
    merkle-root-binds-table fr trace-table final-state

```

```

unfolding alpha-header-supported-trace-table-candidate-state-def
by blast
from verify-monad-header-extraction[OF outcome]
obtain fr' f-fl f-final' as' dg' fl final' query-state where
  header':
    verifier-header-transcript s fr' (map snd f-fl) f-final' as' dg'
    (map snd fl) final' (PTranscript query-state)
and query-out:
  Some (result, final-state) ∈
    set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr' f-fl f-final' as' fl final')
        rounds)
      query-state)
by (rule verify-monad-header-query-extraction[OF outcome])
have eqs:
  fr' = fr ∧ map snd f-fl = f-fri-roots ∧
  f-final' = f-final ∧ as' = as ∧ dg' = dg ∧
  map snd fl = composition-fri-roots ∧ final' = final ∧
  PTranscript query-state = rest
using verifier-header-transcript-unique[OF header' header] by simp
obtain query-idxs' trace-openings where
  len-query-idxs: length query-idxs' = rounds
and len-openings: length trace-openings = rounds
and indices:
   $\bigwedge i. i < rounds \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs' ! i)
and tables:
   $\bigwedge i. i < rounds \implies$ 
    partial-authenticated-table fr'
    (scale * clength) (trace-openings ! i) final-state
proof (rule ntimes-verifier-query-rounds-authenticated-trace-openings-no-raws
  [OF query-out])
fix query-idxs' trace-openings
assume len-query-idxs: length query-idxs' = rounds
and len-openings: length trace-openings = rounds
and indices:
   $\bigwedge i. i < rounds \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs' ! i)
and tables:
   $\bigwedge i. i < rounds \implies$ 
    partial-authenticated-table fr'
    (scale * clength) (trace-openings ! i) final-state
show ?thesis
by (rule that[OF len-query-idxs len-openings indices tables])
qed

```

```

have partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr query-idxs' trace-openings
proof –
  have all-indices:
     $\forall i < \text{rounds}.$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs' ! i)
    using indices by blast
  have all-tables:
     $\forall i < \text{rounds}.$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i final-state)
    using tables eqs by simp
  show ?thesis
    unfolding accepted-with-partial-trace-openings-def accepted-def
    by (intro conjI exI[of - result] exI[of - final-state])
      (use len-query-idxs len-openings all-indices all-tables in simp-all)
qed
have len-table: length trace-table = scale * clength
  using accepted-with-bound-tables-shapes(1)[OF bound] by simp
have partial-candidate:
  partial-trace-table-candidate trace-table trace-openings
  by (rule accepted-bound-trace-table-partial-candidate-if-clean
    [OF bind len-table partial clean])
show ?thesis
  unfolding alpha-header-supported-partial-trace-table-candidates-def
  by (intro CollectI exI[of - Some (result, final-state)]
    exI[of - query-idxs'] exI[of - trace-openings]
    exI[of - as] exI[of - dg]
    exI[of - composition-fri-roots] exI[of - final]
    exI[of - rest] conjI)
    (use outcome partial partial-candidate header in simp-all)
qed

lemma trace-fri-root-trace-table-candidate-states-common-extension-unique-if-clean:
assumes clean:  $\neg$  hash-map-output-collision u
and candidate1:
  trace-fri-root-trace-table-candidate-state s fr trace-table
  final-state
and candidate2:
  trace-fri-root-trace-table-candidate-state s fr trace-table'
  final-state'
and ext1: final-state  $\leq$  u
and ext2: final-state'  $\leq$  u
shows trace-table = trace-table'
proof –
from candidate1 obtain result composition-table as query-idxs trace-bs dg
  comp-bs f-fri-roots f-final header-as composition-fri-roots final rest

```

```

where
  bound1:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and bind1: merkle-root-binds-table fr trace-table final-state
  unfolding trace-fri-root-trace-table-candidate-state-def by blast
from candidate2 obtain result' composition-table' as' query-idxs' trace-bs'
  dg' comp-bs' f-fri-roots' f-final' header-as'
  composition-fri-roots' final' rest' where
  bound2:
    accepted-with-bound-tables s (Some (result', final-state'))
    trace-table' composition-table' as' query-idxs'
  and bind2: merkle-root-binds-table fr trace-table' final-state'
  unfolding trace-fri-root-trace-table-candidate-state-def by blast
have same-len: length trace-table = length trace-table'
  using accepted-with-bound-tables-shapes(1)[OF bound1]
  accepted-with-bound-tables-shapes(1)[OF bound2]
  by simp
have nonempty: trace-table ≠ []
  using accepted-with-bound-tables-shapes(1)[OF bound1]
  eval-domain-nontrivial by auto
show ?thesis
  by (rule merkle-root-binds-same-length-tables-common-extension-unique-if-clean
    [OF clean bind1 bind2 ext1 ext2 same-len nonempty])
qed

```

lemma trace-fri-root-trace-table-candidate-states-compatible-unique-if-clean-merge:

assumes compatible: hash-maps-compatible final-state final-state'

and clean:

¬ hash-map-output-collision
 (hash-state-merge final-state final-state')

and candidate1:

trace-fri-root-trace-table-candidate-state s fr trace-table
 final-state

and candidate2:

trace-fri-root-trace-table-candidate-state s fr trace-table'
 final-state'

shows trace-table = trace-table'

proof –

from candidate1 **obtain** result composition-table as query-idxs trace-bs dg
 comp-bs f-fri-roots f-final header-as composition-fri-roots final rest

where

bound1:

accepted-with-bound-tables s (Some (result, final-state))
 trace-table composition-table as query-idxs

and bind1: merkle-root-binds-table fr trace-table final-state

unfolding trace-fri-root-trace-table-candidate-state-def **by** blast

from candidate2 **obtain** result' composition-table' as' query-idxs' trace-bs'
 dg' comp-bs' f-fri-roots' f-final' header-as'

```

    composition-fri-roots' final' rest' where
bound2:
    accepted-with-bound-tables s (Some (result', final-state'))
    trace-table' composition-table' as' query-idxs'
and bind2: merkle-root-binds-table fr trace-table' final-state'
unfolding trace-fri-root-trace-table-candidate-state-def by blast
have same-len: length trace-table = length trace-table'
using accepted-with-bound-tables-shapes(1)[OF bound1]
    accepted-with-bound-tables-shapes(1)[OF bound2]
by simp
have nonempty: trace-table ≠ []
using accepted-with-bound-tables-shapes(1)[OF bound1]
    eval-domain-nontrivial by auto
show ?thesis
by (rule
    merkle-root-binds-same-length-tables-compatible-states-unique-if-clean-merge
    [OF compatible clean bind1 bind2 same-len nonempty])
qed

lemma trace-fri-root-trace-table-candidate-states-merkle-compatible-unique-if-clean-merge:
assumes compatible:
    merkle-hash-maps-compatible final-state final-state'
and clean:
    ¬ hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
and candidate1:
    trace-fri-root-trace-table-candidate-state s fr trace-table
    final-state
and candidate2:
    trace-fri-root-trace-table-candidate-state s fr trace-table'
    final-state'
shows trace-table = trace-table'
proof –
from candidate1 obtain result composition-table as query-idxs trace-bs dg
    comp-bs f-fri-roots f-final header-as composition-fri-roots final rest
where
bound1:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
and bind1: merkle-root-binds-table fr trace-table final-state
unfolding trace-fri-root-trace-table-candidate-state-def by blast
from candidate2 obtain result' composition-table' as' query-idxs' trace-bs'
    dg' comp-bs' f-fri-roots' f-final' header-as'
    composition-fri-roots' final' rest' where
bound2:
    accepted-with-bound-tables s (Some (result', final-state'))
    trace-table' composition-table' as' query-idxs'
and bind2: merkle-root-binds-table fr trace-table' final-state'
unfolding trace-fri-root-trace-table-candidate-state-def by blast

```



```

have same-len: length trace-table = length trace-table'
using accepted-with-bound-tables-shapes(1)[OF bound1]
      accepted-with-bound-tables-shapes(1)[OF bound2]
by simp
have nonempty: trace-table ≠ []
using accepted-with-bound-tables-shapes(1)[OF bound1]
      eval-domain-nontrivial by auto
show ?thesis
by (rule
      merkle-root-binds-same-length-tables-merkle-compatible-unique-if-clean-merge
      [OF compatible clean bind1 bind2 same-len nonempty])
qed

lemma trace-fri-root-trace-table-candidates-subset-singleton-if-common-clean-extension:
assumes clean: ¬ hash-map-output-collision u
and common-ext:
  ∧ trace-table final-state.
  trace-fri-root-trace-table-candidate-state s fr trace-table
  final-state ⇒
  final-state ≤ u
shows
  ∃ trace-table.
  trace-fri-root-trace-table-candidates s fr ⊆ {trace-table}
proof (cases trace-fri-root-trace-table-candidates s fr = {})
case True
then show ?thesis
by (intro exI[of - []]) simp
next
case False
then obtain trace-table0 where table0:
  trace-table0 ∈ trace-fri-root-trace-table-candidates s fr
by blast
then obtain final-state0 where state0:
  trace-fri-root-trace-table-candidate-state s fr trace-table0
  final-state0
using trace-fri-root-trace-table-candidates-iff-state by blast
have ext0: final-state0 ≤ u
by (rule common-ext[OF state0])
have subset:
  trace-fri-root-trace-table-candidates s fr ⊆ {trace-table0}
proof
fix trace-table
assume table:
  trace-table ∈ trace-fri-root-trace-table-candidates s fr
then obtain final-state where state:
  trace-fri-root-trace-table-candidate-state s fr trace-table
  final-state
using trace-fri-root-trace-table-candidates-iff-state by blast
have ext: final-state ≤ u

```

```

    by (rule common-ext[OF state])
  have trace-table = trace-table0
    by (rule
        trace-fri-root-trace-table-candidate-states-common-extension-unique-if-clean
        [OF clean state state0 ext ext0])
  then show trace-table ∈ {trace-table0}
    by simp
qed
then show ?thesis
  by blast
qed

```

lemma *trace-fri-root-trace-table-candidates-subset-singleton-if-pairwise-clean-merge:*

assumes *pairwise:*

$\bigwedge \text{trace-table final-state trace-table' final-state'.$
 $\text{trace-fri-root-trace-table-candidate-state } s \text{ fr trace-table}$
 $\text{final-state} \implies$
 $\text{trace-fri-root-trace-table-candidate-state } s \text{ fr trace-table'}$
 $\text{final-state'} \implies$
 $\text{hash-maps-compatible final-state final-state'} \wedge$
 $\neg \text{hash-map-output-collision}$
 $(\text{hash-state-merge final-state final-state'})$

shows

$\exists \text{trace-table.}$
 $\text{trace-fri-root-trace-table-candidates } s \text{ fr} \subseteq \{\text{trace-table}\}$

proof (cases *trace-fri-root-trace-table-candidates s fr = {}*)

case *True*

then show *?thesis*

by (intro *exI*[of - []]) *simp*

next

case *False*

then obtain *trace-table0* **where** *table0:*

trace-table0 ∈ *trace-fri-root-trace-table-candidates s fr*

by *blast*

then obtain *final-state0* **where** *state0:*

trace-fri-root-trace-table-candidate-state s fr trace-table0

final-state0

using *trace-fri-root-trace-table-candidates-iff-state* **by** *blast*

have *subset:*

trace-fri-root-trace-table-candidates s fr ⊆ {*trace-table0*}

proof

fix *trace-table*

assume *table:*

trace-table ∈ *trace-fri-root-trace-table-candidates s fr*

then obtain *final-state* **where** *state:*

trace-fri-root-trace-table-candidate-state s fr trace-table

final-state

using *trace-fri-root-trace-table-candidates-iff-state* **by** *blast*

have *compatible:*

```

    hash-maps-compatible final-state final-state0
    using pairwise[OF state state0] by simp
  have clean:
    ¬ hash-map-output-collision
      (hash-state-merge final-state final-state0)
    using pairwise[OF state state0] by simp
  have trace-table = trace-table0
    by (rule
      trace-fri-root-trace-table-candidate-states-compatible-unique-if-clean-merge
      [OF compatible clean state state0])
  then show trace-table ∈ {trace-table0}
    by simp
qed
then show ?thesis
  by blast
qed

lemma trace-fri-supported-root-trace-table-candidates-subset-singleton-if-pairwise-clean-merge:
  assumes pairwise:
    ∧ trace-table final-state trace-table' final-state'.
      trace-fri-supported-root-trace-table-candidate-state s fr trace-table
      final-state ⇒
      trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
      final-state' ⇒
      hash-maps-compatible final-state final-state' ∧
      ¬ hash-map-output-collision
        (hash-state-merge final-state final-state')
  shows
    ∃ trace-table.
      trace-fri-supported-root-trace-table-candidates s fr ⊆
        {trace-table}
proof (cases trace-fri-supported-root-trace-table-candidates s fr = {})
case True
  then show ?thesis
    by (intro exI[of - []]) simp
next
case False
  then obtain trace-table0 where table0:
    trace-table0 ∈ trace-fri-supported-root-trace-table-candidates s fr
  by blast
  then obtain final-state0 where state0:
    trace-fri-supported-root-trace-table-candidate-state s fr trace-table0
    final-state0
  using trace-fri-supported-root-trace-table-candidates-iff-state by blast
  have state0-plain:
    trace-fri-root-trace-table-candidate-state s fr trace-table0
    final-state0
  by (rule
    trace-fri-supported-root-trace-table-candidate-state-imp-candidate-state

```

```

    [OF state0])
  have subset:
    trace-fri-supported-root-trace-table-candidates s fr  $\subseteq$ 
      {trace-table0}
  proof
    fix trace-table
    assume table:
      trace-table  $\in$  trace-fri-supported-root-trace-table-candidates s fr
    then obtain final-state where state:
      trace-fri-supported-root-trace-table-candidate-state s fr trace-table
      final-state
    using trace-fri-supported-root-trace-table-candidates-iff-state by blast
  have state-plain:
    trace-fri-root-trace-table-candidate-state s fr trace-table
    final-state
  by (rule
    trace-fri-supported-root-trace-table-candidate-state-imp-candidate-state
    [OF state])
  have compatible:
    hash-maps-compatible final-state final-state0
  using pairwise[OF state state0] by simp
  have clean:
     $\neg$  hash-map-output-collision
    (hash-state-merge final-state final-state0)
  using pairwise[OF state state0] by simp
  have trace-table = trace-table0
  by (rule
    trace-fri-root-trace-table-candidate-states-compatible-unique-if-clean-merge
    [OF compatible clean state-plain state0-plain])
  then show trace-table  $\in$  {trace-table0}
  by simp
qed
then show ?thesis
  by blast
qed

lemma trace-fri-supported-root-trace-table-candidates-subset-singleton-if-pairwise-no-value-conflict-clean-merge:
  assumes pairwise:
     $\bigwedge$  trace-table final-state trace-table' final-state'.
      trace-fri-supported-root-trace-table-candidate-state s fr trace-table
      final-state  $\implies$ 
      trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
      final-state'  $\implies$ 
       $\neg$  hash-map-value-conflict final-state final-state'  $\wedge$ 
       $\neg$  hash-map-output-collision
      (hash-state-merge final-state final-state')
  shows
     $\exists$  trace-table.
      trace-fri-supported-root-trace-table-candidates s fr  $\subseteq$ 

```

```

      {trace-table}
proof (rule
  trace-fri-supported-root-trace-table-candidates-subset-singleton-if-pairwise-clean-merge)
fix trace-table final-state trace-table' final-state'
assume candidate:
  trace-fri-supported-root-trace-table-candidate-state s fr trace-table
  final-state
and candidate':
  trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
  final-state'
have no-conflict:
  ¬ hash-map-value-conflict final-state final-state'
  using pairwise[OF candidate candidate'] by simp
have compatible:
  hash-maps-compatible final-state final-state'
  using no-conflict
  unfolding hash-maps-compatible-iff-no-value-conflict .
have clean:
  ¬ hash-map-output-collision
  (hash-state-merge final-state final-state')
  using pairwise[OF candidate candidate'] by simp
show hash-maps-compatible final-state final-state' ∧
  ¬ hash-map-output-collision
  (hash-state-merge final-state final-state')
  using compatible clean by simp
qed

```

lemma trace-fri-supported-root-trace-table-candidates-subset-singleton-if-pairwise-merkle-clean-merge:

```

assumes pairwise:
  ∧ trace-table final-state trace-table' final-state'.
  trace-fri-supported-root-trace-table-candidate-state s fr trace-table
  final-state ⇒
  trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
  final-state' ⇒
  merkle-hash-maps-compatible final-state final-state' ∧
  ¬ hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')
shows
  ∃ trace-table.
  trace-fri-supported-root-trace-table-candidates s fr ⊆
  {trace-table}
proof (cases trace-fri-supported-root-trace-table-candidates s fr = {})
case True
  then show ?thesis
  by (intro exI[of - []]) simp
next
case False
  then obtain trace-table0 where table0:
  trace-table0 ∈ trace-fri-supported-root-trace-table-candidates s fr

```

```

    by blast
  then obtain final-state0 where state0:
    trace-fri-supported-root-trace-table-candidate-state s fr trace-table0
    final-state0
    using trace-fri-supported-root-trace-table-candidates-iff-state by blast
  have state0-plain:
    trace-fri-root-trace-table-candidate-state s fr trace-table0
    final-state0
  by (rule
    trace-fri-supported-root-trace-table-candidate-state-imp-candidate-state
    [OF state0])
  have subset:
    trace-fri-supported-root-trace-table-candidates s fr  $\subseteq$ 
    {trace-table0}
  proof
    fix trace-table
    assume table:
      trace-table  $\in$  trace-fri-supported-root-trace-table-candidates s fr
    then obtain final-state where state:
      trace-fri-supported-root-trace-table-candidate-state s fr trace-table
      final-state
      using trace-fri-supported-root-trace-table-candidates-iff-state by blast
    have state-plain:
      trace-fri-root-trace-table-candidate-state s fr trace-table
      final-state
    by (rule
      trace-fri-supported-root-trace-table-candidate-state-imp-candidate-state
      [OF state])
    have compatible:
      merkle-hash-maps-compatible final-state final-state0
      using pairwise[OF state state0] by simp
    have clean:
       $\neg$  hash-map-output-collision
      (merkle-hash-state-merge final-state final-state0)
      using pairwise[OF state state0] by simp
    have trace-table = trace-table0
    by (rule
      trace-fri-root-trace-table-candidate-states-merkle-compatible-unique-if-clean-merge
      [OF compatible clean state-plain state0-plain])
    then show trace-table  $\in$  {trace-table0}
    by simp
  qed
  then show ?thesis
  by blast
qed

```

lemma trace-fri-supported-root-trace-table-candidates-subset-singleton-if-pairwise-no-merkle-value-conflict-clean-merge
assumes pairwise:
 $\bigwedge \text{trace-table final-state trace-table' final-state'.$

```

    trace-fri-supported-root-trace-table-candidate-state s fr trace-table
    final-state  $\implies$ 
    trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
    final-state'  $\implies$ 
     $\neg$  merkle-hash-value-conflict final-state final-state'  $\wedge$ 
     $\neg$  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
  shows
     $\exists$  trace-table.
    trace-fri-supported-root-trace-table-candidates s fr  $\subseteq$ 
    {trace-table}
  proof (rule
    trace-fri-supported-root-trace-table-candidates-subset-singleton-if-pairwise-merkle-clean-merge)
  fix trace-table final-state trace-table' final-state'
  assume candidate:
    trace-fri-supported-root-trace-table-candidate-state s fr trace-table
    final-state
  and candidate':
    trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
    final-state'
  have no-conflict:
     $\neg$  merkle-hash-value-conflict final-state final-state'
  using pairwise[OF candidate candidate'] by simp
  have compatible:
    merkle-hash-maps-compatible final-state final-state'
  using no-conflict
  unfolding merkle-hash-maps-compatible-iff-no-value-conflict .
  have clean:
     $\neg$  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
  using pairwise[OF candidate candidate'] by simp
  show merkle-hash-maps-compatible final-state final-state'  $\wedge$ 
     $\neg$  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
  using compatible clean by simp
qed

lemma trace-fri-supported-root-trace-table-candidates-subset-singleton-if-common-clean-merkle-extension:
  assumes clean:  $\neg$  hash-map-output-collision u
  and common-ext:
     $\bigwedge$  trace-table final-state.
    trace-fri-supported-root-trace-table-candidate-state s fr trace-table
    final-state  $\implies$ 
    merkle-hash-extends final-state u
  shows
     $\exists$  trace-table.
    trace-fri-supported-root-trace-table-candidates s fr  $\subseteq$ 
    {trace-table}
  proof (rule

```

trace-fri-supported-root-trace-table-candidates-subset-singleton-if-pairwise-no-merkle-value-conflict-clean-merge)
fix *trace-table final-state trace-table' final-state'*
assume *candidate*:
 trace-fri-supported-root-trace-table-candidate-state s fr trace-table
 final-state
and *candidate'*:
 trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
 final-state'
have *ext*: *merkle-hash-extends final-state u*
 by (*rule common-ext[OF candidate]*)
have *ext'*: *merkle-hash-extends final-state' u*
 by (*rule common-ext[OF candidate']*)
show $\neg$ *merkle-hash-value-conflict final-state final-state' $\wedge$*
 $\neg$ *hash-map-output-collision*
 (*merkle-hash-state-merge final-state final-state'*)
 by (*rule common-clean-merkle-extension-imp-pairwise-merkle-clean*
 [*OF ext ext' clean*])
qed

lemma *trace-fri-supported-root-trace-table-candidate-states-distinct-imp-merkle-conflict-or-merge-collision*:
assumes *candidate*:
 trace-fri-supported-root-trace-table-candidate-state s fr trace-table
 final-state
and *candidate'*:
 trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
 final-state'
and *distinct-tables*: *trace-table $\neq$ trace-table'*
shows
 merkle-hash-value-conflict final-state final-state' $\vee$
 hash-map-output-collision
 (*merkle-hash-state-merge final-state final-state'*)
proof (*rule ccontr*)
assume *no-bad*:
 $\neg$ (*merkle-hash-value-conflict final-state final-state' $\vee$*
 hash-map-output-collision
 (*merkle-hash-state-merge final-state final-state'*))
have *compatible*:
 merkle-hash-maps-compatible final-state final-state'
 using *no-bad unfolding merkle-hash-maps-compatible-iff-no-value-conflict*
 by *simp*
have *clean*:
 $\neg$ *hash-map-output-collision*
 (*merkle-hash-state-merge final-state final-state'*)
 using *no-bad by simp*
have *candidate-plain*:
 trace-fri-root-trace-table-candidate-state s fr trace-table
 final-state
 by (*rule*
 trace-fri-supported-root-trace-table-candidate-state-imp-candidate-state


```

    [OF candidate])
  have candidate-plain':
    trace-fri-root-trace-table-candidate-state s fr trace-table'
    final-state'
  by (rule
    trace-fri-supported-root-trace-table-candidate-state-imp-candidate-state
    [OF candidate])
  have trace-table = trace-table'
  by (rule
    trace-fri-root-trace-table-candidate-states-merkle-compatible-unique-if-clean-merge
    [OF compatible clean candidate-plain candidate-plain])
  then show False
  using distinct-tables by contradiction
qed

lemma trace-fri-supported-root-trace-table-candidates-not-singleton-imp-pairwise-merkle-bad:
  assumes not-unique:
    ¬ (∃ trace-table.
      trace-fri-supported-root-trace-table-candidates s fr ⊆
        {trace-table})
  shows
    ∃ trace-table final-state trace-table' final-state'.
      trace-fri-supported-root-trace-table-candidate-state s fr trace-table
      final-state ∧
      trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
      final-state' ∧
      trace-table ≠ trace-table' ∧
      (merkle-hash-value-conflict final-state final-state' ∨
      hash-map-output-collision
      (merkle-hash-state-merge final-state final-state'))
  proof -
    let ?C = trace-fri-supported-root-trace-table-candidates s fr
    have nonempty: ?C ≠ {}
    using not-unique by auto
    then obtain trace-table where table: trace-table ∈ ?C
    by blast
    have ¬ ?C ⊆ {trace-table}
    using not-unique by blast
    then obtain trace-table' where table':
      trace-table' ∈ ?C
    and distinct-tables: trace-table ≠ trace-table'
    by auto
    from table obtain final-state where candidate:
      trace-fri-supported-root-trace-table-candidate-state s fr trace-table
      final-state
    using trace-fri-supported-root-trace-table-candidates-iff-state by blast
    from table' obtain final-state' where candidate':
      trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
      final-state'

```

using *trace-fri-supported-root-trace-table-candidates-iff-state* **by** *blast*
have *bad*:
merkle-hash-value-conflict final-state final-state' ∨
hash-map-output-collision
(merkle-hash-state-merge final-state final-state')
by (rule
trace-fri-supported-root-trace-table-candidate-states-distinct-imp-merkle-conflict-or-merge-collision
[OF candidate candidate' distinct-tables])
show *?thesis*
using *candidate candidate' distinct-tables bad* **by** *blast*
qed

definition *trace-fri-supported-root-pairwise-merkle-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow \text{bool}$
where
trace-fri-supported-root-pairwise-merkle-bad s fr $\longleftrightarrow$
 $(\exists \text{ trace-table final-state trace-table' final-state'.$
trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state $\wedge$
trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
final-state' $\wedge$
trace-table $\neq$ *trace-table'* $\wedge$
 $(\text{merkle-hash-value-conflict final-state final-state'} \vee$
 $\text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge final-state final-state'})))$

definition *trace-fri-supported-pairwise-merkle-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
trace-fri-supported-pairwise-merkle-bad s $\longleftrightarrow$
 $(\exists \text{ fr. trace-fri-supported-root-pairwise-merkle-bad s fr})$

lemma *trace-fri-supported-root-pairwise-merkle-bad-extends-initial*:
assumes *bad: trace-fri-supported-root-pairwise-merkle-bad s fr*
obtains *trace-table final-state trace-table' final-state'*
where
trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state
trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
final-state'
trace-table $\neq$ *trace-table'*
 $\text{merkle-hash-value-conflict final-state final-state'} \vee$
 $\text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge final-state final-state'})$
 $s \leq \text{final-state}$
 $s \leq \text{final-state'}$

proof –
from *bad* **obtain** *trace-table final-state trace-table' final-state'* **where**
candidate:

trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state
and *candidate'*:
trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
final-state'
and *distinct*: *trace-table* $\neq$ *trace-table'*
and *pair-bad*:
merkle-hash-value-conflict final-state final-state' $\vee$
hash-map-output-collision
(merkle-hash-state-merge final-state final-state')
unfolding *trace-fri-supported-root-pairwise-merkle-bad-def* **by** *blast*
have *ext*: *s* $\leq$ *final-state*
by (rule *trace-fri-supported-root-trace-table-candidate-state-hash-extends*
[*OF candidate*])
have *ext'*: *s* $\leq$ *final-state'*
by (rule *trace-fri-supported-root-trace-table-candidate-state-hash-extends*
[*OF candidate'*])
show *?thesis*
by (rule *that[OF candidate candidate' distinct pair-bad ext ext']*)
qed

definition *trace-fri-supported-root-pairwise-coupling-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow f \Rightarrow \text{bool}$
where
trace-fri-supported-root-pairwise-coupling-bad s fr $\longleftrightarrow$
 $(\exists \text{ trace-table final-state trace-table' final-state'.$
trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state $\wedge$
trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
final-state' $\wedge$
trace-table $\neq$ *trace-table'* $\wedge$
merkle-hash-pairwise-coupling-bad final-state final-state')

definition *trace-fri-supported-pairwise-coupling-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
trace-fri-supported-pairwise-coupling-bad s $\longleftrightarrow$
 $(\exists \text{ fr. trace-fri-supported-root-pairwise-coupling-bad s fr})$

lemma *trace-fri-supported-root-pairwise-merkle-bad-imp-local-collision-or-coupling-bad*:
assumes *bad*: *trace-fri-supported-root-pairwise-merkle-bad s fr*
shows
 $(\exists \text{ trace-table final-state trace-table' final-state'.$
trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state $\wedge$
trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
final-state' $\wedge$
trace-table $\neq$ *trace-table'* $\wedge$
(hash-map-output-collision final-state $\vee$

$\text{hash-map-output-collision } \text{final-state}') \vee$
 $\text{trace-fri-supported-root-pairwise-coupling-bad } s \text{ fr}$
proof –
from *bad* **obtain** $\text{trace-table } \text{final-state } \text{trace-table}' \text{ final-state}'$
where *candidate*:
 $\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr } \text{trace-table}$
 final-state
and *candidate'*:
 $\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr } \text{trace-table}'$
 $\text{final-state}'$
and *distinct*: $\text{trace-table} \neq \text{trace-table}'$
and *pair-bad*:
 $\text{merkle-hash-value-conflict } \text{final-state } \text{final-state}' \vee$
 $\text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge } \text{final-state } \text{final-state}')$
unfolding $\text{trace-fri-supported-root-pairwise-merkle-bad-def}$ **by** *blast*
have *decomp*:
 $\text{hash-map-output-collision } \text{final-state} \vee$
 $\text{hash-map-output-collision } \text{final-state}' \vee$
 $\text{merkle-hash-pairwise-coupling-bad } \text{final-state } \text{final-state}'$
by (rule $\text{merkle-hash-pairwise-bad-imp-local-collision-or-coupling-bad}$
 $[\text{OF } \text{pair-bad}]$)
show *?thesis*
using *candidate candidate' distinct decomp*
unfolding $\text{trace-fri-supported-root-pairwise-coupling-bad-def}$ **by** *blast*
qed

lemma $\text{trace-fri-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad}$:
assumes *bad*: $\text{trace-fri-supported-pairwise-merkle-bad } s$
shows
 $(\exists \text{ fr } \text{trace-table } \text{final-state } \text{trace-table}' \text{ final-state}'.$
 $\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr } \text{trace-table}$
 $\text{final-state} \wedge$
 $\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr } \text{trace-table}'$
 $\text{final-state}' \wedge$
 $\text{trace-table} \neq \text{trace-table}' \wedge$
 $(\text{hash-map-output-collision } \text{final-state} \vee$
 $\text{hash-map-output-collision } \text{final-state}') \vee$
 $\text{trace-fri-supported-pairwise-coupling-bad } s$
proof –
from *bad* **obtain** *fr* **where** *root-bad*:
 $\text{trace-fri-supported-root-pairwise-merkle-bad } s \text{ fr}$
unfolding $\text{trace-fri-supported-pairwise-merkle-bad-def}$ **by** *blast*
from
 $\text{trace-fri-supported-root-pairwise-merkle-bad-imp-local-collision-or-coupling-bad}$
 $[\text{OF } \text{root-bad}]$
show *?thesis*
unfolding $\text{trace-fri-supported-pairwise-coupling-bad-def}$ **by** *blast*
qed

lemma *trace-fri-supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad*:

assumes *bad*: *trace-fri-supported-pairwise-merkle-bad s*

shows

supported-hash-output-collision-possible s $\vee$
trace-fri-supported-pairwise-coupling-bad s

proof –

have *decomp*:

($\exists$ *fr trace-table final-state trace-table' final-state'*.
trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state $\wedge$
trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
final-state' $\wedge$
trace-table $\neq$ *trace-table'* $\wedge$
(*hash-map-output-collision final-state* $\vee$
hash-map-output-collision final-state')) $\vee$
trace-fri-supported-pairwise-coupling-bad s

by (*rule*
trace-fri-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad
[*OF bad*])

then show *?thesis*

proof

assume *local*:

$\exists$ *fr trace-table final-state trace-table' final-state'*.
trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state $\wedge$
trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
final-state' $\wedge$
trace-table $\neq$ *trace-table'* $\wedge$
(*hash-map-output-collision final-state* $\vee$
hash-map-output-collision final-state')

then obtain *fr trace-table final-state trace-table' final-state'* **where**
candidate:

trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state

and *candidate'*:

trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
final-state'

and *collision*:

hash-map-output-collision final-state $\vee$
hash-map-output-collision final-state'

by *blast*

from *collision* **show** *?thesis*

proof

assume *collision-final*: *hash-map-output-collision final-state*

from *candidate* **obtain** *result composition-table as query-idxs trace-bs dg*
comp-bs f-fri-roots f-final header-as composition-fri-roots final
rest **where**
outcome:

$\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute verify-monad } s)$
and *bound*:
 $\text{accepted-with-bound-tables } s \text{ (Some } (\text{result}, \text{final-state}))$
 $\text{trace-table composition-table as query-idxs}$
unfolding *trace-fri-supported-root-trace-table-candidate-state-def*
by *blast*
have *supported-hash-output-collision-possible s*
by (rule
 $\text{accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible}$
 $[\text{OF outcome bound collision-final}])$
then show *?thesis* **by** *simp*
next
assume *collision-final': hash-map-output-collision final-state'*
from *candidate' obtain result' composition-table' as' query-idxs'*
 $\text{trace-bs' dg' comp-bs' f-fri-roots' f-final' header-as'}$
 $\text{composition-fri-roots' final' rest' where}$
 outcome':
 $\text{Some } (\text{result}', \text{final-state}') \in \text{set-dist } (\text{execute verify-monad } s)$
and *bound'*:
 $\text{accepted-with-bound-tables } s \text{ (Some } (\text{result}', \text{final-state}'))$
 $\text{trace-table' composition-table' as' query-idxs'}$
unfolding *trace-fri-supported-root-trace-table-candidate-state-def*
by *blast*
have *supported-hash-output-collision-possible s*
by (rule
 $\text{accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible}$
 $[\text{OF outcome' bound' collision-final'}])$
then show *?thesis* **by** *simp*
qed
next
assume *trace-fri-supported-pairwise-coupling-bad s*
then show *?thesis* **by** *simp*
qed
qed

lemma *trace-fri-supported-root-candidate-unique-if-no-pairwise-merkle-bad:*
assumes *no-bad: $\neg \text{trace-fri-supported-root-pairwise-merkle-bad } s \text{ fr}$*
shows
 $\exists \text{ trace-table.}$
 $\text{trace-fri-supported-root-trace-table-candidates } s \text{ fr} \subseteq$
 $\{\text{trace-table}\}$
proof (rule *ccontr*)
assume *not-unique:*
 $\neg (\exists \text{ trace-table.}$
 $\text{trace-fri-supported-root-trace-table-candidates } s \text{ fr} \subseteq$
 $\{\text{trace-table}\})$
have *trace-fri-supported-root-pairwise-merkle-bad s fr*
using
 $\text{trace-fri-supported-root-trace-table-candidates-not-singleton-imp-pairwise-merkle-bad}$

$[OF \text{ not-unique}]$
unfolding *trace-fri-supported-root-pairwise-merkle-bad-def*
by *blast*
then show *False*
using *no-bad* **by** *contradiction*
qed

lemma *trace-fri-supported-root-candidate-unique-if-no-global-pairwise-merkle-bad:*
assumes *no-bad: $\neg \text{trace-fri-supported-pairwise-merkle-bad } s$*
shows
 $\exists \text{ trace-table.}$
 $\text{trace-fri-supported-root-trace-table-candidates } s \text{ fr} \subseteq$
 $\{\text{trace-table}\}$
proof (*rule trace-fri-supported-root-candidate-unique-if-no-pairwise-merkle-bad*)
show $\neg \text{trace-fri-supported-root-pairwise-merkle-bad } s \text{ fr}$
using *no-bad* **unfolding** *trace-fri-supported-pairwise-merkle-bad-def* **by** *blast*
qed

definition *trace-fri-root-union-bad-sets*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \text{ list} \Rightarrow f \text{ list set}) \Rightarrow f \Rightarrow f \text{ list set}$
where
 $\text{trace-fri-root-union-bad-sets } s \text{ bad-sets fr} =$
 $\{\text{trace-bs} \in \text{fri-challenge-space } (\text{ceil-log clength}).$
 $\exists \text{ trace-table} \in \text{trace-fri-root-trace-table-candidates } s \text{ fr.}$
 $\text{trace-bs} \in \text{bad-sets trace-table}\}$

definition *trace-fri-supported-root-union-bad-sets*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \text{ list} \Rightarrow f \text{ list set}) \Rightarrow f \Rightarrow f \text{ list set}$
where
 $\text{trace-fri-supported-root-union-bad-sets } s \text{ bad-sets fr} =$
 $\{\text{trace-bs} \in \text{fri-challenge-space } (\text{ceil-log clength}).$
 $\exists \text{ trace-table} \in$
 $\text{trace-fri-supported-root-trace-table-candidates } s \text{ fr.}$
 $\text{trace-bs} \in \text{bad-sets trace-table}\}$

definition *trace-fri-supported-root-partial-union-bad-sets*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \text{ list} \Rightarrow f \text{ list set}) \Rightarrow f \Rightarrow f \text{ list set}$
where
 $\text{trace-fri-supported-root-partial-union-bad-sets } s \text{ bad-sets fr} =$
 $\{\text{trace-bs} \in \text{fri-challenge-space } (\text{ceil-log clength}).$
 $\exists \text{ trace-table} \in$
 $\text{trace-fri-supported-root-partial-trace-table-candidates } s \text{ fr.}$
 $\text{trace-bs} \in \text{bad-sets trace-table}\}$

lemma *trace-fri-root-union-bad-sets-subset:*
 $\text{trace-fri-root-union-bad-sets } s \text{ bad-sets fr} \subseteq$

fri-challenge-space (*ceil-log clength*)
unfolding *trace-fri-root-union-bad-sets-def* **by** *auto*

lemma *trace-fri-supported-root-union-bad-sets-subset*:
trace-fri-supported-root-union-bad-sets s bad-sets fr $\subseteq$
fri-challenge-space (*ceil-log clength*)
unfolding *trace-fri-supported-root-union-bad-sets-def* **by** *auto*

lemma *trace-fri-supported-root-partial-union-bad-sets-subset*:
trace-fri-supported-root-partial-union-bad-sets s bad-sets fr $\subseteq$
fri-challenge-space (*ceil-log clength*)
unfolding *trace-fri-supported-root-partial-union-bad-sets-def* **by** *auto*

lemma *trace-fri-challenge-set-hit-imp-root-union-list-set-hit*:
assumes *hit*: *trace-fri-challenge-set-hit s bad-sets out*
shows

trace-fri-root-list-set-hit s
(*trace-fri-root-union-bad-sets s bad-sets*) *out*

proof —

from *hit* **obtain** *trace-table composition-table as query-idxs trace-bs dg comp-bs*

where *bound*:

accepted-with-bound-tables s out trace-table composition-table as
query-idxs

and *challenges*: *accepted-fri-challenges s out trace-bs dg comp-bs*

and *trace-bs-bad*: *trace-bs* $\in$ *bad-sets trace-table*

unfolding *trace-fri-challenge-set-hit-def* **by** *blast*

have *header-ex*:

$\exists$ *fr f-fri-roots f-final header-as composition-fri-roots final rest*.
verifier-header-transcript s fr f-fri-roots f-final header-as dg
composition-fri-roots final rest

proof —

from *challenges[unfolded accepted-fri-challenges-def]* **show** *?thesis*

proof (*elim exE conjE*)

fix *result0 final-state0 fr0 f-fl0 f-final0 as0 fl0 final0*
query-state0

assume *out0*: *out* = *Some (result0, final-state0)*

and *header0*:

verifier-header-transcript s fr0 (map snd f-fl0) f-final0 as0 dg
(*map snd fl0*) *final0 (PTranscript query-state0)*

show *?thesis*

apply (*rule exI[where x=fr0]*)

apply (*rule exI[where x=map snd f-fl0]*)

apply (*rule exI[where x=f-final0]*)

apply (*rule exI[where x=as0]*)

apply (*rule exI[where x=map snd fl0]*)

apply (*rule exI[where x=final0]*)

apply (*rule exI[where x=PTranscript query-state0]*)

apply (*rule header0*)

done


```

qed
qed
then obtain fr f-fri-roots f-final header-as composition-fri-roots final
  rest where
  header:
    verifier-header-transcript s fr f-fri-roots f-final header-as dg
    composition-fri-roots final rest
  by blast
have candidate:
  trace-table  $\in$  trace-fri-root-trace-table-candidates s fr
  unfolding trace-fri-root-trace-table-candidates-def
  using bound challenges header by blast
have trace-bs-space:
  trace-bs  $\in$  fri-challenge-space (ceil-log clength)
  by (rule accepted-fri-challenges-trace-space[OF challenges])
have trace-bs-union:
  trace-bs  $\in$  trace-fri-root-union-bad-sets s bad-sets fr
  unfolding trace-fri-root-union-bad-sets-def
  using trace-bs-space candidate trace-bs-bad by blast
show ?thesis
  unfolding trace-fri-root-list-set-hit-def
  using challenges header trace-bs-union by blast
qed

lemma trace-fri-challenge-set-hit-imp-supported-root-union-list-set-hit:
  assumes outcome: out  $\in$  set-dist (execute verify-monad s)
  and hit: trace-fri-challenge-set-hit s bad-sets out
  shows
    trace-fri-root-list-set-hit s
    (trace-fri-supported-root-union-bad-sets s bad-sets) out
proof –
from hit obtain trace-table composition-table as query-idxs trace-bs dg comp-bs
  where bound:
    accepted-with-bound-tables s out trace-table composition-table as
    query-idxs
  and challenges: accepted-fri-challenges s out trace-bs dg comp-bs
  and not-low:  $\neg$  trace-table-low-degree trace-table
  and trace-bs-bad: trace-bs  $\in$  bad-sets trace-table
  unfolding trace-fri-challenge-set-hit-def by blast
have header-ex:
   $\exists$  fr f-fri-roots f-final header-as composition-fri-roots final rest.
  verifier-header-transcript s fr f-fri-roots f-final header-as dg
  composition-fri-roots final rest
proof –
from challenges[unfolded accepted-fri-challenges-def] show ?thesis
proof (elim exE conjE)
  fix result0 final-state0 fr0 f-fl0 f-final0 as0 fl0 final0
  query-state0
  assume

```

```

    header0:
      verifier-header-transcript s fr0 (map snd f-fl0) f-final0 as0 dg
      (map snd fl0) final0 (PTranscript query-state0)
  show ?thesis
    apply (rule exI[where x=fr0])
    apply (rule exI[where x=map snd f-fl0])
    apply (rule exI[where x=f-final0])
    apply (rule exI[where x=as0])
    apply (rule exI[where x=map snd fl0])
    apply (rule exI[where x=final0])
    apply (rule exI[where x=PTranscript query-state0])
    apply (rule header0)
  done
qed
qed
then obtain fr f-fri-roots f-final header-as composition-fri-roots final
  rest where
  header:
    verifier-header-transcript s fr f-fri-roots f-final header-as dg
    composition-fri-roots final rest
  by blast
have candidate:
  trace-table ∈ trace-fri-supported-root-trace-table-candidates s fr
  unfolding trace-fri-supported-root-trace-table-candidates-def
  using outcome bound challenges header by blast
have trace-bs-space:
  trace-bs ∈ fri-challenge-space (ceil-log clength)
  by (rule accepted-fri-challenges-trace-space[OF challenges])
have trace-bs-union:
  trace-bs ∈ trace-fri-supported-root-union-bad-sets s bad-sets fr
  unfolding trace-fri-supported-root-union-bad-sets-def
  using trace-bs-space candidate trace-bs-bad by blast
show ?thesis
  unfolding trace-fri-root-list-set-hit-def
  using challenges header trace-bs-union by blast
qed

lemma trace-fri-challenge-set-hit-imp-partial-union-or-collision:
  assumes outcome: out ∈ set-dist (execute verify-monad s)
  and hit: trace-fri-challenge-set-hit s bad-sets out
  shows
    trace-fri-root-list-set-hit s
      (trace-fri-supported-root-partial-union-bad-sets s bad-sets) out ∨
      hash-map-output-collision-bad s out
proof -
  from hit obtain trace-table composition-table as query-idxs trace-bs dg comp-bs
  where bound:
    accepted-with-bound-tables s out trace-table composition-table as
    query-idxs

```

```

    and challenges: accepted-fri-challenges s out trace-bs dg comp-bs
    and not-low:  $\neg$  trace-table-low-degree trace-table
    and trace-bs-bad: trace-bs  $\in$  bad-sets trace-table
    unfolding trace-fri-challenge-set-hit-def by blast
from accepted-with-bound-tables-imp-accepted[OF bound]
obtain result final-state where out-eq: out = Some (result, final-state)
    unfolding accepted-def by (cases out) auto
show ?thesis
proof (cases hash-map-output-collision final-state)
  case True
  then have hash-map-output-collision-bad s out
    unfolding hash-map-output-collision-bad-def accepted-def out-eq by simp
  then show ?thesis by simp
next
case clean: False
have header-ex:
   $\exists$  fr f-fri-roots f-final header-as composition-fri-roots final rest.
    verifier-header-transcript s fr f-fri-roots f-final header-as dg
    composition-fri-roots final rest
proof -
from challenges[unfolded accepted-fri-challenges-def] show ?thesis
proof (elim exE conjE)
  fix result0 final-state0 fr0 f-fl0 f-final0 as0 fl0 final0
  query-state0
  assume
    header0:
      verifier-header-transcript s fr0 (map snd f-fl0) f-final0 as0 dg
      (map snd fl0) final0 (PTranscript query-state0)
  show ?thesis
  apply (rule exI[where x=fr0])
  apply (rule exI[where x=map snd f-fl0])
  apply (rule exI[where x=f-final0])
  apply (rule exI[where x=as0])
  apply (rule exI[where x=map snd fl0])
  apply (rule exI[where x=final0])
  apply (rule exI[where x=PTranscript query-state0])
  apply (rule header0)
  done
qed
qed
then obtain fr f-fri-roots f-final header-as composition-fri-roots final
  rest where
  header:
    verifier-header-transcript s fr f-fri-roots f-final header-as dg
    composition-fri-roots final rest
  by blast
from bound obtain fr' f-fri-roots' f-final' dg'
  composition-fri-roots' final' rest' where
  header-bound:

```

```

    verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
    composition-fri-roots' final' rest'
  and bind:
    merkle-root-binds-table fr' trace-table final-state
  unfolding accepted-with-bound-tables-def out-eq by blast
have fr-eq: fr' = fr
  using verifier-header-transcript-unique[OF header-bound header] by simp
have outcome-some:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
  using outcome out-eq by simp
have bound-some:
  accepted-with-bound-tables s (Some (result, final-state))
  trace-table composition-table as query-idxs
  using bound out-eq by simp
have challenges-some:
  accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
  comp-bs
  using challenges out-eq by simp
have bind-fr: merkle-root-binds-table fr trace-table final-state
  using bind fr-eq by simp
have state:
  trace-fri-supported-root-trace-table-candidate-state s fr trace-table
  final-state
  unfolding trace-fri-supported-root-trace-table-candidate-state-def
  using outcome-some bound-some challenges-some header bind-fr by blast
have candidate:
  trace-table ∈
    trace-fri-supported-root-partial-trace-table-candidates s fr
  by (rule
    trace-fri-supported-root-trace-table-candidate-state-imp-partial-candidate
    [OF state clean])
have trace-bs-space:
  trace-bs ∈ fri-challenge-space (ceil-log clength)
  by (rule accepted-fri-challenges-trace-space[OF challenges])
have trace-bs-union:
  trace-bs ∈
    trace-fri-supported-root-partial-union-bad-sets s bad-sets fr
  unfolding trace-fri-supported-root-partial-union-bad-sets-def
  using trace-bs-space candidate trace-bs-bad by blast
show ?thesis
  unfolding trace-fri-root-list-set-hit-def
  using challenges header trace-bs-union by blast
qed
qed

lemma alpha-bad-set-hit-imp-partial-union-or-collision:
  assumes outcome: out ∈ set-dist (execute verify-monad s)
  and hit: alpha-bad-set-hit s bad-sets out
  and subset:  $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$ 

```

shows
alpha-header-list-set-hit s
(alpha-header-supported-partial-union-bad-sets s bad-sets) out $\vee$
hash-map-output-collision-bad s out
proof –
from *hit* **obtain** *trace-table composition-table as query-idxs* **where**
bound:
accepted-with-bound-tables s out trace-table composition-table
as query-idxs
and *as-bad: as $\in$ bad-sets trace-table*
unfolding *alpha-bad-set-hit-def* **by** *blast*
from *accepted-with-bound-tables-imp-accepted[OF bound]*
obtain *result final-state* **where** *out-eq: out = Some (result, final-state)*
unfolding *accepted-def* **by** *(cases out) auto*
show *?thesis*
proof *(cases hash-map-output-collision final-state)*
case *True*
then have *hash-map-output-collision-bad s out*
unfolding *hash-map-output-collision-bad-def accepted-def out-eq* **by** *simp*
then show *?thesis* **by** *simp*
next
case *clean: False*
from *bound* **obtain** *fr f-fri-roots f-final dg composition-fri-roots final*
rest **where**
header:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *bind:*
merkle-root-binds-table fr trace-table final-state
unfolding *accepted-with-bound-tables-def out-eq* **by** *blast*
have *shape:*
accepted-transcript-shape s out as query-idxs
using *accepted-with-bound-tables-imp-accepted-with-tables[OF bound]*
accepted-with-tables-imp-accepted-transcript-shape
by *blast*
have *state:*
alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table final-state
unfolding *alpha-header-supported-trace-table-candidate-state-def*
by *(intro exI[of - result] exI[of - composition-table]*
exI[of - as] exI[of - query-idxs] exI[of - dg]
exI[of - composition-fri-roots] exI[of - final]
exI[of - rest])
(use outcome out-eq bound header bind in simp)
have *candidate:*
trace-table $\in$
alpha-header-supported-partial-trace-table-candidates s fr
f-fri-roots f-final
by *(rule*

```

    alpha-header-supported-trace-table-candidate-state-imp-partial-candidate
    [OF state clean])
have as-union:
  as ∈
    alpha-header-supported-partial-union-bad-sets s bad-sets fr
    f-fri-roots f-final
  unfolding alpha-header-supported-partial-union-bad-sets-def
  using as-bad candidate subset[of trace-table] by auto
show ?thesis
  unfolding alpha-header-list-set-hit-def
  using shape out-eq header as-union by blast
qed
qed

lemma wp-trace-fri-challenge-set-hit-bound-via-partial-union-or-collision:
fixes C H :: prob
assumes partial-bound:
  wp-event verify-monad
    (trace-fri-root-list-set-hit s
      (trace-fri-supported-root-partial-union-bad-sets s bad-sets)) s ≤ C
and collision-bound:
  wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H
shows
  wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s ≤
    C + H
proof –
have mono:
  wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s ≤
    wp-event verify-monad
      (λout.
        trace-fri-root-list-set-hit s
          (trace-fri-supported-root-partial-union-bad-sets s bad-sets) out ∨
          hash-map-output-collision-bad s out) s
by (rule wp-event-mono-on-support)
    (use trace-fri-challenge-set-hit-imp-partial-union-or-collision in blast)
also have ... ≤ C + H
proof –
have wp-event verify-monad
  (λout.
    trace-fri-root-list-set-hit s
      (trace-fri-supported-root-partial-union-bad-sets s bad-sets) out ∨
      hash-map-output-collision-bad s out) s ≤
    wp-event verify-monad
      (trace-fri-root-list-set-hit s
        (trace-fri-supported-root-partial-union-bad-sets s bad-sets)) s +
    wp-event verify-monad (hash-map-output-collision-bad s) s
by (rule wp-event-union-bound)
also have ... ≤ C + H
by (intro add-mono partial-bound collision-bound)

```

```

    finally show ?thesis .
qed
finally show ?thesis .
qed

lemma accepted-verifier-initial-imp-transcript-target-hit-on-support:
  assumes support:
    out ∈ set-dist (execute verify-monad (verifier-initial-state tr))
  and acc: accepted out
  shows hash-new-output-hit-event (set tr) (verifier-initial-state tr) out
proof -
  from acc obtain result final-state where out-eq:
    out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
  have outcome:
    Some (result, final-state) ∈
      set-dist (execute verify-monad (verifier-initial-state tr))
  using support out-eq by simp
  have hit:
    hash-map-new-output-hit (set tr) (verifier-initial-state tr)
      final-state
  by (rule verify-monad-verifier-initial-trace-root-target-hit[OF outcome])
  show ?thesis
  unfolding hash-new-output-hit-event-def out-eq
  using hit by simp
qed

lemma wp-alpha-bad-set-hit-bound-via-partial-union-or-collision:
  fixes C H :: prob
  assumes subset:  $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$ 
  and partial-bound:
    wp-event verify-monad
      (alpha-header-list-set-hit s
        (alpha-header-supported-partial-union-bad-sets s bad-sets)) s ≤ C
  and collision-bound:
    wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H
  shows
    wp-event verify-monad (alpha-bad-set-hit s bad-sets) s ≤ C + H
proof -
  have mono:
    wp-event verify-monad (alpha-bad-set-hit s bad-sets) s ≤
      wp-event verify-monad
        (λout.
          alpha-header-list-set-hit s
            (alpha-header-supported-partial-union-bad-sets s bad-sets) out ∨
            hash-map-output-collision-bad s out) s
  proof (rule wp-event-mono-on-support)
    fix out
    assume support: out ∈ set-dist (execute verify-monad s)

```

```

    and hit: alpha-bad-set-hit s bad-sets out
  show
    alpha-header-list-set-hit s
    (alpha-header-supported-partial-union-bad-sets s bad-sets) out ∨
    hash-map-output-collision-bad s out
  by (rule alpha-bad-set-hit-imp-partial-union-or-collision
      [OF support hit subset])
qed
also have ... ≤ C + H
proof -
  have wp-event verify-monad
    (λout.
      alpha-header-list-set-hit s
      (alpha-header-supported-partial-union-bad-sets s bad-sets) out ∨
      hash-map-output-collision-bad s out) s ≤
    wp-event verify-monad
      (alpha-header-list-set-hit s
        (alpha-header-supported-partial-union-bad-sets s bad-sets)) s +
    wp-event verify-monad (hash-map-output-collision-bad s) s
  by (rule wp-event-union-bound)
  also have ... ≤ C + H
  by (intro add-mono partial-bound collision-bound)
  finally show ?thesis .
qed
finally show ?thesis .
qed

lemma wp-trace-fri-challenge-set-hit-bound-via-supported-partial-union-or-collision:
  fixes C H :: prob
  assumes future: trace-fri-future-fresh s
  and union-bound:
    ∧fr. nnreal
      (card (trace-fri-supported-root-partial-union-bad-sets s bad-sets fr)) /
      nnreal (CARD('f) ^ ceil-log clength) ≤ C
  and collision-bound:
    wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H
  shows
    wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s ≤
      C + H
proof -
  have partial-bound:
    wp-event verify-monad
      (trace-fri-root-list-set-hit s
        (trace-fri-supported-root-partial-union-bad-sets s bad-sets)) s ≤ C
  by (rule wp-verify-monad-trace-fri-root-list-set-bound[OF future])
    (use trace-fri-supported-root-partial-union-bad-sets-subset union-bound
      in simp-all)
  show ?thesis
  by (rule wp-trace-fri-challenge-set-hit-bound-via-partial-union-or-collision

```


[OF partial-bound collision-bound])

qed

lemma *wp-alpha-bad-set-hit-bound-via-supported-partial-union-or-collision:*
fixes $C\ H :: \text{prob}$
assumes *future: alpha-future-fresh s*
and *subset: $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$*
and *union-bound:*
 $\bigwedge \text{fr f-fri-roots f-final.}$
 nnreal
 $(\text{card}$
 $(\text{alpha-header-supported-partial-union-bad-sets } s \text{ bad-sets fr}$
 $\text{f-fri-roots f-final})) /$
 $\text{nnreal } (\text{card } \text{alpha-space}) \leq C$
and *collision-bound:*
 $\text{wp-event verify-monad } (\text{hash-map-output-collision-bad } s) \ s \leq H$
shows $\text{wp-event verify-monad } (\text{alpha-bad-set-hit } s \text{ bad-sets}) \ s \leq C + H$
proof –
have *partial-bound:*
 $\text{wp-event verify-monad}$
 $(\text{alpha-header-list-set-hit } s$
 $(\text{alpha-header-supported-partial-union-bad-sets } s \text{ bad-sets})) \ s \leq C$
by (rule *wp-verify-monad-alpha-header-list-set-bound*[OF *future*])
 $(\text{use } \text{alpha-header-supported-partial-union-bad-sets-subset-alpha-space}$
 $\text{union-bound in simp-all})$
show *?thesis*
by (rule *wp-alpha-bad-set-hit-bound-via-partial-union-or-collision*
 $[\text{OF subset partial-bound collision-bound}])$
qed

lemma *trace-fri-supported-root-partial-union-bad-sets-fraction-bound-if-unique-candidate:*
fixes $C :: \text{prob}$
assumes *unique:*
 $\exists \text{trace-table.}$
 $\text{trace-fri-supported-root-partial-trace-table-candidates } s \text{ fr} \subseteq$
 $\{\text{trace-table}\}$
and *subset:*
 $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq$
 $\text{fri-challenge-space } (\text{ceil-log clength})$
and *bound:*
 $\bigwedge \text{trace-table.}$
 $\text{nnreal } (\text{card } (\text{bad-sets trace-table})) /$
 $\text{nnreal } (\text{CARD('f)} \wedge \text{ceil-log clength}) \leq C$
shows
 nnreal
 $(\text{card } (\text{trace-fri-supported-root-partial-union-bad-sets } s \text{ bad-sets fr})) /$
 $\text{nnreal } (\text{CARD('f)} \wedge \text{ceil-log clength}) \leq C$
proof –
from *unique* **obtain** *trace-table* **where** *candidates:*

```

    trace-fri-supported-root-partial-trace-table-candidates s fr  $\subseteq$ 
      {trace-table}
  by blast
have union-subset:
  trace-fri-supported-root-partial-union-bad-sets s bad-sets fr  $\subseteq$ 
    bad-sets trace-table
  unfolding trace-fri-supported-root-partial-union-bad-sets-def
  using candidates by auto
have finite-bad: finite (bad-sets trace-table)
  by (rule finite-subset[OF subset finite-fri-challenge-space])
have card-le:
  card (trace-fri-supported-root-partial-union-bad-sets s bad-sets fr)  $\leq$ 
    card (bad-sets trace-table)
  by (rule card-mono[OF finite-bad union-subset])
have nnreal
  (card
    (trace-fri-supported-root-partial-union-bad-sets s bad-sets fr)) /
  nnreal (CARD('f) ^ ceil-log clength)  $\leq$ 
  nnreal (card (bad-sets trace-table)) /
  nnreal (CARD('f) ^ ceil-log clength)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
also have ...  $\leq C$ 
  by (rule bound)
finally show ?thesis .
qed

end

end

```

```

theory Soundness-Reductions-Composition
imports Soundness-Reductions-Trace
begin

```

Composition-table, query-header, and supported candidate reductions.

```

context soundness
begin

```

```

definition trace-fri-bad-challenge-sets-bounded
  :: ('f list  $\Rightarrow$  'f list set)  $\Rightarrow$  bool
where
  trace-fri-bad-challenge-sets-bounded bad-sets  $\longleftrightarrow$ 
    ( $\forall$  trace-table.
      bad-sets trace-table  $\subseteq$  fri-challenge-space (ceil-log clength)  $\wedge$ 
      nnreal (card (bad-sets trace-table)) /
      nnreal (CARD('f) ^ ceil-log clength)  $\leq$  trace-fri-error)

```

```

definition composition-fri-bad-challenge-sets-bounded

```

```

:: ('f ⇒ 'f list ⇒ 'f list set) ⇒ bool
where
  composition-fri-bad-challenge-sets-bounded bad-sets ⟷
    (∀ dg composition-table.
      bad-sets dg composition-table ⊆
        fri-challenge-space (ceil-log (to-nat dg + 1)) ∧
        nnreal (card (bad-sets dg composition-table)) /
        nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤
        composition-fri-error)

lemma wp-trace-fri-challenge-set-hit-bound-if-supported-partial-root-candidate-unique-or-collision:
fixes H :: prob
assumes future: trace-fri-future-fresh s
and unique:
  ∧fr.
    ∃ trace-table.
      trace-fri-supported-root-partial-trace-table-candidates s fr ⊆
        {trace-table}
and bounded: trace-fri-bad-challenge-sets-bounded bad-sets
and collision-bound:
  wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H
shows
  wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s ≤
    trace-fri-error + H
proof -
have union-bound:
  ∧fr.
    nnreal
      (card
        (trace-fri-supported-root-partial-union-bad-sets s bad-sets fr)) /
    nnreal (CARD('f) ^ ceil-log clength) ≤ trace-fri-error
proof -
fix fr
show
  nnreal
    (card
      (trace-fri-supported-root-partial-union-bad-sets s bad-sets fr)) /
  nnreal (CARD('f) ^ ceil-log clength) ≤ trace-fri-error
by (rule
  trace-fri-supported-root-partial-union-bad-sets-fraction-bound-if-unique-candidate
  [OF unique])
  (use bounded in
    ‹auto simp: trace-fri-bad-challenge-sets-bounded-def›)
qed
show ?thesis
by (rule
  wp-trace-fri-challenge-set-hit-bound-via-supported-partial-union-or-collision
  [OF future union-bound collision-bound])
qed

```

lemma *alpha-header-supported-partial-union-bad-sets-fraction-bound-if-unique-candidate:*

fixes $C :: \text{prob}$

assumes *unique:*

$\exists \text{ trace-table.}$

alpha-header-supported-partial-trace-table-candidates s fr

f-fri-roots f-final $\subseteq \{\text{trace-table}\}$

and *subset:* $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$

and *bound:*

$\bigwedge \text{trace-table.}$

$\text{nnreal} (\text{card} (\text{bad-sets trace-table})) / \text{nnreal} (\text{card alpha-space}) \leq C$

shows

nnreal

$(\text{card}$

(alpha-header-supported-partial-union-bad-sets s bad-sets fr

f-fri-roots f-final)) /

nnreal (card alpha-space) ≤ C

proof –

from *unique* **obtain** *trace-table* **where** *candidates:*

alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots

f-final $\subseteq \{\text{trace-table}\}$

by *blast*

have *union-subset:*

alpha-header-supported-partial-union-bad-sets s bad-sets fr f-fri-roots

f-final $\subseteq \text{bad-sets trace-table}$

unfolding *alpha-header-supported-partial-union-bad-sets-def*

using *candidates* **by** *auto*

have *finite-bad:* *finite (bad-sets trace-table)*

by *(rule finite-subset[OF subset finite-alpha-space])*

have *card-le:*

card

(alpha-header-supported-partial-union-bad-sets s bad-sets fr

f-fri-roots f-final) ≤

card (bad-sets trace-table)

by *(rule card-mono[OF finite-bad union-subset])*

have *nnreal*

$(\text{card}$

(alpha-header-supported-partial-union-bad-sets s bad-sets fr

f-fri-roots f-final)) /

nnreal (card alpha-space) ≤

nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)

by *(rule nnreal-nat-divide-right-mono[OF card-le])*

also have $\dots \leq C$

by *(rule bound)*

finally show *?thesis* .

qed

lemma *wp-alpha-bad-set-hit-bound-if-supported-partial-header-candidate-unique-or-collision:*

fixes $C H :: \text{prob}$

assumes *future*: $\alpha\text{-future-fresh } s$
and *unique*:
 $\bigwedge fr \text{ } f\text{-fri-roots } f\text{-final}.$
 $\exists \text{ trace-table}.$
 $\alpha\text{-header-supported-partial-trace-table-candidates } s \text{ } fr$
 $f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\}$
and *subset*: $\bigwedge \text{trace-table}.$ $\text{bad-sets } \text{trace-table} \subseteq \alpha\text{-space}$
and *bounded*:
 $\bigwedge \text{trace-table}.$
 $\text{nnreal } (\text{card } (\text{bad-sets } \text{trace-table})) / \text{nnreal } (\text{card } \alpha\text{-space}) \leq$
 C
and *collision-bound*:
 $\text{wp-event verify-monad } (\text{hash-map-output-collision-bad } s) \text{ } s \leq H$
shows $\text{wp-event verify-monad } (\alpha\text{-bad-set-hit } s \text{ } \text{bad-sets}) \text{ } s \leq C + H$
proof –
have *union-bound*:
 $\bigwedge fr \text{ } f\text{-fri-roots } f\text{-final}.$
 nnreal
 $(\text{card}$
 $(\alpha\text{-header-supported-partial-union-bad-sets } s \text{ } \text{bad-sets } fr$
 $f\text{-fri-roots } f\text{-final})) /$
 $\text{nnreal } (\text{card } \alpha\text{-space}) \leq C$
proof –
fix $fr \text{ } f\text{-fri-roots } f\text{-final}$
show
 nnreal
 $(\text{card}$
 $(\alpha\text{-header-supported-partial-union-bad-sets } s \text{ } \text{bad-sets } fr$
 $f\text{-fri-roots } f\text{-final})) /$
 $\text{nnreal } (\text{card } \alpha\text{-space}) \leq C$
by (rule
 $\alpha\text{-header-supported-partial-union-bad-sets-fraction-bound-if-unique-candidate}$
 $[OF \text{ } \text{unique } \text{subset } \text{bounded}]$)
qed
show *?thesis*
by (rule $\text{wp-alpha-bad-set-hit-bound-via-supported-partial-union-or-collision}$
 $[OF \text{ } \text{future } \text{subset } \text{union-bound } \text{collision-bound}]$)
qed

lemma *composition-randomization-bad-bound-if-alpha-header-supported-partial-candidate-unique-or-collision*:
fixes $H :: \text{prob}$
assumes *future*: $\alpha\text{-future-fresh } s$
and *unique*:
 $\bigwedge fr \text{ } f\text{-fri-roots } f\text{-final}.$
 $\exists \text{ trace-table}.$
 $\alpha\text{-header-supported-partial-trace-table-candidates } s \text{ } fr$
 $f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\}$
and *collision-bound*:
 $\text{wp-event verify-monad } (\text{hash-map-output-collision-bad } s) \text{ } s \leq H$

shows

$$\text{wp-event verify-monad } (\text{composition-randomization-bad } s) \ s \leq \text{composition-error-bound} + H$$
proof –
have *alpha-bound*:

$$\text{wp-event verify-monad } (\text{alpha-bad-set-hit } s \ \text{composition-trace-bad-alpha-space}) \ s \leq \text{composition-error-bound} + H$$
by (rule

$$\text{wp-alpha-bad-set-hit-bound-if-supported-partial-header-candidate-unique-or-collision}$$

$$[\text{OF future unique composition-trace-bad-alpha-space-subset-alpha-space}$$

$$\text{composition-trace-bad-alpha-space-fraction-bound-alpha-space}$$

$$\text{collision-bound}])$$
have $\text{wp-event verify-monad } (\text{composition-randomization-bad } s) \ s \leq$

$$\text{wp-event verify-monad } (\text{alpha-bad-set-hit } s \ \text{composition-trace-bad-alpha-space}) \ s$$
by (rule *wp-event-mono*)

$$(\text{rule composition-randomization-bad-imp-alpha-bad-set-hit})$$
also have $\dots \leq \text{composition-error-bound} + H$
by (rule *alpha-bound*)
finally show *?thesis* .
qed

lemma *wp-trace-fri-challenge-set-hit-bound-via-root-union*:
fixes $C :: \text{prob}$
assumes *future: trace-fri-future-fresh s*
and *union-bound*:

$$\bigwedge \text{fr. nnreal } (\text{card } (\text{trace-fri-root-union-bad-sets } s \ \text{bad-sets } \text{fr})) / \text{nnreal } (\text{CARD('f)} \wedge \text{ceil-log clength}) \leq C$$
shows $\text{wp-event verify-monad } (\text{trace-fri-challenge-set-hit } s \ \text{bad-sets}) \ s \leq C$
proof –
have *root-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-root-list-set-hit } s \ (\text{trace-fri-root-union-bad-sets } s \ \text{bad-sets})) \ s \leq C$$
by (rule *wp-verify-monad-trace-fri-root-list-set-bound*[*OF future*])

$$(\text{use trace-fri-root-union-bad-sets-subset union-bound in simp-all})$$
have $\text{wp-event verify-monad } (\text{trace-fri-challenge-set-hit } s \ \text{bad-sets}) \ s \leq$

$$\text{wp-event verify-monad } (\text{trace-fri-root-list-set-hit } s \ (\text{trace-fri-root-union-bad-sets } s \ \text{bad-sets})) \ s$$
by (rule *wp-event-mono*)

$$(\text{rule trace-fri-challenge-set-hit-imp-root-union-list-set-hit})$$
also have $\dots \leq C$
by (rule *root-bound*)
finally show *?thesis* .
qed

lemma *wp-trace-fri-challenge-set-hit-bound-via-supported-root-union:*
fixes $C :: \text{prob}$
assumes *future: trace-fri-future-fresh s*
and *union-bound:*
 $\bigwedge \text{fr. nnreal}$
 $(\text{card } (\text{trace-fri-supported-root-union-bad-sets } s \text{ bad-sets fr})) /$
 $\text{nnreal } (\text{CARD('f)} \wedge \text{ceil-log clength}) \leq C$
shows *wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s $\leq C$*
proof –
have *root-bound:*
wp-event verify-monad
(trace-fri-root-list-set-hit s
(trace-fri-supported-root-union-bad-sets s bad-sets)) s $\leq C$
by (*rule wp-verify-monad-trace-fri-root-list-set-bound[OF future]*)
(use trace-fri-supported-root-union-bad-sets-subset union-bound in
simp-all)
have *wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s $\leq$*
wp-event verify-monad
(trace-fri-root-list-set-hit s
(trace-fri-supported-root-union-bad-sets s bad-sets)) s
by (*rule wp-event-mono-on-support*)
(rule trace-fri-challenge-set-hit-imp-supported-root-union-list-set-hit)
also have $\dots \leq C$
by (*rule root-bound*)
finally show *?thesis .*
qed

lemma *trace-fri-root-union-bad-sets-fraction-bound-if-unique-candidate:*
fixes $C :: \text{prob}$
assumes *unique:*
 $\exists \text{ trace-table.}$
 $\text{trace-fri-root-trace-table-candidates } s \text{ fr} \subseteq \{\text{trace-table}\}$
and *subset:*
 $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq$
 $\text{fri-challenge-space (ceil-log clength)}$
and *bound:*
 $\bigwedge \text{trace-table.}$
 $\text{nnreal } (\text{card } (\text{bad-sets trace-table})) /$
 $\text{nnreal } (\text{CARD('f)} \wedge \text{ceil-log clength}) \leq C$
shows
 $\text{nnreal } (\text{card } (\text{trace-fri-root-union-bad-sets } s \text{ bad-sets fr})) /$
 $\text{nnreal } (\text{CARD('f)} \wedge \text{ceil-log clength}) \leq C$
proof –
from *unique* **obtain** *trace-table* **where** *candidates:*
 $\text{trace-fri-root-trace-table-candidates } s \text{ fr} \subseteq \{\text{trace-table}\}$
by *blast*
have *union-subset:*
 $\text{trace-fri-root-union-bad-sets } s \text{ bad-sets fr} \subseteq \text{bad-sets trace-table}$
unfolding *trace-fri-root-union-bad-sets-def*

```

    using candidates by auto
  have finite-bad: finite (bad-sets trace-table)
    by (rule finite-subset[OF subset finite-fri-challenge-space])
  have card-le:
    card (trace-fri-root-union-bad-sets s bad-sets fr) ≤
      card (bad-sets trace-table)
    by (rule card-mono[OF finite-bad union-subset])
  have nnreal (card (trace-fri-root-union-bad-sets s bad-sets fr)) /
    nnreal (CARD('f) ^ ceil-log clength) ≤
    nnreal (card (bad-sets trace-table)) /
    nnreal (CARD('f) ^ ceil-log clength)
    by (rule nnreal-nat-divide-right-mono[OF card-le])
  also have ... ≤ C
    by (rule bound)
  finally show ?thesis .
qed

lemma trace-fri-supported-root-union-bad-sets-fraction-bound-if-unique-candidate:
  fixes C :: prob
  assumes unique:
    ∃ trace-table.
      trace-fri-supported-root-trace-table-candidates s fr ⊆
        {trace-table}
  and subset:
    ∧ trace-table. bad-sets trace-table ⊆
      fri-challenge-space (ceil-log clength)
  and bound:
    ∧ trace-table.
      nnreal (card (bad-sets trace-table)) /
      nnreal (CARD('f) ^ ceil-log clength) ≤ C
  shows
    nnreal (card (trace-fri-supported-root-union-bad-sets s bad-sets fr)) /
    nnreal (CARD('f) ^ ceil-log clength) ≤ C
proof -
  from unique obtain trace-table where candidates:
    trace-fri-supported-root-trace-table-candidates s fr ⊆
      {trace-table}
  by blast
  have union-subset:
    trace-fri-supported-root-union-bad-sets s bad-sets fr ⊆
      bad-sets trace-table
  unfolding trace-fri-supported-root-union-bad-sets-def
  using candidates by auto
  have finite-bad: finite (bad-sets trace-table)
    by (rule finite-subset[OF subset finite-fri-challenge-space])
  have card-le:
    card (trace-fri-supported-root-union-bad-sets s bad-sets fr) ≤
      card (bad-sets trace-table)
    by (rule card-mono[OF finite-bad union-subset])

```


have *nnreal*
 $(\text{card } (\text{trace-fri-supported-root-union-bad-sets } s \text{ bad-sets } fr)) /$
 $\text{nnreal } (CARD('f) \wedge \text{ceil-log } \text{clength}) \leq$
 $\text{nnreal } (\text{card } (\text{bad-sets } \text{trace-table})) /$
 $\text{nnreal } (CARD('f) \wedge \text{ceil-log } \text{clength})$
by (*rule nnreal-nat-divide-right-mono*[*OF card-le*])
also have $\dots \leq C$
by (*rule bound*)
finally show *?thesis* .
qed

lemma *composition-fri-challenge-set-hit-imp-list-set-hit*:
assumes *hit*: *composition-fri-challenge-set-hit* *s* *bad-sets* *out*
and *envelope*:
 $\bigwedge dg \text{ composition-table. bad-sets } dg \text{ composition-table} \subseteq B \text{ dg}$
shows *composition-fri-challenge-list-set-hit* *s* *B* *out*
proof –
from *hit* **obtain** *trace-table* *composition-table* *as query-idxs* *trace-bs* *dg comp-bs*
where *challenges*:
 $\text{accepted-fri-challenges } s \text{ out trace-bs } dg \text{ comp-bs}$
and *comp-bad*: $\text{comp-bs} \in \text{bad-sets } dg \text{ composition-table}$
unfolding *composition-fri-challenge-set-hit-def* **by** *blast*
have $\text{comp-bs} \in B \text{ dg}$
using *comp-bad envelope*[*of dg composition-table*] **by** *auto*
then show *?thesis*
unfolding *composition-fri-challenge-list-set-hit-def*
using *challenges* **by** *blast*
qed

definition *composition-fri-degree-table-candidates*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list set}$
where
 $\text{composition-fri-degree-table-candidates } s \text{ dg} =$
 $\{ \text{composition-table.}$
 $\exists \text{ out trace-table as query-idxs trace-bs comp-bs.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table}$
 $\text{as query-idxs} \wedge$
 $\text{accepted-fri-challenges } s \text{ out trace-bs } dg \text{ comp-bs} \}$

definition *composition-fri-degree-union-bad-sets*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow 'f \Rightarrow 'f \text{ list set}$
where
 $\text{composition-fri-degree-union-bad-sets } s \text{ bad-sets } dg =$
 $\{ \text{comp-bs} \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1)).$
 $\exists \text{ composition-table} \in \text{composition-fri-degree-table-candidates } s \text{ dg.}$
 $\text{comp-bs} \in \text{bad-sets } dg \text{ composition-table} \}$

lemma *composition-fri-degree-union-bad-sets-subset*:

$\text{composition-fri-degree-union-bad-sets } s \text{ bad-sets } dg \subseteq$
 $\text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$
unfolding $\text{composition-fri-degree-union-bad-sets-def}$ **by** *auto*

lemma $\text{composition-fri-challenge-set-hit-imp-degree-union-list-set-hit}$:
assumes $\text{hit: composition-fri-challenge-set-hit } s \text{ bad-sets out}$
shows
 $\text{composition-fri-challenge-list-set-hit } s$
 $(\text{composition-fri-degree-union-bad-sets } s \text{ bad-sets}) \text{ out}$

proof –
from hit **obtain** $\text{trace-table composition-table as query-idxs trace-bs } dg \text{ comp-bs}$
where bound :
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as}$
 query-idxs
and $\text{challenges: accepted-fri-challenges } s \text{ out trace-bs } dg \text{ comp-bs}$
and $\text{comp-bs-bad: comp-bs} \in \text{bad-sets } dg \text{ composition-table}$
unfolding $\text{composition-fri-challenge-set-hit-def}$ **by** *blast*
have candidate :
 $\text{composition-table} \in \text{composition-fri-degree-table-candidates } s \text{ dg}$
unfolding $\text{composition-fri-degree-table-candidates-def}$
using bound challenges **by** *blast*
have comp-bs-space :
 $\text{comp-bs} \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$
by $(\text{rule accepted-fri-challenges-composition-space}[OF \text{ challenges}])$
have comp-bs-union :
 $\text{comp-bs} \in \text{composition-fri-degree-union-bad-sets } s \text{ bad-sets } dg$
unfolding $\text{composition-fri-degree-union-bad-sets-def}$
using $\text{comp-bs-space candidate comp-bs-bad}$ **by** *blast*
show $?thesis$
unfolding $\text{composition-fri-challenge-list-set-hit-def}$
using $\text{challenges comp-bs-union}$ **by** *blast*

qed

lemma $\text{wp-composition-fri-challenge-set-hit-bound-via-degree-union}$:
fixes $C :: \text{prob}$
assumes $\text{future: composition-fri-future-fresh } s$
and union-bound :
 $\bigwedge dg. \text{nnreal}$
 $(\text{card } (\text{composition-fri-degree-union-bad-sets } s \text{ bad-sets } dg)) /$
 $\text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq C$
shows
 $\text{wp-event verify-monad } (\text{composition-fri-challenge-set-hit } s \text{ bad-sets}) \text{ } s \leq$
 C

proof –
have list-bound :
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-challenge-list-set-hit } s$
 $(\text{composition-fri-degree-union-bad-sets } s \text{ bad-sets})) \text{ } s \leq C$
by $(\text{rule wp-verify-monad-composition-fri-challenge-list-set-bound}[OF \text{ future}])$

```

    (use composition-fri-degree-union-bad-sets-subset union-bound in
      simp-all)
  have wp-event verify-monad (composition-fri-challenge-set-hit s bad-sets) s ≤
    wp-event verify-monad
      (composition-fri-challenge-list-set-hit s
        (composition-fri-degree-union-bad-sets s bad-sets)) s
  by (rule wp-event-mono)
    (rule composition-fri-challenge-set-hit-imp-degree-union-list-set-hit)
  also have ... ≤ C
  by (rule list-bound)
  finally show ?thesis .
qed

lemma composition-fri-degree-union-bad-sets-fraction-bound-if-unique-candidate:
  fixes C :: prob
  assumes unique:
    ∃ composition-table.
      composition-fri-degree-table-candidates s dg ⊆ {composition-table}
  and subset:
    ∧ composition-table. bad-sets dg composition-table ⊆
      fri-challenge-space (ceil-log (to-nat dg + 1))
  and bound:
    ∧ composition-table.
      nnreal (card (bad-sets dg composition-table)) /
        nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C
  shows
    nnreal (card (composition-fri-degree-union-bad-sets s bad-sets dg)) /
      nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C
proof -
  from unique obtain composition-table where candidates:
    composition-fri-degree-table-candidates s dg ⊆ {composition-table}
  by blast
  have union-subset:
    composition-fri-degree-union-bad-sets s bad-sets dg ⊆
      bad-sets dg composition-table
  unfolding composition-fri-degree-union-bad-sets-def
  using candidates by auto
  have finite-bad: finite (bad-sets dg composition-table)
  by (rule finite-subset[OF subset finite-fri-challenge-space])
  have card-le:
    card (composition-fri-degree-union-bad-sets s bad-sets dg) ≤
      card (bad-sets dg composition-table)
  by (rule card-mono[OF finite-bad union-subset])
  have nnreal (card (composition-fri-degree-union-bad-sets s bad-sets dg)) /
    nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤
    nnreal (card (bad-sets dg composition-table)) /
      nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
  by (rule nnreal-nat-divide-right-mono[OF card-le])
  also have ... ≤ C

```

by (rule bound)
 finally show ?thesis .
 qed

definition *composition-fri-supported-root-composition-table-candidates*

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$
 'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$
 'f list $\Rightarrow$ 'f list set

where

composition-fri-supported-root-composition-table-candidates s fr
 f-fri-roots f-final as dg *composition-fri-roots* =
 { *composition-table*.
 $\exists$ out trace-table query-idxs trace-bs comp-bs final rest.
 out $\in$ set-dist (execute verify-monad s) $\wedge$
 accepted-with-bound-tables s out trace-table *composition-table* as
 query-idxs $\wedge$
 accepted-fri-challenges s out trace-bs dg comp-bs $\wedge$
 verifier-header-transcript s fr f-fri-roots f-final as dg
 composition-fri-roots final rest }

definition *composition-fri-supported-root-composition-table-candidate-state*

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$
 'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$
 'f list $\Rightarrow$ 'f list $\Rightarrow$
 ('f, 'a) protocol-channel-scheme $\Rightarrow$ bool

where

composition-fri-supported-root-composition-table-candidate-state s fr
 f-fri-roots f-final as dg *composition-fri-roots* *composition-table*
final-state $\longleftrightarrow$
 ($\exists$ result trace-table query-idxs trace-bs comp-bs final rest.
 Some (result, final-state) $\in$ set-dist (execute verify-monad s) $\wedge$
 accepted-with-bound-tables s (Some (result, final-state)) trace-table
 composition-table as query-idxs $\wedge$
 accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
 comp-bs $\wedge$
 verifier-header-transcript s fr f-fri-roots f-final as dg
 composition-fri-roots final rest $\wedge$
 composition-fri-roots $\neq$ [] $\wedge$
 merkle-root-binds-table (hd *composition-fri-roots*) *composition-table*
 final-state)

lemma *composition-fri-supported-root-composition-table-candidates-iff-state:*

composition-table $\in$
 composition-fri-supported-root-composition-table-candidates s fr
 f-fri-roots f-final as dg *composition-fri-roots* $\longleftrightarrow$
 ($\exists$ final-state.
 composition-fri-supported-root-composition-table-candidate-state s fr
 f-fri-roots f-final as dg *composition-fri-roots* *composition-table*
 final-state)

proof

assume *candidate*:

composition-table $\in$
composition-fri-supported-root-composition-table-candidates *s* *fr*
f-fri-roots *f-final* *as* *dg* *composition-fri-roots*

then obtain *out* *trace-table* *query-idxs* *trace-bs* *comp-bs* *final* *rest* **where**
outcome: *out* $\in$ *set-dist* (*execute* *verify-monad* *s*)

and *bound*:

accepted-with-bound-tables *s* *out* *trace-table* *composition-table* *as*
query-idxs

and *challenges*: *accepted-fri-challenges* *s* *out* *trace-bs* *dg* *comp-bs*

and *header*:

verifier-header-transcript *s* *fr* *f-fri-roots* *f-final* *as* *dg*
composition-fri-roots *final* *rest*

unfolding *composition-fri-supported-root-composition-table-candidates-def*
by *blast*

from *bound* **obtain** *result* *final-state* *fr'* *f-fri-roots'* *f-final'* *dg'*
composition-fri-roots' *final'* *rest'* **where**

out-eq: *out* = *Some* (*result*, *final-state*)

and *header'*:

verifier-header-transcript *s* *fr'* *f-fri-roots'* *f-final'* *as* *dg'*
composition-fri-roots' *final'* *rest'*

and *comp-nonempty*: *composition-fri-roots'* $\neq$ []

and *comp-bind*:

merkle-root-binds-table (*hd* *composition-fri-roots'*)
composition-table *final-state*

unfolding *accepted-with-bound-tables-def* **by** *blast*

have *eqs*:

fr' = *fr* $\wedge$
f-fri-roots' = *f-fri-roots* $\wedge$
f-final' = *f-final* $\wedge$
dg' = *dg* $\wedge$
composition-fri-roots' = *composition-fri-roots* $\wedge$
final' = *final* $\wedge$
rest' = *rest*

using *verifier-header-transcript-unique*[*OF* *header'* *header*] **by** *simp*

have *state*:

composition-fri-supported-root-composition-table-candidate-state *s* *fr*
f-fri-roots *f-final* *as* *dg* *composition-fri-roots* *composition-table*
final-state

unfolding *composition-fri-supported-root-composition-table-candidate-state-def*
by (*intro* *exI*[*of* - *result*] *exI*[*of* - *trace-table*]
exI[*of* - *query-idxs*] *exI*[*of* - *trace-bs*] *exI*[*of* - *comp-bs*]
exI[*of* - *final*] *exI*[*of* - *rest*])
(use outcome out-eq bound challenges header comp-nonempty comp-bind eqs
in simp)

then show
 $\exists$ *final-state*.
composition-fri-supported-root-composition-table-candidate-state *s* *fr*

*f-fri-roots f-final as dg composition-fri-roots composition-table
 final-state*
by *blast*
next
assume
 $\exists$ *final-state.*
*composition-fri-supported-root-composition-table-candidate-state s fr
 f-fri-roots f-final as dg composition-fri-roots composition-table
 final-state*
then obtain *final-state result trace-table query-idxs trace-bs comp-bs final
 rest where*
outcome:
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *bound:*
*accepted-with-bound-tables s (Some (result, final-state))
 trace-table composition-table as query-idxs*
and *challenges:*
*accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
 comp-bs*
and *header:*
*verifier-header-transcript s fr f-fri-roots f-final as dg
 composition-fri-roots final rest*
unfolding *composition-fri-supported-root-composition-table-candidate-state-def*
by *blast*
show
*composition-table $\in$
 composition-fri-supported-root-composition-table-candidates s fr
 f-fri-roots f-final as dg composition-fri-roots*
unfolding *composition-fri-supported-root-composition-table-candidates-def*
using *outcome bound challenges header by blast*
qed

definition *composition-fri-supported-root-partial-composition-table-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list set}$

where

*composition-fri-supported-root-partial-composition-table-candidates s fr
 f-fri-roots f-final as dg composition-fri-roots =*
 $\{ \text{composition-table.}$
 $\text{composition-fri-roots} \neq [] \wedge$
 $(\exists \text{ out query-idxs composition-openings trace-bs comp-bs final rest.}$
 $\text{out} \in \text{set-dist (execute verify-monad s)} \wedge$
 $\text{accepted-with-partial-composition-openings s out}$
 $(\text{hd composition-fri-roots}) \text{ query-idxs composition-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\text{accepted-fri-challenges s out trace-bs dg comp-bs} \wedge$
 $\text{verifier-header-transcript s fr f-fri-roots f-final as dg}$

composition-fri-roots final rest)}

definition *composition-fri-supported-root-partial-composition-table-candidate-state*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

composition-fri-supported-root-partial-composition-table-candidate-state
 $s \text{ fr } f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots } \text{composition-table}$
 $\text{final-state} \longleftrightarrow$
 $\text{composition-fri-roots} \neq [] \wedge$
 $(\exists \text{result query-idxs } \text{composition-openings } \text{trace-bs } \text{comp-bs } \text{final rest.}$
 $\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute } \text{verify-monad } s) \wedge$
 $\text{accepted-with-partial-composition-openings } s$
 $(\text{Some } (\text{result}, \text{final-state})) (\text{hd } \text{composition-fri-roots}) \text{ query-idxs}$
 $\text{composition-openings} \wedge$
 $\text{partial-composition-table-candidate } \text{composition-table}$
 $\text{composition-openings} \wedge$
 $\text{accepted-fri-challenges } s (\text{Some } (\text{result}, \text{final-state})) \text{ trace-bs dg}$
 $\text{comp-bs} \wedge$
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final as dg}$
 $\text{composition-fri-roots } \text{final rest})$

lemma *composition-fri-supported-root-partial-composition-table-candidates-iff-state:*

$\text{composition-table} \in$
 $\text{composition-fri-supported-root-partial-composition-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots} \longleftrightarrow$
 $(\exists \text{final-state.}$
 $\text{composition-fri-supported-root-partial-composition-table-candidate-state}$
 $s \text{ fr } f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots}$
 $\text{composition-table } \text{final-state})$

proof

assume *candidate:*

$\text{composition-table} \in$
 $\text{composition-fri-supported-root-partial-composition-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots}$

then obtain *out query-idxs composition-openings trace-bs comp-bs final rest*

where *comp-nonempty:* $\text{composition-fri-roots} \neq []$

and *outcome:* $\text{out} \in \text{set-dist } (\text{execute } \text{verify-monad } s)$

and *partial:*

$\text{accepted-with-partial-composition-openings } s \text{ out}$
 $(\text{hd } \text{composition-fri-roots}) \text{ query-idxs } \text{composition-openings}$

and *table:*

$\text{partial-composition-table-candidate } \text{composition-table}$
 $\text{composition-openings}$

and *challenges:* $\text{accepted-fri-challenges } s \text{ out trace-bs dg comp-bs}$

and *header:*

$\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final as dg}$

```

    composition-fri-roots final rest
unfolding
    composition-fri-supported-root-partial-composition-table-candidates-def
by blast
from partial obtain result final-state where out-eq:
    out = Some (result, final-state)
    unfolding accepted-with-partial-composition-openings-def by blast
have outcome-some:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
    using outcome out-eq by simp
have partial-some:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots) query-idxs
    composition-openings
    using partial out-eq by simp
have challenges-some:
    accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
    comp-bs
    using challenges out-eq by simp
have state:
    composition-fri-supported-root-partial-composition-table-candidate-state
    s fr f-fri-roots f-final as dg composition-fri-roots
    composition-table final-state
unfolding
    composition-fri-supported-root-partial-composition-table-candidate-state-def
using comp-nonempty outcome-some partial-some table challenges-some header
by blast
then show
     $\exists$  final-state.
    composition-fri-supported-root-partial-composition-table-candidate-state
    s fr f-fri-roots f-final as dg composition-fri-roots
    composition-table final-state
by blast
next
assume
     $\exists$  final-state.
    composition-fri-supported-root-partial-composition-table-candidate-state
    s fr f-fri-roots f-final as dg composition-fri-roots
    composition-table final-state
then obtain final-state result query-idxs composition-openings trace-bs
    comp-bs final rest where
    comp-nonempty: composition-fri-roots  $\neq$  []
and outcome:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
and partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots) query-idxs
    composition-openings
and table:

```


*partial-composition-table-candidate composition-table
composition-openings*
and *challenges:*
*accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
comp-bs*
and *header:*
*verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest*
unfolding
composition-fri-supported-root-partial-composition-table-candidate-state-def
by *blast*
show
*composition-table ∈
composition-fri-supported-root-partial-composition-table-candidates s fr
f-fri-roots f-final as dg composition-fri-roots*
unfolding
composition-fri-supported-root-partial-composition-table-candidates-def
using *comp-nonempty outcome partial table challenges header* **by** *blast*
qed

lemma *composition-fri-supported-root-composition-table-candidate-state-imp-partial-candidate-state:*
assumes *state:*
*composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table
final-state*
and *clean:* $\neg$ *hash-map-output-collision final-state*
shows
*composition-fri-supported-root-partial-composition-table-candidate-state
s fr f-fri-roots f-final as dg composition-fri-roots composition-table
final-state*
proof —
from *state* **obtain** *result trace-table query-idxs trace-bs comp-bs final rest*
where *outcome:*
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *bound:*
*accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs*
and *challenges:*
*accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
comp-bs*
and *header:*
*verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest*
and *comp-nonempty:* *composition-fri-roots* $\neq$ []
and *bind:*
*merkle-root-binds-table (hd composition-fri-roots) composition-table
final-state*
unfolding *composition-fri-supported-root-composition-table-candidate-state-def*
by *blast*

```

from verify-monad-header-extraction[OF outcome]
obtain fr' f-fl' f-final' as' dg' fl' final' query-state' where
  header':
    verifier-header-transcript s fr' (map snd f-fl') f-final' as' dg'
      (map snd fl') final' (PTranscript query-state')
and query-out:
  Some (result, final-state)  $\in$ 
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr' f-fl' f-final' as' fl'
            final') rounds)
          query-state')
by metis
have eqs:
  fr' = fr  $\wedge$ 
    map snd f-fl' = f-fri-roots  $\wedge$ 
    f-final' = f-final  $\wedge$ 
    as' = as  $\wedge$ 
    dg' = dg  $\wedge$ 
    map snd fl' = composition-fri-roots  $\wedge$ 
    final' = final  $\wedge$ 
    PTranscript query-state' = rest
using verifier-header-transcript-unique[OF header' header] by simp
then have fl-nonempty: fl'  $\neq []$ 
using comp-nonempty by (cases fl') simp-all
then obtain b composition-root fl-tail where fl-eq:
  fl' = (b, composition-root) # fl-tail
by (cases fl') auto
have root-eq: composition-root = hd composition-fri-roots
proof –
  have map-eq: map snd fl' = composition-fri-roots
    using eqs by simp
  have cons-eq: composition-root # map snd fl-tail = composition-fri-roots
    using map-eq fl-eq by simp
  then have hd composition-fri-roots = composition-root
    by (metis list.sel(1))
  then show ?thesis by simp
qed
from query-rounds-accepted-with-partial-composition-openings
  [OF fl-eq query-out]
obtain partial-query-idxs composition-openings where partial:
  accepted-with-partial-composition-openings s
    (Some (result, final-state)) composition-root partial-query-idxs
    composition-openings
by blast
have partial-root:
  accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots)

```

```

    partial-query-idxs composition-openings
  using partial-root-eq by simp
have len-table: length composition-table = scale * clength
  using accepted-with-bound-tables-shapes(2)[OF bound]
  by (simp add: mult.commute)
have table:
  partial-composition-table-candidate composition-table
  composition-openings
  by (rule accepted-bound-composition-table-partial-candidate-if-clean
    [OF bind len-table partial-root clean])
show ?thesis
unfolding
  composition-fri-supported-root-partial-composition-table-candidate-state-def
  using outcome partial-root table challenges header comp-nonempty by blast
qed

```

definition *query-header-supported-partial-table-candidates*

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$ 
  'f list  $\Rightarrow$  'f  $\Rightarrow$  ('f list  $\times$  'f list) set

```

where

```

query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final =
  {(trace-table, composition-table).
    composition-fri-roots  $\neq$  []  $\wedge$ 
    ( $\exists$  out trace-query-idxs composition-query-idxs trace-openings
      composition-openings rest.
        out  $\in$  set-dist (execute verify-monad s)  $\wedge$ 
        accepted-with-partial-trace-openings s out fr trace-query-idxs
          trace-openings  $\wedge$ 
        partial-trace-table-candidate trace-table trace-openings  $\wedge$ 
        accepted-with-partial-composition-openings s out
          (hd composition-fri-roots) composition-query-idxs
            composition-openings  $\wedge$ 
        partial-composition-table-candidate composition-table
          composition-openings  $\wedge$ 
        verifier-header-transcript s fr f-fri-roots f-final as dg
          composition-fri-roots final rest)}}

```

definition *query-header-supported-partial-table-candidate-state*

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$ 
  'f list  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f list  $\Rightarrow$ 
  ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool

```

where

```

query-header-supported-partial-table-candidate-state s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state  $\longleftrightarrow$ 
composition-fri-roots  $\neq$  []  $\wedge$ 

```

$(\exists \text{result trace-query-idxs composition-query-idxs trace-openings}$
 $\text{composition-openings rest.}$
 $\text{Some (result, final-state)} \in \text{set-dist (execute verify-monad s)} \wedge$
 $\text{accepted-with-partial-trace-openings s}$
 $(\text{Some (result, final-state)}) \text{ fr trace-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{accepted-with-partial-composition-openings s}$
 $(\text{Some (result, final-state)}) (\text{hd composition-fri-roots})$
 $\text{composition-query-idxs composition-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\text{verifier-header-transcript s fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final rest})$

lemma *query-header-supported-partial-table-candidates-iff-state:*

$(\text{trace-table, composition-table}) \in$
 $\text{query-header-supported-partial-table-candidates s fr f-fri-roots}$
 $\text{f-final as dg composition-fri-roots final} \longleftrightarrow$
 $(\exists \text{final-state.}$
 $\text{query-header-supported-partial-table-candidate-state s fr f-fri-roots}$
 $\text{f-final as dg composition-fri-roots final trace-table}$
 $\text{composition-table final-state})$

proof

assume *candidate:*

$(\text{trace-table, composition-table}) \in$
 $\text{query-header-supported-partial-table-candidates s fr f-fri-roots}$
 $\text{f-final as dg composition-fri-roots final}$

then obtain *out trace-query-idxs composition-query-idxs trace-openings*
composition-openings rest where

comp-nonempty: $\text{composition-fri-roots} \neq []$

and *outcome:* $\text{out} \in \text{set-dist (execute verify-monad s)}$

and *trace-partial:*

$\text{accepted-with-partial-trace-openings s out fr trace-query-idxs}$
 trace-openings

and *trace-table:*

$\text{partial-trace-table-candidate trace-table trace-openings}$

and *comp-partial:*

$\text{accepted-with-partial-composition-openings s out}$
 $(\text{hd composition-fri-roots}) \text{ composition-query-idxs}$
 $\text{composition-openings}$

and *comp-table:*

$\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings}$

and *header:*

$\text{verifier-header-transcript s fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final rest}$

unfolding *query-header-supported-partial-table-candidates-def* **by** *blast*
from *trace-partial* **obtain** *result final-state where out-eq:*

```

    out = Some (result, final-state)
  unfolding accepted-with-partial-trace-openings-def by blast
have outcome-some:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
  using outcome out-eq by simp
have trace-partial-some:
  accepted-with-partial-trace-openings s (Some (result, final-state)) fr
    trace-query-idxs trace-openings
  using trace-partial out-eq by simp
have comp-partial-some:
  accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots)
    composition-query-idxs composition-openings
  using comp-partial out-eq by simp
have state:
  query-header-supported-partial-table-candidate-state s fr f-fri-roots
    f-final as dg composition-fri-roots final trace-table
    composition-table final-state
  unfolding query-header-supported-partial-table-candidate-state-def
  using comp-nonempty outcome-some trace-partial-some trace-table
    comp-partial-some comp-table header
  by blast
then show
  ∃ final-state.
    query-header-supported-partial-table-candidate-state s fr f-fri-roots
      f-final as dg composition-fri-roots final trace-table
      composition-table final-state
  by blast
next
assume
  ∃ final-state.
    query-header-supported-partial-table-candidate-state s fr f-fri-roots
      f-final as dg composition-fri-roots final trace-table
      composition-table final-state
then obtain final-state result trace-query-idxs composition-query-idxs
  trace-openings composition-openings rest where
  comp-nonempty: composition-fri-roots ≠ []
and outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and trace-partial:
  accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr trace-query-idxs trace-openings
and trace-table:
  partial-trace-table-candidate trace-table trace-openings
and comp-partial:
  accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots)
    composition-query-idxs composition-openings
and comp-table:

```

partial-composition-table-candidate composition-table
composition-openings
and header:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
unfolding *query-header-supported-partial-table-candidate-state-def*
by *blast*
show
(trace-table, composition-table) ∈
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final
unfolding *query-header-supported-partial-table-candidates-def*
using *comp-nonempty outcome trace-partial trace-table comp-partial*
comp-table header
by *blast*
qed

lemma *query-header-supported-partial-table-candidate-state-hash-extends:*
assumes
query-header-supported-partial-table-candidate-state s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state
shows $s \leq \text{final-state}$
proof –
from *assms* **obtain** *result trace-query-idxs composition-query-idxs*
trace-openings composition-openings rest **where**
outcome:
Some (result, final-state) ∈ set-dist (execute verify-monad s)
unfolding *query-header-supported-partial-table-candidate-state-def*
by *blast*
show *?thesis*
by *(rule verify-monad-hash-extends[OF outcome])*
qed

lemma *query-header-supported-table-candidate-state-imp-partial-candidate-state:*
assumes *state:*
query-header-supported-table-candidate-state s fr f-fri-roots f-final
as dg composition-fri-roots final trace-table composition-table
final-state
and *clean: ¬ hash-map-output-collision final-state*
shows
query-header-supported-partial-table-candidate-state s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state
proof –
from *state* **obtain** *result query-idxs rest* **where**
outcome:
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *bound:*

```

    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and trace-bind:
    merkle-root-binds-table fr trace-table final-state
and comp-nonempty: composition-fri-roots  $\neq$  []
and comp-bind:
    merkle-root-binds-table (hd composition-fri-roots) composition-table
    final-state
unfolding query-header-supported-table-candidate-state-def by blast
from verify-monad-header-extraction[OF outcome]
obtain fr' f-fl f-final' as' dg' fl final' query-state where
  header':
    verifier-header-transcript s fr' (map snd f-fl) f-final' as' dg'
    (map snd fl) final' (PTranscript query-state)
and query-out:
    Some (result, final-state)  $\in$ 
      set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr' f-fl f-final' as' fl
            final') rounds)
        query-state)
by metis
have eqs:
  fr' = fr  $\wedge$ 
  map snd f-fl = f-fri-roots  $\wedge$ 
  f-final' = f-final  $\wedge$ 
  as' = as  $\wedge$ 
  dg' = dg  $\wedge$ 
  map snd fl = composition-fri-roots  $\wedge$ 
  final' = final  $\wedge$ 
  PTranscript query-state = rest
using verifier-header-transcript-unique[OF header' header] by simp
obtain trace-query-idxs trace-openings where
  len-trace-query-idxs: length trace-query-idxs = rounds
and len-trace-openings: length trace-openings = rounds
and trace-indices:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (trace-query-idxs ! i)
and trace-tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr'
      (scale * clength) (trace-openings ! i) final-state
proof (rule ntimes-verifier-query-rounds-authenticated-trace-openings-no-raws
  [OF query-out])

```

```

fix trace-query-idxs trace-openings
assume len-trace-query-idxs: length trace-query-idxs = rounds
and len-trace-openings: length trace-openings = rounds
and trace-indices:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (trace-query-idxs ! i)
and trace-tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr'
      (scale * clength) (trace-openings ! i) final-state
show ?thesis
  by (rule that[OF len-trace-query-idxs len-trace-openings
    trace-indices trace-tables])
qed
have trace-partial:
  accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr trace-query-idxs trace-openings
proof -
  have all-indices:
     $\forall i < \text{rounds}.$ 
      map opening-index (trace-openings ! i) =
        powers-scaled (trace-query-idxs ! i)
    using trace-indices by blast
  have all-tables:
     $\forall i < \text{rounds}.$ 
      partial-authenticated-table fr (scale * clength)
        (trace-openings ! i) final-state
    using trace-tables eqs by simp
  show ?thesis
    unfolding accepted-with-partial-trace-openings-def accepted-def
    by (intro conjI exI[of - result] exI[of - final-state])
      (use len-trace-query-idxs len-trace-openings all-indices all-tables
        in simp-all)
qed
have trace-len: length trace-table = scale * clength
  using accepted-with-bound-tables-shapes(1)[OF bound] by simp
have trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
    by (rule accepted-bound-trace-table-partial-candidate-if-clean
      [OF trace-bind trace-len trace-partial clean])
have fl-nonempty: fl  $\neq$  []
  using comp-nonempty eqs by (cases fl) simp-all
then obtain b composition-root fl-tail where fl-eq:
  fl = (b, composition-root) # fl-tail
  by (cases fl) auto
have root-eq: composition-root = hd composition-fri-roots
proof -
  have cons-eq: composition-root # map snd fl-tail = composition-fri-roots

```



```

    using eqs fl-eq by simp
  then have hd composition-fri-roots = composition-root
    by (metis list.sel(1))
  then show ?thesis by simp
qed
from query-rounds-accepted-with-partial-composition-openings
  [OF fl-eq query-out]
obtain composition-query-idxs composition-openings where comp-partial':
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) composition-root composition-query-idxs
  composition-openings
  by blast
have comp-partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) (hd composition-fri-roots)
  composition-query-idxs composition-openings
  using comp-partial' root-eq by simp
have comp-len: length composition-table = scale * clength
  using accepted-with-bound-tables-shapes(2)[OF bound]
  by (simp add: mult.commute)
have comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
  by (rule accepted-bound-composition-table-partial-candidate-if-clean
    [OF comp-bind comp-len comp-partial clean])
show ?thesis
  unfolding query-header-supported-partial-table-candidate-state-def
  using comp-nonempty outcome trace-partial trace-candidate comp-partial
  comp-candidate header
  by blast
qed

lemma accepted-with-bound-tables-header-composition-fri-roots-nonempty:
  assumes bound:
    accepted-with-bound-tables s out trace-table composition-table as
    query-idxs
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  shows composition-fri-roots  $\neq$  []
proof -
  from bound obtain result final-state fr' f-fri-roots' f-final' dg'
    composition-fri-roots' final' rest' where
    out-eq: out = Some (result, final-state)
  and header':
    verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
    composition-fri-roots' final' rest'
  and comp-nonempty': composition-fri-roots'  $\neq$  []
  unfolding accepted-with-bound-tables-def by blast

```

have *composition-fri-roots'* = *composition-fri-roots*
using *verifier-header-transcript-unique*[*OF header' header*] **by** *simp*
then show *?thesis*
using *comp-nonempty'* **by** *simp*
qed

definition *query-header-supported-partial-union-good-sets*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat set}) \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat set}$

where

query-header-supported-partial-union-good-sets s good-sets fr
f-fri-roots f-final as dg composition-fri-roots final =
 $\{idx \in \text{query-sample-space}.$
 $\exists \text{ trace-table composition-table}.$
 $(\text{trace-table}, \text{composition-table}) \in$
 $\text{query-header-supported-partial-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final} \wedge$
 $idx \in \text{good-sets trace-table composition-table as}\}$

lemma *query-header-supported-partial-union-good-sets-subset:*

query-header-supported-partial-union-good-sets s good-sets fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$ query-sample-space

unfolding *query-header-supported-partial-union-good-sets-def* **by** *auto*

lemma *query-header-supported-partial-union-good-sets-fraction-bound-if-unique-candidate:*

assumes *unique:*

$\exists \text{ trace-table composition-table}.$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$
 $\{(\text{trace-table}, \text{composition-table})\}$

and *subset:*

$\bigwedge \text{ trace-table composition-table as}.$
 $\text{good-sets trace-table composition-table as} \subseteq \text{query-sample-space}$

and *bound:*

$\bigwedge \text{ trace-table composition-table as}.$
 $\text{nnreal } (\text{card } (\text{good-sets trace-table composition-table as})) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq \text{query-error-bound}$

shows

nnreal
 $(\text{card}$
 $(\text{query-header-supported-partial-union-good-sets } s \text{ good-sets fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final})) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq \text{query-error-bound}$

proof –

from *unique* **obtain** *trace-table composition-table* **where** *candidates:*
query-header-supported-partial-table-candidates s fr f-fri-roots f-final
as dg composition-fri-roots final $\subseteq$

```

    {(trace-table, composition-table)}
  by blast
have union-subset:
  query-header-supported-partial-union-good-sets s good-sets fr
    f-fri-roots f-final as dg composition-fri-roots final  $\subseteq$ 
    good-sets trace-table composition-table as
  unfolding query-header-supported-partial-union-good-sets-def
  using candidates by auto
have finite-good: finite (good-sets trace-table composition-table as)
  by (rule finite-subset[OF subset finite-query-sample-space])
have card-le:
  card
    (query-header-supported-partial-union-good-sets s good-sets fr
      f-fri-roots f-final as dg composition-fri-roots final)  $\leq$ 
    card (good-sets trace-table composition-table as)
  by (rule card-mono[OF finite-good union-subset])
have nnreal
  (card
    (query-header-supported-partial-union-good-sets s good-sets fr
      f-fri-roots f-final as dg composition-fri-roots final)) /
  nnreal (card query-sample-space)  $\leq$ 
  nnreal (card (good-sets trace-table composition-table as)) /
  nnreal (card query-sample-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
also have ...  $\leq$  query-error-bound
  by (rule bound)
finally show ?thesis .
qed

lemma query-index-round-set-hit-at-imp-partial-union-or-collision:
  assumes outcome: out  $\in$  set-dist (execute verify-monad s)
  and hit: query-index-round-set-hit-at s good-sets i out
  shows
    query-header-rounds-any-index-set-hit s
      (query-header-supported-partial-union-good-sets s good-sets) out  $\vee$ 
      hash-map-output-collision-bad s out
proof -
  from hit obtain trace-table composition-table as query-idxs where
    bound:
      accepted-with-bound-tables s out trace-table composition-table as
      query-idxs
  and i-bound: i < rounds
  and query-hit:
    query-idxs ! i  $\in$  good-sets trace-table composition-table as
  unfolding query-index-round-set-hit-at-def by blast
  from bound obtain result final-state fr f-fri-roots f-final dg
    composition-fri-roots final rest where
    out-eq: out = Some (result, final-state)
  and header:

```

```

    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and trace-bind:
    merkle-root-binds-table fr trace-table final-state
and comp-nonempty: composition-fri-roots  $\neq$  []
and comp-bind:
    merkle-root-binds-table (hd composition-fri-roots) composition-table
    final-state
unfolding accepted-with-bound-tables-def by blast
show ?thesis
proof (cases hash-map-output-collision final-state)
  case True
  then have hash-map-output-collision-bad s out
    unfolding hash-map-output-collision-bad-def accepted-def out-eq by simp
  then show ?thesis by simp
next
case clean: False
have state:
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table composition-table
    final-state
    unfolding query-header-supported-table-candidate-state-def
  by (intro exI[of - result] exI[of - query-idxs] exI[of - rest])
    (use outcome out-eq bound header trace-bind comp-nonempty comp-bind
    in simp)
have candidate:
    (trace-table, composition-table)  $\in$ 
    query-header-supported-partial-table-candidates s fr f-fri-roots
    f-final as dg composition-fri-roots final
    using
    query-header-supported-table-candidate-state-imp-partial-candidate-state
    [OF state clean]
    query-header-supported-partial-table-candidates-iff-state
  by blast
have len: length query-idxs = rounds
  by (rule accepted-with-bound-tables-shapes(4)[OF bound])
have idx-mem: query-idxs ! i  $\in$  set query-idxs
  by (rule nth-mem) (use i-bound len in simp)
have idx-sample: query-idxs ! i  $\in$  query-sample-space
  by (rule accepted-with-bound-tables-query-sample-space
    [OF bound idx-mem])
have union-hit:
    query-idxs ! i  $\in$ 
    query-header-supported-partial-union-good-sets s good-sets fr
    f-fri-roots f-final as dg composition-fri-roots final
    unfolding query-header-supported-partial-union-good-sets-def
    using idx-sample candidate query-hit by blast
have round-hit:
    query-rounds-any-index-set-hit s

```

```

    (query-header-supported-partial-union-good-sets s good-sets fr
      f-fri-roots f-final as dg composition-fri-roots final)
    rounds out
  by (rule accepted-with-bound-tables-query-rounds-any-index-set-hit
    [OF bound i-bound union-hit])
show ?thesis
  unfolding query-header-rounds-any-index-set-hit-def
  using header round-hit by blast
qed
qed

lemma query-index-round-set-hit-imp-partial-union-or-collision:
  assumes outcome: out ∈ set-dist (execute verify-monad s)
  and hit: query-index-round-set-hit s good-sets out
  shows
    query-header-rounds-any-index-set-hit s
      (query-header-supported-partial-union-good-sets s good-sets) out ∨
      hash-map-output-collision-bad s out
proof -
  from hit obtain i where
    query-index-round-set-hit-at s good-sets i out
  unfolding query-index-round-set-hit-def by blast
  then show ?thesis
  by (rule query-index-round-set-hit-at-imp-partial-union-or-collision
    [OF outcome])
qed

lemma wp-query-index-round-set-hit-bound-via-partial-union-or-collision:
  fixes C H :: prob
  assumes partial-bound:
    wp-event verify-monad
      (query-header-rounds-any-index-set-hit s
        (query-header-supported-partial-union-good-sets s good-sets)) s ≤
      C
  and collision-bound:
    wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H
  shows
    wp-event verify-monad (query-index-round-set-hit s good-sets) s ≤
      C + H
proof -
  have mono:
    wp-event verify-monad (query-index-round-set-hit s good-sets) s ≤
      wp-event verify-monad
        (λout.
          query-header-rounds-any-index-set-hit s
            (query-header-supported-partial-union-good-sets s good-sets)
            out ∨
            hash-map-output-collision-bad s out) s
  by (rule wp-event-mono-on-support)

```

(use query-index-round-set-hit-imp-partial-union-or-collision in blast)
 also have ... $\leq C + H$
 proof –
 have wp-event verify-monad
 (λout.
 query-header-rounds-any-index-set-hit s
 (query-header-supported-partial-union-good-sets s good-sets)
 out ∨
 hash-map-output-collision-bad s out) s ≤
 wp-event verify-monad
 (query-header-rounds-any-index-set-hit s
 (query-header-supported-partial-union-good-sets s good-sets)) s +
 wp-event verify-monad (hash-map-output-collision-bad s) s
 by (rule wp-event-union-bound)
 also have ... $\leq C + H$
 by (intro add-mono partial-bound collision-bound)
 finally show ?thesis .
 qed
 finally show ?thesis .
 qed

lemma wp-query-index-round-set-hit-bound-via-supported-partial-union-or-collision:
 fixes H :: prob
 assumes future: query-future-fresh s
 and raw-bound: query-index-raw-preimage-bound
 and union-bound:
 ∧fr f-fri-roots f-final as dg composition-fri-roots final.
 nnreal
 (card
 (query-header-supported-partial-union-good-sets s good-sets fr
 f-fri-roots f-final as dg composition-fri-roots final)) /
 nnreal (card query-sample-space) ≤ query-error-bound
 and collision-bound:
 wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H
 shows
 wp-event verify-monad (query-index-round-set-hit s good-sets) s ≤
 nnreal rounds * query-error-bound + H
 proof –
 have partial-bound:
 wp-event verify-monad
 (query-header-rounds-any-index-set-hit s
 (query-header-supported-partial-union-good-sets s good-sets)) s ≤
 nnreal rounds * query-error-bound
 by (rule wp-verify-monad-query-header-rounds-any-index-set-bound
 [OF future raw-bound])
 (use query-header-supported-partial-union-good-sets-subset union-bound
 in simp-all)
 show ?thesis
 by (rule wp-query-index-round-set-hit-bound-via-partial-union-or-collision)

[OF partial-bound collision-bound])

qed

lemma *wp-query-index-set-hit-bound-via-partial-union-or-collision:*
fixes $C\ H :: \text{prob}$
assumes *partial-bound:*
 wp-event verify-monad
 (*query-header-rounds-any-index-set-hit s*
 (*query-header-supported-partial-union-good-sets s good-sets*)) $s \leq$
 C
and *collision-bound:*
 wp-event verify-monad (hash-map-output-collision-bad s) s \leq H
shows
 wp-event verify-monad (query-index-set-hit s good-sets) s \leq C + H
proof –
have *wp-event verify-monad (query-index-set-hit s good-sets) s \leq*
 wp-event verify-monad (query-index-round-set-hit s good-sets) s
by (rule *wp-event-mono*) (rule *query-index-set-hit-imp-round-set-hit*)
also have $\dots \leq C + H$
by (rule *wp-query-index-round-set-hit-bound-via-partial-union-or-collision*
 [OF *partial-bound collision-bound*])
finally show *?thesis* .
qed

lemma *wp-query-index-round-set-hit-bound-if-supported-partial-header-candidate-unique-or-collision:*
fixes $H :: \text{prob}$
assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *unique:*
 $\bigwedge \text{fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots final.}$
 $\exists \text{ trace-table composition-table.}$
 query-header-supported-partial-table-candidates s fr f-fri-roots
 f-final as dg composition-fri-roots final \subseteq
 $\{(\text{trace-table}, \text{composition-table})\}$
and *subset:*
 $\bigwedge \text{ trace-table composition-table as.}$
 good-sets trace-table composition-table as \subseteq query-sample-space
and *bounded:*
 $\bigwedge \text{ trace-table composition-table as.}$
 $\text{nnreal } (\text{card } (\text{good-sets trace-table composition-table as})) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq \text{query-error-bound}$
and *collision-bound:*
 wp-event verify-monad (hash-map-output-collision-bad s) s \leq H
shows
 wp-event verify-monad (query-index-round-set-hit s good-sets) s \leq
 $\text{nnreal rounds} * \text{query-error-bound} + H$
proof –
have *union-bound:*
 $\bigwedge \text{fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots final.}$

```

nnreal
  (card
    (query-header-supported-partial-union-good-sets s good-sets fr
      f-fri-roots f-final as dg composition-fri-roots final)) /
  nnreal (card query-sample-space) ≤ query-error-bound
by (rule
  query-header-supported-partial-union-good-sets-fraction-bound-if-unique-candidate
  [OF unique subset bounded])
show ?thesis
by (rule
  wp-query-index-round-set-hit-bound-via-supported-partial-union-or-collision
  [OF future raw-bound union-bound collision-bound])
qed

lemma wp-query-index-set-hit-bound-if-supported-partial-header-candidate-unique-or-collision:
  fixes H :: prob
  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and unique:
     $\bigwedge$  fr f-fri-roots f-final as dg composition-fri-roots final.
     $\exists$  trace-table composition-table.
    query-header-supported-partial-table-candidates s fr f-fri-roots
      f-final as dg composition-fri-roots final  $\subseteq$ 
      {(trace-table, composition-table)}
  and subset:
     $\bigwedge$  trace-table composition-table as.
    good-sets trace-table composition-table as  $\subseteq$  query-sample-space
  and bounded:
     $\bigwedge$  trace-table composition-table as.
    nnreal (card (good-sets trace-table composition-table as)) /
    nnreal (card query-sample-space) ≤ query-error-bound
  and collision-bound:
    wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H
  shows
    wp-event verify-monad (query-index-set-hit s good-sets) s ≤
      nnreal rounds * query-error-bound + H
proof -
  have wp-event verify-monad (query-index-set-hit s good-sets) s ≤
    wp-event verify-monad (query-index-round-set-hit s good-sets) s
  by (rule wp-event-mono) (rule query-index-set-hit-imp-round-set-hit)
  also have ... ≤ nnreal rounds * query-error-bound + H
  by (rule
    wp-query-index-round-set-hit-bound-if-supported-partial-header-candidate-unique-or-collision
    [OF future raw-bound unique subset bounded collision-bound])
  finally show ?thesis .
qed

lemma wp-query-sampling-success-set-hit-bound-if-supported-partial-header-candidate-unique-or-collision:
  fixes H :: prob

```


assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *unique:*
 $\bigwedge_{fr} f\text{-fri-roots } f\text{-final as dg composition-fri-roots final.}$
 $\exists \text{ trace-table composition-table.}$
 $\text{query-header-supported-partial-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final as dg composition-fri-roots final} \subseteq$
 $\{(trace\text{-table}, composition\text{-table})\}$
and *collision-bound:*
 $wp\text{-event verify-monad (hash-map-output-collision-bad } s) s \leq H$
shows
 $wp\text{-event verify-monad}$
 $(query\text{-index-set-hit } s \text{ query-sampling-success-space}) s \leq$
 $nnreal \text{ rounds} * query\text{-error-bound} + H$
by (rule
 $wp\text{-query-index-set-hit-bound-if-supported-partial-header-candidate-unique-or-collision}$
 $[OF \text{ future raw-bound unique query-sampling-success-space-subset}$
 $query\text{-sampling-success-space-fraction-bound-query-sample-space}$
 $collision-bound])$

lemma *composition-fri-supported-root-composition-table-candidate-state-hash-extends:*

assumes
 $composition\text{-fri-supported-root-composition-table-candidate-state } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots composition-table}$
 $final\text{-state}$
shows $s \leq final\text{-state}$
proof –
from *assms* **obtain** *result trace-table query-idxs trace-bs comp-bs final rest*
where *outcome:*
 $Some (result, final\text{-state}) \in set\text{-dist (execute verify-monad } s)$
unfolding *composition-fri-supported-root-composition-table-candidate-state-def*
by *blast*
show *?thesis*
by (rule *verify-monad-hash-extends[OF outcome]*)

qed

lemma *composition-fri-supported-root-composition-table-candidate-states-merkle-compatible-unique-if-clean-merge:*

assumes *compatible:*
 $merkle\text{-hash-maps-compatible } final\text{-state } final\text{-state}'$
and *clean:*
 $\neg hash\text{-map-output-collision}$
 $(merkle\text{-hash-state-merge } final\text{-state } final\text{-state}')$
and *candidate:*
 $composition\text{-fri-supported-root-composition-table-candidate-state } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots composition-table}$
 $final\text{-state}$
and *candidate':*
 $composition\text{-fri-supported-root-composition-table-candidate-state } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots composition-table}'$

final-state'
shows *composition-table = composition-table'*
proof –
from *candidate* **obtain** *result trace-table query-idxs trace-bs comp-bs final*
rest **where**
bound:
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
and *comp-nonempty: composition-fri-roots $\neq$ []*
and *bind*:
merkle-root-binds-table (hd composition-fri-roots) composition-table
final-state
unfolding *composition-fri-supported-root-composition-table-candidate-state-def*
by *blast*
from *candidate'* **obtain** *result' trace-table' query-idxs' trace-bs' comp-bs'*
final' rest' **where**
bound':
accepted-with-bound-tables s (Some (result', final-state'))
trace-table' composition-table' as query-idxs'
and *bind'*:
merkle-root-binds-table (hd composition-fri-roots) composition-table'
final-state'
unfolding *composition-fri-supported-root-composition-table-candidate-state-def*
by *blast*
have *same-len: length composition-table = length composition-table'*
using *accepted-with-bound-tables-shapes(2)[OF bound]*
accepted-with-bound-tables-shapes(2)[OF bound']
by *simp*
have *nonempty: composition-table $\neq$ []*
using *accepted-with-bound-tables-shapes(2)[OF bound]*
eval-domain-nontrivial **by** *auto*
show *?thesis*
by (*rule*
merkle-root-binds-same-length-tables-merkle-compatible-unique-if-clean-merge
[OF compatible clean bind bind' same-len nonempty])
qed

lemma *composition-fri-supported-root-composition-table-candidates-subset-singleton-if-common-clean-merkle-extension*
assumes *clean: $\neg$ hash-map-output-collision u*
and *common-ext:*
 $\bigwedge$ composition-table final-state.
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table
final-state $\implies$
merkle-hash-extends final-state u
shows
 $\exists$ *composition-table.*
composition-fri-supported-root-composition-table-candidates s fr
f-fri-roots f-final as dg composition-fri-roots $\subseteq$

```

      {composition-table}
proof (cases
  composition-fri-supported-root-composition-table-candidates s fr
    f-fri-roots f-final as dg composition-fri-roots = {})
case True
then show ?thesis
  by (intro exI[of - []]) simp
next
case False
then obtain composition-table0 where table0:
  composition-table0 ∈
    composition-fri-supported-root-composition-table-candidates s fr
    f-fri-roots f-final as dg composition-fri-roots
by blast
then obtain final-state0 where state0:
  composition-fri-supported-root-composition-table-candidate-state s fr
    f-fri-roots f-final as dg composition-fri-roots composition-table0
    final-state0
using composition-fri-supported-root-composition-table-candidates-iff-state
by blast
have subset:
  composition-fri-supported-root-composition-table-candidates s fr
    f-fri-roots f-final as dg composition-fri-roots ⊆
    {composition-table0}
proof
fix composition-table
assume table:
  composition-table ∈
    composition-fri-supported-root-composition-table-candidates s fr
    f-fri-roots f-final as dg composition-fri-roots
then obtain final-state where state:
  composition-fri-supported-root-composition-table-candidate-state s fr
    f-fri-roots f-final as dg composition-fri-roots composition-table
    final-state
using composition-fri-supported-root-composition-table-candidates-iff-state
by blast
have ext: merkle-hash-extends final-state u
  by (rule common-ext[OF state])
have ext0: merkle-hash-extends final-state0 u
  by (rule common-ext[OF state0])
have clean-pair:
  ¬ merkle-hash-value-conflict final-state final-state0 ∧
  ¬ hash-map-output-collision
    (merkle-hash-state-merge final-state final-state0)
by (rule common-clean-merkle-extension-imp-pairwise-merkle-clean
    [OF ext ext0 clean])
have compatible:
  merkle-hash-maps-compatible final-state final-state0
using clean-pair

```

```

unfolding merkle-hash-maps-compatible-iff-no-value-conflict by simp
have clean-merge:
   $\neg$  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state0)
using clean-pair by simp
have composition-table = composition-table0
by (rule
  composition-fri-supported-root-composition-table-candidate-states-merkle-compatible-unique-if-clean-merge
    [OF compatible clean-merge state state0])
then show composition-table  $\in$  {composition-table0}
by simp
qed
then show ?thesis
by blast
qed

lemma composition-fri-supported-root-composition-table-candidate-states-distinct-imp-merkle-conflict-or-merge-collision
assumes candidate:
  composition-fri-supported-root-composition-table-candidate-state s fr
  f-fri-roots f-final as dg composition-fri-roots composition-table
  final-state
and candidate':
  composition-fri-supported-root-composition-table-candidate-state s fr
  f-fri-roots f-final as dg composition-fri-roots composition-table'
  final-state'
and distinct-tables: composition-table  $\neq$  composition-table'
shows
  merkle-hash-value-conflict final-state final-state'  $\vee$ 
  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
proof (rule ccontr)
assume no-bad:
   $\neg$  (merkle-hash-value-conflict final-state final-state'  $\vee$ 
    hash-map-output-collision
      (merkle-hash-state-merge final-state final-state'))
have compatible:
  merkle-hash-maps-compatible final-state final-state'
using no-bad unfolding merkle-hash-maps-compatible-iff-no-value-conflict
by simp
have clean:
   $\neg$  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
using no-bad by simp
have composition-table = composition-table'
by (rule
  composition-fri-supported-root-composition-table-candidate-states-merkle-compatible-unique-if-clean-merge
    [OF compatible clean candidate candidate'])
then show False
using distinct-tables by contradiction

```

qed

lemma *composition-fri-supported-root-composition-table-candidates-not-singleton-imp-pairwise-merkle-bad:*

assumes *not-unique:*

$\neg (\exists \text{ composition-table.}$
 composition-fri-supported-root-composition-table-candidates s fr
 f-fri-roots f-final as dg composition-fri-roots $\subseteq$
 { composition-table })

shows

$\exists \text{ composition-table final-state composition-table' final-state'.$
 composition-fri-supported-root-composition-table-candidate-state s fr
 f-fri-roots f-final as dg composition-fri-roots composition-table
 final-state $\wedge$
 composition-fri-supported-root-composition-table-candidate-state s fr
 f-fri-roots f-final as dg composition-fri-roots composition-table'
 final-state' $\wedge$
 composition-table $\neq$ composition-table' $\wedge$
 (merkle-hash-value-conflict final-state final-state' $\vee$
 hash-map-output-collision
 (merkle-hash-state-merge final-state final-state'))

proof –

let *?C* =

composition-fri-supported-root-composition-table-candidates s fr
 f-fri-roots f-final as dg composition-fri-roots

have *nonempty: ?C $\neq$ {}*

using *not-unique* **by** *auto*

then obtain *composition-table* **where** *table: composition-table $\in$?C*
 by *blast*

have $\neg ?C \subseteq \{\text{composition-table}\}$

using *not-unique* **by** *blast*

then obtain *composition-table'* **where** *table':*

composition-table' $\in$?C

and *distinct-tables: composition-table $\neq$ composition-table'*

by *auto*

from *table* **obtain** *final-state* **where** *candidate:*

composition-fri-supported-root-composition-table-candidate-state s fr
 f-fri-roots f-final as dg composition-fri-roots composition-table
 final-state

using *composition-fri-supported-root-composition-table-candidates-iff-state*

by *blast*

from *table'* **obtain** *final-state'* **where** *candidate':*

composition-fri-supported-root-composition-table-candidate-state s fr
 f-fri-roots f-final as dg composition-fri-roots composition-table'
 final-state'

using *composition-fri-supported-root-composition-table-candidates-iff-state*

by *blast*

have *bad:*

merkle-hash-value-conflict final-state final-state' $\vee$
 hash-map-output-collision

(merkle-hash-state-merge final-state final-state')
 by (rule
 composition-fri-supported-root-composition-table-candidate-states-distinct-imp-merkle-conflict-or-merge-collision
 [OF candidate candidate' distinct-tables])
 show ?thesis
 using candidate candidate' distinct-tables bad by blast
 qed

definition *composition-fri-supported-root-pairwise-merkle-bad*

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$
 'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$
 'f list $\Rightarrow$ bool

where

composition-fri-supported-root-pairwise-merkle-bad s fr f-fri-roots
 f-final as dg composition-fri-roots $\longleftrightarrow$
 ($\exists$ composition-table final-state composition-table' final-state'.
 composition-fri-supported-root-composition-table-candidate-state s fr
 f-fri-roots f-final as dg composition-fri-roots composition-table
 final-state $\wedge$
 composition-fri-supported-root-composition-table-candidate-state s fr
 f-fri-roots f-final as dg composition-fri-roots composition-table'
 final-state' $\wedge$
 composition-table $\neq$ composition-table' $\wedge$
 (merkle-hash-value-conflict final-state final-state' $\vee$
 hash-map-output-collision
 (merkle-hash-state-merge final-state final-state'))))

definition *composition-fri-supported-pairwise-merkle-bad*

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$ bool

where

composition-fri-supported-pairwise-merkle-bad s $\longleftrightarrow$
 ($\exists$ fr f-fri-roots f-final as dg composition-fri-roots.
 composition-fri-supported-root-pairwise-merkle-bad s fr f-fri-roots
 f-final as dg composition-fri-roots)

lemma *composition-fri-supported-root-pairwise-merkle-bad-extends-initial:*

assumes bad:

 composition-fri-supported-root-pairwise-merkle-bad s fr f-fri-roots
 f-final as dg composition-fri-roots

obtains composition-table final-state composition-table' final-state'

where

composition-fri-supported-root-composition-table-candidate-state s fr
 f-fri-roots f-final as dg composition-fri-roots composition-table
 final-state
 composition-fri-supported-root-composition-table-candidate-state s fr
 f-fri-roots f-final as dg composition-fri-roots composition-table'
 final-state'
 composition-table $\neq$ composition-table'
 merkle-hash-value-conflict final-state final-state' $\vee$

hash-map-output-collision
 (merkle-hash-state-merge *final-state final-state'*)
 $s \leq \text{final-state}$
 $s \leq \text{final-state}'$
proof –
from *bad* **obtain** *composition-table final-state composition-table' final-state'*
where *candidate*:
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table
final-state
and *candidate'*:
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table'
final-state'
and *distinct*: *composition-table* $\neq$ *composition-table'*
and *pair-bad*:
merkle-hash-value-conflict final-state final-state' $\vee$
hash-map-output-collision
 (merkle-hash-state-merge *final-state final-state'*)
unfolding *composition-fri-supported-root-pairwise-merkle-bad-def*
by *blast*
have *ext*: $s \leq \text{final-state}$
by (rule
composition-fri-supported-root-composition-table-candidate-state-hash-extends
 [OF *candidate*])
have *ext'*: $s \leq \text{final-state}'$
by (rule
composition-fri-supported-root-composition-table-candidate-state-hash-extends
 [OF *candidate'*])
show ?thesis
by (rule that[OF *candidate candidate' distinct pair-bad ext ext'*])
qed

definition *composition-fri-supported-root-pairwise-coupling-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow \text{bool}$
where
composition-fri-supported-root-pairwise-coupling-bad s fr f-fri-roots
f-final as dg composition-fri-roots $\longleftrightarrow$
 ($\exists$ *composition-table final-state composition-table' final-state'*.
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table
final-state $\wedge$
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table'
final-state' $\wedge$
composition-table $\neq$ composition-table' $\wedge$
merkle-hash-pairwise-coupling-bad final-state final-state')

definition *composition-fri-supported-pairwise-coupling-bad*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

composition-fri-supported-pairwise-coupling-bad $s \longleftrightarrow$

$(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots}.$

$\text{composition-fri-supported-root-pairwise-coupling-bad } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final as dg } \text{composition-fri-roots})$

lemma *composition-fri-supported-root-pairwise-merkle-bad-imp-local-collision-or-coupling-bad:*

assumes *bad:*

composition-fri-supported-root-pairwise-merkle-bad $s \text{ fr } f\text{-fri-roots}$

$f\text{-final as dg } \text{composition-fri-roots}$

shows

$(\exists \text{ composition-table final-state composition-table' final-state'.$

$\text{composition-fri-supported-root-composition-table-candidate-state } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots } \text{composition-table}$
 $\text{final-state} \wedge$

$\text{composition-fri-supported-root-composition-table-candidate-state } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots } \text{composition-table'}$
 $\text{final-state'} \wedge$

$\text{composition-table} \neq \text{composition-table'} \wedge$

$(\text{hash-map-output-collision final-state} \vee$
 $\text{hash-map-output-collision final-state'}) \vee$

$\text{composition-fri-supported-root-pairwise-coupling-bad } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final as dg } \text{composition-fri-roots})$

proof —

from *bad* **obtain** *composition-table final-state composition-table' final-state'*

where *candidate:*

composition-fri-supported-root-composition-table-candidate-state $s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots } \text{composition-table}$
 final-state

and *candidate':*

composition-fri-supported-root-composition-table-candidate-state $s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots } \text{composition-table'}$
 final-state'

and *distinct:* $\text{composition-table} \neq \text{composition-table'}$

and *pair-bad:*

$\text{merkle-hash-value-conflict final-state final-state'} \vee$

$\text{hash-map-output-collision}$

$(\text{merkle-hash-state-merge final-state final-state'})$

unfolding *composition-fri-supported-root-pairwise-merkle-bad-def*

by *blast*

have *decomp:*

$\text{hash-map-output-collision final-state} \vee$

$\text{hash-map-output-collision final-state'} \vee$

$\text{merkle-hash-pairwise-coupling-bad final-state final-state'}$

by $(\text{rule } \text{merkle-hash-pairwise-bad-imp-local-collision-or-coupling-bad}$

$[\text{OF pair-bad}])$


```

then show ?thesis
proof
  assume hash-map-output-collision final-state
  then show ?thesis
    using candidate candidate' distinct by blast
next
assume rest:
  hash-map-output-collision final-state'  $\vee$ 
  merkle-hash-pairwise-coupling-bad final-state final-state'
then show ?thesis
proof
  assume hash-map-output-collision final-state'
  then show ?thesis
    using candidate candidate' distinct by blast
next
assume merkle-hash-pairwise-coupling-bad final-state final-state'
then have
  composition-fri-supported-root-pairwise-coupling-bad s fr f-fri-roots
  f-final as dg composition-fri-roots
  unfolding composition-fri-supported-root-pairwise-coupling-bad-def
  using candidate candidate' distinct by blast
then show ?thesis by simp
qed
qed
qed

lemma composition-fri-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad:
  assumes bad: composition-fri-supported-pairwise-merkle-bad s
  shows
    ( $\exists$  fr f-fri-roots f-final as dg composition-fri-roots
      composition-table final-state composition-table' final-state'.
      composition-fri-supported-root-composition-table-candidate-state s fr
      f-fri-roots f-final as dg composition-fri-roots composition-table
      final-state  $\wedge$ 
      composition-fri-supported-root-composition-table-candidate-state s fr
      f-fri-roots f-final as dg composition-fri-roots composition-table'
      final-state'  $\wedge$ 
      composition-table  $\neq$  composition-table'  $\wedge$ 
      (hash-map-output-collision final-state  $\vee$ 
      hash-map-output-collision final-state'))  $\vee$ 
      composition-fri-supported-pairwise-coupling-bad s
  proof -
    from bad obtain fr f-fri-roots f-final as dg composition-fri-roots where
      root-bad:
        composition-fri-supported-root-pairwise-merkle-bad s fr f-fri-roots
        f-final as dg composition-fri-roots
    unfolding composition-fri-supported-pairwise-merkle-bad-def by blast
  from
    composition-fri-supported-root-pairwise-merkle-bad-imp-local-collision-or-coupling-bad

```

$[OF \text{ root-bad}]$
show *?thesis*
unfolding *composition-fri-supported-pairwise-coupling-bad-def* **by** *blast*
qed

lemma *composition-fri-supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad*:
assumes *bad: composition-fri-supported-pairwise-merkle-bad s*
shows
supported-hash-output-collision-possible s $\vee$
composition-fri-supported-pairwise-coupling-bad s
proof –
have *decomp*:
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots}$
 $\text{composition-table final-state composition-table' final-state'.$
 $\text{composition-fri-supported-root-composition-table-candidate-state s fr}$
 $f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots composition-table}$
 $\text{final-state} \wedge$
 $\text{composition-fri-supported-root-composition-table-candidate-state s fr}$
 $f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots composition-table'}$
 $\text{final-state'} \wedge$
 $\text{composition-table} \neq \text{composition-table'} \wedge$
 $(\text{hash-map-output-collision final-state} \vee$
 $\text{hash-map-output-collision final-state'})) \vee$
 $\text{composition-fri-supported-pairwise-coupling-bad s}$
by (*rule*
 $\text{composition-fri-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad}$
 $[OF \text{ bad}]$)
then show *?thesis*
proof
assume *local*:
 $\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots}$
 $\text{composition-table final-state composition-table' final-state'.$
 $\text{composition-fri-supported-root-composition-table-candidate-state s fr}$
 $f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots composition-table}$
 $\text{final-state} \wedge$
 $\text{composition-fri-supported-root-composition-table-candidate-state s fr}$
 $f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots composition-table'}$
 $\text{final-state'} \wedge$
 $\text{composition-table} \neq \text{composition-table'} \wedge$
 $(\text{hash-map-output-collision final-state} \vee$
 $\text{hash-map-output-collision final-state'})$
then obtain $\text{fr } f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots}$
 $\text{composition-table final-state composition-table' final-state'}$ **where**
candidate:
 $\text{composition-fri-supported-root-composition-table-candidate-state s fr}$
 $f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots composition-table}$
 final-state
and *candidate'*:
 $\text{composition-fri-supported-root-composition-table-candidate-state s fr}$

```

    f-fri-roots f-final as dg composition-fri-roots composition-table'
    final-state'
  and collision:
    hash-map-output-collision final-state ∨
    hash-map-output-collision final-state'
  by blast
from collision show ?thesis
proof
  assume collision-final: hash-map-output-collision final-state
  from candidate obtain result trace-table query-idxs trace-bs comp-bs final
    rest where
  outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
  unfolding
    composition-fri-supported-root-composition-table-candidate-state-def
  by blast
have supported-hash-output-collision-possible s
  by (rule
    accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible
    [OF outcome bound collision-final])
  then show ?thesis by simp
next
  assume collision-final': hash-map-output-collision final-state'
  from candidate' obtain result' trace-table' query-idxs' trace-bs'
    comp-bs' final' rest' where
  outcome':
    Some (result', final-state') ∈ set-dist (execute verify-monad s)
  and bound':
    accepted-with-bound-tables s (Some (result', final-state'))
    trace-table' composition-table' as query-idxs'
  unfolding
    composition-fri-supported-root-composition-table-candidate-state-def
  by blast
have supported-hash-output-collision-possible s
  by (rule
    accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible
    [OF outcome' bound' collision-final'])
  then show ?thesis by simp
qed
next
  assume composition-fri-supported-pairwise-coupling-bad s
  then show ?thesis by simp
qed
qed
definition alpha-output-local-candidate-ambiguity-bad

```

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{alpha-output-local-candidate-ambiguity-bad } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table alphas query-idxs}$
 $\text{ trace-table' composition-table' alphas' query-idxs'.$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table}$
 $\text{ alphas query-idxs} \wedge$
 $\text{accepted-with-bound-tables } s \text{ out trace-table' composition-table'}$
 $\text{ alphas' query-idxs' } \wedge$
 $\text{trace-table} \neq \text{trace-table'})$

definition *query-output-local-candidate-ambiguity-bad*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{query-output-local-candidate-ambiguity-bad } s \text{ out} \longleftrightarrow$
 $\text{accepted-bound-table-ambiguity-bad } s \text{ out}$

definition *trace-fri-output-local-candidate-ambiguity-bad*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-output-local-candidate-ambiguity-bad } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table alphas query-idxs}$
 $\text{ trace-table' composition-table' alphas' query-idxs'}$
 $\text{ trace-bs dg composition-bs.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table}$
 $\text{ alphas query-idxs} \wedge$
 $\text{accepted-with-bound-tables } s \text{ out trace-table' composition-table'}$
 $\text{ alphas' query-idxs' } \wedge$
 $\text{accepted-fri-challenges } s \text{ out trace-bs dg composition-bs} \wedge$
 $\text{trace-table} \neq \text{trace-table'})$

definition *composition-fri-output-local-candidate-ambiguity-bad*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-fri-output-local-candidate-ambiguity-bad } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table alphas query-idxs}$
 $\text{ trace-table' composition-table' alphas' query-idxs'}$
 $\text{ trace-bs dg composition-bs.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table}$
 $\text{ alphas query-idxs} \wedge$
 $\text{accepted-with-bound-tables } s \text{ out trace-table' composition-table'}$
 $\text{ alphas' query-idxs' } \wedge$
 $\text{accepted-fri-challenges } s \text{ out trace-bs dg composition-bs} \wedge$
 $\text{composition-table} \neq \text{composition-table'})$

definition *supported-output-local-candidate-ambiguity-bad*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

supported-output-local-candidate-ambiguity-bad s out $\longleftrightarrow$
alpha-output-local-candidate-ambiguity-bad s out $\vee$
query-output-local-candidate-ambiguity-bad s out $\vee$
trace-fri-output-local-candidate-ambiguity-bad s out $\vee$
composition-fri-output-local-candidate-ambiguity-bad s out

lemma *alpha-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad:*

assumes *alpha-output-local-candidate-ambiguity-bad s out*

shows *accepted-bound-table-ambiguity-bad s out*

using *assms*

unfolding *alpha-output-local-candidate-ambiguity-bad-def*

accepted-bound-table-ambiguity-bad-def

by *blast*

lemma *query-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad:*

assumes *query-output-local-candidate-ambiguity-bad s out*

shows *accepted-bound-table-ambiguity-bad s out*

using *assms* **unfolding** *query-output-local-candidate-ambiguity-bad-def* .

lemma *trace-fri-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad:*

assumes *trace-fri-output-local-candidate-ambiguity-bad s out*

shows *accepted-bound-table-ambiguity-bad s out*

using *assms*

unfolding *trace-fri-output-local-candidate-ambiguity-bad-def*

accepted-bound-table-ambiguity-bad-def

by *blast*

lemma *composition-fri-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad:*

assumes *composition-fri-output-local-candidate-ambiguity-bad s out*

shows *accepted-bound-table-ambiguity-bad s out*

using *assms*

unfolding *composition-fri-output-local-candidate-ambiguity-bad-def*

accepted-bound-table-ambiguity-bad-def

by *blast*

lemma *supported-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad:*

assumes *supported-output-local-candidate-ambiguity-bad s out*

shows *accepted-bound-table-ambiguity-bad s out*

using *assms*

unfolding *supported-output-local-candidate-ambiguity-bad-def*

using

alpha-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad

query-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad

trace-fri-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad

composition-fri-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad

by blast

lemma supported-output-local-candidate-ambiguity-bad-imp-hash-map-output-collision-bad:

assumes supported-output-local-candidate-ambiguity-bad s out

shows hash-map-output-collision-bad s out

by (rule

accepted-bound-table-ambiguity-bad-imp-hash-map-output-collision-bad)

(rule

supported-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad

[OF assms])

lemma supported-output-local-candidate-ambiguity-bad-mono-hash-map-output-collision-bad:

wp-event verify-monad

(supported-output-local-candidate-ambiguity-bad s) s ≤

wp-event verify-monad (hash-map-output-collision-bad s) s

by (rule wp-event-mono)

(rule

supported-output-local-candidate-ambiguity-bad-imp-hash-map-output-collision-bad)

definition supported-output-local-side-bad

:: (f, 'a) protocol-channel-scheme ⇒

(unit list × (f, 'a) protocol-channel-scheme) option ⇒ bool

where

supported-output-local-side-bad s out ⇔

hash-map-output-collision-bad s out ∨

supported-output-local-candidate-ambiguity-bad s out

lemma supported-output-local-side-bad-iff-hash-map-output-collision-bad:

supported-output-local-side-bad s out ⇔

hash-map-output-collision-bad s out

unfolding supported-output-local-side-bad-def

using

supported-output-local-candidate-ambiguity-bad-imp-hash-map-output-collision-bad

by blast

lemma supported-output-local-side-bad-imp-accepted:

assumes supported-output-local-side-bad s out

shows accepted out

using assms

unfolding supported-output-local-side-bad-iff-hash-map-output-collision-bad

by (rule hash-map-output-collision-bad-imp-accepted)

lemma supported-output-local-side-bad-mono-hash-map-output-collision-bad:

wp-event verify-monad (supported-output-local-side-bad s) s ≤

wp-event verify-monad (hash-map-output-collision-bad s) s

by (rule wp-event-mono)

(simp add: supported-output-local-side-bad-iff-hash-map-output-collision-bad)

end

end

theory *Soundness-Reductions-Side*
imports *Soundness-Reductions-Composition*
begin

Supported pairwise/output-local side-event reductions.

context *soundness*
begin

The following cross-continuation predicates are retained temporarily as a record of the earlier proof route. The refined route below does not charge same-key/different-output lazy-oracle continuations as bad events; it uses *supported-output-local-side-bad* instead.

definition *supported-pairwise-merkle-bad*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
supported-pairwise-merkle-bad $s \longleftrightarrow$
 $\alpha\text{-supported-pairwise-merkle-bad } s \vee$
 $\text{query-supported-pairwise-merkle-bad } s \vee$
 $\text{trace-fri-supported-pairwise-merkle-bad } s \vee$
 $\text{composition-fri-supported-pairwise-merkle-bad } s$

definition *supported-pairwise-coupling-bad*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
supported-pairwise-coupling-bad $s \longleftrightarrow$
 $\alpha\text{-supported-pairwise-coupling-bad } s \vee$
 $\text{query-supported-pairwise-coupling-bad } s \vee$
 $\text{trace-fri-supported-pairwise-coupling-bad } s \vee$
 $\text{composition-fri-supported-pairwise-coupling-bad } s$

lemma *supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad*:
assumes *bad*: *supported-pairwise-merkle-bad* s
shows
 $\text{supported-hash-output-collision-possible } s \vee$
 $\text{supported-pairwise-coupling-bad } s$
using *assms*
unfolding *supported-pairwise-merkle-bad-def*
supported-pairwise-coupling-bad-def
using
 $\alpha\text{-supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad}$
 $\text{query-supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad}$
 $\text{trace-fri-supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad}$
 $\text{composition-fri-supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad}$
by *blast*

lemma *no-supported-pairwise-merkle-bad-if-no-hash-collision-possible-no-coupling-bad*:

assumes *no-collision*: $\neg$ *supported-hash-output-collision-possible* *s*
and *no-coupling*: $\neg$ *supported-pairwise-coupling-bad* *s*
shows $\neg$ *supported-pairwise-merkle-bad* *s*
proof
assume *bad*: *supported-pairwise-merkle-bad* *s*
have *supported-hash-output-collision-possible* *s* $\vee$
supported-pairwise-coupling-bad *s*
by (rule
supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad
[OF *bad*])
then show *False*
using *no-collision no-coupling* **by** blast
qed

lemma *no-alpha-supported-pairwise-merkle-bad-if-no-pairwise-side-bad*:
assumes *no-collision*: $\neg$ *supported-hash-output-collision-possible* *s*
and *no-coupling*: $\neg$ *supported-pairwise-coupling-bad* *s*
shows $\neg$ *alpha-supported-pairwise-merkle-bad* *s*
proof –
have $\neg$ *supported-pairwise-merkle-bad* *s*
by (rule
no-supported-pairwise-merkle-bad-if-no-hash-collision-possible-no-coupling-bad
[OF *no-collision no-coupling*])
then show ?thesis
unfolding *supported-pairwise-merkle-bad-def* **by** blast
qed

lemma *no-query-supported-pairwise-merkle-bad-if-no-pairwise-side-bad*:
assumes *no-collision*: $\neg$ *supported-hash-output-collision-possible* *s*
and *no-coupling*: $\neg$ *supported-pairwise-coupling-bad* *s*
shows $\neg$ *query-supported-pairwise-merkle-bad* *s*
proof –
have $\neg$ *supported-pairwise-merkle-bad* *s*
by (rule
no-supported-pairwise-merkle-bad-if-no-hash-collision-possible-no-coupling-bad
[OF *no-collision no-coupling*])
then show ?thesis
unfolding *supported-pairwise-merkle-bad-def* **by** blast
qed

lemma *no-trace-fri-supported-pairwise-merkle-bad-if-no-pairwise-side-bad*:
assumes *no-collision*: $\neg$ *supported-hash-output-collision-possible* *s*
and *no-coupling*: $\neg$ *supported-pairwise-coupling-bad* *s*
shows $\neg$ *trace-fri-supported-pairwise-merkle-bad* *s*
proof –
have $\neg$ *supported-pairwise-merkle-bad* *s*
by (rule
no-supported-pairwise-merkle-bad-if-no-hash-collision-possible-no-coupling-bad
[OF *no-collision no-coupling*])

then show ?thesis
unfolding supported-pairwise-merkle-bad-def **by** blast
qed

lemma no-composition-fri-supported-pairwise-merkle-bad-if-no-pairwise-side-bad:
assumes no-collision: $\neg$ supported-hash-output-collision-possible s
and no-coupling: $\neg$ supported-pairwise-coupling-bad s
shows $\neg$ composition-fri-supported-pairwise-merkle-bad s
proof –
have $\neg$ supported-pairwise-merkle-bad s
by (rule
no-supported-pairwise-merkle-bad-if-no-hash-collision-possible-no-coupling-bad
[OF no-collision no-coupling])
then show ?thesis
unfolding supported-pairwise-merkle-bad-def **by** blast
qed

definition alpha-supported-pairwise-coupling-event
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
alpha-supported-pairwise-coupling-event $s \text{ out} \longleftrightarrow$
 $(\exists \text{result final-state fr f-fri-roots f-final trace-table}$
 $\text{trace-table}' \text{ final-state'}$
 $\text{out} = \text{Some} (\text{result}, \text{final-state}) \wedge$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr f-fri-roots}$
 $\text{f-final trace-table final-state} \wedge$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr f-fri-roots}$
 $\text{f-final trace-table}' \text{ final-state}' \wedge$
 $\text{trace-table} \neq \text{trace-table}' \wedge$
 $\text{merkle-hash-pairwise-coupling-bad final-state final-state'})$

definition query-supported-pairwise-coupling-event
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
query-supported-pairwise-coupling-event $s \text{ out} \longleftrightarrow$
 $(\exists \text{result final-state fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-table composition-table}$
 $\text{trace-table}' \text{ composition-table}' \text{ final-state'}$
 $\text{out} = \text{Some} (\text{result}, \text{final-state}) \wedge$
 $\text{query-header-supported-table-candidate-state } s \text{ fr f-fri-roots f-final}$
 $\text{as dg composition-fri-roots final trace-table composition-table}$
 $\text{final-state} \wedge$
 $\text{query-header-supported-table-candidate-state } s \text{ fr f-fri-roots f-final}$
 $\text{as dg composition-fri-roots final trace-table}' \text{ composition-table}'$
 $\text{final-state}' \wedge$
 $(\text{trace-table} \neq \text{trace-table}' \vee$
 $\text{composition-table} \neq \text{composition-table}'))$

merkle-hash-pairwise-coupling-bad final-state final-state')

definition *trace-fri-supported-pairwise-coupling-event*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-supported-pairwise-coupling-event s out $\longleftrightarrow$
 $(\exists \text{result final-state fr trace-table trace-table' final-state'.$
 $\text{out} = \text{Some} (\text{result}, \text{final-state}) \wedge$
 $\text{trace-fri-supported-root-trace-table-candidate-state s fr trace-table}$
 $\text{final-state} \wedge$
 $\text{trace-fri-supported-root-trace-table-candidate-state s fr trace-table'}$
 $\text{final-state'} \wedge$
 $\text{trace-table} \neq \text{trace-table'} \wedge$
 $\text{merkle-hash-pairwise-coupling-bad final-state final-state'})$

definition *composition-fri-supported-pairwise-coupling-event*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-supported-pairwise-coupling-event s out $\longleftrightarrow$
 $(\exists \text{result final-state fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots composition-table composition-table'}$
 $\text{final-state'.$
 $\text{out} = \text{Some} (\text{result}, \text{final-state}) \wedge$
 $\text{composition-fri-supported-root-composition-table-candidate-state s fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots composition-table}$
 $\text{final-state} \wedge$
 $\text{composition-fri-supported-root-composition-table-candidate-state s fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots composition-table'}$
 $\text{final-state'} \wedge$
 $\text{composition-table} \neq \text{composition-table'} \wedge$
 $\text{merkle-hash-pairwise-coupling-bad final-state final-state'})$

definition *supported-pairwise-coupling-event*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

supported-pairwise-coupling-event s out $\longleftrightarrow$
 $\alpha\text{-supported-pairwise-coupling-event s out} \vee$
 $\text{query-supported-pairwise-coupling-event s out} \vee$
 $\text{trace-fri-supported-pairwise-coupling-event s out} \vee$
 $\text{composition-fri-supported-pairwise-coupling-event s out}$

definition *supported-pairwise-side-bad*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

supported-pairwise-side-bad s out $\longleftrightarrow$

hash-map-output-collision-bad s out $\vee$
supported-pairwise-coupling-event s out

lemma *alpha-supported-pairwise-coupling-event-imp-accepted:*
assumes *alpha-supported-pairwise-coupling-event s out*
shows *accepted out*
using *assms*
unfolding *alpha-supported-pairwise-coupling-event-def accepted-def*
by *auto*

lemma *query-supported-pairwise-coupling-event-imp-accepted:*
assumes *query-supported-pairwise-coupling-event s out*
shows *accepted out*
using *assms*
unfolding *query-supported-pairwise-coupling-event-def accepted-def*
by *auto*

lemma *trace-fri-supported-pairwise-coupling-event-imp-accepted:*
assumes *trace-fri-supported-pairwise-coupling-event s out*
shows *accepted out*
using *assms*
unfolding *trace-fri-supported-pairwise-coupling-event-def accepted-def*
by *auto*

lemma *composition-fri-supported-pairwise-coupling-event-imp-accepted:*
assumes *composition-fri-supported-pairwise-coupling-event s out*
shows *accepted out*
using *assms*
unfolding *composition-fri-supported-pairwise-coupling-event-def accepted-def*
by *auto*

lemma *supported-pairwise-coupling-event-imp-accepted:*
assumes *supported-pairwise-coupling-event s out*
shows *accepted out*
using *assms*
unfolding *supported-pairwise-coupling-event-def*
using *alpha-supported-pairwise-coupling-event-imp-accepted*
query-supported-pairwise-coupling-event-imp-accepted
trace-fri-supported-pairwise-coupling-event-imp-accepted
composition-fri-supported-pairwise-coupling-event-imp-accepted
by *blast*

lemma *supported-pairwise-side-bad-imp-accepted:*
assumes *supported-pairwise-side-bad s out*
shows *accepted out*
using *assms*
unfolding *supported-pairwise-side-bad-def*
using *hash-map-output-collision-bad-imp-accepted*
supported-pairwise-coupling-event-imp-accepted

by blast

lemma supported-pairwise-side-bad-mono-accepted:

wp-event verify-monad (supported-pairwise-side-bad s) s ≤
wp-event verify-monad accepted s

by (rule wp-event-mono)
(rule supported-pairwise-side-bad-imp-accepted)

lemma supported-pairwise-side-bad-union-bound:

fixes hash-error coupling-error :: prob

assumes hash-bound:

wp-event verify-monad (hash-map-output-collision-bad s) s ≤
hash-error

and coupling-bound:

wp-event verify-monad (supported-pairwise-coupling-event s) s ≤
coupling-error

shows

wp-event verify-monad (supported-pairwise-side-bad s) s ≤
hash-error + coupling-error

proof –

have wp-event verify-monad (supported-pairwise-side-bad s) s ≤
wp-event verify-monad (hash-map-output-collision-bad s) s +
wp-event verify-monad (supported-pairwise-coupling-event s) s

unfolding supported-pairwise-side-bad-def
supported-pairwise-coupling-event-def

by (rule wp-event-union-bound)

also have ... ≤ hash-error + coupling-error

by (intro add-mono hash-bound coupling-bound)

finally show ?thesis .

qed

lemma verifier-initial-hash-map-output-collision-bad-bound-concrete:

wp-event verify-monad
(hash-map-output-collision-bad (verifier-initial-state tr))
(verifier-initial-state tr) ≤ concrete-merkle-binding-error

proof –

let ?s = verifier-initial-state tr

have wp-event verify-monad (hash-map-output-collision-bad ?s) ?s ≤
wp-event verify-monad (hash-map-new-output-collision-bad ?s) ?s

by (rule wp-event-mono)
(rule hash-map-output-collision-bad-imp-new-collision-bad-if-initial-clean,
simp, assumption)

also have ... ≤ concrete-merkle-binding-error

by (rule verifier-initial-hash-map-new-output-collision-bad-bound-concrete)

finally show ?thesis .

qed

lemma verifier-initial-supported-output-local-candidate-ambiguity-bad-bound-concrete:

wp-event verify-monad

(supported-output-local-candidate-ambiguity-bad
 (verifier-initial-state tr))
 (verifier-initial-state tr) $\leq$
 concrete-merkle-binding-error
proof –
have wp-event verify-monad
 (supported-output-local-candidate-ambiguity-bad
 (verifier-initial-state tr))
 (verifier-initial-state tr) $\leq$
 wp-event verify-monad
 (hash-map-output-collision-bad (verifier-initial-state tr))
 (verifier-initial-state tr)
by (rule
 supported-output-local-candidate-ambiguity-bad-mono-hash-map-output-collision-bad)
also have ... $\leq$ concrete-merkle-binding-error
by (rule verifier-initial-hash-map-output-collision-bad-bound-concrete)
finally show ?thesis .
qed

lemma verifier-initial-supported-output-local-side-bad-bound-concrete:
 wp-event verify-monad
 (supported-output-local-side-bad (verifier-initial-state tr))
 (verifier-initial-state tr) $\leq$
 concrete-merkle-binding-error
unfolding supported-output-local-side-bad-iff-hash-map-output-collision-bad
by (rule verifier-initial-hash-map-output-collision-bad-bound-concrete)

lemma verifier-initial-supported-pairwise-side-bad-bound-if-coupling-event-bound:
assumes coupling-bound:
 wp-event verify-monad
 (supported-pairwise-coupling-event (verifier-initial-state tr))
 (verifier-initial-state tr) $\leq$ coupling-error
shows
 wp-event verify-monad
 (supported-pairwise-side-bad (verifier-initial-state tr))
 (verifier-initial-state tr) $\leq$
 concrete-merkle-binding-error + coupling-error
by (rule supported-pairwise-side-bad-union-bound
 [OF verifier-initial-hash-map-output-collision-bad-bound-concrete
 coupling-bound])

lemma supported-hash-output-collision-possible-imp-exists-supported-pairwise-side-bad:
assumes supported-hash-output-collision-possible s
shows $\exists out \in \text{set-dist } (\text{execute verify-monad } s).$
 supported-pairwise-side-bad s out
using assms
unfolding supported-hash-output-collision-possible-def
 supported-pairwise-side-bad-def
by blast

lemma *alpha-supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad*:
assumes *alpha-bad*: *alpha-supported-pairwise-coupling-bad s*
shows $\exists out \in \text{set-dist } (\text{execute verify-monad } s).$
supported-pairwise-side-bad s out

proof —

from *alpha-bad* **obtain** *fr f-fri-roots f-final trace-table final-state*
trace-table' final-state' **where**
candidate:
alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table final-state
and *candidate'*:
alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table' final-state'
and *distinct*: *trace-table* $\neq$ *trace-table'*
and *pair-bad*:
merkle-hash-pairwise-coupling-bad final-state final-state'
unfolding *alpha-supported-pairwise-coupling-bad-def*
alpha-header-supported-pairwise-coupling-bad-def
by *blast*
from *candidate* **obtain** *result composition-table as query-idxs dg*
composition-fri-roots final rest **where**
outcome:
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *bound*:
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
unfolding *alpha-header-supported-trace-table-candidate-state-def* **by** *blast*
have *coupling*:
supported-pairwise-coupling-event s (Some (result, final-state))
unfolding *supported-pairwise-coupling-event-def*
alpha-supported-pairwise-coupling-event-def
using *candidate candidate' distinct pair-bad* **by** *blast*
have *side*:
supported-pairwise-side-bad s (Some (result, final-state))
unfolding *supported-pairwise-side-bad-def*
using *coupling* **by** *simp*
show *?thesis*
using *outcome side* **by** *blast*
qed

lemma *query-supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad*:
assumes *query-bad*: *query-supported-pairwise-coupling-bad s*
shows $\exists out \in \text{set-dist } (\text{execute verify-monad } s).$
supported-pairwise-side-bad s out

proof —

from *query-bad* **obtain** *fr f-fri-roots f-final as dg composition-fri-roots*
final trace-table composition-table final-state trace-table'
composition-table' final-state' **where**

candidate:
query-header-supported-table-candidate-state s fr f-fri-roots f-final
as dg composition-fri-roots final trace-table composition-table
final-state
and *candidate'*:
query-header-supported-table-candidate-state s fr f-fri-roots f-final
as dg composition-fri-roots final trace-table' composition-table'
final-state'
and *distinct:*
trace-table $\neq$ trace-table' $\vee$
composition-table $\neq$ composition-table'
and *pair-bad:*
merkle-hash-pairwise-coupling-bad final-state final-state'
unfolding *query-supported-pairwise-coupling-bad-def*
query-header-supported-pairwise-coupling-bad-def
by *blast*
from *candidate* **obtain** *result query-idxs rest* **where**
outcome:
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *bound:*
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
unfolding *query-header-supported-table-candidate-state-def* **by** *blast*
have *coupling:*
supported-pairwise-coupling-event s (Some (result, final-state))
unfolding *supported-pairwise-coupling-event-def*
query-supported-pairwise-coupling-event-def
using *candidate candidate' distinct pair-bad* **by** *blast*
have *side:*
supported-pairwise-side-bad s (Some (result, final-state))
unfolding *supported-pairwise-side-bad-def*
using *coupling* **by** *simp*
show *?thesis*
using *outcome side* **by** *blast*
qed

lemma *trace-fri-supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad:*
assumes *trace-bad: trace-fri-supported-pairwise-coupling-bad s*
shows $\exists out \in \text{set-dist (execute verify-monad s)}.$
supported-pairwise-side-bad s out

proof –
from *trace-bad* **obtain** *fr trace-table final-state trace-table' final-state'*
where *candidate:*
trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state
and *candidate'*:
trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
final-state'
and *distinct: trace-table $\neq$ trace-table'*

and *pair-bad*:
merkle-hash-pairwise-coupling-bad final-state final-state'
unfolding *trace-fri-supported-pairwise-coupling-bad-def*
trace-fri-supported-root-pairwise-coupling-bad-def
by *blast*
from *candidate* **obtain** *result composition-table as query-idxs trace-bs dg*
comp-bs f-fri-roots f-final header-as composition-fri-roots final rest
where *outcome*:
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *bound*:
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
unfolding *trace-fri-supported-root-trace-table-candidate-state-def*
by *blast*
have *coupling*:
supported-pairwise-coupling-event s (Some (result, final-state))
unfolding *supported-pairwise-coupling-event-def*
trace-fri-supported-pairwise-coupling-event-def
using *candidate candidate' distinct pair-bad* **by** *blast*
have *side*:
supported-pairwise-side-bad s (Some (result, final-state))
unfolding *supported-pairwise-side-bad-def*
using *coupling* **by** *simp*
show *?thesis*
using *outcome side* **by** *blast*
qed

lemma *composition-fri-supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad*:
assumes *composition-bad: composition-fri-supported-pairwise-coupling-bad s*
shows $\exists out \in set-dist (execute verify-monad s).$
supported-pairwise-side-bad s out
proof —
from *composition-bad* **obtain** *fr f-fri-roots f-final as dg*
composition-fri-roots composition-table final-state composition-table'
final-state' where
candidate:
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table
final-state
and *candidate'*:
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table'
final-state'
and *distinct: composition-table ≠ composition-table'*
and *pair-bad*:
merkle-hash-pairwise-coupling-bad final-state final-state'
unfolding *composition-fri-supported-pairwise-coupling-bad-def*
composition-fri-supported-root-pairwise-coupling-bad-def
by *blast*

from *candidate* **obtain** *result trace-table query-idxs trace-bs comp-bs final*
rest where
outcome:
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *bound:*
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
unfolding
composition-fri-supported-root-composition-table-candidate-state-def
by *blast*
have *coupling:*
supported-pairwise-coupling-event s (Some (result, final-state))
unfolding *supported-pairwise-coupling-event-def*
composition-fri-supported-pairwise-coupling-event-def
using *candidate candidate' distinct pair-bad* **by** *blast*
have *side:*
supported-pairwise-side-bad s (Some (result, final-state))
unfolding *supported-pairwise-side-bad-def*
using *coupling* **by** *simp*
show *?thesis*
using *outcome side* **by** *blast*
qed

lemma *supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad:*
assumes *coupling: supported-pairwise-coupling-bad s*
shows $\exists out \in \text{set-dist } (\text{execute verify-monad } s).$
supported-pairwise-side-bad s out
using *coupling*
unfolding *supported-pairwise-coupling-bad-def*
using
alpha-supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad
query-supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad
trace-fri-supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad
composition-fri-supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad
by *blast*

lemma *supported-pairwise-merkle-bad-imp-exists-supported-pairwise-side-bad:*
assumes *bad: supported-pairwise-merkle-bad s*
shows $\exists out \in \text{set-dist } (\text{execute verify-monad } s).$
supported-pairwise-side-bad s out
proof —
have *side:*
supported-hash-output-collision-possible s ∨
supported-pairwise-coupling-bad s
by *(rule*
supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad
[OF bad])
then show *?thesis*
proof

```

assume supported-hash-output-collision-possible s
then show ?thesis
  by (rule
    supported-hash-output-collision-possible-imp-exists-supported-pairwise-side-bad)
next
  assume supported-pairwise-coupling-bad s
  then show ?thesis
    by (rule
      supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad)
qed
qed

```

lemma *no-supported-pairwise-side-bad-on-support-imp-no-hash-collision-possible:*

```

assumes no-side:
   $\bigwedge out. out \in \text{set-dist } (\text{execute verify-monad } s) \implies$ 
   $\neg \text{supported-pairwise-side-bad } s \text{ out}$ 
shows  $\neg \text{supported-hash-output-collision-possible } s$ 
proof
  assume possible: supported-hash-output-collision-possible s
  from supported-hash-output-collision-possible-imp-exists-supported-pairwise-side-bad
    [OF possible]
  obtain out where out:
     $out \in \text{set-dist } (\text{execute verify-monad } s)$ 
  and side: supported-pairwise-side-bad s out
  by blast
  show False
  using no-side[OF out] side by contradiction
qed

```

lemma *no-supported-pairwise-side-bad-on-support-imp-no-coupling-bad:*

```

assumes no-side:
   $\bigwedge out. out \in \text{set-dist } (\text{execute verify-monad } s) \implies$ 
   $\neg \text{supported-pairwise-side-bad } s \text{ out}$ 
shows  $\neg \text{supported-pairwise-coupling-bad } s$ 
proof
  assume coupling: supported-pairwise-coupling-bad s
  from supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad
    [OF coupling]
  obtain out where out:
     $out \in \text{set-dist } (\text{execute verify-monad } s)$ 
  and side: supported-pairwise-side-bad s out
  by blast
  show False
  using no-side[OF out] side by contradiction
qed

```

lemma *no-supported-pairwise-side-bad-on-support-imp-no-pairwise-side-bad:*

```

assumes no-side:
   $\bigwedge out. out \in \text{set-dist } (\text{execute verify-monad } s) \implies$ 

```

$\neg$ *supported-pairwise-side-bad s out*
shows
 $\neg$ *supported-hash-output-collision-possible s*
 $\neg$ *supported-pairwise-coupling-bad s*
proof –
show $\neg$ *supported-hash-output-collision-possible s*
by (rule
no-supported-pairwise-side-bad-on-support-imp-no-hash-collision-possible
[OF no-side])
show $\neg$ *supported-pairwise-coupling-bad s*
by (rule
no-supported-pairwise-side-bad-on-support-imp-no-coupling-bad
[OF no-side])
qed

lemma *composition-fri-supported-root-candidate-unique-if-no-pairwise-merkle-bad*:
assumes *no-bad*:
 $\neg$ *composition-fri-supported-root-pairwise-merkle-bad s fr f-fri-roots*
f-final as dg composition-fri-roots
shows
 $\exists$ *composition-table*.
composition-fri-supported-root-composition-table-candidates s fr
f-fri-roots f-final as dg composition-fri-roots $\subseteq$
{composition-table}
proof (rule *ccontr*)
assume *not-unique*:
 $\neg$ ($\exists$ *composition-table*.
composition-fri-supported-root-composition-table-candidates s fr
f-fri-roots f-final as dg composition-fri-roots $\subseteq$
{composition-table})
have *composition-fri-supported-root-pairwise-merkle-bad s fr f-fri-roots*
f-final as dg composition-fri-roots
using
composition-fri-supported-root-composition-table-candidates-not-singleton-imp-pairwise-merkle-bad
[OF not-unique]
unfolding *composition-fri-supported-root-pairwise-merkle-bad-def*
by *blast*
then show *False*
using *no-bad* **by** *contradiction*
qed

lemma *composition-fri-supported-root-candidate-unique-if-no-global-pairwise-merkle-bad*:
assumes *no-bad*: $\neg$ *composition-fri-supported-pairwise-merkle-bad s*
shows
 $\exists$ *composition-table*.
composition-fri-supported-root-composition-table-candidates s fr
f-fri-roots f-final as dg composition-fri-roots $\subseteq$
{composition-table}
proof (rule

composition-fri-supported-root-candidate-unique-if-no-pairwise-merkle-bad)
show
 $\neg$ *composition-fri-supported-root-pairwise-merkle-bad* *s* *fr* *f*-*fri*-*roots*
f-*final* *as* *dg* *composition-fri-roots*
using *no-bad*
unfolding *composition-fri-supported-pairwise-merkle-bad-def* **by** *blast*
qed

definition *composition-fri-supported-root-union-bad-sets*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list set}$

where

composition-fri-supported-root-union-bad-sets *s* *bad-sets* *fr* *f*-*fri*-*roots*
f-*final* *as* *dg* *composition-fri-roots* =
 $\{ \text{comp-bs} \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1)) .$
 $\exists \text{ composition-table} \in$
 $\text{composition-fri-supported-root-composition-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots.}$
 $\text{comp-bs} \in \text{bad-sets dg composition-table} \}$

lemma *composition-fri-supported-root-union-bad-sets-subset:*

composition-fri-supported-root-union-bad-sets *s* *bad-sets* *fr* *f*-*fri*-*roots*
f-*final* *as* *dg* *composition-fri-roots* $\subseteq$
 $\text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$

unfolding *composition-fri-supported-root-union-bad-sets-def* **by** *auto*

lemma *composition-fri-challenge-set-hit-imp-supported-root-union-list-set-hit:*

assumes *outcome*: *out* $\in$ *set-dist* (*execute verify-monad* *s*)

and *hit*: *composition-fri-challenge-set-hit* *s* *bad-sets* *out*

shows

composition-fri-root-list-set-hit *s*
 $(\text{composition-fri-supported-root-union-bad-sets } s \text{ bad-sets}) \text{ out}$

proof –

from *hit* **obtain** *trace-table* *composition-table* *as* *query-idxs* *trace-bs* *dg* *comp-bs*

where *bound*:

$\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as}$
 query-idxs

and *challenges*: $\text{accepted-fri-challenges } s \text{ out trace-bs dg comp-bs}$

and *comp-bs-bad*: $\text{comp-bs} \in \text{bad-sets dg composition-table}$

unfolding *composition-fri-challenge-set-hit-def* **by** *blast*

from *bound* **obtain** *result-b* *final-state-b* *fr* *f*-*fri*-*roots* *f*-*final* *dg-b*

composition-fri-roots *final* *rest* **where**

header-bound:

$\text{verifier-header-transcript } s \text{ fr f-fri-roots f-final as dg-b}$
 $\text{composition-fri-roots final rest}$

unfolding *accepted-with-bound-tables-def* **by** *blast*

from *challenges*[*unfolded* *accepted-fri-challenges-def*]

```

obtain result-c final-state-c fr-c f-fl-c f-final-c as-c fl-c final-c
  query-state-c where challenge-body:
out = Some (result-c, final-state-c) ∧
verifier-header-transcript s fr-c (map snd f-fl-c) f-final-c as-c dg
  (map snd fl-c) final-c (PTranscript query-state-c) ∧
PState query-state-c =
  verifier-header-state s fr-c (map snd f-fl-c) f-final-c as-c dg
  (map snd fl-c) final-c ∧
Some (result-c, final-state-c) ∈
  set-dist (execute
    (ntimes
      (verifier-query-round-program fr-c f-fl-c f-final-c as-c fl-c
        final-c)
      rounds)
    query-state-c) ∧
PQueryCounter query-state-c = PQueryCounter s ∧
trace-bs = map fst f-fl-c ∧
comp-bs = map fst fl-c ∧
(∀ i < length f-fl-c.
  fmlookup (HashMap query-state-c)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr-c)
        (take (Suc i) (map snd f-fl-c))))) =
    Some (fst (f-fl-c ! i))) ∧
(∀ i < length fl-c.
  fmlookup (HashMap query-state-c)
    (CompositionFriChallenge (PCompositionFriCounter s + i)
      (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr-c)
                (map snd f-fl-c))
              f-final-c)
            as-c)
          dg)
        (take (Suc i) (map snd fl-c))))) =
    Some (fst (fl-c ! i)))
by (elim exE)
have out-challenges: out = Some (result-c, final-state-c)
using challenge-body by simp
have header-challenges:
  verifier-header-transcript s fr-c (map snd f-fl-c) f-final-c as-c dg
  (map snd fl-c) final-c (PTranscript query-state-c)
using challenge-body by simp
have dg-eq: dg-b = dg
using verifier-header-transcript-unique[OF header-challenges header-bound]
by simp
have header:

```

```

    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
    using header-bound dg-eq by simp
  have candidate:
    composition-table ∈
      composition-fri-supported-root-composition-table-candidates s fr
      f-fri-roots f-final as dg composition-fri-roots
    unfolding composition-fri-supported-root-composition-table-candidates-def
    using outcome bound challenges header by blast
  have comp-bs-space:
    comp-bs ∈ fri-challenge-space (ceil-log (to-nat dg + 1))
    by (rule accepted-fri-challenges-composition-space[OF challenges])
  have comp-bs-union:
    comp-bs ∈
      composition-fri-supported-root-union-bad-sets s bad-sets fr
      f-fri-roots f-final as dg composition-fri-roots
    unfolding composition-fri-supported-root-union-bad-sets-def
    using comp-bs-space candidate comp-bs-bad by blast
  show ?thesis
    unfolding composition-fri-root-list-set-hit-def
    using challenges header comp-bs-union by blast
qed

lemma wp-composition-fri-challenge-set-hit-bound-via-supported-root-union:
  fixes C :: prob
  assumes future: composition-fri-future-fresh s
  and union-bound:
     $\bigwedge$ fr f-fri-roots f-final as dg composition-fri-roots.
      nnreal
      (card
        (composition-fri-supported-root-union-bad-sets s bad-sets fr
          f-fri-roots f-final as dg composition-fri-roots)) /
      nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C
  shows
    wp-event verify-monad (composition-fri-challenge-set-hit s bad-sets) s ≤
      C
proof -
  have root-bound:
    wp-event verify-monad
      (composition-fri-root-list-set-hit s
        (composition-fri-supported-root-union-bad-sets s bad-sets)) s ≤ C
    by (rule wp-verify-monad-composition-fri-root-list-set-bound[OF future])
      (use composition-fri-supported-root-union-bad-sets-subset union-bound in
        simp-all)
  have wp-event verify-monad (composition-fri-challenge-set-hit s bad-sets) s ≤
    wp-event verify-monad
      (composition-fri-root-list-set-hit s
        (composition-fri-supported-root-union-bad-sets s bad-sets)) s
    by (rule wp-event-mono-on-support)

```

```

      (rule composition-fri-challenge-set-hit-imp-supported-root-union-list-set-hit)
    also have ... ≤ C
      by (rule root-bound)
    finally show ?thesis .
  qed

lemma composition-fri-supported-root-union-bad-sets-fraction-bound-if-unique-candidate:
  fixes C :: prob
  assumes unique:
    ∃ composition-table.
      composition-fri-supported-root-composition-table-candidates s fr
        f-fri-roots f-final as dg composition-fri-roots ⊆
          {composition-table}
  and subset:
    ∧ composition-table. bad-sets dg composition-table ⊆
      fri-challenge-space (ceil-log (to-nat dg + 1))
  and bound:
    ∧ composition-table.
      nnreal (card (bad-sets dg composition-table)) /
        nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C
  shows
    nnreal
      (card
        (composition-fri-supported-root-union-bad-sets s bad-sets fr
          f-fri-roots f-final as dg composition-fri-roots)) /
        nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C
  proof -
    from unique obtain composition-table where candidates:
      composition-fri-supported-root-composition-table-candidates s fr
        f-fri-roots f-final as dg composition-fri-roots ⊆
          {composition-table}
    by blast
    have union-subset:
      composition-fri-supported-root-union-bad-sets s bad-sets fr f-fri-roots
        f-final as dg composition-fri-roots ⊆
          bad-sets dg composition-table
    unfolding composition-fri-supported-root-union-bad-sets-def
    using candidates by auto
    have finite-bad: finite (bad-sets dg composition-table)
      by (rule finite-subset[OF subset finite-fri-challenge-space])
    have card-le:
      card
        (composition-fri-supported-root-union-bad-sets s bad-sets fr
          f-fri-roots f-final as dg composition-fri-roots) ≤
        card (bad-sets dg composition-table)
      by (rule card-mono[OF finite-bad union-subset])
    have nnreal
      (card
        (composition-fri-supported-root-union-bad-sets s bad-sets fr

```

$f\text{-fri-roots } f\text{-final as dg composition-fri-roots)) /$
 $\text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq$
 $\text{nnreal } (\text{card } (\text{bad-sets } dg \text{ composition-table})) /$
 $\text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (\text{to-nat } dg + 1))$
by (rule *nnreal-nat-divide-right-mono*[*OF card-le*])
also have $\dots \leq C$
by (rule *bound*)
finally show ?thesis .
qed

definition *composition-fri-supported-root-partial-union-bad-sets*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list set}$

where

$\text{composition-fri-supported-root-partial-union-bad-sets } s \text{ bad-sets fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots} =$
 $\{ \text{comp-bs} \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1)) .$
 $\exists \text{ composition-table} \in$
 $\text{composition-fri-supported-root-partial-composition-table-candidates}$
 $s \text{ fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots} .$
 $\text{comp-bs} \in \text{bad-sets } dg \text{ composition-table} \}$

lemma *composition-fri-supported-root-partial-union-bad-sets-subset:*

$\text{composition-fri-supported-root-partial-union-bad-sets } s \text{ bad-sets fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots} \subseteq$
 $\text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$

unfolding *composition-fri-supported-root-partial-union-bad-sets-def* **by** *auto*

lemma *composition-fri-supported-root-partial-union-bad-sets-fraction-bound-if-unique-candidate:*

fixes $C :: \text{prob}$

assumes *unique:*

$\exists \text{ composition-table} .$
 $\text{composition-fri-supported-root-partial-composition-table-candidates } s$
 $\text{fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots} \subseteq$
 $\{ \text{composition-table} \}$

and *subset:*

$\bigwedge \text{composition-table} . \text{bad-sets } dg \text{ composition-table} \subseteq$
 $\text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$

and *bound:*

$\bigwedge \text{composition-table} .$
 $\text{nnreal } (\text{card } (\text{bad-sets } dg \text{ composition-table})) /$
 $\text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq C$

shows

nnreal
 $(\text{card}$
 $(\text{composition-fri-supported-root-partial-union-bad-sets } s \text{ bad-sets fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots})) /$

$nnreal (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq C$
proof –
from *unique* **obtain** *composition-table* **where** *candidates:*
composition-fri-supported-root-partial-composition-table-candidates s fr
f-fri-roots f-final as dg composition-fri-roots $\subseteq$
{composition-table}
by *blast*
have *union-subset:*
composition-fri-supported-root-partial-union-bad-sets s bad-sets fr
f-fri-roots f-final as dg composition-fri-roots $\subseteq$
bad-sets dg composition-table
unfolding *composition-fri-supported-root-partial-union-bad-sets-def*
using *candidates* **by** *auto*
have *finite-bad:* *finite (bad-sets dg composition-table)*
by *(rule finite-subset[OF subset finite-fri-challenge-space])*
have *card-le:*
card
(composition-fri-supported-root-partial-union-bad-sets s bad-sets fr
f-fri-roots f-final as dg composition-fri-roots) $\leq$
card (bad-sets dg composition-table)
by *(rule card-mono[OF finite-bad union-subset])*
have *nnreal*
(card
(composition-fri-supported-root-partial-union-bad-sets s bad-sets fr
f-fri-roots f-final as dg composition-fri-roots)) /
nnreal (CARD('f) $\wedge$ ceil-log (to-nat dg + 1)) $\leq$
nnreal (card (bad-sets dg composition-table)) /
nnreal (CARD('f) $\wedge$ ceil-log (to-nat dg + 1))
by *(rule nnreal-nat-divide-right-mono[OF card-le])*
also have $\dots \leq C$
by *(rule bound)*
finally show *?thesis* .
qed

lemma *composition-fri-challenge-set-hit-imp-partial-union-or-collision:*
assumes *outcome:* *out $\in$ set-dist (execute verify-monad s)*
and *hit:* *composition-fri-challenge-set-hit s bad-sets out*
shows
composition-fri-root-list-set-hit s
(composition-fri-supported-root-partial-union-bad-sets s bad-sets) out $\vee$
hash-map-output-collision-bad s out
proof –
from *hit* **obtain** *trace-table composition-table as query-idxs trace-bs dg comp-bs*
where *bound:*
accepted-with-bound-tables s out trace-table composition-table as
query-idxs
and *challenges:* *accepted-fri-challenges s out trace-bs dg comp-bs*
and *trace-low:* *trace-table-low-degree trace-table*
and *not-low:*

```

    ¬ composition-table-low-degree maxDegree composition-table
  and comp-bs-bad: comp-bs ∈ bad-sets dg composition-table
  unfolding composition-fri-challenge-set-hit-def by blast
from accepted-with-bound-tables-imp-accepted[OF bound]
obtain result final-state where out-eq: out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
show ?thesis
proof (cases hash-map-output-collision final-state)
  case True
  then have hash-map-output-collision-bad s out
    unfolding hash-map-output-collision-bad-def accepted-def out-eq by simp
  then show ?thesis by simp
next
case clean: False
from challenges obtain result-c final-state-c fr-c f-fl-c f-final-c as-c
  fl-c final-c query-state-c where
  out-c: out = Some (result-c, final-state-c)
  and
  header-challenges:
    verifier-header-transcript s fr-c (map snd f-fl-c) f-final-c as-c dg
    (map snd fl-c) final-c (PTranscript query-state-c)
  and query-state-c:
    PState query-state-c =
      verifier-header-state s fr-c (map snd f-fl-c) f-final-c as-c dg
      (map snd fl-c) final-c
  and query-out-c:
    Some (result-c, final-state-c) ∈
      set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr-c f-fl-c f-final-c as-c fl-c
            final-c) rounds)
        query-state-c)
  and query-count-c:
    PQueryCounter query-state-c = PQueryCounter s
  and trace-eq-c: trace-bs = map fst f-fl-c
  and comp-eq-c: comp-bs = map fst fl-c
  and trace-lookup-c:
    ∧ i. i < length f-fl-c ⇒
      fmlookup (HashMap query-state-c)
        (TraceFriChallenge (PTraceFriCounter s + i)
          (foldl concat (concat (PState s) fr-c)
            (take (Suc i) (map snd f-fl-c)))) =
        Some (fst (f-fl-c ! i))
  and comp-lookup-c:
    ∧ i. i < length fl-c ⇒
      fmlookup (HashMap query-state-c)
        (CompositionFriChallenge (PCompositionFriCounter s + i)
          (foldl concat

```

```

      (concat
        (foldl concat
          (concat
            (foldl concat (concat (PState s) fr-c)
              (map snd f-fl-c))
            f-final-c)
          as-c)
        dg)
      (take (Suc i) (map snd fl-c)))) =
    Some (fst (fl-c ! i))
  unfolding accepted-fri-challenges-def by blast
from bound obtain fr-b f-fri-roots-b f-final-b dg-b
  composition-fri-roots-b final-b rest-b where
  header-bound:
    verifier-header-transcript s fr-b f-fri-roots-b f-final-b as dg-b
    composition-fri-roots-b final-b rest-b
  and comp-nonempty:
    composition-fri-roots-b  $\neq$  []
  and comp-bind:
    merkle-root-binds-table (hd composition-fri-roots-b) composition-table
    final-state
  unfolding accepted-with-bound-tables-def out-eq by blast
have dg-eq: dg-b = dg
  using verifier-header-transcript-unique
    [OF header-bound header-challenges]
  by simp
have header:
  verifier-header-transcript s fr-b f-fri-roots-b f-final-b as dg
  composition-fri-roots-b final-b rest-b
  using header-bound dg-eq by simp
have outcome-some:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
  using outcome out-eq by simp
have bound-some:
  accepted-with-bound-tables s (Some (result, final-state)) trace-table
  composition-table as query-idxs
  using bound out-eq by simp
have challenges-some:
  accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
  comp-bs
  using challenges out-eq by simp
have state:
  composition-fri-supported-root-composition-table-candidate-state s
  fr-b f-fri-roots-b f-final-b as dg composition-fri-roots-b
  composition-table final-state
  unfolding composition-fri-supported-root-composition-table-candidate-state-def
  using outcome-some bound-some challenges-some header comp-nonempty
    comp-bind
  by blast

```

```

have candidate:
  composition-table ∈
    composition-fri-supported-root-partial-composition-table-candidates s
    fr-b f-fri-roots-b f-final-b as dg composition-fri-roots-b
using
  composition-fri-supported-root-composition-table-candidate-state-imp-partial-candidate-state
    [OF state clean]
  composition-fri-supported-root-partial-composition-table-candidates-iff-state
by blast
have comp-bs-space:
  comp-bs ∈ fri-challenge-space (ceil-log (to-nat dg + 1))
by (rule accepted-fri-challenges-composition-space[OF challenges])
have comp-bs-union:
  comp-bs ∈
    composition-fri-supported-root-partial-union-bad-sets s bad-sets
    fr-b f-fri-roots-b f-final-b as dg composition-fri-roots-b
unfolding composition-fri-supported-root-partial-union-bad-sets-def
using comp-bs-space candidate comp-bs-bad by blast
show ?thesis
unfolding composition-fri-root-list-set-hit-def
using challenges header comp-bs-union by blast
qed
qed

lemma wp-composition-fri-challenge-set-hit-bound-via-partial-union-or-collision:
  fixes C H :: prob
  assumes partial-bound:
    wp-event verify-monad
      (composition-fri-root-list-set-hit s
        (composition-fri-supported-root-partial-union-bad-sets s bad-sets))
      s ≤ C
  and collision-bound:
    wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H
  shows
    wp-event verify-monad (composition-fri-challenge-set-hit s bad-sets) s ≤
      C + H
proof –
  have mono:
    wp-event verify-monad (composition-fri-challenge-set-hit s bad-sets) s ≤
      wp-event verify-monad
        (λout.
          composition-fri-root-list-set-hit s
            (composition-fri-supported-root-partial-union-bad-sets s bad-sets)
            out ∨
            hash-map-output-collision-bad s out) s
  by (rule wp-event-mono-on-support)
    (use composition-fri-challenge-set-hit-imp-partial-union-or-collision
      in blast)
  also have ... ≤ C + H

```

proof –
have *wp-event verify-monad*
 ($\lambda out.$
 composition-fri-root-list-set-hit s
 (*composition-fri-supported-root-partial-union-bad-sets s bad-sets*)
 $out \vee$
 hash-map-output-collision-bad s out) $s \leq$
wp-event verify-monad
 (*composition-fri-root-list-set-hit s*
 (*composition-fri-supported-root-partial-union-bad-sets s bad-sets*))
 $s +$
wp-event verify-monad (hash-map-output-collision-bad s) s
by (*rule wp-event-union-bound*)
also have $\dots \leq C + H$
by (*intro add-mono partial-bound collision-bound*)
finally show *?thesis* .
qed
finally show *?thesis* .
qed

lemma *wp-composition-fri-challenge-set-hit-bound-via-supported-partial-union-or-collision:*
fixes $C\ H :: \text{prob}$
assumes *future: composition-fri-future-fresh s*
and *union-bound:*
 $\bigwedge fr\ f\text{-fri-roots}\ f\text{-final}\ as\ dg\ \text{composition-fri-roots.}$
 $nnreal$
 ($card$
 (*composition-fri-supported-root-partial-union-bad-sets s bad-sets*
 $fr\ f\text{-fri-roots}\ f\text{-final}\ as\ dg\ \text{composition-fri-roots}$)) /
 $nnreal\ (CARD('f) \wedge \text{ceil-log}\ (to\text{-nat}\ dg + 1)) \leq C$
and *collision-bound:*
wp-event verify-monad (hash-map-output-collision-bad s) s $\leq H$
shows
wp-event verify-monad (composition-fri-challenge-set-hit s bad-sets) s $\leq$
 $C + H$

proof –
have *partial-bound:*
wp-event verify-monad
 (*composition-fri-root-list-set-hit s*
 (*composition-fri-supported-root-partial-union-bad-sets s bad-sets*))
 $s \leq C$
by (*rule wp-verify-monad-composition-fri-root-list-set-bound[OF future]*)
 (*use composition-fri-supported-root-partial-union-bad-sets-subset*
 union-bound in simp-all)
show *?thesis*
by (*rule*
 wp-composition-fri-challenge-set-hit-bound-via-partial-union-or-collision
 [OF partial-bound collision-bound])
qed

lemma *wp-composition-fri-challenge-set-hit-bound-if-supported-partial-root-candidate-unique-or-collision:*
fixes $H :: \text{prob}$
assumes *future: composition-fri-future-fresh s*
and *unique:*
 $\bigwedge \text{fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots.}$
 $\exists \text{ composition-table.}$
 $\text{composition-fri-supported-root-partial-composition-table-candidates}$
 $s \text{ fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots} \subseteq$
 $\{\text{composition-table}\}$
and *bounded: composition-fri-bad-challenge-sets-bounded bad-sets*
and *collision-bound:*
 $\text{wp-event verify-monad } (\text{hash-map-output-collision-bad } s) \ s \leq H$
shows
 $\text{wp-event verify-monad } (\text{composition-fri-challenge-set-hit } s \text{ bad-sets}) \ s \leq$
 $\text{composition-fri-error} + H$
proof –
have *union-bound:*
 $\bigwedge \text{fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots.}$
 nnreal
 $(\text{card}$
 $(\text{composition-fri-supported-root-partial-union-bad-sets } s \text{ bad-sets}$
 $\text{fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots})) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq$
 $\text{composition-fri-error}$
proof –
fix $\text{fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots}$
show nnreal
 $(\text{card}$
 $(\text{composition-fri-supported-root-partial-union-bad-sets } s \text{ bad-sets}$
 $\text{fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots})) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq$
 $\text{composition-fri-error}$
by (rule
 $\text{composition-fri-supported-root-partial-union-bad-sets-fraction-bound-if-unique-candidate}$
 $[\text{OF unique}]$)
(use *bounded in*
 $\langle \text{auto simp: composition-fri-bad-challenge-sets-bounded-def} \rangle$
qed
show *?thesis*
by (rule
 $\text{wp-composition-fri-challenge-set-hit-bound-via-supported-partial-union-or-collision}$
 $[\text{OF future union-bound collision-bound}]$)
qed

definition *trace-fri-bad-with-tables*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where

trace-fri-bad-with-tables s out $\longleftrightarrow$
accepted-with-fri-tables s out $\wedge$ *trace-fri-bad s out*

definition *composition-fri-bad-with-tables*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-bad-with-tables s out $\longleftrightarrow$
accepted-with-fri-tables s out $\wedge$ *composition-fri-bad s out*

definition *trace-fri-bad-with-partial-openings*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-bad-with-partial-openings s out $\longleftrightarrow$
trace-fri-bad s out $\wedge$
 $(\exists \text{ fr query-idxs } \text{trace-openings}.$
 $\text{accepted-with-partial-trace-openings s out fr query-idxs}$
 $\text{trace-openings})$

definition *composition-fri-bad-with-partial-openings*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-bad-with-partial-openings s out $\longleftrightarrow$
composition-fri-bad s out $\wedge$
 $(\exists \text{ fr query-idxs } \text{trace-openings}.$
 $\text{accepted-with-partial-trace-openings s out fr query-idxs}$
 $\text{trace-openings})$

end

end

theory *Soundness-Reductions-Partial*

imports *Soundness-Reductions-Side*

begin

Partial-opening bad-event definitions and accepted-execution partitions.

context *soundness*

begin

Reachable-state FRI interfaces for the partial-opening proof path. These predicates express the verifier-local FRI low-degree reductions over authenticated partial openings and deliberately avoid complete-table witnesses in their statement.

definition *trace-fri-partial-opening-reduction-assumption*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

trace-fri-partial-opening-reduction-assumption $s \longleftrightarrow$
 $wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}bad\text{-}with\text{-}partial\text{-}openings\ s)\ s \leq$
 $trace\text{-}fri\text{-}error$

definition *composition-fri-partial-opening-reduction-assumption*

$:: (f, a)\ protocol\text{-}channel\text{-}scheme \Rightarrow bool$

where

composition-fri-partial-opening-reduction-assumption $s \longleftrightarrow$
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}bad\text{-}with\text{-}partial\text{-}openings\ s)\ s \leq$
 $composition\text{-}fri\text{-}error$

lemma *trace-fri-bad-with-partial-openings-bound-from-partial-reduction:*

assumes *trace-fri-partial-opening-reduction-assumption* s

shows $wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}bad\text{-}with\text{-}partial\text{-}openings\ s)\ s \leq$
 $trace\text{-}fri\text{-}error$

using *assms* **unfolding** *trace-fri-partial-opening-reduction-assumption-def*
by *simp*

lemma *composition-fri-bad-with-partial-openings-bound-from-partial-reduction:*

assumes *composition-fri-partial-opening-reduction-assumption* s

shows

$wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}bad\text{-}with\text{-}partial\text{-}openings\ s)\ s \leq$
 $composition\text{-}fri\text{-}error$

using *assms*

unfolding *composition-fri-partial-opening-reduction-assumption-def* **by** *simp*

lemma *trace-fri-bad-with-partial-openings-imp-trace-fri-bad:*

assumes *trace-fri-bad-with-partial-openings* $s\ out$

shows *trace-fri-bad* $s\ out$

using *assms* **unfolding** *trace-fri-bad-with-partial-openings-def* **by** *simp*

lemma *composition-fri-bad-with-partial-openings-imp-composition-fri-bad:*

assumes *composition-fri-bad-with-partial-openings* $s\ out$

shows *composition-fri-bad* $s\ out$

using *assms* **unfolding** *composition-fri-bad-with-partial-openings-def* **by** *simp*

lemma *wp-trace-fri-bad-with-partial-openings-le-trace-fri-bad:*

$wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}bad\text{-}with\text{-}partial\text{-}openings\ s)\ s \leq$

$wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}bad\ s)\ s$

by (rule *wp-event-mono*)

(rule *trace-fri-bad-with-partial-openings-imp-trace-fri-bad*)

lemma *wp-composition-fri-bad-with-partial-openings-le-composition-fri-bad:*

$wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}bad\text{-}with\text{-}partial\text{-}openings\ s)\ s \leq$

$wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}bad\ s)\ s$

by (rule *wp-event-mono*)

(rule *composition-fri-bad-with-partial-openings-imp-composition-fri-bad*)

lemma *trace-fri-bad-with-tables-imp-partial-openings-on-support:*
assumes *support: out ∈ set-dist (execute verify-monad s)*
and *bad: trace-fri-bad-with-tables s out*
shows *trace-fri-bad-with-partial-openings s out*
proof –
have *tables: accepted-with-fri-tables s out*
using *bad* **unfolding** *trace-fri-bad-with-tables-def* **by** *simp*
have *accepted: accepted out*
by *(rule accepted-with-fri-tables-imp-accepted[OF tables])*
then obtain *result final-state* **where** *out-eq:*
out = Some (result, final-state)
unfolding *accepted-def* **by** *(cases out) auto*
from *verify-monad-accepted-with-partial-trace-openings*
[OF support[unfolded out-eq]]
obtain *fr query-idxs trace-openings* **where** *partial:*
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr query-idxs trace-openings
by *blast*
have *trace-bad: trace-fri-bad s out*
using *bad* **unfolding** *trace-fri-bad-with-tables-def* **by** *simp*
show *?thesis*
unfolding *trace-fri-bad-with-partial-openings-def out-eq*
using *trace-bad[unfolded out-eq] partial* **by** *blast*
qed

lemma *composition-fri-bad-with-tables-imp-partial-openings-on-support:*
assumes *support: out ∈ set-dist (execute verify-monad s)*
and *bad: composition-fri-bad-with-tables s out*
shows *composition-fri-bad-with-partial-openings s out*
proof –
have *tables: accepted-with-fri-tables s out*
using *bad* **unfolding** *composition-fri-bad-with-tables-def* **by** *simp*
have *accepted: accepted out*
by *(rule accepted-with-fri-tables-imp-accepted[OF tables])*
then obtain *result final-state* **where** *out-eq:*
out = Some (result, final-state)
unfolding *accepted-def* **by** *(cases out) auto*
from *verify-monad-accepted-with-partial-trace-openings*
[OF support[unfolded out-eq]]
obtain *fr query-idxs trace-openings* **where** *partial:*
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr query-idxs trace-openings
by *blast*
have *composition-bad: composition-fri-bad s out*
using *bad* **unfolding** *composition-fri-bad-with-tables-def* **by** *simp*
show *?thesis*
unfolding *composition-fri-bad-with-partial-openings-def out-eq*
using *composition-bad[unfolded out-eq] partial* **by** *blast*

qed

lemma *wp-trace-fri-bad-with-tables-le-partial-openings*:
wp-event verify-monad (trace-fri-bad-with-tables s) s ≤
wp-event verify-monad (trace-fri-bad-with-partial-openings s) s
by (rule wp-event-mono-on-support)
(rule trace-fri-bad-with-tables-imp-partial-openings-on-support)

lemma *wp-composition-fri-bad-with-tables-le-partial-openings*:
wp-event verify-monad (composition-fri-bad-with-tables s) s ≤
wp-event verify-monad (composition-fri-bad-with-partial-openings s) s
by (rule wp-event-mono-on-support)
(rule composition-fri-bad-with-tables-imp-partial-openings-on-support)

definition *soundness-bad-event-table-aware*
:: ('f, 'a) protocol-channel-scheme ⇒
(unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
soundness-bad-event-table-aware s out ⟷
composition-bad s out ∨
trace-fri-bad-with-tables s out ∨
composition-fri-bad-with-tables s out ∨
query-bad s out

definition *soundness-bad-event-partial-opening*
:: ('f, 'a) protocol-channel-scheme ⇒
(unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
soundness-bad-event-partial-opening s out ⟷
composition-bad s out ∨
trace-fri-bad-with-partial-openings s out ∨
composition-fri-bad-with-partial-openings s out ∨
query-bad s out

definition *soundness-bad-event-table-aware-with-merkle*
:: ('f, 'a) protocol-channel-scheme ⇒
(unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
soundness-bad-event-table-aware-with-merkle s out ⟷
fri-merkle-binding-bad s out ∨
soundness-bad-event-table-aware s out

definition *soundness-bad-event-partial-opening-with-merkle*
:: ('f, 'a) protocol-channel-scheme ⇒
(unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
soundness-bad-event-partial-opening-with-merkle s out ⟷
fri-merkle-binding-bad s out ∨
soundness-bad-event-partial-opening s out

definition *soundness-bad-event-partial-opening-with-initial-merkle*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{soundness-bad-event-partial-opening-with-initial-merkle } s \text{ out} \longleftrightarrow$
 $\text{initial-merkle-binding-bad } s \text{ out} \vee$
 $\text{soundness-bad-event-partial-opening } s \text{ out}$

definition *soundness-bad-event-partial-opening-with-partial-merkle*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{soundness-bad-event-partial-opening-with-partial-merkle } s \text{ out} \longleftrightarrow$
 $\text{partial-merkle-inconsistency-bad } s \text{ out} \vee$
 $\text{soundness-bad-event-partial-opening } s \text{ out}$

definition *composition-bad-with-partial-candidates*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-bad-with-partial-candidates } s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table } f.$
 $\text{ accepted-with-partial-initial-openings } s \text{ out fr } f\text{-fri-roots } f\text{-final}$
 $\text{ as dg composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{ degree } f < \text{ clength} \wedge$
 $\text{ trace-table} = \text{ map } (\text{poly } f) \text{ eval-domain} \wedge$
 $\text{ violated-constraints } f \neq \{\}$
 $\text{ composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{ all-queries-consistent trace-table composition-table as})$

definition *composition-degree-bad-with-partial-candidates*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-degree-bad-with-partial-candidates } s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table } f.$
 $\text{ accepted-with-partial-initial-openings } s \text{ out fr } f\text{-fri-roots } f\text{-final}$
 $\text{ as dg composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$

partial-composition-table-candidate composition-table
composition-openings $\wedge$
degree f < clength $\wedge$
trace-table = map (poly f) eval-domain $\wedge$
violated-constraints f $\neq$ {} $\wedge$
composition-table-low-degree maxDegree composition-table $\wedge$
all-queries-consistent trace-table composition-table as $\wedge$
 $\neg$ *common-denominator-degree-bounds f as*)

definition *composition-randomization-bad-with-partial-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-randomization-bad-with-partial-candidates s out $\longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table } f.$
 $\text{ accepted-with-partial-initial-openings } s \text{ out fr } f\text{-fri-roots } f\text{-final}$
 $\text{ as dg composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings } \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings } \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings } \wedge$
 $\text{ degree } f < \text{clength} \wedge$
 $\text{ trace-table = map (poly } f) \text{ eval-domain } \wedge$
 $\text{ violated-constraints } f \neq \{\}$ $\wedge$
 $\text{ composition-table-low-degree maxDegree composition-table } \wedge$
 $\text{ all-queries-consistent trace-table composition-table as } \wedge$
 $\text{ common-denominator-degree-bounds } f \text{ as } \wedge$
 $\text{ random-combination-common-denominator-hides-violations } f \text{ as})$

definition *trace-fri-bad-with-partial-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-bad-with-partial-candidates s out $\longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table.}$
 $\text{ accepted-with-partial-initial-openings } s \text{ out fr } f\text{-fri-roots } f\text{-final}$
 $\text{ as dg composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings } \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings } \wedge$
 $\neg \text{ trace-table-low-degree trace-table})$

definition *composition-fri-bad-with-partial-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-bad-with-partial-candidates s out $\longleftrightarrow$
 $(\exists$ fr f -fri-roots f -final as dg *composition-fri-roots* final
trace-query-idxs *trace-openings* *composition-query-idxs*
composition-openings *trace-table* *composition-table*.
accepted-with-partial-initial-openings s out fr f -fri-roots f -final
as dg *composition-fri-roots* final *trace-query-idxs* *trace-openings*
composition-query-idxs *composition-openings* $\wedge$
partial-trace-table-candidate *trace-table* *trace-openings* $\wedge$
partial-composition-table-candidate *composition-table*
composition-openings $\wedge$
trace-table-low-degree *trace-table* $\wedge$
 $\neg$ *composition-table-low-degree* *maxDegree* *composition-table*)

definition *query-bad-with-partial-candidates*

$:: (f, 'a)$ protocol-channel-scheme $\Rightarrow$
 $(\text{unit list} \times (f, 'a)$ protocol-channel-scheme) option $\Rightarrow$ bool

where

query-bad-with-partial-candidates s out $\longleftrightarrow$
 $(\exists$ fr f -fri-roots f -final as dg *composition-fri-roots* final
trace-query-idxs *trace-openings* *composition-query-idxs*
composition-openings *trace-table* *composition-table*.
accepted-with-partial-initial-openings s out fr f -fri-roots f -final
as dg *composition-fri-roots* final *trace-query-idxs* *trace-openings*
composition-query-idxs *composition-openings* $\wedge$
partial-trace-table-candidate *trace-table* *trace-openings* $\wedge$
partial-composition-table-candidate *composition-table*
composition-openings $\wedge$
trace-table-low-degree *trace-table* $\wedge$
composition-table-low-degree *maxDegree* *composition-table* $\wedge$
 $\neg$ *all-queries-consistent* *trace-table* *composition-table* as)

definition *soundness-bad-event-partial-candidate*

$:: (f, 'a)$ protocol-channel-scheme $\Rightarrow$
 $(\text{unit list} \times (f, 'a)$ protocol-channel-scheme) option $\Rightarrow$ bool

where

soundness-bad-event-partial-candidate s out $\longleftrightarrow$
composition-bad-with-partial-candidates s out $\vee$
trace-fri-bad-with-partial-candidates s out $\vee$
composition-fri-bad-with-partial-candidates s out $\vee$
query-bad-with-partial-candidates s out

definition *soundness-bad-event-partial-candidate-with-partial-merkle*

$:: (f, 'a)$ protocol-channel-scheme $\Rightarrow$
 $(\text{unit list} \times (f, 'a)$ protocol-channel-scheme) option $\Rightarrow$ bool

where

soundness-bad-event-partial-candidate-with-partial-merkle s out $\longleftrightarrow$
partial-merkle-inconsistency-bad s out $\vee$
soundness-bad-event-partial-candidate s out

definition *accepted-with-empty-composition-header-candidates*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow$
 $f \Rightarrow \text{nat list} \Rightarrow f \text{ authenticated-opening list list} \Rightarrow$
 $f \text{ list} \Rightarrow f \text{ list} \Rightarrow \text{bool}$

where

accepted-with-empty-composition-header-candidates s out fr f -fri-roots
 f -final as dg final trace-query-idxs trace-openings trace-table
composition-table $\longleftrightarrow$
accepted out $\wedge$
 $(\exists \text{ rest.}$
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final as } dg \text{ []}$
 $\text{final rest}) \wedge$
accepted-with-partial-trace-openings s out fr trace-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
composition-table = replicate (scale * clength) final

definition *composition-bad-with-empty-composition-header-candidates*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-bad-with-empty-composition-header-candidates s out $\longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as } dg \text{ final trace-query-idxs}$
 $\text{trace-openings trace-table composition-table } f.$
accepted-with-empty-composition-header-candidates s out fr
 $f\text{-fri-roots } f\text{-final as } dg \text{ final trace-query-idxs trace-openings}$
 $\text{trace-table composition-table} \wedge$
 $\text{degree } f < \text{clength} \wedge$
 $\text{trace-table} = \text{map } (\text{poly } f) \text{ eval-domain} \wedge$
 $\text{violated-constraints } f \neq \{\}$ $\wedge$
 $\text{composition-table-low-degree } \text{maxDegree } \text{composition-table} \wedge$
 $\text{all-queries-consistent trace-table composition-table as})$

definition *composition-degree-bad-with-empty-composition-header-candidates*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-degree-bad-with-empty-composition-header-candidates s out $\longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as } dg \text{ final trace-query-idxs}$
 $\text{trace-openings trace-table composition-table } f.$
accepted-with-empty-composition-header-candidates s out fr
 $f\text{-fri-roots } f\text{-final as } dg \text{ final trace-query-idxs trace-openings}$
 $\text{trace-table composition-table} \wedge$
 $\text{degree } f < \text{clength} \wedge$
 $\text{trace-table} = \text{map } (\text{poly } f) \text{ eval-domain} \wedge$
 $\text{violated-constraints } f \neq \{\}$ $\wedge$
 $\text{composition-table-low-degree } \text{maxDegree } \text{composition-table} \wedge$

all-queries-consistent trace-table composition-table as $\wedge$
 $\neg$ *common-denominator-degree-bounds f as*)

definition *composition-randomization-bad-with-empty-composition-header-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-randomization-bad-with-empty-composition-header-candidates s out $\longleftrightarrow$
 $(\exists \text{ fr f-fri-roots f-final as dg final trace-query-idxs}$
 $\text{ trace-openings trace-table composition-table f.}$
 $\text{ accepted-with-empty-composition-header-candidates s out fr}$
 $\text{ f-fri-roots f-final as dg final trace-query-idxs trace-openings}$
 $\text{ trace-table composition-table} \wedge$
 $\text{ degree f} < \text{ clength} \wedge$
 $\text{ trace-table} = \text{ map (poly f) eval-domain} \wedge$
 $\text{ violated-constraints f} \neq \{\}$ $\wedge$
 $\text{ composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{ all-queries-consistent trace-table composition-table as} \wedge$
 $\text{ common-denominator-degree-bounds f as} \wedge$
 $\text{ random-combination-common-denominator-hides-violations f as})$

definition *composition-alpha-bad-set-hit-with-empty-composition-header-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-alpha-bad-set-hit-with-empty-composition-header-candidates s out $\longleftrightarrow$
 $(\exists \text{ fr f-fri-roots f-final as dg final trace-query-idxs}$
 $\text{ trace-openings trace-table composition-table.}$
 $\text{ accepted-with-empty-composition-header-candidates s out fr}$
 $\text{ f-fri-roots f-final as dg final trace-query-idxs trace-openings}$
 $\text{ trace-table composition-table} \wedge$
 $\text{ as} \in \text{ composition-trace-bad-alpha-space trace-table})$

definition *composition-alpha-bad-set-hit-with-partial-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-alpha-bad-set-hit-with-partial-candidates s out $\longleftrightarrow$
 $(\exists \text{ fr f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table.}$
 $\text{ accepted-with-partial-initial-openings s out fr f-fri-roots f-final}$
 $\text{ as dg composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ as} \in \text{ composition-trace-bad-alpha-space trace-table})$

definition *partial-trace-opening-alpha-union-bad-sets*

$:: 'f \text{ authenticated-opening list list} \Rightarrow 'f \text{ list set}$

where

partial-trace-opening-alpha-union-bad-sets trace-openings =
 $(\bigcup \text{trace-table} \in \text{partial-trace-table-candidates } \text{trace-openings}.$
composition-trace-bad-alpha-space trace-table)

definition *composition-alpha-partial-opening-union-hit-with-empty-header*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-alpha-partial-opening-union-hit-with-empty-header s out $\longleftrightarrow$
 $(\exists \text{fr } f\text{-fri-roots } f\text{-final as dg final trace-query-idxs}$
 $\text{trace-openings trace-table composition-table.}$
 $\text{accepted-with-empty-composition-header-candidates s out fr}$
 $f\text{-fri-roots } f\text{-final as dg final trace-query-idxs trace-openings}$
 $\text{trace-table composition-table} \wedge$
 $\text{as} \in \text{partial-trace-opening-alpha-union-bad-sets trace-openings})$

lemma *partial-trace-opening-alpha-union-bad-sets-subset-alpha-space:*

partial-trace-opening-alpha-union-bad-sets trace-openings $\subseteq$
alpha-space

unfolding *partial-trace-opening-alpha-union-bad-sets-def*

using *composition-trace-bad-alpha-space-subset-alpha-space* **by** *auto*

lemma *partial-trace-opening-alpha-union-bad-sets-finite:*

finite (partial-trace-opening-alpha-union-bad-sets trace-openings)

by *(rule finite-subset*

$[OF \text{ partial-trace-opening-alpha-union-bad-sets-subset-alpha-space}$
 $\text{finite-alpha-space}]$)

lemma *partial-trace-opening-alpha-union-bad-sets-fraction-bound-if-unique-candidate:*

assumes *unique:*

$\exists \text{ trace-table.}$

$\text{partial-trace-table-candidates trace-openings} \subseteq \{\text{trace-table}\}$

shows

nnreal

$(\text{card } (\text{partial-trace-opening-alpha-union-bad-sets trace-openings})) /$

$\text{nnreal } (\text{card } \text{alpha-space}) \leq \text{composition-error-bound}$

proof –

from *unique* **obtain** *trace-table* **where** *candidates:*

$\text{partial-trace-table-candidates trace-openings} \subseteq \{\text{trace-table}\}$

by *blast*

have *union-subset:*

$\text{partial-trace-opening-alpha-union-bad-sets trace-openings} \subseteq$

$\text{composition-trace-bad-alpha-space trace-table}$

unfolding *partial-trace-opening-alpha-union-bad-sets-def*

using *candidates* **by** *auto*

have *finite-bad: finite (composition-trace-bad-alpha-space trace-table)*

by *(rule finite-subset*

$[OF \text{ composition-trace-bad-alpha-space-subset-alpha-space}$


```

      finite-alpha-space])
have card (partial-trace-opening-alpha-union-bad-sets trace-openings) ≤
  card (composition-trace-bad-alpha-space trace-table)
  by (rule card-mono[OF finite-bad union-subset])
then have
  nnreal
    (card (partial-trace-opening-alpha-union-bad-sets trace-openings)) /
    nnreal (card alpha-space) ≤
    nnreal (card (composition-trace-bad-alpha-space trace-table)) /
    nnreal (card alpha-space)
  by (rule nnreal-nat-divide-right-mono)
also have ... ≤ composition-error-bound
  by (rule composition-trace-bad-alpha-space-fraction-bound-alpha-space)
finally show ?thesis .
qed

lemma composition-alpha-bad-set-hit-with-empty-header-imp-partial-opening-union-hit:
  assumes
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates
    s out
  shows
    composition-alpha-partial-opening-union-hit-with-empty-header s out
proof -
  obtain fr f-fri-roots f-final as dg final trace-query-idxs
    trace-openings trace-table composition-table where partial:
    accepted-with-empty-composition-header-candidates s out fr
    f-fri-roots f-final as dg final trace-query-idxs trace-openings
    trace-table composition-table
  and as-bad: as ∈ composition-trace-bad-alpha-space trace-table
  using assms
  unfolding
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates-def
  by blast
have candidate:
  trace-table ∈ partial-trace-table-candidates trace-openings
  using partial
  unfolding accepted-with-empty-composition-header-candidates-def
    partial-trace-table-candidates-def
  by simp
have
  as ∈ partial-trace-opening-alpha-union-bad-sets trace-openings
  unfolding partial-trace-opening-alpha-union-bad-sets-def
  using candidate as-bad by blast
then show ?thesis
  unfolding composition-alpha-partial-opening-union-hit-with-empty-header-def
  using partial by blast
qed

lemma wp-composition-alpha-bad-set-hit-with-empty-header-le-partial-opening-union-hit:

```

```

wp-event verify-monad
  (composition-alpha-bad-set-hit-with-empty-composition-header-candidates s)
  s ≤
wp-event verify-monad
  (composition-alpha-partial-opening-union-hit-with-empty-header s) s
by (rule wp-event-mono)
(rule
  composition-alpha-bad-set-hit-with-empty-header-imp-partial-opening-union-hit)

```

lemma *composition-alpha-partial-opening-union-hit-with-empty-header-imp-alpha-header-partial-union-on-support:*
assumes *support:* $out \in \text{set-dist}$ (*execute verify-monad s*)

and *hit:*

composition-alpha-partial-opening-union-hit-with-empty-header s out

shows

alpha-header-list-set-hit s

(*alpha-header-supported-partial-union-bad-sets s*
composition-trace-bad-alpha-space) *out*

proof —

obtain *fr f-fri-roots f-final as dg final trace-query-idxs*

trace-openings trace-table composition-table rest **where** *partial:*

accepted-with-empty-composition-header-candidates s out fr

f-fri-roots f-final as dg final trace-query-idxs trace-openings

trace-table composition-table

and *header:*

verifier-header-transcript s fr f-fri-roots f-final as dg []

final rest

and *trace-partial:*

accepted-with-partial-trace-openings s out fr trace-query-idxs

trace-openings

and *as-bad:*

as $\in$ *partial-trace-opening-alpha-union-bad-sets trace-openings*

using *hit*

unfolding *composition-alpha-partial-opening-union-hit-with-empty-header-def*

accepted-with-empty-composition-header-candidates-def

by *blast*

from *trace-partial* **obtain** *result final-state* **where** *out-eq:*

out = *Some (result, final-state)*

unfolding *accepted-with-partial-trace-openings-def* **by** *blast*

from *verify-monad-accepted-transcript-shape[OF support[unfolded out-eq]]*

obtain *as' query-idxs* **where** *shape':*

accepted-transcript-shape s (Some (result, final-state)) as' query-idxs

by *blast*

from *shape'* **obtain** *result' final-state' fr' f-fri-roots' f-final' dg'*

composition-fri-roots' final' rest' where *out-shape:*

Some (result, final-state) = Some (result', final-state')

and *header':*

verifier-header-transcript s fr' f-fri-roots' f-final' as' dg'

composition-fri-roots' final' rest'

by (*elim accepted-transcript-shape-header-query-extraction*)

```

have as'-eq: as' = as
  using verifier-header-transcript-unique[OF header' header] by simp
have shape:
  accepted-transcript-shape s out as query-idxs
  using shape' as'-eq out-eq by simp
from as-bad obtain trace-table' where candidate':
  trace-table' ∈ partial-trace-table-candidates trace-openings
  and as-bad':
    as ∈ composition-trace-bad-alpha-space trace-table'
  unfolding partial-trace-opening-alpha-union-bad-sets-def by blast
have candidate-header:
  trace-table' ∈
    alpha-header-supported-partial-trace-table-candidates s fr
    f-fri-roots f-final
proof -
  have partial-candidate':
    partial-trace-table-candidate trace-table' trace-openings
    using candidate'
    unfolding partial-trace-table-candidates-def by simp
  show ?thesis
    unfolding alpha-header-supported-partial-trace-table-candidates-def
    using support trace-partial partial-candidate' header by blast
qed
have as-union:
  as ∈
    alpha-header-supported-partial-union-bad-sets s
    composition-trace-bad-alpha-space fr f-fri-roots f-final
  unfolding alpha-header-supported-partial-union-bad-sets-def
  using candidate-header as-bad'
    composition-trace-bad-alpha-space-subset-alpha-space[of trace-table']
  by auto
show ?thesis
  unfolding alpha-header-list-set-hit-def
proof (intro exI conjI)
  show accepted-transcript-shape s out as query-idxs
    by (rule shape)
next
  show out = Some (result, final-state)
    by (rule out-eq)
next
  show
    verifier-header-transcript s fr f-fri-roots f-final as dg []
    final rest
    by (rule header)
next
  show
    as ∈
      alpha-header-supported-partial-union-bad-sets s
      composition-trace-bad-alpha-space fr f-fri-roots f-final

```

by (rule as-union)
qed
qed

lemma *wp-composition-alpha-partial-opening-union-hit-with-empty-header-le-alpha-header-partial-union:*
wp-event verify-monad
(composition-alpha-partial-opening-union-hit-with-empty-header s) s ≤
wp-event verify-monad
(alpha-header-list-set-hit s
(alpha-header-supported-partial-union-bad-sets s
composition-trace-bad-alpha-space)) s
by (rule *wp-event-mono-on-support*)
(rule
composition-alpha-partial-opening-union-hit-with-empty-header-imp-alpha-header-partial-union-on-support)

definition *trace-fri-bad-with-empty-composition-header-candidates*
:: (*f*, *a*) *protocol-channel-scheme* ⇒
(*unit list* × (*f*, *a*) *protocol-channel-scheme*) *option* ⇒ *bool*
where

trace-fri-bad-with-empty-composition-header-candidates s out ⇔
(∃ *fr f-fri-roots f-final as dg final trace-query-idxs*
trace-openings trace-table composition-table.
accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table ∧
¬ *trace-table-low-degree trace-table*)

definition *query-bad-with-empty-composition-header-candidates*
:: (*f*, *a*) *protocol-channel-scheme* ⇒
(*unit list* × (*f*, *a*) *protocol-channel-scheme*) *option* ⇒ *bool*
where

query-bad-with-empty-composition-header-candidates s out ⇔
(∃ *fr f-fri-roots f-final as dg final trace-query-idxs*
trace-openings trace-table composition-table.
accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table ∧
trace-table-low-degree trace-table ∧
composition-table-low-degree maxDegree composition-table ∧
¬ *all-queries-consistent trace-table composition-table as*)

definition *soundness-bad-event-partial-candidate-empty-header*
:: (*f*, *a*) *protocol-channel-scheme* ⇒
(*unit list* × (*f*, *a*) *protocol-channel-scheme*) *option* ⇒ *bool*
where

soundness-bad-event-partial-candidate-empty-header s out ⇔
composition-bad-with-empty-composition-header-candidates s out ∨
trace-fri-bad-with-empty-composition-header-candidates s out ∨
query-bad-with-empty-composition-header-candidates s out

definition *soundness-bad-event-table-aware-with-pairwise-side*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{soundness-bad-event-table-aware-with-pairwise-side } s \text{ out} \longleftrightarrow$
 $\text{supported-pairwise-side-bad } s \text{ out} \vee$
 $\text{soundness-bad-event-table-aware-with-merkle } s \text{ out}$

definition *soundness-bad-event-table-aware-with-output-local-side*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{soundness-bad-event-table-aware-with-output-local-side } s \text{ out} \longleftrightarrow$
 $\text{supported-output-local-side-bad } s \text{ out} \vee$
 $\text{soundness-bad-event-table-aware-with-merkle } s \text{ out}$

lemma *soundness-bad-event-table-aware-imp-with-merkle:*
assumes *soundness-bad-event-table-aware* $s \text{ out}$
shows *soundness-bad-event-table-aware-with-merkle* $s \text{ out}$
using *assms* **unfolding** *soundness-bad-event-table-aware-with-merkle-def* **by** *simp*

lemma *soundness-bad-event-partial-opening-imp-with-merkle:*
assumes *soundness-bad-event-partial-opening* $s \text{ out}$
shows *soundness-bad-event-partial-opening-with-merkle* $s \text{ out}$
using *assms*
unfolding *soundness-bad-event-partial-opening-with-merkle-def* **by** *simp*

lemma *initial-merkle-binding-bad-imp-partial-opening-with-initial-merkle:*
assumes *initial-merkle-binding-bad* $s \text{ out}$
shows *soundness-bad-event-partial-opening-with-initial-merkle* $s \text{ out}$
using *assms*
unfolding *soundness-bad-event-partial-opening-with-initial-merkle-def*
by *simp*

lemma *soundness-bad-event-partial-opening-imp-with-initial-merkle:*
assumes *soundness-bad-event-partial-opening* $s \text{ out}$
shows *soundness-bad-event-partial-opening-with-initial-merkle* $s \text{ out}$
using *assms*
unfolding *soundness-bad-event-partial-opening-with-initial-merkle-def*
by *simp*

lemma *partial-merkle-inconsistency-bad-imp-partial-opening-with-partial-merkle:*
assumes *partial-merkle-inconsistency-bad* $s \text{ out}$
shows *soundness-bad-event-partial-opening-with-partial-merkle* $s \text{ out}$
using *assms*
unfolding *soundness-bad-event-partial-opening-with-partial-merkle-def*
by *simp*

lemma *soundness-bad-event-partial-opening-imp-with-partial-merkle:*
assumes *soundness-bad-event-partial-opening s out*
shows *soundness-bad-event-partial-opening-with-partial-merkle s out*
using *assms*
unfolding *soundness-bad-event-partial-opening-with-partial-merkle-def*
by *simp*

lemma *soundness-bad-event-partial-candidate-imp-with-partial-merkle:*
assumes *soundness-bad-event-partial-candidate s out*
shows *soundness-bad-event-partial-candidate-with-partial-merkle s out*
using *assms*
unfolding *soundness-bad-event-partial-candidate-with-partial-merkle-def*
by *simp*

lemma *partial-merkle-inconsistency-bad-imp-partial-candidate-with-partial-merkle:*
assumes *partial-merkle-inconsistency-bad s out*
shows *soundness-bad-event-partial-candidate-with-partial-merkle s out*
using *assms*
unfolding *soundness-bad-event-partial-candidate-with-partial-merkle-def*
by *simp*

lemma *accepted-with-partial-initial-openings-alpha-length:*
assumes
accepted-with-partial-initial-openings s out fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings
shows *length as = length spec*
proof –
obtain *rest* **where**
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
using *assms*
unfolding *accepted-with-partial-initial-openings-def* **by** *blast*
then show *?thesis*
using *verifier-header-transcript-shapes(3)* **by** *blast*
qed

lemma *composition-bad-with-partial-candidates-split:*
assumes *false-statement: $\neg$ exists-valid-trace*
and *bad: composition-bad-with-partial-candidates s out*
shows
composition-degree-bad-with-partial-candidates s out $\vee$
composition-randomization-bad-with-partial-candidates s out
proof –
obtain *fr f-fri-roots f-final as dg composition-fri-roots final*
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table f
where *partial:*
accepted-with-partial-initial-openings s out fr f-fri-roots f-final

```

    as dg composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings
and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
and deg-f: degree f < clength
and trace-table: trace-table = map (poly f) eval-domain
and violated: violated-constraints f ≠ {}
and composition-low:
    composition-table-low-degree maxDegree composition-table
and all-queries:
    all-queries-consistent trace-table composition-table as
using bad
unfolding composition-bad-with-partial-candidates-def by blast
show ?thesis
proof (cases common-denominator-degree-bounds f as)
  case False
    then have composition-degree-bad-with-partial-candidates s out
      unfolding composition-degree-bad-with-partial-candidates-def
      using partial trace-candidate composition-candidate deg-f trace-table
      violated composition-low all-queries
      by blast
    then show ?thesis by blast
  next
    case True
      note bounds = True
      have len-as: length as = length spec
        by (rule accepted-with-partial-initial-openings-alpha-length
          [OF partial])
      have hides:
        random-combination-common-denominator-hides-violations f as
        by (rule random-combination-common-denominator-hides-from-queries
          [OF len-as false-statement deg-f trace-table composition-low
            all-queries bounds])
      have composition-randomization-bad-with-partial-candidates s out
        unfolding composition-randomization-bad-with-partial-candidates-def
        using partial trace-candidate composition-candidate deg-f trace-table
        violated composition-low all-queries bounds hides
        by blast
      then show ?thesis by blast
qed
qed

lemma composition-degree-bad-with-partial-candidates-false:
  ¬ composition-degree-bad-with-partial-candidates s out
using common-denominator-degree-bounds-from-spec-degree-wellformed
  [OF spec-degree-wellformed-from-spec-query-margin]

```

unfolding *composition-degree-bad-with-partial-candidates-def* **by** *blast*

lemma *composition-randomization-bad-with-partial-candidates-imp-alpha-hit:*
assumes *bad: composition-randomization-bad-with-partial-candidates s out*
shows *composition-alpha-bad-set-hit-with-partial-candidates s out*

proof –

obtain *fr f-fri-roots f-final as dg composition-fri-roots final*
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table f

where *partial:*
accepted-with-partial-initial-openings s out fr f-fri-roots f-final
as dg composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings

and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings

and *deg-f: degree f < clength*

and *trace-table: trace-table = map (poly f) eval-domain*

and *violated: violated-constraints f ≠ {}*

and *bounds: common-denominator-degree-bounds f as*

and *hides:*
random-combination-common-denominator-hides-violations f as

using *bad*

unfolding *composition-randomization-bad-with-partial-candidates-def*
by *blast*

have *as-in:*
as ∈ common-denominator-hiding-alpha-space f
using *accepted-with-partial-initial-openings-alpha-length[OF partial]*
hides

unfolding *common-denominator-hiding-alpha-space-def alpha-space-def*
by *simp*

have *witness-eq: low-degree-trace-witness trace-table = f*
by *(rule low-degree-trace-witness-eq[OF deg-f trace-table])*

have *as ∈ composition-trace-bad-alpha-space trace-table*
using *deg-f trace-table violated bounds as-in*

unfolding *composition-trace-bad-alpha-space-def witness-eq*
by *simp*

then show *?thesis*
unfolding *composition-alpha-bad-set-hit-with-partial-candidates-def*
using *partial trace-candidate* **by** *blast*

qed

lemma *composition-alpha-bad-set-hit-with-partial-candidates-imp-alpha-header-partial-union-on-support:*
assumes *support: out ∈ set-dist (execute verify-monad s)*
and *hit: composition-alpha-bad-set-hit-with-partial-candidates s out*
shows
alpha-header-list-set-hit s
(alpha-header-supported-partial-union-bad-sets s
composition-trace-bad-alpha-space) out

proof –


```

obtain fr f-fri-roots f-final as dg composition-fri-roots final
  trace-query-idxs trace-openings composition-query-idxs
  composition-openings trace-table rest
where partial:
  accepted-with-partial-initial-openings s out fr f-fri-roots f-final
    as dg composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and trace-partial:
  accepted-with-partial-trace-openings s out fr trace-query-idxs
    trace-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and as-bad:
  as ∈ composition-trace-bad-alpha-space trace-table
using hit
unfolding composition-alpha-bad-set-hit-with-partial-candidates-def
  accepted-with-partial-initial-openings-def
by blast
from trace-partial obtain result final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-with-partial-trace-openings-def by blast
from verify-monad-accepted-transcript-shape[OF support[unfolded out-eq]]
obtain as' query-idxs where shape':
  accepted-transcript-shape s (Some (result, final-state)) as' query-idxs
by blast
from shape' obtain result' final-state' fr' f-fri-roots' f-final' dg'
  composition-fri-roots' final' rest' where out-shape:
  Some (result, final-state) = Some (result', final-state')
and header':
  verifier-header-transcript s fr' f-fri-roots' f-final' as' dg'
    composition-fri-roots' final' rest'
by (elim accepted-transcript-shape-header-query-extraction)
have as'-eq: as' = as
using verifier-header-transcript-unique[OF header' header] by simp
have shape:
  accepted-transcript-shape s out as query-idxs
using shape' as'-eq out-eq by simp
have candidate-header:
  trace-table ∈
    alpha-header-supported-partial-trace-table-candidates s fr
      f-fri-roots f-final
unfolding alpha-header-supported-partial-trace-table-candidates-def
using support trace-partial trace-candidate header by blast
have as-union:
  as ∈
    alpha-header-supported-partial-union-bad-sets s

```

```

    composition-trace-bad-alpha-space fr f-fri-roots f-final
unfolding alpha-header-supported-partial-union-bad-sets-def
using candidate-header as-bad
    composition-trace-bad-alpha-space-subset-alpha-space[of trace-table]
by auto
show ?thesis
    unfolding alpha-header-list-set-hit-def
    using shape out-eq header as-union by blast
qed

lemma wp-composition-randomization-bad-with-partial-candidates-le-alpha-header-partial-union:
  wp-event verify-monad
    (composition-randomization-bad-with-partial-candidates s) s ≤
  wp-event verify-monad
    (alpha-header-list-set-hit s
      (alpha-header-supported-partial-union-bad-sets s
        composition-trace-bad-alpha-space)) s
by (rule wp-event-mono-on-support)
  (rule
    composition-alpha-bad-set-hit-with-partial-candidates-imp-alpha-header-partial-union-on-support
    [OF - composition-randomization-bad-with-partial-candidates-imp-alpha-hit])

lemma accepted-partial-candidate-partition:
assumes false-statement: ¬ exists-valid-trace
and partial:
  accepted-with-partial-initial-openings s out fr f-fri-roots f-final
  as dg composition-fri-roots final trace-query-idxs trace-openings
  composition-query-idxs composition-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
shows soundness-bad-event-partial-candidate s out
proof (cases trace-table-low-degree trace-table)
case False
have trace-fri-bad-with-partial-candidates s out
  unfolding trace-fri-bad-with-partial-candidates-def
  using partial trace-candidate False by blast
then show ?thesis
  unfolding soundness-bad-event-partial-candidate-def by blast
next
case True
note trace-low = True
show ?thesis
proof (cases composition-table-low-degree maxDegree composition-table)
case False
have composition-fri-bad-with-partial-candidates s out
  unfolding composition-fri-bad-with-partial-candidates-def

```

```

    using partial trace-candidate composition-candidate trace-low False
    by blast
  then show ?thesis
    unfolding soundness-bad-event-partial-candidate-def by blast
next
case True
note composition-low = True
show ?thesis
proof (cases all-queries-consistent trace-table composition-table as)
  case False
  have query-bad-with-partial-candidates s out
    unfolding query-bad-with-partial-candidates-def
    using partial trace-candidate composition-candidate trace-low
      composition-low False
    by blast
  then show ?thesis
    unfolding soundness-bad-event-partial-candidate-def by blast
next
case True
note all-queries = True
from trace-low obtain f where
  deg-f: degree f < clength
  and trace-table-eq: trace-table = map (poly f) eval-domain
  unfolding trace-table-low-degree-def by blast
have violated: violated-constraints f ≠ {}
  by (rule false-statement-violated-constraints
    [OF false-statement deg-f])
have composition-bad-with-partial-candidates s out
  unfolding composition-bad-with-partial-candidates-def
  using partial trace-candidate composition-candidate deg-f
    trace-table-eq violated composition-low all-queries
  by blast
then show ?thesis
  unfolding soundness-bad-event-partial-candidate-def by blast
qed
qed
qed

```

```

lemma constant-composition-table-low-degree:
  composition-table-low-degree maxDegree (replicate (scale * clength) final)
proof -
  have map (poly [:final:]) eval-domain =
    replicate (scale * clength) final
  proof -
    have map (poly [:final:]) eval-domain =
      replicate (length eval-domain) final
    proof (rule nth-equalityI)
      show length (map (poly [:final:]) eval-domain) =
        length (replicate (length eval-domain) final)
    qed
  qed

```

```

    by simp
next
  fix i
  assume i < length (map (poly [:final:]) eval-domain)
  then show map (poly [:final:]) eval-domain ! i =
    replicate (length eval-domain) final ! i
    by simp
qed
then show ?thesis
  using eval-domain-length by (simp add: mult.commute)
qed
then show ?thesis
  unfolding composition-table-low-degree-def
  by (intro exI[of - [:final:]]) simp
qed

lemma accepted-with-empty-composition-header-candidates-alpha-length:
  assumes
    accepted-with-empty-composition-header-candidates s out fr
    f-fri-roots f-final as dg final trace-query-idxs trace-openings
    trace-table composition-table
  shows length as = length spec
proof -
  obtain rest where
    verifier-header-transcript s fr f-fri-roots f-final as dg []
    final rest
  using assms
  unfolding accepted-with-empty-composition-header-candidates-def by blast
  then show ?thesis
    using verifier-header-transcript-shapes(3) by blast
qed

lemma composition-bad-with-empty-composition-header-candidates-split:
  assumes false-statement:  $\neg$  exists-valid-trace
  and bad: composition-bad-with-empty-composition-header-candidates s out
  shows
    composition-degree-bad-with-empty-composition-header-candidates s out  $\vee$ 
    composition-randomization-bad-with-empty-composition-header-candidates s out
proof -
  obtain fr f-fri-roots f-final as dg final trace-query-idxs
    trace-openings trace-table composition-table f
  where partial:
    accepted-with-empty-composition-header-candidates s out fr
    f-fri-roots f-final as dg final trace-query-idxs trace-openings
    trace-table composition-table
  and deg-f: degree f < clength
  and trace-table: trace-table = map (poly f) eval-domain
  and violated: violated-constraints f  $\neq$  {}
  and composition-low:

```

```

    composition-table-low-degree maxDegree composition-table
and all-queries:
    all-queries-consistent trace-table composition-table as
using bad
unfolding composition-bad-with-empty-composition-header-candidates-def
by blast
show ?thesis
proof (cases common-denominator-degree-bounds f as)
  case False
  then have
    composition-degree-bad-with-empty-composition-header-candidates s out
  unfolding composition-degree-bad-with-empty-composition-header-candidates-def
  using partial deg-f trace-table violated composition-low all-queries
  by blast
  then show ?thesis by blast
next
case True
note bounds = True
have len-as: length as = length spec
  by (rule accepted-with-empty-composition-header-candidates-alpha-length
    [OF partial])
have hides:
  random-combination-common-denominator-hides-violations f as
  by (rule random-combination-common-denominator-hides-from-queries
    [OF len-as false-statement deg-f trace-table composition-low
      all-queries bounds])
have
  composition-randomization-bad-with-empty-composition-header-candidates
    s out
  unfolding
    composition-randomization-bad-with-empty-composition-header-candidates-def
  using partial deg-f trace-table violated composition-low all-queries
    bounds hides
  by blast
  then show ?thesis by blast
qed
qed

lemma composition-degree-bad-with-empty-composition-header-candidates-false:
  ¬ composition-degree-bad-with-empty-composition-header-candidates s out
using common-denominator-degree-bounds-from-spec-degree-wellformed
  [OF spec-degree-wellformed-from-spec-query-margin]
unfolding composition-degree-bad-with-empty-composition-header-candidates-def
by blast

lemma composition-degree-bad-with-empty-composition-header-candidates-bound:
  wp-event verify-monad
    (composition-degree-bad-with-empty-composition-header-candidates s) s ≤
    0

```

```

proof –
  have composition-degree-bad-with-empty-composition-header-candidates s =
    ( $\lambda out :: (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option.}$ 
      False)
  by (rule ext)
    (simp add:
      composition-degree-bad-with-empty-composition-header-candidates-false)
  then show ?thesis
    unfolding wp-event-def wp-def dist-expect-def by simp
qed

lemma composition-randomization-bad-with-empty-composition-header-candidates-imp-alpha-hit:
  assumes bad:
    composition-randomization-bad-with-empty-composition-header-candidates
    s out
  shows
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates
    s out
proof –
  obtain fr f-fri-roots f-final as dg final trace-query-idxs
    trace-openings trace-table composition-table f
  where partial:
    accepted-with-empty-composition-header-candidates s out fr
    f-fri-roots f-final as dg final trace-query-idxs trace-openings
    trace-table composition-table
  and deg-f: degree f < clength
  and trace-table: trace-table = map (poly f) eval-domain
  and violated: violated-constraints f  $\neq$  {}
  and bounds: common-denominator-degree-bounds f as
  and hides:
    random-combination-common-denominator-hides-violations f as
  using bad
  unfolding
    composition-randomization-bad-with-empty-composition-header-candidates-def
  by blast
  have as-in:
    as  $\in$  common-denominator-hiding-alpha-space f
  using accepted-with-empty-composition-header-candidates-alpha-length[OF partial]
    hides
  unfolding common-denominator-hiding-alpha-space-def alpha-space-def
  by simp
  have witness-eq: low-degree-trace-witness trace-table = f
    by (rule low-degree-trace-witness-eq[OF deg-f trace-table])
  have as  $\in$  composition-trace-bad-alpha-space trace-table
    using deg-f trace-table violated bounds as-in
  unfolding composition-trace-bad-alpha-space-def witness-eq
  by simp
  then show ?thesis
    unfolding

```

```

      composition-alpha-bad-set-hit-with-empty-composition-header-candidates-def
    using partial by blast
  qed

lemma wp-composition-randomization-bad-with-empty-composition-header-candidates-le-alpha-hit:
  wp-event verify-monad
    (composition-randomization-bad-with-empty-composition-header-candidates s)
  s ≤
  wp-event verify-monad
    (composition-alpha-bad-set-hit-with-empty-composition-header-candidates s)
  s
  by (rule wp-event-mono)
  (rule
    composition-randomization-bad-with-empty-composition-header-candidates-imp-alpha-hit)

lemma accepted-empty-composition-header-candidate-partition:
  assumes false-statement:  $\neg$  exists-valid-trace
  and partial:
    accepted-with-empty-composition-header-candidates s out fr
    f-fri-roots f-final as dg final trace-query-idxs trace-openings
    trace-table composition-table
  shows soundness-bad-event-partial-candidate-empty-header s out
proof (cases trace-table-low-degree trace-table)
  case False
  have trace-fri-bad-with-empty-composition-header-candidates s out
  unfolding trace-fri-bad-with-empty-composition-header-candidates-def
  using partial False by blast
  then show ?thesis
  unfolding soundness-bad-event-partial-candidate-empty-header-def by blast
next
  case True
  note trace-low = True
  have composition-low:
    composition-table-low-degree maxDegree composition-table
  proof -
    have composition-table = replicate (scale * clength) final
    using partial
    unfolding accepted-with-empty-composition-header-candidates-def by simp
  then show ?thesis
  using constant-composition-table-low-degree by simp
qed
show ?thesis
proof (cases all-queries-consistent trace-table composition-table as)
  case False
  have query-bad-with-empty-composition-header-candidates s out
  unfolding query-bad-with-empty-composition-header-candidates-def
  using partial trace-low composition-low False by blast
  then show ?thesis
  unfolding soundness-bad-event-partial-candidate-empty-header-def

```

```

    by blast
next
case True
note all-queries = True
from trace-low obtain f where
  deg-f: degree f < clength
  and trace-table-eq: trace-table = map (poly f) eval-domain
  unfolding trace-table-low-degree-def by blast
have violated: violated-constraints f ≠ {}
  by (rule false-statement-violated-constraints
      [OF false-statement deg-f])
have composition-bad-with-empty-composition-header-candidates s out
  unfolding composition-bad-with-empty-composition-header-candidates-def
  using partial deg-f trace-table-eq violated composition-low all-queries
  by blast
then show ?thesis
  unfolding soundness-bad-event-partial-candidate-empty-header-def
  by blast
qed
qed

lemma fri-merkle-binding-bad-imp-table-aware-with-merkle:
  assumes fri-merkle-binding-bad s out
  shows soundness-bad-event-table-aware-with-merkle s out
  using assms unfolding soundness-bad-event-table-aware-with-merkle-def by simp

lemma fri-merkle-binding-bad-imp-partial-opening-with-merkle:
  assumes fri-merkle-binding-bad s out
  shows soundness-bad-event-partial-opening-with-merkle s out
  using assms
  unfolding soundness-bad-event-partial-opening-with-merkle-def by simp

lemma soundness-bad-event-table-aware-imp-partial-opening-on-support:
  assumes support: out ∈ set-dist (execute verify-monad s)
  and bad: soundness-bad-event-table-aware s out
  shows soundness-bad-event-partial-opening s out
  using bad
proof (unfold soundness-bad-event-table-aware-def, elim disjE)
  assume composition-bad s out
  then show ?thesis
    unfolding soundness-bad-event-partial-opening-def by blast
next
  assume trace-bad: trace-fri-bad-with-tables s out
  then have trace-fri-bad-with-partial-openings s out
    by (rule trace-fri-bad-with-tables-imp-partial-openings-on-support
        [OF support])
  then show ?thesis
    unfolding soundness-bad-event-partial-opening-def by blast
next

```



```

assume composition-bad: composition-fri-bad-with-tables s out
then have composition-fri-bad-with-partial-openings s out
  by (rule composition-fri-bad-with-tables-imp-partial-openings-on-support
    [OF support])
then show ?thesis
  unfolding soundness-bad-event-partial-opening-def by blast
next
  assume query-bad s out
  then show ?thesis
    unfolding soundness-bad-event-partial-opening-def by blast
qed

lemma soundness-bad-event-table-aware-with-merkle-imp-partial-opening-with-merkle-on-support:
  assumes support: out ∈ set-dist (execute verify-monad s)
    and bad: soundness-bad-event-table-aware-with-merkle s out
  shows soundness-bad-event-partial-opening-with-merkle s out
  using bad
proof (unfold soundness-bad-event-table-aware-with-merkle-def, elim disjE)
  assume fri-merkle-binding-bad s out
  then show ?thesis
    by (rule fri-merkle-binding-bad-imp-partial-opening-with-merkle)
next
  assume soundness-bad-event-table-aware s out
  then have soundness-bad-event-partial-opening s out
    by (rule soundness-bad-event-table-aware-imp-partial-opening-on-support
      [OF support])
  then show ?thesis
    by (rule soundness-bad-event-partial-opening-imp-with-merkle)
qed

lemma wp-soundness-bad-event-table-aware-le-partial-opening:
  wp-event verify-monad (soundness-bad-event-table-aware s) s ≤
    wp-event verify-monad (soundness-bad-event-partial-opening s) s
  by (rule wp-event-mono-on-support)
    (rule soundness-bad-event-table-aware-imp-partial-opening-on-support)

lemma wp-soundness-bad-event-table-aware-with-merkle-le-partial-opening-with-merkle:
  wp-event verify-monad (soundness-bad-event-table-aware-with-merkle s) s ≤
    wp-event verify-monad
      (soundness-bad-event-partial-opening-with-merkle s) s
  by (rule wp-event-mono-on-support)
    (rule
      soundness-bad-event-table-aware-with-merkle-imp-partial-opening-with-merkle-on-support)

lemma soundness-bad-event-table-aware-with-merkle-imp-with-pairwise-side:
  assumes soundness-bad-event-table-aware-with-merkle s out
  shows soundness-bad-event-table-aware-with-pairwise-side s out
  using assms
  unfolding soundness-bad-event-table-aware-with-pairwise-side-def by simp

```

lemma *supported-pairwise-side-bad-imp-table-aware-with-pairwise-side:*
assumes *supported-pairwise-side-bad s out*
shows *soundness-bad-event-table-aware-with-pairwise-side s out*
using *assms*
unfolding *soundness-bad-event-table-aware-with-pairwise-side-def* **by** *simp*

lemma *soundness-bad-event-table-aware-with-merkle-imp-with-output-local-side:*
assumes *soundness-bad-event-table-aware-with-merkle s out*
shows *soundness-bad-event-table-aware-with-output-local-side s out*
using *assms*
unfolding *soundness-bad-event-table-aware-with-output-local-side-def*
by *simp*

lemma *supported-output-local-side-bad-imp-table-aware-with-output-local-side:*
assumes *supported-output-local-side-bad s out*
shows *soundness-bad-event-table-aware-with-output-local-side s out*
using *assms*
unfolding *soundness-bad-event-table-aware-with-output-local-side-def*
by *simp*

lemma *accepted-with-fri-tables-table-aware-partition:*
assumes *false-statement: $\neg$ exists-valid-trace*
and *tables: accepted-with-fri-tables s out*
shows *soundness-bad-event-table-aware s out*
proof –
from *accepted-with-fri-tables-obtain-bound-tables[OF tables]*
obtain *trace-table composition-table as query-idxs* **where** *bound:*
accepted-with-bound-tables s out trace-table composition-table as
query-idxs
by *blast*
have *partition:*
composition-bad s out $\vee$ trace-fri-bad s out $\vee$
composition-fri-bad s out $\vee$ query-bad s out
by *(rule accepted-bound-table-partition[OF false-statement bound])*
then show *?thesis*
proof
assume *composition-bad s out*
then show *?thesis*
unfolding *soundness-bad-event-table-aware-def* **by** *blast*
next
assume *rest: trace-fri-bad s out $\vee$*
composition-fri-bad s out $\vee$ query-bad s out
then show *?thesis*
proof
assume *trace-bad: trace-fri-bad s out*
then have *trace-fri-bad-with-tables s out*
using *tables* **unfolding** *trace-fri-bad-with-tables-def* **by** *simp*
then show *?thesis*

```

    unfolding soundness-bad-event-table-aware-def by blast
next
  assume rest': composition-fri-bad s out  $\vee$  query-bad s out
  then show ?thesis
  proof
    assume composition-bad: composition-fri-bad s out
    then have composition-fri-bad-with-tables s out
      using tables unfolding composition-fri-bad-with-tables-def by simp
    then show ?thesis
      unfolding soundness-bad-event-table-aware-def by blast
  next
    assume query-bad s out
    then show ?thesis
      unfolding soundness-bad-event-table-aware-def by blast
  qed
qed
qed
qed

lemma accepted-partition-soundness-bad-event-table-aware-with-merkle:
  assumes false-statement:  $\neg$  exists-valid-trace
  and acc: accepted out
  shows soundness-bad-event-table-aware-with-merkle s out
proof -
  from accepted-partition-fri-merkle-binding[OF acc]
  consider
    (merkle-bad) fri-merkle-binding-bad s out
  | (with-tables) accepted-with-fri-tables s out
  by blast
  then show ?thesis
  proof cases
    case merkle-bad
    then show ?thesis
      by (rule fri-merkle-binding-bad-imp-table-aware-with-merkle)
  next
    case with-tables
    have soundness-bad-event-table-aware s out
      by (rule accepted-with-fri-tables-table-aware-partition
        [OF false-statement with-tables])
    then show ?thesis
      by (rule soundness-bad-event-table-aware-imp-with-merkle)
  qed
qed

lemma accepted-partition-soundness-bad-event-table-aware-with-pairwise-side:
  assumes false-statement:  $\neg$  exists-valid-trace
  and acc: accepted out
  shows soundness-bad-event-table-aware-with-pairwise-side s out
  by (rule soundness-bad-event-table-aware-with-merkle-imp-with-pairwise-side)

```

(rule accepted-partition-soundness-bad-event-table-aware-with-merkle
[OF false-statement acc])

lemma *accepted-partition-soundness-bad-event-table-aware-with-output-local-side:*

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *acc*: *accepted out*

shows *soundness-bad-event-table-aware-with-output-local-side s out*

by (rule

soundness-bad-event-table-aware-with-merkle-imp-with-output-local-side)

(rule accepted-partition-soundness-bad-event-table-aware-with-merkle
[OF false-statement acc])

lemma *accepted-partition-soundness-bad-event-partial-opening-with-merkle:*

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *supp*: *out* $\in$ *set-dist* (*execute verify-monad s*)

and *acc*: *accepted out*

shows *soundness-bad-event-partial-opening-with-merkle s out*

proof –

have *soundness-bad-event-table-aware-with-merkle s out*

by (rule accepted-partition-soundness-bad-event-table-aware-with-merkle
[OF false-statement acc])

then show *?thesis*

by (rule

soundness-bad-event-table-aware-with-merkle-imp-partial-opening-with-merkle-on-support
[OF *supp*])

qed

lemma *accepted-partition-soundness-bad-event-partial-opening-with-initial-merkle:*

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *supp*: *out* $\in$ *set-dist* (*execute verify-monad s*)

and *acc*: *accepted out*

shows *soundness-bad-event-partial-opening-with-initial-merkle s out*

proof –

from *acc* **obtain** *result final-state* **where** *out-eq*:

out = *Some (result, final-state)*

unfolding *accepted-def* **by** (*cases out*) *auto*

from *verify-monad-accepted-transcript-shape*[OF *supp*[*unfolded out-eq*]]

obtain *as query-idxs* **where** *shape*:

accepted-transcript-shape s (Some (result, final-state)) as query-idxs

by *blast*

show *?thesis*

proof (*cases*

$\exists$ *trace-table composition-table*.

accepted-with-bound-tables s (Some (result, final-state))

trace-table composition-table as query-idxs)

case *False*

have *initial-merkle-binding-bad s out*

unfolding *initial-merkle-binding-bad-def out-eq*

using *shape False* **by** *blast*

```

then show ?thesis
  by (rule initial-merkle-binding-bad-imp-partial-opening-with-initial-merkle)
next
case True
then obtain trace-table composition-table where bound:
  accepted-with-bound-tables s (Some (result, final-state))
  trace-table composition-table as query-idxs
  by blast
have partial-bad:
  soundness-bad-event-partial-opening s out
proof –
  have partition:
    composition-bad s (Some (result, final-state)) ∨
    trace-fri-bad s (Some (result, final-state)) ∨
    composition-fri-bad s (Some (result, final-state)) ∨
    query-bad s (Some (result, final-state))
  by (rule accepted-bound-table-partition[OF false-statement bound])
then show ?thesis
proof
  assume composition-bad s (Some (result, final-state))
  then show ?thesis
    unfolding soundness-bad-event-partial-opening-def out-eq by blast
next
  assume rest:
    trace-fri-bad s (Some (result, final-state)) ∨
    composition-fri-bad s (Some (result, final-state)) ∨
    query-bad s (Some (result, final-state))
  then show ?thesis
  proof
    assume trace-bad:
      trace-fri-bad s (Some (result, final-state))
    from verify-monad-accepted-with-partial-trace-openings
      [OF supp[unfolded out-eq]]
    obtain fr query-idxs' trace-openings where partial:
      accepted-with-partial-trace-openings s
      (Some (result, final-state)) fr query-idxs' trace-openings
    by blast
    have trace-fri-bad-with-partial-openings s out
    unfolding trace-fri-bad-with-partial-openings-def out-eq
    using trace-bad partial by blast
    then show ?thesis
    unfolding soundness-bad-event-partial-opening-def by blast
next
  assume rest':
    composition-fri-bad s (Some (result, final-state)) ∨
    query-bad s (Some (result, final-state))
  then show ?thesis
  proof
    assume composition-bad:

```

```

    composition-fri-bad s (Some (result, final-state))
  from verify-monad-accepted-with-partial-trace-openings
    [OF supp[unfolded out-eq]]
  obtain fr query-idxs' trace-openings where partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs' trace-openings
  by blast
  have composition-fri-bad-with-partial-openings s out
    unfolding composition-fri-bad-with-partial-openings-def out-eq
    using composition-bad partial by blast
  then show ?thesis
    unfolding soundness-bad-event-partial-opening-def by blast
next
  assume query-bad s (Some (result, final-state))
  then show ?thesis
    unfolding soundness-bad-event-partial-opening-def out-eq
    by blast
qed
qed
qed
qed
then show ?thesis
  by (rule soundness-bad-event-partial-opening-imp-with-initial-merkle)
qed
qed

lemma accepted-partition-soundness-bad-event-partial-candidate-with-partial-merkle-if-header-nonempty:
  assumes false-statement:  $\neg$  exists-valid-trace
  and supp:  $out \in \text{set-dist } (\text{execute verify-monad } s)$ 
  and acc: accepted out
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and comp-nonempty:  $\text{composition-fri-roots} \neq []$ 
  shows soundness-bad-event-partial-candidate-with-partial-merkle s out
proof -
  from acc obtain result final-state where out-eq:
    out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
  from verify-monad-partial-initial-candidates-or-partial-merkle-bad
    [OF supp[unfolded out-eq] header comp-nonempty]
  consider
    (partial-merkle)
    partial-merkle-inconsistency-bad s (Some (result, final-state))
  | (candidates) trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table where
    accepted-with-partial-initial-openings s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings

```

```

    composition-query-idxs composition-openings
    partial-trace-table-candidate trace-table trace-openings
    partial-composition-table-candidate composition-table
    composition-openings
  by blast
then show ?thesis
proof cases
  case partial-merkle
  then show ?thesis
    unfolding out-eq
    by (rule
      partial-merkle-inconsistency-bad-imp-partial-candidate-with-partial-merkle)
next
  case (candidates trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table)
  have bad:
    soundness-bad-event-partial-candidate s
    (Some (result, final-state))
  by (rule accepted-partial-candidate-partition
    [OF false-statement candidates(1) candidates(2) candidates(3)])
  then show ?thesis
    unfolding out-eq
    by (rule soundness-bad-event-partial-candidate-imp-with-partial-merkle)
qed
qed

lemma accepted-partition-soundness-bad-event-partial-candidate-with-partial-merkle-or-empty-header:
  assumes false-statement:  $\neg$  exists-valid-trace
  and supp:  $out \in \text{set-dist } (\text{execute verify-monad } s)$ 
  and acc: accepted out
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  shows
    soundness-bad-event-partial-candidate-with-partial-merkle s out  $\vee$ 
    soundness-bad-event-partial-candidate-empty-header s out
proof (cases composition-fri-roots = [])
  case False
  then have comp-nonempty: composition-fri-roots  $\neq []$ 
  by simp
  have soundness-bad-event-partial-candidate-with-partial-merkle s out
  by (rule
    accepted-partition-soundness-bad-event-partial-candidate-with-partial-merkle-if-header-nonempty
    [OF false-statement supp acc header comp-nonempty])
  then show ?thesis by simp
next
  case True
  note comp-empty = True
  from acc obtain result final-state where out-eq:

```

```

    out = Some (result, final-state)
    unfolding accepted-def by (cases out) auto
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then have soundness-bad-event-partial-candidate-with-partial-merkle s out
  unfolding out-eq
  by (rule
    partial-merkle-inconsistency-bad-imp-partial-candidate-with-partial-merkle)
then show ?thesis by simp
next
case False
from verify-monad-accepted-with-partial-trace-openings-for-header
  [OF supp[unfolded out-eq] header]
obtain trace-query-idxs trace-openings where trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr trace-query-idxs trace-openings
  by blast
obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
  using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
  [OF trace-partial False]
  by blast
have accepted-out: accepted (Some (result, final-state))
  by (rule accepted-with-partial-trace-openings-imp-accepted
    [OF trace-partial])
let ?composition-table = replicate (scale * clength) final
have empty-partial:
  accepted-with-empty-composition-header-candidates s
  (Some (result, final-state)) fr f-fri-roots f-final as dg final
  trace-query-idxs trace-openings trace-table ?composition-table
  unfolding accepted-with-empty-composition-header-candidates-def
  by (intro conjI exI[of - rest])
  (use accepted-out header comp-empty trace-partial trace-candidate
    in simp-all)
have soundness-bad-event-partial-candidate-empty-header s
  (Some (result, final-state))
  by (rule accepted-empty-composition-header-candidate-partition
    [OF false-statement empty-partial])
then show ?thesis
  unfolding out-eq by simp
qed
qed

definition trace-fri-with-tables-opening-algebraic-cover
:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  ('f list  $\Rightarrow$  'f list set)  $\Rightarrow$  bool
where

```


$\text{trace-fri-with-tables-opening-algebraic-cover } s \text{ bad-sets} \longleftrightarrow$
 $(\forall \text{ out trace-table composition-table as query-idxs opening-query-idxs}$
 $\text{trace-roots trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final trace-round-layers composition-round-layers.}$
 $\text{accepted-with-fri-tables } s \text{ out} \longrightarrow$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as}$
 $\text{query-idxs} \longrightarrow$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs trace-final}$
 $\text{dg composition-roots composition-bs composition-final opening-query-idxs}$
 $\text{trace-round-layers composition-round-layers} \longrightarrow$
 $\neg \text{trace-table-low-degree trace-table} \longrightarrow$
 $\text{trace-bs} \in \text{bad-sets trace-table})$

definition *composition-fri-with-tables-opening-algebraic-cover*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow \text{bool}$

where

$\text{composition-fri-with-tables-opening-algebraic-cover } s \text{ bad-sets} \longleftrightarrow$
 $(\forall \text{ out trace-table composition-table as query-idxs opening-query-idxs}$
 $\text{trace-roots trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final trace-round-layers composition-round-layers.}$
 $\text{accepted-with-fri-tables } s \text{ out} \longrightarrow$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as}$
 $\text{query-idxs} \longrightarrow$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs trace-final}$
 $\text{dg composition-roots composition-bs composition-final opening-query-idxs}$
 $\text{trace-round-layers composition-round-layers} \longrightarrow$
 $\text{trace-table-low-degree trace-table} \longrightarrow$
 $\neg \text{composition-table-low-degree maxDegree composition-table} \longrightarrow$
 $\text{composition-bs} \in \text{bad-sets dg composition-table})$

definition *trace-fri-with-tables-bad-sets*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list set}$

where

$\text{trace-fri-with-tables-bad-sets } s \text{ trace-table} =$
 $\{ \text{trace-bs.}$
 $\exists \text{ out composition-table as query-idxs opening-query-idxs}$
 $\text{trace-roots trace-final dg composition-roots composition-bs}$
 $\text{composition-final trace-round-layers composition-round-layers.}$
 $\text{accepted-with-fri-tables } s \text{ out} \wedge$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as}$
 $\text{query-idxs} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs}$
 $\text{composition-final opening-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \}$

definition *composition-fri-with-tables-bad-sets*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list set}$

where

$\text{composition-fri-with-tables-bad-sets } s \text{ dg composition-table} =$
 $\{ \text{composition-bs.}$

$\exists \text{ out trace-table as query-idxs opening-query-idxs trace-roots}$
 $\text{trace-bs trace-final composition-roots composition-final}$
 $\text{trace-round-layers composition-round-layers.}$

$\text{accepted-with-fri-tables } s \text{ out} \wedge$

$\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as}$
 $\text{query-idxs} \wedge$

$\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{opening-query-idxs trace-round-layers composition-round-layers} \wedge$

$\text{trace-table-low-degree trace-table} \wedge$

$\neg \text{composition-table-low-degree maxDegree composition-table} \}$

lemma $\text{trace-fri-with-tables-bad-sets-cover:}$

$\text{trace-fri-with-tables-opening-algebraic-cover } s$
 $(\text{trace-fri-with-tables-bad-sets } s)$

unfolding $\text{trace-fri-with-tables-opening-algebraic-cover-def}$

$\text{trace-fri-with-tables-bad-sets-def}$

by blast

lemma $\text{composition-fri-with-tables-bad-sets-cover:}$

$\text{composition-fri-with-tables-opening-algebraic-cover } s$
 $(\text{composition-fri-with-tables-bad-sets } s)$

unfolding $\text{composition-fri-with-tables-opening-algebraic-cover-def}$

$\text{composition-fri-with-tables-bad-sets-def}$

by blast

lemma $\text{trace-fri-with-tables-bad-sets-subset:}$

$\text{trace-fri-with-tables-bad-sets } s \text{ trace-table} \subseteq$
 $\text{fri-challenge-space } (\text{ceil-log clength})$

unfolding $\text{trace-fri-with-tables-bad-sets-def}$

using $\text{accepted-fri-opening-transcript-challenges}$

$\text{accepted-fri-challenges-trace-space}$

by blast

lemma $\text{composition-fri-with-tables-bad-sets-subset:}$

$\text{composition-fri-with-tables-bad-sets } s \text{ dg composition-table} \subseteq$
 $\text{fri-challenge-space } (\text{ceil-log } (\text{to-nat dg} + 1))$

unfolding $\text{composition-fri-with-tables-bad-sets-def}$

using $\text{accepted-fri-opening-transcript-challenges}$

$\text{accepted-fri-challenges-composition-space}$

by blast

lemma $\text{trace-fri-with-tables-bad-sets-card-bound:}$

$\text{card } (\text{trace-fri-with-tables-bad-sets } s \text{ trace-table}) \leq$
 $\text{CARD}('f) \wedge \text{ceil-log clength}$

proof –
have $\text{card } (\text{trace-fri-with-tables-bad-sets } s \text{ trace-table}) \leq$
 $\text{card } (\text{fri-challenge-space } (\text{ceil-log } \text{clength}))$
by (rule *card-mono*[*OF finite-fri-challenge-space*])
(rule *trace-fri-with-tables-bad-sets-subset*)
also have $\dots = \text{CARD}('f) \wedge \text{ceil-log } \text{clength}$
by (rule *card-fri-challenge-space*)
finally show ?thesis .
qed

lemma *composition-fri-with-tables-bad-sets-card-bound*:
 $\text{card } (\text{composition-fri-with-tables-bad-sets } s \text{ dg } \text{composition-table}) \leq$
 $\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } \text{dg} + 1)$
proof –
have $\text{card } (\text{composition-fri-with-tables-bad-sets } s \text{ dg } \text{composition-table}) \leq$
 $\text{card } (\text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } \text{dg} + 1)))$
by (rule *card-mono*[*OF finite-fri-challenge-space*])
(rule *composition-fri-with-tables-bad-sets-subset*)
also have $\dots = \text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } \text{dg} + 1)$
by (rule *card-fri-challenge-space*)
finally show ?thesis .
qed

lemma *trace-fri-with-tables-bad-sets-fraction-le-one*:
 $\text{nnreal } (\text{card } (\text{trace-fri-with-tables-bad-sets } s \text{ trace-table})) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } \text{clength}) \leq 1$
proof –
have $\text{nnreal } (\text{card } (\text{trace-fri-with-tables-bad-sets } s \text{ trace-table})) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } \text{clength}) \leq$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } \text{clength}) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } \text{clength})$
by (rule *nnreal-nat-divide-right-mono*)
(rule *trace-fri-with-tables-bad-sets-card-bound*)
also have $\dots = 1$
by (rule *nnreal-nat-divide-self*) *simp*
finally show ?thesis .
qed

lemma *composition-fri-with-tables-bad-sets-fraction-le-one*:
 $\text{nnreal } (\text{card } (\text{composition-fri-with-tables-bad-sets } s \text{ dg } \text{composition-table})) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } \text{dg} + 1)) \leq 1$
proof –
have nnreal
 $(\text{card } (\text{composition-fri-with-tables-bad-sets } s \text{ dg } \text{composition-table})) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } \text{dg} + 1)) \leq$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } \text{dg} + 1)) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } \text{dg} + 1))$
by (rule *nnreal-nat-divide-right-mono*)
(rule *composition-fri-with-tables-bad-sets-card-bound*)

also have ... = 1
 by (rule nnreal-nat-divide-self) simp
 finally show ?thesis .
 qed

definition *trace-fri-bad-with-tables-challenge-cover*
 :: ('f, 'a) protocol-channel-scheme $\Rightarrow$
 ('f list $\Rightarrow$ 'f list set) $\Rightarrow$ bool
where
 trace-fri-bad-with-tables-challenge-cover s bad-sets $\longleftrightarrow$
 ($\forall$ out $\in$ set-dist (execute verify-monad s).
 trace-fri-bad-with-tables s out $\longrightarrow$
 trace-fri-challenge-set-hit s bad-sets out)

definition *composition-fri-bad-with-tables-challenge-cover*
 :: ('f, 'a) protocol-channel-scheme $\Rightarrow$
 ('f $\Rightarrow$ 'f list $\Rightarrow$ 'f list set) $\Rightarrow$ bool
where
 composition-fri-bad-with-tables-challenge-cover s bad-sets $\longleftrightarrow$
 ($\forall$ out $\in$ set-dist (execute verify-monad s).
 composition-fri-bad-with-tables s out $\longrightarrow$
 composition-fri-challenge-set-hit s bad-sets out)

lemma *trace-fri-with-tables-opening-algebraic-cover-imp-bad-challenge-cover:*
assumes cover:
 trace-fri-with-tables-opening-algebraic-cover s bad-sets
shows trace-fri-bad-with-tables-challenge-cover s bad-sets
unfolding trace-fri-bad-with-tables-challenge-cover-def
proof (intro ballI impI)
fix out :: (unit list $\times$ ('f, 'a) protocol-channel-scheme) option
assume outcome: out $\in$ set-dist (execute verify-monad s)
and bad: trace-fri-bad-with-tables s out
then have with-tables: accepted-with-fri-tables s out
and trace-bad: trace-fri-bad s out
unfolding trace-fri-bad-with-tables-def **by** simp-all
from trace-bad **obtain** trace-table composition-table as query-idxs **where** bound:
 accepted-with-bound-tables s out trace-table composition-table as
 query-idxs
and not-low: $\neg$ trace-table-low-degree trace-table
unfolding trace-fri-bad-def **by** blast
from accepted-with-bound-tables-imp-accepted[OF bound]
obtain result final-state **where** out-eq: out = Some (result, final-state)
unfolding accepted-def **by** (cases out) auto
have verify-out:
 Some (result, final-state) $\in$ set-dist (execute verify-monad s)
using outcome out-eq **by** simp
from verify-monad-accepted-fri-opening-transcript[OF verify-out]
obtain trace-roots trace-bs trace-final dg composition-roots composition-bs
 composition-final query-idxs' trace-round-layers composition-round-layers

```

where openings:
  accepted-fri-opening-transcript s (Some (result, final-state))
  trace-roots trace-bs trace-final dg composition-roots composition-bs
  composition-final query-idxs' trace-round-layers
  composition-round-layers
by blast
have trace-bs-bad: trace-bs ∈ bad-sets trace-table
using cover with-tables bound openings not-low out-eq
unfolding trace-fri-with-tables-opening-algebraic-cover-def by blast
have challenges:
  accepted-fri-challenges s out trace-bs dg composition-bs
using accepted-fri-opening-transcript-challenges[OF openings] out-eq
by simp
show trace-fri-challenge-set-hit s bad-sets out
unfolding trace-fri-challenge-set-hit-def
using bound challenges not-low trace-bs-bad by blast
qed

lemma composition-fri-with-tables-opening-algebraic-cover-imp-bad-challenge-cover:
assumes cover:
  composition-fri-with-tables-opening-algebraic-cover s bad-sets
shows composition-fri-bad-with-tables-challenge-cover s bad-sets
unfolding composition-fri-bad-with-tables-challenge-cover-def
proof (intro ballI impI)
fix out :: (unit list × ('f, 'a) protocol-channel-scheme) option
assume outcome: out ∈ set-dist (execute verify-monad s)
and bad: composition-fri-bad-with-tables s out
then have with-tables: accepted-with-fri-tables s out
and composition-bad: composition-fri-bad s out
unfolding composition-fri-bad-with-tables-def by simp-all
from composition-bad obtain trace-table composition-table as query-idxs
where bound:
  accepted-with-bound-tables s out trace-table composition-table as
  query-idxs
and trace-low: trace-table-low-degree trace-table
and not-low: ¬ composition-table-low-degree maxDegree composition-table
unfolding composition-fri-bad-def by blast
from accepted-with-bound-tables-imp-accepted[OF bound]
obtain result final-state where out-eq: out = Some (result, final-state)
unfolding accepted-def by (cases out) auto
have verify-out:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
using outcome out-eq by simp
from verify-monad-accepted-fri-opening-transcript[OF verify-out]
obtain trace-roots trace-bs trace-final dg composition-roots composition-bs
  composition-final query-idxs' trace-round-layers composition-round-layers
where openings:
  accepted-fri-opening-transcript s (Some (result, final-state))
  trace-roots trace-bs trace-final dg composition-roots composition-bs

```

composition-final query-idxs' trace-round-layers
composition-round-layers
by *blast*
have *composition-bs-bad*:
composition-bs $\in$ *bad-sets* *dg composition-table*
using *cover with-tables bound openings trace-low not-low out-eq*
unfolding *composition-fri-with-tables-opening-algebraic-cover-def* **by** *blast*
have *challenges*:
accepted-fri-challenges s out trace-bs dg composition-bs
using *accepted-fri-opening-transcript-challenges[OF openings] out-eq*
by *simp*
show *composition-fri-challenge-set-hit s bad-sets out*
unfolding *composition-fri-challenge-set-hit-def*
using *bound challenges trace-low not-low composition-bs-bad* **by** *blast*
qed

lemma *wp-trace-fri-challenge-set-hit-bound-if-root-candidate-unique*:
assumes *future: trace-fri-future-fresh s*
and *unique*:
 $\bigwedge_{fr.} \exists \text{ trace-table.}$
 $\text{trace-fri-root-trace-table-candidates } s \text{ } fr \subseteq \{\text{trace-table}\}$
and *bounded: trace-fri-bad-challenge-sets-bounded bad-sets*
shows *wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s $\leq$*
trace-fri-error
proof –
have *union-bound*:
 $\bigwedge_{fr.} \text{nnreal}$
 $(\text{card } (\text{trace-fri-root-union-bad-sets } s \text{ } \text{bad-sets } fr)) /$
 $\text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } \text{clength}) \leq \text{trace-fri-error}$
proof –
fix *fr*
show *nnreal*
 $(\text{card } (\text{trace-fri-root-union-bad-sets } s \text{ } \text{bad-sets } fr)) /$
 $\text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } \text{clength}) \leq \text{trace-fri-error}$
by (*rule trace-fri-root-union-bad-sets-fraction-bound-if-unique-candidate*
[OF unique])
(use bounded in
 $\langle \text{auto simp: trace-fri-bad-challenge-sets-bounded-def} \rangle)$
qed
show *?thesis*
by (*rule wp-trace-fri-challenge-set-hit-bound-via-root-union*
[OF future union-bound])
qed

lemma *wp-trace-fri-challenge-set-hit-bound-if-supported-root-candidate-unique*:
assumes *future: trace-fri-future-fresh s*
and *unique*:
 $\bigwedge_{fr.}$

```

     $\exists$  trace-table.
    trace-fri-supported-root-trace-table-candidates s fr  $\subseteq$ 
      {trace-table}
    and bounded: trace-fri-bad-challenge-sets-bounded bad-sets
  shows wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s  $\leq$ 
    trace-fri-error
proof -
  have union-bound:
     $\bigwedge$ fr. nreal
      (card (trace-fri-supported-root-union-bad-sets s bad-sets fr)) /
      nreal (CARD('f)  $\wedge$  ceil-log clength)  $\leq$  trace-fri-error
  proof -
    fix fr
    show nreal
      (card (trace-fri-supported-root-union-bad-sets s bad-sets fr)) /
      nreal (CARD('f)  $\wedge$  ceil-log clength)  $\leq$  trace-fri-error
    by (rule
      trace-fri-supported-root-union-bad-sets-fraction-bound-if-unique-candidate
      [OF unique])
      (use bounded in
         $\langle$ auto simp: trace-fri-bad-challenge-sets-bounded-def $\rangle$ )
  qed
  show ?thesis
    by (rule wp-trace-fri-challenge-set-hit-bound-via-supported-root-union
      [OF future union-bound])
  qed

lemma wp-trace-fri-challenge-set-hit-bound-if-no-supported-pairwise-merkle-bad:
  assumes future: trace-fri-future-fresh s
  and no-bad:  $\neg$  trace-fri-supported-pairwise-merkle-bad s
  and bounded: trace-fri-bad-challenge-sets-bounded bad-sets
  shows wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s  $\leq$ 
    trace-fri-error
proof (rule wp-trace-fri-challenge-set-hit-bound-if-supported-root-candidate-unique
  [OF future - bounded])
  fix fr
  show
     $\exists$  trace-table.
    trace-fri-supported-root-trace-table-candidates s fr  $\subseteq$ 
      {trace-table}
  by (rule
    trace-fri-supported-root-candidate-unique-if-no-global-pairwise-merkle-bad
    [OF no-bad])
  qed

lemma wp-composition-fri-challenge-set-hit-bound-if-degree-candidate-unique:
  assumes future: composition-fri-future-fresh s
  and unique:
     $\bigwedge$ dg.

```

```

     $\exists$  composition-table.
    composition-fri-degree-table-candidates s dg  $\subseteq$ 
      {composition-table}
  and bounded: composition-fri-bad-challenge-sets-bounded bad-sets
shows
  wp-event verify-monad (composition-fri-challenge-set-hit s bad-sets) s  $\leq$ 
    composition-fri-error
proof -
  have union-bound:
     $\bigwedge dg. \text{nnreal}$ 
      (card (composition-fri-degree-union-bad-sets s bad-sets dg)) /
       $\text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq$ 
        composition-fri-error
  proof -
    fix dg
    show nnreal
      (card (composition-fri-degree-union-bad-sets s bad-sets dg)) /
       $\text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq$ 
        composition-fri-error
    by (rule
      composition-fri-degree-union-bad-sets-fraction-bound-if-unique-candidate
      [OF unique])
      (use bounded in
         $\langle \text{auto simp: composition-fri-bad-challenge-sets-bounded-def} \rangle$ )
  qed
  show ?thesis
    by (rule wp-composition-fri-challenge-set-hit-bound-via-degree-union
      [OF future union-bound])
  qed

lemma wp-composition-fri-challenge-set-hit-bound-if-supported-root-candidate-unique:
  assumes future: composition-fri-future-fresh s
  and unique:
     $\bigwedge fr \text{ f-fri-roots f-final as dg composition-fri-roots.}$ 
     $\exists$  composition-table.
      composition-fri-supported-root-composition-table-candidates s fr
      f-fri-roots f-final as dg composition-fri-roots  $\subseteq$ 
        {composition-table}
  and bounded: composition-fri-bad-challenge-sets-bounded bad-sets
shows
  wp-event verify-monad (composition-fri-challenge-set-hit s bad-sets) s  $\leq$ 
    composition-fri-error
proof -
  have union-bound:
     $\bigwedge fr \text{ f-fri-roots f-final as dg composition-fri-roots.}$ 
     $\text{nnreal}$ 
      (card
        (composition-fri-supported-root-union-bad-sets s bad-sets fr
          f-fri-roots f-final as dg composition-fri-roots)) /

```



```

    nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤
    composition-fri-error
proof -
  fix fr f-fri-roots f-final as dg composition-fri-roots
  show nnreal
    (card
      (composition-fri-supported-root-union-bad-sets s bad-sets fr
        f-fri-roots f-final as dg composition-fri-roots)) /
    nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤
    composition-fri-error
  by (rule
    composition-fri-supported-root-union-bad-sets-fraction-bound-if-unique-candidate
    [OF unique])
  (use bounded in
    ‹auto simp: composition-fri-bad-challenge-sets-bounded-def›)
qed
show ?thesis
  by (rule
    wp-composition-fri-challenge-set-hit-bound-via-supported-root-union
    [OF future union-bound])
qed

lemma wp-composition-fri-challenge-set-hit-bound-if-no-supported-pairwise-merkle-bad:
assumes future: composition-fri-future-fresh s
and no-bad: ¬ composition-fri-supported-pairwise-merkle-bad s
and bounded: composition-fri-bad-challenge-sets-bounded bad-sets
shows
  wp-event verify-monad (composition-fri-challenge-set-hit s bad-sets) s ≤
  composition-fri-error
proof (rule
  wp-composition-fri-challenge-set-hit-bound-if-supported-root-candidate-unique
  [OF future - bounded])
fix fr f-fri-roots f-final as dg composition-fri-roots
show
  ∃ composition-table.
    composition-fri-supported-root-composition-table-candidates s fr
    f-fri-roots f-final as dg composition-fri-roots ⊆
    {composition-table}
  by (rule
    composition-fri-supported-root-candidate-unique-if-no-global-pairwise-merkle-bad
    [OF no-bad])
qed

definition trace-fri-opening-algebraic-cover
  :: ('f, 'a) protocol-channel-scheme ⇒
    ('f list ⇒ 'f list set) ⇒ bool
where
  trace-fri-opening-algebraic-cover s bad-sets ⟷
    (∀ out trace-table composition-table as query-idxs opening-query-idxs

```

$\text{trace-roots trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final trace-round-layers composition-round-layers.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as query-idxs} \longrightarrow$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs trace-final}$
 $\text{dg composition-roots composition-bs composition-final opening-query-idxs}$
 $\text{trace-round-layers composition-round-layers} \longrightarrow$
 $\neg \text{trace-table-low-degree trace-table} \longrightarrow$
 $\text{trace-bs} \in \text{bad-sets trace-table})$

definition *composition-fri-opening-algebraic-cover*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow \text{bool}$

where

$\text{composition-fri-opening-algebraic-cover } s \text{ bad-sets} \longleftrightarrow$
 $(\forall \text{ out trace-table composition-table as query-idxs opening-query-idxs}$
 $\text{trace-roots trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final trace-round-layers composition-round-layers.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as query-idxs} \longrightarrow$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs trace-final}$
 $\text{dg composition-roots composition-bs composition-final opening-query-idxs}$
 $\text{trace-round-layers composition-round-layers} \longrightarrow$
 $\text{trace-table-low-degree trace-table} \longrightarrow$
 $\neg \text{composition-table-low-degree maxDegree composition-table} \longrightarrow$
 $\text{composition-bs} \in \text{bad-sets dg composition-table})$

definition *trace-fri-opening-reduction-assumption*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{trace-fri-opening-reduction-assumption } s \longleftrightarrow$
 $(\exists \text{ bad-sets.}$
 $\text{trace-fri-opening-algebraic-cover } s \text{ bad-sets} \wedge$
 $\text{trace-fri-bad-challenge-sets-bounded bad-sets})$

definition *composition-fri-opening-reduction-assumption*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{composition-fri-opening-reduction-assumption } s \longleftrightarrow$
 $(\exists \text{ bad-sets.}$
 $\text{composition-fri-opening-algebraic-cover } s \text{ bad-sets} \wedge$
 $\text{composition-fri-bad-challenge-sets-bounded bad-sets})$

lemma *trace-fri-opening-reductionI:*

assumes $\text{trace-fri-opening-algebraic-cover } s \text{ bad-sets}$

and $\text{trace-fri-bad-challenge-sets-bounded bad-sets}$

shows $\text{trace-fri-opening-reduction-assumption } s$

using *assms unfolding trace-fri-opening-reduction-assumption-def by blast*

lemma *trace-fri-opening-reductionE:*

assumes $\text{trace-fri-opening-reduction-assumption } s$

obtains *bad-sets*
where *trace-fri-opening-algebraic-cover s bad-sets*
and *trace-fri-bad-challenge-sets-bounded bad-sets*
using *assms* **unfolding** *trace-fri-opening-reduction-assumption-def* **by** *blast*

lemma *composition-fri-opening-reductionI:*
assumes *composition-fri-opening-algebraic-cover s bad-sets*
and *composition-fri-bad-challenge-sets-bounded bad-sets*
shows *composition-fri-opening-reduction-assumption s*
using *assms* **unfolding** *composition-fri-opening-reduction-assumption-def* **by** *blast*

lemma *composition-fri-opening-reductionE:*
assumes *composition-fri-opening-reduction-assumption s*
obtains *bad-sets*
where *composition-fri-opening-algebraic-cover s bad-sets*
and *composition-fri-bad-challenge-sets-bounded bad-sets*
using *assms* **unfolding** *composition-fri-opening-reduction-assumption-def*
by *blast*

lemma *trace-fri-multiround-bad-sets-bounded:*
assumes *frac:*
 $\bigwedge \text{trace-table.}$
 $\text{nnreal (card (trace-fri-multiround-bad-sets bad trace-table)) /}$
 $\text{nnreal (CARD('f) ^ ceil-log clength) } \leq \text{trace-fri-error}$
shows *trace-fri-bad-challenge-sets-bounded*
 $(\text{trace-fri-multiround-bad-sets bad})$
unfolding *trace-fri-bad-challenge-sets-bounded-def*
proof (*intro allI conjI*)
fix *trace-table*
show $\text{trace-fri-multiround-bad-sets bad trace-table } \subseteq$
 $\text{fri-challenge-space (ceil-log clength)}$
by (*rule trace-fri-multiround-bad-sets-subset*)
show $\text{nnreal (card (trace-fri-multiround-bad-sets bad trace-table)) /}$
 $\text{nnreal (CARD('f) ^ ceil-log clength) } \leq \text{trace-fri-error}$
by (*rule frac*)
qed

lemma *composition-fri-multiround-bad-sets-bounded:*
assumes *frac:*
 $\bigwedge dg \text{ composition-table.}$
 nnreal (card
 $(\text{composition-fri-multiround-bad-sets bad dg composition-table})) /$
 $\text{nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) } \leq$
 $\text{composition-fri-error}$
shows *composition-fri-bad-challenge-sets-bounded*
 $(\text{composition-fri-multiround-bad-sets bad})$
unfolding *composition-fri-bad-challenge-sets-bounded-def*
proof (*intro allI conjI*)
fix *dg composition-table*

```

show composition-fri-multiround-bad-sets bad dg composition-table  $\subseteq$ 
  fri-challenge-space (ceil-log (to-nat dg + 1))
by (rule composition-fri-multiround-bad-sets-subset)
show nnreal (card
  (composition-fri-multiround-bad-sets bad dg composition-table)) /
  nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤
  composition-fri-error
by (rule frac)
qed

```

```

lemma trace-fri-opening-reduction-from-multiround-bad-sets:
assumes cover:
  trace-fri-opening-algebraic-cover s
  (trace-fri-multiround-bad-sets bad)
and frac:
   $\bigwedge$ trace-table.
  nnreal (card (trace-fri-multiround-bad-sets bad trace-table)) /
  nnreal (CARD('f) ^ ceil-log clength) ≤ trace-fri-error
shows trace-fri-opening-reduction-assumption s
by (rule trace-fri-opening-reductionI
  [OF cover trace-fri-multiround-bad-sets-bounded[OF frac]])

```

```

lemma composition-fri-opening-reduction-from-multiround-bad-sets:
assumes cover:
  composition-fri-opening-algebraic-cover s
  (composition-fri-multiround-bad-sets bad)
and frac:
   $\bigwedge$ dg composition-table.
  nnreal (card
  (composition-fri-multiround-bad-sets bad dg composition-table)) /
  nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤
  composition-fri-error
shows composition-fri-opening-reduction-assumption s
by (rule composition-fri-opening-reductionI
  [OF cover composition-fri-multiround-bad-sets-bounded[OF frac]])

```

end

end

```

theory Soundness-Reductions-Opening
imports Soundness-Reductions-Partial
begin

```

FRI opening algebraic covers and final reduction bounds.

```

context soundness
begin

```

The multiround bad-set constructors above isolate the proof-only part of the FRI reduc-

tions: once an algebraic cover and the corresponding error-fraction bound are available, the existing opening-reduction assumptions follow directly. The current transcript predicate records authenticated local openings, but it does not bind complete intermediate FRI layer tables. Thus the algebraic cover is still an external FRI/table-binding theorem target unless the protocol model is extended with such tables or an equivalent binding interface.

definition *trace-fri-bad-challenge-cover*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow \text{bool}$

where

$\text{trace-fri-bad-challenge-cover } s \text{ bad-sets} \longleftrightarrow$
 $(\forall \text{ out} \in \text{set-dist } (\text{execute verify-monad } s).$
 $\text{trace-fri-bad } s \text{ out} \longrightarrow$
 $\text{trace-fri-challenge-set-hit } s \text{ bad-sets out})$

definition *composition-fri-bad-challenge-cover*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow \text{bool}$

where

$\text{composition-fri-bad-challenge-cover } s \text{ bad-sets} \longleftrightarrow$
 $(\forall \text{ out} \in \text{set-dist } (\text{execute verify-monad } s).$
 $\text{composition-fri-bad } s \text{ out} \longrightarrow$
 $\text{composition-fri-challenge-set-hit } s \text{ bad-sets out})$

lemma *trace-fri-opening-algebraic-cover-imp-bad-challenge-cover:*

assumes *cover:* $\text{trace-fri-opening-algebraic-cover } s \text{ bad-sets}$

shows $\text{trace-fri-bad-challenge-cover } s \text{ bad-sets}$

unfolding *trace-fri-bad-challenge-cover-def*

proof (*intro ballI impI*)

fix $\text{out} :: (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option}$

assume *outcome:* $\text{out} \in \text{set-dist } (\text{execute verify-monad } s)$

and *bad:* $\text{trace-fri-bad } s \text{ out}$

then obtain *trace-table composition-table as query-idxs where bound:*

$\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as query-idxs}$

and *not-low:* $\neg \text{trace-table-low-degree trace-table}$

unfolding *trace-fri-bad-def by blast*

from $\text{accepted-with-bound-tables-imp-accepted}[OF \text{ bound}]$

obtain *result final-state where out-eq:* $\text{out} = \text{Some } (\text{result}, \text{final-state})$

unfolding *accepted-def by (cases out) auto*

have *verify-out:*

$\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute verify-monad } s)$

using *outcome out-eq by simp*

from $\text{verify-monad-accepted-fri-opening-transcript}[OF \text{ verify-out}]$

obtain *trace-roots trace-bs trace-final dg composition-roots composition-bs*

$\text{composition-final query-idxs'} \text{ trace-round-layers composition-round-layers}$

where *openings:*

$\text{accepted-fri-opening-transcript } s (\text{Some } (\text{result}, \text{final-state}))$

$\text{trace-roots trace-bs trace-final dg composition-roots composition-bs}$

composition-final query-idxs' trace-round-layers composition-round-layers
by *blast*
have *trace-bs-bad: trace-bs ∈ bad-sets trace-table*
using *cover bound openings not-low out-eq*
unfolding *trace-fri-opening-algebraic-cover-def* **by** *blast*
have *challenges:*
accepted-fri-challenges s out trace-bs dg composition-bs
using *accepted-fri-opening-transcript-challenges[OF openings] out-eq* **by** *simp*
show *trace-fri-challenge-set-hit s bad-sets out*
unfolding *trace-fri-challenge-set-hit-def*
using *bound challenges not-low trace-bs-bad* **by** *blast*
qed

lemma *composition-fri-opening-algebraic-cover-imp-bad-challenge-cover:*
assumes *cover: composition-fri-opening-algebraic-cover s bad-sets*
shows *composition-fri-bad-challenge-cover s bad-sets*
unfolding *composition-fri-bad-challenge-cover-def*
proof (*intro ballI impI*)
fix *out :: (unit list × ('f, 'a) protocol-channel-scheme) option*
assume *outcome: out ∈ set-dist (execute verify-monad s)*
and *bad: composition-fri-bad s out*
then obtain *trace-table composition-table as query-idxs* **where** *bound:*
accepted-with-bound-tables s out trace-table composition-table as query-idxs
and *trace-low: trace-table-low-degree trace-table*
and *not-low: ¬ composition-table-low-degree maxDegree composition-table*
unfolding *composition-fri-bad-def* **by** *blast*
from *accepted-with-bound-tables-imp-accepted[OF bound]*
obtain *result final-state* **where** *out-eq: out = Some (result, final-state)*
unfolding *accepted-def* **by** (*cases out*) *auto*
have *verify-out:*
Some (result, final-state) ∈ set-dist (execute verify-monad s)
using *outcome out-eq* **by** *simp*
from *verify-monad-accepted-fri-opening-transcript[OF verify-out]*
obtain *trace-roots trace-bs trace-final dg composition-roots composition-bs*
composition-final query-idxs' trace-round-layers composition-round-layers
where *openings:*
accepted-fri-opening-transcript s (Some (result, final-state))
trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final query-idxs' trace-round-layers composition-round-layers
by *blast*
have *composition-bs-bad: composition-bs ∈ bad-sets dg composition-table*
using *cover bound openings trace-low not-low out-eq*
unfolding *composition-fri-opening-algebraic-cover-def* **by** *blast*
have *challenges:*
accepted-fri-challenges s out trace-bs dg composition-bs
using *accepted-fri-opening-transcript-challenges[OF openings] out-eq* **by** *simp*
show *composition-fri-challenge-set-hit s bad-sets out*
unfolding *composition-fri-challenge-set-hit-def*
using *bound challenges trace-low not-low composition-bs-bad* **by** *blast*

qed

definition *trace-fri-challenge-freshness-assumption*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{trace-fri-challenge-freshness-assumption } s \longleftrightarrow$
 $(\forall \text{ bad-sets.}$
 $\text{trace-fri-bad-challenge-sets-bounded bad-sets} \longrightarrow$
 $\text{wp-event verify-monad } (\text{trace-fri-challenge-set-hit } s \text{ bad-sets}) \text{ } s \leq$
 $\text{trace-fri-error})$

definition *composition-fri-challenge-freshness-assumption*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{composition-fri-challenge-freshness-assumption } s \longleftrightarrow$
 $(\forall \text{ bad-sets.}$
 $\text{composition-fri-bad-challenge-sets-bounded bad-sets} \longrightarrow$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-challenge-set-hit } s \text{ bad-sets}) \text{ } s \leq$
 $\text{composition-fri-error})$

definition *random-oracle-freshness-assumption*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{random-oracle-freshness-assumption } s \longleftrightarrow$
 $\alpha\text{-challenge-freshness-assumption } s \wedge$
 $\text{query-index-freshness-assumption } s \wedge$
 $\text{trace-fri-challenge-freshness-assumption } s \wedge$
 $\text{composition-fri-challenge-freshness-assumption } s$

lemma *random-oracle-freshness-alpha:*

assumes *random-oracle-freshness-assumption s*

shows *alpha-challenge-freshness-assumption s*

using *assms unfolding random-oracle-freshness-assumption-def by simp*

lemma *random-oracle-freshness-query:*

assumes *random-oracle-freshness-assumption s*

shows *query-index-freshness-assumption s*

using *assms unfolding random-oracle-freshness-assumption-def by simp*

lemma *random-oracle-freshness-trace-fri:*

assumes *random-oracle-freshness-assumption s*

shows *trace-fri-challenge-freshness-assumption s*

using *assms unfolding random-oracle-freshness-assumption-def by simp*

lemma *random-oracle-freshness-composition-fri:*

assumes *random-oracle-freshness-assumption s*

shows *composition-fri-challenge-freshness-assumption s*

using *assms unfolding random-oracle-freshness-assumption-def by simp*

lemma *alpha-challenge-freshness-if-no-supported-pairwise-merkle-bad*:
assumes *future: alpha-future-fresh s*
and *no-bad: $\neg$ alpha-supported-pairwise-merkle-bad s*
shows *alpha-challenge-freshness-assumption s*
unfolding *alpha-challenge-freshness-assumption-def*
proof (*intro allI impI*)
fix *bad-sets :: 'f list $\Rightarrow$ 'f list set*
assume *subset*:
 $\forall$ *trace-table. bad-sets trace-table $\subseteq$ alpha-space*
and *bounded*:
 $\forall$ *trace-table.*
 $nnreal (card (bad-sets trace-table)) /$
 $nnreal (card alpha-space) \leq composition-error-bound$
show *wp-event verify-monad (alpha-bad-set-hit s bad-sets) s $\leq$*
composition-error-bound
proof (*rule wp-alpha-bad-set-hit-bound-via-supported-header-union*
 $[OF future]$)
fix *trace-table*
show *bad-sets trace-table $\subseteq$ alpha-space*
using *subset by blast*
next
fix *fr f-fri-roots f-final*
have *unique*:
 $\exists$ *trace-table.*
alpha-header-supported-trace-table-candidates s fr f-fri-roots
f-final $\subseteq$ {trace-table}
by (*rule alpha-header-supported-candidate-unique-if-no-global-pairwise-merkle-bad*
 $[OF no-bad]$)
show
 $nnreal$
 $(card$
 $(alpha-header-supported-union-bad-sets s bad-sets fr f-fri-roots$
 $f-final)) /$
 $nnreal (card alpha-space) \leq composition-error-bound$
by (*rule alpha-header-supported-union-bad-sets-fraction-bound-if-unique-candidate*
 $[OF unique]$)
(use subset bounded in blast)+
qed
qed

lemma *trace-fri-challenge-freshness-if-no-supported-pairwise-merkle-bad*:
assumes *future: trace-fri-future-fresh s*
and *no-bad: $\neg$ trace-fri-supported-pairwise-merkle-bad s*
shows *trace-fri-challenge-freshness-assumption s*
unfolding *trace-fri-challenge-freshness-assumption-def*
proof (*intro allI impI*)
fix *bad-sets :: 'f list $\Rightarrow$ 'f list set*
assume *bounded: trace-fri-bad-challenge-sets-bounded bad-sets*

show *wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s ≤ trace-fri-error*
by (rule
wp-trace-fri-challenge-set-hit-bound-if-no-supported-pairwise-merkle-bad
[OF future no-bad bounded])
qed

lemma *composition-fri-challenge-freshness-if-no-supported-pairwise-merkle-bad:*
assumes *future: composition-fri-future-fresh s*
and *no-bad: ¬ composition-fri-supported-pairwise-merkle-bad s*
shows *composition-fri-challenge-freshness-assumption s*
unfolding *composition-fri-challenge-freshness-assumption-def*
proof (intro allI impI)
fix *bad-sets :: 'f ⇒ 'f list ⇒ 'f list set*
assume *bounded: composition-fri-bad-challenge-sets-bounded bad-sets*
show
wp-event verify-monad
(composition-fri-challenge-set-hit s bad-sets) s ≤
composition-fri-error
by (rule
wp-composition-fri-challenge-set-hit-bound-if-no-supported-pairwise-merkle-bad
[OF future no-bad bounded])
qed

lemma *verifier-initial-alpha-challenge-freshness-if-no-supported-pairwise-merkle-bad:*
assumes *no-bad:*
¬ alpha-supported-pairwise-merkle-bad (verifier-initial-state tr)
shows *alpha-challenge-freshness-assumption (verifier-initial-state tr)*
by (rule *alpha-challenge-freshness-if-no-supported-pairwise-merkle-bad*)
(use no-bad in simp-all)

lemma *verifier-initial-trace-fri-challenge-freshness-if-no-supported-pairwise-merkle-bad:*
assumes *no-bad:*
¬ trace-fri-supported-pairwise-merkle-bad (verifier-initial-state tr)
shows
trace-fri-challenge-freshness-assumption (verifier-initial-state tr)
by (rule *trace-fri-challenge-freshness-if-no-supported-pairwise-merkle-bad*)
(use no-bad in simp-all)

lemma *verifier-initial-composition-fri-challenge-freshness-if-no-supported-pairwise-merkle-bad:*
assumes *no-bad:*
¬ composition-fri-supported-pairwise-merkle-bad
(verifier-initial-state tr)
shows
composition-fri-challenge-freshness-assumption
(verifier-initial-state tr)
by (rule
composition-fri-challenge-freshness-if-no-supported-pairwise-merkle-bad)
(use no-bad in simp-all)

lemma *alpha-challenge-freshness-if-no-pairwise-side-bad*:
assumes *future: alpha-future-fresh s*
and *no-collision: $\neg$ supported-hash-output-collision-possible s*
and *no-coupling: $\neg$ supported-pairwise-coupling-bad s*
shows *alpha-challenge-freshness-assumption s*
proof (rule *alpha-challenge-freshness-if-no-supported-pairwise-merkle-bad*
[OF *future*])
show $\neg$ *alpha-supported-pairwise-merkle-bad s*
by (rule *no-alpha-supported-pairwise-merkle-bad-if-no-pairwise-side-bad*
[OF *no-collision no-coupling*])
qed

lemma *query-index-freshness-if-no-pairwise-side-bad*:
assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *no-collision: $\neg$ supported-hash-output-collision-possible s*
and *no-coupling: $\neg$ supported-pairwise-coupling-bad s*
shows *query-index-freshness-assumption s*
proof (rule *query-index-freshness-if-no-query-supported-pairwise-merkle-bad*
[OF *future raw-bound*])
show $\neg$ *query-supported-pairwise-merkle-bad s*
by (rule *no-query-supported-pairwise-merkle-bad-if-no-pairwise-side-bad*
[OF *no-collision no-coupling*])
qed

lemma *trace-fri-challenge-freshness-if-no-pairwise-side-bad*:
assumes *future: trace-fri-future-fresh s*
and *no-collision: $\neg$ supported-hash-output-collision-possible s*
and *no-coupling: $\neg$ supported-pairwise-coupling-bad s*
shows *trace-fri-challenge-freshness-assumption s*
proof (rule *trace-fri-challenge-freshness-if-no-supported-pairwise-merkle-bad*
[OF *future*])
show $\neg$ *trace-fri-supported-pairwise-merkle-bad s*
by (rule
no-trace-fri-supported-pairwise-merkle-bad-if-no-pairwise-side-bad
[OF *no-collision no-coupling*])
qed

lemma *composition-fri-challenge-freshness-if-no-pairwise-side-bad*:
assumes *future: composition-fri-future-fresh s*
and *no-collision: $\neg$ supported-hash-output-collision-possible s*
and *no-coupling: $\neg$ supported-pairwise-coupling-bad s*
shows *composition-fri-challenge-freshness-assumption s*
proof (rule
composition-fri-challenge-freshness-if-no-supported-pairwise-merkle-bad
[OF *future*])
show $\neg$ *composition-fri-supported-pairwise-merkle-bad s*
by (rule

no-composition-fri-supported-pairwise-merkle-bad-if-no-pairwise-side-bad
 $[OF \text{ no-collision no-coupling}]$)

qed

lemma *random-oracle-freshness-if-no-pairwise-side-bad:*

assumes *alpha-future: alpha-future-fresh s*
and *query-future: query-future-fresh s*
and *trace-future: trace-fri-future-fresh s*
and *composition-future: composition-fri-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *no-collision: $\neg$ supported-hash-output-collision-possible s*
and *no-coupling: $\neg$ supported-pairwise-coupling-bad s*

shows *random-oracle-freshness-assumption s*

unfolding *random-oracle-freshness-assumption-def*

proof (*intro conjI*)

show *alpha-challenge-freshness-assumption s*

by (*rule alpha-challenge-freshness-if-no-pairwise-side-bad*
 $[OF \text{ alpha-future no-collision no-coupling}]$)

show *query-index-freshness-assumption s*

by (*rule query-index-freshness-if-no-pairwise-side-bad*
 $[OF \text{ query-future raw-bound no-collision no-coupling}]$)

show *trace-fri-challenge-freshness-assumption s*

by (*rule trace-fri-challenge-freshness-if-no-pairwise-side-bad*
 $[OF \text{ trace-future no-collision no-coupling}]$)

show *composition-fri-challenge-freshness-assumption s*

by (*rule composition-fri-challenge-freshness-if-no-pairwise-side-bad*
 $[OF \text{ composition-future no-collision no-coupling}]$)

qed

lemma *query-bad-bound-if-no-pairwise-side-bad:*

assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *no-collision: $\neg$ supported-hash-output-collision-possible s*
and *no-coupling: $\neg$ supported-pairwise-coupling-bad s*
shows *wp-event verify-monad (query-bad s) s $\leq$*
*nnreal rounds * query-error-bound*

proof –

have *query-fresh: query-index-freshness-assumption s*

by (*rule query-index-freshness-if-no-pairwise-side-bad*
 $[OF \text{ future raw-bound no-collision no-coupling}]$)

show *?thesis*

by (*rule query-bad-bound-from-phase4B* $[OF \text{ query-fresh}]$)

qed

lemma *query-bad-bound-if-no-pairwise-side-event-on-support:*

assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *no-side:*

$\bigwedge \text{out. out} \in \text{set-dist (execute verify-monad s)} \implies$

$\neg \text{supported-pairwise-side-bad } s \text{ out}$
shows $\text{wp-event verify-monad (query-bad } s) s \leq$
 $\text{nnreal rounds} * \text{query-error-bound}$
proof –
have $\text{no-collision: } \neg \text{supported-hash-output-collision-possible } s$
by (rule
 $\text{no-supported-pairwise-side-bad-on-support-imp-no-hash-collision-possible}$
 $[\text{OF no-side}]$)
have $\text{no-coupling: } \neg \text{supported-pairwise-coupling-bad } s$
by (rule $\text{no-supported-pairwise-side-bad-on-support-imp-no-coupling-bad}$
 $[\text{OF no-side}]$)
show ?thesis
by (rule $\text{query-bad-bound-if-no-pairwise-side-bad}$
 $[\text{OF future raw-bound no-collision no-coupling}]$)
qed

lemma $\text{query-bad-bound-if-no-pairwise-side-event-on-support-from-sampler-wellformed:}$
assumes $\text{future: query-future-fresh } s$
and no-side:
 $\bigwedge \text{out. out} \in \text{set-dist (execute verify-monad } s) \implies$
 $\neg \text{supported-pairwise-side-bad } s \text{ out}$
shows $\text{wp-event verify-monad (query-bad } s) s \leq$
 $\text{nnreal rounds} * \text{query-error-bound}$
by (rule $\text{query-bad-bound-if-no-pairwise-side-event-on-support}$
 $[\text{OF future query-index-raw-preimage-bound-from-sampler-wellformed}$
 $\text{no-side}]$)

lemma $\text{composition-bad-bound-if-no-pairwise-side-bad:}$
assumes $\text{wf: spec-degree-wellformed}$
and $\text{future: alpha-future-fresh } s$
and $\text{no-collision: } \neg \text{supported-hash-output-collision-possible } s$
and $\text{no-coupling: } \neg \text{supported-pairwise-coupling-bad } s$
shows $\text{wp-event verify-monad (composition-bad } s) s \leq$
 $\text{composition-error-bound}$
proof –
have $\text{alpha-fresh: alpha-challenge-freshness-assumption } s$
by (rule $\text{alpha-challenge-freshness-if-no-pairwise-side-bad}$
 $[\text{OF future no-collision no-coupling}]$)
show ?thesis
by (rule $\text{composition-bad-bound-from-phase4A}[\text{OF wf alpha-fresh}]$)
qed

lemma $\text{composition-bad-bound-if-no-pairwise-side-event-on-support:}$
assumes $\text{wf: spec-degree-wellformed}$
and $\text{future: alpha-future-fresh } s$
and no-side:
 $\bigwedge \text{out. out} \in \text{set-dist (execute verify-monad } s) \implies$
 $\neg \text{supported-pairwise-side-bad } s \text{ out}$
shows $\text{wp-event verify-monad (composition-bad } s) s \leq$

composition-error-bound

proof —

have *no-collision*: $\neg$ *supported-hash-output-collision-possible* *s*

by (rule *no-supported-pairwise-side-bad-on-support-imp-no-hash-collision-possible* [OF *no-side*])

have *no-coupling*: $\neg$ *supported-pairwise-coupling-bad* *s*

by (rule *no-supported-pairwise-side-bad-on-support-imp-no-coupling-bad* [OF *no-side*])

show *?thesis*

by (rule *composition-bad-bound-if-no-pairwise-side-bad* [OF *wf future no-collision no-coupling*])

qed

lemma *verifier-initial-random-oracle-freshness-if-uniform-sampler-and-no-pairwise-side-bad*:

— The range premise states that *to-nat* enumerates exactly the field indices used by the query sampler; the divisibility premise makes the modulo/index projection uniform over *query-sample-space*.

assumes *range-eq*: *range to-nat* = $\{0..<size\}$

and *dvd*: *query-sample-space-size dvd size*

and *no-collision*:

$\neg$ *supported-hash-output-collision-possible* (*verifier-initial-state tr*)

and *no-coupling*:

$\neg$ *supported-pairwise-coupling-bad* (*verifier-initial-state tr*)

shows *random-oracle-freshness-assumption* (*verifier-initial-state tr*)

proof —

have *raw-bound*: *query-index-raw-preimage-bound*

by (rule *query-index-raw-preimage-bound-if-uniform-range* [OF *range-eq dvd*])

show *?thesis*

by (rule *random-oracle-freshness-if-no-pairwise-side-bad* [OF - - - *raw-bound no-collision no-coupling*])

simp-all

qed

The remaining soundness interfaces have the following status without changing the protocol. The predicate *initial-merkle-binding-no-bad* is a cryptographic/model boundary excluding accepted executions whose local Merkle openings cannot be extended to full committed tables. The predicate *random-oracle-freshness-assumption* is a random-oracle model boundary: its component projections are deterministic, but deriving freshness itself would require assumptions about the hash oracle, and deriving domain separation internally would require protocol labels that are not present in the current formalization. The two FRI reduction interfaces are proof obligations rather than protocol changes. Each packages an opening-transcript algebraic cover with a bounded bad-challenge-set statement. The lemmas below derive the verifier bad-event bounds directly from those opening-based interfaces.

lemma *trace-fri-bad-bound-from-opening-reduction*:

assumes *reduction*: *trace-fri-opening-reduction-assumption s*

and *fresh*: *trace-fri-challenge-freshness-assumption s*

shows *wp-event verify-monad (trace-fri-bad s) s ≤ trace-fri-error*
proof –
from *reduction* **obtain** *bad-sets* **where** *opening-cover*:
 trace-fri-opening-algebraic-cover s bad-sets
and *bounded*:
 trace-fri-bad-challenge-sets-bounded bad-sets
by (rule *trace-fri-opening-reductionE*)
have *cover*: *trace-fri-bad-challenge-cover s bad-sets*
by (rule *trace-fri-opening-algebraic-cover-imp-bad-challenge-cover*
 [*OF opening-cover*])
have *wp-event verify-monad (trace-fri-bad s) s ≤*
 wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s
by (rule *wp-event-mono-on-support*)
 (*use cover in <auto simp: trace-fri-bad-challenge-cover-def>*)
also have ... *≤ trace-fri-error*
 using *fresh bounded*
 unfolding *trace-fri-challenge-freshness-assumption-def* **by** *blast*
finally show *?thesis* .
qed

lemma *composition-fri-bad-bound-from-opening-reduction*:
assumes *reduction*: *composition-fri-opening-reduction-assumption s*
and *fresh*: *composition-fri-challenge-freshness-assumption s*
shows *wp-event verify-monad (composition-fri-bad s) s ≤*
 composition-fri-error
proof –
from *reduction* **obtain** *bad-sets* **where** *opening-cover*:
 composition-fri-opening-algebraic-cover s bad-sets
and *bounded*:
 composition-fri-bad-challenge-sets-bounded bad-sets
by (rule *composition-fri-opening-reductionE*)
have *cover*: *composition-fri-bad-challenge-cover s bad-sets*
by (rule *composition-fri-opening-algebraic-cover-imp-bad-challenge-cover*
 [*OF opening-cover*])
have *wp-event verify-monad (composition-fri-bad s) s ≤*
 wp-event verify-monad
 (*composition-fri-challenge-set-hit s bad-sets*) *s*
by (rule *wp-event-mono-on-support*)
 (*use cover in <auto simp: composition-fri-bad-challenge-cover-def>*)
also have ... *≤ composition-fri-error*
 using *fresh bounded*
 unfolding *composition-fri-challenge-freshness-assumption-def* **by** *blast*
finally show *?thesis* .
qed

lemma *trace-fri-bad-with-partial-openings-bound-from-opening-reduction*:
assumes *reduction*: *trace-fri-opening-reduction-assumption s*
and *fresh*: *trace-fri-challenge-freshness-assumption s*
shows *wp-event verify-monad (trace-fri-bad-with-partial-openings s) s ≤*

trace-fri-error
proof –
 have $\text{wp-event verify-monad } (\text{trace-fri-bad-with-partial-openings } s) \ s \leq$
 $\text{wp-event verify-monad } (\text{trace-fri-bad } s) \ s$
 by (rule $\text{wp-trace-fri-bad-with-partial-openings-le-trace-fri-bad}$)
 also have $\dots \leq \text{trace-fri-error}$
 by (rule $\text{trace-fri-bad-bound-from-opening-reduction}$
 $[\text{OF reduction fresh}]$)
 finally show ?thesis .
qed

lemma $\text{composition-fri-bad-with-partial-openings-bound-from-opening-reduction}$:
 assumes $\text{reduction: composition-fri-opening-reduction-assumption } s$
 and $\text{fresh: composition-fri-challenge-freshness-assumption } s$
 shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-partial-openings } s) \ s \leq$
 $\text{composition-fri-error}$
proof –
 have $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-partial-openings } s) \ s \leq$
 $\text{wp-event verify-monad } (\text{composition-fri-bad } s) \ s$
 by (rule
 $\text{wp-composition-fri-bad-with-partial-openings-le-composition-fri-bad}$)
 also have $\dots \leq \text{composition-fri-error}$
 by (rule $\text{composition-fri-bad-bound-from-opening-reduction}$
 $[\text{OF reduction fresh}]$)
 finally show ?thesis .
qed

lemma $\text{trace-fri-with-tables-bad-sets-bounded-if-error-ge-one}$:
 assumes $\text{error-ge-one: } 1 \leq \text{trace-fri-error}$
 shows $\text{trace-fri-bad-challenge-sets-bounded}$
 $(\text{trace-fri-with-tables-bad-sets } s)$
 unfolding $\text{trace-fri-bad-challenge-sets-bounded-def}$
proof (intro allI conjI)
 fix trace-table
 show $\text{trace-fri-with-tables-bad-sets } s \ \text{trace-table} \subseteq$
 $\text{fri-challenge-space } (\text{ceil-log clength})$
 by (rule $\text{trace-fri-with-tables-bad-sets-subset}$)
 show $\text{nnreal } (\text{card } (\text{trace-fri-with-tables-bad-sets } s \ \text{trace-table})) /$
 $\text{nnreal } (\text{CARD('f') } ^{\text{ceil-log clength}}) \leq \text{trace-fri-error}$
 by (rule order-trans
 $[\text{OF trace-fri-with-tables-bad-sets-fraction-le-one error-ge-one}]$)
qed

lemma $\text{composition-fri-with-tables-bad-sets-bounded-if-error-ge-one}$:
 assumes $\text{error-ge-one: } 1 \leq \text{composition-fri-error}$
 shows $\text{composition-fri-bad-challenge-sets-bounded}$

```

    (composition-fri-with-tables-bad-sets s)
  unfolding composition-fri-bad-challenge-sets-bounded-def
proof (intro allI conjI)
  fix dg composition-table
  show composition-fri-with-tables-bad-sets s dg composition-table  $\subseteq$ 
    fri-challenge-space (ceil-log (to-nat dg + 1))
  by (rule composition-fri-with-tables-bad-sets-subset)
  show nnreal
    (card (composition-fri-with-tables-bad-sets s dg composition-table)) /
    nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))  $\leq$ 
    composition-fri-error
  by (rule order-trans
    [OF composition-fri-with-tables-bad-sets-fraction-le-one error-ge-one])
qed

lemma trace-fri-bad-with-tables-bound-from-cover:
  assumes cover:
    trace-fri-with-tables-opening-algebraic-cover s bad-sets
  and bounded: trace-fri-bad-challenge-sets-bounded bad-sets
  and fresh: trace-fri-challenge-freshness-assumption s
  shows wp-event verify-monad (trace-fri-bad-with-tables s) s  $\leq$ 
    trace-fri-error
proof -
  have challenge-cover:
    trace-fri-bad-with-tables-challenge-cover s bad-sets
  by (rule trace-fri-with-tables-opening-algebraic-cover-imp-bad-challenge-cover
    [OF cover])
  have wp-event verify-monad (trace-fri-bad-with-tables s) s  $\leq$ 
    wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s
  by (rule wp-event-mono-on-support)
  (use challenge-cover in
    ⟨auto simp: trace-fri-bad-with-tables-challenge-cover-def⟩)
  also have ...  $\leq$  trace-fri-error
  using fresh bounded
  unfolding trace-fri-challenge-freshness-assumption-def by blast
  finally show ?thesis .
qed

lemma composition-fri-bad-with-tables-bound-from-cover:
  assumes cover:
    composition-fri-with-tables-opening-algebraic-cover s bad-sets
  and bounded: composition-fri-bad-challenge-sets-bounded bad-sets
  and fresh: composition-fri-challenge-freshness-assumption s
  shows wp-event verify-monad (composition-fri-bad-with-tables s) s  $\leq$ 
    composition-fri-error
proof -
  have challenge-cover:
    composition-fri-bad-with-tables-challenge-cover s bad-sets
  by (rule

```


composition-fri-with-tables-opening-algebraic-cover-imp-bad-challenge-cover
 [OF cover])
have *wp-event verify-monad* (*composition-fri-bad-with-tables* *s*) *s* ≤
wp-event verify-monad
 (*composition-fri-challenge-set-hit* *s* *bad-sets*) *s*
by (*rule wp-event-mono-on-support*)
 (*use challenge-cover in*
 ⟨*auto simp: composition-fri-bad-with-tables-challenge-cover-def*⟩)
also have ... ≤ *composition-fri-error*
using *fresh bounded*
unfolding *composition-fri-challenge-freshness-assumption-def* **by** *blast*
finally show ?thesis .
qed

definition *trace-fri-with-tables-opening-reduction-assumption*
 :: (*f*, *a*) *protocol-channel-scheme* ⇒ *bool*
where
trace-fri-with-tables-opening-reduction-assumption *s* ⇔
 (∃ *bad-sets*.
trace-fri-with-tables-opening-algebraic-cover *s* *bad-sets* ∧
trace-fri-bad-challenge-sets-bounded *bad-sets*)

definition *composition-fri-with-tables-opening-reduction-assumption*
 :: (*f*, *a*) *protocol-channel-scheme* ⇒ *bool*
where
composition-fri-with-tables-opening-reduction-assumption *s* ⇔
 (∃ *bad-sets*.
composition-fri-with-tables-opening-algebraic-cover *s* *bad-sets* ∧
composition-fri-bad-challenge-sets-bounded *bad-sets*)

lemma *trace-fri-with-tables-opening-reductionE*:
assumes *trace-fri-with-tables-opening-reduction-assumption* *s*
obtains *bad-sets*
where *trace-fri-with-tables-opening-algebraic-cover* *s* *bad-sets*
and *trace-fri-bad-challenge-sets-bounded* *bad-sets*
using *assms*
unfolding *trace-fri-with-tables-opening-reduction-assumption-def* **by** *blast*

lemma *composition-fri-with-tables-opening-reductionE*:
assumes *composition-fri-with-tables-opening-reduction-assumption* *s*
obtains *bad-sets*
where *composition-fri-with-tables-opening-algebraic-cover* *s* *bad-sets*
and *composition-fri-bad-challenge-sets-bounded* *bad-sets*
using *assms*
unfolding *composition-fri-with-tables-opening-reduction-assumption-def*
by *blast*

lemma *trace-fri-bad-with-tables-bound-from-opening-reduction*:
assumes *reduction*:

```

    trace-fri-with-tables-opening-reduction-assumption s
  and fresh: trace-fri-challenge-freshness-assumption s
shows wp-event verify-monad (trace-fri-bad-with-tables s) s ≤
  trace-fri-error
proof -
  from reduction obtain bad-sets where cover:
    trace-fri-with-tables-opening-algebraic-cover s bad-sets
  and bounded: trace-fri-bad-challenge-sets-bounded bad-sets
  by (rule trace-fri-with-tables-opening-reductionE)
  show ?thesis
  by (rule trace-fri-bad-with-tables-bound-from-cover
    [OF cover bounded fresh])
qed

lemma composition-fri-bad-with-tables-bound-from-opening-reduction:
  assumes reduction:
    composition-fri-with-tables-opening-reduction-assumption s
  and fresh: composition-fri-challenge-freshness-assumption s
shows wp-event verify-monad (composition-fri-bad-with-tables s) s ≤
  composition-fri-error
proof -
  from reduction obtain bad-sets where cover:
    composition-fri-with-tables-opening-algebraic-cover s bad-sets
  and bounded: composition-fri-bad-challenge-sets-bounded bad-sets
  by (rule composition-fri-with-tables-opening-reductionE)
  show ?thesis
  by (rule composition-fri-bad-with-tables-bound-from-cover
    [OF cover bounded fresh])
qed

end

end

```

```

theory Soundness-Reductions
  imports Soundness-Reductions-Opening
begin

```

Compatibility umbrella for soundness reduction lemmas.

```

end

```

```

theory Soundness-Staged-Query-Prefix
  imports Soundness-Reductions Staged-Security-Experiment
begin

```

Staged query-prefix replay and prefix receive decomposition lemmas.

```

context soundness

```

begin

The deterministic partition above reduces soundness to four probability bounds. The bounds themselves require additional cryptographic/algebraic interfaces: random-linear-combination soundness, query-sampling freshness, and FRI low-degree soundness. Those assumptions are kept in the separate bounded-soundness locale below, rather than hidden as unproved lemmas in this base locale.

lemma *soundness-bound-component-accounting*:

soundness-bound =
 $\text{composition-error-bound} + \text{trace-fri-error} + \text{composition-fri-error} +$
 $\text{nnreal rounds} * \text{query-error-bound}$
unfolding *soundness-bound-def* **by** *simp*

lemma *soundness-bound-with-merkle-component-accounting*:

soundness-bound-with-merkle =
 $\text{merkle-binding-error} + \text{composition-error-bound} + \text{trace-fri-error} +$
 $\text{composition-fri-error} + \text{nnreal rounds} * \text{query-error-bound}$
unfolding *soundness-bound-with-merkle-def* *soundness-bound-def*
by (*simp add: algebra-simps*)

lemma *wp-event-finite-union-bound*:

fixes $B :: 'i \Rightarrow \text{prob}$
assumes *finite*: *finite I*
and *bounds*: $\bigwedge i. i \in I \implies \text{wp-event } m (E \ i) \ s \leq B \ i$
shows
 $\text{wp-event } m (\lambda \text{out}. \exists i \in I. E \ i \ \text{out}) \ s \leq (\sum i \in I. B \ i)$
using *finite bounds*
proof (*induction I*)
case *empty*
show *?case*
unfolding *wp-event-def wp-def dist-expect-def* **by** *simp*
next
case (*insert i I*)
have *split*:
 $\text{wp-event } m (\lambda \text{out}. \exists j \in \text{insert } i \ I. E \ j \ \text{out}) \ s \leq$
 $\text{wp-event } m (E \ i) \ s +$
 $\text{wp-event } m (\lambda \text{out}. \exists j \in I. E \ j \ \text{out}) \ s$
proof –
have $\text{wp-event } m (\lambda \text{out}. \exists j \in \text{insert } i \ I. E \ j \ \text{out}) \ s \leq$
 $\text{wp-event } m (\lambda \text{out}. E \ i \ \text{out} \vee (\exists j \in I. E \ j \ \text{out})) \ s$
by (*rule wp-event-mono*) *auto*
also have $\dots \leq$
 $\text{wp-event } m (E \ i) \ s +$
 $\text{wp-event } m (\lambda \text{out}. \exists j \in I. E \ j \ \text{out}) \ s$
by (*rule wp-event-union-bound*)
finally show *?thesis* .
qed
have *tail*:
 $\text{wp-event } m (\lambda \text{out}. \exists j \in I. E \ j \ \text{out}) \ s \leq (\sum j \in I. B \ j)$

by (rule insert.IH) (use insert.premis in blast)
 have wp-event m (E i) s +
 wp-event m ($\lambda out. \exists j \in I. E$ j out) $s \leq$
 B i + ($\sum j \in I. B$ j)
 by (intro add-mono tail insert.premis) simp
 also have ... = ($\sum j \in insert\ i\ I. B$ j)
 using insert.hyps by simp
 finally show ?case
 by (rule order-trans[OF split])
 qed

lemma hash-extends-verifier-state-from-adversary-right:
 assumes ext: $s \leq t$
 shows $s \leq$ verifier-state-from-adversary t tr
 using ext
 unfolding verifier-state-from-adversary-def less-eq-hash-ext-def
 less-eq-fmap-def
 by simp

lemma checked-staged-security-with-data-state-supported-output-local-side-bad-bound-controlled:
 assumes wf: staged-budget-wellformed budgets
 and controlled: staged-adversary-controlled budgets A
 shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 supported-output-local-side-bad)
 adversary-initial-state $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget)
 unfolding supported-output-local-side-bad-iff-hash-map-output-collision-bad
 by (rule
 checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
 [OF wf controlled])

lemma checked-staged-security-with-data-state-supported-pairwise-side-bad-bound-controlled:
 fixes coupling-error :: prob
 assumes wf: staged-budget-wellformed budgets
 and controlled: staged-adversary-controlled budgets A
 and coupling-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 supported-pairwise-coupling-event)
 adversary-initial-state $\leq$ coupling-error
 shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 supported-pairwise-side-bad)
 adversary-initial-state $\leq$

```

    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget) +
    coupling-error
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  have side-eq:
    ?E supported-pairwise-side-bad =
      (λout. ?E hash-map-output-collision-bad out ∨
       ?E supported-pairwise-coupling-event out)
  unfolding staged-security-with-data-state-verifier-event-def
    supported-pairwise-side-bad-def supported-pairwise-coupling-event-def
  by (rule ext) (auto split: option.splits prod.splits)
  have wp-event ?M (?E supported-pairwise-side-bad)
    adversary-initial-state ≤
    wp-event ?M (?E hash-map-output-collision-bad)
      adversary-initial-state +
    wp-event ?M (?E supported-pairwise-coupling-event)
      adversary-initial-state
  unfolding side-eq
  by (rule wp-event-union-bound)
  also have ... ≤
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget) +
    coupling-error
  by (intro add-mono coupling-bound
    checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
    [OF wf controlled])
  finally show ?thesis .
qed

lemma checked-staged-accepted-imp-transcript-pre-or-new-hit:
  assumes support:
    out ∈
      set-dist
        (execute (checked-staged-security-experiment-with-data-state A)
         adversary-initial-state)
  and acc: accepted out
  shows
    staged-security-with-data-state-transcript-pre-hit out ∨
    staged-security-with-data-state-transcript-new-hit out
proof (cases out)
  case None
  then show ?thesis
    using acc unfolding accepted-def by simp
next
  case (Some packed)

```

then obtain *data attacker-state result final-state* **where** *out-eq*:
out = Some (((data, attacker-state), result), final-state)
by (*cases packed, auto split: prod.splits*)
let *?tr = staged-proof-transcript data*
let *?s = verifier-state-from-adversary attacker-state ?tr*
have *verifier*:
Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
using *checked-staged-security-experiment-with-data-state-outcomeE*
[OF support[unfolded out-eq]]
by *blast*
have *split*:
hash-map-output-values ?s ∩ set ?tr ≠ {} ∨
hash-map-new-output-hit (set ?tr) ?s final-state
by (*rule verify-monad-trace-root-target-preexisting-or-new[OF verifier]*)
simp
then show *?thesis*
unfolding *out-eq*
staged-security-with-data-state-transcript-pre-hit-def
staged-security-with-data-state-transcript-new-hit-def
by *simp*
qed

lemma *checked-staged-acceptance-transcript-hit-union-bound*:
wp-event (checked-staged-security-experiment-with-data-state A) accepted
adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-new-hit
adversary-initial-state

proof –
let *?M = checked-staged-security-experiment-with-data-state A*
let *?pre = staged-security-with-data-state-transcript-pre-hit*
let *?new = staged-security-with-data-state-transcript-new-hit*
have *wp-event ?M accepted adversary-initial-state ≤*
wp-event ?M (λout. ?pre out ∨ ?new out) adversary-initial-state
by (*rule wp-event-mono-on-support*)
(rule checked-staged-accepted-imp-transcript-pre-or-new-hit)
also have *... ≤*
wp-event ?M ?pre adversary-initial-state +
wp-event ?M ?new adversary-initial-state
by (*rule wp-event-union-bound*)
finally show *?thesis* .
qed

definition *staged-security-with-data-state-query-partial-header-key-hit*
 $:: (((f \text{ staged-proof-data} \times f \text{ protocol-channel}) \times \text{unit list}) \times$
 $f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

staged-security-with-data-state-query-partial-header-key-hit out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ *False*
 | *Some* (((*data*, *attacker-state*), -), *final-state*) $\Rightarrow$
 (let *s* =
 verifier-state-from-adversary attacker-state
 (*staged-proof-transcript data*);
 B =
 query-header-supported-partial-union-good-sets s
 query-sampling-success-space
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)
 (*staged-composition-final data*)
 in
 ($\exists$ *i raw*. *i* < *rounds* $\wedge$
 fmlookup (*HashMap final-state*)
 (*QueryIndexChallenge i*
 (*state-after-query-chunks*
 (*staged-query-start-hash data*)
 (*staged-query-chunks data*) *i*)) =
 Some raw $\wedge$
 index (*to-nat raw*) $\in$ *B*)))

definition *staged-security-with-data-state-query-bad-hit-at*

$:: \text{nat} \Rightarrow$
 (((*'f staged-proof-data* $\times$ *'f protocol-channel*) $\times$ *unit list*) $\times$
'f protocol-channel) *option* $\Rightarrow$ *bool*

where

staged-security-with-data-state-query-bad-hit-at i out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ *False*
 | *Some* (((*data*, *attacker-state*), *result*), *final-state*) $\Rightarrow$
 (let *s* =
 verifier-state-from-adversary attacker-state
 (*staged-proof-transcript data*)
 in
 Some (*result*, *final-state*) $\in$ *set-dist* (*execute verify-monad s*) $\wedge$
 query-index-round-set-hit-at s query-sampling-success-space i
 (*Some* (*result*, *final-state*))))

lemma *staged-security-with-data-state-query-bad-hit-imp-round-hit:*

assumes *bad*: *staged-security-with-data-state-query-bad-hit out*

shows $\exists i \in \{..<\text{rounds}\}$.

staged-security-with-data-state-query-bad-hit-at i out

```

proof (cases out)
  case None
  then show ?thesis
    using bad unfolding staged-security-with-data-state-query-bad-hit-def
    by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from bad[unfolded out-eq
    staged-security-with-data-state-query-bad-hit-def Let-def]
  have verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
    and query-bad: query-bad ?s (Some (result, final-state))
    by simp-all
  from query-bad-imp-query-index-round-set-hit[OF query-bad]
  obtain i where i-bound: i < rounds
    and hit-at:
      query-index-round-set-hit-at ?s query-sampling-success-space i
      (Some (result, final-state))
    unfolding query-index-round-set-hit-def by blast
  have staged-security-with-data-state-query-bad-hit-at i out
    unfolding out-eq staged-security-with-data-state-query-bad-hit-at-def
    Let-def
    using verifier hit-at by simp
  then show ?thesis
    using i-bound by blast
qed

lemma checked-staged-security-with-data-state-query-bad-hit-bound-from-rounds:
  assumes round-bound:
     $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-bad-hit-at i)
    adversary-initial-state  $\leq C i$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-bad-hit
    adversary-initial-state  $\leq (\sum i < \text{rounds}. C i)$ 
proof –
  have wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-bad-hit
    adversary-initial-state  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
    ( $\lambda out. \exists i \in \{..<\text{rounds}\}.$ 

```



```

      staged-security-with-data-state-query-bad-hit-at i out)
    adversary-initial-state
  by (rule wp-event-mono)
    (rule staged-security-with-data-state-query-bad-hit-imp-round-hit)
  also have ... ≤ (∑ i<rounds. C i)
  by (rule wp-event-finite-UN-bound)
    (simp-all add: round-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-query-bad-imp-partial-header-key-hit-or-collision-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    out ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  and bad: staged-security-with-data-state-query-bad-hit out
  shows
    staged-security-with-data-state-query-partial-header-key-hit out ∨
    staged-security-with-data-state-verifier-event
    hash-map-output-collision-bad out
proof (cases out)
  case None
  then show ?thesis
    using bad unfolding staged-security-with-data-state-query-bad-hit-def
    by simp
next
  case (Some result-pack)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
  by (cases result-pack, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have support-some:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  using support unfolding out-eq .
  from checked-staged-security-experiment-with-data-state-outcomeE
  [OF support-some]
  have builder:
    Some (data, attacker-state) ∈
      set-dist

```

```

      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast+
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
from bad[unfolded out-eq
  staged-security-with-data-state-query-bad-hit-def Let-def]
have query-bad: query-bad ?s (Some (result, final-state))
by simp
from query-bad-imp-query-index-round-set-hit[OF query-bad]
obtain i trace-table composition-table as query-idxs where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
and i-bound: i < rounds
and query-hit:
  query-idxs ! i ∈
    query-sampling-success-space trace-table composition-table as
unfolding query-index-round-set-hit-def
  query-index-round-set-hit-at-def
by blast
have shape:
  accepted-transcript-shape ?s (Some (result, final-state))
    as query-idxs
using bound
unfolding accepted-with-bound-tables-def accepted-with-tables-def
by simp
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support-some shape]
obtain raw-idxs where
  as-eq: as = staged-alphas data
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup:
   $\bigwedge j. j < rounds \implies$ 
    fmllookup (HashMap final-state)

```

```

      (QueryIndexChallenge j
       (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) j)) =
      Some (raw-idxs ! j)
    by blast
  show ?thesis
  proof (cases hash-map-output-collision final-state)
    case True
    then have
      staged-security-with-data-state-verifier-event
      hash-map-output-collision-bad
      (Some (((data, attacker-state), result), final-state))
    unfolding staged-security-with-data-state-verifier-event-def
      hash-map-output-collision-bad-def accepted-def
    by simp
    then show ?thesis
      unfolding out-eq by simp
  next
  case clean: False
  have idx-sample:
    query-idxs ! i ∈ query-sample-space
  proof -
    have len-query: length query-idxs = rounds
      using query-idxs-eq len-raw by simp
    have query-idxs ! i ∈ set query-idxs
      by (rule nth-mem) (use i-bound len-query in simp)
    then show ?thesis
      by (rule accepted-with-bound-tables-query-sample-space[OF bound])
  qed
  from bound obtain fr f-fri-roots f-final dg composition-fri-roots final
    rest where
    header':
      verifier-header-transcript ?s fr f-fri-roots f-final as dg
      composition-fri-roots final rest
  and trace-bind:
    merkle-root-binds-table fr trace-table final-state
  and comp-nonempty: composition-fri-roots ≠ []
  and comp-bind:
    merkle-root-binds-table (hd composition-fri-roots) composition-table
    final-state
  unfolding accepted-with-bound-tables-def by blast
  have eqs:
    fr = staged-trace-root data ∧
    f-fri-roots = staged-trace-fri-roots data ∧
    f-final = staged-trace-final data ∧
    as = staged-alphas data ∧
    dg = staged-degree data ∧
    composition-fri-roots = staged-composition-fri-roots data ∧

```

```

    final = staged-composition-final data ∧
    rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header' staged-header]
by simp
have state:
  query-header-supported-table-candidate-state ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    trace-table composition-table final-state
unfolding query-header-supported-table-candidate-state-def
by (intro exI[of - result] exI[of - query-idxs]
      exI[of - List.concat (staged-query-chunks data)]])
    (use verifier bound staged-header trace-bind comp-nonempty comp-bind
      eqs in simp)
have candidate:
  (trace-table, composition-table) ∈
    query-header-supported-partial-table-candidates ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
using
  query-header-supported-table-candidate-state-imp-partial-candidate-state
    [OF state clean]
  query-header-supported-partial-table-candidates-iff-state
by blast
have raw-hit:
  index (to-nat (raw-idxs ! i)) ∈
    query-header-supported-partial-union-good-sets ?s
      query-sampling-success-space
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
proof -
  have raw-idx-eq: index (to-nat (raw-idxs ! i)) = query-idxs ! i
    using query-idxs-eq len-raw i-bound by simp
  show ?thesis

```

```

    unfolding query-header-supported-partial-union-good-sets-def
      raw-idx-eq
    using idx-sample candidate query-hit as-eq by blast
qed
have lookup-i:
  fmllookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  by (rule lookup[OF i-bound])
show ?thesis
  unfolding out-eq
    staged-security-with-data-state-query-partial-header-key-hit-def
    Let-def
  using i-bound lookup-i raw-hit by auto
qed
qed

lemma checked-staged-security-with-data-state-query-bad-bound-from-partial-header-key-hit-or-collision:
  fixes C K :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and partial-header-key-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-partial-header-key-hit
      adversary-initial-state  $\leq$  C
  and collision-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        hash-map-output-collision-bad)
      adversary-initial-state  $\leq$  K
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-bad-hit
      adversary-initial-state  $\leq$  C + K
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  have wp-event ?M staged-security-with-data-state-query-bad-hit
    adversary-initial-state  $\leq$ 
    wp-event ?M
      ( $\lambda$ out.
        staged-security-with-data-state-query-partial-header-key-hit out  $\vee$ 
        staged-security-with-data-state-verifier-event
          hash-map-output-collision-bad out)
        adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out

```

```

assume support:
  out ∈ set-dist (execute ?M adversary-initial-state)
  and bad: staged-security-with-data-state-query-bad-hit out
show
  staged-security-with-data-state-query-partial-header-key-hit out ∨
  staged-security-with-data-state-verifier-event
  hash-map-output-collision-bad out
by (rule
  checked-staged-security-with-data-state-query-bad-imp-partial-header-key-hit-or-collision-on-support
  [OF wf controlled support bad])
qed
also have ... ≤
  wp-event ?M staged-security-with-data-state-query-partial-header-key-hit
  adversary-initial-state +
  wp-event ?M
  (staged-security-with-data-state-verifier-event
  hash-map-output-collision-bad)
  adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤ C + K
by (intro add-mono partial-header-key-bound collision-bound)
finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-partial-header-key-hit-or-collision
fixes C K :: prob
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and partial-header-key-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-partial-header-key-hit
  adversary-initial-state ≤ C
and collision-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
  hash-map-output-collision-bad)
  adversary-initial-state ≤ K
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event query-bad)
  adversary-initial-state ≤ C + K
proof –
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event query-bad)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-bad-hit
  adversary-initial-state
by (rule checked-staged-security-with-data-state-query-bad-verifier-event-bound

```

$[OF \text{ order-refl}]$
also have $\dots \leq C + K$
by (rule
checked-staged-security-with-data-state-query-bad-bound-from-partial-header-key-hit-or-collision
 $[OF \text{ wf controlled partial-header-key-bound collision-bound}]$)
finally show *?thesis* .
qed

lemma *checked-staged-security-with-data-state-partial-header-key-hit-bound-from-index-set:*

assumes *cover:*

$\wedge$ *data attacker-state.*
query-header-supported-partial-union-good-sets
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
query-sampling-success-space
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data) $\subseteq B$

and *index-bound:*

wp-event (checked-staged-security-experiment-with-data A)
(staged-security-with-data-query-index-set-hit B)
adversary-initial-state $\leq C$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-query-partial-header-key-hit
adversary-initial-state $\leq C$

proof –

let *?projected =*

$\lambda out. \text{ case out of None } \Rightarrow$
staged-security-with-data-query-index-set-hit B None
 $| \text{ Some } (((data, -), result), t) \Rightarrow$
staged-security-with-data-query-index-set-hit B
(Some ((data, result), t))

have *event-le:*

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-query-partial-header-key-hit
adversary-initial-state $\leq$
wp-event (checked-staged-security-experiment-with-data-state A)
?projected adversary-initial-state

proof (rule *wp-event-mono*)

fix *out*

assume *hit:*

staged-security-with-data-state-query-partial-header-key-hit out

show *?projected out*

proof (cases *out*)

```

case None
then show ?thesis
  using hit
  unfolding
    staged-security-with-data-state-query-partial-header-key-hit-def
  by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
let ?B =
  query-header-supported-partial-union-good-sets ?s
  query-sampling-success-space
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
from hit[unfolded out-eq
  staged-security-with-data-state-query-partial-header-key-hit-def
  Let-def]
obtain i raw where i-bound: i < rounds
and lookup:
  fmlookup (HashMap final-state)
  (QueryIndexChallenge i
  (state-after-query-chunks
  (staged-query-start-hash data)
  (staged-query-chunks data) i)) =
  Some raw
and raw-hit: index (to-nat raw) ∈ ?B
by auto
have raw-in-B: index (to-nat raw) ∈ B
proof –
  have ?B ⊆ B
  by (rule cover)
  then show ?thesis
  using raw-hit by blast
qed
show ?thesis
  unfolding out-eq staged-security-with-data-query-index-set-hit-def
  using i-bound lookup raw-in-B by auto
qed
qed

```


also have

wp-event (checked-staged-security-experiment-with-data-state A)

?projected adversary-initial-state =

wp-event (checked-staged-security-experiment-with-data A)

(staged-security-with-data-query-index-set-hit B)

adversary-initial-state

using *checked-staged-security-experiment-with-data-event-from-data-state*

[of A staged-security-with-data-query-index-set-hit B]

by *simp*

also have $\dots \leq C$

by *(rule index-bound)*

finally show *?thesis .*

qed

lemma *checked-staged-security-with-data-state-partial-header-key-hit-bound-from-index-set-controlled:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *cover:*

$\bigwedge$ *data attacker-state.*

query-header-supported-partial-union-good-sets

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

query-sampling-success-space

(staged-trace-root data)

(staged-trace-fri-roots data)

(staged-trace-final data)

(staged-alphas data)

(staged-degree data)

(staged-composition-fri-roots data)

(staged-composition-final data) $\subseteq B$

shows

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-query-partial-header-key-hit

adversary-initial-state $\leq$

staged-phase-target-error (query-index-raw-preimage B)

(staged-attacker-query-budget budgets +

staged-challenge-query-budget)

by *(rule*

checked-staged-security-with-data-state-partial-header-key-hit-bound-from-index-set

[OF cover])

(rule checked-staged-security-with-data-query-index-set-hit-bound

[OF wf controlled])

lemma *checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-partial-fixed-query-error-cover:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *cover:*

$\bigwedge$ *data attacker-state.*

query-header-supported-partial-union-good-sets

```

    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    query-sampling-success-space
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)  $\subseteq B$ 
  and raw-bound: query-index-raw-preimage-bound
  and subset:  $B \subseteq \text{query-sample-space}$ 
  and frac:
    nnreal (card  $B$ ) / nnreal (card query-sample-space)  $\leq$ 
      query-error-bound
shows
  wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    (staged-security-with-data-state-verifier-event query-bad)
  adversary-initial-state  $\leq$ 
    nnreal
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget) * query-error-bound +
      hash-collision-budget-value 0
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget + verifier-hash-query-budget)
proof -
  have partial-key-bound:
    wp-event (checked-staged-security-experiment-with-data-state  $A$ )
      staged-security-with-data-state-query-partial-header-key-hit
    adversary-initial-state  $\leq$ 
      nnreal
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget) * query-error-bound
  proof -
    have wp-event (checked-staged-security-experiment-with-data-state  $A$ )
      staged-security-with-data-state-query-partial-header-key-hit
      adversary-initial-state  $\leq$ 
        staged-phase-target-error (query-index-raw-preimage  $B$ )
          (staged-attacker-query-budget budgets +
            staged-challenge-query-budget)
    by (rule
      checked-staged-security-with-data-state-partial-header-key-hit-bound-from-index-set-controlled
      [OF wf controlled cover])
  also have ...  $\leq$ 
    nnreal
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget) * query-error-bound
  by (rule staged-phase-query-index-target-error-query-bound
    [OF raw-bound subset frac])

```

finally show *?thesis* .
qed
have *collision-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
hash-map-output-collision-bad)
adversary-initial-state ≤
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
[OF wf controlled])
show *?thesis*
by (rule
checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-partial-header-key-hit-or-collision
[OF wf controlled partial-key-bound collision-bound])
qed

lemma *staged-dynamic-partial-query-set-subset*:
query-header-supported-partial-union-good-sets
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
query-sampling-success-space
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data) ⊆ query-sample-space
by (rule *query-header-supported-partial-union-good-sets-subset*)

lemma *staged-dynamic-partial-query-set-fraction-bound-if-unique-candidate*:
assumes *unique*:
 $\exists$ *trace-table composition-table*.
query-header-supported-partial-table-candidates
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data) ⊆
{(trace-table, composition-table)}
shows
nnreal

(card
 (query-header-supported-partial-union-good-sets
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 query-sampling-success-space
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data))) /
 nnreal (card query-sample-space) ≤ query-error-bound
by (rule
 query-header-supported-partial-union-good-sets-fraction-bound-if-unique-candidate
 [OF unique query-sampling-success-space-subset
 query-sampling-success-space-fraction-bound-query-sample-space])

lemma staged-dynamic-partial-query-set-target-error-bound-if-unique-candidate:

assumes raw-bound: query-index-raw-preimage-bound

and unique:

∃ trace-table composition-table.
 query-header-supported-partial-table-candidates
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) ⊆
 {(trace-table, composition-table)}

shows

staged-phase-target-error
 (query-index-raw-preimage
 (query-header-supported-partial-union-good-sets
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 query-sampling-success-space
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data))) q ≤
 nnreal q * query-error-bound
by (rule staged-phase-query-index-target-error-query-bound

[*OF* raw-bound staged-dynamic-partial-query-set-subset
 staged-dynamic-partial-query-set-fraction-bound-if-unique-candidate
 [*OF* unique]])

lemma *checked-staged-query-prefix-recv-bad-index-imp-new-output-hit*:

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *i-bound*: $i \leq \text{rounds}$

and *absent*: query-index-raw-preimage-absent *B s*

and *out*:

Some (*raw*, *t*) ∈

set-dist

(execute

(checked-staged-query-challenge-prefix-program *A i* $\gg$

(λ -. receive-query-index-challenge)) *s*)

and *bad*: index (to-nat *raw*) ∈ *B*

shows

hash-new-output-hit-event (query-index-raw-preimage *B*) *s*

(Some (*raw*, *t*))

proof –

obtain *prefix-out u* **where** *prefix*:

Some (*prefix-out*, *u*) ∈

set-dist

(execute (checked-staged-query-challenge-prefix-program *A i*) *s*)

and *recv*:

Some (*raw*, *t*) ∈ set-dist (execute receive-query-index-challenge *u*)

using *out* **by** (auto elim!: set-dist-bindE)

have *unchecked-prefix*:

Some

((spp-degree *prefix-out*, spp-composition-final *prefix-out*,

spp-query-chunks *prefix-out*), *u*) ∈

set-dist (execute (staged-query-challenge-prefix-program *A i*) *s*)

by (rule checked-staged-query-challenge-prefix-outcome-imp-staged-query-prefix

[*OF prefix*])

have *prefix-ext*: $s \leq u$

using hash-target-program-staged-query-challenge-prefix-program

[*OF wf controlled i-bound, of query-index-raw-preimage B*]

unchecked-prefix

unfolding hash-target-program-def hash-extension-preserving-def

by blast

have *recv-out*:

$u \leq t \wedge$

fmlookup (HashMap *t*)

(QueryIndexChallenge (PQueryCounter *u*) (PState *u*)) = Some *raw*

using receive-query-index-challenge-outcome[*OF recv*] **by** blast

have *raw-in*: *raw* ∈ query-index-raw-preimage *B*

using *bad* **unfolding** query-index-raw-preimage-def **by** simp

show ?thesis

proof (cases

```

    fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter u) (PState u)))
  case None
  then have hash-map-new-output-hit (query-index-raw-preimage B) s t
    unfolding hash-map-new-output-hit-def
    using recv-out raw-in by blast
  then show ?thesis
    unfolding hash-new-output-hit-event-def by simp
next
case (Some old-raw)
have old-lookup-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter u) (PState u)) =
    Some old-raw
by (rule hash-extension-lookup[OF Some])
  (rule hash-ext-trans[OF prefix-ext conjunct1[OF recv-out]])
then have old-eq: old-raw = raw
  using recv-out by simp
have old-raw  $\notin$  query-index-raw-preimage B
  using absent Some
  unfolding query-index-raw-preimage-absent-def by blast
then show ?thesis
  using old-eq raw-in by simp
qed
qed

```

definition *checked-staged-query-prefix-with-state*

```

:: 'f staged-adversary  $\Rightarrow$  nat  $\Rightarrow$ 
  (((('f staged-query-prefix-data  $\times$  'f protocol-channel),
    'f protocol-channel) state-monad

```

where

```

checked-staged-query-prefix-with-state A i =
  checked-staged-query-challenge-prefix-program A i  $\gg=$ 
  ( $\lambda$ prefix. get  $\gg=$  ( $\lambda$ prefix-state.
    return (prefix, prefix-state)))

```

definition *checked-staged-query-prefix-receive-with-state*

```

:: 'f staged-adversary  $\Rightarrow$  nat  $\Rightarrow$ 
  (((('f staged-query-prefix-data  $\times$  'f protocol-channel)  $\times$  'f),
    'f protocol-channel) state-monad

```

where

```

checked-staged-query-prefix-receive-with-state A i =
  checked-staged-query-prefix-with-state A i  $\gg=$ 
  ( $\lambda$ prefix-pack. receive-query-index-challenge  $\gg=$  ( $\lambda$ raw.
    return (prefix-pack, raw)))

```

lemma *checked-staged-query-program-decomp:*

assumes *i-bound:* $i < n$

shows

```

checked-staged-query-program A trace-roots composition-roots start n =
checked-staged-query-program A trace-roots composition-roots start i ≫=
(λprefix-chunks.
  receive-query-index-challenge ≫= (λraw.
    let idx = index (to-nat raw) in
    query-opening-stage A (start + i) raw ≫= (λchunk.
      assert
        (verifier-query-round-chunk idx trace-roots composition-roots
          chunk) ≫= (λ-.
            record-staged-messages chunk ≫= (λ-.
              checked-staged-query-program A trace-roots composition-roots
                (Suc (start + i)) (n - Suc i) ≫= (λsuffix-chunks.
                  return (prefix-chunks @ chunk # suffix-chunks)))))))
using i-bound
proof (induction i arbitrary: start n)
case 0
then obtain m where n-eq: n = Suc m
by (cases n) simp-all
show ?case
unfolding n-eq
by (simp add: sm-bind-assoc Let-def)
next
case (Suc i)
then obtain m where n-eq: n = Suc m
by (cases n) simp-all
have i-lt-m: i < m
using Suc.premis unfolding n-eq by simp
have IH:
  checked-staged-query-program A trace-roots composition-roots
    (Suc start) m =
  checked-staged-query-program A trace-roots composition-roots
    (Suc start) i ≫=
  (λprefix-chunks.
    receive-query-index-challenge ≫= (λraw.
      let idx = index (to-nat raw) in
      query-opening-stage A (Suc start + i) raw ≫= (λchunk.
        assert
          (verifier-query-round-chunk idx trace-roots composition-roots
            chunk) ≫= (λ-.
              record-staged-messages chunk ≫= (λ-.
                checked-staged-query-program A trace-roots composition-roots
                  (Suc (Suc start + i)) (m - Suc i) ≫= (λsuffix-chunks.
                    return (prefix-chunks @ chunk # suffix-chunks)))))))
by (rule Suc.IH[OF i-lt-m])
show ?case
unfolding n-eq
by (simp add: sm-bind-assoc Let-def IH)
qed

```

lemma *get-bind-const*:
 $(get \gg (\lambda-. m)) = m$
proof *transfer*
fix *m*
show *dist-bind dist-get* $(\lambda-. m) = m$
proof
fix *s*
show *dist-bind dist-get* $(\lambda-. m) s = m s$
apply (*rule dist-inject* [*THEN iffD1*])
unfolding *dist-bind.rep-eq dist-get-def bind-cont-map-def*
apply (*simp add: o-def map-fun-def dist-delta-dist*)
apply (*subst map-bind-delta-left-state* [**where** $f = \lambda s. \text{Some } (s, s)$])
by (*auto split: option.splits prod.splits*)
qed
qed

lemma *get-bind-return-state-support*:
 $\text{Some } ((x, s), s) \in$
 $\text{set-dist } (\text{execute } (get \gg (\lambda t. \text{return } (x, t)))) s$
proof –
have *get-out*: $\text{Some } (s, s) \in \text{set-dist } (\text{execute } get) s$
unfolding *set-dist-def get.rep-eq dist-get-def dist-delta-dist*
delta-map-def
by *simp*
have *ret-out*:
 $\text{Some } ((x, s), s) \in \text{set-dist } (\text{execute } (\text{return } (x, s))) s$
unfolding *set-dist-def return.rep-eq dist-return-def dist-delta-dist*
delta-map-def
by *simp*
show *?thesis*
by (*rule-tac x=s and t=s in set-dist-bindI*)
(rule get-out, rule ret-out)
qed

lemma *get-bind-return-state-outcome*:
assumes *outcome*:
 $\text{Some } ((x\text{-out}, \text{captured}), \text{final}) \in$
 $\text{set-dist } (\text{execute } (get \gg (\lambda t. \text{return } (x, t)))) s$
shows $x\text{-out} = x \wedge \text{captured} = s \wedge \text{final} = s$
proof –
from *outcome* **obtain** *y u* **where** *get-out*:
 $\text{Some } (y, u) \in \text{set-dist } (\text{execute } get) s$
and *ret-out*:
 $\text{Some } ((x\text{-out}, \text{captured}), \text{final}) \in$
 $\text{set-dist } (\text{execute } (\text{return } (x, y))) u$
by (*auto elim!: set-dist-bindE*)
have *y-u*: $y = s \wedge u = s$
using *get-out*
unfolding *set-dist-def get.rep-eq dist-get-def dist-delta-dist*


```

    delta-map-def
  by auto
  have captured-final:  $x\text{-out} = x \wedge \text{captured} = y \wedge \text{final} = u$ 
  using ret-out
  unfolding set-dist-def return.rep-eq dist-return-def dist-delta-dist
    delta-map-def
  by auto
  show ?thesis
    using y-u captured-final by simp
qed

lemma checked-staged-query-prefix-with-state-outcome-state:
  assumes outcome:
    Some ((prefix, prefix-state), u)  $\in$ 
      set-dist (execute (checked-staged-query-prefix-with-state A i) s)
  shows u = prefix-state
proof -
  show ?thesis
    using outcome
    unfolding checked-staged-query-prefix-with-state-def
    by (auto dest!: get-bind-return-state-outcome
      elim!: set-dist-bindE)
qed

lemma checked-staged-query-prefix-with-state-outcome-canonical:
  assumes outcome:
    Some ((prefix, prefix-state), u)  $\in$ 
      set-dist (execute (checked-staged-query-prefix-with-state A i) s)
  shows
    Some ((prefix, prefix-state), prefix-state)  $\in$ 
      set-dist (execute (checked-staged-query-prefix-with-state A i) s)
  using outcome checked-staged-query-prefix-with-state-outcome-state[OF outcome]
  by simp

lemma checked-staged-query-prefix-receive-with-state-prefix-support:
  assumes outcome:
    Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i) s)
  shows
    Some ((prefix, prefix-state), prefix-state)  $\in$ 
      set-dist (execute (checked-staged-query-prefix-with-state A i) s)
proof -
  from outcome obtain u where prefix-u:
    Some ((prefix, prefix-state), u)  $\in$ 
      set-dist (execute (checked-staged-query-prefix-with-state A i) s)
  unfolding checked-staged-query-prefix-receive-with-state-def
  by (auto elim!: set-dist-bindE)
  show ?thesis

```

by (rule checked-staged-query-prefix-with-state-outcome-canonical
[OF prefix-u])
qed

lemma checked-staged-query-prefix-receive-with-state-receive-support:

assumes outcome:

Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i) s)

shows

Some (raw, raw-state) ∈
set-dist (execute receive-query-index-challenge prefix-state)

proof –

from outcome **obtain** u **where** prefix-u:

Some ((prefix, prefix-state), u) ∈
set-dist (execute (checked-staged-query-prefix-with-state A i) s)

and recv-u:

Some (raw, raw-state) ∈
set-dist (execute receive-query-index-challenge u)

unfolding checked-staged-query-prefix-receive-with-state-def

by (auto elim!: set-dist-bindE)

have u = prefix-state

by (rule checked-staged-query-prefix-with-state-outcome-state
[OF prefix-u])

then show ?thesis

using recv-u **by** simp

qed

lemma checked-staged-query-challenge-prefix-program-outcomeE:

assumes outcome:

Some (prefix, prefix-state) ∈
set-dist
(execute (checked-staged-query-challenge-prefix-program A i)
adversary-initial-state)

obtains fr s1 s2 trace-roots trace-bs s3 trace-final s4 s5 as s6 dg
s7 s8 s9 composition-roots composition-bs s10 composition-final s11
query-start query-chunks **where**

Some (fr, s1) ∈
set-dist (execute (trace-root-stage A) adversary-initial-state)
Some ((), s2) ∈ set-dist (execute (record-staged-message fr) s1)
Some ((trace-roots, trace-bs), s3) ∈
set-dist
(execute
(staged-trace-fri-program A 0 (ceil-log clength) []) s2)

Some (trace-final, s4) ∈
set-dist (execute (trace-final-stage A trace-bs) s3)

Some ((), s5) ∈
set-dist (execute (record-staged-message trace-final) s4)

Some (as, s6) ∈

```

    set-dist (execute (staged-alpha-program (length spec)) s5)
Some (dg, s7) ∈ set-dist (execute (degree-stage A as) s6)
Some ((), s8) ∈
  set-dist (execute (record-staged-message dg) s7)
Some ((), s9) ∈
  set-dist
    (execute
      (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)))
      s8)
Some ((composition-roots, composition-bs), s10) ∈
  set-dist
    (execute
      (staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []) s9)
Some (composition-final, s11) ∈
  set-dist
    (execute (composition-final-stage A dg composition-bs) s10)
Some ((), query-start) ∈
  set-dist (execute (record-staged-message composition-final) s11)
Some (query-chunks, prefix-state) ∈
  set-dist
    (execute
      (checked-staged-query-program A trace-roots composition-roots
        0 i)
      query-start)
prefix =
  (|sqp-trace-root = fr,
   sqp-trace-fri-roots = trace-roots,
   sqp-trace-fri-challenges = trace-bs,
   sqp-trace-final = trace-final,
   sqp-alphas = as,
   sqp-degree = dg,
   sqp-composition-fri-roots = composition-roots,
   sqp-composition-fri-challenges = composition-bs,
   sqp-composition-final = composition-final,
   sqp-query-chunks = query-chunks|)
using outcome
unfolding checked-staged-query-challenge-prefix-program-def
  staged-alpha-prefix-program-def Let-def
by (auto elim!: set-dist-bindE intro: that split: prod.splits)

```

definition *checked-staged-after-query-prefix-receive*

```

:: 'f staged-adversary ⇒ nat ⇒
  (('f staged-query-prefix-data × 'f protocol-channel) × 'f) ⇒
  ('f staged-proof-data, 'f protocol-channel) state-monad

```

where

```

checked-staged-after-query-prefix-receive A i prefix-pack =
  (case prefix-pack of ((prefix, -), raw) ⇒
    do {

```

```

let idx = index (to-nat raw);
chunk ← query-opening-stage A i raw;
assert
  (verifier-query-round-chunk idx
    (sqp-trace-fri-roots prefix)
    (sqp-composition-fri-roots prefix) chunk);
record-staged-messages chunk;
suffix-chunks ←
  checked-staged-query-program A
    (sqp-trace-fri-roots prefix)
    (sqp-composition-fri-roots prefix)
    (Suc i) (rounds - Suc i);
return
  (|staged-trace-root = sqp-trace-root prefix,
    staged-trace-fri-roots = sqp-trace-fri-roots prefix,
    staged-trace-fri-challenges =
      sqp-trace-fri-challenges prefix,
    staged-trace-final = sqp-trace-final prefix,
    staged-alphas = sqp-alphas prefix,
    staged-degree = sqp-degree prefix,
    staged-composition-fri-roots =
      sqp-composition-fri-roots prefix,
    staged-composition-fri-challenges =
      sqp-composition-fri-challenges prefix,
    staged-composition-final = sqp-composition-final prefix,
    staged-query-chunks =
      sqp-query-chunks prefix @ chunk # suffix-chunks|)
})

```

definition *checked-staged-after-query-prefix-receive-with-verifier*
 $:: 'f \text{ staged-adversary} \Rightarrow \text{nat} \Rightarrow$
 $((('f \text{ staged-query-prefix-data} \times 'f \text{ protocol-channel}) \times 'f) \Rightarrow$
 $((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}),$
 $'f \text{ protocol-channel}) \text{ state-monad}$

where

checked-staged-after-query-prefix-receive-with-verifier A i prefix-pack =
checked-staged-after-query-prefix-receive A i prefix-pack $\gg=$
 $(\lambda \text{data.}$
 $\text{get} \gg= (\lambda s.$
 put
 $(\text{verifier-state-from-adversary } s$
 $(\text{staged-proof-transcript } \text{data})) \gg=$
 $(\lambda -.$
 $\text{verify-monad} \gg=$
 $(\lambda \text{result. return } ((\text{data}, s), \text{result}))))$

lemma *checked-staged-after-query-prefix-receive-with-verifier-outcome:*

assumes *outcome:*

Some (((data, attacker-state), result), final-state) ∈

```

    set-dist
    (execute
      (checked-staged-after-query-prefix-receive-with-verifier A i
        prefix-pack)
      s)
  shows
    Some (data, attacker-state) ∈
      set-dist
      (execute
        (checked-staged-after-query-prefix-receive A i prefix-pack) s)
    Some (result, final-state) ∈
      set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
  proof –
  from outcome[unfolded
    checked-staged-after-query-prefix-receive-with-verifier-def]
  obtain data' attacker-state' where data':
    Some (data', attacker-state') ∈
      set-dist
      (execute
        (checked-staged-after-query-prefix-receive A i prefix-pack) s)
  and rest:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
      (execute
        (get ≫=
          (λsa.
            put
              (verifier-state-from-adversary sa
                (staged-proof-transcript data')) ≫=
              (λ-.
                verify-monad ≫=
                  (λr. return ((data', sa), r))))))
        attacker-state')
      by (rule set-dist-bindE) (rule that)
  from rest obtain saved-state state-after-get where get-out:
    Some (saved-state, state-after-get) ∈
      set-dist (execute get attacker-state')
  and after-get:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
      (execute
        (put
          (verifier-state-from-adversary saved-state
            (staged-proof-transcript data')) ≫=
          (λ-.
            verify-monad ≫=

```

```

      (λr. return ((data', saved-state), r))))
    state-after-get)
  by (rule set-dist-bindE) (rule that)
from get-out have saved-state-eq: saved-state = attacker-state'
  and state-after-get-eq: state-after-get = attacker-state'
  by simp-all
from after-get[unfolded saved-state-eq state-after-get-eq]
obtain u verifier-start where put-out:
  Some (u, verifier-start) ∈
    set-dist
      (execute
        (put
          (verifier-state-from-adversary attacker-state'
            (staged-proof-transcript data')))
        attacker-state')
  and after-put:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute
        (verify-monad >>=
          (λr. return ((data', attacker-state'), r)))
        verifier-start)
  by (rule set-dist-bindE) (rule that)
from put-out have verifier-start-eq:
  verifier-start =
    verifier-state-from-adversary attacker-state'
    (staged-proof-transcript data')
  by simp
from after-put[unfolded verifier-start-eq]
obtain result' final-state' where verifier:
  Some (result', final-state') ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state'
          (staged-proof-transcript data')))
  and returned:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute (return ((data', attacker-state'), result'))
        final-state')
  by (elim set-dist-bindE)
from returned have data-eq: data = data'
  and attacker-state-eq: attacker-state = attacker-state'
  and result-eq: result = result'
  and final-state-eq: final-state = final-state'
  by simp-all
show
  Some (data, attacker-state) ∈
    set-dist

```

```

      (execute
        (checked-staged-after-query-prefix-receive A i prefix-pack) s)
    using data' unfolding data-eq attacker-state-eq by simp
show
  Some (result, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
    using verifier
    unfolding data-eq attacker-state-eq result-eq final-state-eq
    by simp
qed

lemma checked-staged-security-experiment-with-data-state-outcomeI:
  assumes builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
  shows
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  unfolding checked-staged-security-experiment-with-data-state-def
  apply (rule-tac x=data and t=attacker-state in set-dist-bindI)
  apply (rule builder)
  apply (rule-tac x=attacker-state and t=attacker-state in set-dist-bindI)
  apply simp
  apply (rule-tac x=() and
    t=verifier-state-from-adversary attacker-state
    (staged-proof-transcript data) in set-dist-bindI)
  apply simp
  apply (rule-tac x=result and t=final-state in set-dist-bindI)
  apply (rule verifier)
  apply simp
done

lemma checked-staged-transcript-program-query-prefix-receive-decomp:
  assumes i-bound: i < rounds
  shows

```

```

checked-staged-transcript-program A =
  checked-staged-query-prefix-receive-with-state A i >>=
    checked-staged-after-query-prefix-receive A i
proof –
have query-decomp:
   $\wedge$  trace-roots composition-roots.
  checked-staged-query-program A trace-roots composition-roots 0 rounds =
  checked-staged-query-program A trace-roots composition-roots 0 i >>=
    ( $\lambda$  prefix-chunks.
      receive-query-index-challenge >>= ( $\lambda$  raw.
        let idx = index (to-nat raw) in
        query-opening-stage A (0 + i) raw >>= ( $\lambda$  chunk.
          assert
            (verifier-query-round-chunk idx trace-roots composition-roots
              chunk) >>= ( $\lambda$ -.
                record-staged-messages chunk >>= ( $\lambda$ -.
                  checked-staged-query-program A trace-roots composition-roots
                    (Suc (0 + i)) (rounds – Suc i) >>= ( $\lambda$  suffix-chunks.
                      return (prefix-chunks @ chunk # suffix-chunks))))))
    )
by (rule checked-staged-query-program-decomp[OF i-bound])
show ?thesis
unfolding checked-staged-transcript-program-def
  checked-staged-query-prefix-receive-with-state-def
  checked-staged-query-prefix-with-state-def
  checked-staged-query-challenge-prefix-program-def
  staged-alpha-prefix-program-def
  checked-staged-after-query-prefix-receive-def
apply (simp add: sm-bind-assoc Let-def split-def query-decomp
  get-bind-const)
done
qed

```

```

lemma checked-staged-transcript-program-query-prefix-receive-cont-support:
assumes i-bound: i < rounds
and outcome:
  Some (data, attacker-state)  $\in$ 
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
obtains prefix prefix-state raw raw-state where
  Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  Some (data, attacker-state)  $\in$ 
    set-dist
      (execute
        (checked-staged-after-query-prefix-receive A i
          ((prefix, prefix-state), raw))

```



```

    raw-state)
proof –
  have decomp:
    checked-staged-transcript-program A =
      checked-staged-query-prefix-receive-with-state A i  $\gg$ 
      checked-staged-after-query-prefix-receive A i
    by (rule checked-staged-transcript-program-query-prefix-receive-decomp
      [OF i-bound])
  from outcome[unfolded decomp]
  obtain x raw-state where head:
    Some (x, raw-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  and tail:
    Some (data, attacker-state) ∈
      set-dist
      (execute
        (checked-staged-after-query-prefix-receive A i x)
        raw-state)
    by (auto elim!: set-dist-bindE)
  obtain prefix-pack raw where x-eq: x = (prefix-pack, raw)
  by (cases x) simp
  obtain prefix prefix-state where prefix-pack-eq:
    prefix-pack = (prefix, prefix-state)
  by (cases prefix-pack) simp
  show ?thesis
  proof (rule that[of prefix prefix-state raw raw-state])
    show Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
    using head unfolding x-eq prefix-pack-eq by simp
    show Some (data, attacker-state) ∈
      set-dist
      (execute
        (checked-staged-after-query-prefix-receive A i
          ((prefix, prefix-state), raw))
        raw-state)
    using tail unfolding x-eq prefix-pack-eq by simp
  qed
qed

lemma checked-staged-security-experiment-with-data-state-query-prefix-receive-decomp:
  assumes i-bound: i < rounds
  shows
    checked-staged-security-experiment-with-data-state A =
      checked-staged-query-prefix-receive-with-state A i  $\gg$ 
      checked-staged-after-query-prefix-receive-with-verifier A i

```

unfolding *checked-staged-security-experiment-with-data-state-def*
checked-staged-transcript-program-query-prefix-receive-decomp[*OF i-bound*]
checked-staged-after-query-prefix-receive-with-verifier-def
by (*simp add: sm-bind-assoc*)

definition *staged-query-prefix-start-hash*

:: 'f *staged-query-prefix-data* $\Rightarrow$ 'f

where

staged-query-prefix-start-hash prefix =
concat
 (*foldl concat*
 (*concat*
 (*foldl concat*
 (*concat*
 (*foldl concat*
 (*concat* 0 (*sqp-trace-root prefix*))
 (*sqp-trace-fri-roots prefix*))
 (*sqp-trace-final prefix*))
 (*sqp-alphas prefix*))
 (*sqp-degree prefix*))
 (*sqp-composition-fri-roots prefix*))
 (*sqp-composition-final prefix*))

lemma *checked-staged-query-prefix-with-state-query-chunks-length:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *support:*

Some ((prefix, prefix-state), prefix-state) $\in$
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state)

shows *length (sqp-query-chunks prefix) = i*

proof –

from *support* **have** *prefix-out:*

Some (prefix, prefix-state) $\in$
set-dist
(execute (checked-staged-query-challenge-prefix-program A i)
adversary-initial-state)

unfolding *checked-staged-query-prefix-with-state-def*

by (*auto elim!: set-dist-bindE*)

from *prefix-out* **obtain** *fr s1 s2 trace-roots trace-bs s3 trace-final s4*

s5 as s6 dg s7 s8 s9 composition-roots composition-bs s10

composition-final s11 query-start query-chunks **where**

query-out:

Some (query-chunks, prefix-state) $\in$
set-dist
(execute
(checked-staged-query-program A trace-roots composition-roots

```

    0 i) s11)
and prefix-eq:
  prefix =
    (sqp-trace-root = fr,
     sqp-trace-fri-roots = trace-roots,
     sqp-trace-fri-challenges = trace-bs,
     sqp-trace-final = trace-final,
     sqp-alphas = as,
     sqp-degree = dg,
     sqp-composition-fri-roots = composition-roots,
     sqp-composition-fri-challenges = composition-bs,
     sqp-composition-final = composition-final,
     sqp-query-chunks = query-chunks)
using prefix-out
by (elim checked-staged-query-challenge-prefix-program-outcomeE)
have bound:  $0 + i \leq \text{length } (\text{query-opening-budgets } \text{budgets})$ 
  using wf i-bound unfolding staged-budget-wellformed-def by simp
from checked-staged-query-program-outcome-with-raws
  [OF controlled bound query-out]
obtain raw-idxs query-idxs where len:  $\text{length } \text{query-chunks} = i$ 
by blast
then show ?thesis
  unfolding prefix-eq by simp
qed

```

lemma *checked-staged-query-prefix-with-state-alignment*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *i-bound*: $i < \text{rounds}$

and *support*:

Some (*(prefix, prefix-state), prefix-state*) $\in$
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state)

shows

PQueryCounter prefix-state = $i \wedge$
PState prefix-state =
state-after-query-chunks
(staged-query-prefix-start-hash prefix)
(sqp-query-chunks prefix) i $\wedge$
 $\text{length } (\text{sqp-query-chunks } \text{prefix}) = i$

proof –

from *support* **have** *prefix-out*:

Some (*prefix, prefix-state*) $\in$
set-dist
(execute (checked-staged-query-challenge-prefix-program A i)
adversary-initial-state)

unfolding *checked-staged-query-prefix-with-state-def*

by (*auto elim! set-dist-bindE*)

```

from prefix-out obtain fr s1 s2 trace-roots trace-bs s3 trace-final s4
s5 as s6 dg s7 s8 s9 composition-roots composition-bs s10
composition-final s11 query-start query-chunks where
root-out:
  Some (fr, s1) ∈
    set-dist (execute (trace-root-stage A) adversary-initial-state)
and record-root:
  Some ((), s2) ∈ set-dist (execute (record-staged-message fr) s1)
and trace-out:
  Some ((trace-roots, trace-bs), s3) ∈
    set-dist
      (execute
        (staged-trace-fri-program A 0 (ceil-log clength) []) s2)
and final-out:
  Some (trace-final, s4) ∈
    set-dist (execute (trace-final-stage A trace-bs) s3)
and record-final:
  Some ((), s5) ∈
    set-dist (execute (record-staged-message trace-final) s4)
and alpha-out:
  Some (as, s6) ∈
    set-dist (execute (staged-alpha-program (length spec)) s5)
and degree-out:
  Some (dg, s7) ∈ set-dist (execute (degree-stage A as) s6)
and record-degree:
  Some ((), s8) ∈ set-dist (execute (record-staged-message dg) s7)
and assert-out:
  Some ((), s9) ∈
    set-dist
      (execute
        (assert
          (ceil-log (to-nat dg + 1) ≤
            ceil-log (maxDegree + 1))) s8)
and composition-out:
  Some ((composition-roots, composition-bs), s10) ∈
    set-dist
      (execute
        (staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []) s9)
and composition-final-out:
  Some (composition-final, s11) ∈
    set-dist
      (execute (composition-final-stage A dg composition-bs) s10)
and record-composition-final:
  Some ((), query-start) ∈
    set-dist (execute (record-staged-message composition-final) s11)
and query-out:
  Some (query-chunks, prefix-state) ∈
    set-dist

```

```

    (execute
      (checked-staged-query-program A trace-roots composition-roots
        0 i)
      query-start)
  and prefix-eq:
    prefix =
      (|spp-trace-root = fr,
        spp-trace-fri-roots = trace-roots,
        spp-trace-fri-challenges = trace-bs,
        spp-trace-final = trace-final,
        spp-alphas = as,
        spp-degree = dg,
        spp-composition-fri-roots = composition-roots,
        spp-composition-fri-challenges = composition-bs,
        spp-composition-final = composition-final,
        spp-query-chunks = query-chunks)
    using prefix-out
    by (elim checked-staged-query-challenge-prefix-program-outcomeE)
  have root-controlled:
    controlled-ro-program (trace-root-budget budgets) (trace-root-stage A)
    using controlled unfolding staged-adversary-controlled-def by simp
  have trace-final-controlled:
    controlled-ro-program (trace-final-budget budgets)
      (trace-final-stage A trace-bs)
    using controlled unfolding staged-adversary-controlled-def by blast
  have degree-controlled:
    controlled-ro-program (degree-budget budgets) (degree-stage A as)
    using controlled unfolding staged-adversary-controlled-def by blast
  have composition-final-controlled:
    controlled-ro-program (composition-final-budget budgets)
      (composition-final-stage A dg composition-bs)
    using controlled unfolding staged-adversary-controlled-def by blast
  have root-state: PState s1 = 0
    using controlled-stage-outcome-fields[OF root-controlled root-out]
    by simp
  have s2-eq:
    s2 = s1 (|
      PState := concat (PState s1) fr,
      PTranscript := PTranscript s1 @ [fr])
    by (rule record-staged-message-outcome[OF record-root])
  have trace-bound:
    0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have trace-alignment:
    PState s3 = foldl concat (PState s2) trace-roots
    using staged-trace-fri-program-alignment
      [OF controlled trace-bound trace-out]
    by simp
  have state-s3:

```

```

PState s3 = foldl concat (concat 0 fr) trace-roots
using root-state trace-alignment unfolding s2-eq by simp
have state-s4: PState s4 = PState s3
using controlled-stage-outcome-fields
  [OF trace-final-controlled final-out]
by simp
have s5-eq:
s5 = s4 (
  PState := concat (PState s4) trace-final,
  PTranscript := PTranscript s4 @ [trace-final])
by (rule record-staged-message-outcome[OF record-final])
have alpha-alignment:
PState s6 = foldl concat (PState s5) as
using staged-alpha-program-alignment[OF alpha-out] by simp
have state-s6:
PState s6 =
  foldl concat
    (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
  as
using state-s3 state-s4 alpha-alignment unfolding s5-eq by simp
have state-s7: PState s7 = PState s6
using controlled-stage-outcome-fields[OF degree-controlled degree-out]
by simp
have s8-eq:
s8 = s7 (
  PState := concat (PState s7) dg,
  PTranscript := PTranscript s7 @ [dg])
by (rule record-staged-message-outcome[OF record-degree])
have s9-eq: s9 = s8
using assert-out unfolding assert-def
by (cases
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
  (auto simp: throw-no-outcome)
have round-bound:
ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
using assert-out unfolding assert-def
by (cases
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
  (auto simp: throw-no-outcome)
have composition-bound:
0 + ceil-log (to-nat dg + 1) ≤
  length (composition-fri-budgets budgets)
using wf round-bound unfolding staged-budget-wellformed-def by simp
have composition-alignment:
PState s10 = foldl concat (PState s9) composition-roots
using staged-composition-fri-program-alignment
  [OF controlled composition-bound composition-out]
by simp
have state-s10:

```

```

PState s10 =
  foldl concat
    (concat
      (foldl concat
        (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
        as)
      dg)
    composition-roots
using state-s6 state-s7 composition-alignment
unfolding s8-eq s9-eq by simp
have state-s11: PState s11 = PState s10
using controlled-stage-outcome-fields
  [OF composition-final-controlled composition-final-out]
by simp
have query-start-eq:
  query-start = s11(|
    PState := concat (PState s11) composition-final,
    PTranscript := PTranscript s11 @ [composition-final])
by (rule record-staged-message-outcome[OF record-composition-final])
have start-state:
  PState query-start = staged-query-prefix-start-hash prefix
unfolding prefix-eq staged-query-prefix-start-hash-def
using state-s10 state-s11 unfolding query-start-eq by simp
have query-counter-start: PQueryCounter query-start = 0
proof –
have root-counter:
  PQueryCounter s1 = PQueryCounter adversary-initial-state
using controlled-stage-outcome-fields[OF root-controlled root-out]
by simp
have trace-counter:
  PQueryCounter s3 = PQueryCounter s2
using staged-trace-fri-program-alignment
  [OF controlled trace-bound trace-out]
by simp
have final-counter: PQueryCounter s4 = PQueryCounter s3
using controlled-stage-outcome-fields
  [OF trace-final-controlled final-out]
by simp
have alpha-counter: PQueryCounter s6 = PQueryCounter s5
using staged-alpha-program-alignment[OF alpha-out] by simp
have degree-counter: PQueryCounter s7 = PQueryCounter s6
using controlled-stage-outcome-fields[OF degree-controlled degree-out]
by simp
have comp-counter: PQueryCounter s10 = PQueryCounter s9
using staged-composition-fri-program-alignment
  [OF controlled composition-bound composition-out]
by simp
have comp-final-counter: PQueryCounter s11 = PQueryCounter s10
using controlled-stage-outcome-fields

```

```

    [OF composition-final-controlled composition-final-out]
  by simp
show ?thesis
  using root-counter trace-counter final-counter alpha-counter
    degree-counter comp-counter comp-final-counter
  unfolding s2-eq s5-eq s8-eq s9-eq query-start-eq by simp
qed
have query-bound:  $0 + i \leq \text{length } (\text{query-opening-budgets budgets})$ 
  using wf i-bound unfolding staged-budget-wellformed-def by simp
from checked-staged-query-program-outcome-with-raws
  [OF controlled query-bound query-out]
obtain raw-idxs query-idxs where
  len:  $\text{length } \text{query-chunks} = i$ 
  and state:
    PState prefix-state =
      state-after-query-chunks (PState query-start) query-chunks i
  and counter:
    PQueryCounter prefix-state = PQueryCounter query-start + i
  by blast
show ?thesis
  using start-state query-counter-start state counter len
  unfolding prefix-eq by simp
qed

lemma checked-staged-query-program-prefix-receive-support:
  assumes i-bound:  $i < n$ 
  and outcome:
    Some (chunks, t)  $\in$ 
      set-dist
        (execute
          (checked-staged-query-program A trace-roots composition-roots
            start n) s)
  obtains prefix-chunks prefix-state raw raw-state chunk chunk-state
    record-state suffix-chunks
  where
    Some (prefix-chunks, prefix-state)  $\in$ 
      set-dist
        (execute
          (checked-staged-query-program A trace-roots composition-roots
            start i) s)
    Some (raw, raw-state)  $\in$ 
      set-dist (execute receive-query-index-challenge prefix-state)
    Some (chunk, chunk-state)  $\in$ 
      set-dist
        (execute (query-opening-stage A (start + i) raw) raw-state)
    verifier-query-round-chunk (index (to-nat raw))
      trace-roots composition-roots chunk
    Some ((), record-state)  $\in$ 
      set-dist (execute (record-staged-messages chunk) chunk-state)

```


Some (suffix-chunks, t) ∈
set-dist
(execute
(checked-staged-query-program A trace-roots composition-roots
(Suc (start + i)) (n - Suc i) record-state)
chunks = prefix-chunks @ chunk # suffix-chunks
proof –
have *decomp*:
checked-staged-query-program A trace-roots composition-roots start n =
checked-staged-query-program A trace-roots composition-roots start i ≫=
(λprefix-chunks.
receive-query-index-challenge ≫= (λraw.
let idx = index (to-nat raw) in
query-opening-stage A (start + i) raw ≫= (λchunk.
assert
(verifier-query-round-chunk idx trace-roots composition-roots
chunk) ≫= (λ-
record-staged-messages chunk ≫= (λ-
checked-staged-query-program A trace-roots composition-roots
(Suc (start + i)) (n - Suc i) ≫= (λsuffix-chunks.
return (prefix-chunks @ chunk # suffix-chunks))))))
by (rule *checked-staged-query-program-decomp*[*OF i-bound*])
from *outcome*[*unfolded decomp*]
obtain *prefix-chunks prefix-state raw raw-state chunk chunk-state*
record-state suffix-chunks where
prefix-out:
Some (prefix-chunks, prefix-state) ∈
set-dist
(execute
(checked-staged-query-program A trace-roots composition-roots
start i) s)
and *raw-out:*
Some (raw, raw-state) ∈
set-dist (execute receive-query-index-challenge prefix-state)
and *chunk-out:*
Some (chunk, chunk-state) ∈
set-dist
(execute (query-opening-stage A (start + i) raw) raw-state)
and *chunk-shape:*
verifier-query-round-chunk (index (to-nat raw))
trace-roots composition-roots chunk
and *record-out:*
Some ((), record-state) ∈
set-dist (execute (record-staged-messages chunk) chunk-state)
and *suffix-out:*
Some (suffix-chunks, t) ∈
set-dist
(execute
(checked-staged-query-program A trace-roots composition-roots

```

      (Suc (start + i)) (n - Suc i)) record-state)
    and chunks-eq: chunks = prefix-chunks @ chunk # suffix-chunks
    by (auto simp: assert-def throw-no-outcome elim!: set-dist-bindE
        split: if-splits)
  show ?thesis
    by (rule that[OF prefix-out raw-out chunk-out chunk-shape record-out
        suffix-out chunks-eq])
qed

```

lemma *state-after-query-chunks-append-prefix:*
assumes *len-prefix:* length prefix = i
shows
 state-after-query-chunks st (prefix @ suffix) i =
 state-after-query-chunks st prefix i
using *len-prefix*
unfolding *state-after-query-chunks-def* **by** *simp*

lemma *checked-staged-query-program-prefix-receive-state-alignment:*
assumes *controlled:* staged-adversary-controlled budgets A
and *bound:* start + n ≤ length (query-opening-budgets budgets)
and *i-bound:* i < n
and *outcome:*
 Some (chunks, t) ∈
 set-dist
 (execute
 (checked-staged-query-program A trace-roots composition-roots
 start n) s)
obtains *prefix-chunks prefix-state raw raw-state chunk chunk-state*
 record-state suffix-chunks
where
 Some (prefix-chunks, prefix-state) ∈
 set-dist
 (execute
 (checked-staged-query-program A trace-roots composition-roots
 start i) s)
 Some (raw, raw-state) ∈
 set-dist (execute receive-query-index-challenge prefix-state)
 Some (chunk, chunk-state) ∈
 set-dist
 (execute (query-opening-stage A (start + i) raw) raw-state)
 verifier-query-round-chunk (index (to-nat raw))
 trace-roots composition-roots chunk
 Some ((), record-state) ∈
 set-dist (execute (record-staged-messages chunk) chunk-state)
 Some (suffix-chunks, t) ∈
 set-dist
 (execute
 (checked-staged-query-program A trace-roots composition-roots
 (Suc (start + i)) (n - Suc i)) record-state)

$chunks = prefix-chunks @ chunk \# suffix-chunks$
 $PQueryCounter\ prefix-state = PQueryCounter\ s + i$
 $PState\ prefix-state = state-after-query-chunks\ (PState\ s)\ chunks\ i$

proof –

from *checked-staged-query-program-prefix-receive-support*
 $[OF\ i-bound\ outcome]$

obtain *prefix-chunks prefix-state raw raw-state chunk chunk-state*
record-state suffix-chunks **where**

prefix-out:
 $Some\ (prefix-chunks,\ prefix-state) \in$
 $set-dist$
 $(execute$
 $(checked-staged-query-program\ A\ trace-roots\ composition-roots$
 $start\ i)\ s)$

and *raw-out:*
 $Some\ (raw,\ raw-state) \in$
 $set-dist\ (execute\ receive-query-index-challenge\ prefix-state)$

and *chunk-out:*
 $Some\ (chunk,\ chunk-state) \in$
 $set-dist$
 $(execute\ (query-opening-stage\ A\ (start + i)\ raw)\ raw-state)$

and *chunk-shape:*
 $verifier-query-round-chunk\ (index\ (to-nat\ raw))$
 $trace-roots\ composition-roots\ chunk$

and *record-out:*
 $Some\ ((),\ record-state) \in$
 $set-dist\ (execute\ (record-staged-messages\ chunk)\ chunk-state)$

and *suffix-out:*
 $Some\ (suffix-chunks,\ t) \in$
 $set-dist$
 $(execute$
 $(checked-staged-query-program\ A\ trace-roots\ composition-roots$
 $(Suc\ (start + i))\ (n - Suc\ i))\ record-state)$

and *chunks-eq:* $chunks = prefix-chunks @ chunk \# suffix-chunks$

by $(rule\ checked-staged-query-program-prefix-receive-support$
 $[OF\ i-bound\ outcome])$

have *prefix-bound:* $start + i \leq length\ (query-opening-budgets\ budgets)$
using *bound i-bound* **by** *simp*

from *checked-staged-query-program-outcome-with-raws*
 $[OF\ controlled\ prefix-bound\ prefix-out]$

obtain *raw-prefix query-prefix* **where**

len-prefix: $length\ prefix-chunks = i$

and *state-prefix:*
 $PState\ prefix-state =$
 $state-after-query-chunks\ (PState\ s)\ prefix-chunks\ i$

and *counter-prefix:*
 $PQueryCounter\ prefix-state = PQueryCounter\ s + i$

by *blast*

have *state-full:*

PState prefix-state = state-after-query-chunks (PState s) chunks i
using *state-prefix len-prefix*
unfolding *chunks-eq*
by (*simp add: state-after-query-chunks-append-prefix*)
show *?thesis*
by (*rule that[OF prefix-out raw-out chunk-out chunk-shape record-out*
suffix-out chunks-eq counter-prefix state-full])
qed

lemma *checked-staged-transcript-program-query-prefix-receive-support:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *outcome:*

Some (data, attacker-state) ∈

set-dist

(execute (checked-staged-transcript-program A)

adversary-initial-state)

obtains *prefix prefix-state raw raw-state*

where

Some (((prefix, prefix-state), raw), raw-state) ∈

set-dist

(execute (checked-staged-query-prefix-receive-with-state A i)

adversary-initial-state)

sqp-trace-root prefix = staged-trace-root data

sqp-trace-fri-roots prefix = staged-trace-fri-roots data

sqp-trace-fri-challenges prefix =

staged-trace-fri-challenges data

sqp-trace-final prefix = staged-trace-final data

sqp-alphas prefix = staged-alphas data

sqp-degree prefix = staged-degree data

sqp-composition-fri-roots prefix =

staged-composition-fri-roots data

sqp-composition-fri-challenges prefix =

staged-composition-fri-challenges data

sqp-composition-final prefix = staged-composition-final data

sqp-query-chunks prefix = take i (staged-query-chunks data)

PQueryCounter prefix-state = i

PState prefix-state =

state-after-query-chunks

(staged-query-start-hash data) (staged-query-chunks data) i

prefix-state ≤ attacker-state

raw-state ≤ attacker-state

fmlookup (HashMap raw-state)

(QueryIndexChallenge i

state-after-query-chunks

(staged-query-start-hash data) (staged-query-chunks data) i)) =

Some raw

proof –

```

from outcome obtain fr s1 s2 trace-roots trace-bs s3 trace-final s4 s5
  as s6 dg s7 s8 s9 composition-roots composition-bs s10
  composition-final s11 query-start query-chunks where
root-out:
  Some (fr, s1) ∈
    set-dist (execute (trace-root-stage A) adversary-initial-state)
and record-root:
  Some ((), s2) ∈ set-dist (execute (record-staged-message fr) s1)
and trace-out:
  Some ((trace-roots, trace-bs), s3) ∈
    set-dist
      (execute
        (staged-trace-fri-program A 0 (ceil-log clength) []) s2)
and final-out:
  Some (trace-final, s4) ∈
    set-dist (execute (trace-final-stage A trace-bs) s3)
and record-final:
  Some ((), s5) ∈
    set-dist (execute (record-staged-message trace-final) s4)
and alpha-out:
  Some (as, s6) ∈
    set-dist (execute (staged-alpha-program (length spec)) s5)
and degree-out:
  Some (dg, s7) ∈ set-dist (execute (degree-stage A as) s6)
and record-degree:
  Some ((), s8) ∈ set-dist (execute (record-staged-message dg) s7)
and assert-out:
  Some ((), s9) ∈
    set-dist
      (execute
        (assert
          (ceil-log (to-nat dg + 1) ≤
            ceil-log (maxDegree + 1))) s8)
and composition-out:
  Some ((composition-roots, composition-bs), s10) ∈
    set-dist
      (execute
        (staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []) s9)
and composition-final-out:
  Some (composition-final, s11) ∈
    set-dist
      (execute (composition-final-stage A dg composition-bs) s10)
and record-composition-final:
  Some ((), query-start) ∈
    set-dist (execute (record-staged-message composition-final) s11)
and query-out:
  Some (query-chunks, attacker-state) ∈
    set-dist

```

```

    (execute
      (checked-staged-query-program A trace-roots composition-roots
        0 rounds)
      query-start)
  and data-eq:
    data =
      (|staged-trace-root = fr,
        staged-trace-fri-roots = trace-roots,
        staged-trace-fri-challenges = trace-bs,
        staged-trace-final = trace-final,
        staged-alphas = as,
        staged-degree = dg,
        staged-composition-fri-roots = composition-roots,
        staged-composition-fri-challenges = composition-bs,
        staged-composition-final = composition-final,
        staged-query-chunks = query-chunks|)
    unfolding checked-staged-transcript-program-def Let-def
    by (auto elim!: set-dist-bindE split: prod.splits)
  have assert-out-Suc:
    Some ((), s9) ∈
      set-dist
        (execute
          (assert
            (ceil-log (Suc (to-nat dg)) ≤
              ceil-log (Suc maxDegree))) s8)
    unfolding Suc-eq-plus1
    by (rule assert-out)
  have composition-out-Suc:
    Some ((composition-roots, composition-bs), s10) ∈
      set-dist
        (execute
          (staged-composition-fri-program A dg 0
            (ceil-log (Suc (to-nat dg))) []) s9)
    unfolding Suc-eq-plus1
    by (rule composition-out)
  have query-bound: 0 + rounds ≤ length (query-opening-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  from checked-staged-query-program-prefix-receive-state-alignment
    [OF controlled query-bound i-bound query-out]
  obtain prefix-chunks prefix-state raw raw-state chunk chunk-state
    record-state suffix-chunks where
    prefix-query-out:
      Some (prefix-chunks, prefix-state) ∈
        set-dist
          (execute
            (checked-staged-query-program A trace-roots composition-roots
              0 i) query-start)
  and raw-out:
    Some (raw, raw-state) ∈

```

```

    set-dist (execute receive-query-index-challenge prefix-state)
and chunk-out:
    Some (chunk, chunk-state) ∈
    set-dist
    (execute (query-opening-stage A (0 + i) raw) raw-state)
and chunk-shape:
    verifier-query-round-chunk (index (to-nat raw))
    trace-roots composition-roots chunk
and record-out:
    Some ((), record-state) ∈
    set-dist (execute (record-staged-messages chunk) chunk-state)
and suffix-out:
    Some (suffix-chunks, attacker-state) ∈
    set-dist
    (execute
    (checked-staged-query-program A trace-roots composition-roots
    (Suc (0 + i)) (rounds - Suc i)) record-state)
and chunks-eq: query-chunks = prefix-chunks @ chunk # suffix-chunks
and counter-prefix:
    PQueryCounter prefix-state = PQueryCounter query-start + i
and state-prefix:
    PState prefix-state =
    state-after-query-chunks (PState query-start) query-chunks i
by (rule checked-staged-query-program-prefix-receive-state-alignment
    [OF controlled query-bound i-bound query-out])
let ?prefix =
    (|sqp-trace-root = fr,
    sqp-trace-fri-roots = trace-roots,
    sqp-trace-fri-challenges = trace-bs,
    sqp-trace-final = trace-final,
    sqp-alphas = as,
    sqp-degree = dg,
    sqp-composition-fri-roots = composition-roots,
    sqp-composition-fri-challenges = composition-bs,
    sqp-composition-final = composition-final,
    sqp-query-chunks = prefix-chunks|)
have alpha-prefix-out:
    Some ((fr, trace-roots, trace-bs, trace-final), s5) ∈
    set-dist
    (execute (staged-alpha-prefix-program A) adversary-initial-state)
proof -
have after-record-final:
    Some ((fr, trace-roots, trace-bs, trace-final), s5) ∈
    set-dist
    (execute
    (record-staged-message trace-final ≫=
    (λ-. return (fr, trace-roots, trace-bs, trace-final))))
    s4)
by (rule-tac x=() and t=s5 in set-dist-bindI)

```

```

    (rule record-final, simp)
have after-final:
  Some ((fr, trace-roots, trace-bs, trace-final), s5) ∈
    set-dist
    (execute
      (trace-final-stage A trace-bs >>=
        (λtrace-final.
          record-staged-message trace-final >>=
            (λ-. return (fr, trace-roots, trace-bs, trace-final))))))
    s3)
by (rule-tac x=trace-final and t=s4 in set-dist-bindI)
    (rule final-out, rule after-record-final)
have after-trace:
  Some ((fr, trace-roots, trace-bs, trace-final), s5) ∈
    set-dist
    (execute
      (staged-trace-fri-program A 0 (ceil-log clength) [] >>=
        (λ(trace-roots, trace-bs).
          trace-final-stage A trace-bs >>=
            (λtrace-final.
              record-staged-message trace-final >>=
                (λ-. return
                  (fr, trace-roots, trace-bs, trace-final))))))
    s2)
apply (rule-tac x=(trace-roots, trace-bs) and t=s3
  in set-dist-bindI)
apply (rule trace-out)
apply (simp add: after-final)
done
have after-record-root:
  Some ((fr, trace-roots, trace-bs, trace-final), s5) ∈
    set-dist
    (execute
      (record-staged-message fr >>=
        (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
          (λ(trace-roots, trace-bs).
            trace-final-stage A trace-bs >>=
              (λtrace-final.
                record-staged-message trace-final >>=
                  (λ-. return
                    (fr, trace-roots, trace-bs, trace-final))))))
    s1)
by (rule-tac x=() and t=s2 in set-dist-bindI)
    (rule record-root, rule after-trace)
show ?thesis
unfolding staged-alpha-prefix-program-def
by (rule-tac x=fr and t=s1 in set-dist-bindI)
    (rule root-out, rule after-record-root)
qed

```



```

have prefix-program-out:
  Some (?prefix, prefix-state) ∈
    set-dist
      (execute
        (checked-staged-query-challenge-prefix-program A i)
        adversary-initial-state)
proof –
  have after-prefix-query:
    Some (?prefix, prefix-state) ∈
      set-dist
        (execute
          (checked-staged-query-program A trace-roots composition-roots
            0 i >>=
            (λquery-chunks.
              return
                (|sqp-trace-root = fr,
                  sqp-trace-fri-roots = trace-roots,
                  sqp-trace-fri-challenges = trace-bs,
                  sqp-trace-final = trace-final,
                  sqp-alphas = as,
                  sqp-degree = dg,
                  sqp-composition-fri-roots = composition-roots,
                  sqp-composition-fri-challenges = composition-bs,
                  sqp-composition-final = composition-final,
                  sqp-query-chunks = query-chunks|)))
            query-start)
          by (rule-tac x=prefix-chunks and t=prefix-state in set-dist-bindI)
            (rule prefix-query-out, simp)
  have after-record-composition-final:
    Some (?prefix, prefix-state) ∈
      set-dist
        (execute
          (record-staged-message composition-final >>=
            (λ-.
              checked-staged-query-program A trace-roots composition-roots
                0 i >>=
                (λquery-chunks.
                  return
                    (|sqp-trace-root = fr,
                      sqp-trace-fri-roots = trace-roots,
                      sqp-trace-fri-challenges = trace-bs,
                      sqp-trace-final = trace-final,
                      sqp-alphas = as,
                      sqp-degree = dg,
                      sqp-composition-fri-roots = composition-roots,
                      sqp-composition-fri-challenges = composition-bs,
                      sqp-composition-final = composition-final,
                      sqp-query-chunks = query-chunks|))))
            s11)

```

```

by (rule-tac x=() and t=query-start in set-dist-bindI)
  (rule record-composition-final, rule after-prefix-query)
have after-composition-final:
  Some (?prefix, prefix-state) ∈
    set-dist
    (execute
      (composition-final-stage A dg composition-bs ≫=
        (λcomposition-final.
          record-staged-message composition-final ≫=
            (λ-.
              checked-staged-query-program A trace-roots
                composition-roots 0 i ≫=
                  (λquery-chunks.
                    return
                      (|spp-trace-root = fr,
                        spp-trace-fri-roots = trace-roots,
                        spp-trace-fri-challenges = trace-bs,
                        spp-trace-final = trace-final,
                        spp-alphas = as,
                        spp-degree = dg,
                        spp-composition-fri-roots = composition-roots,
                        spp-composition-fri-challenges = composition-bs,
                        spp-composition-final = composition-final,
                        spp-query-chunks = query-chunks|))))))
    s10)
by (rule-tac x=composition-final and t=s11 in set-dist-bindI)
  (rule composition-final-out, rule after-record-composition-final)
have after-composition:
  Some (?prefix, prefix-state) ∈
    set-dist
    (execute
      (staged-composition-fri-program A dg 0
        (ceil-log (Suc (to-nat dg))) [] ≫=
          (λ(composition-roots, composition-bs).
            composition-final-stage A dg composition-bs ≫=
              (λcomposition-final.
                record-staged-message composition-final ≫=
                  (λ-.
                    checked-staged-query-program A trace-roots
                      composition-roots 0 i ≫=
                        (λquery-chunks.
                          return
                            (|spp-trace-root = fr,
                              spp-trace-fri-roots = trace-roots,
                              spp-trace-fri-challenges = trace-bs,
                              spp-trace-final = trace-final,
                              spp-alphas = as,
                              spp-degree = dg,
                              spp-composition-fri-roots = composition-roots,

```

```

      sqp-composition-fri-challenges = composition-bs,
      sqp-composition-final = composition-final,
      sqp-query-chunks = query-chunks))))))
s9)
apply (rule-tac x=(composition-roots, composition-bs) and t=s10
  in set-dist-bindI)
  apply (rule composition-out-Suc)
apply (simp add: after-composition-final)
done
have after-assert:
  Some (?prefix, prefix-state) ∈
    set-dist
    (execute
      (assert (ceil-log (Suc (to-nat dg)) ≤
        ceil-log (Suc maxDegree)) >>=
      (λ-.
        staged-composition-fri-program A dg 0
        (ceil-log (Suc (to-nat dg))) [] >>=
      (λ(composition-roots, composition-bs).
        composition-final-stage A dg composition-bs >>=
      (λcomposition-final.
        record-staged-message composition-final >>=
      (λ-.
        checked-staged-query-program A trace-roots
        composition-roots 0 i >>=
      (λquery-chunks.
        return
          (|sqp-trace-root = fr,
            sqp-trace-fri-roots = trace-roots,
            sqp-trace-fri-challenges = trace-bs,
            sqp-trace-final = trace-final,
            sqp-alphas = as,
            sqp-degree = dg,
            sqp-composition-fri-roots = composition-roots,
            sqp-composition-fri-challenges = composition-bs,
            sqp-composition-final = composition-final,
            sqp-query-chunks = query-chunks)))))))))
s8)
apply (rule-tac x=() and t=s9 in set-dist-bindI)
  apply (rule assert-out-Suc)
apply (simp add: after-composition)
done
have after-record-degree:
  Some (?prefix, prefix-state) ∈
    set-dist
    (execute
      (record-staged-message dg >>=
      (λ-.
        assert (ceil-log (Suc (to-nat dg)) ≤

```

```

      ceil-log (Suc maxDegree)) >>=
    (λ-.
      staged-composition-fri-program A dg 0
      (ceil-log (Suc (to-nat dg))) [] >>=
      (λ(composition-roots, composition-bs).
        composition-final-stage A dg composition-bs >>=
        (λcomposition-final.
          record-staged-message composition-final >>=
          (λ-.
            checked-staged-query-program A trace-roots
            composition-roots 0 i >>=
            (λquery-chunks.
              return
              (λsqp-trace-root = fr,
                sqp-trace-fri-roots = trace-roots,
                sqp-trace-fri-challenges = trace-bs,
                sqp-trace-final = trace-final,
                sqp-alphas = as,
                sqp-degree = dg,
                sqp-composition-fri-roots = composition-roots,
                sqp-composition-fri-challenges = composition-bs,
                sqp-composition-final = composition-final,
                sqp-query-chunks = query-chunks)))))))))
    s7)
  apply (rule-tac x=() and t=s8 in set-dist-bindI)
  apply (rule record-degree)
  apply (simp add: after-assert)
  done
have after-degree:
  Some (?prefix, prefix-state) ∈
  set-dist
  (execute
    (degree-stage A as >>=
      (λdg.
        record-staged-message dg >>=
        (λ-.
          assert (ceil-log (Suc (to-nat dg)) ≤
            ceil-log (Suc maxDegree)) >>=
          (λ-.
            staged-composition-fri-program A dg 0
            (ceil-log (Suc (to-nat dg))) [] >>=
            (λ(composition-roots, composition-bs).
              composition-final-stage A dg composition-bs >>=
              (λcomposition-final.
                record-staged-message composition-final >>=
                (λ-.
                  checked-staged-query-program A trace-roots
                  composition-roots 0 i >>=
                  (λquery-chunks.

```

```

    return
    (|sqp-trace-root = fr,
     sqp-trace-fri-roots = trace-roots,
     sqp-trace-fri-challenges = trace-bs,
     sqp-trace-final = trace-final,
     sqp-alphas = as,
     sqp-degree = dg,
     sqp-composition-fri-roots = composition-roots,
     sqp-composition-fri-challenges = composition-bs,
     sqp-composition-final = composition-final,
     sqp-query-chunks = query-chunks|)))))))))
s6)
apply (rule-tac x=dg and t=s7 in set-dist-bindI)
apply (rule degree-out)
apply (simp add: after-record-degree)
done
have after-alpha:
  Some (?prefix, prefix-state) ∈
    set-dist
    (execute
      (staged-alpha-program (length spec) >>=
        (λas.
          degree-stage A as >>=
            (λdg.
              record-staged-message dg >>=
                (λ-.
                  assert (ceil-log (Suc (to-nat dg)) ≤
                    ceil-log (Suc maxDegree)) >>=
                      (λ-.
                        staged-composition-fri-program A dg 0
                          (ceil-log (Suc (to-nat dg))) [] >>=
                            (λ(composition-roots, composition-bs).
                              composition-final-stage A dg composition-bs >>=
                                (λcomposition-final.
                                  record-staged-message composition-final >>=
                                    (λ-.
                                      checked-staged-query-program A trace-roots
                                        composition-roots 0 i >>=
                                          (λquery-chunks.
                                            return
                                            (|sqp-trace-root = fr,
                                             sqp-trace-fri-roots = trace-roots,
                                             sqp-trace-fri-challenges = trace-bs,
                                             sqp-trace-final = trace-final,
                                             sqp-alphas = as,
                                             sqp-degree = dg,
                                             sqp-composition-fri-roots = composition-roots,
                                             sqp-composition-fri-challenges = composition-bs,
                                             sqp-composition-final = composition-final,

```

```

s5)
  apply (rule-tac x=as and t=s6 in set-dist-bindI)
  apply (rule alpha-out)
  apply (simp add: after-degree)
done
show ?thesis
  unfolding checked-staged-query-challenge-prefix-program-def Let-def
  apply (rule-tac x=(fr, trace-roots, trace-bs, trace-final) and t=s5
    in set-dist-bindI)
  apply (rule alpha-prefix-out)
  apply (simp add: after-alpha)
done
qed
have prefix-with-state-out:
  Some ((?prefix, prefix-state), prefix-state) ∈
  set-dist
  (execute (checked-staged-query-prefix-with-state A i)
    adversary-initial-state)
  unfolding checked-staged-query-prefix-with-state-def
  by (rule set-dist-bindI[where x=?prefix and t=prefix-state,
    OF prefix-program-out get-bind-return-state-support])
have prefix-receive-out:
  Some ((?prefix, prefix-state), raw), raw-state) ∈
  set-dist
  (execute (checked-staged-query-prefix-receive-with-state A i)
    adversary-initial-state)
  unfolding checked-staged-query-prefix-receive-with-state-def
  by (intro set-dist-bindI[OF prefix-with-state-out]
    set-dist-bindI[OF raw-out])
  simp
have root-controlled:
  controlled-ro-program (trace-root-budget budgets) (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def by simp
have trace-final-controlled:
  controlled-ro-program (trace-final-budget budgets)
  (trace-final-stage A trace-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have degree-controlled:
  controlled-ro-program (degree-budget budgets) (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def by blast
have composition-final-controlled:
  controlled-ro-program (composition-final-budget budgets)
  (composition-final-stage A dg composition-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have root-state: PState s1 = 0
  using controlled-stage-outcome-fields[OF root-controlled root-out]
  by simp
have s2-eq:

```

```

s2 = s1(
  PState := concat (PState s1) fr,
  PTranscript := PTranscript s1 @ [fr])
by (rule record-staged-message-outcome[OF record-root])
have trace-bound:
  0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
  using wf unfolding staged-budget-wellformed-def by simp
have trace-alignment:
  PState s3 = foldl concat (PState s2) trace-roots
  using staged-trace-fri-program-alignment
  [OF controlled trace-bound trace-out]
  by simp
have state-s3:
  PState s3 = foldl concat (concat 0 fr) trace-roots
  using root-state trace-alignment unfolding s2-eq by simp
have state-s4: PState s4 = PState s3
  using controlled-stage-outcome-fields
  [OF trace-final-controlled final-out]
  by simp
have s5-eq:
  s5 = s4(
    PState := concat (PState s4) trace-final,
    PTranscript := PTranscript s4 @ [trace-final])
  by (rule record-staged-message-outcome[OF record-final])
have alpha-alignment:
  PState s6 = foldl concat (PState s5) as
  using staged-alpha-program-alignment[OF alpha-out] by simp
have state-s6:
  PState s6 =
    foldl concat
      (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
    as
  using state-s3 state-s4 alpha-alignment unfolding s5-eq by simp
have state-s7: PState s7 = PState s6
  using controlled-stage-outcome-fields[OF degree-controlled degree-out]
  by simp
have s8-eq:
  s8 = s7(
    PState := concat (PState s7) dg,
    PTranscript := PTranscript s7 @ [dg])
  by (rule record-staged-message-outcome[OF record-degree])
have s9-eq: s9 = s8
  using assert-out unfolding assert-def
  by (cases
    ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
  using assert-out unfolding assert-def

```

```

by (cases
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
(auto simp: throw-no-outcome)
have composition-bound:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
using wf round-bound unfolding staged-budget-wellformed-def by simp
have composition-alignment:
  PState s10 = foldl concat (PState s9) composition-roots
using staged-composition-fri-program-alignment
  [OF controlled composition-bound composition-out]
by simp
have state-s10:
  PState s10 =
    foldl concat
      (concat
        (foldl concat
          (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
          as)
        dg)
      composition-roots
using state-s6 state-s7 composition-alignment
unfolding s8-eq s9-eq by simp
have state-s11: PState s11 = PState s10
using controlled-stage-outcome-fields
  [OF composition-final-controlled composition-final-out]
by simp
have query-start-eq:
  query-start = s11(|
    PState := concat (PState s11) composition-final,
    PTranscript := PTranscript s11 @ [composition-final]|)
by (rule record-staged-message-outcome[OF record-composition-final])
have state-query-start:
  PState query-start =
    concat
      (foldl concat
        (concat
          (foldl concat
            (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
            as)
          dg)
        composition-roots)
      composition-final
using state-s10 state-s11 unfolding query-start-eq by simp
have start-state:
  PState query-start = staged-query-start-hash data
unfolding data-eq staged-query-start-hash-def
  staged-composition-fri-start-hash-def staged-trace-fri-start-hash-def
using state-query-start by simp

```



```

have query-counter-start: PQueryCounter query-start = 0
proof -
  have root-fields:
    PQueryCounter s1 = PQueryCounter adversary-initial-state
    using controlled-stage-outcome-fields[OF root-controlled root-out]
    by simp
  have trace-counters:
    PQueryCounter s3 = PQueryCounter s2
    using staged-trace-fri-program-alignment
      [OF controlled trace-bound trace-out]
    by simp
  have final-counter: PQueryCounter s4 = PQueryCounter s3
    using controlled-stage-outcome-fields
      [OF trace-final-controlled final-out]
    by simp
  have alpha-counter: PQueryCounter s6 = PQueryCounter s5
    using staged-alpha-program-alignment[OF alpha-out] by simp
  have degree-counter: PQueryCounter s7 = PQueryCounter s6
    using controlled-stage-outcome-fields[OF degree-controlled degree-out]
    by simp
  have comp-counter: PQueryCounter s10 = PQueryCounter s9
    using staged-composition-fri-program-alignment
      [OF controlled composition-bound composition-out]
    by simp
  have comp-final-counter: PQueryCounter s11 = PQueryCounter s10
    using controlled-stage-outcome-fields
      [OF composition-final-controlled composition-final-out]
    by simp
  show ?thesis
    using root-fields trace-counters final-counter alpha-counter
      degree-counter comp-counter comp-final-counter
    unfolding s2-eq s5-eq s8-eq s9-eq query-start-eq by simp
qed
have prefix-counter: PQueryCounter prefix-state = i
  using counter-prefix query-counter-start by simp
have prefix-state-eq:
  PState prefix-state =
    state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i
  using state-prefix start-state unfolding data-eq by simp
have prefix-query-bound:  $0 + i \leq \text{length } (\text{query-opening-budgets budgets})$ 
  using query-bound i-bound by simp
have prefix-len:  $\text{length } \text{prefix-chunks} = i$ 
proof -
  from checked-staged-query-program-outcome-with-raws
    [OF controlled prefix-query-bound prefix-query-out]
  show ?thesis
    by blast
qed

```

```

have prefix-chunks-eq:
  prefix-chunks = take i query-chunks
  using chunks-eq prefix-len by simp
have receive-lookup:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
using receive-query-index-challenge-outcome[OF raw-out]
  prefix-counter prefix-state-eq
by simp
have stage-controlled:
  controlled-ro-program (query-opening-budgets budgets ! i)
    (query-opening-stage A i raw)
using controlled wf i-bound
unfolding staged-adversary-controlled-def staged-budget-wellformed-def
by simp
have raw-chunk-ext: raw-state  $\leq$  chunk-state
proof –
  have chunk-out-i:
    Some (chunk, chunk-state)  $\in$ 
      set-dist (execute (query-opening-stage A i raw) raw-state)
    using chunk-out by simp
  show ?thesis
    using controlled-ro-program-extension[OF stage-controlled] chunk-out-i
    unfolding hash-extension-preserving-def by blast
qed
have prefix-raw-ext: prefix-state  $\leq$  raw-state
  by (rule receive-query-index-challenge-extends[OF raw-out])
have record-state-eq:
  record-state = chunk-state(
    PState := foldl concat (PState chunk-state) chunk,
    PTranscript := PTranscript chunk-state @ chunk)
  by (rule record-staged-messages-outcome[OF record-out])
have chunk-record-ext: chunk-state  $\leq$  record-state
  unfolding record-state-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have suffix-bound:
  Suc (0 + i) + (rounds - Suc i)  $\leq$ 
    length (query-opening-budgets budgets)
  using query-bound i-bound by simp
have suffix-target:
  hash-target-program {}
    (sum-list
      (take (rounds - Suc i)
        (drop (Suc (0 + i)) (query-opening-budgets budgets))) +
      (rounds - Suc i))
    (checked-staged-query-program A trace-roots composition-roots
      (Suc (0 + i)) (rounds - Suc i))

```

```

    by (rule hash-target-program-checked-staged-query-program
        [OF controlled suffix-bound])
  have suffix-ext: record-state  $\leq$  attacker-state
    using hash-target-program-extension[OF suffix-target] suffix-out
    unfolding hash-extension-preserving-def by blast
  have raw-ext: raw-state  $\leq$  attacker-state
    by (rule hash-ext-trans[OF raw-chunk-ext])
      (rule hash-ext-trans[OF chunk-record-ext suffix-ext])
  have prefix-ext: prefix-state  $\leq$  attacker-state
    by (rule hash-ext-trans[OF prefix-raw-ext raw-ext])
  show ?thesis
    by (rule that[OF prefix-receive-out])
      (use data-eq prefix-chunks-eq prefix-counter prefix-state-eq prefix-ext
        raw-ext receive-lookup in simp-all)
qed

lemma checked-staged-transcript-program-query-prefix-receive-final-lookup-support:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound:  $i < \text{rounds}$ 
    and outcome:
      Some (data, attacker-state)  $\in$ 
        set-dist
          (execute (checked-staged-transcript-program A)
            adversary-initial-state)
    and verifier:
      Some (result, final-state)  $\in$ 
        set-dist
          (execute verify-monad
            (verifier-state-from-adversary attacker-state
              (staged-proof-transcript data))))
    and final-lookup:
      fmlookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data) (staged-query-chunks data) i)) =
        Some raw
  obtains prefix prefix-state raw-state where
    Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)

proof -
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  show ?thesis
  proof (rule checked-staged-transcript-program-query-prefix-receive-support
    [OF wf controlled i-bound outcome])

```

```

fix prefix prefix-state raw' raw-state
assume prefix-receive:
  Some (((prefix, prefix-state), raw'), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
and raw-ext: raw-state ≤ attacker-state
and raw-lookup:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw'
have raw-ext-s: raw-state ≤ ?s
  by (rule hash-extends-verifier-state-from-adversary-right[OF raw-ext])
have s-ext-final: ?s ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
have raw-ext-final: raw-state ≤ final-state
  by (rule hash-ext-trans[OF raw-ext-s s-ext-final])
have final-lookup-raw':
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw'
  by (rule hash-extension-lookup[OF raw-lookup raw-ext-final])
have raw-eq: raw' = raw
  using final-lookup final-lookup-raw' by simp
have prefix-receive-raw:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
  using prefix-receive raw-eq by simp
show ?thesis
  by (rule that[OF prefix-receive-raw])
qed
qed

lemma checked-staged-transcript-program-query-prefix-receive-final-lookup-support-with-alphas:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and outcome:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
and verifier:

```

```

    Some (result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
  and final-lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw
  obtains prefix prefix-state raw-state where
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
    sqp-alphas prefix = staged-alphas data
  proof —
    let ?s =
      verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)
    show ?thesis
  proof (rule checked-staged-transcript-program-query-prefix-receive-support
    [OF wf controlled i-bound outcome])
    fix prefix prefix-state raw' raw-state
    assume prefix-receive:
      Some (((prefix, prefix-state), raw'), raw-state) ∈
        set-dist
          (execute (checked-staged-query-prefix-receive-with-state A i)
            adversary-initial-state)
    and alphas-eq: sqp-alphas prefix = staged-alphas data
    and raw-ext: raw-state ≤ attacker-state
    and raw-lookup:
      fmlookup (HashMap raw-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw'
    have raw-ext-s: raw-state ≤ ?s
      by (rule hash-extends-verifier-state-from-adversary-right[OF raw-ext])
    have s-ext-final: ?s ≤ final-state
      by (rule verify-monad-hash-extends[OF verifier])
    have raw-ext-final: raw-state ≤ final-state
      by (rule hash-ext-trans[OF raw-ext-s s-ext-final])
    have final-lookup-raw':
      fmlookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data) (staged-query-chunks data) i)) =

```

```

    Some raw'
  by (rule hash-extension-lookup[OF raw-lookup raw-ext-final])
have raw-eq: raw' = raw
  using final-lookup final-lookup-raw' by simp
have prefix-receive-raw:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
  using prefix-receive raw-eq by simp
show ?thesis
  by (rule that[OF prefix-receive-raw alphas-eq])
qed
qed

end

end

```

```

theory Soundness-Staged-Query
imports Soundness-Staged-Query-Prefix
begin

```

Dynamic staged query target events and probability bounds.

```

context soundness
begin

```

```

definition checked-staged-query-prefix-dynamic-index-hit
:: ('f staged-query-prefix-data ⇒ 'f protocol-channel ⇒
  nat set) ⇒
  (((('f staged-query-prefix-data × 'f protocol-channel) × 'f) ×
    'f protocol-channel) option ⇒ bool
where
  checked-staged-query-prefix-dynamic-index-hit B out ⟷
    (case out of
      None ⇒ False
    | Some (((prefix, prefix-state), raw), -) ⇒
      index (to-nat raw) ∈ B prefix prefix-state)

```

```

definition staged-partial-query-target
:: 'f staged-proof-data ⇒ 'f protocol-channel ⇒ nat set
where
  staged-partial-query-target data attacker-state =
    query-header-supported-partial-union-good-sets
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
    query-sampling-success-space
      (staged-trace-root data)

```

(staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)

definition *staged-query-prefix-matches-data*

$:: \text{nat} \Rightarrow 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ staged-proof-data} \Rightarrow$
 bool

where

$\text{staged-query-prefix-matches-data } i \text{ prefix data} \longleftrightarrow$
 $\text{sqp-trace-root prefix} = \text{staged-trace-root data} \wedge$
 $\text{sqp-trace-fri-roots prefix} = \text{staged-trace-fri-roots data} \wedge$
 $\text{sqp-trace-fri-challenges prefix} =$
 $\quad \text{staged-trace-fri-challenges data} \wedge$
 $\text{sqp-trace-final prefix} = \text{staged-trace-final data} \wedge$
 $\text{sqp-alphas prefix} = \text{staged-alphas data} \wedge$
 $\text{sqp-degree prefix} = \text{staged-degree data} \wedge$
 $\text{sqp-composition-fri-roots prefix} =$
 $\quad \text{staged-composition-fri-roots data} \wedge$
 $\text{sqp-composition-fri-challenges prefix} =$
 $\quad \text{staged-composition-fri-challenges data} \wedge$
 $\text{sqp-composition-final prefix} =$
 $\quad \text{staged-composition-final data} \wedge$
 $\text{sqp-query-chunks prefix} = \text{take } i \text{ (staged-query-chunks data)}$

lemma *staged-query-start-hash-of-prefix-completion:*

assumes *match*: $\text{staged-query-prefix-matches-data } i \text{ prefix data}$

shows $\text{staged-query-start-hash data} =$

$\text{staged-query-prefix-start-hash prefix}$

using *match*

unfolding *staged-query-prefix-matches-data-def*

staged-query-start-hash-def staged-composition-fri-start-hash-def

staged-trace-fri-start-hash-def staged-query-prefix-start-hash-def

by *simp*

definition *staged-query-prefix-partial-query-target*

$:: \text{nat} \Rightarrow 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow$
 nat set

where

$\text{staged-query-prefix-partial-query-target } i \text{ prefix prefix-state} =$
 $\{ \text{idx}. \exists \text{ data attacker-state.}$
 $\quad \text{staged-query-prefix-matches-data } i \text{ prefix data} \wedge$
 $\quad \text{prefix-state} \leq \text{attacker-state} \wedge$
 $\quad \text{idx} \in \text{staged-partial-query-target data attacker-state} \}$

definition *staged-query-prefix-committed-query-target*

$:: 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{nat set}$

where

staged-query-prefix-committed-query-target prefix prefix-state =
query-header-committed-union-good-sets prefix-state
query-sampling-success-space
(sqp-trace-root prefix)
(sqp-alphas prefix)
(sqp-composition-fri-roots prefix)

definition *staged-query-prefix-length-committed-table-candidates*

:: 'f staged-query-prefix-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$
('f list $\times$ 'f list) set

where

staged-query-prefix-length-committed-table-candidates prefix prefix-state =
{(trace-table, composition-table).
merkle-root-binds-table (sqp-trace-root prefix) trace-table
prefix-state $\wedge$
sqp-composition-fri-roots prefix $\neq \square \wedge$
merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
composition-table prefix-state $\wedge$
*length trace-table = clength * scale $\wedge$*
*length composition-table = clength * scale}*

definition *staged-query-prefix-length-committed-query-target*

:: 'f staged-query-prefix-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ nat set

where

staged-query-prefix-length-committed-query-target prefix prefix-state =
{idx $\in$ query-sample-space.
 $\exists$ trace-table composition-table.
(trace-table, composition-table) $\in$
staged-query-prefix-length-committed-table-candidates prefix
prefix-state $\wedge$
idx $\in$ query-sampling-success-space trace-table
composition-table (sqp-alphas prefix)}

definition *staged-query-prefix-clean-length-committed-query-target*

:: 'f staged-query-prefix-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ nat set

where

staged-query-prefix-clean-length-committed-query-target prefix
prefix-state =
(if hash-map-output-collision prefix-state then {}
else staged-query-prefix-length-committed-query-target prefix
prefix-state)

definition *checked-staged-query-prefix-hash-map-output-collision*

:: (((('f staged-query-prefix-data $\times$ 'f protocol-channel) $\times$ 'f) $\times$
'f protocol-channel) option $\Rightarrow$ bool

where

checked-staged-query-prefix-hash-map-output-collision out $\longleftrightarrow$
(case out of

$None \Rightarrow False$
 $| Some (((-, prefix-state), -), -) \Rightarrow$
 $hash-map-output-collision\ prefix-state)$

definition *staged-query-prefix-committed-roots-clean*

$:: 'f\ staged-query-prefix-data \Rightarrow 'f\ protocol-channel \Rightarrow bool$

where

$staged-query-prefix-committed-roots-clean\ prefix\ prefix-state \longleftrightarrow$
 $\neg merkle-root-binding-collision\ (sqp-trace-root\ prefix)$
 $prefix-state \wedge$
 $(\forall rt. sqp-composition-fri-roots\ prefix \neq [] \longrightarrow$
 $rt = hd\ (sqp-composition-fri-roots\ prefix) \longrightarrow$
 $\neg merkle-root-binding-collision\ rt\ prefix-state)$

definition *staged-query-prefix-clean-committed-query-target*

$:: 'f\ staged-query-prefix-data \Rightarrow 'f\ protocol-channel \Rightarrow nat\ set$

where

$staged-query-prefix-clean-committed-query-target\ prefix\ prefix-state =$
 $(if\ staged-query-prefix-committed-roots-clean\ prefix\ prefix-state$
 $then\ staged-query-prefix-committed-query-target\ prefix\ prefix-state$
 $else\ \{\})$

definition *checked-staged-query-prefix-committed-root-collision*

$:: (((('f\ staged-query-prefix-data \times 'f\ protocol-channel) \times 'f) \times$
 $'f\ protocol-channel)\ option \Rightarrow bool$

where

$checked-staged-query-prefix-committed-root-collision\ out \longleftrightarrow$
 $(case\ out\ of$
 $None \Rightarrow False$
 $| Some (((prefix, prefix-state), -), -) \Rightarrow$
 $\neg staged-query-prefix-committed-roots-clean\ prefix\ prefix-state)$

definition *staged-security-with-data-state-query-committed-prefix-hit*

$:: 'f\ staged-adversary \Rightarrow$

$((('f\ staged-proof-data \times 'f\ protocol-channel) \times unit\ list) \times$
 $'f\ protocol-channel)\ option \Rightarrow bool$

where

$staged-security-with-data-state-query-committed-prefix-hit\ A\ out \longleftrightarrow$
 $(case\ out\ of$
 $None \Rightarrow False$
 $| Some (((data, attacker-state), -), -) \Rightarrow$
 $(\exists i\ prefix\ prefix-state\ raw\ raw-state.$
 $i < rounds \wedge$
 $Some\ (data, attacker-state) \in$
 $set-dist$
 $(execute\ (checked-staged-transcript-program\ A)$
 $adversary-initial-state) \wedge$
 $Some\ (((prefix, prefix-state), raw), raw-state) \in$
 $set-dist$

$(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i) \\
\text{adversary-initial-state}) \wedge \\
\text{checked-staged-query-prefix-dynamic-index-hit} \\
(\lambda \text{prefix prefix-state.} \\
\text{staged-query-prefix-committed-query-target prefix prefix-state}) \\
(\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}))))$

definition *staged-security-with-data-state-query-length-committed-prefix-hit*

$:: 'f \text{ staged-adversary} \Rightarrow \\
((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}) \times \\
'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-state-query-length-committed-prefix-hit } A \ \text{out} \longleftrightarrow \\
(\text{case out of} \\
\text{None} \Rightarrow \text{False} \\
| \text{Some } (((\text{data}, \text{attacker-state}), -), -) \Rightarrow \\
(\exists i \text{ prefix prefix-state raw raw-state.} \\
i < \text{rounds} \wedge \\
\text{Some } (\text{data}, \text{attacker-state}) \in \\
\text{set-dist} \\
(\text{execute } (\text{checked-staged-transcript-program } A) \\
\text{adversary-initial-state}) \wedge \\
\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in \\
\text{set-dist} \\
(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i) \\
\text{adversary-initial-state}) \wedge \\
\text{checked-staged-query-prefix-dynamic-index-hit} \\
\text{staged-query-prefix-length-committed-query-target} \\
(\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}))))))$

definition *staged-security-with-data-state-query-tree-output-hit*

$:: 'f \text{ staged-adversary} \Rightarrow \\
((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}) \times \\
'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-state-query-tree-output-hit } A \ \text{out} \longleftrightarrow \\
(\text{case out of} \\
\text{None} \Rightarrow \text{False} \\
| \text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state}) \Rightarrow \\
(\text{let } s = \\
\text{verifier-state-from-adversary attacker-state} \\
(\text{staged-proof-transcript data}) \\
\text{in} \\
(\exists i \text{ prefix prefix-state raw raw-state trace-table} \\
\text{composition-table as query-idxs trace-tree composition-tree.} \\
i < \text{rounds} \wedge \\
\text{Some } (\text{data}, \text{attacker-state}) \in \\
\text{set-dist} \\
(\text{execute } (\text{checked-staged-transcript-program } A)$

$\text{adversary-initial-state}) \wedge$
 $\text{Some (result, final-state)} \in$
 $\text{set-dist (execute verify-monad s)} \wedge$
 $\text{accepted-with-bound-tables s (Some (result, final-state))}$
 $\text{trace-table composition-table as query-idxs} \wedge$
 $\text{fmlookup (HashMap final-state)}$
 $(\text{QueryIndexChallenge } i$
 $(\text{state-after-query-chunks}$
 $(\text{staged-query-start-hash data})$
 $(\text{staged-query-chunks data } i)) =$
 $\text{Some raw} \wedge$
 $\text{index (to-nat raw)} \in$
 $\text{query-sampling-success-space trace-table composition-table}$
 $(\text{staged-alphas data}) \wedge$
 $\text{Some (((prefix, prefix-state), raw), raw-state)} \in$
 set-dist
 $(\text{execute (checked-staged-query-prefix-receive-with-state A } i)$
 $\text{adversary-initial-state}) \wedge$
 $\text{sqp-composition-fri-roots prefix} \neq [] \wedge$
 $\text{created-tree trace-table trace-tree final-state} \wedge$
 $\text{sqp-trace-root prefix} = \text{value trace-tree} \wedge$
 $\text{created-tree composition-table composition-tree final-state} \wedge$
 $\text{hd (sqp-composition-fri-roots prefix)} = \text{value composition-tree} \wedge$
 $\text{hash-map-new-output-hit}$
 $(\text{set-tree trace-tree} \cup \text{set-tree composition-tree})$
 $\text{prefix-state final-state}))$

lemma *staged-partial-query-target-subset-prefix-completion:*

assumes *match: staged-query-prefix-matches-data i prefix data*

and ext: *prefix-state $\leq$ attacker-state*

shows

staged-partial-query-target data attacker-state $\subseteq$

staged-query-prefix-partial-query-target i prefix prefix-state

unfolding *staged-query-prefix-partial-query-target-def*

using *match ext* **by** *blast*

lemma *staged-query-prefix-partial-query-target-subset:*

staged-query-prefix-partial-query-target i prefix prefix-state $\subseteq$

query-sample-space

unfolding *staged-query-prefix-partial-query-target-def*

staged-partial-query-target-def

using *query-header-supported-partial-union-good-sets-subset* **by** *blast*

lemma *staged-query-prefix-committed-query-target-subset:*

staged-query-prefix-committed-query-target prefix prefix-state $\subseteq$

query-sample-space

unfolding *staged-query-prefix-committed-query-target-def*

by (rule *query-header-committed-union-good-sets-subset*)

lemma *staged-query-prefix-committed-query-targetI*:
assumes *candidate*:
 (trace-table, composition-table) $\in$
 query-header-committed-table-candidates prefix-state
 (sqp-trace-root prefix) (sqp-composition-fri-roots prefix)
and *hit*:
 idx $\in$ query-sampling-success-space trace-table composition-table
 (sqp-alphas prefix)
shows idx $\in$ staged-query-prefix-committed-query-target prefix prefix-state
proof –
have *idx-space*: idx $\in$ query-sample-space
using query-sampling-success-space-subset hit **by** blast
show ?thesis
unfolding staged-query-prefix-committed-query-target-def
 query-header-committed-union-good-sets-def
using candidate hit idx-space **by** blast
qed

lemma *staged-query-prefix-committed-query-target-fraction-bound-if-unique-candidate*:
assumes *unique*:
 $\exists$ trace-table composition-table.
 query-header-committed-table-candidates prefix-state
 (sqp-trace-root prefix)
 (sqp-composition-fri-roots prefix) $\subseteq$
 {(trace-table, composition-table)}
shows
 nnreal
 (card
 (staged-query-prefix-committed-query-target prefix prefix-state)) /
 nnreal (card query-sample-space) $\leq$ query-error-bound
unfolding staged-query-prefix-committed-query-target-def
by (rule
 query-header-committed-union-good-sets-fraction-bound-if-unique-candidate
 [OF unique query-sampling-success-space-subset
 query-sampling-success-space-fraction-bound-query-sample-space])

lemma *staged-query-prefix-committed-query-target-fraction-bound-if-no-root-collisions*:
assumes *trace-clean*:
 $\neg$ merkle-root-binding-collision (sqp-trace-root prefix)
 prefix-state
and *comp-clean*:
 $\bigwedge rt. \text{sqp-composition-fri-roots prefix} \neq [] \implies$
 $rt = \text{hd (sqp-composition-fri-roots prefix)} \implies$
 $\neg \text{merkle-root-binding-collision rt prefix-state}$
shows
 nnreal
 (card
 (staged-query-prefix-committed-query-target prefix prefix-state)) /
 nnreal (card query-sample-space) $\leq$ query-error-bound

proof –
have *unique*:
 $\exists$ *trace-table composition-table*.
query-header-committed-table-candidates prefix-state
(sqp-trace-root prefix)
(sqp-composition-fri-roots prefix) $\subseteq$
 $\{(trace-table, composition-table)\}$
by (*rule*
query-header-committed-table-candidates-subset-singleton-if-no-root-collisions
 $[OF\ trace-clean\ comp-clean]$)
show *?thesis*
by (*rule*
staged-query-prefix-committed-query-target-fraction-bound-if-unique-candidate
 $[OF\ unique]$)
qed

lemma *staged-query-prefix-length-committed-query-target-subset*:
staged-query-prefix-length-committed-query-target prefix prefix-state
 $\subseteq$ *query-sample-space*
unfolding *staged-query-prefix-length-committed-query-target-def* **by** *auto*

lemma *staged-query-prefix-length-committed-query-targetI*:
assumes *candidate*:
 $(trace-table, composition-table) \in$
staged-query-prefix-length-committed-table-candidates prefix prefix-state
and *hit*:
 $idx \in$ *query-sampling-success-space trace-table composition-table*
 $(sqp-alphas\ prefix)$
shows
 $idx \in$ *staged-query-prefix-length-committed-query-target prefix*
prefix-state

proof –
have *idx-space*: $idx \in$ *query-sample-space*
using *query-sampling-success-space-subset hit* **by** *blast*
show *?thesis*
unfolding *staged-query-prefix-length-committed-query-target-def*
using *candidate hit idx-space* **by** *blast*
qed

lemma *staged-query-prefix-clean-length-committed-query-target-subset*:
staged-query-prefix-clean-length-committed-query-target prefix prefix-state
 $\subseteq$ *query-sample-space*
unfolding *staged-query-prefix-clean-length-committed-query-target-def*
using *staged-query-prefix-length-committed-query-target-subset*
by *simp*

lemma *staged-query-prefix-length-committed-query-target-subset-committed*:
staged-query-prefix-length-committed-query-target prefix prefix-state
 $\subseteq$ *staged-query-prefix-committed-query-target prefix prefix-state*

```

unfolding staged-query-prefix-length-committed-query-target-def
  staged-query-prefix-length-committed-table-candidates-def
  staged-query-prefix-committed-query-target-def
  query-header-committed-union-good-sets-def
  query-header-committed-table-candidates-def
by auto

lemma staged-query-prefix-length-committed-table-candidates-subset-singleton-if-no-hash-collision:
  assumes clean:  $\neg$  hash-map-output-collision prefix-state
  shows
     $\exists$  trace-table composition-table.
      staged-query-prefix-length-committed-table-candidates prefix
        prefix-state  $\subseteq \{(trace-table, composition-table)\}$ 
proof (cases
  staged-query-prefix-length-committed-table-candidates prefix
    prefix-state = {})
  case True
  then show ?thesis
    by (intro exI[of -] [] exI[of - []]) simp
next
  case False
  then obtain pair where pair:
    pair  $\in$ 
      staged-query-prefix-length-committed-table-candidates prefix
        prefix-state
    by auto
  obtain trace-table composition-table where pair-eq:
    pair = (trace-table, composition-table)
  by (cases pair)
  then have candidate:
    (trace-table, composition-table)  $\in$ 
      staged-query-prefix-length-committed-table-candidates prefix
        prefix-state
  using pair by simp
  have trace-bind:
    merkle-root-binds-table (sqp-trace-root prefix) trace-table
      prefix-state
  using candidate
  unfolding staged-query-prefix-length-committed-table-candidates-def
  by simp
  have comp-nonempty: sqp-composition-fri-roots prefix  $\neq []$ 
  using candidate
  unfolding staged-query-prefix-length-committed-table-candidates-def
  by simp
  have comp-bind:
    merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
      composition-table prefix-state
  using candidate
  unfolding staged-query-prefix-length-committed-table-candidates-def

```

```

by simp
have trace-len: length trace-table = clength * scale
  using candidate
  unfolding staged-query-prefix-length-committed-table-candidates-def
  by simp
have comp-len: length composition-table = clength * scale
  using candidate
  unfolding staged-query-prefix-length-committed-table-candidates-def
  by simp
have trace-nonempty: trace-table  $\neq$  []
  using trace-len eval-domain-size-pos by auto
have comp-nonempty-table: composition-table  $\neq$  []
  using comp-len eval-domain-size-pos by auto
have subset:
  staged-query-prefix-length-committed-table-candidates prefix
  prefix-state  $\subseteq \{(trace-table, composition-table)\}$ 
proof
  fix pair
  assume pair-in:
    pair  $\in$ 
      staged-query-prefix-length-committed-table-candidates prefix
      prefix-state
  obtain trace-table' composition-table' where pair-eq:
    pair = (trace-table', composition-table')
    by (cases pair)
  have trace-bind':
    merkle-root-binds-table (sqp-trace-root prefix) trace-table'
    prefix-state
  using pair-in unfolding pair-eq
    staged-query-prefix-length-committed-table-candidates-def
  by simp
  have comp-bind':
    merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
    composition-table' prefix-state
  using pair-in unfolding pair-eq
    staged-query-prefix-length-committed-table-candidates-def
  by simp
  have trace-len':
    length trace-table' = clength * scale
  using pair-in unfolding pair-eq
    staged-query-prefix-length-committed-table-candidates-def
  by simp
  have comp-len':
    length composition-table' = clength * scale
  using pair-in unfolding pair-eq
    staged-query-prefix-length-committed-table-candidates-def
  by simp
  have trace-eq: trace-table' = trace-table
    by (rule merkle-root-binds-same-length-tables-unique-if-no-hash-collision)

```

```

      [OF trace-bind' trace-bind - clean])
    (simp add: trace-len' trace-len)
  have comp-eq: composition-table' = composition-table
  by (rule merkle-root-binds-same-length-tables-unique-if-no-hash-collision
      [OF comp-bind' comp-bind - clean])
    (simp add: comp-len' comp-len)
  show pair ∈ {(trace-table, composition-table)}
  unfolding pair-eq trace-eq comp-eq by simp
qed
show ?thesis
  using subset by blast
qed

lemma staged-query-prefix-length-committed-query-target-fraction-bound-if-no-hash-collision:
  assumes clean: ¬ hash-map-output-collision prefix-state
  shows
    nnreal
      (card
        (staged-query-prefix-length-committed-query-target prefix
          prefix-state)) /
      nnreal (card query-sample-space) ≤ query-error-bound
proof -
  from
    staged-query-prefix-length-committed-table-candidates-subset-singleton-if-no-hash-collision
    [OF clean, of prefix]
  obtain trace-table composition-table where candidates:
    staged-query-prefix-length-committed-table-candidates prefix
    prefix-state ⊆ {(trace-table, composition-table)}
  by blast
  have union-subset:
    staged-query-prefix-length-committed-query-target prefix prefix-state
    ⊆
    query-sampling-success-space trace-table composition-table
    (sqp-alphas prefix)
  unfolding staged-query-prefix-length-committed-query-target-def
  using candidates by auto
  have finite-good:
    finite
      (query-sampling-success-space trace-table composition-table
        (sqp-alphas prefix))
  by (rule finite-subset[OF query-sampling-success-space-subset
    finite-query-sample-space])
  have card-le:
    card
      (staged-query-prefix-length-committed-query-target prefix
        prefix-state) ≤
    card
      (query-sampling-success-space trace-table composition-table
        (sqp-alphas prefix))

```



```

    by (rule card-mono[OF finite-good union-subset])
  have nnreal
    (card
      (staged-query-prefix-length-committed-query-target prefix
        prefix-state)) /
    nnreal (card query-sample-space) ≤
    nnreal
      (card
        (query-sampling-success-space trace-table composition-table
          (sqp-alphas prefix))) /
      nnreal (card query-sample-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
  also have ... ≤ query-error-bound
  by (rule query-sampling-success-space-fraction-bound-query-sample-space)
  finally show ?thesis .
qed

lemma staged-query-prefix-clean-length-committed-query-target-fraction-bound:
  shows
    nnreal
      (card
        (staged-query-prefix-clean-length-committed-query-target prefix
          prefix-state)) /
      nnreal (card query-sample-space) ≤ query-error-bound
  proof (cases hash-map-output-collision prefix-state)
  case True
  then show ?thesis
    unfolding staged-query-prefix-clean-length-committed-query-target-def
    by simp
  next
  case False
  have bound:
    nnreal
      (card
        (staged-query-prefix-length-committed-query-target prefix
          prefix-state)) /
      nnreal (card query-sample-space) ≤ query-error-bound
  by (rule
      staged-query-prefix-length-committed-query-target-fraction-bound-if-no-hash-collision
      [OF False])
  then show ?thesis
    using False
    unfolding staged-query-prefix-clean-length-committed-query-target-def
    by simp
qed

lemma checked-staged-query-prefix-length-committed-hit-imp-clean-hit-or-hash-collision:
  assumes hit:
    checked-staged-query-prefix-dynamic-index-hit

```

```

    staged-query-prefix-length-committed-query-target out
  shows
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-clean-length-committed-query-target out ∨
    checked-staged-query-prefix-hash-map-output-collision out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding checked-staged-query-prefix-dynamic-index-hit-def by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state where out-eq:
    out = Some (((prefix, prefix-state), raw), raw-state)
    by (cases packed, auto split: prod.splits)
  show ?thesis
proof (cases hash-map-output-collision prefix-state)
  case True
  then have checked-staged-query-prefix-hash-map-output-collision out
    unfolding out-eq checked-staged-query-prefix-hash-map-output-collision-def
    by simp
  then show ?thesis by simp
next
  case False
  have
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-clean-length-committed-query-target out
  using hit False
  unfolding out-eq checked-staged-query-prefix-dynamic-index-hit-def
    staged-query-prefix-clean-length-committed-query-target-def
  by simp
  then show ?thesis by simp
qed
qed

lemma staged-query-prefix-clean-committed-query-target-subset:
  staged-query-prefix-clean-committed-query-target prefix prefix-state
  ⊆ query-sample-space
  unfolding staged-query-prefix-clean-committed-query-target-def
  using staged-query-prefix-committed-query-target-subset[of prefix prefix-state]
  by simp

lemma staged-query-prefix-clean-committed-query-target-subset-committed:
  staged-query-prefix-clean-committed-query-target prefix prefix-state
  ⊆ staged-query-prefix-committed-query-target prefix prefix-state
  unfolding staged-query-prefix-clean-committed-query-target-def by auto

lemma staged-query-prefix-clean-committed-query-target-fraction-bound:
  shows nnreal

```

```

    (card
      (staged-query-prefix-clean-committed-query-target prefix prefix-state)) /
    nnreal (card query-sample-space) ≤ query-error-bound
proof (cases
  staged-query-prefix-committed-roots-clean prefix prefix-state)
case True
then have trace-clean:
  ¬ merkle-root-binding-collision (sqp-trace-root prefix)
    prefix-state
and comp-clean:
  ∧ rt. sqp-composition-fri-roots prefix ≠ [] ⇒
    rt = hd (sqp-composition-fri-roots prefix) ⇒
    ¬ merkle-root-binding-collision rt prefix-state
unfolding staged-query-prefix-committed-roots-clean-def by blast+
show ?thesis
unfolding staged-query-prefix-clean-committed-query-target-def
using True
  staged-query-prefix-committed-query-target-fraction-bound-if-no-root-collisions
    [OF trace-clean comp-clean]
by simp
next
case False
then show ?thesis
  unfolding staged-query-prefix-clean-committed-query-target-def by simp
qed

```

lemma checked-staged-query-prefix-committed-hit-imp-clean-hit-or-root-collision:

```

assumes hit:
  checked-staged-query-prefix-dynamic-index-hit
    (λprefix prefix-state.
      staged-query-prefix-committed-query-target prefix prefix-state) out
shows
  checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-clean-committed-query-target out ∨
  checked-staged-query-prefix-committed-root-collision out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding checked-staged-query-prefix-dynamic-index-hit-def by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state where out-eq:
  out = Some (((prefix, prefix-state), raw), raw-state)
by (cases packed, auto split: prod.splits)
show ?thesis
proof (cases
  staged-query-prefix-committed-roots-clean prefix prefix-state)
case True

```

```

have clean-hit:
  checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-clean-committed-query-target out
using hit True
unfolding out-eq checked-staged-query-prefix-dynamic-index-hit-def
  staged-query-prefix-clean-committed-query-target-def
by simp
then show ?thesis by simp
next
case False
then have
  checked-staged-query-prefix-committed-root-collision out
unfolding out-eq checked-staged-query-prefix-committed-root-collision-def
by simp
then show ?thesis by simp
qed
qed

lemma staged-partial-query-target-subset:
  staged-partial-query-target data attacker-state  $\subseteq$ 
    query-sample-space
unfolding staged-partial-query-target-def
by (rule query-header-supported-partial-union-good-sets-subset)

lemma staged-partial-query-target-fraction-bound-if-unique-candidate:
assumes unique:
   $\exists$  trace-table composition-table.
    query-header-supported-partial-table-candidates
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)  $\subseteq$ 
        {(trace-table, composition-table)}
shows
  nnreal (card (staged-partial-query-target data attacker-state)) /
    nnreal (card query-sample-space)  $\leq$  query-error-bound
unfolding staged-partial-query-target-def
by (rule
    query-header-supported-partial-union-good-sets-fraction-bound-if-unique-candidate
    [OF unique query-sampling-success-space-subset
      query-sampling-success-space-fraction-bound-query-sample-space])

definition checked-staged-query-prefix-fixed-transcript-target-hit
  :: 'f staged-proof-data  $\Rightarrow$  'f protocol-channel  $\Rightarrow$ 

```

$((('f \text{ staged-query-prefix-data} \times 'f \text{ protocol-channel}) \times 'f) \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-query-prefix-fixed-transcript-target-hit data
attacker-state out $\longleftrightarrow$
checked-staged-query-prefix-dynamic-index-hit
 $(\lambda - . \text{ staged-partial-query-target data attacker-state}) \text{ out}$

lemma *checked-staged-query-prefix-dynamic-index-hit-bound:*

fixes $C :: \text{prob}$

assumes *raw-bound: query-index-raw-preimage-bound*

and *prefix-bound:*

$\bigwedge \text{prefix prefix-state.}$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s) \Rightarrow$
 $\text{query-future-fresh prefix-state} \wedge$
 $B \text{ prefix prefix-state} \subseteq \text{query-sample-space} \wedge$
 $\text{nnreal } (\text{card } (B \text{ prefix prefix-state})) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq C$

shows

$\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit } B) \ s \leq C$

unfolding *checked-staged-query-prefix-receive-with-state-def*

proof (*rule wp-event-bind-bound-by-cont*)

show $\neg \text{checked-staged-query-prefix-dynamic-index-hit } B \text{ None}$

unfolding *checked-staged-query-prefix-dynamic-index-hit-def* **by** *simp*

next

fix *prefix-pack prefix-state*

assume *prefix-support:*

$\text{Some } (\text{prefix-pack}, \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s)$

obtain *prefix captured-state* **where** *prefix-pack-eq:*

$\text{prefix-pack} = (\text{prefix}, \text{captured-state})$
by (*cases prefix-pack*) *simp*

have *captured-eq: captured-state = prefix-state*

using *checked-staged-query-prefix-with-state-outcome-state*

$[OF \text{ prefix-support}[\text{unfolded } \text{prefix-pack-eq}]]$
by *simp*

have *support':*

$\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s)$

using *checked-staged-query-prefix-with-state-outcome-canonical*

$[OF \text{ prefix-support}[\text{unfolded } \text{prefix-pack-eq}]]$
 captured-eq
by *simp*

from *prefix-bound* $[OF \text{ support'}]$ **have** *future:*

```

    query-future-fresh prefix-state
and subset:
    B prefix prefix-state  $\subseteq$  query-sample-space
and frac:
    nreal (card (B prefix prefix-state)) /
    nreal (card query-sample-space)  $\leq$  C
by blast+
have receive-bound:
    wp-event
    (receive-query-index-challenge  $\gg$  ( $\lambda$ raw.
      return (prefix-pack, raw)))
    (checked-staged-query-prefix-dynamic-index-hit B) prefix-state  $\leq$ 
    C
proof -
let ?P =
     $\lambda$ out. case out of None  $\Rightarrow$  False
    | Some (raw, -)  $\Rightarrow$  index (to-nat raw)  $\in$  B prefix prefix-state
have cont-eq:
    wp-event
    (receive-query-index-challenge  $\gg$  ( $\lambda$ raw.
      return (prefix-pack, raw)))
    (checked-staged-query-prefix-dynamic-index-hit B) prefix-state =
    wp-event receive-query-index-challenge ?P prefix-state
unfolding wp-event-def
apply (subst wp-bind)
apply (rule arg-cong[where
  f= $\lambda$ Q. wp receive-query-index-challenge Q prefix-state])
apply (rule ext)
apply (simp add: wp-return prefix-pack-eq captured-eq
  checked-staged-query-prefix-dynamic-index-hit-def
  split: option.splits prod.splits)
done
show ?thesis
unfolding cont-eq
by (rule order-trans
  [OF wp-receive-query-index-challenge-fresh-index-set-raw-bound
  [OF future raw-bound subset] frac])
qed
show wp-event
    (receive-query-index-challenge  $\gg$  ( $\lambda$ raw.
      return (prefix-pack, raw)))
    (checked-staged-query-prefix-dynamic-index-hit B) prefix-state  $\leq$ 
    C
by (rule receive-bound)
qed

lemma checked-staged-query-prefix-fixed-transcript-target-hit-bound:
fixes C :: prob
assumes raw-bound: query-index-raw-preimage-bound

```

and *prefix-bound*:
 $\bigwedge \text{prefix } \text{prefix-state}.$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s) \implies$
 $\text{query-future-fresh } \text{prefix-state}$
and *subset*: $\text{staged-partial-query-target data attacker-state} \subseteq$
 $\text{query-sample-space}$
and *frac*:
 $\text{nnreal } (\text{card } (\text{staged-partial-query-target data attacker-state})) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq C$
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-fixed-transcript-target-hit data}$
 $\text{attacker-state}) \ s \leq C$
unfolding *checked-staged-query-prefix-fixed-transcript-target-hit-def*
by (*rule checked-staged-query-prefix-dynamic-index-hit-bound*
 $[OF \text{ raw-bound}]$)
(use prefix-bound subset frac in auto)

lemma *checked-staged-query-prefix-committed-target-hit-bound*:
assumes *raw-bound*: *query-index-raw-preimage-bound*
and *prefix-bound*:
 $\bigwedge \text{prefix } \text{prefix-state}.$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s) \implies$
 $\text{query-future-fresh } \text{prefix-state} \wedge$
 $\neg \text{merkle-root-binding-collision } (\text{sqp-trace-root } \text{prefix})$
 $\text{prefix-state} \wedge$
 $(\forall \text{rt. } \text{sqp-composition-fri-roots } \text{prefix} \neq [] \longrightarrow$
 $\text{rt} = \text{hd } (\text{sqp-composition-fri-roots } \text{prefix}) \longrightarrow$
 $\neg \text{merkle-root-binding-collision } \text{rt } \text{prefix-state})$
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $(\lambda \text{prefix } \text{prefix-state}.$
 $\text{staged-query-prefix-committed-query-target } \text{prefix } \text{prefix-state}))$
 $s \leq \text{query-error-bound}$
proof (*rule checked-staged-query-prefix-dynamic-index-hit-bound*
 $[OF \text{ raw-bound}]$)
fix *prefix prefix-state*
assume *support*:
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s)$
from *prefix-bound* $[OF \text{ support}]$
have *future*: *query-future-fresh prefix-state*
and *trace-clean*:

```

    ¬ merkle-root-binding-collision (sqp-trace-root prefix)
      prefix-state
  and comp-clean:
    ∧ rt. sqp-composition-fri-roots prefix ≠ [] ⇒
      rt = hd (sqp-composition-fri-roots prefix) ⇒
        ¬ merkle-root-binding-collision rt prefix-state
  by blast+
show
  query-future-fresh prefix-state ∧
    staged-query-prefix-committed-query-target prefix prefix-state
    ⊆ query-sample-space ∧
  nnreal
    (card
      (staged-query-prefix-committed-query-target prefix prefix-state)) /
  nnreal (card query-sample-space) ≤ query-error-bound
using future
  staged-query-prefix-committed-query-target-subset[of prefix prefix-state]
  staged-query-prefix-committed-query-target-fraction-bound-if-no-root-collisions
    [OF trace-clean comp-clean]
by blast
qed

definition checked-staged-query-prefix-dynamic-index-prehit
  :: ('f staged-query-prefix-data ⇒ 'f protocol-channel ⇒
    nat set) ⇒
    (((('f staged-query-prefix-data × 'f protocol-channel) × 'f) ×
      'f protocol-channel) option ⇒ bool
  where
    checked-staged-query-prefix-dynamic-index-prehit B out ⟷
      (case out of
        None ⇒ False
      | Some (((prefix, prefix-state), -), -) ⇒
          (∃ raw ∈ query-index-raw-preimage (B prefix prefix-state).
            fmllookup (HashMap prefix-state)
              (QueryIndexChallenge (PQueryCounter prefix-state)
                (PState prefix-state)) = Some raw))

definition checked-staged-query-prefix-dynamic-index-prehit-relation
  :: 'f staged-adversary ⇒ nat ⇒ 'f protocol-channel ⇒
    ('f staged-query-prefix-data ⇒ 'f protocol-channel ⇒
      nat set) ⇒
    'f protocol-hash-input ⇒ 'f ⇒ bool
  where
    checked-staged-query-prefix-dynamic-index-prehit-relation A i s B x y
      ⟷
      (∃ prefix prefix-state.
        Some ((prefix, prefix-state), prefix-state) ∈
          set-dist
            (execute (checked-staged-query-prefix-with-state A i) s) ∧

```


$x = \text{QueryIndexChallenge } (P\text{QueryCounter } \text{prefix-state})$
 $(P\text{State } \text{prefix-state}) \wedge$
 $y \in \text{query-index-raw-preimage } (B \text{ prefix } \text{prefix-state})$

lemma *checked-staged-query-prefix-dynamic-index-prehit-imp-relation-hit:*

assumes *empty*: $\text{HashMap } s = \text{fmempty}$

and *prefix-support*:

$\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s)$

and *receive-support*:

$\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in$
set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i) \ s)$

and *prehit*:

$\text{checked-staged-query-prefix-dynamic-index-prehit } B$
 $(\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}))$

shows

hash-relation-hit-event
 $(\text{checked-staged-query-prefix-dynamic-index-prehit-relation } A \ i \ s \ B)$
 $s \ (\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}))$

proof –

from *prehit* **obtain** *old-row* **where** *old-row-in*:

$\text{old-row} \in \text{query-index-raw-preimage } (B \text{ prefix } \text{prefix-state})$

and *lookup-prefix*:

$\text{fmlookup } (\text{HashMap } \text{prefix-state})$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } \text{prefix-state})$
 $(P\text{State } \text{prefix-state})) = \text{Some } \text{old-row}$

unfolding *checked-staged-query-prefix-dynamic-index-prehit-def*

by *auto*

have *recv*:

$\text{Some } (\text{raw}, \text{raw-state}) \in$
set-dist $(\text{execute } \text{receive-query-index-challenge } \text{prefix-state})$

proof –

from *receive-support* **obtain** *u* **where**

prefix-u:

$\text{Some } ((\text{prefix}, \text{prefix-state}), u) \in$
set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s)$

and *recv-u*:

$\text{Some } (\text{raw}, \text{raw-state}) \in$
set-dist $(\text{execute } \text{receive-query-index-challenge } u)$

unfolding *checked-staged-query-prefix-receive-with-state-def*

by $(\text{auto elim!} \text{: set-dist-bindE})$

have $u = \text{prefix-state}$

by $(\text{rule } \text{checked-staged-query-prefix-with-state-outcome-state}$
 $[\text{OF } \text{prefix-u}])$

then show *?thesis*

using *recv-u* **by** *simp*

```

qed
have prefix-ext: prefix-state ≤ raw-state
  by (rule receive-query-index-challenge-extends[OF recv])
have lookup-raw-state:
  fmllookup (HashMap raw-state)
    (QueryIndexChallenge (PQueryCounter prefix-state)
      (PState prefix-state)) = Some old-raw
  by (rule hash-extension-lookup[OF lookup-prefix prefix-ext])
have lookup-initial:
  fmllookup (HashMap s)
    (QueryIndexChallenge (PQueryCounter prefix-state)
      (PState prefix-state)) = None
  using empty by simp
have rel:
  checked-staged-query-prefix-dynamic-index-prehit-relation A i s B
    (QueryIndexChallenge (PQueryCounter prefix-state)
      (PState prefix-state)) old-raw
  unfolding
    checked-staged-query-prefix-dynamic-index-prehit-relation-def
  using prefix-support old-raw-in by blast
have hit:
  hash-relation-hit
    (checked-staged-query-prefix-dynamic-index-prehit-relation A i s B)
    s raw-state
  unfolding hash-relation-hit-def
  by (intro exI[of -
    QueryIndexChallenge (PQueryCounter prefix-state)
      (PState prefix-state)]
    exI[of - old-raw] conjI)
    (use lookup-initial lookup-raw-state rel in simp-all)
show ?thesis
  unfolding hash-relation-hit-event-def using hit by simp
qed

lemma checked-staged-query-prefix-dynamic-index-prehit-bound-by-relation-hit:
  assumes empty: HashMap s = fmempty
  shows
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-prehit B) s ≤
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (hash-relation-hit-event
        (checked-staged-query-prefix-dynamic-index-prehit-relation A i s B)
        s) s
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i) s)

```

```

and prehit: checked-staged-query-prefix-dynamic-index-prehit B out
show
  hash-relation-hit-event
    (checked-staged-query-prefix-dynamic-index-prehit-relation A i s B)
      s out
proof (cases out)
  case None
  then show ?thesis
    using prehit
    unfolding checked-staged-query-prefix-dynamic-index-prehit-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state where out-eq:
    out = Some (((prefix, prefix-state), raw), raw-state)
  by (cases packed, auto split: prod.splits)
  have receive-support:
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i) s)
  using support unfolding out-eq .
  have prefix-support:
    Some ((prefix, prefix-state), prefix-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-with-state A i) s)
  proof –
  from receive-support obtain u where
    prefix-u:
      Some ((prefix, prefix-state), u) ∈
        set-dist
        (execute (checked-staged-query-prefix-with-state A i) s)
  unfolding checked-staged-query-prefix-receive-with-state-def
  by (auto elim!: set-dist-bindE)
  show ?thesis
    by (rule checked-staged-query-prefix-with-state-outcome-canonical
      [OF prefix-u])
  qed
show ?thesis
  unfolding out-eq
  by (rule checked-staged-query-prefix-dynamic-index-prehit-imp-relation-hit
    [OF empty prefix-support receive-support])
    (use prehit out-eq in simp)
  qed
qed

lemma checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program:
assumes empty: HashMap s = fmempty
and relation-program:
  hash-relation-program

```

(*checked-staged-query-prefix-dynamic-index-prehit-relation* A i s B)
fiber-bound query-bound
 (*checked-staged-query-prefix-receive-with-state* A i)
shows
wp-event (*checked-staged-query-prefix-receive-with-state* A i)
 (*checked-staged-query-prefix-dynamic-index-prehit* B) $s \leq$
staged-phase-relation-error fiber-bound query-bound
proof –
have *prehit-le-relation*:
wp-event (*checked-staged-query-prefix-receive-with-state* A i)
 (*checked-staged-query-prefix-dynamic-index-prehit* B) $s \leq$
wp-event (*checked-staged-query-prefix-receive-with-state* A i)
 (*hash-relation-hit-event*
 (*checked-staged-query-prefix-dynamic-index-prehit-relation* A i s B)
 s) s
by (*rule*
checked-staged-query-prefix-dynamic-index-prehit-bound-by-relation-hit
 [*OF empty*])
also have $\dots \leq$ *staged-phase-relation-error fiber-bound query-bound*
using *relation-program*
unfolding *hash-relation-program-def hash-relation-budget-def*
staged-phase-relation-error-def
by *blast*
finally show *?thesis* .
qed

lemma *hash-map-preserving-get*:
hash-map-preserving get
unfolding *hash-map-preserving-def* **by** *simp*

lemma *hash-relation-program-get*:
hash-relation-program R b 0 *get*
by (*rule hash-relation-program-zero*)
 (*rule hash-map-preserving-get*)

lemma *hash-relation-program-modify-preserves-hash-map*:
assumes *bump*:
 $\bigwedge s. \text{HashMap } (\text{bump } s) = \text{HashMap } s$
shows *hash-relation-program* R b 0 (*modify bump*)
by (*rule hash-relation-program-zero*)
 (*rule hash-map-preserving-modify* [*OF bump*])

lemma *hash-relation-program-receive-counted-tagged-random-field-element*:
assumes *bump*:
 $\bigwedge s. \text{HashMap } (\text{bump } s) = \text{HashMap } s$
and fibers: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$
shows
hash-relation-program R b 1
 (*protocol-receive-counted-tagged-random-field-element counter bump tag*)

```

proof –
  have modify-return:
    hash-relation-program R b (0 + 0)
      (modify bump  $\gg$  ( $\lambda$ -. return r))
    for r :: 'f
    by (rule hash-relation-program-bind)
      (rule hash-relation-program-modify-preserves-hash-map[OF bump],
       rule hash-relation-program-zero[OF hash-map-preserving-return])
  have hash-tail:
    hash-relation-program R b (1 + (0 + 0))
      (hash (tag (counter s) (PState s))  $\gg$ 
       ( $\lambda$ r. modify bump  $\gg$  ( $\lambda$ -. return r)))
    for s
    by (rule hash-relation-program-bind)
      (rule hash-relation-program-hash[OF fibers], rule modify-return)
  have hash-relation-program R b (0 + (1 + (0 + 0)))
    (get  $\gg$ 
     ( $\lambda$ s.
      hash (tag (counter s) (PState s))  $\gg$ 
       ( $\lambda$ r. modify bump  $\gg$  ( $\lambda$ -. return r))))
    by (rule hash-relation-program-bind)
      (rule hash-relation-program-get, rule hash-tail)
  then show ?thesis
    unfolding protocol-receive-counted-tagged-random-field-element-def
    by simp
qed

lemma hash-relation-program-receive-query-index-challenge:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R\ x\ y\} \leq b$ 
  shows hash-relation-program R b 1
    receive-query-index-challenge
  unfolding receive-query-index-challenge-def
  by (rule hash-relation-program-receive-counted-tagged-random-field-element
    [OF - fibers])
    simp

lemma hash-map-preserving-get-bind-return-pair:
  hash-map-preserving (get  $\gg$  ( $\lambda$ s. return (x, s)))
  unfolding hash-map-preserving-def
  by (auto elim!: set-dist-bindE)

lemma hash-map-preserving-imp-hash-range-budget-zero:
  fixes m :: ('r, ('f, 'a) protocol-channel-scheme) state-monad
  assumes preserving: hash-map-preserving m
  shows hash-range-budget 0 m
  unfolding hash-range-budget-def
proof (intro allI impI)
  fix s y t
  assume Some (y, t)  $\in$  set-dist (execute m s)

```

```

then have HashMap  $t = \text{HashMap } s$ 
  using preserving unfolding hash-map-preserving-def by blast
then show
   $\text{card } (\text{hash-map-output-values } t) \leq$ 
     $\text{card } (\text{hash-map-output-values } s) + 0$ 
  by (simp add: hash-map-output-values-def)
qed

lemma hash-map-preserving-imp-hash-collision-budget-zero-scheme:
  fixes  $m :: ('r, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$ 
  assumes preserving: hash-map-preserving  $m$ 
  shows hash-collision-budget  $0\ m$ 
  unfolding hash-collision-budget-def
proof (intro allI impI)
  fix  $s :: ('f, 'a) \text{protocol-channel-scheme}$ 
  assume clean:  $\neg \text{hash-map-output-collision } s$ 
  have no-event:
     $\bigwedge \text{out. out} \in \text{set-dist } (\text{execute } m\ s) \implies$ 
       $\neg \text{hash-new-collision-event } s\ \text{out}$ 
  proof -
    fix out
    assume support:  $\text{out} \in \text{set-dist } (\text{execute } m\ s)$ 
    show  $\neg \text{hash-new-collision-event } s\ \text{out}$ 
    proof (cases out)
      case None
      then show ?thesis
        unfolding hash-new-collision-event-def by simp
    next
      case (Some xt)
      then obtain  $x\ t$  where  $xt: xt = (x, t)$ 
        by (cases xt) simp
      have same:  $\text{HashMap } t = \text{HashMap } s$ 
        using preserving support Some xt
        unfolding hash-map-preserving-def by blast
      show ?thesis
        using clean same Some xt
        unfolding hash-new-collision-event-def
          hash-map-new-output-collision-def hash-map-output-collision-def
        by simp
    qed
  qed
  have no-event-dom:
     $\bigwedge \text{out. out} \in \text{dom } (\text{dist } (\text{execute } m\ s)) \implies$ 
       $\neg \text{hash-new-collision-event } s\ \text{out}$ 
    using no-event unfolding set-dist-def by blast
  have wp-event  $m\ (\text{hash-new-collision-event } s)\ s = 0$ 
    unfolding wp-event-def wp-def dist-expect-def
    by (intro sum.neutral) (simp add: no-event-dom)
  then show

```

$wp\text{-}event\ m\ (hash\text{-}new\text{-}collision\text{-}event\ s)\ s \leq$
 $hash\text{-}collision\text{-}budget\text{-}value$
 $(card\ (hash\text{-}map\text{-}output\text{-}values\ s))\ 0$
unfolding $hash\text{-}collision\text{-}budget\text{-}value\text{-}def$ **by** $simp$
qed

lemma $checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}prehit\text{-}relation\text{-}fiber\text{-}bound$:
assumes $fibers$:
 $\bigwedge x. card$
 $(\bigcup \{query\text{-}index\text{-}raw\text{-}preimage\ (B\ prefix\ prefix\text{-}state) \mid$
 $prefix\ prefix\text{-}state.$
 $Some\ ((prefix,\ prefix\text{-}state),\ prefix\text{-}state) \in$
 $set\text{-}dist$
 $(execute\ (checked\text{-}staged\text{-}query\text{-}prefix\text{-}with\text{-}state\ A\ i)\ s) \wedge$
 $x = QueryIndexChallenge\ (PQueryCounter\ prefix\text{-}state)$
 $(PState\ prefix\text{-}state)\}) \leq b$
shows
 $card$
 $\{y. checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}prehit\text{-}relation$
 $A\ i\ s\ B\ x\ y\} \leq b$
proof $-$
let $?U =$
 $\bigcup \{query\text{-}index\text{-}raw\text{-}preimage\ (B\ prefix\ prefix\text{-}state) \mid$
 $prefix\ prefix\text{-}state.$
 $Some\ ((prefix,\ prefix\text{-}state),\ prefix\text{-}state) \in$
 $set\text{-}dist$
 $(execute\ (checked\text{-}staged\text{-}query\text{-}prefix\text{-}with\text{-}state\ A\ i)\ s) \wedge$
 $x = QueryIndexChallenge\ (PQueryCounter\ prefix\text{-}state)$
 $(PState\ prefix\text{-}state)\}$
have $subset$:
 $\{y. checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}prehit\text{-}relation$
 $A\ i\ s\ B\ x\ y\} \subseteq ?U$
unfolding $checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}prehit\text{-}relation\text{-}def$
by $blast$
have $finite\text{-}U$: $finite\ ?U$
by $simp$
have $card$
 $\{y. checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}prehit\text{-}relation$
 $A\ i\ s\ B\ x\ y\} \leq card\ ?U$
by $(rule\ card\text{-}mono[OF\ finite\text{-}U\ subset])$
also have $\dots \leq b$
by $(rule\ fibers)$
finally show $?thesis$.
qed

lemma $checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}prehit\text{-}relation\text{-}fiber\text{-}bound\text{-}size$:
 $card$
 $\{y. checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}prehit\text{-}relation$
 $A\ i\ s\ B\ x\ y\} \leq size$

proof –
have *subset*:
 $\{y. \text{checked-staged-query-prefix-dynamic-index-prehit-relation } A \ i \ s \ B \ x \ y\} \subseteq (UNIV :: 'f \text{ set})$
by *simp*
have *finite-field*: *finite* (*UNIV* :: 'f set)
by *simp*
have *card*
 $\{y. \text{checked-staged-query-prefix-dynamic-index-prehit-relation } A \ i \ s \ B \ x \ y\} \leq \text{card } (UNIV :: 'f \text{ set})$
by (*rule* *card-mono*[*OF* *finite-field subset*])
also have ... = *size*
using *size-card* **by** *simp*
finally show ?thesis .
qed

lemma *hash-relation-program-checked-staged-query-prefix-with-state*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i \leq \text{rounds}$
and *fibers*: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$
shows
hash-relation-program R b
(staged-query-search-queries budgets i)
(checked-staged-query-prefix-with-state A i)
unfolding *checked-staged-query-prefix-with-state-def*

proof –
have *prefix*:
hash-relation-program R b
(staged-query-search-queries budgets i)
(checked-staged-query-challenge-prefix-program A i)
by (*rule* *hash-relation-program-checked-staged-query-challenge-prefix-program*
[*OF* *wf controlled i-bound fibers*])
have *tail*:
 $\bigwedge \text{prefix}. \text{hash-relation-program } R \ b \ 0$
(get $\gg=$ ($\lambda \text{prefix-state}. \text{return } (\text{prefix}, \text{prefix-state}))$)
by (*rule* *hash-relation-program-zero*)
(*rule* *hash-map-preserving-get-bind-return-pair*)
have *hash-relation-program R b*
(staged-query-search-queries budgets i + 0)
(checked-staged-query-challenge-prefix-program A i $\gg=$
($\lambda \text{prefix}. \text{get } \gg= (\lambda \text{prefix-state}. \text{return } (\text{prefix}, \text{prefix-state}))$)
by (*rule* *hash-relation-program-bind*)
(*rule* *prefix*, *rule* *tail*)
then show *hash-relation-program R b*
(staged-query-search-queries budgets i)
(checked-staged-query-challenge-prefix-program A i $\gg=$
($\lambda \text{prefix}. \text{get } \gg= (\lambda \text{prefix-state}. \text{return } (\text{prefix}, \text{prefix-state}))$)


```

      return (prefix, prefix-state))))
    by simp
  qed

lemma hash-relation-program-checked-staged-query-prefix-receive-with-state:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound:  $i \leq \text{rounds}$ 
    and fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
    hash-relation-program R b
      (staged-query-search-queries budgets  $i + 1$ )
      (checked-staged-query-prefix-receive-with-state A i)
  unfolding checked-staged-query-prefix-receive-with-state-def
proof -
  have prefix:
    hash-relation-program R b
      (staged-query-search-queries budgets i)
      (checked-staged-query-prefix-with-state A i)
  by (rule hash-relation-program-checked-staged-query-prefix-with-state
      [OF wf controlled i-bound fibers])
  have receive:
     $\bigwedge \text{prefix-pack}. \text{hash-relation-program R b 1}$ 
      (receive-query-index-challenge  $\ggg (\lambda \text{raw}. \text{return (prefix-pack, raw)})$ )
  proof -
    fix prefix-pack
    have hash-relation-program R b ( $1 + 0$ )
      (receive-query-index-challenge  $\ggg (\lambda \text{raw}. \text{return (prefix-pack, raw)})$ )
    by (rule hash-relation-program-bind)
      (rule hash-relation-program-receive-query-index-challenge[OF fibers],
        rule hash-relation-program-zero[OF hash-map-preserving-return])
    then show hash-relation-program R b 1
      (receive-query-index-challenge  $\ggg (\lambda \text{raw}. \text{return (prefix-pack, raw)})$ )
    by simp
  qed
  show hash-relation-program R b
    (staged-query-search-queries budgets  $i + 1$ )
    (checked-staged-query-prefix-with-state A i  $\ggg (\lambda \text{prefix-pack}. \text{receive-query-index-challenge } \ggg (\lambda \text{raw}. \text{return (prefix-pack, raw)}))$ )
  by (rule hash-relation-program-bind)
    (rule prefix, rule receive)
qed

```

lemma *hash-range-budget-checked-staged-query-prefix-with-state:*
assumes *wf: staged-budget-wellformed budgets*

and *controlled*: staged-adversary-controlled budgets A
and *i-bound*: $i \leq \text{rounds}$
shows
hash-range-budget
 (*staged-query-search-queries* budgets i)
 (*checked-staged-query-prefix-with-state* A i)
unfolding *checked-staged-query-prefix-with-state-def*
proof –
have *prefix*:
 hash-range-budget
 (*staged-query-search-queries* budgets i)
 (*checked-staged-query-challenge-prefix-program* A i)
by (rule *hash-range-budget-checked-staged-query-challenge-prefix-program*
 [*OF wf controlled i-bound*])
have *tail*:
 $\wedge \text{prefix. hash-range-budget } 0$
 (*get* $\gg (\lambda \text{prefix-state. return (prefix, prefix-state)})$)
by (rule *hash-map-preserving-imp-hash-range-budget-zero*)
 (rule *hash-map-preserving-get-bind-return-pair*)
have *hash-range-budget*
 (*staged-query-search-queries* budgets $i + 0$)
 (*checked-staged-query-challenge-prefix-program* A $i \gg$
 $(\lambda \text{prefix. get } \gg (\lambda \text{prefix-state. return (prefix, prefix-state)}))$)
by (rule *hash-range-budget-bind*)
 (rule *prefix*, rule *tail*)
then show *hash-range-budget*
 (*staged-query-search-queries* budgets i)
 (*checked-staged-query-challenge-prefix-program* A $i \gg$
 $(\lambda \text{prefix. get } \gg (\lambda \text{prefix-state. return (prefix, prefix-state)}))$)
by *simp*
qed

lemma *hash-collision-budget-checked-staged-query-prefix-with-state*:
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets A
and *i-bound*: $i \leq \text{rounds}$
shows
hash-collision-budget
 (*staged-query-search-queries* budgets i)
 (*checked-staged-query-prefix-with-state* A i)
unfolding *checked-staged-query-prefix-with-state-def*
proof –
have *prefix-range*:
 hash-range-budget
 (*staged-query-search-queries* budgets i)
 (*checked-staged-query-challenge-prefix-program* A i)
by (rule *hash-range-budget-checked-staged-query-challenge-prefix-program*)

[OF wf controlled i -bound])
have *prefix-collision*:
 hash-collision-budget
 (*staged-query-search-queries* budgets i)
 (*checked-staged-query-challenge-prefix-program* A i)
 by (*rule hash-collision-budget-checked-staged-query-challenge-prefix-program*
 [OF wf controlled i -bound])
have *tail-range*:
 $\wedge$ *prefix*. *hash-range-budget* 0
 (*get* $\gg$ (λ *prefix-state*. *return* (*prefix*, *prefix-state*)))
 by (*rule hash-map-preserving-imp-hash-range-budget-zero*)
 (*rule hash-map-preserving-get-bind-return-pair*)
have *tail-collision*:
 $\wedge$ *prefix*. *hash-collision-budget* 0
 (*get* $\gg$ (λ *prefix-state*. *return* (*prefix*, *prefix-state*)))
 by (*rule hash-map-preserving-imp-hash-collision-budget-zero-scheme*)
 (*rule hash-map-preserving-get-bind-return-pair*)
have *hash-collision-budget*
 (*staged-query-search-queries* budgets $i + 0$)
 (*checked-staged-query-challenge-prefix-program* A i $\gg$
 (λ *prefix*. *get* $\gg$ (λ *prefix-state*.
 return (*prefix*, *prefix-state*))))
 by (*rule hash-collision-budget-bind*)
 (*rule prefix-range*, *rule prefix-collision*, *rule tail-range*,
 rule tail-collision)
then show *hash-collision-budget*
 (*staged-query-search-queries* budgets i)
 (*checked-staged-query-challenge-prefix-program* A i $\gg$
 (λ *prefix*. *get* $\gg$ (λ *prefix-state*.
 return (*prefix*, *prefix-state*))))
 by *simp*
qed

lemma *hash-range-budget-checked-staged-query-prefix-receive-with-state*:
 assumes *wf*: *staged-budget-wellformed* budgets
 and *controlled*: *staged-adversary-controlled* budgets A
 and *i-bound*: $i \leq \text{rounds}$
 shows
 hash-range-budget
 (*staged-query-search-queries* budgets $i + 1$)
 (*checked-staged-query-prefix-receive-with-state* A i)
 unfolding *checked-staged-query-prefix-receive-with-state-def*
proof –
 have *prefix*:
 hash-range-budget
 (*staged-query-search-queries* budgets i)
 (*checked-staged-query-prefix-with-state* A i)
 by (*rule hash-range-budget-checked-staged-query-prefix-with-state*
 [OF wf controlled i -bound])

have *receive*:
 $\wedge$ *prefix-pack*. *hash-range-budget* 1
 (receive-query-index-challenge $\gg$ (λ raw.
 return (*prefix-pack*, raw)))
proof –
fix *prefix-pack*
have *hash-range-budget* (1 + 0)
 (receive-query-index-challenge $\gg$ (λ raw.
 return (*prefix-pack*, raw)))
by (rule *hash-range-budget-bind*)
 (rule *hash-range-budget-receive-query-index-challenge*,
 rule *hash-range-budget-return*)
then show *hash-range-budget* 1
 (receive-query-index-challenge $\gg$ (λ raw.
 return (*prefix-pack*, raw)))
by *simp*
qed
show *hash-range-budget*
 (*staged-query-search-queries* budgets *i* + 1)
 (*checked-staged-query-prefix-with-state* *A* *i* $\gg$
 (λ *prefix-pack*. receive-query-index-challenge $\gg$
 (λ raw. return (*prefix-pack*, raw))))
by (rule *hash-range-budget-bind*)
 (rule *prefix*, rule *receive*)
qed

lemma *hash-collision-budget-checked-staged-query-prefix-receive-with-state*:
assumes *wf*: *staged-budget-wellformed* budgets
and *controlled*: *staged-adversary-controlled* budgets *A*
and *i-bound*: $i \leq \text{rounds}$
shows
hash-collision-budget
 (*staged-query-search-queries* budgets *i* + 1)
 (*checked-staged-query-prefix-receive-with-state* *A* *i*)
unfolding *checked-staged-query-prefix-receive-with-state-def*
proof –
have *prefix-range*:
hash-range-budget
 (*staged-query-search-queries* budgets *i*)
 (*checked-staged-query-prefix-with-state* *A* *i*)
by (rule *hash-range-budget-checked-staged-query-prefix-with-state*
 [OF *wf* *controlled* *i-bound*])
have *prefix-collision*:
hash-collision-budget
 (*staged-query-search-queries* budgets *i*)
 (*checked-staged-query-prefix-with-state* *A* *i*)
by (rule *hash-collision-budget-checked-staged-query-prefix-with-state*
 [OF *wf* *controlled* *i-bound*])
have *receive-range*:

```

 $\wedge$ prefix-pack. hash-range-budget 1
  (receive-query-index-challenge  $\gg$  ( $\lambda$ raw.
    return (prefix-pack, raw)))
proof –
  fix prefix-pack
  have hash-range-budget (1 + 0)
    (receive-query-index-challenge  $\gg$  ( $\lambda$ raw.
      return (prefix-pack, raw)))
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-receive-query-index-challenge,
      rule hash-range-budget-return)
  then show hash-range-budget 1
    (receive-query-index-challenge  $\gg$  ( $\lambda$ raw.
      return (prefix-pack, raw)))
  by simp
qed
have receive-collision:
 $\wedge$ prefix-pack. hash-collision-budget 1
  (receive-query-index-challenge  $\gg$  ( $\lambda$ raw.
    return (prefix-pack, raw)))
proof –
  fix prefix-pack
  have hash-collision-budget (1 + 0)
    (receive-query-index-challenge  $\gg$  ( $\lambda$ raw.
      return (prefix-pack, raw)))
  by (rule hash-collision-budget-bind)
    (rule hash-range-budget-receive-query-index-challenge,
      rule hash-collision-budget-receive-query-index-challenge,
      rule hash-range-budget-return, rule hash-collision-budget-return)
  then show hash-collision-budget 1
    (receive-query-index-challenge  $\gg$  ( $\lambda$ raw.
      return (prefix-pack, raw)))
  by simp
qed
show hash-collision-budget
  (staged-query-search-queries budgets i + 1)
  (checked-staged-query-prefix-with-state A i  $\gg$ 
    ( $\lambda$ prefix-pack. receive-query-index-challenge  $\gg$ 
      ( $\lambda$ raw. return (prefix-pack, raw)))))
  by (rule hash-collision-budget-bind)
    (rule prefix-range, rule prefix-collision, rule receive-range,
      rule receive-collision)
qed

lemma hash-map-output-collision-mono:
  assumes collision: hash-map-output-collision s
  and ext:  $s \leq t$ 
  shows hash-map-output-collision t
proof –

```

```

obtain  $x\ y\ z$  where  $neq: x \neq y$ 
  and  $x\text{-lookup}: \text{fmlookup}\ (\text{HashMap}\ s)\ x = \text{Some}\ z$ 
  and  $y\text{-lookup}: \text{fmlookup}\ (\text{HashMap}\ s)\ y = \text{Some}\ z$ 
  using collision unfolding hash-map-output-collision-def by blast
have  $x\text{-lookup-t}: \text{fmlookup}\ (\text{HashMap}\ t)\ x = \text{Some}\ z$ 
  by (rule hash-extension-lookup[OF  $x\text{-lookup}\ ext$ ])
have  $y\text{-lookup-t}: \text{fmlookup}\ (\text{HashMap}\ t)\ y = \text{Some}\ z$ 
  by (rule hash-extension-lookup[OF  $y\text{-lookup}\ ext$ ])
show ?thesis
  by (rule hash-map-output-collisionI[OF  $neq\ x\text{-lookup-t}\ y\text{-lookup-t}$ ])
qed

lemma checked-staged-query-prefix-hash-map-output-collision-imp-new-collision-on-support:
  assumes support:
     $out \in \text{set-dist}$ 
    (execute (checked-staged-query-prefix-receive-with-state  $A\ i$ )
      adversary-initial-state)
  and collision:
    checked-staged-query-prefix-hash-map-output-collision out
  shows hash-new-collision-event adversary-initial-state out
proof (cases out)
case None
then show ?thesis
  using collision
  unfolding checked-staged-query-prefix-hash-map-output-collision-def
  by simp
next
case (Some packed)
then obtain  $prefix\ prefix\text{-state}\ raw\ raw\text{-state}$  where  $out\text{-eq}$ :
   $out = \text{Some}\ (((prefix,\ prefix\text{-state}),\ raw),\ raw\text{-state})$ 
  by (cases packed) auto
have receive-support:
   $\text{Some}\ (((prefix,\ prefix\text{-state}),\ raw),\ raw\text{-state}) \in$ 
     $\text{set-dist}$ 
    (execute (checked-staged-query-prefix-receive-with-state  $A\ i$ )
      adversary-initial-state)
  using support unfolding  $out\text{-eq}$  .
have recv:
   $\text{Some}\ (raw,\ raw\text{-state}) \in$ 
     $\text{set-dist}\ (\text{execute}\ \text{receive-query-index-challenge}\ prefix\text{-state})$ 
  by (rule
    checked-staged-query-prefix-receive-with-state-receive-support
    [OF receive-support])
have  $prefix\text{-collision}: \text{hash-map-output-collision}\ prefix\text{-state}$ 
  using collision unfolding  $out\text{-eq}$ 
  checked-staged-query-prefix-hash-map-output-collision-def by simp
have  $raw\text{-ext}: prefix\text{-state} \leq raw\text{-state}$ 
  by (rule receive-query-index-challenge-extends[OF recv])
have  $raw\text{-collision}: \text{hash-map-output-collision}\ raw\text{-state}$ 

```

```

    by (rule hash-map-output-collision-mono[OF prefix-collision raw-ext])
  show ?thesis
    unfolding out-eq hash-new-collision-event-def
      hash-map-new-output-collision-def
    using raw-collision by simp
qed

lemma checked-staged-query-prefix-hash-map-output-collision-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound:  $i \leq \text{rounds}$ 
  shows
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      checked-staged-query-prefix-hash-map-output-collision
      adversary-initial-state  $\leq$ 
      hash-collision-budget-value 0
      (staged-query-search-queries budgets i + 1)
  proof -
    let ?M = checked-staged-query-prefix-receive-with-state A i
    let ?q = staged-query-search-queries budgets i + 1
    have event-le:
      wp-event ?M checked-staged-query-prefix-hash-map-output-collision
        adversary-initial-state  $\leq$ 
        wp-event ?M (hash-new-collision-event adversary-initial-state)
        adversary-initial-state
    by (rule wp-event-mono-on-support)
      (rule
        checked-staged-query-prefix-hash-map-output-collision-imp-new-collision-on-support)
    have collision-budget:
      hash-collision-budget ?q ?M
    by (rule hash-collision-budget-checked-staged-query-prefix-receive-with-state
      [OF wf controlled i-bound])
    have new-collision-bound:
      wp-event ?M (hash-new-collision-event adversary-initial-state)
        adversary-initial-state  $\leq$ 
        hash-collision-budget-value
          (card (hash-map-output-values adversary-initial-state)) ?q
    using collision-budget adversary-initial-state-no-output-collision
    unfolding hash-collision-budget-def by blast
    have wp-event ?M checked-staged-query-prefix-hash-map-output-collision
      adversary-initial-state  $\leq$ 
      hash-collision-budget-value
        (card (hash-map-output-values adversary-initial-state)) ?q
    by (rule order-trans[OF event-le new-collision-bound])
    then show ?thesis
      by simp
  qed

lemma checked-staged-query-prefix-partial-target-prehit-bound:

```

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *i-bound*: $i \leq \text{rounds}$
shows
wp-event (*checked-staged-query-prefix-receive-with-state* *A i*)
(*checked-staged-query-prefix-dynamic-index-prehit*
(*staged-query-prefix-partial-query-target i*))
adversary-initial-state $\leq$
staged-phase-relation-error size
(*staged-query-search-queries budgets i + 1*)
proof (*rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)
show *HashMap adversary-initial-state = fmempty*
by (*simp add: adversary-initial-state-def*)
show *hash-relation-program*
(*checked-staged-query-prefix-dynamic-index-prehit-relation A i*
adversary-initial-state
(*staged-query-prefix-partial-query-target i*))
size (*staged-query-search-queries budgets i + 1*)
(*checked-staged-query-prefix-receive-with-state A i*)
by (*rule hash-relation-program-checked-staged-query-prefix-receive-with-state*
[*OF wf controlled i-bound*])
(*rule*
checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
qed

lemma *checked-staged-query-prefix-committed-target-prehit-bound*:
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *i-bound*: $i \leq \text{rounds}$
shows
wp-event (*checked-staged-query-prefix-receive-with-state A i*)
(*checked-staged-query-prefix-dynamic-index-prehit*
($\lambda \text{prefix prefix-state.}$
staged-query-prefix-committed-query-target prefix prefix-state))
adversary-initial-state $\leq$
staged-phase-relation-error size
(*staged-query-search-queries budgets i + 1*)
proof (*rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)
show *HashMap adversary-initial-state = fmempty*
by (*simp add: adversary-initial-state-def*)
show *hash-relation-program*
(*checked-staged-query-prefix-dynamic-index-prehit-relation A i*
adversary-initial-state
($\lambda \text{prefix prefix-state.}$
staged-query-prefix-committed-query-target prefix prefix-state))
size (*staged-query-search-queries budgets i + 1*)
(*checked-staged-query-prefix-receive-with-state A i*)
by (*rule hash-relation-program-checked-staged-query-prefix-receive-with-state*
[*OF wf controlled i-bound*])

(rule
checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
qed

lemma *checked-staged-query-prefix-clean-committed-target-prehit-bound*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i \leq \text{rounds}$
shows
wp-event (*checked-staged-query-prefix-receive-with-state A i*)
(*checked-staged-query-prefix-dynamic-index-prehit*
staged-query-prefix-clean-committed-query-target)
adversary-initial-state $\leq$
staged-phase-relation-error size
(*staged-query-search-queries budgets i + 1*)
proof (rule *checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)
show *HashMap adversary-initial-state = fmempty*
by (*simp add: adversary-initial-state-def*)
show *hash-relation-program*
(*checked-staged-query-prefix-dynamic-index-prehit-relation A i*
adversary-initial-state
staged-query-prefix-clean-committed-query-target)
size (*staged-query-search-queries budgets i + 1*)
(*checked-staged-query-prefix-receive-with-state A i*)
by (rule *hash-relation-program-checked-staged-query-prefix-receive-with-state*
[*OF wf controlled i-bound*])
(rule
checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
qed

lemma *checked-staged-query-prefix-clean-length-committed-target-prehit-bound*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i \leq \text{rounds}$
shows
wp-event (*checked-staged-query-prefix-receive-with-state A i*)
(*checked-staged-query-prefix-dynamic-index-prehit*
staged-query-prefix-clean-length-committed-query-target)
adversary-initial-state $\leq$
staged-phase-relation-error size
(*staged-query-search-queries budgets i + 1*)
proof (rule *checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)
show *HashMap adversary-initial-state = fmempty*
by (*simp add: adversary-initial-state-def*)
show *hash-relation-program*
(*checked-staged-query-prefix-dynamic-index-prehit-relation A i*
adversary-initial-state
staged-query-prefix-clean-length-committed-query-target)
size (*staged-query-search-queries budgets i + 1*)

(*checked-staged-query-prefix-receive-with-state A i*)
by (*rule hash-relation-program-checked-staged-query-prefix-receive-with-state*
 [*OF wf controlled i-bound*])
 (*rule*
checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
qed

definition *checked-staged-query-prefix-dynamic-index-fresh-output-hit*
 $:: ('f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow$
 $\text{nat set}) \Rightarrow$
 $((('f \text{ staged-query-prefix-data} \times 'f \text{ protocol-channel}) \times 'f) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-query-prefix-dynamic-index-fresh-output-hit B out $\longleftrightarrow$
 (*case out of*
None $\Rightarrow$ *False*
 | *Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\Rightarrow$
hash-new-output-hit-event
 (*query-index-raw-preimage* (*B prefix prefix-state*))
prefix-state (*Some* (*raw*, *raw-state*)))

lemma *checked-staged-query-prefix-dynamic-index-hit-imp-prehit-or-fresh:*

assumes *absent:*

$\neg$ *checked-staged-query-prefix-dynamic-index-prehit B*
 (*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*))

and *prefix-support:*

Some ((*prefix*, *prefix-state*), *prefix-state*) $\in$
set-dist
 (*execute* (*checked-staged-query-prefix-with-state A i*) *s*)

and *receive-support:*

Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$
set-dist
 (*execute* (*checked-staged-query-prefix-receive-with-state A i*) *s*)

and *hit:*

checked-staged-query-prefix-dynamic-index-hit B
 (*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*))

shows

hash-new-output-hit-event
 (*query-index-raw-preimage* (*B prefix prefix-state*)) *prefix-state*
 (*Some* (*raw*, *raw-state*))

proof –

have *raw-in:*

raw $\in$ *query-index-raw-preimage* (*B prefix prefix-state*)

using *hit unfolding* *checked-staged-query-prefix-dynamic-index-hit-def*
query-index-raw-preimage-def

by *simp*

have *no-lookup:*

fmlookup (*HashMap prefix-state*)
 (*QueryIndexChallenge* (*PQueryCounter prefix-state*))

```

    (PState prefix-state)) = None
proof (cases
  fmlookup (HashMap prefix-state)
    (QueryIndexChallenge (PQueryCounter prefix-state)
      (PState prefix-state)))
  case None
  then show ?thesis .
next
  case (Some old-raw)
  have old-lookup-prefix:
    fmlookup (HashMap prefix-state)
      (QueryIndexChallenge (PQueryCounter prefix-state)
        (PState prefix-state)) = Some old-raw
  using Some by simp
  have old-in: old-raw ∈ query-index-raw-preimage
    (B prefix prefix-state)
  proof –
  have old-idx:
    index (to-nat old-raw) ∈ B prefix prefix-state
  proof –
  have raw-idx:
    index (to-nat raw) ∈ B prefix prefix-state
  using hit
  unfolding checked-staged-query-prefix-dynamic-index-hit-def
  by simp
  have recv:
    Some (raw, raw-state) ∈
      set-dist
        (execute receive-query-index-challenge prefix-state)
  by (rule
    checked-staged-query-prefix-receive-with-state-receive-support
      [OF receive-support])
  have recv-lookup:
    fmlookup (HashMap raw-state)
      (QueryIndexChallenge (PQueryCounter prefix-state)
        (PState prefix-state)) = Some raw
  using receive-query-index-challenge-outcome[OF recv] by blast
  have ext: prefix-state ≤ raw-state
  using receive-query-index-challenge-extends[OF recv] .
  have old-lookup-raw-state:
    fmlookup (HashMap raw-state)
      (QueryIndexChallenge (PQueryCounter prefix-state)
        (PState prefix-state)) = Some old-raw
  by (rule hash-extension-lookup[OF old-lookup-prefix ext])
  have old-raw-eq: old-raw = raw
  using recv-lookup old-lookup-raw-state by simp
  show ?thesis
  using raw-idx unfolding old-raw-eq .
qed

```

```

  show ?thesis
    using old-idx unfolding query-index-raw-preimage-def by simp
qed
have prehit:
  checked-staged-query-prefix-dynamic-index-prehit B
  (Some (((prefix, prefix-state), raw), raw-state))
  using old-lookup-prefix old-in
  by (auto simp: checked-staged-query-prefix-dynamic-index-prehit-def)
then show ?thesis
  using absent by contradiction
qed
from receive-support have recv:
  Some (raw, raw-state) ∈
  set-dist (execute receive-query-index-challenge prefix-state)
  by (rule
    checked-staged-query-prefix-receive-with-state-receive-support)
have recv-lookup:
  fmlookup (HashMap raw-state)
  (QueryIndexChallenge (PQueryCounter prefix-state)
    (PState prefix-state)) = Some raw
  using receive-query-index-challenge-outcome[OF recv] by blast
have new-hit:
  hash-map-new-output-hit
  (query-index-raw-preimage (B prefix prefix-state))
  prefix-state raw-state
  unfolding hash-map-new-output-hit-def
  using no-lookup recv-lookup raw-in by blast
show ?thesis
  unfolding hash-new-output-hit-event-def using new-hit by simp
qed

lemma checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-fresh:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-hit B) s ≤
  wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-prehit B) s +
  wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-fresh-output-hit B) s
proof -
  let ?M = checked-staged-query-prefix-receive-with-state A i
  let ?hit = checked-staged-query-prefix-dynamic-index-hit B
  let ?pre = checked-staged-query-prefix-dynamic-index-prehit B
  let ?fresh = checked-staged-query-prefix-dynamic-index-fresh-output-hit B
  have wp-event ?M ?hit s ≤ wp-event ?M (λout. ?pre out ∨ ?fresh out) s
  proof (rule wp-event-mono-on-support)
    fix out
    assume support: out ∈ set-dist (execute ?M s)
    and hit: ?hit out
    show ?pre out ∨ ?fresh out

```

```

proof (cases out)
  case None
  then show ?thesis
    using hit unfolding checked-staged-query-prefix-dynamic-index-hit-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state where out-eq:
    out = Some (((prefix, prefix-state), raw), raw-state)
    by (cases packed, auto split: prod.splits)
  show ?thesis
  proof (cases ?pre out)
    case True
    then show ?thesis by simp
  next
  case pre-absent: False
  have prefix-support:
    Some ((prefix, prefix-state), prefix-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-with-state A i) s)
    by (rule
      checked-staged-query-prefix-receive-with-state-prefix-support
      [OF support[unfolded out-eq]])
  have fresh:
    hash-new-output-hit-event
      (query-index-raw-preimage (B prefix prefix-state))
      prefix-state (Some (raw, raw-state)))
    by (rule checked-staged-query-prefix-dynamic-index-hit-imp-prehit-or-fresh
      [OF - prefix-support support[unfolded out-eq]
        hit[unfolded out-eq]])
      (use pre-absent out-eq in simp)
  then show ?thesis
    unfolding out-eq
      checked-staged-query-prefix-dynamic-index-fresh-output-hit-def
    by simp
  qed
qed
qed
also have ... ≤ wp-event ?M ?pre s + wp-event ?M ?fresh s
  by (rule wp-event-union-bound)
finally show ?thesis .
qed

```

```

lemma wp-receive-query-index-challenge-new-output-hit-bound:
  fixes B :: 'f set
  and s :: ('f, 'a) protocol-channel-scheme
  shows
    wp-event receive-query-index-challenge
      (hash-new-output-hit-event B s) s ≤ nnreal (card B) / nnreal size

```

```

proof (cases
  fmlookup (HashMap s)
    (QueryIndexChallenge (PQueryCounter s) (PState s)))
case None
have event-le:
  wp-event receive-query-index-challenge
    (hash-new-output-hit-event B s) s ≤
  wp-event receive-query-index-challenge
    (λout. case out of None ⇒ False | Some (raw, -) ⇒ raw ∈ B) s
proof (rule wp-event-mono-on-support)
fix out
assume support: out ∈ set-dist (execute receive-query-index-challenge s)
and hit: hash-new-output-hit-event B s out
show (case out of None ⇒ False | Some (raw, -) ⇒ raw ∈ B)
proof (cases out)
  case None
  then show ?thesis
    using hit unfolding hash-new-output-hit-event-def by simp
next
  case (Some packed)
  then obtain raw t where packed-eq: packed = (raw, t)
    by (cases packed) simp
  have out':
    Some (raw, t) ∈
      set-dist (execute receive-query-index-challenge s)
    using support unfolding Some packed-eq .
  have lookup:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
    using receive-query-index-challenge-outcome[OF out'] by blast
  have new-hit: hash-map-new-output-hit B s t
    using hit unfolding Some packed-eq hash-new-output-hit-event-def
    by simp
  then show ?thesis
  proof –
    from new-hit obtain x y where none-x:
      fmlookup (HashMap s) x = None
    and some-x: fmlookup (HashMap t) x = Some y
    and y-in: y ∈ B
    unfolding hash-map-new-output-hit-def by blast
  let ?q = QueryIndexChallenge (PQueryCounter s) (PState s)
  have raw ∈ B
  proof (cases x = ?q)
    case True
    then have Some y = Some raw
      using some-x lookup by simp
    then show ?thesis
      using y-in by simp
  next

```

```

    case False
    have fmlookup (HashMap t) x = fmlookup (HashMap s) x
      by (rule receive-query-index-challenge-preserves-other-lookup
        [OF out' False])
    then show ?thesis
      using none-x some-x by simp
    qed
    then show ?thesis
      unfolding Some packed-eq by simp
    qed
  qed
  also have ... = nnreal (card B) / nnreal size
    by (rule wp-receive-query-index-challenge-fresh-set[OF None])
  finally show ?thesis .
next
case (Some old)
have old-lookup:
  fmlookup (HashMap s)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some old
  using Some by simp
have no-new-on-support:
   $\bigwedge out. out \in \text{set-dist } (\text{execute receive-query-index-challenge } s) \implies$ 
     $\neg \text{hash-new-output-hit-event } B \ s \ out$ 
proof -
  fix out
  assume support: out  $\in$  set-dist (execute receive-query-index-challenge s)
  show  $\neg \text{hash-new-output-hit-event } B \ s \ out$ 
  proof (cases out)
    case None
    then show ?thesis
      unfolding hash-new-output-hit-event-def by simp
  next
  case (Some packed)
  then obtain raw t where packed-eq: packed = (raw, t)
    by (cases packed) simp
  have out':
    Some (raw, t)  $\in$ 
      set-dist (execute receive-query-index-challenge s)
    using support unfolding Some packed-eq .
  have ext: s  $\leq$  t
    by (rule receive-query-index-challenge-extends[OF out'])
  show ?thesis
  proof
    assume bad: hash-new-output-hit-event B s out
    have bad':
       $\exists x \ y. \text{fmlookup } (\text{HashMap } s) \ x = \text{None} \wedge$ 
         $\text{fmlookup } (\text{HashMap } t) \ x = \text{Some } y \wedge y \in B$ 
      using bad

```

```

unfolding Some packed-eq hash-new-output-hit-event-def
  hash-map-new-output-hit-def
by simp
then obtain  $x\ y$  where none-x:
  fmlookup (HashMap s) x = None
and some-x: fmlookup (HashMap t) x = Some y
and y-in:  $y \in B$ 
by blast
let  $?q = \text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)$ 
show False
proof (cases x = ?q)
  case True
  then show ?thesis
    using none-x old-lookup by simp
next
  case False
  have fmlookup (HashMap t) x = fmlookup (HashMap s) x
    by (rule receive-query-index-challenge-preserves-other-lookup
      [OF out' False])
  then show ?thesis
    using none-x some-x by simp
qed
qed
qed
qed
have wp-event receive-query-index-challenge
  (hash-new-output-hit-event B s)  $s \leq$ 
wp-event receive-query-index-challenge ( $\lambda\cdot.$  False)  $s$ 
by (rule wp-event-mono-on-support)
  (use no-new-on-support in blast)
also have  $\dots = 0$ 
  unfolding wp-event-def wp-def dist-expect-def by simp
also have  $0 \leq \text{nnreal } (\text{card } B) / \text{nnreal size}$ 
by simp
finally show ?thesis .
qed

```

lemma *checked-staged-query-prefix-dynamic-index-fresh-output-hit-bound*:

fixes $C :: \text{prob}$

assumes *raw-bound: query-index-raw-preimage-bound*

and *prefix-bound*:

$\bigwedge \text{prefix prefix-state.}$

Some ((prefix, prefix-state), prefix-state) \in
set-dist

$(\text{execute } (\text{checked-staged-query-prefix-with-state } A\ i)\ s) \implies$

$B\ \text{prefix}\ \text{prefix-state} \subseteq \text{query-sample-space} \wedge$

$\text{nnreal } (\text{card } (B\ \text{prefix}\ \text{prefix-state})) /$

$\text{nnreal } (\text{card } \text{query-sample-space}) \leq C$

shows


```

    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-fresh-output-hit B) s ≤ C
  unfolding checked-staged-query-prefix-receive-with-state-def
proof (rule wp-event-bind-bound-by-cont)
  show
    ¬ checked-staged-query-prefix-dynamic-index-fresh-output-hit B None
  unfolding checked-staged-query-prefix-dynamic-index-fresh-output-hit-def
  by simp
next
  fix prefix-pack prefix-state
  assume prefix-support:
    Some (prefix-pack, prefix-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-with-state A i) s)
  obtain prefix captured-state where prefix-pack-eq:
    prefix-pack = (prefix, captured-state)
  by (cases prefix-pack) simp
  have captured-eq: captured-state = prefix-state
  using checked-staged-query-prefix-with-state-outcome-state
    [OF prefix-support[unfolded prefix-pack-eq]]
  by simp
  have support':
    Some ((prefix, prefix-state), prefix-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-with-state A i) s)
  using checked-staged-query-prefix-with-state-outcome-canonical
    [OF prefix-support[unfolded prefix-pack-eq]]
    captured-eq
  by simp
  from prefix-bound[OF support'] have subset:
    B prefix prefix-state ⊆ query-sample-space
  and frac:
    nnreal (card (B prefix prefix-state)) /
      nnreal (card query-sample-space) ≤ C
  by blast+
  have raw-fraction:
    nnreal (card (query-index-raw-preimage (B prefix prefix-state))) /
      nnreal size ≤ C
  proof –
    have nnreal (card (query-index-raw-preimage (B prefix prefix-state))) /
      nnreal size ≤
      nnreal (card (B prefix prefix-state)) /
        nnreal (card query-sample-space)
    using raw-bound subset unfolding query-index-raw-preimage-bound-def
    by blast
    also have ... ≤ C
    by (rule frac)
  finally show ?thesis .
qed

```

```

have cont-eq:
  wp-event
    (receive-query-index-challenge  $\gg$  ( $\lambda$ raw.
      return (prefix-pack, raw)))
    (checked-staged-query-prefix-dynamic-index-fresh-output-hit B)
    prefix-state =
  wp-event receive-query-index-challenge
    (hash-new-output-hit-event
      (query-index-raw-preimage (B prefix prefix-state)) prefix-state)
    prefix-state
  unfolding wp-event-def
  apply (subst wp-bind)
  apply (rule arg-cong[where
    f= $\lambda Q$ . wp receive-query-index-challenge Q prefix-state])
  apply (rule ext)
  apply (simp add: wp-return prefix-pack-eq captured-eq
    checked-staged-query-prefix-dynamic-index-fresh-output-hit-def
    hash-new-output-hit-event-def
    split: option.splits prod.splits)
  done
show wp-event
  (receive-query-index-challenge  $\gg$  ( $\lambda$ raw.
    return (prefix-pack, raw)))
  (checked-staged-query-prefix-dynamic-index-fresh-output-hit B)
  prefix-state  $\leq C$ 
  unfolding cont-eq
  by (rule order-trans[
    OF wp-receive-query-index-challenge-new-output-hit-bound raw-fac])
qed

lemma checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error:
  fixes C :: prob
  assumes raw-bound: query-index-raw-preimage-bound
  and prefix-bound:
     $\bigwedge$ prefix prefix-state.
      Some ((prefix, prefix-state), prefix-state)  $\in$ 
        set-dist
        (execute (checked-staged-query-prefix-with-state A i) s)  $\implies$ 
        B prefix prefix-state  $\subseteq$  query-sample-space  $\wedge$ 
        nnreal (card (B prefix prefix-state)) /
        nnreal (card query-sample-space)  $\leq C$ 
  shows
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-hit B) s  $\leq$ 
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-prehit B) s + C
proof –
  have wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-hit B) s  $\leq$ 

```

$wp\text{-}event$ ($checked\text{-}staged\text{-}query\text{-}prefix\text{-}receive\text{-}with\text{-}state$ A i)
 ($checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}prehit$ B) s +
 $wp\text{-}event$ ($checked\text{-}staged\text{-}query\text{-}prefix\text{-}receive\text{-}with\text{-}state$ A i)
 ($checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}fresh\text{-}output\text{-}hit$ B) s
by ($rule$ $checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}hit\text{-}bound\text{-}by\text{-}prehit\text{-}and\text{-}fresh$)
also have ... $\leq$
 $wp\text{-}event$ ($checked\text{-}staged\text{-}query\text{-}prefix\text{-}receive\text{-}with\text{-}state$ A i)
 ($checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}prehit$ B) s + C
by ($intro$ $add\text{-}mono$ $order\text{-}refl$
 $checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}fresh\text{-}output\text{-}hit\text{-}bound$
 $[OF$ $raw\text{-}bound$ $prefix\text{-}bound]$)
finally show $?thesis$.
qed

lemma $query\text{-}sample\text{-}space\text{-}fraction\text{-}le\text{-}one$:
assumes $subset$: $B \subseteq query\text{-}sample\text{-}space$
shows $nnreal$ ($card$ B) / $nnreal$ ($card$ $query\text{-}sample\text{-}space$) $\leq$ ($1::prob$)
proof –
have $finite\text{-}B$: $finite$ B
by ($rule$ $finite\text{-}subset[OF$ $subset$ $finite\text{-}query\text{-}sample\text{-}space]$)
have $card\text{-}le$: $card$ $B \leq card$ $query\text{-}sample\text{-}space$
by ($rule$ $card\text{-}mono[OF$ $finite\text{-}query\text{-}sample\text{-}space$ $subset]$)
have $nnreal$ ($card$ B) / $nnreal$ ($card$ $query\text{-}sample\text{-}space$) $\leq$
 $nnreal$ ($card$ $query\text{-}sample\text{-}space$) / $nnreal$ ($card$ $query\text{-}sample\text{-}space$)
by ($rule$ $nnreal\text{-}nat\text{-}divide\text{-}right\text{-}mono[OF$ $card\text{-}le]$)
also have ... = ($1::prob$)
by ($rule$ $nnreal\text{-}nat\text{-}divide\text{-}self$) ($simp$ add : $query\text{-}sample\text{-}space\text{-}size\text{-}pos$)
finally show $?thesis$.
qed

lemma $checked\text{-}staged\text{-}query\text{-}prefix\text{-}partial\text{-}target\text{-}hit\text{-}bound\text{-}by\text{-}prehit\text{-}plus\text{-}one$:
assumes $raw\text{-}bound$: $query\text{-}index\text{-}raw\text{-}preimage\text{-}bound$
shows
 $wp\text{-}event$ ($checked\text{-}staged\text{-}query\text{-}prefix\text{-}receive\text{-}with\text{-}state$ A i)
 ($checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}hit$
 ($staged\text{-}query\text{-}prefix\text{-}partial\text{-}query\text{-}target$ i)) $s \leq$
 $wp\text{-}event$ ($checked\text{-}staged\text{-}query\text{-}prefix\text{-}receive\text{-}with\text{-}state$ A i)
 ($checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}prehit$
 ($staged\text{-}query\text{-}prefix\text{-}partial\text{-}query\text{-}target$ i)) s + 1
proof ($rule$
 $checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}hit\text{-}bound\text{-}by\text{-}prehit\text{-}and\text{-}query\text{-}error$
 $[OF$ $raw\text{-}bound]$)
fix $prefix$ $prefix\text{-}state$
assume $support$:
 $Some$ (($prefix$, $prefix\text{-}state$), $prefix\text{-}state$) $\in$
 $set\text{-}dist$ ($execute$ ($checked\text{-}staged\text{-}query\text{-}prefix\text{-}with\text{-}state$ A i) s)
have $subset$:
 $staged\text{-}query\text{-}prefix\text{-}partial\text{-}query\text{-}target$ i $prefix$ $prefix\text{-}state \subseteq$
 $query\text{-}sample\text{-}space$

```

    by (rule staged-query-prefix-partial-query-target-subset)
  have frac:
    nnreal
      (card (staged-query-prefix-partial-query-target i prefix prefix-state)) /
      nnreal (card query-sample-space) ≤ (1::prob)
  by (rule query-sample-space-fraction-le-one[OF subset])
  show staged-query-prefix-partial-query-target i prefix prefix-state ⊆
    query-sample-space ∧
    nnreal
      (card (staged-query-prefix-partial-query-target i prefix prefix-state)) /
      nnreal (card query-sample-space) ≤ (1::prob)
  using subset frac by blast
qed

lemma checked-staged-query-prefix-partial-target-hit-bound-by-relation-plus-one:
  assumes raw-bound: query-index-raw-preimage-bound
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i ≤ rounds
  shows
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-hit
        (staged-query-prefix-partial-query-target i))
      adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) + 1
proof -
  have wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-partial-query-target i))
    adversary-initial-state ≤
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-prehit
        (staged-query-prefix-partial-query-target i))
      adversary-initial-state + 1
  by (rule checked-staged-query-prefix-partial-target-hit-bound-by-prehit-plus-one
    [OF raw-bound])
  also have ... ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + 1
  by (intro add-mono order-refl
    checked-staged-query-prefix-partial-target-prehit-bound
    [OF wf controlled i-bound])
  finally show ?thesis .
qed

```

```

lemma checked-staged-query-prefix-committed-target-hit-bound-by-relation-and-query-error:
  assumes raw-bound: query-index-raw-preimage-bound
  and wf: staged-budget-wellformed budgets

```

and *controlled*: staged-adversary-controlled budgets A
and *i-bound*: $i \leq \text{rounds}$
and *prefix-clean*:
 $\bigwedge \text{prefix prefix-state.}$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\neg \text{merkle-root-binding-collision } (\text{sqp-trace-root prefix})$
 $\text{prefix-state} \wedge$
 $(\forall \text{rt. sqp-composition-fri-roots prefix} \neq [] \longrightarrow$
 $\text{rt} = \text{hd } (\text{sqp-composition-fri-roots prefix}) \longrightarrow$
 $\neg \text{merkle-root-binding-collision rt prefix-state})$
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $(\lambda \text{prefix prefix-state.}$
 $\text{staged-query-prefix-committed-query-target prefix prefix-state}))$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$
proof –
have *hit-bound*:
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $(\lambda \text{prefix prefix-state.}$
 $\text{staged-query-prefix-committed-query-target prefix prefix-state}))$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $(\lambda \text{prefix prefix-state.}$
 $\text{staged-query-prefix-committed-query-target prefix prefix-state}))$
 $\text{adversary-initial-state} + \text{query-error-bound}$
proof (rule *checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error*
 $[OF \text{ raw-bound}]$)
fix *prefix prefix-state*
assume *support*:
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state})$
from *prefix-clean* $[OF \text{ support}]$
have *trace-clean*:
 $\neg \text{merkle-root-binding-collision } (\text{sqp-trace-root prefix})$
 prefix-state
and *comp-clean*:
 $\bigwedge \text{rt. sqp-composition-fri-roots prefix} \neq [] \implies$
 $\text{rt} = \text{hd } (\text{sqp-composition-fri-roots prefix}) \implies$
 $\neg \text{merkle-root-binding-collision rt prefix-state}$

by *blast* +
 show
 (λ prefix prefix-state.
 staged-query-prefix-committed-query-target prefix prefix-state)
 prefix prefix-state $\subseteq$ query-sample-space $\wedge$
 nnreal
 (card
 ((λ prefix prefix-state.
 staged-query-prefix-committed-query-target prefix prefix-state)
 prefix prefix-state)) /
 nnreal (card query-sample-space) $\leq$ query-error-bound
 using
 staged-query-prefix-committed-query-target-subset[*of* prefix prefix-state]
 staged-query-prefix-committed-query-target-fraction-bound-if-no-root-collisions
 [*OF* trace-clean comp-clean]
 by *blast*
 qed
 also have ... $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) + query-error-bound
 by (intro add-mono order-refl
 checked-staged-query-prefix-committed-target-prehit-bound
 [*OF* wf controlled i-bound])
 finally show ?thesis .
 qed

lemma checked-staged-query-prefix-clean-committed-target-hit-bound:
assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i \leq$ rounds
shows
 wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-hit
 staged-query-prefix-clean-committed-query-target)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) + query-error-bound
proof –
have hit-bound:
 wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-hit
 staged-query-prefix-clean-committed-query-target)
 adversary-initial-state $\leq$
 wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-prehit
 staged-query-prefix-clean-committed-query-target)
 adversary-initial-state + query-error-bound
proof (rule

```

    checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
    [OF raw-bound])
  fix prefix prefix-state
  assume support:
    Some ((prefix, prefix-state), prefix-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-with-state A i)
        adversary-initial-state)
  have subset:
    staged-query-prefix-clean-committed-query-target prefix prefix-state
      ⊆ query-sample-space
  by (rule staged-query-prefix-clean-committed-query-target-subset)
  have frac:
    nnreal
      (card
        (staged-query-prefix-clean-committed-query-target prefix
          prefix-state)) /
        nnreal (card query-sample-space) ≤ query-error-bound
  by (rule staged-query-prefix-clean-committed-query-target-fraction-bound)
  show staged-query-prefix-clean-committed-query-target prefix prefix-state
    ⊆ query-sample-space ∧
    nnreal
      (card
        (staged-query-prefix-clean-committed-query-target prefix
          prefix-state)) /
        nnreal (card query-sample-space) ≤ query-error-bound
  using subset frac by blast
qed
also have ... ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) + query-error-bound
  by (intro add-mono order-refl
    checked-staged-query-prefix-clean-committed-target-prehit-bound
    [OF wf controlled i-bound])
  finally show ?thesis .
qed

lemma checked-staged-query-prefix-committed-target-hit-bound-by-clean-and-collision:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-hit
      staged-query-prefix-committed-query-target) s ≤
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-hit
      staged-query-prefix-clean-committed-query-target) s +
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    checked-staged-query-prefix-committed-root-collision s
proof -
  let ?M = checked-staged-query-prefix-receive-with-state A i
  let ?hit =

```

checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-committed-query-target
let *?clean* =
checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-clean-committed-query-target
let *?collision* = *checked-staged-query-prefix-committed-root-collision*
have *wp-event ?M ?hit s ≤ wp-event ?M (λout. ?clean out ∨ ?collision out) s*
by (*rule wp-event-mono*)
(*rule checked-staged-query-prefix-committed-hit-imp-clean-hit-or-root-collision*)
also have ... ≤ *wp-event ?M ?clean s + wp-event ?M ?collision s*
by (*rule wp-event-union-bound*)
finally show *?thesis* .
qed

lemma *checked-staged-query-prefix-committed-target-hit-bound-by-relation-query-error-and-collision*:
assumes *raw-bound: query-index-raw-preimage-bound*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i ≤ rounds*
shows
wp-event (checked-staged-query-prefix-receive-with-state A i)
(*checked-staged-query-prefix-dynamic-index-hit*
staged-query-prefix-committed-query-target)
adversary-initial-state ≤
staged-phase-relation-error size
(*staged-query-search-queries budgets i + 1*) +
query-error-bound +
wp-event (checked-staged-query-prefix-receive-with-state A i)
checked-staged-query-prefix-committed-root-collision
adversary-initial-state

proof —
have *split*:
wp-event (checked-staged-query-prefix-receive-with-state A i)
(*checked-staged-query-prefix-dynamic-index-hit*
staged-query-prefix-committed-query-target)
adversary-initial-state ≤
wp-event (checked-staged-query-prefix-receive-with-state A i)
(*checked-staged-query-prefix-dynamic-index-hit*
staged-query-prefix-clean-committed-query-target)
adversary-initial-state +
wp-event (checked-staged-query-prefix-receive-with-state A i)
checked-staged-query-prefix-committed-root-collision
adversary-initial-state
by (*rule*
checked-staged-query-prefix-committed-target-hit-bound-by-clean-and-collision)
have *clean-bound*:
wp-event (checked-staged-query-prefix-receive-with-state A i)
(*checked-staged-query-prefix-dynamic-index-hit*
staged-query-prefix-clean-committed-query-target)

$adversary-initial-state \leq$
 $staged-phase-relation-error\ size$
 $(staged-query-search-queries\ budgets\ i + 1) + query-error-bound$
by (rule *checked-staged-query-prefix-clean-committed-target-hit-bound*
 $[OF\ raw-bound\ wf\ controlled\ i-bound]$)
have *tail-bound*:
 $wp-event\ (checked-staged-query-prefix-receive-with-state\ A\ i)$
 $(checked-staged-query-prefix-dynamic-index-hit$
 $staged-query-prefix-clean-committed-query-target)$
 $adversary-initial-state +$
 $wp-event\ (checked-staged-query-prefix-receive-with-state\ A\ i)$
 $checked-staged-query-prefix-committed-root-collision$
 $adversary-initial-state \leq$
 $staged-phase-relation-error\ size$
 $(staged-query-search-queries\ budgets\ i + 1) +$
 $query-error-bound +$
 $wp-event\ (checked-staged-query-prefix-receive-with-state\ A\ i)$
 $checked-staged-query-prefix-committed-root-collision$
 $adversary-initial-state$
using *clean-bound* **by** (*simp add: add-mono*)
show *?thesis*
by (rule *order-trans* $[OF\ split\ tail-bound]$)
qed

lemma *checked-staged-query-prefix-clean-length-committed-target-hit-bound*:
assumes *raw-bound: query-index-raw-preimage-bound*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i ≤ rounds*
shows
 $wp-event\ (checked-staged-query-prefix-receive-with-state\ A\ i)$
 $(checked-staged-query-prefix-dynamic-index-hit$
 $staged-query-prefix-clean-length-committed-query-target)$
 $adversary-initial-state \leq$
 $staged-phase-relation-error\ size$
 $(staged-query-search-queries\ budgets\ i + 1) + query-error-bound$
proof –
have *hit-bound*:
 $wp-event\ (checked-staged-query-prefix-receive-with-state\ A\ i)$
 $(checked-staged-query-prefix-dynamic-index-hit$
 $staged-query-prefix-clean-length-committed-query-target)$
 $adversary-initial-state \leq$
 $wp-event\ (checked-staged-query-prefix-receive-with-state\ A\ i)$
 $(checked-staged-query-prefix-dynamic-index-prehit$
 $staged-query-prefix-clean-length-committed-query-target)$
 $adversary-initial-state + query-error-bound$
proof (rule
 $checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error$
 $[OF\ raw-bound]$)

```

fix prefix prefix-state
assume support:
  Some ((prefix, prefix-state), prefix-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
      adversary-initial-state)
have subset:
  staged-query-prefix-clean-length-committed-query-target prefix
    prefix-state ⊆ query-sample-space
  by (rule staged-query-prefix-clean-length-committed-query-target-subset)
have frac:
  nnreal
    (card
      (staged-query-prefix-clean-length-committed-query-target prefix
        prefix-state)) /
    nnreal (card query-sample-space) ≤ query-error-bound
  by (rule staged-query-prefix-clean-length-committed-query-target-fraction-bound)
show staged-query-prefix-clean-length-committed-query-target prefix
  prefix-state ⊆ query-sample-space ∧
  nnreal
    (card
      (staged-query-prefix-clean-length-committed-query-target prefix
        prefix-state)) /
    nnreal (card query-sample-space) ≤ query-error-bound
  using subset frac by blast
qed
also have ... ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) + query-error-bound
  by (intro add-mono order-refl
    checked-staged-query-prefix-clean-length-committed-target-prehit-bound
    [OF wf controlled i-bound])
finally show ?thesis .
qed

lemma checked-staged-query-prefix-length-committed-target-hit-bound-by-clean-and-collision:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-hit
      staged-query-prefix-length-committed-query-target) s ≤
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-hit
      staged-query-prefix-clean-length-committed-query-target) s +
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    checked-staged-query-prefix-hash-map-output-collision s
proof –
  let ?M = checked-staged-query-prefix-receive-with-state A i
  let ?hit =
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-length-committed-query-target

```

let $?clean =$
 checked-staged-query-prefix-dynamic-index-hit
 staged-query-prefix-clean-length-committed-query-target
let $?collision =$ *checked-staged-query-prefix-hash-map-output-collision*
have $wp\text{-}event\ ?M\ ?hit\ s \leq wp\text{-}event\ ?M\ (\lambda out. ?clean\ out \vee ?collision\ out)\ s$
 by (rule *wp-event-mono*)
 (rule
 checked-staged-query-prefix-length-committed-hit-imp-clean-hit-or-hash-collision)
also have $\dots \leq wp\text{-}event\ ?M\ ?clean\ s + wp\text{-}event\ ?M\ ?collision\ s$
 by (rule *wp-event-union-bound*)
finally show $?thesis$.
qed

lemma *checked-staged-query-prefix-length-committed-target-hit-bound-by-relation-query-error-and-collision:*

assumes *raw-bound: query-index-raw-preimage-bound*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i \leq rounds$*

shows

$wp\text{-}event\ (checked\text{-}staged\text{-}query\text{-}prefix\text{-}receive\text{-}with\text{-}state\ A\ i)$
 (*checked-staged-query-prefix-dynamic-index-hit*
 staged-query-prefix-length-committed-query-target)
 $adversary\text{-}initial\text{-}state \leq$
 $staged\text{-}phase\text{-}relation\text{-}error\ size$
 (*staged-query-search-queries budgets $i + 1$*) +
 $query\text{-}error\text{-}bound +$
 $wp\text{-}event\ (checked\text{-}staged\text{-}query\text{-}prefix\text{-}receive\text{-}with\text{-}state\ A\ i)$
 checked-staged-query-prefix-hash-map-output-collision
 adversary-initial-state

proof –

have *split:*

$wp\text{-}event\ (checked\text{-}staged\text{-}query\text{-}prefix\text{-}receive\text{-}with\text{-}state\ A\ i)$
 (*checked-staged-query-prefix-dynamic-index-hit*
 staged-query-prefix-length-committed-query-target)
 $adversary\text{-}initial\text{-}state \leq$
 $wp\text{-}event\ (checked\text{-}staged\text{-}query\text{-}prefix\text{-}receive\text{-}with\text{-}state\ A\ i)$
 (*checked-staged-query-prefix-dynamic-index-hit*
 staged-query-prefix-clean-length-committed-query-target)
 $adversary\text{-}initial\text{-}state +$
 $wp\text{-}event\ (checked\text{-}staged\text{-}query\text{-}prefix\text{-}receive\text{-}with\text{-}state\ A\ i)$
 checked-staged-query-prefix-hash-map-output-collision
 adversary-initial-state
by (rule
 checked-staged-query-prefix-length-committed-target-hit-bound-by-clean-and-collision)

have *clean-bound:*

$wp\text{-}event\ (checked\text{-}staged\text{-}query\text{-}prefix\text{-}receive\text{-}with\text{-}state\ A\ i)$
 (*checked-staged-query-prefix-dynamic-index-hit*
 staged-query-prefix-clean-length-committed-query-target)
 $adversary\text{-}initial\text{-}state \leq$

```

    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) + query-error-bound
  by (rule
    checked-staged-query-prefix-clean-length-committed-target-hit-bound
    [OF raw-bound wf controlled i-bound])
  have tail-bound:
    wp-event (checked-staged-query-prefix-receive-with-state  $A$   $i$ )
      (checked-staged-query-prefix-dynamic-index-hit
        staged-query-prefix-clean-length-committed-query-target)
      adversary-initial-state +
    wp-event (checked-staged-query-prefix-receive-with-state  $A$   $i$ )
      checked-staged-query-prefix-hash-map-output-collision
      adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
      query-error-bound +
    wp-event (checked-staged-query-prefix-receive-with-state  $A$   $i$ )
      checked-staged-query-prefix-hash-map-output-collision
      adversary-initial-state
  using clean-bound by (simp add: add-mono)
  show ?thesis
  by (rule order-trans[OF split tail-bound])
qed

```

lemma *checked-staged-query-prefix-length-committed-target-hit-bound:*

assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i \leq \text{rounds}$

shows

```

wp-event (checked-staged-query-prefix-receive-with-state  $A$   $i$ )
  (checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-length-committed-query-target)
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
  (staged-query-search-queries budgets  $i + 1$ ) +
  query-error-bound +
  hash-collision-budget-value 0
  (staged-query-search-queries budgets  $i + 1$ )

```

proof –

have hit-bound:

```

wp-event (checked-staged-query-prefix-receive-with-state  $A$   $i$ )
  (checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-length-committed-query-target)
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
  (staged-query-search-queries budgets  $i + 1$ ) +
  query-error-bound +
  wp-event (checked-staged-query-prefix-receive-with-state  $A$   $i$ )

```

```

    checked-staged-query-prefix-hash-map-output-collision
    adversary-initial-state
  by (rule
    checked-staged-query-prefix-length-committed-target-hit-bound-by-relation-query-error-and-collision
    [OF raw-bound wf controlled i-bound])
  have collision-bound:
    wp-event (checked-staged-query-prefix-receive-with-state A i)
    checked-staged-query-prefix-hash-map-output-collision
    adversary-initial-state ≤
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1)
  by (rule checked-staged-query-prefix-hash-map-output-collision-bound
    [OF wf controlled i-bound])
  have tail:
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    query-error-bound +
    wp-event (checked-staged-query-prefix-receive-with-state A i)
    checked-staged-query-prefix-hash-map-output-collision
    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    query-error-bound +
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1)
  using collision-bound by (intro add-mono order-refl)
  show ?thesis
  by (rule order-trans[OF hit-bound tail])
qed

end

end

```

```

theory Soundness-Staged-Transcript
  imports Soundness-Staged-Query
begin

```

Transcript-level staged query events and their transfer to verifier executions.

```

context soundness
begin

```

```

definition staged-transcript-query-partial-header-key-hit
  :: ('f staged-proof-data × 'f protocol-channel) option ⇒ bool
where
  staged-transcript-query-partial-header-key-hit out ⟷
    (case out of
      None ⇒ False

```

| *Some* (*data*, *attacker-state*) $\Rightarrow$
 (*let* *s* =
 verifier-state-from-adversary *attacker-state*
 (*staged-proof-transcript* *data*);
 B =
 query-header-supported-partial-union-good-sets *s*
 query-sampling-success-space
 (*staged-trace-root* *data*)
 (*staged-trace-fri-roots* *data*)
 (*staged-trace-final* *data*)
 (*staged-alphas* *data*)
 (*staged-degree* *data*)
 (*staged-composition-fri-roots* *data*)
 (*staged-composition-final* *data*)
 in
 ($\exists$ *i raw. i* < *rounds* $\wedge$
 fmlookup (*HashMap* *attacker-state*)
 (*QueryIndexChallenge* *i*
 (*state-after-query-chunks*
 (*staged-query-start-hash* *data*)
 (*staged-query-chunks* *data*) *i*)) =
 Some *raw* $\wedge$
 index (*to-nat* *raw*) $\in$ *B*)))

lemma *staged-transcript-query-partial-header-key-hit-alt:*

staged-transcript-query-partial-header-key-hit *out* $\longleftrightarrow$

(*case out of*

None $\Rightarrow$ *False*

| *Some* (*data*, *attacker-state*) $\Rightarrow$

($\exists$ *i raw. i* < *rounds* $\wedge$

fmlookup (*HashMap* *attacker-state*)

(*QueryIndexChallenge* *i*

(*state-after-query-chunks*

(*staged-query-start-hash* *data*)

(*staged-query-chunks* *data*) *i*)) =

Some *raw* $\wedge$

index (*to-nat* *raw*) $\in$

staged-partial-query-target *data* *attacker-state*))

unfolding *staged-transcript-query-partial-header-key-hit-def*

staged-partial-query-target-def *Let-def*

by (*simp split: option.splits prod.splits*)

definition *staged-transcript-query-partial-header-key-hit-at*

$:: \text{nat} \Rightarrow ('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow$
bool

where

staged-transcript-query-partial-header-key-hit-at *i* *out* $\longleftrightarrow$

(*case out of*

None $\Rightarrow$ *False*

$\mid \text{Some } (data, \text{attacker-state}) \Rightarrow$
 $(\exists \text{raw}. i < \text{rounds} \wedge$
 $\text{fmlookup } (\text{HashMap } \text{attacker-state})$
 $(\text{QueryIndexChallenge } i$
 $(\text{state-after-query-chunks}$
 $(\text{staged-query-start-hash } data)$
 $(\text{staged-query-chunks } data) \ i)) =$
 $\text{Some } \text{raw} \wedge$
 $\text{index } (\text{to-nat } \text{raw}) \in$
 $\text{staged-partial-query-target } data \ \text{attacker-state}))$

lemma *checked-staged-transcript-program-query-prefix-receive-lookup-support:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *outcome:*

$\text{Some } (data, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$

obtains *prefix prefix-state raw raw-state*

where

$\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $\text{adversary-initial-state})$

$\text{raw-state} \leq \text{attacker-state}$

$\text{fmlookup } (\text{HashMap } \text{raw-state})$
 $(\text{QueryIndexChallenge } i$
 $(\text{state-after-query-chunks}$
 $(\text{staged-query-start-hash } data) \ (\text{staged-query-chunks } data) \ i)) =$
 $\text{Some } \text{raw}$

proof (rule *checked-staged-transcript-program-query-prefix-receive-support*
[OF *wf controlled i-bound outcome*])

fix *prefix prefix-state raw raw-state*

assume *prefix-receive:*

$\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $\text{adversary-initial-state})$

and *raw-ext: raw-state ≤ attacker-state*

and *lookup:*

$\text{fmlookup } (\text{HashMap } \text{raw-state})$
 $(\text{QueryIndexChallenge } i$
 $(\text{state-after-query-chunks}$
 $(\text{staged-query-start-hash } data) \ (\text{staged-query-chunks } data) \ i)) =$
 $\text{Some } \text{raw}$

show *?thesis*

by (rule *that*[OF *prefix-receive raw-ext lookup*])

qed

lemma *checked-staged-security-with-data-state-accepted-shape-query-prefix-receive-index:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

Some (((data, attacker-state), result), final-state) ∈ set-dist

(execute (checked-staged-security-experiment-with-data-state A) adversary-initial-state)

and *shape:*

accepted-transcript-shape

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

(Some (result, final-state)) as query-idxs

and *i-bound: i < rounds*

obtains *prefix prefix-state raw raw-state*

where

Some (((prefix, prefix-state), raw), raw-state) ∈ set-dist

(execute (checked-staged-query-prefix-receive-with-state A i) adversary-initial-state)

index (to-nat raw) = query-idxs ! i

proof –

let *?s =*

verifier-state-from-adversary attacker-state

(staged-proof-transcript data)

from *checked-staged-security-experiment-with-data-state-outcomeE*

[OF support]

have *builder:*

Some (data, attacker-state) ∈ set-dist

(execute (checked-staged-transcript-program A) adversary-initial-state)

and *verifier:*

Some (result, final-state) ∈ set-dist (execute verify-monad ?s)

by *blast+*

from *checked-staged-security-with-data-state-accepted-shape-query-keys*

[OF wf controlled support shape]

obtain *raw-idxs* **where**

len-raw: length raw-idxs = rounds

and *query-idxs-eq:*

query-idxs = map (λraw. index (to-nat raw)) raw-idxs

and *lookup-final:*

∧j. j < rounds ⇒

fmlookup (HashMap final-state)

(QueryIndexChallenge j

(state-after-query-chunks

(staged-query-start-hash data)


```

      (staged-query-chunks data) j)) =
    Some (raw-idxs ! j)
  by blast
from checked-staged-transcript-program-query-prefix-receive-lookup-support
  [OF wf controlled i-bound builder]
obtain prefix prefix-state raw raw-state where
  prefix-receive:
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
  and raw-ext: raw-state ≤ attacker-state
  and lookup-raw:
    fmlookup (HashMap raw-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw
  by blast
have attacker-ext-s: attacker-state ≤ ?s
proof –
  have attacker-state ≤ attacker-state
    unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  then show ?thesis
    by (rule hash-extends-verifier-state-from-adversary-right)
qed
have s-ext-final: ?s ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
have raw-ext-final: raw-state ≤ final-state
  by (rule hash-ext-trans[OF raw-ext])
    (rule hash-ext-trans[OF attacker-ext-s s-ext-final])
have lookup-final-raw:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
  by (rule hash-extension-lookup[OF lookup-raw raw-ext-final])
have raw-eq: raw-idxs ! i = raw
  using lookup-final[OF i-bound] lookup-final-raw by simp
have idx-eq: index (to-nat raw) = query-idxs ! i
  using query-idxs-eq len-raw i-bound raw-eq by simp
show ?thesis
  by (rule that[OF prefix-receive idx-eq])
qed

lemma checked-staged-transcript-program-query-prefix-receive-match-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A

```

and *i-bound*: $i < \text{rounds}$
and *outcome*:
Some (*data*, *attacker-state*) $\in$
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
obtains *prefix prefix-state raw raw-state*
where
Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
sqp-trace-root prefix = staged-trace-root data
sqp-trace-fri-roots prefix = staged-trace-fri-roots data
sqp-trace-fri-challenges prefix =
staged-trace-fri-challenges data
sqp-trace-final prefix = staged-trace-final data
sqp-alphas prefix = staged-alphas data
sqp-degree prefix = staged-degree data
sqp-composition-fri-roots prefix =
staged-composition-fri-roots data
sqp-composition-fri-challenges prefix =
staged-composition-fri-challenges data
sqp-composition-final prefix = staged-composition-final data
sqp-query-chunks prefix = take i (staged-query-chunks data)
prefix-state $\leq$ attacker-state
raw-state $\leq$ attacker-state
fmlookup (HashMap raw-state)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data) (staged-query-chunks data) i)) =
Some raw
proof (*rule checked-staged-transcript-program-query-prefix-receive-support*
[OF wf controlled i-bound outcome])
fix *prefix prefix-state raw raw-state*
assume *prefix-receive*:
Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *root-eq*: *sqp-trace-root prefix = staged-trace-root data*
and *trace-roots-eq*:
sqp-trace-fri-roots prefix = staged-trace-fri-roots data
and *trace-bs-eq*:
sqp-trace-fri-challenges prefix =
staged-trace-fri-challenges data
and *trace-final-eq*: *sqp-trace-final prefix = staged-trace-final data*
and *alphas-eq*: *sqp-alphas prefix = staged-alphas data*
and *degree-eq*: *sqp-degree prefix = staged-degree data*

```

and comp-roots-eq:
  sqp-composition-fri-roots prefix =
    staged-composition-fri-roots data
and comp-bs-eq:
  sqp-composition-fri-challenges prefix =
    staged-composition-fri-challenges data
and comp-final-eq:
  sqp-composition-final prefix = staged-composition-final data
and query-chunks-eq:
  sqp-query-chunks prefix = take i (staged-query-chunks data)
and prefix-ext: prefix-state ≤ attacker-state
and raw-ext: raw-state ≤ attacker-state
and lookup:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
show ?thesis
  by (rule that[OF prefix-receive root-eq trace-roots-eq trace-bs-eq
    trace-final-eq alphas-eq degree-eq comp-roots-eq comp-bs-eq
    comp-final-eq query-chunks-eq prefix-ext raw-ext lookup])
qed

lemma checked-staged-transcript-query-hit-at-prefix-receive-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and hit:
  staged-transcript-query-partial-header-key-hit-at i
    (Some (data, attacker-state))
shows
  ∃ prefix prefix-state raw raw-state.
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state) ∧
        checked-staged-query-prefix-fixed-transcript-target-hit
          data attacker-state
        (Some (((prefix, prefix-state), raw), raw-state))
proof –
from hit obtain raw-hit where
  i-bound: i < rounds
and lookup-attacker:
  fmlookup (HashMap attacker-state)

```

```

    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw-hit
  and raw-hit-target:
    index (to-nat raw-hit) ∈ staged-partial-query-target data attacker-state
  unfolding staged-transcript-query-partial-header-key-hit-at-def
  by auto
from checked-staged-transcript-program-query-prefix-receive-lookup-support
  [OF wf controlled i-bound outcome]
obtain prefix prefix-state raw raw-state where
  prefix-receive:
    Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
  and raw-ext: raw-state ≤ attacker-state
  and lookup-raw:
    fmllookup (HashMap raw-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
  by (rule checked-staged-transcript-program-query-prefix-receive-lookup-support
    [OF wf controlled i-bound outcome])
have lookup-attacker-raw:
  fmllookup (HashMap attacker-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
  by (rule hash-extension-lookup[OF lookup-raw raw-ext])
have raw-eq: raw = raw-hit
  using lookup-attacker lookup-attacker-raw by simp
have fixed-hit:
  checked-staged-query-prefix-fixed-transcript-target-hit
    data attacker-state
    (Some (((prefix, prefix-state), raw), raw-state))
  using raw-hit-target unfolding raw-eq
    checked-staged-query-prefix-fixed-transcript-target-hit-def
    checked-staged-query-prefix-dynamic-index-hit-def
  by simp
show ?thesis
  using prefix-receive fixed-hit by blast
qed

```

lemma *checked-staged-transcript-query-hit-at-prefix-completion-receive-support:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*

and *outcome*:
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *hit*:
staged-transcript-query-partial-header-key-hit-at i
(Some (data, attacker-state))
shows
 $\exists$ *prefix prefix-state raw raw-state.*
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state) ∧
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-partial-query-target i)
(Some (((prefix, prefix-state), raw), raw-state))
proof –
from *hit* **obtain** *raw-hit* **where**
i-bound: i < rounds
and *lookup-attacker*:
fmlookup (HashMap attacker-state)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data) (staged-query-chunks data) i)) =
Some raw-hit
and *raw-hit-target*:
index (to-nat raw-hit) ∈ staged-partial-query-target data attacker-state
unfolding *staged-transcript-query-partial-header-key-hit-at-def*
by *auto*
from *checked-staged-transcript-program-query-prefix-receive-match-support*
[OF wf controlled i-bound outcome]
obtain *prefix prefix-state raw raw-state* **where**
prefix-receive:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *root-eq: sqp-trace-root prefix = staged-trace-root data*
and *trace-roots-eq*:
sqp-trace-fri-roots prefix = staged-trace-fri-roots data
and *trace-bs-eq*:
sqp-trace-fri-challenges prefix =
staged-trace-fri-challenges data
and *trace-final-eq*:
sqp-trace-final prefix = staged-trace-final data
and *alphas-eq: sqp-alphas prefix = staged-alphas data*
and *degree-eq: sqp-degree prefix = staged-degree data*
and *comp-roots-eq*:

```

    sqp-composition-fri-roots prefix =
      staged-composition-fri-roots data
  and comp-bs-eq:
    sqp-composition-fri-challenges prefix =
      staged-composition-fri-challenges data
  and comp-final-eq:
    sqp-composition-final prefix = staged-composition-final data
  and query-chunks-eq:
    sqp-query-chunks prefix = take i (staged-query-chunks data)
  and prefix-ext: prefix-state ≤ attacker-state
  and raw-ext: raw-state ≤ attacker-state
  and lookup-raw:
    fmlookup (HashMap raw-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw
  by (rule checked-staged-transcript-program-query-prefix-receive-match-support
    [OF wf controlled i-bound outcome])
  have lookup-attacker-raw:
    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw
  by (rule hash-extension-lookup[OF lookup-raw raw-ext])
  have raw-eq: raw = raw-hit
  using lookup-attacker lookup-attacker-raw by simp
  have match:
    staged-query-prefix-matches-data i prefix data
  unfolding staged-query-prefix-matches-data-def
  using root-eq trace-roots-eq trace-bs-eq trace-final-eq alphas-eq
    degree-eq comp-roots-eq comp-bs-eq comp-final-eq query-chunks-eq
  by simp
  have prefix-target:
    index (to-nat raw) ∈
      staged-query-prefix-partial-query-target i prefix prefix-state
  unfolding raw-eq
  using raw-hit-target
    staged-partial-query-target-subset-prefix-completion[OF match prefix-ext]
  by blast
  have prefix-hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-partial-query-target i)
      (Some (((prefix, prefix-state), raw), raw-state))
  using prefix-target
  unfolding checked-staged-query-prefix-dynamic-index-hit-def
  by simp
  show ?thesis

```

using *prefix-receive prefix-hit* by *blast*
 qed

lemma *checked-staged-after-query-prefix-receive-partial-hit-imp-prefix-hit*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < \text{rounds}$
and *head-support*:
 Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$
 set-dist
 (*execute* (*checked-staged-query-prefix-receive-with-state A i*)
 adversary-initial-state)
and *cont-support*:
 Some (*data*, *attacker-state*) $\in$
 set-dist
 (*execute*
 (*checked-staged-after-query-prefix-receive A i*
 ((*prefix*, *prefix-state*), *raw*))
 raw-state)
and *hit*:
 staged-transcript-query-partial-header-key-hit-at i
 (*Some* (*data*, *attacker-state*))
shows
 checked-staged-query-prefix-dynamic-index-hit
 (*staged-query-prefix-partial-query-target i*)
 (*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*))
proof –
have *prefix-support*:
 Some ((*prefix*, *prefix-state*), *prefix-state*) $\in$
 set-dist
 (*execute* (*checked-staged-query-prefix-with-state A i*)
 adversary-initial-state)
by (*rule checked-staged-query-prefix-receive-with-state-prefix-support*
 [*OF head-support*])
have *recv*:
 Some (*raw*, *raw-state*) $\in$
 set-dist (*execute receive-query-index-challenge prefix-state*)
by (*rule checked-staged-query-prefix-receive-with-state-receive-support*
 [*OF head-support*])
from *checked-staged-query-prefix-with-state-alignment*
 [*OF wf controlled i-bound prefix-support*]
have *prefix-counter*: *PQueryCounter prefix-state = i*
and *prefix-state*:
 PState prefix-state =
 state-after-query-chunks
 (*staged-query-prefix-start-hash prefix*)
 (*sqp-query-chunks prefix*) *i*
and *prefix-len*: *length (sqp-query-chunks prefix) = i*
by *blast+*

from *cont-support* **obtain** *chunk chunk-state record-state suffix-chunks* **where**
chunk-out:
Some (chunk, chunk-state) ∈
set-dist (execute (query-opening-stage A i raw) raw-state)
and *record-out:*
Some ((), record-state) ∈
set-dist (execute (record-staged-messages chunk) chunk-state)
and *suffix-out:*
Some (suffix-chunks, attacker-state) ∈
set-dist
(execute
(checked-staged-query-program A
(sqp-trace-fri-roots prefix)
(sqp-composition-fri-roots prefix)
(Suc i) (rounds − Suc i))
record-state)
and *data-eq:*
data =
(|staged-trace-root = sqp-trace-root prefix,
staged-trace-fri-roots = sqp-trace-fri-roots prefix,
staged-trace-fri-challenges = sqp-trace-fri-challenges prefix,
staged-trace-final = sqp-trace-final prefix,
staged-alphas = sqp-alphas prefix,
staged-degree = sqp-degree prefix,
staged-composition-fri-roots = sqp-composition-fri-roots prefix,
staged-composition-fri-challenges =
sqp-composition-fri-challenges prefix,
staged-composition-final = sqp-composition-final prefix,
staged-query-chunks =
sqp-query-chunks prefix @ chunk # suffix-chunks)
unfolding *checked-staged-after-query-prefix-receive-def Let-def*
by *(auto elim!: set-dist-bindE simp: assert-def throw-no-outcome*
split: if-splits)
have *match: staged-query-prefix-matches-data i prefix data*
unfolding *staged-query-prefix-matches-data-def data-eq*
using *prefix-len* **by** *simp*
have *start-hash:*
staged-query-start-hash data = staged-query-prefix-start-hash prefix
by *(rule staged-query-start-hash-of-prefix-completion[OF match])*
have *prefix-key-state:*
PState prefix-state =
state-after-query-chunks
(staged-query-start-hash data) (staged-query-chunks data) i
using *prefix-state prefix-len start-hash*
unfolding *data-eq*
by *(simp add: state-after-query-chunks-append-prefix)*
have *receive-lookup:*
fmlookup (HashMap raw-state)
(QueryIndexChallenge i


```

    (state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
using receive-query-index-challenge-outcome[OF recv]
    prefix-counter prefix-key-state
by simp
have prefix-raw-ext: prefix-state  $\leq$  raw-state
  by (rule receive-query-index-challenge-extends[OF recv])
have stage-controlled:
  controlled-ro-program (query-opening-budgets budgets ! i)
    (query-opening-stage A i raw)
using controlled wf i-bound
unfolding staged-adversary-controlled-def staged-budget-wellformed-def
by simp
have raw-chunk-ext: raw-state  $\leq$  chunk-state
  using controlled-ro-program-extension[OF stage-controlled] chunk-out
unfolding hash-extension-preserving-def
by blast
have record-state-eq:
  record-state = chunk-state|
    PState := foldl concat (PState chunk-state) chunk,
    PTranscript := PTranscript chunk-state @ chunk|
  by (rule record-staged-messages-outcome[OF record-out])
have chunk-record-ext: chunk-state  $\leq$  record-state
  unfolding record-state-eq less-eq-hash-ext-def less-eq-fmap-def
by simp
have suffix-bound:
  Suc i + (rounds - Suc i)  $\leq$  length (query-opening-budgets budgets)
  using wf i-bound unfolding staged-budget-wellformed-def by simp
have suffix-target:
  hash-target-program {}
    (sum-list
      (take (rounds - Suc i)
        (drop (Suc i) (query-opening-budgets budgets))) +
      (rounds - Suc i))
    (checked-staged-query-program A
      (sqp-trace-fri-roots prefix)
      (sqp-composition-fri-roots prefix)
      (Suc i) (rounds - Suc i))
  by (rule hash-target-program-checked-staged-query-program
    [OF controlled suffix-bound])
have suffix-ext: record-state  $\leq$  attacker-state
  using hash-target-program-extension[OF suffix-target] suffix-out
unfolding hash-extension-preserving-def
by blast
have raw-ext: raw-state  $\leq$  attacker-state
  by (rule hash-ext-trans[OF raw-chunk-ext]
    (rule hash-ext-trans[OF chunk-record-ext suffix-ext]))
have prefix-ext: prefix-state  $\leq$  attacker-state

```

```

    by (rule hash-ext-trans[OF prefix-raw-ext raw-ext])
  from hit obtain raw-hit where
    lookup-attacker:
      fmlookup (HashMap attacker-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data) (staged-query-chunks data) i)) =
        Some raw-hit
  and raw-hit-target:
    index (to-nat raw-hit) ∈
      staged-partial-query-target data attacker-state
  unfolding staged-transcript-query-partial-header-key-hit-at-def
  by auto
  have lookup-attacker-raw:
    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw
  by (rule hash-extension-lookup[OF receive-lookup raw-ext])
  have raw-eq: raw = raw-hit
  using lookup-attacker lookup-attacker-raw by simp
  have prefix-target:
    index (to-nat raw) ∈
      staged-query-prefix-partial-query-target i prefix prefix-state
  unfolding raw-eq
  using raw-hit-target
    staged-partial-query-target-subset-prefix-completion[OF match prefix-ext]
  by blast
  show ?thesis
  using prefix-target
  unfolding checked-staged-query-prefix-dynamic-index-hit-def
  by simp
qed

```

```

lemma checked-staged-transcript-program-query-prefix-receive-compact-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  obtains prefix prefix-state raw raw-state
  where
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)

```

```

    adversary-initial-state)
  sqp-trace-root prefix = staged-trace-root data
  sqp-alphas prefix = staged-alphas data
  sqp-composition-fri-roots prefix =
    staged-composition-fri-roots data
  prefix-state ≤ attacker-state
  raw-state ≤ attacker-state
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
proof (rule checked-staged-transcript-program-query-prefix-receive-match-support
  [OF wf controlled i-bound outcome])
fix prefix prefix-state raw raw-state
assume prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
and root-eq: sqp-trace-root prefix = staged-trace-root data
and alphas-eq: sqp-alphas prefix = staged-alphas data
and comp-roots-eq:
  sqp-composition-fri-roots prefix =
    staged-composition-fri-roots data
and prefix-ext: prefix-state ≤ attacker-state
and raw-ext: raw-state ≤ attacker-state
and lookup:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
show ?thesis
by (rule that[OF prefix-receive root-eq alphas-eq comp-roots-eq
  prefix-ext raw-ext lookup])
qed

```

```

lemma checked-staged-after-query-prefix-receive-extends:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and data-out:
  Some (data, attacker-state) ∈
    set-dist
    (execute
      (checked-staged-after-query-prefix-receive A i
        ((prefix, prefix-state), raw))
      raw-state)

```

shows $\text{raw-state} \leq \text{attacker-state}$

proof –

from data-out **obtain** chunk chunk-state record-state suffix-chunks **where**

chunk-out :

Some $(\text{chunk}, \text{chunk-state}) \in$
 $\text{set-dist } (\text{execute } (\text{query-opening-stage } A \ i \ \text{raw}) \ \text{raw-state})$

and record-out :

Some $((), \text{record-state}) \in$
 $\text{set-dist } (\text{execute } (\text{record-staged-messages } \text{chunk}) \ \text{chunk-state})$

and suffix-out :

Some $(\text{suffix-chunks}, \text{attacker-state}) \in$
 set-dist
 $(\text{execute}$
 $(\text{checked-staged-query-program } A$
 $(\text{sqp-trace-fri-roots } \text{prefix})$
 $(\text{sqp-composition-fri-roots } \text{prefix})$
 $(\text{Suc } i) \ (\text{rounds} - \text{Suc } i))$
 $\text{record-state})$

unfolding $\text{checked-staged-after-query-prefix-receive-def}$ Let-def

by $(\text{auto elim!} : \text{set-dist-bindE simp: assert-def throw-no-outcome}$
 $\text{split: if-splits})$

have stage-controlled :

$\text{controlled-ro-program } (\text{query-opening-budgets } \text{budgets } ! \ i)$
 $(\text{query-opening-stage } A \ i \ \text{raw})$

using $\text{controlled wf i-bound}$

unfolding $\text{staged-adversary-controlled-def}$ $\text{staged-budget-wellformed-def}$

by simp

have $\text{raw-chunk-ext: raw-state} \leq \text{chunk-state}$

using $\text{controlled-ro-program-extension[OF stage-controlled]}$ chunk-out

unfolding $\text{hash-extension-preserving-def}$

by blast

have record-state-eq :

$\text{record-state} = \text{chunk-state} \langle$
 $\text{PState} := \text{foldl concat } (\text{PState } \text{chunk-state}) \ \text{chunk},$
 $\text{PTranscript} := \text{PTranscript } \text{chunk-state} \ @ \ \text{chunk} \rangle$

by $(\text{rule record-staged-messages-outcome[OF record-out]})$

have $\text{chunk-record-ext: chunk-state} \leq \text{record-state}$

unfolding $\text{record-state-eq less-eq-hash-ext-def less-eq-fmap-def}$

by simp

have suffix-bound :

$\text{Suc } i + (\text{rounds} - \text{Suc } i) \leq \text{length } (\text{query-opening-budgets } \text{budgets})$

using wf i-bound **unfolding** $\text{staged-budget-wellformed-def}$ **by** simp

have suffix-target :

$\text{hash-target-program } \{\}$
 $(\text{sum-list}$
 $(\text{take } (\text{rounds} - \text{Suc } i)$
 $(\text{drop } (\text{Suc } i) \ (\text{query-opening-budgets } \text{budgets}))) +$
 $(\text{rounds} - \text{Suc } i))$
 $(\text{checked-staged-query-program } A$

```

    (sqp-trace-fri-roots prefix)
    (sqp-composition-fri-roots prefix)
    (Suc i) (rounds - Suc i)
  by (rule hash-target-program-checked-staged-query-program
      [OF controlled suffix-bound])
  have suffix-ext: record-state ≤ attacker-state
    using hash-target-program-extension[OF suffix-target] suffix-out
    unfolding hash-extension-preserving-def
    by blast
  show ?thesis
  by (rule hash-ext-trans[OF raw-chunk-ext])
    (rule hash-ext-trans[OF chunk-record-ext suffix-ext])
qed

lemma checked-staged-after-query-prefix-receive-with-verifier-query-bad-at-imp-length-prefix-hit-or-tree-output:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and head-support:
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
            adversary-initial-state)
  and cont-support:
    out ∈
      set-dist
        (execute
          (checked-staged-after-query-prefix-receive-with-verifier A i
            ((prefix, prefix-state), raw))
          raw-state)
  and hit:
    staged-security-with-data-state-query-bad-hit-at i out
  shows
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-length-committed-query-target
    (Some (((prefix, prefix-state), raw), raw-state)) ∨
    staged-security-with-data-state-query-tree-output-hit A out
  proof (cases out)
  case None
  then show ?thesis
    using hit unfolding staged-security-with-data-state-query-bad-hit-at-def
    by simp
  next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state

```

```

    (staged-proof-transcript data)
have data-out':
  Some (data, attacker-state) ∈
    set-dist
      (execute
        (checked-staged-after-query-prefix-receive A i
          ((prefix, prefix-state), raw))
        raw-state)
  by (rule
    checked-staged-after-query-prefix-receive-with-verifier-outcome(1)
    [OF cont-support[unfolded out-eq]])
have verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by (rule
    checked-staged-after-query-prefix-receive-with-verifier-outcome(2)
    [OF cont-support[unfolded out-eq]])
from hit[unfolded out-eq
  staged-security-with-data-state-query-bad-hit-at-def Let-def]
have hit-at:
  query-index-round-set-hit-at ?s query-sampling-success-space i
    (Some (result, final-state))
  by simp
from hit-at obtain trace-table composition-table as query-idxs where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
and query-hit:
  query-idxs ! i ∈
    query-sampling-success-space trace-table composition-table as
unfolding query-index-round-set-hit-at-def by blast
have transcript-decomp:
  checked-staged-transcript-program A =
    checked-staged-query-prefix-receive-with-state A i  $\gg$ 
    checked-staged-after-query-prefix-receive A i
  by (rule checked-staged-transcript-program-query-prefix-receive-decomp
    [OF i-bound])
have builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  unfolding transcript-decomp
  by (rule-tac x=((prefix, prefix-state), raw) and t=raw-state
    in set-dist-bindI)
    (rule head-support, rule data-out')
have security-decomp:
  checked-staged-security-experiment-with-data-state A =
    checked-staged-query-prefix-receive-with-state A i  $\gg$ 
    checked-staged-after-query-prefix-receive-with-verifier A i

```

```

by (rule
  checked-staged-security-experiment-with-data-state-query-prefix-receive-decomp
  [OF i-bound])
have support-some:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  unfolding security-decomp out-eq[symmetric]
  by (rule-tac x=((prefix, prefix-state), raw) and t=raw-state
    in set-dist-bindI)
  (rule head-support, rule cont-support)
have shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  as query-idxs
  using bound
  unfolding accepted-with-bound-tables-def accepted-with-tables-def
  by simp
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support-some shape]
obtain final-raw-idxs where
  as-eq: as = staged-alphas data
  and len-final-raw: length final-raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) final-raw-idxs
  and lookup-final:
    ∧j. j < rounds ⇒
      fmllookup (HashMap final-state)
        (QueryIndexChallenge j
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) j)) =
        Some (final-raw-idxs ! j)
  by blast
have prefix-support:
  Some ((prefix, prefix-state), prefix-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-with-state A i)
        adversary-initial-state)
  by (rule checked-staged-query-prefix-receive-with-state-prefix-support
    [OF head-support])
have recv:
  Some (raw, raw-state) ∈
    set-dist (execute receive-query-index-challenge prefix-state)
  by (rule checked-staged-query-prefix-receive-with-state-receive-support
    [OF head-support])
from checked-staged-query-prefix-with-state-alignment
  [OF wf controlled i-bound prefix-support]
have prefix-counter: PQueryCounter prefix-state = i

```

```

and prefix-state-hash:
  PState prefix-state =
    state-after-query-chunks
      (staged-query-prefix-start-hash prefix)
      (sqp-query-chunks prefix) i
and prefix-len: length (sqp-query-chunks prefix) = i
by blast+
from data-out' obtain chunk chunk-state record-state suffix-chunks where
  chunk-out:
    Some (chunk, chunk-state) ∈
      set-dist (execute (query-opening-stage A i raw) raw-state)
and record-out:
    Some ((), record-state) ∈
      set-dist (execute (record-staged-messages chunk) chunk-state)
and suffix-out:
    Some (suffix-chunks, attacker-state) ∈
      set-dist
        (execute
          (checked-staged-query-program A
            (sqp-trace-fri-roots prefix)
            (sqp-composition-fri-roots prefix)
            (Suc i) (rounds - Suc i))
          record-state)
and data-eq:
  data =
    (|staged-trace-root = sqp-trace-root prefix,
      staged-trace-fri-roots = sqp-trace-fri-roots prefix,
      staged-trace-fri-challenges = sqp-trace-fri-challenges prefix,
      staged-trace-final = sqp-trace-final prefix,
      staged-alphas = sqp-alphas prefix,
      staged-degree = sqp-degree prefix,
      staged-composition-fri-roots = sqp-composition-fri-roots prefix,
      staged-composition-fri-challenges =
        sqp-composition-fri-challenges prefix,
      staged-composition-final = sqp-composition-final prefix,
      staged-query-chunks =
        sqp-query-chunks prefix @ chunk # suffix-chunks)
  unfolding checked-staged-after-query-prefix-receive-def Let-def
  by (auto elim!: set-dist-bindE simp: assert-def throw-no-outcome
    split: if-splits)
have match: staged-query-prefix-matches-data i prefix data
  unfolding staged-query-prefix-matches-data-def data-eq
  using prefix-len by simp
have start-hash:
  staged-query-start-hash data = staged-query-prefix-start-hash prefix
  by (rule staged-query-start-hash-of-prefix-completion[OF match])
have prefix-key-state:
  PState prefix-state =
    state-after-query-chunks

```



```

    (staged-query-start-hash data) (staged-query-chunks data) i
using prefix-state-hash prefix-len start-hash
unfolding data-eq
by (simp add: state-after-query-chunks-append-prefix)
have receive-lookup:
  fmllookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
using receive-query-index-challenge-outcome[OF recv]
  prefix-counter prefix-key-state
by simp
have raw-ext-attacker: raw-state ≤ attacker-state
by (rule checked-staged-after-query-prefix-receive-extends
  [OF wf controlled i-bound data-out])
have attacker-ext-s: attacker-state ≤ ?s
proof -
  have attacker-state ≤ attacker-state
    unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  then show ?thesis
    by (rule hash-extends-verifier-state-from-adversary-right)
qed
have s-ext-final: ?s ≤ final-state
by (rule verify-monad-hash-extends[OF verifier])
have raw-ext-final: raw-state ≤ final-state
by (rule hash-ext-trans[OF raw-ext-attacker])
  (rule hash-ext-trans[OF attacker-ext-s s-ext-final])
have lookup-final-raw:
  fmllookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
by (rule hash-extension-lookup[OF receive-lookup raw-ext-final])
have raw-eq: final-raw-idxs ! i = raw
using lookup-final[OF i-bound] lookup-final-raw by simp
have raw-hit-actual:
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table
    (staged-alphas data)
proof -
  have index (to-nat (final-raw-idxs ! i)) = query-idxs ! i
    using query-idxs-eq len-final-raw i-bound by simp
  then show ?thesis
    using query-hit as-eq raw-eq by simp
qed
have prefix-ext-raw: prefix-state ≤ raw-state
by (rule receive-query-index-challenge-extends[OF recv])

```

```

have prefix-ext-final: prefix-state ≤ final-state
  by (rule hash-ext-trans[OF prefix-ext-raw raw-ext-final])
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
from accepted-with-bound-tables-initial-roots-pullback-or-new-tree-output-hit
  [OF prefix-ext-final bound]
obtain fr f-fri-roots f-final dg composition-fri-roots final rest
  trace-tree composition-tree where
  header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and comp-nonempty: composition-fri-roots ≠ []
and trace-created:
  created-tree trace-table trace-tree final-state
and trace-root: fr = value trace-tree
and comp-created:
  created-tree composition-table composition-tree final-state
and comp-root:
  hd composition-fri-roots = value composition-tree
and split:
  (merkle-root-binds-table fr trace-table prefix-state ∧
    merkle-root-binds-table (hd composition-fri-roots)
    composition-table prefix-state) ∨
  hash-map-new-output-hit
  (set-tree trace-tree ∪ set-tree composition-tree)
  prefix-state final-state
by blast
have eqs:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
have trace-len: length trace-table = clength * scale

```

```

using bound unfolding accepted-with-bound-tables-def
  accepted-with-tables-def by simp
have comp-len: length composition-table = clength * scale
using bound unfolding accepted-with-bound-tables-def
  accepted-with-tables-def by simp
from split show ?thesis
proof
  assume binds:
    merkle-root-binds-table fr trace-table prefix-state  $\wedge$ 
    merkle-root-binds-table (hd composition-fri-roots)
      composition-table prefix-state
  have candidate:
    (trace-table, composition-table)  $\in$ 
      staged-query-prefix-length-committed-table-candidates prefix
        prefix-state
  unfolding staged-query-prefix-length-committed-table-candidates-def
  using binds comp-nonempty eqs data-eq trace-len comp-len by simp
  have target:
    index (to-nat raw)  $\in$ 
      staged-query-prefix-length-committed-query-target prefix
        prefix-state
  proof —
    have raw-hit-prefix:
      index (to-nat raw)  $\in$ 
        query-sampling-success-space trace-table composition-table
          (sqp-alphas prefix)
    using raw-hit-actual data-eq by simp
    show ?thesis
      by (rule staged-query-prefix-length-committed-query-targetI
        [OF candidate raw-hit-prefix])
  qed
  then show ?thesis
    unfolding checked-staged-query-prefix-dynamic-index-hit-def
    by simp
next
  assume tree-hit:
    hash-map-new-output-hit
      (set-tree trace-tree  $\cup$  set-tree composition-tree)
      prefix-state final-state
  have comp-nonempty-prefix:
    sqp-composition-fri-roots prefix  $\neq []$ 
  using comp-nonempty eqs data-eq by simp
  have trace-root-prefix:
    sqp-trace-root prefix = value trace-tree
  using trace-root eqs data-eq by simp
  have comp-root-prefix:
    hd (sqp-composition-fri-roots prefix) = value composition-tree
  using comp-root eqs data-eq by simp
  have tree-event-body:

```

```

∃ i prefix prefix-state raw raw-state trace-table
  composition-table as query-idxs trace-tree composition-tree.
i < rounds ∧
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state) ∧
  Some (result, final-state) ∈
    set-dist (execute verify-monad ?s) ∧
  accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs ∧
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some raw ∧
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table
      (staged-alphas data) ∧
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute
        (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state) ∧
  sqp-composition-fri-roots prefix ≠ [] ∧
  created-tree trace-table trace-tree final-state ∧
  sqp-trace-root prefix = value trace-tree ∧
  created-tree composition-table composition-tree final-state ∧
  hd (sqp-composition-fri-roots prefix) =
    value composition-tree ∧
  hash-map-new-output-hit
    (set-tree trace-tree ∪ set-tree composition-tree)
    prefix-state final-state
  apply (rule-tac x=i in exI)
  apply (rule-tac x=prefix in exI)
  apply (rule-tac x=prefix-state in exI)
  apply (rule-tac x=raw in exI)
  apply (rule-tac x=raw-state in exI)
  apply (rule-tac x=trace-table in exI)
  apply (rule-tac x=composition-table in exI)
  apply (rule-tac x=as in exI)
  apply (rule-tac x=query-idxs in exI)
  apply (rule-tac x=trace-tree in exI)
  apply (rule-tac x=composition-tree in exI)
  using i-bound builder verifier bound lookup-final-raw raw-hit-actual
    head-support comp-nonempty-prefix trace-created trace-root-prefix
    comp-created comp-root-prefix tree-hit
  by blast

```

```

have tree-event:
  staged-security-with-data-state-query-tree-output-hit A out
  using tree-event-body
  unfolding out-eq
    staged-security-with-data-state-query-tree-output-hit-def Let-def
  by simp
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-data-state-query-bad-hit-at-bound-by-length-prefix-and-tree:
  assumes raw-bound: query-index-raw-preimage-bound
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-bad-hit-at i)
      adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
      query-error-bound +
      hash-collision-budget-value 0
      (staged-query-search-queries budgets  $i + 1$ ) +
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-with-data-state-query-tree-output-hit A)
        adversary-initial-state
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?Head = checked-staged-query-prefix-receive-with-state A i
  let ?Cont =
    checked-staged-after-query-prefix-receive-with-verifier A i
  let ?Prefix =
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-length-committed-query-target
  let ?Tree = staged-security-with-data-state-query-tree-output-hit A
  let ?Bad = staged-security-with-data-state-query-bad-hit-at i
  let ?BadNoTree =  $\lambda \text{out}. ?\text{Bad out} \wedge \neg ?\text{Tree out}$ 
  have split:
    wp-event ?M ?Bad adversary-initial-state  $\leq$ 
    wp-event ?M ?BadNoTree adversary-initial-state +
    wp-event ?M ?Tree adversary-initial-state
  proof –
    have wp-event ?M ?Bad adversary-initial-state  $\leq$ 
      wp-event ?M ( $\lambda \text{out}. ?\text{BadNoTree out} \vee ?\text{Tree out}$ )
      adversary-initial-state
      by (rule wp-event-mono) blast
    also have ...  $\leq$ 
      wp-event ?M ?BadNoTree adversary-initial-state +

```

```

    wp-event ?M ?Tree adversary-initial-state
  by (rule wp-event-union-bound)
  finally show ?thesis .
qed
have prefix-bound:
  wp-event ?Head ?Prefix adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    query-error-bound +
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1)
  by (rule checked-staged-query-prefix-length-committed-target-hit-bound
    [OF raw-bound wf controlled])
    (use i-bound in simp)
have bad-no-tree-bound:
  wp-event ?M ?BadNoTree adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    query-error-bound +
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1)
unfolding
  checked-staged-security-experiment-with-data-state-query-prefix-receive-decomp
  [OF i-bound]
proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
  show ?BadNoTree None  $\implies$  ?Prefix None
    unfolding staged-security-with-data-state-query-bad-hit-at-def
    checked-staged-query-prefix-dynamic-index-hit-def
    by simp
next
fix x t out
assume head:
  Some (x, t) ∈ set-dist (execute ?Head adversary-initial-state)
  and cont: out ∈ set-dist (execute (?Cont x) t)
  and bad-no-tree: ?BadNoTree out
show ?Prefix (Some (x, t))
proof (cases x)
  case x-eq: (Pair prefix-pack raw)
  then obtain prefix prefix-state where prefix-pack-eq:
    prefix-pack = (prefix, prefix-state)
    by (cases prefix-pack) simp
  have head':
    Some (((prefix, prefix-state), raw), t) ∈
      set-dist (execute ?Head adversary-initial-state)
    using head unfolding x-eq prefix-pack-eq .
  have cont':
    out ∈
      set-dist
        (execute

```

```

      (checked-staged-after-query-prefix-receive-with-verifier A i
        ((prefix, prefix-state), raw))
    t)
  using cont unfolding x-eq prefix-pack-eq .
from checked-staged-after-query-prefix-receive-with-verifier-query-bad-at-imp-length-prefix-hit-or-tree-output
  [OF wf controlled i-bound head' cont']
have ?Prefix (Some (((prefix, prefix-state), raw), t)) ∨
  ?Tree out
  using bad-no-tree by simp
then show ?thesis
  using bad-no-tree unfolding x-eq prefix-pack-eq by simp
qed
qed
have wp-event ?M ?Bad adversary-initial-state ≤
  (staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    query-error-bound +
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1)) +
  wp-event ?M ?Tree adversary-initial-state
by (rule order-trans[OF split])
  (intro add-mono bad-no-tree-bound order-refl)
then show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-data-state-query-bad-hit-at-no-tree-bound-by-length-prefix:
  assumes raw-bound: query-index-raw-preimage-bound
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (λout.
        staged-security-with-data-state-query-bad-hit-at i out ∧
        ¬ staged-security-with-data-state-query-tree-output-hit A out)
    adversary-initial-state ≤
    staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      query-error-bound +
      hash-collision-budget-value 0
      (staged-query-search-queries budgets i + 1)
proof -
  let ?Head = checked-staged-query-prefix-receive-with-state A i
  let ?Cont =
    checked-staged-after-query-prefix-receive-with-verifier A i
  let ?Prefix =
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-length-committed-query-target

```

```

let ?Tree = staged-security-with-data-state-query-tree-output-hit A
let ?Bad = staged-security-with-data-state-query-bad-hit-at i
let ?BadNoTree =  $\lambda out. ?Bad\ out \wedge \neg ?Tree\ out$ 
have prefix-bound:
  wp-event ?Head ?Prefix adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    query-error-bound +
    hash-collision-budget-value 0
    (staged-query-search-queries budgets  $i + 1$ )
by (rule checked-staged-query-prefix-length-committed-target-hit-bound
  [OF raw-bound wf controlled])
  (use i-bound in simp)
show ?thesis
unfolding
  checked-staged-security-experiment-with-data-state-query-prefix-receive-decomp
  [OF i-bound]
proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
show ?BadNoTree None  $\implies$  ?Prefix None
unfolding staged-security-with-data-state-query-bad-hit-at-def
  checked-staged-query-prefix-dynamic-index-hit-def
by simp
next
fix x t out
assume head:
  Some (x, t)  $\in$  set-dist (execute ?Head adversary-initial-state)
and cont: out  $\in$  set-dist (execute (?Cont x) t)
and bad-no-tree: ?BadNoTree out
show ?Prefix (Some (x, t))
proof (cases x)
case x-eq: (Pair prefix-pack raw)
then obtain prefix prefix-state where prefix-pack-eq:
  prefix-pack = (prefix, prefix-state)
by (cases prefix-pack) simp
have head':
  Some (((prefix, prefix-state), raw), t)  $\in$ 
    set-dist (execute ?Head adversary-initial-state)
using head unfolding x-eq prefix-pack-eq .
have cont':
  out  $\in$ 
    set-dist
    (execute
      (checked-staged-after-query-prefix-receive-with-verifier A i
        ((prefix, prefix-state), raw))
      t)
using cont unfolding x-eq prefix-pack-eq .
from checked-staged-after-query-prefix-receive-with-verifier-query-bad-at-imp-length-prefix-hit-or-tree-output
  [OF wf controlled i-bound head' cont']
have ?Prefix (Some (((prefix, prefix-state), raw), t))  $\vee$ 

```



```

      ?Tree out
    using bad-no-tree by simp
  then show ?thesis
    using bad-no-tree unfolding x-eq prefix-pack-eq by simp
qed
qed
qed

lemma checked-staged-security-with-data-state-query-bad-hit-bound-by-length-prefixes-and-tree:
  assumes raw-bound: query-index-raw-preimage-bound
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-bad-hit
      adversary-initial-state  $\leq$ 
      ( $\sum i < \text{rounds}.$ 
        staged-phase-relation-error size
        (staged-query-search-queries budgets  $i + 1$ ) +
        query-error-bound +
        hash-collision-budget-value 0
        (staged-query-search-queries budgets  $i + 1$ )) +
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-tree-output-hit A)
      adversary-initial-state
  proof -
    let ?M = checked-staged-security-experiment-with-data-state A
    let ?Tree = staged-security-with-data-state-query-tree-output-hit A
    let ?NoTree =
       $\lambda i.$  out.
      staged-security-with-data-state-query-bad-hit-at  $i$  out  $\wedge$ 
       $\neg$  ?Tree out
    let ?E =
       $\lambda i.$ 
      staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
      query-error-bound +
      hash-collision-budget-value 0
      (staged-query-search-queries budgets  $i + 1$ )
    have split:
      wp-event ?M staged-security-with-data-state-query-bad-hit
      adversary-initial-state  $\leq$ 
      wp-event ?M ( $\lambda \text{out}.$   $\exists i \in \{..<\text{rounds}\}.$  ?NoTree  $i$  out)
      adversary-initial-state +
      wp-event ?M ?Tree adversary-initial-state
    proof -
      have wp-event ?M staged-security-with-data-state-query-bad-hit
      adversary-initial-state  $\leq$ 
      wp-event ?M

```

```

    ( $\lambda out. (\exists i \in \{..<rounds\}. ?NoTree\ i\ out) \vee ?Tree\ out$ )
    adversary-initial-state
proof (rule wp-event-mono)
  fix out
  assume bad: staged-security-with-data-state-query-bad-hit out
  from staged-security-with-data-state-query-bad-hit-imp-round-hit[OF bad]
  obtain i where i-bound:  $i \in \{..<rounds\}$ 
    and hit: staged-security-with-data-state-query-bad-hit-at i out
    by blast
  show ( $\exists i \in \{..<rounds\}. ?NoTree\ i\ out$ )  $\vee ?Tree\ out$ 
  proof (cases ?Tree out)
    case True
    then show ?thesis by simp
  next
    case False
    then show ?thesis
      using i-bound hit by blast
  qed
qed
also have ...  $\leq$ 
  wp-event ?M ( $\lambda out. \exists i \in \{..<rounds\}. ?NoTree\ i\ out$ )
    adversary-initial-state +
  wp-event ?M ?Tree adversary-initial-state
  by (rule wp-event-union-bound)
finally show ?thesis .
qed
have no-tree-bound:
  wp-event ?M ( $\lambda out. \exists i \in \{..<rounds\}. ?NoTree\ i\ out$ )
    adversary-initial-state  $\leq (\sum i < rounds. ?E\ i)$ 
proof (rule wp-event-finite-union-bound)
  show finite  $\{..<rounds\}$ 
    by simp
next
  fix i
  assume i-bound:  $i \in \{..<rounds\}$ 
  show wp-event ?M ( $?NoTree\ i$ ) adversary-initial-state  $\leq ?E\ i$ 
    by (rule
      checked-staged-security-with-data-state-query-bad-hit-at-no-tree-bound-by-length-prefix
      [OF raw-bound wf controlled])
      (use i-bound in simp)
  qed
have wp-event ?M staged-security-with-data-state-query-bad-hit
  adversary-initial-state  $\leq$ 
  ( $\sum i < rounds. ?E\ i$ ) + wp-event ?M ?Tree adversary-initial-state
  by (rule order-trans[OF split])
  (intro add-mono no-tree-bound order-refl)
then show ?thesis .
qed

```

lemma *checked-staged-security-with-data-state-query-bad-hit-bound-by-length-prefixes-and-tree-from-budgets:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-query-bad-hit

adversary-initial-state $\leq$

$(\sum i < \text{rounds.}$

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) +

query-error-bound +

hash-collision-budget-value 0

(staged-query-search-queries budgets i + 1)) +

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-query-tree-output-hit A)

adversary-initial-state

proof –

have *raw-bound: query-index-raw-preimage-bound*

by *(rule query-index-raw-preimage-bound-from-sampler-wellformed)*

show *?thesis*

by *(rule*

checked-staged-security-with-data-state-query-bad-hit-bound-by-length-prefixes-and-tree

[OF raw-bound wf controlled])

qed

lemma *checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-and-tree:*

assumes *raw-bound: query-index-raw-preimage-bound*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event query-bad)

adversary-initial-state $\leq$

$(\sum i < \text{rounds.}$

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) +

query-error-bound +

hash-collision-budget-value 0

(staged-query-search-queries budgets i + 1)) +

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-query-tree-output-hit A)

adversary-initial-state

proof –

have *wp-event (checked-staged-security-experiment-with-data-state A)*

(staged-security-with-data-state-verifier-event query-bad)

adversary-initial-state $\leq$

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-query-bad-hit

adversary-initial-state

by (rule *checked-staged-security-with-data-state-query-bad-verifier-event-bound*)
simp
also have ... $\leq$
 $(\sum i < \text{rounds}.$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1)) +
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-query-tree-output-hit A)
adversary-initial-state
by (rule
checked-staged-security-with-data-state-query-bad-hit-bound-by-length-prefixes-and-tree
[OF raw-bound wf controlled])
finally show ?thesis .
qed

lemma *checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-and-tree-from-budget*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event query-bad)

adversary-initial-state $\leq$

$(\sum i < \text{rounds}.$

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) +

query-error-bound +

hash-collision-budget-value 0

(staged-query-search-queries budgets i + 1)) +

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-query-tree-output-hit A)

adversary-initial-state

proof –

have *raw-bound: query-index-raw-preimage-bound*

by (rule *query-index-raw-preimage-bound-from-sampler-wellformed*)

show ?thesis

by (rule

checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-and-tree

[OF raw-bound wf controlled])

qed

lemma *checked-staged-transcript-query-partial-header-key-hit-at-bound-from-prefix-receive:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *prefix-bound:*

wp-event (checked-staged-query-prefix-receive-with-state A i)

```

    (checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-partial-query-target i))
    adversary-initial-state ≤ C
  shows
    wp-event (checked-staged-transcript-program A)
      (staged-transcript-query-partial-header-key-hit-at i)
      adversary-initial-state ≤ C
  unfolding checked-staged-transcript-program-query-prefix-receive-decomp
    [OF i-bound]
  proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
  show
    staged-transcript-query-partial-header-key-hit-at i None ⇒
      checked-staged-query-prefix-dynamic-index-hit
        (staged-query-prefix-partial-query-target i) None
  unfolding staged-transcript-query-partial-header-key-hit-at-def
    checked-staged-query-prefix-dynamic-index-hit-def
  by simp
next
  fix x t out
  assume head:
    Some (x, t) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
  and cont:
    out ∈
      set-dist
        (execute (checked-staged-after-query-prefix-receive A i x) t)
  and hit: staged-transcript-query-partial-header-key-hit-at i out
  show
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-partial-query-target i) (Some (x, t))
  proof (cases x)
  case x-eq: (Pair prefix-pack raw)
  then obtain prefix prefix-state where prefix-pack-eq:
    prefix-pack = (prefix, prefix-state)
  by (cases prefix-pack) simp
  show ?thesis
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding staged-transcript-query-partial-header-key-hit-at-def
  by simp
  next
  case (Some packed)
  then obtain data attacker-state where out-eq:
    out = Some (data, attacker-state)
  by (cases packed) simp

```

```

have head':
  Some (((prefix, prefix-state), raw), t) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
using head unfolding x-eq prefix-pack-eq .
have cont':
  Some (data, attacker-state) ∈
    set-dist
      (execute
        (checked-staged-after-query-prefix-receive A i
          ((prefix, prefix-state), raw))
        t)
using cont unfolding x-eq prefix-pack-eq out-eq .
have hit':
  staged-transcript-query-partial-header-key-hit-at i
    (Some (data, attacker-state))
using hit unfolding out-eq .
have prefix-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-partial-query-target i)
    (Some (((prefix, prefix-state), raw), t))
by (rule
  checked-staged-after-query-prefix-receive-partial-hit-imp-prefix-hit
  [OF wf controlled i-bound head' cont' hit'])
show ?thesis
using prefix-hit unfolding x-eq prefix-pack-eq .
qed
qed
qed

lemma checked-staged-transcript-query-bound-table-hit-at-committed-prefix-receive-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and i-bound: i < rounds
and lookup-attacker:
  fmlookup (HashMap attacker-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw-hit
and raw-hit-target:
  index (to-nat raw-hit) ∈
    query-sampling-success-space trace-table composition-table

```

(staged-alphas data)
and *comp-nonempty*:
 staged-composition-fri-roots data $\neq \square$
shows
 $\exists$ prefix prefix-state raw raw-state.
 Some (((prefix, prefix-state), raw), raw-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state) $\wedge$
 (merkle-root-binds-table (sqp-trace-root prefix) trace-table
 prefix-state $\longrightarrow$
 merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
 composition-table prefix-state $\longrightarrow$
 checked-staged-query-prefix-dynamic-index-hit
 (λ prefix prefix-state.
 staged-query-prefix-committed-query-target prefix prefix-state)
 (Some (((prefix, prefix-state), raw), raw-state)))
proof –
from checked-staged-transcript-program-query-prefix-receive-compact-support
 [OF wf controlled i-bound outcome]
obtain prefix prefix-state raw raw-state **where**
 prefix-receive:
 Some (((prefix, prefix-state), raw), raw-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state)
and root-eq: sqp-trace-root prefix = staged-trace-root data
and alphas-eq: sqp-alphas prefix = staged-alphas data
and comp-roots-eq:
 sqp-composition-fri-roots prefix =
 staged-composition-fri-roots data
and prefix-ext: prefix-state $\leq$ attacker-state
and raw-ext: raw-state $\leq$ attacker-state
and lookup-raw:
 fmlookup (HashMap raw-state)
 (QueryIndexChallenge i
 (state-after-query-chunks
 (staged-query-start-hash data) (staged-query-chunks data) i)) =
 Some raw
by (rule checked-staged-transcript-program-query-prefix-receive-compact-support
 [OF wf controlled i-bound outcome])
have lookup-attacker-raw:
 fmlookup (HashMap attacker-state)
 (QueryIndexChallenge i
 (state-after-query-chunks
 (staged-query-start-hash data) (staged-query-chunks data) i)) =
 Some raw
by (rule hash-extension-lookup[OF lookup-raw raw-ext])
have raw-eq: raw = raw-hit

```

using lookup-attacker lookup-attacker-raw by simp
have comp-nonempty-prefix:
  sqp-composition-fri-roots prefix  $\neq []$ 
using comp-nonempty comp-roots-eq by simp
have hit-if-bound:
  merkle-root-binds-table (sqp-trace-root prefix) trace-table
    prefix-state  $\implies$ 
  merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
    composition-table prefix-state  $\implies$ 
  checked-staged-query-prefix-dynamic-index-hit
    ( $\lambda$ prefix prefix-state.
      staged-query-prefix-committed-query-target prefix prefix-state)
    (Some (((prefix, prefix-state), raw), raw-state))
proof –
assume trace-bind:
  merkle-root-binds-table (sqp-trace-root prefix) trace-table
    prefix-state
assume comp-bind:
  merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
    composition-table prefix-state
have candidate:
  (trace-table, composition-table)  $\in$ 
    query-header-committed-table-candidates prefix-state
      (sqp-trace-root prefix)
      (sqp-composition-fri-roots prefix)
unfolding query-header-committed-table-candidates-def
using trace-bind comp-nonempty-prefix comp-bind by simp
have raw-target-prefix:
  index (to-nat raw)  $\in$ 
    query-sampling-success-space trace-table composition-table
      (sqp-alphas prefix)
using raw-hit-target unfolding raw-eq alphas-eq .
have target:
  index (to-nat raw)  $\in$ 
    staged-query-prefix-committed-query-target prefix prefix-state
by (rule staged-query-prefix-committed-query-targetI
    [OF candidate raw-target-prefix])
show ?thesis
unfolding checked-staged-query-prefix-dynamic-index-hit-def
using target by simp
qed
show ?thesis
using prefix-receive hit-if-bound by blast
qed

lemma checked-staged-transcript-query-bound-table-hit-at-length-committed-prefix-receive-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:

```


Some (*data*, *attacker-state*) $\in$
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *i-bound*: *i* < *rounds*
and *lookup-attacker*:
fmlookup (HashMap attacker-state)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data) (staged-query-chunks data) i)) =
Some raw-hit
and *raw-hit-target*:
index (to-nat raw-hit) $\in$
query-sampling-success-space trace-table composition-table
(staged-alphas data)
and *comp-nonempty*:
staged-composition-fri-roots data $\neq$ []
and *trace-len*: *length trace-table = clength * scale*
and *comp-len*: *length composition-table = clength * scale*
shows
 $\exists$ *prefix prefix-state raw raw-state.*
Some (((prefix, prefix-state), raw), raw-state) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state) $\wedge$
(merkle-root-binds-table (sqp-trace-root prefix) trace-table
prefix-state $\longrightarrow$
merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
composition-table prefix-state $\longrightarrow$
checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-length-committed-query-target
(Some (((prefix, prefix-state), raw), raw-state)))
proof –
from *checked-staged-transcript-program-query-prefix-receive-compact-support*
[OF wf controlled i-bound outcome]
obtain *prefix prefix-state raw raw-state where*
prefix-receive:
Some (((prefix, prefix-state), raw), raw-state) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *root-eq*: *sqp-trace-root prefix = staged-trace-root data*
and *alphas-eq*: *sqp-alphas prefix = staged-alphas data*
and *comp-roots-eq*:
sqp-composition-fri-roots prefix =
staged-composition-fri-roots data
and *prefix-ext*: *prefix-state $\leq$ attacker-state*
and *raw-ext*: *raw-state $\leq$ attacker-state*
and *lookup-raw*:

```

    fmlookup (HashMap raw-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
        Some raw
  by (rule checked-staged-transcript-program-query-prefix-receive-compact-support
    [OF wf controlled i-bound outcome])
  have lookup-attacker-raw:
    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
        Some raw
  by (rule hash-extension-lookup[OF lookup-raw raw-ext])
  have raw-eq: raw = raw-hit
  using lookup-attacker lookup-attacker-raw by simp
  have comp-nonempty-prefix:
    sqp-composition-fri-roots prefix ≠ []
  using comp-nonempty comp-roots-eq by simp
  have hit-if-bound:
    merkle-root-binds-table (sqp-trace-root prefix) trace-table
      prefix-state ⇒
    merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
      composition-table prefix-state ⇒
    checked-staged-query-prefix-dynamic-index-hit
      staged-query-prefix-length-committed-query-target
      (Some (((prefix, prefix-state), raw), raw-state))
  proof -
    assume trace-bind:
      merkle-root-binds-table (sqp-trace-root prefix) trace-table
        prefix-state
    assume comp-bind:
      merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
        composition-table prefix-state
    have candidate:
      (trace-table, composition-table) ∈
        staged-query-prefix-length-committed-table-candidates prefix
        prefix-state
    unfolding staged-query-prefix-length-committed-table-candidates-def
    using trace-bind comp-nonempty-prefix comp-bind trace-len comp-len
    by simp
    have raw-target-prefix:
      index (to-nat raw) ∈
        query-sampling-success-space trace-table composition-table
        (sqp-alphas prefix)
    using raw-hit-target unfolding raw-eq alphas-eq .
    have target:
      index (to-nat raw) ∈
        staged-query-prefix-length-committed-query-target prefix

```

```

    prefix-state
  by (rule staged-query-prefix-length-committed-query-targetI
      [OF candidate raw-target-prefix])
show ?thesis
  unfolding checked-staged-query-prefix-dynamic-index-hit-def
  using target by simp
qed
show ?thesis
  using prefix-receive hit-if-bound by blast
qed

lemma checked-staged-bound-tables-initial-roots-late-output-at-query-prefix-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
            adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈
      set-dist
        (execute verify-monad
            (verifier-state-from-adversary attacker-state
              (staged-proof-transcript data))))
  and i-bound: i < rounds
  and bound:
    accepted-with-bound-tables
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
    (Some (result, final-state))
    trace-table composition-table as query-idxs
  shows
    ∃ prefix prefix-state raw raw-state.
      Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
          (execute (checked-staged-query-prefix-receive-with-state A i)
              adversary-initial-state) ∧
      sqp-composition-fri-roots prefix ≠ [] ∧
      merkle-root-binds-table (sqp-trace-root prefix) trace-table
        final-state ∧
      merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
        composition-table final-state ∧
      (((∃ input.
          fmlookup (HashMap prefix-state) input =
            Some (sqp-trace-root prefix)) ∧
        (∃ input.
          fmlookup (HashMap prefix-state) input =
            Some (hd (sqp-composition-fri-roots prefix))))) ∨

```

```

    hash-map-new-output-hit
    {sqp-trace-root prefix, hd (sqp-composition-fri-roots prefix)}
    prefix-state final-state)
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from checked-staged-transcript-program-query-prefix-receive-compact-support
  [OF wf controlled i-bound outcome]
obtain prefix prefix-state raw raw-state where
  prefix-receive:
    Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
    adversary-initial-state)
and root-eq: sqp-trace-root prefix = staged-trace-root data
and comp-roots-eq:
  sqp-composition-fri-roots prefix =
    staged-composition-fri-roots data
and prefix-ext: prefix-state ≤ attacker-state
and raw-ext: raw-state ≤ attacker-state
and lookup-raw:
  fmlookup (HashMap raw-state)
  (QueryIndexChallenge i
  (state-after-query-chunks
  (staged-query-start-hash data) (staged-query-chunks data) i)) =
  Some raw
by (rule checked-staged-transcript-program-query-prefix-receive-compact-support
  [OF wf controlled i-bound outcome])
have prefix-ext-s: prefix-state ≤ ?s
  by (rule hash-extends-verifier-state-from-adversary-right
  [OF prefix-ext])
have s-ext-final: ?s ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
have prefix-ext-final: prefix-state ≤ final-state
  by (rule hash-ext-trans[OF prefix-ext-s s-ext-final])
have staged-header:
  verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled outcome])
from accepted-with-bound-tables-initial-roots-late-output-preexisting-or-union-hit

```

```

[OF prefix-ext-final bound]
obtain fr f-fri-roots f-final dg composition-fri-roots final rest where
  header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and comp-nonempty: composition-fri-roots  $\neq []$ 
and trace-bind:
  merkle-root-binds-table fr trace-table final-state
and comp-bind:
  merkle-root-binds-table (hd composition-fri-roots) composition-table
  final-state
and late:
  (( $\exists$  input. fmlookup (HashMap prefix-state) input = Some fr)  $\wedge$ 
    ( $\exists$  input.
      fmlookup (HashMap prefix-state) input =
        Some (hd composition-fri-roots)))  $\vee$ 
    hash-map-new-output-hit {fr, hd composition-fri-roots}
    prefix-state final-state
by blast
have eqs:
  fr = staged-trace-root data  $\wedge$ 
  f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
  f-final = staged-trace-final data  $\wedge$ 
  as = staged-alphas data  $\wedge$ 
  dg = staged-degree data  $\wedge$ 
  composition-fri-roots = staged-composition-fri-roots data  $\wedge$ 
  final = staged-composition-final data  $\wedge$ 
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
have comp-nonempty-prefix: sqp-composition-fri-roots prefix  $\neq []$ 
using comp-nonempty comp-roots-eq eqs by simp
have trace-bind-prefix:
  merkle-root-binds-table (sqp-trace-root prefix) trace-table
  final-state
using trace-bind root-eq eqs by simp
have comp-bind-prefix:
  merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
  composition-table final-state
using comp-bind comp-roots-eq eqs by simp
have late-prefix:
  (( $\exists$  input.
    fmlookup (HashMap prefix-state) input =
      Some (sqp-trace-root prefix))  $\wedge$ 
    ( $\exists$  input.
      fmlookup (HashMap prefix-state) input =
        Some (hd (sqp-composition-fri-roots prefix))))  $\vee$ 
    hash-map-new-output-hit
    {sqp-trace-root prefix, hd (sqp-composition-fri-roots prefix)}

```

```

    prefix-state final-state
    using late root-eq comp-roots-eq eqs by simp
show ?thesis
  apply (rule-tac x=prefix in exI)
  apply (rule-tac x=prefix-state in exI)
  apply (rule-tac x=raw in exI)
  apply (rule-tac x=raw-state in exI)
  using prefix-receive comp-nonempty-prefix trace-bind-prefix
    comp-bind-prefix late-prefix
  by blast
qed

lemma checked-staged-bound-tables-initial-roots-pullback-at-query-prefix-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
  and i-bound: i < rounds
  and bound:
    accepted-with-bound-tables
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state))
      trace-table composition-table as query-idxs
  and query-hit:
    query-idxs ! i ∈
      query-sampling-success-space trace-table composition-table as
  shows
    ∃ prefix prefix-state raw raw-state trace-tree composition-tree.
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state) ∧
    sqp-alphas prefix = staged-alphas data ∧
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw ∧
    index (to-nat raw) ∈

```

```

    query-sampling-success-space trace-table composition-table
      (staged-alphas data) ∧
    sqp-composition-fri-roots prefix ≠ [] ∧
    created-tree trace-table trace-tree final-state ∧
    sqp-trace-root prefix = value trace-tree ∧
    created-tree composition-table composition-tree final-state ∧
    hd (sqp-composition-fri-roots prefix) = value composition-tree ∧
    ((merkle-root-binds-table (sqp-trace-root prefix) trace-table
      prefix-state ∧
      merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
        composition-table prefix-state) ∨
      hash-map-new-output-hit
        (set-tree trace-tree ∪ set-tree composition-tree)
        prefix-state final-state)
  proof —
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from checked-staged-transcript-program-query-prefix-receive-compact-support
    [OF wf controlled i-bound outcome]
  obtain prefix prefix-state raw raw-state where
    prefix-receive:
      Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
          (execute (checked-staged-query-prefix-receive-with-state A i)
            adversary-initial-state)
  and root-eq: sqp-trace-root prefix = staged-trace-root data
  and alphas-eq: sqp-alphas prefix = staged-alphas data
  and comp-roots-eq:
    sqp-composition-fri-roots prefix =
      staged-composition-fri-roots data
  and prefix-ext: prefix-state ≤ attacker-state
  and raw-ext: raw-state ≤ attacker-state
  and lookup-raw:
    fmlookup (HashMap raw-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw
  by (rule checked-staged-transcript-program-query-prefix-receive-compact-support
    [OF wf controlled i-bound outcome])
  have prefix-ext-s: prefix-state ≤ ?s
  by (rule hash-extends-verifier-state-from-adversary-right
    [OF prefix-ext])
  have s-ext-final: ?s ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
  have prefix-ext-final: prefix-state ≤ final-state
  by (rule hash-ext-trans[OF prefix-ext-s s-ext-final])
  have raw-ext-s: raw-state ≤ ?s

```

```

    by (rule hash-extends-verifier-state-from-adversary-right
        [OF raw-ext])
  have raw-ext-final: raw-state  $\leq$  final-state
    by (rule hash-ext-trans[OF raw-ext-s s-ext-final])
  have lookup-final-raw:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw
    by (rule hash-extension-lookup[OF lookup-raw raw-ext-final])
  have staged-header:
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      (List.concat (staged-query-chunks data))
    by (rule checked-staged-transcript-program-outcome-header-transcript
        [OF wf controlled outcome])
  from accepted-with-bound-tables-initial-roots-pullback-or-new-tree-output-hit
    [OF prefix-ext-final bound]
  obtain fr f-fri-roots f-final dg composition-fri-roots final rest
    trace-tree composition-tree where
    header:
      verifier-header-transcript ?s fr f-fri-roots f-final as dg
      composition-fri-roots final rest
  and comp-nonempty: composition-fri-roots  $\neq$  []
  and trace-created:
    created-tree trace-table trace-tree final-state
  and trace-root: fr = value trace-tree
  and comp-created:
    created-tree composition-table composition-tree final-state
  and comp-root:
    hd composition-fri-roots = value composition-tree
  and split:
    (merkle-root-binds-table fr trace-table prefix-state  $\wedge$ 
      merkle-root-binds-table (hd composition-fri-roots)
        composition-table prefix-state)  $\vee$ 
    hash-map-new-output-hit
      (set-tree trace-tree  $\cup$  set-tree composition-tree)
      prefix-state final-state
  by blast
  have eqs:
    fr = staged-trace-root data  $\wedge$ 
    f-fri-roots = staged-trace-fri-roots data  $\wedge$ 

```



```

f-final = staged-trace-final data ∧
as = staged-alphas data ∧
dg = staged-degree data ∧
composition-fri-roots = staged-composition-fri-roots data ∧
final = staged-composition-final data ∧
rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
have support-some:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  by (rule checked-staged-security-experiment-with-data-state-outcomeI
    [OF outcome verifier])
have shape:
  accepted-transcript-shape ?s (Some (result, final-state))
    as query-idxs
  using bound
  unfolding accepted-with-bound-tables-def accepted-with-tables-def
  by simp
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support-some shape]
obtain final-raw-idxs where
  as-eq: as = staged-alphas data
  and len-final-raw: length final-raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) final-raw-idxs
  and lookup-final:
     $\bigwedge j. j < rounds \implies$ 
      fmlookup (HashMap final-state)
        (QueryIndexChallenge j
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) j)) =
        Some (final-raw-idxs ! j)
  by blast
have raw-eq: final-raw-idxs ! i = raw
  using lookup-final[OF i-bound] lookup-final-raw by simp
have raw-hit:
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table
      (staged-alphas data)
proof –
  have index (to-nat (final-raw-idxs ! i)) = query-idxs ! i
    using query-idxs-eq len-final-raw i-bound by simp
  then show ?thesis
    using query-hit as-eq raw-eq by simp

```

```

qed
have comp-nonempty-prefix: sqp-composition-fri-roots prefix ≠ []
  using comp-nonempty comp-roots-eq eqs by simp
have trace-root-prefix:
  sqp-trace-root prefix = value trace-tree
  using root-eq trace-root eqs by simp
have comp-root-prefix:
  hd (sqp-composition-fri-roots prefix) = value composition-tree
  using comp-root comp-roots-eq eqs by simp
have split-prefix:
  ((merkle-root-binds-table (sqp-trace-root prefix) trace-table
    prefix-state ∧
    merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
      composition-table prefix-state) ∨
    hash-map-new-output-hit
      (set-tree trace-tree ∪ set-tree composition-tree)
      prefix-state final-state)
  using split root-eq comp-roots-eq eqs by simp
show ?thesis
  apply (rule-tac x=prefix in exI)
  apply (rule-tac x=prefix-state in exI)
  apply (rule-tac x=raw in exI)
  apply (rule-tac x=raw-state in exI)
  apply (rule-tac x=trace-tree in exI)
  apply (rule-tac x=composition-tree in exI)
  using prefix-receive alphas-eq lookup-final-raw raw-hit
    comp-nonempty-prefix trace-created trace-root-prefix comp-created
    comp-root-prefix split-prefix
  by blast
qed

lemma checked-staged-transcript-query-bound-table-hit-or-tree-output-at-length-committed-prefix-receive-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))))
  and i-bound: i < rounds
  and bound:
    accepted-with-bound-tables
      (verifier-state-from-adversary attacker-state

```

```

    (staged-proof-transcript data))
    (Some (result, final-state))
    trace-table composition-table as query-idxs
and query-hit:
    query-idxs ! i ∈
    query-sampling-success-space trace-table composition-table as
shows
  ∃ prefix prefix-state raw raw-state trace-tree composition-tree.
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state) ∧
  fmlookup (HashMap final-state)
  (QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw ∧
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table
    (staged-alphas data) ∧
  sqp-composition-fri-roots prefix ≠ [] ∧
  created-tree trace-table trace-tree final-state ∧
  sqp-trace-root prefix = value trace-tree ∧
  created-tree composition-table composition-tree final-state ∧
  hd (sqp-composition-fri-roots prefix) = value composition-tree ∧
  (checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-length-committed-query-target
    (Some (((prefix, prefix-state), raw), raw-state)) ∨
    hash-map-new-output-hit
    (set-tree trace-tree ∪ set-tree composition-tree)
    prefix-state final-state)
proof –
from checked-staged-bound-tables-initial-roots-pullback-at-query-prefix-support
  [OF wf controlled outcome verifier i-bound bound query-hit]
obtain prefix prefix-state raw raw-state trace-tree composition-tree where
  prefix-receive:
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
and alphas-eq: sqp-alphas prefix = staged-alphas data
and lookup-final:
  fmlookup (HashMap final-state)
  (QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
and raw-hit:
  index (to-nat raw) ∈

```

```

    query-sampling-success-space trace-table composition-table
      (staged-alphas data)
  and comp-nonempty: sqp-composition-fri-roots prefix ≠ []
  and trace-created: created-tree trace-table trace-tree final-state
  and trace-root: sqp-trace-root prefix = value trace-tree
  and comp-created:
    created-tree composition-table composition-tree final-state
  and comp-root:
    hd (sqp-composition-fri-roots prefix) = value composition-tree
  and split:
    (merkle-root-binds-table (sqp-trace-root prefix) trace-table
      prefix-state ∧
      merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
        composition-table prefix-state) ∨
    hash-map-new-output-hit
      (set-tree trace-tree ∪ set-tree composition-tree)
      prefix-state final-state
  by blast
  have trace-len: length trace-table = clength * scale
  using bound unfolding accepted-with-bound-tables-def
    accepted-with-tables-def by simp
  have comp-len: length composition-table = clength * scale
  using bound unfolding accepted-with-bound-tables-def
    accepted-with-tables-def by simp
  have prefix-or-tree:
    checked-staged-query-prefix-dynamic-index-hit
      staged-query-prefix-length-committed-query-target
      (Some (((prefix, prefix-state), raw), raw-state)) ∨
    hash-map-new-output-hit
      (set-tree trace-tree ∪ set-tree composition-tree)
      prefix-state final-state
  proof —
  from split show ?thesis
  proof
  assume binds:
    merkle-root-binds-table (sqp-trace-root prefix) trace-table
      prefix-state ∧
    merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
      composition-table prefix-state
  then have candidate:
    (trace-table, composition-table) ∈
      staged-query-prefix-length-committed-table-candidates prefix
        prefix-state
  unfolding staged-query-prefix-length-committed-table-candidates-def
  using comp-nonempty trace-len comp-len by simp
  have target:
    index (to-nat raw) ∈
      staged-query-prefix-length-committed-query-target prefix
        prefix-state

```

```

proof –
  have raw-hit-prefix:
    index (to-nat raw) ∈
      query-sampling-success-space trace-table composition-table
        (sqp-alphas prefix)
    using raw-hit alphas-eq by simp
  show ?thesis
    by (rule staged-query-prefix-length-committed-query-targetI
      [OF candidate raw-hit-prefix])
  qed
then show ?thesis
  unfolding checked-staged-query-prefix-dynamic-index-hit-def
  by simp
next
  assume
    hash-map-new-output-hit
      (set-tree trace-tree ∪ set-tree composition-tree)
    prefix-state final-state
  then show ?thesis by simp
  qed
qed
show ?thesis
  apply (rule-tac x=prefix in exI)
  apply (rule-tac x=prefix-state in exI)
  apply (rule-tac x=raw in exI)
  apply (rule-tac x=raw-state in exI)
  apply (rule-tac x=trace-tree in exI)
  apply (rule-tac x=composition-tree in exI)
  using prefix-receive lookup-final raw-hit comp-nonempty trace-created
    trace-root comp-created comp-root prefix-or-tree
  by blast
qed

lemma checked-staged-transcript-query-bound-table-hit-or-tree-output-at-committed-prefix-receive-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))))
  and i-bound: i < rounds
  and lookup-attacker:

```

```

    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
        Some raw-hit
  and raw-hit-target:
    index (to-nat raw-hit) ∈
      query-sampling-success-space trace-table composition-table
        (staged-alphas data)
  and bound:
    accepted-with-bound-tables
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state))
      trace-table composition-table as query-idxs
  shows
    ∃ prefix prefix-state raw raw-state trace-tree composition-tree.
      Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
          (execute (checked-staged-query-prefix-receive-with-state A i)
            adversary-initial-state) ∧
      fmlookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data) (staged-query-chunks data) i)) =
          Some raw ∧
      index (to-nat raw) ∈
        query-sampling-success-space trace-table composition-table
          (staged-alphas data) ∧
      sqp-composition-fri-roots prefix ≠ [] ∧
      created-tree trace-table trace-tree final-state ∧
      sqp-trace-root prefix = value trace-tree ∧
      created-tree composition-table composition-tree final-state ∧
      hd (sqp-composition-fri-roots prefix) = value composition-tree ∧
      (checked-staged-query-prefix-dynamic-index-hit
        (λprefix prefix-state.
          staged-query-prefix-committed-query-target prefix prefix-state)
        (Some (((prefix, prefix-state), raw), raw-state)) ∨
      hash-map-new-output-hit
        (set-tree trace-tree ∪ set-tree composition-tree)
        prefix-state final-state)
  proof —
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from checked-staged-transcript-program-query-prefix-receive-compact-support
    [OF wf controlled i-bound outcome]
  obtain prefix prefix-state raw raw-state where
    prefix-receive:

```

```

Some (((prefix, prefix-state), raw), raw-state) ∈
  set-dist
  (execute (checked-staged-query-prefix-receive-with-state A i)
    adversary-initial-state)
and root-eg: sqp-trace-root prefix = staged-trace-root data
and alphas-eg: sqp-alphas prefix = staged-alphas data
and comp-roots-eg:
  sqp-composition-fri-roots prefix =
    staged-composition-fri-roots data
and prefix-ext: prefix-state ≤ attacker-state
and raw-ext: raw-state ≤ attacker-state
and lookup-raw:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
by (rule checked-staged-transcript-program-query-prefix-receive-compact-support
  [OF wf controlled i-bound outcome])
have lookup-attacker-raw:
  fmlookup (HashMap attacker-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
by (rule hash-extension-lookup[OF lookup-raw raw-ext])
have raw-eg: raw = raw-hit
using lookup-attacker lookup-attacker-raw by simp
have prefix-ext-s: prefix-state ≤ ?s
by (rule hash-extends-verifier-state-from-adversary-right
  [OF prefix-ext])
have s-ext-final: ?s ≤ final-state
by (rule verify-monad-hash-extends[OF verifier])
have prefix-ext-final: prefix-state ≤ final-state
by (rule hash-ext-trans[OF prefix-ext-s s-ext-final])
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled outcome])
from accepted-with-bound-tables-initial-roots-pullback-or-new-tree-output-hit
  [OF prefix-ext-final bound]

```

```

obtain fr f-fri-roots f-final dg composition-fri-roots final rest
  trace-tree composition-tree where
  header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and comp-nonempty: composition-fri-roots  $\neq \square$ 
and trace-created:
  created-tree trace-table trace-tree final-state
and trace-root: fr = value trace-tree
and comp-created:
  created-tree composition-table composition-tree final-state
and comp-root:
  hd composition-fri-roots = value composition-tree
and split:
  (merkle-root-binds-table fr trace-table prefix-state  $\wedge$ 
   merkle-root-binds-table (hd composition-fri-roots)
    composition-table prefix-state)  $\vee$ 
  hash-map-new-output-hit
  (set-tree trace-tree  $\cup$  set-tree composition-tree)
  prefix-state final-state
by blast
have eqs:
  fr = staged-trace-root data  $\wedge$ 
  f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
  f-final = staged-trace-final data  $\wedge$ 
  as = staged-alphas data  $\wedge$ 
  dg = staged-degree data  $\wedge$ 
  composition-fri-roots = staged-composition-fri-roots data  $\wedge$ 
  final = staged-composition-final data  $\wedge$ 
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
have split-prefix:
  (merkle-root-binds-table (sqp-trace-root prefix) trace-table
   prefix-state  $\wedge$ 
   merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
    composition-table prefix-state)  $\vee$ 
  hash-map-new-output-hit
  (set-tree trace-tree  $\cup$  set-tree composition-tree)
  prefix-state final-state
using split root-eq comp-roots-eq eqs by simp
have comp-nonempty-prefix:
  sqp-composition-fri-roots prefix  $\neq \square$ 
using comp-nonempty comp-roots-eq eqs by simp
have raw-hit-actual:
  index (to-nat raw)  $\in$ 
  query-sampling-success-space trace-table composition-table
  (staged-alphas data)
using raw-hit-target unfolding raw-eq .

```



```

have lookup-initial-raw:
  fmlookup (HashMap ?s)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
using lookup-attacker-raw by simp
have lookup-final-raw:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
by (rule hash-extension-lookup[OF lookup-initial-raw s-ext-final])
have trace-root-prefix:
  sqp-trace-root prefix = value trace-tree
using root-eq trace-root eqs by simp
have comp-root-prefix:
  hd (sqp-composition-fri-roots prefix) = value composition-tree
using comp-root comp-roots-eq eqs by simp
have conclusion:
  checked-staged-query-prefix-dynamic-index-hit
    (λprefix prefix-state.
      staged-query-prefix-committed-query-target prefix prefix-state)
    (Some (((prefix, prefix-state), raw), raw-state)) ∨
  hash-map-new-output-hit
    (set-tree trace-tree ∪ set-tree composition-tree)
    prefix-state final-state
proof —
from split-prefix show ?thesis
proof
  assume prefix-binds:
    merkle-root-binds-table (sqp-trace-root prefix) trace-table
    prefix-state ∧
    merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
    composition-table prefix-state
  then have trace-bind:
    merkle-root-binds-table (sqp-trace-root prefix) trace-table
    prefix-state
  and comp-bind:
    merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
    composition-table prefix-state
  by simp-all
have candidate:
  (trace-table, composition-table) ∈
    query-header-committed-table-candidates prefix-state
    (sqp-trace-root prefix)
    (sqp-composition-fri-roots prefix)
unfolding query-header-committed-table-candidates-def

```

```

    using trace-bind comp-nonempty-prefix comp-bind by simp
have raw-target-prefix:
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table
    (sqp-alphas prefix)
  using raw-hit-target unfolding raw-eq alphas-eq .
have target:
  index (to-nat raw) ∈
    staged-query-prefix-committed-query-target prefix prefix-state
  by (rule staged-query-prefix-committed-query-targetI
    [OF candidate raw-target-prefix])
then show ?thesis
  unfolding checked-staged-query-prefix-dynamic-index-hit-def by simp
next
  assume
    hash-map-new-output-hit
    (set-tree trace-tree ∪ set-tree composition-tree)
    prefix-state final-state
  then show ?thesis by simp
qed
qed
show ?thesis
  apply (rule-tac x=prefix in exI)
  apply (rule-tac x=prefix-state in exI)
  apply (rule-tac x=raw in exI)
  apply (rule-tac x=raw-state in exI)
  apply (rule-tac x=trace-tree in exI)
  apply (rule-tac x=composition-tree in exI)
  using prefix-receive lookup-final-raw raw-hit-actual
    comp-nonempty-prefix trace-created trace-root-prefix comp-created
    comp-root-prefix conclusion
  by blast
qed

lemma checked-staged-security-with-data-state-query-bad-imp-committed-prefix-hit-or-tree-output-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    out ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  and bad: staged-security-with-data-state-query-bad-hit out
  shows
    staged-security-with-data-state-query-committed-prefix-hit A out ∨
    staged-security-with-data-state-query-tree-output-hit A out
proof (cases out)
  case None

```

```

then show ?thesis
  using bad unfolding staged-security-with-data-state-query-bad-hit-def
  by simp
next
case (Some result-pack)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have support-some:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  using support unfolding out-eq .
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF support-some]
have builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast+
from bad[unfolded out-eq
  staged-security-with-data-state-query-bad-hit-def Let-def]
have query-bad: query-bad ?s (Some (result, final-state))
  by simp
from query-bad-imp-query-index-round-set-hit[OF query-bad]
obtain i trace-table composition-table as query-idxs where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and i-bound: i < rounds
  and query-hit:
    query-idxs ! i ∈
      query-sampling-success-space trace-table composition-table as
  unfolding query-index-round-set-hit-def
    query-index-round-set-hit-at-def
  by blast
have shape:
  accepted-transcript-shape ?s (Some (result, final-state))
    as query-idxs
  using bound
  unfolding accepted-with-bound-tables-def accepted-with-tables-def

```

```

  by simp
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support-some shape]
obtain final-raw-idxs where
  as-eq: as = staged-alphas data
  and len-final-raw: length final-raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map ( $\lambda$ raw. index (to-nat raw)) final-raw-idxs
  and lookup-final:
     $\bigwedge j. j < \text{rounds} \implies$ 
      fmllookup (HashMap final-state)
        (QueryIndexChallenge j
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) j)) =
        Some (final-raw-idxs ! j)
  by blast
from checked-staged-transcript-program-outcome-query-lookups
  [OF wf controlled builder]
obtain raw-idxs where
  len-raw: length raw-idxs = rounds
  and lookup-attacker-all:
     $\forall j < \text{rounds}.$ 
      fmllookup (HashMap attacker-state)
        (QueryIndexChallenge j
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) j)) =
        Some (raw-idxs ! j)
  by blast
have s-ext-final:  $?s \leq \text{final-state}$ 
  by (rule verify-monad-hash-extends[OF verifier])
have lookup-initial-i:
  fmllookup (HashMap ?s)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  using lookup-attacker-all i-bound by simp
have lookup-final-i-from-initial:
  fmllookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  by (rule hash-extension-lookup[OF lookup-initial-i s-ext-final])
have raw-eq: final-raw-idxs ! i = raw-idxs ! i
  using lookup-final[OF i-bound] lookup-final-i-from-initial by simp
have raw-hit-target:

```

```

    index (to-nat (raw-idxs ! i)) ∈
      query-sampling-success-space trace-table composition-table
        (staged-alphas data)
  proof —
    have index (to-nat (final-raw-idxs ! i)) = query-idxs ! i
      using query-idxs-eq len-final-raw i-bound by simp
    then show ?thesis
      using query-hit as-eq raw-eq by simp
  qed
  have lookup-attacker-i:
    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
    using lookup-attacker-all i-bound by simp
  from
    checked-staged-transcript-query-bound-table-hit-or-tree-output-at-committed-prefix-receive-support
      [OF wf controlled builder verifier i-bound lookup-attacker-i
        raw-hit-target bound]
  obtain prefix prefix-state raw raw-state trace-tree composition-tree where
    prefix-receive:
      Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
          (execute (checked-staged-query-prefix-receive-with-state A i)
            adversary-initial-state)
  and lookup-final-raw:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw
  and raw-hit-actual:
    index (to-nat raw) ∈
      query-sampling-success-space trace-table composition-table
        (staged-alphas data)
  and comp-nonempty: sqp-composition-fri-roots prefix ≠ []
  and trace-created:
    created-tree trace-table trace-tree final-state
  and trace-root: sqp-trace-root prefix = value trace-tree
  and comp-created:
    created-tree composition-table composition-tree final-state
  and comp-root:
    hd (sqp-composition-fri-roots prefix) = value composition-tree
  and split:
    checked-staged-query-prefix-dynamic-index-hit
      (λprefix prefix-state.
        staged-query-prefix-committed-query-target prefix prefix-state)
    (Some (((prefix, prefix-state), raw), raw-state)) ∨

```

```

    hash-map-new-output-hit
    (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
  by blast
from split show ?thesis
proof
  assume prefix-hit:
    checked-staged-query-prefix-dynamic-index-hit
    ( $\lambda$ prefix prefix-state.
      staged-query-prefix-committed-query-target prefix prefix-state)
    (Some (((prefix, prefix-state), raw), raw-state))
  have committed-body:
     $\exists i$  prefix prefix-state raw raw-state.
     $i < \text{rounds} \wedge$ 
    Some (data, attacker-state)  $\in$ 
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\wedge$ 
    Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
      set-dist
      (execute
        (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)  $\wedge$ 
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-committed-query-target
    (Some (((prefix, prefix-state), raw), raw-state))
  apply (rule-tac x=i in exI)
  apply (rule-tac x=prefix in exI)
  apply (rule-tac x=prefix-state in exI)
  apply (rule-tac x=raw in exI)
  apply (rule-tac x=raw-state in exI)
  using i-bound builder prefix-receive prefix-hit
  by simp
then show ?thesis
  unfolding out-eq
    staged-security-with-data-state-query-committed-prefix-hit-def
  by simp
next
  assume tree-hit:
    hash-map-new-output-hit
    (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
  have tree-body:
     $\exists i$  prefix prefix-state raw raw-state trace-table composition-table
    as query-idxs trace-tree composition-tree.
     $i < \text{rounds} \wedge$ 
    Some (data, attacker-state)  $\in$ 
      set-dist
      (execute (checked-staged-transcript-program A)

```

```

    adversary-initial-state) ∧
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s) ∧
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs ∧
  fmlookup (HashMap final-state)
  (QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data)
      (staged-query-chunks data) i)) =
  Some raw ∧
  index (to-nat raw) ∈
  query-sampling-success-space trace-table composition-table
  (staged-alphas data) ∧
  Some (((prefix, prefix-state), raw), raw-state) ∈
  set-dist
  (execute
    (checked-staged-query-prefix-receive-with-state A i)
    adversary-initial-state) ∧
  sqp-composition-fri-roots prefix ≠ [] ∧
  created-tree trace-table trace-tree final-state ∧
  sqp-trace-root prefix = value trace-tree ∧
  created-tree composition-table composition-tree final-state ∧
  hd (sqp-composition-fri-roots prefix) =
  value composition-tree ∧
  hash-map-new-output-hit
  (set-tree trace-tree ∪ set-tree composition-tree)
  prefix-state final-state
  apply (rule-tac x=i in exI)
  apply (rule-tac x=prefix in exI)
  apply (rule-tac x=prefix-state in exI)
  apply (rule-tac x=raw in exI)
  apply (rule-tac x=raw-state in exI)
  apply (rule-tac x=trace-table in exI)
  apply (rule-tac x=composition-table in exI)
  apply (rule-tac x=as in exI)
  apply (rule-tac x=query-idxs in exI)
  apply (rule-tac x=trace-tree in exI)
  apply (rule-tac x=composition-tree in exI)
  using i-bound builder verifier bound lookup-final-raw raw-hit-actual
  prefix-receive comp-nonempty trace-created trace-root comp-created
  comp-root tree-hit
  by blast
  then show ?thesis
  unfolding out-eq
  staged-security-with-data-state-query-tree-output-hit-def Let-def
  by simp
qed
qed

```

lemma *checked-staged-security-with-data-state-query-bad-imp-length-committed-prefix-hit-or-tree-output-on-support*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *support*:

out $\in$

set-dist

(*execute*

(*checked-staged-security-experiment-with-data-state A*)

adversary-initial-state)

and *bad*: *staged-security-with-data-state-query-bad-hit out*

shows

staged-security-with-data-state-query-length-committed-prefix-hit A out $\vee$

staged-security-with-data-state-query-tree-output-hit A out

proof (*cases out*)

case *None*

then show *?thesis*

using *bad* **unfolding** *staged-security-with-data-state-query-bad-hit-def*

by *simp*

next

case (*Some result-pack*)

then obtain *data attacker-state result final-state* **where** *out-eq*:

out = *Some (((data, attacker-state), result), final-state)*

by (*cases result-pack, auto split: prod.splits*)

let *?s* =

verifier-state-from-adversary attacker-state

(*staged-proof-transcript data*)

have *support-some*:

Some (((data, attacker-state), result), final-state) $\in$

set-dist

(*execute*

(*checked-staged-security-experiment-with-data-state A*)

adversary-initial-state)

using *support* **unfolding** *out-eq* .

from *checked-staged-security-experiment-with-data-state-outcomeE*

[*OF support-some*]

have *builder*:

Some (data, attacker-state) $\in$

set-dist

(*execute (checked-staged-transcript-program A)*

adversary-initial-state)

and *verifier*:

Some (result, final-state) $\in$ set-dist (execute verify-monad ?s)

by *blast+*

from *bad[unfolded out-eq*

staged-security-with-data-state-query-bad-hit-def Let-def]

have *query-bad*: *query-bad ?s (Some (result, final-state))*

by *simp*

from *query-bad-imp-query-index-round-set-hit[OF query-bad]*

obtain *i trace-table composition-table as query-idxs* **where**


```

bound:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs
and i-bound:  $i < \text{rounds}$ 
and query-hit:
  query-idxs !  $i \in$ 
    query-sampling-success-space trace-table composition-table as
unfolding query-index-round-set-hit-def
  query-index-round-set-hit-at-def
by blast
from
  checked-staged-transcript-query-bound-table-hit-or-tree-output-at-length-committed-prefix-receive-support
  [OF wf controlled builder verifier i-bound bound query-hit]
obtain prefix prefix-state raw raw-state trace-tree composition-tree where
  prefix-receive:
    Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
      set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
and lookup-final-raw:
  fmlookup (HashMap final-state)
  (QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
and raw-hit-actual:
  index (to-nat raw)  $\in$ 
    query-sampling-success-space trace-table composition-table
    (staged-alphas data)
and comp-nonempty: sqp-composition-fri-roots prefix  $\neq []$ 
and trace-created:
  created-tree trace-table trace-tree final-state
and trace-root: sqp-trace-root prefix = value trace-tree
and comp-created:
  created-tree composition-table composition-tree final-state
and comp-root:
  hd (sqp-composition-fri-roots prefix) = value composition-tree
and split:
  checked-staged-query-prefix-dynamic-index-hit
  staged-query-prefix-length-committed-query-target
  (Some (((prefix, prefix-state), raw), raw-state))  $\vee$ 
  hash-map-new-output-hit
  (set-tree trace-tree  $\cup$  set-tree composition-tree)
  prefix-state final-state
by blast
from split show ?thesis
proof
  assume prefix-hit:
    checked-staged-query-prefix-dynamic-index-hit

```

```

    staged-query-prefix-length-committed-query-target
    (Some (((prefix, prefix-state), raw), raw-state))
have length-body:
  ∃ i prefix prefix-state raw raw-state.
    i < rounds ∧
    Some (data, attacker-state) ∈
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state) ∧
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
      (execute
        (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state) ∧
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-length-committed-query-target
    (Some (((prefix, prefix-state), raw), raw-state))
apply (rule-tac x=i in exI)
apply (rule-tac x=prefix in exI)
apply (rule-tac x=prefix-state in exI)
apply (rule-tac x=raw in exI)
apply (rule-tac x=raw-state in exI)
using i-bound builder prefix-receive prefix-hit
by simp
then show ?thesis
  unfolding out-eq
    staged-security-with-data-state-query-length-committed-prefix-hit-def
  by simp
next
assume tree-hit:
  hash-map-new-output-hit
  (set-tree trace-tree ∪ set-tree composition-tree)
  prefix-state final-state
have tree-body:
  ∃ i prefix prefix-state raw raw-state trace-table composition-table
    as query-idxs trace-tree composition-tree.
    i < rounds ∧
    Some (data, attacker-state) ∈
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state) ∧
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s) ∧
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs ∧
    fmllookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =

```

```

    Some raw  $\wedge$ 
    index (to-nat raw)  $\in$ 
      query-sampling-success-space trace-table composition-table
      (staged-alphas data)  $\wedge$ 
    Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
      set-dist
      (execute
        (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)  $\wedge$ 
    sqp-composition-fri-roots prefix  $\neq \square$   $\wedge$ 
    created-tree trace-table trace-tree final-state  $\wedge$ 
    sqp-trace-root prefix = value trace-tree  $\wedge$ 
    created-tree composition-table composition-tree final-state  $\wedge$ 
    hd (sqp-composition-fri-roots prefix) =
      value composition-tree  $\wedge$ 
    hash-map-new-output-hit
      (set-tree trace-tree  $\cup$  set-tree composition-tree)
      prefix-state final-state
  apply (rule-tac x=i in exI)
  apply (rule-tac x=prefix in exI)
  apply (rule-tac x=prefix-state in exI)
  apply (rule-tac x=raw in exI)
  apply (rule-tac x=raw-state in exI)
  apply (rule-tac x=trace-table in exI)
  apply (rule-tac x=composition-table in exI)
  apply (rule-tac x=as in exI)
  apply (rule-tac x=query-idxs in exI)
  apply (rule-tac x=trace-tree in exI)
  apply (rule-tac x=composition-tree in exI)
  using i-bound builder verifier bound lookup-final-raw raw-hit-actual
    prefix-receive comp-nonempty trace-created trace-root comp-created
    comp-root tree-hit
  by blast
then show ?thesis
  unfolding out-eq
    staged-security-with-data-state-query-tree-output-hit-def Let-def
  by simp
qed
qed

```

lemma *checked-staged-security-with-data-state-query-bad-bound-from-committed-prefix-hit-or-tree-output:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-query-bad-hit adversary-initial-state $\leq$
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-query-committed-prefix-hit A)
 adversary-initial-state +

```

    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-tree-output-hit A)
      adversary-initial-state
  proof -
    let ?M = checked-staged-security-experiment-with-data-state A
    let ?committed =
      staged-security-with-data-state-query-committed-prefix-hit A
    let ?tree = staged-security-with-data-state-query-tree-output-hit A
    have wp-event ?M staged-security-with-data-state-query-bad-hit
      adversary-initial-state ≤
      wp-event ?M (λout. ?committed out ∨ ?tree out)
      adversary-initial-state
    by (rule wp-event-mono-on-support)
      (rule checked-staged-security-with-data-state-query-bad-imp-committed-prefix-hit-or-tree-output-on-support
        [OF wf controlled])
    also have ... ≤
      wp-event ?M ?committed adversary-initial-state +
      wp-event ?M ?tree adversary-initial-state
    by (rule wp-event-union-bound)
    finally show ?thesis .
  qed

  lemma checked-staged-security-with-data-state-query-bad-bound-from-length-committed-prefix-hit-or-tree-output:
    assumes wf: staged-budget-wellformed budgets
      and controlled: staged-adversary-controlled budgets A
    shows
      wp-event (checked-staged-security-experiment-with-data-state A)
        staged-security-with-data-state-query-bad-hit adversary-initial-state ≤
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-with-data-state-query-length-committed-prefix-hit A)
        adversary-initial-state +
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-with-data-state-query-tree-output-hit A)
        adversary-initial-state
  proof -
    let ?M = checked-staged-security-experiment-with-data-state A
    let ?committed =
      staged-security-with-data-state-query-length-committed-prefix-hit A
    let ?tree = staged-security-with-data-state-query-tree-output-hit A
    have wp-event ?M staged-security-with-data-state-query-bad-hit
      adversary-initial-state ≤
      wp-event ?M (λout. ?committed out ∨ ?tree out)
      adversary-initial-state
    by (rule wp-event-mono-on-support)
      (rule
        checked-staged-security-with-data-state-query-bad-imp-length-committed-prefix-hit-or-tree-output-on-support
        [OF wf controlled])
    also have ... ≤
      wp-event ?M ?committed adversary-initial-state +

```

```

      wp-event ?M ?tree adversary-initial-state
    by (rule wp-event-union-bound)
  finally show ?thesis .
qed

end

end

```

```

theory Soundness-Bound-Table-Partial-Openings
  imports Soundness-Partial-Merkle
begin

```

Bridges from complete Merkle-bound table witnesses to partial authenticated openings. This proof layer is kept separate from `Soundness_Partial_Merkle` to avoid growing the base Merkle/event theory.

```

context soundness
begin

```

```

definition partial-composition-openings-inconsistent-bad
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool

```

where

```

  partial-composition-openings-inconsistent-bad s out  $\longleftrightarrow$ 
    ( $\exists$  composition-root query-idxs composition-openings result final-state
      i opn opn'.
      out = Some (result, final-state)  $\wedge$ 
      accepted-with-partial-composition-openings s out composition-root
        query-idxs composition-openings  $\wedge$ 
      i < rounds  $\wedge$ 
      opn  $\in$  set (composition-openings ! i)  $\wedge$ 
      opn'  $\in$  set (composition-openings ! i)  $\wedge$ 
      opening-index opn = opening-index opn'  $\wedge$ 
      opening-value opn  $\neq$  opening-value opn')

```

lemma partial-composition-openings-inconsistent-bad-imp-partial-merkle-inconsistency-bad:

```

  assumes bad: partial-composition-openings-inconsistent-bad s out
  shows partial-merkle-inconsistency-bad s out

```

proof –

```

  obtain composition-root query-idxs composition-openings result final-state
    i opn opn' where
    out: out = Some (result, final-state)
  and partial:
    accepted-with-partial-composition-openings s out composition-root
      query-idxs composition-openings
  and i-bound: i < rounds
  and opn-in: opn  $\in$  set (composition-openings ! i)
  and opn'-in: opn'  $\in$  set (composition-openings ! i)

```

```

and same-index: opening-index opn = opening-index opn'
and diff-value: opening-value opn  $\neq$  opening-value opn'
using bad
unfolding partial-composition-openings-inconsistent-bad-def by blast
have table:
  partial-authenticated-table composition-root (scale * clength)
    (composition-openings ! i) final-state
proof –
  obtain result' final-state' where
    partial-out: out = Some (result', final-state')
  and tables:
     $\forall i < \text{rounds}.$ 
      partial-authenticated-table composition-root (scale * clength)
        (composition-openings ! i) final-state'
  using partial
  unfolding accepted-with-partial-composition-openings-def by auto
have final-state-eq: final-state' = final-state
  using out partial-out by simp
show ?thesis
  using tables i-bound unfolding final-state-eq by simp
qed
have opn-auth: authenticated-opening-in final-state opn
  using table opn-in unfolding partial-authenticated-table-def by blast
have opn'-auth: authenticated-opening-in final-state opn'
  using table opn'-in unfolding partial-authenticated-table-def by blast
have same-root: opening-root opn = opening-root opn'
proof –
  have root-opn: opening-root opn = composition-root
    using table opn-in unfolding partial-authenticated-table-def by simp
  have root-opn': opening-root opn' = composition-root
    using table opn'-in unfolding partial-authenticated-table-def by simp
  show ?thesis
    using root-opn root-opn' by simp
qed
have same-length: opening-length opn = opening-length opn'
proof –
  have length-opn: opening-length opn = scale * clength
    using table opn-in unfolding partial-authenticated-table-def by simp
  have length-opn': opening-length opn' = scale * clength
    using table opn'-in unfolding partial-authenticated-table-def by simp
  show ?thesis
    using length-opn length-opn' by simp
qed
have accepted-out: accepted out
  using partial
  unfolding accepted-with-partial-composition-openings-def by simp
show ?thesis
  unfolding partial-merkle-inconsistency-bad-def
  using accepted-out out opn-auth opn'-auth same-root same-length

```

same-index diff-value
 by blast
 qed

lemma *partial-composition-openings-inconsistent-bad-imp-hash-map-output-collision-bad*:
 assumes *partial-composition-openings-inconsistent-bad s out*
 shows *hash-map-output-collision-bad s out*
 by (rule *partial-merkle-inconsistency-bad-imp-hash-map-output-collision-bad*)
 (rule
 partial-composition-openings-inconsistent-bad-imp-partial-merkle-inconsistency-bad
 [*OF assms*])

lemma *partial-composition-openings-inconsistent-bad-mono-hash-map-output-collision-bad*:
 wp-event verify-monad (*partial-composition-openings-inconsistent-bad s*) *s*
 $\leq$ wp-event verify-monad (*hash-map-output-collision-bad s*) *s*
 by (rule *wp-event-mono*)
 (rule *partial-composition-openings-inconsistent-bad-imp-hash-map-output-collision-bad*)

lemma *partial-composition-openings-inconsistent-bad-bound-from-hash-map-output-collision*:
 fixes *H :: prob*
 assumes wp-event verify-monad (*hash-map-output-collision-bad s*) $s \leq H$
 shows
 wp-event verify-monad
 (*partial-composition-openings-inconsistent-bad s*) $s \leq H$
 by (rule *order-trans*
 [*OF*
 partial-composition-openings-inconsistent-bad-mono-hash-map-output-collision-bad
 assms])

lemma *table-agrees-with-authenticated-openings-from-maps*:
 assumes *len-table: length table = len*
 and *indices: map opening-index openings = idxs*
 and *value-map: map opening-value openings = map (!) table* *idxs*
 and *idx-bound: $\bigwedge idx. idx \in \text{set } idxs \implies idx < len$*
 shows *table-agrees-with-authenticated-openings table len openings*

proof –

have *opening-props*:
 $\forall opn \in \text{set } openings.$
 opening-index opn $< len \wedge$

proof

fix *opn*
 assume *opn-in: opn $\in$ set openings*
 then obtain *k* where *k-bound: k < length openings*
 and *opn-eq: opn = openings ! k*
 by (*metis in-set-conv-nth*)
 have *idxs-len: length idxs = length openings*
 using *arg-cong*[*OF indices, of length*] by *simp*
 have *idx-eq: opening-index opn = idxs ! k*

```

    using indices k-bound idxs-len opn-eq
    by (metis length-map nth-map)
  have value-eq: opening-value opn = table ! (idxs ! k)
proof -
  have opening-value opn = map opening-value openings ! k
    using k-bound opn-eq by simp
  also have ... = map (! table) idxs ! k
    using value-map by simp
  also have ... = table ! (idxs ! k)
    using k-bound idxs-len by simp
  finally show ?thesis .
qed
have idx-in: idxs ! k ∈ set idxs
  by (rule nth-mem) (use k-bound idxs-len in simp)
show opening-index opn < len ∧
  table ! opening-index opn = opening-value opn
  using idx-bound[OF idx-in] idx-eq value-eq by simp
qed
show ?thesis
  unfolding table-agrees-with-authenticated-openings-def
  using len-table opening-props by simp
qed

lemma accepted-with-bound-tables-partial-opening-witnesses:
  assumes bound:
    accepted-with-bound-tables s out trace-table composition-table as
    query-idxs
  obtains result final-state fr f-fri-roots f-final dg composition-fri-roots
    final rest trace-openings composition-openings where
    out = Some (result, final-state)
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
    composition-fri-roots ≠ []
    accepted-with-partial-trace-openings s out fr query-idxs
    trace-openings
    accepted-with-partial-composition-openings s out
    (hd composition-fri-roots) query-idxs composition-openings
    partial-trace-table-candidate trace-table trace-openings
    partial-composition-table-candidate composition-table
    composition-openings
    merkle-root-binds-table fr trace-table final-state
    merkle-root-binds-table (hd composition-fri-roots) composition-table
    final-state
proof -
  from bound obtain result final-state fr f-fri-roots f-final dg
    composition-fri-roots final rest where out-eq:
    out = Some (result, final-state)
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg

```



```

    composition-fri-roots final rest
and comp-nonempty: composition-fri-roots  $\neq []$ 
and trace-bind:
    merkle-root-binds-table fr trace-table final-state
and comp-bind:
    merkle-root-binds-table (hd composition-fri-roots) composition-table
    final-state
unfolding accepted-with-bound-tables-def by blast
have accepted-out: accepted out
    by (rule accepted-with-bound-tables-imp-accepted[OF bound])
have trace-len-mult: length trace-table = scale * clength
    using accepted-with-bound-tables-shapes(1)[OF bound]
    by (simp add: mult.commute)
have comp-len-mult: length composition-table = scale * clength
    using accepted-with-bound-tables-shapes(2)[OF bound]
    by (simp add: mult.commute)
have query-len: length query-idxs = rounds
    by (rule accepted-with-bound-tables-shapes(4)[OF bound])
have trace-exists:
 $\bigwedge i. i < \text{rounds} \implies$ 
 $\exists \text{openings.}$ 
    map opening-index openings = powers-scaled (query-idxs ! i)  $\wedge$ 
    map opening-value openings =
        map ((! trace-table) (powers-scaled (query-idxs ! i)))  $\wedge$ 
    partial-authenticated-table fr (scale * clength) openings
    final-state
proof –
    fix i
    assume i-bound: i < rounds
    have idx-in: query-idxs ! i  $\in$  set query-idxs
        by (rule nth-mem) (use i-bound query-len in simp)
    have idx-sample: query-idxs ! i  $\in$  query-sample-space
        by (rule accepted-with-bound-tables-query-sample-space[OF bound idx-in])
    have idxs-bound:
 $\bigwedge \text{idx. idx} \in \text{set (powers-scaled (query-idxs ! i))} \implies$ 
    idx < scale * clength
    using query-sample-space-powers-scaled-bound[OF idx-sample]
    by (simp add: mult.commute)
    show
 $\exists \text{openings.}$ 
    map opening-index openings = powers-scaled (query-idxs ! i)  $\wedge$ 
    map opening-value openings =
        map ((! trace-table) (powers-scaled (query-idxs ! i)))  $\wedge$ 
    partial-authenticated-table fr (scale * clength) openings
    final-state
proof (rule merkle-root-binds-table-authenticated-openings[
    OF trace-bind trace-len-mult idxs-bound])
    fix openings
    assume index-map:

```

```

    map opening-index openings = powers-scaled (query-idxs ! i)
  and value-map:
    map opening-value openings =
      map (!) trace-table (powers-scaled (query-idxs ! i))
  and partial:
    partial-authenticated-table fr (scale * clength) openings
    final-state
  show ?thesis
    by (intro exI[of - openings] conjI index-map value-map partial)
qed
qed
let ?trace-opening =  $\lambda i$ . SOME openings.
  map opening-index openings = powers-scaled (query-idxs ! i)  $\wedge$ 
  map opening-value openings =
    map (!) trace-table (powers-scaled (query-idxs ! i))  $\wedge$ 
  partial-authenticated-table fr (scale * clength) openings final-state
define trace-openings where
  trace-openings = map ?trace-opening [0.. $\text{rounds}$ ]
have trace-openings-len: length trace-openings = rounds
  unfolding trace-openings-def by simp
have trace-openings-props:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)  $\wedge$ 
    map opening-value (trace-openings ! i) =
      map (!) trace-table (powers-scaled (query-idxs ! i))  $\wedge$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
proof -
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  have some-prop:
    map opening-index (?trace-opening i) = powers-scaled (query-idxs ! i)  $\wedge$ 
    map opening-value (?trace-opening i) =
      map (!) trace-table (powers-scaled (query-idxs ! i))  $\wedge$ 
    partial-authenticated-table fr (scale * clength)
      (?trace-opening i) final-state
  by (rule someI-ex[OF trace-exists[OF i-bound]])
  show
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)  $\wedge$ 
    map opening-value (trace-openings ! i) =
      map (!) trace-table (powers-scaled (query-idxs ! i))  $\wedge$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
  using some-prop i-bound unfolding trace-openings-def by simp
qed
have comp-exists:
   $\bigwedge i. i < \text{rounds} \implies$ 

```

```

     $\exists$  openings.
    map opening-index openings =
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 
    map opening-value openings =
      map (!) composition-table
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 
    partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) openings final-state
  proof -
    fix i
    assume i-bound: i < rounds
    have idx-in: query-idxs ! i  $\in$  set query-idxs
      by (rule nth-mem) (use i-bound query-len in simp)
    have idx-bound: query-idxs ! i < scale * clength
      using accepted-with-bound-tables-shapes(5)[OF bound idx-in]
      by (simp add: mult.commute)
    have sibling-bound:
      fri-sibling-index (scale * clength) (query-idxs ! i) <
        scale * clength
      unfolding fri-sibling-index-def
      using eval-domain-size-pos by (simp add: mult.commute)
    have idxs-bound:
       $\bigwedge$  idx. idx  $\in$ 
        set [query-idxs ! i,
             fri-sibling-index (scale * clength) (query-idxs ! i)]  $\implies$ 
        idx < scale * clength
    proof -
      fix idx
      assume idx  $\in$ 
        set [query-idxs ! i,
             fri-sibling-index (scale * clength) (query-idxs ! i)]
      then have idx = query-idxs ! i  $\vee$ 
        idx = fri-sibling-index (scale * clength) (query-idxs ! i)
        by simp
      then show idx < scale * clength
        using idx-bound sibling-bound by blast
    qed
  show
     $\exists$  openings.
    map opening-index openings =
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 
    map opening-value openings =
      map (!) composition-table
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 
    partial-authenticated-table (hd composition-fri-roots)

```

```

    (scale * clength) openings final-state
proof (rule merkle-root-binds-table-authenticated-openings[
    OF comp-bind comp-len-mult idxs-bound])
fix openings
assume index-map:
    map opening-index openings =
    [query-idxs ! i,
    fri-sibling-index (scale * clength) (query-idxs ! i)]
and value-map:
    map opening-value openings =
    map (!) composition-table)
    [query-idxs ! i,
    fri-sibling-index (scale * clength) (query-idxs ! i)]
and partial:
    partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) openings final-state
show ?thesis
    by (intro exI[of - openings] conjI index-map value-map partial)
qed
qed
let ?composition-opening =  $\lambda i$ . SOME openings.
    map opening-index openings =
    [query-idxs ! i,
    fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 
    map opening-value openings =
    map (!) composition-table)
    [query-idxs ! i,
    fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 
    partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) openings final-state
define composition-openings where
    composition-openings = map ?composition-opening [0.. $\text{rounds}$ ]
have comp-openings-len: length composition-openings = rounds
unfolding composition-openings-def by simp
have comp-openings-props:
     $\wedge i. i < \text{rounds} \implies$ 
    map opening-index (composition-openings ! i) =
    [query-idxs ! i,
    fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 
    map opening-value (composition-openings ! i) =
    map (!) composition-table)
    [query-idxs ! i,
    fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 
    partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) (composition-openings ! i) final-state
proof -
    fix i
    assume i-bound:  $i < \text{rounds}$ 
    have some-prop:

```

```

map opening-index (?composition-opening i) =
  [query-idxs ! i,
   fri-sibling-index (scale * clength) (query-idxs ! i)] ∧
map opening-value (?composition-opening i) =
  map (!) composition-table
  [query-idxs ! i,
   fri-sibling-index (scale * clength) (query-idxs ! i)] ∧
partial-authenticated-table (hd composition-fri-roots)
  (scale * clength) (?composition-opening i) final-state
by (rule someI-ex[OF comp-exists[OF i-bound]])
show
map opening-index (composition-openings ! i) =
  [query-idxs ! i,
   fri-sibling-index (scale * clength) (query-idxs ! i)] ∧
map opening-value (composition-openings ! i) =
  map (!) composition-table
  [query-idxs ! i,
   fri-sibling-index (scale * clength) (query-idxs ! i)] ∧
partial-authenticated-table (hd composition-fri-roots)
  (scale * clength) (composition-openings ! i) final-state
using some-prop i-bound unfolding composition-openings-def by simp
qed
have trace-partial:
  accepted-with-partial-trace-openings s out fr query-idxs
  trace-openings
proof –
have indices:
  ∧i. i < rounds ⇒
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
using trace-openings-props by simp
have partials:
  ∧i. i < rounds ⇒
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
using trace-openings-props by simp
show ?thesis
unfolding accepted-with-partial-trace-openings-def
by (intro conjI exI[of - result] exI[of - final-state])
  (use accepted-out out-eq query-len trace-openings-len indices partials
   in simp-all)
qed
have comp-partial:
  accepted-with-partial-composition-openings s out
  (hd composition-fri-roots) query-idxs composition-openings
proof –
have indices:
  ∧i. i < rounds ⇒
    map opening-index (composition-openings ! i) =

```

```

    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
  using comp-openings-props by simp
have partials:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table (hd composition-fri-roots)
      (scale * clength) (composition-openings ! i) final-state
  using comp-openings-props by simp
show ?thesis
  unfolding accepted-with-partial-composition-openings-def
  by (intro conjI exI[of - result] exI[of - final-state])
    (use accepted-out out-eq query-len comp-openings-len indices partials
     in simp-all)
qed
have trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
proof -
  have agrees:
     $\bigwedge i. i < \text{rounds} \implies$ 
      table-agrees-with-authenticated-openings trace-table
        (scale * clength) (trace-openings ! i)
  proof -
    fix i
    assume i-bound: i < rounds
    have idx-in: query-idxs ! i  $\in$  set query-idxs
      by (rule nth-mem) (use i-bound query-len in simp)
    have idx-sample: query-idxs ! i  $\in$  query-sample-space
      by (rule accepted-with-bound-tables-query-sample-space[
        OF bound idx-in])
    have idxs-bound:
       $\bigwedge \text{idx}. \text{idx} \in \text{set (powers-scaled (query-idxs ! i))} \implies$ 
        idx < scale * clength
      using query-sample-space-powers-scaled-bound[OF idx-sample]
      by (simp add: mult.commute)
    show
      table-agrees-with-authenticated-openings trace-table
        (scale * clength) (trace-openings ! i)
  proof -
    have index-map:
      map opening-index (trace-openings ! i) =
        powers-scaled (query-idxs ! i)
      using trace-openings-props[OF i-bound] by blast
    have value-map:
      map opening-value (trace-openings ! i) =
        map (!) trace-table (powers-scaled (query-idxs ! i))
      using trace-openings-props[OF i-bound] by blast
    show ?thesis
      by (rule table-agrees-with-authenticated-openings-from-maps[
        OF trace-len-mult index-map value-map idxs-bound])
  end
end

```

```

qed
qed
show ?thesis
  unfolding partial-trace-table-candidate-def
  by (intro conjI allI impI)
    (use trace-len-mult trace-openings-len agrees in simp-all)
qed
have comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
proof -
  have agrees:
     $\bigwedge i. i < \text{rounds} \implies$ 
    table-agrees-with-authenticated-openings composition-table
    (scale * clength) (composition-openings ! i)
  proof -
    fix i
    assume i-bound: i < rounds
    have idx-in: query-idxs ! i  $\in$  set query-idxs
      by (rule nth-mem) (use i-bound query-len in simp)
    have idx-bound: query-idxs ! i < scale * clength
      using accepted-with-bound-tables-shapes(5)[OF bound idx-in]
      by (simp add: mult.commute)
    have sibling-bound:
      fri-sibling-index (scale * clength) (query-idxs ! i) <
      scale * clength
    unfolding fri-sibling-index-def
    using eval-domain-size-pos by (simp add: mult.commute)
    have idxs-bound:
       $\bigwedge \text{idx}. \text{idx} \in$ 
      set [query-idxs ! i,
        fri-sibling-index (scale * clength) (query-idxs ! i)]  $\implies$ 
      idx < scale * clength
    proof -
      fix idx
      assume idx  $\in$ 
      set [query-idxs ! i,
        fri-sibling-index (scale * clength) (query-idxs ! i)]
      then have idx = query-idxs ! i  $\vee$ 
        idx = fri-sibling-index (scale * clength) (query-idxs ! i)
        by simp
      then show idx < scale * clength
        using idx-bound sibling-bound by blast
    qed
  show
    table-agrees-with-authenticated-openings composition-table
    (scale * clength) (composition-openings ! i)
  proof -
    have index-map:

```

```

    map opening-index (composition-openings ! i) =
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]
    using comp-openings-props[OF i-bound] by blast
  have value-map:
    map opening-value (composition-openings ! i) =
      map (!) composition-table
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]
    using comp-openings-props[OF i-bound] by blast
  show ?thesis
    by (rule table-agrees-with-authenticated-openings-from-maps[
      OF comp-len-mult index-map value-map idxs-bound])
qed
qed
show ?thesis
  unfolding partial-composition-table-candidate-def
  by (intro conjI allI impI)
    (use comp-len-mult comp-openings-len agrees in simp-all)
qed
show ?thesis
  by (rule that[OF out-eq header comp-nonempty trace-partial comp-partial
    trace-candidate comp-candidate trace-bind comp-bind])
qed

lemma accepted-with-bound-tables-partial-opening-witnesses-Some:
  assumes bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
  obtains fr f-fri-roots f-final dg composition-fri-roots final rest
    trace-openings composition-openings where
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
    composition-fri-roots ≠ []
    accepted-with-partial-trace-openings s (Some (result, final-state))
    fr query-idxs trace-openings
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots) query-idxs
    composition-openings
    partial-trace-table-candidate trace-table trace-openings
    partial-composition-table-candidate composition-table
    composition-openings
    merkle-root-binds-table fr trace-table final-state
    merkle-root-binds-table (hd composition-fri-roots) composition-table
    final-state
proof -
  show ?thesis
proof (rule accepted-with-bound-tables-partial-opening-witnesses[OF bound])
  fix result' final-state' fr f-fri-roots f-final dg

```



```

    composition-fri-roots final rest trace-openings composition-openings
assume out-eq:
  Some (result, final-state) = Some (result', final-state')
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and comp-nonempty: composition-fri-roots  $\neq$  []
  and trace-partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
  and comp-partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots) query-idxs
    composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  and trace-bind:
    merkle-root-binds-table fr trace-table final-state'
  and comp-bind:
    merkle-root-binds-table (hd composition-fri-roots) composition-table
    final-state'
have final-state-eq: final-state' = final-state
  using out-eq by simp
have trace-bind-outer:
  merkle-root-binds-table fr trace-table final-state
  using trace-bind final-state-eq by simp
have comp-bind-outer:
  merkle-root-binds-table (hd composition-fri-roots) composition-table
  final-state
  using comp-bind final-state-eq by simp
show ?thesis
  by (rule that[OF header comp-nonempty trace-partial comp-partial
    trace-candidate comp-candidate trace-bind-outer comp-bind-outer])
qed
qed

end

end

```

```

theory Soundness-Staged-Partial-Query
imports
  Soundness-Staged-Query-Prefix
  Soundness-Bound-Table-Partial-Openings
begin

```

Partial-opening query events for staged soundness.

This layer records query consistency directly from authenticated openings. It deliberately avoids the stronger, generally false claim that sampled openings determine unique complete trace and composition tables.

context *soundness*

begin

lemma *checked-staged-security-with-data-state-accepted-shape-query-chunks:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *outcome:*

Some (((data, attacker-state), result), final-state) ∈ set-dist

(execute

(checked-staged-security-experiment-with-data-state A)

adversary-initial-state)

and *shape:*

accepted-transcript-shape

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

(Some (result, final-state)) as query-idxs

shows

as = staged-alphas data ∧

(∃ raw-idxs.

length raw-idxs = rounds ∧

query-idxs = map (λraw. index (to-nat raw)) raw-idxs ∧

(∀ i < rounds.

verifier-query-round-chunk (query-idxs ! i)

(staged-trace-fri-roots data)

(staged-composition-fri-roots data)

(staged-query-chunks data ! i)) ∧

(∀ i < rounds.

fmlookup (HashMap final-state)

(QueryIndexChallenge i

(state-after-query-chunks

(staged-query-start-hash data)

(staged-query-chunks data i)) =

Some (raw-idxs ! i) ∧

*(∀ idx ∈ set query-idxs. idx < clength * scale))*

proof —

let *?s =*

verifier-state-from-adversary attacker-state

(staged-proof-transcript data)

from *checked-staged-security-experiment-with-data-state-outcomeE*

[OF outcome]

have *builder:*

Some (data, attacker-state) ∈

set-dist

```

      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
    by blast
  have staged-header:
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
  from accepted-transcript-shape-query-chunksE[OF shape]
  obtain result' final-state' fr f-fri-roots f-final dg
    composition-fri-roots final rest raw-idxs query-chunks trailing
  where out-eq:
    Some (result, final-state) = Some (result', final-state')
  and header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
      composition-fri-roots final rest
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-chunks: length query-chunks = rounds
  and concat-chunks: List.concat query-chunks @ trailing = rest
  and chunk-shape:
    ∧ i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
      f-fri-roots composition-fri-roots (query-chunks ! i)
  and lookup:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap final-state')
      (QueryIndexChallenge (PQueryCounter ?s + i)
        (state-after-query-chunks
          (verifier-header-state ?s fr f-fri-roots f-final as dg
            composition-fri-roots final)
            query-chunks i)) =
        Some (raw-idxs ! i)
  and idx-bound:
    ∀ idx ∈ set query-idxs. idx < clength * scale
  by blast
  have header-eq:
    fr = staged-trace-root data ∧
    f-fri-roots = staged-trace-fri-roots data ∧
    f-final = staged-trace-final data ∧
    as = staged-alphas data ∧

```

```

  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF staged-header header]
  by simp
have final-state-eq: final-state' = final-state
  using out-eq by simp
have header-state-eq:
  verifier-header-state ?s fr f-fri-roots f-final as dg
  composition-fri-roots final =
  staged-query-start-hash data
  using header-eq
  unfolding verifier-header-state-def verifier-header-messages-def
  staged-query-start-hash-def staged-composition-fri-start-hash-def
  staged-trace-fri-start-hash-def
  by simp
from checked-staged-transcript-program-query-chunks-match-verifier-lengths
  [OF builder]
obtain staged-query-idxs where
  staged-match:
    staged-query-chunks-match-verifier-lengths data staged-query-idxs
  by blast
have query-idxs-len: length query-idxs = rounds
  using query-idxs-eq len-raw by simp
have staged-match-query-idxs:
  staged-query-chunks-match-verifier-lengths data query-idxs
  by (rule staged-query-chunks-match-verifier-lengths-transfer
    [OF staged-match query-idxs-len])
have parser-chunk-len:
  ∧i. i < rounds ⇒
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
proof -
  fix i
  assume i-bound: i < rounds
  show
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
  using verifier-query-round-chunk-length[OF chunk-shape[OF i-bound]]
  header-eq by simp
qed
have concat-staged:
  List.concat query-chunks @ trailing =
  List.concat (staged-query-chunks data)

```

```

using concat-chunks header-eq by simp
have query-chunks-eq:
  query-chunks = staged-query-chunks data  $\wedge$  trailing = []
by (rule query-chunks-eq-staged-if-matching-lengths
  [OF len-chunks concat-staged parser-chunk-len
  staged-match-query-idxs])
have chunk-shape-staged:
 $\wedge i. i < rounds \implies$ 
  verifier-query-round-chunk (query-idxs ! i)
    (staged-trace-fri-roots data)
    (staged-composition-fri-roots data)
    (staged-query-chunks data ! i)
using chunk-shape header-eq query-chunks-eq by simp
have lookup-staged:
 $\wedge i. i < rounds \implies$ 
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
proof -
fix i
assume i-bound:  $i < rounds$ 
have lookup-i:
  fmlookup (HashMap final-state')
    (QueryIndexChallenge (PQueryCounter ?s + i)
      (state-after-query-chunks
        (verifier-header-state ?s fr f-fri-roots f-final as dg
          composition-fri-roots final)
        query-chunks i)) =
    Some (raw-idxs ! i)
by (rule lookup[OF i-bound])
show
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
using lookup-i final-state-eq header-state-eq query-chunks-eq by simp
qed
show ?thesis
using header-eq len-raw query-idxs-eq chunk-shape-staged lookup-staged
  idx-bound
by blast
qed

```

lemma checked-staged-security-with-data-state-query-bad-round-chunkE:

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *support*:
Some (((*data*, *attacker-state*), *result*), *final-state*) ∈
set-dist
(execute
(checked-staged-security-experiment-with-data-state A)
adversary-initial-state)
and *bad*:
staged-security-with-data-state-query-bad-hit
(Some (((*data*, *attacker-state*), *result*), *final-state*))
obtains *i* trace-table composition-table query-idxs raw-idxs **where**
i < rounds
accepted-with-bound-tables
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (*result*, *final-state*))
trace-table composition-table (staged-alphas data) query-idxs
query-idxs ! i ∈
query-sampling-success-space trace-table composition-table
(staged-alphas data)
length raw-idxs = rounds
query-idxs = map (λraw. index (to-nat raw)) raw-idxs
verifier-query-round-chunk (query-idxs ! i)
(staged-trace-fri-roots data)
(staged-composition-fri-roots data)
(staged-query-chunks data ! i)
fmlookup (HashMap final-state)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data) (staged-query-chunks data) i)) =
Some (raw-idxs ! i)
proof –
let *?s* =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
from *bad*[*unfolded staged-security-with-data-state-query-bad-hit-def*
Let-def]
have *query-bad*: *query-bad ?s (Some (result, final-state))*
by *simp*
from *query-bad-imp-query-index-round-set-hit*[*OF query-bad*]
obtain *i* trace-table composition-table as *query-idxs* **where**
bound:
accepted-with-bound-tables ?s (Some (result, final-state))
trace-table composition-table as query-idxs
and *i-bound*: *i* < rounds
and *query-hit*:
query-idxs ! i ∈
query-sampling-success-space trace-table composition-table as

```

unfolding query-index-round-set-hit-def
  query-index-round-set-hit-at-def
by blast
have shape:
  accepted-transcript-shape ?s (Some (result, final-state))
    as query-idxs
using bound
unfolding accepted-with-bound-tables-def accepted-with-tables-def
by simp
from checked-staged-security-with-data-state-accepted-shape-query-chunks
  [OF wf controlled support shape]
obtain raw-idxs where
  as-eq: as = staged-alphas data
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and chunk-shape:
   $\bigwedge j. j < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! j)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
      (staged-query-chunks data ! j)
and lookup:
   $\bigwedge j. j < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge j
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data j))) =
      Some (raw-idxs ! j)
by blast
have bound-staged:
  accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table (staged-alphas data) query-idxs
using bound as-eq by simp
have query-hit-staged:
  query-idxs ! i  $\in$ 
    query-sampling-success-space trace-table composition-table
      (staged-alphas data)
using query-hit as-eq by simp
show ?thesis
by (rule that[OF i-bound bound-staged query-hit-staged len-raw
  query-idxs-eq chunk-shape[OF i-bound] lookup[OF i-bound]])
qed

```

```

lemma ntimes-outcome-decomp-at:
assumes outcome:
  Some (results, t)  $\in$  set-dist (execute (ntimes m n) s)
and i-bound: i < n

```

shows
 $\exists \text{prefix } x \text{ suffix } s\text{-}i \text{ s-suc.}$
 $\text{Some } (\text{prefix}, s\text{-}i) \in \text{set-dist } (\text{execute } (\text{ntimes } m \ i) \ s) \wedge$
 $\text{Some } (x, s\text{-}suc) \in \text{set-dist } (\text{execute } m \ s\text{-}i) \wedge$
 $\text{Some } (\text{suffix}, t) \in$
 $\text{set-dist } (\text{execute } (\text{ntimes } m \ (n - \text{Suc } i)) \ s\text{-}suc) \wedge$
 $\text{results} = \text{prefix} @ x \# \text{suffix}$
using *outcome i-bound*
proof (*induction n arbitrary: i s results t*)
case 0
then show ?*case* **by** *simp*
next
case (*Suc n*)
from *Suc.prem*s(1) **obtain** *x results' s1* **where**
 $\text{head: Some } (x, s1) \in \text{set-dist } (\text{execute } m \ s)$
and *tail:*
 $\text{Some } (\text{results}', t) \in \text{set-dist } (\text{execute } (\text{ntimes } m \ n) \ s1)$
and *results-eq:* $\text{results} = x \# \text{results}'$
by (*auto elim!: set-dist-bindE*)
show ?*case*
proof (*cases i*)
case 0
have *prefix0:*
 $\text{Some } ([], s) \in \text{set-dist } (\text{execute } (\text{ntimes } m \ i) \ s)$
using 0 **by** *simp*
have *suffix0:*
 $\text{Some } (\text{results}', t) \in$
 $\text{set-dist } (\text{execute } (\text{ntimes } m \ (\text{Suc } n - \text{Suc } i)) \ s1)$
using *tail 0* **by** *simp*
show ?*thesis*
using *prefix0 head suffix0 results-eq*
by (*intro exI[of - []] exI[of - x] exI[of - results']*
 $\text{exI[of - s] exI[of - s1]] \text{ simp}$)
next
case (*Suc j*)
have *j-bound:* $j < n$
using *Suc.prem*s(2) *Suc* **by** *simp*
from *Suc.IH*[*OF tail j-bound*]
obtain *prefix y suffix s-j s-suc* **where**
prefix:
 $\text{Some } (\text{prefix}, s\text{-}j) \in \text{set-dist } (\text{execute } (\text{ntimes } m \ j) \ s1)$
and *round:* $\text{Some } (y, s\text{-}suc) \in \text{set-dist } (\text{execute } m \ s\text{-}j)$
and *suffix:*
 $\text{Some } (\text{suffix}, t) \in$
 $\text{set-dist } (\text{execute } (\text{ntimes } m \ (n - \text{Suc } j)) \ s\text{-}suc)$
and *results'-eq:* $\text{results}' = \text{prefix} @ y \# \text{suffix}$
by *blast*
have *prefix-suc:*
 $\text{Some } (x \# \text{prefix}, s\text{-}j) \in$


```

    set-dist (execute (ntimes m (Suc j)) s)
  unfolding ntimes.simps
  apply (rule set-dist-bindI[OF head])
  apply (rule set-dist-bindI[OF prefix])
  apply simp
  done
have n-minus: Suc n - Suc (Suc j) = n - Suc j
  by simp
have suffix-suc:
  Some (suffix, t) ∈
    set-dist (execute (ntimes m (Suc n - Suc i)) s-suc)
  using suffix n-minus Suc by simp
show ?thesis
  using prefix-suc round suffix-suc results-eq results'-eq Suc
  by (intro exI[of - x # prefix] exI[of - y] exI[of - suffix]
    exI[of - s-j] exI[of - s-suc]) simp
qed
qed

definition partial-query-round-consistent
:: 'f authenticated-opening list list ⇒
  'f authenticated-opening list list ⇒ 'f list ⇒ nat ⇒
  nat ⇒ bool
where
  partial-query-round-consistent trace-openings composition-openings as i idx
  ⇔
    i < rounds ∧
    idx ∈ query-sample-space ∧
    length trace-openings = rounds ∧
    length composition-openings = rounds ∧
    map opening-index (trace-openings ! i) = powers-scaled idx ∧
    map opening-index (composition-openings ! i) =
      [idx, fri-sibling-index (scale * clength) idx] ∧
    composition-openings ! i ≠ [] ∧
    opening-value ((composition-openings ! i) ! 0) =
      cp-eval as (map opening-value (trace-openings ! i))
      (h ^ idx * shift)

definition partial-query-success-indices-at
:: 'f authenticated-opening list list ⇒
  'f authenticated-opening list list ⇒ 'f list ⇒ nat ⇒
  nat set
where
  partial-query-success-indices-at trace-openings composition-openings as i =
    {idx ∈ query-sample-space.
      partial-query-round-consistent trace-openings composition-openings as
      i idx}

lemma partial-query-success-indices-at-subset:

```

partial-query-success-indices-at trace-openings composition-openings as i
 $\subseteq$ *query-sample-space*
unfolding *partial-query-success-indices-at-def* **by** *auto*

lemma *partial-query-success-indices-at-subset-query-sampling-success-space:*

assumes *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings

and *comp-candidate:*

partial-composition-table-candidate composition-table

composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *comp-low: composition-table-low-degree maxDegree composition-table*

and *not-all:*

$\neg$ *all-queries-consistent trace-table composition-table as*

shows

partial-query-success-indices-at trace-openings composition-openings as i

$\subseteq$ *query-sampling-success-space trace-table composition-table as*

proof

fix *idx*

assume *idx-in:*

idx $\in$ partial-query-success-indices-at trace-openings

composition-openings as i

have *round:*

partial-query-round-consistent trace-openings composition-openings as

i idx

using *idx-in* **unfolding** *partial-query-success-indices-at-def* **by** *simp*

have *i-bound: i < rounds*

using *round* **unfolding** *partial-query-round-consistent-def* **by** *simp*

have *idx-sample: idx $\in$ query-sample-space*

using *round* **unfolding** *partial-query-round-consistent-def* **by** *simp*

have *trace-indices:*

map opening-index (trace-openings ! i) = powers-scaled idx

using *round* **unfolding** *partial-query-round-consistent-def* **by** *simp*

have *comp-indices:*

map opening-index (composition-openings ! i) =

*[idx, fri-sibling-index (scale * clength) idx]*

using *round* **unfolding** *partial-query-round-consistent-def* **by** *simp*

have *comp-value-opened:*

opening-value ((composition-openings ! i) ! 0) =

cp-eval as (map opening-value (trace-openings ! i))

*(h ^ idx * shift)*

using *round* **unfolding** *partial-query-round-consistent-def* **by** *simp*

have *trace-len: length trace-table = scale * clength*

using *trace-candidate* **unfolding** *partial-trace-table-candidate-def*

by *simp*

have *comp-len: length composition-table = scale * clength*

using *comp-candidate* **unfolding** *partial-composition-table-candidate-def*

by *simp*

have *trace-values:*

```

map (!) trace-table) (powers-scaled idx) =
  map opening-value (trace-openings ! i)
by (rule partial-trace-table-candidate-query-values
    [OF trace-candidate i-bound trace-indices])
have comp-value:
  composition-table ! idx =
    opening-value ((composition-openings ! i) ! 0)
by (rule partial-composition-table-candidate-query-value
    [OF comp-candidate i-bound comp-indices])
have idx-domain: idx < clength * scale
  by (rule query-sample-space-less-domain[OF idx-sample])
have powers-bound:
   $\forall j \in \text{set } (\text{powers-scaled } \text{idx}). j < \text{length } \text{trace-table}$ 
  using query-sample-space-powers-scaled-bound[OF idx-sample] trace-len
  by (simp add: mult.commute)
have consistent:
  query-consistent-at trace-table composition-table as idx
  unfolding query-consistent-at-def
  using idx-domain trace-len comp-len powers-bound trace-values comp-value
  comp-value-opened
  by (simp add: mult.commute)
show idx  $\in$  query-sampling-success-space trace-table composition-table as
  unfolding query-sampling-success-space-def query-agreement-indices-def
  using idx-sample consistent trace-low comp-low not-all by simp
qed

```

lemma *partial-query-round-consistent-imp-query-consistent-at:*
assumes *trace-candidate:*
 partial-trace-table-candidate trace-table trace-openings
and *comp-candidate:*
 partial-composition-table-candidate composition-table
 composition-openings
and *round:*
 partial-query-round-consistent trace-openings composition-openings
 as i idx
shows *query-consistent-at trace-table composition-table as idx*
proof –
 have *i-bound: i < rounds*
 using *round* **unfolding** *partial-query-round-consistent-def* **by** *simp*
 have *idx-sample: idx $\in$ query-sample-space*
 using *round* **unfolding** *partial-query-round-consistent-def* **by** *simp*
 have *trace-indices:*
 map opening-index (trace-openings ! i) = powers-scaled idx
 using *round* **unfolding** *partial-query-round-consistent-def* **by** *simp*
 have *comp-indices:*
 map opening-index (composition-openings ! i) =
 *[idx, fri-sibling-index (scale * clength) idx]*
 using *round* **unfolding** *partial-query-round-consistent-def* **by** *simp*
 have *comp-value-opened:*

```

opening-value ((composition-openings ! i) ! 0) =
  cp-eval as (map opening-value (trace-openings ! i))
    (h ^ idx * shift)
using round unfolding partial-query-round-consistent-def by simp
have trace-len: length trace-table = scale * clength
using trace-candidate unfolding partial-trace-table-candidate-def
by simp
have comp-len: length composition-table = scale * clength
using comp-candidate unfolding partial-composition-table-candidate-def
by simp
have trace-values:
  map (! trace-table) (powers-scaled idx) =
    map opening-value (trace-openings ! i)
by (rule partial-trace-table-candidate-query-values
  [OF trace-candidate i-bound trace-indices])
have comp-value:
  composition-table ! idx =
    opening-value ((composition-openings ! i) ! 0)
by (rule partial-composition-table-candidate-query-value
  [OF comp-candidate i-bound comp-indices])
have idx-domain: idx < clength * scale
by (rule query-sample-space-less-domain [OF idx-sample])
have powers-bound:
  ∀ j ∈ set (powers-scaled idx). j < length trace-table
using query-sample-space-powers-scaled-bound [OF idx-sample] trace-len
by (simp add: mult.commute)
show ?thesis
unfolding query-consistent-at-def
using idx-domain trace-len comp-len powers-bound trace-values comp-value
  comp-value-opened
by (simp add: mult.commute)
qed

```

```

lemma partial-query-success-indices-at-fraction-bound-if-candidate-low-degree:
assumes trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-low: composition-table-low-degree maxDegree composition-table
and not-all:
  ¬ all-queries-consistent trace-table composition-table as
shows
  nnreal
    (card
      (partial-query-success-indices-at trace-openings composition-openings
        as i)) /
    nnreal (card query-sample-space) ≤ query-error-bound

```

proof –
have *subset*:
partial-query-success-indices-at trace-openings composition-openings as i
 $\subseteq$ *query-sampling-success-space trace-table composition-table as*
by (rule *partial-query-success-indices-at-subset-query-sampling-success-space*
 $[OF \text{ trace-candidate comp-candidate trace-low comp-low not-all}]$)
have *finite-success*:
finite (query-sampling-success-space trace-table composition-table as)
by (rule *finite-subset* $[OF \text{ query-sampling-success-space-subset}$
 $\text{finite-query-sample-space}]$)
have *card-le*:
card
 $(\text{partial-query-success-indices-at trace-openings composition-openings as } i) \leq$
card (query-sampling-success-space trace-table composition-table as)
by (rule *card-mono* $[OF \text{ finite-success subset}]$)
have *nnreal*
 $(\text{card} (\text{partial-query-success-indices-at trace-openings composition-openings as } i)) /$
 $\text{nnreal (card query-sample-space)} \leq$
 $\text{nnreal (card (query-sampling-success-space trace-table composition-table as))} /$
 $\text{nnreal (card query-sample-space)}$
by (rule *nnreal-nat-divide-right-mono* $[OF \text{ card-le}]$)
also have $\dots \leq \text{query-error-bound}$
by (rule *query-sampling-success-space-fraction-bound-query-sample-space*)
finally show *?thesis* .
qed

lemma *partial-query-round-consistentI*:
assumes *i-bound*: $i < \text{rounds}$
and *idx-sample*: $\text{idx} \in \text{query-sample-space}$
and *trace-len*: $\text{length trace-openings} = \text{rounds}$
and *comp-len*: $\text{length composition-openings} = \text{rounds}$
and *trace-indices*:
 $\text{map opening-index (trace-openings ! } i) = \text{powers-scaled idx}$
and *comp-indices*:
 $\text{map opening-index (composition-openings ! } i) =$
 $[\text{idx, fri-sibling-index (scale * clength) idx}]$
and *comp-value*:
 $\text{opening-value ((composition-openings ! } i) ! 0) =$
 $\text{cp-eval as (map opening-value (trace-openings ! } i))$
 $(h \wedge \text{idx} * \text{shift})$
shows
 $\text{partial-query-round-consistent trace-openings composition-openings as}$
 $i \text{ idx}$

proof –
have *comp-round-nonempty*: $\text{composition-openings ! } i \neq []$

```

proof –
  have length (map opening-index (composition-openings ! i)) =
    length [idx, fri-sibling-index (scale * clength) idx]
  using arg-cong[OF comp-indices, of length] by simp
  then have length (composition-openings ! i) = 2
    by simp
  then show ?thesis by auto
qed
show ?thesis
  unfolding partial-query-round-consistent-def
  using i-bound idx-sample trace-len comp-len trace-indices comp-indices
    comp-round-nonempty comp-value
  by simp
qed

lemma partial-query-success-indices-atI:
  assumes partial-query-round-consistent trace-openings composition-openings
    as i idx
  shows
    idx ∈ partial-query-success-indices-at trace-openings
      composition-openings as i
  using assms unfolding partial-query-success-indices-at-def
    partial-query-round-consistent-def by simp

definition partial-query-openings-consistent
  :: 'f authenticated-opening list ⇒
    'f authenticated-opening list ⇒ 'f list ⇒ nat ⇒ bool
where
  partial-query-openings-consistent trace-openings composition-openings as idx
    ⇔
    idx ∈ query-sample-space ∧
    map opening-index trace-openings = powers-scaled idx ∧
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx] ∧
    composition-openings ≠ [] ∧
    opening-value (composition-openings ! 0) =
      cp-eval as (map opening-value trace-openings) (h ^ idx * shift)

lemma partial-query-openings-consistentI:
  assumes idx-sample: idx ∈ query-sample-space
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx
  and comp-indices:
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx]
  and comp-value:
    opening-value (composition-openings ! 0) =
      cp-eval as (map opening-value trace-openings) (h ^ idx * shift)
  shows

```

```

    partial-query-openings-consistent trace-openings composition-openings
    as idx
  proof -
    have comp-nonempty: composition-openings ≠ []
    proof -
      have length (map opening-index composition-openings) =
        length [idx, fri-sibling-index (scale * clength) idx]
      using arg-cong[OF comp-indices, of length] by simp
      then have length composition-openings = 2
      by simp
      then show ?thesis by auto
    qed
  show ?thesis
    unfolding partial-query-openings-consistent-def
    using idx-sample trace-indices comp-indices comp-nonempty comp-value
    by simp
qed

lemma partial-query-round-consistent-single-roundI:
  assumes i-bound: i < rounds
  and consistent:
    partial-query-openings-consistent trace-openings composition-openings
    as idx
  shows
    partial-query-round-consistent
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    as i idx
  proof -
    have idx-sample: idx ∈ query-sample-space
    using consistent unfolding partial-query-openings-consistent-def by simp
    have trace-indices:
      map opening-index trace-openings = powers-scaled idx
    using consistent unfolding partial-query-openings-consistent-def by simp
    have comp-indices:
      map opening-index composition-openings =
        [idx, fri-sibling-index (scale * clength) idx]
    using consistent unfolding partial-query-openings-consistent-def by simp
    have comp-value:
      opening-value (composition-openings ! 0) =
        cp-eval as (map opening-value trace-openings) (h ^ idx * shift)
    using consistent unfolding partial-query-openings-consistent-def by simp
    have comp-nonempty: composition-openings ≠ []
    using consistent unfolding partial-query-openings-consistent-def by simp
    have comp-round-value:
      opening-value
        (((replicate rounds []) [i := composition-openings]) ! i ! 0) =
        cp-eval as
        (map opening-value

```

```

      (((replicate rounds []) [i := trace-openings]) ! i))
    (h ^ idx * shift)
  using i-bound comp-value by simp
show ?thesis
  unfolding partial-query-round-consistent-def
  using i-bound idx-sample trace-indices comp-indices comp-nonempty
    comp-round-value
  by simp
qed

lemma powers-scaled-hd:
  hd (powers-scaled idx) = idx
  unfolding powers-scaled-def
  using powers-pos by (simp add: upt-conv-Cons)

lemma partial-query-openings-consistent-index-unique:
  assumes left:
    partial-query-openings-consistent trace-openings
    composition-openings as idx
  and right:
    partial-query-openings-consistent trace-openings
    composition-openings as idx'
  shows idx = idx'
proof -
  have powers-scaled idx = powers-scaled idx'
  using left right
  unfolding partial-query-openings-consistent-def by simp
  then have hd (powers-scaled idx) = hd (powers-scaled idx')
  by simp
  then show ?thesis
  by (simp add: powers-scaled-hd)
qed

lemma partial-query-openings-consistent-indices-card-le-one:
  card
    {idx ∈ query-sample-space.
      partial-query-openings-consistent trace-openings composition-openings
      as idx} ≤ 1
proof -
  let ?S =
    {idx ∈ query-sample-space.
      partial-query-openings-consistent trace-openings composition-openings
      as idx}
  have finite-S: finite ?S
  by simp
  have unique:  $\bigwedge idx\ idx'. idx \in ?S \implies idx' \in ?S \implies idx = idx'$ 
  by (rule partial-query-openings-consistent-index-unique) auto
  show ?thesis
proof (cases  $\exists idx. idx \in ?S$ )

```



```

case False
have empty:  $?S = \{\}$ 
proof (rule equals0I)
  fix idx
  assume  $idx \in ?S$ 
  then show False
    using False by blast
qed
show ?thesis
  by (subst empty, simp)
next
case True
then obtain idx where idx-in:  $idx \in ?S$ 
  by blast
have subset-single:  $?S \subseteq \{idx\}$ 
  using idx-in unique by blast
have  $\text{card } ?S \leq \text{card } \{idx\}$ 
  by (rule card-mono) (use subset-single in simp-all)
then show ?thesis
  by simp
qed
qed

lemma partial-query-openings-consistent-imp-round-consistent-list-update:
assumes i-bound:  $i < \text{rounds}$ 
and consistent:
  partial-query-openings-consistent trace-openings composition-openings
  as idx
shows
  partial-query-round-consistent
  ((replicate rounds []) [ $i := \text{trace-openings}$ ])
  ((replicate rounds []) [ $i := \text{composition-openings}$ ])
  as i idx

proof –
have idx-sample:  $idx \in \text{query-sample-space}$ 
using consistent
unfolding partial-query-openings-consistent-def by simp
have trace-indices:
  map opening-index
  (((replicate rounds []) [ $i := \text{trace-openings}$ ]) !  $i$ ) =
  powers-scaled idx
using consistent i-bound
unfolding partial-query-openings-consistent-def by simp
have comp-indices:
  map opening-index
  (((replicate rounds []) [ $i := \text{composition-openings}$ ]) !  $i$ ) =
  [idx, fri-sibling-index (scale * clength) idx]
using consistent i-bound
unfolding partial-query-openings-consistent-def by simp

```

```

have comp-value:
  opening-value
  (((replicate rounds []) [i := composition-openings]) ! i) ! 0) =
  cp-eval as
    (map opening-value
      (((replicate rounds []) [i := trace-openings]) ! i))
    (h ^ idx * shift)
  using consistent i-bound
  unfolding partial-query-openings-consistent-def by simp
show ?thesis
  by (rule partial-query-round-consistentI
    [OF i-bound idx-sample])
    (use i-bound trace-indices comp-indices comp-value in simp-all)
qed

lemma partial-query-openings-consistent-indices-subset-round-success-list-update:
  assumes i-bound: i < rounds
  shows
    {idx ∈ query-sample-space.
      partial-query-openings-consistent trace-openings composition-openings
      as idx}
    ⊆
    partial-query-success-indices-at
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      as i
  unfolding partial-query-success-indices-at-def
  using partial-query-openings-consistent-imp-round-consistent-list-update
    [OF i-bound]
  by blast

lemma partial-query-openings-consistent-indices-fraction-bound-if-candidate-low-degree:
  assumes i-bound: i < rounds
  and trace-candidate:
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
  and comp-candidate:
    partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
  and trace-low: trace-table-low-degree trace-table
  and comp-low: composition-table-low-degree maxDegree composition-table
  and not-all:
    ¬ all-queries-consistent trace-table composition-table as
  shows
    nnreal
    (card
      {idx ∈ query-sample-space.
        partial-query-openings-consistent trace-openings
        composition-openings as idx}) /

```

```

      nnreal (card query-sample-space) ≤ query-error-bound
proof -
  let ?trace-openings = (replicate rounds []) [i := trace-openings]
  let ?composition-openings =
    (replicate rounds []) [i := composition-openings]
  let ?flat =
    {idx ∈ query-sample-space.
     partial-query-openings-consistent trace-openings composition-openings
     as idx}
  let ?round =
    partial-query-success-indices-at ?trace-openings
    ?composition-openings as i
  have subset: ?flat ⊆ ?round
  by (rule
      partial-query-openings-consistent-indices-subset-round-success-list-update
      [OF i-bound])
  have finite-round: finite ?round
  by (rule finite-subset[OF partial-query-success-indices-at-subset
      finite-query-sample-space])
  have card-le: card ?flat ≤ card ?round
  by (rule card-mono[OF finite-round subset])
  have nnreal (card ?flat) / nnreal (card query-sample-space) ≤
    nnreal (card ?round) / nnreal (card query-sample-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
  also have ... ≤ query-error-bound
  by (rule partial-query-success-indices-at-fraction-bound-if-candidate-low-degree
      [OF trace-candidate comp-candidate trace-low comp-low not-all])
  finally show ?thesis .
qed

lemma partial-trace-table-candidate-single-roundI:
  assumes i-bound: i < rounds
  and len-table: length trace-table = scale * clength
  and agrees:
    table-agrees-with-authenticated-openings trace-table
    (scale * clength) trace-openings
  shows
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
  unfolding partial-trace-table-candidate-def
proof (intro conjI allI impI)
  show length trace-table = scale * clength
  by (rule len-table)
next
  show length ((replicate rounds []) [i := trace-openings]) = rounds
  using i-bound by simp
next
  fix j
  assume j-bound: j < rounds

```

```

show
  table-agrees-with-authenticated-openings trace-table
    (scale * clength)
    (((replicate rounds []) [i := trace-openings]) ! j)
proof (cases j = i)
  case True
    then show ?thesis
      using agrees i-bound by simp
  next
    case False
    then show ?thesis
      using j-bound len-table
      unfolding table-agrees-with-authenticated-openings-def
      by simp
qed
qed

lemma partial-composition-table-candidate-single-roundI:
  assumes i-bound: i < rounds
    and len-table: length composition-table = scale * clength
    and agrees:
      table-agrees-with-authenticated-openings composition-table
        (scale * clength) composition-openings
  shows
    partial-composition-table-candidate composition-table
      ((replicate rounds []) [i := composition-openings])
  unfolding partial-composition-table-candidate-def
proof (intro conjI allI impI)
  show length composition-table = scale * clength
    by (rule len-table)
next
  show length ((replicate rounds []) [i := composition-openings]) =
    rounds
    using i-bound by simp
next
  fix j
  assume j-bound: j < rounds
  show
    table-agrees-with-authenticated-openings composition-table
      (scale * clength)
      (((replicate rounds []) [i := composition-openings]) ! j)
proof (cases j = i)
  case True
    then show ?thesis
      using agrees i-bound by simp
  next
    case False
    then show ?thesis
      using j-bound len-table

```

```

    unfolding table-agrees-with-authenticated-openings-def
    by simp
qed
qed

lemma partial-query-openings-consistent-indices-fraction-bound-if-tables-agree-low-degree:
  assumes i-bound:  $i < \text{rounds}$ 
    and trace-len:  $\text{length trace-table} = \text{scale} * \text{clength}$ 
    and comp-len:  $\text{length composition-table} = \text{scale} * \text{clength}$ 
    and trace-agrees:
      table-agrees-with-authenticated-openings trace-table
        (scale * clength) trace-openings
    and comp-agrees:
      table-agrees-with-authenticated-openings composition-table
        (scale * clength) composition-openings
    and trace-low: trace-table-low-degree trace-table
    and comp-low: composition-table-low-degree maxDegree composition-table
    and not-all:
       $\neg \text{all-queries-consistent trace-table composition-table as}$ 
  shows
    nnreal
      (card
        {idx  $\in$  query-sample-space.
          partial-query-openings-consistent trace-openings
            composition-openings as idx}) /
        nnreal (card query-sample-space)  $\leq$  query-error-bound
  proof -
    have trace-candidate:
      partial-trace-table-candidate trace-table
        ((replicate rounds []) [i := trace-openings])
    by (rule partial-trace-table-candidate-single-roundI
        [OF i-bound trace-len trace-agrees])
    have comp-candidate:
      partial-composition-table-candidate composition-table
        ((replicate rounds []) [i := composition-openings])
    by (rule partial-composition-table-candidate-single-roundI
        [OF i-bound comp-len comp-agrees])
    show ?thesis
    by (rule
        partial-query-openings-consistent-indices-fraction-bound-if-candidate-low-degree
        [OF i-bound trace-candidate comp-candidate trace-low comp-low
          not-all])
  qed

lemma table-agrees-with-authenticated-openings-values:
  assumes agrees:
    table-agrees-with-authenticated-openings table len openings
    and indices: map opening-index openings = idxs
  shows map (!) table) idxs = map opening-value openings

```

```

proof (rule nth-equalityI)
  show length (map (!) table) idxs = length (map opening-value openings)
    by (simp add: indices[symmetric])
next
  fix i
  assume i-bound:  $i < \text{length } (\text{map } (!) \text{ table}) \text{ idxs}$ 
  have i-openings:  $i < \text{length } \text{openings}$ 
    using i-bound by (simp add: indices[symmetric])
  have opn-in:  $\text{openings } ! i \in \text{set } \text{openings}$ 
    by (rule nth-mem[OF i-openings])
  have table-value:
     $\text{table } ! \text{ opening-index } (\text{openings } ! i) = \text{opening-value } (\text{openings } ! i)$ 
    using agrees opn-in
    unfolding table-agrees-with-authenticated-openings-def by auto
  have idxs ! i = opening-index (openings ! i)
    using i-bound by (simp add: indices[symmetric])
  then show  $\text{map } (!) \text{ table } \text{idxs } ! i = \text{map } \text{opening-value } \text{openings } ! i$ 
    using table-value i-bound i-openings by simp
qed

```

```

lemma partial-authenticated-table-agrees-with-bound-table-if-clean:
  assumes bind: merkle-root-binds-table rt table final-state
    and len-table:  $\text{length } \text{table} = \text{scale} * \text{clength}$ 
    and partial:
       $\text{partial-authenticated-table } \text{rt } (\text{scale} * \text{clength}) \text{ openings } \text{final-state}$ 
    and clean:  $\neg \text{hash-map-output-collision } \text{final-state}$ 
  shows
     $\text{table-agrees-with-authenticated-openings } \text{table } (\text{scale} * \text{clength})$ 
     $\text{openings}$ 

```

```

proof –
  obtain n where len-pow:  $\text{clength} * \text{scale} = 2^n$ 
    using eval-domain-length-power by blast
  have table-pow:  $\text{length } \text{table} = 2^n$ 
    using len-table len-pow by (simp add: mult.commute)
  have opening-agrees:
     $\forall \text{opn} \in \text{set } \text{openings}.$ 
       $\text{opening-index } \text{opn} < \text{scale} * \text{clength} \wedge$ 
       $\text{table } ! \text{ opening-index } \text{opn} = \text{opening-value } \text{opn}$ 
  proof
    fix opn
    assume opn-in:  $\text{opn} \in \text{set } \text{openings}$ 
    have root:  $\text{opening-root } \text{opn} = \text{rt}$ 
      using partial opn-in unfolding partial-authenticated-table-def
      by blast
    have len:  $\text{opening-length } \text{opn} = \text{length } \text{table}$ 
      using partial opn-in len-table
      unfolding partial-authenticated-table-def by simp
    have auth:  $\text{authenticated-opening-in } \text{final-state } \text{opn}$ 
      using partial opn-in unfolding partial-authenticated-table-def

```

```

    by blast
  have agreement:
    opening-index opn < length table ∧
    table ! opening-index opn = opening-value opn
  by (rule merkle-bound-table-agrees-with-authenticated-opening-if-clean
    [OF bind table-pow auth root len clean])
  then show
    opening-index opn < scale * clength ∧
    table ! opening-index opn = opening-value opn
  using len-table by simp
qed
show ?thesis
  unfolding table-agrees-with-authenticated-openings-def
  using len-table opening-agrees by simp
qed

lemma partial-query-openings-consistent-from-sampling-success:
  assumes trace-bind:
    merkle-root-binds-table fr trace-table final-state
  and comp-bind:
    merkle-root-binds-table composition-root composition-table
    final-state
  and trace-len: length trace-table = scale * clength
  and comp-len: length composition-table = scale * clength
  and trace-auth:
    partial-authenticated-table fr (scale * clength) trace-openings
    final-state
  and comp-auth:
    partial-authenticated-table composition-root (scale * clength)
    composition-openings final-state
  and clean: ¬ hash-map-output-collision final-state
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx
  and comp-indices:
    map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
  and success:
    idx ∈ query-sampling-success-space trace-table composition-table as
  shows
    partial-query-openings-consistent trace-openings composition-openings
    as idx
proof -
  have idx-sample: idx ∈ query-sample-space
  using success
  unfolding query-sampling-success-space-def query-agreement-indices-def
  by (auto split: if-splits)
  have consistent: query-consistent-at trace-table composition-table as idx
  using success
  unfolding query-sampling-success-space-def query-agreement-indices-def

```

```

  by (auto split: if-splits)
have trace-agrees:
  table-agrees-with-authenticated-openings trace-table
  (scale * clength) trace-openings
  by (rule partial-authenticated-table-agrees-with-bound-table-if-clean
      [OF trace-bind trace-len trace-auth clean])
have comp-agrees:
  table-agrees-with-authenticated-openings composition-table
  (scale * clength) composition-openings
  by (rule partial-authenticated-table-agrees-with-bound-table-if-clean
      [OF comp-bind comp-len comp-auth clean])
have trace-values:
  map (! trace-table) (powers-scaled idx) =
    map opening-value trace-openings
  by (rule table-agrees-with-authenticated-openings-values
      [OF trace-agrees trace-indices])
have comp-len-openings: length composition-openings = 2
  using arg-cong[OF comp-indices, of length] by simp
have first-in: composition-openings ! 0 ∈ set composition-openings
  by (rule nth-mem) (use comp-len-openings in simp)
have first-index: opening-index (composition-openings ! 0) = idx
proof -
  have map opening-index composition-openings ! 0 = idx
    using comp-indices by simp
  then show ?thesis
    using comp-len-openings by simp
qed
have composition-value:
  composition-table ! idx = opening-value (composition-openings ! 0)
  using comp-agrees first-in first-index
  unfolding table-agrees-with-authenticated-openings-def by auto
have opened-value:
  opening-value (composition-openings ! 0) =
    cp-eval as (map opening-value trace-openings) (h ^ idx * shift)
  using consistent trace-values composition-value
  unfolding query-consistent-at-def by simp
show ?thesis
  by (rule partial-query-openings-consistentI
      [OF idx-sample trace-indices comp-indices opened-value])
qed

lemma partial-query-round-consistent-from-sampling-success:
  assumes trace-partial:
    accepted-with-partial-trace-openings s out fr query-idxs
    trace-openings
  and comp-partial:
    accepted-with-partial-composition-openings s out composition-root
    query-idxs composition-openings
  and trace-candidate:

```



```

    partial-trace-table-candidate trace-table trace-openings
  and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  and i-bound:  $i < \text{rounds}$ 
  and success:
    query-idxs !  $i \in$ 
    query-sampling-success-space trace-table composition-table as
  shows
    partial-query-round-consistent trace-openings composition-openings as  $i$ 
    (query-idxs !  $i$ )
proof -
  let ?idx = query-idxs !  $i$ 
  have idx-sample: ?idx  $\in$  query-sample-space
  using success
  unfolding query-sampling-success-space-def query-agreement-indices-def
  by (auto split: if-splits)
  have consistent: query-consistent-at trace-table composition-table as ?idx
  using success
  unfolding query-sampling-success-space-def query-agreement-indices-def
  by (auto split: if-splits)
  have trace-len: length trace-openings = rounds
  by (rule accepted-with-partial-trace-openings-shapes(2)
    [OF trace-partial])
  have comp-len: length composition-openings = rounds
  by (rule accepted-with-partial-composition-openings-shapes(2)
    [OF comp-partial])
  have trace-indices:
    map opening-index (trace-openings !  $i$ ) = powers-scaled ?idx
  by (rule accepted-with-partial-trace-openings-shapes(3)
    [OF trace-partial i-bound])
  have comp-indices:
    map opening-index (composition-openings !  $i$ ) =
    [?idx, fri-sibling-index (scale * clength) ?idx]
  by (rule accepted-with-partial-composition-openings-shapes(3)
    [OF comp-partial i-bound])
  have trace-values:
    map ((! trace-table) (powers-scaled ?idx)) =
    map opening-value (trace-openings !  $i$ )
  by (rule partial-trace-table-candidate-query-values
    [OF trace-candidate i-bound trace-indices])
  have composition-value:
    composition-table ! ?idx =
    opening-value ((composition-openings !  $i$ ) ! 0)
  by (rule partial-composition-table-candidate-query-value
    [OF comp-candidate i-bound comp-indices])
  have opened-value:
    opening-value ((composition-openings !  $i$ ) ! 0) =
    cp-eval as (map opening-value (trace-openings !  $i$ ))

```

```

      (h  $\wedge$  ?idx * shift)
    using consistent trace-values composition-value
    unfolding query-consistent-at-def by simp
  show ?thesis
  by (rule partial-query-round-consistentI
      [OF i-bound idx-sample trace-len comp-len trace-indices comp-indices
        opened-value])
qed

```

definition *query-header-supported-partial-opening-witnesses*

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$ 
  'f list  $\Rightarrow$  'f  $\Rightarrow$ 
  ('f authenticated-opening list  $\times$ 
   'f authenticated-opening list) set

```

where

```

query-header-supported-partial-opening-witnesses s fr f-fri-roots
  f-final as dg composition-fri-roots final =
  {(trace-openings, composition-openings).
   composition-fri-roots  $\neq$  []  $\wedge$ 
   ( $\exists$  result final-state rest.
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)  $\wedge$ 
    partial-authenticated-table fr (scale * clength) trace-openings
      final-state  $\wedge$ 
    partial-authenticated-table (hd composition-fri-roots)
      (scale * clength) composition-openings final-state  $\wedge$ 
    verifier-header-transcript s fr f-fri-roots f-final as dg
      composition-fri-roots final rest)}}

```

lemma *query-header-supported-partial-opening-witnessesE*:

assumes

```

(trace-openings, composition-openings)  $\in$ 
  query-header-supported-partial-opening-witnesses s fr f-fri-roots
  f-final as dg composition-fri-roots final

```

obtains result final-state rest **where**

```

composition-fri-roots  $\neq$  []
Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
partial-authenticated-table fr (scale * clength) trace-openings
  final-state
partial-authenticated-table (hd composition-fri-roots)
  (scale * clength) composition-openings final-state
verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest

```

using *assms*

unfolding *query-header-supported-partial-opening-witnesses-def*

by *blast*

definition *query-header-supported-partial-opening-success-indices-at*

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 

```

'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$
 'f list $\Rightarrow$ 'f $\Rightarrow$ nat $\Rightarrow$ nat set

where

query-header-supported-partial-opening-success-indices-at s fr
 f-fri-roots f-final as dg composition-fri-roots final i =
 {idx $\in$ query-sample-space.
 $\exists$ trace-openings composition-openings.
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses s fr
 f-fri-roots f-final as dg composition-fri-roots final $\wedge$
 partial-query-openings-consistent trace-openings composition-openings
 as idx}

lemma query-header-supported-partial-opening-success-indices-at-subset:

query-header-supported-partial-opening-success-indices-at s fr
 f-fri-roots f-final as dg composition-fri-roots final i
 $\subseteq$ query-sample-space

unfolding query-header-supported-partial-opening-success-indices-at-def
by auto

lemma query-header-supported-partial-opening-success-indices-atE:

assumes

idx $\in$ query-header-supported-partial-opening-success-indices-at s fr
 f-fri-roots f-final as dg composition-fri-roots final i

obtains trace-openings composition-openings **where**

idx $\in$ query-sample-space
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses s fr
 f-fri-roots f-final as dg composition-fri-roots final
 partial-query-openings-consistent trace-openings composition-openings
 as idx

using assms

unfolding query-header-supported-partial-opening-success-indices-at-def
by blast

lemma query-header-supported-partial-opening-success-indices-at-UN:

query-header-supported-partial-opening-success-indices-at s fr
 f-fri-roots f-final as dg composition-fri-roots final i =
 ($\bigcup$ (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses s fr
 f-fri-roots f-final as dg composition-fri-roots final.
 {idx $\in$ query-sample-space.
 partial-query-openings-consistent trace-openings
 composition-openings as idx}))

unfolding query-header-supported-partial-opening-success-indices-at-def
by auto

lemma query-header-supported-partial-opening-success-indices-at-subset-if-witnesses:

assumes cover:

$\wedge$ *trace-openings composition-openings*.
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses s fr
f-fri-roots f-final as dg composition-fri-roots final ⇒
{idx ∈ query-sample-space.
partial-query-openings-consistent trace-openings
composition-openings as idx} ⊆ B
shows
query-header-supported-partial-opening-success-indices-at s fr
f-fri-roots f-final as dg composition-fri-roots final i ⊆ B
unfolding *query-header-supported-partial-opening-success-indices-at-UN*
using *cover by auto*

lemma *query-header-supported-partial-opening-success-indices-atI-from-bound-tables:*

assumes *outcome:*
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *header:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *comp-nonempty:* *composition-fri-roots ≠ []*
and *trace-bind:*
merkle-root-binds-table fr trace-table final-state
and *comp-bind:*
merkle-root-binds-table (hd composition-fri-roots) composition-table
final-state
and *trace-len:* *length trace-table = scale * clength*
and *comp-len:* *length composition-table = scale * clength*
and *trace-auth:*
*partial-authenticated-table fr (scale * clength) trace-openings*
final-state
and *comp-auth:*
partial-authenticated-table (hd composition-fri-roots)
*(scale * clength) composition-openings final-state*
and *clean:* $\neg$ *hash-map-output-collision final-state*
and *trace-indices:*
map opening-index trace-openings = powers-scaled idx
and *comp-indices:*
map opening-index composition-openings =
*[idx, fri-sibling-index (scale * clength) idx]*
and *success:*
idx ∈ query-sampling-success-space trace-table composition-table as
shows
idx ∈
query-header-supported-partial-opening-success-indices-at s fr
f-fri-roots f-final as dg composition-fri-roots final i
proof –
have *consistent:*
partial-query-openings-consistent trace-openings composition-openings
as idx

by (rule partial-query-openings-consistent-from-sampling-success
 [OF trace-bind comp-bind trace-len comp-len trace-auth comp-auth
 clean trace-indices comp-indices success])
 have witness:
 (trace-openings, composition-openings) ∈
 query-header-supported-partial-opening-witnesses s fr f-fri-roots
 f-final as dg composition-fri-roots final
 unfolding query-header-supported-partial-opening-witnesses-def
 using comp-nonempty outcome trace-auth comp-auth header
 by (auto split: prod.splits)
 have idx-sample: idx ∈ query-sample-space
 using consistent
 unfolding partial-query-openings-consistent-def by simp
 show ?thesis
 unfolding query-header-supported-partial-opening-success-indices-at-def
 by (intro CollectI conjI exI[of - trace-openings]
 exI[of - composition-openings])
 (use idx-sample witness consistent in simp-all)
 qed

lemma query-header-supported-partial-opening-success-indices-atI-from-partials:
 assumes outcome:
 Some (result, final-state) ∈ set-dist (execute verify-monad s)
 and trace-partial:
 accepted-with-partial-trace-openings s
 (Some (result, final-state)) fr query-idxs trace-openings
 and comp-partial:
 accepted-with-partial-composition-openings s
 (Some (result, final-state)) composition-root query-idxs
 composition-openings
 and trace-candidate:
 partial-trace-table-candidate trace-table trace-openings
 and comp-candidate:
 partial-composition-table-candidate composition-table
 composition-openings
 and header:
 verifier-header-transcript s fr f-fri-roots f-final as dg
 composition-fri-roots final rest
 and comp-nonempty: composition-fri-roots ≠ []
 and root-eq: composition-root = hd composition-fri-roots
 and i-bound: i < rounds
 and success:
 query-idxs ! i ∈
 query-sampling-success-space trace-table composition-table as
 shows
 query-idxs ! i ∈
 query-header-supported-partial-opening-success-indices-at s fr
 f-fri-roots f-final as dg composition-fri-roots final i
 proof —

```

let ?trace-openings = trace-openings ! i
let ?composition-openings = composition-openings ! i
have consistent-round:
  partial-query-round-consistent trace-openings composition-openings as
    i (query-idxs ! i)
  by (rule partial-query-round-consistent-from-sampling-success
    [OF trace-partial comp-partial trace-candidate comp-candidate
      i-bound success])
have consistent:
  partial-query-openings-consistent ?trace-openings
    ?composition-openings as (query-idxs ! i)
  unfolding partial-query-openings-consistent-def
  using consistent-round
  unfolding partial-query-round-consistent-def
  by simp
have trace-table:
  partial-authenticated-table fr (scale * clength) ?trace-openings
    final-state
  using trace-partial i-bound
  unfolding accepted-with-partial-trace-openings-def by blast
have comp-table:
  partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) ?composition-openings final-state
  using comp-partial i-bound root-eq
  unfolding accepted-with-partial-composition-openings-def by blast
have witness:
  (?trace-openings, ?composition-openings) ∈
    query-header-supported-partial-opening-witnesses s fr f-fri-roots
    f-final as dg composition-fri-roots final
  unfolding query-header-supported-partial-opening-witnesses-def
  using comp-nonempty outcome trace-table comp-table header
  by auto
have idx-sample:
  query-idxs ! i ∈ query-sample-space
  using consistent unfolding partial-query-openings-consistent-def by simp
show ?thesis
  unfolding query-header-supported-partial-opening-success-indices-at-def
  by (intro CollectI conjI exI[of - ?trace-openings]
    exI[of - ?composition-openings])
    (use idx-sample witness consistent in simp-all)
qed

```

```

lemma partial-query-round-consistent-from-all-queries-consistent:
  assumes trace-partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
  and comp-partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) composition-root query-idxs

```

```

    composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  and i-bound:  $i < \text{rounds}$ 
  and idx-sample:  $\text{query-idxs} ! i \in \text{query-sample-space}$ 
  and all-queries:
    all-queries-consistent trace-table composition-table as
shows
  partial-query-round-consistent trace-openings composition-openings as
  i ( $\text{query-idxs} ! i$ )
proof -
  let ?idx =  $\text{query-idxs} ! i$ 
  have trace-len:  $\text{length trace-openings} = \text{rounds}$ 
    using trace-partial unfolding accepted-with-partial-trace-openings-def
    by blast
  have comp-len:  $\text{length composition-openings} = \text{rounds}$ 
    using comp-partial
    unfolding accepted-with-partial-composition-openings-def by blast
  have trace-indices:
    map opening-index ( $\text{trace-openings} ! i$ ) = powers-scaled ?idx
    using trace-partial i-bound
    unfolding accepted-with-partial-trace-openings-def by blast
  have comp-indices:
    map opening-index ( $\text{composition-openings} ! i$ ) =
    [ $?idx$ , fri-sibling-index ( $\text{scale} * \text{clength}$ ) ?idx]
    using comp-partial i-bound
    unfolding accepted-with-partial-composition-openings-def by blast
  have qc:
    query-consistent-at trace-table composition-table as ?idx
    using all-queries idx-sample unfolding all-queries-consistent-def
    by simp
  have trace-values:
    map ((!) trace-table) (powers-scaled ?idx) =
    map opening-value ( $\text{trace-openings} ! i$ )
    by (rule partial-trace-table-candidate-query-values
        [OF trace-candidate i-bound trace-indices])
  have comp-value:
    composition-table ! ?idx =
    opening-value (( $\text{composition-openings} ! i$ ) ! 0)
    by (rule partial-composition-table-candidate-query-value
        [OF comp-candidate i-bound comp-indices])
  have opened-value:
    opening-value (( $\text{composition-openings} ! i$ ) ! 0) =
    cp-eval as (map opening-value ( $\text{trace-openings} ! i$ ))
    ( $h \wedge ?idx * \text{shift}$ )
    using qc trace-values comp-value unfolding query-consistent-at-def

```

```

    by simp
  show ?thesis
    by (rule partial-query-round-consistentI
        [OF i-bound idx-sample trace-len comp-len trace-indices
            comp-indices opened-value])
qed

lemma query-header-supported-partial-opening-success-indices-atI-from-partials-consistent:
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and trace-partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
  and comp-partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) composition-root query-idxs
    composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and comp-nonempty: composition-fri-roots ≠ []
  and root-eq: composition-root = hd composition-fri-roots
  and i-bound: i < rounds
  and idx-sample: query-idxs ! i ∈ query-sample-space
  and all-queries:
    all-queries-consistent trace-table composition-table as
  shows
    query-idxs ! i ∈
    query-header-supported-partial-opening-success-indices-at s fr
    f-fri-roots f-final as dg composition-fri-roots final i
proof -
  let ?trace-openings = trace-openings ! i
  let ?composition-openings = composition-openings ! i
  have consistent-round:
    partial-query-round-consistent trace-openings composition-openings as
    i (query-idxs ! i)
  by (rule partial-query-round-consistent-from-all-queries-consistent
      [OF trace-partial comp-partial trace-candidate comp-candidate
          i-bound idx-sample all-queries])
  have consistent:
    partial-query-openings-consistent ?trace-openings
    ?composition-openings as (query-idxs ! i)
  unfolding partial-query-openings-consistent-def
  using consistent-round

```



```

unfolding partial-query-round-consistent-def
by simp
have trace-table:
  partial-authenticated-table fr (scale * clength) ?trace-openings
    final-state
using trace-partial i-bound
unfolding accepted-with-partial-trace-openings-def by blast
have comp-table:
  partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) ?composition-openings final-state
using comp-partial i-bound root-eq
unfolding accepted-with-partial-composition-openings-def by blast
have witness:
  (?trace-openings, ?composition-openings) ∈
    query-header-supported-partial-opening-witnesses s fr f-fri-roots
      f-final as dg composition-fri-roots final
unfolding query-header-supported-partial-opening-witnesses-def
using comp-nonempty outcome trace-table comp-table header
by auto
show ?thesis
unfolding query-header-supported-partial-opening-success-indices-at-def
by (intro CollectI conjI exI[of - ?trace-openings]
      exI[of - ?composition-openings])
      (use idx-sample witness consistent in simp-all)
qed

definition staged-security-with-data-state-query-partial-opening-hit-at
  :: nat ⇒
    (('f staged-proof-data × 'f protocol-channel) × unit list) ×
      'f protocol-channel) option ⇒ bool
where
  staged-security-with-data-state-query-partial-opening-hit-at i out
    ⇔
      (case out of
        None ⇒ False
      | Some (((data, attacker-state), -), final-state) ⇒
        (let s =
          verifier-state-from-adversary attacker-state
            (staged-proof-transcript data);
          B =
            query-header-supported-partial-opening-success-indices-at s
              (staged-trace-root data)
              (staged-trace-fri-roots data)
              (staged-trace-final data)
              (staged-alphas data)
              (staged-degree data)
              (staged-composition-fri-roots data)
              (staged-composition-final data)
          i

```

in
 (∃ *raw*.
 i < *rounds* ∧
 fmlookup (*HashMap final-state*)
 (*QueryIndexChallenge i*
 (*state-after-query-chunks*
 (*staged-query-start-hash data*)
 (*staged-query-chunks data*) *i*)) =
 Some raw ∧
 index (*to-nat raw*) ∈ *B*)))

definition *staged-security-with-data-state-query-partial-opening-hit*
 :: (((*'f staged-proof-data* × *'f protocol-channel*) × *unit list*) ×
 'f protocol-channel) *option* ⇒ *bool*

where

staged-security-with-data-state-query-partial-opening-hit out ⟷
 (∃ *i* < *rounds*.
 staged-security-with-data-state-query-partial-opening-hit-at i out)

lemma *staged-security-with-data-state-query-partial-opening-hit-atD*:
assumes

staged-security-with-data-state-query-partial-opening-hit-at i out

shows *i* < *rounds*

using *assms*

unfolding *staged-security-with-data-state-query-partial-opening-hit-at-def*
by (*cases out*; *auto split*: *prod.splits*)

lemma *staged-security-with-data-state-query-partial-opening-hit-atE*:
assumes

staged-security-with-data-state-query-partial-opening-hit-at i
 (*Some* (((*data*, *attacker-state*), *result*), *final-state*))

obtains *raw* **where**

i < *rounds*
fmlookup (*HashMap final-state*)
 (*QueryIndexChallenge i*
 (*state-after-query-chunks*
 (*staged-query-start-hash data*)
 (*staged-query-chunks data*) *i*)) =
Some raw

index (*to-nat raw*) ∈
query-header-supported-partial-opening-success-indices-at
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)

```

      (staged-composition-final data)
    i
  using assms
  unfolding staged-security-with-data-state-query-partial-opening-hit-at-def
    Let-def
  by auto

lemma staged-security-with-data-state-query-partial-opening-hitI:
  assumes i < rounds
    and staged-security-with-data-state-query-partial-opening-hit-at i out
  shows staged-security-with-data-state-query-partial-opening-hit out
  using assms
  unfolding staged-security-with-data-state-query-partial-opening-hit-def
  by blast

lemma staged-security-with-data-state-query-partial-opening-hit-imp-round-hit:
  assumes hit: staged-security-with-data-state-query-partial-opening-hit out
  shows  $\exists i \in \{..<rounds\}.$ 
    staged-security-with-data-state-query-partial-opening-hit-at i out
  using assms
  unfolding staged-security-with-data-state-query-partial-opening-hit-def
  by blast

lemma checked-staged-security-with-data-state-query-partial-opening-hit-bound-from-rounds:
  assumes round-bound:
     $\bigwedge i. i < rounds \implies$ 
      wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-partial-opening-hit-at i)
      adversary-initial-state  $\leq C i$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-partial-opening-hit
      adversary-initial-state  $\leq (\sum i < rounds. C i)$ 
proof -
  have wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-partial-opening-hit
    adversary-initial-state  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
      ( $\lambda out. \exists i \in \{..<rounds\}.$ 
        staged-security-with-data-state-query-partial-opening-hit-at i out)
    adversary-initial-state
  by (rule wp-event-mono)
    (rule staged-security-with-data-state-query-partial-opening-hit-imp-round-hit)
  also have ...  $\leq (\sum i < rounds. C i)$ 
  by (rule wp-event-finite-UN-bound)
    (simp-all add: round-bound)
  finally show ?thesis .
qed

```

lemma *checked-staged-security-with-data-state-partial-opening-hit-bound-from-index-set:*

assumes *cover:*

$\bigwedge \text{data attacker-state } i.$

$i < \text{rounds} \implies$

query-header-supported-partial-opening-success-indices-at

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

(staged-trace-root data)

(staged-trace-fri-roots data)

(staged-trace-final data)

(staged-alphas data)

(staged-degree data)

(staged-composition-fri-roots data)

(staged-composition-final data)

$i \subseteq B$

and *index-bound:*

wp-event (checked-staged-security-experiment-with-data A)

(staged-security-with-data-query-index-set-hit B)

adversary-initial-state $\leq C$

shows

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-query-partial-opening-hit

adversary-initial-state $\leq C$

proof –

let *?projected =*

$\lambda \text{out. case out of None} \implies$

staged-security-with-data-query-index-set-hit B None

$| \text{Some } (((\text{data}, -), \text{result}), t) \implies$

staged-security-with-data-query-index-set-hit B

(Some ((data, result), t))

have *event-le:*

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-query-partial-opening-hit

adversary-initial-state $\leq$

wp-event (checked-staged-security-experiment-with-data-state A)

?projected adversary-initial-state

proof (rule *wp-event-mono*)

fix *out*

assume *hit:*

staged-security-with-data-state-query-partial-opening-hit out

show *?projected out*

proof (cases *out*)

case *None*

then show *?thesis*

using *hit*

unfolding *staged-security-with-data-state-query-partial-opening-hit-def*

staged-security-with-data-state-query-partial-opening-hit-at-def

by *simp*

next

```

case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
let ?B =
   $\lambda i.$  query-header-supported-partial-opening-success-indices-at ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    i
from hit[unfolded out-eq
  staged-security-with-data-state-query-partial-opening-hit-def
  staged-security-with-data-state-query-partial-opening-hit-at-def
  Let-def]
obtain i raw where i-bound: i < rounds
and lookup:
  fmlookup (HashMap final-state)
  (QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data)
      (staged-query-chunks data) i)) =
    Some raw
and raw-hit: index (to-nat raw) ∈ ?B i
by auto
have raw-in-B: index (to-nat raw) ∈ B
proof –
  have ?B i ⊆ B
  by (rule cover[OF i-bound])
  then show ?thesis
  using raw-hit by blast
qed
show ?thesis
  unfolding out-eq staged-security-with-data-query-index-set-hit-def
  using i-bound lookup raw-in-B by auto
qed
qed
also have
wp-event (checked-staged-security-experiment-with-data-state A)
?projected adversary-initial-state =
wp-event (checked-staged-security-experiment-with-data A)
(staged-security-with-data-query-index-set-hit B)
adversary-initial-state

```

```

using checked-staged-security-experiment-with-data-event-from-data-state
  [of A staged-security-with-data-query-index-set-hit B]
by simp
also have  $\dots \leq C$ 
  by (rule index-bound)
finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-partial-opening-hit-bound-from-index-set-controlled:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and cover:
 $\bigwedge$  data attacker-state i.
   $i < \text{rounds} \implies$ 
    query-header-supported-partial-opening-success-indices-at
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
     $i \subseteq B$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-partial-opening-hit
    adversary-initial-state  $\leq$ 
    staged-phase-target-error (query-index-raw-preimage B)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
proof (rule
  checked-staged-security-with-data-state-partial-opening-hit-bound-from-index-set)
fix data attacker-state i
assume i-bound: i < rounds
show
  query-header-supported-partial-opening-success-indices-at
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
     $i \subseteq B$ 
by (rule cover[OF i-bound])

```

next
show
 $wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data \ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}query\text{-}index\text{-}set\text{-}hit \ B)$
 $adversary\text{-}initial\text{-}state \leq$
 $staged\text{-}phase\text{-}target\text{-}error \ (query\text{-}index\text{-}raw\text{-}preimage \ B)$
 $(staged\text{-}attacker\text{-}query\text{-}budget \ budgets +$
 $staged\text{-}challenge\text{-}query\text{-}budget)$
by $(rule \ checked\text{-}staged\text{-}security\text{-}with\text{-}data\text{-}query\text{-}index\text{-}set\text{-}hit\text{-}bound$
 $[OF \ wf \ controlled])$
qed

lemma *checked-staged-security-with-data-state-partial-opening-hit-bound-from-fixed-query-error-cover:*

assumes $wf: staged\text{-}budget\text{-}wellformed \ budgets$

and $controlled: staged\text{-}adversary\text{-}controlled \ budgets \ A$

and $cover:$

$\bigwedge data \ attacker\text{-}state \ i.$

$i < rounds \implies$

$query\text{-}header\text{-}supported\text{-}partial\text{-}opening\text{-}success\text{-}indices\text{-}at$

$(verifier\text{-}state\text{-}from\text{-}adversary \ attacker\text{-}state$

$(staged\text{-}proof\text{-}transcript \ data))$

$(staged\text{-}trace\text{-}root \ data)$

$(staged\text{-}trace\text{-}fri\text{-}roots \ data)$

$(staged\text{-}trace\text{-}final \ data)$

$(staged\text{-}alphas \ data)$

$(staged\text{-}degree \ data)$

$(staged\text{-}composition\text{-}fri\text{-}roots \ data)$

$(staged\text{-}composition\text{-}final \ data)$

$i \subseteq B$

and $raw\text{-}bound: query\text{-}index\text{-}raw\text{-}preimage\text{-}bound$

and $subset: B \subseteq query\text{-}sample\text{-}space$

and $frac:$

$nnreal \ (card \ B) / nnreal \ (card \ query\text{-}sample\text{-}space) \leq$

$query\text{-}error\text{-}bound$

shows

$wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$

$staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}query\text{-}partial\text{-}opening\text{-}hit$

$adversary\text{-}initial\text{-}state \leq$

$nnreal$

$(staged\text{-}attacker\text{-}query\text{-}budget \ budgets +$

$staged\text{-}challenge\text{-}query\text{-}budget) * query\text{-}error\text{-}bound$

proof –

have $target:$

$wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$

$staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}query\text{-}partial\text{-}opening\text{-}hit$

$adversary\text{-}initial\text{-}state \leq$

$staged\text{-}phase\text{-}target\text{-}error \ (query\text{-}index\text{-}raw\text{-}preimage \ B)$

$(staged\text{-}attacker\text{-}query\text{-}budget \ budgets +$

$staged\text{-}challenge\text{-}query\text{-}budget)$

by (rule
 checked-staged-security-with-data-state-partial-opening-hit-bound-from-index-set-controlled
 [*OF wf controlled cover*])
 also have ... $\leq$
 nnreal
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget) * *query-error-bound*
 by (rule *staged-phase-query-index-target-error-query-bound*
 [*OF raw-bound subset frac*])
 finally show ?thesis .
 qed

lemma *checked-staged-security-with-data-state-query-bad-imp-partial-opening-hit-on-support:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
 Some (((data, attacker-state), result), final-state) ∈
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-data-state A*)
 adversary-initial-state)
and *bad:*
 staged-security-with-data-state-query-bad-hit
 (*Some (((data, attacker-state), result), final-state)*)

shows

staged-security-with-data-state-query-partial-opening-hit
 (*Some (((data, attacker-state), result), final-state)*)

proof —

let ?s =
 verifier-state-from-adversary attacker-state
 (*staged-proof-transcript data*)
from *checked-staged-security-experiment-with-data-state-outcomeE*
 [*OF support*]
obtain *builder* **where** *builder:*
 Some (data, attacker-state) ∈
 set-dist
 (*execute (checked-staged-transcript-program A)*
 adversary-initial-state)
and *verifier:*
 Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by *blast*

have *staged-header:*

verifier-header-transcript ?s
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)


```

    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
show ?thesis
proof (rule checked-staged-security-with-data-state-query-bad-round-chunkE
      [OF wf controlled support bad])
fix i trace-table composition-table query-idxs raw-idxs
assume i-bound: i < rounds
and bound:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table (staged-alphas data) query-idxs
and success:
  query-idxs ! i ∈
  query-sampling-success-space trace-table composition-table
  (staged-alphas data)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and query-chunk:
  verifier-query-round-chunk (query-idxs ! i)
  (staged-trace-fri-roots data)
  (staged-composition-fri-roots data)
  (staged-query-chunks data ! i)
and lookup:
  fmlookup (HashMap final-state)
  (QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i)) =
  Some (raw-idxs ! i)
show ?thesis
proof (rule accepted-with-bound-tables-partial-opening-witnesses-Some
      [OF bound])
fix fr f-fri-roots f-final dg composition-fri-roots final rest
  trace-openings composition-openings
assume header:
  verifier-header-transcript ?s fr f-fri-roots f-final
  (staged-alphas data) dg composition-fri-roots final rest
and comp-nonempty: composition-fri-roots ≠ []
and trace-partial:
  accepted-with-partial-trace-openings ?s
  (Some (result, final-state)) fr query-idxs trace-openings
and comp-partial:
  accepted-with-partial-composition-openings ?s
  (Some (result, final-state)) (hd composition-fri-roots)
  query-idxs composition-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:

```

```

partial-composition-table-candidate composition-table
  composition-openings
and trace-bind:
merkle-root-binds-table fr trace-table final-state
and comp-bind:
merkle-root-binds-table (hd composition-fri-roots)
  composition-table final-state
have header-eq:
fr = staged-trace-root data ∧
f-fri-roots = staged-trace-fri-roots data ∧
f-final = staged-trace-final data ∧
dg = staged-degree data ∧
composition-fri-roots = staged-composition-fri-roots data ∧
final = staged-composition-final data ∧
rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF staged-header header]
by simp
have idx-in-header-set:
query-idxs ! i ∈
  query-header-supported-partial-opening-success-indices-at ?s fr
    f-fri-roots f-final (staged-alphas data) dg
    composition-fri-roots final i
by (rule
  query-header-supported-partial-opening-success-indices-atI-from-partials
  [OF verifier trace-partial comp-partial trace-candidate
    comp-candidate header comp-nonempty refl i-bound success])
have idx-in-staged-set:
query-idxs ! i ∈
  query-header-supported-partial-opening-success-indices-at ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    i
using idx-in-header-set header-eq by simp
have raw-idx:
index (to-nat (raw-idxs ! i)) = query-idxs ! i
using query-idxs-eq len-raw i-bound by simp
have hit-at:
staged-security-with-data-state-query-partial-opening-hit-at i
  (Some (((data, attacker-state), result), final-state))
unfolding
  staged-security-with-data-state-query-partial-opening-hit-at-def
using i-bound lookup idx-in-staged-set raw-idx
by (simp add: Let-def)
show ?thesis

```

```

    by (rule staged-security-with-data-state-query-partial-opening-hitI
      [OF i-bound hit-at])
  qed
qed
qed

lemma checked-staged-security-with-data-state-query-bad-bound-from-partial-opening-hit:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-bad-hit
      adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-partial-opening-hit
      adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
    and bad: staged-security-with-data-state-query-bad-hit out
  show staged-security-with-data-state-query-partial-opening-hit out
proof (cases out)
  case None
  then show ?thesis
    using bad unfolding staged-security-with-data-state-query-bad-hit-def
    by simp
  next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  show ?thesis
    unfolding out-eq
    by (rule
      checked-staged-security-with-data-state-query-bad-imp-partial-opening-hit-on-support
      [OF wf controlled])
      (use support bad out-eq in simp-all)
qed
qed

lemma checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-partial-opening-hit:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and partial-bound:

```

$wp\text{-}event \text{ (checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-query-partial-opening-hit}$
 $adversary\text{-}initial\text{-}state \leq Q$

shows

$wp\text{-}event \text{ (checked-staged-security-experiment-with-data-state } A)$
 $\text{(staged-security-with-data-state-verifier-event query-bad)}$
 $adversary\text{-}initial\text{-}state \leq Q$

proof –

have *query-hit-bound*:

$wp\text{-}event \text{ (checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-query-bad-hit}$
 $adversary\text{-}initial\text{-}state \leq Q$

by (*rule order.trans*
 $[OF \text{ checked-staged-security-with-data-state-query-bad-bound-from-partial-opening-hit}$
 $[OF \text{ wf controlled}] \text{ partial-bound}]$)

show *?thesis*

by (*rule checked-staged-security-with-data-state-query-bad-verifier-event-bound*
 $[OF \text{ query-hit-bound}]$)

qed

lemma *checked-staged-security-with-data-state-query-bad-bound-from-partial-opening-fixed-query-error-cover*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *cover*:

$\bigwedge \text{data attacker-state } i.$
 $i < \text{rounds} \implies$
 $\text{query-header-supported-partial-opening-success-indices-at}$
 $\text{(verifier-state-from-adversary attacker-state}$
 $\text{(staged-proof-transcript data))}$
 $\text{(staged-trace-root data)}$
 $\text{(staged-trace-fri-roots data)}$
 $\text{(staged-trace-final data)}$
 $\text{(staged-alphas data)}$
 $\text{(staged-degree data)}$
 $\text{(staged-composition-fri-roots data)}$
 $\text{(staged-composition-final data)}$
 $i \subseteq B$

and *raw-bound*: *query-index-raw-preimage-bound*

and *subset*: $B \subseteq \text{query-sample-space}$

and *frac*:

$nnreal (\text{card } B) / nnreal (\text{card query-sample-space}) \leq$
 query-error-bound

shows

$wp\text{-}event \text{ (checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-query-bad-hit}$
 $adversary\text{-}initial\text{-}state \leq$
 $nnreal$
 $\text{(staged-attacker-query-budget budgets +}$
 $\text{staged-challenge-query-budget) * query-error-bound}$

proof –
have *wp-event* (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-query-bad-hit
adversary-initial-state $\leq$
wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-query-partial-opening-hit
adversary-initial-state
by (*rule*
checked-staged-security-with-data-state-query-bad-bound-from-partial-opening-hit
 $[OF\ wf\ controlled]$)
also have ... $\leq$
nnreal
(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget) * *query-error-bound*
by (*rule*
checked-staged-security-with-data-state-partial-opening-hit-bound-from-fixed-query-error-cover
 $[OF\ wf\ controlled\ cover\ raw-bound\ subset\ frac]$)
finally show *?thesis* .
qed

lemma *verifier-query-round-program-authenticated-initial-openings*:
fixes *s t* :: (*f*, *a*) *protocol-channel-scheme*
assumes *fl-eq*: *fl* = (*b*, *composition-root*) # *fl-tail*
and *outcome*:
Some (*()*, *t*) $\in$
set-dist
(*execute*
(*verifier-query-round-program fr f-fl f-final as fl final*) *s*)
obtains *raw idx trace-leaves trace-openings composition-openings* **where**
idx = *index* (*to-nat raw*)
map opening-value trace-openings = *trace-leaves*
map opening-index trace-openings = *powers-scaled idx*
*partial-authenticated-table fr (scale * clength) trace-openings t*
*partial-authenticated-table composition-root (scale * clength)*
composition-openings t
map opening-index composition-openings =
 $[idx, fri-sibling-index\ (scale * clength)\ idx]$

proof –
from *outcome[unfolded verifier-query-round-program-alt-def]*
obtain *raw s0* **where** *tail*:
Some (*()*, *t*) $\in$
set-dist
(*execute*
(*verifier-query-round-after-index-program*
fr f-fl f-final as fl final raw) *s0*)
by (*auto elim!:* *set-dist-bindE*)
show *?thesis*
proof (*rule verifier-query-round-after-index-authenticated-trace-openings*
 $[OF\ tail]$)

```

fix idx trace-leaves trace-openings
assume idx-eq: idx = index (to-nat raw)
  and trace-values:
    map opening-value trace-openings = trace-leaves
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx
  and trace-table:
    partial-authenticated-table fr (scale * clength) trace-openings t
show ?thesis
proof (rule
  verifier-query-round-after-index-authenticated-composition-openings
  [OF fl-eq tail])
fix idx' composition-openings
assume idx'-eq: idx' = index (to-nat raw)
  and comp-table:
    partial-authenticated-table composition-root (scale * clength)
    composition-openings t
  and comp-indices:
    map opening-index composition-openings =
    [idx', fri-sibling-index (scale * clength) idx']
have idx'-idx: idx' = idx
  using idx-eq idx'-eq by simp
show ?thesis
  by (rule that[OF idx-eq trace-values trace-indices trace-table
    comp-table])
    (use comp-indices idx'-idx in simp)
qed
qed
qed

lemma verifier-query-round-program-authenticated-initial-openings-with-lookup:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-eq: fl = (b, composition-root) # fl-tail
and outcome:
  Some ((), t) ∈
    set-dist
    (execute
      (verifier-query-round-program fr f-fl f-final as fl final) s)
obtains raw idx trace-leaves trace-openings composition-openings where
  idx = index (to-nat raw)
  map opening-value trace-openings = trace-leaves
  map opening-index trace-openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) trace-openings t
  partial-authenticated-table composition-root (scale * clength)
    composition-openings t
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw

```

```

proof –
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where rand:
  Some (raw, s0) ∈
    set-dist (execute receive-query-index-challenge s)
and tail:
  Some (( ), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s0)
by (auto elim! set-dist-bindE)
have lookup-s0:
  fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
using receive-query-index-challenge-outcome[OF rand] by simp
have lookup-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
using verifier-query-round-after-index-program-preserves-query-lookup
  [OF tail, of PQueryCounter s PState s] lookup-s0
by simp
show ?thesis
proof (rule verifier-query-round-after-index-authenticated-trace-openings
  [OF tail])
fix idx trace-leaves trace-openings
assume idx-eq: idx = index (to-nat raw)
and trace-values:
  map opening-value trace-openings = trace-leaves
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table:
  partial-authenticated-table fr (scale * clength) trace-openings t
show ?thesis
proof (rule
  verifier-query-round-after-index-authenticated-composition-openings
  [OF fl-eq tail])
fix idx' composition-openings
assume idx'-eq: idx' = index (to-nat raw)
and comp-table:
  partial-authenticated-table composition-root (scale * clength)
    composition-openings t
and comp-indices:
  map opening-index composition-openings =
    [idx', fri-sibling-index (scale * clength) idx']
have idx'-idx: idx' = idx
using idx-eq idx'-eq by simp
show ?thesis
by (rule that[OF idx-eq trace-values trace-indices trace-table

```

```

      comp-table - lookup-t])
    (use comp-indices idx'-idx in simp)
  qed
  qed
  qed

lemma verifier-query-round-program-authenticated-initial-openings-with-chunk:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-eq: fl = (b, composition-root) # fl-tail
  and outcome:
    Some (( ), t) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  obtains raw idx chunk trace-leaves trace-openings composition-openings
  where
    idx = index (to-nat raw)
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
    PTranscript s = chunk @ PTranscript t
    PState t = foldl concat (PState s) chunk
    s ≤ t
    PQueryCounter t = Suc (PQueryCounter s)
    map opening-value trace-openings = trace-leaves
    map opening-index trace-openings = powers-scaled idx
    partial-authenticated-table fr (scale * clength) trace-openings t
    partial-authenticated-table composition-root (scale * clength)
      composition-openings t
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx]
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof -
  show ?thesis
proof (rule verifier-query-round-program-outcome[OF outcome])
  fix raw0 idx0 chunk
  assume idx0-eq: idx0 = index (to-nat raw0)
  and chunk-shape:
    verifier-query-round-chunk idx0 (map snd f-fl) (map snd fl)
      chunk
  and transcript:
    PTranscript s = chunk @ PTranscript t
  and state:
    PState t = foldl concat (PState s) chunk
  and ext: s ≤ t
  and counter:
    PQueryCounter t = Suc (PQueryCounter s)
  and lookup0:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw0

```



```

show ?thesis
proof (rule
  verifier-query-round-program-authenticated-initial-openings-with-lookup
  [OF fl-eq outcome])
fix raw idx trace-leaves trace-openings composition-openings
assume idx-eq: idx = index (to-nat raw)
and trace-values:
  map opening-value trace-openings = trace-leaves
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table:
  partial-authenticated-table fr (scale * clength)
  trace-openings t
and comp-table:
  partial-authenticated-table composition-root (scale * clength)
  composition-openings t
and comp-indices:
  map opening-index composition-openings =
  [idx, fri-sibling-index (scale * clength) idx]
and lookup:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
have raw-eq: raw = raw0
  using lookup lookup0 by simp
have idx-idx0: idx = idx0
  using idx-eq idx0-eq raw-eq by simp
show ?thesis
  by (rule that[OF idx-eq - transcript state ext counter
    trace-values trace-indices trace-table comp-table comp-indices
    lookup])
    (use chunk-shape idx-idx0 in simp)
qed
qed
qed

lemma ntimes-verifier-query-rounds-authenticated-initial-openings-at:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-eq: fl = (b, composition-root) # fl-tail
and outcome:
  Some (results, t) ∈
  set-dist
  (execute
    (ntimes
      (verifier-query-round-program fr f-fl f-final as fl final) n)
    s)
and i-bound: i < n
obtains raw idx trace-leaves trace-openings composition-openings where
  idx = index (to-nat raw)
  map opening-value trace-openings = trace-leaves

```

```

map opening-index trace-openings = powers-scaled idx
partial-authenticated-table fr (scale * clength) trace-openings t
partial-authenticated-table composition-root (scale * clength)
  composition-openings t
map opening-index composition-openings =
  [idx, fri-sibling-index (scale * clength) idx]
proof -
have
  ∃ raw idx trace-leaves trace-openings composition-openings.
  idx = index (to-nat raw) ∧
  map opening-value trace-openings = trace-leaves ∧
  map opening-index trace-openings = powers-scaled idx ∧
  partial-authenticated-table fr (scale * clength) trace-openings t ∧
  partial-authenticated-table composition-root (scale * clength)
    composition-openings t ∧
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
using outcome i-bound
proof (induction n arbitrary: i s results t)
case 0
then show ?case
by simp
next
case (Suc n)
from Suc.prem(1) obtain u results' s1 where
  head:
    Some (u, s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
and tail:
    Some (results', t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            n) s1)
by (auto elim!: set-dist-bindE)
have u-eq: u = ()
by (cases u) simp
show ?case
proof (cases i)
case 0
show ?thesis
proof (rule verifier-query-round-program-authenticated-initial-openings
  [OF fl-eq head[unfolded u-eq]])
fix raw idx trace-leaves trace-openings composition-openings
assume idx-eq: idx = index (to-nat raw)
and trace-values:

```

```

    map opening-value trace-openings = trace-leaves
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx
  and trace-table-s1:
    partial-authenticated-table fr (scale * clength)
    trace-openings s1
  and comp-table-s1:
    partial-authenticated-table composition-root (scale * clength)
    composition-openings s1
  and comp-indices:
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx]
  from ntimes-verifier-query-rounds-outcome[OF tail]
  obtain raw-idxs query-idxs query-chunks where
    s1-t: s1 ≤ t
    by (elim exE conjE)
  have trace-table-t:
    partial-authenticated-table fr (scale * clength)
    trace-openings t
    by (rule partial-authenticated-table-mono[OF trace-table-s1 s1-t])
  have comp-table-t:
    partial-authenticated-table composition-root (scale * clength)
    composition-openings t
    by (rule partial-authenticated-table-mono[OF comp-table-s1 s1-t])
  show ?thesis
  apply (rule exI[where x=raw])
  apply (rule exI[where x=idx])
  apply (rule exI[where x=trace-leaves])
  apply (rule exI[where x=trace-openings])
  apply (rule exI[where x=composition-openings])
  apply (intro conjI)
    apply (rule idx-eq)
    apply (rule trace-values)
    apply (rule trace-indices)
    apply (rule trace-table-t)
  apply (rule comp-table-t)
  apply (rule comp-indices)
  done
qed
next
case (Suc j)
have j-bound: j < n
  using Suc.premis(2) ⟨i = Suc j⟩ by simp
from Suc.IH[OF tail j-bound] show ?thesis .
qed
qed
then obtain raw idx trace-leaves trace-openings composition-openings where
  idx-eq: idx = index (to-nat raw)
  and trace-values:

```

```

    map opening-value trace-openings = trace-leaves
and trace-indices:
    map opening-index trace-openings = powers-scaled idx
and trace-table:
    partial-authenticated-table fr (scale * clength) trace-openings t
and comp-table:
    partial-authenticated-table composition-root (scale * clength)
      composition-openings t
and comp-indices:
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx]
by (elim exE conjE)
show ?thesis
by (rule that[OF idx-eq trace-values trace-indices trace-table
  comp-table comp-indices])
qed

lemma accepted-with-bound-tables-partial-opening-header-witness-at:
assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and bound:
  accepted-with-bound-tables s (Some (result, final-state))
  trace-table composition-table as query-idxs
and i-bound: i < rounds
obtains trace-openings composition-openings where
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses s fr f-fri-roots
  f-final as dg composition-fri-roots final
proof —
show ?thesis
proof (rule accepted-with-bound-tables-partial-opening-witnesses-Some[OF bound])
fix fr' f-fri-roots' f-final' dg' composition-fri-roots' final'
  rest' trace-openingss composition-openingss
assume
  header':
  verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
  composition-fri-roots' final' rest'
and comp-nonempty': composition-fri-roots' ≠ []
and trace-partial:
  accepted-with-partial-trace-openings s (Some (result, final-state))
  fr' query-idxs trace-openingss
and comp-partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) (hd composition-fri-roots')
  query-idxs composition-openingss
have eqs:

```

```

fr' = fr ∧
f-fri-roots' = f-fri-roots ∧
f-final' = f-final ∧
dg' = dg ∧
composition-fri-roots' = composition-fri-roots ∧
final' = final ∧
rest' = rest
using verifier-header-transcript-unique[OF header' header] by simp
have trace-table-i:
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) final-state
using trace-partial i-bound eqs
unfolding accepted-with-partial-trace-openings-def by blast
have comp-table-i:
  partial-authenticated-table (hd composition-fri-roots) (scale * clength)
    (composition-openings ! i) final-state
using comp-partial i-bound eqs
unfolding accepted-with-partial-composition-openings-def by simp
have witness:
  (trace-openings ! i, composition-openings ! i) ∈
    query-header-supported-partial-opening-witnesses s fr f-fri-roots
      f-final as dg composition-fri-roots final
unfolding query-header-supported-partial-opening-witnesses-def
using outcome header trace-table-i comp-table-i comp-nonempty' eqs
by auto
show ?thesis
by (rule that[OF witness])
qed
qed

lemma checked-actual-alpha-prefix-support-partial-opening-header-witness-at:
fixes prefix prefix-state data attacker-state result final-state
defines
  s ≡
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
assumes support:
  Some (((((prefix, prefix-state), data), attacker-state),
    result), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and bound:
  accepted-with-bound-tables s (Some (result, final-state))

```

trace-table composition-table as query-idxs
and *i-bound: i < rounds*
obtains *trace-openings composition-openings* **where**
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses s fr f-fri-roots
f-final as dg composition-fri-roots final
proof –
have *verify-out:*
Some (result, final-state) ∈ set-dist (execute verify-monad s)
using *support*
unfolding *checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def*
s-def
by *(auto elim!: set-dist-bindE split: prod.splits)*
show *?thesis*
by *(rule accepted-with-bound-tables-partial-opening-header-witness-at*
[OF verify-out header bound i-bound that])
qed

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-header-witness-at:*
fixes *prefix prefix-state data attacker-state result final-state*
defines
s ≡
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
assumes *support:*
Some (((((prefix, prefix-state), data), attacker-state),
result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)
and *hit:*
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
(Some (((((prefix, prefix-state), data), attacker-state),
result), final-state))
and *i-bound: i < rounds*
obtains *as dg composition-fri-roots final rest trace-table composition-table*
query-idxs trace-openings composition-openings **where**
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
verifier-header-transcript s
(staged-trace-root data) (staged-trace-fri-roots data)
(staged-trace-final data) as dg composition-fri-roots final rest
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses s
(staged-trace-root data) (staged-trace-fri-roots data)
(staged-trace-final data) as dg composition-fri-roots final
proof –

```

obtain trace-table composition-table as query-idxs dg
  composition-fri-roots final rest trace-tree composition-tree where
  verify-out:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
and bound:
  accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
and header:
  verifier-header-transcript s
    (staged-trace-root data) (staged-trace-fri-roots data)
    (staged-trace-final data) as dg composition-fri-roots final rest
using hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit-def
  s-def Let-def
by auto
obtain trace-openings composition-openings where witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses s
    (staged-trace-root data) (staged-trace-fri-roots data)
    (staged-trace-final data) as dg composition-fri-roots final
by (rule accepted-with-bound-tables-partial-opening-header-witness-at
  [OF verify-out header bound i-bound])
show ?thesis
by (rule that[OF bound header witness])
qed

```

definition

```

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
:: ((((((f × 'f list × 'f list × 'f) ×
  'f protocol-channel) × 'f staged-proof-data) ×
  'f protocol-channel) × unit list) ×
  'f protocol-channel) option ⇒ bool
where
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
  out ⇔
  (case out of
    None ⇒ False
  | Some (((((prefix, prefix-state), data), attacker-state),
    result), final-state) ⇒
    (let s =
      verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)
    in ∃ as dg composition-fri-roots final rest trace-table
      composition-table query-idxs trace-openings composition-openings.
      accepted-with-bound-tables s (Some (result, final-state))
        trace-table composition-table as query-idxs ∧
      verifier-header-transcript s
        (staged-trace-root data) (staged-trace-fri-roots data)
    )
  )

```

(staged-trace-final data) as dg composition-fri-roots final rest $\wedge$
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses s
 (staged-trace-root data) (staged-trace-fri-roots data)
 (staged-trace-final data) as dg composition-fri-roots final))

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-imp-partial-opening-witness-hit-on-support:*
assumes *support:*
 out $\in$
 set-dist
 (execute
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 adversary-initial-state)
and *hit:*
 checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
 out
shows
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
 out
proof (cases out)
case *None*
then show ?thesis
using hit
unfolding
 checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit-def
by simp
next
case (Some result-pack)
then obtain prefix prefix-state data attacker-state result final-state **where**
 out-eq:
 out = Some (((((prefix, prefix-state), data), attacker-state),
 result), final-state)
by (cases result-pack, auto split: prod.splits)
let ?s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
have support-some:
 Some (((((prefix, prefix-state), data), attacker-state),
 result), final-state) $\in$
 set-dist
 (execute
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 adversary-initial-state)
using support **unfolding** out-eq .
have hit-some:
 checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
 (Some (((((prefix, prefix-state), data), attacker-state),


```

    result), final-state))
using hit unfolding out-eq .
obtain as dg composition-fri-roots final rest trace-table composition-table
  query-idxs trace-openings composition-openings where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
and header:
  verifier-header-transcript ?s
    (staged-trace-root data) (staged-trace-fri-roots data)
    (staged-trace-final data) as dg composition-fri-roots final rest
and witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data) (staged-trace-fri-roots data)
    (staged-trace-final data) as dg composition-fri-roots final
proof –
obtain trace-table composition-table as query-idxs dg
  composition-fri-roots final rest trace-tree composition-tree where
  verify-out:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
and bound:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs
and header:
  verifier-header-transcript ?s
    (staged-trace-root data) (staged-trace-fri-roots data)
    (staged-trace-final data) as dg composition-fri-roots final rest
using hit-some
unfolding
  checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit-def
  Let-def
by auto
obtain trace-openings composition-openings where witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data) (staged-trace-fri-roots data)
    (staged-trace-final data) as dg composition-fri-roots final
by (rule accepted-with-bound-tables-partial-opening-header-witness-at
  [OF verify-out header bound rounds-positive])
show ?thesis
  by (rule that[OF bound header witness])
qed
show ?thesis
unfolding out-eq
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-def
  Let-def
apply simp
apply (rule exI[where x=as])

```

```

apply (intro conjI)
apply (rule exI[where x=trace-table])
apply (rule exI[where x=composition-table])
apply (rule exI[where x=query-idxs])
apply (rule bound)
apply (rule exI[where x=dg])
apply (rule exI[where x=composition-fri-roots])
apply (rule exI[where x=final])
apply (intro conjI)
apply (rule exI[where x=rest])
apply (rule header)
apply (rule exI[where x=trace-openings])
apply (rule exI[where x=composition-openings])
apply (rule witness)
done
qed

end

end

```

```

theory Soundness-Staged-Partial-Query-Branch
imports Soundness-Staged-Partial-Query
begin

```

Branch-specific probability bridges for the partial-opening query layer. This keeps branch/tree-output integration material out of the size-sensitive base partial-query theory.

```

context soundness
begin

```

```

definition checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
:: (((((('f × 'f list × 'f list × 'f) ×
   'f protocol-channel) × 'f staged-proof-data) ×
   'f protocol-channel) × unit list) ×
   'f protocol-channel) option ⇒ bool

```

```

where
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
    out  $\longleftrightarrow$ 
    (case out of
      None  $\Rightarrow$  False
    | Some (x, t)  $\Rightarrow$ 
      staged-security-with-data-state-query-partial-opening-hit
        (Some (((snd (fst (fst x)), snd (fst x)), snd x), t)))

```

```

lemma checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hitE:
assumes hit:
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit

```

(Some (((((prefix, prefix-state), data), attacker-state), result),
 final-state))
obtains i raw trace-openings composition-openings **where**
 $i < \text{rounds}$
 $\text{fmlookup } (\text{HashMap } \text{final-state})$
 (QueryIndexChallenge i
 (state-after-query-chunks
 (staged-query-start-hash data)
 (staged-query-chunks data) i)) =
 Some raw
 $\text{index } (\text{to-nat raw}) \in \text{query-sample-space}$
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
 partial-query-openings-consistent trace-openings composition-openings
 (staged-alphas data) (index (to-nat raw))
proof –
have data-hit:
 staged-security-with-data-state-query-partial-opening-hit
 (Some (((data, attacker-state), result), final-state))
using hit
unfolding
 checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-def
by simp
then obtain i **where** $i\text{-bound}: i < \text{rounds}$
and hit-at:
 staged-security-with-data-state-query-partial-opening-hit-at i
 (Some (((data, attacker-state), result), final-state))
unfolding staged-security-with-data-state-query-partial-opening-hit-def
by blast
from staged-security-with-data-state-query-partial-opening-hit-atE
 [OF hit-at]
obtain raw **where** lookup:
 $\text{fmlookup } (\text{HashMap } \text{final-state})$
 (QueryIndexChallenge i
 (state-after-query-chunks
 (staged-query-start-hash data)
 (staged-query-chunks data) i)) =
 Some raw
and raw-hit:
 $\text{index } (\text{to-nat raw}) \in$

query-header-supported-partial-opening-success-indices-at
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
 i
 by blast
 from *query-header-supported-partial-opening-success-indices-atE*
 [OF raw-hit]
 obtain trace-openings composition-openings where idx-sample:
 index (to-nat raw) ∈ query-sample-space
 and witness:
 (trace-openings, composition-openings) ∈
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
 and consistent:
 partial-query-openings-consistent trace-openings composition-openings
 (staged-alphas data) (index (to-nat raw))
 by blast
 show ?thesis
 by (rule that[OF i-bound lookup idx-sample witness consistent])
 qed

lemma *checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-projection:*

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
 adversary-initial-state =
wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-query-partial-opening-hit
 adversary-initial-state

proof –

let ?project =
 λx. ((snd (fst (fst x)), snd (fst x)), snd x)

have

wp-event (checked-staged-security-experiment-with-data-state A)

```

    staged-security-with-data-state-query-partial-opening-hit
    adversary-initial-state =
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
     $\gg$  ( $\lambda x.$  return ( $?project\ x$ )))
    staged-security-with-data-state-query-partial-opening-hit
    adversary-initial-state
using
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
  [of A]
by simp
also have ... =
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ( $\lambda out.$  case out of
    None  $\Rightarrow$  staged-security-with-data-state-query-partial-opening-hit None
  | Some (x, t)  $\Rightarrow$ 
    staged-security-with-data-state-query-partial-opening-hit
    (Some ( $?project\ x,$  t)))
    adversary-initial-state
by (rule wp-event-bind-return-map)
also have ... =
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
  adversary-initial-state
unfolding
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-def
  staged-security-with-data-state-query-partial-opening-hit-def
  staged-security-with-data-state-query-partial-opening-hit-at-def
by (rule arg-cong[where
  f= $\lambda Q.$  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    Q adversary-initial-state])
  (rule ext, simp split: option.splits prod.splits)
finally show ?thesis
by simp
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-bound-from-fixed-query-error-cover*
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and cover:
 $\bigwedge$ data attacker-state i.
 i < rounds $\implies$
 query-header-supported-partial-opening-success-indices-at
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)

(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
 $i \subseteq B$
and *raw-bound: query-index-raw-preimage-bound*
and *subset: $B \subseteq \text{query-sample-space}$*
and *frac:*
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space}) \leq$
 query-error-bound
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
adversary-initial-state $\leq$
 nnreal
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}) * \text{query-error-bound}$
proof –
have *data-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-query-partial-opening-hit
adversary-initial-state $\leq$
 nnreal
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}) * \text{query-error-bound}$
by (rule
checked-staged-security-with-data-state-partial-opening-hit-bound-from-fixed-query-error-cover
 $[OF \text{ wf controlled cover raw-bound subset frac}]$)
show *?thesis*
unfolding
checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-projection
by (rule *data-bound*)
qed

lemma *checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support-branch:*
assumes *support:*
Some (((((prefix, prefix-state), data), attacker-state), result),
final-state) $\in$
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)
shows
Some (((data, attacker-state), result), final-state) $\in$
set-dist

```

      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
proof –
  let ?project =
    λx. ((snd (fst (fst x)), snd (fst x)), snd x)
  have projected:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
            A ≫ (λx. return (?project x)))
          adversary-initial-state)
  apply (rule set-dist-bindI[OF support])
  apply simp
  done
  show ?thesis
  using projected
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-bound-tables-all-queries-imp-query-partial-opening-hit-on-sup*

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and support:

Some (((((prefix, prefix-state), data), attacker-state), result),
final-state) ∈

set-dist

(execute

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)

adversary-initial-state)

and bound:

accepted-with-bound-tables

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

(Some (result, final-state)) trace-table composition-table as
query-idxs

and header:

verifier-header-transcript

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

(staged-trace-root data) (staged-trace-fri-roots data)

(staged-trace-final data) as dg composition-fri-roots final rest

and all-queries:

all-queries-consistent trace-table composition-table as

shows

checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit

```

      (Some (((((prefix, prefix-state), data), attacker-state), result),
              final-state))
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
               adversary-initial-state)
  by (rule
      checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support-branch
      [OF support])
  from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
  obtain builder where builder:
    Some (data, attacker-state) ∈
      set-dist
      (execute (checked-staged-transcript-program A)
               adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
  have staged-header:
    verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
  have header-eq:
    as = staged-alphas data ∧
    dg = staged-degree data ∧
    composition-fri-roots = staged-composition-fri-roots data ∧
    final = staged-composition-final data ∧
    rest = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF staged-header header]
  by simp
  have shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as query-idxs
  using bound
  unfolding accepted-with-bound-tables-def accepted-with-tables-def
  by simp

```



```

from checked-staged-security-with-data-state-accepted-shape-query-chunks
  [OF wf controlled data-support shape]
obtain raw-idxs where
  as-eq: as = staged-alphas data
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
  and lookup:
     $\bigwedge i. i < \text{rounds} \implies$ 
      fmlookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)
  by blast
have bound-staged:
  accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table (staged-alphas data) query-idxs
  using bound as-eq by simp
have all-queries-staged:
  all-queries-consistent trace-table composition-table
    (staged-alphas data)
  using all-queries as-eq by simp
have i-bound: 0 < rounds
  by (rule rounds-positive)
have idx-sample:
  query-idxs ! 0  $\in$  query-sample-space
proof –
  have len-query-idxs: length query-idxs = rounds
    by (rule accepted-with-bound-tables-shapes(4)) [OF bound-staged]
  have query-idxs ! 0  $\in$  set query-idxs
    by (rule nth-mem) (use i-bound len-query-idxs in simp)
  then show ?thesis
    by (rule accepted-with-bound-tables-query-sample-space
      [OF bound-staged])
qed
show ?thesis
proof (rule accepted-with-bound-tables-partial-opening-witnesses-Some
  [OF bound-staged])
  fix fr f-fri-roots f-final dg' composition-fri-roots' final'
    rest' trace-openings composition-openings
  assume header':
    verifier-header-transcript ?s fr f-fri-roots f-final
      (staged-alphas data) dg' composition-fri-roots' final' rest'
  and comp-nonempty: composition-fri-roots'  $\neq$  []
  and trace-partial:
    accepted-with-partial-trace-openings ?s
      (Some (result, final-state)) fr query-idxs trace-openings

```

```

and comp-partial:
  accepted-with-partial-composition-openings ?s
    (Some (result, final-state)) (hd composition-fri-roots')
    query-idxs composition-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
    composition-openings
have header-eq':
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  dg' = staged-degree data ∧
  composition-fri-roots' = staged-composition-fri-roots data ∧
  final' = staged-composition-final data ∧
  rest' = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF staged-header header']
by simp
have idx-in:
  query-idxs ! 0 ∈
    query-header-supported-partial-opening-success-indices-at ?s fr
      f-fri-roots f-final (staged-alphas data) dg'
      composition-fri-roots' final' 0
by (rule
  query-header-supported-partial-opening-success-indices-atI-from-partials-consistent
  [OF verifier trace-partial comp-partial trace-candidate
    comp-candidate header' comp-nonempty refl i-bound idx-sample
    all-queries-staged])
have idx-in-staged:
  query-idxs ! 0 ∈
    query-header-supported-partial-opening-success-indices-at ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      0
using idx-in header-eq' by simp
have raw-idx:
  index (to-nat (raw-idxs ! 0)) = query-idxs ! 0
using query-idxs-eq len-raw i-bound by simp
have hit-at:
  staged-security-with-data-state-query-partial-opening-hit-at 0
    (Some (((data, attacker-state), result), final-state))
unfolding staged-security-with-data-state-query-partial-opening-hit-at-def
using i-bound lookup[OF i-bound] idx-in-staged raw-idx

```

```

    by (simp add: Let-def)
  have hit:
    staged-security-with-data-state-query-partial-opening-hit
      (Some (((data, attacker-state), result), final-state))
  by (rule staged-security-with-data-state-query-partial-opening-hitI
      [OF i-bound hit-at])
  show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-def
  using hit by simp
qed
qed

```

definition *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit*

```

:: ((((((f × 'f list × 'f list × 'f) ×
      'f protocol-channel) × 'f staged-proof-data) ×
      'f protocol-channel) × unit list) ×
      'f protocol-channel) option ⇒ bool

```

where

```

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
  out ⇔
  (case out of
    None ⇒ False
  | Some (((((prefix, prefix-state), data), attacker-state),
    result), final-state) ⇒
    (let s =
      verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)
    in ∃ as dg composition-fri-roots final rest trace-table
      composition-table query-idxs trace-openings composition-openings.
      accepted-with-bound-tables s (Some (result, final-state))
      trace-table composition-table as query-idxs ∧
      verifier-header-transcript s
        (staged-trace-root data) (staged-trace-fri-roots data)
        (staged-trace-final data) as dg composition-fri-roots final rest ∧
      (trace-openings, composition-openings) ∈
      query-header-supported-partial-opening-witnesses s
        (staged-trace-root data) (staged-trace-fri-roots data)
        (staged-trace-final data) as dg composition-fri-roots final ∧
      all-queries-consistent trace-table composition-table as))

```

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit-imp-query-hit-on-su*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

```

  out ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state

```

```

      A)
      adversary-initial-state)
and hit:
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
  out
shows
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
  out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit-def
  by simp
next
case (Some result-pack)
then obtain prefix prefix-state data attacker-state result final-state where
  out-eq:
    out = Some (((((prefix, prefix-state), data), attacker-state),
      result), final-state)
  by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have support-some:
  Some (((((prefix, prefix-state), data), attacker-state),
    result), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)
  using support unfolding out-eq .
from hit obtain as dg composition-fri-roots final rest trace-table
  composition-table query-idxs trace-openings composition-openings where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
and header:
  verifier-header-transcript ?s
  (staged-trace-root data) (staged-trace-fri-roots data)
  (staged-trace-final data) as dg composition-fri-roots final rest
and all-queries:
  all-queries-consistent trace-table composition-table as
unfolding out-eq
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit-def
  Let-def
by auto

```

```

show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-actual-alpha-prefix-bound-tables-all-queries-imp-query-partial-opening-hit-on-support
    [OF wf controlled support-some bound header all-queries])
qed

lemma checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit-bound-from-query-hit
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and query-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
  adversary-initial-state ≤ Q
shows
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
  adversary-initial-state ≤ Q
proof –
have
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
  adversary-initial-state ≤
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (rule
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit-imp-query-hit-on-support
    [OF wf controlled])
then show ?thesis
  by (rule order-trans[OF - query-bound])
qed

definition checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit
  :: ((((((f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool
where
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit
  out ⟷
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
  out ∧
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit

```

out

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit-imp-query-hit:*

assumes

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit

out

shows *checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit*

out

using *assms*

unfolding

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit-def

by *simp*

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit-bound-from-query-hit:*

assumes *query-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit

adversary-initial-state $\leq Q$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit

adversary-initial-state $\leq Q$

by (*rule order.trans[OF - query-bound]*)

(*rule wp-event-mono,*

rule

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit-imp-query-hit)

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit-bound-from-fixed-query-e*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *cover:*

$\bigwedge \text{data attacker-state } i.$

$i < \text{rounds} \implies$

query-header-supported-partial-opening-success-indices-at

(*verifier-state-from-adversary attacker-state*

(*staged-proof-transcript data*))

(*staged-trace-root data*)

(*staged-trace-fri-roots data*)

(*staged-trace-final data*)

(*staged-alphas data*)

(*staged-degree data*)

(*staged-composition-fri-roots data*)

(*staged-composition-final data*)

$i \subseteq B$

and *raw-bound: query-index-raw-preimage-bound*

and *subset: $B \subseteq \text{query-sample-space}$*

and *frac:*

$$\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$$

shows

$$\text{wp-event } (\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A) \text{ checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit adversary-initial-state} \leq \text{nnreal } (\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget}) * \text{query-error-bound}$$

proof –

have *query-bound*:

$$\text{wp-event } (\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A) \text{ checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit adversary-initial-state} \leq \text{nnreal } (\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget}) * \text{query-error-bound}$$

by (rule

$$\text{checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-bound-from-fixed-query-error-cover [OF wf controlled cover raw-bound subset frac]}$$

show *?thesis*

by (rule

$$\text{checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit-bound-from-query-hit [OF query-bound]}$$

qed

definition *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-query-hit*

$$\begin{aligned} &:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times \\ &\quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times \\ &\quad 'f \text{ protocol-channel}) \times \text{unit list}) \times \\ &\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool} \end{aligned}$$

where

$$\begin{aligned} &\text{checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-query-hit} \\ &\quad \text{out} \longleftrightarrow \\ &\text{checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit} \\ &\quad \text{out} \wedge \\ &\neg \text{checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit} \\ &\quad \text{out} \end{aligned}$$

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-imp-query-or-without-query*:

assumes

$$\text{checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit out}$$

shows

$$\text{checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit out} \vee \text{checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-query-hit out}$$

```

    out
using assms
unfolding
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit-def
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-query-hit-def
by blast

lemma checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-bound-from-query-and-without-
assumes query-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit
  adversary-initial-state  $\leq Q$ 
and without-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-query-hit
  adversary-initial-state  $\leq W$ 
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
  adversary-initial-state  $\leq Q + W$ 
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?Witness =
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
  let ?Query =
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit
  let ?Without =
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-query-hit
  have wp-event ?M ?Witness adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda out. ?Query\ out \vee ?Without\ out$ )
    adversary-initial-state
  by (rule wp-event-mono)
  (rule
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-imp-query-or-without-query)
  also have ...  $\leq$ 
    wp-event ?M ?Query adversary-initial-state +
    wp-event ?M ?Without adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq Q + W$ 
  by (intro add-mono query-bound without-bound)
  finally show ?thesis .
qed

```


lemma *checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-partial-opening-witness-hit:*
assumes *witness-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit

adversary-initial-state $\leq W$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit

adversary-initial-state $\leq W$

proof —

have

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit

adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit

adversary-initial-state

by (rule *wp-event-mono-on-support*)

(rule

checked-actual-alpha-prefix-branch-subtree-verifier-imp-partial-opening-witness-hit-on-support)

also have ... $\leq W$

by (rule *witness-bound*)

finally show *?thesis* .

qed

definition *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit*

$:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit

out $\longleftrightarrow$

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit

out $\wedge$

$\neg$ *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit*

out

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-imp-all-queries-or-without-all-q*

assumes

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit

out

shows

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit

out $\vee$

```

    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit
    out
using assms
unfolding
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit-def
by blast

lemma checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-bound-from-all-queries-and-wit
assumes all-queries-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
    adversary-initial-state  $\leq Q$ 
and without-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit
    adversary-initial-state  $\leq W$ 
shows
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
    adversary-initial-state  $\leq Q + W$ 
proof –
let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let ?Witness =
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
let ?All =
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
let ?Without =
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit
have wp-event ?M ?Witness adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda out. ?All\ out \vee ?Without\ out$ )
    adversary-initial-state
by (rule wp-event-mono)
    (rule
     checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-imp-all-queries-or-without-all-quer)
also have ...  $\leq$ 
    wp-event ?M ?All adversary-initial-state +
    wp-event ?M ?Without adversary-initial-state
by (rule wp-event-union-bound)
also have ...  $\leq Q + W$ 
by (intro add-mono all-queries-bound without-bound)
finally show ?thesis .
qed

```

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-all-queries-and-without-all-queries:*

assumes *all-queries-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit

adversary-initial-state $\leq Q$

and *without-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit

adversary-initial-state $\leq W$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit

adversary-initial-state $\leq Q + W$

proof —

have *witness-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit

adversary-initial-state $\leq Q + W$

by (rule

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-bound-from-all-queries-and-without-all-queries
[OF *all-queries-bound* *without-bound*])

show *?thesis*

by (rule

checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-partial-opening-witness-hit
[OF *witness-bound*])

qed

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit-imp-fri-or-q*

assumes *support:*

out $\in$

set-dist

(*execute*

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

adversary-initial-state)

and *hit:*

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit

out

shows

checked-staged-security-with-actual-alpha-prefix-verifier-event

trace-fri-bad-with-partial-openings *out* $\vee$

checked-staged-security-with-actual-alpha-prefix-verifier-event

composition-fri-bad-with-partial-openings *out* $\vee$

```

    checked-staged-security-with-actual-alpha-prefix-verifier-event
    query-bad out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit-def
      checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-def
    by simp
next
  case (Some result-pack)
  then obtain prefix prefix-state data attacker-state result final-state where
    out-eq:
      out = Some (((((prefix, prefix-state), data), attacker-state),
        result), final-state)
    by (cases result-pack, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have support-some:
    Some (((((prefix, prefix-state), data), attacker-state),
      result), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  using support unfolding out-eq .
  have witness-hit:
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
      out
    using hit
    unfolding
      checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit-def
    by simp
  have not-all-hit:
    ¬
      checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
        out
    using hit
    unfolding
      checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit-def
    by simp
  from witness-hit obtain as dg composition-fri-roots final rest trace-table
    composition-table query-idxs trace-openings composition-openings where
    bound:
      accepted-with-bound-tables ?s (Some (result, final-state))
      trace-table composition-table as query-idxs

```

```

and header:
  verifier-header-transcript ?s
  (staged-trace-root data) (staged-trace-fri-roots data)
  (staged-trace-final data) as dg composition-fri-roots final rest
and witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses ?s
  (staged-trace-root data) (staged-trace-fri-roots data)
  (staged-trace-final data) as dg composition-fri-roots final
unfolding out-eq
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-def
  Let-def
by auto
have not-all-queries:
  ¬ all-queries-consistent trace-table composition-table as
proof
assume all-queries:
  all-queries-consistent trace-table composition-table as
have
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
  out
unfolding out-eq
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit-def
  Let-def
apply simp
apply (rule exI[where x=as])
apply (rule exI[where x=dg])
apply (rule exI[where x=composition-fri-roots])
apply (rule exI[where x=final])
apply (rule exI[where x=rest])
apply (rule exI[where x=trace-table])
apply (rule exI[where x=composition-table])
apply (intro conjI)
  apply (rule exI[where x=query-idxs])
  apply (rule bound)
  apply (rule header)
apply (rule exI[where x=trace-openings])
apply (rule exI[where x=composition-openings])
apply (rule witness)
apply (rule all-queries)
done
then show False
  using not-all-hit by contradiction
qed
show ?thesis
proof (cases trace-table-low-degree trace-table)
case False
obtain fr f-fri-roots f-final dg' composition-fri-roots' final'
  rest' trace-openings' composition-openings' where

```

```

    trace-partial:
    accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr query-idxs trace-openings'
  by (rule accepted-with-bound-tables-partial-opening-witnesses-Some
    [OF bound]) blast
have trace-bad:
  trace-fri-bad-with-partial-openings ?s
  (Some (result, final-state))
  unfolding trace-fri-bad-with-partial-openings-def trace-fri-bad-def
  using bound False trace-partial by blast
then show ?thesis
  unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  Let-def
  by simp
next
case trace-low: True
show ?thesis
proof (cases
  composition-table-low-degree maxDegree composition-table)
case False
obtain fr f-fri-roots f-final dg' composition-fri-roots' final'
  rest' trace-openings' composition-openings' where
  trace-partial:
  accepted-with-partial-trace-openings ?s
  (Some (result, final-state)) fr query-idxs trace-openings'
  by (rule accepted-with-bound-tables-partial-opening-witnesses-Some
    [OF bound]) blast
have comp-bad:
  composition-fri-bad-with-partial-openings ?s
  (Some (result, final-state))
  unfolding composition-fri-bad-with-partial-openings-def
  composition-fri-bad-def
  using bound trace-low False trace-partial by blast
then show ?thesis
  unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  Let-def
  by simp
next
case composition-low: True
have query-bad:
  query-bad ?s (Some (result, final-state))
  unfolding query-bad-def
  using bound trace-low composition-low not-all-queries by blast
then show ?thesis
  unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  Let-def

```

by simp
 qed
 qed
 qed

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit-bound-from*

assumes *trace-bound*:

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 trace-fri-bad-with-partial-openings)
adversary-initial-state $\leq T$

and *composition-bound*:

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-fri-bad-with-partial-openings)
adversary-initial-state $\leq C$

and *query-bound*:

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 query-bad)
adversary-initial-state $\leq Q$

shows

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state *A*)
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit
adversary-initial-state $\leq T + C + Q$

proof –

let *?M* =

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A

let *?Without* =

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit

let *?Trace* =

checked-staged-security-with-actual-alpha-prefix-verifier-event
trace-fri-bad-with-partial-openings

let *?Composition* =

checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-fri-bad-with-partial-openings

let *?Query* =

checked-staged-security-with-actual-alpha-prefix-verifier-event query-bad

have *wp-event ?M ?Without adversary-initial-state* $\leq$

wp-event ?M ($\lambda out. ?Trace out \vee (?Composition out \vee ?Query out)$)
adversary-initial-state

by (*rule wp-event-mono-on-support*)

```

(rule
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit-imp-fri-or-quer
also have ... ≤
  wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M (λout. ?Composition out ∨ ?Query out)
  adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
  wp-event ?M ?Trace adversary-initial-state +
  (wp-event ?M ?Composition adversary-initial-state +
   wp-event ?M ?Query adversary-initial-state)
by (intro add-mono order-refl wp-event-union-bound)
also have ... ≤ T + (C + Q)
by (intro add-mono trace-bound composition-bound query-bound)
also have ... = T + C + Q
by (simp add: add.assoc)
finally show ?thesis .
qed

```

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-query-hit-partial-fri-and-query:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *query-hit-bound:*

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
   A)
checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
adversary-initial-state ≤ QH

```

and *trace-bound:*

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
   A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
   trace-fri-bad-with-partial-openings)
adversary-initial-state ≤ T

```

and *composition-bound:*

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
   A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
   composition-fri-bad-with-partial-openings)
adversary-initial-state ≤ C

```

and *query-bound:*

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
   A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
   query-bad)
adversary-initial-state ≤ Q

```


shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
adversary-initial-state $\leq QH + (T + C + Q)$

proof –
have *all-queries-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
adversary-initial-state $\leq QH$
by (rule
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit-bound-from-query-hit
[OF wf controlled query-hit-bound])
have *without-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit
adversary-initial-state $\leq T + C + Q$
by (rule
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit-bound-from-par
[OF trace-bound composition-bound query-bound])
show *?thesis*
by (rule
checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-all-queries-and-without-all-queries
[OF all-queries-bound without-bound])
qed

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-partial-opening-witness-query-and-u*
assumes *query-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit
adversary-initial-state $\leq Q$
and *without-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-query-hit
adversary-initial-state $\leq W$
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
adversary-initial-state $\leq Q + W$

proof –
have *witness-bound*:
wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
 adversary-initial-state $\leq Q + W$
 by (rule
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-bound-from-query-and-without-query
 [OF query-bound without-bound])
 show ?thesis
 by (rule
 checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-partial-opening-witness-hit
 [OF witness-bound])
 qed

lemma checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-partial-opening-fixed-query-cover-and-budget
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and cover:
 $\bigwedge$ data attacker-state i .
 $i < \text{rounds} \implies$
 query-header-supported-partial-opening-success-indices-at
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
 $i \subseteq B$
and raw-bound: query-index-raw-preimage-bound
and subset: $B \subseteq \text{query-sample-space}$
and frac:
 $\text{nnreal}(\text{card } B) / \text{nnreal}(\text{card } \text{query-sample-space}) \leq$
 query-error-bound
and without-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-query-hit
 adversary-initial-state $\leq W$
shows
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
 adversary-initial-state $\leq$
 nnreal
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget) * query-error-bound + W
proof –

have *query-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit
adversary-initial-state $\leq$
nnreal
(staged-attacker-query-budget budgets +
*staged-challenge-query-budget) * query-error-bound*
by (rule
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit-bound-from-fixed-query-error
[OF wf controlled cover raw-bound subset frac])
show *?thesis*
by (rule
checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-partial-opening-witness-query-and-without-bound
[OF query-bound without-bound])
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-witness-and-transcript-budget*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *root-pre-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
adversary-initial-state $\leq RP$
and *subtree-pre-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
adversary-initial-state $\leq SP$
and *subtree-verifier-witness-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
adversary-initial-state $\leq WV$
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event composition-bad)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(RP + staged-concrete-transcript-target-error-bound + SP + WV)
proof –

have *subtree-verifier-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
adversary-initial-state $\leq$ *WV*
by (rule
checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-partial-opening-witness-hit
 [OF *subtree-verifier-witness-bound*])
show *?thesis*
by (rule
checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-side-events-and-transcript-budget
 [OF *wf controlled root-pre-bound subtree-pre-bound*
subtree-verifier-bound])
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-query-hit-partial-fri-and-query-hit-bound*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *subtree-pre-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
adversary-initial-state $\leq$ *SP*
and *query-hit-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
adversary-initial-state $\leq$ *QH*
and *trace-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
trace-fri-bad-with-partial-openings)
adversary-initial-state $\leq$ *T*
and *composition-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
composition-fri-bad-with-partial-openings)
adversary-initial-state $\leq$ *C*
and *query-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

```

    (checked-staged-security-with-actual-alpha-prefix-verifier-event
     query-bad)
    adversary-initial-state  $\leq Q$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state  $\leq$ 
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    (staged-phase-relation-error size
     (staged-attacker-query-budget budgets + staged-challenge-query-budget) +
     staged-concrete-transcript-target-error-bound + SP +
     (QH + (T + C + Q)))
  proof -
    have subtree-verifier-bound:
      wp-event
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
         A)
        checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
        adversary-initial-state  $\leq QH + (T + C + Q)$ 
    by (rule
        checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-query-hit-partial-fri-and-query
        [OF wf controlled query-hit-bound trace-bound composition-bound
         query-bound])
    show ?thesis
    by (rule
        checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-side-events-and-transcript-budg
        [OF wf controlled subtree-pre-bound subtree-verifier-bound])
  qed

end

end

theory Staged-Security-Experiment-Composition-Partial
  imports
    Staged-Security-Experiment-Composition-Branch
    Soundness-Reductions-Composition
begin

Partial-candidate composition reductions for the staged security experiment.
This layer avoids treating verifier-side Merkle subtree recomputation as a rare event.
Instead, it lifts the verifier-local reduction from composition alpha badness to a partial-
header candidate event plus hash collision.

context soundness

```

begin

definition *checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit*

```
:: ('f list ⇒ 'f list set) ⇒
  ((((((('f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool
```

where

```
checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
bad-sets out ⇔
(case out of
  None ⇒ False
| Some (x, final-state) ⇒
  (let packed = fst (fst x);
    data = snd packed;
    attacker-state = snd (fst x);
    result = snd x;
    s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  in alpha-header-list-set-hit s
    (alpha-header-supported-partial-union-bad-sets s bad-sets)
    (Some (result, final-state))))
```

definition *checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit*

```
:: ('f list ⇒ 'f list set) ⇒
  ((((((('f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool
```

where

```
checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
bad-sets out ⇔
(case out of
  None ⇒ False
| Some (x, final-state) ⇒
  (let packed = fst (fst x);
    prefix-state = snd (fst packed);
    data = snd packed;
    attacker-state = snd (fst x);
    result = snd x;
    s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  in alpha-header-list-set-hit s
    (alpha-header-supported-partial-union-bad-sets s bad-sets)
    (Some (result, final-state)) ∧
    alpha-vector-prequeried-in-state prefix-state
```

(staged-alphas data) (length spec)))

definition *checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit*

:: ('f list $\Rightarrow$ 'f list set) $\Rightarrow$
 ((((((('f $\times$ 'f list $\times$ 'f list $\times$ 'f) $\times$
 'f protocol-channel) $\times$ 'f staged-proof-data) $\times$
 'f protocol-channel) $\times$ unit list) $\times$
 'f protocol-channel) option $\Rightarrow$ bool

where

checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
bad-sets out $\longleftrightarrow$
checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
bad-sets out $\wedge$
 (case out of
 None $\Rightarrow$ False
 | Some (x, -) $\Rightarrow$
 (let packed = fst (fst x);
 prefix-state = snd (fst packed);
 data = snd packed
 in $\neg$ alpha-vector-prequeried-in-state prefix-state
 (staged-alphas data) (length spec)))

definition *checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*

:: ('f list $\Rightarrow$ 'f list set) $\Rightarrow$
 ((((((('f $\times$ 'f list $\times$ 'f list $\times$ 'f) $\times$
 'f protocol-channel) $\times$ 'f staged-proof-data) $\times$
 'f protocol-channel) $\times$ unit list) $\times$
 'f protocol-channel) option $\Rightarrow$ bool

where

checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
bad-sets out $\longleftrightarrow$
checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
bad-sets out $\wedge$
 $\neg$ *checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
bad-sets out

definition *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift*

:: ((((((('f $\times$ 'f list $\times$ 'f list $\times$ 'f) $\times$
 'f protocol-channel) $\times$ 'f staged-proof-data) $\times$
 'f protocol-channel) $\times$ unit list) $\times$
 'f protocol-channel) option $\Rightarrow$ bool

where

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
out $\longleftrightarrow$
checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-randomization-bad out $\wedge$
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space out

definition *staged-alpha-prefix-partial-header-prequery-hit*

$:: 'f \text{ staged-adversary} \Rightarrow ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

staged-alpha-prefix-partial-header-prequery-hit A *bad-sets* *out*
 $\longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (\text{prefix}, \text{prefix-state}) \Rightarrow$
 $(\exists \text{data attacker-state result final-state.}$
 $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}) \in$
 set-dist
 $(\text{execute}$
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{adversary-initial-state}) \wedge$
 $\text{checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit}$
 bad-sets
 $(\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}))))))$

definition *staged-alpha-prefix-partial-header-prequery-relation*

$:: 'f \text{ staged-adversary} \Rightarrow 'f \text{ protocol-channel} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$

where

staged-alpha-prefix-partial-header-prequery-relation A s *bad-sets* x y
 $\longleftrightarrow$
 $(\exists \text{prefix prefix-state data attacker-state result final-state } i.$
 $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{data}), \text{attacker-state}), \text{result}),$
 $\text{final-state}) \in$
 set-dist
 $(\text{execute}$
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A) \text{ } s) \wedge$
 $\text{checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit}$
 bad-sets
 $(\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{data}), \text{attacker-state}), \text{result}),$
 $\text{final-state}))) \wedge$
 $i < \text{length spec} \wedge$
 $i < \text{length } (\text{staged-alphas data}) \wedge$
 $x = \text{alpha-challenge-key-at prefix-state } i \text{ } (\text{staged-alphas data}) \wedge$
 $y = \text{staged-alphas data } ! i)$

lemma *staged-alpha-prefix-partial-header-prequery-relation-fiber-bound-size:*

card
 $\{y. \text{staged-alpha-prefix-partial-header-prequery-relation } A \text{ } s \text{ bad-sets } x \text{ } y\}$


```

    ≤ size
proof –
  have subset:
    {y. staged-alpha-prefix-partial-header-prequery-relation A s bad-sets x y}
    ⊆ (UNIV :: 'f set)
  by simp
  have finite-field: finite (UNIV :: 'f set)
  by simp
  have card
    {y. staged-alpha-prefix-partial-header-prequery-relation A s bad-sets x y}
    ≤ card (UNIV :: 'f set)
  by (rule card-mono[OF finite-field subset])
  also have ... = size
  using size-card by simp
  finally show ?thesis .
qed

lemma staged-alpha-prefix-partial-header-prequery-hit-imp-relation-hit:
  assumes empty: HashMap s = fmempty
  and support:
    Some (prefix, prefix-state) ∈
      set-dist (execute (staged-alpha-prefix-program A) s)
  and hit:
    staged-alpha-prefix-partial-header-prequery-hit A bad-sets
      (Some (prefix, prefix-state))
  and init: s = adversary-initial-state
  shows
    hash-relation-hit-event
      (staged-alpha-prefix-partial-header-prequery-relation A s bad-sets)
      s (Some (prefix, prefix-state))
proof –
  from hit obtain data attacker-state result final-state where full-support:
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  and prehit:
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
      bad-sets
      (Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state))
  unfolding staged-alpha-prefix-partial-header-prequery-hit-def by auto
  from prehit obtain i where
    i-bound: i < length spec
  and i-len: i < length (staged-alphas data)
  and lookup:

```

```

    fmlookup (HashMap prefix-state)
      (alpha-challenge-key-at prefix-state i (staged-alphas data)) =
        Some (staged-alphas data ! i)
  unfolding
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit-def
    alpha-vector-prequeried-in-state-def
  by (auto simp: Let-def)
have initial-none:
  fmlookup (HashMap s)
    (alpha-challenge-key-at prefix-state i (staged-alphas data)) = None
  using empty by simp
have relation:
  staged-alpha-prefix-partial-header-prequery-relation A s bad-sets
    (alpha-challenge-key-at prefix-state i (staged-alphas data))
    (staged-alphas data ! i)
  unfolding staged-alpha-prefix-partial-header-prequery-relation-def
  using full-support prehit i-bound i-len init by blast
have
  hash-relation-hit
    (staged-alpha-prefix-partial-header-prequery-relation A s bad-sets)
    s prefix-state
  unfolding hash-relation-hit-def
  by (intro exI[of - alpha-challenge-key-at prefix-state i
    (staged-alphas data)]
    exI[of - staged-alphas data ! i] conjI)
    (use initial-none lookup relation in simp-all)
then show ?thesis
  unfolding hash-relation-hit-event-def by simp
qed

lemma staged-alpha-prefix-partial-header-prequery-hit-bound-by-relation-hit:
  assumes init: s = adversary-initial-state
  shows
    wp-event (staged-alpha-prefix-program A)
      (staged-alpha-prefix-partial-header-prequery-hit A bad-sets) s ≤
    wp-event (staged-alpha-prefix-program A)
      (hash-relation-hit-event
        (staged-alpha-prefix-partial-header-prequery-relation A s bad-sets) s)
    s
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈ set-dist (execute (staged-alpha-prefix-program A) s)
  and hit:
    staged-alpha-prefix-partial-header-prequery-hit A bad-sets out
  show
    hash-relation-hit-event
      (staged-alpha-prefix-partial-header-prequery-relation A s bad-sets) s
    out

```

```

proof (cases out)
  case None
  then show ?thesis
    using hit unfolding staged-alpha-prefix-partial-header-prequery-hit-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state where out-eq:
    out = Some (prefix, prefix-state)
    by (cases packed) simp
  show ?thesis
    unfolding out-eq
    by (rule staged-alpha-prefix-partial-header-prequery-hit-imp-relation-hit
      [OF - - - init])
      (use support hit out-eq init in
        ‹simp-all add: adversary-initial-state-def›)
qed
qed

lemma staged-alpha-prefix-partial-header-prequery-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (staged-alpha-prefix-program A)
      (staged-alpha-prefix-partial-header-prequery-hit A bad-sets)
      adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0)
proof —
  have prehit-le-relation:
    wp-event (staged-alpha-prefix-program A)
      (staged-alpha-prefix-partial-header-prequery-hit A bad-sets)
      adversary-initial-state ≤
    wp-event (staged-alpha-prefix-program A)
      (hash-relation-hit-event
        (staged-alpha-prefix-partial-header-prequery-relation A
          adversary-initial-state bad-sets) adversary-initial-state)
      adversary-initial-state
    by (rule staged-alpha-prefix-partial-header-prequery-hit-bound-by-relation-hit)
      simp
  also have ... ≤
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
proof —
  have program:
    hash-relation-program
      (staged-alpha-prefix-partial-header-prequery-relation A
        adversary-initial-state bad-sets)
      size (staged-alpha-search-queries budgets 0)

```

```

    (staged-alpha-prefix-program A)
  by (rule hash-relation-program-staged-alpha-prefix-program
      [OF wf controlled])
    (rule
      staged-alpha-prefix-partial-header-prequery-relation-fiber-bound-size)
  show ?thesis
    using program
    unfolding hash-relation-program-def hash-relation-budget-def
      staged-phase-relation-error-def
    by blast
qed
finally show ?thesis .
qed

lemma checked-staged-security-experiment-with-actual-alpha-prefix-data-stateI:
  assumes prefix:
    Some (prefix, prefix-state) ∈
      set-dist
        (execute (staged-alpha-prefix-program A) adversary-initial-state)
  and after:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-after-alpha-prefix-program A prefix)
          prefix-state)
  and verifier:
    Some (result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
  shows
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
            A)
          adversary-initial-state)
proof -
  have transcript:
    Some (((prefix, prefix-state), data), attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-with-alpha-prefix-program A)
          adversary-initial-state)
  unfolding checked-staged-transcript-with-alpha-prefix-program-def
  apply (rule-tac x=prefix and t=prefix-state in set-dist-bindI)
  apply (rule prefix)
  apply (rule-tac x=prefix-state and t=prefix-state in set-dist-bindI)
  apply simp

```

```

    apply (rule-tac x=data and t=attacker-state in set-dist-bindI)
    apply (rule after)
    apply simp
  done
show ?thesis
unfolding
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
  apply (rule-tac x=((prefix, prefix-state), data) and
    t=attacker-state in set-dist-bindI)
    apply (rule transcript)
  apply (rule-tac x=attacker-state and t=attacker-state in set-dist-bindI)
    apply simp
  apply (rule-tac x=() and
    t=verifier-state-from-adversary attacker-state
    (staged-proof-transcript data) in set-dist-bindI)
    apply simp
  apply (rule-tac x=result and t=final-state in set-dist-bindI)
    apply (rule verifier)
  apply simp
done
qed

lemma checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit-bound-from-prefix:
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
        bad-sets)
      adversary-initial-state ≤
      wp-event (staged-alpha-prefix-program A)
        (staged-alpha-prefix-partial-header-prequery-hit A bad-sets)
        adversary-initial-state
  unfolding checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
    checked-staged-transcript-with-alpha-prefix-program-def
    sm-bind-assoc
  apply (rule wp-event-bind-bound-by-head-event
    [where P = staged-alpha-prefix-partial-header-prequery-hit A bad-sets])
  apply simp
  apply (simp add:
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit-def
    staged-alpha-prefix-partial-header-prequery-hit-def)
  apply (auto simp:
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit-def
    staged-alpha-prefix-partial-header-prequery-hit-def Let-def
    elim!: set-dist-bindE
    intro!: set-dist-bindI
    checked-staged-security-experiment-with-actual-alpha-prefix-data-stateI
    split: option.splits prod.splits)

```

```

apply (rule-tac x=bi in exI)
apply (rule-tac x=bj in exI)
apply (rule-tac x=bk in exI)
apply (rule-tac x=x2a in exI)
apply (intro conjI)
  apply (rule checked-staged-security-experiment-with-actual-alpha-prefix-data-stateI)
    apply assumption
    apply assumption
    apply assumption
    apply assumption
apply assumption
done

```

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit-bound:*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows

```

```

  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
      bad-sets)
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0)

```

proof —

```

have
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
      bad-sets)
  adversary-initial-state ≤
  wp-event (staged-alpha-prefix-program A)
    (staged-alpha-prefix-partial-header-prequery-hit A bad-sets)
  adversary-initial-state
by (rule
  checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit-bound-from-prefix)
also have ... ≤
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0)
by (rule staged-alpha-prefix-partial-header-prequery-hit-bound
  [OF wf controlled])
finally show ?thesis .

```

qed

Vector-valued, prefix-state-dependent accounting for future alpha challenge keys. The bad vector set may be selected from the concrete verifier/header state reached after the alpha prefix; the bound is derived from the existing relation-budget interface for the prefix program, not from a freshness assumption.

lemma *checked-staged-security-with-actual-alpha-prefix-vector-alpha-prequery-accounting:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

(*checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit*
bad-sets)

adversary-initial-state $\leq$
staged-phase-relation-error size

(*staged-alpha-search-queries budgets 0*)

by (*rule*

checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit-bound

[*OF wf controlled*])

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-imp-fresh-or-prequery:*

assumes

checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit

bad-sets out

shows

checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit

bad-sets out $\vee$

checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit

bad-sets out

using *assms*

unfolding

checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-def

checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-def

checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit-def

by (*auto simp: Let-def split: option.splits prod.splits*)

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-bound-from-fresh-and-prequery:*

assumes *fresh-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

(*checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit*
bad-sets)

adversary-initial-state $\leq F$

and *prequery-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

(*checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit*
bad-sets)

adversary-initial-state $\leq P$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit*
bad-sets)
adversary-initial-state $\leq F + P$
proof –
 let *?M* =
 checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
 let *?Bad* =
 checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
 bad-sets
 let *?Fresh* =
 checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
 bad-sets
 let *?Pre* =
 checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
 bad-sets
have *wp-event ?M ?Bad adversary-initial-state* $\leq$
 wp-event ?M (λ out. ?Fresh out $\vee$?Pre out) adversary-initial-state
by (*rule wp-event-mono*)
 (*rule*
 checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-imp-fresh-or-prequery)
also have ... $\leq$
 wp-event ?M ?Fresh adversary-initial-state +
 wp-event ?M ?Pre adversary-initial-state
by (*rule wp-event-union-bound*)
also have ... $\leq F + P$
by (*intro add-mono fresh-bound prequery-bound*)
finally show *?thesis* .
qed

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-bound-from-fresh-and-budgets:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *fresh-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit*
 bad-sets)
 adversary-initial-state $\leq F$
shows
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit*
 bad-sets)
 adversary-initial-state $\leq$
 F + staged-phase-relation-error size

(*staged-alpha-search-queries* budgets 0)

proof –

have *prequery-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit*
bad-sets)
adversary-initial-state ≤
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0)

by (*rule*
checked-staged-security-with-actual-alpha-prefix-vector-alpha-prequery-accounting
 [OF *wf* controlled])

show *?thesis*

by (*rule*
checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-bound-from-fresh-and-prequery
 [OF *fresh-bound prequery-bound*])

qed

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-imp-prefix-or-drift*:

assumes

checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
bad-sets out

shows

checked-staged-security-with-actual-alpha-prefix-bad-set-hit *bad-sets*
 out ∨
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
bad-sets out

using *assms*

unfolding

checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def

by *blast*

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-bound-from-prefix-and-drift*:

assumes *prefix-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
bad-sets)
adversary-initial-state ≤ P

and *drift-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
bad-sets)
adversary-initial-state ≤ D

shows

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
   A)
  (checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
   bad-sets)
  adversary-initial-state  $\leq P + D$ 
proof –
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?Fresh =
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
    bad-sets
  let ?Prefix =
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit bad-sets
  let ?Drift =
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
    bad-sets
  have wp-event ?M ?Fresh adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda$ out. ?Prefix out  $\vee$  ?Drift out)
    adversary-initial-state
  by (rule wp-event-mono)
    (rule
     checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-imp-prefix-or-drift)
  also have ...  $\leq$ 
    wp-event ?M ?Prefix adversary-initial-state +
    wp-event ?M ?Drift adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq P + D$ 
  by (intro add-mono prefix-bound drift-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-imp-branch-tree-on-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out  $\in$ 
  set-dist
  (execute
   (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
   adversary-initial-state)
and drift:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
  out
shows
  checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
  out
proof –

```

```

have random-bad:
  checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-randomization-bad out
using drift
unfolding
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
by simp
have not-prefix:
   $\neg$  checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out
using drift
unfolding
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def
by simp
have split:
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out  $\vee$ 
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
      out
by (rule
  checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-prefix-or-branch-tree-on-sup
    [OF wf controlled support random-bad])
then show ?thesis
  using not-prefix by blast
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-branch-tree:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and branch-tree-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
  checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
  adversary-initial-state  $\leq T$ 
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
  adversary-initial-state  $\leq T$ 
proof –
let ?M =
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
have wp-event ?M
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
  adversary-initial-state  $\leq$ 
  wp-event ?M

```

checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
adversary-initial-state
by (rule *wp-event-mono-on-support*)
(rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-imp-branch-tree-on-support
[*OF wf controlled*])
then show ?thesis
by (rule *order-trans*[*OF - branch-tree-bound*])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-bound-from-prefix-drift-and-budgets*
assumes *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets A*
and *prefix-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
bad-sets)
adversary-initial-state $\leq P$
and *drift-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
bad-sets)
adversary-initial-state $\leq D$
shows
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit*
bad-sets)
adversary-initial-state $\leq$
 $P + D +$
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*)
proof –
have *fresh-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
(*checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit*
bad-sets)
adversary-initial-state $\leq P + D$
by (rule
checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-bound-from-prefix-and-drift
[*OF prefix-bound drift-bound*])
have
wp-event

```

    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
      bad-sets)
    adversary-initial-state ≤
    (P + D) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-bound-from-fresh-and-budgets
    [OF wf controlled fresh-bound])
  then show ?thesis
  by (simp add: add.assoc)
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-partial-header-or-collision
  assumes support:
    out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  and bad:
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-randomization-bad out
  shows
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    hash-map-output-collision-bad out
  proof (cases out)
  case None
  then show ?thesis
  using bad
  unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  by simp
next
  case (Some result-pack)
  then obtain prefix prefix-state data attacker-state result final-state
  where out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state),
        result), final-state)
  by (cases result-pack, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have verify-out:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)

```

```

using support
unfolding out-eq
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
by (auto elim!: set-dist-bindE split: prod.splits)
have random-bad:
  composition-randomization-bad ?s (Some (result, final-state))
using bad
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  Let-def
by simp
have alpha-bad:
  alpha-bad-set-hit ?s composition-trace-bad-alpha-space
  (Some (result, final-state))
by (rule composition-randomization-bad-imp-alpha-bad-set-hit
  [OF random-bad])
have split:
  alpha-header-list-set-hit ?s
  (alpha-header-supported-partial-union-bad-sets ?s
  composition-trace-bad-alpha-space)
  (Some (result, final-state))  $\vee$ 
  hash-map-output-collision-bad ?s (Some (result, final-state))
by (rule alpha-bad-set-hit-imp-partial-union-or-collision
  [OF verify-out alpha-bad
  composition-trace-bad-alpha-space-subset-alpha-space])
then show ?thesis
proof
  assume partial:
    alpha-header-list-set-hit ?s
    (alpha-header-supported-partial-union-bad-sets ?s
    composition-trace-bad-alpha-space)
    (Some (result, final-state))
  have
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
    composition-trace-bad-alpha-space out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-def
    Let-def
    using partial by simp
  then show ?thesis by simp
next
  assume collision:
    hash-map-output-collision-bad ?s (Some (result, final-state))
  have
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    hash-map-output-collision-bad out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    Let-def

```

```

    using collision by simp
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-prefix-prequery-drift-or-collision
  assumes support:
    out ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
            A)
          adversary-initial-state)
  and bad:
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-randomization-bad out
  shows
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
    out ∨
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    hash-map-output-collision-bad out
proof —
  have partial-or-collision:
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    hash-map-output-collision-bad out
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-partial-header-or-collision-or-collision
    [OF support bad])
  then show ?thesis
proof
  assume partial:
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
    composition-trace-bad-alpha-space out
  have fresh-or-prequery:
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space out
  by (rule
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-imp-fresh-or-prequery
    [OF partial])
  then show ?thesis
proof

```

```

assume fresh:
  checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
  composition-trace-bad-alpha-space out
have prefix-or-drift:
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit
  composition-trace-bad-alpha-space out  $\vee$ 
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
  composition-trace-bad-alpha-space out
by (rule
  checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-imp-prefix-or-drift
  [OF fresh])
then show ?thesis
proof
  assume
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out
  then show ?thesis by simp
next
  assume drift:
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
    composition-trace-bad-alpha-space out
  have
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
    out
  unfolding
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
  using bad drift by simp
  then show ?thesis by simp
qed
next
  assume
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space out
  then show ?thesis by simp
qed
next
  assume
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    hash-map-output-collision-bad out
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-bound-from-prefix-prequery-d
assumes prefix-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
  A)
  (checked-staged-security-with-actual-alpha-prefix-bad-set-hit

```



```

    composition-trace-bad-alpha-space)
  adversary-initial-state  $\leq P$ 
and prequery-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq Q$ 
and drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
    adversary-initial-state  $\leq D$ 
and collision-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
     hash-map-output-collision-bad)
    adversary-initial-state  $\leq C$ 
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
     composition-randomization-bad)
    adversary-initial-state  $\leq P + Q + D + C$ 
proof –
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?Prefix =
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space
  let ?Prequery =
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space
  let ?Drift =
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
  let ?Collision =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    hash-map-output-collision-bad
  let ?Random =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-randomization-bad
  have wp-event ?M ?Random adversary-initial-state  $\leq$ 
    wp-event ?M
    ( $\lambda$ out. ?Prefix out  $\vee$  ?Prequery out  $\vee$  ?Drift out  $\vee$ 

```

```

      ?Collision out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (rule
      checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-prefix-prequery-drift-or-collision)
  also have ... ≤
    wp-event ?M ?Prefix adversary-initial-state +
    wp-event ?M ?Prequery adversary-initial-state +
    wp-event ?M ?Drift adversary-initial-state +
    wp-event ?M ?Collision adversary-initial-state
  by (rule wp-event-union-bound4)
  also have ... ≤ P + Q + D + C
  by (intro add-mono prefix-bound prequery-bound drift-bound
    collision-bound)
  finally show ?thesis .
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-bound-from-partial-header-and-collision*

assumes *partial-bound*:

```

  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
      composition-trace-bad-alpha-space)
  adversary-initial-state ≤ P

```

and *collision-bound*:

```

  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      hash-map-output-collision-bad)
  adversary-initial-state ≤ C

```

shows

```

  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-randomization-bad)
  adversary-initial-state ≤ P + C

```

proof –

let *?M* =

```

  checked-staged-security-experiment-with-actual-alpha-prefix-data-state A

```

let *?Partial* =

```

  checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
  composition-trace-bad-alpha-space

```

let *?Collision* =

```

  checked-staged-security-with-actual-alpha-prefix-verifier-event
  hash-map-output-collision-bad

```

let *?Random* =

checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-randomization-bad
have *wp-event ?M ?Random adversary-initial-state* $\leq$
wp-event ?M (λ out. ?Partial out $\vee$?Collision out)
adversary-initial-state
by (rule *wp-event-mono-on-support*)
(rule
checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-partial-header-or-collision-o
also have ... $\leq$
wp-event ?M ?Partial adversary-initial-state +
wp-event ?M ?Collision adversary-initial-state
by (rule *wp-event-union-bound*)
also have ... $\leq P + C$
by (intro *add-mono partial-bound collision-bound*)
finally show *?thesis* .
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-partial-header-and-co*
assumes *partial-bound*:

wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq P$

and *collision-bound*:

wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-verifier-event*
hash-map-output-collision-bad)
adversary-initial-state $\leq C$

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event composition-bad*)
adversary-initial-state $\leq P + C$

proof –

let *?M* =

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A

let *?degree* =

checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-degree-bad

let *?random* =

checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-randomization-bad

let *?bad* =

checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-bad

have *projected*:

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event composition-bad)
  adversary-initial-state =
wp-event ?M ?bad adversary-initial-state
by (rule
  checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
have event-eq: ?bad = ( $\lambda$ out. ?degree out  $\vee$  ?random out)
by (rule ext)
  (auto simp:
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    composition-bad-def split: option.splits prod.splits)
have degree-false: ?degree = ( $\lambda$ out. False)
by (rule ext)
  (auto simp:
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    composition-degree-bad-false[OF spec-degree-wellformed-from-spec-query-margin]
    split: option.splits prod.splits)
have degree-bound:
  wp-event ?M ?degree adversary-initial-state  $\leq 0$ 
  unfolding degree-false wp-event-def wp-def dist-expect-def by simp
have random-bound:
  wp-event ?M ?random adversary-initial-state  $\leq P + C$ 
by (rule
  checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-bound-from-partial-header-and-c
  [OF partial-bound collision-bound])
have wp-event ?M ?bad adversary-initial-state  $\leq$ 
  wp-event ?M ?degree adversary-initial-state +
  wp-event ?M ?random adversary-initial-state
  unfolding event-eq by (rule wp-event-union-bound)
also have ...  $\leq 0 + (P + C)$ 
  by (intro add-mono degree-bound random-bound)
finally show ?thesis
  unfolding projected by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-hash-map-output-collision-bad-bound-controlled:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-verifier-event
        hash-map-output-collision-bad)
      adversary-initial-state  $\leq$ 
      hash-collision-budget-value 0
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget + verifier-hash-query-budget)
proof -
  have data-bound:

```

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 hash-map-output-collision-bad)
adversary-initial-state $\leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*)
by (*rule*
 checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
 [*OF wf controlled*])
then show ?thesis
using
 checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection
 [*of A hash-map-output-collision-bad*]
by *simp*
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-partial-header-and-co*

assumes *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets* *A*

and *partial-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit*
 composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ *P*

shows

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event* *composition-bad*)
adversary-initial-state $\leq$
P +
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*)

proof –

have *collision-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
 hash-map-output-collision-bad)
adversary-initial-state $\leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*)

by (*rule*

checked-staged-security-with-actual-alpha-prefix-hash-map-output-collision-bad-bound-controlled
 [*OF wf controlled*])

show ?thesis

by (rule
 checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-partial-header-and-collision
 [*OF partial-bound collision-bound*])
 qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-partial-header-candidate-drift-and-budget*
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *drift-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq D$
shows
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event composition-bad*)
 adversary-initial-state $\leq$
 composition-error-bound +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 D +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*)
proof –
 let ?*Prefix* =
 composition-error-bound +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*)
have *transcript-bound*:
 wp-event (*checked-staged-transcript-with-alpha-prefix-program A*)
 (*checked-staged-transcript-actual-alpha-prefix-bad-set-hit*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq$?*Prefix*
by (rule
 checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
 [*OF wf controlled*])
have *prefix-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*

$composition_trace_bad_alpha_space)$
 $adversary_initial_state \leq ?Prefix$
by (rule
 $checked_staged_security_with_actual_alpha_prefix_bad_set_hit_bound_from_transcript$
 $[OF transcript_bound])$
have $partial_bound$:
 wp_event
 $(checked_staged_security_experiment_with_actual_alpha_prefix_data_state\ A)$
 $(checked_staged_security_with_actual_alpha_prefix_partial_header_bad_set_hit$
 $composition_trace_bad_alpha_space)$
 $adversary_initial_state \leq$
 $?Prefix + D +$
 $staged_phase_relation_error\ size$
 $(staged_alpha_search_queries\ budgets\ 0)$
by (rule
 $checked_staged_security_with_actual_alpha_prefix_partial_header_bad_set_hit_bound_from_prefix_drift_and_budgets$
 $[OF\ wf\ controlled\ prefix_bound\ drift_bound])$
have
 $wp_event\ (checked_staged_security_experiment_with_data_state\ A)$
 $(staged_security_with_data_state_verifier_event\ composition_bad)$
 $adversary_initial_state \leq$
 $(?Prefix + D +$
 $staged_phase_relation_error\ size$
 $(staged_alpha_search_queries\ budgets\ 0)) +$
 $hash_collision_budget_value\ 0$
 $(staged_attacker_query_budget\ budgets +$
 $staged_challenge_query_budget + verifier_hash_query_budget)$
by (rule
 $checked_staged_security_with_data_state_composition_bad_bound_from_actual_alpha_prefix_partial_header_and_contro$
 $[OF\ wf\ controlled\ partial_bound])$
then show $?thesis$
by (simp add: add.assoc)
qed

lemma $checked_staged_security_with_data_state_composition_bad_bound_from_partial_prequery_branch_tree_and_budgets$
assumes wf : $staged_budget_wellformed\ budgets$
and $controlled$: $staged_adversary_controlled\ budgets\ A$
and $branch_tree_bound$:
 wp_event
 $(checked_staged_security_experiment_with_actual_alpha_prefix_data_state$
 $A)$
 $checked_staged_security_with_actual_alpha_prefix_branch_tree_output_hit$
 $adversary_initial_state \leq T$
shows
 $wp_event\ (checked_staged_security_experiment_with_data_state\ A)$
 $(staged_security_with_data_state_verifier_event\ composition_bad)$
 $adversary_initial_state \leq$
 $composition_error_bound +$
 $staged_phase_relation_error\ size$

```

    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    T +
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget)
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?Prefix =
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0)
  let ?Pre =
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  let ?Coll =
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget)
  have transcript-bound:
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?Prefix
  by (rule
    checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
    [OF wf controlled])
  have prefix-bound:
    wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?Prefix
  by (rule
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
    [OF transcript-bound])
  have prequery-bound:
    wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?Pre
  by (rule
    checked-staged-security-with-actual-alpha-prefix-vector-alpha-prequery-accounting
    [OF wf controlled])

```



```

have drift-bound:
  wp-event ?M
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
  adversary-initial-state  $\leq T$ 
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-branch-tree
    [OF wf controlled branch-tree-bound])
have collision-bound:
  wp-event ?M
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
    hash-map-output-collision-bad)
  adversary-initial-state  $\leq ?Coll$ 
  by (rule
    checked-staged-security-with-actual-alpha-prefix-hash-map-output-collision-bad-bound-controlled
    [OF wf controlled])
let ?degree =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-degree-bad
let ?random =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-randomization-bad
let ?bad =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-bad
have projected:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event composition-bad)
  adversary-initial-state =
  wp-event ?M ?bad adversary-initial-state
  by (rule
    checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
have event-eq: ?bad = ( $\lambda out.$  ?degree out  $\vee$  ?random out)
  by (rule ext)
  (auto simp:
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    composition-bad-def split: option.splits prod.splits)
have degree-false: ?degree = ( $\lambda out.$  False)
  by (rule ext)
  (auto simp:
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    composition-degree-bad-false[OF spec-degree-wellformed-from-spec-query-margin]
    split: option.splits prod.splits)
have degree-bound:
  wp-event ?M ?degree adversary-initial-state  $\leq 0$ 
  unfolding degree-false wp-event-def wp-def dist-expect-def by simp
have random-bound:
  wp-event ?M ?random adversary-initial-state  $\leq$ 
  ?Prefix + ?Pre + T + ?Coll
  by (rule

```

```

    checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-bound-from-prefix-prequery-drift-
    [OF prefix-bound prequery-bound drift-bound collision-bound])
  have wp-event ?M ?bad adversary-initial-state ≤
    wp-event ?M ?degree adversary-initial-state +
    wp-event ?M ?random adversary-initial-state
  unfolding event-eq by (rule wp-event-union-bound)
  also have ... ≤ 0 + (?Prefix + ?Pre + T + ?Coll)
  by (intro add-mono degree-bound random-bound)
  finally show ?thesis
  unfolding projected by (simp add: add.assoc)
qed

end

end

```

theory *Soundness-Staged-Candidate-Query*

imports

Soundness-Staged-Query

Soundness-Staged-Partial-Query

begin

Candidate-indexed dynamic query-prefix targets.

These targets are narrower than the old global partial-opening fixed cover: they are parameterized by one fixed trace/composition candidate pair and the authenticated openings extracted for that pair. This is the shape needed for the composition candidate-drift query case, where the target must be chosen before the query-index challenge is sampled.

context *soundness*

begin

definition *staged-query-prefix-candidate-opening-query-target*

```

:: 'f authenticated-opening list list ⇒
  'f authenticated-opening list list ⇒
  'f list ⇒ nat ⇒
  'f staged-query-prefix-data ⇒ 'f protocol-channel ⇒ nat set

```

where

```

staged-query-prefix-candidate-opening-query-target trace-openings
  composition-openings as i prefix prefix-state =
  partial-query-success-indices-at trace-openings composition-openings as i

```

definition *staged-query-prefix-candidate-pair-query-target*

```

:: 'f list ⇒ 'f list ⇒ 'f list ⇒
  'f staged-query-prefix-data ⇒ 'f protocol-channel ⇒ nat set

```

where

```

staged-query-prefix-candidate-pair-query-target trace-table
  composition-table as prefix prefix-state =
  query-sampling-success-space trace-table composition-table as

```

definition *staged-query-prefix-candidate-opening-query-target-from-prefix*
 $:: 'f \text{ authenticated-opening list list} \Rightarrow$
 $\quad 'f \text{ authenticated-opening list list} \Rightarrow$
 $\quad \text{nat} \Rightarrow 'f \text{ staged-query-prefix-data} \Rightarrow$
 $\quad 'f \text{ protocol-channel} \Rightarrow \text{nat set}$

where

staged-query-prefix-candidate-opening-query-target-from-prefix
 $\text{trace-openings composition-openings } i \text{ prefix prefix-state} =$
 $\text{partial-query-success-indices-at trace-openings composition-openings}$
 $(\text{sqp-alphas prefix}) i$

definition *staged-query-prefix-candidate-pair-query-target-from-prefix*
 $:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ staged-query-prefix-data} \Rightarrow$
 $\quad 'f \text{ protocol-channel} \Rightarrow \text{nat set}$

where

staged-query-prefix-candidate-pair-query-target-from-prefix *trace-table*
 $\text{composition-table prefix prefix-state} =$
 $\text{query-sampling-success-space trace-table composition-table}$
 $(\text{sqp-alphas prefix})$

definition *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit*

where

checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit *A*
 $\text{trace-table composition-table out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\mid \text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}), \text{result}),$
 $\quad \text{final-state}) \Rightarrow$
 $\quad (\exists i \text{ prefix prefix-state raw raw-state.}$
 $\quad \quad i < \text{rounds} \wedge$
 $\quad \quad \text{Some } (\text{data}, \text{attacker-state}) \in$
 $\quad \quad \text{set-dist}$
 $\quad \quad (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\quad \quad \quad \text{adversary-initial-state}) \wedge$
 $\quad \quad \text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in$
 $\quad \quad \text{set-dist}$
 $\quad \quad (\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A i)$
 $\quad \quad \quad \text{adversary-initial-state}) \wedge$
 $\quad \quad \text{checked-staged-query-prefix-dynamic-index-hit}$
 $\quad \quad (\text{staged-query-prefix-candidate-pair-query-target trace-table}$
 $\quad \quad \quad \text{composition-table } (\text{staged-alphas data}))$
 $\quad \quad (\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}))))$

definition *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit*

where

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit *A*
 $\text{trace-openings composition-openings as } i \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$

| *Some* (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
 final-state) $\Rightarrow$
as = staged-alphas data $\wedge$
i < rounds $\wedge$
 ($\exists$ prefix prefix-state raw raw-state.
Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\wedge$
Some (((prefix, prefix-state), raw), raw-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state *A i*)
 adversary-initial-state) $\wedge$
 checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-opening-query-target
 trace-openings composition-openings *as i*)
 (*Some* (((prefix, prefix-state), raw), raw-state))))

lemma staged-query-prefix-candidate-opening-query-target-subset:
 staged-query-prefix-candidate-opening-query-target trace-openings
 composition-openings *as i* prefix prefix-state $\subseteq$ query-sample-space
unfolding staged-query-prefix-candidate-opening-query-target-def
by (rule partial-query-success-indices-at-subset)

lemma staged-query-prefix-candidate-pair-query-target-subset:
 staged-query-prefix-candidate-pair-query-target trace-table
 composition-table *as* prefix prefix-state $\subseteq$ query-sample-space
unfolding staged-query-prefix-candidate-pair-query-target-def
by (rule query-sampling-success-space-subset)

lemma staged-query-prefix-candidate-opening-query-target-from-prefix-subset:
 staged-query-prefix-candidate-opening-query-target-from-prefix
 trace-openings composition-openings *i* prefix prefix-state $\subseteq$
 query-sample-space
unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
by (rule partial-query-success-indices-at-subset)

lemma staged-query-prefix-candidate-pair-query-target-from-prefix-subset:
 staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
 composition-table prefix prefix-state $\subseteq$ query-sample-space
unfolding staged-query-prefix-candidate-pair-query-target-from-prefix-def
by (rule query-sampling-success-space-subset)

lemma staged-query-prefix-candidate-opening-query-target-subset-candidate-pair:
assumes trace-candidate:
 partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
 partial-composition-table-candidate composition-table
 composition-openings

and *trace-low*: *trace-table-low-degree trace-table*
and *comp-low*: *composition-table-low-degree maxDegree composition-table*
and *not-all*:
 $\neg$ *all-queries-consistent trace-table composition-table as*
shows
staged-query-prefix-candidate-opening-query-target trace-openings
composition-openings as i prefix prefix-state $\subseteq$
staged-query-prefix-candidate-pair-query-target trace-table
composition-table as prefix prefix-state
unfolding *staged-query-prefix-candidate-opening-query-target-def*
staged-query-prefix-candidate-pair-query-target-def
by (*rule partial-query-success-indices-at-subset-query-sampling-success-space*
 $[OF \text{ trace-candidate comp-candidate trace-low comp-low not-all}]$)

lemma *staged-query-prefix-candidate-opening-query-target-from-prefix-subset-candidate-pair-from-prefix*:
assumes *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
and *comp-candidate*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-low*: *composition-table-low-degree maxDegree composition-table*
and *not-all*:
 $\neg$ *all-queries-consistent trace-table composition-table*
(sqp-alphas prefix)
shows
staged-query-prefix-candidate-opening-query-target-from-prefix
trace-openings composition-openings i prefix prefix-state $\subseteq$
staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
composition-table prefix prefix-state
unfolding *staged-query-prefix-candidate-opening-query-target-from-prefix-def*
staged-query-prefix-candidate-pair-query-target-from-prefix-def
by (*rule partial-query-success-indices-at-subset-query-sampling-success-space*
 $[OF \text{ trace-candidate comp-candidate trace-low comp-low not-all}]$)

lemma *checked-staged-query-prefix-dynamic-index-hit-mono*:
assumes *subset*: $\bigwedge \text{prefix prefix-state. } A \text{ prefix prefix-state} \subseteq$
 $B \text{ prefix prefix-state}$
and *hit*:
checked-staged-query-prefix-dynamic-index-hit A
(Some (((prefix, prefix-state), raw), raw-state))
shows
checked-staged-query-prefix-dynamic-index-hit B
(Some (((prefix, prefix-state), raw), raw-state))
using *subset hit*
unfolding *checked-staged-query-prefix-dynamic-index-hit-def* **by** *auto*

lemma *partial-trace-table-candidate-single-round-from-all-rounds*:
assumes *cand*: *partial-trace-table-candidate trace-table trace-openings*

```

    and i-bound:  $i < \text{rounds}$ 
  shows
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [ $i := \text{trace-openings} ! i$ ])
  proof -
    have len-table:  $\text{length } \text{trace-table} = \text{scale} * \text{clength}$ 
      using can and unfolding partial-trace-table-candidate-def by simp
    have agrees:
      table-agrees-with-authenticated-openings trace-table
      ( $\text{scale} * \text{clength}$ ) ( $\text{trace-openings} ! i$ )
    using can and i-bound unfolding partial-trace-table-candidate-def by blast
    show ?thesis
      by (rule partial-trace-table-candidate-single-roundI
        [OF i-bound len-table agrees])
  qed

lemma partial-composition-table-candidate-single-round-from-all-rounds:
  assumes can:
    partial-composition-table-candidate composition-table
    composition-openings
  and i-bound:  $i < \text{rounds}$ 
  shows
    partial-composition-table-candidate composition-table
    ((replicate rounds []) [ $i := \text{composition-openings} ! i$ ])
  proof -
    have len-table:  $\text{length } \text{composition-table} = \text{scale} * \text{clength}$ 
      using can and unfolding partial-composition-table-candidate-def by simp
    have agrees:
      table-agrees-with-authenticated-openings composition-table
      ( $\text{scale} * \text{clength}$ ) ( $\text{composition-openings} ! i$ )
    using can and i-bound unfolding partial-composition-table-candidate-def
      by blast
    show ?thesis
      by (rule partial-composition-table-candidate-single-roundI
        [OF i-bound len-table agrees])
  qed

lemma staged-query-prefix-candidate-opening-query-target-single-roundI:
  assumes i-bound:  $i < \text{rounds}$ 
  and consistent:
    partial-query-openings-consistent trace-openings composition-openings
    as idx
  shows
     $\text{idx} \in$ 
    staged-query-prefix-candidate-opening-query-target
    ((replicate rounds []) [ $i := \text{trace-openings}$ ])
    ((replicate rounds []) [ $i := \text{composition-openings}$ ])
    as i prefix prefix-state
  unfolding staged-query-prefix-candidate-opening-query-target-def

```

by (rule partial-query-success-indices-atI,
 rule partial-query-round-consistent-single-roundI
 [OF i-bound consistent])

lemma *checked-staged-query-prefix-candidate-opening-target-single-round-hitI*:

assumes *i-bound*: $i < \text{rounds}$

and *consistent*:

partial-query-openings-consistent trace-openings composition-openings
as (index (to-nat raw))

shows

checked-staged-query-prefix-dynamic-index-hit
 (*staged-query-prefix-candidate-opening-query-target*
 ((*replicate rounds* []) [$i := \text{trace-openings}$])
 ((*replicate rounds* []) [$i := \text{composition-openings}$])
 as *i*)
 (Some (((*prefix*, *prefix-state*), *raw*), *raw-state*))

proof –

have *index (to-nat raw) ∈*

staged-query-prefix-candidate-opening-query-target
 ((*replicate rounds* []) [$i := \text{trace-openings}$])
 ((*replicate rounds* []) [$i := \text{composition-openings}$])
 as *i prefix prefix-state*

by (rule *staged-query-prefix-candidate-opening-query-target-single-roundI*
 [OF *i-bound consistent*])

then show *?thesis*

unfolding *checked-staged-query-prefix-dynamic-index-hit-def* **by** *simp*

qed

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI*:

assumes *as-eq*: $as = \text{staged-alphas data}$

and *i-bound*: $i < \text{rounds}$

and *builder*:

Some (*data*, *attacker-state*) ∈
set-dist
 (*execute (checked-staged-transcript-program A)*
adversary-initial-state)

and *prefix-receive*:

Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) ∈
set-dist
 (*execute (checked-staged-query-prefix-receive-with-state A i)*
adversary-initial-state)

and *target-hit*:

checked-staged-query-prefix-dynamic-index-hit
 (*staged-query-prefix-candidate-opening-query-target*
trace-openings composition-openings as i)
 (Some (((*prefix*, *prefix-state*), *raw*), *raw-state*))

shows

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
trace-openings composition-openings as i

```

    (Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
      final-state)))
proof –
  have body:
    ∃ prefix prefix-state raw raw-state.
      Some (data, attacker-state) ∈
        set-dist
          (execute (checked-staged-transcript-program A)
            adversary-initial-state) ∧
      Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
          (execute (checked-staged-query-prefix-receive-with-state A i)
            adversary-initial-state) ∧
      checked-staged-query-prefix-dynamic-index-hit
        (staged-query-prefix-candidate-opening-query-target
          trace-openings composition-openings as i)
        (Some (((prefix, prefix-state), raw), raw-state))
  by (intro exI[of - prefix] exI[of - prefix-state]
    exI[of - raw] exI[of - raw-state])
    (use builder prefix-receive target-hit in simp)
  show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-def
  using as-eq i-bound body by simp
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI-from-row*:

```

assumes as-eq: as = staged-alphas data
and i-bound: i < rounds
and builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
and consistent:
  partial-query-openings-consistent trace-openings composition-openings
    as (index (to-nat raw))
shows
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
  as i
  (Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
    final-state)))

```


proof –

have *target-hit*:

checked-staged-query-prefix-dynamic-index-hit
 (*staged-query-prefix-candidate-opening-query-target*
 ((*replicate rounds* []) [*i* := *trace-openings*])
 ((*replicate rounds* []) [*i* := *composition-openings*])
 as *i*)

(*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*))

by (*rule checked-staged-query-prefix-candidate-opening-target-single-round-hitI*
 [*OF i-bound consistent*])

show *?thesis*

by (*rule*

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI
 [*OF as-eq i-bound builder prefix-receive target-hit*])

qed

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI-from-prefix-receive*:

assumes *i-bound*: *i* < *rounds*

and *builder*:

Some (*data*, *attacker-state*) ∈
set-dist
 (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state)

and *prefix-receive*:

Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) ∈
set-dist
 (*execute* (*checked-staged-query-prefix-receive-with-state A i*)
adversary-initial-state)

and *consistent*:

partial-query-openings-consistent trace-openings composition-openings
 (*staged-alphas data*) (*index (to-nat raw)*)

shows

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
 ((*replicate rounds* []) [*i* := *trace-openings*])
 ((*replicate rounds* []) [*i* := *composition-openings*])
 (*staged-alphas data*) *i*
 (*Some* ((((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*), *result*),
final-state))

by (*rule*

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI-from-raw
 [*OF refl i-bound builder prefix-receive consistent*])

definition *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix*

where

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
trace-openings composition-openings i out $\longleftrightarrow$

(*case out of*

None $\Rightarrow$ *False*

| *Some* ((((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*), *result*),

$final-state) \Rightarrow$
 $i < rounds \wedge$
 $(\exists prefix\ prefix-state\ raw\ raw-state.$
 $\quad Some\ (data,\ attacker-state) \in$
 $\quad\quad set-dist$
 $\quad\quad (execute\ (checked-staged-transcript-program\ A)$
 $\quad\quad\quad adversary-initial-state) \wedge$
 $\quad Some\ (((prefix,\ prefix-state),\ raw),\ raw-state) \in$
 $\quad\quad set-dist$
 $\quad\quad (execute\ (checked-staged-query-prefix-receive-with-state\ A\ i)$
 $\quad\quad\quad adversary-initial-state) \wedge$
 $checked-staged-query-prefix-dynamic-index-hit$
 $\quad (staged-query-prefix-candidate-opening-query-target-from-prefix$
 $\quad\quad trace-openings\ composition-openings\ i)$
 $\quad (Some\ (((prefix,\ prefix-state),\ raw),\ raw-state))))$

definition *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix*
where

$checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix\ A$
 $\quad trace-table\ composition-table\ i\ out \longleftrightarrow$
 $(case\ out\ of$
 $\quad None \Rightarrow False$
 $\quad | Some\ (((((alpha-prefix,\ alpha-prefix-state),\ data),\ attacker-state),\ result),$
 $\quad\quad final-state) \Rightarrow$
 $\quad\quad i < rounds \wedge$
 $\quad\quad (\exists prefix\ prefix-state\ raw\ raw-state.$
 $\quad\quad\quad Some\ (data,\ attacker-state) \in$
 $\quad\quad\quad\quad set-dist$
 $\quad\quad\quad\quad (execute\ (checked-staged-transcript-program\ A)$
 $\quad\quad\quad\quad\quad adversary-initial-state) \wedge$
 $\quad\quad\quad Some\ (((prefix,\ prefix-state),\ raw),\ raw-state) \in$
 $\quad\quad\quad\quad set-dist$
 $\quad\quad\quad\quad (execute\ (checked-staged-query-prefix-receive-with-state\ A\ i)$
 $\quad\quad\quad\quad\quad adversary-initial-state) \wedge$
 $\quad\quad\quad checked-staged-query-prefix-dynamic-index-hit$
 $\quad\quad\quad (staged-query-prefix-candidate-pair-query-target-from-prefix$
 $\quad\quad\quad\quad trace-table\ composition-table)$
 $\quad\quad\quad (Some\ (((prefix,\ prefix-state),\ raw),\ raw-state))))$

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixI:*

assumes *i-bound:* $i < rounds$

and *builder:*

$Some\ (data,\ attacker-state) \in$
 $\quad set-dist$
 $\quad (execute\ (checked-staged-transcript-program\ A)$
 $\quad\quad adversary-initial-state)$

and *prefix-receive:*

$Some\ (((prefix,\ prefix-state),\ raw),\ raw-state) \in$
 $\quad set-dist$

```

      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
    and target-hit:
      checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-opening-query-target-from-prefix
        trace-openings composition-openings i)
      (Some (((prefix, prefix-state), raw), raw-state))
    shows
      checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
      trace-openings composition-openings i
      (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
        final-state))
  proof -
    have body:
       $\exists$  prefix prefix-state raw raw-state.
      Some (data, attacker-state)  $\in$ 
        set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)  $\wedge$ 
      Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
        set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)  $\wedge$ 
      checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-opening-query-target-from-prefix
        trace-openings composition-openings i)
      (Some (((prefix, prefix-state), raw), raw-state))
    by (intro exI[of - prefix] exI[of - prefix-state]
      exI[of - raw] exI[of - raw-state])
      (use builder prefix-receive target-hit in simp)
    show ?thesis
      unfolding
        checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-def
      using i-bound body by simp
  qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixE*:

assumes *hit*:

```

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
trace-openings composition-openings i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
  result), final-state))

```

obtains *prefix prefix-state raw raw-state* **where**

i < rounds

Some (data, attacker-state) $\in$

set-dist

```

(execute (checked-staged-transcript-program A)
  adversary-initial-state)

```

Some (((prefix, prefix-state), raw), raw-state) $\in$

```

    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-opening-query-target-from-prefix
    trace-openings composition-openings i)
  (Some (((prefix, prefix-state), raw), raw-state))
using hit
proof –
have body:
  i < rounds ∧
  (∃ prefix prefix-state raw raw-state.
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state) ∧
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state) ∧
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-opening-query-target-from-prefix
        trace-openings composition-openings i)
      (Some (((prefix, prefix-state), raw), raw-state))))
using hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-def
by simp
then obtain prefix prefix-state raw raw-state where
  i-bound: i < rounds
and builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
and target-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-opening-query-target-from-prefix
      trace-openings composition-openings i)
    (Some (((prefix, prefix-state), raw), raw-state))
by blast
show ?thesis
by (rule that[OF i-bound builder prefix-receive target-hit])
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixI:*

assumes *i-bound: i < rounds*

and *builder:*

Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)

and *prefix-receive:*

Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)

and *target-hit:*

checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((prefix, prefix-state), raw), raw-state))

shows

checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix A
trace-table composition-table i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
final-state)))

proof —

have *body:*

$\exists$ *prefix prefix-state raw raw-state.*
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state) ∧
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state) ∧
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((prefix, prefix-state), raw), raw-state))
by (*intro exI[of - prefix] exI[of - prefix-state]*
exI[of - raw] exI[of - raw-state])
(use builder prefix-receive target-hit in simp)

show *?thesis*

unfolding

checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-def

using *i-bound body by simp*

qed

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE:*

assumes *hit:*

checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix A
trace-table composition-table i
(Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
obtains *prefix prefix-state raw raw-state* **where**
i < rounds
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((prefix, prefix-state), raw), raw-state))
using *hit*
proof –
have *body:*
i < rounds ∧
(∃ prefix prefix-state raw raw-state.
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state) ∧
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state) ∧
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((prefix, prefix-state), raw), raw-state))))
using *hit*
unfolding
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-def
by *simp*
then obtain *prefix prefix-state raw raw-state* **where**
i-bound: i < rounds
and *builder:*
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive:*
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist

```

      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
    and target-hit:
      checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table)
      (Some (((prefix, prefix-state), raw), raw-state))
    by blast
  show ?thesis
    by (rule that[OF i-bound builder prefix-receive target-hit])
qed

lemma checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-imp-candidate-pair-q
  assumes hit:
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
    trace-openings composition-openings i
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  and trace-low: trace-table-low-degree trace-table
  and comp-low: composition-table-low-degree maxDegree composition-table
  and not-all:
     $\bigwedge$  prefix prefix-state.
    Some ((prefix, prefix-state), prefix-state)  $\in$ 
      set-dist
      (execute (checked-staged-query-prefix-with-state A i)
        adversary-initial-state)  $\implies$ 
     $\neg$  all-queries-consistent trace-table composition-table
    (sqp-alphas prefix)
  shows
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix A
    trace-table composition-table i
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
proof -
  from checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixE
    [OF hit]
  obtain prefix prefix-state raw raw-state where
    i-bound:  $i < \text{rounds}$ 
  and builder:
    Some (data, attacker-state)  $\in$ 
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and prefix-receive:

```

Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) ∈
set-dist
 (execute (checked-staged-query-prefix-receive-with-state *A* *i*)
 adversary-initial-state)
and *target-hit*:
 checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-opening-query-target-from-prefix
 trace-openings composition-openings *i*)
 (Some (((*prefix*, *prefix-state*), *raw*), *raw-state*))
by *blast*
have *prefix-support*:
 Some ((*prefix*, *prefix-state*), *prefix-state*) ∈
 set-dist
 (execute (checked-staged-query-prefix-with-state *A* *i*)
 adversary-initial-state)
by (rule checked-staged-query-prefix-receive-with-state-prefix-support
 [OF *prefix-receive*])
have *pair-target-hit*:
 checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-pair-query-target-from-prefix *trace-table*
 composition-table)
 (Some (((*prefix*, *prefix-state*), *raw*), *raw-state*))
proof –
have *subset*:
 staged-query-prefix-candidate-opening-query-target-from-prefix
 trace-openings composition-openings *i* *prefix* *prefix-state* ⊆
 staged-query-prefix-candidate-pair-query-target-from-prefix *trace-table*
 composition-table *prefix* *prefix-state*
by (rule
 staged-query-prefix-candidate-opening-query-target-from-prefix-subset-candidate-pair-from-prefix
 [OF *trace-candidate comp-candidate trace-low comp-low*
 not-all[OF *prefix-support*]])
show ?thesis
using *target-hit subset*
unfolding checked-staged-query-prefix-dynamic-index-hit-def **by** *auto*
qed
show ?thesis
by (rule
 checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixI
 [OF *i-bound builder prefix-receive pair-target-hit*])
qed
lemma checked-staged-query-prefix-candidate-opening-target-from-prefix-single-round-hitI:
assumes *i-bound*: *i* < *rounds*
and *consistent*:
 partial-query-openings-consistent trace-openings composition-openings
 (sqp-alphas *prefix*) (*index* (*to-nat* *raw*))
shows
 checked-staged-query-prefix-dynamic-index-hit


```

    (staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i)
    (Some (((prefix, prefix-state), raw), raw-state))
  proof -
    have index (to-nat raw) ∈
      staged-query-prefix-candidate-opening-query-target-from-prefix
        ((replicate rounds []) [i := trace-openings])
        ((replicate rounds []) [i := composition-openings])
        i prefix prefix-state
    unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
    by (rule partial-query-success-indices-atI,
        rule partial-query-round-consistent-single-roundI
          [OF i-bound consistent])
    then show ?thesis
      unfolding checked-staged-query-prefix-dynamic-index-hit-def by simp
  qed

lemma checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixI-from-prefix-receive:
  assumes i-bound: i < rounds
  and builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and prefix-receive:
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
  and alphas-eq: sqp-alphas prefix = staged-alphas data
  and consistent:
    partial-query-openings-consistent trace-openings composition-openings
      (staged-alphas data) (index (to-nat raw))
  shows
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i
      (Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
        final-state)))
  proof -
    have consistent-prefix:
      partial-query-openings-consistent trace-openings composition-openings
        (sqp-alphas prefix) (index (to-nat raw))
    using consistent alphas-eq by simp
    have target-hit:
      checked-staged-query-prefix-dynamic-index-hit

```

```

    (staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i)
    (Some (((prefix, prefix-state), raw), raw-state))
  by (rule
      checked-staged-query-prefix-candidate-opening-target-from-prefix-single-round-hitI
      [OF i-bound consistent-prefix])
  show ?thesis
  by (rule
      checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixI
      [OF i-bound builder prefix-receive target-hit])
qed

```

lemma *checked-staged-query-prefix-candidate-pair-target-hitI:*

```

assumes hit:
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table as
shows
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target trace-table
      composition-table as)
    (Some (((prefix, prefix-state), raw), raw-state))
using hit
unfolding checked-staged-query-prefix-dynamic-index-hit-def
  staged-query-prefix-candidate-pair-query-target-def
by simp

```

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hitI:*

```

assumes i-bound: i < rounds
and builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
and target-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target trace-table
      composition-table (staged-alphas data))
    (Some (((prefix, prefix-state), raw), raw-state))
shows
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit A
    trace-table composition-table
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),

```

```

    final-state))
proof –
  have body:
     $\exists i$  prefix prefix-state raw raw-state.
       $i < \text{rounds} \wedge$ 
       $\text{Some } (data, \text{attacker-state}) \in$ 
        set-dist
        (execute (checked-staged-transcript-program A)
         adversary-initial-state)  $\wedge$ 
       $\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in$ 
        set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
         adversary-initial-state)  $\wedge$ 
      checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-pair-query-target trace-table
       composition-table (staged-alphas data))
      ( $\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}))$ )
  by (intro exI[of - i] exI[of - prefix] exI[of - prefix-state]
       exI[of - raw] exI[of - raw-state])
      (use i-bound builder prefix-receive target-hit in simp)
  show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-def
  using body by simp
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-imp-candidate-pair-query-hit*:

```

assumes hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
  trace-openings composition-openings as i
  ( $\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$ 
    result), final-state))
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-low: composition-table-low-degree maxDegree composition-table
and not-all:
   $\neg \text{all-queries-consistent trace-table composition-table as}$ 
shows
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit A
  trace-table composition-table
  ( $\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$ 
    result), final-state))

```

```

proof –
  obtain prefix prefix-state raw raw-state where
    as-eq: as = staged-alphas data

```

and *i-bound*: $i < \text{rounds}$
and *builder*:
Some (*data*, *attacker-state*) $\in$
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive*:
Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *target-hit*:
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target
trace-openings composition-openings as i)
(Some (((prefix, prefix-state), raw), raw-state))
using *hit*
unfolding
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-def
by *auto*
have *pair-target-hit*:
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target trace-table
composition-table as)
(Some (((prefix, prefix-state), raw), raw-state))
by (*rule checked-staged-query-prefix-dynamic-index-hit-mono*
 $[OF - \text{target-hit}]$)
(rule
staged-query-prefix-candidate-opening-query-target-subset-candidate-pair
 $[OF \text{ trace-candidate comp-candidate trace-low comp-low not-all}]$)
have *pair-target-hit'*:
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target trace-table
composition-table (staged-alphas data))
(Some (((prefix, prefix-state), raw), raw-state))
using *pair-target-hit as-eq* **by** *simp*
show *?thesis*
by (*rule checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hitI*
 $[OF \text{ i-bound builder prefix-receive pair-target-hit}']$)
qed

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hitI-from-raw*:
assumes *i-bound*: $i < \text{rounds}$
and *builder*:
Some (*data*, *attacker-state*) $\in$
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive*:

Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *target-raw:*
index (to-nat raw) ∈
query-sampling-success-space trace-table composition-table
(staged-alphas data)
shows
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit A
trace-table composition-table
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
final-state)))
proof –
have *target-hit:*
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target trace-table
composition-table (staged-alphas data))
(Some (((prefix, prefix-state), raw), raw-state))
by *(rule checked-staged-query-prefix-candidate-pair-target-hitI*
[OF target-raw])
show *?thesis*
by *(rule*
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hitI
[OF i-bound builder prefix-receive target-hit])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hitE:*
assumes *hit:*
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit A
trace-table composition-table
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
final-state)))
obtains *i prefix prefix-state raw raw-state where*
i < rounds
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target trace-table
composition-table (staged-alphas data))
(Some (((prefix, prefix-state), raw), raw-state))
using *hit*
proof –

have *body*:
 $\exists i$ *prefix prefix-state raw raw-state*.
 $i < \text{rounds} \wedge$
Some (*data, attacker-state*) $\in$
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state) $\wedge$
Some (((*prefix, prefix-state*), *raw*), *raw-state*) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state) $\wedge$
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target trace-table
composition-table (staged-alphas data))
(Some (((prefix, prefix-state), raw), raw-state))
using *hit*
unfolding
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-def
by *simp*
then obtain *i prefix prefix-state raw raw-state* **where**
i-bound: i < rounds
and *builder*:
Some (*data, attacker-state*) $\in$
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive*:
Some (((*prefix, prefix-state*), *raw*), *raw-state*) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *target-hit*:
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target trace-table
composition-table (staged-alphas data))
(Some (((prefix, prefix-state), raw), raw-state))
by *blast*
show *?thesis*
by (*rule that[OF i-bound builder prefix-receive target-hit]*)
qed

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-rawE*:
assumes *hit*:
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit A
trace-table composition-table
(Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
final-state)))
obtains *i prefix prefix-state raw raw-state* **where**
i < rounds

Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
index (to-nat raw) ∈
query-sampling-success-space trace-table composition-table
(staged-alphas data)
proof –
from *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hitE*
[OF hit]
obtain *i prefix prefix-state raw raw-state* **where**
i-bound: i < rounds
and *builder:*
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive:*
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *target-hit:*
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target trace-table
composition-table (staged-alphas data))
(Some (((prefix, prefix-state), raw), raw-state))
by *blast*
have *raw-hit:*
index (to-nat raw) ∈
query-sampling-success-space trace-table composition-table
(staged-alphas data)
using *target-hit*
unfolding *checked-staged-query-prefix-dynamic-index-hit-def*
staged-query-prefix-candidate-pair-query-target-def
by *simp*
show *?thesis*
by *(rule that[OF i-bound builder prefix-receive raw-hit])*
qed

lemma *staged-query-prefix-candidate-pair-query-target-fraction-bound:*
assumes *trace-low: trace-table-low-degree trace-table*
and *comp-low: composition-table-low-degree maxDegree composition-table*
and *not-all:*
¬ all-queries-consistent trace-table composition-table as

shows
nnreal
 (*card*
 (*staged-query-prefix-candidate-pair-query-target trace-table*
 composition-table as prefix prefix-state)) /
nnreal (card query-sample-space) ≤ query-error-bound
unfolding *staged-query-prefix-candidate-pair-query-target-def*
by (rule *query-sampling-success-space-fraction-bound-query-sample-space*)

lemma *staged-query-prefix-candidate-opening-query-target-fraction-bound:*
assumes *trace-candidate:*
 partial-trace-table-candidate trace-table trace-openings
and *comp-candidate:*
 partial-composition-table-candidate composition-table
 composition-openings
and *trace-low: trace-table-low-degree trace-table*
and *comp-low: composition-table-low-degree maxDegree composition-table*
and *not-all:*
 $\neg$ *all-queries-consistent trace-table composition-table as*

shows
nnreal
 (*card*
 (*staged-query-prefix-candidate-opening-query-target trace-openings*
 composition-openings as i prefix prefix-state)) /
nnreal (card query-sample-space) ≤ query-error-bound
unfolding *staged-query-prefix-candidate-opening-query-target-def*
by (rule *partial-query-success-indices-at-fraction-bound-if-candidate-low-degree*
 [*OF trace-candidate comp-candidate trace-low comp-low not-all*])

lemma *staged-query-prefix-candidate-opening-query-target-from-prefix-fraction-bound:*
assumes *trace-candidate:*
 partial-trace-table-candidate trace-table trace-openings
and *comp-candidate:*
 partial-composition-table-candidate composition-table
 composition-openings
and *trace-low: trace-table-low-degree trace-table*
and *comp-low: composition-table-low-degree maxDegree composition-table*
and *not-all:*
 $\neg$ *all-queries-consistent trace-table composition-table*
 (*sqp-alphas prefix*)

shows
nnreal
 (*card*
 (*staged-query-prefix-candidate-opening-query-target-from-prefix*
 trace-openings composition-openings i prefix prefix-state)) /
nnreal (card query-sample-space) ≤ query-error-bound
unfolding *staged-query-prefix-candidate-opening-query-target-from-prefix-def*
by (rule *partial-query-success-indices-at-fraction-bound-if-candidate-low-degree*
 [*OF trace-candidate comp-candidate trace-low comp-low not-all*])

lemma *staged-query-prefix-candidate-pair-query-target-from-prefix-fraction-bound:*

assumes *trace-low: trace-table-low-degree trace-table*

and *comp-low: composition-table-low-degree maxDegree composition-table*

and *not-all:*

$\neg$ *all-queries-consistent trace-table composition-table*

(sqp-alphas prefix)

shows

nnreal

(card

(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table prefix prefix-state)) /

nnreal (card query-sample-space) $\leq$ query-error-bound

unfolding *staged-query-prefix-candidate-pair-query-target-from-prefix-def*

by *(rule query-sampling-success-space-fraction-bound-query-sample-space)*

lemma *checked-staged-query-prefix-candidate-opening-target-prehit-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: $i \leq$ rounds*

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)

(checked-staged-query-prefix-dynamic-index-prehit

(staged-query-prefix-candidate-opening-query-target
trace-openings composition-openings as i))

adversary-initial-state $\leq$

staged-phase-relation-error size

(staged-query-search-queries budgets $i + 1$)

proof *(rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program)*

show *HashMap adversary-initial-state = fmempty*

by *(simp add: adversary-initial-state-def)*

show *hash-relation-program*

(checked-staged-query-prefix-dynamic-index-prehit-relation A i
adversary-initial-state

(staged-query-prefix-candidate-opening-query-target
trace-openings composition-openings as i))

size (staged-query-search-queries budgets $i + 1$)

(checked-staged-query-prefix-receive-with-state A i)

by *(rule hash-relation-program-checked-staged-query-prefix-receive-with-state*
[OF wf controlled i-bound])

(rule

checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)

qed

lemma *checked-staged-query-prefix-candidate-opening-target-hit-bound:*

assumes *raw-bound: query-index-raw-preimage-bound*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: $i \leq$ rounds*

and *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
and *comp-candidate*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-low*: *composition-table-low-degree maxDegree composition-table*
and *not-all*:
 $\neg$ *all-queries-consistent trace-table composition-table as*
shows
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target
trace-openings composition-openings as i))
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) + query-error-bound
proof –
have *hit-bound*:
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target
trace-openings composition-openings as i))
adversary-initial-state $\leq$
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-prehit
(staged-query-prefix-candidate-opening-query-target
trace-openings composition-openings as i))
adversary-initial-state + query-error-bound
proof (rule
checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
[OF raw-bound])
fix *prefix prefix-state*
assume *support*:
Some ((prefix, prefix-state), prefix-state) $\in$
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state)
have *subset*:
staged-query-prefix-candidate-opening-query-target trace-openings
composition-openings as i prefix prefix-state $\subseteq$
query-sample-space
by (rule *staged-query-prefix-candidate-opening-query-target-subset*)
have *frac*:
nnreal
(card
(staged-query-prefix-candidate-opening-query-target
trace-openings composition-openings as i prefix prefix-state)) /
nnreal (card query-sample-space) $\leq$ query-error-bound

```

  by (rule staged-query-prefix-candidate-opening-query-target-fraction-bound
    [OF trace-candidate comp-candidate trace-low comp-low not-all])
show
  staged-query-prefix-candidate-opening-query-target trace-openings
    composition-openings as i prefix prefix-state  $\subseteq$ 
    query-sample-space  $\wedge$ 
    nnreal
    (card
      (staged-query-prefix-candidate-opening-query-target
        trace-openings composition-openings as i prefix prefix-state)) /
    nnreal (card query-sample-space)  $\leq$  query-error-bound
  using subset frac by blast
qed
have prehit-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-candidate-opening-query-target
        trace-openings composition-openings as i))
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1)
  by (rule checked-staged-query-prefix-candidate-opening-target-prehit-bound
    [OF wf controlled i-bound])
have tail:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-candidate-opening-query-target
        trace-openings composition-openings as i))
  adversary-initial-state + query-error-bound  $\leq$ 
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + query-error-bound
  by (intro add-mono prehit-bound order-refl)
show ?thesis
  by (rule order-trans[OF hit-bound tail])
qed

lemma checked-staged-query-prefix-candidate-opening-target-from-prefix-prehit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i  $\leq$  rounds
  shows
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-prehit
        (staged-query-prefix-candidate-opening-query-target-from-prefix
          trace-openings composition-openings i))
    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1)
proof (rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program)

```

```

show HashMap adversary-initial-state = fmempty
  by (simp add: adversary-initial-state-def)
show hash-relation-program
  (checked-staged-query-prefix-dynamic-index-prehit-relation A i
   adversary-initial-state
   (staged-query-prefix-candidate-opening-query-target-from-prefix
    trace-openings composition-openings i))
  size (staged-query-search-queries budgets i + 1)
  (checked-staged-query-prefix-receive-with-state A i)
by (rule hash-relation-program-checked-staged-query-prefix-receive-with-state
     [OF wf controlled i-bound])
  (rule
   checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
qed

lemma checked-staged-query-prefix-candidate-opening-target-from-prefix-hit-bound:
assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i ≤ rounds
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-low: composition-table-low-degree maxDegree composition-table
and not-all:
   $\bigwedge \text{prefix prefix-state.}$ 
   $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$ 
  set-dist
  (execute (checked-staged-query-prefix-with-state A i)
   adversary-initial-state)  $\implies$ 
   $\neg$  all-queries-consistent trace-table composition-table
  (sqp-alphas prefix)
shows
  wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-hit
   (staged-query-prefix-candidate-opening-query-target-from-prefix
    trace-openings composition-openings i))
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) + query-error-bound
proof –
have hit-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-hit
   (staged-query-prefix-candidate-opening-query-target-from-prefix
    trace-openings composition-openings i))

```

$adversary-initial-state \leq$
 $wp-event (checked-staged-query-prefix-receive-with-state A i)$
 $(checked-staged-query-prefix-dynamic-index-prehit$
 $(staged-query-prefix-candidate-opening-query-target-from-prefix$
 $trace-openings composition-openings i))$
 $adversary-initial-state + query-error-bound$

proof (rule
 $checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error$
 $[OF raw-bound])$

fix $prefix prefix-state$

assume $support:$
 $Some ((prefix, prefix-state), prefix-state) \in$
 $set-dist$
 $(execute (checked-staged-query-prefix-with-state A i)$
 $adversary-initial-state)$

have $subset:$
 $staged-query-prefix-candidate-opening-query-target-from-prefix$
 $trace-openings composition-openings i prefix prefix-state \subseteq$
 $query-sample-space$

by (rule $staged-query-prefix-candidate-opening-query-target-from-prefix-subset$)

have $frac:$
 $nnreal$
 $(card$
 $(staged-query-prefix-candidate-opening-query-target-from-prefix$
 $trace-openings composition-openings i prefix prefix-state)) /$
 $nnreal (card query-sample-space) \leq query-error-bound$

by (rule
 $staged-query-prefix-candidate-opening-query-target-from-prefix-fraction-bound$
 $[OF trace-candidate comp-candidate trace-low comp-low not-all[OF support]])$

show
 $staged-query-prefix-candidate-opening-query-target-from-prefix$
 $trace-openings composition-openings i prefix prefix-state \subseteq$
 $query-sample-space \wedge$
 $nnreal$
 $(card$
 $(staged-query-prefix-candidate-opening-query-target-from-prefix$
 $trace-openings composition-openings i prefix prefix-state)) /$
 $nnreal (card query-sample-space) \leq query-error-bound$

using $subset frac$ **by** $blast$

qed

have $prehit-bound:$
 $wp-event (checked-staged-query-prefix-receive-with-state A i)$
 $(checked-staged-query-prefix-dynamic-index-prehit$
 $(staged-query-prefix-candidate-opening-query-target-from-prefix$
 $trace-openings composition-openings i))$
 $adversary-initial-state \leq$
 $staged-phase-relation-error size$
 $(staged-query-search-queries budgets i + 1)$

by (rule

checked-staged-query-prefix-candidate-opening-target-from-prefix-prehit-bound
 [OF wf controlled i-bound])

have *tail*:
wp-event (*checked-staged-query-prefix-receive-with-state A i*)
 (*checked-staged-query-prefix-dynamic-index-prehit*
 (*staged-query-prefix-candidate-opening-query-target-from-prefix*
trace-openings composition-openings i))
adversary-initial-state + *query-error-bound* ≤
staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) + *query-error-bound*
by (*intro add-mono prehit-bound order-refl*)
show ?thesis
by (*rule order-trans[OF hit-bound tail]*)
qed

lemma *checked-staged-query-prefix-candidate-pair-target-prehit-bound*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i \leq \text{rounds}$
shows
wp-event (*checked-staged-query-prefix-receive-with-state A i*)
 (*checked-staged-query-prefix-dynamic-index-prehit*
 (*staged-query-prefix-candidate-pair-query-target trace-table*
composition-table as))
adversary-initial-state ≤
staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*)
proof (*rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)
show *HashMap adversary-initial-state = fmempty*
by (*simp add: adversary-initial-state-def*)
show *hash-relation-program*
 (*checked-staged-query-prefix-dynamic-index-prehit-relation A i*
adversary-initial-state
 (*staged-query-prefix-candidate-pair-query-target trace-table*
composition-table as))
size (*staged-query-search-queries budgets i + 1*)
 (*checked-staged-query-prefix-receive-with-state A i*)
by (*rule hash-relation-program-checked-staged-query-prefix-receive-with-state*
 [OF wf controlled i-bound])
 (*rule*
checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
qed

lemma *checked-staged-query-prefix-candidate-pair-target-hit-bound*:
assumes *raw-bound*: *query-index-raw-preimage-bound*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i \leq \text{rounds}$
and *trace-low*: *trace-table-low-degree trace-table*

and *comp-low: composition-table-low-degree maxDegree composition-table*
and *not-all:*
 $\neg$ *all-queries-consistent trace-table composition-table as*
shows
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target trace-table
composition-table as))
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) + query-error-bound
proof –
have *hit-bound:*
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target trace-table
composition-table as))
adversary-initial-state $\leq$
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-prehit
(staged-query-prefix-candidate-pair-query-target trace-table
composition-table as))
adversary-initial-state + query-error-bound
proof (rule
checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
[OF raw-bound])
fix *prefix prefix-state*
assume *support:*
Some ((prefix, prefix-state), prefix-state) $\in$
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state)
have *subset:*
staged-query-prefix-candidate-pair-query-target trace-table
composition-table as prefix prefix-state $\subseteq$
query-sample-space
by (rule *staged-query-prefix-candidate-pair-query-target-subset*)
have *frac:*
nnreal
(card
(staged-query-prefix-candidate-pair-query-target trace-table
composition-table as prefix prefix-state)) /
nnreal (card query-sample-space) $\leq$ query-error-bound
by (rule *staged-query-prefix-candidate-pair-query-target-fraction-bound*
[OF trace-low comp-low not-all])
show
staged-query-prefix-candidate-pair-query-target trace-table
composition-table as prefix prefix-state $\subseteq$
query-sample-space $\wedge$

```

nnreal
(card
  (staged-query-prefix-candidate-pair-query-target trace-table
    composition-table as prefix prefix-state)) /
nnreal (card query-sample-space) ≤ query-error-bound
using subset frac by blast
qed
have prehit-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-candidate-pair-query-target trace-table
        composition-table as))
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1)
by (rule checked-staged-query-prefix-candidate-pair-target-prehit-bound
  [OF wf controlled i-bound])
have tail:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-candidate-pair-query-target trace-table
        composition-table as))
  adversary-initial-state + query-error-bound ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) + query-error-bound
by (intro add-mono prehit-bound order-refl)
show ?thesis
by (rule order-trans[OF hit-bound tail])
qed

lemma checked-staged-query-prefix-candidate-pair-target-from-prefix-prehit-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i ≤ rounds
shows
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table))
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1)
proof (rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program)
show HashMap adversary-initial-state = fmempty
by (simp add: adversary-initial-state-def)
show hash-relation-program
  (checked-staged-query-prefix-dynamic-index-prehit-relation A i
    adversary-initial-state
    (staged-query-prefix-candidate-pair-query-target-from-prefix

```


trace-table composition-table))
size (staged-query-search-queries budgets $i + 1$)
(checked-staged-query-prefix-receive-with-state $A\ i$)
by (*rule hash-relation-program-checked-staged-query-prefix-receive-with-state*
[OF wf controlled i -bound])
(rule
checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
qed

lemma *checked-staged-query-prefix-candidate-pair-target-from-prefix-hit-bound:*

assumes *raw-bound: query-index-raw-preimage-bound*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i -bound: $i \leq \text{rounds}$*

and *trace-low: trace-table-low-degree trace-table*

and *comp-low: composition-table-low-degree maxDegree composition-table*

and *not-all:*

$\bigwedge \text{prefix prefix-state.}$

Some ((prefix, prefix-state), prefix-state) $\in$

set-dist

(execute (checked-staged-query-prefix-with-state $A\ i$)

adversary-initial-state) $\implies$

$\neg$ *all-queries-consistent trace-table composition-table*

(sqp-alphas prefix)

shows

wp-event (checked-staged-query-prefix-receive-with-state $A\ i$)

(checked-staged-query-prefix-dynamic-index-hit

(staged-query-prefix-candidate-pair-query-target-from-prefix

trace-table composition-table))

adversary-initial-state $\leq$

staged-phase-relation-error size

(staged-query-search-queries budgets $i + 1$) + query-error-bound

proof –

have *hit-bound:*

wp-event (checked-staged-query-prefix-receive-with-state $A\ i$)

(checked-staged-query-prefix-dynamic-index-hit

(staged-query-prefix-candidate-pair-query-target-from-prefix

trace-table composition-table))

adversary-initial-state $\leq$

wp-event (checked-staged-query-prefix-receive-with-state $A\ i$)

(checked-staged-query-prefix-dynamic-index-prehit

(staged-query-prefix-candidate-pair-query-target-from-prefix

trace-table composition-table))

adversary-initial-state + query-error-bound

proof (*rule*

checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error

[OF raw-bound])

fix *prefix prefix-state*

assume *support:*

Some $((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
set-dist
 (execute (checked-staged-query-prefix-with-state A i)
 adversary-initial-state)
have subset:
 staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table prefix prefix-state $\subseteq$
 query-sample-space
by (rule staged-query-prefix-candidate-pair-query-target-from-prefix-subset)
have frac:
 nnreal
 (card
 (staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table prefix prefix-state)) /
 nnreal (card query-sample-space) $\leq$ query-error-bound
by (rule
 staged-query-prefix-candidate-pair-query-target-from-prefix-fraction-bound)
 (use trace-low comp-low not-all[OF support] in simp-all)
show
 staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table prefix prefix-state $\subseteq$
 query-sample-space $\wedge$
 nnreal
 (card
 (staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table prefix prefix-state)) /
 nnreal (card query-sample-space) $\leq$ query-error-bound
using subset frac **by** blast
qed
have prehit-bound:
 wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-prehit
 (staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table))
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$)
by (rule
 checked-staged-query-prefix-candidate-pair-target-from-prefix-prehit-bound
 [OF wf controlled i-bound])
have tail:
 wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-prehit
 (staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table))
 adversary-initial-state + query-error-bound $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) + query-error-bound
by (intro add-mono prehit-bound order-refl)

```

show ?thesis
  by (rule order-trans[OF hit-bound tail])
qed

lemma checked-staged-query-prefix-candidate-pair-target-from-prefix-hit-bound-from-budgets:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound:  $i < \text{rounds}$ 
    and trace-low: trace-table-low-degree trace-table
    and comp-low: composition-table-low-degree maxDegree composition-table
    and not-all:
       $\bigwedge \text{prefix prefix-state.}$ 
       $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$ 
         $\text{set-dist}$ 
         $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$ 
           $\text{adversary-initial-state}) \implies$ 
         $\neg \text{all-queries-consistent trace-table composition-table}$ 
         $(\text{sqp-alphas prefix})$ 
  shows
     $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$ 
     $(\text{checked-staged-query-prefix-dynamic-index-hit}$ 
       $(\text{staged-query-prefix-candidate-pair-query-target-from-prefix}$ 
         $\text{trace-table composition-table}))$ 
     $\text{adversary-initial-state} \leq$ 
     $\text{staged-phase-relation-error size}$ 
     $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$ 
proof –
  have raw-bound: query-index-raw-preimage-bound
    by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
  show ?thesis
    by (rule
      checked-staged-query-prefix-candidate-pair-target-from-prefix-hit-bound
      [OF raw-bound wf controlled - trace-low comp-low not-all])
      (use i-bound in simp)
qed

end

end

```

```

theory Staged-Security-Experiment-Composition-Query-Residual
imports
  Staged-Security-Experiment-Composition-Partial
  Soundness-Staged-Partial-Query-Branch
  Soundness-Staged-Candidate-Query
begin

```

Composition-specific residual split after the alpha-prequery accounting.

The broad branch-subtree witness event is not rare: honest verifier-side Merkle path re-computation can produce such witnesses. This layer therefore splits the actual remaining composition drift event itself into a part covered by the sampled partial-opening query event and a residual part that is still to be discharged by the Merkle/candidate proof.

context *soundness*
begin

The existing committed query-prefix targets are based on *query-sampling-success-space*. They are suitable for ordinary query inconsistency, but not directly for composition randomization witnesses: *composition-bad-context* already requires all sampled queries to be consistent. Hence the query-sampling success space of the actual composition witness is empty, and the composition-drift query case needs a narrower candidate-indexed target rather than the global partial-opening fixed cover.

lemma *composition-bad-context-query-sampling-success-space-empty:*

assumes

composition-bad-context s out trace-table composition-table as query-idxs f

shows *query-sampling-success-space trace-table composition-table as = {}*

proof –

have *all-queries: all-queries-consistent trace-table composition-table as*

using *assms unfolding composition-bad-context-def* **by** *blast*

then show *?thesis*

unfolding *query-sampling-success-space-def* **by** *simp*

qed

lemma *composition-randomization-bad-witness-query-sampling-success-space-empty:*

assumes *composition-randomization-bad s out*

obtains *trace-table composition-table as query-idxs f* **where**

composition-bad-context s out trace-table composition-table as query-idxs f

common-denominator-degree-bounds f as

random-combination-common-denominator-hides-violations f as

query-sampling-success-space trace-table composition-table as = {}

using *assms*

unfolding *composition-randomization-bad-def*

by (*metis composition-bad-context-query-sampling-success-space-empty*)

lemma *composition-trace-bad-alpha-spaceE:*

assumes *bad: as ∈ composition-trace-bad-alpha-space trace-table*

obtains *f* **where**

f = low-degree-trace-witness trace-table

degree f < clength

trace-table = map (poly f) eval-domain

violated-constraints f ≠ {}

as ∈ common-denominator-hiding-alpha-space f

common-denominator-degree-bounds f as

trace-table-low-degree trace-table

proof –

let *?f = low-degree-trace-witness trace-table*

have *deg: degree ?f < clength*

```

and trace-table: trace-table = map (poly ?f) eval-domain
and violated: violated-constraints ?f ≠ {}
and alpha: as ∈ common-denominator-hiding-alpha-space ?f
and degree-bounds: common-denominator-degree-bounds ?f as
using bad
unfolding composition-trace-bad-alpha-space-def Let-def
by (auto split: if-splits)
have trace-low: trace-table-low-degree trace-table
unfolding trace-table-low-degree-def
using deg trace-table by blast
show ?thesis
by (rule that[OF refl deg trace-table violated alpha degree-bounds
    trace-low])
qed

```

```

lemma composition-trace-bad-alpha-space-trace-low-degree:
  assumes as ∈ composition-trace-bad-alpha-space trace-table
  shows trace-table-low-degree trace-table
  using assms by (elim composition-trace-bad-alpha-spaceE)

```

```

definition checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  :: ((((((f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool

```

where

```

  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    out ⇔
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
    out ∧
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
    out

```

```

definition checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
  :: ((((((f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool

```

where

```

  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
    out ⇔
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
    out ∧
  ¬ checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
    out

```

```

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-imp-drift:
  assumes
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit

```

out
shows
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
 out
using *assms*
unfolding
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
by *simp*

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-imp-not-query-*
assumes
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
 out
shows
 $\neg$ checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
 out
using *assms*
unfolding
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
by *simp*

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-imp-randomiza-*
assumes
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
 out
shows
 checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-randomization-bad out
using *assms*
unfolding
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
by *simp*

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-imp-partial-hea-*
assumes
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
 out
shows
 checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
 composition-trace-bad-alpha-space out
using *assms*
unfolding
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
by *simp*

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-imp-partial-hea-*
assumes

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
out

shows

checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
composition-trace-bad-alpha-space out

using *assms*

unfolding

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def

by *simp*

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-imp-not-prefix-*
assumes

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
out

shows

$\neg$ *checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
composition-trace-bad-alpha-space out

using *assms*

unfolding

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def

by *simp*

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hitE:*
assumes *hit:*

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
(*Some* ((((((*prefix*, *prefix-state*), *data*), *attacker-state*), *result*),
final-state)))

obtains

alpha-header-list-set-hit
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
(*alpha-header-supported-partial-union-bad-sets*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
composition-trace-bad-alpha-space)
(*Some (result, final-state)*)
 $\neg$ *alpha-vector-prequeried-in-state prefix-state*
(*staged-alphas data*) (*length spec*)
 $\neg$ *checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
composition-trace-bad-alpha-space
(*Some* ((((((*prefix*, *prefix-state*), *data*), *attacker-state*), *result*),
final-state)))
 $\neg$ *checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit*
(*Some* ((((((*prefix*, *prefix-state*), *data*), *attacker-state*), *result*),
final-state)))

checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-randomization-bad
(Some (((((prefix, prefix-state), data), attacker-state), result),
final-state)))
using *hit*
unfolding
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def
checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-def
checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-def
Let-def
by *auto*

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-imp-branch-tree*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

out $\in$
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)

and *hit:*

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
out

shows

checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
out

by *(rule*

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-imp-branch-tree-on-support
[OF wf controlled support])

(rule

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-imp-drift
[OF hit])

lemma *checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support:*

assumes *support:*

Some (((((prefix, prefix-state), data), attacker-state), result),
final-state)) $\in$
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)

shows

Some (((data, attacker-state), result), final-state) $\in$
set-dist


```

      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
proof –
  let ?project =
     $\lambda x. ((snd\ (fst\ (fst\ x)),\ snd\ (fst\ x)),\ snd\ x)$ 
  have projected:
    Some (((data, attacker-state), result), final-state)  $\in$ 
      set-dist
        (execute
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
            A  $\gg$  ( $\lambda x. return\ (?project\ x)$ ))
          adversary-initial-state)
  apply (rule set-dist-bindI[OF support])
  apply simp
  done
then show ?thesis
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-query-partial-opening-hi*

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and support:

Some (((((prefix, prefix-state), data), attacker-state), result),
 final-state) $\in$
 set-dist
 (execute
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 adversary-initial-state)

and bad:

checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-randomization-bad
 (Some (((((prefix, prefix-state), data), attacker-state), result),
 final-state))

shows

checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
 (Some (((((prefix, prefix-state), data), attacker-state), result),
 final-state))

proof –

let ?s =

verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)

have data-support:

Some (((data, attacker-state), result), final-state) $\in$
 set-dist

```

    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF data-support]
  obtain builder verifier where
    builder:
      Some (data, attacker-state) ∈
        set-dist
          (execute (checked-staged-transcript-program A)
            adversary-initial-state)
    and verifier:
      Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
  have staged-header:
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
  from bad obtain trace-table composition-table as query-idxs f where
    ctr:
      composition-bad-context ?s (Some (result, final-state))
      trace-table composition-table as query-idxs f
  unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    composition-randomization-bad-def Let-def
  by auto
  have bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
      trace-table composition-table as query-idxs
  using ctr unfolding composition-bad-context-def by blast
  have all-queries:
    all-queries-consistent trace-table composition-table as
  using ctr unfolding composition-bad-context-def by blast
  have shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as query-idxs
  using bound
  unfolding accepted-with-bound-tables-def accepted-with-tables-def
  by simp
  from checked-staged-security-with-data-state-accepted-shape-query-chunks
    [OF wf controlled data-support shape]

```

```

obtain raw-idxs where
  as-eq: as = staged-alphas data
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
  and lookup:
     $\bigwedge i. i < \text{rounds} \implies$ 
      fmlookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)
  and idx-bound:
     $\forall \text{idx} \in \text{set } \text{query-idxs}. \text{idx} < \text{clength} * \text{scale}$ 
  by blast
have bound-staged:
  accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table (staged-alphas data) query-idxs
  using bound as-eq by simp
have all-queries-staged:
  all-queries-consistent trace-table composition-table
    (staged-alphas data)
  using all-queries as-eq by simp
have i-bound: 0 < rounds
  by (rule rounds-positive)
have idx-sample:
  query-idxs ! 0  $\in$  query-sample-space
proof –
  have len-query-idxs: length query-idxs = rounds
    by (rule accepted-with-bound-tables-shapes(4)) [OF bound-staged]
  have query-idxs ! 0  $\in$  set query-idxs
    by (rule nth-mem) (use i-bound len-query-idxs in simp)
  then show ?thesis
    by (rule accepted-with-bound-tables-query-sample-space
      [OF bound-staged])
qed
show ?thesis
proof (rule accepted-with-bound-tables-partial-opening-witnesses-Some
  [OF bound-staged])
  fix fr f-fri-roots f-final dg composition-fri-roots final rest
    trace-openings composition-openings
  assume header:
    verifier-header-transcript ?s fr f-fri-roots f-final
      (staged-alphas data) dg composition-fri-roots final rest
  and comp-nonempty: composition-fri-roots  $\neq$  []
  and trace-partial:
    accepted-with-partial-trace-openings ?s
      (Some (result, final-state)) fr query-idxs trace-openings

```

```

and comp-partial:
  accepted-with-partial-composition-openings ?s
    (Some (result, final-state)) (hd composition-fri-roots)
    query-idxs composition-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
    composition-openings
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF staged-header header]
by simp
have idx-in:
  query-idxs ! 0 ∈
    query-header-supported-partial-opening-success-indices-at ?s fr
    f-fri-roots f-final (staged-alphas data) dg
    composition-fri-roots final 0
by (rule
  query-header-supported-partial-opening-success-indices-atI-from-partials-consistent
  [OF verifier trace-partial comp-partial trace-candidate
    comp-candidate header comp-nonempty refl i-bound idx-sample
    all-queries-staged])
have idx-in-staged:
  query-idxs ! 0 ∈
    query-header-supported-partial-opening-success-indices-at ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      0
using idx-in header-eq by simp
have raw-idx:
  index (to-nat (raw-idxs ! 0)) = query-idxs ! 0
using query-idxs-eq len-raw i-bound by simp
have hit-at:
  staged-security-with-data-state-query-partial-opening-hit-at 0
    (Some (((data, attacker-state), result), final-state))
unfolding staged-security-with-data-state-query-partial-opening-hit-at-def
using i-bound lookup[OF i-bound] idx-in-staged raw-idx

```

```

    by (simp add: Let-def)
  have hit:
    staged-security-with-data-state-query-partial-opening-hit
      (Some (((data, attacker-state), result), final-state))
    by (rule staged-security-with-data-state-query-partial-opening-hitI
        [OF i-bound hit-at])
  show ?thesis
    unfolding
      checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-def
    using hit by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-false-on-support
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    out ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
            A)
          adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
      out
  shows False
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state where
  out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state)
  by (cases packed, auto split: prod.splits)
have support-some:
  Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state) ∈
  set-dist
    (execute

```

```

    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)
  using support unfolding out-eq .
have random-bad:
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-randomization-bad
  (Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state))
  using hit unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
  by simp
have query-hit:
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
  (Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-query-partial-opening-hit-on
    [OF wf controlled support-some random-bad])
have not-query:
  ¬ checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
  (Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state))
  using hit unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
  by simp
show ?thesis
  using query-hit not-query by contradiction
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
    adversary-initial-state ≤ 0
proof -
  have
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
    adversary-initial-state ≤
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state

```

```

    A)
  (λ-. False)
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (use
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-false-on-support
    [OF wf controlled] in blast)
also have ... = 0
  unfolding wp-event-def wp-def dist-expect-def by simp
finally show ?thesis .
qed

```

```

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-imp-query-or-without-query:
assumes
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
  out
shows
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  out ∨
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
  out
using assms
unfolding
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
by blast

```

```

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-query-hit:
assumes
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  out
shows
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
  out
using assms
unfolding
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
by simp

```

```

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-branch-tree-on-sup:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)

```

and *hit*:
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
out
shows
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
out
proof –
have *drift*:
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
out
using *hit*
unfolding
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
by *simp*
show *?thesis*
by (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-imp-branch-tree-on-support
[*OF wf controlled support drift*])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-branch-tree*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *branch-tree-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
adversary-initial-state $\leq T$
shows
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq T$
proof –
have
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq$
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
adversary-initial-state
by (rule *wp-event-mono-on-support*)
(rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-branch-tree-on-support
[*OF wf controlled*])
then show *?thesis*


```

    by (rule order-trans[OF - branch-tree-bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-driftE:
  assumes hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
      (Some (((prefix, prefix-state), data), attacker-state), result),
      final-state)
  obtains s where
    s = verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    composition-randomization-bad s (Some (result, final-state))
    alpha-header-list-set-hit s
      (alpha-header-supported-partial-union-bad-sets s
        composition-trace-bad-alpha-space)
      (Some (result, final-state))
    ¬ staged-alphas data ∈
      alpha-prefix-union-bad-sets prefix-state
        composition-trace-bad-alpha-space (fst prefix)
  proof -
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have random-bad:
    composition-randomization-bad ?s (Some (result, final-state))
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    Let-def
  by simp
  have header-hit:
    alpha-header-list-set-hit ?s
      (alpha-header-supported-partial-union-bad-sets ?s
        composition-trace-bad-alpha-space)
      (Some (result, final-state))
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-def
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-def
    Let-def
  by simp
  have not-prefix:
    ¬ staged-alphas data ∈
      alpha-prefix-union-bad-sets prefix-state
        composition-trace-bad-alpha-space (fst prefix)
  using hit

```

```

unfolding
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit-def
  Let-def
  by simp
show ?thesis
  by (rule that[OF refl random-bad header-hit not-prefix])
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hitE:
assumes hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  (Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state))
obtains s where
  s = verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  composition-randomization-bad s (Some (result, final-state))
  alpha-header-list-set-hit s
    (alpha-header-supported-partial-union-bad-sets s
      composition-trace-bad-alpha-space)
    (Some (result, final-state))
  ¬ staged-alphas data ∈
    alpha-prefix-union-bad-sets prefix-state
      composition-trace-bad-alpha-space (fst prefix)
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
    (Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state))
proof –
have drift:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
  (Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state))
using hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
by simp
have query-hit:
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
  (Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state))
using hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
by simp
from checked-staged-security-with-actual-alpha-prefix-composition-candidate-driftE
  [OF drift]
obtain s where

```

s-def:
s = *verifier-state-from-adversary attacker-state*
(staged-proof-transcript data)
and *random-bad*:
composition-randomization-bad s (Some (result, final-state))
and *header-hit*:
alpha-header-list-set-hit s
(alpha-header-supported-partial-union-bad-sets s
composition-trace-bad-alpha-space)
(Some (result, final-state))
and *not-prefix*:
 $\neg$ *staged-alphas data* $\in$
alpha-prefix-union-bad-sets prefix-state
composition-trace-bad-alpha-space (fst prefix)
by *blast*
show *?thesis*
by (*rule that*[*OF s-def random-bad header-hit not-prefix query-hit*])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supportedE*:

assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *support*:
Some (((((prefix, prefix-state), data), attacker-state), result),
final-state) $\in$
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)
and *hit*:
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
(Some (((((prefix, prefix-state), data), attacker-state), result),
final-state))
obtains *s trace-table* **where**
s = *verifier-state-from-adversary attacker-state*
(staged-proof-transcript data)
composition-randomization-bad s (Some (result, final-state))
staged-alphas data $\in$
alpha-header-supported-partial-union-bad-sets s
composition-trace-bad-alpha-space
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
trace-table $\in$
alpha-header-supported-partial-trace-table-candidates s
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)

```

    staged-alphas data ∈ composition-trace-bad-alpha-space trace-table
  ¬ staged-alphas data ∈
    alpha-prefix-union-bad-sets prefix-state
    composition-trace-bad-alpha-space (fst prefix)
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
    (Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state)))
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hitE
  [OF hit]
have random-bad:
  composition-randomization-bad ?s (Some (result, final-state))
and header-list-hit:
  alpha-header-list-set-hit ?s
    (alpha-header-supported-partial-union-bad-sets ?s
      composition-trace-bad-alpha-space)
    (Some (result, final-state))
and not-prefix:
  ¬ staged-alphas data ∈
    alpha-prefix-union-bad-sets prefix-state
    composition-trace-bad-alpha-space (fst prefix)
and query-hit:
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
    (Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state)))
by blast+
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
by (rule
  checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
  [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
have staged-header:

```

```

verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
from header-list-hit obtain as query-idxs fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  shape: accepted-transcript-shape ?s (Some (result, final-state))
  as query-idxs
and header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and as-bad:
  as ∈ alpha-header-supported-partial-union-bad-sets ?s
  composition-trace-bad-alpha-space fr f-fri-roots f-final
unfolding alpha-header-list-set-hit-def by blast
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
have staged-bad:
  staged-alphas data ∈
    alpha-header-supported-partial-union-bad-sets ?s
    composition-trace-bad-alpha-space
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
using as-bad header-eq by simp
from staged-bad obtain trace-table where trace-candidate:
  trace-table ∈
    alpha-header-supported-partial-trace-table-candidates ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
and trace-bad:
  staged-alphas data ∈ composition-trace-bad-alpha-space trace-table

```

unfolding *alpha-header-supported-partial-union-bad-sets-def* **by** *blast*
show *?thesis*
by (*rule that*[*OF refl random-bad staged-bad trace-candidate trace-bad*
not-prefix query-hit])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supported-openingsE*:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
Some (((((prefix, prefix-state), data), attacker-state), result),
final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)
and *hit:*
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
(Some (((((prefix, prefix-state), data), attacker-state), result),
final-state))
obtains *s trace-table i raw trace-openings composition-openings* **where**
s = verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
composition-randomization-bad s (Some (result, final-state))
trace-table ∈
alpha-header-supported-partial-trace-table-candidates s
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
staged-alphas data ∈ composition-trace-bad-alpha-space trace-table
 $\neg$ *staged-alphas data ∈*
alpha-prefix-union-bad-sets prefix-state
composition-trace-bad-alpha-space (fst prefix)
i < rounds
fmlookup (HashMap final-state)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data)
(staged-query-chunks data) i)) =
Some raw
index (to-nat raw) ∈ query-sample-space
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses s
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)

```

    (staged-composition-fri-roots data)
    (staged-composition-final data)
    partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
proof –
from checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supportedE
    [OF wf controlled support hit]
obtain s trace-table where
  s-def:
    s = verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
and random-bad:
    composition-randomization-bad s (Some (result, final-state))
and trace-candidate:
    trace-table ∈
      alpha-header-supported-partial-trace-table-candidates s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
and trace-bad:
    staged-alphas data ∈ composition-trace-bad-alpha-space trace-table
and not-prefix:
    ¬ staged-alphas data ∈
      alpha-prefix-union-bad-sets prefix-state
      composition-trace-bad-alpha-space (fst prefix)
and query-hit:
    checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
      (Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state)))
by blast
from checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hitE
    [OF query-hit]
obtain i raw trace-openings composition-openings where
  i-bound: i < rounds
and lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some raw
and idx-sample: index (to-nat raw) ∈ query-sample-space
and witness:
    (trace-openings, composition-openings) ∈
      query-header-supported-partial-opening-witnesses
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)

```

```

    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  and consistent:
    partial-query-openings-consistent trace-openings composition-openings
      (staged-alphas data) (index (to-nat raw))
  by blast
have witness-s:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
using witness s-def by simp
show ?thesis
by (rule that[OF s-def random-bad trace-candidate trace-bad not-prefix
  i-bound lookup idx-sample witness-s consistent])
qed

definition checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
:: ('f staged-adversary ⇒
  (((('f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool)
where
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
    A out ⇔
  (case out of
    None ⇒ False
  | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state) ⇒
    (∃ i trace-openings composition-openings.
      checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
        ((replicate rounds []) [i := trace-openings])
        ((replicate rounds []) [i := composition-openings])
        (staged-alphas data) i
        (Some (((((alpha-prefix, alpha-prefix-state), data),
          attacker-state), result), final-state)))))

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hitI:
  assumes component:
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A

```



```

    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    (staged-alphas data) i
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
            final-state))
  shows
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit A
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
            final-state))
  proof -
  have
    ∃ j trace-openings' composition-openings'.
      checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
      ((replicate rounds []) [j := trace-openings'])
      ((replicate rounds []) [j := composition-openings'])
      (staged-alphas data) j
      (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
              final-state))
  by (intro exI[of - i] exI[of - trace-openings]
      exI[of - composition-openings])
      (rule component)
  then show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit-def
  by simp
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-hit*:

assumes *query-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit

adversary-initial-state $\leq Q$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

adversary-initial-state $\leq Q$

by (rule *order.trans[OF - query-bound]*)

(rule *wp-event-mono*,

rule

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-query-hit)

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-q*:

assumes *query-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

```

    adversary-initial-state  $\leq Q$ 
and without-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
       A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
      adversary-initial-state  $\leq W$ 
shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
      adversary-initial-state  $\leq Q + W$ 
proof –
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?Drift =
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
  let ?Query =
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  let ?Without =
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
  have wp-event ?M ?Drift adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda out. ?Query\ out \vee ?Without\ out$ )
    adversary-initial-state
  by (rule wp-event-mono)
    (rule
     checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-imp-query-or-without-query)
  also have ...  $\leq$ 
    wp-event ?M ?Query adversary-initial-state +
    wp-event ?M ?Without adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq Q + W$ 
  by (intro add-mono query-bound without-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-composition-candidate-drift-and-budgets:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
    adversary-initial-state  $\leq D$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state  $\leq$ 

```

```

    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D +
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
    staged-challenge-query-budget + verifier-hash-query-budget)
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?Prefix =
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0)
  let ?Pre =
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  let ?Coll =
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
    staged-challenge-query-budget + verifier-hash-query-budget)
  have transcript-bound:
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?Prefix
  by (rule
    checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
    [OF wf controlled])
  have prefix-bound:
    wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?Prefix
  by (rule
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
    [OF transcript-bound])
  have prequery-bound:
    wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?Pre
  by (rule

```

```

    checked-staged-security-with-actual-alpha-prefix-vector-alpha-prequery-accounting
    [OF wf controlled])
have collision-bound:
  wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
     hash-map-output-collision-bad)
  adversary-initial-state  $\leq$  ?Coll
by (rule
  checked-staged-security-with-actual-alpha-prefix-hash-map-output-collision-bad-bound-controlled
  [OF wf controlled])
let ?degree =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-degree-bad
let ?random =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-randomization-bad
let ?bad =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-bad
have projected:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
  adversary-initial-state =
  wp-event ?M ?bad adversary-initial-state
by (rule
  checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
have event-eq: ?bad = ( $\lambda$ out. ?degree out  $\vee$  ?random out)
by (rule ext)
  (auto simp:
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  composition-bad-def split: option.splits prod.splits)
have degree-false: ?degree = ( $\lambda$ out. False)
by (rule ext)
  (auto simp:
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  composition-degree-bad-false[OF spec-degree-wellformed-from-spec-query-margin]
  split: option.splits prod.splits)
have degree-bound:
  wp-event ?M ?degree adversary-initial-state  $\leq$  0
unfolding degree-false wp-event-def wp-def dist-expect-def by simp
have random-bound:
  wp-event ?M ?random adversary-initial-state  $\leq$ 
  ?Prefix + ?Pre + D + ?Coll
by (rule
  checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-bound-from-prefix-prequery-drift-
  [OF prefix-bound prequery-bound drift-bound collision-bound])
have wp-event ?M ?bad adversary-initial-state  $\leq$ 
  wp-event ?M ?degree adversary-initial-state +
  wp-event ?M ?random adversary-initial-state

```

```

    unfolding event-eq by (rule wp-event-union-bound)
  also have ...  $\leq 0 + (?Prefix + ?Pre + D + ?Coll)$ 
    by (intro add-mono degree-bound random-bound)
  finally show ?thesis
    unfolding projected by (simp add: add.assoc)
qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-partial-opening-hit-and-budgets:
  fixes query-error :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and partial-opening-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-partial-opening-hit
      adversary-initial-state  $\leq$  query-error
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event composition-bad)
      adversary-initial-state  $\leq$ 
      composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      (query-error + 0) +
      hash-collision-budget-value 0
      (staged-attacker-query-budget budgets +
      staged-challenge-query-budget + verifier-hash-query-budget)
  proof -
    let ?M =
      checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  have query-bound:
    wp-event ?M
      checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
      adversary-initial-state  $\leq$  query-error
  unfolding
    checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-projection
  by (rule partial-opening-bound)
  have drift-query-bound:
    wp-event ?M
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
      adversary-initial-state  $\leq$  query-error
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-hit
    [OF query-bound])
  have without-bound:
    wp-event ?M

```

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
adversary-initial-state ≤ 0
by (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-bound
[OF *wf controlled*])
have *drift-bound*:
wp-event ?*M*
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
adversary-initial-state $\leq$ *query-error* + 0
by (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-query
[OF *drift-query-bound without-bound*])
show ?*thesis*
by (rule
checked-staged-security-with-data-state-composition-bad-bound-from-composition-candidate-drift-and-budgets
[OF *wf controlled drift-bound*])
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-partial-query-residual-and-budgets*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *cover*:
 $\bigwedge$ *data attacker-state i*.
i < *rounds* $\implies$
query-header-supported-partial-opening-success-indices-at
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
(*staged-trace-root data*)
(*staged-trace-fri-roots data*)
(*staged-trace-final data*)
(*staged-alphas data*)
(*staged-degree data*)
(*staged-composition-fri-roots data*)
(*staged-composition-final data*)
i $\subseteq B$
and *raw-bound*: *query-index-raw-preimage-bound*
and *subset*: *B* $\subseteq$ *query-sample-space*
and *frac*:
 $\text{nnreal}(\text{card } B) / \text{nnreal}(\text{card } \text{query-sample-space}) \leq$
query-error-bound
and *residual-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
adversary-initial-state $\leq R$
shows
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event composition-bad*)

$$\begin{aligned}
& \text{adversary-initial-state} \leq \\
& \text{composition-error-bound} + \\
& \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{hash-collision-budget-value } 0 \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& (\text{nnreal} \\
& \quad (\text{staged-attacker-query-budget budgets} + \\
& \quad \quad \text{staged-challenge-query-budget}) * \text{query-error-bound} + R) + \\
& \text{hash-collision-budget-value } 0 \\
& \quad (\text{staged-attacker-query-budget budgets} + \\
& \quad \quad \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) \\
\text{proof } - \\
\text{let } ?M = & \\
& \text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A \\
\text{let } ?QueryCost = & \\
& \text{nnreal} \\
& \quad (\text{staged-attacker-query-budget budgets} + \\
& \quad \quad \text{staged-challenge-query-budget}) * \text{query-error-bound} \\
\text{have query-bound:} & \\
& \text{wp-event } ?M \\
& \quad \text{checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit} \\
& \quad \text{adversary-initial-state} \leq ?QueryCost \\
\text{by (rule} & \\
& \quad \text{checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-bound-from-fixed-query-error-cover} \\
& \quad \quad [OF \text{ wf controlled cover raw-bound subset frac}]) \\
\text{have drift-query-bound:} & \\
& \text{wp-event } ?M \\
& \quad \text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit} \\
& \quad \text{adversary-initial-state} \leq ?QueryCost \\
\text{by (rule} & \\
& \quad \text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-hit} \\
& \quad \quad [OF query-bound]) \\
\text{have drift-bound:} & \\
& \text{wp-event } ?M \\
& \quad \text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift} \\
& \quad \text{adversary-initial-state} \leq ?QueryCost + R \\
\text{by (rule} & \\
& \quad \text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-query} \\
& \quad \quad [OF drift-query-bound residual-bound]) \\
\text{have} & \\
& \text{wp-event (checked-staged-security-experiment-with-data-state } A) \\
& \quad (\text{staged-security-with-data-state-verifier-event composition-bad}) \\
& \quad \text{adversary-initial-state} \leq \\
& \quad \text{composition-error-bound} + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) +
\end{aligned}$$

```

    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    (?QueryCost + R) +
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-composition-candidate-drift-and-budgets
    [OF wf controlled drift-bound])
  then show ?thesis
  by (simp add: add.assoc)
qed

```

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-partial-query-and-budgets:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *cover:*

$\bigwedge \text{data attacker-state } i.$

$i < \text{rounds} \implies$

query-header-supported-partial-opening-success-indices-at

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

(staged-trace-root data)

(staged-trace-fri-roots data)

(staged-trace-final data)

(staged-alphas data)

(staged-degree data)

(staged-composition-fri-roots data)

(staged-composition-final data)

$i \subseteq B$

and *raw-bound: query-index-raw-preimage-bound*

and *subset: $B \subseteq \text{query-sample-space}$*

and *frac:*

$\text{nnreal} (\text{card } B) / \text{nnreal} (\text{card } \text{query-sample-space}) \leq$

query-error-bound

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event composition-bad)

adversary-initial-state $\leq$

composition-error-bound +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

hash-collision-budget-value 0

(staged-alpha-search-queries budgets 0) +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

nnreal


```

      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget) * query-error-bound +
      hash-collision-budget-value 0
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget + verifier-hash-query-budget)
proof –
  have residual-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
       A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
      adversary-initial-state ≤ 0
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-bound
    [OF wf controlled])
  have
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state ≤
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    (nnreal
     (staged-attacker-query-budget budgets +
      staged-challenge-query-budget) * query-error-bound + 0) +
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-partial-query-residual-and-budgets
    [OF wf controlled cover raw-bound subset frac residual-bound])
  then show ?thesis
    by simp
qed

end

end

```

```

theory Soundness-Query-Opening-Extraction
imports Soundness-Partial-Merkle
begin

```

Low-level verifier query extraction lemmas used by staged soundness.

This layer is deliberately kept below the staged event definitions: it only talks about verifier query rounds, authenticated openings, transcript chunks, and random-oracle lookups.

context *soundness*

begin

lemma *query-opening-fri-layer-opening-step-partial-authenticated-table-first-value:*

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes $\text{len-pos}: 0 < \text{len}$

and $\text{idx-bound}: \text{idx} < \text{len}$

and *outcome:*

$\text{Some } (out, t) \in$

set-dist

$(\text{execute } (\text{fri-layer-opening-step } (b, rt) (\text{idx}, x, \text{len}, pw))\ s)$

obtains *openings* **where**

$\text{partial-authenticated-table } rt\ \text{len } openings\ t$

$\text{map } \text{opening-index } openings = [\text{idx}, (\text{idx} + \text{len} \text{ div } 2) \bmod \text{len}]$

$\text{length } openings = 2$

$\text{opening-value } (openings\ !\ 0) = x$

proof –

from $\text{outcome}[\text{unfolded } \text{fri-layer-opening-step-unfold-finish}]$

obtain $xp\ s1\ \text{xp-path } s2\ xn\ s3\ \text{xn-path } s4$ **where**

$\text{read-xp}: \text{Some } (xp, s1) \in \text{set-dist } (\text{execute } \text{read } s)$

and $\text{read-xp-path}:$

$\text{Some } (xp\text{-path}, s2) \in$

$\text{set-dist } (\text{execute } (\text{ntimes } \text{read } (\text{floor-log } \text{len}))\ s1)$

and $\text{read-xn}: \text{Some } (xn, s3) \in \text{set-dist } (\text{execute } \text{read } s2)$

and $\text{read-xn-path}:$

$\text{Some } (xn\text{-path}, s4) \in$

$\text{set-dist } (\text{execute } (\text{ntimes } \text{read } (\text{floor-log } \text{len}))\ s3)$

and *finish:*

$\text{Some } (out, t) \in$

set-dist

$(\text{execute}$

$(\text{fri-layer-opening-finish } b\ rt\ \text{idx } x\ \text{len } pw\ xp\ \text{xp-path } xn$

$xn\text{-path})\ s4)$

by $(\text{auto } \text{elim!}: \text{set-dist-bindE})$

have $\text{xp-path-length}: \text{length } xp\text{-path} = \text{floor-log } \text{len}$

using $\text{ntimes-read-any-outcome}[OF\ \text{read-xp-path}]$ **by** *simp*

have $\text{xn-path-length}: \text{length } xn\text{-path} = \text{floor-log } \text{len}$

using $\text{ntimes-read-any-outcome}[OF\ \text{read-xn-path}]$ **by** *simp*

have $\text{sibling-bound}: (\text{idx} + \text{len} \text{ div } 2) \bmod \text{len} < \text{len}$

using len-pos **by** *simp*

have $\text{xp-x}: xp = x$

using *finish*

unfolding $\text{fri-layer-opening-finish-def } \text{assert-def}$

by $(\text{auto } \text{simp}: \text{throw-no-outcome } \text{elim!}: \text{set-dist-bindE } \text{split}: \text{if-splits})$

from $\text{fri-layer-opening-finish-authenticated-openings}$

$[OF\ \text{idx-bound } \text{sibling-bound } \text{xp-path-length } \text{xn-path-length } \text{finish}]$

obtain $\text{xp-opening } \text{xn-opening}$ **where**

```

xp-eq:
  xp-opening =
    (| opening-root = rt,
      opening-length = len,
      opening-index = idx,
      opening-value = xp,
      opening-path = xp-path)
  and xn-eq:
    xn-opening =
      (| opening-root = rt,
        opening-length = len,
        opening-index = (idx + len div 2) mod len,
        opening-value = xn,
        opening-path = xn-path)
  and xp-auth: authenticated-opening-in t xp-opening
  and xn-auth: authenticated-opening-in t xn-opening
  by blast
  let ?openings = [xp-opening, xn-opening]
  have table: partial-authenticated-table rt len ?openings t
    unfolding partial-authenticated-table-def
    using xp-eq xn-eq xp-auth xn-auth by simp
  have indices:
    map opening-index ?openings = [idx, (idx + len div 2) mod len]
    using xp-eq xn-eq by simp
  have first-value: opening-value (?openings ! 0) = x
    using xp-eq xp-x by simp
  show ?thesis
    by (rule that[OF table indices - first-value]) simp
qed

```

```

lemma query-opening-ntimes-outcome-decomp-at:
  assumes outcome:
    Some (results, t) ∈ set-dist (execute (ntimes m n) s)
  and i-bound: i < n
  shows
    ∃ prefix x suffix s-i s-suc.
      Some (prefix, s-i) ∈ set-dist (execute (ntimes m i) s) ∧
      Some (x, s-suc) ∈ set-dist (execute m s-i) ∧
      Some (suffix, t) ∈
        set-dist (execute (ntimes m (n - Suc i)) s-suc) ∧
      results = prefix @ x # suffix
  using outcome i-bound
proof (induction n arbitrary: i s results t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.prem1 obtain x results' s1 where
    head: Some (x, s1) ∈ set-dist (execute m s)

```

```

and tail:
  Some (results', t) ∈ set-dist (execute (ntimes m n) s1)
and results-eq: results = x # results'
by (auto elim!: set-dist-bindE)
show ?case
proof (cases i)
  case 0
  have prefix0:
    Some ([], s) ∈ set-dist (execute (ntimes m i) s)
    using 0 by simp
  have suffix0:
    Some (results', t) ∈
      set-dist (execute (ntimes m (Suc n − Suc i)) s1)
    using tail 0 by simp
  show ?thesis
    using prefix0 head suffix0 results-eq
    by (intro exI[of - []] exI[of - x] exI[of - results']
      exI[of - s] exI[of - s1]) simp
next
  case (Suc j)
  have j-bound: j < n
    using Suc.prems(2) Suc by simp
  from Suc.IH[OF tail j-bound]
  obtain prefix y suffix s-j s-suc where
    prefix:
      Some (prefix, s-j) ∈ set-dist (execute (ntimes m j) s1)
    and round: Some (y, s-suc) ∈ set-dist (execute m s-j)
    and suffix:
      Some (suffix, t) ∈
        set-dist (execute (ntimes m (n − Suc j)) s-suc)
    and results'-eq: results' = prefix @ y # suffix
    by blast
  have prefix-suc:
    Some (x # prefix, s-j) ∈
      set-dist (execute (ntimes m (Suc j)) s)
    unfolding ntimes.simps
    apply (rule set-dist-bindI[OF head])
    apply (rule set-dist-bindI[OF prefix])
    apply simp
    done
  have n-minus: Suc n − Suc (Suc j) = n − Suc j
    by simp
  have suffix-suc:
    Some (suffix, t) ∈
      set-dist (execute (ntimes m (Suc n − Suc i)) s-suc)
    using suffix n-minus Suc by simp
  show ?thesis
    using prefix-suc round suffix-suc results-eq results'-eq Suc
    by (intro exI[of - x # prefix] exI[of - y] exI[of - suffix])

```

```

      exI[of - s-j] exI[of - s-suc]) simp
qed
qed

lemma query-opening-verifier-query-round-program-authenticated-with-lookup:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-eq: fl = (b, composition-root) # fl-tail
  and outcome:
    Some (((), t) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  obtains raw idx trace-leaves trace-openings composition-openings where
    idx = index (to-nat raw)
    map opening-value trace-openings = trace-leaves
    map opening-index trace-openings = powers-scaled idx
    partial-authenticated-table fr (scale * clength) trace-openings t
    partial-authenticated-table composition-root (scale * clength)
      composition-openings t
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx]
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof -
  from outcome[unfolded verifier-query-round-program-alt-def]
  obtain raw s0 where rand:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
  and tail:
    Some (((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s0)
  by (auto elim!: set-dist-bindE)
  have lookup-s0:
    fmlookup (HashMap s0)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using receive-query-index-challenge-outcome[OF rand] by simp
  have lookup-t:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using verifier-query-round-after-index-program-preserves-query-lookup
    [OF tail, of PQueryCounter s PState s] lookup-s0
  by simp
  show ?thesis
proof (rule verifier-query-round-after-index-authenticated-trace-openings
  [OF tail])
  fix idx trace-leaves trace-openings

```

```

assume idx-eq: idx = index (to-nat raw)
and trace-values:
  map opening-value trace-openings = trace-leaves
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table:
  partial-authenticated-table fr (scale * clength)
    trace-openings t
show ?thesis
proof (rule
  verifier-query-round-after-index-authenticated-composition-openings
  [OF fl-eq tail])
fix idx' composition-openings
assume idx'-eq: idx' = index (to-nat raw)
and comp-table:
  partial-authenticated-table composition-root (scale * clength)
    composition-openings t
and comp-indices:
  map opening-index composition-openings =
    [idx', fri-sibling-index (scale * clength) idx']
have idx'-idx: idx' = idx
  using idx-eq idx'-eq by simp
show ?thesis
  by (rule that[OF idx-eq trace-values trace-indices trace-table
    comp-table - lookup-t])
    (use comp-indices idx'-idx in simp)
qed
qed
qed

lemma query-opening-verifier-query-round-program-authenticated-with-chunk:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-eq: fl = (b, composition-root) # fl-tail
and outcome:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
obtains raw idx chunk trace-leaves trace-openings composition-openings
where
  idx = index (to-nat raw)
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
  PTranscript s = chunk @ PTranscript t
  PState t = foldl concat (PState s) chunk
  s ≤ t
  PQueryCounter t = Suc (PQueryCounter s)
  map opening-value trace-openings = trace-leaves
  map opening-index trace-openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) trace-openings t

```

```

partial-authenticated-table composition-root (scale * clength)
  composition-openings t
map opening-index composition-openings =
  [idx, fri-sibling-index (scale * clength) idx]
fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof -
show ?thesis
proof (rule verifier-query-round-program-outcome[OF outcome])
fix raw0 idx0 chunk
assume idx0-eq: idx0 = index (to-nat raw0)
and chunk-shape:
  verifier-query-round-chunk idx0 (map snd f-fl) (map snd fl)
  chunk
and transcript:
  PTranscript s = chunk @ PTranscript t
and state:
  PState t = foldl concat (PState s) chunk
and ext: s ≤ t
and counter:
  PQueryCounter t = Suc (PQueryCounter s)
and lookup0:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw0
show ?thesis
proof (rule
  query-opening-verifier-query-round-program-authenticated-with-lookup
    [OF fl-eq outcome])
fix raw idx trace-leaves trace-openings composition-openings
assume idx-eq: idx = index (to-nat raw)
and trace-values:
  map opening-value trace-openings = trace-leaves
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table:
  partial-authenticated-table fr (scale * clength)
  trace-openings t
and comp-table:
  partial-authenticated-table composition-root (scale * clength)
  composition-openings t
and comp-indices:
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
and lookup:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
have raw-eq: raw = raw0
  using lookup lookup0 by simp
have idx-idx0: idx = idx0

```

```

    using idx-eq idx0-eq raw-eq by simp
  show ?thesis
    by (rule that[OF idx-eq - transcript state ext counter
      trace-values trace-indices trace-table comp-table comp-indices
      lookup])
      (use chunk-shape idx-idx0 in simp)
qed
qed
qed

```

lemma *query-opening-ntimes-prefix-state-alignment:*
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes *outcome:*
Some (prefix, t) ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr f-fl f-final as fl final) i)
s)
obtains *prefix-chunks* **where**
length prefix-chunks = i
PState t = state-after-query-chunks (PState s) prefix-chunks i
PQueryCounter t = PQueryCounter s + i
s ≤ t
proof –
from *ntimes-verifier-query-rounds-outcome[OF outcome]*
obtain *raw-idxs query-idxs prefix-chunks* **where**
len-chunks: length prefix-chunks = i
and *state:*
PState t = state-after-query-chunks (PState s) prefix-chunks i
and *ext: s ≤ t*
and *counter: PQueryCounter t = PQueryCounter s + i*
by *blast*
show *?thesis*
by (rule that[OF *len-chunks state counter ext*])
qed

lemma *query-opening-ntimes-prefix-transcript-state-alignment:*
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes *outcome:*
Some (prefix, t) ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr f-fl f-final as fl final) i)
s)
obtains *prefix-query-idxs prefix-chunks* **where**
length prefix-query-idxs = i
length prefix-chunks = i


```

PTranscript s = List.concat prefix-chunks @ PTranscript t
PState t = state-after-query-chunks (PState s) prefix-chunks i
PQueryCounter t = PQueryCounter s + i
s ≤ t
 $\bigwedge j. j < i \implies$ 
  verifier-query-round-chunk (prefix-query-idxs ! j)
  (map snd f-fl) (map snd fl) (prefix-chunks ! j)
proof –
from ntimes-verifier-query-rounds-outcome[OF outcome]
obtain raw-idxs query-idxs prefix-chunks where
  len-raw-idxs: length raw-idxs = i
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and
  len-chunks: length prefix-chunks = i
and transcript:
  PTranscript s = List.concat prefix-chunks @ PTranscript t
and state:
  PState t = state-after-query-chunks (PState s) prefix-chunks i
and ext: s ≤ t
and counter: PQueryCounter t = PQueryCounter s + i
and rounds:
   $\bigwedge j. j < i \implies$ 
    verifier-query-round-chunk (query-idxs ! j)
    (map snd f-fl) (map snd fl) (prefix-chunks ! j)
by blast
have len-idxs: length query-idxs = i
using len-raw-idxs query-idxs-eq by simp
show ?thesis
by (rule that[OF len-idxs len-chunks transcript state counter ext])
  (use rounds in blast)
qed

lemma query-opening-ntimes-suffix-extends:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (suffix, t) ∈
    set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final) n)
      s)
shows s ≤ t
proof –
from ntimes-verifier-query-rounds-outcome[OF outcome]
show ?thesis
by blast
qed

```

lemma *query-opening-ntimes-suffix-transcript-state-alignment*:
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes *outcome*:
Some (*suffix*, *t*) $\in$
set-dist
(execute
(ntimes
(verifier-query-round-program fr f-fl f-final as fl final) n)
s)
obtains *suffix-query-idxs suffix-chunks* **where**
length suffix-query-idxs = n
length suffix-chunks = n
PTranscript s = List.concat suffix-chunks @ PTranscript t
PState t = state-after-query-chunks (PState s) suffix-chunks n
PQueryCounter t = PQueryCounter s + n
s ≤ t
 $\bigwedge j. j < n \implies$
verifier-query-round-chunk (suffix-query-idxs ! j)
(map snd f-fl) (map snd fl) (suffix-chunks ! j)
proof –
from *ntimes-verifier-query-rounds-outcome[OF outcome]*
obtain *raw-idxs query-idxs suffix-chunks* **where**
len-raw-idxs: length raw-idxs = n
and *query-idxs-eq*:
query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and *len-chunks: length suffix-chunks = n*
and *transcript*:
PTranscript s = List.concat suffix-chunks @ PTranscript t
and *state*:
PState t = state-after-query-chunks (PState s) suffix-chunks n
and *ext: s ≤ t*
and *counter: PQueryCounter t = PQueryCounter s + n*
and *rounds*:
 $\bigwedge j. j < n \implies$
verifier-query-round-chunk (query-idxs ! j)
(map snd f-fl) (map snd fl) (suffix-chunks ! j)
by *blast*
have *len-idxs: length query-idxs = n*
using *len-raw-idxs query-idxs-eq* **by** *simp*
show *?thesis*
by (*rule that[OF len-idxs len-chunks transcript state counter ext]*)
(use rounds in blast)
qed

lemma *query-opening-ntimes-authenticated-initial-openings-at*:
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes *fl-eq: fl = (b, composition-root) # fl-tail*
and *outcome*:
Some (results, t) ∈

```

    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s)
  and i-bound:  $i < n$ 
obtains raw idx prefix-chunks trace-openings composition-openings where
  idx = index (to-nat raw)
  length prefix-chunks = i
  map opening-index trace-openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) trace-openings t
  partial-authenticated-table composition-root (scale * clength)
    composition-openings t
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + i)
      (state-after-query-chunks (PState s) prefix-chunks i)) = Some raw
proof –
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix round suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) i)
          s)
  and round:
    Some (round, s-suc) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s-i)
  and suffix:
    Some (suffix, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
              (n - Suc i))
          s-suc)
  by blast
have round-unit: round = ()
  by (cases round) simp
show ?thesis
proof (rule
  query-opening-verifier-query-round-program-authenticated-with-chunk
  [OF fl-eq round[unfolded round-unit]])
fix raw idx chunk trace-leaves trace-openings composition-openings

```

```

assume idx-eq: idx = index (to-nat raw)
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
  chunk
and transcript: PTranscript s-i = chunk @ PTranscript s-suc
and state-suc: PState s-suc = foldl concat (PState s-i) chunk
and ext-round: s-i ≤ s-suc
and counter-suc: PQueryCounter s-suc = Suc (PQueryCounter s-i)
and trace-values: map opening-value trace-openings = trace-leaves
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table-suc:
  partial-authenticated-table fr (scale * clength)
  trace-openings s-suc
and comp-table-suc:
  partial-authenticated-table composition-root (scale * clength)
  composition-openings s-suc
and comp-indices:
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
and lookup-suc:
  fmlookup (HashMap s-suc)
  (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
show ?thesis
proof (rule query-opening-ntimes-prefix-state-alignment[OF prefix])
fix prefix-chunks
assume len-chunks: length prefix-chunks = i
and state-i:
  PState s-i =
    state-after-query-chunks (PState s) prefix-chunks i
and counter-i: PQueryCounter s-i = PQueryCounter s + i
have suffix-ext: s-suc ≤ t
by (rule query-opening-ntimes-suffix-extends[OF suffix])
have trace-table-t:
  partial-authenticated-table fr (scale * clength)
  trace-openings t
by (rule
    partial-authenticated-table-mono[OF trace-table-suc
    suffix-ext])
have comp-table-t:
  partial-authenticated-table composition-root (scale * clength)
  composition-openings t
by (rule
    partial-authenticated-table-mono[OF comp-table-suc suffix-ext])
have lookup-t:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s + i)
    (state-after-query-chunks (PState s) prefix-chunks i)) =

```

```

    Some raw
    using hash-extension-lookup[OF lookup-suc suffix-ext]
    unfolding counter-i state-i .
  show ?thesis
    by (rule that[OF idx-eq len-chunks trace-indices trace-table-t
        comp-table-t comp-indices lookup-t])
qed
qed
qed
end
end
end

```

```

theory Soundness-Query-Opening-Consistency
imports
  Soundness-Query-Opening-Extraction
  Soundness-Staged-Partial-Query
begin

```

Verifier-local query-opening consistency facts. These lemmas package the composition check enforced by a successful query round together with the authenticated openings extracted from that round.

```

context soundness
begin

```

lemma *verifier-query-round-after-index-authenticated-openings-consistent*:

```

fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-eq: fl = (b, composition-root) # fl-tail
and outcome:
  Some (( ), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s)
obtains idx trace-openings composition-openings where
  idx = index (to-nat raw)
  map opening-index trace-openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) trace-openings t
  partial-authenticated-table composition-root (scale * clength)
    composition-openings t
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
  partial-query-openings-consistent trace-openings composition-openings
    as idx

```

```

proof –
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where

```

```

query-decommit:
  Some (fv, s1) ∈
    set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
and trace-fri:
  Some (f-out, s2) ∈
    set-dist (execute
      (mfold (?idx, hd fv, clength * scale, 1)
        (receive-query-commits f-fl)) s1)
and assert-trace:
  Some ((), s3) ∈
    set-dist
      (execute
        (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
and comp-fri:
  Some (c-out, s4) ∈
    set-dist (execute
      (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
        clength * scale, 1)
        (receive-query-commits fl)) s3)
and assert-comp:
  Some ((), t) ∈
    set-dist
      (execute
        (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
unfolding verifier-query-round-after-index-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have idx-sample: ?idx ∈ query-sample-space
using index-less-query-sample-space
unfolding query-sample-space-def by simp
from check-decommit-on-query-authenticated-openings
  [OF idx-sample query-decommit]
obtain trace-openings where
  trace-values: map opening-value trace-openings = fv
and trace-indices:
  map opening-index trace-openings = powers-scaled ?idx
and trace-table-s1:
  partial-authenticated-table fr (scale * clength) trace-openings s1
by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where s1-s2: s1 ≤ s2
by blast
have s3-eq: s3 = s2
using assert-trace unfolding assert-def
by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
  (auto simp: throw-no-outcome)
have comp-steps:
  receive-query-commits fl =
    fri-layer-opening-step (b, composition-root) #
    receive-query-commits fl-tail

```

```

unfolding fl-eq receive-query-commits-def fri-layer-opening-step-def
by simp
from comp-fri[unfolded comp-steps]
obtain out1 s-head where
  head:
    Some (out1, s-head) ∈
      set-dist
        (execute
          (fri-layer-opening-step (b, composition-root)
            (?idx, cp-eval as fv (h ^ ?idx * shift),
              clength * scale, 1)) s3)
and tail:
    Some (c-out, s4) ∈
      set-dist (execute (mfold out1 (receive-query-commits fl-tail))
        s-head)
by (auto elim!: set-dist-bindE)
have len-pos: 0 < clength * scale
using eval-domain-nontrivial by linarith
have idx-bound: ?idx < clength * scale
by (rule index-less-domain)
from query-opening-fri-layer-opening-step-partial-authenticated-table-first-value
  [OF len-pos idx-bound head]
obtain composition-openings where
  comp-table-head:
    partial-authenticated-table composition-root (clength * scale)
    composition-openings s-head
and comp-indices0:
  map opening-index composition-openings =
    [?idx, (?idx + (clength * scale) div 2) mod (clength * scale)]
and comp-len: length composition-openings = 2
and comp-value:
  opening-value (composition-openings ! 0) =
    cp-eval as fv (h ^ ?idx * shift)
by blast
from fri-layer-opening-step-outcome[OF head]
have s3-s-head: s3 ≤ s-head
by blast
obtain i1 x1 len1 pw1 where out1-eq: out1 = (i1, x1, len1, pw1)
by (cases out1)
from receive-query-commits-layers-outcome[OF tail[unfolded out1-eq]]
obtain tail-layer-chunks tail-chunk where s-head-s4: s-head ≤ s4
by blast
have t-eq: t = s4
using assert-comp unfolding assert-def
by (cases case c-out of (-, x, -, -)  $\Rightarrow$  x = final)
  (auto simp: throw-no-outcome)
have s1-t: s1 ≤ t
using s1-s2 s3-s-head s-head-s4 unfolding s3-eq t-eq
by (meson hash-ext-trans)

```

```

have trace-table-t:
  partial-authenticated-table fr (scale * clength) trace-openings t
  by (rule partial-authenticated-table-mono[OF trace-table-s1 s1-t])
have comp-table-t:
  partial-authenticated-table composition-root (scale * clength)
    composition-openings t
  using partial-authenticated-table-mono[OF comp-table-head s-head-s4]
  unfolding t-eq by (simp add: mult.commute)
have comp-indices:
  map opening-index composition-openings =
    [?idx, fri-sibling-index (scale * clength) ?idx]
  using comp-indices0
  unfolding fri-sibling-index-def
  by (simp add: mult.commute)
have consistent:
  partial-query-openings-consistent trace-openings composition-openings
    as ?idx
  by (rule partial-query-openings-consistentI
    [OF idx-sample trace-indices comp-indices])
    (use comp-value trace-values in simp)
show ?thesis
  by (rule that[OF refl trace-indices trace-table-t comp-table-t
    comp-indices consistent])
qed

```

lemma verifier-query-round-after-index-empty-composition-consistent:

```

fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-empty: fl = []
and outcome:
  Some (((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s)
obtains idx trace-openings where
  idx = index (to-nat raw)
  map opening-index trace-openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) trace-openings t
  cp-eval as (map opening-value trace-openings) (h ^ idx * shift) =
    final

```

proof –

```

let ?idx = index (to-nat raw)
from outcome obtain fv s1 f-out s2 s3 c-out s4 where
  query-decommit:
    Some (fv, s1) ∈
      set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
and trace-fri:
  Some (f-out, s2) ∈
    set-dist (execute

```



```

    (mfold (?idx, hd fv, clength * scale, 1)
      (receive-query-commits f-fl)) s1)
and assert-trace:
  Some (( ), s3) ∈
    set-dist
      (execute
        (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
and comp-fri:
  Some (c-out, s4) ∈
    set-dist (execute
      (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
        clength * scale, 1)
        (receive-query-commits fl)) s3)
and assert-comp:
  Some (( ), t) ∈
    set-dist
      (execute
        (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
unfolding verifier-query-round-after-index-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have idx-sample: ?idx ∈ query-sample-space
using index-less-query-sample-space
unfolding query-sample-space-def by simp
from check-decommit-on-query-authenticated-openings
  [OF idx-sample query-decommit]
obtain trace-openings where
  trace-values: map opening-value trace-openings = fv
and trace-indices:
  map opening-index trace-openings = powers-scaled ?idx
and trace-table-s1:
  partial-authenticated-table fr (scale * clength) trace-openings s1
by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where s1-s2: s1 ≤ s2
by blast
have s3-eq: s3 = s2
using assert-trace unfolding assert-def
by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
  (auto simp: throw-no-outcome)
have comp-out:
  c-out =
    (?idx, cp-eval as fv (h ^ ?idx * shift), clength * scale, 1) ∧
    s4 = s3
using comp-fri fl-empty
unfolding receive-query-commits-def by simp
have final-eq:
  cp-eval as fv (h ^ ?idx * shift) = final
proof (rule ccontr)
  assume ¬ cp-eval as fv (h ^ ?idx * shift) = final

```

```

    then have Some ( $()$ ,  $t$ )  $\in$  set-dist (execute throw s3)
      using assert-comp comp-out unfolding assert-def by simp
    then show False
      by (simp add: throw-no-outcome)
  qed
  have t-eq:  $t = s4$ 
    using assert-comp comp-out final-eq unfolding assert-def by simp
  have t-s2:  $t = s2$ 
    using s3-eq t-eq comp-out by simp
  have s1-t:  $s1 \leq t$ 
    using s1-s2 t-s2 by simp
  have trace-table-t:
    partial-authenticated-table fr (scale * clength) trace-openings t
    by (rule partial-authenticated-table-mono[OF trace-table-s1 s1-t])
  have final-eq':
    cp-eval as (map opening-value trace-openings) ( $h \wedge ?idx * shift$ ) =
      final
    using final-eq trace-values by simp
  show ?thesis
    by (rule that[OF refl trace-indices trace-table-t final-eq'])
  qed

lemma verifier-query-round-program-authenticated-openings-consistent-with-lookup:
  fixes  $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$ 
  assumes fl-eq:  $fl = (b, \text{composition-root}) \# fl\text{-tail}$ 
  and outcome:
    Some ( $()$ ,  $t$ )  $\in$ 
      set-dist
        (execute
          (verifier-query-round-program fr  $f\text{-fl}\ f\text{-final}$  as  $fl\ final$ )  $s$ )
  obtains raw idx trace-openings composition-openings where
     $idx = \text{index } (to\text{-nat } raw)$ 
     $\text{map } opening\text{-index } trace\text{-openings} = \text{powers-scaled } idx$ 
    partial-authenticated-table fr (scale * clength) trace-openings t
    partial-authenticated-table composition-root (scale * clength)
      composition-openings t
     $\text{map } opening\text{-index } composition\text{-openings} =$ 
      [idx, fri-sibling-index (scale * clength) idx]
    partial-query-openings-consistent trace-openings composition-openings
      as idx
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw

proof –
  from outcome[unfolded verifier-query-round-program-alt-def]
  obtain raw s0 where rand:
    Some (raw, s0)  $\in$ 
      set-dist (execute receive-query-index-challenge s)
  and tail:
    Some ( $()$ ,  $t$ )  $\in$ 

```

```

    set-dist
    (execute
      (verifier-query-round-after-index-program
        fr f-fl f-final as fl final raw) s0)
  by (auto elim!: set-dist-bindE)
have lookup-s0:
  fmlookup (HashMap s0)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using receive-query-index-challenge-outcome[OF rand] by simp
have lookup-t:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using verifier-query-round-after-index-program-preserves-query-lookup
    [OF tail, of PQueryCounter s PState s] lookup-s0
  by simp
show ?thesis
proof (rule verifier-query-round-after-index-authenticated-openings-consistent
  [OF fl-eq tail])
  fix idx trace-openings composition-openings
  assume idx-eq: idx = index (to-nat raw)
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx
  and trace-table:
    partial-authenticated-table fr (scale * clength) trace-openings t
  and comp-table:
    partial-authenticated-table composition-root (scale * clength)
      composition-openings t
  and comp-indices:
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx]
  and consistent:
    partial-query-openings-consistent trace-openings
      composition-openings as idx
  show ?thesis
  by (rule that[OF idx-eq trace-indices trace-table comp-table
    comp-indices consistent lookup-t])
qed
qed

lemma verifier-query-round-program-empty-composition-consistent-with-lookup:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-empty: fl = []
  and outcome:
    Some ((), t) ∈
      set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
  obtains raw idx trace-openings where
    idx = index (to-nat raw)

```

```

    map opening-index trace-openings = powers-scaled idx
    partial-authenticated-table fr (scale * clength) trace-openings t
    cp-eval as (map opening-value trace-openings) (h ^ idx * shift) =
      final
    fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof -
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where rand:
  Some (raw, s0) ∈
    set-dist (execute receive-query-index-challenge s)
and tail:
  Some (( ), t) ∈
    set-dist
    (execute
      (verifier-query-round-after-index-program
        fr f-fl f-final as fl final raw) s0)
by (auto elim!: set-dist-bindE)
have lookup-s0:
  fmlookup (HashMap s0)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
using receive-query-index-challenge-outcome[OF rand] by simp
have lookup-t:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
using verifier-query-round-after-index-program-preserves-query-lookup
  [OF tail, of PQueryCounter s PState s] lookup-s0
by simp
show ?thesis
proof (rule verifier-query-round-after-index-empty-composition-consistent
  [OF fl-empty tail])
fix idx trace-openings
assume idx-eq: idx = index (to-nat raw)
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table:
  partial-authenticated-table fr (scale * clength) trace-openings t
and consistent:
  cp-eval as (map opening-value trace-openings) (h ^ idx * shift) =
    final
show ?thesis
by (rule that[OF idx-eq trace-indices trace-table consistent
  lookup-t])
qed
qed

lemma ntimes-verifier-query-rounds-empty-composition-consistent:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-empty: fl = []

```

and *outcome*:
Some (*results*, *t*) $\in$
set-dist
(*execute*
(*ntimes*
(*verifier-query-round-program* *fr* *f-fl* *f-final* *as* *fl* *final*) *n*)
s)
obtains *raw-idxs* *query-idxs* *trace-openings* **where**
length *raw-idxs* = *n*
query-idxs = *map* (λ *raw. index* (*to-nat* *raw*)) *raw-idxs*
length *trace-openings* = *n*
 $\bigwedge i. i < n \implies$
map *opening-index* (*trace-openings* ! *i*) =
powers-scaled (*query-idxs* ! *i*)
 $\bigwedge i. i < n \implies$
partial-authenticated-table *fr* (*scale* * *clength*)
(*trace-openings* ! *i*) *t*
 $\bigwedge i. i < n \implies$
cp-eval *as* (*map* *opening-value* (*trace-openings* ! *i*))
(*h* ^ (*query-idxs* ! *i*) * *shift*) = *final*
proof –
have
 $\exists$ *raw-idxs* *query-idxs* *trace-openings*.
length *raw-idxs* = *n* $\wedge$
query-idxs = *map* (λ *raw. index* (*to-nat* *raw*)) *raw-idxs* $\wedge$
length *trace-openings* = *n* $\wedge$
($\forall i < n.$
map *opening-index* (*trace-openings* ! *i*) =
powers-scaled (*query-idxs* ! *i*) $\wedge$
($\forall i < n.$
partial-authenticated-table *fr* (*scale* * *clength*)
(*trace-openings* ! *i*) *t*) $\wedge$
($\forall i < n.$
cp-eval *as* (*map* *opening-value* (*trace-openings* ! *i*))
(*h* ^ (*query-idxs* ! *i*) * *shift*) = *final*)
using *outcome*
proof (*induction* *n* *arbitrary*: *s* *results* *t*)
case 0
then show ?*case*
by (*intro* *exI*[*of* - []]) *simp*
next
case (*Suc* *n*)
from *Suc.prem*s **obtain** *u* *results'* *s1* **where**
head:
Some (*u*, *s1*) $\in$
set-dist
(*execute*
(*verifier-query-round-program* *fr* *f-fl* *f-final* *as* *fl* *final*) *s*)
and *tail*:

```

    Some (results', t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            n) s1)
  by (auto elim!: set-dist-bindE)
have u-eq: u = ()
  by (cases u) simp
from verifier-query-round-program-empty-composition-consistent-with-lookup
  [OF fl-empty head[unfolded u-eq]]
obtain raw idx openings where
  idx-eq: idx = index (to-nat raw)
  and idx-openings: map opening-index openings = powers-scaled idx
  and table-s1:
    partial-authenticated-table fr (scale * clength) openings s1
  and consistent-head:
    cp-eval as (map opening-value openings) (h ^ idx * shift) =
      final
  by blast
from Suc.IH[OF tail] obtain raw-tail idx-tail openings-tail where
  len-raw-tail: length raw-tail = n
  and idx-tail-eq:
    idx-tail = map (λraw. index (to-nat raw)) raw-tail
  and len-openings-tail: length openings-tail = n
  and tail-indices:
    ∀ i < n.
      map opening-index (openings-tail ! i) =
        powers-scaled (idx-tail ! i)
  and tail-tables:
    ∀ i < n.
      partial-authenticated-table fr (scale * clength)
        (openings-tail ! i) t
  and tail-consistent:
    ∀ i < n.
      cp-eval as (map opening-value (openings-tail ! i))
        (h ^ (idx-tail ! i) * shift) = final
  by blast
have s1-t: s1 ≤ t
  using ntimes-verifier-query-rounds-outcome[OF tail] by blast
have table-t:
  partial-authenticated-table fr (scale * clength) openings t
  by (rule partial-authenticated-table-mono[OF table-s1 s1-t])
let ?raws = raw # raw-tail
let ?idxs = idx # idx-tail
let ?openings = openings # openings-tail
have idxs-eq:
  ?idxs = map (λraw. index (to-nat raw)) ?raws
  using idx-eq idx-tail-eq by simp

```

```

have indices-all:
   $\forall i < \text{Suc } n.$ 
     $\text{map opening-index } (?openings ! i) =$ 
       $\text{powers-scaled } (?idxs ! i)$ 
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{Suc } n$ 
  show  $\text{map opening-index } (?openings ! i) =$ 
     $\text{powers-scaled } (?idxs ! i)$ 
proof (cases i)
  case 0
  then show ?thesis
    using idx-openings by simp
next
  case (Suc j)
  then show ?thesis
    using i-bound tail-indices by simp
qed
qed
have tables-all:
   $\forall i < \text{Suc } n.$ 
     $\text{partial-authenticated-table fr } (\text{scale} * \text{clength})$ 
       $(?openings ! i) \ t$ 
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{Suc } n$ 
  show  $\text{partial-authenticated-table fr } (\text{scale} * \text{clength})$ 
     $(?openings ! i) \ t$ 
proof (cases i)
  case 0
  then show ?thesis
    using table-t by simp
next
  case (Suc j)
  then show ?thesis
    using i-bound tail-tables by simp
qed
qed
have consistent-all:
   $\forall i < \text{Suc } n.$ 
     $\text{cp-eval as } (\text{map opening-value } (?openings ! i))$ 
       $(h \wedge (?idxs ! i) * \text{shift}) = \text{final}$ 
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{Suc } n$ 
  show  $\text{cp-eval as } (\text{map opening-value } (?openings ! i))$ 
     $(h \wedge (?idxs ! i) * \text{shift}) = \text{final}$ 
proof (cases i)
  case 0

```

```

    then show ?thesis
      using consistent-head by simp
  next
    case (Suc j)
    then show ?thesis
      using i-bound tail-consistent by simp
  qed
qed
show ?case
  apply (intro exI[of - ?rows] exI[of - ?idxs]
    exI[of - ?openings] conjI)
    apply (simp add: len-row-tail)
    apply (rule idxs-eq)
    apply (simp add: len-openings-tail)
    apply (rule indices-all)
    apply (rule tables-all)
    apply (rule consistent-all)
  done
qed
then obtain raw-idxs query-idxs trace-openings where
  len-row: length raw-idxs = n
  and query-idxs-eq:
    query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
  and len-trace: length trace-openings = n
  and indices:
     $\forall i < n.$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
  and tables:
     $\forall i < n.$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) t
  and consistent:
     $\forall i < n.$ 
    cp-eval as (map opening-value (trace-openings ! i))
      ( $h \wedge$  (query-idxs ! i) * shift) = final
  by blast
show ?thesis
  by (rule that[OF len-row query-idxs-eq len-trace])
    (use indices tables consistent in auto)
qed

lemma ntimes-verifier-query-rounds-empty-composition-consistent-with-lookup:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-empty: fl = []
  and outcome:
    Some (results, t)  $\in$ 
      set-dist
      (execute

```



```

      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final) n)
      s)
obtains raw-idxs query-idxs query-chunks trace-openings where
  length raw-idxs = n
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  length query-chunks = n
  length trace-openings = n
  ∧i. i < n ⇒
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) query-chunks i)) =
      Some (raw-idxs ! i)
  ∧i. i < n ⇒
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
  ∧i. i < n ⇒
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) t
  ∧i. i < n ⇒
    cp-eval as (map opening-value (trace-openings ! i))
      (h ^ (query-idxs ! i) * shift) = final
proof -
have
  ∃ raw-idxs query-idxs query-chunks trace-openings.
    length raw-idxs = n ∧
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs ∧
    length query-chunks = n ∧
    length trace-openings = n ∧
    (∀ i < n.
      fmlookup (HashMap t)
        (QueryIndexChallenge (PQueryCounter s + i)
          (state-after-query-chunks (PState s) query-chunks i)) =
        Some (raw-idxs ! i)) ∧
    (∀ i < n.
      map opening-index (trace-openings ! i) =
        powers-scaled (query-idxs ! i)) ∧
    (∀ i < n.
      partial-authenticated-table fr (scale * clength)
        (trace-openings ! i) t) ∧
    (∀ i < n.
      cp-eval as (map opening-value (trace-openings ! i))
        (h ^ (query-idxs ! i) * shift) = final)
using outcome
proof (induction n arbitrary: s results t)
case 0
then show ?case
  by (intro exI[of - []]) simp
next

```

```

case (Suc n)
from Suc.premis obtain u results' s1 where
  head:
    Some (u, s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results', t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            n) s1)
  by (auto elim!: set-dist-bindE)
have u-eq: u = ()
  by (cases u) simp
from verifier-query-round-program-empty-composition-consistent-with-lookup
  [OF fl-empty head[unfolded u-eq]]
obtain raw idx openings where
  idx-eq: idx = index (to-nat raw)
  and idx-openings: map opening-index openings = powers-scaled idx
  and table-s1:
    partial-authenticated-table fr (scale * clength) openings s1
  and consistent-head:
    cp-eval as (map opening-value openings) (h ^ idx * shift) =
      final
  and lookup-s1:
    fmlookup (HashMap s1)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  by blast
from verifier-query-round-program-outcome[OF head[unfolded u-eq]]
obtain raw' idx' chunk where
  idx'-eq: idx' = index (to-nat raw')
  and st-s1: PState s1 = foldl concat (PState s) chunk
  and query-count-s1: PQueryCounter s1 = Suc (PQueryCounter s)
  and lookup-s1':
    fmlookup (HashMap s1)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw'
  by blast
have raw'-eq: raw' = raw
  using lookup-s1 lookup-s1' by simp
from Suc.IH[OF tail] obtain raw-tail idx-tail chunk-tail openings-tail
  where len-raw-tail: length raw-tail = n
  and idx-tail-eq:
    idx-tail = map (λraw. index (to-nat raw)) raw-tail
  and len-chunk-tail: length chunk-tail = n
  and len-openings-tail: length openings-tail = n
  and lookup-tail:

```

```

   $\forall i < n.$ 
     $\text{fmlookup } (\text{HashMap } t)$ 
       $(\text{QueryIndexChallenge } (P\text{QueryCounter } s1 + i)$ 
         $(\text{state-after-query-chunks } (P\text{State } s1) \text{ chunk-tail } i)) =$ 
         $\text{Some } (\text{raw-tail } ! i)$ 
and tail-indices:
   $\forall i < n.$ 
     $\text{map opening-index } (\text{openings-tail } ! i) =$ 
     $\text{powers-scaled } (\text{idx-tail } ! i)$ 
and tail-tables:
   $\forall i < n.$ 
     $\text{partial-authenticated-table fr } (\text{scale} * \text{clength})$ 
     $(\text{openings-tail } ! i) \ t$ 
and tail-consistent:
   $\forall i < n.$ 
     $\text{cp-eval as } (\text{map opening-value } (\text{openings-tail } ! i))$ 
     $(h \wedge (\text{idx-tail } ! i) * \text{shift}) = \text{final}$ 
by blast
have s1-t:  $s1 \leq t$ 
  using ntimes-verifier-query-rounds-outcome[OF tail] by blast
have table-t:
   $\text{partial-authenticated-table fr } (\text{scale} * \text{clength}) \text{ openings } t$ 
  by  $(\text{rule } \text{partial-authenticated-table-mono}[\text{OF } \text{table-s1 } s1\text{-t}])$ 
have lookup-head-t:
   $\text{fmlookup } (\text{HashMap } t)$ 
     $(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some } \text{raw}$ 
  using hash-extension-lookup[OF lookup-s1 s1-t] .
let ?raws = raw # raw-tail
let ?idxs = idx # idx-tail
let ?chunks = chunk # chunk-tail
let ?openings = openings # openings-tail
have idxs-eq:
   $?idxs = \text{map } (\lambda \text{raw. index } (\text{to-nat } \text{raw})) \ ?raws$ 
  using idx-eq idx-tail-eq by simp
have state-shift:
   $\bigwedge i. \text{state-after-query-chunks } (P\text{State } s) \ ?chunks \ (\text{Suc } i) =$ 
   $\text{state-after-query-chunks } (P\text{State } s1) \ \text{chunk-tail } i$ 
  using st-s1 unfolding state-after-query-chunks-def by simp
have lookup-all:
   $\forall i < \text{Suc } n.$ 
     $\text{fmlookup } (\text{HashMap } t)$ 
       $(\text{QueryIndexChallenge } (P\text{QueryCounter } s + i)$ 
         $(\text{state-after-query-chunks } (P\text{State } s) \ ?chunks \ i)) =$ 
         $\text{Some } (?raws \ ! \ i)$ 
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{Suc } n$ 
  show  $\text{fmlookup } (\text{HashMap } t)$ 
     $(\text{QueryIndexChallenge } (P\text{QueryCounter } s + i)$ 

```

```

      (state-after-query-chunks (PState s) ?chunks i)) =
      Some (?rows ! i)
proof (cases i)
  case 0
  then show ?thesis
    using lookup-head-t unfolding state-after-query-chunks-def by simp
next
  case (Suc j)
  then show ?thesis
    using i-bound lookup-tail query-count-s1 state-shift[of j] by simp
qed
qed
have indices-all:
   $\forall i < \text{Suc } n.$ 
    map opening-index (?openings ! i) =
    powers-scaled (?idxs ! i)
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{Suc } n$ 
  show map opening-index (?openings ! i) =
    powers-scaled (?idxs ! i)
proof (cases i)
  case 0
  then show ?thesis
    using idx-openings by simp
next
  case (Suc j)
  then show ?thesis
    using i-bound tail-indices by simp
qed
qed
have tables-all:
   $\forall i < \text{Suc } n.$ 
    partial-authenticated-table fr (scale * clength)
    (?openings ! i) t
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{Suc } n$ 
  show partial-authenticated-table fr (scale * clength)
    (?openings ! i) t
proof (cases i)
  case 0
  then show ?thesis
    using table-t by simp
next
  case (Suc j)
  then show ?thesis
    using i-bound tail-tables by simp
qed

```

```

qed
have consistent-all:
   $\forall i < \text{Suc } n.$ 
    cp-eval as (map opening-value (?openings ! i))
      ( $h \wedge (?idxs ! i) * \text{shift}$ ) = final
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{Suc } n$ 
  show cp-eval as (map opening-value (?openings ! i))
    ( $h \wedge (?idxs ! i) * \text{shift}$ ) = final
  proof (cases i)
    case 0
    then show ?thesis
      using consistent-head by simp
  next
    case (Suc j)
    then show ?thesis
      using i-bound tail-consistent by simp
  qed
qed
show ?case
  by (intro exI[of - ?rows] exI[of - ?idxs] exI[of - ?chunks]
    exI[of - ?openings] conjI)
    (use len-row-tail idxs-eq len-chunk-tail len-openings-tail
      lookup-all indices-all tables-all consistent-all in simp-all)
qed
then obtain raw-idxs query-idxs query-chunks trace-openings where
  len-row: length raw-idxs = n
  and query-idxs-eq:
    query-idxs = map ( $\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})$ ) raw-idxs
  and len-chunks: length query-chunks = n
  and len-trace: length trace-openings = n
  and lookup:
     $\forall i < n.$ 
      fmllookup (HashMap t)
        (QueryIndexChallenge (PQueryCounter s + i)
          (state-after-query-chunks (PState s) query-chunks i)) =
        Some (raw-idxs ! i)
  and indices:
     $\forall i < n.$ 
      map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
  and tables:
     $\forall i < n.$ 
      partial-authenticated-table fr (scale * clength)
        (trace-openings ! i) t
  and consistent:
     $\forall i < n.$ 
      cp-eval as (map opening-value (trace-openings ! i))

```

```

      (h ^ (query-idxs ! i) * shift) = final
    by blast
  show ?thesis
    by (rule that[OF len-raw query-idxs-eq len-chunks len-trace])
      (use lookup indices tables consistent in auto)
qed

lemma ntimes-verifier-query-rounds-empty-composition-first-consistent:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-empty: fl = []
  and outcome:
    Some (results, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          s)
  obtains raw trace-openings where
    map opening-index trace-openings =
      powers-scaled (index (to-nat raw))
    partial-authenticated-table fr (scale * clength) trace-openings t
    cp-eval as (map opening-value trace-openings)
    (h ^ index (to-nat raw) * shift) = final
    fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof -
  from outcome obtain u results' s1 where
    head:
      Some (u, s1) ∈
        set-dist
          (execute
            (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results', t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            (rounds - Suc 0))
          s1)
  using rounds-positive
  by (cases rounds) (auto elim!: set-dist-bindE)
  have u-eq: u = ()
  by (cases u) simp
  from verifier-query-round-program-empty-composition-consistent-with-lookup
    [OF fl-empty head[unfolded u-eq]]
  obtain raw idx trace-openings where idx-eq:
    idx = index (to-nat raw)

```

```

and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table-s1:
  partial-authenticated-table fr (scale * clength) trace-openings s1
and consistent:
  cp-eval as (map opening-value trace-openings) (h ^ idx * shift) =
    final
and lookup-s1:
  fmlookup (HashMap s1)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
by blast
have s1-t: s1 ≤ t
  using ntimes-verifier-query-rounds-outcome[OF tail] by blast
have trace-table-t:
  partial-authenticated-table fr (scale * clength) trace-openings t
  by (rule partial-authenticated-table-mono[OF trace-table-s1 s1-t])
have lookup-t:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using hash-extension-lookup[OF lookup-s1 s1-t] .
show ?thesis
  by (rule that[of trace-openings raw])
  (use idx-eq trace-indices trace-table-t consistent lookup-t in simp-all)
qed

lemma ntimes-verifier-query-rounds-authenticated-openings-consistent-at:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-eq: fl = (b, composition-root) # fl-tail
and outcome:
  Some (results, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s)
and i-bound: i < n
obtains raw idx prefix-chunks trace-openings composition-openings where
  idx = index (to-nat raw)
  length prefix-chunks = i
  map opening-index trace-openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) trace-openings t
  partial-authenticated-table composition-root (scale * clength)
    composition-openings t
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
  partial-query-openings-consistent trace-openings composition-openings
    as idx
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + i)

```

```

      (state-after-query-chunks (PState s) prefix-chunks i)) = Some raw
proof –
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix round suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) i)
          s)
and round:
    Some (round, s-suc) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s-i)
and suffix:
    Some (suffix, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            (n – Suc i))
          s-suc)
by blast
have round-unit: round = ()
by (cases round) simp
show ?thesis
proof (rule
  verifier-query-round-program-authenticated-openings-consistent-with-lookup
    [OF fl-eq round[unfolded round-unit]])
fix raw idx trace-openings composition-openings
assume idx-eq: idx = index (to-nat raw)
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table-suc:
  partial-authenticated-table fr (scale * clength)
  trace-openings s-suc
and comp-table-suc:
  partial-authenticated-table composition-root (scale * clength)
  composition-openings s-suc
and comp-indices:
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
and consistent:
  partial-query-openings-consistent trace-openings
  composition-openings as idx
and lookup-suc:
  fmllookup (HashMap s-suc)

```



```

      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw
show ?thesis
proof (rule query-opening-ntimes-prefix-state-alignment[OF prefix])
  fix prefix-chunks
  assume len-chunks: length prefix-chunks = i
  and state-i:
    PState s-i =
      state-after-query-chunks (PState s) prefix-chunks i
  and counter-i: PQueryCounter s-i = PQueryCounter s + i
have suffix-ext: s-suc ≤ t
  by (rule query-opening-ntimes-suffix-extends[OF suffix])
have trace-table-t:
  partial-authenticated-table fr (scale * clength)
    trace-openings t
  by (rule
    partial-authenticated-table-mono[OF trace-table-suc suffix-ext])
have comp-table-t:
  partial-authenticated-table composition-root (scale * clength)
    composition-openings t
  by (rule
    partial-authenticated-table-mono[OF comp-table-suc suffix-ext])
have lookup-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + i)
      (state-after-query-chunks (PState s) prefix-chunks i)) =
    Some raw
  using hash-extension-lookup[OF lookup-suc suffix-ext]
  unfolding counter-i state-i .
show ?thesis
  by (rule that[OF idx-eq len-chunks trace-indices trace-table-t
    comp-table-t comp-indices consistent lookup-t])

qed
qed
qed

lemma ntimes-verifier-query-rounds-empty-composition-consistent-at:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-empty: fl = []
and outcome:
  Some (results, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s)
and i-bound: i < n
obtains raw idx prefix-chunks trace-openings where
  idx = index (to-nat raw)

```

```

length prefix-chunks = i
map opening-index trace-openings = powers-scaled idx
partial-authenticated-table fr (scale * clength) trace-openings t
cp-eval as (map opening-value trace-openings) (h ^ idx * shift) =
  final
fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s + i)
    (state-after-query-chunks (PState s) prefix-chunks i)) = Some raw
proof –
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix round suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) i)
          s)
and round:
    Some (round, s-suc) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s-i)
and suffix:
    Some (suffix, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            (n – Suc i))
          s-suc)
by blast
have round-unit: round = ()
by (cases round) simp
show ?thesis
proof (rule
  verifier-query-round-program-empty-composition-consistent-with-lookup
    [OF fl-empty round[unfolded round-unit]])
fix raw idx trace-openings
assume idx-eq: idx = index (to-nat raw)
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table-suc:
  partial-authenticated-table fr (scale * clength)
    trace-openings s-suc
and consistent:
  cp-eval as (map opening-value trace-openings) (h ^ idx * shift) =
    final
and lookup-suc:

```

```

    fmllookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw
  show ?thesis
  proof (rule query-opening-ntimes-prefix-state-alignment[OF prefix])
    fix prefix-chunks
    assume len-chunks: length prefix-chunks = i
    and state-i:
      PState s-i =
        state-after-query-chunks (PState s) prefix-chunks i
    and counter-i: PQueryCounter s-i = PQueryCounter s + i
  have suffix-ext: s-suc ≤ t
    by (rule query-opening-ntimes-suffix-extends[OF suffix])
  have trace-table-t:
    partial-authenticated-table fr (scale * clength)
      trace-openings t
    by (rule
      partial-authenticated-table-mono[OF trace-table-suc suffix-ext])
  have lookup-t:
    fmllookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) prefix-chunks i)) =
      Some raw
    using hash-extension-lookup[OF lookup-suc suffix-ext]
    unfolding counter-i state-i .
  show ?thesis
    by (rule that[OF idx-eq len-chunks trace-indices trace-table-t
      consistent lookup-t])
  qed
  qed
  qed

  lemma ntimes-verifier-query-rounds-selected-authenticated-chunks-at:
    fixes s t :: ('f, 'a) protocol-channel-scheme
    assumes fl-eq: fl = (b, composition-root) # fl-tail
    and outcome:
      Some (results, t) ∈
        set-dist
          (execute
            (ntimes
              (verifier-query-round-program fr f-fl f-final as fl final) n)
            s)
    and i-bound: i < n
  obtains raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
    suffix-chunks trace-openings composition-openings where
    idx = index (to-nat raw)
    length prefix-query-idxs = i
    length prefix-chunks = i
    length suffix-query-idxs = n - Suc i

```

```

length suffix-chunks = n - Suc i
verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
 $\bigwedge j. j < i \implies$ 
  verifier-query-round-chunk (prefix-query-idxs ! j)
    (map snd f-fl) (map snd fl) (prefix-chunks ! j)
 $\bigwedge j. j < n - \text{Suc } i \implies$ 
  verifier-query-round-chunk (suffix-query-idxs ! j)
    (map snd f-fl) (map snd fl) (suffix-chunks ! j)
PTranscript s =
  List.concat (prefix-chunks @ chunk # suffix-chunks) @ PTranscript t
map opening-index trace-openings = powers-scaled idx
partial-authenticated-table fr (scale * clength) trace-openings t
partial-authenticated-table composition-root (scale * clength)
  composition-openings t
map opening-index composition-openings =
  [idx, fri-sibling-index (scale * clength) idx]
partial-query-openings-consistent trace-openings composition-openings
  as idx
fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s + i)
    (state-after-query-chunks (PState s) prefix-chunks i)) = Some raw
proof -
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix round suffix s-i s-suc where
  prefix:
    Some (prefix, s-i)  $\in$ 
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) i)
          s)
and round:
    Some (round, s-suc)  $\in$ 
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s-i)
and suffix:
    Some (suffix, t)  $\in$ 
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            (n - Suc i))
          s-suc)
by blast
have round-unit: round = ()
by (cases round) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment

```

```

[OF prefix])
fix prefix-query-idxs prefix-chunks
assume len-prefix: length prefix-query-idxs = i
and len-prefix-chunks: length prefix-chunks = i
and transcript-prefix:
  PTranscript s = List.concat prefix-chunks @ PTranscript s-i
and state-i:
  PState s-i =
    state-after-query-chunks (PState s) prefix-chunks i
and counter-i: PQueryCounter s-i = PQueryCounter s + i
and prefix-ext: s ≤ s-i
and prefix-rounds:
  ∧j. j < i ⇒
    verifier-query-round-chunk (prefix-query-idxs ! j)
      (map snd f-fl) (map snd fl) (prefix-chunks ! j)
show ?thesis
proof (rule
  query-opening-verifier-query-round-program-authenticated-with-chunk
  [OF fl-eq round[unfolded round-unit]])
fix raw idx chunk trace-leaves trace-openings composition-openings
assume idx-eq: idx = index (to-nat raw)
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
and transcript-round:
  PTranscript s-i = chunk @ PTranscript s-suc
and state-suc:
  PState s-suc = foldl concat (PState s-i) chunk
and round-ext: s-i ≤ s-suc
and counter-suc: PQueryCounter s-suc = Suc (PQueryCounter s-i)
and trace-values:
  map opening-value trace-openings = trace-leaves
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table-suc:
  partial-authenticated-table fr (scale * clength)
    trace-openings s-suc
and comp-table-suc:
  partial-authenticated-table composition-root (scale * clength)
    composition-openings s-suc
and comp-indices:
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
and lookup-suc:
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
show ?thesis
proof (rule verifier-query-round-program-authenticated-openings-consistent-with-lookup

```

```

[OF fl-eq round[unfolded round-unit]])
fix raw' idx' trace-openings' composition-openings'
assume idx'-eq: idx' = index (to-nat raw')
  and trace-indices':
    map opening-index trace-openings' = powers-scaled idx'
  and trace-table-suc':
    partial-authenticated-table fr (scale * clength)
    trace-openings' s-suc
  and comp-table-suc':
    partial-authenticated-table composition-root (scale * clength)
    composition-openings' s-suc
  and comp-indices':
    map opening-index composition-openings' =
    [idx', fri-sibling-index (scale * clength) idx']
  and consistent':
    partial-query-openings-consistent trace-openings'
    composition-openings' as idx'
  and lookup-suc':
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw'
have raw'-eq: raw' = raw
  using lookup-suc lookup-suc' by simp
have idx'-idx: idx' = idx
  using idx'-eq idx-eq raw'-eq by simp
have suffix-ext: s-suc ≤ t
  by (rule query-opening-ntimes-suffix-extends[OF suffix])
have trace-table-t:
  partial-authenticated-table fr (scale * clength)
  trace-openings' t
  by (rule partial-authenticated-table-mono
    [OF trace-table-suc' suffix-ext])
have comp-table-t:
  partial-authenticated-table composition-root (scale * clength)
  composition-openings' t
  by (rule partial-authenticated-table-mono
    [OF comp-table-suc' suffix-ext])
have lookup-t:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s + i)
    (state-after-query-chunks (PState s) prefix-chunks i)) =
  Some raw
  using hash-extension-lookup[OF lookup-suc suffix-ext]
  unfolding counter-i state-i .
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
  fix suffix-query-idxs suffix-chunks
  assume len-suffix-idxs:

```

```

    length suffix-query-idxs = n - Suc i
  and len-suffix-chunks:
    length suffix-chunks = n - Suc i
  and transcript-suffix:
    PTranscript s-suc =
      List.concat suffix-chunks @ PTranscript t
  and suffix-state:
    PState t =
      state-after-query-chunks (PState s-suc) suffix-chunks
      (n - Suc i)
  and suffix-counter:
    PQueryCounter t = PQueryCounter s-suc + (n - Suc i)
  and suffix-ext2: s-suc ≤ t
  and suffix-rounds:
    ∧j. j < n - Suc i ⇒
      verifier-query-round-chunk (suffix-query-idxs ! j)
      (map snd f-fl) (map snd fl) (suffix-chunks ! j)
  have transcript-all:
    PTranscript s =
      List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript t
  using transcript-prefix transcript-round transcript-suffix
  by simp
  show ?thesis
  by (rule that[OF idx-eq len-prefix len-prefix-chunks
    len-suffix-idxs len-suffix-chunks chunk-shape - -
    transcript-all - trace-table-t comp-table-t - - lookup-t])
    (use trace-indices' comp-indices' consistent' idx'-idx
    prefix-rounds suffix-rounds in simp-all)
qed
qed
qed
qed
qed
qed

lemma ntimes-verifier-query-rounds-aligned-authenticated-openings-consistent:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-eq: fl = (b, composition-root) # fl-tail
  and outcome:
    Some (results, t) ∈
      set-dist
      (execute
      (ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds)
      s)
  obtains query-idxs trace-openings composition-openings where
    length query-idxs = rounds
    length trace-openings = rounds

```

```

length composition-openings = rounds
 $\wedge i. i < \text{rounds} \implies$ 
  map opening-index (trace-openings ! i) =
    powers-scaled (query-idxs ! i)
 $\wedge i. i < \text{rounds} \implies$ 
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) t
 $\wedge i. i < \text{rounds} \implies$ 
  partial-authenticated-table composition-root (scale * clength)
    (composition-openings ! i) t
 $\wedge i. i < \text{rounds} \implies$ 
  map opening-index (composition-openings ! i) =
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
 $\wedge i. i < \text{rounds} \implies$ 
  partial-query-round-consistent trace-openings composition-openings as i
    (query-idxs ! i)
proof (cases rounds = 0)
  case True
  show ?thesis
  by (rule that[of [] [] []]) (use True in simp-all)
next
  case nonzero: False
  let ?P =
     $\lambda i w. \text{map opening-index (fst (snd w))} = \text{powers-scaled (fst w)} \wedge$ 
     $\text{partial-authenticated-table fr (scale * clength) (fst (snd w)) } t \wedge$ 
     $\text{partial-authenticated-table composition-root (scale * clength)}$ 
     $(\text{snd (snd w)}) t \wedge$ 
     $\text{map opening-index (snd (snd w))} =$ 
     $[\text{fst w, fri-sibling-index (scale * clength) (fst w)}] \wedge$ 
     $\text{partial-query-openings-consistent (fst (snd w)) (snd (snd w))}$ 
    as (fst w)
  let ?wit =  $\lambda i. \text{SOME } w. ?P i w$ 
  let ?idx =  $\lambda i. \text{fst (?wit } i)$ 
  let ?trace-openings =  $\lambda i. \text{fst (snd (?wit } i))$ 
  let ?composition-openings =  $\lambda i. \text{snd (snd (?wit } i))$ 
  let ?query-idxs = map ?idx [0..let ?trace-tables = map ?trace-openings [0..let ?comp-tables = map ?composition-openings [0..have ex-round:
     $\wedge i. i < \text{rounds} \implies$ 
     $\exists w. ?P i w$ 
  proof –
    fix i
    assume i-bound:  $i < \text{rounds}$ 
    show  $\exists w. ?P i w$ 
    proof (rule ntimes-verifier-query-rounds-selected-authenticated-chunks-at
      [OF fl-eq outcome i-bound])
      fix raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs

```



```

    suffix-chunks trace-openings composition-openings
assume idx-eq: idx = index (to-nat raw)
and trace-indices:
    map opening-index trace-openings = powers-scaled idx
and trace-table:
    partial-authenticated-table fr (scale * clength) trace-openings t
and comp-table:
    partial-authenticated-table composition-root (scale * clength)
    composition-openings t
and comp-indices:
    map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
and consistent:
    partial-query-openings-consistent trace-openings
    composition-openings as idx
show ?thesis
    by (intro exI[of - (idx, trace-openings, composition-openings)]
    conjI)
    (use idx-eq trace-indices trace-table comp-table comp-indices
    consistent in simp-all)
qed
qed
have round-props:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (?trace-tables ! i) =
    powers-scaled (?query-idxs ! i)  $\wedge$ 
    partial-authenticated-table fr (scale * clength)
    (?trace-tables ! i) t  $\wedge$ 
    partial-authenticated-table composition-root (scale * clength)
    (?comp-tables ! i) t  $\wedge$ 
    map opening-index (?comp-tables ! i) =
    [?query-idxs ! i,
    fri-sibling-index (scale * clength) (?query-idxs ! i)]  $\wedge$ 
    partial-query-round-consistent ?trace-tables ?comp-tables as i
    (?query-idxs ! i)
proof -
  fix i
  assume i-bound: i < rounds
  have chosen: ?P i (?wit i)
    by (rule someI-ex[OF ex-round[OF i-bound]])
  have nths:
    ?query-idxs ! i = ?idx i
    ?trace-tables ! i = ?trace-openings i
    ?comp-tables ! i = ?composition-openings i
    using i-bound by simp-all
  have lengths:
    length ?trace-tables = rounds
    length ?comp-tables = rounds
    by simp-all

```

```

show
  map opening-index (?trace-tables ! i) =
    powers-scaled (?query-idxs ! i) ∧
    partial-authenticated-table fr (scale * clength)
      (?trace-tables ! i) t ∧
    partial-authenticated-table composition-root (scale * clength)
      (?comp-tables ! i) t ∧
  map opening-index (?comp-tables ! i) =
    [?query-idxs ! i,
      fri-sibling-index (scale * clength) (?query-idxs ! i)] ∧
  partial-query-round-consistent ?trace-tables ?comp-tables as i
    (?query-idxs ! i)
using chosen i-bound nth lengths
unfolding partial-query-round-consistent-def
  partial-query-openings-consistent-def
by simp
qed
show ?thesis
by (rule that[of ?query-idxs ?trace-tables ?comp-tables])
  (use round-props in simp-all)
qed

end

end

```

```

theory Soundness-Partial-Initial-Aligned
imports Soundness-Query-Opening-Consistency
begin

```

Aligned partial initial-opening evidence.

The predicate *accepted-with-partial-initial-openings* records trace and composition openings with separate query-index lists. For query soundness bounds this theory packages the stronger verifier-local fact that both opening families come from the same query-round execution and therefore use the same sampled query indices.

```

context soundness
begin

```

definition *accepted-with-partial-initial-openings-aligned*

```

:: ('f, 'a) protocol-channel-scheme ⇒
  (unit list × ('f, 'a) protocol-channel-scheme) option ⇒
  'f ⇒ 'f list ⇒ 'f ⇒ 'f list ⇒ 'f ⇒
  'f list ⇒ 'f ⇒ nat list ⇒
  'f authenticated-opening list list ⇒
  'f authenticated-opening list list ⇒ bool

```

where

```

  accepted-with-partial-initial-openings-aligned s out fr f-fri-roots
    f-final as dg composition-fri-roots final query-idxs trace-openings

```

$composition-openings \longleftrightarrow$
 $accepted\ out \wedge$
 $composition-fri-roots \neq [] \wedge$
 $(\exists rest.$
 $\quad verifier-header-transcript\ s\ fr\ f-fri-roots\ f-final\ as\ dg$
 $\quad\quad composition-fri-roots\ final\ rest) \wedge$
 $accepted-with-partial-trace-openings\ s\ out\ fr\ query-idxs$
 $\quad trace-openings \wedge$
 $accepted-with-partial-composition-openings\ s\ out$
 $\quad (hd\ composition-fri-roots)\ query-idxs\ composition-openings$

lemma *accepted-with-partial-initial-openings-aligned-imp-unaligned:*

assumes

$accepted-with-partial-initial-openings-aligned\ s\ out\ fr\ f-fri-roots$
 $f-final\ as\ dg\ composition-fri-roots\ final\ query-idxs\ trace-openings$
 $composition-openings$

shows

$accepted-with-partial-initial-openings\ s\ out\ fr\ f-fri-roots\ f-final\ as\ dg$
 $composition-fri-roots\ final\ query-idxs\ trace-openings\ query-idxs$
 $composition-openings$

using *assms*

unfolding *accepted-with-partial-initial-openings-aligned-def*

accepted-with-partial-initial-openings-def

by *blast*

lemma *accepted-with-partial-initial-openings-aligned-shapes:*

assumes

$accepted-with-partial-initial-openings-aligned\ s\ out\ fr\ f-fri-roots$
 $f-final\ as\ dg\ composition-fri-roots\ final\ query-idxs\ trace-openings$
 $composition-openings$

shows $composition-fri-roots \neq []$

and $\exists rest.$ $verifier-header-transcript\ s\ fr\ f-fri-roots\ f-final\ as\ dg$
 $composition-fri-roots\ final\ rest$

and $accepted-with-partial-trace-openings\ s\ out\ fr\ query-idxs$
 $trace-openings$

and $accepted-with-partial-composition-openings\ s\ out$
 $(hd\ composition-fri-roots)\ query-idxs\ composition-openings$

using *assms*

unfolding *accepted-with-partial-initial-openings-aligned-def* **by** *blast+*

lemma *accepted-with-partial-initial-openings-aligned-candidates-if-no-partial-merkle-bad:*

assumes *partial:*

$accepted-with-partial-initial-openings-aligned\ s$
 $(Some\ (result,\ final-state))\ fr\ f-fri-roots\ f-final\ as\ dg$
 $composition-fri-roots\ final\ query-idxs\ trace-openings$
 $composition-openings$

and *no-bad:*

$\neg\ partial-merkle-inconsistency-bad\ s$
 $(Some\ (result,\ final-state))$

shows $\exists$ *trace-table composition-table*.
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings
proof –
have *unaligned*:
accepted-with-partial-initial-openings s
(Some (result, final-state)) fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings query-idxs
composition-openings
by (rule *accepted-with-partial-initial-openings-aligned-imp-unaligned*
 $[OF\ partial]$)
show *?thesis*
by (rule
accepted-with-partial-initial-openings-candidates-if-no-partial-merkle-bad
 $[OF\ unaligned\ no-bad]$)
qed

lemma *verify-monad-accepted-with-partial-initial-openings-aligned*:
fixes *s :: ('f, 'a) protocol-channel-scheme*
assumes *outcome*:
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *header0*:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *comp-nonempty*: *composition-fri-roots $\neq []$*
obtains *query-idxs trace-openings composition-openings* **where**
accepted-with-partial-initial-openings-aligned s
(Some (result, final-state)) fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings
composition-openings
 $\wedge i. i < rounds \implies$
partial-query-round-consistent trace-openings composition-openings as i
(query-idxs ! i)
proof –
from *verify-monad-header-extraction* $[OF\ outcome]$
obtain *fr' f-fl f-final' as' dg' fl final' query-state* **where**
header:
verifier-header-transcript s fr' (map snd f-fl) f-final' as' dg'
(map snd fl) final' (PTranscript query-state)
and *query-out*:
Some (result, final-state) $\in$
set-dist
(execute
(ntimes
(verifier-query-round-program fr' f-fl f-final' as' fl
final') rounds)
query-state)
by *metis*

```

have eqs:
  fr' = fr ∧
  map snd f-fl = f-fri-roots ∧
  f-final' = f-final ∧
  as' = as ∧
  dg' = dg ∧
  map snd fl = composition-fri-roots ∧
  final' = final ∧
  PTranscript query-state = rest
using verifier-header-transcript-unique[OF header header0] by simp
have fl-nonempty: fl ≠ []
  using comp-nonempty eqs by (cases fl) simp-all
then obtain b composition-root fl-tail where fl-eq:
  fl = (b, composition-root) # fl-tail
  by (cases fl) auto
have root-eq: composition-root = hd composition-fri-roots
proof -
  have composition-root # map snd fl-tail = composition-fri-roots
    using eqs unfolding fl-eq by simp
  then have hd composition-fri-roots = composition-root
    by (metis list.sel(1))
  then show ?thesis
    by simp
qed
obtain query-idxs trace-openings composition-openings where
  len-query-idxs: length query-idxs = rounds
  and len-trace: length trace-openings = rounds
  and len-comp: length composition-openings = rounds
  and trace-indices:
    ∧ i. i < rounds ⇒
      map opening-index (trace-openings ! i) =
        powers-scaled (query-idxs ! i)
  and trace-tables:
    ∧ i. i < rounds ⇒
      partial-authenticated-table fr' (scale * clength)
        (trace-openings ! i) final-state
  and comp-tables:
    ∧ i. i < rounds ⇒
      partial-authenticated-table composition-root (scale * clength)
        (composition-openings ! i) final-state
  and comp-indices:
    ∧ i. i < rounds ⇒
      map opening-index (composition-openings ! i) =
        [query-idxs ! i,
         fri-sibling-index (scale * clength) (query-idxs ! i)]
  and consistent:
    ∧ i. i < rounds ⇒
      partial-query-round-consistent trace-openings composition-openings
        as' i (query-idxs ! i)

```

```

proof (rule
  ntimes-verifier-query-rounds-aligned-authenticated-openings-consistent
  [OF fl-eq query-out])
fix query-idxs trace-openings composition-openings
assume len-query-idxs: length query-idxs = rounds
and len-trace: length trace-openings = rounds
and len-comp: length composition-openings = rounds
and trace-indices:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
and trace-tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr' (scale * clength)
      (trace-openings ! i) final-state
and comp-tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table composition-root (scale * clength)
      (composition-openings ! i) final-state
and comp-indices:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (composition-openings ! i) =
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]
and consistent:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-query-round-consistent trace-openings composition-openings
      as' i (query-idxs ! i)
show ?thesis
  by (rule that[OF len-query-idxs len-trace len-comp trace-indices
    trace-tables comp-tables comp-indices consistent])
qed
have consistent-as:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-query-round-consistent trace-openings composition-openings as i
      (query-idxs ! i)
using consistent eqs by simp
have trace-partial:
  accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
proof –
have all-indices:
   $\forall i < \text{rounds}.$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
using trace-indices by blast
have all-tables:
   $\forall i < \text{rounds}.$ 
    partial-authenticated-table fr (scale * clength)

```

```

      (trace-openings ! i) final-state
    using trace-tables eqs by simp
  show ?thesis
    unfolding accepted-with-partial-trace-openings-def accepted-def
    by (intro conjI exI[of - result] exI[of - final-state])
      (use len-query-idxs len-trace all-indices all-tables in simp-all)
  qed
  have comp-partial:
    accepted-with-partial-composition-openings s
      (Some (result, final-state)) (hd composition-fri-roots) query-idxs
      composition-openings
  proof -
    have all-indices:
       $\forall i < \text{rounds}.$ 
      map opening-index (composition-openings ! i) =
        [query-idxs ! i,
         fri-sibling-index (scale * clength) (query-idxs ! i)]
    using comp-indices by blast
    have all-tables:
       $\forall i < \text{rounds}.$ 
      partial-authenticated-table (hd composition-fri-roots)
        (scale * clength) (composition-openings ! i) final-state
    using comp-tables root-eq by simp
    show ?thesis
      unfolding accepted-with-partial-composition-openings-def accepted-def
      by (intro conjI exI[of - result] exI[of - final-state])
        (use len-query-idxs len-comp all-indices all-tables in simp-all)
    qed
    have aligned:
      accepted-with-partial-initial-openings-aligned s
        (Some (result, final-state)) fr f-fri-roots f-final as dg
        composition-fri-roots final query-idxs trace-openings
        composition-openings
  proof -
    have accepted-out: accepted (Some (result, final-state))
      using trace-partial
      by (rule accepted-with-partial-trace-openings-imp-accepted)
    show ?thesis
      unfolding accepted-with-partial-initial-openings-aligned-def
      by (intro conjI exI[of - rest])
        (use accepted-out comp-nonempty header0 trace-partial comp-partial in
         simp-all)
    qed
    show ?thesis
      by (rule that[OF aligned consistent-as])
  qed
end

```

end

theory *Soundness-Aligned-Partial-Query*
imports *Soundness-Partial-Initial-Aligned*
begin

Aligned partial-candidate query events.

This layer records the query-bad shape needed by the staged prefix-target proof: one shared verifier query-index list for trace and composition openings, together with the local composition equation checked in each query round. It is downstream of the staged query-opening extraction layer to avoid import cycles with the core deterministic reductions.

context *soundness*
begin

definition *accepted-with-partial-initial-openings-aligned-consistent*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow \text{bool}$

where

accepted-with-partial-initial-openings-aligned-consistent s out fr
 f -fri-roots f -final as dg *composition-fri-roots* *final* *query-idxs*
 $\text{trace-openings} \text{ composition-openings} \longleftrightarrow$
accepted-with-partial-initial-openings-aligned s out fr f -fri-roots
 f -final as dg *composition-fri-roots* *final* *query-idxs* *trace-openings*
 $\text{composition-openings} \wedge$
 $(\forall i < \text{rounds}.$
 $\text{partial-query-round-consistent } \text{trace-openings } \text{composition-openings} \text{ as } i$
 $(\text{query-idxs } ! i))$

definition *query-bad-with-aligned-partial-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

query-bad-with-aligned-partial-candidates s out $\longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as } dg \text{ composition-fri-roots final}$
 $\text{query-idxs } \text{trace-openings } \text{composition-openings } \text{trace-table}$
 $\text{composition-table}.$
accepted-with-partial-initial-openings-aligned-consistent s out fr
 f -fri-roots f -final as dg *composition-fri-roots* *final* *query-idxs*
 $\text{trace-openings } \text{composition-openings} \wedge$
 $\text{partial-trace-table-candidate } \text{trace-table } \text{trace-openings} \wedge$
 $\text{partial-composition-table-candidate } \text{composition-table}$
 $\text{composition-openings} \wedge$

$\text{trace-table-low-degree trace-table} \wedge$
 $\text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\neg \text{all-queries-consistent trace-table composition-table as}$

lemma *verify-monad-accepted-with-partial-initial-openings-aligned-consistent:*

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes *outcome:*

$\text{Some (result, final-state)} \in \text{set-dist (execute verify-monad s)}$

and *header0:*

$\text{verifier-header-transcript s fr f-fri-roots f-final as dg}$

$\text{composition-fri-roots final rest}$

and *comp-nonempty:* $\text{composition-fri-roots} \neq []$

obtains $\text{query-idxs trace-openings composition-openings}$ **where**

accepted-with-partial-initial-openings-aligned-consistent s

$(\text{Some (result, final-state)}) \text{ fr f-fri-roots f-final as dg}$

$\text{composition-fri-roots final query-idxs trace-openings}$

$\text{composition-openings}$

proof –

from *verify-monad-accepted-with-partial-initial-openings-aligned*

$[OF \text{ outcome header0 comp-nonempty}]$

obtain $\text{query-idxs trace-openings composition-openings}$ **where** *aligned:*

accepted-with-partial-initial-openings-aligned s

$(\text{Some (result, final-state)}) \text{ fr f-fri-roots f-final as dg}$

$\text{composition-fri-roots final query-idxs trace-openings}$

$\text{composition-openings}$

and *consistent:*

$\bigwedge i. i < \text{rounds} \implies$

$\text{partial-query-round-consistent trace-openings composition-openings as } i$

$(\text{query-idxs } ! i)$

by *blast*

have *all-consistent:*

$\forall i < \text{rounds}.$

$\text{partial-query-round-consistent trace-openings composition-openings as } i$

$(\text{query-idxs } ! i)$

using *consistent by blast*

have *partial:*

accepted-with-partial-initial-openings-aligned-consistent s

$(\text{Some (result, final-state)}) \text{ fr f-fri-roots f-final as dg}$

$\text{composition-fri-roots final query-idxs trace-openings}$

$\text{composition-openings}$

unfolding *accepted-with-partial-initial-openings-aligned-consistent-def*

using *aligned all-consistent by simp*

show *?thesis*

by *(rule that[OF partial])*

qed

definition *soundness-bad-event-aligned-partial-candidate*

$:: ('f, 'a) \text{ protocol-channel-scheme} \implies$

$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \implies \text{bool}$

where

soundness-bad-event-aligned-partial-candidate s out $\longleftrightarrow$
composition-bad-with-partial-candidates s out $\vee$
trace-fri-bad-with-partial-candidates s out $\vee$
composition-fri-bad-with-partial-candidates s out $\vee$
query-bad-with-aligned-partial-candidates s out

definition *soundness-bad-event-aligned-partial-candidate-with-partial-merkle*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

soundness-bad-event-aligned-partial-candidate-with-partial-merkle s out $\longleftrightarrow$
partial-merkle-inconsistency-bad s out $\vee$
soundness-bad-event-aligned-partial-candidate s out

lemma *accepted-with-partial-initial-openings-aligned-consistent-imp-aligned:*

assumes

accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings

shows

accepted-with-partial-initial-openings-aligned s out fr f-fri-roots
f-final as dg composition-fri-roots final query-idxs trace-openings
composition-openings

using *assms*

unfolding *accepted-with-partial-initial-openings-aligned-consistent-def*

by *simp*

lemma *accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned:*

assumes

accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings

shows

accepted-with-partial-initial-openings s out fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings query-idxs
composition-openings

by (rule *accepted-with-partial-initial-openings-aligned-imp-unaligned*)

(rule *accepted-with-partial-initial-openings-aligned-consistent-imp-aligned*
[*OF assms*])

lemma *accepted-with-partial-initial-openings-aligned-consistent-success-index:*

assumes *partial:*

accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings

and *i-bound: i < rounds*

shows

query-idxs ! i $\in$

partial-query-success-indices-at trace-openings composition-openings
as i
using *partial i-bound*
unfolding *accepted-with-partial-initial-openings-aligned-consistent-def*
partial-query-success-indices-at-def partial-query-round-consistent-def
by *simp*

lemma *query-bad-with-aligned-partial-candidates-imp-query-bad-with-partial-candidates:*
assumes *query-bad-with-aligned-partial-candidates s out*
shows *query-bad-with-partial-candidates s out*
proof —
from *assms obtain fr f-fri-roots f-final as dg composition-fri-roots*
final query-idxs trace-openings composition-openings trace-table
composition-table where partial:
accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *comp-candidate:*
partial-composition-table-candidate composition-table
composition-openings
and *trace-low: trace-table-low-degree trace-table*
and *comp-low:*
composition-table-low-degree maxDegree composition-table
and *not-all:*
 $\neg$ *all-queries-consistent trace-table composition-table as*
unfolding *query-bad-with-aligned-partial-candidates-def* **by** *blast*
have *unaligned:*
accepted-with-partial-initial-openings s out fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings query-idxs
composition-openings
by *(rule*
accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
[OF partial])
show *?thesis*
unfolding *query-bad-with-partial-candidates-def*
using *unaligned trace-candidate comp-candidate trace-low comp-low not-all*
by *blast*
qed

lemma *soundness-bad-event-aligned-partial-candidate-imp-partial-candidate:*
assumes *soundness-bad-event-aligned-partial-candidate s out*
shows *soundness-bad-event-partial-candidate s out*
using *assms query-bad-with-aligned-partial-candidates-imp-query-bad-with-partial-candidates*
unfolding *soundness-bad-event-aligned-partial-candidate-def*
soundness-bad-event-partial-candidate-def
by *blast*

```

lemma soundness-bad-event-aligned-partial-candidate-with-partial-merkle-imp-partial-candidate-with-partial-merkle:
  assumes
    soundness-bad-event-aligned-partial-candidate-with-partial-merkle s out
  shows soundness-bad-event-partial-candidate-with-partial-merkle s out
  using assms soundness-bad-event-aligned-partial-candidate-imp-partial-candidate
  unfolding soundness-bad-event-aligned-partial-candidate-with-partial-merkle-def
    soundness-bad-event-partial-candidate-with-partial-merkle-def
  by blast

lemma accepted-aligned-partial-candidate-partition:
  assumes false-statement:  $\neg$  exists-valid-trace
  and partial:
    accepted-with-partial-initial-openings-aligned-consistent s out fr
    f-fri-roots f-final as dg composition-fri-roots final query-idxs
    trace-openings composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  shows soundness-bad-event-aligned-partial-candidate s out
proof (cases trace-table-low-degree trace-table)
  case False
  have trace-fri-bad-with-partial-candidates s out
  unfolding trace-fri-bad-with-partial-candidates-def
  using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
    [OF partial]
    trace-candidate False by blast
  then show ?thesis
    unfolding soundness-bad-event-aligned-partial-candidate-def by blast
next
  case True
  note trace-low = True
  show ?thesis
  proof (cases composition-table-low-degree maxDegree composition-table)
  case False
  have composition-fri-bad-with-partial-candidates s out
  unfolding composition-fri-bad-with-partial-candidates-def
  using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
    [OF partial]
    trace-candidate composition-candidate trace-low False
  by blast
  then show ?thesis
    unfolding soundness-bad-event-aligned-partial-candidate-def by blast
next
  case True
  note composition-low = True
  show ?thesis
  proof (cases all-queries-consistent trace-table composition-table as)

```

```

case False
have query-bad-with-aligned-partial-candidates s out
  unfolding query-bad-with-aligned-partial-candidates-def
  using partial trace-candidate composition-candidate trace-low
    composition-low False
  by blast
then show ?thesis
  unfolding soundness-bad-event-aligned-partial-candidate-def by blast
next
case True
note all-queries = True
from trace-low obtain f where
  deg-f: degree f < clength
  and trace-table-eq: trace-table = map (poly f) eval-domain
  unfolding trace-table-low-degree-def by blast
have violated: violated-constraints f ≠ {}
  by (rule false-statement-violated-constraints
    [OF false-statement deg-f])
have composition-bad-with-partial-candidates s out
  unfolding composition-bad-with-partial-candidates-def
  using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
    [OF partial]
    trace-candidate composition-candidate deg-f trace-table-eq
    violated composition-low all-queries
  by blast
then show ?thesis
  unfolding soundness-bad-event-aligned-partial-candidate-def by blast
qed
qed
qed

lemma soundness-bad-event-aligned-partial-candidate-union-bound:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes comp-bound:
  wp-event verify-monad
  (composition-bad-with-partial-candidates s) s ≤
  composition-error
and trace-fri-bound:
  wp-event verify-monad
  (trace-fri-bad-with-partial-candidates s) s ≤
  trace-fri-error'
and comp-fri-bound:
  wp-event verify-monad
  (composition-fri-bad-with-partial-candidates s) s ≤
  composition-fri-error'
and query-bound:
  wp-event verify-monad
  (query-bad-with-aligned-partial-candidates s) s ≤
  query-error

```

shows

wp-event verify-monad
(*soundness-bad-event-aligned-partial-candidate s*) *s* ≤
composition-error + *trace-fri-error'* + *composition-fri-error'* +
query-error

proof –

have *wp-event verify-monad*
(*soundness-bad-event-aligned-partial-candidate s*) *s* ≤
wp-event verify-monad
(*composition-bad-with-partial-candidates s*) *s* +
wp-event verify-monad
(*trace-fri-bad-with-partial-candidates s*) *s* +
wp-event verify-monad
(*composition-fri-bad-with-partial-candidates s*) *s* +
wp-event verify-monad
(*query-bad-with-aligned-partial-candidates s*) *s*
unfolding *soundness-bad-event-aligned-partial-candidate-def*
by (rule *wp-event-union-bound4*)
also have ... ≤
composition-error + *trace-fri-error'* + *composition-fri-error'* +
query-error
by (intro *add-mono comp-bound trace-fri-bound comp-fri-bound*
query-bound)
finally show ?thesis .

qed

lemma *soundness-bad-event-aligned-partial-candidate-with-partial-merkle-union-bound*:

fixes *s* :: ('f, 'a) *protocol-channel-scheme*

assumes *merkle-bound*:

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) *s* ≤
merkle-error

and *comp-bound*:

wp-event verify-monad
(*composition-bad-with-partial-candidates s*) *s* ≤
composition-error

and *trace-fri-bound*:

wp-event verify-monad
(*trace-fri-bad-with-partial-candidates s*) *s* ≤
trace-fri-error'

and *comp-fri-bound*:

wp-event verify-monad
(*composition-fri-bad-with-partial-candidates s*) *s* ≤
composition-fri-error'

and *query-bound*:

wp-event verify-monad
(*query-bad-with-aligned-partial-candidates s*) *s* ≤
query-error

shows

wp-event verify-monad

$(\text{soundness-bad-event-aligned-partial-candidate-with-partial-merkle } s) \ s$
 $\leq$
 $\text{merkle-error} + \text{composition-error} + \text{trace-fri-error}' +$
 $\text{composition-fri-error}' + \text{query-error}$

proof –

have *partial-bound*:

wp-event verify-monad
 $(\text{soundness-bad-event-aligned-partial-candidate } s) \ s \leq$
 $\text{composition-error} + \text{trace-fri-error}' + \text{composition-fri-error}' +$
 query-error

by (rule *soundness-bad-event-aligned-partial-candidate-union-bound*
 $[\text{OF comp-bound trace-fri-bound comp-fri-bound query-bound}]$)

have *wp-event verify-monad*
 $(\text{soundness-bad-event-aligned-partial-candidate-with-partial-merkle } s) \ s$
 $\leq$
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s +$
 $\text{wp-event verify-monad}$
 $(\text{soundness-bad-event-aligned-partial-candidate } s) \ s$

unfolding
 $\text{soundness-bad-event-aligned-partial-candidate-with-partial-merkle-def}$

by (rule *wp-event-union-bound*)

also have $\dots \leq$
 $\text{merkle-error} +$
 $(\text{composition-error} + \text{trace-fri-error}' + \text{composition-fri-error}' +$
 $\text{query-error})$

by (intro *add-mono merkle-bound partial-bound*)

finally show *?thesis*

by (*simp add: algebra-simps*)

qed

lemma *checked-staged-security-with-data-state-aligned-partial-candidate-with-partial-merkle-union-bound*:

fixes *merkle-error composition-error trace-fri-error'*
composition-fri-error' query-error :: prob

assumes *merkle-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{partial-merkle-inconsistency-bad})$
 $\text{adversary-initial-state} \leq \text{merkle-error}$

and *comp-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-error}$

and *trace-fri-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$

and *comp-fri-bound*:

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   composition-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ composition-fri-error'
and query-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   query-bad-with-aligned-partial-candidates)
  adversary-initial-state ≤ query-error
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   soundness-bad-event-aligned-partial-candidate-with-partial-merkle)
  adversary-initial-state ≤
  merkle-error + composition-error + trace-fri-error' +
  composition-fri-error' + query-error
proof -
let ?M = checked-staged-security-experiment-with-data-state A
let ?E = staged-security-with-data-state-verifier-event
let ?merkle = ?E partial-merkle-inconsistency-bad
let ?comp = ?E composition-bad-with-partial-candidates
let ?trace = ?E trace-fri-bad-with-partial-candidates
let ?comp-fri = ?E composition-fri-bad-with-partial-candidates
let ?query = ?E query-bad-with-aligned-partial-candidates
let ?partial = ?E soundness-bad-event-aligned-partial-candidate
let ?with-merkle =
  ?E soundness-bad-event-aligned-partial-candidate-with-partial-merkle
have partial-bound:
wp-event ?M ?partial adversary-initial-state ≤
  composition-error + trace-fri-error' + composition-fri-error' +
  query-error
proof -
have wp-event ?M ?partial adversary-initial-state ≤
  wp-event ?M
    (λout. ?comp out ∨ ?trace out ∨ ?comp-fri out ∨
     ?query out)
    adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
   soundness-bad-event-aligned-partial-candidate-def
   split: option.splits prod.splits)
also have ... ≤
  wp-event ?M ?comp adversary-initial-state +
  wp-event ?M ?trace adversary-initial-state +
  wp-event ?M ?comp-fri adversary-initial-state +
  wp-event ?M ?query adversary-initial-state
by (rule wp-event-union-bound4)
also have ... ≤
  composition-error + trace-fri-error' + composition-fri-error' +

```



```

      query-error
    by (intro add-mono comp-bound trace-fri-bound comp-fri-bound
        query-bound)
  finally show ?thesis .
qed
have wp-event ?M ?with-merkle adversary-initial-state ≤
  wp-event ?M (λout. ?merkle out ∨ ?partial out)
  adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
    soundness-bad-event-aligned-partial-candidate-with-partial-merkle-def
    split: option.splits prod.splits)
also have ... ≤
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?partial adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
  merkle-error +
  (composition-error + trace-fri-error' + composition-fri-error' +
    query-error)
by (intro add-mono merkle-bound partial-bound)
finally show ?thesis
by (simp add: algebra-simps)
qed

```

lemma *query-bad-with-aligned-partial-candidates-success-indexE:*
assumes *bad: query-bad-with-aligned-partial-candidates s out*
and *i-bound: i < rounds*
obtains *fr f-fri-roots f-final as dg composition-fri-roots final*
query-idxs trace-openings composition-openings trace-table
composition-table **where**
accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings
partial-trace-table-candidate trace-table trace-openings
partial-composition-table-candidate composition-table
composition-openings
trace-table-low-degree trace-table
composition-table-low-degree maxDegree composition-table
 $\neg$ *all-queries-consistent trace-table composition-table as*
query-idxs ! i ∈
partial-query-success-indices-at trace-openings composition-openings
as i

proof —

from *bad* **obtain** *fr f-fri-roots f-final as dg composition-fri-roots*
final query-idxs trace-openings composition-openings trace-table
composition-table **where** *partial:*
accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs

trace-openings composition-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *comp-candidate:*
partial-composition-table-candidate composition-table
composition-openings
and *trace-low:* *trace-table-low-degree trace-table*
and *comp-low:*
composition-table-low-degree maxDegree composition-table
and *not-all:*
 $\neg$ *all-queries-consistent trace-table composition-table as*
unfolding *query-bad-with-aligned-partial-candidates-def* **by** *blast*
have *success:*
query-idxs ! i $\in$
partial-query-success-indices-at trace-openings composition-openings
as i
by (*rule*
accepted-with-partial-initial-openings-aligned-consistent-success-index
[OF partial i-bound])
show *?thesis*
by (*rule that*[*OF partial trace-candidate comp-candidate trace-low comp-low*
not-all success])
qed

lemma *query-bad-with-aligned-partial-candidates-target-fractionE:*
assumes *bad: query-bad-with-aligned-partial-candidates s out*
and *i-bound: i < rounds*
obtains *fr f-fri-roots f-final as dg composition-fri-roots final*
query-idxs trace-openings composition-openings trace-table
composition-table **where**
accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings
query-idxs ! i $\in$
partial-query-success-indices-at trace-openings composition-openings
as i
nnreal
(card
(partial-query-success-indices-at trace-openings composition-openings
as i)) /
nnreal (card query-sample-space) ≤ query-error-bound
proof –
from *query-bad-with-aligned-partial-candidates-success-indexE*
[OF bad i-bound]
obtain *fr f-fri-roots f-final as dg composition-fri-roots final*
query-idxs trace-openings composition-openings trace-table
composition-table **where** *partial:*
accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs

```

    trace-openings composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  and trace-low: trace-table-low-degree trace-table
  and comp-low:
    composition-table-low-degree maxDegree composition-table
  and not-all:
    ¬ all-queries-consistent trace-table composition-table as
  and success:
    query-idxs ! i ∈
      partial-query-success-indices-at trace-openings composition-openings
      as i
  by blast
have fraction:
  nnreal
  (card
    (partial-query-success-indices-at trace-openings composition-openings
      as i)) /
  nnreal (card query-sample-space) ≤ query-error-bound
  by (rule partial-query-success-indices-at-fraction-bound-if-candidate-low-degree
    [OF trace-candidate comp-candidate trace-low comp-low not-all])
show ?thesis
  by (rule that[OF partial success fraction])
qed

```

lemma *query-bad-with-aligned-partial-candidates-miss-disagreements:*

assumes *bad: query-bad-with-aligned-partial-candidates s out*

shows $\exists$ *trace-table composition-table as query-idxs.*

query-samples-miss-disagreements trace-table composition-table as query-idxs

proof —

from *bad* **obtain** *fr f-fri-roots f-final as dg composition-fri-roots*
final query-idxs trace-openings composition-openings trace-table
composition-table **where** *partial:*
accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings

and *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings

and *comp-candidate:*

partial-composition-table-candidate composition-table
composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *comp-low:*

composition-table-low-degree maxDegree composition-table

and *not-all:*

$\neg$ *all-queries-consistent trace-table composition-table as*
unfolding *query-bad-with-aligned-partial-candidates-def* **by** *blast*
have *aligned*:
accepted-with-partial-initial-openings-aligned s out fr f-fri-roots
f-final as dg composition-fri-roots final query-idxs trace-openings
composition-openings
by (rule *accepted-with-partial-initial-openings-aligned-consistent-imp-aligned*
[OF *partial*])
have *trace-partial*:
accepted-with-partial-trace-openings s out fr query-idxs
trace-openings
using *accepted-with-partial-initial-openings-aligned-shapes(3)*
[OF *aligned*] .
have *len-query*: *length query-idxs = rounds*
using *accepted-with-partial-trace-openings-shapes(1)*[OF *trace-partial*] .
have *nonempty-disagreements*:
query-disagreement-indices trace-table composition-table as $\neq \{\}$
using *not-all*
unfolding *query-disagreement-indices-def bad-query-indices-def*
all-queries-consistent-def
by *auto*
have *subset-agreement*:
set query-idxs $\subseteq$
query-agreement-indices trace-table composition-table as
proof
fix *idx*
assume *idx-in*: *idx $\in$ set query-idxs*
then obtain *i* **where** *i-len*: *i < length query-idxs*
and *idx-eq*: *query-idxs ! i = idx*
by (auto simp: *in-set-conv-nth*)
have *i-bound*: *i < rounds*
using *i-len len-query* **by** *simp*
have *success*:
query-idxs ! i $\in$
partial-query-success-indices-at trace-openings composition-openings
as i
by (rule
accepted-with-partial-initial-openings-aligned-consistent-success-index
[OF *partial i-bound*])
have *success-space*:
query-idxs ! i $\in$
query-sampling-success-space trace-table composition-table as
using *success*
partial-query-success-indices-at-subset-query-sampling-success-space
[OF *trace-candidate comp-candidate trace-low comp-low not-all,*
of i]
by *blast*
then show *idx $\in$*
query-agreement-indices trace-table composition-table as

```

    using idx-eq trace-low comp-low not-all
    unfolding query-sampling-success-space-def by simp
qed
have
  query-samples-miss-disagreements trace-table composition-table as
  query-idxs
  unfolding query-samples-miss-disagreements-def
  using nonempty-disagreements subset-agreement by simp
then show ?thesis by blast
qed

end

end

```

```

theory Soundness-Aligned-Partial-Query-Transcript
imports
  Soundness-Aligned-Partial-Query
  Soundness-Staged-Partial-Query
begin

```

Transcript-tied aligned partial-query events.

The aligned partial-opening predicate records that trace and composition openings use the same sampled query indices. For probabilistic query bounds we additionally need those indices to be the verifier-derived Fiat-Shamir query indices. This layer adds that transcript replay fact without changing the protocol model.

```

context soundness
begin

```

definition *accepted-with-partial-initial-openings-aligned-transcript-consistent*

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$ 
  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$ 
  'f list  $\Rightarrow$  'f  $\Rightarrow$  nat list  $\Rightarrow$ 
  'f authenticated-opening list list  $\Rightarrow$ 
  'f authenticated-opening list list  $\Rightarrow$  bool

```

where

```

accepted-with-partial-initial-openings-aligned-transcript-consistent s out
  fr f-fri-roots f-final as dg composition-fri-roots final query-idxs
  trace-openings composition-openings  $\longleftrightarrow$ 
accepted-with-partial-initial-openings-aligned-consistent s out fr
  f-fri-roots f-final as dg composition-fri-roots final query-idxs
  trace-openings composition-openings  $\wedge$ 
accepted-transcript-shape s out as query-idxs

```

definition *query-bad-with-aligned-transcript-partial-candidates*

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool

```

where

query-bad-with-aligned-transcript-partial-candidates *s out* $\longleftrightarrow$
 $(\exists \text{ fr f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ query-idxs trace-openings composition-openings trace-table}$
 $\text{ composition-table.}$
accepted-with-partial-initial-openings-aligned-transcript-consistent *s*
 $\text{ out fr f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ query-idxs trace-openings composition-openings } \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings } \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings } \wedge$
 $\text{ trace-table-low-degree trace-table } \wedge$
 $\text{ composition-table-low-degree maxDegree composition-table } \wedge$
 $\neg \text{ all-queries-consistent trace-table composition-table as})$

definition *soundness-bad-event-aligned-transcript-partial-candidate*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

soundness-bad-event-aligned-transcript-partial-candidate *s out* $\longleftrightarrow$
 $\text{ composition-bad-with-partial-candidates } s \text{ out } \vee$
 $\text{ trace-fri-bad-with-partial-candidates } s \text{ out } \vee$
 $\text{ composition-fri-bad-with-partial-candidates } s \text{ out } \vee$
 $\text{ query-bad-with-aligned-transcript-partial-candidates } s \text{ out}$

definition *soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
 $s \text{ out} \longleftrightarrow$
 $\text{ partial-merkle-inconsistency-bad } s \text{ out } \vee$
 $\text{ soundness-bad-event-aligned-transcript-partial-candidate } s \text{ out}$

lemma *accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned:*

assumes

$\text{ accepted-with-partial-initial-openings-aligned-transcript-consistent } s \text{ out}$
 $\text{ fr f-fri-roots f-final as dg composition-fri-roots final query-idxs}$
 $\text{ trace-openings composition-openings}$

shows

$\text{ accepted-with-partial-initial-openings-aligned-consistent } s \text{ out fr}$
 $\text{ f-fri-roots f-final as dg composition-fri-roots final query-idxs}$
 $\text{ trace-openings composition-openings}$

using *assms*

unfolding

$\text{ accepted-with-partial-initial-openings-aligned-transcript-consistent-def}$

by *simp*

lemma *accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape:*

assumes

accepted-with-partial-initial-openings-aligned-transcript-consistent s out
fr f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings

shows *accepted-transcript-shape s out as query-idxs*

using *assms*

unfolding

accepted-with-partial-initial-openings-aligned-transcript-consistent-def
by *simp*

lemma *query-bad-with-aligned-transcript-partial-candidates-imp-aligned:*

assumes *query-bad-with-aligned-transcript-partial-candidates s out*

shows *query-bad-with-aligned-partial-candidates s out*

proof –

from *assms* **obtain** *fr f-fri-roots f-final as dg composition-fri-roots*
final query-idxs trace-openings composition-openings trace-table
composition-table **where** *partial:*

accepted-with-partial-initial-openings-aligned-transcript-consistent s
out fr f-fri-roots f-final as dg composition-fri-roots final
query-idxs trace-openings composition-openings

and *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings

and *comp-candidate:*

partial-composition-table-candidate composition-table
composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *comp-low:*

composition-table-low-degree maxDegree composition-table

and *not-all:*

$\neg$ *all-queries-consistent trace-table composition-table as*

unfolding *query-bad-with-aligned-transcript-partial-candidates-def*

by *blast*

have *aligned:*

accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings

by *(rule*

accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
[OF partial])

show *?thesis*

unfolding *query-bad-with-aligned-partial-candidates-def*

using *aligned trace-candidate comp-candidate trace-low comp-low not-all*

by *blast*

qed

lemma *query-bad-with-aligned-transcript-partial-candidates-miss-disagreements:*

assumes *query-bad-with-aligned-transcript-partial-candidates s out*

shows $\exists$ *trace-table composition-table as query-idxs.*

query-samples-miss-disagreements trace-table composition-table as

query-idxs
by (rule *query-bad-with-aligned-partial-candidates-miss-disagreements*)
(rule *query-bad-with-aligned-transcript-partial-candidates-imp-aligned*
[*OF assms*])

lemma *soundness-bad-event-aligned-transcript-partial-candidate-imp-aligned:*
assumes *soundness-bad-event-aligned-transcript-partial-candidate s out*
shows *soundness-bad-event-aligned-transcript-partial-candidate s out*
using *assms query-bad-with-aligned-transcript-partial-candidates-imp-aligned*
unfolding *soundness-bad-event-aligned-transcript-partial-candidate-def*
soundness-bad-event-aligned-transcript-partial-candidate-def
by *blast*

lemma *soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-imp-aligned:*
assumes
soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
s out
shows
soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle s out
using *assms soundness-bad-event-aligned-transcript-partial-candidate-imp-aligned*
unfolding
soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def
soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def
by *blast*

lemma *accepted-aligned-transcript-partial-candidate-partition:*
assumes *false-statement: $\neg$ exists-valid-trace*
and *partial:*
accepted-with-partial-initial-openings-aligned-transcript-consistent s
out fr f-fri-roots f-final as dg composition-fri-roots final
query-idxs trace-openings composition-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *composition-candidate:*
partial-composition-table-candidate composition-table
composition-openings
shows *soundness-bad-event-aligned-transcript-partial-candidate s out*
proof (cases *trace-table-low-degree trace-table*)
case *False*
have *aligned:*
accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings
by (rule
accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
[*OF partial*])
have *trace-fri-bad-with-partial-candidates s out*
unfolding *trace-fri-bad-with-partial-candidates-def*
using *accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned*


```

    [OF aligned]
    trace-candidate False by blast
then show ?thesis
    unfolding soundness-bad-event-aligned-transcript-partial-candidate-def
    by blast
next
    case True
    note trace-low = True
    show ?thesis
    proof (cases composition-table-low-degree maxDegree composition-table)
    case False
    have aligned:
      accepted-with-partial-initial-openings-aligned-consistent s out fr
      f-fri-roots f-final as dg composition-fri-roots final query-idxs
      trace-openings composition-openings
    by (rule
      accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
      [OF partial])
    have composition-fri-bad-with-partial-candidates s out
    unfolding composition-fri-bad-with-partial-candidates-def
    using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
    [OF aligned]
    trace-candidate composition-candidate trace-low False
    by blast
    then show ?thesis
    unfolding soundness-bad-event-aligned-transcript-partial-candidate-def
    by blast
next
    case True
    note composition-low = True
    show ?thesis
    proof (cases all-queries-consistent trace-table composition-table as)
    case False
    have query-bad-with-aligned-transcript-partial-candidates s out
    unfolding query-bad-with-aligned-transcript-partial-candidates-def
    using partial trace-candidate composition-candidate trace-low
    composition-low False
    by blast
    then show ?thesis
    unfolding soundness-bad-event-aligned-transcript-partial-candidate-def
    by blast
next
    case True
    note all-queries = True
    from trace-low obtain f where
      deg-f: degree f < clength
    and trace-table-eq: trace-table = map (poly f) eval-domain
    unfolding trace-table-low-degree-def by blast
    have violated: violated-constraints f ≠ {}

```

```

    by (rule false-statement-violated-constraints
        [OF false-statement deg-f])
  have aligned:
    accepted-with-partial-initial-openings-aligned-consistent s out fr
      f-fri-roots f-final as dg composition-fri-roots final query-idxs
      trace-openings composition-openings
  by (rule
      accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
      [OF partial])
  have composition-bad-with-partial-candidates s out
  unfolding composition-bad-with-partial-candidates-def
  using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
    [OF aligned]
    trace-candidate composition-candidate deg-f trace-table-eq
    violated composition-low all-queries
  by blast
  then show ?thesis
  unfolding soundness-bad-event-aligned-transcript-partial-candidate-def
  by blast
qed
qed
qed

lemma soundness-bad-event-aligned-transcript-partial-candidate-union-bound:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes comp-bound:
    wp-event verify-monad
      (composition-bad-with-partial-candidates s) s ≤
      composition-error
  and trace-fri-bound:
    wp-event verify-monad
      (trace-fri-bad-with-partial-candidates s) s ≤
      trace-fri-error'
  and comp-fri-bound:
    wp-event verify-monad
      (composition-fri-bad-with-partial-candidates s) s ≤
      composition-fri-error'
  and query-bound:
    wp-event verify-monad
      (query-bad-with-aligned-transcript-partial-candidates s) s ≤
      query-error
  shows
    wp-event verify-monad
      (soundness-bad-event-aligned-transcript-partial-candidate s) s ≤
      composition-error + trace-fri-error' + composition-fri-error' +
      query-error
proof -
  have wp-event verify-monad
    (soundness-bad-event-aligned-transcript-partial-candidate s) s ≤

```

```

wp-event verify-monad
  (composition-bad-with-partial-candidates s) s +
wp-event verify-monad
  (trace-fri-bad-with-partial-candidates s) s +
wp-event verify-monad
  (composition-fri-bad-with-partial-candidates s) s +
wp-event verify-monad
  (query-bad-with-aligned-transcript-partial-candidates s) s
unfolding soundness-bad-event-aligned-transcript-partial-candidate-def
by (rule wp-event-union-bound4)
also have ... ≤
  composition-error + trace-fri-error' + composition-fri-error' +
  query-error
by (intro add-mono comp-bound trace-fri-bound comp-fri-bound
    query-bound)
finally show ?thesis .
qed

lemma soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-union-bound:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes merkle-bound:
  wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤
  merkle-error
and comp-bound:
  wp-event verify-monad
    (composition-bad-with-partial-candidates s) s ≤
    composition-error
and trace-fri-bound:
  wp-event verify-monad
    (trace-fri-bad-with-partial-candidates s) s ≤
    trace-fri-error'
and comp-fri-bound:
  wp-event verify-monad
    (composition-fri-bad-with-partial-candidates s) s ≤
    composition-fri-error'
and query-bound:
  wp-event verify-monad
    (query-bad-with-aligned-transcript-partial-candidates s) s ≤
    query-error
shows
  wp-event verify-monad
    (soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle s) s
    ≤
    merkle-error + composition-error + trace-fri-error' +
    composition-fri-error' + query-error
proof –
have partial-bound:
  wp-event verify-monad
    (soundness-bad-event-aligned-transcript-partial-candidate s) s ≤

```

```

    composition-error + trace-fri-error' + composition-fri-error' +
    query-error
  by (rule
    soundness-bad-event-aligned-transcript-partial-candidate-union-bound
    [OF comp-bound trace-fri-bound comp-fri-bound query-bound])
  have wp-event verify-monad
    (soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle s) s
    ≤
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s +
    wp-event verify-monad
      (soundness-bad-event-aligned-transcript-partial-candidate s) s
  unfolding
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def
  by (rule wp-event-union-bound)
  also have ... ≤
    merkle-error +
    (composition-error + trace-fri-error' + composition-fri-error' +
    query-error)
  by (intro add-mono merkle-bound partial-bound)
  finally show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-data-state-aligned-transcript-partial-candidate-union-bound:
  fixes composition-error trace-fri-error' composition-fri-error'
  query-error :: prob
  assumes comp-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-bad-with-partial-candidates)
    adversary-initial-state ≤ composition-error
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ trace-fri-error'
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ composition-fri-error'
  and query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        query-bad-with-aligned-transcript-partial-candidates)
    adversary-initial-state ≤ query-error
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event

```

```

    soundness-bad-event-aligned-transcript-partial-candidate)
  adversary-initial-state ≤
  composition-error + trace-fri-error' + composition-fri-error' +
  query-error
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?comp = ?E composition-bad-with-partial-candidates
  let ?trace = ?E trace-fri-bad-with-partial-candidates
  let ?comp-fri = ?E composition-fri-bad-with-partial-candidates
  let ?query = ?E query-bad-with-aligned-transcript-partial-candidates
  let ?partial = ?E soundness-bad-event-aligned-transcript-partial-candidate
  have wp-event ?M ?partial adversary-initial-state ≤
    wp-event ?M
    (λout. ?comp out ∨ ?trace out ∨ ?comp-fri out ∨
     ?query out)
    adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
     soundness-bad-event-aligned-transcript-partial-candidate-def
     split: option.splits prod.splits)
  also have ... ≤
    wp-event ?M ?comp adversary-initial-state +
    wp-event ?M ?trace adversary-initial-state +
    wp-event ?M ?comp-fri adversary-initial-state +
    wp-event ?M ?query adversary-initial-state
  by (rule wp-event-union-bound4)
  also have ... ≤
    composition-error + trace-fri-error' + composition-fri-error' +
    query-error
  by (intro add-mono comp-bound trace-fri-bound comp-fri-bound
    query-bound)
  finally show ?thesis .
qed

```

```

lemma checked-staged-security-with-data-state-aligned-transcript-partial-candidate-with-partial-merkle-component-un
fixes merkle-error composition-error trace-fri-error'
  composition-fri-error' query-error :: prob
assumes merkle-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     partial-merkle-inconsistency-bad)
  adversary-initial-state ≤ merkle-error
and comp-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-bad-with-partial-candidates)
  adversary-initial-state ≤ composition-error
and trace-fri-bound:

```

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-partial-candidates)
  adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-bad-with-partial-candidates)
  adversary-initial-state  $\leq$  composition-fri-error'
and query-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    query-bad-with-aligned-transcript-partial-candidates)
  adversary-initial-state  $\leq$  query-error
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle)
  adversary-initial-state  $\leq$ 
    merkle-error + composition-error + trace-fri-error' +
    composition-fri-error' + query-error
proof –
have partial-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    soundness-bad-event-aligned-transcript-partial-candidate)
  adversary-initial-state  $\leq$ 
    composition-error + trace-fri-error' + composition-fri-error' +
    query-error
by (rule
  checked-staged-security-with-data-state-aligned-transcript-partial-candidate-union-bound
  [OF comp-bound trace-fri-bound comp-fri-bound query-bound])
let ?M = checked-staged-security-experiment-with-data-state A
let ?E = staged-security-with-data-state-verifier-event
let ?merkle = ?E partial-merkle-inconsistency-bad
let ?partial = ?E soundness-bad-event-aligned-transcript-partial-candidate
let ?with-merkle =
  ?E soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
have wp-event ?M ?with-merkle adversary-initial-state  $\leq$ 
  wp-event ?M ( $\lambda$ out. ?merkle out  $\vee$  ?partial out)
  adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def
    split: option.splits prod.splits)
also have ...  $\leq$ 
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?partial adversary-initial-state
by (rule wp-event-union-bound)

```

also have ... $\leq$
 $\text{merkle-error} +$
 $(\text{composition-error} + \text{trace-fri-error}' + \text{composition-fri-error}' +$
 $\text{query-error})$
by (intro add-mono merkle-bound partial-bound)
finally show ?thesis
by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-data-state-aligned-transcript-partial-candidate-with-partial-merkle-union-bound:

fixes merkle-error composition-error trace-fri-error'

composition-fri-error' query-error :: prob

assumes merkle-bound:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

partial-merkle-inconsistency-bad)

adversary-initial-state $\leq$ merkle-error

and partial-bound:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

soundness-bad-event-aligned-transcript-partial-candidate)

adversary-initial-state $\leq$

composition-error + trace-fri-error' + composition-fri-error' +

query-error

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle)

adversary-initial-state $\leq$

merkle-error + composition-error + trace-fri-error' +

composition-fri-error' + query-error

proof –

let ?M = checked-staged-security-experiment-with-data-state A

let ?E = staged-security-with-data-state-verifier-event

let ?merkle = ?E partial-merkle-inconsistency-bad

let ?partial = ?E soundness-bad-event-aligned-transcript-partial-candidate

have wp-event ?M

(?E soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle)

adversary-initial-state $\leq$

wp-event ?M (λ out. ?merkle out $\vee$?partial out)

adversary-initial-state

by (rule wp-event-mono)

(auto simp: staged-security-with-data-state-verifier-event-def

soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def

split: option.splits prod.splits)

also have ... $\leq$

wp-event ?M ?merkle adversary-initial-state +

wp-event ?M ?partial adversary-initial-state

by (rule wp-event-union-bound)

```

also have ... ≤
  merkle-error +
  (composition-error + trace-fri-error' + composition-fri-error' +
    query-error)
by (intro add-mono merkle-bound partial-bound)
finally show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-data-state-aligned-transcript-query-bad-imp-partial-opening-hit-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  and bad:
    staged-security-with-data-state-verifier-event
      query-bad-with-aligned-transcript-partial-candidates
      (Some (((data, attacker-state), result), final-state))
  shows
    staged-security-with-data-state-query-partial-opening-hit
      (Some (((data, attacker-state), result), final-state))
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF support]
  obtain builder where builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
  have staged-header:
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      (List.concat (staged-query-chunks data))

```



```

by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
from bad obtain fr f-fri-roots f-final as dg composition-fri-roots
  final query-idxs trace-openings composition-openings trace-table
  composition-table where partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent ?s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-low:
  composition-table-low-degree maxDegree composition-table
and not-all:
  ¬ all-queries-consistent trace-table composition-table as
unfolding staged-security-with-data-state-verifier-event-def
  query-bad-with-aligned-transcript-partial-candidates-def
by simp blast
have aligned:
  accepted-with-partial-initial-openings-aligned-consistent ?s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
by (rule
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
  [OF partial])
have shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as
  query-idxs
by (rule
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
  [OF partial])
from checked-staged-security-with-data-state-accepted-shape-query-chunks
  [OF wf controlled support shape]
obtain raw-idxs where
  as-eq: as = staged-alphas data
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup:
  ∀ i < rounds.
  fmlookup (HashMap final-state)
  (QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data)

```

```

      (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  by blast
have aligned0:
  accepted-with-partial-initial-openings-aligned ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
  by (rule accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
      [OF aligned])
from accepted-with-partial-initial-openings-aligned-shapes[OF aligned0]
obtain rest where header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
  by blast
have comp-nonempty: composition-fri-roots ≠ []
  using accepted-with-partial-initial-openings-aligned-shapes(1)
  [OF aligned0] .
have trace-partial:
  accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr query-idxs trace-openings
  using accepted-with-partial-initial-openings-aligned-shapes(3)
  [OF aligned0] .
have comp-partial:
  accepted-with-partial-composition-openings ?s
    (Some (result, final-state)) (hd composition-fri-roots) query-idxs
    composition-openings
  using accepted-with-partial-initial-openings-aligned-shapes(4)
  [OF aligned0] .
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF header staged-header]
  by simp
let ?i = 0
have i-bound: ?i < rounds
  using rounds-positive by simp
have round-consistent:
  partial-query-round-consistent trace-openings composition-openings as
    ?i (query-idxs ! ?i)
  using aligned i-bound
unfolding accepted-with-partial-initial-openings-aligned-consistent-def
by simp

```

```

have opening-consistent:
  partial-query-openings-consistent (trace-openings ! ?i)
    (composition-openings ! ?i) as (query-idxs ! ?i)
  using round-consistent
  unfolding partial-query-round-consistent-def
    partial-query-openings-consistent-def
  by simp
have idx-sample: query-idxs ! ?i ∈ query-sample-space
  using round-consistent
  unfolding partial-query-round-consistent-def by simp
have trace-table:
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! ?i) final-state
  using trace-partial i-bound
  unfolding accepted-with-partial-trace-openings-def by blast
have comp-table:
  partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) (composition-openings ! ?i) final-state
  using comp-partial i-bound
  unfolding accepted-with-partial-composition-openings-def by blast
have witness:
  (trace-openings ! ?i, composition-openings ! ?i) ∈
    query-header-supported-partial-opening-witnesses ?s fr f-fri-roots
    f-final as dg composition-fri-roots final
  unfolding query-header-supported-partial-opening-witnesses-def
  using comp-nonempty verifier trace-table comp-table header by blast
have idx-in-header-set:
  query-idxs ! ?i ∈
    query-header-supported-partial-opening-success-indices-at ?s fr
    f-fri-roots f-final as dg composition-fri-roots final ?i
  unfolding query-header-supported-partial-opening-success-indices-at-def
  using idx-sample witness opening-consistent by blast
have idx-in-staged-set:
  query-idxs ! ?i ∈
    query-header-supported-partial-opening-success-indices-at ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
    ?i
  using idx-in-header-set header-eq by simp
have raw-idx:
  index (to-nat (raw-idxs ! ?i)) = query-idxs ! ?i
  using query-idxs-eq len-raw i-bound by simp
have lookup-i:
  fmlookup (HashMap final-state)

```

```

    (QueryIndexChallenge ?i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) ?i)) =
      Some (raw-idxs ! ?i)
  using lookup i-bound by simp
have hit-at:
  staged-security-with-data-state-query-partial-opening-hit-at ?i
  (Some (((data, attacker-state), result), final-state))
  unfolding staged-security-with-data-state-query-partial-opening-hit-at-def
  using i-bound lookup-i idx-in-staged-set raw-idx
  by (simp add: Let-def)
show ?thesis
  by (rule staged-security-with-data-state-query-partial-opening-hitI
    [OF i-bound hit-at])
qed

lemma checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-partial-opening-hit:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      query-bad-with-aligned-transcript-partial-candidates)
    adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-partial-opening-hit
    adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
  and bad:
    staged-security-with-data-state-verifier-event
    query-bad-with-aligned-transcript-partial-candidates out
  show staged-security-with-data-state-query-partial-opening-hit out
proof (cases out)
  case None
  then show ?thesis
    using bad
    unfolding staged-security-with-data-state-verifier-event-def by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)

```

```

    by (cases packed, auto split: prod.splits)
show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-data-state-aligned-transcript-query-bad-imp-partial-opening-hit-on-support
      [OF wf controlled])
    (use support bad out-eq in simp-all)
qed
qed

lemma selected-query-openings-align-with-replayed-query-key:
  assumes i-bound: i < rounds
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and selected-idx: idx = index (to-nat raw)
  and selected-lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge k prefix-state) = Some raw
  and replay-lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge k prefix-state) = Some (raw-idxs ! i)
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx
  and comp-indices:
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx]
  and consistent:
    partial-query-openings-consistent trace-openings
      composition-openings as idx
  shows
    idx = query-idxs ! i
    map opening-index trace-openings = powers-scaled (query-idxs ! i)
    map opening-index composition-openings =
      [query-idxs ! i,
        fri-sibling-index (scale * clength) (query-idxs ! i)]
    partial-query-openings-consistent trace-openings composition-openings
      as (query-idxs ! i)
proof -
  have raw-eq: raw = raw-idxs ! i
  using selected-lookup replay-lookup by simp
  have idx-eq: idx = query-idxs ! i
  using selected-idx raw-eq query-idxs-eq len-raw i-bound by simp
  show idx = query-idxs ! i
  by (rule idx-eq)
  show map opening-index trace-openings = powers-scaled (query-idxs ! i)
  using trace-indices idx-eq by simp
  show map opening-index composition-openings =
    [query-idxs ! i,

```

```

    fri-sibling-index (scale * clength) (query-idxs ! i)]
  using comp-indices idx-eq by simp
show partial-query-openings-consistent trace-openings composition-openings
  as (query-idxs ! i)
  using consistent idx-eq by simp
qed

```

lemma *selected-query-openings-align-with-replayed-query-index:*

```

assumes i-bound:  $i < \text{rounds}$ 
and len-raw:  $\text{length raw-idxs} = \text{rounds}$ 
and query-idxs-eq:
  query-idxs = map ( $\lambda \text{raw. index (to-nat raw)}$ ) raw-idxs
and selected-idx:  $\text{idx} = \text{index (to-nat raw)}$ 
and selected-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i prefix-state) = Some raw
and replay-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i prefix-state) = Some (raw-idxs ! i)
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and comp-indices:
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
and consistent:
  partial-query-openings-consistent trace-openings
    composition-openings as idx
shows
  idx = query-idxs ! i
  map opening-index trace-openings = powers-scaled (query-idxs ! i)
  map opening-index composition-openings =
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
  partial-query-openings-consistent trace-openings composition-openings
    as (query-idxs ! i)
by (rule selected-query-openings-align-with-replayed-query-key
  [OF i-bound len-raw query-idxs-eq selected-idx selected-lookup
   replay-lookup trace-indices comp-indices consistent])+)

```

lemma *ntimes-verifier-query-rounds-selected-authenticated-chunks-aligned-with-replay-at:*

```

assumes fl-eq:  $\text{fl} = (b, \text{composition-root}) \# \text{fl-tail}$ 
and outcome:
  Some (results, final-state)  $\in$ 
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
        query-state)

```

and *i-bound*: $i < \text{rounds}$
and *len-raw*: $\text{length } \text{raw-idxs} = \text{rounds}$
and *query-idxs-eq*:
 $\text{query-idxs} = \text{map } (\lambda \text{raw. index } (\text{to-nat } \text{raw})) \text{ raw-idxs}$
and *len-query-chunks*: $\text{length } \text{query-chunks} = \text{rounds}$
and *transcript-query*:
 $P\text{Transcript } \text{query-state} =$
 $\text{List.concat } \text{query-chunks} @ P\text{Transcript } \text{final-state}$
and *query-chunk*:
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{verifier-query-round-chunk } (\text{query-idxs } ! i)$
 $(\text{map } \text{snd } f\text{-fl}) (\text{map } \text{snd } \text{fl}) (\text{query-chunks } ! i)$
and *replay-lookup*:
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{fmlookup } (\text{HashMap } \text{final-state})$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } \text{query-state} + i)$
 $(\text{state-after-query-chunks}$
 $(P\text{State } \text{query-state}) \text{ query-chunks } i)) =$
 $\text{Some } (\text{raw-idxs } ! i)$
obtains *trace-openings composition-openings where*
 $\text{map } \text{opening-index } \text{trace-openings} = \text{powers-scaled } (\text{query-idxs } ! i)$
 $\text{partial-authenticated-table } \text{fr } (\text{scale} * \text{clength}) \text{ trace-openings}$
 final-state
 $\text{partial-authenticated-table } \text{composition-root } (\text{scale} * \text{clength})$
 $\text{composition-openings } \text{final-state}$
 $\text{map } \text{opening-index } \text{composition-openings} =$
 $[\text{query-idxs } ! i,$
 $\text{fri-sibling-index } (\text{scale} * \text{clength}) (\text{query-idxs } ! i)]$
 $\text{partial-query-openings-consistent } \text{trace-openings } \text{composition-openings}$
 $\text{as } (\text{query-idxs } ! i)$
proof (*rule ntimes-verifier-query-rounds-selected-authenticated-chunks-at*
 $[\text{OF } \text{fl-eq } \text{outcome } i\text{-bound}])$
fix *raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs*
 $\text{suffix-chunks } \text{trace-openings } \text{composition-openings}$
assume *selected-idx*: $\text{idx} = \text{index } (\text{to-nat } \text{raw})$
and *len-prefix-chunks*: $\text{length } \text{prefix-chunks} = i$
and *len-suffix-chunks*: $\text{length } \text{suffix-chunks} = \text{rounds} - \text{Suc } i$
and *chunk-shape*:
 $\text{verifier-query-round-chunk } \text{idx } (\text{map } \text{snd } f\text{-fl}) (\text{map } \text{snd } \text{fl}) \text{ chunk}$
and *prefix-rounds*:
 $\bigwedge j. j < i \implies$
 $\text{verifier-query-round-chunk } (\text{prefix-query-idxs } ! j)$
 $(\text{map } \text{snd } f\text{-fl}) (\text{map } \text{snd } \text{fl}) (\text{prefix-chunks } ! j)$
and *suffix-rounds*:
 $\bigwedge j. j < \text{rounds} - \text{Suc } i \implies$
 $\text{verifier-query-round-chunk } (\text{suffix-query-idxs } ! j)$
 $(\text{map } \text{snd } f\text{-fl}) (\text{map } \text{snd } \text{fl}) (\text{suffix-chunks } ! j)$
and *selected-transcript*:
 $P\text{Transcript } \text{query-state} =$

```

    List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript final-state
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx
  and trace-table:
    partial-authenticated-table fr (scale * clength) trace-openings
      final-state
  and comp-table:
    partial-authenticated-table composition-root (scale * clength)
      composition-openings final-state
  and comp-indices:
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx]
  and consistent:
    partial-query-openings-consistent trace-openings
      composition-openings as idx
  and selected-lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) prefix-chunks i)) =
      Some raw
  let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
  have len-selected-chunks: length ?selected-chunks = rounds
    using len-prefix-chunks len-suffix-chunks i-bound by simp
  have selected-chunk-lens:
     $\bigwedge j. j < \text{rounds} \implies$ 
    length (?selected-chunks ! j) =
      verifier-query-round-transcript-length (query-idxs ! j)
        (map snd f-fl) (map snd fl)
  proof -
    fix j
    assume j-bound: j < rounds
    show
      length (?selected-chunks ! j) =
        verifier-query-round-transcript-length (query-idxs ! j)
          (map snd f-fl) (map snd fl)
    proof (cases j < i)
      case True
      have nth-eq: ?selected-chunks ! j = prefix-chunks ! j
        using True len-prefix-chunks by (simp add: nth-append)
      have
        length (prefix-chunks ! j) =
          verifier-query-round-transcript-length
            (prefix-query-idxs ! j) (map snd f-fl) (map snd fl)
        by (rule verifier-query-round-chunk-length[OF prefix-rounds[OF True]])
      then show ?thesis
        using nth-eq
        by (simp add: verifier-query-round-transcript-length-index-irrelevant)
    next

```



```

case False
show ?thesis
proof (cases j = i)
  case True
  have nth-eq: ?selected-chunks ! j = chunk
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length chunk =
      verifier-query-round-transcript-length idx
      (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF chunk-shape])
  then show ?thesis
    using nth-eq
    by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
  next
  case False
  let ?k = j - Suc i
  have ge: Suc i ≤ j
    using  $\langle \neg j < i \rangle$  False by linarith
  have k-bound: ?k < rounds - Suc i
    using ge j-bound by linarith
  have nth-eq: ?selected-chunks ! j = suffix-chunks ! ?k
    using ge len-prefix-chunks by (simp add: nth-append)
  have
    length (suffix-chunks ! ?k) =
      verifier-query-round-transcript-length
      (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
      [OF suffix-rounds[OF k-bound]])
  then show ?thesis
    using nth-eq
    by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
  qed
qed
qed
have query-chunk-lens:
   $\bigwedge j. j < rounds \implies$ 
    length (query-chunks ! j) =
      verifier-query-round-transcript-length (query-idxs ! j)
      (map snd f-fl) (map snd fl)
    using query-chunk verifier-query-round-chunk-length by blast
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
     $\bigwedge j. j < length ?selected-chunks \implies$ 

```

```

    length (?selected-chunks ! j) = length (query-chunks ! j)
  using selected-chunk-lens query-chunk-lens len-selected-chunks by simp
have concat-eq-empty:
  List.concat ?selected-chunks @ [] = List.concat query-chunks
  using concat-eq by simp
have len-eq: length ?selected-chunks = length query-chunks
  using len-selected-chunks len-query-chunks by simp
have ?selected-chunks = query-chunks  $\wedge$  ([] :: 'f list) = []
  by (rule concat-append-eq-concat-same-chunk-lengths
      [OF len-eq same-lens concat-eq-empty])
then show ?thesis
  by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
      state-after-query-chunks (PState query-state) prefix-chunks i
    by (rule state-after-query-chunks-append-prefix[OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have selected-lookup-global:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some raw
  using selected-lookup prefix-state-eq by simp
have aligned:
  idx = query-idxs ! i
  map opening-index trace-openings = powers-scaled (query-idxs ! i)
  map opening-index composition-openings =
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
  partial-query-openings-consistent trace-openings composition-openings
  as (query-idxs ! i)
  by (rule selected-query-openings-align-with-replayed-query-key
      [OF i-bound len-raw query-idxs-eq selected-idx
        selected-lookup-global replay-lookup[OF i-bound] trace-indices
        comp-indices consistent])
show ?thesis
  by (rule that)
    (use aligned trace-table comp-table in simp-all)
qed

lemma ntimes-verifier-query-rounds-aligned-authenticated-openings-consistent-with-replay:
  assumes fl-eq: fl = (b, composition-root) # fl-tail

```

```

and outcome:
  Some (results, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and query-chunk:
   $\bigwedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
and replay-lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmllookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
obtains trace-openings composition-openings where
  length trace-openings = rounds
  length composition-openings = rounds
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table composition-root (scale * clength)
      (composition-openings ! i) final-state
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (composition-openings ! i) =
      [query-idxs ! i,
        fri-sibling-index (scale * clength) (query-idxs ! i)]
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-query-round-consistent trace-openings composition-openings as i
      (query-idxs ! i)
proof –
let ?P =
   $\lambda i. w.$ 
    map opening-index (fst w) = powers-scaled (query-idxs ! i) ∧

```

```

partial-authenticated-table fr (scale * clength) (fst w)
  final-state  $\wedge$ 
partial-authenticated-table composition-root (scale * clength)
  (snd w) final-state  $\wedge$ 
map opening-index (snd w) =
  [query-idxs ! i,
   fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 
query-idxs ! i  $\in$  query-sample-space  $\wedge$ 
snd w  $\neq$  []  $\wedge$ 
opening-value (snd w ! 0) =
  cp-eval as (map opening-value (fst w))
  (h  $\wedge$  query-idxs ! i * shift)
have ex-round:  $\bigwedge i. i < \text{rounds} \implies \exists w. ?P\ i\ w$ 
proof -
  fix i
  assume i-bound: i < rounds
  show  $\exists w. ?P\ i\ w$ 
  proof (rule
    ntimes-verifier-query-rounds-selected-authenticated-chunks-aligned-with-replay-at
    [OF fl-eq outcome i-bound len-raw query-idxs-eq len-query-chunks
     transcript-query query-chunk replay-lookup])
    fix trace-openings composition-openings
    assume trace-indices:
      map opening-index trace-openings =
      powers-scaled (query-idxs ! i)
    and trace-table:
      partial-authenticated-table fr (scale * clength) trace-openings
      final-state
    and comp-table:
      partial-authenticated-table composition-root (scale * clength)
      composition-openings final-state
    and comp-indices:
      map opening-index composition-openings =
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]
    and consistent:
      partial-query-openings-consistent trace-openings
      composition-openings as (query-idxs ! i)
    have idx-sample: query-idxs ! i  $\in$  query-sample-space
    using consistent unfolding partial-query-openings-consistent-def
    by simp
    have comp-nonempty: composition-openings  $\neq$  []
    using consistent unfolding partial-query-openings-consistent-def
    by simp
    have comp-value:
      opening-value (composition-openings ! 0) =
      cp-eval as (map opening-value trace-openings)
      (h  $\wedge$  query-idxs ! i * shift)
    using consistent unfolding partial-query-openings-consistent-def

```

```

    by simp
  show ?thesis
    by (intro exI[of - (trace-openings, composition-openings)]
        conjI)
      (use trace-indices trace-table comp-table comp-indices consistent
        idx-sample comp-nonempty comp-value in simp-all)
qed
qed
let ?wit =  $\lambda i. \text{SOME } w. ?P \ i \ w$ 
let ?trace-openings = map ( $\lambda i. \text{fst } (?wit \ i)$ ) [0..\lambda i. \text{snd } (?wit \ i)) [0..\bigwedge i. i < \text{rounds} \implies ?P \ i \ (?wit \ i)
  by (rule someI-ex[OF ex-round])
have consistent:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-query-round-consistent ?trace-openings ?composition-openings as
      i (query-idxs ! i)
proof -
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  have chosen:  $?P \ i \ (?wit \ i)$ 
    by (rule round-props[OF i-bound])
  have nth:
    ?trace-openings ! i = fst (?wit i)
    ?composition-openings ! i = snd (?wit i)
    using i-bound by simp-all
  have lengths:
    length ?trace-openings = rounds
    length ?composition-openings = rounds
    by simp-all
  have idx-sample: query-idxs ! i  $\in$  query-sample-space
    using chosen by blast
  have trace-indices0:
    map opening-index (fst (?wit i)) =
      powers-scaled (query-idxs ! i)
    using chosen by blast
  have comp-indices0:
    map opening-index (snd (?wit i)) =
      [query-idxs ! i,
        fri-sibling-index (scale * clength) (query-idxs ! i)]
    using chosen by blast
  have comp-value0:
    opening-value (snd (?wit i) ! 0) =
      cp-eval as (map opening-value (fst (?wit i)))
        ( $h \wedge \text{query-idxs ! } i * \text{shift}$ )
    using chosen by blast
  have trace-indices:
    map opening-index (?trace-openings ! i) =
      powers-scaled (query-idxs ! i)

```

```

    using trace-indices0 nths by simp
have comp-indices:
  map opening-index (?composition-openings ! i) =
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
    using comp-indices0 nths by simp
have comp-value:
  opening-value ((?composition-openings ! i) ! 0) =
    cp-eval as (map opening-value (?trace-openings ! i))
      (h ^ query-idxs ! i * shift)
    using comp-value0 nths by simp
show
  partial-query-round-consistent ?trace-openings ?composition-openings as
    i (query-idxs ! i)
  by (rule partial-query-round-consistentI
      [OF i-bound idx-sample lengths trace-indices comp-indices comp-value])
qed
show ?thesis
proof (rule that[of ?trace-openings ?composition-openings])
  show length ?trace-openings = rounds
    by simp
  show length ?composition-openings = rounds
    by simp
  show  $\bigwedge i. i < rounds \implies$ 
    map opening-index (?trace-openings ! i) =
      powers-scaled (query-idxs ! i)
  proof -
    fix i
    assume i-bound:  $i < rounds$ 
    have chosen:  $?P\ i\ (?wit\ i)$ 
      by (rule round-props[OF i-bound])
    have nth:  $?trace-openings\ !\ i = fst\ (?wit\ i)$ 
      using i-bound by simp
    have trace-indices0:
      map opening-index (fst (?wit i)) =
        powers-scaled (query-idxs ! i)
      using chosen by blast
    show map opening-index (?trace-openings ! i) =
      powers-scaled (query-idxs ! i)
      using trace-indices0 nth by simp
  qed
  show  $\bigwedge i. i < rounds \implies$ 
    partial-authenticated-table fr (scale * clength)
      (?trace-openings ! i) final-state
  proof -
    fix i
    assume i-bound:  $i < rounds$ 
    have chosen:  $?P\ i\ (?wit\ i)$ 
      by (rule round-props[OF i-bound])

```

```

have nth: ?trace-openings ! i = fst (?wit i)
  using i-bound by simp
have trace-table0:
  partial-authenticated-table fr (scale * clength)
    (fst (?wit i)) final-state
  using chosen by blast
show partial-authenticated-table fr (scale * clength)
  (?trace-openings ! i) final-state
  using trace-table0 nth by simp
qed
show  $\bigwedge i. i < \text{rounds} \implies$ 
  partial-authenticated-table composition-root (scale * clength)
    (?composition-openings ! i) final-state
proof –
  fix i
  assume i-bound: i < rounds
  have chosen: ?P i (?wit i)
    by (rule round-props[OF i-bound])
  have nth: ?composition-openings ! i = snd (?wit i)
    using i-bound by simp
  have comp-table0:
    partial-authenticated-table composition-root (scale * clength)
      (snd (?wit i)) final-state
    using chosen by blast
  show partial-authenticated-table composition-root (scale * clength)
    (?composition-openings ! i) final-state
    using comp-table0 nth by simp
qed
show  $\bigwedge i. i < \text{rounds} \implies$ 
  map opening-index (?composition-openings ! i) =
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
proof –
  fix i
  assume i-bound: i < rounds
  have chosen: ?P i (?wit i)
    by (rule round-props[OF i-bound])
  have nth: ?composition-openings ! i = snd (?wit i)
    using i-bound by simp
  have comp-indices0:
    map opening-index (snd (?wit i)) =
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]
    using chosen by blast
  show map opening-index (?composition-openings ! i) =
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
    using comp-indices0 nth by simp
qed

```

```

show  $\bigwedge i. i < \text{rounds} \implies$ 
  partial-query-round-consistent ?trace-openings ?composition-openings as
    i (query-idxs ! i)
  by (rule consistent)
qed
qed

lemma query-rounds-replay-accepted-with-partial-initial-openings-aligned-consistent:
assumes header0:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and comp-nonempty: composition-fri-roots  $\neq []$ 
and fl-eq: fl = (b, composition-root) # fl-tail
and f-roots: map snd f-fl = f-fri-roots
and comp-roots: map snd fl = composition-fri-roots
and query-out:
  Some (result, final-state) ∈
    set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map ( $\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})$ ) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and query-chunk:
   $\bigwedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
and replay-lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
obtains trace-openings composition-openings where
  accepted-with-partial-initial-openings-aligned-consistent s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
proof –
have root-eq: composition-root = hd composition-fri-roots
proof –
have composition-root # map snd fl-tail = composition-fri-roots

```



```

    using comp-roots unfolding fl-eq by simp
  then have hd composition-fri-roots = composition-root
    by (metis list.sel(1))
  then show ?thesis by simp
qed
show ?thesis
proof (rule
  ntimes-verifier-query-rounds-aligned-authenticated-openings-consistent-with-replay
    [OF fl-eq query-out len-raw query-idxs-eq len-query-chunks
      transcript-query query-chunk replay-lookup])
  fix trace-openings composition-openings
  assume len-trace: length trace-openings = rounds
  and len-comp: length composition-openings = rounds
  and trace-indices:
     $\bigwedge i. i < rounds \implies$ 
    map opening-index (trace-openings ! i) =
    powers-scaled (query-idxs ! i)
  and trace-tables:
     $\bigwedge i. i < rounds \implies$ 
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) final-state
  and comp-tables:
     $\bigwedge i. i < rounds \implies$ 
    partial-authenticated-table composition-root (scale * clength)
    (composition-openings ! i) final-state
  and comp-indices:
     $\bigwedge i. i < rounds \implies$ 
    map opening-index (composition-openings ! i) =
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
  and consistent:
     $\bigwedge i. i < rounds \implies$ 
    partial-query-round-consistent trace-openings composition-openings
    as i (query-idxs ! i)
  have len-query-idxs: length query-idxs = rounds
  using len-raw query-idxs-eq by simp
  have trace-partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
  proof -
    have all-indices:
       $\forall i < rounds.$ 
      map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
    using trace-indices by blast
    have all-tables:
       $\forall i < rounds.$ 
      partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state

```

```

    using trace-tables by blast
show ?thesis
  unfolding accepted-with-partial-trace-openings-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state])
    (use len-query-idxs len-trace all-indices all-tables in simp-all)
qed
have comp-partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) (hd composition-fri-roots) query-idxs
  composition-openings
proof -
  have all-indices:
     $\forall i < \text{rounds}.$ 
    map opening-index (composition-openings ! i) =
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]
  using comp-indices by blast
  have all-tables:
     $\forall i < \text{rounds}.$ 
    partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) (composition-openings ! i) final-state
  using comp-tables root-eq by simp
show ?thesis
  unfolding accepted-with-partial-composition-openings-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state])
    (use len-query-idxs len-comp all-indices all-tables in simp-all)
qed
have aligned:
  accepted-with-partial-initial-openings-aligned s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
proof -
  have accepted-out: accepted (Some (result, final-state))
  using trace-partial
  by (rule accepted-with-partial-trace-openings-imp-accepted)
show ?thesis
  unfolding accepted-with-partial-initial-openings-aligned-def
  by (intro conjI exI[of - rest])
    (use accepted-out comp-nonempty header0 trace-partial comp-partial
     in simp-all)
qed
have all-consistent:
   $\forall i < \text{rounds}.$ 
  partial-query-round-consistent trace-openings composition-openings as i
  (query-idxs ! i)
  using consistent by blast
have aligned-consistent:
  accepted-with-partial-initial-openings-aligned-consistent s

```

```

    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
unfolding accepted-with-partial-initial-openings-aligned-consistent-def
using aligned all-consistent by simp
show ?thesis
by (rule that[OF aligned-consistent])
qed
qed

lemma query-rounds-replay-accepted-with-partial-initial-openings-aligned-transcript-consistent:
assumes header0:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and comp-nonempty: composition-fri-roots  $\neq []$ 
and fl-eq: fl = (b, composition-root) # fl-tail
and f-roots: map snd f-fl = f-fri-roots
and comp-roots: map snd fl = composition-fri-roots
and query-state-transcript: PTranscript query-state = rest
and query-state-state:
    PState query-state =
        verifier-header-state s fr f-fri-roots f-final as dg
        composition-fri-roots final
and query-state-counter: PQueryCounter query-state = PQueryCounter s
and query-out:
    Some (result, final-state)  $\in$ 
        set-dist
        (execute
            (ntimes
                (verifier-query-round-program fr f-fl f-final as fl final)
                rounds)
            query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
    query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
    PTranscript query-state =
        List.concat query-chunks @ PTranscript final-state
and query-chunk:
     $\bigwedge i. i < rounds \implies$ 
        verifier-query-round-chunk (query-idxs ! i)
        (map snd f-fl) (map snd fl) (query-chunks ! i)
and replay-lookup:
     $\bigwedge i. i < rounds \implies$ 
        fmlookup (HashMap final-state)
        (QueryIndexChallenge (PQueryCounter query-state + i)
            (state-after-query-chunks (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)

```

obtains *trace-openings composition-openings* **where**
accepted-with-partial-initial-openings-aligned-transcript-consistent s
(Some (result, final-state)) fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings
composition-openings
proof –
from *query-rounds-replay-accepted-with-partial-initial-openings-aligned-consistent*
[OF header0 comp-nonempty fl-eq f-roots comp-roots query-out
len-raw query-idxs-eq len-query-chunks transcript-query
query-chunk replay-lookup]
obtain *trace-openings composition-openings* **where** *aligned:*
accepted-with-partial-initial-openings-aligned-consistent s
(Some (result, final-state)) fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings
composition-openings
by *blast*
have *transcript-rest:*
List.concat query-chunks @ PTranscript final-state = rest
using *transcript-query query-state-transcript* **by** *simp*
have *chunks:*
 $\forall i < \text{rounds}.$
verifier-query-round-chunk (query-idxs ! i)
f-fri-roots composition-fri-roots (query-chunks ! i)
using *query-chunk f-roots comp-roots* **by** *simp*
have *lookups:*
 $\forall i < \text{rounds}.$
fmlookup (HashMap final-state)
(QueryIndexChallenge (PQueryCounter s + i)
(state-after-query-chunks
(verifier-header-state s fr f-fri-roots f-final as dg
composition-fri-roots final)
query-chunks i)) =
Some (raw-idxs ! i)
using *replay-lookup query-state-state query-state-counter* **by** *simp*
have *idx-bounds:*
 $\forall \text{idx} \in \text{set query-idxs}. \text{idx} < \text{clength} * \text{scale}$
proof
fix *idx*
assume *idx ∈ set query-idxs*
then obtain *raw* **where** *raw: raw ∈ set raw-idxs*
and *idx: idx = index (to-nat raw)*
unfolding *query-idxs-eq* **by** *auto*
have *index (to-nat raw) < query-sample-space-size*
by *(rule index-less-query-sample-space)*
also have $\dots \leq \text{clength} * \text{scale}$
by *(rule query-sample-space-size-le-domain)*
finally show $\text{idx} < \text{clength} * \text{scale}$
using *idx* **by** *simp*
qed

```

have derived:
  verifier-query-indices-derived s (Some (result, final-state))
    (verifier-header-state s fr f-fri-roots f-final as dg
      composition-fri-roots final)
    rest f-fri-roots composition-fri-roots query-idxs
unfolding verifier-query-indices-derived-def
by (intro exI[of - result] exI[of - final-state]
      exI[of - raw-idxs] exI[of - query-chunks]
      exI[of - PTranscript final-state] conjI)
    (use len-raw query-idxs-eq len-query-chunks transcript-rest chunks
      lookups idx-bounds in auto)
have shape:
  accepted-transcript-shape s (Some (result, final-state)) as query-idxs
unfolding accepted-transcript-shape-def
by (intro exI[of - result] exI[of - final-state] exI[of - fr]
      exI[of - f-fri-roots] exI[of - f-final] exI[of - dg]
      exI[of - composition-fri-roots] exI[of - final]
      exI[of - rest] conjI)
    (use header0 derived in simp-all)
have tied:
  accepted-with-partial-initial-openings-aligned-transcript-consistent s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
unfolding
  accepted-with-partial-initial-openings-aligned-transcript-consistent-def
using aligned shape by simp
show ?thesis
  by (rule that[OF tied])
qed

end

end

```

```

theory Soundness-Aligned-Partial-Query-Prefix
imports
  Soundness-Aligned-Partial-Query-Transcript
  Soundness-Staged-Candidate-Query
  Soundness-Staged-Transcript
begin

```

Bridge aligned partial-query evidence to prefix-local staged query targets.

The lemmas here are deterministic: they say that, once a verifier query index is replayed to a prefix receive and the prefix alphas agree with the accepted transcript, the aligned partial-candidate query evidence produces a hit in the existing prefix-local target. Probability bounds remain in the staged prefix-target layer.

```

context soundness

```

begin

definition *checked-staged-security-experiment-with-query-prefix-data-state*

where

checked-staged-security-experiment-with-query-prefix-data-state $A\ i =$
checked-staged-query-prefix-receive-with-state $A\ i \gg=$
 $(\lambda \text{prefix-out}. \text{get} \gg=$
 $(\lambda \text{prefix-state}.$
checked-staged-after-query-prefix-receive-with-verifier $A\ i$
 $\text{prefix-out} \gg=$
 $(\lambda \text{out}. \text{return } ((\text{prefix-out}, \text{prefix-state}), \text{out}))))$

lemma *checked-staged-security-experiment-with-query-prefix-data-state-projection:*

assumes $i\text{-bound}: i < \text{rounds}$

shows

checked-staged-security-experiment-with-query-prefix-data-state $A\ i \gg=$
 $(\lambda \text{packed}. \text{return } (\text{snd } \text{packed})) =$
checked-staged-security-experiment-with-data-state A

unfolding

checked-staged-security-experiment-with-query-prefix-data-state-def
checked-staged-security-experiment-with-data-state-query-prefix-receive-decomp
[*OF* $i\text{-bound}$]

by (*simp* $\text{add: sm-bind-assoc get-bind-const}$)

lemma *checked-staged-security-experiment-with-query-prefix-data-state-projection-event:*

assumes $i\text{-bound}: i < \text{rounds}$

shows

wp-event (*checked-staged-security-experiment-with-data-state* A) E
adversary-initial-state =
wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A\ i)$
 $(\lambda \text{out}. \text{case } \text{out} \text{ of}$
 $\text{None} \Rightarrow E\ \text{None}$
 $| \text{Some } (\text{packed}, t) \Rightarrow E\ (\text{Some } (\text{snd } \text{packed}, t)))$
adversary-initial-state

proof –

have

wp-event (*checked-staged-security-experiment-with-data-state* A) E
adversary-initial-state =
wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A\ i \gg=$
 $(\lambda \text{packed}. \text{return } (\text{snd } \text{packed})))$
 $E\ \text{adversary-initial-state}$

using

checked-staged-security-experiment-with-query-prefix-data-state-projection
[*OF* $i\text{-bound}$, *of* A]

by *simp*

also have ... =

wp-event

```

    (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (λout. case out of
    None ⇒ E None
  | Some (packed, t) ⇒ E (Some (snd packed, t)))
  adversary-initial-state
  by (rule wp-event-bind-return-map)
  finally show ?thesis .
qed

lemma checked-staged-security-experiment-with-query-prefix-data-state-imp-data-state-support:
  assumes i-bound: i < rounds
  and outcome:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
    ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
  shows
    Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
proof -
  from outcome obtain t where prefix-receive:
    Some (((prefix, prefix-state), raw), t) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
  and t-eq: t = raw-state
  and cont:
    Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute
      (checked-staged-after-query-prefix-receive-with-verifier A i
        ((prefix, prefix-state), raw))
      raw-state)
  unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
  have data-out:
    Some (data, attacker-state) ∈
    set-dist
    (execute
      (checked-staged-after-query-prefix-receive A i
        ((prefix, prefix-state), raw))
      raw-state)
  by (rule checked-staged-after-query-prefix-receive-with-verifier-outcome(1)
    [OF cont])

```

```

have verifier:
  Some (result, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
  by (rule checked-staged-after-query-prefix-receive-with-verifier-outcome(2)
    [OF cont])
have builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  unfolding checked-staged-transcript-program-query-prefix-receive-decomp
    [OF i-bound]
  apply (rule-tac x=(prefix, prefix-state), raw) and
    t=raw-state in set-dist-bindI)
  apply (rule prefix-receive[unfolded t-eq])
  apply (rule data-out)
  done
show ?thesis
  by (rule checked-staged-security-experiment-with-data-state-outcomeI
    [OF builder verifier])
qed

lemma checked-staged-security-experiment-with-query-prefix-data-state-bound-by-prefix:
assumes prefix-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i) P
  adversary-initial-state ≤ B
and none-imp: Q None ⇒ P None
and cont-imp:
  ∧prefix-out t out.
  Some (prefix-out, t) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state) ⇒
  out ∈
    set-dist
      (execute
        (get ≫=
          (λprefix-state.
            checked-staged-after-query-prefix-receive-with-verifier A i
              prefix-out ≫=
                (λresult. return ((prefix-out, prefix-state), result))))
          t) ⇒
  Q out ⇒ P (Some (prefix-out, t))
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)

```


$Q \text{ adversary-initial-state} \leq B$
unfolding *checked-staged-security-experiment-with-query-prefix-data-state-def*
by (*rule wp-event-bind-bound-by-head-event*
 $[OF \text{ prefix-bound none-imp cont-imp}]$)

definition *checked-staged-security-with-query-prefix-dynamic-index-hit*
where
checked-staged-security-with-query-prefix-dynamic-index-hit $T \text{ out} \longleftrightarrow$
 (case out of
 $None \Rightarrow False$
 $| \text{Some } (((\text{prefix-pack}, \text{raw}), \text{raw-state}), \text{result}), \text{final-state}) \Rightarrow$
 $\text{checked-staged-query-prefix-dynamic-index-hit } T$
 $(\text{Some } ((\text{prefix-pack}, \text{raw}), \text{raw-state})))$

definition *checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit*
where
checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
 $\text{trace-table composition-table out} \longleftrightarrow$
checked-staged-security-with-query-prefix-dynamic-index-hit
 $(\text{staged-query-prefix-candidate-pair-query-target-from-prefix}$
 $\text{trace-table composition-table}) \text{ out}$

lemma *checked-staged-security-with-query-prefix-dynamic-index-hit-bound:*
assumes *prefix-bound:*
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit } T)$
 $\text{adversary-initial-state} \leq B$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-dynamic-index-hit } T)$
 $\text{adversary-initial-state} \leq B$

proof (*rule*
checked-staged-security-experiment-with-query-prefix-data-state-bound-by-prefix
 $[OF \text{ prefix-bound}]$)
show
 $\text{checked-staged-security-with-query-prefix-dynamic-index-hit } T \text{ None} \implies$
 $\text{checked-staged-query-prefix-dynamic-index-hit } T \text{ None}$
unfolding *checked-staged-security-with-query-prefix-dynamic-index-hit-def*
by *simp*

next
fix *prefix-out t out*
assume *head:*
 $\text{Some } (\text{prefix-out}, t) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $\text{adversary-initial-state})$
and *cont:*
 $\text{out} \in$

```

    set-dist
      (execute
        (get  $\gg$ 
          ( $\lambda$ prefix-state.
            checked-staged-after-query-prefix-receive-with-verifier  $A$   $i$ 
              prefix-out  $\gg$ 
                ( $\lambda$ result. return ((prefix-out, prefix-state), result))))
          t)
      assume hit:
        checked-staged-security-with-query-prefix-dynamic-index-hit  $T$  out
      show
        checked-staged-query-prefix-dynamic-index-hit  $T$ 
          (Some (prefix-out, t))
      using cont hit
      unfolding checked-staged-security-with-query-prefix-dynamic-index-hit-def
      by (auto elim!: set-dist-bindE split: option.splits prod.splits)
qed

lemma checked-staged-query-prefix-dynamic-index-hit-bound-from-fraction:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets  $A$ 
  and i-bound:  $i < \text{rounds}$ 
  and target-bound:
     $\bigwedge$ prefix prefix-state.
      Some ((prefix, prefix-state), prefix-state)  $\in$ 
        set-dist
          (execute (checked-staged-query-prefix-with-state  $A$   $i$ )
            adversary-initial-state)  $\implies$ 
             $T$  prefix prefix-state  $\subseteq$  query-sample-space  $\wedge$ 
            nnreal (card ( $T$  prefix prefix-state)) /
              nnreal (card query-sample-space)  $\leq C$ 
  shows
    wp-event (checked-staged-query-prefix-receive-with-state  $A$   $i$ )
      (checked-staged-query-prefix-dynamic-index-hit  $T$ )
      adversary-initial-state  $\leq$ 
        staged-phase-relation-error size
          (staged-query-search-queries budgets  $i + 1$ ) +  $C$ 
proof -
  have raw-bound: query-index-raw-preimage-bound
  by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
  have hit-bound:
    wp-event (checked-staged-query-prefix-receive-with-state  $A$   $i$ )
      (checked-staged-query-prefix-dynamic-index-hit  $T$ )
      adversary-initial-state  $\leq$ 
        wp-event (checked-staged-query-prefix-receive-with-state  $A$   $i$ )
          (checked-staged-query-prefix-dynamic-index-prehit  $T$ )
          adversary-initial-state +  $C$ 
  by (rule
    checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error

```

$[OF \text{ raw-bound target-bound}]$)
have *prehit-bound*:
 $wp\text{-event} \ (checked\text{-staged-query-prefix-receive-with-state} \ A \ i)$
 $(checked\text{-staged-query-prefix-dynamic-index-prehit} \ T)$
 $adversary\text{-initial-state} \leq$
 $staged\text{-phase-relation-error} \ size$
 $(staged\text{-query-search-queries} \ budgets \ i + 1)$
proof (rule *checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)
show $HashMap \ adversary\text{-initial-state} = fmempty$
by (simp add: *adversary-initial-state-def*)
show *hash-relation-program*
 $(checked\text{-staged-query-prefix-dynamic-index-prehit-relation} \ A \ i)$
 $adversary\text{-initial-state} \ T)$
 $size \ (staged\text{-query-search-queries} \ budgets \ i + 1)$
 $(checked\text{-staged-query-prefix-receive-with-state} \ A \ i)$
by (rule *hash-relation-program-checked-staged-query-prefix-receive-with-state*
 $[OF \ wf \ controlled]$)
 $(use \ i\text{-bound} \ in \ simp\text{-all},$
 $rule \ checked\text{-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size})$
qed
show *?thesis*
by (rule *order-trans* $[OF \ hit\text{-bound}]$)
 $(intro \ add\text{-mono} \ prehit\text{-bound} \ order\text{-refl})$
qed

lemma *checked-staged-security-with-query-prefix-dynamic-index-hit-bound-from-fraction*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < rounds$
and *target-bound*:
 $\bigwedge prefix \ prefix\text{-state}.$
 $Some \ ((prefix, \ prefix\text{-state}), \ prefix\text{-state}) \in$
 $set\text{-dist}$
 $(execute \ (checked\text{-staged-query-prefix-with-state} \ A \ i)$
 $adversary\text{-initial-state}) \implies$
 $T \ prefix \ prefix\text{-state} \subseteq query\text{-sample-space} \wedge$
 $nnreal \ (card \ (T \ prefix \ prefix\text{-state})) /$
 $nnreal \ (card \ query\text{-sample-space}) \leq C$
shows
 $wp\text{-event}$
 $(checked\text{-staged-security-experiment-with-query-prefix-data-state} \ A \ i)$
 $(checked\text{-staged-security-with-query-prefix-dynamic-index-hit} \ T)$
 $adversary\text{-initial-state} \leq$
 $staged\text{-phase-relation-error} \ size$
 $(staged\text{-query-search-queries} \ budgets \ i + 1) + C$
by (rule *checked-staged-security-with-query-prefix-dynamic-index-hit-bound*,
 $rule \ checked\text{-staged-query-prefix-dynamic-index-hit-bound-from-fraction}$
 $[OF \ wf \ controlled \ i\text{-bound} \ target\text{-bound}]$)

lemma *checked-staged-security-with-query-prefix-candidate-opening-target-from-prefix-hit-bound*:

assumes *raw-bound*: *query-index-raw-preimage-bound*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i \leq \text{rounds}$
and *trace-candidate*:
 partial-trace-table-candidate trace-table trace-openings
and *comp-candidate*:
 partial-composition-table-candidate composition-table
 composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-low*:
 composition-table-low-degree maxDegree composition-table
and *not-all*:
 $\bigwedge \text{prefix prefix-state.}$
 Some ((prefix, prefix-state), prefix-state) \in
 set-dist
 (execute (checked-staged-query-prefix-with-state A i)
 adversary-initial-state) \implies
 $\neg \text{all-queries-consistent trace-table composition-table}$
 (sqp-alphas prefix)

shows

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-opening-query-target-from-prefix
 trace-openings composition-openings i))
 adversary-initial-state \leq
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) + query-error-bound
by (*rule checked-staged-security-with-query-prefix-dynamic-index-hit-bound*,
 rule checked-staged-query-prefix-candidate-opening-target-from-prefix-hit-bound
 [*OF raw-bound wf controlled i-bound trace-candidate comp-candidate*
 trace-low comp-low not-all])

lemma *checked-staged-security-with-query-prefix-candidate-opening-target-from-prefix-hit-bound-from-budgets*:

assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < \text{rounds}$
and *trace-candidate*:
 partial-trace-table-candidate trace-table trace-openings
and *comp-candidate*:
 partial-composition-table-candidate composition-table
 composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-low*:
 composition-table-low-degree maxDegree composition-table
and *not-all*:
 $\bigwedge \text{prefix prefix-state.}$

Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) ⇒
 $\neg$ *all-queries-consistent trace-table composition-table*
(sqp-alphas prefix)
shows
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target-from-prefix
trace-openings composition-openings i))
adversary-initial-state ≤
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) + query-error-bound
proof –
have *raw-bound: query-index-raw-preimage-bound*
by *(rule query-index-raw-preimage-bound-from-sampler-wellformed)*
show *?thesis*
by *(rule*
checked-staged-security-with-query-prefix-candidate-opening-target-from-prefix-hit-bound
[OF raw-bound wf controlled - trace-candidate comp-candidate trace-low
comp-low not-all])
(use i-bound in simp)
qed

lemma *checked-staged-security-with-query-prefix-candidate-pair-target-from-prefix-hit-bound:*
assumes *raw-bound: query-index-raw-preimage-bound*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i ≤ rounds*
and *trace-low: trace-table-low-degree trace-table*
and *comp-low:*
composition-table-low-degree maxDegree composition-table
and *not-all:*
 $\bigwedge$ *prefix prefix-state.*
Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) ⇒
 $\neg$ *all-queries-consistent trace-table composition-table*
(sqp-alphas prefix)
shows
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table))
adversary-initial-state ≤

staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) + query-error-bound
by (rule *checked-staged-security-with-query-prefix-dynamic-index-hit-bound*,
rule checked-staged-query-prefix-candidate-pair-target-from-prefix-hit-bound
[OF raw-bound wf controlled i-bound trace-low comp-low not-all])

lemma *checked-staged-security-with-query-prefix-candidate-pair-target-from-prefix-hit-bound-from-budgets:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i < \text{rounds}$*
and *trace-low: trace-table-low-degree trace-table*
and *comp-low:*
composition-table-low-degree maxDegree composition-table
and *not-all:*
 $\bigwedge \text{prefix prefix-state.}$
Some ((prefix, prefix-state), prefix-state) $\in$
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) $\implies$
 $\neg$ *all-queries-consistent trace-table composition-table*
(sqp-alphas prefix)

shows

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table))
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) + query-error-bound

proof –

have *raw-bound: query-index-raw-preimage-bound*
by (rule *query-index-raw-preimage-bound-from-sampler-wellformed*)
show *?thesis*
by (rule
checked-staged-security-with-query-prefix-candidate-pair-target-from-prefix-hit-bound
[OF raw-bound wf controlled - trace-low comp-low not-all])
(use i-bound in simp)

qed

lemma *checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-bound-from-budgets:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i < \text{rounds}$*
and *trace-low: trace-table-low-degree trace-table*
and *comp-low:*
composition-table-low-degree maxDegree composition-table
and *not-all:*
 $\bigwedge \text{prefix prefix-state.}$

Some $((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\neg \text{all-queries-consistent trace-table composition-table}$
 $(\text{sqp-alphas prefix})$

shows

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit}$
 $\text{trace-table composition-table})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$

unfolding

checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-def

by (rule

checked-staged-security-with-query-prefix-candidate-pair-target-from-prefix-hit-bound-from-budgets
 $[OF \text{ wf controlled } i\text{-bound trace-low comp-low not-all}])$

lemma *powers-scaled-eq-imp-eq:*

assumes *powers-scaled idx = powers-scaled idx'*

shows *idx = idx'*

proof –

have *left: powers-scaled idx ! 0 = idx*

unfolding *powers-scaled-def* **using** *powers-pos* **by** *simp*

have *right: powers-scaled idx' ! 0 = idx'*

unfolding *powers-scaled-def* **using** *powers-pos* **by** *simp*

show *?thesis*

using *assms left right* **by** *metis*

qed

lemma *partial-query-success-indices-at-card-le-one:*

card (partial-query-success-indices-at trace-openings

composition-openings as i) ≤ Suc 0

proof –

have *finite-target:*

finite (partial-query-success-indices-at trace-openings

composition-openings as i)

by (rule *finite-subset[OF partial-query-success-indices-at-subset*
finite-query-sample-space])

have *pairwise:*

$\forall \text{idx} \in \text{partial-query-success-indices-at trace-openings}$

$\text{composition-openings as } i.$

$\forall \text{idx}' \in \text{partial-query-success-indices-at trace-openings}$

$\text{composition-openings as } i.$

$\text{idx} = \text{idx}'$

proof (*intro ballI*)

fix *idx idx'*

```

assume idx-in:
  idx ∈ partial-query-success-indices-at trace-openings
    composition-openings as i
and idx'-in:
  idx' ∈ partial-query-success-indices-at trace-openings
    composition-openings as i
have idx-indices:
  map opening-index (trace-openings ! i) = powers-scaled idx
using idx-in
unfolding partial-query-success-indices-at-def
  partial-query-round-consistent-def
by simp
have idx'-indices:
  map opening-index (trace-openings ! i) = powers-scaled idx'
using idx'-in
unfolding partial-query-success-indices-at-def
  partial-query-round-consistent-def
by simp
have powers-scaled idx = powers-scaled idx'
using idx-indices idx'-indices by simp
then show idx = idx'
by (rule powers-scaled-eq-imp-eq)
qed
show ?thesis
using card-le-Suc0-iff-eq[OF finite-target] pairwise by simp
qed

```

```

lemma prob-divide-right-mono:
  assumes  $(a::prob) \leq b$ 
  shows  $a / c \leq b / c$ 
  using assms
  apply transfer
  by (simp add: divide-right-mono)

```

```

lemma partial-query-success-indices-at-singleton-fraction-bound:
  nnreal
    (card
      (partial-query-success-indices-at trace-openings composition-openings
        as i)) /
    nnreal (card query-sample-space) ≤
    (1::prob) / nnreal (card query-sample-space)
proof –
  have card-le:
    nnreal
      (card
        (partial-query-success-indices-at trace-openings composition-openings
          as i)) ≤ (1::prob)
  using partial-query-success-indices-at-card-le-one
  by simp

```


show *?thesis*
by (rule prob-divide-right-mono[OF card-le])
qed

lemma staged-query-prefix-candidate-opening-query-target-from-prefix-singleton-fraction-bound:
nnreal
 (card
 (staged-query-prefix-candidate-opening-query-target-from-prefix
 trace-openings composition-openings i prefix prefix-state)) /
nnreal (card query-sample-space) ≤
 (1::prob) / *nnreal* (card query-sample-space)
unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
by (rule partial-query-success-indices-at-singleton-fraction-bound)

lemma checked-staged-query-prefix-candidate-opening-target-from-prefix-singleton-hit-bound:
assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i \leq \text{rounds}$
shows
wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-opening-query-target-from-prefix
 trace-openings composition-openings i))
adversary-initial-state ≤
staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 (1::prob) / *nnreal* (card query-sample-space)

proof –
have hit-bound:
wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-opening-query-target-from-prefix
 trace-openings composition-openings i))
adversary-initial-state ≤
wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-prehit
 (staged-query-prefix-candidate-opening-query-target-from-prefix
 trace-openings composition-openings i))
adversary-initial-state +
 (1::prob) / *nnreal* (card query-sample-space)

proof (rule checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
 [OF raw-bound])
fix prefix prefix-state
assume
 Some ((prefix, prefix-state), prefix-state) ∈
 set-dist
 (execute (checked-staged-query-prefix-with-state A i)
 adversary-initial-state)

```

show
  staged-query-prefix-candidate-opening-query-target-from-prefix
    trace-openings composition-openings i prefix prefix-state  $\subseteq$ 
    query-sample-space  $\wedge$ 
    nnreal
      (card
        (staged-query-prefix-candidate-opening-query-target-from-prefix
          trace-openings composition-openings i prefix prefix-state)) /
        nnreal (card query-sample-space)
       $\leq (1::prob) / \text{nnreal (card query-sample-space)}$ 
using
  staged-query-prefix-candidate-opening-query-target-from-prefix-subset
  staged-query-prefix-candidate-opening-query-target-from-prefix-singleton-fraction-bound
by blast
qed
have prehit-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-candidate-opening-query-target-from-prefix
        trace-openings composition-openings i))
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1)
by (rule checked-staged-query-prefix-candidate-opening-target-from-prefix-prehit-bound
  [OF wf controlled i-bound])
have tail:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-candidate-opening-query-target-from-prefix
        trace-openings composition-openings i))
  adversary-initial-state +
  (1::prob) / nnreal (card query-sample-space)  $\leq$ 
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space)
by (intro add-mono prehit-bound order-refl)
show ?thesis
by (rule order-trans[OF hit-bound tail])
qed

lemma checked-staged-security-with-query-prefix-candidate-opening-target-from-prefix-singleton-hit-bound:
assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i  $\leq$  rounds
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-dynamic-index-hit

```

$(\text{staged-query-prefix-candidate-opening-query-target-from-prefix}$
 $\text{trace-openings composition-openings } i))$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(1::\text{prob}) / \text{nnreal (card query-sample-space)}$
by $(\text{rule checked-staged-security-with-query-prefix-dynamic-index-hit-bound},$
 rule
 $\text{checked-staged-query-prefix-candidate-opening-target-from-prefix-singleton-hit-bound}$
 $[\text{OF raw-bound wf controlled i-bound}])$

lemma *checked-staged-security-with-query-prefix-candidate-opening-target-from-prefix-singleton-hit-bound-from-budget*

assumes $\text{wf: staged-budget-wellformed budgets}$
and $\text{controlled: staged-adversary-controlled budgets } A$
and $\text{i-bound: } i < \text{rounds}$

shows

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-dynamic-index-hit}$
 $(\text{staged-query-prefix-candidate-opening-query-target-from-prefix}$
 $\text{trace-openings composition-openings } i))$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(1::\text{prob}) / \text{nnreal (card query-sample-space)}$

proof —

have $\text{raw-bound: query-index-raw-preimage-bound}$
by $(\text{rule query-index-raw-preimage-bound-from-sampler-wellformed})$
show $?thesis$
by $(\text{rule}$
 $\text{checked-staged-security-with-query-prefix-candidate-opening-target-from-prefix-singleton-hit-bound}$
 $[\text{OF raw-bound wf controlled}])$
 $(\text{use i-bound in simp})$

qed

definition *checked-staged-security-with-actual-query-prefix-candidate-opening-hit*

where

$\text{checked-staged-security-with-actual-query-prefix-candidate-opening-hit}$
 $\text{trace-openings composition-openings } i \text{ out} \longleftrightarrow$
 $\text{checked-staged-security-with-query-prefix-dynamic-index-hit}$
 $(\text{staged-query-prefix-candidate-opening-query-target-from-prefix}$
 $\text{trace-openings composition-openings } i) \text{ out}$

lemma *checked-staged-security-with-actual-query-prefix-candidate-opening-hitE:*

assumes hit:
 $\text{checked-staged-security-with-actual-query-prefix-candidate-opening-hit}$
 $\text{trace-openings composition-openings } i$
 $(\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}))$

obtains

index (*to-nat raw*) $\in$
staged-query-prefix-candidate-opening-query-target-from-prefix
trace-openings composition-openings i prefix prefix-state

using *hit*

unfolding

checked-staged-security-with-actual-query-prefix-candidate-opening-hit-def
checked-staged-security-with-query-prefix-dynamic-index-hit-def
checked-staged-query-prefix-dynamic-index-hit-def

by *simp*

lemma *checked-staged-security-with-actual-query-prefix-candidate-opening-hit-bound:*

assumes *raw-bound: query-index-raw-preimage-bound*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: $i \leq \text{rounds}$*

shows

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-actual-query-prefix-candidate-opening-hit*
trace-openings composition-openings i)

adversary-initial-state $\leq$

staged-phase-relation-error size

(*staged-query-search-queries budgets i + 1*) +

($1::\text{prob}$) / *nnreal (card query-sample-space)*

unfolding

checked-staged-security-with-actual-query-prefix-candidate-opening-hit-def

by (*rule*

checked-staged-security-with-query-prefix-candidate-opening-target-from-prefix-singleton-hit-bound
[*OF raw-bound wf controlled i-bound*])

lemma *checked-staged-security-with-actual-query-prefix-candidate-opening-hit-bound-from-budgets:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: $i < \text{rounds}$*

shows

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-actual-query-prefix-candidate-opening-hit*
trace-openings composition-openings i)

adversary-initial-state $\leq$

staged-phase-relation-error size

(*staged-query-search-queries budgets i + 1*) +

($1::\text{prob}$) / *nnreal (card query-sample-space)*

unfolding

checked-staged-security-with-actual-query-prefix-candidate-opening-hit-def

by (*rule*

checked-staged-security-with-query-prefix-candidate-opening-target-from-prefix-singleton-hit-bound-from-budgets
[*OF wf controlled i-bound*])

lemma *checked-staged-security-with-data-state-bound-by-actual-query-prefix-candidate-opening-hit*:

assumes *raw-bound*: *query-index-raw-preimage-bound*

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *i-bound*: $i < \text{rounds}$

and *event-imp*:

$\bigwedge \text{out.}$

(*case out of*

$\text{None} \Rightarrow E \text{ None}$

| $\text{Some } (\text{packed}, t) \Rightarrow E (\text{Some } (\text{snd packed}, t))) \Rightarrow$

checked-staged-security-with-actual-query-prefix-candidate-opening-hit

trace-openings composition-openings i out

shows

wp-event (checked-staged-security-experiment-with-data-state A) E

adversary-initial-state $\leq$

staged-phase-relation-error size

(*staged-query-search-queries budgets i + 1*) +

($1::\text{prob}$) / *nnreal (card query-sample-space)*

proof –

have *projection*:

wp-event (checked-staged-security-experiment-with-data-state A) E

adversary-initial-state =

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

($\lambda \text{out. case out of}$

$\text{None} \Rightarrow E \text{ None}$

| $\text{Some } (\text{packed}, t) \Rightarrow E (\text{Some } (\text{snd packed}, t)))$

adversary-initial-state

by (*rule*

checked-staged-security-experiment-with-query-prefix-data-state-projection-event

[*OF i-bound*])

have *lifted-bound*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

($\lambda \text{out. case out of}$

$\text{None} \Rightarrow E \text{ None}$

| $\text{Some } (\text{packed}, t) \Rightarrow E (\text{Some } (\text{snd packed}, t)))$

adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-actual-query-prefix-candidate-opening-hit*

trace-openings composition-openings i)

adversary-initial-state

by (*rule wp-event-mono*) (*use event-imp in blast*)

have *fixed-bound*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-actual-query-prefix-candidate-opening-hit*

```

    trace-openings composition-openings i)
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space)
by (rule
  checked-staged-security-with-actual-query-prefix-candidate-opening-hit-bound
  [OF raw-bound wf controlled])
  (use i-bound in simp)
show ?thesis
  unfolding projection
  by (rule order-trans[OF lifted-bound fixed-bound])
qed

lemma checked-staged-security-with-data-state-bound-by-actual-query-prefix-candidate-opening-hit-from-budgets:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and event-imp:
    ∧ out.
    (case out of
      None ⇒ E None
    | Some (packed, t) ⇒ E (Some (snd packed, t))) ⇒
    checked-staged-security-with-actual-query-prefix-candidate-opening-hit
    trace-openings composition-openings i out
  shows
    wp-event (checked-staged-security-experiment-with-data-state A) E
    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    (1::prob) / nnreal (card query-sample-space)
proof –
  have raw-bound: query-index-raw-preimage-bound
  by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
  show ?thesis
  by (rule
    checked-staged-security-with-data-state-bound-by-actual-query-prefix-candidate-opening-hit
    [OF raw-bound wf controlled i-bound event-imp])
qed

lemma checked-staged-security-with-query-prefix-data-state-bound-by-actual-query-prefix-candidate-opening-hit:
  assumes raw-bound: query-index-raw-preimage-bound
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and event-imp:
    ∧ out. E out ⇒
    checked-staged-security-with-actual-query-prefix-candidate-opening-hit
    trace-openings composition-openings i out

```

shows
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 E adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 (1::prob) / nnreal (card query-sample-space)

proof –
have *event-le*:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 E adversary-initial-state $\leq$
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
 trace-openings composition-openings i)
 adversary-initial-state
by (rule *wp-event-mono*) (rule *event-imp*)
have *fixed-bound*:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
 trace-openings composition-openings i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 (1::prob) / nnreal (card query-sample-space)
by (rule
 checked-staged-security-with-actual-query-prefix-candidate-opening-hit-bound
 [OF raw-bound wf controlled])
 (use *i-bound* **in** *simp*)
show ?thesis
by (rule order-trans[OF *event-le fixed-bound*])
qed

lemma *checked-staged-security-with-query-prefix-data-state-bound-by-actual-query-prefix-candidate-opening-hit-from-*
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets A
and *i-bound*: $i < \text{rounds}$
and *event-imp*:
 $\bigwedge \text{out}. E \text{ out} \implies$
 checked-staged-security-with-actual-query-prefix-candidate-opening-hit
 trace-openings composition-openings i out
shows
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 E adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +

$(1::\text{prob}) / \text{nnreal } (\text{card } \text{query-sample-space})$
proof –
have *raw-bound: query-index-raw-preimage-bound*
by (rule *query-index-raw-preimage-bound-from-sampler-wellformed*)
show *?thesis*
by (rule
checked-staged-security-with-query-prefix-data-state-bound-by-actual-query-prefix-candidate-opening-hit
[OF raw-bound wf controlled i-bound event-imp])
qed

lemma *checked-staged-query-prefix-with-state-alpha-prefix-support:*

assumes *outcome:*

Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state)

obtains *alpha-prefix alpha-prefix-state where*

alpha-prefix =
(sqp-trace-root prefix, sqp-trace-fri-roots prefix,
sqp-trace-fri-challenges prefix, sqp-trace-final prefix)
Some (alpha-prefix, alpha-prefix-state) ∈
set-dist
(execute (staged-alpha-prefix-program A) adversary-initial-state)

proof –

from *outcome* **obtain** *prefix-out prefix-state' where prefix-program-out:*

Some (prefix-out, prefix-state') ∈
set-dist
(execute (checked-staged-query-challenge-prefix-program A i)
adversary-initial-state)

and *prefix-eq0: prefix-out = prefix*

unfolding *checked-staged-query-prefix-with-state-def*

by (auto elim!: *set-dist-bindE dest!: get-bind-return-state-outcome*)

obtain *fr s1 s2 trace-roots trace-bs s3 trace-final s4 s5 as*

s6 dg s7 s8 s9 composition-roots composition-bs s10
composition-final s11 query-start query-chunks where
fr-out:

Some (fr, s1) ∈
set-dist (execute (trace-root-stage A) adversary-initial-state)

and *record-fr:*

Some ((), s2) ∈ set-dist (execute (record-staged-message fr) s1)

and *trace-fri:*

Some ((trace-roots, trace-bs), s3) ∈
set-dist
(execute
(staged-trace-fri-program A 0 (ceil-log clength) []) s2)

and *trace-final-out:*

Some (trace-final, s4) ∈
set-dist (execute (trace-final-stage A trace-bs) s3)

and *record-trace-final:*


```

    Some ( $\langle \rangle$ ,  $s5$ )  $\in$ 
      set-dist (execute (record-staged-message trace-final)  $s4$ )
and alpha-out:
    Some ( $as$ ,  $s6$ )  $\in$ 
      set-dist (execute (staged-alpha-program (length spec))  $s5$ )
and degree-out:
    Some ( $dg$ ,  $s7$ )  $\in$  set-dist (execute (degree-stage  $A$   $as$ )  $s6$ )
and record-degree:
    Some ( $\langle \rangle$ ,  $s8$ )  $\in$ 
      set-dist (execute (record-staged-message  $dg$ )  $s7$ )
and assert-degree:
    Some ( $\langle \rangle$ ,  $s9$ )  $\in$ 
      set-dist
        (execute
          (assert (ceil-log (to-nat  $dg$  + 1)  $\leq$ 
            ceil-log (maxDegree + 1)))  $s8$ )
and comp-fri:
    Some ( $\langle \langle composition\text{-}roots, composition\text{-}bs \rangle \rangle$ ,  $s10$ )  $\in$ 
      set-dist
        (execute
          (staged-composition-fri-program  $A$   $dg$  0
            (ceil-log (to-nat  $dg$  + 1))  $\square$ )  $s9$ )
and comp-final:
    Some (composition-final,  $s11$ )  $\in$ 
      set-dist
        (execute (composition-final-stage  $A$   $dg$  composition-bs)  $s10$ )
and record-comp-final:
    Some ( $\langle \rangle$ , query-start)  $\in$ 
      set-dist (execute (record-staged-message composition-final)  $s11$ )
and query-prefix:
    Some (query-chunks, prefix-state')  $\in$ 
      set-dist
        (execute
          (checked-staged-query-program  $A$  trace-roots composition-roots
            0  $i$ ) query-start)
and prefix-out-eq:
    prefix-out =
      ( $\sqcup$ sqp-trace-root = fr,
        sqp-trace-fri-roots = trace-roots,
        sqp-trace-fri-challenges = trace-bs,
        sqp-trace-final = trace-final,
        sqp-alphas = as,
        sqp-degree = dg,
        sqp-composition-fri-roots = composition-roots,
        sqp-composition-fri-challenges = composition-bs,
        sqp-composition-final = composition-final,
        sqp-query-chunks = query-chunks)
by (rule checked-staged-query-challenge-prefix-program-outcomeE[
  OF prefix-program-out])

```

```

have prefix-eq:
  prefix =
    (|sqp-trace-root = fr,
      sqp-trace-fri-roots = trace-roots,
      sqp-trace-fri-challenges = trace-bs,
      sqp-trace-final = trace-final,
      sqp-alphas = as,
      sqp-degree = dg,
      sqp-composition-fri-roots = composition-roots,
      sqp-composition-fri-challenges = composition-bs,
      sqp-composition-final = composition-final,
      sqp-query-chunks = query-chunks)
    using prefix-eq0 prefix-out-eq by simp
let ?alpha-prefix = (fr, trace-roots, trace-bs, trace-final)
have alpha-prefix-out:
  Some (?alpha-prefix, s5) ∈
    set-dist
    (execute (staged-alpha-prefix-program A) adversary-initial-state)
proof –
  have after-record-final:
    Some (?alpha-prefix, s5) ∈
      set-dist
      (execute
        (record-staged-message trace-final  $\gg$ 
          ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
        s4)
    by (rule-tac x=() and t=s5 in set-dist-bindI)
      (rule record-trace-final, simp)
  have after-final:
    Some (?alpha-prefix, s5) ∈
      set-dist
      (execute
        (trace-final-stage A trace-bs  $\gg$ 
          ( $\lambda$ trace-final.
            record-staged-message trace-final  $\gg$ 
              ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))))
        s3)
    by (rule-tac x=trace-final and t=s4 in set-dist-bindI)
      (rule trace-final-out, rule after-record-final)
  have after-trace:
    Some (?alpha-prefix, s5) ∈
      set-dist
      (execute
        (staged-trace-fri-program A 0 (ceil-log clength) []  $\gg$ 
          ( $\lambda$ (trace-roots, trace-bs).
            trace-final-stage A trace-bs  $\gg$ 
              ( $\lambda$ trace-final.
                record-staged-message trace-final  $\gg$ 
                  ( $\lambda$ -. return

```

```

      (fr, trace-roots, trace-bs, trace-final))))))
    s2)
  apply (rule-tac x=(trace-roots, trace-bs) and t=s3
    in set-dist-bindI)
  apply (rule trace-fri)
  apply (simp add: after-final)
  done
have after-record-root:
  Some (?alpha-prefix, s5) ∈
    set-dist
    (execute
      (record-staged-message fr >>=
        (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
          (λ(trace-roots, trace-bs).
            trace-final-stage A trace-bs >>=
              (λtrace-final.
                record-staged-message trace-final >>=
                  (λ-. return
                    (fr, trace-roots, trace-bs, trace-final)))))))
    s1)
  by (rule-tac x=() and t=s2 in set-dist-bindI)
    (rule record-fr, rule after-trace)
show ?thesis
  unfolding staged-alpha-prefix-program-def
  apply (rule-tac x=fr and t=s1 in set-dist-bindI)
  apply (rule fr-out)
  apply (rule after-record-root)
  done
qed
show ?thesis
  by (rule that[OF - alpha-prefix-out]) (simp add: prefix-eq)
qed

lemma checked-staged-security-with-query-prefix-data-state-alpha-prefix-support:
  assumes outcome:
    Some (((((prefix, prefix-state), raw), raw-state), result), final-state)
    ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
  obtains alpha-prefix alpha-prefix-state where
    alpha-prefix =
      (sqp-trace-root prefix, sqp-trace-fri-roots prefix,
        sqp-trace-fri-challenges prefix, sqp-trace-final prefix)
    Some (alpha-prefix, alpha-prefix-state) ∈
      set-dist
      (execute (staged-alpha-prefix-program A) adversary-initial-state)
proof -

```

from *outcome* **obtain** *t* **where** *prefix-receive*:
Some (((*prefix*, *prefix-state*), *raw*), *t*) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *t-eq*: *t* = *raw-state*
unfolding *checked-staged-security-experiment-with-query-prefix-data-state-def*
by (*auto elim!*: *set-dist-bindE*)
have *prefix-support*:
Some ((*prefix*, *prefix-state*), *prefix-state*) $\in$
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state)
by (*rule checked-staged-query-prefix-receive-with-state-prefix-support*)
(use prefix-receive t-eq in simp)
show *?thesis*
by (*rule*
checked-staged-query-prefix-with-state-alpha-prefix-support
[OF prefix-support])
(rule that)
qed

lemma *checked-staged-security-with-query-prefix-data-state-alphas-eq*:
assumes *outcome*:
Some ((((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((data, *attacker-state*), *result*)), *final-state*)
 $\in$
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)
shows *staged-alphas data* = *sqp-alphas prefix*
using *outcome*
unfolding
checked-staged-security-experiment-with-query-prefix-data-state-def
checked-staged-after-query-prefix-receive-with-verifier-def
checked-staged-after-query-prefix-receive-def
by (*auto elim!*: *set-dist-bindE split: prod.splits*)

definition *checked-staged-security-with-query-prefix-authenticated-opening-hit*
where
checked-staged-security-with-query-prefix-authenticated-opening-hit
trace-openings composition-openings i out $\longleftrightarrow$
(case out of
None $\Rightarrow$ *False*
 $|$ *Some* ((((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((data, *attacker-state*), *result*)), *final-state*) $\Rightarrow$
staged-composition-fri-roots data $\neq$ $\square$ $\wedge$
partial-authenticated-table (staged-trace-root data)

$(scale * clength) \text{ trace-openings final-state} \wedge$
 $partial\text{-authenticated-table } (hd \text{ (staged-composition-fri-roots data)})$
 $(scale * clength) \text{ composition-openings final-state} \wedge$
 $partial\text{-query-openings-consistent trace-openings composition-openings}$
 $(staged\text{-alphas data}) (index \text{ (to-nat raw)})$

definition *checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit*
where

$checked\text{-staged-security-with-query-prefix-prefix-authenticated-opening-hit}$
 $trace\text{-openings composition-openings } i \text{ out} \longleftrightarrow$
 $(case \text{ out of}$
 $\quad None \Rightarrow False$
 $\mid Some \text{ (((((prefix, prefix-state), raw), raw-state),$
 $\quad ((data, attacker-state), result)), final-state) \Rightarrow$
 $\quad staged\text{-composition-fri-roots data} \neq [] \wedge$
 $\quad partial\text{-authenticated-table (staged-trace-root data)}$
 $\quad (scale * clength) \text{ trace-openings prefix-state} \wedge$
 $\quad partial\text{-authenticated-table (hd (staged-composition-fri-roots data)})$
 $\quad (scale * clength) \text{ composition-openings prefix-state} \wedge$
 $\quad partial\text{-query-openings-consistent trace-openings composition-openings}$
 $\quad (staged\text{-alphas data}) (index \text{ (to-nat raw)}))$

definition *checked-staged-security-with-query-prefix-opening-path-output-hit*
where

$checked\text{-staged-security-with-query-prefix-opening-path-output-hit}$
 $trace\text{-openings composition-openings out} \longleftrightarrow$
 $(case \text{ out of}$
 $\quad None \Rightarrow False$
 $\mid Some \text{ (((((prefix, prefix-state), raw), raw-state),$
 $\quad ((data, attacker-state), result)), final-state) \Rightarrow$
 $\quad (\exists \text{ opening} \in set \text{ trace-openings} \cup set \text{ composition-openings.}$
 $\quad hash\text{-map-new-output-hit}$
 $\quad (merkle\text{-path-target-roots final-state}$
 $\quad \quad (opening\text{-root opening})$
 $\quad \quad (opening\text{-length opening})$
 $\quad \quad (opening\text{-index opening})$
 $\quad \quad (opening\text{-value opening})$
 $\quad \quad (opening\text{-path opening}))$
 $\quad prefix\text{-state final-state}))$

lemma *checked-staged-security-with-query-prefix-opening-path-output-hitE:*

assumes *hit:*

$checked\text{-staged-security-with-query-prefix-opening-path-output-hit}$
 $trace\text{-openings composition-openings out}$

obtains $prefix \text{ prefix-state raw raw-state data attacker-state result}$
 $final\text{-state opn}$ **where**

$out = Some \text{ (((((prefix, prefix-state), raw), raw-state),$
 $\quad ((data, attacker-state), result)), final-state)$

$opn \in set \text{ trace-openings} \cup set \text{ composition-openings}$

```

hash-map-new-output-hit
  (merkle-path-target-roots final-state
    (opening-root opn)
    (opening-length opn)
    (opening-index opn)
    (opening-value opn)
    (opening-path opn))
  prefix-state final-state
finite
  (merkle-path-target-roots final-state
    (opening-root opn)
    (opening-length opn)
    (opening-index opn)
    (opening-value opn)
    (opening-path opn))
card
  (merkle-path-target-roots final-state
    (opening-root opn)
    (opening-length opn)
    (opening-index opn)
    (opening-value opn)
    (opening-path opn)) ≤ Suc (length (opening-path opn))
using hit
unfolding checked-staged-security-with-query-prefix-opening-path-output-hit-def
by (cases out)
  (auto intro: that card-merkle-path-target-roots-le split: prod.splits)

```

lemma *checked-staged-security-experiment-with-query-prefix-data-state-prefix-extends-final:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *outcome:*

*Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state)
 ∈
 set-dist
 (execute
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 adversary-initial-state)*

shows *prefix-state ≤ final-state*

proof –

from *outcome* **obtain** *t* **where** *head-support:*

*Some (((prefix, prefix-state), raw), t) ∈
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state)*

and *t-eq: t = raw-state*

and *cont-support:*

Some (((data, attacker-state), result), final-state) ∈

```

    set-dist
    (execute
      (checked-staged-after-query-prefix-receive-with-verifier A i
        ((prefix, prefix-state), raw))
      raw-state)
  unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have head:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  using head-support t-eq by simp
have recv:
  Some (raw, raw-state) ∈
    set-dist (execute receive-query-index-challenge prefix-state)
  by (rule checked-staged-query-prefix-receive-with-state-receive-support
    [OF head])
have prefix-ext-raw: prefix-state ≤ raw-state
  by (rule receive-query-index-challenge-extends[OF recv])
have data-out:
  Some (data, attacker-state) ∈
    set-dist
      (execute
        (checked-staged-after-query-prefix-receive A i
          ((prefix, prefix-state), raw))
        raw-state)
  by (rule
    checked-staged-after-query-prefix-receive-with-verifier-outcome(1)
    [OF cont-support])
have verifier-out:
  Some (result, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
  by (rule
    checked-staged-after-query-prefix-receive-with-verifier-outcome(2)
    [OF cont-support])
have raw-ext-attacker: raw-state ≤ attacker-state
  by (rule checked-staged-after-query-prefix-receive-extends
    [OF wf controlled i-bound data-out])
have attacker-ext-verifier:
  attacker-state ≤
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
proof -
  have attacker-state ≤ attacker-state
    unfolding less-eq-hash-ext-def less-eq-fmap-def by simp

```

```

then show ?thesis
  by (rule hash-extends-verifier-state-from-adversary-right)
qed
have verifier-ext-final:
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)  $\leq$ 
    final-state
  by (rule verify-monad-hash-extends[OF verifier-out])
have raw-ext-final: raw-state  $\leq$  final-state
  by (rule hash-ext-trans[OF raw-ext-attacker])
    (rule hash-ext-trans[OF attacker-ext-verifier verifier-ext-final])
show ?thesis
  by (rule hash-ext-trans[OF prefix-ext-raw raw-ext-final])
qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-prefix-or-path-output-hit:
assumes ext: prefix-state  $\leq$  final-state
and hit:
  checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i
    (Some (((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
shows
  checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
    trace-openings composition-openings i
    (Some (((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))  $\vee$ 
  checked-staged-security-with-query-prefix-opening-path-output-hit
    trace-openings composition-openings
    (Some (((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
proof –
have comp-nonempty: staged-composition-fri-roots data  $\neq$  []
and trace-table:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and comp-table:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings final-state
and consistent:
  partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
using hit
unfolding checked-staged-security-with-query-prefix-authenticated-opening-hit-def
by simp-all
have trace-pullback:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings prefix-state  $\vee$ 
    ( $\exists$  opn  $\in$  set trace-openings.

```



```

    hash-map-new-output-hit
      (merkle-path-target-roots final-state (staged-trace-root data)
        (scale * clength)
        (opening-index opn)
        (opening-value opn)
        (opening-path opn)) prefix-state final-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit
    [OF ext trace-table])
then show ?thesis
proof
  assume trace-prefix:
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings prefix-state
  have comp-pullback:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings prefix-state ∨
    (∃ opn ∈ set composition-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots final-state
          (hd (staged-composition-fri-roots data))
          (scale * clength)
          (opening-index opn)
          (opening-value opn)
          (opening-path opn)) prefix-state final-state)
    by (rule partial-authenticated-table-pullback-or-new-output-hit
      [OF ext comp-table])
  then show ?thesis
proof
  assume comp-prefix:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings prefix-state
  then have
    checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
      trace-openings composition-openings i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
    using comp-nonempty trace-prefix consistent
  unfolding
    checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-def
  by simp
  then show ?thesis by simp
next
  assume comp-path:
    ∃ opn ∈ set composition-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots final-state
          (hd (staged-composition-fri-roots data))
          (scale * clength)
          (opening-index opn)

```

```

      (opening-value opn)
      (opening-path opn)) prefix-state final-state
then obtain opn where opn-in: opn ∈ set composition-openings
and path-hit:
  hash-map-new-output-hit
  (merkle-path-target-roots final-state
   (hd (staged-composition-fri-roots data))
   (scale * clength)
   (opening-index opn)
   (opening-value opn)
   (opening-path opn)) prefix-state final-state
by blast
have root-eq:
  opening-root opn = hd (staged-composition-fri-roots data)
using comp-table opn-in
unfolding partial-authenticated-table-def by blast
have len-eq: opening-length opn = scale * clength
using comp-table opn-in
unfolding partial-authenticated-table-def by blast
have
  checked-staged-security-with-query-prefix-opening-path-output-hit
  trace-openings composition-openings
  (Some (((((prefix, prefix-state), raw), raw-state),
            ((data, attacker-state), result)), final-state))
proof –
  have path-hit':
    hash-map-new-output-hit
    (merkle-path-target-roots final-state
     (opening-root opn)
     (opening-length opn)
     (opening-index opn)
     (opening-value opn)
     (opening-path opn)) prefix-state final-state
    using path-hit root-eq len-eq by simp
  show ?thesis
    unfolding
      checked-staged-security-with-query-prefix-opening-path-output-hit-def
    using opn-in path-hit' by auto
  qed
then show ?thesis by simp
qed
next
assume trace-path:
  ∃ opn ∈ set trace-openings.
  hash-map-new-output-hit
  (merkle-path-target-roots final-state (staged-trace-root data)
   (scale * clength)
   (opening-index opn)
   (opening-value opn)

```

```

      (opening-path opn)) prefix-state final-state
then obtain opn where opn-in: opn ∈ set trace-openings
and path-hit:
  hash-map-new-output-hit
    (merkle-path-target-roots final-state (staged-trace-root data)
      (scale * clength)
      (opening-index opn)
      (opening-value opn)
      (opening-path opn)) prefix-state final-state
  by blast
have root-eq: opening-root opn = staged-trace-root data
  using trace-table opn-in
  unfolding partial-authenticated-table-def by blast
have len-eq: opening-length opn = scale * clength
  using trace-table opn-in
  unfolding partial-authenticated-table-def by blast
have
  checked-staged-security-with-query-prefix-opening-path-output-hit
    trace-openings composition-openings
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
proof –
  have path-hit':
    hash-map-new-output-hit
      (merkle-path-target-roots final-state
        (opening-root opn)
        (opening-length opn)
        (opening-index opn)
        (opening-value opn)
        (opening-path opn)) prefix-state final-state
    using path-hit root-eq len-eq by simp
  show ?thesis
    unfolding checked-staged-security-with-query-prefix-opening-path-output-hit-def
    using opn-in path-hit' by auto
qed
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-prefix-or-path-output-hit-on-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and support:
  out ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)

```

```

and hit:
  checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i out
shows
  checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
    trace-openings composition-openings i out  $\vee$ 
  checked-staged-security-with-query-prefix-opening-path-output-hit
    trace-openings composition-openings out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding checked-staged-security-with-query-prefix-authenticated-opening-hit-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    by (cases packed, auto split: prod.splits)
  have ext: prefix-state  $\leq$  final-state
    by (rule
      checked-staged-security-experiment-with-query-prefix-data-state-prefix-extends-final
      [OF wf controlled i-bound])
      (use support out-eq in simp)
  show ?thesis
    unfolding out-eq
    by (rule
      checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-prefix-or-path-output-hit
      [OF ext])
      (use hit out-eq in simp)
qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-le-actual-query-prefix-hit:
assumes i-bound: i < rounds
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-authenticated-opening-hit
      trace-openings composition-openings i)
    adversary-initial-state  $\leq$ 
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i)

```

```

    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
    out ∈
    set-dist
    (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)
and hit:
    checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i out
show
    checked-staged-security-with-actual-query-prefix-candidate-opening-hit
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings]) i out
proof (cases out)
case None
then show ?thesis
    using hit
    unfolding
    checked-staged-security-with-query-prefix-authenticated-opening-hit-def
    by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
    Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)
    by (cases packed, auto split: prod.splits)
have alphas-eq: staged-alphas data = sqp-alphas prefix
    by (rule checked-staged-security-with-query-prefix-data-state-alphas-eq)
    (use support out-eq in simp)
have consistent:
    partial-query-openings-consistent trace-openings composition-openings
    (sqp-alphas prefix) (index (to-nat raw))
    using hit alphas-eq
    unfolding out-eq
    checked-staged-security-with-query-prefix-authenticated-opening-hit-def
    by simp
have target-hit:
    checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-opening-query-target-from-prefix
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i)
    (Some (((prefix, prefix-state), raw), raw-state))
by (rule

```

```

      checked-staged-query-prefix-candidate-opening-target-from-prefix-single-round-hitI)
    (use i-bound consistent in simp-all)
  show ?thesis
    unfolding out-eq
      checked-staged-security-with-actual-query-prefix-candidate-opening-hit-def
      checked-staged-security-with-query-prefix-dynamic-index-hit-def
    using target-hit by simp
  qed
qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-bound:
  assumes raw-bound: query-index-raw-preimage-bound
    and wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound: i < rounds
  shows
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-authenticated-opening-hit
        trace-openings composition-openings i)
      adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      (1::prob) / nnreal (card query-sample-space)
  proof -
    have actual-bound:
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
          ((replicate rounds []) [i := trace-openings])
          ((replicate rounds []) [i := composition-openings])
          i)
        adversary-initial-state ≤
        staged-phase-relation-error size
        (staged-query-search-queries budgets i + 1) +
        (1::prob) / nnreal (card query-sample-space)
      by (rule checked-staged-security-with-actual-query-prefix-candidate-opening-hit-bound
        [OF raw-bound wf controlled])
        (use i-bound in simp)
    show ?thesis
      by (rule order-trans[
        OF checked-staged-security-with-query-prefix-authenticated-opening-hit-le-actual-query-prefix-hit
          actual-bound])
        (use i-bound in simp)
  qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-budgets:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A

```

and *i-bound*: $i < \text{rounds}$
shows
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
 (checked-staged-security-with-query-prefix-authenticated-opening-hit
 trace-openings composition-openings i)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (staged-query-search-queries *budgets i + 1*) +
 (1::prob) / nnreal (card *query-sample-space*)
proof –
have *raw-bound*: *query-index-raw-preimage-bound*
by (rule *query-index-raw-preimage-bound-from-sampler-wellformed*)
show ?thesis
by (rule
 checked-staged-security-with-query-prefix-authenticated-opening-hit-bound
 [OF *raw-bound wf controlled i-bound*])
qed

lemma *checked-staged-security-with-query-prefix-data-state-bound-by-query-prefix-authenticated-opening-hit*:
assumes *raw-bound*: *query-index-raw-preimage-bound*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < \text{rounds}$
and *event-imp*:
 $\bigwedge \text{out. } E \text{ out} \implies$
 checked-staged-security-with-query-prefix-authenticated-opening-hit
 trace-openings composition-openings i out
shows
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
E adversary-initial-state $\leq$
staged-phase-relation-error size
 (staged-query-search-queries *budgets i + 1*) +
 (1::prob) / nnreal (card *query-sample-space*)
proof –
have *event-le*:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
E adversary-initial-state $\leq$
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
 (checked-staged-security-with-query-prefix-authenticated-opening-hit
 trace-openings composition-openings i)
adversary-initial-state
by (rule *wp-event-mono*) (rule *event-imp*)
have *fixed-bound*:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)

(checked-staged-security-with-query-prefix-authenticated-opening-hit
 trace-openings composition-openings i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 (1::prob) / nnreal (card query-sample-space)
 by (rule checked-staged-security-with-query-prefix-authenticated-opening-hit-bound
 [OF raw-bound wf controlled i-bound])
 show ?thesis
 by (rule order-trans[OF event-le fixed-bound])
 qed

lemma checked-staged-security-with-query-prefix-data-state-bound-by-query-prefix-authenticated-opening-hit-from-bud

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and event-imp:
 $\bigwedge \text{out. } E \text{ out} \implies$
 checked-staged-security-with-query-prefix-authenticated-opening-hit
 trace-openings composition-openings i out

shows

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 E adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 (1::prob) / nnreal (card query-sample-space)

proof –

have raw-bound: query-index-raw-preimage-bound
by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
show ?thesis
by (rule
 checked-staged-security-with-query-prefix-data-state-bound-by-query-prefix-authenticated-opening-hit
 [OF raw-bound wf controlled i-bound event-imp])

qed

lemma checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-le-actual-query-prefix-hit:

assumes i-bound: i < rounds

shows

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
 trace-openings composition-openings i)
 adversary-initial-state $\leq$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])


```

    i)
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
    out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
and hit:
    checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
    trace-openings composition-openings i out
show
    checked-staged-security-with-actual-query-prefix-candidate-opening-hit
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings]) i out
proof (cases out)
case None
then show ?thesis
using hit
unfolding
    checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-def
by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
by (cases packed, auto split: prod.splits)
have alphas-eq: staged-alphas data = sqp-alphas prefix
by (rule checked-staged-security-with-query-prefix-data-state-alphas-eq)
    (use support out-eq in simp)
have consistent:
    partial-query-openings-consistent trace-openings composition-openings
    (sqp-alphas prefix) (index (to-nat raw))
using hit alphas-eq
unfolding out-eq
    checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-def
by simp
have target-hit:
    checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i)
    (Some (((prefix, prefix-state), raw), raw-state))

```

```

    by (rule
        checked-staged-query-prefix-candidate-opening-target-from-prefix-single-round-hitI)
        (use i-bound consistent in simp-all)
  show ?thesis
    unfolding out-eq
        checked-staged-security-with-actual-query-prefix-candidate-opening-hit-def
        checked-staged-security-with-query-prefix-dynamic-index-hit-def
    using target-hit by simp
qed
qed

lemma checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-bound:
  assumes raw-bound: query-index-raw-preimage-bound
    and wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound: i < rounds
  shows
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
        trace-openings composition-openings i)
      adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      (1::prob) / nnreal (card query-sample-space)
  proof -
    have actual-bound:
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
          ((replicate rounds []) [i := trace-openings])
          ((replicate rounds []) [i := composition-openings])
          i)
        adversary-initial-state ≤
        staged-phase-relation-error size
        (staged-query-search-queries budgets i + 1) +
        (1::prob) / nnreal (card query-sample-space)
    by (rule checked-staged-security-with-actual-query-prefix-candidate-opening-hit-bound
        [OF raw-bound wf controlled])
        (use i-bound in simp)
  show ?thesis
    by (rule order-trans[
        OF checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-le-actual-query-prefix-hit
        actual-bound])
        (use i-bound in simp)
  qed

lemma checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-bound-from-budgets:
  assumes wf: staged-budget-wellformed budgets

```

and *controlled*: staged-adversary-controlled budgets A
and *i-bound*: $i < \text{rounds}$
shows
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
 trace-openings composition-openings i)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 (1::prob) / nnreal (card query-sample-space)

proof –

have *raw-bound*: query-index-raw-preimage-bound
by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
show ?thesis
by (rule
 checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-bound
 [OF raw-bound wf controlled i-bound])

qed

lemma *checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-prefix-and-path-output*:

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets A
and *i-bound*: $i < \text{rounds}$
and *prefix-bound*:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
 trace-openings composition-openings i)
 adversary-initial-state $\leq P$
and *path-bound*:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-opening-path-output-hit
 trace-openings composition-openings)
 adversary-initial-state $\leq \text{path-error}$
shows
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-authenticated-opening-hit
 trace-openings composition-openings i)
 adversary-initial-state $\leq P + \text{path-error}$

proof –

have *event-le*:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-authenticated-opening-hit
 trace-openings composition-openings i)
 adversary-initial-state $\leq$

```

wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (λout.
    checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
      trace-openings composition-openings i out ∨
    checked-staged-security-with-query-prefix-opening-path-output-hit
      trace-openings composition-openings out)
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (rule
    checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-prefix-or-path-output-hit-on-support
      [OF wf controlled i-bound])
have union-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (λout.
      checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
        trace-openings composition-openings i out ∨
      checked-staged-security-with-query-prefix-opening-path-output-hit
        trace-openings composition-openings out)
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
      trace-openings composition-openings i)
    adversary-initial-state +
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-opening-path-output-hit
      trace-openings composition-openings)
    adversary-initial-state
by (rule wp-event-union-bound)
have sum-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
      trace-openings composition-openings i)
    adversary-initial-state +
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-opening-path-output-hit
      trace-openings composition-openings)
    adversary-initial-state ≤ P + path-error
by (intro add-mono prefix-bound path-bound)
show ?thesis
by (rule order-trans[OF event-le])
  (rule order-trans[OF union-bound sum-bound])
qed

```

lemma *checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-path-output*:
assumes *raw-bound*: *query-index-raw-preimage-bound*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < \text{rounds}$
and *path-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-opening-path-output-hit*
 trace-openings composition-openings)
 adversary-initial-state $\leq$ *path-error*
shows
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-authenticated-opening-hit*
 trace-openings composition-openings i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
 $(1::\text{prob}) / \text{nnreal } (\text{card } \text{query-sample-space}) + \text{path-error}$
proof –
have *prefix-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit*
 trace-openings composition-openings i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
 $(1::\text{prob}) / \text{nnreal } (\text{card } \text{query-sample-space})$
by (rule *checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-bound*
 [*OF raw-bound wf controlled i-bound*])
show ?thesis
by (rule
 checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-prefix-and-path-output
 [*OF wf controlled i-bound prefix-bound path-bound*])
qed

lemma *accepted-with-partial-initial-openings-aligned-consistent-prefix-target-hit*:

assumes *partial*:
 accepted-with-partial-initial-openings-aligned-consistent s out fr
 f-fri-roots f-final as dg composition-fri-roots final query-idxs
 trace-openings composition-openings
and *i-bound*: $i < \text{rounds}$
and *idx-eq*: *query-idxs ! i = index (to-nat raw)*
and *alphas-eq*: *as = sqp-alphas prefix*

shows
 checked-staged-query-prefix-dynamic-index-hit
 (*staged-query-prefix-candidate-opening-query-target-from-prefix*

```

      trace-openings composition-openings i)
    (Some (((prefix, prefix-state), raw), raw-state))
proof –
  have success:
    query-idxs ! i ∈
      partial-query-success-indices-at trace-openings composition-openings
        as i
  by (rule
    accepted-with-partial-initial-openings-aligned-consistent-success-index
    [OF partial i-bound])
  have index (to-nat raw) ∈
    staged-query-prefix-candidate-opening-query-target-from-prefix
      trace-openings composition-openings i prefix prefix-state
  using success idx-eq alphas-eq
  unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
  by simp
  then show ?thesis
    unfolding checked-staged-query-prefix-dynamic-index-hit-def by simp
qed

```

lemma query-bad-with-aligned-partial-candidates-prefix-target-hitE:

```

assumes bad: query-bad-with-aligned-partial-candidates s out
and i-bound: i < rounds
and idx-eq: query-idxs ! i = index (to-nat raw)
and alphas-eq: as = sqp-alphas prefix
and partial:
  accepted-with-partial-initial-openings-aligned-consistent s out fr
  f-fri-roots f-final as dg composition-fri-roots final query-idxs
  trace-openings composition-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-low:
  composition-table-low-degree maxDegree composition-table
and not-all:
  ¬ all-queries-consistent trace-table composition-table as

```

obtains

```

checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-opening-query-target-from-prefix
    trace-openings composition-openings i)
  (Some (((prefix, prefix-state), raw), raw-state))
nnreal
  (card
    (staged-query-prefix-candidate-opening-query-target-from-prefix
      trace-openings composition-openings i prefix prefix-state)) /
  nnreal (card query-sample-space) ≤ query-error-bound

```

```

proof –
  have hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-opening-query-target-from-prefix
        trace-openings composition-openings i)
      (Some (((prefix, prefix-state), raw), raw-state))
  by (rule
    accepted-with-partial-initial-openings-aligned-consistent-prefix-target-hit
      [OF partial i-bound idx-eq alphas-eq])
  have not-all-prefix:
    ¬ all-queries-consistent trace-table composition-table
      (sqp-alphas prefix)
  using not-all alphas-eq by simp
  have fraction:
    nnreal
      (card
        (staged-query-prefix-candidate-opening-query-target-from-prefix
          trace-openings composition-openings i prefix prefix-state)) /
      nnreal (card query-sample-space) ≤ query-error-bound
  by (rule
    staged-query-prefix-candidate-opening-query-target-from-prefix-fraction-bound
      [OF trace-candidate comp-candidate trace-low comp-low not-all-prefix])
  show ?thesis
    by (rule that[OF hit fraction])
qed

```

lemma *accepted-with-partial-initial-openings-aligned-consistent-candidate-opening-hit-from-prefix*:

```

assumes partial:
  accepted-with-partial-initial-openings-aligned-consistent s out fr
  f-fri-roots f-final as dg composition-fri-roots final query-idxs
  trace-openings composition-openings
and i-bound: i < rounds
and idx-eq: query-idxs ! i = index (to-nat raw)
and alphas-eq: as = sqp-alphas prefix
and builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)

```

shows

```

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
  A trace-openings composition-openings i
  (Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))

```

proof –
have *target-hit*:
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target-from-prefix
trace-openings composition-openings i)
(Some (((prefix, prefix-state), raw), raw-state))
by (rule
accepted-with-partial-initial-openings-aligned-consistent-prefix-target-hit
[OF partial i-bound idx-eq alphas-eq])
show *?thesis*
by (rule
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixI
[OF i-bound builder prefix-receive target-hit])
qed

lemma *accepted-with-partial-initial-openings-aligned-transcript-consistent-candidate-opening-hit-from-prefix*:
assumes *partial*:

accepted-with-partial-initial-openings-aligned-transcript-consistent s
out fr f-fri-roots f-final as dg composition-fri-roots final
query-idxs trace-openings composition-openings

and *i-bound*: *i < rounds*

and *idx-eq*: *query-idxs ! i = index (to-nat raw)*

and *alphas-eq*: *as = sqp-alphas prefix*

and *builder*:

Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)

and *prefix-receive*:

Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)

shows

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A trace-openings composition-openings i
(Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))

proof –

have *aligned*:

accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings

by (rule

accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
[OF partial])

show *?thesis*

by (rule

accepted-with-partial-initial-openings-aligned-consistent-candidate-opening-hit-from-prefix

[OF aligned i-bound idx-eq alphas-eq builder prefix-receive])

qed

lemma *query-bad-with-aligned-partial-candidates-candidate-opening-hit-from-prefixE*:
assumes *bad*: query-bad-with-aligned-partial-candidates *s out*
and *i-bound*: $i < \text{rounds}$
and *idx-eq*: query-idxs ! $i = \text{index } (\text{to-nat raw})$
and *alphas-eq*: $as = \text{sqp-alphas prefix}$
and *partial*:
accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings
and *builder*:
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive*:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
obtains
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A trace-openings composition-openings i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))

proof –
have *hit*:
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A trace-openings composition-openings i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))
by (rule
accepted-with-partial-initial-openings-aligned-consistent-candidate-opening-hit-from-prefix
[OF partial i-bound idx-eq alphas-eq builder prefix-receive])
show ?thesis
by (rule that[OF hit])

qed

lemma *query-bad-with-aligned-transcript-partial-candidates-candidate-opening-hit-from-prefixE*:
assumes *bad*: query-bad-with-aligned-transcript-partial-candidates *s out*
and *i-bound*: $i < \text{rounds}$
and *idx-eq*: query-idxs ! $i = \text{index } (\text{to-nat raw})$
and *alphas-eq*: $as = \text{sqp-alphas prefix}$
and *partial*:
accepted-with-partial-initial-openings-aligned-transcript-consistent s
out fr f-fri-roots f-final as dg composition-fri-roots final
query-idxs trace-openings composition-openings

and *builder*:
Some (data, attacker-state) ∈ set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive*:
Some (((prefix, prefix-state), raw), raw-state) ∈ set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
obtains
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A trace-openings composition-openings i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
proof –
have *hit*:
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A trace-openings composition-openings i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
by (rule
accepted-with-partial-initial-openings-aligned-transcript-consistent-candidate-opening-hit-from-prefix
[OF partial i-bound idx-eq alphas-eq builder prefix-receive])
show *?thesis*
by (rule *that[OF hit]*)
qed

lemma *partial-candidate-query-witness-candidate-opening-hit-from-prefix*:
assumes *partial*:
accepted-with-partial-initial-openings s out fr f-fri-roots f-final as
dg composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings
and *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
and *comp-candidate*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-low*:
composition-table-low-degree maxDegree composition-table
and *not-all*:
 $\neg$ *all-queries-consistent trace-table composition-table as*
and *i-bound*: *i < rounds*
and *alphas-eq*: *as = sqp-alphas prefix*
and *consistent*:
partial-query-openings-consistent (trace-openings ! i)
(composition-openings ! i) as (index (to-nat raw))
and *builder*:

Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive:*
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
shows
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A ((replicate rounds []) [i := trace-openings ! i])
((replicate rounds []) [i := composition-openings ! i]) i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
proof –
have *target-hit:*
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target-from-prefix
((replicate rounds []) [i := trace-openings ! i])
((replicate rounds []) [i := composition-openings ! i])
i)
(Some (((prefix, prefix-state), raw), raw-state))
by (rule
checked-staged-query-prefix-candidate-opening-target-from-prefix-single-round-hitI
(use i-bound consistent alphas-eq in simp-all)
show *?thesis*
by (rule
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixI
[OF i-bound builder prefix-receive target-hit])
qed

lemma *query-bad-with-partial-candidates-witness-candidate-opening-hit-from-prefix:*
assumes *bad: query-bad-with-partial-candidates s out*
and *partial:*
accepted-with-partial-initial-openings s out fr f-fri-roots f-final as
dg composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *comp-candidate:*
partial-composition-table-candidate composition-table
composition-openings
and *trace-low: trace-table-low-degree trace-table*
and *comp-low:*
composition-table-low-degree maxDegree composition-table
and *not-all:*
 $\neg$ *all-queries-consistent trace-table composition-table as*
and *i-bound: i < rounds*

```

and alphas-eq: as = sqp-alphas prefix
and consistent:
  partial-query-openings-consistent (trace-openings ! i)
    (composition-openings ! i) as (index (to-nat raw))
and builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
shows
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
    A ((replicate rounds []) [i := trace-openings ! i])
      ((replicate rounds []) [i := composition-openings ! i]) i
    (Some ((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
by (rule partial-candidate-query-witness-candidate-opening-hit-from-prefix
  [OF partial trace-candidate comp-candidate trace-low comp-low not-all
    i-bound alphas-eq consistent builder prefix-receive])

end

end

```

```

theory Staged-Security-Experiment-Composition-Query-Bridge
imports
  Staged-Security-Experiment-Composition-Query-Residual
  Soundness-Query-Opening-Consistency
  Soundness-Bound-Table-Partial-Openings
  Soundness-Aligned-Partial-Query-Prefix
begin

```

Narrow bridge from the composition-drift query residual to the candidate-opening query target. This is kept separate from the residual split theory because the surrounding staged soundness layers are already heavy to consolidate.

```

context soundness
begin

```

```

lemma partial-query-openings-consistent-synthetic:
  assumes idx-sample: idx ∈ query-sample-space
  obtains trace-openings composition-openings where
    partial-query-openings-consistent trace-openings composition-openings
      as idx
proof –

```

```

let ?trace-openings =
  map ( $\lambda j$ .
    ( $\sqcup$  opening-root = 0,
      opening-length = scale * clength,
      opening-index = j,
      opening-value = 0,
      opening-path = []  $\sqcup$ ) (powers-scaled idx)
  )
let ?composition-openings =
  [ $\sqcup$  opening-root = 0,
    opening-length = scale * clength,
    opening-index = idx,
    opening-value =
      cp-eval as (map opening-value ?trace-openings) ( $h \wedge idx * shift$ ),
    opening-path = []  $\sqcup$ ],
    ( $\sqcup$  opening-root = 0,
      opening-length = scale * clength,
      opening-index = fri-sibling-index (scale * clength) idx,
      opening-value = 0,
      opening-path = []  $\sqcup$ )
have trace-indices:
  map opening-index ?trace-openings = powers-scaled idx
by (simp add: o-def)
have comp-indices:
  map opening-index ?composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
by simp
have consistent:
  partial-query-openings-consistent ?trace-openings
    ?composition-openings as idx
by (rule partial-query-openings-consistentI
    [OF idx-sample trace-indices comp-indices])
  simp
show ?thesis
by (rule that[OF consistent])
qed

```

```

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hitI-from-prefix-receive-
assumes i-bound:  $i < rounds$ 
and builder:
  Some (data, attacker-state)  $\in$ 
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
shows

```

```

checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
proof -
obtain trace-openings composition-openings where consistent:
partial-query-openings-consistent trace-openings composition-openings
(staged-alphas data) (index (to-nat raw))
proof (rule partial-query-openings-consistent-synthetic)
show index (to-nat raw) ∈ query-sample-space
using index-less-query-sample-space
unfolding query-sample-space-def by simp
qed
have component:
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
(staged-alphas data) i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
by (rule
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI-from-prefix-receive
[OF i-bound builder prefix-receive consistent])
show ?thesis
by (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hitI
[OF component])
qed

definition checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
:: ('f staged-adversary ⇒
((((('f × 'f list × 'f list × 'f) ×
'f protocol-channel) × 'f staged-proof-data) ×
'f protocol-channel) × unit list) ×
'f protocol-channel) option ⇒ bool)
where
checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
A out ⇔
(case out of
None ⇒ False
| Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) ⇒
staged-composition-fri-roots data ≠ [] ∧
(∃ i trace-openings composition-openings.
partial-authenticated-table (staged-trace-root data)
(scale * clength) trace-openings final-state ∧
partial-authenticated-table (hd (staged-composition-fri-roots data))
(scale * clength) composition-openings final-state ∧

```

```

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  (staged-alphas data) i
  (Some (((alpha-prefix, alpha-prefix-state), data),
    attacker-state), result), final-state))))

lemma checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hitI:
assumes comp-nonempty: staged-composition-fri-roots data ≠ []
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
  (scale * clength) trace-openings final-state
and comp-auth:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
  (scale * clength) composition-openings final-state
and component:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  (staged-alphas data) i
  (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
shows
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
  A
  (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
proof —
have witness:
  ∃ j trace-openings' composition-openings'.
  partial-authenticated-table (staged-trace-root data)
  (scale * clength) trace-openings' final-state ∧
  partial-authenticated-table (hd (staged-composition-fri-roots data))
  (scale * clength) composition-openings' final-state ∧
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
  ((replicate rounds []) [j := trace-openings'])
  ((replicate rounds []) [j := composition-openings'])
  (staged-alphas data) j
  (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
by (intro exI[of - i] exI[of - trace-openings]
  exI[of - composition-openings] conjI)
  (use trace-auth comp-auth component in simp-all)
show ?thesis
unfolding
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-def
using comp-nonempty witness by simp
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-imp-compact:*
assumes *hit:*
checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
A out
shows
checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
A out
proof (*cases out*)
case *None*
then show *?thesis*
using *hit*
unfolding
checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-def
by *simp*
next
case (*Some packed*)
then obtain *alpha-prefix alpha-prefix-state data attacker-state result*
final-state where out-eq:
out =
Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)
by (*cases packed, auto split: prod.splits*)
from *hit[unfolded out-eq*
checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-def]
obtain *i trace-openings composition-openings where component:*
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
(staged-alphas data) i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
by *auto*
show *?thesis*
unfolding *out-eq*
by (*rule*
checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hitI
[OF component])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-le-compact:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
A)
adversary-initial-state ≤
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
A)


```

    adversary-initial-state
  by (rule wp-event-mono)
    (rule
      checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-imp-compact)

lemma checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hitI-from-current-open
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and comp-nonempty: staged-composition-fri-roots data  $\neq []$ 
and i-bound:  $i < \text{rounds}$ 
and builder:
  Some (data, attacker-state)  $\in$ 
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and verifier:
  Some (result, final-state)  $\in$ 
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))))
and final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and comp-auth:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings final-state
and consistent:
  partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
shows
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
    A
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
proof –
from checked-staged-transcript-program-query-prefix-receive-final-lookup-support
  [OF wf controlled i-bound builder verifier final-lookup]
obtain prefix prefix-state raw-state where prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)

```

```

    by blast
  have component:
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      (staged-alphas data) i
      (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
    by (rule
      checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI-from-prefix-receive
        [OF i-bound builder prefix-receive consistent])
  show ?thesis
    by (rule
      checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hitI
        [OF comp-nonempty trace-auth comp-auth component])
qed

definition checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
  :: ('f staged-adversary  $\Rightarrow$ 
    ((((((f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
      'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
      'f protocol-channel)  $\times$  unit list)  $\times$ 
      'f protocol-channel) option  $\Rightarrow$  bool)
  where
    checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
      A out  $\longleftrightarrow$ 
      (case out of
        None  $\Rightarrow$  False
      | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state)  $\Rightarrow$ 
        staged-composition-fri-roots data  $\neq$  []  $\wedge$ 
        ( $\exists$  i trace-openings composition-openings.
          partial-authenticated-table (staged-trace-root data)
            (scale * clength) trace-openings final-state  $\wedge$ 
          partial-authenticated-table (hd (staged-composition-fri-roots data))
            (scale * clength) composition-openings final-state  $\wedge$ 
          checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
            A
            ((replicate rounds []) [i := trace-openings])
            ((replicate rounds []) [i := composition-openings])
            i
            (Some (((((alpha-prefix, alpha-prefix-state), data),
              attacker-state), result), final-state))))))

lemma checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefixI:
  assumes comp-nonempty: staged-composition-fri-roots data  $\neq$  []
  and trace-auth:
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings final-state

```

```

and comp-auth:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings final-state
and component:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
    A
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i
    (Some ((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
shows
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
    A
    (Some ((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
proof –
  have witness:
    ∃ j trace-openings' composition-openings'.
      partial-authenticated-table (staged-trace-root data)
        (scale * clength) trace-openings' final-state ∧
      partial-authenticated-table (hd (staged-composition-fri-roots data))
        (scale * clength) composition-openings' final-state ∧
      checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
        A
        ((replicate rounds []) [j := trace-openings'])
        ((replicate rounds []) [j := composition-openings'])
        j
        (Some ((((alpha-prefix, alpha-prefix-state), data), attacker-state),
          result), final-state))
    by (intro exI[of - i] exI[of - trace-openings]
      exI[of - composition-openings] conjI)
      (use trace-auth comp-auth component in simp-all)
  show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix-def
  using comp-nonempty witness by simp
qed

```

```

lemma checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefixI-from-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and comp-nonempty: staged-composition-fri-roots data ≠ []
and i-bound: i < rounds
and builder:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)

```

```

and verifier:
  Some (result, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
and final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and comp-auth:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings final-state
and consistent:
  partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
shows
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
    A
    (Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state)))
proof –
from checked-staged-transcript-program-query-prefix-receive-final-lookup-support-with-alphas
  [OF wf controlled i-bound builder verifier final-lookup]
obtain prefix prefix-state raw-state where prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
and alphas-eq: sqp-alphas prefix = staged-alphas data
by blast
have component:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i
    (Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state)))
by (rule
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixI-from-prefix-receive
    [OF i-bound builder prefix-receive alphas-eq consistent])
show ?thesis
by (rule
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefixI

```

```

    [OF comp-nonempty trace-auth comp-auth component])
qed

lemma selected-query-chunks-match-verifier-lengths:
  assumes i-bound:  $i < \text{rounds}$ 
    and len-prefix-chunks:  $\text{length prefix-chunks} = i$ 
    and len-suffix-chunks:  $\text{length suffix-chunks} = \text{rounds} - \text{Suc } i$ 
    and chunk-shape:
      verifier-query-round-chunk idx trace-roots composition-roots chunk
    and prefix-rounds:
       $\bigwedge j. j < i \implies$ 
        verifier-query-round-chunk (prefix-query-idxs ! j)
        trace-roots composition-roots (prefix-chunks ! j)
    and suffix-rounds:
       $\bigwedge j. j < \text{rounds} - \text{Suc } i \implies$ 
        verifier-query-round-chunk (suffix-query-idxs ! j)
        trace-roots composition-roots (suffix-chunks ! j)
  shows
     $\text{length (prefix-chunks @ chunk \# suffix-chunks)} = \text{rounds}$ 
     $\bigwedge j. j < \text{rounds} \implies$ 
       $\text{length ((prefix-chunks @ chunk \# suffix-chunks) ! j)} =$ 
        verifier-query-round-transcript-length (query-idxs ! j)
        trace-roots composition-roots
  proof -
    show  $\text{length (prefix-chunks @ chunk \# suffix-chunks)} = \text{rounds}$ 
      using len-prefix-chunks len-suffix-chunks i-bound by simp
  next
    fix j
    assume j-bound:  $j < \text{rounds}$ 
    let ?chunks = prefix-chunks @ chunk # suffix-chunks
    show
       $\text{length (?chunks ! j)} =$ 
        verifier-query-round-transcript-length (query-idxs ! j)
        trace-roots composition-roots
  proof (cases  $j < i$ )
    case True
    have nth-eq:  $?chunks ! j = \text{prefix-chunks ! } j$ 
      using True len-prefix-chunks by (simp add: nth-append)
    have
       $\text{length (prefix-chunks ! } j) =$ 
        verifier-query-round-transcript-length
          (prefix-query-idxs ! j) trace-roots composition-roots
      by (rule verifier-query-round-chunk-length[OF prefix-rounds[OF True]])
    then show ?thesis
      using nth-eq
      by (simp add: verifier-query-round-transcript-length-index-irrelevant)
  next
    case False
    note not-lt = False

```

```

show ?thesis
proof (cases j = i)
  case True
  have nth-eq: ?chunks ! j = chunk
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length chunk =
      verifier-query-round-transcript-length idx trace-roots
        composition-roots
    by (rule verifier-query-round-chunk-length[OF chunk-shape])
  then show ?thesis
    using nth-eq
    by (simp add: verifier-query-round-transcript-length-index-irrelevant)
next
  case False
  note not-eq = False
  let ?k = j - Suc i
  have ge: Suc i ≤ j
    using not-lt not-eq by linarith
  have k-bound: ?k < rounds - Suc i
    using ge j-bound by linarith
  have nth-eq: ?chunks ! j = suffix-chunks ! ?k
    using ge len-prefix-chunks by (simp add: nth-append)
  have
    length (suffix-chunks ! ?k) =
      verifier-query-round-transcript-length
        (suffix-query-idxs ! ?k) trace-roots composition-roots
    by (rule verifier-query-round-chunk-length
      [OF suffix-rounds[OF k-bound]])
  then show ?thesis
    using nth-eq
    by (simp add: verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed

lemma selected-query-chunks-eq-staged:
  assumes builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and query-idxs-len: length query-idxs = rounds
  and chunks-len: length query-chunks = rounds
  and concat-eq:
    List.concat query-chunks @ trailing =
      List.concat (staged-query-chunks data)
  and parser-chunk-len:
     $\bigwedge j. j < rounds \implies$ 

```

```

length (query-chunks ! j) =
  verifier-query-round-transcript-length (query-idxs ! j)
    (staged-trace-fri-roots data)
    (staged-composition-fri-roots data)
shows query-chunks = staged-query-chunks data  $\wedge$  trailing = []
proof –
from checked-staged-transcript-program-query-chunks-match-verifier-lengths
  [OF builder]
obtain staged-query-idxs where staged-match:
  staged-query-chunks-match-verifier-lengths data staged-query-idxs
by blast
have staged-match-query-idxs:
  staged-query-chunks-match-verifier-lengths data query-idxs
by (rule staged-query-chunks-match-verifier-lengths-transfer
  [OF staged-match query-idxs-len])
show ?thesis
by (rule query-chunks-eq-staged-if-matching-lengths
  [OF chunks-len concat-eq parser-chunk-len
  staged-match-query-idxs])
qed

lemma selected-query-prefix-state-eq-staged:
assumes len-prefix-chunks: length prefix-chunks = i
and chunks-eq:
  prefix-chunks @ chunk # suffix-chunks = staged-query-chunks data
and query-state-hash: PState query-state = staged-query-start-hash data
shows
  state-after-query-chunks (PState query-state) prefix-chunks i =
  state-after-query-chunks
    (staged-query-start-hash data) (staged-query-chunks data) i
proof –
have
  state-after-query-chunks (staged-query-start-hash data)
    (prefix-chunks @ chunk # suffix-chunks) i =
  state-after-query-chunks (staged-query-start-hash data)
    prefix-chunks i
by (rule state-after-query-chunks-append-prefix[OF len-prefix-chunks])
then show ?thesis
using chunks-eq query-state-hash by simp
qed

definition checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit
  :: ('f staged-adversary  $\Rightarrow$ 
    (((('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
      'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
      'f protocol-channel)  $\times$  unit list)  $\times$ 
      'f protocol-channel) option  $\Rightarrow$  bool)
where
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit

```

```

  A out  $\longleftrightarrow$ 
(case out of
  None  $\Rightarrow$  False
| Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
  result), final-state)  $\Rightarrow$ 
  (let s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  in  $\exists$  i trace-openings composition-openings.
    (trace-openings, composition-openings)  $\in$ 
    query-header-supported-partial-opening-witnesses s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)  $\wedge$ 
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    (staged-alphas data) i
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))))))

```

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hitI*:

assumes *witness*:

```

(trace-openings, composition-openings)  $\in$ 
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)

```

and *component*:

```

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
(staged-alphas data) i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
  result), final-state))

```

shows

```

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit
A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),

```



```

      result), final-state))
proof –
  have ex:
    ∃ j trace-openings' composition-openings'.
      (trace-openings', composition-openings') ∈
        query-header-supported-partial-opening-witnesses
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))
            (staged-trace-root data)
            (staged-trace-fri-roots data)
            (staged-trace-final data)
            (staged-alphas data)
            (staged-degree data)
            (staged-composition-fri-roots data)
            (staged-composition-final data) ∧
        checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
          ((replicate rounds []) [j := trace-openings'])
          ((replicate rounds []) [j := composition-openings'])
          (staged-alphas data) j
          (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
            result), final-state))
  by (intro exI[of - i] exI[of - trace-openings]
      exI[of - composition-openings] conjI)
      (use witness component in simp-all)
  show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-def
    Let-def
  using ex by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-imp-compa
assumes hit:
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit
  A out
shows
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
  A out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-def
    by simp
  next
  case (Some packed)
  then obtain alpha-prefix alpha-prefix-state data attacker-state result
    final-state where out-eq:

```

```

    out =
      Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state)
  by (cases packed, auto split: prod.splits)
from hit[unfolded out-eq
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-def
Let-def]
obtain i trace-openings composition-openings where component:
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
(staged-alphas data) i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
  result), final-state))
by auto
show ?thesis
unfolding out-eq
by (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hitI
[OF component])
qed

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hitE:
assumes hit:
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit
A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
  result), final-state))
obtains i trace-openings composition-openings where
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
(staged-alphas data) i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
  result), final-state))
proof –
from hit[unfolded

```

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-def
Let-def]
obtain *i* *trace-openings* *composition-openings* **where** *witness*:
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
and *component*:
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
(staged-alphas data) i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))
by *auto*
show *?thesis*
by *(rule that[OF witness component])*
qed

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-le-compact*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit
A)
adversary-initial-state ≤
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
A)
adversary-initial-state
by *(rule wp-event-mono)*
(rule
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-imp-compact)

definition *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-pr*
 $:: ('f \text{ staged-adversary} \Rightarrow$
 $(((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool})$
where

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
 $A \text{ out} \longleftrightarrow$
(case out of
 None $\Rightarrow$ False
 | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state) $\Rightarrow$
 (let s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in $\exists i$ trace-openings composition-openings.
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses s
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\wedge$
 checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
 A
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])
 i
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))))))

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*

assumes *witness:*

(trace-openings, composition-openings) $\in$
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)

and *component:*

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))

shows

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix

```

A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
proof –
have ex:
  ∃ j trace-openings' composition-openings'.
  (trace-openings', composition-openings') ∈
    query-header-supported-partial-opening-witnesses
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)
        (staged-composition-fri-roots data)
        (staged-composition-final data) ∧
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
  ((replicate rounds []) [j := trace-openings'])
  ((replicate rounds []) [j := composition-openings'])
  j
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
by (intro exI[of - i] exI[of - trace-openings]
  exI[of - composition-openings] conjI)
  (use witness component in simp-all)
show ?thesis
unfolding
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-def
  Let-def
using ex by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
assumes hit:
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
  A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
obtains i trace-openings composition-openings where
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)

```

```

      (staged-composition-fri-roots data)
      (staged-composition-final data)
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
proof –
from hit[unfolded
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-def
Let-def]
obtain i trace-openings composition-openings where witness:
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
and component:
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
by auto
show ?thesis
by (rule that[OF witness component])
qed

```

```

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
assumes hit:
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
obtains i trace-openings composition-openings query-prefix query-prefix-state
raw raw-state where
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state

```

```

      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  i < rounds
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  Some (((query-prefix, query-prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i)
    (Some (((query-prefix, query-prefix-state), raw), raw-state))
proof –
from checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefixE
  [OF hit]
obtain i trace-openings composition-openings where witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
and component:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
  A
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  i
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
by blast
from checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixE
  [OF component]

```

obtain *query-prefix query-prefix-state raw raw-state* **where**
i-bound: $i < \text{rounds}$
and *builder:*
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive:*
Some (((query-prefix, query-prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *target-hit:*
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target-from-prefix
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i)
(Some (((query-prefix, query-prefix-state), raw), raw-state))
by *blast*
show *?thesis*
by *(rule that[OF witness i-bound builder prefix-receive target-hit])*
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-header-authenticat*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
Some (((((prefix, prefix-state), data), attacker-state), result),
final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)
and *hit:*
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
(Some (((((prefix, prefix-state), data), attacker-state), result),
final-state))
shows
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit
A
(Some (((((prefix, prefix-state), data), attacker-state), result),
final-state))
proof –
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
show *?thesis*


```

proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supported-openingsE
  [OF wf controlled support hit])
fix s trace-table i raw trace-openings composition-openings
assume s-def: s = ?s
  and i-bound: i < rounds
  and final-lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        Some raw
  and witness:
    (trace-openings, composition-openings) ∈
      query-header-supported-partial-opening-witnesses s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
  and consistent:
    partial-query-openings-consistent trace-openings composition-openings
      (staged-alphas data) (index (to-nat raw))
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
from checked-staged-transcript-program-query-prefix-receive-final-lookup-support
  [OF wf controlled i-bound builder verifier final-lookup]
obtain query-prefix query-prefix-state raw-state where prefix-receive:
  Some (((query-prefix, query-prefix-state), raw), raw-state) ∈

```

```

    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
  by blast
have component:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit
  A
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  (staged-alphas data) i
  (Some (((prefix, prefix-state), data), attacker-state), result),
  final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI-from-prefix-receive
    [OF i-bound builder prefix-receive consistent])
show ?thesis
  by (rule
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hitI
    [OF - component])
  (use witness s-def in simp)
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-header-authenticated
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    adversary-initial-state ≤
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit
      A)
    adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    out
show

```

```

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit
  A out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state
where out-eq:
  out =
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state)
  by (cases packed, auto split: prod.splits)
show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-header-authenticated-c
    [OF wf controlled])
    (use support hit out-eq in simp-all)
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-header-authenticat
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  (Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state))
shows
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
  A
  (Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state))
proof —

```

```

let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supported-openingsE
  [OF wf controlled support hit])
fix s trace-table i raw trace-openings composition-openings
assume s-def: s = ?s
  and i-bound: i < rounds
  and final-lookup:
fmlookup (HashMap final-state)
  (QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data)
      (staged-query-chunks data) i)) =
    Some raw
  and witness:
(trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  and consistent:
partial-query-openings-consistent trace-openings composition-openings
  (staged-alphas data) (index (to-nat raw))
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast

```

from *checked-staged-transcript-program-query-prefix-receive-final-lookup-support-with-alphas*
[OF wf controlled i-bound builder verifier final-lookup]
obtain *query-prefix query-prefix-state raw-state* **where** *prefix-receive:*
Some (((query-prefix, query-prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *alphas-eq: sqp-alphas query-prefix = staged-alphas data*
by *blast*
have *component:*
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i
(Some (((((prefix, prefix-state), data), attacker-state), result),
final-state))
by *(rule*
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixI-from-prefix-receive
[OF i-bound builder prefix-receive alphas-eq consistent])
show *?thesis*
by *(rule*
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefixI
[OF - component])
(use witness s-def in simp)
qed
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-header-authenticated*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state ≤
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
A)
adversary-initial-state
proof *(rule wp-event-mono-on-support)*
fix *out*
assume *support:*
out ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)

```

      adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  out
show
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
  A out
proof (cases out)
  case None
  then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state
  where out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state)
  by (cases packed, auto split: prod.splits)
show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-header-authenticated
    [OF wf controlled])
  (use support hit out-eq in simp-all)
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-header-auth
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and header-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
  (checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit
    A)
  adversary-initial-state  $\leq$  (Q::prob)
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  adversary-initial-state  $\leq$  Q
by (rule order.trans

```

[OF
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-header-authenticated-can
header-bound])
 (use wf controlled **in** simp-all)

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-header-auth*

assumes wf: *staged-budget-wellformed budgets*

and controlled: *staged-adversary-controlled budgets A*

and header-bound:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

(*checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*
A)

adversary-initial-state $\leq$ (*Q::prob*)

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

adversary-initial-state $\leq$ *Q*

by (rule *order.trans*

[OF

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-header-authenticated-can
header-bound])

(use wf controlled **in** simp-all)

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-header-authenticated*

assumes wf: *staged-budget-wellformed budgets*

and controlled: *staged-adversary-controlled budgets A*

and header-bound:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

(*checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit*
A)

adversary-initial-state $\leq$ (*Q::prob*)

and without-bound:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
adversary-initial-state $\leq$ (*W::prob*)

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift

adversary-initial-state $\leq$ *Q + W*

proof (rule

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-query)

show

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

adversary-initial-state $\leq Q$

by (*rule*

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-header-authen

[*OF wf controlled header-bound*])

show

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit

adversary-initial-state $\leq W$

by (*rule without-bound*)

qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-alpha-bad-trace-openings*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

Some (((((*prefix, prefix-state*), *data*), *attacker-state*), *result*),

final-state) $\in$

set-dist

(*execute*

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

adversary-initial-state)

and *hit:*

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

(*Some* (((((*prefix, prefix-state*), *data*), *attacker-state*), *result*),

final-state))

obtains *s trace-table witness-out query-idxs trace-openings*

witness-as witness-dg witness-composition-fri-roots witness-final

witness-rest **where**

s = verifier-state-from-adversary attacker-state

(*staged-proof-transcript data*)

witness-out $\in$ *set-dist* (*execute verify-monad s*)

accepted-with-partial-trace-openings s witness-out

(*staged-trace-root data*) *query-idxs trace-openings*

partial-trace-table-candidate trace-table trace-openings

verifier-header-transcript s

(*staged-trace-root data*)

(*staged-trace-fri-roots data*)

(*staged-trace-final data*)

witness-as witness-dg witness-composition-fri-roots witness-final

witness-rest

witness-as = staged-alphas data

staged-alphas data $\in$ *composition-trace-bad-alpha-space trace-table*

$\neg$ *staged-alphas data* $\in$


```

    alpha-prefix-union-bad-sets prefix-state
    composition-trace-bad-alpha-space (fst prefix)
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder where
  builder:
    Some (data, attacker-state) ∈
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  by blast
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supportedE
  [OF wf controlled support hit])
fix s trace-table
assume s-def:
  s = verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
and trace-candidate:
  trace-table ∈
    alpha-header-supported-partial-trace-table-candidates s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)

```

```

and trace-bad:
  staged-alphas data ∈ composition-trace-bad-alpha-space trace-table
and not-prefix:
  ¬ staged-alphas data ∈
    alpha-prefix-union-bad-sets prefix-state
    composition-trace-bad-alpha-space (fst prefix)
show ?thesis
proof (rule alpha-header-supported-partial-trace-table-candidatesE
  [OF trace-candidate])
fix witness-out query-idxs trace-openings witness-as witness-dg
  witness-composition-fri-roots witness-final witness-rest
assume witness-out:
  witness-out ∈ set-dist (execute verify-monad s)
and partial:
  accepted-with-partial-trace-openings s witness-out
  (staged-trace-root data) query-idxs trace-openings
and partial-candidate:
  partial-trace-table-candidate trace-table trace-openings
and header:
  verifier-header-transcript s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  witness-as witness-dg witness-composition-fri-roots
  witness-final witness-rest
have staged-header-s:
  verifier-header-transcript s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  (List.concat (staged-query-chunks data))
using staged-header by (simp add: s-def[symmetric])
have as-eq: witness-as = staged-alphas data
using verifier-header-transcript-unique[OF staged-header-s header]
by simp
show ?thesis
by (rule that[OF s-def witness-out partial partial-candidate
  header as-eq trace-bad not-prefix])

qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-table-witnessE:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A

```

and *support*:
Some (((((prefix, prefix-state), data), attacker-state), result),
final-state) $\in$
set-dist
(*execute*
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
adversary-initial-state)
and *hit*:
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
(*Some* (((((prefix, prefix-state), data), attacker-state), result),
final-state))
obtains *s* *trace-table* *composition-table* *query-idxs* *f* *bound-trace-openings*
bound-composition-openings **where**
s = *verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)
composition-bad-context *s* (*Some* (*result*, *final-state*)) *trace-table*
composition-table (*staged-alphas* *data*) *query-idxs* *f*
common-denominator-degree-bounds *f* (*staged-alphas* *data*)
random-combination-common-denominator-hides-violations *f*
(*staged-alphas* *data*)
partial-trace-table-candidate *trace-table* *bound-trace-openings*
partial-composition-table-candidate *composition-table*
bound-composition-openings
trace-table-low-degree *trace-table*
composition-table-low-degree *maxDegree* *composition-table*
all-queries-consistent *trace-table* *composition-table*
(*staged-alphas* *data*)
 $\neg$ *staged-alphas* *data* $\in$
alpha-prefix-union-bad-sets *prefix-state*
composition-trace-bad-alpha-space (*fst* *prefix*)
proof –
from *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supportedE*
[*OF* *wf* *controlled* *support* *hit*]
obtain *s* *trace-table* **where**
s-def:
s = *verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)
and *random-bad*:
composition-randomization-bad *s* (*Some* (*result*, *final-state*))
and *not-prefix*:
 $\neg$ *staged-alphas* *data* $\in$
alpha-prefix-union-bad-sets *prefix-state*
composition-trace-bad-alpha-space (*fst* *prefix*)
by *blast*
from *random-bad*
obtain *trace-table'* *composition-table* *as* *query-idxs* *f* **where** *bad-context*:
composition-bad-context *s* (*Some* (*result*, *final-state*)) *trace-table'*
composition-table *as* *query-idxs* *f*

```

and bounds: common-denominator-degree-bounds f as
and hides:
  random-combination-common-denominator-hides-violations f as
unfolding composition-randomization-bad-def by blast
have bound-as:
  accepted-with-bound-tables s (Some (result, final-state)) trace-table'
  composition-table as query-idxs
using bad-context unfolding composition-bad-context-def by blast
from bound-as obtain fr f-fri-roots f-final dg composition-fri-roots
  final rest where header-as:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
unfolding accepted-with-bound-tables-def by blast
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
  set-dist
  (execute (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
by (rule
  checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
  [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where builder:
  Some (data, attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-program A)
    adversary-initial-state)
and verifier:
  Some (result, final-state) ∈
  set-dist (execute verify-monad
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))))
by blast
have staged-header:
  verifier-header-transcript s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  (List.concat (staged-query-chunks data))
unfolding s-def
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
have as-eq: as = staged-alphas data
using verifier-header-transcript-unique[OF header-as staged-header]

```

```

    by simp
  have bad-context-staged:
    composition-bad-context s (Some (result, final-state)) trace-table'
      composition-table (staged-alphas data) query-idxs f
    using bad-context as-eq by simp
  have bounds-staged:
    common-denominator-degree-bounds f (staged-alphas data)
    using bounds as-eq by simp
  have hides-staged:
    random-combination-common-denominator-hides-violations f
      (staged-alphas data)
    using hides as-eq by simp
  have bound:
    accepted-with-bound-tables s (Some (result, final-state)) trace-table'
      composition-table (staged-alphas data) query-idxs
    using bad-context-staged unfolding composition-bad-context-def by blast
  from accepted-with-bound-tables-partial-opening-witnesses-Some[OF bound]
  obtain fr f-fri-roots f-final dg composition-fri-roots final rest
    bound-trace-openings bound-composition-openings where
    trace-candidate:
      partial-trace-table-candidate trace-table' bound-trace-openings
    and comp-candidate:
      partial-composition-table-candidate composition-table
      bound-composition-openings
    by blast
  have trace-low: trace-table-low-degree trace-table'
  proof -
    have deg-f: degree f < clength
      and trace-table-eq: trace-table' = map (poly f) eval-domain
      using bad-context-staged unfolding composition-bad-context-def by blast+
    show ?thesis
      unfolding trace-table-low-degree-def
      using deg-f trace-table-eq by blast
  qed
  have comp-low:
    composition-table-low-degree maxDegree composition-table
    using bad-context-staged unfolding composition-bad-context-def by blast
  have all-queries:
    all-queries-consistent trace-table' composition-table
      (staged-alphas data)
    using bad-context-staged unfolding composition-bad-context-def by blast
  show ?thesis
    by (rule that[OF s-def bad-context-staged bounds-staged hides-staged
      trace-candidate comp-candidate trace-low comp-low all-queries
      not-prefix])
  qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-aligned-openingsE:
  assumes wf: staged-budget-wellformed budgets

```

and *controlled*: staged-adversary-controlled budgets A
and *support*:
Some (((((prefix, prefix-state), data), attacker-state), result),
final-state) $\in$
set-dist
(*execute*
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
adversary-initial-state)
and *hit*:
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
(*Some* (((((prefix, prefix-state), data), attacker-state), result),
final-state))
obtains s *trace-table composition-table query-idxs f i raw*
header-trace-openings header-composition-openings
bound-trace-openings bound-composition-openings **where**
 $s = \text{verifier-state-from-adversary attacker-state}$
(*staged-proof-transcript data*)
composition-bad-context s (Some (result, final-state)) trace-table
composition-table (staged-alphas data) query-idxs f
 $i < \text{rounds}$
fmlookup (HashMap final-state)
(*QueryIndexChallenge i*
(*state-after-query-chunks*
(*staged-query-start-hash data*)
(*staged-query-chunks data*) i)) =
Some raw
index (to-nat raw) = query-idxs ! i
(*header-trace-openings, header-composition-openings*) $\in$
query-header-supported-partial-opening-witnesses s
(*staged-trace-root data*)
(*staged-trace-fri-roots data*)
(*staged-trace-final data*)
(*staged-alphas data*)
(*staged-degree data*)
(*staged-composition-fri-roots data*)
(*staged-composition-final data*)
partial-query-openings-consistent header-trace-openings
header-composition-openings (staged-alphas data) (query-idxs ! i)
partial-trace-table-candidate trace-table bound-trace-openings
partial-composition-table-candidate composition-table
bound-composition-openings
all-queries-consistent trace-table composition-table
(*staged-alphas data*)
 $\neg \text{staged-alphas data} \in$
alpha-prefix-union-bad-sets prefix-state
composition-trace-bad-alpha-space (fst prefix)
proof –
let $?s =$

```

    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  show ?thesis
  proof (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supported-openingsE
      [OF wf controlled support hit])
    fix s' trace-table' i raw header-trace-openings
      header-composition-openings
    assume s'-def: s' = ?s
    and i-bound: i < rounds
    and final-lookup:
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
          Some raw
    and header-witness:
      (header-trace-openings, header-composition-openings) ∈
        query-header-supported-partial-opening-witnesses s'
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)
        (staged-composition-fri-roots data)
        (staged-composition-final data)
    and header-consistent:
      partial-query-openings-consistent header-trace-openings
        header-composition-openings (staged-alphas data)
        (index (to-nat raw))
  show ?thesis
  proof (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-table-witnessE
      [OF wf controlled support hit])
    fix s trace-table composition-table query-idxs f bound-trace-openings
      bound-composition-openings
    assume s-def: s = ?s
    and bad-context:
      composition-bad-context s (Some (result, final-state)) trace-table
        composition-table (staged-alphas data) query-idxs f
    and trace-candidate:
      partial-trace-table-candidate trace-table bound-trace-openings
    and comp-candidate:
      partial-composition-table-candidate composition-table
        bound-composition-openings
    and all-queries:
      all-queries-consistent trace-table composition-table
        (staged-alphas data)

```

```

and not-prefix:
   $\neg$  staged-alphas data  $\in$ 
    alpha-prefix-union-bad-sets prefix-state
    composition-trace-bad-alpha-space (fst prefix)
have data-support:
  Some (((data, attacker-state), result), final-state)  $\in$ 
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
     adversary-initial-state)
by (rule
     checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
     [OF support])
have bound:
  accepted-with-bound-tables s (Some (result, final-state)) trace-table
  composition-table (staged-alphas data) query-idxs
using bad-context unfolding composition-bad-context-def by blast
have shape:
  accepted-transcript-shape s (Some (result, final-state))
  (staged-alphas data query-idxs)
using bound unfolding accepted-with-bound-tables-def
  accepted-with-tables-def by simp
have shape-s:
  accepted-transcript-shape ?s (Some (result, final-state))
  (staged-alphas data query-idxs)
using shape s-def by simp
from checked-staged-security-with-data-state-accepted-shape-query-chunks
  [OF wf controlled data-support shape-s]
obtain raw-idxs where
  len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and lookup:
   $\bigwedge j. j < rounds \implies$ 
    fmlookup (HashMap final-state)
    (QueryIndexChallenge j
     (state-after-query-chunks
      (staged-query-start-hash data)
      (staged-query-chunks data j))) =
      Some (raw-idxs ! j)
by blast
have raw-eq: raw = raw-idxs ! i
using final-lookup lookup[OF i-bound] by simp
have idx-eq: index (to-nat raw) = query-idxs ! i
using raw-eq query-idxs-eq len-raw i-bound by simp
have header-witness-s:
  (header-trace-openings, header-composition-openings)  $\in$ 
    query-header-supported-partial-opening-witnesses s
    (staged-trace-root data)
    (staged-trace-fri-roots data)

```



```

    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  using header-witness s'-def s-def by simp
have header-consistent-idx:
  partial-query-openings-consistent header-trace-openings
  header-composition-openings (staged-alphas data) (query-idxs ! i)
using header-consistent idx-eq by simp
show ?thesis
  by (rule that[OF s-def bad-context i-bound final-lookup idx-eq
    header-witness-s header-consistent-idx trace-candidate
    comp-candidate all-queries not-prefix])
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-authenticated-witnessE
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    (Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state))
  obtains s trace-table composition-table query-idxs f i raw
    header-trace-openings header-composition-openings
    bound-trace-openings bound-composition-openings
    witness-result witness-final-state witness-rest where
  s = verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  composition-bad-context s (Some (result, final-state)) trace-table
    composition-table (staged-alphas data) query-idxs f
  i < rounds
  index (to-nat raw) = query-idxs ! i
  Some (witness-result, witness-final-state) ∈
    set-dist (execute verify-monad s)
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) header-trace-openings witness-final-state
  partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) header-composition-openings witness-final-state

```

```

verifier-header-transcript s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
witness-rest
partial-query-openings-consistent header-trace-openings
  header-composition-openings (staged-alphas data) (query-idxs ! i)
partial-trace-table-candidate trace-table bound-trace-openings
partial-composition-table-candidate composition-table
  bound-composition-openings
all-queries-consistent trace-table composition-table
  (staged-alphas data)
¬ staged-alphas data ∈
  alpha-prefix-union-bad-sets prefix-state
  composition-trace-bad-alpha-space (fst prefix)
proof —
show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-aligned-openingsE
  [OF wf controlled support hit])
fix s trace-table composition-table query-idxs f i raw
  header-trace-openings header-composition-openings
  bound-trace-openings bound-composition-openings
assume s-def:
  s = verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
and bad-context:
  composition-bad-context s (Some (result, final-state))
  trace-table composition-table (staged-alphas data) query-idxs f
and i-bound: i < rounds
and idx-eq: index (to-nat raw) = query-idxs ! i
and header-witness:
  (header-trace-openings, header-composition-openings) ∈
  query-header-supported-partial-opening-witnesses s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
and header-consistent:
  partial-query-openings-consistent header-trace-openings
  header-composition-openings (staged-alphas data) (query-idxs ! i)
and trace-candidate:

```

```

    partial-trace-table-candidate trace-table bound-trace-openings
and comp-candidate:
    partial-composition-table-candidate composition-table
    bound-composition-openings
and all-queries:
    all-queries-consistent trace-table composition-table
    (staged-alphas data)
and not-prefix:
    ¬ staged-alphas data ∈
    alpha-prefix-union-bad-sets prefix-state
    composition-trace-bad-alpha-space (fst prefix)
show ?thesis
proof (rule query-header-supported-partial-opening-witnessesE
  [OF header-witness])
fix witness-result witness-final-state witness-rest
assume witness-out:
  Some (witness-result, witness-final-state) ∈
  set-dist (execute verify-monad s)
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
  (scale * clength) header-trace-openings witness-final-state
and comp-auth:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
  (scale * clength) header-composition-openings
  witness-final-state
and header:
  verifier-header-transcript s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  witness-rest
show ?thesis
  by (rule that[OF s-def bad-context i-bound idx-eq witness-out
    trace-auth comp-auth header header-consistent trace-candidate
    comp-candidate all-queries not-prefix])
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-current-openingsE:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state) ∈

```

```

    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
      (Some (((prefix, prefix-state), data), attacker-state), result),
      final-state))
obtains s trace-table composition-table query-idxs f i raw
  trace-openings composition-openings
  trace-openingss composition-openingss where
  s = verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  composition-bad-context s (Some (result, final-state)) trace-table
  composition-table (staged-alphas data) query-idxs f
  i < rounds
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some raw
  index (to-nat raw) = query-idxs ! i
  staged-composition-fri-roots data ≠ []
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
  partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings final-state
  partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (query-idxs ! i)
  partial-trace-table-candidate trace-table trace-openingss
  partial-composition-table-candidate composition-table
    composition-openingss
  trace-openings = trace-openingss ! i
  composition-openings = composition-openingss ! i
  all-queries-consistent trace-table composition-table
    (staged-alphas data)
  ¬ staged-alphas data ∈
    alpha-prefix-union-bad-sets prefix-state
    composition-trace-bad-alpha-space (fst prefix)
proof –
show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-aligned-openingsE
    [OF wf controlled support hit])
fix s trace-table composition-table query-idxs f i raw
  header-trace-openings header-composition-openings
  bound-trace-openings bound-composition-openings

```

```

assume s-def:
  s = verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
and bad-context:
  composition-bad-context s (Some (result, final-state))
    trace-table composition-table (staged-alphas data) query-idxs f
and i-bound: i < rounds
and final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some raw
and idx-eq: index (to-nat raw) = query-idxs ! i
and all-queries:
  all-queries-consistent trace-table composition-table
    (staged-alphas data)
and not-prefix:
   $\neg$  staged-alphas data  $\in$ 
    alpha-prefix-union-bad-sets prefix-state
    composition-trace-bad-alpha-space (fst prefix)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have data-support:
  Some (((data, attacker-state), result), final-state)  $\in$ 
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
by (rule
  checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state)  $\in$ 
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and verifier:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)
by blast
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)

```

```

    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
have bound:
  accepted-with-bound-tables s (Some (result, final-state))
  trace-table composition-table (staged-alphas data) query-idxs
  using bad-context unfolding composition-bad-context-def by blast
show ?thesis
proof (rule accepted-with-bound-tables-partial-opening-witnesses-Some
      [OF bound])
  fix fr f-fri-roots f-final dg composition-fri-roots final rest
  trace-openingsss composition-openingsss
  assume header:
    verifier-header-transcript s fr f-fri-roots f-final
    (staged-alphas data) dg composition-fri-roots final rest
  and comp-nonempty: composition-fri-roots  $\neq$  []
  and trace-partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openingsss
  and comp-partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots)
    query-idxs composition-openingsss
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openingsss
  and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openingsss
  have header-s:
    verifier-header-transcript ?s fr f-fri-roots f-final
    (staged-alphas data) dg composition-fri-roots final rest
  using header s-def by simp
  have header-eq:
    fr = staged-trace-root data  $\wedge$ 
    f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
    f-final = staged-trace-final data  $\wedge$ 
    dg = staged-degree data  $\wedge$ 
    composition-fri-roots = staged-composition-fri-roots data  $\wedge$ 
    final = staged-composition-final data  $\wedge$ 
    rest = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF staged-header header-s]
  by simp
  have query-len: length query-idxs = rounds
  by (rule accepted-with-bound-tables-shapes(4)[OF bound])
  have idx-mem: query-idxs ! i  $\in$  set query-idxs

```

```

    by (rule nth-mem) (use i-bound query-len in simp)
  have idx-sample: query-idxs ! i ∈ query-sample-space
    by (rule accepted-with-bound-tables-query-sample-space
        [OF bound idx-mem])
  have consistent-round:
    partial-query-round-consistent trace-openingsss
      composition-openingsss (staged-alphas data) i (query-idxs ! i)
    by (rule partial-query-round-consistent-from-all-queries-consistent
        [OF trace-partial comp-partial trace-candidate comp-candidate
            i-bound idx-sample all-queries])
  have consistent:
    partial-query-openings-consistent (trace-openingsss ! i)
      (composition-openingsss ! i) (staged-alphas data) (query-idxs ! i)
    using consistent-round
    unfolding partial-query-round-consistent-def
      partial-query-openings-consistent-def
    by simp
  have trace-auth:
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) (trace-openingsss ! i) final-state
    using trace-partial header-eq i-bound
    unfolding accepted-with-partial-trace-openings-def by blast
  have comp-auth:
    partial-authenticated-table
      (hd (staged-composition-fri-roots data)) (scale * clength)
      (composition-openingsss ! i) final-state
    using comp-partial header-eq i-bound
    unfolding accepted-with-partial-composition-openings-def by simp
  have comp-nonempty-staged:
    staged-composition-fri-roots data ≠ []
    using comp-nonempty header-eq by simp
  show ?thesis
    by (rule that[OF s-def bad-context i-bound final-lookup idx-eq
        comp-nonempty-staged trace-auth comp-auth consistent
        trace-candidate comp-candidate refl refl all-queries not-prefix])
qed
qed
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-authenticated-candidate*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
Some (((((prefix, prefix-state), data), attacker-state), result),
final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)

```

      adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    (Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state))
shows
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
    A
    (Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state))
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-current-openingsE
  [OF wf controlled support hit])
  fix s trace-table composition-table query-idxs f i raw trace-openings
    composition-openings trace-openingsss composition-openingsss
  assume i-bound: i < rounds
  and final-lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some raw
  and idx-eq: index (to-nat raw) = query-idxs ! i
  and comp-nonempty: staged-composition-fri-roots data ≠ []

```



```

and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and comp-auth:
  partial-authenticated-table
    (hd (staged-composition-fri-roots data)) (scale * clength)
    composition-openings final-state
and consistent:
  partial-query-openings-consistent trace-openings
    composition-openings (staged-alphas data) (query-idxs ! i)
have consistent-raw:
  partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
  using consistent idx-eq by simp
show ?thesis
  by (rule
    checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hitI-from-current-openings
    [OF wf controlled comp-nonempty i-bound builder verifier
      final-lookup trace-auth comp-auth consistent-raw])
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-authenticated-candidate
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    adversary-initial-state  $\leq$ 
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
      A)
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out  $\in$ 
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    out
show
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit

```

```

    A out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state data attacker-state result final-state
  where out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state)
  by (cases packed, auto split: prod.splits)
show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-authenticated-candidate-
    [OF wf controlled])
    (use support hit out-eq in simp-all)
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-authenticated-candidate-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and authenticated-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
      A)
    adversary-initial-state  $\leq$  (Q::prob)
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    adversary-initial-state  $\leq$  Q
by (rule order.trans
  [OF
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-authenticated-candidate-
    authenticated-bound])
    (use wf controlled in simp-all)

```

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-authenticated-candidate-

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    (Some (((((prefix, prefix-state), data), attacker-state), result),
           final-state))
shows
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
    A
    (Some (((((prefix, prefix-state), data), attacker-state), result),
           final-state))
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF data-support]
  obtain builder verifier where
    builder:
      Some (data, attacker-state) ∈
        set-dist
          (execute (checked-staged-transcript-program A)
            adversary-initial-state)
    and verifier:
      Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
  show ?thesis
  proof (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-current-openingsE
    [OF wf controlled support hit])
  fix s trace-table composition-table query-idxs f i raw trace-openings
    composition-openings trace-openingsss composition-openingsss
  assume i-bound: i < rounds

```

```

and final-lookup:
  fmllookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
      Some raw
and idx-eq: index (to-nat raw) = query-idxs ! i
and comp-nonempty: staged-composition-fri-roots data ≠ []
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and comp-auth:
  partial-authenticated-table
    (hd (staged-composition-fri-roots data)) (scale * clength)
    composition-openings final-state
and consistent:
  partial-query-openings-consistent trace-openings
    composition-openings (staged-alphas data) (query-idxs ! i)
have consistent-raw:
  partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
  using consistent idx-eq by simp
show ?thesis
by (rule
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefixI-from-cu
  [OF wf controlled comp-nonempty i-bound builder verifier
    final-lookup trace-auth comp-auth consistent-raw])
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-authenticated-candid
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
      A)
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out ∈
    set-dist

```

```

      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
    and hit:
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    out
  show
    checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
    A out
  proof (cases out)
    case None
    then show ?thesis
      using hit
      unfolding
        checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
        checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
        checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      by simp
  next
    case (Some packed)
    then obtain prefix prefix-state data attacker-state result final-state
      where out-eq:
        out =
          Some (((((prefix, prefix-state), data), attacker-state), result),
            final-state)
      by (cases packed, auto split: prod.splits)
    show ?thesis
      unfolding out-eq
      by (rule
        checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-authenticated-candid
        [OF wf controlled])
      (use support hit out-eq in simp-all)
  qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-authenticat
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and authenticated-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
        A)
      adversary-initial-state  $\leq$  (Q::prob)
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

```

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq Q$
by (*rule order.trans*
 $[OF$
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-authenticated-candidate-
authenticated-bound])
(use wf controlled in simp-all)

lemma *checked-staged-security-with-actual-alpha-prefix-aligned-transcript-query-bad-imp-candidate-opening-hit-from-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)
and *bad:*
checked-staged-security-with-actual-alpha-prefix-verifier-event
query-bad-with-aligned-transcript-partial-candidates
(Some (((((alpha-prefix, alpha-prefix-state), data),
attacker-state), result), final-state))
obtains *i trace-openings composition-openings where*
i < rounds
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A trace-openings composition-openings i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
proof –
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
have *data-support:*
Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute (checked-staged-security-experiment-with-data-state A)
adversary-initial-state)
by (*rule*
checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
 $[OF support]$)
from *checked-staged-security-experiment-with-data-state-outcomeE*
 $[OF data-support]$
obtain *builder verifier where*
builder:
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)

```

      adversary-initial-state)
and verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
from bad obtain fr f-fri-roots f-final as dg composition-fri-roots
  final query-idxs trace-openings composition-openings trace-table
  composition-table where partial:
    accepted-with-partial-initial-openings-aligned-transcript-consistent
    ?s (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  query-bad-with-aligned-transcript-partial-candidates-def
by auto
have shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as
  query-idxs
by (rule
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
  [OF partial])
from checked-staged-security-with-data-state-accepted-shape-query-chunks
  [OF wf controlled data-support shape]
obtain raw-idxs where
  as-eq: as = staged-alphas data
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup:
  ∀ i < rounds.
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
by blast
have zero-bound: 0 < rounds
by (rule rounds-positive)
let ?raw = raw-idxs ! 0
have lookup0:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge 0
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) 0)) =
    Some ?raw
using lookup zero-bound by simp
from checked-staged-transcript-program-query-prefix-receive-final-lookup-support-with-alphas
  [OF wf controlled zero-bound builder verifier lookup0]

```

```

obtain prefix prefix-state raw-state where prefix-receive:
  Some (((prefix, prefix-state), ?raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A 0)
        adversary-initial-state)
  and prefix-alphas: sqp-alphas prefix = staged-alphas data
  by blast
have idx-eq: query-idxs ! 0 = index (to-nat ?raw)
  using query-idxs-eq len-raw zero-bound by simp
have alphas-eq: as = sqp-alphas prefix
  using as-eq prefix-alphas by simp
have hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
    A trace-openings composition-openings 0
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state)))
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-candidate-opening-hit-from-prefix
      [OF partial zero-bound idx-eq alphas-eq builder prefix-receive])
show ?thesis
  by (rule that[OF zero-bound hit])
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-authenticated-and-wf
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and authenticated-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
      A)
    adversary-initial-state ≤ (Q::prob)
and without-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
      adversary-initial-state ≤ (W::prob)
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
      adversary-initial-state ≤ Q + W
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-query)
show
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

```


checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq Q$
by (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-authenticated
[OF wf controlled authenticated-bound])
show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
adversary-initial-state $\leq W$
by (rule *without-bound*)
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-authenticated-from-p*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *authenticated-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
A)
adversary-initial-state $\leq (Q::prob)$
and *without-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
adversary-initial-state $\leq (W::prob)$
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
adversary-initial-state $\leq Q + W$
proof (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-query)
show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq Q$
by (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-authenticated-
[OF wf controlled authenticated-bound])
show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
adversary-initial-state $\leq W$

by (rule without-bound)
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-candidate-opening*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

*Some (((((prefix, prefix-state), data), attacker-state), result),
final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)*

and *hit:*

*checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
(Some (((((prefix, prefix-state), data), attacker-state), result),
final-state))*

shows

*checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
A
(Some (((((prefix, prefix-state), data), attacker-state), result),
final-state))*

proof —

let *?s =*

*verifier-state-from-adversary attacker-state
(staged-proof-transcript data)*

show *?thesis*

proof (rule

*checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supported-openingsE
[OF wf controlled support hit])*

fix *s trace-table i raw trace-openings composition-openings*

assume *i-bound: i < rounds*

and *final-lookup:*

*fmlookup (HashMap final-state)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data)
(staged-query-chunks data) i)) =
Some raw*

and *consistent:*

*partial-query-openings-consistent trace-openings composition-openings
(staged-alphas data) (index (to-nat raw))*

have *data-support:*

*Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute (checked-staged-security-experiment-with-data-state A)
adversary-initial-state)*

by (rule

```

    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
    [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
from checked-staged-transcript-program-query-prefix-receive-final-lookup-support
    [OF wf controlled i-bound builder verifier final-lookup]
obtain query-prefix query-prefix-state raw-state where prefix-receive:
  Some (((query-prefix, query-prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  by blast
have component:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit
  A
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  (staged-alphas data) i
  (Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI-from-prefix-receive
    [OF i-bound builder prefix-receive consistent])
show ?thesis
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hitI
    [OF component])
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-candidate-opening-q
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

```

```

    (checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
     A)
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
    out ∈
    set-dist
    (execute
     (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
     adversary-initial-state)
and hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    out
show
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
    A out
proof (cases out)
case None
then show ?thesis
    using hit
    unfolding
        checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
        checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
        checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state
where out-eq:
    out =
        Some (((((prefix, prefix-state), data), attacker-state), result),
              final-state)
    by (cases packed, auto split: prod.splits)
show ?thesis
unfolding out-eq
by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-candidate-opening-qu
    [OF wf controlled])
    (use support hit out-eq in simp-all)
qed
qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-authenticated-query-and-budgets:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and authenticated-bound:
    wp-event

```

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
A)
adversary-initial-state $\leq Q$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event composition-bad)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
Q +
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)

proof –

have *without-bound*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
adversary-initial-state ≤ 0

by (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-bound
[OF wf controlled])

have *drift-bound*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
adversary-initial-state $\leq Q + 0$

by (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-authenticated-and-without-bound
[OF wf controlled authenticated-bound without-bound])

have *comp-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event composition-bad)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +

```

    (Q + 0) +
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-composition-candidate-drift-and-budgets
    [OF wf controlled drift-bound])
  then show ?thesis
  by simp
qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-header-authenticated-query-from-prefix-a
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and header-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
       A)
    (checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
     A)
    adversary-initial-state ≤ Q
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event composition-bad)
      adversary-initial-state ≤
        composition-error-bound +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
        hash-collision-budget-value 0
        (staged-alpha-search-queries budgets 0) +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
        Q +
        hash-collision-budget-value 0
        (staged-attacker-query-budget budgets +
         staged-challenge-query-budget + verifier-hash-query-budget)
  proof -
  have without-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
       A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
    adversary-initial-state ≤ 0
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-bound
    [OF wf controlled])
  have drift-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

```

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
adversary-initial-state $\leq Q + 0$

proof (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-query)
show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq Q$
by (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-header-authen
[OF wf controlled header-bound])
show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
adversary-initial-state ≤ 0
by (rule *without-bound*)
qed

have *comp-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event composition-bad)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(Q + 0) +
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
checked-staged-security-with-data-state-composition-bad-bound-from-composition-candidate-drift-and-budgets
[OF wf controlled drift-bound])
then show *?thesis*
by *simp*
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-authenticated-query-from-prefix-and-bud*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *authenticated-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

$A)$
(checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
 $A)$
 $\text{adversary-initial-state} \leq Q$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event composition-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $Q +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget})$
proof –
have *without-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit}$
 $\text{adversary-initial-state} \leq 0$
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-bound}$
 $[OF \text{ wf controlled}])$
have *drift-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift}$
 $\text{adversary-initial-state} \leq Q + 0$
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-authenticated-from-prefix}$
 $[OF \text{ wf controlled authenticated-bound without-bound}])$
have *comp-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event composition-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(Q + 0) +$


```

    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-composition-candidate-drift-and-budgets
    [OF wf controlled drift-bound])
  then show ?thesis
  by simp
qed

end

end

```

```

theory Staged-Security-Experiment-Composition-Query-Augmented
imports Staged-Security-Experiment-Composition-Query-Bridge
begin

```

Augmented query-prefix events for the composition query side.

These events live in the query-prefix augmented experiment, so the sampled query challenge is part of the event output. This is the correct setting for applying fixed-witness query-prefix bounds. The fixed-witness event below deliberately does not existentially choose openings after the query challenge; the remaining public-proof work is to derive such fixed witnesses from pre-query Merkle/candidate evidence or charge failures to existing side events.

```

context soundness
begin

```

lemma *checked-staged-security-with-data-state-query-prefix-continuationE:*

assumes *i-bound: $i < \text{rounds}$*

and support:

*Some (((data, attacker-state), result), final-state) ∈
 set-dist
 (execute (checked-staged-security-experiment-with-data-state A)
 adversary-initial-state)*

obtains *prefix prefix-state raw raw-state where*

*Some (((prefix, prefix-state), raw), raw-state) ∈
 set-dist*

*(execute
 (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state)*

*Some (((data, attacker-state), result), final-state) ∈
 set-dist*

*(execute
 (checked-staged-after-query-prefix-receive-with-verifier A i
 ((prefix, prefix-state), raw))
 raw-state)*

proof –
have *decomp*:
checked-staged-security-experiment-with-data-state A =
checked-staged-query-prefix-receive-with-state A i $\gg$
checked-staged-after-query-prefix-receive-with-verifier A i
by (rule
checked-staged-security-experiment-with-data-state-query-prefix-receive-decomp
[OF i-bound])
from *support[unfolded decomp]*
obtain *x raw-state* **where** *prefix-receive*:
Some (x, raw-state) ∈
set-dist
(execute
(checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *continuation*:
Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute
(checked-staged-after-query-prefix-receive-with-verifier A i x)
raw-state)
by (*auto elim! set-dist-bindE*)
obtain *prefix-pack raw* **where** *x-eq: x = (prefix-pack, raw)*
by (*cases x simp*)
obtain *prefix prefix-state* **where** *prefix-pack-eq*:
prefix-pack = (prefix, prefix-state)
by (*cases prefix-pack simp*)
show *?thesis*
by (rule *that[of prefix prefix-state raw raw-state]*)
(use prefix-receive continuation x-eq prefix-pack-eq in simp-all)
qed

definition *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit*
where

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i out $\longleftrightarrow$
(case out of
None $\Rightarrow$ False
| Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) $\Rightarrow$
staged-composition-fri-roots data $\neq$ [] $\wedge$
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)

(staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\wedge$
 checked-staged-security-with-actual-query-prefix-candidate-opening-hit
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])
 i out)

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-imp-actual:*

assumes *hit:*

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
 trace-openings composition-openings i out

shows

checked-staged-security-with-actual-query-prefix-candidate-opening-hit
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])
 i out

using *hit*

unfolding

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-def
by (cases out) (auto split: prod.splits)

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitE:*

assumes *hit:*

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
 trace-openings composition-openings i
 (Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state))

obtains

staged-composition-fri-roots data $\neq$ []
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
 index (to-nat raw) $\in$
 staged-query-prefix-candidate-opening-query-target-from-prefix
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])
 i prefix prefix-state

proof —

have *comp-nonempty:*

staged-composition-fri-roots data $\neq$ []

and *witness*:
 (trace-openings, composition-openings) ∈
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
and *actual*:
 checked-staged-security-with-actual-query-prefix-candidate-opening-hit
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])
 i
 (Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state))
using *hit*
unfolding
 checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-def
by *simp-all*
from checked-staged-security-with-actual-query-prefix-candidate-opening-hitE
 [OF actual]
have *target*:
 index (to-nat raw) ∈
 staged-query-prefix-candidate-opening-query-target-from-prefix
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])
 i prefix prefix-state
 .
show ?thesis
by (rule that[OF comp-nonempty witness target])
qed

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-prefix-facts*:

assumes *support*:

Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) ∈
 set-dist
 (execute
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 adversary-initial-state)

and *hit*:

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
 trace-openings composition-openings i
 (Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state))

shows

staged-alphas data = sqp-alphas prefix
staged-trace-root data = sqp-trace-root prefix
staged-composition-fri-roots data = sqp-composition-fri-roots prefix
index (to-nat raw) ∈
staged-query-prefix-candidate-opening-query-target-from-prefix
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i prefix prefix-state

proof –

show *alphas: staged-alphas data = sqp-alphas prefix*
by (rule *checked-staged-security-with-query-prefix-data-state-alphas-eq*)
(rule support)
from *hit* **show** *target:*
index (to-nat raw) ∈
staged-query-prefix-candidate-opening-query-target-from-prefix
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i prefix prefix-state
by (rule
checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitE)
from support **show** *staged-trace-root data = sqp-trace-root prefix*
unfolding *checked-staged-security-experiment-with-query-prefix-data-state-def*
checked-staged-after-query-prefix-receive-with-verifier-def
checked-staged-after-query-prefix-receive-def
by (auto elim!: *set-dist-bindE* split: *prod.splits*)
from support **show**
staged-composition-fri-roots data = sqp-composition-fri-roots prefix
unfolding *checked-staged-security-experiment-with-query-prefix-data-state-def*
checked-staged-after-query-prefix-receive-with-verifier-def
checked-staged-after-query-prefix-receive-def
by (auto elim!: *set-dist-bindE* split: *prod.splits*)

qed

The next wrapper lets downstream composition proofs conjoin their own side-condition with the fixed authenticated-opening query-prefix hit, without changing the probability bound. This is deliberately fixed-witness: the trace and composition openings are chosen before the query challenge is sampled.

definition *checked-staged-security-with-query-prefix-fixed-header-component*

where

checked-staged-security-with-query-prefix-fixed-header-component
 $E \text{ trace-openings } \text{composition-openings } i \text{ out} \longleftrightarrow$
 $E \text{ out} \wedge$
checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i out

definition *checked-staged-security-with-query-prefix-fixed-current-authenticated-component*

where

checked-staged-security-with-query-prefix-fixed-current-authenticated-component

$E \text{ trace-openings composition-openings } i \text{ out } \longleftrightarrow$
checked-staged-security-with-query-prefix-fixed-header-component
 $E \text{ trace-openings composition-openings } i \text{ out } \wedge$
checked-staged-security-with-query-prefix-authenticated-opening-hit
trace-openings composition-openings } i \text{ out}

definition *checked-staged-security-with-query-prefix-header-witness-not-current*
where

checked-staged-security-with-query-prefix-header-witness-not-current
 $E \text{ trace-openings composition-openings } i \text{ out } \longleftrightarrow$
checked-staged-security-with-query-prefix-fixed-header-component
 $E \text{ trace-openings composition-openings } i \text{ out } \wedge$
 $\neg \text{checked-staged-security-with-query-prefix-authenticated-opening-hit}$
trace-openings composition-openings } i \text{ out}

definition *checked-staged-security-with-query-prefix-header-witness-path-output-hit*
where

checked-staged-security-with-query-prefix-header-witness-path-output-hit
 $E \text{ trace-openings composition-openings } i \text{ out } \longleftrightarrow$
(case out of
 $\text{None} \Rightarrow \text{False}$
| $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}) \Rightarrow$
 $E \text{ out } \wedge$
staged-composition-fri-roots data $\neq [] \wedge$
 $(\exists \text{witness-result witness-state rest.}$
 $\text{let } s = \text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}) \text{ in}$
 $\text{Some } (\text{witness-result}, \text{witness-state}) \in$
 $\text{set-dist } (\text{execute verify-monad } s) \wedge$
verifier-header-transcript s
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $(\text{staged-alphas data})$
 $(\text{staged-degree data})$
 $(\text{staged-composition-fri-roots data})$
 $(\text{staged-composition-final data})$
 $\text{rest } \wedge$
 $((\exists \text{opn} \in \text{set trace-openings.}$
 $\text{hash-map-new-output-hit}$
 $(\text{merkle-path-target-roots witness-state}$
 $(\text{staged-trace-root data})$
 $(\text{scale} * \text{clength})$
 $(\text{opening-index opn})$
 $(\text{opening-value opn})$
 $(\text{opening-path opn}))$
 $s \text{ witness-state}) \vee$
 $(\exists \text{opn} \in \text{set composition-openings.}$

```

hash-map-new-output-hit
  (merkle-path-target-roots witness-state
    (hd (staged-composition-fri-roots data))
    (scale * clength)
    (opening-index opn)
    (opening-value opn)
    (opening-path opn))
  s witness-state))))

```

definition *checked-staged-security-with-query-prefix-header-witness-transcript-hit*
where

```

checked-staged-security-with-query-prefix-header-witness-transcript-hit
  E out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)  $\Rightarrow$ 
    E out  $\wedge$ 
    ( $\exists$  witness-result witness-state.
      let s = verifier-state-from-adversary attacker-state
        (staged-proof-transcript data) in
      Some (witness-result, witness-state)  $\in$ 
        set-dist (execute verify-monad s)  $\wedge$ 
      (hash-map-output-values s  $\cap$ 
        set (staged-proof-transcript data)  $\neq$   $\{\}$ )  $\vee$ 
      hash-map-new-output-hit (set (staged-proof-transcript data))
        s witness-state))))

```

definition *checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit*
where

```

checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
  E out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)  $\Rightarrow$ 
    E out  $\wedge$ 
    hash-map-output-values
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\cap$ 
      set (staged-proof-transcript data)  $\neq$   $\{\}$ )

```

definition *checked-staged-security-with-query-prefix-header-witness-transcript-new-hit*
where

```

checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
  E out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (((((prefix, prefix-state), raw), raw-state),

```

$((data, attacker-state), result)), final-state) \Rightarrow$
 $E \text{ out} \wedge$
 $(\exists witness\text{-}result\ witness\text{-}state.$
 $\quad let\ s = verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $\quad (staged\text{-}proof\text{-}transcript\ data)\ in$
 $\quad Some\ (witness\text{-}result, witness\text{-}state) \in$
 $\quad set\text{-}dist\ (execute\ verify\text{-}monad\ s) \wedge$
 $\quad hash\text{-}map\text{-}new\text{-}output\text{-}hit\ (set\ (staged\text{-}proof\text{-}transcript\ data))$
 $\quad s\ witness\text{-}state))$

lemma *checked-staged-security-with-query-prefix-header-witness-transcript-hit-imp-pre-or-new:*

assumes *hit:*

checked-staged-security-with-query-prefix-header-witness-transcript-hit

E out

shows

checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit

E out $\vee$

checked-staged-security-with-query-prefix-header-witness-transcript-new-hit

E out

using *hit*

unfolding

checked-staged-security-with-query-prefix-header-witness-transcript-hit-def

checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit-def

checked-staged-security-with-query-prefix-header-witness-transcript-new-hit-def

by (*cases out*) (*fastforce split: prod.splits*) $+$

lemma *checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit-imp-data-state-pre-hit:*

assumes *hit:*

checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit

E

$(Some\ (((((prefix, prefix\text{-}state), raw), raw\text{-}state),$

$((data, attacker\text{-}state), result)), final\text{-}state))$

shows

staged-security-with-data-state-transcript-pre-hit

$(Some\ (((data, attacker\text{-}state), result), final\text{-}state))$

using *hit*

unfolding

checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit-def

staged-security-with-data-state-transcript-pre-hit-def

by *simp*

lemma *checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit-bound-from-data-state-pre-hit:*

assumes *i-bound: $i < rounds$*

and *pre-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-transcript-pre-hit

adversary-initial-state $\leq P$

shows

wp-event


```

    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
      E)
    adversary-initial-state ≤ P
proof –
have projection:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (λout. case out of
      None ⇒ staged-security-with-data-state-transcript-pre-hit None
    | Some (packed, t) ⇒
      staged-security-with-data-state-transcript-pre-hit
        (Some (snd packed, t)))
    adversary-initial-state
by (rule
  checked-staged-security-experiment-with-query-prefix-data-state-projection-event
  [OF i-bound])
have event-le:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
      E)
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (λout. case out of
      None ⇒ staged-security-with-data-state-transcript-pre-hit None
    | Some (packed, t) ⇒
      staged-security-with-data-state-transcript-pre-hit
        (Some (snd packed, t)))
    adversary-initial-state
proof (rule wp-event-mono)
fix out
assume hit:
  checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
    E out
show
  (case out of
    None ⇒ staged-security-with-data-state-transcript-pre-hit None
  | Some (packed, t) ⇒
    staged-security-with-data-state-transcript-pre-hit
      (Some (snd packed, t)))
proof (cases out)
case None
then show ?thesis
  using hit

```

```

    unfolding
      checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit-def
    by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
  final-state where out-eq:
  out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
show ?thesis
  using
    checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit-imp-data-state-pre-hit
    [OF hit[unfolded out-eq]]
  unfolding out-eq by simp
qed
qed
show ?thesis
  by (rule order-trans[OF event-le])
  (use pre-bound projection in simp)
qed

```

lemma *checked-staged-security-with-query-prefix-header-witness-path-output-hitE*:

assumes *hit*:

checked-staged-security-with-query-prefix-header-witness-path-output-hit

E trace-openings composition-openings i

(Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state))

and *that*:

$\bigwedge$ *witness-result witness-state rest.*

E (Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state)) $\implies$

staged-composition-fri-roots data $\neq \square \implies$

Some (*witness-result*, *witness-state*) $\in$

set-dist

(*execute verify-monad*

(*verifier-state-from-adversary attacker-state*

(*staged-proof-transcript data*))) $\implies$

verifier-header-transcript

(*verifier-state-from-adversary attacker-state*

(*staged-proof-transcript data*))

(*staged-trace-root data*)

(*staged-trace-fri-roots data*)

(*staged-trace-final data*)

(*staged-alphas data*)

(*staged-degree data*)

(*staged-composition-fri-roots data*)

(*staged-composition-final data*))

```

    rest  $\implies$ 
    (( $\exists$  opn  $\in$  set trace-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots witness-state
          (staged-trace-root data)
          (scale * clength)
          (opening-index opn)
          (opening-value opn)
          (opening-path opn))
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
        witness-state)  $\vee$ 
      ( $\exists$  opn  $\in$  set composition-openings.
        hash-map-new-output-hit
          (merkle-path-target-roots witness-state
            (hd (staged-composition-fri-roots data))
            (scale * clength)
            (opening-index opn)
            (opening-value opn)
            (opening-path opn))
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))
          witness-state))  $\implies$ 
    thesis
  shows thesis
proof -
have E:
  E (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
and nonempty:
  staged-composition-fri-roots data  $\neq$  []
using hit
unfolding checked-staged-security-with-query-prefix-header-witness-path-output-hit-def
by simp-all
from hit obtain witness-result witness-state rest where
witness:
  Some (witness-result, witness-state)  $\in$ 
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
and header:
  verifier-header-transcript
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)

```

```

    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    rest
and path-hit:
  (( $\exists$  opn  $\in$  set trace-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state
        (staged-trace-root data)
        (scale * clength)
        (opening-index opn)
        (opening-value opn)
        (opening-path opn))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      witness-state)  $\vee$ 
    ( $\exists$  opn  $\in$  set composition-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots witness-state
          (hd (staged-composition-fri-roots data))
          (scale * clength)
          (opening-index opn)
          (opening-value opn)
          (opening-path opn))
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
        witness-state))
  unfolding checked-staged-security-with-query-prefix-header-witness-path-output-hit-def
  by (auto simp: Let-def; blast)
show ?thesis
  by (rule that[OF E nonempty witness header path-hit])
qed

lemma checked-staged-security-with-query-prefix-header-witness-path-output-imp-transcript-hit:
  assumes hit:
    checked-staged-security-with-query-prefix-header-witness-path-output-hit
    E trace-openings composition-openings i out
  shows
    checked-staged-security-with-query-prefix-header-witness-transcript-hit
    E out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding checked-staged-security-with-query-prefix-header-witness-path-output-hit-def
      checked-staged-security-with-query-prefix-header-witness-transcript-hit-def
    by simp
next
  case (Some packed)

```

```

then obtain prefix prefix-state raw raw-state data attacker-state result
final-state where out-eq:
  out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from hit[unfolded out-eq]
obtain witness-result witness-state rest where
  E:
    E (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  and witness:
    Some (witness-result, witness-state) ∈
      set-dist (execute verify-monad ?s)
  by (rule
    checked-staged-security-with-query-prefix-header-witness-path-output-hitE)
have tr-eq: PTranscript ?s = staged-proof-transcript data
  by simp
have transcript-hit:
  hash-map-output-values ?s ∩ set (staged-proof-transcript data)
    ≠ {} ∨
  hash-map-new-output-hit (set (staged-proof-transcript data))
    ?s witness-state
  by (rule verify-monad-trace-root-target-preexisting-or-new
    [OF witness tr-eq])
show ?thesis
  unfolding out-eq
    checked-staged-security-with-query-prefix-header-witness-transcript-hit-def
  using E witness transcript-hit by (auto simp: Let-def)
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-imp-header:
assumes hit:
  checked-staged-security-with-query-prefix-fixed-header-component
  E trace-openings composition-openings i out
shows
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
  trace-openings composition-openings i out
using hit
unfolding checked-staged-security-with-query-prefix-fixed-header-component-def
by simp

lemma checked-staged-security-with-query-prefix-fixed-current-authenticated-component-imp-current:
assumes hit:
  checked-staged-security-with-query-prefix-fixed-current-authenticated-component
  E trace-openings composition-openings i out

```

shows

checked-staged-security-with-query-prefix-authenticated-opening-hit
trace-openings composition-openings i out

using *hit*

unfolding

checked-staged-security-with-query-prefix-fixed-current-authenticated-component-def
by *simp*

lemma *checked-staged-security-with-query-prefix-fixed-header-component-split-current:*

assumes *hit:*

checked-staged-security-with-query-prefix-fixed-header-component
E trace-openings composition-openings i out

shows

checked-staged-security-with-query-prefix-fixed-current-authenticated-component
E trace-openings composition-openings i out $\vee$
checked-staged-security-with-query-prefix-header-witness-not-current
E trace-openings composition-openings i out

using *hit*

unfolding

checked-staged-security-with-query-prefix-fixed-current-authenticated-component-def
checked-staged-security-with-query-prefix-header-witness-not-current-def

by *blast*

lemma *checked-staged-security-with-query-prefix-fixed-header-componentE:*

assumes *hit:*

checked-staged-security-with-query-prefix-fixed-header-component
E trace-openings composition-openings i
(*Some* ((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*))

obtains

E (*Some* ((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*))
staged-composition-fri-roots data $\neq []$
(*trace-openings*, *composition-openings*) $\in$
query-header-supported-partial-opening-witnesses
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
(*staged-trace-root data*)
(*staged-trace-fri-roots data*)
(*staged-trace-final data*)
(*staged-alphas data*)
(*staged-degree data*)
(*staged-composition-fri-roots data*)
(*staged-composition-final data*)

index (*to-nat raw*) $\in$

staged-query-prefix-candidate-opening-query-target-from-prefix
(*(replicate rounds [])* [*i* := *trace-openings*])
(*(replicate rounds [])* [*i* := *composition-openings*])
i prefix prefix-state

```

proof –
  have E:
    E (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  and header:
    checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
    trace-openings composition-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  using hit
  unfolding checked-staged-security-with-query-prefix-fixed-header-component-def
  by simp-all
from header show ?thesis
proof (rule
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitE)
assume comp-nonempty:
  staged-composition-fri-roots data ≠ []
and witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
and target:
  index (to-nat raw) ∈
  staged-query-prefix-candidate-opening-query-target-from-prefix
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  i prefix prefix-state
show ?thesis
  by (rule that[OF E comp-nonempty witness target])
qed
qed

```

lemma *checked-staged-security-with-query-prefix-fixed-header-component-supportE*:

```

assumes support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)
and hit:

```

checked-staged-security-with-query-prefix-fixed-header-component
E trace-openings composition-openings i
(Some (((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))
obtains
E (Some (((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute
(checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute
(checked-staged-after-query-prefix-receive-with-verifier A i
((prefix, prefix-state), raw))
raw-state)
staged-alphas data = sqp-alphas prefix
staged-trace-root data = sqp-trace-root prefix
staged-composition-fri-roots data = sqp-composition-fri-roots prefix
staged-composition-fri-roots data ≠ []
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
index (to-nat raw) ∈
staged-query-prefix-candidate-opening-query-target-from-prefix
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i prefix prefix-state
proof –
have *prefix-receive*:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute
(checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *continuation*:
Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute


```

      (checked-staged-after-query-prefix-receive-with-verifier A i
        ((prefix, prefix-state), raw))
      raw-state)
using support
unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
by (auto elim!: set-dist-bindE)
have E:
  E (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
and header:
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
  trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
using hit
unfolding checked-staged-security-with-query-prefix-fixed-header-component-def
by simp-all
from header show ?thesis
proof (rule
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitE)
assume comp-nonempty:
  staged-composition-fri-roots data ≠ []
and witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
and target:
  index (to-nat raw) ∈
  staged-query-prefix-candidate-opening-query-target-from-prefix
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  i prefix prefix-state
have alphas:
  staged-alphas data = sqp-alphas prefix
by (rule checked-staged-security-with-query-prefix-data-state-alphas-eq)
  (rule support)
have trace-root:
  staged-trace-root data = sqp-trace-root prefix
and comp-roots:
  staged-composition-fri-roots data = sqp-composition-fri-roots prefix
using support

```

```

unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
  checked-staged-after-query-prefix-receive-with-verifier-def
  checked-staged-after-query-prefix-receive-def
  by (auto elim!: set-dist-bindE split: prod.splits)
show ?thesis
by (rule that[OF E prefix-receive continuation alphas trace-root
  comp-roots comp-nonempty witness target])
qed
qed

lemma checked-staged-security-with-query-prefix-header-witness-not-current-imp-path-output-on-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-header-witness-not-current
  E trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
shows
  checked-staged-security-with-query-prefix-header-witness-path-output-hit
  E trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof –
let ?out =
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have fixed:
  checked-staged-security-with-query-prefix-fixed-header-component
  E trace-openings composition-openings i ?out
and not-current:
   $\neg$  checked-staged-security-with-query-prefix-authenticated-opening-hit
  trace-openings composition-openings i ?out
using hit
unfolding
  checked-staged-security-with-query-prefix-header-witness-not-current-def
by simp-all
have support-facts:

```

$E \text{ ?out } \wedge$
Some (((*data*, *attacker-state*), *result*), *final-state*) $\in$
set-dist
 (*execute*
 (*checked-staged-after-query-prefix-receive-with-verifier A i*
 ((*prefix*, *prefix-state*), *raw*))
raw-state) $\wedge$
staged-alphas data = sqp-alphas prefix $\wedge$
staged-composition-fri-roots data $\neq \square$ $\wedge$
 (*trace-openings*, *composition-openings*) $\in$
query-header-supported-partial-opening-witnesses ?s
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)
 (*staged-composition-final data*) $\wedge$
index (to-nat raw) \in
staged-query-prefix-candidate-opening-query-target-from-prefix
 ((*replicate rounds* $\square$) [*i := trace-openings*])
 ((*replicate rounds* $\square$) [*i := composition-openings*])
i prefix prefix-state
proof (*rule*
checked-staged-security-with-query-prefix-fixed-header-component-supportE
 [*OF support fixed*])
assume *E: E ?out*
and *continuation:*
Some (((*data*, *attacker-state*), *result*), *final-state*) $\in$
set-dist
 (*execute*
 (*checked-staged-after-query-prefix-receive-with-verifier A i*
 ((*prefix*, *prefix-state*), *raw*))
raw-state)
and *alphas-eq: staged-alphas data = sqp-alphas prefix*
and *comp-nonempty: staged-composition-fri-roots data* $\neq \square$
and *witness:*
 (*trace-openings*, *composition-openings*) $\in$
query-header-supported-partial-opening-witnesses ?s
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)
 (*staged-composition-final data*)
and *target:*
index (to-nat raw) \in
staged-query-prefix-candidate-opening-query-target-from-prefix

```

    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i prefix prefix-state
  show ?thesis
    using E continuation alphas-eq comp-nonempty witness target by simp
qed
have E: E ?out
  using support-facts by blast
have continuation:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute
        (checked-staged-after-query-prefix-receive-with-verifier A i
          ((prefix, prefix-state), raw))
        raw-state)
  using support-facts by blast
have alphas-eq: staged-alphas data = sqp-alphas prefix
  using support-facts by blast
have comp-nonempty: staged-composition-fri-roots data ≠ []
  using support-facts by blast
have witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  using support-facts by blast
have target:
  index (to-nat raw) ∈
    staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i prefix prefix-state
  using support-facts by blast
from checked-staged-after-query-prefix-receive-with-verifier-outcome(2)
  [OF continuation]
have current-verifier:
  Some (result, final-state) ∈
    set-dist (execute verify-monad ?s)
  .
have s-ext-final: ?s ≤ final-state
  by (rule verify-monad-hash-extends[OF current-verifier])
from witness obtain witness-result witness-state rest where
  witness-out:
    Some (witness-result, witness-state) ∈

```

```

    set-dist (execute verify-monad ?s)
and trace-table-witness:
    partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings witness-state
and comp-table-witness:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings witness-state
and header:
    verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    rest
by (rule query-header-supported-partial-opening-witnessesE)
have s-ext-witness: ?s ≤ witness-state
by (rule verify-monad-hash-extends[OF witness-out])
have consistent:
    partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
proof –
have target-success:
    index (to-nat raw) ∈
    partial-query-success-indices-at
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    (sqp-alphas prefix) i
using target
unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
by simp
then have round:
    partial-query-round-consistent
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    (sqp-alphas prefix) i (index (to-nat raw))
unfolding partial-query-success-indices-at-def by simp
have trace-nth:
    ((replicate rounds []) [i := trace-openings]) ! i = trace-openings
using i-bound by simp
have comp-nth:
    ((replicate rounds []) [i := composition-openings]) ! i =
    composition-openings
using i-bound by simp
have idx-sample: index (to-nat raw) ∈ query-sample-space
using round unfolding partial-query-round-consistent-def by simp
have trace-indices-full:

```

```

    map opening-index
      (((replicate rounds []) [i := trace-openings]) ! i) =
      powers-scaled (index (to-nat raw))
  using round unfolding partial-query-round-consistent-def by blast
have comp-indices-full:
  map opening-index
    (((replicate rounds []) [i := composition-openings]) ! i) =
    [index (to-nat raw),
     fri-sibling-index (scale * clength) (index (to-nat raw))]
  using round unfolding partial-query-round-consistent-def by blast
have comp-nonempty-full:
  (((replicate rounds []) [i := composition-openings]) ! i ≠ [])
  using round unfolding partial-query-round-consistent-def by blast
have comp-value-full:
  opening-value
    (((replicate rounds []) [i := composition-openings]) ! i ! 0) =
    cp-eval (sqp-alphas prefix)
    (map opening-value
      (((replicate rounds []) [i := trace-openings]) ! i))
    (h ^ index (to-nat raw) * shift)
  using round unfolding partial-query-round-consistent-def by blast
have
  partial-query-openings-consistent trace-openings composition-openings
    (sqp-alphas prefix) (index (to-nat raw))
  unfolding partial-query-openings-consistent-def
  using idx-sample trace-indices-full comp-indices-full comp-nonempty-full
    comp-value-full trace-nth comp-nth
  by simp
then show ?thesis
  using alphas-eq by simp
qed
have trace-pullback:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings ?s ∨
  (∃ opn ∈ set trace-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state
        (staged-trace-root data)
        (scale * clength)
        (opening-index opn)
        (opening-value opn)
        (opening-path opn)) ?s witness-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit
    [OF s-ext-witness trace-table-witness])
then show ?thesis
proof
  assume trace-table-s:
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings ?s

```

```

have comp-pullback:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings ?s ∨
  (∃ opn ∈ set composition-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state
        (hd (staged-composition-fri-roots data))
        (scale * clength)
        (opening-index opn)
        (opening-value opn)
        (opening-path opn)) ?s witness-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit
    [OF s-ext-witness comp-table-witness])
then show ?thesis
proof
  assume comp-table-s:
    partial-authenticated-table
      (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings ?s
  have trace-table-final:
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings final-state
  by (rule partial-authenticated-table-mono[OF trace-table-s s-ext-final])
  have comp-table-final:
    partial-authenticated-table
      (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings final-state
  by (rule partial-authenticated-table-mono[OF comp-table-s s-ext-final])
  have current:
    checked-staged-security-with-query-prefix-authenticated-opening-hit
      trace-openings composition-openings i ?out
  unfolding
    checked-staged-security-with-query-prefix-authenticated-opening-hit-def
  using comp-nonempty trace-table-final comp-table-final consistent
  by simp
  then show ?thesis
    using not-current by contradiction
next
  assume comp-path:
    ∃ opn ∈ set composition-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots witness-state
          (hd (staged-composition-fri-roots data)) (scale * clength)
          (opening-index opn) (opening-value opn) (opening-path opn))
          ?s witness-state
  have body:
    ∃ witness-result witness-state rest.
      let s = ?s in
      Some (witness-result, witness-state) ∈

```

```

    set-dist (execute verify-monad s) ∧
verifier-header-transcript s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  rest ∧
((∃ opn ∈ set trace-openings.
  hash-map-new-output-hit
    (merkle-path-target-roots witness-state
      (staged-trace-root data)
      (scale * clength)
      (opening-index opn)
      (opening-value opn)
      (opening-path opn))
    s witness-state) ∨
(∃ opn ∈ set composition-openings.
  hash-map-new-output-hit
    (merkle-path-target-roots witness-state
      (hd (staged-composition-fri-roots data))
      (scale * clength)
      (opening-index opn)
      (opening-value opn)
      (opening-path opn))
    s witness-state))
by (intro exI[of - witness-result] exI[of - witness-state]
  exI[of - rest])
  (simp add: witness-out header comp-path)
show ?thesis
  unfolding
    checked-staged-security-with-query-prefix-header-witness-path-output-hit-def
  using E comp-nonempty body by simp
qed
next
assume trace-path:
  ∃ opn∈set trace-openings.
  hash-map-new-output-hit
    (merkle-path-target-roots witness-state
      (staged-trace-root data) (scale * clength)
      (opening-index opn) (opening-value opn) (opening-path opn))
  ?s witness-state
have body:
  ∃ witness-result witness-state rest.
  let s = ?s in
  Some (witness-result, witness-state) ∈
  set-dist (execute verify-monad s) ∧

```



```

verifier-header-transcript s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  rest ∧
((∃ opn ∈ set trace-openings.
  hash-map-new-output-hit
    (merkle-path-target-roots witness-state
      (staged-trace-root data)
      (scale * clength)
      (opening-index opn)
      (opening-value opn)
      (opening-path opn))
    s witness-state) ∨
(∃ opn ∈ set composition-openings.
  hash-map-new-output-hit
    (merkle-path-target-roots witness-state
      (hd (staged-composition-fri-roots data))
      (scale * clength)
      (opening-index opn)
      (opening-value opn)
      (opening-path opn))
    s witness-state))
by (intro exI[of - witness-result] exI[of - witness-state]
  exI[of - rest])
(simp add: witness-out header trace-path)
show ?thesis
unfolding
  checked-staged-security-with-query-prefix-header-witness-path-output-hit-def
using E comp-nonempty body by simp
qed
qed

lemma checked-staged-security-with-query-prefix-fixed-current-authenticated-component-imp-prefix-or-path-output-on
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and support:
  out ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-fixed-current-authenticated-component

```

E trace-openings composition-openings i out
shows
checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
trace-openings composition-openings i out $\vee$
checked-staged-security-with-query-prefix-opening-path-output-hit
trace-openings composition-openings out
by (rule
checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-prefix-or-path-output-hit-on-support
 $[OF\ wf\ controlled\ i\text{-bound}\ support]$)
(rule
checked-staged-security-with-query-prefix-fixed-current-authenticated-component-imp-current
 $[OF\ hit]$)

lemma *checked-staged-security-with-query-prefix-fixed-header-component-bound:*

assumes *raw-bound: query-index-raw-preimage-bound*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*

shows

wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
(*checked-staged-security-with-query-prefix-fixed-header-component*
E trace-openings composition-openings i)
adversary-initial-state $\leq$
staged-phase-relation-error size
(*staged-query-search-queries budgets i + 1*) +
 $(1::prob) / nnreal\ (card\ query\ sample\ space)$

proof —

have *event-le:*

wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
(*checked-staged-security-with-query-prefix-fixed-header-component*
E trace-openings composition-openings i)
adversary-initial-state $\leq$
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
(*checked-staged-security-with-actual-query-prefix-candidate-opening-hit*
 $((replicate\ rounds\ [])\ [i := trace-openings])$
 $((replicate\ rounds\ [])\ [i := composition-openings])$
i)
adversary-initial-state

proof (rule *wp-event-mono*)

fix *out*

assume

checked-staged-security-with-query-prefix-fixed-header-component
E trace-openings composition-openings i out

then have

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i out

```

    by (rule
        checked-staged-security-with-query-prefix-fixed-header-component-imp-header)
  then show
    checked-staged-security-with-actual-query-prefix-candidate-opening-hit
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i out
  by (rule
        checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-imp-actual)
qed
have actual-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i)
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space)
  by (rule checked-staged-security-with-actual-query-prefix-candidate-opening-hit-bound
      [OF raw-bound wf controlled])
  (use i-bound in simp)
  show ?thesis
  by (rule order-trans[OF event-le actual-bound])
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-bound-from-budgets:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  shows
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-fixed-header-component
      E trace-openings composition-openings i)
    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    (1::prob) / nnreal (card query-sample-space)
  proof -
    have raw-bound: query-index-raw-preimage-bound
    by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
    show ?thesis
    by (rule
        checked-staged-security-with-query-prefix-fixed-header-component-bound
        [OF raw-bound wf controlled i-bound])
  qed

```

lemma *checked-staged-security-with-query-prefix-fixed-current-authenticated-component-bound:*

assumes *raw-bound: query-index-raw-preimage-bound*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

shows

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-fixed-current-authenticated-component*
E trace-openings composition-openings i)

adversary-initial-state ≤

staged-phase-relation-error size

(*staged-query-search-queries budgets i + 1*) +

(*1::prob*) / *nnreal (card query-sample-space)*

proof —

have *event-le:*

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-fixed-current-authenticated-component*
E trace-openings composition-openings i)

adversary-initial-state ≤

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-authenticated-opening-hit*
trace-openings composition-openings i)

adversary-initial-state

by (*rule wp-event-mono*)

(*rule*

checked-staged-security-with-query-prefix-fixed-current-authenticated-component-imp-current)

have *current-bound:*

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-authenticated-opening-hit*
trace-openings composition-openings i)

adversary-initial-state ≤

staged-phase-relation-error size

(*staged-query-search-queries budgets i + 1*) +

(*1::prob*) / *nnreal (card query-sample-space)*

by (*rule checked-staged-security-with-query-prefix-authenticated-opening-hit-bound*

[*OF raw-bound wf controlled i-bound*])

show *?thesis*

by (*rule order-trans[OF event-le current-bound]*)

qed

lemma *checked-staged-security-with-query-prefix-fixed-current-authenticated-component-bound-from-budgets:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

shows
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 E trace-openings composition-openings i)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 (1::prob) / nnreal (card query-sample-space)

proof –
have raw-bound: query-index-raw-preimage-bound
by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
show ?thesis
by (rule
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component-bound
 [OF raw-bound wf controlled i-bound])

qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-imp-current-or-witness-path-output-on-supp

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i < \text{rounds}$
and support:
 out $\in$
 set-dist
 (execute
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 adversary-initial-state)
and hit:
 checked-staged-security-with-query-prefix-fixed-header-component
 E trace-openings composition-openings i out

shows
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 E trace-openings composition-openings i out $\vee$
 checked-staged-security-with-query-prefix-header-witness-path-output-hit
 E trace-openings composition-openings i out

proof (cases out)
case None
then have False
using hit
unfolding
 checked-staged-security-with-query-prefix-fixed-header-component-def
 checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-def
by simp
then show ?thesis **by** simp

next
case (Some packed)
obtain prefix prefix-state raw raw-state data attacker-state result final-state
where out-eq:

```

    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
  using Some by (cases packed) (auto split: prod.splits)
from checked-staged-security-with-query-prefix-fixed-header-component-split-current
  [OF hit]
show ?thesis
proof
  assume current:
    checked-staged-security-with-query-prefix-fixed-current-authenticated-component
    E trace-openings composition-openings i out
  then show ?thesis by simp
next
  assume not-current:
    checked-staged-security-with-query-prefix-header-witness-not-current
    E trace-openings composition-openings i out
  have support':
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
  using support out-eq by simp
  have not-current':
    checked-staged-security-with-query-prefix-header-witness-not-current
    E trace-openings composition-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  using not-current out-eq by simp
  have path:
    checked-staged-security-with-query-prefix-header-witness-path-output-hit
    E trace-openings composition-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  by (rule
    checked-staged-security-with-query-prefix-header-witness-not-current-imp-path-output-on-support
    [OF wf controlled i-bound support' not-current'])
  then show ?thesis
    using out-eq by simp
qed
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-bound-from-current-and-witness-path-outpu
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and current-bound:
    wp-event

```

(checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 E trace-openings composition-openings i)
 adversary-initial-state $\leq P$
and path-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-header-witness-path-output-hit
 E trace-openings composition-openings i)
 adversary-initial-state $\leq Q$
shows
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-fixed-header-component
 E trace-openings composition-openings i)
 adversary-initial-state $\leq P + Q$
proof –
have event-le:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-fixed-header-component
 E trace-openings composition-openings i)
 adversary-initial-state $\leq$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (λ out.
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 E trace-openings composition-openings i out $\vee$
 checked-staged-security-with-query-prefix-header-witness-path-output-hit
 E trace-openings composition-openings i out)
 adversary-initial-state
by (rule wp-event-mono-on-support)
 (rule
 checked-staged-security-with-query-prefix-fixed-header-component-imp-current-or-witness-path-output-on-support
 [OF wf controlled i -bound])
have union-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (λ out.
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 E trace-openings composition-openings i out $\vee$
 checked-staged-security-with-query-prefix-header-witness-path-output-hit
 E trace-openings composition-openings i out)
 adversary-initial-state $\leq$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 E trace-openings composition-openings i)
 adversary-initial-state $+$

wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-header-witness-path-output-hit*
 E trace-openings composition-openings i)
adversary-initial-state
by (rule *wp-event-union-bound*)
have *sum-bound*:
wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-fixed-current-authenticated-component*
 E trace-openings composition-openings i)
adversary-initial-state +
wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-header-witness-path-output-hit*
 E trace-openings composition-openings i)
adversary-initial-state $\leq P + Q$
by (intro *add-mono current-bound path-bound*)
show *?thesis*
by (rule *order-trans[OF event-le]*)
 (rule *order-trans[OF union-bound sum-bound]*)
qed

lemma *checked-staged-security-with-query-prefix-fixed-header-component-imp-current-or-witness-transcript-hit-on-su*

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *i-bound*: *i < rounds*

and *support*:

out $\in$

set-dist

 (*execute*

 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)

adversary-initial-state)

and *hit*:

checked-staged-security-with-query-prefix-fixed-header-component

E trace-openings composition-openings i out

shows

checked-staged-security-with-query-prefix-fixed-current-authenticated-component

E trace-openings composition-openings i out $\vee$

checked-staged-security-with-query-prefix-header-witness-transcript-hit

E out

proof –

have *split*:

checked-staged-security-with-query-prefix-fixed-current-authenticated-component

E trace-openings composition-openings i out $\vee$

checked-staged-security-with-query-prefix-header-witness-path-output-hit

E trace-openings composition-openings i out

by (rule

checked-staged-security-with-query-prefix-fixed-header-component-imp-current-or-witness-path-output-on-support


```

    [OF wf controlled i-bound support hit])
then show ?thesis
proof
  assume
    checked-staged-security-with-query-prefix-fixed-current-authenticated-component
    E trace-openings composition-openings i out
  then show ?thesis by simp
next
  assume path:
    checked-staged-security-with-query-prefix-header-witness-path-output-hit
    E trace-openings composition-openings i out
  have
    checked-staged-security-with-query-prefix-header-witness-transcript-hit
    E out
  by (rule
    checked-staged-security-with-query-prefix-header-witness-path-output-imp-transcript-hit
    [OF path])
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-bound-from-current-and-witness-transcript-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i < \text{rounds}$ 
and current-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-fixed-current-authenticated-component
    E trace-openings composition-openings i)
  adversary-initial-state  $\leq P$ 
and transcript-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-header-witness-transcript-hit
    E)
  adversary-initial-state  $\leq Q$ 
shows
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-fixed-header-component
    E trace-openings composition-openings i)
  adversary-initial-state  $\leq P + Q$ 
proof –
  have event-le:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-fixed-header-component
      E trace-openings composition-openings i)

```

```

    adversary-initial-state ≤
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (λout.
    checked-staged-security-with-query-prefix-fixed-current-authenticated-component
      E trace-openings composition-openings i out ∨
    checked-staged-security-with-query-prefix-header-witness-transcript-hit
      E out)
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (rule
    checked-staged-security-with-query-prefix-fixed-header-component-imp-current-or-witness-transcript-hit-on-support
    [OF wf controlled i-bound])
have union-bound:
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (λout.
    checked-staged-security-with-query-prefix-fixed-current-authenticated-component
      E trace-openings composition-openings i out ∨
    checked-staged-security-with-query-prefix-header-witness-transcript-hit
      E out)
  adversary-initial-state ≤
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-fixed-current-authenticated-component
    E trace-openings composition-openings i)
  adversary-initial-state +
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-header-witness-transcript-hit
    E)
  adversary-initial-state
by (rule wp-event-union-bound)
have sum-bound:
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-fixed-current-authenticated-component
    E trace-openings composition-openings i)
  adversary-initial-state +
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-header-witness-transcript-hit
    E)
  adversary-initial-state ≤ P + Q
by (intro add-mono current-bound transcript-bound)
show ?thesis
by (rule order-trans[OF event-le])
  (rule order-trans[OF union-bound sum-bound])
qed

```

lemma *checked-staged-security-with-query-prefix-fixed-header-component-bound-from-current-pre-and-witness-new-hit*

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *i-bound*: $i < \text{rounds}$

and *current-bound*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 E trace-openings composition-openings i)

adversary-initial-state $\leq P$

and *pre-bound*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
 E)

adversary-initial-state $\leq R$

and *new-bound*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
 E)

adversary-initial-state $\leq Q$

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-fixed-header-component
 E trace-openings composition-openings i)

adversary-initial-state $\leq P + R + Q$

proof –

have *transcript-bound*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-header-witness-transcript-hit
 E)

adversary-initial-state $\leq R + Q$

proof –

have *event-le*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-header-witness-transcript-hit
 E)

adversary-initial-state $\leq$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

($\lambda \text{out.}$

 checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
 E out $\vee$

 checked-staged-security-with-query-prefix-header-witness-transcript-new-hit

```

      E out)
    adversary-initial-state
  by (rule wp-event-mono)
    (rule
      checked-staged-security-with-query-prefix-header-witness-transcript-hit-imp-pre-or-new)
have union-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    ( $\lambda out.$ 
      checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
      E out  $\vee$ 
      checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
      E out)
    adversary-initial-state  $\leq$ 
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
      E)
    adversary-initial-state +
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
      E)
    adversary-initial-state
  by (rule wp-event-union-bound)
have sum-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
      E)
    adversary-initial-state +
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
      E)
    adversary-initial-state  $\leq R + Q$ 
  by (intro add-mono pre-bound new-bound)
show ?thesis
  by (rule order-trans[OF event-le])
    (rule order-trans[OF union-bound sum-bound])
qed
have header-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-fixed-header-component
      E trace-openings composition-openings i)
    adversary-initial-state  $\leq P + (R + Q)$ 
  by (rule
    checked-staged-security-with-query-prefix-fixed-header-component-bound-from-current-and-witness-transcript-hit

```

```

    [OF wf controlled i-bound current-bound transcript-bound])
  then show ?thesis
    by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-bound-from-witness-path-output:
  assumes raw-bound: query-index-raw-preimage-bound
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and path-bound:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-path-output-hit
      E trace-openings composition-openings i)
    adversary-initial-state  $\leq Q$ 
  shows
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-fixed-header-component
      E trace-openings composition-openings i)
    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    (1::prob) / nnreal (card query-sample-space) + Q
proof -
  have current-bound:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-fixed-current-authenticated-component
      E trace-openings composition-openings i)
    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    (1::prob) / nnreal (card query-sample-space)
  by (rule
    checked-staged-security-with-query-prefix-fixed-current-authenticated-component-bound
    [OF raw-bound wf controlled i-bound])
  show ?thesis
  by (rule
    checked-staged-security-with-query-prefix-fixed-header-component-bound-from-current-and-witness-path-output
    [OF wf controlled i-bound current-bound path-bound])
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-bound-from-witness-path-output-and-budget:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and path-bound:

```

```

wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-header-witness-path-output-hit
    E trace-openings composition-openings i)
  adversary-initial-state ≤ Q
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-fixed-header-component
      E trace-openings composition-openings i)
    adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      (1::prob) / nnreal (card query-sample-space) + Q
proof -
  have raw-bound: query-index-raw-preimage-bound
  by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
  show ?thesis
  by (rule
    checked-staged-security-with-query-prefix-fixed-header-component-bound-from-witness-path-output
      [OF raw-bound wf controlled i-bound path-bound])
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-bound-from-witness-transcript-hit:
  assumes raw-bound: query-index-raw-preimage-bound
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and transcript-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-header-witness-transcript-hit
        E)
    adversary-initial-state ≤ Q
  shows
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-fixed-header-component
        E trace-openings composition-openings i)
      adversary-initial-state ≤
        staged-phase-relation-error size
        (staged-query-search-queries budgets i + 1) +
        (1::prob) / nnreal (card query-sample-space) + Q
proof -
  have current-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-fixed-current-authenticated-component
        E trace-openings composition-openings i)

```

```

    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    (1::prob) / nnreal (card query-sample-space)
  by (rule
    checked-staged-security-with-query-prefix-fixed-current-authenticated-component-bound
    [OF raw-bound wf controlled i-bound])
  show ?thesis
  by (rule
    checked-staged-security-with-query-prefix-fixed-header-component-bound-from-current-and-witness-transcript-hit
    [OF wf controlled i-bound current-bound transcript-bound])
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-bound-from-witness-transcript-hit-and-budget
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and transcript-bound:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-transcript-hit
      E)
    adversary-initial-state  $\leq Q$ 
  shows
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-fixed-header-component
      E trace-openings composition-openings i)
    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    (1::prob) / nnreal (card query-sample-space) + Q
proof -
  have raw-bound: query-index-raw-preimage-bound
  by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
  show ?thesis
  by (rule
    checked-staged-security-with-query-prefix-fixed-header-component-bound-from-witness-transcript-hit
    [OF raw-bound wf controlled i-bound transcript-bound])
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-bound-from-data-state-pre-and-witness-new
  assumes raw-bound: query-index-raw-preimage-bound
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and pre-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit

```

$\text{adversary-initial-state} \leq R$
and *new-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-header-witness-transcript-new-hit}$
 $\quad E)$
 $\text{adversary-initial-state} \leq Q$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-fixed-header-component}$
 $\quad E \text{ trace-openings composition-openings } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(1::\text{prob}) / \text{nnreal} (\text{card query-sample-space}) + R + Q$
proof –
have *current-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-fixed-current-authenticated-component}$
 $\quad E \text{ trace-openings composition-openings } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(1::\text{prob}) / \text{nnreal} (\text{card query-sample-space})$
by (rule
 $\text{checked-staged-security-with-query-prefix-fixed-current-authenticated-component-bound}$
 $[\text{OF raw-bound wf controlled } i\text{-bound}])$
have *pre-hit-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit}$
 $\quad E)$
 $\text{adversary-initial-state} \leq R$
by (rule
 $\text{checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit-bound-from-data-state-pre-hit}$
 $[\text{OF } i\text{-bound pre-bound}])$
have *header-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-fixed-header-component}$
 $\quad E \text{ trace-openings composition-openings } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(1::\text{prob}) / \text{nnreal} (\text{card query-sample-space})) + R + Q$
by (rule
 $\text{checked-staged-security-with-query-prefix-fixed-header-component-bound-from-current-pre-and-witness-new-hit}$


```

    [OF wf controlled i-bound current-bound pre-hit-bound new-bound])
  then show ?thesis
    by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-data-state-bound-by-query-prefix-fixed-header-component:
  assumes raw-bound: query-index-raw-preimage-bound
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and event-imp:
     $\bigwedge out.$ 
    (case out of
      None  $\Rightarrow E$  None
    | Some (packed, t)  $\Rightarrow E$  (Some (snd packed, t)))  $\Rightarrow$ 
      checked-staged-security-with-query-prefix-fixed-header-component
      C trace-openings composition-openings i out
  shows
    wp-event (checked-staged-security-experiment-with-data-state A) E
      adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      (1::prob) / nnreal (card query-sample-space)
  proof -
    have projection:
      wp-event (checked-staged-security-experiment-with-data-state A) E
        adversary-initial-state =
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        ( $\lambda out.$  case out of
          None  $\Rightarrow E$  None
        | Some (packed, t)  $\Rightarrow E$  (Some (snd packed, t)))
        adversary-initial-state
    by (rule
      checked-staged-security-experiment-with-query-prefix-data-state-projection-event
      [OF i-bound])
    have lifted-bound:
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        ( $\lambda out.$  case out of
          None  $\Rightarrow E$  None
        | Some (packed, t)  $\Rightarrow E$  (Some (snd packed, t)))
        adversary-initial-state  $\leq$ 
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        (checked-staged-security-with-query-prefix-fixed-header-component
          C trace-openings composition-openings i)
        adversary-initial-state
    by (rule wp-event-mono) (use event-imp in blast)

```

have *fixed-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-fixed-header-component*
 C trace-openings composition-openings i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
 (*1::prob*) / *nnreal (card query-sample-space)*
by (*rule checked-staged-security-with-query-prefix-fixed-header-component-bound*
 [*OF raw-bound wf controlled i-bound*])
show *?thesis*
 unfolding *projection*
 by (*rule order-trans[OF lifted-bound fixed-bound]*)
qed

lemma *checked-staged-security-with-data-state-bound-by-query-prefix-fixed-header-component-from-budgets*:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *event-imp:*
 $\bigwedge_{out.}$
 (*case out of*
 None $\Rightarrow$ *E None*
 | *Some (packed, t)* $\Rightarrow$ *E (Some (snd packed, t))*) $\Rightarrow$
 checked-staged-security-with-query-prefix-fixed-header-component
 C trace-openings composition-openings i out
shows
 wp-event (checked-staged-security-experiment-with-data-state A) E
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
 (*1::prob*) / *nnreal (card query-sample-space)*
proof –
 have *raw-bound: query-index-raw-preimage-bound*
 by (*rule query-index-raw-preimage-bound-from-sampler-wellformed*)
 show *?thesis*
 by (*rule*
 checked-staged-security-with-data-state-bound-by-query-prefix-fixed-header-component
 [*OF raw-bound wf controlled i-bound event-imp*])
qed

lemma *checked-staged-security-with-data-state-bound-by-query-prefix-fixed-header-component-from-witness-path-outp*:
assumes *raw-bound: query-index-raw-preimage-bound*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *path-bound:*
 wp-event

```

    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-path-output-hit
      C trace-openings composition-openings i)
    adversary-initial-state ≤ Q
and event-imp:
   $\bigwedge$ out.
    (case out of
      None  $\Rightarrow$  E None
    | Some (packed, t)  $\Rightarrow$  E (Some (snd packed, t)))  $\Rightarrow$ 
      checked-staged-security-with-query-prefix-fixed-header-component
      C trace-openings composition-openings i out
shows
  wp-event (checked-staged-security-experiment-with-data-state A) E
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space) + Q
proof –
have projection:
  wp-event (checked-staged-security-experiment-with-data-state A) E
  adversary-initial-state =
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  ( $\lambda$ out. case out of
    None  $\Rightarrow$  E None
  | Some (packed, t)  $\Rightarrow$  E (Some (snd packed, t)))
  adversary-initial-state
by (rule
  checked-staged-security-experiment-with-query-prefix-data-state-projection-event
  [OF i-bound])
have lifted-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  ( $\lambda$ out. case out of
    None  $\Rightarrow$  E None
  | Some (packed, t)  $\Rightarrow$  E (Some (snd packed, t)))
  adversary-initial-state ≤
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-fixed-header-component
    C trace-openings composition-openings i)
  adversary-initial-state
by (rule wp-event-mono) (use event-imp in blast)
have fixed-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-fixed-header-component
    C trace-openings composition-openings i)
  adversary-initial-state ≤

```

```

    staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
      (1::prob) / nnreal (card query-sample-space) +  $Q$ 
  by (rule
    checked-staged-security-with-query-prefix-fixed-header-component-bound-from-witness-path-output
      [OF raw-bound wf controlled i-bound path-bound])
  show ?thesis
    unfolding projection
    by (rule order-trans[OF lifted-bound fixed-bound])
qed

lemma checked-staged-security-with-data-state-bound-by-query-prefix-fixed-header-component-from-witness-path-output
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets  $A$ 
    and i-bound:  $i < \text{rounds}$ 
    and path-bound:
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state  $A$   $i$ )
        (checked-staged-security-with-query-prefix-header-witness-path-output-hit
           $C$  trace-openings composition-openings  $i$ )
        adversary-initial-state  $\leq Q$ 
    and event-imp:
       $\bigwedge \text{out.}$ 
      (case out of
        None  $\Rightarrow E$  None
        | Some (packed,  $t$ )  $\Rightarrow E$  (Some (snd packed,  $t$ )))  $\Rightarrow$ 
        checked-staged-security-with-query-prefix-fixed-header-component
           $C$  trace-openings composition-openings  $i$  out
  shows
    wp-event (checked-staged-security-experiment-with-data-state  $A$ )  $E$ 
    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
      (1::prob) / nnreal (card query-sample-space) +  $Q$ 
proof -
  have raw-bound: query-index-raw-preimage-bound
    by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
  show ?thesis
    by (rule
      checked-staged-security-with-data-state-bound-by-query-prefix-fixed-header-component-from-witness-path-output
        [OF raw-bound wf controlled i-bound path-bound event-imp])
qed

lemma checked-staged-security-with-actual-alpha-prefix-data-state-bound-by-query-prefix-fixed-header-component:
  assumes raw-bound: query-index-raw-preimage-bound
    and wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets  $A$ 
    and i-bound:  $i < \text{rounds}$ 
    and event-imp:

```

$\wedge out.$
 (case out of
 None $\Rightarrow D$ None
 | Some (packed, t) $\Rightarrow D$ (Some (snd packed, t))) $\Longrightarrow$
 checked-staged-security-with-query-prefix-fixed-header-component
 C trace-openings composition-openings i out
shows
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 ($\lambda out.$ case out of
 None $\Rightarrow D$ None
 | Some (packed, t) $\Rightarrow$
 D (Some (((snd (fst (fst packed)), snd (fst packed)), snd packed),
 t)))
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 (1::prob) / nnreal (card query-sample-space)
proof –
 let ?project =
 $\lambda x.$ ((snd (fst (fst x)), snd (fst x)), snd x)
 have projection:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 ($\lambda out.$ case out of
 None $\Rightarrow D$ None
 | Some (packed, t) $\Rightarrow D$ (Some (?project packed, t)))
 adversary-initial-state =
 wp-event
 (checked-staged-security-experiment-with-data-state A)
 D
 adversary-initial-state
 proof –
 have
 wp-event (checked-staged-security-experiment-with-data-state A)
 D adversary-initial-state =
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
 $\ggg$ ($\lambda x.$ return (?project x)))
 D adversary-initial-state
 using
 checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
 [of A]
 by simp
 also have ... =
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 ($\lambda out.$ case out of
 None $\Rightarrow D$ None

```

    | Some (packed, t)  $\Rightarrow$  D (Some (?project packed, t)))
  adversary-initial-state
  by (rule wp-event-bind-return-map)
  finally show ?thesis
  by simp
qed
have data-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    D adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    (1::prob) / nnreal (card query-sample-space)
  by (rule
    checked-staged-security-with-data-state-bound-by-query-prefix-fixed-header-component
    [OF raw-bound wf controlled i-bound event-imp])
show ?thesis
  unfolding projection
  by (rule data-bound)
qed

lemma checked-staged-security-with-actual-alpha-prefix-data-state-bound-by-query-prefix-fixed-header-component-from
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and event-imp:
     $\bigwedge out.$ 
    (case out of
      None  $\Rightarrow$  D None
    | Some (packed, t)  $\Rightarrow$ 
      D (Some (snd packed, t)))  $\Longrightarrow$ 
      checked-staged-security-with-query-prefix-fixed-header-component
      C trace-openings composition-openings i out
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      ( $\lambda out.$  case out of
        None  $\Rightarrow$  D None
      | Some (packed, t)  $\Rightarrow$ 
        D (Some (((snd (fst (fst packed))), snd (fst packed)), snd packed),
          t)))
      adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      (1::prob) / nnreal (card query-sample-space)
proof -
  have raw-bound: query-index-raw-preimage-bound
  by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
show ?thesis
  by (rule
    checked-staged-security-with-actual-alpha-prefix-data-state-bound-by-query-prefix-fixed-header-component

```

[OF raw-bound wf controlled i-bound event-imp])

qed

lemma *checked-staged-security-with-actual-alpha-prefix-data-state-bound-by-query-prefix-fixed-header-component-from*
assumes raw-bound: *query-index-raw-preimage-bound*
and wf: *staged-budget-wellformed budgets*
and controlled: *staged-adversary-controlled budgets A*
and i-bound: *i < rounds*
and path-bound:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-header-witness-path-output-hit*
 C trace-openings composition-openings i)
 adversary-initial-state ≤ *Q*
and event-imp:
 $\bigwedge out.$
 (case out of
 None $\Rightarrow D$ None
 | Some (packed, t) $\Rightarrow D$ (Some (snd packed, t))) $\Rightarrow$
 checked-staged-security-with-query-prefix-fixed-header-component
 C trace-openings composition-openings i out
shows
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 ($\lambda out.$ case out of
 None $\Rightarrow D$ None
 | Some (packed, t) $\Rightarrow$
 D (Some (((snd (fst (fst packed)), snd (fst packed)), snd packed),
 t)))
 adversary-initial-state ≤
 staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
 (*1::prob*) / *nnreal (card query-sample-space)* + *Q*
proof –
 let ?project =
 $\lambda x.$ ((snd (fst (fst x)), snd (fst x)), snd x)
 have projection:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 ($\lambda out.$ case out of
 None $\Rightarrow D$ None
 | Some (packed, t) $\Rightarrow D$ (Some (?project packed, t)))
 adversary-initial-state =
 wp-event
 (*checked-staged-security-experiment-with-data-state A*)
 D
 adversary-initial-state
proof –
 have

```

wp-event (checked-staged-security-experiment-with-data-state A)
  D adversary-initial-state =
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
     $\gg$  ( $\lambda x. \text{return } (?project\ x)$ ))
  D adversary-initial-state
using
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
  [of A]
by simp
also have ... =
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ( $\lambda out. \text{case out of}$ 
    None  $\Rightarrow$  D None
    | Some (packed, t)  $\Rightarrow$  D (Some (?project packed, t)))
  adversary-initial-state
by (rule wp-event-bind-return-map)
finally show ?thesis
by simp
qed
have data-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  D adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space) + Q
by (rule
  checked-staged-security-with-data-state-bound-by-query-prefix-fixed-header-component-from-witness-path-output
  [OF raw-bound wf controlled i-bound path-bound event-imp])
show ?thesis
unfolding projection
by (rule data-bound)
qed

lemma checked-staged-security-with-actual-alpha-prefix-data-state-bound-by-query-prefix-fixed-header-component-from
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and path-bound:
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-header-witness-path-output-hit
    C trace-openings composition-openings i)
  adversary-initial-state  $\leq$  Q
and event-imp:
 $\bigwedge out.$ 
  (case out of
    None  $\Rightarrow$  D None

```


$| \text{Some } (\text{packed}, t) \Rightarrow D (\text{Some } (\text{snd } \text{packed}, t)) \Rightarrow$
checked-staged-security-with-query-prefix-fixed-header-component
C trace-openings composition-openings i out

shows

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 $(\lambda \text{out. case out of}$
 $\text{None} \Rightarrow D \text{ None}$
 $| \text{Some } (\text{packed}, t) \Rightarrow$
 $D (\text{Some } (((\text{snd } (\text{fst } (\text{fst } \text{packed})), \text{snd } (\text{fst } \text{packed})), \text{snd } \text{packed}),$
 $t)))$
adversary-initial-state $\leq$
staged-phase-relation-error size
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(1::\text{prob}) / \text{nnreal } (\text{card } \text{query-sample-space}) + Q$

proof –

have *raw-bound: query-index-raw-preimage-bound*
by (rule *query-index-raw-preimage-bound-from-sampler-wellformed*)
show *?thesis*
by (rule
checked-staged-security-with-actual-alpha-prefix-data-state-bound-by-query-prefix-fixed-header-component-from-w
 $[OF \text{ raw-bound wf controlled i-bound path-bound event-imp}]$)

qed

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-bound:*

assumes *raw-bound: query-index-raw-preimage-bound*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*

shows

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i)
adversary-initial-state $\leq$
staged-phase-relation-error size
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(1::\text{prob}) / \text{nnreal } (\text{card } \text{query-sample-space})$

proof –

have *event-le:*
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i)
adversary-initial-state $\leq$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-actual-query-prefix-candidate-opening-hit
 $((\text{replicate rounds } []) [i := \text{trace-openings}])$)

```

      ((replicate rounds []) [i := composition-openings])
    i)
  adversary-initial-state
by (rule wp-event-mono)
  (rule
    checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-imp-actual)
have actual-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i)
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space)
by (rule checked-staged-security-with-actual-query-prefix-candidate-opening-hit-bound
  [OF raw-bound wf controlled])
  (use i-bound in simp)
show ?thesis
  by (rule order-trans[OF event-le actual-bound])
qed

lemma checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-bound-from-budgets:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  shows
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
      trace-openings composition-openings i)
    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    (1::prob) / nnreal (card query-sample-space)
proof -
  have raw-bound: query-index-raw-preimage-bound
  by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
  show ?thesis
  by (rule
    checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-bound
    [OF raw-bound wf controlled i-bound])
qed

end

end

```

```

theory Staged-Security-Experiment-Composition-Query-Prefix-Witness
  imports Staged-Security-Experiment-Composition-Query-Augmented
begin

```

Witness-level bridge for the composition query-prefix path.

The existing query-prefix bounds are fixed-witness bounds: the trace and composition openings must already be selected before the query challenge is sampled. This layer records the exact local bridge from such fixed witness data to the augmented query-prefix component event. The remaining public soundness work is to derive these fixed witnesses from prefix Merkle/candidate evidence, or charge failures to existing Merkle/collision side events.

```

context soundness
begin

```

lemma *partial-query-openings-consistent-canonical-trace-index:*

assumes *consistent:*

partial-query-openings-consistent trace-openings composition-openings
as idx

shows *trace-openings* $\neq []$

and *idx* = *opening-index* (*hd trace-openings*)

proof –

have *trace-indices:*

map opening-index trace-openings = *powers-scaled idx*

using *consistent* **unfolding** *partial-query-openings-consistent-def* **by** *simp*

have *powers-nonempty: powers-scaled idx* $\neq []$

unfolding *powers-scaled-def* **using** *powers-pos* **by** *simp*

then show *trace-nonempty: trace-openings* $\neq []$

using *trace-indices* **by** *auto*

have *hd (map opening-index trace-openings)* = *hd (powers-scaled idx)*

using *trace-indices* **by** *simp*

then have *opening-index (hd trace-openings)* = *hd (powers-scaled idx)*

using *trace-nonempty* **by** (*simp add: hd-map*)

then show *idx* = *opening-index (hd trace-openings)*

using *powers-scaled-hd* **by** *simp*

qed

lemma *partial-query-success-indices-at-subset-canonical-trace-index:*

partial-query-success-indices-at trace-openings composition-openings as i

$\subseteq \{ \text{opening-index } (\text{hd } (\text{trace-openings } ! i)) \}$

proof

fix *idx*

assume *idx-in:*

idx $\in$ *partial-query-success-indices-at trace-openings*

composition-openings as i

have *trace-indices:*

map opening-index (trace-openings ! i) = *powers-scaled idx*

```

using idx-in
unfolding partial-query-success-indices-at-def
partial-query-round-consistent-def
by simp
have trace-nonempty: trace-openings ! i ≠ []
proof –
  have powers-scaled idx ≠ []
    unfolding powers-scaled-def using powers-pos by simp
  then show ?thesis
    using trace-indices by auto
qed
have hd (map opening-index (trace-openings ! i)) = hd (powers-scaled idx)
  using trace-indices by simp
then have opening-index (hd (trace-openings ! i)) = hd (powers-scaled idx)
  using trace-nonempty by (simp add: hd-map)
then show idx ∈ {opening-index (hd (trace-openings ! i))}
  using powers-scaled-hd by simp
qed

lemma staged-query-prefix-candidate-opening-query-target-from-prefix-subset-canonical-trace-index:
staged-query-prefix-candidate-opening-query-target-from-prefix
trace-openings composition-openings i prefix prefix-state ⊆
{opening-index (hd (trace-openings ! i))}
unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
by (rule partial-query-success-indices-at-subset-canonical-trace-index)

definition query-header-supported-partial-opening-canonical-indices
:: ('f, 'a) protocol-channel-scheme ⇒
'f ⇒ 'f list ⇒ 'f ⇒ 'f list ⇒ 'f ⇒
'f list ⇒ 'f ⇒ nat set
where
query-header-supported-partial-opening-canonical-indices s fr
f-fri-roots f-final as dg composition-fri-roots final =
{opening-index (hd trace-openings) |
trace-openings composition-openings.
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses s fr
f-fri-roots f-final as dg composition-fri-roots final ∧
trace-openings ≠ []}

definition query-header-supported-partial-opening-canonical-query-indices
:: ('f, 'a) protocol-channel-scheme ⇒
'f ⇒ 'f list ⇒ 'f ⇒ 'f list ⇒ 'f ⇒
'f list ⇒ 'f ⇒ nat set
where
query-header-supported-partial-opening-canonical-query-indices s fr
f-fri-roots f-final as dg composition-fri-roots final =
query-header-supported-partial-opening-canonical-indices s fr
f-fri-roots f-final as dg composition-fri-roots final ∩

```

query-sample-space

lemma *query-header-supported-partial-opening-canonical-query-indices-subset*:

query-header-supported-partial-opening-canonical-query-indices s fr

f-fri-roots f-final as dg composition-fri-roots final $\subseteq$

query-sample-space

unfolding *query-header-supported-partial-opening-canonical-query-indices-def*

by *blast*

lemma *query-header-supported-partial-opening-canonical-query-indices-iff*:

idx $\in$ *query-header-supported-partial-opening-canonical-query-indices s fr*

f-fri-roots f-final as dg composition-fri-roots final $\longleftrightarrow$

idx $\in$ *query-sample-space* $\wedge$

idx $\in$ *query-header-supported-partial-opening-canonical-indices s fr*

f-fri-roots f-final as dg composition-fri-roots final

unfolding *query-header-supported-partial-opening-canonical-query-indices-def*

by *blast*

lemma *finite-query-header-supported-partial-opening-canonical-query-indices[simp]*:

finite

(*query-header-supported-partial-opening-canonical-query-indices s fr*

f-fri-roots f-final as dg composition-fri-roots final)

by (rule *finite-subset*

[*OF query-header-supported-partial-opening-canonical-query-indices-subset*

finite-query-sample-space])

lemma *card-query-header-supported-partial-opening-canonical-query-indices-le*:

card

(*query-header-supported-partial-opening-canonical-query-indices s fr*

f-fri-roots f-final as dg composition-fri-roots final)

$\leq$ *card query-sample-space*

by (rule *card-mono*)

(rule *finite-query-sample-space*,

rule *query-header-supported-partial-opening-canonical-query-indices-subset*)

lemma *query-header-supported-partial-opening-canonical-query-indices-eq-query-sample-space-if-all-indices-witnessed*

assumes *witnessed*:

$\bigwedge idx. idx \in query-sample-space \implies$

$\exists trace-openings\ composition-openings.$

(*trace-openings*, *composition-openings*) $\in$

query-header-supported-partial-opening-witnesses s fr

f-fri-roots f-final as dg composition-fri-roots final $\wedge$

trace-openings $\neq [] \wedge$

opening-index (*hd trace-openings*) = *idx*

shows

query-header-supported-partial-opening-canonical-query-indices s fr

f-fri-roots f-final as dg composition-fri-roots final =

query-sample-space

proof

```

show
  query-header-supported-partial-opening-canonical-query-indices s fr
    f-fri-roots f-final as dg composition-fri-roots final  $\subseteq$ 
    query-sample-space
  by (rule query-header-supported-partial-opening-canonical-query-indices-subset)
next
show
  query-sample-space  $\subseteq$ 
  query-header-supported-partial-opening-canonical-query-indices s fr
    f-fri-roots f-final as dg composition-fri-roots final
proof
  fix idx
  assume idx-sample: idx  $\in$  query-sample-space
  then obtain trace-openings composition-openings where witness:
    (trace-openings, composition-openings)  $\in$ 
    query-header-supported-partial-opening-witnesses s fr
    f-fri-roots f-final as dg composition-fri-roots final
  and nonempty: trace-openings  $\neq \square$ 
  and idx-eq: opening-index (hd trace-openings) = idx
  using witnessed by blast
  have idx-canonical:
    idx  $\in$ 
    query-header-supported-partial-opening-canonical-indices s fr
    f-fri-roots f-final as dg composition-fri-roots final
  unfolding query-header-supported-partial-opening-canonical-indices-def
  using witness nonempty idx-eq by blast
  show
    idx  $\in$ 
    query-header-supported-partial-opening-canonical-query-indices s fr
    f-fri-roots f-final as dg composition-fri-roots final
  unfolding query-header-supported-partial-opening-canonical-query-indices-def
  using idx-sample idx-canonical by simp
  qed
qed

lemma table-agrees-with-authenticated-openings-from-partial-authenticated-table-if-no-partial-merkle-bad:
  assumes table:
    partial-authenticated-table rt (scale * clength) openings final-state
  and outcome:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
  and no-bad:
     $\neg$  partial-merkle-inconsistency-bad s (Some (result, final-state))
  shows  $\exists$  table.
    table-agrees-with-authenticated-openings table (scale * clength)
    openings
proof –
  have accepted-out: accepted (Some (result, final-state))
  unfolding accepted-def by simp
  have idx-bound:

```

```

 $\bigwedge \text{opn}. \text{opn} \in \text{set openings} \implies$ 
   $\text{opening-index } \text{opn} < \text{scale} * \text{clength}$ 
proof –
  fix opn
  assume opn-in:  $\text{opn} \in \text{set openings}$ 
  have auth: authenticated-opening-in final-state opn
    using table opn-in unfolding partial-authenticated-table-def by blast
  have len:  $\text{opening-length } \text{opn} = \text{scale} * \text{clength}$ 
    using table opn-in unfolding partial-authenticated-table-def by blast
  show  $\text{opening-index } \text{opn} < \text{scale} * \text{clength}$ 
    using auth len unfolding authenticated-opening-in-def by simp
qed
have consistent:
   $\bigwedge \text{opn } \text{opn}'. \text{opn} \in \text{set openings} \implies$ 
     $\text{opn}' \in \text{set openings} \implies$ 
     $\text{opening-index } \text{opn} = \text{opening-index } \text{opn}' \implies$ 
     $\text{opening-value } \text{opn} = \text{opening-value } \text{opn}'$ 
proof (rule ccontr)
  fix opn opn'
  assume opn-in:  $\text{opn} \in \text{set openings}$ 
  and opn'-in:  $\text{opn}' \in \text{set openings}$ 
  and same-index:  $\text{opening-index } \text{opn} = \text{opening-index } \text{opn}'$ 
  and neg:  $\text{opening-value } \text{opn} \neq \text{opening-value } \text{opn}'$ 
  have auth: authenticated-opening-in final-state opn
    using table opn-in unfolding partial-authenticated-table-def by blast
  have auth': authenticated-opening-in final-state opn'
    using table opn'-in unfolding partial-authenticated-table-def by blast
  have root:  $\text{opening-root } \text{opn} = \text{opening-root } \text{opn}'$ 
    using table opn-in opn'-in unfolding partial-authenticated-table-def
    by auto
  have len:  $\text{opening-length } \text{opn} = \text{opening-length } \text{opn}'$ 
    using table opn-in opn'-in unfolding partial-authenticated-table-def
    by auto
  have bad:
    partial-merkle-inconsistency-bad s (Some (result, final-state))
    unfolding partial-merkle-inconsistency-bad-def
    by (intro conjI exI[of - result] exI[of - final-state]
      exI[of - opn] exI[of - opn'])
    (use accepted-out auth auth' root len same-index neg in simp-all)
  then show False
    using no-bad by contradiction
qed
show ?thesis
  by (rule table-agrees-with-authenticated-openings-consistent-exists
    [OF idx-bound consistent])
qed

lemma partial-trace-table-candidate-single-witness-if-no-partial-merkle-bad:
  assumes auth:

```

```

    partial-authenticated-table fr (scale * clength) trace-openings
    final-state
  and outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and no-bad:
    ¬ partial-merkle-inconsistency-bad s (Some (result, final-state))
  and i-bound: i < rounds
shows ∃ trace-table.
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings])
proof -
  obtain trace-table where agrees:
    table-agrees-with-authenticated-openings trace-table (scale * clength)
    trace-openings
  using
    table-agrees-with-authenticated-openings-from-partial-authenticated-table-if-no-partial-merkle-bad
    [OF auth outcome no-bad]
  by blast
  have len: length trace-table = scale * clength
  using agrees unfolding table-agrees-with-authenticated-openings-def
  by simp
  have candidate:
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
  by (rule partial-trace-table-candidate-single-roundI
    [OF i-bound len agrees])
  then show ?thesis by blast
qed

```

lemma *partial-composition-table-candidate-single-witness-if-no-partial-merkle-bad*:

```

  assumes auth:
    partial-authenticated-table composition-root (scale * clength)
    composition-openings final-state
  and outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and no-bad:
    ¬ partial-merkle-inconsistency-bad s (Some (result, final-state))
  and i-bound: i < rounds
shows ∃ composition-table.
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
proof -
  obtain composition-table where agrees:
    table-agrees-with-authenticated-openings composition-table
    (scale * clength) composition-openings
  using
    table-agrees-with-authenticated-openings-from-partial-authenticated-table-if-no-partial-merkle-bad
    [OF auth outcome no-bad]
  by blast

```



```

have len: length composition-table = scale * clength
  using agrees unfolding table-agrees-with-authenticated-openings-def
  by simp
have candidate:
  partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
  by (rule partial-composition-table-candidate-single-roundI
    [OF i-bound len agrees])
then show ?thesis by blast
qed

lemma query-header-supported-partial-opening-witness-single-round-candidates-or-partial-merkle-bad:
assumes witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses s fr f-fri-roots
    f-final as dg composition-fri-roots final
and i-bound: i < rounds
shows
  (∃ result final-state.
    partial-merkle-inconsistency-bad s (Some (result, final-state))) ∨
  (∃ trace-table composition-table.
    partial-trace-table-candidate trace-table
      ((replicate rounds []) [i := trace-openings]) ∧
    partial-composition-table-candidate composition-table
      ((replicate rounds []) [i := composition-openings]))
proof –
from query-header-supported-partial-opening-witnessesE[OF witness]
obtain result final-state rest where outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and trace-auth:
  partial-authenticated-table fr (scale * clength) trace-openings
  final-state
and comp-auth:
  partial-authenticated-table (hd composition-fri-roots)
  (scale * clength) composition-openings final-state
by blast
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then show ?thesis by blast
next
case False
obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
using partial-trace-table-candidate-single-witness-if-no-partial-merkle-bad
  [OF trace-auth outcome False i-bound]
by blast

```

```

obtain composition-table where comp-candidate:
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
using
  partial-composition-table-candidate-single-witness-if-no-partial-merkle-bad
  [OF comp-auth outcome False i-bound]
by blast
show ?thesis
using trace-candidate comp-candidate by blast
qed
qed

lemma query-header-supported-partial-opening-witness-success-indices-fraction-bound-if-low-degree-candidates:
assumes i-bound: i < rounds
and trace-candidate:
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
and trace-low: trace-table-low-degree trace-table
and comp-low: composition-table-low-degree maxDegree composition-table
and not-all:
   $\neg$  all-queries-consistent trace-table composition-table as
shows
  nnreal
  (card
    {idx  $\in$  query-sample-space.
      partial-query-openings-consistent trace-openings
      composition-openings as idx} ) /
  nnreal (card query-sample-space)  $\leq$  query-error-bound
by (rule
  partial-query-openings-consistent-indices-fraction-bound-if-candidate-low-degree
  [OF i-bound trace-candidate comp-candidate trace-low comp-low not-all])

lemma accepted-with-partial-trace-openings-update-round:
assumes partial:
  accepted-with-partial-trace-openings s (Some (result, final-state))
  fr query-idxs trace-openings
and i-bound: i < rounds
and consistent:
  partial-query-openings-consistent trace-openings-i
  composition-openings-i as (query-idxs ! i)
and auth:
  partial-authenticated-table fr (scale * clength) trace-openings-i
  final-state
shows
  accepted-with-partial-trace-openings s (Some (result, final-state))
  fr query-idxs (trace-openings[i := trace-openings-i])

```

```

proof –
  have len-query: length query-idxs = rounds
    using accepted-with-partial-trace-openings-shapes(1)[OF partial] .
  have len-trace: length trace-openings = rounds
    using accepted-with-partial-trace-openings-shapes(2)[OF partial] .
  have trace-indices-i:
    map opening-index trace-openings-i = powers-scaled (query-idxs ! i)
    using consistent unfolding partial-query-openings-consistent-def by simp
  have indices:
     $\forall j < \text{rounds}.$ 
    map opening-index ((trace-openings[i := trace-openings-i]) ! j) =
      powers-scaled (query-idxs ! j)
  proof (intro allI impI)
    fix j
    assume j-bound: j < rounds
    show
      map opening-index ((trace-openings[i := trace-openings-i]) ! j) =
        powers-scaled (query-idxs ! j)
    proof (cases j = i)
      case True
      then show ?thesis
        using i-bound len-trace trace-indices-i by simp
    next
      case False
      then show ?thesis
        using accepted-with-partial-trace-openings-shapes(3)[OF partial j-bound]
          len-trace j-bound
        by simp
    qed
  qed
  have tables:
     $\forall j < \text{rounds}.$ 
    partial-authenticated-table fr (scale * clength)
      ((trace-openings[i := trace-openings-i]) ! j) final-state
  proof (intro allI impI)
    fix j
    assume j-bound: j < rounds
    show
      partial-authenticated-table fr (scale * clength)
        ((trace-openings[i := trace-openings-i]) ! j) final-state
    proof (cases j = i)
      case True
      then show ?thesis
        using i-bound len-trace auth by simp
    next
      case False
      then show ?thesis
        using partial j-bound len-trace
          unfolding accepted-with-partial-trace-openings-def

```

```

    by simp
  qed
qed
show ?thesis
  unfolding accepted-with-partial-trace-openings-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state])
    (use len-query len-trace indices tables in simp-all)
qed

lemma accepted-with-partial-composition-openings-update-round:
  assumes partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) composition-root query-idxs
    composition-openings
  and i-bound: i < rounds
  and consistent:
    partial-query-openings-consistent trace-openings-i
    composition-openings-i as (query-idxs ! i)
  and auth:
    partial-authenticated-table composition-root (scale * clength)
    composition-openings-i final-state
  shows
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) composition-root query-idxs
    (composition-openings[i := composition-openings-i])
proof -
  have len-query: length query-idxs = rounds
  using accepted-with-partial-composition-openings-shapes(1)[OF partial] .
  have len-comp: length composition-openings = rounds
  using accepted-with-partial-composition-openings-shapes(2)[OF partial] .
  have comp-indices-i:
    map opening-index composition-openings-i =
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]
  using consistent unfolding partial-query-openings-consistent-def by simp
  have indices:
    ∀ j < rounds.
      map opening-index
        ((composition-openings[i := composition-openings-i]) ! j) =
        [query-idxs ! j,
         fri-sibling-index (scale * clength) (query-idxs ! j)]
  proof (intro allI impI)
    fix j
    assume j-bound: j < rounds
    show
      map opening-index
        ((composition-openings[i := composition-openings-i]) ! j) =
        [query-idxs ! j,
         fri-sibling-index (scale * clength) (query-idxs ! j)]
  end
end

```

```

proof (cases  $j = i$ )
  case True
  then show ?thesis
    using i-bound len-comp comp-indices-i by simp
next
  case False
  then show ?thesis
    using accepted-with-partial-composition-openings-shapes(3)
      [OF partial j-bound]
      len-comp j-bound
    by simp
qed
qed
have tables:
   $\forall j < \text{rounds}.$ 
    partial-authenticated-table composition-root (scale * clength)
      ((composition-openings[ $i := \text{composition-openings-}i$ ]) !  $j$ )
      final-state
proof (intro allI impI)
  fix  $j$ 
  assume j-bound:  $j < \text{rounds}$ 
  show
    partial-authenticated-table composition-root (scale * clength)
      ((composition-openings[ $i := \text{composition-openings-}i$ ]) !  $j$ )
      final-state
proof (cases  $j = i$ )
  case True
  then show ?thesis
    using i-bound len-comp auth by simp
next
  case False
  then show ?thesis
    using partial j-bound len-comp
    unfolding accepted-with-partial-composition-openings-def
    by simp
qed
qed
show ?thesis
  unfolding accepted-with-partial-composition-openings-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state])
    (use len-query len-comp indices tables in simp-all)
qed

lemma accepted-with-partial-initial-openings-update-round:
assumes partial:
  accepted-with-partial-initial-openings s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final trace-query-idxs trace-openings
  composition-query-idxs composition-openings

```

and *i-bound*: $i < \text{rounds}$
and *same-idx*: $\text{trace-query-idxs} ! i = \text{composition-query-idxs} ! i$
and *consistent*:
 partial-query-openings-consistent trace-openings-i
 composition-openings-i as (trace-query-idxs ! i)
and *trace-auth*:
 *partial-authenticated-table fr (scale * clength) trace-openings-i*
 final-state
and *comp-auth*:
 partial-authenticated-table (hd composition-fri-roots)
 *(scale * clength) composition-openings-i final-state*
shows
 accepted-with-partial-initial-openings s
 (Some (result, final-state)) fr f-fri-roots f-final as dg
 composition-fri-roots final trace-query-idxs
 (trace-openings[i := trace-openings-i]) composition-query-idxs
 (composition-openings[i := composition-openings-i])
proof –
have *comp-nonempty*: $\text{composition-fri-roots} \neq []$
 using *accepted-with-partial-initial-openings-shapes(1)[OF partial]* .
have *header*:
 $\exists \text{rest. verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final as } dg$
 composition-fri-roots final rest
 using *accepted-with-partial-initial-openings-shapes(2)[OF partial]* .
have *trace-partial*:
 accepted-with-partial-trace-openings s
 (Some (result, final-state)) fr trace-query-idxs trace-openings
 using *accepted-with-partial-initial-openings-shapes(3)[OF partial]* .
have *comp-partial*:
 accepted-with-partial-composition-openings s
 (Some (result, final-state)) (hd composition-fri-roots)
 composition-query-idxs composition-openings
 using *accepted-with-partial-initial-openings-shapes(4)[OF partial]* .
have *trace-updated*:
 accepted-with-partial-trace-openings s
 (Some (result, final-state)) fr trace-query-idxs
 (trace-openings[i := trace-openings-i])
 by (rule *accepted-with-partial-trace-openings-update-round*
 [*OF trace-partial i-bound consistent trace-auth*])
have *comp-consistent*:
 partial-query-openings-consistent trace-openings-i
 composition-openings-i as (composition-query-idxs ! i)
 using *consistent same-idx by simp*
have *comp-updated*:
 accepted-with-partial-composition-openings s
 (Some (result, final-state)) (hd composition-fri-roots)
 composition-query-idxs
 (composition-openings[i := composition-openings-i])
 by (rule *accepted-with-partial-composition-openings-update-round*

```

    [OF comp-partial i-bound comp-consistent comp-auth])
have accepted-out: accepted (Some (result, final-state))
  by (rule accepted-with-partial-trace-openings-imp-accepted
    [OF trace-updated])
show ?thesis
  unfolding accepted-with-partial-initial-openings-def
  using accepted-out comp-nonempty header trace-updated comp-updated
  by simp
qed

lemma accepted-with-partial-initial-openings-update-round-candidates-if-no-partial-merkle-bad:
assumes partial:
  accepted-with-partial-initial-openings s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final trace-query-idxs trace-openings
  composition-query-idxs composition-openings
and no-bad:
  ¬ partial-merkle-inconsistency-bad s (Some (result, final-state))
and i-bound: i < rounds
and same-idx: trace-query-idxs ! i = composition-query-idxs ! i
and consistent:
  partial-query-openings-consistent trace-openings-i
  composition-openings-i as (trace-query-idxs ! i)
and trace-auth:
  partial-authenticated-table fr (scale * clength) trace-openings-i
  final-state
and comp-auth:
  partial-authenticated-table (hd composition-fri-roots)
  (scale * clength) composition-openings-i final-state
shows
  ∃ trace-table composition-table.
  partial-trace-table-candidate trace-table
  (trace-openings[i := trace-openings-i]) ∧
  partial-composition-table-candidate composition-table
  (composition-openings[i := composition-openings-i])
proof –
have updated:
  accepted-with-partial-initial-openings s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final trace-query-idxs
  (trace-openings[i := trace-openings-i]) composition-query-idxs
  (composition-openings[i := composition-openings-i])
  by (rule accepted-with-partial-initial-openings-update-round
    [OF partial i-bound same-idx consistent trace-auth comp-auth])
show ?thesis
  by (rule accepted-with-partial-initial-openings-candidates-if-no-partial-merkle-bad
    [OF updated no-bad])
qed

```

lemma *query-header-supported-partial-opening-success-indices-at-subset-canonical-indices:*

query-header-supported-partial-opening-success-indices-at s fr
f-fri-roots f-final as dg composition-fri-roots final i $\subseteq$
query-header-supported-partial-opening-canonical-indices s fr
f-fri-roots f-final as dg composition-fri-roots final

proof

fix *idx*

assume *idx-in:*

idx $\in$ query-header-supported-partial-opening-success-indices-at s fr
f-fri-roots f-final as dg composition-fri-roots final i

then obtain *trace-openings composition-openings* **where** *witness:*

(trace-openings, composition-openings) $\in$
query-header-supported-partial-opening-witnesses s fr
f-fri-roots f-final as dg composition-fri-roots final

and *consistent:*

partial-query-openings-consistent trace-openings composition-openings
as idx

by (rule *query-header-supported-partial-opening-success-indices-atE*)

have *nonempty:* *trace-openings $\neq []$*

and *idx-eq:* *idx = opening-index (hd trace-openings)*

by (rule *partial-query-openings-consistent-canonical-trace-index*
 $[OF\ consistent]) +$

show

idx $\in$ query-header-supported-partial-opening-canonical-indices s fr
f-fri-roots f-final as dg composition-fri-roots final

unfolding *query-header-supported-partial-opening-canonical-indices-def*

using *witness nonempty idx-eq* **by** *blast*

qed

lemma *query-header-supported-partial-opening-success-indices-at-subset-canonical-query-indices:*

query-header-supported-partial-opening-success-indices-at s fr
f-fri-roots f-final as dg composition-fri-roots final i $\subseteq$
query-header-supported-partial-opening-canonical-query-indices s fr
f-fri-roots f-final as dg composition-fri-roots final

proof

fix *idx*

assume *idx-in:*

idx $\in$ query-header-supported-partial-opening-success-indices-at s fr
f-fri-roots f-final as dg composition-fri-roots final i

have *idx-canonical:*

idx $\in$ query-header-supported-partial-opening-canonical-indices s fr
f-fri-roots f-final as dg composition-fri-roots final

using *idx-in*

query-header-supported-partial-opening-success-indices-at-subset-canonical-indices

by *blast*

have *idx-sample:* *idx $\in$ query-sample-space*

using *idx-in query-header-supported-partial-opening-success-indices-at-subset*

by *blast*

show


```

    idx ∈ query-header-supported-partial-opening-canonical-query-indices s fr
    f-fri-roots f-final as dg composition-fri-roots final
unfolding query-header-supported-partial-opening-canonical-query-indices-def
using idx-canonical idx-sample by simp
qed

lemma query-header-supported-partial-opening-success-indices-at-fraction-bound-if-canonical-query-indices:
assumes canonical-bound:
    nnreal
    (card
      (query-header-supported-partial-opening-canonical-query-indices
        s fr f-fri-roots f-final as dg composition-fri-roots final)) /
    nnreal (card query-sample-space) ≤ query-error-bound
shows
    nnreal
    (card
      (query-header-supported-partial-opening-success-indices-at s fr
        f-fri-roots f-final as dg composition-fri-roots final i)) /
    nnreal (card query-sample-space) ≤ query-error-bound
proof —
have subset:
    query-header-supported-partial-opening-success-indices-at s fr
    f-fri-roots f-final as dg composition-fri-roots final i ⊆
    query-header-supported-partial-opening-canonical-query-indices s fr
    f-fri-roots f-final as dg composition-fri-roots final
by (rule
      query-header-supported-partial-opening-success-indices-at-subset-canonical-query-indices)
have finite-canonical:
    finite
    (query-header-supported-partial-opening-canonical-query-indices s fr
      f-fri-roots f-final as dg composition-fri-roots final)
by simp
have card-le:
    card
    (query-header-supported-partial-opening-success-indices-at s fr
      f-fri-roots f-final as dg composition-fri-roots final i) ≤
    card
    (query-header-supported-partial-opening-canonical-query-indices s fr
      f-fri-roots f-final as dg composition-fri-roots final)
by (rule card-mono[OF finite-canonical subset])
have
    nnreal
    (card
      (query-header-supported-partial-opening-success-indices-at s fr
        f-fri-roots f-final as dg composition-fri-roots final i)) /
    nnreal (card query-sample-space) ≤
    nnreal
    (card
      (query-header-supported-partial-opening-canonical-query-indices s fr

```

$f\text{-fri-roots } f\text{-final as dg composition-fri-roots final})) /$
 $\text{nnreal (card query-sample-space)}$
by (rule $\text{nnreal-nat-divide-right-mono}[OF \text{ card-le}]$)
also have $\dots \leq \text{query-error-bound}$
by (rule canonical-bound)
finally show $?thesis$.
qed

lemma *query-header-supported-partial-opening-success-indices-at-subset-if-canonical-indices:*

assumes *canonical-cover:*

$\text{query-header-supported-partial-opening-canonical-indices } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final} \subseteq B$

shows

$\text{query-header-supported-partial-opening-success-indices-at } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final } i \subseteq B$

by (rule *order-trans*

[*OF*

$\text{query-header-supported-partial-opening-success-indices-at-subset-canonical-indices}$
 $\text{canonical-cover}]$)

lemma *query-header-supported-partial-opening-success-indices-at-subset-if-canonical-query-indices:*

assumes *canonical-cover:*

$\text{query-header-supported-partial-opening-canonical-query-indices } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final} \subseteq B$

shows

$\text{query-header-supported-partial-opening-success-indices-at } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final } i \subseteq B$

by (rule *order-trans*

[*OF*

$\text{query-header-supported-partial-opening-success-indices-at-subset-canonical-query-indices}$
 $\text{canonical-cover}]$)

lemma *staged-query-header-supported-partial-opening-success-indices-at-subset-if-canonical-indices:*

assumes *canonical-cover:*

$\text{query-header-supported-partial-opening-canonical-indices}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $\quad (\text{staged-proof-transcript data}))$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $(\text{staged-alphas data})$
 $(\text{staged-degree data})$
 $(\text{staged-composition-fri-roots data})$
 $(\text{staged-composition-final data}) \subseteq B$

shows

$\text{query-header-supported-partial-opening-success-indices-at}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $\quad (\text{staged-proof-transcript data}))$
 $(\text{staged-trace-root data})$

(staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
 $i \subseteq B$
by (rule
 query-header-supported-partial-opening-success-indices-at-subset-if-canonical-indices
 [OF canonical-cover])

lemma staged-query-header-supported-partial-opening-success-indices-at-subset-if-canonical-query-indices:

assumes canonical-cover:

query-header-supported-partial-opening-canonical-query-indices
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\subseteq B$

shows

query-header-supported-partial-opening-success-indices-at
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
 $i \subseteq B$

by (rule

query-header-supported-partial-opening-success-indices-at-subset-if-canonical-query-indices
 [OF canonical-cover])

lemma checked-staged-security-with-data-state-partial-opening-hit-bound-from-canonical-query-error-cover:

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and cover:

$\bigwedge$ data attacker-state.

query-header-supported-partial-opening-canonical-query-indices
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)

```

    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)  $\subseteq B$ 
  and subset:  $B \subseteq \text{query-sample-space}$ 
  and frac:
    nnreal (card  $B$ ) / nnreal (card query-sample-space)  $\leq$ 
      query-error-bound
  shows
    wp-event (checked-staged-security-experiment-with-data-state  $A$ )
      staged-security-with-data-state-query-partial-opening-hit
      adversary-initial-state  $\leq$ 
        nnreal
          (staged-attacker-query-budget budgets +
            staged-challenge-query-budget) * query-error-bound
  proof -
  have raw-bound: query-index-raw-preimage-bound
  by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
  show ?thesis
  proof (rule
    checked-staged-security-with-data-state-partial-opening-hit-bound-from-fixed-query-error-cover
    [OF wf controlled - raw-bound subset frac])
  fix data attacker-state  $i$ 
  assume  $i < \text{rounds}$ 
  show
    query-header-supported-partial-opening-success-indices-at
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
       $i \subseteq B$ 
  by (rule
    staged-query-header-supported-partial-opening-success-indices-at-subset-if-canonical-query-indices
    [OF cover])
qed
qed

lemma checked-staged-security-with-data-state-query-bad-bound-from-canonical-query-error-cover:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets  $A$ 
  and cover:
     $\bigwedge$  data attacker-state.
      query-header-supported-partial-opening-canonical-query-indices

```

```

    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)  $\subseteq B$ 
  and subset:  $B \subseteq \text{query-sample-space}$ 
  and frac:
    nnreal (card B) / nnreal (card query-sample-space)  $\leq$ 
      query-error-bound
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-bad-hit
      adversary-initial-state  $\leq$ 
        nnreal
          (staged-attacker-query-budget budgets +
            staged-challenge-query-budget) * query-error-bound
  proof -
    have partial-bound:
      wp-event (checked-staged-security-experiment-with-data-state A)
        staged-security-with-data-state-query-partial-opening-hit
        adversary-initial-state  $\leq$ 
          nnreal
            (staged-attacker-query-budget budgets +
              staged-challenge-query-budget) * query-error-bound
    by (rule
      checked-staged-security-with-data-state-partial-opening-hit-bound-from-canonical-query-error-cover
      [OF wf controlled cover subset frac])
    show ?thesis
      by (rule order.trans
        [OF checked-staged-security-with-data-state-query-bad-bound-from-partial-opening-hit
          [OF wf controlled] partial-bound])
  qed

lemma checked-staged-query-prefix-candidate-opening-target-from-prefix-canonical-indexD:
  assumes i-bound:  $i < \text{rounds}$ 
  and hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-opening-query-target-from-prefix
        ((replicate rounds []) [i := trace-openings])
        ((replicate rounds []) [i := composition-openings])
        i)
      (Some (((prefix, prefix-state), raw), raw-state))
  shows index (to-nat raw) = opening-index (hd trace-openings)
  proof -
    have idx-in:

```

```

index (to-nat raw) ∈
  staged-query-prefix-candidate-opening-query-target-from-prefix
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i prefix prefix-state
using hit unfolding checked-staged-query-prefix-dynamic-index-hit-def
by simp
have singleton:
  staged-query-prefix-candidate-opening-query-target-from-prefix
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i prefix prefix-state ⊆
    {opening-index
      (hd (((replicate rounds []) [i := trace-openings]) ! i))}
by (rule
  staged-query-prefix-candidate-opening-query-target-from-prefix-subset-canonical-trace-index)
have index (to-nat raw) =
  opening-index
    (hd (((replicate rounds []) [i := trace-openings]) ! i))
using idx-in singleton by auto
then show ?thesis
using i-bound by simp
qed

```

lemma checked-staged-security-with-query-prefix-fixed-header-component-canonical-indexD:

assumes i-bound: $i < \text{rounds}$

and hit:

checked-staged-security-with-query-prefix-fixed-header-component
 $E \text{ trace-openings composition-openings } i$
 $(\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data}, \text{attacker-state}), \text{result}), \text{final-state}))$

shows index (to-nat raw) = opening-index (hd trace-openings)

proof –

from checked-staged-security-with-query-prefix-fixed-header-componentE[OF hit]

have target:

index (to-nat raw) ∈
 staged-query-prefix-candidate-opening-query-target-from-prefix
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])
 i prefix prefix-state

by blast

have singleton:

staged-query-prefix-candidate-opening-query-target-from-prefix
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])
 i prefix prefix-state ⊆
 {opening-index
 (hd (((replicate rounds []) [i := trace-openings]) ! i))}

by (rule

```

    staged-query-prefix-candidate-opening-query-target-from-prefix-subset-canonical-trace-index)
have index (to-nat raw) =
    opening-index
    (hd (((replicate rounds []) [i := trace-openings]) ! i))
using target singleton by auto
then show ?thesis
    using i-bound by simp
qed

```

lemma *checked-staged-security-with-query-prefix-fixed-header-component-trace-openings-nonempty*:

```

assumes i-bound:  $i < \text{rounds}$ 
and hit:
    checked-staged-security-with-query-prefix-fixed-header-component
    E trace-openings composition-openings i
    (Some (((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
shows trace-openings  $\neq []$ 
proof –
from checked-staged-security-with-query-prefix-fixed-header-componentE[OF hit]
have target:
    index (to-nat raw)  $\in$ 
    staged-query-prefix-candidate-opening-query-target-from-prefix
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i prefix prefix-state
by blast
then have round:
    partial-query-round-consistent
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    (sqp-alphas prefix) i (index (to-nat raw))
unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
    partial-query-success-indices-at-def
by simp
have trace-indices:
    map opening-index
    (((replicate rounds []) [i := trace-openings]) ! i) =
    powers-scaled (index (to-nat raw))
using round unfolding partial-query-round-consistent-def by blast
have powers-scaled (index (to-nat raw))  $\neq []$ 
unfolding powers-scaled-def using powers-pos by simp
then have ((replicate rounds []) [i := trace-openings]) ! i  $\neq []$ 
using trace-indices by auto
then show ?thesis
    using i-bound by simp
qed

```

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-canonical-query-indexD*:

assumes i-bound: $i < \text{rounds}$

and *hit*:

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))

shows

index (to-nat raw) ∈
query-header-supported-partial-opening-canonical-query-indices
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)

proof –

from *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitE*
[OF hit]

have *witness*:

(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)

and *target*:

index (to-nat raw) ∈
staged-query-prefix-candidate-opening-query-target-from-prefix
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i prefix prefix-state

by *blast+*

have *idx-sample*: *index (to-nat raw) ∈ query-sample-space*

using *target staged-query-prefix-candidate-opening-query-target-from-prefix-subset*

by *blast*

have *round*:

partial-query-round-consistent
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
(sqp-alphas prefix) i (index (to-nat raw))

using *target*

unfolding *staged-query-prefix-candidate-opening-query-target-from-prefix-def*


```

    partial-query-success-indices-at-def
  by simp
have trace-indices:
  map opening-index
  (((replicate rounds []) [i := trace-openings]) ! i) =
  powers-scaled (index (to-nat raw))
  using round unfolding partial-query-round-consistent-def by blast
have trace-nonempty: trace-openings ≠ []
proof -
  have powers-scaled (index (to-nat raw)) ≠ []
  unfolding powers-scaled-def using powers-pos by simp
  then have ((replicate rounds []) [i := trace-openings]) ! i ≠ []
  using trace-indices by auto
  then show ?thesis
  using i-bound by simp
qed
have singleton:
  staged-query-prefix-candidate-opening-query-target-from-prefix
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  i prefix prefix-state ⊆
  {opening-index
   (hd (((replicate rounds []) [i := trace-openings]) ! i))}
  by (rule
    staged-query-prefix-candidate-opening-query-target-from-prefix-subset-canonical-trace-index)
have idx-eq:
  index (to-nat raw) = opening-index (hd trace-openings)
  using target singleton i-bound by auto
have idx-canonical:
  index (to-nat raw) ∈
  query-header-supported-partial-opening-canonical-indices
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  unfolding query-header-supported-partial-opening-canonical-indices-def
  using witness trace-nonempty idx-eq by blast
show ?thesis
  unfolding query-header-supported-partial-opening-canonical-query-indices-def
  using idx-sample idx-canonical by simp
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-canonical-query-indexD:
  assumes i-bound: i < rounds

```

and *hit*:

checked-staged-security-with-query-prefix-fixed-header-component
E trace-openings composition-openings i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))

shows

index (to-nat raw) ∈
query-header-supported-partial-opening-canonical-query-indices
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)

proof —

have *header*:

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))

using *hit*

unfolding *checked-staged-security-with-query-prefix-fixed-header-component-def*

by *simp*

show *?thesis*

by *(rule*
checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-canonical-query-indexD
[OF i-bound header])

qed

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitI-from-dynamic-hit*:

assumes *witness*:

(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)

and *target-hit*:

checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target-from-prefix
((replicate rounds []) [i := trace-openings])

```

      ((replicate rounds []) [i := composition-openings])
    i)
  (Some (((prefix, prefix-state), raw), raw-state))
shows
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
  trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof –
from query-header-supported-partial-opening-witnessesE[OF witness]
have comp-nonempty: staged-composition-fri-roots data ≠ []
by blast
have actual:
  checked-staged-security-with-actual-query-prefix-candidate-opening-hit
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
using target-hit
unfolding checked-staged-security-with-actual-query-prefix-candidate-opening-hit-def
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
by simp
show ?thesis
unfolding
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-def
using comp-nonempty witness actual by simp
qed

lemma checked-staged-security-with-query-prefix-fixed-header-componentI-from-dynamic-hit:
assumes event:
  E (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
and witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
and target-hit:
  checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-opening-query-target-from-prefix
    ((replicate rounds []) [i := trace-openings])

```

```

      ((replicate rounds []) [i := composition-openings])
    i)
  (Some (((prefix, prefix-state), raw), raw-state))
shows
  checked-staged-security-with-query-prefix-fixed-header-component
    E trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
unfolding checked-staged-security-with-query-prefix-fixed-header-component-def
by (intro conjI event
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitI-from-dynamic-hit
    [OF witness target-hit])

lemma checked-staged-security-with-query-prefix-fixed-header-component-bound-from-budgets:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-fixed-header-component
      E trace-openings composition-openings i)
  adversary-initial-state ≤
    staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      (1::prob) / nnreal (card query-sample-space)
proof –
  have raw-bound: query-index-raw-preimage-bound
  by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
  show ?thesis
  by (rule
    checked-staged-security-with-query-prefix-fixed-header-component-bound
      [OF raw-bound wf controlled i-bound])
qed

lemma checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-bound-from-budgets:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
      trace-openings composition-openings i)
  adversary-initial-state ≤
    staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      (1::prob) / nnreal (card query-sample-space)
proof –

```

```

have raw-bound: query-index-raw-preimage-bound
  by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
show ?thesis
  by (rule
    checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-bound
    [OF raw-bound wf controlled i-bound])
qed

lemma checked-staged-query-prefix-candidate-opening-target-from-prefix-hit-bound-from-budgets:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i < \text{rounds}$ 
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-low: composition-table-low-degree maxDegree composition-table
and not-all:
   $\bigwedge \text{prefix prefix-state.}$ 
  Some ((prefix, prefix-state), prefix-state)  $\in$ 
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
      adversary-initial-state)  $\implies$ 
   $\neg$  all-queries-consistent trace-table composition-table
    (sqp-alphas prefix)
shows
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-opening-query-target-from-prefix
        trace-openings composition-openings i))
  adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) + query-error-bound
proof -
have raw-bound: query-index-raw-preimage-bound
  by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
show ?thesis
  by (rule
    checked-staged-query-prefix-candidate-opening-target-from-prefix-hit-bound
    [OF raw-bound wf controlled - trace-candidate comp-candidate
      trace-low comp-low not-all])
    (use i-bound in simp)
qed

end

end

```

```

theory Soundness-Partial-Merkle-Cross
  imports Soundness-Partial-Merkle
begin

```

Cross-state Merkle facts for partial authenticated openings.

The broad query-header candidate relation can be supported by different verifier executions and therefore different final hash-map states. These lemmas isolate the reusable binding fact available in the current model: two authenticated openings for the same root, length, and index cannot carry different values across compatible cleanly-merged states.

```

context soundness
begin

```

definition *partial-openings-cross-cover-all-indices*

```

:: 'f authenticated-opening list list  $\Rightarrow$ 
   'f authenticated-opening list list  $\Rightarrow$  bool

```

where

```

partial-openings-cross-cover-all-indices openings openings'  $\longleftrightarrow$ 
  ( $\forall j < \text{scale} * \text{clength}$ .
     $\exists i < \text{rounds}$ .  $\exists i' < \text{rounds}$ .
       $\exists \text{opn} \in \text{set } (\text{openings} ! i)$ .
         $\exists \text{opn}' \in \text{set } (\text{openings}' ! i')$ .
           $\text{opening-index } \text{opn} = j \wedge$ 
             $\text{opening-index } \text{opn}' = j$ )

```

lemma *partial-openings-cross-cover-all-indicesD*:

assumes

partial-openings-cross-cover-all-indices *openings openings'*

and $j < \text{scale} * \text{clength}$

obtains $i \ i' \ \text{opn} \ \text{opn}'$ **where**

$i < \text{rounds}$

$i' < \text{rounds}$

$\text{opn} \in \text{set } (\text{openings} ! i)$

$\text{opn}' \in \text{set } (\text{openings}' ! i')$

$\text{opening-index } \text{opn} = j$

$\text{opening-index } \text{opn}' = j$

using *assms*

unfolding *partial-openings-cross-cover-all-indices-def* **by** *blast*

lemma *partial-openings-cross-cover-all-indices-sym*:

assumes *partial-openings-cross-cover-all-indices* *openings openings'*

shows *partial-openings-cross-cover-all-indices* *openings'* *openings*

using *assms*

unfolding *partial-openings-cross-cover-all-indices-def* **by** *blast*

lemma *not-partial-openings-cross-cover-all-indicesE*:

assumes $\neg$ *partial-openings-cross-cover-all-indices openings openings'*

obtains j where

$j < \text{scale} * \text{clength}$

$\bigwedge i \ i' \ \text{opn} \ \text{opn}'. \ i < \text{rounds} \implies i' < \text{rounds} \implies$

$\text{opn} \in \text{set} (\text{openings} ! i) \implies$

$\text{opn}' \in \text{set} (\text{openings}' ! i') \implies$

$\text{opening-index} \ \text{opn} = j \implies \text{opening-index} \ \text{opn}' \neq j$

using *assms*

unfolding *partial-openings-cross-cover-all-indices-def* **by** *blast*

lemma *partial-openings-cross-cover-all-indices-imp-left-cover:*

assumes *cover:*

partial-openings-cross-cover-all-indices openings openings'

and *j-bound:* $j < \text{scale} * \text{clength}$

shows $\exists i < \text{rounds}. \exists \text{opn} \in \text{set} (\text{openings} ! i).$

$\text{opening-index} \ \text{opn} = j$

using *partial-openings-cross-cover-all-indicesD[OF cover j-bound]* **by** *blast*

lemma *partial-openings-cross-cover-all-indices-imp-right-cover:*

assumes *cover:*

partial-openings-cross-cover-all-indices openings openings'

and *j-bound:* $j < \text{scale} * \text{clength}$

shows $\exists i < \text{rounds}. \exists \text{opn} \in \text{set} (\text{openings}' ! i).$

$\text{opening-index} \ \text{opn} = j$

using *partial-openings-cross-cover-all-indicesD[OF cover j-bound]* **by** *blast*

lemma *cross-sparse-openings-obtain-unopened-index:*

assumes *rounds-len:* $\text{length} \ \text{openings} = \text{rounds}$

and *sparse:* $\text{length} \ (\text{List.concat} \ \text{openings}) < \text{scale} * \text{clength}$

obtains j where

$j < \text{scale} * \text{clength}$

$\bigwedge i \ \text{opn}. \ i < \text{rounds} \implies \text{opn} \in \text{set} (\text{openings} ! i) \implies$

$\text{opening-index} \ \text{opn} \neq j$

proof –

let $?N = \text{scale} * \text{clength}$

let $?I = \text{opening-index} \ ' \ \text{set} \ (\text{List.concat} \ \text{openings})$

have *finite-I:* *finite* $?I$

by *simp*

have *card-I-le:* $\text{card} \ ?I \leq \text{length} \ (\text{List.concat} \ \text{openings})$

proof –

have $\text{card} \ ?I \leq \text{card} \ (\text{set} \ (\text{List.concat} \ \text{openings}))$

by (*rule card-image-le*) *simp*

also have $\dots \leq \text{length} \ (\text{List.concat} \ \text{openings})$

by (*rule card-length*)

finally show *?thesis* .

qed

have *not-all:* $\neg \{0..<?N\} \subseteq ?I$

proof

assume *subset:* $\{0..<?N\} \subseteq ?I$

```

have ?N = card {0..?N}
  by simp
also have ... ≤ card ?I
  by (rule card-mono[OF finite-I subset])
also have ... ≤ length (List.concat openings)
  by (rule card-I-le)
finally show False
  using sparse by simp
qed
from not-all obtain j where j-in: j ∈ {0..?N} and j-notin: j ∉ ?I
  by blast
then have j-bound: j < ?N
  by simp
have unopened:
  ∧ i opn. i < rounds ⇒ opn ∈ set (openings ! i) ⇒
    opening-index opn ≠ j
proof -
  fix i opn
  assume i-bound: i < rounds
    and opn-in: opn ∈ set (openings ! i)
  have openings ! i ∈ set openings
    using i-bound rounds-len by simp
  then have opn ∈ set (List.concat openings)
    using opn-in by auto
  then show opening-index opn ≠ j
    using j-notin by auto
qed
show ?thesis
  by (rule that[OF j-bound unopened])
qed

lemma partial-openings-cross-cover-all-indices-impossible-if-left-sparse:
  assumes rounds-len: length openings = rounds
    and sparse: length (List.concat openings) < scale * clength
  shows ¬ partial-openings-cross-cover-all-indices openings openings'
proof
  assume cover:
    partial-openings-cross-cover-all-indices openings openings'
  from cross-sparse-openings-obtain-unopened-index[OF rounds-len sparse]
  obtain j where j-bound: j < scale * clength
    and unopened:
      ∧ i opn. i < rounds ⇒ opn ∈ set (openings ! i) ⇒
        opening-index opn ≠ j
  by blast
  from partial-openings-cross-cover-all-indices-imp-left-cover
    [OF cover j-bound]
  obtain i opn where i-bound: i < rounds
    and opn-in: opn ∈ set (openings ! i)
    and opn-idx: opening-index opn = j

```



```

    by blast
  show False
    using unopened[OF i-bound opn-in] opn-idx by simp
qed

lemma partial-openings-cross-cover-all-indices-impossible-if-right-sparse:
  assumes rounds-len: length openings' = rounds
    and sparse: length (List.concat openings') < scale * clength
  shows  $\neg$  partial-openings-cross-cover-all-indices openings openings'
proof -
  have not-sym:
     $\neg$  partial-openings-cross-cover-all-indices openings' openings
  by (rule partial-openings-cross-cover-all-indices-impossible-if-left-sparse
    [OF rounds-len sparse])
  show ?thesis
proof
  assume cover:
    partial-openings-cross-cover-all-indices openings openings'
  have partial-openings-cross-cover-all-indices openings' openings
    by (rule partial-openings-cross-cover-all-indices-sym[OF cover])
  then show False
    using not-sym by contradiction
qed
qed

lemma merkle-path-bound-merkle-mono:
  assumes bound: merkle-path-bound rt len idx v path s
    and ext: merkle-hash-extends s t
  shows merkle-path-bound rt len idx v path t
  using bound
proof (induction path arbitrary: rt len idx)
  case Nil
  then show ?case
    using ext unfolding merkle-hash-extends-def by simp
next
  case (Cons sibling path)
  show ?case
  proof (cases idx < len div 2)
    case True
    then obtain child where
      child:
        merkle-path-bound child (len div 2) idx v path s
    and lookup:
      fmllookup (HashMap s) (MerkleNode child sibling) = Some rt
    using Cons.premis by auto
  have child-t:
    merkle-path-bound child (len div 2) idx v path t
  by (rule Cons.IH[OF child])
  have lookup-t:

```

```

    fmlookup (HashMap t) (MerkleNode child sibling) = Some rt
    using ext lookup unfolding merkle-hash-extends-def by simp
show ?thesis
    using True child-t lookup-t
    by (auto intro!: exI[of - child])
next
case False
then obtain child where
  child:
    merkle-path-bound child (len div 2) (idx - len div 2)
    v path s
  and lookup:
    fmlookup (HashMap s) (MerkleNode sibling child) = Some rt
    using Cons.premis by auto
have child-t:
  merkle-path-bound child (len div 2) (idx - len div 2)
  v path t
  by (rule Cons.IH[OF child])
have lookup-t:
  fmlookup (HashMap t) (MerkleNode sibling child) = Some rt
  using ext lookup unfolding merkle-hash-extends-def by simp
show ?thesis
  using False child-t lookup-t
  by (auto intro!: exI[of - child])
qed
qed

```

```

lemma authenticated-opening-in-merkle-mono:
  assumes opn-auth: authenticated-opening-in s opn
  and ext: merkle-hash-extends s t
  shows authenticated-opening-in t opn
  using opn-auth merkle-path-bound-merkle-mono[OF - ext]
  unfolding authenticated-opening-in-def by blast

```

```

lemma inconsistent-authenticated-openings-imp-merkle-conflict-or-merge-collision:
  assumes first: authenticated-opening-in s opening
  and second: authenticated-opening-in t opening'
  and same-root: opening-root opening = opening-root opening'
  and same-length: opening-length opening = opening-length opening'
  and same-index: opening-index opening = opening-index opening'
  and different-value: opening-value opening ≠ opening-value opening'
  shows
    merkle-hash-value-conflict s t ∨
    hash-map-output-collision (merkle-hash-state-merge s t)
proof (cases merkle-hash-value-conflict s t)
  case True
  then show ?thesis by simp
next
case no-conflict: False

```

```

have compatible: merkle-hash-maps-compatible s t
  using no-conflict
  unfolding merkle-hash-maps-compatible-iff-no-value-conflict .
let ?u = merkle-hash-state-merge s t
have s-ext: merkle-hash-extends s ?u
  by (rule merkle-hash-state-merge-extends-left-if-compatible
    [OF compatible])
have t-ext: merkle-hash-extends t ?u
  by (rule merkle-hash-state-merge-extends-right)
have first-u: authenticated-opening-in ?u opening
  by (rule authenticated-opening-in-merkle-mono[OF first s-ext])
have second-u: authenticated-opening-in ?u opening'
  by (rule authenticated-opening-in-merkle-mono[OF second t-ext])
have collision: hash-map-output-collision ?u
  by (rule inconsistent-authenticated-openings-imp-hash-collision
    [OF first-u second-u same-root same-length same-index
      different-value])
then show ?thesis by simp
qed

lemma authenticated-openings-values-eq-if-no-merkle-conflict-clean-merge:
  assumes first: authenticated-opening-in s opening
    and second: authenticated-opening-in t opening'
    and same-root: opening-root opening = opening-root opening'
    and same-length: opening-length opening = opening-length opening'
    and same-index: opening-index opening = opening-index opening'
    and no-conflict:  $\neg$  merkle-hash-value-conflict s t
    and clean-merge:
       $\neg$  hash-map-output-collision (merkle-hash-state-merge s t)
  shows opening-value opening = opening-value opening'
proof (rule ccontr)
  assume different:
    opening-value opening  $\neq$  opening-value opening'
  have
    merkle-hash-value-conflict s t  $\vee$ 
    hash-map-output-collision (merkle-hash-state-merge s t)
  by (rule
    inconsistent-authenticated-openings-imp-merkle-conflict-or-merge-collision
    [OF first second same-root same-length same-index different])
  then show False
    using no-conflict clean-merge by blast
qed

lemma partial-authenticated-tables-values-eq-if-no-merkle-conflict-clean-merge:
  assumes table:
    partial-authenticated-table rt len openings s
  and table':
    partial-authenticated-table rt len openings' t
  and opn-in: opn  $\in$  set openings

```

and *opn'-in*: $opn' \in \text{set openings'}$
and *same-index*: $\text{opening-index } opn = \text{opening-index } opn'$
and *no-conflict*: $\neg \text{merkle-hash-value-conflict } s \ t$
and *clean-merge*:
 $\neg \text{hash-map-output-collision } (\text{merkle-hash-state-merge } s \ t)$
shows $\text{opening-value } opn = \text{opening-value } opn'$
proof –
have *first*: *authenticated-opening-in* $s \ opn$
using *table* *opn-in* **unfolding** *partial-authenticated-table-def* **by** *blast*
have *second*: *authenticated-opening-in* $t \ opn'$
using *table'* *opn'-in* **unfolding** *partial-authenticated-table-def* **by** *blast*
have *root*: $\text{opening-root } opn = \text{rt}$
using *table* *opn-in* **unfolding** *partial-authenticated-table-def* **by** *blast*
have *root'*: $\text{opening-root } opn' = \text{rt}$
using *table'* *opn'-in* **unfolding** *partial-authenticated-table-def* **by** *blast*
have *same-root*: $\text{opening-root } opn = \text{opening-root } opn'$
using *root* *root'* **by** *simp*
have *len*: $\text{opening-length } opn = \text{len}$
using *table* *opn-in* **unfolding** *partial-authenticated-table-def* **by** *blast*
have *len'*: $\text{opening-length } opn' = \text{len}$
using *table'* *opn'-in* **unfolding** *partial-authenticated-table-def* **by** *blast*
have *same-length*: $\text{opening-length } opn = \text{opening-length } opn'$
using *len* *len'* **by** *simp*
show *?thesis*
by (*rule* *authenticated-openings-values-eq-if-no-merkle-conflict-clean-merge*
 $[\text{OF } \text{first } \text{second } \text{same-root } \text{same-length } \text{same-index } \text{no-conflict}$
 $\text{clean-merge}]$)
qed

lemma *partial-trace-openings-values-eq-if-no-merkle-conflict-clean-merge*:

assumes *partial*:

accepted-with-partial-trace-openings s

$(\text{Some } (\text{result}, \text{final-state})) \text{ fr } \text{query-idxs } \text{trace-openings}$

and *partial'*:

accepted-with-partial-trace-openings s'

$(\text{Some } (\text{result}', \text{final-state}')) \text{ fr } \text{query-idxs}' \text{ trace-openings}'$

and *i-bound*: $i < \text{rounds}$

and *i'-bound*: $i' < \text{rounds}$

and *opn-in*: $opn \in \text{set } (\text{trace-openings } ! i)$

and *opn'-in*: $opn' \in \text{set } (\text{trace-openings}' ! i')$

and *same-index*: $\text{opening-index } opn = \text{opening-index } opn'$

and *no-conflict*:

$\neg \text{merkle-hash-value-conflict } \text{final-state } \text{final-state}'$

and *clean-merge*:

$\neg \text{hash-map-output-collision}$

$(\text{merkle-hash-state-merge } \text{final-state } \text{final-state}')$

shows $\text{opening-value } opn = \text{opening-value } opn'$

proof –

have *table*:

```

partial-authenticated-table fr (scale * clength)
  (trace-openings ! i) final-state
using partial i-bound
unfolding accepted-with-partial-trace-openings-def by blast
have table':
partial-authenticated-table fr (scale * clength)
  (trace-openings' ! i') final-state'
using partial' i'-bound
unfolding accepted-with-partial-trace-openings-def by blast
show ?thesis
by (rule
  partial-authenticated-tables-values-eq-if-no-merkle-conflict-clean-merge
  [OF table table' opn-in opn'-in same-index no-conflict
    clean-merge])
qed

lemma partial-composition-openings-values-eq-if-no-merkle-conflict-clean-merge:
assumes partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) composition-root query-idxs
  composition-openings
and partial':
  accepted-with-partial-composition-openings s'
  (Some (result', final-state')) composition-root query-idxs'
  composition-openings'
and i-bound: i < rounds
and i'-bound: i' < rounds
and opn-in: opn ∈ set (composition-openings ! i)
and opn'-in: opn' ∈ set (composition-openings' ! i')
and same-index: opening-index opn = opening-index opn'
and no-conflict:
  ¬ merkle-hash-value-conflict final-state final-state'
and clean-merge:
  ¬ hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')
shows opening-value opn = opening-value opn'
proof –
have table:
partial-authenticated-table composition-root (scale * clength)
  (composition-openings ! i) final-state
using partial i-bound
unfolding accepted-with-partial-composition-openings-def by blast
have table':
partial-authenticated-table composition-root (scale * clength)
  (composition-openings' ! i') final-state'
using partial' i'-bound
unfolding accepted-with-partial-composition-openings-def by blast
show ?thesis
by (rule

```

partial-authenticated-tables-values-eq-if-no-merkle-conflict-clean-merge
 [OF table table' opn-in opn'-in same-index no-conflict
 clean-merge])

qed

lemma *partial-trace-table-candidates-eq-if-cross-openings-cover-clean-merge:*

assumes *partial:*

accepted-with-partial-trace-openings s
 (Some (result, final-state)) fr query-idxs trace-openings

and *partial':*

accepted-with-partial-trace-openings s'
 (Some (result', final-state')) fr query-idxs' trace-openings'

and *cand:* *partial-trace-table-candidate trace-table trace-openings*

and *cand':*

partial-trace-table-candidate trace-table' trace-openings'

and *cover:*

$\bigwedge j. j < \text{scale} * \text{clength} \implies$
 $\exists i < \text{rounds}. \exists i' < \text{rounds}.$
 $\exists \text{opn} \in \text{set } (\text{trace-openings} ! i).$
 $\exists \text{opn}' \in \text{set } (\text{trace-openings}' ! i').$
 $\text{opening-index } \text{opn} = j \wedge$
 $\text{opening-index } \text{opn}' = j$

and *no-conflict:*

$\neg \text{merkle-hash-value-conflict } \text{final-state } \text{final-state}'$

and *clean-merge:*

$\neg \text{hash-map-output-collision}$
 (merkle-hash-state-merge final-state final-state')

shows *trace-table = trace-table'*

proof (rule *nth-equalityI*)

show *length trace-table = length trace-table'*

using *cand cand' unfolding partial-trace-table-candidate-def by simp*

next

fix *j*

assume *j-bound: j < length trace-table*

have *len: length trace-table = scale * clength*

using *cand unfolding partial-trace-table-candidate-def by simp*

from *cover[OF j-bound[unfolded len]]*

obtain *i i' opn opn' where*

i-bound: i < rounds

and *i'-bound: i' < rounds*

and *opn-in: opn ∈ set (trace-openings ! i)*

and *opn'-in: opn' ∈ set (trace-openings' ! i')*

and *opn-idx: opening-index opn = j*

and *opn'-idx: opening-index opn' = j*

by *blast*

have *trace-val: trace-table ! j = opening-value opn*

using *partial-trace-table-candidateD(3)[OF cand i-bound opn-in]*
opn-idx by simp

have *trace'-val: trace-table' ! j = opening-value opn'*

```

using partial-trace-table-candidateD( $\exists$ )[OF cand' i'-bound opn'-in]
  opn'-idx by simp
have opening-values:
  opening-value opn = opening-value opn'
by (rule partial-trace-openings-values-eq-if-no-merkle-conflict-clean-merge
  [OF partial partial' i-bound i'-bound opn-in opn'-in - no-conflict
  clean-merge])
  (use opn-idx opn'-idx in simp)
show trace-table ! j = trace-table' ! j
  using trace-val trace'-val opening-values by simp
qed

```

lemma *partial-trace-table-candidates-eq-if-cross-cover-clean-merge*:

```

assumes partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr query-idxs trace-openings
and partial':
  accepted-with-partial-trace-openings s'
  (Some (result', final-state')) fr query-idxs' trace-openings'
and cand: partial-trace-table-candidate trace-table trace-openings
and cand':
  partial-trace-table-candidate trace-table' trace-openings'
and cover:
  partial-openings-cross-cover-all-indices trace-openings
  trace-openings'
and no-conflict:
   $\neg$  merkle-hash-value-conflict final-state final-state'
and clean-merge:
   $\neg$  hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')
shows trace-table = trace-table'

```

```

proof (rule
  partial-trace-table-candidates-eq-if-cross-openings-cover-clean-merge
  [OF partial partial' cand cand' - no-conflict clean-merge])
fix j
assume j < scale * clength
from partial-openings-cross-cover-all-indicesD[OF cover this]
obtain i i' opn opn' where
  i < rounds i' < rounds
  opn  $\in$  set (trace-openings ! i)
  opn'  $\in$  set (trace-openings' ! i')
  opening-index opn = j
  opening-index opn' = j
by blast
then show
   $\exists i < rounds. \exists i' < rounds.$ 
   $\exists opn \in \text{set } (trace\_openings ! i).$ 
   $\exists opn' \in \text{set } (trace\_openings' ! i').$ 
  opening-index opn = j  $\wedge$  opening-index opn' = j

```

by blast
qed

lemma *partial-composition-table-candidates-eq-if-cross-openings-cover-clean-merge:*

assumes *partial:*

accepted-with-partial-composition-openings s
(Some (result, final-state)) composition-root query-idxs
composition-openings

and *partial':*

accepted-with-partial-composition-openings s'
(Some (result', final-state')) composition-root query-idxs'
composition-openings'

and *cand:*

partial-composition-table-candidate composition-table
composition-openings

and *cand':*

partial-composition-table-candidate composition-table'
composition-openings'

and *cover:*

$\bigwedge j. j < \text{scale} * \text{clength} \implies$
 $\exists i < \text{rounds}. \exists i' < \text{rounds}.$
 $\exists \text{opn} \in \text{set } (\text{composition-openings} ! i).$
 $\exists \text{opn}' \in \text{set } (\text{composition-openings}' ! i').$
 $\text{opening-index } \text{opn} = j \wedge$
 $\text{opening-index } \text{opn}' = j$

and *no-conflict:*

$\neg \text{merkle-hash-value-conflict } \text{final-state } \text{final-state}'$

and *clean-merge:*

$\neg \text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge } \text{final-state } \text{final-state}')$

shows *composition-table = composition-table'*

proof (rule *nth-equalityI*)

show *length composition-table = length composition-table'*

using *cand cand'*

unfolding *partial-composition-table-candidate-def* **by** *simp*

next

fix *j*

assume *j-bound: j < length composition-table*

have *len: length composition-table = scale * clength*

using *cand* **unfolding** *partial-composition-table-candidate-def* **by** *simp*

from *cover[OF j-bound[unfolded len]]*

obtain *i i' opn opn'* **where**

i-bound: i < rounds

and *i'-bound: i' < rounds*

and *opn-in: opn ∈ set (composition-openings ! i)*

and *opn'-in: opn' ∈ set (composition-openings' ! i')*

and *opn-idx: opening-index opn = j*

and *opn'-idx: opening-index opn' = j*

by *blast*


```

have composition-val:
  composition-table ! j = opening-value opn
  using partial-composition-table-candidateD( $\beta$ )[OF cand i-bound opn-in]
    opn-idx by simp
have composition'-val:
  composition-table' ! j = opening-value opn'
  using partial-composition-table-candidateD( $\beta$ )[OF cand' i'-bound opn'-in]
    opn'-idx by simp
have opening-values:
  opening-value opn = opening-value opn'
  by (rule
    partial-composition-openings-values-eq-if-no-merkle-conflict-clean-merge
    [OF partial partial' i-bound i'-bound opn-in opn'-in - no-conflict
      clean-merge])
    (use opn-idx opn'-idx in simp)
show composition-table ! j = composition-table' ! j
  using composition-val composition'-val opening-values by simp
qed

```

lemma *partial-composition-table-candidates-eq-if-cross-cover-clean-merge*:

```

assumes partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) composition-root query-idxs
  composition-openings
and partial':
  accepted-with-partial-composition-openings s'
  (Some (result', final-state')) composition-root query-idxs'
  composition-openings'
and cand:
  partial-composition-table-candidate composition-table
  composition-openings
and cand':
  partial-composition-table-candidate composition-table'
  composition-openings'
and cover:
  partial-openings-cross-cover-all-indices composition-openings
  composition-openings'
and no-conflict:
   $\neg$  merkle-hash-value-conflict final-state final-state'
and clean-merge:
   $\neg$  hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')
shows composition-table = composition-table'
proof (rule
  partial-composition-table-candidates-eq-if-cross-openings-cover-clean-merge
  [OF partial partial' cand cand' - no-conflict clean-merge])
fix j
assume j < scale * clength
from partial-openings-cross-cover-all-indicesD[OF cover this]

```

```

obtain  $i\ i'\ \text{opn}\ \text{opn}'$  where
   $i < \text{rounds}\ i' < \text{rounds}$ 
   $\text{opn} \in \text{set}\ (\text{composition-openings}\ !\ i)$ 
   $\text{opn}' \in \text{set}\ (\text{composition-openings}'\ !\ i')$ 
   $\text{opening-index}\ \text{opn} = j$ 
   $\text{opening-index}\ \text{opn}' = j$ 
by blast
then show
   $\exists i < \text{rounds}.\ \exists i' < \text{rounds}.$ 
   $\exists \text{opn} \in \text{set}\ (\text{composition-openings}\ !\ i).$ 
   $\exists \text{opn}' \in \text{set}\ (\text{composition-openings}'\ !\ i').$ 
   $\text{opening-index}\ \text{opn} = j \wedge \text{opening-index}\ \text{opn}' = j$ 
by blast
qed

end

end

```

theory *Soundness-Query-Header-Cross-Candidate*

imports

Soundness-Reductions-Composition

Soundness-Partial-Merkle-Cross

begin

Witnessed query-header partial candidates.

The public query-header candidate set hides the authenticated opening witnesses behind existentials. This layer exposes those witnesses so that the cross-state partial-Merkle binding lemmas can be applied before reducing back to the old candidate-set interface.

context *soundness*

begin

definition *query-header-supported-partial-table-candidate-witness*

```

:: ( $f, 'a$ ) protocol-channel-scheme  $\Rightarrow$ 
   $f \Rightarrow 'f\ \text{list} \Rightarrow 'f \Rightarrow 'f\ \text{list} \Rightarrow 'f \Rightarrow$ 
   $'f\ \text{list} \Rightarrow 'f \Rightarrow 'f\ \text{list} \Rightarrow 'f\ \text{list} \Rightarrow$ 
  ( $f, 'a$ ) protocol-channel-scheme  $\Rightarrow$ 
   $'f\ \text{authenticated-opening}\ \text{list}\ \text{list} \Rightarrow$ 
   $'f\ \text{authenticated-opening}\ \text{list}\ \text{list} \Rightarrow \text{bool}$ 

```

where

```

query-header-supported-partial-table-candidate-witness  $s\ \text{fr}\ f\text{-fri-roots}$ 
   $f\text{-final}\ \text{as}\ dg\ \text{composition-fri-roots}\ \text{final}\ \text{trace-table}$ 
   $\text{composition-table}\ \text{final-state}\ \text{trace-openings}\ \text{composition-openings}$ 
 $\longleftrightarrow$ 
   $\text{composition-fri-roots} \neq [] \wedge$ 
   $(\exists \text{result}\ \text{trace-query-idxs}\ \text{composition-query-idxs}\ \text{rest}.$ 
     $\text{Some}\ (\text{result}, \text{final-state}) \in \text{set-dist}\ (\text{execute}\ \text{verify-monad}\ s) \wedge$ 
     $\text{accepted-with-partial-trace-openings}\ s$ 

```

(Some (result, final-state)) fr trace-query-idxs
 trace-openings $\wedge$
 partial-trace-table-candidate trace-table trace-openings $\wedge$
 accepted-with-partial-composition-openings s
 (Some (result, final-state)) (hd composition-fri-roots)
 composition-query-idxs composition-openings $\wedge$
 partial-composition-table-candidate composition-table
 composition-openings $\wedge$
 verifier-header-transcript s fr f-fri-roots f-final as dg
 composition-fri-roots final rest)

lemma query-header-supported-partial-table-candidate-witness-imp-state:

assumes

query-header-supported-partial-table-candidate-witness s fr f-fri-roots
 f-final as dg composition-fri-roots final trace-table
 composition-table final-state trace-openings composition-openings

shows

query-header-supported-partial-table-candidate-state s fr f-fri-roots
 f-final as dg composition-fri-roots final trace-table
 composition-table final-state

using assms

unfolding query-header-supported-partial-table-candidate-witness-def

query-header-supported-partial-table-candidate-state-def

by blast

lemma query-header-supported-partial-table-candidate-state-obtain-witness:

assumes

query-header-supported-partial-table-candidate-state s fr f-fri-roots
 f-final as dg composition-fri-roots final trace-table
 composition-table final-state

obtains trace-openings composition-openings **where**

query-header-supported-partial-table-candidate-witness s fr f-fri-roots
 f-final as dg composition-fri-roots final trace-table
 composition-table final-state trace-openings composition-openings

using assms

unfolding query-header-supported-partial-table-candidate-witness-def

query-header-supported-partial-table-candidate-state-def

by blast

lemma query-header-supported-partial-table-candidate-witness-imp-candidate:

assumes

query-header-supported-partial-table-candidate-witness s fr f-fri-roots
 f-final as dg composition-fri-roots final trace-table
 composition-table final-state trace-openings composition-openings

shows

(trace-table, composition-table) $\in$
 query-header-supported-partial-table-candidates s fr f-fri-roots
 f-final as dg composition-fri-roots final

using query-header-supported-partial-table-candidate-witness-imp-state

[*OF assms*]
query-header-supported-partial-table-candidates-iff-state
by *blast*

lemma *query-header-supported-partial-table-candidates-obtain-witness:*

assumes

(*trace-table*, *composition-table*) ∈
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final

obtains *final-state trace-openings composition-openings* **where**

query-header-supported-partial-table-candidate-witness s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state trace-openings composition-openings

proof –

from *assms* **obtain** *final-state* **where** *state:*

query-header-supported-partial-table-candidate-state s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state

using *query-header-supported-partial-table-candidates-iff-state* **by** *blast*

from *query-header-supported-partial-table-candidate-state-obtain-witness*

[*OF state*]

obtain *trace-openings composition-openings* **where** *witness:*

query-header-supported-partial-table-candidate-witness s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state trace-openings composition-openings

by *blast*

show *?thesis*

by (*rule that[OF witness]*)

qed

lemma *query-header-supported-partial-table-candidate-witness-shapes:*

assumes *witness:*

query-header-supported-partial-table-candidate-witness s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state trace-openings composition-openings

shows

partial-trace-table-candidate trace-table trace-openings
partial-composition-table-candidate composition-table composition-openings

using *witness*

unfolding *query-header-supported-partial-table-candidate-witness-def*

by *blast+*

lemma *query-header-supported-partial-table-candidate-witness-eq-if-cross-cover-clean-merge:*

assumes *witness:*

query-header-supported-partial-table-candidate-witness s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state trace-openings composition-openings

and *witness':*

query-header-supported-partial-table-candidate-witness s fr f-fri-roots

f-final as dg composition-fri-roots final trace-table'
composition-table' final-state' trace-openings' composition-openings'
and *trace-cover:*
partial-openings-cross-cover-all-indices trace-openings
trace-openings'
and *composition-cover:*
partial-openings-cross-cover-all-indices composition-openings
composition-openings'
and *no-conflict:*
 $\neg$ *merkle-hash-value-conflict final-state final-state'*
and *clean-merge:*
 $\neg$ *hash-map-output-collision*
(merkle-hash-state-merge final-state final-state')
shows *trace-table = trace-table' $\wedge$ composition-table = composition-table'*
proof –
from *witness* **obtain** *result trace-query-idxs composition-query-idxs rest* **where**
trace-partial:
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *comp-partial:*
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
and *trace-cand:*
partial-trace-table-candidate trace-table trace-openings
and *comp-cand:*
partial-composition-table-candidate composition-table
composition-openings
unfolding *query-header-supported-partial-table-candidate-witness-def*
by *blast*
from *witness'* **obtain** *result' trace-query-idxs' composition-query-idxs' rest'*
where *trace-partial':*
accepted-with-partial-trace-openings s
(Some (result', final-state')) fr trace-query-idxs'
trace-openings'
and *comp-partial':*
accepted-with-partial-composition-openings s
(Some (result', final-state')) (hd composition-fri-roots)
composition-query-idxs' composition-openings'
and *trace-cand':*
partial-trace-table-candidate trace-table' trace-openings'
and *comp-cand':*
partial-composition-table-candidate composition-table'
composition-openings'
unfolding *query-header-supported-partial-table-candidate-witness-def*
by *blast*
have *trace-eq: trace-table = trace-table'*
by *(rule partial-trace-table-candidates-eq-if-cross-cover-clean-merge*
[OF trace-partial trace-partial' trace-cand trace-cand' trace-cover

```

    no-conflict clean-merge])
have comp-eq: composition-table = composition-table'
by (rule partial-composition-table-candidates-eq-if-cross-cover-clean-merge
    [OF comp-partial comp-partial' comp-cand comp-cand'
    composition-cover no-conflict clean-merge])
show ?thesis
    using trace-eq comp-eq by simp
qed

lemma query-header-supported-partial-table-candidates-subset-singleton-if-pairwise-cross-cover-clean-merge:
assumes pairwise:
     $\bigwedge$  trace-table composition-table final-state trace-openings
        composition-openings trace-table' composition-table' final-state'
        trace-openings' composition-openings'.
    query-header-supported-partial-table-candidate-witness s fr
        f-fri-roots f-final as dg composition-fri-roots final trace-table
        composition-table final-state trace-openings composition-openings
     $\implies$ 
    query-header-supported-partial-table-candidate-witness s fr
        f-fri-roots f-final as dg composition-fri-roots final trace-table'
        composition-table' final-state' trace-openings' composition-openings'
     $\implies$ 
    partial-openings-cross-cover-all-indices trace-openings
        trace-openings'  $\wedge$ 
    partial-openings-cross-cover-all-indices composition-openings
        composition-openings'  $\wedge$ 
     $\neg$  merkle-hash-value-conflict final-state final-state'  $\wedge$ 
     $\neg$  hash-map-output-collision
        (merkle-hash-state-merge final-state final-state')
shows
     $\exists$  trace-table composition-table.
    query-header-supported-partial-table-candidates s fr f-fri-roots
        f-final as dg composition-fri-roots final  $\subseteq$ 
    {(trace-table, composition-table)}
proof (cases
    query-header-supported-partial-table-candidates s fr f-fri-roots
        f-final as dg composition-fri-roots final = {})
case True
then show ?thesis
    by (intro exI[of -] exI[of -]) simp
next
case False
then obtain pair0 where pair0-in:
    pair0  $\in$ 
    query-header-supported-partial-table-candidates s fr f-fri-roots
        f-final as dg composition-fri-roots final
by blast
then obtain trace-table0 composition-table0 where pair0-eq:
    pair0 = (trace-table0, composition-table0)

```

```

    by (cases pair0)
  have cand0:
    (trace-table0, composition-table0) ∈
      query-header-supported-partial-table-candidates s fr f-fri-roots
      f-final as dg composition-fri-roots final
    using pair0-in unfolding pair0-eq .
  from query-header-supported-partial-table-candidates-obtain-witness[OF cand0]
  obtain final-state0 trace-openings0 composition-openings0 where witness0:
    query-header-supported-partial-table-candidate-witness s fr f-fri-roots
    f-final as dg composition-fri-roots final trace-table0
    composition-table0 final-state0 trace-openings0
    composition-openings0
  by blast
  have subset:
    query-header-supported-partial-table-candidates s fr f-fri-roots
    f-final as dg composition-fri-roots final ⊆
      {(trace-table0, composition-table0)}
  proof
    fix pair
    assume pair-in:
      pair ∈
        query-header-supported-partial-table-candidates s fr f-fri-roots
        f-final as dg composition-fri-roots final
    obtain trace-table composition-table where pair-eq:
      pair = (trace-table, composition-table)
    by (cases pair)
  from query-header-supported-partial-table-candidates-obtain-witness
    [OF pair-in[unfolded pair-eq]]
  obtain final-state trace-openings composition-openings where witness:
    query-header-supported-partial-table-candidate-witness s fr
    f-fri-roots f-final as dg composition-fri-roots final trace-table
    composition-table final-state trace-openings composition-openings
  by blast
  have pairwise-facts:
    partial-openings-cross-cover-all-indices trace-openings
      trace-openings0
    partial-openings-cross-cover-all-indices composition-openings
      composition-openings0
    ¬ merkle-hash-value-conflict final-state final-state0
    ¬ hash-map-output-collision
      (merkle-hash-state-merge final-state final-state0)
  using pairwise[OF witness witness0] by simp-all
  have eq:
    trace-table = trace-table0 ∧ composition-table = composition-table0
  by (rule
    query-header-supported-partial-table-candidate-witness-eq-if-cross-cover-clean-merge
    [OF witness witness0 pairwise-facts])
  show pair ∈ {(trace-table0, composition-table0)}
  using pair-eq eq by simp

```

```

qed
then show ?thesis
  by blast
qed

lemma query-header-supported-partial-table-candidates-not-singleton-imp-cross-cover-or-merkle-bad:
  assumes not-unique:
     $\neg (\exists \text{ trace-table } \text{composition-table}.$ 
       $\text{query-header-supported-partial-table-candidates } s \text{ fr } f\text{-fri-roots}$ 
       $f\text{-final as dg } \text{composition-fri-roots } \text{final} \subseteq$ 
       $\{( \text{trace-table}, \text{composition-table} ) \})$ 
  shows
     $\exists \text{ trace-table } \text{composition-table } \text{final-state } \text{trace-openings}$ 
     $\text{composition-openings } \text{trace-table}' \text{ composition-table}' \text{ final-state}'$ 
     $\text{trace-openings}' \text{ composition-openings}'.$ 
     $\text{query-header-supported-partial-table-candidate-witness } s \text{ fr}$ 
     $f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots } \text{final } \text{trace-table}$ 
     $\text{composition-table } \text{final-state } \text{trace-openings } \text{composition-openings} \wedge$ 
     $\text{query-header-supported-partial-table-candidate-witness } s \text{ fr}$ 
     $f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots } \text{final } \text{trace-table}'$ 
     $\text{composition-table}' \text{ final-state}' \text{ trace-openings}'$ 
     $\text{composition-openings}' \wedge$ 
     $(\text{trace-table} \neq \text{trace-table}' \vee$ 
     $\text{composition-table} \neq \text{composition-table}') \wedge$ 
     $(\neg \text{partial-openings-cross-cover-all-indices } \text{trace-openings}$ 
     $\text{trace-openings}' \vee$ 
     $\neg \text{partial-openings-cross-cover-all-indices } \text{composition-openings}$ 
     $\text{composition-openings}' \vee$ 
     $\text{merkle-hash-value-conflict } \text{final-state } \text{final-state}' \vee$ 
     $\text{hash-map-output-collision}$ 
     $(\text{merkle-hash-state-merge } \text{final-state } \text{final-state}'))$ 
  proof -
    let ?C =
       $\text{query-header-supported-partial-table-candidates } s \text{ fr } f\text{-fri-roots}$ 
       $f\text{-final as dg } \text{composition-fri-roots } \text{final}$ 
    have nonempty: ?C  $\neq \{\}$ 
      using not-unique by auto
    then obtain pair where pair-in: pair  $\in$  ?C
      by blast
    obtain trace-table composition-table where pair-eq:
      pair = (trace-table, composition-table)
      by (cases pair)
    have cand:
      (trace-table, composition-table)  $\in$  ?C
      using pair-in unfolding pair-eq .
    have not-subset:  $\neg ?C \subseteq \{( \text{trace-table}, \text{composition-table} ) \}$ 
      using not-unique by blast
    then obtain pair' where pair'-in: pair'  $\in$  ?C
      and pair'-not: pair'  $\notin \{( \text{trace-table}, \text{composition-table} ) \}$ 

```



```

  by auto
obtain trace-table' composition-table' where pair'-eq:
  pair' = (trace-table', composition-table')
  by (cases pair')
have cand':
  (trace-table', composition-table') ∈ ?C
  using pair'-in unfolding pair'-eq .
have distinct:
  trace-table ≠ trace-table' ∨
  composition-table ≠ composition-table'
  using pair'-not unfolding pair'-eq by auto
from query-header-supported-partial-table-candidates-obtain-witness[OF cand]
obtain final-state trace-openings composition-openings where witness:
  query-header-supported-partial-table-candidate-witness s fr
  f-fri-roots f-final as dg composition-fri-roots final trace-table
  composition-table final-state trace-openings composition-openings
  by blast
from query-header-supported-partial-table-candidates-obtain-witness[OF cand]
obtain final-state' trace-openings' composition-openings' where witness':
  query-header-supported-partial-table-candidate-witness s fr
  f-fri-roots f-final as dg composition-fri-roots final trace-table'
  composition-table' final-state' trace-openings'
  composition-openings'
  by blast
have side:
  ¬ partial-openings-cross-cover-all-indices trace-openings
  trace-openings' ∨
  ¬ partial-openings-cross-cover-all-indices composition-openings
  composition-openings' ∨
  merkle-hash-value-conflict final-state final-state' ∨
  hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')
proof (rule ccontr)
assume no-side:
  ¬ (¬ partial-openings-cross-cover-all-indices trace-openings
  trace-openings' ∨
  ¬ partial-openings-cross-cover-all-indices composition-openings
  composition-openings' ∨
  merkle-hash-value-conflict final-state final-state' ∨
  hash-map-output-collision
  (merkle-hash-state-merge final-state final-state'))
have eq:
  trace-table = trace-table' ∧
  composition-table = composition-table'
  by (rule
    query-header-supported-partial-table-candidate-witness-eq-if-cross-cover-clean-merge
    [OF witness witness'])
  (use no-side in auto)
then show False

```

using *distinct* by *simp*
 qed
 show ?thesis
 using *witness witness' distinct side* by *blast*
 qed

definition *query-header-supported-partial-pairwise-cross-or-merkle-bad*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow$
 $f \text{ list} \Rightarrow f \Rightarrow \text{bool}$

where

query-header-supported-partial-pairwise-cross-or-merkle-bad $s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table final-state trace-openings}$
 $\text{composition-openings trace-table' composition-table' final-state'}$
 $\text{trace-openings' composition-openings'}$
query-header-supported-partial-table-candidate-witness $s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final trace-table}$
 $\text{composition-table final-state trace-openings composition-openings} \wedge$
query-header-supported-partial-table-candidate-witness $s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final trace-table'}$
 $\text{composition-table' final-state' trace-openings'}$
 $\text{composition-openings' } \wedge$
 $(\text{trace-table} \neq \text{trace-table'} \vee$
 $\text{composition-table} \neq \text{composition-table'}) \wedge$
 $(\neg \text{partial-openings-cross-cover-all-indices trace-openings}$
 $\text{trace-openings'} \vee$
 $\neg \text{partial-openings-cross-cover-all-indices composition-openings}$
 $\text{composition-openings'} \vee$
 $\text{merkle-hash-value-conflict final-state final-state'} \vee$
 $\text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge final-state final-state'})))$

definition *query-supported-partial-pairwise-cross-or-merkle-bad*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

query-supported-partial-pairwise-cross-or-merkle-bad $s \longleftrightarrow$
 $(\exists f \text{ f-fri-roots } f\text{-final as dg composition-fri-roots final.}$
 $\text{query-header-supported-partial-pairwise-cross-or-merkle-bad } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final})$

lemma *query-header-supported-partial-pairwise-cross-or-merkle-bad-if-not-singleton:*

assumes *not-unique:*

$\neg (\exists \text{ trace-table composition-table.}$
 $\text{query-header-supported-partial-table-candidates } s \text{ fr f-fri-roots}$
 $f\text{-final as dg composition-fri-roots final} \subseteq$
 $\{(\text{trace-table, composition-table})\})$

shows

query-header-supported-partial-pairwise-cross-or-merkle-bad $s \text{ fr}$

f-fri-roots f-final as dg composition-fri-roots final
unfolding *query-header-supported-partial-pairwise-cross-or-merkle-bad-def*
using *query-header-supported-partial-table-candidates-not-singleton-imp-cross-cover-or-merkle-bad*
[OF not-unique]
by *blast*

lemma *query-header-supported-partial-table-candidates-subset-singleton-if-no-cross-or-merkle-bad:*
assumes *no-bad:*
 $\neg$ *query-header-supported-partial-pairwise-cross-or-merkle-bad s fr*
f-fri-roots f-final as dg composition-fri-roots final
shows
 $\exists$ *trace-table composition-table.*
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$
{(trace-table, composition-table)}

proof (*rule ccontr*)
assume *not-unique:*
 $\neg$ ($\exists$ *trace-table composition-table.*
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$
{(trace-table, composition-table)})
then have
query-header-supported-partial-pairwise-cross-or-merkle-bad s fr
f-fri-roots f-final as dg composition-fri-roots final
unfolding *query-header-supported-partial-pairwise-cross-or-merkle-bad-def*
using *query-header-supported-partial-table-candidates-not-singleton-imp-cross-cover-or-merkle-bad*
by *blast*
then show *False*
using *no-bad by contradiction*
qed

lemma *query-supported-partial-pairwise-cross-or-merkle-badD:*
assumes *query-supported-partial-pairwise-cross-or-merkle-bad s*
obtains *fr f-fri-roots f-final as dg composition-fri-roots final* **where**
query-header-supported-partial-pairwise-cross-or-merkle-bad s fr
f-fri-roots f-final as dg composition-fri-roots final
using *assms*
unfolding *query-supported-partial-pairwise-cross-or-merkle-bad-def* **by** *blast*

lemma *query-header-supported-partial-table-candidates-subset-singleton-if-no-global-cross-or-merkle-bad:*
assumes *no-bad:*
 $\neg$ *query-supported-partial-pairwise-cross-or-merkle-bad s*
shows
 $\exists$ *trace-table composition-table.*
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$

$\{(trace-table, composition-table)\}$
proof (rule
query-header-supported-partial-table-candidates-subset-singleton-if-no-cross-or-merkle-bad)
show
 $\neg$ *query-header-supported-partial-pairwise-cross-or-merkle-bad* *s* fr
f-fri-roots *f-final* as *dg* *composition-fri-roots* *final*
using *no-bad*
unfolding *query-supported-partial-pairwise-cross-or-merkle-bad-def* **by** *blast*
qed

lemma *query-header-supported-partial-union-good-sets-fraction-bound-if-no-cross-or-merkle-bad*:
assumes *no-bad*:

$\neg$ *query-header-supported-partial-pairwise-cross-or-merkle-bad* *s* fr
f-fri-roots *f-final* as *dg* *composition-fri-roots* *final*

and *subset*:

$\bigwedge$ *trace-table* *composition-table* as.

good-sets *trace-table* *composition-table* as $\subseteq$ *query-sample-space*

and *bounded*:

$\bigwedge$ *trace-table* *composition-table* as.

$nnreal$ (*card* (*good-sets* *trace-table* *composition-table* as)) /

$nnreal$ (*card* *query-sample-space*) $\leq$ *query-error-bound*

shows

$nnreal$

(*card*

(*query-header-supported-partial-union-good-sets* *s* *good-sets* fr
f-fri-roots *f-final* as *dg* *composition-fri-roots* *final*)) /

$nnreal$ (*card* *query-sample-space*) $\leq$ *query-error-bound*

proof —

have *unique*:

$\exists$ *trace-table* *composition-table*.

query-header-supported-partial-table-candidates *s* fr *f-fri-roots*

f-final as *dg* *composition-fri-roots* *final* $\subseteq$

$\{(trace-table, composition-table)\}$

by (rule

query-header-supported-partial-table-candidates-subset-singleton-if-no-cross-or-merkle-bad
[*OF no-bad*])

show *?thesis*

by (rule

query-header-supported-partial-union-good-sets-fraction-bound-if-unique-candidate
[*OF unique subset bounded*])

qed

lemma *query-header-supported-partial-union-good-sets-fraction-bound-if-no-global-cross-or-merkle-bad*:
assumes *no-bad*:

$\neg$ *query-supported-partial-pairwise-cross-or-merkle-bad* *s*

and *subset*:

$\bigwedge$ *trace-table* *composition-table* as.

good-sets *trace-table* *composition-table* as $\subseteq$ *query-sample-space*

and *bounded*:

$\wedge$ trace-table composition-table as.
 $\text{nnreal } (\text{card } (\text{good-sets trace-table composition-table as})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$
shows
 nnreal
 $(\text{card}$
 $(\text{query-header-supported-partial-union-good-sets } s \text{ good-sets } fr$
 $f\text{-fri-roots } f\text{-final as } dg \text{ composition-fri-roots final})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$
proof (rule
 $\text{query-header-supported-partial-union-good-sets-fraction-bound-if-no-cross-or-merkle-bad}$
 $[OF - \text{subset bounded}]$)
show
 $\neg \text{query-header-supported-partial-pairwise-cross-or-merkle-bad } s \text{ } fr$
 $f\text{-fri-roots } f\text{-final as } dg \text{ composition-fri-roots final}$
using no-bad
unfolding query-supported-partial-pairwise-cross-or-merkle-bad-def **by** blast
qed

lemma wp-query-index-round-set-hit-bound-if-no-global-cross-or-merkle-bad:
fixes $H :: \text{prob}$
assumes future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and no-bad:
 $\neg \text{query-supported-partial-pairwise-cross-or-merkle-bad } s$
and subset:
 $\wedge$ trace-table composition-table as.
 $\text{good-sets trace-table composition-table as} \subseteq \text{query-sample-space}$
and bounded:
 $\wedge$ trace-table composition-table as.
 $\text{nnreal } (\text{card } (\text{good-sets trace-table composition-table as})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$
and collision-bound:
 $\text{wp-event verify-monad } (\text{hash-map-output-collision-bad } s) \text{ } s \leq H$
shows
 $\text{wp-event verify-monad } (\text{query-index-round-set-hit } s \text{ good-sets}) \text{ } s \leq$
 $\text{nnreal rounds} * \text{query-error-bound} + H$
proof (rule
 $\text{wp-query-index-round-set-hit-bound-via-supported-partial-union-or-collision}$
 $[OF \text{ future raw-bound - collision-bound}]$)
fix $fr \text{ } f\text{-fri-roots } f\text{-final as } dg \text{ composition-fri-roots final}$
show
 nnreal
 $(\text{card}$
 $(\text{query-header-supported-partial-union-good-sets } s \text{ good-sets } fr$
 $f\text{-fri-roots } f\text{-final as } dg \text{ composition-fri-roots final})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$
by (rule
 $\text{query-header-supported-partial-union-good-sets-fraction-bound-if-no-global-cross-or-merkle-bad}$

$[OF \text{ no-bad subset bounded}]$)
qed
end
end

theory *Staged-Security-Experiment-Composition-Query-Header-Rounds*
imports
Staged-Security-Experiment-Composition-Query-Prefix-Witness
Soundness-Query-Header-Cross-Candidate
begin

Per-round decomposition for the header-authenticated composition query event.

The global event existentially ranges over the query round and authenticated opening witnesses. This layer peels off the finite round existential. The remaining witness selection problem is deliberately left explicit: later proofs must either choose prefix-fixed witnesses before the query challenge or charge failures to Merkle/collision side events.

context *soundness*
begin

definition *staged-query-prefix-header-supported-query-target*
 $:: 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{nat set}$

where
 $\text{staged-query-prefix-header-supported-query-target prefix prefix-state} =$
 $\text{query-header-supported-partial-union-good-sets prefix-state}$
 $\text{query-sampling-success-space}$
 $(\text{sqp-trace-root prefix})$
 $(\text{sqp-trace-fri-roots prefix})$
 $(\text{sqp-trace-final prefix})$
 $(\text{sqp-alphas prefix})$
 $(\text{sqp-degree prefix})$
 $(\text{sqp-composition-fri-roots prefix})$
 $(\text{sqp-composition-final prefix})$

lemma *staged-query-prefix-header-supported-query-target-subset:*
 $\text{staged-query-prefix-header-supported-query-target prefix prefix-state}$
 $\subseteq \text{query-sample-space}$
unfolding *staged-query-prefix-header-supported-query-target-def*
by (*rule query-header-supported-partial-union-good-sets-subset*)

lemma *staged-query-prefix-header-supported-query-target-fraction-bound-if-no-cross-or-merkle-bad:*
assumes *no-bad:*
 $\neg \text{query-supported-partial-pairwise-cross-or-merkle-bad prefix-state}$
shows
 nnreal
 $(\text{card}$
 $(\text{staged-query-prefix-header-supported-query-target prefix}$

$\text{prefix-state})) /$
 $\text{nnreal (card query-sample-space)} \leq \text{query-error-bound}$
unfolding staged-query-prefix-header-supported-query-target-def
by (rule
 query-header-supported-partial-union-good-sets-fraction-bound-if-no-global-cross-or-merkle-bad
 [OF no-bad query-sampling-success-space-subset
 query-sampling-success-space-fraction-bound-query-sample-space])

lemma checked-staged-query-prefix-header-supported-query-target-prehit-bound:

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i \leq \text{rounds}$

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-prehit
 staged-query-prefix-header-supported-query-target)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$)

proof (rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program)

show HashMap adversary-initial-state = fmempty

by (simp add: adversary-initial-state-def)

show hash-relation-program

(checked-staged-query-prefix-dynamic-index-prehit-relation A i
 adversary-initial-state
 staged-query-prefix-header-supported-query-target)
 size (staged-query-search-queries budgets $i + 1$)
 (checked-staged-query-prefix-receive-with-state A i)

by (rule hash-relation-program-checked-staged-query-prefix-receive-with-state
 [OF wf controlled i-bound])

(rule
 checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)

qed

lemma checked-staged-query-prefix-header-supported-query-target-hit-bound-from-budgets:

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i < \text{rounds}$
and no-bad:

$\bigwedge \text{prefix prefix-state.}$

Some ((prefix, prefix-state), prefix-state) $\in$
 set-dist

(execute (checked-staged-query-prefix-with-state A i)
 adversary-initial-state) $\implies$

$\neg$ query-supported-partial-pairwise-cross-or-merkle-bad
 prefix-state

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-hit

$\text{staged-query-prefix-header-supported-query-target}$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$

proof –

have *raw-bound*: $\text{query-index-raw-preimage-bound}$
by (rule $\text{query-index-raw-preimage-bound-from-sampler-wellformed}$)

have *hit-bound*:
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-header-supported-query-target})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $\text{staged-query-prefix-header-supported-query-target})$
 $\text{adversary-initial-state} + \text{query-error-bound}$

proof (rule
 $\text{checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error}$
 $[\text{OF raw-bound}]$)

fix *prefix prefix-state*
assume *support*:
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state})$

show
 $\text{staged-query-prefix-header-supported-query-target prefix prefix-state}$
 $\subseteq \text{query-sample-space} \wedge$
 nnreal
 $(\text{card}$
 $(\text{staged-query-prefix-header-supported-query-target prefix}$
 $\text{prefix-state})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$

using
 $\text{staged-query-prefix-header-supported-query-target-subset}$
 $[\text{of prefix prefix-state}]$
 $\text{staged-query-prefix-header-supported-query-target-fraction-bound-if-no-cross-or-merkle-bad}$
 $[\text{of prefix-state prefix, OF no-bad}[\text{OF support}]]$

by *blast*

qed

have *prehit-bound*:
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $\text{staged-query-prefix-header-supported-query-target})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1)$

by (rule
 $\text{checked-staged-query-prefix-header-supported-query-target-prehit-bound}$


```

      [OF wf controlled])
    (use i-bound in simp)
  show ?thesis
  by (rule order-trans[OF hit-bound])
    (intro add-mono prehit-bound order-refl)
qed

```

definition *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at*

```

:: nat  $\Rightarrow$  'f staged-adversary  $\Rightarrow$ 
  (((('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
    'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
    'f protocol-channel)  $\times$  unit list)  $\times$ 
    'f protocol-channel) option  $\Rightarrow$  bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
  i A out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state)  $\Rightarrow$ 
    (let s =
      verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)
    in  $\exists$  trace-openings composition-openings.
      (trace-openings, composition-openings)  $\in$ 
        query-header-supported-partial-opening-witnesses s
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)
        (staged-composition-fri-roots data)
        (staged-composition-final data)  $\wedge$ 
      checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
        A
        ((replicate rounds []) [i := trace-openings])
        ((replicate rounds []) [i := composition-openings])
        i
        (Some (((alpha-prefix, alpha-prefix-state), data),
          attacker-state), result), final-state))))

```

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*

assumes *witness*:

```

(trace-openings, composition-openings)  $\in$ 
  query-header-supported-partial-opening-witnesses
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)

```

```

    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  and component:
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  shows
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
    i A
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  unfolding
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at-def
    Let-def
  using witness component
  by (auto split: option.splits prod.splits)

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
  assumes hit:
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
    A out
  shows
    ∃ i < rounds.
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
    i A out
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-def
    by simp
  next
  case (Some packed)
  then obtain alpha-prefix alpha-prefix-state data attacker-state result
    final-state where out-eq:
    out =
    Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state)
    by (cases packed) (auto split: prod.splits)
  from checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefixE
    [OF hit[unfolded out-eq]]
  obtain i trace-openings composition-openings where witness:

```

```

    (trace-openings, composition-openings) ∈
      query-header-supported-partial-opening-witnesses
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)
        (staged-composition-fri-roots data)
        (staged-composition-final data)
  and component:
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
    i
      (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
  by blast
have i-bound: i < rounds
  using component
  unfolding
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-def
  by (simp split: option.splits prod.splits)
have round-hit:
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
    i A out
  unfolding out-eq
  by (rule
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-atI
      [OF witness component])
show ?thesis
  using i-bound round-hit by blast
qed

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
  assumes hit:
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
      i A
      (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
  obtains trace-openings composition-openings where
    (trace-openings, composition-openings) ∈
      query-header-supported-partial-opening-witnesses
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)

```

```

    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
using hit
unfolding
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at-de
Let-def
by (auto split: option.splits prod.splits)

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
assumes hit:
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
i A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
and i-bound: i < rounds
shows
(∃ result' final-state'.
partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result', final-state'))) ∨
(∃ trace-openings composition-openings trace-table composition-table.
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data) ∧
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)) ∧

```

```

    partial-trace-table-candidate trace-table
      ((replicate rounds []) [i := trace-openings]) ∧
    partial-composition-table-candidate composition-table
      ((replicate rounds []) [i := composition-openings]))
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-atE
    [OF hit]
  obtain trace-openings composition-openings where witness:
    (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  and component:
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
    A
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  by blast
from query-header-supported-partial-opening-witness-single-round-candidates-or-partial-merkle-bad
  [OF witness i-bound]
show ?thesis
  using witness component by blast
qed

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
assumes hit:
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
  i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
and i-bound: i < rounds
and trace-low:
  ∧ trace-openings composition-openings trace-table
    composition-table.
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses
  (verifier-state-from-adversary attacker-state

```

(staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\implies$
 partial-trace-table-candidate trace-table
 ((replicate rounds []) [i := trace-openings]) $\implies$
 partial-composition-table-candidate composition-table
 ((replicate rounds []) [i := composition-openings]) $\implies$
 trace-table-low-degree trace-table
and comp-low:
 $\bigwedge$ trace-openings composition-openings trace-table
 composition-table.
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\implies$
 partial-trace-table-candidate trace-table
 ((replicate rounds []) [i := trace-openings]) $\implies$
 partial-composition-table-candidate composition-table
 ((replicate rounds []) [i := composition-openings]) $\implies$
 composition-table-low-degree maxDegree composition-table
and not-all:
 $\bigwedge$ trace-openings composition-openings trace-table
 composition-table prefix prefix-state.
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\implies$
 partial-trace-table-candidate trace-table
 ((replicate rounds []) [i := trace-openings]) $\implies$
 partial-composition-table-candidate composition-table

$((\text{replicate rounds } []) [i := \text{composition-openings}]) \implies$
Some ((*prefix*, *prefix-state*), *prefix-state*) $\in$
set-dist
 (*execute* (*checked-staged-query-prefix-with-state* *A* *i*)
adversary-initial-state) $\implies$
 $\neg$ *all-queries-consistent trace-table composition-table*
 (*sqp-alphas* *prefix*)
shows
 ($\exists$ *result'* *final-state'*.
partial-merkle-inconsistency-bad
 (*verifier-state-from-adversary* *attacker-state*
 (*staged-proof-transcript* *data*))
 (*Some* (*result'*, *final-state'*))) $\vee$
 ($\exists$ *trace-table* *composition-table*.
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
A *trace-table* *composition-table* *i*
 (*Some* (((*alpha-prefix*, *alpha-prefix-state*), *data*),
 (*attacker-state*), *result*), *final-state*)))
proof –
let *?s* =
verifier-state-from-adversary *attacker-state*
 (*staged-proof-transcript* *data*)
from
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at-ca
 [*OF* *hit* *i-bound*]
show *?thesis*
proof
assume
 $\exists$ *result'* *final-state'*.
partial-merkle-inconsistency-bad *?s* (*Some* (*result'*, *final-state'*))
then show *?thesis* **by** *simp*
next
assume *candidates*:
 $\exists$ *trace-openings* *composition-openings* *trace-table*
composition-table.
 (*trace-openings*, *composition-openings*) $\in$
query-header-supported-partial-opening-witnesses *?s*
 (*staged-trace-root* *data*)
 (*staged-trace-fri-roots* *data*)
 (*staged-trace-final* *data*)
 (*staged-alphas* *data*)
 (*staged-degree* *data*)
 (*staged-composition-fri-roots* *data*)
 (*staged-composition-final* *data*) $\wedge$
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A
 ((*replicate rounds* []) [*i* := *trace-openings*])
 ((*replicate rounds* []) [*i* := *composition-openings*])
i

```

    (Some (((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state)) ∧
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings]) ∧
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
then obtain trace-openings composition-openings trace-table
  composition-table where witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
and opening-hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
  A
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  i
  (Some (((alpha-prefix, alpha-prefix-state), data),
    attacker-state), result), final-state))
and trace-candidate:
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
by blast
have pair-hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
  A trace-table composition-table i
  (Some (((alpha-prefix, alpha-prefix-state), data),
    attacker-state), result), final-state))
by (rule
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-imp-candidate-pair-que
  [OF opening-hit trace-candidate comp-candidate
    trace-low[OF witness trace-candidate comp-candidate]
    comp-low[OF witness trace-candidate comp-candidate]])
  (rule not-all[OF witness trace-candidate comp-candidate])
then show ?thesis by blast
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
fixes Q :: nat ⇒ prob

```


assumes *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at}$
 $i \ A)$
 $\text{adversary-initial-state} \leq Q \ i$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at}$
 $A)$
 $\text{adversary-initial-state} \leq (\sum i < \text{rounds}. Q \ i)$
proof –
have *event-le*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at}$
 $A)$
 $\text{adversary-initial-state} \leq$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\lambda \text{out}. \exists i \in \{.. < \text{rounds}\}.$
 $\text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at}$
 $i \ A \ \text{out})$
 $\text{adversary-initial-state}$
by (rule *wp-event-mono*)
 $(\text{use}$
 $\text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-im}$
 $\text{in blast})$
have *union-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\lambda \text{out}. \exists i \in \{.. < \text{rounds}\}.$
 $\text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at}$
 $i \ A \ \text{out})$
 $\text{adversary-initial-state} \leq$
 $(\sum i < \text{rounds}. Q \ i)$
by (rule *wp-event-finite-UN-bound*) (*simp-all add: round-bound*)
show *?thesis*
by (rule *order-trans*[*OF event-le union-bound*])
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-header-authenticated-query-from-prefix-r*
fixes $Q :: \text{nat} \Rightarrow \text{prob}$
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$

```

    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-
        i A)
      adversary-initial-state ≤ Q i
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event composition-bad)
      adversary-initial-state ≤
        composition-error-bound +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
        hash-collision-budget-value 0
        (staged-alpha-search-queries budgets 0) +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
        (∑ i<rounds. Q i) +
        hash-collision-budget-value 0
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget + verifier-hash-query-budget)
  proof -
  have header-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-
        A)
      adversary-initial-state ≤ (∑ i<rounds. Q i)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-round-
    [OF round-bound])
  show ?thesis
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-header-authenticated-query-from-prefix-and-
    [OF wf controlled header-bound])
qed

end

end

```

```

theory Staged-Security-Experiment-Composition-Query-Classification
imports Staged-Security-Experiment-Composition-Query-Prefix-Witness
begin

```

Local classification lemmas for current-final-state partial-opening evidence. These lemmas deliberately live downstream of the fixed-witness bridge to avoid growing the size-

sensitive prefix-witness theory.

context *soundness*

begin

lemma *trace-fri-bad-with-updated-partial-candidatesI:*

assumes *updated:*

accepted-with-partial-initial-openings s

(Some (result, final-state)) fr f-fri-roots f-final as dg

composition-fri-roots final trace-query-idxs trace-openings

composition-query-idxs composition-openings

and *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings

and *not-low: $\neg$ trace-table-low-degree trace-table*

shows *trace-fri-bad-with-partial-candidates s*

(Some (result, final-state))

using *assms*

unfolding *trace-fri-bad-with-partial-candidates-def* **by** *blast*

lemma *composition-fri-bad-with-updated-partial-candidatesI:*

assumes *updated:*

accepted-with-partial-initial-openings s

(Some (result, final-state)) fr f-fri-roots f-final as dg

composition-fri-roots final trace-query-idxs trace-openings

composition-query-idxs composition-openings

and *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings

and *comp-candidate:*

partial-composition-table-candidate composition-table

composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *not-comp-low:*

$\neg$ composition-table-low-degree maxDegree composition-table

shows *composition-fri-bad-with-partial-candidates s*

(Some (result, final-state))

using *assms*

unfolding *composition-fri-bad-with-partial-candidates-def* **by** *blast*

lemma *query-bad-with-updated-partial-candidatesI:*

assumes *updated:*

accepted-with-partial-initial-openings s

(Some (result, final-state)) fr f-fri-roots f-final as dg

composition-fri-roots final trace-query-idxs trace-openings

composition-query-idxs composition-openings

and *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings

and *comp-candidate:*

partial-composition-table-candidate composition-table

composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *comp-low*:
composition-table-low-degree maxDegree composition-table
and *not-all*: $\neg$ *all-queries-consistent trace-table composition-table as*
shows *query-bad-with-partial-candidates s (Some (result, final-state))*
using *assms*
unfolding *query-bad-with-partial-candidates-def* **by** *blast*

lemma *accepted-with-partial-initial-openings-update-round-candidate-classification*:

assumes *partial*:
accepted-with-partial-initial-openings s
(Some (result, final-state)) fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings
and *no-bad*:
 $\neg$ *partial-merkle-inconsistency-bad s (Some (result, final-state))*
and *i-bound*: $i < \text{rounds}$
and *same-idx*: *trace-query-idxs ! i = composition-query-idxs ! i*
and *consistent*:
partial-query-openings-consistent trace-openings-i
composition-openings-i as (trace-query-idxs ! i)
and *trace-auth*:
*partial-authenticated-table fr (scale * clength) trace-openings-i*
final-state
and *comp-auth*:
partial-authenticated-table (hd composition-fri-roots)
*(scale * clength) composition-openings-i final-state*

shows

trace-fri-bad-with-partial-candidates s (Some (result, final-state)) $\vee$
composition-fri-bad-with-partial-candidates s
(Some (result, final-state)) $\vee$
query-bad-with-partial-candidates s (Some (result, final-state)) $\vee$
 $(\exists \text{ trace-table composition-table.}$
partial-trace-table-candidate trace-table
 $(\text{trace-openings}[i := \text{trace-openings-}i]) \wedge$
partial-composition-table-candidate composition-table
 $(\text{composition-openings}[i := \text{composition-openings-}i]) \wedge$
trace-table-low-degree trace-table $\wedge$
composition-table-low-degree maxDegree composition-table $\wedge$
all-queries-consistent trace-table composition-table as)

proof –

let *?trace-openings = trace-openings[i := trace-openings-i]*
let *?composition-openings =*
composition-openings[i := composition-openings-i]
have *updated*:
accepted-with-partial-initial-openings s
(Some (result, final-state)) fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs ?trace-openings
composition-query-idxs ?composition-openings
by (rule *accepted-with-partial-initial-openings-update-round*

```

    [OF partial i-bound same-idx consistent trace-auth comp-auth])
obtain trace-table composition-table where trace-candidate:
    partial-trace-table-candidate trace-table ?trace-openings
and comp-candidate:
    partial-composition-table-candidate composition-table
    ?composition-openings
using accepted-with-partial-initial-openings-candidates-if-no-partial-merkle-bad
    [OF updated no-bad]
by blast
show ?thesis
proof (cases trace-table-low-degree trace-table)
  case False
  then have trace-fri-bad-with-partial-candidates s
    (Some (result, final-state))
    by (rule trace-fri-bad-with-updated-partial-candidatesI
      [OF updated trace-candidate])
  then show ?thesis by simp
next
  case trace-low: True
  show ?thesis
  proof (cases composition-table-low-degree maxDegree composition-table)
    case False
    then have composition-fri-bad-with-partial-candidates s
      (Some (result, final-state))
      by (rule composition-fri-bad-with-updated-partial-candidatesI
        [OF updated trace-candidate comp-candidate trace-low])
    then show ?thesis by simp
  next
  case comp-low: True
  show ?thesis
  proof (cases all-queries-consistent trace-table composition-table as)
    case False
    then have query-bad-with-partial-candidates s
      (Some (result, final-state))
      by (rule query-bad-with-updated-partial-candidatesI
        [OF updated trace-candidate comp-candidate trace-low comp-low])
    then show ?thesis by simp
  next
  case True
  then show ?thesis
    using trace-candidate comp-candidate trace-low comp-low by blast
  qed
qed
qed
qed

lemma accepted-with-partial-initial-openings-aligned-consistent-update-round-candidate-classification:
assumes partial:
  accepted-with-partial-initial-openings-aligned-consistent s

```

```

    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
and no-bad:
  ¬ partial-merkle-inconsistency-bad s (Some (result, final-state))
and i-bound: i < rounds
and consistent:
  partial-query-openings-consistent trace-openings-i
  composition-openings-i as (query-idxs ! i)
and trace-auth:
  partial-authenticated-table fr (scale * clength) trace-openings-i
  final-state
and comp-auth:
  partial-authenticated-table (hd composition-fri-roots)
  (scale * clength) composition-openings-i final-state
shows
  trace-fri-bad-with-partial-candidates s (Some (result, final-state)) ∨
  composition-fri-bad-with-partial-candidates s
  (Some (result, final-state)) ∨
  query-bad-with-partial-candidates s (Some (result, final-state)) ∨
  (∃ trace-table composition-table.
    partial-trace-table-candidate trace-table
    (trace-openings[i := trace-openings-i]) ∧
    partial-composition-table-candidate composition-table
    (composition-openings[i := composition-openings-i]) ∧
    trace-table-low-degree trace-table ∧
    composition-table-low-degree maxDegree composition-table ∧
    all-queries-consistent trace-table composition-table as)
proof —
  have aligned:
    accepted-with-partial-initial-openings-aligned s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
  using partial
  unfolding accepted-with-partial-initial-openings-aligned-consistent-def
  by simp
  have unaligned:
    accepted-with-partial-initial-openings s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings query-idxs
    composition-openings
  by (rule accepted-with-partial-initial-openings-aligned-imp-unaligned
    [OF aligned])
  show ?thesis
  by (rule
    accepted-with-partial-initial-openings-update-round-candidate-classification
    [OF unaligned no-bad i-bound - consistent trace-auth comp-auth])
  simp

```

qed

lemma *accepted-with-partial-initial-openings-aligned-transcript-consistent-update-round-candidate-classification:*

assumes *partial:*

accepted-with-partial-initial-openings-aligned-transcript-consistent s
(Some (result, final-state)) fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings
composition-openings

and *no-bad:*

$\neg$ *partial-merkle-inconsistency-bad s (Some (result, final-state))*

and *i-bound: i < rounds*

and *consistent:*

partial-query-openings-consistent trace-openings-i
composition-openings-i as (query-idxs ! i)

and *trace-auth:*

*partial-authenticated-table fr (scale * clength) trace-openings-i*
final-state

and *comp-auth:*

partial-authenticated-table (hd composition-fri-roots)
*(scale * clength) composition-openings-i final-state*

shows

trace-fri-bad-with-partial-candidates s (Some (result, final-state)) $\vee$
composition-fri-bad-with-partial-candidates s
(Some (result, final-state)) $\vee$
query-bad-with-partial-candidates s (Some (result, final-state)) $\vee$
($\exists$ trace-table composition-table.
partial-trace-table-candidate trace-table
(trace-openings[i := trace-openings-i]) $\wedge$
partial-composition-table-candidate composition-table
(composition-openings[i := composition-openings-i]) $\wedge$
trace-table-low-degree trace-table $\wedge$
composition-table-low-degree maxDegree composition-table $\wedge$
all-queries-consistent trace-table composition-table as)

proof —

have *aligned:*

accepted-with-partial-initial-openings-aligned-consistent s
(Some (result, final-state)) fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings
composition-openings

by (rule

accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
[OF *partial*])

show *?thesis*

by (rule

accepted-with-partial-initial-openings-aligned-consistent-update-round-candidate-classification
[OF *aligned no-bad i-bound consistent trace-auth comp-auth*])

qed

end

end

theory *Staged-Security-Experiment-Composition-Query-Current*
imports *Staged-Security-Experiment-Composition-Query-Classification*
begin

Current-execution projection for the composition query branch.

The broader header-supported query witnesses are existential over possible verifier executions. This layer only reasons about openings authenticated in the actual sampled verifier final state, so the sampled query lookup is available and the event projects to the existing data-state partial-opening query event.

context *soundness*
begin

lemma *checked-staged-security-experiment-with-query-prefix-data-state-raw-final-lookup:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *outcome:*

*Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) ∈
 set-dist*

*(execute
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 adversary-initial-state)*

shows

*fmlookup (HashMap final-state)
 (QueryIndexChallenge i
 (state-after-query-chunks
 (staged-query-start-hash data) (staged-query-chunks data) i)) =
 Some raw*

proof —

have *head:*

*Some (((prefix, prefix-state), raw), raw-state) ∈
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state)*

and *cont:*

*Some (((data, attacker-state), result), final-state) ∈
 set-dist
 (execute
 (checked-staged-after-query-prefix-receive-with-verifier A i
 ((prefix, prefix-state), raw))
 raw-state)*

using *outcome*

unfolding *checked-staged-security-experiment-with-query-prefix-data-state-def*
by *(auto elim!: set-dist-bindE)*


```

have recv:
  Some (raw, raw-state) ∈
    set-dist (execute receive-query-index-challenge prefix-state)
by (rule checked-staged-query-prefix-receive-with-state-receive-support
  [OF head])
have raw-lookup:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge (PQueryCounter prefix-state) (PState prefix-state)) =
    Some raw
  using receive-query-index-challenge-outcome[OF recv] by blast
have data-out:
  Some (data, attacker-state) ∈
    set-dist
      (execute
        (checked-staged-after-query-prefix-receive A i
          ((prefix, prefix-state), raw))
        raw-state)
    by (rule
      checked-staged-after-query-prefix-receive-with-verifier-outcome(1)
      [OF cont])
have verifier-out:
  Some (result, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
    by (rule
      checked-staged-after-query-prefix-receive-with-verifier-outcome(2)
      [OF cont])
have raw-ext-attacker: raw-state ≤ attacker-state
  by (rule checked-staged-after-query-prefix-receive-extends
    [OF wf controlled i-bound data-out])
have attacker-ext-verifier:
  attacker-state ≤
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
proof –
  have attacker-state ≤ attacker-state
    unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  then show ?thesis
    by (rule hash-extends-verifier-state-from-adversary-right)
qed
have verifier-ext-final:
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data) ≤
    final-state
  by (rule verify-monad-hash-extends[OF verifier-out])
have raw-ext-final: raw-state ≤ final-state
  by (rule hash-ext-trans[OF raw-ext-attacker])

```

```

    (rule hash-ext-trans[OF attacker-ext-verifier verifier-ext-final])
have prefix-support:
  Some ((prefix, prefix-state), prefix-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-with-state A i)
        adversary-initial-state)
by (rule checked-staged-query-prefix-receive-with-state-prefix-support
  [OF head])
have prefix-alignment:
  PQueryCounter prefix-state = i ∧
  PState prefix-state =
    state-after-query-chunks
      (staged-query-prefix-start-hash prefix)
      (sqp-query-chunks prefix) i ∧
  length (sqp-query-chunks prefix) = i
by (rule checked-staged-query-prefix-with-state-alignment
  [OF wf controlled i-bound prefix-support])
from cont obtain chunk chunk-state assert-state record-state suffix-chunks
where data-eq:
  data =
    (|staged-trace-root = sqp-trace-root prefix,
      staged-trace-fri-roots = sqp-trace-fri-roots prefix,
      staged-trace-fri-challenges = sqp-trace-fri-challenges prefix,
      staged-trace-final = sqp-trace-final prefix,
      staged-alphas = sqp-alphas prefix,
      staged-degree = sqp-degree prefix,
      staged-composition-fri-roots = sqp-composition-fri-roots prefix,
      staged-composition-fri-challenges =
        sqp-composition-fri-challenges prefix,
      staged-composition-final = sqp-composition-final prefix,
      staged-query-chunks = sqp-query-chunks prefix @ chunk # suffix-chunks)
unfolding checked-staged-after-query-prefix-receive-with-verifier-def
  checked-staged-after-query-prefix-receive-def
by (auto elim!: set-dist-bindE split: prod.splits)
have start-hash-eq:
  staged-query-start-hash data = staged-query-prefix-start-hash prefix
unfolding data-eq staged-query-start-hash-def
  staged-query-prefix-start-hash-def
  staged-composition-fri-start-hash-def
  staged-trace-fri-start-hash-def
by simp
have chunks-state-eq:
  state-after-query-chunks
    (staged-query-start-hash data) (staged-query-chunks data) i =
  state-after-query-chunks
    (staged-query-prefix-start-hash prefix) (sqp-query-chunks prefix) i
proof —
have len-prefix: length (sqp-query-chunks prefix) = i
  using prefix-alignment by simp

```

```

have
  state-after-query-chunks
    (staged-query-start-hash data) (staged-query-chunks data) i =
  state-after-query-chunks
    (staged-query-prefix-start-hash prefix)
    (sqp-query-chunks prefix @ chunk # suffix-chunks) i
  using start-hash-eq unfolding data-eq by simp
also have ... =
  state-after-query-chunks
    (staged-query-prefix-start-hash prefix) (sqp-query-chunks prefix) i
  by (rule state-after-query-chunks-append-prefix[OF len-prefix])
finally show ?thesis .
qed
have fields:
  PQueryCounter prefix-state = i
  PState prefix-state =
    state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i
  using prefix-alignment chunks-state-eq by simp-all
have raw-lookup':
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
  Some raw
  using raw-lookup fields by simp
show ?thesis
  by (rule hash-extension-lookup[OF raw-lookup' raw-ext-final])
qed

definition checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
where
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
    i out  $\longleftrightarrow$ 
    ( $\exists$  trace-openings composition-openings.
      checked-staged-security-with-query-prefix-authenticated-opening-hit
        trace-openings composition-openings i out)

definition checked-staged-security-with-query-prefix-current-query-partial-opening-hit
where
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit
    out  $\longleftrightarrow$ 
    ( $\exists$  i < rounds.
      checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
        i out)

lemma checked-staged-security-with-query-prefix-current-query-partial-opening-hit-atI:
assumes
  checked-staged-security-with-query-prefix-authenticated-opening-hit

```

$\text{trace-openings } \text{composition-openings } i \text{ out}$
shows
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at}$
 $i \text{ out}$
using assms
unfolding
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-def}$
by blast

lemma $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hitI}$:
assumes $i < \text{rounds}$
and
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at}$
 $i \text{ out}$
shows
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit}$
 out
using assms
unfolding
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-def}$
by blast

definition $\text{staged-query-prefix-prefix-authenticated-opening-target}$
 $:: 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{nat set}$
where
 $\text{staged-query-prefix-prefix-authenticated-opening-target prefix prefix-state} =$
 $\{idx \in \text{query-sample-space}.$
 $\exists (\text{trace-openings} :: 'f \text{ authenticated-opening list})$
 $(\text{composition-openings} :: 'f \text{ authenticated-opening list}).$
 $\text{sqp-composition-fri-roots prefix} \neq [] \wedge$
 $\text{partial-authenticated-table (sqp-trace-root prefix)}$
 $(\text{scale} * \text{clength}) \text{ trace-openings prefix-state} \wedge$
 $\text{partial-authenticated-table (hd (sqp-composition-fri-roots prefix))}$
 $(\text{scale} * \text{clength}) \text{ composition-openings prefix-state} \wedge$
 $\text{partial-query-openings-consistent trace-openings composition-openings}$
 $(\text{sqp-alphas prefix}) \text{ idx}\}$

definition $\text{staged-query-prefix-prefix-authenticated-trace-indices}$
 $:: 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{nat set}$
where
 $\text{staged-query-prefix-prefix-authenticated-trace-indices prefix prefix-state} =$
 $\{idx \in \text{query-sample-space}.$
 $\exists \text{opn} :: 'f \text{ authenticated-opening}.$
 $\text{authenticated-opening-in prefix-state opn} \wedge$
 $\text{opening-root opn} = \text{sqp-trace-root prefix} \wedge$
 $\text{opening-length opn} = \text{scale} * \text{clength} \wedge$
 $\text{opening-index opn} = \text{idx}\}$

lemma $\text{staged-query-prefix-prefix-authenticated-opening-target-subset}$:

staged-query-prefix-prefix-authenticated-opening-target prefix prefix-state
 $\subseteq$ *query-sample-space*
unfolding *staged-query-prefix-prefix-authenticated-opening-target-def*
by *blast*

lemma *staged-query-prefix-prefix-authenticated-opening-target-eq-query-sample-space-if-all-indices-witnessed:*

assumes *witnessed:*

$\bigwedge idx. idx \in \text{query-sample-space} \implies$
 $\exists \text{trace-openings composition-openings.}$
 $\text{sqp-composition-fri-roots prefix} \neq [] \wedge$
 $\text{partial-authenticated-table (sqp-trace-root prefix)}$
 $(\text{scale} * \text{clength}) \text{ trace-openings prefix-state} \wedge$
 $\text{partial-authenticated-table}$
 $(\text{hd (sqp-composition-fri-roots prefix)})$
 $(\text{scale} * \text{clength}) \text{ composition-openings prefix-state} \wedge$
 $\text{partial-query-openings-consistent trace-openings}$
 $\text{composition-openings (sqp-alphas prefix) idx}$

shows

staged-query-prefix-prefix-authenticated-opening-target prefix
prefix-state = query-sample-space

proof

show

staged-query-prefix-prefix-authenticated-opening-target prefix
prefix-state $\subseteq$ query-sample-space

by (rule *staged-query-prefix-prefix-authenticated-opening-target-subset*)

next

show

query-sample-space $\subseteq$
staged-query-prefix-prefix-authenticated-opening-target prefix
prefix-state

proof

fix *idx*

assume *idx-sample: idx $\in$ query-sample-space*

then obtain *trace-openings composition-openings* **where**

$\text{sqp-composition-fri-roots prefix} \neq []$
 $\text{partial-authenticated-table (sqp-trace-root prefix)}$
 $(\text{scale} * \text{clength}) \text{ trace-openings prefix-state}$
 $\text{partial-authenticated-table}$
 $(\text{hd (sqp-composition-fri-roots prefix)})$
 $(\text{scale} * \text{clength}) \text{ composition-openings prefix-state}$
 $\text{partial-query-openings-consistent trace-openings composition-openings}$
 $(\text{sqp-alphas prefix} \text{ } idx)$

using *witnessed* **by** *blast*

then show

idx $\in$
staged-query-prefix-prefix-authenticated-opening-target prefix
prefix-state

unfolding *staged-query-prefix-prefix-authenticated-opening-target-def*

using *idx-sample* **by** *blast*

```

qed
qed

lemma staged-query-prefix-prefix-authenticated-opening-target-subset-trace-indices:
  staged-query-prefix-prefix-authenticated-opening-target prefix prefix-state
  ⊆
  staged-query-prefix-prefix-authenticated-trace-indices prefix prefix-state
proof
  fix idx
  assume idx-in:
    idx ∈
      staged-query-prefix-prefix-authenticated-opening-target prefix
      prefix-state
  then obtain trace-openings composition-openings where
    idx-sample: idx ∈ query-sample-space
  and trace-table:
    partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) trace-openings prefix-state
  and consistent:
    partial-query-openings-consistent trace-openings
    composition-openings (sqp-alphas prefix) idx
  unfolding staged-query-prefix-prefix-authenticated-opening-target-def
  by blast
  have trace-indices:
    map opening-index trace-openings = powers-scaled idx
  using consistent unfolding partial-query-openings-consistent-def
  by simp
  have trace-nonempty: trace-openings ≠ []
  proof -
    have powers-scaled idx ≠ []
    unfolding powers-scaled-def using powers-pos by simp
    then show ?thesis
    using trace-indices by auto
  qed
  have hd-idx: opening-index (hd trace-openings) = idx
  proof -
    have hd (map opening-index trace-openings) = hd (powers-scaled idx)
    using trace-indices by simp
    then have opening-index (hd trace-openings) = hd (powers-scaled idx)
    using trace-nonempty by (simp add: hd-map)
    then show ?thesis
    using powers-scaled-hd by simp
  qed
  have hd-in: hd trace-openings ∈ set trace-openings
  using trace-nonempty by simp
  have auth:
    authenticated-opening-in prefix-state (hd trace-openings)
  using trace-table hd-in unfolding partial-authenticated-table-def
  by blast

```

```

have root:
  opening-root (hd trace-openings) = sqp-trace-root prefix
  using trace-table hd-in unfolding partial-authenticated-table-def
  by blast
have len:
  opening-length (hd trace-openings) = scale * clength
  using trace-table hd-in unfolding partial-authenticated-table-def
  by blast
show
  idx ∈
    staged-query-prefix-prefix-authenticated-trace-indices prefix
    prefix-state
  unfolding staged-query-prefix-prefix-authenticated-trace-indices-def
  by (intro CollectI conjI exI[of - hd trace-openings])
    (use idx-sample auth root len hd-idx in simp-all)
qed

lemma staged-query-prefix-prefix-authenticated-trace-indices-subset:
  staged-query-prefix-prefix-authenticated-trace-indices prefix prefix-state
    ⊆ query-sample-space
  unfolding staged-query-prefix-prefix-authenticated-trace-indices-def
  by blast

lemma staged-query-prefix-prefix-authenticated-trace-indices-eq-query-sample-space-if-all-indices-witnessed:
  assumes witnessed:
     $\bigwedge idx. idx \in \text{query-sample-space} \implies$ 
     $\exists \text{opn} :: 'f \text{ authenticated-opening.}$ 
    authenticated-opening-in prefix-state opn  $\wedge$ 
    opening-root opn = sqp-trace-root prefix  $\wedge$ 
    opening-length opn = scale * clength  $\wedge$ 
    opening-index opn = idx
  shows
    staged-query-prefix-prefix-authenticated-trace-indices prefix
    prefix-state = query-sample-space
proof
  show
    staged-query-prefix-prefix-authenticated-trace-indices prefix
    prefix-state ⊆ query-sample-space
  by (rule staged-query-prefix-prefix-authenticated-trace-indices-subset)
next
  show
    query-sample-space ⊆
    staged-query-prefix-prefix-authenticated-trace-indices prefix
    prefix-state
  unfolding staged-query-prefix-prefix-authenticated-trace-indices-def
  using witnessed by blast
qed

lemma finite-staged-query-prefix-prefix-authenticated-trace-indices[simp]:

```

```

finite
  (staged-query-prefix-prefix-authenticated-trace-indices prefix
   prefix-state)
by (rule finite-subset
   [OF staged-query-prefix-prefix-authenticated-trace-indices-subset
    finite-query-sample-space])

lemma staged-query-prefix-prefix-authenticated-opening-target-fraction-bound-if-trace-indices:
  assumes trace-frac:
    nnreal
      (card
        (staged-query-prefix-prefix-authenticated-trace-indices prefix
         prefix-state)) /
    nnreal (card query-sample-space) ≤ query-error-bound
  shows
    nnreal
      (card
        (staged-query-prefix-prefix-authenticated-opening-target prefix
         prefix-state)) /
    nnreal (card query-sample-space) ≤ query-error-bound
proof –
  have card-le:
    card
      (staged-query-prefix-prefix-authenticated-opening-target prefix
       prefix-state) ≤
    card
      (staged-query-prefix-prefix-authenticated-trace-indices prefix
       prefix-state)
  by (rule card-mono)
      (simp,
       rule staged-query-prefix-prefix-authenticated-opening-target-subset-trace-indices)
  have
    nnreal
      (card
        (staged-query-prefix-prefix-authenticated-opening-target prefix
         prefix-state)) /
    nnreal (card query-sample-space) ≤
    nnreal
      (card
        (staged-query-prefix-prefix-authenticated-trace-indices prefix
         prefix-state)) /
    nnreal (card query-sample-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
  also have ... ≤ query-error-bound
  by (rule trace-frac)
  finally show ?thesis .
qed

```

lemma checked-staged-query-prefix-prefix-authenticated-target-hit-bound-from-trace-indices:

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *i-bound*: $i < \text{rounds}$
and *trace-frac*:
 $\bigwedge \text{prefix prefix-state.}$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 nnreal
 $(\text{card}$
 $(\text{staged-query-prefix-prefix-authenticated-trace-indices prefix}$
 $\text{prefix-state})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$

shows

$\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-prefix-authenticated-opening-target})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$

proof –

have *raw-bound*: query-index-raw-preimage-bound
by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
have *hit-bound*:
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-prefix-authenticated-opening-target})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $\text{staged-query-prefix-prefix-authenticated-opening-target})$
 $\text{adversary-initial-state} + \text{query-error-bound}$

proof (rule

$\text{checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error}$
 $[\text{OF raw-bound}])$

fix *prefix prefix-state*
assume *support*:
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state})$

have *subset*:
 $\text{staged-query-prefix-prefix-authenticated-opening-target prefix}$
 $\text{prefix-state} \subseteq \text{query-sample-space}$
by (rule staged-query-prefix-prefix-authenticated-opening-target-subset)

have *frac*:
 nnreal
 $(\text{card}$

```

    (staged-query-prefix-prefix-authenticated-opening-target prefix
     prefix-state)) /
  nnreal (card query-sample-space) ≤ query-error-bound
by (rule
    staged-query-prefix-prefix-authenticated-opening-target-fraction-bound-if-trace-indices)
  (rule trace-frac[OF support])
show
  staged-query-prefix-prefix-authenticated-opening-target prefix
    prefix-state ⊆ query-sample-space ∧
  nnreal
    (card
      (staged-query-prefix-prefix-authenticated-opening-target prefix
       prefix-state)) /
    nnreal (card query-sample-space) ≤ query-error-bound
using subset frac by blast
qed
have prehit-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
     staged-query-prefix-prefix-authenticated-opening-target)
  adversary-initial-state ≤
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1)
proof (rule
  checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program)
show HashMap adversary-initial-state = fmempty
by (simp add: adversary-initial-state-def)
show hash-relation-program
  (checked-staged-query-prefix-dynamic-index-prehit-relation A i
   adversary-initial-state
   staged-query-prefix-prefix-authenticated-opening-target)
  size (staged-query-search-queries budgets i + 1)
  (checked-staged-query-prefix-receive-with-state A i)
by (rule hash-relation-program-checked-staged-query-prefix-receive-with-state
  [OF wf controlled])
  (use i-bound in simp-all,
   rule checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
qed
show ?thesis
by (rule order-trans[OF hit-bound])
  (intro add-mono prehit-bound order-refl)
qed

definition checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
where
  checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
    i out  $\longleftrightarrow$ 
    (∃ (trace-openings :: 'f authenticated-opening list)
     (composition-openings :: 'f authenticated-opening list)).

```

checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
trace-openings composition-openings i out)

definition *checked-staged-security-with-query-prefix-current-path-output-hit-at*
where

checked-staged-security-with-query-prefix-current-path-output-hit-at
(i::nat) out $\longleftrightarrow$
 $(\exists (trace-openings :: 'f \text{ authenticated-opening list})$
 $(composition-openings :: 'f \text{ authenticated-opening list}).$
checked-staged-security-with-query-prefix-opening-path-output-hit
trace-openings composition-openings out)

lemma *checked-staged-security-with-query-prefix-current-path-output-hit-at-imp-lower:*

assumes *hit:*

checked-staged-security-with-query-prefix-current-path-output-hit-at
i out

shows

$\exists (trace-openings :: 'f \text{ authenticated-opening list})$
 $(composition-openings :: 'f \text{ authenticated-opening list}).$
checked-staged-security-with-query-prefix-opening-path-output-hit
trace-openings composition-openings out

using *hit*

unfolding

checked-staged-security-with-query-prefix-current-path-output-hit-at-def

by *assumption*

lemma *checked-staged-security-with-query-prefix-current-path-output-hit-atE:*

assumes *hit:*

checked-staged-security-with-query-prefix-current-path-output-hit-at
i out

obtains *prefix prefix-state raw raw-state data attacker-state result*

final-state rt len idx val pth **where**

out = *Some* ((((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*)

hash-map-new-output-hit

(*merkle-path-target-roots final-state rt len idx val pth*)

prefix-state final-state

finite (*merkle-path-target-roots final-state rt len idx val pth*)

card (*merkle-path-target-roots final-state rt len idx val pth*) $\leq$

Suc (*length pth*)

proof –

from *hit* **have** *lower-ex:*

$\exists (trace-openings :: 'f \text{ authenticated-opening list})$
 $(composition-openings :: 'f \text{ authenticated-opening list}).$

checked-staged-security-with-query-prefix-opening-path-output-hit
trace-openings composition-openings out

unfolding

checked-staged-security-with-query-prefix-current-path-output-hit-at-def

by *assumption*

```

then show ?thesis
proof
  fix trace-openings
  assume composition-ex:
     $\exists$  composition-openings.
      checked-staged-security-with-query-prefix-opening-path-output-hit
      trace-openings composition-openings out
  then show ?thesis
  proof
    fix composition-openings
    assume lower:
      checked-staged-security-with-query-prefix-opening-path-output-hit
      trace-openings composition-openings out
    then show ?thesis
    proof (rule checked-staged-security-with-query-prefix-opening-path-output-hitE)
      fix prefix prefix-state raw raw-state data attacker-state result
      final-state opn
      assume out-eq:
        out = Some (((((prefix, prefix-state), raw), raw-state),
          ((data, attacker-state), result)), final-state)
      assume new-output:
        hash-map-new-output-hit
        (merkle-path-target-roots final-state
          (opening-root opn)
          (opening-length opn)
          (opening-index opn)
          (opening-value opn)
          (opening-path opn))
        prefix-state final-state
      assume finite-roots:
        finite
        (merkle-path-target-roots final-state
          (opening-root opn)
          (opening-length opn)
          (opening-index opn)
          (opening-value opn)
          (opening-path opn))
      assume card-roots:
        card
        (merkle-path-target-roots final-state
          (opening-root opn)
          (opening-length opn)
          (opening-index opn)
          (opening-value opn)
          (opening-path opn))  $\leq$ 
        Suc (length (opening-path opn))
    show thesis
    by (rule that
      [of prefix prefix-state raw raw-state data attacker-state result

```

```

      final-state opening-root opn opening-length opn
      opening-index opn opening-value opn opening-path opn])
    (rule out-eq, rule new-output, rule finite-roots, rule card-roots)
  qed
  qed
  qed
  qed

lemma checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-imp-dynamic-target:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and support:
    out ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
      i out
  shows
    checked-staged-security-with-query-prefix-dynamic-index-hit
      staged-query-prefix-prefix-authenticated-opening-target out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-def
      checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
  by (cases packed, auto split: prod.splits)
  have support-out:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
  using support out-eq by simp
  have head:

```

```

Some (((prefix, prefix-state), raw), raw-state) ∈
  set-dist
  (execute (checked-staged-query-prefix-receive-with-state A i)
    adversary-initial-state)
and cont:
Some (((data, attacker-state), result), final-state) ∈
  set-dist
  (execute
    (checked-staged-after-query-prefix-receive-with-verifier A i
      ((prefix, prefix-state), raw))
    raw-state)
using support-out
unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
by (auto elim!: set-dist-bindE)
have prefix-support:
Some ((prefix, prefix-state), prefix-state) ∈
  set-dist
  (execute (checked-staged-query-prefix-with-state A i)
    adversary-initial-state)
by (rule checked-staged-query-prefix-receive-with-state-prefix-support
  [OF head])
have prefix-alignment:
PQueryCounter prefix-state = i ∧
PState prefix-state =
  state-after-query-chunks
  (staged-query-prefix-start-hash prefix)
  (sqp-query-chunks prefix) i ∧
length (sqp-query-chunks prefix) = i
by (rule checked-staged-query-prefix-with-state-alignment
  [OF wf controlled i-bound prefix-support])
from cont obtain chunk chunk-state assert-state record-state suffix-chunks
where data-eq:
data =
  (|staged-trace-root = sqp-trace-root prefix,
    staged-trace-fri-roots = sqp-trace-fri-roots prefix,
    staged-trace-fri-challenges = sqp-trace-fri-challenges prefix,
    staged-trace-final = sqp-trace-final prefix,
    staged-alphas = sqp-alphas prefix,
    staged-degree = sqp-degree prefix,
    staged-composition-fri-roots = sqp-composition-fri-roots prefix,
    staged-composition-fri-challenges =
      sqp-composition-fri-challenges prefix,
    staged-composition-final = sqp-composition-final prefix,
    staged-query-chunks = sqp-query-chunks prefix @ chunk # suffix-chunks)
unfolding checked-staged-after-query-prefix-receive-with-verifier-def
checked-staged-after-query-prefix-receive-def
by (auto elim!: set-dist-bindE split: prod.splits)
from hit obtain trace-openings composition-openings where prefix-hit:
checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit

```

```

    trace-openings composition-openings i out
  unfolding
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-def
  by blast
have idx-in:
  index (to-nat raw) ∈
    staged-query-prefix-prefix-authenticated-opening-target prefix prefix-state
using prefix-hit index-less-query-sample-space
unfolding out-eq
  checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-def
  staged-query-prefix-prefix-authenticated-opening-target-def
  query-sample-space-def
  data-eq
by auto
show ?thesis
unfolding out-eq
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
  checked-staged-query-prefix-dynamic-index-hit-def
using idx-in by simp
qed

lemma checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-le-dynamic-target:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound: i < rounds
  shows
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
      adversary-initial-state ≤
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-dynamic-index-hit
        staged-query-prefix-prefix-authenticated-opening-target)
      adversary-initial-state
  by (rule wp-event-mono-on-support)
  (rule
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-imp-dynamic-target
    [OF wf controlled i-bound])

lemma checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound: i < rounds
    and prefix-bound:
      wp-event (checked-staged-query-prefix-receive-with-state A i)
        (checked-staged-query-prefix-dynamic-index-hit
          staged-query-prefix-prefix-authenticated-opening-target)
        adversary-initial-state ≤ B

```

shows
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
 adversary-initial-state $\leq B$

proof –
have *lifted*:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-dynamic-index-hit
 staged-query-prefix-prefix-authenticated-opening-target)
 adversary-initial-state $\leq B$
by (rule checked-staged-security-with-query-prefix-dynamic-index-hit-bound
 [OF prefix-bound])
show ?thesis
by (rule order-trans[
 OF checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-le-dynamic-target
 lifted])
 (use wf controlled i-bound in simp-all)

qed

lemma checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-trace-indices:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i < \text{rounds}$
and trace-frac:
 $\bigwedge \text{prefix prefix-state.}$
 Some ((prefix, prefix-state), prefix-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-with-state A i)
 adversary-initial-state) $\implies$
 nnreal
 (card
 (staged-query-prefix-prefix-authenticated-trace-indices prefix
 prefix-state)) /
 nnreal (card query-sample-space) $\leq$ query-error-bound

shows
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) + query-error-bound

proof (rule
 checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target
 [OF wf controlled i-bound])
show
wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-hit

staged-query-prefix-prefix-authenticated-opening-target)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) + query-error-bound
by (rule
checked-staged-query-prefix-prefix-authenticated-target-hit-bound-from-trace-indices
[OF wf controlled i-bound trace-frac])
qed

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-prefix-or-path-on-support*:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *support*:
out $\in$
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)
and *hit*:
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
i out
shows
checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
i out $\vee$
checked-staged-security-with-query-prefix-current-path-output-hit-at
i out
proof —
from *hit* **obtain** *trace-openings composition-openings* **where** *current*:
checked-staged-security-with-query-prefix-authenticated-opening-hit
trace-openings composition-openings i out
unfolding
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-def
by *blast*
have *split*:
checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
trace-openings composition-openings i out $\vee$
checked-staged-security-with-query-prefix-opening-path-output-hit
trace-openings composition-openings out
by (rule
checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-prefix-or-path-output-hit-on-support
[OF wf controlled i-bound support current])
then show *?thesis*
unfolding
checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-def
checked-staged-security-with-query-prefix-current-path-output-hit-at-def
by *blast*
qed

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-path:*

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *i-bound*: $i < \text{rounds}$

and *prefix-bound*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i*)

adversary-initial-state $\leq P$

and *path-bound*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-current-path-output-hit-at i*)

adversary-initial-state $\leq Q$

shows

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i*)

adversary-initial-state $\leq P + Q$

proof –

have *event-le*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i*)

adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

($\lambda \text{out.}$

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at

i out $\vee$

checked-staged-security-with-query-prefix-current-path-output-hit-at

i out)

adversary-initial-state

by (rule *wp-event-mono-on-support*)

(rule

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-prefix-or-path-on-support

[*OF wf controlled i-bound*])

have *union-bound*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

($\lambda \text{out.}$

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at

i out $\vee$

checked-staged-security-with-query-prefix-current-path-output-hit-at

i out)

adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i*)

```

    adversary-initial-state +
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-path-output-hit-at i)
  adversary-initial-state
by (rule wp-event-union-bound)
have sum-bound:
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
  adversary-initial-state +
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-path-output-hit-at i)
  adversary-initial-state ≤ P + Q
by (intro add-mono prefix-bound path-bound)
show ?thesis
by (rule order-trans[OF event-le])
  (rule order-trans[OF union-bound sum-bound])
qed

```

definition *staged-security-with-data-state-current-query-partial-opening-hit-at*

```

:: nat ⇒
  (((f staged-proof-data × f protocol-channel) × unit list) ×
   f protocol-channel) option ⇒ bool

```

where

```

staged-security-with-data-state-current-query-partial-opening-hit-at i out
↔
  (case out of
    None ⇒ False
  | Some (((data, attacker-state), result), final-state) ⇒
    (let s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    in
      (∃ raw trace-openings composition-openings rest.
        i < rounds ∧
        fmllookup (HashMap final-state)
          (QueryIndexChallenge i
            (state-after-query-chunks
              (staged-query-start-hash data)
              (staged-query-chunks data) i)) =
          Some raw ∧
        staged-composition-fri-roots data ≠ [] ∧
        Some (result, final-state) ∈
          set-dist (execute verify-monad s) ∧
        partial-authenticated-table (staged-trace-root data)
          (scale * clength) trace-openings final-state ∧
        partial-authenticated-table

```

(hd (staged-composition-fri-roots data))
 (scale * clength) composition-openings final-state $\wedge$
 verifier-header-transcript s
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
 rest $\wedge$
 partial-query-openings-consistent trace-openings
 composition-openings (staged-alphas data)
 (index (to-nat raw))))))

definition staged-security-with-data-state-current-query-partial-opening-hit
 $:: (((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

staged-security-with-data-state-current-query-partial-opening-hit out
 $\longleftrightarrow$
 ($\exists i < \text{rounds}.$
 staged-security-with-data-state-current-query-partial-opening-hit-at
 i out)

lemma staged-security-with-data-state-current-query-partial-opening-hit-at-aligned-transcript-classification-on-support

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and support:

Some (((data, attacker-state), result), final-state) $\in$
 set-dist
 (execute
 (checked-staged-security-experiment-with-data-state A)
 adversary-initial-state)

and partial:

accepted-with-partial-initial-openings-aligned-transcript-consistent
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result, final-state)) fr f-fri-roots f-final as dg
 composition-fri-roots final query-idxs trace-openings
 composition-openings

and no-bad:

$\neg$ partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result, final-state))

and hit:

staged-security-with-data-state-current-query-partial-opening-hit-at i
 (Some (((data, attacker-state), result), final-state))

shows

trace-fri-bad-with-partial-candidates
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result, final-state)) ∨
composition-fri-bad-with-partial-candidates
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result, final-state)) ∨
query-bad-with-partial-candidates
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result, final-state)) ∨
 (∃ trace-table composition-table trace-openings-i composition-openings-i.
 partial-trace-table-candidate trace-table
 (trace-openings[i := trace-openings-i]) ∧
 partial-composition-table-candidate composition-table
 (composition-openings[i := composition-openings-i]) ∧
 trace-table-low-degree trace-table ∧
 composition-table-low-degree maxDegree composition-table ∧
 all-queries-consistent trace-table composition-table as)

proof —

let ?s =

verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)

from checked-staged-security-experiment-with-data-state-outcomeE
 [OF support]

obtain builder **where** builder:

Some (data, attacker-state) ∈
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state)

by blast

have staged-header:

verifier-header-transcript ?s
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
 (List.concat (staged-query-chunks data))

by (rule checked-staged-transcript-program-outcome-header-transcript
 [OF wf controlled builder])

have shape:

accepted-transcript-shape ?s (Some (result, final-state)) as
 query-idxs

by (rule

```

    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
    [OF partial])
from checked-staged-security-with-data-state-accepted-shape-query-chunks
    [OF wf controlled support shape]
obtain raw-idxs where as-eq: as = staged-alphas data
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup:
    ∀ j < rounds.
        fmllookup (HashMap final-state)
            (QueryIndexChallenge j
              (state-after-query-chunks
                (staged-query-start-hash data)
                (staged-query-chunks data) j)) =
            Some (raw-idxs ! j)
by blast
from partial have aligned:
    accepted-with-partial-initial-openings-aligned-consistent ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned)
have aligned0:
    accepted-with-partial-initial-openings-aligned ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
by (rule
    accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
    [OF aligned])
from accepted-with-partial-initial-openings-aligned-shapes[OF aligned0]
obtain rest where header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
by blast
have header-eq:
    fr = staged-trace-root data ∧
    f-fri-roots = staged-trace-fri-roots data ∧
    f-final = staged-trace-final data ∧
    as = staged-alphas data ∧
    dg = staged-degree data ∧
    composition-fri-roots = staged-composition-fri-roots data ∧
    final = staged-composition-final data ∧
    rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
from hit obtain raw trace-openings-i composition-openings-i rest' where

```

```

i-bound:  $i < \text{rounds}$ 
and final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge  $i$ 
      (state-after-query-chunks
        (staged-query-start-hash  $\text{data}$ )
        (staged-query-chunks  $\text{data}$ )  $i$ )) =
      Some raw
and trace-auth:
  partial-authenticated-table (staged-trace-root  $\text{data}$ )
    ( $\text{scale} * \text{clength}$ ) trace-openings-i final-state
and comp-auth:
  partial-authenticated-table
    (hd (staged-composition-fri-roots  $\text{data}$ )) ( $\text{scale} * \text{clength}$ )
    composition-openings-i final-state
and consistent-row:
  partial-query-openings-consistent trace-openings-i
    composition-openings-i (staged-alphas  $\text{data}$ ) (index (to-nat  $\text{raw}$ ))
unfolding staged-security-with-data-state-current-query-partial-opening-hit-at-def
  Let-def
by auto
have raw-eq:  $\text{raw} = \text{raw-idxs} ! i$ 
  using final-lookup lookup i-bound by simp
have idx-eq:  $\text{index} (\text{to-nat } \text{raw}) = \text{query-idxs} ! i$ 
  using raw-eq query-idxs-eq len-row i-bound by simp
have consistent:
  partial-query-openings-consistent trace-openings-i
    composition-openings-i as (query-idxs !  $i$ )
  using consistent-row idx-eq header-eq by simp
have trace-auth':
  partial-authenticated-table fr ( $\text{scale} * \text{clength}$ ) trace-openings-i
    final-state
  using trace-auth header-eq by simp
have comp-auth':
  partial-authenticated-table (hd composition-fri-roots)
    ( $\text{scale} * \text{clength}$ ) composition-openings-i final-state
  using comp-auth header-eq by simp
have classified:
  trace-fri-bad-with-partial-candidates ? $s$ 
    (Some (result, final-state))  $\vee$ 
  composition-fri-bad-with-partial-candidates ? $s$ 
    (Some (result, final-state))  $\vee$ 
  query-bad-with-partial-candidates ? $s$  (Some (result, final-state))  $\vee$ 
  ( $\exists$  trace-table composition-table.
    partial-trace-table-candidate trace-table
      (trace-openings[ $i := \text{trace-openings-i}$ ])  $\wedge$ 
    partial-composition-table-candidate composition-table
      (composition-openings[ $i := \text{composition-openings-i}$ ])  $\wedge$ 
    trace-table-low-degree trace-table  $\wedge$ 

```

```

    composition-table-low-degree maxDegree composition-table  $\wedge$ 
    all-queries-consistent trace-table composition-table as)
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-update-round-candidate-classification
    [OF partial no-bad i-bound consistent trace-auth' comp-auth'])
  then show ?thesis
    by blast
qed

lemma staged-security-with-data-state-current-query-partial-opening-hit-at-imp-query-prefix-current-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and support:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)  $\in$ 
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
  and hit:
    staged-security-with-data-state-current-query-partial-opening-hit-at i
    (Some (((data, attacker-state), result), final-state))
  shows
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
    i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have hit':
     $\exists \text{raw}' \text{ trace-openings composition-openings rest.}$ 
     $i < \text{rounds} \wedge$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
    Some raw'  $\wedge$ 
    staged-composition-fri-roots data  $\neq [] \wedge$ 
    Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)  $\wedge$ 
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings final-state  $\wedge$ 
    partial-authenticated-table
      (hd (staged-composition-fri-roots data)) (scale * clength)
      composition-openings final-state  $\wedge$ 
    verifier-header-transcript ?s

```



```

    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    rest ∧
  partial-query-openings-consistent trace-openings
    composition-openings (staged-alphas data) (index (to-nat raw'))
using hit
unfolding staged-security-with-data-state-current-query-partial-opening-hit-at-def
  Let-def
by simp
from hit'
obtain raw' trace-openings composition-openings rest where
  final-lookup':
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some raw'
and comp-nonempty: staged-composition-fri-roots data ≠ []
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and comp-auth:
  partial-authenticated-table
    (hd (staged-composition-fri-roots data)) (scale * clength)
    composition-openings final-state
and consistent':
  partial-query-openings-consistent trace-openings
    composition-openings (staged-alphas data) (index (to-nat raw'))
by blast
have final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
by (rule
  checked-staged-security-experiment-with-query-prefix-data-state-raw-final-lookup
    [OF wf controlled i-bound support])
have raw-eq: raw' = raw
using final-lookup' final-lookup by simp
have current:
  checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i

```

```

    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding checked-staged-security-with-query-prefix-authenticated-opening-hit-def
  using comp-nonempty trace-auth comp-auth consistent' raw-eq by simp
show ?thesis
by (rule
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-atI
  [OF current])
qed

lemma checked-staged-security-with-data-state-current-query-partial-opening-hit-at-le-query-prefix-current:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  shows
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-query-partial-opening-hit-at i)
      adversary-initial-state ≤
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state
proof -
  let ?projected =
    λout. case out of
      None ⇒
        staged-security-with-data-state-current-query-partial-opening-hit-at
          i None
      | Some (packed, t) ⇒
        staged-security-with-data-state-current-query-partial-opening-hit-at
          i (Some (snd packed, t))
  have projection:
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-query-partial-opening-hit-at i)
      adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      ?projected
      adversary-initial-state
  by (rule
    checked-staged-security-experiment-with-query-prefix-data-state-projection-event
    [OF i-bound])
  have event-le:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      ?projected
      adversary-initial-state ≤

```

```

wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
  and hit: ?projected out
show
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
    i out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      staged-security-with-data-state-current-query-partial-opening-hit-at-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
show ?thesis
  unfolding out-eq
  by (rule
    staged-security-with-data-state-current-query-partial-opening-hit-at-imp-query-prefix-current-on-support
    [OF wf controlled i-bound])
  (use support hit out-eq in simp-all)
qed
qed
show ?thesis
  unfolding projection
  by (rule event-le)
qed

lemma checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds:
assumes round-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)

```

```

    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state  $\leq C i$ 
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
shows
  wp-event
  (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-current-query-partial-opening-hit
  adversary-initial-state  $\leq (\sum i < \text{rounds}. C i)$ 
proof -
  have round-le:
     $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
    (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-current-query-partial-opening-hit-at i)
    adversary-initial-state  $\leq C i$ 
  proof -
    fix i
    assume i-bound:  $i < \text{rounds}$ 
    show
      wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-query-partial-opening-hit-at i)
      adversary-initial-state  $\leq C i$ 
    by (rule order-trans
        [OF
          checked-staged-security-with-data-state-current-query-partial-opening-hit-at-le-query-prefix-current
            round-bound[OF i-bound]])
      (use wf controlled i-bound in simp-all)
qed
have event-le:
  wp-event
  (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-current-query-partial-opening-hit
  adversary-initial-state  $\leq$ 
  wp-event
  (checked-staged-security-experiment-with-data-state A)
  ( $\lambda \text{out}. \exists i \in \{.. < \text{rounds}\}. \text{staged-security-with-data-state-current-query-partial-opening-hit-at } i \text{ out}$ )
  adversary-initial-state
  by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-current-query-partial-opening-hit-def)
also have ...  $\leq (\sum i < \text{rounds}. C i)$ 
  by (rule wp-event-finite-UN-bound)
  (simp-all add: round-le)
finally show ?thesis .
qed

```

```

lemma staged-security-with-data-state-current-query-partial-opening-hit-at-imp-partial-opening-hit-at:
  assumes hit:
    staged-security-with-data-state-current-query-partial-opening-hit-at i
    out
  shows
    staged-security-with-data-state-query-partial-opening-hit-at i out
proof (cases out)
  case None
  then show ?thesis
  using hit
  unfolding
    staged-security-with-data-state-current-query-partial-opening-hit-at-def
    staged-security-with-data-state-query-partial-opening-hit-at-def
  by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have hit':
     $\exists$  raw trace-openings composition-openings rest.
    i < rounds  $\wedge$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some raw  $\wedge$ 
      staged-composition-fri-roots data  $\neq$  []  $\wedge$ 
      Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)  $\wedge$ 
      partial-authenticated-table (staged-trace-root data)
        (scale * clength) trace-openings final-state  $\wedge$ 
      partial-authenticated-table
        (hd (staged-composition-fri-roots data)) (scale * clength)
        composition-openings final-state  $\wedge$ 
      verifier-header-transcript ?s
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)
        (staged-composition-fri-roots data)
        (staged-composition-final data)
        rest  $\wedge$ 
      partial-query-openings-consistent trace-openings
        composition-openings (staged-alphas data) (index (to-nat raw))

```

```

using hit
unfolding out-eq
  staged-security-with-data-state-current-query-partial-opening-hit-at-def
  Let-def
by simp
from hit'
obtain raw trace-openings composition-openings rest where
  i-bound: i < rounds
and lookup:
  fmlookup (HashMap final-state)
  (QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data)
      (staged-query-chunks data) i)) =
    Some raw
and comp-nonempty: staged-composition-fri-roots data ≠ []
and verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
  (scale * clength) trace-openings final-state
and comp-auth:
  partial-authenticated-table
  (hd (staged-composition-fri-roots data)) (scale * clength)
  composition-openings final-state
and header:
  verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  rest
and consistent:
  partial-query-openings-consistent trace-openings
  composition-openings (staged-alphas data) (index (to-nat raw))
by blast
have idx-sample: index (to-nat raw) ∈ query-sample-space
using consistent
unfolding partial-query-openings-consistent-def by simp
have witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)

```

```

    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
unfolding query-header-supported-partial-opening-witnesses-def
using comp-nonempty verifier trace-auth comp-auth header
by blast
have success:
  index (to-nat raw) ∈
    query-header-supported-partial-opening-success-indices-at ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  i
unfolding query-header-supported-partial-opening-success-indices-at-def
by (intro CollectI conjI exI[of - trace-openings]
    exI[of - composition-openings])
    (use idx-sample witness consistent in simp-all)
show ?thesis
unfolding out-eq staged-security-with-data-state-query-partial-opening-hit-at-def
  Let-def
using i-bound lookup success by simp
qed

lemma staged-security-with-data-state-current-query-partial-opening-hit-imp-partial-opening-hit:
  assumes hit:
    staged-security-with-data-state-current-query-partial-opening-hit out
  shows staged-security-with-data-state-query-partial-opening-hit out
proof —
  from hit obtain i where i-bound: i < rounds
  and round-hit:
    staged-security-with-data-state-current-query-partial-opening-hit-at
    i out
  unfolding staged-security-with-data-state-current-query-partial-opening-hit-def
  by blast
have
  staged-security-with-data-state-query-partial-opening-hit-at i out
by (rule
    staged-security-with-data-state-current-query-partial-opening-hit-at-imp-partial-opening-hit-at
    [OF round-hit])
  then show ?thesis
  by (rule staged-security-with-data-state-query-partial-opening-hitI
    [OF i-bound])
qed

lemma staged-security-with-data-state-current-query-partial-opening-hitI:

```

```

assumes  $i < \text{rounds}$ 
and
   $\text{staged-security-with-data-state-current-query-partial-opening-hit-at}$ 
     $i \text{ out}$ 
shows
   $\text{staged-security-with-data-state-current-query-partial-opening-hit out}$ 
using  $\text{assms}$ 
unfolding  $\text{staged-security-with-data-state-current-query-partial-opening-hit-def}$ 
by  $\text{blast}$ 

lemma  $\text{checked-staged-security-with-data-state-aligned-transcript-query-bad-imp-current-partial-opening-hit-on-suppo}$ 
assumes  $\text{wf: staged-budget-wellformed budgets}$ 
and  $\text{controlled: staged-adversary-controlled budgets } A$ 
and  $\text{support:}$ 
   $\text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state}) \in$ 
     $\text{set-dist}$ 
     $(\text{execute}$ 
       $(\text{checked-staged-security-experiment-with-data-state } A)$ 
       $\text{adversary-initial-state})$ 
and  $\text{bad:}$ 
   $\text{staged-security-with-data-state-verifier-event}$ 
   $\text{query-bad-with-aligned-transcript-partial-candidates}$ 
   $(\text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state}))$ 
shows
   $\text{staged-security-with-data-state-current-query-partial-opening-hit}$ 
     $(\text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state}))$ 
proof –
let  $?s =$ 
   $\text{verifier-state-from-adversary attacker-state}$ 
   $(\text{staged-proof-transcript data})$ 
from  $\text{checked-staged-security-experiment-with-data-state-outcomeE}$ 
   $[\text{OF support}]$ 
obtain  $\text{builder where builder:}$ 
   $\text{Some } (\text{data}, \text{attacker-state}) \in$ 
     $\text{set-dist}$ 
     $(\text{execute } (\text{checked-staged-transcript-program } A)$ 
       $\text{adversary-initial-state})$ 
and  $\text{verifier:}$ 
   $\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute } \text{verify-monad } ?s)$ 
by  $\text{blast}$ 
have  $\text{staged-header:}$ 
   $\text{verifier-header-transcript } ?s$ 
   $(\text{staged-trace-root data})$ 
   $(\text{staged-trace-fri-roots data})$ 
   $(\text{staged-trace-final data})$ 
   $(\text{staged-alphas data})$ 
   $(\text{staged-degree data})$ 
   $(\text{staged-composition-fri-roots data})$ 
   $(\text{staged-composition-final data})$ 

```



```

    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
from bad obtain fr f-fri-roots f-final as dg composition-fri-roots
  final query-idxs trace-openings composition-openings trace-table
  composition-table where partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
unfolding staged-security-with-data-state-verifier-event-def
  query-bad-with-aligned-transcript-partial-candidates-def
by simp blast
have aligned:
  accepted-with-partial-initial-openings-aligned-consistent ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
have shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
    [OF partial])
from checked-staged-security-with-data-state-accepted-shape-query-chunks
  [OF wf controlled support shape]
obtain raw-idxs where
  as-eq: as = staged-alphas data
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
  and lookup:
     $\forall i < \text{rounds}.$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
by blast
have aligned0:
  accepted-with-partial-initial-openings-aligned ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
by (rule

```

```

    accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
    [OF aligned])
from accepted-with-partial-initial-openings-aligned-shapes[OF aligned0]
obtain rest where header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
by blast
have comp-nonempty: composition-fri-roots  $\neq []$ 
    using accepted-with-partial-initial-openings-aligned-shapes(1)
    [OF aligned0] .
have trace-partial:
    accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr query-idxs trace-openings
    using accepted-with-partial-initial-openings-aligned-shapes(3)
    [OF aligned0] .
have comp-partial:
    accepted-with-partial-composition-openings ?s
    (Some (result, final-state)) (hd composition-fri-roots) query-idxs
    composition-openings
    using accepted-with-partial-initial-openings-aligned-shapes(4)
    [OF aligned0] .
have header-eq:
    fr = staged-trace-root data  $\wedge$ 
    f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
    f-final = staged-trace-final data  $\wedge$ 
    as = staged-alphas data  $\wedge$ 
    dg = staged-degree data  $\wedge$ 
    composition-fri-roots = staged-composition-fri-roots data  $\wedge$ 
    final = staged-composition-final data  $\wedge$ 
    rest = List.concat (staged-query-chunks data)
    using verifier-header-transcript-unique[OF header staged-header]
    by simp
let ?i = 0
have i-bound: ?i < rounds
    using rounds-positive by simp
have round-consistent:
    partial-query-round-consistent trace-openings composition-openings as
    ?i (query-idxs ! ?i)
    using aligned i-bound
    unfolding accepted-with-partial-initial-openings-aligned-consistent-def
    by simp
have opening-consistent:
    partial-query-openings-consistent (trace-openings ! ?i)
    (composition-openings ! ?i) as (query-idxs ! ?i)
    using round-consistent
    unfolding partial-query-round-consistent-def
    partial-query-openings-consistent-def
    by simp
have trace-table:

```

```

partial-authenticated-table fr (scale * clength)
  (trace-openings ! ?i) final-state
using trace-partial i-bound
unfolding accepted-with-partial-trace-openings-def by blast
have comp-table:
  partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) (composition-openings ! ?i) final-state
using comp-partial i-bound
unfolding accepted-with-partial-composition-openings-def by blast
have raw-idx:
  index (to-nat (raw-idxs ! ?i)) = query-idxs ! ?i
using query-idxs-eq len-raw i-bound by simp
have lookup-i:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge ?i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) ?i)) =
    Some (raw-idxs ! ?i)
using lookup i-bound by simp
have exists-hit:
  ∃ raw trace-openings' composition-openings' rest'.
    ?i < rounds ∧
    fmlookup (HashMap final-state)
      (QueryIndexChallenge ?i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) ?i)) =
      Some raw ∧
    staged-composition-fri-roots data ≠ [] ∧
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s) ∧
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings' final-state ∧
    partial-authenticated-table (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings' final-state ∧
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      rest' ∧
    partial-query-openings-consistent trace-openings'
      composition-openings' (staged-alphas data) (index (to-nat raw))
apply (rule exI[where x = raw-idxs ! ?i])
apply (rule exI[where x = trace-openings ! ?i])
apply (rule exI[where x = composition-openings ! ?i])
apply (rule exI[where x = List.concat (staged-query-chunks data)])
using i-bound lookup-i comp-nonempty verifier trace-table comp-table

```

```

    opening-consistent raw-idx header-eq staged-header
  by simp
have hit-at:
  staged-security-with-data-state-current-query-partial-opening-hit-at ?i
  (Some (((data, attacker-state), result), final-state))
unfolding
  staged-security-with-data-state-current-query-partial-opening-hit-at-def
  Let-def
using exists-hit by simp
show ?thesis
  by (rule staged-security-with-data-state-current-query-partial-opening-hitI
      [OF i-bound hit-at])
qed

lemma checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     query-bad-with-aligned-transcript-partial-candidates)
    adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-partial-opening-hit
    adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
    set-dist
    (execute
     (checked-staged-security-experiment-with-data-state A)
     adversary-initial-state)
  and bad:
    staged-security-with-data-state-verifier-event
    query-bad-with-aligned-transcript-partial-candidates out
  show staged-security-with-data-state-current-query-partial-opening-hit out
proof (cases out)
  case None
  then show ?thesis
    using bad
    unfolding staged-security-with-data-state-verifier-event-def
    query-bad-with-aligned-transcript-partial-candidates-def
    by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)

```

```

show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-data-state-aligned-transcript-query-bad-imp-current-partial-opening-hit-on-support
    [OF wf controlled])
    (use support bad out-eq in simp-all)
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-current-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state) ∈
  set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
shows
  staged-security-with-data-state-current-query-partial-opening-hit
    (Some (((data, attacker-state), result), final-state))
proof —
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
  set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
by (rule
  checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
  [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)

```

```

by blast
have header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-current-openingsE
  [OF wf controlled support hit])
fix s trace-table composition-table query-idxs f i raw trace-openings
  composition-openings trace-openingss composition-openingss
assume s-def:
  s = verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
and i-bound: i < rounds
and final-lookup:
  fmllookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some raw
and idx-eq: index (to-nat raw) = query-idxs ! i
and comp-nonempty: staged-composition-fri-roots data ≠ []
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and comp-auth:
  partial-authenticated-table
    (hd (staged-composition-fri-roots data)) (scale * clength)
    composition-openings final-state
and consistent:
  partial-query-openings-consistent trace-openings
    composition-openings (staged-alphas data) (query-idxs ! i)
have consistent-raw:
  partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
using consistent idx-eq by simp
have round-hit:
  staged-security-with-data-state-current-query-partial-opening-hit-at i
    (Some (((data, attacker-state), result), final-state))

```

```

unfolding
  staged-security-with-data-state-current-query-partial-opening-hit-at-def
  Let-def
apply simp
by (intro exI[of - raw] exI[of - trace-openings]
      exI[of - composition-openings]
      exI[of - List.concat (staged-query-chunks data)] conjI)
  (use i-bound final-lookup comp-nonempty verifier trace-auth
    comp-auth header consistent-raw in simp-all)
show
  staged-security-with-data-state-current-query-partial-opening-hit
  (Some (((data, attacker-state), result), final-state))
by (rule
    staged-security-with-data-state-current-query-partial-opening-hitI
    [OF i-bound round-hit])
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-data-state-current-po
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  adversary-initial-state ≤
  wp-event
  (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-current-query-partial-opening-hit
  adversary-initial-state
proof -
let ?project =
  λx. ((snd (fst (fst x)), snd (fst x)), snd x)
let ?projected =
  λout. case out of
    None ⇒ False
  | Some (packed, t) ⇒
    staged-security-with-data-state-current-query-partial-opening-hit
    (Some (?project packed, t))
have event-le:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  adversary-initial-state ≤
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ?projected adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out

```

```

assume support:
  out ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    out
show ?projected out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    by simp
next
  case (Some packed)
  then obtain alpha-prefix alpha-prefix-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((alpha-prefix, alpha-prefix-state), data),
        attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  have partial-hit:
    staged-security-with-data-state-current-query-partial-opening-hit
      (Some (((data, attacker-state), result), final-state))
    by (rule
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-current-p
      [OF wf controlled])
      (use support hit out-eq in simp-all)
  show ?thesis
    unfolding out-eq
    using partial-hit by simp
  qed
qed
have projection:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?projected adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-partial-opening-hit
    adversary-initial-state
proof –

```



```

have
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-partial-opening-hit
    adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
       $\gg (\lambda x. \text{return } (?project\ x))$ )
    staged-security-with-data-state-current-query-partial-opening-hit
    adversary-initial-state
using
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
  [of A]
by simp
also have ... =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?projected adversary-initial-state
by (subst wp-event-bind-return-map)
    (simp add:
      staged-security-with-data-state-current-query-partial-opening-hit-def
      staged-security-with-data-state-current-query-partial-opening-hit-at-def)
finally show ?thesis
  by simp
qed
show ?thesis
  by (rule order-trans[OF event-le])
    (simp add: projection)
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-data-state-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and partial-bound:
  wp-event
    (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-partial-opening-hit
    adversary-initial-state  $\leq (Q::prob)$ 
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    adversary-initial-state  $\leq Q$ 
by (rule order.trans
  [OF
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-data-state-current-partic
    partial-bound])
    (use wf controlled in simp-all)

```

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit}$
 $\text{adversary-initial-state} \leq (\sum i < \text{rounds}. C \ i)$
proof –
have *current-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-current-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq (\sum i < \text{rounds}. C \ i)$
by (rule
 $\text{checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds}$
 $[OF \ \text{round-bound} \ wf \ \text{controlled}]$)
show *?thesis*
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-data-state-curr}$
 $[OF \ wf \ \text{controlled} \ \text{current-bound}]$)
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-prefix-target*
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *prefix-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-prefix-authenticated-opening-target})$
 $\text{adversary-initial-state} \leq P \ i$
and *path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq Q \ i$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit}$
 $\text{adversary-initial-state} \leq (\sum i < \text{rounds}. P \ i + Q \ i)$
proof (rule

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-curr
 [OF wf controlled])
fix i
assume $i\text{-bound}: i < \text{rounds}$
have prefix-hit-bound :
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* $A\ i$)
 (*checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at* i)
 $\text{adversary-initial-state} \leq P\ i$
 by (rule
 checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target
 [OF wf controlled $i\text{-bound}$ prefix-bound [OF $i\text{-bound}$]])
show
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* $A\ i$)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* i)
 $\text{adversary-initial-state} \leq P\ i + Q\ i$
 by (rule
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-path
 [OF wf controlled $i\text{-bound}$ prefix-hit-bound path-bound [OF $i\text{-bound}$]])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-trace-indic*
assumes $\text{wf}: \text{staged-budget-wellformed budgets}$
and $\text{controlled}: \text{staged-adversary-controlled budgets } A$
and trace-frac :
 $\bigwedge i\ \text{prefix}\ \text{prefix-state}.$
 $i < \text{rounds} \implies$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 (*execute* (*checked-staged-query-prefix-with-state* $A\ i$)
 $\text{adversary-initial-state}$) $\implies$
 nnreal
 (card
 (*staged-query-prefix-prefix-authenticated-trace-indices* prefix
 prefix-state)) /
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq \text{query-error-bound}$
and path-bound :
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* $A\ i$)
 (*checked-staged-security-with-query-prefix-current-path-output-hit-at* i)
 $\text{adversary-initial-state} \leq Q\ i$
shows
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* A)
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
 $\text{adversary-initial-state} \leq$
 $(\sum i < \text{rounds}.$

staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
query-error-bound + Q i

proof (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-curr
 [OF wf controlled])

fix *i*

assume *i-bound*: *i < rounds*

have *prefix-hit-bound*:

wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i*)
adversary-initial-state ≤
staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) + *query-error-bound*

by (rule
checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-trace-indices
 [OF wf controlled *i-bound*])
 (use *trace-frac*[OF *i-bound*] **in** *blast*)

show

wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i*)
adversary-initial-state ≤
staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
query-error-bound + Q i

by (rule
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-path
 [OF wf controlled *i-bound prefix-hit-bound path-bound*[OF *i-bound*]])

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-trace-indices-path-and-budgets*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *trace-frac*:

$\bigwedge i$ *prefix prefix-state*.
i < rounds $\implies$
 Some ((*prefix, prefix-state*), *prefix-state*) ∈
set-dist
 (*execute (checked-staged-query-prefix-with-state A i)*
adversary-initial-state) $\implies$

nnreal
 (*card*
 (*staged-query-prefix-prefix-authenticated-trace-indices prefix*
prefix-state)) /
nnreal (card query-sample-space) ≤ *query-error-bound*

and *path-bound*:

$\bigwedge i$. *i < rounds* $\implies$

$wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}query\text{-}prefix\text{-}data\text{-}state\ A\ i)$
 $(checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}current\text{-}path\text{-}output\text{-}hit\text{-}at\ i)$
 $adversary\text{-}initial\text{-}state \leq Q\ i$

shows

$wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event\ composition\text{-}bad)$
 $adversary\text{-}initial\text{-}state \leq$
 $composition\text{-}error\text{-}bound +$
 $staged\text{-}phase\text{-}relation\text{-}error\ size$
 $(staged\text{-}\alpha\text{-}search\text{-}queries\ budgets\ 0) +$
 $hash\text{-}collision\text{-}budget\text{-}value\ 0$
 $(staged\text{-}\alpha\text{-}search\text{-}queries\ budgets\ 0) +$
 $staged\text{-}phase\text{-}relation\text{-}error\ size$
 $(staged\text{-}\alpha\text{-}search\text{-}queries\ budgets\ 0) +$
 $(\sum i < rounds.$
 $staged\text{-}phase\text{-}relation\text{-}error\ size$
 $(staged\text{-}query\text{-}search\text{-}queries\ budgets\ i + 1) +$
 $query\text{-}error\text{-}bound + Q\ i) +$
 $hash\text{-}collision\text{-}budget\text{-}value\ 0$
 $(staged\text{-}attacker\text{-}query\text{-}budget\ budgets +$
 $staged\text{-}challenge\text{-}query\text{-}budget + verifier\text{-}hash\text{-}query\text{-}budget)$

proof –

let $?Q =$

$(\sum i < rounds.$
 $staged\text{-}phase\text{-}relation\text{-}error\ size$
 $(staged\text{-}query\text{-}search\text{-}queries\ budgets\ i + 1) +$
 $query\text{-}error\text{-}bound + Q\ i)$

have $query\text{-}bound:$

$wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}data\text{-}state\ A)$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}composition\text{-}candidate\text{-}drift\text{-}query\text{-}hit$
 $adversary\text{-}initial\text{-}state \leq ?Q$

by (rule

$checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}composition\text{-}candidate\text{-}drift\text{-}query\text{-}hit\text{-}bound\text{-}from\text{-}trace\text{-}indices\text{-}a$
 $[OF\ wf\ controlled\ trace\text{-}frac\ path\text{-}bound])$

have $without\text{-}bound:$

$wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}data\text{-}state\ A)$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}composition\text{-}candidate\text{-}drift\text{-}without\text{-}query\text{-}hit$
 $adversary\text{-}initial\text{-}state \leq 0$

by (rule

$checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}composition\text{-}candidate\text{-}drift\text{-}without\text{-}query\text{-}hit\text{-}bound$
 $[OF\ wf\ controlled])$

have $drift\text{-}bound:$

$wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}data\text{-}state\ A)$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}composition\text{-}candidate\text{-}drift$

```

    adversary-initial-state  $\leq$  ?Q + 0
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-query
    [OF query-bound without-bound])
  have comp-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event composition-bad)
      adversary-initial-state  $\leq$ 
        composition-error-bound +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
        hash-collision-budget-value 0
        (staged-alpha-search-queries budgets 0) +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
        (?Q + 0) +
        hash-collision-budget-value 0
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-composition-candidate-drift-and-budgets
    [OF wf controlled drift-bound])
  then show ?thesis
    by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-partial-
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state)  $\in$ 
      set-dist
        (execute
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
            A)
          adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
      (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
  shows
    staged-security-with-data-state-query-partial-opening-hit
      (Some (((data, attacker-state), result), final-state))
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have data-support:

```

```

Some (((data, attacker-state), result), final-state) ∈
  set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
by (rule
  checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
  [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
have header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-current-openingsE
  [OF wf controlled support hit])
fix s trace-table composition-table query-idxs f i raw trace-openings
  composition-openings trace-openingsss composition-openingsss
assume s-def:
  s = verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
and i-bound: i < rounds
and final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some raw
and idx-eq: index (to-nat raw) = query-idxs ! i
and comp-nonempty: staged-composition-fri-roots data ≠ []

```

```

and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and comp-auth:
  partial-authenticated-table
    (hd (staged-composition-fri-roots data)) (scale * clength)
    composition-openings final-state
and consistent:
  partial-query-openings-consistent trace-openings
    composition-openings (staged-alphas data) (query-idxs ! i)
have consistent-raw:
  partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
  using consistent idx-eq by simp
have idx-sample: index (to-nat raw) ∈ query-sample-space
  using consistent-raw
  unfolding partial-query-openings-consistent-def by simp
have witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  unfolding query-header-supported-partial-opening-witnesses-def
  using comp-nonempty verifier trace-auth comp-auth header
  by auto
have success:
  index (to-nat raw) ∈
    query-header-supported-partial-opening-success-indices-at ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    i
  unfolding query-header-supported-partial-opening-success-indices-at-def
  by (intro CollectI conjI exI[of - trace-openings]
    exI[of - composition-openings])
    (use idx-sample witness consistent-raw in simp-all)
have round-hit:
  staged-security-with-data-state-query-partial-opening-hit-at i
    (Some (((data, attacker-state), result), final-state))
  unfolding staged-security-with-data-state-query-partial-opening-hit-at-def

```



```

    Let-def
    using i-bound final-lookup success by simp
  show
    staged-security-with-data-state-query-partial-opening-hit
      (Some (((data, attacker-state), result), final-state))
    by (rule staged-security-with-data-state-query-partial-opening-hitI
      [OF i-bound round-hit])
  qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-data-state-partial-op
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
      adversary-initial-state ≤
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-partial-opening-hit
      adversary-initial-state
  proof -
    let ?project =
      λx. ((snd (fst (fst x))), snd (fst x)), snd x)
    let ?projected =
      λout. case out of
        None ⇒ False
      | Some (packed, t) ⇒
        staged-security-with-data-state-query-partial-opening-hit
          (Some (?project packed, t))
    have event-le:
      wp-event
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
        checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
        adversary-initial-state ≤
      wp-event
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
        ?projected adversary-initial-state
    proof (rule wp-event-mono-on-support)
      fix out
      assume support:
        out ∈
          set-dist
            (execute
              (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
                A)
              adversary-initial-state)
      and hit:

```

```

    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    out
show ?projected out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    by simp
next
  case (Some packed)
  then obtain alpha-prefix alpha-prefix-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((alpha-prefix, alpha-prefix-state), data),
        attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  have partial-hit:
    staged-security-with-data-state-query-partial-opening-hit
      (Some (((data, attacker-state), result), final-state))
    by (rule
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-partial-o
      [OF wf controlled])
      (use support hit out-eq in simp-all)
  show ?thesis
    unfolding out-eq
    using partial-hit by simp
qed
qed
have projection:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?projected adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-partial-opening-hit
    adversary-initial-state
proof –
  have
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-partial-opening-hit
      adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
         $\gg$  (λx. return (?project x)))
      staged-security-with-data-state-query-partial-opening-hit

```

```

    adversary-initial-state
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
also have ... =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?projected adversary-initial-state
  by (subst wp-event-bind-return-map)
    (simp add: staged-security-with-data-state-query-partial-opening-hit-def
      staged-security-with-data-state-query-partial-opening-hit-at-def)
  finally show ?thesis
    by simp
qed
show ?thesis
  by (rule order-trans[OF event-le])
    (simp add: projection)
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-data-state-
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and partial-bound:
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-partial-opening-hit
      adversary-initial-state  $\leq$  (Q::prob)
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
      adversary-initial-state  $\leq$  Q
  by (rule order.trans
    [OF
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-data-state-partial-opening-
      partial-bound])
    (use wf controlled in simp-all)

end

end

theory Staged-Security-Experiment-Composition-Query-Binding-Gap
  imports
    Staged-Security-Experiment-Composition-Query-Header-Rounds
    Staged-Security-Experiment-Composition-Query-Current
begin

```

Candidate-binding gap for the query-prefix composition route.

The dynamic header-supported query target is small only for candidates that are already visible in the prefix oracle state. This layer isolates the remaining proof obligation: if a later authenticated candidate pair is not in that prefix candidate set, the gap must be charged to Merkle/collision side events in the next proof layer.

context *soundness*

begin

lemma *checked-staged-query-prefix-candidate-pair-hit-imp-header-supported-target-hit:*

assumes *candidate:*

(*trace-table*, *composition-table*) ∈
query-header-supported-partial-table-candidates prefix-state
(*sqp-trace-root prefix*)
(*sqp-trace-fri-roots prefix*)
(*sqp-trace-final prefix*)
(*sqp-alphas prefix*)
(*sqp-degree prefix*)
(*sqp-composition-fri-roots prefix*)
(*sqp-composition-final prefix*)

and *hit:*

checked-staged-query-prefix-dynamic-index-hit
(*staged-query-prefix-candidate-pair-query-target-from-prefix*
trace-table composition-table)
(*Some (((prefix, prefix-state), raw), raw-state))*)

shows

checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-header-supported-query-target
(*Some (((prefix, prefix-state), raw), raw-state))*)

proof —

have *subset:*

staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table prefix prefix-state ⊆
staged-query-prefix-header-supported-query-target prefix prefix-state

unfolding

staged-query-prefix-candidate-pair-query-target-from-prefix-def
staged-query-prefix-header-supported-query-target-def
query-header-supported-partial-union-good-sets-def

using *candidate query-sampling-success-space-subset* **by** *auto*

show *?thesis*

using *hit subset*

unfolding *checked-staged-query-prefix-dynamic-index-hit-def*

by *auto*

qed

definition *query-header-supported-single-round-partial-table-candidates*

:: (*'f*, *'a*) *protocol-channel-scheme* ⇒
'f ⇒ *'f list* ⇒ *'f* ⇒ *'f list* ⇒ *'f* ⇒
'f list ⇒ *'f* ⇒ *nat* ⇒ (*'f list* × *'f list*) *set*

where

query-header-supported-single-round-partial-table-candidates s fr
f-fri-roots f-final as dg composition-fri-roots final i =
 $\{(trace-table, composition-table).$
composition-fri-roots $\neq []$ $\wedge$
 $(\exists trace-openings composition-openings rest.$
*partial-authenticated-table fr (scale * clength)*
trace-openings s $\wedge$
partial-authenticated-table (hd composition-fri-roots)
*(scale * clength) composition-openings s $\wedge$*
partial-trace-table-candidate trace-table
 $((replicate\ rounds\ [])\ [i := trace-openings])\ \wedge$
partial-composition-table-candidate composition-table
 $((replicate\ rounds\ [])\ [i := composition-openings])\ \wedge$
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest))}

lemma *query-header-supported-single-round-partial-table-candidatesI:*

assumes *comp-nonempty: composition-fri-roots $\neq []$*

and *trace-auth:*

*partial-authenticated-table fr (scale * clength)*
trace-openings s

and *comp-auth:*

partial-authenticated-table (hd composition-fri-roots)
*(scale * clength) composition-openings s*

and *trace-candidate:*

partial-trace-table-candidate trace-table
 $((replicate\ rounds\ [])\ [i := trace-openings])$

and *comp-candidate:*

partial-composition-table-candidate composition-table
 $((replicate\ rounds\ [])\ [i := composition-openings])$

and *header:*

verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest

shows

$(trace-table, composition-table) \in$
query-header-supported-single-round-partial-table-candidates s fr
f-fri-roots f-final as dg composition-fri-roots final i

unfolding *query-header-supported-single-round-partial-table-candidates-def*

using *assms by blast*

lemma *query-header-supported-single-round-partial-table-candidate-notin-contradicts:*

assumes *notin:*

$(trace-table, composition-table) \notin$
query-header-supported-single-round-partial-table-candidates s fr
f-fri-roots f-final as dg composition-fri-roots final i

and *comp-nonempty: composition-fri-roots $\neq []$*

and *trace-auth:*

*partial-authenticated-table fr (scale * clength)*

trace-openings s
and *comp-auth*:
partial-authenticated-table (hd composition-fri-roots)
*(scale * clength) composition-openings s*
and *trace-candidate*:
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings])
and *comp-candidate*:
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])
and *header*:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
shows *False*
using *notin*
query-header-supported-single-round-partial-table-candidatesI
[OF comp-nonempty trace-auth comp-auth trace-candidate
comp-candidate header]
by *contradiction*

definition *staged-query-prefix-single-round-header-supported-query-target*

$:: \text{nat} \Rightarrow 'f \text{ staged-query-prefix-data} \Rightarrow$
 $'f \text{ protocol-channel} \Rightarrow \text{nat set}$

where

staged-query-prefix-single-round-header-supported-query-target i prefix
prefix-state =
 $\{idx.$
 $\exists \text{ trace-table composition-table.}$
 $(\text{trace-table, composition-table}) \in$
 $\text{query-header-supported-single-round-partial-table-candidates}$
 prefix-state
 $(\text{sqp-trace-root prefix})$
 $(\text{sqp-trace-fri-roots prefix})$
 $(\text{sqp-trace-final prefix})$
 $(\text{sqp-alphas prefix})$
 $(\text{sqp-degree prefix})$
 $(\text{sqp-composition-fri-roots prefix})$
 $(\text{sqp-composition-final prefix})$
 $i \wedge$
 $idx \in$
 $\text{staged-query-prefix-candidate-pair-query-target-from-prefix}$
 $\text{trace-table composition-table prefix prefix-state}\}$

lemma *checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit:*

assumes *candidate:*

$(\text{trace-table, composition-table}) \in$
 $\text{query-header-supported-single-round-partial-table-candidates}$
 prefix-state
 $(\text{sqp-trace-root prefix})$

```

      (sqp-trace-fri-roots prefix)
      (sqp-trace-final prefix)
      (sqp-alphas prefix)
      (sqp-degree prefix)
      (sqp-composition-fri-roots prefix)
      (sqp-composition-final prefix)
    i
  and hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table)
      (Some (((prefix, prefix-state), raw), raw-state))
  shows
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-single-round-header-supported-query-target i)
      (Some (((prefix, prefix-state), raw), raw-state))
  proof -
  have idx-in:
    index (to-nat raw) ∈
      staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table prefix prefix-state
  using hit unfolding checked-staged-query-prefix-dynamic-index-hit-def
  by simp
  have
    index (to-nat raw) ∈
      staged-query-prefix-single-round-header-supported-query-target i
      prefix prefix-state
  unfolding staged-query-prefix-single-round-header-supported-query-target-def
  by (intro CollectI exI[of - trace-table]
    exI[of - composition-table])
    (use candidate idx-in in simp)
  then show ?thesis
    unfolding checked-staged-query-prefix-dynamic-index-hit-def
    by simp
qed

lemma query-header-supported-partial-table-candidatesI-from-partials:
  assumes comp-nonempty: composition-fri-roots ≠ []
  and outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and trace-partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr trace-query-idxs trace-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and comp-partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots)
    composition-query-idxs composition-openings

```

and *comp-candidate*:
partial-composition-table-candidate composition-table
composition-openings
and *header*:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
shows
(trace-table, composition-table) ∈
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final
proof –
have *body*:
composition-fri-roots ≠ [] ∧
(∃ out trace-query-idxs composition-query-idxs trace-openings
composition-openings rest.
out ∈ set-dist (execute verify-monad s) ∧
accepted-with-partial-trace-openings s out fr trace-query-idxs
trace-openings ∧
partial-trace-table-candidate trace-table trace-openings ∧
accepted-with-partial-composition-openings s out
(hd composition-fri-roots) composition-query-idxs
composition-openings ∧
partial-composition-table-candidate composition-table
composition-openings ∧
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest)
using *comp-nonempty outcome trace-partial trace-candidate comp-partial*
comp-candidate header
by *blast*
show *?thesis*
unfolding *query-header-supported-partial-table-candidates-def*
using *body by simp*
qed

lemma *query-header-supported-partial-table-candidate-notin-contradicts-partials*:
assumes *notin*:
(trace-table, composition-table) ∉
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final
and *comp-nonempty*: *composition-fri-roots ≠ []*
and *outcome*:
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *trace-partial*:
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
and *comp-partial*:
accepted-with-partial-composition-openings s

(Some (result, final-state)) (hd composition-fri-roots)
 composition-query-idxs composition-openings
and comp-candidate:
 partial-composition-table-candidate composition-table
 composition-openings
and header:
 verifier-header-transcript s fr f-fri-roots f-final as dg
 composition-fri-roots final rest
shows False
using notin
 query-header-supported-partial-table-candidatesI-from-partials
 [OF comp-nonempty outcome trace-partial trace-candidate comp-partial
 comp-candidate header]
by contradiction

lemma query-header-supported-partial-table-candidate-notin-no-accepted-partials:

assumes notin:
 (trace-table, composition-table) $\notin$
 query-header-supported-partial-table-candidates s fr f-fri-roots
 f-final as dg composition-fri-roots final
and comp-nonempty: composition-fri-roots $\neq []$
and trace-candidate:
 partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
 partial-composition-table-candidate composition-table
 composition-openings
shows
 $\neg (\exists \text{result final-state trace-query-idxs composition-query-idxs rest.}$
 Some (result, final-state) $\in$ set-dist (execute verify-monad s) $\wedge$
 accepted-with-partial-trace-openings s
 (Some (result, final-state)) fr trace-query-idxs trace-openings $\wedge$
 accepted-with-partial-composition-openings s
 (Some (result, final-state)) (hd composition-fri-roots)
 composition-query-idxs composition-openings $\wedge$
 verifier-header-transcript s fr f-fri-roots f-final as dg
 composition-fri-roots final rest)

proof

assume
 $\exists \text{result final-state trace-query-idxs composition-query-idxs rest.}$
 Some (result, final-state) $\in$ set-dist (execute verify-monad s) $\wedge$
 accepted-with-partial-trace-openings s
 (Some (result, final-state)) fr trace-query-idxs trace-openings $\wedge$
 accepted-with-partial-composition-openings s
 (Some (result, final-state)) (hd composition-fri-roots)
 composition-query-idxs composition-openings $\wedge$
 verifier-header-transcript s fr f-fri-roots f-final as dg
 composition-fri-roots final rest
then obtain result final-state trace-query-idxs composition-query-idxs rest
where outcome:

```

    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and trace-partial:
    accepted-with-partial-trace-openings s
      (Some (result, final-state)) fr trace-query-idxs trace-openings
  and comp-partial:
    accepted-with-partial-composition-openings s
      (Some (result, final-state)) (hd composition-fri-roots)
      composition-query-idxs composition-openings
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  by blast
show False
  by (rule query-header-supported-partial-table-candidate-notin-contradicts-partials
    [OF notin comp-nonempty outcome trace-partial trace-candidate
      comp-partial comp-candidate header])
qed

lemma accepted-with-partial-trace-openings-single-round-update-other-empty:
  assumes partial:
    accepted-with-partial-trace-openings s out fr query-idxs
      ((replicate rounds []) [i := trace-openings])
  and j-bound: j < rounds
  and j-ne: j ≠ i
  shows powers-scaled (query-idxs ! j) = []
proof -
  have indices:
    map opening-index (((replicate rounds []) [i := trace-openings]) ! j) =
      powers-scaled (query-idxs ! j)
  using accepted-with-partial-trace-openings-shapes(3)[OF partial j-bound]
  .
  have ((replicate rounds []) [i := trace-openings]) ! j = []
  using j-bound j-ne by simp
  then show ?thesis
  using indices by simp
qed

lemma accepted-with-partial-composition-openings-single-round-update-other-false:
  assumes partial:
    accepted-with-partial-composition-openings s out composition-root
      query-idxs ((replicate rounds []) [i := composition-openings])
  and j-bound: j < rounds
  and j-ne: j ≠ i
  shows False
proof -
  have indices:
    map opening-index (((replicate rounds []) [i := composition-openings]) ! j) =
      [query-idxs ! j, fri-sibling-index (scale * clength) (query-idxs ! j)]
  using

```

```

    accepted-with-partial-composition-openings-shapes(3)
    [OF partial j-bound]
  .
have ((replicate rounds []) [i := composition-openings]) ! j = []
  using j-bound j-ne by simp
then show False
  using indices by simp
qed

lemma query-header-supported-partial-opening-witnesses-pullback-or-path-output:
assumes witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses s fr f-fri-roots
  f-final as dg composition-fri-roots final
shows
  (partial-authenticated-table fr (scale * clength) trace-openings s ∧
  partial-authenticated-table (hd composition-fri-roots) (scale * clength)
  composition-openings s) ∨
  (∃ witness-state. ∃ opn ∈ set trace-openings.
  hash-map-new-output-hit
  (merkle-path-target-roots witness-state fr (scale * clength)
  (opening-index opn) (opening-value opn) (opening-path opn))
  s witness-state) ∨
  (∃ witness-state. ∃ opn ∈ set composition-openings.
  hash-map-new-output-hit
  (merkle-path-target-roots witness-state (hd composition-fri-roots)
  (scale * clength) (opening-index opn) (opening-value opn)
  (opening-path opn))
  s witness-state)
proof –
from witness obtain result witness-state rest where outcome:
  Some (result, witness-state) ∈ set-dist (execute verify-monad s)
and trace-table:
  partial-authenticated-table fr (scale * clength) trace-openings
  witness-state
and comp-table:
  partial-authenticated-table (hd composition-fri-roots)
  (scale * clength) composition-openings witness-state
by (elim query-header-supported-partial-opening-witnessesE)
have ext: s ≤ witness-state
by (rule verify-monad-hash-extends[OF outcome])
have trace-pull:
  partial-authenticated-table fr (scale * clength) trace-openings s ∨
  (∃ opn ∈ set trace-openings.
  hash-map-new-output-hit
  (merkle-path-target-roots witness-state fr (scale * clength)
  (opening-index opn) (opening-value opn) (opening-path opn))
  s witness-state)
by (rule partial-authenticated-table-pullback-or-new-output-hit

```

```

    [OF ext trace-table])
have comp-pull:
  partial-authenticated-table (hd composition-fri-roots) (scale * clength)
    composition-openings s ∨
  (∃ opn ∈ set composition-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state (hd composition-fri-roots)
        (scale * clength) (opening-index opn) (opening-value opn)
        (opening-path opn))
      s witness-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit
    [OF ext comp-table])
from trace-pull comp-pull show ?thesis by blast
qed

lemma query-header-supported-single-round-candidate-from-witness-or-path-output:
assumes witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses s fr f-fri-roots
    f-final as dg composition-fri-roots final
and trace-candidate:
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
shows
  (trace-table, composition-table) ∈
    query-header-supported-single-round-partial-table-candidates s fr
    f-fri-roots f-final as dg composition-fri-roots final i ∨
  (∃ witness-state. ∃ opn ∈ set trace-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state fr (scale * clength)
        (opening-index opn) (opening-value opn) (opening-path opn))
      s witness-state) ∨
  (∃ witness-state. ∃ opn ∈ set composition-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state (hd composition-fri-roots)
        (scale * clength) (opening-index opn) (opening-value opn)
        (opening-path opn))
      s witness-state)
proof –
from witness obtain result witness-state rest where comp-nonempty:
  composition-fri-roots ≠ []
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
by (elim query-header-supported-partial-opening-witnessesE)
have pull:

```

```

    (partial-authenticated-table fr (scale * clength) trace-openings s ∧
      partial-authenticated-table (hd composition-fri-roots) (scale * clength)
        composition-openings s) ∨
    (∃ witness-state. ∃ opn ∈ set trace-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots witness-state fr (scale * clength)
          (opening-index opn) (opening-value opn) (opening-path opn))
        s witness-state) ∨
    (∃ witness-state. ∃ opn ∈ set composition-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots witness-state (hd composition-fri-roots)
          (scale * clength) (opening-index opn) (opening-value opn)
            (opening-path opn))
        s witness-state)
  by (rule query-header-supported-partial-opening-witnesses-pullback-or-path-output
    [OF witness])
then show ?thesis
proof
  assume auths:
    partial-authenticated-table fr (scale * clength) trace-openings s ∧
    partial-authenticated-table (hd composition-fri-roots) (scale * clength)
      composition-openings s
  have candidate:
    (trace-table, composition-table) ∈
      query-header-supported-single-round-partial-table-candidates s fr
        f-fri-roots f-final as dg composition-fri-roots final i
  by (rule query-header-supported-single-round-partial-table-candidatesI
    [OF comp-nonempty - - trace-candidate comp-candidate header])
    (use auths in simp-all)
  then show ?thesis by simp
next
  assume
    (∃ witness-state. ∃ opn ∈ set trace-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots witness-state fr (scale * clength)
          (opening-index opn) (opening-value opn) (opening-path opn))
        s witness-state) ∨
    (∃ witness-state. ∃ opn ∈ set composition-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots witness-state (hd composition-fri-roots)
          (scale * clength) (opening-index opn) (opening-value opn)
            (opening-path opn))
        s witness-state)
  then show ?thesis by simp
qed
qed

```

lemma *query-header-supported-single-round-candidate-notin-imp-path-output:*
assumes *witness:*

(trace-openings, composition-openings) ∈
 query-header-supported-partial-opening-witnesses s fr f-fri-roots
 f-final as dg composition-fri-roots final
and trace-candidate:
 partial-trace-table-candidate trace-table
 ((replicate rounds []) [i := trace-openings])
and comp-candidate:
 partial-composition-table-candidate composition-table
 ((replicate rounds []) [i := composition-openings])
and notin:
 (trace-table, composition-table) ∉
 query-header-supported-single-round-partial-table-candidates s fr
 f-fri-roots f-final as dg composition-fri-roots final i
shows
 (∃ witness-state. ∃ opn ∈ set trace-openings.
 hash-map-new-output-hit
 (merkle-path-target-roots witness-state fr (scale * clength)
 (opening-index opn) (opening-value opn) (opening-path opn))
 s witness-state) ∨
 (∃ witness-state. ∃ opn ∈ set composition-openings.
 hash-map-new-output-hit
 (merkle-path-target-roots witness-state (hd composition-fri-roots)
 (scale * clength) (opening-index opn) (opening-value opn)
 (opening-path opn))
 s witness-state)
using query-header-supported-single-round-candidate-from-witness-or-path-output
 [OF witness trace-candidate comp-candidate] notin
by blast

definition checked-staged-security-with-query-prefix-header-supported-query-target-hit-at
where

checked-staged-security-with-query-prefix-header-supported-query-target-hit-at
 i out ⟷
 (case out of
 None ⇒ False
 | Some (((prefix, prefix-state), raw), raw-state), continuation),
 final-state) ⇒
 checked-staged-query-prefix-dynamic-index-hit
 staged-query-prefix-header-supported-query-target
 (Some (((prefix, prefix-state), raw), raw-state)))

definition checked-staged-security-with-query-prefix-candidate-binding-gap-at
where

checked-staged-security-with-query-prefix-candidate-binding-gap-at
 trace-openings composition-openings trace-table composition-table i out
 ⟷
 (case out of
 None ⇒ False
 | Some (((prefix, prefix-state), raw), raw-state),

$((data, attacker-state), result)), final-state) \Rightarrow$
checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i out $\wedge$
partial-trace-table-candidate trace-table
 $((replicate\ rounds\ [])\ [i := trace-openings]) \wedge$
partial-composition-table-candidate composition-table
 $((replicate\ rounds\ [])\ [i := composition-openings]) \wedge$
 $(trace-table, composition-table) \notin$
query-header-supported-partial-table-candidates prefix-state
 $(spp-trace-root\ prefix)$
 $(spp-trace-fri-roots\ prefix)$
 $(spp-trace-final\ prefix)$
 $(spp-alphas\ prefix)$
 $(spp-degree\ prefix)$
 $(spp-composition-fri-roots\ prefix)$
 $(spp-composition-final\ prefix))$

definition *checked-staged-security-with-query-prefix-candidate-binding-gap-side*
where

checked-staged-security-with-query-prefix-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i out
 $\longleftrightarrow$
(case out of
None $\Rightarrow$ *False*
 $|$ *Some* $(((((prefix, prefix-state), raw), raw-state),$
 $((data, attacker-state), result)), final-state) \Rightarrow$
partial-trace-table-candidate trace-table
 $((replicate\ rounds\ [])\ [i := trace-openings]) \wedge$
partial-composition-table-candidate composition-table
 $((replicate\ rounds\ [])\ [i := composition-openings]) \wedge$
 $(trace-table, composition-table) \notin$
query-header-supported-partial-table-candidates prefix-state
 $(spp-trace-root\ prefix)$
 $(spp-trace-fri-roots\ prefix)$
 $(spp-trace-final\ prefix)$
 $(spp-alphas\ prefix)$
 $(spp-degree\ prefix)$
 $(spp-composition-fri-roots\ prefix)$
 $(spp-composition-final\ prefix))$

definition *checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at*
where

checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out
 $\longleftrightarrow$
checked-staged-security-with-query-prefix-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i out $\wedge$
checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
trace-openings composition-openings i out

lemma *checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-imp-current-prefix-authenticated:*

assumes *gap:*

checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out

shows

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
i out

using *gap*

unfolding

checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at-def
checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-def

by *blast*

lemma *checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-bound-from-dynamic-target:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *prefix-bound:*

wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-prefix-authenticated-opening-target)
adversary-initial-state ≤ B

shows

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i)
adversary-initial-state ≤ B

proof —

have *event-le:*

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i)
adversary-initial-state ≤
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
i)
adversary-initial-state

by *(rule wp-event-mono)*

(rule

checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-imp-current-prefix-authenticated)

have *current-bound:*

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
i)

$adversary-initial-state \leq B$
by (rule
checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target
 $[OF \text{ wf controlled } i\text{-bound prefix-bound}]$)
show *?thesis*
by (rule *order-trans* $[OF \text{ event-le current-bound}]$)
qed

lemma *checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-bound-from-trace-indices*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < rounds$
and *trace-frac*:
 $\bigwedge prefix \text{ prefix-state.}$
 $Some ((prefix, prefix-state), prefix-state) \in$
 $set-dist$
 $(execute (checked-staged-query-prefix-with-state A i)$
 $adversary-initial-state) \implies$
 $nnreal$
 $(card$
 $(staged-query-prefix-prefix-authenticated-trace-indices \text{ prefix}$
 $prefix-state)) /$
 $nnreal (card \text{ query-sample-space}) \leq query-error-bound$
shows
 $wp-event$
 $(checked-staged-security-experiment-with-query-prefix-data-state A i)$
 $(checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at$
 $trace-openings \text{ composition-openings trace-table composition-table } i)$
 $adversary-initial-state \leq$
 $staged-phase-relation-error \text{ size}$
 $(staged-query-search-queries \text{ budgets } i + 1) + query-error-bound$
proof –
have *current-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-query-prefix-data-state A i)$
 $(checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at$
 $i)$
 $adversary-initial-state \leq$
 $staged-phase-relation-error \text{ size}$
 $(staged-query-search-queries \text{ budgets } i + 1) + query-error-bound$
by (rule
checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-trace-indices
 $[OF \text{ wf controlled } i\text{-bound trace-frac}]$)
have *event-le*:
 $wp-event$
 $(checked-staged-security-experiment-with-query-prefix-data-state A i)$
 $(checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at$
 $trace-openings \text{ composition-openings trace-table composition-table } i)$
 $adversary-initial-state \leq$

```

wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
    i)
  adversary-initial-state
by (rule wp-event-mono)
  (rule
    checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-imp-current-prefix-authenticated)
show ?thesis
by (rule order-trans[OF event-le current-bound])
qed

lemma checked-staged-security-with-query-prefix-candidate-binding-gap-iff-fixed-header-component:
  checked-staged-security-with-query-prefix-candidate-binding-gap-at
    trace-openings composition-openings trace-table composition-table i out
 $\longleftrightarrow$ 
  checked-staged-security-with-query-prefix-fixed-header-component
    (checked-staged-security-with-query-prefix-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i)
    trace-openings composition-openings i out
unfolding
  checked-staged-security-with-query-prefix-candidate-binding-gap-at-def
  checked-staged-security-with-query-prefix-candidate-binding-gap-side-def
  checked-staged-security-with-query-prefix-fixed-header-component-def
by (cases out) (auto split: prod.splits)

lemma checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-imp-header-target-or-bir
assumes hit:
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
    trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
and trace-candidate:
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
and trace-low: trace-table-low-degree trace-table
and comp-low: composition-table-low-degree maxDegree composition-table
and not-all:
   $\neg$  all-queries-consistent trace-table composition-table
    (sqp-alphas prefix)
shows
  checked-staged-security-with-query-prefix-header-supported-query-target-hit-at
    i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))  $\vee$ 
  checked-staged-security-with-query-prefix-candidate-binding-gap-at

```

```

    trace-openings composition-openings trace-table composition-table i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
proof –
from hit have opening-target:
  index (to-nat raw) ∈
    staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
    i prefix prefix-state
by (rule
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitE)
have opening-subset:
  staged-query-prefix-candidate-opening-query-target-from-prefix
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
  i prefix prefix-state ⊆
  staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
  composition-table prefix prefix-state
by (rule
  staged-query-prefix-candidate-opening-query-target-from-prefix-subset-candidate-pair-from-prefix
    [OF trace-candidate comp-candidate trace-low comp-low not-all])
have pair-target-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table)
    (Some (((prefix, prefix-state), raw), raw-state))
using opening-target opening-subset
unfolding checked-staged-query-prefix-dynamic-index-hit-def by auto
show ?thesis
proof (cases
  (trace-table, composition-table) ∈
    query-header-supported-partial-table-candidates prefix-state
      (sqp-trace-root prefix)
      (sqp-trace-fri-roots prefix)
      (sqp-trace-final prefix)
      (sqp-alphas prefix)
      (sqp-degree prefix)
      (sqp-composition-fri-roots prefix)
      (sqp-composition-final prefix))
case True
have header-target:
  checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-header-supported-query-target
    (Some (((prefix, prefix-state), raw), raw-state))
by (rule
  checked-staged-query-prefix-candidate-pair-hit-imp-header-supported-target-hit
    [OF True pair-target-hit])
have

```

```

checked-staged-security-with-query-prefix-header-supported-query-target-hit-at
  i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-header-supported-query-target-hit-at-def
using header-target by simp
then show ?thesis by simp
next
case False
have
  checked-staged-security-with-query-prefix-candidate-binding-gap-at
    trace-openings composition-openings trace-table composition-table i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-candidate-binding-gap-at-def
using hit trace-candidate comp-candidate False by simp
then show ?thesis by simp
qed
qed

```

lemma *checked-staged-security-with-query-prefix-candidate-binding-gap-imp-header-authenticated:*
assumes *gap:*
checked-staged-security-with-query-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out
shows
checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i out
using *gap*
unfolding *checked-staged-security-with-query-prefix-candidate-binding-gap-at-def*
by (*cases out*) (*auto split: prod.splits*)

lemma *checked-staged-security-with-query-prefix-candidate-binding-gap-current-imp-prefix-gap-or-opening-path-output:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *support:*
out $\in$
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)
and *current:*
checked-staged-security-with-query-prefix-fixed-current-authenticated-component
(checked-staged-security-with-query-prefix-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i)
trace-openings composition-openings i out
shows

checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out $\vee$
checked-staged-security-with-query-prefix-opening-path-output-hit
trace-openings composition-openings out

proof –

have *side*:

checked-staged-security-with-query-prefix-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i out

and *current-hit*:

checked-staged-security-with-query-prefix-authenticated-opening-hit
trace-openings composition-openings i out

using *current*

unfolding

checked-staged-security-with-query-prefix-fixed-current-authenticated-component-def
checked-staged-security-with-query-prefix-fixed-header-component-def

by *simp-all*

have *split*:

checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
trace-openings composition-openings i out $\vee$
checked-staged-security-with-query-prefix-opening-path-output-hit
trace-openings composition-openings out

by (rule

checked-staged-security-with-query-prefix-fixed-current-authenticated-component-imp-prefix-or-path-output-on-support
[OF wf controlled i-bound support current])

then show *?thesis*

proof

assume *prefix*:

checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
trace-openings composition-openings i out

have

checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out

unfolding

checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at-def

using *side prefix* **by** *simp*

then show *?thesis* **by** *simp*

next

assume

checked-staged-security-with-query-prefix-opening-path-output-hit
trace-openings composition-openings out

then show *?thesis* **by** *simp*

qed

qed

lemma *checked-staged-security-with-query-prefix-candidate-binding-gap-imp-current-or-path-output-on-support*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *i-bound*: *i < rounds*

and *support*:

$out \in$
 $set-dist$
 $(execute$
 $(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$
 $adversary-initial-state)$
and $gap:$
 $checked-staged-security-with-query-prefix-candidate-binding-gap-at$
 $trace-openings\ composition-openings\ trace-table\ composition-table\ i\ out$
shows
 $checked-staged-security-with-query-prefix-fixed-current-authenticated-component$
 $(checked-staged-security-with-query-prefix-candidate-binding-gap-side$
 $trace-openings\ composition-openings\ trace-table\ composition-table\ i)$
 $trace-openings\ composition-openings\ i\ out \vee$
 $checked-staged-security-with-query-prefix-header-witness-path-output-hit$
 $(checked-staged-security-with-query-prefix-candidate-binding-gap-side$
 $trace-openings\ composition-openings\ trace-table\ composition-table\ i)$
 $trace-openings\ composition-openings\ i\ out$
proof –
have $fixed:$
 $checked-staged-security-with-query-prefix-fixed-header-component$
 $(checked-staged-security-with-query-prefix-candidate-binding-gap-side$
 $trace-openings\ composition-openings\ trace-table\ composition-table\ i)$
 $trace-openings\ composition-openings\ i\ out$
using gap
unfolding
 $checked-staged-security-with-query-prefix-candidate-binding-gap-iff-fixed-header-component$
 $\cdot$
show $?thesis$
by $(rule$
 $checked-staged-security-with-query-prefix-fixed-header-component-imp-current-or-witness-path-output-on-support$
 $[OF\ wf\ controlled\ i-bound\ support\ fixed])$
qed

lemma $checked-staged-security-with-query-prefix-candidate-binding-gap-imp-prefix-gap-or-path-outputs-on-support:$
assumes $wf: staged-budget-wellformed\ budgets$
and $controlled: staged-adversary-controlled\ budgets\ A$
and $i-bound: i < rounds$
and $support:$
 $out \in$
 $set-dist$
 $(execute$
 $(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$
 $adversary-initial-state)$
and $gap:$
 $checked-staged-security-with-query-prefix-candidate-binding-gap-at$
 $trace-openings\ composition-openings\ trace-table\ composition-table\ i\ out$
shows
 $checked-staged-security-with-query-prefix-candidate-binding-gap-at$
 $trace-openings\ composition-openings\ trace-table\ composition-table\ i\ out \vee$

checked-staged-security-with-query-prefix-opening-path-output-hit
trace-openings composition-openings out $\vee$
checked-staged-security-with-query-prefix-header-witness-path-output-hit
(checked-staged-security-with-query-prefix-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i)
trace-openings composition-openings i out

proof —

have *split*:

checked-staged-security-with-query-prefix-fixed-current-authenticated-component
(checked-staged-security-with-query-prefix-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i)
trace-openings composition-openings i out $\vee$
checked-staged-security-with-query-prefix-header-witness-path-output-hit
(checked-staged-security-with-query-prefix-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i)
trace-openings composition-openings i out

by (rule
checked-staged-security-with-query-prefix-candidate-binding-gap-imp-current-or-path-output-on-support
[OF wf controlled i-bound support gap]))

then show *?thesis*

proof

assume *current*:

checked-staged-security-with-query-prefix-fixed-current-authenticated-component
(checked-staged-security-with-query-prefix-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i)
trace-openings composition-openings i out

have

checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out $\vee$
checked-staged-security-with-query-prefix-opening-path-output-hit
trace-openings composition-openings out

by (rule
checked-staged-security-with-query-prefix-candidate-binding-gap-current-imp-prefix-gap-or-opening-path-output-on-support
[OF wf controlled i-bound support current]))

then show *?thesis* **by** *blast*

next

assume

checked-staged-security-with-query-prefix-header-witness-path-output-hit
(checked-staged-security-with-query-prefix-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i)
trace-openings composition-openings i out

then show *?thesis* **by** *simp*

qed

qed

lemma *checked-staged-security-with-query-prefix-candidate-binding-gap-bound-from-prefix-target-and-path-outputs*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *i-bound*: *i < rounds*

and *prefix-bound*:
wp-event (*checked-staged-query-prefix-receive-with-state A i*)
(*checked-staged-query-prefix-dynamic-index-hit*
staged-query-prefix-prefix-authenticated-opening-target)
adversary-initial-state $\leq P$
and *opening-path-bound*:
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
(*checked-staged-security-with-query-prefix-opening-path-output-hit*
trace-openings composition-openings)
adversary-initial-state $\leq Q$
and *witness-path-bound*:
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
(*checked-staged-security-with-query-prefix-header-witness-path-output-hit*
(*checked-staged-security-with-query-prefix-candidate-binding-gap-side*
trace-openings composition-openings trace-table composition-table i)
trace-openings composition-openings i)
adversary-initial-state $\leq R$
shows
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
(*checked-staged-security-with-query-prefix-candidate-binding-gap-at*
trace-openings composition-openings trace-table composition-table i)
adversary-initial-state $\leq P + Q + R$
proof –
let *?m* =
checked-staged-security-experiment-with-query-prefix-data-state A i
let *?prefix* =
checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i
let *?open-path* =
checked-staged-security-with-query-prefix-opening-path-output-hit
trace-openings composition-openings
let *?witness-path* =
checked-staged-security-with-query-prefix-header-witness-path-output-hit
(*checked-staged-security-with-query-prefix-candidate-binding-gap-side*
trace-openings composition-openings trace-table composition-table i)
trace-openings composition-openings i
have *event-le*:
wp-event ?m
(*checked-staged-security-with-query-prefix-candidate-binding-gap-at*
trace-openings composition-openings trace-table composition-table i)
adversary-initial-state $\leq$
wp-event ?m
($\lambda out. ?prefix\ out \vee ?open-path\ out \vee ?witness-path\ out$)
adversary-initial-state
by (*rule wp-event-mono-on-support*)
(*rule*


```

checked-staged-security-with-query-prefix-candidate-binding-gap-imp-prefix-gap-or-path-outputs-on-support
[OF wf controlled i-bound])
have union-le:
  wp-event ?m
    (λout. ?prefix out ∨ ?open-path out ∨ ?witness-path out)
    adversary-initial-state ≤
  wp-event ?m ?prefix adversary-initial-state +
  (wp-event ?m ?open-path adversary-initial-state +
    wp-event ?m ?witness-path adversary-initial-state)
proof -
  have
    wp-event ?m
      (λout. ?prefix out ∨ ?open-path out ∨ ?witness-path out)
      adversary-initial-state ≤
    wp-event ?m ?prefix adversary-initial-state +
    wp-event ?m
      (λout. ?open-path out ∨ ?witness-path out)
      adversary-initial-state
    by (rule wp-event-union-bound)
  also have
    ... ≤
    wp-event ?m ?prefix adversary-initial-state +
    (wp-event ?m ?open-path adversary-initial-state +
      wp-event ?m ?witness-path adversary-initial-state)
    by (intro add-mono order-refl wp-event-union-bound)
  finally show ?thesis .
qed
have prefix-gap-bound:
  wp-event ?m ?prefix adversary-initial-state ≤ P
  by (rule
    checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-bound-from-dynamic-target
    [OF wf controlled i-bound prefix-bound])
have sum-bound:
  wp-event ?m ?prefix adversary-initial-state +
  (wp-event ?m ?open-path adversary-initial-state +
    wp-event ?m ?witness-path adversary-initial-state) ≤ P + Q + R
proof -
  have path-sum-bound:
    wp-event ?m ?open-path adversary-initial-state +
    wp-event ?m ?witness-path adversary-initial-state ≤ Q + R
    by (rule add-mono[OF opening-path-bound witness-path-bound])
  have
    wp-event ?m ?prefix adversary-initial-state +
    (wp-event ?m ?open-path adversary-initial-state +
      wp-event ?m ?witness-path adversary-initial-state) ≤ P + (Q + R)
    by (rule add-mono[OF prefix-gap-bound path-sum-bound])
  then show ?thesis
    by (simp add: add.assoc)
qed

```

show ?thesis
by (rule order-trans[OF event-le order-trans[OF union-le sum-bound]])
qed

lemma *checked-staged-security-with-query-prefix-candidate-binding-gap-bound-from-budgets*:
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *i-bound*: $i < \text{rounds}$
shows
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state *A i*)
(checked-staged-security-with-query-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table *i*)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) +
(1::prob) / nnreal (card query-sample-space)

proof –
have *event-le*:
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state *A i*)
(checked-staged-security-with-query-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table *i*)
adversary-initial-state $\leq$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state *A i*)
(checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings *i*)
adversary-initial-state
by (rule wp-event-mono)
(rule
checked-staged-security-with-query-prefix-candidate-binding-gap-imp-header-authenticated)

have *header-bound*:
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state *A i*)
(checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings *i*)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) +
(1::prob) / nnreal (card query-sample-space)
by (rule
checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-bound-from-budgets
[OF *wf controlled i-bound*])

show ?thesis
by (rule order-trans[OF event-le header-bound])
qed

definition *checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at*

$:: \text{nat} \Rightarrow 'f \text{ staged-adversary} \Rightarrow$
 $(((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at}$
 $i \ A \ \text{out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } ((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}) \Rightarrow$
 $(\exists \text{prefix prefix-state raw raw-state.}$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \wedge$
 $\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $\text{adversary-initial-state}) \wedge$
 $\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-header-supported-query-target}$
 $(\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}))))$

definition *checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at*

$:: \text{nat} \Rightarrow 'f \text{ staged-adversary} \Rightarrow$
 $(((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at}$
 $i \ A \ \text{out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } ((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}) \Rightarrow$
 $(\exists \text{prefix prefix-state raw raw-state.}$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \wedge$
 $\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $\text{adversary-initial-state}) \wedge$
 $\text{checked-staged-query-prefix-dynamic-index-hit}$
 $(\text{staged-query-prefix-single-round-header-supported-query-target } i)$

(*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*))))

The preceding actual-alpha event is a diagnostic compatibility predicate: it existentially ranges over query-prefix branches. It should not be bounded directly from query-prefix target budgets unless a separate projection lemma is proved. Bounded public-route arguments should keep the event at the query-prefix data-state layer, or use an explicitly aligned actual-alpha predicate whose query-prefix output is fixed by the run.

definition *checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at*

$:: \text{nat} \Rightarrow 'f \text{ staged-adversary} \Rightarrow$
 $(((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at

$i \ A \ \text{out} \longleftrightarrow$

(*case out of*

None $\Rightarrow \text{False}$

| *Some* ((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
result), *final-state*) $\Rightarrow$

$(\exists \text{ trace-table composition-table prefix prefix-state raw raw-state.}$

Some (*data*, *attacker-state*) $\in$

set-dist

$(\text{execute } (\text{checked-staged-transcript-program } A)$

adversary-initial-state) $\wedge$

Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$

set-dist

$(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$

adversary-initial-state) $\wedge$

checked-staged-query-prefix-dynamic-index-hit

$(\text{staged-query-prefix-candidate-pair-query-target-from-prefix}$

trace-table composition-table)

$(\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state})) \wedge$

$(\text{trace-table}, \text{composition-table}) \notin$

query-header-supported-partial-table-candidates prefix-state

$(\text{sqp-trace-root prefix})$

$(\text{sqp-trace-fri-roots prefix})$

$(\text{sqp-trace-final prefix})$

$(\text{sqp-alphas prefix})$

$(\text{sqp-degree prefix})$

$(\text{sqp-composition-fri-roots prefix})$

$(\text{sqp-composition-final prefix}))$)

definition *checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at*

$:: \text{nat} \Rightarrow 'f \text{ staged-adversary} \Rightarrow$
 $(((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at
i A out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state) $\Rightarrow$
 ($\exists$ trace-table composition-table prefix prefix-state raw raw-state.
 Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\wedge$
 Some (((prefix, prefix-state), raw), raw-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state) $\wedge$
 checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table)
 (Some (((prefix, prefix-state), raw), raw-state)) $\wedge$
 (trace-table, composition-table) $\notin$
 query-header-supported-single-round-partial-table-candidates
 prefix-state
 (sqp-trace-root prefix)
 (sqp-trace-fri-roots prefix)
 (sqp-trace-final prefix)
 (sqp-alphas prefix)
 (sqp-degree prefix)
 (sqp-composition-fri-roots prefix)
 (sqp-composition-final prefix)
 i))

definition *checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-a*

$:: \text{nat} \Rightarrow 'f \text{ staged-adversary} \Rightarrow$
 ((((((('f $\times$ 'f list $\times$ 'f list $\times$ 'f) $\times$
 'f protocol-channel) $\times$ 'f staged-proof-data) $\times$
 'f protocol-channel) $\times$ unit list) $\times$
 'f protocol-channel) option $\Rightarrow$ bool

where

checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
i A out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state) $\Rightarrow$
 ($\exists$ trace-openings composition-openings trace-table composition-table
 prefix prefix-state raw raw-state.
 Some (data, attacker-state) $\in$
 set-dist

(execute (checked-staged-transcript-program A)
 adversary-initial-state) $\wedge$
 Some (((prefix, prefix-state), raw), raw-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state) $\wedge$
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\wedge$
 partial-trace-table-candidate trace-table
 ((replicate rounds []) [i := trace-openings]) $\wedge$
 partial-composition-table-candidate composition-table
 ((replicate rounds []) [i := composition-openings]) $\wedge$
 checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table)
 (Some (((prefix, prefix-state), raw), raw-state)) $\wedge$
 (trace-table, composition-table) $\notin$
 query-header-supported-single-round-partial-table-candidates
 prefix-state
 (sqp-trace-root prefix)
 (sqp-trace-fri-roots prefix)
 (sqp-trace-final prefix)
 (sqp-alphas prefix)
 (sqp-degree prefix)
 (sqp-composition-fri-roots prefix)
 (sqp-composition-final prefix)
 i))

This witness variant has the same diagnostic status: the authenticated openings are carried, but the query-prefix branch is still existential in the actual-alpha output. Use the same-run query-prefix witness-gap predicates for probability bounds.

lemma checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-imp-
assumes gap:
 checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
 i A out
shows
 checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at
 i A out
proof (cases out)
case None

```

then show ?thesis
  using gap
  unfolding
    checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at-def
  by simp
next
  case (Some packed)
  then obtain alpha-prefix alpha-prefix-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state)
    by (cases packed) (auto split: prod.splits)
  from gap obtain trace-openings composition-openings trace-table
    composition-table prefix prefix-state raw raw-state where builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
    and prefix-receive:
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
    and target-hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table)
      (Some (((prefix, prefix-state), raw), raw-state))
    and notin:
    (trace-table, composition-table) ∉
      query-header-supported-single-round-partial-table-candidates
        prefix-state
        (sqp-trace-root prefix)
        (sqp-trace-fri-roots prefix)
        (sqp-trace-final prefix)
        (sqp-alphas prefix)
        (sqp-degree prefix)
        (sqp-composition-fri-roots prefix)
        (sqp-composition-final prefix)
      i
    unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at-def
  by auto
  have body:
  ∃ trace-table composition-table prefix prefix-state raw raw-state.
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)

```

```

    adversary-initial-state) ∧
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state) ∧
  checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-pair-query-target-from-prefix
    trace-table composition-table)
  (Some (((prefix, prefix-state), raw), raw-state)) ∧
  (trace-table, composition-table) ∉
    query-header-supported-single-round-partial-table-candidates
    prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
  i
by (intro exI[of - trace-table] exI[of - composition-table]
  exI[of - prefix] exI[of - prefix-state]
  exI[of - raw] exI[of - raw-state])
  (use builder prefix-receive target-hit notin in simp)
show ?thesis
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at-def
using body by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witnessE:
assumes gap:
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
  i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
obtains trace-openings composition-openings trace-table composition-table
  prefix prefix-state raw raw-state where
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses
    (verifier-state-from-adversary attacker-state)

```


(staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
 partial-trace-table-candidate trace-table
 ((replicate rounds []) [i := trace-openings])
 partial-composition-table-candidate composition-table
 ((replicate rounds []) [i := composition-openings])
 checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table)
 (Some (((prefix, prefix-state), raw), raw-state))
 (trace-table, composition-table) $\notin$
 query-header-supported-single-round-partial-table-candidates
 prefix-state
 (sqp-trace-root prefix)
 (sqp-trace-fri-roots prefix)
 (sqp-trace-final prefix)
 (sqp-alphas prefix)
 (sqp-degree prefix)
 (sqp-composition-fri-roots prefix)
 (sqp-composition-final prefix)
 i
proof –
have body:
 $\exists$ trace-openings composition-openings trace-table composition-table
 prefix prefix-state raw raw-state.
 Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\wedge$
 Some (((prefix, prefix-state), raw), raw-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state) $\wedge$
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)

```

    (staged-composition-final data) ∧
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings]) ∧
    partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings]) ∧
    checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
     trace-table composition-table)
    (Some (((prefix, prefix-state), raw), raw-state)) ∧
    (trace-table, composition-table) ∉
    query-header-supported-single-round-partial-table-candidates
    prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
    i
using gap
unfolding
    checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at-def
by simp
then obtain trace-openings composition-openings trace-table composition-table
    prefix prefix-state raw raw-state where
    Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
     adversary-initial-state)
    Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
     adversary-initial-state)
    (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
    partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])

```

```

checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-pair-query-target-from-prefix
   trace-table composition-table)
  (Some (((prefix, prefix-state), raw), raw-state))
(trace-table, composition-table)  $\notin$ 
query-header-supported-single-round-partial-table-candidates
  prefix-state
  (sqp-trace-root prefix)
  (sqp-trace-fri-roots prefix)
  (sqp-trace-final prefix)
  (sqp-alphas prefix)
  (sqp-degree prefix)
  (sqp-composition-fri-roots prefix)
  (sqp-composition-final prefix)
i
by blast
then show ?thesis
by (rule that)
qed

definition checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at
:: nat  $\Rightarrow$  'f staged-adversary  $\Rightarrow$ 
  ((((((('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
    'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
    'f protocol-channel)  $\times$  unit list)  $\times$ 
    'f protocol-channel) option  $\Rightarrow$  bool

where
checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at
  i A out  $\longleftrightarrow$ 
(case out of
  None  $\Rightarrow$  False
  | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state)  $\Rightarrow$ 
    ( $\exists$  trace-openings composition-openings trace-table composition-table
      prefix prefix-state raw raw-state.
      Some (data, attacker-state)  $\in$ 
        set-dist
          (execute (checked-staged-transcript-program A)
            adversary-initial-state)  $\wedge$ 
      Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
        set-dist
          (execute (checked-staged-query-prefix-receive-with-state A i)
            adversary-initial-state)  $\wedge$ 
      (trace-openings, composition-openings)  $\in$ 
        query-header-supported-partial-opening-witnesses
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))
          (staged-trace-root data)
          (staged-trace-fri-roots data)

```

(staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\wedge$
 partial-trace-table-candidate trace-table
 ((replicate rounds []) [i := trace-openings]) $\wedge$
 partial-composition-table-candidate composition-table
 ((replicate rounds []) [i := composition-openings]) $\wedge$
 checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table)
 (Some (((prefix, prefix-state), raw), raw-state)) $\wedge$
 (trace-table, composition-table) $\notin$
 query-header-supported-partial-table-candidates prefix-state
 (sqp-trace-root prefix)
 (sqp-trace-fri-roots prefix)
 (sqp-trace-final prefix)
 (sqp-alphas prefix)
 (sqp-degree prefix)
 (sqp-composition-fri-roots prefix)
 (sqp-composition-final prefix)))

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-imp-gap:*

assumes *gap:*

checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at

i A out

shows

checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at

i A out

using *gap*

proof (*cases out*)

case *None*

then show *?thesis*

using *gap*

unfolding

checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at-def

by *simp*

next

case (*Some packed*)

then obtain *alpha-prefix alpha-prefix-state data attacker-state result*

final-state where out-eq:

out =

*Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state)*

by (*cases packed*) (*auto split: prod.splits*)

have *gap-body:*

$\exists$ *trace-openings composition-openings trace-table composition-table*

prefix prefix-state raw raw-state.

Some (*data*, *attacker-state*) $\in$
set-dist
 (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\wedge$
Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$
set-dist
 (*execute* (*checked-staged-query-prefix-receive-with-state* *A i*)
adversary-initial-state) $\wedge$
 (*trace-openings*, *composition-openings*) $\in$
query-header-supported-partial-opening-witnesses
 (*verifier-state-from-adversary* *attacker-state*
 (*staged-proof-transcript* *data*))
 (*staged-trace-root* *data*)
 (*staged-trace-fri-roots* *data*)
 (*staged-trace-final* *data*)
 (*staged-alphas* *data*)
 (*staged-degree* *data*)
 (*staged-composition-fri-roots* *data*)
 (*staged-composition-final* *data*) $\wedge$
partial-trace-table-candidate *trace-table*
 ((*replicate* *rounds* []) [*i* := *trace-openings*]) $\wedge$
partial-composition-table-candidate *composition-table*
 ((*replicate* *rounds* []) [*i* := *composition-openings*]) $\wedge$
checked-staged-query-prefix-dynamic-index-hit
 (*staged-query-prefix-candidate-pair-query-target-from-prefix*
trace-table *composition-table*)
 (*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*)) $\wedge$
 (*trace-table*, *composition-table*) $\notin$
query-header-supported-partial-table-candidates *prefix-state*
 (*sqp-trace-root* *prefix*)
 (*sqp-trace-fri-roots* *prefix*)
 (*sqp-trace-final* *prefix*)
 (*sqp-alphas* *prefix*)
 (*sqp-degree* *prefix*)
 (*sqp-composition-fri-roots* *prefix*)
 (*sqp-composition-final* *prefix*)
using *gap*
unfolding *out-eq*
checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at-def
by *simp*
from *gap-body*
obtain *trace-openings* *composition-openings* *trace-table* *composition-table*
prefix *prefix-state* *raw* *raw-state* **where** *builder*:
Some (*data*, *attacker-state*) $\in$
set-dist
 (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state)
and *prefix-receive*:
Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$

```

    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  and target-hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table)
      (Some (((prefix, prefix-state), raw), raw-state))
  and notin:
    (trace-table, composition-table) ∉
      query-header-supported-partial-table-candidates prefix-state
      (sqp-trace-root prefix)
      (sqp-trace-fri-roots prefix)
      (sqp-trace-final prefix)
      (sqp-alphas prefix)
      (sqp-degree prefix)
      (sqp-composition-fri-roots prefix)
      (sqp-composition-final prefix)
  by blast
  have body:
    ∃ trace-table composition-table prefix prefix-state raw raw-state.
      Some (data, attacker-state) ∈
        set-dist
          (execute (checked-staged-transcript-program A)
            adversary-initial-state) ∧
        Some (((prefix, prefix-state), raw), raw-state) ∈
          set-dist
            (execute (checked-staged-query-prefix-receive-with-state A i)
              adversary-initial-state) ∧
        checked-staged-query-prefix-dynamic-index-hit
          (staged-query-prefix-candidate-pair-query-target-from-prefix
            trace-table composition-table)
          (Some (((prefix, prefix-state), raw), raw-state)) ∧
        (trace-table, composition-table) ∉
          query-header-supported-partial-table-candidates prefix-state
          (sqp-trace-root prefix)
          (sqp-trace-fri-roots prefix)
          (sqp-trace-final prefix)
          (sqp-alphas prefix)
          (sqp-degree prefix)
          (sqp-composition-fri-roots prefix)
          (sqp-composition-final prefix)
  by (intro exI[of - trace-table] exI[of - composition-table]
    exI[of - prefix] exI[of - prefix-state]
    exI[of - raw] exI[of - raw-state])
    (use builder prefix-receive target-hit notin in simp)
  show ?thesis
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at-def

```

using *body* by *simp*
qed

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witnessE*:

assumes *gap*:

checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at

i A

(*Some* (((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
result), *final-state*))

obtains *trace-openings composition-openings trace-table composition-table*

prefix prefix-state raw raw-state **where**

Some (*data*, *attacker-state*) ∈

set-dist

(*execute* (*checked-staged-transcript-program A*)
adversary-initial-state)

Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) ∈

set-dist

(*execute* (*checked-staged-query-prefix-receive-with-state A i*)
adversary-initial-state)

(*trace-openings*, *composition-openings*) ∈

query-header-supported-partial-opening-witnesses

(*verifier-state-from-adversary attacker-state*

(*staged-proof-transcript data*))

(*staged-trace-root data*)

(*staged-trace-fri-roots data*)

(*staged-trace-final data*)

(*staged-alphas data*)

(*staged-degree data*)

(*staged-composition-fri-roots data*)

(*staged-composition-final data*)

partial-trace-table-candidate trace-table

((*replicate rounds []*) [*i := trace-openings*])

partial-composition-table-candidate composition-table

((*replicate rounds []*) [*i := composition-openings*])

checked-staged-query-prefix-dynamic-index-hit

(*staged-query-prefix-candidate-pair-query-target-from-prefix*
trace-table composition-table)

(*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*))

(*trace-table*, *composition-table*) ∉

query-header-supported-partial-table-candidates prefix-state

(*sqp-trace-root prefix*)

(*sqp-trace-fri-roots prefix*)

(*sqp-trace-final prefix*)

(*sqp-alphas prefix*)

(*sqp-degree prefix*)

(*sqp-composition-fri-roots prefix*)

(*sqp-composition-final prefix*)

proof —

have *body*:

$\exists$ *trace-openings composition-openings trace-table composition-table*
prefix prefix-state raw raw-state.
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state) ∧
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state) ∧
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data) ∧
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings]) ∧
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings]) ∧
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((prefix, prefix-state), raw), raw-state)) ∧
(trace-table, composition-table) ∉
query-header-supported-partial-table-candidates prefix-state
(sqp-trace-root prefix)
(sqp-trace-fri-roots prefix)
(sqp-trace-final prefix)
(sqp-alphas prefix)
(sqp-degree prefix)
(sqp-composition-fri-roots prefix)
(sqp-composition-final prefix)
using *gap*
unfolding
checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at-def
by *simp*
then obtain *trace-openings composition-openings trace-table composition-table*
prefix prefix-state raw raw-state where
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
Some (((prefix, prefix-state), raw), raw-state) ∈


```

    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
  partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table)
    (Some (((prefix, prefix-state), raw), raw-state))
  (trace-table, composition-table) ∉
    query-header-supported-partial-table-candidates prefix-state
      (sqp-trace-root prefix)
      (sqp-trace-fri-roots prefix)
      (sqp-trace-final prefix)
      (sqp-alphas prefix)
      (sqp-degree prefix)
      (sqp-composition-fri-roots prefix)
      (sqp-composition-final prefix)
  by blast
  then show ?thesis
  by (rule that)
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-imp-single-round-header*

assumes *hit*:

checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix

A trace-table composition-table i

(Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))

shows

checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at

i A

(Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))) ∨

checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at

i A

```

    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
proof –
from
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
  [OF hit]
obtain prefix prefix-state raw raw-state where
  builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and prefix-receive:
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
  and target-hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table)
    (Some (((prefix, prefix-state), raw), raw-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
    [OF hit])
show ?thesis
proof (cases
  (trace-table, composition-table) ∈
    query-header-supported-single-round-partial-table-candidates
    prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
    i)
  case True
  have single-round-hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-single-round-header-supported-query-target i)
      (Some (((prefix, prefix-state), raw), raw-state))
  by (rule
    checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
    [OF True target-hit])
  have
    checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
    i A

```

```

    (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
proof –
  have inner:
     $\exists$  prefix prefix-state raw raw-state.
    Some (data, attacker-state)  $\in$ 
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)  $\wedge$ 
    Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)  $\wedge$ 
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-single-round-header-supported-query-target i)
      (Some (((prefix, prefix-state), raw), raw-state))
  by (intro exI[of - prefix] exI[of - prefix-state]
    exI[of - raw] exI[of - raw-state])
    (use builder prefix-receive single-round-hit in simp)
  show ?thesis
    unfolding
      checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at-def
    using inner by simp
  qed
  then show ?thesis by simp
next
  case False
  have
    checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at
      i A
    (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  proof –
    have inner:
       $\exists$  trace-table composition-table prefix prefix-state raw raw-state.
      Some (data, attacker-state)  $\in$ 
        set-dist
          (execute (checked-staged-transcript-program A)
            adversary-initial-state)  $\wedge$ 
      Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
        set-dist
          (execute (checked-staged-query-prefix-receive-with-state A i)
            adversary-initial-state)  $\wedge$ 
      checked-staged-query-prefix-dynamic-index-hit
        (staged-query-prefix-candidate-pair-query-target-from-prefix
          trace-table composition-table)
        (Some (((prefix, prefix-state), raw), raw-state))  $\wedge$ 
      (trace-table, composition-table)  $\notin$ 
        query-header-supported-single-round-partial-table-candidates

```

```

    prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
  i
  by (intro exI[of - trace-table] exI[of - composition-table]
      exI[of - prefix] exI[of - prefix-state]
      exI[of - raw] exI[of - raw-state])
    (use builder prefix-receive target-hit False in simp)
show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at-def
  using inner by simp
qed
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-with-witness-imp-single-
assumes hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
  A trace-table composition-table i
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
and witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
and trace-candidate:
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
shows
  checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
  i A

```

(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state)) $\vee$
 checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
 i A
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))

proof –

from

checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
 [OF hit]

obtain prefix prefix-state raw raw-state **where**

builder:

Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state)

and prefix-receive:

Some (((prefix, prefix-state), raw), raw-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state)

and target-hit:

checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table)
 (Some (((prefix, prefix-state), raw), raw-state))

by (rule
 checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
 [OF hit])

show ?thesis

proof (cases
 (trace-table, composition-table) $\in$
 query-header-supported-single-round-partial-table-candidates
 prefix-state
 (sqp-trace-root prefix)
 (sqp-trace-fri-roots prefix)
 (sqp-trace-final prefix)
 (sqp-alphas prefix)
 (sqp-degree prefix)
 (sqp-composition-fri-roots prefix)
 (sqp-composition-final prefix)
 i)

case True

have single-round-hit:

checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-single-round-header-supported-query-target i)
 (Some (((prefix, prefix-state), raw), raw-state))

by (rule
 checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit

```

      [OF True target-hit])
have
  checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
    i A
    (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
proof –
  have inner:
    ∃ prefix prefix-state raw raw-state.
      Some (data, attacker-state) ∈
        set-dist
          (execute (checked-staged-transcript-program A)
            adversary-initial-state) ∧
      Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
          (execute (checked-staged-query-prefix-receive-with-state A i)
            adversary-initial-state) ∧
      checked-staged-query-prefix-dynamic-index-hit
        (staged-query-prefix-single-round-header-supported-query-target i)
        (Some (((prefix, prefix-state), raw), raw-state))
  by (intro exI[of - prefix] exI[of - prefix-state]
    exI[of - raw] exI[of - raw-state])
    (use builder prefix-receive single-round-hit in simp)
  show ?thesis
    unfolding
      checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at-def
    using inner by simp
  qed
  then show ?thesis by simp
next
case False
have
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
    i A
    (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
proof –
  have inner:
    ∃ trace-openings composition-openings trace-table composition-table
      prefix prefix-state raw raw-state.
      Some (data, attacker-state) ∈
        set-dist
          (execute (checked-staged-transcript-program A)
            adversary-initial-state) ∧
      Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
          (execute (checked-staged-query-prefix-receive-with-state A i)
            adversary-initial-state) ∧
      (trace-openings, composition-openings) ∈

```

```

query-header-supported-partial-opening-witnesses
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data) ∧
partial-trace-table-candidate trace-table
  ((replicate rounds [] [i := trace-openings]) ∧
partial-composition-table-candidate composition-table
  ((replicate rounds [] [i := composition-openings]) ∧
checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-pair-query-target-from-prefix
    trace-table composition-table)
  (Some (((prefix, prefix-state), raw), raw-state)) ∧
(trace-table, composition-table) ∉
query-header-supported-single-round-partial-table-candidates
  prefix-state
  (sqp-trace-root prefix)
  (sqp-trace-fri-roots prefix)
  (sqp-trace-final prefix)
  (sqp-alphas prefix)
  (sqp-degree prefix)
  (sqp-composition-fri-roots prefix)
  (sqp-composition-final prefix)
  i
by (intro exI[of - trace-openings] exI[of - composition-openings]
  exI[of - trace-table] exI[of - composition-table]
  exI[of - prefix] exI[of - prefix-state]
  exI[of - raw] exI[of - raw-state])
  (use builder prefix-receive witness trace-candidate comp-candidate
    target-hit False in simp)
show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at-def
  using inner by simp
qed
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-with-witness-imp-header
assumes hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
  A trace-table composition-table i
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),

```

```

    result), final-state))
and witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
and trace-candidate:
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
shows
  checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at
    i A
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state)) ∨
  checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at
    i A
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
proof —
from
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
    [OF hit]
obtain prefix prefix-state raw raw-state where
  builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
and prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
and target-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table)
    (Some (((prefix, prefix-state), raw), raw-state))
by (rule

```



```

      checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
      [OF hit])
show ?thesis
proof (cases
  (trace-table, composition-table) ∈
    query-header-supported-partial-table-candidates prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix))
case True
have header-hit:
  checked-staged-query-prefix-dynamic-index-hit
  staged-query-prefix-header-supported-query-target
  (Some (((prefix, prefix-state), raw), raw-state))
by (rule
  checked-staged-query-prefix-candidate-pair-hit-imp-header-supported-target-hit
  [OF True target-hit])
have
  checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at
  i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
proof –
have inner:
  ∃ prefix prefix-state raw raw-state.
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state) ∧
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state) ∧
  checked-staged-query-prefix-dynamic-index-hit
  staged-query-prefix-header-supported-query-target
  (Some (((prefix, prefix-state), raw), raw-state))
by (intro exI[of - prefix] exI[of - prefix-state]
  exI[of - raw] exI[of - raw-state])
  (use builder prefix-receive header-hit in simp)
show ?thesis
unfolding
  checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at-def
using inner by simp
qed
then show ?thesis by simp

```

```

next
case False
have
checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at
  i A
  (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
proof -
have inner:
  ∃ trace-openings composition-openings trace-table composition-table
    prefix prefix-state raw raw-state.
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state) ∧
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state) ∧
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data) ∧
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings]) ∧
  partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings]) ∧
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table)
    (Some (((prefix, prefix-state), raw), raw-state)) ∧
  (trace-table, composition-table) ∉
    query-header-supported-partial-table-candidates prefix-state
      (sqp-trace-root prefix)
      (sqp-trace-fri-roots prefix)
      (sqp-trace-final prefix)
      (sqp-alphas prefix)
      (sqp-degree prefix)
      (sqp-composition-fri-roots prefix)
      (sqp-composition-final prefix)
by (intro exI[of - trace-openings] exI[of - composition-openings]
  exI[of - trace-table] exI[of - composition-table])

```

```

    exI[of - prefix] exI[of - prefix-state]
    exI[of - raw] exI[of - raw-state])
  (use builder prefix-receive witness trace-candidate comp-candidate
   target-hit False in simp)
show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at-def
  using inner by simp
qed
then show ?thesis by simp
qed
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at*

assumes *hit*:

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
i A

(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))

and *i-bound*: $i < \text{rounds}$

and *trace-low*:

$\bigwedge \text{trace-openings composition-openings trace-table}$
composition-table.

(trace-openings, composition-openings) $\in$
query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\implies$

partial-trace-table-candidate trace-table

((replicate rounds []) [i := trace-openings]) $\implies$

partial-composition-table-candidate composition-table

((replicate rounds []) [i := composition-openings]) $\implies$

trace-table-low-degree trace-table

and *comp-low*:

$\bigwedge \text{trace-openings composition-openings trace-table}$
composition-table.

(trace-openings, composition-openings) $\in$
query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)

```

    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)  $\implies$ 
partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])  $\implies$ 
partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])  $\implies$ 
composition-table-low-degree maxDegree composition-table
and not-all:
 $\wedge$  trace-openings composition-openings trace-table
    composition-table prefix prefix-state.
(trace-openings, composition-openings)  $\in$ 
    query-header-supported-partial-opening-witnesses
    (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)  $\implies$ 
partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])  $\implies$ 
partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])  $\implies$ 
Some ((prefix, prefix-state), prefix-state)  $\in$ 
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
    adversary-initial-state)  $\implies$ 
 $\neg$  all-queries-consistent trace-table composition-table
    (sqp-alphas prefix)
shows
( $\exists$  result' final-state'.
    partial-merkle-inconsistency-bad
    (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
    (Some (result', final-state')))  $\vee$ 
checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at
    i A
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state)))  $\vee$ 
checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at
    i A
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state)))
proof -
let ?s =

```

```

    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at-ca
    [OF hit i-bound]
show ?thesis
proof
    assume
         $\exists \text{result}' \text{ final-state}'.$ 
        partial-merkle-inconsistency-bad ?s (Some (result', final-state'))
    then show ?thesis by simp
next
    assume candidates:
         $\exists \text{trace-openings composition-openings trace-table}$ 
        composition-table.
        (trace-openings, composition-openings)  $\in$ 
        query-header-supported-partial-opening-witnesses ?s
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)
        (staged-composition-fri-roots data)
        (staged-composition-final data)  $\wedge$ 
        checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
        A
        ((replicate rounds []) [i := trace-openings])
        ((replicate rounds []) [i := composition-openings])
        i
        (Some (((((alpha-prefix, alpha-prefix-state), data),
            attacker-state), result), final-state)))  $\wedge$ 
        partial-trace-table-candidate trace-table
        ((replicate rounds []) [i := trace-openings])  $\wedge$ 
        partial-composition-table-candidate composition-table
        ((replicate rounds []) [i := composition-openings])
    then obtain trace-openings composition-openings trace-table
        composition-table where witness:
        (trace-openings, composition-openings)  $\in$ 
        query-header-supported-partial-opening-witnesses ?s
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)
        (staged-composition-fri-roots data)
        (staged-composition-final data)
    and opening-hit:
        checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
        A

```

```

    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
  i
  (Some (((alpha-prefix, alpha-prefix-state), data),
    attacker-state), result), final-state))
and trace-candidate:
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
by blast
have pair-hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
  A trace-table composition-table i
  (Some (((alpha-prefix, alpha-prefix-state), data),
    attacker-state), result), final-state))
by (rule
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-imp-candidate-pair-que
  [OF opening-hit trace-candidate comp-candidate
    trace-low[OF witness trace-candidate comp-candidate]
    comp-low[OF witness trace-candidate comp-candidate]])
  (rule not-all[OF witness trace-candidate comp-candidate])
have
  checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at
  i A
  (Some (((alpha-prefix, alpha-prefix-state), data),
    attacker-state), result), final-state)) ∨
  checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at
  i A
  (Some (((alpha-prefix, alpha-prefix-state), data),
    attacker-state), result), final-state))
by (rule
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-with-witness-imp-header-ta
  [OF pair-hit witness trace-candidate comp-candidate])
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
assumes hit:
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
  i A
  (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
and i-bound: i < rounds
and trace-low:
  ∧ trace-openings composition-openings trace-table
  composition-table.

```

$(\text{trace-openings}, \text{composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $(\text{staged-alphas data})$
 $(\text{staged-degree data})$
 $(\text{staged-composition-fri-roots data})$
 $(\text{staged-composition-final data}) \implies$
 $\text{partial-trace-table-candidate trace-table}$
 $((\text{replicate rounds } []) [i := \text{trace-openings}]) \implies$
 $\text{partial-composition-table-candidate composition-table}$
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \implies$
 $\text{trace-table-low-degree trace-table}$
and comp-low:
 $\bigwedge \text{trace-openings composition-openings trace-table}$
 $\text{composition-table.}$
 $(\text{trace-openings}, \text{composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $(\text{staged-alphas data})$
 $(\text{staged-degree data})$
 $(\text{staged-composition-fri-roots data})$
 $(\text{staged-composition-final data}) \implies$
 $\text{partial-trace-table-candidate trace-table}$
 $((\text{replicate rounds } []) [i := \text{trace-openings}]) \implies$
 $\text{partial-composition-table-candidate composition-table}$
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \implies$
 $\text{composition-table-low-degree maxDegree composition-table}$
and not-all:
 $\bigwedge \text{trace-openings composition-openings trace-table}$
 $\text{composition-table prefix prefix-state.}$
 $(\text{trace-openings}, \text{composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $(\text{staged-alphas data})$
 $(\text{staged-degree data})$
 $(\text{staged-composition-fri-roots data})$
 $(\text{staged-composition-final data}) \implies$

partial-trace-table-candidate trace-table
 $((\text{replicate rounds } []) [i := \text{trace-openings}]) \implies$
partial-composition-table-candidate composition-table
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \implies$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\neg \text{all-queries-consistent trace-table composition-table}$
 $(\text{sqp-alphas prefix})$

shows

$(\exists \text{result}' \text{ final-state'}. \text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{Some } (\text{result}', \text{final-state'}))) \vee$
 $\text{checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at}$
 $i \ A$
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}))) \vee$
 $\text{checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at}$
 $i \ A$
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state})))$

proof —

let $?s =$

$\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})$

from

$\text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at-ca}$
 $[OF \text{ hit } i\text{-bound}]$

show $?thesis$

proof

assume

$\exists \text{result}' \text{ final-state'}. \text{partial-merkle-inconsistency-bad } ?s (\text{Some } (\text{result}', \text{final-state'}))$

then show $?thesis$ **by** *simp*

next

assume *candidates:*

$\exists \text{trace-openings composition-openings trace-table}$
 $\text{composition-table.}$
 $(\text{trace-openings}, \text{composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses } ?s$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $(\text{staged-alphas data})$
 $(\text{staged-degree data})$
 $(\text{staged-composition-fri-roots data})$


```

    (staged-composition-final data) ∧
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i
(Some (((alpha-prefix, alpha-prefix-state), data),
  attacker-state), result), final-state)) ∧
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings]) ∧
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])
then obtain trace-openings composition-openings trace-table
composition-table where witness:
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses ?s
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
and opening-hit:
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i
(Some (((alpha-prefix, alpha-prefix-state), data),
  attacker-state), result), final-state))
and trace-candidate:
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings])
and comp-candidate:
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])
by blast
have pair-hit:
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
A trace-table composition-table i
(Some (((alpha-prefix, alpha-prefix-state), data),
  attacker-state), result), final-state))
by (rule
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-imp-candidate-pair-que
[OF opening-hit trace-candidate comp-candidate
  trace-low[OF witness trace-candidate comp-candidate]
  comp-low[OF witness trace-candidate comp-candidate]])
(rule not-all[OF witness trace-candidate comp-candidate])

```

```

have
  checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
    i A
    (Some (((((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state)) ∨
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
    i A
    (Some (((((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-with-witness-imp-single-round-candidate-binding-gap-witness-at
    [OF pair-hit witness trace-candidate comp-candidate])
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-imp-header-target-or-binding-gap-witness-at
assumes hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
  A trace-table composition-table i
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
shows
  checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at
    i A
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state)) ∨
  checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at
    i A
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
proof —
from
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
  [OF hit]
obtain prefix prefix-state raw raw-state where
  i-bound: i < rounds
and builder:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
and prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
and target-hit:
  checked-staged-query-prefix-dynamic-index-hit

```

```

    (staged-query-prefix-candidate-pair-query-target-from-prefix
     trace-table composition-table)
    (Some (((prefix, prefix-state), raw), raw-state))
  by (rule
      checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
      [OF hit])
  show ?thesis
  proof (cases
        (trace-table, composition-table) ∈
        query-header-supported-partial-table-candidates prefix-state
        (sqp-trace-root prefix)
        (sqp-trace-fri-roots prefix)
        (sqp-trace-final prefix)
        (sqp-alphas prefix)
        (sqp-degree prefix)
        (sqp-composition-fri-roots prefix)
        (sqp-composition-final prefix))
      case True
    have header-hit:
      checked-staged-query-prefix-dynamic-index-hit
      staged-query-prefix-header-supported-query-target
      (Some (((prefix, prefix-state), raw), raw-state))
    by (rule
        checked-staged-query-prefix-candidate-pair-hit-imp-header-supported-target-hit
        [OF True target-hit])
    have
      checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at
      i A
      (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
                result), final-state))
    proof -
      have inner:
        ∃ prefix prefix-state raw raw-state.
        Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
                 adversary-initial-state) ∧
        checked-staged-query-prefix-dynamic-index-hit
        staged-query-prefix-header-supported-query-target
        (Some (((prefix, prefix-state), raw), raw-state))
      by (intro exI[of - prefix] exI[of - prefix-state]
          exI[of - raw] exI[of - raw-state])
          (use prefix-receive header-hit in simp)
      show ?thesis
      unfolding
        checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at-def
      using builder inner by simp
    qed
  then show ?thesis by simp

```

```

next
  case False
  have
    checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at
      i A
      (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
  proof –
    have inner:
       $\exists$  trace-table composition-table prefix prefix-state raw raw-state.
        Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
          set-dist
            (execute (checked-staged-query-prefix-receive-with-state A i)
              adversary-initial-state)  $\wedge$ 
            checked-staged-query-prefix-dynamic-index-hit
              (staged-query-prefix-candidate-pair-query-target-from-prefix
                trace-table composition-table)
              (Some (((prefix, prefix-state), raw), raw-state))  $\wedge$ 
            (trace-table, composition-table)  $\notin$ 
              query-header-supported-partial-table-candidates prefix-state
                (sqp-trace-root prefix)
                (sqp-trace-fri-roots prefix)
                (sqp-trace-final prefix)
                (sqp-alphas prefix)
                (sqp-degree prefix)
                (sqp-composition-fri-roots prefix)
                (sqp-composition-final prefix)
            by (intro exI[of - trace-table] exI[of - composition-table]
              exI[of - prefix] exI[of - prefix-state]
              exI[of - raw] exI[of - raw-state])
              (use prefix-receive target-hit False in simp)
            show ?thesis
              unfolding
                checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at-def
              using builder inner by simp
            qed
          then show ?thesis by simp
        qed
      qed

```

```

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
assumes hit:
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
    i A
    (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
and i-bound: i < rounds
and trace-low:
   $\bigwedge$  trace-openings composition-openings trace-table

```

composition-table.
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\implies$
 partial-trace-table-candidate trace-table
 ((replicate rounds []) [i := trace-openings]) $\implies$
 partial-composition-table-candidate composition-table
 ((replicate rounds []) [i := composition-openings]) $\implies$
 trace-table-low-degree trace-table
and comp-low:
 $\bigwedge$ trace-openings composition-openings trace-table
 composition-table.
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\implies$
 partial-trace-table-candidate trace-table
 ((replicate rounds []) [i := trace-openings]) $\implies$
 partial-composition-table-candidate composition-table
 ((replicate rounds []) [i := composition-openings]) $\implies$
 composition-table-low-degree maxDegree composition-table
and not-all:
 $\bigwedge$ trace-openings composition-openings trace-table
 composition-table prefix prefix-state.
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)

$(\text{staged-composition-final data}) \implies$
 $\text{partial-trace-table-candidate trace-table}$
 $((\text{replicate rounds } []) [i := \text{trace-openings}]) \implies$
 $\text{partial-composition-table-candidate composition-table}$
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \implies$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\neg \text{all-queries-consistent trace-table composition-table}$
 $(\text{sqp-alphas prefix})$

shows

$(\exists \text{result}' \text{ final-state'}. \text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{Some } (\text{result}', \text{final-state'}))) \vee$
 $\text{checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at}$
 $i \ A$
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}))) \vee$
 $\text{checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at}$
 $i \ A$
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state})))$

proof —

have *base*:

$(\exists \text{result}' \text{ final-state'}. \text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{Some } (\text{result}', \text{final-state'}))) \vee$
 $(\exists \text{trace-table composition-table}. \text{checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix}$
 $A \ \text{trace-table composition-table } i$
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}),$
 $\text{attacker-state}), \text{result}), \text{final-state}))))$

by (rule

$\text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at-}$
 $[OF \ \text{hit } i\text{-bound trace-low comp-low not-all}])$

then show *?thesis*

proof

assume

$\exists \text{result}' \text{ final-state'}. \text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{Some } (\text{result}', \text{final-state'}))$

then show *?thesis* **by** *simp*

```

next
  assume
     $\exists$  trace-table composition-table.
      checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
      A trace-table composition-table i
      (Some ((((alpha-prefix, alpha-prefix-state), data),
        attacker-state), result), final-state))
  then obtain trace-table composition-table where pair-hit:
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
    A trace-table composition-table i
    (Some ((((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state))
  by blast
  have
    checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at
    i A
    (Some ((((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state))  $\vee$ 
    checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at
    i A
    (Some ((((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-imp-header-target-or-binding-gap
    [OF pair-hit])
  then show ?thesis by simp
  qed
qed

end

end

```

```

theory Staged-Security-Experiment-Composition-Query-Single-Round
  imports Staged-Security-Experiment-Composition-Query-Binding-Gap
begin

```

Same-run single-round candidate binding for the composition query branch.

The actual-alpha diagnostic events in the previous layer intentionally record enough data to relate a candidate pair to a query-prefix target, but they are too broad for Merkle path-output accounting because the query-prefix branch is existential. This layer states the single-round binding gap directly over the query-prefix data-state experiment, where prefix state, sampled query, verifier continuation, and authenticated openings are from the same run.

```

context soundness
begin

```

definition *checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at*
where

checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
 $i \text{ out} \longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((prefix, prefix-state), raw), raw-state), continuation),
 final-state) $\Rightarrow$
 checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-single-round-header-supported-query-target i)
 (Some (((prefix, prefix-state), raw), raw-state)))

lemma *checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-iff-dynamic:*
checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at

$i \text{ out} \longleftrightarrow$
checked-staged-security-with-query-prefix-dynamic-index-hit
 (staged-query-prefix-single-round-header-supported-query-target i) out

unfolding

checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-def
checked-staged-security-with-query-prefix-dynamic-index-hit-def
by (cases out) (auto split: prod.splits)

lemma *checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-bound:*
assumes prefix-bound:

wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-single-round-header-supported-query-target i))
 adversary-initial-state $\leq$ B

shows

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
 i)
 adversary-initial-state $\leq$ B

unfolding

checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-iff-dynamic
by (rule checked-staged-security-with-query-prefix-dynamic-index-hit-bound
 [OF prefix-bound])

lemma *staged-query-prefix-candidate-pair-query-target-from-prefix-empty-if-all-queries-consistent:*

assumes all-queries:

all-queries-consistent trace-table composition-table (sqp-alphas prefix)

shows

staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
 composition-table prefix prefix-state = {}

using all-queries

unfolding staged-query-prefix-candidate-pair-query-target-from-prefix-def
 query-sampling-success-space-def

by simp

lemma *checked-staged-query-prefix-candidate-pair-hit-from-prefix-false-if-all-queries-consistent:*

assumes *all-queries:*

all-queries-consistent trace-table composition-table (sqp-alphas prefix)

shows

$\neg$ *checked-staged-query-prefix-dynamic-index-hit*
(staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
composition-table)
(Some (((prefix, prefix-state), raw), raw-state))

using

staged-query-prefix-candidate-pair-query-target-from-prefix-empty-if-all-queries-consistent
[OF all-queries, of prefix-state]

unfolding *checked-staged-query-prefix-dynamic-index-hit-def*

by *simp*

definition *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side*

where

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side

trace-openings composition-openings trace-table composition-table i out

$\longleftrightarrow$

(case out of

None $\Rightarrow$ False

| *Some (((((prefix, prefix-state), raw), raw-state),*
((data, attacker-state), result)), final-state) $\Rightarrow$

partial-trace-table-candidate trace-table

((replicate rounds []) [i := trace-openings]) $\wedge$

partial-composition-table-candidate composition-table

((replicate rounds []) [i := composition-openings]) $\wedge$

(trace-table, composition-table) $\notin$

query-header-supported-single-round-partial-table-candidates

prefix-state

(sqp-trace-root prefix)

(sqp-trace-fri-roots prefix)

(sqp-trace-final prefix)

(sqp-alphas prefix)

(sqp-degree prefix)

(sqp-composition-fri-roots prefix)

(sqp-composition-final prefix)

i)

definition *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at*

where

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at

trace-openings composition-openings trace-table composition-table i out

$\longleftrightarrow$

(case out of

None $\Rightarrow$ False

| *Some (((((prefix, prefix-state), raw), raw-state),*
((data, attacker-state), result)), final-state) $\Rightarrow$

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i out $\wedge$
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings]) $\wedge$
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings]) $\wedge$
(trace-table, composition-table) $\notin$
query-header-supported-single-round-partial-table-candidates
prefix-state
(sqp-trace-root prefix)
(sqp-trace-fri-roots prefix)
(sqp-trace-final prefix)
(sqp-alphas prefix)
(sqp-degree prefix)
(sqp-composition-fri-roots prefix)
(sqp-composition-final prefix)
i)

definition *checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at*
where

checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out
 $\longleftrightarrow$
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i out $\wedge$
checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
trace-openings composition-openings i out

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-iff-fixed-header-component:*

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out
 $\longleftrightarrow$
checked-staged-security-with-query-prefix-fixed-header-component
(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i)
trace-openings composition-openings i out

unfolding

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at-def
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side-def
checked-staged-security-with-query-prefix-fixed-header-component-def
by (*cases out*) (*auto split: prod.splits*)

lemma *checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-imp-current-prefix-authen*
assumes *gap:*

checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out

shows

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
i out

```

using gap
unfolding
  checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at-def
  checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-def
by blast

lemma checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-bound-from-dynamic-target
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i < \text{rounds}$ 
and prefix-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-prefix-authenticated-opening-target)
  adversary-initial-state  $\leq B$ 
shows
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
    trace-openings composition-openings trace-table composition-table i)
  adversary-initial-state  $\leq B$ 
proof –
have event-le:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
    trace-openings composition-openings trace-table composition-table i)
  adversary-initial-state  $\leq$ 
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
    i)
  adversary-initial-state
by (rule wp-event-mono)
  (rule
    checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-imp-current-prefix-authenticated-hit-at)
have current-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
    i)
  adversary-initial-state  $\leq B$ 
by (rule
  checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target
    [OF wf controlled i-bound prefix-bound])
show ?thesis
by (rule order-trans[OF event-le current-bound])
qed

```

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-imp-single-round-header*

assumes *hit*:

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit

trace-openings composition-openings i

*(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))*

and *trace-candidate*:

partial-trace-table-candidate trace-table

((replicate rounds []) [i := trace-openings])

and *comp-candidate*:

partial-composition-table-candidate composition-table

((replicate rounds []) [i := composition-openings])

and *trace-low*: *trace-table-low-degree trace-table*

and *comp-low*: *composition-table-low-degree maxDegree composition-table*

and *not-all*:

¬ all-queries-consistent trace-table composition-table

(sqp-alphas prefix)

shows

checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at

i

*(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state)) ∨*

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at

trace-openings composition-openings trace-table composition-table i

*(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))*

proof –

from *hit* **have** *opening-target*:

index (to-nat raw) ∈

staged-query-prefix-candidate-opening-query-target-from-prefix

((replicate rounds []) [i := trace-openings])

((replicate rounds []) [i := composition-openings])

i prefix prefix-state

by *(rule*

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitE)

have *opening-subset*:

staged-query-prefix-candidate-opening-query-target-from-prefix

((replicate rounds []) [i := trace-openings])

((replicate rounds []) [i := composition-openings])

i prefix prefix-state ⊆

staged-query-prefix-candidate-pair-query-target-from-prefix trace-table

composition-table prefix prefix-state

by *(rule*

staged-query-prefix-candidate-opening-query-target-from-prefix-subset-candidate-pair-from-prefix

[OF trace-candidate comp-candidate trace-low comp-low not-all])

have *pair-target-hit*:

checked-staged-query-prefix-dynamic-index-hit

(staged-query-prefix-candidate-pair-query-target-from-prefix

trace-table composition-table)

```

    (Some (((prefix, prefix-state), raw), raw-state))
using opening-target opening-subset
unfolding checked-staged-query-prefix-dynamic-index-hit-def by auto
show ?thesis
proof (cases
  (trace-table, composition-table) ∈
    query-header-supported-single-round-partial-table-candidates
    prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
  i)
case True
have target:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-single-round-header-supported-query-target i)
    (Some (((prefix, prefix-state), raw), raw-state))
by (rule
  checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
  [OF True pair-target-hit])
have
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
    i
    (Some ((((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-def
using target by simp
then show ?thesis by simp
next
case False
have
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
    trace-openings composition-openings trace-table composition-table i
    (Some ((((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at-def
using hit trace-candidate comp-candidate False by simp
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-current-imp-prefix-gap-or-openings
assumes wf: staged-budget-wellformed budgets

```

and *controlled*: staged-adversary-controlled budgets A
and *i-bound*: $i < \text{rounds}$
and *support*:
 $\text{out} \in$
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 adversary-initial-state)
and *current*:
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 (*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side*
 trace-openings composition-openings trace-table composition-table i)
 trace-openings composition-openings i out
shows
 checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
 trace-openings composition-openings trace-table composition-table i $\text{out} \vee$
 checked-staged-security-with-query-prefix-opening-path-output-hit
 trace-openings composition-openings out
proof —
 have *side*:
 checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
 trace-openings composition-openings trace-table composition-table i out
 and *current-hit*:
 checked-staged-security-with-query-prefix-authenticated-opening-hit
 trace-openings composition-openings i out
 using *current*
 unfolding
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component-def
 checked-staged-security-with-query-prefix-fixed-header-component-def
 by *simp-all*
 have *split*:
 checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
 trace-openings composition-openings i $\text{out} \vee$
 checked-staged-security-with-query-prefix-opening-path-output-hit
 trace-openings composition-openings out
 by (*rule*
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component-imp-prefix-or-path-output-on-sup
 [*OF wf controlled i-bound support current*])
then show *?thesis*
proof
 assume *prefix*:
 checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
 trace-openings composition-openings i out
 have
 checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
 trace-openings composition-openings trace-table composition-table i out
 unfolding
 checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at-def
 using *side prefix* **by** *simp*

```

    then show ?thesis by simp
next
assume
  checked-staged-security-with-query-prefix-opening-path-output-hit
  trace-openings composition-openings out
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-imp-current-or-path-output-on-s
assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and support:
    out ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-query-prefix-data-state A i)
          adversary-initial-state)
  and gap:
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
    trace-openings composition-openings trace-table composition-table i out
shows
  checked-staged-security-with-query-prefix-fixed-current-authenticated-component
  (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
    trace-openings composition-openings trace-table composition-table i)
  trace-openings composition-openings i out ∨
  checked-staged-security-with-query-prefix-header-witness-path-output-hit
  (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
    trace-openings composition-openings trace-table composition-table i)
  trace-openings composition-openings i out
proof —
  have fixed:
    checked-staged-security-with-query-prefix-fixed-header-component
    (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i)
    trace-openings composition-openings i out
  using gap
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-iff-fixed-header-component
  .
  show ?thesis
  by (rule
    checked-staged-security-with-query-prefix-fixed-header-component-imp-current-or-witness-path-output-on-support
    [OF wf controlled i-bound support fixed])
qed

lemma checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-imp-prefix-gap-or-path-outputs-o
assumes wf: staged-budget-wellformed budgets

```

and *controlled*: staged-adversary-controlled budgets A
and *i-bound*: $i < \text{rounds}$
and *support*:
 $\text{out} \in$
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 adversary-initial-state)
and *gap*:
 checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
 trace-openings composition-openings trace-table composition-table i out
shows
 checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
 trace-openings composition-openings trace-table composition-table i $\text{out} \vee$
 checked-staged-security-with-query-prefix-opening-path-output-hit
 trace-openings composition-openings $\text{out} \vee$
 checked-staged-security-with-query-prefix-header-witness-path-output-hit
 (*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side*
 trace-openings composition-openings trace-table composition-table i)
 trace-openings composition-openings i out
proof —
 have *split*:
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 (*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side*
 trace-openings composition-openings trace-table composition-table i)
 trace-openings composition-openings i $\text{out} \vee$
 checked-staged-security-with-query-prefix-header-witness-path-output-hit
 (*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side*
 trace-openings composition-openings trace-table composition-table i)
 trace-openings composition-openings i out
 by (*rule*
 checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-imp-current-or-path-output-on-supp
 [*OF wf controlled i-bound support gap*])
then show *?thesis*
proof
 assume *current*:
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 (*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side*
 trace-openings composition-openings trace-table composition-table i)
 trace-openings composition-openings i out
 have
 checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
 trace-openings composition-openings trace-table composition-table i $\text{out} \vee$
 checked-staged-security-with-query-prefix-opening-path-output-hit
 trace-openings composition-openings out
 by (*rule*
 checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-current-imp-prefix-gap-or-opening
 [*OF wf controlled i-bound support current*])
then show *?thesis* **by** *blast*


```

next
  assume
    checked-staged-security-with-query-prefix-header-witness-path-output-hit
    (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i)
    trace-openings composition-openings i out
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-bound-from-prefix-target-and-pa
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i < \text{rounds}$ 
and prefix-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-prefix-authenticated-opening-target)
  adversary-initial-state  $\leq P$ 
and opening-path-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-opening-path-output-hit
      trace-openings composition-openings)
  adversary-initial-state  $\leq Q$ 
and witness-path-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-path-output-hit
      (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
        trace-openings composition-openings trace-table composition-table i)
      trace-openings composition-openings i)
  adversary-initial-state  $\leq R$ 
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
      trace-openings composition-openings trace-table composition-table i)
  adversary-initial-state  $\leq P + Q + R$ 
proof -
  let ?m =
    checked-staged-security-experiment-with-query-prefix-data-state A i
  let ?prefix =
    checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
    trace-openings composition-openings trace-table composition-table i
  let ?open-path =
    checked-staged-security-with-query-prefix-opening-path-output-hit
    trace-openings composition-openings
  let ?witness-path =

```

checked-staged-security-with-query-prefix-header-witness-path-output-hit
 (*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side*
 trace-openings composition-openings trace-table composition-table i)
trace-openings composition-openings i
have *event-le*:
wp-event ?m
 (*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at*
 trace-openings composition-openings trace-table composition-table i)
adversary-initial-state $\leq$
wp-event ?m
 ($\lambda out. ?prefix\ out \vee ?open-path\ out \vee ?witness-path\ out$)
adversary-initial-state
by (rule *wp-event-mono-on-support*)
 (rule
 checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-imp-prefix-gap-or-path-outputs-on-s
 [*OF wf controlled i-bound*])
have *union-le*:
wp-event ?m
 ($\lambda out. ?prefix\ out \vee ?open-path\ out \vee ?witness-path\ out$)
adversary-initial-state $\leq$
wp-event ?m ?prefix adversary-initial-state +
 (*wp-event ?m ?open-path adversary-initial-state* +
 wp-event ?m ?witness-path adversary-initial-state)
proof –
have
wp-event ?m
 ($\lambda out. ?prefix\ out \vee ?open-path\ out \vee ?witness-path\ out$)
adversary-initial-state $\leq$
wp-event ?m ?prefix adversary-initial-state +
wp-event ?m
 ($\lambda out. ?open-path\ out \vee ?witness-path\ out$)
adversary-initial-state
by (rule *wp-event-union-bound*)
also have
 ... $\leq$
wp-event ?m ?prefix adversary-initial-state +
 (*wp-event ?m ?open-path adversary-initial-state* +
 wp-event ?m ?witness-path adversary-initial-state)
by (intro *add-mono order-refl wp-event-union-bound*)
finally show *?thesis* .
qed
have *prefix-gap-bound*:
wp-event ?m ?prefix adversary-initial-state $\leq P$
by (rule
 checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-bound-from-dynamic-target
 [*OF wf controlled i-bound prefix-bound*])
have *sum-bound*:
wp-event ?m ?prefix adversary-initial-state +
 (*wp-event ?m ?open-path adversary-initial-state* +

$wp\text{-}event\ ?m\ ?witness\text{-}path\ adversary\text{-}initial\text{-}state) \leq P + Q + R$
proof –
have *path-sum-bound*:
 $wp\text{-}event\ ?m\ ?open\text{-}path\ adversary\text{-}initial\text{-}state +$
 $wp\text{-}event\ ?m\ ?witness\text{-}path\ adversary\text{-}initial\text{-}state \leq Q + R$
by (rule *add-mono*[*OF opening-path-bound witness-path-bound*])
have
 $wp\text{-}event\ ?m\ ?prefix\ adversary\text{-}initial\text{-}state +$
 $(wp\text{-}event\ ?m\ ?open\text{-}path\ adversary\text{-}initial\text{-}state +$
 $wp\text{-}event\ ?m\ ?witness\text{-}path\ adversary\text{-}initial\text{-}state) \leq P + (Q + R)$
by (rule *add-mono*[*OF prefix-gap-bound path-sum-bound*])
then show *?thesis*
by (*simp add: add.assoc*)
qed
show *?thesis*
by (rule *order-trans*[*OF event-le order-trans*[*OF union-le sum-bound*]])
qed

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-bound-from-witness-path-output*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *path-bound*:
 $wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}query\text{-}prefix\text{-}data\text{-}state\ A\ i)$
 $(checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}header\text{-}witness\text{-}path\text{-}output\text{-}hit$
 $(checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}single\text{-}round\text{-}candidate\text{-}binding\text{-}gap\text{-}side$
 $trace\text{-}openings\ composition\text{-}openings\ trace\text{-}table\ composition\text{-}table\ i)$
 $trace\text{-}openings\ composition\text{-}openings\ i)$
 $adversary\text{-}initial\text{-}state \leq Q$
shows
 $wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}query\text{-}prefix\text{-}data\text{-}state\ A\ i)$
 $(checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}single\text{-}round\text{-}candidate\text{-}binding\text{-}gap\text{-}at$
 $trace\text{-}openings\ composition\text{-}openings\ trace\text{-}table\ composition\text{-}table\ i)$
 $adversary\text{-}initial\text{-}state \leq$
 $staged\text{-}phase\text{-}relation\text{-}error\ size$
 $(staged\text{-}query\text{-}search\text{-}queries\ budgets\ i + 1) +$
 $(1::prob) / nnreal\ (card\ query\text{-}sample\text{-}space) + Q$
proof –
have *event-eq*:
 $checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}single\text{-}round\text{-}candidate\text{-}binding\text{-}gap\text{-}at$
 $trace\text{-}openings\ composition\text{-}openings\ trace\text{-}table\ composition\text{-}table\ i =$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}fixed\text{-}header\text{-}component$
 $(checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}single\text{-}round\text{-}candidate\text{-}binding\text{-}gap\text{-}side$
 $trace\text{-}openings\ composition\text{-}openings\ trace\text{-}table\ composition\text{-}table\ i)$
 $trace\text{-}openings\ composition\text{-}openings\ i$
by (rule *ext*)
(rule

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-iff-fixed-header-component)
show ?thesis
unfolding event-eq
by (rule
checked-staged-security-with-query-prefix-fixed-header-component-bound-from-witness-path-output-and-budgets
[OF wf controlled i-bound path-bound])
qed

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-bound-from-budgets*:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i < \text{rounds}$
shows
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state* A i)
(*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at*
trace-openings composition-openings trace-table composition-table i)
adversary-initial-state $\leq$
staged-phase-relation-error size
(*staged-query-search-queries budgets* i + 1) +
(1::prob) / nnreal (card query-sample-space)
proof –
have event-eq:
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i =
checked-staged-security-with-query-prefix-fixed-header-component
(*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side*
trace-openings composition-openings trace-table composition-table i)
trace-openings composition-openings i
by (rule ext)
(rule
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-iff-fixed-header-component)
show ?thesis
unfolding event-eq
by (rule
checked-staged-security-with-query-prefix-fixed-header-component-bound-from-budgets
[OF wf controlled i-bound])
qed

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-bound-from-target-and-f*
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i < \text{rounds}$
and trace-candidate:
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings])
and comp-candidate:
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])

and *trace-low*: *trace-table-low-degree trace-table*
and *comp-low*: *composition-table-low-degree maxDegree composition-table*
and *not-all*:
 $\bigwedge \text{prefix prefix-state raw raw-state data attacker-state result}$
 final-state.
 $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}) \in$
 set-dist
 $(\text{execute}$
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state}$
 $A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\neg \text{all-queries-consistent trace-table composition-table}$
 $(\text{sqr-alphas prefix})$
and *target-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq T$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit}$
 $\text{trace-openings composition-openings } i)$
 $\text{adversary-initial-state} \leq$
 $T +$
 $(\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(1::\text{prob}) / \text{nnreal (card query-sample-space)})$
proof –
let *?m* =
 $\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i$
let *?target* =
 $\text{checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at}$
 i
let *?gap* =
 $\text{checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at}$
 $\text{trace-openings composition-openings trace-table composition-table } i$
have *event-le*:
 $\text{wp-event } ?m$
 $(\text{checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit}$
 $\text{trace-openings composition-openings } i)$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } ?m (\lambda \text{out. } ?\text{target out} \vee ?\text{gap out})$
 $\text{adversary-initial-state}$
proof (*rule wp-event-mono-on-support*)
fix *out*
assume *support*:

```

    out ∈ set-dist (execute ?m adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
    trace-openings composition-openings i out
  show ?target out ∨ ?gap out
  proof (cases out)
    case None
    then show ?thesis
      using hit
      unfolding
        checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-def
      by simp
  next
    case (Some packed)
    then obtain prefix prefix-state raw raw-state data attacker-state
      result final-state where out-eq:
      out =
        Some (((((prefix, prefix-state), raw), raw-state),
          ((data, attacker-state), result)), final-state)
      by (cases packed) (auto split: prod.splits)
    have not-all':
      ¬ all-queries-consistent trace-table composition-table
        (sqp-alphas prefix)
      by (rule not-all)
      (use support out-eq in simp)
    show ?thesis
      unfolding out-eq
      by (rule
        checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-imp-single-round-header-
        [OF - trace-candidate comp-candidate trace-low comp-low not-all'])
      (use hit out-eq in simp)
  qed
qed
have union-le:
  wp-event ?m (λout. ?target out ∨ ?gap out)
  adversary-initial-state ≤
  wp-event ?m ?target adversary-initial-state +
  wp-event ?m ?gap adversary-initial-state
  by (rule wp-event-union-bound)
have gap-bound:
  wp-event ?m ?gap adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space)
  by (rule
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-bound-from-budgets
    [OF wf controlled i-bound])
have sum-bound:
  wp-event ?m ?target adversary-initial-state +

```

$wp\text{-}event\ ?m\ ?gap\ adversary\text{-}initial\text{-}state \leq$
 $T +$
 $(staged\text{-}phase\text{-}relation\text{-}error\ size$
 $(staged\text{-}query\text{-}search\text{-}queries\ budgets\ i + 1) +$
 $(1::prob) / nreal\ (card\ query\text{-}sample\text{-}space))$
by (rule add-mono[*OF* target-bound gap-bound])
show ?thesis
by (rule order-trans[*OF* event-le order-trans[*OF* union-le sum-bound]])
qed

lemma checked-staged-security-with-query-prefix-verifier-unsupported-witness-gap-imp-header-witness-path-output:

assumes witness:

$(trace\text{-}openings, composition\text{-}openings) \in$
 $query\text{-}header\text{-}supported\text{-}partial\text{-}opening\text{-}witnesses$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data))$
 $(staged\text{-}trace\text{-}root\ data)$
 $(staged\text{-}trace\text{-}fri\text{-}roots\ data)$
 $(staged\text{-}trace\text{-}final\ data)$
 $(staged\text{-}alphas\ data)$
 $(staged\text{-}degree\ data)$
 $(staged\text{-}composition\text{-}fri\text{-}roots\ data)$
 $(staged\text{-}composition\text{-}final\ data)$

and trace-candidate:

$partial\text{-}trace\text{-}table\text{-}candidate\ trace\text{-}table$
 $((replicate\ rounds\ [])\ [i := trace\text{-}openings])$

and comp-candidate:

$partial\text{-}composition\text{-}table\text{-}candidate\ composition\text{-}table$
 $((replicate\ rounds\ [])\ [i := composition\text{-}openings])$

and notin-prefix:

$(trace\text{-}table, composition\text{-}table) \notin$
 $query\text{-}header\text{-}supported\text{-}single\text{-}round\text{-}partial\text{-}table\text{-}candidates$
 $prefix\text{-}state$
 $(sqp\text{-}trace\text{-}root\ prefix)$
 $(sqp\text{-}trace\text{-}fri\text{-}roots\ prefix)$
 $(sqp\text{-}trace\text{-}final\ prefix)$
 $(sqp\text{-}alphas\ prefix)$
 $(sqp\text{-}degree\ prefix)$
 $(sqp\text{-}composition\text{-}fri\text{-}roots\ prefix)$
 $(sqp\text{-}composition\text{-}final\ prefix)$
 i

and notin-verifier:

$(trace\text{-}table, composition\text{-}table) \notin$
 $query\text{-}header\text{-}supported\text{-}single\text{-}round\text{-}partial\text{-}table\text{-}candidates$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data))$
 $(staged\text{-}trace\text{-}root\ data)$
 $(staged\text{-}trace\text{-}fri\text{-}roots\ data)$
 $(staged\text{-}trace\text{-}final\ data)$

```

    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  i
shows
  checked-staged-security-with-query-prefix-header-witness-path-output-hit
  (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
    trace-openings composition-openings trace-table composition-table i)
  trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from query-header-supported-partial-opening-witnessesE[OF witness]
  obtain witness-result witness-state rest where comp-nonempty:
    staged-composition-fri-roots data ≠ []
  and outcome:
    Some (witness-result, witness-state) ∈
      set-dist (execute verify-monad ?s)
  and trace-auth:
    partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings witness-state
  and comp-auth:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings witness-state
  and header:
    verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    rest
  by blast
  have ext: ?s ≤ witness-state
  by (rule verify-monad-hash-extends[OF outcome])
  have trace-pull:
    partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings ?s ∨
    (∃ opn ∈ set trace-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots witness-state
          (staged-trace-root data) (scale * clength)
          (opening-index opn) (opening-value opn) (opening-path opn))

```



```

    ?s witness-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit
      [OF ext trace-auth])
have comp-pull:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings ?s ∨
  (∃ opn ∈ set composition-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state
        (hd (staged-composition-fri-roots data)) (scale * clength)
        (opening-index opn) (opening-value opn) (opening-path opn))
      ?s witness-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit
      [OF ext comp-auth])
have path:
  (∃ opn ∈ set trace-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state
        (staged-trace-root data) (scale * clength)
        (opening-index opn) (opening-value opn) (opening-path opn))
      ?s witness-state) ∨
  (∃ opn ∈ set composition-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state
        (hd (staged-composition-fri-roots data)) (scale * clength)
        (opening-index opn) (opening-value opn) (opening-path opn))
      ?s witness-state)
proof (rule ccontr)
  assume not-path: ¬ ?thesis
  have trace-on-s:
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings ?s
  using trace-pull not-path by blast
  have comp-on-s:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings ?s
  using comp-pull not-path by blast
  show False
  by (rule
      query-header-supported-single-round-partial-table-candidate-notin-contradicts
      [OF notin-verifier comp-nonempty trace-on-s comp-on-s
        trace-candidate comp-candidate header])
qed
have side:
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
    trace-openings composition-openings trace-table composition-table i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
unfolding

```

```

    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side-def
using trace-candidate comp-candidate notin-prefix by simp
show ?thesis
unfolding
    checked-staged-security-with-query-prefix-header-witness-path-output-hit-def
using side comp-nonempty outcome header path
apply (simp add: Let-def)
apply (intro exI[of - witness-result] exI[of - witness-state]
    exI[of - rest])
apply auto
done
qed

```

lemma *checked-staged-security-with-query-prefix-witness-gap-side-imp-verifier-supported-or-header-witness-path-outp*

assumes *witness:*

(*trace-openings*, *composition-openings*) ∈
query-header-supported-partial-opening-witnesses
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
(*staged-trace-root data*)
(*staged-trace-fri-roots data*)
(*staged-trace-final data*)
(*staged-alphas data*)
(*staged-degree data*)
(*staged-composition-fri-roots data*)
(*staged-composition-final data*)

and *side:*

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i
(*Some* ((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*))

shows

(*trace-table*, *composition-table*) ∈
query-header-supported-single-round-partial-table-candidates
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
(*staged-trace-root data*)
(*staged-trace-fri-roots data*)
(*staged-trace-final data*)
(*staged-alphas data*)
(*staged-degree data*)
(*staged-composition-fri-roots data*)
(*staged-composition-final data*)
i ∨

checked-staged-security-with-query-prefix-header-witness-path-output-hit
(*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side*
trace-openings composition-openings trace-table composition-table i)
trace-openings composition-openings i
(*Some* ((((*prefix*, *prefix-state*), *raw*), *raw-state*),

```

      ((data, attacker-state), result)), final-state))
proof (cases
  (trace-table, composition-table) ∈
    query-header-supported-single-round-partial-table-candidates
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)
        (staged-composition-fri-roots data)
        (staged-composition-final data)
      i)
  case True
  then show ?thesis by simp
next
  case notin-verifier: False
  from side have trace-candidate:
    partial-trace-table-candidate trace-table
      ((replicate rounds []) [i := trace-openings])
  and comp-candidate:
    partial-composition-table-candidate composition-table
      ((replicate rounds []) [i := composition-openings])
  and notin-prefix:
    (trace-table, composition-table) ∉
      query-header-supported-single-round-partial-table-candidates
        prefix-state
        (sqp-trace-root prefix)
        (sqp-trace-fri-roots prefix)
        (sqp-trace-final prefix)
        (sqp-alphas prefix)
        (sqp-degree prefix)
        (sqp-composition-fri-roots prefix)
        (sqp-composition-final prefix)
      i
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side-def
  by simp-all
have
  checked-staged-security-with-query-prefix-header-witness-path-output-hit
    (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i)
  trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
  by (rule
    checked-staged-security-with-query-prefix-verifier-unsupported-witness-gap-imp-header-witness-path-output
      [OF witness trace-candidate comp-candidate notin-prefix notin-verifier])

```

then show *?thesis* by *simp*
 qed

definition *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side*
 where

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
 trace-openings composition-openings trace-table composition-table i out
 $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
 trace-openings composition-openings trace-table composition-table i
 out $\wedge$
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data))

definition *checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at*
 where

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
 trace-openings composition-openings trace-table composition-table i out
 $\longleftrightarrow$
 checked-staged-security-with-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table) out $\wedge$
 checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
 trace-openings composition-openings trace-table composition-table i out

lemma *checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-imp-dynamic-hit:*

assumes gap:

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
 trace-openings composition-openings trace-table composition-table i out

shows

checked-staged-security-with-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table) out

using gap

unfolding

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-def

by *simp*

lemma *checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-imp-witness-side:*

assumes *gap:*

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
trace-openings composition-openings trace-table composition-table i out

shows

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
trace-openings composition-openings trace-table composition-table i out

using *gap*

unfolding

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-def

by *simp*

definition *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at*

where

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
trace-openings composition-openings trace-table composition-table i out

$\longleftrightarrow$

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out $\wedge$
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
trace-openings composition-openings trace-table composition-table i out

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-imp-gap:*

assumes *hit:*

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
trace-openings composition-openings trace-table composition-table i out

shows

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out

using *hit*

unfolding

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-def

by *simp*

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-imp-witness-side:*

assumes *hit:*

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
trace-openings composition-openings trace-table composition-table i out

shows

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
trace-openings composition-openings trace-table composition-table i out

using *hit*

unfolding

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-def

by *simp*

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-bound-from-budgets:*

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *i-bound*: $i < \text{rounds}$
shows
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state *A i*)
(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
trace-openings composition-openings trace-table composition-table *i*)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) +
 $(1::\text{prob}) / \text{nnreal} (\text{card } \text{query-sample-space})$
proof –
have *event-le*:
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state *A i*)
(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
trace-openings composition-openings trace-table composition-table *i*)
adversary-initial-state $\leq$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state *A i*)
(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table *i*)
adversary-initial-state
by (rule *wp-event-mono*)
(rule
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-imp-gap)
have *gap-bound*:
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state *A i*)
(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table *i*)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) +
 $(1::\text{prob}) / \text{nnreal} (\text{card } \text{query-sample-space})$
by (rule
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-bound-from-budgets
[*OF wf controlled i-bound*])
show ?thesis
by (rule *order-trans*[*OF event-le gap-bound*])
qed

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-bound-from-witness-s*
assumes *side-bound*:

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state *A i*)
(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
trace-openings composition-openings trace-table composition-table *i*)

$adversary-initial-state \leq S$
shows
 $wp-event$
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
 trace-openings composition-openings trace-table composition-table i)
 $adversary-initial-state \leq S$
by (rule order-trans[$OF - side-bound$])
 (rule wp-event-mono,
 rule checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-imp-witness-side)

definition checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side
where

checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side
 trace-openings composition-openings trace-table composition-table i out
 $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
 trace-openings composition-openings trace-table composition-table i
 out $\wedge$
 (trace-table, composition-table) $\in$
 query-header-supported-single-round-partial-table-candidates
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
 i)

lemma checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-imp-supported-or-p
assumes hit:

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
 trace-openings composition-openings trace-table composition-table i out

shows

checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side
 trace-openings composition-openings trace-table composition-table i out $\vee$
 checked-staged-security-with-query-prefix-header-witness-path-output-hit
 (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
 trace-openings composition-openings trace-table composition-table i)
 trace-openings composition-openings i out

proof (cases out)

case None

```

then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-def
  by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
  from hit have side:
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
    trace-openings composition-openings trace-table composition-table i
    out
  and witness:
    (trace-openings, composition-openings) ∈
      query-header-supported-partial-opening-witnesses
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
  unfolding out-eq
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-def
  by simp-all
  have split:
    (trace-table, composition-table) ∈
      query-header-supported-single-round-partial-table-candidates
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
    i ∨
    checked-staged-security-with-query-prefix-header-witness-path-output-hit
    (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i)
    trace-openings composition-openings i out

```



```

unfolding out-eq
by (rule
  checked-staged-security-with-query-prefix-witness-gap-side-imp-verifier-supported-or-header-witness-path-output
  [OF witness side[unfolded out-eq]])
then show ?thesis
proof
  assume supported:
    (trace-table, composition-table) ∈
      query-header-supported-single-round-partial-table-candidates
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      i
  have
    checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side
    trace-openings composition-openings trace-table composition-table i out
  unfolding out-eq
  checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side-def
  using hit[unfolded out-eq] supported by simp
  then show ?thesis by simp
next
  assume
    checked-staged-security-with-query-prefix-header-witness-path-output-hit
    (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i)
    trace-openings composition-openings i out
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-bound-from-support
assumes supported-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side
    trace-openings composition-openings trace-table composition-table i)
  adversary-initial-state ≤ S
and path-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-header-witness-path-output-hit
    (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i))

```

$\text{trace-openings composition-openings } i)$
 $\text{adversary-initial-state} \leq P$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side}$
 $\text{trace-openings composition-openings trace-table composition-table } i)$
 $\text{adversary-initial-state} \leq S + P$
proof –
let $?m =$
 $\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i$
let $?W =$
 $\text{checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side}$
 $\text{trace-openings composition-openings trace-table composition-table } i$
let $?S =$
 $\text{checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side}$
 $\text{trace-openings composition-openings trace-table composition-table } i$
let $?P =$
 $\text{checked-staged-security-with-query-prefix-header-witness-path-output-hit}$
 $(\text{checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side}$
 $\text{trace-openings composition-openings trace-table composition-table } i)$
 $\text{trace-openings composition-openings } i$
have event-le:
 $\text{wp-event } ?m \ ?W \ \text{adversary-initial-state} \leq$
 $\text{wp-event } ?m \ (\lambda \text{out. } ?S \ \text{out} \vee ?P \ \text{out}) \ \text{adversary-initial-state}$
by $(\text{rule wp-event-mono})$
 $(\text{rule}$
 $\text{checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-imp-supported-or-path-}$
have union-le:
 $\text{wp-event } ?m \ (\lambda \text{out. } ?S \ \text{out} \vee ?P \ \text{out}) \ \text{adversary-initial-state} \leq$
 $\text{wp-event } ?m \ ?S \ \text{adversary-initial-state} +$
 $\text{wp-event } ?m \ ?P \ \text{adversary-initial-state}$
by $(\text{rule wp-event-union-bound})$
have sum-le:
 $\text{wp-event } ?m \ ?S \ \text{adversary-initial-state} +$
 $\text{wp-event } ?m \ ?P \ \text{adversary-initial-state} \leq S + P$
by $(\text{rule add-mono}[OF \ \text{supported-bound path-bound}])$
show $?thesis$
by $(\text{rule order-trans}[OF \ \text{event-le order-trans}[OF \ \text{union-le sum-le}]])$
qed

lemma $\text{checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-bound-from-supported-bound}$
assumes supported-bound:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side}$
 $\text{trace-openings composition-openings trace-table composition-table } i)$
 $\text{adversary-initial-state} \leq S$
and transcript-bound:

```

    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-header-witness-transcript-hit
        (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
          trace-openings composition-openings trace-table composition-table i))
      adversary-initial-state  $\leq P$ 
  shows
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
        trace-openings composition-openings trace-table composition-table i)
      adversary-initial-state  $\leq S + P$ 
proof -
  let ?m =
    checked-staged-security-experiment-with-query-prefix-data-state A i
  let ?side =
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i
  have path-bound:
    wp-event ?m
      (checked-staged-security-with-query-prefix-header-witness-path-output-hit
        ?side trace-openings composition-openings i)
      adversary-initial-state  $\leq P$ 
  proof -
    have
      wp-event ?m
        (checked-staged-security-with-query-prefix-header-witness-path-output-hit
          ?side trace-openings composition-openings i)
        adversary-initial-state  $\leq$ 
      wp-event ?m
        (checked-staged-security-with-query-prefix-header-witness-transcript-hit
          ?side)
        adversary-initial-state
      by (rule wp-event-mono)
        (rule
          checked-staged-security-with-query-prefix-header-witness-path-output-imp-transcript-hit)
    then show ?thesis
      by (rule order-trans[OF - transcript-bound])
  qed
  show ?thesis
    by (rule
      checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-bound-from-supported-
        [OF supported-bound path-bound])
  qed

lemma checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-bound-from-supported-
  assumes supported-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)

```

*(checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side
 trace-openings composition-openings trace-table composition-table i)*
adversary-initial-state $\leq S$
and *pre-bound*:
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i))
adversary-initial-state $\leq R$
and *new-bound*:
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i))
adversary-initial-state $\leq Q$
shows
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
trace-openings composition-openings trace-table composition-table i)
adversary-initial-state $\leq S + (R + Q)$
proof –
let *?m* =
checked-staged-security-experiment-with-query-prefix-data-state A i
let *?side* =
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i
have *transcript-bound*:
wp-event ?m
(checked-staged-security-with-query-prefix-header-witness-transcript-hit
?side)
adversary-initial-state $\leq R + Q$
proof –
have *event-le*:
wp-event ?m
(checked-staged-security-with-query-prefix-header-witness-transcript-hit
?side)
adversary-initial-state $\leq$
wp-event ?m
(λ out.
checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
?side out $\vee$
checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
?side out)
adversary-initial-state
by (*rule wp-event-mono*)
(rule

```

      checked-staged-security-with-query-prefix-header-witness-transcript-hit-imp-pre-or-new)
have union-le:
  wp-event ?m
  (λout.
    checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
      ?side out ∨
    checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
      ?side out)
  adversary-initial-state ≤
  wp-event ?m
  (checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
    ?side)
  adversary-initial-state +
  wp-event ?m
  (checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
    ?side)
  adversary-initial-state
by (rule wp-event-union-bound)
have sum-le:
  wp-event ?m
  (checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
    ?side)
  adversary-initial-state +
  wp-event ?m
  (checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
    ?side)
  adversary-initial-state ≤ R + Q
by (rule add-mono[OF pre-bound new-bound])
show ?thesis
by (rule order-trans[OF event-le order-trans[OF union-le sum-le]])
qed
have bound:
  wp-event ?m
  (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
    trace-openings composition-openings trace-table composition-table i)
  adversary-initial-state ≤ S + (R + Q)
by (rule
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-bound-from-supported-
    [OF supported-bound transcript-bound])
then show ?thesis .
qed

lemma checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-bound-from-supported
assumes supported-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side
    trace-openings composition-openings trace-table composition-table i)
  adversary-initial-state ≤ S

```

and *pre-bound*:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state *A i*)
 (checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
 (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
 trace-openings composition-openings trace-table composition-table *i*))
 adversary-initial-state $\leq R$
and *new-bound*:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state *A i*)
 (checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
 (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
 trace-openings composition-openings trace-table composition-table *i*))
 adversary-initial-state $\leq Q$
shows
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state *A i*)
 (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
 trace-openings composition-openings trace-table composition-table *i*)
 adversary-initial-state $\leq S + (R + Q)$
proof –
have *side-bound*:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state *A i*)
 (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
 trace-openings composition-openings trace-table composition-table *i*)
 adversary-initial-state $\leq S + (R + Q)$
by (rule
 checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-bound-from-supported-
 [OF supported-bound pre-bound new-bound])
show ?thesis
by (rule
 checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-bound-from-witness-side
 [OF side-bound])
qed

definition *checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-exists*
where
checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-exists
out $\longleftrightarrow$
 (case *out* of
 None $\Rightarrow$ False
 | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state) $\Rightarrow$
 ($\exists$ result' final-state'.
 partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result', final-state')))))

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*

assumes *hit*:

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at

i A

(*Some* ((((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
result), *final-state*)))

and *i-bound*: *i* < *rounds*

and *trace-low*:

$\bigwedge$ *trace-openings composition-openings trace-table*
composition-table.

(*trace-openings*, *composition-openings*) $\in$
query-header-supported-partial-opening-witnesses
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
(*staged-trace-root data*)
(*staged-trace-fri-roots data*)
(*staged-trace-final data*)
(*staged-alphas data*)
(*staged-degree data*)
(*staged-composition-fri-roots data*)
(*staged-composition-final data*) $\implies$

partial-trace-table-candidate trace-table

((*replicate rounds []*) [*i := trace-openings*]) $\implies$

partial-composition-table-candidate composition-table

((*replicate rounds []*) [*i := composition-openings*]) $\implies$

trace-table-low-degree trace-table

and *comp-low*:

$\bigwedge$ *trace-openings composition-openings trace-table*
composition-table.

(*trace-openings*, *composition-openings*) $\in$
query-header-supported-partial-opening-witnesses
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
(*staged-trace-root data*)
(*staged-trace-fri-roots data*)
(*staged-trace-final data*)
(*staged-alphas data*)
(*staged-degree data*)
(*staged-composition-fri-roots data*)
(*staged-composition-final data*) $\implies$

partial-trace-table-candidate trace-table

((*replicate rounds []*) [*i := trace-openings*]) $\implies$

partial-composition-table-candidate composition-table

((*replicate rounds []*) [*i := composition-openings*]) $\implies$

composition-table-low-degree maxDegree composition-table

and *not-all*:

$\bigwedge$ *trace-openings composition-openings trace-table*
composition-table prefix prefix-state.

$(\text{trace-openings}, \text{composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses}$
 $(\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data}))$
 $(\text{staged-trace-root } \text{data})$
 $(\text{staged-trace-fri-roots } \text{data})$
 $(\text{staged-trace-final } \text{data})$
 $(\text{staged-alphas } \text{data})$
 $(\text{staged-degree } \text{data})$
 $(\text{staged-composition-fri-roots } \text{data})$
 $(\text{staged-composition-final } \text{data}) \implies$
 $\text{partial-trace-table-candidate } \text{trace-table}$
 $((\text{replicate rounds } []) [i := \text{trace-openings}]) \implies$
 $\text{partial-composition-table-candidate } \text{composition-table}$
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \implies$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\neg \text{all-queries-consistent } \text{trace-table } \text{composition-table}$
 $(\text{sqp-alphas } \text{prefix})$

shows

$\text{checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-exists}$
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state})) \vee$
 $\text{checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at}$
 $i \ A$
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state})) \vee$
 $\text{checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at}$
 $i \ A$
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}))$

using

$\text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at-im}$
 $[OF \text{ hit } i\text{-bound } \text{trace-low } \text{comp-low } \text{not-all}]$

unfolding

$\text{checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-exists-def}$

by *simp*

lemma $\text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at-im}$

assumes *split*:

$\bigwedge \text{out.}$

$\text{out} \in$

set-dist

$(\text{execute}$

$(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$

$A)$

$\text{adversary-initial-state}) \implies$

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
i A out $\implies$
checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-exists
out $\vee$
checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
i A out $\vee$
checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
i A out
and *partial-merkle-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-exists
adversary-initial-state $\leq P$
and *target-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at*
i A)
adversary-initial-state $\leq Q$
and *gap-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at*
i A)
adversary-initial-state $\leq R$
shows
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
(*checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at*
i A)
adversary-initial-state $\leq P + Q + R$
proof –
let *?m* =
checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let *?pm* =
checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-exists
let *?target* =
checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
i A
let *?gap* =
checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
i A
have *event-le*:
wp-event ?m
(*checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at*
i A)

```

    adversary-initial-state ≤
    wp-event ?m (λout. ?pm out ∨ ?target out ∨ ?gap out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (rule split)
have union-le:
  wp-event ?m (λout. ?pm out ∨ ?target out ∨ ?gap out)
  adversary-initial-state ≤
  wp-event ?m ?pm adversary-initial-state +
  (wp-event ?m ?target adversary-initial-state +
   wp-event ?m ?gap adversary-initial-state)
proof -
  have
    wp-event ?m (λout. ?pm out ∨ ?target out ∨ ?gap out)
    adversary-initial-state ≤
    wp-event ?m ?pm adversary-initial-state +
    wp-event ?m (λout. ?target out ∨ ?gap out)
    adversary-initial-state
  by (rule wp-event-union-bound)
  also have
    ... ≤
    wp-event ?m ?pm adversary-initial-state +
    (wp-event ?m ?target adversary-initial-state +
     wp-event ?m ?gap adversary-initial-state)
  by (intro add-mono order-refl wp-event-union-bound)
  finally show ?thesis .
qed
have sum-bound:
  wp-event ?m ?pm adversary-initial-state +
  (wp-event ?m ?target adversary-initial-state +
   wp-event ?m ?gap adversary-initial-state) ≤ P + Q + R
proof -
  have
    wp-event ?m ?target adversary-initial-state +
    wp-event ?m ?gap adversary-initial-state ≤ Q + R
  by (rule add-mono[OF target-bound gap-bound])
  then have
    wp-event ?m ?pm adversary-initial-state +
    (wp-event ?m ?target adversary-initial-state +
     wp-event ?m ?gap adversary-initial-state) ≤ P + (Q + R)
  by (rule add-mono[OF partial-merkle-bound])
  then show ?thesis
  by (simp add: add.assoc)
qed
show ?thesis
  by (rule order-trans[OF event-le order-trans[OF union-le sum-bound]])
qed
end

```

end

theory *Staged-Security-Experiment-Composition-Query-Actual-Run*
imports *Staged-Security-Experiment-Composition-Query-Single-Round*
begin

Same-run authenticated-opening split for the composition query branch.

This layer keeps the verifier continuation from the staged experiment fixed, so partial-Merkle inconsistency is expressed as the actual verifier event for the sampled run.

context *soundness*
begin

definition *checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad*
where

checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
 $out \longleftrightarrow$
checked-staged-security-with-actual-alpha-prefix-verifier-event
partial-merkle-inconsistency-bad out

lemma *checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix-imp-a*

assumes *support:*

*Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)*

and *hit:*

checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
A
*(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))*

and *trace-low:*

$\bigwedge i$ *trace-openings composition-openings trace-table*
composition-table.
 $i < rounds \implies$
partial-trace-table-candidate trace-table
 $((\text{replicate } rounds \ \square) \ [i := \text{trace-openings}]) \implies$
partial-composition-table-candidate composition-table
 $((\text{replicate } rounds \ \square) \ [i := \text{composition-openings}]) \implies$
trace-table-low-degree trace-table

and *comp-low:*

$\bigwedge i$ *trace-openings composition-openings trace-table*
composition-table.
 $i < rounds \implies$
partial-trace-table-candidate trace-table

$((\text{replicate rounds } []) [i := \text{trace-openings}]) \Rightarrow$
partial-composition-table-candidate composition-table
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \Rightarrow$
composition-table-low-degree maxDegree composition-table
and not-all:
 $\bigwedge i \text{ trace-openings composition-openings trace-table}$
composition-table prefix prefix-state.
 $i < \text{rounds} \Rightarrow$
partial-trace-table-candidate trace-table
 $((\text{replicate rounds } []) [i := \text{trace-openings}]) \Rightarrow$
partial-composition-table-candidate composition-table
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \Rightarrow$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
*adversary-initial-state}) \Rightarrow
 $\neg \text{all-queries-consistent trace-table composition-table}$
 $(\text{sqp-alphas prefix})$
shows
checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
result), final-state))) \vee
 $(\exists i \text{ trace-table composition-table.}$
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
 $A \text{ trace-table composition-table } i$
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
*result), final-state))))
proof –
let ?out =
 $\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
*result), final-state)
let ?vout = $\text{Some } (\text{result}, \text{final-state})$
let ?s =
verifier-state-from-adversary attacker-state
 $(\text{staged-proof-transcript data})$
have data-support:
 $\text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state}) \in$
set-dist
 $(\text{execute } (\text{checked-staged-security-experiment-with-data-state } A)$
*adversary-initial-state})
by (rule
checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
 $[\text{OF support}]$)
from *checked-staged-security-experiment-with-data-state-outcomeE*
 $[\text{OF data-support}]$
obtain builder verifier **where**
verifier:
 $\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute } \text{verify-monad } ?s)$
by blast****

```

from hit obtain i trace-openings composition-openings where
  comp-nonempty: staged-composition-fri-roots data  $\neq \square$ 
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and comp-auth:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings final-state
and component:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
    A
    ((replicate rounds  $\square$ ) [i := trace-openings])
    ((replicate rounds  $\square$ ) [i := composition-openings])
    i ?out
unfolding
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix-def
by auto
have i-bound: i < rounds
using component
unfolding
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-def
by (simp split: option.splits prod.splits)
show ?thesis
proof (cases partial-merkle-inconsistency-bad ?s ?vout)
case True
then show ?thesis
unfolding
  checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad-def
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
by (simp add: Let-def)
next
case no-bad: False
obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table
    ((replicate rounds  $\square$ ) [i := trace-openings])
using
  partial-trace-table-candidate-single-witness-if-no-partial-merkle-bad
    [OF trace-auth verifier no-bad i-bound]
by blast
obtain composition-table where comp-candidate:
  partial-composition-table-candidate composition-table
    ((replicate rounds  $\square$ ) [i := composition-openings])
using
  partial-composition-table-candidate-single-witness-if-no-partial-merkle-bad
    [OF comp-auth verifier no-bad i-bound]
by blast
have pair-hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
    A trace-table composition-table i ?out

```

```

by (rule
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-imp-candidate-pair-que
    [OF component trace-candidate comp-candidate
      trace-low[OF i-bound trace-candidate comp-candidate]
      comp-low[OF i-bound trace-candidate comp-candidate]])
  (rule not-all[OF i-bound trace-candidate comp-candidate])
then show ?thesis
using i-bound by blast
qed
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix-imp-a*

assumes *support*:

Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state) $\in$
set-dist
 (execute
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 adversary-initial-state)

and *hit*:

checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
 A
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))

and *trace-low*:

$\bigwedge i$ *trace-openings composition-openings trace-table*
composition-table.
i < rounds $\implies$
partial-trace-table-candidate trace-table
 ((replicate rounds []) [*i* := *trace-openings*]) $\implies$
partial-composition-table-candidate composition-table
 ((replicate rounds []) [*i* := *composition-openings*]) $\implies$
trace-table-low-degree trace-table

and *comp-low*:

$\bigwedge i$ *trace-openings composition-openings trace-table*
composition-table.
i < rounds $\implies$
partial-trace-table-candidate trace-table
 ((replicate rounds []) [*i* := *trace-openings*]) $\implies$
partial-composition-table-candidate composition-table
 ((replicate rounds []) [*i* := *composition-openings*]) $\implies$
composition-table-low-degree maxDegree composition-table

and *not-all*:

$\bigwedge i$ *trace-openings composition-openings trace-table*
composition-table prefix prefix-state.
i < rounds $\implies$
partial-trace-table-candidate trace-table
 ((replicate rounds []) [*i* := *trace-openings*]) $\implies$

partial-composition-table-candidate composition-table
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \implies$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\neg \text{all-queries-consistent trace-table composition-table}$
 $(\text{sqp-alphas prefix})$

shows

checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state})) \vee$
 $(\exists i < \text{rounds}.$
checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
 $i \ A$
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state})) \vee$
checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at
 $i \ A$
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}))))$

proof —

have *split*:

checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state})) \vee$
 $(\exists i \text{ trace-table composition-table}.$
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
 $A \text{ trace-table composition-table } i$
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}))))$

by (rule

checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix-imp-actua
 $[OF \text{ support hit trace-low comp-low not-all}]$)

then show *?thesis*

proof

assume

checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}))$

then show *?thesis* **by** *simp*

next

assume

$\exists i \text{ trace-table composition-table}.$
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
 $A \text{ trace-table composition-table } i$
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}))$

then obtain *i trace-table composition-table* **where** *pair-hit*:

```

checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
  A trace-table composition-table i
  (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
by blast
have i-bound: i < rounds
by (rule
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
  [OF pair-hit])
have
checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
  i A
  (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state)) ∨
checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at
  i A
  (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
by (rule
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-imp-single-round-header-ta
  [OF pair-hit])
then show ?thesis
using i-bound by blast
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix-bound-
fixes PM :: prob and T G :: nat ⇒ prob
assumes split:
  ∧ out.
  out ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state) ⇒
checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
  A out ⇒
checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
  out ∨
(∃ i < rounds.
  checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
    i A out ∨
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at
    i A out)
and partial-merkle-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)

```


checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
adversary-initial-state $\leq$ *PM*
and *target-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at } i \ A)$
 $\text{adversary-initial-state} \leq T \ i$
and *gap-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at } i \ A)$
 $\text{adversary-initial-state} \leq G \ i$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix } A)$
 $\text{adversary-initial-state} \leq$
 $PM + (\sum i < \text{rounds}. T \ i + G \ i)$
proof –
let *?M* =
 $\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A$
let *?auth* =
 $\text{checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix } A$
let *?pm* =
 $\text{checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad}$
let *?round* =
 $\lambda i \text{ out.}$
 $\text{checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at } i \ A \ \text{out} \vee$
 $\text{checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at } i \ A \ \text{out}$
have *event-le*:
 $\text{wp-event } ?M \ ?auth \ \text{adversary-initial-state} \leq$
 $\text{wp-event } ?M \ (\lambda \text{out. } ?pm \ \text{out} \vee (\exists i \in \{..<\text{rounds}\}. ?round \ i \ \text{out}))$
 $\text{adversary-initial-state}$
by (*rule wp-event-mono-on-support*)
 $(\text{use split in auto})$
have *union-le*:
 $\text{wp-event } ?M \ (\lambda \text{out. } ?pm \ \text{out} \vee (\exists i \in \{..<\text{rounds}\}. ?round \ i \ \text{out}))$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } ?M \ ?pm \ \text{adversary-initial-state} +$

```

    wp-event ?M (λout. ∃ i ∈ {..rounds}. ?round i out)
    adversary-initial-state
  by (rule wp-event-union-bound)
have round-bound:
  ∧ i. i ∈ {..rounds} ⇒
    wp-event ?M (?round i) adversary-initial-state ≤ T i + G i
proof -
  fix i
  assume i-bound: i ∈ {..rounds}
  have
    wp-event ?M (?round i) adversary-initial-state ≤
      wp-event ?M
        (checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
          i A)
      adversary-initial-state +
      wp-event ?M
        (checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at
          i A)
      adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤ T i + G i
  by (intro add-mono target-bound gap-bound)
    (use i-bound in simp-all)
  finally show wp-event ?M (?round i) adversary-initial-state ≤ T i + G i .
qed
have rounds-bound:
  wp-event ?M (λout. ∃ i ∈ {..rounds}. ?round i out)
  adversary-initial-state ≤ (∑ i < rounds. T i + G i)
  by (rule wp-event-finite-union-bound)
    (simp-all add: round-bound)
have
  wp-event ?M (λout. ?pm out ∨ (∃ i ∈ {..rounds}. ?round i out))
  adversary-initial-state ≤ PM + (∑ i < rounds. T i + G i)
  by (rule order-trans[OF union-le])
    (intro add-mono partial-merkle-bound rounds-bound)
then show ?thesis
  by (rule order-trans[OF event-le])
qed

```

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-actual-single-round-components-and-bud.*

fixes *PM* :: prob **and** *T G* :: nat ⇒ prob

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *split*:

∧ out.

out ∈

set-dist

(execute

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

$$\begin{aligned}
& A) \\
& \text{adversary-initial-state}) \implies \\
& \text{checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix} \\
& A \text{ out} \implies \\
& \text{checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad} \\
& \text{out} \vee \\
& (\exists i < \text{rounds}. \\
& \text{checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at} \\
& i \ A \ \text{out} \vee \\
& \text{checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at} \\
& i \ A \ \text{out}) \\
& \text{and } \text{partial-merkle-bound:} \\
& \text{wp-event} \\
& (\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state} \\
& A) \\
& \text{checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad} \\
& \text{adversary-initial-state} \leq PM \\
& \text{and } \text{target-bound:} \\
& \bigwedge i. i < \text{rounds} \implies \\
& \text{wp-event} \\
& (\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state} \\
& A) \\
& (\text{checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at} \\
& i \ A) \\
& \text{adversary-initial-state} \leq T \ i \\
& \text{and } \text{gap-bound:} \\
& \bigwedge i. i < \text{rounds} \implies \\
& \text{wp-event} \\
& (\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state} \\
& A) \\
& (\text{checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at} \\
& i \ A) \\
& \text{adversary-initial-state} \leq G \ i \\
& \text{shows} \\
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& (\text{staged-security-with-data-state-verifier-event composition-bad}) \\
& \text{adversary-initial-state} \leq \\
& \text{composition-error-bound} + \\
& \text{staged-phase-relation-error size} \\
& (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{hash-collision-budget-value } 0 \\
& (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{staged-phase-relation-error size} \\
& (\text{staged-alpha-search-queries budgets } 0) + \\
& (PM + (\sum i < \text{rounds}. T \ i + G \ i)) + \\
& \text{hash-collision-budget-value } 0 \\
& (\text{staged-attacker-query-budget budgets} + \\
& \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) \\
& \text{proof} -
\end{aligned}$$

```

have authenticated-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
      A)
     $\text{adversary-initial-state} \leq PM + (\sum i < \text{rounds}. T\ i + G\ i)$ 
  by (rule
    checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix-bound-from
    [OF split partial-merkle-bound target-bound gap-bound])
show ?thesis
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-authenticated-query-from-prefix-and-budgets
    [OF wf controlled authenticated-bound])
qed

end

end

```

```

theory Soundness-Staged-Bounds
imports
  Soundness-Staged-Transcript
  Soundness-Staged-Partial-Query
  Soundness-Staged-Partial-Query-Branch
  Staged-Security-Experiment-Composition-Partial
  Staged-Security-Experiment-Composition-Query-Residual
  Staged-Security-Experiment-Composition-Query-Actual-Run
begin

```

Staged soundness bad-event bounds and accepted-execution partitions.

```

context soundness
begin

```

```

lemma checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-query-bad-rounds:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and round-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-bad-hit-at i)
       $\text{adversary-initial-state} \leq C\ i$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event query-bad)
     $\text{adversary-initial-state} \leq (\sum i < \text{rounds}. C\ i)$ 
proof –
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event query-bad)

```

```

    adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-bad-hit
    adversary-initial-state
  by (rule checked-staged-security-with-data-state-query-bad-verifier-event-bound)
    simp
also have ... ≤ (∑ i<rounds. C i)
  by (rule
    checked-staged-security-with-data-state-query-bad-hit-bound-from-rounds)
    (rule round-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-committed-prefix-hit-or-tree-output
  fixes committed-error tree-error :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and committed-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-committed-prefix-hit A)
      adversary-initial-state ≤ committed-error
  and tree-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-tree-output-hit A)
      adversary-initial-state ≤ tree-error
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event query-bad)
      adversary-initial-state ≤ committed-error + tree-error
proof -
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event query-bad)
    adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-bad-hit
    adversary-initial-state
  by (rule checked-staged-security-with-data-state-query-bad-verifier-event-bound)
    simp
also have ... ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-query-committed-prefix-hit A)
  adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-query-tree-output-hit A)
  adversary-initial-state
  by (rule
    checked-staged-security-with-data-state-query-bad-bound-from-committed-prefix-hit-or-tree-output
    [OF wf controlled])
also have ... ≤ committed-error + tree-error

```

by (intro add-mono committed-bound tree-bound)
 finally show ?thesis .
 qed

lemma *checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-length-committed-prefix-hit-or-tree-output*
fixes *committed-error tree-error :: prob*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *committed-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-query-length-committed-prefix-hit A)
 adversary-initial-state ≤ committed-error
and *tree-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-query-tree-output-hit A)
 adversary-initial-state ≤ tree-error
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event query-bad)
 adversary-initial-state ≤ committed-error + tree-error
proof –
 have *wp-event (checked-staged-security-experiment-with-data-state A)*
 (staged-security-with-data-state-verifier-event query-bad)
 adversary-initial-state ≤
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-query-bad-hit
 adversary-initial-state
 by (rule *checked-staged-security-with-data-state-query-bad-verifier-event-bound*)
 simp
 also have ... ≤
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-query-length-committed-prefix-hit A)
 adversary-initial-state +
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-query-tree-output-hit A)
 adversary-initial-state
 by (rule
 checked-staged-security-with-data-state-query-bad-bound-from-length-committed-prefix-hit-or-tree-output
 [OF *wf controlled*])
 also have ... ≤ *committed-error + tree-error*
 by (intro add-mono committed-bound tree-bound)
 finally show ?thesis .
 qed

lemma *staged-security-with-data-state-query-tree-output-hit-imp-partial-header-key-hit-or-collision:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *tree-hit:*
 staged-security-with-data-state-query-tree-output-hit A out

```

shows
  staged-security-with-data-state-query-partial-header-key-hit out  $\vee$ 
  staged-security-with-data-state-verifier-event
  hash-map-output-collision-bad out
proof (cases out)
  case None
  then show ?thesis
  using tree-hit
  unfolding staged-security-with-data-state-query-tree-output-hit-def
  by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have tree-body:
     $\exists i$  prefix prefix-state raw raw-state trace-table composition-table
      as query-idxs trace-tree composition-tree.
     $i < \text{rounds} \wedge$ 
    Some (data, attacker-state)  $\in$ 
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)  $\wedge$ 
    Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)  $\wedge$ 
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs  $\wedge$ 
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw  $\wedge$ 
    index (to-nat raw)  $\in$ 
      query-sampling-success-space trace-table composition-table
      (staged-alphas data)  $\wedge$ 
    Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)  $\wedge$ 
    sqp-composition-fri-roots prefix  $\neq []$   $\wedge$ 
    created-tree trace-table trace-tree final-state  $\wedge$ 
    sqp-trace-root prefix = value trace-tree  $\wedge$ 
    created-tree composition-table composition-tree final-state  $\wedge$ 
    hd (sqp-composition-fri-roots prefix) = value composition-tree  $\wedge$ 
    hash-map-new-output-hit
      (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state

```

```

using tree-hit
unfolding out-eq
  staged-security-with-data-state-query-tree-output-hit-def Let-def
by simp
from tree-body
obtain i prefix prefix-state raw raw-state trace-table composition-table
  as query-idxs trace-tree composition-tree where
    i-bound: i < rounds
and builder:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
     adversary-initial-state)
and verifier:
  Some (result, final-state) ∈
    set-dist (execute verify-monad ?s)
and bound:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs
and lookup-final-raw:
  fmlookup (HashMap final-state)
  (QueryIndexChallenge i
   (state-after-query-chunks
    (staged-query-start-hash data) (staged-query-chunks data i))) =
    Some raw
and raw-hit:
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table
    (staged-alphas data)
by blast
have accepted-out: accepted (Some (result, final-state))
  unfolding accepted-def by simp
show ?thesis
proof (cases hash-map-output-collision final-state)
  case True
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
      hash-map-output-collision-bad-def
    using accepted-out by simp
next
case clean: False
have staged-header:
  verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)

```



```

    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
from bound obtain fr f-fri-roots f-final dg composition-fri-roots final
  rest where
  header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and trace-bind:
    merkle-root-binds-table fr trace-table final-state
  and comp-nonempty: composition-fri-roots ≠ []
  and comp-bind:
    merkle-root-binds-table (hd composition-fri-roots)
    composition-table final-state
  unfolding accepted-with-bound-tables-def by blast
have eqs:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF header staged-header]
  by simp
have state:
  query-header-supported-table-candidate-state ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    trace-table composition-table final-state
  unfolding query-header-supported-table-candidate-state-def
  by (intro exI[of - result] exI[of - query-idxs]
      exI[of - List.concat (staged-query-chunks data)])
    (use verifier bound staged-header trace-bind comp-nonempty comp-bind
      eqs in simp)
have candidate:
  (trace-table, composition-table) ∈
  query-header-supported-partial-table-candidates ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)

```

```

    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
using
  query-header-supported-table-candidate-state-imp-partial-candidate-state
  [OF state clean]
  query-header-supported-partial-table-candidates-iff-state
by blast
have raw-partial-hit:
  index (to-nat raw) ∈
    query-header-supported-partial-union-good-sets ?s
    query-sampling-success-space
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
proof —
  have raw-in-sample:
    index (to-nat raw) ∈ query-sample-space
    using raw-hit query-sampling-success-space-subset by blast
  show ?thesis
  unfolding query-header-supported-partial-union-good-sets-def
    using candidate raw-hit raw-in-sample by blast
qed
then show ?thesis
  unfolding out-eq
    staged-security-with-data-state-query-partial-header-key-hit-def
    Let-def
  using i-bound lookup-final-raw by auto
qed
qed

lemma staged-security-with-data-state-query-tree-output-hit-bound-from-partial-header-key-hit-or-collision:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-tree-output-hit A)
    adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-partial-header-key-hit
    adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      hash-map-output-collision-bad)
    adversary-initial-state

```

proof –
let $?M = \text{checked-staged-security-experiment-with-data-state } A$
let $?partial = \text{staged-security-with-data-state-query-partial-header-key-hit}$
let $?collision =$
 $\text{staged-security-with-data-state-verifier-event}$
 $\text{hash-map-output-collision-bad}$
have $\text{wp-event } ?M (\text{staged-security-with-data-state-query-tree-output-hit } A)$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } ?M (\lambda \text{out. } ?partial \text{ out} \vee ?collision \text{ out})$
 $\text{adversary-initial-state}$
by (rule wp-event-mono)
 (rule
 $\text{staged-security-with-data-state-query-tree-output-hit-imp-partial-header-key-hit-or-collision}$
 [$OF \text{ wf controlled}$])
also have $\dots \leq$
 $\text{wp-event } ?M ?partial \text{ adversary-initial-state} +$
 $\text{wp-event } ?M ?collision \text{ adversary-initial-state}$
by (rule $\text{wp-event-union-bound}$)
finally show $?thesis$.
qed

lemma $\text{checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-partial-header-and-collision-error}$
fixes $\text{partial-error collision-error} :: \text{prob}$
assumes $\text{raw-bound: query-index-raw-preimage-bound}$
and $\text{wf: staged-budget-wellformed budgets}$
and $\text{controlled: staged-adversary-controlled budgets } A$
and partial-bound:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-query-partial-header-key-hit}$
 $\text{adversary-initial-state} \leq \text{partial-error}$
and collision-bound:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{hash-map-output-collision-bad})$
 $\text{adversary-initial-state} \leq \text{collision-error}$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event query-bad})$
 $\text{adversary-initial-state} \leq$
 $(\sum i < \text{rounds.}$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1)) +$
 $(\text{partial-error} + \text{collision-error})$

proof –
let $?M = \text{checked-staged-security-experiment-with-data-state } A$
let $?Tree = \text{staged-security-with-data-state-query-tree-output-hit } A$

```

let ?PrefixSum =
  ( $\sum i < \text{rounds}$ .
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    query-error-bound +
    hash-collision-budget-value 0
    (staged-query-search-queries budgets  $i + 1$ ))
have tree-bound:
  wp-event ?M ?Tree adversary-initial-state  $\leq$ 
    partial-error + collision-error
proof –
  have wp-event ?M ?Tree adversary-initial-state  $\leq$ 
    wp-event ?M staged-security-with-data-state-query-partial-header-key-hit
      adversary-initial-state +
    wp-event ?M
      (staged-security-with-data-state-verifier-event
        hash-map-output-collision-bad)
      adversary-initial-state
    by (rule
      staged-security-with-data-state-query-tree-output-hit-bound-from-partial-header-key-hit-or-collision
      [OF wf controlled])
  also have ...  $\leq$  partial-error + collision-error
    by (intro add-mono partial-bound collision-bound)
  finally show ?thesis .
qed
have wp-event ?M
  (staged-security-with-data-state-verifier-event query-bad)
  adversary-initial-state  $\leq$ 
    ?PrefixSum + wp-event ?M ?Tree adversary-initial-state
  by (rule
    checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-and-tree
    [OF raw-bound wf controlled])
  also have ...  $\leq$  ?PrefixSum + (partial-error + collision-error)
    by (intro add-mono order-refl tree-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-partial-header-and-c
assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and partial-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-partial-header-key-hit
    adversary-initial-state  $\leq$  partial-error
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event query-bad)
    adversary-initial-state  $\leq$ 

```

$(\sum i < \text{rounds}.$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1)) +$
 $(\text{partial-error} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}))$
proof –
have *collision-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{hash-map-output-collision-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget})$
by (rule
 $\text{checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled}$
 $[\text{OF wf controlled}])$
show *?thesis*
by (rule
 $\text{checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-partial-header-and-collision-bound}$
 $[\text{OF raw-bound wf controlled partial-bound collision-bound}])$
qed

lemma *checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-partial-header-and-collision-bound*
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *partial-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-query-partial-header-key-hit}$
 $\text{adversary-initial-state} \leq \text{partial-error}$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event query-bad})$
 $\text{adversary-initial-state} \leq$
 $(\sum i < \text{rounds}.$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1)) +$
 $(\text{partial-error} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}))$

proof –
have *raw-bound: query-index-raw-preimage-bound*
by (rule *query-index-raw-preimage-bound-from-sampler-wellformed*)
show *?thesis*
by (rule
checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-partial-header-and-control
[OF raw-bound wf controlled partial-bound])
qed

lemma *checked-staged-security-with-data-state-partial-trace-openings-inconsistent-bad-bound-controlled:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-trace-openings-inconsistent-bad)
adversary-initial-state ≤
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)

proof –

have *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
partial-trace-openings-inconsistent-bad)
adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
hash-map-output-collision-bad)
adversary-initial-state
by (rule *wp-event-mono*)
(auto simp: staged-security-with-data-state-verifier-event-def
intro: partial-trace-openings-inconsistent-bad-imp-hash-map-output-collision-bad
split: option.splits prod.splits)
also have *... ≤*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
[OF wf controlled])
finally show *?thesis* .

qed

lemma *checked-staged-security-with-data-state-partial-composition-openings-inconsistent-bad-bound-controlled:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event

```

    partial-composition-openings-inconsistent-bad)
  adversary-initial-state ≤
  hash-collision-budget-value 0
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget + verifier-hash-query-budget)
proof –
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     partial-composition-openings-inconsistent-bad)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     hash-map-output-collision-bad)
  adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
     intro:
      partial-composition-openings-inconsistent-bad-imp-hash-map-output-collision-bad
      split: option.splits prod.splits)
  also have ... ≤
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule
    checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
    [OF wf controlled])
  finally show ?thesis .
qed

```

lemma *checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled:*
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     partial-merkle-inconsistency-bad)
  adversary-initial-state ≤
  hash-collision-budget-value 0
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget + verifier-hash-query-budget)

```

```

proof –
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     partial-merkle-inconsistency-bad)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     hash-map-output-collision-bad)
  adversary-initial-state

```

```

    by (rule wp-event-mono)
      (auto simp: staged-security-with-data-state-verifier-event-def
        intro: partial-merkle-inconsistency-bad-imp-hash-map-output-collision-bad
        split: option.splits prod.splits)
  also have ... ≤
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule
    checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
    [OF wf controlled])
  finally show ?thesis .
qed

lemma checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad-bound-controlled:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
      adversary-initial-state ≤
      hash-collision-budget-value 0
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget + verifier-hash-query-budget)
proof -
  have event-eq:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
      adversary-initial-state =
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        partial-merkle-inconsistency-bad)
      adversary-initial-state
  unfolding
    checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad-def
  using
    checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection
    [of A partial-merkle-inconsistency-bad]
  by simp
  show ?thesis
  unfolding event-eq
  by (rule
    checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
    [OF wf controlled])
qed

```

lemma *checked-staged-security-with-data-state-supported-output-local-side-bad-bound-controlled:*

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
shows
wp-event (checked-staged-security-experiment-with-data-state *A*)
(staged-security-with-data-state-verifier-event
supported-output-local-side-bad)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
proof –
have *wp-event* (checked-staged-security-experiment-with-data-state *A*)
(staged-security-with-data-state-verifier-event
supported-output-local-side-bad)
adversary-initial-state $\leq$
wp-event (checked-staged-security-experiment-with-data-state *A*)
(staged-security-with-data-state-verifier-event
hash-map-output-collision-bad)
adversary-initial-state
by (rule *wp-event-mono*)
(auto simp: staged-security-with-data-state-verifier-event-def
supported-output-local-side-bad-iff-hash-map-output-collision-bad
split: option.splits prod.splits)
also have ... $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
[OF *wf controlled*])
finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-supported-pairwise-side-bad-bound-from-coupling:
fixes *coupling-error* :: prob
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *coupling-bound*:
wp-event (checked-staged-security-experiment-with-data-state *A*)
(staged-security-with-data-state-verifier-event
supported-pairwise-coupling-event)
adversary-initial-state $\leq$ *coupling-error*
shows
wp-event (checked-staged-security-experiment-with-data-state *A*)
(staged-security-with-data-state-verifier-event
supported-pairwise-side-bad)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +

```

      staged-challenge-query-budget + verifier-hash-query-budget) +
      coupling-error
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  have event-eq:
    ?E supported-pairwise-side-bad =
      (λout. ?E hash-map-output-collision-bad out ∨
        ?E supported-pairwise-coupling-event out)
  by (rule ext)
    (auto simp: staged-security-with-data-state-verifier-event-def
      supported-pairwise-side-bad-def
      split: option.splits prod.splits)
  have wp-event ?M (?E supported-pairwise-side-bad)
    adversary-initial-state ≤
    wp-event ?M (?E hash-map-output-collision-bad)
      adversary-initial-state +
    wp-event ?M (?E supported-pairwise-coupling-event)
      adversary-initial-state
  unfolding event-eq by (rule wp-event-union-bound)
  also have ... ≤
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget + verifier-hash-query-budget) +
    coupling-error
  by (intro add-mono coupling-bound
    checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
    [OF wf controlled])
  finally show ?thesis .
qed

```

```

lemma checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-committed-prefix-partial-header-a
fixes committed-error partial-error collision-error :: prob
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and committed-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-committed-prefix-hit A)
    adversary-initial-state ≤ committed-error
and partial-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-partial-header-key-hit
    adversary-initial-state ≤ partial-error
and collision-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      hash-map-output-collision-bad)
    adversary-initial-state ≤ collision-error
shows

```

$wp\text{-}event \text{ (checked-staged-security-experiment-with-data-state } A)$
 $\text{ (staged-security-with-data-state-verifier-event query-bad)}$
 $adversary\text{-}initial\text{-}state \leq$
 $committed\text{-}error + (partial\text{-}error + collision\text{-}error)$
proof –
let $?M = \text{checked-staged-security-experiment-with-data-state } A$
have $tree\text{-}bound$:
 $wp\text{-}event ?M \text{ (staged-security-with-data-state-query-tree-output-hit } A)$
 $adversary\text{-}initial\text{-}state \leq partial\text{-}error + collision\text{-}error$
proof –
have $wp\text{-}event ?M$
 $\text{ (staged-security-with-data-state-query-tree-output-hit } A)$
 $adversary\text{-}initial\text{-}state \leq$
 $wp\text{-}event ?M \text{ staged-security-with-data-state-query-partial-header-key-hit}$
 $adversary\text{-}initial\text{-}state +$
 $wp\text{-}event ?M$
 $\text{ (staged-security-with-data-state-verifier-event}$
 $\text{ hash-map-output-collision-bad)}$
 $adversary\text{-}initial\text{-}state$
by (rule
 $\text{ staged-security-with-data-state-query-tree-output-hit-bound-from-partial-header-key-hit-or-collision}$
 $[OF \text{ wf controlled}]$)
also have $\dots \leq partial\text{-}error + collision\text{-}error$
by (intro add-mono partial-bound collision-bound)
finally show $?thesis$.
qed
show $?thesis$
by (rule
 $\text{ checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-committed-prefix-hit-or-tree-output}$
 $[OF \text{ wf controlled committed-bound tree-bound}]$)
qed

lemma $\text{checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-partial-opening-fixed-query-error-}$
assumes wf : $\text{staged-budget-wellformed budgets}$
and controlled : $\text{staged-adversary-controlled budgets } A$
and cover :
 $\bigwedge \text{data attacker-state } i.$
 $i < rounds \implies$
 $\text{query-header-supported-partial-opening-success-indices-at}$
 $\text{ (verifier-state-from-adversary attacker-state}$
 $\text{ (staged-proof-transcript data))}$
 $\text{ (staged-trace-root data)}$
 $\text{ (staged-trace-fri-roots data)}$
 $\text{ (staged-trace-final data)}$
 $\text{ (staged-alphas data)}$
 $\text{ (staged-degree data)}$
 $\text{ (staged-composition-fri-roots data)}$
 $\text{ (staged-composition-final data)}$
 $i \subseteq B$

and *raw-bound*: *query-index-raw-preimage-bound*
and *subset*: $B \subseteq \text{query-sample-space}$
and *frac*:
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space}) \leq$
 query-error-bound
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event } \text{query-bad})$
 $\text{adversary-initial-state} \leq$
 nnreal
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}) * \text{query-error-bound}$
by (*rule checked-staged-security-with-data-state-query-bad-verifier-event-bound*)
(rule
 $\text{checked-staged-security-with-data-state-query-bad-bound-from-partial-opening-fixed-query-error-cover}$
 $[OF \text{ wf controlled cover raw-bound subset frac}]$)

lemma *staged-transcript-query-partial-header-key-hit-iff-round-hit*:
 $\text{staged-transcript-query-partial-header-key-hit out} \longleftrightarrow$
 $(\exists i \in \{..<\text{rounds}\}).$
 $\text{staged-transcript-query-partial-header-key-hit-at } i \text{ out})$
unfolding *staged-transcript-query-partial-header-key-hit-alt*
 $\text{staged-transcript-query-partial-header-key-hit-at-def}$
by (*auto split: option.splits prod.splits*)

lemma *checked-staged-transcript-query-partial-header-key-hit-bound-from-rounds*:
assumes *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{wp-event } (\text{checked-staged-transcript-program } A)$
 $(\text{staged-transcript-query-partial-header-key-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
shows
 $\text{wp-event } (\text{checked-staged-transcript-program } A)$
 $\text{staged-transcript-query-partial-header-key-hit}$
 $\text{adversary-initial-state} \leq (\sum i < \text{rounds}. C \ i)$
proof –
have *event-eq*:
 $\text{staged-transcript-query-partial-header-key-hit} =$
 $(\lambda \text{out}. \exists i \in \{..<\text{rounds}\}).$
 $\text{staged-transcript-query-partial-header-key-hit-at } i \text{ out})$
by (*rule ext*)
(simp add:
 $\text{staged-transcript-query-partial-header-key-hit-iff-round-hit})$
show *?thesis*
unfolding *event-eq*
by (*rule wp-event-finite-union-bound*)
(simp-all add: round-bound)
qed

lemma *checked-staged-security-with-data-state-partial-header-key-hit-bound-from-dynamic-transcript:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *transcript-bound:*

wp-event (checked-staged-transcript-program A)

staged-transcript-query-partial-header-key-hit

adversary-initial-state $\leq C$

shows

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-query-partial-header-key-hit

adversary-initial-state $\leq C$

unfolding *checked-staged-security-experiment-with-data-state-def*

proof (*rule wp-event-bind-bound-by-head-event[OF transcript-bound]*)

show

staged-security-with-data-state-query-partial-header-key-hit None $\implies$

staged-transcript-query-partial-header-key-hit None

unfolding *staged-security-with-data-state-query-partial-header-key-hit-def*

staged-transcript-query-partial-header-key-hit-def

by *simp*

next

fix *data attacker-state out*

assume *builder:*

Some (data, attacker-state) $\in$

set-dist

(execute (checked-staged-transcript-program A)

adversary-initial-state)

and *cont:*

out $\in$

set-dist

(execute

(get $\gg$

(λs . put

(verifier-state-from-adversary s

(staged-proof-transcript data)) $\gg$

(λ -. verify-monad $\gg$

(λ result. return ((data, s), result))))))

attacker-state)

and *hit:*

staged-security-with-data-state-query-partial-header-key-hit out

from *checked-staged-transcript-program-outcome-query-lookups*

[OF wf controlled builder]

obtain *raw-idxs query-idxs where*

len-raw: length raw-idxs = rounds

and *lookup-attacker:*

$\forall j < \text{rounds}.$

fmlookup (HashMap attacker-state)

(QueryIndexChallenge j

(state-after-query-chunks

(staged-query-start-hash data)

```

      (staged-query-chunks data) j)) =
    Some (raw-idxs ! j)
  by blast
from hit obtain data' attacker-state' result final-state i raw where
  out-eq:
    out = Some (((data', attacker-state'), result), final-state)
and i-bound: i < rounds
and lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data')
        (staged-query-chunks data') i)) =
    Some raw
and raw-hit:
  index (to-nat raw) ∈
    query-header-supported-partial-union-good-sets
      (verifier-state-from-adversary attacker-state'
        (staged-proof-transcript data'))
    query-sampling-success-space
      (staged-trace-root data')
      (staged-trace-fri-roots data')
      (staged-trace-final data')
      (staged-alphas data')
      (staged-degree data')
      (staged-composition-fri-roots data')
      (staged-composition-final data')
unfolding staged-security-with-data-state-query-partial-header-key-hit-def
by (auto split: option.splits prod.splits)
from cont[unfolded out-eq] obtain verifier-state verifier-result where
  verifier:
    Some (verifier-result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))))
and data'-eq: data' = data
and attacker-state'-eq: attacker-state' = attacker-state
by (auto elim!: set-dist-bindE)
have ext:
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data) ≤ final-state
by (rule verify-monad-hash-extends[OF verifier])
have lookup-initial:
  fmlookup
    (HashMap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (QueryIndexChallenge i

```

```

    (state-after-query-chunks
      (staged-query-start-hash data)
      (staged-query-chunks data) i)) =
  Some (raw-idxs ! i)
using lookup-attacker i-bound by simp
have lookup-final-from-initial:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
by (rule hash-extension-lookup[OF lookup-initial ext])
have raw-eq: raw = raw-idxs ! i
using lookup-final lookup-final-from-initial data'-eq by simp
have lookup-attacker-i:
  fmlookup (HashMap attacker-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
using lookup-attacker i-bound by simp
have raw-idxs-hit:
  index (to-nat (raw-idxs ! i)) ∈
    query-header-supported-partial-union-good-sets
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
    query-sampling-success-space
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
using raw-hit raw-eq data'-eq attacker-state'-eq by simp
show staged-transcript-query-partial-header-key-hit
  (Some (data, attacker-state))
unfolding staged-transcript-query-partial-header-key-hit-def Let-def
using i-bound lookup-attacker-i raw-idxs-hit by auto
qed

```

definition *checked-staged-event-soundness-bound*
 $:: \text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob}$
where
checked-staged-event-soundness-bound merkle-error composition-error
query-error =
merkle-error + composition-error + trace-fri-error +

composition-fri-error + query-error

lemma *soundness-bad-event-union-bound:*

fixes *s :: ('f, 'a) protocol-channel-scheme*

assumes *comp-bound:*

wp-event verify-monad (composition-bad s) s ≤ composition-error-bound

and *trace-fri-bound:*

wp-event verify-monad (trace-fri-bad s) s ≤ trace-fri-error

and *comp-fri-bound:*

wp-event verify-monad (composition-fri-bad s) s ≤ composition-fri-error

and *query-bound:*

*wp-event verify-monad (query-bad s) s ≤ nnreal rounds * query-error-bound*

shows *wp-event verify-monad (soundness-bad-event s) s ≤ soundness-bound*

proof –

have *wp-event verify-monad (soundness-bad-event s) s ≤*

wp-event verify-monad (composition-bad s) s +

wp-event verify-monad (trace-fri-bad s) s +

wp-event verify-monad (composition-fri-bad s) s +

wp-event verify-monad (query-bad s) s

unfolding *soundness-bad-event-def*

by *(rule wp-event-union-bound4)*

also have *... ≤*

composition-error-bound + trace-fri-error + composition-fri-error +

*nnreal rounds * query-error-bound*

by *(intro add-mono comp-bound trace-fri-bound comp-fri-bound query-bound)*

also have *... = soundness-bound*

by *(simp add: soundness-bound-component-accounting)*

finally show *?thesis .*

qed

lemma *soundness-bad-event-table-aware-union-bound:*

fixes *s :: ('f, 'a) protocol-channel-scheme*

assumes *comp-bound:*

wp-event verify-monad (composition-bad s) s ≤ composition-error-bound

and *trace-fri-bound:*

wp-event verify-monad (trace-fri-bad-with-tables s) s ≤

trace-fri-error

and *comp-fri-bound:*

wp-event verify-monad (composition-fri-bad-with-tables s) s ≤

composition-fri-error

and *query-bound:*

*wp-event verify-monad (query-bad s) s ≤ nnreal rounds * query-error-bound*

shows

wp-event verify-monad (soundness-bad-event-table-aware s) s ≤

soundness-bound

proof –

have *wp-event verify-monad (soundness-bad-event-table-aware s) s ≤*

wp-event verify-monad (composition-bad s) s +

wp-event verify-monad (trace-fri-bad-with-tables s) s +


```

    wp-event verify-monad (composition-fri-bad-with-tables s) s +
    wp-event verify-monad (query-bad s) s
  unfolding soundness-bad-event-table-aware-def
  by (rule wp-event-union-bound4)
also have ... ≤
  composition-error-bound + trace-fri-error + composition-fri-error +
  nnreal rounds * query-error-bound
  by (intro add-mono comp-bound trace-fri-bound comp-fri-bound query-bound)
also have ... = soundness-bound
  by (simp add: soundness-bound-component-accounting)
finally show ?thesis .
qed

```

lemma *soundness-bad-event-partial-opening-union-bound:*
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes *comp-bound:*
 $\text{wp-event verify-monad (composition-bad s) } s \leq \text{composition-error-bound}$
and *trace-fri-bound:*
 $\text{wp-event verify-monad (trace-fri-bad-with-partial-openings s) } s \leq \text{trace-fri-error}$
and *comp-fri-bound:*
 $\text{wp-event verify-monad (composition-fri-bad-with-partial-openings s) } s \leq \text{composition-fri-error}$
and *query-bound:*
 $\text{wp-event verify-monad (query-bad s) } s \leq \text{nnreal rounds * query-error-bound}$
shows
 $\text{wp-event verify-monad (soundness-bad-event-partial-opening s) } s \leq \text{soundness-bound}$
proof –
have $\text{wp-event verify-monad (soundness-bad-event-partial-opening s) } s \leq$
 $\text{wp-event verify-monad (composition-bad s) } s +$
 $\text{wp-event verify-monad (trace-fri-bad-with-partial-openings s) } s +$
 $\text{wp-event verify-monad (composition-fri-bad-with-partial-openings s) } s +$
 $\text{wp-event verify-monad (query-bad s) } s$
unfolding *soundness-bad-event-partial-opening-def*
by (rule wp-event-union-bound4)
also have ... ≤
 composition-error-bound + trace-fri-error + composition-fri-error +
 nnreal rounds * query-error-bound
by (intro add-mono comp-bound trace-fri-bound comp-fri-bound query-bound)
also have ... = soundness-bound
by (simp add: soundness-bound-component-accounting)
finally show ?thesis .
qed

lemma *soundness-bad-event-table-aware-with-merkle-union-bound:*
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes merkle-bound:
 $wp\text{-event verify-monad (fri-merkle-binding-bad } s) s \leq$
 $merkle\text{-binding-error}$
and comp-bound:
 $wp\text{-event verify-monad (composition-bad } s) s \leq$
 $composition\text{-error-bound}$
and trace-fri-bound:
 $wp\text{-event verify-monad (trace-fri-bad-with-tables } s) s \leq$
 $trace\text{-fri-error}$
and comp-fri-bound:
 $wp\text{-event verify-monad (composition-fri-bad-with-tables } s) s \leq$
 $composition\text{-fri-error}$
and query-bound:
 $wp\text{-event verify-monad (query-bad } s) s \leq$
 $nnreal\ rounds * query\text{-error-bound}$
shows
 $wp\text{-event verify-monad}$
 $(soundness\text{-bad-event-table-aware-with-merkle } s) s \leq$
 $soundness\text{-bound-with-merkle}$
proof –
have table-bound:
 $wp\text{-event verify-monad (soundness\text{-bad-event-table-aware } s) s \leq$
 $soundness\text{-bound}$
by (rule soundness-bad-event-table-aware-union-bound
 $[OF\ comp\text{-bound trace-fri-bound comp-fri-bound query-bound}]$)
have wp-event verify-monad
 $(soundness\text{-bad-event-table-aware-with-merkle } s) s \leq$
 $wp\text{-event verify-monad (fri-merkle-binding-bad } s) s +$
 $wp\text{-event verify-monad (soundness\text{-bad-event-table-aware } s) s$
unfolding soundness-bad-event-table-aware-with-merkle-def
by (rule wp-event-union-bound)
also have $\dots \leq merkle\text{-binding-error} + soundness\text{-bound}$
by (intro add-mono merkle-bound table-bound)
also have $\dots = soundness\text{-bound-with-merkle}$
unfolding soundness-bound-with-merkle-def **by** (simp add: algebra-simps)
finally show ?thesis .
qed

lemma soundness-bad-event-partial-opening-with-merkle-union-bound:

fixes $s :: ('f, 'a)\ protocol\text{-channel-scheme}$
assumes merkle-bound:
 $wp\text{-event verify-monad (fri-merkle-binding-bad } s) s \leq$
 $merkle\text{-binding-error}$
and comp-bound:
 $wp\text{-event verify-monad (composition-bad } s) s \leq$
 $composition\text{-error-bound}$
and trace-fri-bound:
 $wp\text{-event verify-monad (trace-fri-bad-with-partial-openings } s) s \leq$
 $trace\text{-fri-error}$

and *comp-fri-bound*:
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}bad\text{-}with\text{-}partial\text{-}openings\ s)\ s \leq$
 $composition\text{-}fri\text{-}error$
and *query-bound*:
 $wp\text{-}event\ verify\text{-}monad\ (query\text{-}bad\ s)\ s \leq$
 $nnreal\ rounds * query\text{-}error\text{-}bound$
shows
 $wp\text{-}event\ verify\text{-}monad$
 $(soundness\text{-}bad\text{-}event\text{-}partial\text{-}opening\text{-}with\text{-}merkle\ s)\ s \leq$
 $soundness\text{-}bound\text{-}with\text{-}merkle$
proof –
have *partial-bound*:
 $wp\text{-}event\ verify\text{-}monad\ (soundness\text{-}bad\text{-}event\text{-}partial\text{-}opening\ s)\ s \leq$
 $soundness\text{-}bound$
by (rule *soundness-bad-event-partial-opening-union-bound*
 $[OF\ comp\text{-}bound\ trace\text{-}fri\text{-}bound\ comp\text{-}fri\text{-}bound\ query\text{-}bound]$)
have *wp-event verify-monad*
 $(soundness\text{-}bad\text{-}event\text{-}partial\text{-}opening\text{-}with\text{-}merkle\ s)\ s \leq$
 $wp\text{-}event\ verify\text{-}monad\ (fri\text{-}merkle\text{-}binding\text{-}bad\ s)\ s +$
 $wp\text{-}event\ verify\text{-}monad\ (soundness\text{-}bad\text{-}event\text{-}partial\text{-}opening\ s)\ s$
unfolding *soundness-bad-event-partial-opening-with-merkle-def*
by (rule *wp-event-union-bound*)
also have $\dots \leq merkle\text{-}binding\text{-}error + soundness\text{-}bound$
by (intro *add-mono merkle-bound partial-bound*)
also have $\dots = soundness\text{-}bound\text{-}with\text{-}merkle$
unfolding *soundness-bound-with-merkle-def* **by** (*simp add: algebra-simps*)
finally show *?thesis* .
qed

lemma *soundness-bad-event-partial-opening-with-initial-merkle-union-bound*:
fixes $s :: ('f, 'a)\ protocol\text{-}channel\text{-}scheme$
assumes *merkle-bound*:
 $wp\text{-}event\ verify\text{-}monad\ (initial\text{-}merkle\text{-}binding\text{-}bad\ s)\ s \leq$
 $merkle\text{-}binding\text{-}error$
and *comp-bound*:
 $wp\text{-}event\ verify\text{-}monad\ (composition\text{-}bad\ s)\ s \leq$
 $composition\text{-}error\text{-}bound$
and *trace-fri-bound*:
 $wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}bad\text{-}with\text{-}partial\text{-}openings\ s)\ s \leq$
 $trace\text{-}fri\text{-}error$
and *comp-fri-bound*:
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}bad\text{-}with\text{-}partial\text{-}openings\ s)\ s \leq$
 $composition\text{-}fri\text{-}error$
and *query-bound*:
 $wp\text{-}event\ verify\text{-}monad\ (query\text{-}bad\ s)\ s \leq$
 $nnreal\ rounds * query\text{-}error\text{-}bound$
shows

$wp\text{-event verify-monad}$
 $(\text{soundness-bad-event-partial-opening-with-initial-merkle } s) \ s \leq$
 $\text{soundness-bound-with-merkle}$

proof –

have partial-bound :

$wp\text{-event verify-monad } (\text{soundness-bad-event-partial-opening } s) \ s \leq$
 soundness-bound

by (rule $\text{soundness-bad-event-partial-opening-union-bound}$
 $[\text{OF comp-bound trace-fri-bound comp-fri-bound query-bound}]$)

have $wp\text{-event verify-monad}$
 $(\text{soundness-bad-event-partial-opening-with-initial-merkle } s) \ s \leq$
 $wp\text{-event verify-monad } (\text{initial-merkle-binding-bad } s) \ s +$
 $wp\text{-event verify-monad } (\text{soundness-bad-event-partial-opening } s) \ s$

unfolding $\text{soundness-bad-event-partial-opening-with-initial-merkle-def}$

by (rule $wp\text{-event-union-bound}$)

also have $\dots \leq \text{merkle-binding-error} + \text{soundness-bound}$

by (intro $\text{add-mono merkle-bound partial-bound}$)

also have $\dots = \text{soundness-bound-with-merkle}$

unfolding $\text{soundness-bound-with-merkle-def}$ **by** (simp $\text{add: algebra-simps}$)

finally show $?thesis$.

qed

lemma $\text{soundness-bad-event-partial-opening-with-partial-merkle-union-bound}$:

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes merkle-bound :

$wp\text{-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq$
 $\text{merkle-binding-error}$

and comp-bound :

$wp\text{-event verify-monad } (\text{composition-bad } s) \ s \leq$
 $\text{composition-error-bound}$

and trace-fri-bound :

$wp\text{-event verify-monad } (\text{trace-fri-bad-with-partial-openings } s) \ s \leq$
 trace-fri-error

and comp-fri-bound :

$wp\text{-event verify-monad}$
 $(\text{composition-fri-bad-with-partial-openings } s) \ s \leq$
 $\text{composition-fri-error}$

and query-bound :

$wp\text{-event verify-monad } (\text{query-bad } s) \ s \leq$
 $\text{nnreal rounds} * \text{query-error-bound}$

shows

$wp\text{-event verify-monad}$
 $(\text{soundness-bad-event-partial-opening-with-partial-merkle } s) \ s \leq$
 $\text{soundness-bound-with-merkle}$

proof –

have partial-bound :

$wp\text{-event verify-monad } (\text{soundness-bad-event-partial-opening } s) \ s \leq$
 soundness-bound

by (rule $\text{soundness-bad-event-partial-opening-union-bound}$)

$[OF \text{ comp-bound trace-fri-bound comp-fri-bound query-bound}]$
have *wp-event verify-monad*
 $(\text{soundness-bad-event-partial-opening-with-partial-merkle } s) \ s \leq$
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s +$
 $\text{wp-event verify-monad } (\text{soundness-bad-event-partial-opening } s) \ s$
unfolding *soundness-bad-event-partial-opening-with-partial-merkle-def*
by *(rule wp-event-union-bound)*
also have $\dots \leq \text{merkle-binding-error} + \text{soundness-bound}$
by *(intro add-mono merkle-bound partial-bound)*
also have $\dots = \text{soundness-bound-with-merkle}$
unfolding *soundness-bound-with-merkle-def* **by** *(simp add: algebra-simps)*
finally show *?thesis* .
qed

lemma *soundness-bad-event-partial-candidate-union-bound:*

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes *comp-bound:*

$\text{wp-event verify-monad}$
 $(\text{composition-bad-with-partial-candidates } s) \ s \leq$
 $\text{composition-error-bound}$

and *trace-fri-bound:*

$\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-partial-candidates } s) \ s \leq$
 trace-fri-error

and *comp-fri-bound:*

$\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-partial-candidates } s) \ s \leq$
 $\text{composition-fri-error}$

and *query-bound:*

$\text{wp-event verify-monad}$
 $(\text{query-bad-with-partial-candidates } s) \ s \leq$
 $\text{nnreal rounds} * \text{query-error-bound}$

shows

$\text{wp-event verify-monad } (\text{soundness-bad-event-partial-candidate } s) \ s \leq$
 soundness-bound

proof –

have $\text{wp-event verify-monad } (\text{soundness-bad-event-partial-candidate } s) \ s \leq$

$\text{wp-event verify-monad}$
 $(\text{composition-bad-with-partial-candidates } s) \ s +$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-partial-candidates } s) \ s +$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-partial-candidates } s) \ s +$
 $\text{wp-event verify-monad } (\text{query-bad-with-partial-candidates } s) \ s$

unfolding *soundness-bad-event-partial-candidate-def*

by *(rule wp-event-union-bound4)*

also have $\dots \leq \text{composition-error-bound} + \text{trace-fri-error} +$
 $\text{composition-fri-error} + \text{nnreal rounds} * \text{query-error-bound}$

by *(intro add-mono comp-bound trace-fri-bound comp-fri-bound)*

```

    query-bound)
  also have ... = soundness-bound
    unfolding soundness-bound-def by (simp add: algebra-simps)
  finally show ?thesis .
qed

lemma soundness-bad-event-partial-candidate-with-partial-merkle-union-bound:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤
      merkle-binding-error
  and comp-bound:
    wp-event verify-monad
      (composition-bad-with-partial-candidates s) s ≤
        composition-error-bound
  and trace-fri-bound:
    wp-event verify-monad
      (trace-fri-bad-with-partial-candidates s) s ≤
        trace-fri-error
  and comp-fri-bound:
    wp-event verify-monad
      (composition-fri-bad-with-partial-candidates s) s ≤
        composition-fri-error
  and query-bound:
    wp-event verify-monad
      (query-bad-with-partial-candidates s) s ≤
        nnreal rounds * query-error-bound
  shows
    wp-event verify-monad
      (soundness-bad-event-partial-candidate-with-partial-merkle s) s ≤
        soundness-bound-with-merkle
proof -
  have partial-bound:
    wp-event verify-monad (soundness-bad-event-partial-candidate s) s ≤
      soundness-bound
  by (rule soundness-bad-event-partial-candidate-union-bound
    [OF comp-bound trace-fri-bound comp-fri-bound query-bound])
  have wp-event verify-monad
    (soundness-bad-event-partial-candidate-with-partial-merkle s) s ≤
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s +
    wp-event verify-monad (soundness-bad-event-partial-candidate s) s
  unfolding soundness-bad-event-partial-candidate-with-partial-merkle-def
  by (rule wp-event-union-bound)
  also have ... ≤ merkle-binding-error + soundness-bound
    by (intro add-mono merkle-bound partial-bound)
  also have ... = soundness-bound-with-merkle
    unfolding soundness-bound-with-merkle-def by (simp add: algebra-simps)
  finally show ?thesis .
qed

```

lemma *checked-staged-security-with-data-state-partial-candidate-with-partial-merkle-union-bound:*
fixes *merkle-error composition-error trace-fri-error'*
composition-fri-error' query-error :: prob
assumes *merkle-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 partial-merkle-inconsistency-bad)
 adversary-initial-state ≤ merkle-error
and *comp-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-bad-with-partial-candidates)
 adversary-initial-state ≤ composition-error
and *trace-fri-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state ≤ trace-fri-error'
and *comp-fri-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state ≤ composition-fri-error'
and *query-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 query-bad-with-partial-candidates)
 adversary-initial-state ≤ query-error
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 soundness-bad-event-partial-candidate-with-partial-merkle)
 adversary-initial-state ≤
 merkle-error + composition-error + trace-fri-error' +
 composition-fri-error' + query-error
proof –
 let *?M = checked-staged-security-experiment-with-data-state A*
 let *?E = staged-security-with-data-state-verifier-event*
 let *?merkle = ?E partial-merkle-inconsistency-bad*
 let *?comp = ?E composition-bad-with-partial-candidates*
 let *?trace = ?E trace-fri-bad-with-partial-candidates*
 let *?comp-fri = ?E composition-fri-bad-with-partial-candidates*
 let *?query = ?E query-bad-with-partial-candidates*
 let *?partial = ?E soundness-bad-event-partial-candidate*
 let *?with-merkle =*
 ?E soundness-bad-event-partial-candidate-with-partial-merkle
 have *partial-bound:*
 wp-event ?M ?partial adversary-initial-state ≤

```

    composition-error + trace-fri-error' + composition-fri-error' +
    query-error
proof -
  have wp-event ?M ?partial adversary-initial-state ≤
    wp-event ?M
      (λout. ?comp out ∨ ?trace out ∨ ?comp-fri out ∨
        ?query out)
      adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
      soundness-bad-event-partial-candidate-def
      split: option.splits prod.splits)
  also have ... ≤
    wp-event ?M ?comp adversary-initial-state +
    wp-event ?M ?trace adversary-initial-state +
    wp-event ?M ?comp-fri adversary-initial-state +
    wp-event ?M ?query adversary-initial-state
  by (rule wp-event-union-bound4)
  also have ... ≤
    composition-error + trace-fri-error' + composition-fri-error' +
    query-error
  by (intro add-mono comp-bound trace-fri-bound comp-fri-bound
    query-bound)
  finally show ?thesis .
qed
have wp-event ?M ?with-merkle adversary-initial-state ≤
  wp-event ?M (λout. ?merkle out ∨ ?partial out)
    adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
    soundness-bad-event-partial-candidate-with-partial-merkle-def
    split: option.splits prod.splits)
also have ... ≤
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?partial adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
  merkle-error +
  (composition-error + trace-fri-error' + composition-fri-error' +
    query-error)
by (intro add-mono merkle-bound partial-bound)
finally show ?thesis
by (simp add: algebra-simps)
qed

lemma soundness-bad-event-partial-candidate-empty-header-union-bound:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes comp-bound:
  wp-event verify-monad

```



```

      (composition-bad-with-empty-composition-header-candidates s) s ≤
      composition-error-bound
and trace-fri-bound:
  wp-event verify-monad
    (trace-fri-bad-with-empty-composition-header-candidates s) s ≤
    trace-fri-error
and query-bound:
  wp-event verify-monad
    (query-bad-with-empty-composition-header-candidates s) s ≤
    nnreal rounds * query-error-bound
shows
  wp-event verify-monad
    (soundness-bad-event-partial-candidate-empty-header s) s ≤
    composition-error-bound + trace-fri-error +
    nnreal rounds * query-error-bound
proof —
  have wp-event verify-monad
    (soundness-bad-event-partial-candidate-empty-header s) s ≤
    wp-event verify-monad
      (λout.
        composition-bad-with-empty-composition-header-candidates s out ∨
        trace-fri-bad-with-empty-composition-header-candidates s out ∨
        query-bad-with-empty-composition-header-candidates s out ∨
        False) s
  unfolding soundness-bad-event-partial-candidate-empty-header-def
  by simp
also have ... ≤
  wp-event verify-monad
    (composition-bad-with-empty-composition-header-candidates s) s +
  wp-event verify-monad
    (trace-fri-bad-with-empty-composition-header-candidates s) s +
  wp-event verify-monad
    (query-bad-with-empty-composition-header-candidates s) s +
  wp-event verify-monad (λ-. False) s
  by (rule wp-event-union-bound4)
also have ... =
  wp-event verify-monad
    (composition-bad-with-empty-composition-header-candidates s) s +
  wp-event verify-monad
    (trace-fri-bad-with-empty-composition-header-candidates s) s +
  wp-event verify-monad
    (query-bad-with-empty-composition-header-candidates s) s
  unfolding wp-event-def wp-def dist-expect-def by simp
also have ... ≤
  composition-error-bound + trace-fri-error +
  nnreal rounds * query-error-bound
  by (intro add-mono comp-bound trace-fri-bound query-bound)
finally show ?thesis .
qed

```

lemma *composition-bad-with-empty-composition-header-candidates-union-bound:*

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes *false-statement:* $\neg \text{exists-valid-trace}$

and *degree-bound:*

wp-event verify-monad

$(\text{composition-degree-bad-with-empty-composition-header-candidates } s) \ s \leq \text{degree-error}$

and *randomization-bound:*

wp-event verify-monad

$(\text{composition-randomization-bad-with-empty-composition-header-candidates } s) \ s \leq \text{randomization-error}$

shows

wp-event verify-monad

$(\text{composition-bad-with-empty-composition-header-candidates } s) \ s \leq \text{degree-error} + \text{randomization-error}$

proof $-$

have *wp-event verify-monad*

$(\text{composition-bad-with-empty-composition-header-candidates } s) \ s \leq$

wp-event verify-monad

$(\lambda \text{out.}$

composition-degree-bad-with-empty-composition-header-candidates

$s \ \text{out} \ \vee$

composition-randomization-bad-with-empty-composition-header-candidates

$s \ \text{out}) \ s$

by $(\text{rule wp-event-mono})$

$(\text{rule composition-bad-with-empty-composition-header-candidates-split}$

$[\text{OF false-statement}])$

also have $\dots \leq$

wp-event verify-monad

$(\text{composition-degree-bad-with-empty-composition-header-candidates } s) \ s +$

wp-event verify-monad

$(\text{composition-randomization-bad-with-empty-composition-header-candidates } s) \ s$

by $(\text{rule wp-event-union-bound})$

also have $\dots \leq \text{degree-error} + \text{randomization-error}$

by $(\text{intro add-mono degree-bound randomization-bound})$

finally show *?thesis* .

qed

lemma *soundness-bad-event-partial-candidate-empty-header-split-union-bound:*

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes *false-statement:* $\neg \text{exists-valid-trace}$

and *degree-bound:*

wp-event verify-monad

$(\text{composition-degree-bad-with-empty-composition-header-candidates } s) \ s \leq \text{degree-error}$

and *randomization-bound:*

wp-event verify-monad

(composition-randomization-bad-with-empty-composition-header-candidates
 s) $s \leq$ randomization-error
and trace-fri-bound:
 wp-event verify-monad
 (trace-fri-bad-with-empty-composition-header-candidates s) $s \leq$
 trace-fri-error
and query-bound:
 wp-event verify-monad
 (query-bad-with-empty-composition-header-candidates s) $s \leq$
 nnreal rounds * query-error-bound
shows
 wp-event verify-monad
 (soundness-bad-event-partial-candidate-empty-header s) $s \leq$
 degree-error + randomization-error + trace-fri-error +
 nnreal rounds * query-error-bound
proof –
have wp-event verify-monad
 (soundness-bad-event-partial-candidate-empty-header s) $s \leq$
 wp-event verify-monad
 (λ out.
 composition-degree-bad-with-empty-composition-header-candidates
 s out $\vee$
 composition-randomization-bad-with-empty-composition-header-candidates
 s out $\vee$
 trace-fri-bad-with-empty-composition-header-candidates s out $\vee$
 query-bad-with-empty-composition-header-candidates s out) s
proof (rule wp-event-mono)
fix out
assume empty:
 soundness-bad-event-partial-candidate-empty-header s out
then consider
 (comp) composition-bad-with-empty-composition-header-candidates s out
 | (trace) trace-fri-bad-with-empty-composition-header-candidates s out
 | (query) query-bad-with-empty-composition-header-candidates s out
unfolding soundness-bad-event-partial-candidate-empty-header-def
by blast
then show
 composition-degree-bad-with-empty-composition-header-candidates
 s out $\vee$
 composition-randomization-bad-with-empty-composition-header-candidates
 s out $\vee$
 trace-fri-bad-with-empty-composition-header-candidates s out $\vee$
 query-bad-with-empty-composition-header-candidates s out
proof cases
case comp
then show ?thesis
using composition-bad-with-empty-composition-header-candidates-split
 [OF false-statement comp]
by blast

```

qed blast+
qed
also have ...  $\leq$ 
  wp-event verify-monad
    (composition-degree-bad-with-empty-composition-header-candidates s) s +
  wp-event verify-monad
    (composition-randomization-bad-with-empty-composition-header-candidates
      s) s +
  wp-event verify-monad
    (trace-fri-bad-with-empty-composition-header-candidates s) s +
  wp-event verify-monad
    (query-bad-with-empty-composition-header-candidates s) s
by (rule wp-event-union-bound4)
also have ...  $\leq$ 
  degree-error + randomization-error + trace-fri-error +
  nnreal rounds * query-error-bound
by (intro add-mono degree-bound randomization-bound trace-fri-bound
  query-bound)
finally show ?thesis .
qed

lemma soundness-bad-event-partial-candidate-empty-header-reduced-union-bound:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes false-statement:  $\neg$  exists-valid-trace
and randomization-bound:
  wp-event verify-monad
    (composition-randomization-bad-with-empty-composition-header-candidates
      s) s  $\leq$  composition-error-bound
and trace-fri-bound:
  wp-event verify-monad
    (trace-fri-bad-with-empty-composition-header-candidates s) s  $\leq$ 
    trace-fri-error
and query-bound:
  wp-event verify-monad
    (query-bad-with-empty-composition-header-candidates s) s  $\leq$ 
    nnreal rounds * query-error-bound
shows
  wp-event verify-monad
    (soundness-bad-event-partial-candidate-empty-header s) s  $\leq$ 
    composition-error-bound + trace-fri-error +
    nnreal rounds * query-error-bound
proof –
have split-bound:
  wp-event verify-monad
    (soundness-bad-event-partial-candidate-empty-header s) s  $\leq$ 
    0 + composition-error-bound + trace-fri-error +
    nnreal rounds * query-error-bound
by (rule soundness-bad-event-partial-candidate-empty-header-split-union-bound
  [OF false-statement

```

```

      composition-degree-bad-with-empty-composition-header-candidates-bound
      randomization-bound trace-fri-bound query-bound])
then show ?thesis by simp
qed

lemma soundness-bad-event-partial-candidate-empty-header-bound-from-alpha-hit:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes false-statement:  $\neg$  exists-valid-trace
  and alpha-bound:
    wp-event verify-monad
    (composition-alpha-bad-set-hit-with-empty-composition-header-candidates
     s) s  $\leq$  composition-error-bound
  and trace-fri-bound:
    wp-event verify-monad
    (trace-fri-bad-with-empty-composition-header-candidates s) s  $\leq$ 
    trace-fri-error
  and query-bound:
    wp-event verify-monad
    (query-bad-with-empty-composition-header-candidates s) s  $\leq$ 
    nnreal rounds * query-error-bound
  shows
    wp-event verify-monad
    (soundness-bad-event-partial-candidate-empty-header s) s  $\leq$ 
    composition-error-bound + trace-fri-error +
    nnreal rounds * query-error-bound
proof -
  have randomization-bound:
    wp-event verify-monad
    (composition-randomization-bad-with-empty-composition-header-candidates
     s) s  $\leq$  composition-error-bound
proof -
  have wp-event verify-monad
    (composition-randomization-bad-with-empty-composition-header-candidates
     s) s  $\leq$ 
    wp-event verify-monad
    (composition-alpha-bad-set-hit-with-empty-composition-header-candidates
     s) s
  by (rule
      wp-composition-randomization-bad-with-empty-composition-header-candidates-le-alpha-hit)
  also have ...  $\leq$  composition-error-bound
  by (rule alpha-bound)
  finally show ?thesis .
qed
show ?thesis
  by (rule soundness-bad-event-partial-candidate-empty-header-reduced-union-bound
      [OF false-statement randomization-bound trace-fri-bound query-bound])
qed

```

lemma *soundness-bad-event-partial-candidate-empty-header-bound-from-partial-opening-alpha-union-hit:*

```

fixes  $s :: ('f, 'a) \text{ protocol-channel-scheme}$ 
assumes  $\text{false-statement} : \neg \text{exists-valid-trace}$ 
and  $\text{alpha-bound} :$ 
   $\text{wp-event verify-monad}$ 
   $(\text{composition-alpha-partial-opening-union-hit-with-empty-header } s) \ s \leq$ 
   $\text{composition-error-bound}$ 
and  $\text{trace-fri-bound} :$ 
   $\text{wp-event verify-monad}$ 
   $(\text{trace-fri-bad-with-empty-composition-header-candidates } s) \ s \leq$ 
   $\text{trace-fri-error}$ 
and  $\text{query-bound} :$ 
   $\text{wp-event verify-monad}$ 
   $(\text{query-bad-with-empty-composition-header-candidates } s) \ s \leq$ 
   $\text{nnreal rounds} * \text{query-error-bound}$ 
shows
   $\text{wp-event verify-monad}$ 
   $(\text{soundness-bad-event-partial-candidate-empty-header } s) \ s \leq$ 
   $\text{composition-error-bound} + \text{trace-fri-error} +$ 
   $\text{nnreal rounds} * \text{query-error-bound}$ 
proof  $-$ 
  have  $\text{alpha-hit-bound} :$ 
     $\text{wp-event verify-monad}$ 
     $(\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates}$ 
       $s) \ s \leq \text{composition-error-bound}$ 
  proof  $-$ 
    have  $\text{wp-event verify-monad}$ 
       $(\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates}$ 
         $s) \ s \leq$ 
       $\text{wp-event verify-monad}$ 
       $(\text{composition-alpha-partial-opening-union-hit-with-empty-header } s) \ s$ 
    by (rule
       $\text{wp-composition-alpha-bad-set-hit-with-empty-header-le-partial-opening-union-hit}$ )
    also have  $\dots \leq \text{composition-error-bound}$ 
    by (rule  $\text{alpha-bound}$ )
    finally show  $?thesis$  .
  qed
show  $?thesis$ 
  by (rule  $\text{soundness-bad-event-partial-candidate-empty-header-bound-from-alpha-hit}$ 
     $[\text{OF false-statement alpha-hit-bound trace-fri-bound query-bound}]$ )
qed

```

```

lemma  $\text{soundness-bad-event-partial-candidate-empty-header-bound-from-alpha-header-partial-union-hit} :$ 
fixes  $s :: ('f, 'a) \text{ protocol-channel-scheme}$ 
assumes  $\text{false-statement} : \neg \text{exists-valid-trace}$ 
and  $\text{alpha-bound} :$ 
   $\text{wp-event verify-monad}$ 
   $(\text{alpha-header-list-set-hit } s$ 
     $(\text{alpha-header-supported-partial-union-bad-sets } s$ 
       $\text{composition-trace-bad-alpha-space})) \ s \leq$ 

```

```

    composition-error-bound
  and trace-fri-bound:
    wp-event verify-monad
      (trace-fri-bad-with-empty-composition-header-candidates s) s ≤
      trace-fri-error
  and query-bound:
    wp-event verify-monad
      (query-bad-with-empty-composition-header-candidates s) s ≤
      nnreal rounds * query-error-bound
  shows
    wp-event verify-monad
      (soundness-bad-event-partial-candidate-empty-header s) s ≤
      composition-error-bound + trace-fri-error +
      nnreal rounds * query-error-bound
  proof -
    have partial-alpha-bound:
      wp-event verify-monad
        (composition-alpha-partial-opening-union-hit-with-empty-header s) s ≤
        composition-error-bound
    proof -
      have wp-event verify-monad
        (composition-alpha-partial-opening-union-hit-with-empty-header s) s
          ≤
          wp-event verify-monad
            (alpha-header-list-set-hit s
              (alpha-header-supported-partial-union-bad-sets s
                composition-trace-bad-alpha-space)) s
        by (rule
          wp-composition-alpha-partial-opening-union-hit-with-empty-header-le-alpha-header-partial-union)
      also have ... ≤ composition-error-bound
        by (rule alpha-bound)
      finally show ?thesis .
    qed
  show ?thesis
    by (rule
      soundness-bad-event-partial-candidate-empty-header-bound-from-partial-opening-alpha-union-hit
      [OF false-statement partial-alpha-bound trace-fri-bound query-bound])
  qed

lemma checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-imp-partial-header-bad-set-h
  assumes support:
    out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-verifier-event

```

```

    composition-randomization-bad-with-partial-candidates out
  shows
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
    composition-trace-bad-alpha-space out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    by simp
next
  case (Some result-pack)
  then obtain prefix prefix-state data attacker-state result final-state
    where out-eq:
      out =
        Some (((((prefix, prefix-state), data), attacker-state),
          result), final-state)
    by (cases result-pack, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
    using support
    unfolding out-eq
      checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
    by (auto elim!: set-dist-bindE split: prod.splits)
  have local-random:
    composition-randomization-bad-with-partial-candidates ?s
    (Some (result, final-state))
    using hit
    unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      Let-def
    by simp
  have local-hit:
    composition-alpha-bad-set-hit-with-partial-candidates ?s
    (Some (result, final-state))
    by (rule composition-randomization-bad-with-partial-candidates-imp-alpha-hit
      [OF local-random])
  have partial-header:
    alpha-header-list-set-hit ?s
    (alpha-header-supported-partial-union-bad-sets ?s
      composition-trace-bad-alpha-space)
    (Some (result, final-state))
    by (rule
      composition-alpha-bad-set-hit-with-partial-candidates-imp-alpha-header-partial-union-on-support
      [OF verifier local-hit])
  show ?thesis

```


unfolding *out-eq*
checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-def
Let-def
using *partial-header* **by** *simp*
qed

lemma *checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-le-partial-header-bad-set-hit:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-randomization-bad-with-partial-candidates)
adversary-initial-state ≤
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state
by *(rule wp-event-mono-on-support)*
(rule
checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-imp-partial-header-bad-set-hit-on

lemma *checked-staged-security-with-actual-alpha-prefix-partial-candidate-composition-randomization-bound-from-pre:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *prefix-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state ≤ P
and *drift-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space)
adversary-initial-state ≤ D
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-randomization-bad-with-partial-candidates)
adversary-initial-state ≤
P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)
proof –
have *partial-header-bound:*

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit*
 composition-trace-bad-alpha-space)
adversary-initial-state $\leq$
P + *D* +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)
by (rule
 checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-bound-from-prefix-drift-and-budgets
 [*OF wf controlled prefix-bound drift-bound*])
have *wp-event*
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
 composition-randomization-bad-with-partial-candidates)
adversary-initial-state $\leq$
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit*
 composition-trace-bad-alpha-space)
adversary-initial-state
by (rule
 checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-le-partial-header-bad-set-hit)
also have ... $\leq$
P + *D* +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)
by (rule *partial-header-bound*)
finally show ?thesis .
qed

lemma *checked-staged-security-with-data-state-partial-candidate-composition-randomization-bound-from-prefix-drift-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *prefix-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq P$
and *drift-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq D$
shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-randomization-bad-with-partial-candidates)
adversary-initial-state $\leq$
 $P + D +$
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)

proof –

have *projected*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-randomization-bad-with-partial-candidates)
adversary-initial-state =
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-randomization-bad-with-partial-candidates)
adversary-initial-state

by (rule
checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)

show *?thesis*

unfolding *projected*

by (rule
checked-staged-security-with-actual-alpha-prefix-partial-candidate-composition-randomization-bound-from-prefix-c
[OF wf controlled prefix-bound drift-bound])

qed

lemma *checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-prefix-drift-and-budgets*

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *prefix-bound:*

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state $\leq P$

and *drift-bound:*

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space)
adversary-initial-state $\leq D$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-bad-with-partial-candidates)

```

    adversary-initial-state ≤
    P + D +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  proof -
    let ?M = checked-staged-security-experiment-with-data-state A
    let ?E = staged-security-with-data-state-verifier-event
    let ?degree = ?E composition-degree-bad-with-partial-candidates
    let ?random = ?E composition-randomization-bad-with-partial-candidates
    let ?bad = ?E composition-bad-with-partial-candidates
    have degree-false: ?degree = (λout. False)
    by (rule ext)
      (auto simp: staged-security-with-data-state-verifier-event-def
        composition-degree-bad-with-partial-candidates-false
        split: option.splits prod.splits)
    have degree-bound:
      wp-event ?M ?degree adversary-initial-state ≤ 0
    unfolding degree-false wp-event-def wp-def dist-expect-def by simp
    have random-bound:
      wp-event ?M ?random adversary-initial-state ≤
      P + D +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0)
    by (rule
      checked-staged-security-with-data-state-partial-candidate-composition-randomization-bound-from-prefix-drift-and-
      [OF wf controlled prefix-bound drift-bound])
    have wp-event ?M ?bad adversary-initial-state ≤
      wp-event ?M (λout. ?degree out ∨ ?random out)
      adversary-initial-state
    by (rule wp-event-mono)
      (auto simp: staged-security-with-data-state-verifier-event-def
        dest: composition-bad-with-partial-candidates-split
        [OF false-statement]
        split: option.splits prod.splits)
    also have ... ≤
      wp-event ?M ?degree adversary-initial-state +
      wp-event ?M ?random adversary-initial-state
    by (rule wp-event-union-bound)
    also have ... ≤
      0 + (P + D +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0))
    by (intro add-mono degree-bound random-bound)
    finally show ?thesis
    by simp
  qed

```

lemma *checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-drift-and-budgets:*
assumes *false-statement:* $\neg \text{exists-valid-trace}$

and *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *drift-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq D$
shows
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 composition-bad-with-partial-candidates)
 adversary-initial-state $\leq$
 composition-error-bound +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 D +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)
proof –
 let *?P* =
 composition-error-bound +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*)
 have *transcript-bound*:
 wp-event (*checked-staged-transcript-with-alpha-prefix-program A*)
 (*checked-staged-transcript-actual-alpha-prefix-bad-set-hit*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq ?P$
 by (*rule*
 checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
 [*OF wf controlled*])
 have *prefix-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq ?P$
 by (*rule*
 checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
 [*OF transcript-bound*])
 have
 wp-event (*checked-staged-security-experiment-with-data-state A*)

```

    (staged-security-with-data-state-verifier-event
      composition-bad-with-partial-candidates)
    adversary-initial-state ≤
    ?P + D +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  by (rule
    checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-prefix-drift-and-budgets
    [OF false-statement wf controlled prefix-bound drift-bound])
  then show ?thesis
    by (simp add: add.assoc)
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-imp-partial-header-bad-set-hit
  assumes support:
    out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-alpha-partial-opening-union-hit-with-empty-header out
  shows
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
    composition-trace-bad-alpha-space out
proof (cases out)
case None
  then show ?thesis
    using hit
    unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    by simp
next
case (Some result-pack)
  then obtain prefix prefix-state data attacker-state result final-state
    where out-eq:
      out =
        Some (((((prefix, prefix-state), data), attacker-state),
          result), final-state)
    by (cases result-pack, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  using support
  unfolding out-eq
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def

```

```

    by (auto elim!: set-dist-bindE split: prod.splits)
  have local-hit:
    composition-alpha-partial-opening-union-hit-with-empty-header ?s
      (Some (result, final-state))
  using hit
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    Let-def
  by simp
  have partial-header:
    alpha-header-list-set-hit ?s
      (alpha-header-supported-partial-union-bad-sets ?s
        composition-trace-bad-alpha-space)
      (Some (result, final-state))
  by (rule
    composition-alpha-partial-opening-union-hit-with-empty-header-imp-alpha-header-partial-union-on-support
      [OF verifier local-hit])
  show ?thesis
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-def
    Let-def
  using partial-header by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-le-partial-header-bad-set-hit:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-alpha-partial-opening-union-hit-with-empty-header)
  adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
      composition-trace-bad-alpha-space)
  adversary-initial-state
  by (rule wp-event-mono-on-support)
  (rule
    checked-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-imp-partial-header-bad-set-hit-on-s)

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-bound-from-prefix-drift-and-bu
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and prefix-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
        composition-trace-bad-alpha-space)
    adversary-initial-state ≤ P

```

and *drift-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq D$
shows
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
 composition-alpha-partial-opening-union-hit-with-empty-header)
 adversary-initial-state $\leq$
 P + *D* +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)
proof –
 have *partial-header-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq$
 P + *D* +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)
 by (rule
 checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-bound-from-prefix-drift-and-budgets
 [*OF wf controlled prefix-bound drift-bound*])
 have *wp-event*
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
 composition-alpha-partial-opening-union-hit-with-empty-header)
 adversary-initial-state $\leq$
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit*
 composition-trace-bad-alpha-space)
 adversary-initial-state
 by (rule
 checked-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-le-partial-header-bad-set-hit)
 also have ... $\leq$
 P + *D* +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)
 by (rule *partial-header-bound*)
 finally show *?thesis* .
qed

lemma *checked-staged-security-with-data-state-empty-header-partial-alpha-hit-bound-from-prefix-drift-and-budgets:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *prefix-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq P$
and *drift-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq D$
shows
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 composition-alpha-partial-opening-union-hit-with-empty-header)
 adversary-initial-state $\leq$
 P + D +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)
proof –
 have *projected:*
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 composition-alpha-partial-opening-union-hit-with-empty-header)
 adversary-initial-state =
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
 composition-alpha-partial-opening-union-hit-with-empty-header)
 adversary-initial-state
 by (rule
 checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
 show *?thesis*
 unfolding *projected*
 by (rule
 checked-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-bound-from-prefix-drift-and-budgets
 [*OF wf controlled prefix-bound drift-bound*])
qed

lemma *checked-staged-security-with-data-state-empty-header-composition-randomization-bound-from-prefix-drift-and-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *prefix-bound:*

$wp\text{-}event$
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq P$

and drift-bound:

$wp\text{-}event$
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq D$

shows

$wp\text{-}event$ (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-randomization-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$
 $P + D +$
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)

proof –

have alpha-bound:

$wp\text{-}event$ (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-alpha-partial-opening-union-hit-with-empty-header)
 adversary-initial-state $\leq$
 $P + D +$
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)

by (rule
 checked-staged-security-with-data-state-empty-header-partial-alpha-hit-bound-from-prefix-drift-and-budgets
 [OF wf controlled prefix-bound drift-bound])

have $wp\text{-}event$ (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-randomization-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$

$wp\text{-}event$ (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-alpha-partial-opening-union-hit-with-empty-header)
 adversary-initial-state

by (rule $wp\text{-}event\text{-}mono$)
 (auto simp: staged-security-with-data-state-verifier-event-def
 intro:
 composition-alpha-bad-set-hit-with-empty-header-imp-partial-opening-union-hit
 composition-randomization-bad-with-empty-composition-header-candidates-imp-alpha-hit
 split: option.splits prod.splits)

also have ... $\leq$
 $P + D +$

```

    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  by (rule alpha-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-empty-header-bad-event-bound-from-prefix-drift-trace-fri-and-query:
  assumes false-statement:  $\neg$  exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and prefix-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq$  P
  and drift-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
     composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq$  D
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state  $\leq$  trace-fri-error
  and query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     query-bad-with-empty-composition-header-candidates)
    adversary-initial-state  $\leq$  empty-query-error
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     soundness-bad-event-partial-candidate-empty-header)
    adversary-initial-state  $\leq$ 
    P + D +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    trace-fri-error + empty-query-error
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?degree =
    ?E composition-degree-bad-with-empty-composition-header-candidates
  let ?random =

```

```

    ?E composition-randomization-bad-with-empty-composition-header-candidates
  let ?trace =
    ?E trace-fri-bad-with-empty-composition-header-candidates
  let ?query =
    ?E query-bad-with-empty-composition-header-candidates
  let ?bad = ?E soundness-bad-event-partial-candidate-empty-header
  have degree-false: ?degree = ( $\lambda$ out. False)
  by (rule ext)
    (auto simp: staged-security-with-data-state-verifier-event-def
      composition-degree-bad-with-empty-composition-header-candidates-false
      split: option.splits prod.splits)
  have degree-bound:
    wp-event ?M ?degree adversary-initial-state  $\leq$  0
  unfolding degree-false wp-event-def wp-def dist-expect-def by simp
  have random-bound:
    wp-event ?M ?random adversary-initial-state  $\leq$ 
      P + D +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0)
  by (rule
    checked-staged-security-with-data-state-empty-header-composition-randomization-bound-from-prefix-drift-and-buo
    [OF wf controlled prefix-bound drift-bound])
  have wp-event ?M ?bad adversary-initial-state  $\leq$ 
    wp-event ?M
      ( $\lambda$ out. ?degree out  $\vee$  ?random out  $\vee$  ?trace out  $\vee$ 
        ?query out)
      adversary-initial-state
  proof (rule wp-event-mono)
    fix out
    assume bad: ?bad out
    show ?degree out  $\vee$  ?random out  $\vee$  ?trace out  $\vee$  ?query out
  proof (cases out)
    case None
    then show ?thesis
      using bad
      unfolding staged-security-with-data-state-verifier-event-def
      by simp
  next
    case (Some packed)
    then obtain data attacker-state result final-state where out-eq:
      out = Some (((data, attacker-state), result), final-state)
      by (cases packed, auto split: prod.splits)
    let ?s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    have local-bad:
      soundness-bad-event-partial-candidate-empty-header ?s
      (Some (result, final-state))
    using bad

```

```

unfolding out-eq staged-security-with-data-state-verifier-event-def
by simp
then consider
  (comp)
    composition-bad-with-empty-composition-header-candidates ?s
    (Some (result, final-state))
  | (trace)
    trace-fri-bad-with-empty-composition-header-candidates ?s
    (Some (result, final-state))
  | (query)
    query-bad-with-empty-composition-header-candidates ?s
    (Some (result, final-state))
unfolding soundness-bad-event-partial-candidate-empty-header-def
by blast
then show ?thesis
proof cases
case comp
then have comp-split:
  composition-degree-bad-with-empty-composition-header-candidates ?s
  (Some (result, final-state))  $\vee$ 
  composition-randomization-bad-with-empty-composition-header-candidates
  ?s (Some (result, final-state))
by (rule
  composition-bad-with-empty-composition-header-candidates-split
  [OF false-statement])
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  using comp-split by (auto split: option.splits prod.splits)
next
case trace
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
next
case query
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
qed
also have ...  $\leq$ 
  wp-event ?M ?degree adversary-initial-state +
  wp-event ?M ?random adversary-initial-state +
  wp-event ?M ?trace adversary-initial-state +
  wp-event ?M ?query adversary-initial-state
by (rule wp-event-union-bound4)
also have ...  $\leq$ 
  0 +

```

$(P + D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error} + \text{empty-query-error}$
by (*intro add-mono degree-bound random-bound trace-fri-bound query-bound*)
finally show *?thesis*
by (*simp add: algebra-simps*)
qed

lemma *checked-staged-security-with-data-state-empty-header-bad-event-bound-from-drift-trace-fri-and-query:*

assumes *false-statement: $\neg \text{exists-valid-trace}$*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *drift-bound:*
wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq D$
and *trace-fri-bound:*
wp-event $(\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *query-bound:*
wp-event $(\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows
wp-event $(\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event soundness-bad-event-partial-candidate-empty-header})$
 $\text{adversary-initial-state} \leq$
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound}$

proof –

let *?P* =
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$

hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0)
have *transcript-bound*:
wp-event (checked-staged-transcript-with-alpha-prefix-program A)
(checked-staged-transcript-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state ≤ ?P
by (rule
checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
[OF wf controlled])
have *prefix-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state ≤ ?P
by (rule
checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
[OF transcript-bound])
have
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state ≤
?P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nnreal rounds * query-error-bound*
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-bound-from-prefix-drift-trace-fri-and-query
[OF false-statement wf controlled prefix-bound drift-bound
trace-fri-bound query-bound])
then show *?thesis*
by (*simp add: add.assoc*)
qed

lemma *soundness-bad-event-table-aware-with-pairwise-side-union-bound*:
fixes *s :: ('f, 'a) protocol-channel-scheme*
and *pairwise-side-error :: prob*
assumes *side-bound*:
wp-event verify-monad (supported-pairwise-side-bad s) s ≤
pairwise-side-error
and *merkle-bound*:
wp-event verify-monad (fri-merkle-binding-bad s) s ≤
merkle-binding-error
and *comp-bound*:
wp-event verify-monad (composition-bad s) s ≤ composition-error-bound
and *trace-fri-bound*:

$wp\text{-event verify-monad (trace-fri-bad-with-tables } s) s \leq$
 trace-fri-error
and comp-fri-bound:
 $wp\text{-event verify-monad (composition-fri-bad-with-tables } s) s \leq$
 $\text{composition-fri-error}$
and query-bound:
 $wp\text{-event verify-monad (query-bad } s) s \leq nnreal\ rounds * \text{query-error-bound}$
shows
 $wp\text{-event verify-monad}$
 $(\text{soundness-bad-event-table-aware-with-pairwise-side } s) s \leq$
 $\text{pairwise-side-error} + \text{soundness-bound-with-merkle}$
proof –
have merkle-table-bound:
 $wp\text{-event verify-monad}$
 $(\text{soundness-bad-event-table-aware-with-merkle } s) s \leq$
 $\text{soundness-bound-with-merkle}$
by (rule soundness-bad-event-table-aware-with-merkle-union-bound
 $[OF\ \text{merkle-bound comp-bound trace-fri-bound comp-fri-bound}$
 $\text{query-bound}]$)
have $wp\text{-event verify-monad}$
 $(\text{soundness-bad-event-table-aware-with-pairwise-side } s) s \leq$
 $wp\text{-event verify-monad (supported-pairwise-side-bad } s) s +$
 $wp\text{-event verify-monad}$
 $(\text{soundness-bad-event-table-aware-with-merkle } s) s$
unfolding soundness-bad-event-table-aware-with-pairwise-side-def
by (rule wp-event-union-bound)
also have $\dots \leq \text{pairwise-side-error} + \text{soundness-bound-with-merkle}$
by (intro add-mono side-bound merkle-table-bound)
finally show ?thesis .
qed

lemma soundness-bad-event-table-aware-with-output-local-side-union-bound:
fixes $s :: ('f, 'a)\ \text{protocol-channel-scheme}$
and $\text{output-local-side-error} :: \text{prob}$
assumes side-bound:
 $wp\text{-event verify-monad (supported-output-local-side-bad } s) s \leq$
 $\text{output-local-side-error}$
and merkle-bound:
 $wp\text{-event verify-monad (fri-merkle-binding-bad } s) s \leq$
 $\text{merkle-binding-error}$
and comp-bound:
 $wp\text{-event verify-monad (composition-bad } s) s \leq \text{composition-error-bound}$
and trace-fri-bound:
 $wp\text{-event verify-monad (trace-fri-bad-with-tables } s) s \leq$
 trace-fri-error
and comp-fri-bound:
 $wp\text{-event verify-monad (composition-fri-bad-with-tables } s) s \leq$
 $\text{composition-fri-error}$
and query-bound:

$wp\text{-event verify-monad (query-bad } s) s \leq$
 $nnreal\ rounds * query\text{-error-bound}$
shows
 $wp\text{-event verify-monad}$
 $(soundness\text{-bad-event-table-aware-with-output-local-side } s) s \leq$
 $output\text{-local-side-error} + soundness\text{-bound-with-merkle}$
proof –
have *merkle-table-bound*:
 $wp\text{-event verify-monad}$
 $(soundness\text{-bad-event-table-aware-with-merkle } s) s \leq$
 $soundness\text{-bound-with-merkle}$
by (rule *soundness-bad-event-table-aware-with-merkle-union-bound*
 $[OF\ merkle\text{-bound comp-bound trace-fri-bound comp-fri-bound}$
 $query\text{-bound}]$)
have *wp-event verify-monad*
 $(soundness\text{-bad-event-table-aware-with-output-local-side } s) s \leq$
 $wp\text{-event verify-monad (supported-output-local-side-bad } s) s +$
 $wp\text{-event verify-monad}$
 $(soundness\text{-bad-event-table-aware-with-merkle } s) s$
unfolding *soundness-bad-event-table-aware-with-output-local-side-def*
by (rule *wp-event-union-bound*)
also have ... $\leq$
 $output\text{-local-side-error} + soundness\text{-bound-with-merkle}$
by (intro *add-mono side-bound merkle-table-bound*)
finally show *?thesis* .
qed

lemma *soundness-from-bad-event-partition*:
fixes $s :: ('f, 'a)\ protocol\text{-channel-scheme}$
assumes *false-statement*: $\neg\ exists\text{-valid-trace}$
and *partition*:
 $\bigwedge out :: (unit\ list \times ('f, 'a)\ protocol\text{-channel-scheme})\ option.$
 $out \in set\text{-dist (execute verify-monad } s) \implies$
 $accepted\ out \implies soundness\text{-bad-event } s\ out$
and *comp-bound*: $wp\text{-event verify-monad (composition-bad } s) s \leq composition\text{-error-bound}$
and *trace-fri-bound*: $wp\text{-event verify-monad (trace-fri-bad } s) s \leq trace\text{-fri-error}$
and *comp-fri-bound*:
 $wp\text{-event verify-monad (composition-fri-bad } s) s \leq composition\text{-fri-error}$
and *query-bound*:
 $wp\text{-event verify-monad (query-bad } s) s \leq nnreal\ rounds * query\text{-error-bound}$
shows $wp\text{-event verify-monad accepted } s \leq soundness\text{-bound}$
proof –
have *bad-event-bound*:
 $wp\text{-event verify-monad (soundness-bad-event } s) s \leq soundness\text{-bound}$
by (rule *soundness-bad-event-union-bound*
 $[OF\ comp-bound trace-fri-bound comp-fri-bound query-bound]$)
have *accepted-subset*:
 $\bigwedge out :: (unit\ list \times ('f, 'a)\ protocol\text{-channel-scheme})\ option.$
 $out \in set\text{-dist (execute verify-monad } s) \implies accepted\ out \implies$

soundness-bad-event s out
proof –
fix *out* :: (unit list × ('f, 'a) protocol-channel-scheme) option
assume *supp*: *out* ∈ *set-dist* (execute *verify-monad s*)
assume *accepted out*
then show *soundness-bad-event s out*
by (rule *partition[OF supp]*)
qed
have *wp-event verify-monad accepted s* ≤
wp-event verify-monad (soundness-bad-event s) s
by (rule *wp-event-mono-on-support*) (rule *accepted-subset*)
also have ... ≤ *soundness-bound*
by (rule *bad-event-bound*)
finally show ?thesis .
qed

lemma *soundness-from-table-aware-bad-event-partition*:
fixes *s* :: ('f, 'a) protocol-channel-scheme
assumes *false-statement*: ¬ *exists-valid-trace*
and *partition*:
 $\bigwedge out :: (unit\ list \times ('f, 'a)\ protocol-channel-scheme)\ option.$
 $out \in set-dist\ (execute\ verify-monad\ s) \implies$
 $accepted\ out \implies$
 $soundness-bad-event-table-aware-with-merkle\ s\ out$
and *merkle-bound*:
 $wp-event\ verify-monad\ (fri-merkle-binding-bad\ s)\ s \leq$
 $merkle-binding-error$
and *comp-bound*:
 $wp-event\ verify-monad\ (composition-bad\ s)\ s \leq$
 $composition-error-bound$
and *trace-fri-bound*:
 $wp-event\ verify-monad\ (trace-fri-bad-with-tables\ s)\ s \leq$
 $trace-fri-error$
and *comp-fri-bound*:
 $wp-event\ verify-monad\ (composition-fri-bad-with-tables\ s)\ s \leq$
 $composition-fri-error$
and *query-bound*:
 $wp-event\ verify-monad\ (query-bad\ s)\ s \leq$
 $nnreal\ rounds * query-error-bound$
shows *wp-event verify-monad accepted s* ≤ *soundness-bound-with-merkle*
proof –
have *bad-event-bound*:
 $wp-event\ verify-monad$
 $(soundness-bad-event-table-aware-with-merkle\ s)\ s \leq$
 $soundness-bound-with-merkle$
by (rule *soundness-bad-event-table-aware-with-merkle-union-bound*
[OF merkle-bound comp-bound trace-fri-bound comp-fri-bound
query-bound])
have *accepted-subset*:

$\bigwedge out :: (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option.}$
 $out \in \text{set-dist } (\text{execute verify-monad } s) \implies \text{accepted out} \implies$
 $\text{soundness-bad-event-table-aware-with-merkle } s \text{ out}$
proof –
fix $out :: (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option}$
assume $\text{supp: } out \in \text{set-dist } (\text{execute verify-monad } s)$
assume accepted out
then show $\text{soundness-bad-event-table-aware-with-merkle } s \text{ out}$
by $(\text{rule partition}[OF \text{ supp}])$
qed
have $\text{wp-event verify-monad accepted } s \leq$
 $\text{wp-event verify-monad}$
 $(\text{soundness-bad-event-table-aware-with-merkle } s) \text{ } s$
by $(\text{rule wp-event-mono-on-support}) (\text{rule accepted-subset})$
also have $\dots \leq \text{soundness-bound-with-merkle}$
by $(\text{rule bad-event-bound})$
finally show $?thesis .$
qed

lemma *soundness-from-partial-opening-bad-event-partition:*
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
and $B :: \text{prob}$
assumes *partition:*
 $\bigwedge out :: (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option.}$
 $out \in \text{set-dist } (\text{execute verify-monad } s) \implies$
 $\text{accepted out} \implies$
 $\text{soundness-bad-event-partial-opening-with-merkle } s \text{ out}$
and *bad-event-bound:*
 $\text{wp-event verify-monad}$
 $(\text{soundness-bad-event-partial-opening-with-merkle } s) \text{ } s \leq B$
shows $\text{wp-event verify-monad accepted } s \leq B$
proof –
have $\text{wp-event verify-monad accepted } s \leq$
 $\text{wp-event verify-monad}$
 $(\text{soundness-bad-event-partial-opening-with-merkle } s) \text{ } s$
by $(\text{rule wp-event-mono-on-support}) (\text{rule partition})$
also have $\dots \leq B$
by $(\text{rule bad-event-bound})$
finally show $?thesis .$
qed

lemma *soundness-from-table-aware-bad-event-partition-with-pairwise-side:*
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
and $\text{pairwise-side-error} :: \text{prob}$
assumes *false-statement:* $\neg \text{exists-valid-trace}$
and *partition:*
 $\bigwedge out :: (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option.}$
 $out \in \text{set-dist } (\text{execute verify-monad } s) \implies$
 $\text{accepted out} \implies$

soundness-bad-event-table-aware-with-pairwise-side s out
and *side-bound:*
wp-event verify-monad (supported-pairwise-side-bad s) s ≤
pairwise-side-error
and *merkle-bound:*
wp-event verify-monad (fri-merkle-binding-bad s) s ≤
merkle-binding-error
and *comp-bound:*
wp-event verify-monad (composition-bad s) s ≤
composition-error-bound
and *trace-fri-bound:*
wp-event verify-monad (trace-fri-bad-with-tables s) s ≤
trace-fri-error
and *comp-fri-bound:*
wp-event verify-monad (composition-fri-bad-with-tables s) s ≤
composition-fri-error
and *query-bound:*
wp-event verify-monad (query-bad s) s ≤
*nnreal rounds * query-error-bound*
shows
wp-event verify-monad accepted s ≤
pairwise-side-error + soundness-bound-with-merkle
proof –
have *bad-event-bound:*
wp-event verify-monad
(soundness-bad-event-table-aware-with-pairwise-side s) s ≤
pairwise-side-error + soundness-bound-with-merkle
by (rule *soundness-bad-event-table-aware-with-pairwise-side-union-bound*
[OF side-bound merkle-bound comp-bound trace-fri-bound comp-fri-bound
query-bound])
have *accepted-subset:*
 $\bigwedge out :: (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option.}$
 $out \in \text{set-dist (execute verify-monad s)} \implies \text{accepted out} \implies$
 $\text{soundness-bad-event-table-aware-with-pairwise-side s out}$
proof –
fix *out :: (unit list × ('f, 'a) protocol-channel-scheme) option*
assume *supp: out ∈ set-dist (execute verify-monad s)*
assume *accepted out*
then show *soundness-bad-event-table-aware-with-pairwise-side s out*
by (rule *partition[OF supp]*)
qed
have *wp-event verify-monad accepted s ≤*
wp-event verify-monad
(soundness-bad-event-table-aware-with-pairwise-side s) s
by (rule *wp-event-mono-on-support*) (rule *accepted-subset*)
also have $\dots \leq \text{pairwise-side-error} + \text{soundness-bound-with-merkle}$
by (rule *bad-event-bound*)
finally show *?thesis .*
qed

lemma *soundness-from-table-aware-bad-event-partition-with-output-local-side:*

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

and $\text{output-local-side-error} :: \text{prob}$

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *partition*:

$\bigwedge \text{out} :: (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option.}$

$\text{out} \in \text{set-dist } (\text{execute verify-monad } s) \implies$

$\text{accepted out} \implies$

$\text{soundness-bad-event-table-aware-with-output-local-side } s \text{ out}$

and *side-bound*:

$\text{wp-event verify-monad } (\text{supported-output-local-side-bad } s) \ s \leq$

$\text{output-local-side-error}$

and *merkle-bound*:

$\text{wp-event verify-monad } (\text{fri-merkle-binding-bad } s) \ s \leq$

$\text{merkle-binding-error}$

and *comp-bound*:

$\text{wp-event verify-monad } (\text{composition-bad } s) \ s \leq$

$\text{composition-error-bound}$

and *trace-fri-bound*:

$\text{wp-event verify-monad } (\text{trace-fri-bad-with-tables } s) \ s \leq$

trace-fri-error

and *comp-fri-bound*:

$\text{wp-event verify-monad } (\text{composition-fri-bad-with-tables } s) \ s \leq$

$\text{composition-fri-error}$

and *query-bound*:

$\text{wp-event verify-monad } (\text{query-bad } s) \ s \leq$

$\text{nnreal rounds} * \text{query-error-bound}$

shows

$\text{wp-event verify-monad accepted } s \leq$

$\text{output-local-side-error} + \text{soundness-bound-with-merkle}$

proof —

have *bad-event-bound*:

$\text{wp-event verify-monad}$

$(\text{soundness-bad-event-table-aware-with-output-local-side } s) \ s \leq$

$\text{output-local-side-error} + \text{soundness-bound-with-merkle}$

by (rule

$\text{soundness-bad-event-table-aware-with-output-local-side-union-bound}$

$[\text{OF side-bound merkle-bound comp-bound trace-fri-bound comp-fri-bound}$

$\text{query-bound}])$

have $\text{wp-event verify-monad accepted } s \leq$

$\text{wp-event verify-monad}$

$(\text{soundness-bad-event-table-aware-with-output-local-side } s) \ s$

by (rule $\text{wp-event-mono-on-support}$) (rule *partition*)

also have $\dots \leq$

$\text{output-local-side-error} + \text{soundness-bound-with-merkle}$

by (rule *bad-event-bound*)

finally show *?thesis* .

qed

lemma *accepted-execution-partition-obligation:*
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes *false-statement:* $\neg \text{exists-valid-trace}$
and *bind:* *initial-merkle-binding-no-bad* s
shows
 $\bigwedge \text{out} :: (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option.}$
 $\text{out} \in \text{set-dist} (\text{execute verify-monad } s) \implies$
 $\text{accepted out} \implies$
 $\text{soundness-bad-event } s \text{ out}$
proof –
fix $\text{out} :: (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option}$
assume *outcome:* $\text{out} \in \text{set-dist} (\text{execute verify-monad } s)$
and *acc:* *accepted out*
from *acc* **obtain** *result final-state* **where** *out-eq:* $\text{out} = \text{Some} (\text{result}, \text{final-state})$
unfolding *accepted-def* **by** *(cases out) auto*
have *verify-out:*
 $\text{Some} (\text{result}, \text{final-state}) \in \text{set-dist} (\text{execute verify-monad } s)$
using *outcome out-eq* **by** *simp*
from *verify-monad-merkle-bound-witnesses* *[OF bind verify-out]*
obtain *trace-table composition-table as query-idxs* **where** *bound:*
 $\text{accepted-with-bound-tables } s (\text{Some} (\text{result}, \text{final-state}))$
 $\text{trace-table composition-table as query-idxs}$
by *blast*
have *composition-bad* $s (\text{Some} (\text{result}, \text{final-state})) \vee$
 $\text{trace-fri-bad } s (\text{Some} (\text{result}, \text{final-state})) \vee$
 $\text{composition-fri-bad } s (\text{Some} (\text{result}, \text{final-state})) \vee$
 $\text{query-bad } s (\text{Some} (\text{result}, \text{final-state}))$
by *(rule accepted-bound-table-partition [OF false-statement bound])*
then show *soundness-bad-event* $s \text{ out}$
unfolding *soundness-bad-event-def* **using** *out-eq* **by** *simp*
qed

lemma *accepted-execution-partition-obligation-partial-opening:*
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes *false-statement:* $\neg \text{exists-valid-trace}$
shows
 $\bigwedge \text{out} :: (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option.}$
 $\text{out} \in \text{set-dist} (\text{execute verify-monad } s) \implies$
 $\text{accepted out} \implies$
 $\text{soundness-bad-event-partial-opening-with-merkle } s \text{ out}$
by *(rule accepted-partition-soundness-bad-event-partial-opening-with-merkle [OF false-statement])*

end

end

```

theory Soundness-Staged-Partial-Merkle
  imports Soundness-Staged-Bounds
begin

```

Aggregator for staged partial-Merkle soundness infrastructure. The old public-path compatibility wrappers based on partial-Merkle event-bound premises were removed before publication; downstream active-route theories import this file only for the underlying staged bounds.

```

end

```

```

theory Soundness-Partial-Candidate-Blocker
  imports Soundness-Partial-Merkle
begin

```

Diagnostic lemmas for the remaining query-bound route.

Partial authenticated openings constrain only the opened indices. Without a separate Merkle-binding/completion theorem, the corresponding partial table candidate sets are not singleton in general: unopened positions can be changed while preserving all partial-opening evidence.

```

context soundness
begin

```

```

lemma table-agrees-with-authenticated-openings-update-unopened:

```

```

  assumes agrees:

```

```

    table-agrees-with-authenticated-openings table len openings

```

```

  and unopened:

```

```

     $\bigwedge \text{opn. } \text{opn} \in \text{set openings} \implies \text{opening-index opn} \neq j$ 

```

```

  shows

```

```

    table-agrees-with-authenticated-openings (table[j := v]) len openings

```

```

proof –

```

```

  have len: len = length table

```

```

    using agrees unfolding table-agrees-with-authenticated-openings-def

```

```

    by simp

```

```

  have vals:

```

```

     $\bigwedge \text{opn. } \text{opn} \in \text{set openings} \implies$ 

```

```

      opening-index opn < length table  $\wedge$ 

```

```

      table ! opening-index opn = opening-value opn

```

```

    using agrees unfolding table-agrees-with-authenticated-openings-def

```

```

    by blast

```

```

  have updated-values:

```

```

     $\bigwedge \text{opn. } \text{opn} \in \text{set openings} \implies$ 

```

```

      opening-index opn < length (table[j := v])  $\wedge$ 

```

```

      (table[j := v]) ! opening-index opn = opening-value opn

```

```

proof –

```

```

  fix opn

```

```

  assume opn-in: opn  $\in$  set openings

```

```

  have idx-bound: opening-index opn < length table

```

```

    using vals[OF opn-in] by simp
  have idx-ne: opening-index opn  $\neq$  j
    by (rule unopened[OF opn-in])
  have (table[j := v]) ! opening-index opn =
    table ! opening-index opn
    using idx-bound idx-ne by simp
  then show
    opening-index opn < length (table[j := v])  $\wedge$ 
    (table[j := v]) ! opening-index opn = opening-value opn
    using vals[OF opn-in] by simp
qed
show ?thesis
  unfolding table-agrees-with-authenticated-openings-def
  using len updated-values by simp
qed

lemma table-agrees-with-authenticated-openings-eq-if-all-indices-opened:
  assumes agrees:
    table-agrees-with-authenticated-openings table len openings
  and agrees':
    table-agrees-with-authenticated-openings table' len openings
  and cover:
     $\bigwedge j. j < len \implies \exists \text{opn} \in \text{set openings}. \text{opening-index opn} = j$ 
  shows table = table'
proof (rule nth-equalityI)
  show length table = length table'
    using agrees agrees'
    unfolding table-agrees-with-authenticated-openings-def by simp
next
  fix j
  assume j-bound: j < length table
  have len-eq: length table = len
    using agrees unfolding table-agrees-with-authenticated-openings-def
    by simp
  then obtain opn where opn-in: opn  $\in$  set openings
    and opn-idx: opening-index opn = j
    using cover j-bound by blast
  have table-val: table ! j = opening-value opn
    using agrees opn-in opn-idx
    unfolding table-agrees-with-authenticated-openings-def by blast
  have table'-val: table' ! j = opening-value opn
    using agrees' opn-in opn-idx
    unfolding table-agrees-with-authenticated-openings-def by blast
  show table ! j = table' ! j
    using table-val table'-val by simp
qed

lemma partial-trace-table-candidate-update-unopened:

```



```

assumes cand: partial-trace-table-candidate trace-table trace-openings
and unopened:
   $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set } (\text{trace-openings} ! i) \implies$ 
    opening-index opn  $\neq j$ 
shows
  partial-trace-table-candidate (trace-table[j := v]) trace-openings
proof –
  have len: length (trace-table[j := v]) = scale * clength
    using cand unfolding partial-trace-table-candidate-def by simp
  have rounds-len: length trace-openings = rounds
    using cand unfolding partial-trace-table-candidate-def by simp
  have agrees:
     $\bigwedge i. i < \text{rounds} \implies$ 
      table-agrees-with-authenticated-openings (trace-table[j := v])
        (scale * clength) (trace-openings ! i)
  proof –
    fix i
    assume i-bound: i < rounds
    have old:
      table-agrees-with-authenticated-openings trace-table
        (scale * clength) (trace-openings ! i)
      using cand i-bound unfolding partial-trace-table-candidate-def by blast
    show
      table-agrees-with-authenticated-openings (trace-table[j := v])
        (scale * clength) (trace-openings ! i)
      by (rule table-agrees-with-authenticated-openings-update-unopened
        [OF old])
        (use unopened[OF i-bound] in blast)
    qed
  show ?thesis
    unfolding partial-trace-table-candidate-def
    using len rounds-len agrees by simp
  qed

lemma partial-trace-table-candidates-not-singleton-if-unopened:
assumes cand: partial-trace-table-candidate trace-table trace-openings
and j-bound: j < scale * clength
and unopened:
   $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set } (\text{trace-openings} ! i) \implies$ 
    opening-index opn  $\neq j$ 
shows
   $\neg (\exists \text{table.}$ 
    partial-trace-table-candidates trace-openings  $\subseteq \{\text{table}\})$ 
proof
assume unique:
   $\exists \text{table. } \text{partial-trace-table-candidates } \text{trace-openings} \subseteq \{\text{table}\}$ 
let ?trace-table' = trace-table[j := trace-table ! j + 1]
have cand-in:
  trace-table  $\in$  partial-trace-table-candidates trace-openings

```

```

    using cand unfolding partial-trace-table-candidates-def by simp
  have cand'-in:
    ?trace-table' ∈ partial-trace-table-candidates trace-openings
    using partial-trace-table-candidate-update-unopened[OF cand unopened]
    unfolding partial-trace-table-candidates-def by simp
  from unique obtain table where subset:
    partial-trace-table-candidates trace-openings ⊆ {table}
    by blast
  have trace-eq: trace-table = table
    using subset cand-in by blast
  have trace'-eq: ?trace-table' = table
    using subset cand'-in by blast
  have len: length trace-table = scale * clength
    using cand unfolding partial-trace-table-candidate-def by simp
  have changed: ?trace-table' ! j ≠ trace-table ! j
    using j-bound len by simp
  show False
    using changed trace-eq trace'-eq by simp
qed

lemma partial-trace-table-candidate-eq-if-all-indices-opened:
  assumes cand: partial-trace-table-candidate trace-table trace-openings
  and cand':
    partial-trace-table-candidate trace-table' trace-openings
  and cover:
     $\bigwedge j. j < \text{scale} * \text{clength} \implies$ 
     $\exists i < \text{rounds}. \exists \text{opn} \in \text{set} (\text{trace-openings} ! i).$ 
     $\text{opening-index } \text{opn} = j$ 
  shows trace-table = trace-table'
proof (rule nth-equalityI)
  show length trace-table = length trace-table'
    using cand cand' unfolding partial-trace-table-candidate-def by simp
next
  fix j
  assume j-bound: j < length trace-table
  have len: length trace-table = scale * clength
    using cand unfolding partial-trace-table-candidate-def by simp
  from cover[OF j-bound[unfolded len]]
  obtain i opn where i-bound: i < rounds
    and opn-in: opn ∈ set (trace-openings ! i)
    and opn-idx: opening-index opn = j
    by blast
  have table-val: trace-table ! j = opening-value opn
    using partial-trace-table-candidateD(3)[OF cand i-bound opn-in]
    opn-idx by simp
  have table'-val: trace-table' ! j = opening-value opn
    using partial-trace-table-candidateD(3)[OF cand' i-bound opn-in]
    opn-idx by simp
  show trace-table ! j = trace-table' ! j

```

```

    using table-val table'-val by simp
qed

lemma partial-trace-table-candidates-subset-singleton-if-all-indices-opened:
  assumes cand: partial-trace-table-candidate trace-table trace-openings
  and cover:
     $\bigwedge j. j < \text{scale} * \text{clength} \implies$ 
     $\exists i < \text{rounds}. \exists \text{opn} \in \text{set } (\text{trace-openings} \ ! \ i).$ 
     $\text{opening-index } \text{opn} = j$ 
  shows partial-trace-table-candidates trace-openings  $\subseteq \{\text{trace-table}\}$ 
proof
  fix trace-table'
  assume trace-table'  $\in$  partial-trace-table-candidates trace-openings
  then have cand':
    partial-trace-table-candidate trace-table' trace-openings
  unfolding partial-trace-table-candidates-def by simp
  have trace-table' = trace-table
  using partial-trace-table-candidate-eq-if-all-indices-opened
    [OF cand' cand cover] .
  then show trace-table'  $\in \{\text{trace-table}\}$ 
  by simp
qed

lemma partial-composition-table-candidate-update-unopened:
  assumes cand:
    partial-composition-table-candidate composition-table
    composition-openings
  and unopened:
     $\bigwedge i \text{ opn}. i < \text{rounds} \implies$ 
     $\text{opn} \in \text{set } (\text{composition-openings} \ ! \ i) \implies$ 
     $\text{opening-index } \text{opn} \neq j$ 
  shows
    partial-composition-table-candidate
    (composition-table[j := v]) composition-openings
proof -
  have len: length (composition-table[j := v]) = scale * clength
  using cand unfolding partial-composition-table-candidate-def by simp
  have rounds-len: length composition-openings = rounds
  using cand unfolding partial-composition-table-candidate-def by simp
  have agrees:
     $\bigwedge i. i < \text{rounds} \implies$ 
    table-agrees-with-authenticated-openings
    (composition-table[j := v]) (scale * clength)
    (composition-openings ! i)
  proof -
    fix i
    assume i-bound: i < rounds
    have old:
      table-agrees-with-authenticated-openings composition-table

```

```

    (scale * clength) (composition-openings ! i)
  using cand i-bound
  unfolding partial-composition-table-candidate-def by blast
show
  table-agrees-with-authenticated-openings
    (composition-table[j := v]) (scale * clength)
    (composition-openings ! i)
  by (rule table-agrees-with-authenticated-openings-update-unopened
      [OF old])
    (use unopened[OF i-bound] in blast)
qed
show ?thesis
  unfolding partial-composition-table-candidate-def
  using len rounds-len agrees by simp
qed

lemma partial-composition-table-candidates-not-singleton-if-unopened:
  assumes cand:
    partial-composition-table-candidate composition-table
    composition-openings
  and j-bound: j < scale * clength
  and unopened:
     $\bigwedge i \text{ opn. } i < \text{rounds} \implies$ 
     $\text{opn} \in \text{set } (\text{composition-openings ! } i) \implies$ 
    opening-index opn  $\neq$  j
  shows
     $\neg (\exists \text{table.}$ 
      partial-composition-table-candidates composition-openings  $\subseteq$ 
      {table})
proof
  assume unique:
     $\exists \text{table. } \text{partial-composition-table-candidates composition-openings}$ 
     $\subseteq \{\text{table}\}$ 
  let ?composition-table' =
    composition-table[j := composition-table ! j + 1]
  have cand-in:
    composition-table  $\in$ 
    partial-composition-table-candidates composition-openings
  using cand unfolding partial-composition-table-candidates-def by simp
  have cand'-in:
    ?composition-table'  $\in$ 
    partial-composition-table-candidates composition-openings
  using partial-composition-table-candidate-update-unopened[OF cand unopened]
  unfolding partial-composition-table-candidates-def by simp
  from unique obtain table where subset:
    partial-composition-table-candidates composition-openings  $\subseteq$ 
    {table}
  by blast
  have comp-eq: composition-table = table

```

```

    using subset cand-in by blast
  have comp'-eq: ?composition-table' = table
    using subset cand'-in by blast
  have len: length composition-table = scale * clength
    using cand unfolding partial-composition-table-candidate-def by simp
  have changed: ?composition-table' ! j  $\neq$  composition-table ! j
    using j-bound len by simp
  show False
    using changed comp-eq comp'-eq by simp
qed

```

lemma *partial-composition-table-candidate-eq-if-all-indices-opened:*

```

  assumes cand:
    partial-composition-table-candidate composition-table
    composition-openings
  and cand':
    partial-composition-table-candidate composition-table'
    composition-openings
  and cover:
     $\bigwedge j. j < \text{scale} * \text{clength} \implies$ 
     $\exists i < \text{rounds}. \exists \text{opn} \in \text{set } (\text{composition-openings} ! i).$ 
    opening-index opn = j
  shows composition-table = composition-table'
proof (rule nth-equalityI)
  show length composition-table = length composition-table'
    using cand cand'
    unfolding partial-composition-table-candidate-def by simp
next
  fix j
  assume j-bound: j < length composition-table
  have len: length composition-table = scale * clength
    using cand unfolding partial-composition-table-candidate-def by simp
  from cover[OF j-bound[unfolded len]]
  obtain i opn where i-bound: i < rounds
    and opn-in: opn  $\in$  set (composition-openings ! i)
    and opn-idx: opening-index opn = j
    by blast
  have table-val: composition-table ! j = opening-value opn
    using partial-composition-table-candidateD(3)[OF cand i-bound opn-in]
    opn-idx by simp
  have table'-val: composition-table' ! j = opening-value opn
    using partial-composition-table-candidateD(3)[OF cand' i-bound opn-in]
    opn-idx by simp
  show composition-table ! j = composition-table' ! j
    using table-val table'-val by simp
qed

```

lemma *partial-composition-table-candidates-subset-singleton-if-all-indices-opened:*

```

  assumes cand:

```

```

    partial-composition-table-candidate composition-table
    composition-openings
  and cover:
     $\bigwedge j. j < scale * clength \implies$ 
     $\exists i < rounds. \exists opn \in set (composition-openings ! i).$ 
    opening-index opn = j
  shows
    partial-composition-table-candidates composition-openings  $\subseteq$ 
    {composition-table}
proof
  fix composition-table'
  assume
    composition-table'  $\in$ 
    partial-composition-table-candidates composition-openings
  then have cand':
    partial-composition-table-candidate composition-table'
    composition-openings
  unfolding partial-composition-table-candidates-def by simp
  have composition-table' = composition-table
  using partial-composition-table-candidate-eq-if-all-indices-opened
  [OF cand' cand cover] .
  then show composition-table'  $\in$  {composition-table}
  by simp
qed

end

end

```

theory *Soundness-Query-Partial-Candidate-Blocker*

imports

Soundness-Query-Header-Cross-Candidate

Soundness-Partial-Candidate-Blocker

begin

Exact query-header version of the partial-candidate non-uniqueness diagnostic. These lemmas show why the old singleton-candidate route cannot be applied to the broad partial-opening candidate relation.

context *soundness*

begin

lemma *sparse-openings-obtain-unopened-index:*

assumes *rounds-len: length openings = rounds*

and *sparse: length (List.concat openings) < scale * clength*

obtains *j where*

*j < scale * clength*

$\bigwedge i opn. i < rounds \implies opn \in set (openings ! i) \implies$

opening-index opn $\neq$ j

```

proof –
  let ?N = scale * clength
  let ?I = opening-index ‘ set (List.concat openings)
  have finite-I: finite ?I
    by simp
  have card-I-le: card ?I ≤ length (List.concat openings)
  proof –
    have card ?I ≤ card (set (List.concat openings))
      by (rule card-image-le) simp
    also have ... ≤ length (List.concat openings)
      by (rule card-length)
    finally show ?thesis .
  qed
  have not-all: ¬ {0.. $?N$ } ⊆ ?I
  proof
    assume subset: {0.. $?N$ } ⊆ ?I
    have ?N = card {0.. $?N$ }
      by simp
    also have ... ≤ card ?I
      by (rule card-mono[OF finite-I subset])
    also have ... ≤ length (List.concat openings)
      by (rule card-I-le)
    finally show False
      using sparse by simp
  qed
  from not-all obtain j where j-in: j ∈ {0.. $?N$ } and j-notin: j ∉ ?I
    by blast
  then have j-bound: j < ?N
    by simp
  have unopened:
     $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set } (\text{openings} ! i) \implies$ 
    opening-index opn ≠ j
  proof –
    fix i opn
    assume i-bound: i < rounds
      and opn-in: opn ∈ set (openings ! i)
    have openings ! i ∈ set openings
      using i-bound rounds-len by simp
    then have opn ∈ set (List.concat openings)
      using opn-in by auto
    then show opening-index opn ≠ j
      using j-notin by auto
  qed
  show ?thesis
    by (rule that[OF j-bound unopened])
  qed

```

lemma accepted-with-partial-trace-openings-concat-length:
assumes partial:

```

    accepted-with-partial-trace-openings s out fr query-idxs trace-openings
  shows length (List.concat trace-openings) = rounds * powers
proof -
  have len: length trace-openings = rounds
    by (rule accepted-with-partial-trace-openings-shapes(2)[OF partial])
  have entry-len:
     $\bigwedge xs. xs \in \text{set trace-openings} \implies \text{length } xs = \text{powers}$ 
  proof -
    fix xs
    assume xs-in: xs  $\in$  set trace-openings
    then obtain i where i-bound: i < length trace-openings
      and xs-eq: xs = trace-openings ! i
      by (auto simp: in-set-conv-nth)
    have idxs:
      map opening-index (trace-openings ! i) =
        powers-scaled (query-idxs ! i)
      by (rule accepted-with-partial-trace-openings-shapes(3)[OF partial])
        (use i-bound len in simp)
    have len-xs: length (map opening-index xs) =
      length (powers-scaled (query-idxs ! i))
      using idxs xs-eq by simp
    show length xs = powers
      using len-xs unfolding powers-scaled-def by simp
  qed
  have map-len: map length trace-openings = replicate rounds powers
  proof (rule nth-equalityI)
    show length (map length trace-openings) =
      length (replicate rounds powers)
      using len by simp
  next
    fix i
    assume i-bound: i < length (map length trace-openings)
    have trace-openings ! i  $\in$  set trace-openings
      by (rule nth-mem) (use i-bound in simp)
    then show
      map length trace-openings ! i = replicate rounds powers ! i
      using entry-len[of trace-openings ! i] i-bound len by simp
  qed
  show ?thesis
  proof -
    have length (List.concat trace-openings) =
      sum-list (map length trace-openings)
      by (induction trace-openings) simp-all
    also have ... = sum-list (replicate rounds powers)
      using map-len by simp
    also have ... = rounds * powers
      by (induction rounds) simp-all
    finally show ?thesis .
  qed

```


qed

lemma *accepted-with-partial-composition-openings-concat-length:*

assumes *partial:*

accepted-with-partial-composition-openings s out composition-root

query-idxs composition-openings

shows $\text{length } (\text{List.concat } \text{composition-openings}) = \text{rounds} * 2$

proof –

have *len:* $\text{length } \text{composition-openings} = \text{rounds}$

by (rule *accepted-with-partial-composition-openings-shapes(2)*)
[OF *partial*])

have *entry-len:*

$\bigwedge xs. xs \in \text{set } \text{composition-openings} \implies \text{length } xs = 2$

proof –

fix *xs*

assume *xs-in:* $xs \in \text{set } \text{composition-openings}$

then obtain *i* **where** *i-bound:* $i < \text{length } \text{composition-openings}$

and *xs-eq:* $xs = \text{composition-openings} ! i$

by (auto simp: *in-set-conv-nth*)

have *idxs:*

$\text{map } \text{opening-index } (\text{composition-openings} ! i) =$

$[\text{query-idxs} ! i,$
 $\text{fri-sibling-index } (\text{scale} * \text{clength}) (\text{query-idxs} ! i)]$

by (rule *accepted-with-partial-composition-openings-shapes(3)*)
[OF *partial*])

(use *i-bound len* **in** *simp*)

have *len-xs:* $\text{length } (\text{map } \text{opening-index } xs) = \text{length}$

$[\text{query-idxs} ! i,$
 $\text{fri-sibling-index } (\text{scale} * \text{clength}) (\text{query-idxs} ! i)]$

using *idxs xs-eq* **by** *simp*

show $\text{length } xs = 2$

using *len-xs* **by** *simp*

qed

have *map-len:* $\text{map } \text{length } \text{composition-openings} = \text{replicate } \text{rounds } 2$

proof (rule *nth-equalityI*)

show $\text{length } (\text{map } \text{length } \text{composition-openings}) =$

$\text{length } (\text{replicate } \text{rounds } 2)$

using *len* **by** *simp*

next

fix *i*

assume *i-bound:* $i < \text{length } (\text{map } \text{length } \text{composition-openings})$

have $\text{composition-openings} ! i \in \text{set } \text{composition-openings}$

by (rule *nth-mem*) (use *i-bound* **in** *simp*)

then show

$\text{map } \text{length } \text{composition-openings} ! i = \text{replicate } \text{rounds } 2 ! i$

using *entry-len[of composition-openings ! i]* *i-bound len* **by** *simp*

qed

show *?thesis*

proof –

```

have length (List.concat composition-openings) =
  sum-list (map length composition-openings)
  by (induction composition-openings) simp-all
also have ... = sum-list (replicate rounds 2)
  using map-len by simp
also have ... = rounds * 2
  by (induction rounds) simp-all
finally show ?thesis .
qed
qed

lemma query-header-supported-partial-table-candidates-update-trace-unopened:
assumes comp-nonempty: composition-fri-roots  $\neq$  []
and outcome:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
and trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr trace-query-idxs trace-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) (hd composition-fri-roots)
  composition-query-idxs composition-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and unopened:
   $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set } (\text{trace-openings } ! i) \implies$ 
  opening-index opn  $\neq$  j
shows
  (trace-table[j := v], composition-table)  $\in$ 
  query-header-supported-partial-table-candidates s fr f-fri-roots
  f-final as dg composition-fri-roots final
proof –
have trace-candidate':
  partial-trace-table-candidate (trace-table[j := v]) trace-openings
by (rule partial-trace-table-candidate-update-unopened
  [OF trace-candidate unopened])
show ?thesis
unfolding query-header-supported-partial-table-candidates-def
using comp-nonempty outcome trace-partial trace-candidate'
  comp-partial comp-candidate header
by blast
qed

```

lemma *query-header-supported-partial-table-candidates-not-singleton-if-trace-unopened:*

assumes *comp-nonempty: composition-fri-roots* $\neq \square$
and *outcome:*
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *trace-partial:*
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *comp-partial:*
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
and *comp-candidate:*
partial-composition-table-candidate composition-table
composition-openings
and *header:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *j-bound: j < scale * clength*
and *unopened:*
 $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set } (\text{trace-openings } ! i) \implies$
opening-index opn $\neq j$

shows

$\neg (\exists \text{ trace-table0 } \text{composition-table0.}$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$
 $\{(\text{trace-table0}, \text{composition-table0})\})$

proof

assume *unique:*
 $\exists \text{ trace-table0 } \text{composition-table0.}$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$
 $\{(\text{trace-table0}, \text{composition-table0})\}$
let *?trace-table' = trace-table[j := trace-table ! j + 1]*
have *candidate:*
 $(\text{trace-table}, \text{composition-table}) \in$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final
unfolding *query-header-supported-partial-table-candidates-def*
using *comp-nonempty outcome trace-partial trace-candidate*
comp-partial comp-candidate header
by *blast*
have *candidate':*
 $(\text{?trace-table'}, \text{composition-table}) \in$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final
by *(rule*
query-header-supported-partial-table-candidates-update-trace-unopened

```

    [OF comp-nonempty outcome trace-partial trace-candidate comp-partial
      comp-candidate header unopened])
from unique obtain trace-table0 composition-table0 where subset:
  query-header-supported-partial-table-candidates s fr f-fri-roots
  f-final as dg composition-fri-roots final  $\subseteq$ 
  {(trace-table0, composition-table0)}
by blast
have pair-eq:
  (trace-table, composition-table) = (trace-table0, composition-table0)
using subset candidate by blast
have pair'-eq:
  (?trace-table', composition-table) = (trace-table0, composition-table0)
using subset candidate' by blast
have len: length trace-table = scale * clength
using trace-candidate unfolding partial-trace-table-candidate-def by simp
have changed: ?trace-table' ! j  $\neq$  trace-table ! j
using j-bound len by simp
show False
using pair-eq pair'-eq changed by simp
qed

lemma query-header-supported-partial-table-candidates-not-singleton-if-trace-sparse:
assumes comp-nonempty: composition-fri-roots  $\neq$  []
and outcome:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
and trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr trace-query-idxs trace-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) (hd composition-fri-roots)
  composition-query-idxs composition-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and sparse: length (List.concat trace-openings) < scale * clength
shows
   $\neg (\exists \text{ trace-table0 composition-table0.}$ 
    query-header-supported-partial-table-candidates s fr f-fri-roots
    f-final as dg composition-fri-roots final  $\subseteq$ 
    {(trace-table0, composition-table0)})
proof –
have rounds-len: length trace-openings = rounds
using trace-candidate unfolding partial-trace-table-candidate-def by simp

```

```

obtain  $j$  where  $j$ -bound:  $j < \text{scale} * \text{clength}$ 
and  $\text{unopened}$ :
   $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set } (\text{trace-openings} ! i) \implies$ 
     $\text{opening-index } \text{opn} \neq j$ 
using  $\text{sparse-openings-obtain-unopened-index}[\text{OF } \text{rounds-len } \text{sparse}]$ 
by  $\text{blast}$ 
show  $?thesis$ 
by (rule
   $\text{query-header-supported-partial-table-candidates-not-singleton-if-trace-unopened}$ 
   $[\text{OF } \text{comp-nonempty } \text{outcome } \text{trace-partial } \text{trace-candidate } \text{comp-partial}$ 
     $\text{comp-candidate } \text{header } j\text{-bound } \text{unopened}])$ 
qed

lemma  $\text{query-header-supported-partial-pairwise-cross-or-merkle-bad-if-trace-sparse}$ :
assumes  $\text{comp-nonempty}$ :  $\text{composition-fri-roots} \neq []$ 
and  $\text{outcome}$ :
   $\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute } \text{verify-monad } s)$ 
and  $\text{trace-partial}$ :
   $\text{accepted-with-partial-trace-openings } s$ 
   $(\text{Some } (\text{result}, \text{final-state})) \text{ fr } \text{trace-query-idxs } \text{trace-openings}$ 
and  $\text{trace-candidate}$ :
   $\text{partial-trace-table-candidate } \text{trace-table } \text{trace-openings}$ 
and  $\text{comp-partial}$ :
   $\text{accepted-with-partial-composition-openings } s$ 
   $(\text{Some } (\text{result}, \text{final-state})) (\text{hd } \text{composition-fri-roots})$ 
   $\text{composition-query-idxs } \text{composition-openings}$ 
and  $\text{comp-candidate}$ :
   $\text{partial-composition-table-candidate } \text{composition-table}$ 
   $\text{composition-openings}$ 
and  $\text{header}$ :
   $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final as } dg$ 
   $\text{composition-fri-roots } \text{final } \text{rest}$ 
and  $\text{sparse}$ :  $\text{length } (\text{List.concat } \text{trace-openings}) < \text{scale} * \text{clength}$ 
shows
   $\text{query-header-supported-partial-pairwise-cross-or-merkle-bad } s \text{ fr}$ 
   $f\text{-fri-roots } f\text{-final as } dg \text{ composition-fri-roots } \text{final}$ 
proof –
have  $\text{not-unique}$ :
   $\neg (\exists \text{ trace-table0 } \text{composition-table0.}$ 
     $\text{query-header-supported-partial-table-candidates } s \text{ fr } f\text{-fri-roots}$ 
     $f\text{-final as } dg \text{ composition-fri-roots } \text{final} \subseteq$ 
     $\{(\text{trace-table0}, \text{composition-table0})\})$ 
by (rule
     $\text{query-header-supported-partial-table-candidates-not-singleton-if-trace-sparse}$ 
     $[\text{OF } \text{comp-nonempty } \text{outcome } \text{trace-partial } \text{trace-candidate } \text{comp-partial}$ 
       $\text{comp-candidate } \text{header } \text{sparse}])$ 
show  $?thesis$ 
by (rule
     $\text{query-header-supported-partial-pairwise-cross-or-merkle-bad-if-not-singleton}$ 

```

[OF not-unique])

qed

lemma *query-header-supported-partial-table-candidates-not-singleton-if-trace-query-sparse:*
assumes *comp-nonempty: composition-fri-roots $\neq \square$*
and *outcome:*
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *trace-partial:*
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *comp-partial:*
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
and *comp-candidate:*
partial-composition-table-candidate composition-table
composition-openings
and *header:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *sparse: rounds * powers < scale * clength*
shows
 $\neg (\exists \text{ trace-table0 } \text{composition-table0}.$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$
{(trace-table0, composition-table0)}))
proof –
have *concat-sparse:*
*length (List.concat trace-openings) < scale * clength*
using *accepted-with-partial-trace-openings-concat-length[OF trace-partial]*
sparse by simp
show *?thesis*
by (rule
query-header-supported-partial-table-candidates-not-singleton-if-trace-sparse
[OF comp-nonempty outcome trace-partial trace-candidate comp-partial
comp-candidate header concat-sparse])
qed

lemma *query-header-supported-partial-pairwise-cross-or-merkle-bad-if-trace-query-sparse:*
assumes *comp-nonempty: composition-fri-roots $\neq \square$*
and *outcome:*
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *trace-partial:*
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings

and *comp-partial*:
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
and *comp-candidate*:
partial-composition-table-candidate composition-table
composition-openings
and *header*:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *sparse: rounds * powers < scale * clength*
shows
query-header-supported-partial-pairwise-cross-or-merkle-bad s fr
f-fri-roots f-final as dg composition-fri-roots final
proof –
have *not-unique*:
 $\neg (\exists \text{ trace-table0 } \text{composition-table0}.$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final \subseteq
 $\{(\text{trace-table0}, \text{composition-table0})\})$
by (rule
query-header-supported-partial-table-candidates-not-singleton-if-trace-query-sparse
 $[OF \text{ comp-nonempty } \text{outcome } \text{trace-partial } \text{trace-candidate } \text{comp-partial}$
 $\text{comp-candidate } \text{header } \text{sparse}]$)
show *?thesis*
by (rule
query-header-supported-partial-pairwise-cross-or-merkle-bad-if-not-singleton
 $[OF \text{ not-unique}]$)
qed

lemma *query-header-supported-partial-table-candidates-update-composition-unopened:*

assumes *comp-nonempty: composition-fri-roots $\neq []$*
and *outcome*:
 $\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute } \text{verify-monad } s)$
and *trace-partial*:
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
and *comp-partial*:
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
and *comp-candidate*:
partial-composition-table-candidate composition-table
composition-openings
and *header*:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest

and *unopened*:
 $\bigwedge i \text{ opn. } i < \text{rounds} \implies$
 $\text{opn} \in \text{set } (\text{composition-openings} \text{ ! } i) \implies$
 $\text{opening-index } \text{opn} \neq j$
shows
 $(\text{trace-table}, \text{composition-table}[j := v]) \in$
 $\text{query-header-supported-partial-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final as dg composition-fri-roots final}$
proof –
have *comp-candidate'*:
 $\text{partial-composition-table-candidate } (\text{composition-table}[j := v])$
 $\text{composition-openings}$
by (rule *partial-composition-table-candidate-update-unopened*
 $[\text{OF comp-candidate unopened}]$)
show *?thesis*
unfolding *query-header-supported-partial-table-candidates-def*
using *comp-nonempty outcome trace-partial trace-candidate*
 $\text{comp-partial comp-candidate' header}$
by *blast*
qed

lemma *query-header-supported-partial-table-candidates-not-singleton-if-composition-unopened*:
assumes *comp-nonempty*: $\text{composition-fri-roots} \neq []$
and *outcome*:
 $\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute verify-monad } s)$
and *trace-partial*:
 $\text{accepted-with-partial-trace-openings } s$
 $(\text{Some } (\text{result}, \text{final-state})) \text{ fr trace-query-idxs trace-openings}$
and *trace-candidate*:
 $\text{partial-trace-table-candidate trace-table trace-openings}$
and *comp-partial*:
 $\text{accepted-with-partial-composition-openings } s$
 $(\text{Some } (\text{result}, \text{final-state})) (\text{hd composition-fri-roots})$
 $\text{composition-query-idxs composition-openings}$
and *comp-candidate*:
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings}$
and *header*:
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final as dg}$
 $\text{composition-fri-roots final rest}$
and *j-bound*: $j < \text{scale} * \text{clength}$
and *unopened*:
 $\bigwedge i \text{ opn. } i < \text{rounds} \implies$
 $\text{opn} \in \text{set } (\text{composition-openings} \text{ ! } i) \implies$
 $\text{opening-index } \text{opn} \neq j$
shows
 $\neg (\exists \text{ trace-table0 composition-table0.}$
 $\text{query-header-supported-partial-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final as dg composition-fri-roots final} \subseteq$


```

      {(trace-table0, composition-table0)})
proof
  assume unique:
     $\exists$  trace-table0 composition-table0.
      query-header-supported-partial-table-candidates s fr f-fri-roots
      f-final as dg composition-fri-roots final  $\subseteq$ 
      {(trace-table0, composition-table0)}
  let ?composition-table' =
    composition-table[j := composition-table ! j + 1]
  have candidate:
    (trace-table, composition-table)  $\in$ 
      query-header-supported-partial-table-candidates s fr f-fri-roots
      f-final as dg composition-fri-roots final
  unfolding query-header-supported-partial-table-candidates-def
  using comp-nonempty outcome trace-partial trace-candidate
    comp-partial comp-candidate header
  by blast
  have candidate':
    (trace-table, ?composition-table')  $\in$ 
      query-header-supported-partial-table-candidates s fr f-fri-roots
      f-final as dg composition-fri-roots final
  by (rule
    query-header-supported-partial-table-candidates-update-composition-unopened
    [OF comp-nonempty outcome trace-partial trace-candidate comp-partial
      comp-candidate header unopened])
  from unique obtain trace-table0 composition-table0 where subset:
    query-header-supported-partial-table-candidates s fr f-fri-roots
    f-final as dg composition-fri-roots final  $\subseteq$ 
    {(trace-table0, composition-table0)}
  by blast
  have pair-eq:
    (trace-table, composition-table) = (trace-table0, composition-table0)
  using subset candidate by blast
  have pair'-eq:
    (trace-table, ?composition-table') =
      (trace-table0, composition-table0)
  using subset candidate' by blast
  have len: length composition-table = scale * clength
  using comp-candidate
  unfolding partial-composition-table-candidate-def by simp
  have changed: ?composition-table' ! j  $\neq$  composition-table ! j
  using j-bound len by simp
  show False
  using pair-eq pair'-eq changed by simp
qed

lemma query-header-supported-partial-table-candidates-not-singleton-if-composition-sparse:
  assumes comp-nonempty: composition-fri-roots  $\neq$  []
  and outcome:

```

Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *trace-partial:*
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *comp-partial:*
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
and *comp-candidate:*
partial-composition-table-candidate composition-table
composition-openings
and *header:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *sparse:*
*length (List.concat composition-openings) < scale * clength*
shows
 $\neg (\exists \text{ trace-table0 } \text{ composition-table0}.$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final ⊆
{(trace-table0, composition-table0)})
proof –
have *rounds-len: length composition-openings = rounds*
using *comp-candidate*
unfolding *partial-composition-table-candidate-def* **by** *simp*
obtain *j* **where** *j-bound: j < scale * clength*
and *unopened:*
 $\bigwedge i \text{ opn. } i < \text{rounds} \implies$
 $\text{opn} \in \text{set } (\text{composition-openings } ! i) \implies$
 $\text{opening-index opn} \neq j$
using *sparse-openings-obtain-unopened-index* [*OF rounds-len sparse*]
by *blast*
show *?thesis*
by (*rule*
query-header-supported-partial-table-candidates-not-singleton-if-composition-unopened
[OF comp-nonempty outcome trace-partial trace-candidate comp-partial
comp-candidate header j-bound unopened])
qed

lemma *query-header-supported-partial-pairwise-cross-or-merkle-bad-if-composition-sparse:*
assumes *comp-nonempty: composition-fri-roots ≠ []*
and *outcome:*
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *trace-partial:*
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings
and *comp-partial:*
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
and *comp-candidate:*
partial-composition-table-candidate composition-table
composition-openings
and *header:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *sparse:*
*length (List.concat composition-openings) < scale * clength*
shows
query-header-supported-partial-pairwise-cross-or-merkle-bad s fr
f-fri-roots f-final as dg composition-fri-roots final
proof –
have *not-unique:*
 $\neg (\exists \text{ trace-table0 } \text{composition-table0}.$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$
 $\{(\text{trace-table0}, \text{composition-table0})\})$
by (rule
query-header-supported-partial-table-candidates-not-singleton-if-composition-sparse
 $[OF \text{ comp-nonempty } \text{outcome } \text{trace-partial } \text{trace-candidate } \text{comp-partial}$
 $\text{comp-candidate } \text{header } \text{sparse}]$)
show *?thesis*
by (rule
query-header-supported-partial-pairwise-cross-or-merkle-bad-if-not-singleton
 $[OF \text{ not-unique}]$)
qed

lemma *query-header-supported-partial-table-candidates-not-singleton-if-composition-query-sparse:*

assumes *comp-nonempty: composition-fri-roots $\neq []$*
and *outcome:*
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *trace-partial:*
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *comp-partial:*
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
and *comp-candidate:*
partial-composition-table-candidate composition-table
composition-openings
and *header:*

verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *sparse: rounds * 2 < scale * clength*
shows
 $\neg (\exists \text{ trace-table0 } \text{composition-table0}.$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$
{(trace-table0, composition-table0)}))
proof –
have *concat-sparse:*
*length (List.concat composition-openings) < scale * clength*
using *accepted-with-partial-composition-openings-concat-length*
[OF comp-partial] sparse by simp
show *?thesis*
by (rule
query-header-supported-partial-table-candidates-not-singleton-if-composition-sparse
[OF comp-nonempty outcome trace-partial trace-candidate comp-partial
comp-candidate header concat-sparse])
qed

lemma *query-header-supported-partial-pairwise-cross-or-merkle-bad-if-composition-query-sparse:*
assumes *comp-nonempty: composition-fri-roots $\neq []$*
and *outcome:*
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *trace-partial:*
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *comp-partial:*
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
and *comp-candidate:*
partial-composition-table-candidate composition-table
composition-openings
and *header:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *sparse: rounds * 2 < scale * clength*
shows
query-header-supported-partial-pairwise-cross-or-merkle-bad s fr
f-fri-roots f-final as dg composition-fri-roots final
proof –
have *not-unique:*
 $\neg (\exists \text{ trace-table0 } \text{composition-table0}.$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$
{(trace-table0, composition-table0)}))

```

    by (rule
      query-header-supported-partial-table-candidates-not-singleton-if-composition-query-sparse
      [OF comp-nonempty outcome trace-partial trace-candidate comp-partial
        comp-candidate header sparse])
  show ?thesis
    by (rule
      query-header-supported-partial-pairwise-cross-or-merkle-bad-if-not-singleton
      [OF not-unique])
qed
end
end

```

```

theory Soundness-Alpha-Header-Candidate-Blocker
imports
  Soundness-Query-Partial-Candidate-Blocker
  Staged-Security-Experiment-Composition
begin

```

Diagnostic lemmas for the alpha-header partial-candidate route.

The staged composition proof uses *alpha-header-supported-partial-trace-table-candidates*. This candidate set is trace-only, but it has the same sparse-opening obstruction as the query-header partial-candidate set: sampled authenticated openings do not determine unopened trace-table positions.

```

context soundness
begin

```

```

definition alpha-header-partial-candidate-not-prefix-bound
  :: 'f protocol-channel  $\Rightarrow$  ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$  bool

```

```

where
  alpha-header-partial-candidate-not-prefix-bound prefix-state s fr
    f-fri-roots f-final  $\longleftrightarrow$ 
    ( $\exists$  trace-table  $\in$ 
      alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
        f-final.
      trace-table  $\notin$  alpha-prefix-trace-table-candidates prefix-state fr)

```

```

lemma alpha-header-partial-candidate-not-prefix-bound-iff-not-subset:
  alpha-header-partial-candidate-not-prefix-bound prefix-state s fr
    f-fri-roots f-final  $\longleftrightarrow$ 
     $\neg$  alpha-header-supported-partial-trace-table-candidates s fr
      f-fri-roots f-final  $\subseteq$ 
      alpha-prefix-trace-table-candidates prefix-state fr
  unfolding alpha-header-partial-candidate-not-prefix-bound-def by blast

```

```

lemma alpha-header-supported-partial-union-bad-sets-subset-prefix-union-if-bound:

```

assumes *bound*:
 $\neg$ *alpha-header-partial-candidate-not-prefix-bound prefix-state s fr f-fri-roots f-final*
shows
alpha-header-supported-partial-union-bad-sets s bad-sets fr f-fri-roots f-final $\subseteq$ *alpha-prefix-union-bad-sets prefix-state bad-sets fr*
using *bound*
unfolding *alpha-header-partial-candidate-not-prefix-bound-def*
alpha-header-supported-partial-union-bad-sets-def
alpha-prefix-union-bad-sets-def
by *blast*

lemma *alpha-header-partial-union-hit-not-prefix-hit-imp-not-prefix-bound*:
assumes *hit*:
as $\in$
alpha-header-supported-partial-union-bad-sets s bad-sets fr f-fri-roots f-final
and *not-prefix*:
as $\notin$ *alpha-prefix-union-bad-sets prefix-state bad-sets fr*
shows
alpha-header-partial-candidate-not-prefix-bound prefix-state s fr f-fri-roots f-final
proof (rule *ccontr*)
assume
 $\neg$ *alpha-header-partial-candidate-not-prefix-bound prefix-state s fr f-fri-roots f-final*
then have
alpha-header-supported-partial-union-bad-sets s bad-sets fr f-fri-roots f-final $\subseteq$
alpha-prefix-union-bad-sets prefix-state bad-sets fr
by (rule *alpha-header-supported-partial-union-bad-sets-subset-prefix-union-if-bound*)
then show *False*
using *hit not-prefix* **by** *blast*
qed

lemma *alpha-header-supported-partial-trace-table-candidates-update-unopened*:
assumes *outcome*:
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *partial*:
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr query-idxs trace-openings
and *candidate*:
partial-trace-table-candidate trace-table trace-openings
and *header*:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *unopened*:
 $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set } (\text{trace-openings } ! i) \implies$
opening-index opn $\neq$ j

```

shows
  trace-table[j := v] ∈
    alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
      f-final
proof –
  have candidate':
    partial-trace-table-candidate (trace-table[j := v]) trace-openings
  by (rule partial-trace-table-candidate-update-unopened
    [OF candidate unopened])
  show ?thesis
    unfolding alpha-header-supported-partial-trace-table-candidates-def
    using outcome partial candidate' header by blast
qed

lemma alpha-header-supported-partial-trace-table-candidates-not-singleton-if-trace-unopened:
assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr query-idxs trace-openings
and candidate:
  partial-trace-table-candidate trace-table trace-openings
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and j-bound: j < scale * clength
and unopened:
   $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set } (\text{trace-openings} \text{ ! } i) \implies$ 
    opening-index opn ≠ j
shows
   $\neg (\exists \text{ trace-table0.}$ 
    alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
      f-final  $\subseteq \{\text{trace-table0}\})$ 
proof
assume unique:
   $\exists \text{ trace-table0.}$ 
    alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
      f-final  $\subseteq \{\text{trace-table0}\}$ 
let ?trace-table' = trace-table[j := trace-table ! j + 1]
have candidate-in:
  trace-table ∈
    alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
      f-final
unfolding alpha-header-supported-partial-trace-table-candidates-def
using outcome partial candidate header by blast
have candidate'-in:
  ?trace-table' ∈
    alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
      f-final

```

```

    by (rule
      alpha-header-supported-partial-trace-table-candidates-update-unopened
      [OF outcome partial candidate header unopened])
  from unique obtain trace-table0 where subset:
    alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
    f-final  $\subseteq$  {trace-table0}
  by blast
  have trace-eq: trace-table = trace-table0
  using subset candidate-in by blast
  have trace'-eq: ?trace-table' = trace-table0
  using subset candidate'-in by blast
  have len: length trace-table = scale * clength
  using candidate unfolding partial-trace-table-candidate-def by simp
  have changed: ?trace-table' ! j  $\neq$  trace-table ! j
  using j-bound len by simp
  show False
  using changed trace-eq trace'-eq by simp
qed

lemma alpha-header-supported-partial-trace-table-candidates-not-singleton-if-trace-sparse:
  assumes outcome:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
  and partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
  and candidate:
    partial-trace-table-candidate trace-table trace-openings
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and sparse: length (List.concat trace-openings) < scale * clength
  shows
     $\neg$  ( $\exists$  trace-table0.
      alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
      f-final  $\subseteq$  {trace-table0})
  proof -
    have rounds-len: length trace-openings = rounds
    using candidate unfolding partial-trace-table-candidate-def by simp
    obtain j where j-bound: j < scale * clength
    and unopened:
       $\bigwedge i$  opn.  $i < \text{rounds} \implies \text{opn} \in \text{set } (\text{trace-openings} ! i) \implies$ 
      opening-index opn  $\neq$  j
    using sparse-openings-obtain-unopened-index [OF rounds-len sparse]
    by blast
    show ?thesis
    by (rule
      alpha-header-supported-partial-trace-table-candidates-not-singleton-if-trace-unopened
      [OF outcome partial candidate header j-bound unopened])
  qed

```


end

end

theory *Soundness-Alpha-Header-Cross-Candidate*

imports

Soundness-Alpha-Header-Candidate-Blocker

Soundness-Partial-Merkle-Cross

begin

Witnessed alpha-header partial trace candidates.

This layer keeps verifier outcomes and authenticated opening witnesses explicit. It proves the trace-only analogue of the query-header cross-candidate decomposition: if witnessed candidates are not singleton, then either pairwise opening coverage fails or a Merkle/collision side condition occurs.

context *soundness*

begin

definition *alpha-header-supported-partial-trace-table-candidate-witness*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow \text{bool}$

where

alpha-header-supported-partial-trace-table-candidate-witness s fr
 f -fri-roots f -final *trace-table* *final-state* *trace-openings* $\longleftrightarrow$
 $(\exists \text{ result query-idxs as dg composition-fri-roots final rest.}$
 $\text{Some (result, final-state)} \in \text{set-dist (execute verify-monad } s) \wedge$
 $\text{accepted-with-partial-trace-openings } s$
 $(\text{Some (result, final-state)}) \text{ fr query-idxs trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final as dg}$
 $\text{composition-fri-roots final rest})$

definition *alpha-header-supported-witnessed-partial-trace-table-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list set}$

where

alpha-header-supported-witnessed-partial-trace-table-candidates s fr
 f -fri-roots f -final =
 { *trace-table*.
 $\exists \text{ final-state trace-openings.}$
 $\text{alpha-header-supported-partial-trace-table-candidate-witness } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final trace-table final-state trace-openings}$ }

definition *alpha-header-supported-witnessed-partial-union-bad-sets*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$(f \text{ list} \Rightarrow f \text{ list set}) \Rightarrow$
 $f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow f \text{ list set}$

where

$\text{alpha-header-supported-witnessed-partial-union-bad-sets } s \text{ bad-sets } fr$
 $f\text{-fri-roots } f\text{-final} =$
 $\{as \in \text{alpha-space}.$
 $\exists \text{ trace-table} \in$
 $\text{alpha-header-supported-witnessed-partial-trace-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final}.$
 $as \in \text{bad-sets trace-table}\}$

lemma *alpha-header-supported-partial-trace-table-candidate-witness-imp-candidate:*

assumes

$\text{alpha-header-supported-partial-trace-table-candidate-witness } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final } \text{trace-table } \text{final-state } \text{trace-openings}$

shows

$\text{trace-table} \in$
 $\text{alpha-header-supported-partial-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final}$

using *assms*

unfolding *alpha-header-supported-partial-trace-table-candidate-witness-def*

alpha-header-supported-partial-trace-table-candidates-def

by *blast*

lemma *alpha-header-supported-witnessed-candidates-subset-candidates:*

$\text{alpha-header-supported-witnessed-partial-trace-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final} \subseteq$
 $\text{alpha-header-supported-partial-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final}$

unfolding *alpha-header-supported-witnessed-partial-trace-table-candidates-def*

using *alpha-header-supported-partial-trace-table-candidate-witness-imp-candidate*

by *blast*

lemma *alpha-header-supported-partial-trace-table-candidates-subset-witnessed-candidates:*

$\text{alpha-header-supported-partial-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final} \subseteq$
 $\text{alpha-header-supported-witnessed-partial-trace-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final}$

unfolding *alpha-header-supported-partial-trace-table-candidates-def*

alpha-header-supported-witnessed-partial-trace-table-candidates-def

alpha-header-supported-partial-trace-table-candidate-witness-def

accepted-with-partial-trace-openings-def

by *blast*

lemma *alpha-header-supported-witnessed-candidates-eq-candidates:*

$\text{alpha-header-supported-witnessed-partial-trace-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final} =$
 $\text{alpha-header-supported-partial-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final}$

using *alpha-header-supported-witnessed-candidates-subset-candidates*
alpha-header-supported-partial-trace-table-candidates-subset-witnessed-candidates
by *blast*

lemma *alpha-header-supported-witnessed-union-bad-sets-subset-union-bad-sets:*
alpha-header-supported-witnessed-partial-union-bad-sets s bad-sets fr
f-fri-roots f-final $\subseteq$
alpha-header-supported-partial-union-bad-sets s bad-sets fr f-fri-roots
f-final

unfolding *alpha-header-supported-witnessed-partial-union-bad-sets-def*
alpha-header-supported-partial-union-bad-sets-def

using *alpha-header-supported-witnessed-candidates-subset-candidates*
by *blast*

lemma *alpha-header-supported-union-bad-sets-subset-witnessed-union-bad-sets:*
alpha-header-supported-partial-union-bad-sets s bad-sets fr f-fri-roots
f-final $\subseteq$
alpha-header-supported-witnessed-partial-union-bad-sets s bad-sets fr
f-fri-roots f-final

unfolding *alpha-header-supported-witnessed-partial-union-bad-sets-def*
alpha-header-supported-partial-union-bad-sets-def

using *alpha-header-supported-partial-trace-table-candidates-subset-witnessed-candidates*
by *blast*

lemma *alpha-header-supported-witnessed-union-bad-sets-eq-union-bad-sets:*
alpha-header-supported-witnessed-partial-union-bad-sets s bad-sets fr
f-fri-roots f-final =
alpha-header-supported-partial-union-bad-sets s bad-sets fr f-fri-roots
f-final

using *alpha-header-supported-witnessed-union-bad-sets-subset-union-bad-sets*
alpha-header-supported-union-bad-sets-subset-witnessed-union-bad-sets

by *blast*

lemma *alpha-header-partial-candidate-not-prefix-bound-obtain-witness:*

assumes

alpha-header-partial-candidate-not-prefix-bound prefix-state s fr
f-fri-roots f-final

obtains *trace-table final-state trace-openings where*

alpha-header-supported-partial-trace-table-candidate-witness s fr
f-fri-roots f-final trace-table final-state trace-openings
trace-table $\notin$ alpha-prefix-trace-table-candidates prefix-state fr

using *assms*

unfolding *alpha-header-partial-candidate-not-prefix-bound-def*

using *alpha-header-supported-partial-trace-table-candidates-subset-witnessed-candidates*

unfolding *alpha-header-supported-witnessed-partial-trace-table-candidates-def*

by *blast*

lemma *alpha-header-supported-witnessed-partial-trace-table-candidates-obtain-witness:*

assumes

trace-table ∈
alpha-header-supported-witnessed-partial-trace-table-candidates *s* *fr*
f-fri-roots *f-final*
obtains *final-state* *trace-openings* **where**
alpha-header-supported-partial-trace-table-candidate-witness *s* *fr*
f-fri-roots *f-final* *trace-table* *final-state* *trace-openings*
using *assms*
unfolding *alpha-header-supported-witnessed-partial-trace-table-candidates-def*
by *blast*

lemma *alpha-header-supported-partial-trace-table-candidate-witness-shape*:
assumes
alpha-header-supported-partial-trace-table-candidate-witness *s* *fr*
f-fri-roots *f-final* *trace-table* *final-state* *trace-openings*
shows *partial-trace-table-candidate* *trace-table* *trace-openings*
using *assms*
unfolding *alpha-header-supported-partial-trace-table-candidate-witness-def*
by *blast*

lemma *alpha-header-supported-partial-trace-table-candidate-witness-eq-if-cross-cover-clean-merge*:
assumes *witness*:
alpha-header-supported-partial-trace-table-candidate-witness *s* *fr*
f-fri-roots *f-final* *trace-table* *final-state* *trace-openings*
and *witness'*:
alpha-header-supported-partial-trace-table-candidate-witness *s* *fr*
f-fri-roots *f-final* *trace-table'* *final-state'* *trace-openings'*
and *trace-cover*:
partial-openings-cross-cover-all-indices *trace-openings*
trace-openings'
and *no-conflict*:
 $\neg$ *merkle-hash-value-conflict* *final-state* *final-state'*
and *clean-merge*:
 $\neg$ *hash-map-output-collision*
(merkle-hash-state-merge *final-state* *final-state')*
shows *trace-table* = *trace-table'*

proof —

from *witness* **obtain** *result* *query-idxs* *as* *dg* *composition-fri-roots* *final* *rest*
where *trace-partial*:
accepted-with-partial-trace-openings *s*
(Some (result, final-state)) *fr* *query-idxs* *trace-openings*
and *trace-cand*:
partial-trace-table-candidate *trace-table* *trace-openings*
unfolding *alpha-header-supported-partial-trace-table-candidate-witness-def*
by *blast*
from *witness'* **obtain** *result'* *query-idxs'* *as'* *dg'* *composition-fri-roots'*
final' *rest'* **where** *trace-partial'*:
accepted-with-partial-trace-openings *s*
(Some (result', final-state')) *fr* *query-idxs'* *trace-openings'*
and *trace-cand'*:

```

    partial-trace-table-candidate trace-table' trace-openings'
  unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
  by blast
show ?thesis
by (rule partial-trace-table-candidates-eq-if-cross-cover-clean-merge
    [OF trace-partial trace-partial' trace-cand trace-cand' trace-cover
        no-conflict clean-merge])
qed

```

lemma *alpha-header-supported-witnessed-partial-trace-table-candidates-subset-singleton-if-pairwise-cross-cover-clean-*
assumes *pairwise:*

```

   $\bigwedge$  trace-table final-state trace-openings trace-table' final-state'
    trace-openings'.
  alpha-header-supported-partial-trace-table-candidate-witness s fr
    f-fri-roots f-final trace-table final-state trace-openings
 $\implies$ 
  alpha-header-supported-partial-trace-table-candidate-witness s fr
    f-fri-roots f-final trace-table' final-state' trace-openings'
 $\implies$ 
  partial-openings-cross-cover-all-indices trace-openings
    trace-openings'  $\wedge$ 
   $\neg$  merkle-hash-value-conflict final-state final-state'  $\wedge$ 
   $\neg$  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')

```

shows

```

 $\exists$  trace-table.
  alpha-header-supported-witnessed-partial-trace-table-candidates s fr
    f-fri-roots f-final  $\subseteq$  {trace-table}

```

proof (cases

```

  alpha-header-supported-witnessed-partial-trace-table-candidates s fr
    f-fri-roots f-final = {})

```

case *True*

then show *?thesis*

```

  by (intro exI[of - []]) simp

```

next

case *False*

then obtain *trace-table0* **where** *cand0*:

```

  trace-table0  $\in$ 
  alpha-header-supported-witnessed-partial-trace-table-candidates s fr
    f-fri-roots f-final
  by blast

```

from *alpha-header-supported-witnessed-partial-trace-table-candidates-obtain-witness*
 [OF *cand0*]

obtain *final-state0* *trace-openings0* **where** *witness0*:

```

  alpha-header-supported-partial-trace-table-candidate-witness s fr
    f-fri-roots f-final trace-table0 final-state0 trace-openings0
  by blast

```

have *subset*:

```

  alpha-header-supported-witnessed-partial-trace-table-candidates s fr

```

$f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table0}\}$
proof
fix *trace-table*
assume *cand*:
 $\text{trace-table} \in$
 $\alpha\text{-header-supported-witnessed-partial-trace-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final}$
from $\alpha\text{-header-supported-witnessed-partial-trace-table-candidates-obtain-witness}$
 $[OF \text{ cand}]$
obtain $\text{final-state } \text{trace-openings}$ **where** *witness*:
 $\alpha\text{-header-supported-partial-trace-table-candidate-witness } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final } \text{trace-table } \text{final-state } \text{trace-openings}$
by *blast*
have *pairwise-facts*:
 $\text{partial-openings-cross-cover-all-indices } \text{trace-openings}$
 trace-openings0
 $\neg \text{merkle-hash-value-conflict } \text{final-state } \text{final-state0}$
 $\neg \text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge } \text{final-state } \text{final-state0})$
using $\text{pairwise}[OF \text{ witness } \text{witness0}]$ **by** *simp-all*
have $\text{trace-table} = \text{trace-table0}$
by (*rule*
 $\alpha\text{-header-supported-partial-trace-table-candidate-witness-eq-if-cross-cover-clean-merge}$
 $[OF \text{ witness } \text{witness0 } \text{pairwise-facts}]$)
then show $\text{trace-table} \in \{\text{trace-table0}\}$
by *simp*
qed
then show *?thesis*
by *blast*
qed

lemma $\alpha\text{-header-supported-witnessed-partial-trace-table-candidates-not-singleton-imp-cross-cover-or-merkle-bad}$:
assumes *not-unique*:
 $\neg (\exists \text{trace-table.}$
 $\alpha\text{-header-supported-witnessed-partial-trace-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\})$
shows
 $\exists \text{trace-table } \text{final-state } \text{trace-openings } \text{trace-table}' \text{ final-state}'$
 $\text{trace-openings}'.$
 $\alpha\text{-header-supported-partial-trace-table-candidate-witness } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final } \text{trace-table } \text{final-state } \text{trace-openings} \wedge$
 $\alpha\text{-header-supported-partial-trace-table-candidate-witness } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final } \text{trace-table}' \text{ final-state}' \text{ trace-openings}' \wedge$
 $\text{trace-table} \neq \text{trace-table}' \wedge$
 $(\neg \text{partial-openings-cross-cover-all-indices } \text{trace-openings}$
 $\text{trace-openings}' \vee$
 $\text{merkle-hash-value-conflict } \text{final-state } \text{final-state}' \vee$
 $\text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge } \text{final-state } \text{final-state}'))$

```

proof –
  let ?C =
    alpha-header-supported-witnessed-partial-trace-table-candidates s fr
    f-fri-roots f-final
  have nonempty: ?C ≠ {}
    using not-unique by auto
  then obtain trace-table where cand: trace-table ∈ ?C
    by blast
  have not-subset: ¬ ?C ⊆ {trace-table}
    using not-unique by blast
  then obtain trace-table' where cand': trace-table' ∈ ?C
    and distinct: trace-table ≠ trace-table'
    by auto
  from alpha-header-supported-witnessed-partial-trace-table-candidates-obtain-witness
    [OF cand]
  obtain final-state trace-openings where witness:
    alpha-header-supported-partial-trace-table-candidate-witness s fr
    f-fri-roots f-final trace-table final-state trace-openings
    by blast
  from alpha-header-supported-witnessed-partial-trace-table-candidates-obtain-witness
    [OF cand]
  obtain final-state' trace-openings' where witness':
    alpha-header-supported-partial-trace-table-candidate-witness s fr
    f-fri-roots f-final trace-table' final-state' trace-openings'
    by blast
  have side:
    ¬ (partial-openings-cross-cover-all-indices trace-openings
      trace-openings' ∨
      merkle-hash-value-conflict final-state final-state' ∨
      hash-map-output-collision
      (merkle-hash-state-merge final-state final-state'))
  proof (rule ccontr)
    assume no-side:
      ¬ (¬ (partial-openings-cross-cover-all-indices trace-openings
        trace-openings' ∨
        merkle-hash-value-conflict final-state final-state' ∨
        hash-map-output-collision
        (merkle-hash-state-merge final-state final-state')))
    have trace-table = trace-table'
      by (rule
        alpha-header-supported-partial-trace-table-candidate-witness-eq-if-cross-cover-clean-merge
        [OF witness witness'])
        (use no-side in auto)
    then show False
      using distinct by simp
  qed
  show ?thesis
    using witness witness' distinct side by blast
  qed

```

definition *alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{bool}$

where

alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
 $s \text{ fr } f\text{-fri-roots } f\text{-final} \longleftrightarrow$
 $(\exists \text{ trace-table final-state trace-openings trace-table' final-state'}$
 $\text{ trace-openings'.$
 $\text{alpha-header-supported-partial-trace-table-candidate-witness } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final trace-table final-state trace-openings} \wedge$
 $\text{alpha-header-supported-partial-trace-table-candidate-witness } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final trace-table' final-state' trace-openings' } \wedge$
 $\text{trace-table} \neq \text{trace-table' } \wedge$
 $(\neg \text{partial-openings-cross-cover-all-indices trace-openings}$
 $\text{ trace-openings' } \vee$
 $\text{merkle-hash-value-conflict final-state final-state' } \vee$
 $\text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge final-state final-state'})))$

definition *alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad s
 $\longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final.}$
 $\text{alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad}$
 $s \text{ fr } f\text{-fri-roots } f\text{-final})$

lemma *alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-if-not-singleton:*

assumes *not-unique:*

$\neg (\exists \text{ trace-table.}$
 $\text{alpha-header-supported-witnessed-partial-trace-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\})$

shows

alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
 $s \text{ fr } f\text{-fri-roots } f\text{-final}$

unfolding

alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-def

using

alpha-header-supported-witnessed-partial-trace-table-candidates-not-singleton-imp-cross-cover-or-merkle-bad
 $[OF \text{ not-unique}]$

by *blast*

lemma *alpha-header-supported-witnessed-partial-trace-table-candidates-subset-singleton-if-no-cross-or-merkle-bad:*

assumes *no-bad:*

$\neg \text{alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad}$
 $s \text{ fr } f\text{-fri-roots } f\text{-final}$

shows

$\exists$ *trace-table*.
 α -header-supported-witnessed-partial-trace-table-candidates *s* fr
 f -fri-roots f -final $\subseteq \{\text{trace-table}\}$
proof (rule *ccontr*)
assume *not-unique*:
 $\neg (\exists \text{trace-table}.$
 α -header-supported-witnessed-partial-trace-table-candidates *s* fr
 f -fri-roots f -final $\subseteq \{\text{trace-table}\})$
then have
 α -header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
s fr f -fri-roots f -final
by (rule
 α -header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-if-not-singleton)
then show *False*
using *no-bad* **by** *contradiction*
qed

lemma α -header-supported-partial-trace-table-candidates-subset-singleton-if-no-witnessed-cross-or-merkle-bad:
assumes *no-bad*:
 $\neg \alpha$ -header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
s fr f -fri-roots f -final
shows
 $\exists$ *trace-table*.
 α -header-supported-partial-trace-table-candidates *s* fr f -fri-roots
 f -final $\subseteq \{\text{trace-table}\}$
proof –
have
 $\exists$ *trace-table*.
 α -header-supported-witnessed-partial-trace-table-candidates *s* fr
 f -fri-roots f -final $\subseteq \{\text{trace-table}\}$
by (rule
 α -header-supported-witnessed-partial-trace-table-candidates-subset-singleton-if-no-cross-or-merkle-bad
[OF *no-bad*])
moreover have
 α -header-supported-partial-trace-table-candidates *s* fr f -fri-roots
 f -final =
 α -header-supported-witnessed-partial-trace-table-candidates *s* fr
 f -fri-roots f -final
using α -header-supported-witnessed-candidates-eq-candidates
[of *s* fr f -fri-roots f -final]
by *simp*
then show *?thesis*
using *calculation* **by** *simp*
qed

lemma α -header-supported-partial-trace-table-candidates-subset-singleton-if-no-global-witnessed-cross-or-merkle-bad:
assumes *no-bad*:
 $\neg \alpha$ -supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
s

shows
 $\exists \text{ trace-table.}$
 $\alpha\text{-header-supported-partial-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final} \subseteq \{\text{trace-table}\}$
proof (rule
 $\alpha\text{-header-supported-partial-trace-table-candidates-subset-singleton-if-no-witnessed-cross-or-merkle-bad}$)
show
 $\neg \alpha\text{-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad}$
 $s \text{ fr } f\text{-fri-roots } f\text{-final}$
using no-bad
unfolding
 $\alpha\text{-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-def}$
by blast
qed

lemma $\alpha\text{-header-supported-witnessed-partial-union-bad-sets-fraction-bound-if-no-cross-or-merkle-bad}$:
fixes $C :: \text{prob}$
assumes no-bad :
 $\neg \alpha\text{-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad}$
 $s \text{ fr } f\text{-fri-roots } f\text{-final}$
and $\text{subset: } \bigwedge \text{trace-table. } \text{bad-sets trace-table} \subseteq \alpha\text{-space}$
and bounded:
 $\bigwedge \text{trace-table.}$
 $\text{nnreal (card (bad-sets trace-table)) / nnreal (card } \alpha\text{-space)} \leq$
 C
shows
 nnreal
 $(\text{card}$
 $(\alpha\text{-header-supported-witnessed-partial-union-bad-sets } s \text{ bad-sets}$
 $\text{fr } f\text{-fri-roots } f\text{-final})) /$
 $\text{nnreal (card } \alpha\text{-space)} \leq C$
proof –
have unique:
 $\exists \text{ trace-table.}$
 $\alpha\text{-header-supported-witnessed-partial-trace-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\}$
by (rule
 $\alpha\text{-header-supported-witnessed-partial-trace-table-candidates-subset-singleton-if-no-cross-or-merkle-bad}$
 $[\text{OF no-bad}]$)
from unique **obtain** trace-table **where** candidates:
 $\alpha\text{-header-supported-witnessed-partial-trace-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\}$
by blast
have union-subset:
 $\alpha\text{-header-supported-witnessed-partial-union-bad-sets } s \text{ bad-sets fr}$
 $f\text{-fri-roots } f\text{-final} \subseteq \text{bad-sets trace-table}$
unfolding $\alpha\text{-header-supported-witnessed-partial-union-bad-sets-def}$
using candidates **by** auto
have $\text{finite-bad: finite (bad-sets trace-table)}$

```

    by (rule finite-subset[OF subset finite-alpha-space])
  have card-le:
    card
      (alpha-header-supported-witnessed-partial-union-bad-sets s bad-sets fr
        f-fri-roots f-final) ≤ card (bad-sets trace-table)
    by (rule card-mono[OF finite-bad union-subset])
  have nnreal
    (card
      (alpha-header-supported-witnessed-partial-union-bad-sets s bad-sets
        fr f-fri-roots f-final)) /
    nnreal (card alpha-space) ≤
    nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
  also have ... ≤ C
  by (rule bounded)
  finally show ?thesis .
qed

lemma alpha-header-supported-partial-union-bad-sets-fraction-bound-if-no-witnessed-cross-or-merkle-bad:
  fixes C :: prob
  assumes no-bad:
    ¬ alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
      s fr f-fri-roots f-final
  and subset:  $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$ 
  and bounded:
     $\bigwedge \text{trace-table.}$ 
    nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space) ≤
    C
  shows
    nnreal
      (card
        (alpha-header-supported-partial-union-bad-sets s bad-sets fr
          f-fri-roots f-final)) /
      nnreal (card alpha-space) ≤ C
  using
    alpha-header-supported-witnessed-partial-union-bad-sets-fraction-bound-if-no-cross-or-merkle-bad
      [OF no-bad subset bounded]
  by (simp add: alpha-header-supported-witnessed-union-bad-sets-eq-union-bad-sets)

lemma alpha-header-supported-witnessed-partial-union-bad-sets-fraction-bound-if-no-global-cross-or-merkle-bad:
  fixes C :: prob
  assumes no-bad:
    ¬ alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
      s
  and subset:  $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$ 
  and bounded:
     $\bigwedge \text{trace-table.}$ 
    nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space) ≤
    C

```

shows

$$\text{nnreal} \left(\text{card} \left(\text{alpha-header-supported-witnessed-partial-union-bad-sets } s \text{ bad-sets} \right. \right. \\ \left. \left. \text{fr } f\text{-fri-roots } f\text{-final} \right) \right) / \\ \text{nnreal} \left(\text{card } \text{alpha-space} \right) \leq C$$
proof –
have *no-header-bad*:
 $\neg \text{alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad}$
 $s \text{ fr } f\text{-fri-roots } f\text{-final}$
using *no-bad*
unfolding
 $\text{alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-def}$
by *blast*
show *?thesis*
by (*rule*
 $\text{alpha-header-supported-witnessed-partial-union-bad-sets-fraction-bound-if-no-cross-or-merkle-bad}$
 $[OF \text{ no-header-bad subset bounded}]$)
qed

lemma *alpha-header-supported-partial-union-bad-sets-fraction-bound-if-no-global-witnessed-cross-or-merkle-bad*:
fixes $C :: \text{prob}$
assumes *no-bad*:
 $\neg \text{alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad}$
 s
and *subset*: $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$
and *bounded*:
 $\bigwedge \text{trace-table.}$
 $\text{nnreal} \left(\text{card} \left(\text{bad-sets trace-table} \right) \right) / \text{nnreal} \left(\text{card } \text{alpha-space} \right) \leq$
 C
shows

$$\text{nnreal} \left(\text{card} \left(\text{alpha-header-supported-partial-union-bad-sets } s \text{ bad-sets fr} \right. \right. \\ \left. \left. f\text{-fri-roots } f\text{-final} \right) \right) / \\ \text{nnreal} \left(\text{card } \text{alpha-space} \right) \leq C$$
using
 $\text{alpha-header-supported-witnessed-partial-union-bad-sets-fraction-bound-if-no-global-cross-or-merkle-bad}$
 $[OF \text{ no-bad subset bounded}]$
by (*simp add: alpha-header-supported-witnessed-union-bad-sets-eq-union-bad-sets*)

lemma *wp-alpha-bad-set-hit-bound-if-no-global-witnessed-cross-or-merkle-bad*:
fixes $C \ H :: \text{prob}$
assumes *future: alpha-future-fresh s*
and *no-bad*:
 $\neg \text{alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad}$
 s
and *subset*: $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$
and *bounded*:

$\bigwedge \text{trace-table.}$
 $\text{nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)} \leq$
 C
and *collision-bound*:
 $\text{wp-event verify-monad (hash-map-output-collision-bad } s) s \leq H$
shows $\text{wp-event verify-monad (alpha-bad-set-hit } s \text{ bad-sets)} s \leq C + H$
proof –
have *unique*:
 $\bigwedge \text{fr } f\text{-fri-roots } f\text{-final.}$
 $\exists \text{ trace-table.}$
 $\text{alpha-header-supported-partial-trace-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\}$
by (rule
 $\text{alpha-header-supported-partial-trace-table-candidates-subset-singleton-if-no-global-witnessed-cross-or-merkle-bad}$
 $[\text{OF no-bad}]$)
show *?thesis*
by (rule
 $\text{wp-alpha-bad-set-hit-bound-if-supported-partial-header-candidate-unique-or-collision}$
 $[\text{OF future unique subset bounded collision-bound}]$)
qed

lemma *composition-randomization-bad-bound-if-no-global-witnessed-cross-or-merkle-bad*:
fixes $H :: \text{prob}$
assumes *future*: $\text{alpha-future-fresh } s$
and *no-bad*:
 $\neg \text{alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad}$
 s
and *collision-bound*:
 $\text{wp-event verify-monad (hash-map-output-collision-bad } s) s \leq H$
shows
 $\text{wp-event verify-monad (composition-randomization-bad } s) s \leq$
 $\text{composition-error-bound} + H$
proof –
have *unique*:
 $\bigwedge \text{fr } f\text{-fri-roots } f\text{-final.}$
 $\exists \text{ trace-table.}$
 $\text{alpha-header-supported-partial-trace-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\}$
by (rule
 $\text{alpha-header-supported-partial-trace-table-candidates-subset-singleton-if-no-global-witnessed-cross-or-merkle-bad}$
 $[\text{OF no-bad}]$)
show *?thesis*
by (rule
 $\text{composition-randomization-bad-bound-if-alpha-header-supported-partial-candidate-unique-or-collision}$
 $[\text{OF future unique collision-bound}]$)
qed

lemma *alpha-header-witness-candidate-in-prefix-if-pullback-clean-cover*:
assumes *prefix-candidate*:

```

    prefix-trace-table ∈
      alpha-prefix-trace-table-candidates prefix-state fr
  and witness:
    alpha-header-supported-partial-trace-table-candidate-witness s fr
    f-fri-roots f-final trace-table final-state trace-openings
  and pullback:
     $\bigwedge i. i < \text{rounds} \implies$ 
      partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) prefix-state
  and cover:
     $\bigwedge j. j < \text{scale} * \text{clength} \implies$ 
       $\exists i < \text{rounds}. \exists \text{opn} \in \text{set } (\text{trace-openings} ! i).$ 
      opening-index opn = j
  and clean-prefix:  $\neg$  hash-map-output-collision prefix-state
shows
  trace-table ∈
    alpha-prefix-trace-table-candidates prefix-state fr
proof -
  have prefix-len:
    length prefix-trace-table = clength * scale
  using prefix-candidate
  unfolding alpha-prefix-trace-table-candidates-def by simp
  have prefix-bind:
    merkle-root-binds-table fr prefix-trace-table prefix-state
  using prefix-candidate
  unfolding alpha-prefix-trace-table-candidates-def by simp
  from witness obtain result query-idxs as dg composition-fri-roots final rest
  where trace-cand:
    partial-trace-table-candidate trace-table trace-openings
  unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
  by blast
  have prefix-trace-cand:
    partial-trace-table-candidate prefix-trace-table trace-openings
proof -
  obtain n where len-pow: clength * scale =  $2^n$ 
  using eval-domain-length-power by blast
  have table-pow: length prefix-trace-table =  $2^n$ 
  using prefix-len len-pow by (simp add: mult.commute)
  have rounds-len: length trace-openings = rounds
  using trace-cand unfolding partial-trace-table-candidate-def by simp
  have agrees:
     $\bigwedge i. i < \text{rounds} \implies$ 
      table-agrees-with-authenticated-openings prefix-trace-table
      (scale * clength) (trace-openings ! i)
proof -
  fix i
  assume i-bound: i < rounds
  have table-i:
    partial-authenticated-table fr (scale * clength)

```

```

    (trace-openings ! i) prefix-state
  by (rule pullback[OF i-bound])
have opening-agrees:
   $\forall \text{opn} \in \text{set } (\text{trace-openings} ! i).$ 
  opening-index opn < scale * clength  $\wedge$ 
  prefix-trace-table ! opening-index opn = opening-value opn
proof
  fix opn
  assume opn-in: opn  $\in$  set (trace-openings ! i)
  have root: opening-root opn = fr
    using table-i opn-in unfolding partial-authenticated-table-def
    by blast
  have len: opening-length opn = length prefix-trace-table
    using table-i opn-in prefix-len
    unfolding partial-authenticated-table-def by simp
  have auth: authenticated-opening-in prefix-state opn
    using table-i opn-in unfolding partial-authenticated-table-def
    by blast
  have agreement:
    opening-index opn < length prefix-trace-table  $\wedge$ 
    prefix-trace-table ! opening-index opn = opening-value opn
  by (rule merkle-bound-table-agrees-with-authenticated-opening-if-clean
    [OF prefix-bind table-pow auth root len clean-prefix])
  then show
    opening-index opn < scale * clength  $\wedge$ 
    prefix-trace-table ! opening-index opn = opening-value opn
    using prefix-len by (simp add: mult.commute)
qed
show
  table-agrees-with-authenticated-openings prefix-trace-table
  (scale * clength) (trace-openings ! i)
  unfolding table-agrees-with-authenticated-openings-def
  using prefix-len opening-agrees by simp
qed
show ?thesis
  unfolding partial-trace-table-candidate-def
  using prefix-len rounds-len agrees by simp
qed
have trace-eq: trace-table = prefix-trace-table
  by (rule partial-trace-table-candidate-eq-if-all-indices-opened
    [OF trace-cand prefix-trace-cand cover])
show ?thesis
  using prefix-candidate unfolding trace-eq .
qed

lemma alpha-header-not-prefix-witness-imp-no-prefix-candidate-or-pullback-gap:
  assumes not-prefix:
    trace-table  $\notin$  alpha-prefix-trace-table-candidates prefix-state fr
  and witness:

```

alpha-header-supported-partial-trace-table-candidate-witness s fr
f-fri-roots f-final trace-table final-state trace-openings
and *clean-prefix*: $\neg$ *hash-map-output-collision prefix-state*
and *pullback*:
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{partial-authenticated-table fr (scale * clength)}$
 $(\text{trace-openings ! } i) \text{ prefix-state}$
and *cover*:
 $\bigwedge j. j < \text{scale * clength} \implies$
 $\exists i < \text{rounds}. \exists \text{opn} \in \text{set } (\text{trace-openings ! } i).$
 $\text{opening-index opn} = j$
shows
 $\text{alpha-prefix-trace-table-candidates prefix-state fr} = \{\}$
proof (rule *ccontr*)
assume $\text{alpha-prefix-trace-table-candidates prefix-state fr} \neq \{\}$
then obtain *prefix-trace-table* **where** *prefix-candidate*:
 $\text{prefix-trace-table} \in$
 $\text{alpha-prefix-trace-table-candidates prefix-state fr}$
by *blast*
have
 $\text{trace-table} \in$
 $\text{alpha-prefix-trace-table-candidates prefix-state fr}$
by (rule *alpha-header-witness-candidate-in-prefix-if-pullback-clean-cover*
 $[\text{OF prefix-candidate witness pullback cover clean-prefix}]$)
then show *False*
using *not-prefix* **by** *contradiction*
qed

lemma *alpha-header-witness-opening-pullback-or-new-output-hit*:
assumes *ext*: $\text{prefix-state} \leq \text{final-state}$
and *witness*:
 $\text{alpha-header-supported-partial-trace-table-candidate-witness s fr}$
 $\text{f-fri-roots f-final trace-table final-state trace-openings}$
and *i-bound*: $i < \text{rounds}$
and *opn-in*: $\text{opn} \in \text{set } (\text{trace-openings ! } i)$
shows
 $\text{authenticated-opening-in prefix-state opn} \vee$
 $\text{hash-map-new-output-hit}$
 $(\text{merkle-path-target-roots final-state fr (scale * clength)})$
 $(\text{opening-index opn}) (\text{opening-value opn}) (\text{opening-path opn})$
 $\text{prefix-state final-state}$
proof –
from *witness* **obtain** *result query-idxs* as *dg composition-fri-roots final rest*
where *partial*:
 $\text{accepted-with-partial-trace-openings s}$
 $(\text{Some (result, final-state)}) \text{ fr query-idxs trace-openings}$
unfolding *alpha-header-supported-partial-trace-table-candidate-witness-def*
by *blast*
have *table-i*:


```

    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
  using partial i-bound
  unfolding accepted-with-partial-trace-openings-def by blast
show ?thesis
  by (rule partial-authenticated-table-opening-pullback-or-new-output-hit
      [OF ext table-i opn-in])
qed

lemma alpha-header-witness-openings-pullback-or-new-output-hit:
  assumes ext: prefix-state ≤ final-state
  and witness:
    alpha-header-supported-partial-trace-table-candidate-witness s fr
      f-fri-roots f-final trace-table final-state trace-openings
  shows
    (∀ i < rounds.
      partial-authenticated-table fr (scale * clength)
        (trace-openings ! i) prefix-state) ∨
    (∃ i < rounds. ∃ opn ∈ set (trace-openings ! i).
      hash-map-new-output-hit
        (merkle-path-target-roots final-state fr (scale * clength)
          (opening-index opn) (opening-value opn) (opening-path opn))
        prefix-state final-state)
  proof (cases
    ∃ i < rounds. ∃ opn ∈ set (trace-openings ! i).
      hash-map-new-output-hit
        (merkle-path-target-roots final-state fr (scale * clength)
          (opening-index opn) (opening-value opn) (opening-path opn))
        prefix-state final-state)
  case True
  then show ?thesis by simp
next
case False
have pullback-all:
  ∀ i < rounds.
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) prefix-state
  proof (intro allI impI)
    fix i
    assume i-bound: i < rounds
    from witness obtain result query-idxs as dg composition-fri-roots final rest
    where partial:
      accepted-with-partial-trace-openings s
        (Some (result, final-state)) fr query-idxs trace-openings
    unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
    by blast
  have table-i:
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state

```

```

using partial i-bound
unfolding accepted-with-partial-trace-openings-def by blast
show
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) prefix-state
unfolding partial-authenticated-table-def
proof
  fix opn
  assume opn-in: opn ∈ set (trace-openings ! i)
  have root-len:
    opening-root opn = fr ∧ opening-length opn = scale * clength
    using table-i opn-in unfolding partial-authenticated-table-def
    by blast
  have pullback:
    authenticated-opening-in prefix-state opn ∨
    hash-map-new-output-hit
    (merkle-path-target-roots final-state fr (scale * clength)
    (opening-index opn) (opening-value opn) (opening-path opn))
    prefix-state final-state
    by (rule alpha-header-witness-opening-pullback-or-new-output-hit
    [OF ext witness i-bound opn-in])
  have auth-prefix: authenticated-opening-in prefix-state opn
    using pullback False i-bound opn-in by blast
  show
    opening-root opn = fr ∧
    opening-length opn = scale * clength ∧
    authenticated-opening-in prefix-state opn
    using root-len auth-prefix by simp
  qed
qed
then show ?thesis by simp
qed

end

end

```

```

theory Staged-Security-Experiment-Alpha-Header-Binding
imports
  Staged-Security-Experiment-Composition-Query-Residual
  Soundness-Alpha-Header-Cross-Candidate
begin

```

Staged wrapper for the alpha-header candidate-binding diagnostic.

This does not bound the event. It isolates the remaining proof obligation: a partial-header alpha hit that is not already a prefix-fixed alpha hit must use a header-supported partial trace candidate that is not bound by the prefix Merkle evidence.

```

context soundness

```

begin

definition *checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*

```

:: ('f list ⇒ 'f list set) ⇒
  ((((((('f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
  bad-sets out ⇔
  (case out of
    None ⇒ False
  | Some (x, final-state) ⇒
    (let packed = fst (fst x);
      prefix = fst (fst packed);
      prefix-state = snd (fst packed);
      data = snd packed;
      attacker-state = snd (fst x);
      result = snd x;
      s =
        verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)
    in ∃ as query-idxs fr f-fri-roots f-final dg composition-fri-roots
      final rest.
      accepted-transcript-shape s (Some (result, final-state))
        as query-idxs ∧
      verifier-header-transcript s fr f-fri-roots f-final as dg
        composition-fri-roots final rest ∧
      as ∈
        alpha-header-supported-partial-union-bad-sets s bad-sets fr
          f-fri-roots f-final ∧
      as ∉ alpha-prefix-union-bad-sets prefix-state bad-sets fr ∧
      alpha-header-partial-candidate-not-prefix-bound prefix-state s
        fr f-fri-roots f-final))

```

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-boundI*:

assumes *s-eq*:

```

s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)

```

and *shape*:

```

accepted-transcript-shape s (Some (result, final-state))
  as query-idxs

```

and *header*:

```

verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest

```

and *hit*:

```

as ∈

```

```

      alpha-header-supported-partial-union-bad-sets s bad-sets fr
      f-fri-roots f-final
    and not-prefix:
      as ∉ alpha-prefix-union-bad-sets prefix-state bad-sets fr
    and not-bound:
      alpha-header-partial-candidate-not-prefix-bound prefix-state s fr
      f-fri-roots f-final
  shows
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
    bad-sets
    (Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state))
proof —
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have shape':
    accepted-transcript-shape ?s (Some (result, final-state))
    as query-idxs
  using shape by (simp add: s-eq)
  have header':
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  using header by (simp add: s-eq)
  have hit':
    as ∈
    alpha-header-supported-partial-union-bad-sets ?s bad-sets fr
    f-fri-roots f-final
  using hit by (simp add: s-eq)
  have not-bound':
    alpha-header-partial-candidate-not-prefix-bound prefix-state ?s fr
    f-fri-roots f-final
  using not-bound by (simp add: s-eq)
  show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound-def
    Let-def
  apply simp
  apply (rule exI[where x=as])
  apply (intro conjI)
  apply (rule exI[where x=query-idxs])
  apply (rule shape')
  apply (rule exI[where x=fr])
  apply (rule exI[where x=f-fri-roots])
  apply (rule exI[where x=f-final])
  apply (intro conjI)
  apply (rule exI[where x=dg])
  apply (rule exI[where x=composition-fri-roots])
  apply (rule exI[where x=final])

```

```

    apply (rule exI[where x=rest])
    apply (rule header')
    apply (rule hit')
    apply (rule not-prefix)
    apply (rule not-bound')
  done
qed

lemma checked-staged-security-with-actual-alpha-prefix-support-prefix-extends-verifier-state:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  shows
    prefix-state ≤
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
proof -
  from support obtain trans-out where trans-out:
    Some (((prefix, prefix-state), data), attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-with-alpha-prefix-program A)
        adversary-initial-state)
  unfolding
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
  from checked-staged-transcript-with-alpha-prefix-program-support
  [OF wf controlled trans-out]
  have prefix-ext: prefix-state ≤ attacker-state
  by blast
  show ?thesis
  by (rule
    alpha-prefix-hash-extends-verifier-state-from-adversary-right
    [OF prefix-ext])
qed

lemma checked-staged-security-with-actual-alpha-prefix-support-prefix-extends-verifier-final:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state) ∈
    set-dist

```

```

      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
    and verifier-out:
      Some (result', witness-final-state) ∈
        set-dist
          (execute verify-monad
            (verifier-state-from-adversary attacker-state
              (staged-proof-transcript data)))
    shows prefix-state ≤ witness-final-state
  proof -
    have prefix-s:
      prefix-state ≤
        verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)
    by (rule
      checked-staged-security-with-actual-alpha-prefix-support-prefix-extends-verifier-state
      [OF wf controlled support])
    have s-final:
      verifier-state-from-adversary attacker-state
        (staged-proof-transcript data) ≤
          witness-final-state
    by (rule verify-monad-hash-extends[OF verifier-out])
    show ?thesis
    by (rule hash-ext-trans[OF prefix-s s-final])
  qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-obtain-witness-final-ext:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
and bound:
  alpha-header-partial-candidate-not-prefix-bound prefix-state
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
  fr f-fri-roots f-final
obtains trace-table witness-final-state trace-openings where
  alpha-header-supported-partial-trace-table-candidate-witness
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
  fr f-fri-roots f-final trace-table witness-final-state

```

trace-openings
trace-table $\notin$ *alpha-prefix-trace-table-candidates* *prefix-state* *fr*
prefix-state $\leq$ *witness-final-state*
proof –
let *?s* =
 verifier-state-from-adversary *attacker-state*
 (*staged-proof-transcript* *data*)
from *alpha-header-partial-candidate-not-prefix-bound-obtain-witness*
 [*OF bound*]
obtain *trace-table* *witness-final-state* *trace-openings* **where** *witness*:
 alpha-header-supported-partial-trace-table-candidate-witness *?s* *fr*
 f-fri-roots *f-final* *trace-table* *witness-final-state*
 trace-openings
and *not-prefix*:
 trace-table $\notin$ *alpha-prefix-trace-table-candidates* *prefix-state* *fr*
by *blast*
from *witness* **obtain** *witness-result* *query-idxs* *as* *dg composition-fri-roots*
 final *rest* **where** *verifier-out*:
 Some (*witness-result*, *witness-final-state*) $\in$
 set-dist (*execute* *verify-monad* *?s*)
unfolding *alpha-header-supported-partial-trace-table-candidate-witness-def*
by *blast*
have *prefix-final*: *prefix-state* $\leq$ *witness-final-state*
by (*rule*
 checked-staged-security-with-actual-alpha-prefix-support-prefix-extends-verifier-final
 [*OF wf controlled support verifier-out*])
show *?thesis*
by (*rule that*[*OF witness not-prefix prefix-final*])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-decompose*:

assumes *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *support*:
 Some (((((*prefix*, *prefix-state*), *data*), *attacker-state*), *result*),
 final-state) $\in$
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 adversary-initial-state)
and *bound*:
 alpha-header-partial-candidate-not-prefix-bound *prefix-state*
 (*verifier-state-from-adversary* *attacker-state*
 (*staged-proof-transcript* *data*))
 fr *f-fri-roots* *f-final*
shows
 hash-map-output-collision *prefix-state* $\vee$
 ($\exists$ *trace-table* *witness-final-state* *trace-openings*.

```

    alpha-header-supported-partial-trace-table-candidate-witness
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      fr f-fri-roots f-final trace-table witness-final-state
      trace-openings  $\wedge$ 
      trace-table  $\notin$ 
      alpha-prefix-trace-table-candidates prefix-state fr  $\wedge$ 
      prefix-state  $\leq$  witness-final-state  $\wedge$ 
      (( $\exists j < \text{scale} * \text{clength}.$ 
         $\forall i \text{ opn}. i < \text{rounds} \longrightarrow$ 
          opn  $\in \text{set } (\text{trace-openings } ! i) \longrightarrow$ 
            opening-index opn  $\neq j$ )  $\vee$ 
        alpha-prefix-trace-table-candidates prefix-state fr = {}  $\vee$ 
        ( $\exists i < \text{rounds}. \exists \text{opn} \in \text{set } (\text{trace-openings } ! i).$ 
          hash-map-new-output-hit
            (merkle-path-target-roots witness-final-state fr
              (scale * clength) (opening-index opn)
              (opening-value opn) (opening-path opn))
            prefix-state witness-final-state)))

proof –
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-obtain-witness-final-ext
    [OF wf controlled support bound]
  obtain trace-table witness-final-state trace-openings where witness:
    alpha-header-supported-partial-trace-table-candidate-witness ?s fr
    f-fri-roots f-final trace-table witness-final-state
    trace-openings
  and not-prefix:
    trace-table  $\notin$ 
    alpha-prefix-trace-table-candidates prefix-state fr
  and prefix-final: prefix-state  $\leq$  witness-final-state
  by blast
  show ?thesis
  proof (cases hash-map-output-collision prefix-state)
    case True
      then show ?thesis by simp
  next
    case clean-prefix: False
    show ?thesis
    proof (cases
       $\forall j < \text{scale} * \text{clength}.$ 
       $\exists i < \text{rounds}. \exists \text{opn} \in \text{set } (\text{trace-openings } ! i).$ 
      opening-index opn = j)
      case False
        then have uncovered:
           $\exists j < \text{scale} * \text{clength}.$ 
           $\forall i \text{ opn}. i < \text{rounds} \longrightarrow$ 

```



```

       $opn \in \text{set } (\text{trace-openings } ! i) \longrightarrow$ 
       $\text{opening-index } opn \neq j$ 
    by blast
  show ?thesis
    using witness not-prefix prefix-final uncovered by blast
next
case True
have pullback-or-new:
  ( $\forall i < \text{rounds}.$ 
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) prefix-state)  $\vee$ 
  ( $\exists i < \text{rounds}.$   $\exists opn \in \text{set } (\text{trace-openings } ! i).$ 
    hash-map-new-output-hit
    (merkle-path-target-roots witness-final-state fr
      (scale * clength) (opening-index opn)
      (opening-value opn) (opening-path opn))
    prefix-state witness-final-state)
  by (rule alpha-header-witness-openings-pullback-or-new-output-hit
    [OF prefix-final witness])
from pullback-or-new show ?thesis
proof
  assume pullback-all:
     $\forall i < \text{rounds}.$ 
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) prefix-state
  have no-prefix-candidate:
    alpha-prefix-trace-table-candidates prefix-state fr = {}
  proof (rule
    alpha-header-not-prefix-witness-imp-no-prefix-candidate-or-pullback-gap
    [OF not-prefix witness clean-prefix])
    fix i
    assume i < rounds
    then show
      partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) prefix-state
      using pullback-all by blast
  next
  fix j
  assume j < scale * clength
  then show
     $\exists i < \text{rounds}.$   $\exists opn \in \text{set } (\text{trace-openings } ! i).$ 
    opening-index opn = j
    using True by blast
qed
show ?thesis
  using witness not-prefix prefix-final no-prefix-candidate by blast
next
assume
   $\exists i < \text{rounds}.$   $\exists opn \in \text{set } (\text{trace-openings } ! i).$ 

```

```

    hash-map-new-output-hit
    (merkle-path-target-roots witness-final-state fr
      (scale * clength) (opening-index opn)
      (opening-value opn) (opening-path opn))
    prefix-state witness-final-state
  then show ?thesis
  using witness not-prefix prefix-final by blast
qed
qed
qed
qed

definition checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
:: ((((((f × 'f list × 'f list × 'f) ×
  'f protocol-channel) × 'f staged-proof-data) ×
  'f protocol-channel) × unit list) ×
  'f protocol-channel) option ⇒ bool
where
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision out
  ⇔
  (case out of
    None ⇒ False
  | Some (x, -) ⇒
    (let packed = fst (fst x);
      prefix-state = snd (fst packed)
    in hash-map-output-collision prefix-state))

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
  adversary-initial-state ≤
  hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0)
proof –
have projection:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
  adversary-initial-state ≤
  wp-event
  (checked-staged-transcript-with-alpha-prefix-program A)
  checked-staged-transcript-actual-alpha-prefix-collision-hit
  adversary-initial-state
unfolding
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
proof (rule wp-event-bind-bound-by-head-event)

```

```

show
  wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    checked-staged-transcript-actual-alpha-prefix-collision-hit
    adversary-initial-state ≤
  wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    checked-staged-transcript-actual-alpha-prefix-collision-hit
    adversary-initial-state
by simp
next
show
  checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
    None ⇒
  checked-staged-transcript-actual-alpha-prefix-collision-hit None
unfolding
  checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-def
  checked-staged-transcript-actual-alpha-prefix-collision-hit-def
by simp
next
fix packed attacker-state out
assume cont:
  out ∈
    set-dist
      (execute
        (get >>=
          (λs. put
            (verifier-state-from-adversary s
              (staged-proof-transcript (snd packed)))) >>=
            (λ-. verify-monad >>=
              (λresult. return ((packed, s), result))))))
        attacker-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
    out
obtain prefix prefix-state data where packed-eq:
  packed = ((prefix, prefix-state), data)
by (cases packed, auto split: prod.splits)
have hash-map-output-collision prefix-state
using cont hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-def
  packed-eq
by (auto elim!: set-dist-bindE split: option.splits prod.splits)
then show
  checked-staged-transcript-actual-alpha-prefix-collision-hit
    (Some (packed, attacker-state))
unfolding checked-staged-transcript-actual-alpha-prefix-collision-hit-def
  packed-eq
by simp
qed

```

```

have transcript-bound:
  wp-event
    (checked-staged-transcript-with-alpha-prefix-program A)
    checked-staged-transcript-actual-alpha-prefix-collision-hit
    adversary-initial-state ≤
  wp-event (staged-alpha-prefix-program A)
    (hash-new-collision-event adversary-initial-state)
    adversary-initial-state
  by (rule checked-staged-transcript-with-alpha-prefix-collision-bound)
have collision-budget:
  hash-collision-budget (staged-alpha-search-queries budgets 0)
    (staged-alpha-prefix-program A)
  by (rule hash-collision-budget-staged-alpha-prefix-program
    [OF wf controlled])
have collision-bound:
  wp-event (staged-alpha-prefix-program A)
    (hash-new-collision-event adversary-initial-state)
    adversary-initial-state ≤
    hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0)
proof –
  have
    wp-event (staged-alpha-prefix-program A)
      (hash-new-collision-event adversary-initial-state)
      adversary-initial-state ≤
      hash-collision-budget-value
        (card (hash-map-output-values adversary-initial-state))
        (staged-alpha-search-queries budgets 0)
    using collision-budget adversary-initial-state-no-output-collision
    unfolding hash-collision-budget-def by blast
  also have ... =
    hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0)
  by simp
  finally show ?thesis .
qed
show ?thesis
  by (rule order-trans[OF projection])
    (rule order-trans[OF transcript-bound collision-bound])
qed

definition checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
  :: ((((((f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool
where
  checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
    out ⇔
  (case out of
    None ⇒ False

```

```

| Some (x, -) ⇒
  (let packed = fst (fst x);
    prefix-state = snd (fst packed);
    data = snd packed;
    attacker-state = snd (fst x);
    s =
      verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)
  in ∃ fr f-fri-roots f-final trace-table witness-final-state
    trace-openings.
    alpha-header-supported-partial-trace-table-candidate-witness
      s fr f-fri-roots f-final trace-table witness-final-state
      trace-openings ∧
    trace-table ∉
      alpha-prefix-trace-table-candidates prefix-state fr ∧
    prefix-state ≤ witness-final-state ∧
    (∃ j < scale * clength.
      ∀ i opn. i < rounds →
        opn ∈ set (trace-openings ! i) →
        opening-index opn ≠ j)))

```

definition *checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate*

```

:: ((((((f × 'f list × 'f list × 'f) ×
  'f protocol-channel) × 'f staged-proof-data) ×
  'f protocol-channel) × unit list) ×
  'f protocol-channel) option ⇒ bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
  out ⇔
  (case out of
    None ⇒ False
  | Some (x, -) ⇒
    (let packed = fst (fst x);
      prefix-state = snd (fst packed);
      data = snd packed;
      attacker-state = snd (fst x);
      s =
        verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)
    in ∃ fr f-fri-roots f-final trace-table witness-final-state
      trace-openings.
      alpha-header-supported-partial-trace-table-candidate-witness
        s fr f-fri-roots f-final trace-table witness-final-state
        trace-openings ∧
      trace-table ∉
        alpha-prefix-trace-table-candidates prefix-state fr ∧
      prefix-state ≤ witness-final-state ∧
      alpha-prefix-trace-table-candidates prefix-state fr = {}))

```

definition *checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output*

```

:: ((((((('f × 'f list × 'f list × 'f) ×
      'f protocol-channel) × 'f staged-proof-data) ×
      'f protocol-channel) × unit list) ×
      'f protocol-channel) option ⇒ bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
  out ⇔
  (case out of
    None ⇒ False
  | Some (x, -) ⇒
    (let packed = fst (fst x);
      prefix-state = snd (fst packed);
      data = snd packed;
      attacker-state = snd (fst x);
      s =
        verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)
    in ∃ fr f-fri-roots f-final trace-table witness-final-state
      trace-openings.
      alpha-header-supported-partial-trace-table-candidate-witness
        s fr f-fri-roots f-final trace-table witness-final-state
        trace-openings ∧
      trace-table ∉
        alpha-prefix-trace-table-candidates prefix-state fr ∧
      prefix-state ≤ witness-final-state ∧
      (∃ i < rounds. ∃ opn ∈ set (trace-openings ! i).
        hash-map-new-output-hit
          (merkle-path-target-roots witness-final-state fr
            (scale * clength) (opening-index opn)
            (opening-value opn) (opening-path opn))
          prefix-state witness-final-state)))

```

definition *checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad*

```

:: ((((((('f × 'f list × 'f list × 'f) ×
      'f protocol-channel) × 'f staged-proof-data) ×
      'f protocol-channel) × unit list) ×
      'f protocol-channel) option ⇒ bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
  out ⇔
  (case out of
    None ⇒ False
  | Some (x, -) ⇒
    (let packed = fst (fst x);
      data = snd packed;
      attacker-state = snd (fst x);
      s =
        verifier-state-from-adversary attacker-state

```

(staged-proof-transcript data)
in alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
s))

definition *checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit*

:: ((((((f × 'f list × 'f list × 'f) ×
'f protocol-channel) × 'f staged-proof-data) ×
'f protocol-channel) × unit list) ×
'f protocol-channel) option ⇒ bool

where

checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
out ⇔
(case out of
None ⇒ False
| Some (x, -) ⇒
(let packed = fst (fst x);
data = snd packed;
attacker-state = snd (fst x);
s =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
in ∃ witness-result witness-state.
Some (witness-result, witness-state) ∈
set-dist (execute verify-monad s) ∧
(hash-map-output-values s ∩
set (staged-proof-transcript data) ≠ {} ∨
hash-map-new-output-hit (set (staged-proof-transcript data))
s witness-state)))

definition *checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit*

:: ((((((f × 'f list × 'f list × 'f) ×
'f protocol-channel) × 'f staged-proof-data) ×
'f protocol-channel) × unit list) ×
'f protocol-channel) option ⇒ bool

where

checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
out ⇔
(case out of
None ⇒ False
| Some (x, -) ⇒
(let packed = fst (fst x);
data = snd packed;
attacker-state = snd (fst x);
s =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
in hash-map-output-values s ∩
set (staged-proof-transcript data) ≠ {}))

definition *checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit*

$$\begin{aligned} &:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times \\ &\quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times \\ &\quad 'f \text{ protocol-channel}) \times \text{unit list}) \times \\ &\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool} \end{aligned}$$

where

checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit

$$\begin{aligned} &\text{out} \longleftrightarrow \\ &(\text{case out of} \\ &\quad \text{None} \Rightarrow \text{False} \\ &| \text{Some } (x, -) \Rightarrow \\ &\quad (\text{let packed} = \text{fst } (\text{fst } x); \\ &\quad \quad \text{data} = \text{snd packed}; \\ &\quad \quad \text{attacker-state} = \text{snd } (\text{fst } x); \\ &\quad \quad s = \\ &\quad \quad \quad \text{verifier-state-from-adversary attacker-state} \\ &\quad \quad \quad (\text{staged-proof-transcript data}) \\ &\quad \text{in } \exists \text{ witness-result witness-state.} \\ &\quad \quad \text{Some } (\text{witness-result}, \text{witness-state}) \in \\ &\quad \quad \quad \text{set-dist } (\text{execute verify-monad } s) \wedge \\ &\quad \quad \quad \text{hash-map-new-output-hit } (\text{set } (\text{staged-proof-transcript data})) \\ &\quad \quad \quad s \text{ witness-state})) \end{aligned}$$

lemma *checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-imp-pre-or-new:*

assumes

checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit

$$\text{out}$$

shows

checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit

$$\text{out} \vee$$
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit

$$\text{out}$$

using *assms*

unfolding

checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-def
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-def
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit-def
by (*auto split: option.splits prod.splits*)

lemma *checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-imp-witness-transcript-hit:*

assumes

checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output

$$\text{out}$$

shows

checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit

$$\text{out}$$

proof (*cases out*)

case *None*

then show *?thesis*


```

using assms
unfolding
  checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-def
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-def
by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state
where out-eq:
  out =
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state)
by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from assms obtain fr f-fri-roots f-final trace-table witness-final-state
  trace-openings where witness:
  alpha-header-supported-partial-trace-table-candidate-witness ?s fr
  f-fri-roots f-final trace-table witness-final-state trace-openings
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-def
  Let-def
by (auto split: option.splits prod.splits)
from witness obtain witness-result query-idxs as dg composition-fri-roots
  final rest where outcome:
  Some (witness-result, witness-final-state) ∈
  set-dist (execute verify-monad ?s)
unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
by blast
have tr-eq: PTranscript ?s = staged-proof-transcript data
by simp
have transcript-hit:
  hash-map-output-values ?s ∩ set (staged-proof-transcript data)
  ≠ {} ∨
  hash-map-new-output-hit (set (staged-proof-transcript data))
  ?s witness-final-state
by (rule verify-monad-trace-root-target-preexisting-or-new
  [OF outcome tr-eq])
show ?thesis
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-def
  Let-def
using outcome transcript-hit
by (auto split: option.splits prod.splits)
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-le-witness-transcript-hit:
wp-event

```

```

    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
    adversary-initial-state ≤
wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
    adversary-initial-state
by (rule wp-event-mono)
    (rule
    checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-imp-witness-transcript-hit)

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-imp-witness-transcript-hit:
assumes
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
    out
shows
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
    out
proof (cases out)
case None
then show ?thesis
using assms
unfolding
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-def
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-def
by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state
where out-eq:
    out =
        Some (((((prefix, prefix-state), data), attacker-state), result),
            final-state)
by (cases packed, auto split: prod.splits)
let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from assms obtain fr f-fri-roots f-final trace-table witness-final-state
    trace-openings where witness:
    alpha-header-supported-partial-trace-table-candidate-witness ?s fr
    f-fri-roots f-final trace-table witness-final-state trace-openings
unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-def
    Let-def
by (auto split: option.splits prod.splits)
from witness obtain witness-result query-idxs as dg composition-fri-roots
    final rest where outcome:
    Some (witness-result, witness-final-state) ∈
    set-dist (execute verify-monad ?s)

```

```

unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
by blast
have tr-eq: PTranscript ?s = staged-proof-transcript data
by simp
have transcript-hit:
  hash-map-output-values ?s  $\cap$  set (staged-proof-transcript data)
     $\neq \{\}$   $\vee$ 
    hash-map-new-output-hit (set (staged-proof-transcript data))
      ?s witness-final-state
by (rule verify-monad-trace-root-target-preexisting-or-new
  [OF outcome tr-eq])
show ?thesis
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-def
  Let-def
using outcome transcript-hit
by (auto split: option.splits prod.splits)
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-le-witness-transcript-hit:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
    adversary-initial-state  $\leq$ 
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
    adversary-initial-state
by (rule wp-event-mono)
  (rule
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-imp-witness-transcript-hit)

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-imp-witnessed-cross-or-merkle-bad
assumes
  checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
  out
shows
  checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
  out
proof (cases out)
case None
then show ?thesis
  using assms
  unfolding
    checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state

```

```

where out-eq:
  out =
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from assms obtain fr f-fri-roots f-final trace-table witness-final-state
  trace-openings j where witness:
  alpha-header-supported-partial-trace-table-candidate-witness ?s fr
  f-fri-roots f-final trace-table witness-final-state trace-openings
and j-bound: j < scale * clength
and unopened:
   $\bigwedge i \text{ opn. } i < \text{rounds} \implies$ 
   $\text{opn} \in \text{set } (\text{trace-openings } ! i) \implies$ 
   $\text{opening-index opn} \neq j$ 
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-def
  Let-def
by (auto split: option.splits prod.splits)
from witness obtain result' query-idxs as dg composition-fri-roots final rest
where outcome:
  Some (result', witness-final-state) ∈ set-dist (execute verify-monad ?s)
and partial:
  accepted-with-partial-trace-openings ?s
  (Some (result', witness-final-state)) fr query-idxs trace-openings
and candidate: partial-trace-table-candidate trace-table trace-openings
and header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
by blast
have not-singleton:
   $\neg (\exists \text{trace-table0.}$ 
  alpha-header-supported-partial-trace-table-candidates ?s fr f-fri-roots
  f-final  $\subseteq \{\text{trace-table0}\})$ 
  by (rule
  alpha-header-supported-partial-trace-table-candidates-not-singleton-if-trace-unopened
  [OF outcome partial candidate header j-bound unopened])
have not-singleton-witnessed:
   $\neg (\exists \text{trace-table0.}$ 
  alpha-header-supported-witnessed-partial-trace-table-candidates ?s fr
  f-fri-roots f-final  $\subseteq \{\text{trace-table0}\})$ 
  using not-singleton
  by (simp add: alpha-header-supported-witnessed-candidates-eq-candidates)
have
  alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
  ?s fr f-fri-roots f-final

```

```

  by (rule
    alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-if-not-singleton
    [OF not-singleton-witnessed])
  then have
    alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad ?s
  unfolding alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-def
  by blast
  then show ?thesis
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad-def
    Let-def
  by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
    adversary-initial-state
  by (rule wp-event-mono)
  (rule
    checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-imp-witnessed-cross-or-merkle-bad)

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-imp-decomposition-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    out ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
            A)
          adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
    bad-sets out
  shows
    checked-staged-security-with-actual-alpha-prefix-not-prefix-collision out ∨
    checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
    out ∨
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
    out ∨
    checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
    out
  proof (cases out)

```

```

case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state
  where out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have hit-body:
   $\exists$  as query-idxs fr f-fri-roots f-final dg composition-fri-roots
    final rest.
    accepted-transcript-shape ?s (Some (result, final-state))
    as query-idxs  $\wedge$ 
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest  $\wedge$ 
    as  $\in$ 
    alpha-header-supported-partial-union-bad-sets ?s bad-sets fr
    f-fri-roots f-final  $\wedge$ 
    as  $\notin$  alpha-prefix-union-bad-sets prefix-state bad-sets fr  $\wedge$ 
    alpha-header-partial-candidate-not-prefix-bound prefix-state ?s fr
    f-fri-roots f-final
  using hit
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound-def
    Let-def
  by simp
from hit-body obtain as query-idxs fr f-fri-roots f-final dg
  composition-fri-roots final rest where bound:
  alpha-header-partial-candidate-not-prefix-bound prefix-state ?s fr
  f-fri-roots f-final
by blast
have support-some:
  Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state)  $\in$ 
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)
  using support out-eq by simp

```

```

from
  checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-decompose
  [OF wf controlled support-some bound]
show ?thesis
proof
  assume collision: hash-map-output-collision prefix-state
  have
    checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
    out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-def
    Let-def
  using collision by simp
  then show ?thesis by simp
next
assume witness-cases:
   $\exists$  trace-table witness-final-state trace-openings.
  alpha-header-supported-partial-trace-table-candidate-witness ?s fr
  f-fri-roots f-final trace-table witness-final-state
  trace-openings  $\wedge$ 
  trace-table  $\notin$ 
    alpha-prefix-trace-table-candidates prefix-state fr  $\wedge$ 
  prefix-state  $\leq$  witness-final-state  $\wedge$ 
  ( $\exists j < \text{scale} * \text{clength}$ .
     $\forall i \text{ opn.}$ 
       $i < \text{rounds} \longrightarrow$ 
       $\text{opn} \in \text{set } (\text{trace-openings} ! i) \longrightarrow$ 
       $\text{opening-index } \text{opn} \neq j) \vee$ 
    alpha-prefix-trace-table-candidates prefix-state fr = {}  $\vee$ 
    ( $\exists i < \text{rounds. } \exists \text{opn} \in \text{set } (\text{trace-openings} ! i).$ 
      hash-map-new-output-hit
      (merkle-path-target-roots witness-final-state fr
        (scale * clength) (opening-index opn)
        (opening-value opn) (opening-path opn))
      prefix-state witness-final-state))
from witness-cases obtain trace-table witness-final-state trace-openings
where witness:
  alpha-header-supported-partial-trace-table-candidate-witness ?s fr
  f-fri-roots f-final trace-table witness-final-state
  trace-openings
and not-prefix:
  trace-table  $\notin$ 
    alpha-prefix-trace-table-candidates prefix-state fr
and ext: prefix-state  $\leq$  witness-final-state
and cases:
  ( $\exists j < \text{scale} * \text{clength}$ .
     $\forall i \text{ opn.}$ 
       $i < \text{rounds} \longrightarrow$ 
       $\text{opn} \in \text{set } (\text{trace-openings} ! i) \longrightarrow$ 

```

```

      opening-index opn ≠ j) ∨
    alpha-prefix-trace-table-candidates prefix-state fr = {} ∨
    (∃ i < rounds. ∃ opn ∈ set (trace-openings ! i).
      hash-map-new-output-hit
        (merkle-path-target-roots witness-final-state fr
          (scale * clength) (opening-index opn)
          (opening-value opn) (opening-path opn))
        prefix-state witness-final-state)
  by blast
from cases show ?thesis
proof
  assume uncovered:
    ∃ j < scale * clength.
      ∀ i opn.
        i < rounds →
          opn ∈ set (trace-openings ! i) →
            opening-index opn ≠ j
  have body:
    ∃ fr' f-fri-roots' f-final' trace-table'
      witness-final-state' trace-openings'.
      alpha-header-supported-partial-trace-table-candidate-witness ?s
        fr' f-fri-roots' f-final' trace-table'
        witness-final-state' trace-openings' ∧
      trace-table' ∉
        alpha-prefix-trace-table-candidates prefix-state fr' ∧
      prefix-state ≤ witness-final-state' ∧
      (∃ j < scale * clength.
        ∀ i opn. i < rounds →
          opn ∈ set (trace-openings' ! i) →
            opening-index opn ≠ j)
  using witness not-prefix ext uncovered by blast
have
  checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
    out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-def
    Let-def
  using body by (simp split: prod.splits)
then show ?thesis by simp
next
  assume rest:
    alpha-prefix-trace-table-candidates prefix-state fr = {} ∨
    (∃ i < rounds. ∃ opn ∈ set (trace-openings ! i).
      hash-map-new-output-hit
        (merkle-path-target-roots witness-final-state fr
          (scale * clength) (opening-index opn)
          (opening-value opn) (opening-path opn))
        prefix-state witness-final-state)
  then show ?thesis

```



```

proof
  assume no-prefix:
    alpha-prefix-trace-table-candidates prefix-state fr = {}
  have body:
     $\exists fr' f\text{-fri-roots}' f\text{-final}' \text{trace-table}'$ 
      witness-final-state' trace-openings'.
    alpha-header-supported-partial-trace-table-candidate-witness ?s
      fr' f-fri-roots' f-final' trace-table'
      witness-final-state' trace-openings'  $\wedge$ 
      trace-table'  $\notin$ 
      alpha-prefix-trace-table-candidates prefix-state fr'  $\wedge$ 
      prefix-state  $\leq$  witness-final-state'  $\wedge$ 
      alpha-prefix-trace-table-candidates prefix-state fr' = {}
  using witness not-prefix ext no-prefix by blast
  have
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
      out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-def
    Let-def
  using body by (simp split: prod.splits)
  then show ?thesis by simp
next
  assume path:
     $\exists i < \text{rounds}. \exists \text{opn} \in \text{set} (\text{trace-openings} ! i).$ 
    hash-map-new-output-hit
      (merkle-path-target-roots witness-final-state fr
        (scale * clength) (opening-index opn)
        (opening-value opn) (opening-path opn))
      prefix-state witness-final-state
  have body:
     $\exists fr' f\text{-fri-roots}' f\text{-final}' \text{trace-table}'$ 
      witness-final-state' trace-openings'.
    alpha-header-supported-partial-trace-table-candidate-witness ?s
      fr' f-fri-roots' f-final' trace-table'
      witness-final-state' trace-openings'  $\wedge$ 
      trace-table'  $\notin$ 
      alpha-prefix-trace-table-candidates prefix-state fr'  $\wedge$ 
      prefix-state  $\leq$  witness-final-state'  $\wedge$ 
      ( $\exists i < \text{rounds}. \exists \text{opn} \in \text{set} (\text{trace-openings}' ! i).$ 
        hash-map-new-output-hit
          (merkle-path-target-roots witness-final-state' fr'
            (scale * clength) (opening-index opn)
            (opening-value opn) (opening-path opn))
          prefix-state witness-final-state')
  using witness not-prefix ext path by blast
  have
    checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
      out

```

```

    unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-def
      Let-def
    using body by (simp split: prod.splits)
  then show ?thesis by simp
qed
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound:
  fixes C U N P :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and collision-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
      adversary-initial-state  $\leq C$ 
  and uncovered-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
      adversary-initial-state  $\leq U$ 
  and no-prefix-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
      adversary-initial-state  $\leq N$ 
  and path-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
      adversary-initial-state  $\leq P$ 
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
        bad-sets)
      adversary-initial-state  $\leq C + U + N + P$ 
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?B =

```

```

    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
    bad-sets
  let ?C = checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
  let ?U =
    checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
  let ?N =
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
  let ?P = checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
  have wp-event ?M ?B adversary-initial-state ≤
    wp-event ?M (λout. ?C out ∨ ?U out ∨ ?N out ∨ ?P out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (rule
    checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-imp-decomposition-on-support
    [OF wf controlled])
  also have ... ≤
    wp-event ?M ?C adversary-initial-state +
    wp-event ?M ?U adversary-initial-state +
    wp-event ?M ?N adversary-initial-state +
    wp-event ?M ?P adversary-initial-state
  by (rule wp-event-union-bound4)
  also have ... ≤ C + U + N + P
  by (intro add-mono collision-bound uncovered-bound no-prefix-bound
    path-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-imp-not-prefix-bound-on-support
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  and drift:
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
    bad-sets out
  shows
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
    bad-sets out
proof (cases out)
case None
then show ?thesis
using drift
unfolding
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def

```

```

    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-def
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-def
  by simp
next
case (Some packed-out)
then obtain prefix prefix-state data attacker-state result final-state
where out-eq:
  out =
    Some (((((prefix, prefix-state), data), attacker-state), result),
          final-state)
  by (cases packed-out, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have partial-hit:
  checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
  bad-sets out
using drift
unfolding
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def
  checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-def
by simp
have not-prefix-event:
  ¬ checked-staged-security-with-actual-alpha-prefix-bad-set-hit
  bad-sets out
using drift
unfolding
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def
by simp
have builder:
  Some (data, attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-program A)
   adversary-initial-state)
proof -
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
     adversary-initial-state)
  by (rule
      checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
      [OF support[unfolded out-eq]])
  from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
  show ?thesis
  by blast
qed
have staged-header:

```

```

verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
have prefix-root-eq: staged-trace-root data = fst prefix
  using support out-eq
  unfolding checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
    checked-staged-transcript-with-alpha-prefix-program-def
    checked-staged-after-alpha-prefix-program-def Let-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have alpha-header-hit:
  alpha-header-list-set-hit ?s
    (alpha-header-supported-partial-union-bad-sets ?s bad-sets)
    (Some (result, final-state))
  using partial-hit unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-def
  by (simp add: Let-def)
from alpha-header-hit obtain as query-idxs fr f-fri-roots f-final dg
  composition-fri-roots final rest where shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as query-idxs
and header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and hit:
  as ∈
    alpha-header-supported-partial-union-bad-sets ?s bad-sets fr
    f-fri-roots f-final
  unfolding alpha-header-list-set-hit-def by blast
have as-eq: as = staged-alphas data
  using verifier-header-transcript-unique[OF staged-header header]
  by simp
have fr-eq: fr = staged-trace-root data
  using verifier-header-transcript-unique[OF header staged-header]
  by simp
have not-prefix:
  as ∉ alpha-prefix-union-bad-sets prefix-state bad-sets fr
proof
  assume as-prefix:
    as ∈ alpha-prefix-union-bad-sets prefix-state bad-sets fr
  have
    staged-alphas data ∈
      alpha-prefix-union-bad-sets prefix-state bad-sets (fst prefix)

```

```

    using as-prefix as-eq fr-eq prefix-root-eq by simp
  then show False
    using not-prefix-event unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-bad-set-hit-def
    by simp
qed
have not-bound:
  alpha-header-partial-candidate-not-prefix-bound prefix-state ?s fr
  f-fri-roots f-final
  by (rule alpha-header-partial-union-hit-not-prefix-hit-imp-not-prefix-bound
      [OF hit not-prefix])
  show ?thesis
    unfolding out-eq
    by (rule
        checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-boundI
        [OF refl shape header hit not-prefix not-bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-le-not-prefix-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
        bad-sets)
      adversary-initial-state ≤
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
        bad-sets)
      adversary-initial-state
  by (rule wp-event-mono-on-support)
  (rule
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-imp-not-prefix-bound-on-support
    [OF wf controlled])

lemma checked-staged-security-with-data-state-composition-bad-bound-from-candidate-not-prefix-and-budgets:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and not-prefix-bound:
      wp-event
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
          composition-trace-bad-alpha-space)
        adversary-initial-state ≤ D
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)

```

(staged-security-with-data-state-verifier-event composition-bad)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)

proof –

have *drift-bound*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space)
adversary-initial-state $\leq D$

proof –

have *wp-event*
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)
adversary-initial-state

by (rule
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-le-not-prefix-bound
[OF wf controlled])

also have ... $\leq D$

by (rule *not-prefix-bound*)

finally show *?thesis* .

qed

show *?thesis*

by (rule
checked-staged-security-with-data-state-composition-bad-bound-from-partial-header-candidate-drift-and-budgets
[OF wf controlled drift-bound])

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-not-prefix-components-and-budgets*:

fixes *C U N P* :: *prob*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *collision-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
 adversary-initial-state $\leq C$
and *uncovered-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
 adversary-initial-state $\leq U$
and *no-prefix-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
 adversary-initial-state $\leq N$
and *path-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
 adversary-initial-state $\leq P$
shows
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event composition-bad*)
 adversary-initial-state $\leq$
 composition-error-bound +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 (*C + U + N + P*) +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget + verifier-hash-query-budget)
proof –
have *not-prefix-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq C + U + N + P$

by (rule
 checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
 [*OF wf controlled collision-bound uncovered-bound no-prefix-bound*
 path-bound])
 show ?thesis
 by (rule
 checked-staged-security-with-data-state-composition-bad-bound-from-candidate-not-prefix-and-budgets
 [*OF wf controlled not-prefix-bound*])
 qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-not-prefix-noncollision-components-and-*
fixes *U N P :: prob*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *uncovered-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
 adversary-initial-state $\leq U$
and *no-prefix-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
 adversary-initial-state $\leq N$
and *path-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
 adversary-initial-state $\leq P$
shows
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event composition-bad*)
 adversary-initial-state $\leq$
 composition-error-bound +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 (*hash-collision-budget-value 0*
 (*staged-alpha-search-queries budgets 0*) + *U* + *N* + *P*) +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*)
proof (rule

checked-staged-security-with-data-state-composition-bad-bound-from-not-prefix-components-and-budgets
 $[OF \text{ wf controlled - uncovered-bound no-prefix-bound path-bound}]$)

show

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
adversary-initial-state $\leq$
hash-collision-budget-value 0 (*staged-alpha-search-queries* budgets 0)
by (rule
 checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
 $[OF \text{ wf controlled}]$)

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-cross-no-prefix-path-components-and-bud*

fixes *X N P :: prob*

assumes *wf: staged-budget-wellformed* budgets

and *controlled: staged-adversary-controlled* budgets *A*

and *cross-bound:*

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state $\leq$ *X*

and *no-prefix-bound:*

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq$ *N*

and *path-bound:*

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq$ *P*

shows

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event composition-bad*)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* budgets 0) +
 (*hash-collision-budget-value* 0
 (*staged-alpha-search-queries* budgets 0) + *X* + *N* + *P*) +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +

hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)

proof (*rule*
checked-staged-security-with-data-state-composition-bad-bound-from-not-prefix-noncollision-components-and-budget
[OF wf controlled - no-prefix-bound path-bound])

show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state ≤ X

by (*rule order-trans*
[OF
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad
cross-bound])

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-cross-no-prefix-transcript-components-and*
fixes *X N T :: prob*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *cross-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state ≤ X
and *no-prefix-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state ≤ N
and *transcript-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
adversary-initial-state ≤ T

shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event composition-bad)
adversary-initial-state ≤
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +

$(\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) + X + N + T) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets } +$
 $\text{staged-challenge-query-budget } + \text{verifier-hash-query-budget})$

proof (rule
 $\text{checked-staged-security-with-data-state-composition-bad-bound-from-cross-no-prefix-path-components-and-budgets}$
 $[\text{OF wf controlled cross-bound no-prefix-bound}]$)

show
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output}$
 $\text{adversary-initial-state} \leq T$

by (rule order-trans
 $[\text{OF}$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-le-witness-transcript-hit}$
 $\text{transcript-bound}]$)

qed

lemma $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new:}$
assumes pre-bound:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$

and new-bound:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit}$
 $\text{adversary-initial-state} \leq Q$

shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit}$
 $\text{adversary-initial-state} \leq P + Q$

proof –
have event-le:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit}$
 $\text{adversary-initial-state} \leq$
 wp-event

```

    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (λout.
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
        out ∨
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
        out)
    adversary-initial-state
  by (rule wp-event-mono)
    (rule
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-imp-pre-or-new)
  have union-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (λout.
        checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
          out ∨
        checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
          out)
      adversary-initial-state ≤
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
      adversary-initial-state +
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
      adversary-initial-state
  by (rule wp-event-union-bound)
  have sum-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
      adversary-initial-state +
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
      adversary-initial-state ≤ P + Q
  by (intro add-mono pre-bound new-bound)
  show ?thesis
  by (rule order-trans[OF event-le])
    (rule order-trans[OF union-bound sum-bound])
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-imp-data-state-pre-hit:*

assumes

checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
*(Some (((((prefix, prefix-state), data), attacker-state), result),
final-state)))*

shows

staged-security-with-data-state-transcript-pre-hit
(Some (((data, attacker-state), result), final-state))

using *assms*

unfolding

checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-def
staged-security-with-data-state-transcript-pre-hit-def

by *simp*

lemma *checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit:*

assumes *pre-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state ≤ P

shows

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
adversary-initial-state ≤ P

proof –

let *?project =*

$\lambda x. ((snd (fst (fst x)), snd (fst x)), snd x)$

have *projection:*

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state =
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(λout. case out of
None ⇒ staged-security-with-data-state-transcript-pre-hit None
| Some (packed, t) ⇒
staged-security-with-data-state-transcript-pre-hit
(Some (?project packed, t)))
adversary-initial-state

proof –

have

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state =
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
 $\gg (\lambda x. return (?project x))$
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state

```

using
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
  [of A]
by simp
also have ... =
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ( $\lambda out. \text{case out of}$ 
    None  $\Rightarrow$  staged-security-with-data-state-transcript-pre-hit None
    | Some (packed, t)  $\Rightarrow$ 
      staged-security-with-data-state-transcript-pre-hit
      (Some (?project packed, t)))
  adversary-initial-state
by (rule wp-event-bind-return-map)
finally show ?thesis .
qed
have event-le:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
  adversary-initial-state  $\leq$ 
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ( $\lambda out. \text{case out of}$ 
    None  $\Rightarrow$  staged-security-with-data-state-transcript-pre-hit None
    | Some (packed, t)  $\Rightarrow$ 
      staged-security-with-data-state-transcript-pre-hit
      (Some (?project packed, t)))
  adversary-initial-state
proof (rule wp-event-mono)
fix out
assume hit:
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
  out
show
  ( $\text{case out of}$ 
    None  $\Rightarrow$  staged-security-with-data-state-transcript-pre-hit None
    | Some (packed, t)  $\Rightarrow$ 
      staged-security-with-data-state-transcript-pre-hit
      (Some (?project packed, t)))
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-def
  by simp
next
case (Some packed)

```

```

then obtain prefix prefix-state data attacker-state result final-state
where out-eq:
  out =
    Some ((((((prefix, prefix-state), data), attacker-state), result),
      final-state)
  by (cases packed) (auto split: prod.splits)
show ?thesis
using
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-imp-data-state-pre-hit
  [OF hit[unfolded out-eq]]
unfolding out-eq by simp
qed
qed
show ?thesis
by (rule order-trans[OF event-le])
  (use pre-bound projection in simp)
qed

definition checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
:: (((((('f × 'f list × 'f list × 'f) ×
  'f protocol-channel) × 'f staged-proof-data) ×
  'f protocol-channel) × unit list) ×
  'f protocol-channel) option ⇒ bool
where
  checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
    out ⇔
  (case out of
    None ⇒ False
  | Some ((((((prefix, prefix-state), data), attacker-state), result),
    final-state) ⇒
    staged-security-with-data-state-transcript-pre-hit
      (Some (((data, attacker-state), result), final-state)) ∨
    staged-security-with-data-state-transcript-new-hit
      (Some (((data, attacker-state), result), final-state))))

lemma checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit-bound-from-data-state:
assumes pre-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-transcript-pre-hit
  adversary-initial-state ≤ P
and new-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-transcript-new-hit
  adversary-initial-state ≤ R
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
  adversary-initial-state ≤ P + R

```



```

proof –
  let ?project =
     $\lambda x. ((\text{snd } (\text{fst } (\text{fst } x)), \text{snd } (\text{fst } x)), \text{snd } x)$ 
  let ?E =
     $\lambda out.$ 
      staged-security-with-data-state-transcript-pre-hit out  $\vee$ 
      staged-security-with-data-state-transcript-new-hit out
  let ?projected =
     $\lambda out.$  case out of
      None  $\Rightarrow$  ?E None
    | Some (packed, t)  $\Rightarrow$  ?E (Some (?project packed, t))
  have projected-eq:
    ?projected =
      checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit-def
    staged-security-with-data-state-transcript-pre-hit-def
    staged-security-with-data-state-transcript-new-hit-def
  by (rule ext) (auto split: option.splits prod.splits)
  have projection:
    wp-event (checked-staged-security-experiment-with-data-state A) ?E
    adversary-initial-state =
      wp-event
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
        checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
        adversary-initial-state
  proof –
    have
      wp-event (checked-staged-security-experiment-with-data-state A) ?E
      adversary-initial-state =
        wp-event
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
             $\gg (\lambda x. \text{return } (?project x))$ )
          ?E adversary-initial-state
    using
      checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
      [of A]
    by simp
  also have ... =
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      ?projected adversary-initial-state
    by (rule wp-event-bind-return-map)
  finally show ?thesis
    unfolding projected-eq .
qed
have union-bound:
  wp-event (checked-staged-security-experiment-with-data-state A) ?E
  adversary-initial-state  $\leq$ 

```

```

wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-transcript-pre-hit
  adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-transcript-new-hit
  adversary-initial-state
by (rule wp-event-union-bound)
have
  wp-event (checked-staged-security-experiment-with-data-state A) ?E
  adversary-initial-state  $\leq P + R$ 
by (rule order-trans[OF union-bound])
  (intro add-mono pre-bound new-bound)
then show ?thesis
  unfolding projection .
qed

lemma checked-staged-security-with-actual-alpha-prefix-some-support-imp-sampled-transcript-hit:
  assumes support:
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state)  $\in$ 
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  shows
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
      (Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state))
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have data-support:
    Some (((data, attacker-state), result), final-state)  $\in$ 
    set-dist
      (execute
        (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
      [OF support])
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF data-support]
  have verifier:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)
  by blast
  have transcript-split:
    hash-map-output-values ?s  $\cap$  set (staged-proof-transcript data)

```

```

    ≠ {} ∨
    hash-map-new-output-hit (set (staged-proof-transcript data))
    ?s final-state
  by (rule verify-monad-trace-root-target-preexisting-or-new[OF verifier])
    simp
show ?thesis
unfolding
  checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit-def
  staged-security-with-data-state-transcript-pre-hit-def
  staged-security-with-data-state-transcript-new-hit-def
using transcript-split by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-bound-from-sampled-transcript
  assumes pre-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state ≤ P
  and new-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-new-hit
    adversary-initial-state ≤ R
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
      adversary-initial-state ≤ P + R
proof -
  have event-le:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
      adversary-initial-state ≤
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
      adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
        adversary-initial-state)
    and hit:
      checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
      out
  show

```

```

    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
    out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state data attacker-state result final-state
    where out-eq:
      out =
        Some (((((prefix, prefix-state), data), attacker-state), result),
              final-state)
    by (cases packed) (auto split: prod.splits)
  show ?thesis
    unfolding out-eq
    by (rule
      checked-staged-security-with-actual-alpha-prefix-some-support-imp-sampled-transcript-hit)
      (rule support[unfolded out-eq])
qed
qed
have transcript-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
    adversary-initial-state  $\leq P + R$ 
  by (rule
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit-bound-from-data-state
    [OF pre-bound new-bound])
  show ?thesis
    by (rule order-trans[OF event-le transcript-bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-bound-from-sampled-transcript:
  assumes pre-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state  $\leq P$ 
  and new-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-new-hit
    adversary-initial-state  $\leq R$ 
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output

```

$adversary-initial-state \leq P + R$
proof –
have *event-le*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq$
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
adversary-initial-state
proof (rule *wp-event-mono-on-support*)
fix *out*
assume *support*:
out $\in$
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
adversary-initial-state)
and *hit*:
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
out
show
checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
out
proof (*cases out*)
case *None*
then show *?thesis*
using *hit*
unfolding
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-def
by *simp*
next
case (*Some packed*)
then obtain *prefix prefix-state data attacker-state result final-state*
where *out-eq*:
out =
Some (((((prefix, prefix-state), data), attacker-state), result),
final-state)
by (*cases packed*) (*auto split: prod.splits*)
show *?thesis*
unfolding *out-eq*
by (rule
checked-staged-security-with-actual-alpha-prefix-some-support-imp-sampled-transcript-hit)
(rule support[unfolded out-eq])
qed
qed
have *transcript-bound*:
wp-event

```

    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
    adversary-initial-state  $\leq P + R$ 
  by (rule
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit-bound-from-data-state
    [OF pre-bound new-bound])
  show ?thesis
  by (rule order-trans[OF event-le transcript-bound])
qed

```

```

lemma checked-staged-security-with-data-state-transcript-new-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-new-hit
    adversary-initial-state  $\leq$  staged-concrete-transcript-target-error-bound
proof -
  let ?Builder = checked-staged-transcript-program A
  let ?Tail =
     $\lambda$ data s.
      wp-event verify-monad
        (hash-new-output-hit-event (set (staged-proof-transcript data))
          (verifier-state-from-adversary s (staged-proof-transcript data)))
        (verifier-state-from-adversary s (staged-proof-transcript data))
  have new-none[simp]:  $\neg$  staged-security-with-data-state-transcript-new-hit None
  unfolding staged-security-with-data-state-transcript-new-hit-def by simp
  have return-eq:
    wp (return ((data, s), result))
    ( $\lambda$ out.
      if staged-security-with-data-state-transcript-new-hit out then 1
      else 0)
    final-state =
    (if hash-map-new-output-hit (set (staged-proof-transcript data))
      (verifier-state-from-adversary s (staged-proof-transcript data))
      final-state then 1 else 0)
  for data s result final-state
  unfolding staged-security-with-data-state-transcript-new-hit-def
  by (simp add: wpsimps)
  have verify-post-eq:
    ( $\lambda$ r. case r of
      None  $\Rightarrow$  0
    | Some (result, final-state)  $\Rightarrow$ 
      wp (return ((data, s), result))
      ( $\lambda$ out.
        if staged-security-with-data-state-transcript-new-hit out
        then 1 else 0)
      final-state) =
    ( $\lambda$ r.

```

```

    if hash-new-output-hit-event (set (staged-proof-transcript data))
      (verifier-state-from-adversary s (staged-proof-transcript data)) r
    then 1 else 0)
  for data s
proof (rule ext)
  fix r
  show (case r of
    None  $\Rightarrow$  0
  | Some (result, final-state)  $\Rightarrow$ 
    wp (return ((data, s), result))
      ( $\lambda$ out.
        if staged-security-with-data-state-transcript-new-hit out
        then 1 else 0)
    final-state) =
    (if hash-new-output-hit-event (set (staged-proof-transcript data))
      (verifier-state-from-adversary s (staged-proof-transcript data)) r
    then 1 else 0)
  proof (cases r)
    case None
    then show ?thesis
      unfolding hash-new-output-hit-event-def by simp
  next
    case (Some packed)
    then obtain result final-state where
      packed-eq: packed = (result, final-state)
    by (cases packed) simp
  show ?thesis
    using return-eq[of data s result final-state]
    unfolding Some packed-eq hash-new-output-hit-event-def
    by simp
qed
qed
have tail-eq:
  wp
    (get  $\gg$  ( $\lambda$ s.
      put (verifier-state-from-adversary s
        (staged-proof-transcript data))  $\gg$ 
      ( $\lambda$ -. verify-monad  $\gg$ 
        ( $\lambda$ result. return ((data, s), result))))))
    ( $\lambda$ out.
      if staged-security-with-data-state-transcript-new-hit out then 1
      else 0)
  s =
  ?Tail data s
  for data s
  unfolding wp-event-def
  by (simp add: wpsimps verify-post-eq)
have post-eq:
  ( $\lambda$ out. case out of

```

```

None  $\Rightarrow$  0
| Some (x, y)  $\Rightarrow$ 
  wp
  (get  $\gg$  ( $\lambda s$ .
    put (verifier-state-from-adversary s
      (staged-proof-transcript x))  $\gg$ 
    ( $\lambda$ -. verify-monad  $\gg$ 
      ( $\lambda$ result. return ((x, s), result)))))
  ( $\lambda$ out.
    if staged-security-with-data-state-transcript-new-hit out
    then 1 else 0)
  y) =
( $\lambda$ out. case out of
  None  $\Rightarrow$  0
  | Some (data, s)  $\Rightarrow$  ?Tail data s)
by (rule ext) (simp add: tail-eq split: option.splits prod.splits)
have post-tail-unfold:
( $\lambda$ out. case out of
  None  $\Rightarrow$  0
  | Some (data, s)  $\Rightarrow$  ?Tail data s) =
( $\lambda$ out. case out of
  None  $\Rightarrow$  0
  | Some (data, s)  $\Rightarrow$ 
    wp verify-monad
    ( $\lambda x$ . if hash-new-output-hit-event
      (set (staged-proof-transcript data))
      (verifier-state-from-adversary s
        (staged-proof-transcript data))
      x then 1 else 0)
    (verifier-state-from-adversary s
      (staged-proof-transcript data)))
by (rule ext) (simp add: wp-event-def split: option.splits prod.splits)
have event-eq:
wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-transcript-new-hit
  adversary-initial-state =
wp ?Builder
( $\lambda$ out. case out of
  None  $\Rightarrow$  0
  | Some (data, s)  $\Rightarrow$  ?Tail data s)
  adversary-initial-state
unfolding checked-staged-security-experiment-with-data-state-def
  wp-event-def
by (simp add: wpsimps post-eq post-tail-unfold
  split: prod.splits option.splits)
have tail-bound:
 $\bigwedge$ out. out  $\in$  set-dist (execute ?Builder adversary-initial-state)  $\implies$ 
  (case out of None  $\Rightarrow$  0 | Some (data, s)  $\Rightarrow$  ?Tail data s)
   $\leq$  staged-concrete-transcript-target-error-bound

```



```

proof –
  fix out
  assume support:
    out ∈ set-dist (execute ?Builder adversary-initial-state)
  show (case out of None ⇒ 0 | Some (data, s) ⇒ ?Tail data s)
    ≤ staged-concrete-transcript-target-error-bound
  proof (cases out)
    case None
    then show ?thesis
      unfolding staged-concrete-transcript-target-error-bound-def by simp
  next
  case (Some packed)
  then obtain data s where packed-eq: packed = (data, s)
    by (cases packed) simp
  have outcome:
    Some (data, s) ∈
      set-dist (execute ?Builder adversary-initial-state)
    using support Some packed-eq by simp
  have verifier-bound:
    ?Tail data s ≤
      concrete-transcript-target-error (staged-proof-transcript data)
    using wp-verify-monad-hash-new-output-hit-bound
      [of set (staged-proof-transcript data)
        verifier-state-from-adversary s
        (staged-proof-transcript data)]
    unfolding concrete-transcript-target-error-def
      hash-target-budget-value-def
    by (simp add: mult.commute)
  have target-bound:
    concrete-transcript-target-error (staged-proof-transcript data)
      ≤ staged-concrete-transcript-target-error-bound
  proof –
    have len:
      length (staged-proof-transcript data) ≤
        staged-proof-transcript-length-bound
      by (rule
        checked-staged-transcript-program-staged-proof-transcript-length-bound
        [OF wf controlled outcome])
    show ?thesis
      unfolding staged-concrete-transcript-target-error-bound-def
      by (rule concrete-transcript-target-error-bound-from-length[OF len])
  qed
  show ?thesis
    using Some packed-eq order-trans[OF verifier-bound target-bound]
    by simp
  qed
qed
show ?thesis
  unfolding event-eq

```

by (rule wp-le-const-on-support[OF tail-bound])
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-cross-no-prefix-transcript-split-components*

fixes $X\ N\ P\ Q :: \text{prob}$

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *cross-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state $\leq X$

and *no-prefix-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq N$

and *pre-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
adversary-initial-state $\leq P$

and *new-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq Q$

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event composition-bad*)

adversary-initial-state $\leq$

composition-error-bound +

staged-phase-relation-error size

(*staged-alpha-search-queries budgets 0*) +

hash-collision-budget-value 0

(*staged-alpha-search-queries budgets 0*) +

(*hash-collision-budget-value 0*

(*staged-alpha-search-queries budgets 0*) + $X + N + (P + Q)$) +

staged-phase-relation-error size

(*staged-alpha-search-queries budgets 0*) +

hash-collision-budget-value 0

(*staged-attacker-query-budget budgets* +

staged-challenge-query-budget + *verifier-hash-query-budget*)

proof (rule

checked-staged-security-with-data-state-composition-bad-bound-from-cross-no-prefix-transcript-components-and-bud

$[OF \text{ wf controlled cross-bound no-prefix-bound}]$
show
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
adversary-initial-state $\leq P + Q$
by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
 $[OF \text{ pre-bound new-bound}]$)
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-cross-no-prefix-data-pre-and-witness-new*
fixes $X \ N \ P \ Q :: \text{prob}$
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *cross-bound:*
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state $\leq X$
and *no-prefix-bound:*
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq N$
and *data-pre-bound:*
wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *new-bound:*
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq Q$
shows
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event composition-bad*)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 (*hash-collision-budget-value 0*
 (*staged-alpha-search-queries budgets 0*) + $X + N + (P + Q)$) +

staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
proof (rule
checked-staged-security-with-data-state-composition-bad-bound-from-cross-no-prefix-transcript-split-components-and
[OF wf controlled cross-bound no-prefix-bound - new-bound])
show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
adversary-initial-state ≤ P
by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
[OF data-pre-bound])
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-cross-and-transcript-components-and-bu*
fixes *X T :: prob*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *cross-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state ≤ X
and *transcript-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
adversary-initial-state ≤ T
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event composition-bad)
adversary-initial-state ≤
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + X + T + T) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0

(staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget)
proof (rule
 checked-staged-security-with-data-state-composition-bad-bound-from-cross-no-prefix-transcript-components-and-bud
 [OF wf controlled cross-bound - transcript-bound])
show
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
 adversary-initial-state $\leq T$
by (rule order-trans
 [OF
 checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-le-witness-transcript-hit
 transcript-bound])
qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-cross-data-pre-and-witness-new-compone
fixes $X P Q :: prob$
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and cross-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
 adversary-initial-state $\leq X$
and data-pre-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-transcript-pre-hit
 adversary-initial-state $\leq P$
and new-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
 adversary-initial-state $\leq Q$
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event composition-bad)
 adversary-initial-state $\leq$
 composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 (hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + X + (P + Q) + (P + Q)) +
 staged-phase-relation-error size

```

      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget + verifier-hash-query-budget)
proof (rule
  checked-staged-security-with-data-state-composition-bad-bound-from-cross-and-transcript-components-and-budgets
  [OF wf controlled cross-bound])
show
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
  adversary-initial-state  $\leq P + Q$ 
proof (rule
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
  [OF - new-bound])
show
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
  adversary-initial-state  $\leq P$ 
by (rule
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
  [OF data-pre-bound])
qed
qed

end

end

```

theory *Staged-Security-Experiment-Composition-Query-Current-Alignment*
imports *Staged-Security-Experiment-Composition-Query-Current*
begin

Query-prefix alignment facts for the current-final-state composition-query route. This theory is intentionally narrow so that the already large current-query projection theory does not grow past the project size threshold.

context *soundness*
begin

lemma *checked-staged-security-experiment-with-query-prefix-data-state-outcomeI*:
assumes *prefix-receive*:
Some (((prefix, prefix-state), raw), raw-state) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)

```

and after:
  Some (data, attacker-state) ∈
    set-dist
      (execute
        (checked-staged-after-query-prefix-receive A i
          ((prefix, prefix-state), raw))
        raw-state)
and verifier:
  Some (result, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))))
shows
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
proof –
  have with-verifier:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-after-query-prefix-receive-with-verifier A i
            ((prefix, prefix-state), raw))
          raw-state)
    unfolding checked-staged-after-query-prefix-receive-with-verifier-def
    apply (rule-tac x=data and t=attacker-state in set-dist-bindI)
    apply (rule after)
    apply (rule-tac x=attacker-state and t=attacker-state in set-dist-bindI)
    apply simp
    apply (rule-tac x=() and
      t=verifier-state-from-adversary attacker-state
      (staged-proof-transcript data) in set-dist-bindI)
    apply simp
    apply (rule-tac x=result and t=final-state in set-dist-bindI)
    apply (rule verifier)
    apply simp
  done
show ?thesis
  unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
  apply (rule set-dist-bindI[OF prefix-receive])
  apply (rule-tac x=raw-state and t=raw-state in set-dist-bindI)
  apply simp
  apply (rule-tac x=((data, attacker-state), result) and
    t=final-state in set-dist-bindI)
  apply (rule with-verifier)

```

apply simp
 done
 qed

lemma *checked-staged-after-query-prefix-receive-alphas:*

assumes *after:*

Some (data, attacker-state) ∈
set-dist
(execute
(checked-staged-after-query-prefix-receive A i
((prefix, prefix-state), raw))
raw-state)

shows *staged-alphas data = sqp-alphas prefix*

using *after*

unfolding *checked-staged-after-query-prefix-receive-def Let-def*

by (*auto elim!: set-dist-bindE split: prod.splits*)

lemma *checked-staged-security-with-actual-alpha-prefix-support-query-prefix-data-stateE:*

assumes *i-bound: i < rounds*

and *support:*

Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)

obtains *prefix prefix-state raw raw-state where*

Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)

proof –

let *?s =*

verifier-state-from-adversary attacker-state
(staged-proof-transcript data)

have *data-support:*

Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute (checked-staged-security-experiment-with-data-state A)
adversary-initial-state)

by (*rule*

checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
[OF support])

from *checked-staged-security-experiment-with-data-state-outcomeE*

[OF data-support]

obtain *builder verifier where builder:*


```

    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈
      set-dist (execute verify-monad ?s)
  by blast
from checked-staged-transcript-program-query-prefix-receive-cont-support
  [OF i-bound builder]
obtain prefix prefix-state raw raw-state where prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  and after:
    Some (data, attacker-state) ∈
      set-dist
        (execute
          (checked-staged-after-query-prefix-receive A i
            ((prefix, prefix-state), raw))
          raw-state)
  by blast
have qprefix-support:
  Some ((((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
  by (rule checked-staged-security-experiment-with-query-prefix-data-state-outcomeI
    [OF prefix-receive after verifier])
show ?thesis
  by (rule that[OF qprefix-support])
qed

lemma checked-staged-security-with-actual-alpha-prefix-support-query-prefix-data-state-alphasE:
  assumes i-bound: i < rounds
  and support:
    Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
            A)
          adversary-initial-state)
  obtains prefix prefix-state raw raw-state where
    Some ((((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈

```

```

    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
    sqp-alphas prefix = staged-alphas data
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and verifier:
  Some (result, final-state) ∈
    set-dist (execute verify-monad ?s)
  by blast
from checked-staged-transcript-program-query-prefix-receive-cont-support
  [OF i-bound builder]
obtain prefix prefix-state raw raw-state where prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  and after:
  Some (data, attacker-state) ∈
    set-dist
      (execute
        (checked-staged-after-query-prefix-receive A i
          ((prefix, prefix-state), raw))
        raw-state)
  by blast
have qprefix-support:
  Some ((((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)

```

adversary-initial-state)
by (rule *checked-staged-security-experiment-with-query-prefix-data-state-outcomeI*
[OF prefix-receive after verifier])
have *alphas-eq: sqp-alphas prefix = staged-alphas data*
using *checked-staged-after-query-prefix-receive-alphas[OF after]*
by *simp*
show *?thesis*
by (rule *that[OF qprefix-support alphas-eq]*)
qed

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-pair-success-query-prefix-hitE*:
assumes *i-bound: i < rounds*
and *support*:
Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)
and *success*:
 $\bigwedge \text{prefix prefix-state raw raw-state.}$
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state
A i)
adversary-initial-state) ⇒
sqp-alphas prefix = staged-alphas data ⇒
index (to-nat raw) ∈
query-sampling-success-space trace-table composition-table
(staged-alphas data)
obtains *prefix prefix-state raw raw-state where*
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)
checked-staged-security-with-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))
proof –
from *checked-staged-security-with-actual-alpha-prefix-support-query-prefix-data-state-alphasE*
[OF i-bound support]
obtain *prefix prefix-state raw raw-state where qprefix-support*:

```

    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
  and alphas-eq: sqp-alphas prefix = staged-alphas data
  by blast
have hit:
  checked-staged-security-with-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table)
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  using success[OF qprefix-support alphas-eq] alphas-eq
  unfolding
    checked-staged-security-with-query-prefix-dynamic-index-hit-def
    checked-staged-query-prefix-dynamic-index-hit-def
    staged-query-prefix-candidate-pair-query-target-from-prefix-def
  by simp
show ?thesis
  by (rule that[OF qprefix-support hit])
qed

lemma checked-staged-actual-alpha-prefix-current-query-partial-opening-hit-at-imp-query-prefix-current-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and support:
    Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  and hit:
    staged-security-with-data-state-current-query-partial-opening-hit-at i
      (Some (((data, attacker-state), result), final-state))
  obtains prefix prefix-state raw raw-state where
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
    i
    (Some (((((prefix, prefix-state), raw), raw-state),

```

```

      ((data, attacker-state), result)), final-state))
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where builder:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
have decomp:
  checked-staged-transcript-program A =
    checked-staged-query-prefix-receive-with-state A i ≫=
    checked-staged-after-query-prefix-receive A i
  by (rule checked-staged-transcript-program-query-prefix-receive-decomp
    [OF i-bound])
from builder[unfolded decomp]
obtain prefix-out raw-state where prefix-receive:
  Some (prefix-out, raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
  and after:
    Some (data, attacker-state) ∈
      set-dist
      (execute
        (checked-staged-after-query-prefix-receive A i prefix-out)
        raw-state)
  by (elim set-dist-bindE)
obtain prefix prefix-state raw where prefix-out-eq:
  prefix-out = ((prefix, prefix-state), raw)
  by (cases prefix-out) auto
have qprefix-support:
  Some ((((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈

```

```

    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
  by (rule checked-staged-security-experiment-with-query-prefix-data-state-outcomeI)
    (use prefix-receive after verifier prefix-out-eq in simp-all)
have qprefix-hit:
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
  i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
  by (rule
    staged-security-with-data-state-current-query-partial-opening-hit-at-imp-query-prefix-current-on-support
    [OF wf controlled i-bound qprefix-support hit])
show ?thesis
  by (rule that[OF qprefix-support qprefix-hit])
qed

definition checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at
:: nat  $\Rightarrow$ 
  ((((((f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
    'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
    'f protocol-channel)  $\times$  unit list)  $\times$ 
    'f protocol-channel) option  $\Rightarrow$  bool
where
  checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at
  i out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state)  $\Rightarrow$ 
    staged-security-with-data-state-current-query-partial-opening-hit-at i
    (Some (((data, attacker-state), result), final-state))))

definition checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
:: ((((((f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
  'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
  'f protocol-channel)  $\times$  unit list)  $\times$ 
  'f protocol-channel) option  $\Rightarrow$  bool
where
  checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
  out  $\longleftrightarrow$ 
  ( $\exists i \in \{..<\text{rounds}\}$ .
    checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at
    i out)

lemma checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-imp-query-prefix-current
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A

```

```

and i-bound:  $i < \text{rounds}$ 
and support:
  out  $\in$ 
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at
    i out
shows
   $\exists \text{prefix prefix-state raw raw-state data attacker-state result final-state.}$ 
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)  $\in$ 
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)  $\wedge$ 
        checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
          i
          (Some (((((prefix, prefix-state), raw), raw-state),
            ((data, attacker-state), result)), final-state))
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-def
    by simp
next
  case (Some packed)
  then obtain alpha-prefix alpha-prefix-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state)
    by (cases packed) (auto split: prod.splits)
  have data-hit:
    staged-security-with-data-state-current-query-partial-opening-hit-at i
      (Some (((data, attacker-state), result), final-state))
    using hit
    unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-def
    by simp
  have support-some:
    Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state)  $\in$ 
      set-dist

```

```

      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
    using support out-eq by simp
  from checked-staged-actual-alpha-prefix-current-query-partial-opening-hit-at-imp-query-prefix-current-on-support
    [OF wf controlled i-bound support-some data-hit]
  obtain prefix prefix-state raw raw-state where qsupport:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
  and qhit:
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
      i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  by blast
  show ?thesis
    using qsupport qhit by blast
qed

lemma checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-bound-from-query-prefix-
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and query-prefix-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state ≤ Q
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state ≤ Q
proof -
  let ?project =
    λx. ((snd (fst (fst x)), snd (fst x)), snd x)
  have projection:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-query-partial-opening-hit-at i)

```



```

    adversary-initial-state
  proof -
  have
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-query-partial-opening-hit-at i)
    adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
         $\gg (\lambda x. \text{return } (?project\ x)))$ )
      (staged-security-with-data-state-current-query-partial-opening-hit-at i)
    adversary-initial-state
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
  also have ... =
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state
  proof -
  have pred-eq:
    ( $\lambda out. \text{case out of}$ 
      None  $\Rightarrow$ 
        staged-security-with-data-state-current-query-partial-opening-hit-at
          i None
      | Some (x, t)  $\Rightarrow$ 
        staged-security-with-data-state-current-query-partial-opening-hit-at
          i (Some (?project x, t))) =
      checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at
        i
    by (rule ext)
    (simp add:
      checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-def
      staged-security-with-data-state-current-query-partial-opening-hit-at-def
      split: option.splits prod.splits)
  show ?thesis
    by (subst wp-event-bind-return-map) (simp add: pred-eq)
  qed
  finally show ?thesis
    by simp
  qed
  have data-bound:
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-query-partial-opening-hit-at i)
    adversary-initial-state  $\leq Q$ 
  by (rule order-trans

```

```

    [OF
      checked-staged-security-with-data-state-current-query-partial-opening-hit-at-le-query-prefix-current
        query-prefix-bound])
    (use wf controlled i-bound in simp-all)
  show ?thesis
    unfolding projection by (rule data-bound)
qed

lemma checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-bound-from-query-prefix-current
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and round-bound:
     $\bigwedge i. i < \text{rounds} \implies$ 
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
        adversary-initial-state  $\leq C i$ 
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
      adversary-initial-state  $\leq (\sum i < \text{rounds}. C i)$ 
proof -
  have round-le:
     $\bigwedge i. i < \text{rounds} \implies$ 
      wp-event
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
        (checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at i)
        adversary-initial-state  $\leq C i$ 
  by (rule
      checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-bound-from-query-prefix-current
        [OF wf controlled - round-bound])
  have
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
      adversary-initial-state  $\leq$ 
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      ( $\lambda \text{out}. \exists i \in \{..<\text{rounds}\}.$ 
        checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at
          i out)
      adversary-initial-state
  by (rule wp-event-mono)
  (simp add:
    checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-def)
  also have ...  $\leq (\sum i < \text{rounds}. C i)$ 
  by (rule wp-event-finite-UN-bound)
  (simp-all add: round-le)

```

finally show ?thesis .
qed

lemma *checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-projection:*

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
 adversary-initial-state =
wp-event
 (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-current-query-partial-opening-hit
 adversary-initial-state

proof –

let ?project =
 $\lambda x. ((\text{snd } (\text{fst } (\text{fst } x)), \text{snd } (\text{fst } x)), \text{snd } x)$

have

wp-event
 (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-current-query-partial-opening-hit
 adversary-initial-state =
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
 $\gg (\lambda x. \text{return } (?project x))$)
 staged-security-with-data-state-current-query-partial-opening-hit
 adversary-initial-state

using

checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
 [of A]

by simp

also have ... =

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
 adversary-initial-state

proof –

have pred-eq:

($\lambda out. \text{case out of}$
 None $\Rightarrow$
 staged-security-with-data-state-current-query-partial-opening-hit
 None

| Some (x, t) $\Rightarrow$
 staged-security-with-data-state-current-query-partial-opening-hit
 (Some (?project x, t))) =

checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit

by (rule ext)

(auto simp:

checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-def
 checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-def
 staged-security-with-data-state-current-query-partial-opening-hit-def

```

      staged-security-with-data-state-current-query-partial-opening-hit-at-def
      split: option.splits prod.splits)
  show ?thesis
    by (subst wp-event-bind-return-map) (simp add: pred-eq)
qed
finally show ?thesis
  by simp
qed

lemma checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-actual-alpha-current-qu
assumes bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
  adversary-initial-state  $\leq Q$ 
shows
  wp-event
  (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-current-query-partial-opening-hit
  adversary-initial-state  $\leq Q$ 
using bound
unfolding
  checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-projection
by simp

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-current-query-part
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out  $\in$ 
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  out
shows
  checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
  out
proof (cases out)
case None
then show ?thesis
  using hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def

```

```

    checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-def
    checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-def
  by simp
next
case (Some packed)
then obtain alpha-prefix alpha-prefix-state data attacker-state result
  final-state where out-eq:
  out =
    Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state)
  by (cases packed) (auto split: prod.splits)
have support-some:
  Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state) ∈
  set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  using support out-eq by simp
have hit-some:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  using hit out-eq by simp
have data-hit:
  staged-security-with-data-state-current-query-partial-opening-hit
    (Some (((data, attacker-state), result), final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-current-par
    [OF wf controlled support-some hit-some])
then obtain i where i-bound: i < rounds
and round-hit:
  staged-security-with-data-state-current-query-partial-opening-hit-at i
    (Some (((data, attacker-state), result), final-state))
unfolding staged-security-with-data-state-current-query-partial-opening-hit-def
by blast
have actual-round:
  checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at
    i
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  using round-hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-def
by simp
show ?thesis
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-def

```

using *i-bound actual-round* by *blast*
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-actual-alpha*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *current-query-bound:*

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit

adversary-initial-state ≤ Q

shows

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

adversary-initial-state ≤ Q

by *(rule order-trans[*

OF - current-query-bound])

(rule wp-event-mono-on-support,

rule checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-current-query-part

[OF wf controlled])

lemma *checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-aligned-transcript-classif*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),

result), final-state) ∈

set-dist

(execute

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

A)

adversary-initial-state)

and *partial:*

accepted-with-partial-initial-openings-aligned-transcript-consistent

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

(Some (result, final-state)) fr f-fri-roots f-final as dg

composition-fri-roots final query-idxs trace-openings

composition-openings

and *no-bad:*

¬ partial-merkle-inconsistency-bad

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

(Some (result, final-state))

and *hit:*

checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at

i

(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),

result), *final-state*))
shows
trace-fri-bad-with-partial-candidates
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result, final-state)) ∨
composition-fri-bad-with-partial-candidates
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result, final-state)) ∨
query-bad-with-partial-candidates
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result, final-state)) ∨
 (∃ *trace-table* *composition-table* *trace-openings-i*
composition-openings-i.
partial-trace-table-candidate *trace-table*
 (trace-openings[i := trace-openings-i]) ∧
partial-composition-table-candidate *composition-table*
 (composition-openings[i := composition-openings-i]) ∧
trace-table-low-degree *trace-table* ∧
composition-table-low-degree *maxDegree* *composition-table* ∧
all-queries-consistent *trace-table* *composition-table* as)
proof –
have *data-support*:
 Some (((data, attacker-state), result), final-state) ∈
 set-dist
 (execute
 (checked-staged-security-experiment-with-data-state A)
 adversary-initial-state)
by (rule
 checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
 [OF support])
have *data-hit*:
 staged-security-with-data-state-current-query-partial-opening-hit-at i
 (Some (((data, attacker-state), result), final-state))
using *hit*
unfolding
 checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-def
by *simp*
show ?thesis
by (rule
 staged-security-with-data-state-current-query-partial-opening-hit-at-aligned-transcript-classification-on-support
 [OF wf controlled data-support partial no-bad data-hit])
qed
lemma *checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-aligned-transcript-classification-on-support*
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets A

and *support*:
Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state) $\in$
set-dist
 (execute
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 adversary-initial-state)
and *partial*:
accepted-with-partial-initial-openings-aligned-transcript-consistent
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result, final-state)) fr f-fri-roots f-final as dg
 composition-fri-roots final query-idxs trace-openings
 composition-openings
and *no-bad*:
 $\neg$ *partial-merkle-inconsistency-bad*
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result, final-state))
and *hit*:
checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))
shows
trace-fri-bad-with-partial-candidates
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result, final-state)) $\vee$
composition-fri-bad-with-partial-candidates
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result, final-state)) $\vee$
query-bad-with-partial-candidates
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result, final-state)) $\vee$
 ($\exists i$ trace-table composition-table trace-openings-*i*
 composition-openings-*i*.
i < rounds $\wedge$
 partial-trace-table-candidate trace-table
 (trace-openings[*i* := trace-openings-*i*]) $\wedge$
 partial-composition-table-candidate composition-table
 (composition-openings[*i* := composition-openings-*i*]) $\wedge$
 trace-table-low-degree trace-table $\wedge$
 composition-table-low-degree maxDegree composition-table $\wedge$
 all-queries-consistent trace-table composition-table as)

proof –
from *hit* **obtain** *i* **where** *i*-bound: *i* < rounds

and *round-hit*:
checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at
i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
unfolding
checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-def
by *blast*
have *classified*:
trace-fri-bad-with-partial-candidates
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result, final-state)) ∨
composition-fri-bad-with-partial-candidates
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result, final-state)) ∨
query-bad-with-partial-candidates
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result, final-state)) ∨
 $(\exists \text{ trace-table composition-table trace-openings-}i$
 $\text{composition-openings-}i.$
partial-trace-table-candidate trace-table
 $(\text{trace-openings}[i := \text{trace-openings-}i]) \wedge$
partial-composition-table-candidate composition-table
 $(\text{composition-openings}[i := \text{composition-openings-}i]) \wedge$
trace-table-low-degree trace-table $\wedge$
composition-table-low-degree maxDegree composition-table $\wedge$
all-queries-consistent trace-table composition-table as)
by *(rule*
checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-aligned-transcript-classificat
 $[OF \text{ wf controlled support partial no-bad round-hit}]$
then show *?thesis*
using *i-bound* **by** *blast*
qed

lemma *checked-staged-actual-alpha-prefix-query-prefix-receive-alignment-from-final-lookup*:

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *support*:

Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) ∈

set-dist

(execute

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)

adversary-initial-state)

```

and final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw
obtains prefix prefix-state raw-state where
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  sqp-alphas prefix = staged-alphas data
  sqp-query-chunks prefix = take i (staged-query-chunks data)
  PQueryCounter prefix-state = i
  PState prefix-state =
    state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
    by (rule
      checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
      [OF support])
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF data-support]
  obtain builder verifier where builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
    and verifier:
      Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
    by blast
  show ?thesis
  proof (rule checked-staged-transcript-program-query-prefix-receive-support
    [OF wf controlled i-bound builder])
  fix prefix prefix-state raw' raw-state
  assume prefix-receive:
    Some (((prefix, prefix-state), raw'), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)

```

```

and alphas-eq: sqp-alphas prefix = staged-alphas data
and chunks-eq:
  sqp-query-chunks prefix = take i (staged-query-chunks data)
and counter-eq: PQueryCounter prefix-state = i
and state-eq:
  PState prefix-state =
    state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i
and raw-ext: raw-state ≤ attacker-state
and raw-lookup:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw'
have raw-ext-s: raw-state ≤ ?s
  by (rule hash-extends-verifier-state-from-adversary-right[OF raw-ext])
have s-ext-final: ?s ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
have raw-ext-final: raw-state ≤ final-state
  by (rule hash-ext-trans[OF raw-ext-s s-ext-final])
have final-lookup-raw':
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw'
  by (rule hash-extension-lookup[OF raw-lookup raw-ext-final])
have raw-eq: raw' = raw
  using final-lookup final-lookup-raw' by simp
have prefix-receive-raw:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  using prefix-receive raw-eq by simp
show ?thesis
  by (rule that[OF prefix-receive-raw alphas-eq chunks-eq counter-eq
    state-eq])
qed
qed

end

end

```

```

theory Soundness-Conceptual-Table
imports Soundness-Partial-Merkle

```

begin

Conceptual Merkle tables.

This layer deliberately does not reconstruct a complete *created-tree* witness from sampled openings. Instead it records the value that is uniquely authenticated for a root, length, and index in a hash-map state. Values at unauthenticated indices are arbitrary Hilbert-choice defaults; later proofs must only use them after showing agreement with authenticated openings or charging disagreement to a Merkle/hash side event.

The intended prefix-fixed use is to build these tables from the query-prefix oracle state, before the query-index challenge is sampled. Later openings checked by the verifier are handled in a downstream layer via pullback or controlled collision/inconsistency events.

context *soundness*

begin

definition *authenticated-value-at*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow 'f \Rightarrow \text{bool}$

where

$\text{authenticated-value-at } s \ r \ \text{len } i \ v \longleftrightarrow$
 $(\exists \text{opn} :: 'f \text{ authenticated-opening.}$
 $\text{authenticated-opening-in } s \ \text{opn} \wedge$
 $\text{opening-root } \text{opn} = r \wedge$
 $\text{opening-length } \text{opn} = \text{len} \wedge$
 $\text{opening-index } \text{opn} = i \wedge$
 $\text{opening-value } \text{opn} = v)$

lemma *authenticated-value-atI:*

assumes *authenticated-opening-in* $s \ \text{opn}$
and $\text{opening-root } \text{opn} = r$
and $\text{opening-length } \text{opn} = \text{len}$
and $\text{opening-index } \text{opn} = i$
and $\text{opening-value } \text{opn} = v$
shows *authenticated-value-at* $s \ r \ \text{len } i \ v$
using *assms unfolding authenticated-value-at-def* **by** *blast*

lemma *authenticated-value-atE:*

assumes *authenticated-value-at* $s \ r \ \text{len } i \ v$
obtains *opn* **where**
 $\text{authenticated-opening-in } s \ \text{opn}$
 $\text{opening-root } \text{opn} = r$
 $\text{opening-length } \text{opn} = \text{len}$
 $\text{opening-index } \text{opn} = i$
 $\text{opening-value } \text{opn} = v$
using *assms unfolding authenticated-value-at-def* **by** *blast*

lemma *authenticated-value-at-unique-if-clean:*

assumes *clean*: $\neg \text{hash-map-output-collision } s$
and v : *authenticated-value-at* $s \ r \ \text{len } i \ v$

and v' : *authenticated-value-at* s r len i v'
 shows $v = v'$
proof –
 from *authenticated-value-atE*[OF v] **obtain** opn **where**
 $auth$: *authenticated-opening-in* s opn
 and $root$: *opening-root* $opn = r$
 and $len-eq$: *opening-length* $opn = len$
 and idx : *opening-index* $opn = i$
 and val : *opening-value* $opn = v$
 by *blast*
 from *authenticated-value-atE*[OF v'] **obtain** opn' **where**
 $auth'$: *authenticated-opening-in* s opn'
 and $root'$: *opening-root* $opn' = r$
 and $len-eq'$: *opening-length* $opn' = len$
 and idx' : *opening-index* $opn' = i$
 and val' : *opening-value* $opn' = v'$
 by *blast*
 show *?thesis*
proof (*rule ccontr*)
 assume $v \neq v'$
 then have $diff$: *opening-value* $opn \neq$ *opening-value* opn'
 using val val' **by** *simp*
 have $collision$: *hash-map-output-collision* s
 by (*rule inconsistent-authenticated-openings-imp-hash-collision*
 $[OF$ $auth$ $auth'$])
 (*use* $root$ $root'$ $len-eq$ $len-eq'$ idx idx' $diff$ **in** *simp-all*)
 then show *False*
 using *clean* **by** *contradiction*
 qed
 qed

lemma *authenticated-value-at-mono*:
 assumes v : *authenticated-value-at* s r len i x
 and ext : $s \leq t$
 shows *authenticated-value-at* t r len i x
proof –
 from *authenticated-value-atE*[OF v] **obtain** opn **where**
 $auth$: *authenticated-opening-in* s opn
 and $root$: *opening-root* $opn = r$
 and $len-eq$: *opening-length* $opn = len$
 and idx : *opening-index* $opn = i$
 and val : *opening-value* $opn = x$
 by *blast*
 have *authenticated-opening-in* t opn
 by (*rule authenticated-opening-in-mono*[OF $auth$ ext])
 then show *?thesis*
 by (*rule authenticated-value-atI*[OF - $root$ $len-eq$ idx val])
 qed

definition *conceptual-opening-value*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow 'f$
where
conceptual-opening-value $s \ r \ \text{len} \ i =$
 $(THE \ v. \ \text{authenticated-value-at} \ s \ r \ \text{len} \ i \ v)$

lemma *conceptual-opening-value-eq-if-authenticated:*
assumes *clean*: $\neg \text{hash-map-output-collision} \ s$
and $v: \text{authenticated-value-at} \ s \ r \ \text{len} \ i \ x$
shows *conceptual-opening-value* $s \ r \ \text{len} \ i = x$
unfolding *conceptual-opening-value-def*
proof (*rule the-equality*)
show *authenticated-value-at* $s \ r \ \text{len} \ i \ x$
by (*rule v*)

next
fix y
assume *authenticated-value-at* $s \ r \ \text{len} \ i \ y$
then show $y = x$
using *authenticated-value-at-unique-if-clean*[*OF clean - v*] **by** *blast*
qed

definition *conceptual-table*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow \text{nat} \Rightarrow 'f \ \text{list}$
where
conceptual-table $s \ r \ \text{len} =$
 $\text{map} \ (\text{conceptual-opening-value} \ s \ r \ \text{len}) \ [0..<\text{len}]$

definition *conceptual-opening-value-with-default*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow \text{nat} \Rightarrow$
 $'f \Rightarrow \text{nat} \Rightarrow 'f$
where
conceptual-opening-value-with-default $s \ r \ \text{len} \ \text{dflt} \ i =$
 $(\text{if } \exists v. \ \text{authenticated-value-at} \ s \ r \ \text{len} \ i \ v$
 $\text{then } \text{conceptual-opening-value} \ s \ r \ \text{len} \ i$
 $\text{else } \text{dflt})$

definition *conceptual-table-with-default*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow \text{nat} \Rightarrow$
 $'f \ \text{list} \Rightarrow 'f \ \text{list}$
where
conceptual-table-with-default $s \ r \ \text{len} \ \text{default-table} =$
 $\text{map} \ (\lambda i. \ \text{conceptual-opening-value-with-default} \ s \ r \ \text{len}$
 $(\text{default-table} \ ! \ i) \ i) \ [0..<\text{len}]$

lemma *length-conceptual-table[simp]:*
 $\text{length} \ (\text{conceptual-table} \ s \ r \ \text{len}) = \text{len}$
unfolding *conceptual-table-def* **by** *simp*

lemma *length-conceptual-table-with-default[simp]:*

length (conceptual-table-with-default s r len default-table) = len
unfolding *conceptual-table-with-default-def* **by** *simp*

lemma *conceptual-table-nth*:

assumes $i < \text{len}$

shows *conceptual-table s r len ! i = conceptual-opening-value s r len i*

using *assms* **unfolding** *conceptual-table-def* **by** *simp*

lemma *conceptual-table-with-default-nth*:

assumes $i < \text{len}$

shows

conceptual-table-with-default s r len default-table ! i =

conceptual-opening-value-with-default s r len (default-table ! i) i

using *assms* **unfolding** *conceptual-table-with-default-def* **by** *simp*

lemma *conceptual-opening-value-with-default-eq-if-authenticated*:

assumes *clean*: $\neg \text{hash-map-output-collision } s$

and *v*: *authenticated-value-at s r len i x*

shows *conceptual-opening-value-with-default s r len dft i = x*

proof –

have *ex*: $\exists v. \text{authenticated-value-at } s \text{ r len } i \ v$

using *v* **by** *blast*

show *?thesis*

using *ex* *conceptual-opening-value-eq-if-authenticated*[*OF clean v*]

unfolding *conceptual-opening-value-with-default-def* **by** *simp*

qed

lemma *conceptual-opening-value-with-default-eq-default-if-not-authenticated*:

assumes $\neg (\exists v. \text{authenticated-value-at } s \text{ r len } i \ v)$

shows *conceptual-opening-value-with-default s r len dft i = dft*

using *assms* **unfolding** *conceptual-opening-value-with-default-def*

by (*auto split: if-splits*)

lemma *conceptual-table-with-default-eq-if-agrees-with-authenticated-values*:

assumes *clean*: $\neg \text{hash-map-output-collision } s$

and *len-eq*: *length default-table = len*

and *agrees*:

$\bigwedge i \ v. i < \text{len} \implies \text{authenticated-value-at } s \text{ r len } i \ v \implies$
default-table ! i = v

shows *conceptual-table-with-default s r len default-table = default-table*

proof (*rule nth-equalityI*)

show *length (conceptual-table-with-default s r len default-table) =*
length default-table

using *len-eq* **by** *simp*

next

fix *i*

assume *i-bound*:

$i < \text{length (conceptual-table-with-default s r len default-table)}$

then have *i-len*: $i < \text{len}$

```

    by simp
  show conceptual-table-with-default s r len default-table ! i =
    default-table ! i
  proof (cases  $\exists v. \text{authenticated-value-at } s \ r \ \text{len } i \ v$ )
    case True
    then obtain v where v: authenticated-value-at s r len i v
    by blast
    have
      conceptual-table-with-default s r len default-table ! i =
        conceptual-opening-value-with-default s r len (default-table ! i) i
    by (rule conceptual-table-with-default-nth[OF i-len])
    also have ... = v
    by (rule conceptual-opening-value-with-default-eq-if-authenticated
        [OF clean v])
    also have ... = default-table ! i
    using agrees[OF i-len v] by simp
    finally show ?thesis .
  next
  case False
  have
    conceptual-table-with-default s r len default-table ! i =
      conceptual-opening-value-with-default s r len (default-table ! i) i
  by (rule conceptual-table-with-default-nth[OF i-len])
  also have ... = default-table ! i
  by (rule
      conceptual-opening-value-with-default-eq-default-if-not-authenticated
      [OF False])
  finally show ?thesis .
qed
qed

lemma conceptual-table-agrees-with-authenticated-opening:
  assumes clean:  $\neg \text{hash-map-output-collision } s$ 
  and auth: authenticated-opening-in s opn
  and root: opening-root opn = r
  and len-eq: opening-length opn = len
  and idx-bound: opening-index opn < len
  shows
    conceptual-table s r len ! opening-index opn = opening-value opn
  proof -
    have v:
      authenticated-value-at s r len (opening-index opn)
      (opening-value opn)
    by (rule authenticated-value-atI[OF auth root len-eq refl refl])
    have conceptual-table s r len ! opening-index opn =
      conceptual-opening-value s r len (opening-index opn)
    by (rule conceptual-table-nth[OF idx-bound])
    also have ... = opening-value opn
    by (rule conceptual-opening-value-eq-if-authenticated[OF clean v])
  
```



```

    finally show ?thesis .
qed

lemma conceptual-table-agrees-with-partial-authenticated-table:
  assumes clean:  $\neg$  hash-map-output-collision s
    and table: partial-authenticated-table r len openings s
    and opn-in:  $opn \in \text{set openings}$ 
  shows
    conceptual-table s r len ! opening-index opn = opening-value opn
proof -
  have auth: authenticated-opening-in s opn
    using table opn-in unfolding partial-authenticated-table-def by blast
  have root: opening-root opn = r
    using table opn-in unfolding partial-authenticated-table-def by blast
  have len-eq: opening-length opn = len
    using table opn-in unfolding partial-authenticated-table-def by blast
  have idx-bound: opening-index opn < len
    using auth len-eq unfolding authenticated-opening-in-def by simp
  show ?thesis
    by (rule conceptual-table-agrees-with-authenticated-opening
        [OF clean auth root len-eq idx-bound])
qed

end

end

```

```

theory Soundness-Conceptual-Query-Prefix
  imports Soundness-Conceptual-Table
begin

```

Prefix-fixed conceptual tables.

These tables are built from a staged query-prefix state, i.e. after the transcript prefix and earlier query rounds have been processed, but before the next query-index challenge is sampled. This is the state selection needed for later dynamic query-target bounds.

```

context soundness
begin

```

```

definition query-prefix-trace-conceptual-table
  :: 'f staged-query-prefix-data  $\Rightarrow$ 
    ('f, 'a) protocol-channel-scheme  $\Rightarrow$  'f list
where
  query-prefix-trace-conceptual-table prefix s =
    conceptual-table s (sqp-trace-root prefix) (scale * clength)

```

```

definition query-prefix-composition-conceptual-table
  :: 'f staged-query-prefix-data  $\Rightarrow$ 
    ('f, 'a) protocol-channel-scheme  $\Rightarrow$  'f list

```

where

query-prefix-composition-conceptual-table prefix s =
conceptual-table s (hd (spp-composition-fri-roots prefix))
*(scale * clength)*

definition *query-prefix-trace-conceptual-table-with-default*

:: 'f staged-query-prefix-data $\Rightarrow$
('f, 'a) protocol-channel-scheme $\Rightarrow$ 'f list $\Rightarrow$ 'f list

where

query-prefix-trace-conceptual-table-with-default prefix s default-table =
conceptual-table-with-default s (spp-trace-root prefix)
*(scale * clength) default-table*

definition *query-prefix-composition-conceptual-table-with-default*

:: 'f staged-query-prefix-data $\Rightarrow$
('f, 'a) protocol-channel-scheme $\Rightarrow$ 'f list $\Rightarrow$ 'f list

where

query-prefix-composition-conceptual-table-with-default prefix s
default-table =
conceptual-table-with-default s (hd (spp-composition-fri-roots prefix))
*(scale * clength) default-table*

lemma *length-query-prefix-trace-conceptual-table[simp]:*

*length (query-prefix-trace-conceptual-table prefix s) = scale * clength*

unfolding *query-prefix-trace-conceptual-table-def by simp*

lemma *length-query-prefix-composition-conceptual-table[simp]:*

length (query-prefix-composition-conceptual-table prefix s) =
*scale * clength*

unfolding *query-prefix-composition-conceptual-table-def by simp*

lemma *length-query-prefix-trace-conceptual-table-with-default[simp]:*

length
(query-prefix-trace-conceptual-table-with-default prefix s
*default-table) = scale * clength*

unfolding *query-prefix-trace-conceptual-table-with-default-def by simp*

lemma *length-query-prefix-composition-conceptual-table-with-default[simp]:*

length
(query-prefix-composition-conceptual-table-with-default prefix s
*default-table) = scale * clength*

unfolding *query-prefix-composition-conceptual-table-with-default-def*
by *simp*

lemma *query-prefix-trace-conceptual-table-agrees-with-authenticated-opening:*

assumes *clean: $\neg$ hash-map-output-collision s*

and *auth: authenticated-opening-in s opn*

and *root: opening-root opn = spp-trace-root prefix*

and *len-eq: opening-length opn = scale * clength*

and *idx-bound*: *opening-index opn* < *scale * clength*
shows
query-prefix-trace-conceptual-table prefix s ! opening-index opn =
opening-value opn
unfolding *query-prefix-trace-conceptual-table-def*
by (*rule conceptual-table-agrees-with-authenticated-opening*
[*OF clean auth root len-eq idx-bound*])

lemma *query-prefix-composition-conceptual-table-agrees-with-authenticated-opening*:

assumes *clean*: $\neg$ *hash-map-output-collision s*
and *auth*: *authenticated-opening-in s opn*
and *roots-nonempty*: *sqp-composition-fri-roots prefix* $\neq$ []
and *root*: *opening-root opn* = *hd (sqp-composition-fri-roots prefix)*
and *len-eq*: *opening-length opn* = *scale * clength*
and *idx-bound*: *opening-index opn* < *scale * clength*
shows
query-prefix-composition-conceptual-table prefix s !
opening-index opn = opening-value opn
unfolding *query-prefix-composition-conceptual-table-def*
by (*rule conceptual-table-agrees-with-authenticated-opening*
[*OF clean auth root len-eq idx-bound*])

lemma

query-prefix-trace-conceptual-table-with-default-agrees-with-authenticated-opening:

assumes *clean*: $\neg$ *hash-map-output-collision s*
and *auth*: *authenticated-opening-in s opn*
and *root*: *opening-root opn* = *sqp-trace-root prefix*
and *len-eq*: *opening-length opn* = *scale * clength*
and *idx-bound*: *opening-index opn* < *scale * clength*
shows
query-prefix-trace-conceptual-table-with-default prefix s
default-table ! opening-index opn =
opening-value opn

proof –

have *v*:
*authenticated-value-at s (sqp-trace-root prefix) (scale * clength)*
(opening-index opn) (opening-value opn)
by (*rule authenticated-value-atI*[*OF auth root len-eq refl refl*])
have
query-prefix-trace-conceptual-table-with-default prefix s
default-table ! opening-index opn =
conceptual-opening-value-with-default s (sqp-trace-root prefix)
*(scale * clength) (default-table ! opening-index opn)*
(opening-index opn)
unfolding *query-prefix-trace-conceptual-table-with-default-def*
by (*rule conceptual-table-with-default-nth*[*OF idx-bound*])
also have ... = *opening-value opn*
by (*rule conceptual-opening-value-with-default-eq-if-authenticated*
[*OF clean v*])

finally show ?thesis .
qed

lemma

query-prefix-composition-conceptual-table-with-default-agrees-with-authenticated-opening:

assumes *clean*: $\neg$ *hash-map-output-collision* *s*

and *auth*: *authenticated-opening-in* *s* *opn*

and *root*: *opening-root* *opn* = *hd* (*sqp-composition-fri-roots* *prefix*)

and *len-eq*: *opening-length* *opn* = *scale* * *clength*

and *idx-bound*: *opening-index* *opn* < *scale* * *clength*

shows

query-prefix-composition-conceptual-table-with-default *prefix* *s*

default-table ! *opening-index* *opn* =

opening-value *opn*

proof —

have *v*:

authenticated-value-at *s* (*hd* (*sqp-composition-fri-roots* *prefix*))

(*scale* * *clength*) (*opening-index* *opn*) (*opening-value* *opn*)

by (*rule* *authenticated-value-atI*[*OF* *auth* *root* *len-eq* *refl* *refl*])

have

query-prefix-composition-conceptual-table-with-default *prefix* *s*

default-table ! *opening-index* *opn* =

conceptual-opening-value-with-default *s*

(*hd* (*sqp-composition-fri-roots* *prefix*)) (*scale* * *clength*)

(*default-table* ! *opening-index* *opn*) (*opening-index* *opn*)

unfolding *query-prefix-composition-conceptual-table-with-default-def*

by (*rule* *conceptual-table-with-default-nth*[*OF* *idx-bound*])

also have ... = *opening-value* *opn*

by (*rule* *conceptual-opening-value-with-default-eq-if-authenticated*

[*OF* *clean* *v*])

finally show ?thesis .

qed

lemma *query-prefix-trace-conceptual-table-agrees-with-partial-authenticated-table:*

assumes *clean*: $\neg$ *hash-map-output-collision* *s*

and *table*:

partial-authenticated-table (*sqp-trace-root* *prefix*)

(*scale* * *clength*) *openings* *s*

and *opn-in*: *opn* $\in$ *set* *openings*

shows

query-prefix-trace-conceptual-table *prefix* *s* ! *opening-index* *opn* =

opening-value *opn*

unfolding *query-prefix-trace-conceptual-table-def*

by (*rule* *conceptual-table-agrees-with-partial-authenticated-table*

[*OF* *clean* *table* *opn-in*])

lemma

query-prefix-composition-conceptual-table-agrees-with-partial-authenticated-table:

assumes *clean*: $\neg$ *hash-map-output-collision* *s*

and *table*:
partial-authenticated-table (*hd* (*sqp-composition-fri-roots prefix*))
*(scale * clength) openings s*
and *opn-in*: *opn* $\in$ *set openings*
shows
query-prefix-composition-conceptual-table prefix s ! opening-index opn =
opening-value opn
unfolding *query-prefix-composition-conceptual-table-def*
by (*rule conceptual-table-agrees-with-partial-authenticated-table*
[OF clean table opn-in])

lemma

query-prefix-trace-conceptual-table-with-default-agrees-with-partial-authenticated-table:

assumes *clean*: $\neg$ *hash-map-output-collision s*

and *table*:

partial-authenticated-table (*sqp-trace-root prefix*)
*(scale * clength) openings s*

and *opn-in*: *opn* $\in$ *set openings*

shows

query-prefix-trace-conceptual-table-with-default prefix s
default-table ! opening-index opn =
opening-value opn

proof —

have *auth*: *authenticated-opening-in s opn*

using *table opn-in* **unfolding** *partial-authenticated-table-def* **by** *blast*

have *root*: *opening-root opn = sqp-trace-root prefix*

using *table opn-in* **unfolding** *partial-authenticated-table-def* **by** *blast*

have *len-eq*: *opening-length opn = scale * clength*

using *table opn-in* **unfolding** *partial-authenticated-table-def* **by** *blast*

have *idx-bound*: *opening-index opn < scale * clength*

using *auth len-eq* **unfolding** *authenticated-opening-in-def* **by** *simp*

show *?thesis*

by (*rule*

query-prefix-trace-conceptual-table-with-default-agrees-with-authenticated-opening
[OF clean auth root len-eq idx-bound])

qed

lemma

query-prefix-composition-conceptual-table-with-default-agrees-with-partial-authenticated-table:

assumes *clean*: $\neg$ *hash-map-output-collision s*

and *table*:

partial-authenticated-table (*hd* (*sqp-composition-fri-roots prefix*))
*(scale * clength) openings s*

and *opn-in*: *opn* $\in$ *set openings*

shows

query-prefix-composition-conceptual-table-with-default prefix s
default-table ! opening-index opn =
opening-value opn

proof —

have *auth*: *authenticated-opening-in s opn*
using *table opn-in unfolding partial-authenticated-table-def* **by** *blast*
have *root*: *opening-root opn = hd (sqp-composition-fri-roots prefix)*
using *table opn-in unfolding partial-authenticated-table-def* **by** *blast*
have *len-eq*: *opening-length opn = scale * clength*
using *table opn-in unfolding partial-authenticated-table-def* **by** *blast*
have *idx-bound*: *opening-index opn < scale * clength*
using *auth len-eq unfolding authenticated-opening-in-def* **by** *simp*
show *?thesis*
by (*rule*
query-prefix-composition-conceptual-table-with-default-agrees-with-authenticated-opening
[OF clean auth root len-eq idx-bound])
qed

lemma *query-prefix-trace-conceptual-table-with-default-eq-default-if-agrees-with-authenticated-values*:

assumes *clean*: $\neg$ *hash-map-output-collision s*
and *len-eq*: *length default-table = scale * clength*
and *agrees*:
 $\bigwedge i v. i < scale * clength \implies$
authenticated-value-at s (sqp-trace-root prefix)
*(scale * clength) i v \implies*
default-table ! i = v
shows
query-prefix-trace-conceptual-table-with-default prefix s
default-table = default-table
unfolding *query-prefix-trace-conceptual-table-with-default-def*
by (*rule conceptual-table-with-default-eq-if-agrees-with-authenticated-values*
[OF clean len-eq agrees])

lemma *query-prefix-composition-conceptual-table-with-default-eq-default-if-agrees-with-authenticated-values*:

assumes *clean*: $\neg$ *hash-map-output-collision s*
and *len-eq*: *length default-table = scale * clength*
and *agrees*:
 $\bigwedge i v. i < scale * clength \implies$
authenticated-value-at s (hd (sqp-composition-fri-roots prefix))
*(scale * clength) i v \implies*
default-table ! i = v
shows
query-prefix-composition-conceptual-table-with-default prefix s
default-table = default-table
unfolding *query-prefix-composition-conceptual-table-with-default-def*
by (*rule conceptual-table-with-default-eq-if-agrees-with-authenticated-values*
[OF clean len-eq agrees])

lemma *query-prefix-trace-conceptual-table-partial-trace-candidate*:

assumes *clean*: $\neg$ *hash-map-output-collision s*
and *len*: *length trace-openings = rounds*
and *tables*:
 $\bigwedge i. i < rounds \implies$

```

      partial-authenticated-table (sqp-trace-root prefix)
      (scale * clength) (trace-openings ! i) s
shows
  partial-trace-table-candidate
    (query-prefix-trace-conceptual-table prefix s) trace-openings
unfolding partial-trace-table-candidate-def
proof (intro conjI allI impI)
  show length (query-prefix-trace-conceptual-table prefix s) =
    scale * clength
  by simp
next
  show length trace-openings = rounds
  by (rule len)
next
  fix i
  assume i-bound: i < rounds
  show
    table-agrees-with-authenticated-openings
      (query-prefix-trace-conceptual-table prefix s) (scale * clength)
      (trace-openings ! i)
  unfolding table-agrees-with-authenticated-openings-def
proof (intro conjI ballI)
  show length (query-prefix-trace-conceptual-table prefix s) =
    scale * clength
  by simp
next
  fix opn
  assume opn-in: opn ∈ set (trace-openings ! i)
  have table: partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) (trace-openings ! i) s
  by (rule tables[OF i-bound])
  show opening-index opn < scale * clength
  using table opn-in
  unfolding partial-authenticated-table-def authenticated-opening-in-def
  by auto
next
  fix opn
  assume opn-in: opn ∈ set (trace-openings ! i)
  have table: partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) (trace-openings ! i) s
  by (rule tables[OF i-bound])
  show
    query-prefix-trace-conceptual-table prefix s ! opening-index opn =
      opening-value opn
  by (rule
    query-prefix-trace-conceptual-table-agrees-with-partial-authenticated-table
    [OF clean table opn-in])
qed
qed

```

```

lemma query-prefix-trace-conceptual-table-with-default-partial-trace-candidate:
  assumes clean:  $\neg$  hash-map-output-collision s
  and len: length trace-openings = rounds
  and tables:
     $\bigwedge i. i < \text{rounds} \implies$ 
      partial-authenticated-table (sqp-trace-root prefix)
        (scale * clength) (trace-openings ! i) s
  shows
    partial-trace-table-candidate
      (query-prefix-trace-conceptual-table-with-default prefix s
        default-table) trace-openings
  unfolding partial-trace-table-candidate-def
proof (intro conjI allI impI)
  show
    length
      (query-prefix-trace-conceptual-table-with-default prefix s
        default-table) =
      scale * clength
    by simp
  next
    show length trace-openings = rounds
    by (rule len)
  next
    fix i
    assume i-bound: i < rounds
    show
      table-agrees-with-authenticated-openings
        (query-prefix-trace-conceptual-table-with-default prefix s
          default-table)
        (scale * clength) (trace-openings ! i)
    unfolding table-agrees-with-authenticated-openings-def
proof (intro conjI ballI)
  show
    length
      (query-prefix-trace-conceptual-table-with-default prefix s
        default-table) =
      scale * clength
    by simp
  next
    fix opn
    assume opn-in: opn  $\in$  set (trace-openings ! i)
    have table: partial-authenticated-table (sqp-trace-root prefix)
      (scale * clength) (trace-openings ! i) s
    by (rule tables[OF i-bound])
    show opening-index opn < scale * clength
    using table opn-in
    unfolding partial-authenticated-table-def authenticated-opening-in-def
    by auto

```



```

next
  fix opn
  assume opn-in:  $opn \in \text{set } (\text{trace-openings} \ ! \ i)$ 
  have table: partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) (trace-openings ! i) s
    by (rule tables[OF i-bound])
  show
    query-prefix-trace-conceptual-table-with-default prefix s
    default-table ! opening-index opn =
    opening-value opn
    by (rule
      query-prefix-trace-conceptual-table-with-default-agrees-with-partial-authenticated-table
      [OF clean table opn-in])
qed
qed

lemma query-prefix-composition-conceptual-table-partial-composition-candidate:
  assumes clean:  $\neg \text{hash-map-output-collision } s$ 
  and len: length composition-openings = rounds
  and tables:
     $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
    (scale * clength) (composition-openings ! i) s
  shows
    partial-composition-table-candidate
    (query-prefix-composition-conceptual-table prefix s)
    composition-openings
  unfolding partial-composition-table-candidate-def
proof (intro conjI allI impI)
  show
    length (query-prefix-composition-conceptual-table prefix s) =
    scale * clength
    by simp
next
  show length composition-openings = rounds
  by (rule len)
next
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  show
    table-agrees-with-authenticated-openings
    (query-prefix-composition-conceptual-table prefix s)
    (scale * clength) (composition-openings ! i)
  unfolding table-agrees-with-authenticated-openings-def
proof (intro conjI ballI)
  show
    length (query-prefix-composition-conceptual-table prefix s) =
    scale * clength
    by simp

```

```

next
  fix opn
  assume opn-in:  $opn \in \text{set } (\text{composition-openings } ! i)$ 
  have table:
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) (composition-openings ! i) s
    by (rule tables[OF i-bound])
  show opening-index opn < scale * clength
    using table opn-in
    unfolding partial-authenticated-table-def authenticated-opening-in-def
    by auto
next
  fix opn
  assume opn-in:  $opn \in \text{set } (\text{composition-openings } ! i)$ 
  have table:
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) (composition-openings ! i) s
    by (rule tables[OF i-bound])
  show
    query-prefix-composition-conceptual-table prefix s !
    opening-index opn = opening-value opn
    by (rule
      query-prefix-composition-conceptual-table-agrees-with-partial-authenticated-table
      [OF clean table opn-in])
qed
qed

lemma
  query-prefix-composition-conceptual-table-with-default-partial-composition-candidate:
  assumes clean:  $\neg \text{hash-map-output-collision } s$ 
  and len:  $\text{length } \text{composition-openings} = \text{rounds}$ 
  and tables:
     $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) (composition-openings ! i) s
  shows
    partial-composition-table-candidate
      (query-prefix-composition-conceptual-table-with-default prefix s
        default-table)
      composition-openings
  unfolding partial-composition-table-candidate-def
proof (intro conjI allI impI)
  show
    length
      (query-prefix-composition-conceptual-table-with-default prefix s
        default-table) =
      scale * clength
    by simp
next

```

```

show length composition-openings = rounds
  by (rule len)
next
  fix i
  assume i-bound: i < rounds
  show
    table-agrees-with-authenticated-openings
      (query-prefix-composition-conceptual-table-with-default prefix s
        default-table)
      (scale * clength) (composition-openings ! i)
    unfolding table-agrees-with-authenticated-openings-def
  proof (intro conjI ballI)
  show
    length
      (query-prefix-composition-conceptual-table-with-default prefix s
        default-table) =
      scale * clength
    by simp
  next
  fix opn
  assume opn-in: opn ∈ set (composition-openings ! i)
  have table:
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) (composition-openings ! i) s
    by (rule tables[OF i-bound])
  show opening-index opn < scale * clength
    using table opn-in
    unfolding partial-authenticated-table-def authenticated-opening-in-def
    by auto
  next
  fix opn
  assume opn-in: opn ∈ set (composition-openings ! i)
  have table:
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) (composition-openings ! i) s
    by (rule tables[OF i-bound])
  show
    query-prefix-composition-conceptual-table-with-default prefix s
      default-table ! opening-index opn =
      opening-value opn
    by (rule
      query-prefix-composition-conceptual-table-with-default-agrees-with-partial-authenticated-table
      [OF clean table opn-in])
  qed
qed

end

end

```

theory *Soundness-Conceptual-Query-Target*

imports

Soundness-Conceptual-Query-Prefix

Soundness-Aligned-Partial-Query-Prefix

begin

Prefix-fixed query targets built from conceptual Merkle tables.

This layer deliberately uses only the query-prefix state. It therefore supports the dynamic query-index probability bound, but it does not claim that openings first authenticated after the query challenge agree with these tables. That later-opening bridge is a separate proof obligation.

context *soundness*

begin

definition *staged-query-prefix-conceptual-query-target*

$:: 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{nat set}$

where

$\text{staged-query-prefix-conceptual-query-target prefix prefix-state} =$
 $\text{query-sampling-success-space}$
 $(\text{query-prefix-trace-conceptual-table prefix prefix-state})$
 $(\text{query-prefix-composition-conceptual-table prefix prefix-state})$
 $(\text{sqp-alphas prefix})$

definition *staged-query-prefix-conceptual-default-query-target*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow$

$'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{nat set}$

where

$\text{staged-query-prefix-conceptual-default-query-target trace-default}$
 $\text{composition-default prefix prefix-state} =$
 $\text{query-sampling-success-space}$
 $(\text{query-prefix-trace-conceptual-table-with-default prefix prefix-state}$
 $\text{trace-default})$
 $(\text{query-prefix-composition-conceptual-table-with-default prefix}$
 $\text{prefix-state composition-default})$
 $(\text{sqp-alphas prefix})$

definition *staged-query-prefix-conceptual-empty-query-target*

$:: 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{nat set}$

where

$\text{staged-query-prefix-conceptual-empty-query-target prefix prefix-state} =$
 $\text{query-sampling-success-space}$
 $(\text{query-prefix-trace-conceptual-table prefix prefix-state})$
 $(\text{replicate (scale * clength) (sqp-composition-final prefix)})$
 $(\text{sqp-alphas prefix})$

lemma *staged-query-prefix-conceptual-query-target-subset:*

$\text{staged-query-prefix-conceptual-query-target prefix prefix-state} \subseteq$

query-sample-space
unfolding *staged-query-prefix-conceptual-query-target-def*
by (rule *query-sampling-success-space-subset*)

lemma *staged-query-prefix-conceptual-default-query-target-subset:*
staged-query-prefix-conceptual-default-query-target trace-default
composition-default prefix prefix-state $\subseteq$
query-sample-space
unfolding *staged-query-prefix-conceptual-default-query-target-def*
by (rule *query-sampling-success-space-subset*)

lemma *staged-query-prefix-conceptual-empty-query-target-subset:*
staged-query-prefix-conceptual-empty-query-target prefix prefix-state
 $\subseteq$ *query-sample-space*
unfolding *staged-query-prefix-conceptual-empty-query-target-def*
by (rule *query-sampling-success-space-subset*)

lemma *staged-query-prefix-conceptual-query-target-fraction-bound:*
nnreal
 (card
 (staged-query-prefix-conceptual-query-target prefix prefix-state)) /
nnreal (card *query-sample-space*) $\leq$ *query-error-bound*
unfolding *staged-query-prefix-conceptual-query-target-def*
by (rule *query-sampling-success-space-fraction-bound-query-sample-space*)

lemma *staged-query-prefix-conceptual-default-query-target-fraction-bound:*
nnreal
 (card
 (staged-query-prefix-conceptual-default-query-target trace-default
 composition-default prefix prefix-state)) /
nnreal (card *query-sample-space*) $\leq$ *query-error-bound*
unfolding *staged-query-prefix-conceptual-default-query-target-def*
by (rule *query-sampling-success-space-fraction-bound-query-sample-space*)

lemma *staged-query-prefix-conceptual-empty-query-target-fraction-bound:*
nnreal
 (card
 (staged-query-prefix-conceptual-empty-query-target prefix
 prefix-state)) /
nnreal (card *query-sample-space*) $\leq$ *query-error-bound*
unfolding *staged-query-prefix-conceptual-empty-query-target-def*
by (rule *query-sampling-success-space-fraction-bound-query-sample-space*)

lemma *query-prefix-trace-conceptual-table-with-default-eq-if-agrees:*
assumes *clean*: $\neg$ *hash-map-output-collision prefix-state*
and *len*: *length trace-default* = *scale* * *clength*
and *agrees*:
 $\bigwedge i v. i < \text{scale} * \text{clength} \implies$
authenticated-value-at prefix-state (sqp-trace-root prefix)

$(scale * clength) i v \implies$
 $trace-default ! i = v$

shows

$query-prefix-trace-conceptual-table-with-default\ prefix\ prefix-state$
 $trace-default = trace-default$

unfolding $query-prefix-trace-conceptual-table-with-default-def$

by $(rule\ conceptual-table-with-default-eq-if-agrees-with-authenticated-values$
 $[OF\ clean\ len\ agrees])$

lemma $query-prefix-composition-conceptual-table-with-default-eq-if-agrees:$

assumes $clean: \neg hash-map-output-collision\ prefix-state$

and $len: length\ composition-default = scale * clength$

and $agrees:$

$\bigwedge i v. i < scale * clength \implies$
 $authenticated-value-at\ prefix-state$
 $(hd\ (sqp-composition-fri-roots\ prefix))\ (scale * clength)\ i v \implies$
 $composition-default ! i = v$

shows

$query-prefix-composition-conceptual-table-with-default\ prefix$
 $prefix-state\ composition-default = composition-default$

unfolding $query-prefix-composition-conceptual-table-with-default-def$

by $(rule\ conceptual-table-with-default-eq-if-agrees-with-authenticated-values$
 $[OF\ clean\ len\ agrees])$

lemma

$staged-query-prefix-candidate-pair-query-target-from-prefix-subset-conceptual-default-if-agrees:$

assumes $clean: \neg hash-map-output-collision\ prefix-state$

and $trace-len: length\ trace-table = scale * clength$

and $composition-len: length\ composition-table = scale * clength$

and $trace-agrees:$

$\bigwedge i v. i < scale * clength \implies$
 $authenticated-value-at\ prefix-state\ (sqp-trace-root\ prefix)$
 $(scale * clength)\ i v \implies$
 $trace-table ! i = v$

and $composition-agrees:$

$\bigwedge i v. i < scale * clength \implies$
 $authenticated-value-at\ prefix-state$
 $(hd\ (sqp-composition-fri-roots\ prefix))\ (scale * clength)\ i v \implies$
 $composition-table ! i = v$

shows

$staged-query-prefix-candidate-pair-query-target-from-prefix\ trace-table$
 $composition-table\ prefix\ prefix-state \subseteq$
 $staged-query-prefix-conceptual-default-query-target\ trace-table$
 $composition-table\ prefix\ prefix-state$

proof –

have $trace-eq:$

$query-prefix-trace-conceptual-table-with-default\ prefix\ prefix-state$
 $trace-table = trace-table$

by $(rule\ query-prefix-trace-conceptual-table-with-default-eq-if-agrees$

```

    [OF clean trace-len trace-agrees])
have composition-eq:
  query-prefix-composition-conceptual-table-with-default prefix
  prefix-state composition-table = composition-table
by (rule
  query-prefix-composition-conceptual-table-with-default-eq-if-agrees
  [OF clean composition-len composition-agrees])
show ?thesis
unfolding staged-query-prefix-candidate-pair-query-target-from-prefix-def
  staged-query-prefix-conceptual-default-query-target-def
using trace-eq composition-eq by simp
qed

```

definition *query-prefix-trace-default-agrees-with-authenticated-values*

```

:: 'f list  $\Rightarrow$  'f staged-query-prefix-data  $\Rightarrow$ 
  'f protocol-channel  $\Rightarrow$  bool

```

where

```

query-prefix-trace-default-agrees-with-authenticated-values trace-default
prefix prefix-state  $\longleftrightarrow$ 
( $\forall i v. i < \text{scale} * \text{clength} \longrightarrow$ 
  authenticated-value-at prefix-state (sqp-trace-root prefix)
  (scale * clength) i v  $\longrightarrow$ 
  trace-default ! i = v)

```

definition *query-prefix-composition-default-agrees-with-authenticated-values*

```

:: 'f list  $\Rightarrow$  'f staged-query-prefix-data  $\Rightarrow$ 
  'f protocol-channel  $\Rightarrow$  bool

```

where

```

query-prefix-composition-default-agrees-with-authenticated-values
composition-default prefix prefix-state  $\longleftrightarrow$ 
( $\forall i v. i < \text{scale} * \text{clength} \longrightarrow$ 
  authenticated-value-at prefix-state
  (hd (sqp-composition-fri-roots prefix)) (scale * clength) i v  $\longrightarrow$ 
  composition-default ! i = v)

```

definition *query-prefix-candidate-defaults-agree-with-authenticated-values*

```

:: 'f list  $\Rightarrow$  'f list  $\Rightarrow$  'f staged-query-prefix-data  $\Rightarrow$ 
  'f protocol-channel  $\Rightarrow$  bool

```

where

```

query-prefix-candidate-defaults-agree-with-authenticated-values
trace-default composition-default prefix prefix-state  $\longleftrightarrow$ 
query-prefix-trace-default-agrees-with-authenticated-values trace-default
prefix prefix-state  $\wedge$ 
query-prefix-composition-default-agrees-with-authenticated-values
composition-default prefix prefix-state

```

lemma *query-prefix-trace-default-agrees-with-authenticated-valuesD:*

assumes

```

query-prefix-trace-default-agrees-with-authenticated-values trace-default

```

```

    prefix prefix-state
  and  $i < \text{scale} * \text{clength}$ 
  and
    authenticated-value-at prefix-state (sqp-trace-root prefix)
      (scale * clength) i v
  shows trace-default !  $i = v$ 
  using assms
  unfolding query-prefix-trace-default-agrees-with-authenticated-values-def
  by blast

```

lemma *query-prefix-composition-default-agrees-with-authenticated-valuesD:*

```

  assumes
    query-prefix-composition-default-agrees-with-authenticated-values
      composition-default prefix prefix-state
  and  $i < \text{scale} * \text{clength}$ 
  and
    authenticated-value-at prefix-state
      (hd (sqp-composition-fri-roots prefix)) (scale * clength) i v
  shows composition-default !  $i = v$ 
  using assms
  unfolding
    query-prefix-composition-default-agrees-with-authenticated-values-def
  by blast

```

lemma

staged-query-prefix-candidate-pair-query-target-from-prefix-subset-conceptual-default-if-prefix-agrees:

```

  assumes clean:  $\neg \text{hash-map-output-collision prefix-state}$ 
  and trace-len:  $\text{length trace-table} = \text{scale} * \text{clength}$ 
  and composition-len:  $\text{length composition-table} = \text{scale} * \text{clength}$ 
  and agrees:
    query-prefix-candidate-defaults-agree-with-authenticated-values
      trace-table composition-table prefix prefix-state

```

shows

```

  staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
    composition-table prefix prefix-state  $\subseteq$ 
  staged-query-prefix-conceptual-default-query-target trace-table
    composition-table prefix prefix-state

```

proof (rule

```

  staged-query-prefix-candidate-pair-query-target-from-prefix-subset-conceptual-default-if-agrees
  [OF clean trace-len composition-len])

```

fix $i v$

assume *i-bound*: $i < \text{scale} * \text{clength}$

and *auth*:

```

  authenticated-value-at prefix-state (sqp-trace-root prefix)
    (scale * clength) i v

```

have *trace-agrees*:

```

  query-prefix-trace-default-agrees-with-authenticated-values trace-table
    prefix prefix-state

```

using *agrees*


```

unfolding
  query-prefix-candidate-defaults-agree-with-authenticated-values-def
by simp
show trace-table ! i = v
by (rule
  query-prefix-trace-default-agrees-with-authenticated-valuesD
  [OF trace-agrees i-bound auth])
next
fix i v
assume i-bound: i < scale * clength
and auth:
  authenticated-value-at prefix-state
  (hd (sqp-composition-fri-roots prefix)) (scale * clength) i v
have composition-agrees:
  query-prefix-composition-default-agrees-with-authenticated-values
  composition-table prefix prefix-state
using agrees
unfolding
  query-prefix-candidate-defaults-agree-with-authenticated-values-def
by simp
show composition-table ! i = v
by (rule
  query-prefix-composition-default-agrees-with-authenticated-valuesD
  [OF composition-agrees i-bound auth])
qed

```

definition query-prefix-candidate-defaults-agree-at-query

```

:: 'f list  $\Rightarrow$  'f list  $\Rightarrow$  nat  $\Rightarrow$ 
  'f staged-query-prefix-data  $\Rightarrow$  'f protocol-channel  $\Rightarrow$  bool

```

where

```

query-prefix-candidate-defaults-agree-at-query trace-table
  composition-table idx prefix prefix-state  $\longleftrightarrow$ 
  ( $\forall j \in \text{set (powers-scaled idx)}$ ).
  query-prefix-trace-conceptual-table-with-default prefix prefix-state
  trace-table ! j = trace-table ! j)  $\wedge$ 
  query-prefix-composition-conceptual-table-with-default prefix prefix-state
  composition-table ! idx = composition-table ! idx

```

lemma query-prefix-candidate-pair-query-hit-imp-conceptual-default-if-agrees-at-query:

assumes hit:

```

  idx  $\in$ 
    staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
    composition-table prefix prefix-state

```

and trace-len: length trace-table = scale * clength

and composition-len: length composition-table = scale * clength

and agrees:

```

  query-prefix-candidate-defaults-agree-at-query trace-table
  composition-table idx prefix prefix-state

```

and default-trace-low:

```

    trace-table-low-degree
      (query-prefix-trace-conceptual-table-with-default prefix prefix-state
       trace-table)
  and default-composition-low:
    composition-table-low-degree maxDegree
      (query-prefix-composition-conceptual-table-with-default prefix
       prefix-state composition-table)
  and default-not-all:
    ¬ all-queries-consistent
      (query-prefix-trace-conceptual-table-with-default prefix prefix-state
       trace-table)
      (query-prefix-composition-conceptual-table-with-default prefix
       prefix-state composition-table)
      (sqp-alphas prefix)
  shows
    idx ∈
      staged-query-prefix-conceptual-default-query-target trace-table
      composition-table prefix prefix-state
  proof —
  let ?trace-default =
    query-prefix-trace-conceptual-table-with-default prefix prefix-state
    trace-table
  let ?composition-default =
    query-prefix-composition-conceptual-table-with-default prefix
    prefix-state composition-table
  have trace-len-eq: length trace-table = length ?trace-default
    using trace-len by simp
  have composition-len-eq:
    length composition-table = length ?composition-default
    using composition-len by simp
  have trace-values:
    map ((! trace-table) (powers-scaled idx)) =
      map ((! ?trace-default) (powers-scaled idx))
  proof (rule nth-equalityI)
  show
    length (map ((! trace-table) (powers-scaled idx))) =
      length (map ((! ?trace-default) (powers-scaled idx)))
    by simp
  next
  fix k
  assume k-bound:
    k < length (map ((! trace-table) (powers-scaled idx)))
  then have ps-bound: k < length (powers-scaled idx)
    by simp
  have ps-mem: powers-scaled idx ! k ∈ set (powers-scaled idx)
    by (rule nth-mem[OF ps-bound])
  have eq:
    ?trace-default ! (powers-scaled idx ! k) =
      trace-table ! (powers-scaled idx ! k)

```

```

using agrees ps-mem
unfolding query-prefix-candidate-defaults-agree-at-query-def
by blast
show
  map ((! trace-table) (powers-scaled idx) ! k =
    map ((! ?trace-default) (powers-scaled idx) ! k
  using ps-bound eq by simp
qed
have composition-value:
  composition-table ! idx = ?composition-default ! idx
using agrees
unfolding query-prefix-candidate-defaults-agree-at-query-def
by simp
have qc-iff:
  query-consistent-at trace-table composition-table (sqp-alphas prefix)
    idx  $\longleftrightarrow$ 
    query-consistent-at ?trace-default ?composition-default
      (sqp-alphas prefix) idx
by (rule query-consistent-at-cong-opened-values
  [OF trace-len-eq composition-len-eq trace-values composition-value])
have qc:
  query-consistent-at trace-table composition-table (sqp-alphas prefix)
    idx
using hit
unfolding staged-query-prefix-candidate-pair-query-target-from-prefix-def
  query-sampling-success-space-def query-agreement-indices-def
by (cases
  trace-table-low-degree trace-table  $\wedge$ 
  composition-table-low-degree maxDegree composition-table  $\wedge$ 
   $\neg$  all-queries-consistent trace-table composition-table
  (sqp-alphas prefix))
  auto
have query-idx: idx  $\in$  query-sample-space
using hit query-sampling-success-space-subset
unfolding staged-query-prefix-candidate-pair-query-target-from-prefix-def
by blast
have default-qc:
  query-consistent-at ?trace-default ?composition-default
    (sqp-alphas prefix) idx
using qc qc-iff by simp
show ?thesis
using query-idx default-qc default-trace-low default-composition-low
  default-not-all
unfolding staged-query-prefix-candidate-pair-query-target-from-prefix-def
  staged-query-prefix-conceptual-default-query-target-def
  query-sampling-success-space-def query-agreement-indices-def
by auto
qed

```

lemma *query-prefix-candidate-defaults-agree-at-query-from-prefix-authenticated-openings:*
assumes *clean:* $\neg$ *hash-map-output-collision* *prefix-state*
and *i-bound:* $i < \text{rounds}$
and *trace-prefix:*
 partial-authenticated-table (*sqp-trace-root* *prefix*)
 (*scale* * *clength*) *trace-openings* *prefix-state*
and *composition-prefix:*
 partial-authenticated-table (*hd* (*sqp-composition-fri-roots* *prefix*))
 (*scale* * *clength*) *composition-openings* *prefix-state*
and *consistent:*
 partial-query-openings-consistent *trace-openings* *composition-openings*
 (*sqp-alphas* *prefix*) *idx*
and *trace-candidate:*
 partial-trace-table-candidate *trace-table*
 ((*replicate* *rounds* []) [*i* := *trace-openings*])
and *composition-candidate:*
 partial-composition-table-candidate *composition-table*
 ((*replicate* *rounds* []) [*i* := *composition-openings*])
shows
 query-prefix-candidate-defaults-agree-at-query *trace-table*
 composition-table *idx* *prefix* *prefix-state*
proof –
let *?trace-openingss* = (*replicate* *rounds* []) [*i* := *trace-openings*]
let *?composition-openingss* =
 (*replicate* *rounds* []) [*i* := *composition-openings*]
have *trace-agrees:*
 table-agrees-with-authenticated-openings *trace-table*
 (*scale* * *clength*) *trace-openings*
proof –
have *all:*
 $\forall ia < \text{rounds}.$
 table-agrees-with-authenticated-openings *trace-table*
 (*scale* * *clength*) (*?trace-openingss* ! *ia*)
using *trace-candidate*
unfolding *partial-trace-table-candidate-def* **by** *blast*
have *at-i:*
 table-agrees-with-authenticated-openings *trace-table*
 (*scale* * *clength*) (*?trace-openingss* ! *i*)
using *all i-bound* **by** *blast*
have *?trace-openingss* ! *i* = *trace-openings*
using *i-bound* **by** *simp*
then show *?thesis*
using *at-i* **by** *simp*
qed
have *composition-agrees:*
 table-agrees-with-authenticated-openings *composition-table*
 (*scale* * *clength*) *composition-openings*
proof –
have *all:*

```

   $\forall ia < rounds.$ 
    table-agrees-with-authenticated-openings composition-table
      (scale * clength) (?composition-openingsss ! ia)
    using composition-candidate
    unfolding partial-composition-table-candidate-def by blast
have at-i:
  table-agrees-with-authenticated-openings composition-table
    (scale * clength) (?composition-openingsss ! i)
    using all i-bound by blast
have ?composition-openingsss ! i = composition-openings
    using i-bound by simp
then show ?thesis
    using at-i by simp
qed
have trace-indices:
  map opening-index trace-openings = powers-scaled idx
    using consistent unfolding partial-query-openings-consistent-def by simp
have composition-indices:
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
    using consistent unfolding partial-query-openings-consistent-def by simp
have composition-nonempty: composition-openings  $\neq$  []
    using consistent unfolding partial-query-openings-consistent-def by simp
have trace-point:
   $\bigwedge j. j \in \text{set } (\text{powers-scaled } idx) \implies$ 
    query-prefix-trace-conceptual-table-with-default prefix prefix-state
      trace-table ! j =
      trace-table ! j
proof –
  fix j
  assume j-in: j  $\in$  set (powers-scaled idx)
  then obtain opn where opn-in: opn  $\in$  set trace-openings
    and opn-idx: opening-index opn = j
    using trace-indices
    by (metis (mono-tags, lifting) in-set-conv-nth length-map nth-map)
  have conceptual:
    query-prefix-trace-conceptual-table-with-default prefix prefix-state
      trace-table ! opening-index opn =
      opening-value opn
    by (rule
      query-prefix-trace-conceptual-table-with-default-agrees-with-partial-authenticated-table
      [OF clean trace-prefix opn-in])
  have table:
    trace-table ! opening-index opn = opening-value opn
    using trace-agrees opn-in
    unfolding table-agrees-with-authenticated-openings-def by blast
show
  query-prefix-trace-conceptual-table-with-default prefix prefix-state
    trace-table ! j =

```

```

    trace-table ! j
  using conceptual table opn-idx by simp
qed
have composition-point:
  query-prefix-composition-conceptual-table-with-default prefix
  prefix-state composition-table ! idx =
  composition-table ! idx
proof -
  have opn0-in: composition-openings ! 0 ∈ set composition-openings
    using composition-nonempty by simp
  have opn0-idx: opening-index (composition-openings ! 0) = idx
    using composition-indices composition-nonempty
    by (cases composition-openings) simp-all
  have conceptual:
    query-prefix-composition-conceptual-table-with-default prefix
    prefix-state composition-table ! opening-index (composition-openings ! 0) =
    opening-value (composition-openings ! 0)
    by (rule
      query-prefix-composition-conceptual-table-with-default-agrees-with-partial-authenticated-table
      [OF clean composition-prefix opn0-in])
  have table:
    composition-table ! opening-index (composition-openings ! 0) =
    opening-value (composition-openings ! 0)
    using composition-agrees opn0-in
    unfolding table-agrees-with-authenticated-openings-def by blast
  show ?thesis
    using conceptual table opn0-idx by simp
qed
show ?thesis
  unfolding query-prefix-candidate-defaults-agree-at-query-def
  using trace-point composition-point by blast
qed

lemma query-prefix-current-openings-imp-conceptual-default-query-target:
  assumes clean: ¬ hash-map-output-collision prefix-state
  and i-bound: i < rounds
  and trace-prefix:
    partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) trace-openings prefix-state
  and composition-prefix:
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
    (scale * clength) composition-openings prefix-state
  and consistent:
    partial-query-openings-consistent trace-openings composition-openings
    (sqp-alphas prefix) idx
  and trace-candidate:
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
  and composition-candidate:

```

```

    partial-composition-table-candidate composition-table
      ((replicate rounds []) [i := composition-openings])
  and default-trace-low:
    trace-table-low-degree
      (query-prefix-trace-conceptual-table-with-default prefix prefix-state
        trace-table)
  and default-composition-low:
    composition-table-low-degree maxDegree
      (query-prefix-composition-conceptual-table-with-default prefix
        prefix-state composition-table)
  and default-not-all:
    ¬ all-queries-consistent
      (query-prefix-trace-conceptual-table-with-default prefix prefix-state
        trace-table)
      (query-prefix-composition-conceptual-table-with-default prefix
        prefix-state composition-table)
      (sqp-alphas prefix)
  shows
    idx ∈
      staged-query-prefix-conceptual-default-query-target trace-table
      composition-table prefix prefix-state
  proof —
    let ?trace-openingss = (replicate rounds []) [i := trace-openings]
    let ?composition-openingss =
      (replicate rounds []) [i := composition-openings]
    let ?trace-default =
      query-prefix-trace-conceptual-table-with-default prefix prefix-state
      trace-table
    let ?composition-default =
      query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-table
    have trace-len: length trace-table = scale * clength
      using trace-candidate unfolding partial-trace-table-candidate-def
      by simp
    have composition-len: length composition-table = scale * clength
      using composition-candidate
      unfolding partial-composition-table-candidate-def by simp
    have idx-sample: idx ∈ query-sample-space
      using consistent unfolding partial-query-openings-consistent-def by simp
    have round:
      partial-query-round-consistent ?trace-openingss ?composition-openingss
        (sqp-alphas prefix) i idx
      using i-bound consistent
      unfolding partial-query-round-consistent-def
      partial-query-openings-consistent-def
      by simp
    have candidate-consistent:
      query-consistent-at trace-table composition-table (sqp-alphas prefix)
      idx

```

```

  by (rule partial-query-round-consistent-imp-query-consistent-at
    [OF trace-candidate composition-candidate round])
have agrees:
  query-prefix-candidate-defaults-agree-at-query trace-table
    composition-table idx prefix prefix-state
  by (rule
    query-prefix-candidate-defaults-agree-at-query-from-prefix-authenticated-openings
    [OF clean i-bound trace-prefix composition-prefix consistent
      trace-candidate composition-candidate])
have trace-len-eq: length trace-table = length ?trace-default
  using trace-len by simp
have composition-len-eq:
  length composition-table = length ?composition-default
  using composition-len by simp
have trace-values-eq:
  map (! trace-table) (powers-scaled idx) =
    map (! ?trace-default) (powers-scaled idx)
proof (rule nth-equalityI)
  show
    length (map (! trace-table) (powers-scaled idx)) =
      length (map (! ?trace-default) (powers-scaled idx))
    by simp
next
  fix k
  assume k-bound:
    k < length (map (! trace-table) (powers-scaled idx))
  then have ps-bound: k < length (powers-scaled idx)
    by simp
  have ps-mem: powers-scaled idx ! k ∈ set (powers-scaled idx)
    by (rule nth-mem[OF ps-bound])
  have eq:
    ?trace-default ! (powers-scaled idx ! k) =
      trace-table ! (powers-scaled idx ! k)
    using agrees ps-mem
  unfolding query-prefix-candidate-defaults-agree-at-query-def
  by blast
  show
    map (! trace-table) (powers-scaled idx) ! k =
      map (! ?trace-default) (powers-scaled idx) ! k
    using ps-bound eq by simp
qed
have composition-value-eq:
  composition-table ! idx = ?composition-default ! idx
  using agrees
  unfolding query-prefix-candidate-defaults-agree-at-query-def by simp
have qc-iff:
  query-consistent-at trace-table composition-table (sqp-alphas prefix)
    idx ⟷
  query-consistent-at ?trace-default ?composition-default

```



```

    (sqp-alphas prefix) idx
  by (rule query-consistent-at-cong-opened-values
    [OF trace-len-eq composition-len-eq trace-values-eq
      composition-value-eq])
  have default-consistent:
    query-consistent-at ?trace-default ?composition-default
      (sqp-alphas prefix) idx
  using candidate-consistent qc-iff by simp
  show ?thesis
  unfolding staged-query-prefix-conceptual-default-query-target-def
    query-sampling-success-space-def query-agreement-indices-def
  using idx-sample default-consistent default-trace-low
    default-composition-low default-not-all by simp
qed

lemma trace-table-agrees-with-prefix-authenticated-values-if-final-merkle-bound:
  assumes ext: prefix-state  $\leq$  final-state
  and clean:  $\neg$  hash-map-output-collision final-state
  and bind:
    merkle-root-binds-table (sqp-trace-root prefix) trace-table
      final-state
  and len: length trace-table = scale * clength
  and i-bound:  $i < \text{scale} * \text{clength}$ 
  and auth:
    authenticated-value-at prefix-state (sqp-trace-root prefix)
      (scale * clength) i v
  shows trace-table ! i = v
proof -
  from authenticated-value-atE[OF auth] obtain opn where
    opn-auth-prefix: authenticated-opening-in prefix-state opn
  and root: opening-root opn = sqp-trace-root prefix
  and opn-len: opening-length opn = scale * clength
  and idx: opening-index opn = i
  and val: opening-value opn = v
  by blast
  have opn-auth-final: authenticated-opening-in final-state opn
  by (rule authenticated-opening-in-mono[OF opn-auth-prefix ext])
  obtain n where len-pow: length trace-table =  $2^n$ 
  using len eval-domain-length-power by (metis mult.commute)
  have agrees:
    opening-index opn < length trace-table  $\wedge$ 
    trace-table ! opening-index opn = opening-value opn
  by (rule merkle-bound-table-agrees-with-authenticated-opening-if-clean
    [OF bind len-pow opn-auth-final root - clean])
    (use len opn-len in simp)
  then show ?thesis
  using idx val by simp
qed

```

lemma *composition-table-agrees-with-prefix-authenticated-values-if-final-merkle-bound:*

assumes *ext*: $\text{prefix-state} \leq \text{final-state}$
and *clean*: $\neg \text{hash-map-output-collision final-state}$
and *bind*:
 $\text{merkle-root-binds-table } (\text{hd } (\text{sqp-composition-fri-roots prefix}))$
 $\text{composition-table final-state}$
and *len*: $\text{length composition-table} = \text{scale} * \text{clength}$
and *i-bound*: $i < \text{scale} * \text{clength}$
and *auth*:
 $\text{authenticated-value-at prefix-state}$
 $(\text{hd } (\text{sqp-composition-fri-roots prefix})) (\text{scale} * \text{clength}) i v$
shows $\text{composition-table} ! i = v$

proof –

from $\text{authenticated-value-atE}[OF \text{ auth}]$ **obtain** *opn* **where**
 opn-auth-prefix : $\text{authenticated-opening-in prefix-state opn}$
and *root*:
 $\text{opening-root opn} = \text{hd } (\text{sqp-composition-fri-roots prefix})$
and *opn-len*: $\text{opening-length opn} = \text{scale} * \text{clength}$
and *idx*: $\text{opening-index opn} = i$
and *val*: $\text{opening-value opn} = v$
by *blast*
have *opn-auth-final*: $\text{authenticated-opening-in final-state opn}$
 by $(\text{rule authenticated-opening-in-mono}[OF \text{ opn-auth-prefix ext}])$
obtain *n* **where** *len-pow*: $\text{length composition-table} = 2^n$
 using *len eval-domain-length-power* **by** $(\text{metis mult.commute})$
have *agrees*:
 $\text{opening-index opn} < \text{length composition-table} \wedge$
 $\text{composition-table} ! \text{opening-index opn} = \text{opening-value opn}$
 by $(\text{rule merkle-bound-table-agrees-with-authenticated-opening-if-clean}$
 $[OF \text{ bind len-pow opn-auth-final root - clean}])$
 $(\text{use len opn-len in simp})$
then show *?thesis*
 using *idx val* **by** *simp*
qed

lemma

staged-query-prefix-candidate-pair-query-target-from-prefix-subset-conceptual-default-if-final-merkle-bound:

assumes *prefix-clean*: $\neg \text{hash-map-output-collision prefix-state}$
and *final-clean*: $\neg \text{hash-map-output-collision final-state}$
and *ext*: $\text{prefix-state} \leq \text{final-state}$
and *trace-bind*:
 $\text{merkle-root-binds-table } (\text{sqp-trace-root prefix}) \text{ trace-table}$
 final-state
and *composition-bind*:
 $\text{merkle-root-binds-table } (\text{hd } (\text{sqp-composition-fri-roots prefix}))$
 $\text{composition-table final-state}$
and *trace-len*: $\text{length trace-table} = \text{scale} * \text{clength}$
and *composition-len*: $\text{length composition-table} = \text{scale} * \text{clength}$
shows

```

    staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
    composition-table prefix prefix-state  $\subseteq$ 
    staged-query-prefix-conceptual-default-query-target trace-table
    composition-table prefix prefix-state
proof (rule
    staged-query-prefix-candidate-pair-query-target-from-prefix-subset-conceptual-default-if-agrees
    [OF prefix-clean trace-len composition-len])
fix i v
assume i-bound:  $i < \text{scale} * \text{clength}$ 
and auth:
    authenticated-value-at prefix-state (sqp-trace-root prefix)
    (scale * clength) i v
show trace-table !  $i = v$ 
by (rule
    trace-table-agrees-with-prefix-authenticated-values-if-final-merkle-bound
    [OF ext final-clean trace-bind trace-len i-bound auth])
next
fix i v
assume i-bound:  $i < \text{scale} * \text{clength}$ 
and auth:
    authenticated-value-at prefix-state
    (hd (sqp-composition-fri-roots prefix)) (scale * clength) i v
show composition-table !  $i = v$ 
by (rule
    composition-table-agrees-with-prefix-authenticated-values-if-final-merkle-bound
    [OF ext final-clean composition-bind composition-len i-bound auth])
qed

lemma
    staged-query-prefix-candidate-opening-query-target-from-prefix-subset-conceptual:
assumes clean:  $\neg \text{hash-map-output-collision prefix-state}$ 
and trace-len:  $\text{length trace-openings} = \text{rounds}$ 
and composition-len:  $\text{length composition-openings} = \text{rounds}$ 
and trace-tables:
     $\bigwedge j. j < \text{rounds} \implies$ 
    partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) (trace-openings ! j) prefix-state
and composition-tables:
     $\bigwedge j. j < \text{rounds} \implies$ 
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
    (scale * clength) (composition-openings ! j) prefix-state
and trace-low:
    trace-table-low-degree
    (query-prefix-trace-conceptual-table prefix prefix-state)
and composition-low:
    composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table prefix prefix-state)
and not-all:
     $\neg \text{all-queries-consistent}$ 

```

```

    (query-prefix-trace-conceptual-table prefix prefix-state)
    (query-prefix-composition-conceptual-table prefix prefix-state)
    (sqp-alphas prefix)
  shows
    staged-query-prefix-candidate-opening-query-target-from-prefix
      trace-openings composition-openings i prefix prefix-state  $\subseteq$ 
      staged-query-prefix-conceptual-query-target prefix prefix-state
  proof -
  have trace-candidate:
    partial-trace-table-candidate
      (query-prefix-trace-conceptual-table prefix prefix-state)
      trace-openings
  by (rule query-prefix-trace-conceptual-table-partial-trace-candidate
      [OF clean trace-len trace-tables])
  have composition-candidate:
    partial-composition-table-candidate
      (query-prefix-composition-conceptual-table prefix prefix-state)
      composition-openings
  by (rule
      query-prefix-composition-conceptual-table-partial-composition-candidate
      [OF clean composition-len composition-tables])
  show ?thesis
  unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
    staged-query-prefix-conceptual-query-target-def
  by (rule partial-query-success-indices-at-subset-query-sampling-success-space
      [OF trace-candidate composition-candidate trace-low composition-low
        not-all])
qed

lemma
  staged-query-prefix-candidate-opening-query-target-from-prefix-subset-conceptual-default:
  assumes clean:  $\neg$  hash-map-output-collision prefix-state
  and trace-len: length trace-openings = rounds
  and composition-len: length composition-openings = rounds
  and trace-tables:
     $\bigwedge j. j < \text{rounds} \implies$ 
    partial-authenticated-table (sqp-trace-root prefix)
      (scale * clength) (trace-openings ! j) prefix-state
  and composition-tables:
     $\bigwedge j. j < \text{rounds} \implies$ 
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) (composition-openings ! j) prefix-state
  and trace-low:
    trace-table-low-degree
      (query-prefix-trace-conceptual-table-with-default prefix
        prefix-state trace-default)
  and composition-low:
    composition-table-low-degree maxDegree
      (query-prefix-composition-conceptual-table-with-default prefix

```

$\text{prefix-state composition-default})$
and *not-all*:
 $\neg \text{all-queries-consistent}$
 $(\text{query-prefix-trace-conceptual-table-with-default prefix}$
 $\text{prefix-state trace-default})$
 $(\text{query-prefix-composition-conceptual-table-with-default prefix}$
 $\text{prefix-state composition-default})$
 $(\text{sqp-alphas prefix})$
shows
 $\text{staged-query-prefix-candidate-opening-query-target-from-prefix}$
 $\text{trace-openings composition-openings } i \text{ prefix prefix-state} \subseteq$
 $\text{staged-query-prefix-conceptual-default-query-target trace-default}$
 $\text{composition-default prefix prefix-state}$
proof –
have *trace-candidate*:
 $\text{partial-trace-table-candidate}$
 $(\text{query-prefix-trace-conceptual-table-with-default prefix prefix-state}$
 $\text{trace-default})$
 trace-openings
by (rule
 $\text{query-prefix-trace-conceptual-table-with-default-partial-trace-candidate}$
 $[\text{OF clean trace-len trace-tables}])$
have *composition-candidate*:
 $\text{partial-composition-table-candidate}$
 $(\text{query-prefix-composition-conceptual-table-with-default prefix}$
 $\text{prefix-state composition-default})$
 $\text{composition-openings}$
by (rule
 $\text{query-prefix-composition-conceptual-table-with-default-partial-composition-candidate}$
 $[\text{OF clean composition-len composition-tables}])$
show *?thesis*
unfolding $\text{staged-query-prefix-candidate-opening-query-target-from-prefix-def}$
 $\text{staged-query-prefix-conceptual-default-query-target-def}$
by (rule $\text{partial-query-success-indices-at-subset-query-sampling-success-space}$
 $[\text{OF trace-candidate composition-candidate trace-low composition-low}$
 $\text{not-all}])$
qed

lemma *checked-staged-query-prefix-conceptual-query-target-hit-bound*:
fixes $C :: \text{prob}$
assumes *raw-bound*: $\text{query-index-raw-preimage-bound}$
and *prefix-bound*:
 $\bigwedge \text{prefix prefix-state.}$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s) \implies$
 $\text{query-future-fresh prefix-state}$
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$

$(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-conceptual-query-target}) s \leq$
 query-error-bound
proof (rule *checked-staged-query-prefix-dynamic-index-hit-bound*[OF *raw-bound*])
fix *prefix prefix-state*
assume *support*:
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s)$
have *future: query-future-fresh prefix-state*
by (rule *prefix-bound*[OF *support*])
show
 $\text{query-future-fresh prefix-state} \wedge$
 $\text{staged-query-prefix-conceptual-query-target prefix prefix-state}$
 $\subseteq \text{query-sample-space} \wedge$
 nnreal
 $(\text{card}$
 $(\text{staged-query-prefix-conceptual-query-target prefix prefix-state})) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq \text{query-error-bound}$
using *future*
 $\text{staged-query-prefix-conceptual-query-target-subset[of prefix prefix-state]}$
 $\text{staged-query-prefix-conceptual-query-target-fraction-bound}$
by *blast*
qed

lemma *checked-staged-query-prefix-conceptual-query-target-hit-bound-from-sampler*:

assumes *prefix-bound*:
 $\bigwedge \text{prefix prefix-state.}$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s) \implies$
 $\text{query-future-fresh prefix-state}$

shows

$\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-conceptual-query-target}) s \leq$
 query-error-bound
by (rule *checked-staged-query-prefix-conceptual-query-target-hit-bound*
[OF *query-index-raw-preimage-bound-from-sampler-wellformed*
prefix-bound])

lemma *checked-staged-query-prefix-conceptual-default-query-target-hit-bound*:

fixes $C :: \text{prob}$
assumes *raw-bound: query-index-raw-preimage-bound*
and *prefix-bound*:
 $\bigwedge \text{prefix prefix-state.}$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s) \implies$

query-future-fresh prefix-state
shows
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-conceptual-default-query-target trace-default
composition-default)) s ≤
query-error-bound
proof (rule *checked-staged-query-prefix-dynamic-index-hit-bound[OF raw-bound]*)
fix *prefix prefix-state*
assume *support:*
Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i) s)
have *future: query-future-fresh prefix-state*
by (rule *prefix-bound[OF support]*)
show
query-future-fresh prefix-state ∧
staged-query-prefix-conceptual-default-query-target trace-default
composition-default prefix prefix-state ⊆ query-sample-space ∧
nnreal
(card
(staged-query-prefix-conceptual-default-query-target trace-default
composition-default prefix prefix-state)) /
nnreal (card query-sample-space) ≤ query-error-bound
using *future*
staged-query-prefix-conceptual-default-query-target-subset
staged-query-prefix-conceptual-default-query-target-fraction-bound
by *blast*
qed

lemma *checked-staged-query-prefix-conceptual-default-query-target-hit-bound-from-sampler:*
assumes *prefix-bound:*
∧ prefix prefix-state.
Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i) s) ⇒
query-future-fresh prefix-state
shows
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-conceptual-default-query-target trace-default
composition-default)) s ≤
query-error-bound
by (rule
checked-staged-query-prefix-conceptual-default-query-target-hit-bound
[OF query-index-raw-preimage-bound-from-sampler-wellformed
prefix-bound])

lemma *checked-staged-query-prefix-conceptual-empty-query-target-hit-bound:*

fixes $C :: \text{prob}$
assumes *raw-bound: query-index-raw-preimage-bound*
and *prefix-bound:*
 $\bigwedge \text{prefix prefix-state.}$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s) \implies$
 $\text{query-future-fresh prefix-state}$
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-conceptual-empty-query-target}) \ s \leq$
 query-error-bound
proof (rule *checked-staged-query-prefix-dynamic-index-hit-bound[OF raw-bound]*)
fix *prefix prefix-state*
assume *support:*
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s)$
have *future: query-future-fresh prefix-state*
by (rule *prefix-bound[OF support]*)
show
 $\text{query-future-fresh prefix-state} \wedge$
 $\text{staged-query-prefix-conceptual-empty-query-target prefix prefix-state}$
 $\subseteq \text{query-sample-space} \wedge$
 nnreal
 $(\text{card}$
 $(\text{staged-query-prefix-conceptual-empty-query-target prefix}$
 $\text{prefix-state})) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq \text{query-error-bound}$
using *future*
 $\text{staged-query-prefix-conceptual-empty-query-target-subset}$
 $\text{staged-query-prefix-conceptual-empty-query-target-fraction-bound}$
by *blast*
qed

lemma *checked-staged-query-prefix-conceptual-empty-query-target-hit-bound-from-sampler:*
assumes *prefix-bound:*
 $\bigwedge \text{prefix prefix-state.}$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s) \implies$
 $\text{query-future-fresh prefix-state}$
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-conceptual-empty-query-target}) \ s \leq$
 query-error-bound
by (rule *checked-staged-query-prefix-conceptual-empty-query-target-hit-bound*)

[*OF query-index-raw-preimage-bound-from-sampler-wellformed
prefix-bound*])

lemma *checked-staged-query-prefix-conceptual-query-target-prehit-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: $i \leq \text{rounds}$*

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)

*(checked-staged-query-prefix-dynamic-index-prehit
staged-query-prefix-conceptual-query-target)*

adversary-initial-state $\leq$

staged-phase-relation-error size

(staged-query-search-queries budgets $i + 1$)

proof (*rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)

show *HashMap adversary-initial-state = fmempty*

by (*simp add: adversary-initial-state-def*)

show *hash-relation-program*

(checked-staged-query-prefix-dynamic-index-prehit-relation A i

adversary-initial-state staged-query-prefix-conceptual-query-target)

size (staged-query-search-queries budgets $i + 1$)

(checked-staged-query-prefix-receive-with-state A i)

by (*rule hash-relation-program-checked-staged-query-prefix-receive-with-state
[OF wf controlled i-bound]*)

(rule

checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)

qed

lemma *checked-staged-query-prefix-conceptual-default-query-target-prehit-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: $i \leq \text{rounds}$*

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)

(checked-staged-query-prefix-dynamic-index-prehit

*(staged-query-prefix-conceptual-default-query-target trace-default
composition-default))*

adversary-initial-state $\leq$

staged-phase-relation-error size

(staged-query-search-queries budgets $i + 1$)

proof (*rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)

show *HashMap adversary-initial-state = fmempty*

by (*simp add: adversary-initial-state-def*)

show *hash-relation-program*

(checked-staged-query-prefix-dynamic-index-prehit-relation A i

adversary-initial-state

*(staged-query-prefix-conceptual-default-query-target trace-default
composition-default))*

size (staged-query-search-queries budgets $i + 1$)

(checked-staged-query-prefix-receive-with-state A i)
 by (rule hash-relation-program-checked-staged-query-prefix-receive-with-state
 [OF wf controlled i -bound])
 (rule
 checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
 qed

lemma checked-staged-query-prefix-conceptual-empty-query-target-prehit-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i -bound: $i \leq \text{rounds}$
shows
 wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-prehit
 staged-query-prefix-conceptual-empty-query-target)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$)
proof (rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program)
show HashMap adversary-initial-state = fmempty
 by (simp add: adversary-initial-state-def)
show hash-relation-program
 (checked-staged-query-prefix-dynamic-index-prehit-relation A i
 adversary-initial-state
 staged-query-prefix-conceptual-empty-query-target)
 size (staged-query-search-queries budgets $i + 1$)
 (checked-staged-query-prefix-receive-with-state A i)
 by (rule hash-relation-program-checked-staged-query-prefix-receive-with-state
 [OF wf controlled i -bound])
 (rule
 checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
 qed

lemma checked-staged-query-prefix-conceptual-query-target-hit-bound-from-budgets:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i -bound: $i < \text{rounds}$
shows
 wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-hit
 staged-query-prefix-conceptual-query-target)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) + query-error-bound
proof –
have raw-bound: query-index-raw-preimage-bound
 by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
have hit-bound:
 wp-event (checked-staged-query-prefix-receive-with-state A i)

$(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\quad \text{staged-query-prefix-conceptual-query-target})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $\quad \text{staged-query-prefix-conceptual-query-target})$
 $\text{adversary-initial-state} + \text{query-error-bound}$

proof (rule
 $\text{checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error}$
 $[\text{OF raw-bound}]$)

fix $\text{prefix prefix-state}$

assume
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\quad \text{adversary-initial-state})$

show
 $\text{staged-query-prefix-conceptual-query-target prefix prefix-state}$
 $\subseteq \text{query-sample-space} \wedge$
 nnreal
 $(\text{card}$
 $\quad (\text{staged-query-prefix-conceptual-query-target prefix}$
 $\quad \text{prefix-state})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$

using $\text{staged-query-prefix-conceptual-query-target-subset}$
 $\text{staged-query-prefix-conceptual-query-target-fraction-bound}$

by blast

qed

have prehit-bound:
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $\quad \text{staged-query-prefix-conceptual-query-target})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1)$

by (rule $\text{checked-staged-query-prefix-conceptual-query-target-prehit-bound}$
 $[\text{OF wf controlled}]$)
 $(\text{use } i\text{-bound in simp})$

show $?thesis$

by (rule $\text{order-trans}[\text{OF hit-bound}]$)
 $(\text{intro add-mono prehit-bound order-refl})$

qed

lemma $\text{checked-staged-query-prefix-conceptual-default-query-target-hit-bound-from-budgets:}$
assumes $\text{wf: staged-budget-wellformed budgets}$
and $\text{controlled: staged-adversary-controlled budgets } A$
and $i\text{-bound: } i < \text{rounds}$
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$

(checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-conceptual-default-query-target trace-default
 composition-default))
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) + query-error-bound
proof –
have raw-bound: query-index-raw-preimage-bound
by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
have hit-bound:
 wp-event (checked-staged-query-prefix-receive-with-state $A \ i$)
 (checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-conceptual-default-query-target trace-default
 composition-default))
 adversary-initial-state $\leq$
 wp-event (checked-staged-query-prefix-receive-with-state $A \ i$)
 (checked-staged-query-prefix-dynamic-index-prehit
 (staged-query-prefix-conceptual-default-query-target trace-default
 composition-default))
 adversary-initial-state + query-error-bound
proof (rule
 checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
 [OF raw-bound])
fix prefix prefix-state
assume
 Some ((prefix, prefix-state), prefix-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-with-state $A \ i$)
 adversary-initial-state)
show
 staged-query-prefix-conceptual-default-query-target trace-default
 composition-default prefix prefix-state $\subseteq$ query-sample-space $\wedge$
 nnreal
 (card
 (staged-query-prefix-conceptual-default-query-target trace-default
 composition-default prefix prefix-state)) /
 nnreal (card query-sample-space) $\leq$ query-error-bound
using staged-query-prefix-conceptual-default-query-target-subset
 staged-query-prefix-conceptual-default-query-target-fraction-bound
by blast
qed
have prehit-bound:
 wp-event (checked-staged-query-prefix-receive-with-state $A \ i$)
 (checked-staged-query-prefix-dynamic-index-prehit
 (staged-query-prefix-conceptual-default-query-target trace-default
 composition-default))
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$)

```

by (rule
  checked-staged-query-prefix-conceptual-default-query-target-prehit-bound
  [OF wf controlled])
  (use i-bound in simp)
show ?thesis
by (rule order-trans[OF hit-bound])
  (intro add-mono prehit-bound order-refl)
qed

lemma checked-staged-query-prefix-conceptual-empty-query-target-hit-bound-from-budgets:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  shows
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-hit
        staged-query-prefix-conceptual-empty-query-target)
      adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) + query-error-bound
  proof -
    have raw-bound: query-index-raw-preimage-bound
    by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
    have hit-bound:
      wp-event (checked-staged-query-prefix-receive-with-state A i)
        (checked-staged-query-prefix-dynamic-index-hit
          staged-query-prefix-conceptual-empty-query-target)
        adversary-initial-state ≤
      wp-event (checked-staged-query-prefix-receive-with-state A i)
        (checked-staged-query-prefix-dynamic-index-prehit
          staged-query-prefix-conceptual-empty-query-target)
        adversary-initial-state + query-error-bound
    proof (rule
      checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
      [OF raw-bound])
      fix prefix prefix-state
      assume
        Some ((prefix, prefix-state), prefix-state) ∈
          set-dist
          (execute (checked-staged-query-prefix-with-state A i)
            adversary-initial-state)
      show
        staged-query-prefix-conceptual-empty-query-target prefix prefix-state
          ⊆ query-sample-space ∧
        nnreal
          (card
            (staged-query-prefix-conceptual-empty-query-target prefix
              prefix-state)) /
          nnreal (card query-sample-space) ≤ query-error-bound
    qed
  qed

```

using *staged-query-prefix-conceptual-empty-query-target-subset*
staged-query-prefix-conceptual-empty-query-target-fraction-bound
by *blast*
qed
have *prehit-bound*:
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-prehit
staged-query-prefix-conceptual-empty-query-target)
adversary-initial-state ≤
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1)
by (rule
checked-staged-query-prefix-conceptual-empty-query-target-prehit-bound
[OF wf controlled])
(use i-bound in simp)
show *?thesis*
by (rule *order-trans[OF hit-bound]*)
(intro add-mono prehit-bound order-refl)
qed

definition *checked-staged-security-with-query-prefix-conceptual-target-hit*
where
checked-staged-security-with-query-prefix-conceptual-target-hit out $\longleftrightarrow$
checked-staged-security-with-query-prefix-dynamic-index-hit
staged-query-prefix-conceptual-query-target out

definition *checked-staged-security-with-query-prefix-conceptual-default-target-hit*
where
checked-staged-security-with-query-prefix-conceptual-default-target-hit
trace-default composition-default out $\longleftrightarrow$
checked-staged-security-with-query-prefix-dynamic-index-hit
(staged-query-prefix-conceptual-default-query-target trace-default
composition-default) out

definition *checked-staged-security-with-query-prefix-conceptual-empty-target-hit*
where
checked-staged-security-with-query-prefix-conceptual-empty-target-hit out
 $\longleftrightarrow$
checked-staged-security-with-query-prefix-dynamic-index-hit
staged-query-prefix-conceptual-empty-query-target out

lemma *checked-staged-security-with-query-prefix-conceptual-target-hit-bound*:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
shows
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
checked-staged-security-with-query-prefix-conceptual-target-hit

adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) + query-error-bound
unfolding *checked-staged-security-with-query-prefix-conceptual-target-hit-def*
by (rule *checked-staged-security-with-query-prefix-dynamic-index-hit-bound*)
 (rule *checked-staged-query-prefix-conceptual-query-target-hit-bound-from-budgets*
 [OF wf controlled i-bound])

lemma *checked-staged-security-with-query-prefix-conceptual-default-target-hit-bound:*

assumes wf: *staged-budget-wellformed budgets*
and controlled: *staged-adversary-controlled budgets A*
and i-bound: *i < rounds*

shows

wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-conceptual-default-target-hit*
trace-default composition-default)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) + query-error-bound

unfolding

checked-staged-security-with-query-prefix-conceptual-default-target-hit-def
by (rule *checked-staged-security-with-query-prefix-dynamic-index-hit-bound*)
 (rule
checked-staged-query-prefix-conceptual-default-query-target-hit-bound-from-budgets
 [OF wf controlled i-bound])

lemma *checked-staged-security-with-query-prefix-conceptual-empty-target-hit-bound:*

assumes wf: *staged-budget-wellformed budgets*
and controlled: *staged-adversary-controlled budgets A*
and i-bound: *i < rounds*

shows

wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
checked-staged-security-with-query-prefix-conceptual-empty-target-hit
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) + query-error-bound

unfolding

checked-staged-security-with-query-prefix-conceptual-empty-target-hit-def
by (rule *checked-staged-security-with-query-prefix-dynamic-index-hit-bound*)
 (rule
checked-staged-query-prefix-conceptual-empty-query-target-hit-bound-from-budgets
 [OF wf controlled i-bound])

end

end

```

theory Soundness-Conceptual-Query-Current
imports
  Soundness-Conceptual-Query-Target
  Staged-Security-Experiment-Composition-Query-Current
begin

```

Connect the existing prefix-authenticated query target to the conceptual prefix tables. This is a deterministic bridge: it only uses openings that are already authenticated in the query-prefix state, so the resulting target remains fixed before the query-index challenge is sampled.

```

context soundness
begin

```

```

definition staged-query-prefix-prefix-authenticated-empty-opening-target
  :: 'f staged-query-prefix-data  $\Rightarrow$  'f protocol-channel  $\Rightarrow$  nat set
where

```

```

  staged-query-prefix-prefix-authenticated-empty-opening-target
    prefix prefix-state =
    {idx  $\in$  query-sample-space.
      $\exists$ (trace-openings :: 'f authenticated-opening list).
     partial-authenticated-table (sqp-trace-root prefix)
       (scale * clength) trace-openings prefix-state  $\wedge$ 
     map opening-index trace-openings = powers-scaled idx  $\wedge$ 
     cp-eval (sqp-alphas prefix) (map opening-value trace-openings)
       (h  $\wedge$  idx * shift) = sqp-composition-final prefix}

```

```

lemma staged-query-prefix-prefix-authenticated-empty-opening-target-subset:
  staged-query-prefix-prefix-authenticated-empty-opening-target
    prefix prefix-state  $\subseteq$  query-sample-space
unfolding staged-query-prefix-prefix-authenticated-empty-opening-target-def
by blast

```

```

definition checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit
where

```

```

  checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit
    trace-openings i out  $\longleftrightarrow$ 
    (case out of
     None  $\Rightarrow$  False
     | Some (((((prefix, prefix-state), raw), raw-state),
                ((data, attacker-state), result)), final-state)  $\Rightarrow$ 
       partial-authenticated-table (sqp-trace-root prefix)
         (scale * clength) trace-openings prefix-state  $\wedge$ 
       map opening-index trace-openings = powers-scaled (index (to-nat raw))  $\wedge$ 
       cp-eval (sqp-alphas prefix) (map opening-value trace-openings)
         (h  $\wedge$  index (to-nat raw) * shift) =
       sqp-composition-final prefix)

```

```

definition checked-staged-security-with-query-prefix-empty-authenticated-opening-hit

```


where

checked-staged-security-with-query-prefix-empty-authenticated-opening-hit
trace-openings i out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ *False*
 | *Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*) $\Rightarrow$
 partial-authenticated-table (*sqp-trace-root prefix*)
 (*scale* * *clength*) *trace-openings final-state* $\wedge$
 map opening-index trace-openings = *powers-scaled* (*index* (*to-nat raw*)) $\wedge$
 cp-eval (*sqp-alphas prefix*) (*map opening-value trace-openings*)
 (*h* $\wedge$ *index* (*to-nat raw*) * *shift*) =
 sqp-composition-final prefix)

definition *checked-staged-security-with-query-prefix-empty-trace-path-output-hit*

where

checked-staged-security-with-query-prefix-empty-trace-path-output-hit
trace-openings out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ *False*
 | *Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*) $\Rightarrow$
 ($\exists$ *opening* $\in$ *set trace-openings*.
 hash-map-new-output-hit
 (*merkle-path-target-roots final-state*
 (*sqp-trace-root prefix*)
 (*scale* * *clength*)
 (*opening-index opening*)
 (*opening-value opening*)
 (*opening-path opening*))
 prefix-state final-state))

definition *checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at*

where

checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at
i out $\longleftrightarrow$
 ($\exists$ (*trace-openings* :: 'f *authenticated-opening list*).
 checked-staged-security-with-query-prefix-empty-authenticated-opening-hit
 trace-openings i out)

definition *checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at*

where

checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at
i out $\longleftrightarrow$
 ($\exists$ (*trace-openings* :: 'f *authenticated-opening list*).
 checked-staged-security-with-query-prefix-empty-authenticated-opening-hit
 trace-openings i out)

definition *checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at*

where

checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at
i out $\longleftrightarrow$
 $(\exists (trace-openings :: 'f \text{ authenticated-opening list}).$
checked-staged-security-with-query-prefix-empty-trace-path-output-hit
trace-openings out)

definition *checked-staged-security-with-query-prefix-conceptual-empty-context-residual-at*

where

checked-staged-security-with-query-prefix-conceptual-empty-context-residual-at
out $\longleftrightarrow$
(case *out* of
None $\Rightarrow$ *False*
| *Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*) $\Rightarrow$
hash-map-output-collision prefix-state $\vee$
 $\neg$ *trace-table-low-degree*
(*query-prefix-trace-conceptual-table prefix prefix-state*) $\vee$
all-queries-consistent
(*query-prefix-trace-conceptual-table prefix prefix-state*)
(*replicate* (*scale* * *clength*) (*sqp-composition-final prefix*))
(*sqp-alphas prefix*))

definition *checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at*

where

checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
(*i :: nat*) *out* $\longleftrightarrow$
checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at
i out $\wedge$
checked-staged-security-with-query-prefix-conceptual-empty-context-residual-at
out

lemma *checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at-imp-dynamic-target:*

assumes *hit:*

checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at
i out

shows

checked-staged-security-with-query-prefix-dynamic-index-hit
staged-query-prefix-authenticated-empty-opening-target out

proof (cases *out*)

case *None*

then show *?thesis*

using *hit*

unfolding

checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at-def
checked-staged-security-with-query-prefix-authenticated-empty-opening-hit-def

by *simp*

next

case (*Some packed*)

```

then obtain prefix prefix-state raw raw-state data attacker-state result
  final-state where out-eq:
  out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
from hit obtain trace-openings where prefix-hit:
  checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit
    trace-openings i out
unfolding
  checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at-def
by blast
have idx-in:
  index (to-nat raw) ∈
    staged-query-prefix-prefix-authenticated-empty-opening-target
    prefix prefix-state
using prefix-hit index-less-query-sample-space
unfolding out-eq
  checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit-def
  staged-query-prefix-prefix-authenticated-empty-opening-target-def
  query-sample-space-def
by auto
show ?thesis
unfolding out-eq
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
  checked-staged-query-prefix-dynamic-index-hit-def
using idx-in by simp
qed

lemma checked-staged-security-with-query-prefix-empty-authenticated-opening-hit-imp-prefix-or-path-output-hit:
assumes ext: prefix-state ≤ final-state
and hit:
  checked-staged-security-with-query-prefix-empty-authenticated-opening-hit
    trace-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
shows
  checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit
    trace-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)) ∨
  checked-staged-security-with-query-prefix-empty-trace-path-output-hit
    trace-openings
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))

proof –
have trace-table:
  partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) trace-openings final-state

```

```

and trace-indices:
  map opening-index trace-openings = powers-scaled (index (to-nat raw))
and consistent:
  cp-eval (sqp-alphas prefix) (map opening-value trace-openings)
    (h  $\wedge$  index (to-nat raw) * shift) =
    sqp-composition-final prefix
using hit
unfolding
  checked-staged-security-with-query-prefix-empty-authenticated-opening-hit-def
by simp-all
have trace-pullback:
  partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) trace-openings prefix-state  $\vee$ 
    ( $\exists$  opn  $\in$  set trace-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots final-state (sqp-trace-root prefix)
        (scale * clength)
        (opening-index opn)
        (opening-value opn)
        (opening-path opn)) prefix-state final-state)
by (rule partial-authenticated-table-pullback-or-new-output-hit
    [OF ext trace-table])
then show ?thesis
proof
  assume trace-prefix:
    partial-authenticated-table (sqp-trace-root prefix)
      (scale * clength) trace-openings prefix-state
  then have
    checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit
      trace-openings i
      (Some ((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
  using trace-indices consistent
  unfolding
    checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit-def
  by simp
then show ?thesis by simp
next
assume
  ( $\exists$  opn  $\in$  set trace-openings.
  hash-map-new-output-hit
    (merkle-path-target-roots final-state (sqp-trace-root prefix)
      (scale * clength)
      (opening-index opn)
      (opening-value opn)
      (opening-path opn)) prefix-state final-state)
then have
  checked-staged-security-with-query-prefix-empty-trace-path-output-hit
    trace-openings

```

```

      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
    unfolding
      checked-staged-security-with-query-prefix-empty-trace-path-output-hit-def
    by simp
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-empty-authenticated-opening-hit-imp-prefix-or-path-output-hit-on-su
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and support:
    out ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-query-prefix-data-state A i)
          adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-empty-authenticated-opening-hit
      trace-openings i out
  shows
    checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit
      trace-openings i out ∨
    checked-staged-security-with-query-prefix-empty-trace-path-output-hit
      trace-openings out
proof (cases out)
case None
  then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-query-prefix-empty-authenticated-opening-hit-def
  by simp
next
case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
  have ext: prefix-state ≤ final-state
  by (rule
    checked-staged-security-experiment-with-query-prefix-data-state-prefix-extends-final
    [OF wf controlled i-bound])
  (use support out-eq in simp)
  show ?thesis
  unfolding out-eq

```

by (rule
 checked-staged-security-with-query-prefix-empty-authenticated-opening-hit-imp-prefix-or-path-output-hit
 [OF *ext*])
 (use *hit out-eq* in *simp*)
 qed

lemma *checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at-imp-prefix-or-path-output-hit-on*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *support:*
 out ∈
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 adversary-initial-state)
and *hit:*
 checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at
 i out
shows
 checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at
 i out ∨
 checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at
 i out
proof —
from *hit* **obtain** *trace-openings* **where** *trace-hit:*
 checked-staged-security-with-query-prefix-empty-authenticated-opening-hit
 trace-openings i out
unfolding
 checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at-def
by *blast*
have *split:*
 checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit
 trace-openings i out ∨
 checked-staged-security-with-query-prefix-empty-trace-path-output-hit
 trace-openings out
by (rule
 checked-staged-security-with-query-prefix-empty-authenticated-opening-hit-imp-prefix-or-path-output-hit-on-suppo
 [OF *wf controlled i-bound support trace-hit*])
then show *?thesis*
unfolding
 checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at-def
 checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at-def
by *blast*
 qed

lemma *low-degree-all-queries-consistent-imp-composition-trace-bad-alpha-space:*
assumes *false-statement: ¬ exists-valid-trace*
and *len-as: length as = length spec*

and *trace-low*: *trace-table-low-degree trace-table*
and *composition-low*:
 composition-table-low-degree maxDegree composition-table
and *all-queries*:
 all-queries-consistent trace-table composition-table as
shows *as* $\in$ *composition-trace-bad-alpha-space trace-table*
proof –
from *trace-low* **obtain** *f* **where** *deg-f*: *degree f < clength*
and *trace-table*: *trace-table = map (poly f) eval-domain*
unfolding *trace-table-low-degree-def* **by** *blast*
have *bounds*: *common-denominator-degree-bounds f as*
by (*rule common-denominator-degree-bounds-from-spec-degree-wellformed*
 [*OF spec-degree-wellformed-from-spec-query-margin deg-f*])
have *hides*:
 random-combination-common-denominator-hides-violations f as
by (*rule random-combination-common-denominator-hides-from-queries*
 [*OF len-as false-statement deg-f trace-table composition-low*
 all-queries bounds])
have *violated*: *violated-constraints f $\neq$ {}*
by (*rule false-statement-violated-constraints*
 [*OF false-statement deg-f*])
have *as-common*: *as* $\in$ *common-denominator-hiding-alpha-space f*
using *len-as hides*
unfolding *common-denominator-hiding-alpha-space-def alpha-space-def*
by *simp*
have *witness-eq*: *low-degree-trace-witness trace-table = f*
by (*rule low-degree-trace-witness-eq*[*OF deg-f trace-table*])
show *?thesis*
 using *deg-f trace-table violated bounds as-common*
 unfolding *composition-trace-bad-alpha-space-def witness-eq*
 by *simp*
qed

lemma *staged-query-prefix-prefix-authenticated-empty-opening-target-subset-conceptual-empty*:
assumes *clean*: $\neg$ *hash-map-output-collision prefix-state*
and *trace-low*:
 trace-table-low-degree
 (*query-prefix-trace-conceptual-table prefix prefix-state*)
and *not-all*:
 $\neg$ *all-queries-consistent*
 (*query-prefix-trace-conceptual-table prefix prefix-state*)
 (*replicate (scale * clength) (sqp-composition-final prefix)*)
 (*sqp-alphas prefix*)
shows
 staged-query-prefix-prefix-authenticated-empty-opening-target
 prefix prefix-state $\subseteq$
 staged-query-prefix-conceptual-empty-query-target prefix prefix-state
proof
fix *idx*

```

assume idx-in:
  idx ∈
    staged-query-prefix-prefix-authenticated-empty-opening-target
    prefix prefix-state
then obtain trace-openings where
  idx-sample: idx ∈ query-sample-space
and trace-table:
  partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) trace-openings prefix-state
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and final-eq:
  cp-eval (sqp-alphas prefix) (map opening-value trace-openings)
    (h ^ idx * shift) = sqp-composition-final prefix
unfolding staged-query-prefix-prefix-authenticated-empty-opening-target-def
by blast
let ?trace-table =
  query-prefix-trace-conceptual-table prefix prefix-state
let ?composition-table =
  replicate (scale * clength) (sqp-composition-final prefix)
have idx-bound: idx < clength * scale
  by (rule query-sample-space-less-domain[OF idx-sample])
have trace-values:
  map (!) ?trace-table (powers-scaled idx) =
    map opening-value trace-openings
proof (rule nth-equalityI)
show
  length (map (!) ?trace-table (powers-scaled idx)) =
    length (map opening-value trace-openings)
  using trace-indices by (metis length-map)
next
fix k
assume k-bound:
  k < length (map (!) ?trace-table (powers-scaled idx))
then have ps-bound: k < length (powers-scaled idx)
  by simp
have opn-bound: k < length trace-openings
  using trace-indices ps-bound by (metis length-map)
have opn-in: trace-openings ! k ∈ set trace-openings
  by (rule nth-mem[OF opn-bound])
have idx-eq:
  opening-index (trace-openings ! k) = powers-scaled idx ! k
  using trace-indices ps-bound opn-bound
  by (metis nth-map)
have conceptual:
  ?trace-table ! opening-index (trace-openings ! k) =
    opening-value (trace-openings ! k)
  by (rule
    query-prefix-trace-conceptual-table-agrees-with-partial-authenticated-table

```



```

    [OF clean trace-table opn-in])
  show
    map (!) ?trace-table (powers-scaled idx) ! k =
      map opening-value trace-openings ! k
    using ps-bound opn-bound idx-eq conceptual by simp
  qed
  have query-consistent:
    query-consistent-at ?trace-table ?composition-table
      (sqp-alphas prefix) idx
  proof -
    have composition-value:
      ?composition-table ! idx = sqp-composition-final prefix
    using idx-bound by (simp add: mult.commute)
    have cp-eq:
      cp-eval (sqp-alphas prefix)
        (map (!) ?trace-table (powers-scaled idx))
        (h ^ idx * shift) =
          sqp-composition-final prefix
    using trace-values final-eq by simp
  show ?thesis
    unfolding query-consistent-at-def
    using idx-bound query-sample-space-powers-scaled-bound[OF idx-sample]
      composition-value cp-eq by (simp add: mult.commute)
  qed
  show
    idx ∈
      staged-query-prefix-conceptual-empty-query-target prefix prefix-state
    unfolding staged-query-prefix-conceptual-empty-query-target-def
      query-sampling-success-space-def query-agreement-indices-def
    using idx-sample trace-low constant-composition-table-low-degree not-all
      query-consistent by simp
  qed

  lemma checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-imp-conceptual-empty-or-residual-hit:
  assumes hit:
    checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at
      i out
  shows
    checked-staged-security-with-query-prefix-conceptual-empty-target-hit out ∨
    checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
      i out
  proof (cases out)
    case None
    then show ?thesis
      using hit
      unfolding
        checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at-def
        checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit-def
      by simp
  end

```

```

next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
  show ?thesis
  proof (cases
    ¬ hash-map-output-collision prefix-state ∧
    trace-table-low-degree
    (query-prefix-trace-conceptual-table prefix prefix-state) ∧
    ¬ all-queries-consistent
    (query-prefix-trace-conceptual-table prefix prefix-state)
    (replicate (scale * clength) (sqp-composition-final prefix))
    (sqp-alphas prefix))
  case False
  then have
    checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
    i out
  unfolding
    checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at-def
    checked-staged-security-with-query-prefix-conceptual-empty-context-residual-at-def
    out-eq
  using hit out-eq by simp
  then show ?thesis by simp
next
  case True
  then have clean:
    ¬ hash-map-output-collision prefix-state
  and trace-low:
    trace-table-low-degree
    (query-prefix-trace-conceptual-table prefix prefix-state)
  and not-all:
    ¬ all-queries-consistent
    (query-prefix-trace-conceptual-table prefix prefix-state)
    (replicate (scale * clength) (sqp-composition-final prefix))
    (sqp-alphas prefix)
  by blast+
  have prefix-hit:
    checked-staged-security-with-query-prefix-dynamic-index-hit
    staged-query-prefix-prefix-authenticated-empty-opening-target out
  by (rule
    checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at-imp-dynamic-target
    [OF hit])
  have subset:
    staged-query-prefix-prefix-authenticated-empty-opening-target
    prefix prefix-state ⊆

```

```

    staged-query-prefix-conceptual-empty-query-target prefix prefix-state
  by (rule
    staged-query-prefix-prefix-authenticated-empty-opening-target-subset-conceptual-empty
    [OF clean trace-low not-all])
  have conceptual-hit:
    checked-staged-security-with-query-prefix-conceptual-empty-target-hit
    out
  using prefix-hit subset
  unfolding out-eq
    checked-staged-security-with-query-prefix-conceptual-empty-target-hit-def
    checked-staged-security-with-query-prefix-dynamic-index-hit-def
    checked-staged-query-prefix-dynamic-index-hit-def
  by auto
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at-bound-from-conceptual-em
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and residual-bound:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
      i)
    adversary-initial-state  $\leq R$ 
  shows
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at
      i)
    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    query-error-bound + R
proof -
  let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i
  let ?Conceptual =
    checked-staged-security-with-query-prefix-conceptual-empty-target-hit
  let ?Residual =
    checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
    i
  have event-le:
    wp-event ?M
    (checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at
      i)
    adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda \text{out. ?Conceptual out} \vee \text{?Residual out}$ )

```

adversary-initial-state
by (rule *wp-event-mono*)
(rule
checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-imp-conceptual-empty-or-residual
have *conceptual-bound*:
wp-event ?*M* ?*Conceptual* *adversary-initial-state* $\leq$
staged-phase-relation-error size
(*staged-query-search-queries budgets* *i* + 1) + *query-error-bound*
by (rule
checked-staged-security-with-query-prefix-conceptual-empty-target-hit-bound
[*OF wf controlled i-bound*])
have *union-bound*:
wp-event ?*M* ($\lambda out. ?Conceptual out \vee ?Residual out$)
adversary-initial-state $\leq$
wp-event ?*M* ?*Conceptual* *adversary-initial-state* +
wp-event ?*M* ?*Residual* *adversary-initial-state*
by (rule *wp-event-union-bound*)
have *sum-bound*:
wp-event ?*M* ?*Conceptual* *adversary-initial-state* +
wp-event ?*M* ?*Residual* *adversary-initial-state* $\leq$
(*staged-phase-relation-error size*
(*staged-query-search-queries budgets* *i* + 1) + *query-error-bound*) +
R
by (intro *add-mono conceptual-bound residual-bound*)
show ?*thesis*
by (rule *order-trans*[*OF event-le*])
(rule *order-trans*[*OF union-bound sum-bound*])
qed

lemma *checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at-bound-from-prefix-and-path*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *i-bound*: *i* < *rounds*

and *prefix-bound*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at*
i)

adversary-initial-state $\leq P$

and *path-bound*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at*
i)

adversary-initial-state $\leq T$

shows

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at*

$i)$
 $\text{adversary-initial-state} \leq P + T$
proof –
let $?M = \text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i$
let $?Prefix =$
 $\text{checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at}$
 i
let $?Path =$
 $\text{checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at}$
 i
have *event-le*:
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } ?M (\lambda \text{out. } ?Prefix \text{ out} \vee ?Path \text{ out})$
 $\text{adversary-initial-state}$
by (rule *wp-event-mono-on-support*)
 $(\text{rule}$
 $\text{checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at-imp-prefix-or-path-output-hit-on-sup}$
 $[OF \text{ wf controlled } i\text{-bound}])$
have *union-bound*:
 $\text{wp-event } ?M (\lambda \text{out. } ?Prefix \text{ out} \vee ?Path \text{ out})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } ?M ?Prefix \text{ adversary-initial-state} +$
 $\text{wp-event } ?M ?Path \text{ adversary-initial-state}$
by (rule *wp-event-union-bound*)
have *sum-bound*:
 $\text{wp-event } ?M ?Prefix \text{ adversary-initial-state} +$
 $\text{wp-event } ?M ?Path \text{ adversary-initial-state} \leq P + T$
by (intro *add-mono prefix-bound path-bound*)
show *?thesis*
by (rule *order-trans*[*OF event-le*])
 $(\text{rule order-trans}[OF \text{ union-bound sum-bound}])$
qed

lemma *staged-query-prefix-prefix-authenticated-opening-target-subset-conceptual*:
assumes *clean*: $\neg \text{hash-map-output-collision prefix-state}$
and *trace-low*:
 $\text{trace-table-low-degree}$
 $(\text{query-prefix-trace-conceptual-table prefix prefix-state})$
and *composition-low*:
 $\text{composition-table-low-degree maxDegree}$
 $(\text{query-prefix-composition-conceptual-table prefix prefix-state})$
and *not-all*:
 $\neg \text{all-queries-consistent}$
 $(\text{query-prefix-trace-conceptual-table prefix prefix-state})$
 $(\text{query-prefix-composition-conceptual-table prefix prefix-state})$
 $(\text{sqp-alphas prefix})$

```

shows
  staged-query-prefix-prefix-authenticated-opening-target prefix
    prefix-state  $\subseteq$ 
    staged-query-prefix-conceptual-query-target prefix prefix-state
proof
  fix idx
  assume idx-in:
    idx  $\in$  staged-query-prefix-prefix-authenticated-opening-target prefix
    prefix-state
  then obtain trace-openings composition-openings where
    trace-table:
      partial-authenticated-table (sqp-trace-root prefix)
        (scale * clength) trace-openings prefix-state
    and composition-table:
      partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
        (scale * clength) composition-openings prefix-state
    and consistent:
      partial-query-openings-consistent trace-openings composition-openings
        (sqp-alphas prefix) idx
    unfolding staged-query-prefix-prefix-authenticated-opening-target-def
    by blast
  let ?trace-openings =
    (replicate rounds []) [0 := trace-openings]
  let ?composition-openings =
    (replicate rounds []) [0 := composition-openings]
  have zero-bound: 0 < rounds
    by (rule rounds-positive)
  have trace-len: length ?trace-openings = rounds
    by simp
  have composition-len: length ?composition-openings = rounds
    by simp
  have trace-tables:
     $\bigwedge j. j < \text{rounds} \implies$ 
      partial-authenticated-table (sqp-trace-root prefix)
        (scale * clength) (?trace-openings ! j) prefix-state
  proof –
    fix j
    assume j-bound: j < rounds
    show
      partial-authenticated-table (sqp-trace-root prefix)
        (scale * clength) (?trace-openings ! j) prefix-state
    proof (cases j = 0)
      case True
        then show ?thesis
          using trace-table zero-bound by simp
      next
        case False
          then have ?trace-openings ! j = []
            using j-bound by simp

```

```

    then show ?thesis
      unfolding partial-authenticated-table-def by simp
    qed
  qed
  have composition-tables:
     $\bigwedge j. j < \text{rounds} \implies$ 
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) (?composition-openings ! j) prefix-state
  proof -
    fix j
    assume j-bound:  $j < \text{rounds}$ 
    show
      partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
        (scale * clength) (?composition-openings ! j) prefix-state
    proof (cases  $j = 0$ )
      case True
      then show ?thesis
        using composition-table zero-bound by simp
    next
      case False
      then have ?composition-openings ! j = []
        using j-bound by simp
      then show ?thesis
        unfolding partial-authenticated-table-def by simp
    qed
  qed
  have opening-target:
    idx  $\in$ 
    staged-query-prefix-candidate-opening-query-target-from-prefix
      ?trace-openings ?composition-openings 0 prefix prefix-state
  unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
  by (rule partial-query-success-indices-atI,
    rule partial-query-round-consistent-single-roundI
      [OF zero-bound consistent])
  have target-subset:
    staged-query-prefix-candidate-opening-query-target-from-prefix
      ?trace-openings ?composition-openings 0 prefix prefix-state  $\subseteq$ 
    staged-query-prefix-conceptual-query-target prefix prefix-state
  by (rule
    staged-query-prefix-candidate-opening-query-target-from-prefix-subset-conceptual
      [OF clean trace-len composition-len trace-tables composition-tables
        trace-low composition-low not-all])
  show
    idx  $\in$  staged-query-prefix-conceptual-query-target prefix prefix-state
  using opening-target target-subset by blast
  qed

  lemma staged-query-prefix-prefix-authenticated-opening-target-fraction-bound-if-conceptual:
    assumes clean:  $\neg \text{hash-map-output-collision prefix-state}$ 

```

```

and trace-low:
  trace-table-low-degree
    (query-prefix-trace-conceptual-table prefix prefix-state)
and composition-low:
  composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table prefix prefix-state)
and not-all:
   $\neg$  all-queries-consistent
    (query-prefix-trace-conceptual-table prefix prefix-state)
    (query-prefix-composition-conceptual-table prefix prefix-state)
    (sqp-alphas prefix)
shows
  nnreal
    (card
      (staged-query-prefix-prefix-authenticated-opening-target prefix
        prefix-state)) /
    nnreal (card query-sample-space)  $\leq$  query-error-bound
proof –
  have subset:
    staged-query-prefix-prefix-authenticated-opening-target prefix
      prefix-state  $\subseteq$ 
      staged-query-prefix-conceptual-query-target prefix prefix-state
  by (rule
    staged-query-prefix-prefix-authenticated-opening-target-subset-conceptual
    [OF clean trace-low composition-low not-all])
  have finite-conceptual:
    finite (staged-query-prefix-conceptual-query-target prefix prefix-state)
  by (rule finite-subset
    [OF staged-query-prefix-conceptual-query-target-subset
      finite-query-sample-space])
  have card-le:
    card
      (staged-query-prefix-prefix-authenticated-opening-target prefix
        prefix-state)  $\leq$ 
      card (staged-query-prefix-conceptual-query-target prefix prefix-state)
  by (rule card-mono[OF finite-conceptual subset])
  have frac-le:
    nnreal
      (card
        (staged-query-prefix-prefix-authenticated-opening-target prefix
          prefix-state)) /
      nnreal (card query-sample-space)  $\leq$ 
      nnreal
        (card
          (staged-query-prefix-conceptual-query-target prefix prefix-state)) /
        nnreal (card query-sample-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
show ?thesis
  by (rule order-trans[OF frac-le

```



```

    staged-query-prefix-conceptual-query-target-fraction-bound])
qed

lemma staged-query-prefix-prefix-authenticated-opening-target-subset-conceptual-default:
assumes clean:  $\neg$  hash-map-output-collision prefix-state
and trace-low:
  trace-table-low-degree
  (query-prefix-trace-conceptual-table-with-default prefix
   prefix-state trace-default)
and composition-low:
  composition-table-low-degree maxDegree
  (query-prefix-composition-conceptual-table-with-default prefix
   prefix-state composition-default)
and not-all:
   $\neg$  all-queries-consistent
  (query-prefix-trace-conceptual-table-with-default prefix
   prefix-state trace-default)
  (query-prefix-composition-conceptual-table-with-default prefix
   prefix-state composition-default)
  (sqp-alphas prefix)
shows
  staged-query-prefix-prefix-authenticated-opening-target prefix
  prefix-state  $\subseteq$ 
  staged-query-prefix-conceptual-default-query-target trace-default
  composition-default prefix prefix-state
proof
fix idx
assume idx-in:
  idx  $\in$  staged-query-prefix-prefix-authenticated-opening-target prefix
  prefix-state
then obtain trace-openings composition-openings where
  trace-table:
    partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) trace-openings prefix-state
and composition-table:
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
    (scale * clength) composition-openings prefix-state
and consistent:
    partial-query-openings-consistent trace-openings composition-openings
    (sqp-alphas prefix) idx
unfolding staged-query-prefix-prefix-authenticated-opening-target-def
by blast
let ?trace-openings =
  (replicate rounds []) [0 := trace-openings]
let ?composition-openings =
  (replicate rounds []) [0 := composition-openings]
have zero-bound: 0 < rounds
by (rule rounds-positive)
have trace-len: length ?trace-openings = rounds

```

```

  by simp
have composition-len: length ?composition-openings = rounds
  by simp
have trace-tables:
   $\bigwedge j. j < rounds \implies$ 
    partial-authenticated-table (sqp-trace-root prefix)
      (scale * clength) (?trace-openings ! j) prefix-state
proof -
  fix j
  assume j-bound: j < rounds
  show
    partial-authenticated-table (sqp-trace-root prefix)
      (scale * clength) (?trace-openings ! j) prefix-state
proof (cases j = 0)
  case True
  then show ?thesis
    using trace-table zero-bound by simp
next
  case False
  then have ?trace-openings ! j = []
    using j-bound by simp
  then show ?thesis
    unfolding partial-authenticated-table-def by simp
qed
qed
have composition-tables:
   $\bigwedge j. j < rounds \implies$ 
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) (?composition-openings ! j) prefix-state
proof -
  fix j
  assume j-bound: j < rounds
  show
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) (?composition-openings ! j) prefix-state
proof (cases j = 0)
  case True
  then show ?thesis
    using composition-table zero-bound by simp
next
  case False
  then have ?composition-openings ! j = []
    using j-bound by simp
  then show ?thesis
    unfolding partial-authenticated-table-def by simp
qed
qed
have opening-target:
  idx  $\in$ 

```

staged-query-prefix-candidate-opening-query-target-from-prefix
?trace-openings ?composition-openings 0 prefix prefix-state
unfolding *staged-query-prefix-candidate-opening-query-target-from-prefix-def*
by (rule *partial-query-success-indices-atI*,
rule *partial-query-round-consistent-single-roundI*
[*OF zero-bound consistent*])
have *target-subset*:
staged-query-prefix-candidate-opening-query-target-from-prefix
?trace-openings ?composition-openings 0 prefix prefix-state $\subseteq$
staged-query-prefix-conceptual-default-query-target trace-default
composition-default prefix prefix-state
by (rule
staged-query-prefix-candidate-opening-query-target-from-prefix-subset-conceptual-default
[*OF clean trace-len composition-len trace-tables composition-tables*
trace-low composition-low not-all])
show
idx $\in$
staged-query-prefix-conceptual-default-query-target trace-default
composition-default prefix prefix-state
using *opening-target target-subset* **by** *blast*
qed

definition *checked-staged-security-with-query-prefix-conceptual-context-residual-at*
where

checked-staged-security-with-query-prefix-conceptual-context-residual-at
out $\longleftrightarrow$
(case *out* of
None $\Rightarrow$ *False*
| *Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*) $\Rightarrow$
hash-map-output-collision prefix-state $\vee$
 $\neg$ *trace-table-low-degree*
(*query-prefix-trace-conceptual-table prefix prefix-state*) $\vee$
 $\neg$ *composition-table-low-degree maxDegree*
(*query-prefix-composition-conceptual-table prefix prefix-state*) $\vee$
all-queries-consistent
(*query-prefix-trace-conceptual-table prefix prefix-state*)
(*query-prefix-composition-conceptual-table prefix prefix-state*)
(*sqp-alphas prefix*))

definition *checked-staged-security-with-query-prefix-conceptual-default-context-residual-at*
where

checked-staged-security-with-query-prefix-conceptual-default-context-residual-at
trace-default composition-default out $\longleftrightarrow$
(case *out* of
None $\Rightarrow$ *False*
| *Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*) $\Rightarrow$
hash-map-output-collision prefix-state $\vee$

$\neg$ *trace-table-low-degree*
 (query-prefix-trace-conceptual-table-with-default prefix
 prefix-state trace-default) $\vee$
 $\neg$ *composition-table-low-degree* maxDegree
 (query-prefix-composition-conceptual-table-with-default prefix
 prefix-state composition-default) $\vee$
all-queries-consistent
 (query-prefix-trace-conceptual-table-with-default prefix
 prefix-state trace-default)
 (query-prefix-composition-conceptual-table-with-default prefix
 prefix-state composition-default)
 (sqp-alphas prefix))

lemma *checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-imp-conceptual-or-residual-on-suppo*

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *i-bound*: $i < \text{rounds}$

and *support*:

out $\in$

set-dist

(execute

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

adversary-initial-state)

and *hit*:

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at

i out

shows

checked-staged-security-with-query-prefix-conceptual-target-hit *out* $\vee$

checked-staged-security-with-query-prefix-conceptual-context-residual-at

out

proof (cases *out*)

case *None*

then show ?thesis

using *hit*

unfolding

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-def

checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-def

by *simp*

next

case (Some *packed*)

then obtain prefix prefix-state raw raw-state data attacker-state result

final-state **where** *out-eq*:

out =

Some (((((prefix, prefix-state), raw), raw-state),

((data, attacker-state), result)), final-state)

by (cases *packed*) (auto split: prod.splits)

show ?thesis

proof (cases

$\neg$ hash-map-output-collision prefix-state $\wedge$

```

    trace-table-low-degree
      (query-prefix-trace-conceptual-table prefix prefix-state) ∧
    composition-table-low-degree maxDegree
      (query-prefix-composition-conceptual-table prefix prefix-state) ∧
    ¬ all-queries-consistent
      (query-prefix-trace-conceptual-table prefix prefix-state)
      (query-prefix-composition-conceptual-table prefix prefix-state)
      (sqp-alphas prefix))
  case False
  then have
    checked-staged-security-with-query-prefix-conceptual-context-residual-at
      out
    unfolding out-eq
      checked-staged-security-with-query-prefix-conceptual-context-residual-at-def
    by simp
  then show ?thesis by simp
next
case True
then have clean:
  ¬ hash-map-output-collision prefix-state
and trace-low:
  trace-table-low-degree
    (query-prefix-trace-conceptual-table prefix prefix-state)
and composition-low:
  composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table prefix prefix-state)
and not-all:
  ¬ all-queries-consistent
    (query-prefix-trace-conceptual-table prefix prefix-state)
    (query-prefix-composition-conceptual-table prefix prefix-state)
    (sqp-alphas prefix)
  by blast+
have prefix-hit:
  checked-staged-security-with-query-prefix-dynamic-index-hit
    staged-query-prefix-prefix-authenticated-opening-target out
  by (rule
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-imp-dynamic-target
      [OF wf controlled i-bound support hit])
have subset:
  staged-query-prefix-prefix-authenticated-opening-target prefix
    prefix-state ⊆
  staged-query-prefix-conceptual-query-target prefix prefix-state
  by (rule
    staged-query-prefix-prefix-authenticated-opening-target-subset-conceptual
      [OF clean trace-low composition-low not-all])
have conceptual-hit:
  checked-staged-security-with-query-prefix-conceptual-target-hit out
  using prefix-hit subset
  unfolding out-eq

```

```

    checked-staged-security-with-query-prefix-conceptual-target-hit-def
    checked-staged-security-with-query-prefix-dynamic-index-hit-def
    checked-staged-query-prefix-dynamic-index-hit-def
  by auto
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-conceptual-residual:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and residual-bound:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    checked-staged-security-with-query-prefix-conceptual-context-residual-at
    adversary-initial-state  $\leq R$ 
  shows
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
     i)
    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    query-error-bound + R
proof -
  let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i
  let ?Conceptual =
    checked-staged-security-with-query-prefix-conceptual-target-hit
  let ?Residual =
    checked-staged-security-with-query-prefix-conceptual-context-residual-at
  have event-le:
    wp-event ?M
    (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
     i)
    adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda \text{out. ?Conceptual out} \vee \text{?Residual out}$ )
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (rule
   checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-imp-conceptual-or-residual-on-support
   [OF wf controlled i-bound])
  have conceptual-bound:
    wp-event ?M ?Conceptual adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) + query-error-bound
  by (rule
      checked-staged-security-with-query-prefix-conceptual-target-hit-bound

```

$[OF \text{ wf controlled } i\text{-bound}]$
have *union-bound*:
 $wp\text{-event } ?M (\lambda out. ?Conceptual \text{ out} \vee ?Residual \text{ out})$
 $adversary\text{-initial-state} \leq$
 $wp\text{-event } ?M ?Conceptual \text{ adversary-initial-state} +$
 $wp\text{-event } ?M ?Residual \text{ adversary-initial-state}$
by (*rule wp-event-union-bound*)
have *sum-bound*:
 $wp\text{-event } ?M ?Conceptual \text{ adversary-initial-state} +$
 $wp\text{-event } ?M ?Residual \text{ adversary-initial-state} \leq$
 $(staged\text{-phase-relation-error } size$
 $(staged\text{-query-search-queries } budgets \ i + 1) + query\text{-error-bound}) +$
 R
by (*intro add-mono conceptual-bound residual-bound*)
show *?thesis*
by (*rule order-trans[OF event-le]*)
 $(rule \text{ order-trans}[OF \text{ union-bound sum-bound}])$
qed

lemma *checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-imp-conceptual-default-or-residual-c*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *support*:
 $out \in$
 $set\text{-dist}$
 $(execute$
 $(checked\text{-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $adversary\text{-initial-state})$
and *hit*:
 $checked\text{-staged-security-with-query-prefix-current-prefix-authenticated-hit-at}$
 $i \ out$
shows
 $checked\text{-staged-security-with-query-prefix-conceptual-default-target-hit}$
 $trace\text{-default } composition\text{-default } out \vee$
 $checked\text{-staged-security-with-query-prefix-conceptual-default-context-residual-at}$
 $trace\text{-default } composition\text{-default } out$
proof (*cases out*)
case *None*
then show *?thesis*
using *hit*
unfolding
 $checked\text{-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-def}$
 $checked\text{-staged-security-with-query-prefix-prefix-authenticated-opening-hit-def}$
by *simp*
next
case (*Some packed*)
then obtain *prefix prefix-state raw raw-state data attacker-state result*
 $final\text{-state}$ **where** *out-eq*:

```

out =
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)
by (cases packed) (auto split: prod.splits)
show ?thesis
proof (cases
  ¬ hash-map-output-collision prefix-state ∧
  trace-table-low-degree
  (query-prefix-trace-conceptual-table-with-default prefix
    prefix-state trace-default) ∧
  composition-table-low-degree maxDegree
  (query-prefix-composition-conceptual-table-with-default prefix
    prefix-state composition-default) ∧
  ¬ all-queries-consistent
  (query-prefix-trace-conceptual-table-with-default prefix
    prefix-state trace-default)
  (query-prefix-composition-conceptual-table-with-default prefix
    prefix-state composition-default)
  (sqp-alphas prefix))
case False
then have
  checked-staged-security-with-query-prefix-conceptual-default-context-residual-at
    trace-default composition-default out
  unfolding out-eq
    checked-staged-security-with-query-prefix-conceptual-default-context-residual-at-def
  by simp
then show ?thesis by simp
next
case True
then have clean:
  ¬ hash-map-output-collision prefix-state
  and trace-low:
    trace-table-low-degree
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-default)
  and composition-low:
    composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-default)
  and not-all:
    ¬ all-queries-consistent
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-default)
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-default)
    (sqp-alphas prefix)
  by blast+
have prefix-hit:
  checked-staged-security-with-query-prefix-dynamic-index-hit

```



```

    staged-query-prefix-prefix-authenticated-opening-target out
  by (rule
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-imp-dynamic-target
    [OF wf controlled i-bound support hit])
  have subset:
    staged-query-prefix-prefix-authenticated-opening-target prefix
    prefix-state  $\subseteq$ 
    staged-query-prefix-conceptual-default-query-target trace-default
    composition-default prefix prefix-state
  by (rule
    staged-query-prefix-prefix-authenticated-opening-target-subset-conceptual-default
    [OF clean trace-low composition-low not-all])
  have conceptual-hit:
    checked-staged-security-with-query-prefix-conceptual-default-target-hit
    trace-default composition-default out
  using prefix-hit subset
  unfolding out-eq
    checked-staged-security-with-query-prefix-conceptual-default-target-hit-def
    checked-staged-security-with-query-prefix-dynamic-index-hit-def
    checked-staged-query-prefix-dynamic-index-hit-def
  by auto
  then show ?thesis by simp
qed
qed

```

lemma *checked-staged-query-prefix-prefix-authenticated-target-hit-bound-from-conceptual:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *conceptual-context:*

$\bigwedge$ *prefix prefix-state.*

Some ((prefix, prefix-state), prefix-state) $\in$
set-dist

(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) $\implies$

$\neg$ *hash-map-output-collision prefix-state $\wedge$*
trace-table-low-degree

(query-prefix-trace-conceptual-table prefix prefix-state) $\wedge$

composition-table-low-degree maxDegree

(query-prefix-composition-conceptual-table prefix prefix-state) $\wedge$

$\neg$ *all-queries-consistent*

(query-prefix-trace-conceptual-table prefix prefix-state)

(query-prefix-composition-conceptual-table prefix prefix-state)

(sqp-alphas prefix)

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)

(checked-staged-query-prefix-dynamic-index-hit

staged-query-prefix-prefix-authenticated-opening-target)

adversary-initial-state $\leq$

```

    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) + query-error-bound
proof –
  have raw-bound: query-index-raw-preimage-bound
    by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
  have hit-bound:
    wp-event (checked-staged-query-prefix-receive-with-state  $A$   $i$ )
      (checked-staged-query-prefix-dynamic-index-hit
        staged-query-prefix-prefix-authenticated-opening-target)
    adversary-initial-state  $\leq$ 
    wp-event (checked-staged-query-prefix-receive-with-state  $A$   $i$ )
      (checked-staged-query-prefix-dynamic-index-prehit
        staged-query-prefix-prefix-authenticated-opening-target)
    adversary-initial-state + query-error-bound
proof (rule
  checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
  [OF raw-bound])
fix prefix prefix-state
assume support:
  Some ((prefix, prefix-state), prefix-state)  $\in$ 
    set-dist
    (execute (checked-staged-query-prefix-with-state  $A$   $i$ )
      adversary-initial-state)
have ctx:
   $\neg$  hash-map-output-collision prefix-state  $\wedge$ 
  trace-table-low-degree
    (query-prefix-trace-conceptual-table prefix prefix-state)  $\wedge$ 
  composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table prefix prefix-state)  $\wedge$ 
   $\neg$  all-queries-consistent
    (query-prefix-trace-conceptual-table prefix prefix-state)
    (query-prefix-composition-conceptual-table prefix prefix-state)
    (sqp-alphas prefix)
  by (rule conceptual-context[OF support])
have subset:
  staged-query-prefix-prefix-authenticated-opening-target prefix
    prefix-state  $\subseteq$  query-sample-space
  by (rule staged-query-prefix-prefix-authenticated-opening-target-subset)
have frac:
  nnreal
    (card
      (staged-query-prefix-prefix-authenticated-opening-target prefix
        prefix-state)) /
  nnreal (card query-sample-space)  $\leq$  query-error-bound
using ctx
by (intro
  staged-query-prefix-prefix-authenticated-opening-target-fraction-bound-if-conceptual)
  blast+
show

```

```

    staged-query-prefix-prefix-authenticated-opening-target prefix
    prefix-state  $\subseteq$  query-sample-space  $\wedge$ 
    nnreal
    (card
      (staged-query-prefix-prefix-authenticated-opening-target prefix
        prefix-state)) /
    nnreal (card query-sample-space)  $\leq$  query-error-bound
  using subset frac by blast
qed
have prehit-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      staged-query-prefix-prefix-authenticated-opening-target)
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1)
proof (rule
  checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program)
show HashMap adversary-initial-state = fmempty
  by (simp add: adversary-initial-state-def)
show hash-relation-program
  (checked-staged-query-prefix-dynamic-index-prehit-relation A i
    adversary-initial-state
    staged-query-prefix-prefix-authenticated-opening-target)
  size (staged-query-search-queries budgets i + 1)
  (checked-staged-query-prefix-receive-with-state A i)
  by (rule hash-relation-program-checked-staged-query-prefix-receive-with-state
    [OF wf controlled])
  (use i-bound in simp-all,
    rule checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
qed
show ?thesis
  by (rule order-trans[OF hit-bound])
  (intro add-mono prehit-bound order-refl)
qed

end

end

```

```

theory Staged-Security-Experiment-Composition-Query-Current-Path
  imports Staged-Security-Experiment-Composition-Query-Current
begin

```

Structured current-query Merkle path-output events.

The broad *checked-staged-security-with-query-prefix-current-path-output-hit-at* event intentionally forgets why the path was relevant. This layer keeps the authenticated-opening root, length, and final-state authentication evidence, so later Merkle/candidate argu-

ments can distinguish ordinary verifier hashing from candidate drift.

context *soundness*

begin

definition *checked-staged-security-with-query-prefix-transcript-hit*

where

checked-staged-security-with-query-prefix-transcript-hit out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ *False*
 | *Some* ((((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*) $\Rightarrow$
 hash-map-output-values
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\cap$
 set (*staged-proof-transcript data*) $\neq$ {} $\vee$
 hash-map-new-output-hit (*set* (*staged-proof-transcript data*))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 final-state)

definition *checked-staged-security-with-query-prefix-current-trace-path-output-hit-at*

where

checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
i out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ *False*
 | *Some* ((((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*) $\Rightarrow$
 ($\exists$ *opn* :: 'f authenticated-opening.
 opening-root opn = *staged-trace-root data* $\wedge$
 opening-length opn = *scale* * *clength* $\wedge$
 authenticated-opening-in final-state opn $\wedge$
 hash-map-new-output-hit
 (*merkle-path-target-roots final-state*
 (*staged-trace-root data*)
 (*scale* * *clength*)
 (*opening-index opn*)
 (*opening-value opn*)
 (*opening-path opn*))
 prefix-state final-state))

definition *checked-staged-security-with-query-prefix-current-composition-path-output-hit-at*

where

checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
i out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ *False*
 | *Some* ((((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*) $\Rightarrow$

staged-composition-fri-roots data $\neq [] \wedge$
 $(\exists \text{opn} :: 'f \text{ authenticated-opening.}$
 opening-root opn = *hd (staged-composition-fri-roots data)* $\wedge$
 opening-length opn = *scale * clength* $\wedge$
 authenticated-opening-in final-state opn $\wedge$
 hash-map-new-output-hit
 (*merkle-path-target-roots final-state*
 (*hd (staged-composition-fri-roots data)*)
 (*scale * clength*)
 (*opening-index opn*)
 (*opening-value opn*)
 (*opening-path opn*)
 prefix-state final-state)

lemma *checked-staged-security-with-query-prefix-structured-path-hit-imp-transcript-hit-on-support:*

assumes *i-bound: i < rounds*

and *support:*

out $\in$
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 adversary-initial-state)

and *hit:*

checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
 i out $\vee$
checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
 i out

shows *checked-staged-security-with-query-prefix-transcript-hit out*

proof (*cases out*)

case *None*

then show *?thesis*

using *hit*

unfolding

checked-staged-security-with-query-prefix-current-trace-path-output-hit-at-def
checked-staged-security-with-query-prefix-current-composition-path-output-hit-at-def

by *simp*

next

case (*Some packed*)

then obtain *prefix prefix-state raw raw-state data attacker-state result*

final-state **where** *out-eq:*

out =
 Some ((((((*prefix, prefix-state*), *raw*), *raw-state*),
 ((*data, attacker-state*), *result*)), *final-state*)

by (*cases packed*) (*auto split: prod.splits*)

have *mapped-support:*

Some (((*data, attacker-state*), *result*), *final-state*) $\in$

set-dist

 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*) $\gg$

```

      (λpacked. return (snd packed)))
    adversary-initial-state)
  unfolding out-eq
  by (rule set-dist-bindI[OF support[unfolded out-eq]]) simp
have projection:
  checked-staged-security-experiment-with-query-prefix-data-state A i ≫=
    (λpacked. return (snd packed)) =
    checked-staged-security-experiment-with-data-state A
  by (rule checked-staged-security-experiment-with-query-prefix-data-state-projection
      [OF i-bound])
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  using mapped-support unfolding projection .
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
have verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
have transcript-split:
  hash-map-output-values ?s ∩ set (staged-proof-transcript data)
    ≠ {} ∨
  hash-map-new-output-hit (set (staged-proof-transcript data))
    ?s final-state
  by (rule verify-monad-trace-root-target-preexisting-or-new[OF verifier])
  simp
show ?thesis
  unfolding out-eq checked-staged-security-with-query-prefix-transcript-hit-def
  using transcript-split by simp
qed

lemma checked-staged-security-with-query-prefix-transcript-hit-bound-from-data-state:
  assumes i-bound: i < rounds
  and pre-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-pre-hit
      adversary-initial-state ≤ P
  and new-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-new-hit
      adversary-initial-state ≤ R
  shows
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)

```

```

    checked-staged-security-with-query-prefix-transcript-hit
    adversary-initial-state  $\leq P + R$ 
proof –
  let ?E =
     $\lambda out.$ 
      staged-security-with-data-state-transcript-pre-hit out  $\vee$ 
      staged-security-with-data-state-transcript-new-hit out
  let ?projected =
     $\lambda out.$  case out of
      None  $\Rightarrow$  ?E None
    | Some (packed, t)  $\Rightarrow$  ?E (Some (snd packed, t))
  have projected-eq:
    ?projected = checked-staged-security-with-query-prefix-transcript-hit
  unfolding
    checked-staged-security-with-query-prefix-transcript-hit-def
    staged-security-with-data-state-transcript-pre-hit-def
    staged-security-with-data-state-transcript-new-hit-def
  by (rule ext) (auto split: option.splits prod.splits)
  have projection:
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?E
    adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      checked-staged-security-with-query-prefix-transcript-hit
      adversary-initial-state
  using
    checked-staged-security-experiment-with-query-prefix-data-state-projection-event
    [OF i-bound, of A ?E]
  unfolding projected-eq .
  have union-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?E
    adversary-initial-state  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-pre-hit
      adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-new-hit
      adversary-initial-state
  by (rule wp-event-union-bound)
  have wp-event (checked-staged-security-experiment-with-data-state A)
    ?E
    adversary-initial-state  $\leq P + R$ 
  by (rule order-trans[OF union-bound])
    (intro add-mono pre-bound new-bound)
  then show ?thesis
  unfolding projection .
qed

```

lemma *checked-staged-security-with-query-prefix-current-trace-path-output-hit-at-bound-from-transcript:*

assumes *i-bound*: $i < \text{rounds}$

and *pre-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)

staged-security-with-data-state-transcript-pre-hit

adversary-initial-state $\leq P$

and *new-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)

staged-security-with-data-state-transcript-new-hit

adversary-initial-state $\leq R$

shows

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)

(*checked-staged-security-with-query-prefix-current-trace-path-output-hit-at* *i*)

adversary-initial-state $\leq P + R$

proof –

have *event-le*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)

(*checked-staged-security-with-query-prefix-current-trace-path-output-hit-at* *i*)

adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)

checked-staged-security-with-query-prefix-transcript-hit

adversary-initial-state

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support*:

out $\in$

set-dist

(*execute*

(*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)

adversary-initial-state)

and *hit*:

checked-staged-security-with-query-prefix-current-trace-path-output-hit-at

i out

show *checked-staged-security-with-query-prefix-transcript-hit out*

by (rule

checked-staged-security-with-query-prefix-structured-path-hit-imp-transcript-hit-on-support

[*OF i-bound support*])

(use *hit* **in** *simp*)

qed

have *transcript-bound*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)

checked-staged-security-with-query-prefix-transcript-hit

adversary-initial-state $\leq P + R$

by (rule


```

      checked-staged-security-with-query-prefix-transcript-hit-bound-from-data-state
      [OF i-bound pre-bound new-bound])
show ?thesis
  by (rule order-trans[OF event-le transcript-bound])
qed

lemma checked-staged-security-with-query-prefix-current-composition-path-output-hit-at-bound-from-transcript:
assumes i-bound:  $i < \text{rounds}$ 
and pre-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-transcript-pre-hit
  adversary-initial-state  $\leq P$ 
and new-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-transcript-new-hit
  adversary-initial-state  $\leq R$ 
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
    adversary-initial-state  $\leq P + R$ 
proof –
have event-le:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
    adversary-initial-state  $\leq$ 
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    checked-staged-security-with-query-prefix-transcript-hit
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out  $\in$ 
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
  i out
show checked-staged-security-with-query-prefix-transcript-hit out
by (rule
  checked-staged-security-with-query-prefix-structured-path-hit-imp-transcript-hit-on-support
  [OF i-bound support])
  (use hit in simp)
qed
have transcript-bound:

```

```

wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  checked-staged-security-with-query-prefix-transcript-hit
  adversary-initial-state  $\leq P + R$ 
by (rule
  checked-staged-security-with-query-prefix-transcript-hit-bound-from-data-state
  [OF i-bound pre-bound new-bound])
show ?thesis
  by (rule order-trans[OF event-le transcript-bound])
qed

lemma checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-prefix-or-structured-path-or
assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and support:
    out  $\in$ 
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
    i out
shows
  checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
    i out  $\vee$ 
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i out  $\vee$ 
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i out
proof (cases out)
case None
  then show ?thesis
    using hit
  unfolding checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-def
    checked-staged-security-with-query-prefix-authenticated-opening-hit-def
  by simp
next
case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
  from hit obtain trace-openings composition-openings where current:
    checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i out

```

```

unfolding checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-def
by blast
have ext: prefix-state ≤ final-state
using support unfolding out-eq
by (rule
  checked-staged-security-experiment-with-query-prefix-data-state-prefix-extends-final
  [OF wf controlled i-bound])
have comp-nonempty: staged-composition-fri-roots data ≠ []
and trace-table:
  partial-authenticated-table (staged-trace-root data)
  (scale * clength) trace-openings final-state
and comp-table:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
  (scale * clength) composition-openings final-state
and consistent:
  partial-query-openings-consistent trace-openings composition-openings
  (staged-alphas data) (index (to-nat raw))
using current unfolding out-eq
  checked-staged-security-with-query-prefix-authenticated-opening-hit-def
by simp-all
have trace-pullback:
  partial-authenticated-table (staged-trace-root data)
  (scale * clength) trace-openings prefix-state ∨
  ( $\exists \text{opn} \in \text{set trace-openings.}$ 
    hash-map-new-output-hit
    (merkle-path-target-roots final-state (staged-trace-root data)
      (scale * clength)
      (opening-index opn)
      (opening-value opn)
      (opening-path opn)) prefix-state final-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit
    [OF ext trace-table])
then show ?thesis
proof
assume trace-prefix:
  partial-authenticated-table (staged-trace-root data)
  (scale * clength) trace-openings prefix-state
have comp-pullback:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
  (scale * clength) composition-openings prefix-state ∨
  ( $\exists \text{opn} \in \text{set composition-openings.}$ 
    hash-map-new-output-hit
    (merkle-path-target-roots final-state
      (hd (staged-composition-fri-roots data))
      (scale * clength)
      (opening-index opn)
      (opening-value opn)
      (opening-path opn)) prefix-state final-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit

```

```

    [OF ext comp-table])
then show ?thesis
proof
  assume comp-prefix:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings prefix-state
  have
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
      i out
  unfolding out-eq
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-def
    checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-def
  using comp-nonempty trace-prefix comp-prefix consistent by auto
  then show ?thesis by simp
next
  assume comp-path:
     $\exists \text{opn} \in \text{set composition-openings.}$ 
    hash-map-new-output-hit
      (merkle-path-target-roots final-state
        (hd (staged-composition-fri-roots data))
        (scale * clength)
        (opening-index opn)
        (opening-value opn)
        (opening-path opn)) prefix-state final-state
  then obtain opn where opn-in: opn  $\in$  set composition-openings
  and path-hit:
    hash-map-new-output-hit
      (merkle-path-target-roots final-state
        (hd (staged-composition-fri-roots data))
        (scale * clength)
        (opening-index opn)
        (opening-value opn)
        (opening-path opn)) prefix-state final-state
  by blast
  have auth: authenticated-opening-in final-state opn
  using comp-table opn-in
  unfolding partial-authenticated-table-def by blast
  have root-eq:
    opening-root opn = hd (staged-composition-fri-roots data)
  using comp-table opn-in
  unfolding partial-authenticated-table-def by blast
  have len-eq: opening-length opn = scale * clength
  using comp-table opn-in
  unfolding partial-authenticated-table-def by blast
  have
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
      i out
  unfolding out-eq
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at-def

```

```

    using comp-nonempty auth root-eq len-eq path-hit by auto
  then show ?thesis by simp
qed
next
assume trace-path:
   $\exists \text{opn} \in \text{set trace-openings.}$ 
  hash-map-new-output-hit
    (merkle-path-target-roots final-state (staged-trace-root data)
      (scale * clength)
      (opening-index opn)
      (opening-value opn)
      (opening-path opn)) prefix-state final-state
  then obtain opn where opn-in:  $\text{opn} \in \text{set trace-openings}$ 
  and path-hit:
    hash-map-new-output-hit
      (merkle-path-target-roots final-state (staged-trace-root data)
        (scale * clength)
        (opening-index opn)
        (opening-value opn)
        (opening-path opn)) prefix-state final-state
    by blast
  have auth: authenticated-opening-in final-state opn
    using trace-table opn-in
    unfolding partial-authenticated-table-def by blast
  have root-eq: opening-root opn = staged-trace-root data
    using trace-table opn-in
    unfolding partial-authenticated-table-def by blast
  have len-eq: opening-length opn = scale * clength
    using trace-table opn-in
    unfolding partial-authenticated-table-def by blast
  have
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
      i out
    unfolding out-eq
      checked-staged-security-with-query-prefix-current-trace-path-output-hit-at-def
    using auth root-eq len-eq path-hit by auto
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-candidate-pair-or-structured-path-on-
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and support:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)  $\in$ 
    set-dist
    (execute

```

```

    (checked-staged-security-experiment-with-query-prefix-data-state  $A$   $i$ )
    adversary-initial-state)
and hit:
    checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings  $i$ 
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
and trace-candidate:
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [ $i$  := trace-openings])
and comp-candidate:
    partial-composition-table-candidate composition-table
    ((replicate rounds []) [ $i$  := composition-openings])
and trace-low: trace-table-low-degree trace-table
and comp-low: composition-table-low-degree maxDegree composition-table
and not-all:
     $\neg$  all-queries-consistent trace-table composition-table
    (sqp-alphas prefix)
shows
    checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
    trace-table composition-table
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))  $\vee$ 
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
     $i$ 
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))  $\vee$ 
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
     $i$ 
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
proof –
    let ?out =
        Some (((((prefix, prefix-state), raw), raw-state),
          ((data, attacker-state), result)), final-state)
    have ext: prefix-state  $\leq$  final-state
    by (rule
        checked-staged-security-experiment-with-query-prefix-data-state-prefix-extends-final
        [OF wf controlled  $i$ -bound support])
    have alphas-eq: staged-alphas data = sqp-alphas prefix
    by (rule checked-staged-security-with-query-prefix-data-state-alphas-eq
        [OF support])
    have comp-nonempty: staged-composition-fri-roots data  $\neq$  []
    and trace-table-final:
        partial-authenticated-table (staged-trace-root data)
        (scale * clength) trace-openings final-state
    and comp-table-final:
        partial-authenticated-table (hd (staged-composition-fri-roots data))
        (scale * clength) composition-openings final-state

```

```

and consistent:
  partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
using hit
unfolding checked-staged-security-with-query-prefix-authenticated-opening-hit-def
by simp-all
have trace-pullback:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings prefix-state  $\vee$ 
    ( $\exists \text{ opn} \in \text{set trace-openings}.$ 
      hash-map-new-output-hit
        (merkle-path-target-roots final-state (staged-trace-root data)
          (scale * clength) (opening-index opn) (opening-value opn)
            (opening-path opn)) prefix-state final-state)
by (rule partial-authenticated-table-pullback-or-new-output-hit
      [OF ext trace-table-final])
then show ?thesis
proof
assume trace-prefix:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings prefix-state
have comp-pullback:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings prefix-state  $\vee$ 
    ( $\exists \text{ opn} \in \text{set composition-openings}.$ 
      hash-map-new-output-hit
        (merkle-path-target-roots final-state
          (hd (staged-composition-fri-roots data))
            (scale * clength) (opening-index opn) (opening-value opn)
              (opening-path opn)) prefix-state final-state)
by (rule partial-authenticated-table-pullback-or-new-output-hit
      [OF ext comp-table-final])
then show ?thesis
proof
assume comp-prefix:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings prefix-state
have opening-target:
  index (to-nat raw)  $\in$ 
    staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i prefix prefix-state
unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
by (rule partial-query-success-indices-atI,
      rule partial-query-round-consistent-single-roundI
      [OF i-bound])
      (use consistent alphas-eq in simp)
have opening-subset:

```

```

    staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i prefix prefix-state  $\subseteq$ 
    staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table prefix prefix-state
  by (rule
    staged-query-prefix-candidate-opening-query-target-from-prefix-subset-candidate-pair-from-prefix
      [OF trace-candidate comp-candidate trace-low comp-low not-all])
  have pair-hit:
    checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
      trace-table composition-table ?out
  unfolding
    checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-def
    checked-staged-security-with-query-prefix-dynamic-index-hit-def
    checked-staged-query-prefix-dynamic-index-hit-def
  using opening-target opening-subset by auto
  then show ?thesis by simp
next
  assume comp-path:
     $\exists \text{opn} \in \text{set composition-openings}.$ 
    hash-map-new-output-hit
      (merkle-path-target-roots final-state
        (hd (staged-composition-fri-roots data))
        (scale * clength) (opening-index opn) (opening-value opn)
        (opening-path opn)) prefix-state final-state
  then obtain opn where opn-in:  $\text{opn} \in \text{set composition-openings}$ 
  and path-hit:
    hash-map-new-output-hit
      (merkle-path-target-roots final-state
        (hd (staged-composition-fri-roots data))
        (scale * clength) (opening-index opn) (opening-value opn)
        (opening-path opn)) prefix-state final-state
  by blast
  have auth: authenticated-opening-in final-state opn
  using comp-table-final opn-in
  unfolding partial-authenticated-table-def by blast
  have root-eq:
    opening-root opn = hd (staged-composition-fri-roots data)
  using comp-table-final opn-in
  unfolding partial-authenticated-table-def by blast
  have len-eq: opening-length opn = scale * clength
  using comp-table-final opn-in
  unfolding partial-authenticated-table-def by blast
  have
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
      i ?out
  unfolding
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at-def

```



```

    using comp-nonempty auth root-eq len-eq path-hit by auto
  then show ?thesis by simp
qed
next
assume trace-path:
  ∃ opn ∈ set trace-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots final-state (staged-trace-root data)
        (scale * clength) (opening-index opn) (opening-value opn)
        (opening-path opn)) prefix-state final-state
  then obtain opn where opn-in: opn ∈ set trace-openings
  and path-hit:
    hash-map-new-output-hit
      (merkle-path-target-roots final-state (staged-trace-root data)
        (scale * clength) (opening-index opn) (opening-value opn)
        (opening-path opn)) prefix-state final-state
  by blast
have auth: authenticated-opening-in final-state opn
  using trace-table-final opn-in
  unfolding partial-authenticated-table-def by blast
have root-eq: opening-root opn = staged-trace-root data
  using trace-table-final opn-in
  unfolding partial-authenticated-table-def by blast
have len-eq: opening-length opn = scale * clength
  using trace-table-final opn-in
  unfolding partial-authenticated-table-def by blast
have
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i ?out
  unfolding
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at-def
  using auth root-eq len-eq path-hit by auto
  then show ?thesis by simp
qed
qed

definition staged-query-prefix-inconsistent-candidate-pair-query-target
where
  staged-query-prefix-inconsistent-candidate-pair-query-target
    trace-table composition-table prefix prefix-state =
    (if all-queries-consistent trace-table composition-table
      (sqp-alphas prefix)
    then {}
    else staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table prefix prefix-state)

lemma staged-query-prefix-inconsistent-candidate-pair-query-target-bound:
assumes trace-low: trace-table-low-degree trace-table
and comp-low:

```

composition-table-low-degree maxDegree composition-table
shows
staged-query-prefix-inconsistent-candidate-pair-query-target
trace-table composition-table prefix prefix-state
 $\subseteq \text{query-sample-space} \wedge$
nnreal
(card
(staged-query-prefix-inconsistent-candidate-pair-query-target
trace-table composition-table prefix prefix-state)) /
nnreal (card query-sample-space) ≤ query-error-bound
proof (*cases*
all-queries-consistent trace-table composition-table (sqp-alphas prefix))
case *True*
then show *?thesis*
unfolding *staged-query-prefix-inconsistent-candidate-pair-query-target-def*
by *simp*
next
case *False*
have *subset:*
staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
composition-table prefix prefix-state ⊆ query-sample-space
by (*rule staged-query-prefix-candidate-pair-query-target-from-prefix-subset*)
have *frac:*
nnreal
(card
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table prefix prefix-state)) /
nnreal (card query-sample-space) ≤ query-error-bound
by (*rule*
staged-query-prefix-candidate-pair-query-target-from-prefix-fraction-bound
[OF trace-low comp-low False])
show *?thesis*
unfolding *staged-query-prefix-inconsistent-candidate-pair-query-target-def*
using *False subset frac by simp*
qed

lemma *checked-staged-security-with-query-prefix-inconsistent-candidate-pair-hit-bound:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *trace-low: trace-table-low-degree trace-table*
and *comp-low:*
composition-table-low-degree maxDegree composition-table
shows
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-dynamic-index-hit
(staged-query-prefix-inconsistent-candidate-pair-query-target
trace-table composition-table))

adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) + query-error-bound
by (*rule*
checked-staged-security-with-query-prefix-dynamic-index-hit-bound-from-fraction
[OF wf controlled i-bound])
(use
staged-query-prefix-inconsistent-candidate-pair-query-target-bound
[OF trace-low comp-low] **in** *blast*)

lemma *checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-imp-inconsistent-target:*

assumes *hit:*

checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
trace-table composition-table
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))

and *not-all:*

$\neg$ *all-queries-consistent trace-table composition-table*
(sqp-alphas prefix)

shows

checked-staged-security-with-query-prefix-dynamic-index-hit
(staged-query-prefix-inconsistent-candidate-pair-query-target
trace-table composition-table)
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))

using *hit not-all*

unfolding

checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-def
checked-staged-security-with-query-prefix-dynamic-index-hit-def
checked-staged-query-prefix-dynamic-index-hit-def
staged-query-prefix-inconsistent-candidate-pair-query-target-def

by *simp*

lemma *checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-candidate-pair-and-structured*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *trace-candidate:*

partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings])

and *comp-candidate:*

partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])

and *trace-low: trace-table-low-degree trace-table*

and *comp-low: composition-table-low-degree maxDegree composition-table*

and *not-all:*

$\bigwedge$ *prefix prefix-state raw raw-state data attacker-state result*
final-state.
Some (((((prefix, prefix-state), raw), raw-state),

$((data, attacker-state), result)), final-state) \in$
set-dist
 $(execute$
 $(checked-staged-security-experiment-with-query-prefix-data-state$
 $A\ i)$
 $adversary-initial-state) \implies$
 $\neg all-queries-consistent\ trace-table\ composition-table$
 $(sqp-alphas\ prefix)$
and *pair-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$
 $(checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit$
 $trace-table\ composition-table)$
 $adversary-initial-state \leq P$
and *trace-path-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$
 $(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at$
 $i)$
 $adversary-initial-state \leq T$
and *composition-path-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$
 $(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at$
 $i)$
 $adversary-initial-state \leq C$
shows
 $wp-event$
 $(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$
 $(checked-staged-security-with-query-prefix-authenticated-opening-hit$
 $trace-openings\ composition-openings\ i)$
 $adversary-initial-state \leq P + T + C$
proof –
let *?M* =
 $checked-staged-security-experiment-with-query-prefix-data-state\ A\ i$
let *?Pair* =
 $checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit$
 $trace-table\ composition-table$
let *?Trace* =
 $checked-staged-security-with-query-prefix-current-trace-path-output-hit-at$
 i
let *?Comp* =
 $checked-staged-security-with-query-prefix-current-composition-path-output-hit-at$
 i
have *event-le*:
 $wp-event\ ?M$
 $(checked-staged-security-with-query-prefix-authenticated-opening-hit$
 $trace-openings\ composition-openings\ i)$
 $adversary-initial-state \leq$

```

wp-event ?M ( $\lambda out. ?Pair\ out \vee ?Trace\ out \vee ?Comp\ out$ )
adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out  $\in$  set-dist (execute ?M adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i out
  show ?Pair out  $\vee$  ?Trace out  $\vee$  ?Comp out
  proof (cases out)
    case None
    then show ?thesis
      using hit
      unfolding
        checked-staged-security-with-query-prefix-authenticated-opening-hit-def
      by simp
    next
    case (Some packed)
    then obtain prefix prefix-state raw raw-state data attacker-state result
      final-state where out-eq:
        out =
          Some (((((prefix, prefix-state), raw), raw-state),
            ((data, attacker-state), result)), final-state)
        by (cases packed) (auto split: prod.splits)
    have support-some:
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)  $\in$ 
        set-dist (execute ?M adversary-initial-state)
    using support out-eq by simp
    show ?thesis
      unfolding out-eq
      by (rule
        checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-candidate-pair-or-structured-path-on-s
        [OF wf controlled i-bound support-some])
      (use hit[unfolded out-eq] trace-candidate comp-candidate
        trace-low comp-low not-all[OF support-some] in simp-all)
    qed
  qed
have union-le:
  wp-event ?M ( $\lambda out. ?Pair\ out \vee ?Trace\ out \vee ?Comp\ out$ )
    adversary-initial-state  $\leq$ 
  wp-event ?M ?Pair adversary-initial-state +
  (wp-event ?M ?Trace adversary-initial-state +
    wp-event ?M ?Comp adversary-initial-state)
proof –
  have
    wp-event ?M ( $\lambda out. ?Pair\ out \vee ?Trace\ out \vee ?Comp\ out$ )
      adversary-initial-state  $\leq$ 

```

```

wp-event ?M ?Pair adversary-initial-state +
wp-event ?M (λout. ?Trace out ∨ ?Comp out)
adversary-initial-state
by (rule wp-event-union-bound)
also have
... ≤
wp-event ?M ?Pair adversary-initial-state +
(wp-event ?M ?Trace adversary-initial-state +
wp-event ?M ?Comp adversary-initial-state)
by (intro add-mono order-refl wp-event-union-bound)
finally show ?thesis .
qed
have sum-bound:
wp-event ?M ?Pair adversary-initial-state +
(wp-event ?M ?Trace adversary-initial-state +
wp-event ?M ?Comp adversary-initial-state) ≤ P + T + C
proof -
have path-sum:
wp-event ?M ?Trace adversary-initial-state +
wp-event ?M ?Comp adversary-initial-state ≤ T + C
by (rule add-mono[OF trace-path-bound composition-path-bound])
have
wp-event ?M ?Pair adversary-initial-state +
(wp-event ?M ?Trace adversary-initial-state +
wp-event ?M ?Comp adversary-initial-state) ≤ P + (T + C)
by (rule add-mono[OF pair-bound path-sum])
then show ?thesis
by (simp add: add.assoc)
qed
show ?thesis
by (rule order-trans[OF event-le order-trans[OF union-le sum-bound]])
qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-inconsistent-candidate-pair-and-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and trace-candidate:
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings])
and comp-candidate:
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])
and trace-low: trace-table-low-degree trace-table
and comp-low: composition-table-low-degree maxDegree composition-table
and not-all-on-hit:
Λprefix prefix-state raw raw-state data attacker-state result
final-state.
Some (((((prefix, prefix-state), raw), raw-state),

```

```

    ((data, attacker-state), result)), final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state
        A i)
      adversary-initial-state) ⇒
    checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)) ⇒
    ¬ all-queries-consistent trace-table composition-table
    (sqp-alphas prefix)
and trace-path-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
      i)
    adversary-initial-state ≤ T
and composition-path-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
      i)
    adversary-initial-state ≤ C
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-authenticated-opening-hit
      trace-openings composition-openings i)
    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    query-error-bound + T + C
proof –
  let ?M =
    checked-staged-security-experiment-with-query-prefix-data-state A i
  let ?Pair =
    checked-staged-security-with-query-prefix-dynamic-index-hit
    (staged-query-prefix-inconsistent-candidate-pair-query-target
      trace-table composition-table)
  let ?Trace =
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i
  let ?Comp =
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i
  have event-le:
    wp-event ?M
    (checked-staged-security-with-query-prefix-authenticated-opening-hit

```

```

    trace-openings composition-openings i)
  adversary-initial-state ≤
wp-event ?M (λout. ?Pair out ∨ ?Trace out ∨ ?Comp out)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out ∈ set-dist (execute ?M adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i out
show ?Pair out ∨ ?Trace out ∨ ?Comp out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-query-prefix-authenticated-opening-hit-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
  final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
have support-some:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
    set-dist (execute ?M adversary-initial-state)
  using support out-eq by simp
have not-all:
  ¬ all-queries-consistent trace-table composition-table
    (sqp-alphas prefix)
  by (rule not-all-on-hit[OF support-some])
    (use hit out-eq in simp)
have split:
  checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
    trace-table composition-table
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)) ∨
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)) ∨
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i
    (Some (((((prefix, prefix-state), raw), raw-state),

```



```

      ((data, attacker-state), result)), final-state))
  by (rule
    checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-candidate-pair-or-structured-path-on-s
      [OF wf controlled i-bound support-some])
    (use hit[unfolded out-eq] trace-candidate comp-candidate
      trace-low comp-low not-all in simp-all)
  then show ?thesis
  proof
    assume pair:
      checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
        trace-table composition-table
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
    have ?Pair
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
    by (rule
      checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-imp-inconsistent-target
        [OF pair not-all])
    then show ?thesis
      unfolding out-eq by simp
  next
    assume
      checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
        i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)) ∨
      checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
        i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
    then show ?thesis
      unfolding out-eq by simp
  qed
  qed
  qed
  have pair-bound:
    wp-event ?M ?Pair adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) + query-error-bound
  by (rule
    checked-staged-security-with-query-prefix-inconsistent-candidate-pair-hit-bound
      [OF wf controlled i-bound trace-low comp-low])
  have union-le:
    wp-event ?M (λout. ?Pair out ∨ ?Trace out ∨ ?Comp out)
      adversary-initial-state ≤
    wp-event ?M ?Pair adversary-initial-state +
    (wp-event ?M ?Trace adversary-initial-state +
    wp-event ?M ?Comp adversary-initial-state)

```

proof –

have

$wp\text{-event } ?M (\lambda out. ?Pair\ out \vee ?Trace\ out \vee ?Comp\ out)$
 $adversary\text{-initial-state} \leq$
 $wp\text{-event } ?M ?Pair\ adversary\text{-initial-state} +$
 $wp\text{-event } ?M (\lambda out. ?Trace\ out \vee ?Comp\ out)$
 $adversary\text{-initial-state}$
by (*rule wp-event-union-bound*)

also have

$\dots \leq$
 $wp\text{-event } ?M ?Pair\ adversary\text{-initial-state} +$
 $(wp\text{-event } ?M ?Trace\ adversary\text{-initial-state} +$
 $wp\text{-event } ?M ?Comp\ adversary\text{-initial-state})$
by (*intro add-mono order-refl wp-event-union-bound*)

finally show *?thesis* .

qed

have *sum-bound*:

$wp\text{-event } ?M ?Pair\ adversary\text{-initial-state} +$
 $(wp\text{-event } ?M ?Trace\ adversary\text{-initial-state} +$
 $wp\text{-event } ?M ?Comp\ adversary\text{-initial-state}) \leq$
 $(staged\text{-phase-relation-error}\ size$
 $(staged\text{-query-search-queries}\ budgets\ i + 1) + query\text{-error-bound}) +$
 $(T + C)$
by (*intro add-mono pair-bound add-mono trace-path-bound*
composition-path-bound)

have *rhs-eq*:

$(staged\text{-phase-relation-error}\ size$
 $(staged\text{-query-search-queries}\ budgets\ i + 1) + query\text{-error-bound}) +$
 $(T + C) =$
 $staged\text{-phase-relation-error}\ size$
 $(staged\text{-query-search-queries}\ budgets\ i + 1) +$
 $query\text{-error-bound} + T + C$
by (*simp add: add.assoc*)

have *bound*:

$wp\text{-event } ?M$
 $(checked\text{-staged-security-with-query-prefix-authenticated-opening-hit}$
 $trace\text{-openings}\ composition\text{-openings}\ i)$
 $adversary\text{-initial-state} \leq$
 $(staged\text{-phase-relation-error}\ size$
 $(staged\text{-query-search-queries}\ budgets\ i + 1) + query\text{-error-bound}) +$
 $(T + C)$
by (*rule order-trans[OF event-le order-trans[OF union-le sum-bound]]*)

show *?thesis*

using *bound rhs-eq* **by** *simp*

qed

lemma *checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-candidate-pair-budgets-and-st*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound*: $i < \text{rounds}$
and *trace-candidate*:
 partial-trace-table-candidate *trace-table*
 $((\text{replicate } \text{rounds } []) [i := \text{trace-openings}])$
and *comp-candidate*:
 partial-composition-table-candidate *composition-table*
 $((\text{replicate } \text{rounds } []) [i := \text{composition-openings}])$
and *trace-low*: *trace-table-low-degree* *trace-table*
and *comp-low*: *composition-table-low-degree* *maxDegree* *composition-table*
and *not-all*:
 $\bigwedge \text{prefix } \text{prefix-state}.$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 adversary-initial-state) $\implies$
 $\neg \text{all-queries-consistent } \text{trace-table } \text{composition-table}$
 $(\text{sqp-alphas } \text{prefix})$
and *trace-path-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at}$
 i)
 $\text{adversary-initial-state} \leq T$
and *composition-path-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at}$
 i)
 $\text{adversary-initial-state} \leq C$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-authenticated-opening-hit}$
 trace-openings *composition-openings* *i*)
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + T + C$
proof –
have *pair-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit}$
 trace-table *composition-table*)
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$
by (rule
 checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-bound-from-budgets

$[OF \text{ wf controlled } i\text{-bound trace-low comp-low not-all}]$
have *not-all-support*:
 $\bigwedge \text{prefix prefix-state raw raw-state data attacker-state result}$
 final-state.
 $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}) \in$
 set-dist
 $(\text{execute}$
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state}$
 $A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\neg \text{all-queries-consistent trace-table composition-table}$
 $(\text{sqp-alphas prefix})$
proof –
fix *prefix prefix-state raw raw-state data attacker-state result final-state*
assume *support*:
 $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}) \in$
 set-dist
 $(\text{execute}$
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state}$
 $A \ i)$
 $\text{adversary-initial-state})$
have *head*:
 $\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $\text{adversary-initial-state})$
using *support*
unfolding *checked-staged-security-experiment-with-query-prefix-data-state-def*
by (*auto elim! set-dist-bindE split prod.splits*)
have *prefix-support*:
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state})$
by (*rule checked-staged-query-prefix-receive-with-state-prefix-support*
 $[OF \text{ head}]$)
show
 $\neg \text{all-queries-consistent trace-table composition-table}$
 $(\text{sqp-alphas prefix})$
by (*rule not-all[OF prefix-support]*)
qed
show *?thesis*
by (*rule*
 $\text{checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-candidate-pair-and-structured-pair}$
 $[OF \text{ wf controlled } i\text{-bound trace-candidate comp-candidate trace-low}$
 $\text{comp-low not-all-support pair-bound trace-path-bound}$
 $\text{composition-path-bound}]$)

qed

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-structure*

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *i-bound*: $i < \text{rounds}$

and *prefix-bound*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at *i*)

adversary-initial-state $\leq P$

and *trace-path-bound*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at *i*)

adversary-initial-state $\leq T$

and *composition-path-bound*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at *i*)

adversary-initial-state $\leq C$

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at *i*)

adversary-initial-state $\leq P + T + C$

proof –

let *?M* = checked-staged-security-experiment-with-query-prefix-data-state *A i*

let *?Prefix* =

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at *i*

let *?Trace* =

checked-staged-security-with-query-prefix-current-trace-path-output-hit-at *i*

let *?Composition* =

checked-staged-security-with-query-prefix-current-composition-path-output-hit-at *i*

have *event-le*:

wp-event *?M*

(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at *i*)

adversary-initial-state $\leq$

wp-event *?M* ($\lambda \text{out. ?Prefix out} \vee \text{?Trace out} \vee \text{?Composition out}$)

adversary-initial-state

by (rule *wp-event-mono-on-support*)

(rule

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-prefix-or-structured-path-on-su

[OF *wf* controlled *i-bound*])

have *first-union*:

wp-event *?M* ($\lambda \text{out. ?Prefix out} \vee \text{?Trace out} \vee \text{?Composition out}$)

adversary-initial-state $\leq$

wp-event *?M* *?Prefix* adversary-initial-state +

wp-event *?M* ($\lambda \text{out. ?Trace out} \vee \text{?Composition out}$)

adversary-initial-state
by (rule *wp-event-union-bound*)
have *second-union*:
wp-event ?*M* ($\lambda out. ?Trace\ out \vee ?Composition\ out$)
adversary-initial-state $\leq$
wp-event ?*M* ?*Trace* *adversary-initial-state* +
wp-event ?*M* ?*Composition* *adversary-initial-state*
by (rule *wp-event-union-bound*)
have *path-sum-bound*:
wp-event ?*M* ($\lambda out. ?Trace\ out \vee ?Composition\ out$)
adversary-initial-state $\leq T + C$
by (rule *order-trans*[*OF second-union*])
(intro *add-mono trace-path-bound composition-path-bound*)
have *sum-bound*:
wp-event ?*M* ?*Prefix* *adversary-initial-state* +
wp-event ?*M* ($\lambda out. ?Trace\ out \vee ?Composition\ out$)
adversary-initial-state $\leq P + (T + C)$
by (intro *add-mono prefix-bound path-sum-bound*)
have *union-sum-bound*:
wp-event ?*M* ($\lambda out. ?Prefix\ out \vee ?Trace\ out \vee ?Composition\ out$)
adversary-initial-state $\leq P + T + C$
by (rule *order-trans*[*OF first-union*])
(use *sum-bound* in $\langle simp\ add: add.assoc \rangle$)
show ?*thesis*
by (rule *order-trans*[*OF event-le*])
(rule *union-sum-bound*)
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-prefix-target*

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *prefix-bound*:

$\bigwedge i. i < rounds \implies$

wp-event (*checked-staged-query-prefix-receive-with-state A i*)
(*checked-staged-query-prefix-dynamic-index-hit*
staged-query-prefix-prefix-authenticated-opening-target)
adversary-initial-state $\leq P\ i$

and *trace-path-bound*:

$\bigwedge i. i < rounds \implies$

wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
(*checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i*)
adversary-initial-state $\leq T\ i$

and *composition-path-bound*:

$\bigwedge i. i < rounds \implies$

wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
(*checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i*)
adversary-initial-state $\leq C\ i$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

adversary-initial-state $\leq (\sum i < \text{rounds}. P\ i + T\ i + C\ i)$

proof (rule

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-curr

[*OF wf controlled*])

fix *i*

assume *i-bound*: $i < \text{rounds}$

have *prefix-hit-bound*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i*)

adversary-initial-state $\leq P\ i$

by (rule

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target

[*OF wf controlled i-bound prefix-bound[OF i-bound]*])

show

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i*)

adversary-initial-state $\leq P\ i + T\ i + C\ i$

by (rule

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-structured-p

[*OF wf controlled i-bound prefix-hit-bound trace-path-bound[OF i-bound]*]

composition-path-bound[OF i-bound]])

qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-trace-indic*

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *trace-frac*:

$\bigwedge i\ \text{prefix}\ \text{prefix-state}.$

$i < \text{rounds} \implies$

Some ((*prefix*, *prefix-state*), *prefix-state*) $\in$

set-dist

(*execute* (*checked-staged-query-prefix-with-state A i*)

adversary-initial-state) $\implies$

nnreal

(*card*

(*staged-query-prefix-prefix-authenticated-trace-indices prefix*
prefix-state)) /

nnreal (*card query-sample-space*) $\leq \text{query-error-bound}$

and *trace-path-bound*:

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i*)

$adversary-initial-state \leq T \ i$
and *composition-path-bound*:
 $\bigwedge i. i < rounds \implies$
 $wp-event$
 $(checked-staged-security-experiment-with-query-prefix-data-state \ A \ i)$
 $(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at \ i)$
 $adversary-initial-state \leq C \ i$
shows
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state \ A)$
 $checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit$
 $adversary-initial-state \leq$
 $(\sum i < rounds.$
 $staged-phase-relation-error \ size$
 $(staged-query-search-queries \ budgets \ i + 1) +$
 $query-error-bound + T \ i + C \ i)$
proof (rule
 $checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-prefix-target-and-$
 $[OF \ wf \ controlled - trace-path-bound \ composition-path-bound])$
fix i
assume $i-bound: i < rounds$
show $wp-event \ (checked-staged-query-prefix-receive-with-state \ A \ i)$
 $(checked-staged-query-prefix-dynamic-index-hit$
 $staged-query-prefix-prefix-authenticated-opening-target)$
 $adversary-initial-state \leq$
 $staged-phase-relation-error \ size$
 $(staged-query-search-queries \ budgets \ i + 1) + query-error-bound$
proof (rule
 $checked-staged-query-prefix-prefix-authenticated-target-hit-bound-from-trace-indices$
 $[OF \ wf \ controlled \ i-bound])$
fix $prefix \ prefix-state$
assume *support*:
 $Some \ ((prefix, \ prefix-state), \ prefix-state) \in$
 $set-dist$
 $(execute \ (checked-staged-query-prefix-with-state \ A \ i)$
 $adversary-initial-state)$
show
 $nnreal$
 $(card$
 $(staged-query-prefix-prefix-authenticated-trace-indices \ prefix$
 $prefix-state)) /$
 $nnreal \ (card \ query-sample-space) \leq query-error-bound$
by (rule $trace-frac[OF \ i-bound \ support]$)
qed
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-trace-indices-structured-paths-and-budget*
assumes $wf: staged-budget-wellformed \ budgets$
and *controlled: staged-adversary-controlled budgets A*

and *trace-frac*:
 $\bigwedge i$ *prefix* *prefix-state*.
 $i < \text{rounds} \implies$
Some ((*prefix*, *prefix-state*), *prefix-state*) $\in$
set-dist
(*execute* (*checked-staged-query-prefix-with-state* *A* *i*)

adversary-initial-state) $\implies$
nnreal
(*card*
(*staged-query-prefix-prefix-authenticated-trace-indices* *prefix*
prefix-state)) /
nnreal (*card* *query-sample-space*) $\leq$ *query-error-bound*

and *trace-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)
(*checked-staged-security-with-query-prefix-current-trace-path-output-hit-at* *i*)
adversary-initial-state $\leq T$ *i*

and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)
(*checked-staged-security-with-query-prefix-current-composition-path-output-hit-at* *i*)
adversary-initial-state $\leq C$ *i*

shows
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event* *composition-bad*)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
(*staged-alpha-search-queries* *budgets* 0) +
staged-phase-relation-error size
(*staged-alpha-search-queries* *budgets* 0) +
($\sum i < \text{rounds}.$
staged-phase-relation-error size
(*staged-query-search-queries* *budgets* *i* + 1) +
query-error-bound + *T* *i* + *C* *i*) +
hash-collision-budget-value 0
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*)

proof –
let ?*Q* =
($\sum i < \text{rounds}.$
staged-phase-relation-error size
(*staged-query-search-queries* *budgets* *i* + 1) +
query-error-bound + *T* *i* + *C* *i*)

have *query-bound*:

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  adversary-initial-state  $\leq ?Q$ 
by (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-trace-indices-a
  [OF wf controlled trace-frac trace-path-bound composition-path-bound])
have without-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
    adversary-initial-state  $\leq 0$ 
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-bound
    [OF wf controlled])
have drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
    adversary-initial-state  $\leq ?Q + 0$ 
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-query
    [OF query-bound without-bound])
have comp-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state  $\leq$ 
      composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      (?Q + 0) +
      hash-collision-budget-value 0
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-composition-candidate-drift-and-budgets
    [OF wf controlled drift-bound])
then show ?thesis
  by simp
qed

end

end

```

theory *Soundness-Conceptual-Query-Empty-Current*

imports

Soundness-Query-Opening-Consistency

Soundness-Conceptual-Query-Current

Staged-Security-Experiment-Composition-Query-Current-Path

Soundness-Staged-Bounds

begin

Empty-composition current-query projection.

This layer connects the verifier-local empty-composition query extraction to the prefixed conceptual empty query target infrastructure. It is kept separate from the non-empty current-query layer to avoid growing the already size-sensitive staged/query theories.

context *soundness*

begin

definition *staged-security-with-data-state-current-empty-query-opening-hit-at*

$:: \text{nat} \Rightarrow$

$((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

staged-security-with-data-state-current-empty-query-opening-hit-at i out

$\longleftrightarrow$

(case out of

None $\Rightarrow$ *False*

| *Some* $(((\text{data}, \text{attacker-state}), \text{result}), \text{final-state}) \Rightarrow$

(let *s* =

verifier-state-from-adversary attacker-state

(*staged-proof-transcript data*)

in

$(\exists \text{ raw trace-openings rest.}$

$i < \text{rounds} \wedge$

fmlookup (*HashMap final-state*)

(*QueryIndexChallenge i*

(*state-after-query-chunks*

(*staged-query-start-hash data*)

(*staged-query-chunks data i*)) =

Some raw $\wedge$

staged-composition-fri-roots data = [] $\wedge$

Some (*result, final-state*) $\in$

set-dist (*execute verify-monad s*) $\wedge$

partial-authenticated-table (*staged-trace-root data*)

(*scale * clength*) *trace-openings final-state* $\wedge$

verifier-header-transcript s

(*staged-trace-root data*)

(*staged-trace-fri-roots data*)

(*staged-trace-final data*)

(*staged-alphas data*)

```

    (staged-degree data)
  []
  (staged-composition-final data)
  rest ∧
  map opening-index trace-openings =
    powers-scaled (index (to-nat raw)) ∧
  cp-eval (staged-alphas data) (map opening-value trace-openings)
  (h ^ index (to-nat raw) * shift) =
  staged-composition-final data)))

```

definition *staged-security-with-data-state-current-empty-query-opening-hit*
 $:: (((\text{'f staged-proof-data} \times \text{'f protocol-channel}) \times \text{unit list}) \times$
 $\text{'f protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

```

  staged-security-with-data-state-current-empty-query-opening-hit out
   $\longleftrightarrow$ 
  ( $\exists i < \text{rounds}.$ 
    staged-security-with-data-state-current-empty-query-opening-hit-at
    i out)

```

lemma *staged-security-with-data-state-current-empty-query-opening-hitI*:
assumes $i < \text{rounds}$
and *staged-security-with-data-state-current-empty-query-opening-hit-at*
 $i \text{ out}$
shows *staged-security-with-data-state-current-empty-query-opening-hit out*
using *assms*
unfolding *staged-security-with-data-state-current-empty-query-opening-hit-def*
by *blast*

lemma *staged-security-with-data-state-current-empty-query-opening-hit-atI*:
assumes $i\text{-bound}: i < \text{rounds}$
and *lookup*:
 $\text{fmlookup } (\text{HashMap final-state})$
 $(\text{QueryIndexChallenge } i$
 $(\text{state-after-query-chunks}$
 $(\text{staged-query-start-hash data})$
 $(\text{staged-query-chunks data } i)) =$
 Some raw
and *comp-empty*: $\text{staged-composition-fri-roots data} = []$
and *verifier*:
 $\text{Some } (\text{result}, \text{final-state}) \in$
 set-dist
 $(\text{execute verify-monad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
and *trace-auth*:
 $\text{partial-authenticated-table } (\text{staged-trace-root data})$
 $(\text{scale} * \text{clength}) \text{ trace-openings final-state}$
and *header*:

```

    verifier-header-transcript
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      []
      (staged-composition-final data)
      rest
  and trace-indices:
    map opening-index trace-openings =
      powers-scaled (index (to-nat raw))
  and consistent:
    cp-eval (staged-alphas data) (map opening-value trace-openings)
      (h ^ index (to-nat raw) * shift) =
        staged-composition-final data
  shows
    staged-security-with-data-state-current-empty-query-opening-hit-at i
      (Some (((data, attacker-state), result), final-state))
  unfolding staged-security-with-data-state-current-empty-query-opening-hit-at-def
  Let-def
  apply simp
  by (intro exI[of - raw] exI[of - trace-openings] exI[of - rest] conjI)
    (use assms in simp-all)

lemma empty-query-bad-header-alignment-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  and bad:
    staged-security-with-data-state-verifier-event
      query-bad-with-empty-composition-header-candidates
      (Some (((data, attacker-state), result), final-state))
  obtains fr f-fri-roots f-final as dg final rest where
    verifier-header-transcript
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
    fr f-fri-roots f-final as dg [] final rest
    fr = staged-trace-root data
    f-fri-roots = staged-trace-fri-roots data
    f-final = staged-trace-final data
    as = staged-alphas data

```

```

    dg = staged-degree data
    staged-composition-fri-roots data = []
    final = staged-composition-final data
    rest = List.concat (staged-query-chunks data)
    verifier-header-state
      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data))
    fr f-fri-roots f-final as dg [] final =
      staged-query-start-hash data
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF support]
obtain builder where builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
               adversary-initial-state)
  by blast
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
       [OF wf controlled builder])
have bad-verifier:
  query-bad-with-empty-composition-header-candidates ?s
    (Some (result, final-state))
  using bad
  unfolding staged-security-with-data-state-verifier-event-def
  by simp
from bad-verifier obtain fr f-fri-roots f-final as dg final
  trace-query-idxs trace-openings trace-table composition-table
  where partial:
    accepted-with-empty-composition-header-candidates ?s
      (Some (result, final-state)) fr f-fri-roots f-final as dg final
    trace-query-idxs trace-openings trace-table composition-table
  unfolding query-bad-with-empty-composition-header-candidates-def
  by blast
from partial obtain rest where header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg []

```

```

    final rest
  unfolding accepted-with-empty-composition-header-candidates-def
  by blast
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  [] = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
have header-state:
  verifier-header-state ?s fr f-fri-roots f-final as dg [] final =
    staged-query-start-hash data
using header-eq
unfolding verifier-header-state-def verifier-header-messages-def
  staged-query-start-hash-def staged-composition-fri-start-hash-def
  staged-trace-fri-start-hash-def
by simp
show ?thesis
by (rule that[OF header])
  (use header-eq header-state in simp-all)
qed

```

```

lemma empty-query-bad-query-rounds-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  and bad:
    staged-security-with-data-state-verifier-event
      query-bad-with-empty-composition-header-candidates
      (Some (((data, attacker-state), result), final-state))
  obtains f-fl query-state where
    PState query-state = staged-query-start-hash data
    PQueryCounter query-state = 0
    Some (result, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program
              (staged-trace-root data)

```

```

      f-fl
      (staged-trace-final data)
      (staged-alphas data)
      []
      (staged-composition-final data))
    rounds)
  query-state)
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF support]
  have verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
  from empty-query-bad-header-alignment-on-support
    [OF wf controlled support bad]
  obtain fr f-fri-roots f-final as dg final rest where header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg []
    final rest
  and fr-eq: fr = staged-trace-root data
  and f-final-eq: f-final = staged-trace-final data
  and as-eq: as = staged-alphas data
  and final-eq: final = staged-composition-final data
  and header-state:
    verifier-header-state ?s fr f-fri-roots f-final as dg [] final =
    staged-query-start-hash data
  by blast
  from verify-monad-supplied-empty-header-query-rounds[OF verifier header]
  obtain f-fl query-state where
    query-state-state:
      PState query-state =
        verifier-header-state ?s fr f-fri-roots f-final as dg [] final
  and query-counter: PQueryCounter query-state = PQueryCounter ?s
  and query-out:
    Some (result, final-state) ∈
      set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as [] final)
          rounds)
        query-state)
  by blast
  show ?thesis
  by (rule that[of query-state f-fl])
    (use query-state-state query-counter query-out header-state fr-eq
      f-final-eq as-eq final-eq in simp-all)
qed

```


lemma *staged-security-with-data-state-current-empty-query-opening-hit-at-imp-query-prefix-current-on-support*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *i-bound*: $i < \text{rounds}$

and *support*:

Some (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*) $\in$
set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
adversary-initial-state)

and *hit*:

staged-security-with-data-state-current-empty-query-opening-hit-at i
 (*Some* (((*data*, *attacker-state*), *result*), *final-state*))

shows

checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at
i
 (*Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*))

proof –

let *?s* =

verifier-state-from-adversary attacker-state
 (*staged-proof-transcript data*)

have *hit'*:

$\exists \text{raw}' \text{ trace-openings rest.}$
 $i < \text{rounds} \wedge$
fmlookup (*HashMap final-state*)
 (*QueryIndexChallenge i*
 (*state-after-query-chunks*
 (*staged-query-start-hash data*)
 (*staged-query-chunks data*) *i*)) =
Some raw' $\wedge$
staged-composition-fri-roots data = $\square \wedge$
Some (*result*, *final-state*) $\in$ *set-dist* (*execute verify-monad ?s*) $\wedge$
partial-authenticated-table (*staged-trace-root data*)
 (*scale * clength*) *trace-openings final-state $\wedge$*
verifier-header-transcript ?s
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 $\square$
 (*staged-composition-final data*)
rest $\wedge$
map opening-index trace-openings =
powers-scaled (*index* (*to-nat raw'*)) $\wedge$
cp-eval (*staged-alphas data*) (*map opening-value trace-openings*)

```

    (h ^ index (to-nat raw') * shift) =
      staged-composition-final data
using hit
unfolding
  staged-security-with-data-state-current-empty-query-opening-hit-at-def
  Let-def
by simp
from hit' obtain raw' trace-openings rest where
  final-lookup':
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some raw'
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and trace-indices:
  map opening-index trace-openings =
    powers-scaled (index (to-nat raw'))
and consistent:
  cp-eval (staged-alphas data) (map opening-value trace-openings)
    (h ^ index (to-nat raw') * shift) =
      staged-composition-final data
by blast
have final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
by (rule
  checked-staged-security-experiment-with-query-prefix-data-state-raw-final-lookup
  [OF wf controlled i-bound support])
have raw-eq: raw' = raw
using final-lookup' final-lookup by simp
from support obtain chunk chunk-state assert-state record-state suffix-chunks
where data-eq:
  data =
    (|staged-trace-root = sqp-trace-root prefix,
      staged-trace-fri-roots = sqp-trace-fri-roots prefix,
      staged-trace-fri-challenges = sqp-trace-fri-challenges prefix,
      staged-trace-final = sqp-trace-final prefix,
      staged-alphas = sqp-alphas prefix,
      staged-degree = sqp-degree prefix,
      staged-composition-fri-roots = sqp-composition-fri-roots prefix,
      staged-composition-fri-challenges =
        sqp-composition-fri-challenges prefix,

```

```

    staged-composition-final = sqp-composition-final prefix,
    staged-query-chunks = sqp-query-chunks prefix @ chunk # suffix-chunks)
unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
    checked-staged-after-query-prefix-receive-with-verifier-def
    checked-staged-after-query-prefix-receive-def
by (auto elim!: set-dist-bindE split: prod.splits)
have current:
    checked-staged-security-with-query-prefix-empty-authenticated-opening-hit
    trace-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
unfolding
    checked-staged-security-with-query-prefix-empty-authenticated-opening-hit-def
using trace-auth trace-indices consistent raw-eq data-eq by simp
show ?thesis
unfolding
    checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at-def
using current by blast
qed

lemma checked-staged-security-with-data-state-current-empty-query-opening-hit-at-le-query-prefix-current:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
shows
    wp-event
    (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-current-empty-query-opening-hit-at i)
    adversary-initial-state ≤
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at i)
    adversary-initial-state
proof –
let ?projected =
    λout. case out of
    None ⇒
        staged-security-with-data-state-current-empty-query-opening-hit-at
        i None
    | Some (packed, t) ⇒
        staged-security-with-data-state-current-empty-query-opening-hit-at
        i (Some (snd packed, t))
have projection:
    wp-event
    (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-current-empty-query-opening-hit-at i)
    adversary-initial-state =
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)

```

```

    ?projected
    adversary-initial-state
  by (rule
    checked-staged-security-experiment-with-query-prefix-data-state-projection-event
    [OF i-bound])
  have event-le:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    ?projected
    adversary-initial-state ≤
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at i)
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
  and hit: ?projected out
  show
    checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at
    i out
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      staged-security-with-data-state-current-empty-query-opening-hit-at-def
    by simp
  next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
  show ?thesis
    unfolding out-eq
  by (rule
    staged-security-with-data-state-current-empty-query-opening-hit-at-imp-query-prefix-current-on-support
    [OF wf controlled i-bound])
    (use support hit out-eq in simp-all)
  qed
  qed

```

```

show ?thesis
  unfolding projection
  by (rule event-le)
qed

lemma empty-query-bad-imp-current-empty-opening-hit-at-zero-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  and bad:
    staged-security-with-data-state-verifier-event
    query-bad-with-empty-composition-header-candidates
    (Some (((data, attacker-state), result), final-state))
  shows
    staged-security-with-data-state-current-empty-query-opening-hit-at 0
    (Some (((data, attacker-state), result), final-state))
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF support]
  have verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
  from empty-query-bad-header-alignment-on-support
    [OF wf controlled support bad]
  obtain rest where header:
    verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    []
    (staged-composition-final data)
    rest
  and comp-empty:
    staged-composition-fri-roots data = []
  by blast
  from empty-query-bad-query-rounds-on-support
    [OF wf controlled support bad]
  obtain f-fl query-state where query-state-state:
    PState query-state = staged-query-start-hash data

```

```

and query-counter:
  PQueryCounter query-state = 0
and query-out:
  Some (result, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program
            (staged-trace-root data)
            f-fl
            (staged-trace-final data)
            (staged-alphas data)
            []
            (staged-composition-final data))
          rounds)
        query-state)
by blast
from ntimes-verifier-query-rounds-empty-composition-first-consistent
  [OF refl query-out]
obtain raw trace-openings where trace-indices:
  map opening-index trace-openings =
    powers-scaled (index (to-nat raw))
and trace-table:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and consistent:
  cp-eval (staged-alphas data) (map opening-value trace-openings)
    (h ^ index (to-nat raw) * shift) =
  staged-composition-final data
and lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state) (PState query-state)) =
  Some raw
by blast
have zero-bound: 0 < rounds
using rounds-positive by simp
have lookup0:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge 0
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) 0)) =
  Some raw
using lookup query-state-state query-counter
unfolding state-after-query-chunks-def by simp
show ?thesis
by (rule
  staged-security-with-data-state-current-empty-query-opening-hit-atI
  [OF zero-bound lookup0 comp-empty verifier trace-table header

```

```

      trace-indices consistent])
qed

lemma checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        query-bad-with-empty-composition-header-candidates)
      adversary-initial-state  $\leq$ 
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
      adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out  $\in$ 
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  and bad:
    staged-security-with-data-state-verifier-event
      query-bad-with-empty-composition-header-candidates out
  show
    staged-security-with-data-state-current-empty-query-opening-hit-at 0 out
proof (cases out)
  case None
  then show ?thesis
    using bad
    unfolding staged-security-with-data-state-verifier-event-def
      query-bad-with-empty-composition-header-candidates-def
    by simp
  next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed) (auto split: prod.splits)
  show ?thesis
    unfolding out-eq
    by (rule empty-query-bad-imp-current-empty-opening-hit-at-zero-on-support
      [OF wf controlled])
      (use support bad out-eq in simp-all)
qed
qed

```

lemma *checked-staged-security-with-data-state-current-empty-query-opening-hit-at-bound-from-prefix-and-path*:

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *i-bound*: $i < \text{rounds}$

and *prefix-bound*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at *i*)

adversary-initial-state $\leq P$

and *path-bound*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at *i*)

adversary-initial-state $\leq T$

shows

wp-event

(checked-staged-security-experiment-with-data-state *A*)

(staged-security-with-data-state-current-empty-query-opening-hit-at *i*)

adversary-initial-state $\leq P + T$

proof —

have *current-le-prefix*:

wp-event

(checked-staged-security-experiment-with-data-state *A*)

(staged-security-with-data-state-current-empty-query-opening-hit-at *i*)

adversary-initial-state $\leq$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at *i*)

adversary-initial-state

by (rule

checked-staged-security-with-data-state-current-empty-query-opening-hit-at-le-query-prefix-current
[OF *wf controlled i-bound*])

have *prefix-bound'*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at *i*)

adversary-initial-state $\leq P + T$

by (rule

checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at-bound-from-prefix-and-path
[OF *wf controlled i-bound prefix-bound path-bound*])

show *?thesis*

by (rule *order-trans*[OF *current-le-prefix prefix-bound'*])

qed

lemma *checked-staged-security-with-data-state-current-empty-query-opening-hit-at-bound-from-conceptual-empty-res*

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *i-bound*: $i < \text{rounds}$
and *residual-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at*
 i)
 adversary-initial-state $\leq R$
and *path-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at*
 i)
 adversary-initial-state $\leq T$
shows
 wp-event
 (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-current-empty-query-opening-hit-at* i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets* $i + 1$) +
 query-error-bound + $R + T$
proof –
 have *prefix-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at*
 i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets* $i + 1$) +
 query-error-bound + R
 by (rule
 checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at-bound-from-conceptual-empty
 [*OF wf controlled i-bound residual-bound*])
 have *current-bound*:
 wp-event
 (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-current-empty-query-opening-hit-at* i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets* $i + 1$) +
 query-error-bound + R) + T
 by (rule
 checked-staged-security-with-data-state-current-empty-query-opening-hit-at-bound-from-prefix-and-path
 [*OF wf controlled i-bound prefix-bound path-bound*])
 then show ?thesis
 by (*simp add: algebra-simps*)
qed

lemma *checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at-bound-from-transcript:*
assumes *i-bound*: $i < \text{rounds}$
and *pre-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 staged-security-with-data-state-transcript-pre-hit
 adversary-initial-state $\leq P$
and *new-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 staged-security-with-data-state-transcript-new-hit
 adversary-initial-state $\leq R$
shows
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)
 (*checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at*
 i)
 adversary-initial-state $\leq P + R$
proof –
have *event-le*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)
 (*checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at*
 i)
 adversary-initial-state $\leq$
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)
 checked-staged-security-with-query-prefix-transcript-hit
 adversary-initial-state
proof (rule *wp-event-mono-on-support*)
fix *out*
assume *support*:
 out $\in$
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)
 adversary-initial-state)
and *hit*:
 checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at
 i out
show *checked-staged-security-with-query-prefix-transcript-hit out*
proof (cases *out*)
case *None*
then show *?thesis*
 using *hit*
 unfolding
 checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at-def
 checked-staged-security-with-query-prefix-empty-trace-path-output-hit-def
 by *simp*
next
case (*Some packed*)

```

then obtain prefix prefix-state raw raw-state data attacker-state result
final-state where out-eq:
  out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
have mapped-support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i >>=
          ( $\lambda$ packed. return (snd packed))))
        adversary-initial-state)
  unfolding out-eq
  by (rule set-dist-bindI[OF support[unfolded out-eq]]) simp
have projection:
  checked-staged-security-experiment-with-query-prefix-data-state A i >>=
    ( $\lambda$ packed. return (snd packed)) =
    checked-staged-security-experiment-with-data-state A
  by (rule checked-staged-security-experiment-with-query-prefix-data-state-projection
    [OF i-bound])
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  using mapped-support unfolding projection .
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
have verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
have transcript-split:
  hash-map-output-values ?s ∩ set (staged-proof-transcript data)
    ≠ {} ∨
  hash-map-new-output-hit (set (staged-proof-transcript data))
    ?s final-state
  by (rule verify-monad-trace-root-target-preexisting-or-new[OF verifier])
    simp
show ?thesis
  unfolding out-eq
    checked-staged-security-with-query-prefix-transcript-hit-def
  using transcript-split by simp
qed
qed
have transcript-bound:

```

```

wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  checked-staged-security-with-query-prefix-transcript-hit
  adversary-initial-state  $\leq P + R$ 
by (rule
  checked-staged-security-with-query-prefix-transcript-hit-bound-from-data-state
  [OF i-bound pre-bound new-bound])
show ?thesis
  by (rule order-trans[OF event-le transcript-bound])
qed

lemma checked-staged-security-with-data-state-current-empty-query-opening-hit-at-bound-from-conceptual-empty-residual-hit-at
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and residual-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at i)
      adversary-initial-state  $\leq E$ 
  and pre-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-pre-hit
      adversary-initial-state  $\leq P$ 
  and new-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-new-hit
      adversary-initial-state  $\leq R$ 
shows
  wp-event
    (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-current-empty-query-opening-hit-at i)
    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    query-error-bound +  $E + (P + R)$ 
proof -
  have path-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at i)
      adversary-initial-state  $\leq P + R$ 
  by (rule
    checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at-bound-from-transcript
    [OF i-bound pre-bound new-bound])
show ?thesis
  by (rule

```

checked-staged-security-with-data-state-current-empty-query-opening-hit-at-bound-from-conceptual-empty-residual
 $[OF \text{ wf controlled } i\text{-bound residual-bound path-bound}]$
qed

lemma *checked-staged-security-with-data-state-empty-header-bad-event-bound-from-prefix-drift-trace-fri-and-current-*

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *prefix-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

(*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
composition-trace-bad-alpha-space)

adversary-initial-state $\leq P$

and *drift-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
composition-trace-bad-alpha-space)

adversary-initial-state $\leq D$

and *trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)

adversary-initial-state $\leq \text{trace-fri-error}$

and *current-empty-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-current-empty-query-opening-hit-at 0*)
adversary-initial-state $\leq \text{empty-query-error}$

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*
soundness-bad-event-partial-candidate-empty-header)

adversary-initial-state $\leq$

$P + D +$

staged-phase-relation-error size

(*staged-alpha-search-queries budgets 0*) +

trace-fri-error + *empty-query-error*

proof –

have *query-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*
query-bad-with-empty-composition-header-candidates)

adversary-initial-state $\leq \text{empty-query-error}$

by (*rule order-trans*[

OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero[
OF wf controlled] *current-empty-bound*])

```

show ?thesis
  by (rule
    checked-staged-security-with-data-state-empty-header-bad-event-bound-from-prefix-drift-trace-fri-and-query
    [OF false-statement wf controlled prefix-bound drift-bound
      trace-fri-bound query-bound])
qed

end

end

theory Soundness-Staged-Aligned-Partial-Merkle
imports
  Soundness-Staged-Partial-Merkle
  Soundness-Aligned-Partial-Query-Transcript
  Staged-Security-Experiment-Alpha-Header-Binding
  Staged-Security-Experiment-Composition-Query-Current-Alignment
  Soundness-Conceptual-Query-Empty-Current
begin

  Staged partial-Merkle packaging that uses aligned nonempty-header query evidence.
  This downstream layer avoids importing verifier-query extraction machinery into the
  upstream deterministic reductions.

context soundness
begin

lemma partial-merkle-inconsistency-bad-imp-aligned-partial-candidate-with-partial-merkle:
  assumes partial-merkle-inconsistency-bad s out
  shows
    soundness-bad-event-aligned-partial-candidate-with-partial-merkle s out
  using assms
  unfolding
    soundness-bad-event-aligned-partial-candidate-with-partial-merkle-def
  by simp

lemma soundness-bad-event-aligned-partial-candidate-imp-with-partial-merkle:
  assumes soundness-bad-event-aligned-partial-candidate s out
  shows
    soundness-bad-event-aligned-partial-candidate-with-partial-merkle s out
  using assms
  unfolding
    soundness-bad-event-aligned-partial-candidate-with-partial-merkle-def
  by simp

lemma accepted-partition-soundness-bad-event-aligned-partial-candidate-with-partial-merkle-if-header-nonempty:
  assumes false-statement:  $\neg$  exists-valid-trace
  and supp: out  $\in$  set-dist (execute verify-monad s)
  and acc: accepted out

```

```

and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and comp-nonempty: composition-fri-roots  $\neq$  []
shows
  soundness-bad-event-aligned-partial-candidate-with-partial-merkle s out
proof –
from acc obtain result final-state where out-eq:
  out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
from
  verify-monad-accepted-with-partial-initial-openings-aligned-consistent
  [OF supp[unfolded out-eq] header comp-nonempty]
obtain query-idxs trace-openings composition-openings where partial:
  accepted-with-partial-initial-openings-aligned-consistent s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
by blast
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then show ?thesis
  unfolding out-eq
  by (rule
    partial-merkle-inconsistency-bad-imp-aligned-partial-candidate-with-partial-merkle)
next
case False
have aligned:
  accepted-with-partial-initial-openings-aligned s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
by (rule
  accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
  [OF partial])
obtain trace-table composition-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
using
  accepted-with-partial-initial-openings-aligned-candidates-if-no-partial-merkle-bad
  [OF aligned False]
by blast
have bad:
  soundness-bad-event-aligned-partial-candidate s
  (Some (result, final-state))

```

```

    by (rule accepted-aligned-partial-candidate-partition
        [OF false-statement partial trace-candidate composition-candidate])
  then show ?thesis
    unfolding out-eq
    by (rule soundness-bad-event-aligned-partial-candidate-imp-with-partial-merkle)
qed
qed

```

lemma *accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-if-header-non*

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *supp*: $\text{out} \in \text{set-dist } (\text{execute verify-monad } s)$

and *acc*: *accepted out*

and *header*:

verifier-header-transcript s fr f-fri-roots f-final as dg

composition-fri-roots final rest

and *comp-nonempty*: *composition-fri-roots* $\neq []$

shows

soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle

s out

proof –

from *acc* **obtain** *result final-state* **where** *out-eq*:

out = Some (result, final-state)

unfolding *accepted-def* **by** (*cases out*) *auto*

from *verify-monad-supplied-header-transcript-shape*

[OF *supp*[*unfolded out-eq*] *header*]

obtain *query-idxs raw-idxs query-chunks* **where** *shape*:

accepted-transcript-shape s (Some (result, final-state)) as query-idxs

and *len-raw*: *length raw-idxs = rounds*

and *query-idxs-eq*:

query-idxs = map ($\lambda \text{raw. index (to-nat raw)}$) raw-idxs

and *len-query-chunks*: *length query-chunks = rounds*

and *transcript-rest*:

List.concat query-chunks @ PTranscript final-state = rest

and *query-chunk-header*:

$\bigwedge i. i < \text{rounds} \implies$

verifier-query-round-chunk (query-idxs ! i)

f-fri-roots composition-fri-roots (query-chunks ! i)

and *replay-lookup-header*:

$\bigwedge i. i < \text{rounds} \implies$

fmlookup (HashMap final-state)

(QueryIndexChallenge (PQueryCounter s + i)

(state-after-query-chunks

(verifier-header-state s fr f-fri-roots f-final as dg

composition-fri-roots final)

query-chunks i)) =

Some (raw-idxs ! i)

by *blast*

from *verify-monad-supplied-header-query-rounds*

[OF *supp*[*unfolded out-eq*] *header*]


```

obtain f-fl fl query-state where f-roots:
  map snd f-fl = f-fri-roots
and comp-roots: map snd fl = composition-fri-roots
and query-transcript-state: PTranscript query-state = rest
and query-hash-state:
  PState query-state =
    verifier-header-state s fr f-fri-roots f-final as dg
    composition-fri-roots final
and query-counter: PQueryCounter query-state = PQueryCounter s
and query-out:
  Some (result, final-state) ∈
    set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      query-state)
by blast
have fl-nonempty: fl ≠ []
  using comp-nonempty comp-roots by (cases fl) simp-all
then obtain b composition-root fl-tail where fl-eq:
  fl = (b, composition-root) # fl-tail
by (cases fl) auto
have transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
  using query-transcript-state transcript-rest by simp
have query-chunk:
  ∧i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
  using query-chunk-header f-roots comp-roots by simp
have replay-lookup:
  ∧i. i < rounds ⇒
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
  using replay-lookup-header query-hash-state query-counter by simp
from
  query-rounds-replay-accepted-with-partial-initial-openings-aligned-transcript-consistent
  [OF header comp-nonempty fl-eq f-roots comp-roots
    query-transcript-state query-hash-state query-counter query-out
    len-raw query-idxs-eq len-query-chunks transcript-query query-chunk
    replay-lookup]
obtain trace-openings composition-openings where partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings

```

```

    composition-openings
  by blast
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then show ?thesis
  unfolding out-eq
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def
  by simp
next
case False
have aligned0:
  accepted-with-partial-initial-openings-aligned s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
using partial
by (blast dest:
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
  accepted-with-partial-initial-openings-aligned-consistent-imp-aligned)
obtain trace-table composition-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
using
  accepted-with-partial-initial-openings-aligned-candidates-if-no-partial-merkle-bad
  [OF aligned0 False]
by blast
have bad:
  soundness-bad-event-aligned-transcript-partial-candidate s
    (Some (result, final-state))
  by (rule accepted-aligned-transcript-partial-candidate-partition
    [OF false-statement partial trace-candidate composition-candidate])
then show ?thesis
  unfolding out-eq
  by (simp add:
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def)
qed
qed

lemma accepted-partition-soundness-bad-event-aligned-partial-candidate-with-partial-merkle-or-empty-header:
  assumes false-statement:  $\neg$  exists-valid-trace
  and supp:  $out \in \text{set-dist } (\text{execute verify-monad } s)$ 
  and acc:  $\text{accepted } out$ 
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest

```

```

shows
  soundness-bad-event-aligned-partial-candidate-with-partial-merkle s out ∨
  soundness-bad-event-partial-candidate-empty-header s out
proof (cases composition-fri-roots = [])
case False
then have comp-nonempty: composition-fri-roots ≠ []
  by simp
have soundness-bad-event-aligned-partial-candidate-with-partial-merkle s
  out
  by (rule
    accepted-partition-soundness-bad-event-aligned-partial-candidate-with-partial-merkle-if-header-nonempty
    [OF false-statement supp acc header comp-nonempty])
then show ?thesis by simp
next
case True
note comp-empty = True
from acc obtain result final-state where out-eq:
  out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then have
  soundness-bad-event-aligned-partial-candidate-with-partial-merkle s out
  unfolding out-eq
  by (rule
    partial-merkle-inconsistency-bad-imp-aligned-partial-candidate-with-partial-merkle)
then show ?thesis by simp
next
case False
from verify-monad-accepted-with-partial-trace-openings-for-header
  [OF supp[unfolded out-eq] header]
obtain trace-query-idxs trace-openings where trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr trace-query-idxs trace-openings
  by blast
obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
  using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
  [OF trace-partial False]
  by blast
have accepted-out: accepted (Some (result, final-state))
  by (rule accepted-with-partial-trace-openings-imp-accepted
    [OF trace-partial])
let ?composition-table = replicate (scale * clength) final
have empty-partial:
  accepted-with-empty-composition-header-candidates s
  (Some (result, final-state)) fr f-fri-roots f-final as dg final

```

```

    trace-query-idxs trace-openings trace-table ?composition-table
unfolding accepted-with-empty-composition-header-candidates-def
by (intro conjI exI[of - rest])
    (use accepted-out header comp-empty trace-partial trace-candidate
    in simp-all)
have soundness-bad-event-partial-candidate-empty-header s
    (Some (result, final-state))
by (rule accepted-empty-composition-header-candidate-partition
    [OF false-statement empty-partial])
then show ?thesis
    unfolding out-eq by simp
qed
qed

lemma accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-or-empty-head
assumes false-statement:  $\neg$  exists-valid-trace
and supp:  $out \in \text{set-dist } (\text{execute verify-monad } s)$ 
and acc: accepted out
and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
shows
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
    s out  $\vee$ 
    soundness-bad-event-partial-candidate-empty-header s out
proof (cases composition-fri-roots = [])
case False
then have comp-nonempty: composition-fri-roots  $\neq$  []
by simp
have
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
    s out
by (rule
    accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-if-header-nonempty
    [OF false-statement supp acc header comp-nonempty])
then show ?thesis by simp
next
case True
note comp-empty = True
from acc obtain result final-state where out-eq:
    out = Some (result, final-state)
unfolding accepted-def by (cases out) auto
show ?thesis
proof (cases
    partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then have
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
    s out

```

```

    unfolding out-eq
      soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def
    by simp
  then show ?thesis by simp
next
case False
from verify-monad-accepted-with-partial-trace-openings-for-header
  [OF supp[unfolded out-eq] header]
obtain trace-query-idxs trace-openings where trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr trace-query-idxs trace-openings
by blast
obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
  [OF trace-partial False]
by blast
have accepted-out: accepted (Some (result, final-state))
  by (rule accepted-with-partial-trace-openings-imp-accepted
    [OF trace-partial])
let ?composition-table = replicate (scale * clength) final
have empty-partial:
  accepted-with-empty-composition-header-candidates s
  (Some (result, final-state)) fr f-fri-roots f-final as dg final
  trace-query-idxs trace-openings trace-table ?composition-table
unfolding accepted-with-empty-composition-header-candidates-def
by (intro conjI exI[of - rest])
  (use accepted-out header comp-empty trace-partial trace-candidate
    in simp-all)
have soundness-bad-event-partial-candidate-empty-header s
  (Some (result, final-state))
  by (rule accepted-empty-composition-header-candidate-partition
    [OF false-statement empty-partial])
then show ?thesis
  unfolding out-eq by simp
qed
qed

lemma checked-staged-soundness-from-aligned-partial-candidate-and-empty-header-bounds:
  fixes partial-candidate-error empty-header-error :: prob
  assumes false-statement:  $\neg$  exists-valid-trace
  and partial-candidate-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-aligned-partial-candidate-with-partial-merkle)
      adversary-initial-state  $\leq$  partial-candidate-error
  and empty-header-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event

```

```

    soundness-bad-event-partial-candidate-empty-header)
  adversary-initial-state ≤ empty-header-error
shows checked-staged-adversary-acceptance-probability  $A \leq$ 
  partial-candidate-error + empty-header-error
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?partial =
    ?E soundness-bad-event-aligned-partial-candidate-with-partial-merkle
  let ?empty = ?E soundness-bad-event-partial-candidate-empty-header
  let ?bad =  $\lambda out. ?partial\ out \vee ?empty\ out$ 
  have accepted-bad:
    wp-event ?M accepted adversary-initial-state ≤
      wp-event ?M ?bad adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:  $out \in \text{set-dist } (\text{execute } ?M \text{ adversary-initial-state})$ 
  and acc: accepted out
  show ?bad out
  proof (cases out)
  case None
  then show ?thesis
    using acc unfolding accepted-def by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have verifier:
    Some (result, final-state)  $\in \text{set-dist } (\text{execute verify-monad } ?s)$ 
    using checked-staged-security-experiment-with-data-state-outcomeE
      [OF support[unfolded out-eq]]
  by blast
  have verifier-acc: accepted (Some (result, final-state))
    unfolding accepted-def by simp
  from verify-monad-accepted-transcript-shape[OF verifier]
  obtain alphas query-idxs where shape:
    accepted-transcript-shape ?s (Some (result, final-state))
    alphas query-idxs
  by blast
  from shape obtain result' final-state' fr f-fri-roots f-final dg
    composition-fri-roots final rest where
    header:
      verifier-header-transcript ?s fr f-fri-roots f-final alphas dg
      composition-fri-roots final rest
  by (elim accepted-transcript-shape-header-query-extraction)

```

```

have local-bad:
  soundness-bad-event-aligned-partial-candidate-with-partial-merkle ?s
    (Some (result, final-state)) ∨
  soundness-bad-event-partial-candidate-empty-header ?s
    (Some (result, final-state))
  by (rule
    accepted-partition-soundness-bad-event-aligned-partial-candidate-with-partial-merkle-or-empty-header
      [OF false-statement verifier verifier-acc header])
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
have bad-bound:
  wp-event ?M ?bad adversary-initial-state ≤
    partial-candidate-error + empty-header-error
proof –
  have wp-event ?M ?bad adversary-initial-state ≤
    wp-event ?M ?partial adversary-initial-state +
    wp-event ?M ?empty adversary-initial-state
    by (rule wp-event-union-bound)
  also have ... ≤ partial-candidate-error + empty-header-error
    by (intro add-mono partial-candidate-bound empty-header-bound)
  finally show ?thesis .
qed
have staged-bound:
  wp-event (checked-staged-security-experiment A) accepted
    adversary-initial-state ≤
    partial-candidate-error + empty-header-error
  unfolding checked-staged-security-experiment-acceptance-with-data-state
  by (rule order-trans[OF accepted-bad bad-bound])
show ?thesis
proof –
  have
    wp-event (checked-staged-security-experiment A)
      (λout. ¬ Option.is-none out) adversary-initial-state =
    wp-event (checked-staged-security-experiment A) accepted
      adversary-initial-state
  unfolding wp-event-def accepted-def
  by (rule arg-cong[where
    f=λQ. wp (checked-staged-security-experiment A) Q
    adversary-initial-state])
    (rule ext, simp)
  then show ?thesis
    unfolding checked-staged-adversary-acceptance-probability-def
    using staged-bound by simp
qed
qed

```

```

lemma checked-staged-soundness-from-aligned-transcript-partial-candidate-and-empty-header-bounds:
  fixes partial-candidate-error empty-header-error :: prob
  assumes false-statement:  $\neg$  exists-valid-trace
  and partial-candidate-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle)
      adversary-initial-state  $\leq$  partial-candidate-error
  and empty-header-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-partial-candidate-empty-header)
      adversary-initial-state  $\leq$  empty-header-error
  shows checked-staged-adversary-acceptance-probability A  $\leq$ 
    partial-candidate-error + empty-header-error
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?partial =
    ?E soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
  let ?empty = ?E soundness-bad-event-partial-candidate-empty-header
  let ?bad =  $\lambda$ out. ?partial out  $\vee$  ?empty out
  have accepted-bad:
    wp-event ?M accepted adversary-initial-state  $\leq$ 
      wp-event ?M ?bad adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support: out  $\in$  set-dist (execute ?M adversary-initial-state)
  and acc: accepted out
  show ?bad out
  proof (cases out)
  case None
  then show ?thesis
    using acc unfolding accepted-def by simp
  next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have verifier:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)
  using checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
  by blast
  have verifier-acc: accepted (Some (result, final-state))
    unfolding accepted-def by simp

```



```

from verify-monad-accepted-transcript-shape[OF verifier]
obtain alphas query-idxs where shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  alphas query-idxs
by blast
from shape obtain result' final-state' fr f-fri-roots f-final dg
  composition-fri-roots final rest where
    header:
      verifier-header-transcript ?s fr f-fri-roots f-final alphas dg
      composition-fri-roots final rest
by (elim accepted-transcript-shape-header-query-extraction)
have local-bad:
  soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
    ?s (Some (result, final-state))  $\vee$ 
  soundness-bad-event-partial-candidate-empty-header ?s
    (Some (result, final-state))
by (rule
  accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-or-empty-head
    [OF false-statement verifier verifier-acc header])
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
have bad-bound:
  wp-event ?M ?bad adversary-initial-state  $\leq$ 
    partial-candidate-error + empty-header-error
proof –
  have wp-event ?M ?bad adversary-initial-state  $\leq$ 
    wp-event ?M ?partial adversary-initial-state +
    wp-event ?M ?empty adversary-initial-state
    by (rule wp-event-union-bound)
  also have ...  $\leq$  partial-candidate-error + empty-header-error
    by (intro add-mono partial-candidate-bound empty-header-bound)
  finally show ?thesis .
qed
have staged-bound:
  wp-event (checked-staged-security-experiment A) accepted
    adversary-initial-state  $\leq$ 
    partial-candidate-error + empty-header-error
  unfolding checked-staged-security-experiment-acceptance-with-data-state
  by (rule order-trans[OF accepted-bad bad-bound])
show ?thesis
proof –
  have
    wp-event (checked-staged-security-experiment A)
      ( $\lambda$ out.  $\neg$  Option.is-none out) adversary-initial-state =
    wp-event (checked-staged-security-experiment A) accepted
      adversary-initial-state

```

```

unfolding wp-event-def accepted-def
by (rule arg-cong[where
  f= $\lambda Q. wp \ (checked-staged-security-experiment \ A) \ Q$ 
  adversary-initial-state])
  (rule ext, simp)
then show ?thesis
  unfolding checked-staged-adversary-acceptance-probability-def
  using staged-bound by simp
qed
qed

lemma checked-staged-soundness-from-aligned-partial-candidate-components-and-empty-header-bound:
fixes merkle-error composition-error trace-fri-error'
  composition-fri-error' query-error empty-header-error :: prob
assumes false-statement:  $\neg \text{exists-valid-trace}$ 
and merkle-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      partial-merkle-inconsistency-bad)
    adversary-initial-state  $\leq$  merkle-error
and comp-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  composition-error
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  composition-fri-error'
and query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      query-bad-with-aligned-partial-candidates)
    adversary-initial-state  $\leq$  query-error
and empty-header-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      soundness-bad-event-partial-candidate-empty-header)
    adversary-initial-state  $\leq$  empty-header-error
shows checked-staged-adversary-acceptance-probability A  $\leq$ 
  merkle-error + composition-error + trace-fri-error' +
  composition-fri-error' + query-error + empty-header-error
proof -

```

```

have partial-candidate-bound:
  wp-event ( checked-staged-security-experiment-with-data-state A )
    ( staged-security-with-data-state-verifier-event
      soundness-bad-event-aligned-partial-candidate-with-partial-merkle )
  adversary-initial-state ≤
    merkle-error + composition-error + trace-fri-error' +
    composition-fri-error' + query-error
  by (rule
    checked-staged-security-with-data-state-aligned-partial-candidate-with-partial-merkle-union-bound
    [OF merkle-bound comp-bound trace-fri-bound comp-fri-bound
      query-bound])
have bound:
  checked-staged-adversary-acceptance-probability A ≤
    (merkle-error + composition-error + trace-fri-error' +
      composition-fri-error' + query-error) + empty-header-error
  by (rule
    checked-staged-soundness-from-aligned-partial-candidate-and-empty-header-bounds
    [OF false-statement partial-candidate-bound empty-header-bound])
then show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-bound:
fixes merkle-error composition-error trace-fri-error'
  composition-fri-error' query-error empty-header-error :: prob
assumes false-statement: ¬ exists-valid-trace
and merkle-bound:
  wp-event ( checked-staged-security-experiment-with-data-state A )
    ( staged-security-with-data-state-verifier-event
      partial-merkle-inconsistency-bad )
  adversary-initial-state ≤ merkle-error
and comp-bound:
  wp-event ( checked-staged-security-experiment-with-data-state A )
    ( staged-security-with-data-state-verifier-event
      composition-bad-with-partial-candidates )
  adversary-initial-state ≤ composition-error
and trace-fri-bound:
  wp-event ( checked-staged-security-experiment-with-data-state A )
    ( staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates )
  adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
  wp-event ( checked-staged-security-experiment-with-data-state A )
    ( staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates )
  adversary-initial-state ≤ composition-fri-error'
and query-bound:
  wp-event ( checked-staged-security-experiment-with-data-state A )
    ( staged-security-with-data-state-verifier-event

```

```

    query-bad-with-aligned-transcript-partial-candidates)
  adversary-initial-state  $\leq$  query-error
and empty-header-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     soundness-bad-event-partial-candidate-empty-header)
  adversary-initial-state  $\leq$  empty-header-error
shows checked-staged-adversary-acceptance-probability A  $\leq$ 
  merkle-error + composition-error + trace-fri-error' +
  composition-fri-error' + query-error + empty-header-error
proof –
have partial-candidate-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle)
  adversary-initial-state  $\leq$ 
  merkle-error + composition-error + trace-fri-error' +
  composition-fri-error' + query-error
by (rule
  checked-staged-security-with-data-state-aligned-transcript-partial-candidate-with-partial-merkle-component-union
  [OF merkle-bound comp-bound trace-fri-bound comp-fri-bound
   query-bound])
have bound:
  checked-staged-adversary-acceptance-probability A  $\leq$ 
  (merkle-error + composition-error + trace-fri-error' +
   composition-fri-error' + query-error) + empty-header-error
by (rule
  checked-staged-soundness-from-aligned-transcript-partial-candidate-and-empty-header-bounds
  [OF false-statement partial-candidate-bound empty-header-bound])
then show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-soundness-from-aligned-partial-candidate-components-and-empty-header-components:
fixes trace-fri-error' composition-fri-error' query-error P D :: prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-partial-candidates)
  adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-fri-bad-with-partial-candidates)
  adversary-initial-state  $\leq$  composition-fri-error'
and query-bound:

```

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-partial-candidates)
adversary-initial-state $\leq$ *query-error*
and *prefix-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
 (*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ *P*
and *drift-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ *D*
and *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *nnreal rounds * query-error-bound*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*P* + *D* +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)) +
trace-fri-error' + composition-fri-error' +
query-error +
 (*P* + *D* +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
*trace-fri-error + nnreal rounds * query-error-bound*)
proof –
have *comp-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
composition-bad-with-partial-candidates)
adversary-initial-state $\leq$
P + D +
staged-phase-relation-error size

(staged-alpha-search-queries budgets 0)
by (rule
 checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-prefix-drift-and-budgets
 [OF false-statement wf controlled prefix-bound drift-bound])
have merkle-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 partial-merkle-inconsistency-bad)
 adversary-initial-state $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
 checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
 [OF wf controlled])
have empty-header-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 soundness-bad-event-partial-candidate-empty-header)
 adversary-initial-state $\leq$
 $P + D +$
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + nnreal rounds * query-error-bound
by (rule
 checked-staged-security-with-data-state-empty-header-bad-event-bound-from-prefix-drift-trace-fri-and-query
 [OF false-statement wf controlled prefix-bound drift-bound
 empty-trace-fri-bound empty-query-bound])
have bound:
 checked-staged-adversary-acceptance-probability $A \leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($P + D +$
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)) +
 trace-fri-error' + composition-fri-error' +
 query-error +
 ($P + D +$
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + nnreal rounds * query-error-bound)
by (rule
 checked-staged-soundness-from-aligned-partial-candidate-components-and-empty-header-bound
 [OF false-statement merkle-bound comp-bound trace-fri-bound
 comp-fri-bound query-bound empty-header-bound])
then show ?thesis .
qed

lemma *checked-staged-soundness-from-aligned-partial-candidate-components-and-current-empty-query:*
fixes *trace-fri-error'* *composition-fri-error'* *query-error* P D
empty-query-error :: *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* A
and *trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*
and *query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-partial-candidates)
adversary-initial-state $\leq$ *query-error*
and *prefix-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
(*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ P
and *drift-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ D
and *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-current-empty-query-opening-hit-at* 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 $(P + D +$

staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
query-error +
(P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error)
proof –
have *comp-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-bad-with-partial-candidates)
adversary-initial-state ≤
P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)
by (rule
checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-prefix-drift-and-budgets
[OF false-statement wf controlled prefix-bound drift-bound])
have *merkle-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state ≤
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[OF wf controlled])
have *empty-header-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state ≤
P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-bound-from-prefix-drift-trace-fri-and-current-empty
[OF false-statement wf controlled prefix-bound drift-bound
empty-trace-fri-bound current-empty-bound])
have *bound*:
checked-staged-adversary-acceptance-probability A ≤
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +

$(P + D +$
 $\quad \text{staged-phase-relation-error size}$
 $\quad (\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $\text{query-error} +$
 $(P + D +$
 $\quad \text{staged-phase-relation-error size}$
 $\quad (\text{staged-alpha-search-queries budgets } 0) +$
 $\quad \text{trace-fri-error} + \text{empty-query-error})$
by (rule
 $\text{checked-staged-soundness-from-aligned-partial-candidate-components-and-empty-header-bound}$
 $[\text{OF false-statement merkle-bound comp-bound trace-fri-bound}$
 $\quad \text{comp-fri-bound query-bound empty-header-bound}])$
then show ?thesis .
qed

lemma *checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-components*
fixes $\text{trace-fri-error}' \text{ composition-fri-error}' \text{ query-error } P \ D :: \text{prob}$
assumes $\text{false-statement}: \neg \text{exists-valid-trace}$
and $\text{wf}: \text{staged-budget-wellformed budgets}$
and $\text{controlled}: \text{staged-adversary-controlled budgets } A$
and $\text{trace-fri-bound}: \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\quad (\text{staged-security-with-data-state-verifier-event}$
 $\quad \text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$
and $\text{comp-fri-bound}: \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\quad (\text{staged-security-with-data-state-verifier-event}$
 $\quad \text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$
and $\text{query-bound}: \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\quad (\text{staged-security-with-data-state-verifier-event}$
 $\quad \text{query-bad-with-aligned-transcript-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{query-error}$
and $\text{prefix-bound}: \text{wp-event}$
 $\quad (\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $\quad A)$
 $\quad (\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit}$
 $\quad \text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq P$
and $\text{drift-bound}: \text{wp-event}$
 $\quad (\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $\quad A)$
 $\quad (\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift}$
 $\quad \text{composition-trace-bad-alpha-space})$

$\text{adversary-initial-state} \leq D$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows *checked-staged-adversary-acceptance-probability* $A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(P + D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $\text{query-error} +$
 $(P + D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$
proof –
have *comp-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq$
 $P + D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)$
by (rule
 $\text{checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-prefix-drift-and-budgets}$
 $[\text{OF false-statement wf controlled prefix-bound drift-bound}])$
have *merkle-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{partial-merkle-inconsistency-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget})$
by (rule
 $\text{checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled}$
 $[\text{OF wf controlled}])$
have *empty-header-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state $\leq$
 $P + D +$
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nnreal rounds * query-error-bound*
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-bound-from-prefix-drift-trace-fri-and-query
[OF false-statement wf controlled prefix-bound drift-bound
empty-trace-fri-bound empty-query-bound])
have bound:
checked-staged-adversary-acceptance-probability A $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
 $(P + D +$
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
query-error +
 $(P + D +$
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nnreal rounds * query-error-bound)*
by (rule
checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-bound
[OF false-statement merkle-bound comp-bound trace-fri-bound
comp-fri-bound query-bound empty-header-bound])
then show ?thesis .
qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-partial-opening-hit:*
fixes *trace-fri-error' composition-fri-error' query-error P D :: prob*
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ trace-fri-error'
and *comp-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ composition-fri-error'
and *partial-opening-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-query-partial-opening-hit
adversary-initial-state $\leq$ *query-error*
and *prefix-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ *P*
and *drift-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ *D*
and *empty-trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *nnreal rounds * query-error-bound*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
query-error +
(P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nnreal rounds * query-error-bound)*
proof –
have *query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq$ *query-error*
by (rule *order-trans*[
OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-partial-opening-hit
partial-opening-bound])
(use wf controlled in simp-all)

show ?thesis

by (rule

checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-components
[*OF false-statement wf controlled trace-fri-bound comp-fri-bound*
query-bound prefix-bound drift-bound empty-trace-fri-bound
empty-query-bound])

qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit*:

fixes *trace-fri-error'* *composition-fri-error'* *query-error* *P* *D* :: prob

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets* *A*

and *trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*

and *comp-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*

and *current-query-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq$ *query-error*

and *prefix-bound*:

wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ *P*

and *drift-bound*:

wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ *D*

and *empty-trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*

and *empty-query-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*

query-bad-with-empty-composition-header-candidates
adversary-initial-state $\leq$ *nnreal rounds* * *query-error-bound*
shows *checked-staged-adversary-acceptance-probability* *A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
query-error +
(P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nnreal rounds * query-error-bound)*
proof –
have *query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq$ *query-error*
by (rule *order-trans*[
OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
current-query-bound])
(use wf controlled in simp-all)
show *?thesis*
by (rule
checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-components
[OF false-statement wf controlled trace-fri-bound comp-fri-bound
query-bound prefix-bound drift-bound empty-trace-fri-bound
empty-query-bound])
qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds*:
fixes *trace-fri-error' composition-fri-error' P D :: prob*
and *C :: nat $\Rightarrow$ prob*
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*

and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq P$
and *drift-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq D$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(P + D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}. C \ i) +$
 $(P + D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$
proof –
have *current-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-current-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq (\sum i < \text{rounds}. C \ i)$
by (rule

checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
 [OF round-bound wf controlled])
show ?thesis
by (rule
checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound
 current-query-bound prefix-bound drift-bound empty-trace-fri-bound
 empty-query-bound])
qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-partial-opening-hit-from-drift*:
fixes trace-fri-error' composition-fri-error' query-error D :: prob
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ trace-fri-error'
and comp-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ composition-fri-error'
and partial-opening-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-query-partial-opening-hit
 adversary-initial-state $\leq$ query-error
and drift-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq$ D
and empty-trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ trace-fri-error
and empty-query-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ nnreal rounds * query-error-bound
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +

$$\begin{aligned}
& \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) + \\
& (\text{composition-error-bound} + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{hash-collision-budget-value } 0 \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& D + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0)) + \\
& \text{trace-fri-error}' + \text{composition-fri-error}' + \\
& \text{query-error} + \\
& (\text{composition-error-bound} + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{hash-collision-budget-value } 0 \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& D + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})
\end{aligned}$$

proof –

have *comp-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

composition-bad-with-partial-candidates)

adversary-initial-state ≤

composition-error-bound +

staged-phase-relation-error size

(*staged-alpha-search-queries budgets* 0) +

hash-collision-budget-value 0

(*staged-alpha-search-queries budgets* 0) +

D +

staged-phase-relation-error size

(*staged-alpha-search-queries budgets* 0)

by (rule

checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-drift-and-budgets

[*OF false-statement wf controlled drift-bound*])

have *query-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

query-bad-with-aligned-transcript-partial-candidates)

adversary-initial-state ≤ *query-error*

by (rule *order-trans*[

OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-partial-opening-hit

partial-opening-bound])

(*use wf controlled in simp-all*)

have *merkle-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

```

    partial-merkle-inconsistency-bad)
  adversary-initial-state  $\leq$ 
  hash-collision-budget-value 0
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
  checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
  [OF wf controlled])
have empty-header-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   soundness-bad-event-partial-candidate-empty-header)
  adversary-initial-state  $\leq$ 
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) +
  D +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  trace-fri-error + nnreal rounds * query-error-bound
by (rule
  checked-staged-security-with-data-state-empty-header-bad-event-bound-from-drift-trace-fri-and-query
  [OF false-statement wf controlled drift-bound empty-trace-fri-bound
   empty-query-bound])
show ?thesis
by (rule
  checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-bound
  [OF false-statement merkle-bound comp-bound trace-fri-bound
   comp-fri-bound query-bound empty-header-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-drift:
  fixes trace-fri-error' composition-fri-error' query-error D :: prob
  assumes false-statement:  $\neg$  exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  trace-fri-error'
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  composition-fri-error'
  and current-query-bound:

```

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq$ query-error
and *drift-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ D
and *empty-trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ trace-fri-error
and *empty-query-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
*adversary-initial-state $\leq$ nnreal rounds * query-error-bound*
shows *checked-staged-adversary-acceptance-probability A $\leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
query-error +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nnreal rounds * query-error-bound)*
proof –
have *comp-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-bad-with-partial-candidates)
adversary-initial-state $\leq$

composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)
by (rule
checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-drift-and-budgets
[OF false-statement wf controlled drift-bound])
have *query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state ≤ query-error
by (rule *order-trans*[
OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
current-query-bound])
(use wf controlled in simp-all)
have *merkle-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state ≤
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[OF wf controlled])
have *empty-header-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state ≤
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nnreal rounds * query-error-bound*
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-bound-from-drift-trace-fri-and-query
[OF false-statement wf controlled drift-bound empty-trace-fri-bound
empty-query-bound])

show *?thesis*

by (rule

checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-bound
[*OF false-statement merkle-bound comp-bound trace-fri-bound*
comp-fri-bound query-bound empty-header-bound])

qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-partial-opening-hit-from-not-prefix:*

fixes *trace-fri-error'* *composition-fri-error'* *query-error* *N* :: *prob*

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets* *A*

and *trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*

and *comp-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*

and *partial-opening-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
staged-security-with-data-state-query-partial-opening-hit
adversary-initial-state $\leq$ *query-error*

and *not-prefix-bound*:

wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ *N*

and *empty-trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*

and *empty-query-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *nnreal* *rounds* * *query-error-bound*

shows *checked-staged-adversary-acceptance-probability* *A* $\leq$

hash-collision-budget-value *0*

(*staged-attacker-query-budget* *budgets* +

staged-challenge-query-budget + *verifier-hash-query-budget*) +

(*composition-error-bound* +

staged-phase-relation-error *size*

$(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $\text{query-error} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$

proof –

have *drift-bound*:

wp-event

$(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$

$(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift}$
 $\text{composition-trace-bad-alpha-space})$

$\text{adversary-initial-state} \leq N$

proof –

have *wp-event*

$(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$

$(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift}$
 $\text{composition-trace-bad-alpha-space})$

$\text{adversary-initial-state} \leq$

wp-event

$(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$

$(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$

$\text{adversary-initial-state}$

by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-le-not-prefix-bound}$
 $[OF \text{ wf controlled}]$)

also have $\dots \leq N$

by (rule *not-prefix-bound*)

finally show *?thesis* .

qed

show *?thesis*

by (rule
 $\text{checked-staged-soundness-from-aligned-transcript-components-and-partial-opening-hit-from-drift}$
 $[OF \text{ false-statement wf controlled trace-fri-bound comp-fri-bound}]$)

*partial-opening-bound drift-bound empty-trace-fri-bound
empty-query-bound*)

qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-not-prefix*
fixes *trace-fri-error'* *composition-fri-error'* *query-error* $N :: \text{prob}$
assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* A
and *trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *composition-fri-error'*
and *current-query-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 staged-security-with-data-state-current-query-partial-opening-hit
 adversary-initial-state $\leq$ *query-error*
and *not-prefix-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq$ N
and *empty-trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *nnreal* *rounds* * *query-error-bound*
shows *checked-staged-adversary-acceptance-probability* $A \leq$
 hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
 staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +

$N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $\text{query-error} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$

proof –

have *drift-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$

proof –

have *wp-event*
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq$

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state}$

by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-le-not-prefix-bound}$
 $[OF \text{ wf controlled}]$)

also have $\dots \leq N$
by (rule *not-prefix-bound*)

finally show *?thesis* .

qed

show *?thesis*

by (rule
 $\text{checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-drift}$
 $[OF \text{ false-statement wf controlled trace-fri-bound comp-fri-bound}$
 $\text{current-query-bound drift-bound empty-trace-fri-bound}$
 $\text{empty-query-bound}]$)

qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-not-prefix*
fixes *trace-fri-error'* *composition-fri-error'* *N* :: *prob*
and *C* :: *nat* $\Rightarrow$ *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *composition-fri-error'*
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i*)
 adversary-initial-state $\leq C i$
and *not-prefix-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq N$
and *empty-trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *nnreal rounds * query-error-bound*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
 hash-collision-budget-value 0
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +

$N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}. C i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$

proof –

have *current-query-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq (\sum i < \text{rounds}. C i)$

by (rule
checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
 $[OF \text{ round-bound wf controlled}]$)

show *?thesis*

by (rule
checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-not-prefix
 $[OF \text{ false-statement wf controlled trace-fri-bound comp-fri-bound}$
 $\text{current-query-bound not-prefix-bound empty-trace-fri-bound}$
 $\text{empty-query-bound}]$)

qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-prefix-target-path-bounds-from-not-prefix*:

fixes *trace-fri-error'* *composition-fri-error'* *N* :: *prob*
and *P Q* :: *nat* $\Rightarrow$ *prob*

assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
adversary-initial-state $\leq \text{trace-fri-error}'$

and *comp-fri-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
adversary-initial-state $\leq \text{composition-fri-error}'$

and *prefix-bound*:

$\bigwedge i. i < \text{rounds} \implies$
wp-event (checked-staged-query-prefix-receive-with-state A i)

(checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-authenticated-opening-target)
adversary-initial-state $\leq P$ *i*
and *path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-path-output-hit-at i)
adversary-initial-state $\leq Q$ *i*
and *not-prefix-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)
adversary-initial-state $\leq N$
and *empty-trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq \text{trace-fri-error}$
and *empty-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq \text{nnreal rounds} * \text{query-error-bound}$
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
N +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
 $(\sum i < \text{rounds}. P\ i + Q\ i) +$
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
N +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nnreal rounds * query-error-bound)*

```

proof –
  have round-bound:
     $\bigwedge i. i < \text{rounds} \implies$ 
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
        adversary-initial-state  $\leq P\ i + Q\ i$ 
proof –
  fix i
  assume i-bound: i < rounds
  have prefix-hit-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
      adversary-initial-state  $\leq P\ i$ 
    by (rule
      checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target
      [OF wf controlled i-bound prefix-bound [OF i-bound]])
  show
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state  $\leq P\ i + Q\ i$ 
    by (rule
      checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-path
      [OF wf controlled i-bound prefix-hit-bound path-bound [OF i-bound]])
  qed
show ?thesis
  by (rule
    checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-not-prefix
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      round-bound not-prefix-bound empty-trace-fri-bound
      empty-query-bound])
  qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-not-prefix:
  fixes trace-fri-error' composition-fri-error' N :: prob
  and Q :: nat  $\Rightarrow$  prob
  assumes false-statement:  $\neg \text{exists-valid-trace}$ 
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-partial-candidates)
      adversary-initial-state  $\leq \text{trace-fri-error}'$ 
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event

```

composition-fri-bad-with-partial-candidates
adversary-initial-state $\leq$ *composition-fri-error'*
and *trace-frac*:
 $\bigwedge i$ *prefix prefix-state*.
 $i < \text{rounds} \implies$
Some ((*prefix, prefix-state*), *prefix-state*) $\in$
set-dist
(*execute* (*checked-staged-query-prefix-with-state A i*)
adversary-initial-state) $\implies$
nnreal
(*card*
(*staged-query-prefix-prefix-authenticated-trace-indices prefix*
prefix-state)) /
nnreal (*card query-sample-space*) $\leq$ *query-error-bound*
and *path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
(*checked-staged-security-with-query-prefix-current-path-output-hit-at i*)
adversary-initial-state $\leq Q i$
and *not-prefix-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq N$
and *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *nnreal rounds * query-error-bound*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
(*composition-error-bound* +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
(*staged-alpha-search-queries budgets 0*) +
N +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*)) +

```

    trace-fri-error' + composition-fri-error' +
    (∑ i < rounds.
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      query-error-bound + Q i) +
    (composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      N +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      trace-fri-error + nnreal rounds * query-error-bound)
  proof -
  have round-bound:
    ∧ i. i < rounds ⇒
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
        adversary-initial-state ≤
      staged-phase-relation-error size
        (staged-query-search-queries budgets i + 1) +
        query-error-bound + Q i
  proof -
  fix i
  assume i-bound: i < rounds
  have prefix-hit-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
      adversary-initial-state ≤
      staged-phase-relation-error size
        (staged-query-search-queries budgets i + 1) + query-error-bound
  by (rule
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-trace-indices
    [OF wf controlled i-bound])
    (use trace-frac[OF i-bound] in blast)
  show
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state ≤
    staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
    query-error-bound + Q i
  by (rule
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-path
    [OF wf controlled i-bound prefix-hit-bound path-bound[OF i-bound]])

```

```

qed
show ?thesis
by (rule
  checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-not-prefix-co
  [OF false-statement wf controlled trace-fri-bound comp-fri-bound
    round-bound not-prefix-bound empty-trace-fri-bound
    empty-query-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-not-prefix-co
fixes trace-fri-error' composition-fri-error' C U M R :: prob
and Q :: nat  $\Rightarrow$  prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  composition-fri-error'
and trace-frac:
   $\bigwedge i$  prefix prefix-state.
   $i < \text{rounds} \implies$ 
  Some ((prefix, prefix-state), prefix-state)  $\in$ 
    set-dist
      (execute (checked-staged-query-prefix-with-state A i)
        adversary-initial-state)  $\implies$ 
  nnreal
    (card
      (staged-query-prefix-prefix-authenticated-trace-indices prefix
        prefix-state)) /
    nnreal (card query-sample-space)  $\leq$  query-error-bound
and path-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-path-output-hit-at i)
    adversary-initial-state  $\leq$  Q i
and collision-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
    adversary-initial-state  $\leq$  C

```

and *uncovered-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state $\leq U$

and *no-prefix-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq M$

and *not-prefix-path-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq R$

and *empty-trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq \text{trace-fri-error}$

and *empty-query-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq \text{nnreal rounds} * \text{query-error-bound}$

shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 (*C* + *U* + *M* + *R*) +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)) +
trace-fri-error' + *composition-fri-error'* +
 ($\sum i < \text{rounds}$.
staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
query-error-bound + *Q i*) +
 (*composition-error-bound* +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0

(staged-alpha-search-queries budgets 0) +
 (C + U + M + R) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + nnreal rounds * query-error-bound)

proof –

have not-prefix-bound:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound composition-trace-bad-alpha-space)

adversary-initial-state ≤ C + U + M + R

by (rule

checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound

[OF wf controlled collision-bound uncovered-bound no-prefix-bound not-prefix-path-bound])

show ?thesis

by (rule

checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-not-prefix-no

[OF false-statement wf controlled trace-fri-bound comp-fri-bound trace-frac path-bound not-prefix-bound empty-trace-fri-bound empty-query-bound])

qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-not-prefix-no

fixes trace-fri-error' composition-fri-error' U M R :: prob

and Q :: nat ⇒ prob

assumes false-statement: ¬ exists-valid-trace

and wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and trace-fri-bound:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

trace-fri-bad-with-partial-candidates)

adversary-initial-state ≤ trace-fri-error'

and comp-fri-bound:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

composition-fri-bad-with-partial-candidates)

adversary-initial-state ≤ composition-fri-error'

and trace-frac:

∧ i prefix prefix-state.

i < rounds ⇒

Some ((prefix, prefix-state), prefix-state) ∈

set-dist

(execute (checked-staged-query-prefix-with-state A i)

adversary-initial-state) ⇒

nnreal

(card
 (staged-query-prefix-prefix-authenticated-trace-indices prefix
 prefix-state)) /
 nnreal (card query-sample-space) $\leq$ query-error-bound
and path-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-path-output-hit-at i)
 adversary-initial-state $\leq Q i$
and uncovered-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
 adversary-initial-state $\leq U$
and no-prefix-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
 adversary-initial-state $\leq M$
and not-prefix-path-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
 adversary-initial-state $\leq R$
and empty-trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq \text{trace-fri-error}$
and empty-query-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq \text{nnreal rounds} * \text{query-error-bound}$
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 (hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + U + M + R) +

staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
($\sum i < \text{rounds}$.
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) +
query-error-bound + $Q i$) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + $U + M + R$) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nnreal rounds * query-error-bound)*

proof (rule
checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-not-prefix-compon
[OF false-statement wf controlled trace-fri-bound comp-fri-bound
trace-frac path-bound - uncovered-bound no-prefix-bound
not-prefix-path-bound empty-trace-fri-bound empty-query-bound])

show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
adversary-initial-state $\leq$
hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0)

by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
[OF wf controlled])

qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-cross-no-pre*
fixes *trace-fri-error' composition-fri-error' $X M R :: \text{prob}$*
and *$Q :: \text{nat} \Rightarrow \text{prob}$*
assumes *false-statement: $\neg \text{exists-valid-trace}$*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ trace-fri-error'
and *comp-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)

$adversary-initial-state \leq composition-fri-error'$
and *trace-frac*:
 $\bigwedge i \text{ prefix prefix-state.}$
 $i < rounds \implies$
 $Some ((prefix, prefix-state), prefix-state) \in$
 $set-dist$
 $(execute (checked-staged-query-prefix-with-state A i)$
 $adversary-initial-state) \implies$
 $nnreal$
 $(card$
 $(staged-query-prefix-prefix-authenticated-trace-indices prefix$
 $prefix-state)) /$
 $nnreal (card query-sample-space) \leq query-error-bound$
and *path-bound*:
 $\bigwedge i. i < rounds \implies$
 $wp-event$
 $(checked-staged-security-experiment-with-query-prefix-data-state A i)$
 $(checked-staged-security-with-query-prefix-current-path-output-hit-at i)$
 $adversary-initial-state \leq Q i$
and *cross-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$
 $checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad$
 $adversary-initial-state \leq X$
and *no-prefix-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$
 $checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate$
 $adversary-initial-state \leq M$
and *not-prefix-path-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$
 $checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output$
 $adversary-initial-state \leq R$
and *empty-trace-fri-bound*:
 $wp-event (checked-staged-security-experiment-with-data-state A)$
 $(staged-security-with-data-state-verifier-event$
 $trace-fri-bad-with-empty-composition-header-candidates)$
 $adversary-initial-state \leq trace-fri-error$
and *empty-query-bound*:
 $wp-event (checked-staged-security-experiment-with-data-state A)$
 $(staged-security-with-data-state-verifier-event$
 $query-bad-with-empty-composition-header-candidates)$
 $adversary-initial-state \leq nnreal rounds * query-error-bound$
shows $checked-staged-adversary-acceptance-probability A \leq$
 $hash-collision-budget-value 0$

(staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 (hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + X + M + R) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)) +
 trace-fri-error' + composition-fri-error' +
 (∑ i < rounds.
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 query-error-bound + Q i) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 (hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + X + M + R) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + nnreal rounds * query-error-bound)

proof (rule
 checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-not-prefix-noncoll
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound
 trace-frac path-bound - no-prefix-bound not-prefix-path-bound
 empty-trace-fri-bound empty-query-bound])

show
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
 adversary-initial-state ≤ X

by (rule order-trans
 [OF
 checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad
 cross-bound])

qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-cross-no-pre.
fixes trace-fri-error' composition-fri-error' X M P R :: prob
and Q :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A

and *trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$

and *comp-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$

and *trace-frac*:
 $\bigwedge i \text{ prefix prefix-state.}$
 $i < \text{rounds} \implies$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 nnreal
 $(\text{card}$
 $(\text{staged-query-prefix-prefix-authenticated-trace-indices prefix}$
 $\text{prefix-state})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$

and *path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq Q \ i$

and *cross-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad}$
 $\text{adversary-initial-state} \leq X$

and *no-prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate}$
 $\text{adversary-initial-state} \leq M$

and *transcript-pre-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$

and *transcript-new-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$

A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq R$
and *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *nnreal rounds* * *query-error-bound*
shows *checked-staged-adversary-acceptance-probability* A $\leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 (*hash-collision-budget-value* 0
 (*staged-alpha-search-queries* *budgets* 0) + X + M + (P + R)) +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0)) +
trace-fri-error' + *composition-fri-error'* +
 ($\sum i < \text{rounds}$.
staged-phase-relation-error *size*
 (*staged-query-search-queries* *budgets* i + 1) +
query-error-bound + Q i) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 (*hash-collision-budget-value* 0
 (*staged-alpha-search-queries* *budgets* 0) + X + M + (P + R)) +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
trace-fri-error + *nnreal rounds* * *query-error-bound*)
proof (rule
checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-cross-no-prefix-path
 [OF *false-statement* wf *controlled trace-fri-bound comp-fri-bound*
trace-frac path-bound cross-bound no-prefix-bound - empty-trace-fri-bound
empty-query-bound])
show
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

```

    A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
    adversary-initial-state  $\leq P + R$ 
  by (rule order-trans
    [OF
      checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-le-witness-transcript-hit])
    (rule
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
      [OF transcript-pre-bound transcript-new-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-cross-no-pre,
fixes trace-fri-error' composition-fri-error' X M P R :: prob
and Q :: nat  $\Rightarrow$  prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  composition-fri-error'
and trace-frac:
   $\bigwedge i$  prefix prefix-state.
     $i < \text{rounds} \implies$ 
    Some ((prefix, prefix-state), prefix-state)  $\in$ 
      set-dist
        (execute (checked-staged-query-prefix-with-state A i)
          adversary-initial-state)  $\implies$ 
      nnreal
        (card
          (staged-query-prefix-prefix-authenticated-trace-indices prefix
            prefix-state)) /
        nnreal (card query-sample-space)  $\leq$  query-error-bound
and path-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-path-output-hit-at i)
    adversary-initial-state  $\leq Q i$ 
and cross-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)

```


checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state $\leq X$
and *no-prefix-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq M$
and *data-pre-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *transcript-new-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq R$
and *empty-trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq \text{trace-fri-error}$
and *empty-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq \text{nnreal rounds} * \text{query-error-bound}$
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + X + M + (P + R)) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
($\sum_{i < \text{rounds}}$.
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound + Q i) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +

```

    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    (hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) + X + M + (P + R)) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    trace-fri-error + nnreal rounds * query-error-bound)
proof (rule
  checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-cross-no-prefix-tra
  [OF false-statement wf controlled trace-fri-bound comp-fri-bound
  trace-frac path-bound cross-bound no-prefix-bound - transcript-new-bound
  empty-trace-fri-bound empty-query-bound])
show
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
  A)
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
  adversary-initial-state ≤ P
by (rule
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
  [OF data-pre-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-cross-data-p
fixes trace-fri-error' composition-fri-error' X P R :: prob
and Q :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
  trace-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
  composition-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ composition-fri-error'
and trace-frac:
  ∧ i prefix prefix-state.
  i < rounds ⇒
  Some ((prefix, prefix-state), prefix-state) ∈
  set-dist
  (execute (checked-staged-query-prefix-with-state A i)
  adversary-initial-state) ⇒
  nnreal
  (card
  (staged-query-prefix-prefix-authenticated-trace-indices prefix

```

$\text{prefix-state})) /$
 $\text{nnreal}(\text{card query-sample-space}) \leq \text{query-error-bound}$
and *path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq Q \ i$
and *cross-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad}$
 $\text{adversary-initial-state} \leq X$
and *data-pre-bound*:
 $\text{wp-event}(\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$
and *transcript-new-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit}$
 $\text{adversary-initial-state} \leq R$
and *empty-trace-fri-bound*:
 $\text{wp-event}(\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *empty-query-bound*:
 $\text{wp-event}(\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) + X + (P + R) + (P + R)) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}.$

staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) +
query-error-bound + $Q i$) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + $X + (P + R) + (P + R)$) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nnreal rounds * query-error-bound)*

proof (rule
checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-cross-no-prefix-da
[OF false-statement wf controlled trace-fri-bound comp-fri-bound
trace-frac path-bound cross-bound - data-pre-bound transcript-new-bound
empty-trace-fri-bound empty-query-bound])

show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq P + R$

proof (rule *order-trans*
[OF
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-le-witness-transcript-hit])

have *pre-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
adversary-initial-state $\leq P$

by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
[OF data-pre-bound])

show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
adversary-initial-state $\leq P + R$

by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
[OF pre-bound transcript-new-bound])

qed
qed
end

end

theory *Soundness-Staged-Aligned-Current-Query*
imports *Soundness-Staged-Aligned-Partial-Merkle*
begin

Thin wrappers that expose the actual-alpha current-query event in the aligned partial-candidate soundness path. The underlying query reduction still lives in the partial-Merkle layer; this theory only uses the projection proved in the current-query alignment layer.

context *soundness*
begin

lemma *checked-staged-soundness-from-aligned-transcript-components-and-actual-alpha-current-partial-opening-hit-fr*
fixes *trace-fri-error'* *composition-fri-error'* *query-error* $N :: \text{prob}$
assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$
and *comp-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$
and *current-query-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq \text{query-error}$
and *not-prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *empty-query-bound*:

$wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $query\text{-}bad\text{-}with\text{-}empty\text{-}composition\text{-}header\text{-}candidates)$
 $adversary\text{-}initial\text{-}state \leq nnreal \ rounds * query\text{-}error\text{-}bound$
shows $checked\text{-}staged\text{-}adversary\text{-}acceptance\text{-}probability \ A \leq$
 $hash\text{-}collision\text{-}budget\text{-}value \ 0$
 $(staged\text{-}attacker\text{-}query\text{-}budget \ budgets +$
 $staged\text{-}challenge\text{-}query\text{-}budget + verifier\text{-}hash\text{-}query\text{-}budget) +$
 $(composition\text{-}error\text{-}bound +$
 $staged\text{-}phase\text{-}relation\text{-}error \ size$
 $(staged\text{-}alpha\text{-}search\text{-}queries \ budgets \ 0) +$
 $hash\text{-}collision\text{-}budget\text{-}value \ 0$
 $(staged\text{-}alpha\text{-}search\text{-}queries \ budgets \ 0) +$
 $N +$
 $staged\text{-}phase\text{-}relation\text{-}error \ size$
 $(staged\text{-}alpha\text{-}search\text{-}queries \ budgets \ 0)) +$
 $trace\text{-}fri\text{-}error' + composition\text{-}fri\text{-}error' +$
 $query\text{-}error +$
 $(composition\text{-}error\text{-}bound +$
 $staged\text{-}phase\text{-}relation\text{-}error \ size$
 $(staged\text{-}alpha\text{-}search\text{-}queries \ budgets \ 0) +$
 $hash\text{-}collision\text{-}budget\text{-}value \ 0$
 $(staged\text{-}alpha\text{-}search\text{-}queries \ budgets \ 0) +$
 $N +$
 $staged\text{-}phase\text{-}relation\text{-}error \ size$
 $(staged\text{-}alpha\text{-}search\text{-}queries \ budgets \ 0) +$
 $trace\text{-}fri\text{-}error + nnreal \ rounds * query\text{-}error\text{-}bound)$
proof –
have $data\text{-}current\text{-}query\text{-}bound$:
 $wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}current\text{-}query\text{-}partial\text{-}opening\text{-}hit$
 $adversary\text{-}initial\text{-}state \leq query\text{-}error$
by (rule
 $checked\text{-}staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}current\text{-}query\text{-}partial\text{-}opening\text{-}hit\text{-}bound\text{-}from\text{-}actual\text{-}alpha\text{-}current\text{-}query$
 $[OF \ current\text{-}query\text{-}bound])$
show $?thesis$
by (rule
 $checked\text{-}staged\text{-}soundness\text{-}from\text{-}aligned\text{-}transcript\text{-}components\text{-}and\text{-}current\text{-}partial\text{-}opening\text{-}hit\text{-}from\text{-}not\text{-}prefix$
 $[OF \ false\text{-}statement \ wf \ controlled \ trace\text{-}fri\text{-}bound \ comp\text{-}fri\text{-}bound$
 $data\text{-}current\text{-}query\text{-}bound \ not\text{-}prefix\text{-}bound \ empty\text{-}trace\text{-}fri\text{-}bound$
 $empty\text{-}query\text{-}bound])$
qed

theorem $stark\text{-}soundness\text{-}from\text{-}aligned\text{-}partial\text{-}actual\text{-}alpha\text{-}current\text{-}query$:
fixes $trace\text{-}fri\text{-}error' \ composition\text{-}fri\text{-}error' \ query\text{-}error \ N :: prob$
assumes $false\text{-}statement: \neg \ exists\text{-}valid\text{-}trace$
and wf : $staged\text{-}budget\text{-}wellformed \ budgets$
and $controlled$: $staged\text{-}adversary\text{-}controlled \ budgets \ A$
and $trace\text{-}fri\text{-}bound$:

wp_event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 wp_event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*
and *current-query-bound*:
 wp_event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
adversary-initial-state $\leq$ *query-error*
and *not-prefix-bound*:
 wp_event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ N
and *empty-trace-fri-bound*:
 wp_event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
 wp_event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *nnreal rounds* * *query-error-bound*
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets* 0)) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets* 0) +
 N +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets* 0)) +
trace-fri-error' + *composition-fri-error'* +
query-error +
 (*composition-error-bound* +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets* 0)) +

hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
N +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nreal rounds * query-error-bound)*
by (rule
checked-staged-soundness-from-aligned-transcript-components-and-actual-alpha-current-partial-opening-hit-from-n
[OF false-statement wf controlled trace-fri-bound comp-fri-bound
current-query-bound not-prefix-bound empty-trace-fri-bound
empty-query-bound])

theorem *stark-soundness-from-aligned-partial-query-prefix-current-rounds:*
fixes *trace-fri-error' composition-fri-error' N :: prob*
and *C :: nat $\Rightarrow$ prob*
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ trace-fri-error'
and *comp-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ composition-fri-error'
and *round-bound:*
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq$ C i
and *not-prefix-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ N
and *empty-trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ trace-fri-error
and *empty-query-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event

query-bad-with-empty-composition-header-candidates
adversary-initial-state $\leq$ *nnreal rounds* * *query-error-bound*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
N +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
($\sum i < \text{rounds}. C i$) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
N +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nnreal rounds * query-error-bound)*
proof –
have *current-query-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
adversary-initial-state $\leq$ *($\sum i < \text{rounds}. C i$)*
by (rule
checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-bound-from-query-prefix-current
[OF wf controlled round-bound])
show *?thesis*
by (rule
stark-soundness-from-aligned-partial-actual-alpha-current-query
[OF false-statement wf controlled trace-fri-bound comp-fri-bound
current-query-bound not-prefix-bound empty-trace-fri-bound
empty-query-bound])
qed

theorem *stark-soundness-from-aligned-partial-query-prefix-current-prefix-path*:
fixes *trace-fri-error' composition-fri-error' N :: prob*
and *P Q :: nat $\Rightarrow$ prob*
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound*:

$wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 $wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*
and *prefix-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 $wp\text{-event}$
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at* i)
adversary-initial-state $\leq P$ i
and *path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 $wp\text{-event}$
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-current-path-output-hit-at* i)
adversary-initial-state $\leq Q$ i
and *not-prefix-bound*:
 $wp\text{-event}$
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq N$
and *empty-trace-fri-bound*:
 $wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
 $wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *nnreal rounds* * *query-error-bound*
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 N +
staged-phase-relation-error *size*

(staged-alpha-search-queries budgets 0)) +
 trace-fri-error' + composition-fri-error' +
 ($\sum i < \text{rounds}. P\ i + Q\ i$) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 N +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + nnreal rounds * query-error-bound)

proof (rule stark-soundness-from-aligned-partial-query-prefix-current-rounds
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound])
fix i
assume i-bound: i < rounds
show
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq P\ i + Q\ i$
by (rule
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-path
 [OF wf controlled i-bound prefix-bound[OF i-bound]
 path-bound[OF i-bound]])
qed (rule not-prefix-bound empty-trace-fri-bound empty-query-bound)+

theorem stark-soundness-from-aligned-partial-query-prefix-dynamic-target-path:
fixes trace-fri-error' composition-fri-error' N :: prob
and P Q :: nat $\Rightarrow$ prob
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ trace-fri-error'
and comp-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ composition-fri-error'
and prefix-target-bound:
 $\bigwedge i. i < \text{rounds} \Rightarrow$
 wp-event
 (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-hit
 staged-query-prefix-prefix-authenticated-opening-target)

$\text{adversary-initial-state} \leq P \ i$
and *path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq Q \ i$
and *not-prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}. P \ i + Q \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$
proof –
have *prefix-bound*:

```

 $\wedge i. i < \text{rounds} \implies$ 
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
  adversary-initial-state  $\leq P$  i
  by (rule
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target
    [OF wf controlled - prefix-target-bound])
  show ?thesis
  by (rule stark-soundness-from-aligned-partial-query-prefix-current-prefix-path
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      prefix-bound path-bound not-prefix-bound empty-trace-fri-bound
      empty-query-bound])
qed
end
end
end

```

theory Soundness-Conceptual-Query-Current-Path

imports

Soundness-Conceptual-Query-Current

Staged-Security-Experiment-Composition-Query-Current-Path

begin

Integration of conceptual prefix-query targets with the existing structured current-query path split.

context soundness

begin

definition checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at

where

checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at

$i \text{ out} \longleftrightarrow$

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at

$i \text{ out} \wedge$

checked-staged-security-with-query-prefix-conceptual-context-residual-at

out

definition checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at

where

checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at

trace-default composition-default $i \text{ out} \longleftrightarrow$

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at

$i \text{ out} \wedge$

checked-staged-security-with-query-prefix-conceptual-default-context-residual-at

trace-default composition-default out

definition *checked-staged-security-with-query-prefix-candidate-pair-default-agreement-gap*
where

checked-staged-security-with-query-prefix-candidate-pair-default-agreement-gap
 $\text{trace-table composition-table out} \longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 $\neg$ query-prefix-candidate-defaults-agree-with-authenticated-values
 trace-table composition-table prefix prefix-state)

definition *checked-staged-security-with-query-prefix-candidate-pair-sampled-default-agreement-gap*
where

checked-staged-security-with-query-prefix-candidate-pair-sampled-default-agreement-gap
 $\text{trace-table composition-table out} \longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 $\neg$ query-prefix-candidate-defaults-agree-at-query trace-table
 composition-table (index (to-nat raw)) prefix prefix-state)

definition *checked-staged-security-with-query-prefix-authenticated-candidate-hit-at*
where

checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
 $\text{trace-table composition-table } i \text{ out} \longleftrightarrow$
 ($\exists$ trace-openings composition-openings.
 checked-staged-security-with-query-prefix-authenticated-opening-hit
 trace-openings composition-openings i out $\wedge$
 partial-trace-table-candidate trace-table
 ((replicate rounds []) [i := trace-openings]) $\wedge$
 partial-composition-table-candidate composition-table
 ((replicate rounds []) [i := composition-openings]))

lemma *checked-staged-security-with-query-prefix-candidate-pair-hit-imp-conceptual-default-or-agreement-gap:*
assumes hit:

 checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
 trace-table composition-table
 (Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state))

and clean: $\neg$ hash-map-output-collision prefix-state

and trace-len: length trace-table = scale * clength

and composition-len: length composition-table = scale * clength

shows

checked-staged-security-with-query-prefix-conceptual-default-target-hit
 trace-table composition-table
 (Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state)) $\vee$
 checked-staged-security-with-query-prefix-candidate-pair-default-agreement-gap

```

    trace-table composition-table
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
proof (cases
  query-prefix-candidate-defaults-agree-with-authenticated-values
    trace-table composition-table prefix prefix-state)
case True
have subset:
  staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
    composition-table prefix prefix-state  $\subseteq$ 
  staged-query-prefix-conceptual-default-query-target trace-table
    composition-table prefix prefix-state
by (rule
  staged-query-prefix-candidate-pair-query-target-from-prefix-subset-conceptual-default-if-prefix-agrees
    [OF clean trace-len composition-len True])
have
  checked-staged-security-with-query-prefix-conceptual-default-target-hit
    trace-table composition-table
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
using hit subset
unfolding
  checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-def
  checked-staged-security-with-query-prefix-conceptual-default-target-hit-def
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
  checked-staged-query-prefix-dynamic-index-hit-def
by auto
then show ?thesis by simp
next
case False
then have
  checked-staged-security-with-query-prefix-candidate-pair-default-agreement-gap
    trace-table composition-table
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-candidate-pair-default-agreement-gap-def
by simp
then show ?thesis by simp
qed

lemma checked-staged-security-with-query-prefix-candidate-pair-hit-imp-conceptual-default-or-sampled-agreement-gap
assumes hit:
  checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
    trace-table composition-table
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
and trace-len: length trace-table = scale * clength
and composition-len: length composition-table = scale * clength

```

```

and default-trace-low:
  trace-table-low-degree
    (query-prefix-trace-conceptual-table-with-default prefix prefix-state
      trace-table)
and default-composition-low:
  composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-table)
and default-not-all:
   $\neg$  all-queries-consistent
    (query-prefix-trace-conceptual-table-with-default prefix prefix-state
      trace-table)
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-table)
    (sqp-alphas prefix)
shows
  checked-staged-security-with-query-prefix-conceptual-default-target-hit
    trace-table composition-table
    (Some ((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))  $\vee$ 
  checked-staged-security-with-query-prefix-candidate-pair-sampled-default-agreement-gap
    trace-table composition-table
    (Some ((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
proof (cases
  query-prefix-candidate-defaults-agree-at-query trace-table
    composition-table (index (to-nat raw)) prefix prefix-state)
case True
have raw-hit:
  index (to-nat raw)  $\in$ 
    staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
      composition-table prefix prefix-state
using hit
unfolding
  checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-def
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
  checked-staged-query-prefix-dynamic-index-hit-def
by simp
have default-raw-hit:
  index (to-nat raw)  $\in$ 
    staged-query-prefix-conceptual-default-query-target trace-table
      composition-table prefix prefix-state
by (rule
  query-prefix-candidate-pair-query-hit-imp-conceptual-default-if-agrees-at-query
  [OF raw-hit trace-len composition-len True default-trace-low
    default-composition-low default-not-all])
have
  checked-staged-security-with-query-prefix-conceptual-default-target-hit
    trace-table composition-table

```



```

    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-conceptual-default-target-hit-def
    checked-staged-security-with-query-prefix-dynamic-index-hit-def
    checked-staged-query-prefix-dynamic-index-hit-def
  using default-raw-hit by simp
  then show ?thesis by simp
next
case False
then have
  checked-staged-security-with-query-prefix-candidate-pair-sampled-default-agreement-gap
  trace-table composition-table
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-candidate-pair-sampled-default-agreement-gap-def
  by simp
  then show ?thesis by simp
qed

lemma checked-staged-security-with-query-prefix-candidate-pair-hit-imp-conceptual-default-if-final-merkle-bound:
  assumes hit:
    checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
    trace-table composition-table
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  and prefix-clean:  $\neg$  hash-map-output-collision prefix-state
  and final-clean:  $\neg$  hash-map-output-collision final-state
  and ext:  $\text{prefix-state} \leq \text{final-state}$ 
  and trace-bind:
    merkle-root-binds-table (sqp-trace-root prefix) trace-table
    final-state
  and composition-bind:
    merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
    composition-table final-state
  and trace-len:  $\text{length trace-table} = \text{scale} * \text{clength}$ 
  and composition-len:  $\text{length composition-table} = \text{scale} * \text{clength}$ 
  shows
    checked-staged-security-with-query-prefix-conceptual-default-target-hit
    trace-table composition-table
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
proof -
  have subset:
    staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
    composition-table prefix prefix-state  $\subseteq$ 
    staged-query-prefix-conceptual-default-query-target trace-table
    composition-table prefix prefix-state

```

```

by (rule
  staged-query-prefix-candidate-pair-query-target-from-prefix-subset-conceptual-default-if-final-merkle-bound
  [OF prefix-clean final-clean ext trace-bind composition-bind trace-len
    composition-len])
show ?thesis
using hit subset
unfolding
  checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-def
  checked-staged-security-with-query-prefix-conceptual-default-target-hit-def
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
  checked-staged-query-prefix-dynamic-index-hit-def
by auto
qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-conceptual-default-or-residual-or-stru
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-authenticated-opening-hit
  trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
and trace-candidate:
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
shows
  checked-staged-security-with-query-prefix-conceptual-default-target-hit
  trace-table composition-table
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)) ∨
  checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
  trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)) ∨
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
  i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)) ∨

```

```

checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof -
  let ?out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
  have current-hit:
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
    i ?out
  by (rule
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-atI
    [OF hit])
  show ?thesis
  proof (cases
    ¬ hash-map-output-collision prefix-state ∧
    trace-table-low-degree
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-table) ∧
    composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-table) ∧
    ¬ all-queries-consistent
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-table)
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-table)
    (sqp-alphas prefix))
  case False
  then have
    checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
    trace-table composition-table i ?out
  unfolding
    checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-def
    checked-staged-security-with-query-prefix-conceptual-default-context-residual-at-def
  using current-hit by simp
  then show ?thesis by simp
next
  case True
  then have clean:
    ¬ hash-map-output-collision prefix-state
  and default-trace-low:
    trace-table-low-degree
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-table)
  and default-composition-low:
    composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table-with-default prefix

```

```

    prefix-state composition-table)
and default-not-all:
   $\neg$  all-queries-consistent
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-table)
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-table)
    (sqp-alphas prefix)
by blast+
have ext: prefix-state  $\leq$  final-state
by (rule
  checked-staged-security-experiment-with-query-prefix-data-state-prefix-extends-final
  [OF wf controlled i-bound support])
have alphas-eq: staged-alphas data = sqp-alphas prefix
by (rule checked-staged-security-with-query-prefix-data-state-alphas-eq
  [OF support])
have trace-root-eq: staged-trace-root data = sqp-trace-root prefix
using support
unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
  checked-staged-after-query-prefix-receive-with-verifier-def
  checked-staged-after-query-prefix-receive-def
by (auto elim!: set-dist-bindE split: prod.splits)
have composition-roots-eq:
  staged-composition-fri-roots data = sqp-composition-fri-roots prefix
using support
unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
  checked-staged-after-query-prefix-receive-with-verifier-def
  checked-staged-after-query-prefix-receive-def
by (auto elim!: set-dist-bindE split: prod.splits)
have comp-nonempty: staged-composition-fri-roots data  $\neq$  []
and trace-table-final:
  partial-authenticated-table (staged-trace-root data)
  (scale * clength) trace-openings final-state
and comp-table-final:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
  (scale * clength) composition-openings final-state
and consistent:
  partial-query-openings-consistent trace-openings composition-openings
  (staged-alphas data) (index (to-nat raw))
using hit
unfolding checked-staged-security-with-query-prefix-authenticated-opening-hit-def
by simp-all
have trace-pullback:
  partial-authenticated-table (staged-trace-root data)
  (scale * clength) trace-openings prefix-state  $\vee$ 
  ( $\exists$  opn  $\in$  set trace-openings.
  hash-map-new-output-hit
  (merkle-path-target-roots final-state (staged-trace-root data)
  (scale * clength) (opening-index opn) (opening-value opn))

```

```

      (opening-path opn)) prefix-state final-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit
      [OF ext trace-table-final])
then show ?thesis
proof
  assume trace-prefix-staged:
    partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings prefix-state
  have comp-pullback:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings prefix-state ∨
    (∃ opn ∈ set composition-openings.
      hash-map-new-output-hit
      (merkle-path-target-roots final-state
        (hd (staged-composition-fri-roots data))
        (scale * clength) (opening-index opn) (opening-value opn)
        (opening-path opn)) prefix-state final-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit
      [OF ext comp-table-final])
then show ?thesis
proof
  assume comp-prefix-staged:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings prefix-state
  have trace-prefix:
    partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) trace-openings prefix-state
  using trace-prefix-staged trace-root-eq by simp
  have composition-prefix:
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
    (scale * clength) composition-openings prefix-state
  using comp-prefix-staged composition-roots-eq by simp
  have consistent-prefix:
    partial-query-openings-consistent trace-openings
    composition-openings (sqp-alphas prefix) (index (to-nat raw))
  using consistent alphas-eq by simp
  have target:
    index (to-nat raw) ∈
    staged-query-prefix-conceptual-default-query-target trace-table
    composition-table prefix prefix-state
  by (rule
    query-prefix-current-openings-imp-conceptual-default-query-target
    [OF clean i-bound trace-prefix composition-prefix
      consistent-prefix trace-candidate comp-candidate
      default-trace-low default-composition-low default-not-all])
  have
    checked-staged-security-with-query-prefix-conceptual-default-target-hit
    trace-table composition-table ?out
  unfolding

```

```

checked-staged-security-with-query-prefix-conceptual-default-target-hit-def
checked-staged-security-with-query-prefix-dynamic-index-hit-def
checked-staged-query-prefix-dynamic-index-hit-def
using target by simp
then show ?thesis by simp
next
assume comp-path:
   $\exists \text{opn} \in \text{set composition-openings.}$ 
  hash-map-new-output-hit
  (merkle-path-target-roots final-state
    (hd (staged-composition-fri-roots data))
    (scale * clength) (opening-index opn) (opening-value opn)
    (opening-path opn)) prefix-state final-state
then obtain opn where opn-in:  $\text{opn} \in \text{set composition-openings}$ 
and path-hit:
  hash-map-new-output-hit
  (merkle-path-target-roots final-state
    (hd (staged-composition-fri-roots data))
    (scale * clength) (opening-index opn) (opening-value opn)
    (opening-path opn)) prefix-state final-state
by blast
have auth: authenticated-opening-in final-state opn
using comp-table-final opn-in
unfolding partial-authenticated-table-def by blast
have root-eq:
  opening-root opn = hd (staged-composition-fri-roots data)
using comp-table-final opn-in
unfolding partial-authenticated-table-def by blast
have len-eq: opening-length opn = scale * clength
using comp-table-final opn-in
unfolding partial-authenticated-table-def by blast
have
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
  i ?out
unfolding
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at-def
using comp-nonempty auth root-eq len-eq path-hit by auto
then show ?thesis by simp
qed
next
assume trace-path:
   $\exists \text{opn} \in \text{set trace-openings.}$ 
  hash-map-new-output-hit
  (merkle-path-target-roots final-state (staged-trace-root data)
    (scale * clength) (opening-index opn) (opening-value opn)
    (opening-path opn)) prefix-state final-state
then obtain opn where opn-in:  $\text{opn} \in \text{set trace-openings}$ 
and path-hit:
  hash-map-new-output-hit

```

```

      (merkle-path-target-roots final-state (staged-trace-root data)
       (scale * clength) (opening-index opn) (opening-value opn)
       (opening-path opn)) prefix-state final-state
    by blast
  have auth: authenticated-opening-in final-state opn
    using trace-table-final opn-in
    unfolding partial-authenticated-table-def by blast
  have root-eq: opening-root opn = staged-trace-root data
    using trace-table-final opn-in
    unfolding partial-authenticated-table-def by blast
  have len-eq: opening-length opn = scale * clength
    using trace-table-final opn-in
    unfolding partial-authenticated-table-def by blast
  have
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
      i ?out
    unfolding
      checked-staged-security-with-query-prefix-current-trace-path-output-hit-at-def
    using auth root-eq len-eq path-hit by auto
  then show ?thesis by simp
qed
qed
qed

lemma checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-imp-conceptual-default-or-residual-or
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and support:
    out ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-query-prefix-data-state A i)
          adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
      trace-table composition-table i out
  shows
    checked-staged-security-with-query-prefix-conceptual-default-target-hit
      trace-table composition-table out ∨
    checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
      trace-table composition-table i out ∨
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
      i out ∨
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
      i out
proof (cases out)
case None
then show ?thesis

```

```

using hit
unfolding
  checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-def
  checked-staged-security-with-query-prefix-authenticated-opening-hit-def
by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
  final-state where out-eq:
  out =
    Some ((((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
from hit[unfolded out-eq
  checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-def]
obtain trace-openings composition-openings where opening-hit:
  checked-staged-security-with-query-prefix-authenticated-opening-hit
  trace-openings composition-openings i
  (Some ((((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
and trace-candidate:
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
by blast
have support':
  Some ((((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)
using support unfolding out-eq .
show ?thesis
unfolding out-eq
by (rule
  checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-conceptual-default-or-residual-or-structu
  [OF wf controlled i-bound support' opening-hit trace-candidate
    comp-candidate])
qed

lemma checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-imp-current:
assumes
  checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
  i out
shows
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at

```


i out
using *assms*
unfolding
checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-def
by *simp*

lemma *checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-imp-residual:*

assumes
checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
i out
shows
checked-staged-security-with-query-prefix-conceptual-context-residual-at
out
using *assms*
unfolding
checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-def
by *simp*

definition *checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at*
where

checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at
i out $\longleftrightarrow$
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
i out $\wedge$
(case out of
None $\Rightarrow$ *False*
| *Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*) $\Rightarrow$
hash-map-output-collision prefix-state)

definition *checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at*
where

checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
i out $\longleftrightarrow$
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
i out $\wedge$
(case out of
None $\Rightarrow$ *False*
| *Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*) $\Rightarrow$
 $\neg$ *trace-table-low-degree*
(*query-prefix-trace-conceptual-table prefix prefix-state*) $\vee$
 $\neg$ *composition-table-low-degree maxDegree*
(*query-prefix-composition-conceptual-table prefix prefix-state*) $\vee$
all-queries-consistent
(*query-prefix-trace-conceptual-table prefix prefix-state*)
(*query-prefix-composition-conceptual-table prefix prefix-state*)
(*sqp-alphas prefix*))

definition *checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at*
where

checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at
 $i \text{ out} \longleftrightarrow$
checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
 $i \text{ out} \wedge$
(case out of
 $\text{None} \Rightarrow \text{False}$
| $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}) \Rightarrow$
 $\neg \text{trace-table-low-degree}$
 $(\text{query-prefix-trace-conceptual-table } \text{prefix } \text{prefix-state}))$

definition *checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at*
where

checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at
 $i \text{ out} \longleftrightarrow$
checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
 $i \text{ out} \wedge$
(case out of
 $\text{None} \Rightarrow \text{False}$
| $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}) \Rightarrow$
 $\neg \text{composition-table-low-degree } \text{maxDegree}$
 $(\text{query-prefix-composition-conceptual-table } \text{prefix } \text{prefix-state}))$

definition *checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at*
where

checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at
 $i \text{ out} \longleftrightarrow$
checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
 $i \text{ out} \wedge$
(case out of
 $\text{None} \Rightarrow \text{False}$
| $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}) \Rightarrow$
 $\text{sqp-alphas } \text{prefix} \in$
 $\text{composition-trace-bad-alpha-space}$
 $(\text{query-prefix-trace-conceptual-table } \text{prefix } \text{prefix-state}))$

lemma *checked-staged-query-prefix-with-state-alias-length:*

assumes *support:*

$\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state})$

shows $\text{length } (\text{sqp-alphas } \text{prefix}) = \text{length } \text{spec}$

proof —

from *support* **have** *prefix-out:*

```

Some (prefix, prefix-state) ∈
  set-dist
    (execute (checked-staged-query-challenge-prefix-program A i)
      adversary-initial-state)
unfolding checked-staged-query-prefix-with-state-def
by (auto elim!: set-dist-bindE)
from prefix-out obtain fr s1 s2 trace-roots trace-bs s3 trace-final s4
  s5 as s6 dg s7 s8 s9 composition-roots composition-bs s10
  composition-final s11 query-start query-chunks where
alpha-out:
  Some (as, s6) ∈
    set-dist (execute (staged-alpha-program (length spec)) s5)
and prefix-eq:
  prefix =
    (|spp-trace-root = fr,
     spp-trace-fri-roots = trace-roots,
     spp-trace-fri-challenges = trace-bs,
     spp-trace-final = trace-final,
     spp-alphas = as,
     spp-degree = dg,
     spp-composition-fri-roots = composition-roots,
     spp-composition-fri-challenges = composition-bs,
     spp-composition-final = composition-final,
     spp-query-chunks = query-chunks|)
using prefix-out
by (elim checked-staged-query-challenge-prefix-program-outcomeE)
have length as = length spec
using staged-alpha-program-alignment[OF alpha-out] by simp
then show ?thesis
unfolding prefix-eq by simp
qed

lemma checked-staged-security-with-query-prefix-data-state-prefix-support:
assumes support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
shows
  Some ((prefix, prefix-state), prefix-state) ∈
  set-dist
    (execute (checked-staged-query-prefix-with-state A i)
      adversary-initial-state)
proof –
from support have receive-support:
  Some (((prefix, prefix-state), raw), raw-state) ∈
  set-dist

```

```

      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
    unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
    by (auto elim!: set-dist-bindE split: prod.splits)
  then show ?thesis
    by (rule checked-staged-query-prefix-receive-with-state-prefix-support)
qed

lemma checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-cases-on-support:
  assumes false-statement:  $\neg$  exists-valid-trace
  and support:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)  $\in$ 
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
    i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  shows
     $\neg$  trace-table-low-degree
    (query-prefix-trace-conceptual-table prefix prefix-state)  $\vee$ 
     $\neg$  composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table prefix prefix-state)  $\vee$ 
    sqp-alphas prefix  $\in$ 
    composition-trace-bad-alpha-space
    (query-prefix-trace-conceptual-table prefix prefix-state)
  proof -
    let ?trace = query-prefix-trace-conceptual-table prefix prefix-state
    let ?composition =
      query-prefix-composition-conceptual-table prefix prefix-state
    from hit have residual:
       $\neg$  trace-table-low-degree ?trace  $\vee$ 
       $\neg$  composition-table-low-degree maxDegree ?composition  $\vee$ 
      all-queries-consistent ?trace ?composition (sqp-alphas prefix)
    unfolding
      checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-def
    by simp
  show ?thesis
  proof (cases trace-table-low-degree ?trace)
    case False
    then show ?thesis by simp
  next
    case trace-low: True
    show ?thesis
    proof (cases composition-table-low-degree maxDegree ?composition)

```

```

case False
then show ?thesis by simp
next
case composition-low: True
from residual trace-low composition-low have all-queries:
  all-queries-consistent ?trace ?composition (sqp-alphas prefix)
  by simp
have prefix-support:
  Some ((prefix, prefix-state), prefix-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
      adversary-initial-state)
  by (rule
    checked-staged-security-with-query-prefix-data-state-prefix-support
    [OF support])
have alpha-len: length (sqp-alphas prefix) = length spec
  by (rule checked-staged-query-prefix-with-state-alphas-length
    [OF prefix-support])
have
  sqp-alphas prefix ∈
    composition-trace-bad-alpha-space ?trace
  by (rule low-degree-all-queries-consistent-imp-composition-trace-bad-alpha-space
    [OF false-statement alpha-len trace-low composition-low
      all-queries])
  then show ?thesis by simp
qed
qed
qed

lemma checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-split-on-support:
assumes false-statement: ¬ exists-valid-trace
and support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
    i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
shows
  checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at
    i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)) ∨
    checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at

```

```

i
(Some (((((prefix, prefix-state), raw), raw-state),
  ((data, attacker-state), result)), final-state)) ∨
checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at
i
(Some (((((prefix, prefix-state), raw), raw-state),
  ((data, attacker-state), result)), final-state))
using
checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-cases-on-support
[OF false-statement support hit] hit
unfolding
checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at-def
checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at-def
checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at-def
by simp

lemma checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-bound-from-split:
assumes false-statement:  $\neg$  exists-valid-trace
and trace-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at
    i)
  adversary-initial-state  $\leq T$ 
and composition-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at
    i)
  adversary-initial-state  $\leq C$ 
and alpha-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at
    i)
  adversary-initial-state  $\leq P$ 
shows
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
    i)
  adversary-initial-state  $\leq T + C + P$ 
proof –
let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i
let ?Trace =
  checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at
    i
let ?Composition =
  checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at

```

```

i
let ?Alpha =
  checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at
  i
have event-le:
  wp-event ?M
    (checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
     i)
  adversary-initial-state ≤
  wp-event ?M (λout. ?Trace out ∨ ?Composition out ∨ ?Alpha out)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out ∈ set-dist (execute ?M adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
  i out
show ?Trace out ∨ ?Composition out ∨ ?Alpha out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
  final-state where out-eq:
  out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
have support':
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist (execute ?M adversary-initial-state)
  using support unfolding out-eq .
have hit':
  checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
  i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
  using hit unfolding out-eq .
show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-split-on-support

```

```

      [OF false-statement support' hit'])
qed
qed
also have ... ≤
  wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M ?Composition adversary-initial-state +
  wp-event ?M ?Alpha adversary-initial-state
proof -
  have ... ≤
    wp-event ?M ?Trace
    adversary-initial-state +
    wp-event ?M (λout. ?Composition out ∨ ?Alpha out)
    adversary-initial-state
  by (rule wp-event-union-bound
    [where P = ?Trace
      and Q = λout. ?Composition out ∨ ?Alpha out])
  also have ... ≤
    wp-event ?M ?Trace adversary-initial-state +
    (wp-event ?M ?Composition adversary-initial-state +
     wp-event ?M ?Alpha adversary-initial-state)
  by (intro add-mono order-refl wp-event-union-bound)
  finally show ?thesis
    by (simp add: add.assoc)
qed
also have ... ≤ T + C + P
  by (intro add-mono trace-bound composition-bound alpha-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-split:
  assumes
    checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
    i out
  shows
    checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at
    i out ∨
    checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
    i out
  using assms
  unfolding
    checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-def
    checked-staged-security-with-query-prefix-conceptual-context-residual-at-def
    checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at-def
    checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-def
  by (cases out) (auto split: prod.splits)

lemma checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A

```


and *i-bound*: $i < \text{rounds}$
shows
wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* $A\ i$)
 (*checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at*
 i)
adversary-initial-state $\leq$
hash-collision-budget-value 0
 (*staged-query-search-queries* *budgets* $i + 1$)
proof –
have *i-le*: $i \leq \text{rounds}$
using *i-bound* **by** *simp*
have *prefix-bound*:
wp-event (*checked-staged-query-prefix-receive-with-state* $A\ i$)
checked-staged-query-prefix-hash-map-output-collision
adversary-initial-state $\leq$
hash-collision-budget-value 0
 (*staged-query-search-queries* *budgets* $i + 1$)
by (*rule checked-staged-query-prefix-hash-map-output-collision-bound*
 [*OF wf controlled i-le*])
show ?thesis
proof (*rule*
 checked-staged-security-experiment-with-query-prefix-data-state-bound-by-prefix
 [*OF prefix-bound*])
show
checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at
 i None $\implies$
checked-staged-query-prefix-hash-map-output-collision *None*
unfolding
checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at-def
checked-staged-query-prefix-hash-map-output-collision-def
by *simp*
next
fix *prefix-out t out*
assume
Some (*prefix-out*, *t*) $\in$
 set-dist
 (*execute* (*checked-staged-query-prefix-receive-with-state* $A\ i$)
 adversary-initial-state)
and *cont*:
out $\in$
 set-dist
 (*execute*
 (*get* $\gg$
 ($\lambda \text{prefix-state.}$
 checked-staged-after-query-prefix-receive-with-verifier $A\ i$
 prefix-out $\gg$
 ($\lambda \text{result. return } ((\text{prefix-out}, \text{prefix-state}), \text{result})$)))
 t)

```

and event:
  checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at
    i out
show
  checked-staged-query-prefix-hash-map-output-collision
    (Some (prefix-out, t))
using cont event
unfolding
  checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at-def
  checked-staged-query-prefix-hash-map-output-collision-def
by (auto elim!: set-dist-bindE split: option.splits prod.splits)
qed
qed

lemma checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-imp-prefix-collision-on-sup
assumes default-context:
   $\bigwedge \text{prefix prefix-state.}$ 
  Some ((prefix, prefix-state), prefix-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
      adversary-initial-state)  $\implies$ 
    trace-table-low-degree
      (query-prefix-trace-conceptual-table-with-default prefix
        prefix-state trace-default)  $\wedge$ 
    composition-table-low-degree maxDegree
      (query-prefix-composition-conceptual-table-with-default prefix
        prefix-state composition-default)  $\wedge$ 
     $\neg$  all-queries-consistent
      (query-prefix-trace-conceptual-table-with-default prefix
        prefix-state trace-default)
      (query-prefix-composition-conceptual-table-with-default prefix
        prefix-state composition-default)
      (sqp-alphas prefix)
and support:
  out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
    trace-default composition-default i out
shows
  checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at
    i out
proof (cases out)
case None
then show ?thesis
using hit

```

```

unfolding
  checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-def
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-def
  checked-staged-security-with-query-prefix-authenticated-opening-hit-def
by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
  final-state where out-eq:
  out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
by (cases packed) (auto split: prod.splits)
have support':
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)
using support unfolding out-eq .
have prefix-support:
  Some ((prefix, prefix-state), prefix-state) ∈
  set-dist
  (execute (checked-staged-query-prefix-with-state A i)
    adversary-initial-state)
by (rule
  checked-staged-security-with-query-prefix-data-state-prefix-support
  [OF support'])
have ctxt-fact:
  trace-table-low-degree
  (query-prefix-trace-conceptual-table-with-default prefix prefix-state
    trace-default) ∧
  composition-table-low-degree maxDegree
  (query-prefix-composition-conceptual-table-with-default prefix
    prefix-state composition-default) ∧
  ¬ all-queries-consistent
  (query-prefix-trace-conceptual-table-with-default prefix prefix-state
    trace-default)
  (query-prefix-composition-conceptual-table-with-default prefix
    prefix-state composition-default)
  (sqp-alphas prefix)
by (rule default-context[OF prefix-support])
from hit ctxt-fact have collision: hash-map-output-collision prefix-state
unfolding out-eq
  checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-def
  checked-staged-security-with-query-prefix-conceptual-default-context-residual-at-def
by auto
have current-hit:

```

```

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
  i out
using hit
unfolding
  checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-def
by simp
show ?thesis
unfolding
  checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at-def
using current-hit collision out-eq by simp
qed

lemma checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-context:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i < \text{rounds}$ 
and default-context:
   $\bigwedge \text{prefix prefix-state.}$ 
  Some ((prefix, prefix-state), prefix-state)  $\in$ 
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
      adversary-initial-state)  $\implies$ 
    trace-table-low-degree
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-default)  $\wedge$ 
    composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-default)  $\wedge$ 
     $\neg$  all-queries-consistent
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-default)
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-default)
    (sqp-alphas prefix)
shows
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
    trace-default composition-default i)
  adversary-initial-state  $\leq$ 
  hash-collision-budget-value 0
  (staged-query-search-queries budgets i + 1)
proof -
let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i
have event-le:
  wp-event ?M
  (checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
    trace-default composition-default i)
  adversary-initial-state  $\leq$ 

```

```

wp-event ?M
(checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at
 i)
adversary-initial-state
by (rule wp-event-mono-on-support)
(rule
checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-imp-prefix-collision-on-support
[OF default-context])
also have ... ≤
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1)
by (rule
checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at-bound
[OF wf controlled i-bound])
finally show ?thesis .
qed

```

lemma *checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-bound-from-split:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *algebraic-bound:*

```

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
 i)
adversary-initial-state ≤ R

```

shows

```

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
 i)
adversary-initial-state ≤
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1) + R

```

proof —

let *?M = checked-staged-security-experiment-with-query-prefix-data-state A i*

let *?Collision =*

```

checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at
 i

```

let *?Algebraic =*

```

checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
 i

```

have *event-le:*

```

wp-event ?M
(checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at i)
adversary-initial-state ≤
wp-event ?M (λout. ?Collision out ∨ ?Algebraic out)
adversary-initial-state

```

```

by (rule wp-event-mono)
  (rule
    checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-split)
have union-bound:
  wp-event ?M ( $\lambda out. ?Collision\ out \vee ?Algebraic\ out$ )
    adversary-initial-state  $\leq$ 
    wp-event ?M ?Collision adversary-initial-state +
    wp-event ?M ?Algebraic adversary-initial-state
  by (rule wp-event-union-bound)
have collision-bound:
  wp-event ?M ?Collision adversary-initial-state  $\leq$ 
    hash-collision-budget-value 0
    (staged-query-search-queries budgets  $i + 1$ )
  by (rule
    checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at-bound
    [OF wf controlled i-bound])
have sum-bound:
  wp-event ?M ?Collision adversary-initial-state +
  wp-event ?M ?Algebraic adversary-initial-state  $\leq$ 
    hash-collision-budget-value 0
    (staged-query-search-queries budgets  $i + 1$ ) + R
  by (intro add-mono collision-bound algebraic-bound)
show ?thesis
  by (rule order-trans[OF event-le order-trans[OF union-bound sum-bound]])
qed

```

lemma *checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-bound-from-current:*

assumes *current-bound:*

```

  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
      i)
  adversary-initial-state  $\leq C$ 

```

shows

```

  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
      i)
  adversary-initial-state  $\leq C$ 

```

by (rule order-trans[OF - current-bound])

```

  (rule wp-event-mono,
  rule checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-imp-current)

```

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-conceptual-or-residual-or-s*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: $i < rounds$*

and *support:*

out $\in$

```

    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
    i out
  shows
    checked-staged-security-with-query-prefix-conceptual-target-hit out ∨
    checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
    i out ∨
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i out ∨
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i out
  proof —
  have split:
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
    i out ∨
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i out ∨
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i out
  by (rule
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-prefix-or-structured-path-on-su
    [OF wf controlled i-bound support hit])
  then show ?thesis
  proof
  assume prefix:
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
    i out
  have
    checked-staged-security-with-query-prefix-conceptual-target-hit out ∨
    checked-staged-security-with-query-prefix-conceptual-context-residual-at
    out
  by (rule
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-imp-conceptual-or-residual-on-support
    [OF wf controlled i-bound support prefix])
  then show ?thesis
  unfolding
    checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-def
  using hit by blast
  next
  assume
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i out ∨
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i out
  then show ?thesis by blast

```

qed
qed

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-conceptual-default-or-residual*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *support:*

out ∈

set-dist

(*execute*

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

adversary-initial-state)

and *hit:*

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at

i out

shows

checked-staged-security-with-query-prefix-conceptual-default-target-hit

trace-default composition-default out ∨

checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at

trace-default composition-default i out ∨

checked-staged-security-with-query-prefix-current-trace-path-output-hit-at

i out ∨

checked-staged-security-with-query-prefix-current-composition-path-output-hit-at

i out

proof —

have *split:*

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at

i out ∨

checked-staged-security-with-query-prefix-current-trace-path-output-hit-at

i out ∨

checked-staged-security-with-query-prefix-current-composition-path-output-hit-at

i out

by (rule

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-prefix-or-structured-path-on-su

[OF *wf controlled i-bound support hit*])

then show *?thesis*

proof

assume *prefix:*

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at

i out

have

checked-staged-security-with-query-prefix-conceptual-default-target-hit

trace-default composition-default out ∨

checked-staged-security-with-query-prefix-conceptual-default-context-residual-at

trace-default composition-default out

by (rule

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-imp-conceptual-default-or-residual-on-

[OF *wf controlled i-bound support prefix*])


```

then show ?thesis
  unfolding
    checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-def
  using hit by blast
next
assume
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
  i out  $\vee$ 
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
  i out
  then show ?thesis by blast
qed
qed

```

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-residual*

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and i-bound: $i < \text{rounds}$

and residual-bound:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at i)

adversary-initial-state $\leq R$

and trace-path-bound:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)

adversary-initial-state $\leq T$

and composition-path-bound:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)

adversary-initial-state $\leq C$

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)

adversary-initial-state $\leq$

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) +

query-error-bound + R + T + C

proof –

let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i

let ?Conceptual =

checked-staged-security-with-query-prefix-conceptual-target-hit

let ?Residual =

checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at

i

```

let ?Trace =
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i
let ?Composition =
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i
have event-le:
  wp-event ?M
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state ≤
  wp-event ?M
    (λout. ?Conceptual out ∨ ?Residual out ∨ ?Trace out ∨
      ?Composition out)
    adversary-initial-state
by (rule wp-event-mono-on-support)
    (rule
      checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-conceptual-or-residual-or-structure
      [OF wf controlled i-bound])
have conceptual-bound:
  wp-event ?M ?Conceptual adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + query-error-bound
by (rule checked-staged-security-with-query-prefix-conceptual-target-hit-bound
    [OF wf controlled i-bound])
have union-bound:
  wp-event ?M
    (λout. ?Conceptual out ∨ ?Residual out ∨ ?Trace out ∨
      ?Composition out)
    adversary-initial-state ≤
  wp-event ?M ?Conceptual adversary-initial-state +
  wp-event ?M ?Residual adversary-initial-state +
  wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M ?Composition adversary-initial-state
by (rule wp-event-union-bound4)
have sum-bound:
  wp-event ?M ?Conceptual adversary-initial-state +
  wp-event ?M ?Residual adversary-initial-state +
  wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M ?Composition adversary-initial-state ≤
    (staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) + query-error-bound) +
    R + T + C
by (intro add-mono conceptual-bound residual-bound trace-path-bound
    composition-path-bound)
have event-bound:
  wp-event ?M
    (λout. ?Conceptual out ∨ ?Residual out ∨ ?Trace out ∨
      ?Composition out)
    adversary-initial-state ≤
    (staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) + query-error-bound) +

```

```

     $R + T + C$ 
  by (rule order-trans[OF union-bound sum-bound])
show ?thesis
  by (rule order-trans[OF event-le])
    (use event-bound in ⟨simp add: algebra-simps⟩)
qed

lemma checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-
assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and residual-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
        trace-default composition-default i)
      adversary-initial-state  $\leq R$ 
  and trace-path-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
      adversary-initial-state  $\leq T$ 
  and composition-path-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
      adversary-initial-state  $\leq C$ 
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
        (staged-query-search-queries budgets  $i + 1$ ) +
        query-error-bound +  $R + T + C$ 
proof -
  let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i
  let ?Conceptual =
    checked-staged-security-with-query-prefix-conceptual-default-target-hit
      trace-default composition-default
  let ?Residual =
    checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
      trace-default composition-default i
  let ?Trace =
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i
  let ?Composition =
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i
  have event-le:
    wp-event ?M

```

```

    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state ≤
    wp-event ?M
    (λout. ?Conceptual out ∨ ?Residual out ∨ ?Trace out ∨
     ?Composition out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (rule
   checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-conceptual-default-or-residual-
    [OF wf controlled i-bound])
  have conceptual-bound:
    wp-event ?M ?Conceptual adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + query-error-bound
  by (rule
    checked-staged-security-with-query-prefix-conceptual-default-target-hit-bound
    [OF wf controlled i-bound])
  have union-bound:
    wp-event ?M
    (λout. ?Conceptual out ∨ ?Residual out ∨ ?Trace out ∨
     ?Composition out)
    adversary-initial-state ≤
    wp-event ?M ?Conceptual adversary-initial-state +
    wp-event ?M ?Residual adversary-initial-state +
    wp-event ?M ?Trace adversary-initial-state +
    wp-event ?M ?Composition adversary-initial-state
  by (rule wp-event-union-bound4)
  have sum-bound:
    wp-event ?M ?Conceptual adversary-initial-state +
    wp-event ?M ?Residual adversary-initial-state +
    wp-event ?M ?Trace adversary-initial-state +
    wp-event ?M ?Composition adversary-initial-state ≤
    (staged-phase-relation-error size
     (staged-query-search-queries budgets i + 1) + query-error-bound) +
    R + T + C
  by (intro add-mono conceptual-bound residual-bound trace-path-bound
    composition-path-bound)
  have event-bound:
    wp-event ?M
    (λout. ?Conceptual out ∨ ?Residual out ∨ ?Trace out ∨
     ?Composition out)
    adversary-initial-state ≤
    (staged-phase-relation-error size
     (staged-query-search-queries budgets i + 1) + query-error-bound) +
    R + T + C
  by (rule order-trans[OF union-bound sum-bound])
  show ?thesis
  by (rule order-trans[OF event-le])
  (use event-bound in ⟨simp add: algebra-simps⟩)

```

qed

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *default-context:*

$\bigwedge \text{prefix prefix-state.}$

Some ((prefix, prefix-state), prefix-state) ∈

set-dist

(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) ⇒

trace-table-low-degree

(query-prefix-trace-conceptual-table-with-default prefix
prefix-state trace-default) ∧

composition-table-low-degree maxDegree

(query-prefix-composition-conceptual-table-with-default prefix
prefix-state composition-default) ∧

$\neg$ *all-queries-consistent*

(query-prefix-trace-conceptual-table-with-default prefix
prefix-state trace-default)

(query-prefix-composition-conceptual-table-with-default prefix
prefix-state composition-default)

(sqp-alphas prefix)

and *trace-path-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)

adversary-initial-state ≤ T

and *composition-path-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)

adversary-initial-state ≤ C

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)

adversary-initial-state ≤

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) +

query-error-bound +

hash-collision-budget-value 0

(staged-query-search-queries budgets i + 1) + T + C

proof –

have *residual-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at

trace-default composition-default i)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1)
by (rule
checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-context
[OF wf controlled i-bound default-context])
have bound:
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1) + T + C
by (rule
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-residual-hit-at-bound
[OF wf controlled i-bound residual-bound trace-path-bound
composition-path-bound])
show ?thesis
using bound **by** simp
qed

lemma *checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-bound-from-conceptual-default-residual-hit-at-bound*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *residual-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
trace-table composition-table i)

adversary-initial-state $\leq R$

and *trace-path-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)

adversary-initial-state $\leq T$

and *composition-path-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)

adversary-initial-state $\leq C$

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-authenticated-candidate-hit-at

```

    trace-table composition-table i)
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  query-error-bound + R + T + C
proof –
let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i
let ?Conceptual =
  checked-staged-security-with-query-prefix-conceptual-default-target-hit
  trace-table composition-table
let ?Residual =
  checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
  trace-table composition-table i
let ?Trace =
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i
let ?Composition =
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i
have event-le:
  wp-event ?M
  (checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
   trace-table composition-table i)
  adversary-initial-state ≤
  wp-event ?M
  (λout. ?Conceptual out ∨ ?Residual out ∨ ?Trace out ∨
   ?Composition out)
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (rule
   checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-imp-conceptual-default-or-residual-or-str
   [OF wf controlled i-bound])
have conceptual-bound:
  wp-event ?M ?Conceptual adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) + query-error-bound
by (rule
  checked-staged-security-with-query-prefix-conceptual-default-target-hit-bound
  [OF wf controlled i-bound])
have union-bound:
  wp-event ?M
  (λout. ?Conceptual out ∨ ?Residual out ∨ ?Trace out ∨
   ?Composition out)
  adversary-initial-state ≤
  wp-event ?M ?Conceptual adversary-initial-state +
  wp-event ?M ?Residual adversary-initial-state +
  wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M ?Composition adversary-initial-state
by (rule wp-event-union-bound4)
have sum-bound:
  wp-event ?M ?Conceptual adversary-initial-state +

```

$wp\text{-}event\ ?M\ ?Residual\ adversary\text{-}initial\text{-}state\ +$
 $wp\text{-}event\ ?M\ ?Trace\ adversary\text{-}initial\text{-}state\ +$
 $wp\text{-}event\ ?M\ ?Composition\ adversary\text{-}initial\text{-}state\ \leq$
 $(staged\text{-}phase\text{-}relation\text{-}error\ size$
 $(staged\text{-}query\text{-}search\text{-}queries\ budgets\ i\ +\ 1) + query\text{-}error\text{-}bound) +$
 $R + T + C$
by (intro add-mono conceptual-bound residual-bound trace-path-bound
composition-path-bound)
have event-bound:
 $wp\text{-}event\ ?M$
 $(\lambda out. ?Conceptual\ out\ \vee\ ?Residual\ out\ \vee\ ?Trace\ out\ \vee$
 $?Composition\ out)$
 $adversary\text{-}initial\text{-}state\ \leq$
 $(staged\text{-}phase\text{-}relation\text{-}error\ size$
 $(staged\text{-}query\text{-}search\text{-}queries\ budgets\ i\ +\ 1) + query\text{-}error\text{-}bound) +$
 $R + T + C$
by (rule order-trans[OF union-bound sum-bound])
show ?thesis
by (rule order-trans[OF event-le])
(use event-bound in ‹simp add: algebra-simps›)
qed

lemma checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-bound-from-conceptual-default-context
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i < rounds$
and default-context:
 $\bigwedge prefix\ prefix\text{-}state.$
 $Some\ ((prefix, prefix\text{-}state), prefix\text{-}state) \in$
 $set\text{-}dist$
 $(execute\ (checked\text{-}staged\text{-}query\text{-}prefix\text{-}with\text{-}state\ A\ i)$
 $adversary\text{-}initial\text{-}state) \implies$
 $trace\text{-}table\text{-}low\text{-}degree$
 $(query\text{-}prefix\text{-}trace\text{-}conceptual\text{-}table\text{-}with\text{-}default\ prefix$
 $prefix\text{-}state\ trace\text{-}table) \wedge$
 $composition\text{-}table\text{-}low\text{-}degree\ maxDegree$
 $(query\text{-}prefix\text{-}composition\text{-}conceptual\text{-}table\text{-}with\text{-}default\ prefix$
 $prefix\text{-}state\ composition\text{-}table) \wedge$
 $\neg\ all\text{-}queries\text{-}consistent$
 $(query\text{-}prefix\text{-}trace\text{-}conceptual\text{-}table\text{-}with\text{-}default\ prefix$
 $prefix\text{-}state\ trace\text{-}table)$
 $(query\text{-}prefix\text{-}composition\text{-}conceptual\text{-}table\text{-}with\text{-}default\ prefix$
 $prefix\text{-}state\ composition\text{-}table)$
 $(sqp\text{-}alphas\ prefix)$
and trace-path-bound:
 $wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}query\text{-}prefix\text{-}data\text{-}state\ A\ i)$
 $(checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}current\text{-}trace\text{-}path\text{-}output\text{-}hit\text{-}at\ i)$
 $adversary\text{-}initial\text{-}state\ \leq\ T$

and *composition-path-bound*:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A\ i$)
 (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
adversary-initial-state $\leq C$
shows
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A\ i$)
 (checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
 trace-table composition-table i)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
query-error-bound +
hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$) + $T + C$
proof –
have *residual-bound*:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A\ i$)
 (checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
 trace-table composition-table i)
adversary-initial-state $\leq$
hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$)
by (rule
 checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-context
 [*OF wf controlled i-bound default-context*])
have *bound*:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A\ i$)
 (checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
 trace-table composition-table i)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
query-error-bound +
hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$) + $T + C$
by (rule
 checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-bound-from-conceptual-default-residual-a
 [*OF wf controlled i-bound residual-bound trace-path-bound*
 composition-path-bound])
show ?thesis
using *bound* **by** *simp*
qed

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-algebraic*
assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: staged-adversary-controlled budgets A
and *i-bound*: $i < \text{rounds}$
and *algebraic-residual-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at*
 i)
 adversary-initial-state $\leq R$
and *trace-path-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-current-trace-path-output-hit-at* i)
 adversary-initial-state $\leq T$
and *composition-path-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-current-composition-path-output-hit-at* i)
 adversary-initial-state $\leq C$
shows
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets* $i + 1$) +
 query-error-bound +
 hash-collision-budget-value 0
 (*staged-query-search-queries budgets* $i + 1$) +
 $R + T + C$
proof –
 have *residual-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at*
 i)
 adversary-initial-state $\leq$
 hash-collision-budget-value 0
 (*staged-query-search-queries budgets* $i + 1$) + R
 by (rule
 checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-bound-from-split
 [*OF wf controlled i-bound algebraic-residual-bound*])
 have *bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets* $i + 1$) +
 query-error-bound +

(hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$) + R) + T + C
 by (rule
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-residual-and
 [OF wf controlled i-bound residual-bound trace-path-bound
 composition-path-bound])
 show ?thesis
 using bound by (simp add: algebra-simps)
 qed

lemma checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-residual-and
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i < \text{rounds}$
and trace-low-degree-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at
 i)
 adversary-initial-state $\leq LT$
and composition-low-degree-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at
 i)
 adversary-initial-state $\leq LC$
and alpha-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at
 i)
 adversary-initial-state $\leq A'$
and trace-path-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
 adversary-initial-state $\leq T$
and composition-path-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
 adversary-initial-state $\leq C$
shows
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size

(staged-query-search-queries budgets $i + 1$) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$) +
 (LT + LC + A') + T + C
proof –
have residual-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
 i)
 adversary-initial-state $\leq$ LT + LC + A'
by (rule
 checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-bound-from-split
 [OF false-statement trace-low-degree-bound
 composition-low-degree-bound alpha-bound])
show ?thesis
by (rule
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-algebraic-re
 [OF wf controlled i-bound residual-bound trace-path-bound
 composition-path-bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and residual-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
 i)
 adversary-initial-state $\leq$ R i
and trace-path-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
 adversary-initial-state $\leq$ T i
and composition-path-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
 adversary-initial-state $\leq$ C i
shows
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

$adversary-initial-state \leq$
 $(\sum_{i < rounds} \text{staged-phase-relation-error size}$
 $\quad (\text{staged-query-search-queries budgets } i + 1) +$
 $\quad \text{query-error-bound} + R \ i + T \ i + C \ i)$

proof (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-current
[OF wf controlled])

fix i
assume $i\text{-bound}: i < rounds$
show
 $wp\text{-event}$
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $adversary-initial-state \leq$
 $\text{staged-phase-relation-error size}$
 $\quad (\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + R \ i + T \ i + C \ i$

by (rule
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-residual-and
[OF wf controlled i-bound residual-bound[OF i-bound]
trace-path-bound[OF i-bound] composition-path-bound[OF i-bound]])

qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual*
assumes $wf: \text{staged-budget-wellformed budgets}$
and $controlled: \text{staged-adversary-controlled budgets } A$
and $residual\text{-bound}: \bigwedge i. i < rounds \implies$
 $wp\text{-event}$
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at}$
 $\quad \text{trace-default composition-default } i)$
 $adversary-initial-state \leq R \ i$
and $trace\text{-path-bound}: \bigwedge i. i < rounds \implies$
 $wp\text{-event}$
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $adversary-initial-state \leq T \ i$
and $composition\text{-path-bound}: \bigwedge i. i < rounds \implies$
 $wp\text{-event}$
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $adversary-initial-state \leq C \ i$

shows
 $wp\text{-event}$
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq$
 $(\sum i < \text{rounds}.$
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 query-error-bound + R i + T i + C i)

proof (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-curr
 [OF wf controlled])

fix *i*
assume *i-bound*: $i < \text{rounds}$
show
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i*)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 query-error-bound + R i + T i + C i

by (rule
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-resi
 [OF wf controlled *i-bound residual-bound*[OF *i-bound*]
 trace-path-bound[OF *i-bound*] *composition-path-bound*[OF *i-bound*]])

qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-*
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *default-context*:
 $\bigwedge i$ *prefix prefix-state. i < rounds* $\implies$
 Some ((prefix, prefix-state), prefix-state) ∈
 set-dist
 (*execute (checked-staged-query-prefix-with-state A i*
 adversary-initial-state) $\implies$
 trace-table-low-degree
 (*query-prefix-trace-conceptual-table-with-default prefix*
 prefix-state trace-default) $\wedge$
 composition-table-low-degree maxDegree
 (*query-prefix-composition-conceptual-table-with-default prefix*
 prefix-state composition-default) $\wedge$
 $\neg$ *all-queries-consistent*
 (*query-prefix-trace-conceptual-table-with-default prefix*
 prefix-state trace-default)
 (*query-prefix-composition-conceptual-table-with-default prefix*
 prefix-state composition-default)
 (*sqp-alphas prefix*)

and *trace-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
 adversary-initial-state $\leq T$ i
and composition-path-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
 adversary-initial-state $\leq C$ i
shows
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
 adversary-initial-state $\leq$
 ($\sum i < \text{rounds}.$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$) +
 T $i + C$ i)
proof (rule
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-curr
 [OF wf controlled])
fix i
assume i -bound: $i < \text{rounds}$
show
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$) +
 T $i + C$ i
by (rule
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-cont
 [OF wf controlled i -bound - trace-path-bound[OF i -bound]
 composition-path-bound[OF i -bound]])
 (rule default-context[OF i -bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and algebraic-residual-bound:
 $\bigwedge i. i < \text{rounds} \implies$

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
 i)
 $\text{adversary-initial-state} \leq R \ i$
and *trace-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
 $\text{adversary-initial-state} \leq T \ i$
and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
 $\text{adversary-initial-state} \leq C \ i$

shows

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
 $\text{adversary-initial-state} \leq$
 $(\sum i < \text{rounds}.$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $R \ i + T \ i + C \ i)$

proof (rule

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-curr
 [OF wf controlled])

fix i

assume $i\text{-bound}: i < \text{rounds}$

show

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $R \ i + T \ i + C \ i$

by (rule

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-algebraic-re
 [OF wf controlled $i\text{-bound}$ algebraic-residual-bound[OF $i\text{-bound}$]
 $\text{trace-path-bound[OF } i\text{-bound}] \text{ composition-path-bound[OF } i\text{-bound}]$])

qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-*

assumes *false-statement:* $\neg \text{exists-valid-trace}$

and *wf:* *staged-budget-wellformed budgets*

and *controlled:* *staged-adversary-controlled budgets A*

and *trace-low-degree-bound:*

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at i)

adversary-initial-state $\leq$ LT i

and *composition-low-degree-bound:*

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at i)

adversary-initial-state $\leq$ LC i

and *alpha-bound:*

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at i)

adversary-initial-state $\leq$ P i

and *trace-path-bound:*

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)

adversary-initial-state $\leq$ T i

and *composition-path-bound:*

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)

adversary-initial-state $\leq$ C i

shows

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

adversary-initial-state $\leq$

($\sum i < \text{rounds}.$

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) +

query-error-bound +

hash-collision-budget-value 0

$(\text{staged-query-search-queries budgets } i + 1) +$
 $(LT\ i + LC\ i + P\ i) + T\ i + C\ i)$
proof (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-current-query-partial-opening-hit-at
 $[OF\ wf\ controlled]$)
fix i
assume $i\text{-bound}: i < \text{rounds}$
show
wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A\ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(LT\ i + LC\ i + P\ i) + T\ i + C\ i$
by (rule
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-residual-split
 $[OF\ \text{false-statement } wf\ \text{controlled } i\text{-bound}]$
 $\text{trace-low-degree-bound}[OF\ i\text{-bound}]$
 $\text{composition-low-degree-bound}[OF\ i\text{-bound}]$
 $\text{alpha-bound}[OF\ i\text{-bound}]$
 $\text{trace-path-bound}[OF\ i\text{-bound}]$
 $\text{composition-path-bound}[OF\ i\text{-bound}]$)]
qed

lemma *checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-trivial-bound:*

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A\ i)$
 $(\text{checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq 1$
by (rule *wp-event-le-1*)

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-trivial-bound:*

assumes $wf: \text{staged-budget-wellformed budgets}$
and $\text{controlled}: \text{staged-adversary-controlled budgets } A$
and $i\text{-bound}: i < \text{rounds}$
and $\text{trace-path-bound}: \text{wp-event}$
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A\ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T$
and $\text{composition-path-bound}: \text{wp-event}$
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A\ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$

adversary-initial-state $\leq C$

shows

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state* $A\ i$)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* i)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-query-search-queries budgets* $i + 1$) +
query-error-bound + 1 + $T + C$

by (rule

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-residual-and
 [*OF wf controlled i-bound*
checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-trivial-bound
trace-path-bound composition-path-bound])

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets* A

and *trace-path-bound:*

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state* $A\ i$)
 (*checked-staged-security-with-query-prefix-current-trace-path-output-hit-at* i)
adversary-initial-state $\leq T\ i$

and *composition-path-bound:*

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state* $A\ i$)
 (*checked-staged-security-with-query-prefix-current-composition-path-output-hit-at* i)
adversary-initial-state $\leq C\ i$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq$
 ($\sum i < \text{rounds}.$
staged-phase-relation-error size
 (*staged-query-search-queries budgets* $i + 1$) +
query-error-bound + 1 + $T\ i + C\ i$)

proof (rule

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-residu
 [*OF wf controlled - trace-path-bound composition-path-bound*])

fix i

assume $i < \text{rounds}$

show

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state* $A\ i$)
 (*checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at* i)
adversary-initial-state ≤ 1

by (rule
checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-trivial-bound)
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-*

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *conceptual-context*:

$\bigwedge i$ *prefix prefix-state*.

$i < \text{rounds} \implies$

Some ((prefix, prefix-state), prefix-state) ∈
set-dist

(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) ⟹

$\neg$ *hash-map-output-collision prefix-state* $\wedge$

trace-table-low-degree

(query-prefix-trace-conceptual-table prefix prefix-state) ∧

composition-table-low-degree maxDegree

(query-prefix-composition-conceptual-table prefix prefix-state) ∧

$\neg$ *all-queries-consistent*

(query-prefix-trace-conceptual-table prefix prefix-state)

(query-prefix-composition-conceptual-table prefix prefix-state)

(sqp-alphas prefix)

and *trace-path-bound*:

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)

adversary-initial-state ≤ T i

and *composition-path-bound*:

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)

adversary-initial-state ≤ C i

shows

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

adversary-initial-state ≤

$(\sum i < \text{rounds}.$

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) +

query-error-bound + T i + C i)

proof (rule

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-prefix-target-and-

[OF wf controlled - trace-path-bound composition-path-bound])

fix *i*

assume *i-bound*: $i < \text{rounds}$

```

show
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-hit
      staged-query-prefix-prefix-authenticated-opening-target)
  adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + query-error-bound
by (rule
  checked-staged-query-prefix-prefix-authenticated-target-hit-bound-from-conceptual
    [OF wf controlled i-bound])
  (rule conceptual-context[OF i-bound])
qed

end

end

```

```

theory Soundness-Conceptual-Query-Current-Candidate
imports Soundness-Conceptual-Query-Current-Path
begin

```

Fixed-candidate projection for current-query openings.

This layer keeps the trace/composition candidate tables as fixed parameters while projecting the data-state current-opening event to the corresponding query-prefix event. It is the form needed by the conceptual/default query target bound: the target is fixed before the query-index challenge.

```

context soundness
begin

```

```

definition staged-security-with-data-state-current-query-partial-candidate-hit-at
where

```

```

  staged-security-with-data-state-current-query-partial-candidate-hit-at
    trace-table composition-table i out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (((data, attacker-state), result), final-state)  $\Rightarrow$ 
    (let s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    in  $\exists$  raw trace-openings composition-openings rest.
      i < rounds  $\wedge$ 
      fmlookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some raw  $\wedge$ 
      staged-composition-fri-roots data  $\neq$  []  $\wedge$ 

```

Some (result, final-state) ∈ set-dist (execute verify-monad s) ∧
partial-authenticated-table (staged-trace-root data)
*(scale * clength) trace-openings final-state ∧*
partial-authenticated-table
*(hd (staged-composition-fri-roots data)) (scale * clength)*
composition-openings final-state ∧
verifier-header-transcript s
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
rest ∧
partial-query-openings-consistent trace-openings
composition-openings (staged-alphas data) (index (to-nat raw)) ∧
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings]) ∧
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings]))

definition *staged-security-with-data-state-current-query-some-partial-candidate-hit*
where

staged-security-with-data-state-current-query-some-partial-candidate-hit
out $\longleftrightarrow$
 $(\exists \text{ trace-table } \text{composition-table } i.$
 $i < \text{rounds} \wedge$
staged-security-with-data-state-current-query-partial-candidate-hit-at
trace-table composition-table i out)

lemma *staged-security-with-data-state-current-query-partial-candidate-hit-at-imp-query-prefix-authenticated-candidate*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *support:*

Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)

and *hit:*

staged-security-with-data-state-current-query-partial-candidate-hit-at
trace-table composition-table i
(Some (((data, attacker-state), result), final-state))

shows

checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
trace-table composition-table i

```

    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from hit obtain raw' trace-openings composition-openings rest where
  final-lookup':
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw'
and comp-nonempty: staged-composition-fri-roots data ≠ []
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and comp-auth:
  partial-authenticated-table
    (hd (staged-composition-fri-roots data)) (scale * clength)
    composition-openings final-state
and consistent':
  partial-query-openings-consistent trace-openings
    composition-openings (staged-alphas data) (index (to-nat raw'))
and trace-candidate:
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
unfolding
  staged-security-with-data-state-current-query-partial-candidate-hit-at-def
  Let-def
by auto
have final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
by (rule
  checked-staged-security-experiment-with-query-prefix-data-state-raw-final-lookup
  [OF wf controlled i-bound support])
have raw-eq: raw' = raw
using final-lookup' final-lookup by simp
have opening-hit:
  checked-staged-security-with-query-prefix-authenticated-opening-hit
  trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),

```

```

      ((data, attacker-state), result)), final-state))
unfolding checked-staged-security-with-query-prefix-authenticated-opening-hit-def
using comp-nonempty trace-auth comp-auth consistent' raw-eq by simp
show ?thesis
unfolding checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-def
by (intro exI[of - trace-openings] exI[of - composition-openings]
      conjI opening-hit trace-candidate comp-candidate)
qed

```

lemma staged-security-with-data-state-current-query-partial-candidate-hit-at-le-query-prefix-authenticated-candidate:

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i < \text{rounds}$

shows

```

wp-event
  (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-current-query-partial-candidate-hit-at
    trace-table composition-table i)
  adversary-initial-state  $\leq$ 
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
    trace-table composition-table i)
  adversary-initial-state

```

proof —

```

let ?projected =
   $\lambda$ out. case out of
    None  $\Rightarrow$ 
      staged-security-with-data-state-current-query-partial-candidate-hit-at
        trace-table composition-table i None
    | Some (packed, t)  $\Rightarrow$ 
      staged-security-with-data-state-current-query-partial-candidate-hit-at
        trace-table composition-table i (Some (snd packed, t))

```

have projection:

```

wp-event
  (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-current-query-partial-candidate-hit-at
    trace-table composition-table i)
  adversary-initial-state =
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  ?projected
  adversary-initial-state
by (rule
      checked-staged-security-experiment-with-query-prefix-data-state-projection-event
      [OF i-bound])

```

have event-le:

```

wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)

```



```

    ?projected
    adversary-initial-state ≤
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
    trace-table composition-table i)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)
and hit: ?projected out
show
  checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
    trace-table composition-table i out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    staged-security-with-data-state-current-query-partial-candidate-hit-at-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
  final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
show ?thesis
  unfolding out-eq
  by (rule
    staged-security-with-data-state-current-query-partial-candidate-hit-at-imp-query-prefix-authenticated-candidate
      [OF wf controlled i-bound])
    (use support hit out-eq in simp-all)
qed
qed
show ?thesis
  unfolding projection
  by (rule event-le)
qed

lemma staged-security-with-data-state-current-query-partial-candidate-hit-at-bound-from-conceptual-default:
  assumes wf: staged-budget-wellformed budgets

```

and *controlled*: staged-adversary-controlled budgets A
and *i-bound*: $i < \text{rounds}$
and *residual-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at*
 trace-table composition-table i)
 adversary-initial-state $\leq R$
and *trace-path-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-current-trace-path-output-hit-at* i)
 adversary-initial-state $\leq T$
and *composition-path-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-current-composition-path-output-hit-at* i)
 adversary-initial-state $\leq C$
shows
 wp-event
 (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-current-query-partial-candidate-hit-at*
 trace-table composition-table i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets* $i + 1$) +
 query-error-bound + $R + T + C$
proof –
 have *prefix-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-authenticated-candidate-hit-at*
 trace-table composition-table i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets* $i + 1$) +
 query-error-bound + $R + T + C$
 by (rule
 checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-bound-from-conceptual-default-residual-a
 [*OF wf controlled i-bound residual-bound trace-path-bound*
 composition-path-bound])
 show ?thesis
 by (rule *order-trans*
 [*OF*
 staged-security-with-data-state-current-query-partial-candidate-hit-at-le-query-prefix-authenticated-candidate
 prefix-bound])
 (*use wf controlled i-bound in simp-all*)
qed

lemma *staged-security-with-data-state-current-query-partial-candidate-hit-at-bound-from-conceptual-default-context:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *default-context:*

$\bigwedge \text{prefix prefix-state.}$

Some ((prefix, prefix-state), prefix-state) ∈ set-dist

(execute (checked-staged-query-prefix-with-state A i) adversary-initial-state) ⇒

trace-table-low-degree

(query-prefix-trace-conceptual-table-with-default prefix prefix-state trace-table) ∧

composition-table-low-degree maxDegree

(query-prefix-composition-conceptual-table-with-default prefix prefix-state composition-table) ∧

$\neg$ *all-queries-consistent*

(query-prefix-trace-conceptual-table-with-default prefix prefix-state trace-table)

(query-prefix-composition-conceptual-table-with-default prefix prefix-state composition-table)

(sqp-alphas prefix)

and *trace-path-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)

adversary-initial-state ≤ T

and *composition-path-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)

adversary-initial-state ≤ C

shows

wp-event

(checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-current-query-partial-candidate-hit-at trace-table composition-table i)

adversary-initial-state ≤

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) +

query-error-bound +

hash-collision-budget-value 0

(staged-query-search-queries budgets i + 1) + T + C

proof –

have *prefix-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-authenticated-candidate-hit-at trace-table composition-table i)

adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1) + T + C
by (rule
checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-bound-from-conceptual-default-context-an
[OF wf controlled i-bound default-context trace-path-bound
composition-path-bound])
show ?thesis
by (rule order-trans
[OF
staged-security-with-data-state-current-query-partial-candidate-hit-at-le-query-prefix-authenticated-candidate
prefix-bound])
(use wf controlled i-bound in simp-all)
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-current-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)
and *hit:*
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
obtains *trace-table composition-table i where*
i < rounds
staged-security-with-data-state-current-query-partial-candidate-hit-at
trace-table composition-table i
(Some (((data, attacker-state), result), final-state))
proof –
let ?s =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
have *data-support:*
Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute (checked-staged-security-experiment-with-data-state A)
adversary-initial-state)
by (rule
checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support

```

    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
    [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
have header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-current-openingsE
  [OF wf controlled support hit])
fix s trace-table composition-table query-idxs f i raw trace-openings
  composition-openings trace-openingss composition-openingss
assume s-def:
  s = verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
and bad-context:
  composition-bad-context s (Some (result, final-state))
    trace-table composition-table (staged-alphas data) query-idxs f
and i-bound: i < rounds
and final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some raw
and idx-eq: index (to-nat raw) = query-idxs ! i
and comp-nonempty: staged-composition-fri-roots data ≠ []
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state

```

```

and comp-auth:
  partial-authenticated-table
    (hd (staged-composition-fri-roots data)) (scale * clength)
    composition-openings final-state
and consistent:
  partial-query-openings-consistent trace-openings
    composition-openings (staged-alphas data) (query-idxs ! i)
and trace-candidate-full:
  partial-trace-table-candidate trace-table trace-openingsss
and comp-candidate-full:
  partial-composition-table-candidate composition-table
    composition-openingsss
and trace-openings-eq: trace-openings = trace-openingsss ! i
and comp-openings-eq: composition-openings = composition-openingsss ! i
and all-queries:
  all-queries-consistent trace-table composition-table
    (staged-alphas data)
and not-alpha-prefix:
  ¬ staged-alphas data ∈
    alpha-prefix-union-bad-sets alpha-prefix-state
    composition-trace-bad-alpha-space (fst alpha-prefix)
have consistent-raw:
  partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
using consistent idx-eq by simp
have trace-candidate:
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
by (rule partial-trace-table-candidate-single-roundI)
    (use i-bound trace-candidate-full trace-openings-eq in
      ⟨auto simp: partial-trace-table-candidate-def⟩)
have comp-candidate:
  partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
by (rule partial-composition-table-candidate-single-roundI)
    (use i-bound comp-candidate-full comp-openings-eq in
      ⟨auto simp: partial-composition-table-candidate-def⟩)
have round-hit:
  staged-security-with-data-state-current-query-partial-candidate-hit-at
    trace-table composition-table i
    (Some (((data, attacker-state), result), final-state))
unfolding
  staged-security-with-data-state-current-query-partial-candidate-hit-at-def
  Let-def
apply simp
by (intro exI[of - raw] exI[of - trace-openings]
  exI[of - composition-openings]
  exI[of - List.concat (staged-query-chunks data)] conjI)
  (use i-bound final-lookup comp-nonempty verifier trace-auth

```

```

      comp-auth header consistent-raw trace-candidate comp-candidate
    in simp-all)
  show ?thesis
    by (rule that[OF i-bound round-hit])
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-current-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
  result), final-state) ∈
set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
  result), final-state))
shows
  staged-security-with-data-state-current-query-some-partial-candidate-hit
  (Some (((data, attacker-state), result), final-state))
proof -
  from
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-current-partia
    [OF wf controlled support hit]
  obtain trace-table composition-table i where
    i-bound: i < rounds
  and round-hit:
    staged-security-with-data-state-current-query-partial-candidate-hit-at
    trace-table composition-table i
    (Some (((data, attacker-state), result), final-state))
  by blast
  show ?thesis
  unfolding
    staged-security-with-data-state-current-query-some-partial-candidate-hit-def
  by (intro exI[of - trace-table] exI[of - composition-table]
    exI[of - i] conjI i-bound round-hit)
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-data-state-current-so
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

```

```

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state ≤
wp-event
(checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-current-query-some-partial-candidate-hit
adversary-initial-state
proof –
  let ?project =
    λx. ((snd (fst (fst x)), snd (fst x)), snd x)
  let ?projected =
    λout. case out of
      None ⇒ False
    | Some (packed, t) ⇒
      staged-security-with-data-state-current-query-some-partial-candidate-hit
        (Some (?project packed, t))
  have event-le:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    adversary-initial-state ≤
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?projected adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    out
  show ?projected out
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    by simp
  next
  case (Some packed)
  then obtain alpha-prefix alpha-prefix-state data attacker-state result
    final-state where out-eq:

```



```

    out =
      Some (((alpha-prefix, alpha-prefix-state), data),
            attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
have candidate-hit:
  staged-security-with-data-state-current-query-some-partial-candidate-hit
  (Some (((data, attacker-state), result), final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-current-s
    [OF wf controlled])
    (use support hit out-eq in simp-all)
show ?thesis
  unfolding out-eq
  using candidate-hit by simp
qed
qed
have projection:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ?projected adversary-initial-state =
  wp-event
  (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-current-query-some-partial-candidate-hit
  adversary-initial-state
proof -
  have
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-some-partial-candidate-hit
    adversary-initial-state =
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
      ≫ (λx. return (?project x)))
    staged-security-with-data-state-current-query-some-partial-candidate-hit
    adversary-initial-state
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
also have ... =
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ?projected adversary-initial-state
  by (subst wp-event-bind-return-map)
  (simp add:
    staged-security-with-data-state-current-query-some-partial-candidate-hit-def
    staged-security-with-data-state-current-query-partial-candidate-hit-at-def)
finally show ?thesis
  by simp
qed

```

```

show ?thesis
  by (rule order-trans[OF event-le])
    (simp add: projection)
qed

end

end

```

```

theory Soundness-Staged-Aligned-Current-Query-Path
imports
  Soundness-Staged-Aligned-Current-Query
  Soundness-Conceptual-Query-Current-Candidate
begin

```

Current-query wrappers that use structured Merkle path-output events.

This layer keeps trace and composition authenticated-opening path outputs separate, which is the form needed for Merkle and candidate uniqueness arguments in the staged soundness proof.

```

context soundness
begin

```

```

theorem stark-soundness-from-aligned-partial-query-prefix-current-prefix-structured-paths:
fixes trace-fri-error' composition-fri-error'  $N :: \text{prob}$ 
and  $P\ T\ C :: \text{nat} \Rightarrow \text{prob}$ 
assumes false-statement:  $\neg \text{exists-valid-trace}$ 
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets  $A$ 
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  composition-fri-error'
and prefix-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state  $A\ i$ )
      (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at  $i$ )
      adversary-initial-state  $\leq P\ i$ 
and trace-path-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state  $A\ i$ )

```

$(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T \ i$
and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *not-prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}. P \ i + T \ i + C \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$
proof (rule *stark-soundness-from-aligned-partial-query-prefix-current-rounds*

```

[OF false-statement wf controlled trace-fri-bound comp-fri-bound])
fix i
assume i-bound:  $i < \text{rounds}$ 
show
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state  $\leq P i + T i + C i$ 
  by (rule
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-structured-paths
    [OF wf controlled i-bound prefix-bound[OF i-bound]
      trace-path-bound[OF i-bound] composition-path-bound[OF i-bound]])
qed (rule not-prefix-bound empty-trace-fri-bound empty-query-bound)+

theorem stark-soundness-from-aligned-partial-query-prefix-dynamic-target-structured-paths:
fixes trace-fri-error' composition-fri-error' N :: prob
and P T C :: nat  $\Rightarrow$  prob
assumes false-statement:  $\neg \text{exists-valid-trace}$ 
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq \text{trace-fri-error}'$ 
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq \text{composition-fri-error}'$ 
and prefix-target-bound:
   $\bigwedge i. i < \text{rounds} \Rightarrow$ 
    wp-event
      (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-hit
        staged-query-prefix-prefix-authenticated-opening-target)
      adversary-initial-state  $\leq P i$ 
and trace-path-bound:
   $\bigwedge i. i < \text{rounds} \Rightarrow$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
      adversary-initial-state  $\leq T i$ 
and composition-path-bound:
   $\bigwedge i. i < \text{rounds} \Rightarrow$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
      adversary-initial-state  $\leq C i$ 

```

and *not-prefix-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq N$
and *empty-trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *nnreal rounds* * *query-error-bound*
shows *checked-staged-adversary-acceptance-probability* *A* $\leq$
 hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
 staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 N +
 staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0)) +
 trace-fri-error' + *composition-fri-error'* +
 ($\sum i < \text{rounds}. P\ i + T\ i + C\ i$) +
 (*composition-error-bound* +
 staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 N +
 staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
 trace-fri-error + *nnreal rounds* * *query-error-bound*)
proof –
have *prefix-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)
 (*checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at* *i*)
 adversary-initial-state $\leq P\ i$
by (rule
 checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target

```

    [OF wf controlled - prefix-target-bound])
show ?thesis
by (rule
  stark-soundness-from-aligned-partial-query-prefix-current-prefix-structured-paths
  [OF false-statement wf controlled trace-fri-bound comp-fri-bound
    prefix-bound trace-path-bound composition-path-bound not-prefix-bound
    empty-trace-fri-bound empty-query-bound])
qed

theorem stark-soundness-from-aligned-partial-query-prefix-trace-index-structured-paths:
fixes trace-fri-error' composition-fri-error' N :: prob
and T C :: nat  $\Rightarrow$  prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  composition-fri-error'
and trace-frac:
   $\bigwedge i$  prefix prefix-state.
    i < rounds  $\Rightarrow$ 
    Some ((prefix, prefix-state), prefix-state)  $\in$ 
      set-dist
        (execute (checked-staged-query-prefix-with-state A i)
          adversary-initial-state)  $\Rightarrow$ 
      nnreal
        (card
          (staged-query-prefix-prefix-authenticated-trace-indices prefix
            prefix-state)) /
        nnreal (card query-sample-space)  $\leq$  query-error-bound
and trace-path-bound:
   $\bigwedge i$ . i < rounds  $\Rightarrow$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
      adversary-initial-state  $\leq$  T i
and composition-path-bound:
   $\bigwedge i$ . i < rounds  $\Rightarrow$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
      adversary-initial-state  $\leq$  C i

```

and *not-prefix-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq N$
and *empty-trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *nnreal rounds* * *query-error-bound*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
 hash-collision-budget-value 0
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 N +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)) +
 trace-fri-error' + *composition-fri-error'* +
 ($\sum i < \text{rounds}$.
 staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
 query-error-bound + *T i* + *C i*) +
 (*composition-error-bound* +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 N +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 trace-fri-error + *nnreal rounds* * *query-error-bound*)
proof (*rule*
 stark-soundness-from-aligned-partial-query-prefix-dynamic-target-structured-paths
 [*OF false-statement wf controlled trace-fri-bound comp-fri-bound - trace-path-bound*
 composition-path-bound not-prefix-bound empty-trace-fri-bound empty-query-bound])
fix *i*
assume *i-bound*: *i* < *rounds*

show

wp-event (*checked-staged-query-prefix-receive-with-state A i*)
 (*checked-staged-query-prefix-dynamic-index-hit*
 staged-query-prefix-prefix-authenticated-opening-target)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) + *query-error-bound*
by (*rule*
 checked-staged-query-prefix-prefix-authenticated-target-hit-bound-from-trace-indices
 [*OF wf controlled i-bound*])
 (*use trace-frac[OF i-bound]* **in** *blast*)

qed

theorem *stark-soundness-from-aligned-partial-query-prefix-conceptual-structured-paths:*

fixes *trace-fri-error'* *composition-fri-error' N :: prob*

and *T C :: nat $\Rightarrow$ prob*

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *trace-fri-bound:*

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*

and *comp-fri-bound:*

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*

and *conceptual-context:*

$\bigwedge i$ *prefix prefix-state.*
i < rounds $\Rightarrow$
Some ((prefix, prefix-state), prefix-state) $\in$
set-dist
 (*execute* (*checked-staged-query-prefix-with-state A i*)
 adversary-initial-state) $\Rightarrow$
 $\neg$ *hash-map-output-collision prefix-state $\wedge$*
trace-table-low-degree
 (*query-prefix-trace-conceptual-table prefix prefix-state*) $\wedge$
composition-table-low-degree maxDegree
 (*query-prefix-composition-conceptual-table prefix prefix-state*) $\wedge$
 $\neg$ *all-queries-consistent*
 (*query-prefix-trace-conceptual-table prefix prefix-state*)
 (*query-prefix-composition-conceptual-table prefix prefix-state*)
 (*sqp-alphas prefix*)

and *trace-path-bound:*

$\bigwedge i.$ *i < rounds $\Rightarrow$*
wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)

$(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T \ i$
and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *not-prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}.$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + T \ i + C \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$

$(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$
proof (rule
 $\text{stark-soundness-from-aligned-partial-query-prefix-dynamic-target-structured-paths}$
 $[\text{OF false-statement wf controlled trace-fri-bound comp-fri-bound - trace-path-bound}$
 $\text{composition-path-bound not-prefix-bound empty-trace-fri-bound empty-query-bound}])$
fix i
assume $i\text{-bound}: i < \text{rounds}$
show
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-prefix-authenticated-opening-target})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$
by (rule
 $\text{checked-staged-query-prefix-prefix-authenticated-target-hit-bound-from-conceptual}$
 $[\text{OF wf controlled } i\text{-bound}])$
 $(\text{rule conceptual-context}[\text{OF } i\text{-bound}])$
qed

theorem $\text{stark-soundness-from-aligned-partial-query-prefix-conceptual-default-structured-paths}$:
fixes $\text{trace-fri-error}' \text{ composition-fri-error}' N :: \text{prob}$
and $T \ C :: \text{nat} \Rightarrow \text{prob}$
and $\text{trace-default composition-default} :: 'f \text{ list}$
assumes $\text{false-statement}: \neg \text{exists-valid-trace}$
and $\text{wf}: \text{staged-budget-wellformed budgets}$
and $\text{controlled}: \text{staged-adversary-controlled budgets } A$
and trace-fri-bound :
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$
and comp-fri-bound :
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$
and default-context :
 $\bigwedge i \text{ prefix prefix-state.}$
 $i < \text{rounds} \implies$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\text{trace-table-low-degree}$
 $(\text{query-prefix-trace-conceptual-table-with-default prefix}$
 $\text{prefix-state trace-default}) \wedge$
 $\text{composition-table-low-degree maxDegree}$

$(\text{query-prefix-composition-conceptual-table-with-default prefix}$
 $\text{prefix-state composition-default}) \wedge$
 $\neg \text{all-queries-consistent}$
 $(\text{query-prefix-trace-conceptual-table-with-default prefix}$
 $\text{prefix-state trace-default})$
 $(\text{query-prefix-composition-conceptual-table-with-default prefix}$
 $\text{prefix-state composition-default})$
 $(\text{sqp-alphas prefix})$
and *trace-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T \ i$
and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *not-prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$

```

    trace-fri-error' + composition-fri-error' +
    (∑ i < rounds.
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      query-error-bound +
      hash-collision-budget-value 0
      (staged-query-search-queries budgets i + 1) +
      T i + C i) +
    (composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      N +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      trace-fri-error + nreal rounds * query-error-bound)
  proof (rule stark-soundness-from-aligned-partial-query-prefix-current-rounds
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound - not-prefix-bound
      empty-trace-fri-bound empty-query-bound])
  fix i
  assume i-bound: i < rounds
  show
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    query-error-bound +
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1) +
    T i + C i
  by (rule
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-cont
    [OF wf controlled i-bound - trace-path-bound[OF i-bound]
      composition-path-bound[OF i-bound]])
    (rule default-context[OF i-bound])
  qed

end

end

theory Soundness-Conceptual-Query-Current-Alpha-Clean
imports Soundness-Staged-Aligned-Current-Query-Path
begin

```

Alpha-clean wrappers for the default-backed conceptual current-query route.

This layer derives the default-context non-consistency premise from the false statement, low degree of the fixed default tables, and the fact that the sampled alpha vector is not in the corresponding composition bad set.

context *soundness*
begin

lemma *checked-staged-security-with-query-prefix-conceptual-default-context-from-alpha-clean:*

assumes *false-statement: $\neg$ exists-valid-trace*
and *alphas-length: $\text{length } (\text{sqp-alphas prefix}) = \text{length spec}$*
and *trace-low:*
 trace-table-low-degree
 (query-prefix-trace-conceptual-table-with-default prefix prefix-state
 trace-default)
and *composition-low:*
 composition-table-low-degree maxDegree
 (query-prefix-composition-conceptual-table-with-default prefix
 prefix-state composition-default)
and *alpha-clean:*
 sqp-alphas prefix $\notin$ composition-trace-bad-alpha-space
 (query-prefix-trace-conceptual-table-with-default prefix prefix-state
 trace-default)

shows

trace-table-low-degree
 (query-prefix-trace-conceptual-table-with-default prefix prefix-state
 trace-default) $\wedge$
composition-table-low-degree maxDegree
 (query-prefix-composition-conceptual-table-with-default prefix
 prefix-state composition-default) $\wedge$
 $\neg$ *all-queries-consistent*
 (query-prefix-trace-conceptual-table-with-default prefix prefix-state
 trace-default)
 (query-prefix-composition-conceptual-table-with-default prefix
 prefix-state composition-default)
 (sqp-alphas prefix)

proof (*intro conjI trace-low composition-low notI*)

assume *all-queries:*
 all-queries-consistent
 (query-prefix-trace-conceptual-table-with-default prefix prefix-state
 trace-default)
 (query-prefix-composition-conceptual-table-with-default prefix
 prefix-state composition-default)
 (sqp-alphas prefix)

have *alpha-bad:*

sqp-alphas prefix $\in$ composition-trace-bad-alpha-space
 (query-prefix-trace-conceptual-table-with-default prefix prefix-state
 trace-default)

by (*rule low-degree-all-queries-consistent-imp-composition-trace-bad-alpha-space*
 [*OF false-statement alphas-length trace-low composition-low all-queries*])

show *False*

using *alpha-clean alpha-bad* by *simp*
qed

lemma *checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-alpha-clean:*

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *i-bound*: $i < \text{rounds}$

and *default-alpha-clean*:

$\bigwedge \text{prefix prefix-state.}$

Some $((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
set-dist

$(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$

trace-table-low-degree

$(\text{query-prefix-trace-conceptual-table-with-default prefix}$
 $\text{prefix-state trace-default}) \wedge$

composition-table-low-degree maxDegree

$(\text{query-prefix-composition-conceptual-table-with-default prefix}$
 $\text{prefix-state composition-default}) \wedge$

sqp-alphas prefix $\notin$ *composition-trace-bad-alpha-space*

$(\text{query-prefix-trace-conceptual-table-with-default prefix}$
 $\text{prefix-state trace-default})$

shows

wp-event

$(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$

$(\text{checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at}$
 $\text{trace-default composition-default } i)$

adversary-initial-state $\leq$

hash-collision-budget-value 0

$(\text{staged-query-search-queries budgets } i + 1)$

proof (rule

checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-context
 $[OF \text{ wf controlled } i\text{-bound}]$)

fix *prefix prefix-state*

assume *prefix-support*:

Some $((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
set-dist

$(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state})$

from *default-alpha-clean* $[OF \text{ prefix-support}]$ **have** *trace-low*:

trace-table-low-degree

$(\text{query-prefix-trace-conceptual-table-with-default prefix}$
 $\text{prefix-state trace-default})$

and *composition-low*:

composition-table-low-degree maxDegree

$(\text{query-prefix-composition-conceptual-table-with-default prefix}$
 $\text{prefix-state composition-default})$

and *alpha-clean*:

```

    sqp-alphas prefix  $\notin$  composition-trace-bad-alpha-space
    (query-prefix-trace-conceptual-table-with-default prefix
     prefix-state trace-default)
  by blast+
have alphas-length: length (sqp-alphas prefix) = length spec
  by (rule checked-staged-query-prefix-with-state-alphas-length
      [OF prefix-support])
show
  trace-table-low-degree
    (query-prefix-trace-conceptual-table-with-default prefix prefix-state
     trace-default)  $\wedge$ 
  composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table-with-default prefix
     prefix-state composition-default)  $\wedge$ 
 $\neg$  all-queries-consistent
    (query-prefix-trace-conceptual-table-with-default prefix prefix-state
     trace-default)
    (query-prefix-composition-conceptual-table-with-default prefix
     prefix-state composition-default)
    (sqp-alphas prefix)
  by (rule
      checked-staged-security-with-query-prefix-conceptual-default-context-from-alpha-clean
      [OF false-statement alphas-length trace-low composition-low alpha-clean])
qed

lemma query-prefix-default-alpha-clean-from-alpha-prefix-union-clean:
  assumes trace-eq:
    query-prefix-trace-conceptual-table-with-default prefix prefix-state
    trace-default = trace-default
  and trace-candidate:
    trace-default  $\in$ 
    alpha-prefix-trace-table-candidates prefix-state (sqp-trace-root prefix)
  and union-clean:
    sqp-alphas prefix  $\notin$ 
    alpha-prefix-union-bad-sets prefix-state composition-trace-bad-alpha-space
    (sqp-trace-root prefix)
  shows
    sqp-alphas prefix  $\notin$  composition-trace-bad-alpha-space
    (query-prefix-trace-conceptual-table-with-default prefix prefix-state
     trace-default)
proof
  assume alpha-bad:
    sqp-alphas prefix  $\in$  composition-trace-bad-alpha-space
    (query-prefix-trace-conceptual-table-with-default prefix prefix-state
     trace-default)
  have alpha-space:
    sqp-alphas prefix  $\in$  alpha-space
  using alpha-bad composition-trace-bad-alpha-space-subset-alpha-space
  by blast

```

have
sqp-alphas prefix $\in$
alpha-prefix-union-bad-sets prefix-state composition-trace-bad-alpha-space
(sqp-trace-root prefix)
unfolding *alpha-prefix-union-bad-sets-def*
using *trace-candidate alpha-bad trace-eq alpha-space* **by** *auto*
then show *False*
using *union-clean* **by** *simp*
qed

lemma *checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-alpha-prefix-u*

assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < \text{rounds}$
and *default-prefix-clean*:
 $\bigwedge \text{prefix prefix-state.}$
Some ((prefix, prefix-state), prefix-state) $\in$
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) $\implies$
query-prefix-trace-conceptual-table-with-default prefix prefix-state
trace-default = trace-default $\wedge$
query-prefix-composition-conceptual-table-with-default prefix
prefix-state composition-default = composition-default $\wedge$
trace-table-low-degree trace-default $\wedge$
composition-table-low-degree maxDegree composition-default $\wedge$
trace-default $\in$
alpha-prefix-trace-table-candidates prefix-state
(sqp-trace-root prefix) $\wedge$
sqp-alphas prefix $\notin$
alpha-prefix-union-bad-sets prefix-state
composition-trace-bad-alpha-space (sqp-trace-root prefix)

shows

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
trace-default composition-default i)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1)

proof (rule

checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-alpha-clean
[OF false-statement wf controlled i-bound])

fix *prefix prefix-state*

assume *prefix-support*:

Some ((prefix, prefix-state), prefix-state) $\in$
set-dist
(execute (checked-staged-query-prefix-with-state A i)

adversary-initial-state)
from *default-prefix-clean*[*OF prefix-support*] **have** *trace-eq*:
query-prefix-trace-conceptual-table-with-default prefix prefix-state
trace-default = trace-default
and *composition-eq*:
query-prefix-composition-conceptual-table-with-default prefix
prefix-state composition-default = composition-default
and *trace-low*: *trace-table-low-degree trace-default*
and *composition-low*:
composition-table-low-degree maxDegree composition-default
and *trace-candidate*:
trace-default $\in$
alpha-prefix-trace-table-candidates prefix-state
(sqp-trace-root prefix)
and *union-clean*:
sqp-alphas prefix $\notin$
alpha-prefix-union-bad-sets prefix-state
composition-trace-bad-alpha-space (sqp-trace-root prefix)
by *blast+*
have *alpha-clean*:
sqp-alphas prefix $\notin$ *composition-trace-bad-alpha-space*
(query-prefix-trace-conceptual-table-with-default prefix prefix-state
trace-default)
by (*rule query-prefix-default-alpha-clean-from-alpha-prefix-union-clean*
[*OF trace-eq trace-candidate union-clean*])
show
trace-table-low-degree
(query-prefix-trace-conceptual-table-with-default prefix prefix-state
trace-default) $\wedge$
composition-table-low-degree maxDegree
(query-prefix-composition-conceptual-table-with-default prefix
prefix-state composition-default) $\wedge$
sqp-alphas prefix $\notin$ *composition-trace-bad-alpha-space*
(query-prefix-trace-conceptual-table-with-default prefix prefix-state
trace-default)
using *trace-eq composition-eq trace-low composition-low alpha-clean* **by** *simp*
qed

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < \text{rounds}$
and *default-alpha-clean*:
 $\bigwedge \text{prefix prefix-state.}$
Some ((prefix, prefix-state), prefix-state) $\in$
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) $\implies$

trace-table-low-degree
 (query-prefix-trace-conceptual-table-with-default prefix
 prefix-state trace-default) $\wedge$
composition-table-low-degree maxDegree
 (query-prefix-composition-conceptual-table-with-default prefix
 prefix-state composition-default) $\wedge$
sqp-alphas prefix $\notin$ composition-trace-bad-alpha-space
 (query-prefix-trace-conceptual-table-with-default prefix
 prefix-state trace-default)
and trace-path-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
 adversary-initial-state $\leq T$
and composition-path-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
 adversary-initial-state $\leq C$
shows
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$) + $T + C$
proof –
have residual-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
 trace-default composition-default i)
 adversary-initial-state $\leq$
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$)
by (rule
 checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-alpha-clean
 [OF false-statement wf controlled i-bound default-alpha-clean])
have bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound +

hash-collision-budget-value 0
(staged-query-search-queries budgets $i + 1$) + $T + C$
by *rule*
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-residual-bound
[OF wf controlled i-bound residual-bound trace-path-bound
composition-path-bound])
show *?thesis*
using *bound by simp*
qed

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-residual-bound*
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i < \text{rounds}$*
and *default-prefix-clean:*
 $\bigwedge \text{prefix prefix-state.}$
Some ((prefix, prefix-state), prefix-state) $\in$
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) $\implies$
query-prefix-trace-conceptual-table-with-default prefix prefix-state
trace-default = trace-default $\wedge$
query-prefix-composition-conceptual-table-with-default prefix
prefix-state composition-default = composition-default $\wedge$
trace-table-low-degree trace-default $\wedge$
composition-table-low-degree maxDegree composition-default $\wedge$
trace-default $\in$
alpha-prefix-trace-table-candidates prefix-state
(sqp-trace-root prefix) $\wedge$
sqp-alphas prefix $\notin$
alpha-prefix-union-bad-sets prefix-state
composition-trace-bad-alpha-space (sqp-trace-root prefix)
and *trace-path-bound:*
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
adversary-initial-state $\leq T$
and *composition-path-bound:*
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
adversary-initial-state $\leq C$
shows
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq$
staged-phase-relation-error size

(staged-query-search-queries budgets $i + 1$) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$) + $T + C$
proof –
have residual-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
 (checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
 trace-default composition-default i)
 adversary-initial-state $\leq$
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$)
by (rule
 checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-alpha-prefix-union
 [OF false-statement wf controlled i -bound default-prefix-clean])
have bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$) + $T + C$
by (rule
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-resi
 [OF wf controlled i -bound residual-bound trace-path-bound
 composition-path-bound])
show ?thesis
 using bound **by** simp
qed

lemma checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-bound-from-conceptual-default-alpha-
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i -bound: $i < \text{rounds}$
and default-alpha-clean:
 $\bigwedge \text{prefix prefix-state.}$
 Some ((prefix, prefix-state), prefix-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-with-state $A \ i$)
 adversary-initial-state) $\implies$
 trace-table-low-degree
 (query-prefix-trace-conceptual-table-with-default prefix
 prefix-state trace-table) $\wedge$
 composition-table-low-degree maxDegree

(query-prefix-composition-conceptual-table-with-default prefix
 prefix-state composition-table) $\wedge$
 sqp-alphas prefix $\notin$ composition-trace-bad-alpha-space
 (query-prefix-trace-conceptual-table-with-default prefix
 prefix-state trace-table)
and trace-path-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
 adversary-initial-state $\leq T$
and composition-path-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
 adversary-initial-state $\leq C$
shows
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
 trace-table composition-table i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets i + 1) + T + C
proof –
have residual-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
 trace-table composition-table i)
 adversary-initial-state $\leq$
 hash-collision-budget-value 0
 (staged-query-search-queries budgets i + 1)
by (rule
 checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-alpha-clean
 [OF false-statement wf controlled i-bound default-alpha-clean])
have bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
 trace-table composition-table i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets i + 1) + T + C

by (rule
 checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-bound-from-conceptual-default-residual-a
 [*OF wf controlled i-bound residual-bound trace-path-bound*
 composition-path-bound])
 show ?thesis
 using bound by simp
 qed

lemma *staged-security-with-data-state-current-query-partial-candidate-hit-at-bound-from-conceptual-default-alpha-clean*
assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < \text{rounds}$
and *default-alpha-clean*:
 $\bigwedge \text{prefix prefix-state.}$
 Some ((*prefix*, *prefix-state*), *prefix-state*) $\in$
 set-dist
 (*execute* (*checked-staged-query-prefix-with-state A i*)
 adversary-initial-state) $\implies$
 trace-table-low-degree
 (*query-prefix-trace-conceptual-table-with-default prefix*
 prefix-state trace-table) $\wedge$
 composition-table-low-degree maxDegree
 (*query-prefix-composition-conceptual-table-with-default prefix*
 prefix-state composition-table) $\wedge$
 sqp-alphas prefix $\notin$ *composition-trace-bad-alpha-space*
 (*query-prefix-trace-conceptual-table-with-default prefix*
 prefix-state trace-table)
and *trace-path-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i*)
 adversary-initial-state $\leq T$
and *composition-path-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i*)
 adversary-initial-state $\leq C$
shows
 wp-event
 (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-current-query-partial-candidate-hit-at*
 trace-table composition-table i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
 query-error-bound +
 hash-collision-budget-value 0
 (*staged-query-search-queries budgets i + 1*) + $T + C$

proof –

have *prefix-bound*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)

(*checked-staged-security-with-query-prefix-authenticated-candidate-hit-at*
trace-table composition-table i)

adversary-initial-state $\leq$

staged-phase-relation-error size

(*staged-query-search-queries budgets i + 1*) +

query-error-bound +

hash-collision-budget-value 0

(*staged-query-search-queries budgets i + 1*) + *T + C*

by (*rule*

checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-bound-from-conceptual-default-alpha-clean

[*OF false-statement wf controlled i-bound default-alpha-clean*

trace-path-bound composition-path-bound])

show *?thesis*

by (*rule order-trans*

[*OF*

staged-security-with-data-state-current-query-partial-candidate-hit-at-le-query-prefix-authenticated-candidate
prefix-bound])

(*use wf controlled i-bound in simp-all*)

qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-*

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *default-alpha-clean*:

$\bigwedge i$ *prefix prefix-state. i < rounds* $\implies$

Some ((prefix, prefix-state), prefix-state) $\in$

set-dist

(*execute (checked-staged-query-prefix-with-state A i)*

adversary-initial-state) $\implies$

trace-table-low-degree

(*query-prefix-trace-conceptual-table-with-default prefix*

prefix-state trace-default) $\wedge$

composition-table-low-degree maxDegree

(*query-prefix-composition-conceptual-table-with-default prefix*

prefix-state composition-default) $\wedge$

sqp-alphas prefix $\notin$ composition-trace-bad-alpha-space

(*query-prefix-trace-conceptual-table-with-default prefix*

prefix-state trace-default)

and *trace-path-bound*:

$\bigwedge i. i < rounds \implies$

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i*)

adversary-initial-state $\leq T i$

and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit}$
 $\text{adversary-initial-state} \leq$
 $(\sum i < \text{rounds}.$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $T \ i + C \ i)$
proof (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-curr}$
 $[\text{OF wf controlled}])$
fix i
assume $i\text{-bound}: i < \text{rounds}$
show
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $T \ i + C \ i$
by (rule
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-alpha}$
 $[\text{OF false-statement wf controlled } i\text{-bound} - \text{trace-path-bound}[\text{OF } i\text{-bound}]$
 $\text{composition-path-bound}[\text{OF } i\text{-bound}]]])$
 $(\text{rule default-alpha-clean}[\text{OF } i\text{-bound}])$
qed

theorem *stark-soundness-from-aligned-partial-query-prefix-conceptual-default-alpha-clean-structured-paths*:
fixes $\text{trace-fri-error}' \ \text{composition-fri-error}' \ N :: \text{prob}$
and $T \ C :: \text{nat} \Rightarrow \text{prob}$
and $\text{trace-default composition-default} :: 'f \ \text{list}$
assumes $\text{false-statement}: \neg \text{exists-valid-trace}$
and $\text{wf}: \text{staged-budget-wellformed budgets}$
and $\text{controlled}: \text{staged-adversary-controlled budgets } A$
and trace-fri-bound :

$wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $\ (staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\ \ \ \ trace\text{-}fri\text{-}bad\text{-}with\text{-}partial\text{-}candidates)$
 $\ \ \ \ adversary\text{-}initial\text{-}state \leq \ trace\text{-}fri\text{-}error'$
and $comp\text{-}fri\text{-}bound$:
 $wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $\ (staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\ \ \ \ composition\text{-}fri\text{-}bad\text{-}with\text{-}partial\text{-}candidates)$
 $\ \ \ \ adversary\text{-}initial\text{-}state \leq \ composition\text{-}fri\text{-}error'$
and $default\text{-}\alpha\text{-}clean$:
 $\bigwedge i \ prefix \ prefix\text{-}state.$
 $\ i < rounds \implies$
 $\ \ Some \ ((prefix, \ prefix\text{-}state), \ prefix\text{-}state) \in$
 $\ \ \ \ set\text{-}dist$
 $\ \ \ \ (execute \ (checked\text{-}staged\text{-}query\text{-}prefix\text{-}with\text{-}state \ A \ i)$
 $\ \ \ \ \ \ adversary\text{-}initial\text{-}state) \implies$
 $\ \ \ trace\text{-}table\text{-}low\text{-}degree$
 $\ \ \ \ (query\text{-}prefix\text{-}trace\text{-}conceptual\text{-}table\text{-}with\text{-}default \ prefix$
 $\ \ \ \ \ \ prefix\text{-}state \ trace\text{-}default) \wedge$
 $\ \ \ composition\text{-}table\text{-}low\text{-}degree \ maxDegree$
 $\ \ \ \ (query\text{-}prefix\text{-}composition\text{-}conceptual\text{-}table\text{-}with\text{-}default \ prefix$
 $\ \ \ \ \ \ prefix\text{-}state \ composition\text{-}default) \wedge$
 $\ \ \ sqp\text{-}\alpha\text{-}space \ prefix \notin \ composition\text{-}trace\text{-}bad\text{-}\alpha\text{-}space$
 $\ \ \ \ (query\text{-}prefix\text{-}trace\text{-}conceptual\text{-}table\text{-}with\text{-}default \ prefix$
 $\ \ \ \ \ \ prefix\text{-}state \ trace\text{-}default)$
and $trace\text{-}path\text{-}bound$:
 $\bigwedge i. \ i < rounds \implies$
 $\ \ wp\text{-}event$
 $\ \ \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}query\text{-}prefix\text{-}data\text{-}state \ A \ i)$
 $\ \ \ (checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}current\text{-}trace\text{-}path\text{-}output\text{-}hit\text{-}at \ i)$
 $\ \ \ adversary\text{-}initial\text{-}state \leq \ T \ i$
and $composition\text{-}path\text{-}bound$:
 $\bigwedge i. \ i < rounds \implies$
 $\ \ wp\text{-}event$
 $\ \ \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}query\text{-}prefix\text{-}data\text{-}state \ A \ i)$
 $\ \ \ (checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}current\text{-}composition\text{-}path\text{-}output\text{-}hit\text{-}at \ i)$
 $\ \ \ adversary\text{-}initial\text{-}state \leq \ C \ i$
and $not\text{-}prefix\text{-}bound$:
 $\ wp\text{-}event$
 $\ \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}data\text{-}state$
 $\ \ \ A)$
 $\ \ (checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}partial\text{-}header\text{-}candidate\text{-}not\text{-}prefix\text{-}bound$
 $\ \ \ \ composition\text{-}trace\text{-}bad\text{-}\alpha\text{-}space)$
 $\ \ \ adversary\text{-}initial\text{-}state \leq \ N$
and $empty\text{-}trace\text{-}fri\text{-}bound$:
 $wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $\ (staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\ \ \ \ trace\text{-}fri\text{-}bad\text{-}with\text{-}empty\text{-}composition\text{-}header\text{-}candidates)$
 $\ \ \ \ adversary\text{-}initial\text{-}state \leq \ trace\text{-}fri\text{-}error$

and *empty-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *nnreal rounds * query-error-bound*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
N +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
 $(\sum i < \text{rounds}.$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1) +
T i + C i) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
N +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nnreal rounds * query-error-bound)*
proof (*rule stark-soundness-from-aligned-partial-query-prefix-current-rounds*
 $[OF \text{ false-statement wf controlled trace-fri-bound comp-fri-bound - not-prefix-bound}$
 $\text{empty-trace-fri-bound empty-query-bound}]$)
fix *i*
assume *i-bound: i < rounds*
show
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1) +

$T\ i + C\ i$
by (rule
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-alpha
 $[OF\ \text{false-statement}\ wf\ \text{controlled}\ i\text{-bound} - \text{trace-path-bound}[OF\ i\text{-bound}]$
 $\text{composition-path-bound}[OF\ i\text{-bound}]]$)
(rule *default-alpha-clean*[$OF\ i\text{-bound}$])
qed

theorem *stark-soundness-from-aligned-partial-query-prefix-conceptual-default-alpha-prefix-union-clean-structured-pat*
fixes *trace-fri-error'* *composition-fri-error'* $N :: \text{prob}$
and $T\ C :: \text{nat} \Rightarrow \text{prob}$
and *trace-default* *composition-default* $:: 'f\ \text{list}$
assumes *false-statement*: $\neg\ \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* A
and *trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*
and *default-prefix-clean*:
 $\bigwedge i\ \text{prefix}\ \text{prefix-state}.$
 $i < \text{rounds} \implies$
Some ((*prefix*, *prefix-state*), *prefix-state*) $\in$
set-dist
(*execute* (*checked-staged-query-prefix-with-state* $A\ i$)
adversary-initial-state) $\implies$
query-prefix-trace-conceptual-table-with-default *prefix* *prefix-state*
trace-default = *trace-default* $\wedge$
query-prefix-composition-conceptual-table-with-default *prefix*
prefix-state *composition-default* = *composition-default* $\wedge$
trace-table-low-degree *trace-default* $\wedge$
composition-table-low-degree *maxDegree* *composition-default* $\wedge$
trace-default $\in$
alpha-prefix-trace-table-candidates *prefix-state*
(*sqp-trace-root* *prefix*) $\wedge$
sqp-alphas *prefix* $\notin$
alpha-prefix-union-bad-sets *prefix-state*
composition-trace-bad-alpha-space (*sqp-trace-root* *prefix*)
and *trace-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state* $A\ i$)
(*checked-staged-security-with-query-prefix-current-trace-path-output-hit-at* i)

$\text{adversary-initial-state} \leq T \ i$
and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *not-prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows *checked-staged-adversary-acceptance-probability* $A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}.$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $T \ i + C \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$

```

    N +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    trace-fri-error + nreal rounds * query-error-bound)
proof (rule stark-soundness-from-aligned-partial-query-prefix-current-rounds
  [OF false-statement wf controlled trace-fri-bound comp-fri-bound - not-prefix-bound
    empty-trace-fri-bound empty-query-bound])
fix i
assume i-bound: i < rounds
show
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  query-error-bound +
  hash-collision-budget-value 0
  (staged-query-search-queries budgets i + 1) +
  T i + C i
by (rule
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-alpha-clean
  [OF false-statement wf controlled i-bound - trace-path-bound[OF i-bound]
    composition-path-bound[OF i-bound]])
  (rule default-prefix-clean[OF i-bound])
qed

end

end

```

theory Soundness-Staged-Aligned-Current-Empty

imports

Soundness-Conceptual-Query-Current-Alpha-Clean

Soundness-Staged-Aligned-Partial-Merkle

begin

Packaging for the aligned-transcript partial-candidate path with the current-empty query bound. This combines the current-query non-empty branch with the empty-composition branch introduced in the current-empty query layer, using a refined empty-header query bound in this downstream route.

context soundness

begin

definition checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift

$$:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$$

$$'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$$

$$'f \text{ protocol-channel}) \times \text{unit list}) \times$$

'f protocol-channel) option $\Rightarrow$ bool
where
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
out $\longleftrightarrow$
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space out $\wedge$
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
composition-randomization-bad-with-partial-candidates out $\vee$
checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-alpha-partial-opening-union-hit-with-empty-header out)
lemma *checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-imp-prefix-prequery-or-relev*
assumes *support*:
out $\in$
set-dist
 (*execute*
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
adversary-initial-state)
and *bad*:
checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-randomization-bad-with-partial-candidates out
shows
checked-staged-security-with-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space out $\vee$
checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
composition-trace-bad-alpha-space out $\vee$
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
out
proof –
have *partial*:
checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
composition-trace-bad-alpha-space out
by (rule
checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-imp-partial-header-bad-set-hit-o
 [OF *support bad*])
have *fresh-or-prequery*:
checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
composition-trace-bad-alpha-space out $\vee$
checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
composition-trace-bad-alpha-space out
by (rule
checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-imp-fresh-or-prequery
 [OF *partial*])
then show *?thesis*
proof
assume *fresh*:
checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
composition-trace-bad-alpha-space out

```

have prefix-or-drift:
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out  $\vee$ 
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
      composition-trace-bad-alpha-space out
by (rule
  checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-imp-prefix-or-drift
    [OF fresh])
then show ?thesis
proof
  assume
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
      composition-trace-bad-alpha-space out
  then show ?thesis by simp
next
  assume drift:
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
      composition-trace-bad-alpha-space out
  have
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
      out
  unfolding
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-def
  using drift bad by simp
  then show ?thesis by simp
qed
next
  assume
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
      composition-trace-bad-alpha-space out
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-imp-prefix-prequery-or-relevant-drift-
assumes support:
  out  $\in$ 
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-alpha-partial-opening-union-hit-with-empty-header out
shows
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out  $\vee$ 
  checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit

```

```

    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
    out
proof —
  have partial:
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
    composition-trace-bad-alpha-space out
  by (rule
    checked-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-imp-partial-header-bad-set-hit-on-
    [OF support hit])
  have fresh-or-prequery:
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space out
  by (rule
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-imp-fresh-or-prequery
    [OF partial])
  then show ?thesis
proof
  assume fresh:
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
    composition-trace-bad-alpha-space out
  have prefix-or-drift:
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
    composition-trace-bad-alpha-space out
  by (rule
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-imp-prefix-or-drift
    [OF fresh])
  then show ?thesis
proof
  assume
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out
  then show ?thesis by simp
next
  assume drift:
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
    composition-trace-bad-alpha-space out
  have
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
    out
  unfolding
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-def
  using drift hit by simp
  then show ?thesis by simp
qed

```



```

next
  assume
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space out
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-bound-from-relevant-drift-an
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and drift-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
    adversary-initial-state ≤ D
  shows
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-randomization-bad-with-partial-candidates)
    adversary-initial-state ≤
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D
  proof -
    let ?M =
      checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
    let ?Prefix =
      checked-staged-security-with-actual-alpha-prefix-bad-set-hit
      composition-trace-bad-alpha-space
    let ?Prequery =
      checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
      composition-trace-bad-alpha-space
    let ?Drift =
      checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
    let ?Random =
      checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-randomization-bad-with-partial-candidates
    let ?P =
      composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +

```

```

    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0)
  let ?Q =
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  have transcript-bound:
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?P
  by (rule
      checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
      [OF wf controlled])
  have prefix-bound:
    wp-event ?M ?Prefix adversary-initial-state ≤ ?P
  by (rule
      checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
      [OF transcript-bound])
  have prequery-bound:
    wp-event ?M ?Prequery adversary-initial-state ≤ ?Q
  by (rule
      checked-staged-security-with-actual-alpha-prefix-vector-alpha-prequery-accounting
      [OF wf controlled])
  have wp-event ?M ?Random adversary-initial-state ≤
    wp-event ?M
    (λout. ?Prefix out ∨ ?Prequery out ∨ ?Drift out ∨ False)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (use
   checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-imp-prefix-prequery-or-relevant-
   [of - A] in blast)
  also have ... ≤
    wp-event ?M ?Prefix adversary-initial-state +
    wp-event ?M ?Prequery adversary-initial-state +
    wp-event ?M ?Drift adversary-initial-state +
    wp-event ?M (λ-. False) adversary-initial-state
  by (rule wp-event-union-bound4)
  also have ... ≤ ?P + ?Q + D + 0
  by (intro add-mono prefix-bound prequery-bound drift-bound)
  (simp add: wp-event-def wp-def dist-expect-def)
  finally show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-bound-from-relevant-drift-and-budget.
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and drift-bound:
    wp-event

```

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq D$

shows

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-alpha-partial-opening-union-hit-with-empty-header)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D

proof –

let *?M* =
checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let *?Prefix* =
checked-staged-security-with-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space
let *?Prequery* =
checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
composition-trace-bad-alpha-space
let *?Drift* =
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
let *?Hit* =
checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-alpha-partial-opening-union-hit-with-empty-header
let *?P* =
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0)
let *?Q* =
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)
have *transcript-bound*:
wp-event (checked-staged-transcript-with-alpha-prefix-program A)
(checked-staged-transcript-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state $\leq ?P$
by (rule
checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
[OF wf controlled])

```

have prefix-bound:
  wp-event ?M ?Prefix adversary-initial-state  $\leq$  ?P
  by (rule
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
    [OF transcript-bound])
have prequery-bound:
  wp-event ?M ?Prequery adversary-initial-state  $\leq$  ?Q
  by (rule
    checked-staged-security-with-actual-alpha-prefix-vector-alpha-prequery-accounting
    [OF wf controlled])
have wp-event ?M ?Hit adversary-initial-state  $\leq$ 
  wp-event ?M
  ( $\lambda$ out. ?Prefix out  $\vee$  ?Prequery out  $\vee$  ?Drift out  $\vee$  False)
  adversary-initial-state
  by (rule wp-event-mono-on-support)
  (use
    checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-imp-prefix-prequery-or-relevant-drift-on-
    [of - A] in blast)
also have ...  $\leq$ 
  wp-event ?M ?Prefix adversary-initial-state +
  wp-event ?M ?Prequery adversary-initial-state +
  wp-event ?M ?Drift adversary-initial-state +
  wp-event ?M ( $\lambda$ -. False) adversary-initial-state
  by (rule wp-event-union-bound4)
also have ...  $\leq$  ?P + ?Q + D + 0
  by (intro add-mono prefix-bound prequery-bound drift-bound)
  (simp add: wp-event-def wp-def dist-expect-def)
finally show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-relevant-drift-and-budg
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and drift-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
   A)
  checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
  adversary-initial-state  $\leq$  D
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   composition-bad-with-partial-candidates)
  adversary-initial-state  $\leq$ 
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +

```

```

    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?degree = ?E composition-degree-bad-with-partial-candidates
  let ?random = ?E composition-randomization-bad-with-partial-candidates
  let ?bad = ?E composition-bad-with-partial-candidates
  have degree-false: ?degree = (λout. False)
    by (rule ext)
      (auto simp: staged-security-with-data-state-verifier-event-def
        composition-degree-bad-with-partial-candidates-false
        split: option.splits prod.splits)
  have degree-bound:
    wp-event ?M ?degree adversary-initial-state ≤ 0
    unfolding degree-false wp-event-def wp-def dist-expect-def by simp
  have projected:
    wp-event ?M ?random adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-verifier-event
        composition-randomization-bad-with-partial-candidates)
      adversary-initial-state
    by (rule
      checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
  have random-bound:
    wp-event ?M ?random adversary-initial-state ≤
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D
    unfolding projected
    by (rule
      checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-bound-from-relevant-drift-and-b
      [OF wf controlled drift-bound])
  have wp-event ?M ?bad adversary-initial-state ≤
    wp-event ?M (λout. ?degree out ∨ ?random out)
      adversary-initial-state
    by (rule wp-event-mono)
      (auto simp: staged-security-with-data-state-verifier-event-def
        dest: composition-bad-with-partial-candidates-split
        [OF false-statement])

```

```

    split: option.splits prod.splits)
  also have ... ≤
    wp-event ?M ?degree adversary-initial-state +
    wp-event ?M ?random adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤
    0 +
    (composition-error-bound +
     staged-phase-relation-error size
     (staged-alpha-search-queries budgets 0) +
     hash-collision-budget-value 0
     (staged-alpha-search-queries budgets 0) +
     staged-phase-relation-error size
     (staged-alpha-search-queries budgets 0) +
     D)
  by (intro add-mono degree-bound random-bound)
  finally show ?thesis
  by simp
qed

lemma checked-staged-security-with-data-state-empty-header-composition-randomization-bound-from-relevant-drift-a
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and drift-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
       A)
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
    adversary-initial-state ≤ D
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       composition-randomization-bad-with-empty-composition-header-candidates)
    adversary-initial-state ≤
      composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      D
  proof -
    let ?M = checked-staged-security-experiment-with-data-state A
    let ?random =
      staged-security-with-data-state-verifier-event
      composition-randomization-bad-with-empty-composition-header-candidates
    let ?alpha =
      staged-security-with-data-state-verifier-event

```

```

    composition-alpha-partial-opening-union-hit-with-empty-header
have alpha-projected:
  wp-event ?M ?alpha adversary-initial-state =
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-alpha-partial-opening-union-hit-with-empty-header)
    adversary-initial-state
  by (rule
    checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
have alpha-bound:
  wp-event ?M ?alpha adversary-initial-state ≤
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D
  unfolding alpha-projected
  by (rule
    checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-bound-from-relevant-drift-and-budgets
    [OF wf controlled drift-bound])
have wp-event ?M ?random adversary-initial-state ≤
  wp-event ?M ?alpha adversary-initial-state
  by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
    intro:
      composition-alpha-bad-set-hit-with-empty-header-imp-partial-opening-union-hit
      composition-randomization-bad-with-empty-composition-header-candidates-imp-alpha-hit
      split: option.splits prod.splits)
also have ... ≤
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  D
  by (rule alpha-bound)
finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-empty-header-bad-event-bound-from-relevant-drift-trace-fri-and-query
  fixes trace-fri-error' :: prob
  assumes false-statement: ¬ exists-valid-trace
  and wf: staged-budget-wellformed budgets

```

and *controlled: staged-adversary-controlled budgets A*
and *drift-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state $\leq D$
and *trace-fri-bound:*
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq \text{trace-fri-error}'$
and *query-bound:*
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq \text{empty-query-error}$
shows
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 soundness-bad-event-partial-candidate-empty-header)
 adversary-initial-state $\leq$
 composition-error-bound +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 D + trace-fri-error' + empty-query-error
proof –
let *?M* = *checked-staged-security-experiment-with-data-state A*
let *?E* = *staged-security-with-data-state-verifier-event*
let *?degree* =
 ?E composition-degree-bad-with-empty-composition-header-candidates
let *?random* =
 ?E composition-randomization-bad-with-empty-composition-header-candidates
let *?trace* =
 ?E trace-fri-bad-with-empty-composition-header-candidates
let *?query* =
 ?E query-bad-with-empty-composition-header-candidates
let *?bad* = *?E soundness-bad-event-partial-candidate-empty-header*
have *degree-false: ?degree = (λ out. False)*
 by (*rule ext*)
 (*auto simp: staged-security-with-data-state-verifier-event-def*
 composition-degree-bad-with-empty-composition-header-candidates-false
 split: option.splits prod.splits)
have *degree-bound:*
 wp-event ?M ?degree adversary-initial-state ≤ 0


```

unfolding degree-false wp-event-def wp-def dist-expect-def by simp
have random-bound:
  wp-event ?M ?random adversary-initial-state ≤
    composition-error-bound +
    staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
    D
by (rule
  checked-staged-security-with-data-state-empty-header-composition-randomization-bound-from-relevant-drift-and-l
  [OF wf controlled drift-bound])
have wp-event ?M ?bad adversary-initial-state ≤
  wp-event ?M
    ( $\lambda out. ?degree\ out \vee ?random\ out \vee ?trace\ out \vee$ 
       $?query\ out$ )
    adversary-initial-state
proof (rule wp-event-mono)
fix out
assume bad:  $?bad\ out$ 
show  $?degree\ out \vee ?random\ out \vee ?trace\ out \vee ?query\ out$ 
proof (cases out)
  case None
  then show  $?thesis$ 
    using bad
    unfolding staged-security-with-data-state-verifier-event-def
    by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  let  $?s =$ 
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have local-bad:
    soundness-bad-event-partial-candidate-empty-header ?s
    (Some (result, final-state))
    using bad
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
  then consider
    (comp)
      composition-bad-with-empty-composition-header-candidates ?s
      (Some (result, final-state))
    | (trace)
      trace-fri-bad-with-empty-composition-header-candidates ?s

```

```

      (Some (result, final-state))
| (query)
  query-bad-with-empty-composition-header-candidates ?s
  (Some (result, final-state))
unfolding soundness-bad-event-partial-candidate-empty-header-def
by blast
then show ?thesis
proof cases
  case comp
  then have comp-split:
    composition-degree-bad-with-empty-composition-header-candidates ?s
    (Some (result, final-state)) ∨
    composition-randomization-bad-with-empty-composition-header-candidates
    ?s (Some (result, final-state))
  by (rule
    composition-bad-with-empty-composition-header-candidates-split
    [OF false-statement])
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    using comp-split by (auto split: option.splits prod.splits)
next
  case trace
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
next
  case query
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
qed
also have ... ≤
  wp-event ?M ?degree adversary-initial-state +
  wp-event ?M ?random adversary-initial-state +
  wp-event ?M ?trace adversary-initial-state +
  wp-event ?M ?query adversary-initial-state
by (rule wp-event-union-bound4)
also have ... ≤
  0 +
  (composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D) +

```

$\text{trace-fri-error}' + \text{empty-query-error}$
by (intro add-mono degree-bound random-bound trace-fri-bound query-bound)
finally show ?thesis
by (simp add: algebra-simps)
qed

lemma checked-staged-soundness-from-aligned-partial-candidate-components-and-current-empty-query-from-relevant-

fixes trace-fri-error' composition-fri-error' query-error D

empty-query-error :: prob

assumes false-statement: $\neg$ exists-valid-trace

and wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and trace-fri-bound:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

trace-fri-bad-with-partial-candidates)

adversary-initial-state $\leq$ trace-fri-error'

and comp-fri-bound:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

composition-fri-bad-with-partial-candidates)

adversary-initial-state $\leq$ composition-fri-error'

and query-bound:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

query-bad-with-aligned-partial-candidates)

adversary-initial-state $\leq$ query-error

and drift-bound:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

A)

checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift

adversary-initial-state $\leq$ D

and empty-trace-fri-bound:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

trace-fri-bad-with-empty-composition-header-candidates)

adversary-initial-state $\leq$ trace-fri-error

and current-empty-bound:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-current-empty-query-opening-hit-at 0)

adversary-initial-state $\leq$ empty-query-error

shows checked-staged-adversary-acceptance-probability A $\leq$

hash-collision-budget-value 0

(staged-attacker-query-budget budgets +

staged-challenge-query-budget + verifier-hash-query-budget) +

(composition-error-bound +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D) +
trace-fri-error' + composition-fri-error' +
query-error +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D + trace-fri-error + empty-query-error)
proof –
have *comp-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-bad-with-partial-candidates)
adversary-initial-state ≤
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D
by (rule
checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-relevant-drift-and-budgets
[OF false-statement wf controlled drift-bound])
have *merkle-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state ≤
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[OF wf controlled])
have *empty-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
adversary-initial-state ≤ empty-query-error
by (rule *order-trans*[

```

    OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero[
      OF wf controlled] current-empty-bound])
have empty-header-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      soundness-bad-event-partial-candidate-empty-header)
  adversary-initial-state ≤
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  D + trace-fri-error + empty-query-error
by (rule
  checked-staged-security-with-data-state-empty-header-bad-event-bound-from-relevant-drift-trace-fri-and-query
    [OF false-statement wf controlled drift-bound empty-trace-fri-bound
      empty-query-bound])
show ?thesis
by (rule
  checked-staged-soundness-from-aligned-partial-candidate-components-and-empty-header-bound
    [OF false-statement merkle-bound comp-bound trace-fri-bound
      comp-fri-bound query-bound empty-header-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query:
fixes trace-fri-error' composition-fri-error' query-error P D
  empty-query-error :: prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ composition-fri-error'
and query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      query-bad-with-aligned-transcript-partial-candidates)
  adversary-initial-state ≤ query-error
and prefix-bound:
  wp-event

```

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-bad-set-hit composition-trace-bad-alpha-space)
adversary-initial-state $\leq P$
and *drift-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift composition-trace-bad-alpha-space)
adversary-initial-state $\leq D$
and *empty-trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ trace-fri-error
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability A $\leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
query-error +
(P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error)
proof –
have *comp-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-bad-with-partial-candidates)
adversary-initial-state $\leq$
P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)
by *(rule*
checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-prefix-drift-and-budgets
[OF false-statement wf controlled prefix-bound drift-bound])
have *merkle-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event

partial-merkle-inconsistency-bad)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*)
by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[*OF wf controlled*])
have *empty-header-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state $\leq$
P + *D* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
trace-fri-error + *empty-query-error*
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-bound-from-prefix-drift-trace-fri-and-current-empty
[*OF false-statement wf controlled prefix-bound drift-bound*
empty-trace-fri-bound current-empty-bound])
have *bound*:
checked-staged-adversary-acceptance-probability *A* $\leq$
hash-collision-budget-value 0
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
(*P* + *D* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0)) +
trace-fri-error' + *composition-fri-error'* +
query-error +
(*P* + *D* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
trace-fri-error + *empty-query-error*)
by (rule
checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-bound
[*OF false-statement merkle-bound comp-bound trace-fri-bound*
comp-fri-bound query-bound empty-header-bound])
then show ?thesis .
qed

lemma *checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from*
fixes *trace-fri-error'* *composition-fri-error'* *query-error* *D*
empty-query-error :: *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *trace-fri-bound*:

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*
and *query-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq$ *query-error*
and *drift-bound*:
 $wp\text{-}event$
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ D
and *empty-trace-fri-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-current-empty-query-opening-hit-at* 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0)) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 D +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0)) +
trace-fri-error' + *composition-fri-error'* +
query-error +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0)) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +

$D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{empty-query-error})$
proof $-$
let $?P =$
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0)$
have transcript-bound:
 $\text{wp-event } (\text{checked-staged-transcript-with-alpha-prefix-program } A)$
 $(\text{checked-staged-transcript-actual-alpha-prefix-bad-set-hit}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq ?P$
by $(\text{rule}$
 $\text{checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets}$
 $[\text{OF wf controlled}])$
have prefix-bound:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq ?P$
by $(\text{rule}$
 $\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript}$
 $[\text{OF transcript-bound}])$
show $?thesis$
by $(\text{rule}$
 $\text{checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query}$
 $[\text{OF false-statement wf controlled trace-fri-bound comp-fri-bound}$
 $\text{query-bound prefix-bound drift-bound empty-trace-fri-bound}$
 $\text{current-empty-bound}])$
qed

lemma $\text{checked-staged-soundness-from-aligned-partial-candidate-components-and-current-empty-query-from-drift:}$
fixes $\text{trace-fri-error' composition-fri-error' query-error } D$
 $\text{empty-query-error} :: \text{prob}$
assumes $\text{false-statement: } \neg \text{exists-valid-trace}$
and $\text{wf: staged-budget-wellformed budgets}$
and $\text{controlled: staged-adversary-controlled budgets } A$
and trace-fri-bound:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error'}$
and comp-fri-bound:

wp_event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
 $adversary_initial_state \leq composition_fri_error'$
and *query-bound*:
 wp_event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-partial-candidates)
 $adversary_initial_state \leq query_error$
and *drift-bound*:
 wp_event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
composition-trace-bad-alpha-space)
 $adversary_initial_state \leq D$
and *empty-trace-fri-bound*:
 wp_event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
 $adversary_initial_state \leq trace_fri_error$
and *current-empty-bound*:
 wp_event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-current-empty-query-opening-hit-at* 0)
 $adversary_initial_state \leq empty_query_error$
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 D +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0)) +
trace-fri-error' + *composition-fri-error'* +
query-error +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 D +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
trace-fri-error + *empty-query-error*)

proof –

let $?P =$
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0)
have *transcript-bound*:
wp-event (checked-staged-transcript-with-alpha-prefix-program A)
(checked-staged-transcript-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$?P
by (rule
checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
[OF wf controlled])
have *prefix-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$?P
by (rule
checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
[OF transcript-bound])
show *?thesis*
by (rule
checked-staged-soundness-from-aligned-partial-candidate-components-and-current-empty-query
[OF false-statement wf controlled trace-fri-bound comp-fri-bound
query-bound prefix-bound drift-bound empty-trace-fri-bound
current-empty-bound])
qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-drift-curr*
fixes *trace-fri-error' composition-fri-error' query-error D*
empty-query-error :: prob
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ trace-fri-error'
and *comp-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ composition-fri-error'
and *current-query-bound*:

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq$ *query-error*
and *drift-bound*:
 $wp\text{-}event$
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ D
and *empty-trace-fri-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-current-empty-query-opening-hit-at* 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
(*composition-error-bound* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
(*staged-alpha-search-queries* *budgets* 0) +
 D +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0)) +
trace-fri-error' + *composition-fri-error'* +
query-error +
(*composition-error-bound* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
(*staged-alpha-search-queries* *budgets* 0) +
 D +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
trace-fri-error + *empty-query-error*)
proof –
have *query-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq$ *query-error*
by (*rule order-trans*[

```

    OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
      current-query-bound])
  (use wf controlled in simp-all)
show ?thesis
by (rule
  checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-d
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      query-bound drift-bound empty-trace-fri-bound current-empty-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-not-prefix
  fixes trace-fri-error' composition-fri-error' query-error N
  empty-query-error :: prob
  assumes false-statement:  $\neg$  exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-partial-candidates)
      adversary-initial-state  $\leq$  trace-fri-error'
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-partial-candidates)
      adversary-initial-state  $\leq$  composition-fri-error'
  and current-query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-current-query-partial-opening-hit
      adversary-initial-state  $\leq$  query-error
  and not-prefix-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
        composition-trace-bad-alpha-space)
      adversary-initial-state  $\leq$  N
  and empty-trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-empty-composition-header-candidates)
      adversary-initial-state  $\leq$  trace-fri-error
  and current-empty-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
      adversary-initial-state  $\leq$  empty-query-error
  shows checked-staged-adversary-acceptance-probability A  $\leq$ 
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +

```

$$\begin{aligned}
& \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) + \\
& (\text{composition-error-bound} + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \quad \text{hash-collision-budget-value } 0 \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \quad N + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0)) + \\
& \text{trace-fri-error}' + \text{composition-fri-error}' + \\
& \text{query-error} + \\
& (\text{composition-error-bound} + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \quad \text{hash-collision-budget-value } 0 \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \quad N + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \quad \text{trace-fri-error} + \text{empty-query-error})
\end{aligned}$$

proof –

have *query-bound*:

$$\begin{aligned}
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\text{staged-security-with-data-state-verifier-event} \\
& \quad \quad \text{query-bad-with-aligned-transcript-partial-candidates}) \\
& \quad \text{adversary-initial-state} \leq \text{query-error}
\end{aligned}$$

by (rule *order-trans*[
OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
current-query-bound])
 (use *wf controlled in simp-all*)

have *drift-bound*:

$$\begin{aligned}
& \text{wp-event} \\
& \quad (\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state} \\
& \quad \quad A) \\
& \quad (\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift} \\
& \quad \quad \text{composition-trace-bad-alpha-space}) \\
& \quad \text{adversary-initial-state} \leq \\
& \quad N
\end{aligned}$$

proof –

have *wp-event*

$$\begin{aligned}
& \quad (\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state} \\
& \quad \quad A) \\
& \quad (\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift} \\
& \quad \quad \text{composition-trace-bad-alpha-space}) \\
& \quad \text{adversary-initial-state} \leq
\end{aligned}$$

wp-event

$$\begin{aligned}
& \quad (\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state} \\
& \quad \quad A) \\
& \quad (\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}
\end{aligned}$$

```

    composition-trace-bad-alpha-space)
  adversary-initial-state
  by (rule
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-le-not-prefix-bound
    [OF wf controlled])
  also have ... ≤ N
    by (rule not-prefix-bound)
  finally show ?thesis .
qed
have prefix-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
      composition-trace-bad-alpha-space)
  adversary-initial-state ≤
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0)
proof -
  let ?P =
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0)
  have transcript-bound:
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
      (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
        composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?P
  by (rule
    checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
    [OF wf controlled])
  show ?thesis
    by (rule
      checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
      [OF transcript-bound])
qed
show ?thesis
  by (rule
    checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      query-bound prefix-bound drift-bound empty-trace-fri-bound
      current-empty-bound])
qed

```

lemma *checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-drift-cur*
fixes *trace-fri-error'* *composition-fri-error'* *D* *empty-query-error* :: *prob*
and *C* :: *nat* $\Rightarrow$ *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *composition-fri-error'*
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* *i*)
 adversary-initial-state $\leq$ *C* *i*
and *drift-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq$ *D*
and *empty-trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-current-empty-query-opening-hit-at* *0*)
 adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability* *A* $\leq$
 hash-collision-budget-value *0*
 (*staged-attacker-query-budget* *budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
 staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* *0*) +
 hash-collision-budget-value *0*
 (*staged-alpha-search-queries* *budgets* *0*) +
 D +
 staged-phase-relation-error *size*

$(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}. C i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{empty-query-error})$

proof –

have *current-query-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq (\sum i < \text{rounds}. C i)$

by (rule
checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
 $[OF \text{ round-bound wf controlled}]$)

show *?thesis*

by (rule
checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-drift-current
 $[OF \text{ false-statement wf controlled trace-fri-bound comp-fri-bound}$
 $\text{current-query-bound drift-bound empty-trace-fri-bound}$
 $\text{current-empty-bound}]$)

qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-not-prefi*

fixes *trace-fri-error'* *composition-fri-error'* *N* *empty-query-error* :: *prob*

and *C* :: *nat* $\Rightarrow$ *prob*

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$

and *comp-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$

and *round-bound*:

$\bigwedge i. i < \text{rounds} \implies$
wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$

$\text{adversary-initial-state} \leq C \ i$
and *not-prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *current-empty-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows *checked-staged-adversary-acceptance-probability* $A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds. } C \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{empty-query-error})$
proof –
have *current-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-current-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq (\sum i < \text{rounds. } C \ i)$
by (rule
 $\text{checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds}$
 $[\text{OF round-bound wf controlled}])$
show *?thesis*

by (rule
 checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-not-prefix-curre
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound
 current-query-bound not-prefix-bound empty-trace-fri-bound
 current-empty-bound])
 qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-not-prefix-cu
fixes trace-fri-error' composition-fri-error' N empty-query-error :: prob
and Q :: nat $\Rightarrow$ prob
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ trace-fri-error'
and comp-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ composition-fri-error'
and trace-frac:
 $\bigwedge i$ prefix prefix-state.
 $i < \text{rounds} \implies$
 Some ((prefix, prefix-state), prefix-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-with-state A i)
 adversary-initial-state) $\implies$
 nnreal
 (card
 (staged-query-prefix-prefix-authenticated-trace-indices prefix
 prefix-state)) /
 nnreal (card query-sample-space) $\leq$ query-error-bound
and path-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-path-output-hit-at i)
 adversary-initial-state $\leq$ Q i
and not-prefix-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq$ N
and empty-trace-fri-bound:

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-current-empty-query-opening-hit-at* 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 N +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0)) +
trace-fri-error' + *composition-fri-error'* +
 ($\sum i < rounds.$
staged-phase-relation-error *size*
 (*staged-query-search-queries* *budgets* $i + 1$) +
query-error-bound + Q i) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 N +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
trace-fri-error + *empty-query-error*)
proof –
have *round-bound*:
 $\bigwedge i. i < rounds \implies$
 $wp\text{-}event$
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* i)
adversary-initial-state $\leq$
staged-phase-relation-error *size*
 (*staged-query-search-queries* *budgets* $i + 1$) +
query-error-bound + Q i
proof –
fix i
assume *i-bound*: $i < rounds$
have *prefix-hit-bound*:
 $wp\text{-}event$

```

    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + query-error-bound
  by (rule
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-trace-indices
    [OF wf controlled i-bound])
    (use trace-frac[OF i-bound] in blast)
show
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    query-error-bound + Q i
  by (rule
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-path
    [OF wf controlled i-bound prefix-hit-bound path-bound[OF i-bound]])
qed
show ?thesis
  by (rule
    checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-not-prefix-c
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
    round-bound not-prefix-bound empty-trace-fri-bound
    current-empty-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-trace-index-structured-path-bounds-from-n
  fixes trace-fri-error' composition-fri-error' N empty-query-error :: prob
  and T C :: nat ⇒ prob
  assumes false-statement: ¬ exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-partial-candidates)
      adversary-initial-state ≤ trace-fri-error'
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-partial-candidates)
      adversary-initial-state ≤ composition-fri-error'
  and trace-frac:
    ∧ i prefix prefix-state.
    i < rounds ⇒
    Some ((prefix, prefix-state), prefix-state) ∈

```

set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \text{ adversary-initial-state}) \implies$
 nnreal
 $(\text{card}$
 $(\text{staged-query-prefix-prefix-authenticated-trace-indices } \text{prefix}$
 $\text{prefix-state})) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq \text{query-error-bound}$
and trace-path-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T \ i$
and $\text{composition-path-bound:}$
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and not-prefix-bound:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and $\text{empty-trace-fri-bound:}$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and $\text{current-empty-bound:}$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$

$(\sum i < \text{rounds}.$
staged-phase-relation-error size
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + T i + C i) +$
 $(\text{composition-error-bound} +$
staged-phase-relation-error size
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
staged-phase-relation-error size
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{empty-query-error})$

proof –

have *round-bound*:

$\bigwedge i. i < \text{rounds} \implies$

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq$
staged-phase-relation-error size
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + T i + C i$

proof –

fix *i*

assume *i-bound*: $i < \text{rounds}$

have *prefix-hit-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at } i)$
 $\text{adversary-initial-state} \leq$
staged-phase-relation-error size
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$

by (rule
 $\text{checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-trace-indices}$
 $[\text{OF wf controlled } i\text{-bound}])$
 $(\text{use trace-frac}[\text{OF } i\text{-bound}] \text{ in blast})$

show

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq$
staged-phase-relation-error size
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + T i + C i$

by (rule
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-structured-}$
 $[\text{OF wf controlled } i\text{-bound prefix-hit-bound trace-path-bound}[\text{OF } i\text{-bound}]$
 $\text{composition-path-bound}[\text{OF } i\text{-bound}]])$

qed
 show ?thesis
 by (rule
 checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-not-prefix-cv
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound
 round-bound not-prefix-bound empty-trace-fri-bound
 current-empty-bound])

qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-conceptual-structured-paths-from-not-prefix-cv

fixes trace-fri-error' composition-fri-error' N empty-query-error :: prob

and T C :: nat $\Rightarrow$ prob

assumes false-statement: $\neg$ exists-valid-trace

and wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and trace-fri-bound:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

trace-fri-bad-with-partial-candidates)

adversary-initial-state $\leq$ trace-fri-error'

and comp-fri-bound:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

composition-fri-bad-with-partial-candidates)

adversary-initial-state $\leq$ composition-fri-error'

and conceptual-context:

$\bigwedge i$ prefix prefix-state.

$i < \text{rounds} \Rightarrow$

Some ((prefix, prefix-state), prefix-state) $\in$

set-dist

(execute (checked-staged-query-prefix-with-state A i)

adversary-initial-state) $\Rightarrow$

$\neg$ hash-map-output-collision prefix-state $\wedge$

trace-table-low-degree

(query-prefix-trace-conceptual-table prefix prefix-state) $\wedge$

composition-table-low-degree maxDegree

(query-prefix-composition-conceptual-table prefix prefix-state) $\wedge$

$\neg$ all-queries-consistent

(query-prefix-trace-conceptual-table prefix prefix-state)

(query-prefix-composition-conceptual-table prefix prefix-state)

(sqp-alphas prefix)

and trace-path-bound:

$\bigwedge i. i < \text{rounds} \Rightarrow$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)

adversary-initial-state $\leq$ T i

and composition-path-bound:

$\bigwedge i. i < \text{rounds} \Rightarrow$

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
 adversary-initial-state $\leq C$ i
and not-prefix-bound:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq N$
and empty-trace-fri-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ trace-fri-error
and current-empty-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq$ empty-query-error
shows checked-staged-adversary-acceptance-probability $A \leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 N +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)) +
 trace-fri-error' + composition-fri-error' +
 ($\sum i < \text{rounds}$.
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound + T i + C i) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 N +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + empty-query-error)
proof –
have round-bound:
 $\bigwedge i. i < \text{rounds} \implies$

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A\ i$)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound + $T\ i + C\ i$

proof –

fix i

assume i -bound: $i < \text{rounds}$

have prefix-target-bound:

wp-event (checked-staged-query-prefix-receive-with-state $A\ i$)
 (checked-staged-query-prefix-dynamic-index-hit
 staged-query-prefix-prefix-authenticated-opening-target)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) + query-error-bound

by (rule
 checked-staged-query-prefix-prefix-authenticated-target-hit-bound-from-conceptual
 [OF wf controlled i -bound])
 (rule conceptual-context[OF i -bound])

have prefix-bound:

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A\ i$)
 (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) + query-error-bound

by (rule
 checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target
 [OF wf controlled i -bound prefix-target-bound])

show

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A\ i$)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound + $T\ i + C\ i$

by (rule
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-structured-
 [OF wf controlled i -bound prefix-bound trace-path-bound[OF i -bound]
 composition-path-bound[OF i -bound]])

qed

show ?thesis

by (rule
 checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-not-prefix-cu
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound
 round-bound not-prefix-bound empty-trace-fri-bound

current-empty-bound])

qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-cross-data-p*
fixes *trace-fri-error'* *composition-fri-error'* *X P R empty-query-error* :: *prob*
and *Q* :: *nat* $\Rightarrow$ *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *composition-fri-error'*
and *trace-frac*:
 $\bigwedge i$ *prefix prefix-state*.
 i < *rounds* $\Rightarrow$
 Some ((*prefix*, *prefix-state*), *prefix-state*) $\in$
 set-dist
 (*execute* (*checked-staged-query-prefix-with-state A i*)
 adversary-initial-state) $\Rightarrow$
 nnreal
 (*card*
 (*staged-query-prefix-prefix-authenticated-trace-indices prefix*
 prefix-state)) /
 nnreal (*card query-sample-space*) $\leq$ *query-error-bound*
and *path-bound*:
 $\bigwedge i$. *i* < *rounds* $\Rightarrow$
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-path-output-hit-at i*)
 adversary-initial-state $\leq$ *Q i*
and *cross-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
 adversary-initial-state $\leq$ *X*
and *data-pre-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 staged-security-with-data-state-transcript-pre-hit
 adversary-initial-state $\leq$ *P*
and *transcript-new-bound*:
 wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq R$
and *empty-trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ trace-fri-error
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability $A \leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + $X + (P + R) + (P + R)$) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
($\sum i < \text{rounds}$.
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) +
query-error-bound + $Q i$) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + $X + (P + R) + (P + R)$) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error)
proof –
have *transcript-pre-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
adversary-initial-state $\leq P$
by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit

$[OF \text{ data-pre-bound}]$)
have *witness-hit-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
adversary-initial-state $\leq P + R$
by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
 $[OF \text{ transcript-pre-bound transcript-new-bound}]$)
have *no-prefix-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq P + R$
by (rule *order-trans*
 $[OF$
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-le-witness-transcript-hit
witness-hit-bound])
have *path-output-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq P + R$
by (rule *order-trans*
 $[OF$
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-le-witness-transcript-hit
witness-hit-bound])
have *uncovered-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state $\leq X$
by (rule *order-trans*
 $[OF$
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad
cross-bound])
have *collision-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
adversary-initial-state $\leq$
hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0)
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
 $[OF \text{ wf controlled}]$)
have *not-prefix-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound

```

    composition-trace-bad-alpha-space)
  adversary-initial-state ≤
  hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0) +
  X + (P + R) + (P + R)
by (rule
  checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
  [OF wf controlled collision-bound uncovered-bound no-prefix-bound
    path-output-bound])
show ?thesis
by (rule
  checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-not-prefix-current-bound
  [OF false-statement wf controlled trace-fri-bound comp-fri-bound
    trace-frac path-bound not-prefix-bound empty-trace-fri-bound
    current-empty-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-trace-index-structured-path-bounds-from-current-bound
fixes trace-fri-error' composition-fri-error' X P R empty-query-error :: prob
and T C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ composition-fri-error'
and trace-frac:
  ∧ i prefix prefix-state.
  i < rounds ⇒
  Some ((prefix, prefix-state), prefix-state) ∈
  set-dist
  (execute (checked-staged-query-prefix-with-state A i)
    adversary-initial-state) ⇒
  nnreal
  (card
    (staged-query-prefix-prefix-authenticated-trace-indices prefix
      prefix-state)) /
  nnreal (card query-sample-space) ≤ query-error-bound
and trace-path-bound:
  ∧ i. i < rounds ⇒
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)

```

$\text{adversary-initial-state} \leq T \ i$
and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *cross-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad}$
 $\text{adversary-initial-state} \leq X$
and *data-pre-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$
and *transcript-new-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit}$
 $\text{adversary-initial-state} \leq R$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *current-empty-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) + X + (P + R) + (P + R)) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}.$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$

$query-error-bound + T\ i + C\ i) +$
 $(composition-error-bound +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $(hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) + X + (P + R) + (P + R)) +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $trace-fri-error + empty-query-error)$

proof –

have *transcript-pre-bound*:

wp-event
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state\ A)$
 $checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit$
 $adversary-initial-state \leq P$

by (rule
 $checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit$
 $[OF\ data-pre-bound])$

have *witness-hit-bound*:

wp-event
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state\ A)$
 $checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit$
 $adversary-initial-state \leq P + R$

by (rule
 $checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new$
 $[OF\ transcript-pre-bound\ transcript-new-bound])$

have *no-prefix-bound*:

wp-event
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state\ A)$
 $checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate$
 $adversary-initial-state \leq P + R$

by (rule *order-trans*
 $[OF$
 $checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-le-witness-transcript-hit$
 $witness-hit-bound])$

have *path-output-bound*:

wp-event
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state\ A)$
 $checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output$
 $adversary-initial-state \leq P + R$

by (rule *order-trans*
 $[OF$
 $checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-le-witness-transcript-hit$
 $witness-hit-bound])$

have *uncovered-bound*:

wp-event
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state\ A)$

checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state $\leq X$
by (*rule order-trans*
 [*OF*
 checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad
 cross-bound])
have *collision-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
 adversary-initial-state $\leq$
 hash-collision-budget-value 0 (*staged-alpha-search-queries budgets 0*)
by (*rule*
 checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
 [*OF wf controlled*])
have *not-prefix-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq$
 hash-collision-budget-value 0 (*staged-alpha-search-queries budgets 0*) +
 X + (P + R) + (P + R)
by (*rule*
 checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
 [*OF wf controlled collision-bound uncovered-bound no-prefix-bound*
 path-output-bound])
show *?thesis*
by (*rule*
 checked-staged-soundness-from-aligned-transcript-components-and-trace-index-structured-path-bounds-from-not-p
 [*OF false-statement wf controlled trace-fri-bound comp-fri-bound*
 trace-frac trace-path-bound composition-path-bound not-prefix-bound
 empty-trace-fri-bound current-empty-bound])
qed

theorem *checked-staged-soundness-from-aligned-transcript-conceptual-default-alpha-prefix-union-clean-structured-pat*
fixes *trace-fri-error'* *composition-fri-error'* *N* *empty-query-error* :: *prob*
and *T C* :: *nat* $\Rightarrow$ *prob*
and *trace-default* *composition-default* :: '*f list*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)

*(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state ≤ composition-fri-error'*
and *default-prefix-clean:*
 $\bigwedge i$ *prefix prefix-state.*
 $i < \text{rounds} \implies$
Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) ⇒
query-prefix-trace-conceptual-table-with-default prefix prefix-state
trace-default = trace-default ∧
query-prefix-composition-conceptual-table-with-default prefix
prefix-state composition-default = composition-default ∧
trace-table-low-degree trace-default ∧
composition-table-low-degree maxDegree composition-default ∧
trace-default ∈
alpha-prefix-trace-table-candidates prefix-state
(spp-trace-root prefix) ∧
spp-alphas prefix ∉
alpha-prefix-union-bad-sets prefix-state
composition-trace-bad-alpha-space (spp-trace-root prefix)
and *trace-path-bound:*
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
adversary-initial-state ≤ T i
and *composition-path-bound:*
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
adversary-initial-state ≤ C i
and *not-prefix-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)
adversary-initial-state ≤ N
and *empty-trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state ≤ trace-fri-error
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)

adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
N +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0)) +
trace-fri-error' + *composition-fri-error'* +
 ($\sum i < \text{rounds}$.
staged-phase-relation-error *size*
 (*staged-query-search-queries* *budgets* $i + 1$) +
query-error-bound +
hash-collision-budget-value 0
 (*staged-query-search-queries* *budgets* $i + 1$) +
T i + *C i*) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
N +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
trace-fri-error + *empty-query-error*)
proof (*rule*
checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-not-prefix-curr
 [*OF false-statement* *wf controlled trace-fri-bound comp-fri-bound - not-prefix-bound*
empty-trace-fri-bound current-empty-bound])
fix *i*
assume *i-bound*: $i < \text{rounds}$
show
wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* i)
adversary-initial-state $\leq$
staged-phase-relation-error *size*
 (*staged-query-search-queries* *budgets* $i + 1$) +
query-error-bound +
hash-collision-budget-value 0
 (*staged-query-search-queries* *budgets* $i + 1$) +
T i + *C i*
by (*rule*
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-alpha

$[OF \text{ false-statement } wf \text{ controlled } i\text{-bound} - \text{trace-path-bound}[OF \text{ } i\text{-bound}]$
 $\text{composition-path-bound}[OF \text{ } i\text{-bound}]])$
 $(\text{rule default-prefix-clean}[OF \text{ } i\text{-bound}])$

qed

theorem *checked-staged-soundness-from-aligned-transcript-conceptual-default-alpha-prefix-union-clean-structured-pat*

fixes *trace-fri-error'* *composition-fri-error'* *D* *empty-query-error* :: *prob*

and *T C* :: *nat* $\Rightarrow$ *prob*

and *trace-default* *composition-default* :: '*f* *list*

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets A*

and *trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*

and *comp-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*

and *default-prefix-clean*:

$\bigwedge i$ *prefix* *prefix-state*.
 $i < \text{rounds} \Rightarrow$
Some (*(prefix, prefix-state), prefix-state*) $\in$
set-dist
(execute (*checked-staged-query-prefix-with-state A i*)
adversary-initial-state) $\Rightarrow$
query-prefix-trace-conceptual-table-with-default *prefix* *prefix-state*
trace-default = *trace-default* $\wedge$
query-prefix-composition-conceptual-table-with-default *prefix*
prefix-state *composition-default* = *composition-default* $\wedge$
trace-table-low-degree *trace-default* $\wedge$
composition-table-low-degree *maxDegree* *composition-default* $\wedge$
trace-default $\in$
alpha-prefix-trace-table-candidates *prefix-state*
(sqp-trace-root *prefix*) $\wedge$
sqp-alphas *prefix* $\notin$
alpha-prefix-union-bad-sets *prefix-state*
composition-trace-bad-alpha-space (*sqp-trace-root* *prefix*)

and *trace-path-bound*:

$\bigwedge i$. $i < \text{rounds} \Rightarrow$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
adversary-initial-state $\leq T \ i$

and *composition-path-bound*:

$\bigwedge i$. $i < \text{rounds} \Rightarrow$

$wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}query\text{-}prefix\text{-}data\text{-}state\ A\ i)$
 $(checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}current\text{-}composition\text{-}path\text{-}output\text{-}hit\text{-}at\ i)$
 $adversary\text{-}initial\text{-}state \leq C\ i$
and $drift\text{-}bound$:
 $wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}data\text{-}state\ A)$
 $(checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}partial\text{-}header\text{-}candidate\text{-}drift\ composition\text{-}trace\text{-}bad\text{-}alpha\text{-}space)$
 $adversary\text{-}initial\text{-}state \leq D$
and $empty\text{-}trace\text{-}fri\text{-}bound$:
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event\ trace\text{-}fri\text{-}bad\text{-}with\text{-}empty\text{-}composition\text{-}header\text{-}candidates)$
 $adversary\text{-}initial\text{-}state \leq trace\text{-}fri\text{-}error$
and $current\text{-}empty\text{-}bound$:
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}current\text{-}empty\text{-}query\text{-}opening\text{-}hit\text{-}at\ 0)$
 $adversary\text{-}initial\text{-}state \leq empty\text{-}query\text{-}error$
shows $checked\text{-}staged\text{-}adversary\text{-}acceptance\text{-}probability\ A \leq$
 $hash\text{-}collision\text{-}budget\text{-}value\ 0$
 $(staged\text{-}attacker\text{-}query\text{-}budget\ budgets +$
 $staged\text{-}challenge\text{-}query\text{-}budget + verifier\text{-}hash\text{-}query\text{-}budget) +$
 $(composition\text{-}error\text{-}bound +$
 $staged\text{-}phase\text{-}relation\text{-}error\ size$
 $(staged\text{-}alpha\text{-}search\text{-}queries\ budgets\ 0) +$
 $hash\text{-}collision\text{-}budget\text{-}value\ 0$
 $(staged\text{-}alpha\text{-}search\text{-}queries\ budgets\ 0) +$
 $D +$
 $staged\text{-}phase\text{-}relation\text{-}error\ size$
 $(staged\text{-}alpha\text{-}search\text{-}queries\ budgets\ 0)) +$
 $trace\text{-}fri\text{-}error' + composition\text{-}fri\text{-}error' +$
 $(\sum i < rounds.$
 $staged\text{-}phase\text{-}relation\text{-}error\ size$
 $(staged\text{-}query\text{-}search\text{-}queries\ budgets\ i + 1) +$
 $query\text{-}error\text{-}bound +$
 $hash\text{-}collision\text{-}budget\text{-}value\ 0$
 $(staged\text{-}query\text{-}search\text{-}queries\ budgets\ i + 1) +$
 $T\ i + C\ i) +$
 $(composition\text{-}error\text{-}bound +$
 $staged\text{-}phase\text{-}relation\text{-}error\ size$
 $(staged\text{-}alpha\text{-}search\text{-}queries\ budgets\ 0) +$
 $hash\text{-}collision\text{-}budget\text{-}value\ 0$
 $(staged\text{-}alpha\text{-}search\text{-}queries\ budgets\ 0) +$
 $D +$
 $staged\text{-}phase\text{-}relation\text{-}error\ size$
 $(staged\text{-}alpha\text{-}search\text{-}queries\ budgets\ 0) +$
 $trace\text{-}fri\text{-}error + empty\text{-}query\text{-}error)$

```

proof (rule
  checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-drift-current-e
  [OF false-statement wf controlled trace-fri-bound comp-fri-bound - drift-bound
    empty-trace-fri-bound current-empty-bound])
fix i
assume i-bound: i < rounds
show
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state ≤
    staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
    query-error-bound +
    hash-collision-budget-value 0
      (staged-query-search-queries budgets i + 1) +
    T i + C i
by (rule
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-alpha
  [OF false-statement wf controlled i-bound - trace-path-bound[OF i-bound]
    composition-path-bound[OF i-bound]])
  (rule default-prefix-clean[OF i-bound])
qed

end

end

```

```

theory Soundness-Alpha-Fixed-Candidate
imports Soundness-Conceptual-Query-Current-Alpha-Clean
begin

```

Fixed-trace-table alpha targets.

This layer records the sound probability component that is available when a trace table has already been fixed before the alpha challenge is sampled. It deliberately does not claim that sampled Merkle openings determine such a table.

```

context soundness
begin

```

definition staged-alpha-prefix-fixed-trace-prequery-hit

```

:: 'f list ⇒
  (('f × 'f list × 'f list × 'f) ×
   'f protocol-channel) option ⇒ bool

```

where

```

  staged-alpha-prefix-fixed-trace-prequery-hit trace-table out ⟷
    (case out of
      None ⇒ False
    | Some (_, prefix-state) ⇒

```

($\exists$ *as* $\in$ *composition-trace-bad-alpha-space trace-table*.
alpha-vector-prequeried-in-state prefix-state as
(length spec)))

definition *staged-alpha-prefix-fixed-trace-prequery-relation*

:: 'f staged-adversary $\Rightarrow$ 'f protocol-channel $\Rightarrow$
'f list $\Rightarrow$ 'f protocol-hash-input $\Rightarrow$ 'f $\Rightarrow$ bool

where

staged-alpha-prefix-fixed-trace-prequery-relation A s trace-table x y
 $\longleftrightarrow$
($\exists$ *prefix prefix-state as i*.
Some (prefix, prefix-state) $\in$
set-dist (execute (staged-alpha-prefix-program A) s) $\wedge$
as $\in$ composition-trace-bad-alpha-space trace-table $\wedge$
i < length spec $\wedge$
i < length as $\wedge$
x = alpha-challenge-key-at prefix-state i as $\wedge$
y = as ! i)

definition *checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried*

:: 'f list $\Rightarrow$
((((('f $\times$ 'f list $\times$ 'f list $\times$ 'f) $\times$
'f protocol-channel) $\times$ 'f staged-proof-data) $\times$
'f protocol-channel) option $\Rightarrow$
bool

where

checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried
trace-table out $\longleftrightarrow$
(case out of
None $\Rightarrow$ False
| Some (((prefix, prefix-state), data), -) $\Rightarrow$
staged-alphas data $\in$ composition-trace-bad-alpha-space trace-table $\wedge$
 $\neg$ alpha-vector-prequeried-in-state prefix-state
(staged-alphas data) (length spec))

definition *checked-staged-transcript-fixed-trace-alpha-prequery-hit*

:: 'f list $\Rightarrow$
((((('f $\times$ 'f list $\times$ 'f list $\times$ 'f) $\times$
'f protocol-channel) $\times$ 'f staged-proof-data) $\times$
'f protocol-channel) option $\Rightarrow$
bool

where

checked-staged-transcript-fixed-trace-alpha-prequery-hit trace-table out
 $\longleftrightarrow$
(case out of
None $\Rightarrow$ False
| Some (((prefix, prefix-state), data), -) $\Rightarrow$
staged-alphas data $\in$ composition-trace-bad-alpha-space trace-table $\wedge$
alpha-vector-prequeried-in-state prefix-state

(staged-alphas data) (length spec))

definition *checked-staged-transcript-fixed-trace-alpha-bad*

```

:: 'f list ⇒
  (((('f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) option ⇒
  bool
where
  checked-staged-transcript-fixed-trace-alpha-bad trace-table out
    ↔
    (case out of
      None ⇒ False
    | Some (((prefix, prefix-state), data), -) ⇒
      staged-alphas data ∈ composition-trace-bad-alpha-space trace-table)

```

lemma *staged-alpha-prefix-fixed-trace-prequery-hit-imp-relation-hit:*

```

assumes empty: HashMap s = fmempty
and support:
  Some (prefix, prefix-state) ∈
    set-dist (execute (staged-alpha-prefix-program A) s)
and hit:
  staged-alpha-prefix-fixed-trace-prequery-hit trace-table
    (Some (prefix, prefix-state))
shows
  hash-relation-hit-event
    (staged-alpha-prefix-fixed-trace-prequery-relation A s trace-table)
    s (Some (prefix, prefix-state))

```

proof —

```

from hit obtain as i where member:
  as ∈ composition-trace-bad-alpha-space trace-table
and i-bound: i < length spec
and i-len: i < length as
and lookup:
  fmlookup (HashMap prefix-state)
    (alpha-challenge-key-at prefix-state i as) = Some (as ! i)
unfolding staged-alpha-prefix-fixed-trace-prequery-hit-def
  alpha-vector-prequeried-in-state-def
by auto
have initial-none:
  fmlookup (HashMap s) (alpha-challenge-key-at prefix-state i as) =
    None
using empty by simp
have relation:
  staged-alpha-prefix-fixed-trace-prequery-relation A s trace-table
    (alpha-challenge-key-at prefix-state i as) (as ! i)
unfolding staged-alpha-prefix-fixed-trace-prequery-relation-def
using support member i-bound i-len by blast
have hit-relation:

```



```

    hash-relation-hit
      (staged-alpha-prefix-fixed-trace-prequery-relation A s trace-table)
      s prefix-state
    unfolding hash-relation-hit-def
    using initial-none lookup relation by blast
  show ?thesis
    unfolding hash-relation-hit-event-def using hit-relation by simp
qed

lemma staged-alpha-prefix-fixed-trace-prequery-hit-bound-by-relation-hit:
  assumes empty: HashMap s = fmempty
  shows
    wp-event (staged-alpha-prefix-program A)
      (staged-alpha-prefix-fixed-trace-prequery-hit trace-table) s ≤
    wp-event (staged-alpha-prefix-program A)
      (hash-relation-hit-event
        (staged-alpha-prefix-fixed-trace-prequery-relation A s trace-table)
        s) s
  proof (rule wp-event-mono-on-support)
    fix out
    assume support: out ∈ set-dist (execute (staged-alpha-prefix-program A) s)
    and hit: staged-alpha-prefix-fixed-trace-prequery-hit trace-table out
    show
      hash-relation-hit-event
        (staged-alpha-prefix-fixed-trace-prequery-relation A s trace-table)
        s out
    proof (cases out)
    case None
    then show ?thesis
      using hit unfolding staged-alpha-prefix-fixed-trace-prequery-hit-def
      by simp
    next
    case (Some result)
    then obtain prefix prefix-state where out-eq:
      out = Some (prefix, prefix-state)
      by (cases result) auto
    show ?thesis
      unfolding out-eq
      by (rule staged-alpha-prefix-fixed-trace-prequery-hit-imp-relation-hit
        [OF empty])
        (use support hit out-eq in simp-all)
  qed
qed

lemma staged-alpha-prefix-fixed-trace-prequery-relation-fiber-bound-size:
  card
    {y. staged-alpha-prefix-fixed-trace-prequery-relation A s trace-table x y}
    ≤ size
  proof —

```

have *subset*:
 {*y*. *staged-alpha-prefix-fixed-trace-prequery-relation* *A* *s* *trace-table* *x* *y*}
 $\subseteq$ (*UNIV* :: '*f* *set*)
 by *simp*
have *finite-field*: *finite* (*UNIV* :: '*f* *set*)
 by *simp*
have *card*
 {*y*. *staged-alpha-prefix-fixed-trace-prequery-relation* *A* *s* *trace-table* *x* *y*}
 $\leq$ *card* (*UNIV* :: '*f* *set*)
 by (*rule* *card-mono*[*OF* *finite-field subset*])
also have ... = *size*
 using *size-card* **by** *simp*
finally show ?*thesis* .
qed

lemma *staged-alpha-prefix-fixed-trace-prequery-hit-bound*:
assumes *wf*: *staged-budget-wellformed* *budgets*
 and *controlled*: *staged-adversary-controlled* *budgets* *A*
shows

wp-event (*staged-alpha-prefix-program* *A*)
 (*staged-alpha-prefix-fixed-trace-prequery-hit* *trace-table*)
 adversary-initial-state $\leq$
 staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0)

proof —

have *prehit-le-relation*:
 wp-event (*staged-alpha-prefix-program* *A*)
 (*staged-alpha-prefix-fixed-trace-prequery-hit* *trace-table*)
 adversary-initial-state $\leq$
 wp-event (*staged-alpha-prefix-program* *A*)
 (*hash-relation-hit-event*
 (*staged-alpha-prefix-fixed-trace-prequery-relation* *A*
 adversary-initial-state *trace-table*)
 adversary-initial-state)
 adversary-initial-state
by (*rule* *staged-alpha-prefix-fixed-trace-prequery-hit-bound-by-relation-hit*)
 (*simp* *add*: *adversary-initial-state-def*)
also have ... $\leq$
 staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0)

proof —

have *program*:
 hash-relation-program
 (*staged-alpha-prefix-fixed-trace-prequery-relation* *A*
 adversary-initial-state *trace-table*)
 size (*staged-alpha-search-queries* *budgets* 0)
 (*staged-alpha-prefix-program* *A*)
by (*rule* *hash-relation-program-staged-alpha-prefix-program*
 [*OF* *wf controlled*])

```

    (rule staged-alpha-prefix-fixed-trace-prequery-relation-fiber-bound-size)
  show ?thesis
    using program
    unfolding hash-relation-program-def hash-relation-budget-def
      staged-phase-relation-error-def
    by blast
qed
finally show ?thesis .
qed

lemma checked-staged-after-alpha-prefix-fixed-trace-alpha-bad-excluding-prequeried-bound:
  shows
    wp-event (checked-staged-after-alpha-prefix-program A prefix)
      (λout. case out of
        None ⇒ False
      | Some (data, -) ⇒
        staged-alphas data ∈
          composition-trace-bad-alpha-space trace-table ∧
        ¬ alpha-vector-prequeried-in-state s
          (staged-alphas data) (length spec))
      s ≤ composition-error-bound
  proof -
    let ?B = composition-trace-bad-alpha-space trace-table
    have bound:
      wp-event (checked-staged-after-alpha-prefix-program A prefix)
        (λout. case out of
          None ⇒ False
        | Some (data, -) ⇒
          staged-alphas data ∈ ?B ∧
          ¬ alpha-vector-prequeried-in-state s
            (staged-alphas data) (length spec))
        s ≤ nnreal (card ?B) / nnreal (CARD('f) ^ length spec)
      by (rule
        checked-staged-after-alpha-prefix-program-alpha-list-bound-excluding-prequeried
        [OF composition-trace-bad-alpha-space-subset-alpha-space])
    also have ... ≤ composition-error-bound
      using composition-trace-bad-alpha-space-fraction-bound-alpha-space
        [of trace-table]
      by simp
    finally show ?thesis .
  qed

lemma checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried-bound:
  shows
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
      (checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried
        trace-table)
      adversary-initial-state ≤ composition-error-bound
  unfolding checked-staged-transcript-with-alpha-prefix-program-def

```

```

proof (rule wp-event-bind-bound-by-cont)
  show
     $\neg$  checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried
      trace-table None
  unfolding
    checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried-def
  by simp
next
  fix prefix prefix-state
  assume prefix-out:
    Some (prefix, prefix-state)  $\in$ 
      set-dist
        (execute (staged-alpha-prefix-program A) adversary-initial-state)
  let ?Q =
    checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried
      trace-table
  have get-eq:
    wp-event
      (get  $\gg$ 
        ( $\lambda$ prefix-state'.
          checked-staged-after-alpha-prefix-program A prefix  $\gg$ 
            ( $\lambda$ data. return ((prefix, prefix-state'), data))))
    ?Q prefix-state =
    wp-event
      (checked-staged-after-alpha-prefix-program A prefix  $\gg$ 
        ( $\lambda$ data. return ((prefix, prefix-state), data)))
    ?Q
      prefix-state
  unfolding wp-event-def by (simp add: wpsimps)
  have return-eq:
    wp-event
      (checked-staged-after-alpha-prefix-program A prefix  $\gg$ 
        ( $\lambda$ data. return ((prefix, prefix-state), data)))
    ?Q prefix-state =
    wp-event (checked-staged-after-alpha-prefix-program A prefix)
      ( $\lambda$ out. case out of
        None  $\Rightarrow$  False
        | Some (data, -)  $\Rightarrow$ 
          staged-alphas data  $\in$ 
            composition-trace-bad-alpha-space trace-table  $\wedge$ 
             $\neg$  alpha-vector-prequeried-in-state prefix-state
              (staged-alphas data) (length spec))
      prefix-state
  apply (subst wp-event-bind-return-map)
  unfolding
    checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried-def
  by simp
show
  wp-event

```

(get $\gg$
 (λprefix-state.
 checked-staged-after-alpha-prefix-program A prefix $\gg$
 (λdata. return ((prefix, prefix-state), data))))
 ?Q prefix-state ≤ composition-error-bound
unfolding get-eq return-eq
by (rule
 checked-staged-after-alpha-prefix-fixed-trace-alpha-bad-excluding-prequeried-bound)
qed

lemma checked-staged-transcript-fixed-trace-alpha-prequery-bound:

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

shows

wp-event (checked-staged-transcript-with-alpha-prefix-program A)
 (checked-staged-transcript-fixed-trace-alpha-prequery-hit trace-table)
 adversary-initial-state ≤
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)

unfolding checked-staged-transcript-with-alpha-prefix-program-def

proof (rule wp-event-bind-bound-by-head-event)

show

wp-event (staged-alpha-prefix-program A)
 (staged-alpha-prefix-fixed-trace-prequery-hit trace-table)
 adversary-initial-state
 ≤ staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)
by (rule staged-alpha-prefix-fixed-trace-prequery-hit-bound
 [OF wf controlled])

next

show

checked-staged-transcript-fixed-trace-alpha-prequery-hit trace-table
 None $\implies$
 staged-alpha-prefix-fixed-trace-prequery-hit trace-table None

unfolding checked-staged-transcript-fixed-trace-alpha-prequery-hit-def
 staged-alpha-prefix-fixed-trace-prequery-hit-def

by simp

next

fix prefix prefix-state out

assume prefix-out:

Some (prefix, prefix-state) ∈
 set-dist
 (execute (staged-alpha-prefix-program A) adversary-initial-state)

and cont:

out ∈
 set-dist
 (execute
 (get $\gg$
 (λprefix-state.

```

      checked-staged-after-alpha-prefix-program A prefix  $\gg$ 
      ( $\lambda$ data. return ((prefix, prefix-state), data)))
    prefix-state)
  and hit:
    checked-staged-transcript-fixed-trace-alpha-prequery-hit trace-table
    out
  from hit cont show
    staged-alpha-prefix-fixed-trace-prequery-hit trace-table
    (Some (prefix, prefix-state))
  unfolding checked-staged-transcript-fixed-trace-alpha-prequery-hit-def
    staged-alpha-prefix-fixed-trace-prequery-hit-def
  by (auto elim!: set-dist-bindE split: option.splits prod.splits)
qed

lemma checked-staged-transcript-fixed-trace-alpha-bad-split:
  assumes hit:
    checked-staged-transcript-fixed-trace-alpha-bad trace-table out
  shows
    checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried
    trace-table out  $\vee$ 
    checked-staged-transcript-fixed-trace-alpha-prequery-hit trace-table out
  using hit
  unfolding checked-staged-transcript-fixed-trace-alpha-bad-def
    checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried-def
    checked-staged-transcript-fixed-trace-alpha-prequery-hit-def
  by (auto split: option.splits prod.splits)

lemma checked-staged-transcript-fixed-trace-alpha-bad-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-fixed-trace-alpha-bad trace-table)
    adversary-initial-state  $\leq$ 
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  proof -
    let ?M = checked-staged-transcript-with-alpha-prefix-program A
    let ?Fresh =
      checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried
      trace-table
    let ?Pre =
      checked-staged-transcript-fixed-trace-alpha-prequery-hit trace-table
    have event-le:
      wp-event ?M
      (checked-staged-transcript-fixed-trace-alpha-bad trace-table)
      adversary-initial-state  $\leq$ 
      wp-event ?M ( $\lambda$ out. ?Fresh out  $\vee$  ?Pre out)

```

```

    adversary-initial-state
  by (rule wp-event-mono) (rule checked-staged-transcript-fixed-trace-alpha-bad-split)
also have ... ≤
  wp-event ?M ?Fresh adversary-initial-state +
  wp-event ?M ?Pre adversary-initial-state
  by (rule wp-event-union-bound)
also have ... ≤
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0)
  by (intro add-mono
    checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried-bound
    checked-staged-transcript-fixed-trace-alpha-prequery-bound
    wf controlled)
  finally show ?thesis .
qed

end

end

```

```

theory Staged-Security-Experiment-Composition-Query-Canonical
imports Staged-Security-Experiment-Composition-Query-Classification
begin

```

Canonical-index projections for the header-authenticated composition query bridge. These lemmas do not bound the broad existential event by themselves; they expose the canonical query-index cover that a later Merkle/candidate binding argument has to control. In particular, the projection from the actual-alpha-prefix event to a query-prefix hit is support-level evidence. It is not by itself a probability reduction to the separately sampled query-prefix program: the authenticated opening witness is still selected existentially from verifier-reachable evidence. A sound probability bound needs either a cover fixed before the query challenge is sampled, or a Merkle/collision/partial-opening side event accounting for failure to select such a witness.

```

context soundness
begin

```

```

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
assumes hit:
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
  A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
obtains i trace-openings composition-openings query-prefix query-prefix-state
  raw raw-state where
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses

```

```

    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  i < rounds
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  Some (((query-prefix, query-prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  index (to-nat raw) ∈
    query-header-supported-partial-opening-canonical-query-indices
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
proof (rule
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-query
    [OF hit])
fix i trace-openings composition-openings query-prefix query-prefix-state
  raw raw-state
assume witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
and i-bound: i < rounds
and builder:
  Some (data, attacker-state) ∈

```


set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state)
and *prefix-receive*:
 Some (((query-prefix, query-prefix-state), raw), raw-state) ∈
set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state)
and *target-hit*:
 checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-opening-query-target-from-prefix
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])
 i)
 (Some (((query-prefix, query-prefix-state), raw), raw-state))
have *header-hit*:
 checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
 trace-openings composition-openings i
 (Some (((((query-prefix, query-prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state))
by (rule
 checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitI-from-dynamic-hit
 [OF witness target-hit])
have *canonical*:
 index (to-nat raw) ∈
 query-header-supported-partial-opening-canonical-query-indices
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
by (rule
 checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-canonical-query-indexD
 [OF i -bound header-hit])
show ?thesis
by (rule that[OF witness i -bound builder prefix-receive canonical])
qed

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
assumes *hit*:
 checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
 A
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))
and *cover*:

query-header-supported-partial-opening-canonical-query-indices
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\subseteq B$

obtains i query-prefix query-prefix-state raw raw-state **where**

$i < \text{rounds}$
 Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state)
 Some (((query-prefix, query-prefix-state), raw), raw-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state)
 checked-staged-query-prefix-dynamic-index-hit
 (λ - -. B)
 (Some (((query-prefix, query-prefix-state), raw), raw-state))

proof (rule

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-canon
 [OF hit])

fix i trace-openings composition-openings query-prefix query-prefix-state raw
 raw-state

assume i -bound: $i < \text{rounds}$

and builder:

Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state)

and prefix-receive:

Some (((query-prefix, query-prefix-state), raw), raw-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state)

and canonical:

index (to-nat raw) $\in$
 query-header-supported-partial-opening-canonical-query-indices
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)

```

      (staged-composition-fri-roots data)
      (staged-composition-final data)
have hit-B:
  checked-staged-query-prefix-dynamic-index-hit
    (λ- -. B)
    (Some (((query-prefix, query-prefix-state), raw), raw-state))
using canonical cover
unfolding checked-staged-query-prefix-dynamic-index-hit-def
by auto
show ?thesis
  by (rule that[OF i-bound builder prefix-receive hit-B])
qed

lemma checked-staged-query-prefix-constant-index-hit-bound-from-budgets:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i < \text{rounds}$ 
and subset:  $B \subseteq \text{query-sample-space}$ 
and frac:
  nnreal (card B) / nnreal (card query-sample-space) ≤
    query-error-bound
shows
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-hit (λ- -. B))
    adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) + query-error-bound
proof -
have raw-bound: query-index-raw-preimage-bound
  by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
have hit-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-hit (λ- -. B))
    adversary-initial-state ≤
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit (λ- -. B))
    adversary-initial-state + query-error-bound
proof (rule
  checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
  [OF raw-bound])
fix prefix prefix-state
assume
  Some ((prefix, prefix-state), prefix-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
      adversary-initial-state)
show
  (λ- -. B) prefix prefix-state ⊆ query-sample-space ∧
  nnreal (card ((λ- -. B) prefix prefix-state)) /

```

```

      nnreal (card query-sample-space) ≤ query-error-bound
    using subset frac by simp
  qed
  have prehit-bound:
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-prehit (λ- -. B))
      adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1)
  proof (rule
    checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program)
  show HashMap adversary-initial-state = fmempty
    by (simp add: adversary-initial-state-def)
  show hash-relation-program
    (checked-staged-query-prefix-dynamic-index-prehit-relation A i
      adversary-initial-state (λ- -. B))
    size (staged-query-search-queries budgets i + 1)
    (checked-staged-query-prefix-receive-with-state A i)
  by (rule hash-relation-program-checked-staged-query-prefix-receive-with-state
    [OF wf controlled])
    (use i-bound in simp-all,
      rule checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
  qed
  show ?thesis
    by (rule order-trans[OF hit-bound])
      (intro add-mono prehit-bound order-refl)
  qed

lemma checked-staged-security-with-query-prefix-constant-index-hit-bound-from-budgets:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and subset: B ⊆ query-sample-space
  and frac:
    nnreal (card B) / nnreal (card query-sample-space) ≤
    query-error-bound
  shows
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-dynamic-index-hit
        (λ- -. B))
      adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) + query-error-bound
  by (rule checked-staged-security-with-query-prefix-dynamic-index-hit-bound,
    rule checked-staged-query-prefix-constant-index-hit-bound-from-budgets
      [OF wf controlled i-bound subset frac])

end

```

end

theory *Soundness*

imports

Soundness-Staged-Aligned-Current-Query-Path
Soundness-Staged-Aligned-Current-Empty
Soundness-Alpha-Fixed-Candidate
Staged-Security-Experiment-Composition-Query-Header-Rounds
Staged-Security-Experiment-Composition-Query-Canonical
Staged-Security-Experiment-Composition-Query-Current

begin

Aggregator for the soundness infrastructure used by the active public endpoint. The public soundness theorem is exported by `Soundness_FRI_Trace_Restricted_Query_Endpoint.thy`.

end

theory *Staged-Security-Experiment-Composition-Query-Single-Round-Witness-Bridge*

imports

Staged-Security-Experiment-Composition-Query-Single-Round
Staged-Security-Experiment-Composition-Query-Current-Alignment
Staged-Security-Experiment-Composition-Query-Current-Path
Staged-Security-Experiment-Alpha-Header-Binding

begin

Bridge from the broad actual-alpha diagnostic witness event to a fixed query-prefix candidate-pair witness event.

The bridge is deterministic: it extracts the concrete prefix branch and candidate pair already present in the diagnostic event. It does not bound the diagnostic event probabilistically; later proof layers must still avoid treating the broad existential diagnostic event as a final public-path event.

context *soundness*

begin

definition *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at*
where

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A out
 $\longleftrightarrow$
(case out of
None $\Rightarrow$ False
| Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) $\Rightarrow$
($\exists$ prefix prefix-state raw raw-state.
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) $\in$

set-dist
 (execute
 (*checked-staged-security-experiment-with-query-prefix-data-state*
 A i)
 adversary-initial-state) $\wedge$
checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
trace-openings composition-openings trace-table composition-table i
 (Some (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*))))

definition *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at*
where

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
i A out $\longleftrightarrow$
 ($\exists$ *trace-openings composition-openings trace-table composition-table*.
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A
out)

definition *checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at*
where

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at
i out $\longleftrightarrow$
 ($\exists$ *trace-openings composition-openings trace-table composition-table*.
checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
trace-openings composition-openings trace-table composition-table i
out)

definition *staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at*
where

staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A out
 $\longleftrightarrow$
 (case *out* of
 None $\Rightarrow$ *False*
 | *Some* (((*data*, *attacker-state*), *result*), *final-state*) $\Rightarrow$
 ($\exists$ *prefix prefix-state raw raw-state*.
 Some (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*) $\in$
 set-dist
 (execute
 (*checked-staged-security-experiment-with-query-prefix-data-state*
 A i)
 adversary-initial-state) $\wedge$
 checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
 trace-openings composition-openings trace-table composition-table i
 (*Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*))))

definition *staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at*
where

staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
i A out $\longleftrightarrow$
 $(\exists \text{ trace-openings composition-openings trace-table composition-table.}$
staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A
out)

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-pr*
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A out

$\longleftrightarrow$
(case out of
None $\Rightarrow$ *False*
 $|$ *Some* $(((((\text{alpha-prefix, alpha-prefix-state}), \text{data}), \text{attacker-state}),$
result), final-state) $\Rightarrow$
staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A
(Some (((data, attacker-state), result), final-state)))

unfolding

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-def
staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-def
by (*auto split: option.splits prod.splits*)

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-wp*
wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A)
adversary-initial-state =
wp-event
(checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A)
adversary-initial-state

proof –

let *?project* =
 $\lambda x. ((\text{snd} (\text{fst} (\text{fst } x)), \text{snd} (\text{fst } x)), \text{snd } x)$

have

wp-event
(checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A)
adversary-initial-state =
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
 $\gg (\lambda x. \text{return } (?project x)))$
(staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at

```

    trace-openings composition-openings trace-table composition-table i A)
  adversary-initial-state
using
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
  [of A]
by simp
also have ... =
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
    trace-openings composition-openings trace-table composition-table i A)
  adversary-initial-state
proof -
have pred-eq:
  (λout. case out of
    None ⇒
      staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
      trace-openings composition-openings trace-table composition-table i A
      None
    | Some (x, t) ⇒
      staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
      trace-openings composition-openings trace-table composition-table i A
      (Some (?project x, t))) =
  checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
  trace-openings composition-openings trace-table composition-table i A
by (rule ext)
  (simp add:
    checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-proje
    staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-def
    split: option.splits prod.splits)
show ?thesis
by (subst wp-event-bind-return-map) (simp add: pred-eq)
qed
finally show ?thesis
by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-bor
assumes bound:
  wp-event
  (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
    trace-openings composition-openings trace-table composition-table i A)
  adversary-initial-state ≤ B
shows
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
    trace-openings composition-openings trace-table composition-table i A)

```


$adversary-initial-state \leq B$
using *bound*
unfolding
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-wp-project
by *simp*

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at*
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
i A out
 $\longleftrightarrow$
(case out of
None $\Rightarrow$ False
| Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) $\Rightarrow$
staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
i A (Some (((data, attacker-state), result), final-state)))
unfolding
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-def
staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-def
by (*auto simp:*
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-projecti
split: option.splits prod.splits)

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
i A)
adversary-initial-state =
wp-event
(checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
i A)
adversary-initial-state

proof –
let *?project =*
 $\lambda x. ((snd (fst (fst x)), snd (fst x)), snd x)$
have
wp-event
(checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
i A)
adversary-initial-state =
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
 $\gg (\lambda x. \text{return } (?project x))$
(staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
i A)
adversary-initial-state

```

using
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
  [of A]
by simp
also have ... =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
    i A)
  adversary-initial-state
proof -
have pred-eq:
  (λout. case out of
    None ⇒
      staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
        i A None
    | Some (x, t) ⇒
      staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
        i A (Some (?project x, t))) =
    checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
      i A
by (rule ext)
  (simp add:
    checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
    staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-def
    staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-def
    split: option.splits prod.splits)
show ?thesis
by (subst wp-event-bind-return-map) (simp add: pred-eq)
qed
finally show ?thesis
by simp
qed

lemma checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at-le-same-run-data-
assumes i-bound: i < rounds
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at
    i)
  adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
    i A)
  adversary-initial-state
proof -
let ?M =

```

```

checked-staged-security-experiment-with-query-prefix-data-state A i
let ?Gap =
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at
  i
let ?DataGap =
  staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
  i A
let ?projected =
   $\lambda out. \text{case } out \text{ of}$ 
     $None \Rightarrow ?DataGap \text{ None}$ 
  |  $Some (packed, t) \Rightarrow ?DataGap (Some (snd \text{ packed}, t))$ 
have projection:
  wp-event (checked-staged-security-experiment-with-data-state A)
  ?DataGap adversary-initial-state =
  wp-event ?M ?projected adversary-initial-state
by (rule checked-staged-security-experiment-with-query-prefix-data-state-projection-event
  [OF i-bound])
have event-le:
  wp-event ?M ?Gap adversary-initial-state  $\leq$ 
  wp-event ?M ?projected adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support: out  $\in$  set-dist (execute ?M adversary-initial-state)
and gap: ?Gap out
show ?projected out
proof (cases out)
case None
then show ?thesis
using gap
unfolding
checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at-def
checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-def
checked-staged-security-with-query-prefix-dynamic-index-hit-def
by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
final-state where out-eq:
out =
  Some (((((prefix, prefix-state), raw), raw-state),
  ((data, attacker-state), result)), final-state)
by (cases packed) (auto split: prod.splits)
from gap obtain trace-openings composition-openings trace-table
composition-table where fixed-gap:
checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
trace-openings composition-openings trace-table composition-table i
out
unfolding
checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at-def

```

```

    by blast
  have qsupport:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
      set-dist (execute ?M adversary-initial-state)
    using support out-eq by simp
  have fixed-gap-out:
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
      trace-openings composition-openings trace-table composition-table i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
    using fixed-gap out-eq by simp
  have data-gap:
    staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
      trace-openings composition-openings trace-table composition-table i A
      (Some (((data, attacker-state), result), final-state))
    unfolding
      staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-def
    using qsupport fixed-gap-out by auto
  have projected-gap:
    ?DataGap (Some (((data, attacker-state), result), final-state))
    unfolding
      staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-def
    using data-gap by blast
  show ?thesis
    unfolding out-eq
    using projected-gap by simp
qed
qed
show ?thesis
  unfolding projection
  by (rule event-le)
qed

lemma checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-atE:
  assumes gap:
    checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
      trace-openings composition-openings trace-table composition-table i A
      (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
  obtains prefix prefix-state raw raw-state where
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-query-prefix-data-state A i)
          adversary-initial-state)
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
      trace-openings composition-openings trace-table composition-table i

```

$((\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}), ((\text{data}, \text{attacker-state}), \text{result})), \text{final-state})),$
using *gap*
unfolding
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-def
by (*auto split: prod.splits*)

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-atI:*

assumes *support:*

$\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}), ((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}) \in$
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)

and *gap:*

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
trace-openings composition-openings trace-table composition-table i
 $((\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}), ((\text{data}, \text{attacker-state}), \text{result})), \text{final-state})),$

shows

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A
 $((\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}), \text{result}), \text{final-state})),$

unfolding

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-def
using *support gap by auto*

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-atI:*

assumes *gap:*

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A out

shows

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
i A out

unfolding

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-def
using *gap by blast*

lemma *checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-atI:*

assumes *gap:*

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
trace-openings composition-openings trace-table composition-table i out

shows

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at
i out

unfolding

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at-def

using gap by blast

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists*

assumes gap:

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
i A out

shows

checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
out

proof (*cases out*)

case *None*

then show *?thesis*

using gap

unfolding

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-d

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-def

by simp

next

case (*Some packed*)

then obtain *alpha-prefix alpha-prefix-state data attacker-state result*
final-state where out-eq:

out =

Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)

by (*cases packed, auto split: prod.splits*)

from gap **obtain** *trace-openings composition-openings trace-table*

composition-table prefix prefix-state raw raw-state where support:

Some (((((prefix, prefix-state), raw), raw-state),

((data, attacker-state), result)), final-state) ∈

set-dist

(execute

(checked-staged-security-experiment-with-query-prefix-data-state
A i)

adversary-initial-state)

and *fixed-gap:*

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at

trace-openings composition-openings trace-table composition-table i

(Some (((((prefix, prefix-state), raw), raw-state),

((data, attacker-state), result)), final-state))

unfolding *out-eq*

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-d

by (*auto elim!:*

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-atE)

have *witness-side:*

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side

trace-openings composition-openings trace-table composition-table i

(Some (((((prefix, prefix-state), raw), raw-state),

((data, attacker-state), result)), final-state))

by (*rule*

```

    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-imp-witness-side
    [OF fixed-gap])
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from witness-side have witness:
    (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-def
  by simp
  from query-header-supported-partial-opening-witnessesE[OF witness]
  obtain witness-result witness-state rest where outcome:
    Some (witness-result, witness-state) ∈
    set-dist (execute verify-monad ?s)
  by blast
  have tr-eq: PTranscript ?s = staged-proof-transcript data
  by simp
  have transcript-hit:
    hash-map-output-values ?s ∩ set (staged-proof-transcript data)
    ≠ {} ∨
    hash-map-new-output-hit (set (staged-proof-transcript data))
    ?s witness-state
  by (rule verify-monad-trace-root-target-preexisting-or-new
    [OF outcome tr-eq])
  show ?thesis
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-def
    Let-def
  using outcome transcript-hit
  by (auto split: prod.splits)
qed

lemma checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
    i A)
  adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit

```

adversary-initial-state
by (rule *wp-event-mono*)
(rule
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-i

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-i*
assumes *pre-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
adversary-initial-state $\leq P$
and *new-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq Q$
shows
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
(*checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-i* *A*)
adversary-initial-state $\leq P + Q$
by (rule *order-trans*
[*OF*
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-i
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
[*OF pre-bound new-bound*]])

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-i*
assumes *data-pre-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *new-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq Q$
shows
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
(*checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-i* *A*)
adversary-initial-state $\leq P + Q$
proof –
have *pre-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
adversary-initial-state $\leq P$
by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
 [OF *data-pre-bound*])
show ?thesis
by (rule
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
 [OF *pre-bound new-bound*])
qed

lemma *staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-bound-from-d*
assumes *data-pre-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *new-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq Q$
shows
wp-event
 (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at*
i A)
adversary-initial-state $\leq P + Q$
using
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-bo
 [OF *data-pre-bound new-bound*, of *i*]
unfolding
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-wp
by *simp*

lemma *checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at-bound-from-data-p*
assumes *i-bound*: $i < \text{rounds}$
and *data-pre-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *new-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq Q$

shows

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at
 i)
 $\text{adversary-initial-state} \leq P + Q$
by (rule order-trans[
 OF checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at-le-same-run-data-
 OF i -bound]
 staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-bound-from-data-
 OF data-pre-bound new-bound]])

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-with-witness-imp-single-*
assumes *hit:*

checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
 A trace-table composition-table i
 (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))

and *witness:*

(trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)

and *trace-candidate:*

partial-trace-table-candidate trace-table
 ((replicate rounds []) [$i := \text{trace-openings}$])

and *comp-candidate:*

partial-composition-table-candidate composition-table
 ((replicate rounds []) [$i := \text{composition-openings}$])

and *same-run-support:*

$\bigwedge \text{prefix prefix-state raw raw-state.}$
 Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 Some ((prefix, prefix-state), raw), raw-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state) $\implies$
 checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table)

$$\begin{aligned}
& (Some\ ((prefix,\ prefix\ state),\ raw),\ raw\ state)) \implies \\
& Some\ (((((prefix,\ prefix\ state),\ raw),\ raw\ state), \\
& \quad ((data,\ attacker\ state),\ result)),\ final\ state) \in \\
& \quad set\ dist \\
& \quad (execute \\
& \quad \quad (checked\ staged\ security\ experiment\ with\ query\ prefix\ data\ state \\
& \quad \quad \quad A\ i) \\
& \quad \quad adversary\ initial\ state)
\end{aligned}$$

shows

$$\begin{aligned}
& checked\ staged\ security\ with\ actual\ alpha\ prefix\ single\ round\ header\ supported\ query\ target\ hit\ at \\
& \quad i\ A \\
& \quad (Some\ (((((alpha\ prefix,\ alpha\ prefix\ state),\ data),\ attacker\ state), \\
& \quad \quad result),\ final\ state)) \vee \\
& \quad checked\ staged\ security\ with\ actual\ alpha\ prefix\ query\ prefix\ fixed\ candidate\ pair\ witness\ gap\ same\ run\ at \\
& \quad \quad trace\ openings\ composition\ openings\ trace\ table\ composition\ table\ i\ A \\
& \quad (Some\ (((((alpha\ prefix,\ alpha\ prefix\ state),\ data),\ attacker\ state), \\
& \quad \quad result),\ final\ state))
\end{aligned}$$

proof –

from

$$\begin{aligned}
& checked\ staged\ security\ with\ actual\ alpha\ prefix\ candidate\ pair\ query\ hit\ from\ prefixE \\
& \quad [OF\ hit]
\end{aligned}$$

obtain $prefix\ prefix\ state\ raw\ raw\ state$ **where**

builder:

$$\begin{aligned}
& Some\ (data,\ attacker\ state) \in \\
& \quad set\ dist \\
& \quad (execute\ (checked\ staged\ transcript\ program\ A) \\
& \quad \quad adversary\ initial\ state)
\end{aligned}$$

and $prefix\ receive$:

$$\begin{aligned}
& Some\ (((prefix,\ prefix\ state),\ raw),\ raw\ state) \in \\
& \quad set\ dist \\
& \quad (execute\ (checked\ staged\ query\ prefix\ receive\ with\ state\ A\ i) \\
& \quad \quad adversary\ initial\ state)
\end{aligned}$$

and $target\ hit$:

$$\begin{aligned}
& checked\ staged\ query\ prefix\ dynamic\ index\ hit \\
& \quad (staged\ query\ prefix\ candidate\ pair\ query\ target\ from\ prefix \\
& \quad \quad trace\ table\ composition\ table) \\
& \quad (Some\ (((prefix,\ prefix\ state),\ raw),\ raw\ state))
\end{aligned}$$

by (rule

$$\begin{aligned}
& checked\ staged\ security\ with\ actual\ alpha\ prefix\ candidate\ pair\ query\ hit\ from\ prefixE \\
& \quad [OF\ hit])
\end{aligned}$$

have $support$:

$$\begin{aligned}
& Some\ (((((prefix,\ prefix\ state),\ raw),\ raw\ state), \\
& \quad ((data,\ attacker\ state),\ result)),\ final\ state) \in \\
& \quad set\ dist \\
& \quad (execute \\
& \quad \quad (checked\ staged\ security\ experiment\ with\ query\ prefix\ data\ state \\
& \quad \quad \quad A\ i) \\
& \quad \quad adversary\ initial\ state)
\end{aligned}$$

by (rule $same\ run\ support[OF\ builder\ prefix\ receive\ target\ hit]$)

```

show ?thesis
proof (cases
  (trace-table, composition-table) ∈
    query-header-supported-single-round-partial-table-candidates
    prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
  i)
case True
have single-round-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-single-round-header-supported-query-target i)
    (Some (((prefix, prefix-state), raw), raw-state))
by (rule
  checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
  [OF True target-hit])
have
  checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
  i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
proof –
have inner:
  ∃ prefix prefix-state raw raw-state.
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state) ∧
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state) ∧
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-single-round-header-supported-query-target i)
    (Some (((prefix, prefix-state), raw), raw-state))
by (intro exI[of - prefix] exI[of - prefix-state]
  exI[of - raw] exI[of - raw-state])
  (use builder prefix-receive single-round-hit in simp)
show ?thesis
unfolding
  checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at-def
using inner by simp
qed
then show ?thesis by simp

```

```

next
case False
have success:
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table
    (sqp-alphas prefix)
using target-hit
unfolding
  checked-staged-query-prefix-dynamic-index-hit-def
  staged-query-prefix-candidate-pair-query-target-from-prefix-def
by simp
have fixed-gap:
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
  trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof -
  have dyn:
    checked-staged-security-with-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table)
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-dynamic-index-hit-def
    checked-staged-query-prefix-dynamic-index-hit-def
    staged-query-prefix-candidate-pair-query-target-from-prefix-def
  using success by simp
  have side:
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
    trace-openings composition-openings trace-table composition-table i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-def
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side-def
  using witness trace-candidate comp-candidate False by simp
  show ?thesis
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-def
  using dyn side by simp
qed
have
  checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
  trace-openings composition-openings trace-table composition-table i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
by (rule
  checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-atI

```

```

    [OF support fixed-gap])
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-candidate-pair-success-with-witness-imp-single-round-header-
assumes i-bound:  $i < \text{rounds}$ 
and support:
  Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state)  $\in$ 
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
and success:
   $\bigwedge \text{prefix prefix-state raw raw-state.}$ 
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)  $\in$ 
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state
          A i)
        adversary-initial-state)  $\implies$ 
    sqp-alphas prefix = staged-alphas data  $\implies$ 
    index (to-nat raw)  $\in$ 
      query-sampling-success-space trace-table composition-table
        (staged-alphas data)
and witness:
  (trace-openings, composition-openings)  $\in$ 
    query-header-supported-partial-opening-witnesses
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
and trace-candidate:
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
shows
  checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
    i A

```

```

    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state)) ∨
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
  trace-openings composition-openings trace-table composition-table i A
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
proof (rule
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-success-query-prefix-hitE
  [OF i-bound support success])
fix prefix prefix-state raw raw-state
assume qsupport:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)
and ghit:
  checked-staged-security-with-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-pair-query-target-from-prefix
    trace-table composition-table)
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
  set-dist
  (execute (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
by (rule
  checked-staged-security-experiment-with-query-prefix-data-state-imp-data-state-support
  [OF i-bound qsupport])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder where builder:
  Some (data, attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-program A)
    adversary-initial-state)
by blast
have prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
  set-dist
  (execute (checked-staged-query-prefix-receive-with-state A i)
    adversary-initial-state)
using qsupport
unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
by (auto elim!: set-dist-bindE split: prod.splits)
have raw-pair-hit:
  checked-staged-query-prefix-dynamic-index-hit

```

```

    (staged-query-prefix-candidate-pair-query-target-from-prefix
     trace-table composition-table)
    (Some (((prefix, prefix-state), raw), raw-state))
using qhit
unfolding checked-staged-security-with-query-prefix-dynamic-index-hit-def
by simp
show ?thesis
proof (cases
  (trace-table, composition-table) ∈
    query-header-supported-single-round-partial-table-candidates
    prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
    i)
case True
have single-round-hit:
  checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-single-round-header-supported-query-target i)
  (Some (((prefix, prefix-state), raw), raw-state))
by (rule
  checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
  [OF True raw-pair-hit])
have
  checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
  i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
proof –
have inner:
  ∃ prefix prefix-state raw raw-state.
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state) ∧
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state) ∧
  checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-single-round-header-supported-query-target i)
  (Some (((prefix, prefix-state), raw), raw-state))
by (intro exI[of - prefix] exI[of - prefix-state]
  exI[of - raw] exI[of - raw-state])
  (use builder prefix-receive single-round-hit in simp)

```



```

show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at-def
  using inner by simp
qed
then show ?thesis by simp
next
case False
have side:
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
    trace-openings composition-openings trace-table composition-table i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-def
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side-def
  using witness trace-candidate comp-candidate False by simp
have fixed-gap:
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
    trace-openings composition-openings trace-table composition-table i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-def
  using qhit side by simp
have
  checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
    trace-openings composition-openings trace-table composition-table i A
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-atI
    [OF qsupport fixed-gap])
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-imp-single-round-header
assumes hit:
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
    trace-openings composition-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
and trace-candidate:
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])

```

and *trace-low*: *trace-table-low-degree trace-table*
and *comp-low*: *composition-table-low-degree maxDegree composition-table*
and *not-all*:
 $\neg$ *all-queries-consistent trace-table composition-table*
 (*sqp-alphas prefix*)
shows
 checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
 i
 (*Some* ((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*)) $\vee$
 checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
 trace-openings composition-openings trace-table composition-table i
 (*Some* ((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*))
proof –
from *hit* **obtain** *comp-nonempty witness opening-target*
where *comp-nonempty*:
 staged-composition-fri-roots data $\neq$ []
and *witness*:
 (*trace-openings*, *composition-openings*) $\in$
 query-header-supported-partial-opening-witnesses
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)
 (*staged-composition-final data*)
and *opening-target*:
 index (to-nat raw) $\in$
 staged-query-prefix-candidate-opening-query-target-from-prefix
 ((*replicate rounds* []) [*i* := *trace-openings*])
 ((*replicate rounds* []) [*i* := *composition-openings*])
 i prefix prefix-state
by (*rule*
 checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitE)
have *opening-subset*:
 staged-query-prefix-candidate-opening-query-target-from-prefix
 ((*replicate rounds* []) [*i* := *trace-openings*])
 ((*replicate rounds* []) [*i* := *composition-openings*])
 i prefix prefix-state $\subseteq$
 staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
 composition-table prefix prefix-state
by (*rule*
 staged-query-prefix-candidate-opening-query-target-from-prefix-subset-candidate-pair-from-prefix
 [*OF trace-candidate comp-candidate trace-low comp-low not-all*])
have *pair-target-hit*:

```

checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-pair-query-target-from-prefix
   trace-table composition-table)
  (Some (((prefix, prefix-state), raw), raw-state))
using opening-target opening-subset
unfolding checked-staged-query-prefix-dynamic-index-hit-def by auto
show ?thesis
proof (cases
  (trace-table, composition-table) ∈
    query-header-supported-single-round-partial-table-candidates
    prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
    i)
case True
have target:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-single-round-header-supported-query-target i)
    (Some (((prefix, prefix-state), raw), raw-state))
by (rule
  checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
  [OF True pair-target-hit])
have
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
    i
    (Some ((((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-def
using target by simp
then show ?thesis by simp
next
case False
have gap:
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
    trace-openings composition-openings trace-table composition-table i
    (Some ((((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at-def
using hit trace-candidate comp-candidate False by simp
have side:
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
    trace-openings composition-openings trace-table composition-table i

```

```

    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-def
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side-def
  using witness trace-candidate comp-candidate False by simp
have
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
    trace-openings composition-openings trace-table composition-table i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-def
  using gap side by simp
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-bound-from-target-and-f
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and trace-candidate:
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
  and comp-candidate:
    partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
  and trace-low: trace-table-low-degree trace-table
  and comp-low: composition-table-low-degree maxDegree composition-table
  and not-all:
     $\bigwedge \text{prefix prefix-state raw raw-state data attacker-state result}$ 
     $\text{final-state.}$ 
     $\text{Some } (((((\text{prefix, prefix-state}), \text{raw}), \text{raw-state}),$ 
       $((\text{data, attacker-state}), \text{result})), \text{final-state}) \in$ 
     $\text{set-dist}$ 
     $(\text{execute}$ 
       $(\text{checked-staged-security-experiment-with-query-prefix-data-state}$ 
         $A \ i)$ 
       $\text{adversary-initial-state}) \implies$ 
     $\neg \text{all-queries-consistent trace-table composition-table}$ 
     $(\text{sqp-alphas prefix})$ 
  and target-bound:
    wp-event
     $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$ 
     $(\text{checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at}$ 
       $i)$ 
     $\text{adversary-initial-state} \leq T$ 
shows

```

```

wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
    trace-openings composition-openings i)
  adversary-initial-state ≤
  T +
  (staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    (1::prob) / nnreal (card query-sample-space))
proof –
  let ?m =
    checked-staged-security-experiment-with-query-prefix-data-state A i
  let ?target =
    checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
      i
  let ?gap =
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
      trace-openings composition-openings trace-table composition-table i
  have event-le:
    wp-event ?m
    (checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
      trace-openings composition-openings i)
    adversary-initial-state ≤
    wp-event ?m (λout. ?target out ∨ ?gap out)
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈ set-dist (execute ?m adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
      trace-openings composition-openings i out
  show ?target out ∨ ?gap out
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-def
    by simp
  next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state
    result final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
  have not-all':

```

```

    ¬ all-queries-consistent trace-table composition-table
      (sqp-alphas prefix)
    by (rule not-all)
      (use support out-eq in simp)
  show ?thesis
    unfolding out-eq
    by (rule
      checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-imp-single-round-header-
      [OF - trace-candidate comp-candidate trace-low comp-low not-all'])
      (use hit out-eq in simp)
  qed
qed
have union-le:
  wp-event ?m (λout. ?target out ∨ ?gap out)
    adversary-initial-state ≤
  wp-event ?m ?target adversary-initial-state +
  wp-event ?m ?gap adversary-initial-state
  by (rule wp-event-union-bound)
have gap-bound:
  wp-event ?m ?gap adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space)
  by (rule
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-bound-from-budgets
    [OF wf controlled i-bound])
have sum-bound:
  wp-event ?m ?target adversary-initial-state +
  wp-event ?m ?gap adversary-initial-state ≤
  T +
  (staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    (1::prob) / nnreal (card query-sample-space))
  by (rule add-mono[OF target-bound gap-bound])
show ?thesis
  by (rule order-trans[OF event-le order-trans[OF union-le sum-bound]])
qed

```

lemma *checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-atI*:

assumes *success*:

index (to-nat raw) ∈
query-sampling-success-space trace-table composition-table
 (*sqp-alphas prefix*)

and *witness*:

(*trace-openings, composition-openings*) ∈
query-header-supported-partial-opening-witnesses
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 (*staged-trace-root data*)

```

    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  and trace-candidate:
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
  and comp-candidate:
    partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
  and notin:
    (trace-table, composition-table) ∉
    query-header-supported-single-round-partial-table-candidates
    prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
  i
shows
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
  trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof —
  have dyn:
    checked-staged-security-with-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
     trace-table composition-table)
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  using success
  unfolding
    checked-staged-security-with-query-prefix-dynamic-index-hit-def
    checked-staged-query-prefix-dynamic-index-hit-def
    staged-query-prefix-candidate-pair-query-target-from-prefix-def
  by simp
  have side:
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
    trace-openings composition-openings trace-table composition-table i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-def

```

```

    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side-def
using witness trace-candidate comp-candidate notin by simp
show ?thesis
unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-def
using dyn side by simp
qed

lemma checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-bound-from-budgets:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i < \text{rounds}$ 
and trace-low: trace-table-low-degree trace-table
and comp-low:
    composition-table-low-degree maxDegree composition-table
and not-all:
     $\bigwedge \text{prefix prefix-state.}$ 
    Some ((prefix, prefix-state), prefix-state)  $\in$ 
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
    adversary-initial-state)  $\implies$ 
     $\neg$  all-queries-consistent trace-table composition-table
    (sqp-alphas prefix)
shows
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
    trace-openings composition-openings trace-table composition-table i)
    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + query-error-bound
proof –
have event-le:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
    trace-openings composition-openings trace-table composition-table i)
    adversary-initial-state  $\leq$ 
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
    trace-table composition-table))
    adversary-initial-state
by (rule wp-event-mono)
    (rule
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-imp-dynamic-hit)
have target-bound:
    wp-event

```



```

    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table))
    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + query-error-bound
  by (rule
    checked-staged-security-with-query-prefix-candidate-pair-target-from-prefix-hit-bound-from-budgets
    [OF wf controlled i-bound trace-low comp-low not-all])
  show ?thesis
  by (rule order-trans[OF event-le target-bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witnessE-can
assumes gap:
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
  i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
obtains trace-openings composition-openings trace-table composition-table
  prefix prefix-state raw raw-state where
  Some (data, attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-program A)
    adversary-initial-state)
  Some (((prefix, prefix-state), raw), raw-state) ∈
  set-dist
  (execute (checked-staged-query-prefix-receive-with-state A i)
    adversary-initial-state)
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
  trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))

proof —
from gap show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witnessE)
fix trace-openings composition-openings trace-table composition-table
  prefix prefix-state raw raw-state
assume builder:
  Some (data, attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-program A)
    adversary-initial-state)
assume prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
  set-dist
  (execute (checked-staged-query-prefix-receive-with-state A i)

```

adversary-initial-state)
assume *witness*:
 (*trace-openings*, *composition-openings*) ∈
 query-header-supported-partial-opening-witnesses
 (*verifier-state-from-adversary* *attacker-state*
 (*staged-proof-transcript* *data*))
 (*staged-trace-root* *data*)
 (*staged-trace-fri-roots* *data*)
 (*staged-trace-final* *data*)
 (*staged-alphas* *data*)
 (*staged-degree* *data*)
 (*staged-composition-fri-roots* *data*)
 (*staged-composition-final* *data*)
assume *trace-candidate*:
 partial-trace-table-candidate *trace-table*
 ((*replicate* *rounds* []) [*i* := *trace-openings*])
assume *comp-candidate*:
 partial-composition-table-candidate *composition-table*
 ((*replicate* *rounds* []) [*i* := *composition-openings*])
assume *target-hit*:
 checked-staged-query-prefix-dynamic-index-hit
 (*staged-query-prefix-candidate-pair-query-target-from-prefix*
 trace-table *composition-table*)
 (*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*))
assume *notin*:
 (*trace-table*, *composition-table*) ∉
 query-header-supported-single-round-partial-table-candidates
 prefix-state
 (*sqp-trace-root* *prefix*)
 (*sqp-trace-fri-roots* *prefix*)
 (*sqp-trace-final* *prefix*)
 (*sqp-alphas* *prefix*)
 (*sqp-degree* *prefix*)
 (*sqp-composition-fri-roots* *prefix*)
 (*sqp-composition-final* *prefix*)
 i
have *dyn*:
 checked-staged-security-with-query-prefix-dynamic-index-hit
 (*staged-query-prefix-candidate-pair-query-target-from-prefix*
 trace-table *composition-table*)
 (*Some* ((((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*))
unfolding *checked-staged-security-with-query-prefix-dynamic-index-hit-def*
using *target-hit* **by** *simp*
have *side*:
 checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
 trace-openings *composition-openings* *trace-table* *composition-table* *i*
 (*Some* ((((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*))

```

unfolding
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-def
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side-def
using witness trace-candidate comp-candidate notin by simp
have fixed-gap:
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
  trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-def
using dyn side by simp
show ?thesis
by (rule that[OF builder prefix-receive fixed-gap])
qed
qed

lemma checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-imp-single-round-header-target-or-fix
assumes pair-hit:
  checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
  trace-table composition-table
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
and witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
and trace-candidate:
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
shows
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
  i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)) ∨
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
  trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),

```

```

      ((data, attacker-state), result)), final-state))
proof -
  let ?out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
  have raw-hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table)
      (Some (((prefix, prefix-state), raw), raw-state))
  using pair-hit
  unfolding
    checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-def
    checked-staged-security-with-query-prefix-dynamic-index-hit-def
  by simp
  show ?thesis
  proof (cases
    (trace-table, composition-table) ∈
      query-header-supported-single-round-partial-table-candidates
        prefix-state
        (sqp-trace-root prefix)
        (sqp-trace-fri-roots prefix)
        (sqp-trace-final prefix)
        (sqp-alphas prefix)
        (sqp-degree prefix)
        (sqp-composition-fri-roots prefix)
        (sqp-composition-final prefix)
      i)
  case True
  have target:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-single-round-header-supported-query-target i)
      (Some (((prefix, prefix-state), raw), raw-state))
  by (rule
    checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
    [OF True raw-hit])
  have
    checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
      i ?out
  unfolding
    checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-def
  using target by simp
  then show ?thesis by simp
next
  case False
  have success:
    index (to-nat raw) ∈
      query-sampling-success-space trace-table composition-table
        (sqp-alphas prefix)

```

```

using raw-hit
unfolding
  checked-staged-query-prefix-dynamic-index-hit-def
  staged-query-prefix-candidate-pair-query-target-from-prefix-def
by simp
have
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
    trace-openings composition-openings trace-table composition-table i
    ?out
  by (rule
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-atI
    [OF success witness trace-candidate comp-candidate False])
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-single-round-header-target-or-fixed-w
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
and witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
and trace-candidate:
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table

```

```

    ((replicate rounds []) [i := composition-openings])
and trace-low: trace-table-low-degree trace-table
and comp-low: composition-table-low-degree maxDegree composition-table
and not-all:
  ¬ all-queries-consistent trace-table composition-table
  (sqp-alphas prefix)
shows
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
  i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)) ∨
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
  trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)) ∨
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
  i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)) ∨
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
  i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof –
  let ?out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
  have split:
    checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
    trace-table composition-table ?out ∨
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i ?out ∨
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i ?out
  by (rule
    checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-candidate-pair-or-structured-path-on-sup
    [OF wf controlled i-bound support hit trace-candidate
      comp-candidate trace-low comp-low not-all])
  then show ?thesis
proof
  assume pair:
    checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
    trace-table composition-table ?out
  from
    checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-imp-single-round-header-target-or-fixed-wi
    [OF pair witness trace-candidate comp-candidate]
  show ?thesis
  by blast
next

```

```

assume
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
  i ?out  $\vee$ 
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
  i ?out
then show ?thesis by blast
qed
qed

```

lemma *checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-single-round-target-fixed-withn*
fixes $H\ G\ T\ C :: \text{prob}$

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: $i < \text{rounds}$*

and *witness:*

$\bigwedge \text{prefix prefix-state raw raw-state data attacker-state result}$
final-state.
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) $\in$
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state
A i)
adversary-initial-state) $\implies$
(trace-openings, composition-openings) $\in$
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)

and *trace-candidate:*

partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings])

and *comp-candidate:*

partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])

and *trace-low: trace-table-low-degree trace-table*

and *comp-low: composition-table-low-degree maxDegree composition-table*

and *not-all:*

$\bigwedge \text{prefix prefix-state raw raw-state data attacker-state result}$
final-state.
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) $\in$
set-dist

```

      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state
          A i)
        adversary-initial-state)  $\implies$ 
 $\neg$  all-queries-consistent trace-table composition-table
      (sqp-alphas prefix)
and target-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
      i)
    adversary-initial-state  $\leq H$ 
and fixed-gap-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
      trace-openings composition-openings trace-table composition-table i)
    adversary-initial-state  $\leq G$ 
and trace-path-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
      i)
    adversary-initial-state  $\leq T$ 
and composition-path-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
      i)
    adversary-initial-state  $\leq C$ 
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-authenticated-opening-hit
      trace-openings composition-openings i)
    adversary-initial-state  $\leq H + G + T + C$ 
proof –
  let ?M =
    checked-staged-security-experiment-with-query-prefix-data-state A i
  let ?Target =
    checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
      i
  let ?Gap =
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
      trace-openings composition-openings trace-table composition-table i
  let ?Trace =
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
      i
  let ?Comp =

```



```

checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
  i
have event-le:
  wp-event ?M
    (checked-staged-security-with-query-prefix-authenticated-opening-hit
      trace-openings composition-openings i)
  adversary-initial-state ≤
  wp-event ?M
    (λout. ?Target out ∨ ?Gap out ∨ ?Trace out ∨ ?Comp out)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈ set-dist (execute ?M adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-authenticated-opening-hit
      trace-openings composition-openings i out
  show ?Target out ∨ ?Gap out ∨ ?Trace out ∨ ?Comp out
  proof (cases out)
    case None
    then show ?thesis
      using hit
      unfolding
        checked-staged-security-with-query-prefix-authenticated-opening-hit-def
      by simp
  next
    case (Some packed)
    then obtain prefix prefix-state raw raw-state data attacker-state result
      final-state where out-eq:
      out =
        Some (((((prefix, prefix-state), raw), raw-state),
          ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
  have support-some:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
      set-dist (execute ?M adversary-initial-state)
    using support out-eq by simp
  show ?thesis
    unfolding out-eq
    by (rule
      checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-single-round-header-target-or-fixed-wit
      [OF wf controlled i-bound support-some - witness[OF support-some]
        trace-candidate comp-candidate trace-low comp-low
        not-all[OF support-some]])
    (use hit out-eq in simp)
  qed
qed
have union-bound:

```

```

wp-event ?M
  (λout. ?Target out ∨ ?Gap out ∨ ?Trace out ∨ ?Comp out)
  adversary-initial-state ≤
  wp-event ?M ?Target adversary-initial-state +
  wp-event ?M ?Gap adversary-initial-state +
  wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M ?Comp adversary-initial-state
by (rule wp-event-union-bound4)
have component-bound:
  wp-event ?M ?Target adversary-initial-state +
  wp-event ?M ?Gap adversary-initial-state +
  wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M ?Comp adversary-initial-state ≤
  H + G + T + C
by (intro add-mono target-bound fixed-gap-bound trace-path-bound
    composition-path-bound)
show ?thesis
  by (rule order-trans[OF event-le order-trans[OF union-bound
    component-bound]])
qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-single-round-target-budgets-an
fixes H T C :: prob
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and witness:
  ∧ prefix prefix-state raw raw-state data attacker-state result
  final-state.
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state
      A i)
    adversary-initial-state) ⇒
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
and trace-candidate:
  partial-trace-table-candidate trace-table

```

((*replicate rounds* $\square$) [$i := \text{trace-openings}$])
and *comp-candidate*:
 partial-composition-table-candidate composition-table
 ((*replicate rounds* $\square$) [$i := \text{composition-openings}$])
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-low*: *composition-table-low-degree maxDegree composition-table*
and *not-all*:
 $\bigwedge \text{prefix prefix-state.}$
 Some ((*prefix, prefix-state*), *prefix-state*) $\in$
 set-dist
 (*execute* (*checked-staged-query-prefix-with-state A i*)
 adversary-initial-state) $\implies$
 $\neg$ *all-queries-consistent trace-table composition-table*
 (*sqp-alphas prefix*)
and *target-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at*
 i)
 adversary-initial-state $\leq H$
and *trace-path-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-trace-path-output-hit-at*
 i)
 adversary-initial-state $\leq T$
and *composition-path-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-composition-path-output-hit-at*
 i)
 adversary-initial-state $\leq C$
shows
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-authenticated-opening-hit*
 trace-openings composition-openings i)
 adversary-initial-state $\leq$
 $H +$
 (*staged-phase-relation-error size*
 (*staged-query-search-queries budgets i + 1*) + *query-error-bound*) +
 $T + C$
proof –
have *fixed-gap-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at*
 trace-openings composition-openings trace-table composition-table i)
 adversary-initial-state $\leq$

```

    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + query-error-bound
  by (rule
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-bound-from-budgets
    [OF wf controlled i-bound trace-low comp-low])
    (use not-all in blast)
  have not-all-on-data-support:
     $\bigwedge$  prefix prefix-state raw raw-state data attacker-state result
      final-state.
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)  $\in$ 
        set-dist
        (execute
          (checked-staged-security-experiment-with-query-prefix-data-state
            A i)
          adversary-initial-state)  $\implies$ 
         $\neg$  all-queries-consistent trace-table composition-table
        (sqp-alphas prefix)
  proof -
  fix prefix prefix-state raw raw-state data attacker-state result
    final-state
  assume support:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)  $\in$ 
      set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state
          A i)
        adversary-initial-state)
  have receive-support:
    Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
      set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  using support
  unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
  have prefix-support:
    Some ((prefix, prefix-state), prefix-state)  $\in$ 
      set-dist
      (execute (checked-staged-query-prefix-with-state A i)
        adversary-initial-state)
  by (rule checked-staged-query-prefix-receive-with-state-prefix-support
    [OF receive-support])
  show  $\neg$  all-queries-consistent trace-table composition-table
    (sqp-alphas prefix)
  by (rule not-all[OF prefix-support])
qed
show ?thesis

```

```

    by (rule
      checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-single-round-target-fixed-witness-
      [OF wf controlled i-bound witness trace-candidate comp-candidate
        trace-low comp-low not-all-on-data-support target-bound fixed-gap-bound
        trace-path-bound composition-path-bound])
qed
end
end
end

```

```

theory Staged-Security-Experiment-Composition-Query-Aligned-Witness-Bridge
imports
  Staged-Security-Experiment-Composition-Query-Current-Alignment
  Staged-Security-Experiment-Composition-Query-Single-Round-Witness-Bridge
begin

```

Same-run bridge from actual-alpha support to the fixed query-prefix candidate-pair witness-gap event.

This layer avoids the broad diagnostic event: the sampled query index, prefix state, verifier output, and witness side facts all live on the same query-prefix data-state output.

```

context soundness
begin

```

lemma *checked-staged-security-with-query-prefix-data-state-accepted-shape-query-index:*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i < \text{rounds}$ 
and support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)  $\in$ 
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
and shape:
  accepted-transcript-shape
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (Some (result, final-state)) as query-idxs
shows index (to-nat raw) = query-idxs ! i
proof –
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have data-support:
  Some (((data, attacker-state), result), final-state)  $\in$ 
    set-dist

```

```

    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
  by (rule
    checked-staged-security-experiment-with-query-prefix-data-state-imp-data-state-support
    [OF i-bound support])
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF data-support]
  obtain verifier where verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
  from checked-staged-security-with-data-state-accepted-shape-query-chunks
    [OF wf controlled data-support shape]
  obtain raw-idxs where len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookup-final:
    ∀ j < rounds.
      fmllookup (HashMap final-state)
        (QueryIndexChallenge j
          (state-after-query-chunks
            (staged-query-start-hash data) (staged-query-chunks data) j)) =
        Some (raw-idxs ! j)
  by blast
  from support obtain t where prefix-receive:
    Some (((prefix, prefix-state), raw), t) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
  and t-eq: t = raw-state
  and cont:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-after-query-prefix-receive-with-verifier A i
            ((prefix, prefix-state), raw))
          raw-state)
  unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
  have data-out:
    Some (data, attacker-state) ∈
      set-dist
        (execute
          (checked-staged-after-query-prefix-receive A i
            ((prefix, prefix-state), raw))
          raw-state)
  by (rule checked-staged-after-query-prefix-receive-with-verifier-outcome(1)
    [OF cont])
  have prefix-support:
    Some ((prefix, prefix-state), prefix-state) ∈

```

```

    set-dist
      (execute (checked-staged-query-prefix-with-state A i)
        adversary-initial-state)
  by (rule checked-staged-query-prefix-receive-with-state-prefix-support)
    (use prefix-receive t-eq in simp)
have recv:
  Some (raw, raw-state) ∈
    set-dist (execute receive-query-index-challenge prefix-state)
  by (rule checked-staged-query-prefix-receive-with-state-receive-support)
    (use prefix-receive t-eq in simp)
from checked-staged-query-prefix-with-state-alignment
  [OF wf controlled i-bound prefix-support]
have prefix-counter: PQueryCounter prefix-state = i
and prefix-state-hash:
  PState prefix-state =
    state-after-query-chunks
      (staged-query-prefix-start-hash prefix)
      (sqp-query-chunks prefix) i
and prefix-len: length (sqp-query-chunks prefix) = i
by blast+
from data-out obtain chunk chunk-state record-state suffix-chunks where
  chunk-out:
    Some (chunk, chunk-state) ∈
      set-dist (execute (query-opening-stage A i raw) raw-state)
and record-out:
  Some ((), record-state) ∈
    set-dist (execute (record-staged-messages chunk) chunk-state)
and suffix-out:
  Some (suffix-chunks, attacker-state) ∈
    set-dist
      (execute
        (checked-staged-query-program A
          (sqp-trace-fri-roots prefix)
          (sqp-composition-fri-roots prefix)
          (Suc i) (rounds - Suc i))
        record-state)
and data-eq:
  data =
    (| staged-trace-root = sqp-trace-root prefix,
      staged-trace-fri-roots = sqp-trace-fri-roots prefix,
      staged-trace-fri-challenges = sqp-trace-fri-challenges prefix,
      staged-trace-final = sqp-trace-final prefix,
      staged-alphas = sqp-alphas prefix,
      staged-degree = sqp-degree prefix,
      staged-composition-fri-roots = sqp-composition-fri-roots prefix,
      staged-composition-fri-challenges =
        sqp-composition-fri-challenges prefix,
      staged-composition-final = sqp-composition-final prefix,
      staged-query-chunks =

```

```

      sqp-query-chunks prefix @ chunk # suffix-chunks)
unfolding checked-staged-after-query-prefix-receive-def Let-def
by (auto elim!: set-dist-bindE simp: assert-def throw-no-outcome
    split: if-splits)
have match: staged-query-prefix-matches-data i prefix data
unfolding staged-query-prefix-matches-data-def data-eq
using prefix-len by simp
have start-hash:
  staged-query-start-hash data = staged-query-prefix-start-hash prefix
by (rule staged-query-start-hash-of-prefix-completion[OF match])
have prefix-key-state:
  PState prefix-state =
    state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i
using prefix-state-hash prefix-len start-hash
unfolding data-eq
by (simp add: state-after-query-chunks-append-prefix)
have receive-lookup:
  fmllookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
using receive-query-index-challenge-outcome[OF recv]
    prefix-counter prefix-key-state
by simp
have raw-ext-attacker: raw-state ≤ attacker-state
by (rule checked-staged-after-query-prefix-receive-extends
  [OF wf controlled i-bound data-out])
have attacker-ext-s: attacker-state ≤ ?s
proof -
  have attacker-state ≤ attacker-state
    unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  then show ?thesis
    by (rule hash-extends-verifier-state-from-adversary-right)
qed
have s-ext-final: ?s ≤ final-state
by (rule verify-monad-hash-extends[OF verifier])
have raw-ext-final: raw-state ≤ final-state
by (rule hash-ext-trans[OF raw-ext-attacker])
  (rule hash-ext-trans[OF attacker-ext-s s-ext-final])
have lookup-final-raw:
  fmllookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
by (rule hash-extension-lookup[OF receive-lookup raw-ext-final])
have raw-eq: raw-idxs ! i = raw

```



```

    using lookup-final[rule-format, OF i-bound] lookup-final-raw by simp
  show ?thesis
    using query-idxs-eq len-raw i-bound raw-eq by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-aligned-transcript-query-bad-imp-header-authenticated-candidate
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  and bad:
    checked-staged-security-with-actual-alpha-prefix-verifier-event
      query-bad-with-aligned-transcript-partial-candidates
      (Some (((((alpha-prefix, alpha-prefix-state), data),
        attacker-state), result), final-state))
  shows
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
      A
      (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF data-support]
  obtain builder verifier where builder:
    Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
  have staged-header:

```

```

verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
from bad obtain fr f-fri-roots f-final as dg composition-fri-roots
  final query-idxs trace-openings composition-openings trace-table
  composition-table where partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent
    ?s (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  query-bad-with-aligned-transcript-partial-candidates-def
by auto
have aligned:
  accepted-with-partial-initial-openings-aligned-consistent ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
have shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
    [OF partial])
from checked-staged-security-with-data-state-accepted-shape-query-chunks
  [OF wf controlled data-support shape]
obtain raw-idxs where as-eq: as = staged-alphas data
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and lookup:
   $\forall i < \text{rounds}.$ 
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)

```

```

  by blast
have aligned0:
  accepted-with-partial-initial-openings-aligned ?s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
  by (rule accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
      [OF aligned])
from accepted-with-partial-initial-openings-aligned-shapes[OF aligned0]
obtain rest where header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
  by blast
have comp-nonempty: composition-fri-roots ≠ []
  using accepted-with-partial-initial-openings-aligned-shapes(1)
  [OF aligned0] .
have trace-partial:
  accepted-with-partial-trace-openings ?s
  (Some (result, final-state)) fr query-idxs trace-openings
  using accepted-with-partial-initial-openings-aligned-shapes(3)
  [OF aligned0] .
have comp-partial:
  accepted-with-partial-composition-openings ?s
  (Some (result, final-state)) (hd composition-fri-roots) query-idxs
  composition-openings
  using accepted-with-partial-initial-openings-aligned-shapes(4)
  [OF aligned0] .
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF header staged-header]
  by simp
let ?i = 0
have i-bound: ?i < rounds
  using rounds-positive by simp
have round-consistent:
  partial-query-round-consistent trace-openings composition-openings as
  ?i (query-idxs ! ?i)
  using aligned i-bound
  unfolding accepted-with-partial-initial-openings-aligned-consistent-def
  by simp
have opening-consistent:
  partial-query-openings-consistent (trace-openings ! ?i)

```

```

    (composition-openings ! ?i) (staged-alphas data)
    (index (to-nat (raw-idxs ! ?i)))
using round-consistent query-idxs-eq len-raw i-bound header-eq
unfolding partial-query-round-consistent-def
    partial-query-openings-consistent-def
by simp
have trace-table-auth:
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! ?i) final-state
using trace-partial i-bound
unfolding accepted-with-partial-trace-openings-def by blast
have comp-table-auth:
    partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) (composition-openings ! ?i) final-state
using comp-partial i-bound
unfolding accepted-with-partial-composition-openings-def by blast
have witness-header:
    (trace-openings ! ?i, composition-openings ! ?i) ∈
    query-header-supported-partial-opening-witnesses ?s fr f-fri-roots
    f-final as dg composition-fri-roots final
unfolding query-header-supported-partial-opening-witnesses-def
using comp-nonempty verifier trace-table-auth comp-table-auth header
by blast
have witness:
    (trace-openings ! ?i, composition-openings ! ?i) ∈
    query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
using witness-header header-eq by simp
have lookup0:
    fmlookup (HashMap final-state)
    (QueryIndexChallenge ?i
    (state-after-query-chunks
    (staged-query-start-hash data)
    (staged-query-chunks data) ?i)) =
    Some (raw-idxs ! ?i)
using lookup i-bound by simp
from checked-staged-transcript-program-query-prefix-receive-final-lookup-support-with-alphas
    [OF wf controlled i-bound builder verifier lookup0]
obtain prefix prefix-state raw-state where prefix-receive:
    Some (((prefix, prefix-state), raw-idxs ! ?i), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A ?i)
    adversary-initial-state)

```

```

and alphas-eq: sqp-alphas prefix = staged-alphas data
by blast
have component:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
    ((replicate rounds []) [?i := trace-openings ! ?i])
    ((replicate rounds []) [?i := composition-openings ! ?i])
    ?i
    (Some ((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
by (rule
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixI-from-prefix-receive
    [OF i-bound builder prefix-receive alphas-eq opening-consistent])
show ?thesis
by (rule
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefixI
    [OF witness component])
qed

lemma checked-staged-security-with-actual-alpha-prefix-aligned-transcript-query-bad-imp-single-round-header-target-c
assumes wf: staged-budget-well-formed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some ((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)
and bad:
  checked-staged-security-with-actual-alpha-prefix-verifier-event
    query-bad-with-aligned-transcript-partial-candidates
    (Some ((((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state))
shows
  checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
    0 A
    (Some ((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state)) ∨
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
    0 A
    (Some ((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
proof –
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have data-support:
  Some (((data, attacker-state), result), final-state) ∈

```

```

    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF data-support]
  obtain builder verifier where builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
  have staged-header:
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
  from bad obtain fr f-fri-roots f-final as dg composition-fri-roots
    final query-idxs trace-openings composition-openings trace-table
    composition-table where partial:
    accepted-with-partial-initial-openings-aligned-transcript-consistent
      ?s (Some (result, final-state)) fr f-fri-roots f-final as dg
      composition-fri-roots final query-idxs trace-openings
      composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  and trace-low: trace-table-low-degree trace-table
  and comp-low:
    composition-table-low-degree maxDegree composition-table
  and not-all:
    ¬ all-queries-consistent trace-table composition-table as
  unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    query-bad-with-aligned-transcript-partial-candidates-def
  by auto
  have aligned:

```

```

    accepted-with-partial-initial-openings-aligned-consistent ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
  have shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
    [OF partial])
  from checked-staged-security-with-data-state-accepted-shape-query-chunks
    [OF wf controlled data-support shape]
  obtain raw-idxs where as-eq: as = staged-alphas data
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookup:
    ∀ i < rounds.
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)
  by blast
  have aligned0:
    accepted-with-partial-initial-openings-aligned ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
  by (rule accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
    [OF aligned])
  from accepted-with-partial-initial-openings-aligned-shapes[OF aligned0]
  obtain rest where header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  by blast
  have comp-nonempty: composition-fri-roots ≠ []
  using accepted-with-partial-initial-openings-aligned-shapes(1)
    [OF aligned0] .
  have trace-partial:
    accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr query-idxs trace-openings
  using accepted-with-partial-initial-openings-aligned-shapes(3)
    [OF aligned0] .
  have comp-partial:

```

```

accepted-with-partial-composition-openings ?s
  (Some (result, final-state)) (hd composition-fri-roots) query-idxs
  composition-openings
using accepted-with-partial-initial-openings-aligned-shapes(4)
  [OF aligned0] .
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
let ?i = 0
have i-bound: ?i < rounds
  using rounds-positive by simp
have round-consistent:
  partial-query-round-consistent trace-openings composition-openings as
    ?i (query-idxs ! ?i)
  using aligned i-bound
  unfolding accepted-with-partial-initial-openings-aligned-consistent-def
  by simp
have opening-consistent:
  partial-query-openings-consistent (trace-openings ! ?i)
    (composition-openings ! ?i) (staged-alphas data)
    (index (to-nat (raw-idxs ! ?i)))
  using round-consistent query-idxs-eq len-raw i-bound header-eq
  unfolding partial-query-round-consistent-def
    partial-query-openings-consistent-def
  by simp
have trace-table-auth:
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! ?i) final-state
  using trace-partial i-bound
  unfolding accepted-with-partial-trace-openings-def by blast
have comp-table-auth:
  partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) (composition-openings ! ?i) final-state
  using comp-partial i-bound
  unfolding accepted-with-partial-composition-openings-def by blast
have witness-header:
  (trace-openings ! ?i, composition-openings ! ?i) ∈
    query-header-supported-partial-opening-witnesses ?s fr f-fri-roots
    f-final as dg composition-fri-roots final
  unfolding query-header-supported-partial-opening-witnesses-def
  using comp-nonempty verifier trace-table-auth comp-table-auth header

```



```

by blast
have witness:
  (trace-openings ! ?i, composition-openings ! ?i) ∈
    query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
using witness-header header-eq by simp
have lookup0:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge ?i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) ?i)) =
    Some (raw-idxs ! ?i)
using lookup i-bound by simp
from checked-staged-transcript-program-query-prefix-receive-final-lookup-support-with-alphas
  [OF wf controlled i-bound builder verifier lookup0]
obtain prefix prefix-state raw-state where prefix-receive:
  Some (((prefix, prefix-state), raw-idxs ! ?i), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A ?i)
        adversary-initial-state)
and alphas-eq: sqp-alphas prefix = staged-alphas data
by blast
have trace-candidate-single:
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [?i := trace-openings ! ?i])
by (rule partial-trace-table-candidate-single-round-from-all-rounds
    [OF trace-candidate i-bound])
have comp-candidate-single:
  partial-composition-table-candidate composition-table
    ((replicate rounds []) [?i := composition-openings ! ?i])
by (rule partial-composition-table-candidate-single-round-from-all-rounds
    [OF comp-candidate i-bound])
have opening-target:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [?i := trace-openings ! ?i])
      ((replicate rounds []) [?i := composition-openings ! ?i])
      ?i)
    (Some (((prefix, prefix-state), raw-idxs ! ?i), raw-state))
by (rule checked-staged-query-prefix-candidate-opening-target-from-prefix-single-round-hitI
    [OF i-bound])
  (use opening-consistent alphas-eq in simp)

```

have *not-all-prefix*:
 $\neg$ *all-queries-consistent trace-table composition-table*
(spp-alphas prefix)
using *not-all header-eq alphas-eq* **by** *simp*
have *opening-subset*:
staged-query-prefix-candidate-opening-query-target-from-prefix
 $((\text{replicate rounds } \square) [?i := \text{trace-openings ! } ?i])$
 $((\text{replicate rounds } \square) [?i := \text{composition-openings ! } ?i])$
 $?i \text{ prefix prefix-state} \subseteq$
staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
composition-table prefix prefix-state
by *(rule*
staged-query-prefix-candidate-opening-query-target-from-prefix-subset-candidate-pair-from-prefix
 $[OF \text{ trace-candidate-single comp-candidate-single trace-low comp-low}$
 $\text{not-all-prefix}]$
have *pair-target*:
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
composition-table)
(Some (((prefix, prefix-state), raw-idxs ! ?i), raw-state))
using *opening-target opening-subset*
unfolding *checked-staged-query-prefix-dynamic-index-hit-def* **by** *auto*
have *pair-hit*:
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
A trace-table composition-table ?i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
by *(rule*
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixI
 $[OF \text{ i-bound builder prefix-receive pair-target}]$
show *?thesis*
by *(rule*
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-with-witness-imp-single-round
 $[OF \text{ pair-hit witness trace-candidate-single comp-candidate-single}]$
qed

definition *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-exists-at*
where
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-exists-at
 $i \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-openings composition-openings trace-table composition-table.}$
 $\text{checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at}$
 $\text{trace-openings composition-openings trace-table composition-table } i \text{ out})$

definition *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-current-exists-at*
where
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-current-exists-at
 $i \text{ out} \longleftrightarrow$
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at

$i \text{ out} \wedge$
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-exists-at
 $i \text{ out}$

definition *checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at*
where

checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at
 $i \text{ out} \longleftrightarrow$
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
 $i \text{ out} \wedge$
checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
 $i \text{ out}$

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-current-exists-bound-from-*
assumes *current-bound:*

wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at*
 i)
 $\text{adversary-initial-state} \leq C$

shows

wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-current-exists-at*
 i)
 $\text{adversary-initial-state} \leq C$

by (rule *order-trans*[*OF* - *current-bound*],
 rule *wp-event-mono*)

(simp add:
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-current-exists-at-def)

lemma *checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-bound-from-*
assumes *current-bound:*

wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at*
 i)
 $\text{adversary-initial-state} \leq C$

shows

wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at*
 i)
 $\text{adversary-initial-state} \leq C$

by (rule *order-trans*[*OF* - *current-bound*],
 rule *wp-event-mono*)

(simp add:
checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at-def)

lemma *checked-staged-security-with-query-prefix-data-state-aligned-transcript-query-bad-imp-single-round-component*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A 0)
adversary-initial-state)
and *bad:*
staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates
(Some (((data, attacker-state), result), final-state))
shows
checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at
0
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state)) ∨
checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at
0
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))
proof –
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
let *?i = 0*
have *i-bound: ?i < rounds*
using *rounds-positive* **by** *simp*
have *data-support:*
Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute (checked-staged-security-experiment-with-data-state A)
adversary-initial-state)
by (rule
checked-staged-security-experiment-with-query-prefix-data-state-imp-data-state-support
[OF i-bound support])
from *checked-staged-security-experiment-with-data-state-outcomeE*
[OF data-support]
obtain *builder verifier* **where** *builder:*
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *verifier:*
Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by *blast*
have *staged-header:*

```

verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
from bad obtain fr f-fri-roots f-final as dg composition-fri-roots
  final query-idxs trace-openings composition-openings trace-table
  composition-table where partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent
    ?s (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-low:
  composition-table-low-degree maxDegree composition-table
and not-all:
  ¬ all-queries-consistent trace-table composition-table as
unfolding staged-security-with-data-state-verifier-event-def
  query-bad-with-aligned-transcript-partial-candidates-def
by auto
have aligned:
  accepted-with-partial-initial-openings-aligned-consistent ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
by (rule
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
  [OF partial])
have shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
by (rule
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
  [OF partial])
have index-eq: index (to-nat raw) = query-idxs ! ?i
by (rule
  checked-staged-security-with-query-prefix-data-state-accepted-shape-query-index
  [OF wf controlled i-bound support shape])

```

```

have aligned0:
  accepted-with-partial-initial-openings-aligned ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
  by (rule accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
    [OF aligned])
from accepted-with-partial-initial-openings-aligned-shapes[OF aligned0]
obtain rest where header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  by blast
have comp-nonempty: composition-fri-roots ≠ []
  using accepted-with-partial-initial-openings-aligned-shapes(1)
    [OF aligned0] .
have trace-partial:
  accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr query-idxs trace-openings
  using accepted-with-partial-initial-openings-aligned-shapes(3)
    [OF aligned0] .
have comp-partial:
  accepted-with-partial-composition-openings ?s
    (Some (result, final-state)) (hd composition-fri-roots) query-idxs
    composition-openings
  using accepted-with-partial-initial-openings-aligned-shapes(4)
    [OF aligned0] .
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF header staged-header]
  by simp
have alphas-eq: staged-alphas data = sqp-alphas prefix
  by (rule checked-staged-security-with-query-prefix-data-state-alphas-eq)
    (rule support)
have round-consistent:
  partial-query-round-consistent trace-openings composition-openings as
    ?i (query-idxs ! ?i)
  using aligned i-bound
  unfolding accepted-with-partial-initial-openings-aligned-consistent-def
  by simp
have opening-consistent:
  partial-query-openings-consistent (trace-openings ! ?i)
    (composition-openings ! ?i) (sqp-alphas prefix)

```

```

    (index (to-nat raw))
using round-consistent index-eq header-eq alphas-eq
unfolding partial-query-round-consistent-def
    partial-query-openings-consistent-def
by simp
have trace-table-auth:
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! ?i) final-state
using trace-partial i-bound
unfolding accepted-with-partial-trace-openings-def by blast
have comp-table-auth:
    partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) (composition-openings ! ?i) final-state
using comp-partial i-bound
unfolding accepted-with-partial-composition-openings-def by blast
have witness-header:
    (trace-openings ! ?i, composition-openings ! ?i) ∈
    query-header-supported-partial-opening-witnesses ?s fr f-fri-roots
    f-final as dg composition-fri-roots final
unfolding query-header-supported-partial-opening-witnesses-def
using comp-nonempty verifier trace-table-auth comp-table-auth header
by blast
have witness:
    (trace-openings ! ?i, composition-openings ! ?i) ∈
    query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
using witness-header header-eq by simp
have trace-candidate-single:
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [?i := trace-openings ! ?i])
by (rule partial-trace-table-candidate-single-round-from-all-rounds
    [OF trace-candidate i-bound])
have comp-candidate-single:
    partial-composition-table-candidate composition-table
    ((replicate rounds []) [?i := composition-openings ! ?i])
by (rule partial-composition-table-candidate-single-round-from-all-rounds
    [OF comp-candidate i-bound])
have opening-target:
    checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-opening-query-target-from-prefix
    ((replicate rounds []) [?i := trace-openings ! ?i])
    ((replicate rounds []) [?i := composition-openings ! ?i])
    ?i)

```

```

    (Some (((prefix, prefix-state), raw), raw-state))
  by (rule checked-staged-query-prefix-candidate-opening-target-from-prefix-single-round-hitI
    [OF i-bound opening-consistent])
  have not-all-prefix:
    ¬ all-queries-consistent trace-table composition-table
      (sqp-alphas prefix)
  using not-all header-eq alphas-eq by simp
  have opening-subset:
    staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [?i := trace-openings ! ?i])
      ((replicate rounds []) [?i := composition-openings ! ?i])
      ?i prefix prefix-state ⊆
    staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
      composition-table prefix prefix-state
  by (rule
    staged-query-prefix-candidate-opening-query-target-from-prefix-subset-candidate-pair-from-prefix
      [OF trace-candidate-single comp-candidate-single trace-low comp-low
        not-all-prefix])
  have pair-target:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
        composition-table)
      (Some (((prefix, prefix-state), raw), raw-state))
  using opening-target opening-subset
  unfolding checked-staged-query-prefix-dynamic-index-hit-def by auto
  have auth-hit:
    checked-staged-security-with-query-prefix-authenticated-opening-hit
      (trace-openings ! ?i) (composition-openings ! ?i) ?i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-authenticated-opening-hit-def
  using comp-nonempty trace-table-auth comp-table-auth opening-consistent
    header-eq alphas-eq
  by simp
  show ?thesis
  proof (cases
    (trace-table, composition-table) ∈
      query-header-supported-single-round-partial-table-candidates
        prefix-state
        (sqp-trace-root prefix)
        (sqp-trace-fri-roots prefix)
        (sqp-trace-final prefix)
        (sqp-alphas prefix)
        (sqp-degree prefix)
        (sqp-composition-fri-roots prefix)
        (sqp-composition-final prefix)
        ?i)
  case True

```



```

have target-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-single-round-header-supported-query-target ?i)
    (Some (((prefix, prefix-state), raw), raw-state))
  by (rule
    checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
    [OF True pair-target])
have
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at
    ?i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at-def
    checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-def
  using target-hit
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-atI
    [OF auth-hit]
  by simp
then show ?thesis by simp
next
case False
have success-prefix:
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table
    (sqp-alphas prefix)
  using pair-target
  unfolding
    checked-staged-query-prefix-dynamic-index-hit-def
    staged-query-prefix-candidate-pair-query-target-from-prefix-def
  by simp
have gap-witness:
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
    (trace-openings ! ?i) (composition-openings ! ?i)
    trace-table composition-table ?i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  by (rule
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-atI
    [OF success-prefix witness trace-candidate-single
      comp-candidate-single False])
have gap-exists:
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at
    ?i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  by (rule
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-atI
    [OF gap-witness])

```

```

then show ?thesis
  using gap-exists by simp
qed
qed

lemma checked-staged-security-with-query-prefix-data-state-aligned-transcript-query-bad-extract-round0-authenticated
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A 0)
      adversary-initial-state)
and bad:
  staged-security-with-data-state-verifier-event
    query-bad-with-aligned-transcript-partial-candidates
    (Some (((data, attacker-state), result), final-state))
obtains trace-opening composition-opening trace-table composition-table where
  checked-staged-security-with-query-prefix-authenticated-opening-hit
  trace-opening composition-opening 0
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
  (trace-opening, composition-opening) ∈
  query-header-supported-partial-opening-witnesses
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [0 := trace-opening])
  partial-composition-table-candidate composition-table
    ((replicate rounds []) [0 := composition-opening])
  trace-table-low-degree trace-table
  composition-table-low-degree maxDegree composition-table
  ¬ all-queries-consistent trace-table composition-table
    (sqp-alphas prefix)
proof -
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
let ?i = 0
have i-bound: ?i < rounds

```

using *rounds-positive* **by** *simp*
have *data-support*:
Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute (checked-staged-security-experiment-with-data-state A)
adversary-initial-state)
by (rule
checked-staged-security-experiment-with-query-prefix-data-state-imp-data-state-support
[OF i-bound support])
from *checked-staged-security-experiment-with-data-state-outcomeE*
[OF data-support]
obtain *builder verifier* **where** *builder*:
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *verifier*:
Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by *blast*
have *staged-header*:
verifier-header-transcript ?s
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
(List.concat (staged-query-chunks data))
by (rule *checked-staged-transcript-program-outcome-header-transcript*
[OF wf controlled builder])
from *bad* **obtain** *fr f-fri-roots f-final as dg composition-fri-roots*
final query-idxs trace-openings composition-openings trace-table
composition-table **where** *partial*:
accepted-with-partial-initial-openings-aligned-transcript-consistent
?s (Some (result, final-state)) fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings
composition-openings
and *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
and *comp-candidate*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-low*:
composition-table-low-degree maxDegree composition-table
and *not-all*:
 $\neg$ *all-queries-consistent trace-table composition-table as*
unfolding *staged-security-with-data-state-verifier-event-def*

```

    query-bad-with-aligned-transcript-partial-candidates-def
  by auto
have aligned:
  accepted-with-partial-initial-openings-aligned-consistent ?s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
have aligned0:
  accepted-with-partial-initial-openings-aligned ?s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
  by (rule accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
    [OF aligned])
from accepted-with-partial-initial-openings-aligned-shapes[OF aligned0]
obtain rest where header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
  by blast
have comp-nonempty: composition-fri-roots ≠ []
  using accepted-with-partial-initial-openings-aligned-shapes(1)
  [OF aligned0] .
have trace-partial:
  accepted-with-partial-trace-openings ?s
  (Some (result, final-state)) fr query-idxs trace-openings
  using accepted-with-partial-initial-openings-aligned-shapes(3)
  [OF aligned0] .
have comp-partial:
  accepted-with-partial-composition-openings ?s
  (Some (result, final-state)) (hd composition-fri-roots) query-idxs
  composition-openings
  using accepted-with-partial-initial-openings-aligned-shapes(4)
  [OF aligned0] .
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF header staged-header]
  by simp
have alphas-eq: staged-alphas data = sqp-alphas prefix
  by (rule checked-staged-security-with-query-prefix-data-state-alphas-eq)

```

```

    (rule support)
have shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
    [OF partial])
have index-eq: index (to-nat raw) = query-idxs ! ?i
by (rule
    checked-staged-security-with-query-prefix-data-state-accepted-shape-query-index
    [OF wf controlled i-bound support shape])
have round-consistent:
  partial-query-round-consistent trace-openings composition-openings as
    ?i (query-idxs ! ?i)
using aligned i-bound
unfolding accepted-with-partial-initial-openings-aligned-consistent-def
by simp
have opening-consistent:
  partial-query-openings-consistent (trace-openings ! ?i)
    (composition-openings ! ?i) (sqp-alphas prefix)
    (index (to-nat raw))
using round-consistent index-eq header-eq alphas-eq
unfolding partial-query-round-consistent-def
  partial-query-openings-consistent-def
by simp
have trace-table-auth:
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! ?i) final-state
using trace-partial i-bound
unfolding accepted-with-partial-trace-openings-def by blast
have comp-table-auth:
  partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) (composition-openings ! ?i) final-state
using comp-partial i-bound
unfolding accepted-with-partial-composition-openings-def by blast
have witness-header:
  (trace-openings ! ?i, composition-openings ! ?i) ∈
    query-header-supported-partial-opening-witnesses ?s fr f-fri-roots
    f-final as dg composition-fri-roots final
unfolding query-header-supported-partial-opening-witnesses-def
using comp-nonempty verifier trace-table-auth comp-table-auth header
by blast
have witness:
  (trace-openings ! ?i, composition-openings ! ?i) ∈
    query-header-supported-partial-opening-witnesses ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)

```

```

    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
using witness-header header-eq by simp
have trace-candidate-single:
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [?i := trace-openings ! ?i])
by (rule partial-trace-table-candidate-single-round-from-all-rounds
    [OF trace-candidate i-bound])
have comp-candidate-single:
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [?i := composition-openings ! ?i])
by (rule partial-composition-table-candidate-single-round-from-all-rounds
    [OF comp-candidate i-bound])
have auth-hit:
  checked-staged-security-with-query-prefix-authenticated-opening-hit
  (trace-openings ! ?i) (composition-openings ! ?i) ?i
  (Some (((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result), final-state))
unfolding
  checked-staged-security-with-query-prefix-authenticated-opening-hit-def
using comp-nonempty trace-table-auth comp-table-auth opening-consistent
  header-eq alphas-eq
by simp
have not-all-prefix:
  ¬ all-queries-consistent trace-table composition-table
  (sqp-alphas prefix)
using not-all header-eq alphas-eq by simp
show ?thesis
by (rule that[OF auth-hit witness trace-candidate-single
  comp-candidate-single trace-low comp-low not-all-prefix])
qed

```

lemma checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-query-prefix-single-round-com

fixes T Gap :: prob

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and target-bound:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A 0)

(checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at 0)

adversary-initial-state $\leq T$

and gap-bound:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A 0)

(checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at 0)

adversary-initial-state $\leq Gap$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq T + \text{Gap}$

proof –

let *?i* = 0

let *?m* =

checked-staged-security-experiment-with-query-prefix-data-state A ?i

let *?query* =

staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates

let *?projected* =

$\lambda \text{out. case out of}$
 $\text{None} \Rightarrow ?query \text{ None}$
 $| \text{Some (packed, t)} \Rightarrow ?query (\text{Some (snd packed, t)})$

let *?target* =

checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at
?i

let *?gap* =

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at
?i

have *i-bound: ?i < rounds*

using *rounds-positive* **by** *simp*

have *projection:*

wp-event (checked-staged-security-experiment-with-data-state A)

?query adversary-initial-state =

wp-event ?m ?projected adversary-initial-state

by (rule

checked-staged-security-experiment-with-query-prefix-data-state-projection-event
 $[OF \text{ i-bound}]$)

have *event-le:*

wp-event ?m ?projected adversary-initial-state $\leq$

wp-event ?m ($\lambda \text{out. ?target out} \vee ?gap \text{ out}$)

adversary-initial-state

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support: out $\in$ set-dist (execute ?m adversary-initial-state)*

and *projected: ?projected out*

show *?target out $\vee$?gap out*

proof (cases *out*)

case *None*

then show *?thesis*

using *projected*

unfolding *staged-security-with-data-state-verifier-event-def* **by** *simp*

next

case (*Some packed*)

then obtain *prefix prefix-state raw raw-state data attacker-state*

result final-state **where** *out-eq:*

```

    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
  have bad:
    staged-security-with-data-state-verifier-event
      query-bad-with-aligned-transcript-partial-candidates
        (Some (((data, attacker-state), result), final-state))
    using projected unfolding out-eq by simp
  show ?thesis
    unfolding out-eq
    by (rule
      checked-staged-security-with-query-prefix-data-state-aligned-transcript-query-bad-imp-single-round-components
        [OF wf controlled])
      (use support bad out-eq in simp-all)
  qed
qed
have union-bound:
  wp-event ?m ( $\lambda out. ?target\ out \vee ?gap\ out$ )
    adversary-initial-state  $\leq$ 
  wp-event ?m ?target adversary-initial-state +
  wp-event ?m ?gap adversary-initial-state
  by (rule wp-event-union-bound)
have sum-bound:
  wp-event ?m ?target adversary-initial-state +
  wp-event ?m ?gap adversary-initial-state  $\leq T + Gap$ 
  by (rule add-mono[OF target-bound gap-bound])
show ?thesis
  unfolding projection
  by (rule order-trans[OF event-le order-trans[OF union-bound sum-bound]])
qed

lemma checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-query-prefix-single-round-target-
  fixes T Gap :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and target-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A 0)
      (checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at
        0)
      adversary-initial-state  $\leq T$ 
  and gap-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A 0)
      (checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at
        0)
      adversary-initial-state  $\leq Gap$ 
  shows

```


wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq T + \text{Gap}$
by (*rule*
checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-query-prefix-single-round-compon
 [*OF wf controlled target-bound gap-bound*])

lemma *checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-query-prefix-single-round-tar*

fixes *T P Q* :: *prob*

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *target-bound*:

wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A 0*)
 (*checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at*
0)
adversary-initial-state $\leq T$

and *data-pre-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$

and *new-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq Q$

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq T + (P + Q)$

proof –

have *i-bound*: $0 < \text{rounds}$

using *rounds-positive* **by** *simp*

have *gap-bound*:

wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A 0*)
 (*checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at*
0)
adversary-initial-state $\leq P + Q$

by (*rule*

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at-bound-from-data-pre-c
 [*OF i-bound data-pre-bound new-bound*])

show *?thesis*

by (*rule*

checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-query-prefix-single-round-target-
 [*OF wf controlled target-bound gap-bound*])

qed

lemma *checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-header-authenticated-candidate*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *header-bound:*

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

(checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix A)

adversary-initial-state ≤ Q

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

query-bad-with-aligned-transcript-partial-candidates)

adversary-initial-state ≤ Q

proof –

let *?project =*

λx. ((snd (fst (fst x)), snd (fst x)), snd x)

let *?query =*

staged-security-with-data-state-verifier-event

query-bad-with-aligned-transcript-partial-candidates

let *?projected =*

λout. case out of

None ⇒ False

| Some (packed, t) ⇒ ?query (Some (?project packed, t))

have *projection:*

wp-event (checked-staged-security-experiment-with-data-state A)

?query adversary-initial-state =

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

?projected adversary-initial-state

proof –

have

wp-event (checked-staged-security-experiment-with-data-state A)

?query adversary-initial-state =

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A

≫ (λx. return (?project x)))

?query adversary-initial-state

using

checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection

[of A]

by *simp*

also have ... =

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

?projected adversary-initial-state

```

    by (subst wp-event-bind-return-map)
      (simp add: staged-security-with-data-state-verifier-event-def)
  finally show ?thesis
    by simp
qed
have event-le:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?projected adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
    A)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
            A)
          adversary-initial-state)
    and projected: ?projected out
  show
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
      A out
  proof (cases out)
    case None
    then show ?thesis
      using projected by simp
  next
    case (Some packed)
    then obtain alpha-prefix alpha-prefix-state data attacker-state
      result final-state where out-eq:
        out =
          Some (((((alpha-prefix, alpha-prefix-state), data),
            attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
    have bad:
      checked-staged-security-with-actual-alpha-prefix-verifier-event
        query-bad-with-aligned-transcript-partial-candidates out
    using projected unfolding out-eq
    by (simp add:
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      staged-security-with-data-state-verifier-event-def Let-def)
    show ?thesis
      unfolding out-eq
      by (rule

```

```

      checked-staged-security-with-actual-alpha-prefix-aligned-transcript-query-bad-imp-header-authenticated-candidate
      [OF wf controlled])
      (use support bad out-eq in simp-all)
qed
qed
show ?thesis
  unfolding projection
  by (rule order-trans[OF event-le header-bound])
qed

lemma checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-single-round-components:
  fixes T Gap :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and target-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
       A)
      (checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
       0 A)
      adversary-initial-state  $\leq$  T
  and gap-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
       A)
      (checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
       0 A)
      adversary-initial-state  $\leq$  Gap
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       query-bad-with-aligned-transcript-partial-candidates)
      adversary-initial-state  $\leq$  T + Gap
proof -
  let ?project =
     $\lambda x. ((snd (fst (fst x)), snd (fst x)), snd x)$ 
  let ?query =
    staged-security-with-data-state-verifier-event
    query-bad-with-aligned-transcript-partial-candidates
  let ?projected =
     $\lambda out. case\ out\ of$ 
      None  $\Rightarrow$  False
    | Some (packed, t)  $\Rightarrow$  ?query (Some (?project packed, t))
  let ?target =
    checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
    0 A
  let ?gap =
    checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
    0 A

```

```

have projection:
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?query adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?projected adversary-initial-state
proof -
  have
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?query adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
         $\gg (\lambda x. \text{return } (?project\ x))$ )
      ?query adversary-initial-state
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
also have ... =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?projected adversary-initial-state
  by (subst wp-event-bind-return-map)
    (simp add: staged-security-with-data-state-verifier-event-def)
finally show ?thesis
  by simp
qed
have event-le:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?projected adversary-initial-state  $\leq$ 
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ( $\lambda out. ?target\ out \vee ?gap\ out$ )
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out  $\in$ 
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  and projected: ?projected out
show ?target out  $\vee$  ?gap out
proof (cases out)
  case None
  then show ?thesis

```

```

    using projected by simp
next
case (Some packed)
then obtain alpha-prefix alpha-prefix-state data attacker-state
  result final-state where out-eq:
  out =
    Some (((((alpha-prefix, alpha-prefix-state), data),
              attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
have bad:
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  query-bad-with-aligned-transcript-partial-candidates out
  using projected unfolding out-eq
  by (simp add:
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    staged-security-with-data-state-verifier-event-def Let-def)
show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-actual-alpha-prefix-aligned-transcript-query-bad-imp-single-round-header-target-or
    [OF wf controlled])
  (use support bad out-eq in simp-all)
qed
qed
have union-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ( $\lambda out. ?target\ out \vee ?gap\ out$ )
  adversary-initial-state  $\leq$ 
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ?target adversary-initial-state +
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ?gap adversary-initial-state
  by (rule wp-event-union-bound)
have sum-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ?target adversary-initial-state +
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ?gap adversary-initial-state  $\leq T + Gap$ 
  by (rule add-mono[OF target-bound gap-bound])
show ?thesis
  unfolding projection
  by (rule order-trans[OF event-le order-trans[OF union-bound sum-bound]])
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-pair-success-witness-gapE*:

assumes *i-bound*: $i < \text{rounds}$

and *support*:

Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state) $\in$
set-dist
 (execute
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 adversary-initial-state)

and *success*:

$\bigwedge \text{prefix prefix-state raw raw-state.}$
Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\in$
set-dist
 (execute
 (checked-staged-security-experiment-with-query-prefix-data-state
 A i)
 adversary-initial-state) $\implies$
 $\text{sqp-alphas prefix} = \text{staged-alphas data} \implies$
index (to-nat raw) $\in$
query-sampling-success-space trace-table composition-table
 (staged-alphas data)

and *witness*:

(trace-openings, composition-openings) $\in$
query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)

and *trace-candidate*:

partial-trace-table-candidate trace-table
 ((replicate rounds []) [i := trace-openings])

and *comp-candidate*:

partial-composition-table-candidate composition-table
 ((replicate rounds []) [i := composition-openings])

and *notin*:

$\bigwedge \text{prefix prefix-state raw raw-state.}$
Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\in$
set-dist
 (execute
 (checked-staged-security-experiment-with-query-prefix-data-state
 A i)

$adversary-initial-state) \implies$
 $sqp-alphas\ prefix = staged-alphas\ data \implies$
 $(trace-table, composition-table) \notin$
 $query-header-supported-single-round-partial-table-candidates$
 $prefix-state$
 $(sqp-trace-root\ prefix)$
 $(sqp-trace-fri-roots\ prefix)$
 $(sqp-trace-final\ prefix)$
 $(sqp-alphas\ prefix)$
 $(sqp-degree\ prefix)$
 $(sqp-composition-fri-roots\ prefix)$
 $(sqp-composition-final\ prefix)$
 i
obtains $prefix\ prefix-state\ raw\ raw-state$ **where**
 $Some\ (((((prefix, prefix-state), raw), raw-state),$
 $((data, attacker-state), result)), final-state) \in$
 $set-dist$
 $(execute$
 $(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$
 $adversary-initial-state)$
 $checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at$
 $trace-openings\ composition-openings\ trace-table\ composition-table\ i$
 $(Some\ (((((prefix, prefix-state), raw), raw-state),$
 $((data, attacker-state), result)), final-state))$
proof –
from $checked-staged-security-with-actual-alpha-prefix-support-query-prefix-data-state-alphasE$
 $[OF\ i-bound\ support]$
obtain $prefix\ prefix-state\ raw\ raw-state$ **where** $qsupport:$
 $Some\ (((((prefix, prefix-state), raw), raw-state),$
 $((data, attacker-state), result)), final-state) \in$
 $set-dist$
 $(execute$
 $(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$
 $adversary-initial-state)$
and $alphas-eq: sqp-alphas\ prefix = staged-alphas\ data$
by $blast$
have $success-prefix:$
 $index\ (to-nat\ raw) \in$
 $query-sampling-success-space\ trace-table\ composition-table$
 $(sqp-alphas\ prefix)$
using $success[OF\ qsupport\ alphas-eq]\ alphas-eq$ **by** $simp$
have $notin-prefix:$
 $(trace-table, composition-table) \notin$
 $query-header-supported-single-round-partial-table-candidates$
 $prefix-state$
 $(sqp-trace-root\ prefix)$
 $(sqp-trace-fri-roots\ prefix)$
 $(sqp-trace-final\ prefix)$
 $(sqp-alphas\ prefix)$


```

    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
  i
  by (rule notin[OF qsupport alphas-eq])
have fixed-gap:
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
  trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
  by (rule checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-atI
    [OF success-prefix witness trace-candidate comp-candidate notin-prefix])
show ?thesis
  by (rule that[OF qsupport fixed-gap])
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-pair-success-header-targetE:*

assumes *i-bound: i < rounds*

and support:

*Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state) ∈
 set-dist
 (execute
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 adversary-initial-state)*

and success:

*∧ prefix prefix-state raw raw-state.
 Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) ∈
 set-dist
 (execute
 (checked-staged-security-experiment-with-query-prefix-data-state
 A i)
 adversary-initial-state) ⇒
 sqp-alphas prefix = staged-alphas data ⇒
 index (to-nat raw) ∈
 query-sampling-success-space trace-table composition-table
 (staged-alphas data)*

and supported:

*∧ prefix prefix-state raw raw-state.
 Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) ∈
 set-dist
 (execute
 (checked-staged-security-experiment-with-query-prefix-data-state
 A i)
 adversary-initial-state) ⇒
 sqp-alphas prefix = staged-alphas data ⇒*

(trace-table, composition-table) ∈
query-header-supported-single-round-partial-table-candidates
prefix-state
(sqp-trace-root prefix)
(sqp-trace-fri-roots prefix)
(sqp-trace-final prefix)
(sqp-alphas prefix)
(sqp-degree prefix)
(sqp-composition-fri-roots prefix)
(sqp-composition-final prefix)
i
obtains *prefix prefix-state raw raw-state* **where**
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)
checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))
proof –
from *checked-staged-security-with-actual-alpha-prefix-support-query-prefix-data-state-alphasE*
[OF i-bound support]
obtain *prefix prefix-state raw raw-state* **where** *qsupport:*
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)
and *alphas-eq: sqp-alphas prefix = staged-alphas data*
by *blast*
have *success-prefix:*
index (to-nat raw) ∈
query-sampling-success-space trace-table composition-table
(sqp-alphas prefix)
using *success[OF qsupport alphas-eq] alphas-eq* **by** *simp*
have *pair-target-hit:*
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((prefix, prefix-state), raw), raw-state))
using *success-prefix*
unfolding
checked-staged-query-prefix-dynamic-index-hit-def
staged-query-prefix-candidate-pair-query-target-from-prefix-def
by *simp*

```

have supported-prefix:
  (trace-table, composition-table) ∈
    query-header-supported-single-round-partial-table-candidates
      prefix-state
        (sqp-trace-root prefix)
        (sqp-trace-fri-roots prefix)
        (sqp-trace-final prefix)
        (sqp-alphas prefix)
        (sqp-degree prefix)
        (sqp-composition-fri-roots prefix)
        (sqp-composition-final prefix)
      i
  by (rule supported[OF qsupport alphas-eq])
have target-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-single-round-header-supported-query-target i)
    (Some (((prefix, prefix-state), raw), raw-state))
  by (rule
    checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
    [OF supported-prefix pair-target-hit])
have query-prefix-hit:
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
    i
    (Some ((((((prefix, prefix-state), raw), raw-state),
      (((data, attacker-state), result)), final-state)))
  unfolding
    checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-def
  using target-hit by simp
show ?thesis
  by (rule that[OF qsupport query-prefix-hit])
qed

end

end

```

theory *Soundness-Staged-Relevant-Drift*

imports

Soundness-Staged-Aligned-Current-Empty

Staged-Security-Experiment-Composition-Query-Aligned-Witness-Bridge

begin

Small downstream layer for the refined relevant-drift event. The purpose is to keep diagnostic and bridge lemmas out of the already large current-empty packaging theory.

context *soundness*

begin

definition *composition-randomization-bad-with-aligned-transcript-partial-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-randomization-bad-with-aligned-transcript-partial-candidates}$
 $s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots final}$
 $\text{query-idxs trace-openings composition-openings trace-table}$
 $\text{composition-table } f.$
 $\text{accepted-with-partial-initial-openings-aligned-transcript-consistent } s$
 $\text{out fr } f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots final}$
 $\text{query-idxs trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\text{degree } f < \text{clength} \wedge$
 $\text{trace-table} = \text{map } (\text{poly } f) \text{ eval-domain} \wedge$
 $\text{violated-constraints } f \neq \{\}$
 $\text{composition-table-low-degree } \text{maxDegree } \text{composition-table} \wedge$
 $\text{all-queries-consistent trace-table composition-table as} \wedge$
 $\text{common-denominator-degree-bounds } f \text{ as} \wedge$
 $\text{random-combination-common-denominator-hides-violations } f \text{ as})$

definition *soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization}$
 $s \text{ out} \longleftrightarrow$
 $\text{composition-randomization-bad-with-aligned-transcript-partial-candidates}$
 $s \text{ out} \vee$
 $\text{trace-fri-bad-with-partial-candidates } s \text{ out} \vee$
 $\text{composition-fri-bad-with-partial-candidates } s \text{ out} \vee$
 $\text{query-bad-with-aligned-transcript-partial-candidates } s \text{ out}$

definition *soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle}$
 $s \text{ out} \longleftrightarrow$
 $\text{partial-merkle-inconsistency-bad } s \text{ out} \vee$
 $\text{soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization}$
 $s \text{ out}$

lemma *accepted-aligned-transcript-partial-candidate-partition-aligned-randomization:*

assumes *false-statement:* $\neg \text{exists-valid-trace}$

and *partial:*

$\text{accepted-with-partial-initial-openings-aligned-transcript-consistent } s$
 $\text{out fr } f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots final}$

```

    query-idxs trace-openings composition-openings
and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
shows
    trace-fri-bad-with-partial-candidates s out  $\vee$ 
    composition-fri-bad-with-partial-candidates s out  $\vee$ 
    query-bad-with-aligned-transcript-partial-candidates s out  $\vee$ 
    composition-randomization-bad-with-aligned-transcript-partial-candidates
    s out
proof (cases trace-table-low-degree trace-table)
case False
have aligned:
    accepted-with-partial-initial-openings-aligned-consistent s out fr
    f-fri-roots f-final as dg composition-fri-roots final query-idxs
    trace-openings composition-openings
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
have trace-fri-bad-with-partial-candidates s out
unfolding trace-fri-bad-with-partial-candidates-def
using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
    [OF aligned]
    trace-candidate False by blast
then show ?thesis by blast
next
case True
note trace-low = True
show ?thesis
proof (cases composition-table-low-degree maxDegree composition-table)
case False
have aligned:
    accepted-with-partial-initial-openings-aligned-consistent s out fr
    f-fri-roots f-final as dg composition-fri-roots final query-idxs
    trace-openings composition-openings
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
have composition-fri-bad-with-partial-candidates s out
unfolding composition-fri-bad-with-partial-candidates-def
using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
    [OF aligned]
    trace-candidate composition-candidate trace-low False
by blast
then show ?thesis by blast
next
case True

```

```

note composition-low = True
show ?thesis
proof (cases all-queries-consistent trace-table composition-table as)
  case False
  have query-bad-with-aligned-transcript-partial-candidates s out
    unfolding query-bad-with-aligned-transcript-partial-candidates-def
    using partial trace-candidate composition-candidate trace-low
      composition-low False
    by blast
  then show ?thesis by blast
next
  case True
  note all-queries = True
  from trace-low obtain f where
    deg-f: degree f < clength
    and trace-table-eq: trace-table = map (poly f) eval-domain
    unfolding trace-table-low-degree-def by blast
  have violated: violated-constraints f ≠ {}
    by (rule false-statement-violated-constraints
      [OF false-statement deg-f])
  have aligned:
    accepted-with-partial-initial-openings-aligned-consistent s out fr
      f-fri-roots f-final as dg composition-fri-roots final query-idxs
      trace-openings composition-openings
    by (rule
      accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
      [OF partial])
  have unaligned:
    accepted-with-partial-initial-openings s out fr f-fri-roots f-final
      as dg composition-fri-roots final query-idxs trace-openings
      query-idxs composition-openings
    by (rule
      accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
      [OF aligned])
  have len-as: length as = length spec
    by (rule accepted-with-partial-initial-openings-alpha-length
      [OF unaligned])
  have bounds: common-denominator-degree-bounds f as
    by (rule common-denominator-degree-bounds-from-spec-degree-wellformed
      [OF spec-degree-wellformed-from-spec-query-margin deg-f])
  have hides:
    random-combination-common-denominator-hides-violations f as
    by (rule random-combination-common-denominator-hides-from-queries
      [OF len-as false-statement deg-f trace-table-eq composition-low
        all-queries bounds])
  have
    composition-randomization-bad-with-aligned-transcript-partial-candidates
      s out
    unfolding

```

```

      composition-randomization-bad-with-aligned-transcript-partial-candidates-def
    using partial trace-candidate composition-candidate deg-f
      trace-table-eq violated composition-low all-queries bounds hides
    by blast
  then show ?thesis by blast
qed
qed
qed

```

lemma *accepted-aligned-transcript-partial-candidate-partition-with-aligned-randomization-event*:

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *partial*:

accepted-with-partial-initial-openings-aligned-transcript-consistent s
 out fr f-fri-roots f-final as dg composition-fri-roots final
 query-idxs trace-openings composition-openings

and *trace-candidate*:

partial-trace-table-candidate trace-table trace-openings

and *composition-candidate*:

partial-composition-table-candidate composition-table
 composition-openings

shows

soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization
 s out

using

accepted-aligned-transcript-partial-candidate-partition-aligned-randomization
 [*OF false-statement partial trace-candidate composition-candidate*]

unfolding

soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-def

by *blast*

lemma *checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-union-bound*:

fixes *composition-error trace-fri-error'* *composition-fri-error'*
query-error :: *prob*

assumes *comp-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 composition-randomization-bad-with-aligned-transcript-partial-candidates)
 adversary-initial-state $\leq$ *composition-error*

and *trace-fri-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *trace-fri-error'*

and *comp-fri-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *composition-fri-error'*

and *query-bound*:

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    query-bad-with-aligned-transcript-partial-candidates)
  adversary-initial-state ≤ query-error
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization)
  adversary-initial-state ≤
  composition-error + trace-fri-error' + composition-fri-error' +
  query-error
proof -
let ?M = checked-staged-security-experiment-with-data-state A
let ?E = staged-security-with-data-state-verifier-event
let ?comp =
  ?E composition-randomization-bad-with-aligned-transcript-partial-candidates
let ?trace = ?E trace-fri-bad-with-partial-candidates
let ?comp-fri = ?E composition-fri-bad-with-partial-candidates
let ?query = ?E query-bad-with-aligned-transcript-partial-candidates
let ?partial =
  ?E soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization
have wp-event ?M ?partial adversary-initial-state ≤
  wp-event ?M
    (λout. ?comp out ∨ ?trace out ∨ ?comp-fri out ∨
      ?query out)
    adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
    soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-def
    split: option.splits prod.splits)
also have ... ≤
  wp-event ?M ?comp adversary-initial-state +
  wp-event ?M ?trace adversary-initial-state +
  wp-event ?M ?comp-fri adversary-initial-state +
  wp-event ?M ?query adversary-initial-state
by (rule wp-event-union-bound4)
also have ... ≤
  composition-error + trace-fri-error' + composition-fri-error' +
  query-error
by (intro add-mono comp-bound trace-fri-bound comp-fri-bound
  query-bound)
finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-with-merkle
fixes merkle-error composition-error trace-fri-error'
  composition-fri-error' query-error :: prob
assumes merkle-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)

```


*(staged-security-with-data-state-verifier-event
 partial-merkle-inconsistency-bad)*
adversary-initial-state $\leq$ *merkle-error*
and *comp-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
*(staged-security-with-data-state-verifier-event
 composition-randomization-bad-with-aligned-transcript-partial-candidates)*
adversary-initial-state $\leq$ *composition-error*
and *trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
*(staged-security-with-data-state-verifier-event
 trace-fri-bad-with-partial-candidates)*
adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
*(staged-security-with-data-state-verifier-event
 composition-fri-bad-with-partial-candidates)*
adversary-initial-state $\leq$ *composition-fri-error'*
and *query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
*(staged-security-with-data-state-verifier-event
 query-bad-with-aligned-transcript-partial-candidates)*
adversary-initial-state $\leq$ *query-error*
shows
wp-event (checked-staged-security-experiment-with-data-state A)
*(staged-security-with-data-state-verifier-event
 soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)*
adversary-initial-state $\leq$
merkle-error + composition-error + trace-fri-error' +
composition-fri-error' + query-error
proof –
have *partial-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
*(staged-security-with-data-state-verifier-event
 soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization)*
adversary-initial-state $\leq$
composition-error + trace-fri-error' + composition-fri-error' +
query-error
by (rule
checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-union-bound
[OF comp-bound trace-fri-bound comp-fri-bound query-bound])
have *wp-event (checked-staged-security-experiment-with-data-state A)*
*(staged-security-with-data-state-verifier-event
 soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)*
adversary-initial-state $\leq$
wp-event (checked-staged-security-experiment-with-data-state A)
*(staged-security-with-data-state-verifier-event
 partial-merkle-inconsistency-bad)*
adversary-initial-state +

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization)
  adversary-initial-state
proof –
  have event-le:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)
      adversary-initial-state ≤
      wp-event (checked-staged-security-experiment-with-data-state A)
        (λout.
          staged-security-with-data-state-verifier-event
            partial-merkle-inconsistency-bad out ∨
            staged-security-with-data-state-verifier-event
              soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization
                out)
          adversary-initial-state
        by (rule wp-event-mono)
          (auto simp: staged-security-with-data-state-verifier-event-def
            soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle-def
            split: option.splits prod.splits)
      also have ... ≤
        wp-event (checked-staged-security-experiment-with-data-state A)
          (staged-security-with-data-state-verifier-event
            partial-merkle-inconsistency-bad)
          adversary-initial-state +
        wp-event (checked-staged-security-experiment-with-data-state A)
          (staged-security-with-data-state-verifier-event
            soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization)
          adversary-initial-state
        by (rule wp-event-union-bound)
      finally show ?thesis .
    qed
  also have ... ≤
    merkle-error +
    (composition-error + trace-fri-error' + composition-fri-error' +
      query-error)
    by (intro add-mono merkle-bound partial-bound)
  finally show ?thesis
    by (simp add: algebra-simps)
qed

lemma accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-n
assumes false-statement: ¬ exists-valid-trace
and supp: out ∈ set-dist (execute verify-monad s)
and acc: accepted out
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg

```

```

    composition-fri-roots final rest
  and comp-nonempty: composition-fri-roots ≠ []
shows
  soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
    s out
proof -
  from acc obtain result final-state where out-eq:
    out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
  from verify-monad-supplied-header-transcript-shape
    [OF supp[unfolded out-eq] header]
  obtain query-idxs raw-idxs query-chunks where shape:
    accepted-transcript-shape s (Some (result, final-state)) as query-idxs
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-query-chunks: length query-chunks = rounds
  and transcript-rest:
    List.concat query-chunks @ PTranscript final-state = rest
  and query-chunk-header:
    ∧ i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
      f-fri-roots composition-fri-roots (query-chunks ! i)
  and replay-lookup-header:
    ∧ i. i < rounds ⇒
      fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter s + i)
      (state-after-query-chunks
      (verifier-header-state s fr f-fri-roots f-final as dg
      composition-fri-roots final)
      query-chunks i)) =
      Some (raw-idxs ! i)
  by blast
  from verify-monad-supplied-header-query-rounds
    [OF supp[unfolded out-eq] header]
  obtain f-fl fl query-state where f-roots:
    map snd f-fl = f-fri-roots
  and comp-roots: map snd fl = composition-fri-roots
  and query-transcript-state: PTranscript query-state = rest
  and query-hash-state:
    PState query-state =
      verifier-header-state s fr f-fri-roots f-final as dg
      composition-fri-roots final
  and query-counter: PQueryCounter query-state = PQueryCounter s
  and query-out:
    Some (result, final-state) ∈
      set-dist
      (execute
      (ntimes

```

```

      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds)
    query-state)
  by blast
have fl-nonempty: fl ≠ []
  using comp-nonempty comp-roots by (cases fl) simp-all
then obtain b composition-root fl-tail where fl-eq:
  fl = (b, composition-root) # fl-tail
  by (cases fl) auto
have transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
  using query-transcript-state transcript-rest by simp
have query-chunk:
  ∧i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
  using query-chunk-header f-roots comp-roots by simp
have replay-lookup:
  ∧i. i < rounds ⇒
    fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some (raw-idxs ! i)
  using replay-lookup-header query-hash-state query-counter by simp
from
  query-rounds-replay-accepted-with-partial-initial-openings-aligned-transcript-consistent
  [OF header comp-nonempty fl-eq f-roots comp-roots
    query-transcript-state query-hash-state query-counter query-out
    len-raw query-idxs-eq len-query-chunks transcript-query query-chunk
    replay-lookup]
obtain trace-openings composition-openings where partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
  by blast
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then show ?thesis
  unfolding out-eq
  soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle-def
  by simp
next
case False
have aligned0:
  accepted-with-partial-initial-openings-aligned s

```

```

    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
using partial
by (blast dest:
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    accepted-with-partial-initial-openings-aligned-consistent-imp-aligned)
obtain trace-table composition-table where trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
using
    accepted-with-partial-initial-openings-aligned-candidates-if-no-partial-merkle-bad
    [OF aligned0 False]
by blast
have bad:
    soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization
    s (Some (result, final-state))
by (rule
    accepted-aligned-transcript-partial-candidate-partition-with-aligned-randomization-event
    [OF false-statement partial trace-candidate composition-candidate])
then show ?thesis
    unfolding out-eq
    by (simp add:
    soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle-def)
qed
qed

lemma accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
assumes false-statement:  $\neg$  exists-valid-trace
and supp:  $out \in \text{set-dist } (\text{execute verify-monad } s)$ 
and acc: accepted out
and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
shows
    soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
    s out  $\vee$ 
    soundness-bad-event-partial-candidate-empty-header s out
proof (cases composition-fri-roots = [])
case False
then have comp-nonempty: composition-fri-roots  $\neq []$ 
by simp
have
    soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
    s out
by (rule
    accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)

```

```

      [OF false-statement supp acc header comp-nonempty])
    then show ?thesis by simp
next
case True
note comp-empty = True
from acc obtain result final-state where out-eq:
  out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then have
  soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
    s out
  unfolding out-eq
  soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle-def
  by simp
then show ?thesis by simp
next
case False
from verify-monad-accepted-with-partial-trace-openings-for-header
  [OF supp[unfolded out-eq] header]
obtain trace-query-idxs trace-openings where trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr trace-query-idxs trace-openings
  by blast
obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
  using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
  [OF trace-partial False]
  by blast
have accepted-out: accepted (Some (result, final-state))
  by (rule accepted-with-partial-trace-openings-imp-accepted
    [OF trace-partial])
let ?composition-table = replicate (scale * clength) final
have empty-partial:
  accepted-with-empty-composition-header-candidates s
  (Some (result, final-state)) fr f-fri-roots f-final as dg final
  trace-query-idxs trace-openings trace-table ?composition-table
  unfolding accepted-with-empty-composition-header-candidates-def
  by (intro conjI exI[of - rest])
  (use accepted-out header comp-empty trace-partial trace-candidate
    in simp-all)
have soundness-bad-event-partial-candidate-empty-header s
  (Some (result, final-state))
  by (rule accepted-empty-composition-header-candidate-partition
    [OF false-statement empty-partial])
then show ?thesis

```

```

    unfolding out-eq by simp
  qed
qed

lemma checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-and-empty-header-
  fixes partial-candidate-error empty-header-error :: prob
  assumes false-statement:  $\neg$  exists-valid-trace
  and partial-candidate-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)
        adversary-initial-state  $\leq$  partial-candidate-error
  and empty-header-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-partial-candidate-empty-header)
        adversary-initial-state  $\leq$  empty-header-error
  shows checked-staged-adversary-acceptance-probability A  $\leq$ 
    partial-candidate-error + empty-header-error
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?partial =
    ?E soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
  let ?empty = ?E soundness-bad-event-partial-candidate-empty-header
  let ?bad =  $\lambda$ out. ?partial out  $\vee$  ?empty out
  have accepted-bad:
    wp-event ?M accepted adversary-initial-state  $\leq$ 
      wp-event ?M ?bad adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support: out  $\in$  set-dist (execute ?M adversary-initial-state)
    and acc: accepted out
    show ?bad out
    proof (cases out)
      case None
      then show ?thesis
        using acc unfolding accepted-def by simp
    next
      case (Some packed)
      then obtain data attacker-state result final-state where out-eq:
        out = Some (((data, attacker-state), result), final-state)
        by (cases packed, auto split: prod.splits)
      let ?s =
        verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)
      have verifier:
        Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)
        using checked-staged-security-experiment-with-data-state-outcomeE

```

```

    [OF support[unfolded out-eq]]
  by blast
have verifier-acc: accepted (Some (result, final-state))
  unfolding accepted-def by simp
from verify-monad-accepted-transcript-shape[OF verifier]
obtain alphas query-idxs where shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  alphas query-idxs
  by blast
from shape obtain result' final-state' fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  header:
    verifier-header-transcript ?s fr f-fri-roots f-final alphas dg
    composition-fri-roots final rest
  by (elim accepted-transcript-shape-header-query-extraction)
have local-bad:
  soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
    ?s (Some (result, final-state))  $\vee$ 
    soundness-bad-event-partial-candidate-empty-header ?s
      (Some (result, final-state))
  by (rule
    accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
      [OF false-statement verifier verifier-acc header])
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
have bad-bound:
  wp-event ?M ?bad adversary-initial-state  $\leq$ 
    partial-candidate-error + empty-header-error
proof -
  have wp-event ?M ?bad adversary-initial-state  $\leq$ 
    wp-event ?M ?partial adversary-initial-state +
    wp-event ?M ?empty adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq$  partial-candidate-error + empty-header-error
  by (intro add-mono partial-candidate-bound empty-header-bound)
  finally show ?thesis .
qed
have staged-bound:
  wp-event (checked-staged-security-experiment A) accepted
    adversary-initial-state  $\leq$ 
    partial-candidate-error + empty-header-error
  unfolding checked-staged-security-experiment-acceptance-with-data-state
  by (rule order-trans[OF accepted-bad bad-bound])
show ?thesis
proof -
  have

```



```

wp-event (checked-staged-security-experiment A)
  (λout. ¬ Option.is-none out) adversary-initial-state =
wp-event (checked-staged-security-experiment A) accepted
  adversary-initial-state
unfolding wp-event-def accepted-def
by (rule arg-cong[where
  f=λQ. wp (checked-staged-security-experiment A) Q
  adversary-initial-state])
  (rule ext, simp)
then show ?thesis
unfolding checked-staged-adversary-acceptance-probability-def
using staged-bound by simp
qed
qed

lemma checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-components-and-en
fixes merkle-error composition-error trace-fri-error'
  composition-fri-error' query-error empty-header-error :: prob
assumes false-statement: ¬ exists-valid-trace
and merkle-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      partial-merkle-inconsistency-bad)
    adversary-initial-state ≤ merkle-error
and comp-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-randomization-bad-with-aligned-transcript-partial-candidates)
    adversary-initial-state ≤ composition-error
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ composition-fri-error'
and query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      query-bad-with-aligned-transcript-partial-candidates)
    adversary-initial-state ≤ query-error
and empty-header-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      soundness-bad-event-partial-candidate-empty-header)
    adversary-initial-state ≤ empty-header-error

```

shows *checked-staged-adversary-acceptance-probability* $A \leq$
merkle-error + *composition-error* + *trace-fri-error'* +
composition-fri-error' + *query-error* + *empty-header-error*
proof –
have *partial-candidate-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)
adversary-initial-state $\leq$
merkle-error + *composition-error* + *trace-fri-error'* +
composition-fri-error' + *query-error*
by (*rule*
checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-with-merkle-u
[*OF merkle-bound comp-bound trace-fri-bound comp-fri-bound*
query-bound])
have *bound*:
checked-staged-adversary-acceptance-probability $A \leq$
(*merkle-error* + *composition-error* + *trace-fri-error'* +
composition-fri-error' + *query-error*) + *empty-header-error*
by (*rule*
checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-and-empty-header-bou
[*OF false-statement partial-candidate-bound empty-header-bound*])
then show *?thesis*
by (*simp add: algebra-simps*)
qed

lemma *accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-current-query-on-support*:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support*:
Some (((*data*, *attacker-state*), *result*), *final-state*) $\in$
set-dist
(*execute*
(*checked-staged-security-experiment-with-data-state* A)
adversary-initial-state)
and *partial*:
accepted-with-partial-initial-openings-aligned-transcript-consistent
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
(*Some* (*result*, *final-state*)) *fr* *f-fri-roots f-final* *as* *dg*
composition-fri-roots final query-idxs trace-openings
composition-openings
shows
staged-security-with-data-state-current-query-partial-opening-hit
(*Some* (((*data*, *attacker-state*), *result*), *final-state*))
proof –
let *?s* =
verifier-state-from-adversary attacker-state
(*staged-proof-transcript data*)

```

from checked-staged-security-experiment-with-data-state-outcomeE
  [OF support]
obtain builder verifier where builder:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A
      adversary-initial-state)
and verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
have shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
by (rule
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
  [OF partial])
from checked-staged-security-with-data-state-accepted-shape-query-chunks
  [OF wf controlled support shape]
obtain raw-idxs where as-eq: as = staged-alphas data
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup:
   $\forall i < \text{rounds.}$ 
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
by blast
have aligned:
  accepted-with-partial-initial-openings-aligned-consistent ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
by (rule

```

```

    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
have aligned0:
  accepted-with-partial-initial-openings-aligned ?s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
by (rule
  accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
  [OF aligned])
from accepted-with-partial-initial-openings-aligned-shapes[OF aligned0]
obtain rest where header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
by blast
have comp-nonempty: composition-fri-roots ≠ []
using accepted-with-partial-initial-openings-aligned-shapes(1)
  [OF aligned0] .
have trace-partial:
  accepted-with-partial-trace-openings ?s
  (Some (result, final-state)) fr query-idxs trace-openings
using accepted-with-partial-initial-openings-aligned-shapes(3)
  [OF aligned0] .
have comp-partial:
  accepted-with-partial-composition-openings ?s
  (Some (result, final-state)) (hd composition-fri-roots) query-idxs
  composition-openings
using accepted-with-partial-initial-openings-aligned-shapes(4)
  [OF aligned0] .
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
let ?i = 0
have i-bound: ?i < rounds
using rounds-positive by simp
have round-consistent:
  partial-query-round-consistent trace-openings composition-openings as
  ?i (query-idxs ! ?i)
using aligned i-bound
unfolding accepted-with-partial-initial-openings-aligned-consistent-def
by simp

```

```

have opening-consistent:
  partial-query-openings-consistent (trace-openings ! ?i)
    (composition-openings ! ?i) as (query-idxs ! ?i)
  using round-consistent
  unfolding partial-query-round-consistent-def
    partial-query-openings-consistent-def
  by simp
have trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) (trace-openings ! ?i) final-state
  using trace-partial i-bound header-eq
  unfolding accepted-with-partial-trace-openings-def by blast
have comp-auth:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) (composition-openings ! ?i) final-state
  using comp-partial i-bound header-eq
  unfolding accepted-with-partial-composition-openings-def by blast
have final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge ?i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) ?i)) =
      Some (raw-idxs ! ?i)
  using lookup i-bound by simp
have idx-raw:
  query-idxs ! ?i = index (to-nat (raw-idxs ! ?i))
  using query-idxs-eq len-raw i-bound by simp
have hit-at:
  staged-security-with-data-state-current-query-partial-opening-hit-at ?i
    (Some (((data, attacker-state), result), final-state))
  unfolding staged-security-with-data-state-current-query-partial-opening-hit-at-def
    Let-def
  apply simp
  by (intro exI[of - raw-idxs ! ?i]
      exI[of - trace-openings ! ?i]
      exI[of - composition-openings ! ?i] exI[of - rest] conjI)
    (use i-bound final-lookup comp-nonempty verifier trace-auth comp-auth
      staged-header opening-consistent idx-raw header-eq as-eq in simp-all)
show ?thesis
  by (rule staged-security-with-data-state-current-query-partial-opening-hitI
    [OF i-bound hit-at])
qed

```

```

lemma checked-staged-security-with-data-state-aligned-partial-candidate-randomization-imp-current-query-on-support
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out ∈

```

```

    set-dist
    (execute
      (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
  and bad:
    staged-security-with-data-state-verifier-event
    composition-randomization-bad-with-aligned-transcript-partial-candidates
    out
  shows staged-security-with-data-state-current-query-partial-opening-hit out
proof (cases out)
case None
then show ?thesis
  using bad
  unfolding staged-security-with-data-state-verifier-event-def
    composition-randomization-bad-with-aligned-transcript-partial-candidates-def
  by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have bad-some:
  composition-randomization-bad-with-aligned-transcript-partial-candidates
  ?s (Some (result, final-state))
using bad
unfolding out-eq staged-security-with-data-state-verifier-event-def
  Let-def
by simp
obtain fr f-fri-roots f-final as dg composition-fri-roots final
  query-idxs trace-openings composition-openings trace-table
  composition-table f
where partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent
  ?s (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
using bad-some
unfolding
  composition-randomization-bad-with-aligned-transcript-partial-candidates-def
by blast
show ?thesis
unfolding out-eq
by (rule
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-current-query-on-support
  [OF wf controlled support[unfolded out-eq] partial])
qed

```

lemma *checked-staged-security-with-data-state-aligned-partial-candidate-randomization-le-current-query*:
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
shows

$$\begin{aligned} & wp\text{-}event \\ & \quad (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A) \\ & \quad (staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event \\ & \quad \quad composition\text{-}randomization\text{-}bad\text{-}with\text{-}aligned\text{-}transcript\text{-}partial\text{-}candidates) \\ & \quad adversary\text{-}initial\text{-}state \leq \\ & wp\text{-}event \\ & \quad (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A) \\ & \quad staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}current\text{-}query\text{-}partial\text{-}opening\text{-}hit \\ & \quad adversary\text{-}initial\text{-}state \\ & \text{by (rule wp-event-mono-on-support)} \\ & \text{(rule} \\ & \quad checked\text{-}staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}aligned\text{-}partial\text{-}candidate\text{-}randomization\text{-}imp\text{-}current\text{-}query\text{-}on\text{-}support \\ & \quad [OF wf controlled]) \end{aligned}$$

lemma *checked-staged-security-with-data-state-aligned-partial-candidate-randomization-bound-from-query-prefix-curr*:
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *round-bound*:

$$\bigwedge i. i < rounds \implies$$

$$\begin{aligned} & wp\text{-}event \\ & \quad (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}query\text{-}prefix\text{-}data\text{-}state\ A\ i) \\ & \quad (checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}current\text{-}query\text{-}partial\text{-}opening\text{-}hit\text{-}at\ i) \\ & \quad adversary\text{-}initial\text{-}state \leq C\ i \end{aligned}$$

shows

$$\begin{aligned} & wp\text{-}event \\ & \quad (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A) \\ & \quad (staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event \\ & \quad \quad composition\text{-}randomization\text{-}bad\text{-}with\text{-}aligned\text{-}transcript\text{-}partial\text{-}candidates) \\ & \quad adversary\text{-}initial\text{-}state \leq (\sum i < rounds. C\ i) \end{aligned}$$

proof –

have *current-bound*:

$$\begin{aligned} & wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A) \\ & \quad staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}current\text{-}query\text{-}partial\text{-}opening\text{-}hit \\ & \quad adversary\text{-}initial\text{-}state \leq (\sum i < rounds. C\ i) \end{aligned}$$

by (rule

$$checked\text{-}staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}current\text{-}query\text{-}partial\text{-}opening\text{-}hit\text{-}bound\text{-}from\text{-}query\text{-}prefix\text{-}rounds$$

$$[OF\ round\text{-}bound\ wf\ controlled])$$

show *?thesis*

by (rule *order-trans*[

$$OF$$

$$checked\text{-}staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}aligned\text{-}partial\text{-}candidate\text{-}randomization\text{-}le\text{-}current\text{-}query$$

$$current\text{-}bound])$$

(use *wf controlled in simp-all*)

qed

lemma *checked-staged-security-with-data-state-aligned-randomization-or-query-bad-le-current-query:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
wp-event
 (*checked-staged-security-experiment-with-data-state A*)
 ($\lambda out.$
 staged-security-with-data-state-verifier-event
 composition-randomization-bad-with-aligned-transcript-partial-candidates
 out $\vee$
 staged-security-with-data-state-verifier-event
 query-bad-with-aligned-transcript-partial-candidates out)
adversary-initial-state $\leq$
wp-event
 (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state
proof (rule *wp-event-mono-on-support*)
fix *out*
assume *support:*
out $\in$
set-dist
 (*execute*
 (*checked-staged-security-experiment-with-data-state A*)
 adversary-initial-state)
and *bad:*
staged-security-with-data-state-verifier-event
composition-randomization-bad-with-aligned-transcript-partial-candidates
out $\vee$
staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates out
show
staged-security-with-data-state-current-query-partial-opening-hit out
proof (cases
 staged-security-with-data-state-verifier-event
 composition-randomization-bad-with-aligned-transcript-partial-candidates
 out)
case *True*
then show *?thesis*
by (rule
 checked-staged-security-with-data-state-aligned-partial-candidate-randomization-imp-current-query-on-support
 [*OF wf controlled support*])
next
case *False*
then have *query-bad:*
staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates out
using *bad* **by** *simp*


```

show ?thesis
proof (cases out)
  case None
  then show ?thesis
    using query-bad
    unfolding staged-security-with-data-state-verifier-event-def
      query-bad-with-aligned-transcript-partial-candidates-def
    by simp
  next
    case (Some packed)
    then obtain data attacker-state result final-state where out-eq:
      out = Some (((data, attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  show ?thesis
    unfolding out-eq
    by (rule
      checked-staged-security-with-data-state-aligned-transcript-query-bad-imp-current-partial-opening-hit-on-support
      [OF wf controlled support[unfolded out-eq]
        query-bad[unfolded out-eq]])
  qed
qed
qed

lemma checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-with-merkle
fixes merkle-error current-query-error trace-fri-error'
  composition-fri-error' :: prob
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and merkle-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      partial-merkle-inconsistency-bad)
  adversary-initial-state ≤ merkle-error
and current-query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-partial-opening-hit
  adversary-initial-state ≤ current-query-error
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ composition-fri-error'
shows
  wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (staged-security-with-data-state-verifier-event
     soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)
    adversary-initial-state ≤
    merkle-error + current-query-error + trace-fri-error' +
    composition-fri-error'
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?merkle = ?E partial-merkle-inconsistency-bad
  let ?comp =
    ?E composition-randomization-bad-with-aligned-transcript-partial-candidates
  let ?trace = ?E trace-fri-bad-with-partial-candidates
  let ?comp-fri = ?E composition-fri-bad-with-partial-candidates
  let ?query = ?E query-bad-with-aligned-transcript-partial-candidates
  let ?current =
    staged-security-with-data-state-current-query-partial-opening-hit
  let ?combined = λout. ?comp out ∨ ?query out
  let ?partial =
    ?E soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
have event-le:
  wp-event ?M ?partial adversary-initial-state ≤
  wp-event ?M
  (λout. ?merkle out ∨ ?combined out ∨ ?trace out ∨
   ?comp-fri out)
  adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
   soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle-def
   soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-def
   split: option.splits prod.splits)
have union-bound:
  wp-event ?M
  (λout. ?merkle out ∨ ?combined out ∨ ?trace out ∨
   ?comp-fri out)
  adversary-initial-state ≤
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?combined adversary-initial-state +
  wp-event ?M ?trace adversary-initial-state +
  wp-event ?M ?comp-fri adversary-initial-state
by (rule wp-event-union-bound4)
have combined-bound:
  wp-event ?M ?combined adversary-initial-state ≤
  wp-event ?M ?current adversary-initial-state
by (rule
  checked-staged-security-with-data-state-aligned-randomization-or-query-bad-le-current-query
  [OF wf controlled])
have
  wp-event ?M ?partial adversary-initial-state ≤
  wp-event ?M ?merkle adversary-initial-state +

```

```

    wp-event ?M ?current adversary-initial-state +
    wp-event ?M ?trace adversary-initial-state +
    wp-event ?M ?comp-fri adversary-initial-state
  by (rule order-trans[OF event-le])
    (rule order-trans[OF union-bound],
     intro add-mono order-refl combined-bound)
also have ... ≤
  merkle-error + current-query-error + trace-fri-error' +
  composition-fri-error'
  by (intro add-mono merkle-bound current-query-bound trace-fri-bound
      comp-fri-bound)
finally show ?thesis .
qed

lemma checked-staged-security-with-actual-alpha-prefix-aligned-partial-candidate-randomization-imp-current-query-o
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
and bad:
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-randomization-bad-with-aligned-transcript-partial-candidates
  out
shows
  (case out of
    None ⇒ False
  | Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state) ⇒
    staged-security-with-data-state-current-query-partial-opening-hit
      (Some (((data, attacker-state), result), final-state)))
proof (cases out)
case None
case Some
  then show ?thesis
  using bad
  unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    composition-randomization-bad-with-aligned-transcript-partial-candidates-def
  by simp
next
case (Some packed)
  then obtain alpha-prefix alpha-prefix-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state)

```

```

    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have bad-some:
    composition-randomization-bad-with-aligned-transcript-partial-candidates
      ?s (Some (result, final-state))
  using bad
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    Let-def
  by simp
  obtain fr f-fri-roots f-final as dg composition-fri-roots final
    query-idxs trace-openings composition-openings trace-table
    composition-table f
  where partial:
    accepted-with-partial-initial-openings-aligned-transcript-consistent
      ?s (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
  using bad-some
  unfolding
    composition-randomization-bad-with-aligned-transcript-partial-candidates-def
  by blast
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support[unfolded out-eq]])
  have data-hit:
    staged-security-with-data-state-current-query-partial-opening-hit
      (Some (((data, attacker-state), result), final-state))
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-current-query-on-support
    [OF wf controlled data-support partial])
  then show ?thesis
    unfolding out-eq by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-aligned-partial-candidate-randomization-le-current-query:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-verifier-event

```

```

    composition-randomization-bad-with-aligned-transcript-partial-candidates)
  adversary-initial-state ≤
wp-event
  (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-current-query-partial-opening-hit
  adversary-initial-state
proof –
  let ?project =
    λx. ((snd (fst (fst x)), snd (fst x)), snd x)
  let ?projected =
    λout. case out of
      None ⇒ False
    | Some (packed, t) ⇒
      staged-security-with-data-state-current-query-partial-opening-hit
        (Some (?project packed, t))
  have event-le:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-verifier-event
        composition-randomization-bad-with-aligned-transcript-partial-candidates)
      adversary-initial-state ≤
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      ?projected adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
            A)
          adversary-initial-state)
  and bad:
    checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-randomization-bad-with-aligned-transcript-partial-candidates
    out
  show ?projected out
  proof (cases out)
  case None
  then show ?thesis
    using bad
    unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      composition-randomization-bad-with-aligned-transcript-partial-candidates-def
    by simp
  next
  case (Some packed)
  then obtain alpha-prefix alpha-prefix-state data attacker-state result
    final-state where out-eq:

```

```

    out =
      Some (((alpha-prefix, alpha-prefix-state), data),
            attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
have hit:
  staged-security-with-data-state-current-query-partial-opening-hit
  (Some (((data, attacker-state), result), final-state))
using
  checked-staged-security-with-actual-alpha-prefix-aligned-partial-candidate-randomization-imp-current-query-on-
  [OF wf controlled support bad]
  unfolding out-eq by simp
show ?thesis
  unfolding out-eq using hit by simp
qed
qed
have projection:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ?projected adversary-initial-state =
  wp-event
  (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-current-query-partial-opening-hit
  adversary-initial-state
proof -
  have
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-partial-opening-hit
    adversary-initial-state =
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
     >>= (λx. return (?project x)))
    staged-security-with-data-state-current-query-partial-opening-hit
    adversary-initial-state
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
also have ... =
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ?projected adversary-initial-state
  by (subst wp-event-bind-return-map)
  (simp add:
    staged-security-with-data-state-current-query-partial-opening-hit-def
    staged-security-with-data-state-current-query-partial-opening-hit-at-def)
  finally show ?thesis
    by simp
qed
show ?thesis

```

by (rule order-trans[*OF event-le*])
 (simp add: projection)
 qed

lemma *checked-staged-security-with-actual-alpha-prefix-aligned-partial-candidate-randomization-bound-from-query-pr*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *round-bound:*
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-randomization-bad-with-aligned-transcript-partial-candidates})$
 $\text{adversary-initial-state} \leq (\sum i < \text{rounds}. C \ i)$
proof –
have *current-bound:*
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-current-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq (\sum i < \text{rounds}. C \ i)$
by (rule
 $\text{checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds}$
 $[OF \text{ round-bound } wf \text{ controlled}]$)
show ?thesis
by (rule order-trans[
 OF
 $\text{checked-staged-security-with-actual-alpha-prefix-aligned-partial-candidate-randomization-le-current-query}$
 $\text{current-bound}]$)
 (use *wf controlled* in simp-all)
 qed

lemma *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-driftE:*
assumes
 $\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift}$
 out
obtains
 $\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift}$
 $\text{composition-trace-bad-alpha-space } out$
 $\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-randomization-bad-with-partial-candidates } out$
 $| \text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift}$
 $\text{composition-trace-bad-alpha-space } out$
 $\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-alpha-partial-opening-union-hit-with-empty-header } out$
using *assms*

unfolding

checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-def
by *blast*

lemma *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-le-branches:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
(*checked-staged-security-with-actual-alpha-prefix-verifier-event*
composition-randomization-bad-with-partial-candidates)
adversary-initial-state +

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
(*checked-staged-security-with-actual-alpha-prefix-verifier-event*
composition-alpha-partial-opening-union-hit-with-empty-header)
adversary-initial-state

proof —

have *event-le:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
($\lambda out.$

checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-randomization-bad-with-partial-candidates out $\vee$
checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-alpha-partial-opening-union-hit-with-empty-header out)
adversary-initial-state

by (rule *wp-event-mono*)

(auto simp:

checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-def)

have *union-le:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
($\lambda out.$

checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-randomization-bad-with-partial-candidates out $\vee$
checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-alpha-partial-opening-union-hit-with-empty-header out)
adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
(*checked-staged-security-with-actual-alpha-prefix-verifier-event*
composition-randomization-bad-with-partial-candidates)


```

    adversary-initial-state +
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-alpha-partial-opening-union-hit-with-empty-header)
  adversary-initial-state
by (rule wp-event-union-bound)
show ?thesis
by (rule order-trans[OF event-le union-le])
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-branch-bound*:

assumes *random-bound*:

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-randomization-bad-with-partial-candidates)
  adversary-initial-state  $\leq R$ 

```

and *empty-bound*:

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-alpha-partial-opening-union-hit-with-empty-header)
  adversary-initial-state  $\leq E$ 

```

shows

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
  adversary-initial-state  $\leq R + E$ 
by (rule order-trans[
  OF
  checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-le-branches])
(intro add-mono random-bound empty-bound)

```

lemma *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-le-not-prefix-bound*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

shows

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
  adversary-initial-state  $\leq$ 
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
    composition-trace-bad-alpha-space)
  adversary-initial-state

```

proof —

have *event-le*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
 composition-trace-bad-alpha-space)
adversary-initial-state
by (rule *wp-event-mono*)
 (auto simp:
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-def)
have *drift-le*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
 composition-trace-bad-alpha-space)
adversary-initial-state $\leq$
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
 composition-trace-bad-alpha-space)
adversary-initial-state
by (rule
 checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-le-not-prefix-bound
 [*OF wf controlled*])
show ?thesis
by (rule *order-trans*[*OF event-le drift-le*])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-not-prefix-*

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *collision-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
adversary-initial-state $\leq C$

and *uncovered-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state $\leq U$

and *no-prefix-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)

checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq N$
and *path-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq P$
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq C + U + N + P$
proof –
have *not-prefix-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)
adversary-initial-state $\leq C + U + N + P$
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
[OF wf controlled collision-bound uncovered-bound no-prefix-bound
path-bound])
have *relevant-le*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)
adversary-initial-state
by (rule
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-le-not-prefix-bound
[OF wf controlled])
show *?thesis*
by (rule *order-trans*[*OF relevant-le not-prefix-bound*])
qed
lemma *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-cross-and-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *cross-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad

$adversary-initial-state \leq X$
and *witness-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
 $adversary-initial-state \leq W$
shows
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 $adversary-initial-state \leq$
 $hash-collision-budget-value\ 0$
 (*staged-alpha-search-queries* *budgets* *0*) + $X + W + W$
proof –
have *collision-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
 $adversary-initial-state \leq$
 $hash-collision-budget-value\ 0$ (*staged-alpha-search-queries* *budgets* *0*)
by (rule
 checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
 [*OF* *wf* *controlled*])
have *uncovered-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
 $adversary-initial-state \leq X$
by (rule *order-trans*
 [*OF*
 checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad
 cross-bound])
have *no-prefix-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
 $adversary-initial-state \leq W$
by (rule *order-trans*
 [*OF*
 checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-le-witness-transcript-hit
 witness-bound])
have *path-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)

checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq W$
by (rule *order-trans*
[*OF*
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-le-witness-transcript-hit
witness-bound])
show *?thesis*
by (rule
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-not-prefix-com
[*OF wf controlled collision-bound uncovered-bound no-prefix-bound*
path-bound])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-cross-data*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *cross-bound:*
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state $\leq X$
and *data-pre-bound:*
wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *transcript-new-bound:*
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq R$
shows
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$
hash-collision-budget-value 0
(*staged-alpha-search-queries budgets 0*) + $X + (P + R) + (P + R)$
proof –
have *transcript-pre-bound:*
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
adversary-initial-state $\leq P$
by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
[*OF data-pre-bound*])
have *witness-bound:*

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
adversary-initial-state $\leq P + R$
by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
 [*OF transcript-pre-bound transcript-new-bound*])
show ?thesis
by (rule
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-cross-and-witness
 [*OF wf controlled cross-bound witness-bound*])
qed

lemma *checked-staged-soundness-from-aligned-partial-candidate-components-and-current-empty-query-from-cross-data*
fixes *trace-fri-error'* *composition-fri-error'* *query-error X P R*
empty-query-error :: prob
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*
and *query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-partial-candidates)
adversary-initial-state $\leq$ *query-error*
and *cross-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state $\leq X$
and *data-pre-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *transcript-new-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit

adversary-initial-state $\leq R$
and *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-current-empty-query-opening-hit-at* 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
(*composition-error-bound* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
(*staged-alpha-search-queries* *budgets* 0) +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
(*hash-collision-budget-value* 0
(*staged-alpha-search-queries* *budgets* 0) + $X + (P + R) + (P + R)$)) +
trace-fri-error' + *composition-fri-error'* +
query-error +
(*composition-error-bound* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
(*staged-alpha-search-queries* *budgets* 0) +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
(*hash-collision-budget-value* 0
(*staged-alpha-search-queries* *budgets* 0) + $X + (P + R) + (P + R)$) +
trace-fri-error + *empty-query-error*)
proof –
let $?D =$
hash-collision-budget-value 0
(*staged-alpha-search-queries* *budgets* 0) + $X + (P + R) + (P + R)$
have *drift-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq ?D$
by (rule
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-cross-data-pre
[*OF wf controlled cross-bound data-pre-bound transcript-new-bound*])
show $?thesis$
by (rule

checked-staged-soundness-from-aligned-partial-candidate-components-and-current-empty-query-from-relevant-dri
 [OF *false-statement* wf *controlled trace-fri-bound comp-fri-bound*
query-bound drift-bound empty-trace-fri-bound current-empty-bound])

qed

lemma *checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from*

fixes *trace-fri-error'* *composition-fri-error'* *query-error D*

empty-query-error :: prob

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

trace-fri-bad-with-partial-candidates)

adversary-initial-state $\leq$ *trace-fri-error'*

and *comp-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

composition-fri-bad-with-partial-candidates)

adversary-initial-state $\leq$ *composition-fri-error'*

and *query-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

query-bad-with-aligned-transcript-partial-candidates)

adversary-initial-state $\leq$ *query-error*

and *drift-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift

adversary-initial-state $\leq$ *D*

and *empty-trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

trace-fri-bad-with-empty-composition-header-candidates)

adversary-initial-state $\leq$ *trace-fri-error*

and *current-empty-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-current-empty-query-opening-hit-at 0*)

adversary-initial-state $\leq$ *empty-query-error*

shows *checked-staged-adversary-acceptance-probability A* $\leq$

hash-collision-budget-value 0

(*staged-attacker-query-budget budgets* +

staged-challenge-query-budget + *verifier-hash-query-budget*) +

(*composition-error-bound* +

staged-phase-relation-error size

(*staged-alpha-search-queries budgets 0*) +

hash-collision-budget-value 0

(staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D) +
 trace-fri-error' + composition-fri-error' +
 query-error +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D + trace-fri-error + empty-query-error)
proof –
have comp-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-bad-with-partial-candidates)
 adversary-initial-state ≤
 composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D
by (rule
 checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-relevant-drift-and-budgets
 [OF false-statement wf controlled drift-bound])
have merkle-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 partial-merkle-inconsistency-bad)
 adversary-initial-state ≤
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
 checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
 [OF wf controlled])
have empty-query-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state ≤ empty-query-error
by (rule order-trans[
 OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero[

```

    OF wf controlled] current-empty-bound])
have empty-header-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      soundness-bad-event-partial-candidate-empty-header)
  adversary-initial-state ≤
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  D + trace-fri-error + empty-query-error
by (rule
  checked-staged-security-with-data-state-empty-header-bad-event-bound-from-relevant-drift-trace-fri-and-query
  [OF false-statement wf controlled drift-bound empty-trace-fri-bound
    empty-query-bound])
show ?thesis
by (rule
  checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-bound
  [OF false-statement merkle-bound comp-bound trace-fri-bound
    comp-fri-bound query-bound empty-header-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-relevant-
fixes trace-fri-error' composition-fri-error' D empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ composition-fri-error'
and round-bound:
  ∧ i. i < rounds ⇒
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state ≤ C i
and drift-bound:
  wp-event

```

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq D$
and *empty-trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ trace-fri-error
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability $A \leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D) +
trace-fri-error' + composition-fri-error' +
($\sum i < \text{rounds. } C i$) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D + trace-fri-error + empty-query-error)
proof –
have *current-query-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq (\sum i < \text{rounds. } C i)$
by (rule
checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
[OF round-bound wf controlled])
have *query-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq (\sum i < \text{rounds. } C i)$
by (rule order-trans[

```

    OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
      current-query-bound])
  (use wf controlled in simp-all)
show ?thesis
by (rule
  checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-re
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      query-bound drift-bound empty-trace-fri-bound current-empty-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix
  fixes trace-fri-error' composition-fri-error' D empty-query-error :: prob
  and C :: nat  $\Rightarrow$  prob
  assumes false-statement:  $\neg$  exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-partial-candidates)
      adversary-initial-state  $\leq$  trace-fri-error'
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-partial-candidates)
      adversary-initial-state  $\leq$  composition-fri-error'
  and round-bound:
     $\bigwedge i. i < \text{rounds} \implies$ 
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
        adversary-initial-state  $\leq$  C i
  and drift-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
      adversary-initial-state  $\leq$  D
  and empty-trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-empty-composition-header-candidates)
      adversary-initial-state  $\leq$  trace-fri-error
  and current-empty-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
      adversary-initial-state  $\leq$  empty-query-error
  shows checked-staged-adversary-acceptance-probability A  $\leq$ 
    hash-collision-budget-value 0

```

$(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum_{i < \text{rounds}} C i) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum_{i < \text{rounds}} C i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $D + \text{trace-fri-error} + \text{empty-query-error})$
proof –
have *comp-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-randomization-bad-with-aligned-transcript-partial-candidates})$
 $\text{adversary-initial-state} \leq (\sum_{i < \text{rounds}} C i)$
by (rule
 $\text{checked-staged-security-with-data-state-aligned-partial-candidate-randomization-bound-from-query-prefix-current-}$
 $[\text{OF wf controlled round-bound}])$
have *current-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-current-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq (\sum_{i < \text{rounds}} C i)$
by (rule
 $\text{checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds}$
 $[\text{OF round-bound wf controlled}])$
have *query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-aligned-transcript-partial-candidates})$
 $\text{adversary-initial-state} \leq (\sum_{i < \text{rounds}} C i)$
by (rule *order-trans*[
 $\text{OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit}$
 $\text{current-query-bound}])$
 $(\text{use wf controlled in simp-all})$
have *merkle-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{partial-merkle-inconsistency-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget})$
by (rule
 $\text{checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled}$
 $[\text{OF wf controlled}])$

have *empty-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *empty-query-error*
by (rule *order-trans*[
OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero[
OF wf controlled] *current-empty-bound*])
have *empty-header-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
(*staged-alpha-search-queries budgets 0*) +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
D + trace-fri-error + empty-query-error
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-bound-from-relevant-drift-trace-fri-and-query
[*OF false-statement wf controlled drift-bound empty-trace-fri-bound*
empty-query-bound])
have *bound*:
checked-staged-adversary-acceptance-probability A $\leq$
hash-collision-budget-value 0
(*staged-attacker-query-budget budgets +*
staged-challenge-query-budget + verifier-hash-query-budget) +
($\sum i < \text{rounds}. C\ i$) +
trace-fri-error' + composition-fri-error' +
($\sum i < \text{rounds}. C\ i$) +
(*composition-error-bound +*
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
(*staged-alpha-search-queries budgets 0*) +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
D + trace-fri-error + empty-query-error)
by (rule
checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-components-and-empty-fri
[*OF false-statement merkle-bound comp-bound trace-fri-bound*
comp-fri-bound query-bound empty-header-bound])
then show ?thesis .
qed

lemma *checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-structured-query-fri*

fixes *trace-fri-error'* *composition-fri-error'* *D* *empty-query-error*
header-query-error :: *prob*
and *C* :: *nat* $\Rightarrow$ *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \Rightarrow$
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)
(*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* *i*)
adversary-initial-state $\leq$ *C* *i*
and *header-query-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*
A)
adversary-initial-state $\leq$ *header-query-error*
and *drift-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$ *D*
and *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-current-empty-query-opening-hit-at* *0*)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability* *A* $\leq$
hash-collision-budget-value *0*
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 $(\sum i < \text{rounds}. C\ i) +$

$\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $\text{header-query-error} +$
 $(\text{composition-error-bound} +$
 $\quad \text{staged-phase-relation-error size}$
 $\quad (\text{staged-alpha-search-queries budgets } 0) +$
 $\quad \text{hash-collision-budget-value } 0$
 $\quad (\text{staged-alpha-search-queries budgets } 0) +$
 $\quad \text{staged-phase-relation-error size}$
 $\quad (\text{staged-alpha-search-queries budgets } 0) +$
 $\quad D + \text{trace-fri-error} + \text{empty-query-error})$
proof –
have *comp-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\quad \text{composition-randomization-bad-with-aligned-transcript-partial-candidates})$
 $\text{adversary-initial-state} \leq (\sum_{i < \text{rounds}} C \ i)$
by (rule
 $\text{checked-staged-security-with-data-state-aligned-partial-candidate-randomization-bound-from-query-prefix-current-}$
 $\quad [\text{OF wf controlled round-bound}])$
have *query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\quad \text{query-bad-with-aligned-transcript-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{header-query-error}$
by (rule
 $\text{checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-header-authenticated-candidate-}$
 $\quad [\text{OF wf controlled header-query-bound}])$
have *merkle-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\quad \text{partial-merkle-inconsistency-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets } +$
 $\quad \text{staged-challenge-query-budget } + \text{verifier-hash-query-budget})$
by (rule
 $\text{checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled}$
 $\quad [\text{OF wf controlled}])$
have *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\quad \text{query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
by (rule *order-trans*[
 $\quad \text{OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero[}$
 $\quad \text{OF wf controlled}] \text{ current-empty-bound}])$
have *empty-header-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$


```

    soundness-bad-event-partial-candidate-empty-header)
  adversary-initial-state ≤
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  D + trace-fri-error + empty-query-error
by (rule
  checked-staged-security-with-data-state-empty-header-bad-event-bound-from-relevant-drift-trace-fri-and-query
  [OF false-statement wf controlled drift-bound empty-trace-fri-bound
    empty-query-bound])
have bound:
  checked-staged-adversary-acceptance-probability A ≤
  (hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget + verifier-hash-query-budget) +
    (∑ i<rounds. C i) + trace-fri-error' +
    composition-fri-error' + header-query-error) +
  (composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D + trace-fri-error + empty-query-error)
by (rule
  checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-components-and-empty-header
  [OF false-statement merkle-bound comp-bound trace-fri-bound
    comp-fri-bound query-bound empty-header-bound])
then show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix
  fixes trace-fri-error' composition-fri-error' D empty-query-error :: prob
  and C :: nat ⇒ prob
  assumes false-statement: ¬ exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ trace-fri-error'
  and comp-fri-bound:

```

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*
and *round-bound*:
 $\bigwedge i. i < rounds \implies$
 $wp\text{-}event$
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* i)
adversary-initial-state $\leq C$ i
and *drift-bound*:
 $wp\text{-}event$
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq D$
and *empty-trace-fri-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-current-empty-query-opening-hit-at* 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 $(\sum i < rounds. C$ $i)$ +
trace-fri-error' + *composition-fri-error'* +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
 $D + trace\text{-}fri\text{-}error + empty\text{-}query\text{-}error$)
proof –
have *current-query-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq (\sum i < rounds. C$ $i)$
by (*rule*
checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
 [*OF* *round-bound* *wf* *controlled*])
have *merkle-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)

(staged-security-with-data-state-verifier-event
 partial-merkle-inconsistency-bad)
 adversary-initial-state $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
 checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
 [OF wf controlled])
have partial-candidate-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)
 adversary-initial-state $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds}. C\ i$) + trace-fri-error' + composition-fri-error'
by (rule
 checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-with-merkle-c
 [OF wf controlled merkle-bound current-query-bound trace-fri-bound
 comp-fri-bound])
have empty-query-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ empty-query-error
by (rule order-trans[
 OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero[
 OF wf controlled] current-empty-bound])
have empty-header-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 soundness-bad-event-partial-candidate-empty-header)
 adversary-initial-state $\leq$
 composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D + trace-fri-error + empty-query-error
by (rule
 checked-staged-security-with-data-state-empty-header-bad-event-bound-from-relevant-drift-trace-fri-and-query
 [OF false-statement wf controlled drift-bound empty-trace-fri-bound
 empty-query-bound])
have bound:
 checked-staged-adversary-acceptance-probability A $\leq$

(*hash-collision-budget-value* 0
 (*staged-attacker-query-budget* *budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*) +
 ($\sum i < \text{rounds}. C\ i$) + *trace-fri-error'* +
 composition-fri-error') +
 (*composition-error-bound* +
 staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
 D + *trace-fri-error* + *empty-query-error*)
 by (rule
 checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-and-empty-header-bound
 [*OF* *false-statement* *partial-candidate-bound* *empty-header-bound*])
 then show ?thesis
 by (simp add: *algebra-simps*)
 qed

lemma *checked-staged-soundness-from-aligned-transcript-conceptual-default-alpha-prefix-union-clean-structured-path*
fixes *trace-fri-error'* *composition-fri-error'* *D* *empty-query-error* :: *prob*
and *T C* :: *nat* $\Rightarrow$ *prob*
and *trace-default* *composition-default* :: '*f* *list*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *composition-fri-error'*
and *default-prefix-clean*:
 $\bigwedge i$ *prefix* *prefix-state*.
 i < *rounds* $\implies$
 Some ((*prefix*, *prefix-state*), *prefix-state*) $\in$
 set-dist
 (*execute* (*checked-staged-query-prefix-with-state* *A* *i*)
 adversary-initial-state) $\implies$
 query-prefix-trace-conceptual-table-with-default *prefix* *prefix-state*
 trace-default = *trace-default* $\wedge$
 query-prefix-composition-conceptual-table-with-default *prefix*
 prefix-state *composition-default* = *composition-default* $\wedge$
 trace-table-low-degree *trace-default* $\wedge$

composition-table-low-degree *maxDegree* *composition-default* $\wedge$
trace-default $\in$
alpha-prefix-trace-table-candidates *prefix-state*
(sqp-trace-root prefix) $\wedge$
sqp-alphas prefix $\notin$
alpha-prefix-union-bad-sets *prefix-state*
composition-trace-bad-alpha-space (*sqp-trace-root prefix*)
and *trace-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
adversary-initial-state $\leq T i$
and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
adversary-initial-state $\leq C i$
and *drift-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq D$
and *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq \text{trace-fri-error}$
and *current-empty-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq \text{empty-query-error}$
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) $+$
 $(\sum i < \text{rounds}.$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1) +
T i + C i) +
trace-fri-error' + composition-fri-error' +
 $(\sum i < \text{rounds}.$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +

```

    query-error-bound +
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1) +
    T i + C i) +
  (composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D + trace-fri-error + empty-query-error)
proof (rule
  checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix-round
  [OF false-statement wf controlled trace-fri-bound comp-fri-bound -
    drift-bound empty-trace-fri-bound current-empty-bound])
fix i
assume i-bound: i < rounds
show
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  query-error-bound +
  hash-collision-budget-value 0
  (staged-query-search-queries budgets i + 1) +
  T i + C i
by (rule
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-alpha
  [OF false-statement wf controlled i-bound - trace-path-bound[OF i-bound]
    composition-path-bound[OF i-bound]])
  (rule default-prefix-clean[OF i-bound])
qed

end

end

```

```

theory Soundness-Staged-Empty-Relevant-Drift
imports Soundness-Staged-Relevant-Drift
begin

```

Empty-header-specific relevant-drift packaging.

This theory is kept downstream of the broader relevant-drift layer so that the latter remains focused on reusable drift decompositions.

```

context soundness

```

begin

definition *checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad*

$:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $\quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $\quad 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad
 $\text{out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\mid \text{Some } (x, -) \Rightarrow$
 $\quad (\text{let packed} = \text{fst } (\text{fst } x);$
 $\quad \quad \text{data} = \text{snd packed};$
 $\quad \quad \text{attacker-state} = \text{snd } (\text{fst } x);$
 $\quad \quad s =$
 $\quad \quad \text{verifier-state-from-adversary attacker-state}$
 $\quad \quad \quad (\text{staged-proof-transcript data})$
 $\text{in } \exists \text{fr } f\text{-fri-roots } f\text{-final trace-table final-state}$
 $\quad \text{trace-openings trace-table' final-state' trace-openings'}$
 $\quad \text{alpha-header-supported-partial-trace-table-candidate-witness}$
 $\quad \quad s \text{ fr } f\text{-fri-roots } f\text{-final trace-table final-state}$
 $\quad \quad \text{trace-openings} \wedge$
 $\quad \text{alpha-header-supported-partial-trace-table-candidate-witness}$
 $\quad \quad s \text{ fr } f\text{-fri-roots } f\text{-final trace-table' final-state'}$
 $\quad \quad \text{trace-openings'} \wedge$
 $\quad \text{trace-table} \neq \text{trace-table'} \wedge$
 $\quad \neg \text{partial-openings-cross-cover-all-indices trace-openings}$
 $\quad \quad \text{trace-openings'})$

definition *checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad*

$:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $\quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $\quad 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
 $\text{out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\mid \text{Some } (x, -) \Rightarrow$
 $\quad (\text{let packed} = \text{fst } (\text{fst } x);$
 $\quad \quad \text{data} = \text{snd packed};$
 $\quad \quad \text{attacker-state} = \text{snd } (\text{fst } x);$
 $\quad \quad s =$
 $\quad \quad \text{verifier-state-from-adversary attacker-state}$
 $\quad \quad \quad (\text{staged-proof-transcript data})$
 $\text{in } \exists \text{fr } f\text{-fri-roots } f\text{-final trace-table final-state}$

trace-openings trace-table' final-state' trace-openings'.
alpha-header-supported-partial-trace-table-candidate-witness
s fr f-fri-roots f-final trace-table final-state
trace-openings ∧
alpha-header-supported-partial-trace-table-candidate-witness
s fr f-fri-roots f-final trace-table' final-state'
trace-openings' ∧
trace-table ≠ trace-table' ∧
(merkle-hash-value-conflict final-state final-state' ∨
hash-map-output-collision
(merkle-hash-state-merge final-state final-state'))))

definition *checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad*

:: ((((((f × 'f list × 'f list × 'f) ×
'f protocol-channel) × 'f staged-proof-data) ×
'f protocol-channel) × unit list) ×
'f protocol-channel) option ⇒ bool

where

checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad
out ⇔
(case out of
None ⇒ False
| Some (x, -) ⇒
(let packed = fst (fst x);
data = snd packed;
attacker-state = snd (fst x);
s =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
in ∃ fr f-fri-roots f-final trace-table final-state
trace-openings.
alpha-header-supported-partial-trace-table-candidate-witness
s fr f-fri-roots f-final trace-table final-state
trace-openings ∧
hash-map-output-collision final-state))

definition *checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad*

:: ((((((f × 'f list × 'f list × 'f) ×
'f protocol-channel) × 'f staged-proof-data) ×
'f protocol-channel) × unit list) ×
'f protocol-channel) option ⇒ bool

where

checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad
out ⇔
(case out of
None ⇒ False
| Some (x, -) ⇒
(let packed = fst (fst x);
data = snd packed;


```

    attacker-state = snd (fst x);
    s =
      verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)
  in  $\exists$  fr f-fri-roots f-final trace-table final-state
    trace-openings trace-table' final-state' trace-openings'.
    alpha-header-supported-partial-trace-table-candidate-witness
      s fr f-fri-roots f-final trace-table final-state
      trace-openings  $\wedge$ 
    alpha-header-supported-partial-trace-table-candidate-witness
      s fr f-fri-roots f-final trace-table' final-state'
      trace-openings'  $\wedge$ 
    trace-table  $\neq$  trace-table'  $\wedge$ 
    merkle-hash-pairwise-coupling-bad final-state final-state')

lemma checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad-split:
assumes
  checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
  out
shows
  checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad
  out  $\vee$ 
  checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad
  out
proof (cases out)
case None
then show ?thesis
using assms
unfolding
  checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad-def
by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state0
where out-eq:
  out =
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state0)
by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from assms obtain fr f-fri-roots f-final trace-table final-state
  trace-openings trace-table' final-state' trace-openings' where
  witness:
    alpha-header-supported-partial-trace-table-candidate-witness ?s fr
    f-fri-roots f-final trace-table final-state trace-openings
and witness':
    alpha-header-supported-partial-trace-table-candidate-witness ?s fr

```

```

    f-fri-roots f-final trace-table' final-state' trace-openings'
and distinct: trace-table  $\neq$  trace-table'
and pair-bad:
    merkle-hash-value-conflict final-state final-state'  $\vee$ 
    hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad-def
    Let-def
by auto
have split:
    hash-map-output-collision final-state  $\vee$ 
    hash-map-output-collision final-state'  $\vee$ 
    merkle-hash-pairwise-coupling-bad final-state final-state'
by (rule merkle-hash-pairwise-bad-imp-local-collision-or-coupling-bad
    [OF pair-bad])
then show ?thesis
proof
    assume collision: hash-map-output-collision final-state
    have
        checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad
        out
    unfolding out-eq
        checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad-def
        Let-def
    apply simp
    apply (intro exI conjI)
    apply (rule witness)
    apply (rule collision)
    done
    then show ?thesis by simp
next
    assume rest:
        hash-map-output-collision final-state'  $\vee$ 
        merkle-hash-pairwise-coupling-bad final-state final-state'
    then show ?thesis
    proof
        assume collision': hash-map-output-collision final-state'
        have
            checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad
            out
        unfolding out-eq
            checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad-def
            Let-def
        apply simp
        apply (intro exI conjI)
        apply (rule witness')
        apply (rule collision')
        done

```

```

    then show ?thesis by simp
next
  assume coupling:
    merkle-hash-pairwise-coupling-bad final-state final-state'
  have
    checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad
      out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad-def
    Let-def
  apply simp
  apply (intro exI conjI)
    apply (rule witness)
    apply (rule witness')
    apply (rule distinct)
    apply (rule coupling)
  done
  then show ?thesis by simp
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad-le-local-or-coupling:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad
    adversary-initial-state +
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad
    adversary-initial-state
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?H =
    checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
  let ?L =
    checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad
  let ?C =
    checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad
  have wp-event ?M ?H adversary-initial-state ≤
    wp-event ?M (λout. ?L out ∨ ?C out) adversary-initial-state
  by (rule wp-event-mono)
    (rule
      checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad-split)

```

also have ... $\leq$
 $wp\text{-event } ?M \ ?L \text{ adversary-initial-state} +$
 $wp\text{-event } ?M \ ?C \text{ adversary-initial-state}$
 by (rule *wp-event-union-bound*)
 finally show *?thesis* .
 qed

lemma *checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad-split:*

assumes

checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
out

shows

checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad
out $\vee$
checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
out

using *assms*

unfolding

checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad-def
checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad-def
checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad-def
alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-def
alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-def
Let-def

by (auto *split: option.splits prod.splits; blast*)

lemma *checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad-le-sparse-or-merkle-side:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad
adversary-initial-state $+$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
adversary-initial-state

proof $-$

let *?M* =

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A

let *?W* =

checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad

let *?S* =

checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad

let *?H* =

checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad

have *wp-event ?M ?W adversary-initial-state* $\leq$

$wp\text{-event } ?M (\lambda out. ?S out \vee ?H out) adversary\text{-initial-state}$
by (rule *wp-event-mono*)
(rule
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad-split)
also have ... $\leq$
 $wp\text{-event } ?M ?S adversary\text{-initial-state} +$
 $wp\text{-event } ?M ?H adversary\text{-initial-state}$
by (rule *wp-event-union-bound*)
finally show *?thesis* .
qed

definition *empty-header-partial-trace-candidate-nonunique*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where

$empty\text{-header-partial-trace-candidate-nonunique } s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg final trace-query-idxs}$
 $\text{ trace-openings trace-table composition-table.}$
 $accepted\text{-with-empty-composition-header-candidates } s \text{ out fr}$
 $f\text{-fri-roots } f\text{-final as dg final trace-query-idxs trace-openings}$
 $\text{ trace-table composition-table} \wedge$
 $\neg (\exists \text{ trace-table0.}$
 $\text{ partial-trace-table-candidates trace-openings} \subseteq$
 $\{\text{trace-table0}\}))$

lemma *composition-alpha-partial-opening-union-hit-with-empty-header-split:*

assumes *hit:*

composition-alpha-partial-opening-union-hit-with-empty-header s out

shows

composition-alpha-bad-set-hit-with-empty-composition-header-candidates
s out $\vee$

empty-header-partial-trace-candidate-nonunique s out

proof –

obtain *fr f-fri-roots f-final as dg final trace-query-idxs*
 $\text{ trace-openings trace-table composition-table}$ **where** *partial:*
 $accepted\text{-with-empty-composition-header-candidates } s \text{ out fr}$
 $f\text{-fri-roots } f\text{-final as dg final trace-query-idxs trace-openings}$
 $\text{ trace-table composition-table}$

and *as-union:*

$as \in \text{ partial-trace-opening-alpha-union-bad-sets trace-openings}$

using *hit*

unfolding *composition-alpha-partial-opening-union-hit-with-empty-header-def*

by *blast*

from *as-union* **obtain** *trace-table'* **where** *candidate':*

$\text{ trace-table}' \in \text{ partial-trace-table-candidates trace-openings}$

and *as-bad':* $as \in \text{ composition-trace-bad-alpha-space trace-table}'$

unfolding *partial-trace-opening-alpha-union-bad-sets-def* **by** *blast*

have *candidate:*

$\text{ trace-table} \in \text{ partial-trace-table-candidates trace-openings}$

```

using partial
unfolding accepted-with-empty-composition-header-candidates-def
  partial-trace-table-candidates-def
by simp
show ?thesis
proof (cases
   $\exists \text{trace-table0}.$ 
    partial-trace-table-candidates trace-openings  $\subseteq$ 
       $\{\text{trace-table0}\}$ )
case True
then obtain trace-table0 where subset:
  partial-trace-table-candidates trace-openings  $\subseteq$ 
     $\{\text{trace-table0}\}$ 
by blast
have trace-eq: trace-table = trace-table0
  using subset candidate by blast
have trace'-eq: trace-table' = trace-table0
  using subset candidate' by blast
have as  $\in$  composition-trace-bad-alpha-space trace-table
  using as-bad' trace-eq trace'-eq by simp
then have
  composition-alpha-bad-set-hit-with-empty-composition-header-candidates
    s out
unfolding
  composition-alpha-bad-set-hit-with-empty-composition-header-candidates-def
using partial by blast
then show ?thesis by simp
next
case False
have empty-header-partial-trace-candidate-nonunique s out
  unfolding empty-header-partial-trace-candidate-nonunique-def
  using partial False by blast
then show ?thesis by simp
qed
qed

```

```

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-nonunique-imp-witnessed-cross-or-merkle-on-s
assumes support:
  out  $\in$ 
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
and nonunique:
  checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-partial-trace-candidate-nonunique out
shows
  checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad

```

```

    out
proof (cases out)
  case None
  then show ?thesis
    using nonunique
    unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    by simp
next
  case (Some result-pack)
  then obtain prefix prefix-state data attacker-state result final-state
    where out-eq:
      out =
        Some (((((prefix, prefix-state), data), attacker-state),
          result), final-state)
    by (cases result-pack, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
    using support
    unfolding out-eq
      checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
    by (auto elim!: set-dist-bindE split: prod.splits)
  have local-nonunique:
    empty-header-partial-trace-candidate-nonunique ?s
      (Some (result, final-state))
    using nonunique
    unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      Let-def
    by simp
  then obtain fr f-fri-roots f-final as dg final trace-query-idxs
    trace-openings trace-table composition-table where empty:
      accepted-with-empty-composition-header-candidates ?s
        (Some (result, final-state)) fr f-fri-roots f-final as dg final
        trace-query-idxs trace-openings trace-table composition-table
  and not-unique-partial:
    ¬ (∃ trace-table0.
      partial-trace-table-candidates trace-openings ⊆
        {trace-table0})
    unfolding empty-header-partial-trace-candidate-nonunique-def
    by blast
  from empty obtain rest where header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg []
    final rest
  and trace-partial:
    accepted-with-partial-trace-openings ?s
      (Some (result, final-state)) fr trace-query-idxs trace-openings

```

```

unfolding accepted-with-empty-composition-header-candidates-def
by blast
have partial-subset:
  partial-trace-table-candidates trace-openings  $\subseteq$ 
    alpha-header-supported-partial-trace-table-candidates ?s fr f-fri-roots
      f-final
proof
  fix trace-table'
  assume
    trace-table' ∈ partial-trace-table-candidates trace-openings
  then have candidate':
    partial-trace-table-candidate trace-table' trace-openings
  unfolding partial-trace-table-candidates-def by simp
show
  trace-table' ∈
    alpha-header-supported-partial-trace-table-candidates ?s fr
      f-fri-roots f-final
  unfolding alpha-header-supported-partial-trace-table-candidates-def
by (intro CollectI exI[of - Some (result, final-state)]
  exI[of - trace-query-idxs] exI[of - trace-openings]
  exI[of - as] exI[of - dg] exI[of - []]
  exI[of - final] exI[of - rest] conjI)
  (use verifier trace-partial candidate' header in simp-all)
qed
have not-unique-header:
   $\neg (\exists \text{trace-table0.}$ 
    alpha-header-supported-partial-trace-table-candidates ?s fr
      f-fri-roots f-final  $\subseteq \{\text{trace-table0}\}$ )
proof
  assume
     $\exists \text{trace-table0.}$ 
    alpha-header-supported-partial-trace-table-candidates ?s fr
      f-fri-roots f-final  $\subseteq \{\text{trace-table0}\}$ 
  then obtain trace-table0 where header-subset:
    alpha-header-supported-partial-trace-table-candidates ?s fr
      f-fri-roots f-final  $\subseteq \{\text{trace-table0}\}$ 
  by blast
have
  partial-trace-table-candidates trace-openings  $\subseteq$ 
     $\{\text{trace-table0}\}$ 
  using partial-subset header-subset by blast
then show False
  using not-unique-partial by blast
qed
have not-unique-witnessed:
   $\neg (\exists \text{trace-table0.}$ 
    alpha-header-supported-witnessed-partial-trace-table-candidates ?s fr
      f-fri-roots f-final  $\subseteq \{\text{trace-table0}\}$ )
using not-unique-header

```



```

    by (simp add: alpha-header-supported-witnessed-candidates-eq-candidates)
  have
    alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
      ?s fr f-fri-roots f-final
    by (rule
      alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-if-not-singleton
      [OF not-unique-witnessed])
  then have
    alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad ?s
  unfolding
    alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-def
  by blast
  then show ?thesis
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad-def
    Let-def
  by simp
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-nonunique-le-witnessed-cross-or-merkle:*

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-partial-trace-candidate-nonunique)
  adversary-initial-state ≤
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
  adversary-initial-state
by (rule wp-event-mono-on-support)
(rule
  checked-staged-security-with-actual-alpha-prefix-empty-header-nonunique-imp-witnessed-cross-or-merkle-on-support)

```

definition *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift*

```

:: ((((((f × 'f list × 'f list × 'f) ×
  'f protocol-channel) × 'f staged-proof-data) ×
  'f protocol-channel) × unit list) ×
  'f protocol-channel) option ⇒ bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
  out ⇔
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
  composition-trace-bad-alpha-space out ∧
checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-alpha-partial-opening-union-hit-with-empty-header out

```

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-le-fixed-or-nonunique:*

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

```

```

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
adversary-initial-state ≤
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
adversary-initial-state +
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-partial-trace-candidate-nonunique)
adversary-initial-state
proof –
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?fixed =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates
  let ?nonunique =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-partial-trace-candidate-nonunique
  have event-le:
    wp-event ?M
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
    adversary-initial-state ≤
    wp-event ?M (λout. ?fixed out ∨ ?nonunique out)
    adversary-initial-state
  by (rule wp-event-mono)
    (auto simp:
      checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-def
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      split: option.splits prod.splits
      dest:
        composition-alpha-partial-opening-union-hit-with-empty-header-split)
  also have ... ≤
    wp-event ?M ?fixed adversary-initial-state +
    wp-event ?M ?nonunique adversary-initial-state
  by (rule wp-event-union-bound)
  finally show ?thesis .
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-le-fixed-or-witnessed-cross:*

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
  adversary-initial-state ≤
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event

```

```

    composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
  adversary-initial-state +
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
  adversary-initial-state
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?fixed =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates
  let ?nonunique =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-partial-trace-candidate-nonunique
  have base:
    wp-event ?M
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
    adversary-initial-state ≤
    wp-event ?M ?fixed adversary-initial-state +
    wp-event ?M ?nonunique adversary-initial-state
  by (rule
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-le-fixed-or-nonunique)
  have nonunique-bound:
    wp-event ?M ?nonunique adversary-initial-state ≤
    wp-event ?M
    checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
    adversary-initial-state
  by (rule
    checked-staged-security-with-actual-alpha-prefix-empty-header-nonunique-le-witnessed-cross-or-merkle)
  show ?thesis
  by (rule order-trans[OF base])
  (intro add-mono order-refl nonunique-bound)
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-bound-from-fixed-and-witnessed-c
assumes fixed-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
  adversary-initial-state ≤ F
and cross-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
  checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
  adversary-initial-state ≤ X

```

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
adversary-initial-state $\leq F + X$

by (*rule order-trans*[

OF

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-le-fixed-or-witnessed-cross])
(*intro add-mono fixed-bound cross-bound*)

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-bound-from-fixed-and-nonunique:*

assumes *fixed-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
(*checked-staged-security-with-actual-alpha-prefix-verifier-event*
composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
adversary-initial-state $\leq F$

and *nonunique-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
(*checked-staged-security-with-actual-alpha-prefix-verifier-event*
empty-header-partial-trace-candidate-nonunique)
adversary-initial-state $\leq N$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
adversary-initial-state $\leq F + N$

by (*rule order-trans*[

OF

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-le-fixed-or-nonunique])
(*intro add-mono fixed-bound nonunique-bound*)

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-le-relevant-drift:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state

by (*rule wp-event-mono*)

(*auto simp:*

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-def
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-def)

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-bound-from-relevant-drift:*
assumes *drift-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state $\leq D$
shows
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
 adversary-initial-state $\leq D$
by (*rule order-trans*[
 OF
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-le-relevant-drift
 drift-bound])

lemma *checked-staged-security-with-actual-alpha-prefix-fixed-empty-alpha-hit-imp-prefix-or-not-prefix-on-support:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
 out $\in$
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 adversary-initial-state)
and *fixed-hit:*
 checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-bad-set-hit-with-empty-composition-header-candidates
 out
shows
 checked-staged-security-with-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space out $\vee$
 checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
 composition-trace-bad-alpha-space out
proof (*cases out*)
case *None*
then show *?thesis*
 using *fixed-hit*
 unfolding *checked-staged-security-with-actual-alpha-prefix-verifier-event-def*
 by *simp*
next
case (*Some result-pack*)
then obtain *prefix prefix-state data attacker-state result final-state*
where *out-eq:*
 out =
 Some (((((prefix, prefix-state), data), attacker-state),
 result), final-state)

```

    by (cases result-pack, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have verify-out:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  using support
  unfolding out-eq
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
  have local-fixed:
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates ?s
    (Some (result, final-state))
  using fixed-hit
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    Let-def
  by simp
  then obtain fr f-fri-roots f-final as dg final trace-query-idxs
    trace-openings trace-table composition-table rest where empty:
    accepted-with-empty-composition-header-candidates ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg final
    trace-query-idxs trace-openings trace-table composition-table
  and header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg []
    final rest
  and trace-partial:
    accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr trace-query-idxs trace-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and as-bad:
    as ∈ composition-trace-bad-alpha-space trace-table
  unfolding
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates-def
    accepted-with-empty-composition-header-candidates-def
  by blast
  from verify-monad-accepted-transcript-shape[OF verify-out]
  obtain as' query-idxs where shape':
    accepted-transcript-shape ?s (Some (result, final-state)) as'
    query-idxs
  by blast
  from shape' obtain result' final-state' fr' f-fri-roots' f-final' dg'
    composition-fri-roots' final' rest' where out-shape:
    Some (result, final-state) = Some (result', final-state')
  and header':
    verifier-header-transcript ?s fr' f-fri-roots' f-final' as' dg'
    composition-fri-roots' final' rest'
  by (elim accepted-transcript-shape-header-query-extraction)

```

```

have as'-eq: as' = as
  using verifier-header-transcript-unique[OF header' header] by simp
have shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
  using shape' as'-eq by simp
have header-union:
  as ∈
    alpha-header-supported-partial-union-bad-sets ?s
    composition-trace-bad-alpha-space fr f-fri-roots f-final
proof —
  have candidate:
    trace-table ∈
      alpha-header-supported-partial-trace-table-candidates ?s fr
        f-fri-roots f-final
  unfolding alpha-header-supported-partial-trace-table-candidates-def
by (intro CollectI exI[of - Some (result, final-state)]
    exI[of - trace-query-idxs] exI[of - trace-openings]
    exI[of - as] exI[of - dg] exI[of - []]
    exI[of - final] exI[of - rest] conjI)
    (use verify-out trace-partial trace-candidate header in simp-all)
show ?thesis
  unfolding alpha-header-supported-partial-union-bad-sets-def
  using candidate as-bad
    composition-trace-bad-alpha-space-subset-alpha-space[of trace-table]
  by auto
qed
have builder:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
proof —
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support[unfolded out-eq]])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
show ?thesis
  by blast
qed
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)

```

```

    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
have prefix-root-eq: staged-trace-root data = fst prefix
  using support out-eq
  unfolding checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
    checked-staged-transcript-with-alpha-prefix-program-def
    checked-staged-after-alpha-prefix-program-def Let-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have as-eq: as = staged-alphas data
  using verifier-header-transcript-unique[OF header staged-header]
  by simp
have fr-eq: fr = staged-trace-root data
  using verifier-header-transcript-unique[OF header staged-header]
  by simp
show ?thesis
proof (cases
  as ∈ alpha-prefix-union-bad-sets prefix-state
    composition-trace-bad-alpha-space fr)
case True
have
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-def
    Let-def
  using True as-eq fr-eq prefix-root-eq by simp
then show ?thesis by simp
next
case not-prefix: False
have not-bound:
  alpha-header-partial-candidate-not-prefix-bound prefix-state ?s fr
    f-fri-roots f-final
  by (rule alpha-header-partial-union-hit-not-prefix-hit-imp-not-prefix-bound
      [OF header-union not-prefix])
have
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
    composition-trace-bad-alpha-space out
  unfolding out-eq
  by (rule
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-boundI
      [OF refl shape header header-union not-prefix not-bound])
then show ?thesis by simp

```


qed
qed

lemma *checked-staged-security-with-actual-alpha-prefix-fixed-empty-alpha-hit-le-prefix-or-not-prefix:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

(*checked-staged-security-with-actual-alpha-prefix-verifier-event*

composition-alpha-bad-set-hit-with-empty-composition-header-candidates)

adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

(*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*

composition-trace-bad-alpha-space)

adversary-initial-state +

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*

composition-trace-bad-alpha-space)

adversary-initial-state

proof —

let *?M* =

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A

let *?Fixed* =

checked-staged-security-with-actual-alpha-prefix-verifier-event

composition-alpha-bad-set-hit-with-empty-composition-header-candidates

let *?Prefix* =

checked-staged-security-with-actual-alpha-prefix-bad-set-hit

composition-trace-bad-alpha-space

let *?NotPrefix* =

checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound

composition-trace-bad-alpha-space

have *event-le:*

wp-event ?M ?Fixed adversary-initial-state $\leq$

wp-event ?M ($\lambda out. ?Prefix out \vee ?NotPrefix out$)

adversary-initial-state

by (*rule wp-event-mono-on-support*)

(*rule*

checked-staged-security-with-actual-alpha-prefix-fixed-empty-alpha-hit-imp-prefix-or-not-prefix-on-support

[*OF wf controlled*])

also have ... $\leq$

wp-event ?M ?Prefix adversary-initial-state +

wp-event ?M ?NotPrefix adversary-initial-state

by (*rule wp-event-union-bound*)

finally show *?thesis* .

qed

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-imp-prefix-prequery-or-empty-relevant*

assumes *support*:

out $\in$
set-dist
 (*execute*
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 adversary-initial-state)

and *hit*:

checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-alpha-partial-opening-union-hit-with-empty-header out

shows

checked-staged-security-with-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space out $\vee$
checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
composition-trace-bad-alpha-space out $\vee$
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
out

proof —

have *partial*:

checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
composition-trace-bad-alpha-space out

by (*rule*
checked-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-imp-partial-header-bad-set-hit-on-
 [*OF support hit*])

have *fresh-or-prequery*:

checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
composition-trace-bad-alpha-space out $\vee$
checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
composition-trace-bad-alpha-space out

by (*rule*
checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-imp-fresh-or-prequery
 [*OF partial*])

then show *?thesis*

proof

assume *fresh*:

checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
composition-trace-bad-alpha-space out

have *prefix-or-drift*:

checked-staged-security-with-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space out $\vee$
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space out

by (*rule*
checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-imp-prefix-or-drift
 [*OF fresh*])

then show *?thesis*

proof

assume

```

    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out
  then show ?thesis by simp
next
  assume drift:
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
    composition-trace-bad-alpha-space out
  have
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
    out
  unfolding
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-def
  using drift hit by simp
  then show ?thesis by simp
qed
next
  assume
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space out
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-bound-from-empty-relevant-drift-and-
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and drift-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
    adversary-initial-state  $\leq D$ 
  shows
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
     composition-alpha-partial-opening-union-hit-with-empty-header)
    adversary-initial-state  $\leq$ 
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A

```

```

let ?Prefix =
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit
  composition-trace-bad-alpha-space
let ?Prequery =
  checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
  composition-trace-bad-alpha-space
let ?Drift =
  checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
let ?Hit =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-alpha-partial-opening-union-hit-with-empty-header
let ?P =
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0)
let ?Q =
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0)
have transcript-bound:
  wp-event (checked-staged-transcript-with-alpha-prefix-program A)
  (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
   composition-trace-bad-alpha-space)
  adversary-initial-state ≤ ?P
by (rule
  checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
  [OF wf controlled])
have prefix-bound:
  wp-event ?M ?Prefix adversary-initial-state ≤ ?P
by (rule
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
  [OF transcript-bound])
have prequery-bound:
  wp-event ?M ?Prequery adversary-initial-state ≤ ?Q
by (rule
  checked-staged-security-with-actual-alpha-prefix-vector-alpha-prequery-accounting
  [OF wf controlled])
have wp-event ?M ?Hit adversary-initial-state ≤
  wp-event ?M
  (λout. ?Prefix out ∨ ?Prequery out ∨ ?Drift out ∨ False)
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (use
  checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-imp-prefix-prequery-or-empty-relevant-drift
  [of - A] in blast)
also have ... ≤
  wp-event ?M ?Prefix adversary-initial-state +
  wp-event ?M ?Prequery adversary-initial-state +

```

```

    wp-event ?M ?Drift adversary-initial-state +
    wp-event ?M (λ-. False) adversary-initial-state
  by (rule wp-event-union-bound4)
also have ... ≤ ?P + ?Q + D + 0
  by (intro add-mono prefix-bound prequery-bound drift-bound)
    (simp add: wp-event-def wp-def dist-expect-def)
finally show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-data-state-empty-header-composition-randomization-bound-from-empty-relevant-
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and drift-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
      adversary-initial-state ≤ D
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-randomization-bad-with-empty-composition-header-candidates)
      adversary-initial-state ≤
        composition-error-bound +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
        hash-collision-budget-value 0
        (staged-alpha-search-queries budgets 0) +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
        D
  proof -
    let ?M = checked-staged-security-experiment-with-data-state A
    let ?random =
      staged-security-with-data-state-verifier-event
        composition-randomization-bad-with-empty-composition-header-candidates
    let ?alpha =
      staged-security-with-data-state-verifier-event
        composition-alpha-partial-opening-union-hit-with-empty-header
    have alpha-projected:
      wp-event ?M ?alpha adversary-initial-state =
        wp-event
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
          (checked-staged-security-with-actual-alpha-prefix-verifier-event
            composition-alpha-partial-opening-union-hit-with-empty-header)
          adversary-initial-state
    by (rule
        checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)

```

have *alpha-bound*:

$$\begin{aligned} & wp\text{-event } ?M \text{ ?alpha adversary-initial-state} \leq \\ & \quad composition\text{-error-bound} + \\ & \quad staged\text{-phase-relation-error size} \\ & \quad (staged\text{-alpha-search-queries budgets } 0) + \\ & \quad hash\text{-collision-budget-value } 0 \\ & \quad (staged\text{-alpha-search-queries budgets } 0) + \\ & \quad staged\text{-phase-relation-error size} \\ & \quad (staged\text{-alpha-search-queries budgets } 0) + \\ & \quad D \end{aligned}$$
unfolding *alpha-projected*
by (rule

$$\begin{aligned} & checked\text{-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-bound-from-empty-relevant-drift-and-bud} \\ & [OF \text{ wf controlled drift-bound}]) \end{aligned}$$
have *wp-event ?M ?random adversary-initial-state* $\leq$

$$wp\text{-event } ?M \text{ ?alpha adversary-initial-state}$$
by (rule *wp-event-mono*)

$$\begin{aligned} & (auto \text{ simp: staged-security-with-data-state-verifier-event-def} \\ & \quad intro: \\ & \quad \quad composition\text{-alpha-bad-set-hit-with-empty-header-imp-partial-opening-union-hit} \\ & \quad \quad composition\text{-randomization-bad-with-empty-composition-header-candidates-imp-alpha-hit} \\ & \quad \quad split: option.splits prod.splits) \end{aligned}$$
also have ... $\leq$

$$\begin{aligned} & composition\text{-error-bound} + \\ & staged\text{-phase-relation-error size} \\ & (staged\text{-alpha-search-queries budgets } 0) + \\ & hash\text{-collision-budget-value } 0 \\ & (staged\text{-alpha-search-queries budgets } 0) + \\ & staged\text{-phase-relation-error size} \\ & (staged\text{-alpha-search-queries budgets } 0) + \\ & D \end{aligned}$$
by (rule *alpha-bound*)
finally show *?thesis* .
qed

lemma *checked-staged-security-with-data-state-empty-header-bad-event-bound-from-empty-relevant-drift-trace-fri-and*
assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *drift-bound*:

$$\begin{aligned} & wp\text{-event} \\ & (checked\text{-staged-security-experiment-with-actual-alpha-prefix-data-state} \\ & \quad A) \\ & checked\text{-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift} \\ & adversary\text{-initial-state} \leq D \end{aligned}$$
and *trace-fri-bound*:

$$\begin{aligned} & wp\text{-event } (checked\text{-staged-security-experiment-with-data-state } A) \\ & (staged\text{-security-with-data-state-verifier-event} \\ & \quad trace\text{-fri-bad-with-empty-composition-header-candidates}) \end{aligned}$$

```

    adversary-initial-state ≤ trace-fri-error
and query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        query-bad-with-empty-composition-header-candidates)
    adversary-initial-state ≤ empty-query-error
shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-partial-candidate-empty-header)
    adversary-initial-state ≤
      composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      D + trace-fri-error + empty-query-error
proof –
    let ?M = checked-staged-security-experiment-with-data-state A
    let ?E = staged-security-with-data-state-verifier-event
    let ?degree =
      ?E composition-degree-bad-with-empty-composition-header-candidates
    let ?random =
      ?E composition-randomization-bad-with-empty-composition-header-candidates
    let ?trace =
      ?E trace-fri-bad-with-empty-composition-header-candidates
    let ?query =
      ?E query-bad-with-empty-composition-header-candidates
    let ?bad = ?E soundness-bad-event-partial-candidate-empty-header
    have degree-false: ?degree = (λout. False)
      by (rule ext)
      (auto simp: staged-security-with-data-state-verifier-event-def
        composition-degree-bad-with-empty-composition-header-candidates-false
        split: option.splits prod.splits)
    have degree-bound:
      wp-event ?M ?degree adversary-initial-state ≤ 0
    unfolding degree-false wp-event-def wp-def dist-expect-def by simp
    have random-bound:
      wp-event ?M ?random adversary-initial-state ≤
        composition-error-bound +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
        hash-collision-budget-value 0
        (staged-alpha-search-queries budgets 0) +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
        D

```

```

by (rule
  checked-staged-security-with-data-state-empty-header-composition-randomization-bound-from-empty-relevant-drij
  [OF wf controlled drift-bound])
have wp-event ?M ?bad adversary-initial-state ≤
  wp-event ?M
  (λout. ?degree out ∨ ?random out ∨ ?trace out ∨
    ?query out)
  adversary-initial-state
proof (rule wp-event-mono)
  fix out
  assume bad: ?bad out
  show ?degree out ∨ ?random out ∨ ?trace out ∨ ?query out
  proof (cases out)
    case None
    then show ?thesis
      using bad
      unfolding staged-security-with-data-state-verifier-event-def
      by simp
  next
    case (Some packed)
    then obtain data attacker-state result final-state where out-eq:
      out = Some (((data, attacker-state), result), final-state)
      by (cases packed, auto split: prod.splits)
    let ?s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    have local-bad:
      soundness-bad-event-partial-candidate-empty-header ?s
      (Some (result, final-state))
      using bad
      unfolding out-eq staged-security-with-data-state-verifier-event-def
      by simp
    then consider
      (comp)
      composition-bad-with-empty-composition-header-candidates ?s
      (Some (result, final-state))
    | (trace)
      trace-fri-bad-with-empty-composition-header-candidates ?s
      (Some (result, final-state))
    | (query)
      query-bad-with-empty-composition-header-candidates ?s
      (Some (result, final-state))
    unfolding soundness-bad-event-partial-candidate-empty-header-def
    by blast
  then show ?thesis
  proof cases
    case comp
    then have comp-split:
      composition-degree-bad-with-empty-composition-header-candidates ?s

```



```

      (Some (result, final-state)) ∨
      composition-randomization-bad-with-empty-composition-header-candidates
      ?s (Some (result, final-state))
    by (rule
      composition-bad-with-empty-composition-header-candidates-split
      [OF false-statement])
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    using comp-split by (auto split: option.splits prod.splits)
next
  case trace
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
next
  case query
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
qed
also have ... ≤
  wp-event ?M ?degree adversary-initial-state +
  wp-event ?M ?random adversary-initial-state +
  wp-event ?M ?trace adversary-initial-state +
  wp-event ?M ?query adversary-initial-state
  by (rule wp-event-union-bound4)
also have ... ≤
  0 +
  (composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  D) +
  trace-fri-error + empty-query-error
  by (intro add-mono degree-bound random-bound trace-fri-bound query-bound)
finally show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix
  fixes trace-fri-error' composition-fri-error' D empty-query-error :: prob
  and C :: nat ⇒ prob
  assumes false-statement: ¬ exists-valid-trace
  and wf: staged-budget-wellformed budgets

```

and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state ≤ trace-fri-error'
and *comp-fri-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state ≤ composition-fri-error'
and *round-bound:*
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i*)
 adversary-initial-state ≤ C i
and *drift-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
 adversary-initial-state ≤ D
and *empty-trace-fri-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state ≤ trace-fri-error
and *current-empty-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-current-empty-query-opening-hit-at 0*)
 adversary-initial-state ≤ empty-query-error
shows *checked-staged-adversary-acceptance-probability A ≤*
 hash-collision-budget-value 0
 (*staged-attacker-query-budget budgets +*
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds}. C i$) +
 trace-fri-error' + composition-fri-error' +
 ($\sum i < \text{rounds}. C i$) +
 (*composition-error-bound +*
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 D + trace-fri-error + empty-query-error)
proof –
 have *comp-bound:*

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
composition-randomization-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq (\sum i < \text{rounds. } C \ i)$
by (*rule*
checked-staged-security-with-data-state-aligned-partial-candidate-randomization-bound-from-query-prefix-current-
[OF wf controlled round-bound])
have *current-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq (\sum i < \text{rounds. } C \ i)$
by (*rule*
checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
[OF round-bound wf controlled])
have *query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq (\sum i < \text{rounds. } C \ i)$
by (*rule order-trans*[
OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
current-query-bound])
(use wf controlled in simp-all)
have *merkle-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by (*rule*
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[OF wf controlled])
have *empty-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *empty-query-error*
by (*rule order-trans*[
OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero
*OF wf controlled] current-empty-bound])
have *empty-header-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state $\leq$
composition-error-bound +
*staged-phase-relation-error size**

(staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D + trace-fri-error + empty-query-error
by (rule
 checked-staged-security-with-data-state-empty-header-bad-event-bound-from-empty-relevant-drift-trace-fri-and-qu
 [OF false-statement wf controlled drift-bound empty-trace-fri-bound
 empty-query-bound])
have bound:
 checked-staged-adversary-acceptance-probability A ≤
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (∑ i<rounds. C i) +
 trace-fri-error' + composition-fri-error' +
 (∑ i<rounds. C i) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D + trace-fri-error + empty-query-error)
by (rule
 checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-components-and-empt
 [OF false-statement merkle-bound comp-bound trace-fri-bound
 comp-fri-bound query-bound empty-header-bound])
then show ?thesis .
qed

lemma checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-structured-query-fr
fixes trace-fri-error' composition-fri-error' D empty-query-error
 header-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-partial-candidates)

$adversary-initial-state \leq composition-fri-error'$
and *round-bound*:
 $\bigwedge i. i < rounds \implies$
 $wp-event$
 $(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$
 $(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at\ i)$
 $adversary-initial-state \leq C\ i$
and *header-query-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state\ A)$
 $(checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix\ A)$
 $adversary-initial-state \leq header-query-error$
and *drift-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state\ A)$
 $checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift$
 $adversary-initial-state \leq D$
and *empty-trace-fri-bound*:
 $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event\ trace-fri-bad-with-empty-composition-header-candidates)$
 $adversary-initial-state \leq trace-fri-error$
and *current-empty-bound*:
 $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-current-empty-query-opening-hit-at\ 0)$
 $adversary-initial-state \leq empty-query-error$
shows $checked-staged-adversary-acceptance-probability\ A \leq$
 $hash-collision-budget-value\ 0$
 $(staged-attacker-query-budget\ budgets +$
 $staged-challenge-query-budget + verifier-hash-query-budget) +$
 $(\sum i < rounds. C\ i) +$
 $trace-fri-error' + composition-fri-error' +$
 $header-query-error +$
 $(composition-error-bound +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $D + trace-fri-error + empty-query-error)$
proof –
have *comp-bound*:
 $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $composition-randomization-bad-with-aligned-transcript-partial-candidates)$

$\text{adversary-initial-state} \leq (\sum i < \text{rounds}. C\ i)$
by (rule
checked-staged-security-with-data-state-aligned-partial-candidate-randomization-bound-from-query-prefix-current-
[OF wf controlled round-bound])
have *query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)
 $\text{adversary-initial-state} \leq \text{header-query-error}$
by (rule
checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-header-authenticated-candidate-
[OF wf controlled header-query-bound])
have *merkle-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget})$
by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[OF wf controlled])
have *empty-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
by (rule *order-trans*[
 $OF\ \text{checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero}$
 $OF\ wf\ controlled]$ *current-empty-bound*])
have *empty-header-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header)
 $\text{adversary-initial-state} \leq$
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $D + \text{trace-fri-error} + \text{empty-query-error}$
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-bound-from-empty-relevant-drift-trace-fri-and-qu
[OF false-statement wf controlled drift-bound empty-trace-fri-bound
empty-query-bound])
have *bound*:

checked-staged-adversary-acceptance-probability $A \leq$
 (hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds}. C\ i$) + trace-fri-error' +
 composition-fri-error' + header-query-error) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D + trace-fri-error + empty-query-error)
by (rule
 checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-components-and-empty-header-relevant-drift
 [OF false-statement merkle-bound comp-bound trace-fri-bound
 comp-fri-bound query-bound empty-header-bound])
then show ?thesis
by (simp add: algebra-simps)
qed

lemma *checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix*
fixes trace-fri-error' composition-fri-error' D empty-query-error :: prob
and C :: nat $\Rightarrow$ prob
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ trace-fri-error'
and comp-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ composition-fri-error'
and round-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq C\ i$
and drift-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift

$\text{adversary-initial-state} \leq D$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *current-empty-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows *checked-staged-adversary-acceptance-probability* $A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum i < \text{rounds. } C i) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $D + \text{trace-fri-error} + \text{empty-query-error})$
proof –
have *current-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-current-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq (\sum i < \text{rounds. } C i)$
by (rule
 $\text{checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds}$
 $[\text{OF round-bound wf controlled}])$
have *merkle-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{partial-merkle-inconsistency-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget})$
by (rule
 $\text{checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled}$
 $[\text{OF wf controlled}])$
have *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
by (rule *order-trans*[

OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero[
 OF wf controlled] current-empty-bound])
have empty-header-bound:
 wp -event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
 D + trace-fri-error + empty-query-error
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-bound-from-empty-relevant-drift-trace-fri-and-qu
[OF false-statement wf controlled drift-bound empty-trace-fri-bound
empty-query-bound])
have partial-candidate-bound:
 wp -event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
($\sum i < \text{rounds. } C \ i$) + trace-fri-error' + composition-fri-error'
by (rule
checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-with-merkle-c
[OF wf controlled merkle-bound current-query-bound trace-fri-bound
comp-fri-bound])
have bound:
checked-staged-adversary-acceptance-probability $A \leq$
(hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
($\sum i < \text{rounds. } C \ i$) + trace-fri-error' +
composition-fri-error') +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
 D + trace-fri-error + empty-query-error)
by (rule
checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-and-empty-header-bou

```

    [OF false-statement partial-candidate-bound empty-header-bound])
  then show ?thesis
    by (simp add: algebra-simps)
qed

end

end

```

```

theory Soundness-Relevant-Drift-Public
imports
  Soundness
  Soundness-Staged-Empty-Relevant-Drift
begin

```

Public migration wrappers for the refined relevant-drift route.

These theorems sit downstream of *Soundness* so the main public compatibility layer does not keep growing. The route below derives the aligned composition-randomization branch and aligned query branch from the same per-round current-query prefix bounds. It exposes only the empty-header-specific relevant-drift event and keeps the partial-candidate FRI premises internal to this route.

```

context soundness
begin

```

```

theorem stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query:
fixes trace-fri-error' composition-fri-error' D empty-query-error :: prob
and C :: nat  $\Rightarrow$  prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  composition-fri-error'
and round-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state  $\leq$  C i
and drift-bound:

```

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
 adversary-initial-state $\leq D$
and *empty-trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ trace-fri-error
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq$ empty-query-error
shows checked-staged-adversary-acceptance-probability *A* $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds}. C\ i$) +
 trace-fri-error' + composition-fri-error' +
 ($\sum i < \text{rounds}. C\ i$) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 D +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + empty-query-error)
proof –
have bound:
 checked-staged-adversary-acceptance-probability *A* $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds}. C\ i$) +
 trace-fri-error' + composition-fri-error' +
 ($\sum i < \text{rounds}. C\ i$) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D + trace-fri-error + empty-query-error)
by (rule
 checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix-ro

```

    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
     round-bound drift-bound empty-trace-fri-bound current-empty-bound])
  then show ?thesis
    by (simp add: algebra-simps)
qed

theorem stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-single-query
fixes trace-fri-error' composition-fri-error' D empty-query-error :: prob
and C :: nat  $\Rightarrow$  prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-partial-candidates)
  adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-fri-bad-with-partial-candidates)
  adversary-initial-state  $\leq$  composition-fri-error'
and round-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state  $\leq$  C i
and drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
    adversary-initial-state  $\leq$  D
and empty-trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state  $\leq$  trace-fri-error
and current-empty-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
  adversary-initial-state  $\leq$  empty-query-error
shows checked-staged-adversary-acceptance-probability A  $\leq$ 
  hash-collision-budget-value 0
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget + verifier-hash-query-budget) +
  ( $\sum i < \text{rounds}. C i$ ) +
  trace-fri-error' + composition-fri-error' +

```

```

    (composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      D +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      trace-fri-error + empty-query-error)
  proof -
    have bound:
      checked-staged-adversary-acceptance-probability A ≤
      hash-collision-budget-value 0
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget + verifier-hash-query-budget) +
      (∑ i<rounds. C i) +
      trace-fri-error' + composition-fri-error' +
      (composition-error-bound +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
        hash-collision-budget-value 0
        (staged-alpha-search-queries budgets 0) +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
        D + trace-fri-error + empty-query-error)
    by (rule
      checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix-ro
      [OF false-statement wf controlled trace-fri-bound comp-fri-bound
        round-bound drift-bound empty-trace-fri-bound current-empty-bound])
    then show ?thesis
    by (simp add: algebra-simps)
qed

end

end

```

```

theory Soundness-Relevant-Drift-Reachable-FRI
imports Soundness-Relevant-Drift-Public
begin

```

Reachable partial-candidate FRI interface for the refined relevant-drift migration path. This layer does not change the compatibility predicates *trace-fri-bad-with-partial-openings* and *composition-fri-bad-with-partial-openings*. Instead it records the partial-candidate FRI obligations used by the aligned partial-opening route: the verifier has authenticated sampled openings, and the FRI low-degree interface rules out a low-degree violation for any candidate table compatible with those openings.

context *soundness*
begin

definition *trace-fri-bad-with-reachable-partial-candidate*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-bad-with-reachable-partial-candidate $s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr query-idxs trace-openings trace-table.}$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table})$

definition *composition-fri-bad-with-reachable-partial-candidate*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-bad-with-reachable-partial-candidate $s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table.}$
 $\text{accepted-with-partial-initial-openings } s \text{ out fr f-fri-roots f-final}$
 $\text{as dg composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table})$

definition *trace-fri-reachable-partial-candidate-reduction-assumption*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

trace-fri-reachable-partial-candidate-reduction-assumption $s \longleftrightarrow$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-reachable-partial-candidate } s) s \leq$
 trace-fri-error

definition *composition-fri-reachable-partial-candidate-reduction-assumption*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

composition-fri-reachable-partial-candidate-reduction-assumption s
 $\longleftrightarrow$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-reachable-partial-candidate } s) s \leq$
 $\text{composition-fri-error}$

lemma *trace-fri-bad-with-partial-candidates-imp-reachable-partial-candidate:*

assumes *trace-fri-bad-with-partial-candidates s out*
shows *trace-fri-bad-with-reachable-partial-candidate s out*
using *assms*
unfolding *trace-fri-bad-with-partial-candidates-def*
trace-fri-bad-with-reachable-partial-candidate-def
accepted-with-partial-initial-openings-def
by *blast*

lemma *composition-fri-bad-with-partial-candidates-imp-reachable-partial-candidate:*
assumes *composition-fri-bad-with-partial-candidates s out*
shows *composition-fri-bad-with-reachable-partial-candidate s out*
using *assms*
unfolding *composition-fri-bad-with-partial-candidates-def*
composition-fri-bad-with-reachable-partial-candidate-def
by *blast*

lemma *trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate:*
assumes *trace-fri-bad-with-empty-composition-header-candidates s out*
shows *trace-fri-bad-with-reachable-partial-candidate s out*
using *assms*
unfolding *trace-fri-bad-with-empty-composition-header-candidates-def*
accepted-with-empty-composition-header-candidates-def
trace-fri-bad-with-reachable-partial-candidate-def
by *blast*

lemma *trace-fri-bad-with-partial-candidates-not-None:*
 $\neg$ *trace-fri-bad-with-partial-candidates s None*
unfolding *trace-fri-bad-with-partial-candidates-def*
accepted-with-partial-initial-openings-def *accepted-def*
by *simp*

lemma *composition-fri-bad-with-partial-candidates-not-None:*
 $\neg$ *composition-fri-bad-with-partial-candidates s None*
unfolding *composition-fri-bad-with-partial-candidates-def*
accepted-with-partial-initial-openings-def *accepted-def*
by *simp*

lemma *trace-fri-bad-with-empty-composition-header-candidates-not-None:*
 $\neg$ *trace-fri-bad-with-empty-composition-header-candidates s None*
unfolding *trace-fri-bad-with-empty-composition-header-candidates-def*
accepted-with-empty-composition-header-candidates-def *accepted-def*
by *simp*

lemma *trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction:*
assumes
 $\bigwedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$

```

    trace-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data))
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ trace-fri-error
  proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    show  $\bigwedge s. \neg \text{trace-fri-bad-with-partial-candidates } s \text{ None}$ 
      by (rule trace-fri-bad-with-partial-candidates-not-None)
  next
    fix data attacker-state
    assume builder:
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
                    adversary-initial-state)
    let ?s =
      verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)
    have reduction:
      wp-event verify-monad
        (trace-fri-bad-with-reachable-partial-candidate ?s) ?s ≤
          trace-fri-error
    using assms[OF builder]
    unfolding trace-fri-reachable-partial-candidate-reduction-assumption-def
    by simp
    show wp-event verify-monad (trace-fri-bad-with-partial-candidates ?s) ?s
      ≤ trace-fri-error
    by (rule order-trans
        [OF - reduction])
      (rule wp-event-mono,
       rule trace-fri-bad-with-partial-candidates-imp-reachable-partial-candidate)
  qed

lemma composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction:
  assumes
     $\bigwedge \text{data attacker-state.}$ 
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
                      adversary-initial-state)  $\implies$ 
    composition-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data))
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       composition-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ composition-fri-error

```



```

proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg \text{composition-fri-bad-with-partial-candidates } s \text{ None}$ 
    by (rule composition-fri-bad-with-partial-candidates-not-None)
next
  fix data attacker-state
  assume builder:
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have reduction:
    wp-event verify-monad
      (composition-fri-bad-with-reachable-partial-candidate ?s) ?s  $\leq$ 
        composition-fri-error
  using assms[OF builder]
  unfolding
    composition-fri-reachable-partial-candidate-reduction-assumption-def
  by simp
  show wp-event verify-monad
    (composition-fri-bad-with-partial-candidates ?s) ?s  $\leq$ 
      composition-fri-error
  by (rule order-trans
    [OF - reduction])
    (rule wp-event-mono,
      rule
        composition-fri-bad-with-partial-candidates-imp-reachable-partial-candidate)
qed

```

```

lemma trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction:
  assumes
     $\bigwedge$  data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
      trace-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-empty-composition-header-candidates)
      adversary-initial-state  $\leq$  trace-fri-error
  proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    show  $\bigwedge s. \neg \text{trace-fri-bad-with-empty-composition-header-candidates } s \text{ None}$ 
      by (rule trace-fri-bad-with-empty-composition-header-candidates-not-None)
  next
    fix data attacker-state

```

assume *builder*:
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state)
let *?s* =
verifier-state-from-adversary *attacker-state*
(*staged-proof-transcript* *data*)
have *reduction*:
wp-event *verify-monad*
(*trace-fri-bad-with-reachable-partial-candidate* *?s*) *?s* $\leq$
trace-fri-error
using *assms*[*OF builder*]
unfolding *trace-fri-reachable-partial-candidate-reduction-assumption-def*
by *simp*
show *wp-event* *verify-monad*
(*trace-fri-bad-with-empty-composition-header-candidates* *?s*) *?s* $\leq$
trace-fri-error
by (*rule* *order-trans*
[*OF - reduction*])
(*rule* *wp-event-mono*,
rule
trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate)
qed

definition *empty-header-low-degree-trace-candidate-nonunique*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
empty-header-low-degree-trace-candidate-nonunique *s out* $\longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg final trace-query-idxs}$
trace-openings *trace-table* *composition-table* *trace-table'*.
accepted-with-empty-composition-header-candidates *s out* *fr*
f-fri-roots *f-final* *as dg final trace-query-idxs* *trace-openings*
trace-table *composition-table* $\wedge$
partial-trace-table-candidate *trace-table'* *trace-openings* $\wedge$
trace-table $\neq$ *trace-table'* $\wedge$
trace-table-low-degree *trace-table* $\wedge$
trace-table-low-degree *trace-table'*)

definition *empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit *s out*
 $\longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg final trace-query-idxs}$
trace-openings *trace-table* *composition-table* *trace-table'* *i*.
accepted-with-empty-composition-header-candidates *s out* *fr*
f-fri-roots *f-final* *as dg final trace-query-idxs* *trace-openings*

$\text{trace-table composition-table} \wedge$
 $\text{partial-trace-table-candidate trace-table}' \text{ trace-openings} \wedge$
 $\text{trace-table} \neq \text{trace-table}' \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\text{trace-table-low-degree trace-table}' \wedge$
 $i < \text{rounds} \wedge$
 $\text{map } (!) \text{ trace-table} \text{ (powers-scaled (trace-query-idxs ! i))} =$
 $\text{map } (!) \text{ trace-table}' \text{ (powers-scaled (trace-query-idxs ! i))}$

definition *empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit}$
 $s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr f-fri-roots f-final as dg final trace-query-idxs}$
 $\text{ trace-openings trace-table composition-table trace-table}' i.$
 $\text{accepted-with-empty-composition-header-candidates s out fr}$
 $\text{ f-fri-roots f-final as dg final trace-query-idxs trace-openings}$
 $\text{ trace-table composition-table} \wedge$
 $\text{accepted-transcript-shape s out as trace-query-idxs} \wedge$
 $\text{partial-trace-table-candidate trace-table}' \text{ trace-openings} \wedge$
 $\text{trace-table} \neq \text{trace-table}' \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\text{trace-table-low-degree trace-table}' \wedge$
 $i < \text{rounds} \wedge$
 $\text{map } (!) \text{ trace-table} \text{ (powers-scaled (trace-query-idxs ! i))} =$
 $\text{map } (!) \text{ trace-table}' \text{ (powers-scaled (trace-query-idxs ! i))})$

lemma *empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-imp-query-agreement:*
assumes

$\text{empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit}$
 $s \text{ out}$

shows

$\text{empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit}$
 $s \text{ out}$

using *assms*

unfolding

$\text{empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit-def}$
 $\text{empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit-def}$

by *blast*

definition *trace-table-agreement-indices* $:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat set}$

where

$\text{trace-table-agreement-indices trace-table trace-table}' =$
 $\{\text{idx} \in \text{query-sample-space}.$
 $\text{map } (!) \text{ trace-table} \text{ (powers-scaled idx)} =$
 $\text{map } (!) \text{ trace-table}' \text{ (powers-scaled idx)}\}$

lemma *empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreementE*:

assumes

empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit
s out

obtains *fr f-fri-roots f-final as dg final trace-query-idxs*
trace-openings trace-table composition-table trace-table' i

where

accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table
accepted-transcript-shape s out as trace-query-idxs
partial-trace-table-candidate trace-table' trace-openings
trace-table $\neq$ trace-table'
trace-table-low-degree trace-table
trace-table-low-degree trace-table'
i < rounds
trace-query-idxs ! i $\in$
trace-table-agreement-indices trace-table trace-table'

proof —

obtain *fr f-fri-roots f-final as dg final trace-query-idxs*
trace-openings trace-table composition-table trace-table' i

where *empty*:

accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table

and *shape*: *accepted-transcript-shape s out as trace-query-idxs*

and *cand'*: *partial-trace-table-candidate trace-table' trace-openings*

and *distinct*: *trace-table $\neq$ trace-table'*

and *trace-low*: *trace-table-low-degree trace-table*

and *trace'-low*: *trace-table-low-degree trace-table'*

and *i-bound*: *i < rounds*

and *agree*:

map (! trace-table) (powers-scaled (trace-query-idxs ! i)) =
map (! trace-table') (powers-scaled (trace-query-idxs ! i))

using *assms*

unfolding

empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit-def

by *blast*

have *len-query*: *length trace-query-idxs = rounds*

using *accepted-transcript-shape-query-shapes(1)[OF shape]* .

have *idx-mem*: *trace-query-idxs ! i $\in$ set trace-query-idxs*
by *(rule nth-mem) (use i-bound len-query in simp)*

have *idx-sample*: *trace-query-idxs ! i $\in$ query-sample-space*
by *(rule accepted-transcript-shape-query-sample-space[OF shape idx-mem])*

have *idx-agree*:

trace-query-idxs ! i $\in$
trace-table-agreement-indices trace-table trace-table'

using *idx-sample agree* **unfolding** *trace-table-agreement-indices-def*
by *simp*

show ?thesis
 by (rule that[OF empty shape cand' distinct trace-low trace'-low
 i-bound idx-agree])

qed

lemma trace-table-agreement-indices-card-bound:

assumes trace-low: trace-table-low-degree trace-table
 and trace'-low: trace-table-low-degree trace-table'
 and distinct: trace-table $\neq$ trace-table'
 shows card (trace-table-agreement-indices trace-table trace-table') <
 clength

proof -

from trace-low obtain f where deg-f: degree f < clength
 and trace-table: trace-table = map (poly f) eval-domain
 unfolding trace-table-low-degree-def by blast
 from trace'-low obtain f' where deg-f': degree f' < clength
 and trace-table': trace-table' = map (poly f') eval-domain
 unfolding trace-table-low-degree-def by blast
 let ?A = trace-table-agreement-indices trace-table trace-table'
 let ?R = f - f'
 have A-subset: ?A $\subseteq$ query-sample-space
 unfolding trace-table-agreement-indices-def by auto
 have finite-A: finite ?A
 unfolding trace-table-agreement-indices-def query-sample-space-def by simp
 have inj: inj-on ($\lambda \text{idx}. h \wedge \text{idx} * \text{shift}$) ?A
 proof (intro inj-onI)
 fix i j
 assume i-in: i $\in$?A
 and j-in: j $\in$?A
 and eq: $h \wedge i * \text{shift} = h \wedge j * \text{shift}$
 have i-bound: i < clength * scale
 by (rule query-sample-space-less-domain[OF set-mp[OF A-subset i-in]])
 have j-bound: j < clength * scale
 by (rule query-sample-space-less-domain[OF set-mp[OF A-subset j-in]])
 have $h \wedge i = h \wedge j$
 using eq shift-nonzero by simp
 then show i = j
 by (rule h-power-inj-on-eval-domain[OF i-bound j-bound])

qed

have residual-nonzero: ?R $\neq$ 0

proof

assume ?R = 0
 then have f = f'
 by simp
 then show False
 using distinct trace-table trace-table' by simp

qed

have image-subset:

($\lambda \text{idx}. h \wedge \text{idx} * \text{shift}$) ' ?A $\subseteq$ {x. poly ?R x = 0}

proof
fix x
assume $x\text{-in}$: $x \in (\lambda \text{idx}. h \wedge \text{idx} * \text{shift}) \text{ ' } ?A$
then obtain idx **where** idx-in : $\text{idx} \in ?A$
and $x\text{-eq}$: $x = h \wedge \text{idx} * \text{shift}$
by *blast*
have idx-sample : $\text{idx} \in \text{query-sample-space}$
by (*rule set-mp*[*OF A-subset idx-in*])
have idx-bound : $\text{idx} < \text{clength} * \text{scale}$
by (*rule query-sample-space-less-domain*[*OF idx-sample*])
have *agree*:
 $\text{map } (!) \text{ trace-table } (\text{powers-scaled } \text{idx}) =$
 $\text{map } (!) \text{ trace-table'} (\text{powers-scaled } \text{idx})$
using idx-in **unfolding** *trace-table-agreement-indices-def* **by** *simp*
have ps0 : $\text{powers-scaled } \text{idx} ! 0 = \text{idx}$
using *powers-pos* **unfolding** *powers-scaled-def* **by** *simp*
have len-ps : $0 < \text{length } (\text{powers-scaled } \text{idx})$
using *powers-pos* **unfolding** *powers-scaled-def* **by** *simp*
have value-eq : $\text{trace-table} ! \text{idx} = \text{trace-table'} ! \text{idx}$
using *arg-cong*[*OF agree, of $\lambda x s. xs ! 0$*] ps0 len-ps
by *simp*
have *left*:
 $\text{trace-table} ! \text{idx} = \text{poly } f (h \wedge \text{idx} * \text{shift})$
using *trace-table eval-domain-nth*[*OF idx-bound*] idx-bound
eval-domain-length **by** *simp*
have *right*:
 $\text{trace-table'} ! \text{idx} = \text{poly } f' (h \wedge \text{idx} * \text{shift})$
using *trace-table' eval-domain-nth*[*OF idx-bound*] idx-bound
eval-domain-length **by** *simp*
have $\text{poly } f (h \wedge \text{idx} * \text{shift}) = \text{poly } f' (h \wedge \text{idx} * \text{shift})$
using *value-eq left right* **by** *simp*
then show $x \in \{x. \text{poly } ?R x = 0\}$
using $x\text{-eq}$ **by** *simp*
qed
have $\text{card } ?A = \text{card } ((\lambda \text{idx}. h \wedge \text{idx} * \text{shift}) \text{ ' } ?A)$
by (*simp add: card-image inj*)
also have $\dots \leq \text{card } \{x. \text{poly } ?R x = 0\}$
by (*rule card-mono*[*OF poly-roots-finite*[*OF residual-nonzero*]
image-subset])
also have $\dots \leq \text{degree } ?R$
by (*rule card-poly-roots-bound*[*OF residual-nonzero*])
also have $\dots < \text{clength}$
using *deg-f deg-f'* **by** (*simp add: degree-diff-less*)
finally show $?thesis$.
qed

lemma *trace-table-agreement-indices-fraction-bound*:
assumes trace-low : *trace-table-low-degree trace-table*
and trace'-low : *trace-table-low-degree trace-table'*

and *distinct*: $\text{trace-table} \neq \text{trace-table}'$
shows *nnreal*
 $(\text{card } (\text{trace-table-agreement-indices } \text{trace-table } \text{trace-table}')) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq$
 $\text{nnreal } \text{clength} / \text{nnreal } (\text{card } \text{query-sample-space})$
proof –
have *card-le*:
 $\text{card } (\text{trace-table-agreement-indices } \text{trace-table } \text{trace-table}') \leq$
 clength
using *trace-table-agreement-indices-card-bound*
 $[OF \text{ trace-low } \text{trace}'\text{-low } \text{distinct}]$
by *simp*
show *?thesis*
by (rule *nnreal-nat-divide-right-mono*[*OF card-le*])
qed

lemma *checked-staged-query-prefix-trace-pair-agreement-hit-bound-by-prehit*:
assumes *trace-low*: *trace-table-low-degree trace-table*
and *trace'-low*: *trace-table-low-degree trace-table'*
and *distinct*: $\text{trace-table} \neq \text{trace-table}'$
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $(\lambda - . \text{trace-table-agreement-indices } \text{trace-table } \text{trace-table}'))$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $(\lambda - . \text{trace-table-agreement-indices } \text{trace-table } \text{trace-table}'))$
 $\text{adversary-initial-state} +$
 $\text{nnreal } \text{clength} / \text{nnreal } (\text{card } \text{query-sample-space})$
proof –
have *raw-bound*: *query-index-raw-preimage-bound*
by (rule *query-index-raw-preimage-bound-from-sampler-wellformed*)
show *?thesis*
proof (rule
 $\text{checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error}$
 $[OF \text{ raw-bound}]$)
fix *prefix prefix-state*
assume
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state})$
show
 $\text{trace-table-agreement-indices } \text{trace-table } \text{trace-table}'$
 $\subseteq \text{query-sample-space} \wedge$
 nnreal
 $(\text{card } (\text{trace-table-agreement-indices } \text{trace-table } \text{trace-table}')) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq$

```

    nnreal clength / nnreal (card query-sample-space)
using trace-table-agreement-indices-fraction-bound
    [OF trace-low trace'-low distinct]
unfolding trace-table-agreement-indices-def by auto
qed
qed

definition checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-from-prefix
:: 'f staged-adversary  $\Rightarrow$  'f list  $\Rightarrow$  'f list  $\Rightarrow$  nat  $\Rightarrow$ 
  (((('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
    'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
    'f protocol-channel)  $\times$  unit list)  $\times$ 
    'f protocol-channel) option  $\Rightarrow$  bool
where
  checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-from-prefix
    A trace-table trace-table' i out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state)  $\Rightarrow$ 
    ( $\exists$  prefix prefix-state raw raw-state.
      Some (data, attacker-state)  $\in$ 
        set-dist
          (execute (checked-staged-transcript-program A)
            adversary-initial-state)  $\wedge$ 
        Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
          set-dist
            (execute (checked-staged-query-prefix-receive-with-state A i)
              adversary-initial-state)  $\wedge$ 
            checked-staged-query-prefix-dynamic-index-hit
              ( $\lambda$ - . trace-table-agreement-indices trace-table trace-table')
              (Some (((prefix, prefix-state), raw), raw-state))))))

lemma checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-from-prefixE:
assumes
  checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-from-prefix
    A trace-table trace-table' i
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
obtains prefix prefix-state raw raw-state where
  Some (data, attacker-state)  $\in$ 
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  checked-staged-query-prefix-dynamic-index-hit

```



```

    (λ- -. trace-table-agreement-indices trace-table trace-table')
    (Some (((prefix, prefix-state), raw), raw-state))
  using assms
proof -
  obtain prefix prefix-state raw raw-state where
    builder:
      Some (data, attacker-state) ∈
        set-dist
          (execute (checked-staged-transcript-program A)
            adversary-initial-state)
    and prefix-receive:
      Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
          (execute (checked-staged-query-prefix-receive-with-state A i)
            adversary-initial-state)
    and hit:
      checked-staged-query-prefix-dynamic-index-hit
        (λ- -. trace-table-agreement-indices trace-table trace-table')
        (Some (((prefix, prefix-state), raw), raw-state))
  using assms
  unfolding
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-from-prefix-def
  by simp blast
show ?thesis
  by (rule that[OF builder prefix-receive hit])
qed

definition checked-staged-security-with-actual-alpha-prefix-empty-header-low-degree-prefix-agreement-hit
:: 'f staged-adversary ⇒
  ((((((('f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool
where
  checked-staged-security-with-actual-alpha-prefix-empty-header-low-degree-prefix-agreement-hit
    A out ⟷
  (case out of
    None ⇒ False
  | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state) ⇒
    (let s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    in
      (∃ fr f-fri-roots f-final as dg final trace-query-idxs
        trace-openings trace-table composition-table trace-table' i
        prefix prefix-state raw raw-state.
        accepted-with-empty-composition-header-candidates s
          (Some (result, final-state)) fr f-fri-roots f-final as dg

```

final trace-query-idxs trace-openings trace-table
composition-table $\wedge$
accepted-transcript-shape s (Some (result, final-state)) as
trace-query-idxs $\wedge$
partial-trace-table-candidate trace-table' trace-openings $\wedge$
trace-table $\neq$ *trace-table'* $\wedge$
trace-table-low-degree trace-table $\wedge$
trace-table-low-degree trace-table' $\wedge$
i < *rounds* $\wedge$
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state) ∧
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state) ∧
index (to-nat raw) ∈
trace-table-agreement-indices trace-table trace-table'))

lemma *empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-imp-prefix-agreement-hit-on-*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

out ∈

set-dist

(execute

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

A)

adversary-initial-state)

and *hit:*

checked-staged-security-with-actual-alpha-prefix-verifier-event

empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit

out

shows

checked-staged-security-with-actual-alpha-prefix-empty-header-low-degree-prefix-agreement-hit

A out

proof *(cases out)*

case *None*

then show *?thesis*

using *hit*

unfolding *checked-staged-security-with-actual-alpha-prefix-verifier-event-def*

checked-staged-security-with-actual-alpha-prefix-empty-header-low-degree-prefix-agreement-hit-def

by *simp*

next

case *(Some packed)*

then obtain *alpha-prefix alpha-prefix-state data attacker-state result*

final-state where out-eq:

out =

```

    Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have local-hit:
  empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit
  ?s (Some (result, final-state))
using hit
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
by simp
from empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreementE
  [OF local-hit]
obtain fr f-fri-roots f-final as dg final trace-query-idxs
  trace-openings trace-table composition-table trace-table' i where
  empty:
    accepted-with-empty-composition-header-candidates ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg final
    trace-query-idxs trace-openings trace-table composition-table
  and shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as
    trace-query-idxs
  and cand': partial-trace-table-candidate trace-table' trace-openings
  and distinct: trace-table  $\neq$  trace-table'
  and trace-low: trace-table-low-degree trace-table
  and trace'-low: trace-table-low-degree trace-table'
  and i-bound: i < rounds
  and idx-agree:
    trace-query-idxs ! i  $\in$ 
    trace-table-agreement-indices trace-table trace-table'
  by blast
have data-support:
  Some (((data, attacker-state), result), final-state)  $\in$ 
  set-dist
  (execute (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support[unfolded out-eq]])
from
  checked-staged-security-with-data-state-accepted-shape-query-prefix-receive-index
  [OF wf controlled data-support shape i-bound]
obtain prefix prefix-state raw raw-state where
  prefix-receive:
    Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)

```

```

      adversary-initial-state)
    and raw-idx: index (to-nat raw) = trace-query-idxs ! i
    by blast
  have raw-agree:
    index (to-nat raw) ∈
      trace-table-agreement-indices trace-table trace-table'
    using raw-idx idx-agree by simp
  have builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  proof -
    from checked-staged-security-experiment-with-data-state-outcomeE
      [OF data-support]
    show ?thesis by blast
  qed
  show ?thesis
    unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-empty-header-low-degree-prefix-agreement-hit-def
    apply (simp add: Let-def)
    by (intro exI[of - fr] exI[of - f-fri-roots]
      exI[of - f-final] exI[of - as] exI[of - dg]
      exI[of - final] exI[of - trace-query-idxs]
      exI[of - trace-openings] exI[of - trace-table]
      exI[of - composition-table] exI[of - trace-table']
      exI[of - i] exI[of - prefix] exI[of - prefix-state]
      exI[of - raw] exI[of - raw-state] conjI empty shape cand'
      distinct trace-low trace'-low i-bound builder prefix-receive
      raw-agree)
  qed

lemma empty-header-low-degree-trace-candidate-nonunique-imp-query-agreement-hit:
  assumes empty-header-low-degree-trace-candidate-nonunique s out
  shows
    empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit
      s out
  proof -
    obtain fr f-fri-roots f-final as dg final trace-query-idxs
      trace-openings trace-table composition-table trace-table' where empty:
      accepted-with-empty-composition-header-candidates s out fr
      f-fri-roots f-final as dg final trace-query-idxs trace-openings
      trace-table composition-table
    and cand': partial-trace-table-candidate trace-table' trace-openings
    and distinct: trace-table ≠ trace-table'
    and trace-low: trace-table-low-degree trace-table
    and trace'-low: trace-table-low-degree trace-table'
    using assms
    unfolding empty-header-low-degree-trace-candidate-nonunique-def by blast

```

```

have partial:
  accepted-with-partial-trace-openings s out fr trace-query-idxs
    trace-openings
  using empty unfolding accepted-with-empty-composition-header-candidates-def
  by blast
have cand:
  partial-trace-table-candidate trace-table trace-openings
  using empty unfolding accepted-with-empty-composition-header-candidates-def
  by blast
have idxs:
  map opening-index (trace-openings ! 0) =
    powers-scaled (trace-query-idxs ! 0)
  using accepted-with-partial-trace-openings-shapes(3)
    [OF partial rounds-positive] .
have vals:
  map (! trace-table) (powers-scaled (trace-query-idxs ! 0)) =
    map (! trace-table') (powers-scaled (trace-query-idxs ! 0))
proof –
  have left:
    map (! trace-table) (powers-scaled (trace-query-idxs ! 0)) =
      map opening-value (trace-openings ! 0)
    by (rule partial-trace-table-candidate-query-values
      [OF cand rounds-positive idxs])
  have right:
    map (! trace-table') (powers-scaled (trace-query-idxs ! 0)) =
      map opening-value (trace-openings ! 0)
    by (rule partial-trace-table-candidate-query-values
      [OF cand' rounds-positive idxs])
  show ?thesis
    using left right by simp
qed
show ?thesis
  unfolding
    empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit-def
  using empty cand' distinct trace-low trace'-low rounds-positive vals
  by blast
qed

lemma empty-header-low-degree-trace-candidate-nonunique-le-query-agreement-hit:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      empty-header-low-degree-trace-candidate-nonunique)
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit)
    adversary-initial-state

```

by (rule wp-event-mono)
(auto simp:
checked-staged-security-with-actual-alpha-prefix-verifier-event-def
split: option.splits prod.splits
dest:
empty-header-low-degree-trace-candidate-nonunique-imp-query-agreement-hit)

lemma empty-header-partial-trace-candidate-nonunique-imp-trace-fri-or-low-degree-nonunique:
assumes empty-header-partial-trace-candidate-nonunique s out
shows

trace-fri-bad-with-empty-composition-header-candidates s out $\vee$
empty-header-low-degree-trace-candidate-nonunique s out

proof —

obtain fr f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table **where** empty:
accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table

and not-singleton:

$\neg (\exists \text{ trace-table0.}$
partial-trace-table-candidates trace-openings $\subseteq$
{trace-table0})

using assms

unfolding empty-header-partial-trace-candidate-nonunique-def **by** blast

have trace-candidate:

trace-table $\in$ partial-trace-table-candidates trace-openings

using empty

unfolding accepted-with-empty-composition-header-candidates-def
partial-trace-table-candidates-def

by simp

obtain trace-table' **where** trace-candidate':

trace-table' $\in$ partial-trace-table-candidates trace-openings

and distinct: trace-table $\neq$ trace-table'

proof —

have $\neg$ partial-trace-table-candidates trace-openings $\subseteq$
{trace-table}

using not-singleton **by** blast

then obtain trace-table' **where**

trace-table' $\in$ partial-trace-table-candidates trace-openings

trace-table' $\notin$ {trace-table}

by blast

then show ?thesis

using that **by** blast

qed

show ?thesis

proof (cases trace-table-low-degree trace-table)

case False

then have trace-fri-bad-with-empty-composition-header-candidates s out

unfolding trace-fri-bad-with-empty-composition-header-candidates-def

```

    using empty by blast
  then show ?thesis by simp
next
case trace-low: True
show ?thesis
proof (cases trace-table-low-degree trace-table')
  case False
  have empty':
    accepted-with-empty-composition-header-candidates s out fr
      f-fri-roots f-final as dg final trace-query-idxs trace-openings
      trace-table' composition-table
  using empty trace-candidate'
  unfolding accepted-with-empty-composition-header-candidates-def
    partial-trace-table-candidates-def
  by blast
  then have
    trace-fri-bad-with-empty-composition-header-candidates s out
  unfolding trace-fri-bad-with-empty-composition-header-candidates-def
  using False by blast
  then show ?thesis by simp
next
case trace'-low: True
have empty-header-low-degree-trace-candidate-nonunique s out
  unfolding empty-header-low-degree-trace-candidate-nonunique-def
  using empty trace-candidate' distinct trace-low trace'-low
  unfolding partial-trace-table-candidates-def
  by blast
  then show ?thesis by simp
qed
qed
qed

```

definition *empty-header-partial-trace-candidate-nonunique-transcript*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

empty-header-partial-trace-candidate-nonunique-transcript s out $\longleftrightarrow$
 $(\exists \text{ fr f-fri-roots f-final as dg final trace-query-idxs}$
 $\text{ trace-openings trace-table composition-table.}$
 $\text{accepted-with-empty-composition-header-candidates s out fr}$
 $\text{ f-fri-roots f-final as dg final trace-query-idxs trace-openings}$
 $\text{ trace-table composition-table} \wedge$
 $\text{accepted-transcript-shape s out as trace-query-idxs} \wedge$
 $\neg (\exists \text{ trace-table0.}$
 $\text{partial-trace-table-candidates trace-openings} \subseteq$
 $\{\text{trace-table0}\}))$

lemma *empty-header-partial-trace-candidate-nonunique-transcript-imp-trace-fri-or-low-degree-transcript-query-agree*

assumes *empty-header-partial-trace-candidate-nonunique-transcript* s out

shows
trace-fri-bad-with-empty-composition-header-candidates s out $\vee$
empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit
s out
proof –
obtain *fr f-fri-roots f-final as dg final trace-query-idxs trace-openings*
trace-table composition-table **where** *empty*:
accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table
and *shape: accepted-transcript-shape s out as trace-query-idxs*
and *not-singleton*:
 $\neg (\exists \text{ trace-table0.}$
 $\text{partial-trace-table-candidates trace-openings} \subseteq$
 $\{\text{trace-table0}\})$
using *assms*
unfolding *empty-header-partial-trace-candidate-nonunique-transcript-def*
by *blast*
have *trace-candidate*:
trace-table $\in$ *partial-trace-table-candidates trace-openings*
using *empty*
unfolding *accepted-with-empty-composition-header-candidates-def*
partial-trace-table-candidates-def
by *simp*
obtain *trace-table'* **where** *trace-candidate'*:
trace-table' $\in$ *partial-trace-table-candidates trace-openings*
and *distinct*: *trace-table* $\neq$ *trace-table'*
proof –
have $\neg \text{partial-trace-table-candidates trace-openings} \subseteq$
 $\{\text{trace-table}\}$
using *not-singleton* **by** *blast*
then obtain *trace-table'* **where**
trace-table' $\in$ *partial-trace-table-candidates trace-openings*
trace-table' $\notin$ $\{\text{trace-table}\}$
by *blast*
then show *?thesis*
using *that* **by** *blast*
qed
show *?thesis*
proof (*cases trace-table-low-degree trace-table*)
case *False*
then have *trace-fri-bad-with-empty-composition-header-candidates s out*
unfolding *trace-fri-bad-with-empty-composition-header-candidates-def*
using *empty* **by** *blast*
then show *?thesis* **by** *simp*
next
case *trace-low: True*
show *?thesis*
proof (*cases trace-table-low-degree trace-table'*)


```

case False
have empty':
  accepted-with-empty-composition-header-candidates s out fr
    f-fri-roots f-final as dg final trace-query-idxs trace-openings
    trace-table' composition-table
  using empty trace-candidate'
  unfolding accepted-with-empty-composition-header-candidates-def
    partial-trace-table-candidates-def
  by blast
then have
  trace-fri-bad-with-empty-composition-header-candidates s out
  unfolding trace-fri-bad-with-empty-composition-header-candidates-def
  using False by blast
then show ?thesis by simp
next
case trace'-low: True
have idxs:
  map opening-index (trace-openings ! 0) =
    powers-scaled (trace-query-idxs ! 0)
proof –
  have partial:
    accepted-with-partial-trace-openings s out fr trace-query-idxs
    trace-openings
  using empty
  unfolding accepted-with-empty-composition-header-candidates-def
  by blast
show ?thesis
  using accepted-with-partial-trace-openings-shapes(3)
    [OF partial rounds-positive] .
qed
have vals:
  map ((! trace-table) (powers-scaled (trace-query-idxs ! 0))) =
    map ((! trace-table') (powers-scaled (trace-query-idxs ! 0)))
proof –
  have cand:
    partial-trace-table-candidate trace-table trace-openings
  using empty
  unfolding accepted-with-empty-composition-header-candidates-def
  by blast
have cand':
  partial-trace-table-candidate trace-table' trace-openings
  using trace-candidate'
  unfolding partial-trace-table-candidates-def by simp
have left:
  map ((! trace-table) (powers-scaled (trace-query-idxs ! 0))) =
    map opening-value (trace-openings ! 0)
  by (rule partial-trace-table-candidate-query-values
    [OF cand rounds-positive idxs])
have right:

```

```

    map ((! trace-table') (powers-scaled (trace-query-idxs ! 0)) =
      map opening-value (trace-openings ! 0)
  by (rule partial-trace-table-candidate-query-values
    [OF cand' rounds-positive idxs])
  show ?thesis
    using left right by simp
qed
have
  empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit
    s out
  unfolding
    empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit-def
  using empty shape trace-candidate' distinct trace-low trace'-low
    rounds-positive vals
  unfolding partial-trace-table-candidates-def
  by blast
  then show ?thesis by simp
qed
qed
qed
qed

lemma empty-header-partial-trace-candidate-nonunique-transcript-le-trace-fri-or-low-degree-transcript-query-agreement
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      empty-header-partial-trace-candidate-nonunique-transcript)
  adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state +
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit)
  adversary-initial-state
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?N =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-partial-trace-candidate-nonunique-transcript
  let ?T =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    trace-fri-bad-with-empty-composition-header-candidates
  let ?L =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit

```

have *wp-event* ?M ?N *adversary-initial-state* ≤
wp-event ?M (λout. ?T out ∨ ?L out) *adversary-initial-state*
by (rule *wp-event-mono*)
(auto simp: *checked-staged-security-with-actual-alpha-prefix-verifier-event-def*
split: option.splits prod.splits
dest:
empty-header-partial-trace-candidate-nonunique-transcript-imp-trace-fri-or-low-degree-transcript-query-agreement)
also have ... ≤
wp-event ?M ?T *adversary-initial-state* +
wp-event ?M ?L *adversary-initial-state*
by (rule *wp-event-union-bound*)
finally show ?thesis .
qed

definition *composition-alpha-partial-opening-union-hit-with-empty-header-transcript*

:: (f, 'a) *protocol-channel-scheme* ⇒
(unit list × (f, 'a) *protocol-channel-scheme*) *option* ⇒ bool

where

composition-alpha-partial-opening-union-hit-with-empty-header-transcript
s out ⇔
(∃ fr f-fri-roots f-final as dg final *trace-query-idxs*
trace-openings trace-table composition-table.
accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final *trace-query-idxs trace-openings*
trace-table composition-table ∧
accepted-transcript-shape s out as trace-query-idxs ∧
as ∈ *partial-trace-opening-alpha-union-bad-sets trace-openings*)

lemma *composition-alpha-partial-opening-union-hit-with-empty-header-transcript-imp-union-hit:*

assumes

composition-alpha-partial-opening-union-hit-with-empty-header-transcript
s out

shows

composition-alpha-partial-opening-union-hit-with-empty-header s out

using *assms*

unfolding

composition-alpha-partial-opening-union-hit-with-empty-header-transcript-def
composition-alpha-partial-opening-union-hit-with-empty-header-def
by *blast*

definition *composition-bad-with-empty-composition-header-candidates-transcript*

:: (f, 'a) *protocol-channel-scheme* ⇒
(unit list × (f, 'a) *protocol-channel-scheme*) *option* ⇒ bool

where

composition-bad-with-empty-composition-header-candidates-transcript
s out ⇔
(∃ fr f-fri-roots f-final as dg final *trace-query-idxs*
trace-openings trace-table composition-table f.
accepted-with-empty-composition-header-candidates s out fr)

f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table $\wedge$
accepted-transcript-shape s out as trace-query-idxs $\wedge$
degree f < clength $\wedge$
trace-table = map (poly f) eval-domain $\wedge$
violated-constraints f $\neq$ {} $\wedge$
composition-table-low-degree maxDegree composition-table $\wedge$
all-queries-consistent trace-table composition-table as)

definition *composition-degree-bad-with-empty-composition-header-candidates-transcript*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-degree-bad-with-empty-composition-header-candidates-transcript
s out $\longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg final trace-query-idxs}$
trace-openings trace-table composition-table f.
accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table $\wedge$
accepted-transcript-shape s out as trace-query-idxs $\wedge$
degree f < clength $\wedge$
trace-table = map (poly f) eval-domain $\wedge$
violated-constraints f $\neq$ {} $\wedge$
composition-table-low-degree maxDegree composition-table $\wedge$
all-queries-consistent trace-table composition-table as $\wedge$
 $\neg \text{ common-denominator-degree-bounds f as})$

definition *composition-randomization-bad-with-empty-composition-header-candidates-transcript*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-randomization-bad-with-empty-composition-header-candidates-transcript
s out $\longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg final trace-query-idxs}$
trace-openings trace-table composition-table f.
accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table $\wedge$
accepted-transcript-shape s out as trace-query-idxs $\wedge$
degree f < clength $\wedge$
trace-table = map (poly f) eval-domain $\wedge$
violated-constraints f $\neq$ {} $\wedge$
composition-table-low-degree maxDegree composition-table $\wedge$
all-queries-consistent trace-table composition-table as $\wedge$
common-denominator-degree-bounds f as $\wedge$
random-combination-common-denominator-hides-violations f as)

lemma *composition-bad-with-empty-composition-header-candidates-transcript-split:*

```

assumes false-statement:  $\neg$  exists-valid-trace
and bad:
  composition-bad-with-empty-composition-header-candidates-transcript
  s out
shows
  composition-degree-bad-with-empty-composition-header-candidates-transcript
  s out  $\vee$ 
  composition-randomization-bad-with-empty-composition-header-candidates-transcript
  s out
proof –
obtain fr f-fri-roots f-final as dg final trace-query-idxs
  trace-openings trace-table composition-table f where partial:
    accepted-with-empty-composition-header-candidates s out fr
    f-fri-roots f-final as dg final trace-query-idxs trace-openings
    trace-table composition-table
and shape: accepted-transcript-shape s out as trace-query-idxs
and deg-f: degree f < clength
and trace-table: trace-table = map (poly f) eval-domain
and violated: violated-constraints f  $\neq$  {}
and composition-low:
  composition-table-low-degree maxDegree composition-table
and all-queries:
  all-queries-consistent trace-table composition-table as
using bad
unfolding
  composition-bad-with-empty-composition-header-candidates-transcript-def
by blast
show ?thesis
proof (cases common-denominator-degree-bounds f as)
case False
then have
  composition-degree-bad-with-empty-composition-header-candidates-transcript
  s out
unfolding
  composition-degree-bad-with-empty-composition-header-candidates-transcript-def
using partial shape deg-f trace-table violated composition-low
  all-queries
by blast
then show ?thesis by simp
next
case True
note bounds = True
have len-as: length as = length spec
proof –
obtain result final-state fr' f-fri-roots' f-final' dg'
  composition-fri-roots' final' rest' where
    verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
    composition-fri-roots' final' rest'
using shape

```

```

    by (elim accepted-transcript-shape-header-query-extraction)
  then show ?thesis
    by (rule verifier-header-transcript-shapes(3))
qed
have hides:
  random-combination-common-denominator-hides-violations f as
  by (rule random-combination-common-denominator-hides-from-queries
    [OF len-as false-statement deg-f trace-table composition-low
      all-queries bounds])
have
  composition-randomization-bad-with-empty-composition-header-candidates-transcript
    s out
  unfolding
    composition-randomization-bad-with-empty-composition-header-candidates-transcript-def
  using partial shape deg-f trace-table violated composition-low
    all-queries bounds hides
  by blast
then show ?thesis by simp
qed
qed

lemma composition-degree-bad-with-empty-composition-header-candidates-transcript-imp-degree-bad:
  assumes
    composition-degree-bad-with-empty-composition-header-candidates-transcript
      s out
  shows
    composition-degree-bad-with-empty-composition-header-candidates s out
  using assms
  unfolding
    composition-degree-bad-with-empty-composition-header-candidates-transcript-def
    composition-degree-bad-with-empty-composition-header-candidates-def
  by blast

lemma composition-degree-bad-with-empty-composition-header-candidates-transcript-false:
   $\neg$  composition-degree-bad-with-empty-composition-header-candidates-transcript
    s out
  using
    composition-degree-bad-with-empty-composition-header-candidates-transcript-imp-degree-bad
    composition-degree-bad-with-empty-composition-header-candidates-false
  by blast

lemma composition-degree-bad-with-empty-composition-header-candidates-transcript-bound:
  wp-event verify-monad
    (composition-degree-bad-with-empty-composition-header-candidates-transcript
      s) s  $\leq$ 
    0
proof –
  have composition-degree-bad-with-empty-composition-header-candidates-transcript
    s =

```

```

    ( $\lambda out :: (unit\ list \times ('f, 'a)\ protocol-channel-scheme)\ option.$ 
      False)
  by (rule ext)
    (simp add:
      composition-degree-bad-with-empty-composition-header-candidates-transcript-false)
  then show ?thesis
    unfolding wp-event-def wp-def dist-expect-def by simp
qed

lemma composition-randomization-bad-with-empty-composition-header-candidates-transcript-imp-alpha-union-hit:
  assumes bad:
    composition-randomization-bad-with-empty-composition-header-candidates-transcript
      s out
  shows
    composition-alpha-partial-opening-union-hit-with-empty-header-transcript
      s out
proof -
  obtain fr f-fri-roots f-final as dg final trace-query-idxs
    trace-openings trace-table composition-table f where partial:
    accepted-with-empty-composition-header-candidates s out fr
      f-fri-roots f-final as dg final trace-query-idxs trace-openings
      trace-table composition-table
  and shape: accepted-transcript-shape s out as trace-query-idxs
  and deg-f: degree f < clength
  and trace-table: trace-table = map (poly f) eval-domain
  and violated: violated-constraints f  $\neq \{\}$ 
  and bounds: common-denominator-degree-bounds f as
  and hides:
    random-combination-common-denominator-hides-violations f as
  using bad
  unfolding
    composition-randomization-bad-with-empty-composition-header-candidates-transcript-def
  by blast
have as-in:
  as  $\in$  common-denominator-hiding-alpha-space f
proof -
  obtain result final-state fr' f-fri-roots' f-final' dg'
    composition-fri-roots' final' rest' where header:
    verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
      composition-fri-roots' final' rest'
  using shape
  by (elim accepted-transcript-shape-header-query-extraction)
have len-as: length as = length spec
  by (rule verifier-header-transcript-shapes(3)[OF header])
show ?thesis
  using len-as hides
  unfolding common-denominator-hiding-alpha-space-def alpha-space-def
  by simp
qed

```

have *candidate*:
trace-table $\in$ *partial-trace-table-candidates* *trace-openings*
using *partial*
unfolding *accepted-with-empty-composition-header-candidates-def*
partial-trace-table-candidates-def
by *simp*
have *witness-eq*: *low-degree-trace-witness* *trace-table* = *f*
by (*rule* *low-degree-trace-witness-eq*[*OF* *deg-f* *trace-table*])
have *as-bad*: *as* $\in$ *composition-trace-bad-alpha-space* *trace-table*
using *deg-f* *trace-table* *violated* *bounds* *as-in*
unfolding *composition-trace-bad-alpha-space-def* *witness-eq*
by *simp*
have *as* $\in$ *partial-trace-opening-alpha-union-bad-sets* *trace-openings*
unfolding *partial-trace-opening-alpha-union-bad-sets-def*
using *candidate* *as-bad* **by** *blast*
then show *?thesis*
unfolding
composition-alpha-partial-opening-union-hit-with-empty-header-transcript-def
using *partial* *shape* **by** *blast*
qed

lemma *composition-alpha-partial-opening-union-hit-with-empty-header-transcript-split*:
assumes *hit*:
composition-alpha-partial-opening-union-hit-with-empty-header-transcript
s out
shows
composition-alpha-bad-set-hit-with-empty-composition-header-candidates
s out $\vee$
empty-header-partial-trace-candidate-nonunique-transcript *s out*
proof –
obtain *fr* *f-fri-roots* *f-final* *as* *dg* *final* *trace-query-idxs*
trace-openings *trace-table* *composition-table* **where** *partial*:
accepted-with-empty-composition-header-candidates *s out* *fr*
f-fri-roots *f-final* *as* *dg* *final* *trace-query-idxs* *trace-openings*
trace-table *composition-table*
and *shape*: *accepted-transcript-shape* *s out* *as* *trace-query-idxs*
and *as-union*:
as $\in$ *partial-trace-opening-alpha-union-bad-sets* *trace-openings*
using *hit*
unfolding
composition-alpha-partial-opening-union-hit-with-empty-header-transcript-def
by *blast*
from *as-union* **obtain** *trace-table'* **where** *candidate'*:
trace-table' $\in$ *partial-trace-table-candidates* *trace-openings*
and *as-bad'*: *as* $\in$ *composition-trace-bad-alpha-space* *trace-table'*
unfolding *partial-trace-opening-alpha-union-bad-sets-def* **by** *blast*
have *candidate*:
trace-table $\in$ *partial-trace-table-candidates* *trace-openings*
using *partial*


```

unfolding accepted-with-empty-composition-header-candidates-def
  partial-trace-table-candidates-def
by simp
show ?thesis
proof (cases
   $\exists \text{ trace-table0.}$ 
   $\text{partial-trace-table-candidates } \text{trace-openings} \subseteq$ 
   $\{\text{trace-table0}\}$ )
case True
then obtain trace-table0 where subset:
   $\text{partial-trace-table-candidates } \text{trace-openings} \subseteq$ 
   $\{\text{trace-table0}\}$ 
by blast
have trace-eq:  $\text{trace-table} = \text{trace-table0}$ 
  using subset candidate by blast
have trace'-eq:  $\text{trace-table}' = \text{trace-table0}$ 
  using subset candidate' by blast
have  $as \in \text{composition-trace-bad-alpha-space } \text{trace-table}$ 
  using as-bad' trace-eq trace'-eq by simp
then have
  composition-alpha-bad-set-hit-with-empty-composition-header-candidates
  s out
unfolding
  composition-alpha-bad-set-hit-with-empty-composition-header-candidates-def
using partial by blast
then show ?thesis by simp
next
case False
have empty-header-partial-trace-candidate-nonunique-transcript s out
  unfolding empty-header-partial-trace-candidate-nonunique-transcript-def
  using partial shape False by blast
then show ?thesis by simp
qed
qed

definition checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
  :: (((((('f × 'f list × 'f list × 'f) ×
  'f protocol-channel) × 'f staged-proof-data) ×
  'f protocol-channel) × unit list) ×
  'f protocol-channel) option  $\Rightarrow$  bool
where
  checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
  out  $\longleftrightarrow$ 
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
  composition-trace-bad-alpha-space out  $\wedge$ 
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-alpha-partial-opening-union-hit-with-empty-header-transcript
  out

```

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-le-fixed-or-nonunique-*
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
 adversary-initial-state $\leq$
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
 adversary-initial-state +
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 empty-header-partial-trace-candidate-nonunique-transcript)
 adversary-initial-state

proof –
 let $?M =$
 checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
 let $?fixed =$
 checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-bad-set-hit-with-empty-composition-header-candidates
 let $?nonunique =$
 checked-staged-security-with-actual-alpha-prefix-verifier-event
 empty-header-partial-trace-candidate-nonunique-transcript
 have *wp-event* $?M$
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
 adversary-initial-state $\leq$
wp-event $?M$ ($\lambda out. ?fixed\ out \vee ?nonunique\ out$)
 adversary-initial-state
 by (rule *wp-event-mono*)
 (auto simp:
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-def
 checked-staged-security-with-actual-alpha-prefix-verifier-event-def
 split: option.splits prod.splits
 dest:
 composition-alpha-partial-opening-union-hit-with-empty-header-transcript-split)
 also have ... $\leq$
wp-event $?M$ $?fixed$ adversary-initial-state +
wp-event $?M$ $?nonunique$ adversary-initial-state
 by (rule *wp-event-union-bound*)
 finally show *?thesis* .
 qed

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-fixed-and-*
assumes *fixed-bound*:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event

$\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq F$
and nonunique-bound :
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{empty-header-partial-trace-candidate-nonunique-transcript})$
 $\text{adversary-initial-state} \leq N$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript}$
 $\text{adversary-initial-state} \leq F + N$
by $(\text{rule order-trans[}$
 OF
 $\text{checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-le-fixed-or-nonunique-tran}$
 $(\text{intro add-mono fixed-bound nonunique-bound})$

lemma $\text{checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-le-relevant-drift}$:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript}$
 $\text{adversary-initial-state} \leq$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift}$
 $\text{adversary-initial-state}$
by $(\text{rule wp-event-mono})$
 $(\text{auto simp}$:
 $\text{checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-def}$
 $\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-def}$
 $\text{checked-staged-security-with-actual-alpha-prefix-verifier-event-def}$
 $\text{split: option.splits prod.splits}$
 intro :
 $\text{composition-alpha-partial-opening-union-hit-with-empty-header-transcript-imp-union-hit})$

lemma $\text{checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-relevant-d}$:
assumes drift-bound :
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift}$
 $\text{adversary-initial-state} \leq D$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript}$
 $\text{adversary-initial-state} \leq D$

by (rule order-trans[
 OF
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-le-relevant-drift
 drift-bound])

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-le-fixed-or-trace-fri-or-*

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state *A*)

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript

adversary-initial-state $\leq$

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state *A*)

(checked-staged-security-with-actual-alpha-prefix-verifier-event

composition-alpha-bad-set-hit-with-empty-composition-header-candidates)

adversary-initial-state +

(*wp-event*

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state *A*)

(checked-staged-security-with-actual-alpha-prefix-verifier-event

trace-fri-bad-with-empty-composition-header-candidates)

adversary-initial-state +

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state *A*)

(checked-staged-security-with-actual-alpha-prefix-verifier-event

empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit)

adversary-initial-state)

proof –

let ?*M* =

checked-staged-security-experiment-with-actual-alpha-prefix-data-state *A*

let ?*fixed* =

checked-staged-security-with-actual-alpha-prefix-verifier-event

composition-alpha-bad-set-hit-with-empty-composition-header-candidates

let ?*nonunique* =

checked-staged-security-with-actual-alpha-prefix-verifier-event

empty-header-partial-trace-candidate-nonunique-transcript

let ?*trace-fri* =

checked-staged-security-with-actual-alpha-prefix-verifier-event

trace-fri-bad-with-empty-composition-header-candidates

let ?*low-degree* =

checked-staged-security-with-actual-alpha-prefix-verifier-event

empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit

have *split*:

wp-event ?*M*

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript

adversary-initial-state $\leq$

wp-event ?*M* ?*fixed* adversary-initial-state +

wp-event ?*M* ?*nonunique* adversary-initial-state

by (rule

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-le-fixed-or-nonunique-tran

have *nonunique*:

$wp\text{-event } ?M \text{ ?nonunique adversary-initial-state} \leq$
 $wp\text{-event } ?M \text{ ?trace-fri adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?low-degree adversary-initial-state}$
by (rule
 $empty\text{-header-partial-trace-candidate-nonunique-transcript-le-trace-fri-or-low-degree-transcript-query-agreement}$)
show $?thesis$
by (rule $order\text{-trans}[OF \text{ split}]$)
 $(intro \text{ add-mono } order\text{-refl } nonunique)$
qed

lemma $checked\text{-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-fixed-trace}$
assumes $fixed\text{-bound}$:

$wp\text{-event}$
 $(checked\text{-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(checked\text{-staged-security-with-actual-alpha-prefix-verifier-event}$
 $composition\text{-alpha-bad-set-hit-with-empty-composition-header-candidates})$
 $adversary\text{-initial-state} \leq F$

and $trace\text{-fri-bound}$:

$wp\text{-event}$
 $(checked\text{-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(checked\text{-staged-security-with-actual-alpha-prefix-verifier-event}$
 $trace\text{-fri-bad-with-empty-composition-header-candidates})$
 $adversary\text{-initial-state} \leq T$

and $low\text{-degree-bound}$:

$wp\text{-event}$
 $(checked\text{-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(checked\text{-staged-security-with-actual-alpha-prefix-verifier-event}$
 $empty\text{-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit})$
 $adversary\text{-initial-state} \leq L$

shows

$wp\text{-event}$
 $(checked\text{-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $checked\text{-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript}$
 $adversary\text{-initial-state} \leq F + (T + L)$

by (rule $order\text{-trans}[$
 OF

$checked\text{-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-le-fixed-or-trace-fri-or-low}$
 $(intro \text{ add-mono } fixed\text{-bound } add\text{-mono } trace\text{-fri-bound } low\text{-degree-bound})$

lemma $empty\text{-header-partial-trace-candidate-nonunique-le-trace-fri-or-low-degree-nonunique}$:

$wp\text{-event}$
 $(checked\text{-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(checked\text{-staged-security-with-actual-alpha-prefix-verifier-event}$
 $empty\text{-header-partial-trace-candidate-nonunique})$
 $adversary\text{-initial-state} \leq$
 $wp\text{-event}$

```

    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state +
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-low-degree-trace-candidate-nonunique)
  adversary-initial-state
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?N =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-partial-trace-candidate-nonunique
  let ?T =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    trace-fri-bad-with-empty-composition-header-candidates
  let ?L =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-low-degree-trace-candidate-nonunique
  have wp-event ?M ?N adversary-initial-state ≤
    wp-event ?M (λout. ?T out ∨ ?L out) adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      split: option.splits prod.splits
      dest:
        empty-header-partial-trace-candidate-nonunique-imp-trace-fri-or-low-degree-nonunique)
  also have ... ≤
    wp-event ?M ?T adversary-initial-state +
    wp-event ?M ?L adversary-initial-state
  by (rule wp-event-union-bound)
  finally show ?thesis .
qed

```

```

theorem stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-f
fixes D empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
  ∧data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    trace-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))

```

and *composition-fri-reduction*:
 $\bigwedge \text{data attacker-state.}$
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)

adversary-initial-state) $\implies$
composition-fri-reachable-partial-candidate-reduction-assumption
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*))

and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
(*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* *i*)
adversary-initial-state $\leq C i$

and *drift-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
adversary-initial-state $\leq D$

and *current-empty-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-current-empty-query-opening-hit-at* *0*)
adversary-initial-state $\leq \text{empty-query-error}$

shows *checked-staged-adversary-acceptance-probability* *A* $\leq$
hash-collision-budget-value *0*
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 $(\sum i < \text{rounds}. C i) +$
trace-fri-error + *composition-fri-error* +
 $(\sum i < \text{rounds}. C i) +$
(*composition-error-bound* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* *0*) +
hash-collision-budget-value *0*
(*staged-alpha-search-queries* *budgets* *0*) +
D +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* *0*) +
trace-fri-error + *empty-query-error*)

proof –
have *trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq \text{trace-fri-error}$

by (*rule*
trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction
[*OF trace-fri-reduction*])

```

have comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ composition-fri-error
by (rule
     composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction
     [OF composition-fri-reduction])
have empty-trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state ≤ trace-fri-error
by (rule
     trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
     [OF trace-fri-reduction])
show ?thesis
by (rule
     stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query
     [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      round-bound drift-bound empty-trace-fri-bound current-empty-bound])
qed

theorem stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-f
fixes F X empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      trace-fri-reachable-partial-candidate-reduction-assumption
        (verifier-state-from-adversary attacker-state
         (staged-proof-transcript data))
and composition-fri-reduction:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      composition-fri-reachable-partial-candidate-reduction-assumption
        (verifier-state-from-adversary attacker-state
         (staged-proof-transcript data))
and round-bound:
  ∧ i. i < rounds ⇒
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)

```


(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq C\ i$
and fixed-empty-alpha-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
 adversary-initial-state $\leq F$
and cross-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
 adversary-initial-state $\leq X$
and current-empty-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq$ empty-query-error
shows checked-staged-adversary-acceptance-probability $A \leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds}. C\ i$) +
 trace-fri-error + composition-fri-error +
 ($\sum i < \text{rounds}. C\ i$) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 ($F + X$) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + empty-query-error)
proof –
have drift-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
 adversary-initial-state $\leq F + X$
by (rule
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-bound-from-fixed-and-witnessed-cross-or-merkle-bad
 [OF fixed-empty-alpha-bound cross-bound])
show ?thesis
by (rule
 stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-reduction
 [OF false-statement wf controlled trace-fri-reduction
 composition-fri-reduction round-bound drift-bound])

current-empty-bound])

qed

theorem stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri

fixes F N empty-query-error :: prob

and $C :: \text{nat} \Rightarrow \text{prob}$

assumes false-statement: $\neg$ exists-valid-trace

and wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and trace-fri-reduction:

$\bigwedge$ data attacker-state.

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\implies$

trace-fri-reachable-partial-candidate-reduction-assumption

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

and composition-fri-reduction:

$\bigwedge$ data attacker-state.

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\implies$

composition-fri-reachable-partial-candidate-reduction-assumption

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

and round-bound:

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)

adversary-initial-state $\leq C$ i

and fixed-empty-alpha-bound:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

A)

(checked-staged-security-with-actual-alpha-prefix-verifier-event

composition-alpha-bad-set-hit-with-empty-composition-header-candidates)

adversary-initial-state $\leq F$

and nonunique-bound:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

A)

(checked-staged-security-with-actual-alpha-prefix-verifier-event

empty-header-partial-trace-candidate-nonunique)

adversary-initial-state $\leq N$

and current-empty-bound:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-current-empty-query-opening-hit-at 0)

adversary-initial-state $\leq$ empty-query-error

shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 ($\sum i < \text{rounds}. C\ i$) +
trace-fri-error + *composition-fri-error* +
 ($\sum i < \text{rounds}. C\ i$) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 ($F + N$) +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
trace-fri-error + *empty-query-error*)
proof –
have *drift-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* A)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
adversary-initial-state $\leq F + N$
by (rule
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-bound-from-fixed-and-nonunique
 [*OF* *fixed-empty-alpha-bound* *nonunique-bound*])
show *?thesis*
by (rule
stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri
 [*OF* *false-statement* *wf* *controlled* *trace-fri-reduction*
composition-fri-reduction *round-bound* *drift-bound*
current-empty-bound])
qed

theorem *stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri*
fixes $F\ L\ \text{empty-query-error} :: \text{prob}$
and $C :: \text{nat} \Rightarrow \text{prob}$
assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* A
and *trace-fri-reduction*:
 $\bigwedge \text{data}\ \text{attacker-state}.$
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* A)
adversary-initial-state) $\implies$
trace-fri-reachable-partial-candidate-reduction-assumption
 (*verifier-state-from-adversary* *attacker-state*
 (*staged-proof-transcript* *data*))
and *composition-fri-reduction*:
 $\bigwedge \text{data}\ \text{attacker-state}.$

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
composition-fri-reachable-partial-candidate-reduction-assumption
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*))
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)
(*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* *i*)
adversary-initial-state $\leq C$ *i*
and *fixed-empty-alpha-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-verifier-event*
composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
adversary-initial-state $\leq F$
and *low-degree-nonunique-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-verifier-event*
empty-header-low-degree-trace-candidate-nonunique)
adversary-initial-state $\leq L$
and *current-empty-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-current-empty-query-opening-hit-at* *0*)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability* *A* $\leq$
hash-collision-budget-value *0*
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 $(\sum i < \text{rounds}. C\ i)$ +
trace-fri-error + *composition-fri-error* +
 $(\sum i < \text{rounds}. C\ i)$ +
(*composition-error-bound* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* *0*) +
hash-collision-budget-value *0*
(*staged-alpha-search-queries* *budgets* *0*) +
(*F* + (*trace-fri-error* + *L*)) +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* *0*) +
trace-fri-error + *empty-query-error*)
proof –
let ?*M* =
checked-staged-security-experiment-with-actual-alpha-prefix-data-state *A*

```

let ?T =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  trace-fri-bad-with-empty-composition-header-candidates
let ?L =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  empty-header-low-degree-trace-candidate-nonunique
have trace-fri-data-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state  $\leq$  trace-fri-error
by (rule
  trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
  [OF trace-fri-reduction])
have trace-fri-prefix-bound:
  wp-event ?M ?T adversary-initial-state  $\leq$  trace-fri-error
proof –
  have projected:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state =
    wp-event ?M ?T adversary-initial-state
  by (rule
    checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
  show ?thesis
    using projected trace-fri-data-bound by simp
qed
have nonunique-bound:
  wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
     empty-header-partial-trace-candidate-nonunique)
  adversary-initial-state  $\leq$  trace-fri-error + L
proof (rule order-trans)
  show wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
     empty-header-partial-trace-candidate-nonunique)
  adversary-initial-state  $\leq$ 
  wp-event ?M ?T adversary-initial-state +
  wp-event ?M ?L adversary-initial-state
  by (rule
    empty-header-partial-trace-candidate-nonunique-le-trace-fri-or-low-degree-nonunique)
  show ...  $\leq$  trace-fri-error + L
  by (intro add-mono trace-fri-prefix-bound low-degree-nonunique-bound)
qed
show ?thesis
  by (rule
    stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli
    [OF false-statement wf controlled trace-fri-reduction]

```

*composition-fri-reduction round-bound fixed-empty-alpha-bound
 nonunique-bound current-empty-bound*])
qed

theorem *stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-f*
fixes $F\ L\ \text{empty-query-error} :: \text{prob}$
and $C :: \text{nat} \Rightarrow \text{prob}$
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-reduction:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *composition-fri-reduction:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *round-bound:*
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state A } i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C\ i$
and *fixed-empty-alpha-bound:*
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq F$
and *low-degree-query-agreement-bound:*
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit})$
 $\text{adversary-initial-state} \leq L$
and *current-empty-bound:*
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$

$adversary-initial-state \leq empty-query-error$
shows $checked-staged-adversary-acceptance-probability\ A \leq$
 $hash-collision-budget-value\ 0$
 $(staged-attacker-query-budget\ budgets +$
 $staged-challenge-query-budget + verifier-hash-query-budget) +$
 $(\sum i < rounds. C\ i) +$
 $trace-fri-error + composition-fri-error +$
 $(\sum i < rounds. C\ i) +$
 $(composition-error-bound +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $(F + (trace-fri-error + L)) +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $trace-fri-error + empty-query-error)$
proof –
have $low-degree-nonunique-bound:$
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state\ A)$
 $(checked-staged-security-with-actual-alpha-prefix-verifier-event$
 $empty-header-low-degree-trace-candidate-nonunique)$
 $adversary-initial-state \leq L$
by $(rule\ order-trans$
 $[OF\ empty-header-low-degree-trace-candidate-nonunique-le-query-agreement-hit$
 $low-degree-query-agreement-bound])$
show $?thesis$
by $(rule$
 $stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli$
 $[OF\ false-statement\ wf\ controlled\ trace-fri-reduction$
 $composition-fri-reduction\ round-bound\ fixed-empty-alpha-bound$
 $low-degree-nonunique-bound\ current-empty-bound])$
qed

theorem $stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli$
fixes $X\ N\ empty-query-error :: prob$
and $C :: nat \Rightarrow prob$
assumes $false-statement: \neg\ exists-valid-trace$
and $wf: staged-budget-wellformed\ budgets$
and $controlled: staged-adversary-controlled\ budgets\ A$
and $trace-fri-reduction:$
 $\bigwedge data\ attacker-state.$
 $Some\ (data,\ attacker-state) \in$
 $set-dist\ (execute\ (checked-staged-transcript-program\ A)$
 $adversary-initial-state) \implies$
 $trace-fri-reachable-partial-candidate-reduction-assumption$
 $(verifier-state-from-adversary\ attacker-state$
 $(staged-proof-transcript\ data))$

and *composition-fri-reduction*:
 $\bigwedge \text{data attacker-state.}$
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)

adversary-initial-state) $\implies$
composition-fri-reachable-partial-candidate-reduction-assumption
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*))

and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
(*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* *i*)
adversary-initial-state $\leq C i$

and *not-prefix-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq N$

and *cross-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state $\leq X$

and *current-empty-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-current-empty-query-opening-hit-at* *0*)
adversary-initial-state $\leq \text{empty-query-error}$

shows *checked-staged-adversary-acceptance-probability* *A* $\leq$
hash-collision-budget-value *0*
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 $(\sum i < \text{rounds}. C i) +$
trace-fri-error + *composition-fri-error* +
 $(\sum i < \text{rounds}. C i) +$
(*composition-error-bound* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* *0*) +
hash-collision-budget-value *0*
(*staged-alpha-search-queries* *budgets* *0*) +
((*composition-error-bound* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* *0*) +
hash-collision-budget-value *0*
(*staged-alpha-search-queries* *budgets* *0*) + *N*) + *X*) +
staged-phase-relation-error *size*

$(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{empty-query-error})$
proof –
let $?M =$
 $\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A$
let $?P =$
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0)$
have *transcript-bound*:
 $\text{wp-event } (\text{checked-staged-transcript-with-alpha-prefix-program } A)$
 $(\text{checked-staged-transcript-actual-alpha-prefix-bad-set-hit}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq ?P$
by (rule
 $\text{checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets}$
 $[OF \text{ wf controlled}]$)
have *prefix-bound*:
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq ?P$
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript}$
 $[OF \text{ transcript-bound}]$)
have *fixed-empty-alpha-bound*:
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq ?P + N$
proof (rule *order-trans*)
show $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} +$
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state}$
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-fixed-empty-alpha-hit-le-prefix-or-not-prefix}$
 $[OF \text{ wf controlled}]$)
show $\dots \leq ?P + N$

```

    by (intro add-mono prefix-bound not-prefix-bound)
qed
show ?thesis
by (rule
  stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli
  [OF false-statement wf controlled trace-fri-reduction
    composition-fri-reduction round-bound fixed-empty-alpha-bound
    cross-bound current-empty-bound])
qed

theorem stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli
fixes  $X P R$  empty-query-error :: prob
and  $C :: \text{nat} \Rightarrow \text{prob}$ 
assumes false-statement:  $\neg \text{exists-valid-trace}$ 
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets  $A$ 
and trace-fri-reduction:
   $\bigwedge \text{data attacker-state.}$ 
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program  $A$ )
      adversary-initial-state)  $\implies$ 
    trace-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
and composition-fri-reduction:
   $\bigwedge \text{data attacker-state.}$ 
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program  $A$ )
      adversary-initial-state)  $\implies$ 
    composition-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
and round-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state  $A$   $i$ )
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at  $i$ )
      adversary-initial-state  $\leq C$   $i$ 
and cross-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
         $A$ )
      checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
      adversary-initial-state  $\leq X$ 
and data-pre-bound:
    wp-event (checked-staged-security-experiment-with-data-state  $A$ )
      staged-security-with-data-state-transcript-pre-hit
      adversary-initial-state  $\leq P$ 
and sampled-transcript-new-bound:

```

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-new-hit
adversary-initial-state $\leq R$
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability A $\leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
($\sum i < \text{rounds. } C i$) +
trace-fri-error + composition-fri-error +
($\sum i < \text{rounds. } C i$) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
((composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
X + (P + R) + (P + R))) + X) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error)
proof –
let *?M =*
checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let *?N =*
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
X + (P + R) + (P + R)
have *no-prefix-bound:*
wp-event ?M
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq P + R$
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-bound-from-sampled-transcript
[OF data-pre-bound sampled-transcript-new-bound])
have *path-bound:*
wp-event ?M
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq P + R$
by (rule

checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-bound-from-sampled-transcript
 [OF *data-pre-bound sampled-transcript-new-bound*])
have *collision-bound*:
wp-event ?M
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
adversary-initial-state $\leq$
hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0)
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
 [OF *wf controlled*])
have *uncovered-bound*:
wp-event ?M
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state $\leq X$
by (rule *order-trans*
 [OF
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad
cross-bound])
have *not-prefix-bound*:
wp-event ?M
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq ?N$
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
 [OF *wf controlled collision-bound uncovered-bound no-prefix-bound*
path-bound])
show *?thesis*
by (rule
stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli
 [OF *false-statement wf controlled trace-fri-reduction*
composition-fri-reduction round-bound not-prefix-bound cross-bound
current-empty-bound])
qed

theorem *stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli*
fixes *X P empty-query-error :: prob*
and *C :: nat $\Rightarrow$ prob*
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-reduction:*
 $\bigwedge$ data attacker-state.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))

and *composition-fri-reduction*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *cross-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad}$
 $\text{adversary-initial-state} \leq X$
and *data-pre-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$
and *current-empty-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum i < \text{rounds. } C \ i) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\sum i < \text{rounds. } C \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $((\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $X + (P + \text{staged-concrete-transcript-target-error-bound}) +$
 $(P + \text{staged-concrete-transcript-target-error-bound}))) + X) +$

staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error)
proof –
have *sampled-transcript-new-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-new-hit
adversary-initial-state ≤ staged-concrete-transcript-target-error-bound
by (rule *checked-staged-security-with-data-state-transcript-new-hit-bound*
[OF wf controlled])
show *?thesis*
by (rule
stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli
[OF false-statement wf controlled trace-fri-reduction
composition-fri-reduction round-bound cross-bound data-pre-bound
sampled-transcript-new-bound current-empty-bound])
qed

theorem *stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli*
fixes *S H P empty-query-error :: prob*
and *C :: nat ⇒ prob*
assumes *false-statement: ¬ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-reduction*:
∧ data attacker-state.
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *composition-fri-reduction*:
∧ data attacker-state.
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
composition-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *round-bound*:
∧ i. i < rounds ⇒
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state ≤ C i
and *sparse-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

$A)$
checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad
adversary-initial-state $\leq S$
and *merkle-side-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 $A)$
checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
adversary-initial-state $\leq H$
and *data-pre-bound*:
wp-event (checked-staged-security-experiment-with-data-state $A)$
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state $A)$
(staged-security-with-data-state-current-empty-query-opening-hit-at $0)$
adversary-initial-state $\leq \text{empty-query-error}$
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
(staged-attacker-query-budget *budgets* $+$
staged-challenge-query-budget $+$ *verifier-hash-query-budget*) $+$
 $(\sum_{i < \text{rounds}} C\ i) +$
trace-fri-error $+$ *composition-fri-error* $+$
 $(\sum_{i < \text{rounds}} C\ i) +$
(composition-error-bound $+$
staged-phase-relation-error *size*
(staged-alpha-search-queries *budgets* $0) +$
hash-collision-budget-value 0
(staged-alpha-search-queries *budgets* $0) +$
 $((\text{composition-error-bound} +$
staged-phase-relation-error *size*
(staged-alpha-search-queries *budgets* $0) +$
hash-collision-budget-value 0
(staged-alpha-search-queries *budgets* $0) +$
 $(\text{hash-collision-budget-value}$ 0
(staged-alpha-search-queries *budgets* $0) +$
 $(S + H) + (P + \text{staged-concrete-transcript-target-error-bound}) +$
 $(P + \text{staged-concrete-transcript-target-error-bound}))) + (S + H)) +$
staged-phase-relation-error *size*
(staged-alpha-search-queries *budgets* $0) +$
trace-fri-error $+$ *empty-query-error*)
proof $-$
have *cross-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 $A)$
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state $\leq S + H$
proof (*rule order-trans*)

```

show wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
  checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
  adversary-initial-state ≤
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
  checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad
  adversary-initial-state +
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
  checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
  adversary-initial-state
  by (rule
    checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad-le-sparse-or-merkle-side)
show ... ≤ S + H
  by (intro add-mono sparse-bound merkle-side-bound)
qed
show ?thesis
  by (rule
    stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli
    [OF false-statement wf controlled trace-fri-reduction
      composition-fri-reduction round-bound cross-bound data-pre-bound
      current-empty-bound])
qed

theorem stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli
fixes S L K P empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
   $\bigwedge$  data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
    trace-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
and composition-fri-reduction:
   $\bigwedge$  data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
    composition-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state

```


(staged-proof-transcript data))
and round-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (staged-security-experiment-with-query-prefix-data-state A i)
 (staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq C$ i
and sparse-bound:
 wp-event
 (staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad
 adversary-initial-state $\leq S$
and local-collision-bound:
 wp-event
 (staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad
 adversary-initial-state $\leq L$
and coupling-bound:
 wp-event
 (staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad
 adversary-initial-state $\leq K$
and data-pre-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-transcript-pre-hit
 adversary-initial-state $\leq P$
and current-empty-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq \text{empty-query-error}$
shows checked-staged-adversary-acceptance-probability $A \leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds}. C$ i) +
 trace-fri-error + composition-fri-error +
 ($\sum i < \text{rounds}. C$ i) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 ((composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0

```

    (staged-alpha-search-queries budgets 0) +
    (hash-collision-budget-value 0
     (staged-alpha-search-queries budgets 0) +
     (S + (L + K)) + (P + staged-concrete-transcript-target-error-bound) +
     (P + staged-concrete-transcript-target-error-bound))) +
    (S + (L + K))) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    trace-fri-error + empty-query-error)
proof -
have merkle-side-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
    adversary-initial-state ≤ L + K
proof (rule order-trans)
show wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
   A)
  checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
  adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad
    adversary-initial-state +
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad
    adversary-initial-state
  by (rule
    checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad-le-local-or-coupling)
show ... ≤ L + K
  by (intro add-mono local-collision-bound coupling-bound)
qed
show ?thesis
  by (rule
    stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-split
    [OF false-statement wf controlled trace-fri-reduction
     composition-fri-reduction round-bound sparse-bound
     merkle-side-bound data-pre-bound current-empty-bound])
qed

end

```

These wrappers expose the broad witnessed cross/Merkle diagnostic split used during development of the reachable-FRI route. They remain available only as internal plumb-

ing; the public route should use aligned transcript-indexed wrappers that do not expose these diagnostic events.

hide-fact

soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable

end

theory *Soundness-FRI-Partial-Evidence*

imports *Soundness-Relevant-Drift-Reachable-FRI*

begin

Normalized partial-opening FRI evidence predicates for the reachable-state FRI obligations.

This theory is intentionally narrow. It does not change the public soundness theorem path; it only factors the remaining reachable FRI obligations into explicit evidence predicates that later proofs can target.

context *soundness*

begin

definition *trace-fri-partial-candidate-evidence*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \Rightarrow \text{nat list} \Rightarrow 'f \text{ authenticated-opening list list} \Rightarrow$
 $'f \text{ list} \Rightarrow \text{bool}$

where

trace-fri-partial-candidate-evidence s out fr query-idxs
 $\text{trace-openings trace-table} \longleftrightarrow$
 $\text{accepted-with-partial-trace-openings s out fr query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table}$

definition *composition-fri-partial-candidate-evidence*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow \text{nat list} \Rightarrow$

'f authenticated-opening list list $\Rightarrow$ 'f list $\Rightarrow$
 'f list $\Rightarrow$ bool

where

composition-fri-partial-candidate-evidence s out fr f-fri-roots
 f-final as dg composition-fri-roots final trace-query-idxs
 trace-openings composition-query-idxs composition-openings trace-table
 composition-table $\longleftrightarrow$
 accepted-with-partial-initial-openings s out fr f-fri-roots f-final
 as dg composition-fri-roots final trace-query-idxs trace-openings
 composition-query-idxs composition-openings $\wedge$
 partial-trace-table-candidate trace-table trace-openings $\wedge$
 partial-composition-table-candidate composition-table
 composition-openings $\wedge$
 trace-table-low-degree trace-table $\wedge$
 $\neg$ composition-table-low-degree maxDegree composition-table

lemma trace-fri-bad-with-reachable-partial-candidate-iff-evidence:

trace-fri-bad-with-reachable-partial-candidate s out $\longleftrightarrow$

($\exists$ fr query-idxs trace-openings trace-table.

trace-fri-partial-candidate-evidence s out fr query-idxs
 trace-openings trace-table)

unfolding trace-fri-bad-with-reachable-partial-candidate-def

trace-fri-partial-candidate-evidence-def

by blast

lemma composition-fri-bad-with-reachable-partial-candidate-iff-evidence:

composition-fri-bad-with-reachable-partial-candidate s out $\longleftrightarrow$

($\exists$ fr f-fri-roots f-final as dg composition-fri-roots final
 trace-query-idxs trace-openings composition-query-idxs
 composition-openings trace-table composition-table.

composition-fri-partial-candidate-evidence s out fr f-fri-roots
 f-final as dg composition-fri-roots final trace-query-idxs
 trace-openings composition-query-idxs composition-openings
 trace-table composition-table)

unfolding composition-fri-bad-with-reachable-partial-candidate-def

composition-fri-partial-candidate-evidence-def

by blast

lemma trace-fri-partial-candidate-evidenceI:

assumes

accepted-with-partial-trace-openings s out fr query-idxs
 trace-openings

partial-trace-table-candidate trace-table trace-openings

$\neg$ trace-table-low-degree trace-table

shows

trace-fri-partial-candidate-evidence s out fr query-idxs
 trace-openings trace-table

using assms **unfolding** trace-fri-partial-candidate-evidence-def **by** blast

lemma *trace-fri-partial-candidate-evidenceD*:
assumes
trace-fri-partial-candidate-evidence s out fr query-idxs
trace-openings trace-table
shows
accepted-with-partial-trace-openings s out fr query-idxs
trace-openings
partial-trace-table-candidate trace-table trace-openings
 $\neg$ *trace-table-low-degree trace-table*
using *assms* **unfolding** *trace-fri-partial-candidate-evidence-def* **by** *blast+*

lemma *trace-fri-bad-with-reachable-partial-candidateE*:
assumes *trace-fri-bad-with-reachable-partial-candidate s out*
obtains *fr query-idxs trace-openings trace-table* **where**
trace-fri-partial-candidate-evidence s out fr query-idxs
trace-openings trace-table
using *assms*
unfolding *trace-fri-bad-with-reachable-partial-candidate-iff-evidence*
by *blast*

lemma *composition-fri-partial-candidate-evidenceI*:
assumes
accepted-with-partial-initial-openings s out fr f-fri-roots f-final
as dg composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings
partial-trace-table-candidate trace-table trace-openings
partial-composition-table-candidate composition-table
composition-openings
trace-table-low-degree trace-table
 $\neg$ *composition-table-low-degree maxDegree composition-table*
shows
composition-fri-partial-candidate-evidence s out fr f-fri-roots
f-final as dg composition-fri-roots final trace-query-idxs
trace-openings composition-query-idxs composition-openings
trace-table composition-table
using *assms*
unfolding *composition-fri-partial-candidate-evidence-def* **by** *blast*

lemma *composition-fri-partial-candidate-evidenceD*:
assumes
composition-fri-partial-candidate-evidence s out fr f-fri-roots
f-final as dg composition-fri-roots final trace-query-idxs
trace-openings composition-query-idxs composition-openings trace-table
composition-table
shows
accepted-with-partial-initial-openings s out fr f-fri-roots f-final
as dg composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings
partial-trace-table-candidate trace-table trace-openings

partial-composition-table-candidate composition-table
composition-openings
trace-table-low-degree trace-table
 $\neg$ *composition-table-low-degree maxDegree composition-table*
using *assms*
unfolding *composition-fri-partial-candidate-evidence-def* **by** *blast+*

lemma *composition-fri-bad-with-reachable-partial-candidateE:*
assumes *composition-fri-bad-with-reachable-partial-candidate s out*
obtains *fr f-fri-roots f-final as dg composition-fri-roots final*
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table **where**
composition-fri-partial-candidate-evidence s out fr f-fri-roots
f-final as dg composition-fri-roots final trace-query-idxs
trace-openings composition-query-idxs composition-openings
trace-table composition-table
using *assms*
unfolding *composition-fri-bad-with-reachable-partial-candidate-iff-evidence*
by *blast*

lemma *trace-fri-reachable-partial-candidate-reductionD:*
assumes *trace-fri-reachable-partial-candidate-reduction-assumption s*
shows
wp-event verify-monad
(trace-fri-bad-with-reachable-partial-candidate s) s $\leq$
trace-fri-error
using *assms*
unfolding *trace-fri-reachable-partial-candidate-reduction-assumption-def*
by *simp*

lemma *composition-fri-reachable-partial-candidate-reductionD:*
assumes *composition-fri-reachable-partial-candidate-reduction-assumption s*
shows
wp-event verify-monad
(composition-fri-bad-with-reachable-partial-candidate s) s $\leq$
composition-fri-error
using *assms*
unfolding *composition-fri-reachable-partial-candidate-reduction-assumption-def*
by *simp*

definition *trace-fri-partial-candidate-opening-evidence*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow 'f \text{ list list list} \Rightarrow$
 $'f \Rightarrow \text{nat list} \Rightarrow 'f \text{ authenticated-opening list list} \Rightarrow$
 $'f \text{ list} \Rightarrow \text{bool}$

where

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table $\longleftrightarrow$
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers $\wedge$
trace-fri-partial-candidate-evidence s out fr candidate-query-idxs
trace-openings trace-table

definition *composition-fri-partial-candidate-opening-evidence*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow 'f \text{ list list list} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow 'f \text{ list} \Rightarrow$
 $'f \text{ list} \Rightarrow \text{bool}$

where

composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\longleftrightarrow$
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
fri-dg opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
composition-fri-partial-candidate-evidence s out fr f-fri-roots
f-final as dg composition-fri-roots final trace-query-idxs
trace-openings composition-query-idxs composition-openings
trace-table composition-table

lemma *trace-fri-partial-candidate-opening-evidenceD:*

assumes

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

shows

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers

trace-fri-partial-candidate-evidence s out fr candidate-query-idxs
trace-openings trace-table
using *assms*
unfolding *trace-fri-partial-candidate-opening-evidence-def* **by** *blast+*

lemma *composition-fri-partial-candidate-opening-evidenceD:*
assumes
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
shows
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
composition-fri-partial-candidate-evidence s out fr f-fri-roots
f-final as dg composition-fri-roots final trace-query-idxs
trace-openings composition-query-idxs composition-openings
trace-table composition-table
using *assms*
unfolding *composition-fri-partial-candidate-opening-evidence-def* **by** *blast+*

lemma *verify-monad-trace-fri-bad-extracts-opening-evidence:*
assumes *outcome:*
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *bad:*
trace-fri-bad-with-reachable-partial-candidate s
(Some (result, final-state))
obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
where
trace-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final dg
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr candidate-query-idxs
trace-openings trace-table
proof –
from *verify-monad-accepted-fri-opening-transcript[OF outcome]*
obtain *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers **where** *fri-opening:*
accepted-fri-opening-transcript s (Some (result, final-state))
trace-roots trace-bs trace-final dg composition-roots composition-bs


```

    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  by blast
from trace-fri-bad-with-reachable-partial-candidateE[OF bad]
obtain fr candidate-query-idxs trace-openings trace-table where evidence:
  trace-fri-partial-candidate-evidence s
  (Some (result, final-state)) fr candidate-query-idxs trace-openings
  trace-table
  by blast
have combined:
  trace-fri-partial-candidate-opening-evidence s
  (Some (result, final-state)) trace-roots trace-bs trace-final dg
  composition-roots composition-bs composition-final fri-query-idxs
  trace-round-layers composition-round-layers fr candidate-query-idxs
  trace-openings trace-table
unfolding trace-fri-partial-candidate-opening-evidence-def
using fri-opening evidence by simp
show ?thesis
  by (rule that[OF combined])
qed

lemma verify-monad-composition-fri-bad-extracts-opening-evidence:
assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and bad:
  composition-fri-bad-with-reachable-partial-candidate s
  (Some (result, final-state))
obtains trace-roots trace-bs trace-final fri-dg opening-composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr f-fri-roots f-final as dg
  composition-fri-roots final trace-query-idxs trace-openings
  composition-query-idxs composition-openings trace-table
  composition-table
where
  composition-fri-partial-candidate-opening-evidence s
  (Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
  opening-composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers fr
  f-fri-roots f-final as dg composition-fri-roots final
  trace-query-idxs trace-openings composition-query-idxs
  composition-openings trace-table composition-table
proof –
from verify-monad-accepted-fri-opening-transcript[OF outcome]
obtain trace-roots trace-bs trace-final fri-dg opening-composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers where fri-opening:
  accepted-fri-opening-transcript s (Some (result, final-state))
  trace-roots trace-bs trace-final fri-dg opening-composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers

```

```

    composition-round-layers
  by blast
from composition-fri-bad-with-reachable-partial-candidateE[OF bad]
obtain fr f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table where evidence:
composition-fri-partial-candidate-evidence s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  by blast
have combined:
composition-fri-partial-candidate-opening-evidence s
  (Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table
unfolding composition-fri-partial-candidate-opening-evidence-def
using fri-opening evidence by simp
show ?thesis
  by (rule that[OF combined])
qed

lemma verify-monad-composition-fri-bad-extracts-opening-evidence-with-degree-bound:
assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and bad:
  composition-fri-bad-with-reachable-partial-candidate s
  (Some (result, final-state))
obtains trace-roots trace-bs trace-final fri-dg opening-composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr f-fri-roots f-final as dg
  composition-fri-roots final trace-query-idxs trace-openings
  composition-query-idxs composition-openings trace-table
  composition-table
where
  composition-fri-partial-candidate-opening-evidence s
  (Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
  opening-composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers fr
  f-fri-roots f-final as dg composition-fri-roots final
  trace-query-idxs trace-openings composition-query-idxs
  composition-openings trace-table composition-table
and to-nat fri-dg ≤ maxDegree
proof (rule verify-monad-composition-fri-bad-extracts-opening-evidence
  [OF outcome bad])

```

```

fix trace-roots trace-bs trace-final fri-dg opening-composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr f-fri-roots f-final as dg
  composition-fri-roots final trace-query-idxs trace-openings
  composition-query-idxs composition-openings trace-table composition-table
assume evidence:
  composition-fri-partial-candidate-opening-evidence s
  (Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
  opening-composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers fr
  f-fri-roots f-final as dg composition-fri-roots final
  trace-query-idxs trace-openings composition-query-idxs
  composition-openings trace-table composition-table
have fri-opening:
  accepted-fri-opening-transcript s (Some (result, final-state))
  trace-roots trace-bs trace-final fri-dg opening-composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers
using composition-fri-partial-candidate-opening-evidenceD(1)
  [OF evidence] .
have challenges:
  accepted-fri-challenges s (Some (result, final-state))
  trace-bs fri-dg composition-bs
by (rule accepted-fri-opening-transcript-challenges[OF fri-opening])
have degree-bound: to-nat fri-dg ≤ maxDegree
by (rule verify-monad-accepted-fri-challenges-degree-bound
  [OF outcome challenges])
show ?thesis
by (rule that[OF evidence degree-bound])
qed

```

```

lemma verifier-query-round-after-index-final-checks:
assumes outcome:
  Some (( ), t) ∈
  set-dist
  (execute
    (verifier-query-round-after-index-program
      fr f-fl f-final as fl final raw) s)
obtains idx fv s1 f-i f-len f-pow s2 c-i c-len c-pow s4 where
  idx = index (to-nat raw)
  Some (fv, s1) ∈
  set-dist (execute (mmap (check-decommit-on-query fr idx)) s)
  Some ((f-i, f-final, f-len, f-pow), s2) ∈
  set-dist
  (execute
    (mfold (idx, hd fv, clength * scale, 1)
      (receive-query-commits f-fl)) s1)
  Some ((c-i, final, c-len, c-pow), s4) ∈
  set-dist

```

```

      (execute
        (mfold (idx, cp-eval as fv (h ^ idx * shift),
          clength * scale, 1)
          (receive-query-commits fl)) s2)
    t = s4
proof -
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1) ∈
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
  and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some ((), s3) ∈
      set-dist
        (execute
          (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
          clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some ((), t) ∈
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
obtain f-i f-x f-len f-pow where f-out-eq:
  f-out = (f-i, f-x, f-len, f-pow)
  by (cases f-out) auto
have f-x-eq: f-x = f-final
  using assert-trace unfolding assert-def f-out-eq
  by (cases f-x = f-final) (auto simp: throw-no-outcome)
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def f-out-eq f-x-eq by simp
obtain c-i c-x c-len c-pow where c-out-eq:
  c-out = (c-i, c-x, c-len, c-pow)
  by (cases c-out) auto
have c-x-eq: c-x = final
  using assert-comp unfolding assert-def c-out-eq
  by (cases c-x = final) (auto simp: throw-no-outcome)
have t-eq: t = s4

```

```

using assert-comp unfolding assert-def c-out-eq c-x-eq by simp
have trace-fri':
  Some ((f-i, f-final, f-len, f-pow), s2) ∈
    set-dist
      (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl) s1)
      )
using trace-fri f-out-eq f-x-eq by simp
have comp-fri':
  Some ((c-i, final, c-len, c-pow), s4) ∈
    set-dist
      (execute
        (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
          clength * scale, 1)
          (receive-query-commits fl) s2)
      )
using comp-fri c-out-eq c-x-eq s3-eq by simp
show ?thesis
by (rule that[OF refl query-decommit trace-fri' comp-fri' t-eq])
qed

```

lemma *verifier-query-round-program-final-checks*:

assumes *outcome*:

```

Some (( ), t) ∈
  set-dist
    (execute
      (verifier-query-round-program fr f-fl f-final as fl final) s)

```

obtains *raw idx fv s0 s1 f-i f-len f-pow s2 c-i c-len c-pow s4*

where

```

Some (raw, s0) ∈ set-dist (execute receive-query-index-challenge s)
idx = index (to-nat raw)
Some (fv, s1) ∈
  set-dist (execute (mmap (check-decommit-on-query fr idx)) s0)
Some ((f-i, f-final, f-len, f-pow), s2) ∈
  set-dist
    (execute
      (mfold (idx, hd fv, clength * scale, 1)
        (receive-query-commits f-fl) s1)
      )
Some ((c-i, final, c-len, c-pow), s4) ∈
  set-dist
    (execute
      (mfold (idx, cp-eval as fv (h ^ idx * shift),
        clength * scale, 1)
        (receive-query-commits fl) s2)
      )
  t = s4

```

proof –

from *outcome[unfolded verifier-query-round-program-alt-def]*

obtain *raw s0* **where**

raw:

Some (raw, s0) ∈

```

      set-dist (execute receive-query-index-challenge s)
and after:
  Some (( ), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s0)
  by (auto elim!: set-dist-bindE)
from verifier-query-round-after-index-final-checks[OF after]
obtain idx fv s1 f-i f-len f-pow s2 c-i c-len c-pow s4 where
  idx-eq: idx = index (to-nat raw)
and query-decommit:
  Some (fv, s1) ∈
    set-dist (execute (mmap (check-decommit-on-query fr idx)) s0)
and trace-fri:
  Some ((f-i, f-final, f-len, f-pow), s2) ∈
    set-dist
      (execute
        (mfold (idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
and comp-fri:
  Some ((c-i, final, c-len, c-pow), s4) ∈
    set-dist
      (execute
        (mfold (idx, cp-eval as fv (h ^ idx * shift),
          clength * scale, 1)
          (receive-query-commits fl)) s2)
and t-eq: t = s4
by blast
show ?thesis
by (rule that[OF raw idx-eq query-decommit trace-fri comp-fri t-eq])
qed

end

end

```

```

theory Soundness-FRI-Layer-Evidence
  imports Soundness-FRI-Partial-Evidence
begin

```

Partial FRI layer evidence extracted from the verifier transcript.

This theory only packages facts already exposed by the trace and composition layer-opening lemmas for *accepted-fri-opening-transcript*. It does not reconstruct complete FRI tables from sampled openings.

```

context soundness
begin

```

definition *fri-layer-step-evidence*

$:: 'f \Rightarrow 'f \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow$
 $\text{nat} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $\text{nat} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

fri-layer-step-evidence *rt ch len idx pw sibling-idx*
 $xp \text{ xp-path } xn \text{ xn-path } \text{next-idx } \text{next-value } \text{chunk} \longleftrightarrow$
 $\text{sibling-idx} = \text{fri-sibling-index } \text{len } \text{idx} \wedge$
 $\text{next-idx} = \text{idx} \bmod (\text{len} \text{ div } 2) \wedge$
 $\text{next-value} =$
 $\text{fri-fold-value } \text{ch } \text{xp } \text{xn}$
 $(\text{fri-fold-denominator } ((h \wedge \text{idx}) * \text{shift}) \text{pw}) \wedge$
 $\text{fri-layer-opening-chunk } \text{len } \text{xp } \text{xp-path } \text{xn } \text{xn-path } \text{chunk}$

definition *fri-round-layer-evidence*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat } \text{list} \Rightarrow$
 $\text{nat} \Rightarrow \text{nat} \Rightarrow 'f \text{ list } \text{list } \text{list} \Rightarrow \text{bool}$

where

fri-round-layer-evidence *roots challenges query-idxs round-idx*
 $\text{layer-idx } \text{round-layers} \longleftrightarrow$
 $\text{round-idx} < \text{length } \text{query-idxs} \wedge$
 $\text{layer-idx} < \text{length } \text{roots} \wedge$
 $\text{length } \text{challenges} = \text{length } \text{roots} \wedge$
 $\text{length } \text{round-layers} = \text{length } \text{query-idxs} \wedge$
 $(\exists xp \text{ xp-path } xn \text{ xn-path.}$
 $\text{fri-layer-step-evidence}$
 $(\text{roots} ! \text{layer-idx})$
 $(\text{challenges} ! \text{layer-idx})$
 $(\text{fri-layer-lengths } (\text{length } \text{roots}) (\text{clength} * \text{scale}) ! \text{layer-idx})$
 $(\text{fri-layer-indices } (\text{length } \text{roots}) (\text{query-idxs} ! \text{round-idx})$
 $(\text{clength} * \text{scale}) ! \text{layer-idx})$
 $(2 \wedge \text{layer-idx})$
 $(\text{fri-sibling-index}$
 $(\text{fri-layer-lengths } (\text{length } \text{roots}) (\text{clength} * \text{scale}) ! \text{layer-idx})$
 $(\text{fri-layer-indices } (\text{length } \text{roots}) (\text{query-idxs} ! \text{round-idx})$
 $(\text{clength} * \text{scale}) ! \text{layer-idx}))$
 $xp \text{ xp-path } xn \text{ xn-path}$
 $((\text{fri-layer-indices } (\text{length } \text{roots}) (\text{query-idxs} ! \text{round-idx})$
 $(\text{clength} * \text{scale}) ! \text{layer-idx}) \bmod$
 $((\text{fri-layer-lengths } (\text{length } \text{roots}) (\text{clength} * \text{scale}) ! \text{layer-idx})$
 $\text{div } 2))$
 $(\text{fri-fold-value } (\text{challenges} ! \text{layer-idx}) \text{xp } \text{xn})$
 $(\text{fri-fold-denominator}$
 $((h \wedge$
 $(\text{fri-layer-indices } (\text{length } \text{roots}) (\text{query-idxs} ! \text{round-idx})$
 $(\text{clength} * \text{scale}) ! \text{layer-idx})) * \text{shift})$
 $(2 \wedge \text{layer-idx})))$
 $(\text{round-layers} ! \text{round-idx} ! \text{layer-idx}))$

definition *fri-all-round-layer-evidence*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow \text{bool}$

where

fri-all-round-layer-evidence roots challenges query-idxs round-layers

$\longleftrightarrow$

$\text{length challenges} = \text{length roots} \wedge$

$\text{length round-layers} = \text{length query-idxs} \wedge$

$(\forall \text{ round-idx} < \text{length query-idxs}.$

$\quad \forall \text{ layer-idx} < \text{length roots}.$

$\quad \text{fri-round-layer-evidence roots challenges query-idxs round-idx}$
 $\quad \text{layer-idx round-layers})$

definition *fri-evidence-layer-len* $:: 'f \text{ list} \Rightarrow \text{nat} \Rightarrow \text{nat}$

where

fri-evidence-layer-len roots layer-idx =

fri-layer-lengths (length roots) ($\text{clength} * \text{scale}$) ! *layer-idx*

definition *fri-evidence-layer-idx*

$:: 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat}$

where

fri-evidence-layer-idx roots query-idxs round-idx layer-idx =

fri-layer-indices (length roots) ($\text{query-idxs} ! \text{round-idx}$)

$(\text{clength} * \text{scale}) ! \text{layer-idx}$

definition *fri-evidence-next-idx*

$:: 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat}$

where

fri-evidence-next-idx roots query-idxs round-idx layer-idx =

fri-evidence-layer-idx roots query-idxs round-idx layer-idx mod

$(\text{fri-evidence-layer-len roots layer-idx div } 2)$

definition *fri-evidence-next-value*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow$

$'f \Rightarrow 'f \Rightarrow 'f$

where

fri-evidence-next-value roots challenges query-idxs round-idx layer-idx

xp xn =

fri-fold-value ($\text{challenges} ! \text{layer-idx}$) *xp xn*

$(\text{fri-fold-denominator}$

$\quad ((h \wedge \text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) * \text{shift})$

$\quad (2 \wedge \text{layer-idx}))$

definition *fri-sampled-table-fold*

$:: 'f \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow$

$'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{bool}$

where

fri-sampled-table-fold ch len idx pw fri-dom layer chunk next-value

$\longleftrightarrow$
 $(\exists xp \text{ } xp\text{-path } xn \text{ } xn\text{-path}.$
 $\text{fri-layer-opening-chunk len } xp \text{ } xp\text{-path } xn \text{ } xn\text{-path chunk} \wedge$
 $\text{fri-opening-matches-table len idx layer } xp \text{ } xn \wedge$
 $\text{fri-dom ! idx} = (h \wedge idx * \text{shift}) \wedge pw \wedge$
 $\text{next-value} = \text{fri-table-fold-value ch layer fri-dom len pw idx})$

definition *generic-fri-round-forced-next-value*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow 'f \Rightarrow \text{bool}$

where

generic-fri-round-forced-next-value roots challenges query-idxs
 $\text{round-layers round-idx layer-idx } v \longleftrightarrow$
 $(\exists xp \text{ } xp\text{-path } xn \text{ } xn\text{-path}.$
 $\text{fri-layer-step-evidence}$
 $(\text{roots ! layer-idx})$
 $(\text{challenges ! layer-idx})$
 $(\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx})$
 $(2 \wedge \text{layer-idx})$
 $(\text{fri-sibling-index} (\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}))$
 $xp \text{ } xp\text{-path } xn \text{ } xn\text{-path}$
 $(\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx})$
 v
 $(\text{round-layers ! round-idx ! layer-idx}))$

lemma *fri-layer-step-evidenceI:*

assumes $\text{sibling-idx} = \text{fri-sibling-index len idx}$
and $\text{next-idx} = \text{idx mod } (\text{len div } 2)$
and $\text{next-value} =$
 $\text{fri-fold-value ch } xp \text{ } xn$
 $(\text{fri-fold-denominator } ((h \wedge \text{idx}) * \text{shift}) \text{ } pw)$
and $\text{fri-layer-opening-chunk len } xp \text{ } xp\text{-path } xn \text{ } xn\text{-path chunk}$
shows $\text{fri-layer-step-evidence rt ch len idx pw sibling-idx}$
 $xp \text{ } xp\text{-path } xn \text{ } xn\text{-path next-idx next-value chunk}$
using *assms unfolding fri-layer-step-evidence-def by simp*

lemma *fri-layer-step-evidenceD:*

assumes $\text{fri-layer-step-evidence rt ch len idx pw sibling-idx}$
 $xp \text{ } xp\text{-path } xn \text{ } xn\text{-path next-idx next-value chunk}$
shows $\text{sibling-idx} = \text{fri-sibling-index len idx}$
and $\text{next-idx} = \text{idx mod } (\text{len div } 2)$
and $\text{next-value} =$
 $\text{fri-fold-value ch } xp \text{ } xn$
 $(\text{fri-fold-denominator } ((h \wedge \text{idx}) * \text{shift}) \text{ } pw)$
and $\text{fri-layer-opening-chunk len } xp \text{ } xp\text{-path } xn \text{ } xn\text{-path chunk}$
using *assms unfolding fri-layer-step-evidence-def by simp-all*

```

lemma fri-layer-opening-chunk-values-unique:
  assumes first:
    fri-layer-opening-chunk len xp xp-path xn xn-path chunk
  and second:
    fri-layer-opening-chunk len yp yp-path yn yn-path chunk
  shows xp = yp
  and xn = yn
proof –
  have chunk-eq:
    [xp] @ xp-path @ [xn] @ xn-path =
      [yp] @ yp-path @ [yn] @ yn-path
  using first second unfolding fri-layer-opening-chunk-def by simp
  then show xp = yp
  by simp
  have len-eq: length xp-path = length yp-path
  using first second unfolding fri-layer-opening-chunk-def by simp
  have left:
    ([xp] @ xp-path @ [xn] @ xn-path) ! Suc (length xp-path) = xn
  by simp
  have right:
    ([yp] @ yp-path @ [yn] @ yn-path) ! Suc (length xp-path) = yn
  using len-eq by simp
  show xn = yn
  using chunk-eq left right by metis
qed

```

```

lemma fri-round-layer-evidenceE:
  assumes evidence:
    fri-round-layer-evidence roots challenges query-idxs round-idx
    layer-idx round-layers
  obtains xp xp-path xn xn-path
  where round-idx < length query-idxs
  and layer-idx < length roots
  and length challenges = length roots
  and length round-layers = length query-idxs
  and fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-layer-lengths (length roots) (clength * scale) ! layer-idx)
    (fri-layer-indices (length roots) (query-idxs ! round-idx)
      (clength * scale ! layer-idx))
    (2 ^ layer-idx)
    (fri-sibling-index
      (fri-layer-lengths (length roots) (clength * scale) ! layer-idx)
      (fri-layer-indices (length roots) (query-idxs ! round-idx)
        (clength * scale ! layer-idx)))
  xp xp-path xn xn-path
  ((fri-layer-indices (length roots) (query-idxs ! round-idx)
    (clength * scale ! layer-idx) mod
```

```

      ((fri-layer-lengths (length roots) (clength * scale) ! layer-idx)
        div 2))
    (fri-fold-value (challenges ! layer-idx) xp xn
      (fri-fold-denominator
        ((h ^
          (fri-layer-indices (length roots) (query-idxs ! round-idx)
            (clength * scale) ! layer-idx)) * shift)
          (2 ^ layer-idx))))
    (round-layers ! round-idx ! layer-idx)
  using evidence unfolding fri-round-layer-evidence-def by blast

```

lemma *fri-round-layer-evidence-compactE*:

```

assumes evidence:
  fri-round-layer-evidence roots challenges query-idxs round-idx
  layer-idx round-layers
obtains xp xp-path xn xn-path
where round-idx < length query-idxs
and layer-idx < length roots
and length challenges = length roots
and length round-layers = length query-idxs
and fri-layer-step-evidence
  (roots ! layer-idx)
  (challenges ! layer-idx)
  (fri-evidence-layer-len roots layer-idx)
  (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
  (2 ^ layer-idx)
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
  xp xp-path xn xn-path
  (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  (fri-evidence-next-value roots challenges query-idxs round-idx
    layer-idx xp xn)
  (round-layers ! round-idx ! layer-idx)

```

proof –

```

from fri-round-layer-evidenceE[OF evidence]
obtain xp xp-path xn xn-path where
  round-bound: round-idx < length query-idxs
and layer-bound: layer-idx < length roots
and challenges-len: length challenges = length roots
and layers-len: length round-layers = length query-idxs
and step:
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-layer-lengths (length roots) (clength * scale) ! layer-idx)
    (fri-layer-indices (length roots) (query-idxs ! round-idx)
      (clength * scale) ! layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index

```

```

    (fri-layer-lengths (length roots) (clength * scale) ! layer-idx)
    (fri-layer-indices (length roots) (query-idxs ! round-idx)
      (clength * scale) ! layer-idx))
  xp xp-path xn xn-path
  ((fri-layer-indices (length roots) (query-idxs ! round-idx)
    (clength * scale) ! layer-idx) mod
    ((fri-layer-lengths (length roots) (clength * scale) ! layer-idx)
      div 2))
  (fri-fold-value (challenges ! layer-idx) xp xn
    (fri-fold-denominator
      ((h ^
        (fri-layer-indices (length roots) (query-idxs ! round-idx)
          (clength * scale) ! layer-idx)) * shift)
        (2 ^ layer-idx)))
    (round-layers ! round-idx ! layer-idx))
  by blast
have compact-step:
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
  xp xp-path xn xn-path
  (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  (fri-evidence-next-value roots challenges query-idxs round-idx
    layer-idx xp xn)
  (round-layers ! round-idx ! layer-idx)
  using step
  unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
    fri-evidence-next-idx-def fri-evidence-next-value-def
  by simp
show ?thesis
  by (rule that[OF round-bound layer-bound challenges-len layers-len
    compact-step])
qed

```

lemma *fri-all-round-layer-evidenceD*:

assumes *evidence*:

fri-all-round-layer-evidence roots challenges query-idxs round-layers

and *round-bound*: *round-idx < length query-idxs*

and *layer-bound*: *layer-idx < length roots*

shows *fri-round-layer-evidence roots challenges query-idxs round-idx*

layer-idx round-layers

using *assms* **unfolding** *fri-all-round-layer-evidence-def* **by** *blast*

lemma *fri-layer-step-evidence-from-outcome*:

assumes *outcome*:
Some ((next-idx, next-value, next-len, next-pw), t) ∈
set-dist (execute (fri-layer-opening-step (ch, rt)
(idx, x, len, pw)) s)
obtains *xp xp-path xn xn-path chunk*
where *fri-layer-step-evidence rt ch len idx pw*
(fri-sibling-index len idx) xp xp-path xn xn-path
next-idx next-value chunk
and *chunk = [xp] @ xp-path @ [xn] @ xn-path*
and *x = xp*
and *next-len = len div 2*
and *next-pw = pw + pw*
proof –
from *fri-layer-opening-step-outcome[OF outcome]*
obtain *xp xp-path xn xn-path x' where*
out-eq:
(next-idx, next-value, next-len, next-pw) =
(idx mod (len div 2), x', len div 2, pw + pw)
and *xp-eq: xp = x*
and *x'-eq:*
x' = fri-fold-value ch xp xn
*(fri-fold-denominator ((h ^ idx) * shift) pw)*
and *chunk:*
fri-layer-opening-chunk len xp xp-path xn xn-path
([xp] @ xp-path @ [xn] @ xn-path)
by *blast*
let *?chunk = [xp] @ xp-path @ [xn] @ xn-path*
have *step:*
fri-layer-step-evidence rt ch len idx pw
(fri-sibling-index len idx) xp xp-path xn xn-path next-idx
next-value ?chunk
using *out-eq x'-eq chunk unfolding fri-layer-step-evidence-def by simp*
show *?thesis*
by *(rule that[OF step refl])*
(use out-eq xp-eq in simp-all)
qed

lemma *fri-layer-step-evidence-fold-table-if-opening-matches:*
assumes *evidence:*
fri-layer-step-evidence rt ch len idx pw sibling-idx
xp xp-path xn xn-path next-idx next-value chunk
and *dom-i: fri-dom ! idx = (h ^ idx * shift) ^ pw*
and *match: fri-opening-matches-table len idx layer xp xn*
shows *next-value = fri-table-fold-value ch layer fri-dom len pw idx*
proof –
have *next-value-eq:*
next-value =
fri-fold-value ch xp xn
*(fri-fold-denominator ((h ^ idx) * shift) pw)*

using *fri-layer-step-evidenceD(3)*[*OF evidence*] .
 also have ... = *fri-table-fold-value ch layer fri-dom len pw idx*
 by (*rule fri-fold-value-eq-table-fold-value*[*OF dom-i match*])
 finally show ?thesis .
 qed

lemma *fri-layer-step-evidence-fold-table-if-matching-chunk*:

assumes *evidence*:
 fri-layer-step-evidence rt ch len idx pw sibling-idx
 xp xp-path xn xn-path next-idx next-value chunk
 and *dom-i*: *fri-dom ! idx = (h ^ idx * shift) ^ pw*
 and *chunk-match*:
 fri-layer-opening-chunk len yp yp-path yn yn-path chunk
 and *table-match*: *fri-opening-matches-table len idx layer yp yn*
 shows *next-value = fri-table-fold-value ch layer fri-dom len pw idx*

proof –

have *step-chunk*: *fri-layer-opening-chunk len xp xp-path xn xn-path chunk*
 by (*rule fri-layer-step-evidenceD(4)*[*OF evidence*])
 have *xp-eq*: *xp = yp*
 by (*rule fri-layer-opening-chunk-values-unique(1)*
 [*OF step-chunk chunk-match*])
 have *xn-eq*: *xn = yn*
 by (*rule fri-layer-opening-chunk-values-unique(2)*
 [*OF step-chunk chunk-match*])
 have *table-match'*:
 fri-opening-matches-table len idx layer xp xn
 using *table-match xp-eq xn-eq* by *simp*
 show ?thesis
 by (*rule fri-layer-step-evidence-fold-table-if-opening-matches*
 [*OF evidence dom-i table-match'*])

qed

lemma *fri-sampled-table-foldI*:

assumes
 fri-layer-opening-chunk len xp xp-path xn xn-path chunk
 and *fri-opening-matches-table len idx layer xp xn*
 and *fri-dom ! idx = (h ^ idx * shift) ^ pw*
 and *next-value = fri-table-fold-value ch layer fri-dom len pw idx*
 shows *fri-sampled-table-fold ch len idx pw fri-dom layer chunk*
 next-value
 using *assms unfolding fri-sampled-table-fold-def* by *blast*

lemma *fri-sampled-table-foldE*:

assumes *fri-sampled-table-fold ch len idx pw fri-dom layer chunk*
 next-value
 obtains *xp xp-path xn xn-path*
 where *fri-layer-opening-chunk len xp xp-path xn xn-path chunk*
 and *fri-opening-matches-table len idx layer xp xn*
 and *fri-dom ! idx = (h ^ idx * shift) ^ pw*

and $next_value = fri_table_fold_value\ ch\ layer\ fri_dom\ len\ pw\ idx$
using *assms* **unfolding** *fri-sampled-table-fold-def* **by** *blast*

lemma *fri-layer-step-evidence-sampled-table-fold-if-matching-chunk*:

assumes *evidence*:

fri-layer-step-evidence\ rt\ ch\ len\ idx\ pw\ sibling-idx

xp\ xp-path\ xn\ xn-path\ next-idx\ next-value\ chunk

and *dom-i*: $fri_dom \ !\ idx = (h \wedge idx * shift) \wedge pw$

and *chunk-match*:

fri-layer-opening-chunk\ len\ yp\ yp-path\ yn\ yn-path\ chunk

and *table-match*: *fri-opening-matches-table\ len\ idx\ layer\ yp\ yn*

shows *fri-sampled-table-fold\ ch\ len\ idx\ pw\ fri-dom\ layer\ chunk*
next-value

proof –

have *fold-eq*:

$next_value = fri_table_fold_value\ ch\ layer\ fri_dom\ len\ pw\ idx$

by (rule *fri-layer-step-evidence-fold-table-if-matching-chunk*
 $[OF\ evidence\ dom-i\ chunk-match\ table-match]$)

show *?thesis*

by (rule *fri-sampled-table-foldI*
 $[OF\ chunk-match\ table-match\ dom-i\ fold-eq]$)

qed

lemma *fri-layer-opening-matches-bound-table-compactE*:

assumes *match*:

fri-layer-opening-matches-bound-table\ query-idx\ layer-idx\ tables
layer-chunks

and *tables-len*: $length\ tables = length\ roots$

obtains *xp\ xp-path\ xn\ xn-path*

where

fri-layer-opening-chunk\ (fri-evidence-layer-len\ roots\ layer-idx)

xp\ xp-path\ xn\ xn-path\ (layer-chunks \ !\ layer-idx)

and *fri-opening-matches-table*

(fri-evidence-layer-len\ roots\ layer-idx)

*(fri-layer-indices\ (length\ roots)\ query-idx\ (clength * scale) \ !\ layer-idx)*

(tables \ !\ layer-idx)\ xp\ xn

proof –

let *?len* = *fri-evidence-layer-len\ roots\ layer-idx*

let *?idx* =

*fri-layer-indices\ (length\ roots)\ query-idx\ (clength * scale) \ !\ layer-idx*

from *match* **obtain** *xp\ xp-path\ xn\ xn-path* **where**

chunk:

fri-layer-opening-chunk\ ?len\ xp\ xp-path\ xn\ xn-path

(layer-chunks \ !\ layer-idx)

and *table-match*:

fri-opening-matches-table\ ?len\ ?idx\ (tables \ !\ layer-idx)\ xp\ xn

unfolding *fri-layer-opening-matches-bound-table-def*

by (auto simp: Let-def tables-len fri-evidence-layer-len-def)
 show ?thesis
 by (rule that[OF chunk table-match])
 qed

lemma *fri-all-round-openings-match-tablesD*:
assumes *all*:
 fri-all-round-openings-match-tables *tables* *query-idxs* *round-layers*
 and *round-bound*: *round-idx* < *length query-idxs*
 and *layer-bound*: *layer-idx* < *length tables*
shows *fri-layer-opening-matches-bound-table*
 (*query-idxs* ! *round-idx*) *layer-idx* *tables*
 (*round-layers* ! *round-idx*)
using *assms*
unfolding *fri-all-round-openings-match-tables-def*
 fri-round-openings-match-tables-def
by *blast*

lemma *accepted-fri-opening-transcript-trace-round-layer-evidence*:
assumes *fri-openings*:
 accepted-fri-opening-transcript *s* *out* *trace-roots* *trace-bs* *trace-final*
 dg *composition-roots* *composition-bs* *composition-final* *query-idxs*
 trace-round-layers *composition-round-layers*
 and *round-bound*: *round-idx* < *rounds*
 and *layer-bound*: *layer-idx* < *length trace-roots*
shows *fri-round-layer-evidence* *trace-roots* *trace-bs* *query-idxs*
 round-idx *layer-idx* *trace-round-layers*

proof –
have *shapes*:
 length trace-bs = *length trace-roots*
 length trace-round-layers = *rounds*
 length query-idxs = *rounds*
using *accepted-fri-opening-transcript-shapes*[*OF fri-openings*] **by** *blast*+
from *accepted-fri-opening-transcript-trace-layer-opening*
 [*OF fri-openings round-bound layer-bound*]
obtain *root* *challenge* *len* *idx* *sibling-idx* *xp* *xp-path* *xn* *xn-path* **where**
 root-eq: *root* = *trace-roots* ! *layer-idx*
 and *challenge-eq*: *challenge* = *trace-bs* ! *layer-idx*
 and *len-eq*:
 len =
 fri-layer-lengths (*length trace-roots*) (*clength * scale*) ! *layer-idx*
 and *idx-eq*:
 idx =
 fri-layer-indices (*length trace-roots*) (*query-idxs* ! *round-idx*)
 (*clength * scale*) ! *layer-idx*
 and *sibling-eq*: *sibling-idx* = *fri-sibling-index* *len* *idx*
 and *chunk*:
 fri-layer-opening-chunk *len* *xp* *xp-path* *xn* *xn-path*
 (*trace-round-layers* ! *round-idx* ! *layer-idx*)


```

by blast
have step:
  fri-layer-step-evidence (trace-roots ! layer-idx)
    (trace-bs ! layer-idx)
    (fri-layer-lengths (length trace-roots) (clength * scale) ! layer-idx)
    (fri-layer-indices (length trace-roots) (query-idxs ! round-idx)
      (clength * scale) ! layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index
      (fri-layer-lengths (length trace-roots) (clength * scale) ! layer-idx)
      (fri-layer-indices (length trace-roots) (query-idxs ! round-idx)
        (clength * scale) ! layer-idx))
  xp xp-path xn xn-path
  ((fri-layer-indices (length trace-roots) (query-idxs ! round-idx)
    (clength * scale) ! layer-idx) mod
    ((fri-layer-lengths (length trace-roots) (clength * scale) ! layer-idx)
      div 2))
  (fri-fold-value (trace-bs ! layer-idx) xp xn
    (fri-fold-denominator
      ((h ^
        (fri-layer-indices (length trace-roots) (query-idxs ! round-idx)
          (clength * scale) ! layer-idx)) * shift)
        (2 ^ layer-idx))))
  (trace-round-layers ! round-idx ! layer-idx)
using sibling-eq chunk unfolding fri-layer-step-evidence-def
by (simp add: len-eq idx-eq)
show ?thesis
unfolding fri-round-layer-evidence-def
using round-bound layer-bound shapes step
by auto
qed

lemma accepted-fri-opening-transcript-composition-round-layer-evidence:
assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers
and round-bound: round-idx < rounds
and layer-bound: layer-idx < length composition-roots
shows fri-round-layer-evidence composition-roots composition-bs query-idxs
  round-idx layer-idx composition-round-layers
proof –
have shapes:
  length composition-bs = length composition-roots
  length composition-round-layers = rounds
  length query-idxs = rounds
using accepted-fri-opening-transcript-shapes[OF fri-openings] by blast+
from accepted-fri-opening-transcript-composition-layer-opening
  [OF fri-openings round-bound layer-bound]

```

```

obtain root challenge len idx sibling-idx xp xp-path xn xn-path where
  root-eq: root = composition-roots ! layer-idx
  and challenge-eq: challenge = composition-bs ! layer-idx
  and len-eq:
    len =
      fri-layer-lengths (length composition-roots) (clength * scale) !
        layer-idx
  and idx-eq:
    idx =
      fri-layer-indices (length composition-roots) (query-idxs ! round-idx)
        (clength * scale) ! layer-idx
  and sibling-eq: sibling-idx = fri-sibling-index len idx
  and chunk:
    fri-layer-opening-chunk len xp xp-path xn xn-path
      (composition-round-layers ! round-idx ! layer-idx)
  by blast
have step:
  fri-layer-step-evidence (composition-roots ! layer-idx)
    (composition-bs ! layer-idx)
    (fri-layer-lengths (length composition-roots) (clength * scale) !
      layer-idx)
    (fri-layer-indices (length composition-roots) (query-idxs ! round-idx)
      (clength * scale) ! layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index
      (fri-layer-lengths (length composition-roots) (clength * scale) !
        layer-idx)
      (fri-layer-indices (length composition-roots) (query-idxs ! round-idx)
        (clength * scale) ! layer-idx))
    xp xp-path xn xn-path
    ((fri-layer-indices (length composition-roots) (query-idxs ! round-idx)
      (clength * scale) ! layer-idx) mod
      ((fri-layer-lengths (length composition-roots) (clength * scale) !
        layer-idx) div 2))
    (fri-fold-value (composition-bs ! layer-idx) xp xn
      (fri-fold-denominator
        ((h ^
          (fri-layer-indices (length composition-roots)
            (query-idxs ! round-idx) (clength * scale) ! layer-idx)) *
          shift)
        (2 ^ layer-idx)))
      (composition-round-layers ! round-idx ! layer-idx)
  using sibling-eq chunk unfolding fri-layer-step-evidence-def
  by (simp add: len-eq idx-eq)
show ?thesis
  unfolding fri-round-layer-evidence-def
  using round-bound layer-bound shapes step
  by auto
qed

```

lemma *accepted-fri-opening-transcript-trace-all-round-layer-evidence:*
assumes *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers
shows *fri-all-round-layer-evidence trace-roots trace-bs query-idxs*
trace-round-layers
proof –
have *shapes:*
length trace-bs = length trace-roots
length trace-round-layers = rounds
length query-idxs = rounds
using *accepted-fri-opening-transcript-shapes[OF fri-openings]* **by** *blast+*
show *?thesis*
unfolding *fri-all-round-layer-evidence-def*
using *shapes*
by (*intro conjI allI impI*)
(*auto intro:*
accepted-fri-opening-transcript-trace-round-layer-evidence
[OF fri-openings])
qed

lemma *accepted-fri-opening-transcript-composition-all-round-layer-evidence:*
assumes *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers
shows *fri-all-round-layer-evidence composition-roots composition-bs*
query-idxs composition-round-layers
proof –
have *shapes:*
length composition-bs = length composition-roots
length composition-round-layers = rounds
length query-idxs = rounds
using *accepted-fri-opening-transcript-shapes[OF fri-openings]* **by** *blast+*
show *?thesis*
unfolding *fri-all-round-layer-evidence-def*
using *shapes*
by (*intro conjI allI impI*)
(*auto intro:*
accepted-fri-opening-transcript-composition-round-layer-evidence
[OF fri-openings])
qed

lemma *ntimes-verifier-query-round-program-final-checks-at:*
assumes *outcome:*
Some (results, t) ∈
set-dist

```

      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s)
    and i-bound:  $i < n$ 
obtains prefix suffix s-i s-suc raw idx fv s0 s1
      f-i f-len f-pow s2 c-i c-len c-pow s4
where
  Some (prefix, s-i) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) i)
        s)
  Some ((), s-suc) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s-i)
  Some (suffix, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
            (n - Suc i))
        s-suc)
  results = prefix @ () # suffix
  Some (raw, s0) ∈ set-dist (execute receive-query-index-challenge s-i)
  idx = index (to-nat raw)
  Some (fv, s1) ∈
    set-dist (execute (mmap (check-decommit-on-query fr idx)) s0)
  Some ((f-i, f-final, f-len, f-pow), s2) ∈
    set-dist
      (execute
        (mfold (idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  Some ((c-i, final, c-len, c-pow), s4) ∈
    set-dist
      (execute
        (mfold (idx, cp-eval as fv (h ^ idx * shift),
          clength * scale, 1)
          (receive-query-commits fl)) s2)
  s-suc = s4
proof -
let ?round = verifier-query-round-program fr f-fl f-final as fl final
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈ set-dist (execute (ntimes ?round i) s)
  and round:

```

```

    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
  and suffix:
    Some (suffix, t) ∈
      set-dist (execute (ntimes ?round (n - Suc i)) s-suc)
  and results-eq: results = prefix @ x # suffix
  by blast
have x-eq: x = ()
  by (cases x) simp
have round-unit:
  Some ((), s-suc) ∈ set-dist (execute ?round s-i)
  using round x-eq by simp
show ?thesis
proof (rule verifier-query-round-program-final-checks[OF round-unit])
  fix raw idx fv s0 s1 f-i f-len f-pow s2 c-i c-len c-pow s4
  assume raw:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s-i)
  and idx-eq: idx = index (to-nat raw)
  and query-decommit:
    Some (fv, s1) ∈
      set-dist (execute (mmap (check-decommit-on-query fr idx)) s0)
  and trace-final:
    Some ((f-i, f-final, f-len, f-pow), s2) ∈
      set-dist
        (execute
          (mfold (idx, hd fv, clength * scale, 1)
            (receive-query-commits f-fl)) s1)
  and composition-final:
    Some ((c-i, final, c-len, c-pow), s4) ∈
      set-dist
        (execute
          (mfold (idx, cp-eval as fv (h ^ idx * shift),
            clength * scale, 1)
            (receive-query-commits fl)) s2)
  and s-suc-eq: s-suc = s4
have results-unit: results = prefix @ () # suffix
  using results-eq x-eq by simp
show ?thesis
  by (rule that[OF prefix round-unit suffix results-unit raw idx-eq
    query-decommit trace-final composition-final s-suc-eq])
qed
qed

end

end

```

theory Soundness-FRI-Generic-Low-Degree

imports *Soundness-FRI-Layer-Evidence*
begin

Generic FRI low-degree proof interface.

This theory factors the trace/composition-independent bad-challenge-list layer. It deliberately stays at the generic FRI level: no trace-specific or composition-specific candidate predicate is mentioned here.

context *soundness*
begin

definition *generic-fri-partial-evidence*

$:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow \text{bool}$

where

generic-fri-partial-evidence *low-degree candidate-table degree-bound*
 $\text{roots challenges final-value query-idxs round-layers} \longleftrightarrow$
 $\text{length roots} = \text{fri-round-count-for-degree-bound degree-bound} \wedge$
 $\text{length challenges} = \text{length roots} \wedge$
 $\text{fri-all-round-layer-evidence roots challenges query-idxs round-layers}$

definition *generic-fri-bad-challenge-lists*

$:: \text{nat} \Rightarrow (\text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow 'f \text{ list set}$

where

generic-fri-bad-challenge-lists *n bad* =
fri-multiround-bad-challenge-lists *n bad*

definition *generic-fri-round-bad-challenge-lists*

$:: \text{nat} \Rightarrow (\text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ list set}$

where

generic-fri-round-bad-challenge-lists *n bad i* =
 $\{bs \in \text{fri-challenge-space } n. i < n \wedge bs ! i \in \text{bad } i (\text{take } i \text{ } bs)\}$

definition *generic-fri-round-bad-witnesses*

$:: \text{nat} \Rightarrow (\text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow \text{nat} \Rightarrow$
 $('f \text{ list} \times 'f \times 'f \text{ list}) \text{ set}$

where

generic-fri-round-bad-witnesses *n bad i* =
 $\{(\text{prefix}, b, \text{suffix}).$
 $i < n \wedge$
 $\text{length prefix} = i \wedge$
 $b \in \text{bad } i \text{ prefix} \wedge$
 $\text{length suffix} = n - \text{Suc } i\}$

definition *generic-fri-bad-candidate*

$:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow$

$(\text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow \text{bool}$
where
generic-fri-bad-candidate low-degree candidate-table degree-bound
 $\text{roots challenges final-value query-idxs round-layers bad} \longleftrightarrow$
generic-fri-partial-evidence low-degree candidate-table degree-bound
 $\text{roots challenges final-value query-idxs round-layers} \wedge$
 $\neg \text{low-degree candidate-table} \wedge$
 $\text{challenges} \in \text{generic-fri-bad-challenge-lists } (\text{length challenges}) \text{ bad}$

definition *generic-fri-proximity-evidence*
 $:: \text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat list} \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list} \Rightarrow$
 $(\text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow \text{bool}$
where
generic-fri-proximity-evidence n d lens pws doms layers challenges bad
 $\longleftrightarrow$
 $\text{fri-multiround-proximity-steps } n \text{ d lens pws doms layers bad} \wedge$
 $\text{challenges} \in \text{fri-challenge-space } n \wedge$
 $(\forall i < n.$
 $\text{fri-algebraic-layer-relation } (\text{lens} ! i) (\text{pws} ! i)$
 $(\text{challenges} ! i) (\text{doms} ! i) (\text{layers} ! i)$
 $(\text{doms} ! \text{Suc } i) (\text{layers} ! \text{Suc } i)) \wedge$
 $\neg \text{fri-table-low-degree-on } d (\text{doms} ! 0) (\text{layers} ! 0) \wedge$
 $\text{fri-table-low-degree-on } (\text{fri-degree-after } n \text{ d}) (\text{doms} ! n)$
 $(\text{layers} ! n)$

definition *generic-fri-canonical-layer-chain-evidence*
 $:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list list} \Rightarrow$
 $(\text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow \text{bool}$
where
generic-fri-canonical-layer-chain-evidence low-degree candidate-table
 $\text{degree-bound roots challenges final-value query-idxs round-layers}$
 $\text{doms layers bad} \longleftrightarrow$
generic-fri-partial-evidence low-degree candidate-table degree-bound
 $\text{roots challenges final-value query-idxs round-layers} \wedge$
 $\text{doms} ! 0 = \text{eval-domain} \wedge$
 $\text{layers} ! 0 = \text{candidate-table} \wedge$
 $\text{fri-final-constant-consistent } (\text{layers} ! \text{length challenges}) \text{ final-value} \wedge$
generic-fri-proximity-evidence (length challenges) degree-bound
 $(\text{fri-layer-lengths } (\text{length challenges}) (\text{clength} * \text{scale}))$
 $(\text{map } (\lambda i. 2 \wedge i) [0..<\text{length challenges}])$
 $\text{doms layers challenges bad}$

definition *generic-fri-sampled-layer-chain-evidence*
 $:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers $\longleftrightarrow$
generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers $\wedge$
length doms = *Suc (length challenges)* $\wedge$
length layers = *Suc (length challenges)* $\wedge$
doms ! 0 = *eval-domain* $\wedge$
layers ! 0 = *candidate-table* $\wedge$
fri-final-constant-consistent (layers ! length challenges) final-value $\wedge$
 $(\forall \text{round-idx} < \text{length query-idxs}.$
 $\quad \forall \text{layer-idx} < \text{length challenges}.$
 $\quad \text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} <$
 $\quad \text{length (layers ! Suc layer-idx)}) \wedge$
fri-sampled-table-fold
 $(\text{challenges ! layer-idx})$
 $(\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx})$
 $(2 \wedge \text{layer-idx})$
 $(\text{doms ! layer-idx})$
 $(\text{layers ! layer-idx})$
 $(\text{round-layers ! round-idx ! layer-idx})$
 $(\text{layers ! Suc layer-idx !}$
 $\quad \text{fri-evidence-next-idx roots query-idxs round-idx layer-idx}))$

definition *fri-sampled-domains-align*

$:: 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

fri-sampled-domains-align roots query-idxs doms $\longleftrightarrow$
 $(\forall \text{round-idx} < \text{length query-idxs}.$
 $\quad \forall \text{layer-idx} < \text{length roots}.$
 $\quad \text{doms ! layer-idx !}$
 $\quad \text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx} =$
 $(h \wedge \text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx} *$
 $\quad \text{shift}) \wedge (2 \wedge \text{layer-idx}))$

definition *fri-sampled-next-domains-align*

$:: 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

fri-sampled-next-domains-align roots query-idxs doms $\longleftrightarrow$
 $(\forall \text{round-idx} < \text{length query-idxs}.$
 $\quad \forall \text{layer-idx} < \text{length roots}.$
 $\quad \text{doms ! layer-idx !}$
 $\quad \text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} =$
 $(h \wedge \text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} *$
 $\quad \text{shift}) \wedge (2 \wedge \text{layer-idx}))$

definition *fri-sampled-indices-in-fold-range*


```

:: 'f list ⇒ nat list ⇒ bool
where
  fri-sampled-indices-in-fold-range roots query-idxs ⇔
    (∀ round-idx < length query-idxs.
      ∀ layer-idx < length roots.
        fri-evidence-next-idx roots query-idxs round-idx layer-idx <
          fri-evidence-layer-len roots layer-idx div 2)

definition fri-sampled-layer-covers-fold-indices
  :: 'f list ⇒ nat list ⇒ nat ⇒ bool
where
  fri-sampled-layer-covers-fold-indices roots query-idxs layer-idx
    ⇔
    (∀ idx < fri-evidence-layer-len roots layer-idx div 2.
      ∃ round-idx < length query-idxs.
        fri-evidence-next-idx roots query-idxs round-idx layer-idx = idx)

definition fri-sampled-layers-cover-fold-indices
  :: 'f list ⇒ nat list ⇒ bool
where
  fri-sampled-layers-cover-fold-indices roots query-idxs ⇔
    (∀ layer-idx < length roots.
      fri-sampled-layer-covers-fold-indices roots query-idxs layer-idx)

definition generic-fri-sampled-layer-chain-full-cover
  :: ('f list ⇒ bool) ⇒ 'f list ⇒ nat ⇒
    'f list ⇒ 'f list ⇒ 'f ⇒ nat list ⇒
    'f list list list ⇒ 'f list list ⇒ 'f list list ⇒
    (nat ⇒ 'f list ⇒ 'f set) ⇒ bool
where
  generic-fri-sampled-layer-chain-full-cover low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers bad ⇔
  generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers ∧
  fri-multiround-proximity-steps (length challenges) degree-bound
    (fri-layer-lengths (length challenges) (clength * scale))
    (map (λi. 2 ^ i) [0..<length challenges])
    doms layers bad ∧
  fri-sampled-layers-cover-fold-indices roots query-idxs ∧
    (∀ layer-idx < length challenges.
      2 dvd fri-evidence-layer-len roots layer-idx) ∧
    (∀ layer-idx < length challenges.
      fri-evidence-layer-len roots layer-idx * 2 ^ layer-idx =
        clength * scale) ∧
    (∀ layer-idx < length challenges.
      fri-evidence-layer-len roots layer-idx ≤
        length (doms ! layer-idx)) ∧

```

$(\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} \leq$
 $\text{length (layers ! layer-idx)}) \wedge$
 $(\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx div 2} \leq$
 $\text{length (doms ! Suc layer-idx)}) \wedge$
 $(\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx div 2} \leq$
 $\text{length (layers ! Suc layer-idx)}) \wedge$
 $(\forall \text{layer-idx} < \text{length challenges.}$
 $\forall \text{idx} < \text{fri-evidence-layer-len roots layer-idx div 2.}$
 $\text{doms ! Suc layer-idx ! idx} =$
 $(\text{doms ! layer-idx ! idx})^2) \wedge$
 $(\forall \text{layer-idx} < \text{length challenges.}$
 $\forall \text{idx} < \text{fri-evidence-layer-len roots layer-idx div 2.}$
 $\text{doms ! layer-idx ! idx} = (\text{h} \wedge \text{idx} * \text{shift}) \wedge (2 \wedge \text{layer-idx})) \wedge$
 $\neg \text{fri-table-low-degree-on degree-bound eval-domain candidate-table} \wedge$
 $\text{fri-table-low-degree-on}$
 $(\text{fri-degree-after (length challenges) degree-bound})$
 $(\text{doms ! length challenges}) (\text{layers ! length challenges})$

lemma *generic-fri-partial-evidence-shapes:*

assumes *generic-fri-partial-evidence low-degree candidate-table degree-bound*
 $\text{roots challenges final-value query-idxs round-layers}$
shows $\text{length roots} = \text{fri-round-count-for-degree-bound degree-bound}$
and $\text{length challenges} = \text{length roots}$
and *fri-all-round-layer-evidence roots challenges query-idxs round-layers*
using *assms unfolding generic-fri-partial-evidence-def by simp-all*

lemma *generic-fri-sampled-layer-chain-evidenceD:*

assumes
 $\text{generic-fri-sampled-layer-chain-evidence low-degree candidate-table}$
 $\text{degree-bound roots challenges final-value query-idxs round-layers}$
 doms layers
shows
 $\text{generic-fri-partial-evidence low-degree candidate-table degree-bound}$
 $\text{roots challenges final-value query-idxs round-layers}$
 $\text{length doms} = \text{Suc (length challenges)}$
 $\text{length layers} = \text{Suc (length challenges)}$
 $\text{doms ! 0} = \text{eval-domain}$
 $\text{layers ! 0} = \text{candidate-table}$
 $\text{fri-final-constant-consistent (layers ! length challenges) final-value}$
using *assms*
unfolding *generic-fri-sampled-layer-chain-evidence-def by simp-all*

lemma *generic-fri-sampled-layer-chain-evidence-sample:*

assumes *chain:*
 $\text{generic-fri-sampled-layer-chain-evidence low-degree candidate-table}$
 $\text{degree-bound roots challenges final-value query-idxs round-layers}$

doms layers
and *round-bound: round-idx < length query-idxs*
and *layer-bound: layer-idx < length challenges*
shows
fri-evidence-next-idx roots query-idxs round-idx layer-idx <
length (layers ! Suc layer-idx)
fri-sampled-table-fold
(challenges ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
(2 ^ layer-idx)
(doms ! layer-idx)
(layers ! layer-idx)
(round-layers ! round-idx ! layer-idx)
(layers ! Suc layer-idx !
fri-evidence-next-idx roots query-idxs round-idx layer-idx)
using *chain round-bound layer-bound*
unfolding *generic-fri-sampled-layer-chain-evidence-def* **by** *simp-all*

lemma *fri-sampled-domains-alignD:*
assumes *fri-sampled-domains-align roots query-idxs doms*
and *round-idx < length query-idxs*
and *layer-idx < length roots*
shows
doms ! layer-idx !
fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
*(h ^ fri-evidence-layer-idx roots query-idxs round-idx layer-idx **
shift) ^ (2 ^ layer-idx)
using *assms* **unfolding** *fri-sampled-domains-align-def* **by** *blast*

lemma *fri-sampled-next-domains-alignD:*
assumes *fri-sampled-next-domains-align roots query-idxs doms*
and *round-idx < length query-idxs*
and *layer-idx < length roots*
shows
doms ! layer-idx !
fri-evidence-next-idx roots query-idxs round-idx layer-idx =
*(h ^ fri-evidence-next-idx roots query-idxs round-idx layer-idx **
shift) ^ (2 ^ layer-idx)
using *assms* **unfolding** *fri-sampled-next-domains-align-def* **by** *blast*

lemma *fri-sampled-indices-in-fold-rangeD:*
assumes *fri-sampled-indices-in-fold-range roots query-idxs*
and *round-idx < length query-idxs*
and *layer-idx < length roots*
shows
fri-evidence-next-idx roots query-idxs round-idx layer-idx <
fri-evidence-layer-len roots layer-idx div 2
using *assms* **unfolding** *fri-sampled-indices-in-fold-range-def* **by** *blast*

lemma *fri-sampled-layers-cover-fold-indicesD*:
assumes *fri-sampled-layers-cover-fold-indices roots query-idxs*
and *layer-idx < length roots*
and *idx < fri-evidence-layer-len roots layer-idx div 2*
obtains *round-idx*
where *round-idx < length query-idxs*
and *fri-evidence-next-idx roots query-idxs round-idx layer-idx = idx*
using *assms*
unfolding *fri-sampled-layers-cover-fold-indices-def*
fri-sampled-layer-covers-fold-indices-def
by *blast*

lemma *fri-sampled-layer-covers-fold-indices-card-le-rounds*:
assumes *cover: fri-sampled-layer-covers-fold-indices roots query-idxs layer-idx*
shows *fri-evidence-layer-len roots layer-idx div 2 ≤ length query-idxs*
proof –
let *?N = fri-evidence-layer-len roots layer-idx div 2*
have *subset*:
 $\{..*?N\} \subseteq*$
 $(\lambda \text{round-idx}.$
 $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx})$ ‘
 $\{..
proof
fix *idx*
assume *idx ∈ {..*?N\}*
then have *idx-bound: idx < ?N*
by *simp*
then obtain *round-idx* **where** *round-bound: round-idx < length query-idxs*
and *idx-eq*:
 $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} = \text{idx}$
using *cover*
unfolding *fri-sampled-layer-covers-fold-indices-def* **by** *blast*
then show *idx ∈*
 $(\lambda \text{round-idx}.$
 $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx})$ ‘
 $\{..
by *blast*
qed
have *?N = card {..*?N\}*
by *simp*
also have $\dots \leq$
 $\text{card } ((\lambda \text{round-idx}.$
 $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx})$ ‘
 $\{..
by *(rule card-mono) (simp-all add: subset)*
also have $\dots \leq \text{card } \{..
by *(rule card-image-le) simp*
also have $\dots = \text{length query-idxs}$$$*$*$

by *simp*
 finally show ?thesis .
 qed

lemma *fri-sampled-layers-cover-fold-indices-card-le-rounds*:
 assumes *cover*: *fri-sampled-layers-cover-fold-indices roots query-idxs*
 and *layer-bound*: *layer-idx < length roots*
 shows *fri-evidence-layer-len roots layer-idx div 2 ≤ length query-idxs*
proof –
 have *layer-cover*:
 fri-sampled-layer-covers-fold-indices roots query-idxs layer-idx
 using *cover layer-bound*
 unfolding *fri-sampled-layers-cover-fold-indices-def* by *blast*
 show ?thesis
 by (rule *fri-sampled-layer-covers-fold-indices-card-le-rounds*
 [*OF layer-cover*])
 qed

lemma *fri-sampled-layers-not-cover-if-layer-exceeds-rounds*:
 assumes *layer-bound*: *layer-idx < length roots*
 and *too-few*:
 length query-idxs < fri-evidence-layer-len roots layer-idx div 2
 shows $\neg$ *fri-sampled-layers-cover-fold-indices roots query-idxs*
proof
 assume *cover*: *fri-sampled-layers-cover-fold-indices roots query-idxs*
 have *fri-evidence-layer-len roots layer-idx div 2 ≤ length query-idxs*
 by (rule *fri-sampled-layers-cover-fold-indices-card-le-rounds*
 [*OF cover layer-bound*])
 then show *False*
 using *too-few* by *simp*
 qed

lemma *generic-fri-proximity-evidenceD*:
 assumes *generic-fri-proximity-evidence n d lens pws doms layers*
 challenges bad
 shows
 fri-multiround-proximity-steps n d lens pws doms layers bad
 challenges ∈ fri-challenge-space n
 $\neg$ *fri-table-low-degree-on d (doms ! 0) (layers ! 0)*
 fri-table-low-degree-on (fri-degree-after n d) (doms ! n)
 (layers ! n)
 using *assms* unfolding *generic-fri-proximity-evidence-def* by *simp-all*

lemma *generic-fri-proximity-evidence-relation*:
 assumes *proximity*:
 generic-fri-proximity-evidence n d lens pws doms layers
 challenges bad
 and *i-bound*: *i < n*
 shows

```

    fri-algebraic-layer-relation (lens ! i) (pws ! i)
      (challenges ! i) (doms ! i) (layers ! i)
      (doms ! Suc i) (layers ! Suc i)
  using proximity i-bound
  unfolding generic-fri-proximity-evidence-def by blast

lemma generic-fri-canonical-layer-chain-proximity:
  assumes chain:
    generic-fri-canonical-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers bad
  shows
    generic-fri-proximity-evidence (length challenges) degree-bound
      (fri-layer-lengths (length challenges) (clength * scale))
      (map (λi. 2 ^ i) [0..<length challenges])
      doms layers challenges bad
  using chain unfolding generic-fri-canonical-layer-chain-evidence-def
  by simp

lemma generic-fri-canonical-layer-chain-relation:
  assumes chain:
    generic-fri-canonical-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers bad
  and i-bound: i < length challenges
  shows
    fri-algebraic-layer-relation
      (fri-layer-lengths (length challenges) (clength * scale) ! i)
      (2 ^ i)
      (challenges ! i) (doms ! i) (layers ! i)
      (doms ! Suc i) (layers ! Suc i)
  proof -
    have proximity:
      generic-fri-proximity-evidence (length challenges) degree-bound
        (fri-layer-lengths (length challenges) (clength * scale))
        (map (λi. 2 ^ i) [0..<length challenges])
        doms layers challenges bad
    by (rule generic-fri-canonical-layer-chain-proximity[OF chain])
    have rel:
      fri-algebraic-layer-relation
        (fri-layer-lengths (length challenges) (clength * scale) ! i)
        ((map (λi. 2 ^ i) [0..<length challenges]) ! i)
        (challenges ! i) (doms ! i) (layers ! i)
        (doms ! Suc i) (layers ! Suc i)
    by (rule generic-fri-proximity-evidence-relation
        [OF proximity i-bound])
    then show ?thesis
      using i-bound by simp
  qed

```

lemma *generic-fri-canonical-layer-chain-lengths*:
assumes *chain*:
generic-fri-canonical-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad
shows *length doms = Suc (length challenges)*
and *length layers = Suc (length challenges)*
proof –
have *proximity*:
generic-fri-proximity-evidence (length challenges) degree-bound
*(fri-layer-lengths (length challenges) (clength * scale))*
*(map ($\lambda i. 2^i$) [0..*length challenges*])*
doms layers challenges bad
by (rule *generic-fri-canonical-layer-chain-proximity*[*OF chain*])
have *steps*:
fri-multiround-proximity-steps (length challenges) degree-bound
*(fri-layer-lengths (length challenges) (clength * scale))*
*(map ($\lambda i. 2^i$) [0..*length challenges*])*
doms layers bad
by (rule *generic-fri-proximity-evidenceD*(1)[*OF proximity*])
then show *length doms = Suc (length challenges)*
and *length layers = Suc (length challenges)*
unfolding *fri-multiround-proximity-steps-def* **by** *simp-all*
qed

lemma *fri-algebraic-layer-relation-next-index-bound*:
assumes *relation*:
fri-algebraic-layer-relation len pw ch fri-dom layer next-dom
next-layer
and *idx-bound: idx < len div 2*
shows *idx < length next-layer*
proof –
have *len div 2 ≤ length next-layer*
using *relation* **unfolding** *fri-algebraic-layer-relation-def* **by** *simp*
then show *?thesis*
using *idx-bound* **by** *linarith*
qed

lemma *fri-algebraic-layer-relation-sampled-fold-eq*:
assumes *relation*:
fri-algebraic-layer-relation len pw ch fri-dom layer next-dom
next-layer
and *idx-bound: idx < len div 2*
shows *next-layer ! idx =*
fri-table-fold-value ch layer fri-dom len pw idx
by (rule *fri-algebraic-layer-relationD*(2)[*OF relation idx-bound*])

lemma *fri-half-dvd-len-if-even*:

```

fixes len :: nat
assumes even-len: 2 dvd len
shows len div 2 dvd len
proof (cases len = 0)
  case True
    then show ?thesis by simp
next
  case False
    then have len = 2 * (len div 2)
      using even-len by simp
    then have len = (len div 2) * 2
      by simp
    then show ?thesis
      unfolding dvd-def by blast
qed

lemma fri-sibling-index-mod-half:
  fixes len idx :: nat
  assumes even-len: 2 dvd len
  shows fri-sibling-index len idx mod (len div 2) =
    idx mod (len div 2)
proof -
  have dvd-half: len div 2 dvd len
    by (rule fri-half-dvd-len-if-even[OF even-len])
  have fri-sibling-index len idx mod (len div 2) =
    ((idx + len div 2) mod len) mod (len div 2)
    unfolding fri-sibling-index-def by simp
  also have ... = (idx + len div 2) mod (len div 2)
    using mod-mod-cancel[OF dvd-half, of idx + len div 2] by simp
  also have ... = idx mod (len div 2)
    by simp
  finally show ?thesis .
qed

lemma fri-fold-value-swap-neg-denominator:
  fri-fold-value b xp xn (- denom) =
    fri-fold-value b xn xp denom
proof (cases denom = 0)
  case True
    then show ?thesis
      unfolding fri-fold-value-def by (simp add: algebra-simps)
next
  case False
    then show ?thesis
      unfolding fri-fold-value-def by (simp add: field-simps)
qed

lemma soundness-h-power-mod-eval-domain:
  h ^ (k mod (clength * scale)) = h ^ k

```



```

proof -
  let ?N = clength * scale
  have order:  $h \wedge ?N = 1$ 
    using h-order .
  have k-eq:  $k = (k \text{ div } ?N) * ?N + k \text{ mod } ?N$ 
    using div-mult-mod-eq[of k ?N] by (simp add: mult.commute)
  have period:  $h \wedge ((k \text{ div } ?N) * ?N) = 1$ 
proof -
  have  $h \wedge ((k \text{ div } ?N) * ?N) = h \wedge (?N * (k \text{ div } ?N))$ 
    by (simp add: mult.commute)
  also have ... =  $(h \wedge ?N) \wedge (k \text{ div } ?N)$ 
    by (simp only: power-mult)
  also have ... = 1
    using order by simp
  finally show ?thesis .
qed
have  $h \wedge k = h \wedge ((k \text{ div } ?N) * ?N + k \text{ mod } ?N)$ 
  using k-eq by simp
also have ... =  $h \wedge ((k \text{ div } ?N) * ?N) * h \wedge (k \text{ mod } ?N)$ 
  by (simp add: power-add)
also have ... =  $h \wedge (k \text{ mod } ?N)$ 
  using period by simp
finally show ?thesis
  by simp
qed

```

```

lemma soundness-h-half-shift:
   $h \wedge (k + (\text{clength} * \text{scale}) \text{ div } 2) = \neg(h \wedge k)$ 
  using h-half-order
  by (simp add: power-add algebra-simps)

```

```

lemma soundness-mod-mult-right-factor-nat:
  fixes a b c :: nat
  assumes b-pos:  $0 < b$ 
    and c-pos:  $0 < c$ 
  shows  $((a \text{ mod } b) * c) \text{ mod } (b * c) = (a * c) \text{ mod } (b * c)$ 
proof -
  have decomp:  $a = a \text{ div } b * b + a \text{ mod } b$ 
    using div-mult-mod-eq[of a b] by simp
  have r-less:  $(a \text{ mod } b) * c < b * c$ 
    using b-pos c-pos mod-less-divisor[OF b-pos]
    by (simp add: mult-less-mono2)
  have r-le:  $(a \text{ mod } b) * c \leq a * c$ 
proof -
  have  $a \text{ mod } b \leq a$ 
    using decomp by linarith
  then show ?thesis
    by simp
qed

```

```

have dvd-diff:  $b * c \text{ dvd } a * c - (a \text{ mod } b) * c$ 
proof -
  have ac-decomp:  $a * c = (a \text{ div } b) * (b * c) + (a \text{ mod } b) * c$ 
  proof -
    have  $a * c = (a \text{ div } b * b + a \text{ mod } b) * c$ 
    using decomp by simp
    also have  $\dots = (a \text{ div } b * b) * c + (a \text{ mod } b) * c$ 
    by (simp only: distrib-right)
    also have  $\dots = (a \text{ div } b) * (b * c) + (a \text{ mod } b) * c$ 
    by (simp add: algebra-simps)
    finally show ?thesis .
  qed
  have  $a * c - (a \text{ mod } b) * c = (a \text{ div } b) * (b * c)$ 
  using ac-decomp by simp
  then show ?thesis by simp
qed

have  $(a * c) \text{ mod } (b * c) = (a \text{ mod } b) * c$ 
  by (rule mod-nat-eqI[OF r-less r-le dvd-diff])
then show ?thesis by simp
qed

lemma fri-layer-lengths-nth-div:
  assumes j-bound:  $j < n$ 
  shows fri-layer-lengths  $n \text{ len } ! j = \text{len div } 2 ^ j$ 
  using j-bound
proof (induction n arbitrary: len j)
  case 0
  then show ?case by simp
next
  case (Suc n)
  then show ?case
proof (cases j)
  case 0
  then show ?thesis by simp
next
  case (Suc k)
  then have k-bound:  $k < n$ 
  using Suc.prem by simp
  have fri-layer-lengths (Suc n)  $\text{len } ! j =$ 
    fri-layer-lengths  $n (\text{len div } 2) ! k$ 
  using Suc by simp
  also have  $\dots = (\text{len div } 2) \text{ div } 2 ^ k$ 
  by (rule Suc.IH[OF k-bound])
  also have  $\dots = \text{len div } 2 ^ j$ 
  unfolding Suc by (simp add: div-mult2-eq mult.commute)
  finally show ?thesis .
qed
qed

```

```

lemma fri-layer-lengths-round-product-if-dvd:
  assumes j-bound:  $j < n$ 
    and dvd:  $2^j \text{ dvd } \text{len}$ 
  shows fri-layer-lengths  $n \text{ len} ! j * 2^j = \text{len}$ 
proof -
  have fri-layer-lengths  $n \text{ len} ! j = \text{len div } 2^j$ 
    by (rule fri-layer-lengths-nth-div[OF j-bound])
  then show ?thesis
    using dvd by (simp add: dvd-div-mult-self)
qed

lemma fri-layer-lengths-even-if-dvd:
  assumes j-bound:  $j < n$ 
    and dvd:  $2^{\text{Suc } j} \text{ dvd } \text{len}$ 
  shows  $2 \text{ dvd } \text{fri-layer-lengths } n \text{ len} ! j$ 
proof -
  obtain k where len-eq:  $\text{len} = 2^{\text{Suc } j} * k$ 
    using dvd by blast
  have fri-layer-lengths  $n \text{ len} ! j = \text{len div } 2^j$ 
    by (rule fri-layer-lengths-nth-div[OF j-bound])
  also have  $\dots = 2 * k$ 
    using len-eq by simp
  finally show ?thesis by simp
qed

lemma fri-layer-lengths-Suc-nth:
  assumes Suc j  $< n$ 
  shows fri-layer-lengths  $n \text{ len} ! \text{Suc } j =$ 
    fri-layer-lengths  $n \text{ len} ! j \text{ div } 2$ 
proof -
  have j-bound:  $j < n$ 
    using assms by simp
  have fri-layer-lengths  $n \text{ len} ! \text{Suc } j = \text{len div } 2^{\text{Suc } j}$ 
    by (rule fri-layer-lengths-nth-div[OF assms])
  also have  $\dots = (\text{len div } 2^j) \text{ div } 2$ 
    by (metis div-mult2-eq mult commute power-Suc)
  also have  $\dots = \text{fri-layer-lengths } n \text{ len} ! j \text{ div } 2$ 
    using fri-layer-lengths-nth-div[OF j-bound, of len] by simp
  finally show ?thesis .
qed

lemma generic-fri-canonical-domain-power:
  assumes chain:
    generic-fri-canonical-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers bad
  and roots-len:  $\text{length } \text{challenges} = \text{length } \text{roots}$ 
  and layer-bound:  $\text{layer-idx} < \text{length } \text{challenges}$ 
  and idx-bound:  $\text{idx} < \text{fri-evidence-layer-len } \text{roots } \text{layer-idx}$ 

```

```

shows doms ! layer-idx ! idx = (h ^ idx * shift) ^ (2 ^ layer-idx)
using layer-bound idx-bound
proof (induction layer-idx arbitrary: idx)
  case 0
  note zero = 0
  have dom0: doms ! 0 = eval-domain
    using chain unfolding generic-fri-canonical-layer-chain-evidence-def
    by simp
  have len0: fri-evidence-layer-len roots 0 = clength * scale
  proof (cases roots)
    case Nil
    then show ?thesis
      using zero roots-len unfolding fri-evidence-layer-len-def by simp
  next
  case (Cons r rs)
  then show ?thesis
    unfolding fri-evidence-layer-len-def by simp
  qed
show ?case
  using eval-domain-nth[of idx] zero len0
  unfolding dom0 by simp
next
  case (Suc j)
  have j-bound: j < length challenges
    using Suc.premis by simp
  have len-eq-j:
    fri-layer-lengths (length challenges) (clength * scale) ! j =
      fri-evidence-layer-len roots j
    using roots-len unfolding fri-evidence-layer-len-def by simp
  have len-eq-suc:
    fri-layer-lengths (length challenges) (clength * scale) ! Suc j =
      fri-evidence-layer-len roots (Suc j)
    using roots-len unfolding fri-evidence-layer-len-def by simp
  have suc-len:
    fri-evidence-layer-len roots (Suc j) =
      fri-evidence-layer-len roots j div 2
    using fri-layer-lengths-Suc-nth[OF Suc.premis(1), of clength * scale]
    len-eq-j len-eq-suc by simp
  have idx-half: idx < fri-evidence-layer-len roots j div 2
    using Suc.premis(2) suc-len by simp
  have idx-prev: idx < fri-evidence-layer-len roots j
    using idx-half by simp
  have rel:
    fri-algebraic-layer-relation
      (fri-evidence-layer-len roots j)
      (2 ^ j)
      (challenges ! j) (doms ! j) (layers ! j)
      (doms ! Suc j) (layers ! Suc j)
    using generic-fri-canonical-layer-chain-relation[OF chain j-bound]

```

```

    len-eq-j by simp
have next-dom:
  doms ! Suc j ! idx = (doms ! j ! idx)2
  by (rule fri-algebraic-layer-relationD(1)[OF rel idx-half])
have prev-dom:
  doms ! j ! idx = (h ^ idx * shift) ^ (2 ^ j)
  by (rule Suc.IH[OF j-bound idx-prev])
let ?x = h ^ idx * shift
have pow-step: (?x ^ (2 ^ j))2 = ?x ^ (2 ^ Suc j)
proof -
  have (?x ^ (2 ^ j))2 = ?x ^ ((2 ^ j) * 2)
    by (simp add: power-mult)
  also have ... = ?x ^ (2 ^ Suc j)
    by (simp add: mult.commute)
  finally show ?thesis .
qed
show ?case
  using next-dom prev-dom pow-step by simp
qed

lemma fri-sibling-domain-round:
assumes len-pos: 0 < len
  and even-len: 2 dvd len
  and round: len * pw = clength * scale
shows
  ((h ^ ((i + len div 2) mod len)) * shift) ^ pw =
  - (((h ^ i) * shift) ^ pw)
proof -
let ?N = clength * scale
have pw-pos: 0 < pw
  using len-pos round eval-domain-size-pos by (cases pw) simp-all
have half-round: (len div 2) * pw = ?N div 2
proof -
  from even-len obtain k where len-eq: len = 2 * k
  by blast
  then have ?N = 2 * (k * pw)
  using round by simp
  then show ?thesis
  using len-eq by simp
qed
have exp-mod:
  (((i + len div 2) mod len) * pw) mod ?N =
  (i * pw + ?N div 2) mod ?N
proof -
  have (((i + len div 2) mod len) * pw) mod ?N =
  ((i + len div 2) * pw) mod ?N
  using soundness-mod-mult-right-factor-nat
  [OF len-pos pw-pos, of i + len div 2] round by simp
  also have ... = (i * pw + ?N div 2) mod ?N

```

```

    using half-round by (simp add: algebra-simps)
  finally show ?thesis .
qed
have h-exp:
  
$$h^{\wedge}(((i + \text{len div } 2) \bmod \text{len}) * pw) = -(h^{\wedge}(i * pw))$$

proof -
  have 
$$h^{\wedge}(((i + \text{len div } 2) \bmod \text{len}) * pw) =$$

    
$$h^{\wedge}((((i + \text{len div } 2) \bmod \text{len}) * pw) \bmod ?N)$$

    using soundness-h-power-mod-eval-domain
    [of  $((i + \text{len div } 2) \bmod \text{len}) * pw$ ]
    by simp
  also have ... =  $h^{\wedge}((i * pw + ?N \text{ div } 2) \bmod ?N)$ 
    using exp-mod by simp
  also have ... =  $h^{\wedge}(i * pw + ?N \text{ div } 2)$ 
    using soundness-h-power-mod-eval-domain[of  $i * pw + ?N \text{ div } 2$ ]
    by simp
  also have ... =  $-(h^{\wedge}(i * pw))$ 
    by (rule soundness-h-half-shift)
  finally show ?thesis .
qed
show ?thesis
  using h-exp by (simp add: power-mult power-mult-distrib)
qed

lemma fri-sibling-fold-denominator:
  assumes len-pos:  $0 < \text{len}$ 
    and even-len:  $2 \text{ dvd } \text{len}$ 
    and round:  $\text{len} * pw = \text{clength} * \text{scale}$ 
  shows
    
$$\text{fri-fold-denominator} ((h^{\wedge} \text{fri-sibling-index } \text{len } i * \text{shift})^{\wedge} pw) \ 1 =$$

    
$$- \text{fri-fold-denominator} ((h^{\wedge} i * \text{shift})^{\wedge} pw) \ 1$$

    using fri-sibling-domain-round[OF len-pos even-len round, of  $i$ ]
    unfolding fri-sibling-index-def fri-fold-denominator-def
    by simp

lemma fri-sibling-index-first-half:
  fixes len idx :: nat
  assumes even-len:  $2 \text{ dvd } \text{len}$ 
    and idx-bound:  $\text{idx} < \text{len div } 2$ 
  shows  $\text{fri-sibling-index } \text{len } \text{idx} = \text{idx} + \text{len div } 2$ 
proof -
  have len-eq:  $\text{len} = 2 * (\text{len div } 2)$ 
    using even-len by simp
  have  $\text{idx} + \text{len div } 2 < \text{len}$ 
    using idx-bound len-eq by linarith
  then show ?thesis
    unfolding fri-sibling-index-def by simp
qed

```

```

lemma fri-sibling-index-involution-first-half:
  fixes len idx :: nat
  assumes even-len:  $2 \text{ dvd } \text{len}$ 
    and idx-bound:  $\text{idx} < \text{len div } 2$ 
  shows fri-sibling-index len (fri-sibling-index len idx) = idx
proof –
  have len-eq:  $\text{len} = 2 * (\text{len div } 2)$ 
    using even-len by simp
  have sib-eq: fri-sibling-index len idx = idx + len div 2
    by (rule fri-sibling-index-first-half[OF even-len idx-bound])
  have idx-len:  $\text{idx} < \text{len}$ 
    using idx-bound len-eq by linarith
  have sum-eq:  $\text{idx} + \text{len div } 2 + \text{len div } 2 = \text{idx} + \text{len}$ 
    using len-eq by linarith
  have fri-sibling-index len (fri-sibling-index len idx) =
     $((\text{idx} + \text{len div } 2) \bmod \text{len} + \text{len div } 2) \bmod \text{len}$ 
    unfolding sib-eq fri-sibling-index-def by simp
  also have  $\dots = (\text{idx} + \text{len div } 2 + \text{len div } 2) \bmod \text{len}$ 
    by (simp add: mod-add-left-eq)
  also have  $\dots = (\text{idx} + \text{len}) \bmod \text{len}$ 
    by (simp only: sum-eq)
  also have  $\dots = \text{idx}$ 
    using idx-len by simp
  finally show ?thesis .
qed

```

```

lemma fri-sibling-index-mod-half-second-half:
  fixes len raw :: nat
  assumes even-len:  $2 \text{ dvd } \text{len}$ 
    and raw-bound:  $\text{raw} < \text{len}$ 
    and second-half:  $\neg \text{raw} < \text{len div } 2$ 
  shows  $\text{raw} = \text{fri-sibling-index len } (\text{raw mod } (\text{len div } 2))$ 
proof –
  let ?half =  $\text{len div } 2$ 
  have len-eq:  $\text{len} = 2 * \text{?half}$ 
    using even-len by simp
  have half-pos:  $0 < \text{?half}$ 
    using raw-bound second-half len-eq by linarith
  have half-le-raw:  $\text{?half} \leq \text{raw}$ 
    using second-half by simp
  define k where  $k = \text{raw} - \text{?half}$ 
  have raw-eq:  $\text{raw} = \text{?half} + k$ 
    using half-le-raw unfolding k-def by simp
  have k-bound:  $k < \text{?half}$ 
    using raw-bound len-eq raw-eq by linarith
  have raw-mod:  $\text{raw mod } \text{?half} = k$ 
    using raw-eq k-bound half-pos by simp
  have fri-sibling-index len (raw mod ?half) =
     $(k + \text{?half}) \bmod \text{len}$ 

```

```

    unfolding raw-mod fri-sibling-index-def by simp
  also have ... = raw
    using raw-eq raw-bound by simp
  finally show ?thesis by simp
qed

lemma fri-table-fold-value-sibling-first-half:
  assumes even-len: 2 dvd len
    and idx-bound: idx < len div 2
    and round: len * pw = clength * scale
    and dom-idx: fri-dom ! idx = (h ^ idx * shift) ^ pw
    and dom-sib:
      fri-dom ! fri-sibling-index len idx =
        (h ^ fri-sibling-index len idx * shift) ^ pw
  shows
    fri-table-fold-value ch layer fri-dom len pw
      (fri-sibling-index len idx) =
        fri-table-fold-value ch layer fri-dom len pw idx
  proof -
    have len-pos: 0 < len
      using idx-bound by simp
    have sib-sib: fri-sibling-index len (fri-sibling-index len idx) = idx
      by (rule fri-sibling-index-involution-first-half
        [OF even-len idx-bound])
    have denom-sib:
      fri-fold-denominator
        ((h ^ fri-sibling-index len idx * shift) ^ pw) 1 =
        - fri-fold-denominator ((h ^ idx * shift) ^ pw) 1
      by (rule fri-sibling-fold-denominator
        [OF len-pos even-len round])
    have fri-table-fold-value ch layer fri-dom len pw
      (fri-sibling-index len idx) =
      fri-fold-value ch
        (layer ! fri-sibling-index len idx)
        (layer ! idx)
        (- fri-fold-denominator ((h ^ idx * shift) ^ pw) 1)
      unfolding fri-table-fold-value-def
      using sib-sib dom-sib denom-sib by simp
    also have ... =
      fri-fold-value ch
        (layer ! idx)
        (layer ! fri-sibling-index len idx)
        (fri-fold-denominator ((h ^ idx * shift) ^ pw) 1)
      by (rule fri-fold-value-swap-neg-denominator)
    also have ... = fri-table-fold-value ch layer fri-dom len pw idx
      unfolding fri-table-fold-value-def using dom-idx by simp
    finally show ?thesis .
  qed

```


lemma *fri-sampled-table-fold-from-algebraic-relation*:
assumes *relation*:
fri-algebraic-layer-relation *len* *pw* *ch* *fri-dom* *layer* *next-dom*
next-layer
and *idx-bound*: $idx < len \text{ div } 2$
and *chunk*:
fri-layer-opening-chunk *len* *xp* *xp-path* *xn* *xn-path* *chunk*
and *match*: *fri-opening-matches-table* *len* *idx* *layer* *xp* *xn*
and *dom-idx*: $fri-dom \ ! \ idx = (h \wedge idx * shift) \wedge pw$
shows $idx < length \ next-layer$
and *fri-sampled-table-fold* *ch* *len* *idx* *pw* *fri-dom* *layer* *chunk*
(*next-layer* ! *idx*)
proof –
show *idx-len*: $idx < length \ next-layer$
by (rule *fri-algebraic-layer-relation-next-index-bound*
[*OF relation idx-bound*])
have *fold-eq*:
next-layer ! *idx* = *fri-table-fold-value* *ch* *layer* *fri-dom* *len* *pw* *idx*
by (rule *fri-algebraic-layer-relation-sampled-fold-eq*
[*OF relation idx-bound*])
show *fri-sampled-table-fold* *ch* *len* *idx* *pw* *fri-dom* *layer* *chunk*
(*next-layer* ! *idx*)
by (rule *fri-sampled-table-foldI*[*OF chunk match dom-idx fold-eq*])
qed

lemma *fri-sampled-table-fold-from-algebraic-relation-sibling*:
assumes *relation*:
fri-algebraic-layer-relation *len* *pw* *ch* *fri-dom* *layer* *next-dom*
next-layer
and *even-len*: $2 \text{ dvd } len$
and *round*: $len * pw = clength * scale$
and *idx-bound*: $idx < len \text{ div } 2$
and *raw-eq*: *raw* = *fri-sibling-index* *len* *idx*
and *chunk*:
fri-layer-opening-chunk *len* *xp* *xp-path* *xn* *xn-path* *chunk*
and *match*: *fri-opening-matches-table* *len* *raw* *layer* *xp* *xn*
and *dom-raw*: $fri-dom \ ! \ raw = (h \wedge raw * shift) \wedge pw$
and *dom-idx*: $fri-dom \ ! \ idx = (h \wedge idx * shift) \wedge pw$
shows $idx < length \ next-layer$
and *fri-sampled-table-fold* *ch* *len* *raw* *pw* *fri-dom* *layer* *chunk*
(*next-layer* ! *idx*)
proof –
show *idx-len*: $idx < length \ next-layer$
by (rule *fri-algebraic-layer-relation-next-index-bound*
[*OF relation idx-bound*])
have *fold-idx*:
next-layer ! *idx* = *fri-table-fold-value* *ch* *layer* *fri-dom* *len* *pw* *idx*
by (rule *fri-algebraic-layer-relation-sampled-fold-eq*
[*OF relation idx-bound*])

```

have fold-row:
  fri-table-fold-value ch layer fri-dom len pw raw =
    fri-table-fold-value ch layer fri-dom len pw idx
  unfolding raw-eq
  by (rule fri-table-fold-value-sibling-first-half
      [OF even-len idx-bound round dom-idx])
    (use dom-row raw-eq in simp)
have fold-eq:
  next-layer ! idx = fri-table-fold-value ch layer fri-dom len pw raw
  using fold-idx fold-row by simp
show fri-sampled-table-fold ch len raw pw fri-dom layer chunk
  (next-layer ! idx)
  by (rule fri-sampled-table-foldI[OF chunk match dom-row fold-eq])
qed

lemma fri-sampled-table-fold-from-algebraic-relation-mod-index:
  assumes relation:
    fri-algebraic-layer-relation len pw ch fri-dom layer next-dom
    next-layer
  and even-len: 2 dvd len
  and round: len * pw = clength * scale
  and chunk:
    fri-layer-opening-chunk len xp xp-path xn xn-path chunk
  and match: fri-opening-matches-table len raw layer xp xn
  and dom-row: fri-dom ! raw = (h ^ raw * shift) ^ pw
  and dom-norm:
    fri-dom ! (raw mod (len div 2)) =
      (h ^ (raw mod (len div 2)) * shift) ^ pw
  shows raw mod (len div 2) < length next-layer
  and fri-sampled-table-fold ch len raw pw fri-dom layer chunk
    (next-layer ! (raw mod (len div 2)))
proof -
  have raw-bound: raw < len
  by (rule fri-opening-matches-tableD(1)[OF match])
  show idx-len: raw mod (len div 2) < length next-layer
  proof (cases raw < len div 2)
  case True
  then have raw-mod: raw mod (len div 2) = raw
  by simp
  show ?thesis
  using fri-sampled-table-fold-from-algebraic-relation(1)
    [OF relation True chunk match dom-row]
  unfolding raw-mod .
  next
  case False
  let ?idx = raw mod (len div 2)
  have half-pos: 0 < len div 2
  using raw-bound False even-len by (cases len) auto
  have idx-bound: ?idx < len div 2

```

```

    using half-pos by simp
show ?thesis
by (rule fri-sampled-table-fold-from-algebraic-relation-sibling(1)
  [OF relation even-len round idx-bound - chunk match dom-row
    dom-norm])
  (rule fri-sibling-index-mod-half-second-half
    [OF even-len raw-bound False])
qed
show fri-sampled-table-fold ch len raw pw fri-dom layer chunk
  (next-layer ! (raw mod (len div 2)))
proof (cases raw < len div 2)
case True
then have raw-mod: raw mod (len div 2) = raw
  by simp
show ?thesis
  using fri-sampled-table-fold-from-algebraic-relation(2)
    [OF relation True chunk match dom-row]
  unfolding raw-mod .
next
case False
let ?idx = raw mod (len div 2)
have half-pos: 0 < len div 2
  using raw-bound False even-len by (cases len) auto
have idx-bound: ?idx < len div 2
  using half-pos by simp
show ?thesis
  by (rule fri-sampled-table-fold-from-algebraic-relation-sibling(2)
    [OF relation even-len round idx-bound - chunk match dom-row
      dom-norm])
    (rule fri-sibling-index-mod-half-second-half
      [OF even-len raw-bound False])
qed
qed

lemma fri-sampled-table-fold-eq-at-mod-index:
  assumes fold:
    fri-sampled-table-fold ch len raw pw fri-dom layer chunk next-value
    and even-len: 2 dvd len
    and round: len * pw = clength * scale
    and dom-norm:
      fri-dom ! (raw mod (len div 2)) =
        (h ^ (raw mod (len div 2)) * shift) ^ pw
  shows next-value =
    fri-table-fold-value ch layer fri-dom len pw (raw mod (len div 2))
proof -
  from fold obtain xp xp-path xn xn-path where
    match: fri-opening-matches-table len raw layer xp xn
    and dom-row: fri-dom ! raw = (h ^ raw * shift) ^ pw
    and next-eq: next-value = fri-table-fold-value ch layer fri-dom len pw raw

```

```

    by (elim fri-sampled-table-foldE)
  have raw-bound: raw < len
    by (rule fri-opening-matches-tableD(1)[OF match])
  show ?thesis
  proof (cases raw < len div 2)
    case True
    then have raw mod (len div 2) = raw
      by simp
    then show ?thesis
      using next-eq by simp
  next
  case False
  let ?idx = raw mod (len div 2)
  have half-pos: 0 < len div 2
    using raw-bound False even-len by (cases len) auto
  have idx-bound: ?idx < len div 2
    using half-pos by simp
  have raw-eq: raw = fri-sibling-index len ?idx
    by (rule fri-sibling-index-mod-half-second-half
        [OF even-len raw-bound False])
  have dom-sib:
    fri-dom ! fri-sibling-index len ?idx =
      (h ^ fri-sibling-index len ?idx * shift) ^ pw
    using dom-row raw-eq by simp
  have fold-sib:
    fri-table-fold-value ch layer fri-dom len pw
      (fri-sibling-index len ?idx) =
      fri-table-fold-value ch layer fri-dom len pw ?idx
    by (rule fri-table-fold-value-sibling-first-half
        [OF even-len idx-bound round dom-norm dom-sib])
  have fri-table-fold-value ch layer fri-dom len pw raw =
    fri-table-fold-value ch layer fri-dom len pw ?idx
    using raw-eq fold-sib by simp
  then show ?thesis
    using next-eq by simp
qed
qed

```

lemma *fri-algebraic-layer-relation-from-sampled-coverage:*

```

  assumes even-len: 2 dvd len
    and round: len * pw = clength * scale
    and len-pos: 0 < len
    and dom-len: len ≤ length fri-dom
    and layer-len: len ≤ length layer
    and next-dom-len: len div 2 ≤ length next-dom
    and next-layer-len: len div 2 ≤ length next-layer
    and next-dom:
       $\bigwedge idx. idx < len \div 2 \implies$ 
      next-dom ! idx = (fri-dom ! idx)2

```

```

and dom-norm:
   $\bigwedge idx. idx < len \text{ div } 2 \implies$ 
     $fri\text{-}dom ! idx = (h \wedge idx * shift) \wedge pw$ 
and sampled-cover:
   $\bigwedge idx. idx < len \text{ div } 2 \implies$ 
     $\exists \text{ raw chunk.}$ 
     $raw \text{ mod } (len \text{ div } 2) = idx \wedge$ 
     $fri\text{-}sampled\text{-}table\text{-}fold \text{ ch len raw pw } fri\text{-}dom \text{ layer chunk}$ 
     $(next\text{-}layer ! idx)$ 
shows
   $fri\text{-}algebraic\text{-}layer\text{-}relation \text{ len pw ch } fri\text{-}dom \text{ layer } next\text{-}dom$ 
   $next\text{-}layer$ 
unfolding fri-algebraic-layer-relation-def
proof (intro conjI allI impI)
show  $0 < len$ 
  by (rule len-pos)
show  $len \leq length \text{ fri-dom}$ 
  by (rule dom-len)
show  $len \leq length \text{ layer}$ 
  by (rule layer-len)
show  $len \text{ div } 2 \leq length \text{ next-dom}$ 
  by (rule next-dom-len)
show  $len \text{ div } 2 \leq length \text{ next-layer}$ 
  by (rule next-layer-len)
fix idx
assume idx-bound:  $idx < len \text{ div } 2$ 
show  $next\text{-}dom ! idx = (fri\text{-}dom ! idx)^2$ 
  by (rule next-dom[OF idx-bound])
obtain raw chunk where raw-mod:  $raw \text{ mod } (len \text{ div } 2) = idx$ 
and fold:
   $fri\text{-}sampled\text{-}table\text{-}fold \text{ ch len raw pw } fri\text{-}dom \text{ layer chunk}$ 
   $(next\text{-}layer ! idx)$ 
using sampled-cover[OF idx-bound] by blast
have fold-eq:
   $next\text{-}layer ! idx =$ 
   $fri\text{-}table\text{-}fold\text{-}value \text{ ch layer } fri\text{-}dom \text{ len pw } (raw \text{ mod } (len \text{ div } 2))$ 
  by (rule fri-sampled-table-fold-eq-at-mod-index
    [OF fold even-len round])
    (use dom-norm idx-bound raw-mod in simp)
then show  $next\text{-}layer ! idx =$ 
   $fri\text{-}table\text{-}fold\text{-}value \text{ ch layer } fri\text{-}dom \text{ len pw } idx$ 
  using raw-mod by simp
qed

lemma generic-fri-sampled-layer-chain-bad-challenge-list-from-coverage:
assumes chain:
  generic-fri-sampled-layer-chain-evidence low-degree candidate-table
  degree-bound roots challenges final-value query-idxs round-layers
  doms layers

```

and *steps*:
fri-multiround-proximity-steps (*length challenges*) *degree-bound*
(*fri-layer-lengths* (*length challenges*) (*clength * scale*))
(*map* ($\lambda i. 2 \wedge i$) [*0..<length challenges*])
doms layers bad
and *covers*: *fri-sampled-layers-cover-fold-indices roots query-idxs*
and *even-len*:
 $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$
 $2 \text{ dvd } \text{fri-evidence-layer-len roots layer-idx}$
and *round-product*:
 $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$
 $\text{fri-evidence-layer-len roots layer-idx} * 2 \wedge \text{layer-idx} =$
 $\text{clength} * \text{scale}$
and *dom-len*:
 $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$
 $\text{fri-evidence-layer-len roots layer-idx} \leq \text{length} (\text{doms} ! \text{layer-idx})$
and *layer-len*:
 $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$
 $\text{fri-evidence-layer-len roots layer-idx} \leq$
 $\text{length} (\text{layers} ! \text{layer-idx})$
and *next-dom-len*:
 $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$
 $\text{fri-evidence-layer-len roots layer-idx} \text{ div } 2 \leq$
 $\text{length} (\text{doms} ! \text{Suc layer-idx})$
and *next-layer-len*:
 $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$
 $\text{fri-evidence-layer-len roots layer-idx} \text{ div } 2 \leq$
 $\text{length} (\text{layers} ! \text{Suc layer-idx})$
and *next-dom*:
 $\bigwedge \text{layer-idx idx}.$
 $\text{layer-idx} < \text{length challenges} \implies$
 $\text{idx} < \text{fri-evidence-layer-len roots layer-idx} \text{ div } 2 \implies$
 $\text{doms} ! \text{Suc layer-idx} ! \text{idx} =$
 $(\text{doms} ! \text{layer-idx} ! \text{idx})^2$
and *dom-norm*:
 $\bigwedge \text{layer-idx idx}.$
 $\text{layer-idx} < \text{length challenges} \implies$
 $\text{idx} < \text{fri-evidence-layer-len roots layer-idx} \text{ div } 2 \implies$
 $\text{doms} ! \text{layer-idx} ! \text{idx} = (\text{h} \wedge \text{idx} * \text{shift}) \wedge (2 \wedge \text{layer-idx})$
and *start-not-low*:
 $\neg \text{fri-table-low-degree-on degree-bound eval-domain candidate-table}$
and *final-low*:
fri-table-low-degree-on
(*fri-degree-after* (*length challenges*) *degree-bound*)
(*doms ! length challenges*) (*layers ! length challenges*)
shows *challenges* $\in$
generic-fri-bad-challenge-lists (*length challenges*) *bad*
proof –
have *partial*:

```

generic-fri-partial-evidence low-degree candidate-table degree-bound
  roots challenges final-value query-idxs round-layers
  by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
have roots-len: length challenges = length roots
  by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
have challenges-space: challenges ∈ fri-challenge-space (length challenges)
  unfolding fri-challenge-space-def by simp
have dom0: doms ! 0 = eval-domain
  by (rule generic-fri-sampled-layer-chain-evidenceD(4)[OF chain])
have layer0: layers ! 0 = candidate-table
  by (rule generic-fri-sampled-layer-chain-evidenceD(5)[OF chain])
have relations:
  ∧i. i < length challenges ⇒
    fri-algebraic-layer-relation
      (fri-layer-lengths (length challenges) (clength * scale) ! i)
      ((map (λi. 2 ^ i) [0..<length challenges]) ! i)
      (challenges ! i) (doms ! i) (layers ! i)
      (doms ! Suc i) (layers ! Suc i)
proof -
  fix i
  assume i-bound: i < length challenges
  have i-root-bound: i < length roots
    using i-bound roots-len by simp
  have len-eq:
    fri-layer-lengths (length challenges) (clength * scale) ! i =
    fri-evidence-layer-len roots i
    using roots-len unfolding fri-evidence-layer-len-def by simp
  have pw-eq:
    (map (λi. 2 ^ i) [0..<length challenges]) ! i = 2 ^ i
    using i-bound by simp
  have sampled-cover:
    ∧idx. idx < fri-evidence-layer-len roots i div 2 ⇒
      ∃ raw chunk.
        raw mod (fri-evidence-layer-len roots i div 2) = idx ∧
        fri-sampled-table-fold (challenges ! i)
          (fri-evidence-layer-len roots i) raw (2 ^ i)
          (doms ! i) (layers ! i) chunk
          (layers ! Suc i ! idx)
proof -
  fix idx
  assume idx-bound: idx < fri-evidence-layer-len roots i div 2
  obtain round-idx where round-bound: round-idx < length query-idxs
    and idx-eq:
      fri-evidence-next-idx roots query-idxs round-idx i = idx
  by (rule fri-sampled-layers-cover-fold-indicesD
    [OF covers i-root-bound idx-bound])
  have sample:
    fri-sampled-table-fold (challenges ! i)
      (fri-evidence-layer-len roots i)

```

```

    (fri-evidence-layer-idx roots query-idxs round-idx i)
    (2 ^ i) (doms ! i) (layers ! i)
    (round-layers ! round-idx ! i)
    (layers ! Suc i !
      fri-evidence-next-idx roots query-idxs round-idx i)
  by (rule generic-fri-sampled-layer-chain-evidence-sample(2)
      [OF chain round-bound i-bound])
show  $\exists$  raw chunk.
  raw mod (fri-evidence-layer-len roots i div 2) = idx  $\wedge$ 
  fri-sampled-table-fold (challenges ! i)
    (fri-evidence-layer-len roots i) raw (2 ^ i)
    (doms ! i) (layers ! i) chunk
    (layers ! Suc i ! idx)
  using sample idx-eq
  unfolding fri-evidence-next-idx-def by blast
qed
have len-pos: 0 < fri-evidence-layer-len roots i
  using round-product[OF i-bound] eval-domain-size-pos
  by (cases fri-evidence-layer-len roots i) simp-all
have rel:
  fri-algebraic-layer-relation
    (fri-evidence-layer-len roots i) (2 ^ i)
    (challenges ! i) (doms ! i) (layers ! i)
    (doms ! Suc i) (layers ! Suc i)
  by (rule fri-algebraic-layer-relation-from-sampled-coverage
      [OF even-len[OF i-bound] round-product[OF i-bound] len-pos
        dom-len[OF i-bound] layer-len[OF i-bound]
        next-dom-len[OF i-bound] next-layer-len[OF i-bound]])
    (use next-dom[OF i-bound] dom-norm[OF i-bound] sampled-cover
      in simp-all)
  then show
    fri-algebraic-layer-relation
      (fri-layer-lengths (length challenges) (clength * scale) ! i)
      ((map ( $\lambda i$ . 2 ^ i) [0.. $\text{length challenges}$ ]) ! i)
      (challenges ! i) (doms ! i) (layers ! i)
      (doms ! Suc i) (layers ! Suc i)
    using rel by (simp add: len-eq pw-eq)
qed
show ?thesis
  unfolding generic-fri-bad-challenge-lists-def
  by (rule fri-multi-round-proximity-skeleton
      [OF steps challenges-space relations])
    (use start-not-low final-low dom0 layer0 in simp-all)
qed

lemma generic-fri-sampled-layer-chain-full-cover-bad-challenge-list:
  assumes
    generic-fri-sampled-layer-chain-full-cover low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers

```



```

    doms layers bad
shows challenges ∈
    generic-fri-bad-challenge-lists (length challenges) bad
proof –
have chain:
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers
and steps:
    fri-multiround-proximity-steps (length challenges) degree-bound
    (fri-layer-lengths (length challenges) (clength * scale))
    (map (λi. 2 ^ i) [0..and covers: fri-sampled-layers-cover-fold-indices roots query-idxs
and even-len:
    ∧layer-idx. layer-idx < length challenges ⇒
    2 dvd fri-evidence-layer-len roots layer-idx
and round-product:
    ∧layer-idx. layer-idx < length challenges ⇒
    fri-evidence-layer-len roots layer-idx * 2 ^ layer-idx =
    clength * scale
and dom-len:
    ∧layer-idx. layer-idx < length challenges ⇒
    fri-evidence-layer-len roots layer-idx ≤
    length (doms ! layer-idx)
and layer-len:
    ∧layer-idx. layer-idx < length challenges ⇒
    fri-evidence-layer-len roots layer-idx ≤
    length (layers ! layer-idx)
and next-dom-len:
    ∧layer-idx. layer-idx < length challenges ⇒
    fri-evidence-layer-len roots layer-idx div 2 ≤
    length (doms ! Suc layer-idx)
and next-layer-len:
    ∧layer-idx. layer-idx < length challenges ⇒
    fri-evidence-layer-len roots layer-idx div 2 ≤
    length (layers ! Suc layer-idx)
and next-dom:
    ∧layer-idx idx.
    layer-idx < length challenges ⇒
    idx < fri-evidence-layer-len roots layer-idx div 2 ⇒
    doms ! Suc layer-idx ! idx =
    (doms ! layer-idx ! idx)2
and dom-norm:
    ∧layer-idx idx.
    layer-idx < length challenges ⇒
    idx < fri-evidence-layer-len roots layer-idx div 2 ⇒
    doms ! layer-idx ! idx = (h ^ idx * shift) ^ (2 ^ layer-idx)
and start-not-low:

```

$\neg$ *fri-table-low-degree-on degree-bound eval-domain candidate-table*
and *final-low*:
fri-table-low-degree-on
(fri-degree-after (length challenges) degree-bound)
(doms ! length challenges) (layers ! length challenges)
using *assms*
unfolding *generic-fri-sampled-layer-chain-full-cover-def*
by *blast+*
show *?thesis*
by (*rule generic-fri-sampled-layer-chain-bad-challenge-list-from-coverage*
[OF chain steps covers even-len round-product dom-len layer-len
next-dom-len next-layer-len next-dom dom-norm start-not-low
final-low])
qed

lemma *generic-fri-bad-candidate-from-sampled-layer-chain-full-cover*:
assumes *cover*:
generic-fri-sampled-layer-chain-full-cover low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad
and *not-low*: $\neg$ *low-degree candidate-table*
shows
generic-fri-bad-candidate low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers bad
proof —
have *sampled*:
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers
using *cover*
unfolding *generic-fri-sampled-layer-chain-full-cover-def* **by** *blast*
have *partial*:
generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
by (*rule generic-fri-sampled-layer-chain-evidenceD(1)[OF sampled]*)
have *bad*:
challenges $\in$ *generic-fri-bad-challenge-lists (length challenges) bad*
by (*rule generic-fri-sampled-layer-chain-full-cover-bad-challenge-list*
[OF cover])
show *?thesis*
unfolding *generic-fri-bad-candidate-def*
using *partial not-low bad* **by** *simp*
qed

lemma *generic-fri-canonical-opening-match-sample*:
assumes *openings*:
fri-all-round-openings-match-tables
(take (length challenges) layers) query-idxs round-layers
and *layers-len*: *length layers = Suc (length challenges)*

```

and roots-len: length challenges = length roots
and round-bound: round-idx < length query-idxs
and layer-bound: layer-idx < length challenges
obtains xp xp-path xn xn-path
where
  fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    xp xp-path xn xn-path (round-layers ! round-idx ! layer-idx)
and fri-opening-matches-table
  (fri-evidence-layer-len roots layer-idx)
  (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
  (layers ! layer-idx) xp xn
proof –
have layer-bound-take:
  layer-idx < length (take (length challenges) layers)
using layer-bound layers-len by simp
have openings-match:
  fri-layer-opening-matches-bound-table (query-idxs ! round-idx)
    layer-idx (take (length challenges) layers)
    (round-layers ! round-idx)
by (rule fri-all-round-openings-match-tablesD
    [OF openings round-bound layer-bound-take])
have tables-len:
  length (take (length challenges) layers) = length roots
using layers-len roots-len by simp
from fri-layer-opening-matches-bound-table-compactE
  [OF openings-match tables-len]
obtain xp xp-path xn xn-path where
  chunk:
    fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
      xp xp-path xn xn-path (round-layers ! round-idx ! layer-idx)
and table-match-take:
  fri-opening-matches-table
    (fri-evidence-layer-len roots layer-idx)
    (fri-layer-indices (length roots) (query-idxs ! round-idx)
      (clength * scale) ! layer-idx)
    (take (length challenges) layers ! layer-idx) xp xn
by blast
have idx-eq:
  fri-layer-indices (length roots) (query-idxs ! round-idx)
    (clength * scale) ! layer-idx =
  fri-evidence-layer-idx roots query-idxs round-idx layer-idx
unfolding fri-evidence-layer-idx-def by simp
have table-eq:
  take (length challenges) layers ! layer-idx = layers ! layer-idx
using layer-bound layers-len by simp
have table-match:
  fri-opening-matches-table
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)

```

```

    (layers ! layer-idx) xp xn
  using table-match-take idx-eq table-eq by simp
show ?thesis
  by (rule that[OF chunk table-match])
qed

lemma generic-fri-canonical-sampled-fold-at-first-half:
  assumes chain:
    generic-fri-canonical-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers bad
  and openings:
    fri-all-round-openings-match-tables
    (take (length challenges) layers) query-idxs round-layers
  and domains: fri-sampled-domains-align roots query-idxs doms
  and idx-bound:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
    fri-evidence-layer-len roots layer-idx div 2
  and round-bound: round-idx < length query-idxs
  and layer-bound: layer-idx < length challenges
  shows
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (layers ! Suc layer-idx)
  fri-sampled-table-fold
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (doms ! layer-idx)
    (layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
proof -
  have partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
  using chain unfolding generic-fri-canonical-layer-chain-evidence-def
  by simp
  have roots-len: length challenges = length roots
  by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
  have layers-len: length layers = Suc (length challenges)
  by (rule generic-fri-canonical-layer-chain-lengths(2)[OF chain])
  have len-eq:
    fri-layer-lengths (length challenges) (clength * scale) ! layer-idx =
    fri-evidence-layer-len roots layer-idx
  using roots-len unfolding fri-evidence-layer-len-def by simp
  have relation:
    fri-algebraic-layer-relation

```

```

    (fri-evidence-layer-len roots layer-idx)
    (2 ^ layer-idx)
    (challenges ! layer-idx) (doms ! layer-idx) (layers ! layer-idx)
    (doms ! Suc layer-idx) (layers ! Suc layer-idx)
using generic-fri-canonical-layer-chain-relation
    [OF chain layer-bound] len-eq by simp
have dom-idx:
  doms ! layer-idx !
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    (h ^ fri-evidence-layer-idx roots query-idxs round-idx layer-idx *
      shift) ^ (2 ^ layer-idx)
  by (rule fri-sampled-domains-alignD[OF domains round-bound])
    (use layer-bound roots-len in simp)
obtain xp xp-path xn xn-path where
  chunk:
    fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    xp xp-path xn xn-path (round-layers ! round-idx ! layer-idx)
and match:
  fri-opening-matches-table
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (layers ! layer-idx) xp xn
  by (rule generic-fri-canonical-opening-match-sample
    [OF openings layers-len roots-len round-bound layer-bound])
have next-idx-eq:
  fri-evidence-next-idx roots query-idxs round-idx layer-idx =
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx
  using idx-bound unfolding fri-evidence-next-idx-def by simp
have idx-len-orig:
  fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
    length (layers ! Suc layer-idx)
  by (rule fri-sampled-table-fold-from-algebraic-relation(1)
    [OF relation idx-bound chunk match dom-idx])
then show
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (layers ! Suc layer-idx)
  using next-idx-eq by simp
have fold-orig:
  fri-sampled-table-fold
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (doms ! layer-idx)
    (layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
  by (rule fri-sampled-table-fold-from-algebraic-relation

```

```

    [OF relation idx-bound chunk match dom-idx])
  then show
    fri-sampled-table-fold
      (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      (2 ^ layer-idx)
      (doms ! layer-idx)
      (layers ! layer-idx)
      (round-layers ! round-idx ! layer-idx)
      (layers ! Suc layer-idx !
        fri-evidence-next-idx roots query-idxs round-idx layer-idx)
    using next-idx-eq by simp
  qed

lemma generic-fri-canonical-sampled-fold-at-any-index:
  assumes chain:
    generic-fri-canonical-layer-chain-evidence low-degree candidate-table
      degree-bound roots challenges final-value query-idxs round-layers
      doms layers bad
  and openings:
    fri-all-round-openings-match-tables
      (take (length challenges) layers) query-idxs round-layers
  and domains: fri-sampled-domains-align roots query-idxs doms
  and even-len: 2 dvd fri-evidence-layer-len roots layer-idx
  and round:
    fri-evidence-layer-len roots layer-idx * 2 ^ layer-idx =
      clength * scale
  and dom-norm:
    doms ! layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx =
      (h ^ fri-evidence-next-idx roots query-idxs round-idx layer-idx *
        shift) ^ (2 ^ layer-idx)
  and round-bound: round-idx < length query-idxs
  and layer-bound: layer-idx < length challenges
  shows
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
      length (layers ! Suc layer-idx)
  fri-sampled-table-fold
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (doms ! layer-idx)
    (layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  proof -

```

```

have partial:
  generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
  using chain unfolding generic-fri-canonical-layer-chain-evidence-def
  by simp
have roots-len: length challenges = length roots
  by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
have layers-len: length layers = Suc (length challenges)
  by (rule generic-fri-canonical-layer-chain-lengths(2)[OF chain])
have len-eq:
  fri-layer-lengths (length challenges) (clength * scale) ! layer-idx =
    fri-evidence-layer-len roots layer-idx
  using roots-len unfolding fri-evidence-layer-len-def by simp
have relation:
  fri-algebraic-layer-relation
    (fri-evidence-layer-len roots layer-idx)
    (2 ^ layer-idx)
    (challenges ! layer-idx) (doms ! layer-idx) (layers ! layer-idx)
    (doms ! Suc layer-idx) (layers ! Suc layer-idx)
  using generic-fri-canonical-layer-chain-relation
    [OF chain layer-bound] len-eq by simp
have dom-raw:
  doms ! layer-idx !
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    (h ^ fri-evidence-layer-idx roots query-idxs round-idx layer-idx *
      shift) ^ (2 ^ layer-idx)
  by (rule fri-sampled-domains-alignD[OF domains round-bound])
    (use layer-bound roots-len in simp)
obtain xp xp-path xn xn-path where
  chunk:
    fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
      xp xp-path xn xn-path (round-layers ! round-idx ! layer-idx)
  and match:
    fri-opening-matches-table
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      (layers ! layer-idx) xp xn
  by (rule generic-fri-canonical-opening-match-sample
    [OF openings layers-len roots-len round-bound layer-bound])
show
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (layers ! Suc layer-idx)
  unfolding fri-evidence-next-idx-def
  by (rule fri-sampled-table-fold-from-algebraic-relation-mod-index(1)
    [OF relation even-len round chunk match dom-raw])
    (use dom-norm fri-evidence-next-idx-def in simp)
show
  fri-sampled-table-fold
    (challenges ! layer-idx)

```

```

(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
(2 ^ layer-idx)
(doms ! layer-idx)
(layers ! layer-idx)
(round-layers ! round-idx ! layer-idx)
(layers ! Suc layer-idx !
  fri-evidence-next-idx roots query-idxs round-idx layer-idx)
unfolding fri-evidence-next-idx-def
by (rule fri-sampled-table-fold-from-algebraic-relation-mod-index(2)
  [OF relation even-len round chunk match dom-row])
  (use dom-norm fri-evidence-next-idx-def in simp)
qed

```

lemma *generic-fri-canonical-sampled-raw-domain-power:*

assumes *chain:*

generic-fri-canonical-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad

and *openings:*

fri-all-round-openings-match-tables
(take (length challenges) layers) query-idxs round-layers

and *round-bound: round-idx < length query-idxs*

and *layer-bound: layer-idx < length challenges*

shows

doms ! layer-idx !
fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
*(h ^ fri-evidence-layer-idx roots query-idxs round-idx layer-idx **
shift) ^ (2 ^ layer-idx)

proof –

have *partial:*

generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers

using *chain* **unfolding** *generic-fri-canonical-layer-chain-evidence-def*

by *simp*

have *roots-len: length challenges = length roots*

by (rule *generic-fri-partial-evidence-shapes(2)*[OF *partial*])

have *layers-len: length layers = Suc (length challenges)*

by (rule *generic-fri-canonical-layer-chain-lengths(2)*[OF *chain*])

obtain *xp xp-path xn xn-path* **where** *match:*

fri-opening-matches-table
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
(layers ! layer-idx) xp xn

by (rule *generic-fri-canonical-opening-match-sample*
 [OF *openings layers-len roots-len round-bound layer-bound*])

have *idx-bound:*

fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
fri-evidence-layer-len roots layer-idx


```

    by (rule fri-opening-matches-tableD(1)[OF match])
  show ?thesis
    by (rule generic-fri-canonical-domain-power
        [OF chain roots-len layer-bound idx-bound])
qed

lemma generic-fri-canonical-layer-chain-as-sampled:
  assumes chain:
    generic-fri-canonical-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers bad
  and openings:
    fri-all-round-openings-match-tables
    (take (length challenges) layers) query-idxs round-layers
  and domains: fri-sampled-domains-align roots query-idxs doms
  and next-domains: fri-sampled-next-domains-align roots query-idxs doms
  and even-len:
     $\bigwedge \text{layer-idx. layer-idx} < \text{length challenges} \implies$ 
    2 dvd fri-evidence-layer-len roots layer-idx
  and round:
     $\bigwedge \text{layer-idx. layer-idx} < \text{length challenges} \implies$ 
    fri-evidence-layer-len roots layer-idx * 2  $^{\text{layer-idx}}$  =
    clength * scale
  shows
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers
proof -
  have partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
  using chain unfolding generic-fri-canonical-layer-chain-evidence-def
  by simp
  have roots-len: length challenges = length roots
  by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
  have doms-len: length doms = Suc (length challenges)
  by (rule generic-fri-canonical-layer-chain-lengths(1)[OF chain])
  have layers-len: length layers = Suc (length challenges)
  by (rule generic-fri-canonical-layer-chain-lengths(2)[OF chain])
  have dom0: doms ! 0 = eval-domain
  using chain unfolding generic-fri-canonical-layer-chain-evidence-def
  by simp
  have layer0: layers ! 0 = candidate-table
  using chain unfolding generic-fri-canonical-layer-chain-evidence-def
  by simp
  have final:
    fri-final-constant-consistent (layers ! length challenges) final-value
  using chain unfolding generic-fri-canonical-layer-chain-evidence-def
  by simp

```

```

have sampled:
   $\wedge \text{round-idx layer-idx.}$ 
   $\text{round-idx} < \text{length query-idxs} \implies$ 
   $\text{layer-idx} < \text{length challenges} \implies$ 
   $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} <$ 
   $\text{length (layers ! Suc layer-idx)} \wedge$ 
   $\text{fri-sampled-table-fold}$ 
   $(\text{challenges ! layer-idx})$ 
   $(\text{fri-evidence-layer-len roots layer-idx})$ 
   $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx})$ 
   $(2 \wedge \text{layer-idx})$ 
   $(\text{doms ! layer-idx})$ 
   $(\text{layers ! layer-idx})$ 
   $(\text{round-layers ! round-idx ! layer-idx})$ 
   $(\text{layers ! Suc layer-idx !}$ 
   $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx})$ 
proof –
  fix round-idx layer-idx
  assume round-bound: round-idx < length query-idxs
  and layer-bound: layer-idx < length challenges
  have layer-bound-roots: layer-idx < length roots
  using layer-bound roots-len by simp
  have dom-norm:
    doms ! layer-idx !
     $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} =$ 
     $(h \wedge \text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} *$ 
     $\text{shift}) \wedge (2 \wedge \text{layer-idx})$ 
  by (rule fri-sampled-next-domains-alignD
    [OF next-domains round-bound layer-bound-roots])
  show
     $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} <$ 
     $\text{length (layers ! Suc layer-idx)} \wedge$ 
     $\text{fri-sampled-table-fold}$ 
     $(\text{challenges ! layer-idx})$ 
     $(\text{fri-evidence-layer-len roots layer-idx})$ 
     $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx})$ 
     $(2 \wedge \text{layer-idx})$ 
     $(\text{doms ! layer-idx})$ 
     $(\text{layers ! layer-idx})$ 
     $(\text{round-layers ! round-idx ! layer-idx})$ 
     $(\text{layers ! Suc layer-idx !}$ 
     $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx})$ 
  by (intro conjI)
    (rule generic-fri-canonical-sampled-fold-at-any-index
      [OF chain openings domains even-len[OF layer-bound]
        round[OF layer-bound] dom-norm round-bound layer-bound])+
qed
show ?thesis
  unfolding generic-fri-sampled-layer-chain-evidence-def

```

using partial doms-len layers-len dom0 layer0 final sampled by blast
qed

lemma generic-fri-bad-challenge-lists-subset:
generic-fri-bad-challenge-lists n bad $\subseteq$ fri-challenge-space n
unfolding generic-fri-bad-challenge-lists-def
by (rule fri-multiround-bad-challenge-lists-subset)

lemma generic-fri-bad-challenge-lists-round-union:
generic-fri-bad-challenge-lists n bad =
($\bigcup i \in \{..<n\}$. generic-fri-round-bad-challenge-lists n bad i)
unfolding generic-fri-bad-challenge-lists-def
fri-multiround-bad-challenge-lists-def
generic-fri-round-bad-challenge-lists-def
by auto

lemma generic-fri-round-bad-challenge-lists-image-subset:
generic-fri-round-bad-challenge-lists n bad $i \subseteq$
($\lambda(\text{prefix}, b, \text{suffix}). \text{prefix} @ b \# \text{suffix}$) '
generic-fri-round-bad-witnesses n bad i

proof
fix bs
assume bs-in: bs $\in$ generic-fri-round-bad-challenge-lists n bad i
then have bs-space: bs $\in$ fri-challenge-space n
and i-bound: $i < n$
and hit: bs ! $i \in$ bad i (take i bs)
unfolding generic-fri-round-bad-challenge-lists-def by simp-all
have len-bs: length bs = n
using bs-space unfolding fri-challenge-space-def by simp
let ?prefix = take i bs
let ?b = bs ! i
let ?suffix = drop (Suc i) bs
have witness:
(?prefix, ?b, ?suffix) $\in$ generic-fri-round-bad-witnesses n bad i
unfolding generic-fri-round-bad-witnesses-def
using i-bound hit len-bs by simp
have bs-eq: bs = ?prefix @ ?b # ?suffix
using i-bound len-bs by (simp add: id-take-nth-drop)
show bs $\in$ ($\lambda(\text{prefix}, b, \text{suffix}). \text{prefix} @ b \# \text{suffix}$) '
generic-fri-round-bad-witnesses n bad i
using witness bs-eq by force
qed

lemma generic-fri-round-bad-witnesses-subset:
generic-fri-round-bad-witnesses n bad $i \subseteq$
{($\text{prefix}, b, \text{suffix}$).
length prefix = $i \wedge$
 $b \in$ bad i prefix $\wedge$
length suffix = $n - \text{Suc } i$ }

unfolding *generic-fri-round-bad-witnesses-def* **by** *auto*

lemma *generic-fri-bad-challenge-lists-card-bound-by-rounds*:

card (generic-fri-bad-challenge-lists n bad) ≤
 $(\sum i \in \{..<n\}. \text{card } (\text{generic-fri-round-bad-challenge-lists } n \text{ bad } i))$

unfolding *generic-fri-bad-challenge-lists-round-union*

by (*rule card-UN-le*) *simp*

lemma *generic-fri-bad-challenge-lists-bounded-byI*:

assumes *card-bound*: *card (generic-fri-bad-challenge-lists n bad) ≤ bd*

shows *fri-bad-challenge-lists-bounded-by n*

(generic-fri-bad-challenge-lists n bad) bd

unfolding *fri-bad-challenge-lists-bounded-by-def*

using *generic-fri-bad-challenge-lists-subset card-bound* **by** *simp*

lemma *generic-fri-round-bad-witnesses-finite*:

finite (generic-fri-round-bad-witnesses n bad i)

proof –

let *?P* = {*prefix* :: 'f list. *length prefix* = *i*}

let *?S* = {*suffix* :: 'f list. *length suffix* = *n – Suc i*}

have *witness-eq*:

generic-fri-round-bad-witnesses n bad i =

*(if i < n then (SIGMA *prefix*:?P. bad i *prefix* × ?S) else {})*

unfolding *generic-fri-round-bad-witnesses-def* **by** *auto*

show *?thesis*

proof (*cases i < n*)

case *True*

have *finite-P*: *finite ?P*

by (*rule finite-length-lists-UNIV*)

have *finite-S*: *finite ?S*

by (*rule finite-length-lists-UNIV*)

have *finite-sigma*: *finite (SIGMA *prefix*:?P. bad i *prefix* × ?S)*

by (*rule finite-SigmaI[OF finite-P]*) (*use finite-S in simp*)

then show *?thesis*

using *True witness-eq* **by** *simp*

next

case *False*

then show *?thesis*

using *witness-eq* **by** *simp*

qed

qed

lemma *generic-fri-round-bad-witnesses-card-bound*:

assumes *local-bound*:

$\bigwedge \text{prefix}. \text{length prefix} = i \implies \text{card } (\text{bad } i \text{ prefix}) \leq bd$

shows *card (generic-fri-round-bad-witnesses n bad i) ≤*

$\text{CARD}('f) \wedge i * bd * \text{CARD}('f) \wedge (n - \text{Suc } i)$

proof –

let *?P* = {*prefix* :: 'f list. *length prefix* = *i*}

```

let ?S = {suffix :: 'f list. length suffix = n - Suc i}
have witness-eq:
  generic-fri-round-bad-witnesses n bad i =
    (if i < n then (SIGMA prefix: ?P. bad i prefix × ?S) else {})
  unfolding generic-fri-round-bad-witnesses-def by auto
show ?thesis
proof (cases i < n)
  case False
  then show ?thesis
    using witness-eq by simp
next
  case True
  have finite-P: finite ?P
    by (rule finite-length-lists-UNIV)
  have finite-S: finite ?S
    by (rule finite-length-lists-UNIV)
  have finite-products: ∀ prefix ∈ ?P. finite (bad i prefix × ?S)
    using finite-S by simp
  have card-witness:
    card (generic-fri-round-bad-witnesses n bad i) =
      (∑ prefix ∈ ?P. card (bad i prefix × ?S))
  proof -
    have card (SIGMA prefix: ?P. bad i prefix × ?S) =
      (∑ prefix ∈ ?P. card (bad i prefix × ?S))
      by (rule card-SigmaI[OF finite-P finite-products])
    then show ?thesis
      using True witness-eq by simp
  qed
  also have ... ≤ (∑ prefix ∈ ?P.
    card (bad i prefix) * CARD('f) ^ (n - Suc i))
  proof (rule sum-mono)
    fix prefix
    assume prefix ∈ ?P
    have card (bad i prefix × ?S) =
      card (bad i prefix) * card ?S
      by (simp add: card-cartesian-product)
    also have ... =
      card (bad i prefix) * CARD('f) ^ (n - Suc i)
      by (simp add: card-length-lists-UNIV)
    finally show card (bad i prefix × ?S) ≤
      card (bad i prefix) * CARD('f) ^ (n - Suc i)
      by simp
  qed
  also have ... ≤ (∑ prefix ∈ ?P.
    bd * CARD('f) ^ (n - Suc i))
  proof (rule sum-mono)
    fix prefix
    assume prefix-in: prefix ∈ ?P
    have card (bad i prefix) ≤ bd

```

```

    by (rule local-bound) (use prefix-in in simp)
  then show card (bad i prefix) * CARD('f) ^ (n - Suc i)
    ≤ bd * CARD('f) ^ (n - Suc i)
    by simp
qed
also have ... = card ?P * (bd * CARD('f) ^ (n - Suc i))
  by simp
also have ... = CARD('f) ^ i * bd * CARD('f) ^ (n - Suc i)
  by (simp add: card-length-lists-UNIV mult.assoc)
finally show ?thesis
  using card-witness by simp
qed
qed

```

lemma generic-fri-round-bad-challenge-lists-card-bound:

assumes local-bound:

$\bigwedge \text{prefix}. \text{length prefix} = i \implies \text{card (bad i prefix)} \leq \text{bd}$

shows card (generic-fri-round-bad-challenge-lists n bad i) ≤
 CARD('f) ^ i * bd * CARD('f) ^ (n - Suc i)

proof —

have image-subset:

generic-fri-round-bad-challenge-lists n bad i ⊆
 (λ(prefix, b, suffix). prefix @ b # suffix) ‘
 generic-fri-round-bad-witnesses n bad i

by (rule generic-fri-round-bad-challenge-lists-image-subset)

have finite-image:

finite ((λ(prefix, b, suffix). prefix @ b # suffix) ‘
 generic-fri-round-bad-witnesses n bad i)

by (rule finite-imageI)

(rule generic-fri-round-bad-witnesses-finite)

have card (generic-fri-round-bad-challenge-lists n bad i) ≤

card ((λ(prefix, b, suffix). prefix @ b # suffix) ‘
 generic-fri-round-bad-witnesses n bad i)

by (rule card-mono[OF finite-image image-subset])

also have ... ≤ card (generic-fri-round-bad-witnesses n bad i)

by (rule card-image-le)

(rule generic-fri-round-bad-witnesses-finite)

also have ... ≤ CARD('f) ^ i * bd * CARD('f) ^ (n - Suc i)

by (rule generic-fri-round-bad-witnesses-card-bound)

(rule local-bound)

finally show ?thesis .

qed

lemma generic-fri-bad-challenge-lists-card-bound-from-round-bounds:

assumes local-bound:

$\bigwedge i \text{ prefix}. i < n \implies \text{length prefix} = i \implies$

card (bad i prefix) ≤ bd i

shows card (generic-fri-bad-challenge-lists n bad) ≤

($\sum i \in \{..<n\}. \text{CARD('f)} ^ i * \text{bd } i * \text{CARD('f)} ^ (n - \text{Suc } i)$)

proof –
have $\text{card } (\text{generic-fri-bad-challenge-lists } n \text{ bad}) \leq$
 $(\sum i \in \{..<n\}. \text{card } (\text{generic-fri-round-bad-challenge-lists } n \text{ bad } i))$
by (rule generic-fri-bad-challenge-lists-card-bound-by-rounds)
also have $\dots \leq$
 $(\sum i \in \{..<n\}. \text{CARD}('f) \wedge i * \text{bd } i * \text{CARD}('f) \wedge (n - \text{Suc } i))$
proof (rule sum-mono)
fix i
assume $i\text{-in}: i \in \{..<n\}$
show $\text{card } (\text{generic-fri-round-bad-challenge-lists } n \text{ bad } i) \leq$
 $\text{CARD}('f) \wedge i * \text{bd } i * \text{CARD}('f) \wedge (n - \text{Suc } i)$
by (rule generic-fri-round-bad-challenge-lists-card-bound)
(rule local-bound[of i], use $i\text{-in}$ in simp-all)
qed
finally show ?thesis .
qed

lemma generic-fri-bad-challenge-lists-bounded-by-from-round-bounds:
assumes local-bound:
 $\bigwedge i \text{ prefix}. i < n \implies \text{length prefix} = i \implies$
 $\text{card } (\text{bad } i \text{ prefix}) \leq \text{bd } i$
shows fri-bad-challenge-lists-bounded-by n
 $(\text{generic-fri-bad-challenge-lists } n \text{ bad})$
 $(\sum i \in \{..<n\}. \text{CARD}('f) \wedge i * \text{bd } i * \text{CARD}('f) \wedge (n - \text{Suc } i))$
by (rule generic-fri-bad-challenge-lists-bounded-byI)
(rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds
[OF local-bound])

lemma generic-fri-bad-challenge-lists-card-bound:
 $\text{card } (\text{generic-fri-bad-challenge-lists } n \text{ bad}) \leq \text{CARD}('f) \wedge n$
unfolding generic-fri-bad-challenge-lists-def
by (rule fri-multiround-bad-challenge-lists-card-bound)

lemma generic-fri-bad-challenge-lists-bounded-by:
fri-bad-challenge-lists-bounded-by n
 $(\text{generic-fri-bad-challenge-lists } n \text{ bad}) (\text{CARD}('f) \wedge n)$
unfolding generic-fri-bad-challenge-lists-def
by (rule fri-multiround-bad-challenge-lists-bounded-by)

lemma generic-fri-low-degree-bad-challenge-listI:
assumes steps:
fri-multiround-proximity-steps n d lens pws doms layers bad
and challenges: $bs \in \text{fri-challenge-space } n$
and relations:
 $\bigwedge i. i < n \implies$
fri-algebraic-layer-relation (lens ! i) (pws ! i) (bs ! i)
(doms ! i) (layers ! i) (doms ! $\text{Suc } i$) (layers ! $\text{Suc } i$)
and start-not-low:
 $\neg \text{fri-table-low-degree-on } d (\text{doms ! } 0) (\text{layers ! } 0)$

and *final-low*:
fri-table-low-degree-on (*fri-degree-after* *n d*) (*doms* ! *n*)
(*layers* ! *n*)
shows *bs* ∈ *generic-fri-bad-challenge-lists* *n bad*
unfolding *generic-fri-bad-challenge-lists-def*
by (*rule fri-multi-round-proximity-skeleton*
[*OF steps challenges relations start-not-low final-low*])

lemma *generic-fri-proximity-evidence-bad-challenge-list*:
assumes *generic-fri-proximity-evidence* *n d lens pws doms layers bs bad*
shows *bs* ∈ *generic-fri-bad-challenge-lists* *n bad*
proof –
have *steps*:
fri-multi-round-proximity-steps *n d lens pws doms layers bad*
and *challenges*: *bs* ∈ *fri-challenge-space* *n*
and *relations*:
 $\bigwedge i. i < n \implies$
fri-algebraic-layer-relation (*lens* ! *i*) (*pws* ! *i*) (*bs* ! *i*)
(*doms* ! *i*) (*layers* ! *i*) (*doms* ! *Suc i*) (*layers* ! *Suc i*)
and *start-not-low*:
 $\neg$ *fri-table-low-degree-on* *d* (*doms* ! 0) (*layers* ! 0)
and *final-low*:
fri-table-low-degree-on (*fri-degree-after* *n d*) (*doms* ! *n*)
(*layers* ! *n*)
using *assms* **unfolding** *generic-fri-proximity-evidence-def* **by** *blast+*
show *?thesis*
by (*rule generic-fri-low-degree-bad-challenge-listI*
[*OF steps challenges relations start-not-low final-low*])
qed

lemma *generic-fri-canonical-layer-chain-bad-challenge-list*:
assumes
generic-fri-canonical-layer-chain-evidence *low-degree candidate-table*
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad
shows *challenges* ∈
generic-fri-bad-challenge-lists (*length challenges*) *bad*
proof –
have *generic-fri-proximity-evidence* (*length challenges*) *degree-bound*
(*fri-layer-lengths* (*length challenges*) (*clength* * *scale*))
(*map* ($\lambda i. 2 \wedge i$) [*0..<length challenges*])
doms layers challenges bad
using *assms*
unfolding *generic-fri-canonical-layer-chain-evidence-def* **by** *simp*
then show *?thesis*
by (*rule generic-fri-proximity-evidence-bad-challenge-list*)
qed

lemma *generic-fri-canonical-layer-chain-evidenceD*:

assumes

*generic-fri-canonical-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad*

shows

*generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
doms ! 0 = eval-domain
layers ! 0 = candidate-table
fri-final-constant-consistent (layers ! length challenges) final-value
generic-fri-proximity-evidence (length challenges) degree-bound
(fri-layer-lengths (length challenges) (clength * scale))
(map ($\lambda i. 2^i$) [0..<length challenges])
doms layers challenges bad*

using *assms* **unfolding** *generic-fri-canonical-layer-chain-evidence-def*
by *simp-all*

lemma *generic-fri-bad-candidate-from-canonical-layer-chain:*

assumes *chain:*

*generic-fri-canonical-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad*

and *not-low: $\neg$ low-degree candidate-table*

shows

*generic-fri-bad-candidate low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers bad*

proof –

have *partial:*

*generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
by (rule generic-fri-canonical-layer-chain-evidenceD(1)[OF chain])*

have *bad:*

*challenges $\in$
generic-fri-bad-challenge-lists (length challenges) bad
by (rule generic-fri-canonical-layer-chain-bad-challenge-list[OF chain])*

show *?thesis*

unfolding *generic-fri-bad-candidate-def*

using *partial not-low bad* **by** *simp*

qed

lemma *generic-fri-bad-challenge-lists-mono:*

assumes *mono:*

*$\bigwedge i \text{ prefix. } i < n \implies \text{length prefix} = i \implies$
*bad i prefix $\subseteq$ bad' i prefix**

shows *generic-fri-bad-challenge-lists n bad $\subseteq$*

generic-fri-bad-challenge-lists n bad'

proof

fix *bs*

assume *bs-in: bs $\in$ generic-fri-bad-challenge-lists n bad*

then have *space*: $bs \in \text{fri-challenge-space } n$
unfolding *generic-fri-bad-challenge-lists-def*
using *fri-multiround-bad-challenge-lists-subset* **by** *blast*
from *bs-in* **obtain** *i* **where** *i-bound*: $i < n$
and *hit*: $bs ! i \in \text{bad } i$ (*take* *i* *bs*)
unfolding *generic-fri-bad-challenge-lists-def*
fri-multiround-bad-challenge-lists-def
by *blast*
have *len-bs*: $\text{length } bs = n$
using *space* **unfolding** *fri-challenge-space-def* **by** *simp*
have *len-take*: $\text{length } (\text{take } i \text{ } bs) = i$
using *i-bound* *len-bs* **by** *simp*
have $bs ! i \in \text{bad}' i$ (*take* *i* *bs*)
using *hit* *mono*[*OF* *i-bound* *len-take*] **by** *blast*
then show $bs \in \text{generic-fri-bad-challenge-lists } n \text{ bad}'$
using *space* *i-bound*
unfolding *generic-fri-bad-challenge-lists-def*
fri-multiround-bad-challenge-lists-def
by *blast*
qed

lemma *generic-fri-bad-candidateD*:
assumes *generic-fri-bad-candidate* *low-degree candidate-table* *degree-bound*
roots *challenges* *final-value* *query-idxs* *round-layers* *bad*
shows *generic-fri-partial-evidence* *low-degree candidate-table* *degree-bound*
roots *challenges* *final-value* *query-idxs* *round-layers*
and $\neg \text{low-degree candidate-table}$
and $\text{challenges} \in \text{generic-fri-bad-challenge-lists } (\text{length } \text{challenges}) \text{ bad}$
using *assms* **unfolding** *generic-fri-bad-candidate-def* **by** *simp-all*

lemma *generic-fri-bad-candidate-challenges-in-space*:
assumes *bad-candidate*:
generic-fri-bad-candidate *low-degree candidate-table* *degree-bound*
roots *challenges* *final-value* *query-idxs* *round-layers* *bad*
shows $\text{challenges} \in \text{fri-challenge-space } (\text{length } \text{challenges})$
proof –
have $\text{challenges} \in \text{generic-fri-bad-challenge-lists } (\text{length } \text{challenges}) \text{ bad}$
using *generic-fri-bad-candidateD*(*3*)[*OF* *bad-candidate*] .
then show *?thesis*
using *generic-fri-bad-challenge-lists-subset*[*of* *length* *challenges* *bad*]
by *blast*
qed

lemma *generic-fri-bad-candidate-challenges-length*:
assumes *bad-candidate*:
generic-fri-bad-candidate *low-degree candidate-table* *degree-bound*
roots *challenges* *final-value* *query-idxs* *round-layers* *bad*
shows $\text{length } \text{challenges} = \text{fri-round-count-for-degree-bound } \text{degree-bound}$
proof –

have *evidence*:
generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
using *generic-fri-bad-candidateD(1)[OF bad-candidate]* .
then show *?thesis*
using *generic-fri-partial-evidence-shapes(1,2)[OF evidence]* **by** *simp*
qed

lemma *trace-table-low-degree-iff-fri-table-low-degree-on-eval-domain*:

trace-table-low-degree trace-table $\longleftrightarrow$
fri-table-low-degree-on (clength - 1) eval-domain trace-table

proof

assume *trace-table-low-degree trace-table*
then obtain *f* **where** *deg-f: degree f < clength*
and *table: trace-table = map (poly f) eval-domain*
unfolding *trace-table-low-degree-def* **by** *blast*
have *degree f ≤ clength - 1*
using *deg-f* **by** *simp*
then show *fri-table-low-degree-on (clength - 1) eval-domain trace-table*
unfolding *fri-table-low-degree-on-def*
using *table* **by** *blast*

next

assume *fri-table-low-degree-on (clength - 1) eval-domain trace-table*
then obtain *p* **where** *deg-p: degree p ≤ clength - 1*
and *table: trace-table = map (poly p) eval-domain*
unfolding *fri-table-low-degree-on-def* **by** *blast*
have *degree p < clength*
using *deg-p clength-pos* **by** *simp*
then show *trace-table-low-degree trace-table*
unfolding *trace-table-low-degree-def*
using *table* **by** *blast*

qed

lemma *composition-table-low-degree-iff-fri-table-low-degree-on-eval-domain*:

composition-table-low-degree d composition-table $\longleftrightarrow$
fri-table-low-degree-on d eval-domain composition-table
unfolding *composition-table-low-degree-def fri-table-low-degree-on-def*
by *simp*

lemma *composition-table-low-degree-mono*:

assumes *d ≤ d'*
and *composition-table-low-degree d composition-table*
shows *composition-table-low-degree d' composition-table*

proof —

from *assms(2)* **obtain** *q* **where** *deg-q: degree q ≤ d*
and *table: composition-table = map (poly q) eval-domain*
unfolding *composition-table-low-degree-def* **by** *blast*
have *degree q ≤ d'*
using *deg-q assms(1)* **by** *simp*

```

then show ?thesis
  unfolding composition-table-low-degree-def
  using table by blast
qed

lemma composition-table-not-low-degree-mono:
  assumes  $d \leq d'$ 
  and  $\neg$  composition-table-low-degree  $d'$  composition-table
  shows  $\neg$  composition-table-low-degree  $d$  composition-table
  using composition-table-low-degree-mono[OF assms(1)] assms(2) by blast

lemma trace-table-not-low-degree-iff-not-fri-table-low-degree-on-eval-domain:
   $\neg$  trace-table-low-degree trace-table  $\longleftrightarrow$ 
   $\neg$  fri-table-low-degree-on (clength - 1) eval-domain trace-table
  by (simp add: trace-table-low-degree-iff-fri-table-low-degree-on-eval-domain)

lemma composition-table-not-low-degree-iff-not-fri-table-low-degree-on-eval-domain:
   $\neg$  composition-table-low-degree  $d$  composition-table  $\longleftrightarrow$ 
   $\neg$  fri-table-low-degree-on  $d$  eval-domain composition-table
  by (simp add:
    composition-table-low-degree-iff-fri-table-low-degree-on-eval-domain)

lemma accepted-fri-opening-transcript-trace-generic-partial-evidence:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  shows
    generic-fri-partial-evidence trace-table-low-degree trace-table
    (clength - 1) trace-roots trace-bs trace-final query-idxs
    trace-round-layers
proof -
  have count:
    length trace-roots = fri-round-count-for-degree-bound (clength - 1)
  using accepted-fri-opening-transcript-algebraic-round-counts(1)
  [OF fri-openings]
  by (simp add: trace-fri-algebraic-round-count-def)
  have len-bs: length trace-bs = length trace-roots
  using accepted-fri-opening-transcript-shapes[OF fri-openings] by blast
  have layers:
    fri-all-round-layer-evidence trace-roots trace-bs query-idxs
    trace-round-layers
  by (rule accepted-fri-opening-transcript-trace-all-round-layer-evidence
    [OF fri-openings])
  show ?thesis
  unfolding generic-fri-partial-evidence-def
  using count len-bs layers by simp
qed

```

lemma *accepted-fri-opening-transcript-composition-generic-partial-evidence:*

assumes *fri-openings:*

*accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers*

shows

*generic-fri-partial-evidence (composition-table-low-degree (to-nat dg))
composition-table (to-nat dg) composition-roots composition-bs
composition-final query-idxs composition-round-layers*

proof —

have *count:*

length composition-roots = fri-round-count-for-degree-bound (to-nat dg)

using *accepted-fri-opening-transcript-algebraic-round-counts(3)
[OF fri-openings]*

by (*simp add: composition-fri-algebraic-round-count-def*)

have *len-bs: length composition-bs = length composition-roots*

using *accepted-fri-opening-transcript-shapes[OF fri-openings]* **by** *blast*

have *layers:*

*fri-all-round-layer-evidence composition-roots composition-bs query-idxs
composition-round-layers*

by (*rule*

*accepted-fri-opening-transcript-composition-all-round-layer-evidence
[OF fri-openings]*)

show *?thesis*

unfolding *generic-fri-partial-evidence-def*

using *count len-bs layers* **by** *simp*

qed

lemma *trace-fri-partial-candidate-opening-evidence-generic-partial:*

assumes *evidence:*

*trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table*

shows

*generic-fri-partial-evidence trace-table-low-degree trace-table
(clength — 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers*

proof —

have *fri-openings:*

*accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers*

using *trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence]* .

show *?thesis*

by (*rule accepted-fri-opening-transcript-trace-generic-partial-evidence
[OF fri-openings]*)

qed

lemma *composition-fri-partial-candidate-opening-evidence-generic-partial:*

assumes *evidence:*

composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table

shows

generic-fri-partial-evidence
(composition-table-low-degree (to-nat fri-dg))
composition-table (to-nat fri-dg) opening-composition-roots
composition-bs composition-final fri-query-idxs
composition-round-layers

proof —

have *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers

using *composition-fri-partial-candidate-opening-evidenceD(1)*
[OF evidence] .

show *?thesis*

by (*rule*
accepted-fri-opening-transcript-composition-generic-partial-evidence
[OF fri-openings])

qed

lemma *trace-fri-partial-candidate-opening-evidence-not-low-on-eval-domain:*

assumes *evidence:*

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

shows $\neg$ *fri-table-low-degree-on (clength - 1) eval-domain trace-table*

proof —

have *partial:*

trace-fri-partial-candidate-evidence s out fr candidate-query-idxs
trace-openings trace-table

using *trace-fri-partial-candidate-opening-evidenceD(2)[OF evidence]* .

have $\neg$ *trace-table-low-degree trace-table*

using *trace-fri-partial-candidate-evidenceD(3)[OF partial]* .

then show *?thesis*

using *trace-table-not-low-degree-iff-not-fri-table-low-degree-on-eval-domain*
by *simp*

qed

lemma *composition-fri-partial-candidate-opening-evidence-not-low-declared:*

assumes *evidence:*

composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table

and *degree-bound: to-nat fri-dg ≤ maxDegree*

shows $\neg$ *composition-table-low-degree (to-nat fri-dg) composition-table*

proof —

have *partial:*

composition-fri-partial-candidate-evidence s out fr f-fri-roots
f-final as dg composition-fri-roots final trace-query-idxs
trace-openings composition-query-idxs composition-openings
trace-table composition-table

by (rule *composition-fri-partial-candidate-opening-evidenceD(2)*
 $[OF\ evidence]$)

have *not-low-max:*

$\neg$ *composition-table-low-degree maxDegree composition-table*

by (rule *composition-fri-partial-candidate-evidenceD(5)* $[OF\ partial]$)

show *?thesis*

by (rule *composition-table-not-low-degree-mono*
 $[OF\ degree-bound\ not-low-max]$)

qed

lemma *composition-fri-partial-candidate-opening-evidence-not-low-declared-on-eval-domain:*

assumes *evidence:*

composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table

and *degree-bound: to-nat fri-dg ≤ maxDegree*

shows $\neg$ *fri-table-low-degree-on (to-nat fri-dg) eval-domain*
composition-table

proof —

have $\neg$ *composition-table-low-degree (to-nat fri-dg) composition-table*

by (rule *composition-fri-partial-candidate-opening-evidence-not-low-declared*
 $[OF\ evidence\ degree-bound]$)

then show *?thesis*

using *composition-table-not-low-degree-iff-not-fri-table-low-degree-on-eval-domain*

by *simp*

qed

lemma *trace-fri-partial-candidate-opening-evidence-canonical-bad-candidate:*

assumes *evidence:*

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

and chain:

generic-fri-canonical-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers bad

shows

generic-fri-bad-candidate trace-table-low-degree trace-table
(clength - 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers bad

proof —

have $\neg$ *trace-table-low-degree trace-table*

using *trace-fri-partial-candidate-opening-evidenceD(2)[OF evidence]*
trace-fri-partial-candidate-evidenceD(3)

by *blast*

then show *?thesis*

by (rule *generic-fri-bad-candidate-from-canonical-layer-chain*
 $[OF\ chain]$)

qed

lemma *composition-fri-partial-candidate-opening-evidence-canonical-bad-candidate:*

assumes *evidence:*

composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table

and *degree-bound: to-nat fri-dg $\leq$ maxDegree*

and chain:

generic-fri-canonical-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers bad

shows

generic-fri-bad-candidate
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers bad

proof —

have $\neg$ *composition-table-low-degree (to-nat fri-dg) composition-table*

by (rule *composition-fri-partial-candidate-opening-evidence-not-low-declared*
 $[OF\ evidence\ degree-bound]$)


```

then show ?thesis
  by (rule generic-fri-bad-candidate-from-canonical-layer-chain
    [OF chain])
qed

lemma trace-fri-multiround-bad-sets-as-generic:
  trace-fri-multiround-bad-sets bad trace-table =
    generic-fri-bad-challenge-lists trace-fri-algebraic-round-count
      (bad trace-table)
unfolding trace-fri-multiround-bad-sets-def
  generic-fri-bad-challenge-lists-def
by simp

lemma composition-fri-multiround-bad-sets-as-generic:
  composition-fri-multiround-bad-sets bad dg composition-table =
    generic-fri-bad-challenge-lists
      (composition-fri-algebraic-round-count dg)
      (bad dg composition-table)
unfolding composition-fri-multiround-bad-sets-def
  generic-fri-bad-challenge-lists-def
by simp

end

end

```

```

theory Soundness-FRI-Internal-Bounds
  imports Soundness-FRI-Generic-Low-Degree
begin

```

Internal concrete FRI challenge-list bounds.

These lemmas deliberately avoid the symbolic parameters *trace-fri-error* and *composition-fri-error*. They expose the concrete bad-set fractions obtained from the generic FRI bad challenge-list interface. Later public-route work can either use these concrete terms directly, or relate the symbolic error parameters to these terms via an explicitly chosen FRI soundness interface.

```

context soundness
begin

```

```

definition fri-bad-challenge-round-mass
  :: nat  $\Rightarrow$  (nat  $\Rightarrow$  nat)  $\Rightarrow$  nat
where
  fri-bad-challenge-round-mass n bd =
    ( $\sum i \in \{..<n\}. \text{CARD}('f) \wedge i * \text{bd } i * \text{CARD}('f) \wedge (n - \text{Suc } i)$ )

```

```

definition fri-bad-challenge-round-fraction
  :: nat  $\Rightarrow$  (nat  $\Rightarrow$  nat)  $\Rightarrow$  prob

```

where

$fri\text{-}bad\text{-}challenge\text{-}round\text{-}fraction\ n\ bd =$
 $nnreal\ (fri\text{-}bad\text{-}challenge\text{-}round\text{-}mass\ n\ bd) / nnreal\ (CARD('f) \wedge n)$

definition $fri\text{-}one\text{-}step\text{-}disagreement\text{-}round\text{-}mass$

$:: nat \Rightarrow (nat \Rightarrow nat) \Rightarrow nat$

where

$fri\text{-}one\text{-}step\text{-}disagreement\text{-}round\text{-}mass\ n\ len =$
 $fri\text{-}bad\text{-}challenge\text{-}round\text{-}mass\ n\ (\lambda i. len\ i\ div\ 2)$

definition $fri\text{-}one\text{-}step\text{-}disagreement\text{-}round\text{-}fraction$

$:: nat \Rightarrow (nat \Rightarrow nat) \Rightarrow prob$

where

$fri\text{-}one\text{-}step\text{-}disagreement\text{-}round\text{-}fraction\ n\ len =$
 $fri\text{-}bad\text{-}challenge\text{-}round\text{-}fraction\ n\ (\lambda i. len\ i\ div\ 2)$

lemma $fri\text{-}one\text{-}step\text{-}disagreement\text{-}round\text{-}fraction\text{-}unfold:$

$fri\text{-}one\text{-}step\text{-}disagreement\text{-}round\text{-}fraction\ n\ len =$
 $nnreal$
 $(\sum i \in \{..<n\}.$
 $CARD('f) \wedge i * (len\ i\ div\ 2) * CARD('f) \wedge (n - Suc\ i)) /$
 $nnreal\ (CARD('f) \wedge n)$

unfolding $fri\text{-}one\text{-}step\text{-}disagreement\text{-}round\text{-}fraction\text{-}def$

$fri\text{-}bad\text{-}challenge\text{-}round\text{-}fraction\text{-}def$

$fri\text{-}bad\text{-}challenge\text{-}round\text{-}mass\text{-}def$

by $simp$

lemma $fri\text{-}one\text{-}step\text{-}disagreement\text{-}round\text{-}fraction\text{-}reft\text{-}bound:$

$nnreal$
 $(\sum i \in \{..<n\}.$
 $CARD('f) \wedge i * (len\ i\ div\ 2) * CARD('f) \wedge (n - Suc\ i)) /$
 $nnreal\ (CARD('f) \wedge n)$
 $\leq fri\text{-}one\text{-}step\text{-}disagreement\text{-}round\text{-}fraction\ n\ len$
by $(simp\ add: fri\text{-}one\text{-}step\text{-}disagreement\text{-}round\text{-}fraction\text{-}unfold)$

definition $trace\text{-}fri\text{-}canonical\text{-}layer\text{-}len :: nat \Rightarrow nat$

where

$trace\text{-}fri\text{-}canonical\text{-}layer\text{-}len\ i =$
 $fri\text{-}layer\text{-}lengths\ (fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound\ (clength - 1))$
 $(clength * scale) ! i$

definition $composition\text{-}fri\text{-}canonical\text{-}layer\text{-}len :: 'f \Rightarrow nat \Rightarrow nat$

where

$composition\text{-}fri\text{-}canonical\text{-}layer\text{-}len\ dg\ i =$
 $fri\text{-}layer\text{-}lengths\ (fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound\ (to\text{-}nat\ dg))$
 $(clength * scale) ! i$

definition $concrete\text{-}trace\text{-}fri\text{-}bad\text{-}challenge\text{-}fraction :: prob$

where

$\text{concrete-trace-fri-bad-challenge-fraction} =$
 $\text{fri-one-step-disagreement-round-fraction}$
 $(\text{fri-round-count-for-degree-bound } (\text{clength} - 1))$
 $\text{trace-fri-canonical-layer-len}$

definition $\text{concrete-composition-fri-bad-challenge-fraction} :: \text{prob}$
where

$\text{concrete-composition-fri-bad-challenge-fraction} =$
 Max
 $((\lambda dg :: 'f.$
 $\text{fri-one-step-disagreement-round-fraction}$
 $(\text{fri-round-count-for-degree-bound } (\text{to-nat } dg))$
 $(\text{composition-fri-canonical-layer-len } dg)) \text{ ' UNIV})$

lemma $\text{concrete-trace-fri-bad-challenge-fraction-eq}$:

$\text{fri-one-step-disagreement-round-fraction}$
 $(\text{fri-round-count-for-degree-bound } (\text{clength} - 1))$
 $\text{trace-fri-canonical-layer-len}$
 $= \text{concrete-trace-fri-bad-challenge-fraction}$
by ($\text{simp add: concrete-trace-fri-bad-challenge-fraction-def}$)

lemma $\text{concrete-composition-fri-bad-challenge-fraction-ge}$:

$\text{fri-one-step-disagreement-round-fraction}$
 $(\text{fri-round-count-for-degree-bound } (\text{to-nat } dg))$
 $(\text{composition-fri-canonical-layer-len } dg)$
 $\leq \text{concrete-composition-fri-bad-challenge-fraction}$
unfolding $\text{concrete-composition-fri-bad-challenge-fraction-def}$
by (rule Max-ge) simp-all

lemma $\text{fri-bad-challenge-round-fraction-bound-from-round-bounds}$:

assumes local-bound :
 $\bigwedge i \text{ prefix. } i < n \implies \text{length prefix} = i \implies$
 $\text{card } (\text{bad } i \text{ prefix}) \leq \text{bd } i$
shows $\text{nnreal } (\text{card } (\text{generic-fri-bad-challenge-lists } n \text{ bad})) /$
 $\text{nnreal } (\text{CARD}('f) ^ n)$
 $\leq \text{fri-bad-challenge-round-fraction } n \text{ bd}$
unfolding $\text{fri-bad-challenge-round-fraction-def}$
 $\text{fri-bad-challenge-round-mass-def}$
by ($\text{rule nnreal-nat-divide-right-mono}$)
 $(\text{rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds}$
 $[\text{OF local-bound}])$

lemma $\text{generic-fri-bad-challenge-lists-card-bound-from-one-step-disagreement-subset}$:

assumes local-subset :
 $\bigwedge i \text{ prefix. } i < n \implies \text{length prefix} = i \implies$
 $\text{bad } i \text{ prefix} \subseteq$
 $\text{fri-one-step-disagreement-challenges}$
 $(\text{len } i) (\text{pw } i) (\text{layer } i \text{ prefix}) (\text{claimed } i \text{ prefix}) (\text{domains } i \text{ prefix})$
shows $\text{card } (\text{generic-fri-bad-challenge-lists } n \text{ bad}) \leq$

$(\sum i \in \{..<n\}.$
 $CARD('f) \wedge i * (len\ i\ div\ 2) * CARD('f) \wedge (n - Suc\ i))$
proof (rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds)
fix $i :: nat$ **and** $prefix :: 'f\ list$
assume $i-bound: i < n$ **and** $prefix-len: length\ prefix = i$
have $card\ (bad\ i\ prefix) \leq$
 $card$
 $(fri-one-step-disagreement-challenges$
 $(len\ i)\ (pw\ i)\ (layer\ i\ prefix)\ (claimed\ i\ prefix)\ (domains\ i\ prefix))$
by (rule card-mono) (simp-all add: local-subset[OF i-bound prefix-len])
also have $\dots \leq len\ i\ div\ 2$
by (rule fri-one-step-disagreement-challenges-card-bound-local)
finally show $card\ (bad\ i\ prefix) \leq len\ i\ div\ 2$.
qed

lemma generic-fri-bad-challenge-lists-bounded-by-from-one-step-disagreement-subset:
assumes local-subset:
 $\bigwedge i\ prefix. i < n \implies length\ prefix = i \implies$
 $bad\ i\ prefix \subseteq$
 $fri-one-step-disagreement-challenges$
 $(len\ i)\ (pw\ i)\ (layer\ i\ prefix)\ (claimed\ i\ prefix)\ (domains\ i\ prefix)$
shows fri-bad-challenge-lists-bounded-by n
 $(generic-fri-bad-challenge-lists\ n\ bad)$
 $(\sum i \in \{..<n\}.$
 $CARD('f) \wedge i * (len\ i\ div\ 2) * CARD('f) \wedge (n - Suc\ i))$
by (rule generic-fri-bad-challenge-lists-bounded-byI)
 $(rule\ generic-fri-bad-challenge-lists-card-bound-from-one-step-disagreement-subset$
 $[OF\ local-subset])$

lemma fri-bad-challenge-round-fraction-bound-from-one-step-disagreement-subset:
assumes local-subset:
 $\bigwedge i\ prefix. i < n \implies length\ prefix = i \implies$
 $bad\ i\ prefix \subseteq$
 $fri-one-step-disagreement-challenges$
 $(len\ i)\ (pw\ i)\ (layer\ i\ prefix)\ (claimed\ i\ prefix)\ (domains\ i\ prefix)$
shows $nnreal\ (card\ (generic-fri-bad-challenge-lists\ n\ bad)) /$
 $nnreal\ (CARD('f) \wedge n)$
 $\leq fri-one-step-disagreement-round-fraction\ n\ len$
unfolding fri-one-step-disagreement-round-fraction-def
 $fri-bad-challenge-round-fraction-def$
 $fri-one-step-disagreement-round-mass-def$
 $fri-bad-challenge-round-mass-def$
by (rule nnreal-nat-divide-right-mono)
 $(rule\ generic-fri-bad-challenge-lists-card-bound-from-one-step-disagreement-subset$
 $[OF\ local-subset])$

lemma wp-verify-monad-trace-generic-fri-bad-challenge-list-bound:
assumes future: trace-fri-future-fresh s
shows

$wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}challenge\text{-}list\text{-}set\text{-}hit\ s$
 $(generic\text{-}fri\text{-}bad\text{-}challenge\text{-}lists\ trace\text{-}fri\text{-}algebraic\text{-}round\text{-}count\ bad))$
 s
 $\leq$
 $nnreal$
 $(card$
 $(generic\text{-}fri\text{-}bad\text{-}challenge\text{-}lists\ trace\text{-}fri\text{-}algebraic\text{-}round\text{-}count$
 $bad)) /$
 $nnreal\ (CARD('f) \wedge ceil\text{-}log\ clength)$
proof (rule $wp\text{-}verify\text{-}monad\text{-}trace\text{-}fri\text{-}challenge\text{-}list\text{-}set\text{-}bound[OF\ future]$)
show $generic\text{-}fri\text{-}bad\text{-}challenge\text{-}lists\ trace\text{-}fri\text{-}algebraic\text{-}round\text{-}count\ bad$
 $\subseteq fri\text{-}challenge\text{-}space\ (ceil\text{-}log\ clength)$
using $generic\text{-}fri\text{-}bad\text{-}challenge\text{-}lists\text{-}subset$
by (simp add: $trace\text{-}fri\text{-}algebraic\text{-}round\text{-}count\text{-}eq$)
qed

lemma $wp\text{-}verify\text{-}monad\text{-}trace\text{-}generic\text{-}fri\text{-}bad\text{-}challenge\text{-}list\text{-}bound\text{-}from\text{-}card$:
assumes $future$: $trace\text{-}fri\text{-}future\text{-}fresh\ s$
and $card\text{-}bound$:
 $card\ (generic\text{-}fri\text{-}bad\text{-}challenge\text{-}lists\ trace\text{-}fri\text{-}algebraic\text{-}round\text{-}count$
 $bad) \leq bd$
shows
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}challenge\text{-}list\text{-}set\text{-}hit\ s$
 $(generic\text{-}fri\text{-}bad\text{-}challenge\text{-}lists\ trace\text{-}fri\text{-}algebraic\text{-}round\text{-}count\ bad))$
 s
 $\leq nnreal\ bd / nnreal\ (CARD('f) \wedge ceil\text{-}log\ clength)$
proof –
have $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}challenge\text{-}list\text{-}set\text{-}hit\ s$
 $(generic\text{-}fri\text{-}bad\text{-}challenge\text{-}lists\ trace\text{-}fri\text{-}algebraic\text{-}round\text{-}count\ bad))$
 s
 $\leq$
 $nnreal$
 $(card$
 $(generic\text{-}fri\text{-}bad\text{-}challenge\text{-}lists\ trace\text{-}fri\text{-}algebraic\text{-}round\text{-}count$
 $bad)) /$
 $nnreal\ (CARD('f) \wedge ceil\text{-}log\ clength)$
by (rule $wp\text{-}verify\text{-}monad\text{-}trace\text{-}generic\text{-}fri\text{-}bad\text{-}challenge\text{-}list\text{-}bound$
 $[OF\ future]$)
also have $\dots \leq nnreal\ bd / nnreal\ (CARD('f) \wedge ceil\text{-}log\ clength)$
by (rule $nnreal\text{-}nat\text{-}divide\text{-}right\text{-}mono[OF\ card\text{-}bound]$)
finally show $?thesis$.
qed

lemma $wp\text{-}verify\text{-}monad\text{-}trace\text{-}generic\text{-}fri\text{-}bad\text{-}challenge\text{-}list\text{-}bound\text{-}from\text{-}rounds$:
assumes $future$: $trace\text{-}fri\text{-}future\text{-}fresh\ s$
and $local\text{-}bound$:

$\bigwedge i \text{ prefix. } i < \text{trace-fri-algebraic-round-count} \implies$
 $\text{length prefix} = i \implies \text{card (bad } i \text{ prefix)} \leq \text{bd } i$

shows

$\text{wp-event verify-monad}$
 $(\text{trace-fri-challenge-list-set-hit } s$
 $(\text{generic-fri-bad-challenge-lists trace-fri-algebraic-round-count bad}))$
 s
 $\leq$
 nnreal
 $(\sum i \in \{..<\text{trace-fri-algebraic-round-count}\}.$
 $\text{CARD('f)} \wedge i * \text{bd } i *$
 $\text{CARD('f)} \wedge (\text{trace-fri-algebraic-round-count} - \text{Suc } i)) /$
 $\text{nnreal (CARD('f)} \wedge \text{ceil-log clength})$

by (rule wp-verify-monad-trace-generic-fri-bad-challenge-list-bound-from-card
[OF future])
(rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds
[OF local-bound])

lemma wp-verify-monad-composition-generic-fri-bad-challenge-list-bound:

assumes future: composition-fri-future-fresh s

and bound:

$\bigwedge dg.$
 nnreal
 $(\text{card}$
 $(\text{generic-fri-bad-challenge-lists}$
 $(\text{composition-fri-algebraic-round-count } dg) (\text{bad } dg))) /$
 $\text{nnreal (CARD('f)} \wedge \text{ceil-log (to-nat } dg + 1)) \leq C$

shows

$\text{wp-event verify-monad}$
 $(\text{composition-fri-challenge-list-set-hit } s$
 $(\lambda dg. \text{generic-fri-bad-challenge-lists}$
 $(\text{composition-fri-algebraic-round-count } dg) (\text{bad } dg)))$
 $s \leq C$

proof (rule wp-verify-monad-composition-fri-challenge-list-set-bound
[OF future])

fix dg

show generic-fri-bad-challenge-lists
 $(\text{composition-fri-algebraic-round-count } dg) (\text{bad } dg)$
 $\subseteq \text{fri-challenge-space (ceil-log (to-nat } dg + 1))$

using generic-fri-bad-challenge-lists-subset

by (simp add: composition-fri-algebraic-round-count-eq)

show nnreal
 $(\text{card}$
 $(\text{generic-fri-bad-challenge-lists}$
 $(\text{composition-fri-algebraic-round-count } dg) (\text{bad } dg))) /$
 $\text{nnreal (CARD('f)} \wedge \text{ceil-log (to-nat } dg + 1)) \leq C$

by (rule bound)

qed

lemma *wp-verify-monad-composition-generic-fri-bad-challenge-list-bound-from-card:*

assumes *future: composition-fri-future-fresh s*

and *card-bound:*

$\bigwedge dg. \text{card}$
 $(\text{generic-fri-bad-challenge-lists}$
 $(\text{composition-fri-algebraic-round-count } dg) (\text{bad } dg)) \leq bd \ dg$

and *fraction-bound:*

$\bigwedge dg. \text{nnreal } (bd \ dg) /$
 $\text{nnreal } (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq C$

shows

wp-event verify-monad
 $(\text{composition-fri-challenge-list-set-hit } s$
 $(\lambda dg. \text{generic-fri-bad-challenge-lists}$
 $(\text{composition-fri-algebraic-round-count } dg) (\text{bad } dg)))$
 $s \leq C$

proof (rule *wp-verify-monad-composition-generic-fri-bad-challenge-list-bound*
 $[OF \text{ future}]$)

fix *dg*

have *nnreal*

$(\text{card}$
 $(\text{generic-fri-bad-challenge-lists}$
 $(\text{composition-fri-algebraic-round-count } dg) (\text{bad } dg))) /$
 $\text{nnreal } (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1))$
 $\leq \text{nnreal } (bd \ dg) /$
 $\text{nnreal } (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1))$
by (rule *nnreal-nat-divide-right-mono* $[OF \text{ card-bound}]$)

also have $\dots \leq C$

by (rule *fraction-bound*)

finally show *nnreal*

$(\text{card}$
 $(\text{generic-fri-bad-challenge-lists}$
 $(\text{composition-fri-algebraic-round-count } dg) (\text{bad } dg))) /$
 $\text{nnreal } (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq C$.

qed

lemma *wp-verify-monad-composition-generic-fri-bad-challenge-list-bound-from-rounds:*

assumes *future: composition-fri-future-fresh s*

and *local-bound:*

$\bigwedge dg \ i \text{ prefix}. i < \text{composition-fri-algebraic-round-count } dg \implies$
 $\text{length prefix} = i \implies \text{card } (\text{bad } dg \ i \text{ prefix}) \leq bd \ dg \ i$

and *fraction-bound:*

$\bigwedge dg. \text{nnreal}$
 $(\sum i \in \{..<\text{composition-fri-algebraic-round-count } dg\}.$
 $CARD('f) \wedge i * bd \ dg \ i *$
 $CARD('f) \wedge (\text{composition-fri-algebraic-round-count } dg - \text{Suc } i)) /$
 $\text{nnreal } (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq C$

shows

wp-event verify-monad
 $(\text{composition-fri-challenge-list-set-hit } s$

```

      (λdg. generic-fri-bad-challenge-lists
        (composition-fri-algebraic-round-count dg) (bad dg)))
    s ≤ C
proof (rule
  wp-verify-monad-composition-generic-fri-bad-challenge-list-bound-from-card
  [OF future - fraction-bound])
fix dg
show card
  (generic-fri-bad-challenge-lists
    (composition-fri-algebraic-round-count dg) (bad dg))
  ≤
  (∑ i ∈ {..composition-fri-algebraic-round-count dg}.
    CARD('f) ^ i * bd dg i *
    CARD('f) ^ (composition-fri-algebraic-round-count dg - Suc i))
  by (rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds)
      (rule local-bound[of - dg], simp-all)
qed

end

end

```

```

theory Soundness-FRI-Refined-Reachable
imports Soundness-FRI-Internal-Bounds
begin

```

Refined reachable FRI layer-chain targets.

Broad reachable partial-candidate events quantify full base tables compatible with sampled initial openings. Sampled base openings alone do not determine the committed FRI layer chain, so this theory introduces the narrower algebraic cover used by the next proof layer.

Its witnesses include the accepted FRI opening transcript and a canonical FRI layer chain. The cover is internal proof infrastructure; it does not change the protocol, verifier, Merkle interface, or public theorem assumptions.

```

context soundness
begin

```

definition *trace-fri-bad-with-refined-layer-chain*

```

:: ('f, 'a) protocol-channel-scheme ⇒
  ('f list ⇒ nat ⇒ 'f list ⇒ 'f set) ⇒
  (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool

```

where

```

trace-fri-bad-with-refined-layer-chain s bad out ⟷
  (∃ trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table doms layers.

```


trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table $\wedge$
generic-fri-canonical-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers (bad trace-table))

definition *composition-fri-bad-with-refined-layer-chain*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-bad-with-refined-layer-chain s bad out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers.
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-canonical-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
(bad fri-dg composition-table))

definition *trace-fri-reachable-missing-refined-layer-chain*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-reachable-missing-refined-layer-chain s bad out $\longleftrightarrow$
trace-fri-bad-with-reachable-partial-candidate s out $\wedge$
 $\neg \text{ trace-fri-bad-with-refined-layer-chain } s \text{ bad out}$

definition *composition-fri-reachable-missing-refined-layer-chain*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-reachable-missing-refined-layer-chain s bad out

$\longleftrightarrow$
composition-fri-bad-with-reachable-partial-candidate *s out* $\wedge$
 $\neg$ *composition-fri-bad-with-refined-layer-chain* *s bad out*

definition *trace-fri-bad-with-sampled-layer-chain*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-bad-with-sampled-layer-chain *s out* $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{ trace-table doms layers.}$
trace-fri-partial-candidate-opening-evidence *s out* *trace-roots*
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table $\wedge$
generic-fri-sampled-layer-chain-evidence *trace-table-low-degree*
trace-table (*clength* - 1) *trace-roots trace-bs trace-final*
fri-query-idxs trace-round-layers doms layers)

definition *composition-fri-bad-with-sampled-layer-chain*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-bad-with-sampled-layer-chain *s out* $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{ opening-composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{ f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table doms layers.}$
composition-fri-partial-candidate-opening-evidence *s out* *trace-roots*
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-sampled-layer-chain-evidence
 $(\text{composition-table-low-degree } (\text{to-nat fri-dg})) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs composition-round-layers doms layers})$

definition *trace-fri-reachable-missing-sampled-layer-chain*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-reachable-missing-sampled-layer-chain } s \text{ out} \longleftrightarrow$
 $\text{trace-fri-bad-with-reachable-partial-candidate } s \text{ out} \wedge$
 $\neg \text{trace-fri-bad-with-sampled-layer-chain } s \text{ out}$

definition *composition-fri-reachable-missing-sampled-layer-chain*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-reachable-missing-sampled-layer-chain } s \text{ out} \longleftrightarrow$
 $\text{composition-fri-bad-with-reachable-partial-candidate } s \text{ out} \wedge$
 $\neg \text{composition-fri-bad-with-sampled-layer-chain } s \text{ out}$

definition *trace-fri-sampled-layer-chain-cover*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow \text{bool}$

where

$\text{trace-fri-sampled-layer-chain-cover } s \text{ bad} \longleftrightarrow$
 $(\forall \text{ out trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
 $\text{trace-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \longrightarrow$
 $\text{generic-fri-sampled-layer-chain-evidence trace-table-low-degree}$
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers} \longrightarrow$
 $\text{trace-bs} \in \text{trace-fri-multiround-bad-sets bad trace-table})$

definition *composition-fri-sampled-layer-chain-cover*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow \text{bool}$

where

$\text{composition-fri-sampled-layer-chain-cover } s \text{ bad} \longleftrightarrow$
 $(\forall \text{ out trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table doms layers.}$
 $\text{composition-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings trace-table}$
 $\text{composition-table} \longrightarrow$

generic-fri-sampled-layer-chain-evidence
 (composition-table-low-degree (to-nat fri-dg)) composition-table
 (to-nat fri-dg) opening-composition-roots composition-bs
 composition-final fri-query-idxs composition-round-layers doms layers
 $\longrightarrow$
 composition-bs $\in$
 composition-fri-multiround-bad-sets bad fri-dg composition-table)

definition *trace-fri-bad-with-full-cover-layer-chain*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \text{ list} \Rightarrow \text{nat} \Rightarrow f \text{ list} \Rightarrow f \text{ set}) \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-bad-with-full-cover-layer-chain s bad out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
trace-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table $\wedge$
generic-fri-sampled-layer-chain-full-cover trace-table-low-degree
 trace-table (clength - 1) trace-roots trace-bs trace-final
 fri-query-idxs trace-round-layers doms layers (bad trace-table))

definition *composition-fri-bad-with-full-cover-layer-chain*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \Rightarrow f \text{ list} \Rightarrow \text{nat} \Rightarrow f \text{ list} \Rightarrow f \text{ set}) \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-bad-with-full-cover-layer-chain s bad out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table doms layers.}$
composition-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final fri-dg opening-composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr f-fri-roots f-final as dg
 composition-fri-roots final trace-query-idxs trace-openings
 composition-query-idxs composition-openings trace-table
 composition-table $\wedge$
generic-fri-sampled-layer-chain-full-cover
 (composition-table-low-degree (to-nat fri-dg)) composition-table
 (to-nat fri-dg) opening-composition-roots composition-bs

composition-final fri-query-idxs composition-round-layers doms layers
(bad fri-dg composition-table))

definition *trace-fri-sampled-missing-full-cover*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-sampled-missing-full-cover s bad out $\longleftrightarrow$
trace-fri-bad-with-sampled-layer-chain s out $\wedge$
 $\neg$ *trace-fri-bad-with-full-cover-layer-chain s bad out*

definition *composition-fri-sampled-missing-full-cover*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-missing-full-cover s bad out $\longleftrightarrow$
composition-fri-bad-with-sampled-layer-chain s out $\wedge$
 $\neg$ *composition-fri-bad-with-full-cover-layer-chain s bad out*

lemma *trace-fri-sampled-layer-chain-cover-from-full-cover:*

fixes *s* $:: ('f, 'a) \text{ protocol-channel-scheme}$

fixes *bad* $:: 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}$

assumes *full-cover:*

$\bigwedge \text{out trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$

trace-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \implies$

generic-fri-sampled-layer-chain-evidence trace-table-low-degree
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers} \implies$
generic-fri-sampled-layer-chain-full-cover trace-table-low-degree
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers (bad trace-table)}$

shows *trace-fri-sampled-layer-chain-cover s bad*

proof (*unfold trace-fri-sampled-layer-chain-cover-def, intro allI impI*)

fix *out trace-roots trace-bs trace-final dg composition-roots*
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers}$

assume *evidence:*

trace-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{trace-bs trace-final dg composition-roots composition-bs}$

```

    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
assume chain:
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers
have full:
    generic-fri-sampled-layer-chain-full-cover trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers (bad trace-table)
    by (rule full-cover[OF evidence chain])
have not-low:  $\neg$  trace-table-low-degree trace-table
    using trace-fri-partial-candidate-opening-evidenceD(2)[OF evidence]
    trace-fri-partial-candidate-evidenceD(3)
    by blast
have bad-candidate:
    generic-fri-bad-candidate trace-table-low-degree trace-table
    (clength - 1) trace-roots trace-bs trace-final fri-query-idxs
    trace-round-layers (bad trace-table)
    by (rule generic-fri-bad-candidate-from-sampled-layer-chain-full-cover
    [OF full not-low])
have bad-generic:
    trace-bs  $\in$  generic-fri-bad-challenge-lists (length trace-bs)
    (bad trace-table)
    by (rule generic-fri-bad-candidateD(3)[OF bad-candidate])
have partial:
    generic-fri-partial-evidence trace-table-low-degree trace-table
    (clength - 1) trace-roots trace-bs trace-final fri-query-idxs
    trace-round-layers
    by (rule generic-fri-bad-candidateD(1)[OF bad-candidate])
have len-trace:
    length trace-bs = trace-fri-algebraic-round-count
    using generic-fri-partial-evidence-shapes(1,2)[OF partial]
    by (simp add: trace-fri-algebraic-round-count-def)
show trace-bs  $\in$  trace-fri-multiround-bad-sets bad trace-table
    using bad-generic len-trace
    by (simp add: trace-fri-multiround-bad-sets-as-generic)
qed

```

lemma composition-fri-sampled-layer-chain-cover-from-full-cover:

```

fixes s :: ('f, 'a) protocol-channel-scheme
fixes bad :: 'f  $\Rightarrow$  'f list  $\Rightarrow$  nat  $\Rightarrow$  'f list  $\Rightarrow$  'f set
assumes full-cover:
     $\bigwedge$  out trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs

```

composition-openings trace-table composition-table doms layers.
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\Rightarrow$
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms
layers $\Rightarrow$
generic-fri-sampled-layer-chain-full-cover
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms
layers (bad fri-dg composition-table)
shows *composition-fri-sampled-layer-chain-cover s bad*
proof (*unfold composition-fri-sampled-layer-chain-cover-def,*
intro allI impI)
fix *out trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
assume *evidence:*
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
assume *chain:*
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms
layers
have *full:*
generic-fri-sampled-layer-chain-full-cover
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms
layers (bad fri-dg composition-table)
by (*rule full-cover[OF evidence chain]*)
have *not-low-on:*

```

  ¬ fri-table-low-degree-on (to-nat fri-dg) eval-domain
    composition-table
  using full unfolding generic-fri-sampled-layer-chain-full-cover-def
  by blast
have not-low:
  ¬ composition-table-low-degree (to-nat fri-dg) composition-table
  using not-low-on
    composition-table-not-low-degree-iff-not-fri-table-low-degree-on-eval-domain
  by simp
have bad-candidate:
  generic-fri-bad-candidate
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
    (bad fri-dg composition-table)
  by (rule generic-fri-bad-candidate-from-sampled-layer-chain-full-cover
      [OF full not-low])
have bad-generic:
  composition-bs ∈ generic-fri-bad-challenge-lists
    (length composition-bs) (bad fri-dg composition-table)
  by (rule generic-fri-bad-candidateD(3)[OF bad-candidate])
have partial:
  generic-fri-partial-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
  by (rule generic-fri-bad-candidateD(1)[OF bad-candidate])
have len-composition:
  length composition-bs = composition-fri-algebraic-round-count fri-dg
  using generic-fri-partial-evidence-shapes(1,2)[OF partial]
  by (simp add: composition-fri-algebraic-round-count-def)
show composition-bs ∈
  composition-fri-multiround-bad-sets bad fri-dg composition-table
  using bad-generic len-composition
  by (simp add: composition-fri-multiround-bad-sets-as-generic)
qed

lemma trace-fri-bad-with-full-cover-layer-chain-imp-list-hit:
  assumes full:
    trace-fri-bad-with-full-cover-layer-chain s bad out
  and envelope:
     $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$ 
  shows trace-fri-challenge-list-set-hit s B out
proof -
  from full obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table doms layers
  where evidence:

```


trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *full-cover*:
generic-fri-sampled-layer-chain-full-cover trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers (bad trace-table)
unfolding *trace-fri-bad-with-full-cover-layer-chain-def* **by** *blast*
have *bad-generic*:
trace-bs ∈ generic-fri-bad-challenge-lists (length trace-bs)
(bad trace-table)
proof –
have *not-low*: $\neg$ *trace-table-low-degree trace-table*
using *trace-fri-partial-candidate-opening-evidenceD(2)[OF evidence]*
trace-fri-partial-candidate-evidenceD(3)
by *blast*
have *bad-candidate*:
generic-fri-bad-candidate trace-table-low-degree trace-table
(clength - 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers (bad trace-table)
by (rule *generic-fri-bad-candidate-from-sampled-layer-chain-full-cover*
 $[OF \text{ full-cover not-low}]$)
show *?thesis*
by (rule *generic-fri-bad-candidateD(3)[OF bad-candidate]*)
qed
have *partial*:
generic-fri-partial-evidence trace-table-low-degree trace-table
(clength - 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers
proof –
have *not-low*: $\neg$ *trace-table-low-degree trace-table*
using *trace-fri-partial-candidate-opening-evidenceD(2)[OF evidence]*
trace-fri-partial-candidate-evidenceD(3)
by *blast*
have *bad-candidate*:
generic-fri-bad-candidate trace-table-low-degree trace-table
(clength - 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers (bad trace-table)
by (rule *generic-fri-bad-candidate-from-sampled-layer-chain-full-cover*
 $[OF \text{ full-cover not-low}]$)
show *?thesis*
by (rule *generic-fri-bad-candidateD(1)[OF bad-candidate]*)
qed
have *len-trace*:
length trace-bs = trace-fri-algebraic-round-count
using *generic-fri-partial-evidence-shapes(1,2)[OF partial]*
by (*simp add: trace-fri-algebraic-round-count-def*)

```

have bad-hit: trace-bs ∈ trace-fri-multiround-bad-sets bad trace-table
  using bad-generic len-trace
  by (simp add: trace-fri-multiround-bad-sets-as-generic)
have challenges:
  accepted-fri-challenges s out trace-bs dg composition-bs
  using trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence]
  accepted-fri-opening-transcript-challenges by blast
have trace-bs ∈ B
  using bad-hit envelope[of trace-table] by blast
then show ?thesis
  unfolding trace-fri-challenge-list-set-hit-def
  using challenges by blast
qed

lemma composition-fri-bad-with-full-cover-layer-chain-imp-list-hit:
  assumes full:
    composition-fri-bad-with-full-cover-layer-chain s bad out
  and envelope:
     $\bigwedge dg$  composition-table.
      composition-fri-multiround-bad-sets bad dg composition-table
       $\subseteq B dg$ 
  shows composition-fri-challenge-list-set-hit s B out
proof –
from full obtain trace-roots trace-bs trace-final fri-dg
  opening-composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers fr
  f-fri-roots f-final as dg composition-fri-roots final
  trace-query-idxs trace-openings composition-query-idxs
  composition-openings trace-table composition-table doms layers
where evidence:
  composition-fri-partial-candidate-opening-evidence s out trace-roots
  trace-bs trace-final fri-dg opening-composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr f-fri-roots f-final as dg
  composition-fri-roots final trace-query-idxs trace-openings
  composition-query-idxs composition-openings trace-table
  composition-table
and full-cover:
  generic-fri-sampled-layer-chain-full-cover
  (composition-table-low-degree (to-nat fri-dg)) composition-table
  (to-nat fri-dg) opening-composition-roots composition-bs
  composition-final fri-query-idxs composition-round-layers doms layers
  (bad fri-dg composition-table)
unfolding composition-fri-bad-with-full-cover-layer-chain-def
by auto
have bad-generic:
  composition-bs ∈ generic-fri-bad-challenge-lists
  (length composition-bs) (bad fri-dg composition-table)
proof –

```

```

have not-low-on:
  ¬ fri-table-low-degree-on (to-nat fri-dg) eval-domain
    composition-table
  using full-cover
  unfolding generic-fri-sampled-layer-chain-full-cover-def by blast
have not-low:
  ¬ composition-table-low-degree (to-nat fri-dg) composition-table
  using not-low-on
    composition-table-not-low-degree-iff-not-fri-table-low-degree-on-eval-domain
  by simp
have bad-candidate:
  generic-fri-bad-candidate
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
    (bad fri-dg composition-table)
  by (rule generic-fri-bad-candidate-from-sampled-layer-chain-full-cover
    [OF full-cover not-low])
show ?thesis
  by (rule generic-fri-bad-candidateD(3)[OF bad-candidate])
qed
have partial:
  generic-fri-partial-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
proof –
  have not-low-on:
    ¬ fri-table-low-degree-on (to-nat fri-dg) eval-domain
      composition-table
    using full-cover
    unfolding generic-fri-sampled-layer-chain-full-cover-def by blast
  have not-low:
    ¬ composition-table-low-degree (to-nat fri-dg) composition-table
    using not-low-on
      composition-table-not-low-degree-iff-not-fri-table-low-degree-on-eval-domain
    by simp
  have bad-candidate:
    generic-fri-bad-candidate
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers
      (bad fri-dg composition-table)
    by (rule generic-fri-bad-candidate-from-sampled-layer-chain-full-cover
      [OF full-cover not-low])
  show ?thesis
    by (rule generic-fri-bad-candidateD(1)[OF bad-candidate])
qed
have len-composition:

```

$\text{length } \text{composition-bs} = \text{composition-fri-algebraic-round-count } \text{fri-dg}$
using $\text{generic-fri-partial-evidence-shapes}(1,2)[OF \text{ partial}]$
by ($\text{simp add: composition-fri-algebraic-round-count-def}$)
have bad-hit :
 $\text{composition-bs} \in$
 $\text{composition-fri-multiround-bad-sets } \text{bad } \text{fri-dg } \text{composition-table}$
using $\text{bad-generic len-composition}$
by ($\text{simp add: composition-fri-multiround-bad-sets-as-generic}$)
have challenges :
 $\text{accepted-fri-challenges } s \text{ out } \text{trace-bs } \text{fri-dg } \text{composition-bs}$
using $\text{composition-fri-partial-candidate-opening-evidenceD}(1)[OF \text{ evidence}]$
 $\text{accepted-fri-opening-transcript-challenges}$ **by** blast
have $\text{composition-bs} \in B \text{ fri-dg}$
using $\text{bad-hit envelope[of fri-dg composition-table]}$ **by** blast
then show $?thesis$
unfolding $\text{composition-fri-challenge-list-set-hit-def}$
using challenges **by** blast
qed

lemma $\text{trace-fri-sampled-imp-full-cover-or-missing}$:
assumes $\text{trace-fri-bad-with-sampled-layer-chain } s \text{ out}$
shows
 $\text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad out} \vee$
 $\text{trace-fri-sampled-missing-full-cover } s \text{ bad out}$
using assms **unfolding** $\text{trace-fri-sampled-missing-full-cover-def}$ **by** blast

lemma $\text{composition-fri-sampled-imp-full-cover-or-missing}$:
assumes $\text{composition-fri-bad-with-sampled-layer-chain } s \text{ out}$
shows
 $\text{composition-fri-bad-with-full-cover-layer-chain } s \text{ bad out} \vee$
 $\text{composition-fri-sampled-missing-full-cover } s \text{ bad out}$
using assms **unfolding** $\text{composition-fri-sampled-missing-full-cover-def}$
by blast

lemma $\text{wp-trace-fri-bad-with-full-cover-layer-chain-bound-from-envelope}$:
assumes $\text{future: trace-fri-future-fresh } s$
and envelope :
 $\bigwedge \text{trace-table. } \text{trace-fri-multiround-bad-sets } \text{bad } \text{trace-table} \subseteq B$
and $\text{subset: } B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$
and bound :
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } \text{clength}) \leq C$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad}) s \leq C$
proof –
have $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad}) s \leq$
 $\text{wp-event verify-monad } (\text{trace-fri-challenge-list-set-hit } s B) s$
by ($\text{rule wp-event-mono}$)

```

    (rule trace-fri-bad-with-full-cover-layer-chain-imp-list-hit
      [OF - envelope])
  also have ... ≤ nreal (card B) /
    nreal (CARD('f) ^ ceil-log clength)
  by (rule wp-verify-monad-trace-fri-challenge-list-set-bound
      [OF future subset])
  also have ... ≤ C
  by (rule bound)
  finally show ?thesis .
qed

lemma wp-composition-fri-bad-with-full-cover-layer-chain-bound-from-envelope:
  assumes future: composition-fri-future-fresh s
  and envelope:
    ∧ dg composition-table.
      composition-fri-multi-round-bad-sets bad dg composition-table
      ⊆ B dg
  and subset:
    ∧ dg. B dg ⊆ fri-challenge-space (ceil-log (to-nat dg + 1))
  and bound:
    ∧ dg. nreal (card (B dg)) /
      nreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C
  shows
    wp-event verify-monad
      (composition-fri-bad-with-full-cover-layer-chain s bad) s ≤ C
proof -
  have wp-event verify-monad
    (composition-fri-bad-with-full-cover-layer-chain s bad) s ≤
    wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s
  by (rule wp-event-mono)
    (rule composition-fri-bad-with-full-cover-layer-chain-imp-list-hit
      [OF - envelope])
  also have ... ≤ C
  by (rule wp-verify-monad-composition-fri-challenge-list-set-bound
      [OF future subset bound])
  finally show ?thesis .
qed

lemma wp-trace-fri-sampled-bound-from-full-cover-and-missing:
  assumes full-bound:
    wp-event verify-monad
      (trace-fri-bad-with-full-cover-layer-chain s bad) s ≤ R
  and missing-bound:
    wp-event verify-monad
      (trace-fri-sampled-missing-full-cover s bad) s ≤ M
  shows
    wp-event verify-monad
      (trace-fri-bad-with-sampled-layer-chain s) s ≤ R + M
proof -

```

have *wp-event verify-monad*
 (*trace-fri-bad-with-sampled-layer-chain s*) $s \leq$
wp-event verify-monad
 ($\lambda out. trace-fri-bad-with-full-cover-layer-chain s bad out \vee$
trace-fri-sampled-missing-full-cover s bad out) s
by (*rule wp-event-mono*)
 (*rule trace-fri-sampled-imp-full-cover-or-missing*)
also have ... $\leq$
wp-event verify-monad
 (*trace-fri-bad-with-full-cover-layer-chain s bad*) $s +$
wp-event verify-monad
 (*trace-fri-sampled-missing-full-cover s bad*) s
by (*rule wp-event-union-bound*)
also have ... $\leq R + M$
by (*intro add-mono full-bound missing-bound*)
finally show ?thesis .
qed

lemma *wp-composition-fri-sampled-bound-from-full-cover-and-missing:*
assumes *full-bound:*
wp-event verify-monad
 (*composition-fri-bad-with-full-cover-layer-chain s bad*) $s \leq R$
and *missing-bound:*
wp-event verify-monad
 (*composition-fri-sampled-missing-full-cover s bad*) $s \leq M$
shows
wp-event verify-monad
 (*composition-fri-bad-with-sampled-layer-chain s*) $s \leq R + M$
proof –
have *wp-event verify-monad*
 (*composition-fri-bad-with-sampled-layer-chain s*) $s \leq$
wp-event verify-monad
 ($\lambda out. composition-fri-bad-with-full-cover-layer-chain s bad out \vee$
composition-fri-sampled-missing-full-cover s bad out) s
by (*rule wp-event-mono*)
 (*rule composition-fri-sampled-imp-full-cover-or-missing*)
also have ... $\leq$
wp-event verify-monad
 (*composition-fri-bad-with-full-cover-layer-chain s bad*) $s +$
wp-event verify-monad
 (*composition-fri-sampled-missing-full-cover s bad*) s
by (*rule wp-event-union-bound*)
also have ... $\leq R + M$
by (*intro add-mono full-bound missing-bound*)
finally show ?thesis .
qed

definition *trace-fri-canonical-layer-chain-cover*
 $:: (f, 'a) protocol-channel-scheme \Rightarrow$

$(\text{'f list} \Rightarrow \text{nat} \Rightarrow \text{'f list} \Rightarrow \text{'f set}) \Rightarrow \text{bool}$
where
 $\text{trace-fri-canonical-layer-chain-cover } s \text{ bad} \longleftrightarrow$
 $(\forall \text{ out trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
 $\text{trace-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \longrightarrow$
 $\text{generic-fri-canonical-layer-chain-evidence trace-table-low-degree}$
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers (bad trace-table)}$
 $\longrightarrow$
 $\text{trace-bs} \in \text{trace-fri-multiround-bad-sets bad trace-table})$

definition *composition-fri-canonical-layer-chain-cover*

$:: (\text{'f}, \text{'a}) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{'f} \Rightarrow \text{'f list} \Rightarrow \text{nat} \Rightarrow \text{'f list} \Rightarrow \text{'f set}) \Rightarrow \text{bool}$

where

$\text{composition-fri-canonical-layer-chain-cover } s \text{ bad} \longleftrightarrow$
 $(\forall \text{ out trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table doms layers.}$
 $\text{composition-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings trace-table}$
 $\text{composition-table} \longrightarrow$
 $\text{generic-fri-canonical-layer-chain-evidence}$
 $(\text{composition-table-low-degree (to-nat fri-dg)}) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs composition-round-layers doms layers}$
 $(\text{bad fri-dg composition-table}) \longrightarrow$
 $\text{composition-bs} \in$
 $\text{composition-fri-multiround-bad-sets bad fri-dg composition-table})$

lemma *trace-fri-canonical-layer-chain-cover-generic:*

fixes $s :: (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}$

fixes $\text{bad} :: \text{'f list} \Rightarrow \text{nat} \Rightarrow \text{'f list} \Rightarrow \text{'f set}$

shows $\text{trace-fri-canonical-layer-chain-cover } s \text{ bad}$

proof (*unfold trace-fri-canonical-layer-chain-cover-def, intro allI impI*)

```

fix out trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table doms layers
assume evidence:
  trace-fri-partial-candidate-opening-evidence s out trace-roots
  trace-bs trace-final dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table
assume chain:
  generic-fri-canonical-layer-chain-evidence trace-table-low-degree
  trace-table (clength - 1) trace-roots trace-bs trace-final
  fri-query-idxs trace-round-layers doms layers (bad trace-table)
have bad-generic:
  trace-bs ∈ generic-fri-bad-challenge-lists (length trace-bs)
  (bad trace-table)
  by (rule generic-fri-canonical-layer-chain-bad-challenge-list[OF chain])
have partial:
  generic-fri-partial-evidence trace-table-low-degree trace-table
  (clength - 1) trace-roots trace-bs trace-final fri-query-idxs
  trace-round-layers
  by (rule generic-fri-canonical-layer-chain-evidenceD(1)[OF chain])
have len-trace:
  length trace-bs = trace-fri-algebraic-round-count
  using generic-fri-partial-evidence-shapes(1,2)[OF partial]
  by (simp add: trace-fri-algebraic-round-count-def)
show trace-bs ∈ trace-fri-multiround-bad-sets bad trace-table
  using bad-generic len-trace
  by (simp add: trace-fri-multiround-bad-sets-as-generic)
qed

```

```

lemma composition-fri-canonical-layer-chain-cover-generic:
  fixes s :: ('f, 'a) protocol-channel-scheme
  fixes bad :: 'f ⇒ 'f list ⇒ nat ⇒ 'f list ⇒ 'f set
  shows composition-fri-canonical-layer-chain-cover s bad
proof (unfold composition-fri-canonical-layer-chain-cover-def,
  intro allI impI)
fix out trace-roots trace-bs trace-final fri-dg
  opening-composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers fr
  f-fri-roots f-final as dg composition-fri-roots final
  trace-query-idxs trace-openings composition-query-idxs
  composition-openings trace-table composition-table doms layers
assume evidence:
  composition-fri-partial-candidate-opening-evidence s out trace-roots
  trace-bs trace-final fri-dg opening-composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr f-fri-roots f-final as dg

```


composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
assume *chain*:
generic-fri-canonical-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
(bad fri-dg composition-table)
have *bad-generic*:
composition-bs $\in$ *generic-fri-bad-challenge-lists*
(length composition-bs) (bad fri-dg composition-table)
by (rule *generic-fri-canonical-layer-chain-bad-challenge-list*[*OF chain*])
have *partial*:
generic-fri-partial-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
by (rule *generic-fri-canonical-layer-chain-evidenceD*(1)[*OF chain*])
have *len-composition*:
length composition-bs = *composition-fri-algebraic-round-count fri-dg*
using *generic-fri-partial-evidence-shapes*(1,2)[*OF partial*]
by (*simp add: composition-fri-algebraic-round-count-def*)
show *composition-bs* $\in$
composition-fri-multiround-bad-sets bad fri-dg composition-table
using *bad-generic len-composition*
by (*simp add: composition-fri-multiround-bad-sets-as-generic*)
qed

lemma *trace-fri-bad-with-refined-layer-chain-imp-list-hit*:
assumes *refined*:
trace-fri-bad-with-refined-layer-chain s bad out
and *envelope*:
 $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
shows *trace-fri-challenge-list-set-hit s B out*
proof —
from *refined* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table doms layers
where *evidence*:
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *chain*:
generic-fri-canonical-layer-chain-evidence trace-table-low-degree
trace-table (clength — 1) trace-roots trace-bs trace-final

fri-query-idxs trace-round-layers doms layers (bad trace-table)
unfolding *trace-fri-bad-with-refined-layer-chain-def* **by** *blast*
have *cover: trace-fri-canonical-layer-chain-cover s bad*
by *(rule trace-fri-canonical-layer-chain-cover-generic)*
have *bad-hit: trace-bs ∈ trace-fri-multiround-bad-sets bad trace-table*
using *cover evidence chain*
unfolding *trace-fri-canonical-layer-chain-cover-def* **by** *blast*
have *challenges:*
accepted-fri-challenges s out trace-bs dg composition-bs
using *trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence]*
accepted-fri-opening-transcript-challenges **by** *blast*
have *trace-bs ∈ B*
using *bad-hit envelope[of trace-table]* **by** *blast*
then show *?thesis*
unfolding *trace-fri-challenge-list-set-hit-def*
using *challenges* **by** *blast*
qed

lemma *composition-fri-bad-with-refined-layer-chain-imp-list-hit:*
assumes *refined:*
composition-fri-bad-with-refined-layer-chain s bad out
and *envelope:*
 $\bigwedge dg$ *composition-table.*
composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B dg$
shows *composition-fri-challenge-list-set-hit s B out*

proof –
from *refined* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
where *evidence:*
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
and *chain:*
generic-fri-canonical-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
(bad fri-dg composition-table)
unfolding *composition-fri-bad-with-refined-layer-chain-def*
by *auto*

```

have bad-hit:
  composition-bs ∈
    composition-fri-multiround-bad-sets bad fri-dg composition-table
proof –
  have bad-generic:
    composition-bs ∈ generic-fri-bad-challenge-lists
      (length composition-bs) (bad fri-dg composition-table)
    by (rule generic-fri-canonical-layer-chain-bad-challenge-list
      [OF chain])
  have partial:
    generic-fri-partial-evidence
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers
    by (rule generic-fri-canonical-layer-chain-evidenceD(1)[OF chain])
  have len-composition:
    length composition-bs = composition-fri-algebraic-round-count fri-dg
    using generic-fri-partial-evidence-shapes(1,2)[OF partial]
    by (simp add: composition-fri-algebraic-round-count-def)
  show ?thesis
    using bad-generic len-composition
    by (simp add: composition-fri-multiround-bad-sets-as-generic)
qed
have challenges:
  accepted-fri-challenges s out trace-bs fri-dg composition-bs
  using composition-fri-partial-candidate-opening-evidenceD(1)[OF evidence]
    accepted-fri-opening-transcript-challenges by blast
have composition-bs ∈ B fri-dg
  using bad-hit envelope[of fri-dg composition-table] by blast
then show ?thesis
  unfolding composition-fri-challenge-list-set-hit-def
  using challenges by blast
qed

lemma wp-trace-fri-bad-with-refined-layer-chain-bound-from-envelope:
  assumes future: trace-fri-future-fresh s
  and envelope:
     $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$ 
  and subset:  $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$ 
  and bound:
     $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log clength}) \leq C$ 
  shows
    wp-event verify-monad
      (trace-fri-bad-with-refined-layer-chain s bad)  $s \leq C$ 
proof –
  have wp-event verify-monad
    (trace-fri-bad-with-refined-layer-chain s bad)  $s \leq$ 
    wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s
    by (rule wp-event-mono)

```

```

    (rule trace-fri-bad-with-refined-layer-chain-imp-list-hit
      [OF - envelope])
  also have ... ≤ nreal (card B) /
    nreal (CARD('f) ^ ceil-log clength)
  by (rule wp-verify-monad-trace-fri-challenge-list-set-bound
      [OF future subset])
  also have ... ≤ C
  by (rule bound)
  finally show ?thesis .
qed

lemma wp-composition-fri-bad-with-refined-layer-chain-bound-from-envelope:
  assumes future: composition-fri-future-fresh s
  and envelope:
    ∧ dg composition-table.
      composition-fri-multi-round-bad-sets bad dg composition-table
        ⊆ B dg
  and subset:
    ∧ dg. B dg ⊆ fri-challenge-space (ceil-log (to-nat dg + 1))
  and bound:
    ∧ dg. nreal (card (B dg)) /
      nreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C
  shows
    wp-event verify-monad
      (composition-fri-bad-with-refined-layer-chain s bad) s ≤ C
proof -
  have wp-event verify-monad
    (composition-fri-bad-with-refined-layer-chain s bad) s ≤
    wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s
  by (rule wp-event-mono)
    (rule composition-fri-bad-with-refined-layer-chain-imp-list-hit
      [OF - envelope])
  also have ... ≤ C
  by (rule wp-verify-monad-composition-fri-challenge-list-set-bound
      [OF future subset bound])
  finally show ?thesis .
qed

lemma trace-fri-bad-with-sampled-layer-chain-imp-list-hit:
  assumes sampled:
    trace-fri-bad-with-sampled-layer-chain s out
  and cover: trace-fri-sampled-layer-chain-cover s bad
  and envelope:
    ∧ trace-table. trace-fri-multi-round-bad-sets bad trace-table ⊆ B
  shows trace-fri-challenge-list-set-hit s B out
proof -
  from sampled obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings

```

trace-table doms *layers*
where *evidence*:
trace-fri-partial-candidate-opening-evidence *s* out *trace-roots*
trace-bs *trace-final* *dg* *composition-roots* *composition-bs*
composition-final *fri-query-idxs* *trace-round-layers*
composition-round-layers fr *candidate-query-idxs* *trace-openings*
trace-table
and *chain*:
generic-fri-sampled-layer-chain-evidence *trace-table-low-degree*
trace-table (clength - 1) *trace-roots* *trace-bs* *trace-final*
fri-query-idxs *trace-round-layers* doms *layers*
unfolding *trace-fri-bad-with-sampled-layer-chain-def* **by** *blast*
have *bad-hit*: *trace-bs* $\in$ *trace-fri-multiround-bad-sets* *bad* *trace-table*
using *cover* *evidence* *chain*
unfolding *trace-fri-sampled-layer-chain-cover-def* **by** *blast*
have *challenges*:
accepted-fri-challenges *s* out *trace-bs* *dg* *composition-bs*
using *trace-fri-partial-candidate-opening-evidence* *D*(1)[*OF* *evidence*]
accepted-fri-opening-transcript-challenges **by** *blast*
have *trace-bs* $\in$ *B*
using *bad-hit* *envelope*[*of* *trace-table*] **by** *blast*
then show ?*thesis*
unfolding *trace-fri-challenge-list-set-hit-def*
using *challenges* **by** *blast*
qed

lemma *composition-fri-bad-with-sampled-layer-chain-imp-list-hit*:

assumes *sampled*:
composition-fri-bad-with-sampled-layer-chain *s* out
and *cover*: *composition-fri-sampled-layer-chain-cover* *s* *bad*
and *envelope*:
 $\bigwedge dg$ *composition-table*.
composition-fri-multiround-bad-sets *bad* *dg* *composition-table*
 $\subseteq B$ *dg*

shows *composition-fri-challenge-list-set-hit* *s* *B* out

proof –

from *sampled* **obtain** *trace-roots* *trace-bs* *trace-final* *fri-dg*
opening-composition-roots *composition-bs* *composition-final*
fri-query-idxs *trace-round-layers* *composition-round-layers* fr
f-fri-roots *f-final* as *dg* *composition-fri-roots* *final*
trace-query-idxs *trace-openings* *composition-query-idxs*
composition-openings *trace-table* *composition-table* doms *layers*
where *evidence*:
composition-fri-partial-candidate-opening-evidence *s* out *trace-roots*
trace-bs *trace-final* *fri-dg* *opening-composition-roots* *composition-bs*
composition-final *fri-query-idxs* *trace-round-layers*
composition-round-layers fr *f-fri-roots* *f-final* as *dg*
composition-fri-roots *final* *trace-query-idxs* *trace-openings*
composition-query-idxs *composition-openings* *trace-table*

```

    composition-table
  and chain:
    generic-fri-sampled-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms layers
  unfolding composition-fri-bad-with-sampled-layer-chain-def by auto
  have bad-hit:
    composition-bs ∈
      composition-fri-multiround-bad-sets bad fri-dg composition-table
  using cover evidence chain
  unfolding composition-fri-sampled-layer-chain-cover-def by metis
  have challenges:
    accepted-fri-challenges s out trace-bs fri-dg composition-bs
  using composition-fri-partial-candidate-opening-evidenceD(1)[OF evidence]
    accepted-fri-opening-transcript-challenges by blast
  have composition-bs ∈ B fri-dg
    using bad-hit envelope[of fri-dg composition-table] by blast
  then show ?thesis
    unfolding composition-fri-challenge-list-set-hit-def
    using challenges by blast
qed

lemma wp-trace-fri-bad-with-sampled-layer-chain-bound-from-envelope:
  assumes future: trace-fri-future-fresh s
  and cover: trace-fri-sampled-layer-chain-cover s bad
  and envelope:
     $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$ 
  and subset:  $B \subseteq \text{fri-challenge-space (ceil-log clength)}$ 
  and bound:
     $\text{nnreal (card B) / nnreal (CARD('f) } ^{\wedge} \text{ceil-log clength)} \leq C$ 
  shows
    wp-event verify-monad
      (trace-fri-bad-with-sampled-layer-chain s) s  $\leq C$ 
proof -
  have wp-event verify-monad
    (trace-fri-bad-with-sampled-layer-chain s) s  $\leq$ 
    wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s
  by (rule wp-event-mono)
    (rule trace-fri-bad-with-sampled-layer-chain-imp-list-hit
      [OF - cover envelope])
  also have ...  $\leq$  nnreal (card B) /
    nnreal (CARD('f)  $^{\wedge}$  ceil-log clength)
  by (rule wp-verify-monad-trace-fri-challenge-list-set-bound
    [OF future subset])
  also have ...  $\leq C$ 
  by (rule bound)
  finally show ?thesis .
qed

```

lemma *wp-composition-fri-bad-with-sampled-layer-chain-bound-from-envelope:*

assumes *future: composition-fri-future-fresh s*
and *cover: composition-fri-sampled-layer-chain-cover s bad*
and *envelope:*
 $\bigwedge dg$ *composition-table.*
composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B dg$
and *subset:*
 $\bigwedge dg. B dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$
and *bound:*
 $\bigwedge dg. \text{nnreal } (\text{card } (B dg)) /$
 $\text{nnreal } (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq C$

shows

wp-event verify-monad
(composition-fri-bad-with-sampled-layer-chain s) s $\leq C$

proof –

have *wp-event verify-monad*
(composition-fri-bad-with-sampled-layer-chain s) s $\leq$
wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s
by (*rule wp-event-mono*)
(rule composition-fri-bad-with-sampled-layer-chain-imp-list-hit
[OF - cover envelope])
also have $\dots \leq C$
by (*rule wp-verify-monad-composition-fri-challenge-list-set-bound*
[OF future subset bound])
finally show *?thesis .*

qed

lemma *trace-fri-reachable-imp-refined-or-missing-layer-chain:*

assumes *trace-fri-bad-with-reachable-partial-candidate s out*
shows
trace-fri-bad-with-refined-layer-chain s bad out $\vee$
trace-fri-reachable-missing-refined-layer-chain s bad out
using *assms*
unfolding *trace-fri-reachable-missing-refined-layer-chain-def* **by** *blast*

lemma *composition-fri-reachable-imp-refined-or-missing-layer-chain:*

assumes *composition-fri-bad-with-reachable-partial-candidate s out*
shows
composition-fri-bad-with-refined-layer-chain s bad out $\vee$
composition-fri-reachable-missing-refined-layer-chain s bad out
using *assms*
unfolding *composition-fri-reachable-missing-refined-layer-chain-def* **by** *blast*

lemma *trace-fri-reachable-imp-sampled-or-missing-layer-chain:*

assumes *trace-fri-bad-with-reachable-partial-candidate s out*
shows
trace-fri-bad-with-sampled-layer-chain s out $\vee$

trace-fri-reachable-missing-sampled-layer-chain s out
using *assms*
unfolding *trace-fri-reachable-missing-sampled-layer-chain-def* **by** *blast*

lemma *composition-fri-reachable-imp-sampled-or-missing-layer-chain:*
assumes *composition-fri-bad-with-reachable-partial-candidate s out*
shows
composition-fri-bad-with-sampled-layer-chain s out $\vee$
composition-fri-reachable-missing-sampled-layer-chain s out
using *assms*
unfolding *composition-fri-reachable-missing-sampled-layer-chain-def*
by *blast*

lemma *verify-monad-trace-fri-missing-extracts-opening-evidence:*
assumes *outcome:*
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *missing:*
trace-fri-reachable-missing-refined-layer-chain s bad
(Some (result, final-state))
obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
where
trace-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final dg
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr candidate-query-idxs
trace-openings trace-table
 $\bigwedge$ *doms layers. $\neg$*
generic-fri-canonical-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers (bad trace-table)
proof –
have *reachable:*
trace-fri-bad-with-reachable-partial-candidate s
(Some (result, final-state))
and *not-refined:*
 $\neg$ *trace-fri-bad-with-refined-layer-chain s bad*
(Some (result, final-state))
using *missing*
unfolding *trace-fri-reachable-missing-refined-layer-chain-def*
by *blast+*
from *verify-monad-trace-fri-bad-extracts-opening-evidence*
[OF outcome reachable]
obtain *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

where *evidence*:
trace-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final dg
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr candidate-query-idxs
trace-openings trace-table
by *blast*
have *no-chain*:
 $\bigwedge \text{doms layers. } \neg$
generic-fri-canonical-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers (bad trace-table)
proof
fix *doms layers*
assume *chain*:
generic-fri-canonical-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers (bad trace-table)
have *trace-fri-bad-with-refined-layer-chain s bad*
(Some (result, final-state))
unfolding *trace-fri-bad-with-refined-layer-chain-def*
using *evidence chain* **by** *blast*
then show *False*
using *not-refined* **by** *contradiction*
qed
show *?thesis*
by *(rule that[OF evidence no-chain])*
qed

lemma *verify-monad-composition-fri-missing-extracts-opening-evidence*:

assumes *outcome*:
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *missing*:
composition-fri-reachable-missing-refined-layer-chain s bad
(Some (result, final-state))

obtains *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table

where
composition-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table

```

to-nat fri-dg ≤ maxDegree
 $\bigwedge$ doms layers.  $\neg$ 
  generic-fri-canonical-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms layers
    (bad fri-dg composition-table)
proof –
have reachable:
  composition-fri-bad-with-reachable-partial-candidate s
    (Some (result, final-state))
and not-refined:
   $\neg$  composition-fri-bad-with-refined-layer-chain s bad
    (Some (result, final-state))
using missing
unfolding composition-fri-reachable-missing-refined-layer-chain-def
by blast+
show ?thesis
proof (rule
  verify-monad-composition-fri-bad-extracts-opening-evidence-with-degree-bound
    [OF outcome reachable])
fix trace-roots trace-bs trace-final fri-dg
  opening-composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers fr
  f-fri-roots f-final as dg composition-fri-roots final
  trace-query-idxs trace-openings composition-query-idxs
  composition-openings trace-table composition-table
assume evidence:
  composition-fri-partial-candidate-opening-evidence s
    (Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table
assume degree-bound: to-nat fri-dg ≤ maxDegree
have no-chain:
   $\bigwedge$ doms layers.  $\neg$ 
    generic-fri-canonical-layer-chain-evidence
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers doms
      layers (bad fri-dg composition-table)
proof
fix doms layers
assume chain:
  generic-fri-canonical-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs

```

```

    composition-final fri-query-idxs composition-round-layers doms
    layers (bad fri-dg composition-table)
  have composition-fri-bad-with-refined-layer-chain s bad
    (Some (result, final-state))
  using evidence chain
  unfolding composition-fri-bad-with-refined-layer-chain-def
  by metis
  then show False
    using not-refined by contradiction
qed
show thesis
  by (rule that[OF evidence degree-bound no-chain])
qed
qed

lemma verify-monad-trace-fri-missing-sampled-extracts-opening-evidence:
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and missing:
    trace-fri-reachable-missing-sampled-layer-chain s
    (Some (result, final-state))
  obtains trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
  where
    trace-fri-partial-candidate-opening-evidence s
    (Some (result, final-state)) trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr candidate-query-idxs
    trace-openings trace-table
    ∧ doms layers. ¬
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers
proof -
  have reachable:
    trace-fri-bad-with-reachable-partial-candidate s
    (Some (result, final-state))
  and not-sampled:
    ¬ trace-fri-bad-with-sampled-layer-chain s
    (Some (result, final-state))
  using missing
  unfolding trace-fri-reachable-missing-sampled-layer-chain-def
  by blast+
from verify-monad-trace-fri-bad-extracts-opening-evidence
  [OF outcome reachable]
obtain trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers

```

```

    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
where evidence:
    trace-fri-partial-candidate-opening-evidence s
    (Some (result, final-state)) trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr candidate-query-idxs
    trace-openings trace-table
by blast
have no-sampled:
     $\bigwedge$  doms layers.  $\neg$ 
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers
proof
    fix doms layers
    assume chain:
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers
    have trace-fri-bad-with-sampled-layer-chain s
    (Some (result, final-state))
    unfolding trace-fri-bad-with-sampled-layer-chain-def
    using evidence chain by blast
    then show False
    using not-sampled by contradiction
qed
show ?thesis
    by (rule that[OF evidence no-sampled])
qed

lemma verify-monad-composition-fri-missing-sampled-extracts-opening-evidence:
assumes outcome:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
and missing:
    composition-fri-reachable-missing-sampled-layer-chain s
    (Some (result, final-state))
obtains trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table
where
    composition-fri-partial-candidate-opening-evidence s
    (Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final

```

```

    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table
    to-nat fri-dg ≤ maxDegree
    ∧ doms layers. ¬
      generic-fri-sampled-layer-chain-evidence
        (composition-table-low-degree (to-nat fri-dg)) composition-table
        (to-nat fri-dg) opening-composition-roots composition-bs
        composition-final fri-query-idxs composition-round-layers doms layers
  proof —
  have reachable:
    composition-fri-bad-with-reachable-partial-candidate s
      (Some (result, final-state))
  and not-sampled:
    ¬ composition-fri-bad-with-sampled-layer-chain s
      (Some (result, final-state))
  using missing
  unfolding composition-fri-reachable-missing-sampled-layer-chain-def
  by blast+
  show ?thesis
  proof (rule
    verify-monad-composition-fri-bad-extracts-opening-evidence-with-degree-bound
      [OF outcome reachable])
  fix trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table
  assume evidence:
    composition-fri-partial-candidate-opening-evidence s
      (Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
      opening-composition-roots composition-bs composition-final
      fri-query-idxs trace-round-layers composition-round-layers fr
      f-fri-roots f-final as dg composition-fri-roots final
      trace-query-idxs trace-openings composition-query-idxs
      composition-openings trace-table composition-table
  assume degree-bound: to-nat fri-dg ≤ maxDegree
  have no-sampled:
    ∧ doms layers. ¬
      generic-fri-sampled-layer-chain-evidence
        (composition-table-low-degree (to-nat fri-dg)) composition-table
        (to-nat fri-dg) opening-composition-roots composition-bs
        composition-final fri-query-idxs composition-round-layers doms
        layers
  proof
  fix doms layers
  assume chain:
    generic-fri-sampled-layer-chain-evidence
      (composition-table-low-degree (to-nat fri-dg)) composition-table

```

```

    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms
    layers
  have composition-fri-bad-with-sampled-layer-chain s
    (Some (result, final-state))
  using evidence chain
  unfolding composition-fri-bad-with-sampled-layer-chain-def
  by metis
  then show False
    using not-sampled by contradiction
qed
show thesis
  by (rule that[OF evidence degree-bound no-sampled])
qed
qed

lemma verify-monad-trace-fri-missing-full-cover-extracts-sampled-evidence:
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and missing:
    trace-fri-sampled-missing-full-cover s bad
    (Some (result, final-state))
  obtains trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table doms layers
  where
    trace-fri-partial-candidate-opening-evidence s
      (Some (result, final-state)) trace-roots trace-bs trace-final dg
      composition-roots composition-bs composition-final fri-query-idxs
      trace-round-layers composition-round-layers fr candidate-query-idxs
      trace-openings trace-table
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers doms layers
    ¬ generic-fri-sampled-layer-chain-full-cover trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers doms layers (bad trace-table)
  proof -
    have sampled:
      trace-fri-bad-with-sampled-layer-chain s
        (Some (result, final-state))
    and not-full:
      ¬ trace-fri-bad-with-full-cover-layer-chain s bad
        (Some (result, final-state))
    using missing
    unfolding trace-fri-sampled-missing-full-cover-def by blast+
  from sampled obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers

```

composition-round-layers fr candidate-query-idxs trace-openings
trace-table doms layers
where *evidence*:
trace-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final dg
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr candidate-query-idxs
trace-openings trace-table
and *chain*:
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
unfolding *trace-fri-bad-with-sampled-layer-chain-def* **by** *blast*
have *no-full*:
 $\neg$ *generic-fri-sampled-layer-chain-full-cover trace-table-low-degree*
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers (bad trace-table)
proof
assume *full*:
generic-fri-sampled-layer-chain-full-cover trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers (bad trace-table)
have *trace-fri-bad-with-full-cover-layer-chain s bad*
(Some (result, final-state))
unfolding *trace-fri-bad-with-full-cover-layer-chain-def*
using *evidence full* **by** *blast*
then show *False*
using *not-full* **by** *contradiction*
qed
show *?thesis*
by *(rule that[OF evidence chain no-full])*
qed

lemma *verify-monad-composition-fri-missing-full-cover-extracts-sampled-evidence*:
assumes *outcome*:
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *missing*:
composition-fri-sampled-missing-full-cover s bad
(Some (result, final-state))
obtains *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
where
composition-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
opening-composition-roots composition-bs composition-final

```

    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table
generic-fri-sampled-layer-chain-evidence
  (composition-table-low-degree (to-nat fri-dg)) composition-table
  (to-nat fri-dg) opening-composition-roots composition-bs
  composition-final fri-query-idxs composition-round-layers doms layers
¬ generic-fri-sampled-layer-chain-full-cover
  (composition-table-low-degree (to-nat fri-dg)) composition-table
  (to-nat fri-dg) opening-composition-roots composition-bs
  composition-final fri-query-idxs composition-round-layers doms layers
  (bad fri-dg composition-table)
proof —
have sampld:
  composition-fri-bad-with-sampled-layer-chain s
    (Some (result, final-state))
and not-full:
  ¬ composition-fri-bad-with-full-cover-layer-chain s bad
    (Some (result, final-state))
using missing
unfolding composition-fri-sampled-missing-full-cover-def by blast+
from sampld obtain trace-roots trace-bs trace-final fri-dg
  opening-composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers fr
  f-fri-roots f-final as dg composition-fri-roots final
  trace-query-idxs trace-openings composition-query-idxs
  composition-openings trace-table composition-table doms layers
where evidence:
  composition-fri-partial-candidate-opening-evidence s
    (Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table
and chain:
  generic-fri-sampled-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms layers
unfolding composition-fri-bad-with-sampled-layer-chain-def by auto
have no-full:
  ¬ generic-fri-sampled-layer-chain-full-cover
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms layers
    (bad fri-dg composition-table)
proof

```


assume *full*:
 generic-fri-sampled-layer-chain-full-cover
 (*composition-table-low-degree* (*to-nat fri-dg*)) *composition-table*
 (*to-nat fri-dg*) *opening-composition-roots* *composition-bs*
 composition-final fri-query-idxs *composition-round-layers* *doms layers*
 (*bad fri-dg composition-table*)
have *composition-fri-bad-with-full-cover-layer-chain s bad*
 (*Some (result, final-state)*)
unfolding *composition-fri-bad-with-full-cover-layer-chain-def*
using *evidence full* **by** *metis*
then show *False*
using *not-full* **by** *contradiction*
qed
show *?thesis*
by (*rule that[OF evidence chain no-full]*)
qed

lemma *wp-trace-fri-reachable-bound-from-refined-and-missing*:
assumes *refined-bound*:
 wp-event verify-monad
 (*trace-fri-bad-with-refined-layer-chain s bad*) $s \leq R$
and *missing-bound*:
 wp-event verify-monad
 (*trace-fri-reachable-missing-refined-layer-chain s bad*) $s \leq M$
shows
 wp-event verify-monad
 (*trace-fri-bad-with-reachable-partial-candidate s*) $s \leq R + M$
proof –
have *wp-event verify-monad*
 (*trace-fri-bad-with-reachable-partial-candidate s*) $s \leq$
 wp-event verify-monad
 ($\lambda out. trace-fri-bad-with-refined-layer-chain s bad out \vee$
 trace-fri-reachable-missing-refined-layer-chain s bad out) s
by (*rule wp-event-mono*)
 (*rule trace-fri-reachable-imp-refined-or-missing-layer-chain*)
also have $\dots \leq$
 wp-event verify-monad
 (*trace-fri-bad-with-refined-layer-chain s bad*) $s +$
 wp-event verify-monad
 (*trace-fri-reachable-missing-refined-layer-chain s bad*) s
by (*rule wp-event-union-bound*)
also have $\dots \leq R + M$
by (*intro add-mono refined-bound missing-bound*)
finally show *?thesis* .
qed

lemma *wp-composition-fri-reachable-bound-from-refined-and-missing*:
assumes *refined-bound*:
 wp-event verify-monad

(composition-fri-bad-with-refined-layer-chain s bad) $s \leq R$
and missing-bound:
 wp-event verify-monad
 (composition-fri-reachable-missing-refined-layer-chain s bad) $s \leq M$
shows
 wp-event verify-monad
 (composition-fri-bad-with-reachable-partial-candidate s) $s \leq R + M$
proof –
have wp-event verify-monad
 (composition-fri-bad-with-reachable-partial-candidate s) $s \leq$
 wp-event verify-monad
 ($\lambda out. \text{composition-fri-bad-with-refined-layer-chain } s \text{ bad out} \vee$
 $\text{composition-fri-reachable-missing-refined-layer-chain } s \text{ bad out}$) s
by (rule wp-event-mono)
 (rule composition-fri-reachable-imp-refined-or-missing-layer-chain)
also have ... $\leq$
 wp-event verify-monad
 (composition-fri-bad-with-refined-layer-chain s bad) $s +$
 wp-event verify-monad
 (composition-fri-reachable-missing-refined-layer-chain s bad) s
by (rule wp-event-union-bound)
also have ... $\leq R + M$
by (intro add-mono refined-bound missing-bound)
finally show ?thesis .
qed

lemma wp-trace-fri-reachable-bound-from-sampled-and-missing:

assumes sampled-bound:

wp-event verify-monad
 (trace-fri-bad-with-sampled-layer-chain s) $s \leq R$

and missing-bound:

wp-event verify-monad
 (trace-fri-reachable-missing-sampled-layer-chain s) $s \leq M$

shows

wp-event verify-monad
 (trace-fri-bad-with-reachable-partial-candidate s) $s \leq R + M$

proof –

have wp-event verify-monad

(trace-fri-bad-with-reachable-partial-candidate s) $s \leq$
 wp-event verify-monad
 ($\lambda out. \text{trace-fri-bad-with-sampled-layer-chain } s \text{ out} \vee$
 $\text{trace-fri-reachable-missing-sampled-layer-chain } s \text{ out}$) s

by (rule wp-event-mono)
 (rule trace-fri-reachable-imp-sampled-or-missing-layer-chain)

also have ... $\leq$

wp-event verify-monad
 (trace-fri-bad-with-sampled-layer-chain s) $s +$
 wp-event verify-monad
 (trace-fri-reachable-missing-sampled-layer-chain s) s

by (rule wp-event-union-bound)
 also have ... $\leq R + M$
 by (intro add-mono sampled-bound missing-bound)
 finally show ?thesis .
 qed

lemma wp-composition-fri-reachable-bound-from-sampled-and-missing:
assumes sampled-bound:
 wp-event verify-monad
 (composition-fri-bad-with-sampled-layer-chain s) $s \leq R$
and missing-bound:
 wp-event verify-monad
 (composition-fri-reachable-missing-sampled-layer-chain s) $s \leq M$
shows
 wp-event verify-monad
 (composition-fri-bad-with-reachable-partial-candidate s) $s \leq R + M$
proof –
have wp-event verify-monad
 (composition-fri-bad-with-reachable-partial-candidate s) $s \leq$
 wp-event verify-monad
 ($\lambda out. \text{composition-fri-bad-with-sampled-layer-chain } s \text{ out} \vee$
 $\text{composition-fri-reachable-missing-sampled-layer-chain } s \text{ out}$) s
by (rule wp-event-mono)
 (rule composition-fri-reachable-imp-sampled-or-missing-layer-chain)
also have ... $\leq$
 wp-event verify-monad
 (composition-fri-bad-with-sampled-layer-chain s) s +
 wp-event verify-monad
 (composition-fri-reachable-missing-sampled-layer-chain s) s
by (rule wp-event-union-bound)
also have ... $\leq R + M$
by (intro add-mono sampled-bound missing-bound)
finally show ?thesis .
 qed

lemma trace-fri-reachable-partial-candidate-reduction-from-refined-and-missing:
assumes refined-bound:
 wp-event verify-monad
 (trace-fri-bad-with-refined-layer-chain s bad) $s \leq R$
and missing-bound:
 wp-event verify-monad
 (trace-fri-reachable-missing-refined-layer-chain s bad) $s \leq M$
and total-bound: $R + M \leq \text{trace-fri-error}$
shows trace-fri-reachable-partial-candidate-reduction-assumption s
proof –
have old-bound:
 wp-event verify-monad
 (trace-fri-bad-with-reachable-partial-candidate s) $s \leq R + M$
by (rule wp-trace-fri-reachable-bound-from-refined-and-missing)

```

    [OF refined-bound missing-bound])
  show ?thesis
    unfolding trace-fri-reachable-partial-candidate-reduction-assumption-def
    by (rule order-trans[OF old-bound total-bound])
qed

lemma composition-fri-reachable-partial-candidate-reduction-from-refined-and-missing:
  assumes refined-bound:
    wp-event verify-monad
      (composition-fri-bad-with-refined-layer-chain s bad)  $s \leq R$ 
  and missing-bound:
    wp-event verify-monad
      (composition-fri-reachable-missing-refined-layer-chain s bad)  $s \leq M$ 
  and total-bound:  $R + M \leq \text{composition-fri-error}$ 
  shows composition-fri-reachable-partial-candidate-reduction-assumption s
proof -
  have old-bound:
    wp-event verify-monad
      (composition-fri-bad-with-reachable-partial-candidate s)  $s \leq R + M$ 
  by (rule wp-composition-fri-reachable-bound-from-refined-and-missing
      [OF refined-bound missing-bound])
  show ?thesis
    unfolding
      composition-fri-reachable-partial-candidate-reduction-assumption-def
    by (rule order-trans[OF old-bound total-bound])
qed

lemma trace-fri-reachable-partial-candidate-reduction-from-sampled-and-missing:
  assumes sampled-bound:
    wp-event verify-monad
      (trace-fri-bad-with-sampled-layer-chain s)  $s \leq R$ 
  and missing-bound:
    wp-event verify-monad
      (trace-fri-reachable-missing-sampled-layer-chain s)  $s \leq M$ 
  and total-bound:  $R + M \leq \text{trace-fri-error}$ 
  shows trace-fri-reachable-partial-candidate-reduction-assumption s
proof -
  have old-bound:
    wp-event verify-monad
      (trace-fri-bad-with-reachable-partial-candidate s)  $s \leq R + M$ 
  by (rule wp-trace-fri-reachable-bound-from-sampled-and-missing
      [OF sampled-bound missing-bound])
  show ?thesis
    unfolding trace-fri-reachable-partial-candidate-reduction-assumption-def
    by (rule order-trans[OF old-bound total-bound])
qed

lemma composition-fri-reachable-partial-candidate-reduction-from-sampled-and-missing:
  assumes sampled-bound:

```

$wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri}\text{-bad}\text{-with}\text{-sampled}\text{-layer}\text{-chain } s) \ s \leq R$
and $missing\text{-bound}$:
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri}\text{-reachable}\text{-missing}\text{-sampled}\text{-layer}\text{-chain } s) \ s \leq M$
and $total\text{-bound}$: $R + M \leq composition\text{-fri}\text{-error}$
shows $composition\text{-fri}\text{-reachable}\text{-partial}\text{-candidate}\text{-reduction}\text{-assumption } s$
proof –
have $old\text{-bound}$:
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri}\text{-bad}\text{-with}\text{-reachable}\text{-partial}\text{-candidate } s) \ s \leq R + M$
by (rule $wp\text{-composition}\text{-fri}\text{-reachable}\text{-bound}\text{-from}\text{-sampled}\text{-and}\text{-missing}$
 $[OF \text{ sampled}\text{-bound } missing\text{-bound}]$)
show $?thesis$
unfolding
 $composition\text{-fri}\text{-reachable}\text{-partial}\text{-candidate}\text{-reduction}\text{-assumption}\text{-def}$
by (rule $order\text{-trans}[OF \text{ old}\text{-bound } total\text{-bound}]$)
qed

lemma $trace\text{-fri}\text{-bad}\text{-with}\text{-partial}\text{-candidates}\text{-bound}\text{-from}\text{-refined}\text{-and}\text{-missing}$:
fixes $R \ M :: prob$
assumes $refined\text{-bound}$:
 $\bigwedge data \text{ attacker}\text{-state}.$
 $Some (data, \text{attacker}\text{-state}) \in$
 $set\text{-dist } (execute (checked\text{-staged}\text{-transcript}\text{-program } A)$
 $\text{adversary}\text{-initial}\text{-state}) \implies$
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri}\text{-bad}\text{-with}\text{-refined}\text{-layer}\text{-chain}$
 $(\text{verifier}\text{-state}\text{-from}\text{-adversary } \text{attacker}\text{-state}$
 $(\text{staged}\text{-proof}\text{-transcript } data)) \text{ bad})$
 $(\text{verifier}\text{-state}\text{-from}\text{-adversary } \text{attacker}\text{-state}$
 $(\text{staged}\text{-proof}\text{-transcript } data)) \leq R$
and $missing\text{-bound}$:
 $\bigwedge data \text{ attacker}\text{-state}.$
 $Some (data, \text{attacker}\text{-state}) \in$
 $set\text{-dist } (execute (checked\text{-staged}\text{-transcript}\text{-program } A)$
 $\text{adversary}\text{-initial}\text{-state}) \implies$
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri}\text{-reachable}\text{-missing}\text{-refined}\text{-layer}\text{-chain}$
 $(\text{verifier}\text{-state}\text{-from}\text{-adversary } \text{attacker}\text{-state}$
 $(\text{staged}\text{-proof}\text{-transcript } data)) \text{ bad})$
 $(\text{verifier}\text{-state}\text{-from}\text{-adversary } \text{attacker}\text{-state}$
 $(\text{staged}\text{-proof}\text{-transcript } data)) \leq M$
shows
 $wp\text{-event } (checked\text{-staged}\text{-security}\text{-experiment}\text{-with}\text{-data}\text{-state } A)$
 $(\text{staged}\text{-security}\text{-with}\text{-data}\text{-state}\text{-verifier}\text{-event}$
 $trace\text{-fri}\text{-bad}\text{-with}\text{-partial}\text{-candidates})$
 $\text{adversary}\text{-initial}\text{-state} \leq R + M$
proof (rule $checked\text{-staged}\text{-security}\text{-with}\text{-data}\text{-state}\text{-verifier}\text{-event}\text{-bound}\text{-from}\text{-cont}$)

```

show  $\bigwedge s. \neg \text{trace-fri-bad-with-partial-candidates } s \text{ None}$ 
  by (rule trace-fri-bad-with-partial-candidates-not-None)
next
fix data attacker-state
assume builder:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have old-bound:
  wp-event verify-monad
    (trace-fri-bad-with-reachable-partial-candidate ?s) ?s  $\leq R + M$ 
by (rule wp-trace-fri-reachable-bound-from-refined-and-missing
    [OF refined-bound[OF builder] missing-bound[OF builder]])
show wp-event verify-monad (trace-fri-bad-with-partial-candidates ?s) ?s
   $\leq R + M$ 
by (rule order-trans[OF - old-bound])
  (rule wp-event-mono,
    rule trace-fri-bad-with-partial-candidates-imp-reachable-partial-candidate)
qed

```

lemma composition-fri-bad-with-partial-candidates-bound-from-refined-and-missing:

```

fixes R M :: prob
assumes refined-bound:
   $\bigwedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  wp-event verify-monad
    (composition-fri-bad-with-refined-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) bad)
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))  $\leq R$ 
and missing-bound:
   $\bigwedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  wp-event verify-monad
    (composition-fri-reachable-missing-refined-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) bad)
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))  $\leq M$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (staged-security-with-data-state-verifier-event
     composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq R + M$ 
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg \text{composition-fri-bad-with-partial-candidates } s \text{ None}$ 
    by (rule composition-fri-bad-with-partial-candidates-not-None)
next
fix data attacker-state
assume builder:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
                  adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have old-bound:
  wp-event verify-monad
    (composition-fri-bad-with-reachable-partial-candidate ?s) ?s  $\leq$ 
    R + M
by (rule wp-composition-fri-reachable-bound-from-refined-and-missing
      [OF refined-bound[OF builder] missing-bound[OF builder]])
show wp-event verify-monad
  (composition-fri-bad-with-partial-candidates ?s) ?s  $\leq R + M$ 
by (rule order-trans[OF - old-bound])
  (rule wp-event-mono,
   rule
    composition-fri-bad-with-partial-candidates-imp-reachable-partial-candidate)
qed

```

lemma trace-fri-bad-with-empty-header-candidates-bound-from-refined-and-missing:

```

fixes R M :: prob
assumes refined-bound:
   $\bigwedge \text{data attacker-state.}$ 
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
                  adversary-initial-state)  $\implies$ 
  wp-event verify-monad
    (trace-fri-bad-with-refined-layer-chain
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) bad)
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data))  $\leq R$ 
and missing-bound:
   $\bigwedge \text{data attacker-state.}$ 
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
                  adversary-initial-state)  $\implies$ 
  wp-event verify-monad
    (trace-fri-reachable-missing-refined-layer-chain

```

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq M$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq R + M$
proof (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
show $\bigwedge s. \neg \text{trace-fri-bad-with-empty-composition-header-candidates } s \text{ None}$
by (rule *trace-fri-bad-with-empty-composition-header-candidates-not-None*)
next
fix *data attacker-state*
assume *builder*:
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$
let $?s =$
 $\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})$
have *old-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-reachable-partial-candidate } ?s) \text{ ?s} \leq R + M$
by (rule *wp-trace-fri-reachable-bound-from-refined-and-missing*
 $[\text{OF refined-bound}[\text{OF builder}] \text{ missing-bound}[\text{OF builder}]]$)
show $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-empty-composition-header-candidates } ?s) \text{ ?s} \leq$
 $R + M$
by (rule *order-trans*[*OF* - *old-bound*])
 $(\text{rule wp-event-mono,}$
 rule
 $\text{trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate})$
qed

lemma *trace-fri-bad-with-partial-candidates-bound-from-sampled-and-missing*:

fixes $R \ M :: \text{prob}$

assumes *sampled-bound*:

$\bigwedge data \text{ attacker-state.}$

$\text{Some } (data, \text{attacker-state}) \in$

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$

$\text{adversary-initial-state}) \implies$

$\text{wp-event verify-monad}$

$(\text{trace-fri-bad-with-sampled-layer-chain}$

$(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data})))$

$(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data})) \leq R$

and *missing-bound*:

$\wedge \text{data attacker-state.}$
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-reachable-missing-sampled-layer-chain*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq M$
shows
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq R + M$
proof (*rule checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
show $\wedge s. \neg \text{trace-fri-bad-with-partial-candidates } s \text{ None}$
by (*rule trace-fri-bad-with-partial-candidates-not-None*)
next
fix *data attacker-state*
assume *builder:*
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state)
let *?s =*
verifier-state-from-adversary attacker-state
(*staged-proof-transcript data*)
have *old-bound:*
wp-event verify-monad
(*trace-fri-bad-with-reachable-partial-candidate ?s*) *?s* $\leq R + M$
by (*rule wp-trace-fri-reachable-bound-from-sampled-and-missing*
[*OF sampled-bound*[*OF builder*] *missing-bound*[*OF builder*]])
show *wp-event verify-monad* (*trace-fri-bad-with-partial-candidates ?s*) *?s*
 $\leq R + M$
by (*rule order-trans*[*OF - old-bound*])
(*rule wp-event-mono*,
rule trace-fri-bad-with-partial-candidates-imp-reachable-partial-candidate)
qed

lemma *composition-fri-bad-with-partial-candidates-bound-from-sampled-and-missing:*
fixes *R M :: prob*
assumes *sampled-bound:*
 $\wedge \text{data attacker-state.}$
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*composition-fri-bad-with-sampled-layer-chain*
(*verifier-state-from-adversary attacker-state*

```

      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤ R
  and missing-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (composition-fri-reachable-missing-sampled-layer-chain
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)) ≤ M
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-partial-candidates)
      adversary-initial-state ≤ R + M
  proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    show ∧ s. ¬ composition-fri-bad-with-partial-candidates s None
      by (rule composition-fri-bad-with-partial-candidates-not-None)
  next
  fix data attacker-state
  assume builder:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have old-bound:
    wp-event verify-monad
      (composition-fri-bad-with-reachable-partial-candidate ?s) ?s ≤
      R + M
  by (rule wp-composition-fri-reachable-bound-from-sampled-and-missing
    [OF sampled-bound[OF builder] missing-bound[OF builder]])
  show wp-event verify-monad
    (composition-fri-bad-with-partial-candidates ?s) ?s ≤ R + M
  by (rule order-trans[OF - old-bound])
    (rule wp-event-mono,
      rule
        composition-fri-bad-with-partial-candidates-imp-reachable-partial-candidate)
  qed

lemma trace-fri-bad-with-empty-header-candidates-bound-from-sampled-and-missing:
  fixes R M :: prob
  assumes sampled-bound:
    ∧ data attacker-state.

```

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-bad-with-sampled-layer-chain
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq R$
and missing-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-reachable-missing-sampled-layer-chain
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq M$
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq R + M$
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
show $\wedge s. \neg \text{trace-fri-bad-with-empty-composition-header-candidates } s \text{ None}$
by (rule trace-fri-bad-with-empty-composition-header-candidates-not-None)
next
fix data attacker-state
assume builder:
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state)
let ?s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
have old-bound:
 wp-event verify-monad
 (trace-fri-bad-with-reachable-partial-candidate ?s) ?s $\leq R + M$
by (rule wp-trace-fri-reachable-bound-from-sampled-and-missing
 [OF sampled-bound[OF builder] missing-bound[OF builder]])
show wp-event verify-monad
 (trace-fri-bad-with-empty-composition-header-candidates ?s) ?s $\leq$
 R + M
by (rule order-trans[OF - old-bound])
 (rule wp-event-mono,
 rule
 trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate)

qed

lemma *trace-fri-bad-with-partial-candidates-bound-from-full-cover-and-missing:*

fixes R MF $MS :: prob$

assumes *full-bound:*

$\wedge data$ *attacker-state.*

$Some (data, attacker-state) \in$
 $set-dist (execute (checked-staged-transcript-program A)$
 $adversary-initial-state) \implies$

$wp-event$ *verify-monad*
 $(trace-fri-bad-with-full-cover-layer-chain$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data)) bad)$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data)) \leq R$

and *missing-full-bound:*

$\wedge data$ *attacker-state.*

$Some (data, attacker-state) \in$
 $set-dist (execute (checked-staged-transcript-program A)$
 $adversary-initial-state) \implies$

$wp-event$ *verify-monad*
 $(trace-fri-sampled-missing-full-cover$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data)) bad)$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data)) \leq MF$

and *missing-sampled-bound:*

$\wedge data$ *attacker-state.*

$Some (data, attacker-state) \in$
 $set-dist (execute (checked-staged-transcript-program A)$
 $adversary-initial-state) \implies$

$wp-event$ *verify-monad*
 $(trace-fri-reachable-missing-sampled-layer-chain$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data)))$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data)) \leq MS$

shows

$wp-event (checked-staged-security-experiment-with-data-state A)$
 $(staged-security-with-data-state-verifier-event$
 $trace-fri-bad-with-partial-candidates)$
 $adversary-initial-state \leq (R + MF) + MS$

proof (rule *trace-fri-bad-with-partial-candidates-bound-from-sampled-and-missing*)

fix *data attacker-state*

assume *builder:*

$Some (data, attacker-state) \in$
 $set-dist (execute (checked-staged-transcript-program A)$
 $adversary-initial-state)$

let $?s =$

```

    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  show wp-event verify-monad
    (trace-fri-bad-with-sampled-layer-chain ?s) ?s ≤ R + MF
  by (rule wp-trace-fri-sampled-bound-from-full-cover-and-missing
      [OF full-bound[OF builder] missing-full-bound[OF builder]])
next
fix data attacker-state
assume builder:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  show wp-event verify-monad
    (trace-fri-reachable-missing-sampled-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤ MS
  by (rule missing-sampled-bound[OF builder])
qed

lemma composition-fri-bad-with-partial-candidates-bound-from-full-cover-and-missing:
  fixes R MF MS :: prob
  assumes full-bound:
    ∧data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
      wp-event verify-monad
        (composition-fri-bad-with-full-cover-layer-chain
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)) bad)
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)) ≤ R
  and missing-full-bound:
    ∧data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
      wp-event verify-monad
        (composition-fri-sampled-missing-full-cover
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)) bad)
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)) ≤ MF
  and missing-sampled-bound:
    ∧data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)

```

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-reachable-missing-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq MS$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq (R + MF) + MS$
proof (rule
 $\text{composition-fri-bad-with-partial-candidates-bound-from-sampled-and-missing})$
fix data attacker-state
assume builder:
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$
let ?s =
 $\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})$
show wp-event verify-monad
 $(\text{composition-fri-bad-with-sampled-layer-chain } ?s) \text{ ?s} \leq R + MF$
by (rule wp-composition-fri-sampled-bound-from-full-cover-and-missing
 $[\text{OF full-bound}[\text{OF builder}] \text{ missing-full-bound}[\text{OF builder}]])$
next
fix data attacker-state
assume builder:
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$
show wp-event verify-monad
 $(\text{composition-fri-reachable-missing-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq MS$
by (rule missing-sampled-bound[OF builder])
qed

lemma trace-fri-bad-with-empty-header-candidates-bound-from-full-cover-and-missing:
fixes R MF MS :: prob
assumes full-bound:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$

(trace-fri-bad-with-full-cover-layer-chain
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) bad)
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq R$
and missing-full-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-sampled-missing-full-cover
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) bad)
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq MF$
and missing-sampled-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-reachable-missing-sampled-layer-chain
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq MS$
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq (R + MF) + MS$
proof (rule
 trace-fri-bad-with-empty-header-candidates-bound-from-sampled-and-missing)
fix data attacker-state
assume builder:
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state)
let ?s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
show wp-event verify-monad
 (trace-fri-bad-with-sampled-layer-chain ?s) ?s $\leq R + MF$
by (rule wp-trace-fri-sampled-bound-from-full-cover-and-missing
 [OF full-bound[OF builder] missing-full-bound[OF builder]])
next
fix data attacker-state
assume builder:

```

    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  show wp-event verify-monad
    (trace-fri-reachable-missing-sampled-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤ MS
  by (rule missing-sampled-bound[OF builder])
qed

end

end

```

```

theory Soundness-FRI-Full-Cover-Reasons
imports Soundness-FRI-Refined-Reachable
begin

```

Diagnostic split for missing full sampled FRI cover.

The predicates below do not add assumptions. They expose which clause of *generic-fri-sampled-layer-chain-j* must fail when verifier sampled layer-chain evidence exists but the corresponding full-cover witness is absent. Later proof layers should bound these failures by existing Merkle/hash/proximity side events.

```

context soundness
begin

```

definition *generic-fri-sampled-full-cover-failure*

```

:: ('f list ⇒ bool) ⇒ 'f list ⇒ nat ⇒
  'f list ⇒ 'f list ⇒ 'f ⇒ nat list ⇒
  'f list list list ⇒ 'f list list ⇒ 'f list list ⇒
  (nat ⇒ 'f list ⇒ 'f set) ⇒ bool

```

where

```

generic-fri-sampled-full-cover-failure low-degree candidate-table
  degree-bound roots challenges final-value query-idxs round-layers
  doms layers bad ⟷
  ¬ fri-multiround-proximity-steps (length challenges) degree-bound
    (fri-layer-lengths (length challenges) (clength * scale))
    (map (λi. 2 ^ i) [0..<length challenges])
    doms layers bad ∨
  ¬ fri-sampled-layers-cover-fold-indices roots query-idxs ∨
  ¬ (∀ layer-idx < length challenges.
    2 dvd fri-evidence-layer-len roots layer-idx) ∨
  ¬ (∀ layer-idx < length challenges.
    fri-evidence-layer-len roots layer-idx * 2 ^ layer-idx =
      clength * scale) ∨
  ¬ (∀ layer-idx < length challenges.

```


$fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx \leq$
 $\quad length\ (doms\ !\ layer\text{-}idx)) \vee$
 $\neg (\forall\ layer\text{-}idx < length\ challenges.$
 $\quad fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx \leq$
 $\quad\quad length\ (layers\ !\ layer\text{-}idx)) \vee$
 $\neg (\forall\ layer\text{-}idx < length\ challenges.$
 $\quad fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx\ div\ 2 \leq$
 $\quad\quad length\ (doms\ !\ Suc\ layer\text{-}idx)) \vee$
 $\neg (\forall\ layer\text{-}idx < length\ challenges.$
 $\quad fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx\ div\ 2 \leq$
 $\quad\quad length\ (layers\ !\ Suc\ layer\text{-}idx)) \vee$
 $\neg (\forall\ layer\text{-}idx < length\ challenges.$
 $\quad \forall\ idx < fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx\ div\ 2.$
 $\quad\quad doms\ !\ Suc\ layer\text{-}idx\ !\ idx =$
 $\quad\quad\quad (doms\ !\ layer\text{-}idx\ !\ idx)^2) \vee$
 $\neg (\forall\ layer\text{-}idx < length\ challenges.$
 $\quad \forall\ idx < fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx\ div\ 2.$
 $\quad\quad doms\ !\ layer\text{-}idx\ !\ idx = (h\ ^\wedge\ idx * shift) ^\wedge (2\ ^\wedge\ layer\text{-}idx)) \vee$
 $fri\text{-}table\text{-}low\text{-}degree\text{-}on\ degree\text{-}bound\ eval\text{-}domain\ candidate\text{-}table \vee$
 $\neg\ fri\text{-}table\text{-}low\text{-}degree\text{-}on$
 $\quad (fri\text{-}degree\text{-}after\ (length\ challenges)\ degree\text{-}bound)$
 $\quad (doms\ !\ length\ challenges)\ (layers\ !\ length\ challenges)$

definition *generic-fri-full-cover-proximity-failure*

$:: nat \Rightarrow 'f\ list \Rightarrow 'f\ list\ list \Rightarrow 'f\ list\ list \Rightarrow$
 $\quad (nat \Rightarrow 'f\ list \Rightarrow 'f\ set) \Rightarrow bool$

where

$generic\text{-}fri\text{-}full\text{-}cover\text{-}proximity\text{-}failure\ degree\text{-}bound\ challenges$
 $\quad doms\ layers\ bad \longleftrightarrow$
 $\neg\ fri\text{-}multiround\text{-}proximity\text{-}steps\ (length\ challenges)\ degree\text{-}bound$
 $\quad (fri\text{-}layer\text{-}lengths\ (length\ challenges)\ (clength * scale))$
 $\quad (map\ (\lambda i. 2\ ^\wedge\ i)\ [0..<length\ challenges])$
 $\quad doms\ layers\ bad$

definition *generic-fri-full-cover-index-failure*

$:: 'f\ list \Rightarrow nat\ list \Rightarrow bool$

where

$generic\text{-}fri\text{-}full\text{-}cover\text{-}index\text{-}failure\ roots\ query\text{-}idxs \longleftrightarrow$
 $\neg\ fri\text{-}sampled\text{-}layers\text{-}cover\text{-}fold\text{-}indices\ roots\ query\text{-}idxs$

definition *generic-fri-full-cover-shape-failure*

$:: 'f\ list \Rightarrow 'f\ list \Rightarrow 'f\ list\ list \Rightarrow 'f\ list\ list \Rightarrow bool$

where

$generic\text{-}fri\text{-}full\text{-}cover\text{-}shape\text{-}failure\ roots\ challenges\ doms\ layers \longleftrightarrow$
 $\neg (\forall\ layer\text{-}idx < length\ challenges.$
 $\quad 2\ dvd\ fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx) \vee$
 $\neg (\forall\ layer\text{-}idx < length\ challenges.$
 $\quad fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx * 2\ ^\wedge\ layer\text{-}idx =$
 $\quad\quad clength * scale) \vee$

$\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} \leq$
 $\text{length (doms ! layer-idx)}) \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} \leq$
 $\text{length (layers ! layer-idx)}) \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx div 2} \leq$
 $\text{length (doms ! Suc layer-idx)}) \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx div 2} \leq$
 $\text{length (layers ! Suc layer-idx)}) \vee$
 $\text{length (doms ! length challenges)} \neq$
 $\text{length (layers ! length challenges)}$

definition *generic-fri-full-cover-arithmetic-shape-failure*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

generic-fri-full-cover-arithmetic-shape-failure roots challenges

$\longleftrightarrow$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $2 \text{ dvd fri-evidence-layer-len roots layer-idx}) \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} * 2 \wedge \text{layer-idx} =$
 $\text{clength} * \text{scale})$

definition *generic-fri-full-cover-witness-length-failure*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

generic-fri-full-cover-witness-length-failure roots challenges doms layers

$\longleftrightarrow$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} \leq$
 $\text{length (doms ! layer-idx)}) \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} \leq$
 $\text{length (layers ! layer-idx)}) \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx div 2} \leq$
 $\text{length (doms ! Suc layer-idx)}) \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx div 2} \leq$
 $\text{length (layers ! Suc layer-idx)}) \vee$
 $\text{length (doms ! length challenges)} \neq$
 $\text{length (layers ! length challenges)}$

lemma *generic-fri-full-cover-shape-failure-split:*

assumes *generic-fri-full-cover-shape-failure roots challenges doms layers*

shows

```

    generic-fri-full-cover-arithmetic-shape-failure roots challenges  $\vee$ 
    generic-fri-full-cover-witness-length-failure roots challenges
    doms layers
using assms
unfolding generic-fri-full-cover-shape-failure-def
    generic-fri-full-cover-arithmetic-shape-failure-def
    generic-fri-full-cover-witness-length-failure-def
by blast

lemma ceil-log-Suc-le-power-local:
  assumes  $d < 2^N$ 
  shows  $\text{ceil-log } (\text{Suc } d) \leq N$ 
proof (cases  $d = 0$ )
  case True
    then show ?thesis
    unfolding ceil-log-def by simp
  next
    case False
    then have d-pos:  $0 < d$ 
    by simp
    have  $2^{\text{floor-log } d} \leq d$ 
    by (rule floor-log-exp2-le[OF d-pos])
    also have  $\dots < 2^N$ 
    using assms .
    finally have  $\text{floor-log } d < N$ 
    by (subst (asm) power-strict-increasing-iff) simp-all
    then show ?thesis
    using False unfolding ceil-log-def by simp
qed

lemma ceil-log-le-power-bound:
  assumes  $n \leq 2^N$ 
  shows  $\text{ceil-log } n \leq N$ 
proof (cases  $n$ )
  case 0
    then show ?thesis
    unfolding ceil-log-def by simp
  next
    case (Suc  $d$ )
    then have  $d < 2^N$ 
    using assms by simp
    then show ?thesis
    unfolding Suc by (rule ceil-log-Suc-le-power-local)
qed

lemma generic-fri-full-cover-arithmetic-shape-failure-false:
  assumes challenges-len:  $\text{length challenges} = \text{length roots}$ 
  and eval-power:  $\text{clength} * \text{scale} = 2^N$ 
  and rounds-bound:  $\text{length challenges} \leq N$ 

```

```

shows  $\neg$  generic-fri-full-cover-arithmetic-shape-failure roots challenges
proof -
  have even:
     $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$ 
       $2 \text{ dvd fri-evidence-layer-len roots layer-idx}$ 
  proof -
    fix layer-idx
    assume layer-bound:  $\text{layer-idx} < \text{length challenges}$ 
    have layer-root-bound:  $\text{layer-idx} < \text{length roots}$ 
      using layer-bound challenges-len by simp
    have suc-le:  $\text{Suc layer-idx} \leq N$ 
      using layer-bound rounds-bound by simp
    have  $(2::\text{nat}) \wedge \text{Suc layer-idx} \text{ dvd } (2::\text{nat}) \wedge N$ 
      by (rule le-imp-power-dvd[OF suc-le])
    then have dvd-eval:  $2 \wedge \text{Suc layer-idx} \text{ dvd } \text{clength} * \text{scale}$ 
      using eval-power by simp
    show  $2 \text{ dvd fri-evidence-layer-len roots layer-idx}$ 
      unfolding fri-evidence-layer-len-def
      by (rule fri-layer-lengths-even-if-dvd[OF layer-root-bound dvd-eval])
    qed
  have product:
     $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$ 
       $\text{fri-evidence-layer-len roots layer-idx} * 2 \wedge \text{layer-idx} =$ 
         $\text{clength} * \text{scale}$ 
  proof -
    fix layer-idx
    assume layer-bound:  $\text{layer-idx} < \text{length challenges}$ 
    have layer-root-bound:  $\text{layer-idx} < \text{length roots}$ 
      using layer-bound challenges-len by simp
    have le-N:  $\text{layer-idx} \leq N$ 
      using layer-bound rounds-bound by simp
    have  $(2::\text{nat}) \wedge \text{layer-idx} \text{ dvd } (2::\text{nat}) \wedge N$ 
      by (rule le-imp-power-dvd[OF le-N])
    then have dvd-eval:  $2 \wedge \text{layer-idx} \text{ dvd } \text{clength} * \text{scale}$ 
      using eval-power by simp
    show  $\text{fri-evidence-layer-len roots layer-idx} * 2 \wedge \text{layer-idx} =$ 
       $\text{clength} * \text{scale}$ 
      unfolding fri-evidence-layer-len-def
      by (rule fri-layer-lengths-round-product-if-dvd
        [OF layer-root-bound dvd-eval])
    qed
  show ?thesis
    unfolding generic-fri-full-cover-arithmetic-shape-failure-def
    using even product by simp
qed

definition generic-fri-full-cover-domain-failure
  :: 'f list  $\Rightarrow$  'f list  $\Rightarrow$  'f list list  $\Rightarrow$  bool
where

```

generic-fri-full-cover-domain-failure $\text{roots challenges doms} \longleftrightarrow$
 $\neg (\forall \text{layer-idx} < \text{length challenges}.$
 $\quad \forall \text{idx} < \text{fri-evidence-layer-len roots layer-idx div } 2.$
 $\quad \text{doms ! Suc layer-idx ! idx} =$
 $\quad (\text{doms ! layer-idx ! idx})^2 \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges}.$
 $\quad \forall \text{idx} < \text{fri-evidence-layer-len roots layer-idx div } 2.$
 $\quad \text{doms ! layer-idx ! idx} = (h \wedge \text{idx} * \text{shift}) \wedge (2 \wedge \text{layer-idx}))$

definition *generic-fri-full-cover-start-low-failure*

$:: \text{nat} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

generic-fri-full-cover-start-low-failure degree-bound candidate-table
 $\longleftrightarrow \text{fri-table-low-degree-on degree-bound eval-domain candidate-table}$

definition *generic-fri-full-cover-final-low-failure*

$:: \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

generic-fri-full-cover-final-low-failure degree-bound challenges
 $\text{doms layers} \longleftrightarrow$
 $\neg \text{fri-table-low-degree-on}$
 $(\text{fri-degree-after} (\text{length challenges}) \text{degree-bound})$
 $(\text{doms ! length challenges}) (\text{layers ! length challenges})$

definition *generic-fri-full-cover-final-length-failure*

$:: 'f \text{ list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

generic-fri-full-cover-final-length-failure challenges doms layers
 $\longleftrightarrow$
 $\text{length} (\text{doms ! length challenges}) \neq$
 $\text{length} (\text{layers ! length challenges})$

lemma *fri-final-constant-consistent-low-degree-if-lengths:*

assumes *final: fri-final-constant-consistent table final-value*

and *len: length table = length fri-dom*

shows *fri-table-low-degree-on d fri-dom table*

proof –

have *table: table = map (poly [:final-value:]) fri-dom*

proof (*rule nth-equalityI*)

show *length table = length (map (poly [:final-value:]) fri-dom)*

using *len* **by** *simp*

next

fix *i*

assume *i-bound: i < length table*

have *table ! i = final-value*

by (*rule fri-final-constant-consistent-nth[OF final i-bound]*)

then show *table ! i = map (poly [:final-value:]) fri-dom ! i*

using *i-bound len* **by** *simp*

qed

show *?thesis*
unfolding *fri-table-low-degree-on-def*
using *table* **by** (intro *exI*[**where** *x*=[:*final-value*:]]) *simp*
qed

lemma *generic-fri-final-low-failure-imp-length-failure*:
assumes *chain*:
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers
and *final-low*:
generic-fri-full-cover-final-low-failure degree-bound challenges
doms layers
shows *generic-fri-full-cover-final-length-failure challenges doms layers*
proof (unfold *generic-fri-full-cover-final-length-failure-def*, rule *notI*)
assume *len*:
length (doms ! length challenges) =
length (layers ! length challenges)
have *final*:
fri-final-constant-consistent (layers ! length challenges) final-value
by (rule *generic-fri-sampled-layer-chain-evidenceD(6)*[*OF chain*])
have *low*:
fri-table-low-degree-on
(fri-degree-after (length challenges) degree-bound)
(doms ! length challenges) (layers ! length challenges)
by (rule *fri-final-constant-consistent-low-degree-if-lengths*[*OF final*])
(rule len[symmetric])
show *False*
using *final-low low*
unfolding *generic-fri-full-cover-final-low-failure-def* **by** *contradiction*
qed

definition *generic-fri-sampled-full-cover-failure-case*
 $:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list list} \Rightarrow$
 $(\text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow \text{bool}$

where
generic-fri-sampled-full-cover-failure-case low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad $\longleftrightarrow$
generic-fri-full-cover-proximity-failure degree-bound challenges
doms layers bad $\vee$
generic-fri-full-cover-index-failure roots query-idxs $\vee$
generic-fri-full-cover-shape-failure roots challenges doms layers $\vee$
generic-fri-full-cover-domain-failure roots challenges doms $\vee$
generic-fri-full-cover-start-low-failure degree-bound candidate-table $\vee$
generic-fri-full-cover-final-low-failure degree-bound challenges
doms layers

lemma *generic-fri-sampled-full-cover-failure-iff-case:*
generic-fri-sampled-full-cover-failure low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad $\longleftrightarrow$
generic-fri-sampled-full-cover-failure-case low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad
unfolding *generic-fri-sampled-full-cover-failure-def*
generic-fri-sampled-full-cover-failure-case-def
generic-fri-full-cover-proximity-failure-def
generic-fri-full-cover-index-failure-def
generic-fri-full-cover-shape-failure-def
generic-fri-full-cover-domain-failure-def
generic-fri-full-cover-start-low-failure-def
generic-fri-full-cover-final-low-failure-def
fri-table-low-degree-on-def
by *auto*

lemma *generic-fri-sampled-full-cover-failure-cases:*
assumes
generic-fri-sampled-full-cover-failure low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad
shows
generic-fri-full-cover-proximity-failure degree-bound challenges
doms layers bad $\vee$
generic-fri-full-cover-index-failure roots query-idxs $\vee$
generic-fri-full-cover-shape-failure roots challenges doms layers $\vee$
generic-fri-full-cover-domain-failure roots challenges doms $\vee$
generic-fri-full-cover-start-low-failure degree-bound candidate-table $\vee$
generic-fri-full-cover-final-low-failure degree-bound challenges
doms layers
using *assms*
unfolding *generic-fri-sampled-full-cover-failure-iff-case*
generic-fri-sampled-full-cover-failure-case-def
by *blast*

definition *trace-fri-sampled-full-cover-failure*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \text{ list} \Rightarrow \text{nat} \Rightarrow f \text{ list} \Rightarrow f \text{ set}) \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
trace-fri-sampled-full-cover-failure s bad out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
trace-fri-partial-candidate-opening-evidence s out trace-roots

trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table $\wedge$
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers $\wedge$
generic-fri-sampled-full-cover-failure trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers (bad trace-table))

definition *trace-fri-sampled-full-cover-failure-case*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-sampled-full-cover-failure-case s bad out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{ trace-table doms layers.}$
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table $\wedge$
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers $\wedge$
generic-fri-sampled-full-cover-failure-case trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers (bad trace-table))

definition *trace-fri-sampled-full-cover-proximity-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-sampled-full-cover-proximity-failure s bad out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{ trace-table doms layers.}$
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table $\wedge$
generic-fri-sampled-layer-chain-evidence trace-table-low-degree

$trace-table\ (clength - 1)\ trace-roots\ trace-bs\ trace-final$
 $fri-query-idxs\ trace-round-layers\ doms\ layers \wedge$
 $generic-fri-full-cover-proximity-failure\ (clength - 1)\ trace-bs$
 $doms\ layers\ (bad\ trace-table))$

definition *trace-fri-sampled-full-cover-index-failure*

$:: (f, 'a)\ protocol-channel-scheme \Rightarrow$
 $(unit\ list \times (f, 'a)\ protocol-channel-scheme)\ option \Rightarrow bool$

where

$trace-fri-sampled-full-cover-index-failure\ s\ out \longleftrightarrow$
 $(\exists\ trace-roots\ trace-bs\ trace-final\ dg\ composition-roots$
 $\quad composition-bs\ composition-final\ fri-query-idxs\ trace-round-layers$
 $\quad composition-round-layers\ fr\ candidate-query-idxs\ trace-openings$
 $\quad trace-table\ doms\ layers.$
 $trace-fri-partial-candidate-opening-evidence\ s\ out\ trace-roots$
 $\quad trace-bs\ trace-final\ dg\ composition-roots\ composition-bs$
 $\quad composition-final\ fri-query-idxs\ trace-round-layers$
 $\quad composition-round-layers\ fr\ candidate-query-idxs\ trace-openings$
 $\quad trace-table \wedge$
 $generic-fri-sampled-layer-chain-evidence\ trace-table-low-degree$
 $\quad trace-table\ (clength - 1)\ trace-roots\ trace-bs\ trace-final$
 $\quad fri-query-idxs\ trace-round-layers\ doms\ layers \wedge$
 $generic-fri-full-cover-index-failure\ trace-roots\ fri-query-idxs)$

definition *trace-fri-sampled-full-cover-shape-failure*

$:: (f, 'a)\ protocol-channel-scheme \Rightarrow$
 $(unit\ list \times (f, 'a)\ protocol-channel-scheme)\ option \Rightarrow bool$

where

$trace-fri-sampled-full-cover-shape-failure\ s\ out \longleftrightarrow$
 $(\exists\ trace-roots\ trace-bs\ trace-final\ dg\ composition-roots$
 $\quad composition-bs\ composition-final\ fri-query-idxs\ trace-round-layers$
 $\quad composition-round-layers\ fr\ candidate-query-idxs\ trace-openings$
 $\quad trace-table\ doms\ layers.$
 $trace-fri-partial-candidate-opening-evidence\ s\ out\ trace-roots$
 $\quad trace-bs\ trace-final\ dg\ composition-roots\ composition-bs$
 $\quad composition-final\ fri-query-idxs\ trace-round-layers$
 $\quad composition-round-layers\ fr\ candidate-query-idxs\ trace-openings$
 $\quad trace-table \wedge$
 $generic-fri-sampled-layer-chain-evidence\ trace-table-low-degree$
 $\quad trace-table\ (clength - 1)\ trace-roots\ trace-bs\ trace-final$
 $\quad fri-query-idxs\ trace-round-layers\ doms\ layers \wedge$
 $generic-fri-full-cover-shape-failure\ trace-roots\ trace-bs$
 $\quad doms\ layers)$

definition *trace-fri-sampled-full-cover-arithmetic-shape-failure*

$:: (f, 'a)\ protocol-channel-scheme \Rightarrow$
 $(unit\ list \times (f, 'a)\ protocol-channel-scheme)\ option \Rightarrow bool$

where

$trace-fri-sampled-full-cover-arithmetic-shape-failure\ s\ out \longleftrightarrow$

$(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
 $\text{trace-fri-partial-candidate-opening-evidence s out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence trace-table-low-degree}$
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers} \wedge$
 $\text{generic-fri-full-cover-arithmetic-shape-failure trace-roots trace-bs})$

definition *trace-fri-sampled-full-cover-witness-length-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-sampled-full-cover-witness-length-failure s out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
 $\text{trace-fri-partial-candidate-opening-evidence s out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence trace-table-low-degree}$
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers} \wedge$
 $\text{generic-fri-full-cover-witness-length-failure trace-roots trace-bs}$
 $\text{doms layers})$

definition *trace-fri-sampled-full-cover-domain-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-sampled-full-cover-domain-failure s out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
 $\text{trace-fri-partial-candidate-opening-evidence s out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence trace-table-low-degree}$

$trace-table\ (clength - 1)\ trace-roots\ trace-bs\ trace-final$
 $fri-query-idxs\ trace-round-layers\ doms\ layers \wedge$
 $generic-fri-full-cover-domain-failure\ trace-roots\ trace-bs\ doms)$

definition *trace-fri-sampled-full-cover-start-low-failure*

$:: (f, 'a)\ protocol-channel-scheme \Rightarrow$
 $(unit\ list \times (f, 'a)\ protocol-channel-scheme)\ option \Rightarrow bool$

where

$trace-fri-sampled-full-cover-start-low-failure\ s\ out \longleftrightarrow$
 $(\exists\ trace-roots\ trace-bs\ trace-final\ dg\ composition-roots$
 $\quad composition-bs\ composition-final\ fri-query-idxs\ trace-round-layers$
 $\quad composition-round-layers\ fr\ candidate-query-idxs\ trace-openings$
 $\quad trace-table\ doms\ layers.$
 $trace-fri-partial-candidate-opening-evidence\ s\ out\ trace-roots$
 $\quad trace-bs\ trace-final\ dg\ composition-roots\ composition-bs$
 $\quad composition-final\ fri-query-idxs\ trace-round-layers$
 $\quad composition-round-layers\ fr\ candidate-query-idxs\ trace-openings$
 $\quad trace-table \wedge$
 $generic-fri-sampled-layer-chain-evidence\ trace-table-low-degree$
 $\quad trace-table\ (clength - 1)\ trace-roots\ trace-bs\ trace-final$
 $\quad fri-query-idxs\ trace-round-layers\ doms\ layers \wedge$
 $generic-fri-full-cover-start-low-failure\ (clength - 1)\ trace-table)$

definition *trace-fri-sampled-full-cover-final-low-failure*

$:: (f, 'a)\ protocol-channel-scheme \Rightarrow$
 $(unit\ list \times (f, 'a)\ protocol-channel-scheme)\ option \Rightarrow bool$

where

$trace-fri-sampled-full-cover-final-low-failure\ s\ out \longleftrightarrow$
 $(\exists\ trace-roots\ trace-bs\ trace-final\ dg\ composition-roots$
 $\quad composition-bs\ composition-final\ fri-query-idxs\ trace-round-layers$
 $\quad composition-round-layers\ fr\ candidate-query-idxs\ trace-openings$
 $\quad trace-table\ doms\ layers.$
 $trace-fri-partial-candidate-opening-evidence\ s\ out\ trace-roots$
 $\quad trace-bs\ trace-final\ dg\ composition-roots\ composition-bs$
 $\quad composition-final\ fri-query-idxs\ trace-round-layers$
 $\quad composition-round-layers\ fr\ candidate-query-idxs\ trace-openings$
 $\quad trace-table \wedge$
 $generic-fri-sampled-layer-chain-evidence\ trace-table-low-degree$
 $\quad trace-table\ (clength - 1)\ trace-roots\ trace-bs\ trace-final$
 $\quad fri-query-idxs\ trace-round-layers\ doms\ layers \wedge$
 $generic-fri-full-cover-final-low-failure\ (clength - 1)\ trace-bs$
 $\quad doms\ layers)$

definition *trace-fri-sampled-full-cover-final-length-failure*

$:: (f, 'a)\ protocol-channel-scheme \Rightarrow$
 $(unit\ list \times (f, 'a)\ protocol-channel-scheme)\ option \Rightarrow bool$

where

$trace-fri-sampled-full-cover-final-length-failure\ s\ out \longleftrightarrow$
 $(\exists\ trace-roots\ trace-bs\ trace-final\ dg\ composition-roots$

composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table doms layers.
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table $\wedge$
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers $\wedge$
generic-fri-full-cover-final-length-failure trace-bs doms layers)

definition *composition-fri-sampled-full-cover-failure*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-failure s bad out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers.
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
 $\wedge$
generic-fri-sampled-full-cover-failure
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
(bad fri-dg composition-table))

definition *composition-fri-sampled-full-cover-failure-case*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-failure-case s bad out $\longleftrightarrow$

$(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table doms layers.}$
 $\text{composition-fri-partial-candidate-opening-evidence s out trace-roots}$
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings trace-table}$
 $\text{composition-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence}$
 $(\text{composition-table-low-degree (to-nat fri-dg)}) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs composition-round-layers doms layers}$
 $\wedge$
 $\text{generic-fri-sampled-full-cover-failure-case}$
 $(\text{composition-table-low-degree (to-nat fri-dg)}) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs composition-round-layers doms layers}$
 $(\text{bad fri-dg composition-table}))$

definition *composition-fri-sampled-full-cover-proximity-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-sampled-full-cover-proximity-failure s bad out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table doms layers.}$
 $\text{composition-fri-partial-candidate-opening-evidence s out trace-roots}$
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings trace-table}$
 $\text{composition-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence}$
 $(\text{composition-table-low-degree (to-nat fri-dg)}) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs composition-round-layers doms layers}$
 $\wedge$
 $\text{generic-fri-full-cover-proximity-failure (to-nat fri-dg)})$

composition-bs doms layers (bad fri-dg composition-table))

definition *composition-fri-sampled-full-cover-index-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-index-failure s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{ opening-composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{ f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table doms layers.}$
 $\text{ composition-fri-partial-candidate-opening-evidence s out trace-roots}$
 $\text{ trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{ composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings trace-table}$
 $\text{ composition-table} \wedge$
 $\text{ generic-fri-sampled-layer-chain-evidence}$
 $(\text{ composition-table-low-degree (to-nat fri-dg)}) \text{ composition-table}$
 $(\text{ to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs composition-round-layers doms layers}$
 $\wedge$
 $\text{ generic-fri-full-cover-index-failure opening-composition-roots}$
 $\text{ fri-query-idxs})$

definition *composition-fri-sampled-full-cover-shape-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-shape-failure s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{ opening-composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{ f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table doms layers.}$
 $\text{ composition-fri-partial-candidate-opening-evidence s out trace-roots}$
 $\text{ trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{ composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings trace-table}$
 $\text{ composition-table} \wedge$
 $\text{ generic-fri-sampled-layer-chain-evidence}$
 $(\text{ composition-table-low-degree (to-nat fri-dg)}) \text{ composition-table}$
 $(\text{ to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$

$\text{composition-final fri-query-idxs composition-round-layers doms layers}$
 $\wedge$
 $\text{generic-fri-full-cover-shape-failure opening-composition-roots}$
 $\text{composition-bs doms layers})$

definition *composition-fri-sampled-full-cover-arithmetic-shape-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-sampled-full-cover-arithmetic-shape-failure } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{ opening-composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{ f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table doms layers.}$
 $\text{composition-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{ trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{ composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings trace-table}$
 $\text{ composition-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence}$
 $(\text{composition-table-low-degree (to-nat fri-dg)}) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs composition-round-layers doms layers}$
 $\wedge$
 $\text{generic-fri-full-cover-arithmetic-shape-failure}$
 $\text{ opening-composition-roots composition-bs})$

definition *composition-fri-sampled-full-cover-witness-length-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-sampled-full-cover-witness-length-failure } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{ opening-composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{ f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table doms layers.}$
 $\text{composition-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{ trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{ composition-fri-roots final trace-query-idxs trace-openings}$

composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
 $\wedge$
generic-fri-full-cover-witness-length-failure
opening-composition-roots composition-bs doms layers)

definition *composition-fri-sampled-full-cover-domain-failure*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-domain-failure s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table doms layers.}$
composition-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings trace-table}$
 $\text{composition-table} \wedge$
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
 $\wedge$
generic-fri-full-cover-domain-failure opening-composition-roots
composition-bs doms)

definition *composition-fri-sampled-full-cover-start-low-failure*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-start-low-failure s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table doms layers.}$
composition-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$

composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
 $\wedge$
generic-fri-full-cover-start-low-failure (to-nat fri-dg)
composition-table)

definition *composition-fri-sampled-full-cover-final-low-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-final-low-failure s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers.
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
 $\wedge$
generic-fri-full-cover-final-low-failure (to-nat fri-dg)
composition-bs doms layers)

definition *composition-fri-sampled-full-cover-final-length-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-final-length-failure s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs

composition-openings trace-table composition-table doms layers.
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
 $\wedge$
generic-fri-full-cover-final-length-failure composition-bs doms
layers)

lemma *trace-fri-sampled-full-cover-failure-iff-case:*
trace-fri-sampled-full-cover-failure s bad out $\longleftrightarrow$
trace-fri-sampled-full-cover-failure-case s bad out
unfolding *trace-fri-sampled-full-cover-failure-def*
trace-fri-sampled-full-cover-failure-case-def
by (*simp only: generic-fri-sampled-full-cover-failure-iff-case*)

lemma *composition-fri-sampled-full-cover-failure-iff-case:*
composition-fri-sampled-full-cover-failure s bad out $\longleftrightarrow$
composition-fri-sampled-full-cover-failure-case s bad out
unfolding *composition-fri-sampled-full-cover-failure-def*
composition-fri-sampled-full-cover-failure-case-def
by (*simp only: generic-fri-sampled-full-cover-failure-iff-case*)

lemma *trace-fri-sampled-full-cover-failure-case-split:*
assumes *trace-fri-sampled-full-cover-failure-case s bad out*
shows
trace-fri-sampled-full-cover-proximity-failure s bad out $\vee$
trace-fri-sampled-full-cover-index-failure s out $\vee$
trace-fri-sampled-full-cover-shape-failure s out $\vee$
trace-fri-sampled-full-cover-domain-failure s out $\vee$
trace-fri-sampled-full-cover-start-low-failure s out $\vee$
trace-fri-sampled-full-cover-final-low-failure s out
using *assms*
unfolding *trace-fri-sampled-full-cover-failure-case-def*
trace-fri-sampled-full-cover-proximity-failure-def
trace-fri-sampled-full-cover-index-failure-def
trace-fri-sampled-full-cover-shape-failure-def
trace-fri-sampled-full-cover-domain-failure-def
trace-fri-sampled-full-cover-start-low-failure-def
trace-fri-sampled-full-cover-final-low-failure-def
generic-fri-sampled-full-cover-failure-case-def
by *meson*

lemma *composition-fri-sampled-full-cover-failure-case-split:*
assumes *composition-fri-sampled-full-cover-failure-case s bad out*
shows
composition-fri-sampled-full-cover-proximity-failure s bad out $\vee$
composition-fri-sampled-full-cover-index-failure s out $\vee$
composition-fri-sampled-full-cover-shape-failure s out $\vee$
composition-fri-sampled-full-cover-domain-failure s out $\vee$
composition-fri-sampled-full-cover-start-low-failure s out $\vee$
composition-fri-sampled-full-cover-final-low-failure s out
using *assms*
unfolding *composition-fri-sampled-full-cover-failure-case-def*
composition-fri-sampled-full-cover-proximity-failure-def
composition-fri-sampled-full-cover-index-failure-def
composition-fri-sampled-full-cover-shape-failure-def
composition-fri-sampled-full-cover-domain-failure-def
composition-fri-sampled-full-cover-start-low-failure-def
composition-fri-sampled-full-cover-final-low-failure-def
generic-fri-sampled-full-cover-failure-case-def
by *meson*

lemma *trace-fri-sampled-full-cover-shape-failure-split:*
assumes *trace-fri-sampled-full-cover-shape-failure s out*
shows
trace-fri-sampled-full-cover-arithmetic-shape-failure s out $\vee$
trace-fri-sampled-full-cover-witness-length-failure s out
proof —
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table doms layers
where *evidence:*
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *chain:*
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
and *shape:*
generic-fri-full-cover-shape-failure trace-roots trace-bs
doms layers
unfolding *trace-fri-sampled-full-cover-shape-failure-def* **by** *blast*
have *split:*
generic-fri-full-cover-arithmetic-shape-failure trace-roots trace-bs $\vee$
generic-fri-full-cover-witness-length-failure trace-roots trace-bs
doms layers

by (rule generic-fri-full-cover-shape-failure-split[OF shape])
 then show ?thesis
 unfolding trace-fri-sampled-full-cover-arithmetic-shape-failure-def
 trace-fri-sampled-full-cover-witness-length-failure-def
 using evidence chain by meson
 qed

lemma composition-fri-sampled-full-cover-shape-failure-split:
 assumes composition-fri-sampled-full-cover-shape-failure s out
 shows
 composition-fri-sampled-full-cover-arithmetic-shape-failure s out $\vee$
 composition-fri-sampled-full-cover-witness-length-failure s out
proof –
 from assms obtain trace-roots trace-bs trace-final fri-dg
 opening-composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers fr
 f-fri-roots f-final as dg composition-fri-roots final
 trace-query-idxs trace-openings composition-query-idxs
 composition-openings trace-table composition-table doms layers
 where evidence:
 composition-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final fri-dg opening-composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr f-fri-roots f-final as dg
 composition-fri-roots final trace-query-idxs trace-openings
 composition-query-idxs composition-openings trace-table
 composition-table
 and chain:
 generic-fri-sampled-layer-chain-evidence
 (composition-table-low-degree (to-nat fri-dg)) composition-table
 (to-nat fri-dg) opening-composition-roots composition-bs
 composition-final fri-query-idxs composition-round-layers doms layers
 and shape:
 generic-fri-full-cover-shape-failure opening-composition-roots
 composition-bs doms layers
 unfolding composition-fri-sampled-full-cover-shape-failure-def
 by meson
 have split:
 generic-fri-full-cover-arithmetic-shape-failure
 opening-composition-roots composition-bs $\vee$
 generic-fri-full-cover-witness-length-failure
 opening-composition-roots composition-bs doms layers
 by (rule generic-fri-full-cover-shape-failure-split[OF shape])
 then show ?thesis
 unfolding
 composition-fri-sampled-full-cover-arithmetic-shape-failure-def
 composition-fri-sampled-full-cover-witness-length-failure-def
 using evidence chain by meson
 qed

lemma *wp-trace-fri-sampled-full-cover-shape-bound-from-split*:
assumes *arithmetic*:
 wp-event verify-monad
 (*trace-fri-sampled-full-cover-arithmetic-shape-failure s*) $s \leq A$
and *witness*:
 wp-event verify-monad
 (*trace-fri-sampled-full-cover-witness-length-failure s*) $s \leq W$
shows
 wp-event verify-monad
 (*trace-fri-sampled-full-cover-shape-failure s*) $s \leq A + W$
proof –
 have *wp-event verify-monad*
 (*trace-fri-sampled-full-cover-shape-failure s*) s
 $\leq$ *wp-event verify-monad*
 ($\lambda out.$
 trace-fri-sampled-full-cover-arithmetic-shape-failure s out $\vee$
 trace-fri-sampled-full-cover-witness-length-failure s out) s
 by (*rule wp-event-mono*)
 (*rule trace-fri-sampled-full-cover-shape-failure-split*)
 also have $\dots \leq A + W$
 by (*rule order-trans*[*OF wp-event-union-bound*
 add-mono[*OF arithmetic witness*]])
 finally show *?thesis* .
qed

lemma *wp-composition-fri-sampled-full-cover-shape-bound-from-split*:
assumes *arithmetic*:
 wp-event verify-monad
 (*composition-fri-sampled-full-cover-arithmetic-shape-failure s*) $s \leq A$
and *witness*:
 wp-event verify-monad
 (*composition-fri-sampled-full-cover-witness-length-failure s*) $s \leq W$
shows
 wp-event verify-monad
 (*composition-fri-sampled-full-cover-shape-failure s*) $s \leq A + W$
proof –
 have *wp-event verify-monad*
 (*composition-fri-sampled-full-cover-shape-failure s*) s
 $\leq$ *wp-event verify-monad*
 ($\lambda out.$
 composition-fri-sampled-full-cover-arithmetic-shape-failure s out $\vee$
 composition-fri-sampled-full-cover-witness-length-failure s out) s
 by (*rule wp-event-mono*)
 (*rule composition-fri-sampled-full-cover-shape-failure-split*)
 also have $\dots \leq A + W$
 by (*rule order-trans*[*OF wp-event-union-bound*

```

      add-mono[OF arithmetic witness]])
  finally show ?thesis .
qed

lemma wp-event-six-union-bound:
  assumes A: wp-event m A s ≤ a
    and B: wp-event m B s ≤ b
    and C: wp-event m C s ≤ c
    and D: wp-event m D s ≤ d
    and E: wp-event m E s ≤ e
    and F: wp-event m F s ≤ f
  shows wp-event m
    (λx. A x ∨ B x ∨ C x ∨ D x ∨ E x ∨ F x) s
    ≤ a + b + c + d + e + f
proof -
  have EF:
    wp-event m (λx. E x ∨ F x) s ≤ e + f
  by (rule order-trans[OF wp-event-union-bound add-mono[OF E F]])
  have DEF:
    wp-event m (λx. D x ∨ E x ∨ F x) s ≤ d + e + f
proof -
  have wp-event m (λx. D x ∨ E x ∨ F x) s
    ≤ wp-event m D s + wp-event m (λx. E x ∨ F x) s
  by (rule wp-event-union-bound)
  also have ... ≤ d + (e + f)
  by (rule add-mono[OF D EF])
  finally show ?thesis
    by (simp add: add.assoc)
qed
have CDEF:
  wp-event m (λx. C x ∨ D x ∨ E x ∨ F x) s
    ≤ c + d + e + f
proof -
  have wp-event m (λx. C x ∨ D x ∨ E x ∨ F x) s
    ≤ wp-event m C s + wp-event m (λx. D x ∨ E x ∨ F x) s
  by (rule wp-event-union-bound)
  also have ... ≤ c + (d + e + f)
  by (rule add-mono[OF C DEF])
  finally show ?thesis
    by (simp add: add.assoc)
qed
have BCDEF:
  wp-event m (λx. B x ∨ C x ∨ D x ∨ E x ∨ F x) s
    ≤ b + c + d + e + f
proof -
  have wp-event m (λx. B x ∨ C x ∨ D x ∨ E x ∨ F x) s
    ≤ wp-event m B s +
      wp-event m (λx. C x ∨ D x ∨ E x ∨ F x) s
  by (rule wp-event-union-bound)

```

```

also have ...  $\leq b + (c + d + e + f)$ 
  by (rule add-mono[OF B CDEF])
finally show ?thesis
  by (simp add: add.assoc)
qed
have wp-event m
  ( $\lambda x. A\ x \vee B\ x \vee C\ x \vee D\ x \vee E\ x \vee F\ x$ ) s
 $\leq$  wp-event m A s +
  wp-event m ( $\lambda x. B\ x \vee C\ x \vee D\ x \vee E\ x \vee F\ x$ ) s
  by (rule wp-event-union-bound)
also have ...  $\leq a + (b + c + d + e + f)$ 
  by (rule add-mono[OF A BCDEF])
finally show ?thesis
  by (simp add: add.assoc)
qed

lemma wp-trace-fri-sampled-full-cover-failure-case-union-bound:
assumes proximity:
  wp-event verify-monad
  (trace-fri-sampled-full-cover-proximity-failure s bad) s  $\leq P$ 
and index:
  wp-event verify-monad
  (trace-fri-sampled-full-cover-index-failure s) s  $\leq I$ 
and shape:
  wp-event verify-monad
  (trace-fri-sampled-full-cover-shape-failure s) s  $\leq Sh$ 
and domain:
  wp-event verify-monad
  (trace-fri-sampled-full-cover-domain-failure s) s  $\leq D$ 
and start-low:
  wp-event verify-monad
  (trace-fri-sampled-full-cover-start-low-failure s) s  $\leq SL$ 
and final-low:
  wp-event verify-monad
  (trace-fri-sampled-full-cover-final-low-failure s) s  $\leq FL$ 
shows
  wp-event verify-monad
  (trace-fri-sampled-full-cover-failure-case s bad) s
 $\leq P + I + Sh + D + SL + FL$ 
proof -
have step1:
  wp-event verify-monad
  (trace-fri-sampled-full-cover-failure-case s bad) s
 $\leq$ 
  wp-event verify-monad
  ( $\lambda out. trace-fri-sampled-full-cover-proximity-failure\ s\ bad\ out \vee$ 
    trace-fri-sampled-full-cover-index-failure s out  $\vee$ 
    trace-fri-sampled-full-cover-shape-failure s out  $\vee$ 
    trace-fri-sampled-full-cover-domain-failure s out  $\vee$ 

```

$\text{trace-fri-sampled-full-cover-start-low-failure } s \text{ out} \vee$
 $\text{trace-fri-sampled-full-cover-final-low-failure } s \text{ out}) \ s$
by (rule *wp-event-mono*)
(rule *trace-fri-sampled-full-cover-failure-case-split*)
have *step2*:
wp-event verify-monad
 $(\lambda \text{out. trace-fri-sampled-full-cover-proximity-failure } s \text{ bad out} \vee$
 $\text{trace-fri-sampled-full-cover-index-failure } s \text{ out} \vee$
 $\text{trace-fri-sampled-full-cover-shape-failure } s \text{ out} \vee$
 $\text{trace-fri-sampled-full-cover-domain-failure } s \text{ out} \vee$
 $\text{trace-fri-sampled-full-cover-start-low-failure } s \text{ out} \vee$
 $\text{trace-fri-sampled-full-cover-final-low-failure } s \text{ out}) \ s$
 $\leq P + I + Sh + D + SL + FL$
by (rule *wp-event-six-union-bound*
[*OF proximity index shape domain start-low final-low*])
show *?thesis*
by (rule *order-trans*[*OF step1 step2*])
qed

lemma *wp-composition-fri-sampled-full-cover-failure-case-union-bound*:

assumes *proximity*:

wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$

and *index*:

wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-index-failure } s) \ s \leq I$

and *shape*:

wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-shape-failure } s) \ s \leq Sh$

and *domain*:

wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-domain-failure } s) \ s \leq D$

and *start-low*:

wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-start-low-failure } s) \ s \leq SL$

and *final-low*:

wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-final-low-failure } s) \ s \leq FL$

shows

wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-failure-case } s \text{ bad}) \ s$
 $\leq P + I + Sh + D + SL + FL$

proof –

have *step1*:

wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-failure-case } s \text{ bad}) \ s$
 $\leq$
wp-event verify-monad
 $(\lambda \text{out.}$


```

    composition-fri-sampled-full-cover-proximity-failure s bad out ∨
    composition-fri-sampled-full-cover-index-failure s out ∨
    composition-fri-sampled-full-cover-shape-failure s out ∨
    composition-fri-sampled-full-cover-domain-failure s out ∨
    composition-fri-sampled-full-cover-start-low-failure s out ∨
    composition-fri-sampled-full-cover-final-low-failure s out) s
  by (rule wp-event-mono)
    (rule composition-fri-sampled-full-cover-failure-case-split)
have step2:
  wp-event verify-monad
    (λout.
      composition-fri-sampled-full-cover-proximity-failure s bad out ∨
      composition-fri-sampled-full-cover-index-failure s out ∨
      composition-fri-sampled-full-cover-shape-failure s out ∨
      composition-fri-sampled-full-cover-domain-failure s out ∨
      composition-fri-sampled-full-cover-start-low-failure s out ∨
      composition-fri-sampled-full-cover-final-low-failure s out) s
    ≤ P + I + Sh + D + SL + FL
  by (rule wp-event-six-union-bound
    [OF proximity index shape domain start-low final-low])
show ?thesis
  by (rule order-trans[OF step1 step2])
qed

lemma trace-fri-sampled-full-cover-start-low-failure-false:
  ¬ trace-fri-sampled-full-cover-start-low-failure s out
proof
  assume failure: trace-fri-sampled-full-cover-start-low-failure s out
  then obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table doms layers
  where evidence:
    trace-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
  and start-low:
    generic-fri-full-cover-start-low-failure (clength - 1)
    trace-table
  unfolding trace-fri-sampled-full-cover-start-low-failure-def by blast
  have not-low:
    ¬ fri-table-low-degree-on (clength - 1) eval-domain trace-table
  by (rule trace-fri-partial-candidate-opening-evidence-not-low-on-eval-domain
    [OF evidence])
  have low:
    fri-table-low-degree-on (clength - 1) eval-domain trace-table
  using start-low

```

```

    unfolding generic-fri-full-cover-start-low-failure-def .
  show False
    using not-low low by contradiction
qed

lemma wp-trace-fri-sampled-full-cover-start-low-failure-zero:
  wp-event verify-monad
    (trace-fri-sampled-full-cover-start-low-failure s) s ≤ 0
proof -
  have wp-event verify-monad
    (trace-fri-sampled-full-cover-start-low-failure s) s
    ≤ wp-event verify-monad (λ-. False) s
  by (rule wp-event-mono)
    (simp add: trace-fri-sampled-full-cover-start-low-failure-false)
  also have ... = 0
    unfolding wp-event-def wp-def dist-expect-def by simp
  finally show ?thesis .
qed

lemma composition-fri-sampled-full-cover-start-low-failure-false-on-support:
  assumes outcome:
    out ∈ set-dist (execute verify-monad s)
  and failure:
    composition-fri-sampled-full-cover-start-low-failure s out
  shows False
proof -
  from failure obtain trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table doms layers
  where evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  and chain:
    generic-fri-sampled-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms layers
  and start-low:
    generic-fri-full-cover-start-low-failure (to-nat fri-dg)
    composition-table
  unfolding composition-fri-sampled-full-cover-start-low-failure-def

```

```

    by meson
  have fri-opening:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    fri-dg opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
    by (rule composition-fri-partial-candidate-opening-evidenceD(1)
        [OF evidence])
  from fri-opening obtain result final-state where
    out-eq: out = Some (result, final-state)
    unfolding accepted-fri-opening-transcript-def by blast
  have outcome':
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
    using outcome out-eq by simp
  have challenges:
    accepted-fri-challenges s (Some (result, final-state)) trace-bs
    fri-dg composition-bs
    using accepted-fri-opening-transcript-challenges[OF fri-opening]
    out-eq by simp
  have degree-bound: to-nat fri-dg ≤ maxDegree
    by (rule verify-monad-accepted-fri-challenges-degree-bound
        [OF outcome' challenges])
  have not-low:
    ¬ fri-table-low-degree-on (to-nat fri-dg) eval-domain
    composition-table
    by (rule
        composition-fri-partial-candidate-opening-evidence-not-low-declared-on-eval-domain
        [OF evidence degree-bound])
  have low:
    fri-table-low-degree-on (to-nat fri-dg) eval-domain composition-table
    using start-low
    unfolding generic-fri-full-cover-start-low-failure-def .
  show False
    using not-low low by contradiction
qed

lemma wp-composition-fri-sampled-full-cover-start-low-failure-zero:
  wp-event verify-monad
  (composition-fri-sampled-full-cover-start-low-failure s) s ≤ 0
proof -
  have wp-event verify-monad
    (composition-fri-sampled-full-cover-start-low-failure s) s
    ≤ wp-event verify-monad (λ-. False) s
    by (rule wp-event-mono-on-support)
    (rule composition-fri-sampled-full-cover-start-low-failure-false-on-support)
  also have ... = 0
    unfolding wp-event-def wp-def dist-expect-def by simp
  finally show ?thesis .
qed

```

lemma *trace-fri-sampled-full-cover-arithmetic-shape-failure-false:*
assumes *trace-fri-sampled-full-cover-arithmetic-shape-failure s out*
shows *False*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table doms layers
where *chain:*
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength – 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
and *arithmetic:*
generic-fri-full-cover-arithmetic-shape-failure trace-roots trace-bs
unfolding *trace-fri-sampled-full-cover-arithmetic-shape-failure-def*
by *meson*
have *partial:*
generic-fri-partial-evidence trace-table-low-degree trace-table
(clength – 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers
by *(rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])*
have *challenges-len: length trace-bs = length trace-roots*
using *generic-fri-partial-evidence-shapes(2)[OF partial]* .
have *rounds-eq: length trace-bs = ceil-log clength*
using *generic-fri-partial-evidence-shapes(1,2)[OF partial]*
unfolding *fri-round-count-for-degree-bound-def*
using *clength-pos* **by** *simp*
obtain *N* **where** *eval-power: clength * scale = 2 ^ N*
using *eval-domain-length-power* **by** *blast*
have *clength-le-eval: clength ≤ clength * scale*
using *scale-pos* **by** *simp*
have *rounds-bound: length trace-bs ≤ N*
unfolding *rounds-eq*
by *(rule ceil-log-le-power-bound)*
(use clength-le-eval eval-power in simp)
have $\neg$ *generic-fri-full-cover-arithmetic-shape-failure*
trace-roots trace-bs
by *(rule generic-fri-full-cover-arithmetic-shape-failure-false*
[OF challenges-len eval-power rounds-bound])
then show *False*
using *arithmetic* **by** *contradiction*
qed

lemma *wp-trace-fri-sampled-full-cover-arithmetic-shape-failure-zero:*
wp-event verify-monad
(trace-fri-sampled-full-cover-arithmetic-shape-failure s) s ≤ 0
proof –
have *wp-event verify-monad*
(trace-fri-sampled-full-cover-arithmetic-shape-failure s) s

```

    ≤ wp-event verify-monad (λ-. False) s
  by (rule wp-event-mono)
    (rule trace-fri-sampled-full-cover-arithmetic-shape-failure-false)
  also have ... = 0
    unfolding wp-event-def wp-def dist-expect-def by simp
  finally show ?thesis .
qed

lemma composition-fri-sampled-full-cover-arithmetic-shape-failure-false-on-support:
  assumes outcome:
    out ∈ set-dist (execute verify-monad s)
  and failure:
    composition-fri-sampled-full-cover-arithmetic-shape-failure s out
  shows False
proof -
  from failure obtain trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table doms layers
  where evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  and chain:
    generic-fri-sampled-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms layers
  and arithmetic:
    generic-fri-full-cover-arithmetic-shape-failure
    opening-composition-roots composition-bs
  unfolding composition-fri-sampled-full-cover-arithmetic-shape-failure-def
  by meson
  have partial:
    generic-fri-partial-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
  by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
  have challenges-len:
    length composition-bs = length opening-composition-roots
  using generic-fri-partial-evidence-shapes(2)[OF partial] .
  have rounds-eq:

```

```

length composition-bs = ceil-log (to-nat fri-dg + 1)
using generic-fri-partial-evidence-shapes(1,2)[OF partial]
unfolding fri-round-count-for-degree-bound-def by simp
have fri-opening:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  fri-dg opening-composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers
by (rule composition-fri-partial-candidate-opening-evidenceD(1)
  [OF evidence])
from fri-opening obtain result final-state where
  out-eq: out = Some (result, final-state)
  unfolding accepted-fri-opening-transcript-def by blast
have outcome':
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
  using outcome out-eq by simp
have challenges:
  accepted-fri-challenges s (Some (result, final-state)) trace-bs
  fri-dg composition-bs
  using accepted-fri-opening-transcript-challenges[OF fri-opening]
  out-eq by simp
have degree-bound: to-nat fri-dg ≤ maxDegree
  by (rule verify-monad-accepted-fri-challenges-degree-bound
  [OF outcome' challenges])
have degree-eval-bound: to-nat fri-dg + 1 ≤ clength * scale
  using degree-bound maxDegree-less-eval-domain by linarith
obtain N where eval-power: clength * scale = 2 ^ N
  using eval-domain-length-power by blast
have rounds-bound: length composition-bs ≤ N
  unfolding rounds-eq
  by (rule ceil-log-le-power-bound)
  (use degree-eval-bound eval-power in simp)
have ¬ generic-fri-full-cover-arithmetic-shape-failure
  opening-composition-roots composition-bs
  by (rule generic-fri-full-cover-arithmetic-shape-failure-false
  [OF challenges-len eval-power rounds-bound])
then show False
  using arithmetic by contradiction
qed

lemma wp-composition-fri-sampled-full-cover-arithmetic-shape-failure-zero:
  wp-event verify-monad
  (composition-fri-sampled-full-cover-arithmetic-shape-failure s) s ≤ 0
proof –
  have wp-event verify-monad
  (composition-fri-sampled-full-cover-arithmetic-shape-failure s) s
  ≤ wp-event verify-monad (λ-. False) s
  by (rule wp-event-mono-on-support)
  (rule
  composition-fri-sampled-full-cover-arithmetic-shape-failure-false-on-support)

```

also have ... = 0
 unfolding *wp-event-def wp-def dist-expect-def* by *simp*
 finally show ?thesis .
 qed

lemma *wp-trace-fri-sampled-full-cover-shape-bound-from-witness-length:*
 assumes *witness:*
 wp-event verify-monad
 $(\text{trace-fri-sampled-full-cover-witness-length-failure } s) \ s \leq W$
 shows
 wp-event verify-monad
 $(\text{trace-fri-sampled-full-cover-shape-failure } s) \ s \leq W$
proof –
 have *wp-event verify-monad*
 $(\text{trace-fri-sampled-full-cover-shape-failure } s) \ s \leq 0 + W$
 by (rule *wp-trace-fri-sampled-full-cover-shape-bound-from-split*
 [*OF wp-trace-fri-sampled-full-cover-arithmetic-shape-failure-zero*
 witness])
 then show ?thesis
 by *simp*
 qed

lemma *wp-composition-fri-sampled-full-cover-shape-bound-from-witness-length:*
 assumes *witness:*
 wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-witness-length-failure } s) \ s \leq W$
 shows
 wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-shape-failure } s) \ s \leq W$
proof –
 have *wp-event verify-monad*
 $(\text{composition-fri-sampled-full-cover-shape-failure } s) \ s \leq 0 + W$
 by (rule *wp-composition-fri-sampled-full-cover-shape-bound-from-split*
 [*OF*
 wp-composition-fri-sampled-full-cover-arithmetic-shape-failure-zero
 witness])
 then show ?thesis
 by *simp*
 qed

lemma *trace-fri-sampled-full-cover-final-low-imp-length:*
 assumes *trace-fri-sampled-full-cover-final-low-failure s out*
 shows *trace-fri-sampled-full-cover-final-length-failure s out*
proof –
 from *assms* obtain *trace-roots trace-bs trace-final dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table doms layers

where *evidence*:
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *chain*:
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
and *final-low*:
generic-fri-full-cover-final-low-failure (clength - 1) trace-bs
doms layers
unfolding *trace-fri-sampled-full-cover-final-low-failure-def* **by** *blast*
have *length-failure*:
generic-fri-full-cover-final-length-failure trace-bs doms layers
by (*rule generic-fri-final-low-failure-imp-length-failure*
[OF chain final-low])
show *?thesis*
unfolding *trace-fri-sampled-full-cover-final-length-failure-def*
using *evidence chain length-failure* **by** *blast*
qed

lemma *composition-fri-sampled-full-cover-final-low-imp-length*:
assumes *composition-fri-sampled-full-cover-final-low-failure s out*
shows *composition-fri-sampled-full-cover-final-length-failure s out*
proof —
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
where *evidence*:
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
and *chain*:
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
and *final-low*:
generic-fri-full-cover-final-low-failure (to-nat fri-dg)
composition-bs doms layers

unfolding *composition-fri-sampled-full-cover-final-low-failure-def*
by *meson*
have *length-failure*:
generic-fri-full-cover-final-length-failure composition-bs doms layers
by (rule *generic-fri-final-low-failure-imp-length-failure*
[*OF chain final-low*])
show *?thesis*
unfolding *composition-fri-sampled-full-cover-final-length-failure-def*
using *evidence chain length-failure* **by** *meson*
qed

lemma *wp-trace-fri-sampled-full-cover-final-low-bound-from-length*:
assumes *length-bound*:
wp-event verify-monad
(*trace-fri-sampled-full-cover-final-length-failure s*) $s \leq L$
shows
wp-event verify-monad
(*trace-fri-sampled-full-cover-final-low-failure s*) $s \leq L$
by (rule *order-trans[OF - length-bound]*)
(rule *wp-event-mono*,
rule *trace-fri-sampled-full-cover-final-low-imp-length*)

lemma *wp-composition-fri-sampled-full-cover-final-low-bound-from-length*:
assumes *length-bound*:
wp-event verify-monad
(*composition-fri-sampled-full-cover-final-length-failure s*) $s \leq L$
shows
wp-event verify-monad
(*composition-fri-sampled-full-cover-final-low-failure s*) $s \leq L$
by (rule *order-trans[OF - length-bound]*)
(rule *wp-event-mono*,
rule *composition-fri-sampled-full-cover-final-low-imp-length*)

lemma *trace-fri-sampled-full-cover-final-length-imp-shape*:
assumes *trace-fri-sampled-full-cover-final-length-failure s out*
shows *trace-fri-sampled-full-cover-shape-failure s out*
using *assms*
unfolding *trace-fri-sampled-full-cover-final-length-failure-def*
trace-fri-sampled-full-cover-shape-failure-def
generic-fri-full-cover-final-length-failure-def
generic-fri-full-cover-shape-failure-def
by *blast*

lemma *composition-fri-sampled-full-cover-final-length-imp-shape*:
assumes *composition-fri-sampled-full-cover-final-length-failure s out*
shows *composition-fri-sampled-full-cover-shape-failure s out*
using *assms*
unfolding *composition-fri-sampled-full-cover-final-length-failure-def*
composition-fri-sampled-full-cover-shape-failure-def

generic-fri-full-cover-final-length-failure-def
generic-fri-full-cover-shape-failure-def
by *meson*

lemma *trace-fri-sampled-full-cover-final-low-imp-shape:*
assumes *trace-fri-sampled-full-cover-final-low-failure s out*
shows *trace-fri-sampled-full-cover-shape-failure s out*
using *trace-fri-sampled-full-cover-final-low-imp-length[OF assms]*
trace-fri-sampled-full-cover-final-length-imp-shape
by *blast*

lemma *composition-fri-sampled-full-cover-final-low-imp-shape:*
assumes *composition-fri-sampled-full-cover-final-low-failure s out*
shows *composition-fri-sampled-full-cover-shape-failure s out*
using *composition-fri-sampled-full-cover-final-low-imp-length[OF assms]*
composition-fri-sampled-full-cover-final-length-imp-shape
by *blast*

lemma *trace-fri-sampled-full-cover-failure-case-split-no-final:*
assumes *trace-fri-sampled-full-cover-failure-case s bad out*
shows
trace-fri-sampled-full-cover-proximity-failure s bad out $\vee$
trace-fri-sampled-full-cover-index-failure s out $\vee$
trace-fri-sampled-full-cover-shape-failure s out $\vee$
trace-fri-sampled-full-cover-domain-failure s out $\vee$
trace-fri-sampled-full-cover-start-low-failure s out
using *trace-fri-sampled-full-cover-failure-case-split[OF assms]*
trace-fri-sampled-full-cover-final-low-imp-shape
by *blast*

lemma *composition-fri-sampled-full-cover-failure-case-split-no-final:*
assumes *composition-fri-sampled-full-cover-failure-case s bad out*
shows
composition-fri-sampled-full-cover-proximity-failure s bad out $\vee$
composition-fri-sampled-full-cover-index-failure s out $\vee$
composition-fri-sampled-full-cover-shape-failure s out $\vee$
composition-fri-sampled-full-cover-domain-failure s out $\vee$
composition-fri-sampled-full-cover-start-low-failure s out
using *composition-fri-sampled-full-cover-failure-case-split[OF assms]*
composition-fri-sampled-full-cover-final-low-imp-shape
by *blast*

lemma *wp-trace-fri-sampled-full-cover-failure-case-union-bound-no-final:*
assumes *proximity:*
wp-event verify-monad
(trace-fri-sampled-full-cover-proximity-failure s bad) s $\leq$ P
and *index:*
wp-event verify-monad
(trace-fri-sampled-full-cover-index-failure s) s $\leq$ I

and *shape*:
wp-event verify-monad
 (trace-fri-sampled-full-cover-shape-failure *s*) $s \leq Sh$
and *domain*:
wp-event verify-monad
 (trace-fri-sampled-full-cover-domain-failure *s*) $s \leq D$
and *start-low*:
wp-event verify-monad
 (trace-fri-sampled-full-cover-start-low-failure *s*) $s \leq SL$
shows
wp-event verify-monad
 (trace-fri-sampled-full-cover-failure-case *s bad*) *s*
 $\leq P + I + Sh + D + SL$
proof –
let *?A* = $\lambda out. \text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad } out$
let *?B* = $\lambda out. \text{trace-fri-sampled-full-cover-index-failure } s \text{ out}$
let *?C* = $\lambda out. \text{trace-fri-sampled-full-cover-shape-failure } s \text{ out}$
let *?D* = $\lambda out. \text{trace-fri-sampled-full-cover-domain-failure } s \text{ out}$
let *?E* = $\lambda out. \text{trace-fri-sampled-full-cover-start-low-failure } s \text{ out}$
have *step1*:
wp-event verify-monad
 (trace-fri-sampled-full-cover-failure-case *s bad*) *s*
 $\leq wp\text{-event verify-monad}$
 ($\lambda out. ?A \text{ out} \vee ?B \text{ out} \vee ?C \text{ out} \vee ?D \text{ out} \vee ?E \text{ out}$) *s*
by (rule *wp-event-mono*)
 (rule *trace-fri-sampled-full-cover-failure-case-split-no-final*)
have *four*:
wp-event verify-monad
 ($\lambda out. ?A \text{ out} \vee ?B \text{ out} \vee ?C \text{ out} \vee ?D \text{ out}$) *s*
 $\leq P + I + Sh + D$
by (rule *order-trans*[*OF wp-event-union-bound*]
 add-mono[*OF add-mono*[*OF add-mono*[*OF proximity index*] *shape*]
 domain]))
have *five*:
wp-event verify-monad
 ($\lambda out. ?A \text{ out} \vee ?B \text{ out} \vee ?C \text{ out} \vee ?D \text{ out} \vee ?E \text{ out}$) *s*
 $\leq P + I + Sh + D + SL$
proof –
have *wp-event verify-monad*
 ($\lambda out. ?A \text{ out} \vee ?B \text{ out} \vee ?C \text{ out} \vee ?D \text{ out} \vee ?E \text{ out}$) *s*
 $\leq wp\text{-event verify-monad}$
 ($\lambda out. ?A \text{ out} \vee ?B \text{ out} \vee ?C \text{ out} \vee ?D \text{ out}$) *s* +
wp-event verify-monad *?E s*
by (rule *order-trans*[*OF - wp-event-union-bound*])
 (rule *wp-event-mono, blast*)
also have ... $\leq (P + I + Sh + D) + SL$
by (rule *add-mono*[*OF four start-low*])
finally show *?thesis*
by (*simp add: add.assoc*)

```

qed
show ?thesis
  by (rule order-trans[OF step1 five])
qed

lemma wp-composition-fri-sampled-full-cover-failure-case-union-bound-no-final:
  assumes proximity:
    wp-event verify-monad
      (composition-fri-sampled-full-cover-proximity-failure s bad) s ≤ P
  and index:
    wp-event verify-monad
      (composition-fri-sampled-full-cover-index-failure s) s ≤ I
  and shape:
    wp-event verify-monad
      (composition-fri-sampled-full-cover-shape-failure s) s ≤ Sh
  and domain:
    wp-event verify-monad
      (composition-fri-sampled-full-cover-domain-failure s) s ≤ D
  and start-low:
    wp-event verify-monad
      (composition-fri-sampled-full-cover-start-low-failure s) s ≤ SL
  shows
    wp-event verify-monad
      (composition-fri-sampled-full-cover-failure-case s bad) s
      ≤ P + I + Sh + D + SL
proof -
  let ?A =
    λout. composition-fri-sampled-full-cover-proximity-failure s bad out
  let ?B = λout. composition-fri-sampled-full-cover-index-failure s out
  let ?C = λout. composition-fri-sampled-full-cover-shape-failure s out
  let ?D = λout. composition-fri-sampled-full-cover-domain-failure s out
  let ?E = λout. composition-fri-sampled-full-cover-start-low-failure s out
  have step1:
    wp-event verify-monad
      (composition-fri-sampled-full-cover-failure-case s bad) s
      ≤ wp-event verify-monad
        (λout. ?A out ∨ ?B out ∨ ?C out ∨ ?D out ∨ ?E out) s
  by (rule wp-event-mono)
    (rule composition-fri-sampled-full-cover-failure-case-split-no-final)
  have four:
    wp-event verify-monad
      (λout. ?A out ∨ ?B out ∨ ?C out ∨ ?D out) s
      ≤ P + I + Sh + D
  by (rule order-trans[OF wp-event-union-bound4
    add-mono[OF add-mono[OF add-mono[OF proximity index] shape]
    domain]])
  have five:
    wp-event verify-monad
      (λout. ?A out ∨ ?B out ∨ ?C out ∨ ?D out ∨ ?E out) s

```

$\leq P + I + Sh + D + SL$
proof –
have *wp-event verify-monad*
 $(\lambda out. ?A \text{ out} \vee ?B \text{ out} \vee ?C \text{ out} \vee ?D \text{ out} \vee ?E \text{ out}) \ s$
 $\leq \text{wp-event verify-monad}$
 $(\lambda out. ?A \text{ out} \vee ?B \text{ out} \vee ?C \text{ out} \vee ?D \text{ out}) \ s +$
 $\text{wp-event verify-monad } ?E \ s$
by (*rule order-trans[OF - wp-event-union-bound]*)
 $(\text{rule wp-event-mono, blast})$
also have $\dots \leq (P + I + Sh + D) + SL$
by (*rule add-mono[OF four start-low]*)
finally show *?thesis*
by (*simp add: add.assoc*)
qed
show *?thesis*
by (*rule order-trans[OF step1 five]*)
qed

lemma *generic-fri-sampled-layer-chain-full-cover-or-failure:*
assumes
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers
shows
generic-fri-sampled-layer-chain-full-cover low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad $\vee$
generic-fri-sampled-full-cover-failure low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad
using *assms*
unfolding *generic-fri-sampled-layer-chain-full-cover-def*
generic-fri-sampled-full-cover-failure-def
by *blast*

lemma *generic-fri-sampled-layer-chain-missing-full-cover-failure:*
assumes *chain:*
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers
and *no-full:*
 $\neg$ *generic-fri-sampled-layer-chain-full-cover low-degree*
candidate-table degree-bound roots challenges final-value query-idxs
round-layers doms layers bad
shows
generic-fri-sampled-full-cover-failure low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad
using *generic-fri-sampled-layer-chain-full-cover-or-failure[OF chain]*

no-full **by** *blast*

lemma *trace-fri-sampled-missing-full-cover-imp-failure:*

assumes *trace-fri-sampled-missing-full-cover s bad out*

shows *trace-fri-sampled-full-cover-failure s bad out*

proof –

have *sampld: trace-fri-bad-with-sampled-layer-chain s out*

and *not-full: $\neg$ trace-fri-bad-with-full-cover-layer-chain s bad out*

using *assms* **unfolding** *trace-fri-sampled-missing-full-cover-def* **by** *blast+*

from *sampld* **obtain** *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr candidate-query-idxs trace-openings

trace-table doms layers

where *evidence:*

trace-fri-partial-candidate-opening-evidence s out trace-roots

trace-bs trace-final dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers fr candidate-query-idxs trace-openings

trace-table

and *chain:*

generic-fri-sampled-layer-chain-evidence trace-table-low-degree

trace-table (clength – 1) trace-roots trace-bs trace-final

fri-query-idxs trace-round-layers doms layers

unfolding *trace-fri-bad-with-sampled-layer-chain-def* **by** *blast*

have *no-full:*

$\neg$ *generic-fri-sampled-layer-chain-full-cover trace-table-low-degree*

trace-table (clength – 1) trace-roots trace-bs trace-final

fri-query-idxs trace-round-layers doms layers (bad trace-table)

proof

assume *full:*

generic-fri-sampled-layer-chain-full-cover trace-table-low-degree

trace-table (clength – 1) trace-roots trace-bs trace-final

fri-query-idxs trace-round-layers doms layers (bad trace-table)

have *trace-fri-bad-with-full-cover-layer-chain s bad out*

unfolding *trace-fri-bad-with-full-cover-layer-chain-def*

using *evidence full* **by** *blast*

then show *False*

using *not-full* **by** *contradiction*

qed

have *failure:*

generic-fri-sampled-full-cover-failure trace-table-low-degree

trace-table (clength – 1) trace-roots trace-bs trace-final

fri-query-idxs trace-round-layers doms layers (bad trace-table)

by (rule *generic-fri-sampled-layer-chain-missing-full-cover-failure*
[*OF chain no-full*])

show *?thesis*

unfolding *trace-fri-sampled-full-cover-failure-def*

using *evidence chain failure* **by** *blast*

qed

lemma *composition-fri-sampled-missing-full-cover-imp-failure:*
assumes *composition-fri-sampled-missing-full-cover s bad out*
shows *composition-fri-sampled-full-cover-failure s bad out*
proof –
have *sampld: composition-fri-bad-with-sampled-layer-chain s out*
and *not-full:*
 $\neg$ *composition-fri-bad-with-full-cover-layer-chain s bad out*
using *assms unfolding composition-fri-sampled-missing-full-cover-def*
by *blast+*
from *sampld obtain trace-roots trace-bs trace-final fri-dg*
 opening-composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers fr
 f-fri-roots f-final as dg composition-fri-roots final
 trace-query-idxs trace-openings composition-query-idxs
 composition-openings trace-table composition-table doms layers
where *evidence:*
 composition-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final fri-dg opening-composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr f-fri-roots f-final as dg
 composition-fri-roots final trace-query-idxs trace-openings
 composition-query-idxs composition-openings trace-table
 composition-table
and *chain:*
 generic-fri-sampled-layer-chain-evidence
 (composition-table-low-degree (to-nat fri-dg)) composition-table
 (to-nat fri-dg) opening-composition-roots composition-bs
 composition-final fri-query-idxs composition-round-layers doms layers
unfolding *composition-fri-bad-with-sampled-layer-chain-def* **by** *auto*
have *no-full:*
 $\neg$ *generic-fri-sampled-layer-chain-full-cover*
 (composition-table-low-degree (to-nat fri-dg)) composition-table
 (to-nat fri-dg) opening-composition-roots composition-bs
 composition-final fri-query-idxs composition-round-layers doms layers
 (bad fri-dg composition-table)
proof
assume *full:*
 generic-fri-sampled-layer-chain-full-cover
 (composition-table-low-degree (to-nat fri-dg)) composition-table
 (to-nat fri-dg) opening-composition-roots composition-bs
 composition-final fri-query-idxs composition-round-layers doms
 layers (bad fri-dg composition-table)
have *composition-fri-bad-with-full-cover-layer-chain s bad out*
unfolding *composition-fri-bad-with-full-cover-layer-chain-def*
using *evidence full by metis*
then show *False*
using *not-full by contradiction*
qed

have *failure*:
generic-fri-sampled-full-cover-failure
 (*composition-table-low-degree* (*to-nat fri-dg*)) *composition-table*
 (*to-nat fri-dg*) *opening-composition-roots* *composition-bs*
composition-final *fri-query-idxs* *composition-round-layers* *doms* *layers*
 (*bad fri-dg* *composition-table*)
by (*rule generic-fri-sampled-layer-chain-missing-full-cover-failure*
 [*OF chain no-full*])
show *?thesis*
unfolding *composition-fri-sampled-full-cover-failure-def*
using *evidence chain failure* **by** *metis*
qed

lemma *wp-trace-fri-sampled-missing-full-cover-bound-from-failure*:

assumes *failure-bound*:
wp-event verify-monad
 (*trace-fri-sampled-full-cover-failure* *s* *bad*) $s \leq M$
shows
wp-event verify-monad
 (*trace-fri-sampled-missing-full-cover* *s* *bad*) $s \leq M$
by (*rule order-trans*[*OF - failure-bound*])
 (*rule wp-event-mono*,
rule trace-fri-sampled-missing-full-cover-imp-failure)

lemma *wp-trace-fri-sampled-missing-full-cover-bound-from-failure-case*:

assumes *failure-bound*:
wp-event verify-monad
 (*trace-fri-sampled-full-cover-failure-case* *s* *bad*) $s \leq M$
shows
wp-event verify-monad
 (*trace-fri-sampled-missing-full-cover* *s* *bad*) $s \leq M$
by (*rule wp-trace-fri-sampled-missing-full-cover-bound-from-failure*)
 (*rule order-trans*[*OF - failure-bound*],
rule wp-event-mono,
simp add: trace-fri-sampled-full-cover-failure-iff-case)

lemma *wp-trace-fri-sampled-missing-full-cover-bound-from-failure-subcases*:

assumes *proximity*:
wp-event verify-monad
 (*trace-fri-sampled-full-cover-proximity-failure* *s* *bad*) $s \leq P$
and *index*:
wp-event verify-monad
 (*trace-fri-sampled-full-cover-index-failure* *s*) $s \leq I$
and *shape*:
wp-event verify-monad
 (*trace-fri-sampled-full-cover-shape-failure* *s*) $s \leq Sh$
and *domain*:
wp-event verify-monad
 (*trace-fri-sampled-full-cover-domain-failure* *s*) $s \leq D$

and *start-low*:
wp-event verify-monad
 $(\text{trace-fri-sampled-full-cover-start-low-failure } s) \ s \leq SL$
and *final-low*:
wp-event verify-monad
 $(\text{trace-fri-sampled-full-cover-final-low-failure } s) \ s \leq FL$
shows
wp-event verify-monad
 $(\text{trace-fri-sampled-missing-full-cover } s \text{ bad}) \ s$
 $\leq P + I + Sh + D + SL + FL$
by (*rule wp-trace-fri-sampled-missing-full-cover-bound-from-failure-case*)
(rule wp-trace-fri-sampled-full-cover-failure-case-union-bound
 $[OF \text{ proximity index shape domain start-low final-low}]$)

lemma *wp-trace-fri-sampled-missing-full-cover-bound-from-five-subcases*:
assumes *proximity*:
wp-event verify-monad
 $(\text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$
and *index*:
wp-event verify-monad
 $(\text{trace-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and *shape*:
wp-event verify-monad
 $(\text{trace-fri-sampled-full-cover-shape-failure } s) \ s \leq Sh$
and *domain*:
wp-event verify-monad
 $(\text{trace-fri-sampled-full-cover-domain-failure } s) \ s \leq D$
and *final-low*:
wp-event verify-monad
 $(\text{trace-fri-sampled-full-cover-final-low-failure } s) \ s \leq FL$
shows
wp-event verify-monad
 $(\text{trace-fri-sampled-missing-full-cover } s \text{ bad}) \ s$
 $\leq P + I + Sh + D + FL$

proof –
have *six*:
wp-event verify-monad
 $(\text{trace-fri-sampled-missing-full-cover } s \text{ bad}) \ s$
 $\leq P + I + Sh + D + 0 + FL$
by (*rule wp-trace-fri-sampled-missing-full-cover-bound-from-failure-subcases*
 $[OF \text{ proximity index shape domain}$
 $\text{wp-trace-fri-sampled-full-cover-start-low-failure-zero final-low}]$)
then show *?thesis*
by *simp*
qed

lemma *wp-trace-fri-sampled-missing-full-cover-bound-from-length-subcases*:
assumes *proximity*:
wp-event verify-monad

(trace-fri-sampled-full-cover-proximity-failure s bad) $s \leq P$
and index:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-index-failure s) $s \leq I$
and shape:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-shape-failure s) $s \leq Sh$
and domain:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-domain-failure s) $s \leq D$
and final-length:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-final-length-failure s) $s \leq FL$
shows
 wp-event verify-monad
 (trace-fri-sampled-missing-full-cover s bad) s
 $\leq P + I + Sh + D + FL$
by (rule wp-trace-fri-sampled-missing-full-cover-bound-from-five-subcases
 [OF proximity index shape domain])
 (rule wp-trace-fri-sampled-full-cover-final-low-bound-from-length
 [OF final-length])

lemma wp-trace-fri-sampled-missing-full-cover-bound-from-shape-subcases:

assumes proximity:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-proximity-failure s bad) $s \leq P$
and index:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-index-failure s) $s \leq I$
and shape:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-shape-failure s) $s \leq Sh$
and domain:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-domain-failure s) $s \leq D$
shows
 wp-event verify-monad
 (trace-fri-sampled-missing-full-cover s bad) s
 $\leq P + I + Sh + D$

proof –

have case-bound:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-failure-case s bad) s
 $\leq P + I + Sh + D + 0$
by (rule wp-trace-fri-sampled-full-cover-failure-case-union-bound-no-final
 [OF proximity index shape domain
 wp-trace-fri-sampled-full-cover-start-low-failure-zero])
have wp-event verify-monad
 (trace-fri-sampled-missing-full-cover s bad) s

$\leq P + I + Sh + D + 0$
by (rule *wp-trace-fri-sampled-missing-full-cover-bound-from-failure-case*
 $[OF\ case-bound]$)
then show *?thesis*
by *simp*
qed

lemma *wp-trace-fri-sampled-missing-full-cover-bound-from-split-shape-subcases:*
assumes *proximity:*
wp-event verify-monad
 $(trace-fri-sampled-full-cover-proximity-failure\ s\ bad)\ s \leq P$
and *index:*
wp-event verify-monad
 $(trace-fri-sampled-full-cover-index-failure\ s)\ s \leq I$
and *arithmetic-shape:*
wp-event verify-monad
 $(trace-fri-sampled-full-cover-arithmetic-shape-failure\ s)\ s \leq A$
and *witness-length:*
wp-event verify-monad
 $(trace-fri-sampled-full-cover-witness-length-failure\ s)\ s \leq W$
and *domain:*
wp-event verify-monad
 $(trace-fri-sampled-full-cover-domain-failure\ s)\ s \leq D$
shows
wp-event verify-monad
 $(trace-fri-sampled-missing-full-cover\ s\ bad)\ s$
 $\leq P + I + (A + W) + D$
by (rule *wp-trace-fri-sampled-missing-full-cover-bound-from-shape-subcases*
 $[OF\ proximity\ index - domain]$)
(rule *wp-trace-fri-sampled-full-cover-shape-bound-from-split*
 $[OF\ arithmetic-shape\ witness-length]$)

lemma *wp-trace-fri-sampled-missing-full-cover-bound-from-witness-length-subcases:*
assumes *proximity:*
wp-event verify-monad
 $(trace-fri-sampled-full-cover-proximity-failure\ s\ bad)\ s \leq P$
and *index:*
wp-event verify-monad
 $(trace-fri-sampled-full-cover-index-failure\ s)\ s \leq I$
and *witness-length:*
wp-event verify-monad
 $(trace-fri-sampled-full-cover-witness-length-failure\ s)\ s \leq W$
and *domain:*
wp-event verify-monad
 $(trace-fri-sampled-full-cover-domain-failure\ s)\ s \leq D$
shows
wp-event verify-monad
 $(trace-fri-sampled-missing-full-cover\ s\ bad)\ s$
 $\leq P + I + W + D$

proof –

have *wp-event verify-monad*

(*trace-fri-sampled-missing-full-cover s bad*) s

$\leq P + I + (0 + W) + D$

by (*rule*

wp-trace-fri-sampled-missing-full-cover-bound-from-split-shape-subcases

[*OF proximity index*

wp-trace-fri-sampled-full-cover-arithmetic-shape-failure-zero

witness-length domain])

then show *?thesis*

by *simp*

qed

lemma *wp-composition-fri-sampled-missing-full-cover-bound-from-failure:*

assumes *failure-bound:*

wp-event verify-monad

(*composition-fri-sampled-full-cover-failure s bad*) $s \leq M$

shows

wp-event verify-monad

(*composition-fri-sampled-missing-full-cover s bad*) $s \leq M$

by (*rule order-trans[OF - failure-bound]*)

(*rule wp-event-mono,*

rule composition-fri-sampled-missing-full-cover-imp-failure)

lemma *wp-composition-fri-sampled-missing-full-cover-bound-from-failure-case:*

assumes *failure-bound:*

wp-event verify-monad

(*composition-fri-sampled-full-cover-failure-case s bad*) $s \leq M$

shows

wp-event verify-monad

(*composition-fri-sampled-missing-full-cover s bad*) $s \leq M$

by (*rule wp-composition-fri-sampled-missing-full-cover-bound-from-failure*)

(*rule order-trans[OF - failure-bound],*

rule wp-event-mono,

simp add: composition-fri-sampled-full-cover-failure-iff-case)

lemma *wp-composition-fri-sampled-missing-full-cover-bound-from-failure-subcases:*

assumes *proximity:*

wp-event verify-monad

(*composition-fri-sampled-full-cover-proximity-failure s bad*) $s \leq P$

and *index:*

wp-event verify-monad

(*composition-fri-sampled-full-cover-index-failure s*) $s \leq I$

and *shape:*

wp-event verify-monad

(*composition-fri-sampled-full-cover-shape-failure s*) $s \leq Sh$

and *domain:*

wp-event verify-monad

(*composition-fri-sampled-full-cover-domain-failure s*) $s \leq D$

and *start-low*:
wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-start-low-failure } s) \ s \leq SL$
and *final-low*:
wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-final-low-failure } s) \ s \leq FL$
shows
wp-event verify-monad
 $(\text{composition-fri-sampled-missing-full-cover } s \text{ bad}) \ s$
 $\leq P + I + Sh + D + SL + FL$
by (*rule wp-composition-fri-sampled-missing-full-cover-bound-from-failure-case*)
(rule wp-composition-fri-sampled-full-cover-failure-case-union-bound
 $[OF \text{ proximity index shape domain start-low final-low}]$)

lemma *wp-composition-fri-sampled-missing-full-cover-bound-from-five-subcases*:
assumes *proximity*:
wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$
and *index*:
wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and *shape*:
wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-shape-failure } s) \ s \leq Sh$
and *domain*:
wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-domain-failure } s) \ s \leq D$
and *final-low*:
wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-final-low-failure } s) \ s \leq FL$
shows
wp-event verify-monad
 $(\text{composition-fri-sampled-missing-full-cover } s \text{ bad}) \ s$
 $\leq P + I + Sh + D + FL$

proof –
have *six*:
wp-event verify-monad
 $(\text{composition-fri-sampled-missing-full-cover } s \text{ bad}) \ s$
 $\leq P + I + Sh + D + 0 + FL$
by (*rule*
 $\text{wp-composition-fri-sampled-missing-full-cover-bound-from-failure-subcases}$
 $[OF \text{ proximity index shape domain}$
 $\text{wp-composition-fri-sampled-full-cover-start-low-failure-zero}$
 $\text{final-low}]$)

then show *?thesis*
by *simp*
qed

lemma *wp-composition-fri-sampled-missing-full-cover-bound-from-length-subcases*:

assumes *proximity*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-proximity-failure s bad*) $s \leq P$

and *index*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-index-failure s*) $s \leq I$

and *shape*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-shape-failure s*) $s \leq Sh$

and *domain*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-domain-failure s*) $s \leq D$

and *final-length*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-final-length-failure s*) $s \leq FL$

shows

wp-event verify-monad

(*composition-fri-sampled-missing-full-cover s bad*) s
 $\leq P + I + Sh + D + FL$

by (*rule*

wp-composition-fri-sampled-missing-full-cover-bound-from-five-subcases
[*OF proximity index shape domain*])

(*rule wp-composition-fri-sampled-full-cover-final-low-bound-from-length*
[*OF final-length*])

lemma *wp-composition-fri-sampled-missing-full-cover-bound-from-shape-subcases*:

assumes *proximity*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-proximity-failure s bad*) $s \leq P$

and *index*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-index-failure s*) $s \leq I$

and *shape*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-shape-failure s*) $s \leq Sh$

and *domain*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-domain-failure s*) $s \leq D$

shows

wp-event verify-monad

(*composition-fri-sampled-missing-full-cover s bad*) s
 $\leq P + I + Sh + D$

proof –

have *case-bound*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-failure-case s bad*) s
 $\leq P + I + Sh + D + 0$

by (*rule*

wp-composition-fri-sampled-full-cover-failure-case-union-bound-no-final

[OF proximity index shape domain
 wp-composition-fri-sampled-full-cover-start-low-failure-zero])
have wp-event verify-monad
 (composition-fri-sampled-missing-full-cover s bad) s
 $\leq P + I + Sh + D + 0$
by (rule
 wp-composition-fri-sampled-missing-full-cover-bound-from-failure-case
 [OF case-bound])
then show ?thesis
by simp
qed

lemma

wp-composition-fri-sampled-missing-full-cover-bound-from-split-shape-subcases:

assumes proximity:

wp-event verify-monad

(composition-fri-sampled-full-cover-proximity-failure s bad) s $\leq P$

and index:

wp-event verify-monad

(composition-fri-sampled-full-cover-index-failure s) s $\leq I$

and arithmetic-shape:

wp-event verify-monad

(composition-fri-sampled-full-cover-arithmetic-shape-failure s) s
 $\leq A$

and witness-length:

wp-event verify-monad

(composition-fri-sampled-full-cover-witness-length-failure s) s $\leq W$

and domain:

wp-event verify-monad

(composition-fri-sampled-full-cover-domain-failure s) s $\leq D$

shows

wp-event verify-monad

(composition-fri-sampled-missing-full-cover s bad) s
 $\leq P + I + (A + W) + D$

by (rule

wp-composition-fri-sampled-missing-full-cover-bound-from-shape-subcases

[OF proximity index - domain])

(rule wp-composition-fri-sampled-full-cover-shape-bound-from-split

[OF arithmetic-shape witness-length])

lemma

wp-composition-fri-sampled-missing-full-cover-bound-from-witness-length-subcases:

assumes proximity:

wp-event verify-monad

(composition-fri-sampled-full-cover-proximity-failure s bad) s $\leq P$

and index:

wp-event verify-monad

(composition-fri-sampled-full-cover-index-failure s) s $\leq I$

and witness-length:

```

    wp-event verify-monad
      (composition-fri-sampled-full-cover-witness-length-failure s) s ≤ W
  and domain:
    wp-event verify-monad
      (composition-fri-sampled-full-cover-domain-failure s) s ≤ D
  shows
    wp-event verify-monad
      (composition-fri-sampled-missing-full-cover s bad) s
      ≤ P + I + W + D
  proof -
    have wp-event verify-monad
      (composition-fri-sampled-missing-full-cover s bad) s
      ≤ P + I + (0 + W) + D
    by (rule
      wp-composition-fri-sampled-missing-full-cover-bound-from-split-shape-subcases
      [OF proximity index
        wp-composition-fri-sampled-full-cover-arithmetic-shape-failure-zero
        witness-length domain])
    then show ?thesis
      by simp
  qed

end

end

```

```

theory Soundness-FRI-Full-Cover-Lengths
  imports Soundness-FRI-Full-Cover-Reasons
begin

```

Length residuals for sampled full-cover FRI witnesses.

Arithmetic shape failures are impossible, and table-layer length failures follow from sampled fold coverage. This theory factors the remaining witness-length residual into domain-length/final-length obligations plus the already-classified sampled-index coverage failure.

```

context soundness
begin

```

```

definition generic-fri-full-cover-domain-length-failure
  :: 'f list ⇒ 'f list ⇒ 'f list list ⇒ 'f list list ⇒ bool

```

```

where

```

```

  generic-fri-full-cover-domain-length-failure roots challenges doms layers
  ⇔
  ¬ (∀ layer-idx < length challenges.
    fri-evidence-layer-len roots layer-idx ≤
      length (doms ! layer-idx)) ∨
  ¬ (∀ layer-idx < length challenges.
    fri-evidence-layer-len roots layer-idx div 2 ≤

```


$\text{length } (\text{doms} ! \text{Suc layer-idx})) \vee$
 $\text{length } (\text{doms} ! \text{length challenges}) \neq$
 $\text{length } (\text{layers} ! \text{length challenges})$

definition *trace-fri-sampled-full-cover-domain-length-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-sampled-full-cover-domain-length-failure } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
 $\text{trace-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence trace-table-low-degree}$
 $\text{trace-table } (\text{clength} - 1) \text{ trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers} \wedge$
 $\text{generic-fri-full-cover-domain-length-failure trace-roots trace-bs}$
 $\text{doms layers})$

definition *composition-fri-sampled-full-cover-domain-length-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-sampled-full-cover-domain-length-failure } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table doms layers.}$
 $\text{composition-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings trace-table}$
 $\text{composition-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence}$
 $(\text{composition-table-low-degree } (\text{to-nat fri-dg})) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs composition-round-layers doms layers}$
 $\wedge$
 $\text{generic-fri-full-cover-domain-length-failure}$

opening-composition-roots composition-bs doms layers)

lemma *generic-fri-sampled-layer-chain-next-layer-lengths-if-cover:*
assumes *chain:*
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers
and *covers:* *fri-sampled-layers-cover-fold-indices roots query-idxs*
and *layer-bound:* *layer-idx < length challenges*
shows
fri-evidence-layer-len roots layer-idx div 2 ≤
length (layers ! Suc layer-idx)
proof (*rule ccontr*)
assume *not-len:*
 $\neg$ *fri-evidence-layer-len roots layer-idx div 2 ≤*
length (layers ! Suc layer-idx)
let *?idx = length (layers ! Suc layer-idx)*
have *idx-bound:*
?idx < fri-evidence-layer-len roots layer-idx div 2
using *not-len by simp*
have *partial:*
generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
by (*rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain]*)
have *roots-len:* *length challenges = length roots*
by (*rule generic-fri-partial-evidence-shapes(2)[OF partial]*)
obtain *round-idx where* *round-bound: round-idx < length query-idxs*
and *next-eq:*
fri-evidence-next-idx roots query-idxs round-idx layer-idx = ?idx
by (*rule fri-sampled-layers-cover-fold-indicesD*
[OF covers - idx-bound])
(use layer-bound roots-len in simp)
have *?idx < length (layers ! Suc layer-idx)*
using *generic-fri-sampled-layer-chain-evidence-sample(1)*
[OF chain round-bound layer-bound]
next-eq by simp
then show *False by simp*
qed

lemma *generic-fri-sampled-layer-chain-layer-lengths-if-cover:*
assumes *chain:*
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers
and *covers:* *fri-sampled-layers-cover-fold-indices roots query-idxs*
and *base-len:* *length candidate-table = clength * scale*
and *layer-bound:* *layer-idx < length challenges*
shows *fri-evidence-layer-len roots layer-idx ≤*
length (layers ! layer-idx)

```

proof (cases layer-idx)
  case 0
  have partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
    by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
  have roots-len: length challenges = length roots
    by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
  have roots-nonempty: 0 < length roots
    using layer-bound roots-len unfolding 0 by simp
  then obtain n where roots-len-suc: length roots = Suc n
    by (cases length roots) auto
  have len0: fri-evidence-layer-len roots 0 = clength * scale
    using roots-len-suc unfolding 0 fri-evidence-layer-len-def by simp
  have layer0: layers ! 0 = candidate-table
    by (rule generic-fri-sampled-layer-chain-evidenceD(5)[OF chain])
  show ?thesis
    using len0 layer0 base-len unfolding 0 by simp
next
  case (Suc j)
  have partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
    by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
  have roots-len: length challenges = length roots
    by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
  have j-bound: j < length challenges
    using layer-bound unfolding Suc by simp
  have len-suc:
    fri-evidence-layer-len roots (Suc j) =
      fri-evidence-layer-len roots j div 2
    using fri-layer-lengths-Suc-nth[of j length roots clength * scale]
    layer-bound roots-len
    unfolding fri-evidence-layer-len-def Suc by simp
  have next-len:
    fri-evidence-layer-len roots j div 2 ≤ length (layers ! Suc j)
    by (rule generic-fri-sampled-layer-chain-next-layer-lengths-if-cover
      [OF chain covers j-bound])
  show ?thesis
    using len-suc next-len unfolding Suc by simp
qed

lemma generic-fri-witness-length-failure-imp-domain-length-if-cover:
  assumes chain:
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers
  and no-index:
    ¬ generic-fri-full-cover-index-failure roots query-idxs

```

```

and base-len: length candidate-table = clength * scale
and witness:
  generic-fri-full-cover-witness-length-failure roots challenges
  doms layers
shows generic-fri-full-cover-domain-length-failure roots challenges
  doms layers
proof –
have covers: fri-sampled-layers-cover-fold-indices roots query-idxs
  using no-index unfolding generic-fri-full-cover-index-failure-def by simp
have layer-len:
   $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$ 
  fri-evidence-layer-len roots layer-idx  $\leq$ 
  length (layers ! layer-idx)
  by (rule generic-fri-sampled-layer-chain-layer-lengths-if-cover
  [OF chain covers base-len])
have next-layer-len:
   $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$ 
  fri-evidence-layer-len roots layer-idx div 2  $\leq$ 
  length (layers ! Suc layer-idx)
  by (rule generic-fri-sampled-layer-chain-next-layer-lengths-if-cover
  [OF chain covers])
show ?thesis
  using witness layer-len next-layer-len
  unfolding generic-fri-full-cover-witness-length-failure-def
  generic-fri-full-cover-domain-length-failure-def
  by blast
qed

```

lemma trace-fri-sampled-full-cover-witness-length-imp-index-or-domain-length:

```

assumes witness:
  trace-fri-sampled-full-cover-witness-length-failure s out
shows
  trace-fri-sampled-full-cover-index-failure s out  $\vee$ 
  trace-fri-sampled-full-cover-domain-length-failure s out

```

proof –

```

from witness obtain trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table doms layers

```

where evidence:

```

  trace-fri-partial-candidate-opening-evidence s out trace-roots
  trace-bs trace-final dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table

```

and chain:

```

  generic-fri-sampled-layer-chain-evidence trace-table-low-degree
  trace-table (clength – 1) trace-roots trace-bs trace-final
  fri-query-idxs trace-round-layers doms layers

```

```

and generic-witness:
  generic-fri-full-cover-witness-length-failure trace-roots trace-bs
    doms layers
unfolding trace-fri-sampled-full-cover-witness-length-failure-def
by blast
show ?thesis
proof (cases
  generic-fri-full-cover-index-failure trace-roots fri-query-idxs)
case True
then show ?thesis
  unfolding trace-fri-sampled-full-cover-index-failure-def
  using evidence chain by blast
next
case False
have partial:
  trace-fri-partial-candidate-evidence s out fr candidate-query-idxs
    trace-openings trace-table
by (rule trace-fri-partial-candidate-opening-evidenceD(2)
    [OF evidence])
have cand:
  partial-trace-table-candidate trace-table trace-openings
by (rule trace-fri-partial-candidate-evidenceD(2)[OF partial])
have base-len: length trace-table = clength * scale
using cand unfolding partial-trace-table-candidate-def by simp
have domain-length:
  generic-fri-full-cover-domain-length-failure trace-roots trace-bs
    doms layers
by (rule generic-fri-witness-length-failure-imp-domain-length-if-cover
    [OF chain False base-len generic-witness])
then show ?thesis
  unfolding trace-fri-sampled-full-cover-domain-length-failure-def
  using evidence chain by blast
qed
qed

lemma composition-fri-sampled-full-cover-witness-length-imp-index-or-domain-length:
assumes witness:
  composition-fri-sampled-full-cover-witness-length-failure s out
shows
  composition-fri-sampled-full-cover-index-failure s out  $\vee$ 
  composition-fri-sampled-full-cover-domain-length-failure s out
proof –
from witness obtain trace-roots trace-bs trace-final fri-dg
  opening-composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers fr
  f-fri-roots f-final as dg composition-fri-roots final
  trace-query-idxs trace-openings composition-query-idxs
  composition-openings trace-table composition-table doms layers
where evidence:

```

```

composition-fri-partial-candidate-opening-evidence s out trace-roots
  trace-bs trace-final fri-dg opening-composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr f-fri-roots f-final as dg
  composition-fri-roots final trace-query-idxs trace-openings
  composition-query-idxs composition-openings trace-table
  composition-table
and chain:
  generic-fri-sampled-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms layers
and generic-witness:
  generic-fri-full-cover-witness-length-failure
    opening-composition-roots composition-bs doms layers
unfolding composition-fri-sampled-full-cover-witness-length-failure-def
by meson
show ?thesis
proof (cases
  generic-fri-full-cover-index-failure opening-composition-roots
    fri-query-idxs)
case True
then show ?thesis
  unfolding composition-fri-sampled-full-cover-index-failure-def
  using evidence chain by meson
next
case False
have partial:
  composition-fri-partial-candidate-evidence s out fr f-fri-roots
    f-final as dg composition-fri-roots final trace-query-idxs
    trace-openings composition-query-idxs composition-openings
    trace-table composition-table
  by (rule composition-fri-partial-candidate-opening-evidenceD(2)
    [OF evidence])
have cand:
  partial-composition-table-candidate composition-table
    composition-openings
  by (rule composition-fri-partial-candidate-evidenceD(3)[OF partial])
have base-len: length composition-table = clength * scale
  using cand unfolding partial-composition-table-candidate-def by simp
have domain-length:
  generic-fri-full-cover-domain-length-failure
    opening-composition-roots composition-bs doms layers
  by (rule generic-fri-witness-length-failure-imp-domain-length-if-cover
    [OF chain False base-len generic-witness])
then show ?thesis
  unfolding composition-fri-sampled-full-cover-domain-length-failure-def
  using evidence chain by meson
qed

```

qed

lemma *wp-trace-fri-sampled-full-cover-witness-length-bound-from-index-or-domain-length:*

assumes *index:*

wp-event verify-monad

(trace-fri-sampled-full-cover-index-failure s) s ≤ I

and *domain-length:*

wp-event verify-monad

(trace-fri-sampled-full-cover-domain-length-failure s) s ≤ DL

shows

wp-event verify-monad

(trace-fri-sampled-full-cover-witness-length-failure s) s
≤ I + DL

proof –

have *wp-event verify-monad*

(trace-fri-sampled-full-cover-witness-length-failure s) s

≤ wp-event verify-monad

(λout. trace-fri-sampled-full-cover-index-failure s out ∨
trace-fri-sampled-full-cover-domain-length-failure s out) s

by *(rule wp-event-mono)*

(rule trace-fri-sampled-full-cover-witness-length-imp-index-or-domain-length)

also have *... ≤ I + DL*

by *(rule order-trans[OF wp-event-union-bound*
add-mono[OF index domain-length]])

finally show *?thesis* .

qed

lemma *wp-composition-fri-sampled-full-cover-witness-length-bound-from-index-or-domain-length:*

assumes *index:*

wp-event verify-monad

(composition-fri-sampled-full-cover-index-failure s) s ≤ I

and *domain-length:*

wp-event verify-monad

(composition-fri-sampled-full-cover-domain-length-failure s) s ≤ DL

shows

wp-event verify-monad

(composition-fri-sampled-full-cover-witness-length-failure s) s
≤ I + DL

proof –

have *wp-event verify-monad*

(composition-fri-sampled-full-cover-witness-length-failure s) s

≤ wp-event verify-monad

(λout. composition-fri-sampled-full-cover-index-failure s out ∨
composition-fri-sampled-full-cover-domain-length-failure s out) s

by *(rule wp-event-mono)*

(rule

composition-fri-sampled-full-cover-witness-length-imp-index-or-domain-length)

also have *... ≤ I + DL*

by *(rule order-trans[OF wp-event-union-bound*

$\text{add-mono}[OF \text{ index domain-length}]])$
finally show *?thesis* .
qed

lemma *wp-trace-fri-sampled-missing-full-cover-bound-from-length-split:*

assumes *proximity:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$

and *index:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-index-failure } s) \ s \leq I$

and *domain-length:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-domain-length-failure } s) \ s \leq DL$

and *domain:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-domain-failure } s) \ s \leq D$

shows

wp-event verify-monad

$(\text{trace-fri-sampled-missing-full-cover } s \text{ bad}) \ s$
 $\leq P + I + (I + DL) + D$

by (*rule wp-trace-fri-sampled-missing-full-cover-bound-from-witness-length-subcases*

[*OF proximity index*

wp-trace-fri-sampled-full-cover-witness-length-bound-from-index-or-domain-length
domain])

(*rule index, rule domain-length*)

lemma *wp-composition-fri-sampled-missing-full-cover-bound-from-length-split:*

assumes *proximity:*

wp-event verify-monad

$(\text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$

and *index:*

wp-event verify-monad

$(\text{composition-fri-sampled-full-cover-index-failure } s) \ s \leq I$

and *domain-length:*

wp-event verify-monad

$(\text{composition-fri-sampled-full-cover-domain-length-failure } s) \ s \leq DL$

and *domain:*

wp-event verify-monad

$(\text{composition-fri-sampled-full-cover-domain-failure } s) \ s \leq D$

shows

wp-event verify-monad

$(\text{composition-fri-sampled-missing-full-cover } s \text{ bad}) \ s$
 $\leq P + I + (I + DL) + D$

by (*rule*

wp-composition-fri-sampled-missing-full-cover-bound-from-witness-length-subcases

[*OF proximity index*

wp-composition-fri-sampled-full-cover-witness-length-bound-from-index-or-domain-length
domain])

(*rule index*, *rule domain-length*)

end

end

theory *Soundness-FRI-Full-Cover-Domains*
imports *Soundness-FRI-Full-Cover-Lengths*
begin

Domain residuals for sampled full-cover FRI witnesses.

This theory keeps the remaining domain issue explicitly verifier-sampled: a global full-cover domain failure is routed to missing sampled-index coverage or to sampled domain facts that still need to be bounded/extracted.

context *soundness*
begin

definition *fri-canonical-domain-at* :: *nat* $\Rightarrow$ '*f list*

where

fri-canonical-domain-at layer-idx =
 $\text{map } (\lambda \text{idx}. (h \wedge \text{idx} * \text{shift}) \wedge (2 \wedge \text{layer-idx}))$
 $[0..<((\text{clength} * \text{scale}) \text{div } 2 \wedge \text{layer-idx})]$

definition *fri-canonical-domains* :: *nat* $\Rightarrow$ '*f list list*

where

fri-canonical-domains n =
 $\text{map } \text{fri-canonical-domain-at } [0..<\text{Suc } n]$

definition *generic-fri-canonical-sampled-layer-chain-evidence*

:: ('*f list* $\Rightarrow$ *bool*) $\Rightarrow$ '*f list* $\Rightarrow$ *nat* $\Rightarrow$
'*f list* $\Rightarrow$ '*f list* $\Rightarrow$ '*f* $\Rightarrow$ *nat list* $\Rightarrow$
'*f list list list* $\Rightarrow$ '*f list list* $\Rightarrow$ *bool*

where

generic-fri-canonical-sampled-layer-chain-evidence low-degree
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers $\longleftrightarrow$
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
(*fri-canonical-domains* (*length challenges*)) *layers*

definition *trace-fri-bad-with-canonical-sampled-layer-chain*

:: ('*f*, '*a*) *protocol-channel-scheme* $\Rightarrow$
(*unit list* $\times$ ('*f*, '*a*) *protocol-channel-scheme*) *option* $\Rightarrow$ *bool*

where

trace-fri-bad-with-canonical-sampled-layer-chain s out $\longleftrightarrow$
 $(\exists \text{trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$

trace-table layers.
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table $\wedge$
generic-fri-canonical-sampled-layer-chain-evidence
trace-table-low-degree trace-table (clength - 1) trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers layers)

definition *composition-fri-bad-with-canonical-sampled-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-bad-with-canonical-sampled-layer-chain s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table layers.
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-canonical-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers layers)

definition *trace-fri-sampled-missing-canonical-sampled-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-sampled-missing-canonical-sampled-layer-chain s out
 $\longleftrightarrow$
trace-fri-bad-with-sampled-layer-chain s out $\wedge$
 $\neg \text{ trace-fri-bad-with-canonical-sampled-layer-chain s out}$

definition *composition-fri-sampled-missing-canonical-sampled-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-missing-canonical-sampled-layer-chain s out
 $\longleftrightarrow$

composition-fri-bad-with-sampled-layer-chain s out $\wedge$
 $\neg$ *composition-fri-bad-with-canonical-sampled-layer-chain s out*

definition *generic-fri-canonical-raw-index-failure*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow \text{bool}$

where

generic-fri-canonical-raw-index-failure roots challenges query-idxs
 $\longleftrightarrow$
 $(\exists \text{round-idx} < \text{length query-idxs}.$
 $\quad \exists \text{layer-idx} < \text{length challenges}.$
 $\quad \text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx} \geq$
 $\quad \text{length (fri-canonical-domain-at layer-idx)})$

definition *trace-fri-canonical-raw-index-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-canonical-raw-index-failure s out $\longleftrightarrow$
 $(\exists \text{trace-roots trace-bs trace-final dg composition-roots}$
 $\quad \text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\quad \text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\quad \text{trace-table doms layers}.$
 $\text{trace-fri-partial-candidate-opening-evidence s out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence trace-table-low-degree}$
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers} \wedge$
 $\text{generic-fri-canonical-raw-index-failure trace-roots trace-bs}$
 $\text{fri-query-idxs})$

definition *composition-fri-canonical-raw-index-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-canonical-raw-index-failure s out $\longleftrightarrow$
 $(\exists \text{trace-roots trace-bs trace-final fri-dg}$
 $\quad \text{opening-composition-roots composition-bs composition-final}$
 $\quad \text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\quad \text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\quad \text{trace-query-idxs trace-openings composition-query-idxs}$
 $\quad \text{composition-openings trace-table composition-table doms layers}.$
 $\text{composition-fri-partial-candidate-opening-evidence s out trace-roots}$
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$

composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
 $\wedge$
generic-fri-canonical-raw-index-failure opening-composition-roots
composition-bs fri-query-idxs)

definition *fri-sampled-successor-domains-square*

$:: 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

fri-sampled-successor-domains-square roots query-idxs doms $\longleftrightarrow$
 $(\forall \text{round-idx} < \text{length query-idxs}.$
 $\quad \forall \text{layer-idx} < \text{length roots}.$
 $\quad \text{doms ! Suc layer-idx !}$
 $\quad \text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} =$
 $(\text{doms ! layer-idx !}$
 $\quad \text{fri-evidence-next-idx roots query-idxs round-idx layer-idx})^2)$

definition *generic-fri-full-cover-sampled-domain-failure*

$:: 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

generic-fri-full-cover-sampled-domain-failure roots query-idxs doms
 $\longleftrightarrow$
 $\neg \text{fri-sampled-next-domains-align roots query-idxs doms} \vee$
 $\neg \text{fri-sampled-successor-domains-square roots query-idxs doms}$

definition *trace-fri-sampled-full-cover-sampled-domain-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-sampled-full-cover-sampled-domain-failure s out $\longleftrightarrow$
 $(\exists \text{trace-roots trace-bs trace-final dg composition-roots}$
 $\quad \text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\quad \text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\quad \text{trace-table doms layers}.$
trace-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
 $\text{trace-table (clength} - 1) \text{ trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers} \wedge$
generic-fri-full-cover-sampled-domain-failure trace-roots
 $\text{fri-query-idxs doms})$

definition *composition-fri-sampled-full-cover-sampled-domain-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-sampled-domain-failure *s out*
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{ opening-composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{ f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table doms layers.}$
 $\text{ composition-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{ trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{ composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings trace-table}$
 $\text{ composition-table} \wedge$
 $\text{ generic-fri-sampled-layer-chain-evidence}$
 $\text{ (composition-table-low-degree (to-nat fri-dg)) composition-table}$
 $\text{ (to-nat fri-dg) opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs composition-round-layers doms layers}$
 $\wedge$
 $\text{ generic-fri-full-cover-sampled-domain-failure}$
 $\text{ opening-composition-roots fri-query-idxs doms})$

lemma *fri-sampled-successor-domains-squareD:*

assumes *fri-sampled-successor-domains-square* *roots query-idxs doms*
and *round-idx < length query-idxs*
and *layer-idx < length roots*

shows

doms ! Suc layer-idx !
 $\text{ fri-evidence-next-idx roots query-idxs round-idx layer-idx} =$
 $(\text{doms ! layer-idx !}$
 $\text{ fri-evidence-next-idx roots query-idxs round-idx layer-idx})^2$

using *assms* **unfolding** *fri-sampled-successor-domains-square-def* **by** *blast*

lemma *fri-canonical-domain-at-length:*

$\text{ length (fri-canonical-domain-at layer-idx) } =$
 $(\text{clength} * \text{scale}) \text{ div } 2 \wedge \text{ layer-idx}$
unfolding *fri-canonical-domain-at-def* **by** *simp*

lemma *fri-canonical-domain-at-nth:*

assumes *idx < length (fri-canonical-domain-at layer-idx)*
shows *fri-canonical-domain-at layer-idx ! idx =*
 $(h \wedge \text{idx} * \text{shift}) \wedge (2 \wedge \text{layer-idx})$
using *assms* **unfolding** *fri-canonical-domain-at-def* **by** *simp*

```

lemma fri-canonical-domains-length:
  length (fri-canonical-domains n) = Suc n
  unfolding fri-canonical-domains-def by simp

lemma fri-canonical-domains-nth:
  assumes layer-idx ≤ n
  shows fri-canonical-domains n ! layer-idx =
    fri-canonical-domain-at layer-idx
proof –
  show ?thesis
  proof (cases layer-idx < n)
  case True
  have in-left:
    layer-idx < length (map fri-canonical-domain-at [0..n])
    using True by simp
  have append-nth:
    (map fri-canonical-domain-at [0..n] @
      [fri-canonical-domain-at n] ! layer-idx =
```

*(if layer-idx < length (map fri-canonical-domain-at [0..*n*])*
*then map fri-canonical-domain-at [0..*n*] ! layer-idx*
else [fri-canonical-domain-at n] !
*(layer-idx – length (map fri-canonical-domain-at [0..*n*]))*

```

    by (rule nth-append)
  have fri-canonical-domains n ! layer-idx =
    map fri-canonical-domain-at [0..n] ! layer-idx
    unfolding fri-canonical-domains-def
    using append-nth in-left by simp
  also have ... = fri-canonical-domain-at layer-idx
    using True by simp
  finally show ?thesis .
next
case False
  then have layer-idx = n
    using assms by simp
  then have eq-n: layer-idx = n .
  have len-left: length (map fri-canonical-domain-at [0..n]) = n
    by simp
  have append-nth:
    (map fri-canonical-domain-at [0..n] @
      [fri-canonical-domain-at n] ! n =
```

*(if n < length (map fri-canonical-domain-at [0..*n*])*
*then map fri-canonical-domain-at [0..*n*] ! n*
else [fri-canonical-domain-at n] !
*(n – length (map fri-canonical-domain-at [0..*n*]))*

```

    by (rule nth-append)
  show ?thesis
    unfolding fri-canonical-domains-def
    using eq-n len-left append-nth by simp
qed

```

qed

lemma *fri-canonical-domain-at-0*:

fri-canonical-domain-at 0 = eval-domain

proof (*rule nth-equalityI*)

show *length (fri-canonical-domain-at 0) = length eval-domain*

by (*simp add: fri-canonical-domain-at-length eval-domain-length*)

next

fix *i*

assume *i-bound: i < length (fri-canonical-domain-at 0)*

then have *i-eval: i < clength * scale*

by (*simp add: fri-canonical-domain-at-length*)

show *fri-canonical-domain-at 0 ! i = eval-domain ! i*

using *fri-canonical-domain-at-nth[OF i-bound] eval-domain-nth[OF i-eval]*

by *simp*

qed

lemma *fri-canonical-domains-0*:

0 < length (fri-canonical-domains n)

by (*simp add: fri-canonical-domains-length*)

lemma *fri-canonical-domains-first*:

fri-canonical-domains n ! 0 = eval-domain

using *fri-canonical-domains-nth[of 0 n] fri-canonical-domain-at-0*

by *simp*

lemma *fri-sampled-table-fold-to-canonical-domain*:

assumes *fold*:

*fri-sampled-table-fold ch len raw (2 ^ layer-idx) fri-dom layer chunk
next-value*

and *raw-bound: raw < length (fri-canonical-domain-at layer-idx)*

shows

*fri-sampled-table-fold ch len raw (2 ^ layer-idx)
(fri-canonical-domain-at layer-idx) layer chunk next-value*

proof —

from *fold* **obtain** *xp xp-path xn xn-path* **where**

chunk: fri-layer-opening-chunk len xp xp-path xn xn-path chunk

and *match: fri-opening-matches-table len raw layer xp xn*

and *dom-raw*:

*fri-dom ! raw = (h ^ raw * shift) ^ (2 ^ layer-idx)*

and *next-eq*:

next-value =

fri-table-fold-value ch layer fri-dom len (2 ^ layer-idx) raw

by (*elim fri-sampled-table-foldE*)

have *canonical-raw*:

fri-canonical-domain-at layer-idx ! raw =

*(h ^ raw * shift) ^ (2 ^ layer-idx)*

by (*rule fri-canonical-domain-at-nth[OF raw-bound]*)

have *fold-eq*:

```

next-value =
  fri-table-fold-value ch layer (fri-canonical-domain-at layer-idx)
    len (2 ^ layer-idx) raw
using next-eq dom-row canonical-row
unfolding fri-table-fold-value-def by simp
show ?thesis
by (rule fri-sampled-table-foldI
    [OF chunk match canonical-row fold-eq])
qed

lemma generic-fri-sampled-layer-chain-to-canonical-domains:
assumes chain:
  generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers
and raw-bounds:
   $\bigwedge$  round-idx layer-idx.
    round-idx < length query-idxs  $\implies$ 
    layer-idx < length challenges  $\implies$ 
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
    length (fri-canonical-domain-at layer-idx)
shows
  generic-fri-canonical-sampled-layer-chain-evidence low-degree
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers
proof –
have partial:
  generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
have len-layers: length layers = Suc (length challenges)
by (rule generic-fri-sampled-layer-chain-evidenceD(3)[OF chain])
have layer0: layers ! 0 = candidate-table
by (rule generic-fri-sampled-layer-chain-evidenceD(5)[OF chain])
have final:
  fri-final-constant-consistent (layers ! length challenges)
  final-value
by (rule generic-fri-sampled-layer-chain-evidenceD(6)[OF chain])
have canonical-chain:
  generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    (fri-canonical-domains (length challenges)) layers
unfolding generic-fri-sampled-layer-chain-evidence-def
proof (intro conjI allI impI)
show
  generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
by (rule partial)
next

```



```

show length (fri-canonical-domains (length challenges)) =
  Suc (length challenges)
  by (rule fri-canonical-domains-length)
next
show length layers = Suc (length challenges)
  by (rule len-layers)
next
show fri-canonical-domains (length challenges) ! 0 = eval-domain
  by (rule fri-canonical-domains-first)
next
show layers ! 0 = candidate-table
  by (rule layer0)
next
show
  fri-final-constant-consistent (layers ! length challenges)
  final-value
  by (rule final)
next
fix round-idx layer-idx
assume round-bound: round-idx < length query-idxs
assume layer-bound: layer-idx < length challenges
show
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (layers ! Suc layer-idx)
  by (rule generic-fri-sampled-layer-chain-evidence-sample(1)
    [OF chain round-bound layer-bound])
next
fix round-idx layer-idx
assume round-bound: round-idx < length query-idxs
assume layer-bound: layer-idx < length challenges
have old-fold:
  fri-sampled-table-fold (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (doms ! layer-idx)
    (layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  by (rule generic-fri-sampled-layer-chain-evidence-sample(2)
    [OF chain round-bound layer-bound])
have canonical-at:
  fri-sampled-table-fold (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-canonical-domain-at layer-idx)
    (layers ! layer-idx)

```

```

    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  by (rule fri-sampled-table-fold-to-canonical-domain
    [OF old-fold raw-bounds[OF round-bound layer-bound]])
have dom-nth:
  fri-canonical-domains (length challenges) ! layer-idx =
    fri-canonical-domain-at layer-idx
  by (rule fri-canonical-domains-nth) (use layer-bound in simp)
show
  fri-sampled-table-fold (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-canonical-domains (length challenges) ! layer-idx)
    (layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  using canonical-at dom-nth by simp
qed
show ?thesis
  using canonical-chain
  unfolding generic-fri-canonical-sampled-layer-chain-evidence-def
  by simp
qed

lemma generic-fri-canonical-sampled-layer-chain-evidenceD:
  assumes
    generic-fri-canonical-sampled-layer-chain-evidence low-degree
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers
  shows
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    (fri-canonical-domains (length challenges)) layers
  using assms
  unfolding generic-fri-canonical-sampled-layer-chain-evidence-def by simp

lemma generic-fri-canonical-sampled-layer-chain-evidenceI:
  assumes
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    (fri-canonical-domains (length challenges)) layers
  shows
    generic-fri-canonical-sampled-layer-chain-evidence low-degree
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers
  using assms

```

unfolding *generic-fri-canonical-sampled-layer-chain-evidence-def* **by** *simp*

lemma *generic-fri-sampled-missing-canonical-imp-raw-index-failure*:

assumes *chain*:

*generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers*

and *missing*:

$\neg$ *generic-fri-canonical-sampled-layer-chain-evidence low-degree
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers*

shows *generic-fri-canonical-raw-index-failure roots challenges query-idxs*

proof (rule *ccontr*)

assume *not-failure*:

$\neg$ *generic-fri-canonical-raw-index-failure roots challenges
query-idxs*

have *raw-bounds*:

$\bigwedge$ *round-idx layer-idx*.

round-idx < length query-idxs $\implies$

layer-idx < length challenges $\implies$

*fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
length (fri-canonical-domain-at layer-idx)*

using *not-failure*

unfolding *generic-fri-canonical-raw-index-failure-def* **by** *force*

have *canonical*:

*generic-fri-canonical-sampled-layer-chain-evidence low-degree
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers*

by (rule *generic-fri-sampled-layer-chain-to-canonical-domains*
[*OF chain raw-bounds*])

show *False*

using *missing canonical* **by** *contradiction*

qed

lemma *trace-fri-missing-canonical-sampled-imp-raw-index-failure*:

assumes *missing*:

trace-fri-sampled-missing-canonical-sampled-layer-chain s out

shows *trace-fri-canonical-raw-index-failure s out*

proof –

from *missing* **have** *sampled*:

trace-fri-bad-with-sampled-layer-chain s out

and *not-canonical*:

$\neg$ *trace-fri-bad-with-canonical-sampled-layer-chain s out*

unfolding *trace-fri-sampled-missing-canonical-sampled-layer-chain-def*

by *blast+*

from *sampled* **obtain** *trace-roots trace-bs trace-final dg*

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr candidate-query-idxs

trace-openings trace-table doms layers

where *evidence*:
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *chain*:
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
unfolding *trace-fri-bad-with-sampled-layer-chain-def* **by** *blast*
have *generic-missing*:
 $\neg$ *generic-fri-canonical-sampled-layer-chain-evidence*
trace-table-low-degree trace-table (clength - 1) trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers layers
proof
assume *canonical*:
generic-fri-canonical-sampled-layer-chain-evidence
trace-table-low-degree trace-table (clength - 1) trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers layers
then have *trace-fri-bad-with-canonical-sampled-layer-chain s out*
unfolding *trace-fri-bad-with-canonical-sampled-layer-chain-def*
using *evidence* **by** *blast*
then show *False*
using *not-canonical* **by** *contradiction*
qed
have *raw-failure*:
generic-fri-canonical-raw-index-failure trace-roots trace-bs
fri-query-idxs
by (*rule generic-fri-sampled-missing-canonical-imp-raw-index-failure*
[OF chain generic-missing])
show *?thesis*
unfolding *trace-fri-canonical-raw-index-failure-def*
using *evidence chain raw-failure* **by** *blast*
qed

lemma *composition-fri-missing-canonical-sampled-imp-raw-index-failure*:
assumes *missing*:
composition-fri-sampled-missing-canonical-sampled-layer-chain s out
shows *composition-fri-canonical-raw-index-failure s out*
proof –
from *missing* **have** *sampled*:
composition-fri-bad-with-sampled-layer-chain s out
and *not-canonical*:
 $\neg$ *composition-fri-bad-with-canonical-sampled-layer-chain s out*
unfolding *composition-fri-sampled-missing-canonical-sampled-layer-chain-def*
by *blast+*
from *sampled* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
where *evidence*:
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
and *chain*:
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
unfolding *composition-fri-bad-with-sampled-layer-chain-def* **by** *auto*
have *generic-missing*:
 $\neg$ *generic-fri-canonical-sampled-layer-chain-evidence*
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers layers
proof
assume *canonical*:
generic-fri-canonical-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers layers
then have *composition-fri-bad-with-canonical-sampled-layer-chain s out*
unfolding *composition-fri-bad-with-canonical-sampled-layer-chain-def*
using *evidence* **by** *meson*
then show *False*
using *not-canonical* **by** *contradiction*
qed
have *raw-failure*:
generic-fri-canonical-raw-index-failure opening-composition-roots
composition-bs fri-query-idxs
by (*rule generic-fri-sampled-missing-canonical-imp-raw-index-failure*
[OF chain generic-missing])
show *?thesis*
unfolding *composition-fri-canonical-raw-index-failure-def*
using *evidence chain raw-failure* **by** *meson*
qed
lemma *wp-trace-fri-missing-canonical-sampled-bound-from-raw-index*:
assumes *raw-index*:
wp-event verify-monad (trace-fri-canonical-raw-index-failure s) s
 $\leq R$

shows
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-sampled-missing-canonical-sampled-layer-chain } s) \ s \leq R$
proof –
have $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-sampled-missing-canonical-sampled-layer-chain } s) \ s$
 $\leq wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-canonical-raw-index-failure } s) \ s$
by $(rule \ wp\text{-event-mono})$
 $(rule \ trace\text{-fri-missing-canonical-sampled-imp-raw-index-failure})$
also have $\dots \leq R$
by $(rule \ raw\text{-index})$
finally show $?thesis$.
qed

lemma $wp\text{-composition-fri-missing-canonical-sampled-bound-from-raw-index}$:
assumes $raw\text{-index}$:
 $wp\text{-event } verify\text{-monad } (composition\text{-fri-canonical-raw-index-failure } s) \ s$
 $\leq R$
shows
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-sampled-missing-canonical-sampled-layer-chain } s) \ s$
 $\leq R$
proof –
have $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-sampled-missing-canonical-sampled-layer-chain } s) \ s$
 $\leq wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-canonical-raw-index-failure } s) \ s$
by $(rule \ wp\text{-event-mono})$
 $(rule \ composition\text{-fri-missing-canonical-sampled-imp-raw-index-failure})$
also have $\dots \leq R$
by $(rule \ raw\text{-index})$
finally show $?thesis$.
qed

lemma $generic\text{-fri-sampled-layer-chain-no-canonical-raw-index-failure}$:
assumes $chain$:
 $generic\text{-fri-sampled-layer-chain-evidence } low\text{-degree } candidate\text{-table}$
 $degree\text{-bound } roots \ challenges \ final\text{-value } query\text{-idxs } round\text{-layers}$
 $doms \ layers$
shows $\neg generic\text{-fri-canonical-raw-index-failure } roots \ challenges$
 $query\text{-idxs}$
proof
assume $failure$:
 $generic\text{-fri-canonical-raw-index-failure } roots \ challenges \ query\text{-idxs}$
then obtain $round\text{-idx } layer\text{-idx}$ **where** $round\text{-bound}$:
 $round\text{-idx} < length \ query\text{-idxs}$
and $layer\text{-bound}$: $layer\text{-idx} < length \ challenges$
and $raw\text{-ge}$:

```

    fri-evidence-layer-idx roots query-idxs round-idx layer-idx ≥
      length (fri-canonical-domain-at layer-idx)
  unfolding generic-fri-canonical-raw-index-failure-def by blast
have partial:
  generic-fri-partial-evidence low-degree candidate-table degree-bound
  roots challenges final-value query-idxs round-layers
  by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
have roots-challenges: length challenges = length roots
  by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
have roots-bound: layer-idx < length roots
  using layer-bound roots-challenges by simp
have fold:
  fri-sampled-table-fold (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (doms ! layer-idx)
    (layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  by (rule generic-fri-sampled-layer-chain-evidence-sample(2)
    [OF chain round-bound layer-bound])
from fold obtain xp xp-path xn xn-path where
  match:
    fri-opening-matches-table (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (layers ! layer-idx) xp xn
  by (elim fri-sampled-table-foldE)
have raw-lt-len:
  fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
    fri-evidence-layer-len roots layer-idx
  by (rule fri-opening-matches-tableD(1)[OF match])
have len-eq:
  fri-evidence-layer-len roots layer-idx =
    length (fri-canonical-domain-at layer-idx)
  unfolding fri-evidence-layer-len-def fri-canonical-domain-at-length
  using fri-layer-lengths-nth-div[OF roots-bound, of clength * scale]
  by simp
show False
  using raw-lt-len raw-ge unfolding len-eq by simp
qed

```

```

lemma trace-fri-canonical-raw-index-failure-false:
  ¬ trace-fri-canonical-raw-index-failure s out
proof
  assume raw: trace-fri-canonical-raw-index-failure s out
  then obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers

```

composition-round-layers fr candidate-query-idxs trace-openings
trace-table doms layers
where *chain*:
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
and *failure*:
generic-fri-canonical-raw-index-failure trace-roots trace-bs
fri-query-idxs
unfolding *trace-fri-canonical-raw-index-failure-def* **by** *blast*
have $\neg$ *generic-fri-canonical-raw-index-failure trace-roots trace-bs*
fri-query-idxs
by (rule *generic-fri-sampled-layer-chain-no-canonical-raw-index-failure*
 $[OF\ chain]$)
then show *False*
using *failure* **by** *contradiction*
qed

lemma *composition-fri-canonical-raw-index-failure-false*:
 $\neg$ *composition-fri-canonical-raw-index-failure s out*
proof
assume *raw*: *composition-fri-canonical-raw-index-failure s out*
then obtain *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
where *chain*:
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
and *failure*:
generic-fri-canonical-raw-index-failure opening-composition-roots
composition-bs fri-query-idxs
unfolding *composition-fri-canonical-raw-index-failure-def* **by** *meson*
have $\neg$ *generic-fri-canonical-raw-index-failure*
opening-composition-roots composition-bs fri-query-idxs
by (rule *generic-fri-sampled-layer-chain-no-canonical-raw-index-failure*
 $[OF\ chain]$)
then show *False*
using *failure* **by** *contradiction*
qed

lemma *wp-trace-fri-canonical-raw-index-failure-zero*:
wp-event verify-monad (trace-fri-canonical-raw-index-failure s) s ≤ 0
proof –
have *wp-event verify-monad (trace-fri-canonical-raw-index-failure s) s*

$\leq \text{wp-event verify-monad } (\lambda\cdot. \text{False}) \ s$
by (rule wp-event-mono)
 (simp add: trace-fri-canonical-raw-index-failure-false)
also have ... = 0
unfolding wp-event-def wp-def dist-expect-def **by** simp
finally show ?thesis .
qed

lemma wp-composition-fri-canonical-raw-index-failure-zero:
 wp-event verify-monad
 (composition-fri-canonical-raw-index-failure s) s ≤ 0

proof –
have wp-event verify-monad
 (composition-fri-canonical-raw-index-failure s) s
 $\leq \text{wp-event verify-monad } (\lambda\cdot. \text{False}) \ s$
by (rule wp-event-mono)
 (simp add: composition-fri-canonical-raw-index-failure-false)
also have ... = 0
unfolding wp-event-def wp-def dist-expect-def **by** simp
finally show ?thesis .
qed

lemma wp-trace-fri-missing-canonical-sampled-zero:
 wp-event verify-monad
 (trace-fri-sampled-missing-canonical-sampled-layer-chain s) s ≤ 0
by (rule wp-trace-fri-missing-canonical-sampled-bound-from-raw-index)
 (rule wp-trace-fri-canonical-raw-index-failure-zero)

lemma wp-composition-fri-missing-canonical-sampled-zero:
 wp-event verify-monad
 (composition-fri-sampled-missing-canonical-sampled-layer-chain s) s
 ≤ 0
by (rule wp-composition-fri-missing-canonical-sampled-bound-from-raw-index)
 (rule wp-composition-fri-canonical-raw-index-failure-zero)

lemma trace-fri-bad-with-canonical-sampled-layer-chain-imp-sampled:
assumes trace-fri-bad-with-canonical-sampled-layer-chain s out
shows trace-fri-bad-with-sampled-layer-chain s out

proof –
from assms **obtain** trace-roots trace-bs trace-final dg composition-roots
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table layers
where evidence:
 trace-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table

and chain:
generic-fri-canonical-sampled-layer-chain-evidence
trace-table-low-degree trace-table (clength - 1) trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers layers
unfolding *trace-fri-bad-with-canonical-sampled-layer-chain-def* **by** *blast*
have *sampled:*
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers
(fri-canonical-domains (length trace-bs)) layers
by (rule *generic-fri-canonical-sampled-layer-chain-evidenceD[OF chain]*)
show *?thesis*
unfolding *trace-fri-bad-with-sampled-layer-chain-def*
using *evidence sampled by blast*
qed

lemma *composition-fri-bad-with-canonical-sampled-layer-chain-imp-sampled:*
assumes *composition-fri-bad-with-canonical-sampled-layer-chain s out*
shows *composition-fri-bad-with-sampled-layer-chain s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table layers
where *evidence:*
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
and chain:
generic-fri-canonical-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers layers
unfolding *composition-fri-bad-with-canonical-sampled-layer-chain-def*
by *meson*
have *sampled:*
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
(fri-canonical-domains (length composition-bs)) layers
by (rule *generic-fri-canonical-sampled-layer-chain-evidenceD[OF chain]*)
show *?thesis*

unfolding *composition-fri-bad-with-sampled-layer-chain-def*
using *evidence sampled* **by** *meson*
qed

lemma *trace-fri-sampled-layer-chain-canonical-or-missing:*
assumes *trace-fri-bad-with-sampled-layer-chain s out*
shows
trace-fri-bad-with-canonical-sampled-layer-chain s out $\vee$
trace-fri-sampled-missing-canonical-sampled-layer-chain s out
using *assms*
unfolding *trace-fri-sampled-missing-canonical-sampled-layer-chain-def*
by *blast*

lemma *composition-fri-sampled-layer-chain-canonical-or-missing:*
assumes *composition-fri-bad-with-sampled-layer-chain s out*
shows
composition-fri-bad-with-canonical-sampled-layer-chain s out $\vee$
composition-fri-sampled-missing-canonical-sampled-layer-chain s out
using *assms*
unfolding
composition-fri-sampled-missing-canonical-sampled-layer-chain-def
by *blast*

lemma *wp-trace-fri-sampled-bound-from-canonical-and-missing:*
assumes *canonical:*
wp-event verify-monad
(trace-fri-bad-with-canonical-sampled-layer-chain s) s $\leq$ C
and *missing:*
wp-event verify-monad
(trace-fri-sampled-missing-canonical-sampled-layer-chain s) s $\leq$ M
shows
wp-event verify-monad (trace-fri-bad-with-sampled-layer-chain s) s
 $\leq C + M$
proof –
have *wp-event verify-monad (trace-fri-bad-with-sampled-layer-chain s) s*
 $\leq$ *wp-event verify-monad*
(λ out. trace-fri-bad-with-canonical-sampled-layer-chain s out $\vee$
trace-fri-sampled-missing-canonical-sampled-layer-chain s out) s
by *(rule wp-event-mono)*
(rule trace-fri-sampled-layer-chain-canonical-or-missing)
also have $\dots \leq C + M$
by *(rule order-trans[OF wp-event-union-bound*
add-mono[OF canonical missing]])
finally show *?thesis* .
qed

lemma *wp-composition-fri-sampled-bound-from-canonical-and-missing:*
assumes *canonical:*
wp-event verify-monad

(composition-fri-bad-with-canonical-sampled-layer-chain s) $s \leq C$
and missing:
 wp-event verify-monad
 (composition-fri-sampled-missing-canonical-sampled-layer-chain s) s
 $\leq M$
shows
 wp-event verify-monad
 (composition-fri-bad-with-sampled-layer-chain s) $s \leq C + M$
proof –
have wp-event verify-monad
 (composition-fri-bad-with-sampled-layer-chain s) s
 $\leq$ wp-event verify-monad
 ($\lambda out.$
 composition-fri-bad-with-canonical-sampled-layer-chain s out $\vee$
 composition-fri-sampled-missing-canonical-sampled-layer-chain s out)
 s
by (rule wp-event-mono)
 (rule composition-fri-sampled-layer-chain-canonical-or-missing)
also have ... $\leq C + M$
by (rule order-trans[OF wp-event-union-bound
 add-mono[OF canonical missing]])
finally show ?thesis .
qed

lemma wp-trace-fri-sampled-bound-from-canonical:
assumes canonical:
 wp-event verify-monad
 (trace-fri-bad-with-canonical-sampled-layer-chain s) $s \leq C$
shows
 wp-event verify-monad (trace-fri-bad-with-sampled-layer-chain s) s
 $\leq C$
proof –
have wp-event verify-monad (trace-fri-bad-with-sampled-layer-chain s) s
 $\leq C + 0$
by (rule wp-trace-fri-sampled-bound-from-canonical-and-missing
 [OF canonical wp-trace-fri-missing-canonical-sampled-zero])
then show ?thesis **by** simp
qed

lemma wp-composition-fri-sampled-bound-from-canonical:
assumes canonical:
 wp-event verify-monad
 (composition-fri-bad-with-canonical-sampled-layer-chain s) $s \leq C$
shows
 wp-event verify-monad (composition-fri-bad-with-sampled-layer-chain s) s
 $\leq C$
proof –
have wp-event verify-monad
 (composition-fri-bad-with-sampled-layer-chain s) $s \leq C + 0$

by (rule wp-composition-fri-sampled-bound-from-canonical-and-missing
 [OF canonical wp-composition-fri-missing-canonical-sampled-zero])
 then show ?thesis by simp
 qed

lemma wp-trace-fri-canonical-sampled-bound-from-sampled:
assumes sampled:
 wp-event verify-monad (trace-fri-bad-with-sampled-layer-chain s) s
 $\leq R$
shows
 wp-event verify-monad
 (trace-fri-bad-with-canonical-sampled-layer-chain s) s $\leq R$

proof –
have wp-event verify-monad
 (trace-fri-bad-with-canonical-sampled-layer-chain s) s $\leq$
 wp-event verify-monad (trace-fri-bad-with-sampled-layer-chain s) s
by (rule wp-event-mono)
 (rule trace-fri-bad-with-canonical-sampled-layer-chain-imp-sampled)
then show ?thesis
by (rule order-trans[OF - sampled])
 qed

lemma wp-composition-fri-canonical-sampled-bound-from-sampled:
assumes sampled:
 wp-event verify-monad
 (composition-fri-bad-with-sampled-layer-chain s) s $\leq R$
shows
 wp-event verify-monad
 (composition-fri-bad-with-canonical-sampled-layer-chain s) s $\leq R$

proof –
have wp-event verify-monad
 (composition-fri-bad-with-canonical-sampled-layer-chain s) s $\leq$
 wp-event verify-monad
 (composition-fri-bad-with-sampled-layer-chain s) s
by (rule wp-event-mono)
 (rule composition-fri-bad-with-canonical-sampled-layer-chain-imp-sampled)
then show ?thesis
by (rule order-trans[OF - sampled])
 qed

lemma wp-trace-fri-canonical-sampled-bound-from-full-cover-and-missing:
assumes full-bound:
 wp-event verify-monad
 (trace-fri-bad-with-full-cover-layer-chain s bad) s $\leq R$
and missing-bound:
 wp-event verify-monad
 (trace-fri-sampled-missing-full-cover s bad) s $\leq M$
shows
 wp-event verify-monad

(*trace-fri-bad-with-canonical-sampled-layer-chain s*) $s \leq R + M$
by (*rule wp-trace-fri-canonical-sampled-bound-from-sampled*)
(*rule wp-trace-fri-sampled-bound-from-full-cover-and-missing*
[*OF full-bound missing-bound*])

lemma *wp-composition-fri-canonical-sampled-bound-from-full-cover-and-missing*:

assumes *full-bound*:

wp-event verify-monad

(*composition-fri-bad-with-full-cover-layer-chain s bad*) $s \leq R$

and *missing-bound*:

wp-event verify-monad

(*composition-fri-sampled-missing-full-cover s bad*) $s \leq M$

shows

wp-event verify-monad

(*composition-fri-bad-with-canonical-sampled-layer-chain s*) s
 $\leq R + M$

by (*rule wp-composition-fri-canonical-sampled-bound-from-sampled*)

(*rule wp-composition-fri-sampled-bound-from-full-cover-and-missing*
[*OF full-bound missing-bound*])

lemma *verify-monad-trace-fri-missing-canonical-sampled-extracts-evidence*:

assumes *outcome*:

Some (result, final-state) ∈ set-dist (execute verify-monad s)

and *missing*:

trace-fri-sampled-missing-canonical-sampled-layer-chain s
(*Some (result, final-state)*)

obtains *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr candidate-query-idxs trace-openings

trace-table doms layers

where

trace-fri-partial-candidate-opening-evidence s

(*Some (result, final-state)*) *trace-roots trace-bs trace-final dg*

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr candidate-query-idxs

trace-openings trace-table

generic-fri-sampled-layer-chain-evidence trace-table-low-degree

trace-table (clength - 1) trace-roots trace-bs trace-final

fri-query-idxs trace-round-layers doms layers

$\wedge$ *canonical-layers*. $\neg$

generic-fri-canonical-sampled-layer-chain-evidence

trace-table-low-degree trace-table (clength - 1) trace-roots

trace-bs trace-final fri-query-idxs trace-round-layers

canonical-layers

proof —

have *sampld*:

trace-fri-bad-with-sampled-layer-chain s

(*Some (result, final-state)*)

and *not-canonical*:

```

    ¬ trace-fri-bad-with-canonical-sampled-layer-chain s
      (Some (result, final-state))
  using missing
  unfolding trace-fri-sampled-missing-canonical-sampled-layer-chain-def
  by blast+
from sampled obtain trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table doms layers
where evidence:
  trace-fri-partial-candidate-opening-evidence s
    (Some (result, final-state)) trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr candidate-query-idxs
    trace-openings trace-table
and chain:
  generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers
  unfolding trace-fri-bad-with-sampled-layer-chain-def by blast
have no-canonical:
  ∧ canonical-layers. ¬
    generic-fri-canonical-sampled-layer-chain-evidence
      trace-table-low-degree trace-table (clength - 1) trace-roots
      trace-bs trace-final fri-query-idxs trace-round-layers
      canonical-layers
proof
  fix canonical-layers
  assume canonical:
    generic-fri-canonical-sampled-layer-chain-evidence
      trace-table-low-degree trace-table (clength - 1) trace-roots
      trace-bs trace-final fri-query-idxs trace-round-layers
      canonical-layers
  have trace-fri-bad-with-canonical-sampled-layer-chain s
    (Some (result, final-state))
    unfolding trace-fri-bad-with-canonical-sampled-layer-chain-def
    using evidence canonical by blast
  then show False
    using not-canonical by contradiction
qed
show ?thesis
  by (rule that[OF evidence chain no-canonical])
qed

lemma verify-monad-composition-fri-missing-canonical-sampled-extracts-evidence:
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and missing:
    composition-fri-sampled-missing-canonical-sampled-layer-chain s

```

(Some (result, final-state))

obtains trace-roots trace-bs trace-final fri-dg
 opening-composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers fr
 f-fri-roots f-final as dg composition-fri-roots final
 trace-query-idxs trace-openings composition-query-idxs
 composition-openings trace-table composition-table doms layers

where
 composition-fri-partial-candidate-opening-evidence s
 (Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
 opening-composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers fr
 f-fri-roots f-final as dg composition-fri-roots final
 trace-query-idxs trace-openings composition-query-idxs
 composition-openings trace-table composition-table
 generic-fri-sampled-layer-chain-evidence
 (composition-table-low-degree (to-nat fri-dg)) composition-table
 (to-nat fri-dg) opening-composition-roots composition-bs
 composition-final fri-query-idxs composition-round-layers doms layers
 $\bigwedge$ canonical-layers. $\neg$
 generic-fri-canonical-sampled-layer-chain-evidence
 (composition-table-low-degree (to-nat fri-dg)) composition-table
 (to-nat fri-dg) opening-composition-roots composition-bs
 composition-final fri-query-idxs composition-round-layers
 canonical-layers

proof —
have sampled:
 composition-fri-bad-with-sampled-layer-chain s
 (Some (result, final-state))
and not-canonical:
 $\neg$ composition-fri-bad-with-canonical-sampled-layer-chain s
 (Some (result, final-state))
using missing
unfolding
 composition-fri-sampled-missing-canonical-sampled-layer-chain-def
by blast+
from sampled **obtain** trace-roots trace-bs trace-final fri-dg
 opening-composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers fr
 f-fri-roots f-final as dg composition-fri-roots final
 trace-query-idxs trace-openings composition-query-idxs
 composition-openings trace-table composition-table doms layers
where evidence:
 composition-fri-partial-candidate-opening-evidence s
 (Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
 opening-composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers fr
 f-fri-roots f-final as dg composition-fri-roots final
 trace-query-idxs trace-openings composition-query-idxs


```

    composition-openings trace-table composition-table
and chain:
    generic-fri-sampled-layer-chain-evidence
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers doms layers
    unfolding composition-fri-bad-with-sampled-layer-chain-def by meson
have no-canonical:
   $\wedge$ canonical-layers.  $\neg$ 
    generic-fri-canonical-sampled-layer-chain-evidence
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers
      canonical-layers
proof
  fix canonical-layers
  assume canonical:
    generic-fri-canonical-sampled-layer-chain-evidence
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers
      canonical-layers
  have composition-fri-bad-with-canonical-sampled-layer-chain s
    (Some (result, final-state))
  unfolding composition-fri-bad-with-canonical-sampled-layer-chain-def
  using evidence canonical by meson
  then show False
    using not-canonical by contradiction
qed
show ?thesis
  by (rule that[OF evidence chain no-canonical])
qed

```

```

lemma fri-canonical-domain-successor-square:
assumes next-bound:
   $idx < \text{length } (\text{fri-canonical-domain-at } (\text{Suc layer-idx}))$ 
and cur-bound:
   $idx < \text{length } (\text{fri-canonical-domain-at layer-idx})$ 
shows  $\text{fri-canonical-domain-at } (\text{Suc layer-idx}) ! idx =$ 
   $(\text{fri-canonical-domain-at layer-idx} ! idx)^2$ 
proof –
  have next-val:
     $\text{fri-canonical-domain-at } (\text{Suc layer-idx}) ! idx =$ 
     $(h \wedge idx * \text{shift}) \wedge (2 \wedge \text{Suc layer-idx})$ 
    by (rule fri-canonical-domain-at-nth[OF next-bound])
  have cur-val:
     $\text{fri-canonical-domain-at layer-idx} ! idx =$ 
     $(h \wedge idx * \text{shift}) \wedge (2 \wedge \text{layer-idx})$ 
    by (rule fri-canonical-domain-at-nth[OF cur-bound])

```

```

have square:
  (h ^ idx * shift) ^ (2 ^ Suc layer-idx) =
    ((h ^ idx * shift) ^ (2 ^ layer-idx))2
  by (metis mult.commute power2-eq-square power-Suc power-mult)
show ?thesis
  using next-val cur-val square by simp
qed

```

```

lemma fri-canonical-domains-successor-square:
  assumes layer-bound: Suc layer-idx ≤ n
  and idx-bound:
    idx < length (fri-canonical-domains n ! Suc layer-idx)
  and cur-idx-bound:
    idx < length (fri-canonical-domains n ! layer-idx)
  shows fri-canonical-domains n ! Suc layer-idx ! idx =
    (fri-canonical-domains n ! layer-idx ! idx)2
proof -
  have cur-bound: layer-idx ≤ n
  using layer-bound by simp
  have next-dom:
    fri-canonical-domains n ! Suc layer-idx =
      fri-canonical-domain-at (Suc layer-idx)
  by (rule fri-canonical-domains-nth[OF layer-bound])
  have cur-dom:
    fri-canonical-domains n ! layer-idx =
      fri-canonical-domain-at layer-idx
  by (rule fri-canonical-domains-nth[OF cur-bound])
  show ?thesis
  using fri-canonical-domain-successor-square
    [of idx layer-idx] idx-bound cur-idx-bound
  unfolding next-dom cur-dom by simp
qed

```

```

lemma fri-canonical-domains-sampled-next-align:
  assumes roots-bound: length roots ≤ n
  and idx-bound:
    ∧ round-idx layer-idx.
    round-idx < length query-idxs ⇒
    layer-idx < length roots ⇒
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
      length (fri-canonical-domain-at layer-idx)
  shows
    fri-sampled-next-domains-align roots query-idxs
      (fri-canonical-domains n)
  unfolding fri-sampled-next-domains-align-def
proof (intro allI impI)
  fix round-idx layer-idx
  assume round-bound: round-idx < length query-idxs
  and layer-bound: layer-idx < length roots

```

```

have layer-le: layer-idx ≤ n
  using layer-bound roots-bound by simp
have dom-eq:
  fri-canonical-domains n ! layer-idx =
    fri-canonical-domain-at layer-idx
  by (rule fri-canonical-domains-nth[OF layer-le])
have idx:
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (fri-canonical-domain-at layer-idx)
  by (rule idx-bound[OF round-bound layer-bound])
show
  fri-canonical-domains n ! layer-idx !
    fri-evidence-next-idx roots query-idxs round-idx layer-idx =
    (h ^ fri-evidence-next-idx roots query-idxs round-idx layer-idx *
      shift) ^ (2 ^ layer-idx)
  unfolding dom-eq
  by (rule fri-canonical-domain-at-nth[OF idx])
qed

```

lemma fri-canonical-domains-sampled-successor-square:

```

assumes roots-bound: length roots ≤ n
and cur-idx-bound:
  ∧ round-idx layer-idx.
  round-idx < length query-idxs ⇒
  layer-idx < length roots ⇒
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (fri-canonical-domains n ! layer-idx)
and next-idx-bound:
  ∧ round-idx layer-idx.
  round-idx < length query-idxs ⇒
  layer-idx < length roots ⇒
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (fri-canonical-domains n ! Suc layer-idx)
shows
  fri-sampled-successor-domains-square roots query-idxs
    (fri-canonical-domains n)
  unfolding fri-sampled-successor-domains-square-def
proof (intro allI impI)
  fix round-idx layer-idx
  assume round-bound: round-idx < length query-idxs
  and layer-bound: layer-idx < length roots
  have layer-le: Suc layer-idx ≤ n
  using layer-bound roots-bound by simp
  show
  fri-canonical-domains n ! Suc layer-idx !
    fri-evidence-next-idx roots query-idxs round-idx layer-idx =
    (fri-canonical-domains n ! layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)2
  by (rule fri-canonical-domains-successor-square[OF layer-le])

```

(rule next-idx-bound[OF round-bound layer-bound],
 rule cur-idx-bound[OF round-bound layer-bound])
qed

lemma *fri-canonical-domains-no-sampled-domain-failure*:
assumes roots-bound: $\text{length roots} \leq n$
and canonical-next-bound:
 $\bigwedge \text{round-idx layer-idx.}$
 $\text{round-idx} < \text{length query-idxs} \implies$
 $\text{layer-idx} < \text{length roots} \implies$
 $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} <$
 $\text{length (fri-canonical-domain-at layer-idx)}$
and canonical-successor-bound:
 $\bigwedge \text{round-idx layer-idx.}$
 $\text{round-idx} < \text{length query-idxs} \implies$
 $\text{layer-idx} < \text{length roots} \implies$
 $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} <$
 $\text{length (fri-canonical-domains } n \text{ ! Suc layer-idx)}$
shows $\neg \text{generic-fri-full-cover-sampled-domain-failure roots query-idxs}$
 $(\text{fri-canonical-domains } n)$
proof –
have next-align:
 $\text{fri-sampled-next-domains-align roots query-idxs}$
 $(\text{fri-canonical-domains } n)$
by (rule *fri-canonical-domains-sampled-next-align*
 [OF roots-bound canonical-next-bound])
have cur-bound:
 $\bigwedge \text{round-idx layer-idx.}$
 $\text{round-idx} < \text{length query-idxs} \implies$
 $\text{layer-idx} < \text{length roots} \implies$
 $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} <$
 $\text{length (fri-canonical-domains } n \text{ ! layer-idx)}$
proof –
fix round-idx layer-idx
assume round-bound: $\text{round-idx} < \text{length query-idxs}$
and layer-bound: $\text{layer-idx} < \text{length roots}$
have layer-le: $\text{layer-idx} \leq n$
using layer-bound roots-bound **by** simp
have dom-eq:
 $\text{fri-canonical-domains } n \text{ ! layer-idx} =$
 $\text{fri-canonical-domain-at layer-idx}$
by (rule *fri-canonical-domains-nth*[OF layer-le])
show
 $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} <$
 $\text{length (fri-canonical-domains } n \text{ ! layer-idx)}$
unfolding dom-eq
by (rule canonical-next-bound[OF round-bound layer-bound])
qed
have succ-square:

```

    fri-sampled-successor-domains-square roots query-idxs
      (fri-canonical-domains n)
  by (rule fri-canonical-domains-sampled-successor-square
      [OF roots-bound cur-bound canonical-successor-bound])
show ?thesis
  unfolding generic-fri-full-cover-sampled-domain-failure-def
  using next-align succ-square by simp
qed

lemma generic-fri-full-cover-domain-failure-imp-index-or-sampled-domain:
  assumes roots-len: length challenges = length roots
  and domain: generic-fri-full-cover-domain-failure roots challenges doms
  shows
    generic-fri-full-cover-index-failure roots query-idxs  $\vee$ 
    generic-fri-full-cover-sampled-domain-failure roots query-idxs doms
proof (cases generic-fri-full-cover-index-failure roots query-idxs)
  case True
  then show ?thesis by simp
next
  case False
  have covers: fri-sampled-layers-cover-fold-indices roots query-idxs
    using False unfolding generic-fri-full-cover-index-failure-def by simp
  show ?thesis
proof (cases
    generic-fri-full-cover-sampled-domain-failure roots query-idxs doms)
  case True
  then show ?thesis by simp
next
  case no-sampled: False
  have next-align: fri-sampled-next-domains-align roots query-idxs doms
  and succ-square:
    fri-sampled-successor-domains-square roots query-idxs doms
  using no-sampled
  unfolding generic-fri-full-cover-sampled-domain-failure-def
  by simp-all
  have no-square-fail:
     $\forall$  layer-idx < length challenges.
     $\forall$  idx < fri-evidence-layer-len roots layer-idx div 2.
    doms ! Suc layer-idx ! idx =
      (doms ! layer-idx ! idx)2
  proof (intro allI impI)
    fix layer-idx idx
    assume layer-bound: layer-idx < length challenges
    and idx-bound: idx < fri-evidence-layer-len roots layer-idx div 2
    have layer-root-bound: layer-idx < length roots
    using layer-bound roots-len by simp
    obtain round-idx where round-bound: round-idx < length query-idxs
    and idx-eq:
      fri-evidence-next-idx roots query-idxs round-idx layer-idx = idx

```

```

    by (rule fri-sampled-layers-cover-fold-indicesD
        [OF covers layer-root-bound idx-bound])
  show doms ! Suc layer-idx ! idx =
    (doms ! layer-idx ! idx)2
    using fri-sampled-successor-domains-squareD
    [OF succ-square round-bound layer-root-bound] idx-eq
  by simp
qed
have no-norm-fail:
  ∀ layer-idx < length challenges.
    ∀ idx < fri-evidence-layer-len roots layer-idx div 2.
      doms ! layer-idx ! idx =
        (h ^ idx * shift) ^ (2 ^ layer-idx)
proof (intro allI impI)
  fix layer-idx idx
  assume layer-bound: layer-idx < length challenges
    and idx-bound: idx < fri-evidence-layer-len roots layer-idx div 2
  have layer-root-bound: layer-idx < length roots
    using layer-bound roots-len by simp
  obtain round-idx where round-bound: round-idx < length query-idxs
    and idx-eq:
      fri-evidence-next-idx roots query-idxs round-idx layer-idx = idx
  by (rule fri-sampled-layers-cover-fold-indicesD
      [OF covers layer-root-bound idx-bound])
  show doms ! layer-idx ! idx =
    (h ^ idx * shift) ^ (2 ^ layer-idx)
    using fri-sampled-next-domains-alignD
    [OF next-align round-bound layer-root-bound] idx-eq
  by simp
qed
have False
  using domain no-square-fail no-norm-fail
  unfolding generic-fri-full-cover-domain-failure-def by simp
then show ?thesis by simp
qed
qed

lemma trace-fri-sampled-full-cover-domain-imp-index-or-sampled-domain:
  assumes domain: trace-fri-sampled-full-cover-domain-failure s out
  shows
    trace-fri-sampled-full-cover-index-failure s out ∨
    trace-fri-sampled-full-cover-sampled-domain-failure s out
proof -
  from domain obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table doms layers
  where evidence:
    trace-fri-partial-candidate-opening-evidence s out trace-roots

```

```

    trace-bs trace-final dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
  and chain:
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers
  and generic-domain:
    generic-fri-full-cover-domain-failure trace-roots trace-bs doms
  unfolding trace-fri-sampled-full-cover-domain-failure-def by blast
have partial:
  generic-fri-partial-evidence trace-table-low-degree trace-table
  (clength - 1) trace-roots trace-bs trace-final fri-query-idxs
  trace-round-layers
  by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
have roots-len: length trace-bs = length trace-roots
  by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
have split:
  generic-fri-full-cover-index-failure trace-roots fri-query-idxs ∨
  generic-fri-full-cover-sampled-domain-failure trace-roots
  fri-query-idxs doms
  by (rule generic-fri-full-cover-domain-failure-imp-index-or-sampled-domain
      [OF roots-len generic-domain])
then show ?thesis
proof
  assume generic-fri-full-cover-index-failure trace-roots fri-query-idxs
  then show ?thesis
    unfolding trace-fri-sampled-full-cover-index-failure-def
    using evidence chain by blast
next
  assume
    generic-fri-full-cover-sampled-domain-failure trace-roots
    fri-query-idxs doms
  then show ?thesis
    unfolding trace-fri-sampled-full-cover-sampled-domain-failure-def
    using evidence chain by blast
qed
qed

lemma composition-fri-sampled-full-cover-domain-imp-index-or-sampled-domain:
  assumes domain: composition-fri-sampled-full-cover-domain-failure s out
  shows
    composition-fri-sampled-full-cover-index-failure s out ∨
    composition-fri-sampled-full-cover-sampled-domain-failure s out
proof -
  from domain obtain trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr

```

```

f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
where evidence:
  composition-fri-partial-candidate-opening-evidence s out trace-roots
  trace-bs trace-final fri-dg opening-composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr f-fri-roots f-final as dg
  composition-fri-roots final trace-query-idxs trace-openings
  composition-query-idxs composition-openings trace-table
  composition-table
and chain:
  generic-fri-sampled-layer-chain-evidence
  (composition-table-low-degree (to-nat fri-dg)) composition-table
  (to-nat fri-dg) opening-composition-roots composition-bs
  composition-final fri-query-idxs composition-round-layers doms layers
and generic-domain:
  generic-fri-full-cover-domain-failure opening-composition-roots
  composition-bs doms
unfolding composition-fri-sampled-full-cover-domain-failure-def
by meson
have partial:
  generic-fri-partial-evidence
  (composition-table-low-degree (to-nat fri-dg)) composition-table
  (to-nat fri-dg) opening-composition-roots composition-bs
  composition-final fri-query-idxs composition-round-layers
by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
have roots-len: length composition-bs = length opening-composition-roots
by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
have split:
  generic-fri-full-cover-index-failure opening-composition-roots
  fri-query-idxs ∨
  generic-fri-full-cover-sampled-domain-failure
  opening-composition-roots fri-query-idxs doms
by (rule generic-fri-full-cover-domain-failure-imp-index-or-sampled-domain
  [OF roots-len generic-domain])
then show ?thesis
proof
assume
  generic-fri-full-cover-index-failure opening-composition-roots
  fri-query-idxs
then show ?thesis
unfolding composition-fri-sampled-full-cover-index-failure-def
using evidence chain by meson
next
assume
  generic-fri-full-cover-sampled-domain-failure
  opening-composition-roots fri-query-idxs doms
then show ?thesis

```


unfolding *composition-fri-sampled-full-cover-sampled-domain-failure-def*
using *evidence chain* **by** *meson*
qed
qed

lemma *wp-trace-fri-sampled-full-cover-domain-bound-from-index-or-sampled-domain:*
assumes *index:*
wp-event verify-monad
(trace-fri-sampled-full-cover-index-failure s) s ≤ I
and *sampled-domain:*
wp-event verify-monad
(trace-fri-sampled-full-cover-sampled-domain-failure s) s ≤ SD
shows
wp-event verify-monad
(trace-fri-sampled-full-cover-domain-failure s) s ≤ I + SD
proof –
have *wp-event verify-monad*
(trace-fri-sampled-full-cover-domain-failure s) s
 $\leq$ *wp-event verify-monad*
(λout. trace-fri-sampled-full-cover-index-failure s out ∨
trace-fri-sampled-full-cover-sampled-domain-failure s out) s
by *(rule wp-event-mono)*
(rule trace-fri-sampled-full-cover-domain-imp-index-or-sampled-domain)
also have $\dots \leq I + SD$
by *(rule order-trans[OF wp-event-union-bound*
add-mono[OF index sampled-domain]])
finally show *?thesis* .
qed

lemma
wp-composition-fri-sampled-full-cover-domain-bound-from-index-or-sampled-domain:
assumes *index:*
wp-event verify-monad
(composition-fri-sampled-full-cover-index-failure s) s ≤ I
and *sampled-domain:*
wp-event verify-monad
(composition-fri-sampled-full-cover-sampled-domain-failure s) s
 $\leq SD$
shows
wp-event verify-monad
(composition-fri-sampled-full-cover-domain-failure s) s ≤ I + SD
proof –
have *wp-event verify-monad*
(composition-fri-sampled-full-cover-domain-failure s) s
 $\leq$ *wp-event verify-monad*
(λout. composition-fri-sampled-full-cover-index-failure s out ∨
composition-fri-sampled-full-cover-sampled-domain-failure s out) s
by *(rule wp-event-mono)*
(rule

$\text{composition-fri-sampled-full-cover-domain-imp-index-or-sampled-domain})$
also have $\dots \leq I + SD$
by (rule order-trans[*OF* wp-event-union-bound
add-mono[*OF* index sampled-domain]])
finally show ?thesis .
qed

lemma wp-trace-fri-sampled-missing-full-cover-bound-from-domain-split:
assumes proximity:
wp-event verify-monad
(trace-fri-sampled-full-cover-proximity-failure s bad) $s \leq P$
and index:
wp-event verify-monad
(trace-fri-sampled-full-cover-index-failure s) $s \leq I$
and domain-length:
wp-event verify-monad
(trace-fri-sampled-full-cover-domain-length-failure s) $s \leq DL$
and sampled-domain:
wp-event verify-monad
(trace-fri-sampled-full-cover-sampled-domain-failure s) $s \leq SD$
shows
wp-event verify-monad
(trace-fri-sampled-missing-full-cover s bad) s
 $\leq P + I + (I + DL) + (I + SD)$
by (rule wp-trace-fri-sampled-missing-full-cover-bound-from-length-split
[*OF* proximity index domain-length
wp-trace-fri-sampled-full-cover-domain-bound-from-index-or-sampled-domain])
(rule index, rule sampled-domain)

lemma wp-composition-fri-sampled-missing-full-cover-bound-from-domain-split:
assumes proximity:
wp-event verify-monad
(composition-fri-sampled-full-cover-proximity-failure s bad) $s \leq P$
and index:
wp-event verify-monad
(composition-fri-sampled-full-cover-index-failure s) $s \leq I$
and domain-length:
wp-event verify-monad
(composition-fri-sampled-full-cover-domain-length-failure s) $s \leq DL$
and sampled-domain:
wp-event verify-monad
(composition-fri-sampled-full-cover-sampled-domain-failure s) s
 $\leq SD$
shows
wp-event verify-monad
(composition-fri-sampled-missing-full-cover s bad) s
 $\leq P + I + (I + DL) + (I + SD)$
by (rule
wp-composition-fri-sampled-missing-full-cover-bound-from-length-split

[*OF proximity index domain-length*
wp-composition-fri-sampled-full-cover-domain-bound-from-index-or-sampled-domain])
(*rule index, rule sampled-domain*)

end

end

theory *Soundness-FRI-Sampled-Obstructions*
imports *Soundness-FRI-Full-Cover-Domains*
begin

Explicit obstruction events for the branch where an accepting verifier execution reaches the broad reachable FRI bad event, but no sampled FRI layer-chain witness exists.

These predicates are proof-routing devices only. They do not reconstruct complete FRI tables from sampled openings; they record that the verifier's partial FRI evidence cannot be organized into the sampled layer-chain object required by the generic low-degree reduction.

context *soundness*
begin

definition *generic-fri-sampled-next-value-conflict*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow \text{bool}$

where

generic-fri-sampled-next-value-conflict roots challenges query-idxs
round-layers $\longleftrightarrow$
 $(\exists \text{ round-idx round-idx' layer-idx } v \ v'.$
 $\text{round-idx} < \text{length query-idxs} \wedge$
 $\text{round-idx'} < \text{length query-idxs} \wedge$
 $\text{layer-idx} < \text{length challenges} \wedge$
 $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} =$
 $\text{fri-evidence-next-idx roots query-idxs round-idx' layer-idx} \wedge$
 $\text{generic-fri-round-forced-next-value roots challenges query-idxs}$
 $\text{round-layers round-idx layer-idx } v \wedge$
 $\text{generic-fri-round-forced-next-value roots challenges query-idxs}$
 $\text{round-layers round-idx' layer-idx } v' \wedge$
 $v \neq v')$

definition *generic-fri-sampled-final-value-conflict*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow \text{bool}$

where

generic-fri-sampled-final-value-conflict roots challenges final-value
query-idxs round-layers $\longleftrightarrow$
 $(\exists \text{ round-idx } v.$
 $0 < \text{length challenges} \wedge$

$round_idx < length\ query_idxs \wedge$
 $generic_fri_round_forced_next_value\ roots\ challenges\ query_idxs$
 $round_layers\ round_idx\ (length\ challenges - 1)\ v \wedge$
 $v \neq final_value)$

definition *generic-fri-sampled-base-opening-conflict*

$:: 'f\ list \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow nat\ list \Rightarrow$
 $'f\ list\ list\ list \Rightarrow bool$

where

$generic_fri_sampled_base_opening_conflict\ candidate_table\ roots\ challenges$
 $query_idxs\ round_layers \longleftrightarrow$
 $(\exists\ round_idx\ xp\ xp_path\ xn\ xn_path.$
 $0 < length\ challenges \wedge$
 $round_idx < length\ query_idxs \wedge$
 $fri_layer_step_evidence$
 $(roots\ !\ 0)$
 $(challenges\ !\ 0)$
 $(fri_evidence_layer_len\ roots\ 0)$
 $(fri_evidence_layer_idx\ roots\ query_idxs\ round_idx\ 0)$
 1
 $(fri_sibling_index\ (fri_evidence_layer_len\ roots\ 0)$
 $(fri_evidence_layer_idx\ roots\ query_idxs\ round_idx\ 0)))$
 $xp\ xp_path\ xn\ xn_path$
 $(fri_evidence_next_idx\ roots\ query_idxs\ round_idx\ 0)$
 $(fri_evidence_next_value\ roots\ challenges\ query_idxs\ round_idx\ 0$
 $xp\ xn)$
 $(round_layers\ !\ round_idx\ !\ 0) \wedge$
 $\neg\ fri_opening_matches_table\ (fri_evidence_layer_len\ roots\ 0)$
 $(fri_evidence_layer_idx\ roots\ query_idxs\ round_idx\ 0)$
 $candidate_table\ xp\ xn)$

definition *generic-fri-sampled-successor-opening-conflict*

$:: 'f\ list \Rightarrow 'f\ list \Rightarrow nat\ list \Rightarrow$
 $'f\ list\ list\ list \Rightarrow bool$

where

$generic_fri_sampled_successor_opening_conflict\ roots\ challenges\ query_idxs$
 $round_layers \longleftrightarrow$
 $(\exists\ source_round\ target_round\ layer_idx\ v\ xp\ xp_path\ xn\ xn_path.$
 $source_round < length\ query_idxs \wedge$
 $target_round < length\ query_idxs \wedge$
 $Suc\ layer_idx < length\ challenges \wedge$
 $generic_fri_round_forced_next_value\ roots\ challenges\ query_idxs$
 $round_layers\ source_round\ layer_idx\ v \wedge$
 $fri_layer_step_evidence$
 $(roots\ !\ Suc\ layer_idx)$
 $(challenges\ !\ Suc\ layer_idx)$
 $(fri_evidence_layer_len\ roots\ (Suc\ layer_idx))$
 $(fri_evidence_layer_idx\ roots\ query_idxs\ target_round$
 $(Suc\ layer_idx))$

$(2 \wedge \text{Suc layer-idx})$
 $(\text{fri-sibling-index } (\text{fri-evidence-layer-len roots } (\text{Suc layer-idx}))$
 $(\text{fri-evidence-layer-idx roots query-idxs target-round}$
 $(\text{Suc layer-idx})))$
 $xp \text{ xp-path } xn \text{ xn-path}$
 $(\text{fri-evidence-next-idx roots query-idxs target-round}$
 $(\text{Suc layer-idx}))$
 $(\text{fri-evidence-next-value roots challenges query-idxs target-round}$
 $(\text{Suc layer-idx}) \text{ xp } xn)$
 $(\text{round-layers ! target-round ! Suc layer-idx}) \wedge$
 $((\text{fri-evidence-next-idx roots query-idxs source-round layer-idx} =$
 $\text{fri-evidence-layer-idx roots query-idxs target-round}$
 $(\text{Suc layer-idx}) \wedge$
 $v \neq xp) \vee$
 $(\text{fri-evidence-next-idx roots query-idxs source-round layer-idx} =$
 $\text{fri-sibling-index } (\text{fri-evidence-layer-len roots } (\text{Suc layer-idx}))$
 $(\text{fri-evidence-layer-idx roots query-idxs target-round}$
 $(\text{Suc layer-idx})) \wedge$
 $v \neq xn)))$

definition *generic-fri-sampled-same-layer-opening-conflict*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow \text{bool}$

where

generic-fri-sampled-same-layer-opening-conflict roots challenges
 $\text{query-idxs round-layers} \longleftrightarrow$
 $(\exists \text{ round-idx round-idx' layer-idx}$
 $xp \text{ xp-path } xn \text{ xn-path } yp \text{ yp-path } yn \text{ yn-path.}$
 $\text{round-idx} < \text{length query-idxs} \wedge$
 $\text{round-idx'} < \text{length query-idxs} \wedge$
 $\text{layer-idx} < \text{length challenges} \wedge$
fri-layer-step-evidence
 $(\text{roots ! layer-idx})$
 $(\text{challenges ! layer-idx})$
 $(\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx})$
 $(2 \wedge \text{layer-idx})$
 $(\text{fri-sibling-index } (\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}))$
 $xp \text{ xp-path } xn \text{ xn-path}$
 $(\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx})$
 $(\text{fri-evidence-next-value roots challenges query-idxs round-idx}$
 $\text{layer-idx } xp \text{ xn})$
 $(\text{round-layers ! round-idx ! layer-idx}) \wedge$
fri-layer-step-evidence
 $(\text{roots ! layer-idx})$
 $(\text{challenges ! layer-idx})$
 $(\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx})$

$(2 \wedge \text{layer-idx})$
 $(\text{fri-sibling-index } (\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx}))$
 $\text{yp yp-path yn yn-path}$
 $(\text{fri-evidence-next-idx roots query-idxs round-idx' layer-idx})$
 $(\text{fri-evidence-next-value roots challenges query-idxs round-idx'}$
 $\text{layer-idx yp yn})$
 $(\text{round-layers ! round-idx' ! layer-idx}) \wedge$
 $((\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx} =$
 $\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx} \wedge$
 $\text{xp} \neq \text{yp}) \vee$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx} =$
 $\text{fri-sibling-index } (\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx}) \wedge$
 $\text{xp} \neq \text{yn}) \vee$
 $(\text{fri-sibling-index } (\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) =$
 $\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx} \wedge$
 $\text{xn} \neq \text{yp}) \vee$
 $(\text{fri-sibling-index } (\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) =$
 $\text{fri-sibling-index } (\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx}) \wedge$
 $\text{xn} \neq \text{yn})))$

lemma *generic-fri-sampled-next-value-conflictE:*

assumes

generic-fri-sampled-next-value-conflict roots challenges query-idxs
round-layers

obtains *round-idx round-idx' layer-idx v v'*

where

round-idx < length query-idxs
round-idx' < length query-idxs
layer-idx < length challenges
fri-evidence-next-idx roots query-idxs round-idx layer-idx =
fri-evidence-next-idx roots query-idxs round-idx' layer-idx
generic-fri-round-forced-next-value roots challenges query-idxs
round-layers round-idx layer-idx v
generic-fri-round-forced-next-value roots challenges query-idxs
round-layers round-idx' layer-idx v'
v ≠ v'

using *assms unfolding generic-fri-sampled-next-value-conflict-def*
by *blast*

lemma *generic-fri-sampled-final-value-conflictE:*

assumes

generic-fri-sampled-final-value-conflict roots challenges final-value
query-idxs round-layers

obtains *round-idx v*

where

$0 < \text{length challenges}$

$\text{round-idx} < \text{length query-idxs}$

$\text{generic-fri-round-forced-next-value roots challenges query-idxs}$

$\text{round-layers round-idx (length challenges - 1) v}$

$v \neq \text{final-value}$

using *assms* **unfolding** *generic-fri-sampled-final-value-conflict-def*

by *blast*

lemma *generic-fri-sampled-successor-opening-conflictE*:

assumes

generic-fri-sampled-successor-opening-conflict roots challenges

query-idxs round-layers

obtains *source-round target-round layer-idx v xp xp-path xn xn-path*

where

$\text{source-round} < \text{length query-idxs}$

$\text{target-round} < \text{length query-idxs}$

$\text{Suc layer-idx} < \text{length challenges}$

$\text{generic-fri-round-forced-next-value roots challenges query-idxs}$

$\text{round-layers source-round layer-idx v}$

fri-layer-step-evidence

$(\text{roots} \text{ ! } \text{Suc layer-idx})$

$(\text{challenges} \text{ ! } \text{Suc layer-idx})$

$(\text{fri-evidence-layer-len roots (Suc layer-idx)})$

$(\text{fri-evidence-layer-idx roots query-idxs target-round}$

$(\text{Suc layer-idx}))$

$(2 \wedge \text{Suc layer-idx})$

$(\text{fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx)})$

$(\text{fri-evidence-layer-idx roots query-idxs target-round}$

$(\text{Suc layer-idx})))$

xp xp-path xn xn-path

$(\text{fri-evidence-next-idx roots query-idxs target-round}$

$(\text{Suc layer-idx}))$

$(\text{fri-evidence-next-value roots challenges query-idxs target-round}$

$(\text{Suc layer-idx}) \text{ xp xn})$

$(\text{round-layers} \text{ ! } \text{target-round} \text{ ! } \text{Suc layer-idx})$

$((\text{fri-evidence-next-idx roots query-idxs source-round layer-idx} =$

$\text{fri-evidence-layer-idx roots query-idxs target-round}$

$(\text{Suc layer-idx}) \wedge$

$v \neq \text{xp}) \vee$

$(\text{fri-evidence-next-idx roots query-idxs source-round layer-idx} =$

$\text{fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx)})$

$(\text{fri-evidence-layer-idx roots query-idxs target-round}$

$(\text{Suc layer-idx})) \wedge$

$v \neq \text{xn}))$

using *assms* **unfolding** *generic-fri-sampled-successor-opening-conflict-def*

by *blast*

definition *generic-fri-sampled-layer-assignment-obstruction*

$$\begin{aligned} &:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow \\ &\quad 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow \\ &\quad 'f \text{ list list list} \Rightarrow \text{bool} \end{aligned}$$

where

generic-fri-sampled-layer-assignment-obstruction *low-degree*
candidate-table degree-bound roots challenges final-value query-idxs
round-layers $\longleftrightarrow$
generic-fri-partial-evidence *low-degree candidate-table degree-bound*
roots challenges final-value query-idxs round-layers $\wedge$
 $(\forall \text{ doms layers. } \neg$
generic-fri-sampled-layer-chain-evidence *low-degree candidate-table*
degree-bound roots challenges final-value query-idxs round-layers
doms layers)

definition *generic-fri-sampled-assignment-conflict*

$$\begin{aligned} &:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \\ &\quad \text{nat list} \Rightarrow 'f \text{ list list list} \Rightarrow \text{bool} \end{aligned}$$

where

generic-fri-sampled-assignment-conflict *candidate-table roots challenges*
final-value query-idxs round-layers $\longleftrightarrow$
generic-fri-sampled-base-opening-conflict *candidate-table roots*
challenges query-idxs round-layers $\vee$
generic-fri-sampled-next-value-conflict *roots challenges query-idxs*
round-layers $\vee$
generic-fri-sampled-successor-opening-conflict *roots challenges*
query-idxs round-layers $\vee$
generic-fri-sampled-same-layer-opening-conflict *roots challenges*
query-idxs round-layers $\vee$
generic-fri-sampled-final-value-conflict *roots challenges final-value*
query-idxs round-layers

definition *generic-fri-sampled-layer-value-constraint*

$$\begin{aligned} &:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \\ &\quad \text{nat list} \Rightarrow 'f \text{ list list list} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow \\ &\quad 'f \Rightarrow \text{bool} \end{aligned}$$

where

generic-fri-sampled-layer-value-constraint *candidate-table roots*
challenges final-value query-idxs round-layers layer-idx idx v
 $\longleftrightarrow$
(if *layer-idx* $= 0$ *then*
 $\text{idx} < \text{length candidate-table} \wedge v = \text{candidate-table} ! \text{idx}$
else if *layer-idx* $= \text{length challenges}$ *then*
 $v = \text{final-value}$
else
 $(\exists \text{ round-idx.}$
 $\text{round-idx} < \text{length query-idxs} \wedge$
generic-fri-round-forced-next-value *roots challenges query-idxs*
round-layers round-idx (layer-idx - 1) v $\wedge$
fri-evidence-next-idx *roots query-idxs round-idx*

$(\text{layer-idx} - 1) = \text{idx}) \vee$
 $(\exists \text{round-idx } xp \text{ } xp\text{-path } xn \text{ } xn\text{-path}.$
 $\text{round-idx} < \text{length } \text{query-idxs} \wedge$
 $\text{layer-idx} < \text{length } \text{challenges} \wedge$
 $\text{fri-layer-step-evidence}$
 $(\text{roots} ! \text{layer-idx})$
 $(\text{challenges} ! \text{layer-idx})$
 $(\text{fri-evidence-layer-len } \text{roots } \text{layer-idx})$
 $(\text{fri-evidence-layer-idx } \text{roots } \text{query-idxs } \text{round-idx } \text{layer-idx})$
 $(2 \wedge \text{layer-idx})$
 $(\text{fri-sibling-index } (\text{fri-evidence-layer-len } \text{roots } \text{layer-idx})$
 $(\text{fri-evidence-layer-idx } \text{roots } \text{query-idxs } \text{round-idx } \text{layer-idx}))$
 $xp \text{ } xp\text{-path } xn \text{ } xn\text{-path}$
 $(\text{fri-evidence-next-idx } \text{roots } \text{query-idxs } \text{round-idx } \text{layer-idx})$
 $(\text{fri-evidence-next-value } \text{roots } \text{challenges } \text{query-idxs } \text{round-idx}$
 $\text{layer-idx } xp \text{ } xn)$
 $(\text{round-layers} ! \text{round-idx} ! \text{layer-idx}) \wedge$
 $((\text{idx} = \text{fri-evidence-layer-idx } \text{roots } \text{query-idxs } \text{round-idx}$
 $\text{layer-idx} \wedge v = xp) \vee$
 $(\text{idx} = \text{fri-sibling-index } (\text{fri-evidence-layer-len } \text{roots } \text{layer-idx})$
 $(\text{fri-evidence-layer-idx } \text{roots } \text{query-idxs } \text{round-idx } \text{layer-idx}) \wedge$
 $v = xn))))$

definition *generic-fri-sampled-layer-value*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $\text{nat list} \Rightarrow 'f \text{ list list list} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow 'f$

where

$\text{generic-fri-sampled-layer-value candidate-table roots challenges}$
 $\text{final-value query-idxs round-layers layer-idx idx} =$
 $(\text{if } \exists v. \text{generic-fri-sampled-layer-value-constraint candidate-table}$
 $\text{roots challenges final-value query-idxs round-layers layer-idx idx } v$
 $\text{then SOME } v. \text{generic-fri-sampled-layer-value-constraint}$
 $\text{candidate-table roots challenges final-value query-idxs round-layers}$
 $\text{layer-idx idx } v$
 $\text{else final-value})$

definition *generic-fri-sampled-assignment-layer*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $\text{nat list} \Rightarrow 'f \text{ list list list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list}$

where

$\text{generic-fri-sampled-assignment-layer candidate-table roots challenges}$
 $\text{final-value query-idxs round-layers layer-idx} =$
 $(\text{if layer-idx} = 0 \text{ then candidate-table}$
 $\text{else map } (\text{generic-fri-sampled-layer-value candidate-table roots}$
 $\text{challenges final-value query-idxs round-layers layer-idx})$
 $[0..<\text{clength} * \text{scale}])$

definition *generic-fri-sampled-assignment-layers*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$

```

    nat list  $\Rightarrow$  'f list list list  $\Rightarrow$  'f list list
where
  generic-fri-sampled-assignment-layers candidate-table roots challenges
    final-value query-idxs round-layers =
  map (generic-fri-sampled-assignment-layer candidate-table roots
    challenges final-value query-idxs round-layers)
    [0.. $\text{Suc}$  (length challenges)]

lemma generic-fri-sampled-assignment-layer-length-nonzero[simp]:
  assumes layer-idx  $\neq$  0
  shows length (generic-fri-sampled-assignment-layer candidate-table roots
    challenges final-value query-idxs round-layers layer-idx) =
    clength * scale
  using assms unfolding generic-fri-sampled-assignment-layer-def by simp

lemma generic-fri-sampled-assignment-layer-zero[simp]:
  generic-fri-sampled-assignment-layer candidate-table roots challenges
    final-value query-idxs round-layers 0 = candidate-table
  unfolding generic-fri-sampled-assignment-layer-def by simp

lemma generic-fri-sampled-assignment-layers-length[simp]:
  length (generic-fri-sampled-assignment-layers candidate-table roots
    challenges final-value query-idxs round-layers) =
     $\text{Suc}$  (length challenges)
  unfolding generic-fri-sampled-assignment-layers-def by simp

lemma generic-fri-sampled-assignment-layers-nth:
  assumes layer-idx  $\leq$  length challenges
  shows
    generic-fri-sampled-assignment-layers candidate-table roots challenges
      final-value query-idxs round-layers ! layer-idx =
      generic-fri-sampled-assignment-layer candidate-table roots challenges
        final-value query-idxs round-layers layer-idx
  proof (cases layer-idx < length challenges)
  case True
  then show ?thesis
    unfolding generic-fri-sampled-assignment-layers-def
    by (simp add: nth-append)
  next
  case False
  then have layer-eq: layer-idx = length challenges
    using assms by simp
  then show ?thesis
    unfolding generic-fri-sampled-assignment-layers-def
    by (simp add: nth-append)
qed

lemma generic-fri-sampled-assignment-layers-base[simp]:
  generic-fri-sampled-assignment-layers candidate-table roots challenges

```

```

    final-value query-idxs round-layers ! 0 = candidate-table
using generic-fri-sampled-assignment-layers-nth[of 0 challenges
    candidate-table roots final-value query-idxs round-layers]
by simp

lemma generic-fri-sampled-layer-value-unique:
assumes constraint:
    generic-fri-sampled-layer-value-constraint candidate-table roots
    challenges final-value query-idxs round-layers layer-idx idx v
and unique:
     $\bigwedge v'. \text{generic-fri-sampled-layer-value-constraint candidate-table}$ 
     $\text{roots challenges final-value query-idxs round-layers layer-idx idx } v'$ 
     $\implies v' = v$ 
shows
    generic-fri-sampled-layer-value candidate-table roots challenges
    final-value query-idxs round-layers layer-idx idx = v
proof -
have ex:  $\exists v. \text{generic-fri-sampled-layer-value-constraint}$ 
    candidate-table roots challenges final-value query-idxs round-layers
    layer-idx idx v
using constraint by blast
have some-eq:
    (SOME v. generic-fri-sampled-layer-value-constraint candidate-table
    roots challenges final-value query-idxs round-layers layer-idx idx v) =
    v
by (rule some-equality) (use constraint unique in blast)+
show ?thesis
unfolding generic-fri-sampled-layer-value-def
using ex some-eq by simp
qed

lemma generic-fri-sampled-assignment-layer-nth-unique:
assumes idx-bound:  $\text{idx} < \text{clength} * \text{scale}$ 
and layer-nonzero:  $\text{layer-idx} \neq 0$ 
and constraint:
    generic-fri-sampled-layer-value-constraint candidate-table roots
    challenges final-value query-idxs round-layers layer-idx idx v
and unique:
     $\bigwedge v'. \text{generic-fri-sampled-layer-value-constraint candidate-table}$ 
     $\text{roots challenges final-value query-idxs round-layers layer-idx idx } v'$ 
     $\implies v' = v$ 
shows
    generic-fri-sampled-assignment-layer candidate-table roots challenges
    final-value query-idxs round-layers layer-idx ! idx = v
unfolding generic-fri-sampled-assignment-layer-def
using idx-bound layer-nonzero
by (simp add: generic-fri-sampled-layer-value-unique[OF constraint unique])

lemma generic-fri-sampled-final-assignment-layer-consistent:

```

```

assumes challenges-nonempty:  $0 < \text{length } \text{challenges}$ 
shows
  fri-final-constant-consistent
    (generic-fri-sampled-assignment-layers candidate-table roots challenges
      final-value query-idxs round-layers ! length challenges)
    final-value
proof –
  let ?layer =
    generic-fri-sampled-assignment-layers candidate-table roots challenges
    final-value query-idxs round-layers ! length challenges
have layer-eq:
  ?layer =
    generic-fri-sampled-assignment-layer candidate-table roots challenges
    final-value query-idxs round-layers (length challenges)
  by (rule generic-fri-sampled-assignment-layers-nth) simp
have layer-len: length ?layer = clength * scale
  using challenges-nonempty layer-eq by simp
have nth-final:
   $\bigwedge \text{idx}. \text{idx} < \text{length } ?\text{layer} \implies ?\text{layer} ! \text{idx} = \text{final-value}$ 
proof –
  fix idx
  assume idx-bound-layer:  $\text{idx} < \text{length } ?\text{layer}$ 
  then have idx-bound:  $\text{idx} < \text{clength} * \text{scale}$ 
  using layer-len by simp
  have constraint:
    generic-fri-sampled-layer-value-constraint candidate-table roots
    challenges final-value query-idxs round-layers
    (length challenges) idx final-value
  using challenges-nonempty
  unfolding generic-fri-sampled-layer-value-constraint-def by simp
  have unique:
     $\bigwedge v'. \text{generic-fri-sampled-layer-value-constraint candidate-table}$ 
    roots challenges final-value query-idxs round-layers
    (length challenges) idx  $v' \implies v' = \text{final-value}$ 
  using challenges-nonempty
  unfolding generic-fri-sampled-layer-value-constraint-def by simp
  show ?layer ! idx = final-value
  unfolding layer-eq
  by (rule generic-fri-sampled-assignment-layer-nth-unique
    [OF idx-bound - constraint unique])
    (use challenges-nonempty in simp)
qed
show ?thesis
  unfolding fri-final-constant-consistent-def
  using nth-final by (auto simp: in-set-conv-nth)
qed

definition trace-fri-sampled-layer-assignment-obstruction
  :: (f, a) protocol-channel-scheme  $\implies$ 

```

$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-sampled-layer-assignment-obstruction } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 trace-table.
 $\text{trace-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
 $\text{generic-fri-sampled-layer-assignment-obstruction}$
 $\text{trace-table-low-degree trace-table (clength - 1) trace-roots}$
 $\text{trace-bs trace-final fri-query-idxs trace-round-layers})$

definition $\text{composition-fri-sampled-layer-assignment-obstruction}$
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where
 $\text{composition-fri-sampled-layer-assignment-obstruction } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table.}$
 $\text{composition-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings trace-table}$
 $\text{composition-table} \wedge$
 $\text{generic-fri-sampled-layer-assignment-obstruction}$
 $(\text{composition-table-low-degree (to-nat fri-dg)}) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs composition-round-layers})$

definition $\text{trace-fri-sampled-assignment-conflict}$
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where
 $\text{trace-fri-sampled-assignment-conflict } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 trace-table.

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table $\wedge$
generic-fri-sampled-assignment-conflict trace-table trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers)

definition *composition-fri-sampled-assignment-conflict*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-assignment-conflict s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table.
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-sampled-assignment-conflict composition-table
opening-composition-roots composition-bs composition-final
fri-query-idxs composition-round-layers)

lemma *generic-fri-sampled-layer-assignment-obstructionD:*

assumes

generic-fri-sampled-layer-assignment-obstruction low-degree
candidate-table degree-bound roots challenges final-value query-idxs
round-layers

shows

generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
 $\wedge \text{doms layers. } \neg$
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers

using *assms*

unfolding *generic-fri-sampled-layer-assignment-obstruction-def*

by *simp-all*

lemma *trace-fri-sampled-layer-assignment-obstructionE:*

assumes

trace-fri-sampled-layer-assignment-obstruction s out

obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
where
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
generic-fri-sampled-layer-assignment-obstruction
trace-table-low-degree trace-table (clength - 1) trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers
using *assms*
unfolding *trace-fri-sampled-layer-assignment-obstruction-def*
apply (*elim exE conjE*)
apply (*rule that*)
apply *assumption*
apply *assumption*
done

lemma *composition-fri-sampled-layer-assignment-obstructionE:*
assumes
composition-fri-sampled-layer-assignment-obstruction s out
obtains *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table
where
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
generic-fri-sampled-layer-assignment-obstruction
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
using *assms*
unfolding *composition-fri-sampled-layer-assignment-obstruction-def*
apply (*elim exE conjE*)
apply (*rule that*)
apply *assumption*
apply *assumption*
done

lemma *generic-fri-no-next-value-conflict-unique*:
assumes *no-conflict*:
 $\neg$ *generic-fri-sampled-next-value-conflict* *roots* *challenges*
query-idxs *round-layers*
and *round-bound*: *round-idx* < *length query-idxs*
and *round-bound'*: *round-idx'* < *length query-idxs*
and *layer-bound*: *layer-idx* < *length challenges*
and *same-idx*:
fri-evidence-next-idx roots query-idxs round-idx layer-idx =
fri-evidence-next-idx roots query-idxs round-idx' layer-idx
and *forced*:
generic-fri-round-forced-next-value *roots challenges query-idxs*
round-layers round-idx layer-idx v
and *forced'*:
generic-fri-round-forced-next-value *roots challenges query-idxs*
round-layers round-idx' layer-idx v'
shows *v* = *v'*
using *assms*
unfolding *generic-fri-sampled-next-value-conflict-def*
by *blast*

lemma *generic-fri-no-final-value-conflict-forced-eq*:
assumes *no-conflict*:
 $\neg$ *generic-fri-sampled-final-value-conflict* *roots challenges*
final-value query-idxs round-layers
and *challenges-nonempty*: 0 < *length challenges*
and *round-bound*: *round-idx* < *length query-idxs*
and *forced*:
generic-fri-round-forced-next-value *roots challenges query-idxs*
round-layers round-idx (length challenges - 1) v
shows *v* = *final-value*
using *assms*
unfolding *generic-fri-sampled-final-value-conflict-def*
by *blast*

lemma *generic-fri-no-assignment-conflict-no-final*:
assumes
 $\neg$ *generic-fri-sampled-assignment-conflict* *candidate-table roots*
challenges final-value query-idxs round-layers
shows
 $\neg$ *generic-fri-sampled-final-value-conflict* *roots challenges*
final-value query-idxs round-layers
using *assms* **unfolding** *generic-fri-sampled-assignment-conflict-def* **by** *simp*

lemma *generic-fri-no-assignment-conflict-no-next*:
assumes
 $\neg$ *generic-fri-sampled-assignment-conflict* *candidate-table roots*
challenges final-value query-idxs round-layers

shows

$\neg$ *generic-fri-sampled-next-value-conflict* roots challenges
query-idxs round-layers

using *assms* **unfolding** *generic-fri-sampled-assignment-conflict-def* **by** *simp*

lemma *generic-fri-sampled-final-layer-forced-value*:

assumes *no-conflict*:

$\neg$ *generic-fri-sampled-assignment-conflict* candidate-table roots
challenges final-value query-idxs round-layers

and *challenges-nonempty*: $0 < \text{length } \text{challenges}$

and *round-bound*: $\text{round-idx} < \text{length } \text{query-idxs}$

and *idx-bound*:

fri-evidence-next-idx roots query-idxs round-idx
 $(\text{length } \text{challenges} - 1) < \text{clength} * \text{scale}$

and *forced*:

generic-fri-round-forced-next-value roots challenges query-idxs
round-layers round-idx $(\text{length } \text{challenges} - 1) v$

shows

generic-fri-sampled-assignment-layers candidate-table roots challenges
final-value query-idxs round-layers ! $\text{length } \text{challenges}$!

fri-evidence-next-idx roots query-idxs round-idx
 $(\text{length } \text{challenges} - 1) = v$

proof –

have *v-final*: $v = \text{final-value}$

by (rule *generic-fri-no-final-value-conflict-forced-eq*
[*OF generic-fri-no-assignment-conflict-no-final*[*OF no-conflict*]
challenges-nonempty round-bound forced])

let *?idx* =

fri-evidence-next-idx roots query-idxs round-idx
 $(\text{length } \text{challenges} - 1)$

have *layer-eq*:

generic-fri-sampled-assignment-layers candidate-table roots challenges
final-value query-idxs round-layers ! $\text{length } \text{challenges} =$
generic-fri-sampled-assignment-layer candidate-table roots challenges
final-value query-idxs round-layers $(\text{length } \text{challenges})$

by (rule *generic-fri-sampled-assignment-layers-nth*) *simp*

have *constraint*:

generic-fri-sampled-layer-value-constraint candidate-table roots
challenges final-value query-idxs round-layers
 $(\text{length } \text{challenges}) \text{ ?idx } \text{final-value}$

using *challenges-nonempty*

unfolding *generic-fri-sampled-layer-value-constraint-def* **by** *simp*

have *unique*:

$\bigwedge v'. \text{generic-fri-sampled-layer-value-constraint candidate-table roots}$
challenges final-value query-idxs round-layers
 $(\text{length } \text{challenges}) \text{ ?idx } v' \implies v' = \text{final-value}$

using *challenges-nonempty*

unfolding *generic-fri-sampled-layer-value-constraint-def* **by** *simp*

have *nth-final*:

```

generic-fri-sampled-assignment-layer candidate-table roots challenges
  final-value query-idxs round-layers (length challenges) ! ?idx =
  final-value
by (rule generic-fri-sampled-assignment-layer-nth-unique
  [OF idx-bound - constraint unique])
  (use challenges-nonempty in simp)
show ?thesis
  using nth-final v-final unfolding layer-eq by simp
qed

lemma generic-fri-no-base-opening-conflict-matches-step:
  assumes no-conflict:
     $\neg$  generic-fri-sampled-base-opening-conflict candidate-table roots
      challenges query-idxs round-layers
  and challenges-nonempty:  $0 < \text{length challenges}$ 
  and round-bound:  $\text{round-idx} < \text{length query-idxs}$ 
  and step:
    fri-layer-step-evidence
      (roots ! 0)
      (challenges ! 0)
      (fri-evidence-layer-len roots 0)
      (fri-evidence-layer-idx roots query-idxs round-idx 0)
      1
      (fri-sibling-index (fri-evidence-layer-len roots 0)
        (fri-evidence-layer-idx roots query-idxs round-idx 0))
      xp xp-path xn xn-path
      (fri-evidence-next-idx roots query-idxs round-idx 0)
      (fri-evidence-next-value roots challenges query-idxs round-idx 0
        xp xn)
      (round-layers ! round-idx ! 0)
  shows
    fri-opening-matches-table (fri-evidence-layer-len roots 0)
      (fri-evidence-layer-idx roots query-idxs round-idx 0)
      candidate-table xp xn
  using assms
  unfolding generic-fri-sampled-base-opening-conflict-def
  by blast

lemma generic-fri-no-assignment-conflict-no-base:
  assumes
     $\neg$  generic-fri-sampled-assignment-conflict candidate-table roots
      challenges final-value query-idxs round-layers
  shows
     $\neg$  generic-fri-sampled-base-opening-conflict candidate-table roots
      challenges query-idxs round-layers
  using assms unfolding generic-fri-sampled-assignment-conflict-def by simp

lemma generic-fri-sampled-assignment-base-matches-step:
  assumes no-conflict:

```

```

    ¬ generic-fri-sampled-assignment-conflict candidate-table roots
      challenges final-value query-idxs round-layers
and challenges-nonempty: 0 < length challenges
and round-bound: round-idx < length query-idxs
and step:
  fri-layer-step-evidence
    (roots ! 0)
    (challenges ! 0)
    (fri-evidence-layer-len roots 0)
    (fri-evidence-layer-idx roots query-idxs round-idx 0)
    1
    (fri-sibling-index (fri-evidence-layer-len roots 0)
      (fri-evidence-layer-idx roots query-idxs round-idx 0))
    xp xp-path xn xn-path
    (fri-evidence-next-idx roots query-idxs round-idx 0)
    (fri-evidence-next-value roots challenges query-idxs round-idx 0
      xp xn)
    (round-layers ! round-idx ! 0)
shows
  fri-opening-matches-table (fri-evidence-layer-len roots 0)
    (fri-evidence-layer-idx roots query-idxs round-idx 0)
    (generic-fri-sampled-assignment-layers candidate-table roots
      challenges final-value query-idxs round-layers ! 0) xp xn
proof –
  have base-match:
    fri-opening-matches-table (fri-evidence-layer-len roots 0)
      (fri-evidence-layer-idx roots query-idxs round-idx 0)
      candidate-table xp xn
  by (rule generic-fri-no-base-opening-conflict-matches-step
    [OF generic-fri-no-assignment-conflict-no-base[OF no-conflict]
      challenges-nonempty round-bound step])
  show ?thesis
    using base-match by simp
qed

lemma generic-fri-no-successor-opening-conflict-matches-left:
assumes no-conflict:
  ¬ generic-fri-sampled-successor-opening-conflict roots challenges
    query-idxs round-layers
and source-bound: source-round < length query-idxs
and target-bound: target-round < length query-idxs
and layer-bound: Suc layer-idx < length challenges
and forced:
  generic-fri-round-forced-next-value roots challenges query-idxs
    round-layers source-round layer-idx v
and step:
  fri-layer-step-evidence
    (roots ! Suc layer-idx)
    (challenges ! Suc layer-idx)

```

(fri-evidence-layer-len roots (Suc layer-idx))
(fri-evidence-layer-idx roots query-idxs target-round
(Suc layer-idx))
(2 ^ Suc layer-idx)
(fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx))
(fri-evidence-layer-idx roots query-idxs target-round
(Suc layer-idx)))
xp xp-path xn xn-path
(fri-evidence-next-idx roots query-idxs target-round
(Suc layer-idx))
(fri-evidence-next-value roots challenges query-idxs target-round
(Suc layer-idx) xp xn)
(round-layers ! target-round ! Suc layer-idx)
and *same-idx:*
fri-evidence-next-idx roots query-idxs source-round layer-idx =
fri-evidence-layer-idx roots query-idxs target-round
(Suc layer-idx)
shows *v = xp*
using *assms*
unfolding *generic-fri-sampled-successor-opening-conflict-def*
by *blast*

lemma *generic-fri-no-successor-opening-conflict-matches-right:*

assumes *no-conflict:*
 $\neg$ *generic-fri-sampled-successor-opening-conflict roots challenges*
query-idxs round-layers
and *source-bound: source-round < length query-idxs*
and *target-bound: target-round < length query-idxs*
and *layer-bound: Suc layer-idx < length challenges*
and *forced:*
generic-fri-round-forced-next-value roots challenges query-idxs
round-layers source-round layer-idx v
and *step:*
fri-layer-step-evidence
(roots ! Suc layer-idx)
(challenges ! Suc layer-idx)
(fri-evidence-layer-len roots (Suc layer-idx))
(fri-evidence-layer-idx roots query-idxs target-round
(Suc layer-idx))
(2 ^ Suc layer-idx)
(fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx))
(fri-evidence-layer-idx roots query-idxs target-round
(Suc layer-idx)))
xp xp-path xn xn-path
(fri-evidence-next-idx roots query-idxs target-round
(Suc layer-idx))
(fri-evidence-next-value roots challenges query-idxs target-round
(Suc layer-idx) xp xn)
(round-layers ! target-round ! Suc layer-idx)

and *same-idx*:
fri-evidence-next-idx roots *query-idxs* *source-round* *layer-idx* =
fri-sibling-index (*fri-evidence-layer-len* roots (*Suc* *layer-idx*))
(*fri-evidence-layer-idx* roots *query-idxs* *target-round*
(*Suc* *layer-idx*))
shows $v = xn$
using *assms*
unfolding *generic-fri-sampled-successor-opening-conflict-def*
by *blast*

lemma *generic-fri-no-same-layer-opening-conflict-left-left*:
assumes *no-conflict*:
 $\neg$ *generic-fri-sampled-same-layer-opening-conflict* roots *challenges*
query-idxs *round-layers*
and *round-bound*: *round-idx* < *length* *query-idxs*
and *round-bound'*: *round-idx'* < *length* *query-idxs*
and *layer-bound*: *layer-idx* < *length* *challenges*
and *step*:
fri-layer-step-evidence
(*roots* ! *layer-idx*)
(*challenges* ! *layer-idx*)
(*fri-evidence-layer-len* roots *layer-idx*)
(*fri-evidence-layer-idx* roots *query-idxs* *round-idx* *layer-idx*)
($2 \wedge$ *layer-idx*)
(*fri-sibling-index* (*fri-evidence-layer-len* roots *layer-idx*)
(*fri-evidence-layer-idx* roots *query-idxs* *round-idx* *layer-idx*))
xp xp-path xn xn-path
(*fri-evidence-next-idx* roots *query-idxs* *round-idx* *layer-idx*)
(*fri-evidence-next-value* roots *challenges* *query-idxs* *round-idx*
layer-idx xp xn)
(*round-layers* ! *round-idx* ! *layer-idx*)
and *step'*:
fri-layer-step-evidence
(*roots* ! *layer-idx*)
(*challenges* ! *layer-idx*)
(*fri-evidence-layer-len* roots *layer-idx*)
(*fri-evidence-layer-idx* roots *query-idxs* *round-idx'* *layer-idx*)
($2 \wedge$ *layer-idx*)
(*fri-sibling-index* (*fri-evidence-layer-len* roots *layer-idx*)
(*fri-evidence-layer-idx* roots *query-idxs* *round-idx'* *layer-idx*))
yp yp-path yn yn-path
(*fri-evidence-next-idx* roots *query-idxs* *round-idx'* *layer-idx*)
(*fri-evidence-next-value* roots *challenges* *query-idxs* *round-idx'*
layer-idx yp yn)
(*round-layers* ! *round-idx'* ! *layer-idx*)
and *same-idx*:
fri-evidence-layer-idx roots *query-idxs* *round-idx* *layer-idx* =
fri-evidence-layer-idx roots *query-idxs* *round-idx'* *layer-idx*
shows $xp = yp$

using *assms*
unfolding *generic-fri-sampled-same-layer-opening-conflict-def*
by *blast*

lemma *generic-fri-no-same-layer-opening-conflict-left-right:*
assumes *no-conflict:*
 $\neg$ *generic-fri-sampled-same-layer-opening-conflict roots challenges*
query-idxs round-layers
and *round-bound: round-idx < length query-idxs*
and *round-bound': round-idx' < length query-idxs*
and *layer-bound: layer-idx < length challenges*
and *step:*
fri-layer-step-evidence
(roots ! layer-idx)
(challenges ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
(2 ^ layer-idx)
(fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
xp xp-path xn xn-path
(fri-evidence-next-idx roots query-idxs round-idx layer-idx)
(fri-evidence-next-value roots challenges query-idxs round-idx
layer-idx xp xn)
(round-layers ! round-idx ! layer-idx)
and *step':*
fri-layer-step-evidence
(roots ! layer-idx)
(challenges ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
(2 ^ layer-idx)
(fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
yp yp-path yn yn-path
(fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
(fri-evidence-next-value roots challenges query-idxs round-idx'
layer-idx yp yn)
(round-layers ! round-idx' ! layer-idx)
and *same-idx:*
fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
shows *xp = yn*
using *assms*
unfolding *generic-fri-sampled-same-layer-opening-conflict-def*
by *blast*

lemma *generic-fri-no-same-layer-opening-conflict-right-left:*

assumes *no-conflict*:
 $\neg$ *generic-fri-sampled-same-layer-opening-conflict* *roots* *challenges*
query-idxs *round-layers*
and *round-bound*: *round-idx* < length *query-idxs*
and *round-bound'*: *round-idx'* < length *query-idxs*
and *layer-bound*: *layer-idx* < length *challenges*
and *step*:
fri-layer-step-evidence
(*roots* ! *layer-idx*)
(*challenges* ! *layer-idx*)
(*fri-evidence-layer-len* *roots* *layer-idx*)
(*fri-evidence-layer-idx* *roots* *query-idxs* *round-idx* *layer-idx*)
($2 \wedge$ *layer-idx*)
(*fri-sibling-index* (*fri-evidence-layer-len* *roots* *layer-idx*)
(*fri-evidence-layer-idx* *roots* *query-idxs* *round-idx* *layer-idx*))
xp xp-path xn xn-path
(*fri-evidence-next-idx* *roots* *query-idxs* *round-idx* *layer-idx*)
(*fri-evidence-next-value* *roots* *challenges* *query-idxs* *round-idx*
layer-idx xp xn)
(*round-layers* ! *round-idx* ! *layer-idx*)
and *step'*:
fri-layer-step-evidence
(*roots* ! *layer-idx*)
(*challenges* ! *layer-idx*)
(*fri-evidence-layer-len* *roots* *layer-idx*)
(*fri-evidence-layer-idx* *roots* *query-idxs* *round-idx'* *layer-idx*)
($2 \wedge$ *layer-idx*)
(*fri-sibling-index* (*fri-evidence-layer-len* *roots* *layer-idx*)
(*fri-evidence-layer-idx* *roots* *query-idxs* *round-idx'* *layer-idx*))
yp yp-path yn yn-path
(*fri-evidence-next-idx* *roots* *query-idxs* *round-idx'* *layer-idx*)
(*fri-evidence-next-value* *roots* *challenges* *query-idxs* *round-idx'*
layer-idx yp yn)
(*round-layers* ! *round-idx'* ! *layer-idx*)
and *same-idx*:
fri-sibling-index (*fri-evidence-layer-len* *roots* *layer-idx*)
(*fri-evidence-layer-idx* *roots* *query-idxs* *round-idx* *layer-idx*) =
fri-evidence-layer-idx *roots* *query-idxs* *round-idx'* *layer-idx*
shows *xn* = *yp*
using *assms*
unfolding *generic-fri-sampled-same-layer-opening-conflict-def*
by *blast*

lemma *generic-fri-no-same-layer-opening-conflict-right-right*:

assumes *no-conflict*:
 $\neg$ *generic-fri-sampled-same-layer-opening-conflict* *roots* *challenges*
query-idxs *round-layers*
and *round-bound*: *round-idx* < length *query-idxs*
and *round-bound'*: *round-idx'* < length *query-idxs*

and *layer-bound*: *layer-idx* < *length challenges*
and *step*:
fri-layer-step-evidence
(*roots* ! *layer-idx*)
(*challenges* ! *layer-idx*)
(*fri-evidence-layer-len roots layer-idx*)
(*fri-evidence-layer-idx roots query-idxs round-idx layer-idx*)
($2^{\text{layer-idx}}$)
(*fri-sibling-index (fri-evidence-layer-len roots layer-idx)*
(*fri-evidence-layer-idx roots query-idxs round-idx layer-idx*))
xp xp-path xn xn-path
(*fri-evidence-next-idx roots query-idxs round-idx layer-idx*)
(*fri-evidence-next-value roots challenges query-idxs round-idx*
layer-idx xp xn)
(*round-layers* ! *round-idx* ! *layer-idx*)
and *step'*:
fri-layer-step-evidence
(*roots* ! *layer-idx*)
(*challenges* ! *layer-idx*)
(*fri-evidence-layer-len roots layer-idx*)
(*fri-evidence-layer-idx roots query-idxs round-idx' layer-idx*)
($2^{\text{layer-idx}}$)
(*fri-sibling-index (fri-evidence-layer-len roots layer-idx)*
(*fri-evidence-layer-idx roots query-idxs round-idx' layer-idx*))
yp yp-path yn yn-path
(*fri-evidence-next-idx roots query-idxs round-idx' layer-idx*)
(*fri-evidence-next-value roots challenges query-idxs round-idx'*
layer-idx yp yn)
(*round-layers* ! *round-idx'* ! *layer-idx*)
and *same-idx*:
fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(*fri-evidence-layer-idx roots query-idxs round-idx layer-idx*) =
fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(*fri-evidence-layer-idx roots query-idxs round-idx' layer-idx*)
shows *xn = yn*
using *assms*
unfolding *generic-fri-sampled-same-layer-opening-conflict-def*
by *blast*

lemma *generic-fri-no-assignment-conflict-no-successor*:
assumes
 $\neg$ *generic-fri-sampled-assignment-conflict candidate-table roots*
challenges final-value query-idxs round-layers
shows
 $\neg$ *generic-fri-sampled-successor-opening-conflict roots challenges*
query-idxs round-layers
using *assms* **unfolding** *generic-fri-sampled-assignment-conflict-def* **by** *simp*

lemma *generic-fri-no-assignment-conflict-no-same*:

assumes
 $\neg$ *generic-fri-sampled-assignment-conflict candidate-table roots*
challenges final-value query-idxs round-layers
shows
 $\neg$ *generic-fri-sampled-same-layer-opening-conflict roots challenges*
query-idxs round-layers
using *assms unfolding generic-fri-sampled-assignment-conflict-def* **by** *simp*

lemma *generic-fri-sampled-layer-value-constraint-current-left:*

assumes *layer-nonzero: 0 < layer-idx*
and *layer-bound: layer-idx < length challenges*
and *round-bound: round-idx < length query-idxs*
and *step:*
fri-layer-step-evidence
(roots ! layer-idx)
(challenges ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
(2 ^ layer-idx)
(fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
xp xp-path xn xn-path
(fri-evidence-next-idx roots query-idxs round-idx layer-idx)
(fri-evidence-next-value roots challenges query-idxs round-idx
layer-idx xp xn)
(round-layers ! round-idx ! layer-idx)

shows
generic-fri-sampled-layer-value-constraint candidate-table roots
challenges final-value query-idxs round-layers layer-idx
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx) xp

proof —

have *nz: layer-idx ≠ 0*
using *layer-nonzero* **by** *simp*
have *not-final: layer-idx ≠ length challenges*
using *layer-nonzero layer-bound* **by** *simp-all*
show *?thesis*
unfolding *generic-fri-sampled-layer-value-constraint-def*
apply (*simp add: nz not-final*)
apply (*rule disjI2*)
apply (*rule exI[of - round-idx]*)
apply (*intro conjI*)
apply (*rule round-bound*)
apply (*rule layer-bound*)
apply (*rule exI[of - xp]*)
apply (*rule exI[of - xp-path]*)
apply (*rule exI[of - xn]*)
apply (*intro conjI*)
apply (*rule exI[of - xn-path]*)
apply (*rule step*)

```

    apply simp
  done
qed

lemma generic-fri-sampled-layer-value-constraint-current-right:
  assumes layer-nonzero:  $0 < \text{layer-idx}$ 
  and layer-bound:  $\text{layer-idx} < \text{length challenges}$ 
  and round-bound:  $\text{round-idx} < \text{length query-idxs}$ 
  and step:
    fri-layer-step-evidence
      (roots ! layer-idx)
      (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      ( $2^{\text{layer-idx}}$ )
      (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
      xp xp-path xn xn-path
      (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
      (fri-evidence-next-value roots challenges query-idxs round-idx
        layer-idx xp xn)
      (round-layers ! round-idx ! layer-idx)
  shows
    generic-fri-sampled-layer-value-constraint candidate-table roots
      challenges final-value query-idxs round-layers layer-idx
      (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)) xn
proof -
  have nz:  $\text{layer-idx} \neq 0$ 
  using layer-nonzero by simp
  have not-final:  $\text{layer-idx} \neq \text{length challenges}$ 
  using layer-nonzero layer-bound by simp-all
  show ?thesis
  unfolding generic-fri-sampled-layer-value-constraint-def
  apply (simp add: nz not-final)
  apply (rule disjI2)
  apply (rule exI[of - round-idx])
  apply (intro conjI)
  apply (rule round-bound)
  apply (rule layer-bound)
  apply (rule exI[of - xp])
  apply (rule exI[of - xp-path])
  apply (rule exI[of - xn])
  apply (intro conjI)
  apply (rule exI[of - xn-path])
  apply (rule step)
  apply simp
done
qed

```

lemma *generic-fri-sampled-assignment-current-left-matches-step*:
assumes *no-conflict*:
 $\neg$ *generic-fri-sampled-assignment-conflict candidate-table roots*
challenges final-value query-idxs round-layers
and *layer-nonzero*: $0 < \text{layer-idx}$
and *layer-bound*: $\text{layer-idx} < \text{length challenges}$
and *round-bound*: $\text{round-idx} < \text{length query-idxs}$
and *idx-bound*:
fri-evidence-layer-idx roots query-idxs round-idx layer-idx $<$
 $\text{clength} * \text{scale}$
and *step*:
fri-layer-step-evidence
(roots ! layer-idx)
(challenges ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
 $(2 \wedge \text{layer-idx})$
(fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
xp xp-path xn xn-path
(fri-evidence-next-idx roots query-idxs round-idx layer-idx)
(fri-evidence-next-value roots challenges query-idxs round-idx
layer-idx xp xn)
(round-layers ! round-idx ! layer-idx)
shows
generic-fri-sampled-assignment-layers candidate-table roots challenges
final-value query-idxs round-layers ! layer-idx !
fri-evidence-layer-idx roots query-idxs round-idx layer-idx = xp
proof –
let *?raw* = *fri-evidence-layer-idx roots query-idxs round-idx layer-idx*
have *layer-eq*:
generic-fri-sampled-assignment-layers candidate-table roots challenges
final-value query-idxs round-layers ! layer-idx =
generic-fri-sampled-assignment-layer candidate-table roots challenges
final-value query-idxs round-layers layer-idx
by (rule *generic-fri-sampled-assignment-layers-nth*)
(use *layer-bound* **in** *simp*)
have *constraint*:
generic-fri-sampled-layer-value-constraint candidate-table roots
challenges final-value query-idxs round-layers layer-idx ?raw xp
by (rule *generic-fri-sampled-layer-value-constraint-current-left*
[OF *layer-nonzero layer-bound round-bound step*])
have *unique*:
 $\bigwedge v'. \text{generic-fri-sampled-layer-value-constraint candidate-table roots}$
challenges final-value query-idxs round-layers layer-idx ?raw v'
 $\implies v' = xp$
proof –
fix *v'*

```

assume c:
  generic-fri-sampled-layer-value-constraint candidate-table roots
    challenges final-value query-idxs round-layers layer-idx ?raw v'
have pred-or-current:
  (∃ source-round.
    source-round < length query-idxs ∧
    generic-fri-round-forced-next-value roots challenges query-idxs
      round-layers source-round (layer-idx - 1) v' ∧
    fri-evidence-next-idx roots query-idxs source-round
      (layer-idx - 1) = ?raw) ∨
  (∃ round-idx' yp yp-path yn yn-path.
    round-idx' < length query-idxs ∧
    layer-idx < length challenges ∧
    fri-layer-step-evidence
      (roots ! layer-idx)
      (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
      (2 ^ layer-idx)
      (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
      yp yp-path yn yn-path
      (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
      (fri-evidence-next-value roots challenges query-idxs round-idx'
        layer-idx yp yn)
      (round-layers ! round-idx' ! layer-idx) ∧
    ((?raw = fri-evidence-layer-idx roots query-idxs round-idx'
      layer-idx ∧ v' = yp) ∨
    (?raw = fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
      v' = yn)))
using c layer-nonzero layer-bound
unfolding generic-fri-sampled-layer-value-constraint-def
by auto
then show v' = xp
proof
assume
  ∃ source-round.
    source-round < length query-idxs ∧
    generic-fri-round-forced-next-value roots challenges query-idxs
      round-layers source-round (layer-idx - 1) v' ∧
    fri-evidence-next-idx roots query-idxs source-round
      (layer-idx - 1) = ?raw
then obtain source-round where source-bound:
  source-round < length query-idxs
and forced:
  generic-fri-round-forced-next-value roots challenges query-idxs
    round-layers source-round (layer-idx - 1) v'
and same-idx:

```

```

    fri-evidence-next-idx roots query-idxs source-round
      (layer-idx - 1) = ?raw
  by blast
have suc: Suc (layer-idx - 1) = layer-idx
  using layer-nonzero by simp
have v' = xp
  using generic-fri-no-successor-opening-conflict-matches-left
    [OF generic-fri-no-assignment-conflict-no-successor[OF no-conflict]
      source-bound round-bound, of layer-idx - 1 v'
      xp xp-path xn xn-path]
    layer-bound forced step same-idx suc by simp
then show ?thesis .
next
assume
  ∃ round-idx' yp yp-path yn yn-path.
    round-idx' < length query-idxs ∧
    layer-idx < length challenges ∧
    fri-layer-step-evidence
      (roots ! layer-idx)
      (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
      (2 ^ layer-idx)
      (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
      yp yp-path yn yn-path
      (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
      (fri-evidence-next-value roots challenges query-idxs round-idx'
        layer-idx yp yn)
      (round-layers ! round-idx' ! layer-idx) ∧
      ((?raw = fri-evidence-layer-idx roots query-idxs round-idx'
        layer-idx ∧ v' = yp) ∨
        (?raw = fri-sibling-index (fri-evidence-layer-len roots layer-idx)
          (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
          v' = yn))
then obtain round-idx' yp yp-path yn yn-path where round-bound':
  round-idx' < length query-idxs
and step':
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
    yp yp-path yn yn-path
    (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
    (fri-evidence-next-value roots challenges query-idxs round-idx'
      layer-idx yp yn)

```

```

    layer-idx yp yn)
  (round-layers ! round-idx' ! layer-idx)
and value-case:
  (?raw = fri-evidence-layer-idx roots query-idxs round-idx'
    layer-idx  $\wedge$  v' = yp)  $\vee$ 
  (?raw = fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)  $\wedge$ 
    v' = yn)
by blast
from value-case show ?thesis
proof
  assume raw-case:
    ?raw = fri-evidence-layer-idx roots query-idxs round-idx'
    layer-idx  $\wedge$  v' = yp
  have xp = yp
  by (rule generic-fri-no-same-layer-opening-conflict-left-left
    [OF generic-fri-no-assignment-conflict-no-same[OF no-conflict]
      round-bound round-bound' layer-bound step step'])
    (use raw-case in simp)
  then show ?thesis
    using raw-case by simp
next
  assume sibling-case:
    ?raw = fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)  $\wedge$ 
    v' = yn
  have xp = yn
  by (rule generic-fri-no-same-layer-opening-conflict-left-right
    [OF generic-fri-no-assignment-conflict-no-same[OF no-conflict]
      round-bound round-bound' layer-bound step step'])
    (use sibling-case in simp)
  then show ?thesis
    using sibling-case by simp
  qed
qed
qed
have nth-raw:
  generic-fri-sampled-assignment-layer candidate-table roots challenges
  final-value query-idxs round-layers layer-idx ! ?raw = xp
  by (rule generic-fri-sampled-assignment-layer-nth-unique
    [OF idx-bound - constraint unique])
    (use layer-nonzero in simp)
  show ?thesis
    using nth-raw unfolding layer-eq by simp
qed

lemma generic-fri-sampled-assignment-current-right-matches-step:
  assumes no-conflict:
     $\neg$  generic-fri-sampled-assignment-conflict candidate-table roots

```

```

    challenges final-value query-idxs round-layers
and layer-nonzero: 0 < layer-idx
and layer-bound: layer-idx < length challenges
and round-bound: round-idx < length query-idxs
and idx-bound:
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) <
    clength * scale
and step:
    fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
    xp xp-path xn xn-path
    (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
    (fri-evidence-next-value roots challenges query-idxs round-idx
    layer-idx xp xn)
    (round-layers ! round-idx ! layer-idx)
shows
    generic-fri-sampled-assignment-layers candidate-table roots challenges
    final-value query-idxs round-layers ! layer-idx !
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) = xn
proof –
let ?sib =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
have layer-eq:
    generic-fri-sampled-assignment-layers candidate-table roots challenges
    final-value query-idxs round-layers ! layer-idx =
    generic-fri-sampled-assignment-layer candidate-table roots challenges
    final-value query-idxs round-layers layer-idx
by (rule generic-fri-sampled-assignment-layers-nth)
    (use layer-bound in simp)
have constraint:
    generic-fri-sampled-layer-value-constraint candidate-table roots
    challenges final-value query-idxs round-layers layer-idx ?sib xn
by (rule generic-fri-sampled-layer-value-constraint-current-right
    [OF layer-nonzero layer-bound round-bound step])
have unique:
     $\bigwedge v'. \text{generic-fri-sampled-layer-value-constraint candidate-table roots}$ 
    challenges final-value query-idxs round-layers layer-idx ?sib  $v'$ 
     $\implies v' = xn$ 
proof –
fix v'

```

```

assume c:
  generic-fri-sampled-layer-value-constraint candidate-table roots
    challenges final-value query-idxs round-layers layer-idx ?sib v'
have pred-or-current:
  (∃ source-round.
    source-round < length query-idxs ∧
    generic-fri-round-forced-next-value roots challenges query-idxs
      round-layers source-round (layer-idx - 1) v' ∧
    fri-evidence-next-idx roots query-idxs source-round
      (layer-idx - 1) = ?sib) ∨
  (∃ round-idx' yp yp-path yn yn-path.
    round-idx' < length query-idxs ∧
    layer-idx < length challenges ∧
    fri-layer-step-evidence
      (roots ! layer-idx)
      (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
      (2 ^ layer-idx)
      (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
      yp yp-path yn yn-path
      (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
      (fri-evidence-next-value roots challenges query-idxs round-idx'
        layer-idx yp yn)
      (round-layers ! round-idx' ! layer-idx) ∧
    ((?sib = fri-evidence-layer-idx roots query-idxs round-idx'
      layer-idx ∧ v' = yp) ∨
    (?sib = fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
      v' = yn)))
using c layer-nonzero layer-bound
unfolding generic-fri-sampled-layer-value-constraint-def
by auto
then show v' = xn
proof
assume
  ∃ source-round.
    source-round < length query-idxs ∧
    generic-fri-round-forced-next-value roots challenges query-idxs
      round-layers source-round (layer-idx - 1) v' ∧
    fri-evidence-next-idx roots query-idxs source-round
      (layer-idx - 1) = ?sib
then obtain source-round where source-bound:
  source-round < length query-idxs
and forced:
  generic-fri-round-forced-next-value roots challenges query-idxs
    round-layers source-round (layer-idx - 1) v'
and same-idx:

```



```

    fri-evidence-next-idx roots query-idxs source-round
      (layer-idx - 1) = ?sib
  by blast
have suc: Suc (layer-idx - 1) = layer-idx
  using layer-nonzero by simp
have v' = xn
  using generic-fri-no-successor-opening-conflict-matches-right
    [OF generic-fri-no-assignment-conflict-no-successor[OF no-conflict]
      source-bound round-bound, of layer-idx - 1 v'
      xp xp-path xn xn-path]
    layer-bound forced step same-idx suc by simp
then show ?thesis .
next
assume
  ∃ round-idx' yp yp-path yn yn-path.
    round-idx' < length query-idxs ∧
    layer-idx < length challenges ∧
    fri-layer-step-evidence
      (roots ! layer-idx)
      (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
      (2 ^ layer-idx)
      (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
      yp yp-path yn yn-path
      (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
      (fri-evidence-next-value roots challenges query-idxs round-idx'
        layer-idx yp yn)
      (round-layers ! round-idx' ! layer-idx) ∧
      ((?sib = fri-evidence-layer-idx roots query-idxs round-idx'
        layer-idx ∧ v' = yp) ∨
        (?sib = fri-sibling-index (fri-evidence-layer-len roots layer-idx)
          (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
          v' = yn))
then obtain round-idx' yp yp-path yn yn-path where round-bound':
  round-idx' < length query-idxs
and step':
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
    yp yp-path yn yn-path
    (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
    (fri-evidence-next-value roots challenges query-idxs round-idx'
      layer-idx yp yn)

```

```

    layer-idx yp yn)
  (round-layers ! round-idx' ! layer-idx)
and value-case:
  (?sib = fri-evidence-layer-idx roots query-idxs round-idx'
    layer-idx  $\wedge$  v' = yp)  $\vee$ 
  (?sib = fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)  $\wedge$ 
    v' = yn)
by blast
from value-case show ?thesis
proof
  assume raw-case:
    ?sib = fri-evidence-layer-idx roots query-idxs round-idx'
    layer-idx  $\wedge$  v' = yp
  have xn = yp
    by (rule generic-fri-no-same-layer-opening-conflict-right-left
      [OF generic-fri-no-assignment-conflict-no-same[OF no-conflict]
        round-bound round-bound' layer-bound step step'])
    (use raw-case in simp)
  then show ?thesis
    using raw-case by simp
next
  assume sibling-case:
    ?sib = fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)  $\wedge$ 
    v' = yn
  have xn = yn
    by (rule generic-fri-no-same-layer-opening-conflict-right-right
      [OF generic-fri-no-assignment-conflict-no-same[OF no-conflict]
        round-bound round-bound' layer-bound step step'])
    (use sibling-case in simp)
  then show ?thesis
    using sibling-case by simp
qed
qed
qed
have nth-sib:
  generic-fri-sampled-assignment-layer candidate-table roots challenges
  final-value query-idxs round-layers layer-idx ! ?sib = xn
  by (rule generic-fri-sampled-assignment-layer-nth-unique
    [OF idx-bound - constraint unique])
    (use layer-nonzero in simp)
show ?thesis
  using nth-sib unfolding layer-eq by simp
qed

lemma generic-fri-sampled-assignment-current-matches-step:
  assumes no-conflict:
     $\neg$  generic-fri-sampled-assignment-conflict candidate-table roots

```

```

    challenges final-value query-idxs round-layers
and layer-nonzero: 0 < layer-idx
and layer-bound: layer-idx < length challenges
and round-bound: round-idx < length query-idxs
and raw-bound:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
    fri-evidence-layer-len roots layer-idx
and len-bound:
    fri-evidence-layer-len roots layer-idx ≤ clength * scale
and sibling-bound:
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) <
    clength * scale
and step:
    fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
    xp xp-path xn xn-path
    (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
    (fri-evidence-next-value roots challenges query-idxs round-idx
    layer-idx xp xn)
    (round-layers ! round-idx ! layer-idx)
shows
    fri-opening-matches-table (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (generic-fri-sampled-assignment-layers candidate-table roots
    challenges final-value query-idxs round-layers ! layer-idx) xp xn
proof –
let ?len = fri-evidence-layer-len roots layer-idx
let ?raw = fri-evidence-layer-idx roots query-idxs round-idx layer-idx
let ?sib = fri-sibling-index ?len ?raw
let ?layer =
    generic-fri-sampled-assignment-layers candidate-table roots challenges
    final-value query-idxs round-layers ! layer-idx
have raw-bound-assignment: ?raw < clength * scale
using raw-bound len-bound by simp
have layer-eq:
    ?layer =
    generic-fri-sampled-assignment-layer candidate-table roots challenges
    final-value query-idxs round-layers layer-idx
by (rule generic-fri-sampled-assignment-layers-nth)
    (use layer-bound in simp)
have layer-len: length ?layer = clength * scale
using layer-eq layer-nonzero by simp

```

```

have left:
  ?layer ! ?raw = xp
  by (rule generic-fri-sampled-assignment-current-left-matches-step
    [OF no-conflict layer-nonzero layer-bound round-bound
      raw-bound-assignment step])
have right:
  ?layer ! ?sib = xn
  by (rule generic-fri-sampled-assignment-current-right-matches-step
    [OF no-conflict layer-nonzero layer-bound round-bound
      sibling-bound step])
show ?thesis
  unfolding fri-opening-matches-table-def
  using raw-bound len-bound layer-len left right by simp
qed

lemma generic-fri-sampled-assignment-successor-forced-value:
  assumes no-conflict:
    ¬ generic-fri-sampled-assignment-conflict candidate-table roots
      challenges final-value query-idxs round-layers
  and layer-bound: Suc layer-idx < length challenges
  and round-bound: round-idx < length query-idxs
  and next-bound:
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
      clength * scale
  and forced:
    generic-fri-round-forced-next-value roots challenges query-idxs
      round-layers round-idx layer-idx v
  shows
    generic-fri-sampled-assignment-layers candidate-table roots challenges
      final-value query-idxs round-layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx = v
proof –
  let ?next = fri-evidence-next-idx roots query-idxs round-idx layer-idx
  have suc-nonzero: Suc layer-idx ≠ 0
  by simp
  have layer-eq:
    generic-fri-sampled-assignment-layers candidate-table roots challenges
      final-value query-idxs round-layers ! Suc layer-idx =
    generic-fri-sampled-assignment-layer candidate-table roots challenges
      final-value query-idxs round-layers (Suc layer-idx)
  by (rule generic-fri-sampled-assignment-layers-nth)
    (use layer-bound in simp)
  have constraint:
    generic-fri-sampled-layer-value-constraint candidate-table roots
      challenges final-value query-idxs round-layers (Suc layer-idx)
      ?next v
  using layer-bound round-bound forced
  unfolding generic-fri-sampled-layer-value-constraint-def
  by auto

```

```

have unique:
   $\wedge v'. \text{generic-fri-sampled-layer-value-constraint candidate-table roots}$ 
     $\text{challenges final-value query-idxs round-layers (Suc layer-idx)}$ 
     $?next\ v' \implies v' = v$ 
proof -
  fix  $v'$ 
  assume  $c$ :
     $\text{generic-fri-sampled-layer-value-constraint candidate-table roots}$ 
       $\text{challenges final-value query-idxs round-layers (Suc layer-idx)}$ 
       $?next\ v'$ 
  have pred-or-current:
     $(\exists \text{source-round.}$ 
       $\text{source-round} < \text{length query-idxs} \wedge$ 
       $\text{generic-fri-round-forced-next-value roots challenges query-idxs}$ 
         $\text{round-layers source-round layer-idx } v' \wedge$ 
       $\text{fri-evidence-next-idx roots query-idxs source-round layer-idx} =$ 
         $?next) \vee$ 
     $(\exists \text{target-round } xp\ \text{xp-path } xn\ \text{xn-path.}$ 
       $\text{target-round} < \text{length query-idxs} \wedge$ 
       $\text{Suc layer-idx} < \text{length challenges} \wedge$ 
       $\text{fri-layer-step-evidence}$ 
         $(\text{roots ! Suc layer-idx})$ 
         $(\text{challenges ! Suc layer-idx})$ 
         $(\text{fri-evidence-layer-len roots (Suc layer-idx)})$ 
         $(\text{fri-evidence-layer-idx roots query-idxs target-round}$ 
           $(\text{Suc layer-idx}))$ 
         $(2 \wedge \text{Suc layer-idx})$ 
         $(\text{fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx)})$ 
           $(\text{fri-evidence-layer-idx roots query-idxs target-round}$ 
             $(\text{Suc layer-idx})))$ 
         $xp\ \text{xp-path } xn\ \text{xn-path}$ 
         $(\text{fri-evidence-next-idx roots query-idxs target-round}$ 
           $(\text{Suc layer-idx}))$ 
         $(\text{fri-evidence-next-value roots challenges query-idxs target-round}$ 
           $(\text{Suc layer-idx})\ xp\ xn)$ 
         $(\text{round-layers ! target-round ! Suc layer-idx}) \wedge$ 
         $((?next = \text{fri-evidence-layer-idx roots query-idxs target-round}$ 
           $(\text{Suc layer-idx}) \wedge v' = xp) \vee$ 
         $(?next = \text{fri-sibling-index}$ 
           $(\text{fri-evidence-layer-len roots (Suc layer-idx)})$ 
           $(\text{fri-evidence-layer-idx roots query-idxs target-round}$ 
             $(\text{Suc layer-idx})) \wedge$ 
           $v' = xn)))$ 
    using  $c\ \text{layer-bound}$ 
    unfolding  $\text{generic-fri-sampled-layer-value-constraint-def}$ 
    by auto
  then show  $v' = v$ 
proof
  assume

```

```

     $\exists$  source-round.
      source-round < length query-idxs  $\wedge$ 
      generic-fri-round-forced-next-value roots challenges query-idxs
        round-layers source-round layer-idx  $v'$   $\wedge$ 
      fri-evidence-next-idx roots query-idxs source-round layer-idx =
        ?next
  then obtain source-round where source-bound:
    source-round < length query-idxs
  and forced':
    generic-fri-round-forced-next-value roots challenges query-idxs
      round-layers source-round layer-idx  $v'$ 
  and same-idx:
    fri-evidence-next-idx roots query-idxs source-round layer-idx =
      ?next
  by blast
have  $v = v'$ 
proof -
  have layer-bound0: layer-idx < length challenges
    using layer-bound by simp
  show ?thesis
  by (rule generic-fri-no-next-value-conflict-unique
    [OF generic-fri-no-assignment-conflict-no-next[OF no-conflict]
      round-bound source-bound layer-bound0 - forced forced'])
    (use same-idx in simp)
qed
then show ?thesis by simp
next
assume
   $\exists$  target-round  $xp$   $xp$ -path  $xn$   $xn$ -path.
    target-round < length query-idxs  $\wedge$ 
    Suc layer-idx < length challenges  $\wedge$ 
    fri-layer-step-evidence
      (roots ! Suc layer-idx)
      (challenges ! Suc layer-idx)
      (fri-evidence-layer-len roots (Suc layer-idx))
      (fri-evidence-layer-idx roots query-idxs target-round
        (Suc layer-idx))
      ( $2 \wedge$  Suc layer-idx)
      (fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx))
        (fri-evidence-layer-idx roots query-idxs target-round
          (Suc layer-idx)))
     $xp$   $xp$ -path  $xn$   $xn$ -path
      (fri-evidence-next-idx roots query-idxs target-round
        (Suc layer-idx))
      (fri-evidence-next-value roots challenges query-idxs target-round
        (Suc layer-idx)  $xp$   $xn$ )
      (round-layers ! target-round ! Suc layer-idx)  $\wedge$ 
      ((?next = fri-evidence-layer-idx roots query-idxs target-round
        (Suc layer-idx)  $\wedge$   $v' = xp$ )  $\vee$ 

```

```

    (?next = fri-sibling-index
      (fri-evidence-layer-len roots (Suc layer-idx))
      (fri-evidence-layer-idx roots query-idxs target-round
        (Suc layer-idx)) ∧
      v' = xn))
then obtain target-round xp xp-path xn xn-path where target-bound:
  target-round < length query-idxs
and step:
  fri-layer-step-evidence
    (roots ! Suc layer-idx)
    (challenges ! Suc layer-idx)
    (fri-evidence-layer-len roots (Suc layer-idx))
    (fri-evidence-layer-idx roots query-idxs target-round
      (Suc layer-idx))
    (2 ^ Suc layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx))
      (fri-evidence-layer-idx roots query-idxs target-round
        (Suc layer-idx)))
    xp xp-path xn xn-path
    (fri-evidence-next-idx roots query-idxs target-round
      (Suc layer-idx))
    (fri-evidence-next-value roots challenges query-idxs target-round
      (Suc layer-idx) xp xn)
    (round-layers ! target-round ! Suc layer-idx)
and value-case:
  (?next = fri-evidence-layer-idx roots query-idxs target-round
    (Suc layer-idx) ∧ v' = xp) ∨
  (?next = fri-sibling-index
    (fri-evidence-layer-len roots (Suc layer-idx))
    (fri-evidence-layer-idx roots query-idxs target-round
      (Suc layer-idx)) ∧
    v' = xn)
by blast
from value-case show ?thesis
proof
  assume raw-case:
    ?next = fri-evidence-layer-idx roots query-idxs target-round
      (Suc layer-idx) ∧ v' = xp
  have v = xp
  by (rule generic-fri-no-successor-opening-conflict-matches-left
    [OF generic-fri-no-assignment-conflict-no-successor
      [OF no-conflict] round-bound target-bound layer-bound
      forced step])
  (use raw-case in simp)
  then show ?thesis
  using raw-case by simp
next
  assume sibling-case:
    ?next = fri-sibling-index

```

```

      (fri-evidence-layer-len roots (Suc layer-idx))
      (fri-evidence-layer-idx roots query-idxs target-round
        (Suc layer-idx))  $\wedge$ 
       $v' = xn$ 
    have  $v = xn$ 
    by (rule generic-fri-no-successor-opening-conflict-matches-right
      [OF generic-fri-no-assignment-conflict-no-successor
        [OF no-conflict] round-bound target-bound layer-bound
        forced step])
      (use sibling-case in simp)
    then show ?thesis
    using sibling-case by simp
  qed
qed
qed
have nth-next:
  generic-fri-sampled-assignment-layer candidate-table roots challenges
  final-value query-idxs round-layers (Suc layer-idx) ! ?next = v
  by (rule generic-fri-sampled-assignment-layer-nth-unique
    [OF next-bound suc-nonzero constraint unique])
  show ?thesis
  using nth-next unfolding layer-eq by simp
qed

lemma accepted-fri-opening-transcript-query-idx-bound:
  assumes
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    query-idxs trace-round-layers composition-round-layers
  and  $idx \in \text{set query-idxs}$ 
  shows  $idx < \text{clength} * \text{scale}$ 
  using assms
  unfolding accepted-fri-opening-transcript-def by blast

lemma trace-fri-partial-candidate-opening-evidence-query-idx-bound:
  assumes
    trace-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
  and  $idx \in \text{set fri-query-idxs}$ 
  shows  $idx < \text{clength} * \text{scale}$ 
proof -
  have fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  by (rule trace-fri-partial-candidate-opening-evidenceD(1)[OF assms(1)])

```



```

show ?thesis
  by (rule accepted-fri-opening-transcript-query-idx-bound
    [OF fri-openings assms(2)])
qed

lemma composition-fri-partial-candidate-opening-evidence-query-idx-bound:
  assumes
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  and idx ∈ set fri-query-idxs
  shows idx < clength * scale
proof –
  have fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  by (rule composition-fri-partial-candidate-opening-evidenceD(1)
    [OF assms(1)])
  show ?thesis
  by (rule accepted-fri-opening-transcript-query-idx-bound
    [OF fri-openings assms(2)])
qed

lemma fri-layer-indices-nth-bound:
  assumes query-bound: query-idx < len
  and layer-bound: layer-idx < n
  and eval-power: len = 2 ^ N
  and rounds-bound: n ≤ N
  shows
    fri-layer-indices n query-idx len ! layer-idx <
    fri-layer-lengths n len ! layer-idx
  using layer-bound query-bound rounds-bound eval-power
proof (induction layer-idx arbitrary: n query-idx len N)
  case 0
  then show ?case
  by (cases n) simp-all
next
  case (Suc layer-idx)
  then obtain n' where n-eq: n = Suc n'
  by (cases n) auto
  have layer-bound': layer-idx < n'
  using Suc.premis(1) unfolding n-eq by simp
  have len-even: 2 dvd len

```

```

    using Suc.premis(3,4) n-eq Suc.premis(1)
    by (cases N) simp-all
  have len-half-power:
     $\exists N'. \text{len div } 2 = 2 \wedge N' \wedge n' \leq N'$ 
  proof (cases N)
    case 0
    then show ?thesis
      using Suc.premis(3) n-eq by simp
  next
    case (Suc N')
    have len div 2 =  $2 \wedge N'$ 
      using Suc.premis(4) unfolding Suc by simp
    moreover have  $n' \leq N'$ 
      using Suc.premis(3) unfolding n-eq Suc by simp
    ultimately show ?thesis by blast
  qed
  then obtain N' where len-half-eq:  $\text{len div } 2 = 2 \wedge N'$ 
    and rounds-bound':  $n' \leq N'$ 
    by blast
  have query-bound':
     $\text{query-idx mod } (\text{len div } 2) < \text{len div } 2$ 
    using len-half-eq by simp
  have ih:
     $\text{fri-layer-indices } n' (\text{query-idx mod } (\text{len div } 2)) (\text{len div } 2) !$ 
     $\text{layer-idx} <$ 
     $\text{fri-layer-lengths } n' (\text{len div } 2) ! \text{layer-idx}$ 
    by (rule Suc.IH[OF layer-bound' query-bound' rounds-bound'
      len-half-eq])
  show ?case
    using ih unfolding n-eq by simp
  qed

lemma fri-evidence-layer-idx-bound-from-query-bound:
  assumes partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
  and query-bound:  $\text{query-idx} < \text{clength} * \text{scale}$ 
  and round-idx-eq:  $\text{query-idx} = \text{query-idxs} ! \text{round-idx}$ 
  and layer-bound:  $\text{layer-idx} < \text{length challenges}$ 
  and eval-power:  $\text{clength} * \text{scale} = 2 \wedge N$ 
  and rounds-bound:  $\text{length challenges} \leq N$ 
  shows
     $\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx} <$ 
     $\text{fri-evidence-layer-len roots layer-idx}$ 
  proof -
    have roots-len:  $\text{length challenges} = \text{length roots}$ 
      by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
    show ?thesis
      unfolding fri-evidence-layer-idx-def fri-evidence-layer-len-def

```

```

using fri-layer-indices-nth-bound
  [OF query-bound, of layer-idx length roots N]
  layer-bound roots-len eval-power rounds-bound round-idx-eq
by simp
qed

```

```

lemma fri-evidence-layer-len-le-eval-domain:
  assumes partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
  and layer-bound: layer-idx < length challenges
  and eval-power: clength * scale = 2 ^ N
  shows fri-evidence-layer-len roots layer-idx ≤ clength * scale
proof –
  have roots-len: length challenges = length roots
    by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
  have roots-bound: layer-idx < length roots
    using layer-bound roots-len by simp
  have len-eq:
    fri-evidence-layer-len roots layer-idx =
      (clength * scale) div 2 ^ layer-idx
    unfolding fri-evidence-layer-len-def
    by (rule fri-layer-lengths-nth-div[OF roots-bound])
  show ?thesis
    unfolding len-eq by simp
qed

```

```

lemma fri-evidence-sibling-index-bound-from-raw:
  assumes raw-bound:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
    fri-evidence-layer-len roots layer-idx
  and len-bound:
    fri-evidence-layer-len roots layer-idx ≤ clength * scale
  shows
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) <
    clength * scale
proof –
  have len-pos: 0 < fri-evidence-layer-len roots layer-idx
    using raw-bound by simp
  have sibling-lt:
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) <
    fri-evidence-layer-len roots layer-idx
    unfolding fri-sibling-index-def
    using len-pos by simp
  then show ?thesis
    using len-bound by linarith
qed

```

```

lemma fri-evidence-next-idx-bound-from-raw:
  assumes raw-bound:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
      fri-evidence-layer-len roots layer-idx
  and len-bound:
    fri-evidence-layer-len roots layer-idx ≤ clength * scale
  shows
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
      clength * scale
proof –
  have len-pos: 0 < fri-evidence-layer-len roots layer-idx
    using raw-bound by simp
  have half-le: fri-evidence-layer-len roots layer-idx div 2 ≤
    clength * scale
    using len-bound by simp
  show ?thesis
proof (cases fri-evidence-layer-len roots layer-idx div 2 = 0)
  case True
    then show ?thesis
      unfolding fri-evidence-next-idx-def
      using raw-bound len-bound by simp
  next
  case False
    then have half-pos:
      0 < fri-evidence-layer-len roots layer-idx div 2
      by simp
    have mod-lt:
      fri-evidence-layer-idx roots query-idxs round-idx layer-idx mod
        (fri-evidence-layer-len roots layer-idx div 2) <
        fri-evidence-layer-len roots layer-idx div 2
      by (rule mod-less-divisor[OF half-pos])
    then show ?thesis
      unfolding fri-evidence-next-idx-def
      using half-le by linarith
  qed
qed

```

```

lemma trace-fri-reachable-missing-sampled-layer-chain-None[simp]:
   $\neg$  trace-fri-reachable-missing-sampled-layer-chain s None
  unfolding trace-fri-reachable-missing-sampled-layer-chain-def
    trace-fri-bad-with-reachable-partial-candidate-def
    accepted-with-partial-trace-openings-def
  by simp

```

```

lemma composition-fri-reachable-missing-sampled-layer-chain-None[simp]:
   $\neg$  composition-fri-reachable-missing-sampled-layer-chain s None
  unfolding composition-fri-reachable-missing-sampled-layer-chain-def
    composition-fri-bad-with-reachable-partial-candidate-def

```

$\text{accepted-with-partial-initial-openings-def}$
 $\text{accepted-with-partial-trace-openings-def}$
by *simp*

lemma $\text{trace-fri-sampled-layer-assignment-obstruction-None}[simp]$:
 $\neg \text{trace-fri-sampled-layer-assignment-obstruction } s \text{ None}$
unfolding $\text{trace-fri-sampled-layer-assignment-obstruction-def}$
 $\text{trace-fri-partial-candidate-opening-evidence-def}$
 $\text{accepted-fri-opening-transcript-def}$
by *simp*

lemma $\text{composition-fri-sampled-layer-assignment-obstruction-None}[simp]$:
 $\neg \text{composition-fri-sampled-layer-assignment-obstruction } s \text{ None}$
unfolding $\text{composition-fri-sampled-layer-assignment-obstruction-def}$
 $\text{composition-fri-partial-candidate-opening-evidence-def}$
 $\text{accepted-fri-opening-transcript-def}$
by *simp*

lemma $\text{trace-fri-sampled-assignment-conflict-None}[simp]$:
 $\neg \text{trace-fri-sampled-assignment-conflict } s \text{ None}$
unfolding $\text{trace-fri-sampled-assignment-conflict-def}$
 $\text{trace-fri-partial-candidate-opening-evidence-def}$
 $\text{accepted-fri-opening-transcript-def}$
by *simp*

lemma $\text{composition-fri-sampled-assignment-conflict-None}[simp]$:
 $\neg \text{composition-fri-sampled-assignment-conflict } s \text{ None}$
unfolding $\text{composition-fri-sampled-assignment-conflict-def}$
 $\text{composition-fri-partial-candidate-opening-evidence-def}$
 $\text{accepted-fri-opening-transcript-def}$
by *simp*

lemma $\text{wp-trace-fri-assignment-obstruction-bound-from-conflict}$:
assumes support-imp :
 $\bigwedge \text{out. out} \in \text{set-dist } (\text{execute verify-monad } s) \implies$
 $\text{trace-fri-sampled-layer-assignment-obstruction } s \text{ out} \implies$
 $\text{trace-fri-sampled-assignment-conflict } s \text{ out}$
and conflict-bound :
 $\text{wp-event verify-monad } (\text{trace-fri-sampled-assignment-conflict } s) \text{ } s$
 $\leq C$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-layer-assignment-obstruction } s) \text{ } s \leq C$
proof –
have $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-layer-assignment-obstruction } s) \text{ } s \leq$
 $\text{wp-event verify-monad } (\text{trace-fri-sampled-assignment-conflict } s) \text{ } s$
by (rule $\text{wp-event-mono-on-support}$) (use support-imp in *blast*)
also have $\dots \leq C$

by (rule conflict-bound)
 finally show ?thesis .
 qed

lemma wp-composition-fri-assignment-obstruction-bound-from-conflict:

assumes support-imp:

$\bigwedge out. out \in \text{set-dist } (\text{execute verify-monad } s) \implies$
 $\text{composition-fri-sampled-layer-assignment-obstruction } s \ out \implies$
 $\text{composition-fri-sampled-assignment-conflict } s \ out$

and conflict-bound:

wp-event verify-monad
 $(\text{composition-fri-sampled-assignment-conflict } s) \ s \leq C$

shows

wp-event verify-monad
 $(\text{composition-fri-sampled-layer-assignment-obstruction } s) \ s \leq C$

proof –

have wp-event verify-monad

$(\text{composition-fri-sampled-layer-assignment-obstruction } s) \ s \leq$

wp-event verify-monad

$(\text{composition-fri-sampled-assignment-conflict } s) \ s$

by (rule wp-event-mono-on-support) (use support-imp in blast)

also have $\dots \leq C$

by (rule conflict-bound)

finally show ?thesis .

qed

lemma trace-fri-reachable-missing-sampled-imp-assignment-obstruction-on-support:

assumes support:

$out \in \text{set-dist } (\text{execute verify-monad } s)$

and missing:

trace-fri-reachable-missing-sampled-layer-chain s out

shows trace-fri-sampled-layer-assignment-obstruction s out

proof (cases out)

case None

then show ?thesis

using missing

unfolding trace-fri-reachable-missing-sampled-layer-chain-def

trace-fri-bad-with-reachable-partial-candidate-def

accepted-with-partial-trace-openings-def

by simp

next

case (Some pair)

then obtain result final-state where pair: pair = (result, final-state)

by (cases pair) simp

have outcome:

$\text{Some } (result, final-state) \in \text{set-dist } (\text{execute verify-monad } s)$

using support Some pair by simp

show ?thesis

proof (rule verify-monad-trace-fri-missing-sampled-extracts-opening-evidence

```

    [OF outcome])
  show
    trace-fri-reachable-missing-sampled-layer-chain s
    (Some (result, final-state))
  using missing Some pair by simp
next
  fix trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
  assume evidence:
    trace-fri-partial-candidate-opening-evidence s
    (Some (result, final-state)) trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr candidate-query-idxs
    trace-openings trace-table
  assume no-chain:
     $\bigwedge$  doms layers.  $\neg$ 
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers
  have partial:
    generic-fri-partial-evidence trace-table-low-degree trace-table
    (clength - 1) trace-roots trace-bs trace-final fri-query-idxs
    trace-round-layers
  by (rule trace-fri-partial-candidate-opening-evidence-generic-partial
    [OF evidence])
  have obstruction:
    generic-fri-sampled-layer-assignment-obstruction
    trace-table-low-degree trace-table (clength - 1) trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers
  unfolding generic-fri-sampled-layer-assignment-obstruction-def
  using partial no-chain by blast
  show trace-fri-sampled-layer-assignment-obstruction s out
  unfolding trace-fri-sampled-layer-assignment-obstruction-def
  using evidence obstruction Some pair by blast
qed
qed

lemma composition-fri-reachable-missing-sampled-imp-assignment-obstruction-on-support:
  assumes support:
    out  $\in$  set-dist (execute verify-monad s)
  and missing:
    composition-fri-reachable-missing-sampled-layer-chain s out
  shows composition-fri-sampled-layer-assignment-obstruction s out
proof (cases out)
  case None
  then show ?thesis
  unfolding composition-fri-sampled-layer-assignment-obstruction-def

```

```

using missing
unfolding composition-fri-reachable-missing-sampled-layer-chain-def
           composition-fri-bad-with-reachable-partial-candidate-def
           accepted-with-partial-initial-openings-def
           accepted-with-partial-trace-openings-def
by simp
next
case (Some pair)
then obtain result final-state where pair: pair = (result, final-state)
  by (cases pair) simp
have outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
  using support Some pair by simp
show ?thesis
proof (rule
  verify-monad-composition-fri-missing-sampled-extracts-opening-evidence
  [OF outcome])
show
  composition-fri-reachable-missing-sampled-layer-chain s
  (Some (result, final-state))
  using missing Some pair by simp
next
fix trace-roots trace-bs trace-final fri-dg opening-composition-roots
     composition-bs composition-final fri-query-idxs trace-round-layers
     composition-round-layers fr f-fri-roots f-final as dg
     composition-fri-roots final trace-query-idxs trace-openings
     composition-query-idxs composition-openings trace-table
     composition-table
assume evidence:
  composition-fri-partial-candidate-opening-evidence s
  (Some (result, final-state)) trace-roots trace-bs trace-final
  fri-dg opening-composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers fr
  f-fri-roots f-final as dg composition-fri-roots final
  trace-query-idxs trace-openings composition-query-idxs
  composition-openings trace-table composition-table
assume degree-bound: to-nat fri-dg ≤ maxDegree
assume no-chain:
   $\bigwedge \text{doms layers. } \neg$ 
  generic-fri-sampled-layer-chain-evidence
  (composition-table-low-degree (to-nat fri-dg))
  composition-table (to-nat fri-dg) opening-composition-roots
  composition-bs composition-final fri-query-idxs
  composition-round-layers doms layers
have partial:
  generic-fri-partial-evidence
  (composition-table-low-degree (to-nat fri-dg)) composition-table
  (to-nat fri-dg) opening-composition-roots composition-bs
  composition-final fri-query-idxs composition-round-layers

```



```

    by (rule composition-fri-partial-candidate-opening-evidence-generic-partial
        [OF evidence])
  have obstruction:
    generic-fri-sampled-layer-assignment-obstruction
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers
  unfolding generic-fri-sampled-layer-assignment-obstruction-def
  using partial no-chain by blast
  show composition-fri-sampled-layer-assignment-obstruction s out
  unfolding composition-fri-sampled-layer-assignment-obstruction-def
  using evidence obstruction Some pair degree-bound by meson
qed
qed

lemma wp-trace-fri-reachable-missing-sampled-bound-from-assignment-obstruction:
  assumes obstruction:
    wp-event verify-monad
      (trace-fri-sampled-layer-assignment-obstruction s) s ≤ M
  shows
    wp-event verify-monad
      (trace-fri-reachable-missing-sampled-layer-chain s) s ≤ M
proof -
  have wp-event verify-monad
    (trace-fri-reachable-missing-sampled-layer-chain s) s
    ≤ wp-event verify-monad
      (trace-fri-sampled-layer-assignment-obstruction s) s
  by (rule wp-event-mono-on-support)
    (rule trace-fri-reachable-missing-sampled-imp-assignment-obstruction-on-support)
  also have ... ≤ M
  by (rule obstruction)
  finally show ?thesis .
qed

lemma wp-composition-fri-reachable-missing-sampled-bound-from-assignment-obstruction:
  assumes obstruction:
    wp-event verify-monad
      (composition-fri-sampled-layer-assignment-obstruction s) s ≤ M
  shows
    wp-event verify-monad
      (composition-fri-reachable-missing-sampled-layer-chain s) s ≤ M
proof -
  have wp-event verify-monad
    (composition-fri-reachable-missing-sampled-layer-chain s) s
    ≤ wp-event verify-monad
      (composition-fri-sampled-layer-assignment-obstruction s) s
  by (rule wp-event-mono-on-support)
    (rule composition-fri-reachable-missing-sampled-imp-assignment-obstruction-on-support)
  also have ... ≤ M

```

by (rule obstruction)
 finally show ?thesis .
 qed

lemma *trace-fri-reachable-partial-candidate-reduction-from-sampled-and-assignment-obstruction:*

assumes *sampled-bound:*

wp-event verify-monad

(*trace-fri-bad-with-sampled-layer-chain s*) $s \leq R$

and *obstruction-bound:*

wp-event verify-monad

(*trace-fri-sampled-layer-assignment-obstruction s*) $s \leq M$

and *total-bound:* $R + M \leq \text{trace-fri-error}$

shows *trace-fri-reachable-partial-candidate-reduction-assumption s*

proof (rule *trace-fri-reachable-partial-candidate-reduction-from-sampled-and-missing*)

show *wp-event verify-monad*

(*trace-fri-bad-with-sampled-layer-chain s*) $s \leq R$

by (rule *sampled-bound*)

next

show *wp-event verify-monad*

(*trace-fri-reachable-missing-sampled-layer-chain s*) $s \leq M$

by (rule *wp-trace-fri-reachable-missing-sampled-bound-from-assignment-obstruction*
 [OF *obstruction-bound*])

next

show $R + M \leq \text{trace-fri-error}$

by (rule *total-bound*)

qed

lemma *composition-fri-reachable-partial-candidate-reduction-from-sampled-and-assignment-obstruction:*

assumes *sampled-bound:*

wp-event verify-monad

(*composition-fri-bad-with-sampled-layer-chain s*) $s \leq R$

and *obstruction-bound:*

wp-event verify-monad

(*composition-fri-sampled-layer-assignment-obstruction s*) $s \leq M$

and *total-bound:* $R + M \leq \text{composition-fri-error}$

shows *composition-fri-reachable-partial-candidate-reduction-assumption s*

proof (rule

composition-fri-reachable-partial-candidate-reduction-from-sampled-and-missing)

show *wp-event verify-monad*

(*composition-fri-bad-with-sampled-layer-chain s*) $s \leq R$

by (rule *sampled-bound*)

next

show *wp-event verify-monad*

(*composition-fri-reachable-missing-sampled-layer-chain s*) $s \leq M$

by (rule
wp-composition-fri-reachable-missing-sampled-bound-from-assignment-obstruction
 [OF *obstruction-bound*])

next

show $R + M \leq \text{composition-fri-error}$

by (rule total-bound)
qed

lemma *trace-fri-reachable-partial-candidate-reduction-from-sampled-and-assignment-conflict:*

assumes *sampled-bound:*

wp-event verify-monad

$(\text{trace-fri-bad-with-sampled-layer-chain } s) \ s \leq R$

and *support-imp:*

$\bigwedge \text{out. out} \in \text{set-dist } (\text{execute verify-monad } s) \implies$

$\text{trace-fri-sampled-layer-assignment-obstruction } s \ \text{out} \implies$

$\text{trace-fri-sampled-assignment-conflict } s \ \text{out}$

and *conflict-bound:*

wp-event verify-monad

$(\text{trace-fri-sampled-assignment-conflict } s) \ s \leq M$

and *total-bound:* $R + M \leq \text{trace-fri-error}$

shows *trace-fri-reachable-partial-candidate-reduction-assumption* s

proof (rule

trace-fri-reachable-partial-candidate-reduction-from-sampled-and-assignment-obstruction)

show *wp-event verify-monad*

$(\text{trace-fri-bad-with-sampled-layer-chain } s) \ s \leq R$

by (rule sampled-bound)

next

show *wp-event verify-monad*

$(\text{trace-fri-sampled-layer-assignment-obstruction } s) \ s \leq M$

by (rule *wp-trace-fri-assignment-obstruction-bound-from-conflict*
[OF *support-imp conflict-bound*])

next

show $R + M \leq \text{trace-fri-error}$

by (rule total-bound)

qed

lemma *composition-fri-reachable-partial-candidate-reduction-from-sampled-and-assignment-conflict:*

assumes *sampled-bound:*

wp-event verify-monad

$(\text{composition-fri-bad-with-sampled-layer-chain } s) \ s \leq R$

and *support-imp:*

$\bigwedge \text{out. out} \in \text{set-dist } (\text{execute verify-monad } s) \implies$

$\text{composition-fri-sampled-layer-assignment-obstruction } s \ \text{out} \implies$

$\text{composition-fri-sampled-assignment-conflict } s \ \text{out}$

and *conflict-bound:*

wp-event verify-monad

$(\text{composition-fri-sampled-assignment-conflict } s) \ s \leq M$

and *total-bound:* $R + M \leq \text{composition-fri-error}$

shows *composition-fri-reachable-partial-candidate-reduction-assumption* s

proof (rule

composition-fri-reachable-partial-candidate-reduction-from-sampled-and-assignment-obstruction)

show *wp-event verify-monad*

$(\text{composition-fri-bad-with-sampled-layer-chain } s) \ s \leq R$

by (rule sampled-bound)

```

next
  show wp-event verify-monad
    (composition-fri-sampled-layer-assignment-obstruction s) s ≤ M
  by (rule wp-composition-fri-assignment-obstruction-bound-from-conflict
      [OF support-imp conflict-bound])
next
  show R + M ≤ composition-fri-error
    by (rule total-bound)
qed

lemma trace-fri-bad-with-partial-candidates-bound-from-sampled-and-assignment-obstruction:
  fixes R M :: prob
  assumes sampled-bound:
     $\bigwedge \text{data attacker-state.}$ 
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (trace-fri-bad-with-sampled-layer-chain
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤ R
  and obstruction-bound:
     $\bigwedge \text{data attacker-state.}$ 
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (trace-fri-sampled-layer-assignment-obstruction
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤ M
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-partial-candidates)
      adversary-initial-state ≤ R + M
proof (rule trace-fri-bad-with-partial-candidates-bound-from-sampled-and-missing)
  fix data attacker-state
  assume builder:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  show wp-event verify-monad
    (trace-fri-bad-with-sampled-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))

```

```

    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤ R
  by (rule sampled-bound[OF builder])
next
fix data attacker-state
assume builder:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
                  adversary-initial-state)
show wp-event verify-monad
  (trace-fri-reachable-missing-sampled-layer-chain
   (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data)) ≤ M
by (rule
    wp-trace-fri-reachable-missing-sampled-bound-from-assignment-obstruction)
  (rule obstruction-bound[OF builder])
qed

lemma composition-fri-bad-with-partial-candidates-bound-from-sampled-and-assignment-obstruction:
  fixes R M :: prob
  assumes sampled-bound:
    ∧ data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
                          adversary-initial-state) ⇒
      wp-event verify-monad
        (composition-fri-bad-with-sampled-layer-chain
         (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
        (verifier-state-from-adversary attacker-state
         (staged-proof-transcript data)) ≤ R
  and obstruction-bound:
    ∧ data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
                          adversary-initial-state) ⇒
      wp-event verify-monad
        (composition-fri-sampled-layer-assignment-obstruction
         (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
        (verifier-state-from-adversary attacker-state
         (staged-proof-transcript data)) ≤ M
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       composition-fri-bad-with-partial-candidates)
      adversary-initial-state ≤ R + M

```

```

proof (rule
  composition-fri-bad-with-partial-candidates-bound-from-sampled-and-missing)
fix data attacker-state
assume builder:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (composition-fri-bad-with-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq R$ 
by (rule sampled-bound[OF builder])
next
fix data attacker-state
assume builder:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (composition-fri-reachable-missing-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq M$ 
by (rule
  wp-composition-fri-reachable-missing-sampled-bound-from-assignment-obstruction)
  (rule obstruction-bound[OF builder])
qed

```

lemma trace-fri-bad-with-partial-candidates-bound-from-sampled-and-assignment-conflict:

```

fixes R M :: prob
assumes sampled-bound:
   $\bigwedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  wp-event verify-monad
  (trace-fri-bad-with-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq R$ 
and support-imp:
   $\bigwedge$  data attacker-state out.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 

```

```

    out ∈ set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))) ⇒
    trace-fri-sampled-layer-assignment-obstruction
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) out ⇒
    trace-fri-sampled-assignment-conflict
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) out
  and conflict-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (trace-fri-sampled-assignment-conflict
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤ M
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-partial-candidates)
      adversary-initial-state ≤ R + M
  proof (rule
    trace-fri-bad-with-partial-candidates-bound-from-sampled-and-assignment-obstruction)
  fix data attacker-state
  assume builder:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  show wp-event verify-monad
    (trace-fri-bad-with-sampled-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤ R
  by (rule sampled-bound[OF builder])
next
  fix data attacker-state
  assume builder:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  show wp-event verify-monad
    (trace-fri-sampled-layer-assignment-obstruction
      (verifier-state-from-adversary attacker-state

```

```

      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤ M
  by (rule wp-trace-fri-assignment-obstruction-bound-from-conflict)
    (use builder support-imp conflict-bound in blast)+
qed

```

lemma *composition-fri-bad-with-partial-candidates-bound-from-sampled-and-assignment-conflict*:
fixes $R\ M :: \text{prob}$

assumes *sampled-bound*:

$\wedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$

$\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq R$

and *support-imp*:

$\wedge \text{data attacker-state out.}$

$\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$

$\text{out} \in \text{set-dist}$
 $(\text{execute verify-monad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))) \implies$
 $\text{composition-fri-sampled-layer-assignment-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ out} \implies$
 $\text{composition-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ out}$

and *conflict-bound*:

$\wedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$

$\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq M$

shows

$\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$


```

      composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq R + M$ 
proof (rule
  composition-fri-bad-with-partial-candidates-bound-from-sampled-and-assignment-obstruction)
fix data attacker-state
assume builder:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (composition-fri-bad-with-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq R$ 
by (rule sampled-bound[OF builder])
next
fix data attacker-state
assume builder:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (composition-fri-sampled-layer-assignment-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq M$ 
by (rule wp-composition-fri-assignment-obstruction-bound-from-conflict)
  (use builder support-imp conflict-bound in blast)+
qed

end

end

```

```

theory Soundness-FRI-Sampled-Assignment-Chain
imports Soundness-FRI-Sampled-Obstructions
begin

```

Construction of sampled FRI layer-chain witnesses from verifier-local sampled assignment layers. This theory is kept separate from the sampled obstruction-routing layer to keep that layer below the project size threshold.

```

context soundness
begin

```

```

lemma ceil-log-le-power:
  assumes le-power:  $m \leq (2::nat) \wedge N$ 

```

```

  shows ceil-log m ≤ N
proof (cases m ≤ 1)
  case True
  then show ?thesis
    unfolding ceil-log-def by simp
next
  case False
  then have m-gt: 1 < m
    by simp
  then have m-pred-pos: 0 < m - 1
    by simp
  have m-pred-lt: m - 1 < (2::nat) ^ N
    using le-power m-gt by linarith
  have floor-less: floor-log (m - 1) < N
proof (rule ccontr)
  assume  $\neg \text{floor-log } (m - 1) < N$ 
  then have N-le: N ≤ floor-log (m - 1)
    by simp
  then have  $(2::nat) ^ N \leq 2 ^ \text{floor-log } (m - 1)$ 
    by simp
  also have  $\dots \leq m - 1$ 
    by (rule floor-log-exp2-le[OF m-pred-pos])
  finally show False
    using m-pred-lt by simp
qed
then show ?thesis
  unfolding ceil-log-def using m-gt by simp
qed

```

lemma *generic-fri-sampled-assignment-fold-at-step:*

assumes *partial:*

generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers

and *no-conflict:*

$\neg$ *generic-fri-sampled-assignment-conflict candidate-table roots*
challenges final-value query-idxs round-layers

and *round-bound: round-idx < length query-idxs*

and *layer-bound: layer-idx < length challenges*

and *raw-bound:*

fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
fri-evidence-layer-len roots layer-idx

and *len-bound:*

*fri-evidence-layer-len roots layer-idx ≤ clength * scale*

and *step:*

fri-layer-step-evidence

(*roots ! layer-idx*)

(*challenges ! layer-idx*)

(*fri-evidence-layer-len roots layer-idx*)

(*fri-evidence-layer-idx roots query-idxs round-idx layer-idx*)

```

    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
    xp xp-path xn xn-path
    (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
    (fri-evidence-next-value roots challenges query-idxs round-idx
      layer-idx xp xn)
    (round-layers ! round-idx ! layer-idx)
  shows
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
      length (generic-fri-sampled-assignment-layers candidate-table roots
        challenges final-value query-idxs round-layers ! Suc layer-idx)
    fri-sampled-table-fold
      (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      (2 ^ layer-idx)
      (fri-canonical-domains (length challenges) ! layer-idx)
      (generic-fri-sampled-assignment-layers candidate-table roots
        challenges final-value query-idxs round-layers ! layer-idx)
      (round-layers ! round-idx ! layer-idx)
      (generic-fri-sampled-assignment-layers candidate-table roots
        challenges final-value query-idxs round-layers ! Suc layer-idx !
        fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  proof -
  let ?layers =
    generic-fri-sampled-assignment-layers candidate-table roots challenges
    final-value query-idxs round-layers
  let ?len = fri-evidence-layer-len roots layer-idx
  let ?raw = fri-evidence-layer-idx roots query-idxs round-idx layer-idx
  let ?next = fri-evidence-next-idx roots query-idxs round-idx layer-idx
  let ?next-value =
    fri-evidence-next-value roots challenges query-idxs round-idx layer-idx
    xp xn
  have roots-len: length challenges = length roots
  by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
  have sibling-bound:
    fri-sibling-index ?len ?raw < clength * scale
  by (rule fri-evidence-sibling-index-bound-from-raw
    [OF raw-bound len-bound])
  have next-bound: ?next < clength * scale
  by (rule fri-evidence-next-idx-bound-from-raw[OF raw-bound len-bound])
  have layer-next-len:
    length (?layers ! Suc layer-idx) = clength * scale
  proof -
  have Suc layer-idx ≤ length challenges
  using layer-bound by simp
  then have ?layers ! Suc layer-idx =
    generic-fri-sampled-assignment-layer candidate-table roots challenges

```

```

      final-value query-idxs round-layers (Suc layer-idx)
    by (rule generic-fri-sampled-assignment-layers-nth)
  then show ?thesis by simp
qed
then show next-bound-layer:
  ?next < length (?layers ! Suc layer-idx)
  using next-bound by simp
have current-match:
  fri-opening-matches-table ?len ?raw (?layers ! layer-idx) xp xn
proof (cases layer-idx = 0)
  case True
  have challenges-nonempty: 0 < length challenges
    using layer-bound by (cases challenges) auto
  have base-match:
    fri-opening-matches-table (fri-evidence-layer-len roots 0)
      (fri-evidence-layer-idx roots query-idxs round-idx 0)
      (?layers ! 0) xp xn
  by (rule generic-fri-sampled-assignment-base-matches-step
      [OF no-conflict challenges-nonempty round-bound])
    (use step True in simp)
  show ?thesis
    using base-match True by simp
next
  case False
  then have layer-nonzero: 0 < layer-idx
    by simp
  show ?thesis
    by (rule generic-fri-sampled-assignment-current-matches-step
        [OF no-conflict layer-nonzero layer-bound round-bound raw-bound
            len-bound sibling-bound step])
qed
have forced:
  generic-fri-round-forced-next-value roots challenges query-idxs
    round-layers round-idx layer-idx ?next-value
  unfolding generic-fri-round-forced-next-value-def
  using step by blast
have next-value:
  ?layers ! Suc layer-idx ! ?next = ?next-value
proof (cases Suc layer-idx < length challenges)
  case True
  show ?thesis
    by (rule generic-fri-sampled-assignment-successor-forced-value
        [OF no-conflict True round-bound next-bound forced])
next
  case False
  have suc-eq: Suc layer-idx = length challenges
    using False layer-bound by simp
  have challenges-nonempty: 0 < length challenges
    using layer-bound by (cases challenges) auto

```

```

have layer-eq: layer-idx = length challenges - 1
  using suc-eq by simp
have forced-last:
  generic-fri-round-forced-next-value roots challenges query-idxs
    round-layers round-idx (length challenges - 1) ?next-value
  using forced layer-eq by simp
have next-bound-last:
  fri-evidence-next-idx roots query-idxs round-idx
    (length challenges - 1) < clength * scale
  using next-bound layer-eq by simp
have final-next:
  ?layers ! length challenges !
    fri-evidence-next-idx roots query-idxs round-idx
    (length challenges - 1) = ?next-value
  apply (rule generic-fri-sampled-final-layer-forced-value)
  using no-conflict challenges-nonempty round-bound next-bound-last forced-last
  by simp-all
show ?thesis
  using final-next layer-eq suc-eq by simp
qed
have chunk:
  fri-layer-opening-chunk ?len xp xp-path xn xn-path
    (round-layers ! round-idx ! layer-idx)
  by (rule fri-layer-step-evidenceD(4)[OF step])
have canonical-layer:
  fri-canonical-domains (length challenges) ! layer-idx =
    fri-canonical-domain-at layer-idx
  by (rule fri-canonical-domains-nth) (use layer-bound in simp)
have canonical-len:
  length (fri-canonical-domain-at layer-idx) = ?len
  unfolding fri-canonical-domain-at-length fri-evidence-layer-len-def
  using roots-len fri-layer-lengths-nth-div
    [of layer-idx length roots clength * scale] layer-bound
  by simp
have dom-row:
  fri-canonical-domains (length challenges) ! layer-idx ! ?raw =
    (h ^ ?raw * shift) ^ (2 ^ layer-idx)
  unfolding canonical-layer
  by (rule fri-canonical-domain-at-nth)
    (use raw-bound canonical-len in simp)
have fold:
  fri-sampled-table-fold
    (challenges ! layer-idx) ?len ?raw (2 ^ layer-idx)
    (fri-canonical-domains (length challenges) ! layer-idx)
    (?layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    ?next-value
  by (rule fri-layer-step-evidence-sampled-table-fold-if-matching-chunk)
    [OF step dom-row chunk current-match]

```

```

then show
  fri-sampled-table-fold
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-canonical-domains (length challenges) ! layer-idx)
    (?layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (?layers ! Suc layer-idx ! ?next)
  using next-value by simp
qed

lemma generic-fri-sampled-assignment-layer-chain-evidence:
  assumes partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
      roots challenges final-value query-idxs round-layers
  and no-conflict:
    ¬ generic-fri-sampled-assignment-conflict candidate-table roots
      challenges final-value query-idxs round-layers
  and challenges-nonempty: 0 < length challenges
  and query-bounds:
    ∧ round-idx. round-idx < length query-idxs ⇒
      query-idxs ! round-idx < clength * scale
  and eval-power: clength * scale = 2 ^ N
  and rounds-bound: length challenges ≤ N
  shows
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
      degree-bound roots challenges final-value query-idxs round-layers
      (fri-canonical-domains (length challenges))
      (generic-fri-sampled-assignment-layers candidate-table roots
        challenges final-value query-idxs round-layers)
  proof –
    let ?doms = fri-canonical-domains (length challenges)
    let ?layers =
      generic-fri-sampled-assignment-layers candidate-table roots challenges
        final-value query-idxs round-layers
    have doms-len: length ?doms = Suc (length challenges)
      by (rule fri-canonical-domains-length)
    have layers-len: length ?layers = Suc (length challenges)
      by (rule generic-fri-sampled-assignment-layers-length)
    have doms0: ?doms ! 0 = eval-domain
      by (rule fri-canonical-domains-first)
    have layers0: ?layers ! 0 = candidate-table
      by simp
    have final:
      fri-final-constant-consistent (?layers ! length challenges)
        final-value
      by (rule generic-fri-sampled-final-assignment-layer-consistent

```

[*OF challenges-nonempty*])

have *samples*:

$\wedge$ *round-idx layer-idx*.

round-idx < length query-idxs $\implies$

layer-idx < length challenges $\implies$

fri-evidence-next-idx roots query-idxs round-idx layer-idx <

length (?layers ! Suc layer-idx) $\wedge$

fri-sampled-table-fold

(challenges ! layer-idx)

(fri-evidence-layer-len roots layer-idx)

(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)

(2 ^ layer-idx)

(?doms ! layer-idx)

(?layers ! layer-idx)

(round-layers ! round-idx ! layer-idx)

(?layers ! Suc layer-idx !

fri-evidence-next-idx roots query-idxs round-idx layer-idx)

proof –

fix *round-idx layer-idx*

assume *round-bound: round-idx < length query-idxs*

and *layer-bound: layer-idx < length challenges*

have *roots-len: length challenges = length roots*

by (*rule generic-fri-partial-evidence-shapes*(2)[*OF partial*])

have *all-layers*:

fri-all-round-layer-evidence roots challenges query-idxs round-layers

by (*rule generic-fri-partial-evidence-shapes*(3)[*OF partial*])

have *layer-root-bound: layer-idx < length roots*

using *layer-bound roots-len* **by** *simp*

have *round-layer*:

fri-round-layer-evidence roots challenges query-idxs round-idx

layer-idx round-layers

by (*rule fri-all-round-layer-evidenceD*

[*OF all-layers round-bound layer-root-bound*])

then obtain *xp xp-path xn xn-path* **where** *step*:

fri-layer-step-evidence

(roots ! layer-idx)

(challenges ! layer-idx)

(fri-evidence-layer-len roots layer-idx)

(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)

(2 ^ layer-idx)

(fri-sibling-index (fri-evidence-layer-len roots layer-idx)

(fri-evidence-layer-idx roots query-idxs round-idx layer-idx))

xp xp-path xn xn-path

(fri-evidence-next-idx roots query-idxs round-idx layer-idx)

(fri-evidence-next-value roots challenges query-idxs round-idx

layer-idx xp xn)

(round-layers ! round-idx ! layer-idx)

by (*elim fri-round-layer-evidence-compactE*)

have *raw-bound*:

```

    fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
      fri-evidence-layer-len roots layer-idx
  by (rule fri-evidence-layer-idx-bound-from-query-bound
      [OF partial query-bounds[OF round-bound] refl layer-bound
       eval-power rounds-bound])
  have len-bound:
    fri-evidence-layer-len roots layer-idx ≤ clength * scale
  by (rule fri-evidence-layer-len-le-eval-domain
      [OF partial layer-bound eval-power])
  show
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
      length (?layers ! Suc layer-idx) ∧
    fri-sampled-table-fold
      (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      (2 ^ layer-idx)
      (?doms ! layer-idx)
      (?layers ! layer-idx)
      (round-layers ! round-idx ! layer-idx)
      (?layers ! Suc layer-idx !
        fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  by (rule conjI)
    (rule generic-fri-sampled-assignment-fold-at-step
      [OF partial no-conflict round-bound layer-bound raw-bound
       len-bound step])+
qed
show ?thesis
  unfolding generic-fri-sampled-layer-chain-evidence-def
  using partial doms-len layers-len doms0 layers0 final samples
  by blast
qed

lemma generic-fri-sampled-assignment-layer-chain-evidence-zero-round:
  assumes partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
  and challenges-empty: length challenges = 0
  and final:
    fri-final-constant-consistent candidate-table final-value
  shows
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    (fri-canonical-domains (length challenges))
    (generic-fri-sampled-assignment-layers candidate-table roots
     challenges final-value query-idxs round-layers)
proof -
  let ?doms = fri-canonical-domains (length challenges)
  let ?layers =

```



```

    generic-fri-sampled-assignment-layers candidate-table roots challenges
    final-value query-idxs round-layers
  have doms-len: length ?doms = Suc (length challenges)
    by (rule fri-canonical-domains-length)
  have layers-len: length ?layers = Suc (length challenges)
    by (rule generic-fri-sampled-assignment-layers-length)
  have doms0: ?doms ! 0 = eval-domain
    by (rule fri-canonical-domains-first)
  have layers0: ?layers ! 0 = candidate-table
    by simp
  have final-layer:
    fri-final-constant-consistent (?layers ! length challenges)
    final-value
    using challenges-empty final by simp
  have samples:
     $\bigwedge$ round-idx layer-idx.
    round-idx < length query-idxs  $\implies$ 
    layer-idx < length challenges  $\implies$ 
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
      length (?layers ! Suc layer-idx)  $\wedge$ 
    fri-sampled-table-fold
      (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      ( $2^{\text{layer-idx}}$ )
      (?doms ! layer-idx)
      (?layers ! layer-idx)
      (round-layers ! round-idx ! layer-idx)
      (?layers ! Suc layer-idx !
        fri-evidence-next-idx roots query-idxs round-idx layer-idx)
    using challenges-empty by simp
  show ?thesis
    unfolding generic-fri-sampled-layer-chain-evidence-def
    using partial doms-len layers-len doms0 layers0 final-layer samples
    by blast
qed

```

```

lemma generic-fri-sampled-layer-assignment-obstruction-imp-conflict:
  assumes obstruction:
    generic-fri-sampled-layer-assignment-obstruction low-degree
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers
  and challenges-nonempty: 0 < length challenges
  and query-bounds:
     $\bigwedge$ round-idx. round-idx < length query-idxs  $\implies$ 
      query-idxs ! round-idx < clength * scale
  and eval-power: clength * scale =  $2^N$ 
  and rounds-bound: length challenges  $\leq N$ 
  shows

```

generic-fri-sampled-assignment-conflict candidate-table roots
challenges final-value query-idxs round-layers
proof (rule *ccontr*)
assume *no-conflict*:
 $\neg$ *generic-fri-sampled-assignment-conflict candidate-table roots*
challenges final-value query-idxs round-layers
have *partial*:
generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
by (rule *generic-fri-sampled-layer-assignment-obstructionD*(1)
[*OF obstruction*])
have *no-chain*:
 $\bigwedge$ *doms layers*. $\neg$
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers
by (rule *generic-fri-sampled-layer-assignment-obstructionD*(2)
[*OF obstruction*])
have *chain*:
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
(*fri-canonical-domains* (*length challenges*))
(*generic-fri-sampled-assignment-layers candidate-table roots*
challenges final-value query-idxs round-layers)
by (rule *generic-fri-sampled-assignment-layer-chain-evidence*
[*OF partial no-conflict challenges-nonempty query-bounds*
eval-power rounds-bound])
show *False*
using *no-chain*[*of fri-canonical-domains* (*length challenges*)
generic-fri-sampled-assignment-layers candidate-table roots
challenges final-value query-idxs round-layers] *chain*
by *blast*
qed

lemma *generic-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final*:
assumes *obstruction*:
generic-fri-sampled-layer-assignment-obstruction low-degree
candidate-table degree-bound roots challenges final-value query-idxs
round-layers
and *query-bounds*:
 $\bigwedge$ *round-idx*. *round-idx* < *length query-idxs* $\implies$
query-idxs ! *round-idx* < *clength* * *scale*
and *eval-power*: *clength* * *scale* = 2^N
and *rounds-bound*: *length challenges* $\leq N$
shows
generic-fri-sampled-assignment-conflict candidate-table roots
challenges final-value query-idxs round-layers $\vee$
length challenges = 0 $\wedge$
 $\neg$ *fri-final-constant-consistent candidate-table final-value*

```

proof (cases 0 < length challenges)
  case True
  then show ?thesis
    by (intro disjI1
      generic-fri-sampled-layer-assignment-obstruction-imp-conflict
      [OF obstruction - query-bounds eval-power rounds-bound])
next
  case False
  then have challenges-empty: length challenges = 0
    by simp
  have partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
  by (rule generic-fri-sampled-layer-assignment-obstructionD(1)
    [OF obstruction])
  have no-chain:
     $\bigwedge$ doms layers.  $\neg$ 
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers
  by (rule generic-fri-sampled-layer-assignment-obstructionD(2)
    [OF obstruction])
  show ?thesis
proof (cases fri-final-constant-consistent candidate-table final-value)
  case True
  have chain:
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    (fri-canonical-domains (length challenges))
    (generic-fri-sampled-assignment-layers candidate-table roots
    challenges final-value query-idxs round-layers)
  by (rule generic-fri-sampled-assignment-layer-chain-evidence-zero-round
    [OF partial challenges-empty True])
  then show ?thesis
    using no-chain by blast
next
  case False
  then show ?thesis
    using challenges-empty by blast
qed
qed

```

definition trace-fri-zero-round-final-obstruction

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-zero-round-final-obstruction } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$

composition-round-layers fr candidate-query-idxs trace-openings
trace-table.
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table $\wedge$
length trace-bs = 0 $\wedge$
 $\neg$ *fri-final-constant-consistent trace-table trace-final*)

definition *composition-fri-zero-round-final-obstruction*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-zero-round-final-obstruction s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table.
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
length composition-bs = 0 $\wedge$
 $\neg$ *fri-final-constant-consistent composition-table composition-final*)

lemma *trace-fri-sampled-layer-assignment-obstruction-imp-conflict-if-nonempty:*

assumes *obstruction: trace-fri-sampled-layer-assignment-obstruction s out*

and *challenges-nonempty:*

$\wedge$ *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table.
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table $\implies$
 $0 < \text{length trace-bs}$

shows *trace-fri-sampled-assignment-conflict s out*

proof (rule *trace-fri-sampled-layer-assignment-obstructionE[OF obstruction]*)

fix *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings

```

    trace-table
assume evidence:
    trace-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
assume obstruction-generic:
    generic-fri-sampled-layer-assignment-obstruction
    trace-table-low-degree trace-table (clength - 1) trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers
obtain N where eval-power: clength * scale =  $2^N$ 
    using eval-domain-length-power by blast
have trace-rounds-le: length trace-bs ≤ N
proof -
    have fri-openings:
        accepted-fri-opening-transcript s out trace-roots trace-bs
        trace-final dg composition-roots composition-bs composition-final
        fri-query-idxs trace-round-layers composition-round-layers
        by (rule trace-fri-partial-candidate-opening-evidenceD(1)
            [OF evidence])
    have len: length trace-bs = ceil-log clength
        using accepted-fri-opening-transcript-shapes(1,2)[OF fri-openings]
        by simp
    have clength ≤ length * scale
        using scale-pos by simp
    also have ... =  $2^N$ 
        by (rule eval-power)
    finally have ceil-log clength ≤ N
        by (rule ceil-log-le-power)
    then show ?thesis
        using len by simp
qed
have query-bounds:
     $\bigwedge \text{round-idx. round-idx} < \text{length fri-query-idxs} \implies$ 
     $\text{fri-query-idxs} ! \text{round-idx} < \text{clength} * \text{scale}$ 
proof -
    fix round-idx
    assume round-bound: round-idx < length fri-query-idxs
    have fri-query-idxs ! round-idx ∈ set fri-query-idxs
        using round-bound by simp
    then show fri-query-idxs ! round-idx < clength * scale
        by (rule trace-fri-partial-candidate-opening-evidence-query-idx-bound
            [OF evidence])
qed
have conflict:
    generic-fri-sampled-assignment-conflict trace-table trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers
    by (rule generic-fri-sampled-layer-assignment-obstruction-imp-conflict

```

```

    [OF obstruction-generic challenges-nonempty[OF evidence]
      query-bounds eval-power trace-rounds-le])
show ?thesis
  unfolding trace-fri-sampled-assignment-conflict-def
  using evidence conflict by blast
qed

lemma composition-fri-sampled-layer-assignment-obstruction-imp-conflict-if-nonempty:
  assumes support:
    out ∈ set-dist (execute verify-monad s)
  and obstruction:
    composition-fri-sampled-layer-assignment-obstruction s out
  and challenges-nonempty:
    ∧ trace-roots trace-bs trace-final fri-dg opening-composition-roots
      composition-bs composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr f-fri-roots f-final as dg
      composition-fri-roots final trace-query-idxs trace-openings
      composition-query-idxs composition-openings trace-table
      composition-table.
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table ⇒
    0 < length composition-bs
  shows composition-fri-sampled-assignment-conflict s out
proof (rule composition-fri-sampled-layer-assignment-obstructionE
  [OF obstruction])
  fix trace-roots trace-bs trace-final fri-dg opening-composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr f-fri-roots f-final as dg
  composition-fri-roots final trace-query-idxs trace-openings
  composition-query-idxs composition-openings trace-table composition-table
  assume evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  assume obstruction-generic:
    generic-fri-sampled-layer-assignment-obstruction
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
  obtain N where eval-power: clength * scale = 2 ^ N

```

```

using eval-domain-length-power by blast
have fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg opening-composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers
by (rule composition-fri-partial-candidate-opening-evidenceD(1)
  [OF evidence])
obtain result final-state where out-eq: out = Some (result, final-state)
  using fri-openings unfolding accepted-fri-opening-transcript-def by blast
have outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
  using support out-eq by simp
have challenges:
  accepted-fri-challenges s (Some (result, final-state))
  trace-bs fri-dg composition-bs
using accepted-fri-opening-transcript-challenges[OF fri-openings]
  out-eq by simp
have degree-bound: to-nat fri-dg ≤ maxDegree
  by (rule verify-monad-accepted-fri-challenges-degree-bound
  [OF outcome challenges])
have composition-rounds-le: length composition-bs ≤ N
proof –
  have len: length composition-bs = ceil-log (to-nat fri-dg + 1)
    using accepted-fri-opening-transcript-shapes(3,4)[OF fri-openings]
    by simp
  have to-nat fri-dg + 1 ≤ clength * scale
    using degree-bound maxDegree-less-eval-domain by linarith
  also have ... = 2 ^ N
    by (rule eval-power)
  finally have ceil-log (to-nat fri-dg + 1) ≤ N
    by (rule ceil-log-le-power)
  then show ?thesis
    using len by simp
qed
have query-bounds:
   $\bigwedge \text{round-idx. round-idx} < \text{length fri-query-idxs} \implies$ 
   $\text{fri-query-idxs} ! \text{round-idx} < \text{clength} * \text{scale}$ 
proof –
  fix round-idx
  assume round-bound: round-idx < length fri-query-idxs
  have fri-query-idxs ! round-idx ∈ set fri-query-idxs
    using round-bound by simp
  then show fri-query-idxs ! round-idx < clength * scale
    by (rule composition-fri-partial-candidate-opening-evidence-query-idx-bound
    [OF evidence])
qed
have conflict:
  generic-fri-sampled-assignment-conflict composition-table

```

```

    opening-composition-roots composition-bs composition-final
    fri-query-idxs composition-round-layers
  by (rule generic-fri-sampled-layer-assignment-obstruction-imp-conflict
    [OF obstruction-generic challenges-nonempty[OF evidence]
      query-bounds eval-power composition-rounds-le])
  show ?thesis
  unfolding composition-fri-sampled-assignment-conflict-def
  apply (intro exI conjI)
  apply (rule evidence)
  apply (rule conflict)
  done
qed

lemma trace-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final:
  assumes obstruction: trace-fri-sampled-layer-assignment-obstruction s out
  shows
    trace-fri-sampled-assignment-conflict s out  $\vee$ 
    trace-fri-zero-round-final-obstruction s out
  proof (rule trace-fri-sampled-layer-assignment-obstructionE[OF obstruction])
    fix trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr candidate-query-idxs trace-openings
      trace-table
    assume evidence:
      trace-fri-partial-candidate-opening-evidence s out trace-roots
      trace-bs trace-final dg composition-roots composition-bs
      composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr candidate-query-idxs trace-openings
      trace-table
    assume obstruction-generic:
      generic-fri-sampled-layer-assignment-obstruction
      trace-table-low-degree trace-table (clength - 1) trace-roots
      trace-bs trace-final fri-query-idxs trace-round-layers
    obtain N where eval-power: clength * scale =  $2^N$ 
    using eval-domain-length-power by blast
    have trace-rounds-le: length trace-bs  $\leq$  N
    proof -
      have fri-openings:
        accepted-fri-opening-transcript s out trace-roots trace-bs
        trace-final dg composition-roots composition-bs composition-final
        fri-query-idxs trace-round-layers composition-round-layers
      by (rule trace-fri-partial-candidate-opening-evidenceD(1)
        [OF evidence])
    have len: length trace-bs = ceil-log clength
      using accepted-fri-opening-transcript-shapes(1,2)[OF fri-openings]
      by simp
    have clength  $\leq$  clength * scale
      using scale-pos by simp
    also have ... =  $2^N$ 
  end

```



```

    by (rule eval-power)
  finally have  $\text{ceil-log } \text{clength} \leq N$ 
    by (rule ceil-log-le-power)
  then show ?thesis
    using len by simp
qed
have query-bounds:
 $\bigwedge \text{round-idx}. \text{round-idx} < \text{length } \text{fri-query-idxs} \implies$ 
 $\text{fri-query-idxs} ! \text{round-idx} < \text{clength} * \text{scale}$ 
proof -
  fix round-idx
  assume round-bound:  $\text{round-idx} < \text{length } \text{fri-query-idxs}$ 
  have  $\text{fri-query-idxs} ! \text{round-idx} \in \text{set } \text{fri-query-idxs}$ 
    using round-bound by simp
  then show  $\text{fri-query-idxs} ! \text{round-idx} < \text{clength} * \text{scale}$ 
    by (rule trace-fri-partial-candidate-opening-evidence-query-idx-bound
        [OF evidence])
qed
have generic:
  generic-fri-sampled-assignment-conflict trace-table trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers  $\vee$ 
     $\text{length } \text{trace-bs} = 0 \wedge$ 
     $\neg \text{fri-final-constant-consistent } \text{trace-table } \text{trace-final}$ 
  by (rule
    generic-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final
    [OF obstruction-generic query-bounds eval-power trace-rounds-le])
  then show ?thesis
proof
  assume conflict:
    generic-fri-sampled-assignment-conflict trace-table trace-roots
      trace-bs trace-final fri-query-idxs trace-round-layers
  then show ?thesis
    unfolding trace-fri-sampled-assignment-conflict-def
    using evidence by blast
next
  assume zero:
     $\text{length } \text{trace-bs} = 0 \wedge$ 
     $\neg \text{fri-final-constant-consistent } \text{trace-table } \text{trace-final}$ 
  then show ?thesis
    unfolding trace-fri-zero-round-final-obstruction-def
    using evidence by blast
qed
qed

lemma composition-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final:
  assumes support:
     $\text{out} \in \text{set-dist } (\text{execute } \text{verify-monad } s)$ 
  and obstruction:
    composition-fri-sampled-layer-assignment-obstruction  $s$  out

```

shows

composition-fri-sampled-assignment-conflict s out $\vee$
composition-fri-zero-round-final-obstruction s out

proof (rule *composition-fri-sampled-layer-assignment-obstructionE*
 $[OF\ obstruction]$)

fix *trace-roots trace-bs trace-final fri-dg opening-composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table composition-table

assume *evidence*:

composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table

assume *obstruction-generic*:

generic-fri-sampled-layer-assignment-obstruction
 $(composition-table-low-degree\ (to-nat\ fri-dg))\ composition-table$
 $(to-nat\ fri-dg)\ opening-composition-roots\ composition-bs$
composition-final fri-query-idxs composition-round-layers

obtain *N* **where** *eval-power*: $length * scale = 2^N$

using *eval-domain-length-power* **by** *blast*

have *fri-openings*:

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers

by (rule *composition-fri-partial-candidate-opening-evidenceD(1)*
 $[OF\ evidence]$)

obtain *result final-state* **where** *out-eq*: *out* = *Some (result, final-state)*

using *fri-openings unfolding accepted-fri-opening-transcript-def* **by** *blast*

have *outcome*:

Some (result, final-state) ∈ set-dist (execute verify-monad s)
using *support out-eq* **by** *simp*

have *challenges*:

accepted-fri-challenges s (Some (result, final-state))
trace-bs fri-dg composition-bs
using *accepted-fri-opening-transcript-challenges[OF fri-openings]*
out-eq **by** *simp*

have *degree-bound*: $to-nat\ fri-dg \leq maxDegree$

by (rule *verify-monad-accepted-fri-challenges-degree-bound*
 $[OF\ outcome\ challenges]$)

have *composition-rounds-le*: $length\ composition-bs \leq N$

proof –

have *len*: $length\ composition-bs = ceil-log\ (to-nat\ fri-dg + 1)$
using *accepted-fri-opening-transcript-shapes(3,4)[OF fri-openings]*

```

    by simp
  have to-nat fri-dg + 1 ≤ clength * scale
    using degree-bound maxDegree-less-eval-domain by linarith
  also have ... = 2 ^ N
    by (rule eval-power)
  finally have ceil-log (to-nat fri-dg + 1) ≤ N
    by (rule ceil-log-le-power)
  then show ?thesis
    using len by simp
qed
have query-bounds:
  ∧ round-idx. round-idx < length fri-query-idxs ⇒
    fri-query-idxs ! round-idx < clength * scale
proof -
  fix round-idx
  assume round-bound: round-idx < length fri-query-idxs
  have fri-query-idxs ! round-idx ∈ set fri-query-idxs
    using round-bound by simp
  then show fri-query-idxs ! round-idx < clength * scale
    by (rule composition-fri-partial-candidate-opening-evidence-query-idx-bound
        [OF evidence])
qed
have generic:
  generic-fri-sampled-assignment-conflict composition-table
    opening-composition-roots composition-bs composition-final
    fri-query-idxs composition-round-layers ∨
  length composition-bs = 0 ∧
  ¬ fri-final-constant-consistent composition-table
    composition-final
  by (rule
    generic-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final
    [OF obstruction-generic query-bounds eval-power
      composition-rounds-le])
then show ?thesis
proof
  assume conflict:
    generic-fri-sampled-assignment-conflict composition-table
      opening-composition-roots composition-bs composition-final
      fri-query-idxs composition-round-layers
  then show ?thesis
    unfolding composition-fri-sampled-assignment-conflict-def
    apply (intro disjI1 exI conjI)
    apply (rule evidence)
    apply assumption
  done
next
  assume zero:
    length composition-bs = 0 ∧
    ¬ fri-final-constant-consistent composition-table

```

```

      composition-final
    then show ?thesis
      unfolding composition-fri-zero-round-final-obstruction-def
      apply (intro disjI2 exI conjI)
      apply (rule evidence)
      using zero
      apply simp-all
    done
  qed
qed

end

end

```

```

theory Soundness-FRI-Zero-Round-Target
  imports Soundness-FRI-Sampled-Assignment-Chain
begin

```

Query-style target events for the zero-round FRI branch.

If no FRI fold round is present, sampled fold equations cannot produce a successor layer. Acceptance can only test that sampled base-layer entries agree with the final value. This theory factors the corresponding target set without changing the protocol or the public soundness interface.

```

context soundness
begin

```

```

definition fri-final-value-agreement-set :: 'f list  $\Rightarrow$  'f  $\Rightarrow$  nat set
  where
    fri-final-value-agreement-set table final =
      {idx  $\in$  query-sample-space. idx < length table  $\wedge$  table ! idx = final}

```

```

definition fri-zero-round-final-value-target-hit
  :: 'f list  $\Rightarrow$  'f  $\Rightarrow$  nat list  $\Rightarrow$  bool
  where
    fri-zero-round-final-value-target-hit table final query-idxs
       $\longleftrightarrow$ 
       $\neg$  fri-final-constant-consistent table final  $\wedge$ 
      set query-idxs  $\subseteq$  fri-final-value-agreement-set table final

```

```

definition trace-zero-round-final-checked-candidate
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$ 
    'f  $\Rightarrow$  nat list  $\Rightarrow$  'f authenticated-opening list list  $\Rightarrow$ 
    'f list  $\Rightarrow$  'f  $\Rightarrow$  bool
  where
    trace-zero-round-final-checked-candidate s out fr query-idxs
      trace-openings trace-table final  $\longleftrightarrow$ 

```

accepted-with-partial-trace-openings *s* out fr *query-idxs*
trace-openings $\wedge$
partial-trace-table-candidate *trace-table* *trace-openings* $\wedge$
 $(\forall \text{idx} \in \text{set } \text{query-idxs}. \text{idx} \in \text{query-sample-space}) \wedge$
 $(\forall i < \text{rounds}. \text{opening-value } (\text{hd } (\text{trace-openings } ! i)) = \text{final})$

definition *trace-fri-zero-round-checked-final-obstruction*

$:: ('f, 'a) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$

where

trace-fri-zero-round-checked-final-obstruction *s* out $\longleftrightarrow$
 $(\exists \text{trace-roots } \text{trace-bs } \text{trace-final } \text{dg } \text{composition-roots}$
 $\text{composition-bs } \text{composition-final } \text{fri-query-idxs } \text{trace-round-layers}$
 $\text{composition-round-layers fr } \text{trace-openings } \text{trace-table}.$
accepted-fri-opening-transcript *s* out *trace-roots* *trace-bs*
trace-final *dg* *composition-roots* *composition-bs* *composition-final*
fri-query-idxs *trace-round-layers* *composition-round-layers* $\wedge$
 $\text{length } \text{trace-bs} = 0 \wedge$
trace-zero-round-final-checked-candidate *s* out fr *fri-query-idxs*
trace-openings *trace-table* *trace-final* $\wedge$
 $\neg \text{fri-final-constant-consistent } \text{trace-table } \text{trace-final})$

lemma *fri-final-value-agreement-set-subset*:

fri-final-value-agreement-set *table* *final* $\subseteq \text{query-sample-space}$
unfolding *fri-final-value-agreement-set-def* **by** *blast*

lemma *finite-fri-final-value-agreement-set[simp]*:

$\text{finite } (\text{fri-final-value-agreement-set } \text{table } \text{final})$
by (*rule* *finite-subset*[*OF* *fri-final-value-agreement-set-subset*])
simp

lemma *fri-final-value-agreement-setD*:

assumes $\text{idx} \in \text{fri-final-value-agreement-set } \text{table } \text{final}$
shows $\text{idx} \in \text{query-sample-space}$
and $\text{idx} < \text{length } \text{table}$
and $\text{table } ! \text{idx} = \text{final}$
using *assms* **unfolding** *fri-final-value-agreement-set-def* **by** *blast*

lemma *fri-zero-round-final-value-target-hitD*:

assumes *fri-zero-round-final-value-target-hit* *table* *final* *query-idxs*
shows $\neg \text{fri-final-constant-consistent } \text{table } \text{final}$
and $\bigwedge \text{idx}. \text{idx} \in \text{set } \text{query-idxs} \implies$
 $\text{idx} \in \text{fri-final-value-agreement-set } \text{table } \text{final}$
using *assms*
unfolding *fri-zero-round-final-value-target-hit-def* **by** *blast*

lemma *powers-scaled-hd*:

$\text{hd } (\text{powers-scaled } \text{idx}) = \text{idx}$
unfolding *powers-scaled-def*

```

using powers-pos by (simp add: upt-conv-Cons)

lemma trace-zero-round-final-checked-candidate-query-agrees:
  assumes checked:
    trace-zero-round-final-checked-candidate s out fr query-idxs
    trace-openings trace-table final
  and i-bound: i < rounds
  shows
    query-idxs ! i ∈ fri-final-value-agreement-set trace-table final
proof -
  have partial:
    accepted-with-partial-trace-openings s out fr query-idxs trace-openings
  using checked unfolding trace-zero-round-final-checked-candidate-def
  by simp
  have cand:
    partial-trace-table-candidate trace-table trace-openings
  using checked unfolding trace-zero-round-final-checked-candidate-def
  by simp
  have query-sample:
    query-idxs ! i ∈ query-sample-space
  using checked i-bound accepted-with-partial-trace-openings-shapes(1)
  [OF partial]
  unfolding trace-zero-round-final-checked-candidate-def
  by simp
  have idxs:
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
  by (rule accepted-with-partial-trace-openings-shapes(3)
    [OF partial i-bound])
  have openings-nonempty: trace-openings ! i ≠ []
proof -
  have map opening-index (trace-openings ! i) ≠ []
  using idxs powers-pos unfolding powers-scaled-def by simp
  then show ?thesis by auto
qed
  have hd-index:
    opening-index (hd (trace-openings ! i)) = query-idxs ! i
proof -
  have opening-index (hd (trace-openings ! i)) =
    hd (map opening-index (trace-openings ! i))
  using openings-nonempty by (cases trace-openings ! i) auto
  also have ... = hd (powers-scaled (query-idxs ! i))
  using idxs by simp
  also have ... = query-idxs ! i
  by (rule powers-scaled-hd)
  finally show ?thesis .
qed
  have hd-in: hd (trace-openings ! i) ∈ set (trace-openings ! i)
  using openings-nonempty by simp

```

```

have table-value:
  trace-table ! (query-idxs ! i) =
    opening-value (hd (trace-openings ! i))
  using partial-trace-table-candidateD(3)[OF cand i-bound hd-in]
  hd-index by simp
have final:
  opening-value (hd (trace-openings ! i)) = final
  using checked i-bound
  unfolding trace-zero-round-final-checked-candidate-def by simp
have table-len:
  length trace-table = scale * clength
  using cand unfolding partial-trace-table-candidate-def by blast
have idx-bound: query-idxs ! i < length trace-table
  using partial-trace-table-candidateD(2)[OF cand i-bound hd-in]
  hd-index table-len by simp
show ?thesis
  unfolding fri-final-value-agreement-set-def
  using query-sample idx-bound table-value final by simp
qed

lemma trace-zero-round-final-checked-candidate-target-hit:
  assumes checked:
    trace-zero-round-final-checked-candidate s out fr query-idxs
    trace-openings trace-table final
  and not-final:
    ¬ fri-final-constant-consistent trace-table final
  shows
    fri-zero-round-final-value-target-hit trace-table final query-idxs
proof -
  have partial:
    accepted-with-partial-trace-openings s out fr query-idxs trace-openings
  using checked unfolding trace-zero-round-final-checked-candidate-def
  by simp
have len-query: length query-idxs = rounds
  by (rule accepted-with-partial-trace-openings-shapes(1)[OF partial])
have subset:
  set query-idxs ⊆ fri-final-value-agreement-set trace-table final
proof
  fix idx
  assume idx-in: idx ∈ set query-idxs
  then obtain i where i-bound: i < length query-idxs
    and idx-eq: idx = query-idxs ! i
    by (auto simp: in-set-conv-nth)
  then have i < rounds
    using len-query by simp
  then show idx ∈ fri-final-value-agreement-set trace-table final
    using idx-eq
    by (simp add:
      trace-zero-round-final-checked-candidate-query-agrees[OF checked])

```

```

qed
show ?thesis
  unfolding fri-zero-round-final-value-target-hit-def
  using not-final subset by simp
qed

lemma trace-fri-zero-round-checked-final-obstructionE:
  assumes trace-fri-zero-round-checked-final-obstruction s out
  obtains trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr trace-openings trace-table
  where
    accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final dg composition-roots composition-bs composition-final
      fri-query-idxs trace-round-layers composition-round-layers
    length trace-bs = 0
    trace-zero-round-final-checked-candidate s out fr fri-query-idxs
      trace-openings trace-table trace-final
     $\neg$  fri-final-constant-consistent trace-table trace-final
  using assms
  unfolding trace-fri-zero-round-checked-final-obstruction-def
  by blast

lemma trace-fri-zero-round-checked-final-obstruction-target-hit:
  assumes trace-fri-zero-round-checked-final-obstruction s out
  obtains trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr trace-openings trace-table
  where
    accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final dg composition-roots composition-bs composition-final
      fri-query-idxs trace-round-layers composition-round-layers
    length trace-bs = 0
    trace-zero-round-final-checked-candidate s out fr fri-query-idxs
      trace-openings trace-table trace-final
     $\neg$  fri-final-constant-consistent trace-table trace-final
    fri-zero-round-final-value-target-hit trace-table trace-final
    fri-query-idxs
  proof (rule trace-fri-zero-round-checked-final-obstructionE[OF assms])
    fix trace-roots trace-bs trace-final dg composition-roots composition-bs
      composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr trace-openings trace-table
    assume fri-openings:
      accepted-fri-opening-transcript s out trace-roots trace-bs
        trace-final dg composition-roots composition-bs composition-final
        fri-query-idxs trace-round-layers composition-round-layers
    and rounds-empty: length trace-bs = 0
    and checked:
      trace-zero-round-final-checked-candidate s out fr fri-query-idxs

```



```

    trace-openings trace-table trace-final
  and not-final:
    ¬ fri-final-constant-consistent trace-table trace-final
  have target:
    fri-zero-round-final-value-target-hit trace-table trace-final
    fri-query-idxs
  by (rule trace-zero-round-final-checked-candidate-target-hit
    [OF checked not-final])
  show ?thesis
  by (rule that[OF fri-openings rounds-empty checked not-final target])
qed

```

lemma *fri-final-value-agreement-set-card-le:*
card (fri-final-value-agreement-set table final) ≤
query-sample-space-size

```

proof —
  have card (fri-final-value-agreement-set table final) ≤
    card query-sample-space
  by (rule card-mono[OF finite-query-sample-space
    fri-final-value-agreement-set-subset])
  then show ?thesis by simp
qed

```

lemma *fri-final-value-agreement-set-card-lt-if-query-sample-disagrees:*

```

assumes idx-sample: idx ∈ query-sample-space
and idx-bound: idx < length table
and neq: table ! idx ≠ final
shows card (fri-final-value-agreement-set table final) <
query-sample-space-size

```

```

proof —
  have proper:
    fri-final-value-agreement-set table final ⊂ query-sample-space
proof
  show fri-final-value-agreement-set table final ⊆ query-sample-space
    by (rule fri-final-value-agreement-set-subset)
next
  show fri-final-value-agreement-set table final ≠ query-sample-space
proof
  assume eq: fri-final-value-agreement-set table final =
    query-sample-space
  then have idx ∈ fri-final-value-agreement-set table final
    using idx-sample by simp
  then show False
    using fri-final-value-agreement-setD(3) neq by blast
qed
qed
  have card (fri-final-value-agreement-set table final) <
    card query-sample-space
  by (rule psubset-card-mono[OF finite-query-sample-space proper])

```

then show ?thesis by simp
qed

lemma *fri-final-value-agreement-set-all-query-sample-iff*:
assumes *table-covers-query-sample*:
 $\bigwedge idx. idx \in \text{query-sample-space} \implies idx < \text{length table}$
shows
 $\text{fri-final-value-agreement-set table final} = \text{query-sample-space}$
 $\iff$
 $(\forall idx \in \text{query-sample-space}. \text{table} ! idx = \text{final})$
unfolding *fri-final-value-agreement-set-def*
using *table-covers-query-sample* **by** *blast*

lemma *fri-final-constant-consistent-if-query-sample-is-whole-table*:
assumes *table-len*: $\text{length table} = \text{query-sample-space-size}$
and *all-query*: $\forall idx \in \text{query-sample-space}. \text{table} ! idx = \text{final}$
shows *fri-final-constant-consistent table final*
proof –
have *all-idx*: $\bigwedge idx. idx < \text{length table} \implies \text{table} ! idx = \text{final}$
proof –
fix *idx*
assume *idx-bound*: $idx < \text{length table}$
then have $idx \in \text{query-sample-space}$
using *table-len* **unfolding** *query-sample-space-def* **by** *simp*
then show $\text{table} ! idx = \text{final}$
using *all-query* **by** *simp*
qed
show ?thesis
unfolding *fri-final-constant-consistent-def*
using *all-idx* **by** (auto simp: *in-set-conv-nth*)
qed

lemma *fri-final-value-agreement-set-card-lt-if-not-constant-whole-sample*:
assumes *table-len*: $\text{length table} = \text{query-sample-space-size}$
and *not-final*: $\neg \text{fri-final-constant-consistent table final}$
shows $\text{card} (\text{fri-final-value-agreement-set table final}) < \text{query-sample-space-size}$
proof (rule *ccontr*)
assume $\neg \text{card} (\text{fri-final-value-agreement-set table final}) < \text{query-sample-space-size}$
then have *card-eq*:
 $\text{card} (\text{fri-final-value-agreement-set table final}) = \text{query-sample-space-size}$
using *fri-final-value-agreement-set-card-le[of table final]* **by** *linarith*
have *set-eq*:
 $\text{fri-final-value-agreement-set table final} = \text{query-sample-space}$
by (rule *card-subset-eq[OF finite-query-sample-space fri-final-value-agreement-set-subset]*)
 (use *card-eq* **in** *simp*)

have *all-query*:
 $\forall idx \in \text{query-sample-space. table} ! idx = \text{final}$
using *set-eq fri-final-value-agreement-setD(3)* **by** *metis*
have *fri-final-constant-consistent table final*
by (*rule fri-final-constant-consistent-if-query-sample-is-whole-table*
 $[OF \text{ table-len all-query}]$)
then show *False*
using *not-final* **by** *simp*
qed

lemma *trace-zero-round-table-len-equals-query-sample-space*:
assumes *trace-rounds-empty: length trace-bs = 0*
and *fri-openings*:
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers}$
and *cand: partial-trace-table-candidate trace-table trace-openings*
shows $\text{length trace-table} = \text{query-sample-space-size}$
proof –
have *len-trace-bs: length trace-bs = ceil-log clength*
using *accepted-fri-opening-transcript-shapes(1,2)[OF fri-openings]*
by *simp*
have *ceil-zero: ceil-log clength = 0*
using *trace-rounds-empty len-trace-bs* **by** *simp*
have *clength-le-one: clength ≤ 1*
using *ceil-zero unfolding ceil-log-def*
by (*cases clength ≤ 1*) *simp-all*
have *clength-eq: clength = 1*
using *clength-le-one clength-pos* **by** *simp*
have *powers-eq: powers = 1*
using *powers-pos powers-le-clength clength-eq* **by** *simp*
have *table-len: length trace-table = scale * clength*
using *cand unfolding partial-trace-table-candidate-def* **by** *blast*
show *?thesis*
unfolding *query-sample-space-size-def*
using *table-len clength-eq powers-eq* **by** *simp*
qed

lemma *trace-fri-zero-round-checked-final-obstruction-agreement-set-card-lt*:
assumes *obstruction*:
 $\text{trace-fri-zero-round-checked-final-obstruction } s \text{ out}$
obtains *trace-roots trace-bs trace-final dg composition-roots*
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr trace-openings trace-table}$
where
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers}$
 $\text{length trace-bs} = 0$

```

    trace-zero-round-final-checked-candidate s out fr fri-query-idxs
    trace-openings trace-table trace-final
  ¬ fri-final-constant-consistent trace-table trace-final
  fri-zero-round-final-value-target-hit trace-table trace-final
  fri-query-idxs
  card (fri-final-value-agreement-set trace-table trace-final) <
    query-sample-space-size
proof (rule trace-fri-zero-round-checked-final-obstruction-target-hit
  [OF obstruction])
fix trace-roots trace-bs trace-final dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr trace-openings trace-table
assume fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final dg composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers
and rounds-empty: length trace-bs = 0
and checked:
  trace-zero-round-final-checked-candidate s out fr fri-query-idxs
  trace-openings trace-table trace-final
and not-final:
  ¬ fri-final-constant-consistent trace-table trace-final
and target:
  fri-zero-round-final-value-target-hit trace-table trace-final
  fri-query-idxs
have cand:
  partial-trace-table-candidate trace-table trace-openings
using checked unfolding trace-zero-round-final-checked-candidate-def
by simp
have table-len:
  length trace-table = query-sample-space-size
by (rule trace-zero-round-table-len-equals-query-sample-space
  [OF rounds-empty fri-openings cand])
have card-lt:
  card (fri-final-value-agreement-set trace-table trace-final) <
    query-sample-space-size
by (rule fri-final-value-agreement-set-card-lt-if-not-constant-whole-sample
  [OF table-len not-final])
show ?thesis
by (rule that[OF fri-openings rounds-empty checked not-final target
  card-lt])
qed

```

```

lemma trace-fri-zero-round-checked-final-obstruction-first-query-target:
assumes obstruction:
  trace-fri-zero-round-checked-final-obstruction s out
obtains trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr trace-openings trace-table

```

where

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
 $\text{length } \text{trace-bs} = 0$
trace-zero-round-final-checked-candidate s out fr fri-query-idxs
trace-openings trace-table trace-final
 $\neg \text{fri-final-constant-consistent trace-table trace-final}$
fri-query-idxs ! 0 $\in$
fri-final-value-agreement-set trace-table trace-final
 $\text{card } (\text{fri-final-value-agreement-set trace-table trace-final}) <$
 $\text{query-sample-space-size}$

proof (rule *trace-fri-zero-round-checked-final-obstruction-agreement-set-card-lt*
 $[OF \text{ obstruction}]$)

fix *trace-roots trace-bs trace-final dg composition-roots composition-bs*
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-openings trace-table

assume *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers

and *rounds-empty:* $\text{length } \text{trace-bs} = 0$

and *checked:*

trace-zero-round-final-checked-candidate s out fr fri-query-idxs
trace-openings trace-table trace-final

and *not-final:*

$\neg \text{fri-final-constant-consistent trace-table trace-final}$

and *target:*

fri-zero-round-final-value-target-hit trace-table trace-final
fri-query-idxs

and *card-lt:*

$\text{card } (\text{fri-final-value-agreement-set trace-table trace-final}) <$
 $\text{query-sample-space-size}$

have *first-in:*

fri-query-idxs ! 0 $\in$
fri-final-value-agreement-set trace-table trace-final

proof –

have *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings

using *checked unfolding trace-zero-round-final-checked-candidate-def*
by *simp*

have $\text{length } \text{fri-query-idxs} = \text{rounds}$

by (rule *accepted-with-partial-trace-openings-shapes(1)*) $[OF \text{ partial}]$

then have *fri-query-idxs ! 0* $\in \text{set } \text{fri-query-idxs}$

using *rounds-positive* **by** (*simp add: nth-mem*)

then show *?thesis*

by (rule *fri-zero-round-final-value-target-hitD(2)*) $[OF \text{ target}]$

qed

show *?thesis*
by (rule that[*OF fri-openings rounds-empty checked not-final first-in card-lt*])
qed

lemma *trace-zero-round-checked-candidate-not-trace-low-degree*:
assumes *rounds-empty*: $\text{length trace-bs} = 0$
and *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *checked*:
trace-zero-round-final-checked-candidate s out fr fri-query-idxs
trace-openings trace-table trace-final
and *not-final*:
 $\neg$ *fri-final-constant-consistent trace-table trace-final*
shows $\neg$ *trace-table-low-degree trace-table*
proof
assume *low*: *trace-table-low-degree trace-table*
have *len-trace-bs*: $\text{length trace-bs} = \text{ceil-log clength}$
using *accepted-fri-opening-transcript-shapes(1,2)[OF fri-openings]*
by *simp*
have *ceil-zero*: $\text{ceil-log clength} = 0$
using *rounds-empty len-trace-bs* **by** *simp*
have *clength-le-one*: $\text{clength} \leq 1$
using *ceil-zero unfolding ceil-log-def*
by (cases $\text{clength} \leq 1$) *simp-all*
have *clength-eq*: $\text{clength} = 1$
using *clength-le-one clength-pos* **by** *simp*
obtain *f* **where** *deg-f*: $\text{degree } f < \text{clength}$
and *trace-table*: $\text{trace-table} = \text{map } (\text{poly } f) \text{ eval-domain}$
using *low unfolding trace-table-low-degree-def* **by** *blast*
have *deg0*: $\text{degree } f = 0$
using *deg-f clength-eq* **by** *simp*
obtain *a* **where** *f-eq*: $f = [:a:]$
using *degree0-coeffs[OF deg0]* **by** *blast*
have *table-const*:
 $\bigwedge y. y \in \text{set trace-table} \implies y = a$
using *trace-table f-eq* **by** *auto*
have *sample-agrees*:
fri-query-idxs ! 0 $\in$
fri-final-value-agreement-set trace-table trace-final
by (rule *trace-zero-round-final-checked-candidate-query-agrees*
 $[OF \text{checked rounds-positive}]$)
have *idx-bound*: *fri-query-idxs ! 0* $< \text{length trace-table}$
using *fri-final-value-agreement-setD(2)[OF sample-agrees]* .
have *table-at-final*:
 $\text{trace-table ! (fri-query-idxs ! 0)} = \text{trace-final}$
using *fri-final-value-agreement-setD(3)[OF sample-agrees]* .

```

have trace-table ! (fri-query-idxs ! 0) ∈ set trace-table
  using idx-bound by simp
then have a-eq: a = trace-final
  using table-const table-at-final by simp
have fri-final-constant-consistent trace-table trace-final
  unfolding fri-final-constant-consistent-def
  using table-const a-eq by auto
then show False
  using not-final by contradiction
qed

lemma trace-fri-zero-round-checked-final-obstruction-not-trace-low-degree:
  assumes obstruction:
    trace-fri-zero-round-checked-final-obstruction s out
  obtains trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr trace-openings trace-table
  where
    accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final dg composition-roots composition-bs composition-final
      fri-query-idxs trace-round-layers composition-round-layers
    length trace-bs = 0
    trace-zero-round-final-checked-candidate s out fr fri-query-idxs
      trace-openings trace-table trace-final
    ¬ fri-final-constant-consistent trace-table trace-final
    ¬ trace-table-low-degree trace-table
  proof (rule trace-fri-zero-round-checked-final-obstructionE[OF obstruction])
  fix trace-roots trace-bs trace-final dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr trace-openings trace-table
  assume fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final dg composition-roots composition-bs composition-final
      fri-query-idxs trace-round-layers composition-round-layers
  and rounds-empty: length trace-bs = 0
  and checked:
    trace-zero-round-final-checked-candidate s out fr fri-query-idxs
      trace-openings trace-table trace-final
  and not-final:
    ¬ fri-final-constant-consistent trace-table trace-final
  have not-low:
    ¬ trace-table-low-degree trace-table
  by (rule trace-zero-round-checked-candidate-not-trace-low-degree
    [OF rounds-empty fri-openings checked not-final])
  show ?thesis
  by (rule that[OF fri-openings rounds-empty checked not-final not-low])
qed

definition trace-fri-zero-round-checked-first-query-target-hit

```

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  trace-fri-zero-round-checked-first-query-target-hit s out  $\longleftrightarrow$ 
    ( $\exists$  trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr trace-openings trace-table.
      accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final dg composition-roots composition-bs composition-final
      fri-query-idxs trace-round-layers composition-round-layers  $\wedge$ 
      length trace-bs = 0  $\wedge$ 
      trace-zero-round-final-checked-candidate s out fr fri-query-idxs
      trace-openings trace-table trace-final  $\wedge$ 
       $\neg$  fri-final-constant-consistent trace-table trace-final  $\wedge$ 
      fri-query-idxs ! 0  $\in$ 
      fri-final-value-agreement-set trace-table trace-final  $\wedge$ 
      card (fri-final-value-agreement-set trace-table trace-final) <
      query-sample-space-size)

lemma trace-fri-zero-round-checked-final-obstruction-imp-first-query-target-hit:
  assumes trace-fri-zero-round-checked-final-obstruction s out
  shows trace-fri-zero-round-checked-first-query-target-hit s out
proof (rule trace-fri-zero-round-checked-final-obstruction-first-query-target
  [OF assms])
  fix trace-roots trace-bs trace-final dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers composition-round-layers
  fr trace-openings trace-table
  assume fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  and rounds-empty: length trace-bs = 0
  and checked:
    trace-zero-round-final-checked-candidate s out fr fri-query-idxs
    trace-openings trace-table trace-final
  and not-final:
     $\neg$  fri-final-constant-consistent trace-table trace-final
  and first-hit:
    fri-query-idxs ! 0  $\in$ 
    fri-final-value-agreement-set trace-table trace-final
  and card-lt:
    card (fri-final-value-agreement-set trace-table trace-final) <
    query-sample-space-size
  show ?thesis
  unfolding trace-fri-zero-round-checked-first-query-target-hit-def
  by (intro exI conjI)
    (rule fri-openings, rule rounds-empty, rule checked, rule not-final,
    rule first-hit, rule card-lt)
qed

```


lemma *wp-trace-fri-zero-round-checked-final-obstruction-bound-from-first-query-target-hit:*

assumes *target-bound:*

wp-event verify-monad

$(\text{trace-fri-zero-round-checked-first-query-target-hit } s) \ s \leq T$

shows

wp-event verify-monad

$(\text{trace-fri-zero-round-checked-final-obstruction } s) \ s \leq T$

proof –

have *wp-event verify-monad*

$(\text{trace-fri-zero-round-checked-final-obstruction } s) \ s \leq$

wp-event verify-monad

$(\text{trace-fri-zero-round-checked-first-query-target-hit } s) \ s$

by (rule *wp-event-mono*)

(rule

trace-fri-zero-round-checked-final-obstruction-imp-first-query-target-hit)

then show *?thesis*

by (rule *order-trans[OF - target-bound]*)

qed

lemma *trace-fri-zero-round-final-obstruction-agreement-set-card-lt:*

assumes *obstruction: trace-fri-zero-round-final-obstruction s out*

obtains *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr candidate-query-idxs trace-openings

trace-table

where

trace-fri-partial-candidate-opening-evidence s out trace-roots

trace-bs trace-final dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers fr candidate-query-idxs trace-openings

trace-table

length trace-bs = 0

$\neg$ *fri-final-constant-consistent trace-table trace-final*

card (fri-final-value-agreement-set trace-table trace-final) <

query-sample-space-size

proof –

from *obstruction* **obtain** *trace-roots trace-bs trace-final dg*

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr candidate-query-idxs

trace-openings trace-table **where** *evidence:*

trace-fri-partial-candidate-opening-evidence s out trace-roots

trace-bs trace-final dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers fr candidate-query-idxs trace-openings

trace-table

and *rounds-empty: length trace-bs = 0*

and *not-final:*

$\neg$ *fri-final-constant-consistent trace-table trace-final*

unfolding *trace-fri-zero-round-final-obstruction-def* **by** *blast*
have *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
by (rule *trace-fri-partial-candidate-opening-evidenceD(1)*)
[*OF evidence*])
have *partial*:
trace-fri-partial-candidate-evidence s out fr candidate-query-idxs
trace-openings trace-table
by (rule *trace-fri-partial-candidate-opening-evidenceD(2)*)
[*OF evidence*])
have *cand*:
partial-trace-table-candidate trace-table trace-openings
by (rule *trace-fri-partial-candidate-evidenceD(2)*[*OF partial*])
have *table-len*:
length trace-table = query-sample-space-size
by (rule *trace-zero-round-table-len-equals-query-sample-space*)
[*OF rounds-empty fri-openings cand*])
have *card-lt*:
card (fri-final-value-agreement-set trace-table trace-final) <
query-sample-space-size
by (rule *fri-final-value-agreement-set-card-lt-if-not-constant-whole-sample*)
[*OF table-len not-final*])
show *?thesis*
by (rule *that*[*OF evidence rounds-empty not-final card-lt*])
qed

lemma *trace-fri-zero-round-final-obstructionE*:
assumes *trace-fri-zero-round-final-obstruction s out*
obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
where
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
length trace-bs = 0
 $\neg$ *fri-final-constant-consistent trace-table trace-final*
using *assms*
unfolding *trace-fri-zero-round-final-obstruction-def* **by** *blast*

definition *trace-fri-zero-round-candidate-small-agreement-set*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where

$\text{trace-fri-zero-round-candidate-small-agreement-set } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 trace-table.
 $\text{trace-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
 $\text{length trace-bs} = 0 \wedge$
 $\neg \text{fri-final-constant-consistent trace-table trace-final} \wedge$
 $\text{card (fri-final-value-agreement-set trace-table trace-final)} <$
 $\text{query-sample-space-size})$

definition *trace-fri-zero-round-candidate-first-query-target-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-zero-round-candidate-first-query-target-hit } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 trace-table.
 $\text{trace-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
 $\text{length trace-bs} = 0 \wedge$
 $\neg \text{fri-final-constant-consistent trace-table trace-final} \wedge$
 $\text{candidate-query-idxs} ! 0 \in$
 $\text{fri-final-value-agreement-set trace-table trace-final} \wedge$
 $\text{card (fri-final-value-agreement-set trace-table trace-final)} <$
 $\text{query-sample-space-size})$

definition *trace-fri-zero-round-candidate-first-query-alignment-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-zero-round-candidate-first-query-alignment-gap } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 trace-table.
 $\text{trace-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$

$trace-table \wedge$
 $length\ trace-bs = 0 \wedge$
 $\neg fri-final-constant-consistent\ trace-table\ trace-final \wedge$
 $candidate-query-idxs ! 0 \in$
 $fri-final-value-agreement-set\ trace-table\ trace-final \wedge$
 $fri-query-idxs ! 0 \notin$
 $fri-final-value-agreement-set\ trace-table\ trace-final \wedge$
 $card\ (fri-final-value-agreement-set\ trace-table\ trace-final) <$
 $query-sample-space-size)$

definition *trace-fri-zero-round-candidate-first-query-value-gap*
 $:: (f, 'a)\ protocol-channel-scheme \Rightarrow$
 $(unit\ list \times (f, 'a)\ protocol-channel-scheme)\ option \Rightarrow bool$

where

$trace-fri-zero-round-candidate-first-query-value-gap\ s\ out \longleftrightarrow$
 $(\exists\ trace-roots\ trace-bs\ trace-final\ dg\ composition-roots$
 $\quad composition-bs\ composition-final\ fri-query-idxs\ trace-round-layers$
 $\quad composition-round-layers\ fr\ candidate-query-idxs\ trace-openings$
 $\quad trace-table.$
 $trace-fri-partial-candidate-opening-evidence\ s\ out\ trace-roots$
 $\quad trace-bs\ trace-final\ dg\ composition-roots\ composition-bs$
 $\quad composition-final\ fri-query-idxs\ trace-round-layers$
 $\quad composition-round-layers\ fr\ candidate-query-idxs\ trace-openings$
 $\quad trace-table \wedge$
 $length\ trace-bs = 0 \wedge$
 $\neg fri-final-constant-consistent\ trace-table\ trace-final \wedge$
 $candidate-query-idxs ! 0 \notin$
 $fri-final-value-agreement-set\ trace-table\ trace-final \wedge$
 $card\ (fri-final-value-agreement-set\ trace-table\ trace-final) <$
 $query-sample-space-size)$

lemma *trace-fri-zero-round-candidate-small-agreement-setE:*
assumes *trace-fri-zero-round-candidate-small-agreement-set s out*
obtains *trace-roots trace-bs trace-final dg composition-roots*
 $\quad composition-bs\ composition-final\ fri-query-idxs\ trace-round-layers$
 $\quad composition-round-layers\ fr\ candidate-query-idxs\ trace-openings$
 $\quad trace-table$

where

$trace-fri-partial-candidate-opening-evidence\ s\ out\ trace-roots$
 $\quad trace-bs\ trace-final\ dg\ composition-roots\ composition-bs$
 $\quad composition-final\ fri-query-idxs\ trace-round-layers$
 $\quad composition-round-layers\ fr\ candidate-query-idxs\ trace-openings$
 $\quad trace-table$
 $length\ trace-bs = 0$
 $\neg fri-final-constant-consistent\ trace-table\ trace-final$
 $card\ (fri-final-value-agreement-set\ trace-table\ trace-final) <$
 $query-sample-space-size$

using *assms*

unfolding *trace-fri-zero-round-candidate-small-agreement-set-def*

by blast

lemma *trace-fri-zero-round-candidate-first-query-target-hitE*:
assumes *trace-fri-zero-round-candidate-first-query-target-hit s out*
obtains *trace-roots trace-bs trace-final dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table
where
 trace-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table
 length trace-bs = 0
 $\neg$ *fri-final-constant-consistent trace-table trace-final*
 candidate-query-idxs ! 0 ∈
 fri-final-value-agreement-set trace-table trace-final
 card (fri-final-value-agreement-set trace-table trace-final) <
 query-sample-space-size
using *assms*
unfolding *trace-fri-zero-round-candidate-first-query-target-hit-def*
by *blast*

lemma *trace-fri-zero-round-candidate-first-query-alignment-gapE*:
assumes *trace-fri-zero-round-candidate-first-query-alignment-gap s out*
obtains *trace-roots trace-bs trace-final dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table
where
 trace-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table
 length trace-bs = 0
 $\neg$ *fri-final-constant-consistent trace-table trace-final*
 candidate-query-idxs ! 0 ∈
 fri-final-value-agreement-set trace-table trace-final
 fri-query-idxs ! 0 ∉
 fri-final-value-agreement-set trace-table trace-final
 card (fri-final-value-agreement-set trace-table trace-final) <
 query-sample-space-size
using *assms*
unfolding *trace-fri-zero-round-candidate-first-query-alignment-gap-def*
by *blast*

lemma *trace-fri-zero-round-candidate-first-query-value-gapE*:

assumes *trace-fri-zero-round-candidate-first-query-value-gap s out*
obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

where

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

length trace-bs = 0

$\neg$ *fri-final-constant-consistent trace-table trace-final*

candidate-query-idxs ! 0 $\notin$

fri-final-value-agreement-set trace-table trace-final

card (fri-final-value-agreement-set trace-table trace-final) <
query-sample-space-size

using *assms*

unfolding *trace-fri-zero-round-candidate-first-query-value-gap-def*
by *blast*

lemma *trace-fri-zero-round-candidate-first-query-target-hit-imp-small:*

assumes *trace-fri-zero-round-candidate-first-query-target-hit s out*

shows *trace-fri-zero-round-candidate-small-agreement-set s out*

proof (*rule trace-fri-zero-round-candidate-first-query-target-hitE[OF assms]*)

fix *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

assume *evidence:*

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

and *rounds-empty: length trace-bs = 0*

and *not-final:*

$\neg$ *fri-final-constant-consistent trace-table trace-final*

and *card-lt:*

card (fri-final-value-agreement-set trace-table trace-final) <
query-sample-space-size

show *?thesis*

unfolding *trace-fri-zero-round-candidate-small-agreement-set-def*

by (*intro exI conjI*)

(*rule evidence, rule rounds-empty, rule not-final, rule card-lt*)

qed

lemma *trace-fri-zero-round-checked-first-query-target-hit-imp-candidate:*

assumes *trace-fri-zero-round-checked-first-query-target-hit s out*

shows *trace-fri-zero-round-candidate-first-query-target-hit s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-openings trace-table
where *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *rounds-empty*: *length trace-bs = 0*
and *checked*:
trace-zero-round-final-checked-candidate s out fr fri-query-idxs
trace-openings trace-table trace-final
and *not-final*:
 $\neg$ *fri-final-constant-consistent trace-table trace-final*
and *first-hit*:
fri-query-idxs ! 0 $\in$
fri-final-value-agreement-set trace-table trace-final
and *card-lt*:
card (fri-final-value-agreement-set trace-table trace-final) <
query-sample-space-size
unfolding *trace-fri-zero-round-checked-first-query-target-hit-def*
by *blast*
have *partial*:
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
using *checked* **unfolding** *trace-zero-round-final-checked-candidate-def*
by *simp*
have *candidate*:
partial-trace-table-candidate trace-table trace-openings
using *checked* **unfolding** *trace-zero-round-final-checked-candidate-def*
by *simp*
have *not-low*:
 $\neg$ *trace-table-low-degree trace-table*
by (*rule trace-zero-round-checked-candidate-not-trace-low-degree*
[*OF rounds-empty fri-openings checked not-final*])
have *evidence*:
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr fri-query-idxs trace-openings trace-table
unfolding *trace-fri-partial-candidate-opening-evidence-def*
trace-fri-partial-candidate-evidence-def
by (*intro conjI fri-openings partial candidate not-low*)
show *?thesis*
unfolding *trace-fri-zero-round-candidate-first-query-target-hit-def*
by (*intro exI conjI*)
(*rule evidence, rule rounds-empty, rule not-final, rule first-hit,*
rule card-lt)

qed

lemma *trace-fri-zero-round-final-obstruction-imp-candidate-small-agreement-set:*

assumes *trace-fri-zero-round-final-obstruction s out*

shows *trace-fri-zero-round-candidate-small-agreement-set s out*

proof (rule *trace-fri-zero-round-final-obstruction-agreement-set-card-lt*
[*OF assms*])

fix *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

assume *evidence:*

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

and *rounds-empty: length trace-bs = 0*

and *not-final:*

¬ *fri-final-constant-consistent trace-table trace-final*

and *card-lt:*

card (fri-final-value-agreement-set trace-table trace-final) <
query-sample-space-size

show *?thesis*

unfolding *trace-fri-zero-round-candidate-small-agreement-set-def*

by (*intro exI conjI*)

(rule *evidence*, rule *rounds-empty*, rule *not-final*, rule *card-lt*)

qed

lemma *wp-trace-fri-zero-round-final-obstruction-bound-from-candidate-small-agreement-set:*

assumes *small-bound:*

wp-event verify-monad

(*trace-fri-zero-round-candidate-small-agreement-set s*) $s \leq T$

shows

wp-event verify-monad

(*trace-fri-zero-round-final-obstruction s*) $s \leq T$

proof –

have *wp-event verify-monad*

(*trace-fri-zero-round-final-obstruction s*) $s \leq$

wp-event verify-monad

(*trace-fri-zero-round-candidate-small-agreement-set s*) s

by (rule *wp-event-mono*)

(rule

trace-fri-zero-round-final-obstruction-imp-candidate-small-agreement-set)

then show *?thesis*

by (rule *order-trans[OF - small-bound]*)

qed

end

end

theory *Soundness-Query-Transcript-Openings*
imports *Soundness-Query-Opening-Consistency*
begin

Combined query-round extraction lemmas.

The existing execution layer extracts transcript chunks and authenticated Merkle openings separately. This layer keeps the witnesses coupled to the same verifier query-round execution, so downstream transcript-indexed reductions can use verifier-derived query indices.

context *soundness*
begin

lemma *verifier-query-round-program-outcome-with-authenticated-trace-openings:*

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes *outcome:*

$\text{Some } ((), t) \in$

set-dist

$(\text{execute}$

$(\text{verifier-query-round-program } fr\ f\text{-}fl\ f\text{-}final\ \text{as } fl\ final)\ s)$

obtains *raw idx chunk openings where*

$idx = \text{index } (to\text{-}nat\ raw)$

$\text{verifier-query-round-chunk } idx\ (\text{map } snd\ f\text{-}fl)\ (\text{map } snd\ fl)\ chunk$

$P\text{Transcript } s = chunk\ @\ P\text{Transcript } t$

$P\text{State } t = \text{foldl } concat\ (P\text{State } s)\ chunk$

$s \leq t$

$P\text{QueryCounter } t = \text{Suc } (P\text{QueryCounter } s)$

$\text{fmlookup } (\text{HashMap } t)\ (\text{QueryIndexChallenge } (P\text{QueryCounter } s)\ (P\text{State } s)) =$

$\text{Some } raw$

$\text{length } openings = \text{length } (\text{powers-scaled } idx)$

$\text{map } \text{opening-index } openings = \text{powers-scaled } idx$

$\text{partial-authenticated-table } fr\ (\text{scale } * \text{ clength})\ openings\ t$

proof –

from *outcome[unfolded verifier-query-round-program-alt-def]*

obtain *raw s0 where*

rand:

$\text{Some } (raw, s0) \in$

$\text{set-dist } (\text{execute } \text{receive-query-index-challenge } s)$

and *tail:*

$\text{Some } ((), t) \in$

set-dist

$(\text{execute}$

$(\text{verifier-query-round-after-index-program}$

$fr\ f\text{-}fl\ f\text{-}final\ \text{as } fl\ final\ raw)\ s0)$

by *(auto elim!: set-dist-bindE)*

let $?idx = \text{index } (to\text{-}nat\ raw)$

```

from tail obtain fv s1 f-out s2 s3 c-out s4 where
  query-decommit:
    Some (fv, s1) ∈
      set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s0)
  and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some (( ), s3) ∈
      set-dist
        (execute
          (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold
          (?idx, cp-eval as fv (h ^ ?idx * shift), clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some (( ), t) ∈
      set-dist
        (execute (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have rand-res:
  s ≤ s0 ∧
  PState s0 = PState s ∧
  PTranscript s0 = PTranscript s ∧
  fmlookup (HashMap s0)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using receive-query-index-challenge-outcome[OF rand] .
have rand-counter:
  PQueryCounter s0 = Suc (PQueryCounter s)
  using receive-query-index-challenge-counter-outcome[OF rand] by simp
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript ?idx fv query-paths query-chunk
  and tr-s0: PTranscript s0 = query-chunk @ PTranscript s1
  and st-s1: PState s1 = foldl concat (PState s0) query-chunk
  and ext-s0-s1: s0 ≤ s1
  and query-count-s1: PQueryCounter s1 = PQueryCounter s0
  by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  trace-layers:
    fri-layers-transcript (length f-fl) (clength * scale)

```

```

    trace-layer-chunks trace-fri-chunk
  and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
  and st-s2: PState s2 = foldl concat (PState s1) trace-fri-chunk
  and ext-s1-s2: s1 ≤ s2
  and query-count-s2: PQueryCounter s2 = PQueryCounter s1
  by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk
  and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
  and st-s4: PState s4 = foldl concat (PState s3) comp-fri-chunk
  and ext-s3-s4: s3 ≤ s4
  and query-count-s4: PQueryCounter s4 = PQueryCounter s3
  by blast
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
let ?chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have round-chunk:
  verifier-query-round-chunk ?idx (map snd f-fl) (map snd fl) ?chunk
  unfolding verifier-query-round-chunk-def
  apply (rule exI[where x=query-chunk])
  apply (rule exI[where x=trace-fri-chunk])
  apply (rule exI[where x=comp-fri-chunk])
  apply (rule exI[where x=fv])
  apply (rule exI[where x=query-paths])
  apply (rule exI[where x=trace-layer-chunks])
  apply (rule exI[where x=comp-layer-chunks])
  using query-chunk-shape trace-layers comp-layers
  by simp
have tr-t: PTranscript s = ?chunk @ PTranscript t
  using rand-res tr-s0 tr-s1 tr-s3 s3-eq t-eq by simp
have st-t: PState t = foldl concat (PState s) ?chunk
  using rand-res st-s1 st-s2 st-s4 s3-eq t-eq by simp
have ext-s0-t: s0 ≤ t
  using ext-s0-s1 ext-s1-s2 ext-s3-s4 t-eq unfolding s3-eq
  by (meson hash-ext-trans)
have ext-s-t: s ≤ t
  using rand-res ext-s0-t by (meson hash-ext-trans)
have query-count-t: PQueryCounter t = Suc (PQueryCounter s)
  using rand-counter query-count-s1 query-count-s2 query-count-s4
  unfolding s3-eq t-eq by simp

```

```

have lookup-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof -
  let ?key = QueryIndexChallenge (PQueryCounter s) (PState s)
  have fmlookup (HashMap s0) ?key = fmlookup (HashMap t) ?key
    using ext-s0-t rand-res
    unfolding less-eq-hash-ext-def less-eq-fmap-def
    by (metis option.distinct(1))
  then show ?thesis
    using rand-res by simp
qed
from verifier-query-round-after-index-authenticated-trace-openings[OF tail]
obtain idx' leaves openings where
  idx'-eq: idx' = ?idx
  and len-openings: length openings = length (powers-scaled idx')
  and idx-openings: map opening-index openings = powers-scaled idx'
  and table-t:
    partial-authenticated-table fr (scale * clength) openings t
  by blast
show ?thesis
proof (rule that)
  show ?idx = index (to-nat raw)
    by simp
  show verifier-query-round-chunk ?idx (map snd f-fl) (map snd fl)
    ?chunk
    by (rule round-chunk)
  show PTranscript s = ?chunk @ PTranscript t
    by (rule tr-t)
  show PState t = foldl concat (PState s) ?chunk
    by (rule st-t)
  show s ≤ t
    by (rule ext-s-t)
  show PQueryCounter t = Suc (PQueryCounter s)
    by (rule query-count-t)
  show fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
    by (rule lookup-t)
  show length openings = length (powers-scaled ?idx)
    using idx'-eq len-openings by simp
  show map opening-index openings = powers-scaled ?idx
    using idx'-eq idx-openings by simp
  show partial-authenticated-table fr (scale * clength) openings t
    by (rule table-t)
qed
qed

lemma ntimes-verifier-query-rounds-selected-trace-openings-aligned-with-replay-at:
  fixes query-state final-state :: ('f, 'a) protocol-channel-scheme

```

```

assumes outcome:
  Some (results, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          query-state)
and i-bound: i < rounds
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and query-chunk:
   $\bigwedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
and replay-lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmllookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
obtains openings where
  map opening-index openings = powers-scaled (query-idxs ! i)
  partial-authenticated-table fr (scale * clength) openings final-state
proof –
let ?round = verifier-query-round-program fr f-fl f-final as fl final
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist (execute (ntimes ?round i) query-state)
and round:
  Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
  Some (suffix, final-state) ∈
    set-dist (execute (ntimes ?round (rounds – Suc i)) s-suc)
by blast
have x-eq: x = ()
by (cases x) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
fix prefix-query-idxs prefix-chunks

```

```

assume len-prefix-chunks: length prefix-chunks = i
and transcript-prefix:
  PTranscript query-state =
    List.concat prefix-chunks @ PTranscript s-i
and state-i:
  PState s-i =
    state-after-query-chunks (PState query-state) prefix-chunks i
and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
and prefix-rounds:
   $\bigwedge k. k < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
show ?thesis
proof (rule
  verifier-query-round-program-outcome-with-authenticated-trace-openings
  [OF round[unfolded x-eq]])
fix raw idx chunk openings
assume idx-eq: idx = index (to-nat raw)
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
and transcript-round: PTranscript s-i = chunk @ PTranscript s-suc
and lookup-suc:
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw
and opening-indices: map opening-index openings = powers-scaled idx
and auth-table-suc:
  partial-authenticated-table fr (scale * clength) openings s-suc
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-chunks:
  length suffix-chunks = rounds - Suc i
and transcript-suffix:
  PTranscript s-suc =
    List.concat suffix-chunks @ PTranscript final-state
and suffix-ext: s-suc ≤ final-state
and suffix-rounds:
   $\bigwedge k. k < \text{rounds} - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
  using len-prefix-chunks len-suffix-chunks i-bound by simp
have selected-chunk-lens:
   $\bigwedge k. k < \text{rounds} \implies$ 
    length (?selected-chunks ! k) =

```

```

      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound: k < rounds
  show
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
proof (cases k < i)
  case True
  have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length (prefix-chunks ! k) =
      verifier-query-round-transcript-length
        (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
  by (rule verifier-query-round-chunk-length
      [OF prefix-rounds[OF True]])
  then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
next
  case False
  show ?thesis
proof (cases k = i)
  case True
  have nth-eq: ?selected-chunks ! k = chunk
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length chunk =
      verifier-query-round-transcript-length idx
        (map snd f-fl) (map snd fl)
  by (rule verifier-query-round-chunk-length[OF chunk-shape])
  then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
next
  case False
  let ?k = k - Suc i
  have ge: Suc i ≤ k
    using ⟨¬ k < i⟩ False by linarith
  have k-bound': ?k < rounds - Suc i
    using ge k-bound by linarith
  have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
    using ge len-prefix-chunks by (simp add: nth-append)
  have

```

```

length (suffix-chunks ! ?k) =
  verifier-query-round-transcript-length
    (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
by (rule verifier-query-round-chunk-length
  [OF suffix-rounds[OF k-bound']])
then show ?thesis
  using nth-eq
  by (simp add:
    verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-lens:
   $\bigwedge k. k < \text{rounds} \implies$ 
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  using query-chunk verifier-query-round-chunk-length by blast
have transcript-all:
  PTranscript query-state =
    List.concat ?selected-chunks @ PTranscript final-state
  using transcript-prefix transcript-round transcript-suffix by simp
have concat-eq:
  List.concat ?selected-chunks = List.concat query-chunks
  using transcript-all transcript-query by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
     $\bigwedge k. k < \text{length } ?\text{selected-chunks} \implies$ 
      length (?selected-chunks ! k) = length (query-chunks ! k)
    using selected-chunk-lens query-chunk-lens len-selected-chunks
    by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
    using concat-eq by simp
  have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks  $\wedge$  ([] :: 'f list) = []
    by (rule concat-append-eq-concat-same-chunk-lengths
      [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state)

```



```

      ?selected-chunks i =
      state-after-query-chunks (PState query-state)
      prefix-chunks i
    by (rule state-after-query-chunks-append-prefix
      [OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have lookup-final:
  fmlookup (HashMap final-state)
  (QueryIndexChallenge (PQueryCounter query-state + i)
    (state-after-query-chunks
      (PState query-state) query-chunks i)) =
  Some raw
  using hash-extension-lookup[OF lookup-suc suffix-ext]
  unfolding counter-i state-i prefix-state-eq .
have raw-eq: raw = raw-idxs ! i
  using lookup-final replay-lookup[OF i-bound] by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-eq query-idxs-eq len-raw i-bound by simp
have auth-table-final:
  partial-authenticated-table fr (scale * clength) openings
  final-state
  by (rule partial-authenticated-table-mono
    [OF auth-table-suc suffix-ext])
show ?thesis
  by (rule that[OF - auth-table-final])
  (use opening-indices idx-eq-query in simp)
qed
qed
qed
qed

lemma verifier-query-round-program-outcome-with-zero-trace-final-opening:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes f-fl-empty: f-fl = []
  and outcome:
    Some (((), t) ∈
      set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s))
  obtains raw idx chunk openings where
    idx = index (to-nat raw)
    verifier-query-round-chunk idx [] (map snd fl) chunk
    PTranscript s = chunk @ PTranscript t
    PState t = foldl concat (PState s) chunk
    s ≤ t
    PQueryCounter t = Suc (PQueryCounter s)
    fmlookup (HashMap t) (QueryIndexChallenge (PQueryCounter s) (PState s)) =

```

```

    Some raw
    map opening-index openings = powers-scaled idx
    partial-authenticated-table fr (scale * clength) openings t
    opening-value (hd openings) = f-final
proof –
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where
  rand:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
and tail:
    Some ((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s0)
by (auto elim!: set-dist-bindE)
let ?idx = index (to-nat raw)
from tail obtain fv s1 f-out s2 s3 c-out s4 where
  query-decommit:
    Some (fv, s1) ∈
      set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s0)
and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
and assert-trace:
    Some ((), s3) ∈
      set-dist
        (execute
          (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold
          (?idx, cp-eval as fv (h ^ ?idx * shift), clength * scale, 1)
          (receive-query-commits fl)) s3)
and assert-comp:
    Some ((), t) ∈
      set-dist
        (execute (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
unfolding verifier-query-round-after-index-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have rand-res:
  s ≤ s0 ∧
  PState s0 = PState s ∧
  PTranscript s0 = PTranscript s ∧
  fmlookup (HashMap s0)

```

```

    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
using receive-query-index-challenge-outcome[OF rand] .
have rand-counter:
  PQueryCounter s0 = Suc (PQueryCounter s)
  using receive-query-index-challenge-counter-outcome[OF rand] by simp
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript ?idx fv query-paths query-chunk
  and tr-s0: PTranscript s0 = query-chunk @ PTranscript s1
  and st-s1: PState s1 = foldl concat (PState s0) query-chunk
  and ext-s0-s1: s0 ≤ s1
  and query-count-s1: PQueryCounter s1 = PQueryCounter s0
  by blast
have trace-out:
  f-out = (?idx, hd fv, clength * scale, 1)
  and s2-eq: s2 = s1
  using trace-fri f-fl-empty
  unfolding receive-query-commits-def by simp-all
have hd-fv: hd fv = f-final
  using assert-trace unfolding assert-def trace-out
  by (cases hd fv = f-final) (auto simp: throw-no-outcome)
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def trace-out hd-fv by simp
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk
  and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
  and st-s4: PState s4 = foldl concat (PState s3) comp-fri-chunk
  and ext-s3-s4: s3 ≤ s4
  and query-count-s4: PQueryCounter s4 = PQueryCounter s3
  by blast
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
have trace-layers-empty:
  fri-layers-transcript 0 (clength * scale) [] []
  unfolding fri-layers-transcript-def by simp
let ?chunk = query-chunk @ [] @ comp-fri-chunk
have round-chunk:
  verifier-query-round-chunk ?idx [] (map snd fl) ?chunk
  unfolding verifier-query-round-chunk-def
  apply (rule exI[where x=query-chunk])
  apply (rule exI[where x=[]])
  apply (rule exI[where x=comp-fri-chunk])
  apply (rule exI[where x=fv])

```

```

apply (rule exI[where  $x = \text{query-paths}$ ])
apply (rule exI[where  $x = []$ ])
apply (rule exI[where  $x = \text{comp-layer-chunks}$ ])
using query-chunk-shape trace-layers-empty comp-layers
by simp
have tr-t: PTranscript  $s = ?\text{chunk} @ \text{PTranscript } t$ 
  using rand-res tr-s0 tr-s3 s3-eq s2-eq t-eq by simp
have st-t: PState  $t = \text{foldl concat (PState } s) ?\text{chunk}$ 
  using rand-res st-s1 st-s4 s3-eq s2-eq t-eq by simp
have ext-s0-t:  $s0 \leq t$ 
  using ext-s0-s1 ext-s3-s4 t-eq unfolding s3-eq s2-eq
  by (meson hash-ext-trans)
have ext-s-t:  $s \leq t$ 
  using rand-res ext-s0-t by (meson hash-ext-trans)
have query-count-t: PQueryCounter  $t = \text{Suc (PQueryCounter } s)$ 
  using rand-counter query-count-s1 query-count-s4
  unfolding s3-eq s2-eq t-eq by simp
have lookup-t:
  fmlookup (HashMap  $t$ )
    (QueryIndexChallenge (PQueryCounter  $s$ ) (PState  $s$ )) = Some raw
using hash-extension-lookup[OF conjunct2[OF conjunct2[OF conjunct2[OF rand-res]]] ext-s0-t]
  rand-res
by simp
have idx-in:  $?idx \in \text{query-sample-space}$ 
  using index-less-query-sample-space
  unfolding query-sample-space-def by simp
from check-decommit-on-query-authenticated-openings
  [OF idx-in query-decommit]
obtain openings where values-openings:
  map opening-value openings = fv
  and idx-openings: map opening-index openings = powers-scaled  $?idx$ 
  and table-s1:
  partial-authenticated-table fr (scale * clength) openings s1
by blast
have s1-t:  $s1 \leq t$ 
  using ext-s3-s4 unfolding s3-eq s2-eq t-eq .
have table-t:
  partial-authenticated-table fr (scale * clength) openings  $t$ 
  by (rule partial-authenticated-table-mono[OF table-s1 s1-t])
have openings-nonempty: openings  $\neq []$ 
proof -
  have map opening-index openings  $\neq []$ 
  using idx-openings powers-pos unfolding powers-scaled-def by simp
  then show ?thesis by auto
qed
have hd-value: opening-value (hd openings) = hd fv
  using values-openings openings-nonempty
  by (cases openings; cases fv) simp-all
show ?thesis

```

```

proof (rule that)
  show ?idx = index (to-nat raw)
    by simp
  show verifier-query-round-chunk ?idx [] (map snd fl) ?chunk
    by (rule round-chunk)
  show PTranscript s = ?chunk @ PTranscript t
    by (rule tr-t)
  show PState t = foldl concat (PState s) ?chunk
    by (rule st-t)
  show s ≤ t
    by (rule ext-s-t)
  show PQueryCounter t = Suc (PQueryCounter s)
    by (rule query-count-t)
  show fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
    by (rule lookup-t)
  show map opening-index openings = powers-scaled ?idx
    by (rule idx-openings)
  show partial-authenticated-table fr (scale * clength) openings t
    by (rule table-t)
  show opening-value (hd openings) = f-final
    using hd-value hd-fv by simp
qed
qed

```

```

lemma ntimes-verifier-query-rounds-selected-empty-trace-chunks-at:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-empty: fl = []
  and outcome:
    Some (results, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) n)
          s)
  and i-bound: i < n
obtains raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
  suffix-chunks trace-openings where
    idx = index (to-nat raw)
    length prefix-query-idxs = i
    length prefix-chunks = i
    length suffix-query-idxs = n - Suc i
    length suffix-chunks = n - Suc i
    verifier-query-round-chunk idx (map snd f-fl) [] chunk
     $\bigwedge j. j < i \implies$ 
      verifier-query-round-chunk (prefix-query-idxs ! j)
      (map snd f-fl) [] (prefix-chunks ! j)
     $\bigwedge j. j < n - Suc i \implies$ 
      verifier-query-round-chunk (suffix-query-idxs ! j)

```

```

    (map snd f-fl) [] (suffix-chunks ! j)
PTranscript s =
  List.concat (prefix-chunks @ chunk # suffix-chunks) @ PTranscript t
map opening-index trace-openings = powers-scaled idx
partial-authenticated-table fr (scale * clength) trace-openings t
fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s + i)
    (state-after-query-chunks (PState s) prefix-chunks i)) = Some raw
proof –
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix round suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) i)
          s)
and round:
    Some (round, s-suc) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s-i)
and suffix:
    Some (suffix, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            (n – Suc i))
          s-suc)
by blast
have round-unit: round = ()
by (cases round) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
fix prefix-query-idxs prefix-chunks
assume len-prefix: length prefix-query-idxs = i
and len-prefix-chunks: length prefix-chunks = i
and transcript-prefix:
  PTranscript s = List.concat prefix-chunks @ PTranscript s-i
and state-i:
  PState s-i =
    state-after-query-chunks (PState s) prefix-chunks i
and counter-i: PQueryCounter s-i = PQueryCounter s + i
and prefix-ext: s ≤ s-i
and prefix-rounds:
   $\bigwedge j. j < i \implies$ 

```

```

    verifier-query-round-chunk (prefix-query-idxs ! j)
      (map snd f-fl) (map snd fl) (prefix-chunks ! j)
show ?thesis
proof (rule
  verifier-query-round-program-outcome-with-authenticated-trace-openings
    [OF round[unfolded round-unit]])
fix raw idx chunk trace-openings
assume idx-eq: idx = index (to-nat raw)
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
and transcript-round:
  PTranscript s-i = chunk @ PTranscript s-suc
and state-suc:
  PState s-suc = foldl concat (PState s-i) chunk
and round-ext: s-i ≤ s-suc
and counter-suc: PQueryCounter s-suc = Suc (PQueryCounter s-i)
and lookup-suc:
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
and len-trace:
  length trace-openings = length (powers-scaled idx)
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table-suc:
  partial-authenticated-table fr (scale * clength)
    trace-openings s-suc
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs:
  length suffix-query-idxs = n - Suc i
and len-suffix-chunks:
  length suffix-chunks = n - Suc i
and transcript-suffix:
  PTranscript s-suc =
    List.concat suffix-chunks @ PTranscript t
and suffix-state:
  PState t =
    state-after-query-chunks (PState s-suc) suffix-chunks
      (n - Suc i)
and suffix-counter:
  PQueryCounter t = PQueryCounter s-suc + (n - Suc i)
and suffix-ext: s-suc ≤ t
and suffix-rounds:
   $\bigwedge j. j < n - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! j)

```

```

      (map snd f-fl) (map snd fl) (suffix-chunks ! j)
have transcript-all:
  PTranscript s =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript t
  using transcript-prefix transcript-round transcript-suffix
  by simp
have trace-table-t:
  partial-authenticated-table fr (scale * clength)
    trace-openings t
  by (rule partial-authenticated-table-mono
    [OF trace-table-suc suffix-ext])
have lookup-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + i)
      (state-after-query-chunks (PState s) prefix-chunks i)) =
    Some raw
  using hash-extension-lookup[OF lookup-suc suffix-ext]
  unfolding counter-i state-i .
show ?thesis
  by (rule that[OF idx-eq len-prefix len-prefix-chunks
    len-suffix-idxs len-suffix-chunks - - - transcript-all
    trace-indices trace-table-t lookup-t])
    (use chunk-shape prefix-rounds suffix-rounds fl-empty in simp-all)
qed
qed
qed
qed

lemma ntimes-verifier-query-rounds-selected-empty-trace-chunks-aligned-with-replay-at:
assumes outcome:
  Some (results, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as [] final)
            rounds)
          query-state)
and i-bound: i < rounds
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and query-chunk:
  ∧ i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)

```



```

    (map snd f-fl) [] (query-chunks ! i)
and replay-lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
obtains trace-openings where
  map opening-index trace-openings = powers-scaled (query-idxs ! i)
  partial-authenticated-table fr (scale * clength) trace-openings
  final-state
proof (rule ntimes-verifier-query-rounds-selected-empty-trace-chunks-at
  [OF refl outcome i-bound])
fix raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
  suffix-chunks trace-openings
assume selected-idx: idx = index (to-nat raw)
and len-prefix-chunks: length prefix-chunks = i
and len-suffix-chunks: length suffix-chunks = rounds - Suc i
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) [] chunk
and prefix-rounds:
   $\bigwedge j. j < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! j)
      (map snd f-fl) [] (prefix-chunks ! j)
and suffix-rounds:
   $\bigwedge j. j < \text{rounds} - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! j)
      (map snd f-fl) [] (suffix-chunks ! j)
and selected-transcript:
  PTranscript query-state =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript final-state
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table:
  partial-authenticated-table fr (scale * clength) trace-openings
  final-state
and selected-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) prefix-chunks i)) =
    Some raw
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
  using len-prefix-chunks len-suffix-chunks i-bound by simp
have selected-chunk-lens:
   $\bigwedge j. j < \text{rounds} \implies$ 
    length (?selected-chunks ! j) =

```

```

    verifier-query-round-transcript-length (query-idxs ! j)
      (map snd f-fl) []
  proof -
    fix j
    assume j-bound: j < rounds
    show
      length (?selected-chunks ! j) =
        verifier-query-round-transcript-length (query-idxs ! j)
          (map snd f-fl) []
  proof (cases j < i)
    case True
    have nth-eq: ?selected-chunks ! j = prefix-chunks ! j
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length (prefix-chunks ! j) =
        verifier-query-round-transcript-length
          (prefix-query-idxs ! j) (map snd f-fl) []
      by (rule verifier-query-round-chunk-length[OF prefix-rounds[OF True]])
    then show ?thesis
      using nth-eq
      by (simp add: verifier-query-round-transcript-length-index-irrelevant)
  next
    case False
    show ?thesis
    proof (cases j = i)
      case True
      have nth-eq: ?selected-chunks ! j = chunk
        using True len-prefix-chunks by (simp add: nth-append)
      have
        length chunk =
          verifier-query-round-transcript-length idx (map snd f-fl) []
        by (rule verifier-query-round-chunk-length[OF chunk-shape])
      then show ?thesis
        using nth-eq
        by (simp add:
            verifier-query-round-transcript-length-index-irrelevant)
    next
      case False
      let ?k = j - Suc i
      have ge: Suc i ≤ j
        using <¬ j < i> False by linarith
      have k-bound: ?k < rounds - Suc i
        using ge j-bound by linarith
      have nth-eq: ?selected-chunks ! j = suffix-chunks ! ?k
        using ge len-prefix-chunks by (simp add: nth-append)
      have
        length (suffix-chunks ! ?k) =
          verifier-query-round-transcript-length
            (suffix-query-idxs ! ?k) (map snd f-fl) []

```

```

    by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound]])
  then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-lens:
   $\bigwedge j. j < \text{rounds} \implies$ 
    length (query-chunks ! j) =
      verifier-query-round-transcript-length (query-idxs ! j)
      (map snd f-fl) []
  using query-chunk verifier-query-round-chunk-length by blast
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
     $\bigwedge j. j < \text{length } ?\text{selected-chunks} \implies$ 
      length (?selected-chunks ! j) = length (query-chunks ! j)
    using selected-chunk-lens query-chunk-lens len-selected-chunks by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
    using concat-eq by simp
  have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks  $\wedge$  ([] :: 'f list) = []
    by (rule concat-append-eq-concat-same-chunk-lengths
        [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
      state-after-query-chunks (PState query-state) prefix-chunks i
    by (rule state-after-query-chunks-append-prefix[OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have selected-lookup-global:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =

```

```

    Some raw
  using selected-lookup prefix-state-eq by simp
have raw-eq: raw = raw-idxs ! i
  using selected-lookup-global replay-lookup[OF i-bound] by simp
have idx-eq: idx = query-idxs ! i
  using selected-idx raw-eq query-idxs-eq len-raw i-bound by simp
show ?thesis
  by (rule that)
    (use idx-eq trace-indices trace-table in simp-all)
qed

lemma ntimes-verifier-query-rounds-empty-trace-openings-aligned-with-replay:
  assumes outcome:
    Some (results, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as [] final)
            rounds)
          query-state)
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-query-chunks: length query-chunks = rounds
  and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state
  and query-chunk:
    ∧ i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
        (map snd f-fl) [] (query-chunks ! i)
  and replay-lookup:
    ∧ i. i < rounds ⇒
      fmlookup (HashMap final-state)
        (QueryIndexChallenge (PQueryCounter query-state + i)
          (state-after-query-chunks
            (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
  obtains trace-openings where
    length trace-openings = rounds
    ∧ i. i < rounds ⇒
      map opening-index (trace-openings ! i) =
        powers-scaled (query-idxs ! i)
    ∧ i. i < rounds ⇒
      partial-authenticated-table fr (scale * clength)
        (trace-openings ! i) final-state
proof -
  let ?P =
    λi openings.

```

```

    map opening-index openings = powers-scaled (query-idxs ! i) ∧
    partial-authenticated-table fr (scale * clength) openings final-state
have ex-round: ∧i. i < rounds ⇒ ∃ openings. ?P i openings
proof -
  fix i
  assume i-bound: i < rounds
  show ∃ openings. ?P i openings
  proof (rule
    ntimes-verifier-query-rounds-selected-empty-trace-chunks-aligned-with-replay-at
    [OF outcome i-bound len-raw query-idxs-eq len-query-chunks
      transcript-query query-chunk replay-lookup])
    fix trace-openings
    assume
      map opening-index trace-openings =
        powers-scaled (query-idxs ! i)
        partial-authenticated-table fr (scale * clength)
        trace-openings final-state
    then show ?thesis
      by blast
  qed
qed
let ?w = λi. SOME openings. ?P i openings
let ?trace-openings = map ?w [0..<rounds]
have len-trace: length ?trace-openings = rounds
  by simp
have props:
  ∧i. i < rounds ⇒ ?P i (?trace-openings ! i)
proof -
  fix i
  assume i-bound: i < rounds
  have ?P i (?w i)
    by (rule someI-ex[OF ex-round[OF i-bound]])
  then show ?P i (?trace-openings ! i)
    using i-bound by simp
qed
show ?thesis
  by (rule that[OF len-trace])
  (use props in simp-all)
qed

lemma ntimes-verifier-query-rounds-selected-zero-trace-final-opening-at:
  assumes outcome:
    Some (results, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr [] f-final as fl final)
            rounds)
          query-state)

```

```

and i-bound:  $i < \text{rounds}$ 
and len-raw:  $\text{length } \text{raw-idxs} = \text{rounds}$ 
and query-idxs-eq:
   $\text{query-idxs} = \text{map } (\lambda \text{raw. index (to-nat raw)}) \text{ raw-idxs}$ 
and len-query-chunks:  $\text{length } \text{query-chunks} = \text{rounds}$ 
and transcript-query:
   $P\text{Transcript } \text{query-state} =$ 
     $\text{List.concat } \text{query-chunks} @ P\text{Transcript } \text{final-state}$ 
and query-chunk:
   $\bigwedge i. i < \text{rounds} \implies$ 
     $\text{verifier-query-round-chunk } (\text{query-idxs } ! i)$ 
     $\quad [] (\text{map } \text{snd } \text{fl}) (\text{query-chunks } ! i)$ 
and replay-lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
     $\text{fmlookup } (\text{HashMap } \text{final-state})$ 
     $\quad (\text{QueryIndexChallenge } (P\text{QueryCounter } \text{query-state} + i)$ 
     $\quad (\text{state-after-query-chunks}$ 
     $\quad (\text{PState } \text{query-state}) \text{ query-chunks } i)) =$ 
     $\text{Some } (\text{raw-idxs } ! i)$ 
obtains trace-openings where
   $\text{map } \text{opening-index } \text{trace-openings} = \text{powers-scaled } (\text{query-idxs } ! i)$ 
   $\text{partial-authenticated-table } \text{fr } (\text{scale} * \text{clength}) \text{ trace-openings}$ 
   $\text{final-state}$ 
   $\text{opening-value } (\text{hd } \text{trace-openings}) = \text{f-final}$ 
proof –
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix round suffix s-i s-suc where
  prefix:
     $\text{Some } (\text{prefix}, \text{s-i}) \in$ 
     $\text{set-dist}$ 
     $(\text{execute}$ 
     $\quad (\text{ntimes}$ 
     $\quad (\text{verifier-query-round-program } \text{fr } [] \text{ f-final as fl final}) i)$ 
     $\quad \text{query-state})$ 
and round:
     $\text{Some } (\text{round}, \text{s-suc}) \in$ 
     $\text{set-dist}$ 
     $(\text{execute}$ 
     $\quad (\text{verifier-query-round-program } \text{fr } [] \text{ f-final as fl final}) \text{ s-i})$ 
and suffix:
     $\text{Some } (\text{suffix}, \text{final-state}) \in$ 
     $\text{set-dist}$ 
     $(\text{execute}$ 
     $\quad (\text{ntimes}$ 
     $\quad (\text{verifier-query-round-program } \text{fr } [] \text{ f-final as fl final})$ 
     $\quad (\text{rounds} - \text{Suc } i))$ 
     $\quad \text{s-suc})$ 
by blast
have round-unit:  $\text{round} = ()$ 

```

```

  by (cases round) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
  fix prefix-query-idxs prefix-chunks
  assume len-prefix: length prefix-query-idxs = i
  and len-prefix-chunks: length prefix-chunks = i
  and transcript-prefix:
    PTranscript query-state = List.concat prefix-chunks @ PTranscript s-i
  and state-i:
    PState s-i =
      state-after-query-chunks (PState query-state) prefix-chunks i
  and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
  and prefix-ext: query-state ≤ s-i
  and prefix-rounds:
     $\bigwedge j. j < i \implies$ 
      verifier-query-round-chunk (prefix-query-idxs ! j)
      (map snd []) (map snd fl) (prefix-chunks ! j)
show ?thesis
proof (rule
  verifier-query-round-program-outcome-with-zero-trace-final-opening
  [OF refl round[unfolded round-unit]])
  fix raw idx chunk trace-openings
  assume selected-idx: idx = index (to-nat raw)
  and chunk-shape:
    verifier-query-round-chunk idx [] (map snd fl) chunk
  and transcript-round:
    PTranscript s-i = chunk @ PTranscript s-suc
  and state-suc:
    PState s-suc = foldl concat (PState s-i) chunk
  and round-ext: s-i ≤ s-suc
  and counter-suc: PQueryCounter s-suc = Suc (PQueryCounter s-i)
  and lookup-suc:
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx
  and trace-table-suc:
    partial-authenticated-table fr (scale * clength)
    trace-openings s-suc
  and trace-value:
    opening-value (hd trace-openings) = f-final
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
  fix suffix-query-idxs suffix-chunks
  assume len-suffix-idxs:
    length suffix-query-idxs = rounds - Suc i

```

```

and len-suffix-chunks:
  length suffix-chunks = rounds - Suc i
and transcript-suffix:
  PTranscript s-suc =
    List.concat suffix-chunks @ PTranscript final-state
and suffix-state:
  PState final-state =
    state-after-query-chunks (PState s-suc) suffix-chunks
    (rounds - Suc i)
and suffix-counter:
  PQueryCounter final-state =
    PQueryCounter s-suc + (rounds - Suc i)
and suffix-ext: s-suc ≤ final-state
and suffix-rounds:
   $\bigwedge j. j < \text{rounds} - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! j)
    (map snd []) (map snd fl) (suffix-chunks ! j)
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have selected-transcript:
  PTranscript query-state =
    List.concat ?selected-chunks @ PTranscript final-state
using transcript-prefix transcript-round transcript-suffix
by simp
have len-selected-chunks: length ?selected-chunks = rounds
using len-prefix-chunks len-suffix-chunks i-bound by simp
have selected-chunk-lens:
   $\bigwedge j. j < \text{rounds} \implies$ 
    length (?selected-chunks ! j) =
      verifier-query-round-transcript-length (query-idxs ! j)
       $\square$  (map snd fl)
proof -
  fix j
  assume j-bound: j < rounds
  show
    length (?selected-chunks ! j) =
      verifier-query-round-transcript-length (query-idxs ! j)
       $\square$  (map snd fl)
proof (cases j < i)
  case True
  have nth-eq: ?selected-chunks ! j = prefix-chunks ! j
  using True len-prefix-chunks by (simp add: nth-append)
  have
    length (prefix-chunks ! j) =
      verifier-query-round-transcript-length
      (prefix-query-idxs ! j)  $\square$  (map snd fl)
  using verifier-query-round-chunk-length
    [OF prefix-rounds [OF True]]
  by simp
then show ?thesis

```



```

    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
next
case False
show ?thesis
proof (cases j = i)
case True
have nth-eq: ?selected-chunks ! j = chunk
    using True len-prefix-chunks by (simp add: nth-append)
have
    length chunk =
        verifier-query-round-transcript-length idx [] (map snd fl)
    by (rule verifier-query-round-chunk-length[OF chunk-shape])
then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
next
case False
let ?k = j - Suc i
have ge: Suc i ≤ j
    using <¬ j < i> False by linarith
have k-bound: ?k < rounds - Suc i
    using ge j-bound by linarith
have nth-eq: ?selected-chunks ! j = suffix-chunks ! ?k
    using ge len-prefix-chunks by (simp add: nth-append)
have
    length (suffix-chunks ! ?k) =
        verifier-query-round-transcript-length
            (suffix-query-idxs ! ?k) [] (map snd fl)
    using verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound]]
    by simp
then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-lens:
    ∧j. j < rounds ⇒
        length (query-chunks ! j) =
            verifier-query-round-transcript-length (query-idxs ! j)
            [] (map snd fl)
    using query-chunk verifier-query-round-chunk-length by blast
have concat-eq:
    List.concat ?selected-chunks = List.concat query-chunks

```

```

using selected-transcript transcript-query by simp
have chunks-eq: ?selected-chunks = query-chunks
proof —
  have same-lens:
     $\bigwedge j. j < \text{length } ?\text{selected-chunks} \implies$ 
       $\text{length } (?selected-chunks ! j) = \text{length } (query-chunks ! j)$ 
    using selected-chunk-lens query-chunk-lens len-selected-chunks
    by simp
  have concat-eq-empty:
     $List.concat ?selected-chunks @ [] = List.concat query-chunks$ 
    using concat-eq by simp
  have len-eq:  $\text{length } ?selected-chunks = \text{length } query-chunks$ 
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks  $\wedge ([] :: 'f list) = []$ 
    by (rule concat-append-eq-concat-same-chunk-lengths
      [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
   $state\text{-after}\text{-query}\text{-chunks } (PState \text{ query-state}) \text{ prefix-chunks } i =$ 
   $state\text{-after}\text{-query}\text{-chunks } (PState \text{ query-state}) \text{ query-chunks } i$ 
proof —
  have
     $state\text{-after}\text{-query}\text{-chunks } (PState \text{ query-state}) ?selected-chunks i =$ 
     $state\text{-after}\text{-query}\text{-chunks } (PState \text{ query-state}) \text{ prefix-chunks } i$ 
    by (rule state-after-query-chunks-append-prefix
      [OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have selected-lookup-global:
   $fmlookup (HashMap \text{ final-state})$ 
   $(QueryIndexChallenge (PQueryCounter \text{ query-state} + i)$ 
   $(state\text{-after}\text{-query}\text{-chunks } (PState \text{ query-state})$ 
   $query-chunks i)) =$ 
   $Some \text{ raw}$ 
  using hash-extension-lookup [OF lookup-suc suffix-ext]
  unfolding counter-i state-i prefix-state-eq .
have raw-eq:  $\text{raw} = \text{raw-idxs} ! i$ 
  using selected-lookup-global replay-lookup [OF i-bound] by simp
have idx-eq:  $\text{idx} = \text{query-idxs} ! i$ 
  using selected-idx raw-eq query-idxs-eq len-raw i-bound by simp
have trace-table:
   $partial\text{-authenticated-table } fr (\text{scale} * \text{clength}) \text{ trace-openings}$ 
   $\text{final-state}$ 
  by (rule partial-authenticated-table-mono
    [OF trace-table-suc suffix-ext])
show ?thesis

```

```

      by (rule that)
      (use idx-eq trace-indices trace-table trace-value in simp-all)
    qed
  qed
  qed
  qed

lemma ntimes-verifier-query-rounds-zero-trace-final-openings-aligned-with-replay:
assumes outcome:
  Some (results, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr [] f-final as fl final)
          rounds)
        query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and query-chunk:
  ∧i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
    [] (map snd fl) (query-chunks ! i)
and replay-lookup:
  ∧i. i < rounds ⇒
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
obtains trace-openings where
  length trace-openings = rounds
  ∧i. i < rounds ⇒
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
  ∧i. i < rounds ⇒
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
  ∧i. i < rounds ⇒
    opening-value (hd (trace-openings ! i)) = f-final
proof -
let ?P =
  λi openings.
    map opening-index openings = powers-scaled (query-idxs ! i) ∧
    partial-authenticated-table fr (scale * clength) openings final-state ∧

```

```

    opening-value (hd openings) = f-final
have ex-round:  $\bigwedge i. i < \text{rounds} \implies \exists \text{openings}. ?P\ i\ \text{openings}$ 
proof -
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  show  $\exists \text{openings}. ?P\ i\ \text{openings}$ 
  proof (rule
    ntimes-verifier-query-rounds-selected-zero-trace-final-opening-at
    [OF outcome i-bound len-raw query-idxs-eq len-query-chunks
      transcript-query query-chunk replay-lookup])
  fix trace-openings
  assume
    map opening-index trace-openings =
      powers-scaled (query-idxs ! i)
    partial-authenticated-table fr (scale * clength)
    trace-openings final-state
    opening-value (hd trace-openings) = f-final
  then show ?thesis
    by blast
qed
qed
let ?w =  $\lambda i. \text{SOME openings}. ?P\ i\ \text{openings}$ 
let ?trace-openings = map ?w [0.. $\text{rounds}$ ]
have len-trace: length ?trace-openings = rounds
  by simp
have props:
   $\bigwedge i. i < \text{rounds} \implies ?P\ i\ (?trace-openings\ !\ i)$ 
proof -
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  have  $?P\ i\ (?w\ i)$ 
    by (rule someI-ex[OF ex-round[OF i-bound]])
  then show  $?P\ i\ (?trace-openings\ !\ i)$ 
    using i-bound by simp
qed
show ?thesis
  by (rule that[OF len-trace])
  (use props in simp-all)
qed

lemma verify-monad-supplied-empty-header-partial-trace-openings:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
  and header0:
    verifier-header-transcript s fr f-fri-roots f-final as dg []
    final rest
  obtains query-idxs trace-openings where
    accepted-transcript-shape s (Some (result, final-state)) as

```

```

    query-idxs
  accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
proof –
obtain f-fl query-state where f-roots:
  map snd f-fl = f-fri-roots
and query-rest: PTranscript query-state = rest
and query-state-start:
  PState query-state =
    verifier-header-state s fr f-fri-roots f-final as dg [] final
and query-count:
  PQueryCounter query-state = PQueryCounter s
and query-out:
  Some (result, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as [] final)
          rounds)
        query-state)
by (rule verify-monad-supplied-empty-header-query-rounds
      [OF outcome header0])
show ?thesis
proof (rule verify-monad-supplied-header-transcript-shape
      [OF outcome header0])
fix query-idxs raw-idxs query-chunks
assume shape:
  accepted-transcript-shape s (Some (result, final-state)) as
    query-idxs
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  List.concat query-chunks @ PTranscript final-state = rest
and query-chunk:
  ∧ i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
    f-fri-roots [] (query-chunks ! i)
and replay-lookup:
  ∧ i. i < rounds ⇒
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks
          (verifier-header-state s fr f-fri-roots f-final as dg []
            final)
          query-chunks i)) =
      Some (raw-idxs ! i)
have transcript-query-state:

```

```

PTranscript query-state =
  List.concat query-chunks @ PTranscript final-state
using transcript-query query-rest by simp
have query-chunk-empty:
   $\bigwedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) [] (query-chunks ! i)
  using query-chunk f-roots by simp
have replay-lookup-query-state:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmllookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
  using replay-lookup query-state-start query-count by simp
show ?thesis
proof (rule
  ntimes-verifier-query-rounds-empty-trace-openings-aligned-with-replay
    [OF query-out len-raw query-idxs-eq len-query-chunks
      transcript-query-state query-chunk-empty
      replay-lookup-query-state])
fix trace-openings
assume len-trace: length trace-openings = rounds
and trace-indices:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (trace-openings ! i) =
    powers-scaled (query-idxs ! i)
and trace-tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
have len-query-idxs: length query-idxs = rounds
  using len-raw query-idxs-eq by simp
have partial:
  accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
  unfolding accepted-with-partial-trace-openings-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state])
    (use len-query-idxs len-trace trace-indices trace-tables in auto)
show ?thesis
  by (rule that[OF shape partial])
qed
qed
qed

end

end

```

```

theory Soundness-FRI-Verifier-Tied
  imports
    Soundness-FRI-Zero-Round-Target
    Soundness-Query-Transcript-Openings
begin

```

Verifier-tied FRI partial-candidate events.

Broad reachable partial-candidate events allow the candidate table to be compatible with sampled authenticated openings whose indices are not tied to the FRI opening transcript. This theory introduces narrower internal events where the candidate openings use the same query indices as the verifier's FRI transcript. The protocol and public soundness statement are unchanged.

```

context soundness
begin

```

```

lemma partial-authenticated-tables-values-unique-if-clean:
  assumes table:
    partial-authenticated-table rt len openings s
  and table':
    partial-authenticated-table rt len openings' s
  and clean:  $\neg$  hash-map-output-collision s
  and first: opening  $\in$  set openings
  and second: opening'  $\in$  set openings'
  and same-index: opening-index opening = opening-index opening'
  shows opening-value opening = opening-value opening'
proof –
  have combined:
    partial-authenticated-table rt len (openings @ openings') s
  using table table'
  unfolding partial-authenticated-table-def by auto
  show ?thesis
  by (rule partial-authenticated-table-values-unique-if-clean[
    OF combined clean])
    (use first second same-index in auto)
qed

```

```

lemma verifier-query-round-after-index-zero-trace-final-opening:
  fixes s t :: (f, a) protocol-channel-scheme
  assumes empty-trace: f-fl = []
  and outcome:
    Some ((), t)  $\in$ 
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s)
  obtains idx openings where

```

```

  idx = index (to-nat raw)
  and map opening-index openings = powers-scaled idx
  and partial-authenticated-table fr (scale * clength) openings t
  and opening-value (hd openings) = f-final
proof -
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1) ∈
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
  and trace-fri:
    Some (f-out, s2) ∈
      set-dist
        (execute
          (mfold (?idx, hd fv, clength * scale, 1)
            (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some ((), s3) ∈
      set-dist
        (execute
          (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
          clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some ((), t) ∈
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have trace-out:
  f-out = (?idx, hd fv, clength * scale, 1)
  and s2-eq: s2 = s1
  using trace-fri empty-trace
  unfolding receive-query-commits-def by simp-all
have hd-fv: hd fv = f-final
  using assert-trace unfolding assert-def trace-out
  by (cases hd fv = f-final) (auto simp: throw-no-outcome)
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def trace-out hd-fv by simp
have idx-in: ?idx ∈ query-sample-space
  using index-less-query-sample-space
  unfolding query-sample-space-def by simp
from check-decommit-on-query-authenticated-openings
  [OF idx-in query-decommit]

```


obtain *openings* **where** *len-openings*:
 length openings = length (powers-scaled ?idx)
and *values-openings*: *map opening-value openings = fv*
and *idx-openings*: *map opening-index openings = powers-scaled ?idx*
and *table-s1*:
 *partial-authenticated-table fr (scale * clength) openings s1*
by *blast*
from *receive-query-commits-layers-outcome[OF comp-fri]*
obtain *comp-layer-chunks comp-fri-chunk* **where** *s3-s4: s3 ≤ s4*
by *blast*
have *t-eq: t = s4*
 using *assert-comp unfolding assert-def*
 by (*cases case c-out of (-, x, -, -) ⇒ x = final*)
 (*auto simp: throw-no-outcome*)
have *s1-t: s1 ≤ t*
 using *s3-s4 unfolding s3-eq s2-eq t-eq .*
have *table-t*:
 *partial-authenticated-table fr (scale * clength) openings t*
 by (*rule partial-authenticated-table-mono[OF table-s1 s1-t]*)
have *openings-nonempty: openings ≠ []*
 using *len-openings powers-pos unfolding powers-scaled-def by auto*
have *hd-value: opening-value (hd openings) = hd fv*
 using *values-openings openings-nonempty*
 by (*cases openings; cases fv*) *simp-all*
show *?thesis*
 by (*rule that[OF refl idx-openings table-t]*)
 (*simp add: hd-value hd-fv*)

qed

lemma *verifier-query-round-program-zero-trace-final-opening*:

fixes *s t :: ('f, 'a) protocol-channel-scheme*
assumes *empty-trace: f-fl = []*
and *outcome*:
 Some ((), t) ∈
 set-dist
 (*execute*
 (*verifier-query-round-program fr f-fl f-final as fl final*) *s*)
shows $\exists$ *raw idx openings.*
 idx = index (to-nat raw) ∧
 map opening-index openings = powers-scaled idx ∧
 *partial-authenticated-table fr (scale * clength) openings t ∧*
 opening-value (hd openings) = f-final

proof –

from *outcome[unfolded verifier-query-round-program-alt-def]*
obtain *raw s0* **where**
 Some (raw, s0) ∈ set-dist (execute receive-query-index-challenge s)
and *after*:
 Some ((), t) ∈
 set-dist

```

      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s0)
    by (auto elim!: set-dist-bindE)
  from verifier-query-round-after-index-zero-trace-final-opening
    [OF empty-trace after]
  show ?thesis
    by blast
qed

lemma trace-zero-round-checked-candidate-from-verified-openings-if-no-merkle:
  fixes final-state :: ('f, 'a) protocol-channel-scheme
  assumes partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
  and candidate:
    partial-trace-table-candidate trace-table trace-openings
  and no-merkle:
    ¬ partial-merkle-inconsistency-bad s (Some (result, final-state))
  and query-sample:
     $\bigwedge \text{idx}. \text{idx} \in \text{set query-idxs} \implies \text{idx} \in \text{query-sample-space}$ 
  and verifier-openings:
     $\bigwedge i. i < \text{rounds} \implies$ 
       $\exists \text{openings}.$ 
      map opening-index openings = powers-scaled (query-idxs ! i)  $\wedge$ 
      partial-authenticated-table fr (scale * clength) openings
      final-state  $\wedge$ 
      opening-value (hd openings) = final
  shows
    trace-zero-round-final-checked-candidate s
    (Some (result, final-state)) fr query-idxs trace-openings
    trace-table final
proof -
  have final-values:
     $\bigwedge i. i < \text{rounds} \implies$ 
      opening-value (hd (trace-openings ! i)) = final
proof -
  fix i
  assume i-bound: i < rounds
  have candidate-indices:
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
  by (rule accepted-with-partial-trace-openings-shapes(3)
    [OF partial i-bound])
  have candidate-table:
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) final-state
  using partial i-bound
  unfolding accepted-with-partial-trace-openings-def by blast

```

```

from verifier-openings[OF i-bound]
obtain openings where verified-indices:
  map opening-index openings = powers-scaled (query-idxs ! i)
and verified-table:
  partial-authenticated-table fr (scale * clength) openings
  final-state
and verified-value: opening-value (hd openings) = final
by blast
have candidate-nonempty: trace-openings ! i ≠ []
proof –
  have map opening-index (trace-openings ! i) ≠ []
  using candidate-indices powers-pos unfolding powers-scaled-def by simp
  then show ?thesis by auto
qed
have verified-nonempty: openings ≠ []
proof –
  have map opening-index openings ≠ []
  using verified-indices powers-pos unfolding powers-scaled-def by simp
  then show ?thesis by auto
qed
let ?candidate = hd (trace-openings ! i)
let ?verified = hd openings
have candidate-in: ?candidate ∈ set (trace-openings ! i)
  using candidate-nonempty by simp
have verified-in: ?verified ∈ set openings
  using verified-nonempty by simp
have same-index:
  opening-index ?candidate = opening-index ?verified
proof –
  have opening-index ?candidate =
    hd (map opening-index (trace-openings ! i))
  using candidate-nonempty by (cases trace-openings ! i) auto
  also have ... = hd (powers-scaled (query-idxs ! i))
  using candidate-indices by simp
  also have ... = hd (map opening-index openings)
  using verified-indices by simp
  also have ... = opening-index ?verified
  using verified-nonempty by (cases openings) auto
  finally show ?thesis .
qed
have candidate-auth:
  authenticated-opening-in final-state ?candidate
  using candidate-table candidate-in
  unfolding partial-authenticated-table-def by blast
have verified-auth:
  authenticated-opening-in final-state ?verified
  using verified-table verified-in
  unfolding partial-authenticated-table-def by blast
have same-root:

```

```

opening-root ?candidate = opening-root ?verified
using candidate-table verified-table candidate-in verified-in
unfolding partial-authenticated-table-def by auto
have same-length:
  opening-length ?candidate = opening-length ?verified
  using candidate-table verified-table candidate-in verified-in
  unfolding partial-authenticated-table-def by auto
show opening-value ?candidate = final
proof (rule ccontr)
  assume neq-final: opening-value ?candidate  $\neq$  final
  have diff:
    opening-value ?candidate  $\neq$  opening-value ?verified
    using neq-final verified-value by simp
  have partial-merkle-inconsistency-bad s
    (Some (result, final-state))
    unfolding partial-merkle-inconsistency-bad-def
    by (intro conjI exI)
    (use partial candidate-auth verified-auth same-root same-length
      same-index diff in
       $\langle$ auto simp: accepted-with-partial-trace-openings-def $\rangle$ )
  then show False
    using no-merkle by contradiction
qed
qed
show ?thesis
  unfolding trace-zero-round-final-checked-candidate-def
  using partial candidate query-sample final-values by blast
qed

```

definition trace-fri-bad-with-verifier-tied-partial-candidate

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-bad-with-verifier-tied-partial-candidate s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr trace-openings trace-table.}$
 $\text{accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{accepted-with-partial-trace-openings s out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table})$

definition trace-fri-bad-with-header-tied-partial-candidate

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-bad-with-header-tied-partial-candidate } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table})$

lemma *trace-fri-bad-with-header-tied-imp-verifier-tied:*
assumes *trace-fri-bad-with-header-tied-partial-candidate s out*
shows *trace-fri-bad-with-verifier-tied-partial-candidate s out*
using *assms*
unfolding *trace-fri-bad-with-header-tied-partial-candidate-def*
trace-fri-bad-with-verifier-tied-partial-candidate-def
by *blast*

lemma *accepted-fri-opening-transcript-query-idx-in-sample-space:*
assumes *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
query-idxs trace-round-layers composition-round-layers
and *idx-in: idx ∈ set query-idxs*
shows *idx ∈ query-sample-space*
using *fri-openings idx-in index-less-query-sample-space*
unfolding *accepted-fri-opening-transcript-def query-sample-space-def* **by** *auto*

definition *trace-fri-header-tied-zero-round-final-obstruction*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where

$\text{trace-fri-header-tied-zero-round-final-obstruction } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$

$\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{length trace-bs} = 0 \wedge$
 $\neg \text{fri-final-constant-consistent trace-table trace-final})$

lemma *trace-fri-header-tied-zero-round-final-obstruction-imp-checked-or-merkle:*

assumes *obstruction:*

trace-fri-header-tied-zero-round-final-obstruction s out

shows

trace-fri-zero-round-checked-final-obstruction s out $\vee$

partial-merkle-inconsistency-bad s out

proof —

from *obstruction* **obtain** *trace-roots trace-bs trace-final dg*

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr as rest trace-openings

trace-table **where** *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final dg composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers

and *header:*

verifier-header-transcript s fr trace-roots trace-final as dg

composition-roots composition-final rest

and *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs

trace-openings

and *candidate:*

partial-trace-table-candidate trace-table trace-openings

and *not-low:* $\neg \text{trace-table-low-degree trace-table}$

and *zero-rounds:* $\text{length trace-bs} = 0$

and *not-final:*

$\neg \text{fri-final-constant-consistent trace-table trace-final}$

unfolding *trace-fri-header-tied-zero-round-final-obstruction-def*

by *blast*

from *fri-openings* **obtain** *result final-state fr0 f-fl as0 fl query-state*

raw-idxs query-chunks **where** *out-eq:*

out = Some (result, final-state)

and *trace-roots-eq:* $\text{trace-roots} = \text{map snd f-fl}$

and *trace-bs-eq:* $\text{trace-bs} = \text{map fst f-fl}$

and *composition-roots-eq:* $\text{composition-roots} = \text{map snd fl}$

and *composition-bs-eq:* $\text{composition-bs} = \text{map fst fl}$

and *header0:*

verifier-header-transcript s fr0 trace-roots trace-final as0 dg

composition-roots composition-final (PTranscript query-state)

and *query-state-eq:*

PState query-state =

verifier-header-state s fr0 trace-roots trace-final as0 dg

composition-roots composition-final

and *query-counter-eq:*

PQueryCounter query-state = PQueryCounter s

and *query-out:*

```

Some (result, final-state) ∈
  set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr0 f-fl trace-final as0 fl
          composition-final)
        rounds)
      query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  fri-query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and query-chunk:
  ∧i. i < rounds ⇒
    verifier-query-round-chunk (fri-query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
and replay-lookup:
  ∧i. i < rounds ⇒
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
unfolding accepted-fri-opening-transcript-def by blast
have f-fl-empty: f-fl = []
  using zero-rounds trace-bs-eq by simp
have trace-roots-empty: trace-roots = []
  using trace-roots-eq f-fl-empty by simp
have header-egs:
  fr0 = fr ∧
  as0 = as ∧
  PTranscript query-state = rest
  using verifier-header-transcript-unique[OF header0 header] by simp
have query-out-empty:
  Some (result, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr [] trace-final as fl
            composition-final)
          rounds)
        query-state)
    using query-out f-fl-empty header-egs by simp
have query-chunk-empty:
  ∧i. i < rounds ⇒
    verifier-query-round-chunk (fri-query-idxs ! i)
    [] (map snd fl) (query-chunks ! i)

```

```

using query-chunk trace-roots-empty composition-roots-eq by simp
show ?thesis
proof (cases partial-merkle-inconsistency-bad s out)
  case True
  then show ?thesis by simp
next
  case False
  have no-merkle:
     $\neg$  partial-merkle-inconsistency-bad s
    (Some (result, final-state))
  using False out-eq by simp
  have verifier-openings:
     $\bigwedge i. i < \text{rounds} \implies$ 
     $\exists \text{openings.}$ 
    map opening-index openings =
      powers-scaled (fri-query-idxs ! i)  $\wedge$ 
      partial-authenticated-table fr (scale * clength) openings
      final-state  $\wedge$ 
      opening-value (hd openings) = trace-final
  proof -
    fix i
    assume i-bound: i < rounds
    show
       $\exists \text{openings.}$ 
      map opening-index openings =
        powers-scaled (fri-query-idxs ! i)  $\wedge$ 
        partial-authenticated-table fr (scale * clength) openings
        final-state  $\wedge$ 
        opening-value (hd openings) = trace-final
  proof (rule
    ntimes-verifier-query-rounds-selected-zero-trace-final-opening-at
    [OF query-out-empty i-bound len-raw query-idxs-eq len-query-chunks
      transcript-query query-chunk-empty replay-lookup])
    fix verified-openings
    assume
      map opening-index verified-openings =
        powers-scaled (fri-query-idxs ! i)
        partial-authenticated-table fr (scale * clength)
        verified-openings final-state
      opening-value (hd verified-openings) = trace-final
    then show
       $\exists \text{openings.}$ 
      map opening-index openings =
        powers-scaled (fri-query-idxs ! i)  $\wedge$ 
        partial-authenticated-table fr (scale * clength) openings
        final-state  $\wedge$ 
        opening-value (hd openings) = trace-final
    by blast
  qed

```



```

qed
have query-sample:
   $\bigwedge idx. idx \in \text{set } \text{fri-query-idxs} \implies idx \in \text{query-sample-space}$ 
  by (rule accepted-fri-opening-transcript-query-idx-in-sample-space
      [OF fri-openings])
have partial-some:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr fri-query-idxs trace-openings
  using partial out-eq by simp
have checked:
  trace-zero-round-final-checked-candidate s
  (Some (result, final-state)) fr fri-query-idxs trace-openings
  trace-table trace-final
  by (rule trace-zero-round-checked-candidate-from-verified-openings-if-no-merkle
      [OF partial-some candidate no-merkle query-sample verifier-openings])
have trace-fri-zero-round-checked-final-obstruction s out
  unfolding trace-fri-zero-round-checked-final-obstruction-def out-eq
  by (intro exI conjI)
  (use fri-openings[unfolded out-eq] zero-rounds checked not-final
      in simp-all)
then show ?thesis by simp
qed
qed

definition composition-fri-bad-with-verifier-tied-partial-candidate
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  composition-fri-bad-with-verifier-tied-partial-candidate s out
   $\iff$ 
  ( $\exists$  trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as trace-openings composition-openings
    trace-table composition-table.
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers  $\wedge$ 
    accepted-with-partial-initial-openings-aligned s out fr trace-roots
    trace-final as fri-dg composition-roots composition-final fri-query-idxs
    trace-openings composition-openings  $\wedge$ 
    partial-trace-table-candidate trace-table trace-openings  $\wedge$ 
    partial-composition-table-candidate composition-table
    composition-openings  $\wedge$ 
    trace-table-low-degree trace-table  $\wedge$ 
     $\neg$  composition-table-low-degree maxDegree composition-table)

definition trace-fri-bad-with-verifier-tied-sampled-layer-chain
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 

```

$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-bad-with-verifier-tied-sampled-layer-chain } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr trace-openings trace-table doms layers.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence trace-table-low-degree}$
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers})$

definition *composition-fri-bad-with-verifier-tied-sampled-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$
 $\text{trace-table composition-table doms layers.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence}$
 $(\text{composition-table-low-degree (to-nat fri-dg)}) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs composition-round-layers doms layers})$

definition *trace-fri-verifier-tied-missing-sampled-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-verifier-tied-missing-sampled-layer-chain } s \text{ out} \longleftrightarrow$

trace-fri-bad-with-verifier-tied-partial-candidate s out $\wedge$
 $\neg$ *trace-fri-bad-with-verifier-tied-sampled-layer-chain s out*

definition *composition-fri-verifier-tied-missing-sampled-layer-chain*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-missing-sampled-layer-chain s out
 $\longleftrightarrow$
composition-fri-bad-with-verifier-tied-partial-candidate s out $\wedge$
 $\neg$ *composition-fri-bad-with-verifier-tied-sampled-layer-chain s out*

definition *trace-fri-verifier-tied-reduction*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

trace-fri-verifier-tied-reduction s $\longleftrightarrow$
wp-event verify-monad
 $(\text{trace-fri-bad-with-verifier-tied-partial-candidate } s) \leq$
trace-fri-error

definition *composition-fri-verifier-tied-reduction*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-reduction s $\longleftrightarrow$
wp-event verify-monad
 $(\text{composition-fri-bad-with-verifier-tied-partial-candidate } s) \leq$
composition-fri-error

definition *soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
s out $\longleftrightarrow$
composition-bad-with-partial-candidates s out $\vee$
trace-fri-bad-with-header-tied-partial-candidate s out $\vee$
composition-fri-bad-with-verifier-tied-partial-candidate s out $\vee$
query-bad-with-aligned-transcript-partial-candidates s out

definition *soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
s out $\longleftrightarrow$
partial-merkle-inconsistency-bad s out $\vee$
soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
s out

lemma *accepted-fri-opening-transcript-obtains-aligned-partial-openings:*

assumes *fri-openings:*

accepted-fri-opening-transcript s (Some (result, final-state))
trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final query-idxs trace-round-layers
composition-round-layers

and *comp-nonempty:* *composition-roots* $\neq$ []

obtains *fr as rest trace-openings composition-openings* **where**

verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
accepted-with-partial-initial-openings-aligned-transcript-consistent s
(Some (result, final-state)) fr trace-roots trace-final as dg
composition-roots composition-final query-idxs trace-openings
composition-openings

proof –

from *fri-openings* **obtain** *result' final-state'* *fr f-fl as fl query-state*
raw-idxs query-chunks **where** *out-eq:*

Some (result, final-state) = Some (result', final-state')

and *trace-roots-eq:* *trace-roots = map snd f-fl*

and *comp-roots-eq:* *composition-roots = map snd fl*

and *header:*

verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final (PTranscript query-state)

and *query-state-state:*

PState query-state =
verifier-header-state s fr trace-roots trace-final as dg
composition-roots composition-final

and *query-state-counter:*

PQueryCounter query-state = PQueryCounter s

and *query-out:*

Some (result', final-state') ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr f-fl trace-final as fl
composition-final)
rounds)
query-state)

and *len-raw:* *length raw-idxs = rounds*

and *query-idxs-eq:*

query-idxs = map (λraw. index (to-nat raw)) raw-idxs

and *len-query-chunks:* *length query-chunks = rounds*

and *transcript-query:*

PTranscript query-state =
List.concat query-chunks @ PTranscript final-state'

and *query-chunk:*

$\bigwedge i. i < rounds \implies$
verifier-query-round-chunk (query-idxs ! i)
trace-roots composition-roots (query-chunks ! i)

```

and replay-lookup:
   $\wedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge
        (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
    unfolding accepted-fri-opening-transcript-def by blast
obtain b composition-root fl-tail where fl-eq:
  fl = (b, composition-root) # fl-tail
  using comp-nonempty comp-roots-eq by (cases fl) auto
have f-roots: map snd f-fl = trace-roots
  using trace-roots-eq by simp
have comp-roots: map snd fl = composition-roots
  using comp-roots-eq by simp
have result-eq: result' = result
  and final-state-eq: final-state' = final-state
  using out-eq by simp-all
have query-out':
  Some (result, final-state)  $\in$ 
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl trace-final as fl
            composition-final)
          rounds)
        query-state)
  using query-out result-eq final-state-eq by simp
have transcript-query':
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
  using transcript-query final-state-eq by simp
have query-chunk':
   $\wedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
  using query-chunk trace-roots-eq comp-roots-eq by simp
have replay-lookup':
   $\wedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge
        (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
    using replay-lookup final-state-eq by simp
from
  query-rounds-replay-accepted-with-partial-initial-openings-aligned-transcript-consistent
  [OF header comp-nonempty fl-eq f-roots comp-roots refl
    query-state-state query-state-counter query-out' len-raw query-idxs-eq

```

```

    len-query-chunks transcript-query' query-chunk' replay-lookup']
obtain trace-openings composition-openings where partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent s
  (Some (result, final-state)) fr trace-roots trace-final as dg
  composition-roots composition-final query-idxs trace-openings
  composition-openings
by blast
show ?thesis
by (rule that[OF header partial])
qed

lemma accepted-fri-opening-transcript-obtains-aligned-partial-candidates-or-partial-merkle:
assumes fri-openings:
  accepted-fri-opening-transcript s (Some (result, final-state))
  trace-roots trace-bs trace-final dg composition-roots composition-bs
  composition-final query-idxs trace-round-layers
  composition-round-layers
and comp-nonempty: composition-roots  $\neq$  []
shows
  partial-merkle-inconsistency-bad s (Some (result, final-state))  $\vee$ 
  ( $\exists$  fr as rest trace-openings composition-openings trace-table
   composition-table.
   verifier-header-transcript s fr trace-roots trace-final as dg
   composition-roots composition-final rest  $\wedge$ 
   accepted-with-partial-initial-openings-aligned s
   (Some (result, final-state)) fr trace-roots trace-final as dg
   composition-roots composition-final query-idxs trace-openings
   composition-openings  $\wedge$ 
   partial-trace-table-candidate trace-table trace-openings  $\wedge$ 
   partial-composition-table-candidate composition-table
   composition-openings)
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then show ?thesis by simp
next
case no-merkle: False
from accepted-fri-opening-transcript-obtains-aligned-partial-openings
  [OF fri-openings comp-nonempty]
obtain fr as rest trace-openings composition-openings where header:
  verifier-header-transcript s fr trace-roots trace-final as dg
  composition-roots composition-final rest
and partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent s
  (Some (result, final-state)) fr trace-roots trace-final as dg
  composition-roots composition-final query-idxs trace-openings
  composition-openings
by blast
have aligned:

```

```

    accepted-with-partial-initial-openings-aligned s
    (Some (result, final-state)) fr trace-roots trace-final as dg
    composition-roots composition-final query-idxs trace-openings
    composition-openings
  using partial
  by (blast dest:
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    accepted-with-partial-initial-openings-aligned-consistent-imp-aligned)
  obtain trace-table composition-table where trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  using
    accepted-with-partial-initial-openings-aligned-candidates-if-no-partial-merkle-bad
    [OF aligned no-merkle]
  by blast
  show ?thesis
  by (intro disjI2 exI conjI)
    (rule header, rule aligned, rule trace-candidate,
     rule composition-candidate)
qed

lemma accepted-verifier-tied-aligned-transcript-partial-candidate-partition:
  assumes false-statement:  $\neg$  exists-valid-trace
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final query-idxs trace-round-layers
    composition-round-layers
  and partial:
    accepted-with-partial-initial-openings-aligned-transcript-consistent s
    out fr trace-roots trace-final as fri-dg composition-roots
    composition-final
    query-idxs trace-openings composition-openings
  and header:
    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  shows
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
    s out
  proof (cases trace-table-low-degree trace-table)
  case False
  have aligned:

```

```

accepted-with-partial-initial-openings-aligned-consistent s out fr
  trace-roots trace-final as fri-dg composition-roots composition-final query-idxs
  trace-openings composition-openings
by (rule
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
  [OF partial])
have aligned0:
  accepted-with-partial-initial-openings-aligned s out fr trace-roots
  trace-final as fri-dg composition-roots composition-final query-idxs
  trace-openings composition-openings
by (rule accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
  [OF aligned])
have trace-partial:
  accepted-with-partial-trace-openings s out fr query-idxs
  trace-openings
by (rule accepted-with-partial-initial-openings-aligned-shapes(3)
  [OF aligned0])
have trace-fri-bad-with-header-tied-partial-candidate s out
  unfolding trace-fri-bad-with-header-tied-partial-candidate-def
by (intro exI conjI)
  (rule fri-openings, rule header, rule trace-partial,
   rule trace-candidate, rule False)
then show ?thesis
  unfolding
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-def
  by blast
next
case True
note trace-low = True
show ?thesis
proof (cases composition-table-low-degree maxDegree composition-table)
case False
have aligned:
  accepted-with-partial-initial-openings-aligned-consistent s out fr
  trace-roots trace-final as fri-dg composition-roots composition-final query-idxs
  trace-openings composition-openings
by (rule
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
  [OF partial])
have aligned0:
  accepted-with-partial-initial-openings-aligned s out fr trace-roots
  trace-final as fri-dg composition-roots composition-final query-idxs
  trace-openings composition-openings
by (rule
  accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
  [OF aligned])
have composition-fri-bad-with-verifier-tied-partial-candidate s out
  unfolding composition-fri-bad-with-verifier-tied-partial-candidate-def
by (intro exI conjI)

```



```

    (rule fri-openings, rule aligned0, rule trace-candidate,
     rule composition-candidate, rule trace-low, rule False)
  then show ?thesis
    unfolding
      soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-def
    by blast
next
case True
note composition-low = True
show ?thesis
proof (cases all-queries-consistent trace-table composition-table as)
  case False
  have query-bad-with-aligned-transcript-partial-candidates s out
    unfolding query-bad-with-aligned-transcript-partial-candidates-def
    using partial trace-candidate composition-candidate trace-low
      composition-low False
    by blast
  then show ?thesis
    unfolding
      soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-def
    by blast
next
case True
note all-queries = True
from trace-low obtain f where
  deg-f: degree f < clength
  and trace-table-eq: trace-table = map (poly f) eval-domain
  unfolding trace-table-low-degree-def by blast
have violated: violated-constraints f ≠ {}
  by (rule false-statement-violated-constraints
      [OF false-statement deg-f])
have aligned:
  accepted-with-partial-initial-openings-aligned-consistent s out fr
    trace-roots trace-final as fri-dg composition-roots composition-final query-idxs
    trace-openings composition-openings
  by (rule
      accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
      [OF partial])
have composition-bad-with-partial-candidates s out
  unfolding composition-bad-with-partial-candidates-def
  using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
    [OF aligned]
    trace-candidate composition-candidate deg-f trace-table-eq
    violated composition-low all-queries
  by blast
then show ?thesis
  unfolding
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-def
  by blast

```

qed
 qed
 qed

lemma *accepted-fri-opening-transcript-verifier-tied-partition-with-partial-merkle:*
assumes *false-statement:* $\neg$ *exists-valid-trace*
and *fri-openings:*
 accepted-fri-opening-transcript *s* (*Some* (*result*, *final-state*))
 trace-roots *trace-bs* *trace-final* *dg* *composition-roots* *composition-bs*
 composition-final *query-idxs* *trace-round-layers*
 composition-round-layers
and *comp-nonempty:* *composition-roots* $\neq$ []
shows
 soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
 s (*Some* (*result*, *final-state*))
proof –
from *accepted-fri-opening-transcript-obtains-aligned-partial-openings*
 [*OF* *fri-openings* *comp-nonempty*]
obtain *fr* as *rest* *trace-openings* *composition-openings* **where** *header:*
 verifier-header-transcript *s* *fr* *trace-roots* *trace-final* as *dg*
 composition-roots *composition-final* *rest*
and *partial:*
 accepted-with-partial-initial-openings-aligned-transcript-consistent *s*
 (*Some* (*result*, *final-state*)) *fr* *trace-roots* *trace-final* as *dg*
 composition-roots *composition-final* *query-idxs* *trace-openings*
 composition-openings
by *blast*
show *?thesis*
proof (*cases*
 partial-merkle-inconsistency-bad *s* (*Some* (*result*, *final-state*)))
case *True*
then show *?thesis*
 unfolding
 soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-def
 by *simp*
next
case *False*
have *aligned:*
 accepted-with-partial-initial-openings-aligned *s*
 (*Some* (*result*, *final-state*)) *fr* *trace-roots* *trace-final* as *dg*
 composition-roots *composition-final* *query-idxs* *trace-openings*
 composition-openings
using *partial*
by (*blast* *dest:*
 accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
 accepted-with-partial-initial-openings-aligned-consistent-imp-aligned)
from *accepted-with-partial-initial-openings-aligned-candidates-if-no-partial-merkle-bad*
 [*OF* *aligned* *False*]
obtain *trace-table* *composition-table* **where** *trace-candidate:*

```

    partial-trace-table-candidate trace-table trace-openings
  and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  by blast
have bad:
  soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
  s (Some (result, final-state))
  by (rule accepted-verifier-tied-aligned-transcript-partial-candidate-partition
    [OF false-statement fri-openings partial header trace-candidate
      composition-candidate])
then show ?thesis
  unfolding
  soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-def
  by simp
qed
qed

```

lemma *accepted-fri-opening-transcript-headerE*:

```

  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  obtains fr as rest where
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest
  using fri-openings
  unfolding accepted-fri-opening-transcript-def
  by blast

```

lemma *accepted-partition-soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-o*

```

  assumes false-statement:  $\neg$  exists-valid-trace
  and supp:  $out \in \text{set-dist } (\text{execute verify-monad } s)$ 
  and acc: accepted out
  shows
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
    s out  $\vee$ 
    soundness-bad-event-partial-candidate-empty-header s out

```

proof –

```

  from acc obtain result final-state where out-eq:
    out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
  from verify-monad-accepted-fri-opening-transcript[OF supp[unfolded out-eq]]
  obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final query-idxs trace-round-layers
    composition-round-layers
  where fri-openings:
    accepted-fri-opening-transcript s (Some (result, final-state))
    trace-roots trace-bs trace-final dg composition-roots composition-bs

```

```

    composition-final query-idxs trace-round-layers
    composition-round-layers
  by blast
show ?thesis
proof (cases composition-roots = [])
case False
have
  soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
  s (Some (result, final-state))
  by (rule
    accepted-fri-opening-transcript-verifier-tied-partition-with-partial-merkle
    [OF false-statement fri-openings False])
then show ?thesis
  unfolding out-eq by simp
next
case True
note comp-empty = True
from accepted-fri-opening-transcript-headerE[OF fri-openings]
obtain fr as rest where header:
  verifier-header-transcript s fr trace-roots trace-final as dg
  composition-roots composition-final rest
  by blast
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then have
  soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
  s (Some (result, final-state))
  unfolding
  soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-def
  by simp
then show ?thesis
  unfolding out-eq by simp
next
case False
from verify-monad-accepted-with-partial-trace-openings-for-header
  [OF supp[unfolded out-eq] header]
obtain trace-query-idxs trace-openings where trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr trace-query-idxs trace-openings
  by blast
obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
  using
    accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
    [OF trace-partial False]
  by blast
have accepted-out: accepted (Some (result, final-state))

```

```

    by (rule accepted-with-partial-trace-openings-imp-accepted
        [OF trace-partial])
  let ?composition-table = replicate (scale * clength) composition-final
  have empty-partial:
    accepted-with-empty-composition-header-candidates s
      (Some (result, final-state)) fr trace-roots trace-final as dg
      composition-final trace-query-idxs trace-openings trace-table
      ?composition-table
  unfolding accepted-with-empty-composition-header-candidates-def
  by (intro conjI exI[of - rest])
    (use accepted-out header comp-empty trace-partial trace-candidate
      in simp-all)
  have soundness-bad-event-partial-candidate-empty-header s
    (Some (result, final-state))
  by (rule accepted-empty-composition-header-candidate-partition
      [OF false-statement empty-partial])
  then show ?thesis
    unfolding out-eq by simp
qed
qed
qed

lemma soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-union-bound:
  assumes comp-bound:
    wp-event verify-monad
      (composition-bad-with-partial-candidates s) s ≤
      composition-error
  and trace-fri-bound:
    wp-event verify-monad
      (trace-fri-bad-with-header-tied-partial-candidate s) s ≤
      trace-fri-error'
  and comp-fri-bound:
    wp-event verify-monad
      (composition-fri-bad-with-verifier-tied-partial-candidate s) s ≤
      composition-fri-error'
  and query-bound:
    wp-event verify-monad
      (query-bad-with-aligned-transcript-partial-candidates s) s ≤
      query-error
  shows
    wp-event verify-monad
      (soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
        s) s ≤
      composition-error + trace-fri-error' + composition-fri-error' +
      query-error
proof -
  have wp-event verify-monad
    (soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
      s) s ≤

```

```

wp-event verify-monad
  (composition-bad-with-partial-candidates s) s +
wp-event verify-monad
  (trace-fri-bad-with-header-tied-partial-candidate s) s +
wp-event verify-monad
  (composition-fri-bad-with-verifier-tied-partial-candidate s) s +
wp-event verify-monad
  (query-bad-with-aligned-transcript-partial-candidates s) s
unfolding
  soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-def
by (rule wp-event-union-bound4)
also have ... ≤
  composition-error + trace-fri-error' + composition-fri-error' +
  query-error
by (intro add-mono comp-bound trace-fri-bound comp-fri-bound
  query-bound)
finally show ?thesis .
qed

lemma soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-union-bound:
assumes merkle-bound:
  wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤
  merkle-error
and comp-bound:
  wp-event verify-monad
    (composition-bad-with-partial-candidates s) s ≤
    composition-error
and trace-fri-bound:
  wp-event verify-monad
    (trace-fri-bad-with-header-tied-partial-candidate s) s ≤
    trace-fri-error'
and comp-fri-bound:
  wp-event verify-monad
    (composition-fri-bad-with-verifier-tied-partial-candidate s) s ≤
    composition-fri-error'
and query-bound:
  wp-event verify-monad
    (query-bad-with-aligned-transcript-partial-candidates s) s ≤
    query-error
shows
  wp-event verify-monad
    (soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
    s) s ≤
    merkle-error + composition-error + trace-fri-error' +
    composition-fri-error' + query-error
proof –
have partial-bound:
  wp-event verify-monad
    (soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate

```

```

    s) s ≤
    composition-error + trace-fri-error' + composition-fri-error' +
    query-error
  by (rule
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-union-bound
    [OF comp-bound trace-fri-bound comp-fri-bound query-bound])
  have wp-event verify-monad
    (soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
    s) s ≤
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s +
    wp-event verify-monad
    (soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
    s) s
  unfolding
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-def
  by (rule wp-event-union-bound)
  also have ... ≤
    merkle-error +
    (composition-error + trace-fri-error' + composition-fri-error' +
    query-error)
  by (intro add-mono merkle-bound partial-bound)
  finally show ?thesis
  by (simp add: algebra-simps)
qed

```

lemma *checked-staged-security-with-data-state-verifier-tied-aligned-transcript-partial-candidate-union-bound:*

assumes *comp-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-bad-with-partial-candidates)
adversary-initial-state ≤ composition-error

and *trace-fri-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state ≤ trace-fri-error'

and *comp-fri-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state ≤ composition-fri-error'

and *query-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state ≤ query-error

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event

```

    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate)
  adversary-initial-state ≤
  composition-error + trace-fri-error' + composition-fri-error' +
  query-error
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?comp = ?E composition-bad-with-partial-candidates
  let ?trace = ?E trace-fri-bad-with-header-tied-partial-candidate
  let ?comp-fri =
    ?E composition-fri-bad-with-verifier-tied-partial-candidate
  let ?query = ?E query-bad-with-aligned-transcript-partial-candidates
  let ?partial =
    ?E soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
  have wp-event ?M ?partial adversary-initial-state ≤
    wp-event ?M
    (λout. ?comp out ∨ ?trace out ∨ ?comp-fri out ∨
    ?query out)
    adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-def
    split: option.splits prod.splits)
  also have ... ≤
    wp-event ?M ?comp adversary-initial-state +
    wp-event ?M ?trace adversary-initial-state +
    wp-event ?M ?comp-fri adversary-initial-state +
    wp-event ?M ?query adversary-initial-state
  by (rule wp-event-union-bound4)
  also have ... ≤
    composition-error + trace-fri-error' + composition-fri-error' +
    query-error
  by (intro add-mono comp-bound trace-fri-bound comp-fri-bound
    query-bound)
  finally show ?thesis .
qed

```

lemma *checked-staged-security-with-data-state-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-u*

assumes merkle-bound:

```

  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
  partial-merkle-inconsistency-bad)
  adversary-initial-state ≤ merkle-error

```

and comp-bound:

```

  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
  composition-bad-with-partial-candidates)
  adversary-initial-state ≤ composition-error

```

and trace-fri-bound:


```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-bad-with-verifier-tied-partial-candidate)
  adversary-initial-state  $\leq$  composition-fri-error'
and query-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    query-bad-with-aligned-transcript-partial-candidates)
  adversary-initial-state  $\leq$  query-error
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle)
  adversary-initial-state  $\leq$ 
    merkle-error + composition-error + trace-fri-error' +
    composition-fri-error' + query-error
proof –
have partial-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate)
  adversary-initial-state  $\leq$ 
    composition-error + trace-fri-error' + composition-fri-error' +
    query-error
by (rule
  checked-staged-security-with-data-state-verifier-tied-aligned-transcript-partial-candidate-union-bound
    [OF comp-bound trace-fri-bound comp-fri-bound query-bound])
let ?M = checked-staged-security-experiment-with-data-state A
let ?E = staged-security-with-data-state-verifier-event
let ?merkle = ?E partial-merkle-inconsistency-bad
let ?partial =
  ?E soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
let ?with-merkle =
  ?E soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
have wp-event ?M ?with-merkle adversary-initial-state  $\leq$ 
  wp-event ?M ( $\lambda$ out. ?merkle out  $\vee$  ?partial out)
  adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-def
    split: option.splits prod.splits)
also have ...  $\leq$ 
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?partial adversary-initial-state

```

```

    by (rule wp-event-union-bound)
  also have ...  $\leq$ 
    merkle-error +
    (composition-error + trace-fri-error' + composition-fri-error' +
     query-error)
  by (intro add-mono merkle-bound partial-bound)
  finally show ?thesis
    by (simp add: algebra-simps)
qed

lemma checked-staged-soundness-from-verifier-tied-aligned-transcript-partial-candidate-and-empty-header-bounds:
  fixes partial-candidate-error empty-header-error :: prob
  assumes false-statement:  $\neg$  exists-valid-trace
  and partial-candidate-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle)
      adversary-initial-state  $\leq$  partial-candidate-error
  and empty-header-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       soundness-bad-event-partial-candidate-empty-header)
      adversary-initial-state  $\leq$  empty-header-error
  shows checked-staged-adversary-acceptance-probability A  $\leq$ 
    partial-candidate-error + empty-header-error
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?partial =
    ?E soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
  let ?empty = ?E soundness-bad-event-partial-candidate-empty-header
  let ?bad =  $\lambda$ out. ?partial out  $\vee$  ?empty out
  have accepted-bad:
    wp-event ?M accepted adversary-initial-state  $\leq$ 
    wp-event ?M ?bad adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support: out  $\in$  set-dist (execute ?M adversary-initial-state)
    and acc: accepted out
    show ?bad out
    proof (cases out)
      case None
      then show ?thesis
        using acc unfolding accepted-def by simp
    next
      case (Some packed)
      then obtain data attacker-state result final-state where out-eq:
        out = Some (((data, attacker-state), result), final-state)
      by (cases packed, auto split: prod.splits)
    end
  end
end

```

```

let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  using checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
  by blast
have verifier-acc: accepted (Some (result, final-state))
  unfolding accepted-def by simp
have local-bad:
  soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
    ?s (Some (result, final-state)) ∨
  soundness-bad-event-partial-candidate-empty-header ?s
    (Some (result, final-state))
  by (rule
    accepted-partition-soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-or-
    [OF false-statement verifier verifier-acc])
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
have bad-bound:
  wp-event ?M ?bad adversary-initial-state ≤
  partial-candidate-error + empty-header-error
proof -
  have wp-event ?M ?bad adversary-initial-state ≤
    wp-event ?M ?partial adversary-initial-state +
    wp-event ?M ?empty adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤ partial-candidate-error + empty-header-error
  by (intro add-mono partial-candidate-bound empty-header-bound)
  finally show ?thesis .
qed
have staged-bound:
  wp-event (checked-staged-security-experiment A) accepted
  adversary-initial-state ≤
  partial-candidate-error + empty-header-error
  unfolding checked-staged-security-experiment-acceptance-with-data-state
  by (rule order-trans[OF accepted-bad bad-bound])
show ?thesis
proof -
  have
    wp-event (checked-staged-security-experiment A)
    (λout. ¬ Option.is-none out) adversary-initial-state =
    wp-event (checked-staged-security-experiment A) accepted
    adversary-initial-state
  unfolding wp-event-def accepted-def

```

```

    by (rule arg-cong[where
      f= $\lambda Q. wp \text{ (checked-staged-security-experiment } A) \text{ } Q$ 
      adversary-initial-state])
      (rule ext, simp)
  then show ?thesis
    unfolding checked-staged-adversary-acceptance-probability-def
    using staged-bound by simp
qed
qed

lemma checked-staged-soundness-from-verifier-tied-aligned-transcript-partial-candidate-components-and-empty-header
  fixes merkle-error composition-error trace-fri-error'
    composition-fri-error' query-error empty-header-error :: prob
  assumes false-statement:  $\neg \text{exists-valid-trace}$ 
  and merkle-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        partial-merkle-inconsistency-bad)
    adversary-initial-state  $\leq$  merkle-error
  and comp-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  composition-error
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-header-tied-partial-candidate)
    adversary-initial-state  $\leq$  trace-fri-error'
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-verifier-tied-partial-candidate)
    adversary-initial-state  $\leq$  composition-fri-error'
  and query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        query-bad-with-aligned-transcript-partial-candidates)
    adversary-initial-state  $\leq$  query-error
  and empty-header-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-partial-candidate-empty-header)
    adversary-initial-state  $\leq$  empty-header-error
  shows checked-staged-adversary-acceptance-probability A  $\leq$ 
    merkle-error + composition-error + trace-fri-error' +
    composition-fri-error' + query-error + empty-header-error
proof -
  have partial-candidate-bound:

```

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle)
adversary-initial-state ≤
merkle-error + composition-error + trace-fri-error' +
composition-fri-error' + query-error
by (rule
checked-staged-security-with-data-state-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-union
[OF merkle-bound comp-bound trace-fri-bound comp-fri-bound
query-bound])
have bound:
checked-staged-adversary-acceptance-probability A ≤
(merkle-error + composition-error + trace-fri-error' +
composition-fri-error' + query-error) + empty-header-error
by (rule
checked-staged-soundness-from-verifier-tied-aligned-transcript-partial-candidate-and-empty-header-bounds
[OF false-statement partial-candidate-bound empty-header-bound])
then show ?thesis
by (simp add: algebra-simps)
qed

lemma *trace-fri-bad-with-verifier-tied-partial-candidateE:*
assumes
trace-fri-bad-with-verifier-tied-partial-candidate s out
obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-openings trace-table
where
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
partial-trace-table-candidate trace-table trace-openings
¬ trace-table-low-degree trace-table
using *assms*
unfolding *trace-fri-bad-with-verifier-tied-partial-candidate-def*
by *blast*

lemma *composition-fri-bad-with-verifier-tied-partial-candidateE:*
assumes
composition-fri-bad-with-verifier-tied-partial-candidate s out
obtains *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table
where
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs

```

    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  accepted-with-partial-initial-openings-aligned s out fr trace-roots
    trace-final as fri-dg composition-roots composition-final fri-query-idxs
    trace-openings composition-openings
  partial-trace-table-candidate trace-table trace-openings
  partial-composition-table-candidate composition-table
    composition-openings
  trace-table-low-degree trace-table
  ¬ composition-table-low-degree maxDegree composition-table
proof –
obtain trace-roots trace-bs trace-final fri-dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr as trace-openings composition-openings
  trace-table composition-table
where fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
and aligned:
  accepted-with-partial-initial-openings-aligned s out fr trace-roots
    trace-final as fri-dg composition-roots composition-final fri-query-idxs
    trace-openings composition-openings
and trace-cand:
  partial-trace-table-candidate trace-table trace-openings
and comp-cand:
  partial-composition-table-candidate composition-table
    composition-openings
and trace-low:
  trace-table-low-degree trace-table
and comp-not-low:
  ¬ composition-table-low-degree maxDegree composition-table
using assms
unfolding composition-fri-bad-with-verifier-tied-partial-candidate-def
by metis
show ?thesis
by (rule that[OF fri-openings aligned trace-cand comp-cand trace-low
  comp-not-low])
qed

```

```

lemma trace-fri-bad-with-verifier-tied-imp-reachable-partial-candidate:
assumes
  trace-fri-bad-with-verifier-tied-partial-candidate s out
shows trace-fri-bad-with-reachable-partial-candidate s out
proof (rule trace-fri-bad-with-verifier-tied-partial-candidateE[OF assms])
fix trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr trace-openings trace-table

```

```

assume partial:
  accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
and cand:
  partial-trace-table-candidate trace-table trace-openings
and not-low:  $\neg$  trace-table-low-degree trace-table
show ?thesis
  unfolding trace-fri-bad-with-reachable-partial-candidate-def
  by (intro exI conjI; (rule partial | rule cand | rule not-low))
qed

lemma composition-fri-bad-with-verifier-tied-imp-reachable-partial-candidate:
assumes
  composition-fri-bad-with-verifier-tied-partial-candidate s out
shows composition-fri-bad-with-reachable-partial-candidate s out
proof (rule
  composition-fri-bad-with-verifier-tied-partial-candidateE[OF assms])
fix trace-roots trace-bs trace-final fri-dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr as trace-openings composition-openings
  trace-table composition-table
assume aligned:
  accepted-with-partial-initial-openings-aligned s out fr trace-roots
  trace-final as fri-dg composition-roots composition-final fri-query-idxs
  trace-openings composition-openings
and trace-cand:
  partial-trace-table-candidate trace-table trace-openings
and comp-cand:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-not-low:
   $\neg$  composition-table-low-degree maxDegree composition-table
have unaligned:
  accepted-with-partial-initial-openings s out fr trace-roots
  trace-final as fri-dg composition-roots composition-final
  fri-query-idxs trace-openings fri-query-idxs composition-openings
by (rule accepted-with-partial-initial-openings-aligned-imp-unaligned
  [OF aligned])
show ?thesis
  unfolding composition-fri-bad-with-reachable-partial-candidate-def
  by (intro exI conjI; (rule unaligned | rule trace-cand |
    rule comp-cand | rule trace-low | rule comp-not-low))
qed

lemma trace-fri-bad-with-verifier-tied-partial-candidate-None[simp]:
 $\neg$  trace-fri-bad-with-verifier-tied-partial-candidate s None
unfolding trace-fri-bad-with-verifier-tied-partial-candidate-def
  accepted-fri-opening-transcript-def

```

by *simp*

lemma *composition-fri-bad-with-verifier-tied-partial-candidate-None*[*simp*]:

¬ *composition-fri-bad-with-verifier-tied-partial-candidate* *s* *None*

unfolding *composition-fri-bad-with-verifier-tied-partial-candidate-def*
accepted-fri-opening-transcript-def

by *simp*

lemma *trace-fri-verifier-tied-imp-sampled-or-missing-layer-chain*:

assumes *trace-fri-bad-with-verifier-tied-partial-candidate* *s* *out*

shows

trace-fri-bad-with-verifier-tied-sampled-layer-chain *s* *out* ∨

trace-fri-verifier-tied-missing-sampled-layer-chain *s* *out*

using *assms*

unfolding *trace-fri-verifier-tied-missing-sampled-layer-chain-def*

by *blast*

lemma *composition-fri-verifier-tied-imp-sampled-or-missing-layer-chain*:

assumes *composition-fri-bad-with-verifier-tied-partial-candidate* *s* *out*

shows

composition-fri-bad-with-verifier-tied-sampled-layer-chain *s* *out* ∨

composition-fri-verifier-tied-missing-sampled-layer-chain *s* *out*

using *assms*

unfolding *composition-fri-verifier-tied-missing-sampled-layer-chain-def*

by *blast*

lemma *wp-trace-fri-verifier-tied-bound-from-sampled-and-missing*:

assumes *sampled-bound*:

wp-event *verify-monad*

(*trace-fri-bad-with-verifier-tied-sampled-layer-chain* *s*) $s \leq R$

and *missing-bound*:

wp-event *verify-monad*

(*trace-fri-verifier-tied-missing-sampled-layer-chain* *s*) $s \leq M$

shows

wp-event *verify-monad*

(*trace-fri-bad-with-verifier-tied-partial-candidate* *s*) $s \leq R + M$

proof –

have *wp-event* *verify-monad*

(*trace-fri-bad-with-verifier-tied-partial-candidate* *s*) $s \leq$

wp-event *verify-monad*

($\lambda out. trace-fri-bad-with-verifier-tied-sampled-layer-chain$ *s* *out* ∨

trace-fri-verifier-tied-missing-sampled-layer-chain *s* *out*) *s*

by (rule *wp-event-mono*)

(rule *trace-fri-verifier-tied-imp-sampled-or-missing-layer-chain*)

also have ... $\leq$

wp-event *verify-monad*

(*trace-fri-bad-with-verifier-tied-sampled-layer-chain* *s*) $s +$

wp-event *verify-monad*

(*trace-fri-verifier-tied-missing-sampled-layer-chain* *s*) *s*

by (rule wp-event-union-bound)
 also have ... $\leq R + M$
 by (rule add-mono[OF sampled-bound missing-bound])
 finally show ?thesis .
 qed

lemma wp-composition-fri-verifier-tied-bound-from-sampled-and-missing:
 assumes sampled-bound:
 wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s $\leq R$
 and missing-bound:
 wp-event verify-monad
 (composition-fri-verifier-tied-missing-sampled-layer-chain s) s $\leq M$
 shows
 wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-partial-candidate s) s $\leq$
 R + M
proof –
 have wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-partial-candidate s) s $\leq$
 wp-event verify-monad
 (λ out.
 composition-fri-bad-with-verifier-tied-sampled-layer-chain s out $\vee$
 composition-fri-verifier-tied-missing-sampled-layer-chain s out) s
 by (rule wp-event-mono)
 (rule composition-fri-verifier-tied-imp-sampled-or-missing-layer-chain)
 also have ... $\leq$
 wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s +
 wp-event verify-monad
 (composition-fri-verifier-tied-missing-sampled-layer-chain s) s
 by (rule wp-event-union-bound)
 also have ... $\leq R + M$
 by (rule add-mono[OF sampled-bound missing-bound])
 finally show ?thesis .
 qed

definition trace-fri-verifier-tied-sampled-layer-assignment-obstruction
 :: ('f, 'a) protocol-channel-scheme $\Rightarrow$
 (unit list $\times$ ('f, 'a) protocol-channel-scheme) option $\Rightarrow$ bool
where
 trace-fri-verifier-tied-sampled-layer-assignment-obstruction s out
 $\longleftrightarrow$
 ($\exists$ trace-roots trace-bs trace-final dg composition-roots
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr trace-openings trace-table.
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final dg composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers $\wedge$

accepted-with-partial-trace-openings *s out fr fri-query-idxs*
trace-openings $\wedge$
partial-trace-table-candidate *trace-table trace-openings* $\wedge$
 $\neg$ *trace-table-low-degree* *trace-table* $\wedge$
generic-fri-sampled-layer-assignment-obstruction
trace-table-low-degree *trace-table* (*clength* - 1) *trace-roots*
trace-bs trace-final fri-query-idxs trace-round-layers)

definition *trace-fri-verifier-tied-sampled-assignment-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-verifier-tied-sampled-assignment-conflict *s out*
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr trace-openings trace-table.}$
accepted-fri-opening-transcript *s out trace-roots trace-bs*
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers} \wedge$
accepted-with-partial-trace-openings *s out fr fri-query-idxs*
trace-openings $\wedge$
partial-trace-table-candidate *trace-table trace-openings* $\wedge$
 $\neg$ *trace-table-low-degree* *trace-table* $\wedge$
generic-fri-sampled-assignment-conflict *trace-table trace-roots*
trace-bs trace-final fri-query-idxs trace-round-layers)

definition *trace-fri-verifier-tied-zero-round-final-obstruction*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-verifier-tied-zero-round-final-obstruction *s out*
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr trace-openings trace-table.}$
accepted-fri-opening-transcript *s out trace-roots trace-bs*
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers} \wedge$
accepted-with-partial-trace-openings *s out fr fri-query-idxs*
trace-openings $\wedge$
partial-trace-table-candidate *trace-table trace-openings* $\wedge$
 $\neg$ *trace-table-low-degree* *trace-table* $\wedge$
 $\text{length trace-bs} = 0 \wedge$
 $\neg \text{ fri-final-constant-consistent trace-table trace-final}$)

definition *composition-fri-verifier-tied-sampled-layer-assignment-obstruction*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-sampled-layer-assignment-obstruction s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table.}$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{ accepted-with-partial-initial-openings-aligned s out fr trace-roots}$
 $\text{ trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{ trace-openings composition-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{ trace-table-low-degree trace-table} \wedge$
 $\neg \text{ composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{ generic-fri-sampled-layer-assignment-obstruction}$
 $(\text{ composition-table-low-degree (to-nat fri-dg)}) \text{ composition-table}$
 $(\text{ to-nat fri-dg}) \text{ composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs composition-round-layers})$

definition *composition-fri-verifier-tied-sampled-assignment-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-sampled-assignment-conflict s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table.}$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{ accepted-with-partial-initial-openings-aligned s out fr trace-roots}$
 $\text{ trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{ trace-openings composition-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{ trace-table-low-degree trace-table} \wedge$
 $\neg \text{ composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{ generic-fri-sampled-assignment-conflict composition-table}$
 $\text{ composition-roots composition-bs composition-final fri-query-idxs}$
 $\text{ composition-round-layers})$

definition *composition-fri-verifier-tied-zero-round-final-obstruction*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-zero-round-final-obstruction s out

$\longleftrightarrow$

$(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table.}$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{ accepted-with-partial-initial-openings-aligned s out fr trace-roots}$
 $\text{ trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{ trace-openings composition-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{ trace-table-low-degree trace-table} \wedge$
 $\neg \text{ composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{ length composition-bs} = 0 \wedge$
 $\neg \text{ fri-final-constant-consistent composition-table}$
 $\text{ composition-final})$

lemma *trace-fri-header-tied-zero-round-final-obstruction-imp-verifier-tied:*

assumes *trace-fri-header-tied-zero-round-final-obstruction s out*

shows *trace-fri-verifier-tied-zero-round-final-obstruction s out*

using *assms*

unfolding *trace-fri-header-tied-zero-round-final-obstruction-def*

trace-fri-verifier-tied-zero-round-final-obstruction-def

by *blast*

lemma *composition-fri-verifier-tied-zero-round-final-obstruction-false:*

$\neg \text{ composition-fri-verifier-tied-zero-round-final-obstruction s out}$

proof

assume *obstruction:*

composition-fri-verifier-tied-zero-round-final-obstruction s out

then obtain *trace-roots trace-bs trace-final fri-dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as trace-openings composition-openings

trace-table composition-table

where *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers

and *aligned*:
accepted-with-partial-initial-openings-aligned *s* out fr *trace-roots*
trace-final as *fri-dg* *composition-roots* *composition-final* *fri-query-idxs*
trace-openings *composition-openings*
and *zero*: *length* *composition-bs* = 0
unfolding *composition-fri-verifier-tied-zero-round-final-obstruction-def*
by *blast*
have *nonempty*: *composition-roots* ≠ []
by (*rule* *accepted-with-partial-initial-openings-aligned-shapes*(1)
[*OF aligned*])
have *composition-roots* = []
using *accepted-fri-opening-transcript-shapes*(4)[*OF fri-openings*] *zero*
by *simp*
then show *False*
using *nonempty* **by** *simp*
qed

lemma *wp-composition-fri-verifier-tied-zero-round-final-obstruction-zero*:
wp-event *verify-monad*
(*composition-fri-verifier-tied-zero-round-final-obstruction* *s*) *s* = 0
unfolding *wp-event-def* *wp-def* *dist-expect-def*
by (*simp* *add*:
composition-fri-verifier-tied-zero-round-final-obstruction-false)

lemma *trace-fri-verifier-tied-assignment-obstruction-imp-conflict-or-zero*:
assumes *obstruction*:
trace-fri-verifier-tied-sampled-layer-assignment-obstruction *s* out
shows
trace-fri-verifier-tied-sampled-assignment-conflict *s* out ∨
trace-fri-verifier-tied-zero-round-final-obstruction *s* out
proof —
obtain *trace-roots* *trace-bs'* *trace-final* *dg* *composition-roots*
composition-bs *composition-final* *fri-query-idxs* *trace-round-layers*
composition-round-layers fr *trace-openings* *trace-table*
where *fri-openings*:
accepted-fri-opening-transcript *s* out *trace-roots* *trace-bs'*
trace-final *dg* *composition-roots* *composition-bs* *composition-final*
fri-query-idxs *trace-round-layers* *composition-round-layers*
and *partial*:
accepted-with-partial-trace-openings *s* out fr *fri-query-idxs*
trace-openings
and *cand*: *partial-trace-table-candidate* *trace-table* *trace-openings*
and *not-low*: ¬ *trace-table-low-degree* *trace-table*
and *generic*:
generic-fri-sampled-layer-assignment-obstruction
trace-table-low-degree *trace-table* (*clength* — 1) *trace-roots*
trace-bs' *trace-final* *fri-query-idxs* *trace-round-layers*
using *obstruction*
unfolding *trace-fri-verifier-tied-sampled-layer-assignment-obstruction-def*

```

    by blast
  obtain  $N$  where  $eval\_power: clength * scale = 2 \wedge N$ 
    using  $eval\_domain\_length\_power$  by blast
  have  $trace\_rounds\_le: length\ trace\_bs' \leq N$ 
  proof -
    have  $len: length\ trace\_bs' = ceil\_log\ clength$ 
      using  $accepted\_fri\_opening\_transcript\_shapes(1,2)[OF\ fri\_openings]$ 
      by simp
    have  $clength \leq clength * scale$ 
      using  $scale\_pos$  by simp
    also have  $\dots = 2 \wedge N$ 
      by (rule  $eval\_power$ )
    finally have  $ceil\_log\ clength \leq N$ 
      by (rule  $ceil\_log\_le\_power$ )
    then show ?thesis
      using  $len$  by simp
  qed
  have  $query\_bounds'$ :
     $\bigwedge round\_idx. round\_idx < length\ fri\_query\_idxs \implies$ 
     $fri\_query\_idxs ! round\_idx < clength * scale$ 
  proof -
    fix  $round\_idx$ 
    assume  $round\_bound: round\_idx < length\ fri\_query\_idxs$ 
    have  $fri\_query\_idxs ! round\_idx \in set\ fri\_query\_idxs$ 
      using  $round\_bound$  by simp
    then show  $fri\_query\_idxs ! round\_idx < clength * scale$ 
      by (rule  $accepted\_fri\_opening\_transcript\_query\_idx\_bound$ 
        [OF  $fri\_openings$ ])
  qed
  have  $split$ :
     $generic\_fri\_sampled\_assignment\_conflict\ trace\_table\ trace\_roots$ 
     $trace\_bs'\ trace\_final\ fri\_query\_idxs\ trace\_round\_layers \vee$ 
     $length\ trace\_bs' = 0 \wedge$ 
     $\neg fri\_final\_constant\_consistent\ trace\_table\ trace\_final$ 
  by (rule
     $generic\_fri\_sampled\_layer\_assignment\_obstruction\_imp\_conflict\_or\_zero\_round\_final$ 
    [OF  $generic\ query\_bounds'\ eval\_power\ trace\_rounds\_le$ ])
  then show ?thesis
  proof
    assume  $conflict$ :
       $generic\_fri\_sampled\_assignment\_conflict\ trace\_table\ trace\_roots$ 
       $trace\_bs'\ trace\_final\ fri\_query\_idxs\ trace\_round\_layers$ 
    then show ?thesis
      unfolding  $trace\_fri\_verifier\_tied\_sampled\_assignment\_conflict\_def$ 
      by (intro  $disjI1\ exI\ conjI$ )
      (rule  $fri\_openings, rule\ partial, rule\ cand, rule\ not\_low,$ 
        rule  $conflict$ )
  next
    assume  $zero$ :

```

```

length trace-bs' = 0 ∧
  ¬ fri-final-constant-consistent trace-table trace-final
then show ?thesis
  unfolding trace-fri-verifier-tied-zero-round-final-obstruction-def
  by (intro disjI2 exI conjI)
    (rule fri-openings, rule partial, rule cand, rule not-low,
     simp-all)
qed
qed

lemma composition-fri-verifier-tied-assignment-obstruction-imp-conflict-or-zero-on-support:
  assumes support: out ∈ set-dist (execute verify-monad s)
  and obstruction:
    composition-fri-verifier-tied-sampled-layer-assignment-obstruction
      s out
  shows
    composition-fri-verifier-tied-sampled-assignment-conflict s out ∨
    composition-fri-verifier-tied-zero-round-final-obstruction s out
proof —
  obtain trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as trace-openings composition-openings
    trace-table composition-table
  where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  and aligned:
    accepted-with-partial-initial-openings-aligned s out fr trace-roots
    trace-final as fri-dg composition-roots composition-final fri-query-idxs
    trace-openings composition-openings
  and trace-cand:
    partial-trace-table-candidate trace-table trace-openings
  and comp-cand:
    partial-composition-table-candidate composition-table
    composition-openings
  and trace-low: trace-table-low-degree trace-table
  and comp-not-low:
    ¬ composition-table-low-degree maxDegree composition-table
  and generic:
    generic-fri-sampled-layer-assignment-obstruction
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers
  using obstruction
  unfolding
    composition-fri-verifier-tied-sampled-layer-assignment-obstruction-def
  by metis

```

```

obtain  $N$  where  $eval\_power: length * scale = 2 \wedge N$ 
  using  $eval\_domain\_length\_power$  by  $blast$ 
obtain  $result\ final\_state$  where  $out\_eq: out = Some\ (result, final\_state)$ 
  using  $fri\_openings$  unfolding  $accepted\_fri\_opening\_transcript\_def$  by  $blast$ 
have  $outcome:$ 
   $Some\ (result, final\_state) \in set\_dist\ (execute\ verify\_monad\ s)$ 
  using  $support\ out\_eq$  by  $simp$ 
have  $challenges:$ 
   $accepted\_fri\_challenges\ s\ (Some\ (result, final\_state))$ 
   $trace\_bs\ fri\_dg\ composition\_bs$ 
  using  $accepted\_fri\_opening\_transcript\_challenges[OF\ fri\_openings]$ 
   $out\_eq$  by  $simp$ 
have  $degree\_bound: to\_nat\ fri\_dg \leq maxDegree$ 
  by  $(rule\ verify\_monad\_accepted\_fri\_challenges\_degree\_bound$ 
     $[OF\ outcome\ challenges])$ 
have  $composition\_rounds\_le: length\ composition\_bs \leq N$ 
proof –
  have  $len: length\ composition\_bs = ceil\_log\ (to\_nat\ fri\_dg + 1)$ 
    using  $accepted\_fri\_opening\_transcript\_shapes(3,4)[OF\ fri\_openings]$ 
    by  $simp$ 
  have  $to\_nat\ fri\_dg + 1 \leq length * scale$ 
    using  $degree\_bound\ maxDegree\_less\_eval\_domain$  by  $linarith$ 
  also have  $\dots = 2 \wedge N$ 
    by  $(rule\ eval\_power)$ 
  finally have  $ceil\_log\ (to\_nat\ fri\_dg + 1) \leq N$ 
    by  $(rule\ ceil\_log\_le\_power)$ 
  then show  $?thesis$ 
    using  $len$  by  $simp$ 
qed
have  $query\_bounds:$ 
   $\bigwedge round\_idx. round\_idx < length\ fri\_query\_idxs \implies$ 
   $fri\_query\_idxs\ !\ round\_idx < length * scale$ 
proof –
  fix  $round\_idx$ 
  assume  $round\_bound: round\_idx < length\ fri\_query\_idxs$ 
  have  $fri\_query\_idxs\ !\ round\_idx \in set\ fri\_query\_idxs$ 
    using  $round\_bound$  by  $simp$ 
  then show  $fri\_query\_idxs\ !\ round\_idx < length * scale$ 
    by  $(rule\ accepted\_fri\_opening\_transcript\_query\_idx\_bound$ 
       $[OF\ fri\_openings])$ 
qed
have  $split:$ 
   $generic\_fri\_sampled\_assignment\_conflict\ composition\_table$ 
   $composition\_roots\ composition\_bs\ composition\_final\ fri\_query\_idxs$ 
   $composition\_round\_layers \vee$ 
   $length\ composition\_bs = 0 \wedge$ 
   $\neg\ fri\_final\_constant\_consistent\ composition\_table$ 
   $composition\_final$ 
by  $(rule$ 

```



```

    generic-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final
    [OF generic query-bounds eval-power composition-rounds-le])
then show ?thesis
proof
  assume conflict:
    generic-fri-sampled-assignment-conflict composition-table
    composition-roots composition-bs composition-final fri-query-idxs
    composition-round-layers
  then show ?thesis
  unfolding
    composition-fri-verifier-tied-sampled-assignment-conflict-def
  by (intro disjI1 exI conjI)
    (rule fri-openings, rule aligned, rule trace-cand, rule comp-cand,
     rule trace-low, rule comp-not-low, rule conflict)
next
  assume zero:
    length composition-bs = 0 ∧
    ¬ fri-final-constant-consistent composition-table
    composition-final
  then show ?thesis
  unfolding
    composition-fri-verifier-tied-zero-round-final-obstruction-def
  by (intro disjI2 exI conjI)
    (rule fri-openings, rule aligned, rule trace-cand, rule comp-cand,
     rule trace-low, rule comp-not-low, simp-all)
qed
qed

lemma trace-fri-verifier-tied-missing-sampled-imp-assignment-obstruction:
  assumes missing:
    trace-fri-verifier-tied-missing-sampled-layer-chain s out
  shows
    trace-fri-verifier-tied-sampled-layer-assignment-obstruction s out
proof –
  have bad:
    trace-fri-bad-with-verifier-tied-partial-candidate s out
  and not-sampled:
    ¬ trace-fri-bad-with-verifier-tied-sampled-layer-chain s out
  using missing
  unfolding trace-fri-verifier-tied-missing-sampled-layer-chain-def
  by blast+
from trace-fri-bad-with-verifier-tied-partial-candidateE[OF bad]
obtain trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr trace-openings trace-table
where fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final dg composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers

```

and *partial*:
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *cand*: *partial-trace-table-candidate trace-table trace-openings*
and *not-low*: $\neg$ *trace-table-low-degree trace-table*
by *blast*
have *partial-generic*:
generic-fri-partial-evidence trace-table-low-degree trace-table
(clength - 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers
by (rule *accepted-fri-opening-transcript-trace-generic-partial-evidence*
 $[OF\ fri-openings]$)
have *no-chain*:
 $\bigwedge doms\ layers. \neg$
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
proof
fix *doms layers*
assume *chain*:
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
have *trace-fri-bad-with-verifier-tied-sampled-layer-chain s out*
unfolding *trace-fri-bad-with-verifier-tied-sampled-layer-chain-def*
by (intro *exI conjI*)
(rule *fri-openings*, rule *partial*, rule *cand*, rule *not-low*,
rule *chain*)
then show *False*
using *not-sampled by contradiction*
qed
have *obstruction*:
generic-fri-sampled-layer-assignment-obstruction
trace-table-low-degree trace-table (clength - 1) trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers
unfolding *generic-fri-sampled-layer-assignment-obstruction-def*
using *partial-generic no-chain by blast*
show *?thesis*
unfolding *trace-fri-verifier-tied-sampled-layer-assignment-obstruction-def*
by (intro *exI conjI*)
(rule *fri-openings*, rule *partial*, rule *cand*, rule *not-low*,
rule *obstruction*)
qed

lemma *composition-fri-verifier-tied-missing-sampled-imp-assignment-obstruction*:
assumes *missing*:
composition-fri-verifier-tied-missing-sampled-layer-chain s out
shows
composition-fri-verifier-tied-sampled-layer-assignment-obstruction s out

proof –

have *bad*:

composition-fri-bad-with-verifier-tied-partial-candidate s out

and *not-sampled*:

$\neg$ *composition-fri-bad-with-verifier-tied-sampled-layer-chain s out*

using *missing*

unfolding *composition-fri-verifier-tied-missing-sampled-layer-chain-def*

by *blast+*

from *composition-fri-bad-with-verifier-tied-partial-candidateE[OF bad]*

obtain *trace-roots trace-bs trace-final fri-dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as trace-openings composition-openings

trace-table composition-table

where *fri-openings*:

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers

and *aligned*:

accepted-with-partial-initial-openings-aligned s out fr trace-roots

trace-final as fri-dg composition-roots composition-final fri-query-idxs

trace-openings composition-openings

and *trace-cand*:

partial-trace-table-candidate trace-table trace-openings

and *comp-cand*:

partial-composition-table-candidate composition-table

composition-openings

and *trace-low*: *trace-table-low-degree trace-table*

and *comp-not-low*:

$\neg$ *composition-table-low-degree maxDegree composition-table*

by *metis*

have *partial-generic*:

generic-fri-partial-evidence

(composition-table-low-degree (to-nat fri-dg)) composition-table

(to-nat fri-dg) composition-roots composition-bs composition-final

fri-query-idxs composition-round-layers

by (rule

accepted-fri-opening-transcript-composition-generic-partial-evidence

[OF fri-openings])

have *no-chain*:

$\bigwedge$ *doms layers. $\neg$*

generic-fri-sampled-layer-chain-evidence

(composition-table-low-degree (to-nat fri-dg)) composition-table

(to-nat fri-dg) composition-roots composition-bs

composition-final fri-query-idxs composition-round-layers doms

layers

proof

fix *doms layers*

assume *chain*:

```

generic-fri-sampled-layer-chain-evidence
  (composition-table-low-degree (to-nat fri-dg)) composition-table
  (to-nat fri-dg) composition-roots composition-bs
  composition-final fri-query-idxs composition-round-layers doms
  layers
have composition-fri-bad-with-verifier-tied-sampled-layer-chain s out
unfolding composition-fri-bad-with-verifier-tied-sampled-layer-chain-def
by (intro exI conjI)
  (rule fri-openings, rule aligned, rule trace-cand, rule comp-cand,
   rule trace-low, rule comp-not-low, rule chain)
then show False
  using not-sampled by contradiction
qed
have obstruction:
generic-fri-sampled-layer-assignment-obstruction
  (composition-table-low-degree (to-nat fri-dg)) composition-table
  (to-nat fri-dg) composition-roots composition-bs composition-final
  fri-query-idxs composition-round-layers
unfolding generic-fri-sampled-layer-assignment-obstruction-def
using partial-generic no-chain by blast
show ?thesis
unfolding
  composition-fri-verifier-tied-sampled-layer-assignment-obstruction-def
by (intro exI conjI)
  (rule fri-openings, rule aligned, rule trace-cand, rule comp-cand,
   rule trace-low, rule comp-not-low, rule obstruction)
qed

lemma wp-trace-fri-verifier-tied-missing-sampled-bound-from-assignment-obstruction:
assumes obstruction-bound:
  wp-event verify-monad
  (trace-fri-verifier-tied-sampled-layer-assignment-obstruction s) s
  ≤ M
shows
  wp-event verify-monad
  (trace-fri-verifier-tied-missing-sampled-layer-chain s) s ≤ M
proof –
have wp-event verify-monad
  (trace-fri-verifier-tied-missing-sampled-layer-chain s) s ≤
  wp-event verify-monad
  (trace-fri-verifier-tied-sampled-layer-assignment-obstruction s) s
by (rule wp-event-mono)
  (rule trace-fri-verifier-tied-missing-sampled-imp-assignment-obstruction)
also have ... ≤ M
by (rule obstruction-bound)
finally show ?thesis .
qed

lemma wp-composition-fri-verifier-tied-missing-sampled-bound-from-assignment-obstruction:

```

assumes *obstruction-bound*:
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}layer\text{-}assignment\text{-}obstruction\ s)\ s$
 $\leq M$
shows
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}missing\text{-}sampled\text{-}layer\text{-}chain\ s)\ s \leq M$
proof –
have $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}missing\text{-}sampled\text{-}layer\text{-}chain\ s)\ s \leq$
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}layer\text{-}assignment\text{-}obstruction\ s)\ s$
by (rule *wp-event-mono*)
(rule
 $composition\text{-}fri\text{-}verifier\text{-}tied\text{-}missing\text{-}sampled\text{-}imp\text{-}assignment\text{-}obstruction$)
also have $\dots \leq M$
by (rule *obstruction-bound*)
finally show *?thesis* .
qed

lemma *wp-trace-fri-verifier-tied-bound-from-sampled-and-assignment-obstruction*:

assumes *sampled-bound*:
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}tied\text{-}sampled\text{-}layer\text{-}chain\ s)\ s \leq R$
and *obstruction-bound*:
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}layer\text{-}assignment\text{-}obstruction\ s)\ s$
 $\leq M$
shows
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}tied\text{-}partial\text{-}candidate\ s)\ s \leq R + M$
by (rule *wp-trace-fri-verifier-tied-bound-from-sampled-and-missing*
 $[OF\ sampled\text{-}bound$
 $wp\text{-}trace\text{-}fri\text{-}verifier\text{-}tied\text{-}missing\text{-}sampled\text{-}bound\text{-}from\text{-}assignment\text{-}obstruction$
 $[OF\ obstruction\text{-}bound]]$)

lemma *wp-composition-fri-verifier-tied-bound-from-sampled-and-assignment-obstruction*:

assumes *sampled-bound*:
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}tied\text{-}sampled\text{-}layer\text{-}chain\ s)\ s \leq R$
and *obstruction-bound*:
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}layer\text{-}assignment\text{-}obstruction\ s)\ s$
 $\leq M$
shows
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}tied\text{-}partial\text{-}candidate\ s)\ s \leq$
 $R + M$
by (rule *wp-composition-fri-verifier-tied-bound-from-sampled-and-missing*

[OF sampled-bound
 wp-composition-fri-verifier-tied-missing-sampled-bound-from-assignment-obstruction
 [OF obstruction-bound]])

lemma *trace-fri-bad-with-verifier-tied-partial-candidates-staged-bound:*

assumes *verifier-bound:*

$\wedge$ *data attacker-state.*

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$*

*wp-event verify-monad
 (trace-fri-bad-with-verifier-tied-partial-candidate
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq C$*

shows

*wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state $\leq C$*

by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
 (simp-all add: *verifier-bound*)

lemma *composition-fri-bad-with-verifier-tied-partial-candidates-staged-bound:*

assumes *verifier-bound:*

$\wedge$ *data attacker-state.*

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$*

*wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-partial-candidate
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq C$*

shows

*wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state $\leq C$*

by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
 (simp-all add: *verifier-bound*)

lemma *wp-trace-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero:*

assumes *conflict-bound:*

wp-event verify-monad

(trace-fri-verifier-tied-sampled-assignment-conflict s) s $\leq C$

and *zero-bound:*

$wp\text{-event verify-monad}$
 $(trace\text{-fri-verifier-tied-zero-round-final-obstruction } s) \ s \leq Z$
shows
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-verifier-tied-sampled-layer-assignment-obstruction } s) \ s$
 $\leq C + Z$
proof –
have $wp\text{-event verify-monad}$
 $(trace\text{-fri-verifier-tied-sampled-layer-assignment-obstruction } s) \ s$
 $\leq wp\text{-event verify-monad}$
 $(\lambda out. trace\text{-fri-verifier-tied-sampled-assignment-conflict } s \ out \ \vee$
 $trace\text{-fri-verifier-tied-zero-round-final-obstruction } s \ out) \ s$
by (rule $wp\text{-event-mono}$)
(rule $trace\text{-fri-verifier-tied-assignment-obstruction-imp-conflict-or-zero}$)
also have $\dots \leq$
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-verifier-tied-sampled-assignment-conflict } s) \ s +$
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-verifier-tied-zero-round-final-obstruction } s) \ s$
by (rule $wp\text{-event-union-bound}$)
also have $\dots \leq C + Z$
by (rule $add\text{-mono}[OF \ conflict\text{-bound} \ zero\text{-bound}]$)
finally show $?thesis$.
qed

lemma $wp\text{-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero}$:
assumes $conflict\text{-bound}$:
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-verifier-tied-sampled-assignment-conflict } s) \ s \leq C$
and $zero\text{-bound}$:
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-verifier-tied-zero-round-final-obstruction } s) \ s \leq Z$
shows
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-verifier-tied-sampled-layer-assignment-obstruction } s) \ s$
 $\leq C + Z$
proof –
have $wp\text{-event verify-monad}$
 $(composition\text{-fri-verifier-tied-sampled-layer-assignment-obstruction } s) \ s$
 $\leq wp\text{-event verify-monad}$
 $(\lambda out. composition\text{-fri-verifier-tied-sampled-assignment-conflict } s \ out \ \vee$
 $composition\text{-fri-verifier-tied-zero-round-final-obstruction } s \ out) \ s$
by (rule $wp\text{-event-mono-on-support}$)
(rule
 $composition\text{-fri-verifier-tied-assignment-obstruction-imp-conflict-or-zero-on-support}$)
also have $\dots \leq$
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-verifier-tied-sampled-assignment-conflict } s) \ s +$
 $wp\text{-event verify-monad}$

(composition-fri-verifier-tied-zero-round-final-obstruction s) s
 by (rule wp-event-union-bound)
 also have ... $\leq C + Z$
 by (rule add-mono[OF conflict-bound zero-bound])
 finally show ?thesis .
 qed

lemma wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict:
assumes conflict-bound:
 wp-event verify-monad
 (composition-fri-verifier-tied-sampled-assignment-conflict s) s $\leq C$
shows
 wp-event verify-monad
 (composition-fri-verifier-tied-sampled-layer-assignment-obstruction s) s
 $\leq C$
proof –
have zero-bound:
 wp-event verify-monad
 (composition-fri-verifier-tied-zero-round-final-obstruction s) s ≤ 0
by (subst wp-composition-fri-verifier-tied-zero-round-final-obstruction-zero)
 simp
have wp-event verify-monad
 (composition-fri-verifier-tied-sampled-layer-assignment-obstruction s) s
 $\leq C + 0$
by (rule
 wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero
 [OF conflict-bound zero-bound])
then show ?thesis
by simp
 qed

lemma wp-trace-fri-verifier-tied-bound-from-sampled-conflict-and-zero:
assumes sampled-bound:
 wp-event verify-monad
 (trace-fri-bad-with-verifier-tied-sampled-layer-chain s) s $\leq R$
and conflict-bound:
 wp-event verify-monad
 (trace-fri-verifier-tied-sampled-assignment-conflict s) s $\leq C$
and zero-bound:
 wp-event verify-monad
 (trace-fri-verifier-tied-zero-round-final-obstruction s) s $\leq Z$
shows
 wp-event verify-monad
 (trace-fri-bad-with-verifier-tied-partial-candidate s) s
 $\leq R + (C + Z)$
by (rule
 wp-trace-fri-verifier-tied-bound-from-sampled-and-assignment-obstruction
 [OF sampled-bound
 wp-trace-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero

$[OF \text{ conflict-bound zero-bound}]]])$

lemma *wp-composition-fri-verifier-tied-bound-from-sampled-conflict-and-zero:*

assumes *sampled-bound:*

wp-event verify-monad

(composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s ≤ R

and *conflict-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-sampled-assignment-conflict s) s ≤ C

and *zero-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-zero-round-final-obstruction s) s ≤ Z

shows

wp-event verify-monad

(composition-fri-bad-with-verifier-tied-partial-candidate s) s
 $\leq R + (C + Z)$

by *(rule*

wp-composition-fri-verifier-tied-bound-from-sampled-and-assignment-obstruction

$[OF \text{ sampled-bound}$

wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero

$[OF \text{ conflict-bound zero-bound}]]])$

lemma *wp-composition-fri-verifier-tied-bound-from-sampled-conflict:*

assumes *sampled-bound:*

wp-event verify-monad

(composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s ≤ R

and *conflict-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-sampled-assignment-conflict s) s ≤ C

shows

wp-event verify-monad

(composition-fri-bad-with-verifier-tied-partial-candidate s) s
 $\leq R + C$

by *(rule*

wp-composition-fri-verifier-tied-bound-from-sampled-and-assignment-obstruction

$[OF \text{ sampled-bound}$

wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict

$[OF \text{ conflict-bound}]]])$

lemma *trace-fri-verifier-tied-reductionD:*

assumes *trace-fri-verifier-tied-reduction s*

shows

wp-event verify-monad

(trace-fri-bad-with-verifier-tied-partial-candidate s) s ≤
trace-fri-error

using *assms unfolding trace-fri-verifier-tied-reduction-def by simp*

lemma *composition-fri-verifier-tied-reductionD:*

assumes *composition-fri-verifier-tied-reduction s*

shows

wp-event verify-monad

(composition-fri-bad-with-verifier-tied-partial-candidate s) s ≤

composition-fri-error

using *assms unfolding composition-fri-verifier-tied-reduction-def* **by** *simp*

lemma *trace-fri-verifier-tied-reduction-from-sampled-conflict-and-zero:*

assumes *sampled-bound:*

wp-event verify-monad

(trace-fri-bad-with-verifier-tied-sampled-layer-chain s) s ≤ R

and *conflict-bound:*

wp-event verify-monad

(trace-fri-verifier-tied-sampled-assignment-conflict s) s ≤ C

and *zero-bound:*

wp-event verify-monad

(trace-fri-verifier-tied-zero-round-final-obstruction s) s ≤ Z

and *total-bound: R + (C + Z) ≤ trace-fri-error*

shows *trace-fri-verifier-tied-reduction s*

unfolding *trace-fri-verifier-tied-reduction-def*

by *(rule order-trans[*

OF wp-trace-fri-verifier-tied-bound-from-sampled-conflict-and-zero

[OF sampled-bound conflict-bound zero-bound] total-bound])

lemma *composition-fri-verifier-tied-reduction-from-sampled-conflict-and-zero:*

assumes *sampled-bound:*

wp-event verify-monad

(composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s ≤ R

and *conflict-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-sampled-assignment-conflict s) s ≤ C

and *zero-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-zero-round-final-obstruction s) s ≤ Z

and *total-bound: R + (C + Z) ≤ composition-fri-error*

shows *composition-fri-verifier-tied-reduction s*

unfolding *composition-fri-verifier-tied-reduction-def*

by *(rule order-trans[*

OF wp-composition-fri-verifier-tied-bound-from-sampled-conflict-and-zero

[OF sampled-bound conflict-bound zero-bound] total-bound])

lemma *composition-fri-verifier-tied-reduction-from-sampled-conflict:*

assumes *sampled-bound:*

wp-event verify-monad

(composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s ≤ R

and *conflict-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-sampled-assignment-conflict s) s ≤ C

and *total-bound: R + C ≤ composition-fri-error*

shows *composition-fri-verifier-tied-reduction s*

```

unfolding composition-fri-verifier-tied-reduction-def
by (rule order-trans[
  OF wp-composition-fri-verifier-tied-bound-from-sampled-conflict
  [OF sampled-bound conflict-bound] total-bound])

lemma trace-fri-zero-round-checked-final-obstruction-imp-verifier-tied:
  assumes
    trace-fri-zero-round-checked-final-obstruction s out
  shows trace-fri-bad-with-verifier-tied-partial-candidate s out
proof (rule trace-fri-zero-round-checked-final-obstruction-not-trace-low-degree
  [OF assms])
fix trace-roots trace-bs trace-final dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr trace-openings trace-table
assume fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final dg composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers
and checked:
  trace-zero-round-final-checked-candidate s out fr fri-query-idxs
  trace-openings trace-table trace-final
and not-low:  $\neg$  trace-table-low-degree trace-table
have partial:
  accepted-with-partial-trace-openings s out fr fri-query-idxs
  trace-openings
using checked unfolding trace-zero-round-final-checked-candidate-def
by simp
have cand:
  partial-trace-table-candidate trace-table trace-openings
using checked unfolding trace-zero-round-final-checked-candidate-def
by simp
show ?thesis
  unfolding trace-fri-bad-with-verifier-tied-partial-candidate-def
  by (intro exI conjI)
    (rule fri-openings, rule partial, rule cand, rule not-low)
qed

lemma trace-fri-zero-round-checked-final-obstruction-mono-verifier-tied:
  wp-event verify-monad (trace-fri-zero-round-checked-final-obstruction s) s  $\leq$ 
  wp-event verify-monad
  (trace-fri-bad-with-verifier-tied-partial-candidate s) s
by (rule wp-event-mono)
  (rule trace-fri-zero-round-checked-final-obstruction-imp-verifier-tied)

end

end

```

theory *Soundness-Relevant-Drift-Query-Agreement*
imports *Soundness-Relevant-Drift-Reachable-FRI*
begin

Fixed trace-pair query-agreement bounds for the empty-header relevant-drift residual. This layer keeps the candidate pair fixed before the query challenge is sampled. It therefore avoids the invalid broad fixed-cover route where partial openings selected after the query determine the target.

context *soundness*
begin

definition *staged-security-with-data-state-trace-pair-agreement-hit-at*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow$
 $((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

staged-security-with-data-state-trace-pair-agreement-hit-at
 $\text{trace-table trace-table}' i \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state}) \Rightarrow$
 $(\exists \text{ as query-idxs.}$
 $\text{accepted-transcript-shape}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{Some } (\text{result}, \text{final-state})) \text{ as query-idxs} \wedge$
 $i < \text{rounds} \wedge$
 $\text{query-idxs} ! i \in$
 $\text{trace-table-agreement-indices trace-table trace-table}'))$

definition *checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow$
 $(((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
 $\text{trace-table trace-table}' i \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } ((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}) \Rightarrow$
 $\text{staged-security-with-data-state-trace-pair-agreement-hit-at}$
 $\text{trace-table trace-table}' i$
 $(\text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state})))$

definition *checked-staged-security-with-query-prefix-trace-pair-agreement-hit-at*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow$

$(((((('f \text{ staged-query-prefix-data} \times 'f \text{ protocol-channel}) \times$
 $'f) \times 'f \text{ protocol-channel}) \times$
 $((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list})) \times$
 $'f \text{ protocol-channel}) \text{ option}) \Rightarrow \text{bool}$

where

$\text{checked-staged-security-with-query-prefix-trace-pair-agreement-hit-at}$
 $\text{trace-table trace-table}' i \text{ out} \longleftrightarrow$
 $\text{checked-staged-security-with-query-prefix-dynamic-index-hit}$
 $(\lambda - . \text{trace-table-agreement-indices trace-table trace-table}') \text{ out}$

lemma *checked-staged-query-prefix-trace-pair-agreement-prehit-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

shows

$\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $(\lambda - . \text{trace-table-agreement-indices trace-table trace-table}'))$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1)$

proof (rule *checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)

show $\text{HashMap adversary-initial-state} = \text{fmempty}$

by (simp add: *adversary-initial-state-def*)

show *hash-relation-program*

$(\text{checked-staged-query-prefix-dynamic-index-prehit-relation } A \ i$
 $\text{adversary-initial-state}$
 $(\lambda - . \text{trace-table-agreement-indices trace-table trace-table}'))$
 $\text{size } (\text{staged-query-search-queries budgets } i + 1)$
 $(\text{checked-staged-query-prefix-receive-with-state } A \ i)$

by (rule *hash-relation-program-checked-staged-query-prefix-receive-with-state*
 $[\text{OF } wf \text{ controlled less-imp-le}[\text{OF } i\text{-bound}]]$)

(rule *checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size*)

qed

lemma *checked-staged-query-prefix-trace-pair-agreement-hit-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *trace-low: trace-table-low-degree trace-table*

and *trace'-low: trace-table-low-degree trace-table'*

and *distinct: trace-table $\neq$ trace-table'*

shows

$\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $(\lambda - . \text{trace-table-agreement-indices trace-table trace-table}'))$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$

$\text{nnreal } \text{clength} / \text{nnreal } (\text{card } \text{query-sample-space})$
proof –
have *hit*:
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $(\lambda - . \text{trace-table-agreement-indices } \text{trace-table } \text{trace-table}'))$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $(\lambda - . \text{trace-table-agreement-indices } \text{trace-table } \text{trace-table}'))$
 $\text{adversary-initial-state} +$
 $\text{nnreal } \text{clength} / \text{nnreal } (\text{card } \text{query-sample-space})$
by (rule *checked-staged-query-prefix-trace-pair-agreement-hit-bound-by-prehit*
 $[\text{OF } \text{trace-low } \text{trace}'\text{-low } \text{distinct}]$)
have *pre*:
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $(\lambda - . \text{trace-table-agreement-indices } \text{trace-table } \text{trace-table}'))$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1)$
by (rule *checked-staged-query-prefix-trace-pair-agreement-prehit-bound*
 $[\text{OF } \text{wf controlled } i\text{-bound}]$)
show *?thesis*
by (rule *order-trans* $[\text{OF } \text{hit}]$) (intro *add-mono pre order-refl*)
qed

lemma *checked-staged-security-with-query-prefix-trace-pair-agreement-hit-bound*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < \text{rounds}$
and *trace-low*: *trace-table-low-degree trace-table*
and *trace'-low*: *trace-table-low-degree trace-table'*
and *distinct*: $\text{trace-table} \neq \text{trace-table}'$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-trace-pair-agreement-hit-at}$
 $\text{trace-table } \text{trace-table}' \ i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{nnreal } \text{clength} / \text{nnreal } (\text{card } \text{query-sample-space})$
unfolding *checked-staged-security-with-query-prefix-trace-pair-agreement-hit-at-def*
by (rule *checked-staged-security-with-query-prefix-dynamic-index-hit-bound*)
(rule *checked-staged-query-prefix-trace-pair-agreement-hit-bound*
 $[\text{OF } \text{wf controlled } i\text{-bound } \text{trace-low } \text{trace}'\text{-low } \text{distinct}]$)

lemma *staged-security-with-data-state-trace-pair-agreement-hit-at-le-query-prefix*:

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i < \text{rounds}$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-trace-pair-agreement-hit-at
      trace-table trace-table' i)
    adversary-initial-state  $\leq$ 
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-trace-pair-agreement-hit-at
      trace-table trace-table' i)
    adversary-initial-state
proof –
  let ?E =
    staged-security-with-data-state-trace-pair-agreement-hit-at
      trace-table trace-table' i
  let ?Q =
    checked-staged-security-with-query-prefix-trace-pair-agreement-hit-at
      trace-table trace-table' i
  have proj:
    wp-event (checked-staged-security-experiment-with-data-state A) ?E
      adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      ( $\lambda \text{out. case out of}$ 
         $\text{None} \Rightarrow ?E \text{ None}$ 
         $| \text{Some (packed, t)} \Rightarrow ?E (\text{Some (snd packed, t)})$ )
      adversary-initial-state
  by (rule checked-staged-security-experiment-with-query-prefix-data-state-projection-event
    [OF i-bound])
  show ?thesis
    unfolding proj
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out  $\in$ 
        set-dist
          (execute
            (checked-staged-security-experiment-with-query-prefix-data-state
              A i)
            adversary-initial-state)
    and hit:
      ( $\text{case out of None} \Rightarrow ?E \text{ None}$ 
         $| \text{Some (packed, t)} \Rightarrow ?E (\text{Some (snd packed, t)})$ )
    show ?Q out
  proof (cases out)
    case None
    then show ?thesis

```

```

using hit
unfolding staged-security-with-data-state-trace-pair-agreement-hit-at-def
  checked-staged-security-with-query-prefix-trace-pair-agreement-hit-at-def
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state
  result final-state where out-eq:
  out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
  by (cases packed, auto split: prod.splits)
have support':
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)
  ∈
  set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state
        A i)
      adversary-initial-state)
  using support unfolding out-eq .
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
  set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
  by (rule
    checked-staged-security-experiment-with-query-prefix-data-state-imp-data-state-support
    [OF i-bound support'])
have data-hit:
  staged-security-with-data-state-trace-pair-agreement-hit-at
  trace-table trace-table' i
  (Some (((data, attacker-state), result), final-state))
  using hit unfolding out-eq by simp
obtain as query-idxs where shape:
  accepted-transcript-shape
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (Some (result, final-state)) as query-idxs
and idx-agree:
  query-idxs ! i ∈
  trace-table-agreement-indices trace-table trace-table'
using data-hit unfolding
  staged-security-with-data-state-trace-pair-agreement-hit-at-def
by auto
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled data-support shape]

```



```

obtain raw-idxs where len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup-final:
   $\bigwedge j. j < rounds \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge j
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data j))) =
      Some (raw-idxs ! j)
by blast
have final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data i))) =
    Some raw
by (rule
  checked-staged-security-experiment-with-query-prefix-data-state-raw-final-lookup
  [OF wf controlled i-bound support'])
have raw-eq: raw-idxs ! i = raw
using lookup-final [OF i-bound] final-lookup by simp
have raw-agree:
  index (to-nat raw) ∈
    trace-table-agreement-indices trace-table trace-table'
using idx-agree query-idxs-eq len-raw i-bound raw-eq by simp
show ?thesis
unfolding out-eq
  checked-staged-security-with-query-prefix-trace-pair-agreement-hit-at-def
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
  checked-staged-query-prefix-dynamic-index-hit-def
using raw-agree by simp
qed
qed
qed

lemma staged-security-with-data-state-trace-pair-agreement-hit-at-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and trace-low: trace-table-low-degree trace-table
and trace'-low: trace-table-low-degree trace-table'
and distinct: trace-table ≠ trace-table'
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-trace-pair-agreement-hit-at
      trace-table trace-table' i)
  adversary-initial-state ≤

```

```

    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    nnreal clength / nnreal (card query-sample-space)
  by (rule order-trans[
    OF staged-security-with-data-state-trace-pair-agreement-hit-at-le-query-prefix
    checked-staged-security-with-query-prefix-trace-pair-agreement-hit-bound])
  (use assms in simp-all)

```

lemma *checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at-projection:*

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
    trace-table trace-table' i)
  adversary-initial-state =
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-trace-pair-agreement-hit-at
    trace-table trace-table' i)
  adversary-initial-state

```

proof —

```

let ?project =
  λx. ((snd (fst (fst x)), snd (fst x)), snd x)

```

have

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-trace-pair-agreement-hit-at
    trace-table trace-table' i)
  adversary-initial-state =
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
    ≫ (λx. return (?project x)))
  (staged-security-with-data-state-trace-pair-agreement-hit-at
    trace-table trace-table' i)
  adversary-initial-state

```

using

```

  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
  [of A]

```

by *simp*

also have ... =

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
    trace-table trace-table' i)
  adversary-initial-state

```

proof —

have *pred-eq*:

```

(λout. case out of
  None ⇒
    staged-security-with-data-state-trace-pair-agreement-hit-at
    trace-table trace-table' i None
  | Some (x, t) ⇒

```

```

    staged-security-with-data-state-trace-pair-agreement-hit-at
    trace-table trace-table' i (Some (?project x, t))) =
checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
    trace-table trace-table' i
by (rule ext)
  (auto simp:
    staged-security-with-data-state-trace-pair-agreement-hit-at-def
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at-def
    split: option.splits prod.splits)
show ?thesis
  by (subst wp-event-bind-return-map) (simp add: pred-eq)
qed
finally show ?thesis
  by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound: i < rounds
    and trace-low: trace-table-low-degree trace-table
    and trace'-low: trace-table-low-degree trace-table'
    and distinct: trace-table ≠ trace-table'
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
        trace-table trace-table' i)
      adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      nnreal clength / nnreal (card query-sample-space)
  unfolding
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at-projection
  by (rule staged-security-with-data-state-trace-pair-agreement-hit-at-bound
    [OF wf controlled i-bound trace-low trace'-low distinct])

definition low-degree-trace-pair-space :: ('f list × 'f list) set
where
  low-degree-trace-pair-space =
    {(trace-table, trace-table').
      trace-table-low-degree trace-table ∧
      trace-table-low-degree trace-table' ∧
      trace-table ≠ trace-table'}

lemma trace-table-low-degree-length:
  assumes trace-table-low-degree trace-table
  shows length trace-table = clength * scale
  using assms unfolding trace-table-low-degree-def

```

```

by (auto simp: eval-domain-length)

lemma finite-low-degree-trace-pair-space[simp]:
  finite low-degree-trace-pair-space
proof -
  have subset:
    low-degree-trace-pair-space  $\subseteq$ 
    {xs. length xs = clength * scale}  $\times$ 
    {xs. length xs = clength * scale}
  unfolding low-degree-trace-pair-space-def
  by (auto dest: trace-table-low-degree-length)
  show ?thesis
  by (rule finite-subset[OF subset])
    (simp add: finite-length-lists-UNIV)
qed

definition low-degree-trace-pair-agreement-round-error
where
  low-degree-trace-pair-agreement-round-error budgets i =
    ( $\sum$  (trace-table, trace-table')  $\in$  low-degree-trace-pair-space.
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      nnreal clength / nnreal (card query-sample-space))

definition low-degree-trace-pair-agreement-error
where
  low-degree-trace-pair-agreement-error budgets =
    ( $\sum$  x  $\in$  low-degree-trace-pair-space  $\times$  {..\Rightarrow
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      nnreal clength / nnreal (card query-sample-space))

lemma empty-header-low-degree-transcript-query-agreement-imp-pair-hit:
  assumes
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit
  out
  shows
     $\exists$  pair  $\in$  low-degree-trace-pair-space.
     $\exists$  i  $\in$  {..

```

```

next
  case (Some packed)
  then obtain alpha-prefix alpha-prefix-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state)
    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have hit:
    empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit
      ?s (Some (result, final-state))
  using assms
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  by simp
  then obtain fr f-fri-roots f-final as dg final trace-query-idxs
    trace-openings trace-table composition-table trace-table' i where
    shape:
      accepted-transcript-shape ?s (Some (result, final-state)) as
        trace-query-idxs
    and distinct: trace-table  $\neq$  trace-table'
    and trace-low: trace-table-low-degree trace-table
    and trace'-low: trace-table-low-degree trace-table'
    and i-bound:  $i < \text{rounds}$ 
    and idx-agree:
      trace-query-idxs !  $i \in$ 
        trace-table-agreement-indices trace-table trace-table'
    by (rule
      empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreementE)
  have pair-in:
    (trace-table, trace-table')  $\in$  low-degree-trace-pair-space
  unfolding low-degree-trace-pair-space-def
  using trace-low trace'-low distinct by simp
  have pair-hit:
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
      trace-table trace-table' i out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at-def
    staged-security-with-data-state-trace-pair-agreement-hit-at-def
  using shape i-bound idx-agree by auto
  show ?thesis
    by (intro bexI[of - (trace-table, trace-table')]
      bexI[of - i])
      (use pair-in i-bound pair-hit in simp-all)
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-low-degree-transcript-query-agreement-bound*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

(*checked-staged-security-with-actual-alpha-prefix-verifier-event*

empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit)

adversary-initial-state $\leq$

low-degree-trace-pair-agreement-error budgets

proof –

let *?M* =

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A

let *?pair-hit* =

$\lambda(\text{pair}, i) \text{ out.}$

checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at

(*fst pair*) (*snd pair*) *i out*

let *?A* = *low-degree-trace-pair-space* $\times \{..<\text{rounds}\}$

have *event-le*:

wp-event ?M

(*checked-staged-security-with-actual-alpha-prefix-verifier-event*

empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit)

adversary-initial-state $\leq$

wp-event ?M ($\lambda \text{out. } \exists x \in ?A. ?\text{pair-hit } x \text{ out}$)

adversary-initial-state

by (*rule wp-event-mono*)

(*auto dest*:

empty-header-low-degree-transcript-query-agreement-imp-pair-hit)

have *finite-A*: *finite ?A*

by *simp*

have *union-bound*:

wp-event ?M ($\lambda \text{out. } \exists x \in ?A. ?\text{pair-hit } x \text{ out}$)

adversary-initial-state $\leq$

($\sum x \in ?A.$

case x of (*pair, i*) $\Rightarrow$

staged-phase-relation-error size

(*staged-query-search-queries budgets i + 1*) +

nnreal clength / nnreal (card query-sample-space))

proof (*rule wp-event-finite-UN-bound[OF finite-A]*)

fix *x*

assume *x-in*: $x \in ?A$

then obtain *trace-table trace-table' i* **where** *x-eq*:

$x = ((\text{trace-table}, \text{trace-table}'), i)$

and *pair-in*:

(*trace-table, trace-table'*) $\in$ *low-degree-trace-pair-space*

and *i-bound*: $i < \text{rounds}$

by (*cases x, auto*)

have *trace-low*: *trace-table-low-degree trace-table*

using *pair-in* **unfolding** *low-degree-trace-pair-space-def* **by** *auto*

```

have trace'-low: trace-table-low-degree trace-table'
  using pair-in unfolding low-degree-trace-pair-space-def by auto
have distinct: trace-table  $\neq$  trace-table'
  using pair-in unfolding low-degree-trace-pair-space-def by auto
have bound:
  wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
      trace-table trace-table' i)
    adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
        (staged-query-search-queries budgets i + 1) +
        nnreal clength / nnreal (card query-sample-space)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at-bound
    [OF wf controlled i-bound trace-low trace'-low distinct])
show wp-event ?M (?pair-hit x) adversary-initial-state
   $\leq$  (case x of (pair, i)  $\Rightarrow$ 
    staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      nnreal clength / nnreal (card query-sample-space))
  unfolding x-eq
  using bound by simp
qed
have sum-eq:
  ( $\sum x \in ?A.$ 
    case x of (pair, i)  $\Rightarrow$ 
      staged-phase-relation-error size
        (staged-query-search-queries budgets i + 1) +
        nnreal clength / nnreal (card query-sample-space)) =
    low-degree-trace-pair-agreement-error budgets
  unfolding low-degree-trace-pair-agreement-error-def
  low-degree-trace-pair-agreement-round-error-def
  by (simp add: sum.cartesian-product case-prod-unfold)
show ?thesis
proof (rule order-trans[OF event-le])
  show wp-event ?M ( $\lambda out. \exists x \in ?A. ?pair-hit x out$ )
    adversary-initial-state  $\leq$ 
      low-degree-trace-pair-agreement-error budgets
    using union-bound sum-eq by simp
qed
qed

end

end

```

```

theory Soundness-Relevant-Drift-Transcript-Empty
imports

```

begin

Transcript-indexed empty-header packaging.

This layer avoids the broad empty-header composition residual whose authenticated-opening witnesses carry existential query indices. The composition branch instead carries the verifier transcript shape, so later query-agreement bounds can use verifier-derived query indices.

context *soundness*

begin

definition *soundness-bad-event-partial-candidate-empty-header-transcript*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

soundness-bad-event-partial-candidate-empty-header-transcript s out
 $\longleftrightarrow$
composition-bad-with-empty-composition-header-candidates-transcript
s out $\vee$
trace-fri-bad-with-empty-composition-header-candidates s out $\vee$
query-bad-with-empty-composition-header-candidates s out

lemma *accepted-empty-composition-header-candidate-partition-transcript:*

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *partial*:

accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table

and *shape*: *accepted-transcript-shape s out as trace-query-idxs*

shows *soundness-bad-event-partial-candidate-empty-header-transcript s out*

proof (*cases trace-table-low-degree trace-table*)

case *False*

have *trace-fri-bad-with-empty-composition-header-candidates s out*

unfolding *trace-fri-bad-with-empty-composition-header-candidates-def*

using *partial False* **by** *blast*

then show *?thesis*

unfolding *soundness-bad-event-partial-candidate-empty-header-transcript-def*

by *blast*

next

case *True*

note *trace-low = True*

have *composition-low*:

composition-table-low-degree maxDegree composition-table

proof –

have *composition-table = replicate (scale * clength) final*

using *partial*

unfolding *accepted-with-empty-composition-header-candidates-def* **by** *simp*

then show *?thesis*


```

    using constant-composition-table-low-degree by simp
qed
show ?thesis
proof (cases all-queries-consistent trace-table composition-table as)
  case False
  have query-bad-with-empty-composition-header-candidates s out
    unfolding query-bad-with-empty-composition-header-candidates-def
    using partial trace-low composition-low False by blast
  then show ?thesis
    unfolding soundness-bad-event-partial-candidate-empty-header-transcript-def
    by blast
next
  case True
  note all-queries = True
  from trace-low obtain f where
    deg-f: degree f < clength
    and trace-table-eq: trace-table = map (poly f) eval-domain
    unfolding trace-table-low-degree-def by blast
  have violated: violated-constraints f ≠ {}
    by (rule false-statement-violated-constraints
      [OF false-statement deg-f])
  have composition-bad-with-empty-composition-header-candidates-transcript
    s out
    unfolding composition-bad-with-empty-composition-header-candidates-transcript-def
    using partial shape deg-f trace-table-eq violated composition-low
      all-queries
    by blast
  then show ?thesis
    unfolding soundness-bad-event-partial-candidate-empty-header-transcript-def
    by blast
qed
qed

```

```

lemma accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-or-empty-head
  assumes false-statement: ¬ exists-valid-trace
  and supp: out ∈ set-dist (execute verify-monad s)
  and acc: accepted out
  and shape: accepted-transcript-shape s out as query-idxs
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and trace-partial-if-empty:
    composition-fri-roots = [] ⇒
      accepted-with-partial-trace-openings s out fr query-idxs
      trace-openings
  shows
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
      s out ∨
    soundness-bad-event-partial-candidate-empty-header-transcript s out

```

```

proof (cases composition-fri-roots = [])
  case False
  then have comp-nonempty: composition-fri-roots ≠ []
    by simp
  have
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
      s out
    by (rule
      accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-if-header-nonemp
        [OF false-statement supp acc header comp-nonempty])
  then show ?thesis by simp
next
  case True
  note comp-empty = True
  from acc obtain result final-state where out-eq:
    out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
  have trace-partial:
    accepted-with-partial-trace-openings s (Some (result, final-state))
      fr query-idxs trace-openings
    using trace-partial-if-empty[OF comp-empty] unfolding out-eq .
  show ?thesis
  proof (cases
    partial-merkle-inconsistency-bad s (Some (result, final-state)))
    case True
    then have
      soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
        s out
      unfolding out-eq
        soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def
      by simp
    then show ?thesis by simp
  next
  case False
  obtain trace-table where trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
    [OF trace-partial False]
  by blast
  have accepted-out: accepted (Some (result, final-state))
    by (rule accepted-with-partial-trace-openings-imp-accepted
      [OF trace-partial])
  let ?composition-table = replicate (scale * clength) final
  have empty-partial:
    accepted-with-empty-composition-header-candidates s
      (Some (result, final-state)) fr f-fri-roots f-final as dg final
      query-idxs trace-openings trace-table ?composition-table
  unfolding accepted-with-empty-composition-header-candidates-def
  by (intro conjI exI[of - rest])

```

```

      (use accepted-out header comp-empty trace-partial trace-candidate
        in simp-all)
    have soundness-bad-event-partial-candidate-empty-header-transcript s
      (Some (result, final-state))
    by (rule accepted-empty-composition-header-candidate-partition-transcript
      [OF false-statement empty-partial shape[unfolded out-eq]])
    then show ?thesis
      unfolding out-eq by simp
  qed
qed

lemma accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-or-empty-header
  assumes false-statement:  $\neg$  exists-valid-trace
  and supp:  $out \in \text{set-dist } (\text{execute verify-monad } s)$ 
  and acc: accepted out
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  shows
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
      s out  $\vee$ 
    soundness-bad-event-partial-candidate-empty-header-transcript s out
proof (cases composition-fri-roots = [])
case False
  then have comp-nonempty: composition-fri-roots  $\neq$  []
  by simp
  have
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
      s out
  by (rule
    accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-if-header-nonempty
    [OF false-statement supp acc header comp-nonempty])
  then show ?thesis by simp
next
case True
  note comp-empty = True
  from acc obtain result final-state where out-eq:
    out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
  have header-empty:
    verifier-header-transcript s fr f-fri-roots f-final as dg []
    final rest
  using header comp-empty by simp
  from verify-monad-supplied-empty-header-partial-trace-openings
    [OF supp[unfolded out-eq] header-empty]
  obtain query-idxs trace-openings where shape:
    accepted-transcript-shape s (Some (result, final-state)) as
    query-idxs
  and trace-partial:

```

```

    accepted-with-partial-trace-openings s
      (Some (result, final-state)) fr query-idxs trace-openings
  by blast
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then have
  soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
    s out
  unfolding out-eq
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def
  by simp
then show ?thesis by simp
next
case False
obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
  [OF trace-partial False]
by blast
have accepted-out: accepted (Some (result, final-state))
  by (rule accepted-with-partial-trace-openings-imp-accepted
    [OF trace-partial])
let ?composition-table = replicate (scale * clength) final
have empty-partial:
  accepted-with-empty-composition-header-candidates s
    (Some (result, final-state)) fr f-fri-roots f-final as dg final
    query-idxs trace-openings trace-table ?composition-table
  unfolding accepted-with-empty-composition-header-candidates-def
  by (intro conjI exI[of - rest])
    (use accepted-out header-empty trace-partial trace-candidate
      in simp-all)
have soundness-bad-event-partial-candidate-empty-header-transcript s
  (Some (result, final-state))
  by (rule accepted-empty-composition-header-candidate-partition-transcript
    [OF false-statement empty-partial shape])
then show ?thesis
  unfolding out-eq by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-transcript-imp-prefix-prequery-or-emp
  assumes support:
    out ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
            A)

```

```

      adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-alpha-partial-opening-union-hit-with-empty-header-transcript
  out
shows
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit
  composition-trace-bad-alpha-space out ∨
  checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
  composition-trace-bad-alpha-space out ∨
  checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
  out
proof —
have broad-hit:
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-alpha-partial-opening-union-hit-with-empty-header out
using hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
by (auto split: option.splits prod.splits
  intro:
    composition-alpha-partial-opening-union-hit-with-empty-header-transcript-imp-union-hit)
have partial:
  checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
  composition-trace-bad-alpha-space out
by (rule
  checked-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-imp-partial-header-bad-set-hit-on-
  [OF support broad-hit])
have fresh-or-prequery:
  checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
  composition-trace-bad-alpha-space out ∨
  checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
  composition-trace-bad-alpha-space out
by (rule
  checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-imp-fresh-or-prequery
  [OF partial])
then show ?thesis
proof
assume fresh:
  checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
  composition-trace-bad-alpha-space out
have prefix-or-drift:
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit
  composition-trace-bad-alpha-space out ∨
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
  composition-trace-bad-alpha-space out
by (rule
  checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-imp-prefix-or-drift
  [OF fresh])

```

```

then show ?thesis
proof
  assume
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out
  then show ?thesis by simp
next
  assume drift:
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
    composition-trace-bad-alpha-space out
  have
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
    out
  unfolding
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-def
  using drift hit by simp
  then show ?thesis by simp
qed
next
  assume
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space out
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-transcript-bound-from-empty-relevant
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and drift-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
   A)
  checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
  adversary-initial-state  $\leq D$ 
shows
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
   composition-alpha-partial-opening-union-hit-with-empty-header-transcript)
  adversary-initial-state  $\leq$ 
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  D

```

```

proof –
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?Prefix =
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space
  let ?Prequery =
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space
  let ?Drift =
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
  let ?Hit =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-alpha-partial-opening-union-hit-with-empty-header-transcript
  let ?P =
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0)
  let ?Q =
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  have transcript-bound:
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?P
    by (rule
      checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
      [OF wf controlled])
  have prefix-bound:
    wp-event ?M ?Prefix adversary-initial-state ≤ ?P
    by (rule
      checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
      [OF transcript-bound])
  have prequery-bound:
    wp-event ?M ?Prequery adversary-initial-state ≤ ?Q
    by (rule
      checked-staged-security-with-actual-alpha-prefix-vector-alpha-prequery-accounting
      [OF wf controlled])
  have event-le:
    wp-event ?M ?Hit adversary-initial-state ≤
    wp-event ?M
    (λout. ?Prefix out ∨ ?Prequery out ∨ ?Drift out ∨ False)
    adversary-initial-state
    by (rule wp-event-mono-on-support)
    (use
     checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-transcript-imp-prefix-prequery-or-empty-

```

```

      [of - A] in blast)
also have ... ≤
  wp-event ?M ?Prefix adversary-initial-state +
  wp-event ?M ?Prequery adversary-initial-state +
  wp-event ?M ?Drift adversary-initial-state +
  wp-event ?M (λ-. False) adversary-initial-state
by (rule wp-event-union-bound4)
also have ... ≤ ?P + ?Q + D + 0
by (intro add-mono prefix-bound prequery-bound drift-bound)
  (simp add: wp-event-def wp-def dist-expect-def)
finally show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-data-state-empty-header-composition-randomization-transcript-bound-from-empty
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
  checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
  adversary-initial-state ≤ D
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-randomization-bad-with-empty-composition-header-candidates-transcript)
  adversary-initial-state ≤
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D
proof -
let ?M = checked-staged-security-experiment-with-data-state A
let ?random =
  staged-security-with-data-state-verifier-event
    composition-randomization-bad-with-empty-composition-header-candidates-transcript
let ?alpha =
  staged-security-with-data-state-verifier-event
    composition-alpha-partial-opening-union-hit-with-empty-header-transcript
have alpha-projected:
  wp-event ?M ?alpha adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event

```



```

    composition-alpha-partial-opening-union-hit-with-empty-header-transcript)
  adversary-initial-state
  by (rule
    checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
  have alpha-bound:
    wp-event ?M ?alpha adversary-initial-state ≤
      composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      D
  unfolding alpha-projected
  by (rule
    checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-transcript-bound-from-empty-relevant-drift-trace
    [OF wf controlled drift-bound])
  have wp-event ?M ?random adversary-initial-state ≤
    wp-event ?M ?alpha adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
      intro:
        composition-randomization-bad-with-empty-composition-header-candidates-transcript-imp-alpha-union-hit
        split: option.splits prod.splits)
  also have ... ≤
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D
  by (rule alpha-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-empty-header-bad-event-transcript-bound-from-empty-relevant-drift-trace
  fixes empty-trace-fri-error :: prob
  assumes false-statement: ¬ exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and drift-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
    adversary-initial-state ≤ D

```

and *trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{empty-trace-fri-error}$

and *query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$

shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{soundness-bad-event-partial-candidate-empty-header-transcript})$
 $\text{adversary-initial-state} \leq$
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $D + \text{empty-trace-fri-error} + \text{empty-query-error}$

proof –
let $?M = \text{checked-staged-security-experiment-with-data-state } A$
let $?E = \text{staged-security-with-data-state-verifier-event}$
let $?degree =$
 $?E \text{ composition-degree-bad-with-empty-composition-header-candidates-transcript}$
let $?random =$
 $?E \text{ composition-randomization-bad-with-empty-composition-header-candidates-transcript}$
let $?trace =$
 $?E \text{ trace-fri-bad-with-empty-composition-header-candidates}$
let $?query =$
 $?E \text{ query-bad-with-empty-composition-header-candidates}$
let $?bad =$
 $?E \text{ soundness-bad-event-partial-candidate-empty-header-transcript}$
have $\text{degree-false: } ?degree = (\lambda \text{out. False})$
by (rule ext)
 $(\text{auto simp: staged-security-with-data-state-verifier-event-def}$
 $\text{composition-degree-bad-with-empty-composition-header-candidates-transcript-false}$
 $\text{split: option.splits prod.splits})$
have *degree-bound*:
 $\text{wp-event } ?M ?degree \text{ adversary-initial-state} \leq 0$
unfolding *degree-false* *wp-event-def* *wp-def* *dist-expect-def* **by** *simp*
have *random-bound*:
 $\text{wp-event } ?M ?random \text{ adversary-initial-state} \leq$
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$

```

    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D
  by (rule
    checked-staged-security-with-data-state-empty-header-composition-randomization-transcript-bound-from-empty-re
    [OF wf controlled drift-bound])
  have wp-event ?M ?bad adversary-initial-state ≤
    wp-event ?M
    (λout. ?degree out ∨ ?random out ∨ ?trace out ∨
      ?query out)
    adversary-initial-state
  proof (rule wp-event-mono)
  fix out
  assume bad: ?bad out
  show ?degree out ∨ ?random out ∨ ?trace out ∨ ?query out
  proof (cases out)
  case None
  then show ?thesis
    using bad
    unfolding staged-security-with-data-state-verifier-event-def
    by simp
  next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have local-bad:
    soundness-bad-event-partial-candidate-empty-header-transcript ?s
    (Some (result, final-state))
    using bad
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
  then consider
    (comp)
      composition-bad-with-empty-composition-header-candidates-transcript ?s
      (Some (result, final-state))
    | (trace)
      trace-fri-bad-with-empty-composition-header-candidates ?s
      (Some (result, final-state))
    | (query)
      query-bad-with-empty-composition-header-candidates ?s
      (Some (result, final-state))
  unfolding
    soundness-bad-event-partial-candidate-empty-header-transcript-def

```

```

    by blast
  then show ?thesis
proof cases
  case comp
  then have comp-split:
    composition-degree-bad-with-empty-composition-header-candidates-transcript
      ?s (Some (result, final-state)) ∨
    composition-randomization-bad-with-empty-composition-header-candidates-transcript
      ?s (Some (result, final-state))
  by (rule
    composition-bad-with-empty-composition-header-candidates-transcript-split
    [OF false-statement])
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    using comp-split by (auto split: option.splits prod.splits)
next
  case trace
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
next
  case query
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
qed
also have ... ≤
  wp-event ?M ?degree adversary-initial-state +
  wp-event ?M ?random adversary-initial-state +
  wp-event ?M ?trace adversary-initial-state +
  wp-event ?M ?query adversary-initial-state
  by (rule wp-event-union-bound4)
also have ... ≤
  0 +
  (composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D) +
  empty-trace-fri-error + empty-query-error
  by (intro add-mono degree-bound random-bound trace-fri-bound query-bound)
finally show ?thesis
  by (simp add: algebra-simps)
qed

```

lemma *checked-staged-soundness-from-aligned-transcript-partial-candidate-and-empty-header-transcript-bounds:*
fixes *partial-candidate-error empty-header-error :: prob*
assumes *false-statement: $\neg$ exists-valid-trace*
and *partial-candidate-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle)
 adversary-initial-state $\leq$ partial-candidate-error
and *empty-header-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 soundness-bad-event-partial-candidate-empty-header-transcript)
 adversary-initial-state $\leq$ empty-header-error
shows *checked-staged-adversary-acceptance-probability A $\leq$*
 partial-candidate-error + empty-header-error
proof –
 let *?M = checked-staged-security-experiment-with-data-state A*
 let *?E = staged-security-with-data-state-verifier-event*
 let *?partial =*
 ?E soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
 let *?empty = ?E soundness-bad-event-partial-candidate-empty-header-transcript*
 let *?bad = λ out. ?partial out $\vee$?empty out*
 have *accepted-bad:*
 wp-event ?M accepted adversary-initial-state $\leq$
 wp-event ?M ?bad adversary-initial-state
 proof (rule *wp-event-mono-on-support*)
 fix *out*
 assume *support: out $\in$ set-dist (execute ?M adversary-initial-state)*
 and *acc: accepted out*
 show *?bad out*
 proof (cases *out*)
 case *None*
 then show *?thesis*
 using *acc unfolding accepted-def by simp*
 next
 case (Some *packed*)
 then obtain *data attacker-state result final-state where out-eq:*
 out = Some (((data, attacker-state), result), final-state)
 by (cases *packed*, auto split: *prod.splits*)
 let *?s =*
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 have *verifier:*
 Some (result, final-state) $\in$ set-dist (execute verify-monad ?s)
 using *checked-staged-security-experiment-with-data-state-outcomeE*
 [OF support[unfolded out-eq]]
 by *blast*
 have *verifier-acc: accepted (Some (result, final-state))*

```

    unfolding accepted-def by simp
  from verify-monad-accepted-transcript-shape[OF verifier]
  obtain alphas query-idxs where shape:
    accepted-transcript-shape ?s (Some (result, final-state))
    alphas query-idxs
  by blast
  from shape obtain result' final-state' fr f-fri-roots f-final dg
    composition-fri-roots final rest where
    header:
      verifier-header-transcript ?s fr f-fri-roots f-final alphas dg
      composition-fri-roots final rest
  by (elim accepted-transcript-shape-header-query-extraction)
  have local-bad:
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
      ?s (Some (result, final-state))  $\vee$ 
    soundness-bad-event-partial-candidate-empty-header-transcript ?s
      (Some (result, final-state))
  by (rule
    accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-or-empty-head
      [OF false-statement verifier verifier-acc header])
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
have bad-bound:
  wp-event ?M ?bad adversary-initial-state  $\leq$ 
  partial-candidate-error + empty-header-error
proof -
  have wp-event ?M ?bad adversary-initial-state  $\leq$ 
    wp-event ?M ?partial adversary-initial-state +
    wp-event ?M ?empty adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq$  partial-candidate-error + empty-header-error
  by (intro add-mono partial-candidate-bound empty-header-bound)
  finally show ?thesis .
qed
have staged-bound:
  wp-event (checked-staged-security-experiment A) accepted
  adversary-initial-state  $\leq$ 
  partial-candidate-error + empty-header-error
  unfolding checked-staged-security-experiment-acceptance-with-data-state
  by (rule order-trans[OF accepted-bad bad-bound])
show ?thesis
proof -
  have
    wp-event (checked-staged-security-experiment A)
      ( $\lambda$ out.  $\neg$  Option.is-none out) adversary-initial-state =
    wp-event (checked-staged-security-experiment A) accepted

```

```

    adversary-initial-state
unfolding wp-event-def accepted-def
by (rule arg-cong[where
  f= $\lambda Q. wp \text{ (checked-staged-security-experiment } A) \ Q$ 
  adversary-initial-state])
  (rule ext, simp)
then show ?thesis
  unfolding checked-staged-adversary-acceptance-probability-def
  using staged-bound by simp
qed
qed

lemma checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from
fixes trace-fri-error' composition-fri-error' query-error D E
  empty-trace-fri-error
  empty-query-error :: prob
assumes false-statement:  $\neg \text{exists-valid-trace}$ 
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  composition-fri-error'
and query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      query-bad-with-aligned-transcript-partial-candidates)
    adversary-initial-state  $\leq$  query-error
and drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
    adversary-initial-state  $\leq$  D
and empty-drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
    adversary-initial-state  $\leq$  E
and empty-trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event

```

trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *empty-trace-fri-error*
and *current-empty-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-current-empty-query-opening-hit-at 0*)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
(*composition-error-bound* +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
(*staged-alpha-search-queries budgets 0*) +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
D) +
trace-fri-error' + *composition-fri-error'* +
query-error +
(*composition-error-bound* +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
(*staged-alpha-search-queries budgets 0*) +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
E + *empty-trace-fri-error* + *empty-query-error*)
proof –
have *comp-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
composition-bad-with-partial-candidates)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
(*staged-alpha-search-queries budgets 0*) +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
D
by (*rule*
checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-relevant-drift-and-budgets
[*OF false-statement wf controlled drift-bound*])
have *merkle-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
partial-merkle-inconsistency-bad)

$\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget})$
by (rule
 $\text{checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled}$
 $[\text{OF wf controlled}]$)
have empty-query-bound :
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
by (rule $\text{order-trans}[\text{OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero}[\text{OF wf controlled}] \text{ current-empty-bound}]$)
have $\text{empty-header-bound}$:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{soundness-bad-event-partial-candidate-empty-header-transcript})$
 $\text{adversary-initial-state} \leq$
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $E + \text{empty-trace-fri-error} + \text{empty-query-error}$
by (rule
 $\text{checked-staged-security-with-data-state-empty-header-bad-event-transcript-bound-from-empty-relevant-drift-trans}$
 $[\text{OF false-statement wf controlled empty-drift-bound}$
 $\text{empty-trace-fri-bound empty-query-bound}]$)
have $\text{partial-candidate-bound}$:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle})$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $D) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' + \text{query-error}$

by (rule
checked-staged-security-with-data-state-aligned-transcript-partial-candidate-with-partial-merkle-component-union
 [OF merkle-bound comp-bound trace-fri-bound comp-fri-bound
 query-bound])
have bound:
checked-staged-adversary-acceptance-probability $A \leq$
 (hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D) +
 trace-fri-error' + composition-fri-error' + query-error) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 E + empty-trace-fri-error + empty-query-error)
by (rule
checked-staged-soundness-from-aligned-transcript-partial-candidate-and-empty-header-transcript-bounds
 [OF false-statement partial-candidate-bound empty-header-bound])
then show ?thesis
by (simp add: algebra-simps)
qed

lemma *checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from*
fixes trace-fri-error' composition-fri-error' query-error D
 empty-trace-fri-error empty-query-error :: prob
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ trace-fri-error'
and comp-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ composition-fri-error'

and *query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq$ *query-error*
and *drift-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$ *D*
and *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *empty-trace-fri-error*
and *current-empty-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D) +
trace-fri-error' + composition-fri-error' +
query-error +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D + empty-trace-fri-error + empty-query-error)
proof –
have *empty-drift-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
adversary-initial-state $\leq$ *D*
by (*rule*

```

    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-relevant-drift
    [OF drift-bound])
  show ?thesis
  by (rule
    checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-re
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      query-bound drift-bound empty-drift-bound empty-trace-fri-bound
      current-empty-bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-reachable-
fixes F L :: prob
assumes trace-fri-reduction:
   $\bigwedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    trace-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
and fixed-empty-alpha-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
    adversary-initial-state  $\leq$  F
and low-degree-query-agreement-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit)
    adversary-initial-state  $\leq$  L
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
    adversary-initial-state  $\leq$  F + (trace-fri-error + L)
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?trace =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    trace-fri-bad-with-empty-composition-header-candidates
  let ?low-degree-transcript =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit
  let ?low-degree =

```

checked-staged-security-with-actual-alpha-prefix-verifier-event
empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit
have *trace-data-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state ≤ trace-fri-error
by (rule
trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
[OF trace-fri-reduction])
have *trace-bound*:
wp-event ?M ?trace adversary-initial-state ≤ trace-fri-error
using *trace-data-bound*
checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection
[of A trace-fri-bad-with-empty-composition-header-candidates]
by *simp*
have *low-degree-transcript-bound*:
wp-event ?M ?low-degree-transcript adversary-initial-state ≤ L
proof (rule *order-trans*)
show *wp-event ?M ?low-degree-transcript adversary-initial-state ≤*
wp-event ?M ?low-degree adversary-initial-state
by (rule *wp-event-mono*)
(auto simp: checked-staged-security-with-actual-alpha-prefix-verifier-event-def
intro:
empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-imp-query-agreement
split: option.splits prod.splits)
show *wp-event ?M ?low-degree adversary-initial-state ≤ L*
by (rule *low-degree-query-agreement-bound*)
qed
show *?thesis*
by (rule
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-fixed-trace-fri-
[OF fixed-empty-alpha-bound trace-bound low-degree-transcript-bound])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-reachable-*
fixes *F :: prob*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-reduction:*
∧ data attacker-state.
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *fixed-empty-alpha-bound:*
wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
adversary-initial-state $\leq F$

shows

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
adversary-initial-state $\leq$
 $F + (\text{trace-fri-error} + \text{low-degree-trace-pair-agreement-error budgets})$

proof –

let $?M =$

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
have *trace-data-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq \text{trace-fri-error}$
by (rule
trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
 $[OF \text{ trace-fri-reduction}]$)
have *trace-bound*:
wp-event ?M
(checked-staged-security-with-actual-alpha-prefix-verifier-event trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq \text{trace-fri-error}$
using *trace-data-bound*
checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection
 $[of A \text{ trace-fri-bad-with-empty-composition-header-candidates}]$
by *simp*
have *low-degree-bound*:
wp-event ?M
(checked-staged-security-with-actual-alpha-prefix-verifier-event empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit)
adversary-initial-state $\leq$
 $\text{low-degree-trace-pair-agreement-error budgets}$
by (rule
checked-staged-security-with-actual-alpha-prefix-empty-header-low-degree-transcript-query-agreement-bound
 $[OF \text{ wf controlled}]$)
show $?thesis$
by (rule
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-fixed-trace-fri-
 $[OF \text{ fixed-empty-alpha-bound trace-bound low-degree-bound}]$)
qed

lemma *checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from*
fixes *trace-fri-error' composition-fri-error' query-error D F L*
empty-query-error :: prob

assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *composition-fri-error'*
and *query-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-aligned-transcript-partial-candidates)
 adversary-initial-state $\leq$ *query-error*
and *drift-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state $\leq$ *D*
and *trace-fri-reduction*:
 $\bigwedge$ *data attacker-state.*
 Some (*data, attacker-state*) $\in$
 set-dist (*execute* (*checked-staged-transcript-program A*)
 adversary-initial-state) $\implies$
 trace-fri-reachable-partial-candidate-reduction-assumption
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
and *fixed-empty-alpha-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
 composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *F*
and *low-degree-query-agreement-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
 empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit)
 adversary-initial-state $\leq$ *L*
and *current-empty-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-current-empty-query-opening-hit-at 0*)

$adversary-initial-state \leq empty-query-error$
shows $checked-staged-adversary-acceptance-probability\ A \leq$
 $hash-collision-budget-value\ 0$
 $(staged-attacker-query-budget\ budgets +$
 $staged-challenge-query-budget + verifier-hash-query-budget) +$
 $(composition-error-bound +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $D) +$
 $trace-fri-error' + composition-fri-error' +$
 $query-error +$
 $(composition-error-bound +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $(F + (trace-fri-error + L)) + trace-fri-error +$
 $empty-query-error)$
proof –
have $empty-drift-bound:$
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state\ A)$
 $checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript$
 $adversary-initial-state \leq F + (trace-fri-error + L)$
by (rule
 $checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-reachable-trac$
 $[OF\ trace-fri-reduction\ fixed-empty-alpha-bound$
 $low-degree-query-agreement-bound])$
have $empty-trace-fri-bound:$
 $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $trace-fri-bad-with-empty-composition-header-candidates)$
 $adversary-initial-state \leq trace-fri-error$
by (rule
 $trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction$
 $[OF\ trace-fri-reduction])$
show $?thesis$
by (rule
 $checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-re$
 $[OF\ false-statement\ wf\ controlled\ trace-fri-bound\ comp-fri-bound$
 $query-bound\ drift-bound\ empty-drift-bound\ empty-trace-fri-bound$
 $current-empty-bound])$
qed

lemma *checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from*
fixes *trace-fri-error'* *composition-fri-error'* *query-error* *D* *F*
empty-query-error :: *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*
and *query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq$ *query-error*
and *drift-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$ *D*
and *trace-fri-reduction*:
 $\bigwedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
trace-fri-reachable-partial-candidate-reduction-assumption
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*))
and *fixed-empty-alpha-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-verifier-event*
composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *F*
and *current-empty-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-current-empty-query-opening-hit-at* *0*)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability* *A* $\leq$
hash-collision-budget-value *0*

(staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D) +
 trace-fri-error' + composition-fri-error' +
 query-error +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (F + (trace-fri-error +
 low-degree-trace-pair-agreement-error budgets)) +
 trace-fri-error + empty-query-error)

proof –

have empty-drift-bound:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
 adversary-initial-state ≤
 F + (trace-fri-error + low-degree-trace-pair-agreement-error budgets)

by (rule
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-reachable-trac
 [OF wf controlled trace-fri-reduction fixed-empty-alpha-bound])

have empty-trace-fri-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state ≤ trace-fri-error

by (rule
 trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
 [OF trace-fri-reduction])

show ?thesis

by (rule
 checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-re
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound
 query-bound drift-bound empty-drift-bound empty-trace-fri-bound
 current-empty-bound])

qed

theorem stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-r

fixes $D\ F\ L\ \text{empty-query-error} :: \text{prob}$
and $C :: \text{nat} \Rightarrow \text{prob}$
assumes $\text{false-statement} : \neg \text{exists-valid-trace}$
and $\text{wf} : \text{staged-budget-wellformed budgets}$
and $\text{controlled} : \text{staged-adversary-controlled budgets } A$
and $\text{trace-fri-reduction} :$
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \Rightarrow$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and $\text{composition-fri-reduction} :$
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \Rightarrow$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and $\text{round-bound} :$
 $\bigwedge i. i < \text{rounds} \Rightarrow$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A\ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C\ i$
and $\text{drift-bound} :$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift}$
 $\text{adversary-initial-state} \leq D$
and $\text{fixed-empty-alpha-bound} :$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq F$
and $\text{low-degree-query-agreement-bound} :$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit})$
 $\text{adversary-initial-state} \leq L$
and $\text{current-empty-bound} :$
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$

$(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $D) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\sum_{i < \text{rounds.}} C \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(F + (\text{trace-fri-error} + L)) + \text{trace-fri-error} +$
 $\text{empty-query-error})$
proof –
have trace-fri-bound :
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
by (rule
 $\text{trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction}$
 $[\text{OF trace-fri-reduction}])$
have comp-fri-bound :
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}$
by (rule
 $\text{composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction}$
 $[\text{OF composition-fri-reduction}])$
have $\text{current-query-bound}$:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-current-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq (\sum_{i < \text{rounds.}} C \ i)$
by (rule
 $\text{checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds}$
 $[\text{OF round-bound wf controlled}])$

have *query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq (\sum i < \text{rounds}. C\ i)$
by (*rule order-trans*[
OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
current-query-bound])
(*use wf controlled in simp-all*)
show *?thesis*
by (*rule*
checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-r
[*OF false-statement wf controlled trace-fri-bound comp-fri-bound*
query-bound drift-bound trace-fri-reduction fixed-empty-alpha-bound
low-degree-query-agreement-bound current-empty-bound])
qed

theorem *stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-r*
fixes *D F empty-query-error :: prob*
and *C :: nat $\Rightarrow$ prob*
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-reduction:*
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-reachable-partial-candidate-reduction-assumption
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
and *composition-fri-reduction:*
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
composition-fri-reachable-partial-candidate-reduction-assumption
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
and *round-bound:*
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
(*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i*)
adversary-initial-state $\leq C\ i$
and *drift-bound:*
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq D$
and *fixed-empty-alpha-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
adversary-initial-state $\leq F$
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D) +
trace-fri-error + composition-fri-error +
($\sum_{i < \text{rounds}} C i$) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(F + (trace-fri-error +
low-degree-trace-pair-agreement-error budgets)) +
trace-fri-error + empty-query-error)
proof –
have *trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
by (*rule*
trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction
[OF trace-fri-reduction])
have *comp-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event

```

    composition-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ composition-fri-error
by (rule
  composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction
  [OF composition-fri-reduction])
have current-query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-current-query-partial-opening-hit
  adversary-initial-state ≤ (∑ i<rounds. C i)
by (rule
  checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
  [OF round-bound wf controlled])
have query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    query-bad-with-aligned-transcript-partial-candidates)
  adversary-initial-state ≤ (∑ i<rounds. C i)
by (rule order-trans[
  OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
  current-query-bound])
  (use wf controlled in simp-all)
show ?thesis
by (rule
  checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-re
  [OF false-statement wf controlled trace-fri-bound comp-fri-bound
    query-bound drift-bound trace-fri-reduction fixed-empty-alpha-bound
    current-empty-bound])
qed

theorem stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-r
fixes D empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
    trace-fri-reachable-partial-candidate-reduction-assumption
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
and composition-fri-reduction:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
    composition-fri-reachable-partial-candidate-reduction-assumption

```

(verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
and round-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq C$ i
and drift-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state $\leq D$
and current-empty-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq$ empty-query-error
shows checked-staged-adversary-acceptance-probability $A \leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D) +
 trace-fri-error + composition-fri-error +
 ($\sum i < \text{rounds}. C$ i) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D + trace-fri-error + empty-query-error)
proof –
have trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ trace-fri-error
by (rule
 trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction
 [OF trace-fri-reduction])

have *comp-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error*
by (*rule*
composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction
[*OF composition-fri-reduction*])
have *current-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq$ ($\sum i < \text{rounds. } C \ i$)
by (*rule*
checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
[*OF round-bound wf controlled*])
have *query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq$ ($\sum i < \text{rounds. } C \ i$)
by (*rule order-trans*[
OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
current-query-bound])
(*use wf controlled in simp-all*)
have *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
by (*rule*
trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
[*OF trace-fri-reduction*])
show *?thesis*
by (*rule*
checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-re
[*OF false-statement wf controlled trace-fri-bound comp-fri-bound*
query-bound drift-bound empty-trace-fri-bound current-empty-bound])
qed

theorem *stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-*
fixes *D N L empty-query-error :: prob*
and *C :: nat $\Rightarrow$ prob*
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-reduction:*
 $\bigwedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A))

$\text{adversary-initial-state}) \implies$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and $\text{composition-fri-reduction:}$
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and round-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state A } i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and drift-bound:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift}$
 $\text{adversary-initial-state} \leq D$
and not-prefix-bound:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and $\text{low-degree-query-agreement-bound:}$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit})$
 $\text{adversary-initial-state} \leq L$
and $\text{current-empty-bound:}$
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability A} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$

hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D) +
trace-fri-error + composition-fri-error +
($\sum i < \text{rounds}. C i$) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
((composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + N) +
(trace-fri-error + L)) +
trace-fri-error + empty-query-error)

proof –

let *?M* =

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A

let *?P* =

composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0)

have *transcript-bound*:

wp-event (checked-staged-transcript-with-alpha-prefix-program A)
(checked-staged-transcript-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state ≤ ?P

by (rule

checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
[OF wf controlled])

have *prefix-bound*:

wp-event ?M
(checked-staged-security-with-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state ≤ ?P

by (rule

checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
[OF transcript-bound])

have *fixed-empty-alpha-bound*:

wp-event ?M
(checked-staged-security-with-actual-alpha-prefix-verifier-event

```

      composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
    adversary-initial-state ≤ ?P + N
proof (rule order-trans)
  show wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
    adversary-initial-state ≤
      wp-event ?M
      (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
        composition-trace-bad-alpha-space)
      adversary-initial-state +
      wp-event ?M
      (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
        composition-trace-bad-alpha-space)
      adversary-initial-state
  by (rule
    checked-staged-security-with-actual-alpha-prefix-fixed-empty-alpha-hit-le-prefix-or-not-prefix
    [OF wf controlled])
  show ... ≤ ?P + N
    by (intro add-mono prefix-bound not-prefix-bound)
qed
show ?thesis
  by (rule
    stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-reach
    [OF false-statement wf controlled trace-fri-reduction
      composition-fri-reduction round-bound drift-bound
      fixed-empty-alpha-bound low-degree-query-agreement-bound
      current-empty-bound])
qed

theorem stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-r
fixes D N empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      trace-fri-reachable-partial-candidate-reduction-assumption
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
and composition-fri-reduction:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒

```

composition-fri-reachable-partial-candidate-reduction-assumption
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
and round-bound:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A\ i$)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq C\ i$
and drift-bound:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state $\leq D$
and not-prefix-bound:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq N$
and current-empty-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq \text{empty-query-error}$
shows checked-staged-adversary-acceptance-probability $A \leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D) +
 trace-fri-error + composition-fri-error +
 ($\sum i < \text{rounds}. C\ i$) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 ((composition-error-bound +
 staged-phase-relation-error size

$(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) + N) +$
 $(\text{trace-fri-error} + \text{low-degree-trace-pair-agreement-error budgets})) +$
 $\text{trace-fri-error} + \text{empty-query-error})$

proof –

let $?M =$
 $\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A$

let $?P =$
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0)$

have *transcript-bound*:
 $\text{wp-event } (\text{checked-staged-transcript-with-alpha-prefix-program } A)$
 $(\text{checked-staged-transcript-actual-alpha-prefix-bad-set-hit}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq ?P$

by (rule
 $\text{checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets}$
 $[OF \text{ wf controlled}]$)

have *prefix-bound*:
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq ?P$

by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript}$
 $[OF \text{ transcript-bound}]$)

have *fixed-empty-alpha-bound*:
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq ?P + N$

proof (rule *order-trans*)

show $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} +$
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state}$

by (rule

```

    checked-staged-security-with-actual-alpha-prefix-fixed-empty-alpha-hit-le-prefix-or-not-prefix
    [OF wf controlled])
  show ... ≤ ?P + N
  by (intro add-mono prefix-bound not-prefix-bound)
qed
show ?thesis
by (rule
  stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-reach
  [OF false-statement wf controlled trace-fri-reduction
    composition-fri-reduction round-bound drift-bound
    fixed-empty-alpha-bound current-empty-bound])
qed

theorem stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-j
fixes D N empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
    trace-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
and composition-fri-reduction:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
    composition-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
and round-bound:
  ∧ i. i < rounds ⇒
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state ≤ C i
and drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
    adversary-initial-state ≤ D
and not-prefix-bound:
  wp-event

```

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound composition-trace-bad-alpha-space)
adversary-initial-state $\leq N$
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability $A \leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D) +
trace-fri-error + composition-fri-error +
($\sum i < \text{rounds}. C i$) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D + trace-fri-error + empty-query-error)
by *(rule*
stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-relevant
[OF false-statement wf controlled trace-fri-reduction
composition-fri-reduction round-bound drift-bound
current-empty-bound])
theorem *stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-relevant*
fixes *X P R L empty-query-error :: prob*
and *C :: nat $\Rightarrow$ prob*
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-reduction:*
 $\bigwedge$ data attacker-state.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-reachable-partial-candidate-reduction-assumption

$(\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data}))$
and *composition-fri-reduction*:
 $\bigwedge \text{data } \text{attacker-state.}$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data}))$
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *cross-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad}$
 $\text{adversary-initial-state} \leq X$
and *data-pre-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$
and *witness-new-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit}$
 $\text{adversary-initial-state} \leq R$
and *low-degree-query-agreement-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit})$
 $\text{adversary-initial-state} \leq L$
and *current-empty-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error } \text{size})$

(staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + $X + (P + R) + (P + R)$)) +
 trace-fri-error + composition-fri-error +
 ($\sum i < \text{rounds}. C\ i$) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 ((composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 (hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + $X + (P + R) + (P + R)$)) +
 (trace-fri-error + L)) +
 trace-fri-error + empty-query-error)

proof –
let ?D =
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + $X + (P + R) + (P + R)$
have drift-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state $\leq$?D
by (rule
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-cross-data-pre
 [OF wf controlled cross-bound data-pre-bound witness-new-bound])
have transcript-pre-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
 adversary-initial-state $\leq P$
by (rule
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
 [OF data-pre-bound])
have witness-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit

$adversary-initial-state \leq P + R$
by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
 $[OF \text{ transcript-pre-bound witness-new-bound}]$)
have *no-prefix-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
 $adversary-initial-state \leq P + R$
by (rule *order-trans*
 $[OF$
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-le-witness-transcript-hit
*witness-bound])
have *path-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
 $adversary-initial-state \leq P + R$
by (rule *order-trans*
 $[OF$
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-le-witness-transcript-hit
*witness-bound])
have *collision-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
 $adversary-initial-state \leq$
hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0)
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
 $[OF \text{ wf controlled}]$)
have *uncovered-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
 $adversary-initial-state \leq X$
by (rule *order-trans*
 $[OF$
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad
*cross-bound])
have *not-prefix-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)
 $adversary-initial-state \leq ?D$
by (rule***

```

    checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
    [OF wf controlled collision-bound uncovered-bound no-prefix-bound
     path-bound])
show ?thesis
by (rule
    stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-reach
    [OF false-statement wf controlled trace-fri-reduction
     composition-fri-reduction round-bound drift-bound not-prefix-bound
     low-degree-query-agreement-bound current-empty-bound])
qed

theorem stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-r
fixes X P R empty-query-error :: prob
and C :: nat  $\Rightarrow$  prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
   $\bigwedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    trace-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
and composition-fri-reduction:
   $\bigwedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    composition-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
and round-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state  $\leq$  C i
and cross-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
    adversary-initial-state  $\leq$  X
and data-pre-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state  $\leq$  P

```

and *witness-new-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq R$
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + X + (P + R) + (P + R))) +
trace-fri-error + composition-fri-error +
($\sum i < \text{rounds}. C i$) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
((composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + X + (P + R) + (P + R))) +
(trace-fri-error + low-degree-trace-pair-agreement-error budgets)) +
trace-fri-error + empty-query-error)
proof –
let *?D* =
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + X + (P + R) + (P + R)
have *drift-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift

$adversary-initial-state \leq ?D$
by (rule
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-cross-data-pre
 $[OF \text{ wf controlled cross-bound data-pre-bound witness-new-bound}]$)
have *transcript-pre-bound*:
wp-event
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state \ A)$
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
 $adversary-initial-state \leq P$
by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
 $[OF \text{ data-pre-bound}]$)
have *witness-bound*:
wp-event
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state \ A)$
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
 $adversary-initial-state \leq P + R$
by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
 $[OF \text{ transcript-pre-bound witness-new-bound}]$)
have *no-prefix-bound*:
wp-event
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state \ A)$
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
 $adversary-initial-state \leq P + R$
by (rule *order-trans*
 $[OF$
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-le-witness-transcript-hit
witness-bound])
have *path-bound*:
wp-event
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state \ A)$
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
 $adversary-initial-state \leq P + R$
by (rule *order-trans*
 $[OF$
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-le-witness-transcript-hit
witness-bound])
have *collision-bound*:
wp-event
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state \ A)$
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
 $adversary-initial-state \leq$
 $hash-collision-budget-value \ 0 \ (staged-alpha-search-queries \ budgets \ 0)$
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
 $[OF \text{ wf controlled}]$)
have *uncovered-bound*:

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
   A)
  checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
  adversary-initial-state  $\leq X$ 
by (rule order-trans
   [OF
    checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad
    cross-bound])
have not-prefix-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
   composition-trace-bad-alpha-space)
  adversary-initial-state  $\leq ?D$ 
by (rule
   checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
   [OF wf controlled collision-bound uncovered-bound no-prefix-bound
    path-bound])
show ?thesis
by (rule
   stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-reach
   [OF false-statement wf controlled trace-fri-reduction
    composition-fri-reduction round-bound drift-bound not-prefix-bound
    current-empty-bound])
qed

end

These transcript-empty wrappers expose the broad witnessed cross/Merkle diagnos-
tic event directly. They are retained only for internal comparison with the sampled-
transcript route and are not part of the exported public path.

hide-fact
soundness.stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query
soundness.stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query
soundness.stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query
soundness.stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query
soundness.stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query
soundness.stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query
soundness.stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query
soundness.stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query

end

theory Soundness-Relevant-Drift-Transcript-Public
imports Soundness-Relevant-Drift-Transcript-Empty

```

begin

Public-facing transcript-indexed packaging for the current relevant-drift proof route. This layer keeps new wrappers out of the larger transcript-empty theory and avoids the diagnostic broad witnessed cross/Merkle route for the nonempty partial-candidate branch.

context *soundness*

begin

lemma *accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle*

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *supp*: $\text{out} \in \text{set-dist } (\text{execute verify-monad } s)$

and *acc*: *accepted out*

and *header*:

verifier-header-transcript s fr f-fri-roots f-final as dg

composition-fri-roots final rest

shows

soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
s out $\vee$

soundness-bad-event-partial-candidate-empty-header-transcript s out

proof (*cases composition-fri-roots = []*)

case *False*

then have *comp-nonempty*: *composition-fri-roots* $\neq []$

by *simp*

have

soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
s out

by (*rule*

accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
[*OF false-statement supp acc header comp-nonempty*])

then show *?thesis* **by** *simp*

next

case *True*

note *comp-empty = True*

from *acc* **obtain** *result final-state* **where** *out-eq*:

out = Some (result, final-state)

unfolding *accepted-def* **by** (*cases out*) *auto*

have *header-empty*:

verifier-header-transcript s fr f-fri-roots f-final as dg []

final rest

using *header comp-empty* **by** *simp*

from *verify-monad-supplied-empty-header-partial-trace-openings*

[*OF supp[unfolded out-eq] header-empty*]

obtain *query-idxs trace-openings* **where** *shape*:

accepted-transcript-shape s (Some (result, final-state)) as
query-idxs

and *trace-partial*:

accepted-with-partial-trace-openings s

(*Some (result, final-state)*) *fr query-idxs trace-openings*


```

  by blast
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then have
  soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
    s out
  unfolding out-eq
  soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle-def
  by simp
then show ?thesis by simp
next
case False
obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
  [OF trace-partial False]
by blast
have accepted-out: accepted (Some (result, final-state))
  by (rule accepted-with-partial-trace-openings-imp-accepted
    [OF trace-partial])
let ?composition-table = replicate (scale * clength) final
have empty-partial:
  accepted-with-empty-composition-header-candidates s
    (Some (result, final-state)) fr f-fri-roots f-final as dg final
  query-idxs trace-openings trace-table ?composition-table
  unfolding accepted-with-empty-composition-header-candidates-def
  by (intro conjI exI[of - rest])
    (use accepted-out header-empty trace-partial trace-candidate
      in simp-all)
have soundness-bad-event-partial-candidate-empty-header-transcript s
  (Some (result, final-state))
  by (rule accepted-empty-composition-header-candidate-partition-transcript
    [OF false-statement empty-partial shape])
then show ?thesis
  unfolding out-eq by simp
qed
qed

lemma checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-and-empty-header-
  fixes partial-candidate-error empty-header-error :: prob
  assumes false-statement:  $\neg$  exists-valid-trace
  and partial-candidate-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)
    adversary-initial-state  $\leq$  partial-candidate-error
  and empty-header-bound:

```

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    soundness-bad-event-partial-candidate-empty-header-transcript)
  adversary-initial-state ≤ empty-header-error
shows checked-staged-adversary-acceptance-probability A ≤
  partial-candidate-error + empty-header-error
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?partial =
    ?E soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
  let ?empty = ?E soundness-bad-event-partial-candidate-empty-header-transcript
  let ?bad = λout. ?partial out ∨ ?empty out
  have accepted-bad:
    wp-event ?M accepted adversary-initial-state ≤
      wp-event ?M ?bad adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support: out ∈ set-dist (execute ?M adversary-initial-state)
  and acc: accepted out
  show ?bad out
  proof (cases out)
  case None
  then show ?thesis
  using acc unfolding accepted-def by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  using checked-staged-security-experiment-with-data-state-outcomeE
  [OF support[unfolded out-eq]]
by blast
have verifier-acc: accepted (Some (result, final-state))
  unfolding accepted-def by simp
from verify-monad-accepted-transcript-shape[OF verifier]
obtain alphas query-idxs where shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  alphas query-idxs
by blast
from shape obtain result' final-state' fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  header:
    verifier-header-transcript ?s fr f-fri-roots f-final alphas dg

```

```

      composition-fri-roots final rest
    by (elim accepted-transcript-shape-header-query-extraction)
  have local-bad:
    soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
      ?s (Some (result, final-state)) ∨
      soundness-bad-event-partial-candidate-empty-header-transcript ?s
      (Some (result, final-state))
    by (rule
      accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-m
      [OF false-statement verifier verifier-acc header])
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
have bad-bound:
  wp-event ?M ?bad adversary-initial-state ≤
  partial-candidate-error + empty-header-error
proof -
  have wp-event ?M ?bad adversary-initial-state ≤
    wp-event ?M ?partial adversary-initial-state +
    wp-event ?M ?empty adversary-initial-state
    by (rule wp-event-union-bound)
  also have ... ≤ partial-candidate-error + empty-header-error
    by (intro add-mono partial-candidate-bound empty-header-bound)
  finally show ?thesis .
qed
have staged-bound:
  wp-event (checked-staged-security-experiment A) accepted
  adversary-initial-state ≤
  partial-candidate-error + empty-header-error
  unfolding checked-staged-security-experiment-acceptance-with-data-state
  by (rule order-trans[OF accepted-bad bad-bound])
show ?thesis
proof -
  have
    wp-event (checked-staged-security-experiment A)
    (λout. ¬ Option.is-none out) adversary-initial-state =
    wp-event (checked-staged-security-experiment A) accepted
    adversary-initial-state
  unfolding wp-event-def accepted-def
  by (rule arg-cong[where
    f=λQ. wp (checked-staged-security-experiment A) Q
    adversary-initial-state])
    (rule ext, simp)
  then show ?thesis
    unfolding checked-staged-adversary-acceptance-probability-def
    using staged-bound by simp
qed

```

qed

theorem *stark-soundness-from-transcript-indexed-partial-opening-route:*

fixes F *empty-query-error* :: prob

and $C :: \text{nat} \Rightarrow \text{prob}$

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *trace-fri-reduction*:

$\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\implies$

trace-fri-reachable-partial-candidate-reduction-assumption

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

and *composition-fri-reduction*:

$\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\implies$

composition-fri-reachable-partial-candidate-reduction-assumption

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

and *round-bound*:

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)

adversary-initial-state $\leq C i$

and *fixed-empty-alpha-bound*:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

A)

(checked-staged-security-with-actual-alpha-prefix-verifier-event

composition-alpha-bad-set-hit-with-empty-composition-header-candidates)

adversary-initial-state $\leq F$

and *current-empty-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-current-empty-query-opening-hit-at 0)

adversary-initial-state $\leq \text{empty-query-error}$

shows *checked-staged-adversary-acceptance-probability A $\leq$*

hash-collision-budget-value 0

(staged-attacker-query-budget budgets +

staged-challenge-query-budget + verifier-hash-query-budget) +

($\sum i < \text{rounds}. C i$) +

trace-fri-error + composition-fri-error +

($\sum i < \text{rounds}. C i$) +

(composition-error-bound +

staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(F + (trace-fri-error +
low-degree-trace-pair-agreement-error budgets)) +
trace-fri-error + empty-query-error)

proof –

have *merkle-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state ≤
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)

by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[OF wf controlled])

have *comp-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-randomization-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state ≤ (∑ i<rounds. C i)

by (rule
checked-staged-security-with-data-state-aligned-partial-candidate-randomization-bound-from-query-prefix-current-
[OF wf controlled round-bound])

have *trace-fri-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state ≤ trace-fri-error

by (rule
trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction
[OF trace-fri-reduction])

have *comp-fri-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state ≤ composition-fri-error

by (rule
composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction
[OF composition-fri-reduction])

have *current-query-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state ≤ (∑ i<rounds. C i)

by (rule
checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
[OF round-bound wf controlled])

have *query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq$ $(\sum i < \text{rounds}. C\ i)$

by (rule *order-trans*[
OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
current-query-bound])
(use wf controlled in simp-all)

have *partial-candidate-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
 $(\sum i < \text{rounds}. C\ i) + \text{trace-fri-error} + \text{composition-fri-error} +$
 $(\sum i < \text{rounds}. C\ i)$

by (rule
checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-with-merkle-u
[OF merkle-bound comp-bound trace-fri-bound comp-fri-bound
query-bound])

have *empty-drift-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
adversary-initial-state $\leq$
 $F + (\text{trace-fri-error} + \text{low-degree-trace-pair-agreement-error budgets})$

by (rule
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-reachable-trac
[OF wf controlled trace-fri-reduction fixed-empty-alpha-bound])

have *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*

by (rule
trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
[OF trace-fri-reduction])

have *empty-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *empty-query-error*

by (rule *order-trans*[

```

    OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero[
      OF wf controlled] current-empty-bound])
have empty-header-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      soundness-bad-event-partial-candidate-empty-header-transcript)
  adversary-initial-state ≤
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  (F + (trace-fri-error +
    low-degree-trace-pair-agreement-error budgets)) +
  trace-fri-error + empty-query-error
by (rule
  checked-staged-security-with-data-state-empty-header-bad-event-transcript-bound-from-empty-relevant-drift-trans
    [OF false-statement wf controlled empty-drift-bound
      empty-trace-fri-bound empty-query-bound])
have bound:
  checked-staged-adversary-acceptance-probability A ≤
  (hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget + verifier-hash-query-budget) +
    (∑ i<rounds. C i) + trace-fri-error + composition-fri-error +
    (∑ i<rounds. C i)) +
  (composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    (F + (trace-fri-error +
      low-degree-trace-pair-agreement-error budgets)) +
    trace-fri-error + empty-query-error)
by (rule
  checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-and-empty-header-tran
    [OF false-statement partial-candidate-bound empty-header-bound])
then show ?thesis
by (simp add: algebra-simps)
qed

theorem stark-soundness-from-transcript-indexed-partial-opening-route-single-query-charge:
fixes F empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace

```

and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-reduction:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *composition-fri-reduction:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *round-bound:*
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state A } i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *fixed-empty-alpha-bound:*
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq F$
and *current-empty-bound:*
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum i < \text{rounds. } C \ i) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(F + (\text{trace-fri-error} +$

$\text{low-degree-trace-pair-agreement-error budgets})) +$
 $\text{trace-fri-error} + \text{empty-query-error})$

proof –

have *merkle-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\quad \text{partial-merkle-inconsistency-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\quad \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget})$

by (rule
 $\text{checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled}$
 $[\text{OF wf controlled}])$

have *trace-fri-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\quad \text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$

by (rule
 $\text{trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction}$
 $[\text{OF trace-fri-reduction}])$

have *comp-fri-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\quad \text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}$

by (rule
 $\text{composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction}$
 $[\text{OF composition-fri-reduction}])$

have *current-query-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-current-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq (\sum_{i < \text{rounds. } C} i)$

by (rule
 $\text{checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds}$
 $[\text{OF round-bound wf controlled}])$

have *partial-candidate-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle})$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\quad \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum_{i < \text{rounds. } C} i) + \text{trace-fri-error} + \text{composition-fri-error}$

by (rule
 $\text{checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-with-merkle-c}$
 $[\text{OF wf controlled merkle-bound current-query-bound trace-fri-bound}]$

$\text{comp-fri-bound}]$
have *empty-drift-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript}$
 $\text{adversary-initial-state} \leq$
 $F + (\text{trace-fri-error} + \text{low-degree-trace-pair-agreement-error budgets})$
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-reachable-trac}$
 $[OF \text{ wf controlled trace-fri-reduction fixed-empty-alpha-bound}]$)
have *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
by (rule
 $\text{trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction}$
 $[OF \text{ trace-fri-reduction}]$)
have *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
by (rule *order-trans*[
 $OF \text{ checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero}$
 $OF \text{ wf controlled}] \text{ current-empty-bound}]$)
have *empty-header-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{soundness-bad-event-partial-candidate-empty-header-transcript})$
 $\text{adversary-initial-state} \leq$
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(F + (\text{trace-fri-error} +$
 $\text{low-degree-trace-pair-agreement-error budgets})) +$
 $\text{trace-fri-error} + \text{empty-query-error}$
by (rule
 $\text{checked-staged-security-with-data-state-empty-header-bad-event-transcript-bound-from-empty-relevant-drift-trans}$
 $[OF \text{ false-statement wf controlled empty-drift-bound}$
 $\text{empty-trace-fri-bound empty-query-bound}]$)
have *bound*:
 $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $(\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$

$$\begin{aligned}
& \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) + \\
& (\sum_{i < \text{rounds}} C \ i) + \text{trace-fri-error} + \text{composition-fri-error}) + \\
& (\text{composition-error-bound} + \\
& \text{staged-phase-relation-error size} \\
& (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{hash-collision-budget-value } 0 \\
& (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{staged-phase-relation-error size} \\
& (\text{staged-alpha-search-queries budgets } 0) + \\
& (F + (\text{trace-fri-error} + \\
& \text{low-degree-trace-pair-agreement-error budgets})) + \\
& \text{trace-fri-error} + \text{empty-query-error}) \\
& \text{by (rule} \\
& \text{checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-and-empty-header-tran} \\
& \text{[OF false-statement partial-candidate-bound empty-header-bound])} \\
& \text{then show ?thesis} \\
& \text{by (simp add: algebra-simps)} \\
& \text{qed}
\end{aligned}$$

theorem *stark-soundness-from-transcript-indexed-partial-opening-route-from-not-prefix:*

fixes N *empty-query-error* :: *prob*
and C :: *nat* $\Rightarrow$ *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* A
and *trace-fri-reduction*:
 $\bigwedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* A)
adversary-initial-state) $\implies$
trace-fri-reachable-partial-candidate-reduction-assumption
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*))
and *composition-fri-reduction*:
 $\bigwedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* A)
adversary-initial-state) $\implies$
composition-fri-reachable-partial-candidate-reduction-assumption
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*))
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state* A i)
(*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* i)
adversary-initial-state $\leq C \ i$
and *not-prefix-bound*:
wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound composition-trace-bad-alpha-space)
adversary-initial-state $\leq N$
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability $A \leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
($\sum i < \text{rounds. } C i$) +
trace-fri-error + composition-fri-error +
($\sum i < \text{rounds. } C i$) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
((composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + N) +
(trace-fri-error +
low-degree-trace-pair-agreement-error budgets)) +
trace-fri-error + empty-query-error)
proof –
let *?M =*
checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let *?P =*
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0)
have *transcript-bound:*
wp-event (checked-staged-transcript-with-alpha-prefix-program A)
(checked-staged-transcript-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state $\leq ?P$
by (rule
checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
[OF wf controlled])
have *prefix-bound:*

```

wp-event ?M
  (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
   composition-trace-bad-alpha-space)
adversary-initial-state ≤ ?P
by (rule
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
  [OF transcript-bound])
have fixed-empty-alpha-bound:
  wp-event ?M
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
   composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
  adversary-initial-state ≤ ?P + N
proof (rule order-trans)
show wp-event ?M
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
   composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
  adversary-initial-state ≤
wp-event ?M
  (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
   composition-trace-bad-alpha-space)
  adversary-initial-state +
wp-event ?M
  (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
   composition-trace-bad-alpha-space)
  adversary-initial-state
by (rule
  checked-staged-security-with-actual-alpha-prefix-fixed-empty-alpha-hit-le-prefix-or-not-prefix
  [OF wf controlled])
show ... ≤ ?P + N
  by (intro add-mono prefix-bound not-prefix-bound)
qed
show ?thesis
  by (rule
    stark-soundness-from-transcript-indexed-partial-opening-route
    [OF false-statement wf controlled trace-fri-reduction
     composition-fri-reduction round-bound fixed-empty-alpha-bound
     current-empty-bound])
qed

theorem stark-soundness-from-transcript-indexed-partial-opening-route-from-not-prefix-single-query-charge:
fixes N empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)

```

$\text{adversary-initial-state}) \implies$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and $\text{composition-fri-reduction:}$
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and round-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state A } i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and not-prefix-bound:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and $\text{current-empty-bound:}$
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum i < \text{rounds. } C \ i) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $((\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) + N) +$
 $(\text{trace-fri-error} +$
 $\text{low-degree-trace-pair-agreement-error budgets})) +$

```

    trace-fri-error + empty-query-error)
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?P =
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0)
  have transcript-bound:
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?P
  by (rule
    checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
    [OF wf controlled])
  have prefix-bound:
    wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?P
  by (rule
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
    [OF transcript-bound])
  have fixed-empty-alpha-bound:
    wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
     composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
    adversary-initial-state ≤ ?P + N
  proof (rule order-trans)
  show wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
     composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
    adversary-initial-state ≤
    wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state +
    wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
     composition-trace-bad-alpha-space)
    adversary-initial-state
  by (rule
    checked-staged-security-with-actual-alpha-prefix-fixed-empty-alpha-hit-le-prefix-or-not-prefix
    [OF wf controlled])
  show ... ≤ ?P + N
  by (intro add-mono prefix-bound not-prefix-bound)

```

```

qed
show ?thesis
by (rule
  stark-soundness-from-transcript-indexed-partial-opening-route-single-query-charge
  [OF false-statement wf controlled trace-fri-reduction
    composition-fri-reduction round-bound fixed-empty-alpha-bound
    current-empty-bound])
qed

theorem stark-soundness-from-transcript-indexed-partial-opening-route-from-uncovered-and-transcript:
fixes  $U$   $P$  empty-query-error :: prob
and  $C :: \text{nat} \Rightarrow \text{prob}$ 
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets  $A$ 
and trace-fri-reduction:
   $\bigwedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program  $A$ )
      adversary-initial-state)  $\implies$ 
    trace-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
and composition-fri-reduction:
   $\bigwedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program  $A$ )
      adversary-initial-state)  $\implies$ 
    composition-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
and round-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state  $A$   $i$ )
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at  $i$ )
      adversary-initial-state  $\leq C$   $i$ 
and uncovered-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
       $A$ )
    checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
    adversary-initial-state  $\leq U$ 
and data-pre-bound:
  wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state  $\leq P$ 
and current-empty-bound:
  wp-event (checked-staged-security-experiment-with-data-state  $A$ )

```


$(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum_{i < \text{rounds}} C i) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\sum_{i < \text{rounds}} C i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $((\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) + U +$
 $(P + \text{staged-concrete-transcript-target-error-bound}) +$
 $(P + \text{staged-concrete-transcript-target-error-bound}))) +$
 $(\text{trace-fri-error} +$
 $\text{low-degree-trace-pair-agreement-error budgets})) +$
 $\text{trace-fri-error} + \text{empty-query-error})$

proof –

have $\text{transcript-new-bound}$:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-new-hit}$
 $\text{adversary-initial-state} \leq \text{staged-concrete-transcript-target-error-bound}$

by (rule

$\text{checked-staged-security-with-data-state-transcript-new-hit-bound}$
 $[\text{OF } \text{wf controlled}])$

have no-prefix-bound :

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate}$
 $\text{adversary-initial-state} \leq$
 $P + \text{staged-concrete-transcript-target-error-bound}$

by (rule

$\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-bound-from-sampled-transcript}$
 $[\text{OF } \text{data-pre-bound transcript-new-bound}])$

have path-bound :

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output}$

adversary-initial-state $\leq$
P + *staged-concrete-transcript-target-error-bound*
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-bound-from-sampled-transcript
[*OF data-pre-bound transcript-new-bound*])
have *collision-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
adversary-initial-state $\leq$
hash-collision-budget-value 0 (*staged-alpha-search-queries budgets 0*)
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
[*OF wf controlled*])
have *not-prefix-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$
hash-collision-budget-value 0 (*staged-alpha-search-queries budgets 0*) +
U + (*P* + *staged-concrete-transcript-target-error-bound*) +
(*P* + *staged-concrete-transcript-target-error-bound*)
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
[*OF wf controlled collision-bound uncovered-bound no-prefix-bound*
path-bound])
show ?thesis
by (rule
stark-soundness-from-transcript-indexed-partial-opening-route-from-not-prefix
[*OF false-statement wf controlled trace-fri-reduction*
composition-fri-reduction round-bound not-prefix-bound
current-empty-bound])
qed

theorem *stark-soundness-from-transcript-indexed-partial-opening-route-from-uncovered-and-transcript-single-query-c*
fixes *U P empty-query-error :: prob*
and *C :: nat $\Rightarrow$ prob*
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-reduction:*
 $\bigwedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))
and composition-fri-reduction:
 $\bigwedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 composition-fri-reachable-partial-candidate-reduction-assumption
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
and round-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq C i$
and uncovered-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
 adversary-initial-state $\leq U$
and data-pre-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-transcript-pre-hit
 adversary-initial-state $\leq P$
and current-empty-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq \text{empty-query-error}$
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds}. C i$) +
 trace-fri-error + composition-fri-error +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 ((composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 (hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + U +

$(P + \text{staged-concrete-transcript-target-error-bound}) +$
 $(P + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\text{trace-fri-error} +$
 $\text{low-degree-trace-pair-agreement-error budgets})) +$
 $\text{trace-fri-error} + \text{empty-query-error})$

proof –

have *transcript-new-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-new-hit}$
 $\text{adversary-initial-state} \leq \text{staged-concrete-transcript-target-error-bound}$

by (rule
 $\text{checked-staged-security-with-data-state-transcript-new-hit-bound}$
 $[\text{OF wf controlled}]$)

have *no-prefix-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate}$
 $\text{adversary-initial-state} \leq$
 $P + \text{staged-concrete-transcript-target-error-bound}$

by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-bound-from-sampled-transcript}$
 $[\text{OF data-pre-bound transcript-new-bound}]$)

have *path-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output}$
 $\text{adversary-initial-state} \leq$
 $P + \text{staged-concrete-transcript-target-error-bound}$

by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-bound-from-sampled-transcript}$
 $[\text{OF data-pre-bound transcript-new-bound}]$)

have *collision-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-collision}$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0 \text{ (staged-alpha-search-queries budgets } 0)$

by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound}$
 $[\text{OF wf controlled}]$)

have *not-prefix-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0 \text{ (staged-alpha-search-queries budgets } 0) +$
 $U + (P + \text{staged-concrete-transcript-target-error-bound}) +$

```

    (P + staged-concrete-transcript-target-error-bound)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
    [OF wf controlled collision-bound uncovered-bound no-prefix-bound
      path-bound])
  show ?thesis
  by (rule
    stark-soundness-from-transcript-indexed-partial-opening-route-from-not-prefix-single-query-charge
    [OF false-statement wf controlled trace-fri-reduction
      composition-fri-reduction round-bound not-prefix-bound
      current-empty-bound])
qed
end
end
end

```

theory *Soundness-Not-Prefix-Partial-Opening*
imports *Soundness-Relevant-Drift-Transcript-Public*
begin

Same-run partial-opening packaging for the remaining not-prefix residual.

The residual is not a probability-small sparse-cover event by itself. Instead, it records the precise verifier evidence still carried by the current route: an accepting verifier run, authenticated partial trace openings from that same run, and a trace-domain index left unopened by those sampled openings. This layer gives that evidence a non-diagnostic name and keeps the broad witnessed cross/Merkle classifier out of the checked wrapper.

context *soundness*
begin

definition *partial-trace-openings-leave-domain-gap*
 $:: 'f \text{ authenticated-opening list list} \Rightarrow \text{bool}$
where
partial-trace-openings-leave-domain-gap *trace-openings* $\longleftrightarrow$
 $(\exists j < \text{scale} * \text{clength}.$
 $\forall i \text{ opn. } i < \text{rounds} \longrightarrow$
 $\text{opn} \in \text{set } (\text{trace-openings} ! i) \longrightarrow$
 $\text{opening-index } \text{opn} \neq j)$

definition *checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap*
 $:: (((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where
checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
 $\text{out} \longleftrightarrow$

```

(case out of
  None  $\Rightarrow$  False
| Some (x, -)  $\Rightarrow$ 
  (let packed = fst (fst x);
    prefix-state = snd (fst packed);
    data = snd packed;
    attacker-state = snd (fst x);
    s =
      verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)
  in  $\exists$  fr f-fri-roots f-final trace-table witness-final-state
    trace-openings.
    alpha-header-supported-partial-trace-table-candidate-witness
      s fr f-fri-roots f-final trace-table witness-final-state
      trace-openings  $\wedge$ 
    trace-table  $\notin$ 
      alpha-prefix-trace-table-candidates prefix-state fr  $\wedge$ 
    prefix-state  $\leq$  witness-final-state  $\wedge$ 
    partial-trace-openings-leave-domain-gap trace-openings))

```

lemma *checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap-iff-uncovered-index:*
checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
out $\longleftrightarrow$
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
out

unfolding

```

checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap-def
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-def
partial-trace-openings-leave-domain-gap-def
by (auto split: option.splits prod.splits)

```

definition *checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement*

```

:: ((((((f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
  'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
  'f protocol-channel)  $\times$  unit list)  $\times$ 
  'f protocol-channel) option  $\Rightarrow$  bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement
  out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (x, -)  $\Rightarrow$ 
    (let packed = fst (fst x);
      prefix-state = snd (fst packed);
      data = snd packed;
      attacker-state = snd (fst x);
      s =
        verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)
    )
  )

```

in $\exists fr\ f\text{-fri-roots}\ f\text{-final}\ trace\text{-table}\ prefix\text{-trace-table}$
witness-final-state *trace-openings* *result* *query-idxs* *i*.
alpha-header-supported-partial-trace-table-candidate-witness
s *fr* *f-fri-roots* *f-final* *trace-table* *witness-final-state*
trace-openings $\wedge$
trace-table $\notin$
alpha-prefix-trace-table-candidates *prefix-state* *fr* $\wedge$
prefix-trace-table $\in$
alpha-prefix-trace-table-candidates *prefix-state* *fr* $\wedge$
prefix-state $\leq$ *witness-final-state* $\wedge$
accepted-with-partial-trace-openings *s*
(Some (result, witness-final-state)) *fr* *query-idxs*
trace-openings $\wedge$
i $<$ *rounds* $\wedge$
query-idxs ! *i* $\in$
trace-table-agreement-indices *trace-table*
prefix-trace-table))

definition *checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-sampled-agreement*

$:: 'f\ staged\text{-adversary} \Rightarrow$
 $(((((('f \times 'f\ list \times 'f\ list \times 'f) \times$
 $'f\ protocol\text{-channel}) \times 'f\ staged\text{-proof-data}) \times$
 $'f\ protocol\text{-channel}) \times unit\ list) \times$
 $'f\ protocol\text{-channel})\ option \Rightarrow bool$

where

checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-sampled-agreement
A out $\longleftrightarrow$
(case out of
None $\Rightarrow$ *False*
| Some (x, -) $\Rightarrow$
 $(let\ packed = fst\ (fst\ x);$
 $prefix\text{-state} = snd\ (fst\ packed);$
 $data = snd\ packed;$
 $attacker\text{-state} = snd\ (fst\ x);$
 $s =$
 $verifier\text{-state-from-adversary}\ attacker\text{-state}$
 $(staged\text{-proof-transcript}\ data)$
in $\exists fr\ f\text{-fri-roots}\ f\text{-final}\ trace\text{-table}\ prefix\text{-trace-table}$
witness-final-state *trace-openings* *result* *query-idxs* *as i*
prefix *prefix-state'* *raw* *raw-state*.
alpha-header-supported-partial-trace-table-candidate-witness
s *fr* *f-fri-roots* *f-final* *trace-table* *witness-final-state*
trace-openings $\wedge$
trace-table $\notin$
alpha-prefix-trace-table-candidates *prefix-state* *fr* $\wedge$
prefix-trace-table $\in$
alpha-prefix-trace-table-candidates *prefix-state* *fr* $\wedge$
trace-table-low-degree *trace-table* $\wedge$
trace-table-low-degree *prefix-trace-table* $\wedge$

$trace-table \neq prefix-trace-table \wedge$
 $accepted-with-partial-trace-openings\ s$
 $(Some\ (result,\ witness-final-state))\ fr\ query-idxs$
 $trace-openings \wedge$
 $accepted-transcript-shape\ s\ (Some\ (result,\ witness-final-state))$
 $as\ query-idxs \wedge$
 $i < rounds \wedge$
 $Some\ (data,\ attacker-state) \in$
 $set-dist$
 $(execute\ (checked-staged-transcript-program\ A)$
 $adversary-initial-state) \wedge$
 $Some\ (((prefix,\ prefix-state'),\ raw),\ raw-state) \in$
 $set-dist$
 $(execute\ (checked-staged-query-prefix-receive-with-state\ A\ i)$
 $adversary-initial-state) \wedge$
 $index\ (to-nat\ raw) \in$
 $trace-table-agreement-indices\ trace-table$
 $prefix-trace-table))$

definition *checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement*

$:: ((((((f \times 'f\ list \times 'f\ list \times 'f) \times$
 $'f\ protocol-channel) \times 'f\ staged-proof-data) \times$
 $'f\ protocol-channel) \times unit\ list) \times$
 $'f\ protocol-channel)\ option \Rightarrow bool$

where

$checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement$
 $out \longleftrightarrow$
 $(case\ out\ of$
 $None \Rightarrow False$
 $| Some\ (((((alpha-prefix,\ prefix-state),\ data),\ attacker-state),$
 $result),\ final-state) \Rightarrow$
 $(let\ s =$
 $verifier-state-from-adversary\ attacker-state$
 $(staged-proof-transcript\ data)$
 $in\ \exists fr\ f-fri-roots\ f-final\ trace-table\ prefix-trace-table$
 $witness-final-state\ trace-openings\ as\ query-idxs\ i.$
 $alpha-header-supported-partial-trace-table-candidate-witness$
 $s\ fr\ f-fri-roots\ f-final\ trace-table\ witness-final-state$
 $trace-openings \wedge$
 $trace-table \notin$
 $alpha-prefix-trace-table-candidates\ prefix-state\ fr \wedge$
 $prefix-trace-table \in$
 $alpha-prefix-trace-table-candidates\ prefix-state\ fr \wedge$
 $trace-table-low-degree\ trace-table \wedge$
 $trace-table-low-degree\ prefix-trace-table \wedge$
 $trace-table \neq prefix-trace-table \wedge$
 $accepted-transcript-shape\ s\ (Some\ (result,\ final-state))$
 $as\ query-idxs \wedge$
 $i < rounds \wedge$

query-idxs ! i ∈
trace-table-agreement-indices trace-table
prefix-trace-table))

lemma *checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-same-run-partial-opening-gap:*

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state ≤
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
adversary-initial-state
by *(rule wp-event-mono)*
(simp add:
checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap-iff-uncovered-index)

lemma *checked-staged-security-with-actual-alpha-prefix-same-run-gap-imp-witness-transcript-hit:*

assumes
checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
out
shows
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
out
proof *(cases out)*
case *None*
then show *?thesis*
using *assms*
unfolding
checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap-def
by *simp*
next
case *(Some packed)*
then obtain *prefix prefix-state data attacker-state result final-state*
where *out-eq:*
out =
Some (((((prefix, prefix-state), data), attacker-state), result),
final-state)
by *(cases packed, auto split: prod.splits)*
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
from *assms* **obtain** *fr f-fri-roots f-final trace-table*
witness-final-state trace-openings **where** *witness:*
alpha-header-supported-partial-trace-table-candidate-witness ?s fr
f-fri-roots f-final trace-table witness-final-state trace-openings
unfolding *out-eq*
checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap-def
Let-def

```

    by (auto split: prod.splits)
  from witness obtain result' query-idxs as dg composition-fri-roots final
    rest where outcome:
      Some (result', witness-final-state) ∈
        set-dist (execute verify-monad ?s)
  unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
  by blast
  have tr-eq: PTranscript ?s = staged-proof-transcript data
  by simp
  have transcript-hit:
    hash-map-output-values ?s ∩ set (staged-proof-transcript data)
      ≠ {} ∨
    hash-map-new-output-hit (set (staged-proof-transcript data))
      ?s witness-final-state
  by (rule verify-monad-trace-root-target-preexisting-or-new
    [OF outcome tr-eq])
  show ?thesis
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-def
    Let-def
  using outcome transcript-hit
  by (auto split: prod.splits)
qed

lemma checked-staged-security-with-actual-alpha-prefix-same-run-gap-le-witness-transcript-hit:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
    adversary-initial-state
  by (rule wp-event-mono)
    (rule
      checked-staged-security-with-actual-alpha-prefix-same-run-gap-imp-witness-transcript-hit)

lemma checked-staged-security-with-actual-alpha-prefix-same-run-gap-bound-from-pre-and-new:
  assumes pre-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
      adversary-initial-state ≤ P
  and new-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit

```

$adversary-initial-state \leq Q$
shows
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$
 $checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap$
 $adversary-initial-state \leq P + Q$
by (rule $order-trans$
 $[OF$
 $checked-staged-security-with-actual-alpha-prefix-same-run-gap-le-witness-transcript-hit$
 $checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new$
 $[OF\ pre-bound\ new-bound]])$

lemma $checked-staged-security-with-actual-alpha-prefix-same-run-gap-bound-from-data-pre-and-new$:
assumes $data-pre-bound$:
 $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $staged-security-with-data-state-transcript-pre-hit$
 $adversary-initial-state \leq P$
and $new-bound$:
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$
 $checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit$
 $adversary-initial-state \leq Q$
shows
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$
 $checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap$
 $adversary-initial-state \leq P + Q$

proof –
have $pre-bound$:
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state\ A)$
 $checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit$
 $adversary-initial-state \leq P$
by (rule
 $checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit$
 $[OF\ data-pre-bound])$
show $?thesis$
by (rule
 $checked-staged-security-with-actual-alpha-prefix-same-run-gap-bound-from-pre-and-new$
 $[OF\ pre-bound\ new-bound])$

qed

lemma $checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit-bound-from-sampled-transcript$:
assumes $pre-bound$:
 $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $staged-security-with-data-state-transcript-pre-hit$

```

    adversary-initial-state  $\leq P$ 
and new-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-new-hit
    adversary-initial-state  $\leq R$ 
shows
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
    adversary-initial-state  $\leq P + R$ 
proof –
have event-le:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
    adversary-initial-state  $\leq$ 
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
    out  $\in$ 
    set-dist
    (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    adversary-initial-state)
and hit:
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
    out
show
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
    out
proof (cases out)
case None
then show ?thesis
    using hit
    unfolding
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit-def
    by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state
where out-eq:
    out =
    Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state)
by (cases packed) (auto split: prod.splits)

```

```

show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-actual-alpha-prefix-some-support-imp-sampled-transcript-hit)
    (rule support[unfolded out-eq])
qed
qed
have sampled-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
    adversary-initial-state  $\leq P + R$ 
  by (rule
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit-bound-from-data-state
    [OF pre-bound new-bound])
show ?thesis
  by (rule order-trans[OF event-le sampled-bound])
qed

```

```

lemma checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-same-run-
fixes G P R :: prob
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and gap-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
    adversary-initial-state  $\leq G$ 
and data-pre-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state  $\leq P$ 
and transcript-new-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-new-hit
    adversary-initial-state  $\leq R$ 
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
    adversary-initial-state  $\leq$ 
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) + G + (P + R) + (P + R)
proof –
have collision-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)

```

checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
adversary-initial-state $\leq$
hash-collision-budget-value 0 (*staged-alpha-search-queries* budgets 0)
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
[*OF wf controlled*])
have *uncovered-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state $\leq G$
by (rule *order-trans*
[*OF*
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-same-run-partial-opening-gap
gap-bound])
have *no-prefix-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq P + R$
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-bound-from-sampled-transcript
[*OF data-pre-bound transcript-new-bound*])
have *path-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq P + R$
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-bound-from-sampled-transcript
[*OF data-pre-bound transcript-new-bound*])
show *?thesis*
by (rule
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-not-prefix-com
[*OF wf controlled collision-bound uncovered-bound no-prefix-bound*
path-bound])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-same-run-*
fixes *P Q R :: prob*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *data-pre-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$

and *witness-new-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
 adversary-initial-state $\leq Q$
and *transcript-new-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 staged-security-with-data-state-transcript-new-hit
 adversary-initial-state $\leq R$
shows
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state $\leq$
 hash-collision-budget-value 0
 (*staged-alpha-search-queries* budgets 0) + (*P* + *Q*) + (*P* + *R*) +
 (*P* + *R*)
proof –
 have *gap-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
 adversary-initial-state $\leq P + Q$
 by (rule
 checked-staged-security-with-actual-alpha-prefix-same-run-gap-bound-from-data-pre-and-new
 [*OF data-pre-bound witness-new-bound*])
 show ?thesis
 by (rule
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-same-run-gap
 [*OF wf controlled gap-bound data-pre-bound transcript-new-bound*])
qed

theorem *stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-from-same-*
fixes *G P empty-query-error* :: *prob*
and *C* :: *nat* $\Rightarrow$ *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed* budgets
and *controlled*: *staged-adversary-controlled* budgets *A*
and *trace-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
 Some (*data*, *attacker-state*) $\in$
 set-dist (*execute* (*checked-staged-transcript-program* *A*)
 adversary-initial-state) $\implies$
 trace-fri-reachable-partial-candidate-reduction-assumption
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
and *composition-fri-reduction*:

$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 composition-fri-reachable-partial-candidate-reduction-assumption
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
and round-bound:
 $\wedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq C i$
and gap-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
 adversary-initial-state $\leq G$
and data-pre-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-transcript-pre-hit
 adversary-initial-state $\leq P$
and current-empty-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq \text{empty-query-error}$
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds}. C i$) +
 trace-fri-error + composition-fri-error +
 ($\sum i < \text{rounds}. C i$) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 (hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + G +
 (P + staged-concrete-transcript-target-error-bound) +
 (P + staged-concrete-transcript-target-error-bound)) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + empty-query-error)
proof –
have trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ trace-fri-error
by (rule
 trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction
 [OF trace-fri-reduction])
have comp-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ composition-fri-error
by (rule
 composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction
 [OF composition-fri-reduction])
have empty-trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ trace-fri-error
by (rule
 trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
 [OF trace-fri-reduction])
have transcript-new-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-transcript-new-hit
 adversary-initial-state $\leq$ staged-concrete-transcript-target-error-bound
by (rule checked-staged-security-with-data-state-transcript-new-hit-bound
 [OF wf controlled])
have drift-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state $\leq$
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + G +
 (P + staged-concrete-transcript-target-error-bound) +
 (P + staged-concrete-transcript-target-error-bound)
by (rule
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-same-run-gap
 [OF wf controlled gap-bound data-pre-bound transcript-new-bound])
have base:
 checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds. } C \ i$) +
 trace-fri-error + composition-fri-error +
 ($\sum i < \text{rounds. } C \ i$) +
 (composition-error-bound +

```

    staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
    (hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) + G +
      (P + staged-concrete-transcript-target-error-bound) +
      (P + staged-concrete-transcript-target-error-bound)) +
    trace-fri-error + empty-query-error)
  by (rule
    checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix-ro
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      round-bound drift-bound empty-trace-fri-bound current-empty-bound])
  then show ?thesis
    by (simp add: algebra-simps)
qed

```

lemma *powers-scaled-domain-bound-imp-query-sample-space:*

assumes *bounded:*

$\bigwedge j. j \in \text{set } (\text{powers-scaled } \text{idx}) \implies j < \text{clength} * \text{scale}$

shows $\text{idx} \in \text{query-sample-space}$

proof –

let $?m = \text{Max } (\text{set } [0..<\text{powers}])$

have $m\text{-in}: ?m \in \text{set } [0..<\text{powers}]$

using *powers-pos* **by** (intro *Max-in*) *simp-all*

have $m\text{-eq}: ?m = \text{powers} - 1$

using *powers-pos* **by** (intro *Max-eqI*) *auto*

have $\text{idx-m-bound}: \text{idx} + ?m * \text{scale} < \text{clength} * \text{scale}$

by (rule *bounded*) (use *m-in* **in** $\langle \text{auto simp: powers-scaled-def} \rangle$)

have $m\text{-le}: ?m * \text{scale} \leq \text{clength} * \text{scale}$

proof –

have $?m < \text{clength}$

using *m-eq powers-le-clength powers-pos* **by** *simp*

then show *?thesis*

by (simp add: *mult-le-mono*)

qed

have $\text{idx} < \text{clength} * \text{scale} - ?m * \text{scale}$

using *idx-m-bound m-le* **by** *linarith*

then show *?thesis*

unfolding *query-sample-space-def query-sample-space-size-def* **by** *simp*

qed

lemma *partial-authenticated-trace-openings-query-index-in-sample-space:*

assumes *indices:*

$\text{map } \text{opening-index } \text{openings} = \text{powers-scaled } \text{idx}$

and *auth:*

$\text{partial-authenticated-table } \text{fr } (\text{scale} * \text{clength}) \text{ openings } s$

shows $idx \in \text{query-sample-space}$
proof (rule powers-scaled-domain-bound-imp-query-sample-space)
fix j
assume $j\text{-in}: j \in \text{set (powers-scaled idx)}$
have $j \in \text{opening-index ' set openings}$
by (metis indices $j\text{-in}$ set-map)
then obtain opn **where** $opn\text{-in}: opn \in \text{set openings}$
and $j\text{-eq}: j = \text{opening-index } opn$
by blast
have $\text{opening-index } opn < \text{opening-length } opn$
using $\text{auth } opn\text{-in}$ **unfolding** $\text{partial-authenticated-table-def}$
 $\text{authenticated-opening-in-def}$ **by** blast
moreover have $\text{opening-length } opn = \text{scale} * \text{clength}$
using $\text{auth } opn\text{-in}$ **unfolding** $\text{partial-authenticated-table-def}$ **by** blast
ultimately show $j < \text{clength} * \text{scale}$
using $j\text{-eq}$ **by** (simp add: mult.commute)
qed

lemma *accepted-partial-trace-openings-leave-domain-gap-if-sparse-queries:*
assumes *partial:*
 $\text{accepted-with-partial-trace-openings } s \text{ out fr query-idxs trace-openings}$
and *sparse:* $\text{rounds} * \text{powers} < \text{scale} * \text{clength}$
shows *partial-trace-openings-leave-domain-gap trace-openings*
proof –
have $\text{rounds-len}: \text{length trace-openings} = \text{rounds}$
by (rule *accepted-with-partial-trace-openings-shapes(2)*[OF *partial*])
have *concat-sparse:*
 $\text{length (List.concat trace-openings)} < \text{scale} * \text{clength}$
using *accepted-with-partial-trace-openings-concat-length*[OF *partial*]
 sparse **by** simp
obtain j **where** $j\text{-bound}: j < \text{scale} * \text{clength}$
and *unopened:*
 $\bigwedge i \text{ opn. } i < \text{rounds} \implies$
 $opn \in \text{set (trace-openings ! } i) \implies$
 $\text{opening-index } opn \neq j$
using *sparse-openings-obtain-unopened-index*[OF $\text{rounds-len concat-sparse}$]
by blast
show *?thesis*
unfolding *partial-trace-openings-leave-domain-gap-def*
using $j\text{-bound}$ *unopened* **by** blast
qed

lemma *checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap-if-sparse-queries:*
assumes *wf:* *staged-budget-wellformed budgets*
and *controlled:* *staged-adversary-controlled budgets A*
and *sparse:* $\text{rounds} * \text{powers} < \text{scale} * \text{clength}$
and *support:*
 $\text{out} \in$
 set-dist

```

      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
    and hit:
      checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
      bad-sets out
    shows
      checked-staged-security-with-actual-alpha-prefix-not-prefix-collision out ∨
      checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate out ∨
      checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output out ∨
      checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap out
  proof (cases out)
  case None
  then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state
  where out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from hit obtain fr f-fri-roots f-final as where not-prefix:
  alpha-header-partial-candidate-not-prefix-bound prefix-state ?s fr
  f-fri-roots f-final
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound-def
  Let-def
by (auto split: option.splits prod.splits)
from checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-obtain-witness-final-ext
  [OF wf controlled support[unfolded out-eq] not-prefix]
obtain trace-table witness-final-state trace-openings where witness:
  alpha-header-supported-partial-trace-table-candidate-witness ?s fr
  f-fri-roots f-final trace-table witness-final-state trace-openings
and table-not-prefix:
  trace-table ∉ alpha-prefix-trace-table-candidates prefix-state fr
and prefix-ext: prefix-state ≤ witness-final-state
by blast
from witness obtain result' query-idxs as' dg composition-fri-roots final rest
where partial:
  accepted-with-partial-trace-openings ?s

```

```

    (Some (result', witness-final-state)) fr query-idxs trace-openings
unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
by blast
have gap:
  partial-trace-openings-leave-domain-gap trace-openings
by (rule accepted-partial-trace-openings-leave-domain-gap-if-sparse-queries
    [OF partial sparse])
have
  checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
  out
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap-def
  Let-def
using witness table-not-prefix prefix-ext gap
by (simp split: prod.splits, blast)
then show ?thesis by simp
qed

lemma alpha-header-witness-prefix-candidate-values-agree-on-openings:
assumes prefix-candidate:
  prefix-trace-table ∈
  alpha-prefix-trace-table-candidates prefix-state fr
and witness:
  alpha-header-supported-partial-trace-table-candidate-witness s fr
  f-fri-roots f-final trace-table final-state trace-openings
and pullback:
  partial-authenticated-table fr (scale * clength)
  (trace-openings ! i) prefix-state
and clean-prefix:  $\neg$  hash-map-output-collision prefix-state
and i-bound:  $i < \text{rounds}$ 
shows
  map ((! trace-table) (map opening-index (trace-openings ! i))) =
  map ((! prefix-trace-table) (map opening-index (trace-openings ! i)))
proof –
have prefix-len:
  length prefix-trace-table = clength * scale
using prefix-candidate
unfolding alpha-prefix-trace-table-candidates-def by simp
have prefix-bind:
  merkle-root-binds-table fr prefix-trace-table prefix-state
using prefix-candidate
unfolding alpha-prefix-trace-table-candidates-def by simp
from witness obtain result query-idxs as dg composition-fri-roots final rest
where trace-cand:
  partial-trace-table-candidate trace-table trace-openings
unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
by blast
have trace-agrees:
  table-agrees-with-authenticated-openings trace-table

```

```

    (scale * clength) (trace-openings ! i)
  using trace-cand i-bound
  unfolding partial-trace-table-candidate-def by blast
have point-eq:
   $\bigwedge \text{opn. } \text{opn} \in \text{set } (\text{trace-openings} ! i) \implies$ 
    trace-table ! opening-index opn =
    prefix-trace-table ! opening-index opn
proof -
  fix opn
  assume opn-in: opn  $\in$  set (trace-openings ! i)
  have trace-value:
    trace-table ! opening-index opn = opening-value opn
  using trace-agrees opn-in
  unfolding table-agrees-with-authenticated-openings-def by blast
have root: opening-root opn = fr
  using pullback opn-in unfolding partial-authenticated-table-def
  by blast
have len: opening-length opn = length prefix-trace-table
  using pullback opn-in prefix-len
  unfolding partial-authenticated-table-def by simp
have auth: authenticated-opening-in prefix-state opn
  using pullback opn-in unfolding partial-authenticated-table-def
  by blast
obtain n where len-pow: clength * scale =  $2^n$ 
  using eval-domain-length-power by blast
have table-pow: length prefix-trace-table =  $2^n$ 
  using prefix-len len-pow by (simp add: mult.commute)
have prefix-value:
  prefix-trace-table ! opening-index opn = opening-value opn
  using
    merkle-bound-table-agrees-with-authenticated-opening-if-clean
    [OF prefix-bind table-pow auth root len clean-prefix]
  by simp
show
  trace-table ! opening-index opn =
  prefix-trace-table ! opening-index opn
  using trace-value prefix-value by simp
qed
show ?thesis
  by (rule nth-equalityI)
  (use point-eq in auto)
qed

lemma alpha-header-witness-prefix-candidate-trace-table-agreement-index:
  assumes prefix-candidate:
    prefix-trace-table  $\in$ 
    alpha-prefix-trace-table-candidates prefix-state fr
  and witness:
    alpha-header-supported-partial-trace-table-candidate-witness s fr

```

```

    f-fri-roots f-final trace-table final-state trace-openings
  and pullback:
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) prefix-state
  and clean-prefix:  $\neg$  hash-map-output-collision prefix-state
  and i-bound:  $i < \text{rounds}$ 
  and idx-sample:  $\text{idx} \in \text{query-sample-space}$ 
  and idx-openings:
    map opening-index (trace-openings ! i) = powers-scaled idx
  shows  $\text{idx} \in \text{trace-table-agreement-indices trace-table prefix-trace-table}$ 
proof -
  have agree:
    map (! trace-table) (map opening-index (trace-openings ! i)) =
    map (! prefix-trace-table) (map opening-index (trace-openings ! i))
  by (rule alpha-header-witness-prefix-candidate-values-agree-on-openings
    [OF prefix-candidate witness pullback clean-prefix i-bound])
  show ?thesis
    unfolding trace-table-agreement-indices-def
    using idx-sample agree idx-openings by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-imp-sampled-agreement-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    out  $\in$ 
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
    bad-sets out
  shows
    checked-staged-security-with-actual-alpha-prefix-not-prefix-collision out  $\vee$ 
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate out  $\vee$ 
    checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output out  $\vee$ 
    checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state data attacker-state result final-state

```

```

where out-eq:
  out =
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from hit obtain fr f-fri-roots f-final as where not-prefix-bound:
  alpha-header-partial-candidate-not-prefix-bound prefix-state ?s fr
  f-fri-roots f-final
unfolding out-eq
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound-def
Let-def
by (auto split: option.splits prod.splits)
from checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-obtain-witness-final-ext
  [OF wf controlled support[unfolded out-eq] not-prefix-bound]
obtain trace-table witness-final-state trace-openings where witness:
  alpha-header-supported-partial-trace-table-candidate-witness ?s fr
  f-fri-roots f-final trace-table witness-final-state trace-openings
and table-not-prefix:
  trace-table  $\notin$  alpha-prefix-trace-table-candidates prefix-state fr
and prefix-ext: prefix-state  $\leq$  witness-final-state
by blast
show ?thesis
proof (cases hash-map-output-collision prefix-state)
case True
then show ?thesis
  unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-def
  by simp
next
case clean-prefix: False
show ?thesis
proof (cases alpha-prefix-trace-table-candidates prefix-state fr = {})
case True
then have
  checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
  out
  unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-def
  Let-def
  using witness table-not-prefix prefix-ext by (simp, blast)
then show ?thesis by simp
next
case nonempty: False
then obtain prefix-trace-table where prefix-candidate:
  prefix-trace-table  $\in$ 
  alpha-prefix-trace-table-candidates prefix-state fr

```



```

by blast
have pullback-or-new:
  ( $\forall i < \text{rounds}.$ 
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) prefix-state)  $\vee$ 
  ( $\exists i < \text{rounds}.$   $\exists \text{opn} \in \text{set}$  (trace-openings ! i).
    hash-map-new-output-hit
    (merkle-path-target-roots witness-final-state fr
      (scale * clength) (opening-index opn)
      (opening-value opn) (opening-path opn))
    prefix-state witness-final-state)
by (rule alpha-header-witness-openings-pullback-or-new-output-hit
  [OF prefix-ext witness])
from pullback-or-new show ?thesis
proof
  assume pullback-all:
     $\forall i < \text{rounds}.$ 
      partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) prefix-state
  from witness obtain witness-result query-idxs as dg
    composition-fri-roots final rest where partial:
      accepted-with-partial-trace-openings ?s
      (Some (witness-result, witness-final-state)) fr query-idxs
      trace-openings
  unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
  by blast
have i-bound:  $0 < \text{rounds}$ 
  by (rule rounds-positive)
have idx-openings:
    map opening-index (trace-openings ! 0) =
    powers-scaled (query-idxs ! 0)
  by (rule accepted-with-partial-trace-openings-shapes(3)
    [OF partial i-bound])
have pullback0:
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! 0) prefix-state
  using pullback-all i-bound by blast
have idx-sample: query-idxs ! 0  $\in$  query-sample-space
  by (rule partial-authenticated-trace-openings-query-index-in-sample-space
    [OF idx-openings pullback0])
have agreement:
    query-idxs ! 0  $\in$ 
    trace-table-agreement-indices trace-table prefix-trace-table
  by (rule
    alpha-header-witness-prefix-candidate-trace-table-agreement-index
    [OF prefix-candidate witness pullback0 clean-prefix i-bound
      idx-sample idx-openings])
have
  checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement

```

```

    out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement-def
    Let-def
  using witness table-not-prefix prefix-candidate prefix-ext partial
    i-bound agreement
  by (simp split: prod.splits, blast)
then show ?thesis by simp
next
  assume new:
     $\exists i < \text{rounds}. \exists \text{opn} \in \text{set } (\text{trace-openings } ! i).$ 
    hash-map-new-output-hit
    (merkle-path-target-roots witness-final-state fr
      (scale * clength) (opening-index opn)
      (opening-value opn) (opening-path opn))
    prefix-state witness-final-state
  have
    checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
    out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-def
    Let-def
  using witness table-not-prefix prefix-ext new
  by (simp split: prod.splits, blast)
then show ?thesis by simp
qed
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-sampled-agreement-union-bound:
  fixes C S N P :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and collision-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
    adversary-initial-state  $\leq C$ 
  and sampled-agreement-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement
    adversary-initial-state  $\leq S$ 
  and no-prefix-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state

```

A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq N$
and *path-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq P$
shows
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
bad-sets)
adversary-initial-state $\leq C + S + N + P$
proof –
let $?M =$
checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let $?B =$
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
bad-sets
let $?C =$ *checked-staged-security-with-actual-alpha-prefix-not-prefix-collision*
let $?S =$
checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement
let $?N =$
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
let $?P =$ *checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output*
have *wp-event* $?M ?B$ *adversary-initial-state* $\leq$
wp-event $?M (\lambda out. ?C out \vee ?S out \vee ?N out \vee ?P out)$
adversary-initial-state
proof (rule *wp-event-mono-on-support*)
fix *out*
assume *support*:
out $\in$ *set-dist* (*execute* $?M$ *adversary-initial-state*)
and *hit*: $?B out$
from *checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-imp-sampled-agreement-on-support*
 [*OF* *wf* *controlled support hit*]
show $?C out \vee ?S out \vee ?N out \vee ?P out$
by *blast*
qed
also have $\dots \leq$
wp-event $?M ?C$ *adversary-initial-state* +
wp-event $?M ?S$ *adversary-initial-state* +
wp-event $?M ?N$ *adversary-initial-state* +
wp-event $?M ?P$ *adversary-initial-state*
by (rule *wp-event-union-bound4*)
also have $\dots \leq C + S + N + P$
by (*intro add-mono collision-bound sampled-agreement-bound*)

```

      no-prefix-bound path-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement-imp-witness-transcript-hit:
  assumes
    checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement
    out
  shows
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
    out
proof (cases out)
case None
  then show ?thesis
  using assms
  unfolding
    checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement-def
  by simp
next
case (Some packed)
  then obtain prefix prefix-state data attacker-state result final-state
  where out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state)
  by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from assms obtain fr f-fri-roots f-final trace-table prefix-trace-table
    witness-final-state trace-openings witness-result query-idxs i where
    witness:
      alpha-header-supported-partial-trace-table-candidate-witness ?s fr
      f-fri-roots f-final trace-table witness-final-state trace-openings
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement-def
    Let-def
  by (auto split: prod.splits)
  from witness obtain result' query-idxs' as dg composition-fri-roots final
  rest where outcome:
    Some (result', witness-final-state) ∈
      set-dist (execute verify-monad ?s)
  unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
  by blast
  have tr-eq: PTranscript ?s = staged-proof-transcript data
  by simp
  have transcript-hit:
    hash-map-output-values ?s ∩ set (staged-proof-transcript data)
    ≠ {} ∨

```

```

    hash-map-new-output-hit (set (staged-proof-transcript data))
    ?s witness-final-state
  by (rule verify-monad-trace-root-target-preexisting-or-new
      [OF outcome tr-eq])
show ?thesis
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-def
  Let-def
using outcome transcript-hit
by (auto split: prod.splits)
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement-le-witness-transcript-hit:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement
  adversary-initial-state  $\leq$ 
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
  adversary-initial-state
by (rule wp-event-mono)
(rule
  checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement-imp-witness-transcript-hit)

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement-bound-from-pre-and-new:
  assumes pre-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
    adversary-initial-state  $\leq P$ 
  and new-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
    adversary-initial-state  $\leq Q$ 
  shows
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement
    adversary-initial-state  $\leq P + Q$ 
by (rule order-trans
    [OF
      checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement-le-witness-transcript-hit
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
      [OF pre-bound new-bound]])

```

lemma *checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-imp-sampled-transcript-hit-on-support*:
assumes *support*:
 out ∈
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 adversary-initial-state)
and *hit*:
 checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
 bad-sets out
shows
 checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
 out
proof (*cases out*)
case *None*
then show *?thesis*
 using *hit*
 unfolding
 checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound-def
 by *simp*
next
case (*Some packed*)
then obtain *prefix prefix-state data attacker-state result final-state*
where *out-eq*:
 out =
 Some (((((prefix, prefix-state), data), attacker-state), result),
 final-state)
 by (*cases packed, auto split: prod.splits*)
let *?s* =
 verifier-state-from-adversary attacker-state
 (*staged-proof-transcript data*)
have *data-support*:
 Some (((data, attacker-state), result), final-state) ∈
 set-dist
 (*execute (checked-staged-security-experiment-with-data-state A)*
 adversary-initial-state)
 by (*rule*
 checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
 [*OF support[unfolded out-eq]*])
have *verifier*:
 Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
 using *checked-staged-security-experiment-with-data-state-outcomeE*
 [*OF data-support*]
 by *blast*
have *tr-eq*: *PTranscript ?s = staged-proof-transcript data*
 by *simp*
have *split*:

```

hash-map-output-values ?s  $\cap$  set (staged-proof-transcript data)
 $\neq \{\}$   $\vee$ 
hash-map-new-output-hit (set (staged-proof-transcript data))
?s final-state
by (rule verify-monad-trace-root-target-preexisting-or-new
[OF verifier tr-eq])
show ?thesis
unfolding out-eq
checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit-def
staged-security-with-data-state-transcript-pre-hit-def
staged-security-with-data-state-transcript-new-hit-def
using split by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-le-sampled-transcript-hit:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
bad-sets)
adversary-initial-state  $\leq$ 
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
adversary-initial-state
by (rule wp-event-mono-on-support)
(rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-imp-sampled-transcript-hit-on-support)

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-from-sampled-transcript:
assumes pre-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state  $\leq P$ 
and new-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-new-hit
adversary-initial-state  $\leq R$ 
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
bad-sets)
adversary-initial-state  $\leq P + R$ 
by (rule order-trans
[OF
checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-le-sampled-transcript-hit
checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit-bound-from-data-state
[OF pre-bound new-bound]]))

```

lemma *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-sampled-tr*
fixes $P\ R :: \text{prob}$
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *data-pre-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-transcript-pre-hit
 adversary-initial-state $\leq P$
and *transcript-new-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-transcript-new-hit
 adversary-initial-state $\leq R$
shows
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state $\leq P + R$
proof –
have *not-prefix-bound:*
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq P + R$
by (rule
 checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-from-sampled-transcript
 [OF *data-pre-bound transcript-new-bound*])
show *?thesis*
by (rule *order-trans*
 [OF
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-le-not-prefix-bound
 not-prefix-bound])
 (use wf controlled in simp-all)
qed

theorem *stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-from-samp*
fixes $P\ \text{empty-query-error} :: \text{prob}$
and $C :: \text{nat} \Rightarrow \text{prob}$
assumes *false-statement: $\neg \text{exists-valid-trace}$*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-reduction:*
 $\bigwedge \text{data attacker-state.}$
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 trace-fri-reachable-partial-candidate-reduction-assumption
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))

and *composition-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
composition-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq C i$
and *data-pre-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability A $\leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
($\sum i < \text{rounds}. C i$) +
trace-fri-error + composition-fri-error +
($\sum i < \text{rounds}. C i$) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(P + staged-concrete-transcript-target-error-bound) +
trace-fri-error + empty-query-error)
proof –
have *trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ trace-fri-error
by (rule
trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction
[OF trace-fri-reduction])
have *comp-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)

```

    (staged-security-with-data-state-verifier-event
     composition-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ composition-fri-error
  by (rule
      composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction
      [OF composition-fri-reduction])
  have empty-trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-bad-with-empty-composition-header-candidates)
      adversary-initial-state ≤ trace-fri-error
  by (rule
      trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
      [OF trace-fri-reduction])
  have transcript-new-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-new-hit
      adversary-initial-state ≤ staged-concrete-transcript-target-error-bound
  by (rule checked-staged-security-with-data-state-transcript-new-hit-bound
      [OF wf controlled])
  have drift-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
      adversary-initial-state ≤
        P + staged-concrete-transcript-target-error-bound
  by (rule
      checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-sampled-trans
      [OF wf controlled data-pre-bound transcript-new-bound])
  show ?thesis
  by (rule
      checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix-ro
      [OF false-statement wf controlled trace-fri-bound comp-fri-bound
       round-bound drift-bound empty-trace-fri-bound current-empty-bound])
qed

theorem stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-from-sampled-
fixes P empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
                      adversary-initial-state) ⇒
      trace-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state

```

(staged-proof-transcript data))
and composition-fri-reduction:
 $\bigwedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 composition-fri-reachable-partial-candidate-reduction-assumption
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
and round-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq C i$
and data-pre-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-transcript-pre-hit
 adversary-initial-state $\leq P$
and current-empty-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq \text{empty-query-error}$
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds}. C i$) +
 trace-fri-error + composition-fri-error +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (P + staged-concrete-transcript-target-error-bound) +
 trace-fri-error + empty-query-error)
proof –
have trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq \text{trace-fri-error}$
by (rule
 trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction
 [OF trace-fri-reduction])
have comp-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)

```

    (staged-security-with-data-state-verifier-event
     composition-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ composition-fri-error
  by (rule
      composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction
      [OF composition-fri-reduction])
  have empty-trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-bad-with-empty-composition-header-candidates)
      adversary-initial-state ≤ trace-fri-error
  by (rule
      trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
      [OF trace-fri-reduction])
  have transcript-new-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-new-hit
      adversary-initial-state ≤ staged-concrete-transcript-target-error-bound
  by (rule checked-staged-security-with-data-state-transcript-new-hit-bound
      [OF wf controlled])
  have drift-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
      adversary-initial-state ≤
        P + staged-concrete-transcript-target-error-bound
  by (rule
      checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-sampled-trans
      [OF wf controlled data-pre-bound transcript-new-bound])
  show ?thesis
  by (rule
      checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix-ro
      [OF false-statement wf controlled trace-fri-bound comp-fri-bound
       round-bound drift-bound empty-trace-fri-bound current-empty-bound])
qed

theorem stark-soundness-from-aligned-partial-components-structured-query-and-current-empty-query-from-sampled-t
  fixes P empty-query-error :: prob
  and C :: nat ⇒ prob
  assumes false-statement: ¬ exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-reduction:
    ∧ data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
        trace-fri-reachable-partial-candidate-reduction-assumption
        (verifier-state-from-adversary attacker-state

```

(staged-proof-transcript data))
and composition-fri-reduction:
 $\bigwedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 composition-fri-reachable-partial-candidate-reduction-assumption
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
and round-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq C i$
and data-pre-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-transcript-pre-hit
 adversary-initial-state $\leq P$
and current-empty-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq \text{empty-query-error}$
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds}. C i$) +
 trace-fri-error + composition-fri-error + 1 +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (P + staged-concrete-transcript-target-error-bound) +
 trace-fri-error + empty-query-error)
proof –
have trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq \text{trace-fri-error}$
by (rule
 trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction
 [OF trace-fri-reduction])
have comp-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)

```

    (staged-security-with-data-state-verifier-event
     composition-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ composition-fri-error
  by (rule
      composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction
      [OF composition-fri-reduction])
  have empty-trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-bad-with-empty-composition-header-candidates)
      adversary-initial-state ≤ trace-fri-error
  by (rule
      trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
      [OF trace-fri-reduction])
  have transcript-new-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-new-hit
      adversary-initial-state ≤ staged-concrete-transcript-target-error-bound
  by (rule checked-staged-security-with-data-state-transcript-new-hit-bound
      [OF wf controlled])
  have drift-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
      adversary-initial-state ≤
        P + staged-concrete-transcript-target-error-bound
  by (rule
      checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-sampled-trans
      [OF wf controlled data-pre-bound transcript-new-bound])
  have header-query-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
       A)
      (checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
       A)
      adversary-initial-state ≤ (1::prob)
  by (rule wp-event-le-1)
  show ?thesis
  by (rule
      checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-structured-query-from-
      [OF false-statement wf controlled trace-fri-bound comp-fri-bound
       round-bound header-query-bound drift-bound empty-trace-fri-bound
       current-empty-bound])
qed

theorem stark-soundness-from-transcript-indexed-partial-opening-route-from-sampled-transcript:
  fixes P empty-query-error :: prob
  and C :: nat ⇒ prob
  assumes false-statement: ¬ exists-valid-trace

```

and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-reduction:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *composition-fri-reduction:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *round-bound:*
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state A } i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *data-pre-bound:*
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$
and *current-empty-bound:*
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows *checked-staged-adversary-acceptance-probability A ≤*
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum i < \text{rounds. } C \ i) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\sum i < \text{rounds. } C \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $((\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$

```

    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    (P + staged-concrete-transcript-target-error-bound)) +
    (trace-fri-error +
     low-degree-trace-pair-agreement-error budgets)) +
    trace-fri-error + empty-query-error)
proof -
  have sampled-transcript-new-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-new-hit
    adversary-initial-state ≤ staged-concrete-transcript-target-error-bound
  by (rule
    checked-staged-security-with-data-state-transcript-new-hit-bound
    [OF wf controlled])
  have not-prefix-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤
    P + staged-concrete-transcript-target-error-bound
  by (rule
    checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-from-sampled-transcript
    [OF data-pre-bound sampled-transcript-new-bound])
  show ?thesis
  by (rule
    stark-soundness-from-transcript-indexed-partial-opening-route-from-not-prefix
    [OF false-statement wf controlled trace-fri-reduction
     composition-fri-reduction round-bound not-prefix-bound
     current-empty-bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-sampled-agreement-imp-prefix-pair-hit:
  assumes
    checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-sampled-agreement
    A out
  shows
    ∃ pair ∈ low-degree-trace-pair-space.
    ∃ i ∈ {.. $\text{rounds}$ }.
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-from-prefix
    A (fst pair) (snd pair) i out
proof (cases out)
  case None
  then show ?thesis
  using assms
  unfolding
    checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-sampled-agreement-def
  by simp
next

```



```

case (Some packed)
then obtain alpha-prefix alpha-prefix-state data attacker-state result
  final-state where out-eq:
  out =
    Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state)
  by (cases packed, auto split: prod.splits)
from assms obtain fr f-fri-roots f-final trace-table prefix-trace-table
  witness-final-state trace-openings witness-result query-idxs as i
  prefix prefix-state' raw raw-state where
  trace-low: trace-table-low-degree trace-table
  and prefix-low: trace-table-low-degree prefix-trace-table
  and distinct: trace-table  $\neq$  prefix-trace-table
  and i-bound:  $i < \text{rounds}$ 
  and builder:
    Some (data, attacker-state)  $\in$ 
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and prefix-receive:
    Some (((prefix, prefix-state'), raw), raw-state)  $\in$ 
      set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  and raw-agree:
    index (to-nat raw)  $\in$ 
      trace-table-agreement-indices trace-table prefix-trace-table
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-sampled-agreement-def
    Let-def
  by (auto split: prod.splits)
have pair-in:
  (trace-table, prefix-trace-table)  $\in$  low-degree-trace-pair-space
  unfolding low-degree-trace-pair-space-def
  using trace-low prefix-low distinct by simp
have prefix-hit:
  checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-from-prefix
  A trace-table prefix-trace-table i out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-from-prefix-def
    checked-staged-query-prefix-dynamic-index-hit-def
  apply (simp add: builder)
  using prefix-receive raw-agree by blast
show ?thesis
  by (intro bexI[of - (trace-table, prefix-trace-table)]
    bexI[of - i])
    (use pair-in i-bound prefix-hit in simp-all)
qed

```

```

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement-imp-pair-hit:
  assumes
    checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement
    out
  shows
     $\exists \text{pair} \in \text{low-degree-trace-pair-space}.$ 
     $\exists i \in \{..<\text{rounds}\}.$ 
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
      (fst pair) (snd pair) i out
proof (cases out)
  case None
  then show ?thesis
    using assms
    unfolding
      checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement-def
    by simp
next
  case (Some packed)
  then obtain alpha-prefix prefix-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((alpha-prefix, prefix-state), data), attacker-state),
        result), final-state)
    by (cases packed, auto split: prod.splits)
  from assms obtain fr f-fri-roots f-final trace-table prefix-trace-table
    witness-final-state trace-openings as query-idxs i where
    trace-low: trace-table-low-degree trace-table
    and prefix-low: trace-table-low-degree prefix-trace-table
    and distinct: trace-table  $\neq$  prefix-trace-table
    and shape:
      accepted-transcript-shape
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state)) as query-idxs
    and i-bound: i < rounds
    and idx-agree:
      query-idxs ! i  $\in$ 
      trace-table-agreement-indices trace-table prefix-trace-table
    unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement-def
    Let-def
    by auto
  have pair-in:
    (trace-table, prefix-trace-table)  $\in$  low-degree-trace-pair-space
    unfolding low-degree-trace-pair-space-def
    using trace-low prefix-low distinct by simp
  have pair-hit:
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
      trace-table prefix-trace-table i out

```

```

unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at-def
  staged-security-with-data-state-trace-pair-agreement-hit-at-def
using shape i-bound idx-agree by auto
show ?thesis
  by (intro bexI[of - (trace-table, prefix-trace-table)])
    (bexI[of - i])
    (use pair-in i-bound pair-hit in simp-all)
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement
    adversary-initial-state ≤
    low-degree-trace-pair-agreement-error budgets
proof -
let ?M =
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let ?pair-hit =
   $\lambda(\text{pair}, i) \text{ out.}$ 
  checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
    (fst pair) (snd pair) i out
let ?A = low-degree-trace-pair-space × {..rounds}
have event-le:
  wp-event ?M
  checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement
    adversary-initial-state ≤
  wp-event ?M ( $\lambda \text{out. } \exists x \in ?A. \text{ ?pair-hit } x \text{ out}$ )
  adversary-initial-state
by (rule wp-event-mono)
  (auto dest:
    checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement-imp-pair-hit)
have finite-A: finite ?A
by simp
have union-bound:
  wp-event ?M ( $\lambda \text{out. } \exists x \in ?A. \text{ ?pair-hit } x \text{ out}$ )
  adversary-initial-state ≤
  ( $\sum x \in ?A.$ 
    case x of (pair, i)  $\Rightarrow$ 
    staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      nnreal clength / nnreal (card query-sample-space))
proof (rule wp-event-finite-UN-bound[OF finite-A])
fix x
assume x-in: x ∈ ?A

```

```

then obtain trace-table prefix-trace-table i where x-eq:
  x = ((trace-table, prefix-trace-table), i)
and pair-in:
  (trace-table, prefix-trace-table) ∈ low-degree-trace-pair-space
and i-bound: i < rounds
by (cases x, auto)
have trace-low: trace-table-low-degree trace-table
  using pair-in unfolding low-degree-trace-pair-space-def by auto
have prefix-low: trace-table-low-degree prefix-trace-table
  using pair-in unfolding low-degree-trace-pair-space-def by auto
have distinct: trace-table ≠ prefix-trace-table
  using pair-in unfolding low-degree-trace-pair-space-def by auto
have bound:
  wp-event ?M
  (checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
    trace-table prefix-trace-table i)
  adversary-initial-state ≤
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    nnreal clength / nnreal (card query-sample-space)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at-bound
    [OF wf controlled i-bound trace-low prefix-low distinct])
show wp-event ?M (?pair-hit x) adversary-initial-state
  ≤ (case x of (pair, i) ⇒
    staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      nnreal clength / nnreal (card query-sample-space))
  unfolding x-eq
  using bound by simp
qed
have sum-eq:
  ( $\sum x \in ?A.$ 
    case x of (pair, i) ⇒
      staged-phase-relation-error size
        (staged-query-search-queries budgets i + 1) +
        nnreal clength / nnreal (card query-sample-space)) =
    low-degree-trace-pair-agreement-error budgets
  unfolding low-degree-trace-pair-agreement-error-def
    low-degree-trace-pair-agreement-round-error-def
  by (simp add: sum.cartesian-product case-prod-unfold)
show ?thesis
proof (rule order-trans[OF event-le])
  show wp-event ?M ( $\lambda out. \exists x \in ?A. ?pair-hit x out$ )
    adversary-initial-state ≤
    low-degree-trace-pair-agreement-error budgets
    using union-bound sum-eq by simp
qed
qed

```

theorem *stark-soundness-from-transcript-indexed-partial-opening-route-from-sampled-transcript-not-prefix:*
fixes P *empty-query-error* :: *prob*
and C :: *nat* $\Rightarrow$ *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *composition-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
composition-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \Rightarrow$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq C i$
and *data-pre-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability A $\leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
 $(\sum i < \text{rounds}. C i) +$
trace-fri-error + composition-fri-error +
 $(\sum i < \text{rounds}. C i) +$
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +

```

    staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
    ((composition-error-bound +
      staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
        (staged-alpha-search-queries budgets 0) +
      (P + staged-concrete-transcript-target-error-bound)) +
      (trace-fri-error +
        low-degree-trace-pair-agreement-error budgets)) +
      trace-fri-error + empty-query-error)
  proof -
    have transcript-new-bound:
      wp-event (checked-staged-security-experiment-with-data-state A)
        staged-security-with-data-state-transcript-new-hit
        adversary-initial-state ≤
        staged-concrete-transcript-target-error-bound
    by (rule
      checked-staged-security-with-data-state-transcript-new-hit-bound
      [OF wf controlled])
    have not-prefix-bound:
      wp-event
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
        (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
          composition-trace-bad-alpha-space)
        adversary-initial-state ≤
        P + staged-concrete-transcript-target-error-bound
    by (rule
      checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-from-sampled-transcript
      [OF data-pre-bound transcript-new-bound])
    show ?thesis
    by (rule
      stark-soundness-from-transcript-indexed-partial-opening-route-from-not-prefix
      [OF false-statement wf controlled trace-fri-reduction
        composition-fri-reduction round-bound not-prefix-bound
        current-empty-bound])
  qed

theorem stark-soundness-from-transcript-indexed-partial-opening-route-from-residual-sampled-transcript-not-prefix:
  fixes P empty-query-error :: prob
  and R :: nat ⇒ prob
  assumes false-statement: ¬ exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-reduction:
    ∧ data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒

```

trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *composition-fri-reduction:*
 $\bigwedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
composition-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *residual-bound:*
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
i)
adversary-initial-state $\leq R i$
and *data-pre-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability A $\leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
 $(\sum i < \text{rounds}.$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound + R i +
(P + staged-concrete-transcript-target-error-bound) +
(P + staged-concrete-transcript-target-error-bound)) +
trace-fri-error + composition-fri-error +
 $(\sum i < \text{rounds}.$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound + R i +
(P + staged-concrete-transcript-target-error-bound) +
(P + staged-concrete-transcript-target-error-bound)) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +
 ((composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 (P + staged-concrete-transcript-target-error-bound)) +
 (trace-fri-error +
 low-degree-trace-pair-agreement-error budgets)) +
 trace-fri-error + empty-query-error)

proof –

have transcript-new-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-transcript-new-hit
 adversary-initial-state ≤
 staged-concrete-transcript-target-error-bound

by (rule
 checked-staged-security-with-data-state-transcript-new-hit-bound
 [OF wf controlled])

have trace-path-bound:

∧i. i < rounds ⇒
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
 i)
 adversary-initial-state ≤
 P + staged-concrete-transcript-target-error-bound

by (rule
 checked-staged-security-with-query-prefix-current-trace-path-output-hit-at-bound-from-transcript
 [OF - data-pre-bound transcript-new-bound])

have composition-path-bound:

∧i. i < rounds ⇒
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
 i)
 adversary-initial-state ≤
 P + staged-concrete-transcript-target-error-bound

by (rule
 checked-staged-security-with-query-prefix-current-composition-path-output-hit-at-bound-from-transcript
 [OF - data-pre-bound transcript-new-bound])

have round-bound:

∧i. i < rounds ⇒
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state ≤
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +


```

    query-error-bound + R i +
    (P + staged-concrete-transcript-target-error-bound) +
    (P + staged-concrete-transcript-target-error-bound)
  by (rule
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-residual-and
    [OF wf controlled - residual-bound trace-path-bound
      composition-path-bound])
  show ?thesis
  by (rule
    stark-soundness-from-transcript-indexed-partial-opening-route-from-sampled-transcript-not-prefix
    [OF false-statement wf controlled trace-fri-reduction
      composition-fri-reduction round-bound data-pre-bound
      current-empty-bound])
qed

theorem stark-soundness-from-transcript-indexed-partial-opening-route-from-residual-split-sampled-transcript-not-pr
fixes P empty-query-error :: prob
and LT LC Alpha :: nat  $\Rightarrow$  prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
   $\bigwedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    trace-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
and composition-fri-reduction:
   $\bigwedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    composition-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
and trace-low-degree-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at
        i)
    adversary-initial-state  $\leq$  LT i
and composition-low-degree-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at

```

$i)$
 $\text{adversary-initial-state} \leq LC\ i$
and α -bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A\ i)$
 $(\text{checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq \text{Alpha } i$
and data-pre-bound :
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$
and $\text{current-empty-bound}$:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum i < \text{rounds}.$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(LT\ i + LC\ i + \text{Alpha } i) +$
 $(P + \text{staged-concrete-transcript-target-error-bound}) +$
 $(P + \text{staged-concrete-transcript-target-error-bound})) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\sum i < \text{rounds}.$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(LT\ i + LC\ i + \text{Alpha } i) +$
 $(P + \text{staged-concrete-transcript-target-error-bound}) +$
 $(P + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $((\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$

$(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(P + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\text{trace-fri-error} +$
 $\text{low-degree-trace-pair-agreement-error budgets})) +$
 $\text{trace-fri-error} + \text{empty-query-error})$

proof –

have *residual-bound*:

$\bigwedge i. i < \text{rounds} \implies$

wp-event

$(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(LT \ i + LC \ i + Alpha \ i)$

proof –

fix *i*

assume *i-bound*: $i < \text{rounds}$

have *algebraic-bound*:

wp-event

$(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq LT \ i + LC \ i + Alpha \ i$

by (rule

$\text{checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-bound-from-split}$
 $[OF \ \text{false-statement trace-low-degree-bound}[OF \ i\text{-bound}]$
 $\text{composition-low-degree-bound}[OF \ i\text{-bound}]$
 $\text{alpha-bound}[OF \ i\text{-bound}]])$

show

wp-event

$(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(LT \ i + LC \ i + Alpha \ i)$

by (rule

$\text{checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-bound-from-split}$
 $[OF \ \text{wf controlled } i\text{-bound algebraic-bound}]])$

qed

have *bound*:

$\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$

```

    staged-challenge-query-budget + verifier-hash-query-budget) +
  (∑ i<rounds.
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    query-error-bound +
    (hash-collision-budget-value 0
      (staged-query-search-queries budgets i + 1) +
      (LT i + LC i + Alpha i)) +
    (P + staged-concrete-transcript-target-error-bound) +
    (P + staged-concrete-transcript-target-error-bound)) +
  trace-fri-error + composition-fri-error +
  (∑ i<rounds.
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    query-error-bound +
    (hash-collision-budget-value 0
      (staged-query-search-queries budgets i + 1) +
      (LT i + LC i + Alpha i)) +
    (P + staged-concrete-transcript-target-error-bound) +
    (P + staged-concrete-transcript-target-error-bound)) +
  (composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    ((composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      (P + staged-concrete-transcript-target-error-bound)) +
      (trace-fri-error +
        low-degree-trace-pair-agreement-error budgets)) +
    trace-fri-error + empty-query-error)
  by (rule
    stark-soundness-from-transcript-indexed-partial-opening-route-from-residual-sampled-transcript-not-prefix
    [OF false-statement wf controlled trace-fri-reduction
      composition-fri-reduction residual-bound data-pre-bound
      current-empty-bound])
  show ?thesis
  using bound by (simp add: algebra-simps)
qed

theorem stark-soundness-from-transcript-indexed-partial-opening-route-from-same-run-partial-opening-gap-and-tran
fixes G P empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace

```

and *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *trace-fri-reduction*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $\text{(verifier-state-from-adversary attacker-state}$
 $\text{(staged-proof-transcript data))}$
and *composition-fri-reduction*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $\text{(verifier-state-from-adversary attacker-state}$
 $\text{(staged-proof-transcript data))}$
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $\text{(checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $\text{(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *gap-bound*:
 wp-event
 $\text{(checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 A)
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap}$
 $\text{adversary-initial-state} \leq G$
and *data-pre-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$
and *current-empty-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $\text{(staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $\text{(staged-attacker-query-budget budgets +}$
 $\text{staged-challenge-query-budget + verifier-hash-query-budget) +}$
 $\text{(\sum } i < \text{rounds. } C \ i) +$
 $\text{trace-fri-error + composition-fri-error +}$
 $\text{(\sum } i < \text{rounds. } C \ i) +$
 $\text{(composition-error-bound +}$
 $\text{staged-phase-relation-error size}$
 $\text{(staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$

```

    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    ((composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    (hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) + G +
    (P + staged-concrete-transcript-target-error-bound) +
    (P + staged-concrete-transcript-target-error-bound))) +
    (trace-fri-error +
    low-degree-trace-pair-agreement-error budgets)) +
    trace-fri-error + empty-query-error)
proof -
  have uncovered-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
    adversary-initial-state ≤ G
  by (rule order-trans
    [OF checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-same-run-partial-opening-gap
     gap-bound])
  show ?thesis
  by (rule
    stark-soundness-from-transcript-indexed-partial-opening-route-from-uncovered-and-transcript
    [OF false-statement wf controlled trace-fri-reduction
     composition-fri-reduction round-bound uncovered-bound data-pre-bound
     current-empty-bound])
qed

end

end

```

```

theory Soundness-Public-Route-Single-Query
imports
  Soundness-FRI-Verifier-Tied
  Soundness-Not-Prefix-Partial-Opening
begin

```

Narrow downstream wrappers for the public route that use the transcript-indexed single-current-query charge. This theory depends directly on the partial-opening route needed by the final endpoint.

```

context soundness
begin

```

theorem *stark-soundness-from-transcript-indexed-partial-opening-route-from-sampled-transcript-relevant-drift-only-s*
fixes P *trace-fri-error'* *composition-fri-error'*
empty-trace-fri-error' *empty-query-error* :: *prob*
and $C :: \text{nat} \Rightarrow \text{prob}$
assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq C\ i$
and *data-pre-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq \text{empty-query-error}$
and *trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state $\leq \text{trace-fri-error}'$
and *comp-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state $\leq \text{composition-fri-error}'$
and *empty-trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq \text{empty-trace-fri-error}'$
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(P + staged-concrete-transcript-target-error-bound)) +

$\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum_{i < \text{rounds}} C\ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(P + \text{staged-concrete-transcript-target-error-bound}) +$
 $\text{empty-trace-fri-error}' + \text{empty-query-error})$

proof –

have *current-query-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-current-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq (\sum_{i < \text{rounds}} C\ i)$

by (rule
 $\text{checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds}$
 $[\text{OF round-bound wf controlled}]$)

have *query-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-aligned-transcript-partial-candidates})$
 $\text{adversary-initial-state} \leq (\sum_{i < \text{rounds}} C\ i)$

by (rule *order-trans*[
 $\text{OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit}$
 $\text{current-query-bound}]$)
 $(\text{use wf controlled in simp-all})$

have *transcript-new-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-new-hit}$
 $\text{adversary-initial-state} \leq \text{staged-concrete-transcript-target-error-bound}$

by (rule
 $\text{checked-staged-security-with-data-state-transcript-new-hit-bound}$
 $[\text{OF wf controlled}]$)

have *drift-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift}$
 $\text{adversary-initial-state} \leq$
 $P + \text{staged-concrete-transcript-target-error-bound}$

by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-sampled-trans}$
 $[\text{OF wf controlled data-pre-bound transcript-new-bound}]$)

have *comp-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq$

composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(P + staged-concrete-transcript-target-error-bound)
by (rule
checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-relevant-drift-and-budgets
[OF false-statement wf controlled drift-bound])
have *merkle-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state ≤
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[OF wf controlled])
have *empty-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
adversary-initial-state ≤ empty-query-error
by (rule *order-trans*[
OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero
OF wf controlled] *current-empty-bound*])
have *empty-header-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state ≤
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(P + staged-concrete-transcript-target-error-bound) +
empty-trace-fri-error' + empty-query-error
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-bound-from-relevant-drift-trace-fri-and-query
[OF false-statement wf controlled drift-bound empty-trace-fri-bound
empty-query-bound])
show *?thesis*

by (rule
 checked-staged-soundness-from-verifier-tied-aligned-transcript-partial-candidate-components-and-empty-header-b
 [OF false-statement merkle-bound comp-bound trace-fri-bound
 comp-fri-bound query-bound empty-header-bound])
 qed

end

These single-query-charge wrappers are downstream plumbing for the public same-run route. The visible theorem remains in *Soundness-Public-Route-Same-Run*.

hide-fact
soundness.stark-soundness-from-transcript-indexed-partial-opening-route-from-sampled-transcript-relevant-drift-only
 end

theory *Soundness-FRI-Header-Tied-Reduction*
imports *Soundness-FRI-Verifier-Tied*
begin

Header-tied trace FRI reductions.

The active public route uses trace candidates tied to the verifier header root. This layer mirrors the verifier-tied sampled/missing decomposition without widening back to arbitrary authenticated roots.

context *soundness*
begin

lemma *trace-fri-bad-with-header-tied-partial-candidate-None[simp]*:
 $\neg$ *trace-fri-bad-with-header-tied-partial-candidate s None*
unfolding *trace-fri-bad-with-header-tied-partial-candidate-def*
accepted-fri-opening-transcript-def
by *simp*

definition *trace-fri-bad-with-header-tied-sampled-layer-chain*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-bad-with-header-tied-sampled-layer-chain s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table}$
 doms layers.
 $\text{accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript s fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings s out fr fri-query-idxs}$

$\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate } \text{trace-table } \text{trace-openings} \wedge$
 $\neg \text{trace-table-low-degree } \text{trace-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence } \text{trace-table-low-degree}$
 $\text{trace-table } (\text{clength} - 1) \text{ trace-roots } \text{trace-bs } \text{trace-final}$
 $\text{fri-query-idxs } \text{trace-round-layers } \text{doms layers})$

definition *trace-fri-header-tied-missing-sampled-layer-chain*

$:: (\text{'f}, \text{'a}) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-missing-sampled-layer-chain } s \text{ out} \longleftrightarrow$
 $\text{trace-fri-bad-with-header-tied-partial-candidate } s \text{ out} \wedge$
 $\neg \text{trace-fri-bad-with-header-tied-sampled-layer-chain } s \text{ out}$

definition *trace-fri-header-tied-sampled-layer-assignment-obstruction*

$:: (\text{'f}, \text{'a}) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-sampled-layer-assignment-obstruction } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots } \text{trace-bs } \text{trace-final } dg \text{ composition-roots}$
 $\text{composition-bs } \text{composition-final } \text{fri-query-idxs } \text{trace-round-layers}$
 $\text{composition-round-layers } fr \text{ as rest } \text{trace-openings } \text{trace-table}.$
 $\text{accepted-fri-opening-transcript } s \text{ out } \text{trace-roots } \text{trace-bs}$
 $\text{trace-final } dg \text{ composition-roots } \text{composition-bs } \text{composition-final}$
 $\text{fri-query-idxs } \text{trace-round-layers } \text{composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr } \text{trace-roots } \text{trace-final } \text{as } dg$
 $\text{composition-roots } \text{composition-final } \text{rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out } \text{fr } \text{fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate } \text{trace-table } \text{trace-openings} \wedge$
 $\neg \text{trace-table-low-degree } \text{trace-table} \wedge$
 $\text{generic-fri-sampled-layer-assignment-obstruction}$
 $\text{trace-table-low-degree } \text{trace-table } (\text{clength} - 1) \text{ trace-roots}$
 $\text{trace-bs } \text{trace-final } \text{fri-query-idxs } \text{trace-round-layers})$

definition *trace-fri-header-tied-sampled-assignment-conflict*

$:: (\text{'f}, \text{'a}) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-sampled-assignment-conflict } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots } \text{trace-bs } \text{trace-final } dg \text{ composition-roots}$
 $\text{composition-bs } \text{composition-final } \text{fri-query-idxs } \text{trace-round-layers}$
 $\text{composition-round-layers } fr \text{ as rest } \text{trace-openings } \text{trace-table}.$
 $\text{accepted-fri-opening-transcript } s \text{ out } \text{trace-roots } \text{trace-bs}$
 $\text{trace-final } dg \text{ composition-roots } \text{composition-bs } \text{composition-final}$
 $\text{fri-query-idxs } \text{trace-round-layers } \text{composition-round-layers} \wedge$

verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table $\wedge$*
generic-fri-sampled-assignment-conflict trace-table trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers)

lemma *trace-fri-header-tied-sampled-assignment-conflict-imp-sampled:*

assumes

trace-fri-header-tied-sampled-assignment-conflict s out

shows *trace-fri-sampled-assignment-conflict s out*

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table

where *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers

and *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings

and *cand:* *partial-trace-table-candidate trace-table trace-openings*

and *not-low:* $\neg$ *trace-table-low-degree trace-table*

and *conflict:*

generic-fri-sampled-assignment-conflict trace-table trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers

unfolding *trace-fri-header-tied-sampled-assignment-conflict-def*
by *blast*

have *evidence:*

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr fri-query-idxs trace-openings
trace-table

unfolding *trace-fri-partial-candidate-opening-evidence-def*
trace-fri-partial-candidate-evidence-def

by (*intro conjI fri-openings partial cand not-low*)

show *?thesis*

unfolding *trace-fri-sampled-assignment-conflict-def*

by (*intro exI conjI*)
(rule evidence, rule conflict)

qed

lemma *wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-sampled:*

assumes *sampled-bound:*

wp-event verify-monad (trace-fri-sampled-assignment-conflict s) s $\leq C$

shows
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-sampled-assignment-conflict } s) \ s \leq C$
proof –
have $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-sampled-assignment-conflict } s) \ s \leq$
 $wp\text{-event } verify\text{-monad } (trace\text{-fri-sampled-assignment-conflict } s) \ s$
by $(rule \ wp\text{-event-mono})$
 $(rule \ trace\text{-fri-header-tied-sampled-assignment-conflict-imp-sampled})$
then show $?thesis$
by $(rule \ order\text{-trans}[OF \ - \ sampled\text{-bound}])$
qed

lemma $trace\text{-fri-header-tied-missing-sampled-imp-assignment-obstruction}$:
assumes $missing$:
 $trace\text{-fri-header-tied-missing-sampled-layer-chain } s \ out$
shows
 $trace\text{-fri-header-tied-sampled-layer-assignment-obstruction } s \ out$
proof –
have bad :
 $trace\text{-fri-bad-with-header-tied-partial-candidate } s \ out$
and $not\text{-sampled}$:
 $\neg trace\text{-fri-bad-with-header-tied-sampled-layer-chain } s \ out$
using $missing$
unfolding $trace\text{-fri-header-tied-missing-sampled-layer-chain-def}$
by $blast+$
from bad **obtain** $trace\text{-roots } trace\text{-bs } trace\text{-final } dg \ composition\text{-roots}$
 $composition\text{-bs } composition\text{-final } fri\text{-query-idxs } trace\text{-round-layers}$
 $composition\text{-round-layers } fr \ as \ rest \ trace\text{-openings } trace\text{-table}$
where $fri\text{-openings}$:
 $accepted\text{-fri-opening-transcript } s \ out \ trace\text{-roots } trace\text{-bs}$
 $trace\text{-final } dg \ composition\text{-roots } composition\text{-bs } composition\text{-final}$
 $fri\text{-query-idxs } trace\text{-round-layers } composition\text{-round-layers}$
and $header$:
 $verifier\text{-header-transcript } s \ fr \ trace\text{-roots } trace\text{-final } as \ dg$
 $composition\text{-roots } composition\text{-final } rest$
and $partial$:
 $accepted\text{-with-partial-trace-openings } s \ out \ fr \ fri\text{-query-idxs}$
 $trace\text{-openings}$
and $cand$: $partial\text{-trace-table-candidate } trace\text{-table } trace\text{-openings}$
and $not\text{-low}$: $\neg trace\text{-table-low-degree } trace\text{-table}$
unfolding $trace\text{-fri-bad-with-header-tied-partial-candidate-def}$
by $blast$
have $partial\text{-generic}$:
 $generic\text{-fri-partial-evidence } trace\text{-table-low-degree } trace\text{-table}$
 $(clength - 1) \ trace\text{-roots } trace\text{-bs } trace\text{-final } fri\text{-query-idxs}$
 $trace\text{-round-layers}$
by $(rule \ accepted\text{-fri-opening-transcript-trace-generic-partial-evidence}$
 $[OF \ fri\text{-openings}])$

```

have no-chain:
   $\wedge$  doms layers.  $\neg$ 
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers doms layers
proof
  fix doms layers
  assume chain:
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers doms layers
  have trace-fri-bad-with-header-tied-sampled-layer-chain s out
    unfolding trace-fri-bad-with-header-tied-sampled-layer-chain-def
    by (intro exI conjI)
      (rule fri-openings, rule header, rule partial, rule cand,
       rule not-low, rule chain)
  then show False
    using not-sampled by contradiction
qed
have obstruction:
  generic-fri-sampled-layer-assignment-obstruction
    trace-table-low-degree trace-table (clength - 1) trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers
  unfolding generic-fri-sampled-layer-assignment-obstruction-def
  using partial-generic no-chain by blast
show ?thesis
  unfolding trace-fri-header-tied-sampled-layer-assignment-obstruction-def
  by (intro exI conjI)
    (rule fri-openings, rule header, rule partial, rule cand,
     rule not-low, rule obstruction)
qed

lemma trace-fri-header-tied-assignment-obstruction-imp-conflict-or-zero:
  assumes obstruction:
    trace-fri-header-tied-sampled-layer-assignment-obstruction s out
  shows
    trace-fri-header-tied-sampled-assignment-conflict s out  $\vee$ 
    trace-fri-header-tied-zero-round-final-obstruction s out
proof -
  obtain trace-roots trace-bs' trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as rest trace-openings trace-table
  where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs'
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  and header:
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest

```

and *partial*:
 accepted-with-partial-trace-openings *s* out fr *fri-query-idxs*
 trace-openings
and *cand*: *partial-trace-table-candidate* *trace-table* *trace-openings*
and *not-low*: $\neg$ *trace-table-low-degree* *trace-table*
and *generic*:
 generic-fri-sampled-layer-assignment-obstruction
 trace-table-low-degree *trace-table* (*clength* - 1) *trace-roots*
 trace-bs' *trace-final* *fri-query-idxs* *trace-round-layers*
using *obstruction*
unfolding *trace-fri-header-tied-sampled-layer-assignment-obstruction-def*
by *blast*
obtain *N* **where** *eval-power*: *clength* * *scale* = 2^N
 using *eval-domain-length-power* **by** *blast*
have *trace-rounds-le*: *length* *trace-bs'* $\leq N$
proof -
 have *len*: *length* *trace-bs'* = *ceil-log* *clength*
 using *accepted-fri-opening-transcript-shapes*(1,2)[*OF* *fri-openings*]
 by *simp*
 have *clength* $\leq$ *clength* * *scale*
 using *scale-pos* **by** *simp*
 also have ... = 2^N
 by (rule *eval-power*)
 finally have *ceil-log* *clength* $\leq N$
 by (rule *ceil-log-le-power*)
 then show ?thesis
 using *len* **by** *simp*
qed
have *query-bounds'*:
 $\bigwedge \text{round-idx. round-idx} < \text{length } \text{fri-query-idxs} \implies$
 $\text{fri-query-idxs} ! \text{round-idx} < \text{clength} * \text{scale}$
proof -
 fix *round-idx*
 assume *round-bound*: *round-idx* < *length* *fri-query-idxs*
 have *fri-query-idxs* ! *round-idx* $\in$ *set* *fri-query-idxs*
 using *round-bound* **by** *simp*
 then show *fri-query-idxs* ! *round-idx* < *clength* * *scale*
 by (rule *accepted-fri-opening-transcript-query-idx-bound*
 [*OF* *fri-openings*])
qed
have *split*:
 generic-fri-sampled-assignment-conflict *trace-table* *trace-roots*
 trace-bs' *trace-final* *fri-query-idxs* *trace-round-layers* $\vee$
 length *trace-bs'* = 0 $\wedge$
 $\neg$ *fri-final-constant-consistent* *trace-table* *trace-final*
 by (rule
 generic-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final
 [*OF* *generic* *query-bounds'* *eval-power* *trace-rounds-le*])
then show ?thesis

proof
assume *conflict*:
generic-fri-sampled-assignment-conflict trace-table trace-roots
trace-bs' trace-final fri-query-idxs trace-round-layers
then show *?thesis*
unfolding *trace-fri-header-tied-sampled-assignment-conflict-def*
by (*intro disjI1 exI conjI*)
(rule fri-openings, rule header, rule partial, rule cand,
rule not-low, rule conflict)
next
assume *zero*:
length trace-bs' = 0 ∧
 $\neg$ *fri-final-constant-consistent trace-table trace-final*
then show *?thesis*
unfolding *trace-fri-header-tied-zero-round-final-obstruction-def*
by (*intro disjI2 exI conjI*)
(rule fri-openings, rule header, rule partial, rule cand,
rule not-low,
simp-all)
qed
qed

lemma *wp-trace-fri-header-tied-missing-sampled-bound-from-assignment-obstruction*:
assumes *assignment-bound*:
wp-event verify-monad
(trace-fri-header-tied-sampled-layer-assignment-obstruction s) s
 $\leq M$
shows
wp-event verify-monad
(trace-fri-header-tied-missing-sampled-layer-chain s) s ≤ M

proof –
have *wp-event verify-monad*
(trace-fri-header-tied-missing-sampled-layer-chain s) s ≤
wp-event verify-monad
(trace-fri-header-tied-sampled-layer-assignment-obstruction s) s
by (*rule wp-event-mono*)
(rule trace-fri-header-tied-missing-sampled-imp-assignment-obstruction)
then show *?thesis*
by (*rule order-trans[OF - assignment-bound]*)
qed

lemma *wp-trace-fri-header-tied-bound-from-sampled-and-missing*:
assumes *sampled-bound*:
wp-event verify-monad
(trace-fri-bad-with-header-tied-sampled-layer-chain s) s ≤ R
and *missing-bound*:
wp-event verify-monad
(trace-fri-header-tied-missing-sampled-layer-chain s) s ≤ M
shows

$wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-bad-with-header-tied-partial-candidate } s) \ s \leq R + M$
proof –
have $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-bad-with-header-tied-partial-candidate } s) \ s \leq$
 $wp\text{-event } verify\text{-monad}$
 $(\lambda out. trace\text{-fri-bad-with-header-tied-sampled-layer-chain } s \ out \ \vee$
 $trace\text{-fri-header-tied-missing-sampled-layer-chain } s \ out) \ s$
unfolding $trace\text{-fri-header-tied-missing-sampled-layer-chain-def}$
by $(rule \ wp\text{-event-mono}) \ blast$
also have $\dots \leq$
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-bad-with-header-tied-sampled-layer-chain } s) \ s +$
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-missing-sampled-layer-chain } s) \ s$
by $(rule \ wp\text{-event-union-bound})$
also have $\dots \leq R + M$
by $(rule \ add\text{-mono}[OF \ sampled\text{-bound} \ missing\text{-bound}])$
finally show $?thesis$.
qed

lemma $wp\text{-trace-fri-header-tied-assignment-obstruction-bound-from-conflict-and-zero}$:
assumes $conflict\text{-bound}$:
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-sampled-assignment-conflict } s) \ s \leq C$
and $zero\text{-bound}$:
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-zero-round-final-obstruction } s) \ s \leq Z$
shows
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-sampled-layer-assignment-obstruction } s) \ s$
 $\leq C + Z$
proof –
have $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-sampled-layer-assignment-obstruction } s) \ s$
 $\leq wp\text{-event } verify\text{-monad}$
 $(\lambda out. trace\text{-fri-header-tied-sampled-assignment-conflict } s \ out \ \vee$
 $trace\text{-fri-header-tied-zero-round-final-obstruction } s \ out) \ s$
by $(rule \ wp\text{-event-mono})$
 $(rule \ trace\text{-fri-header-tied-assignment-obstruction-imp-conflict-or-zero})$
also have $\dots \leq$
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-sampled-assignment-conflict } s) \ s +$
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-zero-round-final-obstruction } s) \ s$
by $(rule \ wp\text{-event-union-bound})$
also have $\dots \leq C + Z$
by $(rule \ add\text{-mono}[OF \ conflict\text{-bound} \ zero\text{-bound}])$
finally show $?thesis$.

qed

lemma *wp-trace-fri-header-tied-bound-from-sampled-conflict-and-zero:*

assumes *sampled-bound:*

wp-event verify-monad

(trace-fri-bad-with-header-tied-sampled-layer-chain s) s ≤ R

and *conflict-bound:*

wp-event verify-monad

(trace-fri-header-tied-sampled-assignment-conflict s) s ≤ C

and *zero-bound:*

wp-event verify-monad

(trace-fri-header-tied-zero-round-final-obstruction s) s ≤ Z

shows

wp-event verify-monad

(trace-fri-bad-with-header-tied-partial-candidate s) s

≤ R + (C + Z)

by (*rule wp-trace-fri-header-tied-bound-from-sampled-and-missing*

[OF sampled-bound

wp-trace-fri-header-tied-missing-sampled-bound-from-assignment-obstruction

[OF wp-trace-fri-header-tied-assignment-obstruction-bound-from-conflict-and-zero

[OF conflict-bound zero-bound]]])

lemma *trace-fri-bad-with-header-tied-partial-candidates-staged-bound:*

assumes *verifier-bound:*

∧ data attacker-state.

Some (data, attacker-state) ∈

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) ⇒

wp-event verify-monad

(trace-fri-bad-with-header-tied-partial-candidate

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)) ≤ C

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

trace-fri-bad-with-header-tied-partial-candidate)

adversary-initial-state ≤ C

by (*rule checked-staged-security-with-data-state-verifier-event-bound-from-cont*)

(simp-all add: verifier-bound)

lemma *wp-trace-fri-header-tied-zero-round-bound-from-checked-and-merkle:*

assumes *checked-bound:*

wp-event verify-monad

(trace-fri-zero-round-checked-final-obstruction s) s ≤ Z

and *merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M

shows

```

    wp-event verify-monad
      (trace-fri-header-tied-zero-round-final-obstruction s) s ≤ Z + M
proof –
  have wp-event verify-monad
    (trace-fri-header-tied-zero-round-final-obstruction s) s ≤
      wp-event verify-monad
        (λout. trace-fri-zero-round-checked-final-obstruction s out ∨
          partial-merkle-inconsistency-bad s out) s
  by (rule wp-event-mono)
    (rule trace-fri-header-tied-zero-round-final-obstruction-imp-checked-or-merkle)
  also have ... ≤
    wp-event verify-monad
      (trace-fri-zero-round-checked-final-obstruction s) s +
      wp-event verify-monad (partial-merkle-inconsistency-bad s) s
  by (rule wp-event-union-bound)
  also have ... ≤ Z + M
  by (rule add-mono[OF checked-bound merkle-bound])
  finally show ?thesis .
qed

end

end

```

```

theory Soundness-FRI-Layer-Merkle
imports Soundness-FRI-Header-Tied-Reduction
begin

```

Authenticated-opening facts for sampled FRI layer chunks.

The sampled-assignment conflict predicates are stated over transcript chunks. This layer connects those chunks back to the Merkle checks performed by the verifier. It is intentionally local: it extracts only sampled openings from the verifier execution and does not reconstruct complete FRI tables.

```

context soundness
begin

```

definition *fri-layer-chunk-authenticated*

```

:: 'f ⇒ nat ⇒ nat ⇒ 'f list ⇒
  ('f, 'a) protocol-channel-scheme ⇒ bool

```

where

```

fri-layer-chunk-authenticated rt len idx chunk s ⟷
  (∃ xp xp-path xn xn-path.
    fri-layer-opening-chunk len xp xp-path xn xn-path chunk ∧
    authenticated-opening-in s
    (opening-root = rt,
     opening-length = len,
     opening-index = idx,
     opening-value = xp,

```

$\text{opening-path} = \text{xp-path}) \wedge$
 $\text{authenticated-opening-in } s$
 $(\text{opening-root} = \text{rt},$
 $\text{opening-length} = \text{len},$
 $\text{opening-index} = \text{fri-sibling-index len idx},$
 $\text{opening-value} = \text{xn},$
 $\text{opening-path} = \text{xn-path}))$

definition *fri-mfold-chunks-authenticated*

$:: ('f \times 'f) \text{ list} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow 'f \text{ list list} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{fri-mfold-chunks-authenticated bfs len idx chunks } s \longleftrightarrow$
 $\text{length chunks} = \text{length bfs} \wedge$
 $(\forall j < \text{length bfs}.$
 $\text{fri-layer-chunk-authenticated (snd (bfs ! j))}$
 $(\text{fri-layer-lengths (length bfs) len ! j})$
 $(\text{fri-layer-indices (length bfs) idx len ! j})$
 $(\text{chunks ! j}) s)$

lemma *fri-layers-transcript-nth-length:*

assumes *layers: fri-layers-transcript n len layer-chunks chunk*
and *j-bound: j < n*

shows

$\text{length (layer-chunks ! j)} =$
 $2 + 2 * \text{floor-log (fri-layer-lengths n len ! j)}$

proof –

from *fri-layers-transcript-nth-opening[OF layers j-bound]*

obtain *xp xp-path xn xn-path* **where**

layer-open:

$\text{fri-layer-opening-chunk (fri-layer-lengths n len ! j)}$
 $\text{xp xp-path xn xn-path (layer-chunks ! j)}$

by *blast*

show *?thesis*

using *fri-layer-opening-chunk-length[OF layer-open]* .

qed

lemma *fri-layers-transcript-unique:*

assumes *left: fri-layers-transcript n len layer-chunks chunk*

and *right: fri-layers-transcript n len layer-chunks' chunk*

shows *layer-chunks = layer-chunks'*

proof –

have *len-left: length layer-chunks = n*

using *left unfolding fri-layers-transcript-def* **by** *simp*

have *len-right: length layer-chunks' = n*

using *right unfolding fri-layers-transcript-def* **by** *simp*

have *concat-eq:*

$\text{List.concat layer-chunks @ []} = \text{List.concat layer-chunks'}$

using *left right unfolding fri-layers-transcript-def* **by** *simp*

```

have same-lens:
   $\bigwedge j. j < \text{length } \text{layer-chunks} \implies$ 
     $\text{length } (\text{layer-chunks} ! j) = \text{length } (\text{layer-chunks}' ! j)$ 
  using fri-layers-transcript-nth-length[OF left]
    fri-layers-transcript-nth-length[OF right]
    len-left len-right
  by simp
have layer-chunks = layer-chunks'  $\wedge$  ( $[] :: 'f \text{ list}$ ) = []
  by (rule concat-append-eq-concat-same-chunk-lengths
    [OF - same-lens concat-eq])
    (use len-left len-right in simp)
then show ?thesis
  by simp
qed

lemma query-round-fri-layer-transcripts-unique:
  assumes left:
    query-round-fri-layer-transcripts idx trace-roots composition-roots
    chunk trace-layers composition-layers
  and right:
    query-round-fri-layer-transcripts idx trace-roots composition-roots
    chunk trace-layers' composition-layers'
  shows trace-layers = trace-layers'
    and composition-layers = composition-layers'
proof -
  from left obtain query-chunk trace-fri-chunk composition-fri-chunk
    leaves paths where
    chunk-left:
      chunk = query-chunk @ trace-fri-chunk @ composition-fri-chunk
  and query-left:
    query-decommitment-transcript idx leaves paths query-chunk
  and trace-left:
    fri-layers-transcript (length trace-roots) (clength * scale)
    trace-layers trace-fri-chunk
  and composition-left:
    fri-layers-transcript (length composition-roots) (clength * scale)
    composition-layers composition-fri-chunk
  unfolding query-round-fri-layer-transcripts-def by blast
  from right obtain query-chunk' trace-fri-chunk' composition-fri-chunk'
    leaves' paths' where
    chunk-right:
      chunk = query-chunk' @ trace-fri-chunk' @ composition-fri-chunk'
  and query-right:
    query-decommitment-transcript idx leaves' paths' query-chunk'
  and trace-right:
    fri-layers-transcript (length trace-roots) (clength * scale)
    trace-layers' trace-fri-chunk'
  and composition-right:
    fri-layers-transcript (length composition-roots) (clength * scale)

```

```

      composition-layers' composition-fri-chunk'
unfolding query-round-fri-layer-transcripts-def by blast
have len-query:
  length query-chunk = length query-chunk'
using query-decommitment-transcript-length[OF query-left]
  query-decommitment-transcript-length[OF query-right]
by simp
have len-trace:
  length trace-fri-chunk = length trace-fri-chunk'
using fri-layers-transcript-length[OF trace-left]
  fri-layers-transcript-length[OF trace-right]
by simp
have trace-chunk-eq: trace-fri-chunk = trace-fri-chunk'
proof -
  have left-drop:
    drop (length query-chunk) chunk =
      trace-fri-chunk @ composition-fri-chunk
    using chunk-left by simp
  have right-drop:
    drop (length query-chunk) chunk =
      trace-fri-chunk' @ composition-fri-chunk'
    using chunk-right len-query by simp
  have left-take:
    trace-fri-chunk =
      take (length trace-fri-chunk) (drop (length query-chunk) chunk)
    using left-drop by simp
  have right-take:
    trace-fri-chunk' =
      take (length trace-fri-chunk') (drop (length query-chunk) chunk)
    using right-drop by simp
  show ?thesis
    using left-take right-take len-trace by simp
qed
have composition-chunk-eq:
  composition-fri-chunk = composition-fri-chunk'
proof -
  have left-drop:
    drop (length query-chunk + length trace-fri-chunk) chunk =
      composition-fri-chunk
    using chunk-left by simp
  have right-drop:
    drop (length query-chunk + length trace-fri-chunk) chunk =
      composition-fri-chunk'
    using chunk-right len-query len-trace by simp
  show ?thesis
    using left-drop right-drop by simp
qed
have trace-right':
  fri-layers-transcript (length trace-roots) (clength * scale)

```

```

    trace-layers' trace-fri-chunk
  using trace-right trace-chunk-eq by simp
have composition-right':
  fri-layers-transcript (length composition-roots) (clength * scale)
    composition-layers' composition-fri-chunk
  using composition-right composition-chunk-eq by simp
show trace-layers = trace-layers'
  by (rule fri-layers-transcript-unique[OF trace-left trace-right'])
show composition-layers = composition-layers'
  by (rule fri-layers-transcript-unique
    [OF composition-left composition-right'])
qed

```

lemma *fri-layer-chunk-authenticatedI*:

assumes *fri-layer-opening-chunk len xp xp-path xn xn-path chunk*
and *authenticated-opening-in s*
 (*opening-root = rt,*
opening-length = len,
opening-index = idx,
opening-value = xp,
opening-path = xp-path)
and *authenticated-opening-in s*
 (*opening-root = rt,*
opening-length = len,
opening-index = fri-sibling-index len idx,
opening-value = xn,
opening-path = xn-path)
shows *fri-layer-chunk-authenticated rt len idx chunk s*
using *assms unfolding fri-layer-chunk-authenticated-def* **by** *blast*

lemma *fri-layer-chunk-authenticatedE*:

assumes *fri-layer-chunk-authenticated rt len idx chunk s*
obtains *xp xp-path xn xn-path* **where**
fri-layer-opening-chunk len xp xp-path xn xn-path chunk
authenticated-opening-in s
 (*opening-root = rt,*
opening-length = len,
opening-index = idx,
opening-value = xp,
opening-path = xp-path)
authenticated-opening-in s
 (*opening-root = rt,*
opening-length = len,
opening-index = fri-sibling-index len idx,
opening-value = xn,
opening-path = xn-path)
using *assms unfolding fri-layer-chunk-authenticated-def* **by** *blast*

lemma *fri-layer-chunk-authenticated-matching-values*:

assumes *auth: fri-layer-chunk-authenticated rt len idx chunk s*
and *chunk:*
fri-layer-opening-chunk len yp yp-path yn yn-path chunk
obtains *xp-path xn-path* **where**
fri-layer-opening-chunk len yp xp-path yn xn-path chunk
authenticated-opening-in s
(*opening-root = rt,*
opening-length = len,
opening-index = idx,
opening-value = yp,
opening-path = xp-path)
authenticated-opening-in s
(*opening-root = rt,*
opening-length = len,
opening-index = fri-sibling-index len idx,
opening-value = yn,
opening-path = xn-path)
proof –
from *auth* **obtain** *xp xp-path xn xn-path* **where**
chunk': fri-layer-opening-chunk len xp xp-path xn xn-path chunk
and *xp-auth:*
authenticated-opening-in s
(*opening-root = rt,*
opening-length = len,
opening-index = idx,
opening-value = xp,
opening-path = xp-path)
and *xn-auth:*
authenticated-opening-in s
(*opening-root = rt,*
opening-length = len,
opening-index = fri-sibling-index len idx,
opening-value = xn,
opening-path = xn-path)
by (*rule fri-layer-chunk-authenticatedE*)
have *xp-eq: xp = yp*
by (*rule fri-layer-opening-chunk-values-unique(1)[OF chunk' chunk]*)
have *xn-eq: xn = yn*
by (*rule fri-layer-opening-chunk-values-unique(2)[OF chunk' chunk]*)
have *chunk-y:*
fri-layer-opening-chunk len yp xp-path yn xn-path chunk
using *chunk' xp-eq xn-eq* **by** *simp*
show *?thesis*
by (*rule that[OF chunk-y]*)
(use xp-auth xn-auth xp-eq xn-eq in simp-all)
qed

lemma *authenticated-openings-values-eq-if-no-partial-merkle:*
assumes *no-bad:*


```

    ¬ partial-merkle-inconsistency-bad s
      (Some (result, final-state))
  and first: authenticated-opening-in final-state opn
  and second: authenticated-opening-in final-state opn'
  and same-root: opening-root opn = opening-root opn'
  and same-length: opening-length opn = opening-length opn'
  and same-index: opening-index opn = opening-index opn'
  shows opening-value opn = opening-value opn'
proof (rule ccontr)
  assume neg: opening-value opn ≠ opening-value opn'
  have partial-merkle-inconsistency-bad s
    (Some (result, final-state))
  unfolding partial-merkle-inconsistency-bad-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state]
      exI[of - opn] exI[of - opn'])
    (use first second same-root same-length same-index neg in simp-all)
  then show False using no-bad by contradiction
qed

```

lemma *fri-layer-authenticated-base-values-eq-if-no-partial-merkle:*

```

  assumes no-bad:
    ¬ partial-merkle-inconsistency-bad s
      (Some (result, final-state))
  and auth:
    fri-layer-chunk-authenticated rt len idx chunk final-state
  and auth':
    fri-layer-chunk-authenticated rt len idx chunk' final-state
  and chunk:
    fri-layer-opening-chunk len xp xp-path xn xn-path chunk
  and chunk':
    fri-layer-opening-chunk len yp yp-path yn yn-path chunk'
  shows xp = yp
proof (rule ccontr)
  assume neg: xp ≠ yp
  from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
  obtain xp-path' xn-path' where
    xp-auth:
      authenticated-opening-in final-state
      (|opening-root = rt,
        opening-length = len,
        opening-index = idx,
        opening-value = xp,
        opening-path = xp-path'|)
  by blast
  from fri-layer-chunk-authenticated-matching-values[OF auth' chunk']
  obtain yp-path' yn-path' where
    yp-auth:
      authenticated-opening-in final-state
      (|opening-root = rt,

```

```

    opening-length = len,
    opening-index = idx,
    opening-value = yp,
    opening-path = yp-path')
  by blast
have partial-merkle-inconsistency-bad s
  (Some (result, final-state))
unfolding partial-merkle-inconsistency-bad-def accepted-def
by (intro conjI exI[of - result] exI[of - final-state]
    exI[of - (opening-root = rt,
    opening-length = len,
    opening-index = idx,
    opening-value = xp,
    opening-path = xp-path')])
    exI[of - (opening-root = rt,
    opening-length = len,
    opening-index = idx,
    opening-value = yp,
    opening-path = yp-path')])
  (use xp-auth yp-auth neg in simp-all)
then show False using no-bad by contradiction
qed

lemma fri-layer-authenticated-sibling-values-eq-if-no-partial-merkle:
  assumes no-bad:
    ¬ partial-merkle-inconsistency-bad s
    (Some (result, final-state))
  and auth:
    fri-layer-chunk-authenticated rt len idx chunk final-state
  and auth':
    fri-layer-chunk-authenticated rt len idx chunk' final-state
  and chunk:
    fri-layer-opening-chunk len xp xp-path xn xn-path chunk
  and chunk':
    fri-layer-opening-chunk len yp yp-path yn yn-path chunk'
  shows xn = yn
proof (rule ccontr)
  assume neg: xn ≠ yn
  from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
  obtain xp-path' xn-path' where
    xn-auth:
      authenticated-opening-in final-state
      (opening-root = rt,
      opening-length = len,
      opening-index = fri-sibling-index len idx,
      opening-value = xn,
      opening-path = xn-path')
  by blast
  from fri-layer-chunk-authenticated-matching-values[OF auth' chunk']

```

```

obtain yp-path' yn-path' where
  yn-auth:
    authenticated-opening-in final-state
    ( $\ell$  opening-root = rt,
     opening-length = len,
     opening-index = fri-sibling-index len idx,
     opening-value = yn,
     opening-path = yn-path')
  by blast
have partial-merkle-inconsistency-bad s
  (Some (result, final-state))
unfolding partial-merkle-inconsistency-bad-def accepted-def
by (intro conjI exI[of - result] exI[of - final-state]
     exI[of - ( $\ell$  opening-root = rt,
       opening-length = len,
       opening-index = fri-sibling-index len idx,
       opening-value = xn,
       opening-path = xn-path')])
     exI[of - ( $\ell$  opening-root = rt,
       opening-length = len,
       opening-index = fri-sibling-index len idx,
       opening-value = yn,
       opening-path = yn-path')]])
  (use xn-auth yn-auth neq in simp-all)
then show False using no-bad by contradiction
qed

lemma fri-layer-authenticated-base-sibling-values-eq-if-no-partial-merkle:
assumes no-bad:
   $\neg$  partial-merkle-inconsistency-bad s
  (Some (result, final-state))
and auth:
  fri-layer-chunk-authenticated rt len idx chunk final-state
and auth':
  fri-layer-chunk-authenticated rt len idx' chunk' final-state
and chunk:
  fri-layer-opening-chunk len xp xp-path xn xn-path chunk
and chunk':
  fri-layer-opening-chunk len yp yp-path yn yn-path chunk'
and same-index: idx = fri-sibling-index len idx'
shows xp = yn
proof –
from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
obtain xp-path' xn-path' where
  xp-auth:
    authenticated-opening-in final-state
    ( $\ell$  opening-root = rt,
     opening-length = len,
     opening-index = idx,

```

```

      opening-value = xp,
      opening-path = xp-path')
  by blast
from fri-layer-chunk-authenticated-matching-values[OF auth' chunk']
obtain yp-path' yn-path' where
  yn-auth:
    authenticated-opening-in final-state
    (|opening-root = rt,
      opening-length = len,
      opening-index = fri-sibling-index len idx',
      opening-value = yn,
      opening-path = yn-path')
  by blast
show ?thesis
using authenticated-openings-values-eq-if-no-partial-merkle
  [OF no-bad xp-auth yn-auth] same-index
by simp
qed

```

lemma fri-layer-authenticated-sibling-base-values-eq-if-no-partial-merkle:

```

assumes no-bad:
  ¬ partial-merkle-inconsistency-bad s
  (Some (result, final-state))
and auth:
  fri-layer-chunk-authenticated rt len idx chunk final-state
and auth':
  fri-layer-chunk-authenticated rt len idx' chunk' final-state
and chunk:
  fri-layer-opening-chunk len xp xp-path xn xn-path chunk
and chunk':
  fri-layer-opening-chunk len yp yp-path yn yn-path chunk'
and same-index: fri-sibling-index len idx = idx'
shows xn = yp

```

proof –

```

from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
obtain xp-path' xn-path' where
  xn-auth:
    authenticated-opening-in final-state
    (|opening-root = rt,
      opening-length = len,
      opening-index = fri-sibling-index len idx,
      opening-value = xn,
      opening-path = xn-path')
  by blast
from fri-layer-chunk-authenticated-matching-values[OF auth' chunk']
obtain yp-path' yn-path' where
  yp-auth:
    authenticated-opening-in final-state
    (|opening-root = rt,

```

```

    opening-length = len,
    opening-index = idx',
    opening-value = yp,
    opening-path = yp-path')
  by blast
show ?thesis
  using authenticated-openings-values-eq-if-no-partial-merkle
    [OF no-bad xn-auth yp-auth] same-index
  by simp
qed

lemma fri-layer-authenticated-sibling-sibling-values-eq-if-no-partial-merkle:
  assumes no-bad:
    ¬ partial-merkle-inconsistency-bad s
    (Some (result, final-state))
  and auth:
    fri-layer-chunk-authenticated rt len idx chunk final-state
  and auth':
    fri-layer-chunk-authenticated rt len idx' chunk' final-state
  and chunk:
    fri-layer-opening-chunk len xp xp-path xn xn-path chunk
  and chunk':
    fri-layer-opening-chunk len yp yp-path yn yn-path chunk'
  and same-index:
    fri-sibling-index len idx = fri-sibling-index len idx'
  shows xn = yn
proof -
  from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
  obtain xp-path' xn-path' where
    xn-auth:
      authenticated-opening-in final-state
      (|opening-root = rt,
        opening-length = len,
        opening-index = fri-sibling-index len idx,
        opening-value = xn,
        opening-path = xn-path'|)
  by blast
  from fri-layer-chunk-authenticated-matching-values[OF auth' chunk']
  obtain yp-path' yn-path' where
    yn-auth:
      authenticated-opening-in final-state
      (|opening-root = rt,
        opening-length = len,
        opening-index = fri-sibling-index len idx',
        opening-value = yn,
        opening-path = yn-path'|)
  by blast
show ?thesis
  using authenticated-openings-values-eq-if-no-partial-merkle

```

$[OF \text{ no-bad } xn\text{-auth } yn\text{-auth}] \text{ same-index}$
 by *simp*
 qed

lemma *fri-layer-chunk-authenticated-mono*:
 assumes *auth*: *fri-layer-chunk-authenticated* *rt len idx chunk s*
 and *ext*: $s \leq t$
 shows *fri-layer-chunk-authenticated* *rt len idx chunk t*
 using *auth*
 unfolding *fri-layer-chunk-authenticated-def*
 by (*meson authenticated-opening-in-mono ext*)

lemma *fri-mfold-chunks-authenticated-mono*:
 assumes *auth*: *fri-mfold-chunks-authenticated* *bfs len idx chunks s*
 and *ext*: $s \leq t$
 shows *fri-mfold-chunks-authenticated* *bfs len idx chunks t*
 using *auth*
 unfolding *fri-mfold-chunks-authenticated-def*
 by (*meson fri-layer-chunk-authenticated-mono ext*)

lemma *fri-layer-opening-step-authenticated-chunk*:
 fixes $s \ t :: ('f, 'a) \text{ protocol-channel-scheme}$
 assumes *len-pos*: $0 < \text{len}$
 and *idx-bound*: $\text{idx} < \text{len}$
 and *outcome*:
 Some (*out*, *t*) $\in$
 set-dist
 (*execute* (*fri-layer-opening-step* (*b*, *rt*) (*idx*, *x*, *len*, *pw*)) *s*)
 obtains *xp xp-path xn xn-path* **where**
 fri-layer-opening-chunk *len xp xp-path xn xn-path*
 (*[xp]* @ *xp-path* @ *[xn]* @ *xn-path*)
 authenticated-opening-in t
 ($\text{opening-root} = \text{rt},$
 $\text{opening-length} = \text{len},$
 $\text{opening-index} = \text{idx},$
 $\text{opening-value} = \text{xp},$
 $\text{opening-path} = \text{xp-path}$)
 authenticated-opening-in t
 ($\text{opening-root} = \text{rt},$
 $\text{opening-length} = \text{len},$
 $\text{opening-index} = \text{fri-sibling-index len idx},$
 $\text{opening-value} = \text{xn},$
 $\text{opening-path} = \text{xn-path}$)

proof –
 from *outcome*[*unfolded fri-layer-opening-step-unfold-finish*]
 obtain *xp s1 xp-path s2 xn s3 xn-path s4* **where**
 read-xp: *Some* (*xp*, *s1*) $\in$ *set-dist* (*execute read s*)
 and *read-xp-path*:
 Some (*xp-path*, *s2*) $\in$

```

    set-dist (execute (ntimes read (floor-log len)) s1)
and read-xn: Some (xn, s3) ∈ set-dist (execute read s2)
and read-xn-path:
  Some (xn-path, s4) ∈
    set-dist (execute (ntimes read (floor-log len)) s3)
and finish:
  Some (out, t) ∈
    set-dist
      (execute
        (fri-layer-opening-finish b rt idx x len pw
          xp xp-path xn xn-path) s4)
by (auto elim!: set-dist-bindE)
have xp-path-length: length xp-path = floor-log len
  using ntimes-read-any-outcome[OF read-xp-path] by simp
have xn-path-length: length xn-path = floor-log len
  using ntimes-read-any-outcome[OF read-xn-path] by simp
have sibling-bound: (idx + len div 2) mod len < len
  using len-pos by simp
from fri-layer-opening-finish-authenticated-openings
  [OF idx-bound sibling-bound xp-path-length xn-path-length finish]
have xp-auth:
  authenticated-opening-in t
    (|opening-root = rt,
      opening-length = len,
      opening-index = idx,
      opening-value = xp,
      opening-path = xp-path)
and xn-auth:
  authenticated-opening-in t
    (|opening-root = rt,
      opening-length = len,
      opening-index = fri-sibling-index len idx,
      opening-value = xn,
      opening-path = xn-path)
unfolding fri-sibling-index-def
by blast+
have chunk:
  fri-layer-opening-chunk len xp xp-path xn xn-path
    ([xp] @ xp-path @ [xn] @ xn-path)
  using xp-path-length xn-path-length
  unfolding fri-layer-opening-chunk-def by simp
show ?thesis
  by (rule that[OF chunk xp-auth xn-auth])
qed

```

```

lemma fri-layer-opening-step-authenticated-chunk-pred:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes len-pos: 0 < len
and idx-bound: idx < len

```

and *outcome*:
Some (out, t) ∈
set-dist
(execute (fri-layer-opening-step (b, rt) (idx, x, len, pw)) s)
obtains *chunk* **where** *fri-layer-chunk-authenticated rt len idx chunk t*
proof –
from *fri-layer-opening-step-authenticated-chunk*
[OF len-pos idx-bound outcome]
obtain *xp xp-path xn xn-path* **where**
chunk:
fri-layer-opening-chunk len xp xp-path xn xn-path
([xp] @ xp-path @ [xn] @ xn-path)
and *xp-auth:*
authenticated-opening-in t
(|opening-root = rt,
opening-length = len,
opening-index = idx,
opening-value = xp,
opening-path = xp-path)
and *xn-auth:*
authenticated-opening-in t
(|opening-root = rt,
opening-length = len,
opening-index = fri-sibling-index len idx,
opening-value = xn,
opening-path = xn-path)
by *blast*
have *fri-layer-chunk-authenticated rt len idx*
([xp] @ xp-path @ [xn] @ xn-path) t
by *(rule fri-layer-chunk-authenticatedI[OF chunk xp-auth xn-auth])*
then show *?thesis*
by *(rule that)*
qed

lemma *fri-layer-opening-step-outcome-authenticated:*
fixes *s t :: ('f, 'a) protocol-channel-scheme*
assumes *len-pos: 0 < len*
and *idx-bound: idx < len*
and *outcome:*
Some (out, t) ∈
set-dist
(execute (fri-layer-opening-step (b, rt) (idx, x, len, pw)) s)
obtains *xp xp-path xn xn-path x' chunk* **where**
out = (idx mod (len div 2), x', len div 2, pw + pw)
xp = x
x' = fri-fold-value b xp xn
*(fri-fold-denominator ((h ^ idx) * shift) pw)*
chunk = [xp] @ xp-path @ [xn] @ xn-path
fri-layer-opening-chunk len xp xp-path xn xn-path chunk


```

PTranscript s = chunk @ PTranscript t
PState t = foldl concat (PState s) chunk
s ≤ t
PQueryCounter t = PQueryCounter s
authenticated-opening-in t
  (opening-root = rt,
   opening-length = len,
   opening-index = idx,
   opening-value = xp,
   opening-path = xp-path)
authenticated-opening-in t
  (opening-root = rt,
   opening-length = len,
   opening-index = fri-sibling-index len idx,
   opening-value = xn,
   opening-path = xn-path)
proof –
from outcome obtain xp s1 xp-path s2 xn s3 xn-path s4 s5 ap1 s6 s7
  ap2 s8 s9 where
  read-xp: Some (xp, s1) ∈ set-dist (execute read s)
and read-xp-path:
  Some (xp-path, s2) ∈
    set-dist (execute (ntimes read (floor-log len)) s1)
and read-xn: Some (xn, s3) ∈ set-dist (execute read s2)
and read-xn-path:
  Some (xn-path, s4) ∈
    set-dist (execute (ntimes read (floor-log len)) s3)
and assert-xp: Some ((), s5) ∈ set-dist (execute (assert (xp = x)) s4)
and check1:
  Some (ap1, s6) ∈
    set-dist (execute (check-authentication-path len idx xp xp-path) s5)
and assert-ap1:
  Some ((), s7) ∈ set-dist (execute (assert (ap1 = rt)) s6)
and check2:
  Some (ap2, s8) ∈
    set-dist (execute
      (check-authentication-path len ((idx + len div 2) mod len) xn
        xn-path) s7)
and assert-ap2:
  Some ((), s9) ∈ set-dist (execute (assert (ap2 = rt)) s8)
and ret:
  Some (out, t) ∈
    set-dist (execute
      (return
        (idx mod (len div 2),
          (xp + xn) div 2 +
            b * ((xp - xn) div (2 * ((h ^ idx) * shift) ^ pw)),
          len div 2, pw + pw)) s9)
unfolding fri-layer-opening-step-def

```

```

    by (auto simp: Let-def elim!: set-dist-bindE)
  from read-outcome[OF read-xp] obtain rest1 where
    tr-s: PTranscript s = xp # rest1
    and st-s1: PState s1 = concat (PState s) xp
    and tr-s1: PTranscript s1 = rest1
    and ext-s-s1: s ≤ s1
    and query-count-s1: PQueryCounter s1 = PQueryCounter s
  by blast
  have xp-path-res:
    length xp-path = floor-log len ∧
    PTranscript s1 = xp-path @ PTranscript s2 ∧
    PState s2 = foldl concat (PState s1) xp-path ∧
    s1 ≤ s2 ∧
    PQueryCounter s2 = PQueryCounter s1
  using ntimes-read-any-outcome[OF read-xp-path] by simp
  from read-outcome[OF read-xn] obtain rest3 where
    tr-s2: PTranscript s2 = xn # rest3
    and st-s3: PState s3 = concat (PState s2) xn
    and tr-s3: PTranscript s3 = rest3
    and ext-s2-s3: s2 ≤ s3
    and query-count-s3: PQueryCounter s3 = PQueryCounter s2
  by blast
  have xn-path-res:
    length xn-path = floor-log len ∧
    PTranscript s3 = xn-path @ PTranscript s4 ∧
    PState s4 = foldl concat (PState s3) xn-path ∧
    s3 ≤ s4 ∧
    PQueryCounter s4 = PQueryCounter s3
  using ntimes-read-any-outcome[OF read-xn-path] by simp
  have xp-eq: xp = x
    using assert-xp unfolding assert-def
    by (cases xp = x) (auto simp: throw-no-outcome)
  have s5-eq: s5 = s4
    using assert-xp xp-eq unfolding assert-def by simp
  have s5-s6: s5 ≤ s6
    by (rule check-authentication-path-hash-extends[OF check1])
  have st-s6: PState s6 = PState s5
    using check-authentication-path-preserves-channel(1)[OF check1] .
  have tr-s6: PTranscript s6 = PTranscript s5
    using check-authentication-path-preserves-channel(2)[OF check1] .
  have query-count-s6: PQueryCounter s6 = PQueryCounter s5
    using check-authentication-path-preserves-channel(6)[OF check1] .
  have ap1-eq: ap1 = rt
    using assert-ap1 unfolding assert-def
    by (cases ap1 = rt) (auto simp: throw-no-outcome)
  have s7-eq: s7 = s6
    using assert-ap1 ap1-eq unfolding assert-def by simp
  have s7-s8: s7 ≤ s8
    by (rule check-authentication-path-hash-extends[OF check2])

```

```

have st-s8: PState s8 = PState s7
  using check-authentication-path-preserves-channel(1)[OF check2] .
have tr-s8: PTranscript s8 = PTranscript s7
  using check-authentication-path-preserves-channel(2)[OF check2] .
have query-count-s8: PQueryCounter s8 = PQueryCounter s7
  using check-authentication-path-preserves-channel(6)[OF check2] .
have ap2-eq: ap2 = rt
  using assert-ap2 unfolding assert-def
  by (cases ap2 = rt) (auto simp: throw-no-outcome)
have s9-eq: s9 = s8
  using assert-ap2 ap2-eq unfolding assert-def by simp
let ?x' =
  (xp + xn) div 2 +
  b * ((xp - xn) div (2 * ((h ^ idx) * shift) ^ pw))
let ?chunk = [xp] @ xp-path @ [xn] @ xn-path
have out-t:
  out = (idx mod (len div 2), ?x', len div 2, pw + pw) ^ t = s9
  using ret by simp
have x'-fold:
  ?x' = fri-fold-value b xp xn
  (fri-fold-denominator ((h ^ idx) * shift) pw)
  unfolding fri-fold-value-def fri-fold-denominator-def by simp
have chunk-shape:
  fri-layer-opening-chunk len xp xp-path xn xn-path ?chunk
  using xp-path-res xn-path-res unfolding fri-layer-opening-chunk-def by simp
have tr-t: PTranscript s = ?chunk @ PTranscript t
  using tr-s tr-s1 xp-path-res tr-s2 tr-s3 xn-path-res s5-eq tr-s6
  s7-eq tr-s8 s9-eq out-t
  by simp
have st-t: PState t = foldl concat (PState s) ?chunk
  using st-s1 xp-path-res st-s3 xn-path-res s5-eq st-s6 s7-eq st-s8
  s9-eq out-t
  by simp
have s-t: s ≤ t
  using ext-s-s1 xp-path-res ext-s2-s3 xn-path-res s5-s6 s7-s8 out-t
  unfolding s5-eq s7-eq s9-eq
  by (meson hash-ext-trans)
have query-count-t: PQueryCounter t = PQueryCounter s
  using out-t query-count-s1
  xp-path-res query-count-s3 xn-path-res query-count-s6 query-count-s8
  s5-eq s7-eq s9-eq
  by simp
have sibling-bound: (idx + len div 2) mod len < len
  using len-pos by simp
have xp-path-length: length xp-path = floor-log len
  using xp-path-res by simp
have xn-path-length: length xn-path = floor-log len
  using xn-path-res by simp
have s6-t: s6 ≤ t

```

```

    using s7-s8 out-t unfolding s7-eq s9-eq by simp
have xp-bound:
  merkle-path-bound rt len idx xp xp-path t
proof -
  have merkle-path-bound ap1 len idx xp xp-path s6
    by (rule check-authentication-path-outcome-bound[OF check1])
  then have merkle-path-bound rt len idx xp xp-path s6
    unfolding ap1-eq .
  then show ?thesis
    by (rule merkle-path-bound-mono[OF - s6-t])
qed
have xn-bound:
  merkle-path-bound rt len (fri-sibling-index len idx) xn xn-path t
proof -
  have merkle-path-bound ap2 len ((idx + len div 2) mod len)
    xn xn-path s8
    by (rule check-authentication-path-outcome-bound[OF check2])
  then show ?thesis
    using ap2-eq out-t unfolding fri-sibling-index-def s9-eq by simp
qed
have xp-auth:
  authenticated-opening-in t
  (opening-root = rt, opening-length = len,
   opening-index = idx, opening-value = xp,
   opening-path = xp-path)
  unfolding authenticated-opening-in-def
  using idx-bound xp-path-length xp-bound by simp
have xn-auth:
  authenticated-opening-in t
  (opening-root = rt, opening-length = len,
   opening-index = fri-sibling-index len idx,
   opening-value = xn, opening-path = xn-path)
  unfolding authenticated-opening-in-def
  using sibling-bound xn-path-length xn-bound
  unfolding fri-sibling-index-def by simp
show ?thesis
proof (rule that)
  show out = (idx mod (len div 2), ?x', len div 2, pw + pw)
    using out-t by simp
  show xp = x
    by (rule xp-eq)
  show ?x' =
    fri-fold-value b xp xn
    (fri-fold-denominator ((h ^ idx) * shift) pw)
    by (rule x'-fold)
  show ?chunk = [xp] @ xp-path @ [xn] @ xn-path
    by simp
  show fri-layer-opening-chunk len xp xp-path xn xn-path ?chunk
    by (rule chunk-shape)

```

```

show  $P\text{Transcript } s = ?\text{chunk } @ \ P\text{Transcript } t$ 
  by (rule tr-t)
show  $P\text{State } t = \text{foldl concat } (P\text{State } s) \ ?\text{chunk}$ 
  by (rule st-t)
show  $s \leq t$ 
  by (rule s-t)
show  $P\text{QueryCounter } t = P\text{QueryCounter } s$ 
  by (rule query-count-t)
show authenticated-opening-in t
  (opening-root = rt, opening-length = len,
   opening-index = idx, opening-value = xp,
   opening-path = xp-path)
  by (rule xp-auth)
show authenticated-opening-in t
  (opening-root = rt, opening-length = len,
   opening-index = fri-sibling-index len idx,
   opening-value = xn, opening-path = xn-path)
  by (rule xn-auth)
qed
qed

lemma mfold-fri-layer-opening-steps-hash-extends:
  fixes  $s \ t :: ('f, 'a) \text{protocol-channel-scheme}$ 
  assumes outcome:
     $\text{Some } (out, t) \in$ 
      set-dist
      (execute (mfold (idx, x, len, pw)
        (map fri-layer-opening-step bfs))  $s$ )
  shows  $s \leq t$ 
proof –
  from mfold-fri-layer-openings-outcome[OF outcome]
  obtain layer-chunks chunk where
     $s \leq t$ 
    by blast
  then show ?thesis .
qed

lemma mfold-fri-layer-opening-steps-authenticated-opening-mono:
  fixes  $s \ t :: ('f, 'a) \text{protocol-channel-scheme}$ 
  assumes auth: authenticated-opening-in s opening
  and outcome:
     $\text{Some } (out, t) \in$ 
      set-dist
      (execute (mfold (idx, x, len, pw)
        (map fri-layer-opening-step bfs))  $s$ )
  shows authenticated-opening-in t opening
  by (rule authenticated-opening-in-mono
    [OF auth mfold-fri-layer-opening-steps-hash-extends[OF outcome]])

```

lemma *mfold-fri-layer-opening-head-chunk-authenticated*:
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes $\text{len-pos}: 0 < \text{len}$
and $\text{idx-bound}: \text{idx} < \text{len}$
and outcome :
 $\text{Some } (\text{out}, t) \in$
 set-dist
 $(\text{execute}$
 $(\text{mfold } (\text{idx}, x, \text{len}, \text{pw})$
 $(\text{map } \text{fri-layer-opening-step } ((b, \text{rt}) \# \text{bfs})))\ s)$
obtains chunk **where** $\text{fri-layer-chunk-authenticated } \text{rt } \text{len } \text{idx } \text{chunk } t$
proof –
from outcome **obtain** $\text{out1 } s1$ **where**
 head :
 $\text{Some } (\text{out1}, s1) \in$
 set-dist
 $(\text{execute } (\text{fri-layer-opening-step } (b, \text{rt}) (\text{idx}, x, \text{len}, \text{pw}))\ s)$
and tail :
 $\text{Some } (\text{out}, t) \in$
 $\text{set-dist } (\text{execute } (\text{mfold } \text{out1 } (\text{map } \text{fri-layer-opening-step } \text{bfs}))\ s1)$
by $(\text{auto elim!}: \text{set-dist-bindE})$
from $\text{fri-layer-opening-step-authenticated-chunk-pred}$
 $[\text{OF } \text{len-pos } \text{idx-bound } \text{head}]$
obtain chunk **where** auth-s1 :
 $\text{fri-layer-chunk-authenticated } \text{rt } \text{len } \text{idx } \text{chunk } s1$
by blast
obtain $\text{idx1 } x1\ \text{len1 } \text{pw1}$ **where** out1-eq :
 $\text{out1} = (\text{idx1}, x1, \text{len1}, \text{pw1})$
by $(\text{cases } \text{out1})$
have $s1\text{-t}: s1 \leq t$
by $(\text{rule } \text{mfold-fri-layer-opening-steps-hash-extends}$
 $[\text{OF } \text{tail}[\text{unfolded } \text{out1-eq}]])$
have $\text{fri-layer-chunk-authenticated } \text{rt } \text{len } \text{idx } \text{chunk } t$
by $(\text{rule } \text{fri-layer-chunk-authenticated-mono}[\text{OF } \text{auth-s1 } s1\text{-t}])$
then show $?thesis$
by $(\text{rule } \text{that})$
qed

lemma *mfold-fri-layer-opening-selected-chunk-authenticated-at-exists*:
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes outcome :
 $\text{Some } (\text{out}, t) \in$
 set-dist
 $(\text{execute}$
 $(\text{mfold } (\text{idx}, x, \text{len}, \text{pw})$
 $(\text{map } \text{fri-layer-opening-step } \text{bfs}))\ s)$
and $j\text{-bound}: j < \text{length } \text{bfs}$
and layer-len-pos :
 $0 < \text{fri-layer-lengths } (\text{length } \text{bfs})\ \text{len} ! j$

```

and layer-idx-bound:
  fri-layer-indices (length bfs) idx len ! j <
  fri-layer-lengths (length bfs) len ! j
shows  $\exists$  chunk.
  fri-layer-chunk-authenticated (snd (bfs ! j))
  (fri-layer-lengths (length bfs) len ! j)
  (fri-layer-indices (length bfs) idx len ! j)
  chunk t
using outcome j-bound layer-len-pos layer-idx-bound
proof (induction bfs arbitrary: idx x len pw s out t j)
case Nil
then show ?case by simp
next
case (Cons bf bfs)
obtain b rt where bf-eq: bf = (b, rt)
by (cases bf)
from Cons.prems(1)[unfolded bf-eq]
obtain idx1 x1 len1 pw1 s1 where
  head:
    Some ((idx1, x1, len1, pw1), s1)  $\in$ 
      set-dist
      (execute (fri-layer-opening-step (b, rt) (idx, x, len, pw)) s)
and tail:
  Some (out, t)  $\in$ 
    set-dist
    (execute
      (mfold (idx1, x1, len1, pw1)
        (map fri-layer-opening-step bfs)) s1)
by (auto elim!: set-dist-bindE)
show ?case
proof (cases j)
case 0
have len-pos: 0 < len
using Cons.prems(3) 0 by simp
have idx-bound: idx < len
using Cons.prems(4) 0 by simp
from fri-layer-opening-step-authenticated-chunk-pred
  [OF len-pos idx-bound head]
obtain chunk where auth-s1:
  fri-layer-chunk-authenticated rt len idx chunk s1
by blast
have s1-t: s1  $\leq$  t
by (rule mfold-fri-layer-opening-steps-hash-extends
  [OF tail])
have auth-t:
  fri-layer-chunk-authenticated rt len idx chunk t
by (rule fri-layer-chunk-authenticated-mono[OF auth-s1 s1-t])
show ?thesis
using auth-t 0 bf-eq by auto

```

```

next
case (Suc k)
from fri-layer-opening-step-outcome[OF head]
obtain xp xp-path xn xn-path x' where out1-eq:
  (idx1, x1, len1, pw1) =
    (idx mod (len div 2), x', len div 2, pw + pw)
by blast
have k-bound: k < length bfs
  using Cons.premis(2) Suc by simp
have tail-len-pos:
  0 < fri-layer-lengths (length bfs) (len div 2) ! k
  using Cons.premis(3) Suc by simp
have tail-idx-bound:
  fri-layer-indices (length bfs) (idx mod (len div 2))
    (len div 2) ! k <
    fri-layer-lengths (length bfs) (len div 2) ! k
  using Cons.premis(4) Suc by simp
show ?thesis
  using Cons.IH[OF tail[unfolded out1-eq] k-bound
    tail-len-pos tail-idx-bound] Suc
  by auto
qed
qed

lemma mfold-fri-layer-opening-selected-chunk-authenticated-at:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (out, t) ∈
      set-dist
        (execute
          (mfold (idx, x, len, pw)
            (map fri-layer-opening-step bfs)) s)
  and j-bound: j < length bfs
  and layer-len-pos:
    0 < fri-layer-lengths (length bfs) len ! j
  and layer-idx-bound:
    fri-layer-indices (length bfs) idx len ! j <
    fri-layer-lengths (length bfs) len ! j
  obtains chunk where
    fri-layer-chunk-authenticated (snd (bfs ! j))
      (fri-layer-lengths (length bfs) len ! j)
      (fri-layer-indices (length bfs) idx len ! j)
    chunk t
  using mfold-fri-layer-opening-selected-chunk-authenticated-at-exists
    [OF outcome j-bound layer-len-pos layer-idx-bound]
  by blast

lemma receive-query-commits-selected-chunk-authenticated-at:
  fixes s t :: ('f, 'a) protocol-channel-scheme

```



```

assumes outcome:
  Some (out, t) ∈
    set-dist
      (execute
        (mfold (idx, x, len, pw) (receive-query-commits bfs)) s)
and j-bound: j < length bfs
and layer-len-pos:
  0 < fri-layer-lengths (length bfs) len ! j
and layer-idx-bound:
  fri-layer-indices (length bfs) idx len ! j <
    fri-layer-lengths (length bfs) len ! j
obtains chunk where
  fri-layer-chunk-authenticated (snd (bfs ! j))
    (fri-layer-lengths (length bfs) len ! j)
    (fri-layer-indices (length bfs) idx len ! j)
  chunk t
proof –
  have map-eq: receive-query-commits bfs = map fri-layer-opening-step bfs
    unfolding receive-query-commits-def fri-layer-opening-step-def
    by (simp add: fun-eq-iff split: prod.splits)
  show ?thesis
    by (rule mfold-fri-layer-opening-selected-chunk-authenticated-at
      [OF outcome[unfolded map-eq] j-bound layer-len-pos
        layer-idx-bound that])
qed

lemma receive-query-commits-mfold-hash-extends:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (out, t) ∈
      set-dist
        (execute (mfold (idx, x, len, pw)
          (receive-query-commits bfs)) s)
  shows s ≤ t
proof –
  have map-eq: receive-query-commits bfs = map fri-layer-opening-step bfs
    unfolding receive-query-commits-def fri-layer-opening-step-def
    by (simp add: fun-eq-iff split: prod.splits)
  show ?thesis
    by (rule mfold-fri-layer-opening-steps-hash-extends
      [OF outcome[unfolded map-eq]])
qed

lemma receive-query-commits-mfold-authenticated-opening-mono:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes auth: authenticated-opening-in s opening
  and outcome:
    Some (out, t) ∈
      set-dist

```

```

      (execute (mfold (idx, x, len, pw)
        (receive-query-commits bfs)) s)
shows authenticated-opening-in t opening
by (rule authenticated-opening-in-mono
  [OF auth receive-query-commits-mfold-hash-extends[OF outcome]])

lemma receive-query-commits-mfold-head-chunk-authenticated:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes len-pos: 0 < len
and idx-bound: idx < len
and outcome:
  Some (out, t) ∈
    set-dist
      (execute
        (mfold (idx, x, len, pw)
          (receive-query-commits ((b, rt) # bfs))) s)
obtains chunk where fri-layer-chunk-authenticated rt len idx chunk t
proof –
have map-eq:
  receive-query-commits ((b, rt) # bfs) =
    map fri-layer-opening-step ((b, rt) # bfs)
unfolding receive-query-commits-def fri-layer-opening-step-def
by (simp add: fun-eq-iff split: prod.splits)
from mfold-fri-layer-opening-head-chunk-authenticated
  [OF len-pos idx-bound outcome[unfolded map-eq]]
obtain chunk where fri-layer-chunk-authenticated rt len idx chunk t
by blast
then show ?thesis
by (rule that)
qed

lemma verifier-query-round-after-index-trace-fri-head-chunk-authenticated:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes f-fl-eq: f-fl = (b, rt) # f-fl-tail
and outcome:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s)
obtains idx chunk where
  idx = index (to-nat raw)
  fri-layer-chunk-authenticated rt (clength * scale) idx chunk t
proof –
let ?idx = index (to-nat raw)
from outcome obtain fv s1 f-out s2 s3 c-out s4 where
  query-decommit:
  Some (fv, s1) ∈
    set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)

```

```

and trace-fri:
  Some (f-out, s2) ∈
    set-dist (execute
      (mfold (?idx, hd fv, clength * scale, 1)
        (receive-query-commits f-fl)) s1)
and assert-trace:
  Some (( ), s3) ∈
    set-dist
      (execute
        (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
and comp-fri:
  Some (c-out, s4) ∈
    set-dist (execute
      (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
        clength * scale, 1)
        (receive-query-commits fl)) s3)
and assert-comp:
  Some (( ), t) ∈
    set-dist
      (execute
        (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
unfolding verifier-query-round-after-index-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have len-pos: 0 < clength * scale
  using eval-domain-nontrivial by linarith
have idx-bound: ?idx < clength * scale
  by (rule index-less-domain)
from receive-query-commits-mfold-head-chunk-authenticated
  [OF len-pos idx-bound trace-fri[unfolded f-fl-eq]]
obtain chunk where auth-s2:
  fri-layer-chunk-authenticated rt (clength * scale) ?idx chunk s2
by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
    (auto simp: throw-no-outcome)
have s3-s4: s3 ≤ s4
  by (rule receive-query-commits-mfold-hash-extends[OF comp-fri])
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
have s2-t: s2 ≤ t
  using s3-s4 unfolding s3-eq t-eq by simp
have auth-t:
  fri-layer-chunk-authenticated rt (clength * scale) ?idx chunk t
  by (rule fri-layer-chunk-authenticated-mono[OF auth-s2 s2-t])
show ?thesis
  by (rule that[OF refl auth-t])

```

qed

lemma *verifier-query-round-after-index-trace-fri-selected-chunk-authenticated:*

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes *outcome:*

$\text{Some } ((), t) \in$

set-dist

(execute

(verifier-query-round-after-index-program

fr f-fl f-final as fl final raw) s)

and *j-bound:* $j < \text{length } f\text{-fl}$

and *layer-len-pos:*

$0 < \text{fri-layer-lengths } (\text{length } f\text{-fl})\ (\text{clength} * \text{scale})\ !\ j$

and *layer-idx-bound:*

$\text{fri-layer-indices } (\text{length } f\text{-fl})\ (\text{index } (\text{to-nat } \text{raw}))$

$(\text{clength} * \text{scale})\ !\ j <$

$\text{fri-layer-lengths } (\text{length } f\text{-fl})\ (\text{clength} * \text{scale})\ !\ j$

obtains *chunk where*

fri-layer-chunk-authenticated (snd (f-fl ! j))

*(fri-layer-lengths (length f-fl) (clength * scale) ! j)*

(fri-layer-indices (length f-fl) (index (to-nat raw)))

*(clength * scale) ! j)*

chunk t

proof $-$

let $?idx = \text{index } (\text{to-nat } \text{raw})$

from *outcome* **obtain** $fv\ s1\ f\text{-out}\ s2\ s3\ c\text{-out}\ s4$ **where**

trace-fri:

$\text{Some } (f\text{-out}, s2) \in$

set-dist (execute

*(mfold (?idx, hd fv, clength * scale, 1)*

(receive-query-commits f-fl)) s1)

and *assert-trace:*

$\text{Some } ((), s3) \in$

set-dist

(execute

(assert (case f-out of (-, f-x, -, -) $\Rightarrow$ $f\text{-x} = f\text{-final}$)) s2)

and *comp-fri:*

$\text{Some } (c\text{-out}, s4) \in$

set-dist (execute

*(mfold (?idx, cp-eval as fv (h $\wedge$?idx * shift),*

*clength * scale, 1)*

(receive-query-commits fl)) s3)

and *assert-comp:*

$\text{Some } ((), t) \in$

set-dist

(execute

(assert (case c-out of (-, x, -, -) $\Rightarrow$ $x = \text{final}$)) s4)

unfolding *verifier-query-round-after-index-program-def*

by *(auto simp: Let-def split: prod.splits elim!: set-dist-bindE)*

```

from receive-query-commits-selected-chunk-authenticated-at
  [OF trace-fri j-bound layer-len-pos layer-idx-bound]
obtain chunk where auth-s2:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) ?idx (clength * scale) ! j)
  chunk s2
by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -)  $\Rightarrow$  f-x = f-final)
    (auto simp: throw-no-outcome)
have s3-s4: s3  $\leq$  s4
  by (rule receive-query-commits-mfold-hash-extends[OF comp-fri])
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -)  $\Rightarrow$  x = final)
    (auto simp: throw-no-outcome)
have s2-t: s2  $\leq$  t
  using s3-s4 unfolding s3-eq t-eq by simp
have auth-t:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) ?idx (clength * scale) ! j)
  chunk t
  by (rule fri-layer-chunk-authenticated-mono[OF auth-s2 s2-t])
then show ?thesis
  by (rule that)
qed

lemma verifier-query-round-after-index-composition-fri-head-chunk-authenticated:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-eq: fl = (b, rt) # fl-tail
  and outcome:
    Some (( ), t)  $\in$ 
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s)
  obtains idx chunk where
    idx = index (to-nat raw)
    fri-layer-chunk-authenticated rt (clength * scale) idx chunk t
proof -
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1)  $\in$ 
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
  and trace-fri:

```

```

    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some (( ), s3) ∈
      set-dist
        (execute
          (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
          clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some (( ), t) ∈
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have len-pos: 0 < clength * scale
  using eval-domain-nontrivial by linarith
have idx-bound: ?idx < clength * scale
  by (rule index-less-domain)
from receive-query-commits-mfold-head-chunk-authenticated
  [OF len-pos idx-bound comp-fri[unfolded fl-eq]]
obtain chunk where auth-s4:
  fri-layer-chunk-authenticated rt (clength * scale) ?idx chunk s4
  by blast
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
have auth-t:
  fri-layer-chunk-authenticated rt (clength * scale) ?idx chunk t
  using auth-s4 unfolding t-eq .
show ?thesis
  by (rule that[OF refl auth-t])
qed

lemma verifier-query-round-after-index-composition-fri-selected-chunk-authenticated:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (( ), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program

```

```

      fr f-fl f-final as fl final raw) s)
and j-bound: j < length fl
and layer-len-pos:
  0 < fri-layer-lengths (length fl) (clength * scale) ! j
and layer-idx-bound:
  fri-layer-indices (length fl) (index (to-nat raw))
  (clength * scale) ! j <
  fri-layer-lengths (length fl) (clength * scale) ! j
obtains chunk where
  fri-layer-chunk-authenticated (snd (fl ! j))
  (fri-layer-lengths (length fl) (clength * scale) ! j)
  (fri-layer-indices (length fl) (index (to-nat raw))
  (clength * scale) ! j)
  chunk t
proof -
let ?idx = index (to-nat raw)
from outcome obtain fv s1 f-out s2 s3 c-out s4 where
comp-fri:
  Some (c-out, s4) ∈
  set-dist (execute
    (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
    clength * scale, 1)
    (receive-query-commits fl)) s3)
and assert-comp:
  Some (( ), t) ∈
  set-dist
  (execute
    (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
unfolding verifier-query-round-after-index-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
from receive-query-commits-selected-chunk-authenticated-at
[OF comp-fri j-bound layer-len-pos layer-idx-bound]
obtain chunk where auth-s4:
  fri-layer-chunk-authenticated (snd (fl ! j))
  (fri-layer-lengths (length fl) (clength * scale) ! j)
  (fri-layer-indices (length fl) ?idx (clength * scale) ! j)
  chunk s4
by blast
have t-eq: t = s4
using assert-comp unfolding assert-def
by (cases case c-out of (-, x, -, -) ⇒ x = final)
(auto simp: throw-no-outcome)
have auth-t:
  fri-layer-chunk-authenticated (snd (fl ! j))
  (fri-layer-lengths (length fl) (clength * scale) ! j)
  (fri-layer-indices (length fl) ?idx (clength * scale) ! j)
  chunk t
using auth-s4 unfolding t-eq .
then show ?thesis

```

by (rule that)
qed

lemma *verifier-query-round-program-trace-fri-head-chunk-authenticated:*
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes $f\text{-fl}\text{-eq}: f\text{-fl} = (b, rt) \# f\text{-fl}\text{-tail}$
and *outcome:*
 $\text{Some } ((), t) \in$
 set-dist
 $(\text{execute}$
 $(\text{verifier-query-round-program } fr\ f\text{-fl}\ f\text{-final}\ as\ fl\ final)\ s)$
obtains $raw\ idx\ chunk$ **where**
 $idx = \text{index } (to\text{-nat } raw)$
 $fri\text{-layer-chunk-authenticated } rt\ (clength * scale)\ idx\ chunk\ t$
proof –
from $\text{outcome}[\text{unfolded } \text{verifier-query-round-program-alt-def}]$
obtain $raw\ s0$ **where**
 $raw:$
 $\text{Some } (raw, s0) \in$
 $\text{set-dist } (\text{execute } \text{receive-query-index-challenge } s)$
and *tail:*
 $\text{Some } ((), t) \in$
 set-dist
 $(\text{execute}$
 $(\text{verifier-query-round-after-index-program}$
 $fr\ f\text{-fl}\ f\text{-final}\ as\ fl\ final\ raw)\ s0)$
by $(\text{auto } elim!: \text{set-dist-bindE})$
from $\text{verifier-query-round-after-index-trace-fri-head-chunk-authenticated}$
 $[OF\ f\text{-fl}\text{-eq}\ tail]$
obtain $idx\ chunk$ **where**
 $idx\text{-eq}: idx = \text{index } (to\text{-nat } raw)$
and $auth: fri\text{-layer-chunk-authenticated } rt\ (clength * scale)\ idx\ chunk\ t$
by $blast$
show $?thesis$
by $(\text{rule that}[OF\ idx\text{-eq}\ auth])$
qed

lemma *verifier-query-round-program-trace-fri-selected-chunk-authenticated:*
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes *outcome:*
 $\text{Some } ((), t) \in$
 set-dist
 $(\text{execute}$
 $(\text{verifier-query-round-program } fr\ f\text{-fl}\ f\text{-final}\ as\ fl\ final)\ s)$
and $j\text{-bound}: j < \text{length } f\text{-fl}$
and $raw\text{-layer}:$
 $\bigwedge raw.$
 $0 < fri\text{-layer-lengths } (length\ f\text{-fl})\ (clength * scale) ! j \wedge$
 $fri\text{-layer-indices } (length\ f\text{-fl})\ (\text{index } (to\text{-nat } raw))$


```

      (clength * scale) ! j <
      fri-layer-lengths (length f-fl) (clength * scale) ! j
obtains raw chunk where
  fri-layer-chunk-authenticated (snd (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! j)
  chunk t
proof -
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where
  raw:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
and tail:
    Some ((), t) ∈
      set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s0)
by (auto elim!: set-dist-bindE)
have layer-len-pos:
  0 < fri-layer-lengths (length f-fl) (clength * scale) ! j
using raw-layer[of raw] by simp
have layer-idx-bound:
  fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! j <
    fri-layer-lengths (length f-fl) (clength * scale) ! j
using raw-layer[of raw] by simp
from verifier-query-round-after-index-trace-fri-selected-chunk-authenticated
  [OF tail j-bound layer-len-pos layer-idx-bound]
obtain chunk where auth:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! j)
  chunk t
by blast
show ?thesis
by (rule that[OF auth])
qed

```

```

lemma verifier-query-round-program-trace-fri-selected-chunk-authenticated-with-lookup:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some ((), t) ∈
    set-dist
    (execute
      (verifier-query-round-program fr f-fl f-final as fl final) s)

```

```

and j-bound:  $j < \text{length } f\text{-fl}$ 
and raw-layer:
   $\wedge \text{raw.}$ 
   $0 < \text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! j \wedge$ 
   $\text{fri-layer-indices } (\text{length } f\text{-fl}) (\text{index } (\text{to-nat } \text{raw}))$ 
   $(\text{clength} * \text{scale}) ! j <$ 
   $\text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! j$ 
obtains raw chunk where
  fri-layer-chunk-authenticated (snd (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) (index (to-nat raw))
  (clength * scale) ! j)
  chunk t
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof –
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where
  raw:
    Some (raw, s0)  $\in$ 
    set-dist (execute receive-query-index-challenge s)
and tail:
  Some ((), t)  $\in$ 
  set-dist
  (execute
    (verifier-query-round-after-index-program
      fr f-fl f-final as fl final raw) s0)
by (auto elim!: set-dist-bindE)
have layer-len-pos:
   $0 < \text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! j$ 
using raw-layer[of raw] by simp
have layer-idx-bound:
  fri-layer-indices (length f-fl) (index (to-nat raw))
  (clength * scale) ! j <
  fri-layer-lengths (length f-fl) (clength * scale) ! j
using raw-layer[of raw] by simp
from verifier-query-round-after-index-trace-fri-selected-chunk-authenticated
  [OF tail j-bound layer-len-pos layer-idx-bound]
obtain chunk where auth:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) (index (to-nat raw))
  (clength * scale) ! j)
  chunk t
by blast
have lookup-s0:
  fmlookup (HashMap s0)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
using receive-query-index-challenge-outcome[OF raw] by simp

```

```

have s0-t:  $s0 \leq t$ 
  using verifier-query-round-after-index-program-outcome[OF tail] by simp
have lookup-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  by (rule hash-extension-lookup[OF lookup-s0 s0-t])
show ?thesis
  by (rule that[OF auth lookup-t])
qed

lemma verifier-query-round-program-composition-fri-head-chunk-authenticated:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-eq: fl = (b, rt) # fl-tail
and outcome:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
obtains raw idx chunk where
  idx = index (to-nat raw)
  fri-layer-chunk-authenticated rt (clength * scale) idx chunk t
proof -
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where
  raw:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
and tail:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s0)
  by (auto elim!: set-dist-bindE)
from verifier-query-round-after-index-composition-fri-head-chunk-authenticated
  [OF fl-eq tail]
obtain idx chunk where
  idx-eq: idx = index (to-nat raw)
  and auth: fri-layer-chunk-authenticated rt (clength * scale) idx chunk t
  by blast
show ?thesis
  by (rule that[OF idx-eq auth])
qed

lemma verifier-query-round-program-composition-fri-selected-chunk-authenticated:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some ((), t) ∈
    set-dist

```

```

      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
    and j-bound: j < length fl
    and raw-layer:
       $\wedge$ raw.
      0 < fri-layer-lengths (length fl) (clength * scale) ! j  $\wedge$ 
      fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j <
      fri-layer-lengths (length fl) (clength * scale) ! j
    obtains raw chunk where
      fri-layer-chunk-authenticated (snd (fl ! j))
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! j)
      chunk t
  proof -
  from outcome[unfolded verifier-query-round-program-alt-def]
  obtain raw s0 where
    raw:
      Some (raw, s0)  $\in$ 
      set-dist (execute receive-query-index-challenge s)
  and tail:
    Some ((), t)  $\in$ 
    set-dist
    (execute
      (verifier-query-round-after-index-program
        fr f-fl f-final as fl final raw) s0)
  by (auto elim!: set-dist-bindE)
  have layer-len-pos:
    0 < fri-layer-lengths (length fl) (clength * scale) ! j
  using raw-layer[of raw] by simp
  have layer-idx-bound:
    fri-layer-indices (length fl) (index (to-nat raw))
    (clength * scale) ! j <
    fri-layer-lengths (length fl) (clength * scale) ! j
  using raw-layer[of raw] by simp
  from
    verifier-query-round-after-index-composition-fri-selected-chunk-authenticated
    [OF tail j-bound layer-len-pos layer-idx-bound]
  obtain chunk where auth:
    fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j)
    chunk t
  by blast
  show ?thesis
  by (rule that[OF auth])
qed

```

lemma *verifier-query-round-program-composition-fri-selected-chunk-authenticated-with-lookup*:
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes *outcome*:
 $\text{Some } ((\), t) \in$
 set-dist
 $(\text{execute}$
 $(\text{verifier-query-round-program}\ \text{fr}\ f\text{-fl}\ f\text{-final}\ \text{as}\ \text{fl}\ \text{final})\ s)$
and *j-bound*: $j < \text{length}\ \text{fl}$
and *raw-layer*:
 $\bigwedge \text{raw.}$
 $0 < \text{fri-layer-lengths}\ (\text{length}\ \text{fl})\ (\text{clength} * \text{scale})\ !\ j \wedge$
 $\text{fri-layer-indices}\ (\text{length}\ \text{fl})\ (\text{index}\ (\text{to-nat}\ \text{raw}))$
 $(\text{clength} * \text{scale})\ !\ j <$
 $\text{fri-layer-lengths}\ (\text{length}\ \text{fl})\ (\text{clength} * \text{scale})\ !\ j$
obtains *raw chunk* **where**
 $\text{fri-layer-chunk-authenticated}\ (\text{snd}\ (\text{fl}\ !\ j))$
 $(\text{fri-layer-lengths}\ (\text{length}\ \text{fl})\ (\text{clength} * \text{scale})\ !\ j)$
 $(\text{fri-layer-indices}\ (\text{length}\ \text{fl})\ (\text{index}\ (\text{to-nat}\ \text{raw})))$
 $(\text{clength} * \text{scale})\ !\ j)$
 $\text{chunk}\ t$
 $\text{fmlookup}\ (\text{HashMap}\ t)$
 $(\text{QueryIndexChallenge}\ (\text{PQueryCounter}\ s)\ (\text{PState}\ s)) = \text{Some}\ \text{raw}$
proof –
from *outcome*[*unfolded verifier-query-round-program-alt-def*]
obtain *raw s0* **where**
 raw:
 $\text{Some}\ (\text{raw}, s0) \in$
 $\text{set-dist}\ (\text{execute}\ \text{receive-query-index-challenge}\ s)$
and *tail*:
 $\text{Some}\ ((\), t) \in$
 set-dist
 $(\text{execute}$
 $(\text{verifier-query-round-after-index-program}$
 $\text{fr}\ f\text{-fl}\ f\text{-final}\ \text{as}\ \text{fl}\ \text{final}\ \text{raw})\ s0)$
by (*auto elim!:* *set-dist-bindE*)
have *layer-len-pos*:
 $0 < \text{fri-layer-lengths}\ (\text{length}\ \text{fl})\ (\text{clength} * \text{scale})\ !\ j$
using *raw-layer*[*of raw*] **by** *simp*
have *layer-idx-bound*:
 $\text{fri-layer-indices}\ (\text{length}\ \text{fl})\ (\text{index}\ (\text{to-nat}\ \text{raw}))$
 $(\text{clength} * \text{scale})\ !\ j <$
 $\text{fri-layer-lengths}\ (\text{length}\ \text{fl})\ (\text{clength} * \text{scale})\ !\ j$
using *raw-layer*[*of raw*] **by** *simp*
from
 $\text{verifier-query-round-after-index-composition-fri-selected-chunk-authenticated}$
 $[\text{OF}\ \text{tail}\ j\text{-bound}\ \text{layer-len-pos}\ \text{layer-idx-bound}]$
obtain *chunk* **where** *auth*:
 $\text{fri-layer-chunk-authenticated}\ (\text{snd}\ (\text{fl}\ !\ j))$

```

    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j)
    chunk t
  by blast
have lookup-s0:
  fmllookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using receive-query-index-challenge-outcome[OF raw] by simp
have s0-t: s0 ≤ t
  using verifier-query-round-after-index-program-outcome[OF tail] by simp
have lookup-t:
  fmllookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  by (rule hash-extension-lookup[OF lookup-s0 s0-t])
show ?thesis
  by (rule that[OF auth lookup-t])
qed

lemma ntimes-verifier-query-rounds-selected-trace-fri-head-chunk-authenticated-at:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes f-fl-eq: f-fl = (b, rt) # f-fl-tail
  and outcome:
    Some (results, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) n)
          s)
  and i-bound: i < n
  obtains raw idx chunk where
    idx = index (to-nat raw)
    fri-layer-chunk-authenticated rt (clength * scale) idx chunk t
proof -
  let ?round = verifier-query-round-program fr f-fl f-final as fl final
  from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
  obtain prefix x suffix s-i s-suc where
    round:
      Some (x, s-suc) ∈ set-dist (execute ?round s-i)
  and suffix:
    Some (suffix, t) ∈
      set-dist (execute (ntimes ?round (n - Suc i)) s-suc)
  by blast
have x-eq: x = ()
  by (cases x) simp
from verifier-query-round-program-trace-fri-head-chunk-authenticated
  [OF f-fl-eq round[unfolded x-eq]]
  obtain raw idx chunk where
    idx-eq: idx = index (to-nat raw)

```

```

and auth-suc:
  fri-layer-chunk-authenticated rt (clength * scale) idx chunk s-suc
by blast
have s-suc-t: s-suc ≤ t
  using ntimes-verifier-query-rounds-outcome[OF suffix] by blast
have auth-t:
  fri-layer-chunk-authenticated rt (clength * scale) idx chunk t
  by (rule fri-layer-chunk-authenticated-mono[OF auth-suc s-suc-t])
show ?thesis
  by (rule that[OF idx-eq auth-t])
qed

lemma ntimes-verifier-query-rounds-selected-trace-fri-chunk-authenticated-at:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (results, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s)
and i-bound: i < n
and j-bound: j < length f-fl
and raw-layer:
   $\bigwedge$ raw.
    0 < fri-layer-lengths (length f-fl) (clength * scale) ! j ∧
    fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! j <
    fri-layer-lengths (length f-fl) (clength * scale) ! j
obtains raw chunk where
  fri-layer-chunk-authenticated (snd (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! j)
  chunk t
proof –
let ?round = verifier-query-round-program fr f-fl f-final as fl final
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix x suffix s-i s-suc where
  round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
  Some (suffix, t) ∈
    set-dist (execute (ntimes ?round (n – Suc i)) s-suc)
by blast
have x-eq: x = ()
by (cases x) simp
from verifier-query-round-program-trace-fri-selected-chunk-authenticated
  [OF round[unfolded x-eq] j-bound raw-layer]

```

```

obtain raw chunk where auth-suc:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! j)
  chunk s-suc
by blast
have s-suc-t: s-suc ≤ t
  using ntimes-verifier-query-rounds-outcome[OF suffix] by blast
have auth-t:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! j)
  chunk t
  by (rule fri-layer-chunk-authenticated-mono[OF auth-suc s-suc-t])
show ?thesis
  by (rule that[OF auth-t])
qed

lemma ntimes-verifier-query-rounds-selected-trace-fri-chunk-authenticated-with-prefix-at:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (results, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s)
and i-bound: i < n
and j-bound: j < length f-fl
and raw-layer:
   $\bigwedge^{\text{raw.}}$ 
    0 < fri-layer-lengths (length f-fl) (clength * scale) ! j ∧
    fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! j <
    fri-layer-lengths (length f-fl) (clength * scale) ! j
obtains raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
  suffix-chunks layer-chunk where
  idx = index (to-nat raw)
  length prefix-query-idxs = i
  length prefix-chunks = i
  length suffix-query-idxs = n - Suc i
  length suffix-chunks = n - Suc i
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
   $\bigwedge k. k < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
   $\bigwedge k. k < n - \text{Suc } i \implies$ 

```



```

    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
  PTranscript s =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @ PTranscript t
  fri-layer-chunk-authenticated (snd (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) idx (clength * scale) ! j)
    layer-chunk t
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + i)
      (state-after-query-chunks (PState s) prefix-chunks i)) =
    Some raw
proof –
  let ?round = verifier-query-round-program fr f-fl f-final as fl final
  from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
  obtain prefix x suffix s-i s-suc where
    prefix:
      Some (prefix, s-i) ∈
        set-dist (execute (ntimes ?round i) s)
    and round:
      Some (x, s-suc) ∈ set-dist (execute ?round s-i)
    and suffix:
      Some (suffix, t) ∈
        set-dist (execute (ntimes ?round (n - Suc i)) s-suc)
  by blast
  have x-eq: x = ()
  by (cases x) simp
  show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
  fix prefix-query-idxs prefix-chunks
  assume len-prefix-idxs: length prefix-query-idxs = i
  and len-prefix-chunks: length prefix-chunks = i
  and transcript-prefix:
    PTranscript s = List.concat prefix-chunks @ PTranscript s-i
  and state-i:
    PState s-i =
      state-after-query-chunks (PState s) prefix-chunks i
  and counter-i: PQueryCounter s-i = PQueryCounter s + i
  and prefix-ext: s ≤ s-i
  and prefix-rounds:
     $\bigwedge k. k < i \implies$ 
      verifier-query-round-chunk (prefix-query-idxs ! k)
        (map snd f-fl) (map snd fl) (prefix-chunks ! k)
  show ?thesis
proof (rule verifier-query-round-program-outcome-with-authenticated-trace-openings
  [OF round[unfolded x-eq]])
  fix raw idx chunk openings
  assume idx-eq: idx = index (to-nat raw)

```

```

and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
and transcript-round:
  PTranscript s-i = chunk @ PTranscript s-suc
and state-suc:
  PState s-suc = foldl concat (PState s-i) chunk
and round-ext: s-i ≤ s-suc
and counter-suc: PQueryCounter s-suc = Suc (PQueryCounter s-i)
and lookup-suc:
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
and len-openings:
  length openings = length (powers-scaled idx)
and trace-indices:
  map opening-index openings = powers-scaled idx
and trace-table-suc:
  partial-authenticated-table fr (scale * clength) openings
    s-suc
show ?thesis
proof (rule
  verifier-query-round-program-trace-fri-selected-chunk-authenticated-with-lookup
    [OF round[unfolded x-eq] j-bound raw-layer])
fix raw' layer-chunk
assume auth-suc:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) (index (to-nat raw'))
      (clength * scale) ! j)
    layer-chunk s-suc
and lookup-suc':
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw'
have raw'-eq: raw' = raw
  using lookup-suc lookup-suc' by simp
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs: length suffix-query-idxs = n - Suc i
and len-suffix-chunks:
  length suffix-chunks = n - Suc i
and transcript-suffix:
  PTranscript s-suc =
    List.concat suffix-chunks @ PTranscript t
and suffix-state:
  PState t =

```

```

      state-after-query-chunks (PState s-suc) suffix-chunks
      (n - Suc i)
    and suffix-counter:
      PQueryCounter t = PQueryCounter s-suc + (n - Suc i)
    and suffix-ext: s-suc ≤ t
    and suffix-rounds:
      ∧k. k < n - Suc i ⇒
        verifier-query-round-chunk (suffix-query-idxs ! k)
        (map snd f-fl) (map snd fl) (suffix-chunks ! k)
  have transcript-all:
    PTranscript s =
      List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript t
  using transcript-prefix transcript-round transcript-suffix
  by simp
  have auth-t-row:
    fri-layer-chunk-authenticated (snd (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) (index (to-nat row))
      (clength * scale) ! j)
    layer-chunk t
  by (rule fri-layer-chunk-authenticated-mono
    [OF auth-suc[unfolded raw'-eq] suffix-ext])
  have auth-t:
    fri-layer-chunk-authenticated (snd (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) idx (clength * scale) ! j)
    layer-chunk t
  using auth-t-row idx-eq by simp
  have lookup-t:
    fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + i)
      (state-after-query-chunks (PState s) prefix-chunks i)) =
    Some raw
  using hash-extension-lookup[OF lookup-suc suffix-ext]
  unfolding counter-i state-i .
  show ?thesis
  by (rule that[OF idx-eq len-prefix-idxs len-prefix-chunks
    len-suffix-idxs len-suffix-chunks chunk-shape
    prefix-rounds suffix-rounds transcript-all auth-t
    lookup-t])
qed
qed
qed
qed
qed

lemma ntimes-verifier-query-rounds-selected-trace-fri-chunk-aligned-with-replay-at:
  fixes query-state final-state :: ('f, 'a) protocol-channel-scheme

```

```

assumes outcome:
  Some (results, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
        query-state)
and i-bound:  $i < \text{rounds}$ 
and j-bound:  $j < \text{length } f\text{-fl}$ 
and raw-layer:
   $\bigwedge \text{raw.}$ 
     $0 < \text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! j \wedge$ 
     $\text{fri-layer-indices } (\text{length } f\text{-fl}) (\text{index } (\text{to-nat } \text{raw}))$ 
     $(\text{clength} * \text{scale}) ! j <$ 
     $\text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! j$ 
and len-raw:  $\text{length } \text{raw-idxs} = \text{rounds}$ 
and query-idxs-eq:
   $\text{query-idxs} = \text{map } (\lambda \text{raw. index } (\text{to-nat } \text{raw})) \text{ raw-idxs}$ 
and len-query-chunks:  $\text{length } \text{query-chunks} = \text{rounds}$ 
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and query-chunk:
   $\bigwedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
and replay-lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks
        (PState query-state) query-chunks i)) =
    Some (raw-idxs ! i)
obtains layer-chunk where
  fri-layer-chunk-authenticated (snd (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) (query-idxs ! i)
    (clength * scale) ! j)
  layer-chunk final-state
proof (rule
  ntimes-verifier-query-rounds-selected-trace-fri-chunk-authenticated-with-prefix-at
  [OF outcome i-bound j-bound raw-layer])
fix raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
  suffix-chunks layer-chunk
assume selected-idx:  $\text{idx} = \text{index } (\text{to-nat } \text{raw})$ 
and len-prefix-idxs:  $\text{length } \text{prefix-query-idxs} = i$ 
and len-prefix-chunks:  $\text{length } \text{prefix-chunks} = i$ 
and len-suffix-idxs:  $\text{length } \text{suffix-query-idxs} = \text{rounds} - \text{Suc } i$ 

```

```

and len-suffix-chunks: length suffix-chunks = rounds - Suc i
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
and prefix-rounds:
   $\bigwedge k. k < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
    (map snd f-fl) (map snd fl) (prefix-chunks ! k)
and suffix-rounds:
   $\bigwedge k. k < rounds - Suc i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
    (map snd f-fl) (map snd fl) (suffix-chunks ! k)
and selected-transcript:
  PTranscript query-state =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
    PTranscript final-state
and auth:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) idx (clength * scale) ! j)
  layer-chunk final-state
and selected-lookup:
  fmlookup (HashMap final-state)
  (QueryIndexChallenge (PQueryCounter query-state + i)
    (state-after-query-chunks (PState query-state) prefix-chunks i)) =
    Some raw
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
  using len-prefix-chunks len-suffix-chunks i-bound by simp
have selected-chunk-lens:
   $\bigwedge k. k < rounds \implies$ 
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound: k < rounds
  show
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
proof (cases k < i)
  case True
  have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length (prefix-chunks ! k) =
      verifier-query-round-transcript-length
      (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF prefix-rounds[OF True]])

```

```

then show ?thesis
  using nth-eq
  by (simp add: verifier-query-round-transcript-length-index-irrelevant)
next
case False
show ?thesis
proof (cases k = i)
  case True
  have nth-eq: ?selected-chunks ! k = chunk
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length chunk =
      verifier-query-round-transcript-length idx
        (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF chunk-shape])
  then show ?thesis
    using nth-eq
    by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
next
case False
let ?k = k - Suc i
have ge: Suc i ≤ k
  using <¬ k < i> False by linarith
have k-bound': ?k < rounds - Suc i
  using ge k-bound by linarith
have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
  using ge len-prefix-chunks by (simp add: nth-append)
have
  length (suffix-chunks ! ?k) =
    verifier-query-round-transcript-length
      (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
  by (rule verifier-query-round-chunk-length
    [OF suffix-rounds[OF k-bound']])
then show ?thesis
  using nth-eq
  by (simp add:
    verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-lens:
  ∧k. k < rounds ⇒
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  using query-chunk verifier-query-round-chunk-length by blast
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query by simp

```

```

have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
     $\wedge k. k < \text{length } ?\text{selected-chunks} \implies$ 
     $\text{length } (?selected-chunks ! k) = \text{length } (query-chunks ! k)$ 
    using selected-chunk-lens query-chunk-lens len-selected-chunks by simp
  have concat-eq-empty:
     $List.concat ?selected-chunks @ [] = List.concat query-chunks$ 
    using concat-eq by simp
  have len-eq:  $\text{length } ?selected-chunks = \text{length } query-chunks$ 
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks  $\wedge ([] :: 'f \text{ list}) = []$ 
    by (rule concat-append-eq-concat-same-chunk-lengths
        [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
   $state\text{-after}\text{-query}\text{-chunks } (PState \text{ query}\text{-state}) \text{ prefix}\text{-chunks } i =$ 
   $state\text{-after}\text{-query}\text{-chunks } (PState \text{ query}\text{-state}) \text{ query}\text{-chunks } i$ 
proof -
  have
     $state\text{-after}\text{-query}\text{-chunks } (PState \text{ query}\text{-state}) ?selected-chunks i =$ 
     $state\text{-after}\text{-query}\text{-chunks } (PState \text{ query}\text{-state}) \text{ prefix}\text{-chunks } i$ 
    by (rule state-after-query-chunks-append-prefix[OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have selected-lookup-global:
   $fmlookup (HashMap \text{ final}\text{-state})$ 
   $(QueryIndexChallenge (PQueryCounter \text{ query}\text{-state} + i)$ 
   $(state\text{-after}\text{-query}\text{-chunks } (PState \text{ query}\text{-state}) \text{ query}\text{-chunks } i)) =$ 
   $Some \text{ raw}$ 
  using selected-lookup prefix-state-eq by simp
have raw-eq:  $\text{raw} = \text{raw}\text{-idxs } ! i$ 
  using selected-lookup-global replay-lookup[OF i-bound] by simp
have idx-eq:  $\text{idx} = \text{query}\text{-idxs } ! i$ 
  using selected-idx raw-eq query-idxs-eq len-raw i-bound by simp
show ?thesis
  by (rule that[of layer-chunk]) (use auth idx-eq in simp)
qed

lemma ntimes-verifier-query-rounds-selected-composition-fri-head-chunk-authenticated-at:
  fixes  $s \ t :: ('f, 'a) \text{ protocol-channel-scheme}$ 
  assumes fl-eq:  $fl = (b, rt) \# fl\text{-tail}$ 
  and outcome:
     $Some (results, t) \in$ 
     $set\text{-dist}$ 
     $(execute$ 

```

```

      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final) n)
      s)
  and i-bound:  $i < n$ 
  obtains raw idx chunk where
    idx = index (to-nat raw)
    fri-layer-chunk-authenticated rt (clength * scale) idx chunk t
  proof -
  let ?round = verifier-query-round-program fr f-fl f-final as fl final
  from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
  obtain prefix x suffix s-i s-suc where
    round:
      Some (x, s-suc) ∈ set-dist (execute ?round s-i)
  and suffix:
    Some (suffix, t) ∈
      set-dist (execute (ntimes ?round (n - Suc i)) s-suc)
  by blast
  have x-eq:  $x = ()$ 
  by (cases x) simp
  from verifier-query-round-program-composition-fri-head-chunk-authenticated
    [OF fl-eq round[unfolded x-eq]]
  obtain raw idx chunk where
    idx-eq:  $idx = index (to-nat raw)$ 
  and auth-suc:
    fri-layer-chunk-authenticated rt (clength * scale) idx chunk s-suc
  by blast
  have s-suc-t:  $s-suc \leq t$ 
  using ntimes-verifier-query-rounds-outcome[OF suffix] by blast
  have auth-t:
    fri-layer-chunk-authenticated rt (clength * scale) idx chunk t
  by (rule fri-layer-chunk-authenticated-mono[OF auth-suc s-suc-t])
  show ?thesis
  by (rule that[OF idx-eq auth-t])
qed

```

lemma *ntimes-verifier-query-rounds-selected-composition-fri-chunk-authenticated-at:*
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes *outcome:*
 Some (results, t) ∈
 set-dist
 (execute
 (ntimes
 (verifier-query-round-program fr f-fl f-final as fl final) n)
 s)
and *i-bound:* $i < n$
and *j-bound:* $j < \text{length fl}$
and *raw-layer:*
 $\bigwedge^{\text{raw.}}$
 $0 < \text{fri-layer-lengths (length fl) (clength * scale) ! j} \wedge$


```

    fri-layer-indices (length fl) (index (to-nat raw))
    (clength * scale) ! j <
    fri-layer-lengths (length fl) (clength * scale) ! j
obtains raw chunk where
    fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw))
    (clength * scale) ! j)
    chunk t
proof –
let ?round = verifier-query-round-program fr f-fl f-final as fl final
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix x suffix s-i s-suc where
    round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
    Some (suffix, t) ∈
    set-dist (execute (ntimes ?round (n – Suc i)) s-suc)
by blast
have x-eq: x = ()
by (cases x) simp
from
    verifier-query-round-program-composition-fri-selected-chunk-authenticated
    [OF round[unfolded x-eq] j-bound raw-layer]
obtain raw chunk where auth-suc:
    fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw))
    (clength * scale) ! j)
    chunk s-suc
by blast
have s-suc-t: s-suc ≤ t
using ntimes-verifier-query-rounds-outcome[OF suffix] by blast
have auth-t:
    fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw))
    (clength * scale) ! j)
    chunk t
by (rule fri-layer-chunk-authenticated-mono[OF auth-suc s-suc-t])
show ?thesis
by (rule that[OF auth-t])
qed

lemma ntimes-verifier-query-rounds-selected-composition-fri-chunk-authenticated-with-prefix-at:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
    Some (results, t) ∈
    set-dist

```

```

      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s)
    and i-bound:  $i < n$ 
    and j-bound:  $j < \text{length } fl$ 
    and raw-layer:
       $\wedge_{\text{raw.}}$ 
       $0 < \text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! j \wedge$ 
       $\text{fri-layer-indices } (\text{length } fl) (\text{index } (\text{to-nat } \text{raw}))$ 
       $(\text{clength} * \text{scale}) ! j <$ 
       $\text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! j$ 
    obtains raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
      suffix-chunks layer-chunk where
      idx = index (to-nat raw)
      length prefix-query-idxs = i
      length prefix-chunks = i
      length suffix-query-idxs =  $n - \text{Suc } i$ 
      length suffix-chunks =  $n - \text{Suc } i$ 
      verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
       $\wedge k. k < i \implies$ 
      verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
       $\wedge k. k < n - \text{Suc } i \implies$ 
      verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
    PTranscript s =
      List.concat (prefix-chunks @ chunk # suffix-chunks) @ PTranscript t
    fri-layer-chunk-authenticated (snd (fl ! j))
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) idx (clength * scale) ! j)
      layer-chunk t
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) prefix-chunks i)) =
      Some raw
proof –
let ?round = verifier-query-round-program fr f-fl f-final as fl final
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i)  $\in$ 
    set-dist (execute (ntimes ?round i) s)
  and round:
    Some (x, s-suc)  $\in$  set-dist (execute ?round s-i)
  and suffix:
    Some (suffix, t)  $\in$ 
    set-dist (execute (ntimes ?round (n - Suc i)) s-suc)
by blast

```

```

have  $x\text{-eq}$ :  $x = ()$ 
  by (cases  $x$ ) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
  fix prefix-query-idxs prefix-chunks
  assume len-prefix-idxs: length prefix-query-idxs =  $i$ 
  and len-prefix-chunks: length prefix-chunks =  $i$ 
  and transcript-prefix:
     $PTranscript\ s = List.concat\ prefix-chunks\ @\ PTranscript\ s\text{-}i$ 
  and state-i:
     $PState\ s\text{-}i =$ 
       $state\text{-}after\text{-}query\text{-}chunks\ (PState\ s)\ prefix-chunks\ i$ 
  and counter-i:  $PQueryCounter\ s\text{-}i = PQueryCounter\ s + i$ 
  and prefix-ext:  $s \leq s\text{-}i$ 
  and prefix-rounds:
     $\bigwedge k. k < i \implies$ 
       $verifier\text{-}query\text{-}round\text{-}chunk\ (prefix\text{-}query\text{-}idxs\ !\ k)$ 
       $(map\ snd\ f\text{-}fl)\ (map\ snd\ fl)\ (prefix\text{-}chunks\ !\ k)$ 
  show ?thesis
proof (rule verifier-query-round-program-outcome-with-authenticated-trace-openings
  [OF round[unfolded x-eq]])
  fix raw idx chunk openings
  assume idx-eq:  $idx = index\ (to\text{-}nat\ raw)$ 
  and chunk-shape:
     $verifier\text{-}query\text{-}round\text{-}chunk\ idx\ (map\ snd\ f\text{-}fl)\ (map\ snd\ fl)$ 
     $chunk$ 
  and transcript-round:
     $PTranscript\ s\text{-}i = chunk\ @\ PTranscript\ s\text{-}suc$ 
  and state-suc:
     $PState\ s\text{-}suc = foldl\ concat\ (PState\ s\text{-}i)\ chunk$ 
  and round-ext:  $s\text{-}i \leq s\text{-}suc$ 
  and counter-suc:  $PQueryCounter\ s\text{-}suc = Suc\ (PQueryCounter\ s\text{-}i)$ 
  and lookup-suc:
     $flookup\ (HashMap\ s\text{-}suc)$ 
     $(QueryIndexChallenge\ (PQueryCounter\ s\text{-}i)\ (PState\ s\text{-}i)) =$ 
     $Some\ raw$ 
  and len-openings:
     $length\ openings = length\ (powers\text{-}scaled\ idx)$ 
  and trace-indices:
     $map\ opening\text{-}index\ openings = powers\text{-}scaled\ idx$ 
  and trace-table-suc:
     $partial\text{-}authenticated\text{-}table\ fr\ (scale * clength)\ openings$ 
     $s\text{-}suc$ 
  show ?thesis
proof (rule
    verifier-query-round-program-composition-fri-selected-chunk-authenticated-with-lookup
    [OF round[unfolded x-eq] j-bound raw-layer])
  fix  $raw'\ layer\text{-}chunk$ 

```

```

assume auth-suc:
  fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw'))
      (clength * scale) ! j)
    layer-chunk s-suc
and lookup-suc':
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw'
have raw'-eq: raw' = raw
  using lookup-suc lookup-suc' by simp
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs: length suffix-query-idxs = n - Suc i
  and len-suffix-chunks:
    length suffix-chunks = n - Suc i
  and transcript-suffix:
    PTranscript s-suc =
      List.concat suffix-chunks @ PTranscript t
  and suffix-state:
    PState t =
      state-after-query-chunks (PState s-suc) suffix-chunks
      (n - Suc i)
  and suffix-counter:
    PQueryCounter t = PQueryCounter s-suc + (n - Suc i)
  and suffix-ext: s-suc ≤ t
  and suffix-rounds:
     $\bigwedge k. k < n - Suc\ i \implies$ 
      verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
have transcript-all:
  PTranscript s =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
    PTranscript t
  using transcript-prefix transcript-round transcript-suffix
  by simp
have auth-t-row:
  fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j)
    layer-chunk t
  by (rule fri-layer-chunk-authenticated-mono
    [OF auth-suc[unfolded raw'-eq] suffix-ext])
have auth-t:
  fri-layer-chunk-authenticated (snd (fl ! j))

```

```

    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) idx (clength * scale) ! j)
    layer-chunk t
  using auth-t-raw idx-eq by simp
have lookup-t:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s + i)
    (state-after-query-chunks (PState s) prefix-chunks i)) =
  Some raw
  using hash-extension-lookup[OF lookup-suc suffix-ext]
  unfolding counter-i state-i .
show ?thesis
  by (rule that[OF idx-eq len-prefix-idxs len-prefix-chunks
    len-suffix-idxs len-suffix-chunks chunk-shape
    prefix-rounds suffix-rounds transcript-all auth-t
    lookup-t])
qed
qed
qed
qed
qed

lemma ntimes-verifier-query-rounds-selected-composition-fri-chunk-aligned-with-replay-at:
  fixes query-state final-state :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (results, final-state) ∈
    set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      query-state)
  and i-bound: i < rounds
  and j-bound: j < length fl
  and raw-layer:
     $\bigwedge_{raw.}$ 
    0 < fri-layer-lengths (length fl) (clength * scale) ! j  $\wedge$ 
    fri-layer-indices (length fl) (index (to-nat raw))
    (clength * scale) ! j <
    fri-layer-lengths (length fl) (clength * scale) ! j
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map ( $\lambda raw. index (to-nat raw)$ ) raw-idxs
  and len-query-chunks: length query-chunks = rounds
  and transcript-query:
    PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
  and query-chunk:
     $\bigwedge i. i < rounds \implies$ 

```

```

    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
and replay-lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
obtains layer-chunk where
  fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (query-idxs ! i)
      (clength * scale) ! j)
  layer-chunk final-state
proof (rule
  ntimes-verifier-query-rounds-selected-composition-fri-chunk-authenticated-with-prefix-at
    [OF outcome i-bound j-bound raw-layer])
fix raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
  suffix-chunks layer-chunk
assume selected-idx: idx = index (to-nat raw)
and len-prefix-idxs: length prefix-query-idxs = i
and len-prefix-chunks: length prefix-chunks = i
and len-suffix-idxs: length suffix-query-idxs = rounds - Suc i
and len-suffix-chunks: length suffix-chunks = rounds - Suc i
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
and prefix-rounds:
   $\bigwedge k. k < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
and suffix-rounds:
   $\bigwedge k. k < \text{rounds} - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
and selected-transcript:
  PTranscript query-state =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript final-state
and auth:
  fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) idx (clength * scale) ! j)
  layer-chunk final-state
and selected-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) prefix-chunks i)) =
    Some raw

```

```

let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
  using len-prefix-chunks len-suffix-chunks i-bound by simp
have selected-chunk-lens:
   $\wedge k. k < rounds \implies$ 
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound: k < rounds
  show
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  proof (cases k < i)
    case True
    have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length (prefix-chunks ! k) =
        verifier-query-round-transcript-length
          (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
      by (rule verifier-query-round-chunk-length[OF prefix-rounds[OF True]])
    then show ?thesis
      using nth-eq
      by (simp add: verifier-query-round-transcript-length-index-irrelevant)
  next
    case False
    show ?thesis
    proof (cases k = i)
      case True
      have nth-eq: ?selected-chunks ! k = chunk
        using True len-prefix-chunks by (simp add: nth-append)
      have
        length chunk =
          verifier-query-round-transcript-length idx
            (map snd f-fl) (map snd fl)
        by (rule verifier-query-round-chunk-length[OF chunk-shape])
      then show ?thesis
        using nth-eq
        by (simp add:
            verifier-query-round-transcript-length-index-irrelevant)
    next
      case False
      let ?k = k - Suc i
      have ge: Suc i  $\leq$  k
        using  $\langle \neg k < i \rangle$  False by linarith
      have k-bound': ?k < rounds - Suc i

```

```

    using ge k-bound by linarith
  have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
    using ge len-prefix-chunks by (simp add: nth-append)
  have
    length (suffix-chunks ! ?k) =
      verifier-query-round-transcript-length
        (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound]])
  then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-lens:
   $\bigwedge k. k < rounds \implies$ 
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
    using query-chunk verifier-query-round-chunk-length by blast
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
     $\bigwedge k. k < length \text{ ?selected-chunks } \implies$ 
      length (?selected-chunks ! k) = length (query-chunks ! k)
    using selected-chunk-lens query-chunk-lens len-selected-chunks by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
    using concat-eq by simp
  have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks  $\wedge$  ( [] :: 'f list ) = []
    by (rule concat-append-eq-concat-same-chunk-lengths
        [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
      state-after-query-chunks (PState query-state) prefix-chunks i
    by (rule state-after-query-chunks-append-prefix[OF len-prefix-chunks])

```



```

then show ?thesis
  using chunks-eq by simp
qed
have selected-lookup-global:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some raw
  using selected-lookup prefix-state-eq by simp
have raw-eq: raw = raw-idxs ! i
  using selected-lookup-global replay-lookup[OF i-bound] by simp
have idx-eq: idx = query-idxs ! i
  using selected-idx raw-eq query-idxs-eq len-raw i-bound by simp
show ?thesis
  by (rule that[of layer-chunk]) (use auth idx-eq in simp)
qed

lemma accepted-fri-opening-transcript-trace-selected-chunk-authenticated-at:
assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers
and out-eq: out = Some (result, final-state)
and round-bound: i < rounds
and layer-bound: j < length trace-roots
and raw-layer:
   $\bigwedge$ raw.
    0 < fri-layer-lengths (length trace-roots) (clength * scale) ! j  $\wedge$ 
    fri-layer-indices (length trace-roots) (index (to-nat raw))
      (clength * scale) ! j <
    fri-layer-lengths (length trace-roots) (clength * scale) ! j
obtains layer-chunk where
  fri-layer-chunk-authenticated (trace-roots ! j)
  (fri-layer-lengths (length trace-roots) (clength * scale) ! j)
  (fri-layer-indices (length trace-roots) (query-idxs ! i)
    (clength * scale) ! j)
  layer-chunk final-state
proof –
from fri-openings obtain result' final-state' fr f-fl as fl query-state
  raw-idxs query-chunks where
  out-def: out = Some (result', final-state')
and trace-roots-eq: trace-roots = map snd f-fl
and composition-roots-eq: composition-roots = map snd fl
and outcome:
  Some (result', final-state')  $\in$ 
  set-dist (execute
    (ntimes
      (verifier-query-round-program fr f-fl trace-final as fl
        composition-final)))

```

```

    rounds)
    query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
    query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
    PTranscript query-state =
        List.concat query-chunks @ PTranscript final-state'
and query-chunk:
     $\bigwedge i. i < \text{rounds} \implies$ 
        verifier-query-round-chunk (query-idxs ! i)
        trace-roots composition-roots (query-chunks ! i)
and replay-lookup:
     $\bigwedge i. i < \text{rounds} \implies$ 
        fmllookup (HashMap final-state')
        (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
unfolding accepted-fri-opening-transcript-def by blast
have result-eq: result' = result
using out-def out-eq by simp
have final-state-eq: final-state' = final-state
using out-def out-eq by simp
have outcome':
    Some (result, final-state)  $\in$ 
        set-dist (execute
            (ntimes
                (verifier-query-round-program fr f-fl trace-final as fl
                composition-final)
            rounds)
            query-state)
using outcome result-eq final-state-eq by simp
have j-bound-f: j < length f-fl
using layer-bound trace-roots-eq by simp
have raw-layer-f:
     $\bigwedge$ raw.
        0 < fri-layer-lengths (length f-fl) (clength * scale) ! j  $\wedge$ 
        fri-layer-indices (length f-fl) (index (to-nat raw))
        (clength * scale) ! j <
        fri-layer-lengths (length f-fl) (clength * scale) ! j
using raw-layer trace-roots-eq by simp
have query-chunk-f:
     $\bigwedge i. i < \text{rounds} \implies$ 
        verifier-query-round-chunk (query-idxs ! i)
        (map snd f-fl) (map snd fl) (query-chunks ! i)
using query-chunk trace-roots-eq composition-roots-eq by simp
from ntimes-verifier-query-rounds-selected-trace-fri-chunk-aligned-with-replay-at
    [OF outcome' round-bound j-bound-f raw-layer-f len-raw query-idxs-eq

```

```

    len-query-chunks transcript-query[unfolded final-state-eq]
    query-chunk-f replay-lookup[unfolded final-state-eq]]
obtain layer-chunk where auth:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) (query-idxs ! i)
    (clength * scale) ! j)
  layer-chunk final-state
by blast
have root-eq: trace-roots ! j = snd (f-fl ! j)
  using trace-roots-eq layer-bound by simp
show ?thesis
  by (rule that[of layer-chunk])
    (use auth trace-roots-eq root-eq in simp)
qed

lemma accepted-fri-opening-transcript-composition-selected-chunk-authenticated-at:
assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers
and out-eq: out = Some (result, final-state)
and round-bound: i < rounds
and layer-bound: j < length composition-roots
and raw-layer:
   $\bigwedge$ raw.
    0 < fri-layer-lengths (length composition-roots) (clength * scale) ! j  $\wedge$ 
    fri-layer-indices (length composition-roots) (index (to-nat raw))
      (clength * scale) ! j <
    fri-layer-lengths (length composition-roots) (clength * scale) ! j
obtains layer-chunk where
  fri-layer-chunk-authenticated (composition-roots ! j)
  (fri-layer-lengths (length composition-roots) (clength * scale) ! j)
  (fri-layer-indices (length composition-roots) (query-idxs ! i)
    (clength * scale) ! j)
  layer-chunk final-state
proof –
from fri-openings obtain result' final-state' fr f-fl as fl query-state
  raw-idxs query-chunks where
  out-def: out = Some (result', final-state')
and trace-roots-eq: trace-roots = map snd f-fl
and composition-roots-eq: composition-roots = map snd fl
and outcome:
  Some (result', final-state')  $\in$ 
  set-dist (execute
    (ntimes
      (verifier-query-round-program fr f-fl trace-final as fl
        composition-final)
      rounds)

```

```

    query-state)
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-query-chunks: length query-chunks = rounds
  and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state'
  and query-chunk:
    ∧i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)
  and replay-lookup:
    ∧i. i < rounds ⇒
      fmllookup (HashMap final-state')
      (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
  unfolding accepted-fri-opening-transcript-def by blast
  have result-eq: result' = result
  using out-def out-eq by simp
  have final-state-eq: final-state' = final-state
  using out-def out-eq by simp
  have outcome':
    Some (result, final-state) ∈
      set-dist (execute
        (ntimes
          (verifier-query-round-program fr f-fl trace-final as fl
            composition-final)
          rounds)
        query-state)
  using outcome result-eq final-state-eq by simp
  have j-bound-f: j < length fl
  using layer-bound composition-roots-eq by simp
  have raw-layer-f:
    ∧raw.
      0 < fri-layer-lengths (length fl) (clength * scale) ! j ∧
      fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j <
      fri-layer-lengths (length fl) (clength * scale) ! j
  using raw-layer composition-roots-eq by simp
  have query-chunk-f:
    ∧i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
  using query-chunk trace-roots-eq composition-roots-eq by simp
  from ntimes-verifier-query-rounds-selected-composition-fri-chunk-aligned-with-replay-at
  [OF outcome' round-bound j-bound-f raw-layer-f len-raw query-idxs-eq
    len-query-chunks transcript-query[unfolded final-state-eq]]

```

```

      query-chunk-f replay-lookup[unfolded final-state-eq]]
obtain layer-chunk where auth:
  fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (query-idxs ! i)
      (clength * scale) ! j)
    layer-chunk final-state
  by blast
have root-eq: composition-roots ! j = snd (fl ! j)
  using composition-roots-eq layer-bound by simp
show ?thesis
  by (rule that[of layer-chunk])
    (use auth composition-roots-eq root-eq in simp)
qed

end

end

```

```

theory Soundness-FRI-Layer-Authenticated
imports Soundness-FRI-Layer-Merkle
begin

```

Authenticated FRI layer evidence paired with transcript layer evidence.

The exact equality between the layer chunk recorded in *accepted-fri-opening-transcript* and the layer chunk authenticated by the verifier is still intentionally not asserted here. Instead this theory packages the checked facts that are currently available without a broad replay induction: the transcript gives a local FRI step, and the verifier execution gives an authenticated FRI chunk at the same root, layer length, and replayed query index.

```

context soundness
begin

```

```

definition generic-fri-transcript-step-with-authenticated-chunk
  :: 'f list  $\Rightarrow$  'f list  $\Rightarrow$  nat list  $\Rightarrow$ 
    'f list list list  $\Rightarrow$  ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    nat  $\Rightarrow$  nat  $\Rightarrow$  bool

```

where

```

generic-fri-transcript-step-with-authenticated-chunk roots challenges
  query-idxs round-layers final-state round-idx layer-idx  $\longleftrightarrow$ 
  fri-round-layer-evidence roots challenges query-idxs round-idx
  layer-idx round-layers  $\wedge$ 
  ( $\exists$  auth-chunk.
    fri-layer-chunk-authenticated (roots ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      auth-chunk final-state)

```

lemma *generic-fri-transcript-step-with-authenticated-chunkI*:
assumes *step*:
fri-round-layer-evidence roots challenges query-idxs round-idx
layer-idx round-layers
and *auth*:
fri-layer-chunk-authenticated (roots ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
auth-chunk final-state
shows
generic-fri-transcript-step-with-authenticated-chunk roots challenges
query-idxs round-layers final-state round-idx layer-idx
using *assms*
unfolding *generic-fri-transcript-step-with-authenticated-chunk-def*
by *blast*

lemma *generic-fri-transcript-step-with-authenticated-chunkE*:
assumes
generic-fri-transcript-step-with-authenticated-chunk roots challenges
query-idxs round-layers final-state round-idx layer-idx
obtains *auth-chunk* **where**
fri-round-layer-evidence roots challenges query-idxs round-idx
layer-idx round-layers
fri-layer-chunk-authenticated (roots ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
auth-chunk final-state
using *assms*
unfolding *generic-fri-transcript-step-with-authenticated-chunk-def*
by *blast*

definition *generic-fri-authenticated-same-layer-opening-conflict*
 $:: 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
generic-fri-authenticated-same-layer-opening-conflict roots query-idxs
final-state $\longleftrightarrow$
 $(\exists \text{ round-idx round-idx'} \text{ layer-idx opn opn'.$
 $\text{round-idx} < \text{length query-idxs} \wedge$
 $\text{round-idx'} < \text{length query-idxs} \wedge$
 $\text{layer-idx} < \text{length roots} \wedge$
 $\text{authenticated-opening-in final-state opn} \wedge$
 $\text{authenticated-opening-in final-state opn'} \wedge$
 $\text{opening-root opn} = \text{roots ! layer-idx} \wedge$
 $\text{opening-root opn'} = \text{roots ! layer-idx} \wedge$
 $\text{opening-length opn} = \text{fri-evidence-layer-len roots layer-idx} \wedge$
 $\text{opening-length opn'} = \text{fri-evidence-layer-len roots layer-idx} \wedge$
 $(\text{opening-index opn} =$

$fri\text{-}evidence\text{-}layer\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx\ layer\text{-}idx \vee$
 $opening\text{-}index\ opn =$
 $fri\text{-}sibling\text{-}index\ (fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx\ layer\text{-}idx)) \wedge$
 $(opening\text{-}index\ opn' =$
 $fri\text{-}evidence\text{-}layer\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx'\ layer\text{-}idx \vee$
 $opening\text{-}index\ opn' =$
 $fri\text{-}sibling\text{-}index\ (fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx'\ layer\text{-}idx)) \wedge$
 $opening\text{-}index\ opn = opening\text{-}index\ opn' \wedge$
 $opening\text{-}value\ opn \neq opening\text{-}value\ opn')$

lemma *generic-fri-authenticated-same-layer-opening-conflictI:*

assumes *round-bound:* $round\text{-}idx < length\ query\text{-}idxs$
and *round-bound':* $round\text{-}idx' < length\ query\text{-}idxs$
and *layer-bound:* $layer\text{-}idx < length\ roots$
and *auth:* *authenticated-opening-in final-state* opn
authenticated-opening-in final-state opn'
and *root:* $opening\text{-}root\ opn = roots\ !\ layer\text{-}idx$
 $opening\text{-}root\ opn' = roots\ !\ layer\text{-}idx$
and *len:*
 $opening\text{-}length\ opn = fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx$
 $opening\text{-}length\ opn' = fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx$
and *idx:*
 $opening\text{-}index\ opn =$
 $fri\text{-}evidence\text{-}layer\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx\ layer\text{-}idx \vee$
 $opening\text{-}index\ opn =$
 $fri\text{-}sibling\text{-}index\ (fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx\ layer\text{-}idx)$
 $opening\text{-}index\ opn' =$
 $fri\text{-}evidence\text{-}layer\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx'\ layer\text{-}idx \vee$
 $opening\text{-}index\ opn' =$
 $fri\text{-}sibling\text{-}index\ (fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx'\ layer\text{-}idx)$
and *same-index:* $opening\text{-}index\ opn = opening\text{-}index\ opn'$
and *neg:* $opening\text{-}value\ opn \neq opening\text{-}value\ opn'$

shows

generic-fri-authenticated-same-layer-opening-conflict $roots\ query\text{-}idxs$
final-state

unfolding *generic-fri-authenticated-same-layer-opening-conflict-def*

by (*intro* $exI[of - round\text{-}idx]$ $exI[of - round\text{-}idx']$
 $exI[of - layer\text{-}idx]$ $exI[of - opn]$ $exI[of - opn']$ *conjI*)
(use assms in auto)

lemma *generic-fri-authenticated-same-layer-opening-conflict-imp-partial-merkle:*

assumes

generic-fri-authenticated-same-layer-opening-conflict $roots\ query\text{-}idxs$
final-state

shows *partial-merkle-inconsistency-bad* s (*Some* (*result*, *final-state*))

proof –
from *assms* **obtain** *round-idx round-idx' layer-idx opn opn'* **where**
auth: authenticated-opening-in final-state opn
authenticated-opening-in final-state opn'
and *roots:*
opening-root opn = roots ! layer-idx
opening-root opn' = roots ! layer-idx
and *lengths:*
opening-length opn = fri-evidence-layer-len roots layer-idx
opening-length opn' = fri-evidence-layer-len roots layer-idx
and *same-index: opening-index opn = opening-index opn'*
and *neq: opening-value opn ≠ opening-value opn'*
unfolding *generic-fri-authenticated-same-layer-opening-conflict-def*
by *blast*
have *same-root: opening-root opn = opening-root opn'*
using *roots* **by** *simp*
have *same-length: opening-length opn = opening-length opn'*
using *lengths* **by** *simp*
show *?thesis*
unfolding *partial-merkle-inconsistency-bad-def accepted-def*
by (*intro conjI exI[of - result] exI[of - final-state]*
exI[of - opn] exI[of - opn'])
(use auth same-root same-length same-index neq in simp-all)
qed

lemma *generic-fri-authenticated-same-layer-opening-conflictI-base-base:*
assumes *round-bound: round-idx < length query-idxs*
and *round-bound': round-idx' < length query-idxs*
and *layer-bound: layer-idx < length roots*
and *auth:*
fri-layer-chunk-authenticated (roots ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
chunk final-state
and *auth':*
fri-layer-chunk-authenticated (roots ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
chunk' final-state
and *chunk:*
fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
xp xp-path xn xn-path chunk
and *chunk':*
fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
yp yp-path yn yn-path chunk'
and *same-index:*
fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
fri-evidence-layer-idx roots query-idxs round-idx' layer-idx
and *neq: xp ≠ yp*

shows
generic-fri-authenticated-same-layer-opening-conflict roots query-idxs
final-state

proof –
from *fri-layer-chunk-authenticated-matching-values*[*OF auth chunk*]
obtain *xp-path' xn-path'* **where** *auth-xp*:
authenticated-opening-in final-state
 (opening-root = roots ! layer-idx,
 opening-length = fri-evidence-layer-len roots layer-idx,
 opening-index = fri-evidence-layer-idx roots query-idxs round-idx
 layer-idx,
 opening-value = xp,
 opening-path = xp-path')
by blast
from *fri-layer-chunk-authenticated-matching-values*[*OF auth' chunk'*]
obtain *yp-path' yn-path'* **where** *auth-yp*:
authenticated-opening-in final-state
 (opening-root = roots ! layer-idx,
 opening-length = fri-evidence-layer-len roots layer-idx,
 opening-index = fri-evidence-layer-idx roots query-idxs round-idx'
 layer-idx,
 opening-value = yp,
 opening-path = yp-path')
by blast
let ?opn =
 (opening-root = roots ! layer-idx,
 opening-length = fri-evidence-layer-len roots layer-idx,
 opening-index = fri-evidence-layer-idx roots query-idxs round-idx
 layer-idx,
 opening-value = xp,
 opening-path = xp-path')
let ?opn' =
 (opening-root = roots ! layer-idx,
 opening-length = fri-evidence-layer-len roots layer-idx,
 opening-index = fri-evidence-layer-idx roots query-idxs round-idx'
 layer-idx,
 opening-value = yp,
 opening-path = yp-path')
show ?thesis
by (rule *generic-fri-authenticated-same-layer-opening-conflictI*
 [*OF round-bound round-bound' layer-bound auth-xp auth-yp*])
 (use same-index neq **in** simp-all)
qed

lemma *generic-fri-authenticated-same-layer-opening-conflictI-base-sibling*:
assumes round-bound: round-idx < length query-idxs
and round-bound': round-idx' < length query-idxs
and layer-bound: layer-idx < length roots
and auth:

```

    fri-layer-chunk-authenticated (roots ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      chunk final-state
  and auth':
    fri-layer-chunk-authenticated (roots ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
      chunk' final-state
  and chunk:
    fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
      xp xp-path xn xn-path chunk
  and chunk':
    fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
      yp yp-path yn yn-path chunk'
  and same-index:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
  and neq: xp ≠ yn
  shows
    generic-fri-authenticated-same-layer-opening-conflict roots query-idxs
      final-state
  proof -
  from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
  obtain xp-path' xn-path' where auth-xp:
    authenticated-opening-in final-state
    (|opening-root = roots ! layer-idx,
      opening-length = fri-evidence-layer-len roots layer-idx,
      opening-index = fri-evidence-layer-idx roots query-idxs round-idx
        layer-idx,
      opening-value = xp,
      opening-path = xp-path'|)
  by blast
  from fri-layer-chunk-authenticated-matching-values[OF auth' chunk']
  obtain yp-path' yn-path' where auth-yn:
    authenticated-opening-in final-state
    (|opening-root = roots ! layer-idx,
      opening-length = fri-evidence-layer-len roots layer-idx,
      opening-index =
        fri-sibling-index (fri-evidence-layer-len roots layer-idx)
          (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx),
      opening-value = yn,
      opening-path = yn-path'|)
  by blast
  show ?thesis
  by (rule generic-fri-authenticated-same-layer-opening-conflictI
    [OF round-bound round-bound' layer-bound auth-xp auth-yn])
    (use same-index neq in simp-all)

```

qed

lemma *generic-fri-authenticated-same-layer-opening-conflictI-sibling-base:*

assumes *round-bound*: $\text{round-idx} < \text{length query-idxs}$
and *round-bound'*: $\text{round-idx}' < \text{length query-idxs}$
and *layer-bound*: $\text{layer-idx} < \text{length roots}$
and *auth*:
 fri-layer-chunk-authenticated (*roots* ! *layer-idx*)
 (*fri-evidence-layer-len* *roots* *layer-idx*)
 (*fri-evidence-layer-idx* *roots* *query-idxs* *round-idx* *layer-idx*)
 chunk *final-state*
and *auth'*:
 fri-layer-chunk-authenticated (*roots* ! *layer-idx*)
 (*fri-evidence-layer-len* *roots* *layer-idx*)
 (*fri-evidence-layer-idx* *roots* *query-idxs* *round-idx'* *layer-idx*)
 chunk' *final-state*
and *chunk*:
 fri-layer-opening-chunk (*fri-evidence-layer-len* *roots* *layer-idx*)
 xp xp-path xn xn-path chunk
and *chunk'*:
 fri-layer-opening-chunk (*fri-evidence-layer-len* *roots* *layer-idx*)
 yp yp-path yn yn-path chunk'
and *same-index*:
 fri-sibling-index (*fri-evidence-layer-len* *roots* *layer-idx*)
 (*fri-evidence-layer-idx* *roots* *query-idxs* *round-idx* *layer-idx*) =
 fri-evidence-layer-idx *roots* *query-idxs* *round-idx'* *layer-idx*
and *neq*: $xn \neq yp$
shows
 generic-fri-authenticated-same-layer-opening-conflict *roots* *query-idxs*
 final-state

proof –

from *fri-layer-chunk-authenticated-matching-values*[*OF auth chunk*]
obtain *xp-path' xn-path'* **where** *auth-xn*:
 authenticated-opening-in final-state
 ($\text{opening-root} = \text{roots} ! \text{layer-idx}$,
 $\text{opening-length} = \text{fri-evidence-layer-len roots layer-idx}$,
 $\text{opening-index} =$
 fri-sibling-index (*fri-evidence-layer-len* *roots* *layer-idx*)
 (*fri-evidence-layer-idx* *roots* *query-idxs* *round-idx* *layer-idx*),
 $\text{opening-value} = xn$,
 $\text{opening-path} = xn\text{-path}'$)
by *blast*
from *fri-layer-chunk-authenticated-matching-values*[*OF auth' chunk'*]
obtain *yp-path' yn-path'* **where** *auth-yp*:
 authenticated-opening-in final-state
 ($\text{opening-root} = \text{roots} ! \text{layer-idx}$,
 $\text{opening-length} = \text{fri-evidence-layer-len roots layer-idx}$,
 $\text{opening-index} = \text{fri-evidence-layer-idx roots query-idxs round-idx}'$
 layer-idx ,

```

    opening-value = yp,
    opening-path = yp-path')
  by blast
show ?thesis
  by (rule generic-fri-authenticated-same-layer-opening-conflictI
      [OF round-bound round-bound' layer-bound auth-xn auth-yp])
      (use same-index neq in simp-all)
qed

lemma generic-fri-authenticated-same-layer-opening-conflictI-sibling-sibling:
  assumes round-bound: round-idx < length query-idxs
  and round-bound': round-idx' < length query-idxs
  and layer-bound: layer-idx < length roots
  and auth:
    fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    chunk final-state
  and auth':
    fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    chunk' final-state
  and chunk:
    fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    xp xp-path xn xn-path chunk
  and chunk':
    fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    yp yp-path yn yn-path chunk'
  and same-index:
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
  and neq: xn ≠ yn
  shows
    generic-fri-authenticated-same-layer-opening-conflict roots query-idxs
    final-state
proof -
  from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
  obtain xp-path' xn-path' where auth-xn:
    authenticated-opening-in final-state
    (|opening-root = roots ! layer-idx,
     opening-length = fri-evidence-layer-len roots layer-idx,
     opening-index =
       fri-sibling-index (fri-evidence-layer-len roots layer-idx)
       (fri-evidence-layer-idx roots query-idxs round-idx layer-idx),
     opening-value = xn,
     opening-path = xn-path')

```

```

    by blast
  from fri-layer-chunk-authenticated-matching-values[OF auth' chunk']
  obtain yp-path' yn-path' where auth-yn:
    authenticated-opening-in final-state
    (opening-root = roots ! layer-idx,
     opening-length = fri-evidence-layer-len roots layer-idx,
     opening-index =
       fri-sibling-index (fri-evidence-layer-len roots layer-idx)
       (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx),
     opening-value = yn,
     opening-path = yn-path')
  by blast
  show ?thesis
  by (rule generic-fri-authenticated-same-layer-opening-conflictI
      [OF round-bound round-bound' layer-bound auth-xn auth-yn])
    (use same-index neq in simp-all)
qed

```

definition *generic-fri-recorded-layer-chunk-authenticated*
 $:: 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow 'f \text{ list list list} \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow \text{bool}$

where
generic-fri-recorded-layer-chunk-authenticated roots query-idxs
 round-layers final-state round-idx layer-idx $\longleftrightarrow$
fri-layer-chunk-authenticated (roots ! layer-idx)
 (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
 (round-layers ! round-idx ! layer-idx) final-state

definition *generic-fri-authenticated-slot-recorded-chunk-gap*
 $:: 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow 'f \text{ list list list} \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where
generic-fri-authenticated-slot-recorded-chunk-gap roots query-idxs
 round-layers final-state $\longleftrightarrow$
 $(\exists \text{ round-idx layer-idx auth-chunk.}$
 $\text{round-idx} < \text{length query-idxs} \wedge$
 $\text{layer-idx} < \text{length roots} \wedge$
fri-layer-chunk-authenticated (roots ! layer-idx)
 (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
 auth-chunk final-state $\wedge$
 $\neg \text{generic-fri-recorded-layer-chunk-authenticated roots query-idxs}$
 $\text{round-layers final-state round-idx layer-idx})$

lemma *generic-fri-transcript-step-auth-missing-imp-authenticated-slot-recorded-chunk-gap*:
assumes step:
generic-fri-transcript-step-with-authenticated-chunk roots challenges
 query-idxs round-layers final-state round-idx layer-idx

and *missing*:
 $\neg$ *generic-fri-recorded-layer-chunk-authenticated roots query-idxs*
round-layers final-state round-idx layer-idx
shows
generic-fri-authenticated-slot-recorded-chunk-gap roots query-idxs
round-layers final-state
proof –
from *step obtain auth-chunk where evidence*:
fri-round-layer-evidence roots challenges query-idxs round-idx layer-idx
round-layers
and *auth*:
fri-layer-chunk-authenticated (roots ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
auth-chunk final-state
by (*rule generic-fri-transcript-step-with-authenticated-chunkE*)
from *evidence have round-bound: round-idx < length query-idxs*
and *layer-bound: layer-idx < length roots*
by (*auto elim: fri-round-layer-evidenceE*)
show *?thesis*
unfolding *generic-fri-authenticated-slot-recorded-chunk-gap-def*
by (*intro exI conjI*)
(rule round-bound, rule layer-bound, rule auth, rule missing)
qed

definition *generic-fri-sampled-same-layer-recorded-auth-gap*
 $:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where
generic-fri-sampled-same-layer-recorded-auth-gap roots challenges
query-idxs round-layers final-state $\longleftrightarrow$
 $(\exists \text{ round-idx round-idx' layer-idx.}$
 $\text{round-idx} < \text{length query-idxs} \wedge$
 $\text{round-idx'} < \text{length query-idxs} \wedge$
 $\text{layer-idx} < \text{length challenges} \wedge$
generic-fri-sampled-same-layer-opening-conflict roots challenges
query-idxs round-layers $\wedge$
 $(\neg \text{generic-fri-recorded-layer-chunk-authenticated roots query-idxs}$
 $\text{round-layers final-state round-idx layer-idx} \vee$
 $\neg \text{generic-fri-recorded-layer-chunk-authenticated roots query-idxs}$
 $\text{round-layers final-state round-idx' layer-idx}))$

definition *generic-fri-sampled-same-layer-authenticated-recorded-auth-gap*
 $:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where
generic-fri-sampled-same-layer-authenticated-recorded-auth-gap roots
challenges query-idxs round-layers final-state $\longleftrightarrow$
 $(\exists \text{ round-idx round-idx' layer-idx.}$

$round_idx < length\ query_idxs \wedge$
 $round_idx' < length\ query_idxs \wedge$
 $layer_idx < length\ challenges \wedge$
 $generic_fri_sampled_same_layer_opening_conflict\ roots\ challenges$
 $query_idxs\ round_layers \wedge$
 $((generic_fri_transcript_step_with_authenticated_chunk\ roots\ challenges$
 $query_idxs\ round_layers\ final_state\ round_idx\ layer_idx \wedge$
 $\neg generic_fri_recorded_layer_chunk_authenticated\ roots\ query_idxs$
 $round_layers\ final_state\ round_idx\ layer_idx) \vee$
 $(generic_fri_transcript_step_with_authenticated_chunk\ roots\ challenges$
 $query_idxs\ round_layers\ final_state\ round_idx'\ layer_idx \wedge$
 $\neg generic_fri_recorded_layer_chunk_authenticated\ roots\ query_idxs$
 $round_layers\ final_state\ round_idx'\ layer_idx)))$

lemma *generic-fri-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap:*

assumes

generic-fri-sampled-same-layer-authenticated-recorded-auth-gap roots
challenges query-idxs round-layers final-state

shows

generic-fri-authenticated-slot-recorded-chunk-gap roots query-idxs
round-layers final-state

proof —

from *assms* **obtain** $round_idx\ round_idx'\ layer_idx$ **where**

gap:

$(generic_fri_transcript_step_with_authenticated_chunk\ roots\ challenges$
 $query_idxs\ round_layers\ final_state\ round_idx\ layer_idx \wedge$
 $\neg generic_fri_recorded_layer_chunk_authenticated\ roots\ query_idxs$
 $round_layers\ final_state\ round_idx\ layer_idx) \vee$
 $(generic_fri_transcript_step_with_authenticated_chunk\ roots\ challenges$
 $query_idxs\ round_layers\ final_state\ round_idx'\ layer_idx \wedge$
 $\neg generic_fri_recorded_layer_chunk_authenticated\ roots\ query_idxs$
 $round_layers\ final_state\ round_idx'\ layer_idx)$

unfolding *generic-fri-sampled-same-layer-authenticated-recorded-auth-gap-def*

by *blast*

then show *?thesis*

proof

assume *left:*

$generic_fri_transcript_step_with_authenticated_chunk\ roots\ challenges$
 $query_idxs\ round_layers\ final_state\ round_idx\ layer_idx \wedge$
 $\neg generic_fri_recorded_layer_chunk_authenticated\ roots\ query_idxs$
 $round_layers\ final_state\ round_idx\ layer_idx$

show *?thesis*

by (*rule*

$generic_fri_transcript_step_auth_missing_imp_authenticated_slot_recorded_chunk_gap$
 $[of\ roots\ challenges\ query_idxs\ round_layers\ final_state\ round_idx$
 $layer_idx])$

(*use left in simp-all*)

next

assume *right:*

```

generic-fri-transcript-step-with-authenticated-chunk roots challenges
  query-idxs round-layers final-state round-idx' layer-idx  $\wedge$ 
 $\neg$  generic-fri-recorded-layer-chunk-authenticated roots query-idxs
  round-layers final-state round-idx' layer-idx
show ?thesis
by (rule
  generic-fri-transcript-step-auth-missing-imp-authenticated-slot-recorded-chunk-gap
  [of roots challenges query-idxs round-layers final-state round-idx'
    layer-idx])
  (use right in simp-all)
qed
qed

lemma generic-fri-recorded-auth-gap-imp-authenticated-recorded-auth-gap:
  assumes gap:
    generic-fri-sampled-same-layer-recorded-auth-gap roots challenges
    query-idxs round-layers final-state
  and auth-step:
     $\wedge$  round-idx layer-idx.
    round-idx < length query-idxs  $\implies$ 
    layer-idx < length challenges  $\implies$ 
    generic-fri-transcript-step-with-authenticated-chunk roots
    challenges query-idxs round-layers final-state round-idx layer-idx
  shows
    generic-fri-sampled-same-layer-authenticated-recorded-auth-gap roots
    challenges query-idxs round-layers final-state
proof -
  from gap obtain round-idx round-idx' layer-idx where
    round-bound: round-idx < length query-idxs
  and round-bound': round-idx' < length query-idxs
  and layer-bound: layer-idx < length challenges
  and conflict:
    generic-fri-sampled-same-layer-opening-conflict roots challenges
    query-idxs round-layers
  and missing:
     $\neg$  generic-fri-recorded-layer-chunk-authenticated roots query-idxs
    round-layers final-state round-idx layer-idx  $\vee$ 
     $\neg$  generic-fri-recorded-layer-chunk-authenticated roots query-idxs
    round-layers final-state round-idx' layer-idx
  unfolding generic-fri-sampled-same-layer-recorded-auth-gap-def
  by blast
show ?thesis
proof (cases
   $\neg$  generic-fri-recorded-layer-chunk-authenticated roots query-idxs
  round-layers final-state round-idx layer-idx)
case True
have step:
  generic-fri-transcript-step-with-authenticated-chunk roots challenges
  query-idxs round-layers final-state round-idx layer-idx

```



```

    by (rule auth-step[OF round-bound layer-bound])
  show ?thesis
    unfolding
      generic-fri-sampled-same-layer-authenticated-recorded-auth-gap-def
    by (intro exI conjI disjI1)
      (rule round-bound, rule round-bound', rule layer-bound,
       rule conflict, rule step, rule True)
next
case False
then have missing':
  ¬ generic-fri-recorded-layer-chunk-authenticated roots query-idxs
    round-layers final-state round-idx' layer-idx
  using missing by simp
have step:
  generic-fri-transcript-step-with-authenticated-chunk roots challenges
    query-idxs round-layers final-state round-idx' layer-idx
  by (rule auth-step[OF round-bound' layer-bound])
show ?thesis
  unfolding
    generic-fri-sampled-same-layer-authenticated-recorded-auth-gap-def
  by (intro exI conjI disjI2)
    (rule round-bound, rule round-bound', rule layer-bound,
     rule conflict, rule step, rule missing')
qed
qed

definition generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
:: 'f list ⇒ 'f list ⇒ nat list ⇒
  'f list list list ⇒ ('f, 'a) protocol-channel-scheme ⇒ bool
where
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
  roots challenges query-idxs round-layers final-state ⟷
(∃ round-idx round-idx' layer-idx
  xp xp-path xn xn-path yp yp-path yn yn-path.
  round-idx < length query-idxs ∧
  round-idx' < length query-idxs ∧
  layer-idx < length challenges ∧
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
  xp xp-path xn xn-path
  (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  (fri-evidence-next-value roots challenges query-idxs round-idx
    layer-idx xp xn))

```

$(\text{round-layers} ! \text{round-idx} ! \text{layer-idx}) \wedge$
 $\text{fri-layer-step-evidence}$
 $(\text{roots} ! \text{layer-idx})$
 $(\text{challenges} ! \text{layer-idx})$
 $(\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx}' \text{layer-idx})$
 $(2 \wedge \text{layer-idx})$
 $(\text{fri-sibling-index} (\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx}' \text{layer-idx}))$
 $\text{yp yp-path yn yn-path}$
 $(\text{fri-evidence-next-idx roots query-idxs round-idx}' \text{layer-idx})$
 $(\text{fri-evidence-next-value roots challenges query-idxs round-idx}'$
 $\text{layer-idx yp yn})$
 $(\text{round-layers} ! \text{round-idx}' ! \text{layer-idx}) \wedge$
 $\text{fri-layer-chunk-authenticated} (\text{roots} ! \text{layer-idx})$
 $(\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx})$
 $(\text{round-layers} ! \text{round-idx} ! \text{layer-idx}) \text{final-state} \wedge$
 $\text{fri-layer-chunk-authenticated} (\text{roots} ! \text{layer-idx})$
 $(\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx}' \text{layer-idx})$
 $(\text{round-layers} ! \text{round-idx}' ! \text{layer-idx}) \text{final-state} \wedge$
 $((\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx} =$
 $\text{fri-evidence-layer-idx roots query-idxs round-idx}' \text{layer-idx} \wedge$
 $xp \neq yp) \vee$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx} =$
 $\text{fri-sibling-index} (\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx}' \text{layer-idx}) \wedge$
 $xp \neq yn) \vee$
 $(\text{fri-sibling-index} (\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) =$
 $\text{fri-evidence-layer-idx roots query-idxs round-idx}' \text{layer-idx} \wedge$
 $xn \neq yp) \vee$
 $(\text{fri-sibling-index} (\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) =$
 $\text{fri-sibling-index} (\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx}' \text{layer-idx}) \wedge$
 $xn \neq yn)))$

lemma *generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-authenticated:*

assumes *conflict:*

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks

roots challenges query-idxs round-layers final-state

and *len-challenges: length challenges = length roots*

shows

generic-fri-authenticated-same-layer-opening-conflict roots query-idxs

final-state

proof —

from *conflict* **obtain** *round-idx round-idx' layer-idx*

xp xp -path xn xn -path yp yp -path yn yn -path **where**
 round-bound: round-idx < length query-idxs
and round-bound': round-idx' < length query-idxs
and layer-bound-ch: layer-idx < length challenges
and step:
 fri-layer-step-evidence
 (roots ! layer-idx)
 (challenges ! layer-idx)
 (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
 ($2^{\text{layer-idx}}$)
 (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
 xp xp -path xn xn -path
 (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
 (fri-evidence-next-value roots challenges query-idxs round-idx
 layer-idx xp xn)
 (round-layers ! round-idx ! layer-idx)
and step':
 fri-layer-step-evidence
 (roots ! layer-idx)
 (challenges ! layer-idx)
 (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
 ($2^{\text{layer-idx}}$)
 (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
 yp yp -path yn yn -path
 (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
 (fri-evidence-next-value roots challenges query-idxs round-idx'
 layer-idx yp yn)
 (round-layers ! round-idx' ! layer-idx)
and auth:
 fri-layer-chunk-authenticated (roots ! layer-idx)
 (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
 (round-layers ! round-idx ! layer-idx) final-state
and auth':
 fri-layer-chunk-authenticated (roots ! layer-idx)
 (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
 (round-layers ! round-idx' ! layer-idx) final-state
and cases:
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
 fri-evidence-layer-idx roots query-idxs round-idx' layer-idx $\wedge$
 $xp \neq yp$) $\vee$
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
 fri-sibling-index (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) $\wedge$

```

   $xp \neq yn) \vee$ 
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx  $\wedge$ 
     $xn \neq yp) \vee$ 
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)  $\wedge$ 
     $xn \neq yn$ )
unfolding
  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-def
by blast
have layer-bound: layer-idx < length roots
  using layer-bound-ch len-challenges by simp
have chunk:
  fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    xp xp-path xn xn-path (round-layers ! round-idx ! layer-idx)
  using fri-layer-step-evidenceD(4)[OF step] .
have chunk':
  fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    yp yp-path yn yn-path (round-layers ! round-idx' ! layer-idx)
  using fri-layer-step-evidenceD(4)[OF step'] .
from cases show ?thesis
proof
  assume c:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
      fri-evidence-layer-idx roots query-idxs round-idx' layer-idx  $\wedge$ 
       $xp \neq yp$ 
  show ?thesis
  by (rule generic-fri-authenticated-same-layer-opening-conflictI-base-base
    [OF round-bound round-bound' layer-bound auth auth' chunk chunk'])
    (use c in simp-all)
next
  assume rest:
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
      fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)  $\wedge$ 
       $xp \neq yn) \vee$ 
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
      fri-evidence-layer-idx roots query-idxs round-idx' layer-idx  $\wedge$ 
       $xn \neq yp) \vee$ 
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
      fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)  $\wedge$ 
       $xn \neq yn$ )
  then show ?thesis

```

```

proof
  assume c:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
      fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
      xp ≠ yn
  show ?thesis
    by (rule generic-fri-authenticated-same-layer-opening-conflictI-base-sibling
      [OF round-bound round-bound' layer-bound auth auth' chunk chunk'])
      (use c in simp-all)
  next
    assume rest':
      (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
          fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
          xn ≠ yp) ∨
      (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
          fri-sibling-index (fri-evidence-layer-len roots layer-idx)
            (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
            xn ≠ yn)
    then show ?thesis
    proof
      assume c:
        fri-sibling-index (fri-evidence-layer-len roots layer-idx)
          (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
            fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
            xn ≠ yp
      show ?thesis
        by (rule generic-fri-authenticated-same-layer-opening-conflictI-sibling-base
          [OF round-bound round-bound' layer-bound auth auth' chunk chunk'])
          (use c in simp-all)
    next
      assume c:
        fri-sibling-index (fri-evidence-layer-len roots layer-idx)
          (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
            fri-sibling-index (fri-evidence-layer-len roots layer-idx)
              (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
              xn ≠ yn
      show ?thesis
        by (rule generic-fri-authenticated-same-layer-opening-conflictI-sibling-sibling
          [OF round-bound round-bound' layer-bound auth auth' chunk chunk'])
          (use c in simp-all)
    qed
  qed
qed
qed

```

lemma generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-sampled:

assumes

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state

shows

generic-fri-sampled-same-layer-opening-conflict roots challenges
query-idxs round-layers

using *assms*

unfolding

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-def
generic-fri-sampled-same-layer-opening-conflict-def

by *blast*

lemma *generic-fri-sampled-same-layer-imp-authenticated-or-recorded-auth-gap:*

assumes *conflict:*

generic-fri-sampled-same-layer-opening-conflict roots challenges
query-idxs round-layers

shows

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state $\vee$
generic-fri-sampled-same-layer-recorded-auth-gap roots challenges
query-idxs round-layers final-state

proof —

from *conflict* **obtain** *round-idx round-idx' layer-idx*

xp xp-path xn xn-path yp yp-path yn yn-path **where**

round-bound: round-idx < length query-idxs

and *round-bound': round-idx' < length query-idxs*

and *layer-bound: layer-idx < length challenges*

and *step:*

fri-layer-step-evidence

(roots ! layer-idx)

(challenges ! layer-idx)

(fri-evidence-layer-len roots layer-idx)

(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)

(2 ^ layer-idx)

(fri-sibling-index (fri-evidence-layer-len roots layer-idx)

(fri-evidence-layer-idx roots query-idxs round-idx layer-idx))

xp xp-path xn xn-path

(fri-evidence-next-idx roots query-idxs round-idx layer-idx)

(fri-evidence-next-value roots challenges query-idxs round-idx
layer-idx xp xn)

(round-layers ! round-idx ! layer-idx)

and *step':*

fri-layer-step-evidence

(roots ! layer-idx)

(challenges ! layer-idx)

(fri-evidence-layer-len roots layer-idx)

(fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)

(2 ^ layer-idx)

(fri-sibling-index (fri-evidence-layer-len roots layer-idx)

```

    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
  yp yp-path yn yn-path
  (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
  (fri-evidence-next-value roots challenges query-idxs round-idx'
    layer-idx yp yn)
  (round-layers ! round-idx' ! layer-idx)
and cases:
  (fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
    xp ≠ yp) ∨
  (fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
    xp ≠ yn) ∨
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
    xn ≠ yp) ∨
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
    xn ≠ yn)
unfolding generic-fri-sampled-same-layer-opening-conflict-def
by blast
show ?thesis
proof (cases
  generic-fri-recorded-layer-chunk-authenticated roots query-idxs
    round-layers final-state round-idx layer-idx ∧
  generic-fri-recorded-layer-chunk-authenticated roots query-idxs
    round-layers final-state round-idx' layer-idx)
case True
have auth:
  fri-layer-chunk-authenticated (roots ! layer-idx)
  (fri-evidence-layer-len roots layer-idx)
  (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
  (round-layers ! round-idx ! layer-idx) final-state
using True
unfolding generic-fri-recorded-layer-chunk-authenticated-def
by simp
have auth':
  fri-layer-chunk-authenticated (roots ! layer-idx)
  (fri-evidence-layer-len roots layer-idx)
  (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
  (round-layers ! round-idx' ! layer-idx) final-state
using True
unfolding generic-fri-recorded-layer-chunk-authenticated-def
by simp
have

```

```

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
  roots challenges query-idxs round-layers final-state
unfolding
  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-def
by (intro exI conjI)
  (rule round-bound, rule round-bound', rule layer-bound, rule step,
   rule step', rule auth, rule auth', rule cases)
then show ?thesis by simp
next
case False
have
  generic-fri-sampled-same-layer-recorded-auth-gap roots challenges
  query-idxs round-layers final-state
unfolding generic-fri-sampled-same-layer-recorded-auth-gap-def
by (intro exI conjI)
  (rule round-bound, rule round-bound', rule layer-bound,
   rule conflict, use False in simp)
then show ?thesis by simp
qed
qed

lemma generic-fri-sampled-same-layer-auth-gap-imp-recorded-auth-gap:
assumes conflict:
  generic-fri-sampled-same-layer-opening-conflict roots challenges
  query-idxs round-layers
and no-auth:
  ¬ generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
  roots challenges query-idxs round-layers final-state
shows
  generic-fri-sampled-same-layer-recorded-auth-gap roots challenges
  query-idxs round-layers final-state
using generic-fri-sampled-same-layer-imp-authenticated-or-recorded-auth-gap
  [OF conflict, of final-state] no-auth
by blast

lemma generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-partial-merkle:
assumes conflict:
  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
  roots challenges query-idxs round-layers final-state
and len-challenges: length challenges = length roots
shows partial-merkle-inconsistency-bad s (Some (result, final-state))
proof –
have auth-conflict:
  generic-fri-authenticated-same-layer-opening-conflict roots query-idxs
  final-state
by (rule
  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-authenticated
  [OF conflict len-challenges])
show ?thesis

```


by (rule generic-fri-authenticated-same-layer-opening-conflict-imp-partial-merkle
 [OF auth-conflict])
 qed

definition generic-fri-sampled-assignment-conflict-with-authenticated-same-layer

:: 'f list $\Rightarrow$ 'f list $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$
 nat list $\Rightarrow$ 'f list list list $\Rightarrow$
 ('f, 'a) protocol-channel-scheme $\Rightarrow$ bool

where

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
 candidate-table roots challenges final-value query-idxs round-layers
 final-state $\longleftrightarrow$
 generic-fri-sampled-base-opening-conflict candidate-table roots
 challenges query-idxs round-layers $\vee$
 generic-fri-sampled-next-value-conflict roots challenges query-idxs
 round-layers $\vee$
 generic-fri-sampled-successor-opening-conflict roots challenges
 query-idxs round-layers $\vee$
 generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
 roots challenges query-idxs round-layers final-state $\vee$
 generic-fri-sampled-final-value-conflict roots challenges final-value
 query-idxs round-layers

definition generic-fri-sampled-assignment-conflict-without-same-layer

:: 'f list $\Rightarrow$ 'f list $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$
 nat list $\Rightarrow$ 'f list list list $\Rightarrow$ bool

where

generic-fri-sampled-assignment-conflict-without-same-layer
 candidate-table roots challenges final-value query-idxs round-layers
 $\longleftrightarrow$
 generic-fri-sampled-base-opening-conflict candidate-table roots
 challenges query-idxs round-layers $\vee$
 generic-fri-sampled-next-value-conflict roots challenges query-idxs
 round-layers $\vee$
 generic-fri-sampled-successor-opening-conflict roots challenges
 query-idxs round-layers $\vee$
 generic-fri-sampled-final-value-conflict roots challenges final-value
 query-idxs round-layers

lemma generic-fri-sampled-assignment-conflict-without-same-layer-split:

assumes

generic-fri-sampled-assignment-conflict-without-same-layer
 candidate-table roots challenges final-value query-idxs round-layers

shows

generic-fri-sampled-base-opening-conflict candidate-table roots
 challenges query-idxs round-layers $\vee$
 generic-fri-sampled-next-value-conflict roots challenges query-idxs
 round-layers $\vee$
 generic-fri-sampled-successor-opening-conflict roots challenges

query-idxs round-layers $\vee$
generic-fri-sampled-final-value-conflict roots challenges final-value
query-idxs round-layers
using *assms*
unfolding *generic-fri-sampled-assignment-conflict-without-same-layer-def*
by *blast*

lemma *generic-fri-sampled-assignment-conflict-without-same-layerI-base:*
assumes
generic-fri-sampled-base-opening-conflict candidate-table roots
challenges query-idxs round-layers
shows
generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers
using *assms*
unfolding *generic-fri-sampled-assignment-conflict-without-same-layer-def*
by *blast*

lemma *generic-fri-sampled-assignment-conflict-without-same-layerI-next:*
assumes
generic-fri-sampled-next-value-conflict roots challenges query-idxs
round-layers
shows
generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers
using *assms*
unfolding *generic-fri-sampled-assignment-conflict-without-same-layer-def*
by *blast*

lemma *generic-fri-sampled-assignment-conflict-without-same-layerI-successor:*
assumes
generic-fri-sampled-successor-opening-conflict roots challenges
query-idxs round-layers
shows
generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers
using *assms*
unfolding *generic-fri-sampled-assignment-conflict-without-same-layer-def*
by *blast*

lemma *generic-fri-sampled-assignment-conflict-without-same-layerI-final:*
assumes
generic-fri-sampled-final-value-conflict roots challenges final-value
query-idxs round-layers
shows
generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers
using *assms*
unfolding *generic-fri-sampled-assignment-conflict-without-same-layer-def*

by blast

lemma *generic-fri-sampled-assignment-conflict-without-same-layer-iff:*

generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers

$\longleftrightarrow$

generic-fri-sampled-base-opening-conflict candidate-table roots
challenges query-idxs round-layers $\vee$

generic-fri-sampled-next-value-conflict roots challenges query-idxs
round-layers $\vee$

generic-fri-sampled-successor-opening-conflict roots challenges
query-idxs round-layers $\vee$

generic-fri-sampled-final-value-conflict roots challenges final-value
query-idxs round-layers

unfolding *generic-fri-sampled-assignment-conflict-without-same-layer-def*

by *simp*

lemma *generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-split:*

assumes

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
candidate-table roots challenges final-value query-idxs round-layers
final-state

shows

generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers $\vee$
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state

using *assms*

unfolding

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-def
generic-fri-sampled-assignment-conflict-without-same-layer-def

by *blast*

lemma *generic-fri-sampled-assignment-conflict-with-authenticated-same-layerI-without:*

assumes

generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers

shows

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
candidate-table roots challenges final-value query-idxs round-layers
final-state

using *assms*

unfolding

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-def
generic-fri-sampled-assignment-conflict-without-same-layer-def

by *blast*

lemma *generic-fri-sampled-assignment-conflict-with-authenticated-same-layerI-same:*

assumes

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state
shows
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
candidate-table roots challenges final-value query-idxs round-layers
final-state
using *assms*
unfolding *generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-def*
by *blast*

lemma *generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-iff:*
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
candidate-table roots challenges final-value query-idxs round-layers
final-state $\longleftrightarrow$
generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers $\vee$
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state

proof
assume
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
candidate-table roots challenges final-value query-idxs round-layers
final-state
then show
generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers $\vee$
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state
by (*rule generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-split*)

next
assume
generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers $\vee$
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state
then show
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
candidate-table roots challenges final-value query-idxs round-layers
final-state
by (*elim disjE*)
 (*rule generic-fri-sampled-assignment-conflict-with-authenticated-same-layerI-without,*
assumption,
rule generic-fri-sampled-assignment-conflict-with-authenticated-same-layerI-same,
assumption)

qed

lemma *generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-imp-sampled:*
assumes
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer

candidate-table roots challenges final-value query-idxs round-layers
final-state
shows
generic-fri-sampled-assignment-conflict candidate-table roots challenges
final-value query-idxs round-layers
using *assms*
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-sampled
unfolding
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-def
generic-fri-sampled-assignment-conflict-def
by *blast*

lemma *generic-fri-sampled-assignment-conflict-imp-authenticated-or-same-layer-auth-gap:*

assumes
generic-fri-sampled-assignment-conflict candidate-table roots
challenges final-value query-idxs round-layers
shows
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
candidate-table roots challenges final-value query-idxs round-layers
final-state $\vee$
(generic-fri-sampled-same-layer-opening-conflict roots challenges
query-idxs round-layers $\wedge$
 $\neg$ *generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks*
roots challenges query-idxs round-layers final-state)
using *assms*
unfolding *generic-fri-sampled-assignment-conflict-def*
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-def
generic-fri-sampled-assignment-conflict-without-same-layer-def
by *blast*

lemma *generic-fri-authenticated-same-layer-branch-imp-partial-merkle:*

assumes
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state
and *length challenges = length roots*
shows *partial-merkle-inconsistency-bad s (Some (result, final-state))*
by (*rule*
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-partial-merkle
 $[OF\ assms]$)

lemma *generic-fri-sampled-same-layer-opening-conflict-imp-authenticated-if-recorded:*

assumes *conflict:*
generic-fri-sampled-same-layer-opening-conflict roots challenges
query-idxs round-layers
and *len-challenges: length challenges = length roots*
and *recorded:*
 $\bigwedge \text{round-idx layer-idx.}$
 $\text{round-idx} < \text{length query-idxs} \implies$
 $\text{layer-idx} < \text{length roots} \implies$

generic-fri-recorded-layer-chunk-authenticated roots query-idxs
round-layers final-state round-idx layer-idx
shows
generic-fri-authenticated-same-layer-opening-conflict roots query-idxs
final-state
proof –
from *conflict* **obtain** *round-idx round-idx' layer-idx*
xp xp-path xn xn-path yp yp-path yn yn-path **where**
round-bound: round-idx < length query-idxs
and *round-bound': round-idx' < length query-idxs*
and *layer-bound-ch: layer-idx < length challenges*
and *step:*
fri-layer-step-evidence
(roots ! layer-idx)
(challenges ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
(2 ^ layer-idx)
(fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
xp xp-path xn xn-path
(fri-evidence-next-idx roots query-idxs round-idx layer-idx)
(fri-evidence-next-value roots challenges query-idxs round-idx
layer-idx xp xn)
(round-layers ! round-idx ! layer-idx)
and *step':*
fri-layer-step-evidence
(roots ! layer-idx)
(challenges ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
(2 ^ layer-idx)
(fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
yp yp-path yn yn-path
(fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
(fri-evidence-next-value roots challenges query-idxs round-idx'
layer-idx yp yn)
(round-layers ! round-idx' ! layer-idx)
and *cases:*
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
xp ≠ yp) ∨
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
xp ≠ yn) ∨
(fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =

```

    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
    xn ≠ yp) ∨
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
      fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
      xn ≠ yn)
  unfolding generic-fri-sampled-same-layer-opening-conflict-def
    fri-evidence-layer-len-def fri-evidence-layer-idx-def
    fri-evidence-next-idx-def fri-evidence-next-value-def
  by blast
have layer-bound: layer-idx < length roots
  using layer-bound-ch len-challenges by simp
have chunk:
  fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    xp xp-path xn xn-path (round-layers ! round-idx ! layer-idx)
  using fri-layer-step-evidenceD(4)[OF step] .
have chunk':
  fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    yp yp-path yn yn-path (round-layers ! round-idx' ! layer-idx)
  using fri-layer-step-evidenceD(4)[OF step'] .
have auth:
  fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (round-layers ! round-idx ! layer-idx) final-state
  using recorded[OF round-bound layer-bound]
  unfolding generic-fri-recorded-layer-chunk-authenticated-def .
have auth':
  fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    (round-layers ! round-idx' ! layer-idx) final-state
  using recorded[OF round-bound' layer-bound]
  unfolding generic-fri-recorded-layer-chunk-authenticated-def .
from cases show ?thesis
proof
  assume c:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
      fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
    xp ≠ yp
  show ?thesis
  by (rule generic-fri-authenticated-same-layer-opening-conflictI-base-base
      [OF round-bound round-bound' layer-bound auth auth' chunk chunk'])
    (use c in simp-all)
next
  assume rest:
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
      fri-sibling-index (fri-evidence-layer-len roots layer-idx)

```

$(\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx}) \wedge$
 $xp \neq yn) \vee$
 $(\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) =$
 $\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx} \wedge$
 $xn \neq yp) \vee$
 $(\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) =$
 $\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx}) \wedge$
 $xn \neq yn)$
then show ?thesis
proof
assume c:
 $\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx} =$
 $\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx}) \wedge$
 $xp \neq yn$
show ?thesis
by (rule generic-fri-authenticated-same-layer-opening-conflictI-base-sibling
 $[\text{OF round-bound round-bound' layer-bound auth auth' chunk chunk'}])$
 (use c **in** simp-all)
next
assume rest':
 $(\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) =$
 $\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx} \wedge$
 $xn \neq yp) \vee$
 $(\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) =$
 $\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx}) \wedge$
 $xn \neq yn)$
then show ?thesis
proof
assume c:
 $\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) =$
 $\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx} \wedge$
 $xn \neq yp$
show ?thesis
by (rule generic-fri-authenticated-same-layer-opening-conflictI-sibling-base
 $[\text{OF round-bound round-bound' layer-bound auth auth' chunk chunk'}])$
 (use c **in** simp-all)
next
assume c:
 $\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) =$
 $\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$


```

      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
      xn ≠ yn
    show ?thesis
    by (rule generic-fri-authenticated-same-layer-opening-conflictI-sibling-sibling
        [OF round-bound round-bound' layer-bound auth auth' chunk chunk'])
        (use c in simp-all)
  qed
qed
qed
qed

```

lemma *accepted-fri-opening-transcript-trace-step-with-authenticated-chunk-at:*

assumes *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers

and *out-eq:* *out* = *Some (result, final-state)*

and *round-bound:* *i* < *rounds*

and *layer-bound:* *j* < *length trace-roots*

and *raw-layer:*

$\bigwedge \text{raw.}$

$0 < \text{fri-layer-lengths } (\text{length trace-roots}) (\text{clength} * \text{scale}) ! j \wedge$
 $\text{fri-layer-indices } (\text{length trace-roots}) (\text{index } (\text{to-nat raw}))$
 $(\text{clength} * \text{scale}) ! j <$
 $\text{fri-layer-lengths } (\text{length trace-roots}) (\text{clength} * \text{scale}) ! j$

shows

generic-fri-transcript-step-with-authenticated-chunk trace-roots
trace-bs query-idxs trace-round-layers final-state i j

proof —

have *step:*

fri-round-layer-evidence trace-roots trace-bs query-idxs i j
trace-round-layers

by (rule *accepted-fri-opening-transcript-trace-round-layer-evidence*
 $[OF \text{ fri-openings round-bound layer-bound}]$)

from *accepted-fri-opening-transcript-trace-selected-chunk-authenticated-at*
 $[OF \text{ fri-openings out-eq round-bound layer-bound raw-layer}]$

obtain *auth-chunk where auth:*

fri-layer-chunk-authenticated (trace-roots ! j)
 $(\text{fri-layer-lengths } (\text{length trace-roots}) (\text{clength} * \text{scale}) ! j)$
 $(\text{fri-layer-indices } (\text{length trace-roots}) (\text{query-idxs} ! i))$
 $(\text{clength} * \text{scale}) ! j)$
auth-chunk final-state

by *blast*

have *auth-compact:*

fri-layer-chunk-authenticated (trace-roots ! j)
 $(\text{fri-evidence-layer-len trace-roots } j)$
 $(\text{fri-evidence-layer-idx trace-roots query-idxs } i j)$
auth-chunk final-state

using *auth*

```

    unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
  by simp
show ?thesis
  by (rule generic-fri-transcript-step-with-authenticated-chunkI
      [OF step auth-compact])
qed

lemma accepted-fri-opening-transcript-composition-step-with-authenticated-chunk-at:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  and out-eq: out = Some (result, final-state)
  and round-bound: i < rounds
  and layer-bound: j < length composition-roots
  and raw-layer:
     $\bigwedge \text{raw.}$ 
    0 < fri-layer-lengths (length composition-roots) (clength * scale) ! j  $\wedge$ 
    fri-layer-indices (length composition-roots) (index (to-nat raw))
    (clength * scale) ! j <
    fri-layer-lengths (length composition-roots) (clength * scale) ! j
  shows
    generic-fri-transcript-step-with-authenticated-chunk composition-roots
    composition-bs query-idxs composition-round-layers final-state i j
proof -
  have step:
    fri-round-layer-evidence composition-roots composition-bs query-idxs i j
    composition-round-layers
  by (rule accepted-fri-opening-transcript-composition-round-layer-evidence
      [OF fri-openings round-bound layer-bound])
  from
    accepted-fri-opening-transcript-composition-selected-chunk-authenticated-at
    [OF fri-openings out-eq round-bound layer-bound raw-layer]
  obtain auth-chunk where auth:
    fri-layer-chunk-authenticated (composition-roots ! j)
    (fri-layer-lengths (length composition-roots) (clength * scale) ! j)
    (fri-layer-indices (length composition-roots) (query-idxs ! i)
     (clength * scale) ! j)
    auth-chunk final-state
  by blast
  have auth-compact:
    fri-layer-chunk-authenticated (composition-roots ! j)
    (fri-evidence-layer-len composition-roots j)
    (fri-evidence-layer-idx composition-roots query-idxs i j)
    auth-chunk final-state
  using auth
  unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
  by simp
show ?thesis

```

```

    by (rule generic-fri-transcript-step-with-authenticated-chunkI
        [OF step auth-compact])
qed

end

end

```

```

theory Soundness-FRI-Layer-Replay
  imports Soundness-FRI-Layer-Authenticated
begin

```

Narrow replay facts for selected FRI layer openings.

The first lemma is a generic selected-position decomposition for *mfold*. It avoids adding another proof by induction over the full FRI verifier layer in the already-large Merkle theory.

```

context soundness
begin

```

```

lemma fri-layer-lengths-prefix-nth:
  assumes  $j < n$ 
  shows  $\text{fri-layer-lengths } n \text{ len} ! j = \text{fri-layer-lengths } (\text{Suc } j) \text{ len} ! j$ 
  using assms
proof (induction  $j$  arbitrary:  $n \text{ len}$ )
  case 0
  then show ?case
    by (cases  $n$ ) simp-all
next
  case (Suc  $j$ )
  then obtain  $n'$  where  $n\text{-eq}: n = \text{Suc } n'$ 
    by (cases  $n$ ) auto
  have  $j\text{-bound}: j < n'$ 
    using Suc.prems unfolding  $n\text{-eq}$  by simp
  show ?case
    using Suc.IH[OF  $j\text{-bound}$ , of  $\text{len div } 2$ ]
    unfolding  $n\text{-eq}$  by simp
qed

```

```

lemma fri-layer-indices-prefix-nth:
  assumes  $j < n$ 
  shows
     $\text{fri-layer-indices } n \text{ idx len} ! j =$ 
     $\text{fri-layer-indices } (\text{Suc } j) \text{ idx len} ! j$ 
  using assms
proof (induction  $j$  arbitrary:  $n \text{ idx len}$ )
  case 0
  then show ?case
    by (cases  $n$ ) simp-all

```

```

next
  case (Suc j)
  then obtain n' where n-eq: n = Suc n'
    by (cases n) auto
  have j-bound: j < n'
    using Suc.premis unfolding n-eq by simp
  show ?case
    using Suc.IH[OF j-bound, of idx mod (len div 2) len div 2]
    unfolding n-eq by simp
qed

lemma fri-layer-lengths-prefix-same:
  assumes k-bound: k < j
  shows fri-layer-lengths (Suc j) len ! k = fri-layer-lengths j len ! k
  using fri-layer-lengths-prefix-nth[OF less-trans[OF k-bound lessI], of len]
    fri-layer-lengths-prefix-nth[OF k-bound, of len]
  by simp

lemma fri-layers-transcript-snoc:
  assumes prefix:
    fri-layers-transcript j len prefix-chunks prefix-chunk
  and last:
    fri-layer-opening-chunk (fri-layer-lengths (Suc j) len ! j)
    xp xp-path xn xn-path last-chunk
  shows
    fri-layers-transcript (Suc j) len (prefix-chunks @ [last-chunk])
    (prefix-chunk @ last-chunk)
  using prefix last
proof (induction j arbitrary: len prefix-chunks prefix-chunk)
  case 0
  then have chunks-eq: prefix-chunks = []
    and prefix-chunk-eq: prefix-chunk = []
    unfolding fri-layers-transcript-def by simp-all
  have last-fact:
    fri-layer-opening-chunk len xp xp-path xn xn-path last-chunk
    using 0.premis(2) by simp
  have last-open:
    ∃ yp yp-path yn yn-path.
      fri-layer-opening-chunk len yp yp-path yn yn-path last-chunk
  apply (rule exI[where x=xp])
  apply (rule exI[where x=xp-path])
  apply (rule exI[where x=xn])
  apply (rule exI[where x=xn-path])
  apply (rule last-fact)
  done
  show ?case
    unfolding chunks-eq prefix-chunk-eq fri-layers-transcript-def
    using last-open by simp
next

```

```

case (Suc j)
obtain first rest where chunks-eq: prefix-chunks = first # rest
  using Suc.premis(1) unfolding fri-layers-transcript-def
  by (cases prefix-chunks) auto
have first-open:
   $\exists$  yp yp-path yn yn-path.
    fri-layer-opening-chunk len yp yp-path yn yn-path first
  using Suc.premis(1) unfolding fri-layers-transcript-def chunks-eq
  by auto
have rest-layers:
  fri-layers-transcript j (len div 2) rest (List.concat rest)
  using Suc.premis(1) unfolding fri-layers-transcript-def chunks-eq
  by auto
have last-tail:
  fri-layer-opening-chunk
    (fri-layer-lengths (Suc j) (len div 2) ! j)
    xp xp-path xn xn-path last-chunk
  using Suc.premis(2) by simp
have tail:
  fri-layers-transcript (Suc j) (len div 2) (rest @ [last-chunk])
    (List.concat rest @ last-chunk)
  by (rule Suc.IH[OF rest-layers last-tail])
have prefix-chunk-eq: prefix-chunk = first @ List.concat rest
  using Suc.premis(1) unfolding fri-layers-transcript-def chunks-eq by simp
show ?case
  unfolding chunks-eq prefix-chunk-eq fri-layers-transcript-def
  using first-open tail unfolding fri-layers-transcript-def by auto
qed

```

```

lemma fri-layers-transcript-last-chunk-eq:
assumes layers:
  fri-layers-transcript (Suc j) len layer-chunks chunk
and prefix:
  fri-layers-transcript j len prefix-chunks prefix-chunk
and last:
  fri-layer-opening-chunk (fri-layer-lengths (Suc j) len ! j)
    xp xp-path xn xn-path last-chunk
and chunk-eq: chunk = prefix-chunk @ last-chunk
shows layer-chunks ! j = last-chunk
proof –
have combined:
  fri-layers-transcript (Suc j) len (prefix-chunks @ [last-chunk])
    (prefix-chunk @ last-chunk)
  by (rule fri-layers-transcript-snoc[OF prefix last])
have layers-eq: layer-chunks = prefix-chunks @ [last-chunk]
  by (rule fri-layers-transcript-unique
    [OF layers combined[unfolded chunk-eq[symmetric]]])
have len-prefix: length prefix-chunks = j
  using prefix unfolding fri-layers-transcript-def by simp

```

```

show ?thesis
  using layers-eq len-prefix by (simp add: nth-append)
qed

lemma generic-fri-round-forced-next-value-from-step-evidence:
  assumes step:
    fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
    xp xp-path xn xn-path
    (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
    v
    (round-layers ! round-idx ! layer-idx)
  shows
    generic-fri-round-forced-next-value roots challenges query-idxs
    round-layers round-idx layer-idx v
  using step
  unfolding generic-fri-round-forced-next-value-def by blast

lemma mfold-outcome-decomp-at:
  assumes outcome:
    Some (out, t) ∈ set-dist (execute (mfold a ms) s)
    and j-bound: j < length ms
  shows
    ∃ a-j s-j out-j s-suc.
      Some (a-j, s-j) ∈ set-dist (execute (mfold a (take j ms)) s) ∧
      Some (out-j, s-suc) ∈ set-dist (execute ((ms ! j) a-j) s-j) ∧
      Some (out, t) ∈
        set-dist (execute (mfold out-j (drop (Suc j) ms)) s-suc)
  using outcome j-bound
proof (induction j arbitrary: a ms s)
case 0
  then obtain m ms' where ms-eq: ms = m # ms'
  by (cases ms) auto
from 0.premis(1)[unfolded ms-eq]
obtain out-j s-suc where
  head: Some (out-j, s-suc) ∈ set-dist (execute (m a) s)
  and tail:
    Some (out, t) ∈ set-dist (execute (mfold out-j ms') s-suc)
  by (auto elim!: set-dist-bindE)
have prefix: Some (a, s) ∈ set-dist (execute (mfold a (take 0 ms)) s)
  by simp
have selected-orig:
  Some (out-j, s-suc) ∈ set-dist (execute ((ms ! 0) a) s)

```

```

    using head unfolding ms-eq by simp
  have suffix-orig:
    Some (out, t) ∈
      set-dist (execute (mfold out-j (drop (Suc 0) ms)) s-suc)
    using tail unfolding ms-eq by simp
  show ?case
    by (intro exI[of - a] exI[of - s] exI[of - out-j]
        exI[of - s-suc] conjI)
      (use prefix selected-orig suffix-orig in simp-all)
next
  case (Suc j)
  then obtain m ms' where ms-eq: ms = m # ms'
    by (cases ms) auto
  from Suc.prem1(1)[unfolded ms-eq]
  obtain out1 s1 where
    head: Some (out1, s1) ∈ set-dist (execute (m a) s)
    and tail:
      Some (out, t) ∈ set-dist (execute (mfold out1 ms') s1)
    by (auto elim!: set-dist-bindE)
  have j-bound': j < length ms'
    using Suc.prem2(2) unfolding ms-eq by simp
  from Suc.IH[OF tail j-bound']
  obtain a-j s-j out-j s-suc where
    prefix-tail:
      Some (a-j, s-j) ∈
        set-dist (execute (mfold out1 (take j ms')) s1)
    and selected:
      Some (out-j, s-suc) ∈
        set-dist (execute ((ms' ! j) a-j) s-j)
    and suffix:
      Some (out, t) ∈
        set-dist (execute (mfold out-j (drop (Suc j) ms')) s-suc)
    by blast
  have prefix-unfolded:
    Some (a-j, s-j) ∈
      set-dist (execute ((m a) >>= (λx. mfold x (take j ms')))) s)
    by (rule-tac x=out1 and t=s1 in set-dist-bindI)
      (use head prefix-tail in simp-all)
  have prefix-mfold:
    Some (a-j, s-j) ∈
      set-dist (execute (mfold a (m # take j ms')) s)
    using prefix-unfolded by simp
  have prefix:
    Some (a-j, s-j) ∈
      set-dist (execute (mfold a (take (Suc j) ms)) s)
    using prefix-mfold unfolding ms-eq by simp
  have selected-orig:
    Some (out-j, s-suc) ∈
      set-dist (execute ((ms ! Suc j) a-j) s-j)

```

```

using selected unfolding ms-eq by simp
have suffix-orig:
  Some (out, t) ∈
    set-dist (execute (mfold out-j (drop (Suc (Suc j)) ms)) s-suc)
using suffix unfolding ms-eq by simp
show ?case
  by (intro exI[of - a-j] exI[of - s-j] exI[of - out-j]
    exI[of - s-suc] conjI)
    (use prefix selected-orig suffix-orig in simp-all)
qed

lemma mfold-fri-layer-openings-decomp-at:
fixes s t :: ('f, 'a') protocol-channel-scheme
assumes outcome:
  Some (out, t) ∈
    set-dist
      (execute
        (mfold (idx, x, len, pw) (map fri-layer-opening-step bfs)) s)
and j-bound: j < length bfs
obtains idx-j x-j len-j pw-j out-j s-j s-suc where
  Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
    set-dist
      (execute
        (mfold (idx, x, len, pw)
          (take j (map fri-layer-opening-step bfs)))) s)
  Some (out-j, s-suc) ∈
    set-dist
      (execute
        (fri-layer-opening-step (bfs ! j)
          (idx-j, x-j, len-j, pw-j)) s-j)
  Some (out, t) ∈
    set-dist
      (execute
        (mfold out-j (drop (Suc j) (map fri-layer-opening-step bfs)))
          s-suc)

proof –
have mapped-bound: j < length (map fri-layer-opening-step bfs)
using j-bound by simp
from mfold-outcome-decomp-at[OF outcome mapped-bound]
obtain a-j s-j out-j s-suc where
  prefix:
    Some (a-j, s-j) ∈
      set-dist
        (execute
          (mfold (idx, x, len, pw)
            (take j (map fri-layer-opening-step bfs)))) s)
and selected:
  Some (out-j, s-suc) ∈
    set-dist

```



```

      (execute
        ((map fri-layer-opening-step bfs ! j) a-j) s-j)
    and suffix:
      Some (out, t) ∈
        set-dist
          (execute
            (mfold out-j (drop (Suc j) (map fri-layer-opening-step bfs)))
            s-suc)
        by blast
  obtain idx-j x-j len-j pw-j where a-j-eq:
    a-j = (idx-j, x-j, len-j, pw-j)
  by (cases a-j)
  have selected':
    Some (out-j, s-suc) ∈
      set-dist
        (execute
          (fri-layer-opening-step (bfs ! j)
            (idx-j, x-j, len-j, pw-j)) s-j)
    using selected j-bound unfolding a-j-eq by simp
  show ?thesis
    by (rule that[OF prefix[unfolded a-j-eq] selected' suffix])
qed

```

```

lemma mfold-fri-layer-openings-prefix-idx-len:
  fixes s s-j :: ('f, 'a) protocol-channel-scheme
  assumes prefix:
    Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
      set-dist
        (execute
          (mfold (idx, x, len, pw)
            (take j (map fri-layer-opening-step bfs))) s)
  and j-bound: j ≤ length bfs
  shows
    idx-j = fri-layer-indices (Suc j) idx len ! j ∧
    len-j = fri-layer-lengths (Suc j) len ! j
  using prefix j-bound
proof (induction j arbitrary: idx x len pw bfs s idx-j x-j len-j pw-j s-j)
  case 0
  then show ?case
    by simp
next
  case (Suc j)
  then obtain bf bfs' where bfs-eq: bfs = bf # bfs'
    by (cases bfs) auto
  obtain b rt where bf-eq: bf = (b, rt)
    by (cases bf)
  from Suc.prem1[unfolded bfs-eq]
  obtain out1 s1 where
    head:

```

```

    Some (out1, s1) ∈
      set-dist
        (execute (fri-layer-opening-step bf (idx, x, len, pw)) s)
  and tail:
    Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
      set-dist
        (execute
          (mfold out1 (take j (map fri-layer-opening-step bfs'))) s1)
  by (auto elim!: set-dist-bindE)
from fri-layer-opening-step-outcome[OF head[unfolded bf-eq]]
obtain x' xp xp-path xn xn-path where out1-eq:
  out1 = (idx mod (len div 2), x', len div 2, pw + pw)
  by blast
have j-bound': j ≤ length bfs'
  using Suc.premis(2) unfolding bfs-eq by simp
have tail-shape:
  idx-j =
    fri-layer-indices (Suc j) (idx mod (len div 2)) (len div 2) ! j ∧
  len-j = fri-layer-lengths (Suc j) (len div 2) ! j
  by (rule Suc.IH[OF tail[unfolded out1-eq] j-bound'])
show ?case
  using tail-shape by simp
qed

lemma mfold-fri-layer-openings-prefix-pow:
  fixes s s-j :: ('f, 'a) protocol-channel-scheme
  assumes prefix:
    Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
      set-dist
        (execute
          (mfold (idx, x, len, pw)
            (take j (map fri-layer-opening-step bfs)))) s)
  and j-bound: j ≤ length bfs
  shows pw-j = pw * 2 ^ j
  using prefix j-bound
proof (induction j arbitrary: idx x len pw bfs s idx-j x-j len-j pw-j s-j)
case 0
  then show ?case
    by simp
next
case (Suc j)
  then obtain bf bfs' where bfs-eq: bfs = bf # bfs'
    by (cases bfs) auto
  obtain b rt where bf-eq: bf = (b, rt)
    by (cases bf)
  from Suc.premis(1)[unfolded bfs-eq]
  obtain out1 s1 where
    head:
      Some (out1, s1) ∈

```

```

    set-dist
    (execute (fri-layer-opening-step bf (idx, x, len, pw)) s)
and tail:
  Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
    set-dist
    (execute
      (mfold out1 (take j (map fri-layer-opening-step bfs'))) s1)
  by (auto elim!: set-dist-bindE)
from fri-layer-opening-step-outcome[OF head[unfolded bf-eq]]
obtain x' xp xp-path xn xn-path where out1-eq:
  out1 = (idx mod (len div 2), x', len div 2, pw + pw)
  by blast
have j-bound': j ≤ length bfs'
  using Suc.premis(2) unfolding bfs-eq by simp
have tail-pow: pw-j = (pw + pw) * 2 ^ j
  by (rule Suc.IH[OF tail[unfolded out1-eq] j-bound'])
then show ?case
  by simp
qed

```

lemma *mfold-fri-layer-openings-prefix-pow-full:*
fixes $s \ s\text{-}j :: ('f, 'a) \text{protocol-channel-scheme}$
assumes *prefix:*
 Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
 set-dist
 (execute
 (mfold (idx, x, len, pw)
 (take j (map fri-layer-opening-step bfs')))) s)
and *j-bound:* $j < \text{length } \text{bfs}$
shows $\text{pw-j} = \text{pw} * 2^j$
by (rule *mfold-fri-layer-openings-prefix-pow*[OF *prefix*])
 (use *j-bound* **in** *simp*)

lemma *mfold-fri-layer-openings-prefix-idx-len-full:*
fixes $s \ s\text{-}j :: ('f, 'a) \text{protocol-channel-scheme}$
assumes *prefix:*
 Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
 set-dist
 (execute
 (mfold (idx, x, len, pw)
 (take j (map fri-layer-opening-step bfs')))) s)
and *j-bound:* $j < \text{length } \text{bfs}$
shows
 $\text{idx-j} = \text{fri-layer-indices } (\text{length } \text{bfs}) \ \text{idx} \ \text{len} \ ! \ j \wedge$
 $\text{len-j} = \text{fri-layer-lengths } (\text{length } \text{bfs}) \ \text{len} \ ! \ j$
proof –
have *shape:*
 $\text{idx-j} = \text{fri-layer-indices } (\text{Suc } j) \ \text{idx} \ \text{len} \ ! \ j \wedge$
 $\text{len-j} = \text{fri-layer-lengths } (\text{Suc } j) \ \text{len} \ ! \ j$

```

    by (rule mfold-fri-layer-openings-prefix-idx-len[OF prefix])
      (use j-bound in simp)
  show ?thesis
    using shape fri-layer-indices-prefix-nth[OF j-bound, of idx len]
      fri-layer-lengths-prefix-nth[OF j-bound, of len]
    by simp
qed

```

lemma *fri-layer-opening-selected-step-authenticated-full*:

fixes *s s-j s-suc* :: ('f, 'a) protocol-channel-scheme

assumes *prefix*:

Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
set-dist
(execute
(mfold (idx, x, len, pw)
(take j (map fri-layer-opening-step bfs))) s)

and *selected*:

Some (out-j, s-suc) ∈
set-dist
(execute
(fri-layer-opening-step (bfs ! j)
(idx-j, x-j, len-j, pw-j)) s-j)

and *j-bound*: *j < length bfs*

and *layer-len-pos*:

0 < fri-layer-lengths (length bfs) len ! j

and *layer-idx-bound*:

fri-layer-indices (length bfs) idx len ! j <
fri-layer-lengths (length bfs) len ! j

obtains *chunk* **where**

fri-layer-chunk-authenticated (snd (bfs ! j))
(fri-layer-lengths (length bfs) len ! j)
(fri-layer-indices (length bfs) idx len ! j)
chunk s-suc

proof –

have *shape*:

idx-j = fri-layer-indices (length bfs) idx len ! j ∧
len-j = fri-layer-lengths (length bfs) len ! j

by (rule *mfold-fri-layer-openings-prefix-idx-len-full*
 $[OF\ prefix\ j-bound]$)

obtain *b rt* **where** *bf-eq*: *bfs ! j = (b, rt)*

by (cases *bfs ! j*)

from *fri-layer-opening-step-authenticated-chunk-pred*
 $[OF\ -\ selected[unfolded\ bf-eq]]$

obtain *chunk* **where** *auth*:

fri-layer-chunk-authenticated rt len-j idx-j chunk s-suc
using *layer-len-pos layer-idx-bound shape*
by *blast*

show *?thesis*

by (rule *that*[*of chunk*])

```

      (use auth shape bf-eq in simp)
qed

lemma receive-query-commits-decomp-at:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (out, t) ∈
      set-dist
        (execute
          (mfold (idx, x, len, pw) (receive-query-commits bfs)) s)
  and j-bound: j < length bfs
  obtains idx-j x-j len-j pw-j out-j s-j s-suc where
    Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
      set-dist
        (execute
          (mfold (idx, x, len, pw)
            (take j (map fri-layer-opening-step bfs)))) s)
    Some (out-j, s-suc) ∈
      set-dist
        (execute
          (fri-layer-opening-step (bfs ! j)
            (idx-j, x-j, len-j, pw-j)) s-j)
    Some (out, t) ∈
      set-dist
        (execute
          (mfold out-j (drop (Suc j) (map fri-layer-opening-step bfs)))
            s-suc))
  proof -
    have map-eq: receive-query-commits bfs = map fri-layer-opening-step bfs
    unfolding receive-query-commits-def fri-layer-opening-step-def
    by (simp add: fun-eq-iff split: prod.splits)
    show ?thesis
    by (rule mfold-fri-layer-openings-decomp-at
      [OF outcome[unfolded map-eq] j-bound that])
  qed

lemma receive-query-commits-prefix-idx-len:
  fixes s s-j :: ('f, 'a) protocol-channel-scheme
  assumes prefix:
    Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
      set-dist
        (execute
          (mfold (idx, x, len, pw)
            (take j (map fri-layer-opening-step bfs)))) s)
  and j-bound: j ≤ length bfs
  shows
    idx-j = fri-layer-indices (Suc j) idx len ! j ∧
    len-j = fri-layer-lengths (Suc j) len ! j
  by (rule mfold-fri-layer-openings-prefix-idx-len[OF prefix j-bound])

```

lemma *receive-query-commits-prefix-pow*:
fixes $s \ s\text{-}j :: ('f, 'a) \text{protocol-channel-scheme}$
assumes *prefix*:
Some $((idx\text{-}j, x\text{-}j, len\text{-}j, pw\text{-}j), s\text{-}j) \in$
set-dist
(execute
(mfold (idx, x, len, pw)
(take j (map fri-layer-opening-step bfs)))) s)
and $j\text{-bound}: j \leq \text{length } bfs$
shows $pw\text{-}j = pw * 2^j$
by (rule *mfold-fri-layer-openings-prefix-pow*[*OF prefix j-bound*])

lemma *receive-query-commits-prefix-pow-full*:
fixes $s \ s\text{-}j :: ('f, 'a) \text{protocol-channel-scheme}$
assumes *prefix*:
Some $((idx\text{-}j, x\text{-}j, len\text{-}j, pw\text{-}j), s\text{-}j) \in$
set-dist
(execute
(mfold (idx, x, len, pw)
(take j (map fri-layer-opening-step bfs)))) s)
and $j\text{-bound}: j < \text{length } bfs$
shows $pw\text{-}j = pw * 2^j$
by (rule *mfold-fri-layer-openings-prefix-pow-full*[*OF prefix j-bound*])

lemma *receive-query-commits-prefix-idx-len-full*:
fixes $s \ s\text{-}j :: ('f, 'a) \text{protocol-channel-scheme}$
assumes *prefix*:
Some $((idx\text{-}j, x\text{-}j, len\text{-}j, pw\text{-}j), s\text{-}j) \in$
set-dist
(execute
(mfold (idx, x, len, pw)
(take j (map fri-layer-opening-step bfs)))) s)
and $j\text{-bound}: j < \text{length } bfs$
shows
 $idx\text{-}j = \text{fri-layer-indices } (\text{length } bfs) \ idx \ len \ ! \ j \wedge$
 $len\text{-}j = \text{fri-layer-lengths } (\text{length } bfs) \ len \ ! \ j$
by (rule *mfold-fri-layer-openings-prefix-idx-len-full*[*OF prefix j-bound*])

lemma *fri-layer-opening-step-outcome-evidence*:
fixes $s \ t :: ('f, 'a) \text{protocol-channel-scheme}$
assumes *outcome*:
Some $((next\text{-}idx, next\text{-}value, next\text{-}len, next\text{-}pow), t) \in$
set-dist (execute (fri-layer-opening-step (b, rt) (idx, x, len, pw)) s)
obtains $xp \ xp\text{-}path \ xn \ xn\text{-}path$ **where**
 $xp = x$
 $next\text{-}idx = idx \bmod (\text{length } bfs)$
 $next\text{-}len = \text{length } bfs$
 $next\text{-}pow = pw + pw$

```

    fri-layer-step-evidence rt b len idx pw
    (fri-sibling-index len idx) xp xp-path xn xn-path
    next-idx next-value ([xp] @ xp-path @ [xn] @ xn-path)
proof –
  from fri-layer-opening-step-outcome[OF outcome]
  obtain xp xp-path xn xn-path x' where out-eq:
    (next-idx, next-value, next-len, next-pow) =
    (idx mod (len div 2), x', len div 2, pw + pw)
  and xp-eq: xp = x
  and x'-eq:
    x' = fri-fold-value b xp xn
    (fri-fold-denominator ((h ^ idx) * shift) pw)
  and chunk:
    fri-layer-opening-chunk len xp xp-path xn xn-path
    ([xp] @ xp-path @ [xn] @ xn-path)
  by blast
  have next-idx-eq: next-idx = idx mod (len div 2)
  using out-eq by simp
  have next-value-eq: next-value = x'
  using out-eq by simp
  have next-len-eq: next-len = len div 2
  using out-eq by simp
  have next-pow-eq: next-pow = pw + pw
  using out-eq by simp
  have evidence:
    fri-layer-step-evidence rt b len idx pw
    (fri-sibling-index len idx) xp xp-path xn xn-path
    next-idx next-value ([xp] @ xp-path @ [xn] @ xn-path)
  by (rule fri-layer-step-evidenceI)
    (use next-idx-eq next-value-eq x'-eq chunk in simp-all)
  show ?thesis
  by (rule that[OF xp-eq next-idx-eq next-len-eq next-pow-eq evidence])
qed

```

```

lemma receive-query-commits-last-step-evidence:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some ((out-idx, out-value, out-len, out-pow), t) ∈
    set-dist
    (execute
      (mfold (idx, x, len, pw) (receive-query-commits bfs)) s)
  and nonempty: 0 < length bfs
  obtains j idx-j x-j len-j pw-j b rt xp xp-path xn xn-path s-j
    s-suc where
    j = length bfs – 1
    j < length bfs
    bfs ! j = (b, rt)
    Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
    set-dist

```

```

      (execute
        (mfold (idx, x, len, pw)
          (take j (map fri-layer-opening-step bfs))) s)
    Some ((out-idx, out-value, out-len, out-pow), s-suc) ∈
      set-dist
      (execute
        (fri-layer-opening-step (b, rt)
          (idx-j, x-j, len-j, pw-j)) s-j)
    t = s-suc
    idx-j = fri-layer-indices (length bfs) idx len ! j
    len-j = fri-layer-lengths (length bfs) len ! j
    pw-j = pw * 2 ^ j
    fri-layer-step-evidence rt b len-j idx-j pw-j
      (fri-sibling-index len-j idx-j) xp xp-path xn xn-path
      out-idx out-value ([xp] @ xp-path @ [xn] @ xn-path)
proof –
  let ?j = length bfs – 1
  have j-bound: ?j < length bfs
    using nonempty by simp
  show ?thesis
proof (rule receive-query-commits-decomp-at[OF outcome j-bound])
  fix idx-j x-j len-j pw-j out-j s-j s-suc
  assume prefix:
    Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
      set-dist
      (execute
        (mfold (idx, x, len, pw)
          (take ?j (map fri-layer-opening-step bfs))) s)
  and selected:
    Some (out-j, s-suc) ∈
      set-dist
      (execute
        (fri-layer-opening-step (bfs ! ?j)
          (idx-j, x-j, len-j, pw-j)) s-j)
  and suffix:
    Some ((out-idx, out-value, out-len, out-pow), t) ∈
      set-dist
      (execute
        (mfold out-j
          (drop (Suc ?j) (map fri-layer-opening-step bfs))) s-suc)
  have drop-empty:
    drop (Suc ?j) (map fri-layer-opening-step bfs) = []
    using nonempty by simp
  have out-j-eq: out-j = (out-idx, out-value, out-len, out-pow)
    and t-eq: t = s-suc
    using suffix unfolding drop-empty by simp-all
  obtain b rt where bf-eq: bfs ! ?j = (b, rt)
    by (cases bfs ! ?j)
  have shape:

```



```

    idx-j = fri-layer-indices (length bfs) idx len ! ?j ∧
    len-j = fri-layer-lengths (length bfs) len ! ?j
  by (rule receive-query-commits-prefix-idx-len-full
    [OF prefix j-bound])
  have pow: pw-j = pw * 2 ^ ?j
  by (rule receive-query-commits-prefix-pow-full[OF prefix j-bound])
  show ?thesis
  proof (rule fri-layer-opening-step-outcome-evidence
    [OF selected[unfolded bf-eq out-j-eq]])
    fix xp xp-path xn xn-path
    assume
      xp = x-j
      out-idx = idx-j mod (len-j div 2)
      out-len = len-j div 2
      out-pow = pw-j + pw-j
    and step:
      fri-layer-step-evidence rt b len-j idx-j pw-j
      (fri-sibling-index len-j idx-j) xp xp-path xn xn-path
      out-idx out-value ([xp] @ xp-path @ [xn] @ xn-path)
    show ?thesis
    by (rule that[OF refl j-bound bf-eq prefix
      selected[unfolded bf-eq out-j-eq] t-eq conjunct1[OF shape]
      conjunct2[OF shape] pow step])
  qed
qed
qed

lemma verifier-query-round-after-index-trace-fri-last-step-evidence:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some ((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s)
  and nonempty: 0 < length f-fl
  and raw-layer:
    ∧ raw'.
      0 < fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1) ∧
      fri-layer-indices (length f-fl) (index (to-nat raw'))
      (clength * scale) ! (length f-fl - 1) <
      fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1)
  obtains actual-chunk axp axp-path axn axn-path where
    fri-layer-chunk-authenticated
      (snd (f-fl ! (length f-fl - 1)))
      (fri-layer-lengths (length f-fl) (clength * scale) !
        (length f-fl - 1))

```

```

(fri-layer-indices (length f-fl) (index (to-nat raw))
 (clength * scale) ! (length f-fl - 1))
actual-chunk t
fri-layer-step-evidence
(snd (f-fl ! (length f-fl - 1)))
(fst (f-fl ! (length f-fl - 1)))
(fri-layer-lengths (length f-fl) (clength * scale) !
 (length f-fl - 1))
(fri-layer-indices (length f-fl) (index (to-nat raw))
 (clength * scale) ! (length f-fl - 1))
(2 ^ (length f-fl - 1))
(fri-sibling-index
 (fri-layer-lengths (length f-fl) (clength * scale) !
 (length f-fl - 1))
 (fri-layer-indices (length f-fl) (index (to-nat raw))
 (clength * scale) ! (length f-fl - 1)))
axp axp-path axn axn-path
((fri-layer-indices (length f-fl) (index (to-nat raw))
 (clength * scale) ! (length f-fl - 1)) mod
 ((fri-layer-lengths (length f-fl) (clength * scale) !
 (length f-fl - 1)) div 2))
f-final actual-chunk
proof —
show ?thesis
proof (rule verifier-query-round-after-index-final-checks[OF outcome])
fix idx fv s1 f-i f-len f-pow s2 c-i c-len c-pow s4
assume idx-eq: idx = index (to-nat raw)
and trace-fri:
  Some ((f-i, f-final, f-len, f-pow), s2) ∈
    set-dist
    (execute
      (mfold (idx, hd fv, clength * scale, 1)
        (receive-query-commits f-fl)) s1)
and comp-fri:
  Some ((c-i, final, c-len, c-pow), s4) ∈
    set-dist
    (execute
      (mfold (idx, cp-eval as fv (h ^ idx * shift),
        clength * scale, 1)
        (receive-query-commits fl)) s2)
and t-eq: t = s4
let ?j = length f-fl - 1
let ?len = fri-layer-lengths (length f-fl) (clength * scale) ! ?j
let ?idx =
  fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! ?j
have len-pos: 0 < ?len
using raw-layer[of raw] by simp
have idx-bound: ?idx < ?len

```

```

using raw-layer[of raw] by simp
show ?thesis
proof (rule receive-query-commits-last-step-evidence
  [OF trace-fri[unfolded idx-eq] nonempty])
fix j idx-j x-j len-j pw-j b rt xp xp-path xn xn-path s-j s-suc
assume j-eq: j = ?j
and bf-eq: f-fl ! j = (b, rt)
and selected:
  Some ((f-i, f-final, f-len, f-pow), s-suc) ∈
    set-dist
    (execute
      (fri-layer-opening-step (b, rt)
        (idx-j, x-j, len-j, pw-j)) s-j)
and s2-eq: s2 = s-suc
and idx-j-eq:
  idx-j =
    fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! j
and len-j-eq:
  len-j =
    fri-layer-lengths (length f-fl) (clength * scale) ! j
and pw-j-eq: pw-j = 1 * 2 ^ j
have len-pos-j: 0 < len-j
using len-pos len-j-eq j-eq by simp
have idx-bound-j: idx-j < len-j
using idx-bound idx-j-eq len-j-eq j-eq by simp
show ?thesis
proof (rule fri-layer-opening-step-outcome-authenticated
  [OF len-pos-j idx-bound-j selected])
fix axp axp-path axn axn-path next-value actual-chunk
assume selected-out:
  (f-i, f-final, f-len, f-pow) =
    (idx-j mod (len-j div 2), next-value, len-j div 2,
      pw-j + pw-j)
and next-value-eq:
  next-value =
    fri-fold-value b axp axn
    (fri-fold-denominator ((h ^ idx-j) * shift) pw-j)
and chunk:
  fri-layer-opening-chunk len-j axp axp-path axn axn-path
  actual-chunk
and axp-auth:
  authenticated-opening-in s-suc
  (opening-root = rt, opening-length = len-j,
    opening-index = idx-j, opening-value = axp,
    opening-path = axp-path)
and axn-auth:
  authenticated-opening-in s-suc
  (opening-root = rt, opening-length = len-j,

```

```

      opening-index = fri-sibling-index len-j idx-j,
      opening-value = axn, opening-path = axn-path))
have f-final-eq: f-final = next-value
  using selected-out by simp
have auth-s2:
  fri-layer-chunk-authenticated rt len-j idx-j actual-chunk s2
  unfolding s2-eq
  by (rule fri-layer-chunk-authenticatedI[OF chunk axp-auth axn-auth])
have s2-t: s2 ≤ t
  using receive-query-commits-mfold-hash-extends[OF comp-fri] t-eq
  by simp
have auth-t:
  fri-layer-chunk-authenticated rt len-j idx-j actual-chunk t
  by (rule fri-layer-chunk-authenticated-mono[OF auth-s2 s2-t])
have step:
  fri-layer-step-evidence rt b len-j idx-j pw-j
  (fri-sibling-index len-j idx-j) axp axp-path axn axn-path
  (idx-j mod (len-j div 2)) f-final actual-chunk
  by (rule fri-layer-step-evidenceI)
  (use next-value-eq f-final-eq chunk in simp-all)
have step':
  fri-layer-step-evidence
  (snd (f-fl ! ?j)) (fst (f-fl ! ?j)) ?len ?idx (2 ^ ?j)
  (fri-sibling-index ?len ?idx) axp axp-path axn axn-path
  (?idx mod (?len div 2)) f-final actual-chunk
  using step bf-eq idx-j-eq len-j-eq pw-j-eq j-eq by simp
show ?thesis
  by (rule that[OF - step'])
  (use auth-t bf-eq idx-j-eq len-j-eq j-eq in simp)
qed
qed
qed
qed

```

lemma verifier-query-round-after-index-composition-fri-last-step-evidence:

fixes s t :: ('f, 'a) protocol-channel-scheme

assumes outcome:

Some ((), t) ∈

set-dist

(execute

(verifier-query-round-after-index-program

fr f-fl f-final as fl final raw) s)

and nonempty: 0 < length fl

and raw-layer:

∧ raw'.

0 < fri-layer-lengths (length fl) (clength * scale) !

(length fl - 1) ∧

fri-layer-indices (length fl) (index (to-nat raw'))

(clength * scale) ! (length fl - 1) <

```

    fri-layer-lengths (length fl) (clength * scale) !
    (length fl - 1)
obtains actual-chunk axp axp-path axn axn-path where
  fri-layer-chunk-authenticated
    (snd (fl ! (length fl - 1)))
    (fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1))
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! (length fl - 1))
    actual-chunk t
  fri-layer-step-evidence
    (snd (fl ! (length fl - 1)))
    (fst (fl ! (length fl - 1)))
    (fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1))
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! (length fl - 1))
    (2 ^ (length fl - 1))
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1))
      (fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! (length fl - 1)))
    axp axp-path axn axn-path
    ((fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! (length fl - 1)) mod
      ((fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1)) div 2))
    final actual-chunk
proof -
  show ?thesis
proof (rule verifier-query-round-after-index-final-checks[OF outcome])
  fix idx fv s1 f-i f-len f-pow s2 c-i c-len c-pow s4
  assume idx-eq: idx = index (to-nat raw)
  and comp-fri:
    Some ((c-i, final, c-len, c-pow), s4) ∈
      set-dist
      (execute
        (mfold (idx, cp-eval as fv (h ^ idx * shift),
          clength * scale, 1)
          (receive-query-commits fl)) s2)
  and t-eq: t = s4
  let ?j = length fl - 1
  let ?len = fri-layer-lengths (length fl) (clength * scale) ! ?j
  let ?idx =
    fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! ?j
  have len-pos: 0 < ?len
  using raw-layer[of raw] by simp

```

```

have idx-bound: ?idx < ?len
  using raw-layer[of raw] by simp
show ?thesis
proof (rule receive-query-commits-last-step-evidence
  [OF comp-fri[unfolded idx-eq] nonempty])
fix j idx-j x-j len-j pw-j b rt xp xp-path xn xn-path s-j s-suc
assume j-eq: j = ?j
  and bf-eq: fl ! j = (b, rt)
  and selected:
    Some ((c-i, final, c-len, c-pow), s-suc) ∈
      set-dist
      (execute
        (fri-layer-opening-step (b, rt)
          (idx-j, x-j, len-j, pw-j)) s-j)
and s4-eq: s4 = s-suc
and idx-j-eq:
  idx-j =
    fri-layer-indices (length fl) (index (to-nat raw))
    (clength * scale) ! j
and len-j-eq:
  len-j =
    fri-layer-lengths (length fl) (clength * scale) ! j
and pw-j-eq: pw-j = 1 * 2 ^ j
have len-pos-j: 0 < len-j
  using len-pos len-j-eq j-eq by simp
have idx-bound-j: idx-j < len-j
  using idx-bound idx-j-eq len-j-eq j-eq by simp
show ?thesis
proof (rule fri-layer-opening-step-outcome-authenticated
  [OF len-pos-j idx-bound-j selected])
fix axp axp-path axn axn-path next-value actual-chunk
assume selected-out:
  (c-i, final, c-len, c-pow) =
    (idx-j mod (len-j div 2), next-value, len-j div 2,
      pw-j + pw-j)
and next-value-eq:
  next-value =
    fri-fold-value b axp axn
    (fri-fold-denominator ((h ^ idx-j) * shift) pw-j)
and chunk:
  fri-layer-opening-chunk len-j axp axp-path axn axn-path
  actual-chunk
and axp-auth:
  authenticated-opening-in s-suc
  (opening-root = rt, opening-length = len-j,
    opening-index = idx-j, opening-value = axp,
    opening-path = axp-path)
and axn-auth:
  authenticated-opening-in s-suc

```

```

      (opening-root = rt, opening-length = len-j,
       opening-index = fri-sibling-index len-j idx-j,
       opening-value = axn, opening-path = axn-path)
have final-eq: final = next-value
  using selected-out by simp
have auth-t:
  fri-layer-chunk-authenticated rt len-j idx-j actual-chunk t
  unfolding t-eq s4-eq
  by (rule fri-layer-chunk-authenticatedI[OF chunk axp-auth axn-auth])
have step:
  fri-layer-step-evidence rt b len-j idx-j pw-j
  (fri-sibling-index len-j idx-j) axp axp-path axn axn-path
  (idx-j mod (len-j div 2)) final actual-chunk
  by (rule fri-layer-step-evidenceI)
  (use next-value-eq final-eq chunk in simp-all)
have step':
  fri-layer-step-evidence
  (snd (fl ! ?j)) (fst (fl ! ?j)) ?len ?idx (2 ^ ?j)
  (fri-sibling-index ?len ?idx) axp axp-path axn axn-path
  (?idx mod (?len div 2)) final actual-chunk
  using step bf-eq idx-j-eq len-j-eq pw-j-eq j-eq by simp
show ?thesis
  by (rule that[OF - step'])
  (use auth-t bf-eq idx-j-eq len-j-eq j-eq in simp)
qed
qed
qed
qed

lemma ntimes-verifier-query-rounds-selected-trace-fri-last-step-with-prefix-at:
fixes query-state final-state :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (results, final-state) ∈
  set-dist
  (execute
   (ntimes
    (verifier-query-round-program fr f-fl f-final as fl final)
    rounds)
   query-state)
and i-bound: i < rounds
and nonempty: 0 < length f-fl
and raw-layer:
   $\bigwedge$  raw.
  0 < fri-layer-lengths (length f-fl) (clength * scale) !
  (length f-fl - 1) ∧
  fri-layer-indices (length f-fl) (index (to-nat raw))
  (clength * scale) ! (length f-fl - 1) <
  fri-layer-lengths (length f-fl) (clength * scale) !
  (length f-fl - 1)

```

```

obtains raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
  suffix-chunks actual-chunk axp axp-path axn axn-path where
  idx = index (to-nat raw)
  length prefix-query-idxs = i
  length prefix-chunks = i
  length suffix-query-idxs = rounds - Suc i
  length suffix-chunks = rounds - Suc i
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
   $\bigwedge k. k < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
   $\bigwedge k. k < rounds - Suc i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
  PTranscript query-state =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript final-state
  fri-layer-chunk-authenticated
    (snd (f-fl ! (length f-fl - 1)))
    (fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1))
    (fri-layer-indices (length f-fl) idx (clength * scale) !
      (length f-fl - 1))
    actual-chunk final-state
  fri-layer-step-evidence
    (snd (f-fl ! (length f-fl - 1)))
    (fst (f-fl ! (length f-fl - 1)))
    (fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1))
    (fri-layer-indices (length f-fl) idx (clength * scale) !
      (length f-fl - 1))
    (2 ^ (length f-fl - 1))
    (fri-sibling-index
      (fri-layer-lengths (length f-fl) (clength * scale) !
        (length f-fl - 1))
      (fri-layer-indices (length f-fl) idx (clength * scale) !
        (length f-fl - 1)))
    axp axp-path axn axn-path
    ((fri-layer-indices (length f-fl) idx (clength * scale) !
      (length f-fl - 1)) mod
      ((fri-layer-lengths (length f-fl) (clength * scale) !
        (length f-fl - 1)) div 2))
    f-final actual-chunk
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) prefix-chunks i)) =
    Some raw

```

proof —

let ?round = verifier-query-round-program fr f-fl f-final as fl final


```

from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist (execute (ntimes ?round i) query-state)
  and round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
  and suffix:
    Some (suffix, final-state) ∈
      set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
  by blast
have x-eq: x = ()
  by (cases x) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
  fix prefix-query-idxs prefix-chunks
  assume len-prefix-idxs: length prefix-query-idxs = i
  and len-prefix-chunks: length prefix-chunks = i
  and transcript-prefix:
    PTranscript query-state =
      List.concat prefix-chunks @ PTranscript s-i
  and state-i:
    PState s-i =
      state-after-query-chunks (PState query-state) prefix-chunks i
  and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
  and prefix-rounds:
     $\bigwedge k. k < i \implies$ 
      verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
  from round[unfolded x-eq verifier-query-round-program-alt-def]
  obtain raw s0 where
    raw-out:
      Some (raw, s0) ∈
        set-dist (execute receive-query-index-challenge s-i)
  and after:
    Some ((), s-suc) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program fr f-fl f-final as
            fl final raw) s0)
  by (auto elim!: set-dist-bindE)
have lookup-s0:
  fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
  using receive-query-index-challenge-outcome[OF raw-out] by simp
show ?thesis
proof (rule verifier-query-round-program-outcome[OF round[unfolded x-eq]])

```

```

fix raw' idx chunk
assume idx-eq: idx = index (to-nat raw')
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
  chunk
and transcript-round:
  PTranscript s-i = chunk @ PTranscript s-suc
and lookup-suc:
  fmlookup (HashMap s-suc)
  (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
  Some raw'
have s0-suc: s0 ≤ s-suc
using verifier-query-round-after-index-program-outcome[OF after]
by simp
have lookup-suc-raw:
  fmlookup (HashMap s-suc)
  (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
  Some raw
by (rule hash-extension-lookup[OF lookup-s0 s0-suc])
have raw-eq': raw = raw'
using lookup-suc lookup-suc-raw by simp
from verifier-query-round-after-index-trace-fri-last-step-evidence
  [OF after nonempty raw-layer]
obtain actual-chunk axp axp-path arn arn-path where
  auth-suc:
    fri-layer-chunk-authenticated
    (snd (f-fl ! (length f-fl - 1)))
    (fri-layer-lengths (length f-fl) (clength * scale) !
     (length f-fl - 1))
    (fri-layer-indices (length f-fl) (index (to-nat raw))
     (clength * scale) ! (length f-fl - 1))
    actual-chunk s-suc
and step-raw:
  fri-layer-step-evidence
  (snd (f-fl ! (length f-fl - 1)))
  (fst (f-fl ! (length f-fl - 1)))
  (fri-layer-lengths (length f-fl) (clength * scale) !
   (length f-fl - 1))
  (fri-layer-indices (length f-fl) (index (to-nat raw))
   (clength * scale) ! (length f-fl - 1))
  (2 ^ (length f-fl - 1))
  (fri-sibling-index
   (fri-layer-lengths (length f-fl) (clength * scale) !
    (length f-fl - 1))
   (fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! (length f-fl - 1)))
  axp axp-path arn arn-path
  ((fri-layer-indices (length f-fl) (index (to-nat raw))
   (clength * scale) ! (length f-fl - 1)) mod

```

```

      ((fri-layer-lengths (length f-fl) (clength * scale) !
        (length f-fl - 1)) div 2))
    f-final actual-chunk
  by blast
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
  fix suffix-query-idxs suffix-chunks
  assume len-suffix-idxs:
    length suffix-query-idxs = rounds - Suc i
  and len-suffix-chunks:
    length suffix-chunks = rounds - Suc i
  and transcript-suffix:
    PTranscript s-suc =
      List.concat suffix-chunks @ PTranscript final-state
  and suffix-ext: s-suc ≤ final-state
  and suffix-rounds:
     $\bigwedge k. k < rounds - Suc i \implies$ 
      verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
  have transcript-all:
    PTranscript query-state =
      List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript final-state
  using transcript-prefix transcript-round transcript-suffix by simp
  have auth-final-raw:
    fri-layer-chunk-authenticated
      (snd (f-fl ! (length f-fl - 1)))
      (fri-layer-lengths (length f-fl) (clength * scale) !
        (length f-fl - 1))
      (fri-layer-indices (length f-fl) (index (to-nat raw))
        (clength * scale) !
        (length f-fl - 1))
      actual-chunk final-state
  by (rule fri-layer-chunk-authenticated-mono[OF auth-suc suffix-ext])
  have auth-final:
    fri-layer-chunk-authenticated
      (snd (f-fl ! (length f-fl - 1)))
      (fri-layer-lengths (length f-fl) (clength * scale) !
        (length f-fl - 1))
      (fri-layer-indices (length f-fl) idx (clength * scale) !
        (length f-fl - 1))
      actual-chunk final-state
  using auth-final-raw raw-eq' idx-eq by simp
  have step-idx:
    fri-layer-step-evidence
      (snd (f-fl ! (length f-fl - 1)))
      (fst (f-fl ! (length f-fl - 1)))
      (fri-layer-lengths (length f-fl) (clength * scale) !

```

```

    (length f-fl - 1))
  (fri-layer-indices (length f-fl) idx (clength * scale) !
    (length f-fl - 1))
  (2 ^ (length f-fl - 1))
  (fri-sibling-index
    (fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1))
    (fri-layer-indices (length f-fl) idx (clength * scale) !
      (length f-fl - 1)))
  axp axp-path axn axn-path
  ((fri-layer-indices (length f-fl) idx (clength * scale) !
    (length f-fl - 1)) mod
    ((fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1)) div 2))
  f-final actual-chunk
using step-raw raw-eq' idx-eq by simp
have lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state)
        prefix-chunks i)) =
    Some raw
proof -
  have lookup-suc':
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
      Some raw
    using lookup-suc-raw state-i counter-i by simp
  show ?thesis
    by (rule hash-extension-lookup[OF lookup-suc' suffix-ext])
qed
show ?thesis
  by (rule that[OF - len-prefix-idxs len-prefix-chunks
    len-suffix-idxs len-suffix-chunks chunk-shape prefix-rounds
    suffix-rounds transcript-all auth-final step-idx
    lookup-final])
    (use idx-eq raw-eq' in simp)
qed
qed
qed
qed

lemma ntimes-verifier-query-rounds-selected-composition-fri-last-step-with-prefix-at:
  fixes query-state final-state :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (results, final-state) ∈
    set-dist

```

```

    (execute
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      query-state)
  and i-bound:  $i < \text{rounds}$ 
  and nonempty:  $0 < \text{length fl}$ 
  and raw-layer:
     $\bigwedge_{\text{raw.}}$ 
       $0 < \text{fri-layer-lengths (length fl) (clength * scale) !}$ 
       $(\text{length fl} - 1) \wedge$ 
       $\text{fri-layer-indices (length fl) (index (to-nat raw))}$ 
       $(\text{clength * scale) ! (length fl} - 1) <$ 
       $\text{fri-layer-lengths (length fl) (clength * scale) !}$ 
       $(\text{length fl} - 1)$ 
  obtains raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
    suffix-chunks actual-chunk axp axp-path axn axn-path where
    idx = index (to-nat raw)
    length prefix-query-idxs = i
    length prefix-chunks = i
    length suffix-query-idxs = rounds - Suc i
    length suffix-chunks = rounds - Suc i
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
     $\bigwedge k. k < i \implies$ 
      verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
     $\bigwedge k. k < \text{rounds} - \text{Suc } i \implies$ 
      verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
    PTranscript query-state =
      List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript final-state
    fri-layer-chunk-authenticated
      (snd (fl ! (length fl - 1)))
      (fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1))
      (fri-layer-indices (length fl) idx (clength * scale) !
        (length fl - 1))
      actual-chunk final-state
    fri-layer-step-evidence
      (snd (fl ! (length fl - 1)))
      (fst (fl ! (length fl - 1)))
      (fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1))
      (fri-layer-indices (length fl) idx (clength * scale) !
        (length fl - 1))
       $(2 \wedge (\text{length fl} - 1))$ 
      (fri-sibling-index
        (fri-layer-lengths (length fl) (clength * scale) !

```

```

    (length fl - 1))
  (fri-layer-indices (length fl) idx (clength * scale) !
    (length fl - 1)))
  axp axp-path axn axn-path
  ((fri-layer-indices (length fl) idx (clength * scale) !
    (length fl - 1)) mod
    ((fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1)) div 2))
  final actual-chunk
  fmlookup (HashMap final-state)
  (QueryIndexChallenge (PQueryCounter query-state + i)
    (state-after-query-chunks (PState query-state) prefix-chunks i)) =
  Some raw
proof -
  let ?round = verifier-query-round-program fr f-fl f-final as fl final
  from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
  obtain prefix x suffix s-i s-suc where
    prefix:
      Some (prefix, s-i) ∈
        set-dist (execute (ntimes ?round i) query-state)
  and round:
      Some (x, s-suc) ∈ set-dist (execute ?round s-i)
  and suffix:
      Some (suffix, final-state) ∈
        set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
  by blast
  have x-eq: x = ()
  by (cases x) simp
  show ?thesis
  proof (rule query-opening-ntimes-prefix-transcript-state-alignment
    [OF prefix])
    fix prefix-query-idxs prefix-chunks
    assume len-prefix-idxs: length prefix-query-idxs = i
    and len-prefix-chunks: length prefix-chunks = i
    and transcript-prefix:
      PTranscript query-state =
        List.concat prefix-chunks @ PTranscript s-i
    and state-i:
      PState s-i =
        state-after-query-chunks (PState query-state) prefix-chunks i
    and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
    and prefix-rounds:
       $\bigwedge k. k < i \implies$ 
        verifier-query-round-chunk (prefix-query-idxs ! k)
          (map snd f-fl) (map snd fl) (prefix-chunks ! k)
    from round[unfolded x-eq verifier-query-round-program-alt-def]
    obtain raw s0 where
      raw-out:
        Some (raw, s0) ∈

```

```

    set-dist (execute receive-query-index-challenge s-i)
and after:
    Some ((), s-suc) ∈
    set-dist
    (execute
    (verifier-query-round-after-index-program fr f-fl f-final as
    fl final raw) s0)
by (auto elim!: set-dist-bindE)
have lookup-s0:
    fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
    using receive-query-index-challenge-outcome[OF raw-out] by simp
show ?thesis
proof (rule verifier-query-round-program-outcome[OF round[unfolded x-eq]])
    fix raw' idx chunk
    assume idx-eq: idx = index (to-nat raw')
    and chunk-shape:
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
    and transcript-round:
    PTranscript s-i = chunk @ PTranscript s-suc
    and lookup-suc:
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw'
have s0-suc: s0 ≤ s-suc
    using verifier-query-round-after-index-program-outcome[OF after]
    by simp
have lookup-suc-raw:
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
    by (rule hash-extension-lookup[OF lookup-s0 s0-suc])
have raw-eq': raw = raw'
    using lookup-suc lookup-suc-raw by simp
from verifier-query-round-after-index-composition-fri-last-step-evidence
    [OF after nonempty raw-layer]
obtain actual-chunk axp axp-path axn axn-path where
    auth-suc:
    fri-layer-chunk-authenticated
    (snd (fl ! (length fl - 1)))
    (fri-layer-lengths (length fl) (clength * scale) !
    (length fl - 1))
    (fri-layer-indices (length fl) (index (to-nat raw))
    (clength * scale) ! (length fl - 1))
    actual-chunk s-suc
and step-raw:
    fri-layer-step-evidence

```

```

    (snd (fl ! (length fl - 1)))
    (fst (fl ! (length fl - 1)))
    (fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1))
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! (length fl - 1))
    (2 ^ (length fl - 1))
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1))
      (fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! (length fl - 1)))
    axp axp-path axn axn-path
    ((fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! (length fl - 1)) mod
      ((fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1)) div 2))
    final actual-chunk
  by blast
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
  fix suffix-query-idxs suffix-chunks
  assume len-suffix-idxs:
    length suffix-query-idxs = rounds - Suc i
  and len-suffix-chunks:
    length suffix-chunks = rounds - Suc i
  and transcript-suffix:
    PTranscript s-suc =
      List.concat suffix-chunks @ PTranscript final-state
  and suffix-ext: s-suc ≤ final-state
  and suffix-rounds:
     $\bigwedge k. k < \text{rounds} - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
  have transcript-all:
    PTranscript query-state =
      List.concat (prefix-chunks @ chunk # suffix-chunks) @
        PTranscript final-state
  using transcript-prefix transcript-round transcript-suffix by simp
  have auth-final-raw:
    fri-layer-chunk-authenticated
    (snd (fl ! (length fl - 1)))
    (fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1))
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) !
      (length fl - 1))
    actual-chunk final-state

```



```

by (rule fri-layer-chunk-authenticated-mono[OF auth-suc suffix-ext])
have auth-final:
  fri-layer-chunk-authenticated
    (snd (fl ! (length fl - 1)))
    (fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1))
    (fri-layer-indices (length fl) idx (clength * scale) !
      (length fl - 1))
  actual-chunk final-state
using auth-final-raw raw-eq' idx-eq by simp
have step-idx:
  fri-layer-step-evidence
    (snd (fl ! (length fl - 1)))
    (fst (fl ! (length fl - 1)))
    (fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1))
    (fri-layer-indices (length fl) idx (clength * scale) !
      (length fl - 1))
    (2 ^ (length fl - 1))
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1))
      (fri-layer-indices (length fl) idx (clength * scale) !
        (length fl - 1)))
    axp axp-path axn axn-path
    ((fri-layer-indices (length fl) idx (clength * scale) !
      (length fl - 1)) mod
      ((fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1)) div 2))
    final actual-chunk
using step-raw raw-eq' idx-eq by simp
have lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state)
        prefix-chunks i)) =
  Some raw
proof -
have lookup-suc':
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state)
        prefix-chunks i)) =
  Some raw
using lookup-suc-raw state-i counter-i by simp
show ?thesis
by (rule hash-extension-lookup[OF lookup-suc' suffix-ext])
qed
show ?thesis

```

by (rule that[OF - len-prefix-idxs len-prefix-chunks
 len-suffix-idxs len-suffix-chunks chunk-shape prefix-rounds
 suffix-rounds transcript-all auth-final step-idx
 lookup-final])
 (use idx-eq raw-eq' in simp)
 qed
 qed
 qed
 qed

lemma generic-fri-forced-value-eq-from-authenticated-step:

assumes forced:

generic-fri-round-forced-next-value roots challenges query-idxs
 round-layers round-idx layer-idx v

and recorded-auth:

generic-fri-recorded-layer-chunk-authenticated roots query-idxs
 round-layers final-state round-idx layer-idx

and actual-auth:

fri-layer-chunk-authenticated (roots ! layer-idx)
 (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
 actual-chunk final-state

and actual-step:

fri-layer-step-evidence
 (roots ! layer-idx)
 (challenges ! layer-idx)
 (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
 (2 ^ layer-idx)
 (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
 axp axp-path axn axn-path
 (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
 actual-value actual-chunk

and no-merkle:

¬ partial-merkle-inconsistency-bad s (Some (result, final-state))

shows v = actual-value

proof –

from forced **obtain** rxp rxp-path rxn rxn-path **where** recorded-step:

fri-layer-step-evidence
 (roots ! layer-idx)
 (challenges ! layer-idx)
 (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
 (2 ^ layer-idx)
 (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
 rxp rxp-path rxn rxn-path
 (fri-evidence-next-idx roots query-idxs round-idx layer-idx)

```

    v
    (round-layers ! round-idx ! layer-idx)
unfolding generic-fri-round-forced-next-value-def by blast
have recorded-chunk:
  fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (round-layers ! round-idx ! layer-idx) final-state
using recorded-auth
unfolding generic-fri-recorded-layer-chunk-authenticated-def .
have rxp-eq: rxp = axp
by (rule fri-layer-authenticated-base-values-eq-if-no-partial-merkle
    [OF no-merkle recorded-chunk actual-auth])
  (use fri-layer-step-evidenceD(4)[OF recorded-step]
    fri-layer-step-evidenceD(4)[OF actual-step] in simp-all)
have rxn-eq: rxn = axn
by (rule fri-layer-authenticated-sibling-values-eq-if-no-partial-merkle
    [OF no-merkle recorded-chunk actual-auth])
  (use fri-layer-step-evidenceD(4)[OF recorded-step]
    fri-layer-step-evidenceD(4)[OF actual-step] in simp-all)
show ?thesis
using fri-layer-step-evidenceD(3)[OF recorded-step]
  fri-layer-step-evidenceD(3)[OF actual-step]
  rxp-eq rxn-eq
by simp
qed

```

lemma generic-fri-final-conflict-false-from-authenticated-step:

assumes forced:

generic-fri-round-forced-next-value roots challenges query-idxs
 round-layers round-idx layer-idx v

and recorded-auth:

generic-fri-recorded-layer-chunk-authenticated roots query-idxs
 round-layers final-state round-idx layer-idx

and actual-auth:

fri-layer-chunk-authenticated (roots ! layer-idx)
 (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
 actual-chunk final-state

and actual-step:

fri-layer-step-evidence
 (roots ! layer-idx)
 (challenges ! layer-idx)
 (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
 (2 ^ layer-idx)
 (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
 axp axp-path axn axn-path

```

    (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
    final-value actual-chunk
  and no-merkle:
    ¬ partial-merkle-inconsistency-bad s (Some (result, final-state))
  and conflict: v ≠ final-value
shows False
using generic-fri-forced-value-eq-from-authenticated-step
  [OF forced recorded-auth actual-auth actual-step no-merkle]
  conflict
by simp

lemma generic-fri-selected-final-step-missing-false-from-authenticated-step:
  fixes root challenge final-value :: 'f
  and final-state :: ('f, 'a) protocol-channel-scheme
  assumes actual-auth:
    fri-layer-chunk-authenticated root len idx actual-chunk final-state
  and actual-step:
    fri-layer-step-evidence root challenge len idx pw sibling-idx
    arp arp-path arn arn-path next-idx final-value actual-chunk
  and missing:
    ∧ actual-chunk arp arp-path arn arn-path.
    ¬ (fri-layer-chunk-authenticated root len idx actual-chunk final-state ∧
      fri-layer-step-evidence root challenge len idx pw sibling-idx
      arp arp-path arn arn-path next-idx final-value actual-chunk)
  shows False
  using actual-auth actual-step missing by blast

end

end

theory Soundness-FRI-Authenticated-Route
  imports Soundness-FRI-Layer-Replay
begin

Route-level FRI events that keep the existing public reduction shape but refine the
sampled same-layer branch to carry authenticated Merkle evidence.

context soundness
begin

lemma fri-first-layer-raw-bound:
  assumes roots-nonempty: 0 < length roots
  shows
    0 < fri-layer-lengths (length roots) (clength * scale) ! 0 ∧
    fri-layer-indices (length roots) (index (to-nat raw))
      (clength * scale) ! 0 <
    fri-layer-lengths (length roots) (clength * scale) ! 0
proof -

```

```

have idx-bound: index (to-nat raw) < clength * scale
proof -
  have index (to-nat raw) < query-sample-space-size
    by (rule index-less-query-sample-space)
  also have ... ≤ clength * scale
    by (rule query-sample-space-size-le-domain)
  finally show ?thesis .
qed
show ?thesis
  using roots-nonempty idx-bound clength-pos scale-pos
  by (cases roots) simp-all
qed

lemma fri-layer-raw-bound:
  assumes layer-bound: j < length roots
    and eval-power: clength * scale = 2 ^ N
    and roots-rounds: length roots ≤ N
  shows
    0 < fri-layer-lengths (length roots) (clength * scale) ! j ∧
    fri-layer-indices (length roots) (index (to-nat raw))
      (clength * scale) ! j <
    fri-layer-lengths (length roots) (clength * scale) ! j
proof -
  have idx-bound: index (to-nat raw) < clength * scale
  proof -
    have index (to-nat raw) < query-sample-space-size
      by (rule index-less-query-sample-space)
    also have ... ≤ clength * scale
      by (rule query-sample-space-size-le-domain)
    finally show ?thesis .
  qed
  have idx-layer:
    fri-layer-indices (length roots) (index (to-nat raw))
      (clength * scale) ! j <
    fri-layer-lengths (length roots) (clength * scale) ! j
  by (rule fri-layer-indices-nth-bound
    [OF idx-bound layer-bound eval-power roots-rounds])
  have len-pos:
    0 < fri-layer-lengths (length roots) (clength * scale) ! j
  using idx-layer by linarith
  show ?thesis
    using len-pos idx-layer by simp
qed

definition trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
  :: ('f, 'a) protocol-channel-scheme ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
  trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer

```

$s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table}$
 $\text{result final-state.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{generic-fri-sampled-assignment-conflict-with-authenticated-same-layer}$
 $\text{trace-table trace-roots trace-bs trace-final fri-query-idxs}$
 $\text{trace-round-layers final-state})$

definition *trace-fri-header-tied-sampled-assignment-conflict-without-same-layer*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-sampled-assignment-conflict-without-same-layer}$
 $s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{generic-fri-sampled-assignment-conflict-without-same-layer}$
 $\text{trace-table trace-roots trace-bs trace-final fri-query-idxs}$
 $\text{trace-round-layers})$

definition *trace-fri-header-tied-sampled-base-opening-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-sampled-base-opening-conflict } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table* $\wedge$
generic-fri-sampled-base-opening-conflict trace-table trace-roots
trace-bs fri-query-idxs trace-round-layers)

definition *trace-fri-header-tied-sampled-next-value-conflict*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-sampled-next-value-conflict s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table* $\wedge$
generic-fri-sampled-next-value-conflict trace-roots trace-bs
fri-query-idxs trace-round-layers)

definition *trace-fri-header-tied-sampled-successor-opening-conflict*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-sampled-successor-opening-conflict s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table* $\wedge$

*generic-fri-sampled-successor-opening-conflict trace-roots trace-bs
fri-query-idxs trace-round-layers)*

definition *trace-fri-header-tied-sampled-final-value-conflict*

*:: ('f, 'a) protocol-channel-scheme $\Rightarrow$
(unit list $\times$ ('f, 'a) protocol-channel-scheme) option $\Rightarrow$ bool*

where

*trace-fri-header-tied-sampled-final-value-conflict s out $\longleftrightarrow$
($\exists$ trace-roots trace-bs trace-final dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table.
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ trace-table-low-degree trace-table $\wedge$
generic-fri-sampled-final-value-conflict trace-roots trace-bs
trace-final fri-query-idxs trace-round-layers)*

lemma *trace-fri-header-tied-without-same-imp-branch:*

assumes

*trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out*

shows

*trace-fri-header-tied-sampled-base-opening-conflict s out $\vee$
trace-fri-header-tied-sampled-next-value-conflict s out $\vee$
trace-fri-header-tied-sampled-successor-opening-conflict s out $\vee$
trace-fri-header-tied-sampled-final-value-conflict s out*

proof —

from *assms obtain* *trace-roots trace-bs trace-final dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table*

where *fri-openings:*

*accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers*

and *header:*

*verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest*

and *partial:*

*accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings*

and *cand:* *partial-trace-table-candidate trace-table trace-openings*

and *not-low:* $\neg$ *trace-table-low-degree trace-table*

and *without:*


```

    generic-fri-sampled-assignment-conflict-without-same-layer
      trace-table trace-roots trace-bs trace-final fri-query-idxs
      trace-round-layers
unfolding
  trace-fri-header-tied-sampled-assignment-conflict-without-same-layer-def
by blast
have split:
  generic-fri-sampled-base-opening-conflict trace-table trace-roots
    trace-bs fri-query-idxs trace-round-layers  $\vee$ 
  generic-fri-sampled-next-value-conflict trace-roots trace-bs
    fri-query-idxs trace-round-layers  $\vee$ 
  generic-fri-sampled-successor-opening-conflict trace-roots trace-bs
    fri-query-idxs trace-round-layers  $\vee$ 
  generic-fri-sampled-final-value-conflict trace-roots trace-bs
    trace-final fri-query-idxs trace-round-layers
by (rule generic-fri-sampled-assignment-conflict-without-same-layer-split
      [OF without])
then show ?thesis
proof
  assume base:
    generic-fri-sampled-base-opening-conflict trace-table trace-roots
      trace-bs fri-query-idxs trace-round-layers
  then show ?thesis
    unfolding trace-fri-header-tied-sampled-base-opening-conflict-def
    by (intro disjI1 exI conjI)
      (rule fri-openings, rule header, rule partial, rule cand,
        rule not-low, rule base)
next
  assume rest-split:
    generic-fri-sampled-next-value-conflict trace-roots trace-bs
      fri-query-idxs trace-round-layers  $\vee$ 
    generic-fri-sampled-successor-opening-conflict trace-roots trace-bs
      fri-query-idxs trace-round-layers  $\vee$ 
    generic-fri-sampled-final-value-conflict trace-roots trace-bs
      trace-final fri-query-idxs trace-round-layers
  then show ?thesis
proof
  assume next-conflict:
    generic-fri-sampled-next-value-conflict trace-roots trace-bs
      fri-query-idxs trace-round-layers
  then show ?thesis
    unfolding trace-fri-header-tied-sampled-next-value-conflict-def
    by (intro disjI2 disjI1 exI conjI)
      (rule fri-openings, rule header, rule partial, rule cand,
        rule not-low, rule next-conflict)
next
  assume rest-split':
    generic-fri-sampled-successor-opening-conflict trace-roots trace-bs
      fri-query-idxs trace-round-layers  $\vee$ 

```

```

    generic-fri-sampled-final-value-conflict trace-roots trace-bs
    trace-final fri-query-idxs trace-round-layers
then show ?thesis
proof
  assume successor:
    generic-fri-sampled-successor-opening-conflict trace-roots
    trace-bs fri-query-idxs trace-round-layers
  then show ?thesis
    unfolding
      trace-fri-header-tied-sampled-successor-opening-conflict-def
    by (intro disjI2 disjI2 disjI1 exI conjI)
      (rule fri-openings, rule header, rule partial, rule cand,
       rule not-low, rule successor)
  next
    assume final-conflict:
      generic-fri-sampled-final-value-conflict trace-roots trace-bs
      trace-final fri-query-idxs trace-round-layers
    then show ?thesis
      unfolding trace-fri-header-tied-sampled-final-value-conflict-def
      by (intro disjI2 disjI2 disjI2 exI conjI)
        (rule fri-openings, rule header, rule partial, rule cand,
         rule not-low, rule final-conflict)
    qed
  qed
qed
qed
qed

lemma wp-trace-fri-header-tied-without-same-bound-from-branches:
assumes base-bound:
  wp-event verify-monad
  (trace-fri-header-tied-sampled-base-opening-conflict s)  $s \leq B$ 
and next-bound:
  wp-event verify-monad
  (trace-fri-header-tied-sampled-next-value-conflict s)  $s \leq N$ 
and successor-bound:
  wp-event verify-monad
  (trace-fri-header-tied-sampled-successor-opening-conflict s)  $s \leq S$ 
and final-bound:
  wp-event verify-monad
  (trace-fri-header-tied-sampled-final-value-conflict s)  $s \leq F$ 
shows
  wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer s)
   $s \leq B + N + S + F$ 
proof –
  have wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer s)
   $s \leq$ 
  wp-event verify-monad

```

($\lambda out.$
 $trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}base\text{-}opening\text{-}conflict\ s\ out \vee$
 $trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}next\text{-}value\text{-}conflict\ s\ out \vee$
 $trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}successor\text{-}opening\text{-}conflict\ s\ out \vee$
 $trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}final\text{-}value\text{-}conflict\ s\ out) \ s$
 by (rule *wp-event-mono*)
 (rule *trace-fri-header-tied-without-same-imp-branch*)
 also have ... $\leq$
 $wp\text{-}event\ verify\text{-}monad$
 ($trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}base\text{-}opening\text{-}conflict\ s) \ s +$
 $wp\text{-}event\ verify\text{-}monad$
 ($trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}next\text{-}value\text{-}conflict\ s) \ s +$
 $wp\text{-}event\ verify\text{-}monad$
 ($trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}successor\text{-}opening\text{-}conflict\ s) \ s +$
 $wp\text{-}event\ verify\text{-}monad$
 ($trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}final\text{-}value\text{-}conflict\ s) \ s$
 by (rule *wp-event-union-bound₄*)
 also have ... $\leq B + N + S + F$
 by (intro *add-mono base-bound next-bound successor-bound final-bound*)
 finally show ?thesis .
 qed

lemma *trace-fri-header-tied-without-sameI-base:*

assumes *trace-fri-header-tied-sampled-base-opening-conflict s out*

shows

trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out

using *assms*

unfolding *trace-fri-header-tied-sampled-base-opening-conflict-def*

trace-fri-header-tied-sampled-assignment-conflict-without-same-layer-def

by (blast intro:

generic-fri-sampled-assignment-conflict-without-same-layerI-base)

lemma *trace-fri-header-tied-without-sameI-next:*

assumes *trace-fri-header-tied-sampled-next-value-conflict s out*

shows

trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out

using *assms*

unfolding *trace-fri-header-tied-sampled-next-value-conflict-def*

trace-fri-header-tied-sampled-assignment-conflict-without-same-layer-def

by (blast intro:

generic-fri-sampled-assignment-conflict-without-same-layerI-next)

lemma *trace-fri-header-tied-without-sameI-successor:*

assumes *trace-fri-header-tied-sampled-successor-opening-conflict s out*

shows

trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out

using *assms*
unfolding *trace-fri-header-tied-sampled-successor-opening-conflict-def*
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer-def
by (*blast intro*:
generic-fri-sampled-assignment-conflict-without-same-layerI-successor)

lemma *trace-fri-header-tied-without-sameI-final*:
assumes *trace-fri-header-tied-sampled-final-value-conflict s out*
shows
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out
using *assms*
unfolding *trace-fri-header-tied-sampled-final-value-conflict-def*
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer-def
by (*blast intro*:
generic-fri-sampled-assignment-conflict-without-same-layerI-final)

lemma *trace-fri-header-tied-without-same-iff-branches*:
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer s out
 $\longleftrightarrow$
trace-fri-header-tied-sampled-base-opening-conflict s out $\vee$
trace-fri-header-tied-sampled-next-value-conflict s out $\vee$
trace-fri-header-tied-sampled-successor-opening-conflict s out $\vee$
trace-fri-header-tied-sampled-final-value-conflict s out
using *trace-fri-header-tied-without-same-imp-branch*
trace-fri-header-tied-without-sameI-base
trace-fri-header-tied-without-sameI-next
trace-fri-header-tied-without-sameI-successor
trace-fri-header-tied-without-sameI-final
by *blast*

lemma *trace-fri-header-tied-base-conflict-obtains-authenticated-first-chunk*:
assumes *base*:
trace-fri-header-tied-sampled-base-opening-conflict
s (Some (result, final-state))
obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table
round-idx xp xp-path xn xn-path auth-chunk
where
accepted-fri-opening-transcript s (Some (result, final-state))
trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr fri-query-idxs trace-openings
partial-trace-table-candidate trace-table trace-openings

```

¬ trace-table-low-degree trace-table
round-idx < length fri-query-idxs
0 < length trace-bs
fri-layer-step-evidence
  (trace-roots ! 0)
  (trace-bs ! 0)
  (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
  1
  (fri-sibling-index (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
  xp xp-path xn xn-path
  (fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
  (fri-evidence-next-value trace-roots trace-bs fri-query-idxs
    round-idx 0 xp xn)
  (trace-round-layers ! round-idx ! 0)
¬ fri-opening-matches-table
  (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
  trace-table xp xn
fri-layer-chunk-authenticated (trace-roots ! 0)
  (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
  auth-chunk final-state
proof —
from base obtain trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr as rest trace-openings trace-table
where fri-openings:
  accepted-fri-opening-transcript s (Some (result, final-state))
  trace-roots trace-bs trace-final dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers
and header:
  verifier-header-transcript s fr trace-roots trace-final as dg
  composition-roots composition-final rest
and partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr fri-query-idxs trace-openings
and cand: partial-trace-table-candidate trace-table trace-openings
and not-low: ¬ trace-table-low-degree trace-table
and generic:
  generic-fri-sampled-base-opening-conflict trace-table trace-roots
  trace-bs fri-query-idxs trace-round-layers
unfolding trace-fri-header-tied-sampled-base-opening-conflict-def
by blast
from generic obtain round-idx xp xp-path xn xn-path where
  trace-bs-nonempty: 0 < length trace-bs
and round-bound: round-idx < length fri-query-idxs

```

and *step*:
fri-layer-step-evidence
 (trace-roots ! 0)
 (trace-bs ! 0)
 (fri-evidence-layer-len trace-roots 0)
 (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
 1
 (fri-sibling-index (fri-evidence-layer-len trace-roots 0)
 (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
xp xp-path xn xn-path
 (fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
 (fri-evidence-next-value trace-roots trace-bs fri-query-idxs
 round-idx 0 *xp xn*)
 (trace-round-layers ! round-idx ! 0)
and *no-match*:
 ¬ *fri-opening-matches-table*
 (fri-evidence-layer-len trace-roots 0)
 (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
 trace-table *xp xn*
unfolding *generic-fri-sampled-base-opening-conflict-def*
by *blast*
have *roots-nonempty*: 0 < length trace-roots
using *accepted-fri-opening-transcript-shapes*(2)[*OF fri-openings*]
trace-bs-nonempty **by** *simp*
have *round-bound-rounds*: round-idx < rounds
using *round-bound accepted-fri-opening-transcript-shapes*(7)[*OF fri-openings*]
by *simp*
obtain *auth-chunk* **where** *auth*:
fri-layer-chunk-authenticated (trace-roots ! 0)
 (fri-evidence-layer-len trace-roots 0)
 (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
auth-chunk final-state
proof –
have *layer-bound*: 0 < length trace-roots
by (rule *roots-nonempty*)
have *raw-layer*:
 ∧_{raw}.
 0 < *fri-layer-lengths* (length trace-roots) (clength * scale) ! 0 ∧
fri-layer-indices (length trace-roots) (index (to-nat raw))
 (clength * scale) ! 0 <
fri-layer-lengths (length trace-roots) (clength * scale) ! 0
by (rule *fri-first-layer-raw-bound*[*OF roots-nonempty*])
from *accepted-fri-opening-transcript-trace-step-with-authenticated-chunk-at*
 [*OF fri-openings refl round-bound-rounds layer-bound raw-layer*]
obtain *auth-chunk* **where**
generic-fri-transcript-step-with-authenticated-chunk trace-roots
 trace-bs fri-query-idxs trace-round-layers final-state round-idx 0
by *blast*
then obtain *auth-chunk* **where**

```

    fri-layer-chunk-authenticated (trace-roots ! 0)
    (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    auth-chunk final-state
  by (rule generic-fri-transcript-step-with-authenticated-chunkE)
then show ?thesis
  by (rule that)
qed
show ?thesis
  by (rule that[OF fri-openings header partial cand not-low round-bound
    trace-bs-nonempty step no-match auth])
qed

lemma trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer-imp-header-tied:
  assumes
    trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
    s out
  shows trace-fri-header-tied-sampled-assignment-conflict s out
proof -
  from assms obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as rest trace-openings trace-table
    result final-state
  where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  and out-eq: out = Some (result, final-state)
  and header:
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest
  and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  and cand: partial-trace-table-candidate trace-table trace-openings
  and not-low:  $\neg$  trace-table-low-degree trace-table
  and conflict:
    generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
    trace-table trace-roots trace-bs trace-final fri-query-idxs
    trace-round-layers final-state
  unfolding
    trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
  by blast
have sampled:
  generic-fri-sampled-assignment-conflict trace-table trace-roots
  trace-bs trace-final fri-query-idxs trace-round-layers
  by (rule
    generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-imp-sampled
    [OF conflict])

```

show *?thesis*
unfolding *trace-fri-header-tied-sampled-assignment-conflict-def*
by (*intro exI conjI*)
 (*rule fri-openings, rule header, rule partial, rule cand,*
 rule not-low, rule sampled)
qed

lemma *wp-trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer-bound-from-header-tied:*
assumes *header-bound:*
 wp-event verify-monad
 (*trace-fri-header-tied-sampled-assignment-conflict s*) $s \leq C$
shows
 wp-event verify-monad
 (*trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer s*)
 $s \leq C$
proof –
have *wp-event verify-monad*
 (*trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer s*)
 $s \leq$
 wp-event verify-monad
 (*trace-fri-header-tied-sampled-assignment-conflict s*) s
by (*rule wp-event-mono*)
 (*rule*
 trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer-imp-header-tied)
then show *?thesis*
by (*rule order-trans[OF - header-bound]*)
qed

definition *trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks
 $s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table}$
 $\text{ result final-state.}$
 $\text{ accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{ out} = \text{Some (result, final-state)} \wedge$
 $\text{ verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{ composition-roots composition-final rest} \wedge$
 $\text{ accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{ trace-table-low-degree trace-table} \wedge$
 $\text{ generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks}$

trace-roots trace-bs fri-query-idxs trace-round-layers final-state)

lemma *trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks-imp-partial-merkle:*

assumes

trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks
s out

shows *partial-merkle-inconsistency-bad s out*

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table result
final-state

where *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers

and *out-eq:* *out = Some (result, final-state)*

and *conflict:*

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
trace-roots trace-bs fri-query-idxs trace-round-layers final-state

unfolding

trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks-def

by *blast*

have *len:* *length trace-bs = length trace-roots*

using *accepted-fri-opening-transcript-shapes(2)[OF fri-openings]* .

have *partial-merkle-inconsistency-bad s (Some (result, final-state))*

by *(rule*

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-partial-merkle
[OF conflict len])

then show *?thesis*

by *(simp add: out-eq)*

qed

lemma *wp-trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks-bound-from-partial-merkle:*

assumes *merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M

shows

wp-event verify-monad
(trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks s)
s ≤ M

proof —

have *wp-event verify-monad*

(trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks s)

s ≤ wp-event verify-monad (partial-merkle-inconsistency-bad s) s

by *(rule wp-event-mono)*

(rule

trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks-imp-partial-merkle)

then show *?thesis*

by *(rule order-trans[OF - merkle-bound])*

qed

lemma *trace-fri-header-tied-authenticated-conflict-imp-without-same-or-same-layer*:

assumes

trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out

shows

trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out $\vee$
trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks s out

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table result
final-state

where *fri-openings*:

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers

and *out-eq*: *out* = *Some (result, final-state)*

and *header*:

verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest

and *partial*:

accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings

and *cand*: *partial-trace-table-candidate trace-table trace-openings*

and *not-low*: $\neg$ *trace-table-low-degree trace-table*

and *conflict*:

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
trace-table trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers final-state

unfolding

trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer-def

by *blast*

have *split*:

generic-fri-sampled-assignment-conflict-without-same-layer
trace-table trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers $\vee$
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
trace-roots trace-bs fri-query-idxs trace-round-layers final-state

by (*rule*

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-split
[*OF conflict*])

then show *?thesis*

proof

assume *without*:

generic-fri-sampled-assignment-conflict-without-same-layer
trace-table trace-roots trace-bs trace-final fri-query-idxs

```

    trace-round-layers
  then show ?thesis
    unfolding
      trace-fri-header-tied-sampled-assignment-conflict-without-same-layer-def
    by (intro disjI1 exI conjI)
      (rule fri-openings, rule header, rule partial, rule cand,
       rule not-low, rule without)
  next
  assume same:
    generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    trace-roots trace-bs fri-query-idxs trace-round-layers final-state
  then show ?thesis
    unfolding
      trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks-def
    by (intro disjI2 exI conjI)
      (rule fri-openings, rule out-eq, rule header, rule partial,
       rule cand, rule not-low, rule same)
qed
qed

lemma trace-fri-header-tied-authenticated-conflictI-without-same:
  assumes
    trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
    s out
  shows
    trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
    s out
proof -
  from assms obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as rest trace-openings trace-table
  where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  and header:
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest
  and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  and cand: partial-trace-table-candidate trace-table trace-openings
  and not-low:  $\neg$  trace-table-low-degree trace-table
  and without:
    generic-fri-sampled-assignment-conflict-without-same-layer
    trace-table trace-roots trace-bs trace-final fri-query-idxs
    trace-round-layers
  unfolding
    trace-fri-header-tied-sampled-assignment-conflict-without-same-layer-def

```

by blast
 from *fri-openings* **obtain** *result final-state* **where** *out-eq*:
 out = Some (result, final-state)
 unfolding *accepted-fri-opening-transcript-def* by blast
 have *conflict*:
 generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
 trace-table trace-roots trace-bs trace-final fri-query-idxs
 trace-round-layers final-state
 by (rule
 generic-fri-sampled-assignment-conflict-with-authenticated-same-layerI-without
 [*OF without*])
 show ?thesis
 unfolding
 trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
 by (intro *exI conjI*)
 (rule *fri-openings*, rule *out-eq*, rule *header*, rule *partial*, rule *cand*,
 rule *not-low*, rule *conflict*)
 qed

lemma *trace-fri-header-tied-authenticated-conflictI-same-layer*:
 assumes
 trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks s out
 shows
 trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
 s out
proof —
 from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr as rest trace-openings trace-table result
 final-state
where *fri-openings*:
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final dg composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers
and *out-eq*: *out = Some (result, final-state)*
and *header*:
 verifier-header-transcript s fr trace-roots trace-final as dg
 composition-roots composition-final rest
and *partial*:
 accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings
and *cand*: *partial-trace-table-candidate trace-table trace-openings*
and *not-low*: $\neg$ *trace-table-low-degree trace-table*
and *same*:
 generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
 trace-roots trace-bs fri-query-idxs trace-round-layers final-state
 unfolding
 trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks-def
 by blast

have *conflict*:
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
trace-table trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers final-state
by (rule
generic-fri-sampled-assignment-conflict-with-authenticated-same-layerI-same
[*OF same*])
show *?thesis*
unfolding
trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
by (intro *exI conjI*)
(rule *fri-openings*, rule *out-eq*, rule *header*, rule *partial*, rule *cand*,
rule *not-low*, rule *conflict*)
qed

lemma *trace-fri-header-tied-authenticated-conflict-iff*:
trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out $\longleftrightarrow$
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out $\vee$
trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks s out
using *trace-fri-header-tied-authenticated-conflict-imp-without-same-or-same-layer*
trace-fri-header-tied-authenticated-conflictI-without-same
trace-fri-header-tied-authenticated-conflictI-same-layer
by *blast*

lemma *wp-trace-fri-header-tied-authenticated-conflict-bound-from-without-same-and-merkle*:
assumes *without-bound*:
wp-event verify-monad
(*trace-fri-header-tied-sampled-assignment-conflict-without-same-layer s*)
s $\leq$ W
and *merkle-bound*:
wp-event verify-monad (partial-merkle-inconsistency-bad s) s $\leq$ M
shows
wp-event verify-monad
(*trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer s*)
s $\leq$ W + M

proof –
have *wp-event verify-monad*
(*trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer s*)
s $\leq$
wp-event verify-monad
(λ *out.*
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out $\vee$
trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks
s out) s)
by (rule *wp-event-mono*)
(rule

$\text{trace-fri-header-tied-authenticated-conflict-imp-without-same-or-same-layer})$
also have ... $\leq$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict-without-same-layer } s)$
 $s +$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks } s)$
 s
by (rule $\text{wp-event-union-bound}$)
also have ... $\leq W + M$
by (rule $\text{add-mono}[OF \text{ without-bound}$
 $\text{wp-trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks-bound-from-partial-merkle}$
 $[OF \text{ merkle-bound}]])$
finally show ?thesis .
qed

definition $\text{trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer}$

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer}$
 $s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr trace-openings trace-table result}$
 final-state.
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{generic-fri-sampled-assignment-conflict-with-authenticated-same-layer}$
 $\text{trace-table trace-roots trace-bs trace-final fri-query-idxs}$
 $\text{trace-round-layers final-state})$

lemma $\text{trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-imp-verifier-tied:}$

assumes

$\text{trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer}$
 $s \text{ out}$

shows $\text{trace-fri-verifier-tied-sampled-assignment-conflict } s \text{ out}$

proof —

from **assms** **obtain** $\text{trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr trace-openings trace-table result}$
 final-state

where fri-openings:

```

    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
and out-eq: out = Some (result, final-state)
and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
and cand: partial-trace-table-candidate trace-table trace-openings
and not-low:  $\neg$  trace-table-low-degree trace-table
and conflict:
    generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
    trace-table trace-roots trace-bs trace-final fri-query-idxs
    trace-round-layers final-state
unfolding
    trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
by blast
have sampled:
    generic-fri-sampled-assignment-conflict trace-table trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers
by (rule
    generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-imp-sampled
    [OF conflict])
show ?thesis
unfolding trace-fri-verifier-tied-sampled-assignment-conflict-def
by (intro exI conjI)
    (rule fri-openings, rule partial, rule cand, rule not-low,
    rule sampled)

```

qed

lemma wp-trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-bound-from-verifier-tied:

assumes verifier-bound:

wp-event verify-monad

(trace-fri-verifier-tied-sampled-assignment-conflict s) $s \leq C$

shows

wp-event verify-monad

(trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)

$s \leq C$

proof –

have wp-event verify-monad

(trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)

$s \leq$

wp-event verify-monad

(trace-fri-verifier-tied-sampled-assignment-conflict s) s

by (rule wp-event-mono)

(rule

trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-imp-verifier-tied)

then show ?thesis

by (rule order-trans[OF - verifier-bound])

qed

definition *trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
 $s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr trace-openings trace-table result}$
 final-state.
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks}$
 $\text{trace-roots trace-bs fri-query-idxs trace-round-layers final-state})$

lemma *trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-imp-partial-merkle:*

assumes

trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
 $s \text{ out}$

shows *partial-merkle-inconsistency-bad* $s \text{ out}$

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr trace-openings trace-table result}$
 final-state

where *fri-openings:*

$\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers}$

and *out-eq:* $\text{out} = \text{Some (result, final-state)}$

and *conflict:*

$\text{generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks}$
 $\text{trace-roots trace-bs fri-query-idxs trace-round-layers final-state}$

unfolding

trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-def

by *blast*

have *len:* $\text{length trace-bs} = \text{length trace-roots}$

using *accepted-fri-opening-transcript-shapes(2)[OF fri-openings]* .

have *partial-merkle-inconsistency-bad* $s (\text{Some (result, final-state)})$

by (*rule*

$\text{generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-partial-merkle}$
 $[\text{OF conflict len}]$)

then show *?thesis*
by (*simp add: out-eq*)
qed

lemma *wp-trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-bound-from-partial-merkle:*

assumes *merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M

shows

wp-event verify-monad

(trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks s)

s ≤ M

proof –

have *wp-event verify-monad*

(trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks s)

s ≤ wp-event verify-monad (partial-merkle-inconsistency-bad s) s

by (*rule wp-event-mono*)

(*rule*

trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-imp-partial-merkle)

then show *?thesis*

by (*rule order-trans[OF - merkle-bound]*)

qed

definition *composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer*

:: ('f, 'a) protocol-channel-scheme ⇒

(unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool

where

composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer

s out ⟷

(∃ trace-roots trace-bs trace-final fri-dg composition-roots

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as trace-openings composition-openings

trace-table composition-table result final-state.

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers ∧

out = Some (result, final-state) ∧

accepted-with-partial-initial-openings-aligned s out fr trace-roots

trace-final as fri-dg composition-roots composition-final fri-query-idxs

trace-openings composition-openings ∧

partial-trace-table-candidate trace-table trace-openings ∧

partial-composition-table-candidate composition-table

composition-openings ∧

trace-table-low-degree trace-table ∧

¬ composition-table-low-degree maxDegree composition-table ∧

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer

composition-table composition-roots composition-bs

composition-final fri-query-idxs composition-round-layers

final-state)

definition *composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
 $s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table.}$
 $\text{ accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{ accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{ trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{ trace-openings composition-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{ trace-table-low-degree trace-table} \wedge$
 $\neg \text{ composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{ generic-fri-sampled-assignment-conflict-without-same-layer}$
 $\text{ composition-table composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs composition-round-layers})$

definition *composition-fri-verifier-tied-sampled-base-opening-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-sampled-base-opening-conflict $s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table.}$
 $\text{ accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{ accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{ trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{ trace-openings composition-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{ trace-table-low-degree trace-table} \wedge$
 $\neg \text{ composition-table-low-degree maxDegree composition-table} \wedge$

*generic-fri-sampled-base-opening-conflict composition-table
composition-roots composition-bs fri-query-idxs
composition-round-layers)*

definition *composition-fri-verifier-tied-sampled-next-value-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-sampled-next-value-conflict s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table.}$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{ accepted-with-partial-initial-openings-aligned s out fr trace-roots}$
 $\text{ trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{ trace-openings composition-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{ trace-table-low-degree trace-table} \wedge$
 $\neg \text{ composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{ generic-fri-sampled-next-value-conflict composition-roots}$
 $\text{ composition-bs fri-query-idxs composition-round-layers})$

definition *composition-fri-verifier-tied-sampled-successor-opening-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-sampled-successor-opening-conflict s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table.}$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{ accepted-with-partial-initial-openings-aligned s out fr trace-roots}$
 $\text{ trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{ trace-openings composition-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$

$\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{generic-fri-sampled-successor-opening-conflict composition-roots}$
 $\text{composition-bs fri-query-idxs composition-round-layers})$

definition *composition-fri-verifier-tied-sampled-final-value-conflict*

$:: ('f, 'a) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-sampled-final-value-conflict } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$
 $\text{trace-table composition-table.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{generic-fri-sampled-final-value-conflict composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs}$
 $\text{composition-round-layers})$

lemma *composition-fri-verifier-tied-without-same-imp-branch:*

assumes

$\text{composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer}$
 $s \text{ out}$

shows

$\text{composition-fri-verifier-tied-sampled-base-opening-conflict } s \text{ out} \vee$
 $\text{composition-fri-verifier-tied-sampled-next-value-conflict } s \text{ out} \vee$
 $\text{composition-fri-verifier-tied-sampled-successor-opening-conflict}$
 $s \text{ out} \vee$
 $\text{composition-fri-verifier-tied-sampled-final-value-conflict } s \text{ out}$

proof —

from **assms** **obtain** $\text{trace-roots trace-bs trace-final fri-dg}$
 $\text{composition-roots composition-bs composition-final fri-query-idxs}$
 $\text{trace-round-layers composition-round-layers fr as trace-openings}$
 $\text{composition-openings trace-table composition-table}$

where *fri-openings:*

$\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$

```

    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
and aligned:
    accepted-with-partial-initial-openings-aligned s out fr trace-roots
    trace-final as fri-dg composition-roots composition-final fri-query-idxs
    trace-openings composition-openings
and trace-cand:
    partial-trace-table-candidate trace-table trace-openings
and comp-cand:
    partial-composition-table-candidate composition-table
    composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-not-low:
    ¬ composition-table-low-degree maxDegree composition-table
and without:
    generic-fri-sampled-assignment-conflict-without-same-layer
    composition-table composition-roots composition-bs composition-final
    fri-query-idxs composition-round-layers
unfolding
    composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
by blast
have split:
    generic-fri-sampled-base-opening-conflict composition-table
    composition-roots composition-bs fri-query-idxs
    composition-round-layers ∨
    generic-fri-sampled-next-value-conflict composition-roots
    composition-bs fri-query-idxs composition-round-layers ∨
    generic-fri-sampled-successor-opening-conflict composition-roots
    composition-bs fri-query-idxs composition-round-layers ∨
    generic-fri-sampled-final-value-conflict composition-roots
    composition-bs composition-final fri-query-idxs
    composition-round-layers
by (rule generic-fri-sampled-assignment-conflict-without-same-layer-split
    [OF without])
then show ?thesis
proof
    assume base:
    generic-fri-sampled-base-opening-conflict composition-table
    composition-roots composition-bs fri-query-idxs
    composition-round-layers
    then show ?thesis
    unfolding composition-fri-verifier-tied-sampled-base-opening-conflict-def
    by (intro disjI1 exI conjI)
    (rule fri-openings, rule aligned, rule trace-cand, rule comp-cand,
    rule trace-low, rule comp-not-low, rule base)
next
    assume rest-split:
    generic-fri-sampled-next-value-conflict composition-roots
    composition-bs fri-query-idxs composition-round-layers ∨

```

```

generic-fri-sampled-successor-opening-conflict composition-roots
  composition-bs fri-query-idxs composition-round-layers ∨
generic-fri-sampled-final-value-conflict composition-roots
  composition-bs composition-final fri-query-idxs
  composition-round-layers
then show ?thesis
proof
  assume next-conflict:
    generic-fri-sampled-next-value-conflict composition-roots
    composition-bs fri-query-idxs composition-round-layers
  then show ?thesis
    unfolding
      composition-fri-verifier-tied-sampled-next-value-conflict-def
    by (intro disjI2 disjI1 exI conjI)
      (rule fri-openings, rule aligned, rule trace-cand, rule comp-cand,
       rule trace-low, rule comp-not-low, rule next-conflict)
next
  assume rest-split':
    generic-fri-sampled-successor-opening-conflict composition-roots
    composition-bs fri-query-idxs composition-round-layers ∨
    generic-fri-sampled-final-value-conflict composition-roots
    composition-bs composition-final fri-query-idxs
    composition-round-layers
  then show ?thesis
  proof
    assume successor:
      generic-fri-sampled-successor-opening-conflict composition-roots
      composition-bs fri-query-idxs composition-round-layers
    then show ?thesis
      unfolding
        composition-fri-verifier-tied-sampled-successor-opening-conflict-def
      by (intro disjI2 disjI2 disjI1 exI conjI)
        (rule fri-openings, rule aligned, rule trace-cand,
         rule comp-cand, rule trace-low, rule comp-not-low,
         rule successor)
  next
  assume final-conflict:
    generic-fri-sampled-final-value-conflict composition-roots
    composition-bs composition-final fri-query-idxs
    composition-round-layers
  then show ?thesis
    unfolding
      composition-fri-verifier-tied-sampled-final-value-conflict-def
    by (intro disjI2 disjI2 disjI2 exI conjI)
      (rule fri-openings, rule aligned, rule trace-cand,
       rule comp-cand, rule trace-low, rule comp-not-low,
       rule final-conflict)
qed
qed

```

qed
qed

lemma *wp-composition-fri-verifier-tied-without-same-bound-from-branches:*

assumes *base-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-sampled-base-opening-conflict s) s ≤ B

and *next-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-sampled-next-value-conflict s) s ≤ N

and *successor-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-sampled-successor-opening-conflict s)
s ≤ S

and *final-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-sampled-final-value-conflict s) s ≤ F

shows

wp-event verify-monad

(composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s)
s ≤ B + N + S + F

proof –

have *wp-event verify-monad*

(composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s)
s ≤

wp-event verify-monad

(λout.

composition-fri-verifier-tied-sampled-base-opening-conflict

s out ∨

composition-fri-verifier-tied-sampled-next-value-conflict

s out ∨

composition-fri-verifier-tied-sampled-successor-opening-conflict

s out ∨

composition-fri-verifier-tied-sampled-final-value-conflict

s out) s

by *(rule wp-event-mono)*

(rule composition-fri-verifier-tied-without-same-imp-branch)

also have ... ≤

wp-event verify-monad

(composition-fri-verifier-tied-sampled-base-opening-conflict s) s +

wp-event verify-monad

(composition-fri-verifier-tied-sampled-next-value-conflict s) s +

wp-event verify-monad

(composition-fri-verifier-tied-sampled-successor-opening-conflict s)

s +

wp-event verify-monad

(composition-fri-verifier-tied-sampled-final-value-conflict s) s

by *(rule wp-event-union-bound4)*

also have ... ≤ *B + N + S + F*

by (intro add-mono base-bound next-bound successor-bound final-bound)
 finally show ?thesis .
 qed

lemma *composition-fri-verifier-tied-without-sameI-base:*
assumes *composition-fri-verifier-tied-sampled-base-opening-conflict s out*
shows
 composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
 s out
using *assms*
unfolding
 composition-fri-verifier-tied-sampled-base-opening-conflict-def
 composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
by (blast intro:
 generic-fri-sampled-assignment-conflict-without-same-layerI-base)

lemma *composition-fri-verifier-tied-without-sameI-next:*
assumes *composition-fri-verifier-tied-sampled-next-value-conflict s out*
shows
 composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
 s out
using *assms*
unfolding
 composition-fri-verifier-tied-sampled-next-value-conflict-def
 composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
by (blast intro:
 generic-fri-sampled-assignment-conflict-without-same-layerI-next)

lemma *composition-fri-verifier-tied-without-sameI-successor:*
assumes
 composition-fri-verifier-tied-sampled-successor-opening-conflict s out
shows
 composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
 s out
using *assms*
unfolding
 composition-fri-verifier-tied-sampled-successor-opening-conflict-def
 composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
by (blast intro:
 generic-fri-sampled-assignment-conflict-without-same-layerI-successor)

lemma *composition-fri-verifier-tied-without-sameI-final:*
assumes *composition-fri-verifier-tied-sampled-final-value-conflict s out*
shows
 composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
 s out
using *assms*
unfolding
 composition-fri-verifier-tied-sampled-final-value-conflict-def

composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
by (blast intro:
generic-fri-sampled-assignment-conflict-without-same-layerI-final)

lemma *composition-fri-verifier-tied-without-same-iff-branches:*
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out
 $\longleftrightarrow$
composition-fri-verifier-tied-sampled-base-opening-conflict s out $\vee$
composition-fri-verifier-tied-sampled-next-value-conflict s out $\vee$
composition-fri-verifier-tied-sampled-successor-opening-conflict s out $\vee$
composition-fri-verifier-tied-sampled-final-value-conflict s out
using *composition-fri-verifier-tied-without-same-imp-branch*
composition-fri-verifier-tied-without-sameI-base
composition-fri-verifier-tied-without-sameI-next
composition-fri-verifier-tied-without-sameI-successor
composition-fri-verifier-tied-without-sameI-final
by blast

lemma *composition-fri-verifier-tied-base-conflict-obtains-authenticated-first-chunk:*

assumes *base:*
composition-fri-verifier-tied-sampled-base-opening-conflict
s (Some (result, final-state))
obtains *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table round-idx xp xp-path xn xn-path
auth-chunk

where

accepted-fri-opening-transcript s (Some (result, final-state))
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers
accepted-with-partial-initial-openings-aligned s
(Some (result, final-state)) fr trace-roots trace-final as fri-dg
composition-roots composition-final fri-query-idxs trace-openings
composition-openings
partial-trace-table-candidate trace-table trace-openings
partial-composition-table-candidate composition-table
composition-openings
trace-table-low-degree trace-table
 $\neg$ *composition-table-low-degree maxDegree composition-table*
round-idx < length fri-query-idxs
0 < length composition-bs
fri-layer-step-evidence
(composition-roots ! 0)
(composition-bs ! 0)
(fri-evidence-layer-len composition-roots 0)
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)

```

1
(fri-sibling-index (fri-evidence-layer-len composition-roots 0)
 (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0))
xp xp-path xn xn-path
(fri-evidence-next-idx composition-roots fri-query-idxs round-idx 0)
(fri-evidence-next-value composition-roots composition-bs
 fri-query-idxs round-idx 0 xp xn)
(composition-round-layers ! round-idx ! 0)
¬ fri-opening-matches-table
(fri-evidence-layer-len composition-roots 0)
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
composition-table xp xn
fri-layer-chunk-authenticated (composition-roots ! 0)
(fri-evidence-layer-len composition-roots 0)
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
auth-chunk final-state
proof –
from base obtain trace-roots trace-bs trace-final fri-dg
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
composition-openings trace-table composition-table
where fri-openings:
accepted-fri-opening-transcript s (Some (result, final-state))
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers
and aligned:
accepted-with-partial-initial-openings-aligned s
(Some (result, final-state)) fr trace-roots trace-final as fri-dg
composition-roots composition-final fri-query-idxs trace-openings
composition-openings
and trace-cand:
partial-trace-table-candidate trace-table trace-openings
and comp-cand:
partial-composition-table-candidate composition-table
composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-not-low:
¬ composition-table-low-degree maxDegree composition-table
and generic:
generic-fri-sampled-base-opening-conflict composition-table
composition-roots composition-bs fri-query-idxs
composition-round-layers
unfolding composition-fri-verifier-tied-sampled-base-opening-conflict-def
by blast
from generic obtain round-idx xp xp-path xn xn-path where
composition-bs-nonempty: 0 < length composition-bs
and round-bound: round-idx < length fri-query-idxs
and step:

```

```

fri-layer-step-evidence
  (composition-roots ! 0)
  (composition-bs ! 0)
  (fri-evidence-layer-len composition-roots 0)
  (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
  1
  (fri-sibling-index (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0))
  xp xp-path xn xn-path
  (fri-evidence-next-idx composition-roots fri-query-idxs round-idx 0)
  (fri-evidence-next-value composition-roots composition-bs
    fri-query-idxs round-idx 0 xp xn)
  (composition-round-layers ! round-idx ! 0)
and no-match:
  ¬ fri-opening-matches-table
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    composition-table xp xn
unfolding generic-fri-sampled-base-opening-conflict-def
by blast
have roots-nonempty: 0 < length composition-roots
  using accepted-fri-opening-transcript-shapes(4)[OF fri-openings]
  composition-bs-nonempty by simp
have round-bound-rounds: round-idx < rounds
  using round-bound accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
  by simp
obtain auth-chunk where auth:
  fri-layer-chunk-authenticated (composition-roots ! 0)
  (fri-evidence-layer-len composition-roots 0)
  (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
  auth-chunk final-state
proof -
  have layer-bound: 0 < length composition-roots
    by (rule roots-nonempty)
  have raw-layer:
    ∧ raw.
      0 < fri-layer-lengths (length composition-roots) (clength * scale) ! 0 ∧
      fri-layer-indices (length composition-roots) (index (to-nat raw))
      (clength * scale) ! 0 <
      fri-layer-lengths (length composition-roots) (clength * scale) ! 0
    by (rule fri-first-layer-raw-bound[OF roots-nonempty])
  from accepted-fri-opening-transcript-composition-step-with-authenticated-chunk-at
    [OF fri-openings refl round-bound-rounds layer-bound raw-layer]
  obtain auth-chunk where
    generic-fri-transcript-step-with-authenticated-chunk composition-roots
    composition-bs fri-query-idxs composition-round-layers final-state
    round-idx 0
  by blast
then obtain auth-chunk where

```

```

    fri-layer-chunk-authenticated (composition-roots ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    auth-chunk final-state
  by (rule generic-fri-transcript-step-with-authenticated-chunkE)
then show ?thesis
  by (rule that)
qed
show ?thesis
  by (rule that[OF fri-openings aligned trace-cand comp-cand trace-low
    comp-not-low round-bound composition-bs-nonempty step no-match auth])
qed

```

definition *composition-fri-verifier-tied-authenticated-base-opening-conflict*

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool

```

where

```

composition-fri-verifier-tied-authenticated-base-opening-conflict s out
 $\longleftrightarrow$ 
( $\exists$  trace-roots trace-bs trace-final fri-dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr as trace-openings composition-openings
  trace-table composition-table round-idx xp xp-path xn xn-path
  result final-state.
out = Some (result, final-state)  $\wedge$ 
accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers  $\wedge$ 
accepted-with-partial-initial-openings-aligned s out fr trace-roots
  trace-final as fri-dg composition-roots composition-final fri-query-idxs
  trace-openings composition-openings  $\wedge$ 
partial-trace-table-candidate trace-table trace-openings  $\wedge$ 
partial-composition-table-candidate composition-table
  composition-openings  $\wedge$ 
trace-table-low-degree trace-table  $\wedge$ 
 $\neg$  composition-table-low-degree maxDegree composition-table  $\wedge$ 
round-idx < length fri-query-idxs  $\wedge$ 
0 < length composition-bs  $\wedge$ 
fri-layer-step-evidence
  (composition-roots ! 0)
  (composition-bs ! 0)
  (fri-evidence-layer-len composition-roots 0)
  (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
  1
  (fri-sibling-index (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0))
  xp xp-path xn xn-path
  (fri-evidence-next-idx composition-roots fri-query-idxs round-idx 0)

```

```

(fri-evidence-next-value composition-roots composition-bs
 fri-query-idxs round-idx 0 xp xn)
(composition-round-layers ! round-idx ! 0) ∧
¬ fri-opening-matches-table
(fri-evidence-layer-len composition-roots 0)
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
composition-table xp xn ∧
fri-layer-chunk-authenticated (composition-roots ! 0)
(fri-evidence-layer-len composition-roots 0)
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
(composition-round-layers ! round-idx ! 0) final-state)

```

definition *composition-fri-verifier-tied-base-chunk-auth-gap*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

```

composition-fri-verifier-tied-base-chunk-auth-gap s out  $\longleftrightarrow$ 
composition-fri-verifier-tied-sampled-base-opening-conflict s out ∧
¬ composition-fri-verifier-tied-authenticated-base-opening-conflict
s out

```

lemma *composition-fri-verifier-tied-sampled-base-imp-authenticated-or-auth-gap:*

assumes

composition-fri-verifier-tied-sampled-base-opening-conflict s out

shows

composition-fri-verifier-tied-authenticated-base-opening-conflict s out $\vee$
composition-fri-verifier-tied-base-chunk-auth-gap s out

using *assms*

unfolding *composition-fri-verifier-tied-base-chunk-auth-gap-def*

by *blast*

lemma *wp-composition-fri-verifier-tied-sampled-base-bound-from-authenticated-and-auth-gap:*

fixes *A G :: prob*

assumes *authenticated-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-authenticated-base-opening-conflict s)

s ≤ A

and *gap-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-base-chunk-auth-gap s) s ≤ G

shows

wp-event verify-monad

(composition-fri-verifier-tied-sampled-base-opening-conflict s) s

≤ A + G

proof –

have *wp-event verify-monad*

(composition-fri-verifier-tied-sampled-base-opening-conflict s) s ≤

wp-event verify-monad

(λout.

composition-fri-verifier-tied-authenticated-base-opening-conflict
s out $\vee$
composition-fri-verifier-tied-base-chunk-auth-gap s out *s*
by (rule *wp-event-mono*)
(rule *composition-fri-verifier-tied-sampled-base-imp-authenticated-or-auth-gap*)
also have ... $\leq$
wp-event verify-monad
(*composition-fri-verifier-tied-authenticated-base-opening-conflict s*) *s* +
wp-event verify-monad
(*composition-fri-verifier-tied-base-chunk-auth-gap s*) *s*
by (rule *wp-event-union-bound*)
also have ... $\leq A + G$
by (intro *add-mono authenticated-bound gap-bound*)
finally show ?thesis .
qed

lemma *composition-fri-verifier-tied-authenticated-base-conflict-imp-sampled-base:*
assumes

composition-fri-verifier-tied-authenticated-base-opening-conflict s out

shows *composition-fri-verifier-tied-sampled-base-opening-conflict s out*

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table round-idx xp xp-path xn xn-path
result final-state

where *out-eq*: *out* = *Some (result, final-state)*

and *fri-openings*:

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers

and *aligned*:

accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings

and *trace-cand*:

partial-trace-table-candidate trace-table trace-openings

and *comp-cand*:

partial-composition-table-candidate composition-table
composition-openings

and *trace-low*: *trace-table-low-degree trace-table*

and *comp-not-low*:

$\neg$ *composition-table-low-degree maxDegree composition-table*

and *round-bound*: *round-idx* < *length fri-query-idxs*

and *composition-bs-nonempty*: *0* < *length composition-bs*

and *step*:

fri-layer-step-evidence
(*composition-roots* ! *0*)

```

    (composition-bs ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    1
    (fri-sibling-index (fri-evidence-layer-len composition-roots 0)
      (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0))
    xp xp-path xn xn-path
    (fri-evidence-next-idx composition-roots fri-query-idxs round-idx 0)
    (fri-evidence-next-value composition-roots composition-bs
      fri-query-idxs round-idx 0 xp xn)
    (composition-round-layers ! round-idx ! 0)
  and no-match:
    ¬ fri-opening-matches-table
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    composition-table xp xn
  and auth:
    fri-layer-chunk-authenticated (composition-roots ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    (composition-round-layers ! round-idx ! 0) final-state
  unfolding
    composition-fri-verifier-tied-authenticated-base-opening-conflict-def
  by auto
  have base:
    generic-fri-sampled-base-opening-conflict composition-table
      composition-roots composition-bs fri-query-idxs composition-round-layers
  unfolding generic-fri-sampled-base-opening-conflict-def
  by (intro exI[of - round-idx] exI[of - xp] exI[of - xp-path]
    exI[of - xn] exI[of - xn-path] conjI)
    (use round-bound composition-bs-nonempty step no-match in auto)
  show ?thesis
  unfolding composition-fri-verifier-tied-sampled-base-opening-conflict-def
  by (intro exI conjI)
    (use out-eq fri-openings aligned trace-cand comp-cand trace-low
      comp-not-low base in auto)
qed

lemma composition-fri-verifier-tied-authenticated-base-conflict-imp-partial-merkle:
  assumes base:
    composition-fri-verifier-tied-authenticated-base-opening-conflict
    s (Some (result, final-state))
  shows
    partial-merkle-inconsistency-bad s (Some (result, final-state))
proof -
  show ?thesis
proof (rule ccontr)
  assume no-bad:
    ¬ partial-merkle-inconsistency-bad s

```

```

    (Some (result, final-state))
obtain trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as trace-openings composition-openings
    trace-table composition-table round-idx xp xp-path xn xn-path
where aligned:
    accepted-with-partial-initial-openings-aligned s
    (Some (result, final-state)) fr trace-roots trace-final as fri-dg
    composition-roots composition-final fri-query-idxs trace-openings
    composition-openings
and comp-cand:
    partial-composition-table-candidate composition-table
    composition-openings
and round-bound: round-idx < length fri-query-idxs
and step:
    fri-layer-step-evidence
    (composition-roots ! 0)
    (composition-bs ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    1
    (fri-sibling-index (fri-evidence-layer-len composition-roots 0)
      (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0))
    xp xp-path xn xn-path
    (fri-evidence-next-idx composition-roots fri-query-idxs round-idx 0)
    (fri-evidence-next-value composition-roots composition-bs
      fri-query-idxs round-idx 0 xp xn)
    (composition-round-layers ! round-idx ! 0)
and no-match:
    ¬ fri-opening-matches-table
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    composition-table xp xn
and auth:
    fri-layer-chunk-authenticated (composition-roots ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    (composition-round-layers ! round-idx ! 0) final-state
using base
unfolding
    composition-fri-verifier-tied-authenticated-base-opening-conflict-def
by blast
have initial:
    accepted-with-partial-initial-openings s
    (Some (result, final-state)) fr trace-roots trace-final as fri-dg
    composition-roots composition-final fri-query-idxs trace-openings
    fri-query-idxs composition-openings
by (rule accepted-with-partial-initial-openings-aligned-imp-unaligned
    [OF aligned])

```



```

have comp-partial:
  accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-roots) fri-query-idxs
    composition-openings
  by (rule accepted-with-partial-initial-openings-shapes(4)
    [OF initial])
have roots-nonempty: composition-roots  $\neq []$ 
  by (rule accepted-with-partial-initial-openings-shapes(1)[OF initial])
have round-bound-rounds: round-idx < rounds
  using round-bound
    accepted-with-partial-composition-openings-shapes(1)[OF comp-partial]
  by simp
let ?idx =
  fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0
have idx-eq:
  ?idx = fri-query-idxs ! round-idx
  unfolding fri-evidence-layer-idx-def
  using roots-nonempty by (cases composition-roots) simp-all
have len-eq:
  fri-evidence-layer-len composition-roots 0 = scale * clength
  unfolding fri-evidence-layer-len-def
  using roots-nonempty by (cases composition-roots) simp-all
have root-eq: composition-roots ! 0 = hd composition-roots
  using roots-nonempty by (cases composition-roots) simp-all
have idxs:
  map opening-index (composition-openings ! round-idx) =
    [?idx, fri-sibling-index (scale * clength) ?idx]
  using accepted-with-partial-composition-openings-shapes(3)
    [OF comp-partial round-bound-rounds]
  unfolding idx-eq by simp
have comp-value:
  composition-table ! ?idx =
    opening-value ((composition-openings ! round-idx) ! 0)
  by (rule partial-composition-table-candidate-query-value
    [OF comp-cand round-bound-rounds idxs])
have len-openings: length (composition-openings ! round-idx) = 2
proof -
  have length (map opening-index (composition-openings ! round-idx)) =
    length [?idx, fri-sibling-index (scale * clength) ?idx]
    using arg-cong[OF idxs, of length] by simp
  then show ?thesis by simp
qed
have opn0-in:
  (composition-openings ! round-idx) ! 0
     $\in$  set (composition-openings ! round-idx)
  using len-openings by (cases composition-openings ! round-idx) auto
have opn1-in:
  (composition-openings ! round-idx) ! 1
     $\in$  set (composition-openings ! round-idx)

```

```

using len-openings
by (cases composition-openings ! round-idx; cases tl (composition-openings ! round-idx))
  auto
have comp-table:
  partial-authenticated-table (hd composition-roots) (scale * clength)
    (composition-openings ! round-idx) final-state
using comp-partial round-bound-rounds
unfolding accepted-with-partial-composition-openings-def by blast
have opn0-auth:
  authenticated-opening-in final-state
    ((composition-openings ! round-idx) ! 0)
using comp-table opn0-in
unfolding partial-authenticated-table-def by blast
have opn0-root:
  opening-root ((composition-openings ! round-idx) ! 0) =
    composition-roots ! 0
using comp-table opn0-in root-eq
unfolding partial-authenticated-table-def by simp
have opn0-len:
  opening-length ((composition-openings ! round-idx) ! 0) =
    fri-evidence-layer-len composition-roots 0
using comp-table opn0-in len-eq
unfolding partial-authenticated-table-def by simp
have opn0-idx:
  opening-index ((composition-openings ! round-idx) ! 0) = ?idx
using idxs len-openings by (cases composition-openings ! round-idx) auto
have opn0-value: opening-value ((composition-openings ! round-idx) ! 0) = xp
proof —
from step have chunk:
  fri-layer-opening-chunk
    (fri-evidence-layer-len composition-roots 0)
    xp xp-path xn xn-path
    (composition-round-layers ! round-idx ! 0)
unfolding fri-layer-step-evidence-def by blast
from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
obtain xp-path' xn-path' where xp-auth:
  authenticated-opening-in final-state
    (opening-root = composition-roots ! 0,
     opening-length = fri-evidence-layer-len composition-roots 0,
     opening-index =
       fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0,
     opening-value = xp,
     opening-path = xp-path')
by (metis)
have opening-value ((composition-openings ! round-idx) ! 0) =
  opening-value
    (opening-root = composition-roots ! 0,
     opening-length = fri-evidence-layer-len composition-roots 0,
     opening-index =

```

```

    fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0,
    opening-value = xp,
    opening-path = xp-path')
  by (rule authenticated-openings-values-eq-if-no-partial-merkle
      [OF no-bad opn0-auth xp-auth])
    (use opn0-root opn0-len opn0-idx in simp-all)
  then show ?thesis by simp
qed
have comp-match-base: composition-table ! ?idx = xp
  using comp-value opn0-value by simp
have comp-sibling:
  composition-table ! fri-sibling-index (scale * clength) ?idx =
    opening-value ((composition-openings ! round-idx) ! 1)
proof -
  have vals:
    map (!) composition-table
      [?idx, fri-sibling-index (scale * clength) ?idx] =
    map opening-value (composition-openings ! round-idx)
  by (rule partial-composition-table-candidate-query-values
      [OF comp-cand round-bound-rounds idxs])
  have lhs:
    map (!) composition-table
      [?idx, fri-sibling-index (scale * clength) ?idx] ! 1 =
    composition-table ! fri-sibling-index (scale * clength) ?idx
  by simp
  have rhs:
    map opening-value (composition-openings ! round-idx) ! 1 =
    opening-value ((composition-openings ! round-idx) ! 1)
  using len-openings by simp
  show ?thesis
    using arg-cong[OF vals, of  $\lambda x s. xs ! 1$ ] lhs rhs by simp
qed
have opn1-auth:
  authenticated-opening-in final-state
    ((composition-openings ! round-idx) ! 1)
  using comp-table opn1-in
  unfolding partial-authenticated-table-def by blast
have opn1-root:
  opening-root ((composition-openings ! round-idx) ! 1) =
    composition-roots ! 0
  using comp-table opn1-in root-eq
  unfolding partial-authenticated-table-def by simp
have opn1-len:
  opening-length ((composition-openings ! round-idx) ! 1) =
    fri-evidence-layer-len composition-roots 0
  using comp-table opn1-in len-eq
  unfolding partial-authenticated-table-def by simp
have opn1-idx:
  opening-index ((composition-openings ! round-idx) ! 1) =

```

```

    fri-sibling-index (fri-evidence-layer-len composition-roots 0) ?idx
using idxs len-openings len-eq
by (cases composition-openings ! round-idx; cases tl (composition-openings ! round-idx))
    auto
have opn1-value: opening-value ((composition-openings ! round-idx) ! 1) = xn
proof -
  from step have chunk:
    fri-layer-opening-chunk
      (fri-evidence-layer-len composition-roots 0)
      xp xp-path xn xn-path
      (composition-round-layers ! round-idx ! 0)
    unfolding fri-layer-step-evidence-def by blast
  from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
  obtain xp-path' xn-path' where xn-auth:
    authenticated-opening-in final-state
      (opening-root = composition-roots ! 0,
       opening-length = fri-evidence-layer-len composition-roots 0,
       opening-index =
         fri-sibling-index (fri-evidence-layer-len composition-roots 0)
           (fri-evidence-layer-idx composition-roots fri-query-idxs
            round-idx 0),
       opening-value = xn,
       opening-path = xn-path')
  by (metis)
  have opening-value ((composition-openings ! round-idx) ! 1) =
    opening-value
      (opening-root = composition-roots ! 0,
       opening-length = fri-evidence-layer-len composition-roots 0,
       opening-index =
         fri-sibling-index (fri-evidence-layer-len composition-roots 0)
           (fri-evidence-layer-idx composition-roots fri-query-idxs
            round-idx 0),
       opening-value = xn,
       opening-path = xn-path')
  by (rule authenticated-openings-values-eq-if-no-partial-merkle
       [OF no-bad opn1-auth xn-auth])
    (use opn1-root opn1-len opn1-idx in simp-all)
  then show ?thesis by simp
qed
have comp-match-sibling:
  composition-table !
    fri-sibling-index (fri-evidence-layer-len composition-roots 0) ?idx =
    xn
  using comp-sibling opn1-value len-eq by simp
have idx-bound: ?idx < fri-evidence-layer-len composition-roots 0
proof -
  from auth obtain xp0 xp0-path xn0 xn0-path where
    authenticated-opening-in final-state
      (opening-root = composition-roots ! 0,

```

```

    opening-length = fri-evidence-layer-len composition-roots 0,
    opening-index = ?idx,
    opening-value = xp0,
    opening-path = xp0-path)
  by (rule fri-layer-chunk-authenticatedE)
then show ?thesis
  unfolding authenticated-opening-in-def by simp
qed
have table-len:
  fri-evidence-layer-len composition-roots 0 ≤
    length composition-table
  using partial-composition-table-candidateD(1)
    [OF comp-cand round-bound-rounds opn0-in]
  unfolding len-eq by simp
have match:
  fri-opening-matches-table
    (fri-evidence-layer-len composition-roots 0)
    ?idx composition-table xp xn
  unfolding fri-opening-matches-table-def
  using idx-bound table-len comp-match-base comp-match-sibling
  by simp
then show False
  using no-match by contradiction
qed
qed

lemma wp-composition-fri-verifier-tied-authenticated-base-conflict-bound-by-partial-merkle:
  wp-event verify-monad
    (composition-fri-verifier-tied-authenticated-base-opening-conflict s) s
  ≤ wp-event verify-monad (partial-merkle-inconsistency-bad s) s
proof (rule wp-event-mono)
  fix out
  assume conflict:
    composition-fri-verifier-tied-authenticated-base-opening-conflict s out
  from conflict obtain result final-state where out-eq:
    out = Some (result, final-state)
  unfolding composition-fri-verifier-tied-authenticated-base-opening-conflict-def
  by blast
  show partial-merkle-inconsistency-bad s out
  unfolding out-eq
  by (rule composition-fri-verifier-tied-authenticated-base-conflict-imp-partial-merkle)
    (use conflict out-eq in simp)
qed

lemma wp-composition-fri-verifier-tied-sampled-base-bound-from-partial-merkle-and-auth-gap:
  fixes G :: prob
  assumes gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-base-chunk-auth-gap s) s ≤ G

```

shows

$$\begin{aligned} & \text{wp-event verify-monad} \\ & \quad (\text{composition-fri-verifier-tied-sampled-base-opening-conflict } s) \ s \\ & \leq \text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s + G \end{aligned}$$
proof (rule wp-composition-fri-verifier-tied-sampled-base-bound-from-authenticated-and-auth-gap)

show wp-event verify-monad

$$\begin{aligned} & \quad (\text{composition-fri-verifier-tied-authenticated-base-opening-conflict } s) \ s \\ & \leq \text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \end{aligned}$$
by (rule wp-composition-fri-verifier-tied-authenticated-base-conflict-bound-by-partial-merkle)

show wp-event verify-monad

$$(\text{composition-fri-verifier-tied-base-chunk-auth-gap } s) \ s \leq G$$
by (rule gap-bound)

qed

lemma wp-composition-fri-verifier-tied-without-same-bound-from-partial-merkle-auth-gap-and-branches:
fixes $G \ N \ S \ F :: \text{prob}$
assumes gap-bound:

$$\text{wp-event verify-monad } (\text{composition-fri-verifier-tied-base-chunk-auth-gap } s) \ s \leq G$$
and next-bound:

$$\text{wp-event verify-monad } (\text{composition-fri-verifier-tied-sampled-next-value-conflict } s) \ s \leq N$$
and successor-bound:

$$\begin{aligned} & \text{wp-event verify-monad} \\ & \quad (\text{composition-fri-verifier-tied-sampled-successor-opening-conflict } s) \\ & \quad s \leq S \end{aligned}$$
and final-bound:

$$\text{wp-event verify-monad } (\text{composition-fri-verifier-tied-sampled-final-value-conflict } s) \ s \leq F$$
shows

$$\begin{aligned} & \text{wp-event verify-monad} \\ & \quad (\text{composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer } s) \\ & \quad s \leq (\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s + G) + N + S + F \end{aligned}$$
by (rule wp-composition-fri-verifier-tied-without-same-bound-from-branches)
 (rule wp-composition-fri-verifier-tied-sampled-base-bound-from-partial-merkle-and-auth-gap
 [OF gap-bound],
 rule next-bound,
 rule successor-bound,
 rule final-bound)

lemma composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-imp-verifier-tied:
assumes

$$\begin{aligned} & \text{composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer} \\ & \quad s \ \text{out} \end{aligned}$$
shows composition-fri-verifier-tied-sampled-assignment-conflict $s \ \text{out}$

proof –
from **assms** **obtain** trace-roots trace-bs trace-final fri-dg
 composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers fr as trace-openings

```

    composition-openings trace-table composition-table result final-state
where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
and out-eq: out = Some (result, final-state)
and aligned:
    accepted-with-partial-initial-openings-aligned s out fr trace-roots
    trace-final as fri-dg composition-roots composition-final fri-query-idxs
    trace-openings composition-openings
and trace-cand:
    partial-trace-table-candidate trace-table trace-openings
and comp-cand:
    partial-composition-table-candidate composition-table
    composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-not-low:
    ¬ composition-table-low-degree maxDegree composition-table
and conflict:
    generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
    composition-table composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
    final-state
unfolding
    composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
by blast
have sampled:
    generic-fri-sampled-assignment-conflict composition-table
    composition-roots composition-bs composition-final fri-query-idxs
    composition-round-layers
by (rule
    generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-imp-sampled
    [OF conflict])
show ?thesis
unfolding composition-fri-verifier-tied-sampled-assignment-conflict-def
by (intro exI conjI)
    (rule fri-openings, rule aligned, rule trace-cand, rule comp-cand,
    rule trace-low, rule comp-not-low, rule sampled)
qed

lemma wp-composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-bound-from-verifi
assumes verifier-bound:
    wp-event verify-monad
    (composition-fri-verifier-tied-sampled-assignment-conflict s) s ≤ C
shows
    wp-event verify-monad
    (composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
    s ≤ C

```

proof –
have *wp-event verify-monad*
 (*composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s*)
 $s \leq$
wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-assignment-conflict s*) *s*
by (*rule wp-event-mono*)
 (*rule*
composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-imp-verifier-tied)
then show *?thesis*
by (*rule order-trans[OF - verifier-bound]*)
qed

definition *composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 (*unit list* $\times$ (*f, 'a*) *protocol-channel-scheme*) *option* $\Rightarrow$ *bool*

where

composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
 $s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table result final-state.
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers $\wedge$
 $\text{out} = \text{Some} (\text{result}, \text{final-state}) \wedge$
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
trace-table-low-degree trace-table $\wedge$
 $\neg \text{composition-table-low-degree } \text{maxDegree } \text{composition-table} \wedge$
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
composition-roots composition-bs fri-query-idxs
composition-round-layers final-state)

lemma *composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-imp-partial-merkle:*

assumes

composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
 $s \text{ out}$

shows *partial-merkle-inconsistency-bad s out*

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings


```

    composition-openings trace-table composition-table result final-state
where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
and out-eq: out = Some (result, final-state)
and conflict:
    generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    composition-roots composition-bs fri-query-idxs
    composition-round-layers final-state
unfolding
    composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-def
by blast
have len: length composition-bs = length composition-roots
    using accepted-fri-opening-transcript-shapes(4)[OF fri-openings] .
have partial-merkle-inconsistency-bad s (Some (result, final-state))
    by (rule
        generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-partial-merkle
        [OF conflict len])
then show ?thesis
    by (simp add: out-eq)
qed

lemma wp-composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-bound-from-partial-merkle:
assumes merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s)  $s \leq M$ 
shows
    wp-event verify-monad
        (composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks s)
         $s \leq M$ 
proof -
    have wp-event verify-monad
        (composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks s)
         $s \leq$  wp-event verify-monad (partial-merkle-inconsistency-bad s) s
    by (rule wp-event-mono)
        (rule
            composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-imp-partial-merkle)
then show ?thesis
    by (rule order-trans[OF - merkle-bound])
qed

lemma composition-fri-verifier-tied-authenticated-conflict-imp-without-same-or-same-layer:
assumes
    composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
    s out
shows
    composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
    s out  $\vee$ 

```

composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
s out
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
composition-openings trace-table composition-table result final-state
where *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *out-eq*: *out = Some (result, final-state)*
and *aligned*:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings
and *trace-cand*:
partial-trace-table-candidate trace-table trace-openings
and *comp-cand*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-not-low*:
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *conflict*:
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
composition-table composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
final-state
unfolding
composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
by *blast*
have *split*:
generic-fri-sampled-assignment-conflict-without-same-layer
composition-table composition-roots composition-bs composition-final
fri-query-idxs composition-round-layers $\vee$
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
composition-roots composition-bs fri-query-idxs composition-round-layers
final-state
by (*rule*
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-split
[*OF conflict*])
then show *?thesis*
proof
assume *without*:
generic-fri-sampled-assignment-conflict-without-same-layer
composition-table composition-roots composition-bs composition-final
fri-query-idxs composition-round-layers

then show *?thesis*
unfolding
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
by (*intro disjI1 exI conjI*)
(rule fri-openings, rule aligned, rule trace-cand, rule comp-cand,
rule trace-low, rule comp-not-low, rule without)
next
assume *same:*
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
composition-roots composition-bs fri-query-idxs
composition-round-layers final-state
then show *?thesis*
unfolding
composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-def
by (*intro disjI2 exI conjI*)
(rule fri-openings, rule out-eq, rule aligned, rule trace-cand,
rule comp-cand, rule trace-low, rule comp-not-low, rule same)
qed
qed

lemma *composition-fri-verifier-tied-authenticated-conflictI-without-same:*
assumes
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out
shows
composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out
proof —
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
composition-openings trace-table composition-table
where *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *aligned:*
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings
and *trace-cand:*
partial-trace-table-candidate trace-table trace-openings
and *comp-cand:*
partial-composition-table-candidate composition-table
composition-openings
and *trace-low:* *trace-table-low-degree trace-table*
and *comp-not-low:*
 $\neg$ *composition-table-low-degree maxDegree composition-table*

and *without*:
generic-fri-sampled-assignment-conflict-without-same-layer
composition-table composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
unfolding
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
by *blast*
from *fri-openings* **obtain** *result final-state* **where** *out-eq*:
out = Some (result, final-state)
unfolding *accepted-fri-opening-transcript-def* **by** *blast*
have *conflict*:
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
composition-table composition-roots composition-bs composition-final
fri-query-idxs composition-round-layers final-state
by (*rule*
generic-fri-sampled-assignment-conflict-with-authenticated-same-layerI-without
[OF without])
show *?thesis*
unfolding
composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
by (*intro exI conjI*)
(rule fri-openings, rule out-eq, rule aligned, rule trace-cand,
rule comp-cand, rule trace-low, rule comp-not-low, rule conflict)
qed

lemma *composition-fri-verifier-tied-authenticated-conflictI-same-layer*:
assumes
composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
s out
shows
composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out
proof —
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
composition-openings trace-table composition-table result final-state
where *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *out-eq*: *out = Some (result, final-state)*
and *aligned*:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings
and *trace-cand*:
partial-trace-table-candidate trace-table trace-openings

and *comp-cand*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-not-low*:
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *same*:
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
composition-roots composition-bs fri-query-idxs
composition-round-layers final-state
unfolding
composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-def
by *blast*
have *conflict*:
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
composition-table composition-roots composition-bs composition-final
fri-query-idxs composition-round-layers final-state
by (rule
generic-fri-sampled-assignment-conflict-with-authenticated-same-layerI-same
[OF *same*])
show *?thesis*
unfolding
composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
by (*intro exI conjI*)
(rule *fri-openings*, rule *out-eq*, rule *aligned*, rule *trace-cand*,
rule *comp-cand*, rule *trace-low*, rule *comp-not-low*, rule *conflict*)
qed

lemma *composition-fri-verifier-tied-authenticated-conflict-iff*:
composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out $\longleftrightarrow$
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out $\vee$
composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
s out
using *composition-fri-verifier-tied-authenticated-conflict-imp-without-same-or-same-layer*
composition-fri-verifier-tied-authenticated-conflictI-without-same
composition-fri-verifier-tied-authenticated-conflictI-same-layer
by *blast*

lemma *wp-composition-fri-verifier-tied-authenticated-conflict-bound-from-without-same-and-merkle*:
assumes *without-bound*:
wp-event verify-monad
(*composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s*)
s $\leq$ W
and *merkle-bound*:
wp-event verify-monad (partial-merkle-inconsistency-bad s) s $\leq$ M
shows
wp-event verify-monad

```

      (composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
      s ≤ W + M
proof –
  have wp-event verify-monad
    (composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
    s ≤
    wp-event verify-monad
      (λout.
        composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
          s out ∨
          composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
            s out) s
  by (rule wp-event-mono)
    (rule
      composition-fri-verifier-tied-authenticated-conflict-imp-without-same-or-same-layer)
  also have ... ≤
    wp-event verify-monad
      (composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s)
      s +
    wp-event verify-monad
      (composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks s)
      s
  by (rule wp-event-union-bound)
  also have ... ≤ W + M
  by (rule add-mono[OF without-bound
    wp-composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-bound-from-partial-merkle
    [OF merkle-bound]])
  finally show ?thesis .
qed

end

end

```

```

theory Soundness-FRI-Base-Recorded-Chunk
imports Soundness-FRI-Layer-Merkle
begin

```

Base-layer recorded chunk replay. This is the small case needed for the existing first-layer composition residual: the head chunk read by *receive-query-commits* is the first chunk in the extracted layer transcript, hence the recorded first layer is authenticated.

```

context soundness
begin

```

```

lemma mfold-fri-layer-opening-head-recorded-chunk-authenticated:
assumes outcome:
  Some (out, t) ∈
  set-dist

```

```

      (execute
        (mfold (idx, x, len, pw)
          (map fri-layer-opening-step ((b, rt) # bfs))) s)
    and layers:
      fri-layers-transcript (length ((b, rt) # bfs)) len layer-chunks
      total-chunk
    and transcript: PTranscript s = total-chunk @ PTranscript t
    and len-pos: 0 < len
    and idx-bound: idx < len
  shows
    fri-layer-chunk-authenticated rt len idx (layer-chunks ! 0) t
proof -
  from outcome obtain out1 s1 where
    head:
      Some (out1, s1) ∈
        set-dist
          (execute (fri-layer-opening-step (b, rt) (idx, x, len, pw)) s)
    and tail:
      Some (out, t) ∈
        set-dist (execute (mfold out1 (map fri-layer-opening-step bfs)) s1)
    by (auto elim!: set-dist-bindE)
  obtain xp xp-path xn xn-path x' head-chunk where
    out1-eq:
      out1 = (idx mod (len div 2), x', len div 2, pw + pw)
    and head-open:
      fri-layer-opening-chunk len xp xp-path xn xn-path head-chunk
    and tr-head: PTranscript s = head-chunk @ PTranscript s1
    and auth-xp:
      authenticated-opening-in s1
      (|opening-root = rt,
        opening-length = len,
        opening-index = idx,
        opening-value = xp,
        opening-path = xp-path)
    and auth-xn:
      authenticated-opening-in s1
      (|opening-root = rt,
        opening-length = len,
        opening-index = fri-sibling-index len idx,
        opening-value = xn,
        opening-path = xn-path)
    by (rule fri-layer-opening-step-outcome-authenticated
      [OF len-pos idx-bound head])
  from mfold-fri-layer-openings-outcome[OF tail[unfolded out1-eq]]
  obtain tail-layers tail-chunk where
    tail-layers:
      fri-layers-transcript (length bfs) (len div 2) tail-layers
      tail-chunk
    and tr-tail: PTranscript s1 = tail-chunk @ PTranscript t

```

```

    and ext-tail:  $s1 \leq t$ 
  by auto
let ?layers = head-chunk # tail-layers
let ?chunk = head-chunk @ tail-chunk
have exec-layers:
  fri-layers-transcript (length ((b, rt) # bfs)) len ?layers ?chunk
  using head-open tail-layers
  unfolding fri-layers-transcript-def by auto
have exec-transcript: PTranscript s = ?chunk @ PTranscript t
  using tr-head tr-tail by simp
have chunk-eq: ?chunk = total-chunk
  using exec-transcript transcript by simp
have layers-eq: ?layers = layer-chunks
  by (rule fri-layers-transcript-unique[OF exec-layers])
  (use layers chunk-eq in simp)
have head-auth-s1:
  fri-layer-chunk-authenticated rt len idx head-chunk s1
  by (rule fri-layer-chunk-authenticatedI[OF head-open auth-xp auth-xn])
have head-auth-t:
  fri-layer-chunk-authenticated rt len idx head-chunk t
  by (rule fri-layer-chunk-authenticated-mono[OF head-auth-s1 ext-tail])
show ?thesis
  using head-auth-t layers-eq[symmetric] by simp
qed

```

lemma *receive-query-commits-head-recorded-chunk-authenticated:*

assumes *outcome:*

Some (out, t) ∈
set-dist
(execute
(mfold (idx, x, len, pw)
(receive-query-commits ((b, rt) # bfs))) s)

and *layers:*

fri-layers-transcript (length ((b, rt) # bfs)) len layer-chunks
total-chunk

and *transcript:* PTranscript s = total-chunk @ PTranscript t

and *len-pos:* 0 < len

and *idx-bound:* idx < len

shows

fri-layer-chunk-authenticated rt len idx (layer-chunks ! 0) t

proof –

have *map-eq:*

receive-query-commits ((b, rt) # bfs) =
map fri-layer-opening-step ((b, rt) # bfs)
unfolding *receive-query-commits-def fri-layer-opening-step-def*
by (*simp add: fun-eq-iff split: prod.splits*)

show *?thesis*

by (*rule mfold-fri-layer-opening-head-recorded-chunk-authenticated*
[OF outcome[unfolded map-eq] layers transcript len-pos idx-bound])

qed

lemma *verifier-query-round-after-index-trace-base-recorded-chunk-authenticated:*

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes $f\text{-fl}\text{-eq}: f\text{-fl} = (b, rt) \# f\text{-fl}\text{-tail}$

and *outcome:*

$\text{Some } ((), t) \in$

set-dist

(execute

(verifier-query-round-after-index-program

fr f-fl f-final as fl final raw) s)

and *recorded:*

query-round-fri-layer-transcripts (index (to-nat raw))

(map snd f-fl) (map snd fl) round-chunk trace-layers

composition-layers

and *transcript:* $P\text{Transcript } s = \text{round-chunk} @ P\text{Transcript } t$

shows

*fri-layer-chunk-authenticated rt (clength * scale)*

(index (to-nat raw)) (trace-layers ! 0) t

proof –

let $?idx = \text{index } (to\text{-nat } raw)$

from *outcome* **obtain** $fv\ s1\ f\text{-out}\ s2\ s3\ c\text{-out}\ s4$ **where**

query-decommit:

$\text{Some } (fv, s1) \in$

set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)

and *trace-fri:*

$\text{Some } (f\text{-out}, s2) \in$

set-dist (execute

*(mfold (?idx, hd fv, clength * scale, 1)*

(receive-query-commits f-fl)) s1)

and *assert-trace:*

$\text{Some } ((), s3) \in$

set-dist

(execute

(assert (case f-out of (-, f-x, -, -) $\Rightarrow$ $f\text{-x} = f\text{-final}$)) s2)

and *comp-fri:*

$\text{Some } (c\text{-out}, s4) \in$

set-dist (execute

(mfold

*(?idx, cp-eval as fv ($h \wedge ?idx * \text{shift}$), clength * scale, 1)*

(receive-query-commits fl)) s3)

and *assert-comp:*

$\text{Some } ((), t) \in$

set-dist

(execute

(assert (case c-out of (-, x, -, -) $\Rightarrow$ $x = \text{final}$)) s4)

unfolding *verifier-query-round-after-index-program-def*

by *(auto simp: Let-def split: prod.splits elim!: set-dist-bindE)*

from *check-decommit-on-query-outcome[OF query-decommit]*

```

obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript ?idx fv query-paths query-chunk
  and tr-s: PTranscript s = query-chunk @ PTranscript s1
  by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  trace-layers-actual:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
  and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
  and ext-s1-s2: s1 ≤ s2
  by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers-actual:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk
  and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
  and ext-s3-s4: s3 ≤ s4
  by blast
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
let ?actual-chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have actual-recorded:
  query-round-fri-layer-transcripts ?idx (map snd f-fl) (map snd fl)
  ?actual-chunk trace-layer-chunks comp-layer-chunks
  unfolding query-round-fri-layer-transcripts-def
  by (intro exI[of - query-chunk] exI[of - trace-fri-chunk]
    exI[of - comp-fri-chunk] exI[of - fv] exI[of - query-paths] conjI)
    (use query-chunk-shape trace-layers-actual comp-layers-actual in
      ⟨simp-all add: mult.commute⟩)
have actual-transcript:
  PTranscript s = ?actual-chunk @ PTranscript t
  using tr-s tr-s1 tr-s3 s3-eq t-eq by simp
have actual-chunk-eq: ?actual-chunk = round-chunk
  using actual-transcript transcript by simp
have trace-layer-chunks-eq:
  trace-layer-chunks = trace-layers
  by (rule query-round-fri-layer-transcripts-unique(1)
    [OF actual-recorded[unfolded actual-chunk-eq] recorded])
have len-pos: 0 < clength * scale
  by (rule eval-domain-size-pos)

```

```

have idx-bound: ?idx < clength * scale
  by (rule index-less-domain)
have auth-s2:
  fri-layer-chunk-authenticated rt (clength * scale) ?idx
    (trace-layer-chunks ! 0) s2
  by (rule receive-query-commits-head-recorded-chunk-authenticated
    [OF trace-fri[unfolded f-fl-eq]
      trace-layers-actual[unfolded f-fl-eq] tr-s1 len-pos idx-bound])
have s2-t: s2 ≤ t
  using ext-s3-s4 s3-eq t-eq by simp
have auth-t:
  fri-layer-chunk-authenticated rt (clength * scale) ?idx
    (trace-layer-chunks ! 0) t
  by (rule fri-layer-chunk-authenticated-mono[OF auth-s2 s2-t])
show ?thesis
  using auth-t trace-layer-chunks-eq by simp
qed

lemma verifier-query-round-after-index-composition-base-recorded-chunk-authenticated:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-eq: fl = (b, rt) # fl-tail
and outcome:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s)
and recorded:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) round-chunk trace-layers
    composition-layers
and transcript: PTranscript s = round-chunk @ PTranscript t
shows
  fri-layer-chunk-authenticated rt (clength * scale)
    (index (to-nat raw)) (composition-layers ! 0) t
proof –
let ?idx = index (to-nat raw)
from outcome obtain fv s1 f-out s2 s3 c-out s4 where
  query-decommit:
    Some (fv, s1) ∈
      set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
and trace-fri:
  Some (f-out, s2) ∈
    set-dist (execute
      (mfold (?idx, hd fv, clength * scale, 1)
        (receive-query-commits f-fl)) s1)
and assert-trace:
  Some ((), s3) ∈
    set-dist

```

```

    (execute
      (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold
          (?idx, cp-eval as fv (h ^ ?idx * shift), clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some (( ), t) ∈
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript ?idx fv query-paths query-chunk
  and tr-s: PTranscript s = query-chunk @ PTranscript s1
  by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  trace-layers-actual:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
  and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
  by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers-actual:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk
  and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
  by blast
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
let ?actual-chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have actual-recorded:
  query-round-fri-layer-transcripts ?idx (map snd f-fl) (map snd fl)
  ?actual-chunk trace-layer-chunks comp-layer-chunks
  unfolding query-round-fri-layer-transcripts-def
  by (intro exI[of - query-chunk] exI[of - trace-fri-chunk])

```

```

    exI[of - comp-fri-chunk] exI[of - fv] exI[of - query-paths] conjI)
  (use query-chunk-shape trace-layers-actual comp-layers-actual in
    ⟨simp-all add: mult.commute⟩)
have actual-transcript:
  PTranscript s = ?actual-chunk @ PTranscript t
  using tr-s tr-s1 tr-s3 s3-eq t-eq by simp
have actual-chunk-eq: ?actual-chunk = round-chunk
  using actual-transcript transcript by simp
have comp-layer-chunks-eq:
  comp-layer-chunks = composition-layers
  by (rule query-round-fri-layer-transcripts-unique(2)
    [OF actual-recorded[unfolded actual-chunk-eq] recorded])
have len-pos: 0 < clength * scale
  by (rule eval-domain-size-pos)
have idx-bound: ?idx < clength * scale
  by (rule index-less-domain)
have auth-s4:
  fri-layer-chunk-authenticated rt (clength * scale) ?idx
    (comp-layer-chunks ! 0) s4
  by (rule receive-query-commits-head-recorded-chunk-authenticated
    [OF comp-fri[unfolded fl-eq] comp-layers-actual[unfolded fl-eq]
      tr-s3 len-pos idx-bound])
show ?thesis
  using auth-s4 comp-layer-chunks-eq t-eq by simp
qed

lemma accepted-fri-opening-transcript-trace-base-recorded-chunk-authenticated-at:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  and out-eq: out = Some (result, final-state)
  and round-bound: i < rounds
  and trace-nonempty: 0 < length trace-roots
  shows
    fri-layer-chunk-authenticated (trace-roots ! 0) (clength * scale)
      (query-idxs ! i) (trace-round-layers ! i ! 0) final-state
proof -
  from fri-openings obtain result' final-state' fr f-fl as fl query-state
    raw-idxs query-chunks where
    out-def: out = Some (result', final-state')
  and trace-roots-eq: trace-roots = map snd f-fl
  and composition-roots-eq: composition-roots = map snd fl
  and outcome:
    Some (result', final-state') ∈
      set-dist (execute
        (ntimes
          (verifier-query-round-program fr f-fl trace-final as fl
            composition-final)))

```

```

    rounds)
  query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state'
and query-chunk:
  ∧i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
and replay-lookup:
  ∧i. i < rounds ⇒
    fmlookup (HashMap final-state')
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
and recorded:
  ∧i. i < rounds ⇒
    query-round-fri-layer-transcripts (query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
    (trace-round-layers ! i) (composition-round-layers ! i)
unfolding accepted-fri-opening-transcript-def by blast
have result-eq: result' = result
  using out-def out-eq by simp
have final-state-eq: final-state' = final-state
  using out-def out-eq by simp
have outcome':
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
  using outcome result-eq final-state-eq by simp
have f-fl-nonempty: f-fl ≠ []
  using trace-nonempty trace-roots-eq by auto
then obtain bf f-fl-tail where f-fl-cons: f-fl = bf # f-fl-tail
  by (cases f-fl) auto
obtain b rt where bf-eq: bf = (b, rt)
  by (cases bf)
have f-fl-eq: f-fl = (b, rt) # f-fl-tail
  using f-fl-cons bf-eq by simp
have rt-eq: rt = trace-roots ! 0
  using f-fl-cons bf-eq trace-roots-eq by simp
let ?round =

```

```

    verifier-query-round-program fr f-fl trace-final as fl composition-final
from query-opening-ntimes-outcome-decomp-at[OF outcome' round-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist (execute (ntimes ?round i) query-state)
and round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
    Some (suffix, final-state) ∈
      set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
by blast
have x-eq: x = ()
  by (cases x) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
  fix prefix-query-idxs prefix-chunks
  assume len-prefix-idxs: length prefix-query-idxs = i
    and len-prefix-chunks: length prefix-chunks = i
    and transcript-prefix:
      PTranscript query-state =
        List.concat prefix-chunks @ PTranscript s-i
  and state-i:
    PState s-i =
      state-after-query-chunks (PState query-state) prefix-chunks i
  and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
  and prefix-ext: query-state ≤ s-i
  and prefix-rounds:
     $\bigwedge k. k < i \implies$ 
      verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
  from round[unfolded x-eq verifier-query-round-program-alt-def]
  obtain raw s0 where
    raw-out:
      Some (raw, s0) ∈
        set-dist (execute receive-query-index-challenge s-i)
  and after:
    Some ((), s-suc) ∈
      set-dist (execute
        (verifier-query-round-after-index-program fr f-fl trace-final as
          fl composition-final raw) s0)
  by (auto elim!: set-dist-bindE)
  have tr-s0: PTranscript s0 = PTranscript s-i
    using receive-query-index-challenge-outcome[OF raw-out] by simp
  show ?thesis
  proof (rule verifier-query-round-program-outcome[OF round[unfolded x-eq]])
    fix raw' idx chunk
    assume idx-eq: idx = index (to-nat raw')

```

```

and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
and transcript-round:
  PTranscript s-i = chunk @ PTranscript s-suc
and state-suc:
  PState s-suc = foldl concat (PState s-i) chunk
and round-ext: s-i ≤ s-suc
and counter-suc:
  PQueryCounter s-suc = Suc (PQueryCounter s-i)
and lookup-suc:
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw'
have raw-eq': raw = raw'
proof –
  have lookup-s0:
    fmlookup (HashMap s0)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw
    using receive-query-index-challenge-outcome[OF raw-out] by simp
have s0-suc: s0 ≤ s-suc
    using verifier-query-round-after-index-program-outcome[OF after]
    by simp
have lookup-suc-raw:
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw
    using hash-extension-lookup[OF lookup-s0 s0-suc] .
show ?thesis
    using lookup-suc lookup-suc-raw by simp
qed
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
  fix suffix-query-idxs suffix-chunks
  assume len-suffix-idxs:
    length suffix-query-idxs = rounds - Suc i
  and len-suffix-chunks:
    length suffix-chunks = rounds - Suc i
  and transcript-suffix:
    PTranscript s-suc =
      List.concat suffix-chunks @ PTranscript final-state
  and suffix-state:
    PState final-state =
      state-after-query-chunks (PState s-suc) suffix-chunks
      (rounds - Suc i)
  and suffix-counter:
    PQueryCounter final-state =

```



```

    PQueryCounter s-suc + (rounds - Suc i)
  and suffix-ext: s-suc ≤ final-state
  and suffix-rounds:
    ∧k. k < rounds - Suc i ⇒
      verifier-query-round-chunk (suffix-query-idxs ! k)
        (map snd f-fl) (map snd fl) (suffix-chunks ! k)
  let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
  have len-selected-chunks: length ?selected-chunks = rounds
    using len-prefix-chunks len-suffix-chunks round-bound by simp
  have selected-chunk-lens:
    ∧k. k < rounds ⇒
      length (?selected-chunks ! k) =
        verifier-query-round-transcript-length (query-idxs ! k)
          (map snd f-fl) (map snd fl)
  proof -
    fix k
    assume k-bound: k < rounds
    show
      length (?selected-chunks ! k) =
        verifier-query-round-transcript-length (query-idxs ! k)
          (map snd f-fl) (map snd fl)
    proof (cases k < i)
      case True
      have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
        using True len-prefix-chunks by (simp add: nth-append)
      have
        length (prefix-chunks ! k) =
          verifier-query-round-transcript-length
            (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
        by (rule verifier-query-round-chunk-length
            [OF prefix-rounds[OF True]])
      then show ?thesis
        using nth-eq
        by (simp add:
            verifier-query-round-transcript-length-index-irrelevant)
    next
      case False
      show ?thesis
      proof (cases k = i)
        case True
        have nth-eq: ?selected-chunks ! k = chunk
          using True len-prefix-chunks by (simp add: nth-append)
        have
          length chunk =
            verifier-query-round-transcript-length idx
              (map snd f-fl) (map snd fl)
          by (rule verifier-query-round-chunk-length[OF chunk-shape])
        then show ?thesis
          using nth-eq

```

```

    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
next
case False
let ?k = k - Suc i
have ge: Suc i ≤ k
    using ⟨¬ k < i⟩ False by linarith
have k-bound': ?k < rounds - Suc i
    using ge k-bound by linarith
have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
    using ge len-prefix-chunks by (simp add: nth-append)
have
    length (suffix-chunks ! ?k) =
        verifier-query-round-transcript-length
            (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound']])
then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-lens:
    ∧k. k < rounds ⇒
        length (query-chunks ! k) =
            verifier-query-round-transcript-length (query-idxs ! k)
                (map snd f-fl) (map snd fl)
    using query-chunk verifier-query-round-chunk-length by blast
have selected-transcript:
    PTranscript query-state =
        List.concat ?selected-chunks @ PTranscript final-state
    using transcript-prefix transcript-round transcript-suffix by simp
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
    using selected-transcript transcript-query final-state-eq by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
    have same-lens:
        ∧k. k < length ?selected-chunks ⇒
            length (?selected-chunks ! k) = length (query-chunks ! k)
        using selected-chunk-lens query-chunk-lens len-selected-chunks
        by simp
    have concat-eq-empty:
        List.concat ?selected-chunks @ [] = List.concat query-chunks
        using concat-eq by simp
    have len-eq: length ?selected-chunks = length query-chunks
        using len-selected-chunks len-query-chunks by simp
    have ?selected-chunks = query-chunks ∧ ([] :: 'f list) = []

```

```

    by (rule concat-append-eq-concat-same-chunk-lengths
        [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
      state-after-query-chunks (PState query-state) prefix-chunks i
    by (rule state-after-query-chunks-append-prefix
        [OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some raw'
proof -
  have lookup-suc':
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
    Some raw'
    using lookup-suc state-i counter-i by simp
  have lookup-final-prefix:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
    Some raw'
    using hash-extension-lookup[OF lookup-suc' suffix-ext] .
  show ?thesis
    using lookup-final-prefix prefix-state-eq by simp
qed
have raw-idx-eq: raw' = raw-idxs ! i
  using lookup-final replay-lookup[OF round-bound] final-state-eq
  by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-idx-eq query-idxs-eq len-raw round-bound by simp
have selected-i: ?selected-chunks ! i = chunk
  using len-prefix-chunks by (simp add: nth-append)
have chunk-i-eq: chunk = query-chunks ! i
  using chunks-eq selected-i by simp

```

```

have recorded-i:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) chunk (trace-round-layers ! i)
    (composition-round-layers ! i)
using recorded[OF round-bound] idx-eq idx-eq-query raw-eq'
  chunk-i-eq trace-roots-eq composition-roots-eq
by simp
have tr-s0-suc:
  PTranscript s0 = chunk @ PTranscript s-suc
using tr-s0 transcript-round by simp
have auth-suc:
  fri-layer-chunk-authenticated rt (clength * scale)
    (index (to-nat raw)) (trace-round-layers ! i ! 0) s-suc
by (rule
  verifier-query-round-after-index-trace-base-recorded-chunk-authenticated
    [OF f-fl-eq after recorded-i tr-s0-suc])
have auth-final:
  fri-layer-chunk-authenticated rt (clength * scale)
    (index (to-nat raw)) (trace-round-layers ! i ! 0)
    final-state
by (rule fri-layer-chunk-authenticated-mono[OF auth-suc suffix-ext])
have raw-idx-query: index (to-nat raw) = query-idxs ! i
using idx-eq idx-eq-query raw-eq' by simp
show ?thesis
using auth-final rt-eq raw-idx-query by simp
qed
qed
qed
qed

lemma accepted-fri-opening-transcript-composition-base-recorded-chunk-authenticated-at:
assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers
and out-eq: out = Some (result, final-state)
and round-bound: i < rounds
and composition-nonempty: 0 < length composition-roots
shows
  fri-layer-chunk-authenticated (composition-roots ! 0) (clength * scale)
    (query-idxs ! i) (composition-round-layers ! i ! 0) final-state
proof -
from fri-openings obtain result' final-state' fr f-fl as fl query-state
  raw-idxs query-chunks where
  out-def: out = Some (result', final-state')
and trace-roots-eq: trace-roots = map snd f-fl
and composition-roots-eq: composition-roots = map snd fl
and outcome:
  Some (result', final-state') ∈

```

```

    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-query-chunks: length query-chunks = rounds
  and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state'
  and query-chunk:
    ∧ i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
  and replay-lookup:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap final-state')
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
  and recorded:
    ∧ i. i < rounds ⇒
      query-round-fri-layer-transcripts (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)
      (trace-round-layers ! i) (composition-round-layers ! i)
  unfolding accepted-fri-opening-transcript-def by blast
  have result-eq: result' = result
  using out-def out-eq by simp
  have final-state-eq: final-state' = final-state
  using out-def out-eq by simp
  have outcome':
    Some (result, final-state) ∈
      set-dist (execute
        (ntimes
          (verifier-query-round-program fr f-fl trace-final as fl
            composition-final)
          rounds)
        query-state)
  using outcome result-eq final-state-eq by simp
  have fl-nonempty: fl ≠ []
  using composition-nonempty composition-roots-eq by auto
  then obtain bf fl-tail where fl-cons: fl = bf # fl-tail
  by (cases fl) auto
  obtain b rt where bf-eq: bf = (b, rt)
  by (cases bf)
  have fl-eq: fl = (b, rt) # fl-tail

```

```

using fl-cons bf-eq by simp
have rt-eq: rt = composition-roots ! 0
using fl-cons bf-eq composition-roots-eq by simp
let ?round =
  verifier-query-round-program fr f-fl trace-final as fl composition-final
from query-opening-ntimes-outcome-decomp-at[OF outcome' round-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist (execute (ntimes ?round i) query-state)
and round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
    Some (suffix, final-state) ∈
      set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
by blast
have x-eq: x = ()
by (cases x) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
fix prefix-query-idxs prefix-chunks
assume len-prefix-idxs: length prefix-query-idxs = i
and len-prefix-chunks: length prefix-chunks = i
and transcript-prefix:
  PTranscript query-state =
    List.concat prefix-chunks @ PTranscript s-i
and state-i:
  PState s-i =
    state-after-query-chunks (PState query-state) prefix-chunks i
and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
and prefix-ext: query-state ≤ s-i
and prefix-rounds:
   $\bigwedge k. k < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
from round[unfolded x-eq verifier-query-round-program-alt-def]
obtain raw s0 where
  raw-out:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s-i)
and after:
    Some ((), s-suc) ∈
      set-dist (execute
        (verifier-query-round-after-index-program fr f-fl trace-final as
          fl composition-final raw) s0)
by (auto elim!: set-dist-bindE)
have tr-s0: PTranscript s0 = PTranscript s-i
using receive-query-index-challenge-outcome[OF raw-out] by simp

```

```

show ?thesis
proof (rule verifier-query-round-program-outcome[OF round[unfolded x-eq]])
  fix raw' idx chunk
  assume idx-eq: idx = index (to-nat raw')
  and chunk-shape:
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
  and transcript-round:
    PTranscript s-i = chunk @ PTranscript s-suc
  and state-suc:
    PState s-suc = foldl concat (PState s-i) chunk
  and round-ext: s-i ≤ s-suc
  and counter-suc:
    PQueryCounter s-suc = Suc (PQueryCounter s-i)
  and lookup-suc:
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw'
  have raw-eq': raw = raw'
  proof -
    have lookup-s0:
      fmlookup (HashMap s0)
        (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
        Some raw
    using receive-query-index-challenge-outcome[OF raw-out] by simp
  have s0-suc: s0 ≤ s-suc
    using verifier-query-round-after-index-program-outcome[OF after]
    by simp
  have lookup-suc-raw:
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw
    using hash-extension-lookup[OF lookup-s0 s0-suc] .
  show ?thesis
    using lookup-suc lookup-suc-raw by simp
qed
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
  fix suffix-query-idxs suffix-chunks
  assume len-suffix-idxs:
    length suffix-query-idxs = rounds - Suc i
  and len-suffix-chunks:
    length suffix-chunks = rounds - Suc i
  and transcript-suffix:
    PTranscript s-suc =
      List.concat suffix-chunks @ PTranscript final-state
  and suffix-state:
    PState final-state =

```

```

    state-after-query-chunks (PState s-suc) suffix-chunks
      (rounds - Suc i)
  and suffix-counter:
    PQueryCounter final-state =
      PQueryCounter s-suc + (rounds - Suc i)
  and suffix-ext: s-suc ≤ final-state
  and suffix-rounds:
     $\bigwedge k. k < rounds - Suc\ i \implies$ 
      verifier-query-round-chunk (suffix-query-idxs ! k)
        (map snd f-fl) (map snd fl) (suffix-chunks ! k)
  let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
  have len-selected-chunks: length ?selected-chunks = rounds
    using len-prefix-chunks len-suffix-chunks round-bound by simp
  have selected-chunk-lens:
     $\bigwedge k. k < rounds \implies$ 
      length (?selected-chunks ! k) =
        verifier-query-round-transcript-length (query-idxs ! k)
          (map snd f-fl) (map snd fl)
  proof -
    fix k
    assume k-bound: k < rounds
    show
      length (?selected-chunks ! k) =
        verifier-query-round-transcript-length (query-idxs ! k)
          (map snd f-fl) (map snd fl)
  proof (cases k < i)
    case True
      have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
        using True len-prefix-chunks by (simp add: nth-append)
      have
        length (prefix-chunks ! k) =
          verifier-query-round-transcript-length
            (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
        by (rule verifier-query-round-chunk-length
            [OF prefix-rounds[OF True]])
      then show ?thesis
        using nth-eq
        by (simp add:
            verifier-query-round-transcript-length-index-irrelevant)
    next
      case False
      show ?thesis
    proof (cases k = i)
      case True
        have nth-eq: ?selected-chunks ! k = chunk
          using True len-prefix-chunks by (simp add: nth-append)
        have
          length chunk =
            verifier-query-round-transcript-length idx

```



```

      (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF chunk-shape])
  then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
  next
  case False
  let ?k = k - Suc i
  have ge: Suc i ≤ k
    using ⟨¬ k < i⟩ False by linarith
  have k-bound': ?k < rounds - Suc i
    using ge k-bound by linarith
  have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
    using ge len-prefix-chunks by (simp add: nth-append)
  have
    length (suffix-chunks ! ?k) =
      verifier-query-round-transcript-length
        (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound']])
  then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-lens:
  ∧k. k < rounds ⇒
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  using query-chunk verifier-query-round-chunk-length by blast
have selected-transcript:
  PTranscript query-state =
    List.concat ?selected-chunks @ PTranscript final-state
  using transcript-prefix transcript-round transcript-suffix by simp
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query final-state-eq by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
    ∧k. k < length ?selected-chunks ⇒
      length (?selected-chunks ! k) = length (query-chunks ! k)
    using selected-chunk-lens query-chunk-lens len-selected-chunks
    by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks

```

```

    using concat-eq by simp
  have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks  $\wedge$  ( $\square :: 'f \text{ list}$ ) =  $\square$ 
    by (rule concat-append-eq-concat-same-chunk-lengths
        [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
  state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
    state-after-query-chunks (PState query-state) prefix-chunks i
  by (rule state-after-query-chunks-append-prefix
      [OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
  Some raw'
proof -
  have lookup-suc':
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
    Some raw'
  using lookup-suc state-i counter-i by simp
  have lookup-final-prefix:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
    Some raw'
  using hash-extension-lookup[OF lookup-suc' suffix-ext] .
  show ?thesis
    using lookup-final-prefix prefix-state-eq by simp
qed
have raw-idx-eq: raw' = raw-idxs ! i
  using lookup-final replay-lookup[OF round-bound] final-state-eq
  by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-idx-eq query-idxs-eq len-raw round-bound by simp

```

```

have selected-i: ?selected-chunks ! i = chunk
  using len-prefix-chunks by (simp add: nth-append)
have chunk-i-eq: chunk = query-chunks ! i
  using chunks-eq selected-i by simp
have recorded-i:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) chunk (trace-round-layers ! i)
    (composition-round-layers ! i)
  using recorded[OF round-bound] idx-eq idx-eq-query raw-eq'
    chunk-i-eq trace-roots-eq composition-roots-eq
  by simp
have tr-s0-suc:
  PTranscript s0 = chunk @ PTranscript s-suc
  using tr-s0 transcript-round by simp
have auth-suc:
  fri-layer-chunk-authenticated rt (clength * scale)
    (index (to-nat raw)) (composition-round-layers ! i ! 0) s-suc
  by (rule
    verifier-query-round-after-index-composition-base-recorded-chunk-authenticated
      [OF fl-eq after recorded-i tr-s0-suc])
have auth-final:
  fri-layer-chunk-authenticated rt (clength * scale)
    (index (to-nat raw)) (composition-round-layers ! i ! 0)
    final-state
  by (rule fri-layer-chunk-authenticated-mono[OF auth-suc suffix-ext])
have raw-idx-query: index (to-nat raw) = query-idxs ! i
  using idx-eq idx-eq-query raw-eq' by simp
show ?thesis
  using auth-final rt-eq raw-idx-query by simp
qed
qed
qed
qed

end

end

```

```

theory Soundness-FRI-Raw-Layer-Bounds
  imports Soundness-FRI-Authenticated-Route
begin

```

Small reusable bounds for FRI layer indices derived from accepted verifier transcripts. These keep later replay proofs from repeatedly rebuilding the same *fri-layer-raw-bound* arithmetic.

```

context soundness
begin

```

lemma *accepted-fri-opening-transcript-trace-raw-layer-bound*:
assumes *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers
and *layer-bound*: $j < \text{length } \text{trace-roots}$
shows
 $\bigwedge_{\text{raw.}}$
 $0 < \text{fri-layer-lengths } (\text{length } \text{trace-roots}) (\text{clength} * \text{scale}) ! j \wedge$
 $\text{fri-layer-indices } (\text{length } \text{trace-roots}) (\text{index } (\text{to-nat } \text{raw}))$
 $(\text{clength} * \text{scale}) ! j <$
 $\text{fri-layer-lengths } (\text{length } \text{trace-roots}) (\text{clength} * \text{scale}) ! j$
proof –
obtain N **where** *eval-power*: $\text{clength} * \text{scale} = 2^N$
using *eval-domain-length-power* **by** *blast*
have *trace-roots-le*: $\text{length } \text{trace-roots} \leq N$
proof –
have *len*: $\text{length } \text{trace-roots} = \text{ceil-log } \text{clength}$
using *accepted-fri-opening-transcript-shapes(1)[OF fri-openings]* .
have $\text{clength} \leq \text{clength} * \text{scale}$
using *scale-pos* **by** *simp*
also have $\dots = 2^N$
by (*rule eval-power*)
finally have $\text{ceil-log } \text{clength} \leq N$
by (*rule ceil-log-le-power*)
then show *?thesis*
using *len* **by** *simp*
qed
show $\bigwedge_{\text{raw.}}$
 $0 < \text{fri-layer-lengths } (\text{length } \text{trace-roots}) (\text{clength} * \text{scale}) ! j \wedge$
 $\text{fri-layer-indices } (\text{length } \text{trace-roots}) (\text{index } (\text{to-nat } \text{raw}))$
 $(\text{clength} * \text{scale}) ! j <$
 $\text{fri-layer-lengths } (\text{length } \text{trace-roots}) (\text{clength} * \text{scale}) ! j$
by (*rule fri-layer-raw-bound[OF layer-bound eval-power trace-roots-le]*)
qed

lemma *accepted-fri-opening-transcript-composition-raw-layer-bound*:
assumes *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers
and *degree-bound*: $\text{to-nat } dg \leq \text{maxDegree}$
and *layer-bound*: $j < \text{length } \text{composition-roots}$
shows
 $\bigwedge_{\text{raw.}}$
 $0 < \text{fri-layer-lengths } (\text{length } \text{composition-roots}) (\text{clength} * \text{scale}) ! j \wedge$
 $\text{fri-layer-indices } (\text{length } \text{composition-roots}) (\text{index } (\text{to-nat } \text{raw}))$
 $(\text{clength} * \text{scale}) ! j <$
 $\text{fri-layer-lengths } (\text{length } \text{composition-roots}) (\text{clength} * \text{scale}) ! j$

```

proof –
  obtain  $N$  where  $eval\_power: length * scale = 2 \wedge N$ 
    using  $eval\_domain\_length\_power$  by  $blast$ 
  have  $composition\_roots\_le: length\ composition\_roots \leq N$ 
  proof –
    have  $len: length\ composition\_roots = \text{ceil-log}\ (to\_nat\ dg + 1)$ 
      using  $accepted\_fri\_opening\_transcript\_shapes(3)[OF\ fri\_openings]$  .
    have  $to\_nat\ dg + 1 \leq length * scale$ 
      using  $degree\_bound\ maxDegree\_less\_eval\_domain$  by  $linarith$ 
    also have  $\dots = 2 \wedge N$ 
      by  $(rule\ eval\_power)$ 
    finally have  $\text{ceil-log}\ (to\_nat\ dg + 1) \leq N$ 
      by  $(rule\ \text{ceil-log-le-power})$ 
    then show  $?thesis$ 
      using  $len$  by  $simp$ 
  qed
show  $\bigwedge raw.$ 
   $0 < fri\_layer\_lengths\ (length\ composition\_roots)$ 
   $(length * scale) ! j \wedge$ 
   $fri\_layer\_indices\ (length\ composition\_roots)\ (index\ (to\_nat\ raw))$ 
   $(length * scale) ! j <$ 
   $fri\_layer\_lengths\ (length\ composition\_roots)\ (length * scale) ! j$ 
by  $(rule\ fri\_layer\_raw\_bound$ 
   $[OF\ layer\_bound\ eval\_power\ composition\_roots\_le])$ 
qed

lemma  $accepted\_fri\_opening\_transcript\_trace\_step\_with\_authenticated\_chunk\_at\_raw:$ 
assumes  $fri\_openings:$ 
   $accepted\_fri\_opening\_transcript\ s\ out\ trace\_roots\ trace\_bs\ trace\_final$ 
   $dg\ composition\_roots\ composition\_bs\ composition\_final\ query\_idxs$ 
   $trace\_round\_layers\ composition\_round\_layers$ 
and  $out\_eq: out = Some\ (result, final\_state)$ 
and  $round\_bound: i < rounds$ 
and  $layer\_bound: j < length\ trace\_roots$ 
shows
   $generic\_fri\_transcript\_step\_with\_authenticated\_chunk\ trace\_roots$ 
   $trace\_bs\ query\_idxs\ trace\_round\_layers\ final\_state\ i\ j$ 
by  $(rule\ accepted\_fri\_opening\_transcript\_trace\_step\_with\_authenticated\_chunk\_at$ 
   $[OF\ fri\_openings\ out\_eq\ round\_bound\ layer\_bound])$ 
   $(rule\ accepted\_fri\_opening\_transcript\_trace\_raw\_layer\_bound$ 
   $[OF\ fri\_openings\ layer\_bound])$ 

lemma  $accepted\_fri\_opening\_transcript\_composition\_step\_with\_authenticated\_chunk\_at\_raw:$ 
assumes  $fri\_openings:$ 
   $accepted\_fri\_opening\_transcript\ s\ out\ trace\_roots\ trace\_bs\ trace\_final$ 
   $dg\ composition\_roots\ composition\_bs\ composition\_final\ query\_idxs$ 
   $trace\_round\_layers\ composition\_round\_layers$ 
and  $out\_eq: out = Some\ (result, final\_state)$ 
and  $degree\_bound: to\_nat\ dg \leq maxDegree$ 

```

```

and round-bound:  $i < \text{rounds}$ 
and layer-bound:  $j < \text{length composition-roots}$ 
shows
  generic-fri-transcript-step-with-authenticated-chunk composition-roots
    composition-bs query-idxs composition-round-layers final-state  $i\ j$ 
by (rule
  accepted-fri-opening-transcript-composition-step-with-authenticated-chunk-at
    [OF fri-openings out-eq round-bound layer-bound])
  (rule accepted-fri-opening-transcript-composition-raw-layer-bound
    [OF fri-openings degree-bound layer-bound])

```

end

end

```

theory Soundness-FRI-Final-Step-Replay
  imports Soundness-FRI-Raw-Layer-Bounds
begin

```

Selected final-step replay for trace FRI openings.

This layer lifts the monadic replay fact for the last trace FRI fold step through the accepted FRI transcript. It keeps the result local to sampled verifier evidence and does not reconstruct complete FRI tables.

```

context soundness
begin

```

```

lemma accepted-fri-opening-transcript-trace-final-step-evidence-at:
  assumes fri-openings:
    accepted-fri-opening-transcript  $s\ out\ trace\ roots\ trace\ bs\ trace\ final$ 
       $dg\ composition\ roots\ composition\ bs\ composition\ final\ query\ idxs$ 
       $trace\ round\ layers\ composition\ round\ layers$ 
  and out-eq:  $out = \text{Some } (result, final\ state)$ 
  and round-bound:  $i < \text{rounds}$ 
  and nonempty:  $0 < \text{length } trace\ roots$ 
  and raw-layer:
     $\bigwedge raw.$ 
       $0 < \text{fri-layer-lengths } (length\ trace\ roots) (clength * scale) !$ 
       $(length\ trace\ roots - 1) \wedge$ 
       $\text{fri-layer-indices } (length\ trace\ roots) (\text{index } (to\ nat\ raw))$ 
       $(clength * scale) ! (length\ trace\ roots - 1) <$ 
       $\text{fri-layer-lengths } (length\ trace\ roots) (clength * scale) !$ 
       $(length\ trace\ roots - 1)$ 
  obtains actual-chunk  $axp\ axp\ path\ axn\ axn\ path$  where
    fri-layer-chunk-authenticated
       $(trace\ roots ! (length\ trace\ roots - 1))$ 
       $(\text{fri-evidence-layer-len } trace\ roots (length\ trace\ roots - 1))$ 
       $(\text{fri-evidence-layer-idx } trace\ roots\ query\ idxs\ i$ 
         $(length\ trace\ roots - 1))$ 

```

```

    actual-chunk final-state
  fri-layer-step-evidence
    (trace-roots ! (length trace-roots - 1))
    (trace-bs ! (length trace-roots - 1))
    (fri-evidence-layer-len trace-roots (length trace-roots - 1))
    (fri-evidence-layer-idx trace-roots query-idxs i
      (length trace-roots - 1))
    (2 ^ (length trace-roots - 1))
    (fri-sibling-index
      (fri-evidence-layer-len trace-roots (length trace-roots - 1))
      (fri-evidence-layer-idx trace-roots query-idxs i
        (length trace-roots - 1)))
    axp axp-path axn axn-path
    (fri-evidence-next-idx trace-roots query-idxs i
      (length trace-roots - 1))
    trace-final actual-chunk
proof -
  from fri-openings obtain result' final-state' fr f-fl as fl query-state
    raw-idxs query-chunks where
    out-def: out = Some (result', final-state')
  and trace-roots-eq: trace-roots = map snd f-fl
  and trace-bs-eq: trace-bs = map fst f-fl
  and composition-roots-eq: composition-roots = map snd fl
  and query-out:
    Some (result', final-state') ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl trace-final as fl
              composition-final)
            rounds)
          query-state)
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-query-chunks: length query-chunks = rounds
  and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state'
  and query-chunk:
    ∧ i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
        trace-roots composition-roots (query-chunks ! i)
  and replay-lookup:
    ∧ i. i < rounds ⇒
      fmlookup (HashMap final-state')
        (QueryIndexChallenge (PQueryCounter query-state + i)
          (state-after-query-chunks (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)

```

```

unfolding accepted-fri-opening-transcript-def by blast
have result-eq: result' = result
using out-def out-eq by simp
have final-state-eq: final-state' = final-state
using out-def out-eq by simp
have query-out':
  Some (result, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl trace-final as fl
            composition-final)
          rounds)
        query-state)
using query-out result-eq final-state-eq by simp
have f-fl-nonempty: 0 < length f-fl
using nonempty trace-roots-eq by simp
have raw-layer-f:
   $\bigwedge$ raw.
    0 < fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1)  $\wedge$ 
      fri-layer-indices (length f-fl) (index (to-nat raw))
        (clength * scale) ! (length f-fl - 1) <
      fri-layer-lengths (length f-fl) (clength * scale) !
        (length f-fl - 1)
using raw-layer trace-roots-eq by simp
from ntimes-verifier-query-rounds-selected-trace-fri-last-step-with-prefix-at
  [OF query-out' round-bound f-fl-nonempty raw-layer-f]
obtain raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
  suffix-chunks actual-chunk axp axp-path axn axn-path where
idx-eq: idx = index (to-nat raw)
and len-prefix-idxs: length prefix-query-idxs = i
and len-prefix-chunks: length prefix-chunks = i
and len-suffix-idxs: length suffix-query-idxs = rounds - Suc i
and len-suffix-chunks: length suffix-chunks = rounds - Suc i
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
and prefix-rounds:
   $\bigwedge$ k. k < i  $\implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
and suffix-rounds:
   $\bigwedge$ k. k < rounds - Suc i  $\implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
and selected-transcript:
  PTranscript query-state =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript final-state

```



```

and auth:
  fri-layer-chunk-authenticated
    (snd (f-fl ! (length f-fl - 1)))
    (fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1))
    (fri-layer-indices (length f-fl) idx (clength * scale) !
      (length f-fl - 1))
    actual-chunk final-state
and step:
  fri-layer-step-evidence
    (snd (f-fl ! (length f-fl - 1)))
    (fst (f-fl ! (length f-fl - 1)))
    (fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1))
    (fri-layer-indices (length f-fl) idx (clength * scale) !
      (length f-fl - 1))
    (2 ^ (length f-fl - 1))
    (fri-sibling-index
      (fri-layer-lengths (length f-fl) (clength * scale) !
        (length f-fl - 1))
      (fri-layer-indices (length f-fl) idx (clength * scale) !
        (length f-fl - 1)))
    axp axp-path axn axn-path
    ((fri-layer-indices (length f-fl) idx (clength * scale) !
      (length f-fl - 1)) mod
      ((fri-layer-lengths (length f-fl) (clength * scale) !
        (length f-fl - 1)) div 2))
    trace-final actual-chunk
and selected-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) prefix-chunks i)) =
    Some raw
by (rule
  ntimes-verifier-query-rounds-selected-trace-fri-last-step-with-prefix-at
    [OF query-out' round-bound f-fl-nonempty raw-layer-f])
  (rule that, assumption+)
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
using len-prefix-chunks len-suffix-chunks round-bound by simp
have selected-chunk-lens:
   $\bigwedge k. k < \text{rounds} \implies$ 
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound: k < rounds
  show

```

```

length (?selected-chunks ! k) =
  verifier-query-round-transcript-length (query-idxs ! k)
  (map snd f-fl) (map snd fl)
proof (cases k < i)
  case True
  have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length (prefix-chunks ! k) =
      verifier-query-round-transcript-length
        (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF prefix-rounds[OF True]])
  then show ?thesis
    using nth-eq
    by (simp add: verifier-query-round-transcript-length-index-irrelevant)
next
  case False
  show ?thesis
  proof (cases k = i)
    case True
    have nth-eq: ?selected-chunks ! k = chunk
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length chunk =
        verifier-query-round-transcript-length idx
        (map snd f-fl) (map snd fl)
      by (rule verifier-query-round-chunk-length[OF chunk-shape])
    then show ?thesis
      using nth-eq
      by (simp add: verifier-query-round-transcript-length-index-irrelevant)
  next
  case False
  let ?k = k - Suc i
  have ge: Suc i ≤ k
    using <¬ k < i> False by linarith
  have k-bound': ?k < rounds - Suc i
    using ge k-bound by linarith
  have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
    using ge len-prefix-chunks by (simp add: nth-append)
  have
    length (suffix-chunks ! ?k) =
      verifier-query-round-transcript-length
        (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
      [OF suffix-rounds[OF k-bound']])
  then show ?thesis
    using nth-eq
    by (simp add: verifier-query-round-transcript-length-index-irrelevant)
qed

```

```

qed
qed
have query-chunk-lens:
   $\bigwedge k. k < \text{rounds} \implies$ 
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound: k < rounds
  have
    verifier-query-round-chunk (query-idxs ! k)
      (map snd f-fl) (map snd fl) (query-chunks ! k)
  using query-chunk[OF k-bound] trace-roots-eq composition-roots-eq
  by simp
  then show
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length)
qed
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query final-state-eq by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
     $\bigwedge k. k < \text{length } ?\text{selected-chunks} \implies$ 
      length (?selected-chunks ! k) = length (query-chunks ! k)
    using selected-chunk-lens query-chunk-lens len-selected-chunks by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
    using concat-eq by simp
  have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks  $\wedge$  ( [] :: 'f list) = []
    by (rule concat-append-eq-concat-same-chunk-lengths
      [OF len-eq same-lens concat-eq-empty])
  then show ?thesis by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
      state-after-query-chunks (PState query-state) prefix-chunks i
    by (rule state-after-query-chunks-append-prefix[OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp

```

```

qed
have selected-lookup-global:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
  Some raw
  using selected-lookup prefix-state-eq by simp
have raw-eq: raw = raw-idxs ! i
  using selected-lookup-global replay-lookup[OF round-bound]
    final-state-eq by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-eq query-idxs-eq len-raw round-bound by simp
have auth':
  fri-layer-chunk-authenticated
    (trace-roots ! (length trace-roots - 1))
    (fri-evidence-layer-len trace-roots (length trace-roots - 1))
    (fri-evidence-layer-idx trace-roots query-idxs i
      (length trace-roots - 1))
    actual-chunk final-state
  using auth idx-eq-query trace-roots-eq f-fl-nonempty
  unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
  by simp
have step':
  fri-layer-step-evidence
    (trace-roots ! (length trace-roots - 1))
    (trace-bs ! (length trace-roots - 1))
    (fri-evidence-layer-len trace-roots (length trace-roots - 1))
    (fri-evidence-layer-idx trace-roots query-idxs i
      (length trace-roots - 1))
    (2 ^ (length trace-roots - 1))
    (fri-sibling-index
      (fri-evidence-layer-len trace-roots (length trace-roots - 1))
      (fri-evidence-layer-idx trace-roots query-idxs i
        (length trace-roots - 1)))
    arp arp-path axn axn-path
    (fri-evidence-next-idx trace-roots query-idxs i
      (length trace-roots - 1))
    trace-final actual-chunk
  using step idx-eq-query trace-roots-eq trace-bs-eq f-fl-nonempty
  unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
    fri-evidence-next-idx-def
  by simp
show ?thesis
  by (rule that[OF auth' step'])
qed

```

```

lemma accepted-fri-opening-transcript-composition-final-step-evidence-at:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final

```

```

    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
and out-eq: out = Some (result, final-state)
and round-bound: i < rounds
and nonempty: 0 < length composition-roots
and raw-layer:
   $\wedge$ raw.
    0 < fri-layer-lengths (length composition-roots) (clength * scale) !
    (length composition-roots - 1)  $\wedge$ 
    fri-layer-indices (length composition-roots) (index (to-nat raw))
    (clength * scale) ! (length composition-roots - 1) <
    fri-layer-lengths (length composition-roots) (clength * scale) !
    (length composition-roots - 1)
obtains actual-chunk axp axp-path axn axn-path where
  fri-layer-chunk-authenticated
    (composition-roots ! (length composition-roots - 1))
    (fri-evidence-layer-len composition-roots (length composition-roots - 1))
    (fri-evidence-layer-idx composition-roots query-idxs i
      (length composition-roots - 1))
    actual-chunk final-state
  fri-layer-step-evidence
    (composition-roots ! (length composition-roots - 1))
    (composition-bs ! (length composition-roots - 1))
    (fri-evidence-layer-len composition-roots (length composition-roots - 1))
    (fri-evidence-layer-idx composition-roots query-idxs i
      (length composition-roots - 1))
    (2  $\wedge$  (length composition-roots - 1))
    (fri-sibling-index
      (fri-evidence-layer-len composition-roots (length composition-roots - 1))
      (fri-evidence-layer-idx composition-roots query-idxs i
        (length composition-roots - 1)))
    axp axp-path axn axn-path
    (fri-evidence-next-idx composition-roots query-idxs i
      (length composition-roots - 1))
    composition-final actual-chunk
proof -
from fri-openings obtain result' final-state' fr f-fl as fl query-state
  raw-idxs query-chunks where
  out-def: out = Some (result', final-state')
and trace-roots-eq: trace-roots = map snd f-fl
and composition-bs-eq: composition-bs = map fst fl
and composition-roots-eq: composition-roots = map snd fl
and query-out:
  Some (result', final-state')  $\in$ 
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl trace-final as fl
            composition-final)

```

```

    rounds)
    query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
    query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state'
and query-chunk:
     $\bigwedge i. i < \text{rounds} \implies$ 
      verifier-query-round-chunk (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)
and replay-lookup:
     $\bigwedge i. i < \text{rounds} \implies$ 
      fmllookup (HashMap final-state')
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
unfolding accepted-fri-opening-transcript-def by blast
have result-eq: result' = result
using out-def out-eq by simp
have final-state-eq: final-state' = final-state
using out-def out-eq by simp
have query-out':
    Some (result, final-state)  $\in$ 
      set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl trace-final as fl
            composition-final)
          rounds)
        query-state)
using query-out result-eq final-state-eq by simp
have fl-nonempty: 0 < length fl
using nonempty composition-roots-eq by simp
have raw-layer-fl:
     $\bigwedge$ raw.
      0 < fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1)  $\wedge$ 
      fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! (length fl - 1) <
      fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1)
using raw-layer composition-roots-eq by simp
from ntimes-verifier-query-rounds-selected-composition-fri-last-step-with-prefix-at
  [OF query-out' round-bound fl-nonempty raw-layer-fl]
obtain raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
  suffix-chunks actual-chunk axp axp-path axn axn-path where

```

```

idx-eg: idx = index (to-nat raw)
and len-prefix-idxs: length prefix-query-idxs = i
and len-prefix-chunks: length prefix-chunks = i
and len-suffix-idxs: length suffix-query-idxs = rounds - Suc i
and len-suffix-chunks: length suffix-chunks = rounds - Suc i
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
and prefix-rounds:
   $\bigwedge k. k < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
and suffix-rounds:
   $\bigwedge k. k < \text{rounds} - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
and selected-transcript:
  PTranscript query-state =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript final-state
and auth:
  fri-layer-chunk-authenticated
    (snd (fl ! (length fl - 1)))
    (fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1))
    (fri-layer-indices (length fl) idx (clength * scale) !
      (length fl - 1))
    actual-chunk final-state
and step:
  fri-layer-step-evidence
    (snd (fl ! (length fl - 1)))
    (fst (fl ! (length fl - 1)))
    (fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1))
    (fri-layer-indices (length fl) idx (clength * scale) !
      (length fl - 1))
    (2 ^ (length fl - 1))
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1))
      (fri-layer-indices (length fl) idx (clength * scale) !
        (length fl - 1)))
    axp axp-path axn axn-path
    ((fri-layer-indices (length fl) idx (clength * scale) !
      (length fl - 1)) mod
      ((fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1)) div 2))
    composition-final actual-chunk
and selected-lookup:
  fmlookup (HashMap final-state)

```



```

next
  case False
  let ?k = k - Suc i
  have ge: Suc i ≤ k
    using ⟨¬ k < i⟩ False by linarith
  have k-bound': ?k < rounds - Suc i
    using ge k-bound by linarith
  have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
    using ge len-prefix-chunks by (simp add: nth-append)
  have
    length (suffix-chunks ! ?k) =
      verifier-query-round-transcript-length
        (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound']])
  then show ?thesis
    using nth-eq
    by (simp add: verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-lens:
  ∧k. k < rounds ⇒
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound: k < rounds
  have
    verifier-query-round-chunk (query-idxs ! k)
      (map snd f-fl) (map snd fl) (query-chunks ! k)
    using query-chunk[OF k-bound] trace-roots-eq composition-roots-eq
    by simp
  then show
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length)
qed
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query final-state-eq by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
    ∧k. k < length ?selected-chunks ⇒
      length (?selected-chunks ! k) = length (query-chunks ! k)
    using selected-chunk-lens query-chunk-lens len-selected-chunks by simp
  have concat-eq-empty:

```

```

    List.concat ?selected-chunks @ [] = List.concat query-chunks
  using concat-eq by simp
have len-eq: length ?selected-chunks = length query-chunks
  using len-selected-chunks len-query-chunks by simp
have ?selected-chunks = query-chunks  $\wedge$  ([] :: 'f list) = []
  by (rule concat-append-eq-concat-same-chunk-lengths
      [OF len-eq same-lens concat-eq-empty])
then show ?thesis by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
      state-after-query-chunks (PState query-state) prefix-chunks i
  by (rule state-after-query-chunks-append-prefix[OF len-prefix-chunks])
then show ?thesis
  using chunks-eq by simp
qed
have selected-lookup-global:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some raw
  using selected-lookup prefix-state-eq by simp
have raw-eq: raw = raw-idxs ! i
  using selected-lookup-global replay-lookup[OF round-bound]
    final-state-eq by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-eq query-idxs-eq len-raw round-bound by simp
have auth':
  fri-layer-chunk-authenticated
    (composition-roots ! (length composition-roots - 1))
    (fri-evidence-layer-len composition-roots (length composition-roots - 1))
    (fri-evidence-layer-idx composition-roots query-idxs i
      (length composition-roots - 1))
    actual-chunk final-state
  using auth idx-eq-query composition-roots-eq fl-nonempty
  unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
  by simp
have step':
  fri-layer-step-evidence
    (composition-roots ! (length composition-roots - 1))
    (composition-bs ! (length composition-roots - 1))
    (fri-evidence-layer-len composition-roots (length composition-roots - 1))
    (fri-evidence-layer-idx composition-roots query-idxs i
      (length composition-roots - 1))
    (2  $\wedge$  (length composition-roots - 1))

```

```

    (fri-sibling-index
      (fri-evidence-layer-len composition-roots (length composition-roots - 1))
      (fri-evidence-layer-idx composition-roots query-idxs i
        (length composition-roots - 1)))
    axp axp-path axn axn-path
    (fri-evidence-next-idx composition-roots query-idxs i
      (length composition-roots - 1))
    composition-final actual-chunk
  using step idx-eq-query composition-roots-eq composition-bs-eq fl-nonempty
  unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
    fri-evidence-next-idx-def
  by simp
show ?thesis
  by (rule that[OF auth' step])
qed

end

end

```

```

theory Soundness-FRI-Recorded-Chunk-Replay
imports Soundness-FRI-Raw-Layer-Bounds
begin

```

Selected recorded FRI layer replay.

The authenticated-route layer already proves that the verifier authenticates some chunk at the replayed root/index of a selected FRI layer. The residual proofs need the stronger fact that the specific chunk recorded in the extracted verifier transcript is authenticated. This theory proves that narrow replay fact for one selected layer, avoiding the earlier broad all-layer replay induction.

```

context soundness
begin

```

```

lemma fri-layer-lengths-nth-pos-imp-initial-pos:
  assumes j-bound:  $j < n$ 
    and pos:  $0 < \text{fri-layer-lengths } n \text{ len} \text{ ! } j$ 
  shows  $0 < \text{len}$ 
  using assms
proof (induction j arbitrary: n len)
  case 0
  then show ?case
    by (cases n) simp-all
next
  case (Suc j)
  then obtain n' where n-eq:  $n = \text{Suc } n'$ 
    by (cases n) auto
  have j-bound':  $j < n'$ 

```

```

    using Suc.prem1(1) unfolding n-eq by simp
have pos': 0 < fri-layer-lengths n' (len div 2) ! j
    using Suc.prem1(2) unfolding n-eq by simp
have 0 < len div 2
    by (rule Suc.IH[OF j-bound' pos'])
then show ?case
    by simp
qed

lemma mfold-fri-layer-opening-selected-recorded-chunk-authenticated-at:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
    Some (out, t) ∈
        set-dist
            (execute
                (mfold (idx, x, len, pw)
                    (map fri-layer-opening-step bfs)) s)
and layers:
    fri-layers-transcript (length bfs) len layer-chunks total-chunk
and transcript:
    PTranscript s = total-chunk @ PTranscript t
and initial-idx-bound: idx < len
and all-layer-bounds:
    ∧k. k < length bfs ⇒
        0 < fri-layer-lengths (length bfs) len ! k ∧
        fri-layer-indices (length bfs) idx len ! k <
        fri-layer-lengths (length bfs) len ! k
and j-bound: j < length bfs
shows
    fri-layer-chunk-authenticated (snd (bfs ! j))
        (fri-layer-lengths (length bfs) len ! j)
        (fri-layer-indices (length bfs) idx len ! j)
        (layer-chunks ! j) t
using outcome layers transcript initial-idx-bound all-layer-bounds j-bound
proof (induction bfs arbitrary: idx x len pw s out t j layer-chunks total-chunk)
case Nil
then show ?case by simp
next
case (Cons bf bfs)
obtain b rt where bf-eq: bf = (b, rt)
    by (cases bf)
from Cons.prem1(1)[unfolded bf-eq]
obtain out1 s1 where
    head:
        Some (out1, s1) ∈
            set-dist
                (execute (fri-layer-opening-step (b, rt) (idx, x, len, pw)) s)
and tail:
    Some (out, t) ∈

```

```

    set-dist
    (execute (mfold out1 (map fri-layer-opening-step bfs)) s1)
  by (auto elim!: set-dist-bindE)
obtain first rest where chunks-eq: layer-chunks = first # rest
using Cons.premis(6) Cons.premis(2)
unfolding fri-layers-transcript-def by (cases layer-chunks) auto
have total-eq: total-chunk = first @ List.concat rest
using Cons.premis(2) unfolding fri-layers-transcript-def chunks-eq
by simp
have first-open:
  ∃ xp xp-path xn xn-path.
    fri-layer-opening-chunk len xp xp-path xn xn-path first
using Cons.premis(2) unfolding fri-layers-transcript-def chunks-eq
by auto
have rest-layers:
  fri-layers-transcript (length bfs) (len div 2) rest
  (List.concat rest)
using Cons.premis(2) unfolding fri-layers-transcript-def chunks-eq
by auto
obtain x' xp xp-path xn xn-path head-chunk where
  out1-eq:
    out1 = (idx mod (len div 2), x', len div 2, pw + pw)
and head-chunk-def: head-chunk = [xp] @ xp-path @ [xn] @ xn-path
and head-open:
  fri-layer-opening-chunk len xp xp-path xn xn-path head-chunk
and tr-head: PTranscript s = head-chunk @ PTranscript s1
and xp-auth:
  authenticated-opening-in s1
  (|opening-root = rt,
   opening-length = len,
   opening-index = idx,
   opening-value = xp,
   opening-path = xp-path)
and xn-auth:
  authenticated-opening-in s1
  (|opening-root = rt,
   opening-length = len,
   opening-index = fri-sibling-index len idx,
   opening-value = xn,
   opening-path = xn-path)
proof –
have len-pos: 0 < len
using Cons.premis(5)[of 0] by simp
have idx-bound: idx < len
by (rule Cons.premis(4))
show ?thesis
proof (rule fri-layer-opening-step-outcome-authenticated
  [OF len-pos idx-bound head])
fix xp' xp-path' xn' xn-path' x'' chunk

```

```

assume out-eq:
  out1 = (idx mod (len div 2), x'', len div 2, pw + pw)
and chunk-eq: chunk = [xp'] @ xp-path' @ [xn'] @ xn-path'
and chunk-open:
  fri-layer-opening-chunk len xp' xp-path' xn' xn-path' chunk
and tr: PTranscript s = chunk @ PTranscript s1
and ext: s ≤ s1
and auth-xp:
  authenticated-opening-in s1
  (opening-root = rt, opening-length = len,
   opening-index = idx, opening-value = xp',
   opening-path = xp-path')
and auth-xn:
  authenticated-opening-in s1
  (opening-root = rt, opening-length = len,
   opening-index = fri-sibling-index len idx,
   opening-value = xn', opening-path = xn-path')
show ?thesis
  by (rule that[OF out-eq chunk-eq chunk-open tr auth-xp auth-xn])
qed
qed
from mfold-fri-layer-openings-outcome[OF tail[unfolded out1-eq]]
obtain tail-layers tail-chunk where
  tail-transcript:
    PTranscript s1 = tail-chunk @ PTranscript t
and tail-ext: s1 ≤ t
by blast
have head-len: length head-chunk = length first
proof –
  obtain yp yp-path yn yn-path where first-open':
    fri-layer-opening-chunk len yp yp-path yn yn-path first
    using first-open by blast
show ?thesis
  using fri-layer-opening-chunk-length[OF head-open]
    fri-layer-opening-chunk-length[OF first-open']
  by simp
qed
have chunks-total-eq:
  head-chunk @ tail-chunk = first @ List.concat rest
  using tr-head tail-transcript Cons.prem3 total-eq by simp
have head-eq: head-chunk = first
  using arg-cong[OF chunks-total-eq, of take (length head-chunk)]
    head-len
  by simp
have tail-transcript-rest:
  PTranscript s1 = List.concat rest @ PTranscript t
  using chunks-total-eq head-eq tail-transcript by simp
show ?case
proof (cases j)

```

```

case 0
have auth-s1:
  fri-layer-chunk-authenticated rt len idx head-chunk s1
  by (rule fri-layer-chunk-authenticatedI[OF head-open xp-auth xn-auth])
have auth-t:
  fri-layer-chunk-authenticated rt len idx head-chunk t
  by (rule fri-layer-chunk-authenticated-mono[OF auth-s1 tail-ext])
show ?thesis
  using auth-t head-eq 0 chunks-eq bf-eq by simp
next
case (Suc k)
have k-bound: k < length bfs
  using Cons.premis(6) Suc by simp
have tail-all-layer-bounds:
   $\bigwedge m. m < \text{length } bfs \implies$ 
  0 < fri-layer-lengths (length bfs) (len div 2) ! m  $\wedge$ 
  fri-layer-indices (length bfs) (idx mod (len div 2))
  (len div 2) ! m <
  fri-layer-lengths (length bfs) (len div 2) ! m
  using Cons.premis(5)[of Suc m for m] by simp
have tail-initial-idx-bound: idx mod (len div 2) < len div 2
  using tail-all-layer-bounds[of 0] k-bound by (cases bfs) simp-all
have tail-auth:
  fri-layer-chunk-authenticated (snd (bfs ! k))
  (fri-layer-lengths (length bfs) (len div 2) ! k)
  (fri-layer-indices (length bfs) (idx mod (len div 2))
  (len div 2) ! k)
  (rest ! k) t
  by (rule Cons.IH[OF tail[unfolded out1-eq] rest-layers
  tail-transcript-rest tail-initial-idx-bound tail-all-layer-bounds
  k-bound])
show ?thesis
  using tail-auth Suc chunks-eq by simp
qed
qed

lemma receive-query-commits-selected-recorded-chunk-authenticated-at:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (out, t)  $\in$ 
  set-dist
  (execute
  (mfold (idx, x, len, pw) (receive-query-commits bfs)) s)
and layers:
  fri-layers-transcript (length bfs) len layer-chunks total-chunk
and transcript:
  PTranscript s = total-chunk @ PTranscript t
and initial-idx-bound: idx < len
and all-layer-bounds:

```

$\bigwedge k. k < \text{length } bfs \implies$
 $0 < \text{fri-layer-lengths } (\text{length } bfs) \text{ len} ! k \wedge$
 $\text{fri-layer-indices } (\text{length } bfs) \text{ idx len} ! k <$
 $\text{fri-layer-lengths } (\text{length } bfs) \text{ len} ! k$
and $j\text{-bound}: j < \text{length } bfs$
shows
 $\text{fri-layer-chunk-authenticated } (\text{snd } (bfs ! j))$
 $(\text{fri-layer-lengths } (\text{length } bfs) \text{ len} ! j)$
 $(\text{fri-layer-indices } (\text{length } bfs) \text{ idx len} ! j)$
 $(\text{layer-chunks} ! j) t$
proof –
have $\text{map-eq}: \text{receive-query-commits } bfs = \text{map } \text{fri-layer-opening-step } bfs$
unfolding $\text{receive-query-commits-def } \text{fri-layer-opening-step-def}$
by $(\text{simp add: fun-eq-iff split: prod.splits})$
show $?thesis$
by $(\text{rule } \text{mfold-fri-layer-opening-selected-recorded-chunk-authenticated-at}$
 $[OF \text{ outcome}[\text{unfolded map-eq}] \text{ layers transcript initial-idx-bound}$
 $\text{all-layer-bounds } j\text{-bound}])$
qed

lemma $\text{verifier-query-round-after-index-trace-recorded-layer-chunk-authenticated:}$
fixes $s \ t :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes outcome:
 $\text{Some } ((), t) \in$
 set-dist
 $(\text{execute}$
 $(\text{verifier-query-round-after-index-program}$
 $\text{fr } f\text{-fl } f\text{-final as fl final raw}) \ s)$
and recorded:
 $\text{query-round-fri-layer-transcripts } (\text{index } (\text{to-nat raw}))$
 $(\text{map snd } f\text{-fl}) (\text{map snd fl}) \text{ round-chunk trace-layers}$
 $\text{composition-layers}$
and $\text{transcript: } P\text{Transcript } s = \text{round-chunk } @ \ P\text{Transcript } t$
and $j\text{-bound}: j < \text{length } f\text{-fl}$
and all-layer-bounds:
 $\bigwedge k. k < \text{length } f\text{-fl} \implies$
 $0 < \text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! k \wedge$
 $\text{fri-layer-indices } (\text{length } f\text{-fl}) (\text{index } (\text{to-nat raw}))$
 $(\text{clength} * \text{scale}) ! k <$
 $\text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! k$
shows
 $\text{fri-layer-chunk-authenticated } (\text{snd } (f\text{-fl} ! j))$
 $(\text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! j)$
 $(\text{fri-layer-indices } (\text{length } f\text{-fl}) (\text{index } (\text{to-nat raw})))$
 $(\text{clength} * \text{scale}) ! j)$
 $(\text{trace-layers} ! j) t$
proof –
let $?idx = \text{index } (\text{to-nat raw})$
from outcome **obtain** $fv \ s1 \ f\text{-out} \ s2 \ s3 \ c\text{-out} \ s4$ **where**


```

query-decommit:
  Some (fv, s1) ∈
    set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
and trace-fri:
  Some (f-out, s2) ∈
    set-dist (execute
      (mfold (?idx, hd fv, clength * scale, 1)
        (receive-query-commits f-fl)) s1)
and assert-trace:
  Some (( ), s3) ∈
    set-dist
      (execute
        (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
and comp-fri:
  Some (c-out, s4) ∈
    set-dist (execute
      (mfold
        (?idx, cp-eval as fv (h ^ ?idx * shift), clength * scale, 1)
        (receive-query-commits fl)) s3)
and assert-comp:
  Some (( ), t) ∈
    set-dist
      (execute
        (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
unfolding verifier-query-round-after-index-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript ?idx fv query-paths query-chunk
and tr-s: PTranscript s = query-chunk @ PTranscript s1
by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  trace-layers-actual:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
and ext-s1-s2: s1 ≤ s2
by blast
have s3-eq: s3 = s2
using assert-trace unfolding assert-def
by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
  (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers-actual:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk

```

```

and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
and ext-s3-s4: s3 ≤ s4
by blast
have t-eq: t = s4
using assert-comp unfolding assert-def
by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
let ?actual-chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have actual-recorded:
    query-round-fri-layer-transcripts ?idx (map snd f-fl) (map snd fl)
    ?actual-chunk trace-layer-chunks comp-layer-chunks
unfolding query-round-fri-layer-transcripts-def
by (intro exI[of - query-chunk] exI[of - trace-fri-chunk]
    exI[of - comp-fri-chunk] exI[of - fv] exI[of - query-paths] conjI)
    (use query-chunk-shape trace-layers-actual comp-layers-actual in
    ⟨simp-all add: mult.commute⟩)
have actual-transcript:
    PTranscript s = ?actual-chunk @ PTranscript t
using tr-s tr-s1 tr-s3 s3-eq t-eq by simp
have actual-chunk-eq: ?actual-chunk = round-chunk
using actual-transcript transcript by simp
have trace-layer-chunks-eq:
    trace-layer-chunks = trace-layers
by (rule query-round-fri-layer-transcripts-unique(1)
    [OF actual-recorded[unfolded actual-chunk-eq] recorded])
have idx-bound: ?idx < clength * scale
by (rule index-less-domain)
have auth-s2:
    fri-layer-chunk-authenticated (snd (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) ?idx (clength * scale) ! j)
    (trace-layer-chunks ! j) s2
by (rule receive-query-commits-selected-recorded-chunk-authenticated-at
    [OF trace-fri trace-layers-actual tr-s1 idx-bound
    all-layer-bounds j-bound])
have s2-t: s2 ≤ t
using ext-s3-s4 s3-eq t-eq by simp
have auth-t:
    fri-layer-chunk-authenticated (snd (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) ?idx (clength * scale) ! j)
    (trace-layer-chunks ! j) t
by (rule fri-layer-chunk-authenticated-mono[OF auth-s2 s2-t])
show ?thesis
using auth-t trace-layer-chunks-eq by simp
qed

```

lemma *verifier-query-round-after-index-composition-recorded-layer-chunk-authenticated*:
fixes *s t* :: (*f*, *'a*) *protocol-channel-scheme*

```

assumes outcome:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s)
and recorded:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) round-chunk trace-layers
    composition-layers
and transcript: PTranscript s = round-chunk @ PTranscript t
and j-bound: j < length fl
and all-layer-bounds:
   $\bigwedge k. k < \text{length } fl \implies$ 
     $0 < \text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! k \wedge$ 
     $\text{fri-layer-indices } (\text{length } fl) (\text{index } (\text{to-nat raw}))$ 
     $(\text{clength} * \text{scale}) ! k <$ 
     $\text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! k$ 
shows
  fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw)))
    (clength * scale) ! j)
    (composition-layers ! j) t
proof –
  let ?idx = index (to-nat raw)
from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1) ∈
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
    and trace-fri:
      Some (f-out, s2) ∈
        set-dist (execute
          (mfold (?idx, hd fv, clength * scale, 1)
            (receive-query-commits f-fl)) s1)
    and assert-trace:
      Some ((), s3) ∈
        set-dist
          (execute
            (assert (case f-out of (-, f-x, -, -)  $\Rightarrow$  f-x = f-final)) s2)
    and comp-fri:
      Some (c-out, s4) ∈
        set-dist (execute
          (mfold
            (?idx, cp-eval as fv (h  $\wedge$  ?idx * shift), clength * scale, 1)
            (receive-query-commits fl)) s3)
    and assert-comp:
      Some ((), t) ∈
        set-dist

```

```

    (execute
      (assert (case c-out of (-, x, -, -)  $\Rightarrow$   $x = \text{final}$ ))  $s_4$ )
    unfolding verifier-query-round-after-index-program-def
    by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript ?idx fv query-paths query-chunk
  and tr-s: PTranscript s = query-chunk @ PTranscript s1
  by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  trace-layers-actual:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
  and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
  by blast
have s3-eq:  $s_3 = s_2$ 
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -)  $\Rightarrow$   $f-x = f\text{-final}$ )
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers-actual:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk
  and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
  by blast
have t-eq:  $t = s_4$ 
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -)  $\Rightarrow$   $x = \text{final}$ )
    (auto simp: throw-no-outcome)
let ?actual-chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have actual-recorded:
  query-round-fri-layer-transcripts ?idx (map snd f-fl) (map snd fl)
    ?actual-chunk trace-layer-chunks comp-layer-chunks
  unfolding query-round-fri-layer-transcripts-def
  by (intro exI[of - query-chunk] exI[of - trace-fri-chunk]
    exI[of - comp-fri-chunk] exI[of - fv] exI[of - query-paths] conjI)
    (use query-chunk-shape trace-layers-actual comp-layers-actual in
       $\langle \text{simp-all add: mult commute} \rangle$ )
have actual-transcript:
  PTranscript s = ?actual-chunk @ PTranscript t
  using tr-s tr-s1 tr-s3 s3-eq t-eq by simp
have actual-chunk-eq: ?actual-chunk = round-chunk
  using actual-transcript transcript by simp
have comp-layer-chunks-eq:
  comp-layer-chunks = composition-layers
  by (rule query-round-fri-layer-transcripts-unique(2))

```

```

    [OF actual-recorded[unfolded actual-chunk-eq] recorded])
  have idx-bound: ?idx < clength * scale
  by (rule index-less-domain)
  have auth-s4:
    fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) ?idx (clength * scale) ! j)
    (comp-layer-chunks ! j) s4
  by (rule receive-query-commits-selected-recorded-chunk-authenticated-at
    [OF comp-fri comp-layers-actual tr-s3 idx-bound
      all-layer-bounds j-bound])
  show ?thesis
  using auth-s4 comp-layer-chunks-eq t-eq by simp
qed

lemma accepted-fri-opening-transcript-trace-recorded-layer-chunk-authenticated-at:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  and out-eq: out = Some (result, final-state)
  and round-bound: i < rounds
  and layer-bound: j < length trace-roots
  and all-layer-bounds:
     $\bigwedge \text{raw } k. k < \text{length trace-roots} \implies$ 
     $0 < \text{fri-layer-lengths (length trace-roots) (clength * scale) ! } k \wedge$ 
     $\text{fri-layer-indices (length trace-roots) (index (to-nat raw))}$ 
     $(\text{clength * scale}) ! k <$ 
     $\text{fri-layer-lengths (length trace-roots) (clength * scale) ! } k$ 
  shows
    generic-fri-recorded-layer-chunk-authenticated trace-roots query-idxs
    trace-round-layers final-state i j
proof -
  from fri-openings obtain result' final-state' fr f-fl as fl query-state
    raw-idxs query-chunks where
    out-def: out = Some (result', final-state')
  and trace-roots-eq: trace-roots = map snd f-fl
  and trace-bs-eq: trace-bs = map fst f-fl
  and composition-roots-eq: composition-roots = map snd fl
  and outcome:
    Some (result', final-state')  $\in$ 
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:

```

```

    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state'
and query-chunk:
    ∧i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)
and replay-lookup:
    ∧i. i < rounds ⇒
      fmllookup (HashMap final-state')
        (QueryIndexChallenge (PQueryCounter query-state + i)
          (state-after-query-chunks (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
and recorded:
    ∧i. i < rounds ⇒
      query-round-fri-layer-transcripts (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)
      (trace-round-layers ! i) (composition-round-layers ! i)
unfolding accepted-fri-opening-transcript-def by blast
have result-eq: result' = result
using out-def out-eq by simp
have final-state-eq: final-state' = final-state
using out-def out-eq by simp
have outcome':
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
using outcome result-eq final-state-eq by simp
let ?round =
  verifier-query-round-program fr f-fl trace-final as fl composition-final
from query-opening-ntimes-outcome-decomp-at[OF outcome' round-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist (execute (ntimes ?round i) query-state)
and round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
    Some (suffix, final-state) ∈
      set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
by blast
have x-eq: x = ()
by (cases x) simp

```

```

show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
  fix prefix-query-idxs prefix-chunks
  assume len-prefix-idxs: length prefix-query-idxs = i
  and len-prefix-chunks: length prefix-chunks = i
  and transcript-prefix:
    PTranscript query-state =
      List.concat prefix-chunks @ PTranscript s-i
  and state-i:
    PState s-i =
      state-after-query-chunks (PState query-state) prefix-chunks i
  and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
  and prefix-ext: query-state ≤ s-i
  and prefix-rounds:
     $\bigwedge k. k < i \implies$ 
      verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
  from round[unfolded x-eq verifier-query-round-program-alt-def]
  obtain raw s0 where
    raw-out:
      Some (raw, s0) ∈
        set-dist (execute receive-query-index-challenge s-i)
  and after:
    Some ((), s-suc) ∈
      set-dist (execute
        (verifier-query-round-after-index-program fr f-fl trace-final as
          fl composition-final raw) s0)
  by (auto elim!: set-dist-bindE)
  have tr-s0: PTranscript s0 = PTranscript s-i
  using receive-query-index-challenge-outcome[OF raw-out] by simp
  show ?thesis
  proof (rule verifier-query-round-program-outcome[OF round[unfolded x-eq]])
    fix raw' idx chunk
    assume idx-eq: idx = index (to-nat raw')
    and chunk-shape:
      verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
      chunk
    and transcript-round:
      PTranscript s-i = chunk @ PTranscript s-suc
    and state-suc:
      PState s-suc = foldl concat (PState s-i) chunk
    and round-ext: s-i ≤ s-suc
    and counter-suc:
      PQueryCounter s-suc = Suc (PQueryCounter s-i)
    and lookup-suc:
      fmllookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
        Some raw'

```

```

have raw-eq': raw = raw'
proof -
  have lookup-s0:
    fmlookup (HashMap s0)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw
    using receive-query-index-challenge-outcome[OF raw-out] by simp
  have s0-suc: s0 ≤ s-suc
    using verifier-query-round-after-index-program-outcome[OF after]
    by simp
  have lookup-suc-raw:
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw
    using hash-extension-lookup[OF lookup-s0 s0-suc] .
  show ?thesis
    using lookup-suc lookup-suc-raw by simp
qed
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
  fix suffix-query-idxs suffix-chunks
  assume len-suffix-idxs:
    length suffix-query-idxs = rounds - Suc i
  and len-suffix-chunks:
    length suffix-chunks = rounds - Suc i
  and transcript-suffix:
    PTranscript s-suc =
      List.concat suffix-chunks @ PTranscript final-state
  and suffix-state:
    PState final-state =
      state-after-query-chunks (PState s-suc) suffix-chunks
      (rounds - Suc i)
  and suffix-counter:
    PQueryCounter final-state =
      PQueryCounter s-suc + (rounds - Suc i)
  and suffix-ext: s-suc ≤ final-state
  and suffix-rounds:
     $\bigwedge k. k < \text{rounds} - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
  let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
  have len-selected-chunks: length ?selected-chunks = rounds
    using len-prefix-chunks len-suffix-chunks round-bound by simp
  have selected-chunk-lens:
     $\bigwedge k. k < \text{rounds} \implies$ 
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)

```



```

proof —
  fix  $k$ 
  assume  $k\text{-bound}$ :  $k < \text{rounds}$ 
  show
     $\text{length } (?selected\text{-chunks } ! k) =$ 
       $\text{verifier-query-round-transcript-length } (query\text{-idxs } ! k)$ 
       $(\text{map } snd\ f\text{-fl}) (\text{map } snd\ fl)$ 
  proof ( $\text{cases } k < i$ )
    case  $True$ 
    have  $nth\text{-eq}$ :  $?selected\text{-chunks } ! k = \text{prefix-chunks } ! k$ 
      using  $True\ len\text{-prefix-chunks}$  by ( $\text{simp add: } nth\text{-append}$ )
    have
       $\text{length } (\text{prefix-chunks } ! k) =$ 
         $\text{verifier-query-round-transcript-length}$ 
         $(\text{prefix-query-idxs } ! k) (\text{map } snd\ f\text{-fl}) (\text{map } snd\ fl)$ 
      by ( $\text{rule } \text{verifier-query-round-chunk-length}$ 
         $[OF\ \text{prefix-rounds}[OF\ True]])$ 
    then show  $?thesis$ 
      using  $nth\text{-eq}$ 
      by ( $\text{simp add:}$ 
         $\text{verifier-query-round-transcript-length-index-irrelevant}$ )
  next
    case  $False$ 
    show  $?thesis$ 
    proof ( $\text{cases } k = i$ )
      case  $True$ 
      have  $nth\text{-eq}$ :  $?selected\text{-chunks } ! k = \text{chunk}$ 
        using  $True\ len\text{-prefix-chunks}$  by ( $\text{simp add: } nth\text{-append}$ )
      have
         $\text{length } \text{chunk} =$ 
           $\text{verifier-query-round-transcript-length } idx$ 
           $(\text{map } snd\ f\text{-fl}) (\text{map } snd\ fl)$ 
        by ( $\text{rule } \text{verifier-query-round-chunk-length}[OF\ \text{chunk-shape}]$ )
      then show  $?thesis$ 
        using  $nth\text{-eq}$ 
        by ( $\text{simp add:}$ 
           $\text{verifier-query-round-transcript-length-index-irrelevant}$ )
    next
      case  $False$ 
      let  $?k = k - Suc\ i$ 
      have  $ge$ :  $Suc\ i \leq k$ 
        using  $\langle \neg k < i \rangle\ False$  by  $linarith$ 
      have  $k\text{-bound}'$ :  $?k < \text{rounds} - Suc\ i$ 
        using  $ge\ k\text{-bound}$  by  $linarith$ 
      have  $nth\text{-eq}$ :  $?selected\text{-chunks } ! k = \text{suffix-chunks } ! ?k$ 
        using  $ge\ len\text{-prefix-chunks}$  by ( $\text{simp add: } nth\text{-append}$ )
      have
         $\text{length } (\text{suffix-chunks } ! ?k) =$ 
           $\text{verifier-query-round-transcript-length}$ 

```

```

      (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound']])
  then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-f:
   $\bigwedge k. k < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! k)
    (map snd f-fl) (map snd fl) (query-chunks ! k)
  using query-chunk trace-roots-eq composition-roots-eq by simp
have query-chunk-lens:
   $\bigwedge k. k < \text{rounds} \implies$ 
    length (query-chunks ! k) =
    verifier-query-round-transcript-length (query-idxs ! k)
    (map snd f-fl) (map snd fl)
  using query-chunk-f verifier-query-round-chunk-length by blast
have selected-transcript:
  PTranscript query-state =
    List.concat ?selected-chunks @ PTranscript final-state
  using transcript-prefix transcript-round transcript-suffix by simp
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query final-state-eq by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
     $\bigwedge k. k < \text{length } ?\text{selected-chunks} \implies$ 
      length (?selected-chunks ! k) = length (query-chunks ! k)
    using selected-chunk-lens query-chunk-lens len-selected-chunks
    by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
    using concat-eq by simp
  have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks  $\wedge$  ([] :: 'f list) = []
    by (rule concat-append-eq-concat-same-chunk-lengths
        [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
  state-after-query-chunks (PState query-state) query-chunks i
proof -

```

```

have
  state-after-query-chunks (PState query-state) ?selected-chunks i =
    state-after-query-chunks (PState query-state) prefix-chunks i
by (rule state-after-query-chunks-append-prefix
  [OF len-prefix-chunks])
then show ?thesis
  using chunks-eq by simp
qed
have lookup-final:
  fmllookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some raw'
proof -
  have lookup-suc':
    fmllookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
    Some raw'
  using lookup-suc state-i counter-i by simp
  have lookup-final-prefix:
    fmllookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
    Some raw'
  using hash-extension-lookup[OF lookup-suc' suffix-ext] .
  show ?thesis
    using lookup-final-prefix prefix-state-eq by simp
qed
have raw-idx-eq: raw' = raw-idxs ! i
  using lookup-final replay-lookup[OF round-bound] final-state-eq
  by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-idx-eq query-idxs-eq len-raw round-bound by simp
have selected-i: ?selected-chunks ! i = chunk
  using len-prefix-chunks by (simp add: nth-append)
have chunk-i-eq: chunk = query-chunks ! i
  using chunks-eq selected-i by simp
have recorded-i:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) chunk (trace-round-layers ! i)
    (composition-round-layers ! i)
  using recorded[OF round-bound] idx-eq idx-eq-query raw-eq'
    chunk-i-eq trace-roots-eq composition-roots-eq
  by simp
have tr-s0-suc:
  PTranscript s0 = chunk @ PTranscript s-suc

```

```

    using tr-s0 transcript-round by simp
  have j-bound-f:  $j < \text{length } f\text{-fl}$ 
    using layer-bound trace-roots-eq by simp
  have all-layer-bounds-f:
     $\bigwedge k. k < \text{length } f\text{-fl} \implies$ 
       $0 < \text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! k \wedge$ 
       $\text{fri-layer-indices } (\text{length } f\text{-fl}) (\text{index } (\text{to-nat } \text{raw}))$ 
       $(\text{clength} * \text{scale}) ! k <$ 
       $\text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! k$ 
    using all-layer-bounds trace-roots-eq by simp
  have auth-suc:
    fri-layer-chunk-authenticated (snd (f-fl ! j))
      (fri-layer-lengths (length f-fl) (clength * scale) ! j)
      (fri-layer-indices (length f-fl) (index (to-nat raw)))
      (clength * scale) ! j
      (trace-round-layers ! i ! j) s-suc
  by (rule
    verifier-query-round-after-index-trace-recorded-layer-chunk-authenticated
      [OF after recorded-i tr-s0-suc j-bound-f
        all-layer-bounds-f])
  have auth-final:
    fri-layer-chunk-authenticated (snd (f-fl ! j))
      (fri-layer-lengths (length f-fl) (clength * scale) ! j)
      (fri-layer-indices (length f-fl) (index (to-nat raw)))
      (clength * scale) ! j
      (trace-round-layers ! i ! j) final-state
  by (rule fri-layer-chunk-authenticated-mono[OF auth-suc suffix-ext])
  have raw-idx-query:  $\text{index } (\text{to-nat } \text{raw}) = \text{query-idxs} ! i$ 
    using idx-eq idx-eq-query raw-eq' by simp
  have auth:
    fri-layer-chunk-authenticated (trace-roots ! j)
      (fri-evidence-layer-len trace-roots j)
      (fri-evidence-layer-idx trace-roots query-idxs i j)
      (trace-round-layers ! i ! j) final-state
    using auth-final trace-roots-eq raw-idx-query layer-bound
    unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
    by simp
  show ?thesis
    unfolding generic-fri-recorded-layer-chunk-authenticated-def
    by (rule auth)
qed
qed
qed
qed

lemma accepted-fri-opening-transcript-composition-recorded-layer-chunk-authenticated-at:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs

```

```

    trace-round-layers composition-round-layers
  and out-eq: out = Some (result, final-state)
  and round-bound: i < rounds
  and layer-bound: j < length composition-roots
  and all-layer-bounds:
     $\bigwedge \text{raw } k. k < \text{length composition-roots} \implies$ 
      0 < fri-layer-lengths (length composition-roots) (clength * scale) ! k  $\wedge$ 
      fri-layer-indices (length composition-roots) (index (to-nat raw))
      (clength * scale) ! k <
      fri-layer-lengths (length composition-roots) (clength * scale) ! k
  shows
    generic-fri-recorded-layer-chunk-authenticated composition-roots
    query-idxs composition-round-layers final-state i j
  proof -
  from fri-openings obtain result' final-state' fr f-fl as fl query-state
    raw-idxs query-chunks where
    out-def: out = Some (result', final-state')
  and trace-roots-eq: trace-roots = map snd f-fl
  and composition-roots-eq: composition-roots = map snd fl
  and composition-bs-eq: composition-bs = map fst fl
  and outcome:
    Some (result', final-state')  $\in$ 
      set-dist (execute
        (ntimes
          (verifier-query-round-program fr f-fl trace-final as fl
            composition-final)
          rounds)
        query-state)
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map ( $\lambda \text{raw}. \text{index (to-nat raw)}$ ) raw-idxs
  and len-query-chunks: length query-chunks = rounds
  and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state'
  and query-chunk:
     $\bigwedge i. i < \text{rounds} \implies$ 
      verifier-query-round-chunk (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)
  and replay-lookup:
     $\bigwedge i. i < \text{rounds} \implies$ 
      fmlookup (HashMap final-state')
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
  and recorded:
     $\bigwedge i. i < \text{rounds} \implies$ 
      query-round-fri-layer-transcripts (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)

```

```

      (trace-round-layers ! i) (composition-round-layers ! i)
unfolding accepted-fri-opening-transcript-def by blast
have result-eq: result' = result
using out-def out-eq by simp
have final-state-eq: final-state' = final-state
using out-def out-eq by simp
have outcome':
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
using outcome result-eq final-state-eq by simp
let ?round =
  verifier-query-round-program fr f-fl trace-final as fl composition-final
from query-opening-ntimes-outcome-decomp-at[OF outcome' round-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist (execute (ntimes ?round i) query-state)
and round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
    Some (suffix, final-state) ∈
      set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
by blast
have x-eq: x = ()
by (cases x) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
fix prefix-query-idxs prefix-chunks
assume len-prefix-idxs: length prefix-query-idxs = i
and len-prefix-chunks: length prefix-chunks = i
and transcript-prefix:
  PTranscript query-state =
    List.concat prefix-chunks @ PTranscript s-i
and state-i:
  PState s-i =
    state-after-query-chunks (PState query-state) prefix-chunks i
and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
and prefix-ext: query-state ≤ s-i
and prefix-rounds:
   $\bigwedge k. k < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
from round[unfolded x-eq verifier-query-round-program-alt-def]

```

```

obtain raw s0 where
  raw-out:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s-i)
  and after:
    Some ((), s-suc) ∈
      set-dist (execute
        (verifier-query-round-after-index-program fr f-fl trace-final as
          fl composition-final raw) s0)
  by (auto elim!: set-dist-bindE)
have tr-s0: PTranscript s0 = PTranscript s-i
  using receive-query-index-challenge-outcome[OF raw-out] by simp
show ?thesis
proof (rule verifier-query-round-program-outcome[OF round[unfolded x-eq]])
  fix raw' idx chunk
  assume idx-eq: idx = index (to-nat raw')
  and chunk-shape:
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
  and transcript-round:
    PTranscript s-i = chunk @ PTranscript s-suc
  and state-suc:
    PState s-suc = foldl concat (PState s-i) chunk
  and round-ext: s-i ≤ s-suc
  and counter-suc:
    PQueryCounter s-suc = Suc (PQueryCounter s-i)
  and lookup-suc:
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw'
  have raw-eq': raw = raw'
  proof -
    have lookup-s0:
      fmlookup (HashMap s0)
        (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
        Some raw
    using receive-query-index-challenge-outcome[OF raw-out] by simp
  have s0-suc: s0 ≤ s-suc
  using verifier-query-round-after-index-program-outcome[OF after]
  by simp
  have lookup-suc-raw:
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw
  using hash-extension-lookup[OF lookup-s0 s0-suc] .
  show ?thesis
  using lookup-suc lookup-suc-raw by simp
qed
show ?thesis

```

```

proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs:
  length suffix-query-idxs = rounds - Suc i
and len-suffix-chunks:
  length suffix-chunks = rounds - Suc i
and transcript-suffix:
  PTranscript s-suc =
    List.concat suffix-chunks @ PTranscript final-state
and suffix-state:
  PState final-state =
    state-after-query-chunks (PState s-suc) suffix-chunks
    (rounds - Suc i)
and suffix-counter:
  PQueryCounter final-state =
    PQueryCounter s-suc + (rounds - Suc i)
and suffix-ext: s-suc ≤ final-state
and suffix-rounds:
   $\bigwedge k. k < \text{rounds} - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
    (map snd f-fl) (map snd fl) (suffix-chunks ! k)
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
  using len-prefix-chunks len-suffix-chunks round-bound by simp
have selected-chunk-lens:
   $\bigwedge k. k < \text{rounds} \implies$ 
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound: k < rounds
  show
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
proof (cases k < i)
  case True
  have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length (prefix-chunks ! k) =
      verifier-query-round-transcript-length
        (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
      [OF prefix-rounds[OF True]])
  then show ?thesis
    using nth-eq

```



```

    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
next
case False
show ?thesis
proof (cases k = i)
case True
have nth-eq: ?selected-chunks ! k = chunk
    using True len-prefix-chunks by (simp add: nth-append)
have
    length chunk =
        verifier-query-round-transcript-length idx
        (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF chunk-shape])
then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
next
case False
let ?k = k - Suc i
have ge: Suc i ≤ k
    using <¬ k < i> False by linarith
have k-bound!: ?k < rounds - Suc i
    using ge k-bound by linarith
have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
    using ge len-prefix-chunks by (simp add: nth-append)
have
    length (suffix-chunks ! ?k) =
        verifier-query-round-transcript-length
        (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound!]])
then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-f:
     $\bigwedge k. k < rounds \implies$ 
    verifier-query-round-chunk (query-idxs ! k)
    (map snd f-fl) (map snd fl) (query-chunks ! k)
    using query-chunk trace-roots-eq composition-roots-eq by simp
have query-chunk-lens:
     $\bigwedge k. k < rounds \implies$ 
    length (query-chunks ! k) =
        verifier-query-round-transcript-length (query-idxs ! k)

```

```

      (map snd f-fl) (map snd fl)
    using query-chunk-f verifier-query-round-chunk-length by blast
  have selected-transcript:
    PTranscript query-state =
      List.concat ?selected-chunks @ PTranscript final-state
    using transcript-prefix transcript-round transcript-suffix by simp
  have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
    using selected-transcript transcript-query final-state-eq by simp
  have chunks-eq: ?selected-chunks = query-chunks
  proof -
    have same-lens:
       $\bigwedge k. k < \text{length } ?\text{selected-chunks} \implies$ 
         $\text{length } (?selected-chunks ! k) = \text{length } (query-chunks ! k)$ 
      using selected-chunk-lens query-chunk-lens len-selected-chunks
      by simp
    have concat-eq-empty:
      List.concat ?selected-chunks @ [] = List.concat query-chunks
      using concat-eq by simp
    have len-eq:  $\text{length } ?\text{selected-chunks} = \text{length } query-chunks$ 
      using len-selected-chunks len-query-chunks by simp
    have ?selected-chunks = query-chunks  $\wedge$  ( $[] :: 'f \text{ list}$ ) =  $[]$ 
      by (rule concat-append-eq-concat-same-chunk-lengths
        [OF len-eq same-lens concat-eq-empty])
    then show ?thesis
      by simp
  qed
  have prefix-state-eq:
    state-after-query-chunks (PState query-state) prefix-chunks i =
      state-after-query-chunks (PState query-state) query-chunks i
  proof -
    have
      state-after-query-chunks (PState query-state) ?selected-chunks i =
        state-after-query-chunks (PState query-state) prefix-chunks i
      by (rule state-after-query-chunks-append-prefix
        [OF len-prefix-chunks])
    then show ?thesis
      using chunks-eq by simp
  qed
  have lookup-final:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
      Some raw'
  proof -
    have lookup-suc':
      fmlookup (HashMap s-suc)
        (QueryIndexChallenge (PQueryCounter query-state + i)
          (state-after-query-chunks (PState query-state)
            prefix-chunks i)) =

```

```

    Some raw'
    using lookup-suc state-i counter-i by simp
  have lookup-final-prefix:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
    Some raw'
    using hash-extension-lookup[OF lookup-suc' suffix-ext] .
  show ?thesis
    using lookup-final-prefix prefix-state-eq by simp
qed
have raw-idx-eq: raw' = raw-idxs ! i
  using lookup-final replay-lookup[OF round-bound] final-state-eq
  by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-idx-eq query-idxs-eq len-raw round-bound by simp
have selected-i: ?selected-chunks ! i = chunk
  using len-prefix-chunks by (simp add: nth-append)
have chunk-i-eq: chunk = query-chunks ! i
  using chunks-eq selected-i by simp
have recorded-i:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) chunk (trace-round-layers ! i)
    (composition-round-layers ! i)
  using recorded[OF round-bound] idx-eq idx-eq-query raw-eq'
    chunk-i-eq trace-roots-eq composition-roots-eq
  by simp
have tr-s0-suc:
  PTranscript s0 = chunk @ PTranscript s-suc
  using tr-s0 transcript-round by simp
have j-bound-fl: j < length fl
  using layer-bound composition-roots-eq by simp
have all-layer-bounds-fl:
   $\bigwedge k. k < \text{length } fl \implies$ 
  0 < fri-layer-lengths (length fl) (clength * scale) ! k  $\wedge$ 
  fri-layer-indices (length fl) (index (to-nat raw))
    (clength * scale) ! k <
  fri-layer-lengths (length fl) (clength * scale) ! k
  using all-layer-bounds composition-roots-eq by simp
have auth-suc:
  fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j)
    (composition-round-layers ! i ! j) s-suc
  by (rule
    verifier-query-round-after-index-composition-recorded-layer-chunk-authenticated
      [OF after recorded-i tr-s0-suc j-bound-fl])

```

```

      all-layer-bounds-fl])
have auth-final:
  fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j)
    (composition-round-layers ! i ! j) final-state
  by (rule fri-layer-chunk-authenticated-mono[OF auth-suc suffix-ext])
have raw-idx-query: index (to-nat raw) = query-idxs ! i
  using idx-eq idx-eq-query raw-eq' by simp
have auth:
  fri-layer-chunk-authenticated (composition-roots ! j)
    (fri-evidence-layer-len composition-roots j)
    (fri-evidence-layer-idx composition-roots query-idxs i j)
    (composition-round-layers ! i ! j) final-state
  using auth-final composition-roots-eq raw-idx-query layer-bound
  unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
  by simp
show ?thesis
  unfolding generic-fri-recorded-layer-chunk-authenticated-def
  by (rule auth)
qed
qed
qed
qed
end

end
end

```

theory *Soundness-FRI-Composition-Residuals*

imports

Soundness-FRI-Authenticated-Route
Soundness-FRI-Base-Recorded-Chunk
Soundness-FRI-Final-Step-Replay
Soundness-FRI-Raw-Layer-Bounds
Soundness-FRI-Recorded-Chunk-Replay

begin

Narrow residual refinements for the verifier-tied composition FRI route. This layer keeps the larger authenticated route theory stable while naming the exact remaining base-opening gap: the recorded first-layer composition FRI chunk is not known to be authenticated.

context *soundness*

begin

definition *composition-fri-reachable-verifier-tie-gap*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-fri-reachable-verifier-tie-gap } s \text{ out} \longleftrightarrow$
 $\text{composition-fri-bad-with-reachable-partial-candidate } s \text{ out} \wedge$
 $\neg \text{composition-fri-bad-with-verifier-tied-partial-candidate } s \text{ out}$

definition $\text{composition-fri-verifier-tied-candidate-transfer-gap}$
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where

$\text{composition-fri-verifier-tied-candidate-transfer-gap } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$
 $\text{trace-table composition-table.}$
 $\text{composition-fri-bad-with-reachable-partial-candidate } s \text{ out} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{trace-final as fri-dg composition-roots composition-final}$
 $\text{fri-query-idxs trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\neg (\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table}))$

definition $\text{composition-fri-verifier-tied-candidate-available-gap}$
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where

$\text{composition-fri-verifier-tied-candidate-available-gap } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$
 $\text{trace-table composition-table.}$
 $\text{composition-fri-reachable-verifier-tie-gap } s \text{ out} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{trace-final as fri-dg composition-roots composition-final}$
 $\text{fri-query-idxs trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings})$

lemma *composition-fri-aligned-candidate-bad-contradicts-verifier-tie-gap:*
assumes *gap: composition-fri-reachable-verifier-tie-gap s out*
and *fri-openings:*
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
and *aligned:*
 accepted-with-partial-initial-openings-aligned s out fr trace-roots
 trace-final as fri-dg composition-roots composition-final
 fri-query-idxs trace-openings composition-openings
and *trace-candidate:*
 partial-trace-table-candidate trace-table trace-openings
and *composition-candidate:*
 partial-composition-table-candidate composition-table
 composition-openings
and *trace-low: trace-table-low-degree trace-table*
and *composition-not-low:*
 $\neg$ *composition-table-low-degree maxDegree composition-table*
shows *False*
proof –
have *composition-fri-bad-with-verifier-tied-partial-candidate s out*
unfolding *composition-fri-bad-with-verifier-tied-partial-candidate-def*
by (*intro exI conjI*)
 (*rule fri-openings, rule aligned, rule trace-candidate,*
 rule composition-candidate, rule trace-low, rule composition-not-low)
then show *False*
using *gap unfolding composition-fri-reachable-verifier-tie-gap-def* **by** *simp*
qed

lemma *composition-fri-aligned-candidate-transfer-imp-transfer-gap:*
assumes *gap: composition-fri-reachable-verifier-tie-gap s out*
and *fri-openings:*
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
and *aligned:*
 accepted-with-partial-initial-openings-aligned s out fr trace-roots
 trace-final as fri-dg composition-roots composition-final
 fri-query-idxs trace-openings composition-openings
and *trace-candidate:*
 partial-trace-table-candidate trace-table trace-openings
and *composition-candidate:*
 partial-composition-table-candidate composition-table
 composition-openings
and *transfer:*
 $\neg$ (*trace-table-low-degree trace-table* $\wedge$

```

    ¬ composition-table-low-degree maxDegree composition-table)
  shows composition-fri-verifier-tied-candidate-transfer-gap s out
proof —
  have raw: composition-fri-bad-with-reachable-partial-candidate s out
    using gap unfolding composition-fri-reachable-verifier-tie-gap-def
    by simp
  show ?thesis
    unfolding composition-fri-verifier-tied-candidate-transfer-gap-def
    by (intro exI conjI)
      (rule raw, rule fri-openings, rule aligned, rule trace-candidate,
       rule composition-candidate, rule transfer)
qed

lemma composition-fri-verifier-tied-candidate-available-gap-imp-transfer-gap:
  assumes composition-fri-verifier-tied-candidate-available-gap s out
  shows composition-fri-verifier-tied-candidate-transfer-gap s out
proof —
  obtain trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as trace-openings composition-openings
    trace-table composition-table
  where gap: composition-fri-reachable-verifier-tie-gap s out
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  and aligned:
    accepted-with-partial-initial-openings-aligned s out fr trace-roots
    trace-final as fri-dg composition-roots composition-final
    fri-query-idxs trace-openings composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  using assms
  unfolding composition-fri-verifier-tied-candidate-available-gap-def
  by blast
  show ?thesis
proof (cases
  trace-table-low-degree trace-table ∧
  ¬ composition-table-low-degree maxDegree composition-table)
  case True
  have False
  by (rule composition-fri-aligned-candidate-bad-contradicts-verifier-tie-gap
    [OF gap fri-openings aligned trace-candidate composition-candidate
      True[THEN conjunct1] True[THEN conjunct2]])
  then show ?thesis by simp

```

```

next
  case False
  show ?thesis
    by (rule composition-fri-aligned-candidate-transfer-imp-transfer-gap
      [OF gap fri-openings aligned trace-candidate composition-candidate
        False])
qed
qed

lemma wp-composition-fri-verifier-tied-candidate-available-gap-bound-from-transfer-gap:
  assumes transfer-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-candidate-transfer-gap s) s ≤ T
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-candidate-available-gap s) s ≤ T
  by (rule order-trans[OF - transfer-bound])
    (rule wp-event-mono,
      rule composition-fri-verifier-tied-candidate-available-gap-imp-transfer-gap)

lemma composition-fri-verifier-tied-reduction-staged-bound:
  assumes reductions:
    ∧ data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
        composition-fri-verifier-tied-reduction
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-verifier-tied-partial-candidate)
      adversary-initial-state ≤ composition-fri-error
  by (rule composition-fri-bad-with-verifier-tied-partial-candidates-staged-bound)
    (rule composition-fri-verifier-tied-reductionD[OF reductions])

lemma composition-fri-verifier-tied-reduction-from-reachable-reduction:
  assumes composition-fri-reachable-partial-candidate-reduction-assumption s
  shows composition-fri-verifier-tied-reduction s
proof -
  have reachable:
    wp-event verify-monad
      (composition-fri-bad-with-reachable-partial-candidate s) s ≤
        composition-fri-error
  using assms
  unfolding
    composition-fri-reachable-partial-candidate-reduction-assumption-def
  by simp

```



```

show ?thesis
  unfolding composition-fri-verifier-tied-reduction-def
  by (rule order-trans[OF - reachable])
    (rule wp-event-mono,
     rule composition-fri-bad-with-verifier-tied-imp-reachable-partial-candidate)
qed

lemma composition-fri-reachable-imp-verifier-tied-or-gap:
  assumes composition-fri-bad-with-reachable-partial-candidate s out
  shows
    composition-fri-bad-with-verifier-tied-partial-candidate s out  $\vee$ 
    composition-fri-reachable-verifier-tie-gap s out
  using assms
  unfolding composition-fri-reachable-verifier-tie-gap-def
  by blast

lemma wp-composition-fri-reachable-bound-from-verifier-tied-and-gap:
  fixes V G :: prob
  assumes verifier-bound:
    wp-event verify-monad
    (composition-fri-bad-with-verifier-tied-partial-candidate s) s  $\leq$  V
  and gap-bound:
    wp-event verify-monad
    (composition-fri-reachable-verifier-tie-gap s) s  $\leq$  G
  shows
    wp-event verify-monad
    (composition-fri-bad-with-reachable-partial-candidate s) s  $\leq$  V + G
proof -
  have wp-event verify-monad
    (composition-fri-bad-with-reachable-partial-candidate s) s  $\leq$ 
    wp-event verify-monad
    ( $\lambda$ out.
     composition-fri-bad-with-verifier-tied-partial-candidate s out  $\vee$ 
     composition-fri-reachable-verifier-tie-gap s out) s
  by (rule wp-event-mono)
    (rule composition-fri-reachable-imp-verifier-tied-or-gap)
  also have ...  $\leq$ 
    wp-event verify-monad
    (composition-fri-bad-with-verifier-tied-partial-candidate s) s +
    wp-event verify-monad
    (composition-fri-reachable-verifier-tie-gap s) s
  by (rule wp-event-union-bound)
  also have ...  $\leq$  V + G
  by (rule add-mono[OF verifier-bound gap-bound])
  finally show ?thesis .
qed

lemma composition-fri-reachable-partial-candidate-reduction-from-verifier-tied-and-gap:
  fixes V G :: prob

```

assumes *verifier-bound*:
wp-event verify-monad
(composition-fri-bad-with-verifier-tied-partial-candidate s) s ≤ V
and *gap-bound*:
wp-event verify-monad
(composition-fri-reachable-verifier-tie-gap s) s ≤ G
and *total-bound*: $V + G \leq \text{composition-fri-error}$
shows *composition-fri-reachable-partial-candidate-reduction-assumption s*
proof –
have *old-bound*:
wp-event verify-monad
(composition-fri-bad-with-reachable-partial-candidate s) s ≤ V + G
by (rule *wp-composition-fri-reachable-bound-from-verifier-tied-and-gap*
 $[OF \text{ verifier-bound gap-bound}]$)
show *?thesis*
unfolding
composition-fri-reachable-partial-candidate-reduction-assumption-def
by (rule *order-trans* $[OF \text{ old-bound total-bound}]$)
qed

definition *composition-fri-verifier-tied-recorded-base-chunk-auth-missing*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-recorded-base-chunk-auth-missing s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table round-idx xp xp-path xn xn-path}$
 $\text{ result final-state.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned s out fr trace-roots}$
 $\text{ trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{ trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{round-idx} < \text{length fri-query-idxs} \wedge$
 $0 < \text{length composition-bs} \wedge$
 $\text{fri-layer-step-evidence}$
 $(\text{composition-roots} ! 0)$
 $(\text{composition-bs} ! 0)$

```

(fri-evidence-layer-len composition-roots 0)
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
1
(fri-sibling-index (fri-evidence-layer-len composition-roots 0)
  (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0))
xp xp-path xn xn-path
(fri-evidence-next-idx composition-roots fri-query-idxs round-idx 0)
(fri-evidence-next-value composition-roots composition-bs
  fri-query-idxs round-idx 0 xp xn)
(composition-round-layers ! round-idx ! 0) ∧
¬ fri-opening-matches-table
(fri-evidence-layer-len composition-roots 0)
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
composition-table xp xn ∧
¬ fri-layer-chunk-authenticated (composition-roots ! 0)
(fri-evidence-layer-len composition-roots 0)
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
(composition-round-layers ! round-idx ! 0) final-state)

```

lemma *composition-fri-verifier-tied-recorded-base-chunk-auth-missing-false:*

assumes

composition-fri-verifier-tied-recorded-base-chunk-auth-missing s out

shows *False*

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr as trace-openings

composition-openings trace-table composition-table round-idx xp xp-path

xn xn-path result final-state

where *out-eq*: *out* = *Some (result, final-state)*

and *fri-openings*:

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers

and *round-bound*: *round-idx* < *length fri-query-idxs*

and *composition-bs-nonempty*: *0* < *length composition-bs*

and *missing*:

¬ *fri-layer-chunk-authenticated (composition-roots ! 0)*

(fri-evidence-layer-len composition-roots 0)

(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)

(composition-round-layers ! round-idx ! 0) final-state

unfolding

composition-fri-verifier-tied-recorded-base-chunk-auth-missing-def

by *blast*

have *len-composition-bs*:

length composition-bs = *length composition-roots*

using *accepted-fri-opening-transcript-shapes(4)[OF fri-openings]* .

have *composition-nonempty*: *0* < *length composition-roots*

```

    using composition-bs-nonempty len-composition-bs by simp
  have len-query: length fri-query-idxs = rounds
    using accepted-fri-opening-transcript-shapes(7)[OF fri-openings] .
  have round-bound-rounds: round-idx < rounds
    using round-bound len-query by simp
  have auth:
    fri-layer-chunk-authenticated (composition-roots ! 0) (clength * scale)
      (fri-query-idxs ! round-idx)
      (composition-round-layers ! round-idx ! 0) final-state
    by (rule
      accepted-fri-opening-transcript-composition-base-recorded-chunk-authenticated-at
      [OF fri-openings out-eq round-bound-rounds composition-nonempty])
  have len0:
    fri-evidence-layer-len composition-roots 0 = clength * scale
    using composition-nonempty
    unfolding fri-evidence-layer-len-def
    by (cases composition-roots) simp-all
  have idx0:
    fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0 =
      fri-query-idxs ! round-idx
    using composition-nonempty
    unfolding fri-evidence-layer-idx-def
    by (cases composition-roots) simp-all
  show False
    using missing auth unfolding len0 idx0 by contradiction
qed

```

```

lemma wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero:
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
      s ≤ 0
  proof -
    have wp-event verify-monad
      (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s) s
      ≤ wp-event verify-monad (λ-. False) s
      by (rule wp-event-mono)
      (rule composition-fri-verifier-tied-recorded-base-chunk-auth-missing-false)
    also have ... = 0
      unfolding wp-event-def wp-def dist-expect-def by simp
    finally show ?thesis .
  qed

```

```

lemma composition-fri-verifier-tied-base-chunk-auth-gap-imp-recorded-missing:
  assumes composition-fri-verifier-tied-base-chunk-auth-gap s out
  shows
    composition-fri-verifier-tied-recorded-base-chunk-auth-missing s out
  proof -
    from assms have base:

```

```

composition-fri-verifier-tied-sampled-base-opening-conflict s out
and no-auth:
  ¬ composition-fri-verifier-tied-authenticated-base-opening-conflict
    s out
unfolding composition-fri-verifier-tied-base-chunk-auth-gap-def
by simp-all
from base obtain trace-roots trace-bs trace-final fri-dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr as trace-openings composition-openings
  trace-table composition-table
where fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
and aligned:
  accepted-with-partial-initial-openings-aligned s out fr trace-roots
    trace-final as fri-dg composition-roots composition-final fri-query-idxs
    trace-openings composition-openings
and trace-cand:
  partial-trace-table-candidate trace-table trace-openings
and comp-cand:
  partial-composition-table-candidate composition-table
    composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-not-low:
  ¬ composition-table-low-degree maxDegree composition-table
and generic:
  generic-fri-sampled-base-opening-conflict composition-table
    composition-roots composition-bs fri-query-idxs
    composition-round-layers
unfolding composition-fri-verifier-tied-sampled-base-opening-conflict-def
by blast
from fri-openings obtain result final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-fri-opening-transcript-def by blast
from generic obtain round-idx xp xp-path xn xn-path where
  round-bound: round-idx < length fri-query-idxs
and composition-bs-nonempty: 0 < length composition-bs
and step:
  fri-layer-step-evidence
    (composition-roots ! 0)
    (composition-bs ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
  1
  (fri-sibling-index (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0))
  xp xp-path xn xn-path

```

```

    (fri-evidence-next-idx composition-roots fri-query-idxs round-idx 0)
    (fri-evidence-next-value composition-roots composition-bs
      fri-query-idxs round-idx 0 xp xn)
    (composition-round-layers ! round-idx ! 0)
  and no-match:
    ¬ fri-opening-matches-table
      (fri-evidence-layer-len composition-roots 0)
      (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
      composition-table xp xn
  unfolding generic-fri-sampled-base-opening-conflict-def
  by blast
have missing:
  ¬ fri-layer-chunk-authenticated (composition-roots ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    (composition-round-layers ! round-idx ! 0) final-state
proof
  assume auth:
    fri-layer-chunk-authenticated (composition-roots ! 0)
      (fri-evidence-layer-len composition-roots 0)
      (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
      (composition-round-layers ! round-idx ! 0) final-state
  have
    composition-fri-verifier-tied-authenticated-base-opening-conflict
      s out
  unfolding composition-fri-verifier-tied-authenticated-base-opening-conflict-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
      rule comp-cand, rule trace-low, rule comp-not-low, rule round-bound,
      rule composition-bs-nonempty, rule step, rule no-match, rule auth)
  then show False
    using no-auth by contradiction
qed
show ?thesis
  unfolding composition-fri-verifier-tied-recorded-base-chunk-auth-missing-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
      rule comp-cand, rule trace-low, rule comp-not-low, rule round-bound,
      rule composition-bs-nonempty, rule step, rule no-match, rule missing)
qed

lemma wp-composition-fri-verifier-tied-base-chunk-auth-gap-bound-from-recorded-missing:
  fixes M :: prob
  assumes missing-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
    s ≤ M
  shows
    wp-event verify-monad

```

$(\text{composition-fri-verifier-tied-base-chunk-auth-gap } s) \ s \leq M$
proof –
have $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-base-chunk-auth-gap } s) \ s \leq$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-recorded-base-chunk-auth-missing } s) \ s$
by $(\text{rule wp-event-mono})$
 $(\text{rule composition-fri-verifier-tied-base-chunk-auth-gap-imp-recorded-missing})$
then show $?thesis$
by $(\text{rule order-trans}[OF - \text{missing-bound}])$
qed

lemma $\text{wp-composition-fri-verifier-tied-sampled-base-bound-from-partial-merkle-and-recorded-missing}$:
fixes $M :: \text{prob}$
assumes missing-bound :
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-recorded-base-chunk-auth-missing } s)$
 $s \leq M$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-base-opening-conflict } s) \ s$
 $\leq \text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s + M$
by $(\text{rule wp-composition-fri-verifier-tied-sampled-base-bound-from-partial-merkle-and-auth-gap})$
 $(\text{rule}$
 $\text{wp-composition-fri-verifier-tied-base-chunk-auth-gap-bound-from-recorded-missing}$
 $[\text{OF missing-bound}])$

lemma $\text{wp-composition-fri-verifier-tied-without-same-bound-from-partial-merkle-recorded-missing-and-branches}$:
fixes $M \ N \ S \ F :: \text{prob}$
assumes missing-bound :
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-recorded-base-chunk-auth-missing } s)$
 $s \leq M$
and next-bound :
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-next-value-conflict } s) \ s \leq N$
and successor-bound :
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-successor-opening-conflict } s)$
 $s \leq S$
and final-bound :
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-final-value-conflict } s) \ s \leq F$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer } s)$
 $s \leq (\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s + M) + N + S + F$
by $(\text{rule wp-composition-fri-verifier-tied-without-same-bound-from-branches})$
 $(\text{rule wp-composition-fri-verifier-tied-sampled-base-bound-from-partial-merkle-and-recorded-missing})$

[*OF missing-bound*],
rule next-bound,
rule successor-bound,
rule final-bound)

lemma *wp-composition-fri-verifier-tied-authenticated-conflict-bound-from-recorded-missing-and-branches*:

fixes $P\ M\ N\ S\ F :: \text{prob}$

assumes *partial-merkle-bound*:

wp-event verify-monad (partial-merkle-inconsistency-bad s) s $\leq P$

and *missing-bound*:

wp-event verify-monad
(composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
 $s \leq M$

and *next-bound*:

wp-event verify-monad
(composition-fri-verifier-tied-sampled-next-value-conflict s) s $\leq N$

and *successor-bound*:

wp-event verify-monad
(composition-fri-verifier-tied-sampled-successor-opening-conflict s)
 $s \leq S$

and *final-bound*:

wp-event verify-monad
(composition-fri-verifier-tied-sampled-final-value-conflict s) s $\leq F$

shows

wp-event verify-monad
(composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
 $s \leq (P + M) + N + S + F + P$

proof –

have *without-bound*:

wp-event verify-monad
(composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s)
 $s \leq (P + M) + N + S + F$

proof –

have *wp-event verify-monad*

(composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s)
 $s \leq$
(wp-event verify-monad (partial-merkle-inconsistency-bad s) s + M)
 $+ N + S + F$

by (*rule*

wp-composition-fri-verifier-tied-without-same-bound-from-partial-merkle-recorded-missing-and-branches
[*OF missing-bound next-bound successor-bound final-bound*])

also have $\dots \leq (P + M) + N + S + F$

by (*intro add-mono partial-merkle-bound order-refl*)

finally show *?thesis* .

qed

show *?thesis*

by (*rule*

wp-composition-fri-verifier-tied-authenticated-conflict-bound-from-without-same-and-merkle
[*OF without-bound partial-merkle-bound*])

qed

definition *composition-fri-verifier-tied-next-value-replay-checked-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-next-value-replay-checked-conflict s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table round-idx round-idx' layer-idx v v'.$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{ accepted-with-partial-initial-openings-aligned s out fr trace-roots}$
 $\text{ trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{ trace-openings composition-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{ trace-table-low-degree trace-table} \wedge$
 $\neg \text{ composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{ round-idx} < \text{ length fri-query-idxs} \wedge$
 $\text{ round-idx'} < \text{ length fri-query-idxs} \wedge$
 $\text{ layer-idx} < \text{ length composition-bs} \wedge$
 $\text{ fri-evidence-next-idx composition-roots fri-query-idxs round-idx}$
 $\text{ layer-idx} =$
 $\text{ fri-evidence-next-idx composition-roots fri-query-idxs round-idx'}$
 $\text{ layer-idx} \wedge$
 $\text{ generic-fri-round-forced-next-value composition-roots composition-bs}$
 $\text{ fri-query-idxs composition-round-layers round-idx layer-idx v} \wedge$
 $\text{ generic-fri-round-forced-next-value composition-roots composition-bs}$
 $\text{ fri-query-idxs composition-round-layers round-idx' layer-idx v'} \wedge$
 $v \neq v' \wedge$
 $v = v')$

definition *composition-fri-verifier-tied-next-value-replay-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-next-value-replay-gap s out $\longleftrightarrow$
 $\text{ composition-fri-verifier-tied-sampled-next-value-conflict s out} \wedge$
 $\neg \text{ composition-fri-verifier-tied-next-value-replay-checked-conflict}$
 s out

lemma *composition-fri-verifier-tied-next-value-replay-checked-conflict-false:*

$\neg \text{ composition-fri-verifier-tied-next-value-replay-checked-conflict s out}$

unfolding *composition-fri-verifier-tied-next-value-replay-checked-conflict-def*
by *blast*

lemma *composition-fri-verifier-tied-sampled-next-value-imp-replay-gap:*
assumes *composition-fri-verifier-tied-sampled-next-value-conflict s out*
shows *composition-fri-verifier-tied-next-value-replay-gap s out*
using *assms composition-fri-verifier-tied-next-value-replay-checked-conflict-false*
unfolding *composition-fri-verifier-tied-next-value-replay-gap-def*
by *simp*

lemma *composition-fri-verifier-tied-sampled-next-value-conflictE:*
assumes *composition-fri-verifier-tied-sampled-next-value-conflict s out*
obtains *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table round-idx round-idx' layer-idx v v'

where

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings
partial-trace-table-candidate trace-table trace-openings
partial-composition-table-candidate composition-table
composition-openings
trace-table-low-degree trace-table
 $\neg$ *composition-table-low-degree maxDegree composition-table*
round-idx < length fri-query-idxs
round-idx' < length fri-query-idxs
layer-idx < length composition-bs
fri-evidence-next-idx composition-roots fri-query-idxs round-idx
layer-idx =
fri-evidence-next-idx composition-roots fri-query-idxs round-idx'
layer-idx
generic-fri-round-forced-next-value composition-roots composition-bs
fri-query-idxs composition-round-layers round-idx layer-idx v
generic-fri-round-forced-next-value composition-roots composition-bs
fri-query-idxs composition-round-layers round-idx' layer-idx v'
 $v \neq v'$

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
composition-openings trace-table composition-table

where *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers

composition-round-layers
and *aligned*:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
and *trace-cand*:
partial-trace-table-candidate trace-table trace-openings
and *comp-cand*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-not-low*:
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *conflict*:
generic-fri-sampled-next-value-conflict composition-roots
composition-bs fri-query-idxs composition-round-layers
unfolding *composition-fri-verifier-tied-sampled-next-value-conflict-def*
by *blast*
from *generic-fri-sampled-next-value-conflictE[OF conflict]*
obtain *round-idx round-idx' layer-idx v v'*
where *round-bound*: *round-idx < length fri-query-idxs*
and *round-bound'*: *round-idx' < length fri-query-idxs*
and *layer-bound*: *layer-idx < length composition-bs*
and *same-idx*:
fri-evidence-next-idx composition-roots fri-query-idxs round-idx
layer-idx =
fri-evidence-next-idx composition-roots fri-query-idxs round-idx'
layer-idx
and *forced*:
generic-fri-round-forced-next-value composition-roots composition-bs
fri-query-idxs composition-round-layers round-idx layer-idx v
and *forced'*:
generic-fri-round-forced-next-value composition-roots composition-bs
fri-query-idxs composition-round-layers round-idx' layer-idx v'
and *neq*: *v $\neq$ v'*
by *blast*
show *?thesis*
by (*rule that[OF fri-openings aligned trace-cand comp-cand trace-low*
comp-not-low round-bound round-bound' layer-bound same-idx forced
forced' neq])
qed

lemma *wp-composition-fri-verifier-tied-sampled-next-value-bound-from-replay-gap*:
fixes *G :: prob*
assumes *gap-bound*:
wp-event verify-monad
(composition-fri-verifier-tied-next-value-replay-gap s) s $\leq$ G
shows
wp-event verify-monad

$(\text{composition-fri-verifier-tied-sampled-next-value-conflict } s) \ s \leq G$
proof –
have $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-next-value-conflict } s) \ s \leq$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap } s) \ s$
by $(\text{rule wp-event-mono})$
 $(\text{rule composition-fri-verifier-tied-sampled-next-value-imp-replay-gap})$
then show $?thesis$
by $(\text{rule order-trans}[OF - \text{gap-bound}])$
qed

definition $\text{composition-fri-verifier-tied-successor-replay-checked-conflict}$

$:: (\text{'f}, \text{'a}) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-successor-replay-checked-conflict } s \text{ out}$

$\longleftrightarrow$

$(\exists \text{trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$
 $\text{trace-table composition-table source-round target-round layer-idx}$
 $v \text{ xp xp-path xn xn-path.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{source-round} < \text{length fri-query-idxs} \wedge$
 $\text{target-round} < \text{length fri-query-idxs} \wedge$
 $\text{Suc layer-idx} < \text{length composition-bs} \wedge$
 $\text{generic-fri-round-forced-next-value composition-roots composition-bs}$
 $\text{fri-query-idxs composition-round-layers source-round layer-idx } v \wedge$
 $\text{fri-layer-step-evidence}$
 $(\text{composition-roots ! Suc layer-idx})$
 $(\text{composition-bs ! Suc layer-idx})$
 $(\text{fri-evidence-layer-len composition-roots (Suc layer-idx)})$
 $(\text{fri-evidence-layer-idx composition-roots fri-query-idxs target-round}$
 $(\text{Suc layer-idx}))$
 $(2 \wedge \text{Suc layer-idx})$
 $(\text{fri-sibling-index}$
 $(\text{fri-evidence-layer-len composition-roots (Suc layer-idx)}))$

$$\begin{aligned}
& (fri\text{-}evidence\text{-}layer\text{-}idx\ composition\text{-}roots\ fri\text{-}query\text{-}idxs \\
& \quad target\text{-}round\ (Suc\ layer\text{-}idx))) \\
& xp\ xp\text{-}path\ xn\ xn\text{-}path \\
& (fri\text{-}evidence\text{-}next\text{-}idx\ composition\text{-}roots\ fri\text{-}query\text{-}idxs\ target\text{-}round \\
& \quad (Suc\ layer\text{-}idx)) \\
& (fri\text{-}evidence\text{-}next\text{-}value\ composition\text{-}roots\ composition\text{-}bs \\
& \quad fri\text{-}query\text{-}idxs\ target\text{-}round\ (Suc\ layer\text{-}idx)\ xp\ xn) \\
& (composition\text{-}round\text{-}layers\ !\ target\text{-}round\ !\ Suc\ layer\text{-}idx)\ \wedge \\
& ((fri\text{-}evidence\text{-}next\text{-}idx\ composition\text{-}roots\ fri\text{-}query\text{-}idxs\ source\text{-}round \\
& \quad layer\text{-}idx = \\
& \quad fri\text{-}evidence\text{-}layer\text{-}idx\ composition\text{-}roots\ fri\text{-}query\text{-}idxs\ target\text{-}round \\
& \quad (Suc\ layer\text{-}idx)\ \wedge \\
& \quad v \neq xp \wedge v = xp) \vee \\
& (fri\text{-}evidence\text{-}next\text{-}idx\ composition\text{-}roots\ fri\text{-}query\text{-}idxs\ source\text{-}round \\
& \quad layer\text{-}idx = \\
& \quad fri\text{-}sibling\text{-}index \\
& \quad (fri\text{-}evidence\text{-}layer\text{-}len\ composition\text{-}roots\ (Suc\ layer\text{-}idx)) \\
& \quad (fri\text{-}evidence\text{-}layer\text{-}idx\ composition\text{-}roots\ fri\text{-}query\text{-}idxs \\
& \quad \quad target\text{-}round\ (Suc\ layer\text{-}idx))\ \wedge \\
& \quad v \neq xn \wedge v = xn)))
\end{aligned}$$

definition *composition-fri-verifier-tied-successor-replay-gap*

$$\begin{aligned}
& :: (f, 'a)\ protocol\text{-}channel\text{-}scheme \Rightarrow \\
& \quad (unit\ list \times (f, 'a)\ protocol\text{-}channel\text{-}scheme)\ option \Rightarrow bool
\end{aligned}$$

where

$$\begin{aligned}
& composition\text{-}fri\text{-}verifier\text{-}tied\text{-}successor\text{-}replay\text{-}gap\ s\ out \longleftrightarrow \\
& \quad composition\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}successor\text{-}opening\text{-}conflict\ s\ out \wedge \\
& \quad \neg\ composition\text{-}fri\text{-}verifier\text{-}tied\text{-}successor\text{-}replay\text{-}checked\text{-}conflict \\
& \quad \quad s\ out
\end{aligned}$$

lemma *composition-fri-verifier-tied-successor-replay-checked-conflict-false:*

$$\neg\ composition\text{-}fri\text{-}verifier\text{-}tied\text{-}successor\text{-}replay\text{-}checked\text{-}conflict\ s\ out$$

unfolding

$$\begin{aligned}
& composition\text{-}fri\text{-}verifier\text{-}tied\text{-}successor\text{-}replay\text{-}checked\text{-}conflict\text{-}def \\
& \text{by } blast
\end{aligned}$$

lemma *composition-fri-verifier-tied-sampled-successor-imp-replay-gap:*

assumes

$composition\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}successor\text{-}opening\text{-}conflict\ s\ out$

shows $composition\text{-}fri\text{-}verifier\text{-}tied\text{-}successor\text{-}replay\text{-}gap\ s\ out$

using *assms*

$composition\text{-}fri\text{-}verifier\text{-}tied\text{-}successor\text{-}replay\text{-}checked\text{-}conflict\text{-}false$

unfolding $composition\text{-}fri\text{-}verifier\text{-}tied\text{-}successor\text{-}replay\text{-}gap\text{-}def$

by *simp*

lemma *composition-fri-verifier-tied-sampled-successor-opening-conflictE:*

assumes

$composition\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}successor\text{-}opening\text{-}conflict\ s\ out$

obtains $trace\text{-}roots\ trace\text{-}bs\ trace\text{-}final\ fri\text{-}dg\ composition\text{-}roots$

composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table source-round target-round layer-idx v
xp xp-path xn xn-path

where

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings
partial-trace-table-candidate trace-table trace-openings
partial-composition-table-candidate composition-table
composition-openings
trace-table-low-degree trace-table
 $\neg$ *composition-table-low-degree maxDegree composition-table*
source-round < length fri-query-idxs
target-round < length fri-query-idxs
Suc layer-idx < length composition-bs
generic-fri-round-forced-next-value composition-roots composition-bs
fri-query-idxs composition-round-layers source-round layer-idx v
fri-layer-step-evidence
(composition-roots ! Suc layer-idx)
(composition-bs ! Suc layer-idx)
(fri-evidence-layer-len composition-roots (Suc layer-idx))
(fri-evidence-layer-idx composition-roots fri-query-idxs target-round
(Suc layer-idx))
(2 ^ Suc layer-idx)
(fri-sibling-index
(fri-evidence-layer-len composition-roots (Suc layer-idx))
(fri-evidence-layer-idx composition-roots fri-query-idxs
target-round (Suc layer-idx)))
xp xp-path xn xn-path
(fri-evidence-next-idx composition-roots fri-query-idxs target-round
(Suc layer-idx))
(fri-evidence-next-value composition-roots composition-bs
fri-query-idxs target-round (Suc layer-idx) xp xn)
(composition-round-layers ! target-round ! Suc layer-idx)
(fri-evidence-next-idx composition-roots fri-query-idxs source-round
layer-idx =
fri-evidence-layer-idx composition-roots fri-query-idxs target-round
(Suc layer-idx) $\wedge$
v $\neq$ xp) $\vee$
(fri-evidence-next-idx composition-roots fri-query-idxs source-round
layer-idx =
fri-sibling-index
(fri-evidence-layer-len composition-roots (Suc layer-idx))
(fri-evidence-layer-idx composition-roots fri-query-idxs
target-round (Suc layer-idx)) $\wedge$

$v \neq xn)$
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
composition-openings trace-table composition-table
where *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *aligned*:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
and *trace-cand*:
partial-trace-table-candidate trace-table trace-openings
and *comp-cand*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-not-low*:
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *conflict*:
generic-fri-sampled-successor-opening-conflict composition-roots
composition-bs fri-query-idxs composition-round-layers
unfolding
composition-fri-verifier-tied-sampled-successor-opening-conflict-def
by *blast*
from *generic-fri-sampled-successor-opening-conflictE[OF conflict]*
obtain *source-round target-round layer-idx v xp xp-path xn xn-path*
where *source-bound*: *source-round < length fri-query-idxs*
and *target-bound*: *target-round < length fri-query-idxs*
and *layer-bound*: *Suc layer-idx < length composition-bs*
and *forced*:
generic-fri-round-forced-next-value composition-roots composition-bs
fri-query-idxs composition-round-layers source-round layer-idx v
and *step*:
fri-layer-step-evidence
(composition-roots ! Suc layer-idx)
(composition-bs ! Suc layer-idx)
(fri-evidence-layer-len composition-roots (Suc layer-idx))
(fri-evidence-layer-idx composition-roots fri-query-idxs
target-round (Suc layer-idx))
(2 ^ Suc layer-idx)
(fri-sibling-index
(fri-evidence-layer-len composition-roots (Suc layer-idx))
(fri-evidence-layer-idx composition-roots fri-query-idxs
target-round (Suc layer-idx)))

```

  xp xp-path xn xn-path
  (fri-evidence-next-idx composition-roots fri-query-idxs
   target-round (Suc layer-idx))
  (fri-evidence-next-value composition-roots composition-bs
   fri-query-idxs target-round (Suc layer-idx) xp xn)
  (composition-round-layers ! target-round ! Suc layer-idx)
and cases:
  (fri-evidence-next-idx composition-roots fri-query-idxs source-round
   layer-idx =
   fri-evidence-layer-idx composition-roots fri-query-idxs target-round
   (Suc layer-idx) ∧
   v ≠ xp) ∨
  (fri-evidence-next-idx composition-roots fri-query-idxs source-round
   layer-idx =
   fri-sibling-index
   (fri-evidence-layer-len composition-roots (Suc layer-idx))
   (fri-evidence-layer-idx composition-roots fri-query-idxs
    target-round (Suc layer-idx)) ∧
   v ≠ xn)
by blast
show ?thesis
by (rule that[OF fri-openings aligned trace-cand comp-cand trace-low
  comp-not-low source-bound target-bound layer-bound forced step
  cases])
qed

lemma wp-composition-fri-verifier-tied-sampled-successor-bound-from-replay-gap:
  fixes G :: prob
  assumes gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-successor-replay-gap s) s ≤ G
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-sampled-successor-opening-conflict s)
      s ≤ G
proof -
  have wp-event verify-monad
    (composition-fri-verifier-tied-sampled-successor-opening-conflict s)
    s ≤
    wp-event verify-monad
      (composition-fri-verifier-tied-successor-replay-gap s) s
  by (rule wp-event-mono)
  (rule composition-fri-verifier-tied-sampled-successor-imp-replay-gap)
  then show ?thesis
  by (rule order-trans[OF - gap-bound])
qed

definition composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
  :: ('f, 'a) protocol-channel-scheme ⇒

```


$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
 $s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table result final-state.}$
 $\text{out} = \text{Some}(\text{result}, \text{final-state}) \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{ trace-final as fri-dg composition-roots composition-final}$
 $\text{ fri-query-idxs trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{ composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{generic-fri-authenticated-slot-recorded-chunk-gap composition-roots}$
 $\text{ fri-query-idxs composition-round-layers final-state})$

definition *composition-fri-verifier-tied-final-value-replay-checked-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-final-value-replay-checked-conflict } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table round-idx } v.$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{ trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{ trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{ composition-table-low-degree maxDegree composition-table} \wedge$
 $0 < \text{length composition-bs} \wedge$
 $\text{round-idx} < \text{length fri-query-idxs} \wedge$
 $\text{generic-fri-round-forced-next-value composition-roots composition-bs}$

fri_query_idxs $composition_round_layers$ $round_idx$
 $(length\ composition_bs - 1) \ v \wedge$
 $v \neq composition_final \wedge$
 $v = composition_final$

definition $composition_fri_verifier_tied_final_value_replay_gap$

$:: ('f, 'a) protocol_channel_scheme \Rightarrow$
 $(unit\ list \times ('f, 'a) protocol_channel_scheme) option \Rightarrow bool$

where

$composition_fri_verifier_tied_final_value_replay_gap\ s\ out \longleftrightarrow$
 $composition_fri_verifier_tied_sampled_final_value_conflict\ s\ out \wedge$
 $\neg composition_fri_verifier_tied_final_value_replay_checked_conflict$
 $s\ out$

lemma $composition_fri_verifier_tied_sampled_final_value_conflictE$:

assumes

$composition_fri_verifier_tied_sampled_final_value_conflict\ s\ out$

obtains $trace_roots\ trace_bs\ trace_final\ fri_dg\ composition_roots$

$composition_bs\ composition_final\ fri_query_idxs\ trace_round_layers$

$composition_round_layers\ fr\ as\ trace_openings\ composition_openings$

$trace_table\ composition_table\ round_idx\ v$

where

$accepted_fri_opening_transcript\ s\ out\ trace_roots\ trace_bs$
 $trace_final\ fri_dg\ composition_roots\ composition_bs\ composition_final$
 $fri_query_idxs\ trace_round_layers\ composition_round_layers$
 $accepted_with_partial_initial_openings_aligned\ s\ out\ fr\ trace_roots$
 $trace_final\ as\ fri_dg\ composition_roots\ composition_final\ fri_query_idxs$
 $trace_openings\ composition_openings$
 $partial_trace_table_candidate\ trace_table\ trace_openings$
 $partial_composition_table_candidate\ composition_table$
 $composition_openings$
 $trace_table_low_degree\ trace_table$
 $\neg composition_table_low_degree\ maxDegree\ composition_table$
 $0 < length\ composition_bs$
 $round_idx < length\ fri_query_idxs$
 $generic_fri_round_forced_next_value\ composition_roots\ composition_bs$
 $fri_query_idxs\ composition_round_layers\ round_idx$
 $(length\ composition_bs - 1) \ v$
 $v \neq composition_final$

proof –

from $assms$ **obtain** $trace_roots\ trace_bs\ trace_final\ fri_dg$

$composition_roots\ composition_bs\ composition_final\ fri_query_idxs$

$trace_round_layers\ composition_round_layers\ fr\ as\ trace_openings$

$composition_openings\ trace_table\ composition_table$ **where**

$fri_openings$:

$accepted_fri_opening_transcript\ s\ out\ trace_roots\ trace_bs$

$trace_final\ fri_dg\ composition_roots\ composition_bs$

$composition_final\ fri_query_idxs\ trace_round_layers$

$composition_round_layers$

and *aligned*:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
and *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
and *composition-candidate*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *composition-nonlow*:
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *final-conflict*:
generic-fri-sampled-final-value-conflict composition-roots
composition-bs composition-final fri-query-idxs
composition-round-layers
unfolding *composition-fri-verifier-tied-sampled-final-value-conflict-def*
by *blast*
from *generic-fri-sampled-final-value-conflictE[OF final-conflict]*
obtain *round-idx v* **where**
nonempty: 0 < length composition-bs
and *round-bound: round-idx < length fri-query-idxs*
and *forced*:
generic-fri-round-forced-next-value composition-roots composition-bs
fri-query-idxs composition-round-layers round-idx
(length composition-bs - 1) v
and *neq: v $\neq$ composition-final*
by *blast*
show *?thesis*
by (*rule that[OF fri-openings aligned trace-candidate*
composition-candidate trace-low composition-nonlow nonempty
round-bound forced neq])
qed

lemma *composition-fri-verifier-tied-final-value-replay-checked-conflict-false*:
 $\neg$ *composition-fri-verifier-tied-final-value-replay-checked-conflict s out*
unfolding *composition-fri-verifier-tied-final-value-replay-checked-conflict-def*
by *blast*

lemma *composition-fri-verifier-tied-sampled-final-value-imp-replay-gap*:
assumes *composition-fri-verifier-tied-sampled-final-value-conflict s out*
shows *composition-fri-verifier-tied-final-value-replay-gap s out*
using *assms composition-fri-verifier-tied-final-value-replay-checked-conflict-false*
unfolding *composition-fri-verifier-tied-final-value-replay-gap-def*
by *simp*

lemma *wp-composition-fri-verifier-tied-sampled-final-value-bound-from-replay-gap*:
fixes *G :: prob*
assumes *gap-bound*:

$wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq G$
shows
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-final-value-conflict } s) \ s \leq G$
proof –
have $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-final-value-conflict } s) \ s \leq$
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s$
by (rule $wp\text{-event-mono}$)
(rule $\text{composition-fri-verifier-tied-sampled-final-value-imp-replay-gap}$)
then show ?thesis
by (rule $\text{order-trans}[OF - \text{gap-bound}]$)
qed

definition $\text{composition-fri-verifier-tied-final-value-authenticated-replay-gap}$
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-final-value-authenticated-replay-gap } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$
 $\text{trace-table composition-table result final-state round-idx } v.$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table} \wedge$
 $0 < \text{length composition-bs} \wedge$
 $\text{round-idx} < \text{length fri-query-idxs} \wedge$
 $\text{generic-fri-round-forced-next-value composition-roots composition-bs}$
 $\text{fri-query-idxs composition-round-layers round-idx}$
 $(\text{length composition-bs} - 1) \ v \wedge$
 $v \neq \text{composition-final} \wedge$
 $\neg \text{partial-merkle-inconsistency-bad } s \text{ out} \wedge$
 $\text{generic-fri-recorded-layer-chunk-authenticated composition-roots}$
 $\text{fri-query-idxs composition-round-layers final-state round-idx}$
 $(\text{length composition-bs} - 1))$

definition *composition-fri-verifier-tied-final-value-selected-step-missing-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-final-value-selected-step-missing-gap s out
 $\longleftrightarrow$
composition-fri-verifier-tied-final-value-authenticated-replay-gap s out $\wedge$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers result final-state round-idx } v.$
 $\text{out} = \text{Some} (\text{result}, \text{final-state}) \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $0 < \text{length composition-bs} \wedge$
 $\text{round-idx} < \text{length fri-query-idxs} \wedge$
 $\text{generic-fri-round-forced-next-value composition-roots composition-bs}$
 $\text{ fri-query-idxs composition-round-layers round-idx}$
 $(\text{length composition-bs} - 1) \vee \wedge$
 $v \neq \text{composition-final} \wedge$
 $\neg \text{partial-merkle-inconsistency-bad } s \text{ out} \wedge$
 $\text{generic-fri-recorded-layer-chunk-authenticated composition-roots}$
 $\text{ fri-query-idxs composition-round-layers final-state round-idx}$
 $(\text{length composition-bs} - 1) \wedge$
 $(\forall \text{ actual-chunk axp axp-path axn axn-path.}$
 $\neg (\text{fri-layer-chunk-authenticated}$
 $(\text{composition-roots} ! (\text{length composition-bs} - 1))$
 $(\text{fri-evidence-layer-len composition-roots} (\text{length composition-bs} - 1))$
 $(\text{fri-evidence-layer-idx composition-roots fri-query-idxs round-idx}$
 $(\text{length composition-bs} - 1))$
 $\text{actual-chunk final-state} \wedge$
 $\text{fri-layer-step-evidence}$
 $(\text{composition-roots} ! (\text{length composition-bs} - 1))$
 $(\text{composition-bs} ! (\text{length composition-bs} - 1))$
 $(\text{fri-evidence-layer-len composition-roots} (\text{length composition-bs} - 1))$
 $(\text{fri-evidence-layer-idx composition-roots fri-query-idxs round-idx}$
 $(\text{length composition-bs} - 1))$
 $(2 \wedge (\text{length composition-bs} - 1))$
 $(\text{fri-sibling-index}$
 $(\text{fri-evidence-layer-len composition-roots} (\text{length composition-bs} - 1))$
 $(\text{fri-evidence-layer-idx composition-roots fri-query-idxs round-idx}$
 $(\text{length composition-bs} - 1)))$
 $\text{axp axp-path axn axn-path}$
 $(\text{fri-evidence-next-idx composition-roots fri-query-idxs round-idx}$
 $(\text{length composition-bs} - 1))$
 $\text{composition-final actual-chunk})))$

lemma *composition-fri-verifier-tied-final-value-selected-step-missing-gap-false-on-support:*
assumes *support:* $out \in \text{set-dist } (\text{execute verify-monad } s)$
and *gap:*
 composition-fri-verifier-tied-final-value-selected-step-missing-gap
 s out
shows *False*
proof –
from *gap* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers result final-state round-idx v
where *out-eq:* $out = \text{Some } (result, final-state)$
and *fri-openings:*
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
and *nonempty-bs:* $0 < \text{length } composition-bs$
and *round-bound:* $round-idx < \text{length } fri-query-idxs$
and *missing:*
 $\bigwedge \text{actual-chunk } axp \ axp\text{-path } axn \ axn\text{-path}.$
 $\neg (fri\text{-layer-chunk-authenticated}$
 $(composition-roots \ ! \ (\text{length } composition-bs - 1))$
 $(fri\text{-evidence-layer-len } composition-roots \ (\text{length } composition-bs - 1))$
 $(fri\text{-evidence-layer-idx } composition-roots \ fri\text{-query-idxs } round-idx$
 $(\text{length } composition-bs - 1))$
 $actual\text{-chunk } final\text{-state} \wedge$
 fri-layer-step-evidence
 $(composition-roots \ ! \ (\text{length } composition-bs - 1))$
 $(composition-bs \ ! \ (\text{length } composition-bs - 1))$
 $(fri\text{-evidence-layer-len } composition-roots \ (\text{length } composition-bs - 1))$
 $(fri\text{-evidence-layer-idx } composition-roots \ fri\text{-query-idxs } round-idx$
 $(\text{length } composition-bs - 1))$
 $(2 \wedge (\text{length } composition-bs - 1))$
 fri-sibling-index
 $(fri\text{-evidence-layer-len } composition-roots \ (\text{length } composition-bs - 1))$
 $(fri\text{-evidence-layer-idx } composition-roots \ fri\text{-query-idxs } round-idx$
 $(\text{length } composition-bs - 1)))$
 $axp \ axp\text{-path } axn \ axn\text{-path}$
 $(fri\text{-evidence-next-idx } composition-roots \ fri\text{-query-idxs } round-idx$
 $(\text{length } composition-bs - 1))$
 $composition\text{-final } actual\text{-chunk})$
unfolding
 composition-fri-verifier-tied-final-value-selected-step-missing-gap-def
by *blast*
have *outcome:*
 $\text{Some } (result, final-state) \in \text{set-dist } (\text{execute verify-monad } s)$
using *support out-eq* **by** *simp*
have *challenges:*
 accepted-fri-challenges s (Some (result, final-state))

```

    trace-bs fri-dg composition-bs
using accepted-fri-opening-transcript-challenges[OF fri-openings]
    out-eq by simp
have degree-bound: to-nat fri-dg ≤ maxDegree
    by (rule verify-monad-accepted-fri-challenges-degree-bound
        [OF outcome challenges])
have len-bs-roots: length composition-bs = length composition-roots
    using accepted-fri-opening-transcript-shapes(4)[OF fri-openings] .
have nonempty-roots: 0 < length composition-roots
    using nonempty-bs len-bs-roots by simp
have round-bound-rounds: round-idx < rounds
    using round-bound
        accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
    by simp
have raw-layer:
    ∧ raw.
    0 < fri-layer-lengths (length composition-roots) (clength * scale) !
    (length composition-roots - 1) ∧
    fri-layer-indices (length composition-roots) (index (to-nat raw))
    (clength * scale) ! (length composition-roots - 1) <
    fri-layer-lengths (length composition-roots) (clength * scale) !
    (length composition-roots - 1)
    by (rule accepted-fri-opening-transcript-composition-raw-layer-bound
        [OF fri-openings degree-bound])
        (use nonempty-roots in simp)
from accepted-fri-opening-transcript-composition-final-step-evidence-at
    [OF fri-openings out-eq round-bound-rounds nonempty-roots raw-layer]
obtain actual-chunk axp axp-path axn axn-path where
    auth:
    fri-layer-chunk-authenticated
    (composition-roots ! (length composition-roots - 1))
    (fri-evidence-layer-len composition-roots (length composition-roots - 1))
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
        (length composition-roots - 1))
    actual-chunk final-state
and step:
    fri-layer-step-evidence
    (composition-roots ! (length composition-roots - 1))
    (composition-bs ! (length composition-roots - 1))
    (fri-evidence-layer-len composition-roots (length composition-roots - 1))
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
        (length composition-roots - 1))
    (2 ^ (length composition-roots - 1))
    (fri-sibling-index
        (fri-evidence-layer-len composition-roots (length composition-roots - 1))
        (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
            (length composition-roots - 1)))
    axp axp-path axn axn-path
    (fri-evidence-next-idx composition-roots fri-query-idxs round-idx

```

```

      (length composition-roots - 1))
    composition-final actual-chunk
  by blast
show False
using missing[of actual-chunk axp axp-path axn axn-path]
  auth step len-bs-roots
  by simp
qed

lemma wp-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero:
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-selected-step-missing-gap s)
      s ≤ (0::prob)
proof -
  have
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-selected-step-missing-gap s)
      s ≤ wp-event verify-monad (λ-. False) s
  by (rule wp-event-mono-on-support)
  (rule
    composition-fri-verifier-tied-final-value-selected-step-missing-gap-false-on-support)
  also have ... = 0
  unfolding wp-event-def wp-def dist-expect-def by simp
  finally show ?thesis by simp
qed

lemma composition-fri-verifier-tied-final-value-authenticated-replay-gap-imp-selected-step-missing:
  assumes
    composition-fri-verifier-tied-final-value-authenticated-replay-gap s out
  shows
    composition-fri-verifier-tied-final-value-selected-step-missing-gap s out
proof -
  from assms obtain trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as trace-openings composition-openings
    trace-table composition-table result final-state round-idx v
  where out-eq: out = Some (result, final-state)
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  and nonempty: 0 < length composition-bs
  and round-bound: round-idx < length fri-query-idxs
  and forced:
    generic-fri-round-forced-next-value composition-roots composition-bs
    fri-query-idxs composition-round-layers round-idx
    (length composition-bs - 1) v

```



```

and neg:  $v \neq \text{composition-final}$ 
and no-merkle:  $\neg \text{partial-merkle-inconsistency-bad } s \text{ out}$ 
and recorded-auth:
  generic-fri-recorded-layer-chunk-authenticated composition-roots
  fri-query-idxs composition-round-layers final-state round-idx
  (length composition-bs - 1)
unfolding
  composition-fri-verifier-tied-final-value-authenticated-replay-gap-def
by blast
have no-step:
   $\wedge \text{actual-chunk } \text{axp } \text{axp-path } \text{axn } \text{axn-path}.$ 
   $\neg (\text{fri-layer-chunk-authenticated}$ 
    (composition-roots ! (length composition-bs - 1))
    (fri-evidence-layer-len composition-roots (length composition-bs - 1))
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
      (length composition-bs - 1))
    actual-chunk final-state  $\wedge$ 
    fri-layer-step-evidence
      (composition-roots ! (length composition-bs - 1))
      (composition-bs ! (length composition-bs - 1))
      (fri-evidence-layer-len composition-roots (length composition-bs - 1))
      (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
        (length composition-bs - 1))
      ( $2 \wedge (\text{length } \text{composition-bs} - 1)$ )
      (fri-sibling-index
        (fri-evidence-layer-len composition-roots (length composition-bs - 1))
        (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
          (length composition-bs - 1)))
      axp axp-path axn axn-path
      (fri-evidence-next-idx composition-roots fri-query-idxs round-idx
        (length composition-bs - 1))
      composition-final actual-chunk)
  proof
  fix actual-chunk axp axp-path axn axn-path
  assume step-auth:
    fri-layer-chunk-authenticated
      (composition-roots ! (length composition-bs - 1))
      (fri-evidence-layer-len composition-roots (length composition-bs - 1))
      (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
        (length composition-bs - 1))
      actual-chunk final-state  $\wedge$ 
      fri-layer-step-evidence
        (composition-roots ! (length composition-bs - 1))
        (composition-bs ! (length composition-bs - 1))
        (fri-evidence-layer-len composition-roots (length composition-bs - 1))
        (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
          (length composition-bs - 1))
        ( $2 \wedge (\text{length } \text{composition-bs} - 1)$ )
        (fri-sibling-index
          (fri-evidence-layer-len composition-roots (length composition-bs - 1))
          (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
            (length composition-bs - 1)))

```

```

    (fri-evidence-layer-len composition-roots (length composition-bs - 1))
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
      (length composition-bs - 1)))
  arp arp-path arn arn-path
  (fri-evidence-next-idx composition-roots fri-query-idxs round-idx
    (length composition-bs - 1))
  composition-final actual-chunk
have False
  by (rule generic-fri-final-conflict-false-from-authenticated-step
    [OF forced recorded-auth conjunct1[OF step-auth]
      conjunct2[OF step-auth] no-merkle[unfolded out-eq] neq])
  then show False .
qed
have witness:
  ∃ trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers result final-state round-idx v.
  out = Some (result, final-state) ∧
  accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers ∧
  0 < length composition-bs ∧
  round-idx < length fri-query-idxs ∧
  generic-fri-round-forced-next-value composition-roots composition-bs
    fri-query-idxs composition-round-layers round-idx
    (length composition-bs - 1) v ∧
  v ≠ composition-final ∧
  ¬ partial-merkle-inconsistency-bad s out ∧
  generic-fri-recorded-layer-chunk-authenticated composition-roots
    fri-query-idxs composition-round-layers final-state round-idx
    (length composition-bs - 1) ∧
  (∀ actual-chunk arp arp-path arn arn-path.
    ¬ (fri-layer-chunk-authenticated
      (composition-roots ! (length composition-bs - 1))
      (fri-evidence-layer-len composition-roots (length composition-bs - 1))
      (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
        (length composition-bs - 1))
      actual-chunk final-state ∧
      fri-layer-step-evidence
        (composition-roots ! (length composition-bs - 1))
        (composition-bs ! (length composition-bs - 1))
        (fri-evidence-layer-len composition-roots (length composition-bs - 1))
        (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
          (length composition-bs - 1))
        (2 ^ (length composition-bs - 1))
        (fri-sibling-index
          (fri-evidence-layer-len composition-roots (length composition-bs - 1))
          (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
            (length composition-bs - 1))

```

```

      (length composition-bs - 1)))
    arp arp-path arn arn-path
    (fri-evidence-next-idx composition-roots fri-query-idxs round-idx
      (length composition-bs - 1))
    composition-final actual-chunk))
  using out-eq fri-openings nonempty round-bound forced neq no-merkle
    recorded-auth no-step
  by blast
show ?thesis
unfolding
  composition-fri-verifier-tied-final-value-selected-step-missing-gap-def
using assms witness by blast
qed

lemma wp-composition-fri-verifier-tied-final-value-authenticated-replay-gap-bound-from-selected-step-missing:
  fixes G :: prob
  assumes gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-selected-step-missing-gap s)
    s ≤ G
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-authenticated-replay-gap s)
    s ≤ G
proof -
  have wp-event verify-monad
    (composition-fri-verifier-tied-final-value-authenticated-replay-gap s)
    s ≤
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-selected-step-missing-gap s)
    s
  by (rule wp-event-mono)
  (rule
    composition-fri-verifier-tied-final-value-authenticated-replay-gap-imp-selected-step-missing)
  then show ?thesis
  by (rule order-trans[OF - gap-bound])
qed

lemma composition-fri-verifier-tied-final-value-replay-gap-imp-merkle-or-slot-or-authenticated-on-support:
  assumes support: out ∈ set-dist (execute verify-monad s)
  and gap: composition-fri-verifier-tied-final-value-replay-gap s out
  shows
    partial-merkle-inconsistency-bad s out ∨
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
      s out ∨
    composition-fri-verifier-tied-final-value-authenticated-replay-gap s out
proof -
  from gap have sampled:
    composition-fri-verifier-tied-sampled-final-value-conflict s out

```

```

unfolding composition-fri-verifier-tied-final-value-replay-gap-def
by simp
from composition-fri-verifier-tied-sampled-final-value-conflictE[OF sampled]
obtain trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table round-idx v
where fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers
and aligned:
  accepted-with-partial-initial-openings-aligned s out fr trace-roots
  trace-final as fri-dg composition-roots composition-final
  fri-query-idxs trace-openings composition-openings
and trace-cand: partial-trace-table-candidate trace-table trace-openings
and comp-cand:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-not-low:
   $\neg$  composition-table-low-degree maxDegree composition-table
and nonempty:  $0 < \text{length } \text{composition-bs}$ 
and round-bound:  $\text{round-idx} < \text{length } \text{fri-query-idxs}$ 
and forced:
  generic-fri-round-forced-next-value composition-roots composition-bs
  fri-query-idxs composition-round-layers round-idx
   $(\text{length } \text{composition-bs} - 1) v$ 
and neq:  $v \neq \text{composition-final}$ 
by blast
from fri-openings obtain result final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-fri-opening-transcript-def by blast
show ?thesis
proof (cases partial-merkle-inconsistency-bad s out)
  case True
    then show ?thesis by simp
next
  case no-merkle: False
    let  $?j = \text{length } \text{composition-bs} - 1$ 
    have len-composition-bs:  $\text{length } \text{composition-bs} = \text{length } \text{composition-roots}$ 
      using accepted-fri-opening-transcript-shapes(4)[OF fri-openings] .
    have layer-bound:  $?j < \text{length } \text{composition-roots}$ 
      using nonempty len-composition-bs by simp
    show ?thesis
    proof (cases
      generic-fri-recorded-layer-chunk-authenticated composition-roots
      fri-query-idxs composition-round-layers final-state round-idx ?j)

```

```

case True
have authenticated-gap:
  composition-fri-verifier-tied-final-value-authenticated-replay-gap
  s out
unfolding
  composition-fri-verifier-tied-final-value-authenticated-replay-gap-def
by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
   rule comp-cand, rule trace-low, rule comp-not-low, rule nonempty,
   rule round-bound, rule forced, rule neq,
   use no-merkle True in simp-all)
then show ?thesis by simp
next
case missing-recorded: False
have outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
  using support out-eq by simp
have challenges:
  accepted-fri-challenges s (Some (result, final-state))
  trace-bs fri-dg composition-bs
  using accepted-fri-opening-transcript-challenges[OF fri-openings]
  out-eq by simp
have degree-bound: to-nat fri-dg ≤ maxDegree
  by (rule verify-monad-accepted-fri-challenges-degree-bound
    [OF outcome challenges])
have round-bound-rounds: round-idx < rounds
  using round-bound
  accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
  by simp
have raw-layer:
   $\bigwedge \text{raw.}$ 
   $0 < \text{fri-layer-lengths (length composition-roots)}$ 
   $(\text{clength} * \text{scale}) ! ?j \wedge$ 
   $\text{fri-layer-indices (length composition-roots) (index (to-nat raw))}$ 
   $(\text{clength} * \text{scale}) ! ?j <$ 
   $\text{fri-layer-lengths (length composition-roots)}$ 
   $(\text{clength} * \text{scale}) ! ?j$ 
  by (rule accepted-fri-opening-transcript-composition-raw-layer-bound
    [OF fri-openings degree-bound layer-bound])
from accepted-fri-opening-transcript-composition-selected-chunk-authenticated-at
  [OF fri-openings out-eq round-bound-rounds layer-bound raw-layer]
obtain auth-chunk where auth:
  fri-layer-chunk-authenticated (composition-roots ! ?j)
  (fri-layer-lengths (length composition-roots) (clength * scale) ! ?j)
  (fri-layer-indices (length composition-roots)
    (fri-query-idxs ! round-idx) (clength * scale) ! ?j)
  auth-chunk final-state
  by blast
have step:

```

```

    fri-round-layer-evidence composition-roots composition-bs
    fri-query-idxs round-idx ?j composition-round-layers
  by (rule accepted-fri-opening-transcript-composition-round-layer-evidence
    [OF fri-openings round-bound-rounds layer-bound])
  have generic-step:
    generic-fri-transcript-step-with-authenticated-chunk
    composition-roots composition-bs fri-query-idxs
    composition-round-layers final-state round-idx ?j
  by (rule generic-fri-transcript-step-with-authenticated-chunkI
    [OF step])
    (use auth in
      ⟨simp add: fri-evidence-layer-len-def
        fri-evidence-layer-idx-def⟩)
  have slot-gap:
    generic-fri-authenticated-slot-recorded-chunk-gap
    composition-roots fri-query-idxs composition-round-layers
    final-state
  by (rule
    generic-fri-transcript-step-auth-missing-imp-authenticated-slot-recorded-chunk-gap
    [OF generic-step missing-recorded])
  have route-slot-gap:
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
    s out
  unfolding
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
      rule comp-cand, rule trace-low, rule comp-not-low, rule slot-gap)
  then show ?thesis by simp
qed
qed
qed

lemma wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-merkle-slot-and-authenticated:
  fixes M A R :: prob
  assumes merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M
  and slot-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
      s ≤ A
  and replay-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-authenticated-replay-gap s)
      s ≤ R
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-replay-gap s) s
      ≤ M + A + R

```

proof –
let $?P = \text{partial-merkle-inconsistency-bad } s$
let $?Q = \text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap } s$
let $?R =$
 $\quad \text{composition-fri-verifier-tied-final-value-authenticated-replay-gap } s$
have *event-le*:
 $\quad \text{wp-event verify-monad}$
 $\quad \quad (\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq$
 $\quad \quad \text{wp-event verify-monad } (\lambda \text{out. } ?P \text{ out} \vee ?Q \text{ out} \vee ?R \text{ out}) \ s$
by (*rule wp-event-mono-on-support*)
 $\quad (\text{rule}$
 $\quad \quad \text{composition-fri-verifier-tied-final-value-replay-gap-imp-merkle-or-slot-or-authenticated-on-support})$
have *union-le*:
 $\quad \text{wp-event verify-monad } (\lambda \text{out. } ?P \text{ out} \vee ?Q \text{ out} \vee ?R \text{ out}) \ s \leq$
 $\quad \quad \text{wp-event verify-monad } ?P \ s +$
 $\quad \quad \text{wp-event verify-monad } ?Q \ s +$
 $\quad \quad \text{wp-event verify-monad } ?R \ s$
proof –
have $\text{wp-event verify-monad } (\lambda \text{out. } ?P \text{ out} \vee ?Q \text{ out} \vee ?R \text{ out}) \ s \leq$
 $\quad \text{wp-event verify-monad } ?P \ s +$
 $\quad \text{wp-event verify-monad } (\lambda \text{out. } ?Q \text{ out} \vee ?R \text{ out}) \ s$
by (*rule wp-event-union-bound*)
also have $\dots \leq$
 $\quad \text{wp-event verify-monad } ?P \ s +$
 $\quad (\text{wp-event verify-monad } ?Q \ s + \text{wp-event verify-monad } ?R \ s)$
by (*intro add-mono order.refl wp-event-union-bound*)
finally show *?thesis*
by (*simp add: algebra-simps*)
qed
have *bounds*:
 $\quad \text{wp-event verify-monad } ?P \ s +$
 $\quad \text{wp-event verify-monad } ?Q \ s +$
 $\quad \text{wp-event verify-monad } ?R \ s \leq M + A + R$
using *merkle-bound slot-bound replay-bound*
by (*intro add-mono*)
show *?thesis*
by (*rule order-trans[OF event-le order-trans[OF union-le bounds]]*)
qed

lemma *wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step*:
fixes $M \ A \ R :: \text{prob}$
assumes *merkle-bound*:
 $\quad \text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq M$
and *slot-bound*:
 $\quad \text{wp-event verify-monad}$
 $\quad \quad (\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap } s)$
 $\quad \quad s \leq A$
and *selected-step-bound*:
 $\quad \text{wp-event verify-monad}$

(*composition-fri-verifier-tied-final-value-selected-step-missing-gap* s)
 $s \leq R$
shows
wp-event verify-monad
 (*composition-fri-verifier-tied-final-value-replay-gap* s) s
 $\leq M + A + R$
proof –
have *authenticated-bound*:
wp-event verify-monad
 (*composition-fri-verifier-tied-final-value-authenticated-replay-gap* s)
 $s \leq R$
by (*rule*
wp-composition-fri-verifier-tied-final-value-authenticated-replay-gap-bound-from-selected-step-missing
 [*OF selected-step-bound*])
show *?thesis*
by (*rule*
wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-merkle-slot-and-authenticated)
 (*rule merkle-bound*, *rule slot-bound*, *rule authenticated-bound*)
qed

lemma *wp-composition-fri-verifier-tied-authenticated-conflict-bound-from-recorded-missing-next-successor-and-final-g*
fixes $P\ M\ N\ S\ G :: \text{prob}$
assumes *partial-merkle-bound*:
wp-event verify-monad (*partial-merkle-inconsistency-bad* s) $s \leq P$
and *missing-bound*:
wp-event verify-monad
 (*composition-fri-verifier-tied-recorded-base-chunk-auth-missing* s)
 $s \leq M$
and *next-bound*:
wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-next-value-conflict* s) $s \leq N$
and *successor-bound*:
wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-successor-opening-conflict* s)
 $s \leq S$
and *final-gap-bound*:
wp-event verify-monad
 (*composition-fri-verifier-tied-final-value-replay-gap* s) $s \leq G$
shows
wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer* s)
 $s \leq (P + M) + N + S + G + P$
by (*rule*
wp-composition-fri-verifier-tied-authenticated-conflict-bound-from-recorded-missing-and-branches)
 (*rule partial-merkle-bound*,
rule missing-bound,
rule next-bound,
rule successor-bound,
rule wp-composition-fri-verifier-tied-sampled-final-value-bound-from-replay-gap

[*OF final-gap-bound*])

lemma *wp-composition-fri-verifier-tied-authenticated-conflict-bound-from-all-replay-gaps:*

fixes *P M N S G :: prob*

assumes *partial-merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ P

and *missing-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)

s ≤ M

and *next-gap-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-next-value-replay-gap s) s ≤ N

and *successor-gap-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-successor-replay-gap s) s ≤ S

and *final-gap-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-final-value-replay-gap s) s ≤ G

shows

wp-event verify-monad

(composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)

s ≤ (P + M) + N + S + G + P

by (*rule*

wp-composition-fri-verifier-tied-authenticated-conflict-bound-from-recorded-missing-next-successor-and-final-gap)

(*rule partial-merkle-bound,*

rule missing-bound,

rule wp-composition-fri-verifier-tied-sampled-next-value-bound-from-replay-gap

[*OF next-gap-bound*],

rule wp-composition-fri-verifier-tied-sampled-successor-bound-from-replay-gap

[*OF successor-gap-bound*],

rule final-gap-bound)

definition *composition-fri-verifier-tied-assignment-auth-gap*

:: ('f, 'a) protocol-channel-scheme ⇒

(unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool

where

composition-fri-verifier-tied-assignment-auth-gap s out ⇔

composition-fri-verifier-tied-sampled-assignment-conflict s out ∧

¬ composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s out

definition *composition-fri-verifier-tied-same-layer-auth-gap*

:: ('f, 'a) protocol-channel-scheme ⇒

(unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool

where

composition-fri-verifier-tied-same-layer-auth-gap s out ⇔

(∃ trace-roots trace-bs trace-final fri-dg composition-roots

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as trace-openings composition-openings
trace-table composition-table result final-state.
out = Some (result, final-state) ∧
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers ∧
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings ∧
partial-trace-table-candidate trace-table trace-openings ∧
partial-composition-table-candidate composition-table
composition-openings ∧
trace-table-low-degree trace-table ∧
 $\neg$ *composition-table-low-degree maxDegree composition-table ∧*
generic-fri-sampled-same-layer-opening-conflict composition-roots
composition-bs fri-query-idxs composition-round-layers ∧
 $\neg$ *generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks*
composition-roots composition-bs fri-query-idxs composition-round-layers
final-state)

definition *composition-fri-verifier-tied-same-layer-recorded-auth-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-same-layer-recorded-auth-gap s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table result final-state.
out = Some (result, final-state) ∧
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers ∧
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings ∧
partial-trace-table-candidate trace-table trace-openings ∧
partial-composition-table-candidate composition-table
composition-openings ∧
trace-table-low-degree trace-table ∧
 $\neg$ *composition-table-low-degree maxDegree composition-table ∧*
generic-fri-sampled-same-layer-recorded-auth-gap composition-roots
composition-bs fri-query-idxs composition-round-layers final-state)

definition *composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap
s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table result final-state.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned s out fr trace-roots}$
 $\text{ trace-final as fri-dg composition-roots composition-final}$
 $\text{ fri-query-idxs trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{generic-fri-sampled-same-layer-authenticated-recorded-auth-gap}$
 $\text{ composition-roots composition-bs fri-query-idxs composition-round-layers}$
 $\text{ final-state})$

lemma *composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-false-on-support:*

assumes *support: out* $\in$ *set-dist (execute verify-monad s)*

and event:

composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
s out

shows *False*

proof —

from *event* **obtain** *trace-roots trace-bs trace-final fri-dg*
 $\text{ composition-roots composition-bs composition-final fri-query-idxs}$
 $\text{ trace-round-layers composition-round-layers fr as trace-openings}$
 $\text{ composition-openings trace-table composition-table result final-state}$

where *out-eq: out* $=$ *Some (result, final-state)*

and *fri-openings:*

$\text{accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers}$

and *gap:*

$\text{generic-fri-authenticated-slot-recorded-chunk-gap composition-roots}$
 $\text{ fri-query-idxs composition-round-layers final-state}$

unfolding

composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-def

by *blast*

from *gap* **obtain** *round-idx layer-idx auth-chunk* **where**

round-bound: round-idx $<$ *length fri-query-idxs*

```

and layer-bound: layer-idx < length composition-roots
and missing:
  ¬ generic-fri-recorded-layer-chunk-authenticated composition-roots
    fri-query-idxs composition-round-layers final-state round-idx
    layer-idx
unfolding generic-fri-authenticated-slot-recorded-chunk-gap-def
by blast
have support-some:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
using support out-eq by simp
have challenges:
  accepted-fri-challenges s (Some (result, final-state))
    trace-bs fri-dg composition-bs
using accepted-fri-opening-transcript-challenges[OF fri-openings]
  out-eq by simp
have degree-bound: to-nat fri-dg ≤ maxDegree
by (rule verify-monad-accepted-fri-challenges-degree-bound
  [OF support-some challenges])
have round-bound-rounds: round-idx < rounds
using round-bound accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
by simp
have all-layer-bounds:
  ∧ raw k. k < length composition-roots ⇒
    0 < fri-layer-lengths (length composition-roots) (clength * scale) ! k ∧
    fri-layer-indices (length composition-roots) (index (to-nat raw))
    (clength * scale) ! k <
    fri-layer-lengths (length composition-roots) (clength * scale) ! k
by (rule accepted-fri-opening-transcript-composition-raw-layer-bound
  [OF fri-openings degree-bound])
have recorded:
  generic-fri-recorded-layer-chunk-authenticated composition-roots
    fri-query-idxs composition-round-layers final-state round-idx layer-idx
by (rule
    accepted-fri-opening-transcript-composition-recorded-layer-chunk-authenticated-at
    [OF fri-openings out-eq round-bound-rounds layer-bound
    all-layer-bounds])
show False
using missing recorded by simp
qed

lemma wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero:
shows
  wp-event verify-monad
    (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
    s ≤ (0::prob)
proof -
have
  wp-event verify-monad
    (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)

```

```

    s ≤ wp-event verify-monad (λ-. False) s
  by (rule wp-event-mono-on-support)
    (rule
      composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-false-on-support)
  also have ... = 0
    unfolding wp-event-def wp-def dist-expect-def by simp
  finally show ?thesis by simp
qed

lemma composition-fri-verifier-tied-recorded-base-chunk-auth-missing-imp-slot-recorded-chunk-gap:
  assumes
    composition-fri-verifier-tied-recorded-base-chunk-auth-missing s out
  shows
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s out
proof -
  from assms obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as trace-openings
    composition-openings trace-table composition-table round-idx xp xp-path
    xn xn-path result final-state
  where out-eq: out = Some (result, final-state)
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  and aligned:
    accepted-with-partial-initial-openings-aligned s out fr trace-roots
    trace-final as fri-dg composition-roots composition-final
    fri-query-idxs trace-openings composition-openings
  and trace-cand:
    partial-trace-table-candidate trace-table trace-openings
  and comp-cand:
    partial-composition-table-candidate composition-table
    composition-openings
  and trace-low: trace-table-low-degree trace-table
  and comp-not-low:
    ¬ composition-table-low-degree maxDegree composition-table
  and round-bound: round-idx < length fri-query-idxs
  and composition-bs-nonempty: 0 < length composition-bs
  and missing:
    ¬ fri-layer-chunk-authenticated (composition-roots ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    (composition-round-layers ! round-idx ! 0) final-state
  unfolding
    composition-fri-verifier-tied-recorded-base-chunk-auth-missing-def
  by blast
have len-composition-bs:

```

```

length composition-bs = length composition-roots
using accepted-fri-opening-transcript-shapes(4)[OF fri-openings] .
have layer-bound: 0 < length composition-roots
using composition-bs-nonempty len-composition-bs by simp
have len-query: length fri-query-idxs = rounds
using accepted-fri-opening-transcript-shapes(7)[OF fri-openings] .
have round-bound-rounds: round-idx < rounds
using round-bound len-query by simp
have raw-layer:
   $\wedge$ raw.
  0 < fri-layer-lengths (length composition-roots) (clength * scale) ! 0  $\wedge$ 
  fri-layer-indices (length composition-roots) (index (to-nat raw))
  (clength * scale) ! 0 <
  fri-layer-lengths (length composition-roots) (clength * scale) ! 0
by (rule fri-first-layer-raw-bound[OF layer-bound])
have auth-step:
  generic-fri-transcript-step-with-authenticated-chunk composition-roots
  composition-bs fri-query-idxs composition-round-layers final-state
  round-idx 0
by (rule
  accepted-fri-opening-transcript-composition-step-with-authenticated-chunk-at
  [OF fri-openings out-eq round-bound-rounds layer-bound raw-layer])
have recorded-missing:
   $\neg$  generic-fri-recorded-layer-chunk-authenticated composition-roots
  fri-query-idxs composition-round-layers final-state round-idx 0
using missing
unfolding generic-fri-recorded-layer-chunk-authenticated-def
by simp
have slot-gap:
  generic-fri-authenticated-slot-recorded-chunk-gap composition-roots
  fri-query-idxs composition-round-layers final-state
by (rule
  generic-fri-transcript-step-auth-missing-imp-authenticated-slot-recorded-chunk-gap
  [OF auth-step recorded-missing])
show ?thesis
unfolding
  composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-def
by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
  rule comp-cand, rule trace-low, rule comp-not-low, rule slot-gap)
qed

lemma wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-bound-from-slot-recorded-chunk-gap:
fixes A :: prob
assumes slot-gap-bound:
  wp-event verify-monad
  (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
  s  $\leq$  A
shows

```

```

wp-event verify-monad
  (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
  s ≤ A
proof –
  have wp-event verify-monad
    (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
    s ≤
    wp-event verify-monad
      (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
      s
  by (rule wp-event-mono)
    (rule
      composition-fri-verifier-tied-recorded-base-chunk-auth-missing-imp-slot-recorded-chunk-gap)
  then show ?thesis
    by (rule order-trans[OF - slot-gap-bound])
qed

```

```

lemma composition-fri-verifier-tied-assignment-imp-authenticated-or-auth-gap:
assumes composition-fri-verifier-tied-sampled-assignment-conflict s out
shows
  composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
    s out ∨
    composition-fri-verifier-tied-assignment-auth-gap s out
using assms
unfolding composition-fri-verifier-tied-assignment-auth-gap-def
by blast

```

```

lemma composition-fri-verifier-tied-assignment-auth-gap-imp-same-layer-auth-gap:
assumes composition-fri-verifier-tied-assignment-auth-gap s out
shows composition-fri-verifier-tied-same-layer-auth-gap s out
proof –
from assms have conflict:
  composition-fri-verifier-tied-sampled-assignment-conflict s out
and no-auth:
  ¬ composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
    s out
unfolding composition-fri-verifier-tied-assignment-auth-gap-def
by simp-all
from conflict obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final fri-query-idxs
  trace-round-layers composition-round-layers fr as trace-openings
  composition-openings trace-table composition-table
where fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers
and aligned:
  accepted-with-partial-initial-openings-aligned s out fr trace-roots

```

```

    trace-final as fri-dg composition-roots composition-final
    fri-query-idxs trace-openings composition-openings
and trace-cand:
    partial-trace-table-candidate trace-table trace-openings
and comp-cand:
    partial-composition-table-candidate composition-table
    composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-not-low:
    ¬ composition-table-low-degree maxDegree composition-table
and sampled:
    generic-fri-sampled-assignment-conflict composition-table
    composition-roots composition-bs composition-final fri-query-idxs
    composition-round-layers
unfolding
    composition-fri-verifier-tied-sampled-assignment-conflict-def
by blast
from fri-openings obtain result final-state where out-eq:
    out = Some (result, final-state)
    unfolding accepted-fri-opening-transcript-def by blast
have split:
    generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
    composition-table composition-roots composition-bs composition-final
    fri-query-idxs composition-round-layers final-state ∨
    (generic-fri-sampled-same-layer-opening-conflict composition-roots
    composition-bs fri-query-idxs composition-round-layers ∧
    ¬ generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    composition-roots composition-bs fri-query-idxs composition-round-layers
    final-state)
by (rule
    generic-fri-sampled-assignment-conflict-imp-authenticated-or-same-layer-auth-gap
    [OF sampled])
then show ?thesis
proof
    assume auth:
    generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
    composition-table composition-roots composition-bs composition-final
    fri-query-idxs composition-round-layers final-state
    have
    composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
    s out
    unfolding
    composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
    by (intro exI conjI)
    (rule fri-openings, rule out-eq, rule aligned, rule trace-cand,
    rule comp-cand, rule trace-low, rule comp-not-low, rule auth)
    then show ?thesis
    using no-auth by contradiction
next

```



```

assume gap:
  generic-fri-sampled-same-layer-opening-conflict composition-roots
  composition-bs fri-query-idxs composition-round-layers  $\wedge$ 
   $\neg$  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
  composition-roots composition-bs fri-query-idxs composition-round-layers
  final-state
show ?thesis
unfolding composition-fri-verifier-tied-same-layer-auth-gap-def
by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
   rule comp-cand, rule trace-low, rule comp-not-low, use gap in simp-all)
qed
qed

lemma composition-fri-verifier-tied-same-layer-auth-gap-imp-recorded-auth-gap:
  assumes composition-fri-verifier-tied-same-layer-auth-gap s out
  shows composition-fri-verifier-tied-same-layer-recorded-auth-gap s out
proof -
from assms obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final fri-query-idxs
  trace-round-layers composition-round-layers fr as trace-openings
  composition-openings trace-table composition-table result final-state
where out-eq: out = Some (result, final-state)
and fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers
and aligned:
  accepted-with-partial-initial-openings-aligned s out fr trace-roots
  trace-final as fri-dg composition-roots composition-final
  fri-query-idxs trace-openings composition-openings
and trace-cand:
  partial-trace-table-candidate trace-table trace-openings
and comp-cand:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-not-low:
   $\neg$  composition-table-low-degree maxDegree composition-table
and same:
  generic-fri-sampled-same-layer-opening-conflict composition-roots
  composition-bs fri-query-idxs composition-round-layers
and no-auth:
   $\neg$  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
  composition-roots composition-bs fri-query-idxs composition-round-layers
  final-state
unfolding composition-fri-verifier-tied-same-layer-auth-gap-def
by blast

```

```

have recorded-gap:
  generic-fri-sampled-same-layer-recorded-auth-gap composition-roots
    composition-bs fri-query-idxs composition-round-layers final-state
by (rule generic-fri-sampled-same-layer-auth-gap-imp-recorded-auth-gap
    [OF same no-auth])
show ?thesis
  unfolding composition-fri-verifier-tied-same-layer-recorded-auth-gap-def
by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
    rule comp-cand, rule trace-low, rule comp-not-low, rule recorded-gap)
qed

lemma composition-fri-verifier-tied-same-layer-recorded-auth-gap-imp-authenticated-recorded-auth-gap-on-support:
assumes support: out  $\in$  set-dist (execute verify-monad s)
and gap: composition-fri-verifier-tied-same-layer-recorded-auth-gap s out
shows
  composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap
    s out
proof –
from gap obtain trace-roots trace-bs trace-final fri-dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr as trace-openings composition-openings
  trace-table composition-table result final-state
where out-eq: out = Some (result, final-state)
and fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers
and aligned:
  accepted-with-partial-initial-openings-aligned s out fr trace-roots
  trace-final as fri-dg composition-roots composition-final
  fri-query-idxs trace-openings composition-openings
and trace-cand:
  partial-trace-table-candidate trace-table trace-openings
and comp-cand:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-not-low:
   $\neg$  composition-table-low-degree maxDegree composition-table
and recorded-gap:
  generic-fri-sampled-same-layer-recorded-auth-gap composition-roots
    composition-bs fri-query-idxs composition-round-layers final-state
unfolding composition-fri-verifier-tied-same-layer-recorded-auth-gap-def
by blast
have outcome:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
using support out-eq by simp

```

```

have challenges:
  accepted-fri-challenges s (Some (result, final-state))
  trace-bs fri-dg composition-bs
using accepted-fri-opening-transcript-challenges[OF fri-openings]
  out-eq by simp
have degree-bound: to-nat fri-dg ≤ maxDegree
by (rule verify-monad-accepted-fri-challenges-degree-bound
  [OF outcome challenges])
have len-composition-bs:
  length composition-bs = length composition-roots
using accepted-fri-opening-transcript-shapes(4)[OF fri-openings] .
have len-query: length fri-query-idxs = rounds
using accepted-fri-opening-transcript-shapes(7)[OF fri-openings] .
have auth-step:
  ∧round-idx layer-idx.
  round-idx < length fri-query-idxs ⇒
  layer-idx < length composition-bs ⇒
  generic-fri-transcript-step-with-authenticated-chunk composition-roots
  composition-bs fri-query-idxs composition-round-layers final-state
  round-idx layer-idx
proof –
  fix round-idx layer-idx
  assume round-bound: round-idx < length fri-query-idxs
  and layer-bound-bs: layer-idx < length composition-bs
  have round-bound-rounds: round-idx < rounds
  using round-bound len-query by simp
  have layer-bound-roots: layer-idx < length composition-roots
  using layer-bound-bs len-composition-bs by simp
  show
  generic-fri-transcript-step-with-authenticated-chunk
  composition-roots composition-bs fri-query-idxs
  composition-round-layers final-state round-idx layer-idx
  by (rule
    accepted-fri-opening-transcript-composition-step-with-authenticated-chunk-at-raw
    [OF fri-openings out-eq degree-bound round-bound-rounds
    layer-bound-roots])
qed
have authenticated-recorded-gap:
  generic-fri-sampled-same-layer-authenticated-recorded-auth-gap
  composition-roots composition-bs fri-query-idxs composition-round-layers
  final-state
by (rule
  generic-fri-recorded-auth-gap-imp-authenticated-recorded-auth-gap
  [OF recorded-gap auth-step])
show ?thesis
unfolding
  composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap-def
by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,

```

*rule comp-cand, rule trace-low, rule comp-not-low,
rule authenticated-recorded-gap)*

qed

lemma *composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap:*

assumes

composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap

s out

shows

composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap

s out

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as trace-openings composition-openings

trace-table composition-table result final-state

where *out-eq: out = Some (result, final-state)*

and *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers

and *aligned:*

accepted-with-partial-initial-openings-aligned s out fr trace-roots

trace-final as fri-dg composition-roots composition-final

fri-query-idxs trace-openings composition-openings

and *trace-cand:*

partial-trace-table-candidate trace-table trace-openings

and *comp-cand:*

partial-composition-table-candidate composition-table

composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *comp-not-low:*

¬ composition-table-low-degree maxDegree composition-table

and *gap:*

generic-fri-sampled-same-layer-authenticated-recorded-auth-gap

composition-roots composition-bs fri-query-idxs composition-round-layers

final-state

unfolding

composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap-def

by *blast*

have *slot-gap:*

generic-fri-authenticated-slot-recorded-chunk-gap composition-roots

fri-query-idxs composition-round-layers final-state

by *(rule*

generic-fri-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap

[OF gap])

show *?thesis*

unfolding

composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-def
by (*intro exI conjI*)
 (*rule out-eq, rule fri-openings, rule aligned, rule trace-cand,*
rule comp-cand, rule trace-low, rule comp-not-low, rule slot-gap)
qed

lemma *wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-authenticated-and-auth-gap:*
fixes $C\ G :: \text{prob}$
assumes *authenticated-bound:*
wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s*)
 $s \leq C$
and *auth-gap-bound:*
wp-event verify-monad
 (*composition-fri-verifier-tied-assignment-auth-gap s*) $s \leq G$
shows
wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-assignment-conflict s*) s
 $\leq C + G$
proof –
have *wp-event verify-monad*
 (*composition-fri-verifier-tied-sampled-assignment-conflict s*) $s \leq$
wp-event verify-monad
 ($\lambda \text{out.}$
composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
 $s\ \text{out} \vee$
composition-fri-verifier-tied-assignment-auth-gap s out) s
by (*rule wp-event-mono*)
 (*rule*
composition-fri-verifier-tied-assignment-imp-authenticated-or-auth-gap)
also have $\dots \leq$
wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s*)
 $s +$
wp-event verify-monad
 (*composition-fri-verifier-tied-assignment-auth-gap s*) s
by (*rule wp-event-union-bound*)
also have $\dots \leq C + G$
by (*intro add-mono authenticated-bound auth-gap-bound*)
finally show *?thesis* .
qed

lemma *wp-composition-fri-verifier-tied-assignment-auth-gap-bound-from-same-layer-auth-gap:*
fixes $G :: \text{prob}$
assumes *gap-bound:*
wp-event verify-monad
 (*composition-fri-verifier-tied-same-layer-auth-gap s*) $s \leq G$
shows
wp-event verify-monad

```

      (composition-fri-verifier-tied-assignment-auth-gap s) s ≤ G
proof –
  have wp-event verify-monad
    (composition-fri-verifier-tied-assignment-auth-gap s) s ≤
      wp-event verify-monad
        (composition-fri-verifier-tied-same-layer-auth-gap s) s
  by (rule wp-event-mono)
    (rule
      composition-fri-verifier-tied-assignment-auth-gap-imp-same-layer-auth-gap)
  then show ?thesis
    by (rule order-trans[OF - gap-bound])
qed

lemma wp-composition-fri-verifier-tied-same-layer-auth-gap-bound-from-recorded-auth-gap:
  fixes G :: prob
  assumes gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-same-layer-recorded-auth-gap s) s
      ≤ G
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-same-layer-auth-gap s) s ≤ G
proof –
  have wp-event verify-monad
    (composition-fri-verifier-tied-same-layer-auth-gap s) s ≤
      wp-event verify-monad
        (composition-fri-verifier-tied-same-layer-recorded-auth-gap s) s
  by (rule wp-event-mono)
    (rule
      composition-fri-verifier-tied-same-layer-auth-gap-imp-recorded-auth-gap)
  then show ?thesis
    by (rule order-trans[OF - gap-bound])
qed

lemma wp-composition-fri-verifier-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap:
  fixes G :: prob
  assumes gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap
        s) s ≤ G
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-same-layer-recorded-auth-gap s) s
      ≤ G
proof –
  have wp-event verify-monad
    (composition-fri-verifier-tied-same-layer-recorded-auth-gap s) s ≤
      wp-event verify-monad
        (composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap

```

```

      s) s
    by (rule wp-event-mono-on-support)
      (rule
        composition-fri-verifier-tied-same-layer-recorded-auth-gap-imp-authenticated-recorded-auth-gap-on-support)
  then show ?thesis
    by (rule order-trans[OF - gap-bound])
qed

lemma wp-composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap
fixes G :: prob
assumes gap-bound:
  wp-event verify-monad
    (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
  s ≤ G
shows
  wp-event verify-monad
    (composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap
      s) s ≤ G
proof -
  have wp-event verify-monad
    (composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap
      s) s ≤
    wp-event verify-monad
      (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
      s
  by (rule wp-event-mono)
    (rule
      composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap)
  then show ?thesis
    by (rule order-trans[OF - gap-bound])
qed

lemma wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-auth-gap-and-residuals:
fixes A P M N S G :: prob
assumes auth-gap-bound:
  wp-event verify-monad
    (composition-fri-verifier-tied-assignment-auth-gap s) s ≤ A
and partial-merkle-bound:
  wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ P
and missing-bound:
  wp-event verify-monad
    (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
    s ≤ M
and next-gap-bound:
  wp-event verify-monad
    (composition-fri-verifier-tied-next-value-replay-gap s) s ≤ N
and successor-gap-bound:
  wp-event verify-monad
    (composition-fri-verifier-tied-successor-replay-gap s) s ≤ S

```

and *final-gap-bound*:
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}final\text{-}value\text{-}replay\text{-}gap\ s)\ s \leq G$
shows
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\ s)\ s$
 $\leq ((P + M) + N + S + G + P) + A$
by (rule
 $wp\text{-}composition\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\text{-}bound\text{-}from\text{-}authenticated\text{-}and\text{-}auth\text{-}gap$)
(rule $wp\text{-}composition\text{-}fri\text{-}verifier\text{-}tied\text{-}authenticated\text{-}conflict\text{-}bound\text{-}from\text{-}all\text{-}replay\text{-}gaps$
[OF $partial\text{-}merkle\text{-}bound$ $missing\text{-}bound$ $next\text{-}gap\text{-}bound$
 $successor\text{-}gap\text{-}bound$ $final\text{-}gap\text{-}bound$],
rule $auth\text{-}gap\text{-}bound$)

lemma *wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-same-layer-auth-gap-and-residuals*:

fixes $A\ P\ M\ N\ S\ G :: prob$

assumes *same-layer-gap-bound*:

$wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}same\text{-}layer\text{-}auth\text{-}gap\ s)\ s \leq A$

and *partial-merkle-bound*:

$wp\text{-}event\ verify\text{-}monad\ (partial\text{-}merkle\text{-}inconsistency\text{-}bad\ s)\ s \leq P$

and *missing-bound*:

$wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}recorded\text{-}base\text{-}chunk\text{-}auth\text{-}missing\ s)$
 $s \leq M$

and *next-gap-bound*:

$wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}next\text{-}value\text{-}replay\text{-}gap\ s)\ s \leq N$

and *successor-gap-bound*:

$wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}successor\text{-}replay\text{-}gap\ s)\ s \leq S$

and *final-gap-bound*:

$wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}final\text{-}value\text{-}replay\text{-}gap\ s)\ s \leq G$

shows

$wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\ s)\ s$
 $\leq ((P + M) + N + S + G + P) + A$

by (rule

$wp\text{-}composition\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\text{-}bound\text{-}from\text{-}auth\text{-}gap\text{-}and\text{-}residuals$)

(rule

$wp\text{-}composition\text{-}fri\text{-}verifier\text{-}tied\text{-}assignment\text{-}auth\text{-}gap\text{-}bound\text{-}from\text{-}same\text{-}layer\text{-}auth\text{-}gap$
[OF $same\text{-}layer\text{-}gap\text{-}bound$],

rule $partial\text{-}merkle\text{-}bound$,

rule $missing\text{-}bound$,

rule $next\text{-}gap\text{-}bound$,

rule $successor\text{-}gap\text{-}bound$,

rule $final\text{-}gap\text{-}bound$)

lemma *wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-recorded-auth-gap-and-residuals:*

fixes $A\ P\ M\ N\ S\ G :: \text{prob}$

assumes *recorded-gap-bound:*

wp-event verify-monad
 (*composition-fri-verifier-tied-same-layer-recorded-auth-gap s*) $s \leq A$

and *partial-merkle-bound:*

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq P$

and *missing-bound:*

wp-event verify-monad
 (*composition-fri-verifier-tied-recorded-base-chunk-auth-missing s*)
 $s \leq M$

and *next-gap-bound:*

wp-event verify-monad
 (*composition-fri-verifier-tied-next-value-replay-gap s*) $s \leq N$

and *successor-gap-bound:*

wp-event verify-monad
 (*composition-fri-verifier-tied-successor-replay-gap s*) $s \leq S$

and *final-gap-bound:*

wp-event verify-monad
 (*composition-fri-verifier-tied-final-value-replay-gap s*) $s \leq G$

shows

wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-assignment-conflict s*) $s \leq ((P + M) + N + S + G + P) + A$

by (*rule*

wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-same-layer-auth-gap-and-residuals)

(*rule*

wp-composition-fri-verifier-tied-same-layer-auth-gap-bound-from-recorded-auth-gap

[*OF recorded-gap-bound*],

rule partial-merkle-bound,

rule missing-bound,

rule next-gap-bound,

rule successor-gap-bound,

rule final-gap-bound)

lemma *wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-authenticated-recorded-auth-gap-and*

fixes $A\ P\ M\ N\ S\ G :: \text{prob}$

assumes *gap-bound:*

wp-event verify-monad
 (*composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap s*) $s \leq A$

and *partial-merkle-bound:*

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq P$

and *missing-bound:*

wp-event verify-monad
 (*composition-fri-verifier-tied-recorded-base-chunk-auth-missing s*)
 $s \leq M$

and *next-gap-bound:*

wp-event verify-monad
 (composition-fri-verifier-tied-next-value-replay-gap s) $s \leq N$
and *successor-gap-bound*:
wp-event verify-monad
 (composition-fri-verifier-tied-successor-replay-gap s) $s \leq S$
and *final-gap-bound*:
wp-event verify-monad
 (composition-fri-verifier-tied-final-value-replay-gap s) $s \leq G$
shows
wp-event verify-monad
 (composition-fri-verifier-tied-sampled-assignment-conflict s) s
 $\leq ((P + M) + N + S + G + P) + A$
by (rule
wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-recorded-auth-gap-and-residuals)
 (rule
wp-composition-fri-verifier-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap
 [OF gap-bound],
rule partial-merkle-bound,
rule missing-bound,
rule next-gap-bound,
rule successor-gap-bound,
rule final-gap-bound)

lemma *wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-slot-recorded-chunk-gap-and-residuals*

fixes $A P M N S G :: \text{prob}$

assumes *gap-bound*:

wp-event verify-monad
 (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
 $s \leq A$

and *partial-merkle-bound*:

wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$

and *missing-bound*:

wp-event verify-monad
 (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
 $s \leq M$

and *next-gap-bound*:

wp-event verify-monad
 (composition-fri-verifier-tied-next-value-replay-gap s) $s \leq N$

and *successor-gap-bound*:

wp-event verify-monad
 (composition-fri-verifier-tied-successor-replay-gap s) $s \leq S$

and *final-gap-bound*:

wp-event verify-monad
 (composition-fri-verifier-tied-final-value-replay-gap s) $s \leq G$

shows

wp-event verify-monad
 (composition-fri-verifier-tied-sampled-assignment-conflict s) s
 $\leq ((P + M) + N + S + G + P) + A$

by (rule

wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-authenticated-recorded-auth-gap-and-residuals
 (rule
wp-composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap
 [*OF gap-bound*],
 rule *partial-merkle-bound*,
 rule *missing-bound*,
 rule *next-gap-bound*,
 rule *successor-gap-bound*,
 rule *final-gap-bound*)

lemma *wp-composition-fri-verifier-tied-bound-from-sampled-auth-gap-and-residuals*:

fixes $R\ A\ P\ M\ N\ S\ G :: \text{prob}$

assumes *sampled-bound*:

wp-event verify-monad

(*composition-fri-bad-with-verifier-tied-sampled-layer-chain* s) $s \leq R$

and *auth-gap-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-assignment-auth-gap* s) $s \leq A$

and *partial-merkle-bound*:

wp-event verify-monad (*partial-merkle-inconsistency-bad* s) $s \leq P$

and *missing-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-recorded-base-chunk-auth-missing* s)
 $s \leq M$

and *next-gap-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-next-value-replay-gap* s) $s \leq N$

and *successor-gap-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-successor-replay-gap* s) $s \leq S$

and *final-gap-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-final-value-replay-gap* s) $s \leq G$

shows

wp-event verify-monad

(*composition-fri-bad-with-verifier-tied-partial-candidate* s) s
 $\leq R + (((P + M) + N + S + G + P) + A)$

proof –

have *conflict-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-sampled-assignment-conflict* s) s
 $\leq ((P + M) + N + S + G + P) + A$

by (rule

wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-auth-gap-and-residuals)

(rule *auth-gap-bound*,

rule *partial-merkle-bound*,

rule *missing-bound*,

rule *next-gap-bound*,

rule *successor-gap-bound*,

```

    rule final-gap-bound)
show ?thesis
by (rule
    wp-composition-fri-verifier-tied-bound-from-sampled-conflict
    [OF sampled-bound conflict-bound])
qed

lemma wp-composition-fri-verifier-tied-bound-from-sampled-same-layer-auth-gap-and-residuals:
fixes R A P M N S G :: prob
assumes sampled-bound:
    wp-event verify-monad
      (composition-fri-bad-with-verifier-tied-sampled-layer-chain s)  $s \leq R$ 
and same-layer-gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-same-layer-auth-gap s)  $s \leq A$ 
and partial-merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s)  $s \leq P$ 
and missing-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
       $s \leq M$ 
and next-gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-next-value-replay-gap s)  $s \leq N$ 
and successor-gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-successor-replay-gap s)  $s \leq S$ 
and final-gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-replay-gap s)  $s \leq G$ 
shows
    wp-event verify-monad
      (composition-fri-bad-with-verifier-tied-partial-candidate s)  $s$ 
       $\leq R + (((P + M) + N + S + G + P) + A)$ 
by (rule
    wp-composition-fri-verifier-tied-bound-from-sampled-auth-gap-and-residuals)
(rule sampled-bound,
rule
    wp-composition-fri-verifier-tied-assignment-auth-gap-bound-from-same-layer-auth-gap
    [OF same-layer-gap-bound],
rule partial-merkle-bound,
rule missing-bound,
rule next-gap-bound,
rule successor-gap-bound,
rule final-gap-bound)

```

```

lemma wp-composition-fri-verifier-tied-bound-from-sampled-recorded-auth-gap-and-residuals:
fixes R A P M N S G :: prob
assumes sampled-bound:

```

wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-sampled-layer-chain s) $s \leq R$
and recorded-gap-bound:
wp-event verify-monad
 (composition-fri-verifier-tied-same-layer-recorded-auth-gap s) $s \leq A$
and partial-merkle-bound:
wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$
and missing-bound:
wp-event verify-monad
 (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
 $s \leq M$
and next-gap-bound:
wp-event verify-monad
 (composition-fri-verifier-tied-next-value-replay-gap s) $s \leq N$
and successor-gap-bound:
wp-event verify-monad
 (composition-fri-verifier-tied-successor-replay-gap s) $s \leq S$
and final-gap-bound:
wp-event verify-monad
 (composition-fri-verifier-tied-final-value-replay-gap s) $s \leq G$
shows
wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-partial-candidate s) s
 $\leq R + (((P + M) + N + S + G + P) + A)$
by (rule
 wp-composition-fri-verifier-tied-bound-from-sampled-same-layer-auth-gap-and-residuals)
 (rule sampled-bound,
 rule
 wp-composition-fri-verifier-tied-same-layer-auth-gap-bound-from-recorded-auth-gap
 [OF recorded-gap-bound],
 rule partial-merkle-bound,
 rule missing-bound,
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound)

lemma wp-composition-fri-verifier-tied-bound-from-sampled-authenticated-recorded-auth-gap-and-residuals:

fixes $R A P M N S G :: \text{prob}$

assumes sampled-bound:

wp-event verify-monad

(composition-fri-bad-with-verifier-tied-sampled-layer-chain s) $s \leq R$

and gap-bound:

wp-event verify-monad

(composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap
 s) $s \leq A$

and partial-merkle-bound:

wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$

and missing-bound:

wp-event verify-monad
 (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
 $s \leq M$
and *next-gap-bound*:
wp-event verify-monad
 (composition-fri-verifier-tied-next-value-replay-gap s) $s \leq N$
and *successor-gap-bound*:
wp-event verify-monad
 (composition-fri-verifier-tied-successor-replay-gap s) $s \leq S$
and *final-gap-bound*:
wp-event verify-monad
 (composition-fri-verifier-tied-final-value-replay-gap s) $s \leq G$
shows
wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-partial-candidate s) s
 $\leq R + (((P + M) + N + S + G + P) + A)$
by (rule
 wp-composition-fri-verifier-tied-bound-from-sampled-recorded-auth-gap-and-residuals)
 (rule sampled-bound,
 rule
 wp-composition-fri-verifier-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap
 [OF gap-bound],
 rule partial-merkle-bound,
 rule missing-bound,
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound)

lemma *wp-composition-fri-verifier-tied-bound-from-sampled-slot-recorded-chunk-gap-and-residuals*:

fixes $R A P M N S G :: \text{prob}$

assumes *sampled-bound*:

wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-sampled-layer-chain s) $s \leq R$

and *gap-bound*:

wp-event verify-monad
 (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
 s) $s \leq A$

and *partial-merkle-bound*:

wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$

and *missing-bound*:

wp-event verify-monad
 (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
 $s \leq M$

and *next-gap-bound*:

wp-event verify-monad
 (composition-fri-verifier-tied-next-value-replay-gap s) $s \leq N$

and *successor-gap-bound*:

wp-event verify-monad
 (composition-fri-verifier-tied-successor-replay-gap s) $s \leq S$

and *final-gap-bound*:

$$\text{wp-event verify-monad } (\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq G$$
shows

$$\begin{aligned} &\text{wp-event verify-monad} \\ &(\text{composition-fri-bad-with-verifier-tied-partial-candidate } s) \ s \\ &\leq R + (((P + M) + N + S + G + P) + A) \end{aligned}$$
by (rule

$$\begin{aligned} &\text{wp-composition-fri-verifier-tied-bound-from-sampled-authenticated-recorded-auth-gap-and-residuals}) \\ &(\text{rule sampled-bound,} \\ &\text{rule} \\ &\text{wp-composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap} \\ &[\text{OF gap-bound}], \\ &\text{rule partial-merkle-bound,} \\ &\text{rule missing-bound,} \\ &\text{rule next-gap-bound,} \\ &\text{rule successor-gap-bound,} \\ &\text{rule final-gap-bound}) \end{aligned}$$

lemma *composition-fri-reachable-partial-candidate-reduction-from-verifier-tied-residual-bounds*:
fixes $R \ A \ P \ M \ N \ S \ G \ T :: \text{prob}$
assumes *sampled-bound*:

$$\text{wp-event verify-monad } (\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain } s) \ s \leq R$$
and *slot-gap-bound*:

$$\text{wp-event verify-monad } (\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap } s) \ s \leq A$$
and *partial-merkle-bound*:

$$\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq P$$
and *missing-bound*:

$$\begin{aligned} &\text{wp-event verify-monad} \\ &(\text{composition-fri-verifier-tied-recorded-base-chunk-auth-missing } s) \\ &s \leq M \end{aligned}$$
and *next-gap-bound*:

$$\text{wp-event verify-monad } (\text{composition-fri-verifier-tied-next-value-replay-gap } s) \ s \leq N$$
and *successor-gap-bound*:

$$\text{wp-event verify-monad } (\text{composition-fri-verifier-tied-successor-replay-gap } s) \ s \leq S$$
and *final-gap-bound*:

$$\text{wp-event verify-monad } (\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq G$$
and *verifier-gap-bound*:

$$\text{wp-event verify-monad } (\text{composition-fri-reachable-verifier-tie-gap } s) \ s \leq T$$
and *total-bound*:

$$\begin{aligned} &R + (((P + M) + N + S + G + P) + A) + T \\ &\leq \text{composition-fri-error} \end{aligned}$$

shows *composition-fri-reachable-partial-candidate-reduction-assumption s*
proof –
have *verifier-bound*:
 wp-event verify-monad
 (*composition-fri-bad-with-verifier-tied-partial-candidate s*) *s*
 $\leq R + (((P + M) + N + S + G + P) + A)$
by (*rule*
 wp-composition-fri-verifier-tied-bound-from-sampled-slot-recorded-chunk-gap-and-residuals
 [*OF sampled-bound slot-gap-bound partial-merkle-bound missing-bound*
 next-gap-bound successor-gap-bound final-gap-bound])
show *?thesis*
by (*rule*
 composition-fri-reachable-partial-candidate-reduction-from-verifier-tied-and-gap
 [*OF verifier-bound verifier-gap-bound total-bound*])
qed

lemma *composition-fri-reachable-partial-candidate-reduction-from-verifier-tied-residual-bounds-with-recorded-missing*
fixes *R A P N S G T :: prob*

assumes *sampled-bound*:
 wp-event verify-monad
 (*composition-fri-bad-with-verifier-tied-sampled-layer-chain s*) $s \leq R$
and *slot-gap-bound*:
 wp-event verify-monad
 (*composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap*
 s) $s \leq A$
and *partial-merkle-bound*:
 wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq P$
and *next-gap-bound*:
 wp-event verify-monad
 (*composition-fri-verifier-tied-next-value-replay-gap s*) $s \leq N$
and *successor-gap-bound*:
 wp-event verify-monad
 (*composition-fri-verifier-tied-successor-replay-gap s*) $s \leq S$
and *final-gap-bound*:
 wp-event verify-monad
 (*composition-fri-verifier-tied-final-value-replay-gap s*) $s \leq G$
and *verifier-gap-bound*:
 wp-event verify-monad
 (*composition-fri-reachable-verifier-tie-gap s*) $s \leq T$
and *total-bound*:
 $R + (((P + 0) + N + S + G + P) + A) + T$
 $\leq \text{composition-fri-error}$

shows *composition-fri-reachable-partial-candidate-reduction-assumption s*
by (*rule composition-fri-reachable-partial-candidate-reduction-from-verifier-tied-residual-bounds*
 [**where** $M = 0$])
 (*rule sampled-bound*,
 rule slot-gap-bound,
 rule partial-merkle-bound,
 rule wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero,

rule next-gap-bound,
rule successor-gap-bound,
rule final-gap-bound,
rule verifier-gap-bound,
rule total-bound)

lemma *composition-fri-verifier-tied-reduction-from-residual-bounds:*

fixes $R\ A\ P\ M\ N\ S\ G :: \text{prob}$

assumes *sampled-bound:*

wp-event verify-monad

(composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s ≤ R

and *slot-gap-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s) s ≤ A

and *partial-merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ P

and *missing-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-recorded-base-chunk-auth-missing s) s ≤ M

and *next-gap-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-next-value-replay-gap s) s ≤ N

and *successor-gap-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-successor-replay-gap s) s ≤ S

and *final-gap-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-final-value-replay-gap s) s ≤ G

and *total-bound:*

$R + (((P + M) + N + S + G + P) + A)$
 $\leq \text{composition-fri-error}$

shows *composition-fri-verifier-tied-reduction s*

unfolding *composition-fri-verifier-tied-reduction-def*

by *(rule order-trans[*

OF wp-composition-fri-verifier-tied-bound-from-sampled-slot-recorded-chunk-gap-and-residuals
[OF sampled-bound slot-gap-bound partial-merkle-bound missing-bound
next-gap-bound successor-gap-bound final-gap-bound] total-bound])

lemma *composition-fri-verifier-tied-reduction-from-residual-bounds-without-recorded-missing:*

fixes $R\ A\ P\ N\ S\ G :: \text{prob}$

assumes *sampled-bound:*

wp-event verify-monad

(composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s ≤ R

and *slot-gap-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s) s ≤ A

and *partial-merkle-bound*:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq P$
and *next-gap-bound*:
 $\text{wp-event verify-monad } (\text{composition-fri-verifier-tied-next-value-replay-gap } s) \ s \leq N$
and *successor-gap-bound*:
 $\text{wp-event verify-monad } (\text{composition-fri-verifier-tied-successor-replay-gap } s) \ s \leq S$
and *final-gap-bound*:
 $\text{wp-event verify-monad } (\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq G$
and *total-bound*:
 $R + (((P + A) + N + S + G + P) + A) \leq \text{composition-fri-error}$
shows *composition-fri-verifier-tied-reduction s*
proof –
have *missing-bound*:
 $\text{wp-event verify-monad } (\text{composition-fri-verifier-tied-recorded-base-chunk-auth-missing } s) \ s \leq A$
by (rule
 $\text{wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-bound-from-slot-recorded-chunk-gap}$
 $[\text{OF slot-gap-bound}])$
show *?thesis*
by (rule *composition-fri-verifier-tied-reduction-from-residual-bounds*
 $[\text{OF sampled-bound slot-gap-bound partial-merkle-bound missing-bound}$
 $\text{next-gap-bound successor-gap-bound final-gap-bound total-bound}])$

qed

lemma *composition-fri-verifier-tied-reduction-from-residual-bounds-with-recorded-missing-zero*:

fixes $R \ A \ P \ N \ S \ G :: \text{prob}$

assumes *sampled-bound*:

$\text{wp-event verify-monad } (\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain } s) \ s \leq R$

and *slot-gap-bound*:

$\text{wp-event verify-monad } (\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap } s) \ s \leq A$

and *partial-merkle-bound*:

$\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq P$

and *next-gap-bound*:

$\text{wp-event verify-monad } (\text{composition-fri-verifier-tied-next-value-replay-gap } s) \ s \leq N$

and *successor-gap-bound*:

$\text{wp-event verify-monad } (\text{composition-fri-verifier-tied-successor-replay-gap } s) \ s \leq S$

and *final-gap-bound*:

$\text{wp-event verify-monad } (\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq G$

```

and total-bound:
   $R + (((P + 0) + N + S + G + P) + A)$ 
   $\leq \text{composition-fri-error}$ 
shows composition-fri-verifier-tied-reduction s
by (rule composition-fri-verifier-tied-reduction-from-residual-bounds
  [where  $M = 0$ , OF sampled-bound slot-gap-bound partial-merkle-bound
   wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero
   next-gap-bound successor-gap-bound final-gap-bound total-bound])

end

end

```

```

theory Soundness-FRI-Value-Aligned-Events
  imports Soundness-FRI-Composition-Residuals
begin

```

Value-aligned FRI branch events.

These events are narrower than the raw sampled base-opening conflicts: they keep the trace query opening that supplies the first trace FRI input value. The intended use is to avoid exporting broad verifier-query-round decomposition lemmas, which are expensive for Isabelle to finalize.

```

context soundness
begin

```

```

lemma receive-query-commits-head-layer-outcome:
  fixes  $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$ 
  assumes bfs-eq:  $\text{bfs} = (b, \text{rt}) \# \text{bfs-tail}$ 
  and outcome:
     $\text{Some } (out, t) \in$ 
      set-dist
      (execute
        (mfold (idx, x, len, pw) (receive-query-commits bfs)) s)
  obtains layer-chunks chunk xp xp-path xn xn-path where
    fri-layers-transcript (length bfs) len layer-chunks chunk
     $P\text{Transcript } s = \text{chunk} \ @ \ P\text{Transcript } t$ 
     $P\text{State } t = \text{foldl concat } (P\text{State } s) \ \text{chunk}$ 
     $s \leq t$ 
     $P\text{QueryCounter } t = P\text{QueryCounter } s$ 
     $\text{layer-chunks} \neq []$ 
     $xp = x$ 
    fri-layer-opening-chunk len xp xp-path xn xn-path (hd layer-chunks)
proof –
  have steps:
    receive-query-commits bfs =
      fri-layer-opening-step (b, rt) #
      receive-query-commits bfs-tail
  unfolding bfs-eq receive-query-commits-def fri-layer-opening-step-def

```

```

    by simp
  from outcome[unfolded steps]
  obtain out1 s1 where head:
    Some (out1, s1) ∈
      set-dist
        (execute
          (fri-layer-opening-step (b, rt) (idx, x, len, pw)) s)
  and tail:
    Some (out, t) ∈
      set-dist (execute
        (mfold out1 (receive-query-commits bfs-tail)) s1)
  by (auto elim!: set-dist-bindE)
  from fri-layer-opening-step-outcome[OF head]
  obtain xp xp-path xn xn-path x' where out1-eq:
    out1 = (idx mod (len div 2), x', len div 2, pw + pw)
  and xp-eq: xp = x
  and head-chunk:
    fri-layer-opening-chunk len xp xp-path xn xn-path
      ([xp] @ xp-path @ [xn] @ xn-path)
  and transcript-head:
    PTranscript s =
      ([xp] @ xp-path @ [xn] @ xn-path) @ PTranscript s1
  and state-head:
    PState s1 =
      foldl concat (PState s) ([xp] @ xp-path @ [xn] @ xn-path)
  and ext-head: s ≤ s1
  and counter-head: PQueryCounter s1 = PQueryCounter s
  using fri-layer-opening-step-outcome[OF head]
  by blast
  from receive-query-commits-layers-outcome[OF tail[unfolded out1-eq]]
  obtain tail-layers tail-chunk where tail-layers:
    fri-layers-transcript (length bfs-tail) (len div 2)
      tail-layers tail-chunk
  and transcript-tail:
    PTranscript s1 = tail-chunk @ PTranscript t
  and state-tail:
    PState t = foldl concat (PState s1) tail-chunk
  and ext-tail: s1 ≤ t
  and counter-tail: PQueryCounter t = PQueryCounter s1
  by blast
  let ?head-chunk = [xp] @ xp-path @ [xn] @ xn-path
  let ?layers = ?head-chunk # tail-layers
  let ?chunk = ?head-chunk @ tail-chunk
  have layers:
    fri-layers-transcript (length bfs) len ?layers ?chunk
    using tail-layers head-chunk unfolding bfs-eq fri-layers-transcript-def
    by auto
  have transcript:
    PTranscript s = ?chunk @ PTranscript t

```

```

    using transcript-head transcript-tail by simp
have state:
  PState t = foldl concat (PState s) ?chunk
  using state-head state-tail by simp
have ext: s ≤ t
  by (rule hash-ext-trans[OF ext-head ext-tail])
have counter: PQueryCounter t = PQueryCounter s
  using counter-head counter-tail by simp
show ?thesis
  by (rule that[OF layers transcript state ext counter - xp-eq])
    (use head-chunk in simp-all)
qed

lemma verifier-query-round-after-index-trace-fri-head-value-aligned:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes f-fl-eq: f-fl = (b, rt) # f-fl-tail
  and outcome:
    Some ((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s)
  obtains idx openings xp xp-path xn xn-path where
    idx = index (to-nat raw)
    map opening-index openings = powers-scaled idx
    partial-authenticated-table fr (scale * clength) openings t
    openings ≠ []
    opening-value (hd openings) = xp
    fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
      ([xp] @ xp-path @ [xn] @ xn-path)
  proof -
    let ?idx = index (to-nat raw)
    from outcome obtain fv s1 f-out s2 s3 c-out s4 where
      query-decommit:
        Some (fv, s1) ∈
          set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
    and trace-fri:
      Some (f-out, s2) ∈
        set-dist (execute
          (mfold (?idx, hd fv, clength * scale, 1)
            (receive-query-commits f-fl)) s1)
    and assert-trace:
      Some ((), s3) ∈
        set-dist
          (execute
            (assert (case f-out of (_, f-x, -, -) ⇒ f-x = f-final)) s2)
    and comp-fri:
      Some (c-out, s4) ∈
        set-dist (execute

```

```

    (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
      clength * scale, 1)
      (receive-query-commits fl)) s3)
and assert-comp:
  Some ((), t) ∈
    set-dist
      (execute
        (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have idx-in: ?idx ∈ query-sample-space
  using index-less-query-sample-space
  unfolding query-sample-space-def by simp
from check-decommit-on-query-authenticated-openings
  [OF idx-in query-decommit]
obtain openings where len-openings:
  length openings = length (powers-scaled ?idx)
and values-openings: map opening-value openings = fv
and idx-openings: map opening-index openings = powers-scaled ?idx
and table-s1:
  partial-authenticated-table fr (scale * clength) openings s1
by blast
from receive-query-commits-decomp-at
  [OF trace-fri[unfolded f-fl-eq], of 0]
obtain idx0 x0 len0 pw0 out1 s1' s-suc where
  head:
  Some (out1, s1') ∈
    set-dist
      (execute (fri-layer-opening-step (b, rt)
        (?idx, hd fv, clength * scale, 1)) s1)
  by simp blast
from fri-layer-opening-step-outcome[OF head]
obtain xp xp-path xn xn-path x' where
  xp-eq: xp = hd fv
and chunk:
  fri-layer-opening-chunk (clength * scale) xp xp-path xn xn-path
    ([xp] @ xp-path @ [xn] @ xn-path)
  by blast
have s1-t: s1 ≤ t
proof -
  obtain idx1 x1 len1 pw1 where f-out-eq:
  f-out = (idx1, x1, len1, pw1)
  by (cases f-out)
  have s1-s2:
  s1 ≤ s2
  by (rule receive-query-commits-mfold-hash-extends
    [OF trace-fri[unfolded f-out-eq]])
  have s3-eq: s3 = s2
  using assert-trace unfolding assert-def f-out-eq

```

```

    by (cases x1 = f-final) (auto simp: throw-no-outcome)
  have s3-s4:
    s3 ≤ s4
    by (rule receive-query-commits-mfold-hash-extends[OF comp-fri])
  have t-eq: t = s4
    using assert-comp unfolding assert-def
    by (cases case c-out of (-, x, -, -) ⇒ x = final)
      (auto simp: throw-no-outcome)
  have s2-s4: s2 ≤ s4
    using s3-s4 unfolding s3-eq .
  show ?thesis
    using hash-ext-trans[OF s1-s2 s2-s4] t-eq by simp
qed
have table-t:
  partial-authenticated-table fr (scale * clength) openings t
  by (rule partial-authenticated-table-mono[OF table-s1 s1-t])
have openings-nonempty: openings ≠ []
  using len-openings powers-pos unfolding powers-scaled-def by auto
have hd-value: opening-value (hd openings) = hd fv
  using values-openings openings-nonempty
  by (cases openings; cases fv) simp-all
show ?thesis
  by (rule that[OF refl idx-openings table-t openings-nonempty])
    (use hd-value xp-eq chunk in ⟨simp-all add: mult commute⟩)
qed

lemma verifier-query-round-after-index-trace-fri-head-layer-value-aligned:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes f-fl-eq: f-fl = (b, rt) # f-fl-tail
  and outcome:
    Some (( ), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s)
  obtains idx openings query-chunk trace-layer-chunks composition-layer-chunks
    xp xp-path xn xn-path where
    idx = index (to-nat raw)
    query-round-fri-layer-transcripts idx (map snd f-fl) (map snd fl)
      query-chunk trace-layer-chunks composition-layer-chunks
    PTranscript s = query-chunk @ PTranscript t
    PState t = foldl concat (PState s) query-chunk
    s ≤ t
    PQueryCounter t = PQueryCounter s
    map opening-index openings = powers-scaled idx
    partial-authenticated-table fr (scale * clength) openings t
    openings ≠ []
    opening-value (hd openings) = xp
    fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path

```

```

    (trace-layer-chunks ! 0)
proof -
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1) ∈
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
  and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some ((), s3) ∈
      set-dist
        (execute
          (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
          clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some ((), t) ∈
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have idx-in: ?idx ∈ query-sample-space
  using index-less-query-sample-space
  unfolding query-sample-space-def by simp
from check-decommit-on-query-authenticated-openings
  [OF idx-in query-decommit]
obtain openings where values-openings:
  map opening-value openings = fv
  and idx-openings: map opening-index openings = powers-scaled ?idx
  and table-s1:
    partial-authenticated-table fr (scale * clength) openings s1
  by blast
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths decommit-chunk where query-chunk-shape:
  query-decommitment-transcript ?idx fv query-paths decommit-chunk
  and tr-s-s1:
    PTranscript s = decommit-chunk @ PTranscript s1
  and st-s1:
    PState s1 = foldl concat (PState s) decommit-chunk
  and ext-s-s1: s ≤ s1

```



```

and query-counter-s1: PQueryCounter s1 = PQueryCounter s
by blast
from receive-query-commits-head-layer-outcome[OF f-fl-eq trace-fri]
obtain trace-layer-chunks trace-fri-chunk xp xp-path xn xn-path where
  trace-layers:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
and tr-s1-s2:
  PTranscript s1 = trace-fri-chunk @ PTranscript s2
and st-s2:
  PState s2 = foldl concat (PState s1) trace-fri-chunk
and ext-s1-s2: s1 ≤ s2
and query-counter-s2: PQueryCounter s2 = PQueryCounter s1
and trace-layers-nonempty: trace-layer-chunks ≠ []
and xp-eq: xp = hd fv
and head-chunk:
  fri-layer-opening-chunk (clength * scale) xp xp-path xn xn-path
    (hd trace-layer-chunks)
by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain composition-layer-chunks composition-fri-chunk where
  composition-layers:
    fri-layers-transcript (length fl) (clength * scale)
    composition-layer-chunks composition-fri-chunk
and tr-s3-s4:
  PTranscript s3 = composition-fri-chunk @ PTranscript s4
and st-s4:
  PState s4 = foldl concat (PState s3) composition-fri-chunk
and ext-s3-s4: s3 ≤ s4
and query-counter-s4: PQueryCounter s4 = PQueryCounter s3
by blast
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
let ?query-chunk =
  decommit-chunk @ trace-fri-chunk @ composition-fri-chunk
have round-layers:
  query-round-fri-layer-transcripts ?idx (map snd f-fl) (map snd fl)
    ?query-chunk trace-layer-chunks composition-layer-chunks
unfolding query-round-fri-layer-transcripts-def
by (intro exI[of - decommit-chunk] exI[of - trace-fri-chunk]
  exI[of - composition-fri-chunk] exI[of - fv]
  exI[of - query-paths] conjI)
  (use query-chunk-shape trace-layers composition-layers in

```

```

    ⟨simp-all add: mult.commute⟩)
have tr-t: PTranscript s = ?query-chunk @ PTranscript t
  using tr-s-s1 tr-s1-s2 tr-s3-s4 s3-eq t-eq by simp
have st-t: PState t = foldl concat (PState s) ?query-chunk
  using st-s1 st-s2 st-s4 s3-eq t-eq by simp
have ext-s-t: s ≤ t
  using ext-s-s1 ext-s1-s2 ext-s3-s4 t-eq unfolding s3-eq
  by (meson hash-ext-trans)
have query-counter-t: PQueryCounter t = PQueryCounter s
  using query-counter-s1 query-counter-s2 query-counter-s4
  unfolding s3-eq t-eq by simp
have table-t:
  partial-authenticated-table fr (scale * clength) openings t
  by (rule partial-authenticated-table-mono[OF table-s1])
  (use ext-s1-s2 ext-s3-s4 t-eq s3-eq in
    ⟨meson hash-ext-trans⟩)
have openings-nonempty: openings ≠ []
  using idx-openings powers-pos unfolding powers-scaled-def by auto
have hd-value: opening-value (hd openings) = hd fv
  using values-openings openings-nonempty
  by (cases openings; cases fv) simp-all
have head-chunk-nth:
  fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
  (trace-layer-chunks ! 0)
  using head-chunk trace-layers-nonempty
  by (cases trace-layer-chunks) (simp-all add: mult.commute)
show ?thesis
  by (rule that[OF refl round-layers tr-t st-t ext-s-t query-counter-t
    idx-openings table-t openings-nonempty - head-chunk-nth])
  (use hd-value xp-eq in simp)
qed

```

```

lemma verifier-query-round-program-trace-fri-head-value-aligned-with-lookup:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes f-fl-eq: f-fl = (b, rt) # f-fl-tail
  and outcome:
    Some (( ), t) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  obtains raw idx openings xp xp-path xn xn-path where
    idx = index (to-nat raw)
    map opening-index openings = powers-scaled idx
    partial-authenticated-table fr (scale * clength) openings t
    openings ≠ []
    opening-value (hd openings) = xp
    fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
      ([xp] @ xp-path @ [xn] @ xn-path)
    fmlookup (HashMap t)

```

$(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some } \text{raw}$
proof –
from $\text{outcome}[\text{unfolded verifier-query-round-program-alt-def}]$
obtain $\text{raw } s0$ **where** raw :
 $\text{Some } (\text{raw}, s0) \in$
 $\text{set-dist } (\text{execute receive-query-index-challenge } s)$
and tail :
 $\text{Some } ((), t) \in$
 set-dist
 $(\text{execute}$
 $(\text{verifier-query-round-after-index-program}$
 $\text{fr } f\text{-fl } f\text{-final as } \text{fl final } \text{raw}) s0)$
by $(\text{auto elim!} : \text{set-dist-bindE})$
obtain $\text{idx openings } xp \text{ } xp\text{-path } xn \text{ } xn\text{-path}$ **where**
 $\text{idx-eq} : \text{idx} = \text{index } (\text{to-nat } \text{raw})$
and $\text{idxs} : \text{map opening-index openings} = \text{powers-scaled idx}$
and auth-table :
 $\text{partial-authenticated-table fr } (\text{scale} * \text{clength}) \text{ openings } t$
and $\text{nonempty} : \text{openings} \neq []$
and $\text{opening-value-eq} : \text{opening-value } (\text{hd openings}) = xp$
and chunk :
 $\text{fri-layer-opening-chunk } (\text{scale} * \text{clength}) xp \text{ } xp\text{-path } xn \text{ } xn\text{-path}$
 $([xp] @ xp\text{-path} @ [xn] @ xn\text{-path})$
using $\text{verifier-query-round-after-index-trace-fri-head-value-aligned}$
 $[OF f\text{-fl-eq } \text{tail}]$
by metis
have lookup-s0 :
 $\text{fmlookup } (\text{HashMap } s0)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some } \text{raw}$
using $\text{receive-query-index-challenge-outcome}[OF \text{raw}]$ **by** simp
have $s0\text{-t} : s0 \leq t$
using $\text{verifier-query-round-after-index-program-outcome}[OF \text{tail}]$
by simp
have lookup-t :
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some } \text{raw}$
by $(\text{rule hash-extension-lookup}[OF \text{lookup-s0 } s0\text{-t}])$
show $?thesis$
by $(\text{rule that}[OF \text{idx-eq idxs auth-table nonempty opening-value-eq chunk lookup-t}])$
qed

lemma $\text{ntimes-verifier-query-rounds-selected-trace-fri-head-value-aligned-with-replay-at}$:
fixes $\text{query-state final-state} :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes $f\text{-fl-eq} : f\text{-fl} = (b, rt) \# f\text{-fl-tail}$
and outcome :
 $\text{Some } (\text{results}, \text{final-state}) \in$
 set-dist
 $(\text{execute}$
 $(\text{ntimes}$

```

      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds)
    query-state)
  and i-bound:  $i < \text{rounds}$ 
  and len-raw:  $\text{length raw-idxs} = \text{rounds}$ 
  and query-idxs-eq:
    query-idxs = map ( $\lambda \text{raw. index (to-nat raw)}$ ) raw-idxs
  and len-query-chunks:  $\text{length query-chunks} = \text{rounds}$ 
  and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state
  and query-chunk:
     $\bigwedge i. i < \text{rounds} \implies$ 
      verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
  and replay-lookup:
     $\bigwedge i. i < \text{rounds} \implies$ 
      fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks
      (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
  obtains openings xp xp-path xn xn-path where
    map opening-index openings = powers-scaled (query-idxs ! i)
    partial-authenticated-table fr (scale * clength) openings final-state
    openings  $\neq []$ 
    opening-value (hd openings) = xp
    fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
    ([xp] @ xp-path @ [xn] @ xn-path)
  proof -
  let ?round = verifier-query-round-program fr f-fl f-final as fl final
  from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
  obtain prefix x suffix s-i s-suc where
    prefix:
      Some (prefix, s-i)  $\in$ 
      set-dist (execute (ntimes ?round i) query-state)
  and round:
      Some (x, s-suc)  $\in$  set-dist (execute ?round s-i)
  and suffix:
      Some (suffix, final-state)  $\in$ 
      set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
  by blast
  have x-eq:  $x = ()$ 
  by (cases x) simp
  show ?thesis
  proof (rule query-opening-ntimes-prefix-transcript-state-alignment
    [OF prefix])
    fix prefix-query-idxs prefix-chunks
    assume len-prefix-idxs:  $\text{length prefix-query-idxs} = i$ 

```

```

and len-prefix-chunks: length prefix-chunks = i
and transcript-prefix:
  PTranscript query-state =
    List.concat prefix-chunks @ PTranscript s-i
and state-i:
  PState s-i =
    state-after-query-chunks (PState query-state) prefix-chunks i
and counter-i:
  PQueryCounter s-i = PQueryCounter query-state + i
and prefix-ext: query-state ≤ s-i
and prefix-rounds:
   $\bigwedge k. k < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
show ?thesis
proof (rule verifier-query-round-program-outcome-with-authenticated-trace-openings
  [OF round[unfolded x-eq]])
fix raw idx chunk round-openings
assume idx-eq: idx = index (to-nat raw)
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
and transcript-round:
  PTranscript s-i = chunk @ PTranscript s-suc
and state-suc:
  PState s-suc = foldl concat (PState s-i) chunk
and round-ext: s-i ≤ s-suc
and counter-suc:
  PQueryCounter s-suc = Suc (PQueryCounter s-i)
and lookup-suc:
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
and len-round-openings:
  length round-openings = length (powers-scaled idx)
and trace-indices:
  map opening-index round-openings = powers-scaled idx
and trace-table-suc:
  partial-authenticated-table fr (scale * clength)
    round-openings s-suc
show ?thesis
proof (rule verifier-query-round-program-trace-fri-head-value-aligned-with-lookup
  [OF f-fl-eq round[unfolded x-eq]])
fix raw' idx' openings xp xp-path xn xn-path
assume idx'-eq: idx' = index (to-nat raw')
and opening-indices:
  map opening-index openings = powers-scaled idx'
and auth-table-suc:
  partial-authenticated-table fr (scale * clength) openings

```

```

    s-suc
  and openings-nonempty: openings ≠ []
  and opening-value-eq: opening-value (hd openings) = xp
  and head-chunk:
    fri-layer-opening-chunk (scale * clength) xp xp-path xn
    xn-path ([xp] @ xp-path @ [xn] @ xn-path)
  and lookup-suc':
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw'
  have raw'-eq: raw' = raw
  using lookup-suc lookup-suc' by simp
  show ?thesis
  proof (rule query-opening-ntimes-suffix-transcript-state-alignment
    [OF suffix])
    fix suffix-query-idxs suffix-chunks
    assume len-suffix-idxs:
      length suffix-query-idxs = rounds - Suc i
    and len-suffix-chunks:
      length suffix-chunks = rounds - Suc i
    and transcript-suffix:
      PTranscript s-suc =
      List.concat suffix-chunks @ PTranscript final-state
    and suffix-state:
      PState final-state =
      state-after-query-chunks (PState s-suc) suffix-chunks
      (rounds - Suc i)
    and suffix-counter:
      PQueryCounter final-state =
      PQueryCounter s-suc + (rounds - Suc i)
    and suffix-ext: s-suc ≤ final-state
    and suffix-rounds:
      ∧k. k < rounds - Suc i ⇒
      verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
    let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
    have len-selected-chunks: length ?selected-chunks = rounds
      using len-prefix-chunks len-suffix-chunks i-bound by simp
    have selected-chunk-lens:
      ∧k. k < rounds ⇒
      length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
    proof -
      fix k
      assume k-bound: k < rounds
      show
        length (?selected-chunks ! k) =
        verifier-query-round-transcript-length (query-idxs ! k)

```

```

      (map snd f-fl) (map snd fl)
proof (cases k < i)
case True
have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
  using True len-prefix-chunks by (simp add: nth-append)
have
  length (prefix-chunks ! k) =
    verifier-query-round-transcript-length
      (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
  by (rule verifier-query-round-chunk-length
      [OF prefix-rounds[OF True]])
then show ?thesis
  using nth-eq
  by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
next
case False
show ?thesis
proof (cases k = i)
case True
have nth-eq: ?selected-chunks ! k = chunk
  using True len-prefix-chunks by (simp add: nth-append)
have
  length chunk =
    verifier-query-round-transcript-length idx
      (map snd f-fl) (map snd fl)
  by (rule verifier-query-round-chunk-length[OF chunk-shape])
then show ?thesis
  using nth-eq
  by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
next
case False
let ?k = k - Suc i
have ge: Suc i ≤ k
  using <¬ k < i> False by linarith
have k-bound': ?k < rounds - Suc i
  using ge k-bound by linarith
have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
  using ge len-prefix-chunks by (simp add: nth-append)
have
  length (suffix-chunks ! ?k) =
    verifier-query-round-transcript-length
      (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
  by (rule verifier-query-round-chunk-length
      [OF suffix-rounds[OF k-bound']])
then show ?thesis
  using nth-eq
  by (simp add:

```

```

      verifier-query-round-transcript-length-index-irrelevant)
    qed
  qed
  qed
  have query-chunk-lens:
     $\bigwedge k. k < \text{rounds} \implies$ 
      length (query-chunks ! k) =
        verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
    using query-chunk verifier-query-round-chunk-length by blast
  have transcript-all:
    PTranscript query-state =
      List.concat ?selected-chunks @ PTranscript final-state
    using transcript-prefix transcript-round transcript-suffix by simp
  have concat-eq:
    List.concat ?selected-chunks = List.concat query-chunks
    using transcript-all transcript-query by simp
  have chunks-eq: ?selected-chunks = query-chunks
  proof -
    have same-lens:
       $\bigwedge k. k < \text{length } ?\text{selected-chunks} \implies$ 
        length (?selected-chunks ! k) = length (query-chunks ! k)
      using selected-chunk-lens query-chunk-lens len-selected-chunks
      by simp
    have concat-eq-empty:
      List.concat ?selected-chunks @ [] = List.concat query-chunks
      using concat-eq by simp
    have len-eq: length ?selected-chunks = length query-chunks
      using len-selected-chunks len-query-chunks by simp
    have ?selected-chunks = query-chunks  $\wedge$  ([] :: 'f list) = []
      by (rule concat-append-eq-concat-same-chunk-lengths
        [OF len-eq same-lens concat-eq-empty])
    then show ?thesis
      by simp
  qed
  have prefix-state-eq:
    state-after-query-chunks (PState query-state) prefix-chunks i =
      state-after-query-chunks (PState query-state) query-chunks i
  proof -
    have
      state-after-query-chunks (PState query-state)
        ?selected-chunks i =
        state-after-query-chunks (PState query-state)
          prefix-chunks i
      by (rule state-after-query-chunks-append-prefix
        [OF len-prefix-chunks])
    then show ?thesis
      using chunks-eq by simp
  qed

```



```

have lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks
        (PState query-state) query-chunks i)) =
    Some raw
  using hash-extension-lookup[OF lookup-suc suffix-ext]
  unfolding counter-i state-i prefix-state-eq .
have raw-eq: raw = raw-idxs ! i
  using lookup-final replay-lookup[OF i-bound] by simp
have idx'-eq-query: idx' = query-idxs ! i
  using idx'-eq raw'-eq raw-eq query-idxs-eq len-row i-bound
  by simp
have auth-table-final:
  partial-authenticated-table fr (scale * clength) openings
    final-state
  by (rule partial-authenticated-table-mono
    [OF auth-table-suc suffix-ext])
show ?thesis
  by (rule that[OF - auth-table-final openings-nonempty
    opening-value-eq head-chunk])
    (use opening-indices idx'-eq-query in simp)
qed
qed
qed
qed
qed

```

definition trace-fri-header-tied-value-aligned-base-opening-conflict

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-value-aligned-base-opening-conflict s out
 $\iff$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table}$
 $\text{ round-idx xp xp-path xn xn-path result final-state.}$
 $\text{ out} = \text{Some (result, final-state)} \wedge$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs trace-final}$
 $\text{ dg composition-roots composition-bs composition-final fri-query-idxs}$
 $\text{ trace-round-layers composition-round-layers} \wedge$
 $\text{ verifier-header-transcript s fr trace-roots trace-final as dg}$
 $\text{ composition-roots composition-final rest} \wedge$
 $\text{ accepted-with-partial-trace-openings s out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{ trace-table-low-degree trace-table} \wedge$
 $\text{ round-idx} < \text{length fri-query-idxs} \wedge$

$0 < \text{length } \text{trace-bs} \wedge$
 $\text{trace-openings} ! \text{round-idx} \neq [] \wedge$
 $\text{opening-index } (\text{hd } (\text{trace-openings} ! \text{round-idx})) =$
 $\text{fri-evidence-layer-idx } \text{trace-roots } \text{fri-query-idxs } \text{round-idx } 0 \wedge$
 $\text{opening-value } (\text{hd } (\text{trace-openings} ! \text{round-idx})) = \text{xp} \wedge$
 $\text{fri-layer-step-evidence}$
 $(\text{trace-roots} ! 0)$
 $(\text{trace-bs} ! 0)$
 $(\text{fri-evidence-layer-len } \text{trace-roots } 0)$
 $(\text{fri-evidence-layer-idx } \text{trace-roots } \text{fri-query-idxs } \text{round-idx } 0)$
 1
 $(\text{fri-sibling-index } (\text{fri-evidence-layer-len } \text{trace-roots } 0)$
 $(\text{fri-evidence-layer-idx } \text{trace-roots } \text{fri-query-idxs } \text{round-idx } 0))$
 $\text{xp } \text{xp-path } \text{xn } \text{xn-path}$
 $(\text{fri-evidence-next-idx } \text{trace-roots } \text{fri-query-idxs } \text{round-idx } 0)$
 $(\text{fri-evidence-next-value } \text{trace-roots } \text{trace-bs } \text{fri-query-idxs}$
 $\text{round-idx } 0 \text{ xp } \text{xn})$
 $(\text{trace-round-layers} ! \text{round-idx} ! 0) \wedge$
 $\neg \text{fri-opening-matches-table}$
 $(\text{fri-evidence-layer-len } \text{trace-roots } 0)$
 $(\text{fri-evidence-layer-idx } \text{trace-roots } \text{fri-query-idxs } \text{round-idx } 0)$
 $\text{trace-table } \text{xp } \text{xn})$

definition *trace-fri-header-tied-value-aligned-sibling-mismatch*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-value-aligned-sibling-mismatch } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots } \text{trace-bs } \text{trace-final } \text{dg } \text{composition-roots}$
 $\text{composition-bs } \text{composition-final } \text{fri-query-idxs } \text{trace-round-layers}$
 $\text{composition-round-layers } \text{fr } \text{as } \text{rest } \text{trace-openings } \text{trace-table}$
 $\text{round-idx } \text{xp } \text{xp-path } \text{xn } \text{xn-path } \text{result } \text{final-state}.$
 $\text{out} = \text{Some } (\text{result}, \text{final-state}) \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out } \text{trace-roots } \text{trace-bs } \text{trace-final}$
 $\text{dg } \text{composition-roots } \text{composition-bs } \text{composition-final } \text{fri-query-idxs}$
 $\text{trace-round-layers } \text{composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr } \text{trace-roots } \text{trace-final } \text{as } \text{dg}$
 $\text{composition-roots } \text{composition-final } \text{rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out } \text{fr } \text{fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate } \text{trace-table } \text{trace-openings} \wedge$
 $\neg \text{trace-table-low-degree } \text{trace-table} \wedge$
 $\text{round-idx} < \text{length } \text{fri-query-idxs} \wedge$
 $0 < \text{length } \text{trace-bs} \wedge$
 $\text{trace-openings} ! \text{round-idx} \neq [] \wedge$
 $\text{opening-index } (\text{hd } (\text{trace-openings} ! \text{round-idx})) =$
 $\text{fri-evidence-layer-idx } \text{trace-roots } \text{fri-query-idxs } \text{round-idx } 0 \wedge$
 $\text{opening-value } (\text{hd } (\text{trace-openings} ! \text{round-idx})) = \text{xp} \wedge$

```

fri-layer-step-evidence
  (trace-roots ! 0)
  (trace-bs ! 0)
  (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
  1
  (fri-sibling-index (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
  xp xp-path xn xn-path
  (fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
  (fri-evidence-next-value trace-roots trace-bs fri-query-idxs
    round-idx 0 xp xn)
  (trace-round-layers ! round-idx ! 0) ∧
trace-table !
  fri-sibling-index (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
  ≠ xn)

```

lemma *trace-fri-header-tied-value-aligned-base-imp-sampled-base:*

assumes

trace-fri-header-tied-value-aligned-base-opening-conflict s out

shows *trace-fri-header-tied-sampled-base-opening-conflict s out*

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as rest trace-openings trace-table

round-idx xp xp-path xn xn-path result final-state

where *out-eq: out = Some (result, final-state)*

and *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final

dg composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers

and *header:*

verifier-header-transcript s fr trace-roots trace-final as dg

composition-roots composition-final rest

and *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs

trace-openings

and *cand: partial-trace-table-candidate trace-table trace-openings*

and *not-low: ¬ trace-table-low-degree trace-table*

and *round-bound: round-idx < length fri-query-idxs*

and *trace-bs-nonempty: 0 < length trace-bs*

and *openings-nonempty: trace-openings ! round-idx ≠ []*

and *hd-idx:*

opening-index (hd (trace-openings ! round-idx)) =

fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0

and *hd-value: opening-value (hd (trace-openings ! round-idx)) = xp*

and *step:*

fri-layer-step-evidence

```

(trace-roots ! 0)
(trace-bs ! 0)
(fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
1
(fri-sibling-index (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
xp xp-path xn xn-path
(fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
(fri-evidence-next-value trace-roots trace-bs fri-query-idxs
  round-idx 0 xp xn)
(trace-round-layers ! round-idx ! 0)
and no-match:
  ¬ fri-opening-matches-table
    (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    trace-table xp xn
unfolding
  trace-fri-header-tied-value-aligned-base-opening-conflict-def
by auto
have base:
  generic-fri-sampled-base-opening-conflict trace-table trace-roots
    trace-bs fri-query-idxs trace-round-layers
unfolding generic-fri-sampled-base-opening-conflict-def
by (intro exI[of - round-idx] exI[of - xp] exI[of - xp-path]
  exI[of - xn] exI[of - xn-path] conjI)
  (use round-bound trace-bs-nonempty step no-match in auto)
show ?thesis
unfolding trace-fri-header-tied-sampled-base-opening-conflict-def
by (intro exI conjI)
  (use out-eq fri-openings header partial cand not-low base in auto)
qed

lemma trace-fri-header-tied-value-aligned-base-imp-sibling-mismatch:
assumes
  trace-fri-header-tied-value-aligned-base-opening-conflict s out
shows trace-fri-header-tied-value-aligned-sibling-mismatch s out
proof —
from assms obtain trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr as rest trace-openings trace-table
  round-idx xp xp-path xn xn-path result final-state
where out-eq: out = Some (result, final-state)
and fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final fri-query-idxs
  trace-round-layers composition-round-layers
and header:
  verifier-header-transcript s fr trace-roots trace-final as dg

```

```

    composition-roots composition-final rest
and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
and cand: partial-trace-table-candidate trace-table trace-openings
and not-low:  $\neg$  trace-table-low-degree trace-table
and round-bound: round-idx < length fri-query-idxs
and trace-bs-nonempty: 0 < length trace-bs
and openings-nonempty: trace-openings ! round-idx  $\neq$  []
and hd-idx:
    opening-index (hd (trace-openings ! round-idx)) =
    fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0
and hd-value: opening-value (hd (trace-openings ! round-idx)) = xp
and step:
    fri-layer-step-evidence
    (trace-roots ! 0)
    (trace-bs ! 0)
    (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    1
    (fri-sibling-index (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
    xp xp-path xn xn-path
    (fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
    (fri-evidence-next-value trace-roots trace-bs fri-query-idxs
    round-idx 0 xp xn)
    (trace-round-layers ! round-idx ! 0)
and no-match:
     $\neg$  fri-opening-matches-table
    (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    trace-table xp xn
unfolding
    trace-fri-header-tied-value-aligned-base-opening-conflict-def
by auto
let ?idx =
    fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0
let ?len = fri-evidence-layer-len trace-roots 0
let ?opn = hd (trace-openings ! round-idx)
have round-bound-rounds: round-idx < rounds
    using round-bound accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
    by simp
have opn-in: ?opn  $\in$  set (trace-openings ! round-idx)
    using openings-nonempty by simp
have table-len: length trace-table = scale * clength
    by (rule partial-trace-table-candidateD(1)
    [OF cand round-bound-rounds opn-in])
have idx-bound-eval: ?idx < scale * clength
    using partial-trace-table-candidateD(2)

```

```

    [OF cand round-bound-rounds opn-in] hd-idx
  by simp
have table-left: trace-table ! ?idx = xp
  using partial-trace-table-candidateD(3)
    [OF cand round-bound-rounds opn-in] hd-idx hd-value
  by simp
have roots-nonempty: 0 < length trace-roots
  using accepted-fri-opening-transcript-shapes(2)[OF fri-openings]
    trace-bs-nonempty
  by simp
have len-eq: ?len = scale * clength
  unfolding fri-evidence-layer-len-def
  using fri-layer-lengths-nth-div[of 0 length trace-roots
    clength * scale] roots-nonempty
  by (simp add: mult.commute)
have sibling-mismatch:
  trace-table ! fri-sibling-index ?len ?idx ≠ xn
proof
  assume sibling-eq: trace-table ! fri-sibling-index ?len ?idx = xn
  have fri-opening-matches-table ?len ?idx trace-table xp xn
    unfolding fri-opening-matches-table-def
    using idx-bound-eval table-len table-left sibling-eq len-eq
    by (simp add: mult.commute)
  then show False
    using no-match by contradiction
qed
show ?thesis
  unfolding trace-fri-header-tied-value-aligned-sibling-mismatch-def
  by (intro exI conjI)
    (use out-eq fri-openings header partial cand not-low round-bound
      trace-bs-nonempty openings-nonempty hd-idx hd-value step
      sibling-mismatch in auto)
qed

lemma wp-trace-fri-header-tied-value-aligned-base-bound-from-sibling-mismatch:
  assumes mismatch-bound:
    wp-event verify-monad
      (trace-fri-header-tied-value-aligned-sibling-mismatch s) s ≤ M
  shows
    wp-event verify-monad
      (trace-fri-header-tied-value-aligned-base-opening-conflict s) s ≤ M
proof -
  have wp-event verify-monad
    (trace-fri-header-tied-value-aligned-base-opening-conflict s) s ≤
      wp-event verify-monad
        (trace-fri-header-tied-value-aligned-sibling-mismatch s) s
  by (rule wp-event-mono)
    (rule trace-fri-header-tied-value-aligned-base-imp-sibling-mismatch)
  then show ?thesis

```

by (rule order-trans[*OF - mismatch-bound*])
qed

definition *trace-fri-header-tied-sibling-mismatch-structural-gap*
:: (*f*, *a*) *protocol-channel-scheme* $\Rightarrow$
(*unit list* $\times$ (*f*, *a*) *protocol-channel-scheme*) *option* $\Rightarrow$ *bool*

where
trace-fri-header-tied-sibling-mismatch-structural-gap s out
 $\longleftrightarrow$
trace-fri-header-tied-value-aligned-sibling-mismatch s out $\wedge$
 $\neg$ *partial-merkle-inconsistency-bad s out*

lemma *trace-fri-header-tied-value-aligned-sibling-mismatch-imp-partial-merkle-or-structural-gap*:
assumes *mismatch*:
trace-fri-header-tied-value-aligned-sibling-mismatch s out
shows
partial-merkle-inconsistency-bad s out $\vee$
trace-fri-header-tied-sibling-mismatch-structural-gap s out

proof –
show ?thesis
using *mismatch*
unfolding *trace-fri-header-tied-sibling-mismatch-structural-gap-def*
by blast
qed

lemma *wp-trace-fri-header-tied-value-aligned-sibling-mismatch-bound-from-partial-merkle-and-structural-gap*:
fixes *P G* :: *prob*
assumes *partial-merkle-bound*:
wp-event verify-monad (*partial-merkle-inconsistency-bad s*) *s* $\leq P$
and *gap-bound*:
wp-event verify-monad
(*trace-fri-header-tied-sibling-mismatch-structural-gap s*) *s* $\leq G$
shows
wp-event verify-monad
(*trace-fri-header-tied-value-aligned-sibling-mismatch s*) *s* $\leq P + G$

proof –
have *wp-event verify-monad*
(*trace-fri-header-tied-value-aligned-sibling-mismatch s*) *s* $\leq$
wp-event verify-monad
($\lambda out. \text{partial-merkle-inconsistency-bad } s \text{ out} \vee$
trace-fri-header-tied-sibling-mismatch-structural-gap s out) *s*
by (rule *wp-event-mono*)
(rule
trace-fri-header-tied-value-aligned-sibling-mismatch-imp-partial-merkle-or-structural-gap)
also have ... $\leq$
wp-event verify-monad (*partial-merkle-inconsistency-bad s*) *s* +
wp-event verify-monad
(*trace-fri-header-tied-sibling-mismatch-structural-gap s*) *s*
by (rule *wp-event-union-bound*)

also have ... $\leq P + G$
 by (intro add-mono partial-merkle-bound gap-bound)
 finally show ?thesis .
 qed

definition *trace-fri-header-tied-base-value-alignment-gap*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-header-tied-base-value-alignment-gap } s \text{ out} \longleftrightarrow$
 $\text{trace-fri-header-tied-sampled-base-opening-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-value-aligned-base-opening-conflict } s \text{ out}$

lemma *trace-fri-header-tied-sampled-base-imp-value-aligned-or-gap*:
assumes *trace-fri-header-tied-sampled-base-opening-conflict* $s \text{ out}$
shows
 $\text{trace-fri-header-tied-value-aligned-base-opening-conflict } s \text{ out} \vee$
 $\text{trace-fri-header-tied-base-value-alignment-gap } s \text{ out}$
using *assms*
unfolding *trace-fri-header-tied-base-value-alignment-gap-def*
by *blast*

lemma *wp-trace-fri-header-tied-sampled-base-bound-from-value-aligned-and-gap*:
fixes $V \ G :: \text{prob}$
assumes *value-aligned-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-value-aligned-base-opening-conflict } s) \ s \leq V$
and *gap-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-value-alignment-gap } s) \ s \leq G$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-base-opening-conflict } s) \ s \leq V + G$

proof –
have $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-base-opening-conflict } s) \ s \leq$
 $\text{wp-event verify-monad}$
 $(\lambda \text{out.}$
 $\text{trace-fri-header-tied-value-aligned-base-opening-conflict } s \text{ out} \vee$
 $\text{trace-fri-header-tied-base-value-alignment-gap } s \text{ out}) \ s$
by (rule *wp-event-mono*)
(rule *trace-fri-header-tied-sampled-base-imp-value-aligned-or-gap*)
also have ... $\leq$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-value-aligned-base-opening-conflict } s) \ s +$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-value-alignment-gap } s) \ s$
by (rule *wp-event-union-bound*)
also have ... $\leq V + G$

by (intro add-mono value-aligned-bound gap-bound)
 finally show ?thesis .
 qed

lemma *wp-trace-fri-header-tied-sampled-base-bound-from-sibling-mismatch-and-gap:*

fixes $M\ G :: \text{prob}$

assumes *mismatch-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-value-aligned-sibling-mismatch } s) \ s \leq M$

and *gap-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-base-value-alignment-gap } s) \ s \leq G$

shows

wp-event verify-monad

$(\text{trace-fri-header-tied-sampled-base-opening-conflict } s) \ s \leq M + G$

by (rule

wp-trace-fri-header-tied-sampled-base-bound-from-value-aligned-and-gap)

(rule *wp-trace-fri-header-tied-value-aligned-base-bound-from-sibling-mismatch*
 [OF *mismatch-bound*], rule *gap-bound*)

end

end

theory *Soundness-FRI-Trace-Residuals*

imports

Soundness-FRI-Value-Aligned-Events

Soundness-FRI-Final-Step-Replay

Soundness-FRI-Recorded-Chunk-Replay

begin

Narrow residual refinements for the header-tied trace FRI route. These predicates name the remaining replay gaps for non-base FRI branches without changing the protocol or exporting broad selected-step replay lemmas.

context *soundness*

begin

definition *trace-fri-reachable-header-tie-gap*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-reachable-header-tie-gap } s \text{ out} \longleftrightarrow$

$\text{trace-fri-bad-with-reachable-partial-candidate } s \text{ out} \wedge$

$\neg \text{trace-fri-bad-with-header-tied-partial-candidate } s \text{ out}$

definition *trace-fri-header-tied-candidate-transfer-gap*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-candidate-transfer-gap s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
trace-fri-bad-with-reachable-partial-candidate s out $\wedge$
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
trace-table-low-degree trace-table)

definition *trace-fri-header-tied-candidate-available-gap*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-candidate-available-gap s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
trace-fri-reachable-header-tie-gap s out $\wedge$
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings)

lemma *trace-fri-zero-round-final-obstruction-imp-header-tied-or-gap:*

assumes *trace-fri-zero-round-final-obstruction s out*

shows

trace-fri-bad-with-header-tied-partial-candidate s out $\vee$
trace-fri-reachable-header-tie-gap s out

proof –

have *reachable:*

trace-fri-bad-with-reachable-partial-candidate s out

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr candidate-query-idxs
trace-openings trace-table

```

where evidence:
  trace-fri-partial-candidate-opening-evidence s out trace-roots
  trace-bs trace-final dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table
  unfolding trace-fri-zero-round-final-obstruction-def by blast
have partial:
  trace-fri-partial-candidate-evidence s out fr candidate-query-idxs
  trace-openings trace-table
  using trace-fri-partial-candidate-opening-evidenceD(2)[OF evidence] .
show ?thesis
  unfolding trace-fri-bad-with-reachable-partial-candidate-def
  by (intro exI conjI)
    (rule trace-fri-partial-candidate-evidenceD(1)[OF partial],
     rule trace-fri-partial-candidate-evidenceD(2)[OF partial],
     rule trace-fri-partial-candidate-evidenceD(3)[OF partial])
qed
show ?thesis
proof (cases trace-fri-bad-with-header-tied-partial-candidate s out)
  case True
    then show ?thesis by simp
  next
    case False
      then have trace-fri-reachable-header-tie-gap s out
        unfolding trace-fri-reachable-header-tie-gap-def
        using reachable by simp
      then show ?thesis by simp
qed
qed

lemma wp-trace-fri-zero-round-final-obstruction-bound-from-header-tied-and-gap:
  fixes H G :: prob
  assumes header-bound:
    wp-event verify-monad
    (trace-fri-bad-with-header-tied-partial-candidate s)  $s \leq H$ 
  and gap-bound:
    wp-event verify-monad
    (trace-fri-reachable-header-tie-gap s)  $s \leq G$ 
  shows
    wp-event verify-monad
    (trace-fri-zero-round-final-obstruction s)  $s \leq H + G$ 
proof –
  have wp-event verify-monad
    (trace-fri-zero-round-final-obstruction s)  $s \leq$ 
  wp-event verify-monad
    ( $\lambda out.$ 
      trace-fri-bad-with-header-tied-partial-candidate s out  $\vee$ 
      trace-fri-reachable-header-tie-gap s out)  $s$ 

```

by (rule wp-event-mono)
 (rule trace-fri-zero-round-final-obstruction-imp-header-tied-or-gap)
 also have ... $\leq$
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-partial-candidate s) s +
 wp-event verify-monad (trace-fri-reachable-header-tie-gap s) s
 by (rule wp-event-union-bound)
 also have ... $\leq H + G$
 by (rule add-mono[OF header-bound gap-bound])
 finally show ?thesis .
 qed

definition trace-fri-header-tied-reduction
 :: (f, 'a) protocol-channel-scheme $\Rightarrow$ bool
where
 trace-fri-header-tied-reduction s $\longleftrightarrow$
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-partial-candidate s) s $\leq$
 trace-fri-error

lemma trace-fri-header-tied-reductionD:
assumes trace-fri-header-tied-reduction s
shows
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-partial-candidate s) s $\leq$
 trace-fri-error
using assms **unfolding** trace-fri-header-tied-reduction-def **by** simp

lemma trace-fri-header-tied-reduction-staged-bound:
assumes reductions:
 $\bigwedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 trace-fri-header-tied-reduction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state $\leq$ trace-fri-error
by (rule trace-fri-bad-with-header-tied-partial-candidates-staged-bound)
 (rule trace-fri-header-tied-reductionD[OF reductions])

lemma trace-fri-header-tied-reduction-from-reachable-reduction:
assumes trace-fri-reachable-partial-candidate-reduction-assumption s
shows trace-fri-header-tied-reduction s
proof –

```

have reachable:
  wp-event verify-monad
    (trace-fri-bad-with-reachable-partial-candidate s)  $s \leq$ 
    trace-fri-error
  using assms
  unfolding trace-fri-reachable-partial-candidate-reduction-assumption-def
  by simp
have verifier-tied:
  wp-event verify-monad
    (trace-fri-bad-with-verifier-tied-partial-candidate s)  $s \leq$ 
    trace-fri-error
  by (rule order-trans[OF - reachable])
    (rule wp-event-mono,
      rule trace-fri-bad-with-verifier-tied-imp-reachable-partial-candidate)
show ?thesis
  unfolding trace-fri-header-tied-reduction-def
  by (rule order-trans[OF - verifier-tied])
    (rule wp-event-mono,
      rule trace-fri-bad-with-header-tied-imp-verifier-tied)
qed

lemma trace-fri-header-candidate-non-low-contradicts-header-tie-gap:
assumes gap: trace-fri-reachable-header-tie-gap s out
and fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers
and header:
  verifier-header-transcript s fr trace-roots trace-final as fri-dg
  composition-roots composition-final rest
and partial:
  accepted-with-partial-trace-openings s out fr fri-query-idxs
  trace-openings
and candidate:
  partial-trace-table-candidate trace-table trace-openings
and not-low:  $\neg$  trace-table-low-degree trace-table
shows False
proof –
have trace-fri-bad-with-header-tied-partial-candidate s out
  unfolding trace-fri-bad-with-header-tied-partial-candidate-def
  by (intro exI conjI)
    (rule fri-openings, rule header, rule partial, rule candidate,
      rule not-low)
then show False
  using gap unfolding trace-fri-reachable-header-tie-gap-def by simp
qed

lemma trace-fri-header-candidate-low-imp-transfer-gap:

```

assumes *gap: trace-fri-reachable-header-tie-gap s out*
and *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate:*
partial-trace-table-candidate trace-table trace-openings
and *low: trace-table-low-degree trace-table*
shows *trace-fri-header-tied-candidate-transfer-gap s out*
proof –
have *raw: trace-fri-bad-with-reachable-partial-candidate s out*
using *gap unfolding trace-fri-reachable-header-tie-gap-def* **by** *simp*
show *?thesis*
unfolding *trace-fri-header-tied-candidate-transfer-gap-def*
by *(intro exI conjI)*
(rule raw, rule fri-openings, rule header, rule partial,
rule candidate, rule low)
qed

lemma *trace-fri-header-tied-candidate-available-gap-imp-transfer-gap:*
assumes *trace-fri-header-tied-candidate-available-gap s out*
shows *trace-fri-header-tied-candidate-transfer-gap s out*
proof –
obtain *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table
where *gap: trace-fri-reachable-header-tie-gap s out*
and *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate:*
partial-trace-table-candidate trace-table trace-openings
using *assms*
unfolding *trace-fri-header-tied-candidate-available-gap-def* **by** *blast*

```

show ?thesis
proof (cases trace-table-low-degree trace-table)
  case True
    show ?thesis
    by (rule trace-fri-header-candidate-low-imp-transfer-gap
      [OF gap fri-openings header partial candidate True])
  next
    case False
    have False
    by (rule trace-fri-header-candidate-non-low-contradicts-header-tie-gap
      [OF gap fri-openings header partial candidate False])
    then show ?thesis by simp
qed
qed

lemma wp-trace-fri-header-tied-candidate-available-gap-bound-from-transfer-gap:
assumes transfer-bound:
  wp-event verify-monad
  (trace-fri-header-tied-candidate-transfer-gap s) s ≤ T
shows
  wp-event verify-monad
  (trace-fri-header-tied-candidate-available-gap s) s ≤ T
by (rule order-trans[OF - transfer-bound])
  (rule wp-event-mono,
    rule trace-fri-header-tied-candidate-available-gap-imp-transfer-gap)

lemma trace-fri-reachable-imp-header-tied-or-gap:
assumes trace-fri-bad-with-reachable-partial-candidate s out
shows
  trace-fri-bad-with-header-tied-partial-candidate s out ∨
  trace-fri-reachable-header-tie-gap s out
using assms
unfolding trace-fri-reachable-header-tie-gap-def
by blast

lemma wp-trace-fri-reachable-bound-from-header-tied-and-gap:
fixes H G :: prob
assumes header-bound:
  wp-event verify-monad
  (trace-fri-bad-with-header-tied-partial-candidate s) s ≤ H
and gap-bound:
  wp-event verify-monad
  (trace-fri-reachable-header-tie-gap s) s ≤ G
shows
  wp-event verify-monad
  (trace-fri-bad-with-reachable-partial-candidate s) s ≤ H + G
proof –
have wp-event verify-monad
  (trace-fri-bad-with-reachable-partial-candidate s) s ≤

```

```

    wp-event verify-monad
      (λout.
        trace-fri-bad-with-header-tied-partial-candidate s out ∨
        trace-fri-reachable-header-tie-gap s out) s
  by (rule wp-event-mono)
    (rule trace-fri-reachable-imp-header-tied-or-gap)
also have ... ≤
  wp-event verify-monad
    (trace-fri-bad-with-header-tied-partial-candidate s) s +
  wp-event verify-monad
    (trace-fri-reachable-header-tie-gap s) s
  by (rule wp-event-union-bound)
also have ... ≤ H + G
  by (rule add-mono[OF header-bound gap-bound])
finally show ?thesis .
qed

lemma trace-fri-reachable-partial-candidate-reduction-from-header-tied-and-gap:
  fixes H G :: prob
  assumes header-bound:
    wp-event verify-monad
      (trace-fri-bad-with-header-tied-partial-candidate s) s ≤ H
  and gap-bound:
    wp-event verify-monad
      (trace-fri-reachable-header-tie-gap s) s ≤ G
  and total-bound: H + G ≤ trace-fri-error
  shows trace-fri-reachable-partial-candidate-reduction-assumption s
proof -
  have old-bound:
    wp-event verify-monad
      (trace-fri-bad-with-reachable-partial-candidate s) s ≤ H + G
  by (rule wp-trace-fri-reachable-bound-from-header-tied-and-gap
    [OF header-bound gap-bound])
  show ?thesis
    unfolding trace-fri-reachable-partial-candidate-reduction-assumption-def
    by (rule order-trans[OF old-bound total-bound])
qed

definition trace-fri-header-tied-recorded-base-chunk-auth-missing
  :: ('f, 'a) protocol-channel-scheme ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
  trace-fri-header-tied-recorded-base-chunk-auth-missing s out ⟷
    (∃ trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr as rest trace-openings trace-table
      round-idx xp xp-path xn xn-path result final-state.
      out = Some (result, final-state) ∧
      accepted-fri-opening-transcript s out trace-roots trace-bs)

```


trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table* $\wedge$
round-idx < length fri-query-idxs $\wedge$
0 < length trace-bs $\wedge$
fri-layer-step-evidence
 (*trace-roots ! 0*)
 (*trace-bs ! 0*)
 (*fri-evidence-layer-len trace-roots 0*)
 (*fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0*)
 1
 (*fri-sibling-index (fri-evidence-layer-len trace-roots 0)*
 (*fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0*))
 xp xp-path xn xn-path
 (*fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0*)
 (*fri-evidence-next-value trace-roots trace-bs fri-query-idxs*
 round-idx 0 xp xn)
 (*trace-round-layers ! round-idx ! 0*) $\wedge$
 $\neg$ *fri-opening-matches-table*
 (*fri-evidence-layer-len trace-roots 0*)
 (*fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0*)
 trace-table xp xn $\wedge$
 $\neg$ *fri-layer-chunk-authenticated (trace-roots ! 0)*
 (*fri-evidence-layer-len trace-roots 0*)
 (*fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0*)
 (*trace-round-layers ! round-idx ! 0*) *final-state*)

lemma *trace-fri-header-tied-recorded-base-chunk-auth-missing-false:*

assumes

trace-fri-header-tied-recorded-base-chunk-auth-missing s out

shows *False*

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as rest trace-openings trace-table

round-idx xp xp-path xn xn-path result final-state

where *out-eq: out = Some (result, final-state)*

and *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final dg composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers

and *round-bound: round-idx < length fri-query-idxs*

and *trace-bs-nonempty: 0 < length trace-bs*

and *missing:*

```

  ¬ fri-layer-chunk-authenticated (trace-roots ! 0)
    (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    (trace-round-layers ! round-idx ! 0) final-state
unfolding
  trace-fri-header-tied-recorded-base-chunk-auth-missing-def
by blast
have len-trace-bs:
  length trace-bs = length trace-roots
  using accepted-fri-opening-transcript-shapes(2)[OF fri-openings] .
have trace-nonempty: 0 < length trace-roots
  using trace-bs-nonempty len-trace-bs by simp
have len-query: length fri-query-idxs = rounds
  using accepted-fri-opening-transcript-shapes(7)[OF fri-openings] .
have round-bound-rounds: round-idx < rounds
  using round-bound len-query by simp
have auth:
  fri-layer-chunk-authenticated (trace-roots ! 0) (clength * scale)
    (fri-query-idxs ! round-idx)
    (trace-round-layers ! round-idx ! 0) final-state
by (rule
  accepted-fri-opening-transcript-trace-base-recorded-chunk-authenticated-at
    [OF fri-openings out-eq round-bound-rounds trace-nonempty])
have len0:
  fri-evidence-layer-len trace-roots 0 = clength * scale
  using trace-nonempty
  unfolding fri-evidence-layer-len-def
  by (cases trace-roots) simp-all
have idx0:
  fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0 =
    fri-query-idxs ! round-idx
  using trace-nonempty
  unfolding fri-evidence-layer-idx-def
  by (cases trace-roots) simp-all
show False
  using missing auth unfolding len0 idx0 by contradiction
qed

lemma wp-trace-fri-header-tied-recorded-base-chunk-auth-missing-zero:
shows
  wp-event verify-monad
    (trace-fri-header-tied-recorded-base-chunk-auth-missing s) s ≤ 0
proof -
have wp-event verify-monad
  (trace-fri-header-tied-recorded-base-chunk-auth-missing s) s
  ≤ wp-event verify-monad (λ-. False) s
by (rule wp-event-mono)
  (rule trace-fri-header-tied-recorded-base-chunk-auth-missing-false)
also have ... = 0

```

unfolding *wp-event-def wp-def dist-expect-def* **by** *simp*
finally show *?thesis* .
qed

definition *trace-fri-header-tied-next-value-replay-checked-conflict*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-next-value-replay-checked-conflict s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr trace-openings trace-table}$
 $\text{ round-idx round-idx' layer-idx } v \ v').$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{fr} = \text{hd trace-roots} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{round-idx} < \text{length fri-query-idxs} \wedge$
 $\text{round-idx'} < \text{length fri-query-idxs} \wedge$
 $\text{layer-idx} < \text{length trace-bs} \wedge$
 $\text{fri-evidence-next-idx trace-roots fri-query-idxs round-idx layer-idx} =$
 $\text{fri-evidence-next-idx trace-roots fri-query-idxs round-idx'}$
 $\text{ layer-idx} \wedge$
 $\text{generic-fri-round-forced-next-value trace-roots trace-bs}$
 $\text{ fri-query-idxs trace-round-layers round-idx layer-idx } v \wedge$
 $\text{generic-fri-round-forced-next-value trace-roots trace-bs}$
 $\text{ fri-query-idxs trace-round-layers round-idx' layer-idx } v' \wedge$
 $v \neq v' \wedge$
 $v = v')$

definition *trace-fri-header-tied-next-value-replay-gap*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-next-value-replay-gap s out $\longleftrightarrow$
 $\text{trace-fri-header-tied-sampled-next-value-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-next-value-replay-checked-conflict } s \text{ out}$

lemma *trace-fri-header-tied-next-value-replay-checked-conflict-false:*

$\neg \text{trace-fri-header-tied-next-value-replay-checked-conflict } s \text{ out}$

unfolding *trace-fri-header-tied-next-value-replay-checked-conflict-def*
by *blast*

lemma *trace-fri-header-tied-sampled-next-value-imp-replay-gap*:
assumes *trace-fri-header-tied-sampled-next-value-conflict s out*
shows *trace-fri-header-tied-next-value-replay-gap s out*
using *assms trace-fri-header-tied-next-value-replay-checked-conflict-false*
unfolding *trace-fri-header-tied-next-value-replay-gap-def*
by *simp*

lemma *wp-trace-fri-header-tied-sampled-next-value-bound-from-replay-gap*:
fixes *G :: prob*
assumes *gap-bound*:
wp-event verify-monad
(trace-fri-header-tied-next-value-replay-gap s) s ≤ G
shows
wp-event verify-monad
(trace-fri-header-tied-sampled-next-value-conflict s) s ≤ G
proof –
have *wp-event verify-monad*
(trace-fri-header-tied-sampled-next-value-conflict s) s ≤
wp-event verify-monad
(trace-fri-header-tied-next-value-replay-gap s) s
by *(rule wp-event-mono)*
(rule trace-fri-header-tied-sampled-next-value-imp-replay-gap)
then show *?thesis*
by *(rule order-trans[OF - gap-bound])*
qed

definition *trace-fri-header-tied-successor-replay-checked-conflict*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where
trace-fri-header-tied-successor-replay-checked-conflict s out
 $\iff$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr trace-openings trace-table}$
 $\text{ source-round target-round layer-idx } v \text{ xp xp-path xn xn-path.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{fr} = \text{hd trace-roots} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{source-round} < \text{length fri-query-idxs} \wedge$
 $\text{target-round} < \text{length fri-query-idxs} \wedge$
 $\text{Suc layer-idx} < \text{length trace-bs} \wedge$
 $\text{generic-fri-round-forced-next-value trace-roots trace-bs}$

$fri_query_idxs \text{ trace_round_layers source_round layer_idx } v \wedge$
 $fri_layer_step_evidence$
 $(trace_roots ! Suc \text{ layer_idx})$
 $(trace_bs ! Suc \text{ layer_idx})$
 $(fri_evidence_layer_len \text{ trace_roots } (Suc \text{ layer_idx}))$
 $(fri_evidence_layer_idx \text{ trace_roots } fri_query_idxs \text{ target_round}$
 $(Suc \text{ layer_idx}))$
 $(2 \wedge Suc \text{ layer_idx})$
 $(fri_sibling_index$
 $(fri_evidence_layer_len \text{ trace_roots } (Suc \text{ layer_idx}))$
 $(fri_evidence_layer_idx \text{ trace_roots } fri_query_idxs$
 $\text{ target_round } (Suc \text{ layer_idx})))$
 $xp \text{ xp_path } xn \text{ xn_path}$
 $(fri_evidence_next_idx \text{ trace_roots } fri_query_idxs \text{ target_round}$
 $(Suc \text{ layer_idx}))$
 $(fri_evidence_next_value \text{ trace_roots } trace_bs \text{ fri_query_idxs}$
 $\text{ target_round } (Suc \text{ layer_idx}) \text{ xp } xn)$
 $(trace_round_layers ! \text{ target_round } ! Suc \text{ layer_idx}) \wedge$
 $((fri_evidence_next_idx \text{ trace_roots } fri_query_idxs \text{ source_round}$
 $\text{ layer_idx} =$
 $fri_evidence_layer_idx \text{ trace_roots } fri_query_idxs \text{ target_round}$
 $(Suc \text{ layer_idx}) \wedge$
 $v \neq xp \wedge v = xp) \vee$
 $(fri_evidence_next_idx \text{ trace_roots } fri_query_idxs \text{ source_round}$
 $\text{ layer_idx} =$
 $fri_sibling_index$
 $(fri_evidence_layer_len \text{ trace_roots } (Suc \text{ layer_idx}))$
 $(fri_evidence_layer_idx \text{ trace_roots } fri_query_idxs \text{ target_round}$
 $(Suc \text{ layer_idx})) \wedge$
 $v \neq xn \wedge v = xn)))$

definition *trace-fri-header-tied-successor-replay-gap*

$:: (f, 'a) \text{ protocol_channel_scheme} \Rightarrow$
 $(unit \text{ list} \times (f, 'a) \text{ protocol_channel_scheme}) \text{ option} \Rightarrow bool$

where

$trace_fri_header_tied_successor_replay_gap \text{ s out} \longleftrightarrow$
 $trace_fri_header_tied_sampled_successor_opening_conflict \text{ s out} \wedge$
 $\neg trace_fri_header_tied_successor_replay_checked_conflict \text{ s out}$

lemma *trace-fri-header-tied-successor-replay-checked-conflict-false:*

$\neg trace_fri_header_tied_successor_replay_checked_conflict \text{ s out}$

unfolding *trace-fri-header-tied-successor-replay-checked-conflict-def*
by *blast*

lemma *trace-fri-header-tied-sampled-successor-imp-replay-gap:*

assumes *trace-fri-header-tied-sampled-successor-opening-conflict s out*

shows *trace-fri-header-tied-successor-replay-gap s out*

using *assms trace-fri-header-tied-successor-replay-checked-conflict-false*

unfolding *trace-fri-header-tied-successor-replay-gap-def*

by *simp*

lemma *wp-trace-fri-header-tied-sampled-successor-bound-from-replay-gap*:

fixes $G :: \text{prob}$

assumes *gap-bound*:

wp-event verify-monad

$(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq G$

shows

wp-event verify-monad

$(\text{trace-fri-header-tied-sampled-successor-opening-conflict } s) \ s \leq G$

proof –

have *wp-event verify-monad*

$(\text{trace-fri-header-tied-sampled-successor-opening-conflict } s) \ s \leq$

wp-event verify-monad

$(\text{trace-fri-header-tied-successor-replay-gap } s) \ s$

by (rule *wp-event-mono*)

(rule *trace-fri-header-tied-sampled-successor-imp-replay-gap*)

then show *?thesis*

by (rule *order-trans[OF - gap-bound]*)

qed

definition *trace-fri-header-tied-final-value-replay-checked-conflict*

$:: (\text{'f}, \text{'a}) \text{ protocol-channel-scheme} \Rightarrow$

$(\text{unit list} \times (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-final-value-replay-checked-conflict s out

$\longleftrightarrow$

$(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$

$\text{composition-bs composition-final fri-query-idxs trace-round-layers}$

$\text{composition-round-layers fr trace-openings trace-table round-idx } v.$

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers $\wedge$

fr = *hd trace-roots* $\wedge$

accepted-with-partial-trace-openings s out fr fri-query-idxs

trace-openings $\wedge$

partial-trace-table-candidate trace-table trace-openings $\wedge$

$\neg \text{trace-table-low-degree trace-table} \wedge$

$0 < \text{length trace-bs} \wedge$

$\text{round-idx} < \text{length fri-query-idxs} \wedge$

generic-fri-round-forced-next-value trace-roots trace-bs

fri-query-idxs trace-round-layers round-idx ($\text{length trace-bs} - 1$)

v $\wedge$

v $\neq \text{trace-final} \wedge$

v = *trace-final*)

definition *trace-fri-header-tied-final-value-replay-gap*

$:: (\text{'f}, \text{'a}) \text{ protocol-channel-scheme} \Rightarrow$

$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-header-tied-final-value-replay-gap } s \text{ out} \longleftrightarrow$
 $\text{trace-fri-header-tied-sampled-final-value-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-final-value-replay-checked-conflict } s \text{ out}$

lemma *trace-fri-header-tied-sampled-final-value-conflictE*:
assumes *trace-fri-header-tied-sampled-final-value-conflict* *s out*
obtains *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table
round-idx v
where
accepted-fri-opening-transcript *s out trace-roots trace-bs*
trace-final fri-dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
verifier-header-transcript *s fr trace-roots trace-final as fri-dg*
composition-roots composition-final rest
accepted-with-partial-trace-openings *s out fr fri-query-idxs*
trace-openings
partial-trace-table-candidate trace-table trace-openings
 $\neg \text{trace-table-low-degree } \text{trace-table}$
 $0 < \text{length } \text{trace-bs}$
 $\text{round-idx} < \text{length } \text{fri-query-idxs}$
generic-fri-round-forced-next-value *trace-roots trace-bs*
fri-query-idxs trace-round-layers round-idx $(\text{length } \text{trace-bs} - 1) \text{ } v$
 $v \neq \text{trace-final}$

proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as rest trace-openings
trace-table **where**
fri-openings:
accepted-fri-opening-transcript *s out trace-roots trace-bs*
trace-final fri-dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *header*:
verifier-header-transcript *s fr trace-roots trace-final as fri-dg*
composition-roots composition-final rest
and *partial*:
accepted-with-partial-trace-openings *s out fr fri-query-idxs*
trace-openings
and *candidate*:
partial-trace-table-candidate trace-table trace-openings
and *nonlow*: $\neg \text{trace-table-low-degree } \text{trace-table}$
and *final-conflict*:
generic-fri-sampled-final-value-conflict *trace-roots trace-bs*
trace-final fri-query-idxs trace-round-layers
unfolding *trace-fri-header-tied-sampled-final-value-conflict-def*

by blast
 from generic-fri-sampled-final-value-conflictE[OF final-conflict]
 obtain round-idx v where
 nonempty: $0 < \text{length } \text{trace-bs}$
 and round-bound: $\text{round-idx} < \text{length } \text{fri-query-idxs}$
 and forced:
 generic-fri-round-forced-next-value trace-roots trace-bs
 fri-query-idxs trace-round-layers round-idx $(\text{length } \text{trace-bs} - 1) v$
 and neg: $v \neq \text{trace-final}$
 by blast
 show ?thesis
 by (rule that[OF fri-openings header partial candidate nonlow nonempty
 round-bound forced neg])
 qed

lemma trace-fri-header-tied-final-value-replay-checked-conflict-false:
 $\neg \text{trace-fri-header-tied-final-value-replay-checked-conflict } s \text{ out}$
unfolding trace-fri-header-tied-final-value-replay-checked-conflict-def
 by blast

lemma trace-fri-header-tied-sampled-final-value-imp-replay-gap:
assumes trace-fri-header-tied-sampled-final-value-conflict s out
shows trace-fri-header-tied-final-value-replay-gap s out
using assms trace-fri-header-tied-final-value-replay-checked-conflict-false
unfolding trace-fri-header-tied-final-value-replay-gap-def
 by simp

lemma wp-trace-fri-header-tied-sampled-final-value-bound-from-replay-gap:
fixes $G :: \text{prob}$
assumes gap-bound:
 wp-event verify-monad
 $(\text{trace-fri-header-tied-final-value-replay-gap } s) s \leq G$
shows
 wp-event verify-monad
 $(\text{trace-fri-header-tied-sampled-final-value-conflict } s) s \leq G$
proof –
 have wp-event verify-monad
 $(\text{trace-fri-header-tied-sampled-final-value-conflict } s) s \leq$
 wp-event verify-monad
 $(\text{trace-fri-header-tied-final-value-replay-gap } s) s$
 by (rule wp-event-mono)
 (rule trace-fri-header-tied-sampled-final-value-imp-replay-gap)
then show ?thesis
 by (rule order-trans[OF gap-bound])
 qed

lemma wp-trace-fri-header-tied-without-same-bound-from-replay-gaps:
fixes $B N S F :: \text{prob}$
assumes base-bound:

wp-event verify-monad
 (trace-fri-header-tied-sampled-base-opening-conflict s) $s \leq B$
and next-gap-bound:
wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap s) $s \leq N$
and successor-gap-bound:
wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap s) $s \leq S$
and final-gap-bound:
wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap s) $s \leq F$
shows
wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer s)
 $s \leq B + N + S + F$
by (rule wp-trace-fri-header-tied-without-same-bound-from-branches)
 (rule base-bound,
 rule wp-trace-fri-header-tied-sampled-next-value-bound-from-replay-gap
 [OF next-gap-bound],
 rule wp-trace-fri-header-tied-sampled-successor-bound-from-replay-gap
 [OF successor-gap-bound],
 rule wp-trace-fri-header-tied-sampled-final-value-bound-from-replay-gap
 [OF final-gap-bound])

lemma wp-trace-fri-header-tied-without-same-bound-from-value-alignment-and-replay-gaps:

fixes $M G N S F :: \text{prob}$

assumes mismatch-bound:

wp-event verify-monad
 (trace-fri-header-tied-value-aligned-sibling-mismatch s) $s \leq M$

and alignment-gap-bound:

wp-event verify-monad
 (trace-fri-header-tied-base-value-alignment-gap s) $s \leq G$

and next-gap-bound:

wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap s) $s \leq N$

and successor-gap-bound:

wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap s) $s \leq S$

and final-gap-bound:

wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap s) $s \leq F$

shows

wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer s)
 $s \leq (M + G) + N + S + F$

by (rule wp-trace-fri-header-tied-without-same-bound-from-replay-gaps)

(rule
 wp-trace-fri-header-tied-sampled-base-bound-from-sibling-mismatch-and-gap
 [OF mismatch-bound alignment-gap-bound],

rule next-gap-bound,
rule successor-gap-bound,
rule final-gap-bound)

lemma *wp-trace-fri-header-tied-without-same-bound-from-partial-merkle-structural-and-replay-gaps:*

fixes $P\ H\ G\ N\ S\ F :: \text{prob}$

assumes *partial-merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ P

and *structural-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-sibling-mismatch-structural-gap s) s ≤ H

and *alignment-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-base-value-alignment-gap s) s ≤ G

and *next-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-next-value-replay-gap s) s ≤ N

and *successor-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-successor-replay-gap s) s ≤ S

and *final-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-final-value-replay-gap s) s ≤ F

shows

wp-event verify-monad

(trace-fri-header-tied-sampled-assignment-conflict-without-same-layer s)

$s \leq ((P + H) + G) + N + S + F$

by (*rule*

wp-trace-fri-header-tied-without-same-bound-from-value-alignment-and-replay-gaps)

(*rule*

wp-trace-fri-header-tied-value-aligned-sibling-mismatch-bound-from-partial-merkle-and-structural-gap

[OF partial-merkle-bound structural-gap-bound],

rule alignment-gap-bound,

rule next-gap-bound,

rule successor-gap-bound,

rule final-gap-bound)

lemma *wp-trace-fri-header-tied-authenticated-conflict-bound-from-partial-merkle-structural-and-replay-gaps:*

fixes $P\ H\ G\ N\ S\ F :: \text{prob}$

assumes *partial-merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ P

and *structural-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-sibling-mismatch-structural-gap s) s ≤ H

and *alignment-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-base-value-alignment-gap s) s ≤ G

and *next-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-next-value-replay-gap s) $s \leq N$
and successor-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap s) $s \leq S$
and final-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap s) $s \leq F$
shows
 wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
 $s \leq (((P + H) + G) + N + S + F) + P$
proof –
have without-bound:
 wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer s)
 $s \leq ((P + H) + G) + N + S + F$
by (rule
 wp-trace-fri-header-tied-without-same-bound-from-partial-merkle-structural-and-replay-gaps)
 (rule partial-merkle-bound,
 rule structural-gap-bound,
 rule alignment-gap-bound,
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound)
show ?thesis
by (rule
 wp-trace-fri-header-tied-authenticated-conflict-bound-from-without-same-and-merkle)
 (rule without-bound, rule partial-merkle-bound)
qed

definition trace-fri-header-tied-assignment-auth-gap

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-assignment-auth-gap s out $\longleftrightarrow$
 trace-fri-header-tied-sampled-assignment-conflict s out $\wedge$
 $\neg \text{trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer}$
 s out

definition trace-fri-header-tied-same-layer-auth-gap

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-same-layer-auth-gap s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table}$
 $\text{result final-state.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table* $\wedge$
generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs
fri-query-idxs trace-round-layers $\wedge$
 $\neg$ *generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks*
trace-roots trace-bs fri-query-idxs trace-round-layers final-state)

definition *trace-fri-header-tied-same-layer-recorded-auth-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-same-layer-recorded-auth-gap s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table}$
 $\text{ result final-state.}$
 $\text{out} = \text{Some}(\text{result}, \text{final-state}) \wedge$
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table* $\wedge$
generic-fri-sampled-same-layer-recorded-auth-gap trace-roots trace-bs
fri-query-idxs trace-round-layers final-state)

definition *trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table}$
 $\text{ result final-state.}$
 $\text{out} = \text{Some}(\text{result}, \text{final-state}) \wedge$
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table* $\wedge$
generic-fri-sampled-same-layer-authenticated-recorded-auth-gap
trace-roots trace-bs fri-query-idxs trace-round-layers final-state)

definition *trace-fri-header-tied-authenticated-slot-recorded-chunk-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table}$
 $\text{ result final-state.}$
 $\text{ out} = \text{Some (result, final-state)} \wedge$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{ verifier-header-transcript s fr trace-roots trace-final as dg}$
 $\text{ composition-roots composition-final rest} \wedge$
 $\text{ accepted-with-partial-trace-openings s out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{ trace-table-low-degree trace-table} \wedge$
 $\text{ generic-fri-authenticated-slot-recorded-chunk-gap trace-roots}$
 $\text{ fri-query-idxs trace-round-layers final-state})$

definition *trace-fri-header-tied-final-value-authenticated-replay-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-final-value-authenticated-replay-gap s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table}$
 $\text{ result final-state round-idx v.}$
 $\text{ out} = \text{Some (result, final-state)} \wedge$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{ verifier-header-transcript s fr trace-roots trace-final as dg}$
 $\text{ composition-roots composition-final rest} \wedge$

accepted-with-partial-trace-openings *s* out fr *fri-query-idxs*
trace-openings $\wedge$
partial-trace-table-candidate *trace-table* *trace-openings* $\wedge$
 $\neg$ *trace-table-low-degree* *trace-table* $\wedge$
 $0 < \text{length } \text{trace-bs}$ $\wedge$
 $\text{round-idx} < \text{length } \text{fri-query-idxs}$ $\wedge$
generic-fri-round-forced-next-value *trace-roots* *trace-bs*
fri-query-idxs *trace-round-layers* *round-idx* ($\text{length } \text{trace-bs} - 1$)
v $\wedge$
v $\neq$ *trace-final* $\wedge$
 $\neg$ *partial-merkle-inconsistency-bad* *s* out $\wedge$
generic-fri-recorded-layer-chunk-authenticated *trace-roots*
fri-query-idxs *trace-round-layers* *final-state* *round-idx*
($\text{length } \text{trace-bs} - 1$)

lemma *trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-false*:
assumes

trace-fri-header-tied-authenticated-slot-recorded-chunk-gap *s* out

shows *False*

proof —

from *assms* **obtain** *trace-roots* *trace-bs* *trace-final* *dg* *composition-roots*
composition-bs *composition-final* *fri-query-idxs* *trace-round-layers*
composition-round-layers fr as *rest* *trace-openings* *trace-table*
result *final-state*

where *out-eq*: *out* = *Some* (*result*, *final-state*)

and *fri-openings*:

accepted-fri-opening-transcript *s* out *trace-roots* *trace-bs*
trace-final *dg* *composition-roots* *composition-bs* *composition-final*
fri-query-idxs *trace-round-layers* *composition-round-layers*

and *gap*:

generic-fri-authenticated-slot-recorded-chunk-gap *trace-roots*
fri-query-idxs *trace-round-layers* *final-state*

unfolding *trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-def*

by *blast*

from *gap* **obtain** *round-idx* *layer-idx* *auth-chunk* **where**

round-bound: $\text{round-idx} < \text{length } \text{fri-query-idxs}$

and *layer-bound*: $\text{layer-idx} < \text{length } \text{trace-roots}$

and *missing*:

$\neg$ *generic-fri-recorded-layer-chunk-authenticated* *trace-roots*
fri-query-idxs *trace-round-layers* *final-state* *round-idx* *layer-idx*

unfolding *generic-fri-authenticated-slot-recorded-chunk-gap-def*

by *blast*

have *round-bound-rounds*: $\text{round-idx} < \text{rounds}$

using *round-bound* *accepted-fri-opening-transcript-shapes*(7)[*OF* *fri-openings*]

by *simp*

have *all-layer-bounds*:

$\bigwedge \text{raw } k. k < \text{length } \text{trace-roots} \implies$

$0 < \text{fri-layer-lengths } (\text{length } \text{trace-roots}) (\text{clength} * \text{scale}) ! k \wedge$
fri-layer-indices ($\text{length } \text{trace-roots}$) (*index* (*to-nat* *raw*))

```

      (clength * scale) ! k <
      fri-layer-lengths (length trace-roots) (clength * scale) ! k
    by (rule accepted-fri-opening-transcript-trace-raw-layer-bound
        [OF fri-openings])
  have recorded:
    generic-fri-recorded-layer-chunk-authenticated trace-roots
    fri-query-idxs trace-round-layers final-state round-idx layer-idx
  by (rule
      accepted-fri-opening-transcript-trace-recorded-layer-chunk-authenticated-at
      [OF fri-openings out-eq round-bound-rounds layer-bound
        all-layer-bounds])
  show False
  using missing recorded by simp
qed

```

lemma *wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero:*
shows

```

  wp-event verify-monad
    (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s) s
    ≤ (0::prob)

```

proof –

have

```

  wp-event verify-monad
    (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s) s
    ≤ wp-event verify-monad (λ-. False) s
  by (rule wp-event-mono)
    (rule trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-false)

```

also have ... = 0

unfolding *wp-event-def wp-def dist-expect-def* **by** *simp*

finally show *?thesis* **by** *simp*

qed

definition *trace-fri-header-tied-final-value-selected-step-missing-gap*

```

:: ('f, 'a) protocol-channel-scheme ⇒
  (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool

```

where

```

trace-fri-header-tied-final-value-selected-step-missing-gap s out
  ⇔
  trace-fri-header-tied-final-value-authenticated-replay-gap s out ∧
  (∃ trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers result final-state round-idx v.
    out = Some (result, final-state) ∧
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers ∧
    0 < length trace-bs ∧
    round-idx < length fri-query-idxs ∧
    generic-fri-round-forced-next-value trace-roots trace-bs

```

fri-query-idxs trace-round-layers round-idx (length trace-bs - 1)
v $\wedge$
v $\neq$ *trace-final* $\wedge$
 $\neg$ *partial-merkle-inconsistency-bad s out* $\wedge$
generic-fri-recorded-layer-chunk-authenticated trace-roots
fri-query-idxs trace-round-layers final-state round-idx
(length trace-bs - 1) $\wedge$
 $(\forall$ *actual-chunk axp axp-path axn axn-path.
 $\neg$ (*fri-layer-chunk-authenticated*
(trace-roots ! (length trace-bs - 1))
(fri-evidence-layer-len trace-roots (length trace-bs - 1))
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
(length trace-bs - 1))
actual-chunk final-state $\wedge$
fri-layer-step-evidence
(trace-roots ! (length trace-bs - 1))
(trace-bs ! (length trace-bs - 1))
(fri-evidence-layer-len trace-roots (length trace-bs - 1))
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
(length trace-bs - 1))
 $(2 \wedge (length trace-bs - 1))$
(fri-sibling-index
(fri-evidence-layer-len trace-roots (length trace-bs - 1))
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
(length trace-bs - 1)))
axp axp-path axn axn-path
(fri-evidence-next-idx trace-roots fri-query-idxs round-idx
(length trace-bs - 1))
trace-final actual-chunk)))*

lemma *trace-fri-header-tied-final-value-selected-step-missing-gap-false:*

assumes *gap:*

trace-fri-header-tied-final-value-selected-step-missing-gap s out

shows *False*

proof —

from *gap* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers result final-state round-idx v

where *out-eq:* *out = Some (result, final-state)*

and *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers

and *nonempty-bs:* $0 < length trace-bs$

and *round-bound:* *round-idx < length fri-query-idxs*

and *missing:*

$\wedge$ *actual-chunk axp axp-path axn axn-path.*

$\neg$ (*fri-layer-chunk-authenticated*
(trace-roots ! (length trace-bs - 1)))


```

    (fri-evidence-layer-len trace-roots (length trace-bs - 1))
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
      (length trace-bs - 1))
    actual-chunk final-state  $\wedge$ 
    fri-layer-step-evidence
      (trace-roots ! (length trace-bs - 1))
      (trace-bs ! (length trace-bs - 1))
      (fri-evidence-layer-len trace-roots (length trace-bs - 1))
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
        (length trace-bs - 1))
      (2  $\wedge$  (length trace-bs - 1))
      (fri-sibling-index
        (fri-evidence-layer-len trace-roots (length trace-bs - 1))
        (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
          (length trace-bs - 1)))
      arp arp-path arn arn-path
      (fri-evidence-next-idx trace-roots fri-query-idxs round-idx
        (length trace-bs - 1))
      trace-final actual-chunk)
  unfolding trace-fri-header-tied-final-value-selected-step-missing-gap-def
  by blast
have len-bs-roots: length trace-bs = length trace-roots
  using accepted-fri-opening-transcript-shapes(2)[OF fri-openings] .
have nonempty-roots: 0 < length trace-roots
  using nonempty-bs len-bs-roots by simp
have round-bound-rounds: round-idx < rounds
  using round-bound
    accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
  by simp
have raw-layer:
   $\wedge$ raw.
    0 < fri-layer-lengths (length trace-roots) (clength * scale) !
    (length trace-roots - 1)  $\wedge$ 
    fri-layer-indices (length trace-roots) (index (to-nat raw))
    (clength * scale) ! (length trace-roots - 1) <
    fri-layer-lengths (length trace-roots) (clength * scale) !
    (length trace-roots - 1)
  by (rule accepted-fri-opening-transcript-trace-raw-layer-bound
    [OF fri-openings])
    (use nonempty-roots in simp)
from accepted-fri-opening-transcript-trace-final-step-evidence-at
  [OF fri-openings out-eq round-bound-rounds nonempty-roots raw-layer]
obtain actual-chunk arp arp-path arn arn-path where
  auth:
    fri-layer-chunk-authenticated
      (trace-roots ! (length trace-roots - 1))
      (fri-evidence-layer-len trace-roots (length trace-roots - 1))
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
        (length trace-roots - 1))

```

```

    actual-chunk final-state
  and step:
    fri-layer-step-evidence
      (trace-roots ! (length trace-roots - 1))
      (trace-bs ! (length trace-roots - 1))
      (fri-evidence-layer-len trace-roots (length trace-roots - 1))
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
        (length trace-roots - 1))
      (2 ^ (length trace-roots - 1))
      (fri-sibling-index
        (fri-evidence-layer-len trace-roots (length trace-roots - 1))
        (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
          (length trace-roots - 1)))
      arp arp-path arn arn-path
      (fri-evidence-next-idx trace-roots fri-query-idxs round-idx
        (length trace-roots - 1))
      trace-final actual-chunk
  by blast
show False
using missing[of actual-chunk arp arp-path arn arn-path]
  auth step len-bs-roots
by simp
qed

lemma wp-trace-fri-header-tied-final-value-selected-step-missing-gap-zero:
  shows
    wp-event verify-monad
      (trace-fri-header-tied-final-value-selected-step-missing-gap s) s
      ≤ (0::prob)
proof -
  have
    wp-event verify-monad
      (trace-fri-header-tied-final-value-selected-step-missing-gap s) s ≤
      wp-event verify-monad (λ-. False) s
  by (rule wp-event-mono)
  (rule trace-fri-header-tied-final-value-selected-step-missing-gap-false)
  also have ... = 0
  unfolding wp-event-def wp-def dist-expect-def by simp
  finally show ?thesis by simp
qed

lemma trace-fri-header-tied-final-value-authenticated-replay-gap-imp-selected-step-missing:
  assumes
    trace-fri-header-tied-final-value-authenticated-replay-gap s out
  shows
    trace-fri-header-tied-final-value-selected-step-missing-gap s out
proof -
  from assms obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers

```

```

    composition-round-layers fr as rest trace-openings trace-table
    result final-state round-idx v
where out-eq: out = Some (result, final-state)
and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
and nonempty: 0 < length trace-bs
and round-bound: round-idx < length fri-query-idxs
and forced:
    generic-fri-round-forced-next-value trace-roots trace-bs
    fri-query-idxs trace-round-layers round-idx (length trace-bs - 1) v
and neq: v ≠ trace-final
and no-merkle: ¬ partial-merkle-inconsistency-bad s out
and recorded-auth:
    generic-fri-recorded-layer-chunk-authenticated trace-roots
    fri-query-idxs trace-round-layers final-state round-idx
    (length trace-bs - 1)
unfolding trace-fri-header-tied-final-value-authenticated-replay-gap-def
by blast
have no-step:
  ∧ actual-chunk arp arp-path arn arn-path.
  ¬ (fri-layer-chunk-authenticated
    (trace-roots ! (length trace-bs - 1))
    (fri-evidence-layer-len trace-roots (length trace-bs - 1))
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
      (length trace-bs - 1))
    actual-chunk final-state ∧
    fri-layer-step-evidence
    (trace-roots ! (length trace-bs - 1))
    (trace-bs ! (length trace-bs - 1))
    (fri-evidence-layer-len trace-roots (length trace-bs - 1))
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
      (length trace-bs - 1))
    (2 ^ (length trace-bs - 1))
    (fri-sibling-index
      (fri-evidence-layer-len trace-roots (length trace-bs - 1))
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
        (length trace-bs - 1))))
    arp arp-path arn arn-path
    (fri-evidence-next-idx trace-roots fri-query-idxs round-idx
      (length trace-bs - 1))
    trace-final actual-chunk)
proof
fix actual-chunk arp arp-path arn arn-path
assume step-auth:
  fri-layer-chunk-authenticated
    (trace-roots ! (length trace-bs - 1))
    (fri-evidence-layer-len trace-roots (length trace-bs - 1))

```

```

(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
 (length trace-bs - 1))
actual-chunk final-state ∧
fri-layer-step-evidence
(trace-roots ! (length trace-bs - 1))
(trace-bs ! (length trace-bs - 1))
(fri-evidence-layer-len trace-roots (length trace-bs - 1))
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
 (length trace-bs - 1))
(2 ^ (length trace-bs - 1))
(fri-sibling-index
 (fri-evidence-layer-len trace-roots (length trace-bs - 1))
 (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
 (length trace-bs - 1)))
axp axp-path axn axn-path
(fri-evidence-next-idx trace-roots fri-query-idxs round-idx
 (length trace-bs - 1))
trace-final actual-chunk
have False
  by (rule generic-fri-final-conflict-false-from-authenticated-step
    [OF forced recorded-auth conjunct1[OF step-auth]
     conjunct2[OF step-auth] no-merkle[unfolded out-eq] neq])
  then show False .
qed
have witness:
  ∃ trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers result final-state round-idx v.
  out = Some (result, final-state) ∧
  accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers ∧
  0 < length trace-bs ∧
  round-idx < length fri-query-idxs ∧
  generic-fri-round-forced-next-value trace-roots trace-bs
    fri-query-idxs trace-round-layers round-idx (length trace-bs - 1)
    v ∧
  v ≠ trace-final ∧
  ¬ partial-merkle-inconsistency-bad s out ∧
  generic-fri-recorded-layer-chunk-authenticated trace-roots
    fri-query-idxs trace-round-layers final-state round-idx
    (length trace-bs - 1) ∧
  (∀ actual-chunk axp axp-path axn axn-path.
    ¬ (fri-layer-chunk-authenticated
      (trace-roots ! (length trace-bs - 1))
      (fri-evidence-layer-len trace-roots (length trace-bs - 1))
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
        (length trace-bs - 1))
      actual-chunk final-state ∧

```

```

    fri-layer-step-evidence
      (trace-roots ! (length trace-bs - 1))
      (trace-bs ! (length trace-bs - 1))
      (fri-evidence-layer-len trace-roots (length trace-bs - 1))
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
        (length trace-bs - 1))
      (2 ^ (length trace-bs - 1))
      (fri-sibling-index
        (fri-evidence-layer-len trace-roots (length trace-bs - 1))
        (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
          (length trace-bs - 1)))
      arp arp-path arn arn-path
      (fri-evidence-next-idx trace-roots fri-query-idxs round-idx
        (length trace-bs - 1))
      trace-final actual-chunk))
  using out-eq fri-openings nonempty round-bound forced neq no-merkle
    recorded-auth no-step
  by blast
show ?thesis
  unfolding trace-fri-header-tied-final-value-selected-step-missing-gap-def
  using assms witness by blast
qed

lemma wp-trace-fri-header-tied-final-value-authenticated-replay-gap-bound-from-selected-step-missing:
  fixes G :: prob
  assumes gap-bound:
    wp-event verify-monad
      (trace-fri-header-tied-final-value-selected-step-missing-gap s) s
      ≤ G
  shows
    wp-event verify-monad
      (trace-fri-header-tied-final-value-authenticated-replay-gap s) s
      ≤ G
proof -
  have wp-event verify-monad
    (trace-fri-header-tied-final-value-authenticated-replay-gap s) s ≤
    wp-event verify-monad
      (trace-fri-header-tied-final-value-selected-step-missing-gap s) s
  by (rule wp-event-mono)
  (rule
    trace-fri-header-tied-final-value-authenticated-replay-gap-imp-selected-step-missing)
  then show ?thesis
  by (rule order-trans[OF gap-bound])
qed

lemma trace-fri-header-tied-final-value-replay-gap-imp-merkle-or-slot-or-authenticated:
  assumes trace-fri-header-tied-final-value-replay-gap s out
  shows
    partial-merkle-inconsistency-bad s out ∨

```

```

    trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s out ∨
    trace-fri-header-tied-final-value-authenticated-replay-gap s out
proof –
from assms have sampled:
    trace-fri-header-tied-sampled-final-value-conflict s out
    unfolding trace-fri-header-tied-final-value-replay-gap-def by simp
from trace-fri-header-tied-sampled-final-value-conflictE[OF sampled]
obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as rest trace-openings trace-table
    round-idx v
where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
and header:
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest
and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
and cand: partial-trace-table-candidate trace-table trace-openings
and not-low:  $\neg$  trace-table-low-degree trace-table
and nonempty:  $0 < \text{length } \text{trace-bs}$ 
and round-bound: round-idx < length fri-query-idxs
and forced:
    generic-fri-round-forced-next-value trace-roots trace-bs
    fri-query-idxs trace-round-layers round-idx (length trace-bs - 1) v
and neq:  $v \neq \text{trace-final}$ 
by blast
from fri-openings obtain result final-state where out-eq:
    out = Some (result, final-state)
    unfolding accepted-fri-opening-transcript-def by blast
show ?thesis
proof (cases partial-merkle-inconsistency-bad s out)
    case True
    then show ?thesis by simp
next
    case no-merkle: False
    let ?j = length trace-bs - 1
    have len-trace-bs: length trace-bs = length trace-roots
    using accepted-fri-opening-transcript-shapes(2)[OF fri-openings] .
    have layer-bound: ?j < length trace-roots
    using nonempty len-trace-bs by simp
    show ?thesis
    proof (cases generic-fri-recorded-layer-chunk-authenticated trace-roots
        fri-query-idxs trace-round-layers final-state round-idx ?j)
        case True
        have authenticated-gap:

```

```

    trace-fri-header-tied-final-value-authenticated-replay-gap s out
  unfolding
    trace-fri-header-tied-final-value-authenticated-replay-gap-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule header, rule partial,
     rule cand, rule not-low, rule nonempty, rule round-bound,
     rule forced, rule neg, use no-merkle True in simp-all)
  then show ?thesis by simp
next
case missing-recorded: False
have round-bound-rounds: round-idx < rounds
  using round-bound
    accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
  by simp
have raw-layer:
   $\bigwedge_{\text{raw.}}$ 
    0 < fri-layer-lengths (length trace-roots) (clength * scale) !
    ?j  $\wedge$ 
    fri-layer-indices (length trace-roots) (index (to-nat raw))
    (clength * scale) ! ?j <
    fri-layer-lengths (length trace-roots) (clength * scale) ! ?j
  by (rule accepted-fri-opening-transcript-trace-raw-layer-bound
    [OF fri-openings layer-bound])
from accepted-fri-opening-transcript-trace-selected-chunk-authenticated-at
  [OF fri-openings out-eq round-bound-rounds layer-bound raw-layer]
obtain auth-chunk where auth:
  fri-layer-chunk-authenticated (trace-roots ! ?j)
  (fri-layer-lengths (length trace-roots) (clength * scale) ! ?j)
  (fri-layer-indices (length trace-roots) (fri-query-idxs ! round-idx)
   (clength * scale) ! ?j)
  auth-chunk final-state
  by blast
have step:
  fri-round-layer-evidence trace-roots trace-bs fri-query-idxs
  round-idx ?j trace-round-layers
  by (rule accepted-fri-opening-transcript-trace-round-layer-evidence
    [OF fri-openings round-bound-rounds layer-bound])
have generic-step:
  generic-fri-transcript-step-with-authenticated-chunk trace-roots
  trace-bs fri-query-idxs trace-round-layers final-state
  round-idx ?j
  by (rule generic-fri-transcript-step-with-authenticated-chunkI
    [OF step])
    (use auth in
      $\langle$ simp add: fri-evidence-layer-len-def
      fri-evidence-layer-idx-def $\rangle$ )
have slot-gap:
  generic-fri-authenticated-slot-recorded-chunk-gap trace-roots
  fri-query-idxs trace-round-layers final-state

```

```

    by (rule
      generic-fri-transcript-step-auth-missing-imp-authenticated-slot-recorded-chunk-gap
      [OF generic-step missing-recorded])
  have route-slot-gap:
    trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s out
  unfolding trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule header, rule partial,
     rule cand, rule not-low, rule slot-gap)
  then show ?thesis by simp
qed
qed
qed

lemma wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-authenticated:
  fixes M A R :: prob
  assumes merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M
  and slot-bound:
    wp-event verify-monad
      (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s) s ≤ A
  and replay-bound:
    wp-event verify-monad
      (trace-fri-header-tied-final-value-authenticated-replay-gap s) s ≤ R
  shows
    wp-event verify-monad
      (trace-fri-header-tied-final-value-replay-gap s) s ≤ M + A + R
proof -
  let ?P = partial-merkle-inconsistency-bad s
  let ?Q = trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s
  let ?R =
    trace-fri-header-tied-final-value-authenticated-replay-gap s
  have event-le:
    wp-event verify-monad
      (trace-fri-header-tied-final-value-replay-gap s) s ≤
      wp-event verify-monad (λout. ?P out ∨ ?Q out ∨ ?R out) s
  by (rule wp-event-mono)
    (rule
      trace-fri-header-tied-final-value-replay-gap-imp-merkle-or-slot-or-authenticated)
  have union-le:
    wp-event verify-monad (λout. ?P out ∨ ?Q out ∨ ?R out) s ≤
    wp-event verify-monad ?P s +
    wp-event verify-monad ?Q s +
    wp-event verify-monad ?R s
proof -
  have wp-event verify-monad (λout. ?P out ∨ ?Q out ∨ ?R out) s ≤
    wp-event verify-monad ?P s +
    wp-event verify-monad (λout. ?Q out ∨ ?R out) s
  by (rule wp-event-union-bound)

```



```

also have ...  $\leq$ 
  wp-event verify-monad ?P s +
    (wp-event verify-monad ?Q s + wp-event verify-monad ?R s)
  by (intro add-mono order.refl wp-event-union-bound)
finally show ?thesis
  by (simp add: algebra-simps)
qed
have bounds:
  wp-event verify-monad ?P s +
    wp-event verify-monad ?Q s +
    wp-event verify-monad ?R s  $\leq$  M + A + R
  using merkle-bound slot-bound replay-bound
  by (intro add-mono)
show ?thesis
  by (rule order-trans[OF event-le order-trans[OF union-le bounds]])
qed

lemma wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step:
fixes M A R :: prob
assumes merkle-bound:
  wp-event verify-monad (partial-merkle-inconsistency-bad s) s  $\leq$  M
and slot-bound:
  wp-event verify-monad
    (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s) s  $\leq$  A
and selected-step-bound:
  wp-event verify-monad
    (trace-fri-header-tied-final-value-selected-step-missing-gap s) s
     $\leq$  R
shows
  wp-event verify-monad
    (trace-fri-header-tied-final-value-replay-gap s) s  $\leq$  M + A + R
proof -
have authenticated-bound:
  wp-event verify-monad
    (trace-fri-header-tied-final-value-authenticated-replay-gap s) s
     $\leq$  R
  by (rule
    wp-trace-fri-header-tied-final-value-authenticated-replay-gap-bound-from-selected-step-missing
    [OF selected-step-bound])
show ?thesis
  by (rule
    wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-authenticated)
    (rule merkle-bound, rule slot-bound, rule authenticated-bound)
qed

definition trace-fri-header-tied-base-head-value-mismatch-gap
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where

```

$\text{trace-fri-header-tied-base-head-value-mismatch-gap } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\quad \text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\quad \text{composition-round-layers fr as rest trace-openings trace-table}$
 $\quad \text{round-idx xp xp-path xn xn-path result final-state.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\quad \text{trace-final dg composition-roots composition-bs composition-final}$
 $\quad \text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\quad \text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\quad \text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{round-idx} < \text{length fri-query-idxs} \wedge$
 $0 < \text{length trace-bs} \wedge$
 $\text{fri-layer-step-evidence}$
 $\quad (\text{trace-roots} ! 0)$
 $\quad (\text{trace-bs} ! 0)$
 $\quad (\text{fri-evidence-layer-len trace-roots } 0)$
 $\quad (\text{fri-evidence-layer-idx trace-roots fri-query-idxs round-idx } 0)$
 $\quad 1$
 $\quad (\text{fri-sibling-index (fri-evidence-layer-len trace-roots } 0)$
 $\quad \quad (\text{fri-evidence-layer-idx trace-roots fri-query-idxs round-idx } 0))$
 $\quad \text{xp xp-path xn xn-path}$
 $\quad (\text{fri-evidence-next-idx trace-roots fri-query-idxs round-idx } 0)$
 $\quad (\text{fri-evidence-next-value trace-roots trace-bs fri-query-idxs}$
 $\quad \quad \text{round-idx } 0 \text{ xp xn})$
 $\quad (\text{trace-round-layers} ! \text{round-idx} ! 0) \wedge$
 $\neg \text{fri-opening-matches-table}$
 $\quad (\text{fri-evidence-layer-len trace-roots } 0)$
 $\quad (\text{fri-evidence-layer-idx trace-roots fri-query-idxs round-idx } 0)$
 $\quad \text{trace-table xp xn} \wedge$
 $\neg (\text{trace-openings} ! \text{round-idx} \neq [] \wedge$
 $\quad \text{opening-index (hd (trace-openings} ! \text{round-idx}))} =$
 $\quad \text{fri-evidence-layer-idx trace-roots fri-query-idxs round-idx } 0 \wedge$
 $\quad \text{opening-value (hd (trace-openings} ! \text{round-idx}))} = \text{xp}))$

lemma *trace-fri-header-tied-base-value-alignment-gap-imp-head-value-mismatch-gap:*

assumes *trace-fri-header-tied-base-value-alignment-gap s out*

shows *trace-fri-header-tied-base-head-value-mismatch-gap s out*

proof –

from *assms* **have** *sampled:*

trace-fri-header-tied-sampled-base-opening-conflict s out

and *no-aligned:*

$\neg \text{trace-fri-header-tied-value-aligned-base-opening-conflict } s \text{ out}$

unfolding *trace-fri-header-tied-base-value-alignment-gap-def* **by** *simp-all*

from *sampled* **obtain** *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table
where *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *header*:
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
and *partial*:
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *cand*: *partial-trace-table-candidate trace-table trace-openings*
and *not-low*: $\neg$ *trace-table-low-degree trace-table*
and *generic*:
generic-fri-sampled-base-opening-conflict trace-table trace-roots
trace-bs fri-query-idxs trace-round-layers
unfolding *trace-fri-header-tied-sampled-base-opening-conflict-def*
by *blast*
from *fri-openings* **obtain** *result final-state* **where** *out-eq*:
out = Some (result, final-state)
unfolding *accepted-fri-opening-transcript-def* **by** *blast*
from *generic* **obtain** *round-idx xp xp-path xn xn-path* **where**
trace-bs-nonempty: $0 < \text{length } \text{trace-bs}$
and *round-bound*: $\text{round-idx} < \text{length } \text{fri-query-idxs}$
and *step*:
fri-layer-step-evidence
(trace-roots ! 0)
(trace-bs ! 0)
(fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
1
(fri-sibling-index (fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
xp xp-path xn xn-path
(fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
(fri-evidence-next-value trace-roots trace-bs fri-query-idxs
round-idx 0 xp xn)
(trace-round-layers ! round-idx ! 0)
and *no-match*:
 $\neg$ *fri-opening-matches-table*
(fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
trace-table xp xn
unfolding *generic-fri-sampled-base-opening-conflict-def*
by *blast*
have *no-head*:
 $\neg (\text{trace-openings} ! \text{round-idx} \neq [] \wedge$
 $\text{opening-index } (\text{hd } (\text{trace-openings} ! \text{round-idx})) =$

```

    fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0  $\wedge$ 
    opening-value (hd (trace-openings ! round-idx)) = xp)
proof
  assume head:
    trace-openings ! round-idx  $\neq []$   $\wedge$ 
    opening-index (hd (trace-openings ! round-idx)) =
      fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0  $\wedge$ 
      opening-value (hd (trace-openings ! round-idx)) = xp
  have aligned:
    trace-fri-header-tied-value-aligned-base-opening-conflict s out
  unfolding trace-fri-header-tied-value-aligned-base-opening-conflict-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,
     rule not-low, rule round-bound, rule trace-bs-nonempty,
     use head step no-match in simp-all)
  then show False
    using no-aligned by contradiction
qed
show ?thesis
  unfolding trace-fri-header-tied-base-head-value-mismatch-gap-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,
     rule not-low, rule round-bound, rule trace-bs-nonempty, rule step,
     rule no-match, rule no-head)
qed

lemma wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap:
  fixes G :: prob
  assumes mismatch-bound:
    wp-event verify-monad
    (trace-fri-header-tied-base-head-value-mismatch-gap s) s  $\leq$  G
  shows
    wp-event verify-monad
    (trace-fri-header-tied-base-value-alignment-gap s) s  $\leq$  G
proof -
  have wp-event verify-monad
    (trace-fri-header-tied-base-value-alignment-gap s) s  $\leq$ 
    wp-event verify-monad
    (trace-fri-header-tied-base-head-value-mismatch-gap s) s
  by (rule wp-event-mono)
    (rule
      trace-fri-header-tied-base-value-alignment-gap-imp-head-value-mismatch-gap)
  then show ?thesis
    by (rule order-trans[OF - mismatch-bound])
qed

definition trace-fri-header-tied-base-head-value-neq-gap
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool

```

where

$\text{trace-fri-header-tied-base-head-value-neq-gap } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table}$
 $\text{round-idx xp xp-path xn xn-path result final-state.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{round-idx} < \text{length fri-query-idxs} \wedge$
 $0 < \text{length trace-bs} \wedge$
 $\text{trace-openings} ! \text{round-idx} \neq [] \wedge$
 $\text{opening-index (hd (trace-openings ! round-idx))} =$
 $\text{fri-evidence-layer-idx trace-roots fri-query-idxs round-idx } 0 \wedge$
 $\text{opening-value (hd (trace-openings ! round-idx))} \neq \text{xp} \wedge$
 $\text{fri-layer-step-evidence}$
 $(\text{trace-roots} ! 0)$
 $(\text{trace-bs} ! 0)$
 $(\text{fri-evidence-layer-len trace-roots } 0)$
 $(\text{fri-evidence-layer-idx trace-roots fri-query-idxs round-idx } 0)$
 1
 $(\text{fri-sibling-index (fri-evidence-layer-len trace-roots } 0)$
 $(\text{fri-evidence-layer-idx trace-roots fri-query-idxs round-idx } 0))$
 $\text{xp xp-path xn xn-path}$
 $(\text{fri-evidence-next-idx trace-roots fri-query-idxs round-idx } 0)$
 $(\text{fri-evidence-next-value trace-roots trace-bs fri-query-idxs}$
 $\text{round-idx } 0 \text{ xp xn})$
 $(\text{trace-round-layers} ! \text{round-idx} ! 0) \wedge$
 $\neg \text{fri-opening-matches-table}$
 $(\text{fri-evidence-layer-len trace-roots } 0)$
 $(\text{fri-evidence-layer-idx trace-roots fri-query-idxs round-idx } 0)$
 $\text{trace-table xp xn})$

lemma *trace-fri-header-tied-base-head-value-mismatch-gap-imp-neq-gap:*

assumes *trace-fri-header-tied-base-head-value-mismatch-gap s out*

shows *trace-fri-header-tied-base-head-value-neq-gap s out*

proof —

from *assms* **show** *?thesis*

unfolding *trace-fri-header-tied-base-head-value-mismatch-gap-def*

proof (*elim exE conjE*)

fix *trace-roots trace-bs trace-final dg composition-roots composition-bs*
composition-final fri-query-idxs trace-round-layers

```

    composition-round-layers fr as rest trace-openings trace-table
    round-idx xp xp-path xn xn-path result final-state
assume out-eq: out = Some (result, final-state)
assume fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
assume header:
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest
assume partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
assume cand: partial-trace-table-candidate trace-table trace-openings
assume not-low:  $\neg$  trace-table-low-degree trace-table
assume round-bound: round-idx < length fri-query-idxs
assume trace-bs-nonempty: 0 < length trace-bs
assume step:
    fri-layer-step-evidence
    (trace-roots ! 0)
    (trace-bs ! 0)
    (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    1
    (fri-sibling-index (fri-evidence-layer-len trace-roots 0)
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
    xp xp-path xn xn-path
    (fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
    (fri-evidence-next-value trace-roots trace-bs fri-query-idxs
      round-idx 0 xp xn)
    (trace-round-layers ! round-idx ! 0)
assume no-match:
     $\neg$  fri-opening-matches-table
    (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    trace-table xp xn
assume no-head:
     $\neg$  (trace-openings ! round-idx  $\neq$  []  $\wedge$ 
      opening-index (hd (trace-openings ! round-idx)) =
        fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0  $\wedge$ 
      opening-value (hd (trace-openings ! round-idx)) = xp)
have round-bound-rounds: round-idx < rounds
    using round-bound
    accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
    by simp
have roots-nonempty: 0 < length trace-roots
    using accepted-fri-opening-transcript-shapes(2)[OF fri-openings]
    trace-bs-nonempty by simp
have idxs:

```

```

    map opening-index (trace-openings ! round-idx) =
      powers-scaled (fri-query-idxs ! round-idx)
  by (rule accepted-with-partial-trace-openings-shapes(3)
    [OF partial round-bound-rounds])
have openings-nonempty: trace-openings ! round-idx ≠ []
  using idxs powers-pos unfolding powers-scaled-def by auto
have hd-idx:
  opening-index (hd (trace-openings ! round-idx)) =
    fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0
proof -
  have opening-index (hd (trace-openings ! round-idx)) =
    fri-query-idxs ! round-idx
  proof -
    have opening-index (hd (trace-openings ! round-idx)) =
      hd (map opening-index (trace-openings ! round-idx))
    using openings-nonempty by (cases trace-openings ! round-idx) auto
  also have ... = hd (powers-scaled (fri-query-idxs ! round-idx))
    using idxs by simp
  also have ... = fri-query-idxs ! round-idx
    by (rule powers-scaled-hd)
  finally show ?thesis .
qed
moreover have
  fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0 =
    fri-query-idxs ! round-idx
  using roots-nonempty
  unfolding fri-evidence-layer-idx-def
  by (cases trace-roots) simp-all
ultimately show ?thesis by simp
qed
have hd-neq:
  opening-value (hd (trace-openings ! round-idx)) ≠ xp
  using no-head openings-nonempty hd-idx by blast
show trace-fri-header-tied-base-head-value-neq-gap s out
  unfolding trace-fri-header-tied-base-head-value-neq-gap-def
  by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,
    rule not-low, rule round-bound, rule trace-bs-nonempty,
    rule openings-nonempty, rule hd-idx, rule hd-neq, rule step,
    rule no-match)
qed
qed

lemma wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap:
  fixes G :: prob
  assumes neq-bound:
    wp-event verify-monad
    (trace-fri-header-tied-base-head-value-neq-gap s) s ≤ G
  shows

```

$wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-base-head-value-mismatch-gap } s) \ s \leq G$
proof –
have $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-base-head-value-mismatch-gap } s) \ s \leq$
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-base-head-value-neq-gap } s) \ s$
by $(rule \ wp\text{-event-mono})$
 $(rule \ trace\text{-fri-header-tied-base-head-value-mismatch-gap-imp-neq-gap})$
then show $?thesis$
by $(rule \ order\text{-trans}[OF - neq\text{-bound}])$
qed

lemma $trace\text{-fri-header-tied-assignment-imp-authenticated-or-auth-gap}$:
assumes $trace\text{-fri-header-tied-sampled-assignment-conflict } s \ out$
shows
 $trace\text{-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer}$
 $s \ out \vee$
 $trace\text{-fri-header-tied-assignment-auth-gap } s \ out$
using $assms$
unfolding $trace\text{-fri-header-tied-assignment-auth-gap-def}$
by $blast$

lemma $trace\text{-fri-header-tied-assignment-auth-gap-imp-same-layer-auth-gap}$:
assumes $trace\text{-fri-header-tied-assignment-auth-gap } s \ out$
shows $trace\text{-fri-header-tied-same-layer-auth-gap } s \ out$
proof –
from $assms$ **have** $conflict$:
 $trace\text{-fri-header-tied-sampled-assignment-conflict } s \ out$
and $no\text{-auth}$:
 $\neg \ trace\text{-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer}$
 $s \ out$
unfolding $trace\text{-fri-header-tied-assignment-auth-gap-def}$
by $simp\text{-all}$
from $conflict$ **obtain** $trace\text{-roots } trace\text{-bs } trace\text{-final } dg \ composition\text{-roots}$
 $composition\text{-bs } composition\text{-final } fri\text{-query-idxs } trace\text{-round-layers}$
 $composition\text{-round-layers } fr \ as \ rest \ trace\text{-openings } trace\text{-table}$
where $fri\text{-openings}$:
 $accepted\text{-fri-opening-transcript } s \ out \ trace\text{-roots } trace\text{-bs}$
 $trace\text{-final } dg \ composition\text{-roots } composition\text{-bs } composition\text{-final}$
 $fri\text{-query-idxs } trace\text{-round-layers } composition\text{-round-layers}$
and $header$:
 $verifier\text{-header-transcript } s \ fr \ trace\text{-roots } trace\text{-final } as \ dg$
 $composition\text{-roots } composition\text{-final } rest$
and $partial$:
 $accepted\text{-with-partial-trace-openings } s \ out \ fr \ fri\text{-query-idxs}$
 $trace\text{-openings}$
and $cand$: $partial\text{-trace-table-candidate } trace\text{-table } trace\text{-openings}$
and $not\text{-low}$: $\neg \ trace\text{-table-low-degree } trace\text{-table}$


```

and sampled:
  generic-fri-sampled-assignment-conflict trace-table trace-roots
  trace-bs trace-final fri-query-idxs trace-round-layers
unfolding trace-fri-header-tied-sampled-assignment-conflict-def
by blast
from fri-openings obtain result final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-fri-opening-transcript-def by blast
have split:
  generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
  trace-table trace-roots trace-bs trace-final fri-query-idxs
  trace-round-layers final-state  $\vee$ 
  (generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs
   fri-query-idxs trace-round-layers  $\wedge$ 
    $\neg$  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    trace-roots trace-bs fri-query-idxs trace-round-layers final-state)
by (rule
  generic-fri-sampled-assignment-conflict-imp-authenticated-or-same-layer-auth-gap
  [OF sampled])
then show ?thesis
proof
assume auth:
  generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
  trace-table trace-roots trace-bs trace-final fri-query-idxs
  trace-round-layers final-state
have
  trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
  s out
unfolding
  trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
by (intro exI conjI)
  (rule fri-openings, rule out-eq, rule header, rule partial, rule cand,
   rule not-low, rule auth)
then show ?thesis
using no-auth by contradiction
next
assume gap:
  generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs
  fri-query-idxs trace-round-layers  $\wedge$ 
   $\neg$  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    trace-roots trace-bs fri-query-idxs trace-round-layers final-state
show ?thesis
unfolding trace-fri-header-tied-same-layer-auth-gap-def
by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,
   rule not-low, use gap in simp-all)
qed
qed

```

lemma *trace-fri-header-tied-same-layer-auth-gap-imp-recorded-auth-gap:*
assumes *trace-fri-header-tied-same-layer-auth-gap s out*
shows *trace-fri-header-tied-same-layer-recorded-auth-gap s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table
result final-state
where *out-eq: out = Some (result, final-state)*
and *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *cand: partial-trace-table-candidate trace-table trace-openings*
and *not-low: ¬ trace-table-low-degree trace-table*
and *same:*
generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs
fri-query-idxs trace-round-layers
and *no-auth:*
¬ generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
trace-roots trace-bs fri-query-idxs trace-round-layers final-state
unfolding *trace-fri-header-tied-same-layer-auth-gap-def*
by *blast*
have *recorded-gap:*
generic-fri-sampled-same-layer-recorded-auth-gap trace-roots trace-bs
fri-query-idxs trace-round-layers final-state
by (rule *generic-fri-sampled-same-layer-auth-gap-imp-recorded-auth-gap*
[OF *same no-auth*])
show *?thesis*
unfolding *trace-fri-header-tied-same-layer-recorded-auth-gap-def*
by (*intro exI conjI*)
(*rule out-eq, rule fri-openings, rule header, rule partial, rule cand,*
rule not-low, rule recorded-gap)
qed

lemma *trace-fri-header-tied-same-layer-recorded-auth-gap-imp-authenticated-recorded-auth-gap:*
assumes *trace-fri-header-tied-same-layer-recorded-auth-gap s out*
shows
trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap s out
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table

```

  result final-state
where out-eq: out = Some (result, final-state)
and fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final dg composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers
and header:
  verifier-header-transcript s fr trace-roots trace-final as dg
  composition-roots composition-final rest
and partial:
  accepted-with-partial-trace-openings s out fr fri-query-idxs
  trace-openings
and cand: partial-trace-table-candidate trace-table trace-openings
and not-low:  $\neg$  trace-table-low-degree trace-table
and recorded-gap:
  generic-fri-sampled-same-layer-recorded-auth-gap trace-roots
  trace-bs fri-query-idxs trace-round-layers final-state
unfolding trace-fri-header-tied-same-layer-recorded-auth-gap-def
by blast
have len-trace-bs: length trace-bs = length trace-roots
  using accepted-fri-opening-transcript-shapes(2)[OF fri-openings] .
have len-query: length fri-query-idxs = rounds
  using accepted-fri-opening-transcript-shapes(7)[OF fri-openings] .
have auth-step:
   $\bigwedge$ round-idx layer-idx.
  round-idx < length fri-query-idxs  $\implies$ 
  layer-idx < length trace-bs  $\implies$ 
  generic-fri-transcript-step-with-authenticated-chunk trace-roots
  trace-bs fri-query-idxs trace-round-layers final-state round-idx
  layer-idx
proof –
  fix round-idx layer-idx
  assume round-bound: round-idx < length fri-query-idxs
  and layer-bound-bs: layer-idx < length trace-bs
  have round-bound-rounds: round-idx < rounds
  using round-bound len-query by simp
  have layer-bound-roots: layer-idx < length trace-roots
  using layer-bound-bs len-trace-bs by simp
  show
  generic-fri-transcript-step-with-authenticated-chunk trace-roots
  trace-bs fri-query-idxs trace-round-layers final-state round-idx
  layer-idx
  by (rule
    accepted-fri-opening-transcript-trace-step-with-authenticated-chunk-at-raw
    [OF fri-openings out-eq round-bound-rounds layer-bound-roots])
qed
have authenticated-recorded-gap:
  generic-fri-sampled-same-layer-authenticated-recorded-auth-gap
  trace-roots trace-bs fri-query-idxs trace-round-layers final-state

```

by (rule
 generic-fri-recorded-auth-gap-imp-authenticated-recorded-auth-gap
 [*OF recorded-gap auth-step*])
 show ?thesis
 unfolding
 trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-def
 by (intro exI conjI)
 (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,
 rule not-low, rule authenticated-recorded-gap)
 qed

lemma *trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap*:
 assumes
 trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap s out
 shows *trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s out*
 proof –
 from assms obtain *trace-roots trace-bs trace-final dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr as rest trace-openings trace-table
 result final-state
 where out-eq: *out = Some (result, final-state)*
 and fri-openings:
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final dg composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers
 and header:
 verifier-header-transcript s fr trace-roots trace-final as dg
 composition-roots composition-final rest
 and partial:
 accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings
 and cand: *partial-trace-table-candidate trace-table trace-openings*
 and not-low: $\neg$ *trace-table-low-degree trace-table*
 and gap:
 generic-fri-sampled-same-layer-authenticated-recorded-auth-gap
 trace-roots trace-bs fri-query-idxs trace-round-layers final-state
 unfolding
 trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-def
 by blast
 have slot-gap:
 generic-fri-authenticated-slot-recorded-chunk-gap trace-roots
 fri-query-idxs trace-round-layers final-state
 by (rule
 generic-fri-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap
 [*OF gap*])
 show ?thesis
 unfolding *trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-def*
 by (intro exI conjI)
 (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,

rule not-low, rule slot-gap)

qed

lemma *wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-authenticated-and-auth-gap:*
fixes $C\ G :: \text{prob}$
assumes *authenticated-bound:*
 wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
 $s \leq C$
and *auth-gap-bound:*
 wp-event verify-monad
 (trace-fri-header-tied-assignment-auth-gap s) s ≤ G
shows
 wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict s) s ≤ C + G

proof –
have *wp-event verify-monad*
 (trace-fri-header-tied-sampled-assignment-conflict s) s ≤
 wp-event verify-monad
 (λout.
 trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
 s out ∨
 trace-fri-header-tied-assignment-auth-gap s out) s
by *(rule wp-event-mono)*
 (rule trace-fri-header-tied-assignment-imp-authenticated-or-auth-gap)
also have $\dots \leq$
 wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
 $s +$
 wp-event verify-monad
 (trace-fri-header-tied-assignment-auth-gap s) s
by *(rule wp-event-union-bound)*
also have $\dots \leq C + G$
by *(intro add-mono authenticated-bound auth-gap-bound)*
finally show *?thesis .*

qed

lemma *wp-trace-fri-header-tied-assignment-auth-gap-bound-from-same-layer-auth-gap:*
fixes $G :: \text{prob}$
assumes *gap-bound:*
 wp-event verify-monad
 (trace-fri-header-tied-same-layer-auth-gap s) s ≤ G
shows
 wp-event verify-monad
 (trace-fri-header-tied-assignment-auth-gap s) s ≤ G

proof –
have *wp-event verify-monad*
 (trace-fri-header-tied-assignment-auth-gap s) s ≤
 wp-event verify-monad

```

      (trace-fri-header-tied-same-layer-auth-gap s) s
    by (rule wp-event-mono)
      (rule trace-fri-header-tied-assignment-auth-gap-imp-same-layer-auth-gap)
  then show ?thesis
    by (rule order-trans[OF - gap-bound])
qed

```

lemma *wp-trace-fri-header-tied-same-layer-auth-gap-bound-from-recorded-auth-gap:*
fixes $G :: \text{prob}$
assumes *gap-bound:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-same-layer-recorded-auth-gap } s) \ s \leq G$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-same-layer-auth-gap } s) \ s \leq G$
proof –
have $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-same-layer-auth-gap } s) \ s \leq$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-same-layer-recorded-auth-gap } s) \ s$
by (rule wp-event-mono)
 (rule trace-fri-header-tied-same-layer-auth-gap-imp-recorded-auth-gap)
then show ?thesis
 by (rule order-trans[OF - gap-bound])
qed

lemma *wp-trace-fri-header-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap:*
fixes $G :: \text{prob}$
assumes *gap-bound:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap } s)$
 $s \leq G$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-same-layer-recorded-auth-gap } s) \ s \leq G$
proof –
have $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-same-layer-recorded-auth-gap } s) \ s \leq$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap } s)$
 s
by (rule wp-event-mono)
 (rule
 trace-fri-header-tied-same-layer-recorded-auth-gap-imp-authenticated-recorded-auth-gap)
then show ?thesis
 by (rule order-trans[OF - gap-bound])
qed

lemma *wp-trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap:*

```

fixes  $G :: \text{prob}$ 
assumes  $\text{gap-bound}$ :
   $\text{wp-event verify-monad}$ 
     $(\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap } s) \ s$ 
     $\leq G$ 
shows
   $\text{wp-event verify-monad}$ 
     $(\text{trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap } s)$ 
     $s \leq G$ 
proof –
  have  $\text{wp-event verify-monad}$ 
     $(\text{trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap } s)$ 
     $s \leq$ 
     $\text{wp-event verify-monad}$ 
       $(\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap } s) \ s$ 
  by  $(\text{rule wp-event-mono})$ 
     $(\text{rule}$ 
       $\text{trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap})$ 
  then show  $?thesis$ 
    by  $(\text{rule order-trans}[OF - \text{gap-bound}])$ 
qed

lemma  $\text{wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-auth-gap-and-residuals}$ :
fixes  $A \ P \ H \ G \ N \ S \ F :: \text{prob}$ 
assumes  $\text{auth-gap-bound}$ :
   $\text{wp-event verify-monad}$ 
     $(\text{trace-fri-header-tied-assignment-auth-gap } s) \ s \leq A$ 
and  $\text{partial-merkle-bound}$ :
   $\text{wp-event verify-monad}$   $(\text{partial-merkle-inconsistency-bad } s) \ s \leq P$ 
and  $\text{structural-gap-bound}$ :
   $\text{wp-event verify-monad}$ 
     $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq H$ 
and  $\text{alignment-gap-bound}$ :
   $\text{wp-event verify-monad}$ 
     $(\text{trace-fri-header-tied-base-value-alignment-gap } s) \ s \leq G$ 
and  $\text{next-gap-bound}$ :
   $\text{wp-event verify-monad}$ 
     $(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq N$ 
and  $\text{successor-gap-bound}$ :
   $\text{wp-event verify-monad}$ 
     $(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq S$ 
and  $\text{final-gap-bound}$ :
   $\text{wp-event verify-monad}$ 
     $(\text{trace-fri-header-tied-final-value-replay-gap } s) \ s \leq F$ 
shows
   $\text{wp-event verify-monad}$ 
     $(\text{trace-fri-header-tied-sampled-assignment-conflict } s) \ s$ 
     $\leq (((((P + H) + G) + N + S + F) + P) + A$ 
by  $(\text{rule}$ 

```

wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-authenticated-and-auth-gap)
 (rule
wp-trace-fri-header-tied-authenticated-conflict-bound-from-partial-merkle-structural-and-replay-gaps
 [OF *partial-merkle-bound structural-gap-bound alignment-gap-bound*
next-gap-bound successor-gap-bound final-gap-bound],
 rule *auth-gap-bound*)

lemma *wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-same-layer-auth-gap-and-residuals:*
fixes *A P H G N S F :: prob*

assumes *same-layer-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-same-layer-auth-gap s*) $s \leq A$

and *partial-merkle-bound:*

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq P$

and *structural-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-sibling-mismatch-structural-gap s*) $s \leq H$

and *alignment-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-base-value-alignment-gap s*) $s \leq G$

and *next-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-next-value-replay-gap s*) $s \leq N$

and *successor-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-successor-replay-gap s*) $s \leq S$

and *final-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-final-value-replay-gap s*) $s \leq F$

shows

wp-event verify-monad

(*trace-fri-header-tied-sampled-assignment-conflict s*) s
 $\leq (((((P + H) + G) + N + S + F) + P) + A$

by (rule

wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-auth-gap-and-residuals)

(rule

wp-trace-fri-header-tied-assignment-auth-gap-bound-from-same-layer-auth-gap

[OF *same-layer-gap-bound*],

rule *partial-merkle-bound*,

rule *structural-gap-bound*,

rule *alignment-gap-bound*,

rule *next-gap-bound*,

rule *successor-gap-bound*,

rule *final-gap-bound*)

lemma *wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-recorded-auth-gap-and-residuals:*

fixes *A P H G N S F :: prob*

assumes *recorded-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-same-layer-recorded-auth-gap s) $s \leq A$
and partial-merkle-bound:
 wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$
and structural-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap s) $s \leq H$
and alignment-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-base-value-alignment-gap s) $s \leq G$
and next-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap s) $s \leq N$
and successor-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap s) $s \leq S$
and final-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap s) $s \leq F$
shows
 wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict s) s
 $\leq (((((P + H) + G) + N + S + F) + P) + A$
by (rule
 wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-same-layer-auth-gap-and-residuals)
 (rule
 wp-trace-fri-header-tied-same-layer-auth-gap-bound-from-recorded-auth-gap
 [OF recorded-gap-bound],
 rule partial-merkle-bound,
 rule structural-gap-bound,
 rule alignment-gap-bound,
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound)

lemma wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-authenticated-recorded-auth-gap-and-residuals
fixes $A P H G N S F :: \text{prob}$

assumes gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap s)
 $s \leq A$
and partial-merkle-bound:
 wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$
and structural-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap s) $s \leq H$
and alignment-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-base-value-alignment-gap s) $s \leq G$
and next-gap-bound:

wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap s) $s \leq N$
and *successor-gap-bound*:
wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap s) $s \leq S$
and *final-gap-bound*:
wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap s) $s \leq F$
shows
wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict s) s
 $\leq (((P + H) + G) + N + S + F) + P + A$
by (rule
 wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-recorded-auth-gap-and-residuals)
 (rule
 wp-trace-fri-header-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap
 [OF gap-bound],
 rule partial-merkle-bound,
 rule structural-gap-bound,
 rule alignment-gap-bound,
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound)

lemma wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-slot-recorded-chunk-gap-and-residuals:

fixes $A P H G N S F :: \text{prob}$

assumes gap-bound:

wp-event verify-monad
 (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s) s
 $\leq A$
and *partial-merkle-bound*:
wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$
and *structural-gap-bound*:
wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap s) $s \leq H$
and *alignment-gap-bound*:
wp-event verify-monad
 (trace-fri-header-tied-base-value-alignment-gap s) $s \leq G$
and *next-gap-bound*:
wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap s) $s \leq N$
and *successor-gap-bound*:
wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap s) $s \leq S$
and *final-gap-bound*:
wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap s) $s \leq F$

shows

wp-event verify-monad

$(\text{trace-fri-header-tied-sampled-assignment-conflict } s) \ s$
 $\leq (((((P + H) + G) + N + S + F) + P) + A$
by (rule
 $\text{wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-authenticated-recorded-auth-gap-and-residuals}$)
(rule
 $\text{wp-trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap}$
 $[OF \text{ gap-bound}]$,
rule $\text{partial-merkle-bound}$,
rule $\text{structural-gap-bound}$,
rule $\text{alignment-gap-bound}$,
rule next-gap-bound ,
rule $\text{successor-gap-bound}$,
rule final-gap-bound)

lemma $\text{wp-trace-fri-header-tied-bound-from-sampled-zero-auth-gap-and-residuals}$:

fixes $R \ Z \ A \ P \ H \ G \ N \ S \ F :: \text{prob}$

assumes sampled-bound :

$\text{wp-event verify-monad}$

$(\text{trace-fri-bad-with-header-tied-sampled-layer-chain } s) \ s \leq R$

and zero-bound :

$\text{wp-event verify-monad}$

$(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq Z$

and auth-gap-bound :

$\text{wp-event verify-monad}$

$(\text{trace-fri-header-tied-assignment-auth-gap } s) \ s \leq A$

and $\text{partial-merkle-bound}$:

$\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq P$

and $\text{structural-gap-bound}$:

$\text{wp-event verify-monad}$

$(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq H$

and $\text{alignment-gap-bound}$:

$\text{wp-event verify-monad}$

$(\text{trace-fri-header-tied-base-value-alignment-gap } s) \ s \leq G$

and next-gap-bound :

$\text{wp-event verify-monad}$

$(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq N$

and $\text{successor-gap-bound}$:

$\text{wp-event verify-monad}$

$(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq S$

and final-gap-bound :

$\text{wp-event verify-monad}$

$(\text{trace-fri-header-tied-final-value-replay-gap } s) \ s \leq F$

shows

$\text{wp-event verify-monad}$

$(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s$

$\leq R + (((((P + H) + G) + N + S + F) + P) + A + Z)$

proof –

have conflict-bound :

$\text{wp-event verify-monad}$

$(\text{trace-fri-header-tied-sampled-assignment-conflict } s) \ s$
 $\leq (((((P + H) + G) + N + S + F) + P) + A$
by (rule
 $\text{wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-auth-gap-and-residuals})$
(rule *auth-gap-bound*,
rule *partial-merkle-bound*,
rule *structural-gap-bound*,
rule *alignment-gap-bound*,
rule *next-gap-bound*,
rule *successor-gap-bound*,
rule *final-gap-bound*)
have *wp-event verify-monad*
 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s$
 $\leq R + (((((P + H) + G) + N + S + F) + P) + A + Z)$
by (rule *order-trans*[
 $OF \ \text{wp-trace-fri-header-tied-bound-from-sampled-conflict-and-zero}$
 $[OF \ \text{sampled-bound} \ \text{conflict-bound} \ \text{zero-bound}]])$
 $(\text{simp add: algebra-simps})$
then show *?thesis* .
qed

lemma *wp-trace-fri-header-tied-bound-from-sampled-zero-same-layer-auth-gap-and-residuals:*

fixes $R \ Z \ A \ P \ H \ G \ N \ S \ F :: \text{prob}$

assumes *sampled-bound:*

wp-event verify-monad

$(\text{trace-fri-bad-with-header-tied-sampled-layer-chain } s) \ s \leq R$

and *zero-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq Z$

and *same-layer-gap-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-same-layer-auth-gap } s) \ s \leq A$

and *partial-merkle-bound:*

wp-event verify-monad $(\text{partial-merkle-inconsistency-bad } s) \ s \leq P$

and *structural-gap-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq H$

and *alignment-gap-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-base-value-alignment-gap } s) \ s \leq G$

and *next-gap-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq N$

and *successor-gap-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq S$

and *final-gap-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-final-value-replay-gap } s) \ s \leq F$

shows

wp-event verify-monad
 (*trace-fri-bad-with-header-tied-partial-candidate s*) s
 $\leq R + (((((P + H) + G) + N + S + F) + P) + A + Z)$
by (*rule*
wp-trace-fri-header-tied-bound-from-sampled-zero-auth-gap-and-residuals)
 (*rule sampled-bound*,
rule zero-bound,
rule
wp-trace-fri-header-tied-assignment-auth-gap-bound-from-same-layer-auth-gap
[OF same-layer-gap-bound],
rule partial-merkle-bound,
rule structural-gap-bound,
rule alignment-gap-bound,
rule next-gap-bound,
rule successor-gap-bound,
rule final-gap-bound)

lemma *wp-trace-fri-header-tied-bound-from-sampled-zero-recorded-auth-gap-and-residuals*:

fixes $R\ Z\ A\ P\ H\ G\ N\ S\ F :: \text{prob}$

assumes *sampled-bound*:

wp-event verify-monad
 (*trace-fri-bad-with-header-tied-sampled-layer-chain s*) $s \leq R$

and *zero-bound*:

wp-event verify-monad
 (*trace-fri-header-tied-zero-round-final-obstruction s*) $s \leq Z$

and *recorded-gap-bound*:

wp-event verify-monad
 (*trace-fri-header-tied-same-layer-recorded-auth-gap s*) $s \leq A$

and *partial-merkle-bound*:

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq P$

and *structural-gap-bound*:

wp-event verify-monad
 (*trace-fri-header-tied-sibling-mismatch-structural-gap s*) $s \leq H$

and *alignment-gap-bound*:

wp-event verify-monad
 (*trace-fri-header-tied-base-value-alignment-gap s*) $s \leq G$

and *next-gap-bound*:

wp-event verify-monad
 (*trace-fri-header-tied-next-value-replay-gap s*) $s \leq N$

and *successor-gap-bound*:

wp-event verify-monad
 (*trace-fri-header-tied-successor-replay-gap s*) $s \leq S$

and *final-gap-bound*:

wp-event verify-monad
 (*trace-fri-header-tied-final-value-replay-gap s*) $s \leq F$

shows

wp-event verify-monad
 (*trace-fri-bad-with-header-tied-partial-candidate s*) s

$\leq R + (((((P + H) + G) + N + S + F) + P) + A + Z)$
by (rule
wp-trace-fri-header-tied-bound-from-sampled-zero-same-layer-auth-gap-and-residuals)
(rule *sampled-bound*,
rule *zero-bound*,
rule
wp-trace-fri-header-tied-same-layer-auth-gap-bound-from-recorded-auth-gap
[*OF recorded-gap-bound*],
rule *partial-merkle-bound*,
rule *structural-gap-bound*,
rule *alignment-gap-bound*,
rule *next-gap-bound*,
rule *successor-gap-bound*,
rule *final-gap-bound*)

lemma *wp-trace-fri-header-tied-bound-from-sampled-zero-authenticated-recorded-auth-gap-and-residuals*:

fixes $R\ Z\ A\ P\ H\ G\ N\ S\ F :: \text{prob}$

assumes *sampled-bound*:

wp-event verify-monad

(*trace-fri-bad-with-header-tied-sampled-layer-chain s*) $s \leq R$

and *zero-bound*:

wp-event verify-monad

(*trace-fri-header-tied-zero-round-final-obstruction s*) $s \leq Z$

and *gap-bound*:

wp-event verify-monad

(*trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap s*)
 $s \leq A$

and *partial-merkle-bound*:

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq P$

and *structural-gap-bound*:

wp-event verify-monad

(*trace-fri-header-tied-sibling-mismatch-structural-gap s*) $s \leq H$

and *alignment-gap-bound*:

wp-event verify-monad

(*trace-fri-header-tied-base-value-alignment-gap s*) $s \leq G$

and *next-gap-bound*:

wp-event verify-monad

(*trace-fri-header-tied-next-value-replay-gap s*) $s \leq N$

and *successor-gap-bound*:

wp-event verify-monad

(*trace-fri-header-tied-successor-replay-gap s*) $s \leq S$

and *final-gap-bound*:

wp-event verify-monad

(*trace-fri-header-tied-final-value-replay-gap s*) $s \leq F$

shows

wp-event verify-monad

(*trace-fri-bad-with-header-tied-partial-candidate s*) s

$\leq R + (((((P + H) + G) + N + S + F) + P) + A + Z)$

by (rule

$wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}bound\text{-}from\text{-}sampled\text{-}zero\text{-}recorded\text{-}auth\text{-}gap\text{-}and\text{-}residuals)$
(rule sampled-bound,
rule zero-bound,
rule
 $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}same\text{-}layer\text{-}recorded\text{-}auth\text{-}gap\text{-}bound\text{-}from\text{-}authenticated\text{-}recorded\text{-}auth\text{-}gap$
[OF gap-bound],
rule partial-merkle-bound,
rule structural-gap-bound,
rule alignment-gap-bound,
rule next-gap-bound,
rule successor-gap-bound,
rule final-gap-bound)

lemma $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}bound\text{-}from\text{-}sampled\text{-}zero\text{-}slot\text{-}recorded\text{-}chunk\text{-}gap\text{-}and\text{-}residuals$:

fixes $R\ Z\ A\ P\ H\ G\ N\ S\ F :: prob$

assumes sampled-bound:

$wp\text{-}event\ verify\text{-}monad$

($trace\text{-}fri\text{-}bad\text{-}with\text{-}header\text{-}tied\text{-}sampled\text{-}layer\text{-}chain\ s$) $s \leq R$

and zero-bound:

$wp\text{-}event\ verify\text{-}monad$

($trace\text{-}fri\text{-}header\text{-}tied\text{-}zero\text{-}round\text{-}final\text{-}obstruction\ s$) $s \leq Z$

and gap-bound:

$wp\text{-}event\ verify\text{-}monad$

($trace\text{-}fri\text{-}header\text{-}tied\text{-}authenticated\text{-}slot\text{-}recorded\text{-}chunk\text{-}gap\ s$)
 $s \leq A$

and partial-merkle-bound:

$wp\text{-}event\ verify\text{-}monad$ ($partial\text{-}merkle\text{-}inconsistency\text{-}bad\ s$) $s \leq P$

and structural-gap-bound:

$wp\text{-}event\ verify\text{-}monad$

($trace\text{-}fri\text{-}header\text{-}tied\text{-}sibling\text{-}mismatch\text{-}structural\text{-}gap\ s$) $s \leq H$

and alignment-gap-bound:

$wp\text{-}event\ verify\text{-}monad$

($trace\text{-}fri\text{-}header\text{-}tied\text{-}base\text{-}value\text{-}alignment\text{-}gap\ s$) $s \leq G$

and next-gap-bound:

$wp\text{-}event\ verify\text{-}monad$

($trace\text{-}fri\text{-}header\text{-}tied\text{-}next\text{-}value\text{-}replay\text{-}gap\ s$) $s \leq N$

and successor-gap-bound:

$wp\text{-}event\ verify\text{-}monad$

($trace\text{-}fri\text{-}header\text{-}tied\text{-}successor\text{-}replay\text{-}gap\ s$) $s \leq S$

and final-gap-bound:

$wp\text{-}event\ verify\text{-}monad$

($trace\text{-}fri\text{-}header\text{-}tied\text{-}final\text{-}value\text{-}replay\text{-}gap\ s$) $s \leq F$

shows

$wp\text{-}event\ verify\text{-}monad$

($trace\text{-}fri\text{-}bad\text{-}with\text{-}header\text{-}tied\text{-}partial\text{-}candidate\ s$) s

$\leq R + (((((P + H) + G) + N + S + F) + P) + A + Z)$

by (rule

$wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}bound\text{-}from\text{-}sampled\text{-}zero\text{-}authenticated\text{-}recorded\text{-}auth\text{-}gap\text{-}and\text{-}residuals)$

(rule sampled-bound,

rule zero-bound,
rule
wp-trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap
[OF gap-bound],
rule partial-merkle-bound,
rule structural-gap-bound,
rule alignment-gap-bound,
rule next-gap-bound,
rule successor-gap-bound,
rule final-gap-bound)

lemma *wp-trace-fri-header-tied-bound-from-sampled-zero-slot-and-authenticated-final-residuals:*

fixes $R\ Z\ A\ P\ H\ G\ N\ S\ F :: \text{prob}$

assumes *sampled-bound:*

wp-event verify-monad

(trace-fri-bad-with-header-tied-sampled-layer-chain s) s ≤ R

and *zero-bound:*

wp-event verify-monad

(trace-fri-header-tied-zero-round-final-obstruction s) s ≤ Z

and *slot-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s)
 $s \leq A$

and *partial-merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ P

and *structural-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-sibling-mismatch-structural-gap s) s ≤ H

and *alignment-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-base-value-alignment-gap s) s ≤ G

and *next-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-next-value-replay-gap s) s ≤ N

and *successor-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-successor-replay-gap s) s ≤ S

and *authenticated-final-bound:*

wp-event verify-monad

(trace-fri-header-tied-final-value-authenticated-replay-gap s) s
 $\leq F$

shows

wp-event verify-monad

(trace-fri-bad-with-header-tied-partial-candidate s) s

$\leq R + (((((P + H) + G) + N + S + (P + A + F)) + P) + A + Z)$

proof –

have *final-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-final-value-replay-gap s) s ≤ P + A + F

by (rule
 wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-authenticated)
 (rule partial-merkle-bound, rule slot-gap-bound,
 rule authenticated-final-bound)
 have old-bound:
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-partial-candidate s) s
 $\leq R + (((((P + H) + G) + N + S + (P + A + F)) + P) + A + Z)$
 by (rule
 wp-trace-fri-header-tied-bound-from-sampled-zero-slot-recorded-chunk-gap-and-residuals)
 (rule sampled-bound,
 rule zero-bound,
 rule slot-gap-bound,
 rule partial-merkle-bound,
 rule structural-gap-bound,
 rule alignment-gap-bound,
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound)
 then show ?thesis .
 qed

lemma wp-trace-fri-header-tied-bound-from-sampled-zero-slot-and-selected-step-final-residuals:
fixes R Z A P H G N S F :: prob
assumes sampled-bound:
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-sampled-layer-chain s) s $\leq R$
and zero-bound:
 wp-event verify-monad
 (trace-fri-header-tied-zero-round-final-obstruction s) s $\leq Z$
and slot-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s)
 s $\leq A$
and partial-merkle-bound:
 wp-event verify-monad (partial-merkle-inconsistency-bad s) s $\leq P$
and structural-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap s) s $\leq H$
and alignment-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-base-value-alignment-gap s) s $\leq G$
and next-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap s) s $\leq N$
and successor-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap s) s $\leq S$
and selected-step-bound:

wp-event verify-monad
 (trace-fri-header-tied-final-value-selected-step-missing-gap s) s
 $\leq F$
shows
wp-event verify-monad
 (trace-fri-bad-with-header-tied-partial-candidate s) s
 $\leq R + (((((P + H) + G) + N + S + (P + A + F)) + P) + A + Z)$
proof –
have *final-gap-bound*:
wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap s) $s \leq P + A + F$
by (rule
 wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
 (rule *partial-merkle-bound*, rule *slot-gap-bound*,
 rule *selected-step-bound*)
show *?thesis*
by (rule
 wp-trace-fri-header-tied-bound-from-sampled-zero-slot-recorded-chunk-gap-and-residuals)
 (rule *sampled-bound*,
 rule *zero-bound*,
 rule *slot-gap-bound*,
 rule *partial-merkle-bound*,
 rule *structural-gap-bound*,
 rule *alignment-gap-bound*,
 rule *next-gap-bound*,
 rule *successor-gap-bound*,
 rule *final-gap-bound*)

qed

lemma *trace-fri-reachable-partial-candidate-reduction-from-header-tied-residual-bounds*:

fixes $R\ Z\ A\ P\ H\ G\ N\ S\ F\ T :: \text{prob}$

assumes *sampled-bound*:

wp-event verify-monad
 (trace-fri-bad-with-header-tied-sampled-layer-chain s) $s \leq R$

and *zero-bound*:

wp-event verify-monad
 (trace-fri-header-tied-zero-round-final-obstruction s) $s \leq Z$

and *slot-gap-bound*:

wp-event verify-monad
 (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s)
 $s \leq A$

and *partial-merkle-bound*:

wp-event verify-monad (*partial-merkle-inconsistency-bad* s) $s \leq P$

and *structural-gap-bound*:

wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap s) $s \leq H$

and *alignment-gap-bound*:

wp-event verify-monad
 (trace-fri-header-tied-base-value-alignment-gap s) $s \leq G$

and *next-gap-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq N$
and *successor-gap-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq S$
and *final-gap-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-final-value-replay-gap } s) \ s \leq F$
and *header-gap-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-reachable-header-tie-gap } s) \ s \leq T$
and *total-bound*:
 $R + (((((P + H) + G) + N + S + F) + P) + A + Z) + T$
 $\leq \text{trace-fri-error}$
shows *trace-fri-reachable-partial-candidate-reduction-assumption* s
proof –
have *header-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s$
 $\leq R + (((((P + H) + G) + N + S + F) + P) + A + Z)$
by (rule
 $wp\text{-trace-fri-header-tied-bound-from-sampled-zero-slot-recorded-chunk-gap-and-residuals}$
 $[OF \text{ sampled-bound zero-bound slot-gap-bound partial-merkle-bound}$
 $\text{ structural-gap-bound alignment-gap-bound next-gap-bound}$
 $\text{ successor-gap-bound final-gap-bound}]$)
show *?thesis*
by (rule
 $\text{trace-fri-reachable-partial-candidate-reduction-from-header-tied-and-gap}$
 $[OF \text{ header-bound header-gap-bound total-bound}]$)
qed

lemma *trace-fri-header-tied-reduction-from-residual-bounds*:
fixes $R \ Z \ A \ P \ H \ G \ N \ S \ F :: \text{prob}$
assumes *sampled-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-sampled-layer-chain } s) \ s \leq R$
and *zero-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq Z$
and *slot-gap-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap } s)$
 $s \leq A$
and *partial-merkle-bound*:
 $wp\text{-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq P$
and *structural-gap-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq H$

```

and alignment-gap-bound:
  wp-event verify-monad
    (trace-fri-header-tied-base-value-alignment-gap s)  $s \leq G$ 
and next-gap-bound:
  wp-event verify-monad
    (trace-fri-header-tied-next-value-replay-gap s)  $s \leq N$ 
and successor-gap-bound:
  wp-event verify-monad
    (trace-fri-header-tied-successor-replay-gap s)  $s \leq S$ 
and final-gap-bound:
  wp-event verify-monad
    (trace-fri-header-tied-final-value-replay-gap s)  $s \leq F$ 
and total-bound:
   $R + (((((P + H) + G) + N + S + F) + P) + A + Z)$ 
     $\leq \text{trace-fri-error}$ 
shows trace-fri-header-tied-reduction s
unfolding trace-fri-header-tied-reduction-def
by (rule order-trans[
  OF wp-trace-fri-header-tied-bound-from-sampled-zero-slot-recorded-chunk-gap-and-residuals
  [OF sampled-bound zero-bound slot-gap-bound partial-merkle-bound
    structural-gap-bound alignment-gap-bound next-gap-bound
    successor-gap-bound final-gap-bound] total-bound])

end

end

theory Soundness-FRI-Query-Openings
imports
  Soundness-FRI
  Soundness-Query-Transcript-Openings
begin

Bridge from the FRI opening transcript evidence to the authenticated trace openings
for the exact verifier-derived query-index list.

context soundness
begin

lemma accepted-fri-opening-transcript-trace-openings-for-query-indices:
assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final dg composition-roots composition-bs composition-final
  query-idxs trace-round-layers composition-round-layers
and header:
  verifier-header-transcript s fr trace-roots trace-final as dg
  composition-roots composition-final rest
and out-eq: out = Some (result, final-state)
obtains trace-openings where

```

```

    accepted-with-partial-trace-openings s out fr query-idxs trace-openings
proof –
  let ?P =
     $\lambda i$  openings. map opening-index openings =
      powers-scaled (query-idxs ! i)  $\wedge$ 
      partial-authenticated-table fr (scale * clength) openings final-state
  have len-query-idxs: length query-idxs = rounds
    by (rule accepted-fri-opening-transcript-shapes(7)[OF fri-openings])
  from fri-openings obtain result' final-state' fr' f-fl header-as fl
    query-state raw-idxs query-chunks
  where out-def: out = Some (result', final-state')
  and trace-roots-eq: trace-roots = map snd f-fl
  and composition-roots-eq: composition-roots = map snd fl
  and header':
    verifier-header-transcript s fr' trace-roots trace-final header-as dg
    composition-roots composition-final (PTranscript query-state)
  and query-out:
    Some (result', final-state')  $\in$ 
      set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr' f-fl trace-final header-as fl
            composition-final)
          rounds)
        query-state)
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
  and len-query-chunks: length query-chunks = rounds
  and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state'
  and query-chunk:
     $\bigwedge i. i < \text{rounds} \implies$ 
      verifier-query-round-chunk (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)
  and replay-lookup:
     $\bigwedge i. i < \text{rounds} \implies$ 
      fmlookup (HashMap final-state')
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
  unfolding accepted-fri-opening-transcript-def
  by blast
  have final-state-eq: final-state' = final-state
    using out-def out-eq by simp
  have fr-eq: fr' = fr
    using verifier-header-transcript-unique[OF header' header] by simp

```

```

have ex-round:  $\bigwedge i. i < \text{rounds} \implies \exists \text{openings}. ?P\ i\ \text{openings}$ 
proof -
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  show  $\exists \text{openings}. ?P\ i\ \text{openings}$ 
  proof (rule
    ntimes-verifier-query-rounds-selected-trace-openings-aligned-with-replay-at
    [OF query-out i-bound len-raw query-idxs-eq len-query-chunks])
  show PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state'
  by (rule transcript-query)
  show  $\bigwedge j. j < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! j)
    (map snd f-fl) (map snd fl) (query-chunks ! j)
  using query-chunk trace-roots-eq composition-roots-eq by simp
  show  $\bigwedge j. j < \text{rounds} \implies$ 
    fmlookup (HashMap final-state')
    (QueryIndexChallenge (PQueryCounter query-state + j)
    (state-after-query-chunks
    (PState query-state) query-chunks j)) =
    Some (raw-idxs ! j)
  using replay-lookup by simp
next
  fix openings
  assume
    map opening-index openings = powers-scaled (query-idxs ! i)
    partial-authenticated-table fr' (scale * clength) openings
    final-state'
  then show ?thesis
    using fr-eq final-state-eq by blast
qed
qed
let ?w =  $\lambda i. \text{SOME openings}. ?P\ i\ \text{openings}$ 
let ?trace-openings = map ?w [0.. $\text{rounds}$ ]
have len-trace: length ?trace-openings = rounds
  by simp
have props:  $\bigwedge i. i < \text{rounds} \implies ?P\ i\ (?trace-openings\ !\ i)$ 
proof -
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  have ?P i (?w i)
    by (rule someI-ex[OF ex-round[OF i-bound]])
  then show ?P i (?trace-openings ! i)
    using i-bound by simp
qed
have partial:
  accepted-with-partial-trace-openings s out fr query-idxs
  ?trace-openings
  unfolding accepted-with-partial-trace-openings-def accepted-def

```

```

    by (intro conjI exI[of - result] exI[of - final-state])
      (use out-eq len-query-idxs len-trace props in auto)
  show ?thesis
    by (rule that[OF partial])
qed

end

end

```

```

theory Soundness-FRI-Trace-Head-Value
imports
  Soundness-FRI-Trace-Residuals
  Soundness-FRI-Query-Openings
begin

```

Local replay facts for the trace base head-value residual. This theory keeps the exact selected trace-layer replay needed by the residual proof out of the already large trace residual layer.

```

context soundness
begin

```

lemma *verifier-query-round-program-trace-fri-head-layer-value-aligned-with-lookup:*

```

fixes s t :: ('f, 'a) protocol-channel-scheme
assumes f-fl-eq: f-fl = (b, rt) # f-fl-tail
and outcome:
  Some (( ), t) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
obtains raw idx openings query-chunk trace-layer-chunks
  composition-layer-chunks xp xp-path xn xn-path where
  idx = index (to-nat raw)
  query-round-fri-layer-transcripts idx (map snd f-fl) (map snd fl)
  query-chunk trace-layer-chunks composition-layer-chunks
  PTranscript s = query-chunk @ PTranscript t
  PState t = foldl concat (PState s) query-chunk
  s ≤ t
  map opening-index openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) openings t
  openings ≠ []
  opening-value (hd openings) = xp
  fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
    (trace-layer-chunks ! 0)
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof –
from outcome[unfolded verifier-query-round-program-alt-def]

```

```

obtain raw s0 where raw:
  Some (raw, s0) ∈
    set-dist (execute receive-query-index-challenge s)
and tail:
  Some (( ), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s0)
by (auto elim!: set-dist-bindE)
obtain idx openings query-chunk trace-layer-chunks composition-layer-chunks
  xp xp-path xn xn-path
where idx-eq: idx = index (to-nat raw)
and layers:
  query-round-fri-layer-transcripts idx (map snd f-fl) (map snd fl)
  query-chunk trace-layer-chunks composition-layer-chunks
and tr-t: PTranscript s0 = query-chunk @ PTranscript t
and st-t: PState t = foldl concat (PState s0) query-chunk
and ext-s0-t: s0 ≤ t
and idxs: map opening-index openings = powers-scaled idx
and auth-table:
  partial-authenticated-table fr (scale * clength) openings t
and nonempty: openings ≠ []
and opening-value-eq: opening-value (hd openings) = xp
and head-chunk:
  fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
  (trace-layer-chunks ! 0)
using verifier-query-round-after-index-trace-fri-head-layer-value-aligned
  [OF f-fl-eq tail]
by metis
have lookup-s0:
  fmlookup (HashMap s0)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
using receive-query-index-challenge-outcome[OF raw] by simp
have lookup-t:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
by (rule hash-extension-lookup[OF lookup-s0 ext-s0-t])
have tr-s0: PTranscript s = PTranscript s0
using receive-query-index-challenge-outcome[OF raw] by simp
have st-s0: PState s0 = PState s
using receive-query-index-challenge-outcome[OF raw] by simp
have ext-s-s0: s ≤ s0
using receive-query-index-challenge-outcome[OF raw] by simp
have ext-s-t: s ≤ t
by (rule hash-ext-trans[OF ext-s-s0 ext-s0-t])
have tr-goal: PTranscript s = query-chunk @ PTranscript t
using tr-t tr-s0 by simp
have st-goal: PState t = foldl concat (PState s) query-chunk

```



```

using st-t st-s0 by simp
show ?thesis
by (rule that[OF idx-eq layers tr-goal st-goal ext-s-t idxs auth-table
      nonempty opening-value-eq head-chunk lookup-t])
qed

lemma ntimes-verifier-query-rounds-selected-trace-fri-head-layer-value-aligned-with-replay-at:
fixes query-state final-state :: ('f, 'a) protocol-channel-scheme
assumes f-fl-eq: f-fl = (b, rt) # f-fl-tail
and outcome:
  Some (results, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
        query-state)
and i-bound: i < rounds
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and query-chunk:
   $\bigwedge i. i < rounds \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
and query-layers:
  query-round-fri-layer-transcripts (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
    trace-layer-chunks composition-layer-chunks
and replay-lookup:
   $\bigwedge i. i < rounds \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
obtains openings xp xp-path xn xn-path where
  map opening-index openings = powers-scaled (query-idxs ! i)
  partial-authenticated-table fr (scale * clength) openings final-state
  openings ≠ []
  opening-value (hd openings) = xp
  fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
    (trace-layer-chunks ! 0)
proof –
  let ?round = verifier-query-round-program fr f-fl f-final as fl final

```

```

from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist (execute (ntimes ?round i) query-state)
and round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
    Some (suffix, final-state) ∈
      set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
by blast
have x-eq: x = ()
  by (cases x) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
  fix prefix-query-idxs prefix-chunks
  assume len-prefix-idxs: length prefix-query-idxs = i
    and len-prefix-chunks: length prefix-chunks = i
    and transcript-prefix:
      PTranscript query-state =
        List.concat prefix-chunks @ PTranscript s-i
  and state-i:
    PState s-i =
      state-after-query-chunks (PState query-state) prefix-chunks i
  and counter-i:
    PQueryCounter s-i = PQueryCounter query-state + i
  and prefix-ext: query-state ≤ s-i
  and prefix-rounds:
     $\bigwedge k. k < i \implies$ 
      verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
  show ?thesis
proof (rule
  verifier-query-round-program-trace-fri-head-layer-value-aligned-with-lookup
  [OF f-fl-eq round[unfolded x-eq]])
  fix raw idx chunk actual-trace-layers actual-composition-layers
    openings xp xp-path xn xn-path
  assume idx-eq: idx = index (to-nat raw)
    and actual-layers:
      query-round-fri-layer-transcripts idx (map snd f-fl)
      (map snd fl) chunk actual-trace-layers actual-composition-layers
  and transcript-round: PTranscript s-i = chunk @ PTranscript s-suc
  and state-suc: PState s-suc = foldl concat (PState s-i) chunk
  and round-ext: s-i ≤ s-suc
  and opening-indices: map opening-index openings = powers-scaled idx
  and auth-table-suc:
    partial-authenticated-table fr (scale * clength) openings s-suc
  and openings-nonempty: openings ≠ []

```

```

and opening-value-eq: opening-value (hd openings) = xp
and head-chunk:
  fri-layer-opening-chunk (scale * clength) xp xp-path xn
  xn-path (actual-trace-layers ! 0)
and lookup-suc:
  fmlookup (HashMap s-suc)
  (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
  Some raw
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs:
  length suffix-query-idxs = rounds - Suc i
and len-suffix-chunks:
  length suffix-chunks = rounds - Suc i
and transcript-suffix:
  PTranscript s-suc =
  List.concat suffix-chunks @ PTranscript final-state
and suffix-state:
  PState final-state =
  state-after-query-chunks (PState s-suc) suffix-chunks
  (rounds - Suc i)
and suffix-counter:
  PQueryCounter final-state =
  PQueryCounter s-suc + (rounds - Suc i)
and suffix-ext: s-suc ≤ final-state
and suffix-rounds:
   $\bigwedge k. k < rounds - Suc i \implies$ 
  verifier-query-round-chunk (suffix-query-idxs ! k)
  (map snd f-fl) (map snd fl) (suffix-chunks ! k)
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
  using len-prefix-chunks len-suffix-chunks i-bound by simp
have selected-chunk-lens:
   $\bigwedge k. k < rounds \implies$ 
  length (?selected-chunks ! k) =
  verifier-query-round-transcript-length (query-idxs ! k)
  (map snd f-fl) (map snd fl)
proof -
fix k
assume k-bound: k < rounds
show
  length (?selected-chunks ! k) =
  verifier-query-round-transcript-length (query-idxs ! k)
  (map snd f-fl) (map snd fl)
proof (cases k < i)
case True
have nth-eq: ?selected-chunks ! k = prefix-chunks ! k

```

```

    using True len-prefix-chunks by (simp add: nth-append)
  have
    length (prefix-chunks ! k) =
      verifier-query-round-transcript-length
        (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
  by (rule verifier-query-round-chunk-length
      [OF prefix-rounds[OF True]])
  then show ?thesis
    using nth-eq
  by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
next
case False
show ?thesis
proof (cases k = i)
  case True
  have nth-eq: ?selected-chunks ! k = chunk
    using True len-prefix-chunks by (simp add: nth-append)
  have chunk-shape:
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
  unfolding verifier-query-round-chunk-def
  using actual-layers
  unfolding query-round-fri-layer-transcripts-def by blast
  have
    length chunk =
      verifier-query-round-transcript-length idx
        (map snd f-fl) (map snd fl)
  by (rule verifier-query-round-chunk-length[OF chunk-shape])
  then show ?thesis
    using nth-eq
  by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
next
case False
let ?k = k - Suc i
have ge: Suc i ≤ k
  using <¬ k < i> False by linarith
have k-bound': ?k < rounds - Suc i
  using ge k-bound by linarith
have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
  using ge len-prefix-chunks by (simp add: nth-append)
have
  length (suffix-chunks ! ?k) =
    verifier-query-round-transcript-length
      (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
  by (rule verifier-query-round-chunk-length
      [OF suffix-rounds[OF k-bound']])
then show ?thesis

```

```

      using nth-eq
      by (simp add:
          verifier-query-round-transcript-length-index-irrelevant)
    qed
  qed
qed
have query-chunk-lens:
   $\bigwedge k. k < \text{rounds} \implies$ 
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
  using query-chunk verifier-query-round-chunk-length by blast
have transcript-all:
  PTranscript query-state =
    List.concat ?selected-chunks @ PTranscript final-state
  using transcript-prefix transcript-round transcript-suffix by simp
have concat-eq:
  List.concat ?selected-chunks = List.concat query-chunks
  using transcript-all transcript-query by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
     $\bigwedge k. k < \text{length } ?\text{selected-chunks} \implies$ 
      length (?selected-chunks ! k) = length (query-chunks ! k)
    using selected-chunk-lens query-chunk-lens len-selected-chunks
    by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
    using concat-eq by simp
  have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks  $\wedge$  ([] :: 'f list) = []
    by (rule concat-append-eq-concat-same-chunk-lengths
        [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state)
      ?selected-chunks i =
        state-after-query-chunks (PState query-state)
          prefix-chunks i
    by (rule state-after-query-chunks-append-prefix
        [OF len-prefix-chunks])
  then show ?thesis

```

```

    using chunks-eq by simp
qed
have lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks
        (PState query-state) query-chunks i)) =
    Some raw
  using hash-extension-lookup[OF lookup-suc suffix-ext]
  unfolding counter-i state-i prefix-state-eq .
have raw-eq: raw = raw-idxs ! i
  using lookup-final replay-lookup[OF i-bound] by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-eq query-idxs-eq len-raw i-bound by simp
have auth-table-final:
  partial-authenticated-table fr (scale * clength) openings
  final-state
  by (rule partial-authenticated-table-mono
    [OF auth-table-suc suffix-ext])
have selected-nth: ?selected-chunks ! i = chunk
  using len-prefix-chunks by (simp add: nth-append)
have actual-layers-query:
  query-round-fri-layer-transcripts (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
  actual-trace-layers actual-composition-layers
  using actual-layers idx-eq-query chunks-eq selected-nth by simp
have actual-trace-layers-eq:
  actual-trace-layers = trace-layer-chunks
  by (rule query-round-fri-layer-transcripts-unique(1)
    [OF actual-layers-query query-layers])
show ?thesis
  by (rule that[OF - auth-table-final openings-nonempty
    opening-value-eq])
    (use opening-indices idx-eq-query head-chunk
      actual-trace-layers-eq in simp-all)
qed
qed
qed
qed

```

lemma *accepted-fri-opening-transcript-trace-head-layer-value-aligned-at:*
assumes *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
query-idxs trace-round-layers composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final header-as dg
composition-roots composition-final header-rest
and *out-eq:* *out = Some (result, final-state)*

```

and round-bound: round-idx < length query-idxs
and trace-bs-nonempty: 0 < length trace-bs
obtains openings xp xp-path xn xn-path where
  map opening-index openings = powers-scaled (query-idxs ! round-idx)
  partial-authenticated-table fr (scale * clength) openings final-state
  openings ≠ []
  opening-value (hd openings) = xp
  fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
    (trace-round-layers ! round-idx ! 0)
proof –
from fri-openings obtain result' final-state' fr' f-fl as fl query-state
  raw-idxs query-chunks
where out-def: out = Some (result', final-state')
and trace-roots-eq: trace-roots = map snd f-fl
and trace-bs-eq: trace-bs = map fst f-fl
and composition-roots-eq: composition-roots = map snd fl
and header':
  verifier-header-transcript s fr' trace-roots trace-final as dg
    composition-roots composition-final (PTranscript query-state)
and query-out:
  Some (result', final-state') ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr' f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq: query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state'
and query-chunk:
  ∧ i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)
and query-layers:
  ∧ i. i < rounds ⇒
    query-round-fri-layer-transcripts (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)
      (trace-round-layers ! i) (composition-round-layers ! i)
and replay-lookup:
  ∧ i. i < rounds ⇒
    fmlookup (HashMap final-state')
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)

```

```

unfolding accepted-fri-opening-transcript-def by blast
have fr'-eq: fr' = fr
  using verifier-header-transcript-unique[OF header header'] by simp
have result'-eq: result' = result
  using out-def out-eq by simp
have final-state'-eq: final-state' = final-state
  using out-def out-eq by simp
have query-out':
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr' f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
  using query-out result'-eq final-state'-eq by simp
have round-bound-rounds: round-idx < rounds
  using round-bound
    accepted-fri-opening-transcript-shapes( $\gamma$ )[OF fri-openings]
  by simp
obtain b rt f-fl-tail where f-fl-eq: f-fl = (b, rt) # f-fl-tail
proof (cases f-fl)
  case Nil
  then show ?thesis
    using trace-bs-nonempty trace-bs-eq by simp
next
  case (Cons head tail)
  then obtain b rt where head-eq: head = (b, rt)
    by (cases head)
  then show ?thesis
    using Cons that by simp
qed
have query-layer:
  query-round-fri-layer-transcripts (query-idxs ! round-idx)
    (map snd f-fl) (map snd fl) (query-chunks ! round-idx)
    (trace-round-layers ! round-idx)
    (composition-round-layers ! round-idx)
  using query-layers[OF round-bound-rounds]
    trace-roots-eq composition-roots-eq by simp
have ex:
   $\exists$  openings xp xp-path xn xn-path.
    map opening-index openings = powers-scaled (query-idxs ! round-idx) ∧
    partial-authenticated-table fr' (scale * clength) openings final-state ∧
    openings ≠ [] ∧
    opening-value (hd openings) = xp ∧
    fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
    (trace-round-layers ! round-idx ! 0)
proof (rule
  ntimes-verifier-query-rounds-selected-trace-fri-head-layer-value-aligned-with-replay-at

```



```

    [OF f-fl-eq query-out' round-bound-rounds len-raw query-idxs-eq
      len-query-chunks])
  show PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
    using transcript-query final-state'-eq by simp
  show  $\bigwedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
    using query-chunk trace-roots-eq composition-roots-eq by simp
  show query-round-fri-layer-transcripts (query-idxs ! round-idx)
    (map snd f-fl) (map snd fl) (query-chunks ! round-idx)
    (trace-round-layers ! round-idx)
    (composition-round-layers ! round-idx)
    by (rule query-layer)
  show  $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
    using replay-lookup final-state'-eq by simp
next
  fix openings xp xp-path xn xn-path
  assume map opening-index openings =
    powers-scaled (query-idxs ! round-idx)
  and partial-authenticated-table fr' (scale * clength) openings
    final-state
  and openings  $\neq []$ 
  and opening-value (hd openings) = xp
  and fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
    (trace-round-layers ! round-idx ! 0)
  then show ?thesis
    by blast
qed
from ex show ?thesis
  by (metis fr'-eq that)
qed

lemma trace-fri-header-tied-base-head-value-neq-gap-imp-partial-merkle:
  assumes trace-fri-header-tied-base-head-value-neq-gap s out
  shows partial-merkle-inconsistency-bad s out
proof -
  from assms obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as rest trace-openings trace-table
    round-idx xp xp-path xn xn-path result final-state
  where out-eq: out = Some (result, final-state)
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs

```

```

    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
and header:
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest
and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
and candidate: partial-trace-table-candidate trace-table trace-openings
and nonlow:  $\neg$  trace-table-low-degree trace-table
and round-bound: round-idx < length fri-query-idxs
and trace-bs-nonempty: 0 < length trace-bs
and openings-nonempty: trace-openings ! round-idx  $\neq$  []
and hd-idx:
    opening-index (hd (trace-openings ! round-idx)) =
    fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0
and hd-neq: opening-value (hd (trace-openings ! round-idx))  $\neq$  xp
and step:
    fri-layer-step-evidence
    (trace-roots ! 0)
    (trace-bs ! 0)
    (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    1
    (fri-sibling-index (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
    xp xp-path xn xn-path
    (fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
    (fri-evidence-next-value trace-roots trace-bs fri-query-idxs
    round-idx 0 xp xn)
    (trace-round-layers ! round-idx ! 0)
and no-match:
     $\neg$  fri-opening-matches-table (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    trace-table xp xn
unfolding trace-fri-header-tied-base-head-value-neq-gap-def
by metis
have round-bound-rounds: round-idx < rounds
using round-bound
    accepted-fri-opening-transcript-shapes( $\gamma$ )[OF fri-openings]
by simp
from accepted-fri-opening-transcript-trace-head-layer-value-aligned-at
    [OF fri-openings header out-eq round-bound trace-bs-nonempty]
obtain openings yp yp-path yn yn-path where actual-idxs:
    map opening-index openings = powers-scaled (fri-query-idxs ! round-idx)
and actual-auth:
    partial-authenticated-table fr (scale * clength) openings final-state
and actual-nonempty: openings  $\neq$  []
and actual-value: opening-value (hd openings) = yp

```

```

and actual-chunk:
  fri-layer-opening-chunk (scale * clength) yp yp-path yn yn-path
    (trace-round-layers ! round-idx ! 0)
by blast
have roots-nonempty: 0 < length trace-roots
using accepted-fri-opening-transcript-shapes(2)[OF fri-openings]
  trace-bs-nonempty by simp
have len-eq:
  fri-evidence-layer-len trace-roots 0 = scale * clength
unfolding fri-evidence-layer-len-def
using roots-nonempty
by (cases trace-roots) (simp-all add: mult.commute)
have step-chunk:
  fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
    (trace-round-layers ! round-idx ! 0)
using fri-layer-step-evidenceD(4)[OF step]
unfolding len-eq .
have yp-eq: yp = xp
by (rule fri-layer-opening-chunk-values-unique(1)
  [OF actual-chunk step-chunk])
have actual-head-idx:
  opening-index (hd openings) =
    fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0
proof –
  have opening-index (hd openings) = fri-query-idxs ! round-idx
proof –
  have opening-index (hd openings) =
    hd (map opening-index openings)
    using actual-nonempty by (cases openings) auto
  also have ... = hd (powers-scaled (fri-query-idxs ! round-idx))
    using actual-idxs by simp
  also have ... = fri-query-idxs ! round-idx
    by (rule powers-scaled-hd)
  finally show ?thesis .
qed
moreover have
  fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0 =
    fri-query-idxs ! round-idx
  using roots-nonempty
  unfolding fri-evidence-layer-idx-def
  by (cases trace-roots) simp-all
  ultimately show ?thesis by simp
qed
let ?opn = hd (trace-openings ! round-idx)
let ?opn' = hd openings
have trace-table:
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! round-idx) final-state
  using partial round-bound-rounds out-eq

```

```

    unfolding accepted-with-partial-trace-openings-def by blast
have opn-auth: authenticated-opening-in final-state ?opn
  using trace-table openings-nonempty
  unfolding partial-authenticated-table-def by simp
have opn'-auth: authenticated-opening-in final-state ?opn'
  using actual-auth actual-nonempty
  unfolding partial-authenticated-table-def by simp
have same-root: opening-root ?opn = opening-root ?opn'
  using trace-table actual-auth openings-nonempty actual-nonempty
  unfolding partial-authenticated-table-def by auto
have same-length: opening-length ?opn = opening-length ?opn'
  using trace-table actual-auth openings-nonempty actual-nonempty
  unfolding partial-authenticated-table-def by auto
have same-index: opening-index ?opn = opening-index ?opn'
  using hd-idx actual-head-idx by simp
have diff-value: opening-value ?opn  $\neq$  opening-value ?opn'
  using hd-neq actual-value yp-eq by simp
have acc: accepted out
  unfolding accepted-def out-eq by simp
show ?thesis
  unfolding partial-merkle-inconsistency-bad-def
  by (intro conjI exI)
    (rule acc, rule out-eq, rule opn-auth, rule opn'-auth,
     rule same-root, rule same-length, rule same-index, rule diff-value)
qed

lemma wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle:
  assumes merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s  $\leq$  M
  shows
    wp-event verify-monad
      (trace-fri-header-tied-base-head-value-neq-gap s) s  $\leq$  M
proof -
  have wp-event verify-monad
    (trace-fri-header-tied-base-head-value-neq-gap s) s  $\leq$ 
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s
  by (rule wp-event-mono)
    (rule trace-fri-header-tied-base-head-value-neq-gap-imp-partial-merkle)
  then show ?thesis
    by (rule order-trans[OF merkle-bound])
qed

end

end

```

```

theory Soundness-FRI-Trace-Candidate-Gap
  imports Soundness-FRI-Trace-Head-Value

```

begin

Candidate-availability split for the reachable/header-tied trace FRI gap. This layer depends on the exact FRI-query opening bridge but keeps the exported split separate from the base head-value residual theory.

context *soundness*

begin

lemma *trace-fri-reachable-header-tie-gap-imp-merkle-or-candidate-available-on-support:*

assumes *support: out ∈ set-dist (execute verify-monad s)*

and *gap: trace-fri-reachable-header-tie-gap s out*

shows

partial-merkle-inconsistency-bad s out ∨

trace-fri-header-tied-candidate-available-gap s out

proof (*cases partial-merkle-inconsistency-bad s out*)

case *True*

then show *?thesis* **by** *simp*

next

case *no-merkle: False*

from *gap* **have** *raw: trace-fri-bad-with-reachable-partial-candidate s out*

unfolding *trace-fri-reachable-header-tie-gap-def* **by** *simp*

from *trace-fri-bad-with-reachable-partial-candidateE[OF raw]*

obtain *fr0 query-idxs0 trace-openings0 trace-table0* **where** *partial0:*

accepted-with-partial-trace-openings s out fr0 query-idxs0

trace-openings0

using *raw* **unfolding** *trace-fri-bad-with-reachable-partial-candidate-def*

by *blast*

have *acc: accepted out*

by (*rule accepted-with-partial-trace-openings-imp-accepted[OF partial0]*)

from *acc* **obtain** *result final-state* **where** *out-eq:*

out = Some (result, final-state)

unfolding *accepted-def* **by** (*cases out*) *auto*

from *verify-monad-trace-fri-bad-extracts-opening-evidence*

[OF support[unfolded out-eq] raw[unfolded out-eq]]

obtain *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr1 candidate-query-idxs trace-openings1

trace-table1

where *evidence:*

trace-fri-partial-candidate-opening-evidence s

(Some (result, final-state)) trace-roots trace-bs trace-final dg

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr1 candidate-query-idxs

trace-openings1 trace-table1

by (*rule verify-monad-trace-fri-bad-extracts-opening-evidence*

[OF support[unfolded out-eq] raw[unfolded out-eq]])

have *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final dg composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers
using *trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence]*
unfolding *out-eq* .
from *accepted-fri-opening-transcript-headerE[OF fri-openings]*
obtain *fr as rest where header:*
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
by (rule *accepted-fri-opening-transcript-headerE[OF fri-openings]*)
from *accepted-fri-opening-transcript-trace-openings-for-query-indices*
[OF fri-openings header out-eq]
obtain *trace-openings where partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
by (rule *accepted-fri-opening-transcript-trace-openings-for-query-indices*
[OF fri-openings header out-eq])
obtain *trace-table where candidate:*
partial-trace-table-candidate trace-table trace-openings
using *accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad*
[OF partial[unfolded out-eq] no-merkle[unfolded out-eq]]
by *blast*
show *?thesis*
unfolding *trace-fri-header-tied-candidate-available-gap-def*
by (intro *disjI2 exI conjI*)
(rule gap, rule fri-openings, rule header, rule partial,
rule candidate)
qed

lemma *wp-trace-fri-reachable-header-tie-gap-bound-from-merkle-and-candidate-transfer:*
assumes *merkle-bound:*
wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M
and *transfer-bound:*
wp-event verify-monad
(trace-fri-header-tied-candidate-transfer-gap s) s ≤ T
shows
wp-event verify-monad
(trace-fri-reachable-header-tie-gap s) s ≤ M + T
proof –
have *split-bound:*
wp-event verify-monad
(trace-fri-reachable-header-tie-gap s) s ≤
wp-event verify-monad
(λout. partial-merkle-inconsistency-bad s out ∨
trace-fri-header-tied-candidate-available-gap s out) s
by (rule *wp-event-mono-on-support*)
(rule
trace-fri-reachable-header-tie-gap-imp-merkle-or-candidate-available-on-support)
also have ... ≤
wp-event verify-monad (partial-merkle-inconsistency-bad s) s +
wp-event verify-monad

```

      (trace-fri-header-tied-candidate-available-gap s) s
    by (rule wp-event-union-bound)
  also have ...  $\leq M + T$ 
  by (rule add-mono[OF merkle-bound])
    (rule
      wp-trace-fri-header-tied-candidate-available-gap-bound-from-transfer-gap
      [OF transfer-bound])
  finally show ?thesis .
qed

lemma trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-candidate-transfer:
  assumes header-bound:
    wp-event verify-monad
      (trace-fri-bad-with-header-tied-partial-candidate s) s  $\leq Ht$ 
  and merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s  $\leq Mt$ 
  and transfer-bound:
    wp-event verify-monad
      (trace-fri-header-tied-candidate-transfer-gap s) s  $\leq Tt$ 
  and total-bound:  $Ht + (Mt + Tt) \leq \text{trace-fri-error}$ 
  shows trace-fri-reachable-partial-candidate-reduction-assumption s
proof -
  have gap-bound:
    wp-event verify-monad
      (trace-fri-reachable-header-tie-gap s) s  $\leq Mt + Tt$ 
  by (rule
      wp-trace-fri-reachable-header-tie-gap-bound-from-merkle-and-candidate-transfer
      [OF merkle-bound transfer-bound])
  have reachable-bound:
    wp-event verify-monad
      (trace-fri-bad-with-reachable-partial-candidate s) s  $\leq$ 
       $Ht + (Mt + Tt)$ 
  by (rule wp-trace-fri-reachable-bound-from-header-tied-and-gap
      [OF header-bound gap-bound])
  show ?thesis
  unfolding trace-fri-reachable-partial-candidate-reduction-assumption-def
  by (rule order-trans[OF reachable-bound total-bound])
qed

end

end

```

```

theory Soundness-FRI-Composition-Support
  imports Soundness-FRI-Composition-Residuals
begin

```

Small support/extraction facts for the composition FRI reachable gap. These are fac-

tored out to avoid large direct proofs that stall theory finalization.

context *soundness*
begin

definition *composition-fri-reachable-verifier-tie-empty-header-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-reachable-verifier-tie-empty-header-gap *s out*
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers.}$
 $\text{ composition-fri-reachable-verifier-tie-gap } s \text{ out} \wedge$
 $\text{ accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{ composition-roots} = [])$

lemma *composition-fri-bad-with-reachable-partial-candidate-imp-accepted:*

assumes *composition-fri-bad-with-reachable-partial-candidate* *s out*
shows *accepted out*

proof (rule *composition-fri-bad-with-reachable-partial-candidateE*[*OF assms*])

fix *fr f-roots f-final as dg composition-roots final*
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table}$

assume *evidence:*

$\text{ composition-fri-partial-candidate-evidence } s \text{ out fr f-roots}$
 $\text{ f-final as dg composition-roots final trace-query-idxs}$
 $\text{ trace-openings composition-query-idxs composition-openings}$
 $\text{ trace-table composition-table}$

have *partial:*

$\text{ accepted-with-partial-initial-openings } s \text{ out fr f-roots f-final}$
 $\text{ as dg composition-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings}$

by (rule *composition-fri-partial-candidate-evidenceD*(1)[*OF evidence*])

then show *accepted out*

by (rule *accepted-with-partial-initial-openings-imp-accepted*)

qed

lemma *composition-fri-reachable-verifier-tie-gap-imp-reachable-bad:*

assumes *composition-fri-reachable-verifier-tie-gap* *s out*
shows *composition-fri-bad-with-reachable-partial-candidate* *s out*
using *assms unfolding composition-fri-reachable-verifier-tie-gap-def* **by** *simp*

lemma *composition-fri-reachable-verifier-tie-gap-imp-accepted:*

assumes *composition-fri-reachable-verifier-tie-gap* *s out*
shows *accepted out*

by (rule *composition-fri-bad-with-reachable-partial-candidate-imp-accepted*)
 (rule *composition-fri-reachable-verifier-tie-gap-imp-reachable-bad*[*OF assms*])

lemma *composition-fri-reachable-verifier-tie-empty-header-gapI*:
assumes *gap*: *composition-fri-reachable-verifier-tie-gap s out*
and *fri-openings*:
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
and *empty*: *composition-roots = []*
shows *composition-fri-reachable-verifier-tie-empty-header-gap s out*
unfolding *composition-fri-reachable-verifier-tie-empty-header-gap-def*
by (*intro exI conjI*)
 (rule *gap*, rule *fri-openings*, rule *empty*)

lemma *composition-fri-reachable-verifier-tie-empty-header-gap-false*:
 $\neg$ *composition-fri-reachable-verifier-tie-empty-header-gap s out*

proof

assume *empty-gap*:
 composition-fri-reachable-verifier-tie-empty-header-gap s out
then obtain *trace-roots trace-bs trace-final fri-dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers
where *gap*: *composition-fri-reachable-verifier-tie-gap s out*
and *fri-openings*:
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
and *empty*: *composition-roots = []*
unfolding *composition-fri-reachable-verifier-tie-empty-header-gap-def*
by *blast*
obtain *fr as rest* **where** *verifier-header*:
 verifier-header-transcript s fr trace-roots trace-final as fri-dg
 composition-roots composition-final rest
by (rule *accepted-fri-opening-transcript-headerE*[*OF fri-openings*])
have *raw*: *composition-fri-bad-with-reachable-partial-candidate s out*
using *gap* **unfolding** *composition-fri-reachable-verifier-tie-gap-def*
by *simp*
from *composition-fri-bad-with-reachable-partial-candidateE*[*OF raw*]
obtain *fr' f-fri-roots f-final as' dg' composition-fri-roots final*
 trace-query-idxs trace-openings composition-query-idxs
 composition-openings trace-table composition-table
where *evidence*:
 composition-fri-partial-candidate-evidence s out fr' f-fri-roots
 f-final as' dg' composition-fri-roots final trace-query-idxs
 trace-openings composition-query-idxs composition-openings
 trace-table composition-table

```

    by blast
  have partial:
    accepted-with-partial-initial-openings s out fr' f-fri-roots
      f-final as' dg' composition-fri-roots final trace-query-idxs
      trace-openings composition-query-idxs composition-openings
    by (rule composition-fri-partial-candidate-evidenceD(1)[OF evidence])
  have nonempty: composition-fri-roots ≠ []
    by (rule accepted-with-partial-initial-openings-shapes(1)[OF partial])
  obtain rest' where candidate-header:
    verifier-header-transcript s fr' f-fri-roots f-final as' dg'
      composition-fri-roots final rest'
  using accepted-with-partial-initial-openings-shapes(2)[OF partial]
  by blast
  have composition-fri-roots = composition-roots
    using verifier-header-transcript-unique
      [OF verifier-header candidate-header]
  by simp
  then show False
    using empty nonempty by simp
qed

lemma wp-composition-fri-reachable-verifier-tie-empty-header-gap-zero:
  wp-event verify-monad
    (composition-fri-reachable-verifier-tie-empty-header-gap s) s = 0
  unfolding wp-event-def wp-def dist-expect-def
  by (simp add:
    composition-fri-reachable-verifier-tie-empty-header-gap-false)

lemma composition-fri-reachable-verifier-tie-gap-with-fri-openings-empty-or-nonempty:
  assumes gap: composition-fri-reachable-verifier-tie-gap s out
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  shows
    composition-fri-reachable-verifier-tie-empty-header-gap s out ∨
    composition-roots ≠ []
  proof (cases composition-roots = [])
  case True
  have composition-fri-reachable-verifier-tie-empty-header-gap s out
    by (rule composition-fri-reachable-verifier-tie-empty-header-gapI
      [OF gap fri-openings True])
  then show ?thesis by simp
  next
  case False
  then show ?thesis by simp
qed

```

lemma *composition-fri-verifier-tied-candidate-available-gapI*:
assumes *gap*: *composition-fri-reachable-verifier-tie-gap s out*
and *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *aligned*:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
and *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
and *composition-candidate*:
partial-composition-table-candidate composition-table
composition-openings
shows *composition-fri-verifier-tied-candidate-available-gap s out*
unfolding *composition-fri-verifier-tied-candidate-available-gap-def*
by (*intro exI conjI*)
(*rule gap*, *rule fri-openings*, *rule aligned*, *rule trace-candidate*,
rule composition-candidate)

lemma *composition-fri-reachable-verifier-tie-gap-with-aligned-openings-imp-merkle-or-candidate-available*:
assumes *out-eq*: *out = Some (result, final-state)*
and *gap*: *composition-fri-reachable-verifier-tie-gap s out*
and *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *aligned*:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
shows
partial-merkle-inconsistency-bad s out $\vee$
composition-fri-verifier-tied-candidate-available-gap s out
proof (*cases partial-merkle-inconsistency-bad s out*)
case *True*
then show *?thesis* **by** *simp*
next
case *no-merkle*: *False*
obtain *trace-table composition-table* **where** *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
and *composition-candidate*:
partial-composition-table-candidate composition-table
composition-openings
using
accepted-with-partial-initial-openings-aligned-candidates-if-no-partial-merkle-bad

```

    [OF aligned[unfolded out-eq] no-merkle[unfolded out-eq]]
  by blast
have available:
  composition-fri-verifier-tied-candidate-available-gap s out
  by (rule composition-fri-verifier-tied-candidate-available-gapI
    [OF gap fri-openings aligned trace-candidate composition-candidate])
then show ?thesis by simp
qed

lemma composition-fri-reachable-verifier-tie-gap-with-fri-openings-headerE:
  assumes
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  obtains fr as rest where
    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest
  by (rule accepted-fri-opening-transcript-headerE[OF assms])

lemma composition-fri-reachable-verifier-tie-gap-with-fri-openings-header-and-branchE:
  assumes gap: composition-fri-reachable-verifier-tie-gap s out
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  obtains fr as rest where
    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest
    composition-fri-reachable-verifier-tie-empty-header-gap s out  $\vee$ 
    composition-roots  $\neq$  []
proof -
  obtain fr as rest where header:
    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest
  by (rule composition-fri-reachable-verifier-tie-gap-with-fri-openings-headerE
    [OF fri-openings])
  have branch:
    composition-fri-reachable-verifier-tie-empty-header-gap s out  $\vee$ 
    composition-roots  $\neq$  []
  by (rule
    composition-fri-reachable-verifier-tie-gap-with-fri-openings-empty-or-nonempty
    [OF gap fri-openings])
  show ?thesis
  by (rule that[OF header branch])
qed

lemma composition-fri-reachable-verifier-tie-gap-nonempty-imp-merkle-or-candidate-available:

```

assumes *out-eq*: *out* = *Some* (*result*, *final-state*)
and *gap*: *composition-fri-reachable-verifier-tie-gap* *s out*
and *fri-openings*:
 accepted-fri-opening-transcript *s out trace-roots trace-bs*
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
and *nonempty*: *composition-roots* $\neq$ []
shows
 partial-merkle-inconsistency-bad *s out* $\vee$
 composition-fri-verifier-tied-candidate-available-gap *s out*
proof –
have *split*:
 partial-merkle-inconsistency-bad *s* (*Some* (*result*, *final-state*)) $\vee$
 ($\exists$ *fr as rest trace-openings composition-openings trace-table*
 composition-table.
 verifier-header-transcript *s fr trace-roots trace-final as fri-dg*
 composition-roots composition-final rest $\wedge$
 accepted-with-partial-initial-openings-aligned *s*
 (*Some* (*result*, *final-state*)) *fr trace-roots trace-final as fri-dg*
 composition-roots composition-final fri-query-idxs trace-openings
 composition-openings $\wedge$
 partial-trace-table-candidate trace-table trace-openings $\wedge$
 partial-composition-table-candidate composition-table
 composition-openings)
by (*rule*
 accepted-fri-opening-transcript-obtains-aligned-partial-candidates-or-partial-merkle
 [*OF fri-openings*[*unfolded out-eq*] *nonempty*])
show ?thesis
proof (*rule disjE*[*OF split*])
assume *partial-merkle-inconsistency-bad* *s* (*Some* (*result*, *final-state*))
then show ?thesis
by (*simp add*: *out-eq*)
next
assume
 $\exists$ *fr as rest trace-openings composition-openings trace-table*
 composition-table.
 verifier-header-transcript *s fr trace-roots trace-final as fri-dg*
 composition-roots composition-final rest $\wedge$
 accepted-with-partial-initial-openings-aligned *s*
 (*Some* (*result*, *final-state*)) *fr trace-roots trace-final as fri-dg*
 composition-roots composition-final fri-query-idxs trace-openings
 composition-openings $\wedge$
 partial-trace-table-candidate trace-table trace-openings $\wedge$
 partial-composition-table-candidate composition-table
 composition-openings
then obtain *fr as rest trace-openings composition-openings trace-table*
 composition-table **where** *aligned*:
 accepted-with-partial-initial-openings-aligned *s out fr trace-roots*

```

    trace-final as fri-dg composition-roots composition-final
    fri-query-idxs trace-openings composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  by (auto simp: out-eq)
have available:
  composition-fri-verifier-tied-candidate-available-gap s out
  by (rule composition-fri-verifier-tied-candidate-available-gapI
    [OF gap fri-openings aligned trace-candidate composition-candidate])
then show ?thesis by simp
qed
qed

lemma composition-fri-reachable-verifier-tie-gap-imp-empty-or-merkle-or-candidate-available-on-support:
  assumes support: out ∈ set-dist (execute verify-monad s)
  and gap: composition-fri-reachable-verifier-tie-gap s out
  shows
    composition-fri-reachable-verifier-tie-empty-header-gap s out ∨
    partial-merkle-inconsistency-bad s out ∨
    composition-fri-verifier-tied-candidate-available-gap s out
proof -
  have raw: composition-fri-bad-with-reachable-partial-candidate s out
  using gap
  unfolding composition-fri-reachable-verifier-tie-gap-def by simp
  have acc: accepted out
  by (rule composition-fri-bad-with-reachable-partial-candidate-imp-accepted
    [OF raw])
  obtain result final-state where out-eq: out = Some (result, final-state)
  using acc unfolding accepted-def by (cases out) auto
  from verify-monad-accepted-fri-opening-transcript[OF support[unfolded out-eq]]
  obtain trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  unfolding out-eq by blast
  show ?thesis
proof (cases composition-roots = [])
  case True
  have composition-fri-reachable-verifier-tie-empty-header-gap s out
  by (rule composition-fri-reachable-verifier-tie-empty-header-gapI
    [OF gap fri-openings True])
  then show ?thesis by simp

```

```

next
  case False
  have
    partial-merkle-inconsistency-bad s out  $\vee$ 
    composition-fri-verifier-tied-candidate-available-gap s out
  by (rule
    composition-fri-reachable-verifier-tie-gap-nonempty-imp-merkle-or-candidate-available
    [OF out-eq gap fri-openings False])
  then show ?thesis by simp
qed
qed

lemma wp-composition-fri-reachable-verifier-tie-gap-bound-from-empty-merkle-and-candidate-transfer:
  assumes empty-bound:
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-empty-header-gap s)  $s \leq E$ 
  and merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s)  $s \leq M$ 
  and transfer-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-candidate-transfer-gap s)  $s \leq T$ 
  shows
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-gap s)  $s \leq E + (M + T)$ 
proof -
  have split-bound:
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-gap s)  $s \leq$ 
    wp-event verify-monad
      ( $\lambda out.$ 
        composition-fri-reachable-verifier-tie-empty-header-gap s out  $\vee$ 
        partial-merkle-inconsistency-bad s out  $\vee$ 
        composition-fri-verifier-tied-candidate-available-gap s out)  $s$ 
  by (rule wp-event-mono-on-support)
  (rule
    composition-fri-reachable-verifier-tie-gap-imp-empty-or-merkle-or-candidate-available-on-support)
  also have ...  $\leq$ 
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-empty-header-gap s)  $s +$ 
    wp-event verify-monad
      ( $\lambda out.$ 
        partial-merkle-inconsistency-bad s out  $\vee$ 
        composition-fri-verifier-tied-candidate-available-gap s out)  $s$ 
  by (rule wp-event-union-bound)
  also have ...  $\leq$ 
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-empty-header-gap s)  $s +$ 
    (wp-event verify-monad (partial-merkle-inconsistency-bad s)  $s +$ 
    wp-event verify-monad

```

(composition-fri-verifier-tied-candidate-available-gap s) s)
 by (rule add-mono[OF order-refl wp-event-union-bound])
 also have ... $\leq E + (M + T)$
 by (intro add-mono empty-bound merkle-bound
 wp-composition-fri-verifier-tied-candidate-available-gap-bound-from-transfer-gap
 transfer-bound)
 finally show ?thesis .
 qed

lemma composition-fri-reachable-partial-candidate-reduction-from-split-residual-bounds:
 fixes R A P M N S G E Q T :: prob
 assumes sampled-bound:
 wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s $\leq R$
 and slot-gap-bound:
 wp-event verify-monad
 (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
 s) s $\leq A$
 and verifier-partial-merkle-bound:
 wp-event verify-monad (partial-merkle-inconsistency-bad s) s $\leq P$
 and missing-bound:
 wp-event verify-monad
 (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
 s $\leq M$
 and next-gap-bound:
 wp-event verify-monad
 (composition-fri-verifier-tied-next-value-replay-gap s) s $\leq N$
 and successor-gap-bound:
 wp-event verify-monad
 (composition-fri-verifier-tied-successor-replay-gap s) s $\leq S$
 and final-gap-bound:
 wp-event verify-monad
 (composition-fri-verifier-tied-final-value-replay-gap s) s $\leq G$
 and empty-bound:
 wp-event verify-monad
 (composition-fri-reachable-verifier-tie-empty-header-gap s) s $\leq E$
 and gap-partial-merkle-bound:
 wp-event verify-monad (partial-merkle-inconsistency-bad s) s $\leq Q$
 and candidate-transfer-bound:
 wp-event verify-monad
 (composition-fri-verifier-tied-candidate-transfer-gap s) s $\leq T$
 and total-bound:
 R + (((P + M) + N + S + G + P) + A) + (E + (Q + T))
 $\leq$ composition-fri-error
 shows composition-fri-reachable-partial-candidate-reduction-assumption s
 proof –
 have verifier-bound:
 wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-partial-candidate s) s

$\leq R + (((P + M) + N + S + G + P) + A)$
by (rule
wp-composition-fri-verifier-tied-bound-from-sampled-slot-recorded-chunk-gap-and-residuals
 $[OF \text{ sampled-bound slot-gap-bound verifier-partial-merkle-bound}$
 $\text{missing-bound next-gap-bound successor-gap-bound final-gap-bound}]$)
have *gap-bound*:
wp-event verify-monad
 $(\text{composition-fri-reachable-verifier-tie-gap } s) \ s \leq$
 $E + (Q + T)$
by (rule
wp-composition-fri-reachable-verifier-tie-gap-bound-from-empty-merkle-and-candidate-transfer
 $[OF \text{ empty-bound gap-partial-merkle-bound candidate-transfer-bound}]$)
show *?thesis*
by (rule
composition-fri-reachable-partial-candidate-reduction-from-verifier-tied-and-gap
 $[OF \text{ verifier-bound gap-bound total-bound}]$)
qed

lemma *composition-fri-reachable-partial-candidate-reduction-from-split-residual-bounds-with-recorded-missing-zero*:
fixes $R \ A \ P \ N \ S \ G \ E \ Q \ T :: \text{prob}$
assumes *sampled-bound*:
wp-event verify-monad
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain } s) \ s \leq R$
and *slot-gap-bound*:
wp-event verify-monad
 $(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap}$
 $s) \ s \leq A$
and *verifier-partial-merkle-bound*:
wp-event verify-monad $(\text{partial-merkle-inconsistency-bad } s) \ s \leq P$
and *next-gap-bound*:
wp-event verify-monad
 $(\text{composition-fri-verifier-tied-next-value-replay-gap } s) \ s \leq N$
and *successor-gap-bound*:
wp-event verify-monad
 $(\text{composition-fri-verifier-tied-successor-replay-gap } s) \ s \leq S$
and *final-gap-bound*:
wp-event verify-monad
 $(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq G$
and *empty-bound*:
wp-event verify-monad
 $(\text{composition-fri-reachable-verifier-tie-empty-header-gap } s) \ s \leq E$
and *gap-partial-merkle-bound*:
wp-event verify-monad $(\text{partial-merkle-inconsistency-bad } s) \ s \leq Q$
and *candidate-transfer-bound*:
wp-event verify-monad
 $(\text{composition-fri-verifier-tied-candidate-transfer-gap } s) \ s \leq T$
and *total-bound*:
 $R + (((P + 0) + N + S + G + P) + A) + (E + (Q + T))$
 $\leq \text{composition-fri-error}$

```

shows composition-fri-reachable-partial-candidate-reduction-assumption s
by (rule
  composition-fri-reachable-partial-candidate-reduction-from-split-residual-bounds
  [where  $M = 0$ ])
  (rule sampled-bound,
   rule slot-gap-bound,
   rule verifier-partial-merkle-bound,
   rule wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero,
   rule next-gap-bound,
   rule successor-gap-bound,
   rule final-gap-bound,
   rule empty-bound,
   rule gap-partial-merkle-bound,
   rule candidate-transfer-bound,
   rule total-bound)

```

end

end

theory *Soundness-FRI-Candidate-Transfer-Support*

imports

Soundness-FRI-Trace-Candidate-Gap

Soundness-FRI-Composition-Support

begin

Small destructors for the remaining FRI candidate-transfer branches. These branches are intentionally not closed by complete-table uniqueness: sampled partial openings can admit multiple completions. The lemmas here expose the exact low/non-low candidate shape that later proofs must charge to a partial-opening inconsistency or a narrower candidate-indexed event.

context *soundness*

begin

definition *trace-fri-header-tied-low-candidate-available*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-low-candidate-available s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table.}$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{ verifier-header-transcript s fr trace-roots trace-final as fri-dg}$
 $\text{ composition-roots composition-final rest} \wedge$

accepted-with-partial-trace-openings *s out fr fri-query-idxs*
trace-openings $\wedge$
partial-trace-table-candidate *trace-table trace-openings* $\wedge$
trace-table-low-degree *trace-table*)

definition *trace-fri-header-tied-low-candidate-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-low-candidate-gap *s out* $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
trace-fri-reachable-header-tie-gap *s out* $\wedge$
accepted-fri-opening-transcript *s out trace-roots trace-bs*
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers $\wedge$
verifier-header-transcript *s fr trace-roots trace-final as fri-dg*
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings *s out fr fri-query-idxs*
trace-openings $\wedge$
partial-trace-table-candidate *trace-table trace-openings* $\wedge$
trace-table-low-degree *trace-table*)

definition *composition-fri-verifier-tied-misaligned-candidate-classification*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-misaligned-candidate-classification *s out*
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$
 $\text{trace-table composition-table.}$
accepted-fri-opening-transcript *s out trace-roots trace-bs*
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers $\wedge$
accepted-with-partial-initial-openings-aligned *s out fr trace-roots*
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings $\wedge$
partial-trace-table-candidate *trace-table trace-openings* $\wedge$
partial-composition-table-candidate *composition-table*
composition-openings $\wedge$
 $\neg (\text{trace-table-low-degree } \text{trace-table} \wedge$
 $\neg \text{ composition-table-low-degree } \text{maxDegree } \text{composition-table}))$

definition *composition-fri-verifier-tied-transcript-consistent-misaligned-classification*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-transcript-consistent-misaligned-classification}$
 $s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings}$
 $\text{composition-openings trace-table composition-table.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as fri-dg}$
 $\text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned-transcript-consistent } s$
 $\text{out fr trace-roots trace-final as fri-dg composition-roots}$
 $\text{composition-final fri-query-idxs trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\neg (\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table}))$

definition $\text{composition-fri-verifier-tied-transcript-consistent-classification-cover}$

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-transcript-consistent-classification-cover}$
 $s \text{ out} \longleftrightarrow$
 $\text{composition-bad-with-partial-candidates } s \text{ out} \vee$
 $\text{trace-fri-bad-with-header-tied-partial-candidate } s \text{ out} \vee$
 $\text{query-bad-with-aligned-transcript-partial-candidates } s \text{ out}$

lemma $\text{trace-fri-header-tied-candidate-transfer-gapD:}$

assumes $\text{trace-fri-header-tied-candidate-transfer-gap } s \text{ out}$

shows

$\text{trace-fri-bad-with-reachable-partial-candidate } s \text{ out}$
 $\text{trace-fri-header-tied-low-candidate-available } s \text{ out}$

proof —

from assms **obtain** $\text{trace-roots trace-bs trace-final fri-dg}$
 $\text{composition-roots composition-bs composition-final fri-query-idxs}$
 $\text{trace-round-layers composition-round-layers fr as rest trace-openings}$
 trace-table **where**

$\text{bad: trace-fri-bad-with-reachable-partial-candidate } s \text{ out}$
and fri-openings:
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$

composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate:*
partial-trace-table-candidate trace-table trace-openings
and *low:* *trace-table-low-degree trace-table*
unfolding *trace-fri-header-tied-candidate-transfer-gap-def*
by *blast*
show *trace-fri-bad-with-reachable-partial-candidate s out*
by (*rule bad*)
show *trace-fri-header-tied-low-candidate-available s out*
unfolding *trace-fri-header-tied-low-candidate-available-def*
by (*intro exI conjI*)
(*rule fri-openings, rule header, rule partial, rule candidate,*
rule low)
qed

lemma *trace-fri-header-tied-candidate-transfer-gap-imp-accepted:*
assumes *trace-fri-header-tied-candidate-transfer-gap s out*
shows *accepted out*
proof –
have *bad:* *trace-fri-bad-with-reachable-partial-candidate s out*
by (*rule trace-fri-header-tied-candidate-transfer-gapD(1)[OF assms]*)
from *trace-fri-bad-with-reachable-partial-candidateE[OF bad]*
obtain *fr query-idxs trace-openings trace-table* **where** *evidence:*
trace-fri-partial-candidate-evidence s out fr query-idxs
trace-openings trace-table
by *blast*
have *partial:*
accepted-with-partial-trace-openings s out fr query-idxs
trace-openings
by (*rule trace-fri-partial-candidate-evidenceD(1)[OF evidence]*)
show *?thesis*
by (*rule accepted-with-partial-trace-openings-imp-accepted[OF partial]*)
qed

lemma *wp-trace-fri-header-tied-candidate-transfer-gap-bound-from-low-candidate:*
assumes *low-bound:*
wp-event verify-monad
(*trace-fri-header-tied-low-candidate-available s*) $s \leq T$
shows
wp-event verify-monad
(*trace-fri-header-tied-candidate-transfer-gap s*) $s \leq T$
proof –
have *wp-event verify-monad*

```

    (trace-fri-header-tied-candidate-transfer-gap s) s ≤
wp-event verify-monad
    (trace-fri-header-tied-low-candidate-available s) s
by (rule wp-event-mono)
    (rule trace-fri-header-tied-candidate-transfer-gapD(2))
then show ?thesis
    by (rule order-trans[OF - low-bound])
qed

lemma trace-fri-header-tied-low-candidate-gap-imp-low-candidate-available:
  assumes trace-fri-header-tied-low-candidate-gap s out
  shows trace-fri-header-tied-low-candidate-available s out
  using assms
  unfolding trace-fri-header-tied-low-candidate-gap-def
    trace-fri-header-tied-low-candidate-available-def
  by blast

lemma trace-fri-header-tied-low-candidate-gap-imp-transfer-gap:
  assumes trace-fri-header-tied-low-candidate-gap s out
  shows trace-fri-header-tied-candidate-transfer-gap s out
proof -
  from assms obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as rest trace-openings
    trace-table where
  gap: trace-fri-reachable-header-tie-gap s out
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  and header:
    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest
  and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  and candidate:
    partial-trace-table-candidate trace-table trace-openings
  and low: trace-table-low-degree trace-table
  unfolding trace-fri-header-tied-low-candidate-gap-def by blast
show ?thesis
  by (rule trace-fri-header-candidate-low-imp-transfer-gap
    [OF gap fri-openings header partial candidate low])
qed

lemma trace-fri-header-tied-candidate-available-gap-imp-low-candidate-gap:
  assumes trace-fri-header-tied-candidate-available-gap s out
  shows trace-fri-header-tied-low-candidate-gap s out

```

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as rest trace-openings
trace-table **where**

gap: trace-fri-reachable-header-tie-gap s out

and *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers

and *header:*

verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest

and *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings

and *candidate:*

partial-trace-table-candidate trace-table trace-openings

unfolding *trace-fri-header-tied-candidate-available-gap-def* **by** *blast*

show *?thesis*

proof (*cases trace-table-low-degree trace-table*)

case *True*

show *?thesis*

unfolding *trace-fri-header-tied-low-candidate-gap-def*

by (*intro exI conjI*)

(*rule gap, rule fri-openings, rule header, rule partial,*
rule candidate, rule True)

next

case *False*

have *False*

by (*rule trace-fri-header-candidate-non-low-contradicts-header-tie-gap*
[OF gap fri-openings header partial candidate False])

then show *?thesis* **by** *simp*

qed

qed

lemma *wp-trace-fri-header-tied-candidate-available-gap-bound-from-low-candidate-gap:*

assumes *low-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-low-candidate-gap s*) $s \leq T$

shows

wp-event verify-monad

(*trace-fri-header-tied-candidate-available-gap s*) $s \leq T$

proof –

have *wp-event verify-monad*

(*trace-fri-header-tied-candidate-available-gap s*) $s \leq$

wp-event verify-monad

(*trace-fri-header-tied-low-candidate-gap s*) s

by (rule wp-event-mono)
 (rule trace-fri-header-tied-candidate-available-gap-imp-low-candidate-gap)
 then show ?thesis
 by (rule order-trans[OF low-gap-bound])
 qed

lemma wp-trace-fri-reachable-header-tie-gap-bound-from-merkle-and-low-candidate-gap:
assumes merkle-bound:
 wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq M$
and low-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-low-candidate-gap s) $s \leq T$
shows
 wp-event verify-monad
 (trace-fri-reachable-header-tie-gap s) $s \leq M + T$
proof –
have split-bound:
 wp-event verify-monad
 (trace-fri-reachable-header-tie-gap s) $s \leq$
 wp-event verify-monad
 ($\lambda out. \text{partial-merkle-inconsistency-bad } s \text{ out} \vee$
 trace-fri-header-tied-candidate-available-gap s out) s
by (rule wp-event-mono-on-support)
 (rule
 trace-fri-reachable-header-tie-gap-imp-merkle-or-candidate-available-on-support)
also have ... $\leq$
 wp-event verify-monad (partial-merkle-inconsistency-bad s) s +
 wp-event verify-monad
 (trace-fri-header-tied-candidate-available-gap s) s
by (rule wp-event-union-bound)
also have ... $\leq M + T$
by (rule add-mono[OF merkle-bound])
 (rule
 wp-trace-fri-header-tied-candidate-available-gap-bound-from-low-candidate-gap
 [OF low-gap-bound])
finally show ?thesis .
 qed

lemma trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-low-candidate-gap:
assumes header-bound:
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-partial-candidate s) $s \leq Ht$
and merkle-bound:
 wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq Mt$
and low-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-low-candidate-gap s) $s \leq Tt$
and total-bound: $Ht + (Mt + Tt) \leq \text{trace-fri-error}$
shows trace-fri-reachable-partial-candidate-reduction-assumption s

proof –
have *gap-bound*:
 wp-event verify-monad
 (*trace-fri-reachable-header-tie-gap s*) $s \leq Mt + Tt$
by (*rule*
 wp-trace-fri-reachable-header-tie-gap-bound-from-merkle-and-low-candidate-gap
 [*OF merkle-bound low-gap-bound*])
have *reachable-bound*:
 wp-event verify-monad
 (*trace-fri-bad-with-reachable-partial-candidate s*) $s \leq$
 $Ht + (Mt + Tt)$
by (*rule wp-trace-fri-reachable-bound-from-header-tied-and-gap*
 [*OF header-bound gap-bound*])
show *?thesis*
 unfolding *trace-fri-reachable-partial-candidate-reduction-assumption-def*
 by (*rule order-trans*[*OF reachable-bound total-bound*])
qed

lemma *composition-fri-verifier-tied-candidate-transfer-gapD*:
assumes *composition-fri-verifier-tied-candidate-transfer-gap s out*
shows
 composition-fri-bad-with-reachable-partial-candidate s out
 composition-fri-verifier-tied-misaligned-candidate-classification s out

proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
 composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers fr as trace-openings
 composition-openings trace-table composition-table **where**
 bad: composition-fri-bad-with-reachable-partial-candidate s out
and *fri-openings*:
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
and *aligned*:
 accepted-with-partial-initial-openings-aligned s out fr trace-roots
 trace-final as fri-dg composition-roots composition-final
 fri-query-idxs trace-openings composition-openings
and *trace-candidate*:
 partial-trace-table-candidate trace-table trace-openings
and *composition-candidate*:
 partial-composition-table-candidate composition-table
 composition-openings
and *classified*:
 $\neg$ (*trace-table-low-degree trace-table* $\wedge$
 $\neg$ *composition-table-low-degree maxDegree composition-table*)
unfolding *composition-fri-verifier-tied-candidate-transfer-gap-def*
by *blast*
show *composition-fri-bad-with-reachable-partial-candidate s out*

by (rule bad)
 show
 composition-fri-verifier-tied-misaligned-candidate-classification s out
 unfolding
 composition-fri-verifier-tied-misaligned-candidate-classification-def
 by (intro exI conjI)
 (rule fri-openings, rule aligned, rule trace-candidate,
 rule composition-candidate, rule classified)
 qed

lemma composition-fri-verifier-tied-candidate-transfer-gap-imp-accepted:
 assumes composition-fri-verifier-tied-candidate-transfer-gap s out
 shows accepted out
 by (rule composition-fri-bad-with-reachable-partial-candidate-imp-accepted)
 (rule composition-fri-verifier-tied-candidate-transfer-gapD(1)[OF assms])

lemma wp-composition-fri-verifier-tied-candidate-transfer-gap-bound-from-misaligned-classification:
 assumes classification-bound:
 wp-event verify-monad
 (composition-fri-verifier-tied-misaligned-candidate-classification s)
 $s \leq T$
 shows
 wp-event verify-monad
 (composition-fri-verifier-tied-candidate-transfer-gap s) $s \leq T$
 proof –
 have wp-event verify-monad
 (composition-fri-verifier-tied-candidate-transfer-gap s) $s \leq$
 wp-event verify-monad
 (composition-fri-verifier-tied-misaligned-candidate-classification s)
 s
 by (rule wp-event-mono)
 (rule composition-fri-verifier-tied-candidate-transfer-gapD(2))
 then show ?thesis
 by (rule order-trans[OF - classification-bound])
 qed

lemma composition-fri-verifier-tied-transcript-consistent-misaligned-classification-imp-misaligned-classification:
 assumes
 composition-fri-verifier-tied-transcript-consistent-misaligned-classification
 s out
 shows composition-fri-verifier-tied-misaligned-candidate-classification
 s out
 proof –
 from assms obtain trace-roots trace-bs trace-final fri-dg
 composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers fr as rest trace-openings
 composition-openings trace-table composition-table **where**
 fri-openings:
 accepted-fri-opening-transcript s out trace-roots trace-bs

```

    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
and partial:
    accepted-with-partial-initial-openings-aligned-transcript-consistent s
    out fr trace-roots trace-final as fri-dg composition-roots
    composition-final fri-query-idxs trace-openings composition-openings
and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
and classified:
    ¬ (trace-table-low-degree trace-table ∧
    ¬ composition-table-low-degree maxDegree composition-table)
unfolding
    composition-fri-verifier-tied-transcript-consistent-misaligned-classification-def
by blast
have aligned:
    accepted-with-partial-initial-openings-aligned s out fr trace-roots
    trace-final as fri-dg composition-roots composition-final fri-query-idxs
    trace-openings composition-openings
using partial
by (blast dest:
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    accepted-with-partial-initial-openings-aligned-consistent-imp-aligned)
show ?thesis
unfolding
    composition-fri-verifier-tied-misaligned-candidate-classification-def
by (intro exI conjI)
    (rule fri-openings, rule aligned, rule trace-candidate,
    rule composition-candidate, rule classified)
qed

lemma composition-fri-verifier-tied-transcript-consistent-misaligned-classification-imp-aligned-bad-event:
assumes false-statement: ¬ exists-valid-trace
and classification:
    composition-fri-verifier-tied-transcript-consistent-misaligned-classification
    s out
shows
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
    s out
proof –
from classification obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as rest trace-openings
    composition-openings trace-table composition-table where
    fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs

```

```

    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
and header:
    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest
and partial:
    accepted-with-partial-initial-openings-aligned-transcript-consistent s
    out fr trace-roots trace-final as fri-dg composition-roots
    composition-final fri-query-idxs trace-openings composition-openings
and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
unfolding
    composition-fri-verifier-tied-transcript-consistent-misaligned-classification-def
by blast
show ?thesis
by (rule
    accepted-verifier-tied-aligned-transcript-partial-candidate-partition
    [OF false-statement fri-openings partial header trace-candidate
    composition-candidate])
qed

lemma wp-composition-fri-verifier-tied-transcript-consistent-misaligned-classification-bound-from-aligned-bad-event:
assumes false-statement:  $\neg$  exists-valid-trace
and aligned-bound:
    wp-event verify-monad
    (soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
    s)  $s \leq T$ 
shows
    wp-event verify-monad
    (composition-fri-verifier-tied-transcript-consistent-misaligned-classification
    s)  $s \leq T$ 
proof –
have wp-event verify-monad
    (composition-fri-verifier-tied-transcript-consistent-misaligned-classification
    s)  $s \leq$ 
    wp-event verify-monad
    (soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
    s) s
by (rule wp-event-mono)
    (rule
    composition-fri-verifier-tied-transcript-consistent-misaligned-classification-imp-aligned-bad-event
    [OF false-statement])
then show ?thesis
by (rule order-trans[OF - aligned-bound])
qed

```

lemma *composition-fri-verifier-tied-transcript-consistent-misaligned-classification-imp-cover*:
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *classification*:
composition-fri-verifier-tied-transcript-consistent-misaligned-classification
s out
shows
composition-fri-verifier-tied-transcript-consistent-classification-cover
s out
proof –
from *classification* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as rest trace-openings
composition-openings trace-table composition-table **where**
fri-openings:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *header*:
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
and *partial*:
accepted-with-partial-initial-openings-aligned-transcript-consistent s
out fr trace-roots trace-final as fri-dg composition-roots
composition-final fri-query-idxs trace-openings composition-openings
and *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
and *composition-candidate*:
partial-composition-table-candidate composition-table
composition-openings
and *classified*:
 $\neg$ (*trace-table-low-degree trace-table* $\wedge$
 $\neg$ *composition-table-low-degree maxDegree composition-table*)
unfolding
composition-fri-verifier-tied-transcript-consistent-misaligned-classification-def
by *blast*
show *?thesis*
proof (*cases trace-table-low-degree trace-table*)
case *False*
have *aligned*:
accepted-with-partial-initial-openings-aligned-consistent s out fr
trace-roots trace-final as fri-dg composition-roots
composition-final fri-query-idxs trace-openings composition-openings
by (*rule*
accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
[*OF partial*])
have *aligned0*:
accepted-with-partial-initial-openings-aligned s out fr trace-roots

```

    trace-final as fri-dg composition-roots composition-final
    fri-query-idxs trace-openings composition-openings
  by (rule
    accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
    [OF aligned])
  have trace-partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  by (rule accepted-with-partial-initial-openings-aligned-shapes(3)
    [OF aligned0])
  have trace-fri-bad-with-header-tied-partial-candidate s out
    unfolding trace-fri-bad-with-header-tied-partial-candidate-def
  by (intro exI conjI)
    (rule fri-openings, rule header, rule trace-partial,
    rule trace-candidate, rule False)
  then show ?thesis
    unfolding
    composition-fri-verifier-tied-transcript-consistent-classification-cover-def
  by blast
next
  case True
  note trace-low = True
  have composition-low:
    composition-table-low-degree maxDegree composition-table
    using classified trace-low by blast
  show ?thesis
  proof (cases all-queries-consistent trace-table composition-table as)
    case False
    have query-bad-with-aligned-transcript-partial-candidates s out
      unfolding query-bad-with-aligned-transcript-partial-candidates-def
      using partial trace-candidate composition-candidate trace-low
      composition-low False
    by blast
    then show ?thesis
      unfolding
      composition-fri-verifier-tied-transcript-consistent-classification-cover-def
    by blast
  next
    case True
    note all-queries = True
    from trace-low obtain f where
      deg-f: degree f < clength
      and trace-table-eq: trace-table = map (poly f) eval-domain
      unfolding trace-table-low-degree-def by blast
    have violated: violated-constraints f ≠ {}
      by (rule false-statement-violated-constraints
        [OF false-statement deg-f])
    have aligned:
      accepted-with-partial-initial-openings-aligned-consistent s out fr

```

```

    trace-roots trace-final as fri-dg composition-roots
    composition-final fri-query-idxs trace-openings
    composition-openings
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
  have composition-bad-with-partial-candidates s out
  unfolding composition-bad-with-partial-candidates-def
  using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
    [OF aligned]
    trace-candidate composition-candidate deg-f trace-table-eq
    violated composition-low all-queries
  by blast
  then show ?thesis
  unfolding
    composition-fri-verifier-tied-transcript-consistent-classification-cover-def
  by blast
qed
qed
qed

lemma wp-composition-fri-verifier-tied-transcript-consistent-classification-cover-union-bound:
  assumes comp-bound:
    wp-event verify-monad
      (composition-bad-with-partial-candidates s)  $s \leq C$ 
  and trace-bound:
    wp-event verify-monad
      (trace-fri-bad-with-header-tied-partial-candidate s)  $s \leq Ft$ 
  and query-bound:
    wp-event verify-monad
      (query-bad-with-aligned-transcript-partial-candidates s)  $s \leq Q$ 
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-transcript-consistent-classification-cover
        s)  $s \leq C + Ft + Q$ 
proof -
  have wp-event verify-monad
    (composition-fri-verifier-tied-transcript-consistent-classification-cover
      s)  $s \leq$ 
  wp-event verify-monad
    (composition-bad-with-partial-candidates s)  $s +$ 
  wp-event verify-monad
    (trace-fri-bad-with-header-tied-partial-candidate s)  $s +$ 
  wp-event verify-monad
    (query-bad-with-aligned-transcript-partial-candidates s)  $s$ 
proof -
  have wp-event verify-monad
    (composition-fri-verifier-tied-transcript-consistent-classification-cover
      s)  $s \leq$ 

```

```

wp-event verify-monad
  (composition-bad-with-partial-candidates s) s +
wp-event verify-monad
  (λout.
    trace-fri-bad-with-header-tied-partial-candidate s out ∨
    query-bad-with-aligned-transcript-partial-candidates s out) s
unfolding
  composition-fri-verifier-tied-transcript-consistent-classification-cover-def
by (rule wp-event-union-bound)
also have ... ≤
  wp-event verify-monad
    (composition-bad-with-partial-candidates s) s +
  (wp-event verify-monad
    (trace-fri-bad-with-header-tied-partial-candidate s) s +
  wp-event verify-monad
    (query-bad-with-aligned-transcript-partial-candidates s) s)
by (rule add-mono[OF order-refl wp-event-union-bound])
finally show ?thesis by (simp add: algebra-simps)
qed
also have ... ≤ C + Ft + Q
by (intro add-mono comp-bound trace-bound query-bound)
finally show ?thesis .
qed

lemma wp-composition-fri-verifier-tied-transcript-consistent-misaligned-classification-bound-from-cover-components:
assumes false-statement: ¬ exists-valid-trace
and comp-bound:
  wp-event verify-monad
    (composition-bad-with-partial-candidates s) s ≤ C
and trace-bound:
  wp-event verify-monad
    (trace-fri-bad-with-header-tied-partial-candidate s) s ≤ Ft
and query-bound:
  wp-event verify-monad
    (query-bad-with-aligned-transcript-partial-candidates s) s ≤ Q
shows
  wp-event verify-monad
    (composition-fri-verifier-tied-transcript-consistent-misaligned-classification
      s) s ≤ C + Ft + Q
proof –
have wp-event verify-monad
  (composition-fri-verifier-tied-transcript-consistent-misaligned-classification
    s) s ≤
wp-event verify-monad
  (composition-fri-verifier-tied-transcript-consistent-classification-cover
    s) s
by (rule wp-event-mono)
  (rule
    composition-fri-verifier-tied-transcript-consistent-misaligned-classification-imp-cover

```


$[OF \text{ false-statement}]$)
also have $\dots \leq C + Ft + Q$
by (rule
 $wp\text{-composition-fri-verifier-tied-transcript-consistent-classification-cover-union-bound}$
 $[OF \text{ comp-bound trace-bound query-bound}]$)
finally show ?thesis .
qed

lemma *composition-fri-verifier-tied-candidate-available-gap-imp-empty-or-merkle-or-transcript-consistent-classification*
assumes *available*:

$composition\text{-fri-verifier-tied-candidate-available-gap } s \text{ out}$

shows

$composition\text{-fri-reachable-verifier-tie-empty-header-gap } s \text{ out} \vee$

$partial\text{-merkle-inconsistency-bad } s \text{ out} \vee$

$composition\text{-fri-verifier-tied-transcript-consistent-misaligned-classification}$

$s \text{ out}$

proof —

from *available* **obtain** $trace\text{-roots } trace\text{-bs } trace\text{-final } fri\text{-dg}$

$composition\text{-roots } composition\text{-bs } composition\text{-final } fri\text{-query-idxs}$

$trace\text{-round-layers } composition\text{-round-layers } fr \text{ as } trace\text{-openings}$

$composition\text{-openings } trace\text{-table } composition\text{-table}$ **where**

gap : $composition\text{-fri-reachable-verifier-tie-gap } s \text{ out}$

and $fri\text{-openings}$:

$accepted\text{-fri-opening-transcript } s \text{ out } trace\text{-roots } trace\text{-bs}$

$trace\text{-final } fri\text{-dg } composition\text{-roots } composition\text{-bs}$

$composition\text{-final } fri\text{-query-idxs } trace\text{-round-layers}$

$composition\text{-round-layers}$

and $aligned$:

$accepted\text{-with-partial-initial-openings-aligned } s \text{ out } fr \text{ } trace\text{-roots}$

$trace\text{-final } fr \text{ } fri\text{-dg } composition\text{-roots } composition\text{-final}$

$fri\text{-query-idxs } trace\text{-openings } composition\text{-openings}$

and $trace\text{-candidate}$:

$partial\text{-trace-table-candidate } trace\text{-table } trace\text{-openings}$

and $composition\text{-candidate}$:

$partial\text{-composition-table-candidate } composition\text{-table}$

$composition\text{-openings}$

unfolding $composition\text{-fri-verifier-tied-candidate-available-gap-def}$

by *blast*

from $fri\text{-openings}$ **obtain** $result \text{ final-state}$ **where** $out\text{-eq}$:

$out = Some (result, final\text{-state})$

unfolding $accepted\text{-fri-opening-transcript-def}$ **by** *blast*

show ?thesis

proof (cases $composition\text{-roots} = []$)

case *True*

have $composition\text{-fri-reachable-verifier-tie-empty-header-gap } s \text{ out}$

by (rule $composition\text{-fri-reachable-verifier-tie-empty-header-gapI}$

$[OF \text{ gap } fri\text{-openings } True]$)

then show ?thesis **by** *simp*

next

```

case comp-nonempty: False
show ?thesis
proof (cases partial-merkle-inconsistency-bad s out)
  case True
  then show ?thesis by simp
next
  case no-merkle: False
  from accepted-fri-opening-transcript-obtains-aligned-partial-openings
    [OF fri-openings[unfolded out-eq] comp-nonempty]
  obtain fr' as' rest' trace-openings' composition-openings'
  where header':
    verifier-header-transcript s fr' trace-roots trace-final as' fri-dg
      composition-roots composition-final rest'
  and partial':
    accepted-with-partial-initial-openings-aligned-transcript-consistent s
      (Some (result, final-state)) fr' trace-roots trace-final as'
      fri-dg composition-roots composition-final fri-query-idxs
      trace-openings' composition-openings'
  by blast
have aligned':
  accepted-with-partial-initial-openings-aligned s out fr'
    trace-roots trace-final as' fri-dg composition-roots
    composition-final fri-query-idxs trace-openings'
    composition-openings'
  using partial'[folded out-eq]
  by (blast dest:
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    accepted-with-partial-initial-openings-aligned-consistent-imp-aligned)
obtain trace-table' composition-table'
where trace-candidate':
  partial-trace-table-candidate trace-table' trace-openings'
  and composition-candidate':
  partial-composition-table-candidate composition-table'
    composition-openings'
  using
    accepted-with-partial-initial-openings-aligned-candidates-if-no-partial-merkle-bad
    [OF aligned'[unfolded out-eq] no-merkle[unfolded out-eq]]
  by blast
have classified:
   $\neg$  (trace-table-low-degree trace-table'  $\wedge$ 
    composition-table-low-degree maxDegree composition-table')
proof
  assume bad-class:
    trace-table-low-degree trace-table'  $\wedge$ 
     $\neg$  composition-table-low-degree maxDegree composition-table'
  have composition-fri-bad-with-verifier-tied-partial-candidate s out
  unfolding composition-fri-bad-with-verifier-tied-partial-candidate-def
  by (intro exI conjI)
    (rule fri-openings, rule aligned', rule trace-candidate',

```

```

      rule composition-candidate', rule bad-class[THEN conjunct1],
      rule bad-class[THEN conjunct2])
  then show False
    using gap
    unfolding composition-fri-reachable-verifier-tie-gap-def by simp
qed
have
  composition-fri-verifier-tied-transcript-consistent-misaligned-classification
    s out
  unfolding
    composition-fri-verifier-tied-transcript-consistent-misaligned-classification-def
  by (intro exI conjI)
    (rule fri-openings, rule header'[folded out-eq],
     rule partial'[folded out-eq], rule trace-candidate',
     rule composition-candidate', rule classified)
  then show ?thesis by simp
qed
qed
qed

lemma wp-composition-fri-verifier-tied-candidate-available-gap-bound-from-empty-merkle-and-transcript-consistent-c
  assumes empty-bound:
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-empty-header-gap s) s ≤ E
  and merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M
  and classification-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-transcript-consistent-misaligned-classification
        s) s ≤ T
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-candidate-available-gap s) s ≤
      E + (M + T)
proof -
  have wp-event verify-monad
    (composition-fri-verifier-tied-candidate-available-gap s) s ≤
    wp-event verify-monad
      (λout.
        composition-fri-reachable-verifier-tie-empty-header-gap s out ∨
        partial-merkle-inconsistency-bad s out ∨
        composition-fri-verifier-tied-transcript-consistent-misaligned-classification
          s out) s
  by (rule wp-event-mono)
    (rule
      composition-fri-verifier-tied-candidate-available-gap-imp-empty-or-merkle-or-transcript-consistent-classification)
  also have ... ≤
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-empty-header-gap s) s +

```

wp-event verify-monad
 (λout.
 partial-merkle-inconsistency-bad s out ∨
 composition-fri-verifier-tied-transcript-consistent-misaligned-classification
 s out) *s*
 by (rule *wp-event-union-bound*)
 also have ... ≤
 wp-event verify-monad
 (*composition-fri-reachable-verifier-tie-empty-header-gap s*) *s* +
 (*wp-event verify-monad* (*partial-merkle-inconsistency-bad s*) *s* +
 wp-event verify-monad
 (*composition-fri-verifier-tied-transcript-consistent-misaligned-classification*
 s) *s*)
 by (rule *add-mono[OF order-refl wp-event-union-bound]*)
 also have ... ≤ *E* + (*M* + *T*)
 by (intro *add-mono empty-bound merkle-bound classification-bound*)
 finally show ?thesis .
 qed

lemma *wp-composition-fri-reachable-verifier-tie-gap-bound-from-empty-merkle-and-candidate-available:*

assumes *empty-bound:*

wp-event verify-monad
 (*composition-fri-reachable-verifier-tie-empty-header-gap s*) *s* ≤ *E*

and *merkle-bound:*

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) *s* ≤ *M*

and *available-bound:*

wp-event verify-monad
 (*composition-fri-verifier-tied-candidate-available-gap s*) *s* ≤ *T*

shows

wp-event verify-monad
 (*composition-fri-reachable-verifier-tie-gap s*) *s* ≤ *E* + (*M* + *T*)

proof –

have *split-bound:*

wp-event verify-monad
 (*composition-fri-reachable-verifier-tie-gap s*) *s* ≤
 wp-event verify-monad
 (λout.
 composition-fri-reachable-verifier-tie-empty-header-gap s out ∨
 partial-merkle-inconsistency-bad s out ∨
 composition-fri-verifier-tied-candidate-available-gap s out) *s*

by (rule *wp-event-mono-on-support*)

(rule
composition-fri-reachable-verifier-tie-gap-imp-empty-or-merkle-or-candidate-available-on-support)

also have ... ≤

wp-event verify-monad
 (*composition-fri-reachable-verifier-tie-empty-header-gap s*) *s* +
 wp-event verify-monad
 (λout.
 partial-merkle-inconsistency-bad s out ∨

composition-fri-verifier-tied-candidate-available-gap s out) s
 by (rule wp-event-union-bound)
 also have ... $\leq$
 wp-event verify-monad
 (composition-fri-reachable-verifier-tie-empty-header-gap s) s +
 (wp-event verify-monad (partial-merkle-inconsistency-bad s) s) +
 wp-event verify-monad
 (composition-fri-verifier-tied-candidate-available-gap s) s)
 by (rule add-mono[OF order-refl wp-event-union-bound])
 also have ... $\leq E + (M + T)$
 by (intro add-mono empty-bound merkle-bound available-bound)
 finally show ?thesis .
 qed

lemma composition-fri-reachable-partial-candidate-reduction-from-split-residual-bounds-with-candidate-available:

fixes R A P M N S G E Q T :: prob

assumes sampled-bound:

wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s $\leq R$

and slot-gap-bound:

wp-event verify-monad
 (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
 s) s $\leq A$

and verifier-partial-merkle-bound:

wp-event verify-monad (partial-merkle-inconsistency-bad s) s $\leq P$

and missing-bound:

wp-event verify-monad
 (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
 s $\leq M$

and next-gap-bound:

wp-event verify-monad
 (composition-fri-verifier-tied-next-value-replay-gap s) s $\leq N$

and successor-gap-bound:

wp-event verify-monad
 (composition-fri-verifier-tied-successor-replay-gap s) s $\leq S$

and final-gap-bound:

wp-event verify-monad
 (composition-fri-verifier-tied-final-value-replay-gap s) s $\leq G$

and empty-bound:

wp-event verify-monad
 (composition-fri-reachable-verifier-tie-empty-header-gap s) s $\leq E$

and gap-partial-merkle-bound:

wp-event verify-monad (partial-merkle-inconsistency-bad s) s $\leq Q$

and candidate-available-bound:

wp-event verify-monad
 (composition-fri-verifier-tied-candidate-available-gap s) s $\leq T$

and total-bound:

$R + (((P + M) + N + S + G + P) + A) + (E + (Q + T))$
 $\leq$ composition-fri-error

shows *composition-fri-reachable-partial-candidate-reduction-assumption s*
proof –
have *verifier-bound*:
 wp-event verify-monad
 (*composition-fri-bad-with-verifier-tied-partial-candidate s*) *s*
 $\leq R + (((P + M) + N + S + G + P) + A)$
by (*rule*
 wp-composition-fri-verifier-tied-bound-from-sampled-slot-recorded-chunk-gap-and-residuals
 [*OF sampled-bound slot-gap-bound verifier-partial-merkle-bound*
 missing-bound next-gap-bound successor-gap-bound final-gap-bound])
have *gap-bound*:
 wp-event verify-monad
 (*composition-fri-reachable-verifier-tie-gap s*) *s* $\leq$
 $E + (Q + T)$
by (*rule*
 wp-composition-fri-reachable-verifier-tie-gap-bound-from-empty-merkle-and-candidate-available
 [*OF empty-bound gap-partial-merkle-bound candidate-available-bound*])
show *?thesis*
by (*rule*
 composition-fri-reachable-partial-candidate-reduction-from-verifier-tied-and-gap
 [*OF verifier-bound gap-bound total-bound*])
qed

lemma *composition-fri-reachable-partial-candidate-reduction-from-split-residual-bounds-with-candidate-available-and-*

fixes *R A P N S G E Q T :: prob*

assumes *sampled-bound*:

wp-event verify-monad
 (*composition-fri-bad-with-verifier-tied-sampled-layer-chain s*) *s* $\leq R$

and *slot-gap-bound*:

wp-event verify-monad
 (*composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap*
 s) *s* $\leq A$

and *verifier-partial-merkle-bound*:

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) *s* $\leq P$

and *next-gap-bound*:

wp-event verify-monad
 (*composition-fri-verifier-tied-next-value-replay-gap s*) *s* $\leq N$

and *successor-gap-bound*:

wp-event verify-monad
 (*composition-fri-verifier-tied-successor-replay-gap s*) *s* $\leq S$

and *final-gap-bound*:

wp-event verify-monad
 (*composition-fri-verifier-tied-final-value-replay-gap s*) *s* $\leq G$

and *empty-bound*:

wp-event verify-monad
 (*composition-fri-reachable-verifier-tie-empty-header-gap s*) *s* $\leq E$

and *gap-partial-merkle-bound*:

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) *s* $\leq Q$

and *candidate-available-bound*:

```

    wp-event verify-monad
      (composition-fri-verifier-tied-candidate-available-gap s) s ≤ T
  and total-bound:
    R + (((P + 0) + N + S + G + P) + A) + (E + (Q + T))
      ≤ composition-fri-error
  shows composition-fri-reachable-partial-candidate-reduction-assumption s
  by (rule
    composition-fri-reachable-partial-candidate-reduction-from-split-residual-bounds-with-candidate-available
      [where M = 0])
    (rule sampled-bound,
     rule slot-gap-bound,
     rule verifier-partial-merkle-bound,
     rule wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero,
     rule next-gap-bound,
     rule successor-gap-bound,
     rule final-gap-bound,
     rule empty-bound,
     rule gap-partial-merkle-bound,
     rule candidate-available-bound,
     rule total-bound)

end

end

theory Soundness-FRI-Empty-Header-Reduction
  imports Soundness-FRI-Trace-Residuals
begin

Named reduction interface for the empty-composition-header trace FRI branch. This
keeps the large reachable-FRI theory stable while giving the final route the same active-
bound shape for all three FRI components.

context soundness
begin

definition trace-fri-empty-header-reduction
  :: ('f, 'a) protocol-channel-scheme ⇒ bool
where
  trace-fri-empty-header-reduction s ⟷
    wp-event verify-monad
      (trace-fri-bad-with-empty-composition-header-candidates s) s ≤
      trace-fri-error

lemma trace-fri-empty-header-reductionD:
  assumes trace-fri-empty-header-reduction s
  shows
    wp-event verify-monad
      (trace-fri-bad-with-empty-composition-header-candidates s) s ≤

```

```

    trace-fri-error
using assms unfolding trace-fri-empty-header-reduction-def by simp

lemma trace-fri-empty-header-reduction-staged-bound:
  assumes reductions:
     $\bigwedge \text{data attacker-state.}$ 
     $\text{Some (data, attacker-state)} \in$ 
       $\text{set-dist (execute (checked-staged-transcript-program } A)$ 
         $\text{adversary-initial-state)} \implies$ 
     $\text{trace-fri-empty-header-reduction}$ 
       $(\text{verifier-state-from-adversary attacker-state}$ 
         $(\text{staged-proof-transcript data}))$ 
  shows
     $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$ 
       $(\text{staged-security-with-data-state-verifier-event}$ 
         $\text{trace-fri-bad-with-empty-composition-header-candidates})$ 
     $\text{adversary-initial-state} \leq \text{trace-fri-error}$ 
  by (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    (simp-all add: trace-fri-bad-with-empty-composition-header-candidates-not-None
      trace-fri-empty-header-reductionD reductions)

lemma trace-fri-empty-header-reduction-from-reachable-reduction:
  assumes trace-fri-reachable-partial-candidate-reduction-assumption s
  shows trace-fri-empty-header-reduction s
proof –
  have reachable:
     $\text{wp-event verify-monad}$ 
       $(\text{trace-fri-bad-with-reachable-partial-candidate } s) \ s \leq$ 
       $\text{trace-fri-error}$ 
  using assms
  unfolding trace-fri-reachable-partial-candidate-reduction-assumption-def
  by simp
show ?thesis
  unfolding trace-fri-empty-header-reduction-def
  by (rule order-trans[OF - reachable])
    (rule wp-event-mono,
      rule
        trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate)
qed

end

end

theory Soundness-FRI-Active-Reductions
imports
  Soundness-FRI-Candidate-Transfer-Support
  Soundness-FRI-Empty-Header-Reduction

```


begin

Staged packaging for the active FRI reductions used by the public route. This layer does not add assumptions; it names the exact verifier-local reductions consumed by the public-route bounds.

Internal symbolic FRI soundness interface for the public staged theorem.

The trace side is intentionally stated in terms of the verifier-consumed header-tied branch and the empty-composition-header branch. The composition side is stated in terms of the verifier-tied branch consumed by the active public route.

locale *soundness-fri* = *soundness* +
assumes *trace-fri-active-reductions*:
 $\bigwedge A$ *data attacker-state*.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 trace-fri-header-tied-reduction
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\wedge$
 trace-fri-empty-header-reduction
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
assumes *composition-fri-active-reduction*:
 $\bigwedge A$ *data attacker-state*.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 composition-fri-verifier-tied-reduction
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))

context *soundness*

begin

lemma *active-composition-candidate-transfer-bound-from-classification-bound*:

assumes *classification-bound*:
 $\bigwedge$ *data attacker-state*.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 wp-event verify-monad
 (*composition-fri-verifier-tied-misaligned-candidate-classification*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq$
 T data attacker-state
and *support*:
 Some (data, attacker-state) ∈

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state})$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-candidate-transfer-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $T \text{ data attacker-state}$
by (rule
 $\text{wp-composition-fri-verifier-tied-candidate-transfer-gap-bound-from-misaligned-classification}$)
 $(\text{rule classification-bound}[OF \text{ support}])$

lemma *active-composition-candidate-available-bound-from-transcript-consistent-classification-bound:*

assumes *empty-bound:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-reachable-verifier-tie-empty-header-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $E \text{ data attacker-state}$
and *merkle-bound:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $M \text{ data attacker-state}$
and *classification-bound:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-transcript-consistent-misaligned-classification}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data})) \leq$
 $T \text{ data attacker-state}$
and support:
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})}$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-candidate-available-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $E \text{ data attacker-state} + (M \text{ data attacker-state} +$
 $T \text{ data attacker-state})$
by (rule
 $\text{wp-composition-fri-verifier-tied-candidate-available-gap-bound-from-empty-merkle-and-transcript-consistent-classi}$
 $(\text{rule empty-bound[OF support], rule merkle-bound[OF support],}$
 $\text{rule classification-bound[OF support]})$

lemma *active-composition-transcript-consistent-classification-bound-from-components:*

assumes *false-statement:* $\neg \text{exists-valid-trace}$
and comp-bound:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-bad-with-partial-candidates}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $C \text{ data attacker-state}$
and trace-fri-bound:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-partial-candidate}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ft \text{ data attacker-state}$
and query-bound:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$

$set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$
 $wp-event \ verify-monad$
 $(query-bad-with-aligned-transcript-partial-candidates$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)))$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)) \leq$
 $Q \ data \ attacker-state$
and support:
 $Some \ (data, \ attacker-state) \in$
 $set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state)$
shows
 $wp-event \ verify-monad$
 $(composition-fri-verifier-tied-transcript-consistent-misaligned-classification$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)))$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)) \leq$
 $C \ data \ attacker-state + Ft \ data \ attacker-state +$
 $Q \ data \ attacker-state$
by (rule
 $wp-composition-fri-verifier-tied-transcript-consistent-misaligned-classification-bound-from-cover-components$
 $[OF \ false-statement])$
 $(rule \ comp-bound[OF \ support], \ rule \ trace-fri-bound[OF \ support],$
 $rule \ query-bound[OF \ support])$

lemma *active-trace-fri-reachable-reduction-from-header-merkle-and-low-candidate-gap:*

fixes $Ht \ Mt \ Tt ::$

$(\text{'f staged-proof-data} \Rightarrow \text{'f protocol-channel} \Rightarrow prob)$

assumes *trace-header-bound:*

$\bigwedge data \ attacker-state.$

$Some \ (data, \ attacker-state) \in$

$set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$

$wp-event \ verify-monad$

$(trace-fri-bad-with-header-tied-partial-candidate$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)))$

$(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)) \leq$

$Ht \ data \ attacker-state$

and *trace-merkle-bound:*

$\bigwedge data \ attacker-state.$

$Some \ (data, \ attacker-state) \in$

$set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$

$wp-event \ verify-monad$

$(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Mt \text{ data attacker-state}$
and $\text{trace-low-gap-bound}$:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-low-candidate-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Tt \text{ data attacker-state}$
and total-bound :
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $Ht \text{ data attacker-state} +$
 $(Mt \text{ data attacker-state} + Tt \text{ data attacker-state}) \leq$
 trace-fri-error
and support :
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})}$
shows
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
by $(\text{rule}$
 $\text{trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-low-candidate-gap})$
 $(\text{rule trace-header-bound}[OF \text{ support}],$
 $\text{rule trace-merkle-bound}[OF \text{ support}],$
 $\text{rule trace-low-gap-bound}[OF \text{ support}],$
 $\text{rule total-bound}[OF \text{ support}])$

definition $\text{active-fri-reductions-for}$ **where**

$\text{active-fri-reductions-for } A \longleftrightarrow$
 $(\forall \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \longrightarrow$
 $\text{trace-fri-header-tied-reduction}$
 $(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data})) \wedge$
 $\text{composition-fri-verifier-tied-reduction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \wedge$
 $\text{trace-fri-empty-header-reduction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$

definition *trace-fri-header-tied-residual-obligations*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $\text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow$
 $\text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-residual-obligations } s \text{ } r\text{-}b \text{ } z\text{-}b \text{ } a\text{-}b \text{ } p\text{-}b \text{ } h\text{-}b \text{ } g\text{-}b \text{ } n\text{-}b \text{ } s\text{-}b \text{ } f\text{-}b$
 $\longleftrightarrow$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-sampled-layer-chain } s) \text{ } s \leq r\text{-}b \wedge$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \text{ } s \leq z\text{-}b \wedge$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap } s) \text{ } s$
 $\leq a\text{-}b \wedge$
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \text{ } s \leq p\text{-}b \wedge$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \text{ } s \leq h\text{-}b \wedge$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-head-value-neq-gap } s) \text{ } s \leq g\text{-}b \wedge$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap } s) \text{ } s \leq n\text{-}b \wedge$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap } s) \text{ } s \leq s\text{-}b \wedge$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-final-value-selected-step-missing-gap } s) \text{ } s$
 $\leq f\text{-}b \wedge$
 $r\text{-}b + (((((p\text{-}b + h\text{-}b) + g\text{-}b) + n\text{-}b + s\text{-}b +$
 $(p\text{-}b + a\text{-}b + f\text{-}b)) + p\text{-}b) + a\text{-}b + z\text{-}b)$
 $\leq \text{trace-fri-error}$

definition *composition-fri-verifier-tied-residual-obligations*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $\text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow$
 $\text{prob} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-residual-obligations } s \text{ } r\text{-}b \text{ } a\text{-}b \text{ } p\text{-}b \text{ } n\text{-}b \text{ } s\text{-}b \text{ } g\text{-}b$
 $\longleftrightarrow$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain } s) \text{ } s$
 $\leq r\text{-}b \wedge$
 $\text{wp-event verify-monad}$

$(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap } s)$
 $s \leq a-b \wedge$
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq p-b \wedge$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap } s) \ s \leq n-b \wedge$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap } s) \ s \leq s-b \wedge$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq g-b \wedge$
 $r-b + (((p-b + a-b) + n-b + s-b + g-b + p-b) + a-b)$
 $\leq \text{composition-fri-error}$

lemma *trace-fri-header-tied-residual-obligations-from-bounds-without-head:*

assumes *sampled:*

$\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-sampled-layer-chain } s) \ s \leq r-b$

and *zero:*

$\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq z-b$

and *slot:*

$\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap } s) \ s$
 $\leq a-b$

and *merkle:*

$\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq p-b$

and *structural:*

$\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq h-b$

and *next-bound:*

$\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq n-b$

and *successor:*

$\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq s-b$

and *final-bound:*

$\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-final-value-selected-step-missing-gap } s) \ s$
 $\leq f-b$

and *total:*

$r-b + (((((p-b + h-b) + p-b) + n-b + s-b +$
 $(p-b + a-b + f-b)) + p-b) + a-b + z-b)$
 $\leq \text{trace-fri-error}$

shows

$\text{trace-fri-header-tied-residual-obligations } s$
 $r-b \ z-b \ a-b \ p-b \ h-b \ p-b \ n-b \ s-b \ f-b$

proof —

have *head:*

$\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-head-value-neq-gap } s) \ s \leq p-b$

by (rule
 wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle
 [OF merkle])
 show ?thesis
 using sampled zero slot merkle structural head next-bound successor
 final-bound total
 unfolding trace-fri-header-tied-residual-obligations-def by blast
 qed

lemma trace-fri-header-tied-residual-obligations-from-bounds-without-head-and-selected-zero:
assumes sampled:
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-sampled-layer-chain s) $s \leq r-b$
and zero:
 wp-event verify-monad
 (trace-fri-header-tied-zero-round-final-obstruction s) $s \leq z-b$
and slot:
 wp-event verify-monad
 (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s) $s \leq a-b$
and merkle:
 wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq p-b$
and structural:
 wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap s) $s \leq h-b$
and next-bound:
 wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap s) $s \leq n-b$
and successor:
 wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap s) $s \leq s-b$
and total:
 $r-b + (((p-b + h-b) + p-b) + n-b + s-b +$
 $(p-b + a-b + 0)) + p-b + a-b + z-b$
 $\leq \text{trace-fri-error}$
shows
 trace-fri-header-tied-residual-obligations s
 r-b z-b a-b p-b h-b p-b n-b s-b 0
by (rule trace-fri-header-tied-residual-obligations-from-bounds-without-head)
 (rule sampled, rule zero, rule slot, rule merkle, rule structural,
 rule next-bound, rule successor,
 rule wp-trace-fri-header-tied-final-value-selected-step-missing-gap-zero,
 rule total)

lemma trace-fri-header-tied-residual-obligations-from-bounds-without-head-slot-and-selected-zero:
assumes sampled:
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-sampled-layer-chain s) $s \leq r-b$
and zero:

$wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq z\text{-b}$
and merkle:
 $wp\text{-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq p\text{-b}$
and structural:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq h\text{-b}$
and next-bound:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq n\text{-b}$
and successor:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq s\text{-b}$
and total:
 $r\text{-b} + (((((p\text{-b} + h\text{-b}) + p\text{-b}) + n\text{-b} + s\text{-b} +$
 $(p\text{-b} + 0 + 0)) + p\text{-b}) + 0 + z\text{-b})$
 $\leq \text{trace-fri-error}$
shows
 $\text{trace-fri-header-tied-residual-obligations } s$
 $r\text{-b } z\text{-b } 0 \ p\text{-b } h\text{-b } p\text{-b } n\text{-b } s\text{-b } 0$
by (rule
 $\text{trace-fri-header-tied-residual-obligations-from-bounds-without-head-and-selected-zero})$
(rule sampled, rule zero,
rule $wp\text{-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero}$,
rule merkle, rule structural, rule next-bound, rule successor, rule total)

lemma $\text{composition-fri-verifier-tied-residual-obligations-from-bounds}$:
assumes sampled:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain } s) \ s$
 $\leq r\text{-b}$
and slot:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap } s)$
 $s \leq a\text{-b}$
and merkle:
 $wp\text{-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq p\text{-b}$
and next-bound:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap } s) \ s \leq n\text{-b}$
and successor:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap } s) \ s \leq s\text{-b}$
and final-bound:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq g\text{-b}$
and total:
 $r\text{-b} + (((p\text{-b} + a\text{-b}) + n\text{-b} + s\text{-b} + g\text{-b} + p\text{-b}) + a\text{-b})$
 $\leq \text{composition-fri-error}$

shows

composition-fri-verifier-tied-residual-obligations s

r-b a-b p-b n-b s-b g-b

using *sampled slot merkle next-bound successor final-bound total*

unfolding *composition-fri-verifier-tied-residual-obligations-def* **by** *blast*

lemma *composition-fri-verifier-tied-residual-obligations-from-bounds-slot-zero:*

assumes *sampled:*

wp-event verify-monad

(composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s

≤ r-b

and *merkle:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ p-b

and *next-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-next-value-replay-gap s) s ≤ n-b

and *successor:*

wp-event verify-monad

(composition-fri-verifier-tied-successor-replay-gap s) s ≤ s-b

and *final-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-final-value-replay-gap s) s ≤ g-b

and *total:*

r-b + ((p-b + 0) + n-b + s-b + g-b + p-b) + 0)

≤ composition-fri-error

shows

composition-fri-verifier-tied-residual-obligations s

r-b 0 p-b n-b s-b g-b

by (*rule composition-fri-verifier-tied-residual-obligations-from-bounds*)

(*rule sampled,*

rule wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero,

rule merkle, rule next-bound, rule successor, rule final-bound,

rule total)

lemma *active-composition-fri-reachable-reduction-from-split-bounds-without-empty-with-candidate-available:*

fixes

Rc Ac Pc Nc Sc Gc Qc Tc ::

('f staged-proof-data ⇒

'f protocol-channel ⇒ prob)

assumes *comp-sampled-bound:*

∧ data attacker-state.

Some (data, attacker-state) ∈

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) ⇒

wp-event verify-monad

(composition-fri-bad-with-verifier-tied-sampled-layer-chain

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state

$(\text{staged-proof-transcript data}) \leq$
 $Rc \text{ data attacker-state}$
and *comp-slot-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ac \text{ data attacker-state}$
and *comp-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Pc \text{ data attacker-state}$
and *comp-next-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Nc \text{ data attacker-state}$
and *comp-successor-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$

$Sc \text{ data attacker-state}$
and $comp\text{-}final\text{-}bound$:
 $\wedge data \text{ attacker-state.}$
 $Some (data, attacker-state) \in$
 $set\text{-}dist (execute (checked\text{-}staged\text{-}transcript\text{-}program A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $wp\text{-}event \text{ verify-monad}$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}final\text{-}value\text{-}replay\text{-}gap$
 $(verifier\text{-}state\text{-}from\text{-}adversary attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript data)))$
 $(verifier\text{-}state\text{-}from\text{-}adversary attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript data)) \leq$
 $Gc \text{ data attacker-state}$
and $comp\text{-}gap\text{-}merkle\text{-}bound$:
 $\wedge data \text{ attacker-state.}$
 $Some (data, attacker-state) \in$
 $set\text{-}dist (execute (checked\text{-}staged\text{-}transcript\text{-}program A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $wp\text{-}event \text{ verify-monad}$
 $(partial\text{-}merkle\text{-}inconsistency\text{-}bad$
 $(verifier\text{-}state\text{-}from\text{-}adversary attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript data)))$
 $(verifier\text{-}state\text{-}from\text{-}adversary attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript data)) \leq$
 $Qc \text{ data attacker-state}$
and $comp\text{-}candidate\text{-}available\text{-}bound$:
 $\wedge data \text{ attacker-state.}$
 $Some (data, attacker-state) \in$
 $set\text{-}dist (execute (checked\text{-}staged\text{-}transcript\text{-}program A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $wp\text{-}event \text{ verify-monad}$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}candidate\text{-}available\text{-}gap$
 $(verifier\text{-}state\text{-}from\text{-}adversary attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript data)))$
 $(verifier\text{-}state\text{-}from\text{-}adversary attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript data)) \leq$
 $Tc \text{ data attacker-state}$
and $comp\text{-}total\text{-}bound$:
 $\wedge data \text{ attacker-state.}$
 $Some (data, attacker-state) \in$
 $set\text{-}dist (execute (checked\text{-}staged\text{-}transcript\text{-}program A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $Rc \text{ data attacker-state} +$
 $((Pc \text{ data attacker-state} + 0) +$
 $Nc \text{ data attacker-state} + Sc \text{ data attacker-state} +$
 $Gc \text{ data attacker-state} + Pc \text{ data attacker-state}) +$
 $Ac \text{ data attacker-state}) +$
 $(Qc \text{ data attacker-state} + Tc \text{ data attacker-state})$
 $\leq composition\text{-}fri\text{-}error$

assumes *support*:
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
shows
composition-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
proof –
let *?s* =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
show *?thesis*
by (rule
composition-fri-reachable-partial-candidate-reduction-from-split-residual-bounds-with-candidate-available-and-rec
[**where** *E = 0*])
(rule *comp-sampled-bound*[*OF support*],
rule *comp-slot-bound*[*OF support*],
rule *comp-merkle-bound*[*OF support*],
rule *comp-next-bound*[*OF support*],
rule *comp-successor-bound*[*OF support*],
rule *comp-final-bound*[*OF support*],
simp add: *wp-composition-fri-reachable-verifier-tie-empty-header-gap-zero*,
rule *comp-gap-merkle-bound*[*OF support*],
rule *comp-candidate-available-bound*[*OF support*],
insert *comp-total-bound*[*OF support*], simp)
qed

definition *active-fri-residual-obligations-for where*
active-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft
Rc Ac Pc Nc Sc Gc ↔
(∀ *data attacker-state*.
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) →
(let *s* =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
in
trace-fri-header-tied-residual-obligations s
(*Rt data attacker-state*) (*Zt data attacker-state*)
(*At data attacker-state*) (*Pt data attacker-state*)
(*Ht data attacker-state*) (*Gt data attacker-state*)
(*Nt data attacker-state*) (*St data attacker-state*)
(*Ft data attacker-state*) ∧
composition-fri-verifier-tied-residual-obligations s
(*Rc data attacker-state*) (*Ac data attacker-state*)
(*Pc data attacker-state*)
(*Nc data attacker-state*) (*Sc data attacker-state*)

$(Gc \text{ data attacker-state}) \wedge$
 $\text{trace-fri-empty-header-reduction } s))$

definition *active-trace-fri-residual-obligations-for* **where**
active-trace-fri-residual-obligations-for $A \text{ Rt Zt At Pt Ht Gt Nt St Ft}$

$\longleftrightarrow$
 $(\forall \text{ data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \longrightarrow$
 $\text{trace-fri-header-tied-residual-obligations}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{Rt data attacker-state}) (\text{Zt data attacker-state})$
 $(\text{At data attacker-state}) (\text{Pt data attacker-state})$
 $(\text{Ht data attacker-state}) (\text{Gt data attacker-state})$
 $(\text{Nt data attacker-state}) (\text{St data attacker-state})$
 $(\text{Ft data attacker-state}))$

definition *active-composition-fri-residual-obligations-for* **where**
active-composition-fri-residual-obligations-for $A \text{ Rc Ac Pc Nc Sc Gc}$

$\longleftrightarrow$
 $(\forall \text{ data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \longrightarrow$
 $\text{composition-fri-verifier-tied-residual-obligations}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{Rc data attacker-state}) (\text{Ac data attacker-state})$
 $(\text{Pc data attacker-state})$
 $(\text{Nc data attacker-state}) (\text{Sc data attacker-state})$
 $(\text{Gc data attacker-state}))$

definition *active-empty-trace-fri-reduction-for* **where**
active-empty-trace-fri-reduction-for $A \longleftrightarrow$

$(\forall \text{ data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \longrightarrow$
 $\text{trace-fri-empty-header-reduction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$

lemma *active-fri-residual-obligations-from-components:*

assumes

active-trace-fri-residual-obligations-for $A \text{ Rt Zt At Pt Ht Gt Nt St Ft}$
active-composition-fri-residual-obligations-for $A \text{ Rc Ac Pc Nc Sc Gc}$
active-empty-trace-fri-reduction-for A

shows *active-fri-residual-obligations-for* $A \text{ Rt Zt At Pt Ht Gt Nt St Ft}$
 $Rc \text{ Ac Pc Nc Sc Gc}$

using *assms*

unfolding *active-fri-residual-obligations-for-def*
active-trace-fri-residual-obligations-for-def
active-composition-fri-residual-obligations-for-def
active-empty-trace-fri-reduction-for-def

by (*simp add: Let-def*)

lemma *active-fri-residual-obligations-trace-component:*

assumes *active-fri-residual-obligations-for* $A \text{ Rt Zt At Pt Ht Gt Nt St Ft}$
 $Rc \text{ Ac Pc Nc Sc Gc}$

shows *active-trace-fri-residual-obligations-for* $A \text{ Rt Zt At Pt Ht Gt Nt St Ft}$

using *assms*

unfolding *active-fri-residual-obligations-for-def*
active-trace-fri-residual-obligations-for-def

by (*simp add: Let-def*)

lemma *active-fri-residual-obligations-composition-component:*

assumes *active-fri-residual-obligations-for* $A \text{ Rt Zt At Pt Ht Gt Nt St Ft}$
 $Rc \text{ Ac Pc Nc Sc Gc}$

shows *active-composition-fri-residual-obligations-for* $A \text{ Rc Ac Pc Nc Sc Gc}$

using *assms*

unfolding *active-fri-residual-obligations-for-def*
active-composition-fri-residual-obligations-for-def

by (*simp add: Let-def*)

lemma *active-fri-residual-obligations-empty-trace-component:*

assumes *active-fri-residual-obligations-for* $A \text{ Rt Zt At Pt Ht Gt Nt St Ft}$
 $Rc \text{ Ac Pc Nc Sc Gc}$

shows *active-empty-trace-fri-reduction-for* A

using *assms*

unfolding *active-fri-residual-obligations-for-def*
active-empty-trace-fri-reduction-for-def

by (*simp add: Let-def*)

lemma *active-trace-fri-residual-obligations-from-bounds-without-head:*

fixes

$Rt \text{ Zt At Pt Ht Nt St Ft} ::$

$(\text{'f staged-proof-data} \Rightarrow$

$\text{'f protocol-channel} \Rightarrow \text{prob})$

assumes *trace-sampled-bound:*

$\bigwedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$

$\text{set-dist (execute (checked-staged-transcript-program } A)$

$\text{adversary-initial-state)} \implies$

$\text{wp-event verify-monad}$

$(\text{trace-fri-bad-with-header-tied-sampled-layer-chain}$

$(\text{verifier-state-from-adversary attacker-state}$

(staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤
 Rt data attacker-state
and trace-zero-bound:
 ∧ data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 wp-event verify-monad
 (trace-fri-header-tied-zero-round-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤
 Zt data attacker-state
and trace-slot-bound:
 ∧ data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 wp-event verify-monad
 (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤
 At data attacker-state
and trace-merkle-bound:
 ∧ data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤
 Pt data attacker-state
and trace-structural-bound:
 ∧ data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ht \text{ data attacker-state}$
and trace-next-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Nt \text{ data attacker-state}$
and $\text{trace-successor-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $St \text{ data attacker-state}$
and $\text{trace-final-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-final-value-selected-step-missing-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ft \text{ data attacker-state}$
and $\text{trace-total-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $Rt \text{ data attacker-state} +$
 $(((((Pt \text{ data attacker-state} + Ht \text{ data attacker-state}) +$
 $Pt \text{ data attacker-state}) +$
 $Nt \text{ data attacker-state} + St \text{ data attacker-state} +$
 $(Pt \text{ data attacker-state} + At \text{ data attacker-state} +$

```

      Ft data attacker-state)) +
      Pt data attacker-state) +
      At data attacker-state + Zt data attacker-state)
    ≤ trace-fri-error
shows active-trace-fri-residual-obligations-for A Rt Zt At Pt Ht Pt
      Nt St Ft
unfolding active-trace-fri-residual-obligations-for-def
proof (intro allI impI)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show trace-fri-header-tied-residual-obligations ?s
  (Rt data attacker-state) (Zt data attacker-state)
  (At data attacker-state) (Pt data attacker-state)
  (Ht data attacker-state) (Pt data attacker-state)
  (Nt data attacker-state) (St data attacker-state)
  (Ft data attacker-state)
by (rule trace-fri-header-tied-residual-obligations-from-bounds-without-head)
  (rule trace-sampled-bound[OF support],
    rule trace-zero-bound[OF support],
    rule trace-slot-bound[OF support],
    rule trace-merkle-bound[OF support],
    rule trace-structural-bound[OF support],
    rule trace-next-bound[OF support],
    rule trace-successor-bound[OF support],
    rule trace-final-bound[OF support],
    rule trace-total-bound[OF support])
qed

lemma active-trace-fri-residual-obligations-from-bounds-without-head-and-selected-zero:
fixes
  Rt Zt At Pt Ht Nt St ::
    ('f staged-proof-data ⇒
      'f protocol-channel ⇒ prob)
assumes trace-sampled-bound:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  wp-event verify-monad
    (trace-fri-bad-with-header-tied-sampled-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state)

```

$(\text{staged-proof-transcript data}) \leq$
Rt data attacker-state
and *trace-zero-bound:*
 $\wedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-zero-round-final-obstruction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Zt data attacker-state
and *trace-slot-bound:*
 $\wedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-authenticated-slot-recorded-chunk-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
At data attacker-state
and *trace-merkle-bound:*
 $\wedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Pt data attacker-state
and *trace-structural-bound:*
 $\wedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-sibling-mismatch-structural-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$

```

    Ht data attacker-state
and trace-next-bound:
 $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-header-tied-next-value-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$ 
    Nt data attacker-state
and trace-successor-bound:
 $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-header-tied-successor-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$ 
    St data attacker-state
and trace-total-bound:
 $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
    Rt data attacker-state +
      ((((((Pt data attacker-state + Ht data attacker-state) +
        Pt data attacker-state) +
        Nt data attacker-state + St data attacker-state +
        (Pt data attacker-state + At data attacker-state + 0)) +
        Pt data attacker-state) +
        At data attacker-state + Zt data attacker-state)
       $\leq$  trace-fri-error
shows active-trace-fri-residual-obligations-for A Rt Zt At Pt Ht Pt
  Nt St ( $\lambda$ - -. (0::prob))
unfolding active-trace-fri-residual-obligations-for-def
proof (intro allI impI)
fix data attacker-state
assume support:
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state

```

```

    (staged-proof-transcript data)
show trace-fri-header-tied-residual-obligations ?s
    (Rt data attacker-state) (Zt data attacker-state)
    (At data attacker-state) (Pt data attacker-state)
    (Ht data attacker-state) (Pt data attacker-state)
    (Nt data attacker-state) (St data attacker-state) 0
by (rule
    trace-fri-header-tied-residual-obligations-from-bounds-without-head-and-selected-zero)
    (rule trace-sampled-bound[OF support],
    rule trace-zero-bound[OF support],
    rule trace-slot-bound[OF support],
    rule trace-merkle-bound[OF support],
    rule trace-structural-bound[OF support],
    rule trace-next-bound[OF support],
    rule trace-successor-bound[OF support],
    rule trace-total-bound[OF support])
qed

lemma active-trace-fri-residual-obligations-from-bounds-without-head-slot-and-selected-zero:
fixes
    Rt Zt Pt Ht Nt St ::
    ('f staged-proof-data  $\Rightarrow$ 
    'f protocol-channel  $\Rightarrow$  prob)
assumes trace-sampled-bound:
     $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
    adversary-initial-state)  $\Longrightarrow$ 
    wp-event verify-monad
    (trace-fri-bad-with-header-tied-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$ 
    Rt data attacker-state
and trace-zero-bound:
     $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
    adversary-initial-state)  $\Longrightarrow$ 
    wp-event verify-monad
    (trace-fri-header-tied-zero-round-final-obstruction
    (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$ 
    Zt data attacker-state
and trace-merkle-bound:
     $\wedge$ data attacker-state.

```

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Pt data attacker-state
and *trace-structural-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-sibling-mismatch-structural-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Ht data attacker-state
and *trace-next-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-next-value-replay-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Nt data attacker-state
and *trace-successor-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-successor-replay-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
St data attacker-state
and *trace-total-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈

```

    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  Rt data attacker-state +
  (((((Pt data attacker-state + Ht data attacker-state) +
    Pt data attacker-state) +
    Nt data attacker-state + St data attacker-state +
    (Pt data attacker-state + 0 + 0)) +
    Pt data attacker-state) +
    0 + Zt data attacker-state)
 $\leq$  trace-fri-error
shows
  active-trace-fri-residual-obligations-for A Rt Zt
  ( $\lambda$ - -. (0::prob)) Pt Ht Pt Nt St ( $\lambda$ - -. (0::prob))
unfolding active-trace-fri-residual-obligations-for-def
proof (intro allI impI)
fix data attacker-state
assume support:
  Some (data, attacker-state)  $\in$ 
  set-dist (execute (checked-staged-transcript-program A)
    adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
show trace-fri-header-tied-residual-obligations ?s
  (Rt data attacker-state) (Zt data attacker-state) 0
  (Pt data attacker-state) (Ht data attacker-state)
  (Pt data attacker-state) (Nt data attacker-state)
  (St data attacker-state) 0
by (rule
  trace-fri-header-tied-residual-obligations-from-bounds-without-head-slot-and-selected-zero)
  (rule trace-sampled-bound[OF support],
  rule trace-zero-bound[OF support],
  rule trace-merkle-bound[OF support],
  rule trace-structural-bound[OF support],
  rule trace-next-bound[OF support],
  rule trace-successor-bound[OF support],
  rule trace-total-bound[OF support])
qed

lemma active-composition-fri-residual-obligations-from-bounds:
fixes
  Rc Ac Pc Nc Sc Gc ::
  ('f staged-proof-data  $\implies$ 
  'f protocol-channel  $\implies$  prob)
assumes comp-sampled-bound:
 $\bigwedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
  set-dist (execute (checked-staged-transcript-program A)
    adversary-initial-state)  $\implies$ 

```

$wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri}\text{-bad}\text{-with}\text{-verifier}\text{-tied}\text{-sampled}\text{-layer}\text{-chain}$
 $(verifier\text{-state}\text{-from}\text{-adversary } attacker\text{-state}$
 $(staged\text{-proof}\text{-transcript } data)))$
 $(verifier\text{-state}\text{-from}\text{-adversary } attacker\text{-state}$
 $(staged\text{-proof}\text{-transcript } data)) \leq$
 $Rc \text{ data } attacker\text{-state}$
and $comp\text{-slot}\text{-bound}$:
 $\wedge data \text{ attacker}\text{-state}.$
 $Some (data, attacker\text{-state}) \in$
 $set\text{-dist } (execute (checked\text{-staged}\text{-transcript}\text{-program } A)$
 $adversary\text{-initial}\text{-state}) \implies$
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri}\text{-verifier}\text{-tied}\text{-authenticated}\text{-slot}\text{-recorded}\text{-chunk}\text{-gap}$
 $(verifier\text{-state}\text{-from}\text{-adversary } attacker\text{-state}$
 $(staged\text{-proof}\text{-transcript } data)))$
 $(verifier\text{-state}\text{-from}\text{-adversary } attacker\text{-state}$
 $(staged\text{-proof}\text{-transcript } data)) \leq$
 $Ac \text{ data } attacker\text{-state}$
and $comp\text{-merkle}\text{-bound}$:
 $\wedge data \text{ attacker}\text{-state}.$
 $Some (data, attacker\text{-state}) \in$
 $set\text{-dist } (execute (checked\text{-staged}\text{-transcript}\text{-program } A)$
 $adversary\text{-initial}\text{-state}) \implies$
 $wp\text{-event } verify\text{-monad}$
 $(partial\text{-merkle}\text{-inconsistency}\text{-bad}$
 $(verifier\text{-state}\text{-from}\text{-adversary } attacker\text{-state}$
 $(staged\text{-proof}\text{-transcript } data)))$
 $(verifier\text{-state}\text{-from}\text{-adversary } attacker\text{-state}$
 $(staged\text{-proof}\text{-transcript } data)) \leq$
 $Pc \text{ data } attacker\text{-state}$
and $comp\text{-next}\text{-bound}$:
 $\wedge data \text{ attacker}\text{-state}.$
 $Some (data, attacker\text{-state}) \in$
 $set\text{-dist } (execute (checked\text{-staged}\text{-transcript}\text{-program } A)$
 $adversary\text{-initial}\text{-state}) \implies$
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri}\text{-verifier}\text{-tied}\text{-next}\text{-value}\text{-replay}\text{-gap}$
 $(verifier\text{-state}\text{-from}\text{-adversary } attacker\text{-state}$
 $(staged\text{-proof}\text{-transcript } data)))$
 $(verifier\text{-state}\text{-from}\text{-adversary } attacker\text{-state}$
 $(staged\text{-proof}\text{-transcript } data)) \leq$
 $Nc \text{ data } attacker\text{-state}$
and $comp\text{-successor}\text{-bound}$:
 $\wedge data \text{ attacker}\text{-state}.$
 $Some (data, attacker\text{-state}) \in$
 $set\text{-dist } (execute (checked\text{-staged}\text{-transcript}\text{-program } A)$
 $adversary\text{-initial}\text{-state}) \implies$
 $wp\text{-event } verify\text{-monad}$


```

    (composition-fri-verifier-tied-successor-replay-gap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
  Sc data attacker-state
and comp-final-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-replay-gap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤
    Gc data attacker-state
and comp-total-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    Rc data attacker-state +
      (((Pc data attacker-state + Ac data attacker-state) +
        Nc data attacker-state + Sc data attacker-state +
        Gc data attacker-state + Pc data attacker-state) +
        Ac data attacker-state)
    ≤ composition-fri-error
shows active-composition-fri-residual-obligations-for A Rc Ac Pc Nc Sc Gc
unfolding active-composition-fri-residual-obligations-for-def
proof (intro allI impI)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show composition-fri-verifier-tied-residual-obligations ?s
  (Rc data attacker-state) (Ac data attacker-state)
  (Pc data attacker-state)
  (Nc data attacker-state) (Sc data attacker-state)
  (Gc data attacker-state)
by (rule composition-fri-verifier-tied-residual-obligations-from-bounds)
  (rule comp-sampled-bound[OF support],
    rule comp-slot-bound[OF support],
    rule comp-merkle-bound[OF support],

```

```

    rule comp-next-bound[OF support],
    rule comp-successor-bound[OF support],
    rule comp-final-bound[OF support],
    rule comp-total-bound[OF support])
qed

lemma active-composition-fri-residual-obligations-from-bounds-slot-zero:
fixes
  Rc Pc Nc Sc Gc ::
    ('f staged-proof-data  $\Rightarrow$ 
     'f protocol-channel  $\Rightarrow$  prob)
assumes comp-sampled-bound:
   $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\Longrightarrow$ 
    wp-event verify-monad
      (composition-fri-bad-with-verifier-tied-sampled-layer-chain
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq$ 
    Rc data attacker-state
and comp-merkle-bound:
   $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\Longrightarrow$ 
    wp-event verify-monad
      (partial-merkle-inconsistency-bad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq$ 
    Pc data attacker-state
and comp-next-bound:
   $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\Longrightarrow$ 
    wp-event verify-monad
      (composition-fri-verifier-tied-next-value-replay-gap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq$ 
    Nc data attacker-state
and comp-successor-bound:
   $\wedge$ data attacker-state.

```

```

Some (data, attacker-state) ∈
  set-dist (execute (checked-staged-transcript-program A)
    adversary-initial-state) ⇒
wp-event verify-monad
  (composition-fri-verifier-tied-successor-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
Sc data attacker-state
and comp-final-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
wp-event verify-monad
  (composition-fri-verifier-tied-final-value-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
Gc data attacker-state
and comp-total-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
Rc data attacker-state +
  (((Pc data attacker-state + 0) +
    Nc data attacker-state + Sc data attacker-state +
    Gc data attacker-state + Pc data attacker-state) +
    0)
  ≤ composition-fri-error
shows
  active-composition-fri-residual-obligations-for A Rc
  (λ- -. (0::prob)) Pc Nc Sc Gc
unfolding active-composition-fri-residual-obligations-for-def
proof (intro allI impI)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show composition-fri-verifier-tied-residual-obligations ?s
  (Rc data attacker-state) 0 (Pc data attacker-state)
  (Nc data attacker-state) (Sc data attacker-state)

```

(*Gc data attacker-state*)
by (*rule composition-fri-verifier-tied-residual-obligations-from-bounds-slot-zero*)
 (*rule comp-sampled-bound*[*OF support*],
rule comp-merkle-bound[*OF support*],
rule comp-next-bound[*OF support*],
rule comp-successor-bound[*OF support*],
rule comp-final-bound[*OF support*],
rule comp-total-bound[*OF support*])
qed

lemma *active-fri-reductions-forD*:
assumes *active-fri-reductions-for A*
and *Some (data, attacker-state) ∈*
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
shows
trace-fri-header-tied-reduction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
composition-fri-verifier-tied-reduction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
trace-fri-empty-header-reduction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
using *assms unfolding active-fri-reductions-for-def* **by** *blast+*

lemma *trace-fri-header-tied-reduction-from-residual-obligations*:
assumes
trace-fri-header-tied-residual-obligations s r-b z-b a-b p-b h-b g-b n-b s-b f-b
shows *trace-fri-header-tied-reduction s*
proof –
have *sampled*:
wp-event verify-monad
(trace-fri-bad-with-header-tied-sampled-layer-chain s) s ≤ r-b
using *assms unfolding trace-fri-header-tied-residual-obligations-def*
by *blast*
have *zero*:
wp-event verify-monad
(trace-fri-header-tied-zero-round-final-obstruction s) s ≤ z-b
using *assms unfolding trace-fri-header-tied-residual-obligations-def*
by *blast*
have *slot*:
wp-event verify-monad
(trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s) s
≤ a-b
using *assms unfolding trace-fri-header-tied-residual-obligations-def*
by *blast*
have *merkle*:

$wp\text{-event verify-monad } (partial\text{-merkle-inconsistency-bad } s) \ s \leq p\text{-b}$
using *assms* **unfolding** *trace-fri-header-tied-residual-obligations-def*
by *blast*

have *structural*:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq h\text{-b}$
using *assms* **unfolding** *trace-fri-header-tied-residual-obligations-def*
by *blast*

have *head-neq*:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-base-head-value-neq-gap } s) \ s \leq g\text{-b}$
using *assms* **unfolding** *trace-fri-header-tied-residual-obligations-def*
by *blast*

have *head-mismatch*:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-base-head-value-mismatch-gap } s) \ s \leq g\text{-b}$
by (rule
 $wp\text{-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap}$
 $[OF \ head\text{-neq}]$)

have *alignment*:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-base-value-alignment-gap } s) \ s \leq g\text{-b}$
by (rule
 $wp\text{-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap}$
 $[OF \ head\text{-mismatch}]$)

have *next-bound*:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-next-value-replay-gap } s) \ s \leq n\text{-b}$
using *assms* **unfolding** *trace-fri-header-tied-residual-obligations-def*
by *blast*

have *successor*:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-successor-replay-gap } s) \ s \leq s\text{-b}$
using *assms* **unfolding** *trace-fri-header-tied-residual-obligations-def*
by *blast*

have *selected-final*:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-final-value-selected-step-missing-gap } s) \ s$
 $\leq f\text{-b}$
using *assms* **unfolding** *trace-fri-header-tied-residual-obligations-def*
by *blast*

have *final-bound*:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-final-value-replay-gap } s) \ s \leq$
 $p\text{-b} + a\text{-b} + f\text{-b}$
by (rule
 $wp\text{-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step}$
 $(rule \ merkle, rule \ slot, rule \ selected\text{-final})$)

have *total*:

```


$$r-b + (((((p-b + h-b) + g-b) + n-b + s-b + (p-b + a-b + f-b)) + p-b) + a-b + z-b) \leq \text{trace-fri-error}$$

using assms unfolding trace-fri-header-tied-residual-obligations-def
by blast
show ?thesis
by (rule trace-fri-header-tied-reduction-from-residual-bounds
  [OF sampled zero slot merkle structural alignment next-bound successor final-bound total])
qed

lemma composition-fri-verifier-tied-reduction-from-residual-obligations:
assumes
  composition-fri-verifier-tied-residual-obligations s r-b a-b p-b n-b s-b g-b
shows composition-fri-verifier-tied-reduction s
proof –
have sampled:
  wp-event verify-monad
  (composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s
   $\leq r-b$ 
using assms
unfolding composition-fri-verifier-tied-residual-obligations-def
by blast
have slot:
  wp-event verify-monad
  (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
   $s \leq a-b$ 
using assms
unfolding composition-fri-verifier-tied-residual-obligations-def
by blast
have merkle:
  wp-event verify-monad (partial-merkle-inconsistency-bad s) s  $\leq p-b$ 
using assms
unfolding composition-fri-verifier-tied-residual-obligations-def
by blast
have next-bound:
  wp-event verify-monad
  (composition-fri-verifier-tied-next-value-replay-gap s)  $s \leq n-b$ 
using assms
unfolding composition-fri-verifier-tied-residual-obligations-def
by blast
have successor:
  wp-event verify-monad
  (composition-fri-verifier-tied-successor-replay-gap s)  $s \leq s-b$ 
using assms
unfolding composition-fri-verifier-tied-residual-obligations-def
by blast
have final-bound:
  wp-event verify-monad

```

```

    (composition-fri-verifier-tied-final-value-replay-gap s) s ≤ g-b
using assms
unfolding composition-fri-verifier-tied-residual-obligations-def
by blast
have total:
  r-b + (((p-b + a-b) + n-b + s-b + g-b + p-b) + a-b)
    ≤ composition-fri-error
using assms
unfolding composition-fri-verifier-tied-residual-obligations-def
by blast
have total-zero:
  r-b + (((p-b + 0) + n-b + s-b + g-b + p-b) + a-b)
    ≤ composition-fri-error
by (rule order-trans[OF - total]) simp
show ?thesis
by (rule
    composition-fri-verifier-tied-reduction-from-residual-bounds-with-recorded-missing-zero
    [OF sampled slot merkle next-bound successor final-bound total-zero])
qed

```

```

lemma active-fri-reductions-from-residual-obligations:
  assumes active-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft
    Rc Ac Pc Nc Sc Gc
  shows active-fri-reductions-for A
  unfolding active-fri-reductions-for-def
proof (intro allI impI conjI)
  fix data attacker-state
  assume support:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have trace-obs:
    trace-fri-header-tied-residual-obligations ?s
      (Rt data attacker-state) (Zt data attacker-state)
      (At data attacker-state) (Pt data attacker-state)
      (Ht data attacker-state) (Gt data attacker-state)
      (Nt data attacker-state) (St data attacker-state)
      (Ft data attacker-state)
  using assms support
  unfolding active-fri-residual-obligations-for-def by (simp add: Let-def)
  have composition-obs:
    composition-fri-verifier-tied-residual-obligations ?s
      (Rc data attacker-state) (Ac data attacker-state)
      (Pc data attacker-state)
      (Nc data attacker-state) (Sc data attacker-state)
      (Gc data attacker-state)

```

```

using assms support
unfolding active-fri-residual-obligations-for-def by (simp add: Let-def)
have empty-obs:
  trace-fri-empty-header-reduction ?s
  using assms support
  unfolding active-fri-residual-obligations-for-def by (simp add: Let-def)
show trace-fri-header-tied-reduction ?s
  by (rule trace-fri-header-tied-reduction-from-residual-obligations
    [OF trace-obs])
show composition-fri-verifier-tied-reduction ?s
  by (rule
    composition-fri-verifier-tied-reduction-from-residual-obligations
    [OF composition-obs])
show trace-fri-empty-header-reduction ?s
  by (rule empty-obs)
qed

```

```

lemma active-fri-reductions-from-reachable:
  assumes trace-reachable:
     $\bigwedge \text{data attacker-state.}$ 
     $\text{Some (data, attacker-state)} \in$ 
       $\text{set-dist (execute (checked-staged-transcript-program } A \text{)}$ 
         $\text{adversary-initial-state)} \implies$ 
      trace-fri-reachable-partial-candidate-reduction-assumption
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
  and composition-reachable:
     $\bigwedge \text{data attacker-state.}$ 
     $\text{Some (data, attacker-state)} \in$ 
       $\text{set-dist (execute (checked-staged-transcript-program } A \text{)}$ 
         $\text{adversary-initial-state)} \implies$ 
      composition-fri-reachable-partial-candidate-reduction-assumption
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
  shows active-fri-reductions-for A
  unfolding active-fri-reductions-for-def
proof (intro allI impI conjI)
  fix data attacker-state
  assume support:
     $\text{Some (data, attacker-state)} \in$ 
       $\text{set-dist (execute (checked-staged-transcript-program } A \text{)}$ 
         $\text{adversary-initial-state)}$ 
  show trace-fri-header-tied-reduction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
  by (rule trace-fri-header-tied-reduction-from-reachable-reduction)
    (rule trace-reachable[OF support])
  show composition-fri-verifier-tied-reduction
    (verifier-state-from-adversary attacker-state

```



```

    (staged-proof-transcript data))
  by (rule composition-fri-verifier-tied-reduction-from-reachable-reduction)
    (rule composition-reachable[OF support])
show trace-fri-empty-header-reduction
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))
  by (rule trace-fri-empty-header-reduction-from-reachable-reduction)
    (rule trace-reachable[OF support])
qed

```

lemma *active-fri-residual-obligations-from-bounds:*

```

fixes
  Rt Zt At Pt Ht Gt Nt St Ft ::
    ('f staged-proof-data  $\Rightarrow$ 
     'f protocol-channel  $\Rightarrow$  prob)
and Rc Ac Pc Nc Sc Gc ::
    ('f staged-proof-data  $\Rightarrow$ 
     'f protocol-channel  $\Rightarrow$  prob)
assumes trace-sampled-bound:
   $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
      wp-event verify-monad
        (trace-fri-bad-with-header-tied-sampled-layer-chain
         (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
        (verifier-state-from-adversary attacker-state
         (staged-proof-transcript data))  $\leq$ 
      Rt data attacker-state
and trace-zero-bound:
   $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
      wp-event verify-monad
        (trace-fri-header-tied-zero-round-final-obstruction
         (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
        (verifier-state-from-adversary attacker-state
         (staged-proof-transcript data))  $\leq$ 
      Zt data attacker-state
and trace-slot-bound:
   $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
      wp-event verify-monad
        (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap

```

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{At data attacker-state}$
and $\text{trace-merkle-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Pt data attacker-state}$
and $\text{trace-structural-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Ht data attacker-state}$
and $\text{trace-head-neq-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-head-value-neq-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Gt data attacker-state}$
and trace-next-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Nt \text{ data attacker-state}$
and *trace-successor-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $St \text{ data attacker-state}$
and *trace-final-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-final-value-selected-step-missing-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ft \text{ data attacker-state}$
and *trace-total-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $Rt \text{ data attacker-state} +$
 $(((((Pt \text{ data attacker-state} + Ht \text{ data attacker-state}) +$
 $Gt \text{ data attacker-state}) +$
 $Nt \text{ data attacker-state} + St \text{ data attacker-state} +$
 $(Pt \text{ data attacker-state} + At \text{ data attacker-state} +$
 $Ft \text{ data attacker-state})) +$
 $Pt \text{ data attacker-state}) +$
 $At \text{ data attacker-state} + Zt \text{ data attacker-state})$
 $\leq \text{trace-fri-error}$
and *comp-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}$

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Rc \text{ data attacker-state}$
and *comp-slot-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ac \text{ data attacker-state}$
and *comp-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Pc \text{ data attacker-state}$
and *comp-next-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Nc \text{ data attacker-state}$
and *comp-successor-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$

```

      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
    Sc data attacker-state
and comp-final-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-replay-gap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤
    Gc data attacker-state
and comp-total-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    Rc data attacker-state +
      (((Pc data attacker-state + Ac data attacker-state) +
        Nc data attacker-state + Sc data attacker-state +
        Gc data attacker-state + Pc data attacker-state) +
        Ac data attacker-state)
    ≤ composition-fri-error
and empty-header-reduction:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    trace-fri-empty-header-reduction
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
shows active-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft
  Rc Ac Pc Nc Sc Gc
unfolding active-fri-residual-obligations-for-def
proof (intro allI impI)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show (let s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  in
  trace-fri-header-tied-residual-obligations s

```

```

    (Rt data attacker-state) (Zt data attacker-state)
    (At data attacker-state) (Pt data attacker-state)
    (Ht data attacker-state) (Gt data attacker-state)
    (Nt data attacker-state) (St data attacker-state)
    (Ft data attacker-state) ∧
    composition-fri-verifier-tied-residual-obligations s
    (Rc data attacker-state) (Ac data attacker-state)
    (Pc data attacker-state)
    (Nc data attacker-state) (Sc data attacker-state)
    (Gc data attacker-state) ∧
    trace-fri-empty-header-reduction s)
using trace-sampled-bound[OF support] trace-zero-bound[OF support]
    trace-slot-bound[OF support] trace-merkle-bound[OF support]
    trace-structural-bound[OF support] trace-head-neq-bound[OF support]
    trace-next-bound[OF support] trace-successor-bound[OF support]
    trace-final-bound[OF support] trace-total-bound[OF support]
    comp-sampled-bound[OF support] comp-slot-bound[OF support]
    comp-merkle-bound[OF support] comp-next-bound[OF support]
    comp-successor-bound[OF support]
    comp-final-bound[OF support] comp-total-bound[OF support]
    empty-header-reduction[OF support]
unfolding trace-fri-header-tied-residual-obligations-def
    composition-fri-verifier-tied-residual-obligations-def
by (simp add: Let-def)
qed

```

lemma *active-fri-residual-obligations-from-bounds-without-head:*

fixes

Rt Zt At Pt Ht Nt St Ft ::

('f staged-proof-data ⇒

'f protocol-channel ⇒ prob)

and *Rc Ac Pc Nc Sc Gc ::*

('f staged-proof-data ⇒

'f protocol-channel ⇒ prob)

assumes *trace-sampled-bound:*

$\bigwedge$ *data attacker-state.*

Some (data, attacker-state) ∈

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) ⇒

wp-event verify-monad

(trace-fri-bad-with-header-tied-sampled-layer-chain

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)) ≤

Rt data attacker-state

and *trace-zero-bound:*

$\bigwedge$ *data attacker-state.*

Some (data, attacker-state) ∈

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Zt data attacker-state}$
and trace-slot-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{At data attacker-state}$
and $\text{trace-merkle-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Pt data attacker-state}$
and $\text{trace-structural-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Ht data attacker-state}$
and trace-next-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Nt data attacker-state}$
and $\text{trace-successor-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{St data attacker-state}$
and $\text{trace-final-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-final-value-selected-step-missing-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Ft data attacker-state}$
and $\text{trace-total-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{Rt data attacker-state} +$
 $(((((\text{Pt data attacker-state} + \text{Ht data attacker-state}) +$
 $\text{Pt data attacker-state}) +$
 $\text{Nt data attacker-state} + \text{St data attacker-state} +$
 $(\text{Pt data attacker-state} + \text{At data attacker-state} +$
 $\text{Ft data attacker-state})) +$
 $\text{Pt data attacker-state}) +$
 $\text{At data attacker-state} + \text{Zt data attacker-state})$
 $\leq \text{trace-fri-error}$
and $\text{comp-sampled-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $R_c \text{ data attacker-state}$
and comp-slot-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $A_c \text{ data attacker-state}$
and comp-merkle-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $P_c \text{ data attacker-state}$
and comp-next-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $N_c \text{ data attacker-state}$
and comp-successor-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Sc data attacker-state}$
and comp-final-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Gc data attacker-state}$
and comp-total-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{Rc data attacker-state} +$
 $((\text{Pc data attacker-state} + \text{Ac data attacker-state}) +$
 $\text{Nc data attacker-state} + \text{Sc data attacker-state} +$
 $\text{Gc data attacker-state} + \text{Pc data attacker-state}) +$
 $\text{Ac data attacker-state})$
 $\leq \text{composition-fri-error}$
and $\text{empty-header-reduction:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{trace-fri-empty-header-reduction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
shows $\text{active-fri-residual-obligations-for A Rt Zt At Pt Ht Pt Nt St Ft}$
 Rc Ac Pc Nc Sc Gc
proof $(\text{rule active-fri-residual-obligations-from-bounds})$
fix $\text{data attacker-state}$
assume support:
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})$
show $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-sampled-layer-chain}$

```

    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Rt data attacker-state
  by (rule trace-sampled-bound[OF support])
show wp-event verify-monad
  (trace-fri-header-tied-zero-round-final-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Zt data attacker-state
  by (rule trace-zero-bound[OF support])
show wp-event verify-monad
  (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  At data attacker-state
  by (rule trace-slot-bound[OF support])
show wp-event verify-monad
  (partial-merkle-inconsistency-bad
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Pt data attacker-state
  by (rule trace-merkle-bound[OF support])
show wp-event verify-monad
  (trace-fri-header-tied-sibling-mismatch-structural-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Ht data attacker-state
  by (rule trace-structural-bound[OF support])
show wp-event verify-monad
  (trace-fri-header-tied-base-head-value-neq-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Pt data attacker-state
  by (rule
    wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
  (rule trace-merkle-bound[OF support])
show wp-event verify-monad

```

```

    (trace-fri-header-tied-next-value-replay-gap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
  Nt data attacker-state
  by (rule trace-next-bound[OF support])
show wp-event verify-monad
  (trace-fri-header-tied-successor-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  St data attacker-state
  by (rule trace-successor-bound[OF support])
show wp-event verify-monad
  (trace-fri-header-tied-final-value-selected-step-missing-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Ft data attacker-state
  by (rule trace-final-bound[OF support])
show Rt data attacker-state +
  (((((Pt data attacker-state + Ht data attacker-state) +
    Pt data attacker-state) +
    Nt data attacker-state + St data attacker-state +
    (Pt data attacker-state + At data attacker-state +
      Ft data attacker-state)) +
    Pt data attacker-state) +
    At data attacker-state + Zt data attacker-state)
  ≤ trace-fri-error
  by (rule trace-total-bound[OF support])
show wp-event verify-monad
  (composition-fri-bad-with-verifier-tied-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Rc data attacker-state
  by (rule comp-sampled-bound[OF support])
show wp-event verify-monad
  (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Ac data attacker-state
  by (rule comp-slot-bound[OF support])

```

```

show wp-event verify-monad
  (partial-merkle-inconsistency-bad
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Pc data attacker-state
by (rule comp-merkle-bound[OF support])
show wp-event verify-monad
  (composition-fri-verifier-tied-next-value-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Nc data attacker-state
by (rule comp-next-bound[OF support])
show wp-event verify-monad
  (composition-fri-verifier-tied-successor-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Sc data attacker-state
by (rule comp-successor-bound[OF support])
show wp-event verify-monad
  (composition-fri-verifier-tied-final-value-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Gc data attacker-state
by (rule comp-final-bound[OF support])
show Rc data attacker-state +
  (((Pc data attacker-state + Ac data attacker-state) +
    Nc data attacker-state + Sc data attacker-state +
    Gc data attacker-state + Pc data attacker-state) +
    Ac data attacker-state)
  ≤ composition-fri-error
by (rule comp-total-bound[OF support])
show trace-fri-empty-header-reduction
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
by (rule empty-header-reduction[OF support])
qed

lemma active-fri-residual-obligations-from-bounds-without-head-and-selected-zero:
fixes
  Rt Zt At Pt Ht Nt St ::
  ('f staged-proof-data ⇒

```

```

    'f protocol-channel  $\Rightarrow$  prob)
and Rc Ac Pc Nc Sc Gc ::
    ('f staged-proof-data  $\Rightarrow$ 
     'f protocol-channel  $\Rightarrow$  prob)
assumes trace-sampled-bound:
 $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\Rightarrow$ 
wp-event verify-monad
  (trace-fri-bad-with-header-tied-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$ 
    Rt data attacker-state
and trace-zero-bound:
 $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\Rightarrow$ 
wp-event verify-monad
  (trace-fri-header-tied-zero-round-final-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$ 
    Zt data attacker-state
and trace-slot-bound:
 $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\Rightarrow$ 
wp-event verify-monad
  (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$ 
    At data attacker-state
and trace-merkle-bound:
 $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\Rightarrow$ 
wp-event verify-monad
  (partial-merkle-inconsistency-bad
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))

```

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Pt \text{ data attacker-state}$
and *trace-structural-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ht \text{ data attacker-state}$
and *trace-next-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Nt \text{ data attacker-state}$
and *trace-successor-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $St \text{ data attacker-state}$
and *trace-total-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $Rt \text{ data attacker-state} +$
 $(((((Pt \text{ data attacker-state} + Ht \text{ data attacker-state}) +$
 $Pt \text{ data attacker-state}) +$
 $Nt \text{ data attacker-state} + St \text{ data attacker-state} +$
 $(Pt \text{ data attacker-state} + At \text{ data attacker-state} + 0))) +$

$Pt \text{ data attacker-state}) +$
 $At \text{ data attacker-state} + Zt \text{ data attacker-state})$
 $\leq \text{trace-fri-error}$
and comp-sampled-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Rc \text{ data attacker-state}$
and comp-slot-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ac \text{ data attacker-state}$
and comp-merkle-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Pc \text{ data attacker-state}$
and comp-next-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data}) \leq$
 $Nc \text{ data attacker-state}$
and *comp-successor-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Sc \text{ data attacker-state}$
and *comp-final-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Gc \text{ data attacker-state}$
and *comp-total-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $Rc \text{ data attacker-state} +$
 $((Pc \text{ data attacker-state} + Ac \text{ data attacker-state}) +$
 $Nc \text{ data attacker-state} + Sc \text{ data attacker-state} +$
 $Gc \text{ data attacker-state} + Pc \text{ data attacker-state}) +$
 $Ac \text{ data attacker-state})$
 $\leq \text{composition-fri-error}$
and *empty-header-reduction*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-empty-header-reduction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
shows *active-fri-residual-obligations-for A Rt Zt At Pt Ht Pt Nt St*
 $(\lambda - . (0::\text{prob})) Rc Ac Pc Nc Sc Gc$
proof (rule *active-fri-residual-obligations-from-components*)
show *active-trace-fri-residual-obligations-for A Rt Zt At Pt Ht Pt*

```

  Nt St ( $\lambda$ - -. (0::prob))
by (rule
  active-trace-fri-residual-obligations-from-bounds-without-head-and-selected-zero)
(erule trace-sampled-bound, erule trace-zero-bound,
  erule trace-slot-bound, erule trace-merkle-bound,
  erule trace-structural-bound, erule trace-next-bound,
  erule trace-successor-bound, erule trace-total-bound)
show active-composition-fri-residual-obligations-for A Rc Ac Pc Nc Sc Gc
by (rule active-composition-fri-residual-obligations-from-bounds)
(erule comp-sampled-bound, erule comp-slot-bound,
  erule comp-merkle-bound, erule comp-next-bound,
  erule comp-successor-bound, erule comp-final-bound,
  erule comp-total-bound)
show active-empty-trace-fri-reduction-for A
  unfolding active-empty-trace-fri-reduction-for-def
  by (intro allI impI, erule empty-header-reduction)
qed

```

lemma *active-fri-residual-obligations-from-bounds-without-head-slot-and-selected-zero:*
fixes

```

  Rt Zt Pt Ht Nt St ::
    ('f staged-proof-data  $\Rightarrow$ 
      'f protocol-channel  $\Rightarrow$  prob)
  and Rc Pc Nc Sc Gc ::
    ('f staged-proof-data  $\Rightarrow$ 
      'f protocol-channel  $\Rightarrow$  prob)

```

assumes *trace-sampled-bound:*

```

 $\wedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  wp-event verify-monad
    (trace-fri-bad-with-header-tied-sampled-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))  $\leq$ 
  Rt data attacker-state

```

and *trace-zero-bound:*

```

 $\wedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  wp-event verify-monad
    (trace-fri-header-tied-zero-round-final-obstruction
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))  $\leq$ 

```

Zt data attacker-state
and *trace-merkle-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Pt data attacker-state
and *trace-structural-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-sibling-mismatch-structural-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Ht data attacker-state
and *trace-next-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-next-value-replay-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Nt data attacker-state
and *trace-successor-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-successor-replay-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
St data attacker-state

and *trace-total-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
Rt data attacker-state +
 ((((*Pt data attacker-state* + *Ht data attacker-state*) +
Pt data attacker-state) +
Nt data attacker-state + *St data attacker-state* +
 (*Pt data attacker-state* + 0 + 0)) +
Pt data attacker-state) +
 0 + *Zt data attacker-state*)

$\leq$ *trace-fri-error*

and *comp-sampled-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
 (*composition-fri-bad-with-verifier-tied-sampled-layer-chain*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq$

Rc data attacker-state

and *comp-merkle-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
 (*partial-merkle-inconsistency-bad*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq$

Pc data attacker-state

and *comp-next-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
 (*composition-fri-verifier-tied-next-value-replay-gap*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq$

Nc data attacker-state

and *comp-successor-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Sc data attacker-state}$

and *comp-final-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Gc data attacker-state}$

and *comp-total-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{Rc data attacker-state} +$
 $((\text{Pc data attacker-state} + 0) +$
 $\text{Nc data attacker-state} + \text{Sc data attacker-state} +$
 $\text{Gc data attacker-state} + \text{Pc data attacker-state}) +$
 $0)$
 $\leq \text{composition-fri-error}$

and *empty-header-reduction*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-empty-header-reduction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$

shows
 $\text{active-fri-residual-obligations-for } A \text{ Rt Zt } (\lambda \text{ - . } (0::\text{prob}))$
 $\text{Pt Ht Pt Nt St } (\lambda \text{ - . } (0::\text{prob})) \text{ Rc } (\lambda \text{ - . } (0::\text{prob}))$
 Pc Nc Sc Gc

proof (rule *active-fri-residual-obligations-from-components*)
show *active-trace-fri-residual-obligations-for* $A \text{ Rt Zt}$

```

  (λ- -. (0::prob)) Pt Ht Pt Nt St (λ- -. (0::prob))
by (rule
  active-trace-fri-residual-obligations-from-bounds-without-head-slot-and-selected-zero)
(erule trace-sampled-bound, erule trace-zero-bound,
erule trace-merkle-bound, erule trace-structural-bound,
erule trace-next-bound, erule trace-successor-bound,
erule trace-total-bound)
show active-composition-fri-residual-obligations-for A Rc
  (λ- -. (0::prob)) Pc Nc Sc Gc
by (rule active-composition-fri-residual-obligations-from-bounds-slot-zero)
(erule comp-sampled-bound, erule comp-merkle-bound,
erule comp-next-bound, erule comp-successor-bound,
erule comp-final-bound, erule comp-total-bound)
show active-empty-trace-fri-reduction-for A
  unfolding active-empty-trace-fri-reduction-for-def
  by (intro allI impI, erule empty-header-reduction)
qed

lemmas active-fri-reductions-from-residual-bounds =
  active-fri-reductions-from-residual-obligations[
    OF active-fri-residual-obligations-from-bounds]

lemmas active-fri-reductions-from-residual-bounds-without-head =
  active-fri-reductions-from-residual-obligations[
    OF active-fri-residual-obligations-from-bounds-without-head]

lemmas active-fri-reductions-from-residual-bounds-without-head-and-selected-zero =
  active-fri-reductions-from-residual-obligations[
    OF active-fri-residual-obligations-from-bounds-without-head-and-selected-zero]

lemmas active-fri-reductions-from-residual-bounds-without-head-slot-and-selected-zero =
  active-fri-reductions-from-residual-obligations[
    OF active-fri-residual-obligations-from-bounds-without-head-slot-and-selected-zero]

lemma active-fri-reductions-staged-trace-bound:
  assumes active: active-fri-reductions-for A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-header-tied-partial-candidate)
    adversary-initial-state ≤ trace-fri-error
  by (rule trace-fri-header-tied-reduction-staged-bound)
    (rule active-fri-reductions-forD(1)[OF active])

lemma active-fri-reductions-staged-composition-bound:
  assumes active: active-fri-reductions-for A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event

```

$\text{composition-fri-bad-with-verifier-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}$
by (rule $\text{composition-fri-verifier-tied-reduction-staged-bound}$)
(rule $\text{active-fri-reductions-forD}(2)[OF \text{ active}]$)

lemma *active-fri-reductions-staged-empty-header-bound:*

assumes *active: active-fri-reductions-for A*

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$

by (rule $\text{trace-fri-empty-header-reduction-staged-bound}$)
(rule $\text{active-fri-reductions-forD}(3)[OF \text{ active}]$)

lemma *active-trace-fri-residual-obligations-staged-trace-bound:*

assumes

active-trace-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$

proof (rule $\text{trace-fri-header-tied-reduction-staged-bound}$)

fix *data attacker-state*

assume *support:*

$\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$

let $?s =$

$\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data})$

have *residual:*

$\text{trace-fri-header-tied-residual-obligations } ?s$
 $(\text{Rt } \text{data } \text{attacker-state}) (\text{Zt } \text{data } \text{attacker-state})$
 $(\text{At } \text{data } \text{attacker-state}) (\text{Pt } \text{data } \text{attacker-state})$
 $(\text{Ht } \text{data } \text{attacker-state}) (\text{Gt } \text{data } \text{attacker-state})$
 $(\text{Nt } \text{data } \text{attacker-state}) (\text{St } \text{data } \text{attacker-state})$
 $(\text{Ft } \text{data } \text{attacker-state})$

using *assms support*

unfolding *active-trace-fri-residual-obligations-for-def* **by** *simp*

show $\text{trace-fri-header-tied-reduction } ?s$

by (rule $\text{trace-fri-header-tied-reduction-from-residual-obligations}$
 $[OF \text{ residual}]$)

qed

lemma *active-composition-fri-residual-obligations-staged-composition-bound:*

assumes

active-composition-fri-residual-obligations-for A Rc Ac Pc Nc Sc Gc

shows
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad composition\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}tied\text{-}partial\text{-}candidate)$
 $adversary\text{-}initial\text{-}state \leq composition\text{-}fri\text{-}error$
proof (rule $composition\text{-}fri\text{-}verifier\text{-}tied\text{-}reduction\text{-}staged\text{-}bound$)
fix data attacker-state
assume support:
 $Some\ (data, attacker\text{-}state) \in$
 $set\text{-}dist\ (execute\ (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $\quad adversary\text{-}initial\text{-}state)$
let ?s =
 $verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)$
have residual:
 $composition\text{-}fri\text{-}verifier\text{-}tied\text{-}residual\text{-}obligations\ ?s$
 $(Rc\ data\ attacker\text{-}state)\ (Ac\ data\ attacker\text{-}state)$
 $(Pc\ data\ attacker\text{-}state)$
 $(Nc\ data\ attacker\text{-}state)\ (Sc\ data\ attacker\text{-}state)$
 $(Gc\ data\ attacker\text{-}state)$
using assms support
unfolding active-composition-fri-residual-obligations-for-def **by** simp
show composition-fri-verifier-tied-reduction ?s
by (rule $composition\text{-}fri\text{-}verifier\text{-}tied\text{-}reduction\text{-}from\text{-}residual\text{-}obligations$
 $[OF\ residual])$
qed

lemma active-empty-trace-fri-reduction-staged-empty-header-bound:

assumes active-empty-trace-fri-reduction-for A
shows
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad trace\text{-}fri\text{-}bad\text{-}with\text{-}empty\text{-}composition\text{-}header\text{-}candidates)$
 $adversary\text{-}initial\text{-}state \leq trace\text{-}fri\text{-}error$
by (rule $trace\text{-}fri\text{-}empty\text{-}header\text{-}reduction\text{-}staged\text{-}bound$)
 $(use\ assms\ in$
 $\quad \langle unfold\ active\text{-}empty\text{-}trace\text{-}fri\text{-}reduction\text{-}for\text{-}def,\ blast \rangle)$

lemma active-fri-residual-obligations-staged-trace-bound:

assumes active-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft
Rc Ac Pc Nc Sc Gc
shows
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad trace\text{-}fri\text{-}bad\text{-}with\text{-}header\text{-}tied\text{-}partial\text{-}candidate)$
 $adversary\text{-}initial\text{-}state \leq trace\text{-}fri\text{-}error$
by (rule $active\text{-}fri\text{-}reductions\text{-}staged\text{-}trace\text{-}bound$)
 $(rule\ active\text{-}fri\text{-}reductions\text{-}from\text{-}residual\text{-}obligations[OF\ assms])$

lemma *active-fri-residual-obligations-staged-composition-bound*:
assumes *active-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft Rc Ac Pc Nc Sc Gc*
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state ≤ composition-fri-error
by (*rule active-fri-reductions-staged-composition-bound*)
(*rule active-fri-reductions-from-residual-obligations[OF assms]*)

lemma *active-fri-residual-obligations-staged-empty-header-bound*:
assumes *active-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft Rc Ac Pc Nc Sc Gc*
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state ≤ trace-fri-error
by (*rule active-fri-reductions-staged-empty-header-bound*)
(*rule active-fri-reductions-from-residual-obligations[OF assms]*)

end

context *soundness-fri*
begin

lemma *active-fri-reductions-for-from-locale*:
active-fri-reductions-for A
unfolding *active-fri-reductions-for-def*
proof (*intro allI impI conjI*)
fix *data attacker-state*
assume *support*:
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
show *trace-fri-header-tied-reduction*
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
using *trace-fri-active-reductions[OF support]* **by** *blast*

next
fix *data attacker-state*
assume *support*:
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
show *composition-fri-verifier-tied-reduction*
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))

```

    by (rule composition-fri-active-reduction[OF support])
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show trace-fri-empty-header-reduction
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
using trace-fri-active-reductions[OF support] by blast
qed

end

end

```

```

theory Soundness-FRI-Obligation-Normalization
imports Soundness-FRI-Active-Reductions
begin

```

Normal form for the three FRI obligations that remain internal to locale *soundness-fri*. This theory does not change the protocol or the public soundness route. It only records the exact verifier-local events behind the internal FRI assumptions and exposes the generic partial FRI evidence already available from accepted verifier transcripts.

```

context soundness
begin

```

```

lemma trace-fri-header-tied-reduction-event:
  trace-fri-header-tied-reduction s  $\longleftrightarrow$ 
  wp-event verify-monad
    (λout. ∃ trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr as rest trace-openings trace-table.
    accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final dg composition-roots composition-bs composition-final
      fri-query-idxs trace-round-layers composition-round-layers ∧
    verifier-header-transcript s fr trace-roots trace-final as dg
      composition-roots composition-final rest ∧
    accepted-with-partial-trace-openings s out fr fri-query-idxs
      trace-openings ∧
    partial-trace-table-candidate trace-table trace-openings ∧
    ¬ trace-table-low-degree trace-table) s ≤
  trace-fri-error
unfolding trace-fri-header-tied-reduction-def
  trace-fri-bad-with-header-tied-partial-candidate-def
by simp

```

lemma *trace-fri-empty-header-reduction-event*:
trace-fri-empty-header-reduction $s \longleftrightarrow$
wp-event verify-monad
 $(\lambda out. \exists fr\ f\text{-fri-roots}\ f\text{-final}\ as\ dg\ final\ trace\text{-query-idxs}$
 $\quad trace\text{-openings}\ trace\text{-table}\ composition\text{-table}.$
 $\quad accepted\text{-with-empty-composition-header-candidates}\ s\ out\ fr$
 $\quad f\text{-fri-roots}\ f\text{-final}\ as\ dg\ final\ trace\text{-query-idxs}\ trace\text{-openings}$
 $\quad trace\text{-table}\ composition\text{-table} \wedge$
 $\quad \neg\ trace\text{-table-low-degree}\ trace\text{-table})\ s \leq$
trace-fri-error
unfolding *trace-fri-empty-header-reduction-def*
trace-fri-bad-with-empty-composition-header-candidates-def
by *simp*

lemma *composition-fri-verifier-tied-reduction-event*:
composition-fri-verifier-tied-reduction $s \longleftrightarrow$
wp-event verify-monad
 $(\lambda out. \exists trace\text{-roots}\ trace\text{-bs}\ trace\text{-final}\ fri\text{-dg}\ composition\text{-roots}$
 $\quad composition\text{-bs}\ composition\text{-final}\ fri\text{-query-idxs}\ trace\text{-round-layers}$
 $\quad composition\text{-round-layers}\ fr\ as\ trace\text{-openings}\ composition\text{-openings}$
 $\quad trace\text{-table}\ composition\text{-table}.$
 $\quad accepted\text{-fri-opening-transcript}\ s\ out\ trace\text{-roots}\ trace\text{-bs}$
 $\quad trace\text{-final}\ fri\text{-dg}\ composition\text{-roots}\ composition\text{-bs}$
 $\quad composition\text{-final}\ fri\text{-query-idxs}\ trace\text{-round-layers}$
 $\quad composition\text{-round-layers} \wedge$
 $\quad accepted\text{-with-partial-initial-openings-aligned}\ s\ out\ fr\ trace\text{-roots}$
 $\quad trace\text{-final}\ as\ fri\text{-dg}\ composition\text{-roots}\ composition\text{-final}$
 $\quad fri\text{-query-idxs}\ trace\text{-openings}\ composition\text{-openings} \wedge$
 $\quad partial\text{-trace-table-candidate}\ trace\text{-table}\ trace\text{-openings} \wedge$
 $\quad partial\text{-composition-table-candidate}\ composition\text{-table}$
 $\quad composition\text{-openings} \wedge$
 $\quad trace\text{-table-low-degree}\ trace\text{-table} \wedge$
 $\quad \neg\ composition\text{-table-low-degree}\ maxDegree\ composition\text{-table})\ s \leq$
composition-fri-error
unfolding *composition-fri-verifier-tied-reduction-def*
composition-fri-bad-with-verifier-tied-partial-candidate-def
by *simp*

lemma *trace-fri-header-tied-badE*:
assumes *trace-fri-bad-with-header-tied-partial-candidate* $s\ out$
obtains *trace-roots* *trace-bs* *trace-final* *dg* *composition-roots*
 $\quad composition\text{-bs}\ composition\text{-final}\ fri\text{-query-idxs}\ trace\text{-round-layers}$
 $\quad composition\text{-round-layers}\ fr\ as\ rest\ trace\text{-openings}\ trace\text{-table}$ **where**
 $\quad accepted\text{-fri-opening-transcript}\ s\ out\ trace\text{-roots}\ trace\text{-bs}$
 $\quad trace\text{-final}\ dg\ composition\text{-roots}\ composition\text{-bs}\ composition\text{-final}$
 $\quad fri\text{-query-idxs}\ trace\text{-round-layers}\ composition\text{-round-layers}$
 $\quad verifier\text{-header-transcript}\ s\ fr\ trace\text{-roots}\ trace\text{-final}\ as\ dg$
 $\quad composition\text{-roots}\ composition\text{-final}\ rest$
 $\quad accepted\text{-with-partial-trace-openings}\ s\ out\ fr\ fri\text{-query-idxs}$

trace-openings
partial-trace-table-candidate trace-table trace-openings
 $\neg$ *trace-table-low-degree trace-table*
using *assms*
unfolding *trace-fri-bad-with-header-tied-partial-candidate-def*
by *blast*

lemma *trace-fri-empty-header-badE*:
assumes *trace-fri-bad-with-empty-composition-header-candidates s out*
obtains *fr f-fri-roots f-final as dg final trace-query-idxs*
trace-openings trace-table composition-table **where**
accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table
 $\neg$ *trace-table-low-degree trace-table*
using *assms*
unfolding *trace-fri-bad-with-empty-composition-header-candidates-def*
by *blast*

lemma *trace-fri-empty-header-bad-extracts-generic-partial-evidence*:
assumes *outcome*:
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *bad*:
trace-fri-bad-with-empty-composition-header-candidates s
(Some (result, final-state))
obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table **where**
trace-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final dg
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr candidate-query-idxs
trace-openings trace-table
generic-fri-partial-evidence trace-table-low-degree trace-table
(clength - 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers

proof –
have *reachable*:
trace-fri-bad-with-reachable-partial-candidate s
(Some (result, final-state))
by (*rule*
trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate
 $[OF\ bad]$)
from *verify-monad-trace-fri-bad-extracts-opening-evidence*
 $[OF\ outcome\ reachable]$
obtain *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings

trace-table
where *evidence*:
trace-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final dg
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr candidate-query-idxs
trace-openings trace-table
by *blast*
have *generic*:
generic-fri-partial-evidence trace-table-low-degree trace-table
(clength - 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers
by (rule *trace-fri-partial-candidate-opening-evidence-generic-partial*
[*OF evidence*])
show *?thesis*
by (rule *that[OF evidence generic]*)
qed

lemma *trace-fri-header-tied-sampled-layer-chain-imp-sampled-layer-chain*:
assumes *trace-fri-bad-with-header-tied-sampled-layer-chain s out*
shows *trace-fri-bad-with-sampled-layer-chain s out*
proof —
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table doms
layers
where *fri*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *partial*:
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate*:
partial-trace-table-candidate trace-table trace-openings
and *not-low*: $\neg$ *trace-table-low-degree trace-table*
and *chain*:
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
unfolding *trace-fri-bad-with-header-tied-sampled-layer-chain-def*
by *blast*
have *evidence*:
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr fri-query-idxs trace-openings
trace-table
unfolding *trace-fri-partial-candidate-opening-evidence-def*

trace-fri-partial-candidate-evidence-def
by (intro conjI fri partial candidate not-low)
show ?thesis
unfolding trace-fri-bad-with-sampled-layer-chain-def
by (intro exI conjI)
(rule evidence, rule chain)
qed

lemma wp-trace-fri-header-tied-sampled-layer-chain-bound-from-sampled:
assumes sampled-bound:
wp-event verify-monad (trace-fri-bad-with-sampled-layer-chain s) $s \leq R$
shows
wp-event verify-monad
(trace-fri-bad-with-header-tied-sampled-layer-chain s) $s \leq R$
proof –
have wp-event verify-monad
(trace-fri-bad-with-header-tied-sampled-layer-chain s) $s \leq$
wp-event verify-monad (trace-fri-bad-with-sampled-layer-chain s) s
by (rule wp-event-mono)
(rule trace-fri-header-tied-sampled-layer-chain-imp-sampled-layer-chain)
then show ?thesis
by (rule order-trans[OF - sampled-bound])
qed

lemma wp-trace-fri-header-tied-sampled-layer-chain-bound-from-full-cover-and-missing:
assumes full-bound:
wp-event verify-monad
(trace-fri-bad-with-full-cover-layer-chain s bad) $s \leq R$
and missing-bound:
wp-event verify-monad
(trace-fri-sampled-missing-full-cover s bad) $s \leq M$
shows
wp-event verify-monad
(trace-fri-bad-with-header-tied-sampled-layer-chain s) $s \leq R + M$
by (rule wp-trace-fri-header-tied-sampled-layer-chain-bound-from-sampled)
(rule wp-trace-fri-sampled-bound-from-full-cover-and-missing
[OF full-bound missing-bound])

lemma composition-fri-verifier-tied-sampled-layer-chain-imp-sampled-layer-chain:
assumes composition-fri-bad-with-verifier-tied-sampled-layer-chain s out
shows composition-fri-bad-with-sampled-layer-chain s out
proof –
from assms **obtain** trace-roots trace-bs trace-final fri-dg
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
composition-openings trace-table composition-table doms layers
where fri:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *aligned*:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
and *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
and *composition-candidate*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *composition-not-low*:
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *chain*:
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) composition-roots composition-bs composition-final
fri-query-idxs composition-round-layers doms layers
unfolding *composition-fri-bad-with-verifier-tied-sampled-layer-chain-def*
by *blast*
have *unaligned*:
accepted-with-partial-initial-openings s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings fri-query-idxs composition-openings
by (*rule accepted-with-partial-initial-openings-aligned-imp-unaligned*
[OF aligned])
have *evidence*:
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-roots trace-final as fri-dg
composition-roots composition-final fri-query-idxs trace-openings
fri-query-idxs composition-openings trace-table composition-table
unfolding *composition-fri-partial-candidate-opening-evidence-def*
composition-fri-partial-candidate-evidence-def
by (*intro conjI fri unaligned trace-candidate composition-candidate*
trace-low composition-not-low)
show *?thesis*
unfolding *composition-fri-bad-with-sampled-layer-chain-def*
by (*intro exI conjI*)
(rule evidence, rule chain)
qed

lemma *wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-sampled*:
assumes *sampled-bound*:
wp-event verify-monad (composition-fri-bad-with-sampled-layer-chain s) s
 $\leq R$
shows

$wp\text{-event verify-monad}$
 $(composition\text{-fri-bad-with-verifier-tied-sampled-layer-chain } s) \ s \leq R$
proof –
have $wp\text{-event verify-monad}$
 $(composition\text{-fri-bad-with-verifier-tied-sampled-layer-chain } s) \ s \leq$
 $wp\text{-event verify-monad } (composition\text{-fri-bad-with-sampled-layer-chain } s) \ s$
by (rule $wp\text{-event-mono}$)
(rule
 $composition\text{-fri-verifier-tied-sampled-layer-chain-imp-sampled-layer-chain}$)
then show $?thesis$
by (rule $order\text{-trans}[OF - sampled-bound]$)
qed

lemma $wp\text{-composition-fri-verifier-tied-sampled-layer-chain-bound-from-full-cover-and-missing}$:
assumes $full\text{-bound}$:
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-bad-with-full-cover-layer-chain } s \text{ bad}) \ s \leq R$
and $missing\text{-bound}$:
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-sampled-missing-full-cover } s \text{ bad}) \ s \leq M$
shows
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-bad-with-verifier-tied-sampled-layer-chain } s) \ s \leq R + M$
by (rule
 $wp\text{-composition-fri-verifier-tied-sampled-layer-chain-bound-from-sampled}$)
(rule $wp\text{-composition-fri-sampled-bound-from-full-cover-and-missing}$
 $[OF full\text{-bound missing-bound}]$)

lemma $trace\text{-fri-header-tied-residual-obligations-from-full-cover-missing-without-head-slot-and-selected-zero}$:
assumes $full\text{-bound}$:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-bad-with-full-cover-layer-chain } s \text{ bad}) \ s \leq R$
and $missing\text{-bound}$:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-sampled-missing-full-cover } s \text{ bad}) \ s \leq M$
and $zero$:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-zero-round-final-obstruction } s) \ s \leq z\text{-b}$
and $merkle$:
 $wp\text{-event verify-monad } (partial\text{-merkle-inconsistency-bad } s) \ s \leq p\text{-b}$
and $structural$:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq h\text{-b}$
and $next\text{-bound}$:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-next-value-replay-gap } s) \ s \leq n\text{-b}$
and $successor$:

wp-event verify-monad
(trace-fri-header-tied-successor-replay-gap s) s ≤ s-b
and total:
 $(R + M) + (((((p-b + h-b) + p-b) + n-b + s-b +$
 $(p-b + 0 + 0)) + p-b) + 0 + z-b)$
 $≤ \text{trace-fri-error}$
shows
trace-fri-header-tied-residual-obligations s
 $(R + M) z-b 0 p-b h-b p-b n-b s-b 0$
by (*rule*
trace-fri-header-tied-residual-obligations-from-bounds-without-head-slot-and-selected-zero)
(*rule*
wp-trace-fri-header-tied-sampled-layer-chain-bound-from-full-cover-and-missing
 $[OF \text{ full-bound missing-bound}]$,
rule zero, rule merkle, rule structural, rule next-bound,
rule successor, rule total)

lemma *composition-fri-verifier-tied-residual-obligations-from-full-cover-missing-slot-zero:*

assumes *full-bound:*
wp-event verify-monad
(composition-fri-bad-with-full-cover-layer-chain s bad) s ≤ R
and *missing-bound:*
wp-event verify-monad
(composition-fri-sampled-missing-full-cover s bad) s ≤ M
and *merkle:*
wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ p-b
and *next-bound:*
wp-event verify-monad
(composition-fri-verifier-tied-next-value-replay-gap s) s ≤ n-b
and *successor:*
wp-event verify-monad
(composition-fri-verifier-tied-successor-replay-gap s) s ≤ s-b
and *final-bound:*
wp-event verify-monad
(composition-fri-verifier-tied-final-value-replay-gap s) s ≤ g-b
and total:
 $(R + M) + (((p-b + 0) + n-b + s-b + g-b + p-b) + 0)$
 $≤ \text{composition-fri-error}$

shows
composition-fri-verifier-tied-residual-obligations s
 $(R + M) 0 p-b n-b s-b g-b$
by (*rule composition-fri-verifier-tied-residual-obligations-from-bounds-slot-zero*)
(*rule*
wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-full-cover-and-missing
 $[OF \text{ full-bound missing-bound}]$,
rule merkle, rule next-bound, rule successor, rule final-bound,
rule total)

lemma *composition-fri-verifier-tied-badE:*

assumes *composition-fri-bad-with-verifier-tied-partial-candidate s out*
obtains *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table **where**
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
partial-trace-table-candidate trace-table trace-openings
partial-composition-table-candidate composition-table
composition-openings
trace-table-low-degree trace-table
 $\neg$ *composition-table-low-degree maxDegree composition-table*
using *assms*
unfolding *composition-fri-bad-with-verifier-tied-partial-candidate-def*
by *blast*

lemma *trace-fri-header-tied-bad-generic-partial-evidence:*

assumes *trace-fri-bad-with-header-tied-partial-candidate s out*
obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table **where**
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
partial-trace-table-candidate trace-table trace-openings
 $\neg$ *trace-table-low-degree trace-table*
generic-fri-partial-evidence trace-table-low-degree trace-table
(clength - 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers

proof —

from *trace-fri-header-tied-badE[OF assms]* **obtain** *trace-roots trace-bs*
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr as rest
trace-openings trace-table

where *fri:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers

and *header:*

verifier-header-transcript s fr trace-roots trace-final as dg

composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate:*
partial-trace-table-candidate trace-table trace-openings
and *not-low:* $\neg$ *trace-table-low-degree trace-table*
by *blast*
have *generic:*
generic-fri-partial-evidence trace-table-low-degree trace-table
(clength - 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers
by (rule *accepted-fri-opening-transcript-trace-generic-partial-evidence*
 $[OF\ fri]$)
show *?thesis*
by (rule *that[OF fri header partial candidate not-low generic]*)
qed

lemma *composition-fri-verifier-tied-bad-generic-partial-evidence:*
assumes *composition-fri-bad-with-verifier-tied-partial-candidate s out*
obtains *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table **where**
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
partial-trace-table-candidate trace-table trace-openings
partial-composition-table-candidate composition-table
composition-openings
trace-table-low-degree trace-table
 $\neg$ *composition-table-low-degree maxDegree composition-table*
generic-fri-partial-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) composition-roots composition-bs composition-final
fri-query-idxs composition-round-layers

proof –
from *composition-fri-verifier-tied-badE[OF assms]* **obtain** *trace-roots*
trace-bs trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table
where *fri:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs

```

    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
and aligned:
    accepted-with-partial-initial-openings-aligned s out fr trace-roots
    trace-final as fri-dg composition-roots composition-final
    fri-query-idxs trace-openings composition-openings
and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
and trace-low: trace-table-low-degree trace-table
and composition-not-low:
    ¬ composition-table-low-degree maxDegree composition-table
by blast
have generic:
    generic-fri-partial-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) composition-roots composition-bs composition-final
    fri-query-idxs composition-round-layers
by (rule
    accepted-fri-opening-transcript-composition-generic-partial-evidence
    [OF fri])
show ?thesis
by (rule that[OF fri aligned trace-candidate composition-candidate
    trace-low composition-not-low generic])
qed

end

end

```

```

theory Soundness-FRI-Prequery-Core
imports
    Soundness-FRI-Obligation-Normalization
    Staged-Security-Experiment-Composition-Branch
begin

```

Verifier-local FRI challenge prequery accounting.

The concrete FRI challenge-list bounds require freshness of future Trace/Composition FRI challenge keys. In the staged adversary experiment the verifier receives the adversary's final oracle map, so freshness is not automatic. This layer defines the exact verifier-header key paths and proves deterministic fresh/prequeried splits. It does not add assumptions or change the oracle model.

```

context soundness
begin

```

definition *trace-fri-challenge-key-at*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $\text{nat} \Rightarrow 'f \text{ protocol-hash-input}$
where
 $\text{trace-fri-challenge-key-at } s \text{ fr roots } i =$
 $\text{TraceFriChallenge } (P\text{TraceFriCounter } s + i)$
 $(\text{foldl concat } (\text{concat } (P\text{State } s) \text{ fr}) (\text{take } (\text{Suc } i) \text{ roots}))$

definition *composition-fri-challenge-key-at*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ protocol-hash-input}$
where
 $\text{composition-fri-challenge-key-at } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots } i =$
 $\text{CompositionFriChallenge } (P\text{CompositionFriCounter } s + i)$
 $(\text{foldl concat}$
 $(\text{concat}$
 $(\text{foldl concat}$
 $(\text{concat}$
 $(\text{foldl concat } (\text{concat } (P\text{State } s) \text{ fr}) \text{ trace-roots})$
 $\text{trace-final})$
 $\text{as})$
 $\text{dg})$
 $(\text{take } (\text{Suc } i) \text{ composition-roots}))$

definition *trace-fri-challenge-path-fresh*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $\text{nat} \Rightarrow \text{bool}$
where
 $\text{trace-fri-challenge-path-fresh } s \text{ fr roots } n \longleftrightarrow$
 $(\forall i < n.$
 $i < \text{length roots} \longrightarrow$
 $\text{fmlookup } (\text{HashMap } s) (\text{trace-fri-challenge-key-at } s \text{ fr roots } i) =$
 $\text{None})$

definition *trace-fri-challenge-path-prequeried*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $\text{nat} \Rightarrow \text{bool}$
where
 $\text{trace-fri-challenge-path-prequeried } s \text{ fr roots } n \longleftrightarrow$
 $(\exists i < n.$
 $i < \text{length roots} \wedge$
 $\text{fmlookup } (\text{HashMap } s) (\text{trace-fri-challenge-key-at } s \text{ fr roots } i)$
 $\neq \text{None})$

definition *composition-fri-challenge-path-fresh*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow \text{bool}$

where

composition-fri-challenge-path-fresh *s* *fr* *trace-roots* *trace-final* *as* *dg*
composition-roots *n* $\longleftrightarrow$
 $(\forall i < n.$
 $i < \text{length } \text{composition-roots} \longrightarrow$
 $\text{fmlookup } (\text{HashMap } s)$
 $(\text{composition-fri-challenge-key-at } s \text{ fr } \text{trace-roots } \text{trace-final}$
 $\text{as } dg \text{ composition-roots } i) = \text{None})$

definition *composition-fri-challenge-path-prequeried*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow \text{bool}$

where

composition-fri-challenge-path-prequeried *s* *fr* *trace-roots* *trace-final* *as* *dg*
composition-roots *n* $\longleftrightarrow$
 $(\exists i < n.$
 $i < \text{length } \text{composition-roots} \wedge$
 $\text{fmlookup } (\text{HashMap } s)$
 $(\text{composition-fri-challenge-key-at } s \text{ fr } \text{trace-roots } \text{trace-final}$
 $\text{as } dg \text{ composition-roots } i) \neq \text{None})$

definition *trace-fri-challenge-list-fresh-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ list set} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-challenge-list-fresh-hit *s* *B* *out* $\longleftrightarrow$
 $(\exists \text{ trace-bs } dg \text{ comp-bs fr } \text{trace-roots } \text{trace-final as}$
 $\text{composition-roots final rest.}$
 $\text{accepted-fri-challenges } s \text{ out trace-bs } dg \text{ comp-bs} \wedge$
 $\text{verifier-header-transcript } s \text{ fr } \text{trace-roots } \text{trace-final as } dg$
 $\text{composition-roots final rest} \wedge$
 $\text{trace-bs} \in B \wedge$
 $\text{trace-fri-challenge-path-fresh } s \text{ fr } \text{trace-roots}$
 $(\text{length } \text{trace-roots}))$

definition *trace-fri-challenge-list-prequery-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ list set} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-challenge-list-prequery-hit *s* *B* *out* $\longleftrightarrow$
 $(\exists \text{ trace-bs } dg \text{ comp-bs fr } \text{trace-roots } \text{trace-final as}$
 $\text{composition-roots final rest.}$
 $\text{accepted-fri-challenges } s \text{ out trace-bs } dg \text{ comp-bs} \wedge$
 $\text{verifier-header-transcript } s \text{ fr } \text{trace-roots } \text{trace-final as } dg$
 $\text{composition-roots final rest} \wedge$
 $\text{trace-bs} \in B \wedge$
 $\text{trace-fri-challenge-path-prequeried } s \text{ fr } \text{trace-roots}$
 $(\text{length } \text{trace-roots}))$

definition *composition-fri-challenge-list-fresh-hit*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-challenge-list-fresh-hit $s \ B \ \text{out} \longleftrightarrow$
 $(\exists \text{ trace-bs } dg \ \text{comp-bs } fr \ \text{trace-roots } \text{trace-final} \ \text{as}$
 $\quad \text{composition-roots } \text{final } \text{rest}.$
 $\text{accepted-fri-challenges } s \ \text{out } \text{trace-bs } dg \ \text{comp-bs} \wedge$
 $\text{verifier-header-transcript } s \ \text{fr } \text{trace-roots } \text{trace-final} \ \text{as } dg$
 $\quad \text{composition-roots } \text{final } \text{rest} \wedge$
 $\text{comp-bs} \in B \ dg \wedge$
 $\text{composition-fri-challenge-path-fresh } s \ \text{fr } \text{trace-roots } \text{trace-final}$
 $\quad \text{as } dg \ \text{composition-roots } (\text{length } \text{composition-roots}))$

definition *composition-fri-challenge-list-prequery-hit*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-challenge-list-prequery-hit $s \ B \ \text{out} \longleftrightarrow$
 $(\exists \text{ trace-bs } dg \ \text{comp-bs } fr \ \text{trace-roots } \text{trace-final} \ \text{as}$
 $\quad \text{composition-roots } \text{final } \text{rest}.$
 $\text{accepted-fri-challenges } s \ \text{out } \text{trace-bs } dg \ \text{comp-bs} \wedge$
 $\text{verifier-header-transcript } s \ \text{fr } \text{trace-roots } \text{trace-final} \ \text{as } dg$
 $\quad \text{composition-roots } \text{final } \text{rest} \wedge$
 $\text{comp-bs} \in B \ dg \wedge$
 $\text{composition-fri-challenge-path-prequeried } s \ \text{fr } \text{trace-roots}$
 $\quad \text{trace-final} \ \text{as } dg \ \text{composition-roots } (\text{length } \text{composition-roots}))$

definition *trace-fri-challenge-key-relation*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ list set} \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$

where

trace-fri-challenge-key-relation $s \ B \ n \ x \ y \longleftrightarrow$
 $(\exists bs \in B. \exists fr \ \text{roots } i.$
 $\quad i < n \wedge$
 $\quad i < \text{length } bs \wedge$
 $\quad i < \text{length } \text{roots} \wedge$
 $\quad x = \text{trace-fri-challenge-key-at } s \ \text{fr } \text{roots } i \wedge$
 $\quad y = bs \ ! \ i)$

definition *composition-fri-challenge-key-relation*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $\text{nat} \Rightarrow 'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$

where

composition-fri-challenge-key-relation $s \ B \ n \ x \ y \longleftrightarrow$
 $(\exists dg. \exists bs \in B \ dg. \exists fr \ \text{trace-roots } \text{trace-final} \ \text{as}$
 $\quad \text{composition-roots } i.$
 $\quad i < n \wedge$
 $\quad i < \text{length } bs \wedge$

$i < \text{length composition-roots} \wedge$
 $x = \text{composition-fri-challenge-key-at } s \text{ fr trace-roots trace-final}$
 $\text{as } dg \text{ composition-roots } i \wedge$
 $y = bs ! i)$

definition *checked-staged-trace-fri-prequery-hit*

$:: 'f \text{ list set} \Rightarrow$
 $('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{checked-staged-trace-fri-prequery-hit } B \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (data, \text{attacker-state}) \Rightarrow$
 $\text{let } s = \text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript } data)$
 $\text{in staged-trace-fri-challenges } data \in B \wedge$
 $\text{trace-fri-challenge-path-prequeried } s$
 $(\text{staged-trace-root } data)$
 $(\text{staged-trace-fri-roots } data)$
 $(\text{length } (\text{staged-trace-fri-roots } data))))$

definition *checked-staged-composition-fri-prequery-hit*

$:: ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{checked-staged-composition-fri-prequery-hit } B \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (data, \text{attacker-state}) \Rightarrow$
 $\text{let } s = \text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript } data)$
 $\text{in staged-composition-fri-challenges } data \in$
 $B (\text{staged-degree } data) \wedge$
 $\text{composition-fri-challenge-path-prequeried } s$
 $(\text{staged-trace-root } data)$
 $(\text{staged-trace-fri-roots } data)$
 $(\text{staged-trace-final } data)$
 $(\text{staged-alphas } data)$
 $(\text{staged-degree } data)$
 $(\text{staged-composition-fri-roots } data)$
 $(\text{length } (\text{staged-composition-fri-roots } data))))$

definition *checked-staged-trace-fri-prequery-relation*

$:: 'f \text{ staged-adversary} \Rightarrow 'f \text{ list set} \Rightarrow$
 $'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$

where

$\text{checked-staged-trace-fri-prequery-relation } A \text{ B } x \text{ y} \longleftrightarrow$
 $(\exists data \text{ attacker-state } i.$
 $\text{Some } (data, \text{attacker-state}) \in$

set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\wedge$
 staged-trace-fri-challenges data $\in B \wedge$
 i < ceil-log clength $\wedge$
 i < length (staged-trace-fri-challenges data) $\wedge$
 i < length (staged-trace-fri-roots data) $\wedge$
 x = trace-fri-challenge-key-at
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data) (staged-trace-fri-roots data) i $\wedge$
 y = staged-trace-fri-challenges data ! i)

definition checked-staged-composition-fri-prequery-relation

:: 'f staged-adversary $\Rightarrow$ ('f $\Rightarrow$ 'f list set) $\Rightarrow$
 'f protocol-hash-input $\Rightarrow$ 'f $\Rightarrow$ bool

where

checked-staged-composition-fri-prequery-relation A B x y $\longleftrightarrow$
 ($\exists$ data attacker-state i.
 Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\wedge$
 staged-composition-fri-challenges data $\in B$ (staged-degree data) $\wedge$
 i < ceil-log (maxDegree + 1) $\wedge$
 i < length (staged-composition-fri-challenges data) $\wedge$
 i < length (staged-composition-fri-roots data) $\wedge$
 x = composition-fri-challenge-key-at
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data) (staged-trace-fri-roots data)
 (staged-trace-final data) (staged-alphas data)
 (staged-degree data) (staged-composition-fri-roots data) i $\wedge$
 y = staged-composition-fri-challenges data ! i)

definition checked-staged-trace-fri-verifier-prequery-relation

:: 'f staged-adversary $\Rightarrow$ 'f list set $\Rightarrow$
 'f protocol-hash-input $\Rightarrow$ 'f $\Rightarrow$ bool

where

checked-staged-trace-fri-verifier-prequery-relation A B x y $\longleftrightarrow$
 ($\exists$ data attacker-state bs fr roots i.
 Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\wedge$
 bs $\in B \wedge$
 i < ceil-log clength $\wedge$
 i < length bs $\wedge$
 i < length roots $\wedge$

```

x = trace-fri-challenge-key-at
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))
fr roots i ∧
y = bs ! i)

```

definition *checked-staged-composition-fri-verifier-prequery-relation*

```

:: 'f staged-adversary ⇒ ('f ⇒ 'f list set) ⇒
  'f protocol-hash-input ⇒ 'f ⇒ bool

```

where

```

checked-staged-composition-fri-verifier-prequery-relation A B x y
↔
(∃ data attacker-state dg bs fr trace-roots trace-final as
 composition-roots i.
 Some (data, attacker-state) ∈
   set-dist
   (execute (checked-staged-transcript-program A)
    adversary-initial-state) ∧
 bs ∈ B dg ∧
 i < ceil-log (maxDegree + 1) ∧
 i < length bs ∧
 i < length composition-roots ∧
 x = composition-fri-challenge-key-at
   (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
 fr trace-roots trace-final as dg composition-roots i ∧
 y = bs ! i)

```

lemma *trace-fri-challenge-key-relation-fiber-card-bound:*

assumes *finite-B: finite B*

shows

```

card {y. trace-fri-challenge-key-relation s B n x y} ≤ card B * n

```

proof –

let *?pairs = B × {..*n*}*

let *?values = (λ(bs, i). bs ! i) ‘ ?pairs*

have *subset:*

```

{y. trace-fri-challenge-key-relation s B n x y} ⊆ ?values

```

unfolding *trace-fri-challenge-key-relation-def*

by *force*

have *finite-values: finite ?values*

using *finite-B* **by** *simp*

```

have card {y. trace-fri-challenge-key-relation s B n x y} ≤
  card ?values

```

by (rule *card-mono*[*OF* *finite-values subset*])

also have *... ≤ card ?pairs*

by (rule *card-image-le*) (*use* *finite-B* **in** *simp*)

also have *... = card B * n*

using *finite-B* **by** *simp*

finally show *?thesis* .

qed

lemma *composition-fri-challenge-key-relation-fiber-card-bound:*

assumes *finite-B*: $\bigwedge dg. \text{finite } (B \ dg)$

shows

$\text{card } \{y. \text{composition-fri-challenge-key-relation } s \ B \ n \ x \ y\} \leq$
 $(\sum dg \in (UNIV :: 'f \ \text{set}). \text{card } (B \ dg) * n)$

proof –

let *?pairs* = $(\text{SIGMA } dg:(UNIV :: 'f \ \text{set}). B \ dg \times \{..<n\})$

let *?values* = $(\lambda(dg, bs, i). bs \ ! \ i) \ ` \ ?pairs$

have *subset*:

$\{y. \text{composition-fri-challenge-key-relation } s \ B \ n \ x \ y\} \subseteq ?values$

unfolding *composition-fri-challenge-key-relation-def*

by *force*

have *finite-values*: *finite* *?values*

using *finite-B* **by** *simp*

have $\text{card } \{y. \text{composition-fri-challenge-key-relation } s \ B \ n \ x \ y\} \leq$
 $\text{card } ?values$

by $(\text{rule } \text{card-mono}[\text{OF } \text{finite-values } \text{subset}])$

also have $\dots \leq \text{card } ?pairs$

by $(\text{rule } \text{card-image-le}) \ (use \ \text{finite-B} \ \text{in } \text{simp})$

also have $\dots =$

$(\sum dg \in (UNIV :: 'f \ \text{set}). \text{card } (B \ dg \times \{..<n\}))$

using *finite-B* **by** $(\text{simp } add: \text{card-SigmaI})$

also have $\dots = (\sum dg \in (UNIV :: 'f \ \text{set}). \text{card } (B \ dg) * n)$

using *finite-B* **by** *simp*

finally show *?thesis* .

qed

lemma *hash-relation-program-checked-staged-transcript-trace-fri-prequery:*

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *finite-B*: *finite B*

shows

hash-relation-program $(\text{trace-fri-challenge-key-relation } s \ B \ n)$
 $(\text{card } B * n)$

$(\text{staged-attacker-query-budget } \text{budgets} + \text{staged-challenge-query-budget})$

$(\text{checked-staged-transcript-program } A)$

by $(\text{rule } \text{hash-relation-program-checked-staged-transcript-program}$
 $[\text{OF } wf \ \text{controlled}])$

$(\text{rule } \text{trace-fri-challenge-key-relation-fiber-card-bound}[\text{OF } \text{finite-B}])$

lemma *checked-staged-transcript-trace-fri-prequery-relation-bound:*

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *finite-B*: *finite B*

shows

wp-event $(\text{checked-staged-transcript-program } A)$

$(\text{hash-relation-hit-event } (\text{trace-fri-challenge-key-relation } s \ B \ n))$

$adversary-initial-state)$
 $adversary-initial-state \leq$
 $hash-relation-budget-value \ (card \ B * n)$
 $(staged-attacker-query-budget \ budgets + staged-challenge-query-budget)$
proof –
have *program*:
 $hash-relation-program \ (trace-fri-challenge-key-relation \ s \ B \ n)$
 $(card \ B * n)$
 $(staged-attacker-query-budget \ budgets + staged-challenge-query-budget)$
 $(checked-staged-transcript-program \ A)$
by (rule *hash-relation-program-checked-staged-transcript-trace-fri-prequery*
 $[OF \ wf \ controlled \ finite-B]$)
show *?thesis*
using *program*
unfolding *hash-relation-program-def hash-relation-budget-def* **by** *blast*
qed

lemma *hash-relation-program-checked-staged-transcript-composition-fri-prequery*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *finite-B*: $\bigwedge dg. \ finite \ (B \ dg)$
shows
 $hash-relation-program \ (composition-fri-challenge-key-relation \ s \ B \ n)$
 $(\sum dg \in (UNIV :: 'f \ set). \ card \ (B \ dg) * n)$
 $(staged-attacker-query-budget \ budgets + staged-challenge-query-budget)$
 $(checked-staged-transcript-program \ A)$
by (rule *hash-relation-program-checked-staged-transcript-program*
 $[OF \ wf \ controlled]$)
(rule *composition-fri-challenge-key-relation-fiber-card-bound*
 $[OF \ finite-B]$)

lemma *checked-staged-transcript-composition-fri-prequery-relation-bound*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *finite-B*: $\bigwedge dg. \ finite \ (B \ dg)$
shows
 $wp-event \ (checked-staged-transcript-program \ A)$
 $(hash-relation-hit-event$
 $(composition-fri-challenge-key-relation \ s \ B \ n)$
 $adversary-initial-state)$
 $adversary-initial-state \leq$
 $hash-relation-budget-value$
 $(\sum dg \in (UNIV :: 'f \ set). \ card \ (B \ dg) * n)$
 $(staged-attacker-query-budget \ budgets + staged-challenge-query-budget)$
proof –

have *program*:
 $hash-relation-program \ (composition-fri-challenge-key-relation \ s \ B \ n)$
 $(\sum dg \in (UNIV :: 'f \ set). \ card \ (B \ dg) * n)$
 $(staged-attacker-query-budget \ budgets + staged-challenge-query-budget)$

```

    (checked-staged-transcript-program A)
  by (rule
    hash-relation-program-checked-staged-transcript-composition-fri-prequery
    [OF wf controlled finite-B])
  show ?thesis
    using program
    unfolding hash-relation-program-def hash-relation-budget-def by blast
qed

end

context soundness
begin

lemma checked-staged-trace-fri-prequery-relation-fiber-card-bound:
  assumes finite-B: finite B
  shows
    card {y. checked-staged-trace-fri-prequery-relation A B x y} ≤
      card B * ceil-log clength
  proof -
    let ?pairs = B × {..

```

```

have subset:
  {y. checked-staged-composition-fri-prequery-relation A B x y}
    ⊆ ?values
  unfolding checked-staged-composition-fri-prequery-relation-def
  by force
have finite-values: finite ?values
  using finite-B by simp
have card {y. checked-staged-composition-fri-prequery-relation A B x y}
  ≤ card ?values
  by (rule card-mono[OF finite-values subset])
also have ... ≤ card ?pairs
  by (rule card-image-le) (use finite-B in simp)
also have ... =
  (∑ dg ∈ (UNIV :: 'f set).
    card (B dg × {.. $\text{ceil-log } (\text{maxDegree} + 1)\}$ }))
  using finite-B by (simp add: card-SigmaI)
also have ... =
  (∑ dg ∈ (UNIV :: 'f set).
    card (B dg) *  $\text{ceil-log } (\text{maxDegree} + 1)$ )
  using finite-B by simp
finally show ?thesis .
qed

```

lemma *checked-staged-trace-fri-verifier-prequery-relation-fiber-card-bound:*
assumes *finite-B: finite B*
shows

```

card {y. checked-staged-trace-fri-verifier-prequery-relation A B x y}
  ≤ card B *  $\text{ceil-log clength}$ 

```

proof –

```

let ?pairs = B × {.. $\text{ceil-log clength}$ }
let ?values = (λ(bs, i). bs ! i) ‘ ?pairs
have subset:
  {y. checked-staged-trace-fri-verifier-prequery-relation A B x y}
    ⊆ ?values
  unfolding checked-staged-trace-fri-verifier-prequery-relation-def
  by auto
have card {y. checked-staged-trace-fri-verifier-prequery-relation A B x y}
  ≤ card ?values
  by (rule card-mono[OF finite-imageI[OF finite-cartesian-product
    [OF finite-B finite-lessThan]] subset])
also have ... ≤ card ?pairs
  by (rule card-image-le) (use finite-B in simp)
also have ... = card B *  $\text{ceil-log clength}$ 
  using finite-B by simp
finally show ?thesis .
qed

```

lemma

checked-staged-composition-fri-verifier-prequery-relation-fiber-card-bound:

```

assumes finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
shows
  card
    {y. checked-staged-composition-fri-verifier-prequery-relation A B x y}
     $\leq (\sum dg \in (UNIV :: 'f \ set)).$ 
      card (B dg) * ceil-log (maxDegree + 1))
proof –
  let ?pairs =
    SIGMA dg:(UNIV :: 'f set). B dg  $\times \{..<\text{ceil-log } (\text{maxDegree} + 1)\}$ 
  let ?values = ( $\lambda(dg, bs, i). bs \ ! \ i$ ) ‘ ?pairs
  have subset:
    {y. checked-staged-composition-fri-verifier-prequery-relation A B x y}
       $\subseteq ?values$ 
  unfolding checked-staged-composition-fri-verifier-prequery-relation-def
  by force
  have finite-pairs: finite ?pairs
  using finite-B by simp
  have card
    {y. checked-staged-composition-fri-verifier-prequery-relation A B x y}
     $\leq \text{card } ?values$ 
  by (rule card-mono[OF finite-imageI[OF finite-pairs] subset])
  also have ...  $\leq \text{card } ?pairs$ 
  by (rule card-image-le) (rule finite-pairs)
  also have ... =
    ( $\sum dg \in (UNIV :: 'f \ set).$ 
      card (B dg) * ceil-log (maxDegree + 1))
  using finite-B by (simp add: card-SigmaI)
  finally show ?thesis .
qed

lemma checked-staged-transcript-trace-fri-verifier-prequery-relation-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
shows
  wp-event (checked-staged-transcript-program A)
    (hash-relation-hit-event
      (checked-staged-trace-fri-verifier-prequery-relation A B)
      adversary-initial-state)
  adversary-initial-state  $\leq$ 
    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof –
  have program:
    hash-relation-program
      (checked-staged-trace-fri-verifier-prequery-relation A B)
      (card B * ceil-log clength)
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
      (checked-staged-transcript-program A)

```

```

    by (rule hash-relation-program-checked-staged-transcript-program
        [OF wf controlled])
      (rule checked-staged-trace-fri-verifier-prequery-relation-fiber-card-bound
        [OF finite-B])
  then show ?thesis
    unfolding hash-relation-program-def hash-relation-budget-def by blast
qed

lemma
  checked-staged-transcript-composition-fri-verifier-prequery-relation-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
  shows
    wp-event (checked-staged-transcript-program A)
      (hash-relation-hit-event
        (checked-staged-composition-fri-verifier-prequery-relation A B)
        adversary-initial-state)
    adversary-initial-state  $\leq$ 
    hash-relation-budget-value
      ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
        card (B dg) * ceil-log (maxDegree + 1))
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  proof -
    have program:
      hash-relation-program
        (checked-staged-composition-fri-verifier-prequery-relation A B)
        ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
          card (B dg) * ceil-log (maxDegree + 1))
        (staged-attacker-query-budget budgets + staged-challenge-query-budget)
        (checked-staged-transcript-program A)
    by (rule hash-relation-program-checked-staged-transcript-program
        [OF wf controlled])
      (rule
        checked-staged-composition-fri-verifier-prequery-relation-fiber-card-bound
        [OF finite-B])
    then show ?thesis
      unfolding hash-relation-program-def hash-relation-budget-def by blast
  qed

end

context soundness
begin

lemma checked-staged-trace-fri-verifier-prequery-hit-imp-relation-hit:
  assumes builder:
    Some (data, attacker-state)  $\in$ 
      set-dist

```



```

      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
    and verifier:
      Some (result, final-state) ∈
        set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
    and hit:
      trace-fri-challenge-list-prequery-hit
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      B (Some (result, final-state))
  shows
    hash-relation-hit
    (checked-staged-trace-fri-verifier-prequery-relation A B)
    adversary-initial-state attacker-state
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from hit obtain trace-bs dg comp-bs fr roots trace-final as
    composition-roots final rest where
    challenges:
      accepted-fri-challenges ?s (Some (result, final-state))
      trace-bs dg comp-bs
  and header:
    verifier-header-transcript ?s fr roots trace-final as dg
    composition-roots final rest
  and trace-bs-B: trace-bs ∈ B
  and prequeried:
    trace-fri-challenge-path-prequeried ?s fr roots (length roots)
  unfolding trace-fri-challenge-list-prequery-hit-def by blast
  from prequeried obtain i y where
    i-len: i < length roots
  and lookup-s:
    fmlookup (HashMap ?s) (trace-fri-challenge-key-at ?s fr roots i) =
      Some y
  unfolding trace-fri-challenge-path-prequeried-def
  by (auto split: option.splits)
  have lookup-info:
    i < length trace-bs ∧ y = trace-bs ! i
  by (rule verify-monad-trace-fri-prequery-lookup
    [OF verifier challenges header i-len])
    (use lookup-s in ‹simp add: trace-fri-challenge-key-at-def›)
  have i-round: i < ceil-log clength
proof -
  have trace-bs ∈ fri-challenge-space (ceil-log clength)
  by (rule accepted-fri-challenges-trace-space[OF challenges])

```

```

then have length trace-bs = ceil-log clength
  unfolding fri-challenge-space-def by simp
then show ?thesis
  using lookup-info by simp
qed
have relation:
  checked-staged-trace-fri-verifier-prequery-relation A B
  (trace-fri-challenge-key-at ?s fr roots i) y
unfolding checked-staged-trace-fri-verifier-prequery-relation-def
apply (rule exI[where x = data])
apply (rule exI[where x = attacker-state])
apply (rule exI[where x = trace-bs])
apply (rule exI[where x = fr])
apply (rule exI[where x = roots])
apply (rule exI[where x = i])
using builder trace-bs-B i-round i-len lookup-info by simp
have initial-absent:
  fmlookup (HashMap adversary-initial-state)
  (trace-fri-challenge-key-at ?s fr roots i) = None
unfolding adversary-initial-state-def by simp
have attacker-lookup:
  fmlookup (HashMap attacker-state)
  (trace-fri-challenge-key-at ?s fr roots i) = Some y
using lookup-s unfolding verifier-state-from-adversary-def by simp
show ?thesis
unfolding hash-relation-hit-def
using initial-absent attacker-lookup relation by blast
qed

lemma checked-staged-composition-fri-verifier-prequery-hit-imp-relation-hit:
assumes builder:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
and verifier:
  Some (result, final-state) ∈
    set-dist
    (execute verify-monad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))))
and hit:
  composition-fri-challenge-list-prequery-hit
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  B (Some (result, final-state))
shows
  hash-relation-hit
  (checked-staged-composition-fri-verifier-prequery-relation A B)

```

```

    adversary-initial-state attacker-state
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from hit obtain trace-bs dg comp-bs fr trace-roots trace-final as
    composition-roots final rest where
    challenges:
      accepted-fri-challenges ?s (Some (result, final-state))
      trace-bs dg comp-bs
  and header:
    verifier-header-transcript ?s fr trace-roots trace-final as dg
    composition-roots final rest
  and comp-bs-B: comp-bs ∈ B dg
  and prequeried:
    composition-fri-challenge-path-prequeried ?s fr trace-roots
    trace-final as dg composition-roots (length composition-roots)
  unfolding composition-fri-challenge-list-prequery-hit-def by blast
  from prequeried obtain i y where
    i-len: i < length composition-roots
  and lookup-s:
    fmlookup (HashMap ?s)
    (composition-fri-challenge-key-at ?s fr trace-roots trace-final as
      dg composition-roots i) =
      Some y
  unfolding composition-fri-challenge-path-prequeried-def
  by (auto split: option.splits)
  have lookup-info:
    i < length comp-bs ∧ y = comp-bs ! i
  by (rule verify-monad-composition-fri-prequery-lookup
    [OF verifier challenges header i-len])
    (use lookup-s in
      ⟨simp add: composition-fri-challenge-key-at-def⟩)
  have i-round: i < ceil-log (maxDegree + 1)
proof -
  have comp-bs ∈ fri-challenge-space (ceil-log (to-nat dg + 1))
    by (rule accepted-fri-challenges-composition-space[OF challenges])
  then have len-comp: length comp-bs = ceil-log (to-nat dg + 1)
    unfolding fri-challenge-space-def by simp
  have dg-bound: to-nat dg ≤ maxDegree
    by (rule verify-monad-accepted-fri-challenges-degree-bound
      [OF verifier challenges])
  have ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
    by (rule ceil-log-to-nat-degree-bound[OF dg-bound])
  then show ?thesis
    using lookup-info len-comp by simp
qed
  have relation:
    checked-staged-composition-fri-verifier-prequery-relation A B

```

```

    (composition-fri-challenge-key-at ?s fr trace-roots trace-final as dg
      composition-roots i) y
  unfolding checked-staged-composition-fri-verifier-prequery-relation-def
  apply (rule exI[where x = data])
  apply (rule exI[where x = attacker-state])
  apply (rule exI[where x = dg])
  apply (rule exI[where x = comp-bs])
  apply (rule exI[where x = fr])
  apply (rule exI[where x = trace-roots])
  apply (rule exI[where x = trace-final])
  apply (rule exI[where x = as])
  apply (rule exI[where x = composition-roots])
  apply (rule exI[where x = i])
  using builder comp-bs-B i-round i-len lookup-info by simp
have initial-absent:
  fmlookup (HashMap adversary-initial-state)
    (composition-fri-challenge-key-at ?s fr trace-roots trace-final as dg
      composition-roots i) = None
  unfolding adversary-initial-state-def by simp
have attacker-lookup:
  fmlookup (HashMap attacker-state)
    (composition-fri-challenge-key-at ?s fr trace-roots trace-final as dg
      composition-roots i) = Some y
  using lookup-s unfolding verifier-state-from-adversary-def by simp
show ?thesis
  unfolding hash-relation-hit-def
  using initial-absent attacker-lookup relation by blast
qed

lemma checked-staged-security-trace-fri-verifier-prequery-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B: finite B
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-challenge-list-prequery-hit s B))
      adversary-initial-state ≤
      hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  unfolding checked-staged-security-experiment-with-data-state-def
proof (rule wp-event-bind-bound-by-head-event)
  show wp-event (checked-staged-transcript-program A)
    (hash-relation-hit-event
      (checked-staged-trace-fri-verifier-prequery-relation A B)
      adversary-initial-state)
  adversary-initial-state
  ≤ hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)

```

```

    by (rule checked-staged-transcript-trace-fri-verifier-prequery-relation-bound
        [OF wf controlled finite-B])
next
show staged-security-with-data-state-verifier-event
  (λs. trace-fri-challenge-list-prequery-hit s B) None ⇒
hash-relation-hit-event
  (checked-staged-trace-fri-verifier-prequery-relation A B)
  adversary-initial-state None
unfolding staged-security-with-data-state-verifier-event-def by simp
next
fix data attacker-state out
assume builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and cont:
  out ∈
    set-dist
      (execute
        (get ≫=
          (λs. put
            (verifier-state-from-adversary s
              (staged-proof-transcript data)) ≫=
            (λ-. verify-monad ≫=
              (λresult. return ((data, s), result))))))
        attacker-state)
and hit:
  staged-security-with-data-state-verifier-event
    (λs. trace-fri-challenge-list-prequery-hit s B) out
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from cont hit obtain result final-state where
  verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
and prequery:
  trace-fri-challenge-list-prequery-hit ?s B
  (Some (result, final-state))
unfolding staged-security-with-data-state-verifier-event-def
by (auto elim!: set-dist-bindE split: option.splits prod.splits)
have relation-hit:
  hash-relation-hit
    (checked-staged-trace-fri-verifier-prequery-relation A B)
    adversary-initial-state attacker-state
  by (rule checked-staged-trace-fri-verifier-prequery-hit-imp-relation-hit
      [OF builder verifier prequery])
then show
  hash-relation-hit-event

```

```

    (checked-staged-trace-fri-verifier-prequery-relation A B)
    adversary-initial-state (Some (data, attacker-state))
  unfolding hash-relation-hit-event-def by simp
qed

lemma checked-staged-security-composition-fri-verifier-prequery-hit-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. \text{composition-fri-challenge-list-prequery-hit } s \ B$ ))
    adversary-initial-state  $\leq$ 
    hash-relation-budget-value
      ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
         $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1)$ )
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  unfolding checked-staged-security-experiment-with-data-state-def
proof (rule wp-event-bind-bound-by-head-event)
  show wp-event (checked-staged-transcript-program A)
    (hash-relation-hit-event
      (checked-staged-composition-fri-verifier-prequery-relation A B)
      adversary-initial-state)
  adversary-initial-state
   $\leq$  hash-relation-budget-value
    ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
       $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1)$ )
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule
    checked-staged-transcript-composition-fri-verifier-prequery-relation-bound
    [OF wf controlled finite-B])
next
  show staged-security-with-data-state-verifier-event
    ( $\lambda s. \text{composition-fri-challenge-list-prequery-hit } s \ B$ ) None  $\implies$ 
    hash-relation-hit-event
      (checked-staged-composition-fri-verifier-prequery-relation A B)
      adversary-initial-state None
  unfolding staged-security-with-data-state-verifier-event-def by simp
next
  fix data attacker-state out
  assume builder:
    Some (data, attacker-state)  $\in$ 
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and cont:
    out  $\in$ 
    set-dist

```

```

    (execute
      (get >>=
        (λs. put
          (verifier-state-from-adversary s
            (staged-proof-transcript data)) >>=
            (λ-. verify-monad >>=
              (λresult. return ((data, s), result))))))
      attacker-state)
  and hit:
    staged-security-with-data-state-verifier-event
      (λs. composition-fri-challenge-list-prequery-hit s B) out
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from cont hit obtain result final-state where
    verifier:
      Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  and prequery:
    composition-fri-challenge-list-prequery-hit ?s B
      (Some (result, final-state))
  unfolding staged-security-with-data-state-verifier-event-def
  by (auto elim!: set-dist-bindE split: option.splits prod.splits)
  have relation-hit:
    hash-relation-hit
      (checked-staged-composition-fri-verifier-prequery-relation A B)
      adversary-initial-state attacker-state
  by (rule
    checked-staged-composition-fri-verifier-prequery-hit-imp-relation-hit
      [OF builder verifier prequery])
  then show
    hash-relation-hit-event
      (checked-staged-composition-fri-verifier-prequery-relation A B)
      adversary-initial-state (Some (data, attacker-state))
  unfolding hash-relation-hit-event-def by simp
qed

lemma checked-staged-trace-fri-prequery-hit-imp-fixed-relation-hit:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and hit:
    checked-staged-trace-fri-prequery-hit B
      (Some (data, attacker-state))
  shows
    hash-relation-hit

```

```

    (checked-staged-trace-fri-prequery-relation A B)
    adversary-initial-state attacker-state
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have member: staged-trace-fri-challenges data ∈ B
    using hit unfolding checked-staged-trace-fri-prequery-hit-def by simp
  have prequeried:
    trace-fri-challenge-path-prequeried ?s
    (staged-trace-root data) (staged-trace-fri-roots data)
    (length (staged-trace-fri-roots data))
    using hit unfolding checked-staged-trace-fri-prequery-hit-def by simp
  from prequeried obtain i where
    i-len: i < length (staged-trace-fri-roots data)
    and lookup-s:
      fmlookup (HashMap ?s)
      (trace-fri-challenge-key-at ?s (staged-trace-root data)
        (staged-trace-fri-roots data) i) ≠ None
    unfolding trace-fri-challenge-path-prequeried-def by blast
  have shape:
    length (staged-trace-fri-roots data) = ceil-log clength ∧
    length (staged-trace-fri-challenges data) = ceil-log clength ∧
    length (staged-alphas data) = length spec ∧
    length (staged-composition-fri-roots data) =
      ceil-log (to-nat (staged-degree data) + 1) ∧
    length (staged-composition-fri-challenges data) =
      ceil-log (to-nat (staged-degree data) + 1) ∧
    ceil-log (to-nat (staged-degree data) + 1) ≤
      ceil-log (maxDegree + 1) ∧
    length (staged-query-chunks data) = rounds
    by (rule checked-staged-transcript-program-outcome-shape
      [OF wf controlled outcome])
  have unchecked-out:
    Some (data, attacker-state) ∈
      set-dist (execute (staged-transcript-program A)
        adversary-initial-state)
    by (rule checked-staged-transcript-program-outcome-imp-staged-transcript-program
      [OF outcome])
  have staged-lookup:
    fmlookup (HashMap attacker-state)
    (TraceFriChallenge i
      (foldl concat (staged-trace-fri-start-hash data)
        (take (Suc i) (staged-trace-fri-roots data)))) =
      Some (staged-trace-fri-challenges data ! i)
    by (rule staged-transcript-program-outcome-trace-challenge-lookup
      [OF wf controlled unchecked-out])
    (use i-len shape in simp)
  have key-eq:

```



```

    trace-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) i =
      TraceFriChallenge i
      (foldl concat (staged-trace-fri-start-hash data)
        (take (Suc i) (staged-trace-fri-roots data)))
  unfolding trace-fri-challenge-key-at-def staged-trace-fri-start-hash-def
  by simp
have relation:
  checked-staged-trace-fri-prequery-relation A B
    (trace-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) i)
    (staged-trace-fri-challenges data ! i)
  unfolding checked-staged-trace-fri-prequery-relation-def
  by (intro exI[of - data] exI[of - attacker-state] exI[of - i])
    (use outcome member i-len shape in simp-all)
have initial-absent:
  fmllookup (HashMap adversary-initial-state)
    (trace-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) i) = None
  unfolding adversary-initial-state-def by simp
have staged-lookup-key:
  fmllookup (HashMap attacker-state)
    (trace-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) i) =
    Some (staged-trace-fri-challenges data ! i)
  using staged-lookup key-eq by simp
show ?thesis
  unfolding hash-relation-hit-def
  using initial-absent staged-lookup-key relation by blast
qed

lemma checked-staged-composition-fri-prequery-hit-imp-fixed-relation-hit:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and hit:
    checked-staged-composition-fri-prequery-hit B
      (Some (data, attacker-state))
  shows
    hash-relation-hit
      (checked-staged-composition-fri-prequery-relation A B)
      adversary-initial-state attacker-state
proof -
  let ?s =
    verifier-state-from-adversary attacker-state

```

```

    (staged-proof-transcript data)
have member:
    staged-composition-fri-challenges data ∈ B (staged-degree data)
    using hit unfolding checked-staged-composition-fri-prequery-hit-def
    by simp
have prequeried:
    composition-fri-challenge-path-prequeried ?s
    (staged-trace-root data) (staged-trace-fri-roots data)
    (staged-trace-final data) (staged-alphas data) (staged-degree data)
    (staged-composition-fri-roots data)
    (length (staged-composition-fri-roots data))
    using hit unfolding checked-staged-composition-fri-prequery-hit-def
    by simp
from prequeried obtain i where
    i-len: i < length (staged-composition-fri-roots data)
    and lookup-s:
    fmlookup (HashMap ?s)
    (composition-fri-challenge-key-at ?s (staged-trace-root data)
    (staged-trace-fri-roots data) (staged-trace-final data)
    (staged-alphas data) (staged-degree data)
    (staged-composition-fri-roots data) i) ≠ None
    unfolding composition-fri-challenge-path-prequeried-def by blast
have shape:
    length (staged-trace-fri-roots data) = ceil-log clength ∧
    length (staged-trace-fri-challenges data) = ceil-log clength ∧
    length (staged-alphas data) = length spec ∧
    length (staged-composition-fri-roots data) =
    ceil-log (to-nat (staged-degree data) + 1) ∧
    length (staged-composition-fri-challenges data) =
    ceil-log (to-nat (staged-degree data) + 1) ∧
    ceil-log (to-nat (staged-degree data) + 1) ≤
    ceil-log (maxDegree + 1) ∧
    length (staged-query-chunks data) = rounds
    by (rule checked-staged-transcript-program-outcome-shape
    [OF wf controlled outcome])
have unchecked-out:
    Some (data, attacker-state) ∈
    set-dist (execute (staged-transcript-program A)
    adversary-initial-state)
    by (rule checked-staged-transcript-program-outcome-imp-staged-transcript-program
    [OF outcome])
have staged-lookup:
    fmlookup (HashMap attacker-state)
    (CompositionFriChallenge i
    (foldl concat (staged-composition-fri-start-hash data)
    (take (Suc i) (staged-composition-fri-roots data)))) =
    Some (staged-composition-fri-challenges data ! i)
    by (rule staged-transcript-program-outcome-composition-challenge-lookup
    [OF wf controlled unchecked-out])

```

```

    (use i-len shape in simp)
have key-eq:
  composition-fri-challenge-key-at ?s (staged-trace-root data)
    (staged-trace-fri-roots data) (staged-trace-final data)
    (staged-alphas data) (staged-degree data)
    (staged-composition-fri-roots data) i =
    CompositionFriChallenge i
    (foldl concat (staged-composition-fri-start-hash data)
      (take (Suc i) (staged-composition-fri-roots data)))
unfolding composition-fri-challenge-key-at-def
  staged-composition-fri-start-hash-def staged-trace-fri-start-hash-def
by simp
have relation:
  checked-staged-composition-fri-prequery-relation A B
    (composition-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) (staged-trace-final data)
      (staged-alphas data) (staged-degree data)
      (staged-composition-fri-roots data) i)
    (staged-composition-fri-challenges data ! i)
unfolding checked-staged-composition-fri-prequery-relation-def
by (intro exI[of - data] exI[of - attacker-state] exI[of - i])
    (use outcome member i-len shape in simp-all)
have initial-absent:
  fmlookup (HashMap adversary-initial-state)
    (composition-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) (staged-trace-final data)
      (staged-alphas data) (staged-degree data)
      (staged-composition-fri-roots data) i) = None
unfolding adversary-initial-state-def by simp
have staged-lookup-key:
  fmlookup (HashMap attacker-state)
    (composition-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) (staged-trace-final data)
      (staged-alphas data) (staged-degree data)
      (staged-composition-fri-roots data) i) =
    Some (staged-composition-fri-challenges data ! i)
using staged-lookup key-eq by simp
show ?thesis
unfolding hash-relation-hit-def
using initial-absent staged-lookup-key relation by blast
qed

lemma checked-staged-transcript-trace-fri-prequery-hit-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
shows
  wp-event (checked-staged-transcript-program A)
    (checked-staged-trace-fri-prequery-hit B)

```

```

    adversary-initial-state ≤
    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof –
  have prehit-le-relation:
    wp-event (checked-staged-transcript-program A)
    (checked-staged-trace-fri-prequery-hit B)
    adversary-initial-state ≤
    wp-event (checked-staged-transcript-program A)
    (hash-relation-hit-event
    (checked-staged-trace-fri-prequery-relation A B)
    adversary-initial-state)
    adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈ set-dist
    (execute (checked-staged-transcript-program A)
    adversary-initial-state)
  and hit: checked-staged-trace-fri-prequery-hit B out
show
  hash-relation-hit-event
  (checked-staged-trace-fri-prequery-relation A B)
  adversary-initial-state out
proof (cases out)
  case None
  then show ?thesis
    using hit unfolding checked-staged-trace-fri-prequery-hit-def by simp
next
  case (Some result)
  then obtain data attacker-state where out-eq:
    out = Some (data, attacker-state)
  by (cases result) simp
  have relation-hit:
    hash-relation-hit
    (checked-staged-trace-fri-prequery-relation A B)
    adversary-initial-state attacker-state
  by (rule checked-staged-trace-fri-prequery-hit-imp-fixed-relation-hit
    [OF wf controlled - -])
    (use support hit out-eq in simp-all)
  then show ?thesis
    unfolding out-eq hash-relation-hit-event-def by simp
qed
qed
also have ... ≤
  hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof –
  have program:

```

```

hash-relation-program
  (checked-staged-trace-fri-prequery-relation A B)
  (card B * ceil-log clength)
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  (checked-staged-transcript-program A)
  by (rule hash-relation-program-checked-staged-transcript-program
      [OF wf controlled])
      (rule checked-staged-trace-fri-prequery-relation-fiber-card-bound
        [OF finite-B])
  show ?thesis
  using program
  unfolding hash-relation-program-def hash-relation-budget-def by blast
qed
finally show ?thesis .
qed

```

lemma *checked-staged-transcript-composition-fri-prequery-hit-bound:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: $\bigwedge dg. \text{finite } (B \text{ } dg)$*

shows

wp-event (checked-staged-transcript-program A)
(checked-staged-composition-fri-prequery-hit B)
adversary-initial-state $\leq$
hash-relation-budget-value
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \text{ } dg) * \text{ceil-log } (\text{maxDegree} + 1))$
(staged-attacker-query-budget budgets + staged-challenge-query-budget)

proof –

have *prehit-le-relation:*

wp-event (checked-staged-transcript-program A)
(checked-staged-composition-fri-prequery-hit B)
adversary-initial-state $\leq$
wp-event (checked-staged-transcript-program A)
(hash-relation-hit-event
(checked-staged-composition-fri-prequery-relation A B)
adversary-initial-state)
adversary-initial-state

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support:*

out $\in$ set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)

and *hit: checked-staged-composition-fri-prequery-hit B out*

show

hash-relation-hit-event
(checked-staged-composition-fri-prequery-relation A B)
adversary-initial-state out

```

proof (cases out)
  case None
  then show ?thesis
    using hit unfolding checked-staged-composition-fri-prequery-hit-def
    by simp
next
  case (Some result)
  then obtain data attacker-state where out-eq:
    out = Some (data, attacker-state)
    by (cases result) simp
  have relation-hit:
    hash-relation-hit
      (checked-staged-composition-fri-prequery-relation A B)
      adversary-initial-state attacker-state
    by (rule
      checked-staged-composition-fri-prequery-hit-imp-fixed-relation-hit
      [OF wf controlled - -])
      (use support hit out-eq in simp-all)
  then show ?thesis
    unfolding out-eq hash-relation-hit-event-def by simp
qed
qed
also have ...  $\leq$ 
  hash-relation-budget-value
    ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
      card (B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof -
  have program:
    hash-relation-program
      (checked-staged-composition-fri-prequery-relation A B)
      ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
        card (B dg) * ceil-log (maxDegree + 1))
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
      (checked-staged-transcript-program A)
    by (rule hash-relation-program-checked-staged-transcript-program
      [OF wf controlled])
      (rule
        checked-staged-composition-fri-prequery-relation-fiber-card-bound
        [OF finite-B])
  show ?thesis
    using program
    unfolding hash-relation-program-def hash-relation-budget-def by blast
qed
finally show ?thesis .
qed

end

```

context *soundness*
begin

lemma *checked-staged-trace-fri-prequery-hit-imp-relation-hit*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *outcome*:
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *hit*:
checked-staged-trace-fri-prequery-hit B
(Some (data, attacker-state))
shows
hash-relation-hit
(trace-fri-challenge-key-relation
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
B (length (staged-trace-fri-roots data)))
adversary-initial-state attacker-state
proof –
let *?s* =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
have *member*: *staged-trace-fri-challenges data ∈ B*
using *hit unfolding checked-staged-trace-fri-prequery-hit-def* **by** *simp*
have *prequeried*:
trace-fri-challenge-path-prequeried ?s
(staged-trace-root data) (staged-trace-fri-roots data)
(length (staged-trace-fri-roots data))
using *hit unfolding checked-staged-trace-fri-prequery-hit-def* **by** *simp*
from *prequeried* **obtain** *i* **where**
i-len: *i < length (staged-trace-fri-roots data)*
and *lookup-s*:
fmlookup (HashMap ?s)
(trace-fri-challenge-key-at ?s (staged-trace-root data)
(staged-trace-fri-roots data) i) ≠ None
unfolding *trace-fri-challenge-path-prequeried-def* **by** *blast*
have *shape*:
length (staged-trace-fri-roots data) = ceil-log clength ∧
length (staged-trace-fri-challenges data) = ceil-log clength ∧
length (staged-alphas data) = length spec ∧
length (staged-composition-fri-roots data) =
ceil-log (to-nat (staged-degree data) + 1) ∧
length (staged-composition-fri-challenges data) =
ceil-log (to-nat (staged-degree data) + 1) ∧
ceil-log (to-nat (staged-degree data) + 1) ≤
ceil-log (maxDegree + 1) ∧

```

    length (staged-query-chunks data) = rounds
  by (rule checked-staged-transcript-program-outcome-shape
      [OF wf controlled outcome])
  have unchecked-out:
    Some (data, attacker-state) ∈
      set-dist (execute (staged-transcript-program A)
        adversary-initial-state)
  by (rule checked-staged-transcript-program-outcome-imp-staged-transcript-program
      [OF outcome])
  have staged-lookup:
    fmlookup (HashMap attacker-state)
      (TraceFriChallenge i
        (foldl concat (staged-trace-fri-start-hash data)
          (take (Suc i) (staged-trace-fri-roots data)))) =
      Some (staged-trace-fri-challenges data ! i)
  by (rule staged-transcript-program-outcome-trace-challenge-lookup
      [OF wf controlled unchecked-out])
    (use i-len shape in simp)
  have key-eq:
    trace-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) i =
      TraceFriChallenge i
        (foldl concat (staged-trace-fri-start-hash data)
          (take (Suc i) (staged-trace-fri-roots data)))
  unfolding trace-fri-challenge-key-at-def staged-trace-fri-start-hash-def
  by simp
  have relation:
    trace-fri-challenge-key-relation ?s B
      (length (staged-trace-fri-roots data))
      (trace-fri-challenge-key-at ?s (staged-trace-root data)
        (staged-trace-fri-roots data) i)
      (staged-trace-fri-challenges data ! i)
  proof -
    have i-challenges: i < length (staged-trace-fri-challenges data)
      using i-len shape by simp
    show ?thesis
      unfolding trace-fri-challenge-key-relation-def
      by (intro bexI[of - staged-trace-fri-challenges data]
          exI[of - staged-trace-root data]
          exI[of - staged-trace-fri-roots data]
          exI[of - i])
        (use member i-len i-challenges in simp-all)
  qed
  have initial-absent:
    fmlookup (HashMap adversary-initial-state)
      (trace-fri-challenge-key-at ?s (staged-trace-root data)
        (staged-trace-fri-roots data) i) = None
  unfolding adversary-initial-state-def by simp
  have staged-lookup-key:

```



```

    fmlookup (HashMap attacker-state)
      (trace-fri-challenge-key-at ?s (staged-trace-root data)
        (staged-trace-fri-roots data) i) =
      Some (staged-trace-fri-challenges data ! i)
    using staged-lookup key-eq by simp
  show ?thesis
    unfolding hash-relation-hit-def
    using initial-absent staged-lookup-key relation by blast
qed

lemma checked-staged-composition-fri-prequery-hit-imp-relation-hit:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and hit:
    checked-staged-composition-fri-prequery-hit B
      (Some (data, attacker-state))
  shows
    hash-relation-hit
      (composition-fri-challenge-key-relation
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      B (length (staged-composition-fri-roots data)))
      adversary-initial-state attacker-state
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have member:
    staged-composition-fri-challenges data ∈ B (staged-degree data)
  using hit unfolding checked-staged-composition-fri-prequery-hit-def
  by simp
  have prequeried:
    composition-fri-challenge-path-prequeried ?s
      (staged-trace-root data) (staged-trace-fri-roots data)
      (staged-trace-final data) (staged-alphas data) (staged-degree data)
      (staged-composition-fri-roots data)
      (length (staged-composition-fri-roots data))
  using hit unfolding checked-staged-composition-fri-prequery-hit-def
  by simp
  from prequeried obtain i where
    i-len: i < length (staged-composition-fri-roots data)
  and lookup-s:
    fmlookup (HashMap ?s)
      (composition-fri-challenge-key-at ?s (staged-trace-root data)

```

```

    (staged-trace-fri-roots data) (staged-trace-final data)
    (staged-alphas data) (staged-degree data)
    (staged-composition-fri-roots data) i) ≠ None
unfolding composition-fri-challenge-path-prequeried-def by blast
have shape:
  length (staged-trace-fri-roots data) = ceil-log clength ∧
  length (staged-trace-fri-challenges data) = ceil-log clength ∧
  length (staged-alphas data) = length spec ∧
  length (staged-composition-fri-roots data) =
    ceil-log (to-nat (staged-degree data) + 1) ∧
  length (staged-composition-fri-challenges data) =
    ceil-log (to-nat (staged-degree data) + 1) ∧
  ceil-log (to-nat (staged-degree data) + 1) ≤
    ceil-log (maxDegree + 1) ∧
  length (staged-query-chunks data) = rounds
by (rule checked-staged-transcript-program-outcome-shape
  [OF wf controlled outcome])
have unchecked-out:
  Some (data, attacker-state) ∈
    set-dist (execute (staged-transcript-program A)
      adversary-initial-state)
by (rule checked-staged-transcript-program-outcome-imp-staged-transcript-program
  [OF outcome])
have staged-lookup:
  fmlookup (HashMap attacker-state)
    (CompositionFriChallenge i
      (foldl concat (staged-composition-fri-start-hash data)
        (take (Suc i) (staged-composition-fri-roots data)))) =
    Some (staged-composition-fri-challenges data ! i)
by (rule staged-transcript-program-outcome-composition-challenge-lookup
  [OF wf controlled unchecked-out])
  (use i-len shape in simp)
have key-eq:
  composition-fri-challenge-key-at ?s (staged-trace-root data)
    (staged-trace-fri-roots data) (staged-trace-final data)
    (staged-alphas data) (staged-degree data)
    (staged-composition-fri-roots data) i =
  CompositionFriChallenge i
    (foldl concat (staged-composition-fri-start-hash data)
      (take (Suc i) (staged-composition-fri-roots data)))
unfolding composition-fri-challenge-key-at-def
  staged-composition-fri-start-hash-def staged-trace-fri-start-hash-def
by simp
have relation:
  composition-fri-challenge-key-relation ?s B
    (length (staged-composition-fri-roots data))
    (composition-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) (staged-trace-final data)
      (staged-alphas data) (staged-degree data))

```

```

      (staged-composition-fri-roots data) i)
    (staged-composition-fri-challenges data ! i)
  proof -
    have i-challenges: i < length (staged-composition-fri-challenges data)
      using i-len shape by simp
    show ?thesis
      unfolding composition-fri-challenge-key-relation-def
      by (intro exI[of - staged-degree data]
          bexI[of - staged-composition-fri-challenges data]
          exI[of - staged-trace-root data]
          exI[of - staged-trace-fri-roots data]
          exI[of - staged-trace-final data]
          exI[of - staged-alphas data]
          exI[of - staged-composition-fri-roots data]
          exI[of - i])
        (use member i-len i-challenges in simp-all)
    qed
  have initial-absent:
    fmllookup (HashMap adversary-initial-state)
      (composition-fri-challenge-key-at ?s (staged-trace-root data)
        (staged-trace-fri-roots data) (staged-trace-final data)
        (staged-alphas data) (staged-degree data)
        (staged-composition-fri-roots data) i) = None
    unfolding adversary-initial-state-def by simp
  have staged-lookup-key:
    fmllookup (HashMap attacker-state)
      (composition-fri-challenge-key-at ?s (staged-trace-root data)
        (staged-trace-fri-roots data) (staged-trace-final data)
        (staged-alphas data) (staged-degree data)
        (staged-composition-fri-roots data) i) =
      Some (staged-composition-fri-challenges data ! i)
    using staged-lookup key-eq by simp
  show ?thesis
    unfolding hash-relation-hit-def
    using initial-absent staged-lookup-key relation by blast
  qed

end

end

```

```

theory Soundness-FRI-Prequery-Challenge-Bounds
  imports Soundness-FRI-Prequery-Core
begin

```

Staged and verifier-local FRI challenge-list accounting built on the prequery core definitions and relation bounds.

```

context soundness

```

begin

definition *trace-fri-root-path-fresh-from*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow \text{bool}$

where

$\text{trace-fri-root-path-fresh-from } s \text{ roots } n \longleftrightarrow$

$(\forall i < n.$

$i < \text{length roots} \longrightarrow$

$\text{fmlookup } (\text{HashMap } s)$

$(\text{TraceFriChallenge } (P\text{TraceFriCounter } s + i)$

$(\text{foldl concat } (P\text{State } s) (\text{take } (\text{Suc } i) \text{ roots}))) = \text{None})$

definition *composition-fri-root-path-fresh-from*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow \text{bool}$

where

$\text{composition-fri-root-path-fresh-from } s \text{ roots } n \longleftrightarrow$

$(\forall i < n.$

$i < \text{length roots} \longrightarrow$

$\text{fmlookup } (\text{HashMap } s)$

$(\text{CompositionFriChallenge } (P\text{CompositionFriCounter } s + i)$

$(\text{foldl concat } (P\text{State } s) (\text{take } (\text{Suc } i) \text{ roots}))) = \text{None})$

lemma *trace-fri-challenge-path-fresh-after-read-iff:*

assumes *read-fr:* $\text{Some } (fr, s1) \in \text{set-dist } (\text{execute read } s)$

shows

$\text{trace-fri-challenge-path-fresh } s \text{ fr roots } n \longleftrightarrow$

$\text{trace-fri-root-path-fresh-from } s1 \text{ roots } n$

proof –

have *state:* $P\text{State } s1 = \text{concat } (P\text{State } s) \text{ fr}$

using *read-outcome*[*OF read-fr*] **by** *blast*

have *counter:* $P\text{TraceFriCounter } s1 = P\text{TraceFriCounter } s$

using *read-outcome*[*OF read-fr*] **by** *blast*

have *hash:* $\text{HashMap } s1 = \text{HashMap } s$

by (*rule read-preserves-hash-map*[*OF read-fr*])

show *?thesis*

unfolding *trace-fri-challenge-path-fresh-def*

trace-fri-root-path-fresh-from-def

by (*simp add:* *trace-fri-challenge-key-at-def state counter hash*)

qed

lemma *receive-trace-fri-commits-preserves-trace-root-path-fresh-tail:*

assumes *path:*

$\text{trace-fri-root-path-fresh-from } s \text{ } (r \# \text{ roots}) \text{ } (\text{Suc } n)$

and *outcome:*

$\text{Some } ((b, r), t) \in \text{set-dist } (\text{execute receive-trace-fri-commits } s)$

shows $\text{trace-fri-root-path-fresh-from } t \text{ roots } n$

proof –

from *outcome* **obtain** *s-read* **where**

read-r: $\text{Some } (r, s\text{-read}) \in \text{set-dist } (\text{execute read } s)$

```

and rand-b:
  Some (b, t) ∈ set-dist (execute receive-trace-fri-challenge s-read)
unfolding receive-trace-fri-commits-def receive-fri-commits-with-def
by (auto elim!:: set-dist-bindE)
from read-outcome[OF read-r] have read-state:
  PState s-read = concat (PState s) r
by blast
from read-outcome[OF read-r] have read-counter:
  PTraceFriCounter s-read = PTraceFriCounter s
by blast
have read-hash: HashMap s-read = HashMap s
by (rule read-preserves-hash-map[OF read-r])
have counter-t:
  PTraceFriCounter t = Suc (PTraceFriCounter s)
using receive-trace-fri-challenge-counter-outcome[OF rand-b]
  read-counter by simp
have state-t: PState t = concat (PState s) r
using receive-trace-fri-challenge-outcome[OF rand-b] read-state by simp
show ?thesis
unfolding trace-fri-root-path-fresh-from-def
proof (intro allI impI)
fix i
assume i-lt: i < n
and i-len: i < length roots
let ?key =
  TraceFriChallenge (PTraceFriCounter t + i)
  (foldl concat (PState t) (take (Suc i) roots))
have key-eq:
  ?key =
    TraceFriChallenge (PTraceFriCounter s + Suc i)
    (foldl concat (PState s) (take (Suc (Suc i)) (r # roots)))
using counter-t state-t by simp
have neg:
  ?key ≠ TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)
using counter-t read-counter by simp
have fmlookup (HashMap t) ?key =
  fmlookup (HashMap s-read) ?key
by (rule receive-trace-fri-challenge-preserves-other-lookup
  [OF rand-b neg])
also have ... = fmlookup (HashMap s) ?key
using read-hash by simp
also have ... = None
proof –
have Suc i < Suc n
using i-lt by simp
moreover have Suc i < length (r # roots)
using i-len by simp
ultimately show ?thesis
using path unfolding trace-fri-root-path-fresh-from-def key-eq

```

```

    by blast
qed
finally show fmlookup (HashMap t) ?key = None .
qed
qed

lemma wp-receive-trace-fri-commits-actual-fresh-set:
shows
  wp-event receive-trace-fri-commits
    (λout. case out of
      None ⇒ False
    | Some ((b, r), -) ⇒
      b ∈ B ∧
      fmlookup (HashMap s)
        (TraceFriChallenge (PTraceFriCounter s)
          (concat (PState s) r)) = None) s
    ≤ nnreal (card B) / nnreal size
  unfolding receive-trace-fri-commits-def receive-fri-commits-with-def
proof (rule wp-event-bind-bound-by-cont)
  show ¬ (case None of None ⇒ False
    | Some ((b, r), -) ⇒
      b ∈ B ∧
      fmlookup (HashMap s)
        (TraceFriChallenge (PTraceFriCounter s)
          (concat (PState s) r)) = None)
    by simp
next
  fix r s-read
  assume read-r: Some (r, s-read) ∈ set-dist (execute read s)
  have read-state: PState s-read = concat (PState s) r
    using read-outcome[OF read-r] by blast
  have read-counter: PTraceFriCounter s-read = PTraceFriCounter s
    using read-outcome[OF read-r] by blast
  have read-hash: HashMap s-read = HashMap s
    by (rule read-preserves-hash-map[OF read-r])
  let ?fresh =
    fmlookup (HashMap s-read)
      (TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)) = None
  let ?Q =
    λout :: (('f × 'f) × ('f, 'a) protocol-channel-scheme) option.
      case out of
        None ⇒ False
      | Some ((b, r'), -) ⇒
        b ∈ B ∧
        fmlookup (HashMap s)
          (TraceFriChallenge (PTraceFriCounter s)
            (concat (PState s) r')) = None
  show wp-event
    (receive-trace-fri-challenge ≫ (λb. return (b, r))) ?Q

```

```

    s-read
    ≤ nnreal (card B) / nnreal size
proof (cases ?fresh)
case True
have fresh-key:
  fmlookup (HashMap s)
    (TraceFriChallenge (PTraceFriCounter s) (concat (PState s) r)) =
    None
using True by (simp add: read-state read-counter read-hash)
have cont-eq:
  (λout. case out of
    None ⇒ (0 :: prob)
    | Some (b, u) ⇒
      wp (return (b, r)) (λout. if ?Q out then 1 else 0) u) =
  (λout :: ('f × ('f, 'a) protocol-channel-scheme) option.
    if (case out of None ⇒ False | Some (b, -) ⇒ b ∈ B)
    then 1 else 0)
by (rule ext)
  (auto simp: wpsimps fresh-key split: option.splits prod.splits)
have wp-event
  (receive-trace-fri-challenge ≫= (λb. return (b, r))) ?Q
  s-read =
  wp-event receive-trace-fri-challenge
  (λout. case out of None ⇒ False | Some (b, -) ⇒ b ∈ B)
  s-read
unfolding wp-event-def by (simp add: wpsimps cont-eq)
also have ... = nnreal (card B) / nnreal size
by (rule wp-receive-trace-fri-challenge-fresh-set[OF True])
finally show ?thesis by simp
next
case False
have not-fresh-s:
  fmlookup (HashMap s)
    (TraceFriChallenge (PTraceFriCounter s) (concat (PState s) r))
    ≠ None
using False by (simp add: read-state read-counter read-hash)
have wp-event
  (receive-trace-fri-challenge ≫= (λb. return (b, r))) ?Q
  s-read
  ≤ wp-event
  (receive-trace-fri-challenge ≫= (λb. return (b, r)))
  (λ- :: (('f × 'f) ×
    ('f, 'a) protocol-channel-scheme) option. False) s-read
by (rule wp-event-mono-on-support)
  (use not-fresh-s in
    ⟨auto simp: wpsimps elim!: set-dist-bindE
      split: option.splits prod.splits⟩)
also have ... = 0
unfolding wp-event-def wp-def dist-expect-def by simp

```

```

finally have zero-bound:
  wp-event
    (receive-trace-fri-challenge  $\gg$  ( $\lambda b$ . return (b, r))) ?Q
    s-read  $\leq 0$  .
also have ...  $\leq$  nnreal (card B) / nnreal size
  by simp
finally show ?thesis .
qed
qed

lemma wp-ntimes-receive-trace-fri-commits-exact-challenges-actual-fresh-bound:
assumes len: length bs = n
shows
  wp-event (ntimes receive-trace-fri-commits n)
    ( $\lambda$ out. case out of
      None  $\Rightarrow$  False
    | Some (brs, -)  $\Rightarrow$ 
      map fst brs = bs  $\wedge$ 
      trace-fri-root-path-fresh-from s (map snd brs) n) s
     $\leq (1 / \text{nnreal size})^n$ 
using len
proof (induction n arbitrary: s bs)
case 0
then have bs-empty: bs = []
by simp
show ?case
  unfolding bs-empty by (simp add: wp-event-def wpsimps
    trace-fri-root-path-fresh-from-def)
next
case (Suc n)
obtain b bs' where bs-eq: bs = b # bs'
using Suc.prem by (cases bs) auto
have len-bs': length bs' = n
using Suc.prem bs-eq by simp
let ?tail =
  (ntimes receive-trace-fri-commits n ::
    (('f  $\times$  'f) list, ('f, 'a) protocol-channel-scheme) state-monad)
let ?Q =
   $\lambda$ out :: (('f  $\times$  'f) list)  $\times$ 
    ('f, 'a) protocol-channel-scheme) option.
    case out of
      None  $\Rightarrow$  False
    | Some (brs, -)  $\Rightarrow$ 
      map fst brs = bs  $\wedge$ 
      trace-fri-root-path-fresh-from s (map snd brs) (Suc n)
let ?Head =
   $\lambda$ out :: (('f  $\times$  'f)  $\times$  ('f, 'a) protocol-channel-scheme) option.
    case out of
      None  $\Rightarrow$  False

```



```

| Some ((b0, r0), -) ⇒
  b0 ∈ {b} ∧
  fmlookup (HashMap s)
    (TraceFriChallenge (PTraceFriCounter s)
      (concat (PState s) r0)) = None
have head-bound:
  wp-event receive-trace-fri-commits ?Head s ≤ 1 / nnreal size
proof -
  have wp-event receive-trace-fri-commits ?Head s
    ≤ nnreal (card ({b} :: 'f set)) / nnreal size
    by (rule wp-receive-trace-fri-commits-actual-fresh-set)
  then show ?thesis by simp
qed
have bind-bound:
  wp-event
    (receive-trace-fri-commits >>=
      (λbr :: 'f × 'f.
        ?tail >>= (λbrs :: ('f × 'f) list. return (br # brs))))
    ?Q s
  ≤ wp-event receive-trace-fri-commits ?Head s *
    (1 / nnreal size) ^ n
proof (rule wp-event-bind-bound-by-head-and-cont[where P = ?Head])
  show ¬ ?Q None
  by simp
next
fix br t
assume not-head: ¬ ?Head (Some (br, t))
obtain b0 r0 where br-eq: br = (b0, r0)
  by (cases br)
have cont-false:
  wp-event (?tail >>= (λbrs :: ('f × 'f) list. return (br # brs)))
    ?Q t
  ≤ wp-event (?tail >>= (λbrs :: ('f × 'f) list. return (br # brs)))
    (λ- :: ((('f × 'f) list) ×
      ('f, 'a) protocol-channel-scheme) option. False) t
  by (rule wp-event-mono-on-support)
    (use not-head bs-eq br-eq in
      ⟨auto simp: wpsimps trace-fri-root-path-fresh-from-def
        elim!: set-dist-bindE split: option.splits prod.splits⟩)
also have ... = 0
  unfolding wp-event-def wp-def dist-expect-def by simp
finally have le-zero:
  wp-event (?tail >>= (λbrs :: ('f × 'f) list. return (br # brs)))
    ?Q t ≤ 0 .
show
  wp-event (?tail >>= (λbrs :: ('f × 'f) list. return (br # brs)))
    ?Q t = 0
  by (rule antisym[OF le-zero]) simp
next

```

```

fix br t
assume head:
  Some (br, t) ∈ set-dist (execute receive-trace-fri-commits s)
  and head-hit: ?Head (Some (br, t))
obtain b0 r0 where br-eq: br = (b0, r0)
  by (cases br)
have b0-eq: b0 = b
  using head-hit unfolding br-eq by simp
let ?TailQ =
  λout :: (((f × f) list) ×
    (f, 'a) protocol-channel-scheme) option.
    case out of
      None ⇒ False
    | Some (brs, -) ⇒
      map fst brs = bs' ∧
      trace-fri-root-path-fresh-from t (map snd brs) n
have cont-le:
  wp-event (?tail ≫ (λbrs :: (f × f) list. return (br # brs)))
  ?Q t ≤ wp-event ?tail ?TailQ t
proof (rule wp-event-bind-bound-by-head-event)
  show wp-event ?tail ?TailQ t ≤ wp-event ?tail ?TailQ t
  by simp
next
  show ?Q None ⇒ ?TailQ None
  by simp
next
  fix brs u out
  assume tail-out:
    Some (brs, u) ∈ set-dist (execute ?tail t)
  and ret-out:
    out ∈ set-dist (execute (return (br # brs)) u)
  and hit: ?Q out
  have out-eq: out = Some (br # brs, u)
  using ret-out
  unfolding set-dist-def return.rep-eq dist-return-def
    dist-delta-dist delta-map-def by simp
  have map-eq: map fst brs = bs'
  using hit out-eq b0-eq bs-eq br-eq by simp
  have path-full:
    trace-fri-root-path-fresh-from s (map snd (br # brs)) (Suc n)
  using hit out-eq by simp
  have path-full':
    trace-fri-root-path-fresh-from s (r0 # map snd brs) (Suc n)
  using path-full unfolding br-eq by simp
  have path-tail:
    trace-fri-root-path-fresh-from t (map snd brs) n
  by (rule receive-trace-fri-commits-preserves-trace-root-path-fresh-tail
    [OF path-full' head[unfolded br-eq]])
  show ?TailQ (Some (brs, u))

```

```

    using map-eq path-tail by simp
  qed
  also have ... ≤ (1 / nnreal size) ^ n
    by (rule Suc.IH[OF len-bs])
  finally show
    wp-event (?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs)))
      ?Q t ≤ (1 / nnreal size) ^ n .
  qed
  have wp-event (ntimes receive-trace-fri-commits (Suc n)) ?Q s
    ≤ wp-event receive-trace-fri-commits ?Head s *
      (1 / nnreal size) ^ n
    using bind-bound by simp
  also have ... ≤ (1 / nnreal size) * (1 / nnreal size) ^ n
    by (rule mult-right-mono[OF head-bound]) simp
  also have ... = (1 / nnreal size) ^ Suc n
    by simp
  finally show ?case .
  qed

lemma wp-ntimes-receive-trace-fri-commits-finite-set-actual-fresh-bound:
  assumes finite-B: finite B
  and lengths: ⋀bs. bs ∈ B ⇒ length bs = n
  shows
    wp-event (ntimes receive-trace-fri-commits n)
      (λout. case out of
        None ⇒ False
        | Some (brs, -) ⇒
          map fst brs ∈ B ∧
          trace-fri-root-path-fresh-from s (map snd brs) n) s
      ≤ nnreal (card B) / (nnreal size) ^ n
  proof -
    let ?P =
      λbs out. case out of
        None ⇒ False
        | Some (brs, -) ⇒
          map fst brs = bs ∧
          trace-fri-root-path-fresh-from s (map snd brs) n
    have event-mono:
      wp-event (ntimes receive-trace-fri-commits n)
        (λout. case out of
          None ⇒ False
          | Some (brs, -) ⇒
            map fst brs ∈ B ∧
            trace-fri-root-path-fresh-from s (map snd brs) n) s
        ≤ wp-event (ntimes receive-trace-fri-commits n)
          (λout. ∃ bs ∈ B. ?P bs out) s
      by (rule wp-event-mono) (auto split: option.splits prod.splits)
    also have ... ≤ (∑ bs ∈ B. (1 / nnreal size) ^ n)
      by (rule wp-event-finite-UN-bound[OF finite-B])
  
```

(rule *wp-ntimes-receive-trace-fri-commits-exact-challenges-actual-fresh-bound*
 [OF *lengths*])
also have ... = $\text{nnreal } (\text{card } B) * ((1 / \text{nnreal size}) ^ n)$
using *finite-B* **by** *simp*
also have ... = $\text{nnreal } (\text{card } B) / (\text{nnreal size}) ^ n$
using *size-card* **by** *transfer (simp add: power-divide)*
finally show ?thesis .
qed

lemma *wp-ntimes-receive-trace-fri-commits-challenge-space-actual-fresh-bound*:
assumes *subset*: $B \subseteq \text{fri-challenge-space } n$
shows

wp-event (ntimes receive-trace-fri-commits n)
 (λout. case out of
 None ⇒ False
 | Some (brs, -) ⇒
 map fst brs ∈ B ∧
 trace-fri-root-path-fresh-from s (map snd brs) n) s
 ≤ $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD('f)} ^ n)$

proof –

have *finite-B*: finite B
by (rule *finite-subset*[OF *subset finite-fri-challenge-space*])
have *lengths*: $\bigwedge bs. bs \in B \implies \text{length } bs = n$
using *subset unfolding fri-challenge-space-def* **by** *auto*
have *wp-event (ntimes receive-trace-fri-commits n)*
 (λout. case out of
 None ⇒ False
 | Some (brs, -) ⇒
 map fst brs ∈ B ∧
 trace-fri-root-path-fresh-from s (map snd brs) n) s
 ≤ $\text{nnreal } (\text{card } B) / (\text{nnreal size}) ^ n$
by (rule *wp-ntimes-receive-trace-fri-commits-finite-set-actual-fresh-bound*
 [OF *finite-B lengths*])
also have ... = $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD('f)} ^ n)$
using *size-card* **by** *simp*
finally show ?thesis .
qed

lemma *wp-verifier-trace-fri-prefix-challenge-list-actual-fresh-bound*:
assumes *subset*: $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$
shows

wp-event verifier-trace-fri-prefix
 (λout. case out of
 None ⇒ False
 | Some ((fr, f-fl), -) ⇒
 map fst f-fl ∈ B ∧
 trace-fri-challenge-path-fresh s fr (map snd f-fl)
 (length (map snd f-fl))) s
 ≤ $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD('f)} ^ \text{ceil-log clength})$

```

proof –
  let ?Q =
    λout. case out of
      None ⇒ False
    | Some ((fr, f-fl), -) ⇒
      map fst f-fl ∈ B ∧
      trace-fri-challenge-path-fresh s fr (map snd f-fl)
      (length (map snd f-fl))
  let ?C = nnreal (card B) / nnreal (CARD('f) ^ ceil-log clength)
  show ?thesis
    unfolding verifier-trace-fri-prefix-def
  proof (rule wp-event-bind-bound-by-cont)
    show ¬ ?Q None
      by simp
  next
    fix fr s1
    assume read-fr: Some (fr, s1) ∈ set-dist (execute read s)
    let ?Head =
      λout. case out of
        None ⇒ False
      | Some (f-fl, -) ⇒
        map fst f-fl ∈ B ∧
        trace-fri-root-path-fresh-from s1 (map snd f-fl)
        (ceil-log clength)
    have trace-bound:
      wp-event (ntimes receive-trace-fri-commits (ceil-log clength))
      ?Head s1 ≤ ?C
    by (rule wp-ntimes-receive-trace-fri-commits-challenge-space-actual-fresh-bound
      [OF subset])
    show wp-event
      (ntimes receive-trace-fri-commits (ceil-log clength) ⋈
      (λf-fl. return (fr, f-fl)))
      ?Q s1 ≤ ?C
    proof (rule wp-event-bind-bound-by-head-event[OF trace-bound])
      show ?Q None ⇒ ?Head None
        by simp
    next
      fix f-fl t out
      assume trace-out:
        Some (f-fl, t) ∈
          set-dist
            (execute (ntimes receive-trace-fri-commits (ceil-log clength))
              s1)
      and cont:
        out ∈ set-dist (execute (return (fr, f-fl)) t)
      and hit: ?Q out
      have out-eq: out = Some ((fr, f-fl), t)
      using cont
      unfolding set-dist-def return.rep-eq dist-return-def

```

```

    dist-delta-dist delta-map-def by simp
have len: length f-fl = ceil-log clength
    using ntimes-receive-trace-fri-commits-outcome[OF trace-out] by simp
have path:
    trace-fri-root-path-fresh-from s1 (map snd f-fl)
    (ceil-log clength)
    using hit out-eq len
    trace-fri-challenge-path-fresh-after-read-iff[OF read-fr]
    by simp
show ?Head (Some (f-fl, t))
    using hit out-eq path by simp
qed
qed
qed

lemma wp-verify-monad-trace-fri-challenge-list-fresh-hit-actual-bound:
  assumes subset:  $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$ 
  shows
    wp-event verify-monad (trace-fri-challenge-list-fresh-hit s B)  $s \leq$ 
    nnreal (card B) / nnreal ( $\text{CARD}(f) \wedge \text{ceil-log clength}$ )
proof -
  let ?Head =
     $\lambda \text{out. case out of}$ 
    None  $\Rightarrow$  False
    | Some ((fr, f-fl), -)  $\Rightarrow$ 
      map fst f-fl  $\in B \wedge$ 
      trace-fri-challenge-path-fresh s fr (map snd f-fl)
      (length (map snd f-fl))
  have prefix-bound:
    wp-event verifier-trace-fri-prefix ?Head  $s \leq$ 
    nnreal (card B) / nnreal ( $\text{CARD}(f) \wedge \text{ceil-log clength}$ )
  by (rule wp-verifier-trace-fri-prefix-challenge-list-actual-fresh-bound
    [OF subset])
  show ?thesis
    unfolding verify-monad-trace-fri-decomposition
  proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
  show trace-fri-challenge-list-fresh-hit s B None  $\implies$ 
    ?Head None
    unfolding trace-fri-challenge-list-fresh-hit-def
    accepted-fri-challenges-def by blast
next
  fix header prefix-state out
  assume prefix:
    Some (header, prefix-state)  $\in$ 
    set-dist (execute verifier-trace-fri-prefix s)
  and suffix:
    out  $\in$ 
    set-dist (execute (verifier-after-trace-fri header) prefix-state)
  and hit: trace-fri-challenge-list-fresh-hit s B out

```

```

obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
from hit obtain trace-bs dg comp-bs fr' trace-roots' trace-final' as'
  composition-roots' final' rest' where
    challenges-hit: accepted-fri-challenges s out trace-bs dg comp-bs
  and header-hit:
    verifier-header-transcript s fr' trace-roots' trace-final' as' dg
    composition-roots' final' rest'
  and trace-bs-B: trace-bs ∈ B
  and fresh-hit:
    trace-fri-challenge-path-fresh s fr' trace-roots'
    (length trace-roots')
  unfolding trace-fri-challenge-list-fresh-hit-def by blast
from challenges-hit obtain result final-state where out-eq:
  out = Some (result, final-state)
  unfolding accepted-fri-challenges-def by blast
have prefix':
  Some ((fr, f-fl), prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix s)
  using prefix header-eq by simp
have suffix':
  Some (result, final-state) ∈
    set-dist
    (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
  using suffix header-eq out-eq by simp
from verifier-after-trace-fri-accepted-fri-challenges[OF prefix' suffix']
obtain dg' comp-bs' where
  challenges-prefix:
    accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg' comp-bs'
  by blast
have trace-bs-eq: trace-bs = map fst f-fl
  using accepted-fri-challenges-unique
  [OF challenges-prefix challenges-hit[unfolded out-eq]]
  by simp
from challenges-prefix obtain result0 final-state0 fr0 f-fl0
  trace-final0 as0 fl0 final0 query-state0 where body0:
  Some (result, final-state) = Some (result0, final-state0) ∧
  verifier-header-transcript s fr0 (map snd f-fl0) trace-final0 as0
  dg' (map snd fl0) final0 (PTranscript query-state0) ∧
  PState query-state0 =
  verifier-header-state s fr0 (map snd f-fl0) trace-final0 as0
  dg' (map snd fl0) final0 ∧
  Some (result0, final-state0) ∈
  set-dist (execute
    (ntimes
      (verifier-query-round-program fr0 f-fl0 trace-final0 as0
        fl0 final0)
      rounds)

```

```

    query-state0) ∧
  PQueryCounter query-state0 = PQueryCounter s ∧
  map fst f-fl = map fst f-fl0 ∧
  comp-bs' = map fst fl0 ∧
  (∀ i < length f-fl0.
    fmlookup (HashMap query-state0)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr0)
          (take (Suc i) (map snd f-fl0)))) =
    Some (fst (f-fl0 ! i))) ∧
  (∀ i < length fl0.
    fmlookup (HashMap query-state0)
      (CompositionFriChallenge (PCompositionFriCounter s + i)
        (foldl concat
          (concat
            (foldl concat
              (concat
                (foldl concat (concat (PState s) fr0)
                  (map snd f-fl0))
                trace-final0)
              as0)
            dg')
          (take (Suc i) (map snd fl0)))) =
    Some (fst (fl0 ! i)))
unfolding accepted-fri-challenges-def by blast
have header-prefix0:
  verifier-header-transcript s fr0 (map snd f-fl0) trace-final0 as0
  dg' (map snd fl0) final0 (PTranscript query-state0)
using body0 by simp
have trace-bs0: map fst f-fl = map fst f-fl0
using body0 by simp
have prefix-res:
  PTranscript s = [fr] @ map snd f-fl @ PTranscript prefix-state
using verifier-trace-fri-prefix-outcome[OF prefix'] by simp
have len-f-fl: length f-fl = ceil-log clength
using verifier-trace-fri-prefix-outcome[OF prefix'] by simp
have fr0-eq: fr0 = fr
using prefix-res header-prefix0
unfolding verifier-header-transcript-def verifier-header-messages-def
by simp
have len-f-fl0: length f-fl0 = ceil-log clength
using header-prefix0
unfolding verifier-header-transcript-def by simp
have roots0-eq: map snd f-fl0 = map snd f-fl
proof -
  let ?rest0 =
    trace-final0 # as0 @ dg' # map snd fl0 @ final0 #
    PTranscript query-state0
have prefix0:

```



```

    PTranscript s = [fr] @ map snd f-fl0 @ ?rest0
    using header-prefix0 fr0-eq
    unfolding verifier-header-transcript-def verifier-header-messages-def
    by simp
  have eq:
    ([fr] @ map snd f-fl @ PTranscript prefix-state =
     [fr] @ map snd f-fl0 @ ?rest0)
    using prefix-res prefix0 by simp
  have take-eq:
    take (Suc (length (map snd f-fl)))
      ([fr] @ map snd f-fl @ PTranscript prefix-state) =
    take (Suc (length (map snd f-fl)))
      ([fr] @ map snd f-fl0 @ ?rest0)
    using eq by simp
  then show ?thesis
    using len-f-fl len-f-fl0 by simp
qed
have header-prefix:
  verifier-header-transcript s fr (map snd f-fl) trace-final0 as0
  dg' (map snd fl0) final0 (PTranscript query-state0)
  using header-prefix0 fr0-eq roots0-eq by simp
have roots-eq:
  fr' = fr  $\wedge$  trace-roots' = map snd f-fl
  using verifier-header-transcript-unique[OF header-prefix header-hit]
  by simp
then have fresh-prefix:
  trace-fri-challenge-path-fresh s fr (map snd f-fl)
  (length (map snd f-fl))
  using fresh-hit by simp
show ?Head (Some (header, prefix-state))
  using trace-bs-B trace-bs-eq fresh-prefix header-eq by simp
qed
qed

lemma receive-composition-fri-commits-preserves-composition-root-path-fresh-tail:
  assumes path:
    composition-fri-root-path-fresh-from s (r # roots) (Suc n)
  and outcome:
    Some ((b, r), t)  $\in$  set-dist (execute receive-composition-fri-commits s)
  shows composition-fri-root-path-fresh-from t roots n
proof -
  from outcome obtain s-read where
    read-r: Some (r, s-read)  $\in$  set-dist (execute read s)
  and rand-b:
    Some (b, t)  $\in$ 
      set-dist (execute receive-composition-fri-challenge s-read)
  unfolding receive-composition-fri-commits-def receive-fri-commits-with-def
  by (auto elim!: set-dist-bindE)
  from read-outcome[OF read-r] have read-state:

```

```

PState s-read = concat (PState s) r
by blast
from read-outcome[OF read-r] have read-counter:
  PCompositionFriCounter s-read = PCompositionFriCounter s
by blast
have read-hash: HashMap s-read = HashMap s
by (rule read-preserves-hash-map[OF read-r])
have counter-t:
  PCompositionFriCounter t = Suc (PCompositionFriCounter s)
using receive-composition-fri-challenge-counter-outcome[OF rand-b]
  read-counter by simp
have state-t: PState t = concat (PState s) r
using receive-composition-fri-challenge-outcome[OF rand-b] read-state
by simp
show ?thesis
  unfolding composition-fri-root-path-fresh-from-def
proof (intro allI impI)
  fix i
  assume i-lt: i < n
  and i-len: i < length roots
  let ?key =
    CompositionFriChallenge (PCompositionFriCounter t + i)
    (foldl concat (PState t) (take (Suc i) roots))
  have key-eq:
    ?key =
      CompositionFriChallenge (PCompositionFriCounter s + Suc i)
      (foldl concat (PState s) (take (Suc (Suc i)) (r # roots)))
  using counter-t state-t by simp
  have neg:
    ?key ≠
      CompositionFriChallenge (PCompositionFriCounter s-read)
      (PState s-read)
  using counter-t read-counter by simp
  have fmllookup (HashMap t) ?key =
    fmllookup (HashMap s-read) ?key
  by (rule receive-composition-fri-challenge-preserves-other-lookup
    [OF rand-b neg])
  also have ... = fmllookup (HashMap s) ?key
  using read-hash by simp
  also have ... = None
proof -
  have Suc i < Suc n
  using i-lt by simp
  moreover have Suc i < length (r # roots)
  using i-len by simp
  ultimately show ?thesis
  using path unfolding composition-fri-root-path-fresh-from-def key-eq
  by blast
qed

```

finally show $\text{fmlookup } (\text{HashMap } t) \text{ ?key} = \text{None} .$
qed
qed

lemma *wp-receive-composition-fri-commits-actual-fresh-set:*
shows
 $\text{wp-event receive-composition-fri-commits}$
 $(\lambda \text{out. case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } ((b, r), -) \Rightarrow$
 $\quad \quad b \in B \wedge$
 $\quad \quad \text{fmlookup } (\text{HashMap } s)$
 $\quad \quad (\text{CompositionFriChallenge } (\text{PCompositionFriCounter } s)$
 $\quad \quad \quad (\text{concat } (\text{PState } s) \text{ } r)) = \text{None}) \text{ } s$
 $\leq \text{nnreal } (\text{card } B) / \text{nnreal size}$

unfolding *receive-composition-fri-commits-def receive-fri-commits-with-def*

proof (rule *wp-event-bind-bound-by-cont*)
show $\neg (\text{case None of None} \Rightarrow \text{False})$
 $| \text{Some } ((b, r), -) \Rightarrow$
 $\quad b \in B \wedge$
 $\quad \text{fmlookup } (\text{HashMap } s)$
 $\quad (\text{CompositionFriChallenge } (\text{PCompositionFriCounter } s)$
 $\quad \quad (\text{concat } (\text{PState } s) \text{ } r)) = \text{None})$

by simp

next
fix $r \text{ s-read}$
assume $\text{read-r: Some } (r, \text{s-read}) \in \text{set-dist } (\text{execute read } s)$
have $\text{read-state: PState s-read} = \text{concat } (\text{PState } s) \text{ } r$
using $\text{read-outcome[OF read-r]}$ **by blast**
have read-counter:
 $\text{PCompositionFriCounter s-read} = \text{PCompositionFriCounter } s$
using $\text{read-outcome[OF read-r]}$ **by blast**
have $\text{read-hash: HashMap s-read} = \text{HashMap } s$
by (rule $\text{read-preserves-hash-map[OF read-r]}$)
let $\text{?fresh} =$
 $\text{fmlookup } (\text{HashMap s-read})$
 $(\text{CompositionFriChallenge } (\text{PCompositionFriCounter s-read})$
 $\quad (\text{PState s-read})) = \text{None}$

let $\text{?Q} =$
 $\lambda \text{out} :: ((f \times f) \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option.}$
 case out of
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } ((b, r'), -) \Rightarrow$
 $\quad \quad b \in B \wedge$
 $\quad \quad \text{fmlookup } (\text{HashMap } s)$
 $\quad \quad (\text{CompositionFriChallenge } (\text{PCompositionFriCounter } s)$
 $\quad \quad \quad (\text{concat } (\text{PState } s) \text{ } r')) = \text{None}$

show wp-event
 $(\text{receive-composition-fri-challenge} \gg (\lambda b. \text{return } (b, r))) \text{ ?Q}$

```

    s-read
    ≤ nnreal (card B) / nnreal size
proof (cases ?fresh)
case True
have fresh-key:
  fmlookup (HashMap s)
    (CompositionFriChallenge (PCompositionFriCounter s)
      (concat (PState s) r)) = None
using True by (simp add: read-state read-counter read-hash)
have cont-eq:
  (λout. case out of
    None ⇒ (0 :: prob)
    | Some (b, u) ⇒
      wp (return (b, r)) (λout. if ?Q out then 1 else 0) u) =
    (λout :: ('f × ('f, 'a) protocol-channel-scheme) option.
      if (case out of None ⇒ False | Some (b, -) ⇒ b ∈ B)
      then 1 else 0)
by (rule ext)
  (auto simp: wpsimps fresh-key split: option.splits prod.splits)
have wp-event
  (receive-composition-fri-challenge ≫ (λb. return (b, r))) ?Q
  s-read =
  wp-event receive-composition-fri-challenge
    (λout. case out of None ⇒ False | Some (b, -) ⇒ b ∈ B)
    s-read
unfolding wp-event-def by (simp add: wpsimps cont-eq)
also have ... = nnreal (card B) / nnreal size
by (rule wp-receive-composition-fri-challenge-fresh-set[OF True])
finally show ?thesis by simp
next
case False
have not-fresh-s:
  fmlookup (HashMap s)
    (CompositionFriChallenge (PCompositionFriCounter s)
      (concat (PState s) r)) ≠ None
using False by (simp add: read-state read-counter read-hash)
have wp-event
  (receive-composition-fri-challenge ≫ (λb. return (b, r))) ?Q
  s-read
  ≤ wp-event
    (receive-composition-fri-challenge ≫ (λb. return (b, r)))
    (λ- :: (('f × 'f) ×
      ('f, 'a) protocol-channel-scheme) option. False) s-read
by (rule wp-event-mono-on-support)
  (use not-fresh-s in
    ⟨auto simp: wpsimps elim!: set-dist-bindE
      split: option.splits prod.splits⟩)
also have ... = 0
unfolding wp-event-def wp-def dist-expect-def by simp

```

```

finally have zero-bound:
  wp-event
    (receive-composition-fri-challenge  $\gg$  ( $\lambda b. \text{return } (b, r)$ )) ?Q
  s-read  $\leq 0$  .
also have ...  $\leq \text{nnreal } (\text{card } B) / \text{nnreal size}$ 
  by simp
finally show ?thesis .
qed
qed

lemma wp-ntimes-receive-composition-fri-commits-exact-challenges-actual-fresh-bound:
assumes len: length bs = n
shows
  wp-event (ntimes receive-composition-fri-commits n)
    ( $\lambda \text{out. case out of}$ 
      None  $\Rightarrow$  False
      | Some (brs, -)  $\Rightarrow$ 
        map fst brs = bs  $\wedge$ 
        composition-fri-root-path-fresh-from s (map snd brs) n) s
     $\leq (1 / \text{nnreal size})^n$ 
using len
proof (induction n arbitrary: s bs)
case 0
then have bs-empty: bs = []
  by simp
show ?case
  unfolding bs-empty by (simp add: wp-event-def wpsimps
    composition-fri-root-path-fresh-from-def)
next
case (Suc n)
obtain b bs' where bs-eq: bs = b # bs'
  using Suc.premis by (cases bs) auto
have len-bs': length bs' = n
  using Suc.premis bs-eq by simp
let ?tail =
  (ntimes receive-composition-fri-commits n ::
    (('f  $\times$  'f) list, ('f, 'a) protocol-channel-scheme) state-monad)
let ?Q =
   $\lambda \text{out} :: (((\text{'f} \times \text{'f}) \text{ list}) \times$ 
    ('f, 'a) protocol-channel-scheme) option.
    case out of
      None  $\Rightarrow$  False
      | Some (brs, -)  $\Rightarrow$ 
        map fst brs = bs  $\wedge$ 
        composition-fri-root-path-fresh-from s (map snd brs) (Suc n)
let ?Head =
   $\lambda \text{out} :: (((\text{'f} \times \text{'f}) \times (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ option.}$ 
    case out of
      None  $\Rightarrow$  False

```

```

| Some ((b0, r0), -) =>
  b0 ∈ {b} ∧
  fmlookup (HashMap s)
    (CompositionFriChallenge (PCompositionFriCounter s)
      (concat (PState s) r0)) = None
have head-bound:
  wp-event receive-composition-fri-commits ?Head s ≤ 1 / nnreal size
proof -
  have wp-event receive-composition-fri-commits ?Head s
    ≤ nnreal (card ({b} :: 'f set)) / nnreal size
  by (rule wp-receive-composition-fri-commits-actual-fresh-set)
  then show ?thesis by simp
qed
have bind-bound:
  wp-event
    (receive-composition-fri-commits >>=
      (λbr :: 'f × 'f.
        ?tail >>= (λbrs :: ('f × 'f) list. return (br # brs))))
  ?Q s
  ≤ wp-event receive-composition-fri-commits ?Head s *
    (1 / nnreal size) ^ n
proof (rule wp-event-bind-bound-by-head-and-cont[where P = ?Head])
  show ¬ ?Q None
  by simp
next
fix br t
assume not-head: ¬ ?Head (Some (br, t))
obtain b0 r0 where br-eq: br = (b0, r0)
  by (cases br)
have cont-false:
  wp-event (?tail >>= (λbrs :: ('f × 'f) list. return (br # brs)))
  ?Q t
  ≤ wp-event (?tail >>= (λbrs :: ('f × 'f) list. return (br # brs)))
    (λ- :: (((('f × 'f) list) ×
      ('f, 'a) protocol-channel-scheme) option. False) t
  by (rule wp-event-mono-on-support)
    (use not-head bs-eq br-eq in
      ⟨auto simp: wpsimps composition-fri-root-path-fresh-from-def
        elim!: set-dist-bindE split: option.splits prod.splits⟩)
also have ... = 0
  unfolding wp-event-def wp-def dist-expect-def by simp
finally have le-zero:
  wp-event (?tail >>= (λbrs :: ('f × 'f) list. return (br # brs)))
  ?Q t ≤ 0 .
show
  wp-event (?tail >>= (λbrs :: ('f × 'f) list. return (br # brs)))
  ?Q t = 0
  by (rule antisym[OF le-zero]) simp
next

```

```

fix br t
assume head:
  Some (br, t) ∈
    set-dist (execute receive-composition-fri-commits s)
  and head-hit: ?Head (Some (br, t))
obtain b0 r0 where br-eq: br = (b0, r0)
  by (cases br)
have b0-eq: b0 = b
  using head-hit unfolding br-eq by simp
let ?TailQ =
  λout :: (((f × f) list) ×
    (f, 'a) protocol-channel-scheme) option.
    case out of
      None ⇒ False
    | Some (brs, -) ⇒
      map fst brs = bs' ∧
      composition-fri-root-path-fresh-from t (map snd brs) n
have cont-le:
  wp-event (?tail ≫= (λbrs :: (f × f) list. return (br # brs)))
  ?Q t ≤ wp-event ?tail ?TailQ t
proof (rule wp-event-bind-bound-by-head-event)
  show wp-event ?tail ?TailQ t ≤ wp-event ?tail ?TailQ t
  by simp
next
  show ?Q None ⇒ ?TailQ None
  by simp
next
  fix brs u out
  assume tail-out:
    Some (brs, u) ∈ set-dist (execute ?tail t)
  and ret-out:
    out ∈ set-dist (execute (return (br # brs)) u)
  and hit: ?Q out
  have out-eq: out = Some (br # brs, u)
  using ret-out
  unfolding set-dist-def return.rep-eq dist-return-def
    dist-delta-dist delta-map-def by simp
  have map-eq: map fst brs = bs'
  using hit out-eq b0-eq bs-eq br-eq by simp
  have path-full:
    composition-fri-root-path-fresh-from s (map snd (br # brs))
    (Suc n)
  using hit out-eq by simp
  have path-full':
    composition-fri-root-path-fresh-from s (r0 # map snd brs)
    (Suc n)
  using path-full unfolding br-eq by simp
  have path-tail:
    composition-fri-root-path-fresh-from t (map snd brs) n

```

```

    by (rule
      receive-composition-fri-commits-preserves-composition-root-path-fresh-tail
      [OF path-full' head[unfolded br-eq]])
  show ?TailQ (Some (brs, u))
    using map-eq path-tail by simp
qed
also have ... ≤ (1 / nnreal size) ^ n
  by (rule Suc.IH[OF len-bs])
finally show
  wp-event (?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs)))
    ?Q t ≤ (1 / nnreal size) ^ n .
qed
have wp-event (ntimes receive-composition-fri-commits (Suc n)) ?Q s
  ≤ wp-event receive-composition-fri-commits ?Head s *
    (1 / nnreal size) ^ n
  using bind-bound by simp
also have ... ≤ (1 / nnreal size) * (1 / nnreal size) ^ n
  by (rule mult-right-mono[OF head-bound]) simp
also have ... = (1 / nnreal size) ^ Suc n
  by simp
finally show ?case .
qed

lemma wp-ntimes-receive-composition-fri-commits-finite-set-actual-fresh-bound:
  assumes finite-B: finite B
  and lengths: ⋀bs. bs ∈ B ⇒ length bs = n
  shows
    wp-event (ntimes receive-composition-fri-commits n)
      (λout. case out of
        None ⇒ False
        | Some (brs, -) ⇒
          map fst brs ∈ B ∧
          composition-fri-root-path-fresh-from s (map snd brs) n) s
      ≤ nnreal (card B) / (nnreal size) ^ n
proof -
  let ?P =
    λbs out. case out of
      None ⇒ False
      | Some (brs, -) ⇒
        map fst brs = bs ∧
        composition-fri-root-path-fresh-from s (map snd brs) n
  have event-mono:
    wp-event (ntimes receive-composition-fri-commits n)
      (λout. case out of
        None ⇒ False
        | Some (brs, -) ⇒
          map fst brs ∈ B ∧
          composition-fri-root-path-fresh-from s (map snd brs) n) s
      ≤ wp-event (ntimes receive-composition-fri-commits n)

```



```

    (λout. ∃ bs ∈ B. ?P bs out) s
  by (rule wp-event-mono) (auto split: option.splits prod.splits)
also have ... ≤ (∑ bs ∈ B. (1 / nnreal size) ^ n)
  by (rule wp-event-finite-UN-bound[OF finite-B])
    (rule wp-ntimes-receive-composition-fri-commits-exact-challenges-actual-fresh-bound
      [OF lengths])
also have ... = nnreal (card B) * ((1 / nnreal size) ^ n)
  using finite-B by simp
also have ... = nnreal (card B) / (nnreal size) ^ n
  using size-card by transfer (simp add: power-divide)
finally show ?thesis .
qed

```

lemma *wp-ntimes-receive-composition-fri-commits-challenge-space-actual-fresh-bound:*

assumes *subset: B ⊆ fri-challenge-space n*

shows

```

wp-event (ntimes receive-composition-fri-commits n)
  (λout. case out of
    None ⇒ False
  | Some (brs, -) ⇒
    map fst brs ∈ B ∧
    composition-fri-root-path-fresh-from s (map snd brs) n) s
  ≤ nnreal (card B) / nnreal (CARD('f) ^ n)

```

proof –

```

  have finite-B: finite B
  by (rule finite-subset[OF subset finite-fri-challenge-space])
  have lengths: ⋀ bs. bs ∈ B ⇒ length bs = n
  using subset unfolding fri-challenge-space-def by auto
  have wp-event (ntimes receive-composition-fri-commits n)
    (λout. case out of
      None ⇒ False
    | Some (brs, -) ⇒
      map fst brs ∈ B ∧
      composition-fri-root-path-fresh-from s (map snd brs) n) s
    ≤ nnreal (card B) / (nnreal size) ^ n
  by (rule
    wp-ntimes-receive-composition-fri-commits-finite-set-actual-fresh-bound
      [OF finite-B lengths])
  also have ... = nnreal (card B) / nnreal (CARD('f) ^ n)
  using size-card by simp
  finally show ?thesis .

```

qed

lemma *composition-fri-challenge-path-fresh-before-composition-iff:*

assumes *state:*

```

PState s-comp =
  concat
    (foldl concat
      (concat

```

```

      (foldl concat (concat (PState s) fr) trace-roots)
      trace-final)
    as)
  dg
and counter: PCompositionFriCounter s-comp = PCompositionFriCounter s
and hash: HashMap s-comp = HashMap s
shows
  composition-fri-challenge-path-fresh s fr trace-roots trace-final as dg
  composition-roots n  $\longleftrightarrow$ 
  composition-fri-root-path-fresh-from s-comp composition-roots n
unfolding composition-fri-challenge-path-fresh-def
  composition-fri-root-path-fresh-from-def
by (simp add: composition-fri-challenge-key-at-def state counter hash)

lemma composition-fri-challenge-path-fresh-from-preservedI:
assumes state:
  PState s-comp =
    concat
      (foldl concat
        (concat
          (foldl concat (concat (PState s) fr) trace-roots)
          trace-final)
        as)
      dg
and counter: PCompositionFriCounter s-comp = PCompositionFriCounter s
and lookup:
   $\bigwedge i x. \text{fmlookup (HashMap s-comp) (CompositionFriChallenge i x) =}$ 
    fmlookup (HashMap s) (CompositionFriChallenge i x)
and fresh:
  composition-fri-root-path-fresh-from s-comp composition-roots n
shows
  composition-fri-challenge-path-fresh s fr trace-roots trace-final as dg
  composition-roots n
unfolding composition-fri-challenge-path-fresh-def
  composition-fri-challenge-key-at-def
proof (intro allI impI)
fix i
assume i-lt:  $i < n$ 
and i-len:  $i < \text{length composition-roots}$ 
let ?key =
  CompositionFriChallenge (PCompositionFriCounter s + i)
  (foldl concat
    (concat
      (foldl concat
        (concat (foldl concat (concat (PState s) fr) trace-roots)
        trace-final)
      as)
    dg)
    (take (Suc i) composition-roots))

```

have ?key =
 CompositionFriChallenge (PCompositionFriCounter s-comp + i)
 (foldl concat (PState s-comp) (take (Suc i) composition-roots))
using state counter **by** simp
moreover have
 fmlookup (HashMap s-comp)
 (CompositionFriChallenge (PCompositionFriCounter s-comp + i)
 (foldl concat (PState s-comp) (take (Suc i) composition-roots))) =
 None
using fresh i-lt i-len
unfolding composition-fri-root-path-fresh-from-def **by** blast
ultimately show fmlookup (HashMap s) ?key = None
using lookup[of PCompositionFriCounter s-comp + i
 foldl concat (PState s-comp) (take (Suc i) composition-roots)]
by simp
qed

lemma receive-trace-fri-commits-preserves-composition-fri-lookup:
assumes outcome:
 Some (br, t) ∈ set-dist (execute receive-trace-fri-commits s)
shows
 fmlookup (HashMap t) (CompositionFriChallenge i x) =
 fmlookup (HashMap s) (CompositionFriChallenge i x)
proof –
obtain b r **where** br-eq: br = (b, r)
by (cases br)
from outcome[unfolded br-eq] **obtain** s-read **where**
 read-r: Some (r, s-read) ∈ set-dist (execute read s)
and rand-b:
 Some (b, t) ∈ set-dist (execute receive-trace-fri-challenge s-read)
unfolding receive-trace-fri-commits-def receive-fri-commits-with-def
by (auto elim!: set-dist-bindE)
have read-hash: HashMap s-read = HashMap s
by (rule read-preserves-hash-map[OF read-r])
have neq:
 CompositionFriChallenge i x ≠
 TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)
by simp
have fmlookup (HashMap t) (CompositionFriChallenge i x) =
 fmlookup (HashMap s-read) (CompositionFriChallenge i x)
by (rule receive-trace-fri-challenge-preserves-other-lookup
 [OF rand-b neq])
also have ... = fmlookup (HashMap s) (CompositionFriChallenge i x)
using read-hash **by** simp
finally show ?thesis .
qed

lemma ntimes-receive-trace-fri-commits-preserves-composition-fri-lookup:
assumes outcome:

```

    Some (brs, t) ∈
      set-dist (execute (ntimes receive-trace-fri-commits n) s)
  shows
    fmlookup (HashMap t) (CompositionFriChallenge i x) =
      fmlookup (HashMap s) (CompositionFriChallenge i x)
  using outcome
proof (induction n arbitrary: s brs t)
  case 0
  then show ?case
    by simp
next
  case (Suc n)
  from Suc.prem1 obtain br brs' s1 where
    head: Some (br, s1) ∈ set-dist (execute receive-trace-fri-commits s)
  and tail:
    Some (brs', t) ∈
      set-dist (execute (ntimes receive-trace-fri-commits n) s1)
  by (auto elim!: set-dist-bindE)
  have fmlookup (HashMap t) (CompositionFriChallenge i x) =
    fmlookup (HashMap s1) (CompositionFriChallenge i x)
  by (rule Suc.IH[OF tail])
  also have ... = fmlookup (HashMap s) (CompositionFriChallenge i x)
  by (rule receive-trace-fri-commits-preserves-composition-fri-lookup
      [OF head])
  finally show ?case .
qed

lemma alpha-round-preserves-composition-fri-lookup:
  assumes outcome: Some (a, t) ∈ set-dist (execute alpha-round s)
  shows
    fmlookup (HashMap t) (CompositionFriChallenge i x) =
      fmlookup (HashMap s) (CompositionFriChallenge i x)
proof -
  from outcome obtain a0 s0 a1 s1 s2 where
    rand: Some (a0, s0) ∈ set-dist (execute receive-alpha-challenge s)
  and read-a: Some (a1, s1) ∈ set-dist (execute read s0)
  and assert-a: Some ((), s2) ∈ set-dist (execute (assert (a0 = a1)) s1)
  and ret-a: Some (a, t) ∈ set-dist (execute (return a1) s2)
  unfolding alpha-round-def
  by (auto simp: Let-def elim!: set-dist-bindE)
  have neq:
    CompositionFriChallenge i x ≠
      AlphaChallenge (PAlphaCounter s) (PState s)
  by simp
  have fmlookup (HashMap s0) (CompositionFriChallenge i x) =
    fmlookup (HashMap s) (CompositionFriChallenge i x)
  by (rule receive-alpha-challenge-preserves-other-lookup[OF rand neq])
  moreover have HashMap s1 = HashMap s0
  by (rule read-preserves-hash-map[OF read-a])

```

```

moreover have  $s2 = s1$ 
  using assert-a unfolding assert-def
  by (cases a0 = a1) (auto simp: throw-no-outcome)
moreover have  $t = s2$ 
  using ret-a by simp
ultimately show ?thesis
  by simp
qed

lemma mmap-alpha-round-preserves-composition-fri-lookup:
  assumes outcome:
     $\text{Some } (as, t) \in \text{set-dist } (\text{execute } (\text{mmap } (\text{replicate } n \text{ alpha-round}))) \ s$ 
  shows
     $\text{fmlookup } (\text{HashMap } t) \ (\text{CompositionFriChallenge } i \ x) =$ 
     $\text{fmlookup } (\text{HashMap } s) \ (\text{CompositionFriChallenge } i \ x)$ 
  using outcome
proof (induction n arbitrary: s as t)
  case 0
  then show ?case
    by simp
next
  case (Suc n)
  from Suc.prems obtain  $a \ as' \ s1$  where
    head:  $\text{Some } (a, s1) \in \text{set-dist } (\text{execute } \text{alpha-round } s)$ 
    and tail:
       $\text{Some } (as', t) \in \text{set-dist } (\text{execute } (\text{mmap } (\text{replicate } n \text{ alpha-round}))) \ s1$ 
    by (auto elim!: set-dist-bindE)
  have  $\text{fmlookup } (\text{HashMap } t) \ (\text{CompositionFriChallenge } i \ x) =$ 
     $\text{fmlookup } (\text{HashMap } s1) \ (\text{CompositionFriChallenge } i \ x)$ 
    by (rule Suc.IH[OF tail])
  also have  $\dots = \text{fmlookup } (\text{HashMap } s) \ (\text{CompositionFriChallenge } i \ x)$ 
    by (rule alpha-round-preserves-composition-fri-lookup[OF head])
  finally show ?case .
qed

```

```

lemma wp-verifier-composition-fri-prefix-challenge-list-actual-fresh-bound:
  fixes  $C :: \text{prob}$ 
  assumes subset:
     $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$ 
  and bound:
     $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) /$ 
     $\text{nnreal } (CARD('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq C$ 
  shows
    wp-event verifier-composition-fri-prefix
    ( $\lambda out. \text{case out of}$ 
       $\text{None} \Rightarrow \text{False}$ 
       $| \text{Some } ((fr, f-fl, f-final, as, dg, fl), -) \Rightarrow$ 

```

```

      map fst fl ∈ B dg ∧
      composition-fri-challenge-path-fresh s fr (map snd f-fl)
      f-final as dg (map snd fl) (length (map snd fl)) s
    ≤ C
  proof -
    let ?Q =
      λout. case out of
        None ⇒ False
      | Some ((fr, f-fl, f-final, as, dg, fl), -) ⇒
        map fst fl ∈ B dg ∧
        composition-fri-challenge-path-fresh s fr (map snd f-fl)
        f-final as dg (map snd fl) (length (map snd fl))
    show ?thesis
      unfolding verifier-composition-fri-prefix-def
    proof (rule wp-event-bind-bound-by-cont)
      show ¬ ?Q None
        by simp
    next
      fix fr s1
      assume read-fr: Some (fr, s1) ∈ set-dist (execute read s)
      from read-outcome[OF read-fr] have st-s1:
        PState s1 = concat (PState s) fr
        by blast
      from read-outcome[OF read-fr] have comp-count-s1:
        PCompositionFriCounter s1 = PCompositionFriCounter s
        by blast
      have hash-s1: HashMap s1 = HashMap s
        by (rule read-preserves-hash-map[OF read-fr])
      show wp-event
        (ntimes receive-trace-fri-commits (ceil-log clength) ≫=
        (λf-fl. read ≫=
        (λf-final.
        mmap (replicate (length spec) alpha-round) ≫=
        (λas. read ≫=
        (λdg.
        assert (to-nat dg ≤ maxDegree) ≫=
        (λ-. ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1)) ≫=
        (λfl.
        return (fr, f-fl, f-final, as, dg, fl))))))))))
        ?Q s1 ≤ C
    proof (rule wp-event-bind-bound-by-cont)
      show ¬ ?Q None
        by simp
    next
      fix f-fl s2
      assume trace-fri:
        Some (f-fl, s2) ∈
        set-dist

```

```

      (execute (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
have trace-res:
  PState s2 = foldl concat (PState s1) (map snd f-fl) ∧
  PCompositionFriCounter s2 = PCompositionFriCounter s1
using ntimes-receive-trace-fri-commits-outcome[OF trace-fri]
by simp
have lookup-s2:
  ∧i x. fmlookup (HashMap s2) (CompositionFriChallenge i x) =
    fmlookup (HashMap s) (CompositionFriChallenge i x)
proof -
  fix i x
  have fmlookup (HashMap s2) (CompositionFriChallenge i x) =
    fmlookup (HashMap s1) (CompositionFriChallenge i x)
  by (rule ntimes-receive-trace-fri-commits-preserves-composition-fri-lookup
    [OF trace-fri])
  also have ... = fmlookup (HashMap s) (CompositionFriChallenge i x)
  using hash-s1 by simp
  finally show
    fmlookup (HashMap s2) (CompositionFriChallenge i x) =
      fmlookup (HashMap s) (CompositionFriChallenge i x) .
qed
show wp-event
  (read >>=
    (λf-final.
      mmap (replicate (length spec) alpha-round) >>=
        (λas. read >>=
          (λdg.
            assert (to-nat dg ≤ maxDegree) >>=
              (λ-. ntimes receive-composition-fri-commits
                (ceil-log (to-nat dg + 1)) >>=
                  (λfl.
                    return (fr, f-fl, f-final, as, dg, fl)))))))
  ?Q s2 ≤ C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?Q None
  by simp
next
  fix f-final s3
  assume read-final: Some (f-final, s3) ∈ set-dist (execute read s2)
  from read-outcome[OF read-final] have st-s3:
    PState s3 = concat (PState s2) f-final
  by blast
  from read-outcome[OF read-final] have comp-count-s3:
    PCompositionFriCounter s3 = PCompositionFriCounter s2
  by blast
  have hash-s3: HashMap s3 = HashMap s2
  by (rule read-preserves-hash-map[OF read-final])
  have lookup-s3:
    ∧i x. fmlookup (HashMap s3) (CompositionFriChallenge i x) =

```

```

    fmlookup (HashMap s) (CompositionFriChallenge i x)
  using hash-s3 lookup-s2 by simp
show wp-event
  (mmap (replicate (length spec) alpha-round) >>=
    (λas. read >>=
      (λdg.
        assert (to-nat dg ≤ maxDegree) >>=
          (λ-. ntimes receive-composition-fri-commits
            (ceil-log (to-nat dg + 1)) >>=
              (λfl.
                return (fr, f-fl, f-final, as, dg, fl)))))))
  ?Q s3 ≤ C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?Q None
  by simp
next
  fix as s4
  assume alpha:
    Some (as, s4) ∈
      set-dist
        (execute (mmap (replicate (length spec) alpha-round)) s3)
  have alpha-res:
    PState s4 = foldl concat (PState s3) as ∧
    PCompositionFriCounter s4 = PCompositionFriCounter s3
  using mmap-alpha-round-outcome[OF alpha] by simp
  have lookup-s4:
    ∧i x. fmlookup (HashMap s4) (CompositionFriChallenge i x) =
      fmlookup (HashMap s) (CompositionFriChallenge i x)
  proof -
    fix i x
    have fmlookup (HashMap s4) (CompositionFriChallenge i x) =
      fmlookup (HashMap s3) (CompositionFriChallenge i x)
    by (rule mmap-alpha-round-preserves-composition-fri-lookup
      [OF alpha])
    also have ... =
      fmlookup (HashMap s) (CompositionFriChallenge i x)
    by (rule lookup-s3)
    finally show
      fmlookup (HashMap s4) (CompositionFriChallenge i x) =
        fmlookup (HashMap s) (CompositionFriChallenge i x) .
  qed
show wp-event
  (read >>=
    (λdg.
      assert (to-nat dg ≤ maxDegree) >>=
        (λ-. ntimes receive-composition-fri-commits
          (ceil-log (to-nat dg + 1)) >>=
            (λfl.
              return (fr, f-fl, f-final, as, dg, fl))))))

```



```

    ?Q s4 ≤ C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?Q None
    by simp
next
  fix dg s5
  assume read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
  from read-outcome[OF read-dg] have st-s5:
    PState s5 = concat (PState s4) dg
    by blast
  from read-outcome[OF read-dg] have comp-count-s5:
    PCompositionFriCounter s5 = PCompositionFriCounter s4
    by blast
  have hash-s5: HashMap s5 = HashMap s4
    by (rule read-preserves-hash-map[OF read-dg])
  have lookup-s5:
    ∧i x. fmlookup (HashMap s5) (CompositionFriChallenge i x) =
      fmlookup (HashMap s) (CompositionFriChallenge i x)
    using hash-s5 lookup-s4 by simp
  show wp-event
    (assert (to-nat dg ≤ maxDegree) >>=
      (λ-. ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1)) >>=
          (λfl.
            return (fr, f-fl, f-final, as, dg, fl))))
    ?Q s5 ≤ C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?Q None
    by simp
next
  fix u s6
  assume assert-out:
    Some (u, s6) ∈
      set-dist (execute (assert (to-nat dg ≤ maxDegree))) s5)
  have s6-eq: s6 = s5
    using assert-out unfolding assert-def
    by (cases to-nat dg ≤ maxDegree)
      (auto simp: throw-no-outcome)
  have state-s6:
    PState s6 =
      concat
        (foldl concat
          (concat
            (foldl concat (concat (PState s) fr)
              (map snd f-fl))
            f-final)
          as)
        dg
    using st-s1 trace-res st-s3 alpha-res st-s5 s6-eq

```

```

    by simp
  have counter-s6:
    PCompositionFriCounter s6 = PCompositionFriCounter s
    using comp-count-s1 trace-res comp-count-s3 alpha-res
      comp-count-s5 s6-eq
    by simp
  have lookup-s6:
     $\bigwedge i x. \text{fmlookup } (\text{HashMap } s6) \text{ } ( \text{CompositionFriChallenge } i \ x ) =$ 
    fmlookup (HashMap s) (CompositionFriChallenge i x)
    using lookup-s5 s6-eq by simp
  let ?Head =
     $\lambda out. \text{case out of}$ 
    None  $\Rightarrow$  False
    | Some (fl, -)  $\Rightarrow$ 
      map fst fl  $\in B \ dg \wedge$ 
      composition-fri-root-path-fresh-from s6 (map snd fl)
      (ceil-log (to-nat dg + 1))
  have comp-bound:
    wp-event
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1)))
    ?Head s6  $\leq$ 
    nnreal (card (B dg)) /
    nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
    by (rule
      wp-ntimes-receive-composition-fri-commits-challenge-space-actual-fresh-bound
      [OF subset])
  have comp-bound-C:
    wp-event
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1)))
    ?Head s6  $\leq C$ 
    by (rule order-trans[OF comp-bound bound])
  show wp-event
    (ntimes receive-composition-fri-commits
      (ceil-log (to-nat dg + 1))  $\gg=$ 
      ( $\lambda fl.$ 
        return (fr, f-fl, f-final, as, dg, fl)))
    ?Q s6  $\leq C$ 
  proof (rule wp-event-bind-bound-by-head-event[OF comp-bound-C])
    show ?Q None  $\Rightarrow$  ?Head None
    by simp
  next
  fix fl t out
  assume comp-out:
    Some (fl, t)  $\in$ 
    set-dist
    (execute

```

```

      (ntimes receive-composition-fri-commits
       (ceil-log (to-nat dg + 1))) s6)
and cont:
  out ∈
    set-dist
      (execute
        (return (fr, f-fl, f-final, as, dg, fl)) t)
and hit: ?Q out
have out-eq:
  out = Some ((fr, f-fl, f-final, as, dg, fl), t)
  using cont
  unfolding set-dist-def return.rep-eq dist-return-def
    dist-delta-dist delta-map-def
  by simp
have len-fl: length fl = ceil-log (to-nat dg + 1)
  using ntimes-receive-composition-fri-commits-outcome
    [OF comp-out] by simp
have path-fresh-len:
  composition-fri-challenge-path-fresh s fr
    (map snd f-fl) f-final as dg (map snd fl)
    (length fl)
  using hit out-eq by simp
have root-fresh-head:
  composition-fri-root-path-fresh-from s6 (map snd fl)
    (ceil-log (to-nat dg + 1))
  using path-fresh-len len-fl state-s6 counter-s6 lookup-s6
  unfolding composition-fri-challenge-path-fresh-def
    composition-fri-root-path-fresh-from-def
    composition-fri-challenge-key-at-def
  by simp
have root-fresh:
  composition-fri-root-path-fresh-from s6 (map snd fl)
    (ceil-log (Suc (to-nat dg)))
  using root-fresh-head by simp
have path-fresh:
  composition-fri-challenge-path-fresh s fr (map snd f-fl)
    f-final as dg (map snd fl) (length (map snd fl))
  using composition-fri-challenge-path-fresh-from-preservedI
    [OF state-s6 counter-s6 lookup-s6 root-fresh]
    len-fl
  by simp
show ?Head (Some (fl, t))
  using hit out-eq root-fresh-head by simp
qed
qed
qed
qed
qed
qed

```

```

qed
qed

lemma wp-verify-monad-composition-fri-challenge-list-fresh-hit-actual-bound:
  fixes C :: prob
  assumes subset:
     $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$ 
  and bound:
     $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) / \text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq C$ 
  shows
    wp-event verify-monad (composition-fri-challenge-list-fresh-hit s B) s
       $\leq C$ 
proof -
  let ?Head =
     $\lambda out. \text{case out of}$ 
     $\text{None} \Rightarrow \text{False}$ 
     $| \text{Some } ((fr, f-fl, f-final, as, dg, fl), -) \Rightarrow$ 
     $\text{map fst } fl \in B \ dg \wedge$ 
     $\text{composition-fri-challenge-path-fresh } s \ fr \ (\text{map snd } f-fl)$ 
     $f-final \ as \ dg \ (\text{map snd } fl) \ (\text{length } (\text{map snd } fl))$ 
  have prefix-bound:
    wp-event verifier-composition-fri-prefix ?Head s  $\leq C$ 
  by (rule
    wp-verifier-composition-fri-prefix-challenge-list-actual-fresh-bound
    [OF subset bound])
  show ?thesis
    unfolding verify-monad-composition-fri-decomposition
  proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
    show composition-fri-challenge-list-fresh-hit s B None  $\implies$ 
      ?Head None
    unfolding composition-fri-challenge-list-fresh-hit-def
      accepted-fri-challenges-def by blast
  next
    fix header prefix-state out
    assume prefix:
      Some (header, prefix-state)  $\in$ 
      set-dist (execute verifier-composition-fri-prefix s)
    and suffix:
      out  $\in$ 
      set-dist (execute
        (verifier-after-composition-fri header) prefix-state)
    and hit: composition-fri-challenge-list-fresh-hit s B out
    obtain fr f-fl f-final as dg fl where header-eq:
      header = (fr, f-fl, f-final, as, dg, fl)
    by (cases header) auto
    from hit obtain trace-bs dg' comp-bs fr' trace-roots' trace-final'
      as' composition-roots' final' rest' where
      challenges-hit: accepted-fri-challenges s out trace-bs dg' comp-bs

```

and *header-hit*:
verifier-header-transcript s fr' trace-roots' trace-final' as' dg'
composition-roots' final' rest'
and *comp-bs-B*: *comp-bs* $\in B$ *dg'*
and *fresh-hit*:
composition-fri-challenge-path-fresh s fr' trace-roots'
trace-final' as' dg' composition-roots'
(length composition-roots[^])
unfolding *composition-fri-challenge-list-fresh-hit-def* **by** *blast*
from *challenges-hit* **obtain** *result final-state* **where** *out-eq*:
out = Some (result, final-state)
unfolding *accepted-fri-challenges-def* **by** *blast*
have *prefix'*:
Some ((fr, f-fl, f-final, as, dg, fl), prefix-state) $\in$
set-dist (execute verifier-composition-fri-prefix s)
using *prefix header-eq* **by** *simp*
have *suffix'*:
Some (result, final-state) $\in$
set-dist
(execute
(verifier-after-composition-fri
(fr, f-fl, f-final, as, dg, fl))
prefix-state)
using *suffix header-eq out-eq* **by** *simp*
from *suffix'* **obtain** *final query-state* **where**
read-final:
Some (final, query-state) $\in$ set-dist (execute read prefix-state)
and *query-out*:
Some (result, final-state) $\in$
set-dist
(execute
(ntimes
(verifier-query-round-program fr f-fl f-final as fl final)
rounds)
query-state)
unfolding *verifier-after-composition-fri-def*
by *(auto elim!: set-dist-bindE)*
have *prefix-res*:
length f-fl = ceil-log clength $\wedge$
length as = length spec $\wedge$
length fl = ceil-log (to-nat dg + 1) $\wedge$
PTranscript s =
[fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
map snd fl @ PTranscript prefix-state
using *verifier-composition-fri-prefix-outcome[OF prefix']* **by** *simp*
from *read-outcome[OF read-final]* **obtain** *rest-query* **where**
tr-prefix: PTranscript prefix-state = final # rest-query
and *tr-query: PTranscript query-state = rest-query*
by *blast*

```

have header-prefix:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
using prefix-res tr-prefix tr-query
unfolding verifier-header-transcript-def verifier-header-messages-def
by simp
have challenges-prefix:
  accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg (map fst fl)
by (rule verifier-after-composition-fri-accepted-fri-challenges
    [OF prefix' suffix'])
have eqs:
  trace-bs = map fst f-fl  $\wedge$  dg' = dg  $\wedge$  comp-bs = map fst fl
using accepted-fri-challenges-unique
  [OF challenges-prefix challenges-hit[unfolded out-eq]]
by simp
have header-eqs:
  fr' = fr  $\wedge$ 
  trace-roots' = map snd f-fl  $\wedge$ 
  trace-final' = f-final  $\wedge$ 
  as' = as  $\wedge$ 
  composition-roots' = map snd fl
using verifier-header-transcript-unique[OF header-prefix header-hit]
  eqs
by simp
then have fresh-prefix:
  composition-fri-challenge-path-fresh s fr (map snd f-fl)
    f-final as dg (map snd fl) (length (map snd fl))
using fresh-hit eqs by simp
show ?Head (Some (header, prefix-state))
using comp-bs-B eqs fresh-prefix header-eq by simp
qed
qed

```

```

lemma trace-fri-challenge-list-fresh-hit-imp-set-hit:
assumes trace-fri-challenge-list-fresh-hit s B out
shows trace-fri-challenge-list-set-hit s B out
using assms
unfolding trace-fri-challenge-list-fresh-hit-def
  trace-fri-challenge-list-set-hit-def
by blast

```

```

lemma composition-fri-challenge-list-fresh-hit-imp-set-hit:
assumes composition-fri-challenge-list-fresh-hit s B out
shows composition-fri-challenge-list-set-hit s B out
using assms
unfolding composition-fri-challenge-list-fresh-hit-def
  composition-fri-challenge-list-set-hit-def
by blast

```

lemma *wp-trace-fri-challenge-list-fresh-hit-bound-from-future-fresh:*

assumes *future: trace-fri-future-fresh s*

and *subset: B ⊆ fri-challenge-space (ceil-log clength)*

shows

*wp-event verify-monad (trace-fri-challenge-list-fresh-hit s B) s ≤
nnreal (card B) / nnreal (CARD('f) ^ ceil-log clength)*

proof –

have *wp-event verify-monad (trace-fri-challenge-list-fresh-hit s B) s
≤ wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s*

by (rule *wp-event-mono*)

(rule *trace-fri-challenge-list-fresh-hit-imp-set-hit*)

also have ... ≤

nnreal (card B) / nnreal (CARD('f) ^ ceil-log clength)

by (rule *wp-verify-monad-trace-fri-challenge-list-set-bound*
[*OF future subset*])

finally show *?thesis* .

qed

lemma *wp-composition-fri-challenge-list-fresh-hit-bound-from-future-fresh:*

assumes *future: composition-fri-future-fresh s*

and *subset:*

∧ dg. B dg ⊆ fri-challenge-space (ceil-log (to-nat dg + 1))

and *bound:*

*∧ dg. nnreal (card (B dg)) /
nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C*

shows

*wp-event verify-monad (composition-fri-challenge-list-fresh-hit s B) s
≤ C*

proof –

have *wp-event verify-monad (composition-fri-challenge-list-fresh-hit s B) s
≤ wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s*

by (rule *wp-event-mono*)

(rule *composition-fri-challenge-list-fresh-hit-imp-set-hit*)

also have ... ≤ C

by (rule *wp-verify-monad-composition-fri-challenge-list-set-bound*
[*OF future subset bound*])

finally show *?thesis* .

qed

lemma *accepted-fri-challenges-headerE:*

assumes *accepted-fri-challenges s out trace-bs dg comp-bs*

obtains *fr f-fl trace-final alphas fl final rest* **where**

*verifier-header-transcript s fr (map snd f-fl) trace-final alphas dg
(map snd fl) final rest*

trace-bs = map fst f-fl

comp-bs = map fst fl

proof –

from *assms* **obtain** *result final-state fr f-fl trace-final alphas fl final*

```

  query-state where body:
out = Some (result, final-state) ∧
  verifier-header-transcript s fr (map snd f-fl) trace-final alphas dg
  (map snd fl) final (PTranscript query-state) ∧
PState query-state =
  verifier-header-state s fr (map snd f-fl) trace-final alphas dg
  (map snd fl) final ∧
Some (result, final-state) ∈
  set-dist (execute
    (ntimes (verifier-query-round-program fr f-fl trace-final alphas fl final)
      rounds)
    query-state) ∧
PQueryCounter query-state = PQueryCounter s ∧
trace-bs = map fst f-fl ∧
comp-bs = map fst fl ∧
(∀ i < length f-fl.
  fmllookup (HashMap query-state)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr)
        (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))) ∧
(∀ i < length fl.
  fmllookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter s + i)
      (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) (map snd f-fl))
                trace-final)
            alphas)
          dg)
        (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i)))
unfolding accepted-fri-challenges-def by blast
have header:
  verifier-header-transcript s fr (map snd f-fl) trace-final alphas dg
  (map snd fl) final (PTranscript query-state)
using body by simp
have trace-eq: trace-bs = map fst f-fl
using body by simp
have comp-eq: comp-bs = map fst fl
using body by simp
show thesis
by (rule that[of fr f-fl trace-final alphas fl final
  PTranscript query-state,
  OF header trace-eq comp-eq])
qed

```


lemma *trace-fri-challenge-path-fresh-or-prequeried:*
trace-fri-challenge-path-fresh s fr roots n $\vee$
trace-fri-challenge-path-prequeried s fr roots n

proof (cases $\exists i < n. i < \text{length roots} \wedge$
fmlookup (HashMap s) (trace-fri-challenge-key-at s fr roots i) $\neq$
None)
case *True*
then have *trace-fri-challenge-path-prequeried s fr roots n*
unfolding *trace-fri-challenge-path-prequeried-def* .
then show ?thesis **by** *simp*

next
case *False*
then have *trace-fri-challenge-path-fresh s fr roots n*
unfolding *trace-fri-challenge-path-fresh-def*
by *auto*
then show ?thesis **by** *simp*

qed

lemma *composition-fri-challenge-path-fresh-or-prequeried:*
composition-fri-challenge-path-fresh s fr trace-roots trace-final as dg
composition-roots n $\vee$
composition-fri-challenge-path-prequeried s fr trace-roots trace-final as
dg composition-roots n

proof (cases $\exists i < n. i < \text{length composition-roots} \wedge$
fmlookup (HashMap s)
(composition-fri-challenge-key-at s fr trace-roots trace-final as dg
composition-roots i) $\neq$ None)
case *True*
then have *composition-fri-challenge-path-prequeried s fr trace-roots*
trace-final as dg composition-roots n
unfolding *composition-fri-challenge-path-prequeried-def* .
then show ?thesis **by** *simp*

next
case *False*
then have *composition-fri-challenge-path-fresh s fr trace-roots*
trace-final as dg composition-roots n
unfolding *composition-fri-challenge-path-fresh-def*
by *auto*
then show ?thesis **by** *simp*

qed

lemma *trace-fri-challenge-list-set-hit-imp-fresh-or-prequery:*
assumes *trace-fri-challenge-list-set-hit s B out*
shows
trace-fri-challenge-list-fresh-hit s B out $\vee$
trace-fri-challenge-list-prequery-hit s B out

proof –
from *assms* **obtain** *trace-bs dg comp-bs* **where** *hit:*
accepted-fri-challenges s out trace-bs dg comp-bs

```

    trace-bs ∈ B
    unfolding trace-fri-challenge-list-set-hit-def by auto
from hit(1) obtain fr f-fl trace-final alphas fl final rest where
  header:
    verifier-header-transcript s fr (map snd f-fl) trace-final alphas dg
    (map snd fl) final rest
  trace-bs = map fst f-fl
  comp-bs = map fst fl
  by (rule accepted-fri-challenges-headerE)
have split: trace-fri-challenge-path-fresh s fr (map snd f-fl)
  (length (map snd f-fl)) ∨
  trace-fri-challenge-path-prequeried s fr (map snd f-fl)
  (length (map snd f-fl))
  by (rule trace-fri-challenge-path-fresh-or-prequeried)
then show ?thesis
proof
  assume fresh: trace-fri-challenge-path-fresh s fr (map snd f-fl)
  (length (map snd f-fl))
  have trace-fri-challenge-list-fresh-hit s B out
    unfolding trace-fri-challenge-list-fresh-hit-def
    by (intro exI conjI; (rule hit(1) | rule header | rule hit(2) |
      rule fresh))
  then show ?thesis by simp
next
  assume prequeried: trace-fri-challenge-path-prequeried s fr
  (map snd f-fl) (length (map snd f-fl))
  have trace-fri-challenge-list-prequery-hit s B out
    unfolding trace-fri-challenge-list-prequery-hit-def
    by (intro exI conjI; (rule hit(1) | rule header | rule hit(2) |
      rule prequeried))
  then show ?thesis by simp
qed
qed

lemma composition-fri-challenge-list-set-hit-imp-fresh-or-prequery:
  assumes composition-fri-challenge-list-set-hit s B out
  shows
    composition-fri-challenge-list-fresh-hit s B out ∨
    composition-fri-challenge-list-prequery-hit s B out
proof –
  from assms obtain trace-bs dg comp-bs where hit:
    accepted-fri-challenges s out trace-bs dg comp-bs
    comp-bs ∈ B dg
  unfolding composition-fri-challenge-list-set-hit-def by auto
from hit(1) obtain fr f-fl trace-final alphas fl final rest where
  header:
    verifier-header-transcript s fr (map snd f-fl) trace-final alphas dg
    (map snd fl) final rest
  trace-bs = map fst f-fl

```

```

    comp-bs = map fst fl
  by (rule accepted-fri-challenges-headerE)
have split: composition-fri-challenge-path-fresh s fr (map snd f-fl)
  trace-final alphas dg (map snd fl) (length (map snd fl)) ∨
  composition-fri-challenge-path-prequeried s fr (map snd f-fl)
  trace-final alphas dg (map snd fl) (length (map snd fl))
  by (rule composition-fri-challenge-path-fresh-or-prequeried)
then show ?thesis
proof
  assume fresh: composition-fri-challenge-path-fresh s fr (map snd f-fl)
  trace-final alphas dg (map snd fl) (length (map snd fl))
  have composition-fri-challenge-list-fresh-hit s B out
  unfolding composition-fri-challenge-list-fresh-hit-def
  by (intro exI conjI; (rule hit(1) | rule header | rule hit(2) |
    rule fresh))
  then show ?thesis by simp
next
  assume prequeried: composition-fri-challenge-path-prequeried s fr
    (map snd f-fl) trace-final alphas dg (map snd fl) (length (map snd fl))
  have composition-fri-challenge-list-prequery-hit s B out
  unfolding composition-fri-challenge-list-prequery-hit-def
  by (intro exI conjI; (rule hit(1) | rule header | rule hit(2) |
    rule prequeried))
  then show ?thesis by simp
qed
qed

lemma wp-trace-fri-challenge-list-set-hit-bound-from-fresh-and-prequery:
  assumes fresh-bound:
    wp-event verify-monad (trace-fri-challenge-list-fresh-hit s B) s ≤ F
  and prequery-bound:
    wp-event verify-monad (trace-fri-challenge-list-prequery-hit s B) s
      ≤ P
  shows wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s
    ≤ F + P
proof -
  have wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s ≤
    wp-event verify-monad
      (λout. trace-fri-challenge-list-fresh-hit s B out ∨
        trace-fri-challenge-list-prequery-hit s B out) s
  by (rule wp-event-mono)
    (rule trace-fri-challenge-list-set-hit-imp-fresh-or-prequery)
  also have ... ≤
    wp-event verify-monad (trace-fri-challenge-list-fresh-hit s B) s +
    wp-event verify-monad (trace-fri-challenge-list-prequery-hit s B) s
  by (rule wp-event-union-bound)
  also have ... ≤ F + P
  by (rule add-mono[OF fresh-bound prequery-bound])
  finally show ?thesis .

```

qed

lemma *wp-composition-fri-challenge-list-set-hit-bound-from-fresh-and-prequery:*

assumes *fresh-bound:*

wp-event verify-monad (composition-fri-challenge-list-fresh-hit s B) s
 $\leq F$

and *prequery-bound:*

wp-event verify-monad (composition-fri-challenge-list-prequery-hit s B) s
 $\leq P$

shows *wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s*
 $\leq F + P$

proof –

have *wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s*
 $\leq$ *wp-event verify-monad*

$(\lambda out. \text{composition-fri-challenge-list-fresh-hit } s \ B \ out \ \vee$
 $\text{composition-fri-challenge-list-prequery-hit } s \ B \ out) \ s$

by (rule *wp-event-mono*)

(rule *composition-fri-challenge-list-set-hit-imp-fresh-or-prequery*)

also have ... $\leq$

wp-event verify-monad (composition-fri-challenge-list-fresh-hit s B) s +
wp-event verify-monad (composition-fri-challenge-list-prequery-hit s B) s

by (rule *wp-event-union-bound*)

also have ... $\leq F + P$

by (rule *add-mono[OF fresh-bound prequery-bound]*)

finally show *?thesis* .

qed

end

context *soundness*

begin

lemma *checked-staged-security-trace-fri-challenge-list-set-hit-bound-from-fresh-and-prequery:*

assumes *fresh-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
 $(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B))$
adversary-initial-state $\leq F$

and *prequery-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
 $(\lambda s. \text{trace-fri-challenge-list-prequery-hit } s \ B))$
adversary-initial-state $\leq P$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
 $(\lambda s. \text{trace-fri-challenge-list-set-hit } s \ B))$
adversary-initial-state $\leq F + P$

proof –

```

have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s$ . trace-fri-challenge-list-set-hit s B))
adversary-initial-state  $\leq$ 
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s$  out.
      trace-fri-challenge-list-fresh-hit s B out  $\vee$ 
      trace-fri-challenge-list-prequery-hit s B out))
adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume
  out  $\in$ 
  set-dist
  (execute (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
and hit:
  staged-security-with-data-state-verifier-event
    ( $\lambda s$ . trace-fri-challenge-list-set-hit s B) out
then show
  staged-security-with-data-state-verifier-event
    ( $\lambda s$  out.
      trace-fri-challenge-list-fresh-hit s B out  $\vee$ 
      trace-fri-challenge-list-prequery-hit s B out) out
unfolding staged-security-with-data-state-verifier-event-def
by (auto dest: trace-fri-challenge-list-set-hit-imp-fresh-or-prequery
  split: option.splits prod.splits)
qed
also have ...  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    ( $\lambda$ out.
      staged-security-with-data-state-verifier-event
        ( $\lambda s$ . trace-fri-challenge-list-fresh-hit s B) out  $\vee$ 
        staged-security-with-data-state-verifier-event
          ( $\lambda s$ . trace-fri-challenge-list-prequery-hit s B) out)
    adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    split: option.splits prod.splits)
also have ...  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s$ . trace-fri-challenge-list-fresh-hit s B))
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s$ . trace-fri-challenge-list-prequery-hit s B))
  adversary-initial-state

```

```

    by (rule wp-event-union-bound)
  also have ...  $\leq F + P$ 
    by (rule add-mono[OF fresh-bound prequery-bound])
  finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-challenge-list-set-hit-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and finite-B: finite B
    and fresh-bound:
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-with-data-state-verifier-event
          ( $\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B$ ))
        adversary-initial-state  $\leq F$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. \text{trace-fri-challenge-list-set-hit } s \ B$ ))
      adversary-initial-state  $\leq$ 
        F +
        hash-relation-budget-value (card B * ceil-log clength)
        (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule
    checked-staged-security-trace-fri-challenge-list-set-hit-bound-from-fresh-and-prequery)
  (rule fresh-bound,
    rule checked-staged-security-trace-fri-verifier-prequery-hit-bound
      [OF wf controlled finite-B])

lemma checked-staged-security-trace-fri-challenge-list-fresh-hit-actual-bound:
  assumes subset:  $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B$ ))
      adversary-initial-state  $\leq$ 
        nnreal (card B) / nnreal ( $\text{CARD}(f) \wedge \text{ceil-log clength}$ )
  proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    show  $\bigwedge s. \neg \text{trace-fri-challenge-list-fresh-hit } s \ B \ \text{None}$ 
      unfolding trace-fri-challenge-list-fresh-hit-def
        accepted-fri-challenges-def by blast
  next
    fix data attacker-state
    assume Some (data, attacker-state)  $\in$ 
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
    show wp-event verify-monad
      (trace-fri-challenge-list-fresh-hit

```

$$\begin{aligned}
& (\text{verifier-state-from-adversary attacker-state} \\
& \quad (\text{staged-proof-transcript data})) B) \\
& (\text{verifier-state-from-adversary attacker-state} \\
& \quad (\text{staged-proof-transcript data})) \\
& \leq \text{nnreal} (\text{card } B) / \text{nnreal} (\text{CARD}('f) \wedge \text{ceil-log clength}) \\
& \text{by (rule wp-verify-monad-trace-fri-challenge-list-fresh-hit-actual-bound} \\
& \quad [\text{OF subset}])
\end{aligned}$$

qed

lemma *checked-staged-security-trace-fri-bad-challenge-list-fresh-hit-bound-from-round-bounds:*

assumes *local-bound:*

$\bigwedge i$ *prefix.*

$i < \text{fri-round-count-for-degree-bound} (\text{clength} - 1) \implies$

$\text{length prefix} = i \implies$

$\text{card} (\text{bad } i \text{ prefix}) \leq \text{bd } i$

shows

$\text{wp-event} (\text{checked-staged-security-experiment-with-data-state } A)$

$(\text{staged-security-with-data-state-verifier-event}$

$(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s$

$(\text{generic-fri-bad-challenge-lists}$

$(\text{fri-round-count-for-degree-bound} (\text{clength} - 1)) \text{ bad}))$

$\text{adversary-initial-state} \leq$

nnreal

$(\sum i \in \{..<\text{fri-round-count-for-degree-bound} (\text{clength} - 1)\}.$

$\text{CARD}('f) \wedge i * \text{bd } i *$

$\text{CARD}('f) \wedge$

$(\text{fri-round-count-for-degree-bound} (\text{clength} - 1) - \text{Suc } i)) /$

$\text{nnreal} (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound} (\text{clength} - 1))$

proof –

let $?n = \text{fri-round-count-for-degree-bound} (\text{clength} - 1)$

let $?B = \text{generic-fri-bad-challenge-lists } ?n \text{ bad}$

let $?K =$

$(\sum i \in \{..<?n\}.$

$\text{CARD}('f) \wedge i * \text{bd } i * \text{CARD}('f) \wedge (?n - \text{Suc } i))$

have *subset:* $?B \subseteq \text{fri-challenge-space} (\text{ceil-log clength})$

using *generic-fri-bad-challenge-lists-subset*[*of* $?n \text{ bad}$]

by (*simp add: fri-round-count-for-degree-bound-def clength-pos*)

have *base:*

$\text{wp-event} (\text{checked-staged-security-experiment-with-data-state } A)$

$(\text{staged-security-with-data-state-verifier-event}$

$(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s ?B))$

$\text{adversary-initial-state} \leq$

$\text{nnreal} (\text{card } ?B) / \text{nnreal} (\text{CARD}('f) \wedge \text{ceil-log clength})$

by (*rule checked-staged-security-trace-fri-challenge-list-fresh-hit-actual-bound*

$[\text{OF subset}]$)

have *card-bound:* $\text{card } ?B \leq ?K$

by (*rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds*)

$(\text{rule local-bound})$

have *ratio-bound:*

```

nnreal (card ?B) / nnreal (CARD('f) ^ ceil-log clength)
  ≤ nnreal ?K / nnreal (CARD('f) ^ ?n)
proof -
  have nnreal (card ?B) / nnreal (CARD('f) ^ ceil-log clength)
    ≤ nnreal ?K / nnreal (CARD('f) ^ ceil-log clength)
    using card-bound by (rule nnreal-nat-divide-right-mono)
    also have ... = nnreal ?K / nnreal (CARD('f) ^ ?n)
      by (simp add: fri-round-count-for-degree-bound-def clength-pos)
    finally show ?thesis .
qed
show ?thesis
  by (rule order-trans[OF base ratio-bound])
qed

lemma checked-staged-security-composition-fri-challenge-list-fresh-hit-actual-bound:
  fixes C :: prob
  assumes subset:
     $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$ 
  and bound:
     $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) / \text{nnreal } (\text{CARD}('f) ^ \text{ceil-log } (\text{to-nat } dg + 1)) \leq C$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$ ))
      adversary-initial-state ≤ C
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg \text{composition-fri-challenge-list-fresh-hit } s \ B \ \text{None}$ 
    unfolding composition-fri-challenge-list-fresh-hit-def
      accepted-fri-challenges-def by blast
next
  fix data attacker-state
  assume Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  show wp-event verify-monad
    (composition-fri-challenge-list-fresh-hit
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) B)
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    ≤ C
  by (rule wp-verify-monad-composition-fri-challenge-list-fresh-hit-actual-bound
    [OF subset bound])
qed

lemma checked-staged-security-composition-fri-bad-challenge-list-fresh-hit-bound-from-round-bounds:
  fixes C :: prob

```


assumes *local-bound*:

$$\bigwedge dg \ i \ \text{prefix}. \\ i < \text{fri-round-count-for-degree-bound} \ (to\text{-nat} \ dg) \implies \\ \text{length} \ \text{prefix} = i \implies \\ \text{card} \ (\text{bad} \ dg \ i \ \text{prefix}) \leq \text{bd} \ dg \ i$$

and *fraction-bound*:

$$\bigwedge dg. \\ \text{nnreal} \\ \left(\sum i \in \{..<\text{fri-round-count-for-degree-bound} \ (to\text{-nat} \ dg)\}. \right. \\ \quad \text{CARD}('f) \wedge^i * \text{bd} \ dg \ i * \\ \quad \left. \text{CARD}('f) \wedge \right. \\ \quad \left. (\text{fri-round-count-for-degree-bound} \ (to\text{-nat} \ dg) - \text{Suc} \ i) \right) / \\ \text{nnreal} \ (\text{CARD}('f) \wedge^{\text{fri-round-count-for-degree-bound} \ (to\text{-nat} \ dg)}) \leq C$$

shows

$$\text{wp-event} \ (\text{checked-staged-security-experiment-with-data-state} \ A) \\ (\text{staged-security-with-data-state-verifier-event} \\ (\lambda s. \text{composition-fri-challenge-list-fresh-hit} \ s \\ (\lambda dg. \text{generic-fri-bad-challenge-lists} \\ (\text{fri-round-count-for-degree-bound} \ (to\text{-nat} \ dg)) \ (\text{bad} \ dg)))) \\ \text{adversary-initial-state} \leq C$$

proof –

let *?B* =

$$\lambda dg. \text{generic-fri-bad-challenge-lists} \\ (\text{fri-round-count-for-degree-bound} \ (to\text{-nat} \ dg)) \ (\text{bad} \ dg)$$

have *subset*:

$$\bigwedge dg. \ ?B \ dg \subseteq \text{fri-challenge-space} \ (\text{ceil-log} \ (to\text{-nat} \ dg + 1))$$

using *generic-fri-bad-challenge-lists-subset*

by (*simp add: fri-round-count-for-degree-bound-def*)

have *ratio-bound*:

$$\bigwedge dg. \ \text{nnreal} \ (\text{card} \ (\ ?B \ dg)) / \\ \text{nnreal} \ (\text{CARD}('f) \wedge^{\text{ceil-log} \ (to\text{-nat} \ dg + 1)}) \leq C$$

proof –

fix *dg*

let *?n* = *fri-round-count-for-degree-bound* (*to-nat dg*)

let *?K* =

$$\left(\sum i \in \{..<?n\}. \right. \\ \quad \text{CARD}('f) \wedge^i * \text{bd} \ dg \ i * \text{CARD}('f) \wedge^{(?n - \text{Suc} \ i)})$$

have *card-bound*: *card* (*?B dg*) ≤ *?K*

by (*rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds*)
(*rule local-bound*)

have *nnreal* (*card* (*?B dg*)) /

$$\text{nnreal} \ (\text{CARD}('f) \wedge^{\text{ceil-log} \ (to\text{-nat} \ dg + 1)}) \\ \leq \text{nnreal} \ ?K / \text{nnreal} \ (\text{CARD}('f) \wedge^{\text{ceil-log} \ (to\text{-nat} \ dg + 1)})$$

using *card-bound* **by** (*rule nnreal-nat-divide-right-mono*)

also have ... = *nnreal* *?K* / *nnreal* (*CARD*('f) $\wedge^{?n}$)

by (*simp add: fri-round-count-for-degree-bound-def*)

also have ... ≤ *C*

by (*rule fraction-bound*)

finally show

$$\frac{\text{nnreal} (\text{card} (?B \text{ dg}))}{\text{nnreal} (\text{CARD}('f) \wedge \text{ceil-log} (\text{to-nat} \text{ dg} + 1))} \leq C .$$

qed
show *?thesis*
by (rule
checked-staged-security-composition-fri-challenge-list-fresh-hit-actual-bound
[OF subset ratio-bound])

qed

lemma *checked-staged-security-composition-fri-challenge-list-set-hit-bound-from-fresh-and-prequery:*
assumes *fresh-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
*($\lambda s.$ *composition-fri-challenge-list-fresh-hit s B*))*
adversary-initial-state $\leq F$
and *prequery-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
*($\lambda s.$ *composition-fri-challenge-list-prequery-hit s B*))*
adversary-initial-state $\leq P$
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
*($\lambda s.$ *composition-fri-challenge-list-set-hit s B*))*
adversary-initial-state $\leq F + P$

proof –
have *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
*($\lambda s.$ *composition-fri-challenge-list-set-hit s B*))*
adversary-initial-state $\leq$
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
(λs out.
composition-fri-challenge-list-fresh-hit s B out $\vee$
composition-fri-challenge-list-prequery-hit s B out))
adversary-initial-state

proof (rule *wp-event-mono-on-support*)
fix *out*
assume
out $\in$
set-dist
(execute (checked-staged-security-experiment-with-data-state A)
adversary-initial-state)
and *hit:*
staged-security-with-data-state-verifier-event
*($\lambda s.$ *composition-fri-challenge-list-set-hit s B*) out*
then show
staged-security-with-data-state-verifier-event
(λs out.

```

      composition-fri-challenge-list-fresh-hit s B out ∨
      composition-fri-challenge-list-prequery-hit s B out) out
unfolding staged-security-with-data-state-verifier-event-def
by (auto dest:
      composition-fri-challenge-list-set-hit-imp-fresh-or-prequery
      split: option.splits prod.splits)
qed
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (λout.
      staged-security-with-data-state-verifier-event
        (λs. composition-fri-challenge-list-fresh-hit s B) out ∨
      staged-security-with-data-state-verifier-event
        (λs. composition-fri-challenge-list-prequery-hit s B) out)
    adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    split: option.splits prod.splits)
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-challenge-list-fresh-hit s B))
    adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-challenge-list-prequery-hit s B))
    adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤ F + P
by (rule add-mono[OF fresh-bound prequery-bound])
finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-challenge-list-set-hit-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-challenge-list-fresh-hit s B))
    adversary-initial-state ≤ F
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-challenge-list-set-hit s B))
    adversary-initial-state ≤
    F +
    hash-relation-budget-value

```

```

    ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
       $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$ 
    ( $\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget}$ )
  by (rule
    checked-staged-security-composition-fri-challenge-list-set-hit-bound-from-fresh-and-prequery)
  (rule fresh-bound,
    rule checked-staged-security-composition-fri-verifier-prequery-hit-bound
      [OF wf controlled finite-B])

end

end

```

```

theory Soundness-FRI-Prequery-Trace-Sampled-Bounds
imports Soundness-FRI-Prequery-Challenge-Bounds
begin

```

Staged bounds for trace FRI sampled-layer events. The lemmas combine the fresh-challenge case with prequery accounting from the shared random oracle.

```

context soundness
begin

```

lemma *checked-staged-security-trace-fri-sampled-layer-chain-bound-from-fresh-and-prequery:*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and cover:
   $\bigwedge \text{data attacker-state.}$ 
     $\text{Some } (\text{data}, \text{attacker-state}) \in$ 
       $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$ 
         $\text{adversary-initial-state}) \implies$ 
       $\text{trace-fri-sampled-layer-chain-cover}$ 
       $(\text{verifier-state-from-adversary attacker-state}$ 
         $(\text{staged-proof-transcript data})) \text{ bad}$ 
and envelope:
   $\bigwedge \text{data attacker-state trace-table.}$ 
     $\text{Some } (\text{data}, \text{attacker-state}) \in$ 
       $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$ 
         $\text{adversary-initial-state}) \implies$ 
       $\text{trace-fri-multiround-bad-sets bad trace-table} \subseteq B$ 
and fresh-bound:
   $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$ 
     $(\text{staged-security-with-data-state-verifier-event}$ 
       $(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B))$ 
     $\text{adversary-initial-state} \leq F$ 

```

shows

```

   $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$ 
     $(\text{staged-security-with-data-state-verifier-event}$ 

```

```

    trace-fri-bad-with-sampled-layer-chain)
  adversary-initial-state ≤
  F +
  hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof –
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-sampled-layer-chain)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     (λs. trace-fri-challenge-list-set-hit s B))
  adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
     adversary-initial-state)
  and hit:
    staged-security-with-data-state-verifier-event
    trace-fri-bad-with-sampled-layer-chain out
  from hit obtain data attacker-state result final-state where
    out-eq: out = Some (((data, attacker-state), result), final-state)
  unfolding staged-security-with-data-state-verifier-event-def
  by (cases out) (auto split: option.splits prod.splits)
  have builder:
    Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
     adversary-initial-state)
  using checked-staged-security-experiment-with-data-state-outcomeE
  [OF support[unfolded out-eq]]
  by blast
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have sampled:
    trace-fri-bad-with-sampled-layer-chain ?s
    (Some (result, final-state))
  using hit unfolding staged-security-with-data-state-verifier-event-def
  out-eq by simp
  have list-hit:
    trace-fri-challenge-list-set-hit ?s B (Some (result, final-state))
  by (rule trace-fri-bad-with-sampled-layer-chain-imp-list-hit
    [OF sampled cover[OF builder]])
    (rule envelope[OF builder])
  show

```

```

    staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{trace-fri-challenge-list-set-hit } s \ B$ ) out
    unfolding staged-security-with-data-state-verifier-event-def out-eq
    using list-hit by simp
  qed
  also have ...  $\leq$ 
     $F +$ 
    hash-relation-budget-value (card  $B * \text{ceil-log } \text{clength}$ )
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule checked-staged-security-trace-fri-challenge-list-set-hit-bound
    [ $OF \text{ wf controlled finite-}B \text{ fresh-bound}$ ])
  finally show ?thesis .
  qed

lemma checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-fresh-and-prequery:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets  $A$ 
  and finite- $B$ : finite  $B$ 
  and cover:
     $\bigwedge \text{data attacker-state.}$ 
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program  $A$ )
        adversary-initial-state)  $\implies$ 
      trace-fri-sampled-layer-chain-cover
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) bad
  and envelope:
     $\bigwedge \text{data attacker-state trace-table.}$ 
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program  $A$ )
        adversary-initial-state)  $\implies$ 
      trace-fri-multiround-bad-sets bad trace-table  $\subseteq B$ 
  and fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state  $A$ )
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B$ ))
    adversary-initial-state  $\leq F$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state  $A$ )
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-header-tied-sampled-layer-chain)
    adversary-initial-state  $\leq$ 
       $F +$ 
      hash-relation-budget-value (card  $B * \text{ceil-log } \text{clength}$ )
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  proof -
  have wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-sampled-layer-chain)

```

```

    adversary-initial-state  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-sampled-layer-chain)
      adversary-initial-state
  by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    intro: trace-fri-header-tied-sampled-layer-chain-imp-sampled-layer-chain
    split: option.splits prod.splits)
also have ...  $\leq$ 
  F +
  hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
by (rule
  checked-staged-security-trace-fri-sampled-layer-chain-bound-from-fresh-and-prequery
  [OF wf controlled finite-B cover envelope fresh-bound])
finally show ?thesis .
qed

end

end

```

theory Soundness-FRI-Prequery-Composition-Sampled-Bounds
imports Soundness-FRI-Prequery-Trace-Sampled-Bounds
begin

Staged bounds for composition FRI sampled-layer events. This theory mirrors the trace-side prequery accounting with the composition degree parameter.

context soundness
begin

lemma checked-staged-security-composition-fri-sampled-layer-chain-bound-from-fresh-and-prequery:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: $\bigwedge dg. \text{finite } (B \ dg)$
and cover:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{composition-fri-sampled-layer-chain-cover}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad}$
and envelope:
 $\bigwedge \text{data attacker-state dg composition-table.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$

```

    adversary-initial-state)  $\implies$ 
    composition-fri-multiround-bad-sets bad dg composition-table
     $\subseteq B$  dg
and fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
    ( $\lambda s$ . composition-fri-challenge-list-fresh-hit s B))
    adversary-initial-state  $\leq F$ 
shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
    composition-fri-bad-with-sampled-layer-chain)
    adversary-initial-state  $\leq$ 
    F +
    hash-relation-budget-value
    ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
    card (B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof -
have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
    composition-fri-bad-with-sampled-layer-chain)
    adversary-initial-state  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
    ( $\lambda s$ . composition-fri-challenge-list-set-hit s B))
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
    out  $\in$ 
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
and hit:
    staged-security-with-data-state-verifier-event
    composition-fri-bad-with-sampled-layer-chain out
from hit obtain data attacker-state result final-state where
    out-eq: out = Some (((data, attacker-state), result), final-state)
unfolding staged-security-with-data-state-verifier-event-def
by (cases out) (auto split: option.splits prod.splits)
have builder:
    Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
    adversary-initial-state)
using checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
by blast
let ?s =

```



```

    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have sampled:
    composition-fri-bad-with-sampled-layer-chain ?s
      (Some (result, final-state))
  using hit unfolding staged-security-with-data-state-verifier-event-def
    out-eq by simp
  have list-hit:
    composition-fri-challenge-list-set-hit ?s B
      (Some (result, final-state))
  by (rule composition-fri-bad-with-sampled-layer-chain-imp-list-hit
      [OF sampled cover [OF builder]])
    (rule envelope [OF builder])
  show
    staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-challenge-list-set-hit s B) out
  unfolding staged-security-with-data-state-verifier-event-def out-eq
  using list-hit by simp
qed
also have ...  $\leq$ 
  F +
  hash-relation-budget-value
    ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
      card (B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule
      checked-staged-security-composition-fri-challenge-list-set-hit-bound
      [OF wf controlled finite-B fresh-bound])
  finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-fresh-and-prequery:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
  and cover:
     $\bigwedge \text{data attacker-state.}$ 
      Some (data, attacker-state)  $\in$ 
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state)  $\implies$ 
        composition-fri-sampled-layer-chain-cover
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)) bad
  and envelope:
     $\bigwedge \text{data attacker-state dg composition-table.}$ 
      Some (data, attacker-state)  $\in$ 
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state)  $\implies$ 
        composition-fri-multiround-bad-sets bad dg composition-table

```

```

    ⊆ B dg
  and fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. composition-fri-challenge-list-fresh-hit s B))
    adversary-initial-state ≤ F
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-verifier-tied-sampled-layer-chain)
    adversary-initial-state ≤
      F +
      hash-relation-budget-value
        (∑ dg ∈ (UNIV :: 'f set).
          card (B dg) * ceil-log (maxDegree + 1))
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  proof -
    have wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-verifier-tied-sampled-layer-chain)
    adversary-initial-state ≤
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-with-data-state-verifier-event
          composition-fri-bad-with-sampled-layer-chain)
      adversary-initial-state
    by (rule wp-event-mono-on-support)
      (auto simp: staged-security-with-data-state-verifier-event-def
        intro:
          composition-fri-verifier-tied-sampled-layer-chain-imp-sampled-layer-chain
          split: option.splits prod.splits)
    also have ... ≤
      F +
      hash-relation-budget-value
        (∑ dg ∈ (UNIV :: 'f set).
          card (B dg) * ceil-log (maxDegree + 1))
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    by (rule
      checked-staged-security-composition-fri-sampled-layer-chain-bound-from-fresh-and-prequery
        [OF wf controlled finite-B cover envelope fresh-bound])
    finally show ?thesis .
  qed

end

end

```

```

theory Soundness-FRI-Staged-Active-Bounds
  imports Soundness-FRI-Prequery-Composition-Sampled-Bounds

```

begin

Staged integration lemmas for the active FRI route.

The prequery accounting layer proves whole checked-security-experiment bounds for sampled FRI layer-chain events. The active route still reasons about partial-candidate FRI events. This layer bridges those levels by splitting a partial-candidate FRI bad event into the sampled layer-chain case and the remaining missing-sampled-chain residual.

context *soundness*

begin

lemma *trace-fri-header-tied-partial-candidate-imp-sampled-conflict-or-zero:*

assumes *bad: trace-fri-bad-with-header-tied-partial-candidate s out*

shows

trace-fri-bad-with-header-tied-sampled-layer-chain s out $\vee$

trace-fri-header-tied-sampled-assignment-conflict s out $\vee$

trace-fri-header-tied-zero-round-final-obstruction s out

proof (*cases trace-fri-bad-with-header-tied-sampled-layer-chain s out*)

case *True*

then show *?thesis* **by** *blast*

next

case *False*

have *missing: trace-fri-header-tied-missing-sampled-layer-chain s out*

unfolding *trace-fri-header-tied-missing-sampled-layer-chain-def*

using *bad False* **by** *blast*

have *obstruction:*

trace-fri-header-tied-sampled-layer-assignment-obstruction s out

by (*rule trace-fri-header-tied-missing-sampled-imp-assignment-obstruction*
[OF missing])

then show *?thesis*

using *trace-fri-header-tied-assignment-obstruction-imp-conflict-or-zero*

by *blast*

qed

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-and-missing:*

assumes *sampled-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

trace-fri-bad-with-header-tied-sampled-layer-chain)

adversary-initial-state $\leq R$

and *missing-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\implies$

wp-event verify-monad

(trace-fri-header-tied-missing-sampled-layer-chain

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state

$(\text{staged-proof-transcript data}) \leq M$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq R + M$
proof –
have *missing-staged*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-header-tied-missing-sampled-layer-chain})$
 $\text{adversary-initial-state} \leq M$
by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
 $(\text{auto simp: trace-fri-header-tied-missing-sampled-layer-chain-def}$
 $\text{missing-bound})$
have $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\lambda \text{out.}$
 $\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-sampled-layer-chain out} \vee$
 $\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-header-tied-missing-sampled-layer-chain out})$
 $\text{adversary-initial-state}$
unfolding *trace-fri-header-tied-missing-sampled-layer-chain-def*
by (rule *wp-event-mono-on-support*)
 $(\text{auto simp: staged-security-with-data-state-verifier-event-def}$
 $\text{split: option.splits prod.splits})$
also have $\dots \leq$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-sampled-layer-chain})$
 $\text{adversary-initial-state} +$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-header-tied-missing-sampled-layer-chain})$
 $\text{adversary-initial-state}$
by (rule *wp-event-union-bound*)
also have $\dots \leq R + M$
by (rule *add-mono[OF sampled-bound missing-staged]*)
finally show *?thesis* .
qed

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-conflict-and-staged-zero*:
assumes *sampled-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$

```

    trace-fri-bad-with-header-tied-sampled-layer-chain)
  adversary-initial-state  $\leq R$ 
and conflict-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-sampled-assignment-conflict)
  adversary-initial-state  $\leq C$ 
and zero-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-zero-round-final-obstruction)
  adversary-initial-state  $\leq Z$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state  $\leq R + (C + Z)$ 
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?sampled =
    staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-sampled-layer-chain
  let ?conflict =
    staged-security-with-data-state-verifier-event
    trace-fri-header-tied-sampled-assignment-conflict
  let ?zero =
    staged-security-with-data-state-verifier-event
    trace-fri-header-tied-zero-round-final-obstruction
  have split:
    wp-event ?M
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-header-tied-partial-candidate)
    adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda out. ?sampled\ out \vee ?conflict\ out \vee ?zero\ out$ )
    adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def
  by (rule wp-event-mono-on-support)
    (auto dest:
      trace-fri-header-tied-partial-candidate-imp-sampled-conflict-or-zero
      split: option.splits prod.splits)
  also have ...  $\leq$ 
    wp-event ?M ?sampled adversary-initial-state +
    wp-event ?M ( $\lambda out. ?conflict\ out \vee ?zero\ out$ )
    adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq$ 
    wp-event ?M ?sampled adversary-initial-state +
    (wp-event ?M ?conflict adversary-initial-state +
    wp-event ?M ?zero adversary-initial-state)

```

```

    by (intro add-mono order-refl wp-event-union-bound)
  also have ...  $\leq R + (C + Z)$ 
    by (intro add-mono sampled-bound conflict-bound zero-bound)
  finally show ?thesis .
qed

```

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-and-missing:*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and cover:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    trace-fri-sampled-layer-chain-cover
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) bad
and envelope:
 $\wedge$ data attacker-state trace-table.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    trace-fri-multiround-bad-sets bad trace-table  $\subseteq B$ 
and fresh-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s$ . trace-fri-challenge-list-fresh-hit s B))
  adversary-initial-state  $\leq F$ 
and missing-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-header-tied-missing-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq M$ 
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state  $\leq$ 
  (F +
    hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) + M

```

proof –

have *sampled*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-sampled-layer-chain)

adversary-initial-state $\leq$

F +

hash-relation-budget-value (*card B* * *ceil-log clength*)

(*staged-attacker-query-budget budgets* +

staged-challenge-query-budget)

by (*rule*

checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-fresh-and-prequery

[*OF wf controlled finite-B cover envelope fresh-bound*])

show *?thesis*

by (*rule*

checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-and-missing

[*OF sampled missing-bound*])

qed

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-and-assignment-obstruction*

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *finite-B*: *finite B*

and *cover*:

$\wedge$ *data attacker-state*.

Some (*data, attacker-state*) $\in$

set-dist (*execute* (*checked-staged-transcript-program A*)

adversary-initial-state) $\implies$

trace-fri-sampled-layer-chain-cover

(*verifier-state-from-adversary attacker-state*

(*staged-proof-transcript data*)) *bad*

and *envelope*:

$\wedge$ *data attacker-state trace-table*.

Some (*data, attacker-state*) $\in$

set-dist (*execute* (*checked-staged-transcript-program A*)

adversary-initial-state) $\implies$

trace-fri-multi-round-bad-sets bad trace-table $\subseteq B$

and *fresh-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

($\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B$))

adversary-initial-state $\leq F$

and *assignment-bound*:

$\wedge$ *data attacker-state*.

Some (*data, attacker-state*) $\in$

set-dist (*execute* (*checked-staged-transcript-program A*)

adversary-initial-state) $\implies$

wp-event verify-monad

(*trace-fri-header-tied-sampled-layer-assignment-obstruction*)

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq M$

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq$
 $(F +$
 $\text{hash-relation-budget-value } (\text{card } B * \text{ceil-log clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) + M$

proof (rule

$\text{checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-and-missing}$
 $[\text{OF wf controlled finite-B cover envelope fresh-bound}])$

fix data attacker-state

assume support:

$\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$

show wp-event verify-monad

$(\text{trace-fri-header-tied-missing-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq M$

by (rule wp-trace-fri-header-tied-missing-sampled-bound-from-assignment-obstruction)

(rule assignment-bound[OF support])

qed

lemma checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-conflict-and-staged-

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and finite-B: finite B

and cover:

$\bigwedge \text{data attacker-state.}$
 $\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{trace-fri-sampled-layer-chain-cover}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad}$

and envelope:

$\bigwedge \text{data attacker-state trace-table.}$
 $\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{trace-fri-multi-round-bad-sets bad trace-table} \subseteq B$

and *fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B$))
adversary-initial-state $\leq F$
and *conflict-bound*:
 $\wedge \text{data attacker-state.}$
 Some (*data*, *attacker-state*) $\in$
 set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) $\implies$
wp-event *verify-monad*
 (*trace-fri-header-tied-sampled-assignment-conflict*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq C$
and *zero-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-header-tied-zero-round-final-obstruction)
adversary-initial-state $\leq Z$
shows
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state $\leq$
 ($F +$
hash-relation-budget-value (*card B* * *ceil-log clength*)
 (*staged-attacker-query-budget budgets* +
staged-challenge-query-budget)) + ($C + Z$)
proof (rule
checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-conflict-and-staged-zero)
show *wp-event* (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-sampled-layer-chain)
adversary-initial-state $\leq$
 $F +$
hash-relation-budget-value (*card B* * *ceil-log clength*)
 (*staged-attacker-query-budget budgets* +
staged-challenge-query-budget)
by (rule
checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-fresh-and-prequery
 [*OF wf controlled finite-B cover envelope fresh-bound*])
show *wp-event* (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-header-tied-sampled-assignment-conflict)
adversary-initial-state $\leq C$
by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
 (*simp-all add: conflict-bound*)

$\text{trace-fri-header-tied-sampled-assignment-conflict-def}$
 $\text{accepted-fri-opening-transcript-def}$
show $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-header-tied-zero-round-final-obstruction})$
 $\text{adversary-initial-state} \leq Z$
by (rule zero-bound)
qed

lemma $\text{checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-conflict-and-zero:}$
assumes $\text{wf: staged-budget-wellformed budgets}$
and $\text{controlled: staged-adversary-controlled budgets } A$
and $\text{finite-B: finite } B$
and cover:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{trace-fri-sampled-layer-chain-cover}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad}$
and envelope:
 $\bigwedge \text{data attacker-state trace-table.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and fresh-bound:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B))$
 $\text{adversary-initial-state} \leq F$
and conflict-bound:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq C$
and zero-bound:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$

$$\begin{aligned}
& (\text{trace-fri-header-tied-zero-round-final-obstruction} \\
& \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad (\text{staged-proof-transcript data}))) \\
& (\text{verifier-state-from-adversary attacker-state} \\
& \quad (\text{staged-proof-transcript data})) \leq Z
\end{aligned}$$

shows

$$\begin{aligned}
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\text{staged-security-with-data-state-verifier-event} \\
& \quad \quad \text{trace-fri-bad-with-header-tied-partial-candidate}) \\
& \text{adversary-initial-state} \leq \\
& (F + \\
& \quad \text{hash-relation-budget-value } (\text{card } B * \text{ceil-log clength}) \\
& \quad (\text{staged-attacker-query-budget budgets} + \\
& \quad \quad \text{staged-challenge-query-budget})) + (C + Z)
\end{aligned}$$

proof (rule

$$\begin{aligned}
& \text{checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-and-assignment-obstruction} \\
& \quad [\text{OF wf controlled finite-B cover envelope fresh-bound}]
\end{aligned}$$

fix data attacker-state

assume support:

$$\begin{aligned}
& \text{Some } (data, \text{attacker-state}) \in \\
& \quad \text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \\
& \quad \quad \text{adversary-initial-state})
\end{aligned}$$

show wp-event verify-monad

$$\begin{aligned}
& (\text{trace-fri-header-tied-sampled-layer-assignment-obstruction} \\
& \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad (\text{staged-proof-transcript data}))) \\
& (\text{verifier-state-from-adversary attacker-state} \\
& \quad (\text{staged-proof-transcript data})) \leq C + Z
\end{aligned}$$

by (rule

$$\begin{aligned}
& \text{wp-trace-fri-header-tied-assignment-obstruction-bound-from-conflict-and-zero}) \\
& (\text{rule conflict-bound}[\text{OF support}], \text{rule zero-bound}[\text{OF support}])
\end{aligned}$$

qed

lemma checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-and-residual-gaps:

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and finite-B: finite B

and cover:

$$\begin{aligned}
& \bigwedge \text{data attacker-state.} \\
& \quad \text{Some } (data, \text{attacker-state}) \in \\
& \quad \quad \text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \\
& \quad \quad \quad \text{adversary-initial-state}) \implies \\
& \quad \text{trace-fri-sampled-layer-chain-cover} \\
& \quad \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad \quad (\text{staged-proof-transcript data})) \text{ bad}
\end{aligned}$$

and envelope:

$$\begin{aligned}
& \bigwedge \text{data attacker-state trace-table.} \\
& \quad \text{Some } (data, \text{attacker-state}) \in \\
& \quad \quad \text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)
\end{aligned}$$

$\text{adversary-initial-state}) \implies$
 $\text{trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *fresh-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B))$
 $\text{adversary-initial-state} \leq \text{Fresh}$
and *slot-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Slot}$
and *merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Merkle}$
and *structural-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Structural}$
and *alignment-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-value-alignment-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Align}$
and *next-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Next}$
and *successor-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Successor}$
and *final-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-final-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Final}$
and *zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Zero}$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$

```

    trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state ≤
  (Fresh +
    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)) +
  ((((((Merkle + Structural) + Align) + Next + Successor + Final) +
    Merkle) + Slot + Zero)
proof (rule
  checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-conflict-and-zero
  [OF wf controlled finite-B cover envelope fresh-bound])
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
show wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict ?s) ?s
  ≤ ((((((Merkle + Structural) + Align) + Next + Successor + Final) +
    Merkle) + Slot)
by (rule
  wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-slot-recorded-chunk-gap-and-residuals)
  (rule slot-bound[OF support],
    rule merkle-bound[OF support],
    rule structural-bound[OF support],
    rule alignment-bound[OF support],
    rule next-bound[OF support],
    rule successor-bound[OF support],
    rule final-bound[OF support])
show wp-event verify-monad
  (trace-fri-header-tied-zero-round-final-obstruction ?s) ?s ≤ Zero
by (rule zero-bound[OF support])
qed

```

lemma checked-staged-security-trace-fri-empty-header-bound-from-sampled-and-missing:

```

assumes sampled-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-sampled-layer-chain)
  adversary-initial-state ≤ R
and missing-bound:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  wp-event verify-monad

```

$$\begin{aligned}
& (\text{trace-fri-reachable-missing-sampled-layer-chain} \\
& \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad (\text{staged-proof-transcript data}))) \\
& (\text{verifier-state-from-adversary attacker-state} \\
& \quad (\text{staged-proof-transcript data})) \leq M
\end{aligned}$$

shows

$$\begin{aligned}
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\text{staged-security-with-data-state-verifier-event} \\
& \quad \quad \text{trace-fri-bad-with-empty-composition-header-candidates}) \\
& \text{adversary-initial-state} \leq R + M
\end{aligned}$$

proof –

have *missing-staged*:

$$\begin{aligned}
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\text{staged-security-with-data-state-verifier-event} \\
& \quad \quad \text{trace-fri-reachable-missing-sampled-layer-chain}) \\
& \text{adversary-initial-state} \leq M
\end{aligned}$$

by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
(simp add: *trace-fri-reachable-missing-sampled-layer-chain-def*
trace-fri-bad-with-reachable-partial-candidate-def
accepted-with-partial-trace-openings-def,
rule *missing-bound*)

have *wp-event* (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$

$$\begin{aligned}
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\lambda \text{out.} \\
& \quad \quad \text{staged-security-with-data-state-verifier-event} \\
& \quad \quad \quad \text{trace-fri-bad-with-sampled-layer-chain out} \vee \\
& \quad \quad \text{staged-security-with-data-state-verifier-event} \\
& \quad \quad \quad \text{trace-fri-reachable-missing-sampled-layer-chain out}) \\
& \text{adversary-initial-state}
\end{aligned}$$

by (rule *wp-event-mono-on-support*)
(auto simp: *staged-security-with-data-state-verifier-event-def*
dest:
trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate
trace-fri-reachable-imp-sampled-or-missing-layer-chain
split: *option.splits prod.splits*)

also have ... $\leq$

$$\begin{aligned}
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\text{staged-security-with-data-state-verifier-event} \\
& \quad \quad \text{trace-fri-bad-with-sampled-layer-chain}) \\
& \text{adversary-initial-state} + \\
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\text{staged-security-with-data-state-verifier-event} \\
& \quad \quad \text{trace-fri-reachable-missing-sampled-layer-chain}) \\
& \text{adversary-initial-state}
\end{aligned}$$

by (rule *wp-event-union-bound*)

also have ... $\leq R + M$

by (rule add-mono[OF sampled-bound missing-staged])
 finally show ?thesis .
 qed

lemma *checked-staged-security-trace-fri-empty-header-bound-from-fresh-prequery-and-missing:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: finite B*
and *cover:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-sampled-layer-chain-cover}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad}$
and *envelope:*
 $\bigwedge \text{data attacker-state trace-table.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *fresh-bound:*
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B))$
 $\text{adversary-initial-state} \leq F$
and *missing-bound:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-reachable-missing-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq M$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq$
 $(F +$
 $\text{hash-relation-budget-value (card } B * \text{ceil-log clength)}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) + M$
proof –
have *sampled:*


```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-sampled-layer-chain)
adversary-initial-state ≤
F +
  hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)
by (rule
  checked-staged-security-trace-fri-sampled-layer-chain-bound-from-fresh-and-prequery
  [OF wf controlled finite-B cover envelope fresh-bound])
show ?thesis
by (rule
  checked-staged-security-trace-fri-empty-header-bound-from-sampled-and-missing
  [OF sampled missing-bound])
qed

```

lemma *checked-staged-security-trace-fri-empty-header-bound-from-fresh-prequery-and-assignment-obstruction:*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and cover:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      trace-fri-sampled-layer-chain-cover
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) bad
and envelope:
  ∧ data attacker-state trace-table.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      trace-fri-multi-round-bad-sets bad trace-table ⊆ B
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-challenge-list-fresh-hit s B))
    adversary-initial-state ≤ F
and assignment-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (trace-fri-sampled-layer-assignment-obstruction
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))

```

$(\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})) \leq M$

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ (\text{staged-security-with-data-state-verifier-event} \\ \text{trace-fri-bad-with-empty-composition-header-candidates}) \\ \text{adversary-initial-state} \leq \\ (F + \\ \text{hash-relation-budget-value } (\text{card } B * \text{ceil-log clength}) \\ (\text{staged-attacker-query-budget budgets} + \\ \text{staged-challenge-query-budget})) + M$

proof (rule

$\text{checked-staged-security-trace-fri-empty-header-bound-from-fresh-prequery-and-missing} \\ [\text{OF wf controlled finite-B cover envelope fresh-bound}])$

fix data attacker-state

assume support:

$\text{Some } (\text{data}, \text{attacker-state}) \in \\ \text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \\ \text{adversary-initial-state})$

show wp-event verify-monad

$(\text{trace-fri-reachable-missing-sampled-layer-chain} \\ (\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data}))) \\ (\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})) \leq M$

by (rule wp-trace-fri-reachable-missing-sampled-bound-from-assignment-obstruction)

(rule assignment-bound[OF support])

qed

lemma checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-sampled-and-missing:

assumes sampled-bound:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ (\text{staged-security-with-data-state-verifier-event} \\ \text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}) \\ \text{adversary-initial-state} \leq R$

and missing-bound:

$\bigwedge \text{data attacker-state.} \\ \text{Some } (\text{data}, \text{attacker-state}) \in \\ \text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \\ \text{adversary-initial-state}) \implies \\ \text{wp-event verify-monad} \\ (\text{composition-fri-verifier-tied-missing-sampled-layer-chain} \\ (\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data}))) \\ (\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})) \leq M$

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ (\text{staged-security-with-data-state-verifier-event}$

$\text{composition-fri-bad-with-verifier-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq R + M$
proof –
have *missing-staged*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-verifier-tied-missing-sampled-layer-chain})$
 $\text{adversary-initial-state} \leq M$
by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
 $(\text{auto simp: composition-fri-verifier-tied-missing-sampled-layer-chain-def}$
 $\text{missing-bound})$
have $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-verifier-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\lambda \text{out.}$
 $\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain out } \vee$
 $\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-verifier-tied-missing-sampled-layer-chain out})$
 $\text{adversary-initial-state}$
unfolding *composition-fri-verifier-tied-missing-sampled-layer-chain-def*
by (rule *wp-event-mono-on-support*)
 $(\text{auto simp: staged-security-with-data-state-verifier-event-def}$
 $\text{split: option.splits prod.splits})$
also have $\dots \leq$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain})$
 $\text{adversary-initial-state} +$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-verifier-tied-missing-sampled-layer-chain})$
 $\text{adversary-initial-state}$
by (rule *wp-event-union-bound*)
also have $\dots \leq R + M$
by (rule *add-mono[OF sampled-bound missing-staged]*)
finally show *?thesis* .
qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-and-missing*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: $\bigwedge dg. \text{finite } (B dg)$*
and *cover:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A))$

$\text{adversary-initial-state}) \implies$
 $\text{composition-fri-sampled-layer-chain-cover}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad}$
and *envelope*:
 $\wedge \text{data attacker-state dg composition-table.}$
 $\text{Some (data, attacker-state) } \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{composition-fri-multiround-bad-sets bad dg composition-table}$
 $\subseteq B \text{ dg}$
and *fresh-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-challenge-list-fresh-hit s B}))$
 $\text{adversary-initial-state} \leq F$
and *missing-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state) } \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-missing-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq M$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-verifier-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq$
 $(F +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card (B dg) * ceil-log (maxDegree + 1))}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) + M$
proof –
have *sampled*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain})$
 $\text{adversary-initial-state} \leq$
 $F +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card (B dg) * ceil-log (maxDegree + 1))}$
 $(\text{staged-attacker-query-budget budgets} +$

staged-challenge-query-budget)

by (rule
checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-fresh-and-prequery
 [OF wf controlled finite-B cover envelope fresh-bound])

show ?thesis

by (rule
checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-sampled-and-missing
 [OF sampled missing-bound])

qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-and-assignment*

assumes wf: *staged-budget-wellformed* budgets

and controlled: *staged-adversary-controlled* budgets A

and finite-B: $\bigwedge dg. \text{finite } (B \ dg)$

and cover:
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (*execute* (*checked-staged-transcript-program* A)
adversary-initial-state) $\implies$
composition-fri-sampled-layer-chain-cover
 (*verifier-state-from-adversary* attacker-state
 (*staged-proof-transcript* data)) bad

and envelope:
 $\bigwedge \text{data attacker-state dg composition-table.}$
Some (data, attacker-state) $\in$
set-dist (*execute* (*checked-staged-transcript-program* A)
adversary-initial-state) $\implies$
composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \ dg$

and fresh-bound:
wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$))
adversary-initial-state $\leq F$

and assignment-bound:
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (*execute* (*checked-staged-transcript-program* A)
adversary-initial-state) $\implies$
wp-event *verify-monad*
 (*composition-fri-verifier-tied-sampled-layer-assignment-obstruction*
 (*verifier-state-from-adversary* attacker-state
 (*staged-proof-transcript* data)))
 (*verifier-state-from-adversary* attacker-state
 (*staged-proof-transcript* data)) $\leq M$

shows
wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-verifier-tied-partial-candidate)

$adversary-initial-state \leq$
 $(F +$
 $hash-relation-budget-value$
 $(\sum dg \in (UNIV :: 'f set).$
 $card (B dg) * ceil-log (maxDegree + 1))$
 $(staged-attacker-query-budget budgets +$
 $staged-challenge-query-budget)) + M$

proof (rule
checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-and-missing
 $[OF\ wf\ controlled\ finite-B\ cover\ envelope\ fresh-bound]$)

fix data attacker-state

assume support:
 $Some (data, attacker-state) \in$
 $set-dist (execute (checked-staged-transcript-program A)$
 $adversary-initial-state)$

show wp-event verify-monad
 $(composition-fri-verifier-tied-missing-sampled-layer-chain$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data)))$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data)) \leq M$

by (rule
 $wp-composition-fri-verifier-tied-missing-sampled-bound-from-assignment-obstruction)$
 $(rule\ assignment-bound[OF\ support])$)

qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-conflict-and-*

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and finite-B: $\bigwedge dg. finite (B dg)$

and cover:
 $\bigwedge data\ attacker-state.$
 $Some (data, attacker-state) \in$
 $set-dist (execute (checked-staged-transcript-program A)$
 $adversary-initial-state) \implies$
 $composition-fri-sampled-layer-chain-cover$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data))\ bad$

and envelope:
 $\bigwedge data\ attacker-state\ dg\ composition-table.$
 $Some (data, attacker-state) \in$
 $set-dist (execute (checked-staged-transcript-program A)$
 $adversary-initial-state) \implies$
 $composition-fri-multiround-bad-sets\ bad\ dg\ composition-table$
 $\subseteq B\ dg$

and fresh-bound:
 $wp-event (checked-staged-security-experiment-with-data-state A)$
 $(staged-security-with-data-state-verifier-event$
 $(\lambda s. composition-fri-challenge-list-fresh-hit\ s\ B))$

$\text{adversary-initial-state} \leq F$
and *conflict-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq C$
and *zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq Z$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-verifier-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq$
 $(F +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card } (B \text{ } dg) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) + (C + Z)$
proof (rule
 $\text{checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-and-assignment-ob}$
 $[\text{OF wf controlled finite-B cover envelope fresh-bound}])$
fix *data attacker-state*
assume *support*:
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$
show $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-layer-assignment-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq C + Z$
by (rule

wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero
(rule conflict-bound[OF support], rule zero-bound[OF support])

qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-and-residual*

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *finite-B*: $\bigwedge dg. \text{finite } (B \ dg)$

and *cover*:

$\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program *A*)
adversary-initial-state) $\implies$

composition-fri-sampled-layer-chain-cover
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad

and *envelope*:

$\bigwedge \text{data attacker-state dg composition-table.}$

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program *A*)
adversary-initial-state) $\implies$
composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \ dg$

and *fresh-bound*:

wp-event (checked-staged-security-experiment-with-data-state *A*)
(staged-security-with-data-state-verifier-event
($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$))
adversary-initial-state $\leq$ Fresh

and *slot-bound*:

$\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap*
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$ Slot

and *merkle-bound*:

$\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*partial-merkle-inconsistency-bad*
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state

$(\text{staged-proof-transcript data}) \leq \text{Merkle}$
and *missing-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-recorded-base-chunk-auth-missing}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Missing}$
and *next-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Next}$
and *successor-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Successor}$
and *final-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Final}$
and *zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$

```

    adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-zero-round-final-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  Zero
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-verifier-tied-partial-candidate)
  adversary-initial-state  $\leq$ 
    (Fresh +
      hash-relation-budget-value
        ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
          card (B dg) * ceil-log (maxDegree + 1))
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget)) +
      (((Merkle + Missing) + Next + Successor + Final + Merkle) +
        Slot + Zero)
proof (rule
  checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-conflict-and-zero
    [OF wf controlled finite-B cover envelope fresh-bound])
fix data attacker-state
assume support:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show wp-event verify-monad
  (composition-fri-verifier-tied-sampled-assignment-conflict ?s) ?s
 $\leq$  ((Merkle + Missing) + Next + Successor + Final + Merkle) + Slot
by (rule
  wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-slot-recorded-chunk-gap-and-residuals)
  (rule slot-bound[OF support],
    rule merkle-bound[OF support],
    rule missing-bound[OF support],
    rule next-bound[OF support],
    rule successor-bound[OF support],
    rule final-bound[OF support])
show wp-event verify-monad
  (composition-fri-verifier-tied-zero-round-final-obstruction ?s) ?s  $\leq$  Zero
by (rule zero-bound[OF support])
qed

end

```

end

theory *Soundness-FRI-Trace-Candidate-Blocker*
imports *Soundness-FRI-Candidate-Transfer-Support*
begin

Diagnostic normalization facts for the remaining trace candidate-transfer branch. These lemmas do not close the branch; they identify the exact obstacle: the broad reachable witness may use a different authenticated root, query-index list, or opening set than the verifier-consumed/header-tied FRI witness.

context *soundness*
begin

definition *trace-fri-header-tied-low-candidate-untied-reachable-witness-gap*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
 $\iff$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table}$
 $\text{ fr0 query-idxs0 trace-openings0 trace-table0.}$
 $\text{ trace-fri-reachable-header-tie-gap s out} \wedge$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{ verifier-header-transcript s fr trace-roots trace-final as fri-dg}$
 $\text{ composition-roots composition-final rest} \wedge$
 $\text{ accepted-with-partial-trace-openings s out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ trace-table-low-degree trace-table} \wedge$
 $\text{ accepted-with-partial-trace-openings s out fr0 query-idxs0}$
 $\text{ trace-openings0} \wedge$
 $\text{ partial-trace-table-candidate trace-table0 trace-openings0} \wedge$
 $\neg \text{ trace-table-low-degree trace-table0} \wedge$
 $(\text{fr0} \neq \text{fr} \vee \text{query-idxs0} \neq \text{fri-query-idxs} \vee$
 $\text{ trace-openings0} \neq \text{trace-openings}))$

lemma *trace-fri-header-tied-low-candidate-gap-imp-untied-reachable-witness:*

assumes *trace-fri-header-tied-low-candidate-gap s out*

shows

trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr as rest trace-openings
trace-table
where *gap*: *trace-fri-reachable-header-tie-gap s out*
and *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *header*:
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
and *partial*:
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate*:
partial-trace-table-candidate trace-table trace-openings
and *low*: *trace-table-low-degree trace-table*
unfolding *trace-fri-header-tied-low-candidate-gap-def* **by** *blast*
have *raw*: *trace-fri-bad-with-reachable-partial-candidate s out*
using *gap* **unfolding** *trace-fri-reachable-header-tie-gap-def* **by** *simp*
from *trace-fri-bad-with-reachable-partial-candidateE[OF raw]*
obtain *fr0 query-idxs0 trace-openings0 trace-table0* **where** *evidence0*:
trace-fri-partial-candidate-evidence s out fr0 query-idxs0
trace-openings0 trace-table0
by *blast*
have *partial0*:
accepted-with-partial-trace-openings s out fr0 query-idxs0
trace-openings0
by (rule *trace-fri-partial-candidate-evidenceD(1)[OF evidence0]*)
have *candidate0*:
partial-trace-table-candidate trace-table0 trace-openings0
by (rule *trace-fri-partial-candidate-evidenceD(2)[OF evidence0]*)
have *not-low0*: $\neg$ *trace-table-low-degree trace-table0*
by (rule *trace-fri-partial-candidate-evidenceD(3)[OF evidence0]*)
have *untied*:
fr0 $\neq$ fr $\vee$ query-idxs0 $\neq$ fri-query-idxs $\vee$
trace-openings0 $\neq$ trace-openings
proof (rule *ccontr*)
assume $\neg$ (*fr0 $\neq$ fr $\vee$ query-idxs0 $\neq$ fri-query-idxs $\vee$*
trace-openings0 $\neq$ trace-openings)
then have *eqs*:
fr0 = fr
query-idxs0 = fri-query-idxs
trace-openings0 = trace-openings
by *auto*
have *trace-fri-bad-with-header-tied-partial-candidate s out*
unfolding *trace-fri-bad-with-header-tied-partial-candidate-def*
by (intro *exI conjI*)
(rule fri-openings, rule header,

```

      use partial0 candidate0 not-low0 eqs in auto)
    then show False
      using gap unfolding trace-fri-reachable-header-tie-gap-def by simp
qed
show ?thesis
  unfolding
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap-def
  by (intro exI conjI)
    (rule gap, rule fri-openings, rule header, rule partial,
     rule candidate, rule low, rule partial0, rule candidate0,
     rule not-low0, rule untied)
qed

lemma wp-trace-fri-header-tied-low-candidate-gap-bound-from-untied-witness:
  assumes
    wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
     $s \leq T$ 
  shows
    wp-event verify-monad
    (trace-fri-header-tied-low-candidate-gap s)  $s \leq T$ 
  by (rule order-trans[OF - assms])
    (rule wp-event-mono,
     rule trace-fri-header-tied-low-candidate-gap-imp-untied-reachable-witness)

lemma trace-fri-reachable-header-tie-gap-imp-merkle-or-untied-on-support:
  assumes support: out  $\in$  set-dist (execute verify-monad s)
    and gap: trace-fri-reachable-header-tie-gap s out
  shows
    partial-merkle-inconsistency-bad s out  $\vee$ 
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
  proof -
    have split:
      partial-merkle-inconsistency-bad s out  $\vee$ 
      trace-fri-header-tied-candidate-available-gap s out
    by (rule
        trace-fri-reachable-header-tie-gap-imp-merkle-or-candidate-available-on-support
        [OF support gap])
    then show ?thesis
  proof
    assume partial-merkle-inconsistency-bad s out
    then show ?thesis by simp
  next
    assume available: trace-fri-header-tied-candidate-available-gap s out
    have low-gap: trace-fri-header-tied-low-candidate-gap s out
      by (rule trace-fri-header-tied-candidate-available-gap-imp-low-candidate-gap
          [OF available])
    have
      trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out

```

by (rule
 trace-fri-header-tied-low-candidate-gap-imp-untied-reachable-witness
 [*OF low-gap*])
 then show ?thesis by simp
 qed
 qed

lemma *wp-trace-fri-reachable-header-tie-gap-bound-from-merkle-and-untied:*

assumes *merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M

and *untied-bound:*

wp-event verify-monad

(trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)

s ≤ U

shows

wp-event verify-monad

(trace-fri-reachable-header-tie-gap s) s ≤ M + U

proof –

have *split-bound:*

wp-event verify-monad

(trace-fri-reachable-header-tie-gap s) s ≤

wp-event verify-monad

(λout. partial-merkle-inconsistency-bad s out ∨

trace-fri-header-tied-low-candidate-untied-reachable-witness-gap

s out) s

by (rule *wp-event-mono-on-support*)

(rule trace-fri-reachable-header-tie-gap-imp-merkle-or-untied-on-support)

also have ... ≤

wp-event verify-monad (partial-merkle-inconsistency-bad s) s +

wp-event verify-monad

(trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)

s

by (rule *wp-event-union-bound*)

also have ... ≤ *M + U*

by (rule *add-mono[OF merkle-bound untied-bound]*)

finally show ?thesis .

qed

lemma *trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied:*

fixes *H M U :: prob*

assumes *header-bound:*

wp-event verify-monad

(trace-fri-bad-with-header-tied-partial-candidate s) s ≤ H

and *merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M

and *untied-bound:*

wp-event verify-monad

(trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)

s ≤ U

```

    and total-bound:  $H + (M + U) \leq \text{trace-fri-error}$ 
  shows trace-fri-reachable-partial-candidate-reduction-assumption  $s$ 
proof -
  have gap-bound:
    wp-event verify-monad
      (trace-fri-reachable-header-tie-gap  $s$ )  $s \leq M + U$ 
  by (rule
    wp-trace-fri-reachable-header-tie-gap-bound-from-merkle-and-untied
    [OF merkle-bound untied-bound])
  have reachable-bound:
    wp-event verify-monad
      (trace-fri-bad-with-reachable-partial-candidate  $s$ )  $s \leq H + (M + U)$ 
  by (rule wp-trace-fri-reachable-bound-from-header-tied-and-gap
    [OF header-bound gap-bound])
  show ?thesis
    unfolding trace-fri-reachable-partial-candidate-reduction-assumption-def
    by (rule order-trans[OF reachable-bound total-bound])
qed

end

end

```

```

theory Soundness-FRI-Trace-Untied-Cases
  imports Soundness-FRI-Trace-Candidate-Blocker
begin

```

Case split for the remaining trace FRI untied-witness residual.

The broad residual says that the reachable low-degree witness consumed by the verifier and the separate non-low-degree reachable witness disagree in at least one of: authenticated root, query-index list, or sampled openings. This theory exposes these three causes as separate events so subsequent proof layers can bound them independently.

```

context soundness
begin

```

definition *trace-fri-header-tied-low-candidate-untied-reachable-witness-case*

```

:: ('f  $\Rightarrow$  'f  $\Rightarrow$  nat list  $\Rightarrow$  nat list  $\Rightarrow$ 
  'f authenticated-opening list list  $\Rightarrow$ 
  'f authenticated-opening list list  $\Rightarrow$  bool)  $\Rightarrow$ 
('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
(unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool

```

where

```

trace-fri-header-tied-low-candidate-untied-reachable-witness-case  $P$   $s$  out
 $\longleftrightarrow$ 
( $\exists$  trace-roots trace-bs trace-final fri-dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr as rest trace-openings trace-table
  fr0 query-idxs0 trace-openings0 trace-table0.

```

trace-fri-reachable-header-tie-gap s out $\wedge$
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
trace-table-low-degree trace-table $\wedge$
accepted-with-partial-trace-openings s out fr0 query-idxs0
trace-openings0 $\wedge$
partial-trace-table-candidate trace-table0 trace-openings0 $\wedge$
 $\neg$ *trace-table-low-degree trace-table0* $\wedge$
P fr0 fr query-idxs0 fri-query-idxs trace-openings0 trace-openings)

lemma *trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI*:

assumes

trace-fri-reachable-header-tie-gap s out
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
partial-trace-table-candidate trace-table trace-openings
trace-table-low-degree trace-table
accepted-with-partial-trace-openings s out fr0 query-idxs0
trace-openings0
partial-trace-table-candidate trace-table0 trace-openings0
 $\neg$ *trace-table-low-degree trace-table0*
P fr0 fr query-idxs0 fri-query-idxs trace-openings0 trace-openings

shows

trace-fri-header-tied-low-candidate-untied-reachable-witness-case
P s out

unfolding

trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def

by (*intro exI conjI*)

(*rule assms*) $+$

definition *trace-fri-header-tied-low-candidate-untied-root-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-low-candidate-untied-root-gap s out
 $\longleftrightarrow$

trace-fri-header-tied-low-candidate-untied-reachable-witness-case
 $(\lambda fr0\ fr\ \dots\ fr0 \neq fr)\ s\ out$

definition *trace-fri-header-tied-low-candidate-untied-query-gap*
 $:: (f, 'a)\ protocol-channel-scheme \Rightarrow$
 $(unit\ list \times (f, 'a)\ protocol-channel-scheme)\ option \Rightarrow bool$

where

trace-fri-header-tied-low-candidate-untied-query-gap $s\ out$
 $\longleftrightarrow$
trace-fri-header-tied-low-candidate-untied-reachable-witness-case
 $(\lambda -\ query-idxs0\ fri-query-idxs\ \dots$
 $query-idxs0 \neq fri-query-idxs)\ s\ out$

definition *trace-fri-header-tied-low-candidate-untied-openings-gap*
 $:: (f, 'a)\ protocol-channel-scheme \Rightarrow$
 $(unit\ list \times (f, 'a)\ protocol-channel-scheme)\ option \Rightarrow bool$

where

trace-fri-header-tied-low-candidate-untied-openings-gap $s\ out$
 $\longleftrightarrow$
trace-fri-header-tied-low-candidate-untied-reachable-witness-case
 $(\lambda -\ \dots\ trace-openings0\ trace-openings.$
 $trace-openings0 \neq trace-openings)\ s\ out$

definition *trace-fri-header-tied-low-candidate-untied-normalized-openings-gap*
 $:: (f, 'a)\ protocol-channel-scheme \Rightarrow$
 $(unit\ list \times (f, 'a)\ protocol-channel-scheme)\ option \Rightarrow bool$

where

trace-fri-header-tied-low-candidate-untied-normalized-openings-gap $s\ out$
 $\longleftrightarrow$
trace-fri-header-tied-low-candidate-untied-reachable-witness-case
 $(\lambda fr0\ fr\ query-idxs0\ fri-query-idxs\ trace-openings0\ trace-openings.$
 $fr0 = fr \wedge query-idxs0 = fri-query-idxs \wedge$
 $trace-openings0 \neq trace-openings)\ s\ out$

definition *trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap*
 $:: (f, 'a)\ protocol-channel-scheme \Rightarrow$
 $(unit\ list \times (f, 'a)\ protocol-channel-scheme)\ option \Rightarrow bool$

where

trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap $s\ out$
 $\longleftrightarrow$
trace-fri-header-tied-low-candidate-untied-reachable-witness-case
 $(\lambda fr0\ fr\ query-idxs0\ fri-query-idxs\ trace-openings0\ trace-openings.$
 $(fr0 \neq fr \vee query-idxs0 \neq fri-query-idxs) \wedge$
 $trace-openings0 \neq trace-openings)\ s\ out$

lemma *trace-fri-header-tied-low-candidate-untied-reachable-witness-gap-imp-cases:*

assumes

trace-fri-header-tied-low-candidate-untied-reachable-witness-gap $s\ out$

shows

```

    trace-fri-header-tied-low-candidate-untied-root-gap s out ∨
    trace-fri-header-tied-low-candidate-untied-query-gap s out ∨
    trace-fri-header-tied-low-candidate-untied-openings-gap s out
proof –
from assms obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as rest trace-openings
    trace-table fr0 query-idxs0 trace-openings0 trace-table0 where
gap: trace-fri-reachable-header-tie-gap s out
and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
and header:
    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest
and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
and candidate:
    partial-trace-table-candidate trace-table trace-openings
and low: trace-table-low-degree trace-table
and partial0:
    accepted-with-partial-trace-openings s out fr0 query-idxs0
    trace-openings0
and candidate0:
    partial-trace-table-candidate trace-table0 trace-openings0
and not-low0: ¬ trace-table-low-degree trace-table0
and mismatch:
    fr0 ≠ fr ∨ query-idxs0 ≠ fri-query-idxs ∨
    trace-openings0 ≠ trace-openings
unfolding
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap-def
by blast
from mismatch show ?thesis
proof
    assume root-mismatch: fr0 ≠ fr
    have trace-fri-header-tied-low-candidate-untied-root-gap s out
    unfolding trace-fri-header-tied-low-candidate-untied-root-gap-def
    by (rule
        trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI)
        (rule gap, rule fri-openings, rule header, rule partial,
         rule candidate, rule low, rule partial0, rule candidate0,
         rule not-low0, rule root-mismatch)
    then show ?thesis by simp
next
    assume tail:
        query-idxs0 ≠ fri-query-idxs ∨

```

```

    trace-openings0 ≠ trace-openings
  then show ?thesis
proof
  assume query-mismatch: query-idxs0 ≠ fri-query-idxs
  have trace-fri-header-tied-low-candidate-untied-query-gap s out
    unfolding trace-fri-header-tied-low-candidate-untied-query-gap-def
    by (rule
      trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI)
      (rule gap, rule fri-openings, rule header, rule partial,
        rule candidate, rule low, rule partial0, rule candidate0,
        rule not-low0, rule query-mismatch)
  then show ?thesis by simp
next
  assume openings-mismatch:
    trace-openings0 ≠ trace-openings
  have trace-fri-header-tied-low-candidate-untied-openings-gap s out
    unfolding trace-fri-header-tied-low-candidate-untied-openings-gap-def
    by (rule
      trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI)
      (rule gap, rule fri-openings, rule header, rule partial,
        rule candidate, rule low, rule partial0, rule candidate0,
        rule not-low0, rule openings-mismatch)
  then show ?thesis by simp
qed
qed
qed

lemma trace-fri-header-tied-low-candidate-untied-root-gap-imp-witness-gap:
  assumes trace-fri-header-tied-low-candidate-untied-root-gap s out
  shows
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
  using assms
  unfolding
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap-def
    trace-fri-header-tied-low-candidate-untied-root-gap-def
    trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
  by blast

lemma trace-fri-header-tied-low-candidate-untied-query-gap-imp-witness-gap:
  assumes trace-fri-header-tied-low-candidate-untied-query-gap s out
  shows
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
  using assms
  unfolding
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap-def
    trace-fri-header-tied-low-candidate-untied-query-gap-def
    trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
  by blast

```

lemma *trace-fri-header-tied-low-candidate-untied-openings-gap-imp-witness-gap:*
assumes *trace-fri-header-tied-low-candidate-untied-openings-gap s out*
shows
trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
using *assms*
unfolding
trace-fri-header-tied-low-candidate-untied-reachable-witness-gap-def
trace-fri-header-tied-low-candidate-untied-openings-gap-def
trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
by *blast*

lemma *merkle-path-bound-unique-path-if-clean:*
assumes *clean: $\neg$ hash-map-output-collision s*
and *first: merkle-path-bound rt len idx v path s*
and *second: merkle-path-bound rt len idx v path' s*
and *same-length: length path = length path'*
shows *path = path'*
using *first second same-length*
proof (*induction path arbitrary: path' rt len idx*)
case *Nil*
then show *?case by simp*
next
case (*Cons sibling path*)
from *Cons.premis(3)* **obtain** *sibling' path'' where*
path': path' = sibling' # path''
and *lengths: length path = length path''*
by (*cases path'*) *auto*
show *?case*
proof (*cases idx < len div 2*)
case *True*
obtain *child where*
child:
merkle-path-bound child (len div 2) idx v path s
and *first-lookup:*
fmllookup (HashMap s) (MerkleNode child sibling) = Some rt
using *Cons.premis(1) True by auto*
obtain *child' where*
child':
merkle-path-bound child' (len div 2) idx v path'' s
and *second-lookup:*
fmllookup (HashMap s) (MerkleNode child' sibling') = Some rt
using *Cons.premis(2) True path' by auto*
have *node-eq:*
MerkleNode child sibling = MerkleNode child' sibling'
proof (*rule ccontr*)
assume *neg:*
MerkleNode child sibling $\neq$ MerkleNode child' sibling'
have *hash-map-output-collision s*
by (*rule hash-map-output-collisionI*)

```

    [OF neq first-lookup second-lookup])
  then show False using clean by contradiction
qed
have child-eq: child = child' using node-eq by simp
have sibling-eq: sibling = sibling' using node-eq by simp
have path-eq: path = path''
  by (rule Cons.IH[OF child - lengths])
  (use child' child-eq in simp)
show ?thesis
  using path' sibling-eq path-eq by simp
next
case False
obtain child where
  child:
    merkle-path-bound child (len div 2) (idx - len div 2)
    v path s
  and first-lookup:
    fmllookup (HashMap s) (MerkleNode sibling child) = Some rt
  using Cons.prem1 False by auto
obtain child' where
  child':
    merkle-path-bound child' (len div 2) (idx - len div 2)
    v path'' s
  and second-lookup:
    fmllookup (HashMap s) (MerkleNode sibling' child') = Some rt
  using Cons.prem2 False path' by auto
have node-eq:
  MerkleNode sibling child = MerkleNode sibling' child'
proof (rule ccontr)
  assume neg:
    MerkleNode sibling child  $\neq$  MerkleNode sibling' child'
  have hash-map-output-collision s
    by (rule hash-map-output-collisionI
      [OF neq first-lookup second-lookup])
  then show False using clean by contradiction
qed
have child-eq: child = child' using node-eq by simp
have sibling-eq: sibling = sibling' using node-eq by simp
have path-eq: path = path''
  by (rule Cons.IH[OF child - lengths])
  (use child' child-eq in simp)
show ?thesis
  using path' sibling-eq path-eq by simp
qed
qed

lemma authenticated-opening-eq-if-clean:
  assumes clean:  $\neg$  hash-map-output-collision s
  and first: authenticated-opening-in s opening

```

and *second*: *authenticated-opening-in s opening'*
and *same-root*: *opening-root opening = opening-root opening'*
and *same-length*: *opening-length opening = opening-length opening'*
and *same-index*: *opening-index opening = opening-index opening'*
and *same-value*: *opening-value opening = opening-value opening'*
shows *opening = opening'*
proof –
have *path-lengths*:
length (opening-path opening) = length (opening-path opening')
using *first second same-length*
unfolding *authenticated-opening-in-def* **by** *simp*
have *first-bound*:
merkle-path-bound
(opening-root opening)
(opening-length opening)
(opening-index opening)
(opening-value opening)
(opening-path opening)
s
using *first unfolding authenticated-opening-in-def* **by** *blast*
have *second-bound*:
merkle-path-bound
(opening-root opening)
(opening-length opening)
(opening-index opening)
(opening-value opening)
(opening-path opening')
s
using *second same-root same-length same-index same-value*
unfolding *authenticated-opening-in-def* **by** *simp*
have *paths*:
opening-path opening = opening-path opening'
by (*rule merkle-path-bound-unique-path-if-clean*
[*OF clean first-bound second-bound path-lengths*])
show *?thesis*
using *same-root same-length same-index same-value paths*
by (*cases opening; cases opening'; simp*)
qed

lemma *accepted-with-partial-trace-openings-same-root-query-openings-eq-if-clean*:
assumes *partial*:
accepted-with-partial-trace-openings s (Some (result, final-state)) fr
query-idxs trace-openings
and *partial0*:
accepted-with-partial-trace-openings s (Some (result, final-state)) fr
query-idxs trace-openings0
and *clean*: $\neg$ *hash-map-output-collision final-state*
shows *trace-openings0 = trace-openings*
proof –

```

have len-openings: length trace-openings = rounds
  using partial unfolding accepted-with-partial-trace-openings-def by blast
have len-openings0: length trace-openings0 = rounds
  using partial0 unfolding accepted-with-partial-trace-openings-def by blast
have outer-len: length trace-openings0 = length trace-openings
  using len-openings len-openings0 by simp
have outer-nth:
   $\bigwedge i. i < \text{length } \text{trace-openings0} \implies$ 
     $\text{trace-openings0} ! i = \text{trace-openings} ! i$ 
proof -
  fix i
  assume i-bound0:  $i < \text{length } \text{trace-openings0}$ 
  then have i-bound:  $i < \text{rounds}$ 
    using len-openings0 by simp
  have idxs:
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
    using partial i-bound
    unfolding accepted-with-partial-trace-openings-def by blast
  have idxs0:
    map opening-index (trace-openings0 ! i) =
      powers-scaled (query-idxs ! i)
    using partial0 i-bound
    unfolding accepted-with-partial-trace-openings-def by blast
  have table:
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
    using partial i-bound
    unfolding accepted-with-partial-trace-openings-def by blast
  have table0:
    partial-authenticated-table fr (scale * clength)
      (trace-openings0 ! i) final-state
    using partial0 i-bound
    unfolding accepted-with-partial-trace-openings-def by blast
  have len-inner:
    length (trace-openings0 ! i) = length (trace-openings ! i)
    using idxs idxs0 by (metis length-map)
  have inner-nth:
     $\bigwedge j. j < \text{length } (\text{trace-openings0} ! i) \implies$ 
       $\text{nth } (\text{nth } \text{trace-openings0 } i) j = \text{nth } (\text{nth } \text{trace-openings } i) j$ 
  proof -
    fix j
    assume j-bound0:  $j < \text{length } (\text{trace-openings0} ! i)$ 
    have j-bound:  $j < \text{length } (\text{trace-openings} ! i)$ 
      using j-bound0 len-inner by simp
    define opn0 where opn0 = nth (nth trace-openings0 i) j
    define opn where opn = nth (nth trace-openings i) j
    have opn0-in:  $\text{opn0} \in \text{set } (\text{trace-openings0} ! i)$ 
      unfolding opn0-def using j-bound0 by simp

```

```

have opn-in: opn ∈ set (trace-openings ! i)
  unfolding opn-def using j-bound by simp
have auth0: authenticated-opening-in final-state opn0
  using table0 opn0-in unfolding partial-authenticated-table-def
  by blast
have auth: authenticated-opening-in final-state opn
  using table opn-in unfolding partial-authenticated-table-def
  by blast
have root0: opening-root opn0 = fr
  using table0 opn0-in unfolding partial-authenticated-table-def
  by blast
have root: opening-root opn = fr
  using table opn-in unfolding partial-authenticated-table-def
  by blast
have length0: opening-length opn0 = scale * clength
  using table0 opn0-in unfolding partial-authenticated-table-def
  by blast
have length: opening-length opn = scale * clength
  using table opn-in unfolding partial-authenticated-table-def
  by blast
have same-index: opening-index opn0 = opening-index opn
proof -
  have map opening-index (trace-openings0 ! i) ! j =
    map opening-index (trace-openings ! i) ! j
  using idxs idxs0 by simp
  then show ?thesis
    unfolding opn0-def opn-def using j-bound0 j-bound by simp
qed
have same-value: opening-value opn0 = opening-value opn
proof (rule ccontr)
  assume neq: opening-value opn0 ≠ opening-value opn
  have hash-map-output-collision final-state
    by (rule inconsistent-authenticated-openings-imp-hash-collision
      [OF auth0 auth])
  (use root0 root length0 length same-index neq in simp-all)
  then show False using clean by contradiction
qed
have opn-eq: opn0 = opn
  by (rule authenticated-opening-eq-if-clean[OF clean auth0 auth])
  (use root0 root length0 length same-index same-value in simp-all)
show nth (nth trace-openings0 i) j = nth (nth trace-openings i) j
  using opn-eq unfolding opn0-def opn-def by simp
qed
show trace-openings0 ! i = trace-openings ! i
  using len-inner inner-nth by (metis nth-equalityI)
qed
show ?thesis
  using outer-len outer-nth by (metis nth-equalityI)
qed

```


lemma *trace-fri-header-tied-low-candidate-untied-normalized-openings-gap-imp-hash-collision:*
assumes
trace-fri-header-tied-low-candidate-untied-normalized-openings-gap s out
shows *hash-map-output-collision-bad s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as rest trace-openings
trace-table fr0 query-idxs0 trace-openings0 trace-table0 **where**
partial:
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *partial0:*
accepted-with-partial-trace-openings s out fr0 query-idxs0
trace-openings0
and *root-eq: fr0 = fr*
and *query-eq: query-idxs0 = fri-query-idxs*
and *openings-neq: trace-openings0 ≠ trace-openings*
unfolding
trace-fri-header-tied-low-candidate-untied-normalized-openings-gap-def
trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
by *blast*
from *partial* **obtain** *result final-state* **where** *out:*
out = Some (result, final-state)
unfolding *accepted-with-partial-trace-openings-def* **by** *blast*
show *?thesis*
proof (*cases hash-map-output-collision final-state*)
case *True*
then show *?thesis*
unfolding *hash-map-output-collision-bad-def accepted-def*
using *out* **by** *simp*
next
case *clean: False*
have *partial-some:*
accepted-with-partial-trace-openings s (Some (result, final-state))
fr fri-query-idxs trace-openings
using *partial out* **by** *simp*
have *partial0-some:*
accepted-with-partial-trace-openings s (Some (result, final-state))
fr fri-query-idxs trace-openings0
using *partial0 out root-eq query-eq* **by** *simp*
have *trace-openings0 = trace-openings*
by (*rule*
accepted-with-partial-trace-openings-same-root-query-openings-eq-if-clean
[OF partial-some partial0-some clean])
then show *?thesis*
using *openings-neq* **by** *contradiction*
qed

qed

lemma *trace-fri-header-tied-low-candidate-untied-normalized-openings-gap-mono-hash-collision:*

wp-event verify-monad

(trace-fri-header-tied-low-candidate-untied-normalized-openings-gap s) s ≤

wp-event verify-monad (hash-map-output-collision-bad s) s

by *(rule wp-event-mono)*

(rule

trace-fri-header-tied-low-candidate-untied-normalized-openings-gap-imp-hash-collision)

lemma *trace-fri-header-tied-low-candidate-untied-openings-gap-imp-normalized-or-nonnormalized:*

assumes *trace-fri-header-tied-low-candidate-untied-openings-gap s out*

shows

trace-fri-header-tied-low-candidate-untied-normalized-openings-gap s out ∨

trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap s out

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr as rest trace-openings

trace-table fr0 query-idxs0 trace-openings0 trace-table0 **where**

gap: trace-fri-reachable-header-tie-gap s out

and *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers

and *header:*

verifier-header-transcript s fr trace-roots trace-final as fri-dg

composition-roots composition-final rest

and *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs

trace-openings

and *candidate:*

partial-trace-table-candidate trace-table trace-openings

and *low: trace-table-low-degree trace-table*

and *partial0:*

accepted-with-partial-trace-openings s out fr0 query-idxs0

trace-openings0

and *candidate0:*

partial-trace-table-candidate trace-table0 trace-openings0

and *not-low0: ¬ trace-table-low-degree trace-table0*

and *openings-neg: trace-openings0 ≠ trace-openings*

unfolding *trace-fri-header-tied-low-candidate-untied-openings-gap-def*

trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def

by *blast*

show *?thesis*

proof *(cases fr0 = fr ∧ query-idxs0 = fri-query-idxs)*

case *True*

have *trace-fri-header-tied-low-candidate-untied-normalized-openings-gap*

```

  s out
  unfolding
    trace-fri-header-tied-low-candidate-untied-normalized-openings-gap-def
  by (rule
    trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI
    [OF gap fri-openings header partial candidate low partial0 candidate0
      not-low0])
    (use True openings-neq in simp)
  then show ?thesis by simp
next
case False
then have diff: fr0 ≠ fr ∨ query-idxs0 ≠ fri-query-idxs
  by blast
have trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap
  s out
  unfolding
    trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-def
  by (rule
    trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI
    [OF gap fri-openings header partial candidate low partial0 candidate0
      not-low0])
    (use diff openings-neq in simp)
  then show ?thesis by simp
qed
qed

lemma wp-trace-fri-header-tied-low-candidate-untied-openings-gap-bound-from-hash-and-nonnormalized:
  fixes hash-err nonnormalized-err :: prob
  assumes hash-bound:
    wp-event verify-monad (hash-map-output-collision-bad s) s ≤
      hash-err
  and nonnormalized-bound:
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap s)
      s ≤ nonnormalized-err
  shows
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-openings-gap s) s ≤
      hash-err + nonnormalized-err
proof -
  have wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-openings-gap s) s ≤
    wp-event verify-monad
      (λout.
        trace-fri-header-tied-low-candidate-untied-normalized-openings-gap
          s out ∨
          trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap
            s out) s
  by (rule wp-event-mono)

```

(rule
 trace-fri-header-tied-low-candidate-untied-openings-gap-imp-normalized-or-nonnormalized)
also have ... $\leq$
 wp-event verify-monad
 (trace-fri-header-tied-low-candidate-untied-normalized-openings-gap s)
 s +
 wp-event verify-monad
 (trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap
 s) s
by (rule wp-event-union-bound)
also have ... $\leq$ hash-err + nonnormalized-err
by (rule add-mono)
 (rule order-trans[
 OF trace-fri-header-tied-low-candidate-untied-normalized-openings-gap-mono-hash-collision
 hash-bound],
 rule nonnormalized-bound)
finally show ?thesis .
qed

lemma trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-imp-root-or-query:
assumes
 trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap
 s out
shows
 trace-fri-header-tied-low-candidate-untied-root-gap s out $\vee$
 trace-fri-header-tied-low-candidate-untied-query-gap s out
proof –
from assms **obtain** trace-roots trace-bs trace-final fri-dg
 composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers fr as rest trace-openings
 trace-table fr0 query-idxs0 trace-openings0 trace-table0 **where**
 gap: trace-fri-reachable-header-tie-gap s out
and fri-openings:
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
and header:
 verifier-header-transcript s fr trace-roots trace-final as fri-dg
 composition-roots composition-final rest
and partial:
 accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings
and candidate:
 partial-trace-table-candidate trace-table trace-openings
and low: trace-table-low-degree trace-table
and partial0:
 accepted-with-partial-trace-openings s out fr0 query-idxs0
 trace-openings0

```

and candidate0:
  partial-trace-table-candidate trace-table0 trace-openings0
and not-low0:  $\neg$  trace-table-low-degree trace-table0
and mismatch:  $fr0 \neq fr \vee query-idxs0 \neq fri-query-idxs$ 
unfolding
  trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-def
  trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
by blast
from mismatch show ?thesis
proof
  assume root-mismatch:  $fr0 \neq fr$ 
  have trace-fri-header-tied-low-candidate-untied-root-gap s out
    unfolding trace-fri-header-tied-low-candidate-untied-root-gap-def
    by (rule
      trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI)
      (rule gap, rule fri-openings, rule header, rule partial,
       rule candidate, rule low, rule partial0, rule candidate0,
       rule not-low0, rule root-mismatch)
  then show ?thesis by simp
next
  assume query-mismatch:  $query-idxs0 \neq fri-query-idxs$ 
  have trace-fri-header-tied-low-candidate-untied-query-gap s out
    unfolding trace-fri-header-tied-low-candidate-untied-query-gap-def
    by (rule
      trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI)
      (rule gap, rule fri-openings, rule header, rule partial,
       rule candidate, rule low, rule partial0, rule candidate0,
       rule not-low0, rule query-mismatch)
  then show ?thesis by simp
qed
qed

lemma wp-trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-bound-from-root-query:
  fixes root-err query-err :: prob
  assumes root-bound:
    wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-root-gap s) s  $\leq$ 
    root-err
  and query-bound:
    wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-query-gap s) s  $\leq$ 
    query-err
  shows
    wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap s)
    s  $\leq$  root-err + query-err
proof –
  have wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap s)

```

$s \leq$
wp-event verify-monad
 ($\lambda out.$
 trace-fri-header-tied-low-candidate-untied-root-gap s out $\vee$
 trace-fri-header-tied-low-candidate-untied-query-gap s out) s
 by (rule *wp-event-mono*)
 (rule
 trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-imp-root-or-query)
 also have ... $\leq$
 wp-event verify-monad
 (*trace-fri-header-tied-low-candidate-untied-root-gap s*) $s +$
 wp-event verify-monad
 (*trace-fri-header-tied-low-candidate-untied-query-gap s*) s
 by (rule *wp-event-union-bound*)
 also have ... $\leq$ *root-err + query-err*
 by (rule *add-mono[OF root-bound query-bound]*)
 finally show ?thesis .
 qed

lemma *trace-fri-header-tied-low-candidate-untied-root-gap-imp-query-or-openings:*

assumes *trace-fri-header-tied-low-candidate-untied-root-gap s out*

shows

trace-fri-header-tied-low-candidate-untied-query-gap s out $\vee$
trace-fri-header-tied-low-candidate-untied-openings-gap s out

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
 composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers fr as rest trace-openings
 trace-table fr0 query-idxs0 trace-openings0 trace-table0 **where**
gap: trace-fri-reachable-header-tie-gap s out
and *fri-openings:*
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
and *header:*
 verifier-header-transcript s fr trace-roots trace-final as fri-dg
 composition-roots composition-final rest
and *partial:*
 accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings
and *candidate:*
 partial-trace-table-candidate trace-table trace-openings
and *low: trace-table-low-degree trace-table*
and *partial0:*
 accepted-with-partial-trace-openings s out fr0 query-idxs0
 trace-openings0
and *candidate0:*
 partial-trace-table-candidate trace-table0 trace-openings0

```

and not-low0:  $\neg$  trace-table-low-degree trace-table0
and root-mismatch: fr0  $\neq$  fr
unfolding trace-fri-header-tied-low-candidate-untied-root-gap-def
  trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
by blast
show ?thesis
proof (cases query-idxs0  $\neq$  fri-query-idxs)
  case True
  have trace-fri-header-tied-low-candidate-untied-query-gap s out
    unfolding trace-fri-header-tied-low-candidate-untied-query-gap-def
    by (rule
      trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI)
    (rule gap, rule fri-openings, rule header, rule partial,
      rule candidate, rule low, rule partial0, rule candidate0,
      rule not-low0, rule True)
  then show ?thesis by simp
next
  case query-same: False
  show ?thesis
  proof (cases trace-openings0  $\neq$  trace-openings)
    case True
    have trace-fri-header-tied-low-candidate-untied-openings-gap s out
      unfolding trace-fri-header-tied-low-candidate-untied-openings-gap-def
      by (rule
        trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI)
      (rule gap, rule fri-openings, rule header, rule partial,
        rule candidate, rule low, rule partial0, rule candidate0,
        rule not-low0, rule True)
    then show ?thesis by simp
  next
  case openings-same: False
  have same-query: query-idxs0 = fri-query-idxs
    using query-same by simp
  have same-openings: trace-openings0 = trace-openings
    using openings-same by simp
  have len-openings: length trace-openings = rounds
    using partial unfolding accepted-with-partial-trace-openings-def
    by blast
  have idxs0:
    map opening-index (trace-openings ! 0) =
      powers-scaled (fri-query-idxs ! 0)
    using partial rounds-positive
    unfolding accepted-with-partial-trace-openings-def by blast
  have nonempty: trace-openings ! 0  $\neq$  []
  proof -
    have powers-scaled (fri-query-idxs ! 0)  $\neq$  []
      unfolding powers-scaled-def using powers-pos by simp
    then show ?thesis
      using idxs0 by auto

```

```

qed
then obtain opn where opn-in: opn ∈ set (trace-openings ! 0)
  by (cases trace-openings ! 0) auto
have table:
  partial-authenticated-table fr (scale * clength)
  (trace-openings ! 0) (snd (the out))
proof -
  from partial obtain result final-state where
    out: out = Some (result, final-state)
  and table':
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! 0) final-state
  using rounds-positive
  unfolding accepted-with-partial-trace-openings-def by blast
  then show ?thesis by simp
qed
have table0:
  partial-authenticated-table fr0 (scale * clength)
  (trace-openings ! 0) (snd (the out))
proof -
  from partial0 obtain result final-state where
    out: out = Some (result, final-state)
  and table':
    partial-authenticated-table fr0 (scale * clength)
    (trace-openings0 ! 0) final-state
  using rounds-positive
  unfolding accepted-with-partial-trace-openings-def by blast
  then show ?thesis
    using same-openings by simp
qed
have opening-root opn = fr
  using table opn-in unfolding partial-authenticated-table-def by blast
moreover have opening-root opn = fr0
  using table0 opn-in unfolding partial-authenticated-table-def by blast
ultimately have fr0 = fr by simp
then show ?thesis
  using root-mismatch by contradiction
qed
qed
qed

```

lemma *trace-fri-header-tied-low-candidate-untied-gap-imp-query-or-openings:*
assumes
trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
shows
trace-fri-header-tied-low-candidate-untied-query-gap s out ∨
trace-fri-header-tied-low-candidate-untied-openings-gap s out
proof -
have *cases:*


```

    trace-fri-header-tied-low-candidate-untied-root-gap s out ∨
    trace-fri-header-tied-low-candidate-untied-query-gap s out ∨
    trace-fri-header-tied-low-candidate-untied-openings-gap s out
  by (rule
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap-imp-cases
    [OF assms])
  then show ?thesis
proof
  assume trace-fri-header-tied-low-candidate-untied-root-gap s out
  then show ?thesis
    by (rule
      trace-fri-header-tied-low-candidate-untied-root-gap-imp-query-or-openings)
  next
  assume
    trace-fri-header-tied-low-candidate-untied-query-gap s out ∨
    trace-fri-header-tied-low-candidate-untied-openings-gap s out
  then show ?thesis .
qed
qed

lemma trace-untied-powers-scaled-eq-imp-eq:
  assumes powers-scaled idx = powers-scaled idx'
  shows idx = idx'
proof -
  have left: powers-scaled idx ! 0 = idx
    unfolding powers-scaled-def using powers-pos by simp
  have right: powers-scaled idx' ! 0 = idx'
    unfolding powers-scaled-def using powers-pos by simp
  have powers-scaled idx ! 0 = powers-scaled idx' ! 0
    using assms by simp
  then show ?thesis
    using left right by simp
qed

lemma trace-fri-header-tied-low-candidate-untied-query-gap-imp-openings:
  assumes trace-fri-header-tied-low-candidate-untied-query-gap s out
  shows trace-fri-header-tied-low-candidate-untied-openings-gap s out
proof -
  from assms obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as rest trace-openings
    trace-table fr0 query-idxs0 trace-openings0 trace-table0 where
    gap: trace-fri-reachable-header-tie-gap s out
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  and header:

```

```

    verifier-header-transcript s fr trace-roots trace-final as fri-dg
      composition-roots composition-final rest
  and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
      trace-openings
  and candidate:
    partial-trace-table-candidate trace-table trace-openings
  and low: trace-table-low-degree trace-table
  and partial0:
    accepted-with-partial-trace-openings s out fr0 query-idxs0
      trace-openings0
  and candidate0:
    partial-trace-table-candidate trace-table0 trace-openings0
  and not-low0:  $\neg$  trace-table-low-degree trace-table0
  and query-mismatch: query-idxs0  $\neq$  fri-query-idxs
  unfolding trace-fri-header-tied-low-candidate-untied-query-gap-def
    trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
  by blast
show ?thesis
proof (cases trace-openings0  $\neq$  trace-openings)
  case True
  show ?thesis
    unfolding trace-fri-header-tied-low-candidate-untied-openings-gap-def
    by (rule
      trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI)
      (rule gap, rule fri-openings, rule header, rule partial,
        rule candidate, rule low, rule partial0, rule candidate0,
        rule not-low0, rule True)
next
  case openings-same: False
  have same-openings: trace-openings0 = trace-openings
    using openings-same by simp
  have len-query: length fri-query-idxs = rounds
    using partial unfolding accepted-with-partial-trace-openings-def
    by blast
  have len-query0: length query-idxs0 = rounds
    using partial0 unfolding accepted-with-partial-trace-openings-def
    by blast
  have nth-round-eq:
     $\bigwedge i. i < \text{rounds} \implies \text{query-idxs0} ! i = \text{fri-query-idxs} ! i$ 
  proof -
    fix i
    assume i-bound:  $i < \text{rounds}$ 
    have idxs:
      map opening-index (trace-openings ! i) =
        powers-scaled (fri-query-idxs ! i)
    using partial i-bound
    unfolding accepted-with-partial-trace-openings-def by blast
  have idxs0:

```

```

    map opening-index (trace-openings0 ! i) =
      powers-scaled (query-idxs0 ! i)
    using partial0 i-bound
    unfolding accepted-with-partial-trace-openings-def by blast
  have powers-scaled (query-idxs0 ! i) =
    powers-scaled (fri-query-idxs ! i)
    using idxs idxs0 same-openings by simp
  then show query-idxs0 ! i = fri-query-idxs ! i
    by (rule trace-untied-powers-scaled-eq-imp-eq)
qed
have len-eq: length query-idxs0 = length fri-query-idxs
  using len-query len-query0 by simp
have query-idxs0 = fri-query-idxs
proof (rule nth-equalityI)
  show length query-idxs0 = length fri-query-idxs
    by (rule len-eq)
next
  fix i
  assume i < length query-idxs0
  then have i < rounds
    using len-query0 by simp
  then show query-idxs0 ! i = fri-query-idxs ! i
    by (rule nth-round-eq)
qed
then show ?thesis
  using query-mismatch by contradiction
qed
qed

lemma trace-fri-header-tied-low-candidate-untied-gap-imp-openings:
  assumes
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
  shows
    trace-fri-header-tied-low-candidate-untied-openings-gap s out
proof -
  have split:
    trace-fri-header-tied-low-candidate-untied-query-gap s out ∨
    trace-fri-header-tied-low-candidate-untied-openings-gap s out
  by (rule
    trace-fri-header-tied-low-candidate-untied-gap-imp-query-or-openings
    [OF assms])
  then show ?thesis
proof
  assume trace-fri-header-tied-low-candidate-untied-query-gap s out
  then show ?thesis
    by (rule
      trace-fri-header-tied-low-candidate-untied-query-gap-imp-openings)
next
  assume trace-fri-header-tied-low-candidate-untied-openings-gap s out

```

```

    then show ?thesis .
qed
qed

lemma trace-fri-header-tied-low-candidate-untied-gap-imp-hash-root-or-query:
  assumes
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
  shows
    hash-map-output-collision-bad s out  $\vee$ 
    trace-fri-header-tied-low-candidate-untied-root-gap s out  $\vee$ 
    trace-fri-header-tied-low-candidate-untied-query-gap s out
proof -
  have openings:
    trace-fri-header-tied-low-candidate-untied-openings-gap s out
  by (rule trace-fri-header-tied-low-candidate-untied-gap-imp-openings
      [OF assms])
  have normalized-or:
    trace-fri-header-tied-low-candidate-untied-normalized-openings-gap
      s out  $\vee$ 
    trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap
      s out
  by (rule
      trace-fri-header-tied-low-candidate-untied-openings-gap-imp-normalized-or-nonnormalized
      [OF openings])
  then show ?thesis
proof
  assume
    trace-fri-header-tied-low-candidate-untied-normalized-openings-gap
      s out
  then have hash-map-output-collision-bad s out
  by (rule
      trace-fri-header-tied-low-candidate-untied-normalized-openings-gap-imp-hash-collision)
  then show ?thesis by simp
next
  assume
    trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap
      s out
  then show ?thesis
  using
    trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-imp-root-or-query
  by blast
qed
qed

lemma wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-cases:
  fixes root-err query-err opening-err :: prob
  assumes root-bound:
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-root-gap s) s  $\leq$ 

```

```

    root-err
  and query-bound:
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-query-gap s) s ≤
    query-err
  and openings-bound:
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-openings-gap s) s ≤
    opening-err
  shows
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
      s ≤ root-err + (query-err + opening-err)
  proof -
  have split-bound:
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
      s ≤
    wp-event verify-monad
      (λout.
        trace-fri-header-tied-low-candidate-untied-root-gap s out ∨
        trace-fri-header-tied-low-candidate-untied-query-gap s out ∨
        trace-fri-header-tied-low-candidate-untied-openings-gap s out)
      s
  by (rule wp-event-mono)
  (rule
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap-imp-cases)
  also have ... ≤
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-root-gap s) s +
    wp-event verify-monad
      (λout.
        trace-fri-header-tied-low-candidate-untied-query-gap s out ∨
        trace-fri-header-tied-low-candidate-untied-openings-gap s out)
      s
  by (rule wp-event-union-bound)
  also have ... ≤
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-root-gap s) s +
    (wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-query-gap s) s +
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-openings-gap s) s)
  by (intro add-mono order-refl wp-event-union-bound)
  also have ... ≤ root-err + (query-err + opening-err)
  by (intro add-mono root-bound query-bound openings-bound)
  finally show ?thesis .
qed

```

lemma *wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-query-and-openings:*

fixes *query-err opening-err :: prob*

assumes *query-bound:*

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-query-gap s) s ≤
query-err

and *openings-bound:*

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-openings-gap s) s ≤
opening-err

shows

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
s ≤ query-err + opening-err

proof –

have *split-bound:*

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
s ≤
wp-event verify-monad
(λout.
trace-fri-header-tied-low-candidate-untied-query-gap s out ∨
trace-fri-header-tied-low-candidate-untied-openings-gap s out)
s

by *(rule wp-event-mono)*

(rule trace-fri-header-tied-low-candidate-untied-gap-imp-query-or-openings)

also have *... ≤*

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-query-gap s) s +
wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-openings-gap s) s

by *(rule wp-event-union-bound)*

also have *... ≤ query-err + opening-err*

by *(rule add-mono[OF query-bound openings-bound])*

finally show *?thesis .*

qed

lemma *wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-openings:*

fixes *opening-err :: prob*

assumes *openings-bound:*

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-openings-gap s) s ≤
opening-err

shows

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
s ≤ opening-err

by *(rule order-trans[OF - openings-bound])*

(rule wp-event-mono,

rule trace-fri-header-tied-low-candidate-untied-gap-imp-openings)

lemma *trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied-cases:*

fixes *header-err merkle-err root-err query-err opening-err :: prob*

assumes *header-bound:*

wp-event verify-monad

(trace-fri-bad-with-header-tied-partial-candidate s) s ≤
header-err

and *merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤
merkle-err

and *root-bound:*

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-root-gap s) s ≤
root-err

and *query-bound:*

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-query-gap s) s ≤
query-err

and *openings-bound:*

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-openings-gap s) s ≤
opening-err

and *total-bound:*

header-err + (merkle-err + (root-err + (query-err + opening-err)))
≤ trace-fri-error

shows *trace-fri-reachable-partial-candidate-reduction-assumption s*

proof —

have *untied-bound:*

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
s ≤ root-err + (query-err + opening-err)

by *(rule*

wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-cases
[OF root-bound query-bound openings-bound])

have *reduction:*

trace-fri-reachable-partial-candidate-reduction-assumption s

by *(rule*

trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied
[OF header-bound merkle-bound untied-bound])
(rule total-bound)

show *?thesis*

by *(rule reduction)*

qed

lemma *trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied-openings:*

fixes *header-err merkle-err opening-err :: prob*

assumes *header-bound:*

wp-event verify-monad

```

    (trace-fri-bad-with-header-tied-partial-candidate s) s ≤
    header-err
and merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤
    merkle-err
and openings-bound:
    wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-openings-gap s) s ≤
    opening-err
and total-bound:
    header-err + (merkle-err + opening-err) ≤ trace-fri-error
shows trace-fri-reachable-partial-candidate-reduction-assumption s
proof –
have untied-bound:
    wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
    s ≤ opening-err
by (rule
    wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-openings
    [OF openings-bound])
have reduction:
    trace-fri-reachable-partial-candidate-reduction-assumption s
by (rule
    trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied
    [OF header-bound merkle-bound untied-bound])
    (rule total-bound)
show ?thesis
by (rule reduction)
qed

```

```

lemma trace-fri-reachable-partial-candidate-reduction-from-header-merkle-hash-and-nonnormalized-openings:
fixes header-err merkle-err hash-err nonnormalized-err :: prob
assumes header-bound:
    wp-event verify-monad
    (trace-fri-bad-with-header-tied-partial-candidate s) s ≤
    header-err
and merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤
    merkle-err
and hash-bound:
    wp-event verify-monad (hash-map-output-collision-bad s) s ≤
    hash-err
and nonnormalized-bound:
    wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap s)
    s ≤ nonnormalized-err
and total-bound:
    header-err + (merkle-err + (hash-err + nonnormalized-err)) ≤
    trace-fri-error

```


shows *trace-fri-reachable-partial-candidate-reduction-assumption s*
proof –
have *openings-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-header-tied-low-candidate-untied-openings-gap } s) \leq \text{hash-err} + \text{nonnormalized-err}$$
by (rule

$$\text{wp-trace-fri-header-tied-low-candidate-untied-openings-gap-bound-from-hash-and-nonnormalized}$$

$$[\text{OF hash-bound nonnormalized-bound}])$$
show *?thesis*
by (rule

$$\text{trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied-openings}$$

$$[\text{OF header-bound merkle-bound openings-bound}])$$

$$(\text{rule total-bound})$$
qed

lemma *trace-fri-reachable-partial-candidate-reduction-from-header-merkle-hash-root-query*:

fixes *header-err merkle-err hash-err root-err query-err :: prob*

assumes *header-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-bad-with-header-tied-partial-candidate } s) \leq \text{header-err}$$

and *merkle-bound*:

$$\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \leq \text{merkle-err}$$

and *hash-bound*:

$$\text{wp-event verify-monad } (\text{hash-map-output-collision-bad } s) \leq \text{hash-err}$$

and *root-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-header-tied-low-candidate-untied-root-gap } s) \leq \text{root-err}$$

and *query-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-header-tied-low-candidate-untied-query-gap } s) \leq \text{query-err}$$

and *total-bound*:

$$\text{header-err} + (\text{merkle-err} + (\text{hash-err} + (\text{root-err} + \text{query-err}))) \leq \text{trace-fri-error}$$

shows *trace-fri-reachable-partial-candidate-reduction-assumption s*

proof –

have *nonnormalized-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap } s) \leq \text{root-err} + \text{query-err}$$

by (rule

$$\text{wp-trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-bound-from-root-query}$$

$$[\text{OF root-bound query-bound}])$$

show *?thesis*

by (rule
 trace-fri-reachable-partial-candidate-reduction-from-header-merkle-hash-and-nonnormalized-openings
 [*OF header-bound merkle-bound hash-bound nonnormalized-bound*])
 (rule *total-bound*)
 qed

lemma *wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-hash-root-query*:
fixes *hash-err root-err query-err :: prob*
assumes *hash-bound*:
 wp-event verify-monad (hash-map-output-collision-bad s) s ≤
 hash-err
and *root-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-low-candidate-untied-root-gap s*) *s ≤*
 root-err
and *query-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-low-candidate-untied-query-gap s*) *s ≤*
 query-err
shows
 wp-event verify-monad
 (*trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s*)
 s ≤ hash-err + (root-err + query-err)
proof –
have *nonnormalized-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap s*)
 s ≤ root-err + query-err
by (rule
 wp-trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-bound-from-root-query
 [*OF root-bound query-bound*])
have *openings-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-low-candidate-untied-openings-gap s*) *s ≤*
 hash-err + (root-err + query-err)
by (rule
 wp-trace-fri-header-tied-low-candidate-untied-openings-gap-bound-from-hash-and-nonnormalized
 [*OF hash-bound nonnormalized-bound*])
show *?thesis*
by (rule
 wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-openings
 [*OF openings-bound*])
 qed

lemma *trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied-query-openings*:
fixes *header-err merkle-err query-err opening-err :: prob*
assumes *header-bound*:
 wp-event verify-monad
 (*trace-fri-bad-with-header-tied-partial-candidate s*) *s ≤*

```

    header-err
  and merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤
    merkle-err
  and query-bound:
    wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-query-gap s) s ≤
    query-err
  and openings-bound:
    wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-openings-gap s) s ≤
    opening-err
  and total-bound:
    header-err + (merkle-err + (query-err + opening-err))
    ≤ trace-fri-error
  shows trace-fri-reachable-partial-candidate-reduction-assumption s
proof -
  have untied-bound:
    wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
    s ≤ query-err + opening-err
  by (rule
    wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-query-and-openings
    [OF query-bound openings-bound])
  have reduction:
    trace-fri-reachable-partial-candidate-reduction-assumption s
  by (rule
    trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied
    [OF header-bound merkle-bound untied-bound])
    (rule total-bound)
  show ?thesis
  by (rule reduction)
qed

end

end

```

```

theory Soundness-FRI-Trace-Untied-Active
  imports
    Soundness-FRI-Active-Reductions
    Soundness-FRI-Trace-Untied-Cases
begin

```

Active-level packaging for the trace FRI untied-witness blocker. This theory is intentionally downstream of the large active-reduction layer so that the new diagnostic event can be used without growing that file further.

```
context soundness
```

begin

lemma *active-trace-fri-reachable-reduction-from-header-merkle-and-untied:*

fixes *Ht Mt Ut ::*

('f staged-proof-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ prob)

assumes *trace-header-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$

wp-event verify-monad
(trace-fri-bad-with-header-tied-partial-candidate
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$

Ht data attacker-state

and *trace-merkle-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$

wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$

Mt data attacker-state

and *trace-untied-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-reachable-witness-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$

Ut data attacker-state

and *total-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$

Ht data attacker-state +
(Mt data attacker-state + Ut data attacker-state) $\leq$
trace-fri-error

and *support*:
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
shows
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
by (*rule*
trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied)
(rule trace-header-bound[OF support],
rule trace-merkle-bound[OF support],
rule trace-untied-bound[OF support],
rule total-bound[OF support])

lemma *active-trace-fri-reachable-reduction-from-header-merkle-and-untied-cases*:

fixes *Ht Mt Rt Qt Ot ::*

('f staged-proof-data ⇒ 'f protocol-channel ⇒ prob)

assumes *trace-header-bound*:

∧ data attacker-state.

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒

wp-event verify-monad
(trace-fri-bad-with-header-tied-partial-candidate
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤

Ht data attacker-state

and *trace-merkle-bound*:

∧ data attacker-state.

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒

wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤

Mt data attacker-state

and *trace-root-bound*:

∧ data attacker-state.

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-root-gap

$$\begin{aligned}
& (\text{verifier-state-from-adversary attacker-state} \\
& \quad (\text{staged-proof-transcript data}))) \\
& (\text{verifier-state-from-adversary attacker-state} \\
& \quad (\text{staged-proof-transcript data})) \leq \\
& R_t \text{ data attacker-state} \\
\text{and trace-query-bound:} \\
& \wedge \text{data attacker-state.} \\
& \text{Some (data, attacker-state)} \in \\
& \quad \text{set-dist (execute (checked-staged-transcript-program A) } \\
& \quad \quad \text{adversary-initial-state)} \implies \\
& \text{wp-event verify-monad} \\
& \quad (\text{trace-fri-header-tied-low-candidate-untied-query-gap} \\
& \quad \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad \quad (\text{staged-proof-transcript data}))) \\
& \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad (\text{staged-proof-transcript data})) \leq \\
& Q_t \text{ data attacker-state} \\
\text{and trace-openings-bound:} \\
& \wedge \text{data attacker-state.} \\
& \text{Some (data, attacker-state)} \in \\
& \quad \text{set-dist (execute (checked-staged-transcript-program A) } \\
& \quad \quad \text{adversary-initial-state)} \implies \\
& \text{wp-event verify-monad} \\
& \quad (\text{trace-fri-header-tied-low-candidate-untied-openings-gap} \\
& \quad \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad \quad (\text{staged-proof-transcript data}))) \\
& \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad (\text{staged-proof-transcript data})) \leq \\
& O_t \text{ data attacker-state} \\
\text{and total-bound:} \\
& \wedge \text{data attacker-state.} \\
& \text{Some (data, attacker-state)} \in \\
& \quad \text{set-dist (execute (checked-staged-transcript-program A) } \\
& \quad \quad \text{adversary-initial-state)} \implies \\
& H_t \text{ data attacker-state} + \\
& \quad (M_t \text{ data attacker-state} + \\
& \quad \quad (R_t \text{ data attacker-state} + \\
& \quad \quad \quad (Q_t \text{ data attacker-state} + O_t \text{ data attacker-state}))) \leq \\
& \text{trace-fri-error} \\
\text{and support:} \\
& \text{Some (data, attacker-state)} \in \\
& \quad \text{set-dist (execute (checked-staged-transcript-program A) } \\
& \quad \quad \text{adversary-initial-state)} \\
\text{shows} \\
& \text{trace-fri-reachable-partial-candidate-reduction-assumption} \\
& \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad (\text{staged-proof-transcript data})) \\
\text{by (rule} \\
& \quad \text{trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied-cases)}
\end{aligned}$$

```

(rule trace-header-bound[OF support],
 rule trace-merkle-bound[OF support],
 rule trace-root-bound[OF support],
 rule trace-query-bound[OF support],
 rule trace-openings-bound[OF support],
 rule total-bound[OF support])

```

lemma *active-trace-fri-reachable-reduction-from-header-merkle-and-untied-query-openings:*

fixes *Ht Mt Qt Ot ::*

(*'f staged-proof-data* $\Rightarrow$ *'f protocol-channel* $\Rightarrow$ *prob*)

assumes *trace-header-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$

wp-event verify-monad

(*trace-fri-bad-with-header-tied-partial-candidate*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))

(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$

Ht data attacker-state

and *trace-merkle-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$

wp-event verify-monad

(*partial-merkle-inconsistency-bad*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))

(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$

Mt data attacker-state

and *trace-query-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$

wp-event verify-monad

(*trace-fri-header-tied-low-candidate-untied-query-gap*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))

(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$

Qt data attacker-state

and *trace-openings-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-low-candidate-untied-openings-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Ot data attacker-state}$
and total-bound:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{Ht data attacker-state} +$
 $(\text{Mt data attacker-state} +$
 $(\text{Qt data attacker-state} + \text{Ot data attacker-state})) \leq$
 trace-fri-error
and support:
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state})$
shows
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
by $(\text{rule}$
 $\text{trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied-query-openings})$
 $(\text{rule trace-header-bound}[OF \text{ support}],$
 $\text{rule trace-merkle-bound}[OF \text{ support}],$
 $\text{rule trace-query-bound}[OF \text{ support}],$
 $\text{rule trace-openings-bound}[OF \text{ support}],$
 $\text{rule total-bound}[OF \text{ support}])$

lemma *active-trace-fri-reachable-reduction-from-header-merkle-and-untied-openings:*

fixes $Ht \ Mt \ Ot ::$

$(\text{'f staged-proof-data} \Rightarrow \text{'f protocol-channel} \Rightarrow \text{prob})$

assumes $\text{trace-header-bound:}$

$\bigwedge \text{data attacker-state.}$

$\text{Some } (data, \text{attacker-state}) \in$

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$

$\text{wp-event verify-monad}$

$(\text{trace-fri-bad-with-header-tied-partial-candidate}$

$(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data})))$

$(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data})) \leq$

$\text{Ht data attacker-state}$

and *trace-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Mt data attacker-state}$
and *trace-openings-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-low-candidate-untied-openings-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Ot data attacker-state}$
and *total-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{Ht data attacker-state} +$
 $(\text{Mt data attacker-state} + \text{Ot data attacker-state}) \leq$
 trace-fri-error
and *support*:
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)}$
shows
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
by (rule
 $\text{trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied-openings})$
 $(\text{rule trace-header-bound[OF support],}$
 $\text{rule trace-merkle-bound[OF support],}$
 $\text{rule trace-openings-bound[OF support],}$
 $\text{rule total-bound[OF support]})$

lemma *active-trace-fri-reachable-reduction-from-header-merkle-hash-root-query*:
fixes *Ht Mt Kt Rt Qt* ::

('f staged-proof-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ prob)
assumes trace-header-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-partial-candidate
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 Ht data attacker-state
and trace-merkle-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 Mt data attacker-state
and trace-hash-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (hash-map-output-collision-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 Kt data attacker-state
and trace-root-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (trace-fri-header-tied-low-candidate-untied-root-gap
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 Rt data attacker-state

```

and trace-query-bound:
 $\wedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-query-gap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq$ 
      Qt data attacker-state
and total-bound:
 $\wedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    Ht data attacker-state +
      (Mt data attacker-state +
        (Kt data attacker-state +
          (Rt data attacker-state + Qt data attacker-state)))  $\leq$ 
      trace-fri-error
and support:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
shows
  trace-fri-reachable-partial-candidate-reduction-assumption
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
by (rule
  trace-fri-reachable-partial-candidate-reduction-from-header-merkle-hash-root-query)
  (rule trace-header-bound[OF support],
    rule trace-merkle-bound[OF support],
    rule trace-hash-bound[OF support],
    rule trace-root-bound[OF support],
    rule trace-query-bound[OF support],
    rule total-bound[OF support])

end

end

theory Soundness-FRI-Replay-Gap-Assignment
imports
  Soundness-FRI-Trace-Untied-Active
begin

Small adapters from the replay-gap residual predicates to the existing sampled-assignment

```

conflict branches. The replay gaps are represented as sampled next, successor, or final conflicts, so they are charged to the same FRI sampled-assignment layer rather than treated as independent probabilistic obligations.

context *soundness*
begin

lemma *trace-fri-header-tied-next-value-replay-gap-imp-without-same:*
assumes *trace-fri-header-tied-next-value-replay-gap s out*
shows
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out
using *assms*
unfolding *trace-fri-header-tied-next-value-replay-gap-def*
by (*blast intro: trace-fri-header-tied-without-sameI-next*)

lemma *trace-fri-header-tied-successor-replay-gap-imp-without-same:*
assumes *trace-fri-header-tied-successor-replay-gap s out*
shows
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out
using *assms*
unfolding *trace-fri-header-tied-successor-replay-gap-def*
by (*blast intro: trace-fri-header-tied-without-sameI-successor*)

lemma *trace-fri-header-tied-final-value-replay-gap-imp-without-same:*
assumes *trace-fri-header-tied-final-value-replay-gap s out*
shows
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out
using *assms*
unfolding *trace-fri-header-tied-final-value-replay-gap-def*
by (*blast intro: trace-fri-header-tied-without-sameI-final*)

lemma *trace-fri-header-tied-without-same-imp-sampled-assignment-conflict:*
assumes
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out
shows *trace-fri-header-tied-sampled-assignment-conflict s out*
using *assms*
unfolding
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer-def
trace-fri-header-tied-sampled-assignment-conflict-def
generic-fri-sampled-assignment-conflict-without-same-layer-def
generic-fri-sampled-assignment-conflict-def
by *blast*

lemma *wp-trace-fri-header-tied-next-value-replay-gap-bound-from-without-same:*
fixes *C :: prob*
assumes *conflict-bound:*

wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
 s) $s \leq C$

shows

wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap *s*) $s \leq C$

by (rule order-trans[*OF* - conflict-bound])
 (rule wp-event-mono,
 rule trace-fri-header-tied-next-value-replay-gap-imp-without-same)

lemma wp-trace-fri-header-tied-successor-replay-gap-bound-from-without-same:

fixes *C* :: prob

assumes conflict-bound:

wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
 s) $s \leq C$

shows

wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap *s*) $s \leq C$

by (rule order-trans[*OF* - conflict-bound])
 (rule wp-event-mono,
 rule trace-fri-header-tied-successor-replay-gap-imp-without-same)

lemma wp-trace-fri-header-tied-final-value-replay-gap-bound-from-without-same:

fixes *C* :: prob

assumes conflict-bound:

wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
 s) $s \leq C$

shows

wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap *s*) $s \leq C$

by (rule order-trans[*OF* - conflict-bound])
 (rule wp-event-mono,
 rule trace-fri-header-tied-final-value-replay-gap-imp-without-same)

lemma wp-trace-fri-header-tied-without-same-bound-from-sampled-assignment-conflict:

fixes *C* :: prob

assumes conflict-bound:

wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict *s*) $s \leq C$

shows

wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
 s) $s \leq C$

by (rule order-trans[*OF* - conflict-bound])
 (rule wp-event-mono,
 rule trace-fri-header-tied-without-same-imp-sampled-assignment-conflict)

lemma *composition-fri-verifier-tied-next-value-replay-gap-imp-without-same:*
assumes *composition-fri-verifier-tied-next-value-replay-gap s out*
shows
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out
using *assms*
unfolding *composition-fri-verifier-tied-next-value-replay-gap-def*
by (*blast intro: composition-fri-verifier-tied-without-sameI-next*)

lemma *composition-fri-verifier-tied-successor-replay-gap-imp-without-same:*
assumes *composition-fri-verifier-tied-successor-replay-gap s out*
shows
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out
using *assms*
unfolding *composition-fri-verifier-tied-successor-replay-gap-def*
by (*blast intro: composition-fri-verifier-tied-without-sameI-successor*)

lemma *composition-fri-verifier-tied-final-value-replay-gap-imp-without-same:*
assumes *composition-fri-verifier-tied-final-value-replay-gap s out*
shows
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out
using *assms*
unfolding *composition-fri-verifier-tied-final-value-replay-gap-def*
by (*blast intro: composition-fri-verifier-tied-without-sameI-final*)

lemma *composition-fri-verifier-tied-without-same-imp-sampled-assignment-conflict:*
assumes
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out
shows *composition-fri-verifier-tied-sampled-assignment-conflict s out*
using *assms*
unfolding
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
composition-fri-verifier-tied-sampled-assignment-conflict-def
generic-fri-sampled-assignment-conflict-without-same-layer-def
generic-fri-sampled-assignment-conflict-def
by *blast*

lemma *wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-without-same:*
fixes *C :: prob*
assumes *conflict-bound:*
wp-event verify-monad
(composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s) s ≤ C
shows
wp-event verify-monad
(composition-fri-verifier-tied-next-value-replay-gap s) s ≤ C

by (rule order-trans[OF - conflict-bound])
 (rule wp-event-mono,
 rule composition-fri-verifier-tied-next-value-replay-gap-imp-without-same)

lemma wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-without-same:

fixes $C :: \text{prob}$

assumes conflict-bound:

wp-event verify-monad

(composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
 s) $s \leq C$

shows

wp-event verify-monad

(composition-fri-verifier-tied-successor-replay-gap s) $s \leq C$

by (rule order-trans[OF - conflict-bound])

(rule wp-event-mono,

rule composition-fri-verifier-tied-successor-replay-gap-imp-without-same)

lemma wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-without-same:

fixes $C :: \text{prob}$

assumes conflict-bound:

wp-event verify-monad

(composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
 s) $s \leq C$

shows

wp-event verify-monad

(composition-fri-verifier-tied-final-value-replay-gap s) $s \leq C$

by (rule order-trans[OF - conflict-bound])

(rule wp-event-mono,

rule composition-fri-verifier-tied-final-value-replay-gap-imp-without-same)

lemma wp-composition-fri-verifier-tied-without-same-bound-from-sampled-assignment-conflict:

fixes $C :: \text{prob}$

assumes conflict-bound:

wp-event verify-monad

(composition-fri-verifier-tied-sampled-assignment-conflict s) $s \leq C$

shows

wp-event verify-monad

(composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
 s) $s \leq C$

by (rule order-trans[OF - conflict-bound])

(rule wp-event-mono,

rule composition-fri-verifier-tied-without-same-imp-sampled-assignment-conflict)

lemma trace-fri-header-tied-residual-obligations-from-without-same-bounds:

fixes $C :: \text{prob}$

assumes sampled:

wp-event verify-monad

(trace-fri-bad-with-header-tied-sampled-layer-chain s) $s \leq r-b$

and zero:

wp-event verify-monad
 (trace-fri-header-tied-zero-round-final-obstruction s) $s \leq z-b$
and merkle:
wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq p-b$
and structural:
wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap s) $s \leq h-b$
and without-same:
wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
 s) $s \leq C$
and total:
 $r-b + (((((p-b + h-b) + p-b) + C + C +$
 $(p-b + 0 + 0)) + p-b) + 0 + z-b)$
 $\leq \text{trace-fri-error}$
shows
trace-fri-header-tied-residual-obligations s
 $r-b \ z-b \ 0 \ p-b \ h-b \ p-b \ C \ C \ 0$
by (rule
trace-fri-header-tied-residual-obligations-from-bounds-without-head-slot-and-selected-zero)
 (rule *sampled*, rule *zero*, rule *merkle*, rule *structural*,
 rule *wp-trace-fri-header-tied-next-value-replay-gap-bound-from-without-same*
 [OF *without-same*],
 rule *wp-trace-fri-header-tied-successor-replay-gap-bound-from-without-same*
 [OF *without-same*],
 rule *total*)

lemma *composition-fri-verifier-tied-residual-obligations-from-without-same-bounds:*

fixes $C :: \text{prob}$

assumes *sampled:*

wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s
 $\leq r-b$

and merkle:

wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq p-b$

and without-same:

wp-event verify-monad
 (composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
 s) $s \leq C$

and total:

$r-b + (((p-b + 0) + C + C + C + p-b) + 0)$
 $\leq \text{composition-fri-error}$

shows

composition-fri-verifier-tied-residual-obligations $s \ r-b \ 0 \ p-b \ C \ C \ C$

by (rule *composition-fri-verifier-tied-residual-obligations-from-bounds-slot-zero*)

(rule *sampled*, rule *merkle*,

rule *wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-without-same*
 [OF *without-same*],

rule *wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-without-same*

$[OF \text{ without-same}]$,
rule wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-without-same
 $[OF \text{ without-same}]$,
rule total)

lemma *active-trace-fri-residual-obligations-from-without-same-bounds:*

fixes

Rt Zt Pt Ht Ct ::

('f staged-proof-data $\Rightarrow$

'f protocol-channel $\Rightarrow$ prob)

assumes *trace-sampled-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\Rightarrow$

wp-event verify-monad

(trace-fri-bad-with-header-tied-sampled-layer-chain

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)) $\leq$

Rt data attacker-state

and *trace-zero-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\Rightarrow$

wp-event verify-monad

(trace-fri-header-tied-zero-round-final-obstruction

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)) $\leq$

Zt data attacker-state

and *trace-merkle-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\Rightarrow$

wp-event verify-monad

(partial-merkle-inconsistency-bad

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)) $\leq$

Pt data attacker-state

and *trace-structural-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$

```

    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-header-tied-sibling-mismatch-structural-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$ 
Ht data attacker-state
and trace-without-same-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$ 
Ct data attacker-state
and trace-total-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
Rt data attacker-state +
  (((((Pt data attacker-state + Ht data attacker-state) +
    Pt data attacker-state) +
    Ct data attacker-state + Ct data attacker-state +
    (Pt data attacker-state + 0 + 0)) +
    Pt data attacker-state) +
    0 + Zt data attacker-state)
 $\leq$  trace-fri-error
shows active-trace-fri-residual-obligations-for A Rt Zt
  ( $\lambda$ - -. (0::prob)) Pt Ht Pt Ct Ct ( $\lambda$ - -. (0::prob))
unfolding active-trace-fri-residual-obligations-for-def
proof (intro allI impI)
fix data attacker-state
assume support:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show trace-fri-header-tied-residual-obligations ?s
  (Rt data attacker-state) (Zt data attacker-state) 0
  (Pt data attacker-state) (Ht data attacker-state)

```

```

(Pt data attacker-state) (Ct data attacker-state)
(Ct data attacker-state) 0
by (rule trace-fri-header-tied-residual-obligations-from-without-same-bounds)
(rule trace-sampled-bound[OF support],
rule trace-zero-bound[OF support],
rule trace-merkle-bound[OF support],
rule trace-structural-bound[OF support],
rule trace-without-same-bound[OF support],
rule trace-total-bound[OF support])
qed

lemma active-composition-fri-residual-obligations-from-without-same-bounds:
fixes
  Rc Pc Cc ::
    ('f staged-proof-data  $\Rightarrow$ 
     'f protocol-channel  $\Rightarrow$  prob)
assumes comp-sampled-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\Longrightarrow$ 
wp-event verify-monad
  (composition-fri-bad-with-verifier-tied-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$ 
  Rc data attacker-state
and comp-merkle-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\Longrightarrow$ 
wp-event verify-monad
  (partial-merkle-inconsistency-bad
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$ 
  Pc data attacker-state
and comp-without-same-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\Longrightarrow$ 
wp-event verify-monad
  (composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))

```

```

    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
  Cc data attacker-state
and comp-total-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
  Rc data attacker-state +
    (((Pc data attacker-state + 0) +
      Cc data attacker-state + Cc data attacker-state +
      Cc data attacker-state + Pc data attacker-state) +
      0)
    ≤ composition-fri-error
shows active-composition-fri-residual-obligations-for A Rc
  (λ- -. (0::prob)) Pc Cc Cc Cc
unfolding active-composition-fri-residual-obligations-for-def
proof (intro allI impI)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show composition-fri-verifier-tied-residual-obligations ?s
  (Rc data attacker-state) 0 (Pc data attacker-state)
  (Cc data attacker-state) (Cc data attacker-state)
  (Cc data attacker-state)
by (rule composition-fri-verifier-tied-residual-obligations-from-without-same-bounds)
  (rule comp-sampled-bound[OF support],
    rule comp-merkle-bound[OF support],
    rule comp-without-same-bound[OF support],
    rule comp-total-bound[OF support])
qed

```

lemma trace-fri-reachable-partial-candidate-reduction-from-header-without-same-and-untied:

fixes R Z P H C Q U :: prob

assumes sampled-bound:

wp-event verify-monad

(trace-fri-bad-with-header-tied-sampled-layer-chain s) s ≤ R

and zero-bound:

wp-event verify-monad

(trace-fri-header-tied-zero-round-final-obstruction s) s ≤ Z

and header-partial-merkle-bound:

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ P

and structural-gap-bound:

wp-event verify-monad

(*trace-fri-header-tied-sibling-mismatch-structural-gap* s) $s \leq H$
and *without-same-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-sampled-assignment-conflict-without-same-layer*
 s) $s \leq C$
and *gap-partial-merkle-bound*:
 wp-event verify-monad (*partial-merkle-inconsistency-bad* s) $s \leq Q$
and *untied-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-low-candidate-untied-reachable-witness-gap* s)
 $s \leq U$
and *total-bound*:
 $R + (((((P + H) + P) + C + C + C) + P) + 0 + Z) +$
 $(Q + U) \leq \text{trace-fri-error}$
shows *trace-fri-reachable-partial-candidate-reduction-assumption* s
proof –
have *head-neq*:
 wp-event verify-monad
 (*trace-fri-header-tied-base-head-value-neq-gap* s) $s \leq P$
by (*rule*
 wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle
 [*OF header-partial-merkle-bound*])
have *head-mismatch*:
 wp-event verify-monad
 (*trace-fri-header-tied-base-head-value-mismatch-gap* s) $s \leq P$
by (*rule*
 wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
 [*OF head-neq*])
have *alignment*:
 wp-event verify-monad
 (*trace-fri-header-tied-base-value-alignment-gap* s) $s \leq P$
by (*rule*
 wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
 [*OF head-mismatch*])
have *next-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-next-value-replay-gap* s) $s \leq C$
by (*rule*
 wp-trace-fri-header-tied-next-value-replay-gap-bound-from-without-same
 [*OF without-same-bound*])
have *successor-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-successor-replay-gap* s) $s \leq C$
by (*rule*
 wp-trace-fri-header-tied-successor-replay-gap-bound-from-without-same
 [*OF without-same-bound*])
have *final-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-final-value-replay-gap* s) $s \leq C$

by (rule
 wp-trace-fri-header-tied-final-value-replay-gap-bound-from-without-same
 [*OF without-same-bound*])
 have header-bound:
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-partial-candidate s) $s \leq$
 $R + (((((P + H) + P) + C + C + C) + P) + 0 + Z)$
 by (rule
 wp-trace-fri-header-tied-bound-from-sampled-zero-slot-recorded-chunk-gap-and-residuals)
 (rule sampled-bound, rule zero-bound,
 rule *wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero*,
 rule *header-partial-merkle-bound*, rule *structural-gap-bound*,
 rule *alignment*, rule *next-bound*, rule *successor-bound*,
 rule *final-bound*)
 show ?thesis
 by (rule
 trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied)
 (rule header-bound, rule *gap-partial-merkle-bound*,
 rule *untied-bound*, rule *total-bound*)
 qed

lemma *trace-fri-reachable-partial-candidate-reduction-from-header-without-same-hash-root-query*:
 fixes $R\ Z\ P\ H\ C\ Q\ K\ Rt\ Qt :: prob$
 assumes sampled-bound:
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-sampled-layer-chain s) $s \leq R$
 and zero-bound:
 wp-event verify-monad
 (trace-fri-header-tied-zero-round-final-obstruction s) $s \leq Z$
 and header-partial-merkle-bound:
 wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$
 and structural-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap s) $s \leq H$
 and without-same-bound:
 wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
 s) $s \leq C$
 and gap-partial-merkle-bound:
 wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq Q$
 and hash-bound:
 wp-event verify-monad (hash-map-output-collision-bad s) $s \leq K$
 and root-bound:
 wp-event verify-monad
 (trace-fri-header-tied-low-candidate-untied-root-gap s) $s \leq Rt$
 and query-bound:
 wp-event verify-monad
 (trace-fri-header-tied-low-candidate-untied-query-gap s) $s \leq Qt$
 and total-bound:

$$R + (((((P + H) + P) + C + C + C) + P) + 0 + Z) + (Q + (K + (Rt + Qt))) \leq \text{trace-fri-error}$$
shows *trace-fri-reachable-partial-candidate-reduction-assumption* s
proof –
 have *head-neq*:

 wp-event verify-monad

 $(\text{trace-fri-header-tied-base-head-value-neq-gap } s) \ s \leq P$

 by (*rule*

 wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle

 [*OF header-partial-merkle-bound*])

have *head-mismatch*:

 wp-event verify-monad

 $(\text{trace-fri-header-tied-base-head-value-mismatch-gap } s) \ s \leq P$

 by (*rule*

 wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap

 [*OF head-neq*])

have *alignment*:

 wp-event verify-monad

 $(\text{trace-fri-header-tied-base-value-alignment-gap } s) \ s \leq P$

 by (*rule*

 wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap

 [*OF head-mismatch*])

have *next-bound*:

 wp-event verify-monad

 $(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq C$

 by (*rule*

 wp-trace-fri-header-tied-next-value-replay-gap-bound-from-without-same

 [*OF without-same-bound*])

have *successor-bound*:

 wp-event verify-monad

 $(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq C$

 by (*rule*

 wp-trace-fri-header-tied-successor-replay-gap-bound-from-without-same

 [*OF without-same-bound*])

have *final-bound*:

 wp-event verify-monad

 $(\text{trace-fri-header-tied-final-value-replay-gap } s) \ s \leq C$

 by (*rule*

 wp-trace-fri-header-tied-final-value-replay-gap-bound-from-without-same

 [*OF without-same-bound*])

have *header-bound*:

 wp-event verify-monad

 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s \leq$

 $R + (((((P + H) + P) + C + C + C) + P) + 0 + Z)$

 by (*rule*

 wp-trace-fri-header-tied-bound-from-sampled-zero-slot-recorded-chunk-gap-and-residuals)

 (*rule sampled-bound*, *rule zero-bound*,

 rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero,

 rule header-partial-merkle-bound, *rule structural-gap-bound*,

```

    rule alignment, rule next-bound, rule successor-bound,
    rule final-bound)
show ?thesis
by (rule
    trace-fri-reachable-partial-candidate-reduction-from-header-merkle-hash-root-query)
    (rule header-bound, rule gap-partial-merkle-bound, rule hash-bound,
    rule root-bound, rule query-bound, rule total-bound)
qed

lemma active-trace-fri-reachable-reduction-from-header-without-same-and-untied:
fixes Rt Zt Pt Ht Ct Qt Ut ::
    ('f staged-proof-data  $\Rightarrow$  'f protocol-channel  $\Rightarrow$  prob)
assumes trace-sampled-bound:
 $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
        set-dist (execute (checked-staged-transcript-program A)
            adversary-initial-state)  $\Rightarrow$ 
wp-event verify-monad
    (trace-fri-bad-with-header-tied-sampled-layer-chain
        (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq$ 
    Rt data attacker-state
and trace-zero-bound:
 $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
        set-dist (execute (checked-staged-transcript-program A)
            adversary-initial-state)  $\Rightarrow$ 
wp-event verify-monad
    (trace-fri-header-tied-zero-round-final-obstruction
        (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq$ 
    Zt data attacker-state
and trace-header-merkle-bound:
 $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
        set-dist (execute (checked-staged-transcript-program A)
            adversary-initial-state)  $\Rightarrow$ 
wp-event verify-monad
    (partial-merkle-inconsistency-bad
        (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq$ 
    Pt data attacker-state
and trace-structural-bound:

```


$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 Ht data attacker-state
and trace-without-same-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 Ct data attacker-state
and trace-gap-merkle-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 Qt data attacker-state
and trace-untied-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 Ut data attacker-state
and trace-total-bound:
 $\wedge$ data attacker-state.

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
Rt data attacker-state +
(((((Pt data attacker-state + Ht data attacker-state) +
Pt data attacker-state) +
Ct data attacker-state + Ct data attacker-state +
Ct data attacker-state) +
Pt data attacker-state) +
0 + Zt data attacker-state) +
(Qt data attacker-state + Ut data attacker-state)
≤ trace-fri-error
and *support:*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
shows
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
by (*rule*
trace-fri-reachable-partial-candidate-reduction-from-header-without-same-and-untied)
(rule trace-sampled-bound[OF support],
rule trace-zero-bound[OF support],
rule trace-header-merkle-bound[OF support],
rule trace-structural-bound[OF support],
rule trace-without-same-bound[OF support],
rule trace-gap-merkle-bound[OF support],
rule trace-untied-bound[OF support],
rule trace-total-bound[OF support])

lemma *active-trace-fri-reachable-reduction-from-header-without-same-hash-root-query:*
fixes *Rt Zt Pt Ht Ct Qt Kt Ut Vt ::*
('f staged-proof-data ⇒ 'f protocol-channel ⇒ prob)
assumes *trace-sampled-bound:*
 $\bigwedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-bad-with-header-tied-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Rt data attacker-state
and *trace-zero-bound:*
 $\bigwedge$ *data attacker-state.*
Some (data, attacker-state) ∈

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Zt data attacker-state}$
and $\text{trace-header-merkle-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Pt data attacker-state}$
and $\text{trace-structural-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Ht data attacker-state}$
and $\text{trace-without-same-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict-without-same-layer}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Ct data attacker-state}$
and $\text{trace-gap-merkle-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$

adversary-initial-state) $\implies$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Qt data attacker-state
and *trace-hash-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(hash-map-output-collision-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Kt data attacker-state
and *trace-root-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-root-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Ut data attacker-state
and *trace-query-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-query-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Vt data attacker-state
and *trace-total-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$

$Rt \text{ data attacker-state} +$
 $(((((Pt \text{ data attacker-state} + Ht \text{ data attacker-state}) +$
 $Pt \text{ data attacker-state}) +$
 $Ct \text{ data attacker-state} + Ct \text{ data attacker-state} +$
 $Ct \text{ data attacker-state}) +$
 $Pt \text{ data attacker-state}) +$
 $0 + Zt \text{ data attacker-state}) +$
 $(Qt \text{ data attacker-state} +$
 $(Kt \text{ data attacker-state} +$
 $(Ut \text{ data attacker-state} + Vt \text{ data attacker-state})))$
 $\leq \text{trace-fri-error}$
and *support*:
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$
shows
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
by (*rule*
 $\text{trace-fri-reachable-partial-candidate-reduction-from-header-without-same-hash-root-query})$
 $(\text{rule trace-sampled-bound}[OF \text{ support}],$
 $\text{rule trace-zero-bound}[OF \text{ support}],$
 $\text{rule trace-header-merkle-bound}[OF \text{ support}],$
 $\text{rule trace-structural-bound}[OF \text{ support}],$
 $\text{rule trace-without-same-bound}[OF \text{ support}],$
 $\text{rule trace-gap-merkle-bound}[OF \text{ support}],$
 $\text{rule trace-hash-bound}[OF \text{ support}],$
 $\text{rule trace-root-bound}[OF \text{ support}],$
 $\text{rule trace-query-bound}[OF \text{ support}],$
 $\text{rule trace-total-bound}[OF \text{ support}])$

lemma *composition-fri-reachable-partial-candidate-reduction-from-split-without-same-bounds*:
fixes $R \ P \ C \ E \ Q \ T :: \text{prob}$
assumes *sampled-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain } s) \ s \leq R$
and *verifier-partial-merkle-bound*:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq P$
and *without-same-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer}$
 $s) \ s \leq C$
and *empty-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-reachable-verifier-tie-empty-header-gap } s) \ s \leq E$
and *gap-partial-merkle-bound*:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq Q$
and *candidate-available-bound*:

wp-event verify-monad
 (*composition-fri-verifier-tied-candidate-available-gap s*) $s \leq T$
and *total-bound*:
 $R + (((P + 0) + C + C + C + P) + 0) + (E + (Q + T))$
 $\leq \text{composition-fri-error}$
shows *composition-fri-reachable-partial-candidate-reduction-assumption s*
by (*rule*
composition-fri-reachable-partial-candidate-reduction-from-split-residual-bounds-with-candidate-available-and-recon
(rule sampled-bound,
rule wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero,
rule verifier-partial-merkle-bound,
rule wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-without-same
[OF without-same-bound],
rule wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-without-same
[OF without-same-bound],
rule wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-without-same
[OF without-same-bound],
rule empty-bound,
rule gap-partial-merkle-bound,
rule candidate-available-bound,
rule total-bound)

lemma *composition-fri-reachable-partial-candidate-reduction-from-split-sampled-assignment-bounds*:
fixes $R\ P\ C\ E\ Q\ T :: \text{prob}$
assumes *sampled-bound*:
wp-event verify-monad
 (*composition-fri-bad-with-verifier-tied-sampled-layer-chain s*) $s \leq R$
and *verifier-partial-merkle-bound*:
wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$
and *sampled-assignment-bound*:
wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-assignment-conflict s*) $s \leq C$
and *empty-bound*:
wp-event verify-monad
 (*composition-fri-reachable-verifier-tie-empty-header-gap s*) $s \leq E$
and *gap-partial-merkle-bound*:
wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq Q$
and *candidate-available-bound*:
wp-event verify-monad
 (*composition-fri-verifier-tied-candidate-available-gap s*) $s \leq T$
and *total-bound*:
 $R + (((P + 0) + C + C + C + P) + 0) + (E + (Q + T))$
 $\leq \text{composition-fri-error}$
shows *composition-fri-reachable-partial-candidate-reduction-assumption s*
by (*rule*
composition-fri-reachable-partial-candidate-reduction-from-split-without-same-bounds)
(rule sampled-bound,
rule verifier-partial-merkle-bound,

rule wp-composition-fri-verifier-tied-without-same-bound-from-sampled-assignment-conflict
 [OF *sampled-assignment-bound*],
rule empty-bound,
rule gap-partial-merkle-bound,
rule candidate-available-bound,
rule total-bound)

lemma *active-composition-fri-reachable-reduction-from-split-without-same-bounds*:

fixes *Rc Pc Cc Ec Qc Tc* ::

(*'f staged-proof-data* $\Rightarrow$ *'f protocol-channel* $\Rightarrow$ *prob*)

assumes *comp-sampled-bound*:

$\wedge$ *data attacker-state*.

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
wp-event verify-monad
(composition-fri-bad-with-verifier-tied-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Rc data attacker-state

and *comp-merkle-bound*:

$\wedge$ *data attacker-state*.

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Pc data attacker-state

and *comp-without-same-bound*:

$\wedge$ *data attacker-state*.

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
wp-event verify-monad
(composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Cc data attacker-state

and *comp-empty-bound*:

$\wedge$ *data attacker-state*.

Some (data, attacker-state) $\in$

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-reachable-verifier-tie-empty-header-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Ec data attacker-state}$
and comp-gap-merkle-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Qc data attacker-state}$
and comp-candidate-available-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-candidate-available-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Tc data attacker-state}$
and comp-total-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{Rc data attacker-state} +$
 $((\text{Pc data attacker-state} + 0) +$
 $\text{Cc data attacker-state} + \text{Cc data attacker-state} +$
 $\text{Cc data attacker-state} + \text{Pc data attacker-state}) +$
 $0) +$
 $(\text{Ec data attacker-state} +$
 $(\text{Qc data attacker-state} + \text{Tc data attacker-state}))$
 $\leq \text{composition-fri-error}$
and support:
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$

adversary-initial-state)
shows
composition-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
by (*rule*
composition-fri-reachable-partial-candidate-reduction-from-split-without-same-bounds)
(rule comp-sampled-bound[OF support],
rule comp-merkle-bound[OF support],
rule comp-without-same-bound[OF support],
rule comp-empty-bound[OF support],
rule comp-gap-merkle-bound[OF support],
rule comp-candidate-available-bound[OF support],
rule comp-total-bound[OF support])

lemma *active-fri-reductions-from-without-same-and-untied-bounds:*

fixes

Rt Zt Pt Ht Ct Qt Ut Rc Pc Cc Ec Qc Tc ::
('f staged-proof-data ⇒ 'f protocol-channel ⇒ prob)

assumes *trace-sampled-bound:*

$\bigwedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-bad-with-header-tied-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Rt data attacker-state

and *trace-zero-bound:*

$\bigwedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-zero-round-final-obstruction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Zt data attacker-state

and *trace-header-merkle-bound:*

$\bigwedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad

(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Pt data attacker-state
and *trace-structural-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-sibling-mismatch-structural-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Ht data attacker-state
and *trace-without-same-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Ct data attacker-state
and *trace-gap-merkle-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Qt data attacker-state
and *trace-untied-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-reachable-witness-gap

$$\begin{aligned}
& (\text{verifier-state-from-adversary attacker-state} \\
& \quad (\text{staged-proof-transcript data}))) \\
& (\text{verifier-state-from-adversary attacker-state} \\
& \quad (\text{staged-proof-transcript data})) \leq \\
& \text{Ut data attacker-state} \\
\text{and trace-total-bound:} \\
\wedge \text{data attacker-state.} \\
& \text{Some (data, attacker-state)} \in \\
& \quad \text{set-dist (execute (checked-staged-transcript-program A) } \\
& \quad \quad \text{adversary-initial-state)} \implies \\
& \text{Rt data attacker-state} + \\
& \quad (((((\text{Pt data attacker-state} + \text{Ht data attacker-state}) + \\
& \quad \quad \text{Pt data attacker-state}) + \\
& \quad \quad \text{Ct data attacker-state} + \text{Ct data attacker-state} + \\
& \quad \quad \text{Ct data attacker-state}) + \\
& \quad \quad \text{Pt data attacker-state}) + \\
& \quad \quad 0 + \text{Zt data attacker-state}) + \\
& \quad (\text{Qt data attacker-state} + \text{Ut data attacker-state}) \\
& \leq \text{trace-fri-error} \\
\text{and comp-sampled-bound:} \\
\wedge \text{data attacker-state.} \\
& \text{Some (data, attacker-state)} \in \\
& \quad \text{set-dist (execute (checked-staged-transcript-program A) } \\
& \quad \quad \text{adversary-initial-state)} \implies \\
& \text{wp-event verify-monad} \\
& \quad (\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain} \\
& \quad \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad \quad (\text{staged-proof-transcript data}))) \\
& \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad (\text{staged-proof-transcript data})) \leq \\
& \text{Rc data attacker-state} \\
\text{and comp-merkle-bound:} \\
\wedge \text{data attacker-state.} \\
& \text{Some (data, attacker-state)} \in \\
& \quad \text{set-dist (execute (checked-staged-transcript-program A) } \\
& \quad \quad \text{adversary-initial-state)} \implies \\
& \text{wp-event verify-monad} \\
& \quad (\text{partial-merkle-inconsistency-bad} \\
& \quad \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad \quad (\text{staged-proof-transcript data}))) \\
& \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad (\text{staged-proof-transcript data})) \leq \\
& \text{Pc data attacker-state} \\
\text{and comp-without-same-bound:} \\
\wedge \text{data attacker-state.} \\
& \text{Some (data, attacker-state)} \in \\
& \quad \text{set-dist (execute (checked-staged-transcript-program A) } \\
& \quad \quad \text{adversary-initial-state)} \implies \\
& \text{wp-event verify-monad}
\end{aligned}$$

$(\text{composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Cc \text{ data attacker-state}$
and *comp-empty-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-reachable-verifier-tie-empty-header-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ec \text{ data attacker-state}$
and *comp-gap-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Qc \text{ data attacker-state}$
and *comp-candidate-available-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-candidate-available-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Tc \text{ data attacker-state}$
and *comp-total-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $Rc \text{ data attacker-state} +$
 $((Pc \text{ data attacker-state} + 0) +$

```

      Cc data attacker-state + Cc data attacker-state +
      Cc data attacker-state + Pc data attacker-state) +
      0) +
      (Ec data attacker-state +
      (Qc data attacker-state + Tc data attacker-state))
    ≤ composition-fri-error
  shows active-fri-reductions-for A
proof (rule active-fri-reductions-from-reachable)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
have total:
  Rt data attacker-state +
  (Pt data attacker-state + Ht data attacker-state +
  Pt data attacker-state +
  Ct data attacker-state +
  Ct data attacker-state +
  Ct data attacker-state +
  Pt data attacker-state +
  0 +
  Zt data attacker-state) +
  (Qt data attacker-state + Ut data attacker-state)
  ≤ trace-fri-error
  using trace-total-bound[OF support] by (simp add: add.assoc)
show trace-fri-reachable-partial-candidate-reduction-assumption
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  using trace-sampled-bound[OF support]
    trace-zero-bound[OF support]
    trace-header-merkle-bound[OF support]
    trace-structural-bound[OF support]
    trace-without-same-bound[OF support]
    trace-gap-merkle-bound[OF support]
    trace-untied-bound[OF support]
    total
  by (rule
    trace-fri-reachable-partial-candidate-reduction-from-header-without-same-and-untied
    [where R = Rt data attacker-state
      and Z = Zt data attacker-state
      and P = Pt data attacker-state
      and H = Ht data attacker-state
      and C = Ct data attacker-state
      and Q = Qt data attacker-state
      and U = Ut data attacker-state])
next
fix data attacker-state
assume support:

```

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
have *total*:
Rc data attacker-state +
(Pc data attacker-state + 0 + Cc data attacker-state +
Cc data attacker-state +
Cc data attacker-state +
Pc data attacker-state +
0) +
(Ec data attacker-state +
(Qc data attacker-state + Tc data attacker-state))
≤ composition-fri-error
using *comp-total-bound[OF support]* **by** (*simp add: add.assoc*)
show *composition-fri-reachable-partial-candidate-reduction-assumption*
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
using *comp-sampled-bound[OF support]*
comp-merkle-bound[OF support]
comp-without-same-bound[OF support]
comp-empty-bound[OF support]
comp-gap-merkle-bound[OF support]
comp-candidate-available-bound[OF support]
total
by (*rule*
composition-fri-reachable-partial-candidate-reduction-from-split-without-same-bounds
[where *R = Rc data attacker-state*
and *P = Pc data attacker-state*
and *C = Cc data attacker-state*
and *E = Ec data attacker-state*
and *Q = Qc data attacker-state*
and *T = Tc data attacker-state]*)
qed

lemma *active-fri-reductions-from-without-same-hash-root-query-bounds*:
fixes
Rt Zt Pt Ht Ct Qt Kt Ut Vt Rc Pc Cc Ec Qc Tc ::
('f staged-proof-data ⇒ 'f protocol-channel ⇒ prob)
assumes *trace-sampled-bound*:
 $\bigwedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-bad-with-header-tied-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤

Rt data attacker-state
and *trace-zero-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-zero-round-final-obstruction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Zt data attacker-state
and *trace-header-merkle-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Pt data attacker-state
and *trace-structural-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-sibling-mismatch-structural-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Ht data attacker-state
and *trace-without-same-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Ct data attacker-state

and *trace-gap-merkle-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

wp-event verify-monad
(*partial-merkle-inconsistency-bad*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$

Qt data attacker-state

and *trace-hash-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

wp-event verify-monad
(*hash-map-output-collision-bad*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$

Kt data attacker-state

and *trace-root-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

wp-event verify-monad
(*trace-fri-header-tied-low-candidate-untied-root-gap*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$

Ut data attacker-state

and *trace-query-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

wp-event verify-monad
(*trace-fri-header-tied-low-candidate-untied-query-gap*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$

Vt data attacker-state

and *trace-total-bound*:

$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 Rt data attacker-state +
 (((((Pt data attacker-state + Ht data attacker-state) +
 Pt data attacker-state) +
 Ct data attacker-state + Ct data attacker-state +
 Ct data attacker-state) +
 Pt data attacker-state) +
 0 + Zt data attacker-state) +
 (Qt data attacker-state +
 (Kt data attacker-state +
 (Ut data attacker-state + Vt data attacker-state))))
 $\leq$ trace-fri-error
and comp-sampled-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-sampled-layer-chain
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 Rc data attacker-state
and comp-merkle-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 Pc data attacker-state
and comp-without-same-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state

$(\text{staged-proof-transcript data}) \leq$
 $Cc \text{ data attacker-state}$
and *comp-empty-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-reachable-verifier-tie-empty-header-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ec \text{ data attacker-state}$
and *comp-gap-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Qc \text{ data attacker-state}$
and *comp-candidate-available-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-candidate-available-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Tc \text{ data attacker-state}$
and *comp-total-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $Rc \text{ data attacker-state} +$
 $((Pc \text{ data attacker-state} + 0) +$
 $Cc \text{ data attacker-state} + Cc \text{ data attacker-state} +$
 $Cc \text{ data attacker-state} + Pc \text{ data attacker-state}) +$
 $0) +$
 $(Ec \text{ data attacker-state} +$

```

    (Qc data attacker-state + Tc data attacker-state))
  ≤ composition-fri-error
shows active-fri-reductions-for A
apply (rule active-fri-reductions-from-without-same-and-untied-bounds
[where Rt = Rt and Zt = Zt and Pt = Pt and Ht = Ht and Ct = Ct
and Qt = Qt
and Ut =
λdata attacker-state.
Kt data attacker-state +
(Ut data attacker-state + Vt data attacker-state)
and Rc = Rc and Pc = Pc and Cc = Cc and Ec = Ec and Qc = Qc
and Tc = Tc])
  apply (rule trace-sampled-bound, assumption)
  apply (rule trace-zero-bound, assumption)
  apply (rule trace-header-merkle-bound, assumption)
  apply (rule trace-structural-bound, assumption)
  apply (rule trace-without-same-bound, assumption)
  apply (rule trace-gap-merkle-bound, assumption)
  apply (rule
    wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-hash-root-query)
  apply (rule trace-hash-bound, assumption)
  apply (rule trace-root-bound, assumption)
  apply (rule trace-query-bound, assumption)
  apply (rule trace-total-bound, assumption)
  apply (rule comp-sampled-bound, assumption)
  apply (rule comp-merkle-bound, assumption)
  apply (rule comp-without-same-bound, assumption)
  apply (rule comp-empty-bound, assumption)
  apply (rule comp-gap-merkle-bound, assumption)
  apply (rule comp-candidate-available-bound, assumption)
  apply (rule comp-total-bound, assumption)
done

end

end

```

```

theory Soundness-FRI-Composition-Replay-Gaps
imports
  Soundness-FRI-Composition-Support
  Soundness-FRI-Generic-Low-Degree
begin

```

Narrow downstream layer for the remaining composition verifier-tied replay gaps. This avoids growing the composition residuals theory, which is already above the advisory size threshold.

```

context soundness
begin

```

```

lemma fri-same-mod-half-index-cases:
  fixes len i j :: nat
  assumes even-len:  $2 \text{ dvd } \text{len}$ 
    and i-bound:  $i < \text{len}$ 
    and j-bound:  $j < \text{len}$ 
    and same-mod:  $i \bmod (\text{len} \text{ div } 2) = j \bmod (\text{len} \text{ div } 2)$ 
  shows
     $i = j \vee$ 
     $i = \text{fri-sibling-index len } j \vee$ 
     $\text{fri-sibling-index len } i = j \vee$ 
     $\text{fri-sibling-index len } i = \text{fri-sibling-index len } j$ 
proof -
  let ?half =  $\text{len} \text{ div } 2$ 
  have len-eq:  $\text{len} = 2 * ?\text{half}$ 
    using even-len by simp
  have half-pos:  $0 < ?\text{half}$ 
    using i-bound len-eq by linarith
  consider
    (ff)  $i < ?\text{half} \wedge j < ?\text{half}$ 
  | (fs)  $i < ?\text{half} \wedge j < ?\text{half}$ 
  | (sf)  $\neg i < ?\text{half} \wedge j < ?\text{half}$ 
  | (ss)  $\neg i < ?\text{half} \wedge j < ?\text{half}$ 
    by blast
  then show ?thesis
proof cases
  case ff
    then have  $i = j$ 
      using same-mod half-pos by simp
    then show ?thesis by simp
next
  case fs
    have j-eq:  $j = \text{fri-sibling-index len } (j \bmod ?\text{half})$ 
      by (rule fri-sibling-index-mod-half-second-half
        [OF even-len j-bound fs(2)])
    have i-eq:  $j \bmod ?\text{half} = i$ 
      using same-mod fs(1) by simp
    have  $\text{fri-sibling-index len } i = j$ 
      using j-eq i-eq by simp
    then show ?thesis by simp
next
  case sf
    have i-eq:  $i = \text{fri-sibling-index len } (i \bmod ?\text{half})$ 
      by (rule fri-sibling-index-mod-half-second-half
        [OF even-len i-bound sf(1)])
    have j-eq:  $i \bmod ?\text{half} = j$ 
      using same-mod sf(2) by simp
    have  $i = \text{fri-sibling-index len } j$ 
      using i-eq j-eq by simp

```

```

    then show ?thesis by simp
next
case ss
have i-eq:  $i = \text{fri-sibling-index len } (i \bmod ?half)$ 
  by (rule fri-sibling-index-mod-half-second-half
      [OF even-len i-bound ss(1)])
have j-eq:  $j = \text{fri-sibling-index len } (j \bmod ?half)$ 
  by (rule fri-sibling-index-mod-half-second-half
      [OF even-len j-bound ss(2)])
have i = j
  using i-eq j-eq same-mod by simp
then show ?thesis by simp
qed
qed

lemma fri-same-mod-half-index-cases3:
  fixes len i j :: nat
  assumes even-len: 2 dvd len
    and i-bound:  $i < \text{len}$ 
    and j-bound:  $j < \text{len}$ 
    and same-mod:  $i \bmod (\text{len div } 2) = j \bmod (\text{len div } 2)$ 
  shows
     $i = j \vee$ 
     $i = \text{fri-sibling-index len } j \vee$ 
     $\text{fri-sibling-index len } i = j$ 
proof -
  let ?half = len div 2
  have len-eq:  $\text{len} = 2 * ?half$ 
    using even-len by simp
  have half-pos:  $0 < ?half$ 
    using i-bound len-eq by linarith
  consider
    (ff)  $i < ?half \wedge j < ?half$ 
  | (fs)  $i < ?half \wedge j < ?half$ 
  | (sf)  $\neg i < ?half \wedge j < ?half$ 
  | (ss)  $\neg i < ?half \wedge j < ?half$ 
  by blast
  then show ?thesis
proof cases
case ff
  then have i = j
    using same-mod half-pos by simp
  then show ?thesis by simp
next
case fs
  have j-eq:  $j = \text{fri-sibling-index len } (j \bmod ?half)$ 
    by (rule fri-sibling-index-mod-half-second-half
        [OF even-len j-bound fs(2)])
  have i-eq:  $j \bmod ?half = i$ 

```

```

    using same-mod fs(1) by simp
  have fri-sibling-index len i = j
    using j-eq i-eq by simp
  then show ?thesis by simp
next
case sf
have i-eq: i = fri-sibling-index len (i mod ?half)
  by (rule fri-sibling-index-mod-half-second-half
    [OF even-len i-bound sf(1)])
have j-eq: i mod ?half = j
  using same-mod sf(2) by simp
have i = fri-sibling-index len j
  using i-eq j-eq by simp
then show ?thesis by simp
next
case ss
have i-eq: i = fri-sibling-index len (i mod ?half)
  by (rule fri-sibling-index-mod-half-second-half
    [OF even-len i-bound ss(1)])
have j-eq: j = fri-sibling-index len (j mod ?half)
  by (rule fri-sibling-index-mod-half-second-half
    [OF even-len j-bound ss(2)])
have i = j
  using i-eq j-eq same-mod by simp
then show ?thesis by simp
qed
qed

lemma fri-sibling-index-involution:
  fixes len idx :: nat
  assumes even-len: 2 dvd len
    and idx-bound: idx < len
  shows fri-sibling-index len (fri-sibling-index len idx) = idx
proof -
  let ?half = len div 2
  have len-eq: len = 2 * ?half
    using even-len by simp
  have half-pos: 0 < ?half
    using idx-bound len-eq by linarith
  show ?thesis
proof (cases idx < ?half)
case True
  then show ?thesis
    by (rule fri-sibling-index-involution-first-half[OF even-len])
next
case False
  define k where k = idx - ?half
  have idx-eq: idx = ?half + k
    using False unfolding k-def by simp

```

```

have k-bound:  $k < ?half$ 
  using idx-bound len-eq idx-eq by linarith
have sib-eq:  $fri\_sibling\_index\ len\ idx = k$ 
proof -
  have sum-eq:  $(?half + k + ?half) = k + len$ 
    using len-eq by linarith
  have k-len:  $k < len$ 
    using k-bound len-eq by linarith
  have  $(?half + k + ?half) \bmod len = (k + len) \bmod len$ 
    by (simp only: sum-eq)
  also have  $\dots = k$ 
    using k-len by simp
  finally have mod-eq:  $(?half + k + ?half) \bmod len = k$  .
  then show ?thesis
    unfolding fri-sibling-index-def idx-eq
    using mod-eq by simp
qed
have fri-sibling-index len (fri-sibling-index len idx) =
  fri-sibling-index len k
  using sib-eq by simp
also have  $\dots = idx$ 
  using fri-sibling-index-first-half[OF even-len k-bound]
  unfolding idx-eq by simp
finally show ?thesis .
qed
qed

lemma fri-sibling-fold-denominator-verifier:
  assumes len-pos:  $0 < len$ 
  and even-len:  $2 \mid len$ 
  and round:  $len * pw = clength * scale$ 
  shows
    fri-fold-denominator  $((h \wedge fri\_sibling\_index\ len\ i) * shift)\ pw =$ 
       $- fri\_fold\_denominator\ ((h \wedge i) * shift)\ pw$ 
  using fri-sibling-domain-round[OF len-pos even-len round, of i]
  unfolding fri-sibling-index-def fri-fold-denominator-def
  by simp

lemma fri-fold-value-sibling-verifier:
  assumes len-pos:  $0 < len$ 
  and even-len:  $2 \mid len$ 
  and round:  $len * pw = clength * scale$ 
  shows
    fri-fold-value  $b\ xn\ xp$ 
       $(fri\_fold\_denominator\ ((h \wedge fri\_sibling\_index\ len\ i) * shift)\ pw) =$ 
      fri-fold-value  $b\ xp\ xn$ 
       $(fri\_fold\_denominator\ ((h \wedge i) * shift)\ pw)$ 
  proof -
    have denom:

```

```

    fri-fold-denominator ((h ^ fri-sibling-index len i) * shift) pw =
      - fri-fold-denominator ((h ^ i) * shift) pw
  by (rule fri-sibling-fold-denominator-verifier
      [OF len-pos even-len round])
  show ?thesis
    unfolding denom
    by (rule fri-fold-value-swap-neg-denominator)
qed

lemma generic-fri-forced-next-value-eq-if-no-same-layer-conflict:
  assumes no-same:
    ¬ generic-fri-sampled-same-layer-opening-conflict roots challenges
      query-idxs round-layers
  and round-bound: round-idx < length query-idxs
  and round-bound': round-idx' < length query-idxs
  and layer-bound: layer-idx < length challenges
  and even-len: 2 dvd fri-evidence-layer-len roots layer-idx
  and raw-bound:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
      fri-evidence-layer-len roots layer-idx
  and raw-bound':
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx <
      fri-evidence-layer-len roots layer-idx
  and round-product:
    fri-evidence-layer-len roots layer-idx * 2 ^ layer-idx =
      clength * scale
  and same-next:
    fri-evidence-next-idx roots query-idxs round-idx layer-idx =
      fri-evidence-next-idx roots query-idxs round-idx' layer-idx
  and forced:
    generic-fri-round-forced-next-value roots challenges query-idxs
      round-layers round-idx layer-idx v
  and forced':
    generic-fri-round-forced-next-value roots challenges query-idxs
      round-layers round-idx' layer-idx v'
  shows v = v'
proof -
  let ?len = fri-evidence-layer-len roots layer-idx
  let ?raw = fri-evidence-layer-idx roots query-idxs round-idx layer-idx
  let ?raw' =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx
  let ?ch = challenges ! layer-idx
  let ?pw = 2 ^ layer-idx
  have len-pos: 0 < ?len
  using raw-bound by simp
  from forced obtain xp xp-path xn xn-path where step:
    fri-layer-step-evidence
      (roots ! layer-idx)
      ?ch

```



```

?len
?raw
?pw
(fri-sibling-index ?len ?raw)
xp xp-path xn xn-path
(fri-evidence-next-idx roots query-idxs round-idx layer-idx)
v
(round-layers ! round-idx ! layer-idx)
unfolding generic-fri-round-forced-next-value-def by blast
have v-eq:
v = fri-evidence-next-value roots challenges query-idxs round-idx
layer-idx xp xn
using fri-layer-step-evidenceD(3)[OF step]
unfolding fri-evidence-next-value-def by simp
have step-norm:
fri-layer-step-evidence
  (roots ! layer-idx)
  ?ch
  ?len
  ?raw
  ?pw
  (fri-sibling-index ?len ?raw)
  xp xp-path xn xn-path
  (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  (fri-evidence-next-value roots challenges query-idxs round-idx
    layer-idx xp xn)
  (round-layers ! round-idx ! layer-idx)
using step v-eq by simp
from forced' obtain yp yp-path yn yn-path where step':
fri-layer-step-evidence
  (roots ! layer-idx)
  ?ch
  ?len
  ?raw'
  ?pw
  (fri-sibling-index ?len ?raw')
  yp yp-path yn yn-path
  (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
  v'
  (round-layers ! round-idx' ! layer-idx)
unfolding generic-fri-round-forced-next-value-def by blast
have v'-eq:
v' = fri-evidence-next-value roots challenges query-idxs round-idx'
layer-idx yp yn
using fri-layer-step-evidenceD(3)[OF step']
unfolding fri-evidence-next-value-def by simp
have step'-norm:
fri-layer-step-evidence
  (roots ! layer-idx)

```

```

?ch
?len
?raw'
?pw
(fri-sibling-index ?len ?raw')
yp yp-path yn yn-path
(fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
(fri-evidence-next-value roots challenges query-idxs round-idx'
  layer-idx yp yn)
(round-layers ! round-idx' ! layer-idx)
using step' v'-eq by simp
have same-mod: ?raw mod (?len div 2) = ?raw' mod (?len div 2)
  using same-next unfolding fri-evidence-next-idx-def by simp
have idx-cases:
  ?raw = ?raw' ∨
  ?raw = fri-sibling-index ?len ?raw' ∨
  fri-sibling-index ?len ?raw = ?raw'
  by (rule fri-same-mod-half-index-cases3
    [OF even-len raw-bound raw-bound' same-mod])
from idx-cases show ?thesis
proof
  assume raw-eq: ?raw = ?raw'
  have xp-eq: xp = yp
    by (rule generic-fri-no-same-layer-opening-conflict-left-left
      [OF no-same round-bound round-bound' layer-bound step-norm
        step'-norm])
    (rule raw-eq)
  have xn-eq: xn = yn
    by (rule generic-fri-no-same-layer-opening-conflict-right-right
      [OF no-same round-bound round-bound' layer-bound step-norm
        step'-norm])
    (use raw-eq in simp)
  show ?thesis
    using v-eq v'-eq raw-eq xp-eq xn-eq
    unfolding fri-evidence-next-value-def
    by simp
next
  assume raw-or-sib:
    ?raw = fri-sibling-index ?len ?raw' ∨
    fri-sibling-index ?len ?raw = ?raw'
  then show ?thesis
  proof
    assume raw-sib: ?raw = fri-sibling-index ?len ?raw'
    have sib-law: fri-sibling-index ?len ?raw = ?raw'
      using raw-sib
      fri-sibling-index-involution[OF even-len raw-bound']
      by simp
    have xp-eq: xp = yn
      by (rule generic-fri-no-same-layer-opening-conflict-left-right

```

```

    [OF no-same round-bound round-bound' layer-bound step-norm
      step'-norm])
  (rule raw-sib)
have xn-eq: xn = yp
  by (rule generic-fri-no-same-layer-opening-conflict-right-left
    [OF no-same round-bound round-bound' layer-bound step-norm
      step'-norm])
    (rule sib-row)
have fold-eq:
  fri-fold-value ?ch xp xn
    (fri-fold-denominator ((h ^ ?raw) * shift) ?pw) =
  fri-fold-value ?ch yp yn
    (fri-fold-denominator ((h ^ ?raw') * shift) ?pw)
using fri-fold-value-sibling-verifier
  [OF len-pos even-len round-product,
    where i=?raw' and b=?ch and xn=yn and xp=yp]
  raw-sib xp-eq xn-eq
  by simp
show ?thesis
  using v-eq v'-eq fold-eq
  unfolding fri-evidence-next-value-def
  by simp
next
assume sib-row: fri-sibling-index ?len ?raw = ?raw'
have raw-sib: ?raw = fri-sibling-index ?len ?raw'
  using sib-row fri-sibling-index-involution[OF even-len raw-bound]
  by simp
have xn-eq: xn = yp
  by (rule generic-fri-no-same-layer-opening-conflict-right-left
    [OF no-same round-bound round-bound' layer-bound step-norm
      step'-norm])
    (rule sib-row)
have xp-eq: xp = yn
  by (rule generic-fri-no-same-layer-opening-conflict-left-right
    [OF no-same round-bound round-bound' layer-bound step-norm
      step'-norm])
    (rule raw-sib)
have fold-eq:
  fri-fold-value ?ch yp yn
    (fri-fold-denominator ((h ^ ?raw') * shift) ?pw) =
  fri-fold-value ?ch xp xn
    (fri-fold-denominator ((h ^ ?raw) * shift) ?pw)
using fri-fold-value-sibling-verifier
  [OF len-pos even-len round-product,
    where i=?raw and b=?ch and xn=xn and xp=xp]
  sib-row xp-eq xn-eq
  by simp
show ?thesis
  using v-eq v'-eq fold-eq

```

```

    unfolding fri-evidence-next-value-def
  by simp
qed
qed
qed

lemma accepted-fri-opening-transcript-composition-layer-arithmetic:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  and degree-bound: to-nat dg ≤ maxDegree
  and layer-bound: layer-idx < length composition-bs
  shows 2 dvd fri-evidence-layer-len composition-roots layer-idx
  and fri-evidence-layer-len composition-roots layer-idx * 2 ^ layer-idx =
    clength * scale
  proof -
    obtain N where eval-power: clength * scale = 2 ^ N
    using eval-domain-length-power by blast
    have bs-len: length composition-bs = length composition-roots
    using accepted-fri-opening-transcript-shapes(4)[OF fri-openings] .
    have roots-bound: layer-idx < length composition-roots
    using layer-bound bs-len by simp
    have roots-le: length composition-roots ≤ N
    proof -
      have len: length composition-roots = ceil-log (to-nat dg + 1)
      using accepted-fri-opening-transcript-shapes(3)[OF fri-openings] .
      have to-nat dg + 1 ≤ clength * scale
      using degree-bound maxDegree-less-eval-domain by linarith
      also have ... = 2 ^ N
      by (rule eval-power)
      finally have ceil-log (to-nat dg + 1) ≤ N
      by (rule ceil-log-le-power)
      then show ?thesis
      using len by simp
    qed
    have suc-le: Suc layer-idx ≤ N
    using roots-bound roots-le by simp
    have dvd-eval-suc: (2::nat) ^ Suc layer-idx dvd clength * scale
    using le-imp-power-dvd[OF suc-le] eval-power by simp
    show 2 dvd fri-evidence-layer-len composition-roots layer-idx
    unfolding fri-evidence-layer-len-def
    by (rule fri-layer-lengths-even-if-dvd[OF roots-bound dvd-eval-suc])
    have le-N: layer-idx ≤ N
    using roots-bound roots-le by simp
    have dvd-eval: (2::nat) ^ layer-idx dvd clength * scale
    using le-imp-power-dvd[OF le-N] eval-power by simp
    show fri-evidence-layer-len composition-roots layer-idx *
      2 ^ layer-idx = clength * scale

```

```

unfolding fri-evidence-layer-len-def
by (rule fri-layer-lengths-round-product-if-dvd
      [OF roots-bound dvd-eval])
qed

lemma accepted-fri-opening-transcript-composition-evidence-layer-idx-bound:
assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers
and degree-bound: to-nat dg ≤ maxDegree
and round-bound: round-idx < length query-idxs
and layer-bound: layer-idx < length composition-roots
shows
  fri-evidence-layer-idx composition-roots query-idxs round-idx layer-idx <
  fri-evidence-layer-len composition-roots layer-idx
proof –
obtain raw-idxs query-chunks query-state where
  len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
using fri-openings
by (auto simp: accepted-fri-opening-transcript-def)
have len-query: length query-idxs = rounds
  using accepted-fri-opening-transcript-shapes(?) [OF fri-openings] .
have round-raw-bound: round-idx < length raw-idxs
  using round-bound len-query len-raw by simp
have query-idx-eq:
  query-idxs ! round-idx = index (to-nat (raw-idxs ! round-idx))
  using query-idxs-eq round-raw-bound by simp
have raw-layer:
  0 < fri-layer-lengths (length composition-roots)
  (clength * scale) ! layer-idx ∧
  fri-layer-indices (length composition-roots)
  (index (to-nat (raw-idxs ! round-idx))) (clength * scale) !
  layer-idx <
  fri-layer-lengths (length composition-roots)
  (clength * scale) ! layer-idx
by (rule accepted-fri-opening-transcript-composition-raw-layer-bound
      [OF fri-openings degree-bound layer-bound])
show ?thesis
  using raw-layer query-idx-eq
  unfolding fri-evidence-layer-idx-def fri-evidence-layer-len-def
  by simp
qed

lemma generic-fri-forced-next-value-eq-from-recorded-step:
assumes forced:
  generic-fri-round-forced-next-value roots challenges query-idxs

```

```

    round-layers round-idx layer-idx v
  and step:
    fri-layer-step-evidence
      (roots ! layer-idx)
      (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      (2 ^ layer-idx)
      (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
      xp xp-path xn xn-path
      (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
      v'
      (round-layers ! round-idx ! layer-idx)
  shows v = v'
proof -
  from forced obtain yp yp-path yn yn-path where forced-step:
    fri-layer-step-evidence
      (roots ! layer-idx)
      (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      (2 ^ layer-idx)
      (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
      yp yp-path yn yn-path
      (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
      v
      (round-layers ! round-idx ! layer-idx)
  unfolding generic-fri-round-forced-next-value-def by blast
  have yp-eq: yp = xp
  by (rule fri-layer-opening-chunk-values-unique(1))
    (rule fri-layer-step-evidenceD(4)[OF forced-step],
     rule fri-layer-step-evidenceD(4)[OF step])
  have yn-eq: yn = xn
  by (rule fri-layer-opening-chunk-values-unique(2))
    (rule fri-layer-step-evidenceD(4)[OF forced-step],
     rule fri-layer-step-evidenceD(4)[OF step])
  show ?thesis
  using fri-layer-step-evidenceD(3)[OF forced-step]
    fri-layer-step-evidenceD(3)[OF step]
    yp-eq yn-eq
  by simp
qed

lemma fri-layer-indices-tail-nth:
  fri-layer-indices (Suc j) (q mod (len div 2)) (len div 2) ! j =
  fri-layer-indices (Suc j) q len ! j mod
  (fri-layer-lengths (Suc j) len ! j div 2)

```

by (*induction j arbitrary: q len*) *simp-all*

lemma *fri-layer-indices-Suc-nth*:

assumes *layer-bound: Suc j < n*

shows

*fri-layer-indices n q len ! Suc j =
 fri-layer-indices n q len ! j mod
 (fri-layer-lengths n len ! j div 2)*

proof –

have *idx-Suc*:

*fri-layer-indices n q len ! Suc j =
 fri-layer-indices (Suc (Suc j)) q len ! Suc j*

using *fri-layer-indices-prefix-nth[OF layer-bound, of q len]* **by** *simp*

have *idx-j*:

*fri-layer-indices n q len ! j =
 fri-layer-indices (Suc j) q len ! j*

using *fri-layer-indices-prefix-nth[of j n q len] layer-bound* **by** *simp*

have *len-j*:

*fri-layer-lengths n len ! j =
 fri-layer-lengths (Suc j) len ! j*

using *fri-layer-lengths-prefix-nth[of j n len] layer-bound* **by** *simp*

show *?thesis*

unfolding *idx-Suc idx-j len-j*

using *fri-layer-indices-tail-nth[of j q len]*

by *simp*

qed

lemma *fri-evidence-layer-idx-Suc*:

assumes *layer-bound: Suc layer-idx < length roots*

shows

*fri-evidence-layer-idx roots query-idxs round-idx (Suc layer-idx) =
 fri-evidence-next-idx roots query-idxs round-idx layer-idx*

using *fri-layer-indices-Suc-nth[OF layer-bound,
 of query-idxs ! round-idx clength * scale]*

unfolding *fri-evidence-layer-idx-def fri-evidence-next-idx-def
 fri-evidence-layer-len-def*

by *simp*

lemma *mfold-fri-layer-openings-successor-prefix-step*:

fixes *s s-suc :: ('f, 'a) protocol-channel-scheme*

assumes *prefix-suc*:

*Some ((idx-suc, x-suc, len-suc, pw-suc), s-suc) ∈
 set-dist*

(execute

(mfold (idx, x, len, pw)

(take (Suc j) (map fri-layer-opening-step bfs))) s)

and *j-bound: j < length bfs*

obtains *idx-j x-j len-j pw-j b rt xp xp-path xn xn-path s-j* **where**
bfs ! j = (b, rt)

```

Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
  set-dist
    (execute
      (mfold (idx, x, len, pw)
        (take j (map fri-layer-opening-step bfs))) s)
Some ((idx-suc, x-suc, len-suc, pw-suc), s-suc) ∈
  set-dist
    (execute
      (fri-layer-opening-step (b, rt) (idx-j, x-j, len-j, pw-j))
        s-j)
fri-layer-step-evidence rt b len-j idx-j pw-j
  (fri-sibling-index len-j idx-j) xp xp-path xn xn-path
  idx-suc x-suc ([xp] @ xp-path @ [xn] @ xn-path)
proof –
  let ?bfs = take (Suc j) bfs
  have j-bound-take: j < length ?bfs
    using j-bound by simp
  have take-map:
    take (Suc j) (map fri-layer-opening-step bfs) =
      map fri-layer-opening-step ?bfs
    by (simp add: take-map)
  show ?thesis
proof (rule mfold-fri-layer-openings-decomp-at
  [OF prefix-suc[unfolded take-map] j-bound-take])
fix idx-j x-j len-j pw-j out-j s-j s-j-suc
assume prefix:
  Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
    set-dist
      (execute
        (mfold (idx, x, len, pw)
          (take j (map fri-layer-opening-step ?bfs))) s)
and selected:
  Some (out-j, s-j-suc) ∈
    set-dist
      (execute
        (fri-layer-opening-step (?bfs ! j)
          (idx-j, x-j, len-j, pw-j)) s-j)
and suffix:
  Some ((idx-suc, x-suc, len-suc, pw-suc), s-suc) ∈
    set-dist
      (execute
        (mfold out-j
          (drop (Suc j) (map fri-layer-opening-step ?bfs))) s-j-suc)
have drop-empty:
  drop (Suc j) (map fri-layer-opening-step ?bfs) = []
  by simp
have out-j-eq: out-j = (idx-suc, x-suc, len-suc, pw-suc)
  and s-suc-eq: s-suc = s-j-suc
  using suffix unfolding drop-empty by simp-all

```



```

have take-prefix-list: take j ?bfs = take j bfs
  by (simp add: take-take)
have take-prefix:
  take j (map fri-layer-opening-step ?bfs) =
    take j (map fri-layer-opening-step bfs)
proof (rule nth-equalityI)
  show length (take j (map fri-layer-opening-step ?bfs)) =
    length (take j (map fri-layer-opening-step bfs))
    by simp
next
  fix i
  assume i-bound: i < length (take j (map fri-layer-opening-step ?bfs))
  then have i < j
    by simp
  then show take j (map fri-layer-opening-step ?bfs) ! i =
    take j (map fri-layer-opening-step bfs) ! i
    using j-bound by simp
qed
have bfs-j: ?bfs ! j = bfs ! j
  using j-bound by simp
obtain b rt where bf-eq: bfs ! j = (b, rt)
  by (cases bfs ! j)
show ?thesis
proof (rule fri-layer-opening-step-outcome-evidence
  [OF selected[unfolded bfs-j bf-eq out-j-eq]])
  fix xp xp-path xn xn-path
  assume xp-eq: xp = x-j
  and next-idx-eq: idx-suc = idx-j mod (len-j div 2)
  and next-len-eq: len-suc = len-j div 2
  and next-pow-eq: pw-suc = pw-j + pw-j
  and step:
    fri-layer-step-evidence rt b len-j idx-j pw-j
    (fri-sibling-index len-j idx-j) xp xp-path xn xn-path
    idx-suc x-suc ([xp] @ xp-path @ [xn] @ xn-path)
  have selected-suc:
    Some ((idx-suc, x-suc, len-suc, pw-suc), s-suc) ∈
      set-dist
      (execute
        (fri-layer-opening-step (b, rt)
          (idx-j, x-j, len-j, pw-j)) s-j)
    using selected[unfolded bfs-j bf-eq out-j-eq] s-suc-eq by simp
  show ?thesis
    by (rule that[OF bf-eq prefix[unfolded take-prefix]
      selected-suc step])
qed
qed
qed

```

lemma mfold-fri-layer-opening-selected-recorded-step-evidence-at:

```

fixes  $s\ t :: ('f, 'a) \text{ protocol-channel-scheme}$ 
assumes  $\text{outcome}$ :
   $\text{Some } (\text{out}, t) \in$ 
     $\text{set-dist}$ 
     $(\text{execute}$ 
       $(\text{mfold } (\text{idx}, x, \text{len}, \text{pw})$ 
         $(\text{map } \text{fri-layer-opening-step } \text{bfs}))\ s)$ 
and  $\text{layers}$ :
   $\text{fri-layers-transcript } (\text{length } \text{bfs})\ \text{len } \text{layer-chunks } \text{total-chunk}$ 
and  $\text{transcript}$ :
   $P\text{Transcript } s = \text{total-chunk } @\ P\text{Transcript } t$ 
and  $\text{initial-idx-bound}$ :  $\text{idx} < \text{len}$ 
and  $\text{all-layer-bounds}$ :
   $\bigwedge k. k < \text{length } \text{bfs} \implies$ 
     $0 < \text{fri-layer-lengths } (\text{length } \text{bfs})\ \text{len} ! k \wedge$ 
     $\text{fri-layer-indices } (\text{length } \text{bfs})\ \text{idx } \text{len} ! k <$ 
     $\text{fri-layer-lengths } (\text{length } \text{bfs})\ \text{len} ! k$ 
and  $\text{j-bound}$ :  $j < \text{length } \text{bfs}$ 
shows
   $\exists xp\ xp\text{-path } xn\ xn\text{-path } \text{next-value}.$ 
   $\text{fri-layer-step-evidence } (\text{snd } (\text{bfs} ! j))\ (\text{fst } (\text{bfs} ! j))$ 
   $(\text{fri-layer-lengths } (\text{length } \text{bfs})\ \text{len} ! j)$ 
   $(\text{fri-layer-indices } (\text{length } \text{bfs})\ \text{idx } \text{len} ! j)$ 
   $(\text{pw} * 2^j)$ 
   $(\text{fri-sibling-index } (\text{fri-layer-lengths } (\text{length } \text{bfs})\ \text{len} ! j)$ 
     $(\text{fri-layer-indices } (\text{length } \text{bfs})\ \text{idx } \text{len} ! j))$ 
   $xp\ xp\text{-path } xn\ xn\text{-path}$ 
   $(\text{fri-layer-indices } (\text{length } \text{bfs})\ \text{idx } \text{len} ! j \bmod$ 
     $(\text{fri-layer-lengths } (\text{length } \text{bfs})\ \text{len} ! j \text{ div } 2))$ 
   $\text{next-value}$ 
   $(\text{layer-chunks} ! j)$ 
proof –
  show  $?thesis$ 
  using  $\text{outcome } \text{layers } \text{transcript } \text{initial-idx-bound } \text{all-layer-bounds}$ 
   $\text{j-bound}$ 
proof  $(\text{induction } \text{bfs } \text{arbitrary};\ \text{idx } x\ \text{len } \text{pw } s\ \text{out } t\ j\ \text{layer-chunks } \text{total-chunk})$ 
  case  $\text{Nil}$ 
  then show  $?case$  by  $\text{simp}$ 
next
  case  $(\text{Cons } \text{bf } \text{bfs})$ 
  obtain  $b\ rt$  where  $\text{bf-eq}: \text{bf} = (b, rt)$ 
  by  $(\text{cases } \text{bf})$ 
  from  $\text{Cons.premis}(1)[\text{unfolded } \text{bf-eq}]$ 
  obtain  $\text{out1 } s1$  where
     $\text{head}:$ 
     $\text{Some } (\text{out1}, s1) \in$ 
       $\text{set-dist}$ 
       $(\text{execute } (\text{fri-layer-opening-step } (b, rt)$ 
         $(\text{idx}, x, \text{len}, \text{pw}))\ s)$ 

```

```

and tail:
  Some (out, t) ∈
    set-dist
      (execute (mfold out1 (map fri-layer-opening-step bfs)) s1)
by (auto elim!: set-dist-bindE)
obtain first rest where chunks-eq: layer-chunks = first # rest
using Cons.premis(6) Cons.premis(2)
unfolding fri-layers-transcript-def by (cases layer-chunks) auto
have total-eq: total-chunk = first @ List.concat rest
using Cons.premis(2) unfolding fri-layers-transcript-def chunks-eq
by simp
have first-open:
  ∃ yp yp-path yn yn-path.
    fri-layer-opening-chunk len yp yp-path yn yn-path first
using Cons.premis(2) unfolding fri-layers-transcript-def chunks-eq
by auto
have rest-layers:
  fri-layers-transcript (length bfs) (len div 2) rest
    (List.concat rest)
using Cons.premis(2) unfolding fri-layers-transcript-def chunks-eq
by auto
obtain x1 xp xp-path xn xn-path head-chunk where out1-eq:
  out1 = (idx mod (len div 2), x1, len div 2, pw + pw)
and head-chunk-def: head-chunk = [xp] @ xp-path @ [xn] @ xn-path
and head-open:
  fri-layer-opening-chunk len xp xp-path xn xn-path head-chunk
and head-transcript:
  PTranscript s = head-chunk @ PTranscript s1
and head-step:
  fri-layer-step-evidence rt b len idx pw
    (fri-sibling-index len idx) xp xp-path xn xn-path
    (idx mod (len div 2)) x1 head-chunk
proof –
have len-pos: 0 < len
using Cons.premis(5)[of 0] by simp
have idx-bound: idx < len
by (rule Cons.premis(4))
show ?thesis
proof (rule fri-layer-opening-step-outcome-authenticated
  [OF len-pos idx-bound head])
fix xp' xp-path' xn' xn-path' x' chunk
assume out-eq:
  out1 = (idx mod (len div 2), x', len div 2, pw + pw)
and xp-eq: xp' = x
and x'-eq:
  x' = fri-fold-value b xp' xn'
    (fri-fold-denominator ((h ^ idx) * shift) pw)
and chunk-eq: chunk = [xp'] @ xp-path' @ [xn'] @ xn-path'
and chunk-open:

```

```

    fri-layer-opening-chunk len xp' xp-path' xn' xn-path'
    chunk
  and tr: PTranscript s = chunk @ PTranscript s1
  assume
    s ≤ s1
    PState s1 = foldl concat (PState s) chunk
    PQueryCounter s1 = PQueryCounter s
    authenticated-opening-in s1
      (opening-root = rt, opening-length = len,
       opening-index = idx, opening-value = xp',
       opening-path = xp-path')
    authenticated-opening-in s1
      (opening-root = rt, opening-length = len,
       opening-index = fri-sibling-index len idx,
       opening-value = xn', opening-path = xn-path')
  have step:
    fri-layer-step-evidence rt b len idx pw
    (fri-sibling-index len idx) xp' xp-path' xn' xn-path'
    (idx mod (len div 2)) x' chunk
  by (rule fri-layer-step-evidenceI)
    (use x'-eq chunk-open in simp-all)
  show ?thesis
  by (rule that[OF out-eq chunk-eq chunk-open tr step])
qed
qed
from mfold-fri-layer-openings-outcome[OF tail[unfolded out1-eq]]
obtain tail-layers tail-chunk where
  tail-transcript:
    PTranscript s1 = tail-chunk @ PTranscript t
  by blast
have head-len:
  length head-chunk = length first
proof -
  obtain yp yp-path yn yn-path where first-open':
    fri-layer-opening-chunk len yp yp-path yn yn-path first
  using first-open by blast
show ?thesis
  using fri-layer-opening-chunk-length[OF head-open]
    fri-layer-opening-chunk-length[OF first-open']
  by simp
qed
have chunks-total-eq:
  head-chunk @ tail-chunk = first @ List.concat rest
  using head-transcript tail-transcript Cons.prem3 total-eq by simp
have head-eq: head-chunk = first
  using arg-cong[OF chunks-total-eq,
    of take (length head-chunk)]
  head-len
  by simp

```

```

have tail-chunk-eq: tail-chunk = List.concat rest
  using chunks-total-eq head-eq by simp
have tail-transcript-rest:
  PTranscript s1 = List.concat rest @ PTranscript t
  using tail-transcript tail-chunk-eq by simp
show ?case
proof (cases j)
  case 0
  have step:
    fri-layer-step-evidence (snd ((bf # bfs) ! j))
      (fst ((bf # bfs) ! j))
      (fri-layer-lengths (length (bf # bfs)) len ! j)
      (fri-layer-indices (length (bf # bfs)) idx len ! j)
      (pw * 2 ^ j)
      (fri-sibling-index
        (fri-layer-lengths (length (bf # bfs)) len ! j)
        (fri-layer-indices (length (bf # bfs)) idx len ! j))
      xp xp-path xn xn-path
      (fri-layer-indices (length (bf # bfs)) idx len ! j mod
        (fri-layer-lengths (length (bf # bfs)) len ! j div 2))
      x1 (layer-chunks ! j)
    using head-step 0 bf-eq head-eq chunks-eq by simp
  show ?thesis
    using step by blast
next
  case (Suc k)
  have k-bound: k < length bfs
    using Cons.prem(6) Suc by simp
  have tail-all-layer-bounds:
     $\bigwedge m. m < \text{length } bfs \implies$ 
    0 < fri-layer-lengths (length bfs) (len div 2) ! m  $\wedge$ 
    fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! m <
    fri-layer-lengths (length bfs) (len div 2) ! m
    using Cons.prem(5)[of Suc m for m] by simp
  have tail-initial-idx-bound:
    idx mod (len div 2) < len div 2
    using tail-all-layer-bounds[of 0] k-bound by (cases bfs) simp-all
  from Cons.IH[OF tail[unfolded out1-eq] rest-layers
    tail-transcript-rest tail-initial-idx-bound tail-all-layer-bounds
    k-bound]
  obtain txp txp-path txn txn-path tnext where tail-step:
    fri-layer-step-evidence (snd (bfs ! k)) (fst (bfs ! k))
      (fri-layer-lengths (length bfs) (len div 2) ! k)
      (fri-layer-indices (length bfs) (idx mod (len div 2))
        (len div 2) ! k)
      ((pw + pw) * 2 ^ k)
      (fri-sibling-index
        (fri-layer-lengths (length bfs) (len div 2) ! k)

```

```

      (fri-layer-indices (length bfs) (idx mod (len div 2))
        (len div 2) ! k))
    txp txp-path txn txn-path
    (fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! k mod
      (fri-layer-lengths (length bfs) (len div 2) ! k div 2))
    tnext (rest ! k)
  by blast
have pow-eq: (pw + pw) * 2 ^ k = pw * 2 ^ Suc k
  by (simp add: algebra-simps)
have step:
  fri-layer-step-evidence (snd ((bf # bfs) ! j))
    (fst ((bf # bfs) ! j))
    (fri-layer-lengths (length (bf # bfs)) len ! j)
    (fri-layer-indices (length (bf # bfs)) idx len ! j)
    (pw * 2 ^ j)
    (fri-sibling-index
      (fri-layer-lengths (length (bf # bfs)) len ! j)
      (fri-layer-indices (length (bf # bfs)) idx len ! j))
    txp txp-path txn txn-path
    (fri-layer-indices (length (bf # bfs)) idx len ! j mod
      (fri-layer-lengths (length (bf # bfs)) len ! j div 2))
    tnext (layer-chunks ! j)
  using tail-step Suc chunks-eq pow-eq by simp
show ?thesis
  using step by blast
qed
qed
qed

lemma mfold-fri-layer-opening-first-recorded-step-evidence:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (out, t) ∈
      set-dist
      (execute
        (mfold (idx, x, len, pw)
          (map fri-layer-opening-step bfs)) s)
  and layers:
    fri-layers-transcript (length bfs) len layer-chunks total-chunk
  and transcript:
    PTranscript s = total-chunk @ PTranscript t
  and nonempty: 0 < length bfs
  shows
    ∃ xp-path xn xn-path next-value.
      fri-layer-step-evidence (snd (bfs ! 0)) (fst (bfs ! 0))
        (fri-layer-lengths (length bfs) len ! 0)
        (fri-layer-indices (length bfs) idx len ! 0)
        pw

```

```

(fri-sibling-index (fri-layer-lengths (length bfs) len ! 0)
 (fri-layer-indices (length bfs) idx len ! 0))
x xp-path xn xn-path
(fri-layer-indices (length bfs) idx len ! 0 mod
 (fri-layer-lengths (length bfs) len ! 0 div 2))
next-value
(layer-chunks ! 0)
proof -
obtain bf bfs' where bfs-eq: bfs = bf # bfs'
using nonempty by (cases bfs) auto
obtain b rt where bf-eq: bf = (b, rt)
by (cases bf)
from outcome[unfolded bfs-eq bf-eq]
obtain out1 s1 where
  head:
    Some (out1, s1) ∈
      set-dist
        (execute (fri-layer-opening-step (b, rt)
          (idx, x, len, pw)) s)
  and tail:
    Some (out, t) ∈
      set-dist
        (execute (mfold out1 (map fri-layer-opening-step bfs') s1)
          by (auto elim!: set-dist-bindE))
obtain first rest where chunks-eq: layer-chunks = first # rest
using layers nonempty unfolding bfs-eq fri-layers-transcript-def
by (cases layer-chunks) auto
have total-eq: total-chunk = first @ List.concat rest
using layers unfolding bfs-eq fri-layers-transcript-def chunks-eq
by simp
have first-open:
  ∃ yp yp-path yn yn-path.
    fri-layer-opening-chunk len yp yp-path yn yn-path first
using layers unfolding bfs-eq fri-layers-transcript-def chunks-eq
by auto
obtain x1 xp xp-path xn xn-path head-chunk where out1-eq:
  out1 = (idx mod (len div 2), x1, len div 2, pw + pw)
and head-chunk-def: head-chunk = [xp] @ xp-path @ [xn] @ xn-path
and xp-eq: xp = x
and head-open:
  fri-layer-opening-chunk len xp xp-path xn xn-path head-chunk
and head-transcript:
  PTranscript s = head-chunk @ PTranscript s1
and head-step:
  fri-layer-step-evidence rt b len idx pw
  (fri-sibling-index len idx) xp xp-path xn xn-path
  (idx mod (len div 2)) x1 head-chunk
proof -
from fri-layer-opening-step-outcome[OF head]

```

```

obtain  $xp'$   $xp\text{-path}'$   $xn'$   $xn\text{-path}'$   $x'$  where  $out1\text{-eq}'$ :
   $out1 = (idx \bmod (len \div 2), x', len \div 2, pw + pw)$ 
and  $xp'\text{-eq}$ :  $xp' = x$ 
and  $x'\text{-eq}$ :
   $x' = \text{fri-fold-value } b \text{ } xp' \text{ } xn'$ 
   $(\text{fri-fold-denominator } ((h \wedge idx) * shift) \text{ } pw)$ 
and  $chunk\text{-open}$ :
   $\text{fri-layer-opening-chunk } len \text{ } xp' \text{ } xp\text{-path}' \text{ } xn' \text{ } xn\text{-path}'$ 
   $([xp'] @ xp\text{-path}' @ [xn'] @ xn\text{-path}')$ 
and  $tr$ :
   $PTranscript \text{ } s =$ 
   $([xp'] @ xp\text{-path}' @ [xn'] @ xn\text{-path}') @ PTranscript \text{ } s1$ 
by  $blast$ 
have  $step$ :
   $\text{fri-layer-step-evidence } rt \text{ } b \text{ } len \text{ } idx \text{ } pw$ 
   $(\text{fri-sibling-index } len \text{ } idx) \text{ } xp' \text{ } xp\text{-path}' \text{ } xn' \text{ } xn\text{-path}'$ 
   $(idx \bmod (len \div 2)) \text{ } x'$ 
   $([xp'] @ xp\text{-path}' @ [xn'] @ xn\text{-path}')$ 
by  $(\text{rule } \text{fri-layer-step-evidenceI})$ 
   $(\text{use } x'\text{-eq } chunk\text{-open} \text{ in } simp\text{-all})$ 
show  $?thesis$ 
  by  $(\text{rule that}[OF \text{ } out1\text{-eq}' \text{ refl } xp'\text{-eq } chunk\text{-open } tr \text{ } step])$ 
qed
from  $\text{mfold-fri-layer-openings-outcome}[OF \text{ } tail[unfolded \text{ } out1\text{-eq}]]$ 
obtain  $tail\text{-layers}$   $tail\text{-chunk}$  where
   $tail\text{-transcript}$ :
   $PTranscript \text{ } s1 = tail\text{-chunk} @ PTranscript \text{ } t$ 
by  $blast$ 
have  $head\text{-len}$ :
   $length \text{ } head\text{-chunk} = length \text{ } first$ 
proof –
  obtain  $yp$   $yp\text{-path}$   $yn$   $yn\text{-path}$  where  $first\text{-open}'$ :
   $\text{fri-layer-opening-chunk } len \text{ } yp \text{ } yp\text{-path} \text{ } yn \text{ } yn\text{-path} \text{ } first$ 
using  $first\text{-open}$  by  $blast$ 
show  $?thesis$ 
using  $\text{fri-layer-opening-chunk-length}[OF \text{ } head\text{-open}]$ 
   $\text{fri-layer-opening-chunk-length}[OF \text{ } first\text{-open}']$ 
by  $simp$ 
qed
have  $chunks\text{-total}\text{-eq}$ :
   $head\text{-chunk} @ tail\text{-chunk} = first @ List.concat \text{ } rest$ 
using  $head\text{-transcript } tail\text{-transcript } transcript \text{ } total\text{-eq}$  by  $simp$ 
have  $head\text{-eq}$ :  $head\text{-chunk} = first$ 
using  $\text{arg-cong}[OF \text{ } chunks\text{-total}\text{-eq},$ 
   $\text{of take } (length \text{ } head\text{-chunk})]$ 
   $head\text{-len}$ 
by  $simp$ 
have  $step$ :
   $\text{fri-layer-step-evidence } (snd \text{ } (bfs ! 0)) \text{ } (fst \text{ } (bfs ! 0))$ 

```



```

(fri-layer-lengths (length bfs) len ! 0)
(fri-layer-indices (length bfs) idx len ! 0)
pw
(fri-sibling-index (fri-layer-lengths (length bfs) len ! 0)
  (fri-layer-indices (length bfs) idx len ! 0))
x xp-path xn xn-path
(fri-layer-indices (length bfs) idx len ! 0 mod
  (fri-layer-lengths (length bfs) len ! 0 div 2))
x1
(layer-chunks ! 0)
using head-step bfs-eq bf-eq chunks-eq head-eq xp-eq by simp
show ?thesis
using step by blast
qed

lemma mfold-fri-layer-opening-successor-recorded-step-from-previous:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (out, t) ∈
    set-dist
    (execute
      (mfold (idx, x, len, pw)
        (map fri-layer-opening-step bfs)) s)
and layers:
  fri-layers-transcript (length bfs) len layer-chunks total-chunk
and transcript:
  PTranscript s = total-chunk @ PTranscript t
and initial-idx-bound: idx < len
and all-layer-bounds:
   $\bigwedge k. k < \text{length } \text{bfs} \implies$ 
    0 < fri-layer-lengths (length bfs) len ! k  $\wedge$ 
    fri-layer-indices (length bfs) idx len ! k <
    fri-layer-lengths (length bfs) len ! k
and j-bound: Suc j < length bfs
and previous-step:
  fri-layer-step-evidence (snd (bfs ! j)) (fst (bfs ! j))
  (fri-layer-lengths (length bfs) len ! j)
  (fri-layer-indices (length bfs) idx len ! j)
  (pw * 2 ^ j)
  (fri-sibling-index
    (fri-layer-lengths (length bfs) len ! j)
    (fri-layer-indices (length bfs) idx len ! j))
  xp xp-path xn xn-path
  (fri-layer-indices (length bfs) idx len ! j mod
    (fri-layer-lengths (length bfs) len ! j div 2))
  v
  (layer-chunks ! j)
shows
   $\exists xp\text{-path}' xn' xn\text{-path}' \text{ next-value.}$ 

```

```

    fri-layer-step-evidence (snd (bfs ! Suc j)) (fst (bfs ! Suc j))
    (fri-layer-lengths (length bfs) len ! Suc j)
    (fri-layer-indices (length bfs) idx len ! Suc j)
    (pw * 2 ^ Suc j)
    (fri-sibling-index
      (fri-layer-lengths (length bfs) len ! Suc j)
      (fri-layer-indices (length bfs) idx len ! Suc j))
    v xp-path' xn' xn-path'
    (fri-layer-indices (length bfs) idx len ! Suc j mod
      (fri-layer-lengths (length bfs) len ! Suc j div 2))
    next-value
    (layer-chunks ! Suc j)
  using outcome layers transcript initial-idx-bound all-layer-bounds j-bound
  previous-step
proof (induction bfs arbitrary: idx x len pw s out t j layer-chunks total-chunk
  xp xp-path xn xn-path v)
  case Nil
  then show ?case by simp
next
  case (Cons bf bfs)
  obtain b rt where bf-eq: bf = (b, rt)
  by (cases bf)
  from Cons.prem1[1][unfolded bf-eq]
  obtain out1 s1 where
    head:
      Some (out1, s1) ∈
        set-dist
          (execute (fri-layer-opening-step (b, rt)
            (idx, x, len, pw)) s)
  and tail:
      Some (out, t) ∈
        set-dist
          (execute (mfold out1 (map fri-layer-opening-step bfs)) s1)
  by (auto elim!: set-dist-bindE)
  obtain first rest where chunks-eq: layer-chunks = first # rest
  using Cons.prem2 Cons.prem6
  unfolding fri-layers-transcript-def by (cases layer-chunks) auto
  have total-eq: total-chunk = first @ List.concat rest
  using Cons.prem2 unfolding fri-layers-transcript-def chunks-eq
  by simp
  have first-open:
    ∃ yp yp-path yn yn-path.
      fri-layer-opening-chunk len yp yp-path yn yn-path first
  using Cons.prem2 unfolding fri-layers-transcript-def chunks-eq
  by auto
  have rest-layers:
    fri-layers-transcript (length bfs) (len div 2) rest
    (List.concat rest)
  using Cons.prem2 unfolding fri-layers-transcript-def chunks-eq

```

```

by auto
obtain  $x1$   $hxp$   $hxp\text{-}path$   $hxn$   $hxn\text{-}path$   $head\text{-}chunk$  where  $out1\text{-}eq$ :
   $out1 = (idx \bmod (len \div 2), x1, len \div 2, pw + pw)$ 
and  $hxp\text{-}eq$ :  $hxp = x$ 
and  $head\text{-}open$ :
   $fri\text{-}layer\text{-}opening\text{-}chunk\ len\ hxp\ hxp\text{-}path\ hxn\ hxn\text{-}path\ head\text{-}chunk$ 
and  $head\text{-}transcript$ :
   $PTranscript\ s = head\text{-}chunk\ @\ PTranscript\ s1$ 
and  $head\text{-}step$ :
   $fri\text{-}layer\text{-}step\text{-}evidence\ rt\ b\ len\ idx\ pw$ 
   $(fri\text{-}sibling\text{-}index\ len\ idx)\ hxp\ hxp\text{-}path\ hxn\ hxn\text{-}path$ 
   $(idx \bmod (len \div 2))\ x1\ head\text{-}chunk$ 
proof –
from  $fri\text{-}layer\text{-}opening\text{-}step\text{-}outcome[OF\ head]$ 
obtain  $xp'$   $xp\text{-}path'$   $xn'$   $xn\text{-}path'$   $x'$  where  $out1\text{-}eq'$ :
   $out1 = (idx \bmod (len \div 2), x', len \div 2, pw + pw)$ 
and  $xp'\text{-}eq$ :  $xp' = x$ 
and  $x'\text{-}eq$ :
   $x' = fri\text{-}fold\text{-}value\ b\ xp'\ xn'$ 
   $(fri\text{-}fold\text{-}denominator\ ((h \wedge idx) * shift)\ pw)$ 
and  $chunk\text{-}open$ :
   $fri\text{-}layer\text{-}opening\text{-}chunk\ len\ xp'\ xp\text{-}path'\ xn'\ xn\text{-}path'$ 
   $([xp'] @ xp\text{-}path' @ [xn'] @ xn\text{-}path')$ 
and  $tr$ :
   $PTranscript\ s =$ 
   $([xp'] @ xp\text{-}path' @ [xn'] @ xn\text{-}path') @ PTranscript\ s1$ 
by  $blast$ 
have  $step$ :
   $fri\text{-}layer\text{-}step\text{-}evidence\ rt\ b\ len\ idx\ pw$ 
   $(fri\text{-}sibling\text{-}index\ len\ idx)\ xp'\ xp\text{-}path'\ xn'\ xn\text{-}path'$ 
   $(idx \bmod (len \div 2))\ x'$ 
   $([xp'] @ xp\text{-}path' @ [xn'] @ xn\text{-}path')$ 
by  $(rule\ fri\text{-}layer\text{-}step\text{-}evidenceI)$ 
   $(use\ x'\text{-}eq\ chunk\text{-}open\ in\ simp\text{-}all)$ 
show  $?thesis$ 
by  $(rule\ that[OF\ out1\text{-}eq'\ xp'\text{-}eq\ chunk\text{-}open\ tr\ step])$ 
qed
from  $mfold\text{-}fri\text{-}layer\text{-}openings\text{-}outcome[OF\ tail[unfolded\ out1\text{-}eq]]$ 
obtain  $tail\text{-}layers$   $tail\text{-}chunk$  where
   $tail\text{-}transcript$ :
   $PTranscript\ s1 = tail\text{-}chunk\ @\ PTranscript\ t$ 
by  $blast$ 
have  $head\text{-}len$ :
   $length\ head\text{-}chunk = length\ first$ 
proof –
obtain  $yp$   $yp\text{-}path$   $yn$   $yn\text{-}path$  where  $first\text{-}open'$ :
   $fri\text{-}layer\text{-}opening\text{-}chunk\ len\ yp\ yp\text{-}path\ yn\ yn\text{-}path\ first$ 
using  $first\text{-}open$  by  $blast$ 
show  $?thesis$ 

```

```

    using fri-layer-opening-chunk-length[OF head-open]
      fri-layer-opening-chunk-length[OF first-open]
  by simp
qed
have chunks-total-eq:
  head-chunk @ tail-chunk = first @ List.concat rest
  using head-transcript tail-transcript Cons.prems(3) total-eq by simp
have head-eq: head-chunk = first
  using arg-cong[OF chunks-total-eq,
    of take (length head-chunk)]
    head-len
  by simp
have tail-chunk-eq: tail-chunk = List.concat rest
  using chunks-total-eq head-eq by simp
have tail-transcript-rest:
  PTranscript s1 = List.concat rest @ PTranscript t
  using tail-transcript tail-chunk-eq by simp
show ?case
proof (cases j)
  case 0
  have prev-head:
    fri-layer-step-evidence rt b len idx pw
      (fri-sibling-index len idx) xp xp-path xn xn-path
      (idx mod (len div 2)) v first
    using Cons.prems(7) 0 bf-eq chunks-eq by simp
  have v-eq-x1: v = x1
  proof -
    have xp-eq: xp = hxp
      by (rule fri-layer-opening-chunk-values-unique(1))
        (rule fri-layer-step-evidenceD(4)[OF prev-head],
          rule fri-layer-step-evidenceD(4)[OF head-step[unfolded head-eq]])
    have xn-eq: xn = hxn
      by (rule fri-layer-opening-chunk-values-unique(2))
        (rule fri-layer-step-evidenceD(4)[OF prev-head],
          rule fri-layer-step-evidenceD(4)[OF head-step[unfolded head-eq]])
    show ?thesis
      using fri-layer-step-evidenceD(3)[OF prev-head]
        fri-layer-step-evidenceD(3)[OF head-step[unfolded head-eq]]
        xp-eq xn-eq
      by simp
  qed
  have bfs-nonempty: 0 < length bfs
    using Cons.prems(6) 0 by simp
  from mfold-fri-layer-opening-first-recorded-step-evidence
    [OF tail[unfolded out1-eq] rest-layers tail-transcript-rest
      bfs-nonempty]
  obtain sxp-path sxn sxn-path snext where succ-step-tail:
    fri-layer-step-evidence (snd (bfs ! 0)) (fst (bfs ! 0))
    (fri-layer-lengths (length bfs) (len div 2) ! 0)

```

```

(fri-layer-indices (length bfs) (idx mod (len div 2))
 (len div 2) ! 0)
(pw + pw)
(fri-sibling-index
 (fri-layer-lengths (length bfs) (len div 2) ! 0)
 (fri-layer-indices (length bfs) (idx mod (len div 2))
 (len div 2) ! 0))
x1 sxp-path sxn sxn-path
(fri-layer-indices (length bfs) (idx mod (len div 2))
 (len div 2) ! 0 mod
 (fri-layer-lengths (length bfs) (len div 2) ! 0 div 2))
snext
(rest ! 0)
by blast
have pow-eq: pw + pw = pw * 2 ^ Suc 0
  by (simp add: algebra-simps)
have succ-step:
  fri-layer-step-evidence (snd ((bf # bfs) ! Suc j))
    (fst ((bf # bfs) ! Suc j))
    (fri-layer-lengths (length (bf # bfs)) len ! Suc j)
    (fri-layer-indices (length (bf # bfs)) idx len ! Suc j)
    (pw * 2 ^ Suc j)
    (fri-sibling-index
      (fri-layer-lengths (length (bf # bfs)) len ! Suc j)
      (fri-layer-indices (length (bf # bfs)) idx len ! Suc j))
    v sxp-path sxn sxn-path
    (fri-layer-indices (length (bf # bfs)) idx len ! Suc j mod
      (fri-layer-lengths (length (bf # bfs)) len ! Suc j div 2))
    snext
    (layer-chunks ! Suc j)
  using succ-step-tail 0 chunks-eq v-eq-x1 pow-eq by simp
show ?thesis
  using succ-step by blast
next
case (Suc k)
have k-bound: Suc k < length bfs
  using Cons.premis(6) Suc by simp
have tail-all-layer-bounds:
   $\bigwedge m. m < \text{length } bfs \implies$ 
    0 < fri-layer-lengths (length bfs) (len div 2) ! m  $\wedge$ 
    fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! m <
    fri-layer-lengths (length bfs) (len div 2) ! m
  using Cons.premis(5)[of Suc m for m] by simp
have tail-initial-idx-bound:
  idx mod (len div 2) < len div 2
  using tail-all-layer-bounds[of 0] k-bound by (cases bfs) simp-all
have prev-tail:
  fri-layer-step-evidence (snd (bfs ! k)) (fst (bfs ! k))

```

```

(fri-layer-lengths (length bfs) (len div 2) ! k)
(fri-layer-indices (length bfs) (idx mod (len div 2))
  (len div 2) ! k)
((pw + pw) * 2 ^ k)
(fri-sibling-index
  (fri-layer-lengths (length bfs) (len div 2) ! k)
  (fri-layer-indices (length bfs) (idx mod (len div 2))
    (len div 2) ! k))
xp xp-path xn xn-path
(fri-layer-indices (length bfs) (idx mod (len div 2))
  (len div 2) ! k mod
  (fri-layer-lengths (length bfs) (len div 2) ! k div 2))
v (rest ! k)
proof -
  have pow-eq: (pw + pw) * 2 ^ k = pw * (2 * 2 ^ k)
    by (simp add: algebra-simps)
  have raw:
    fri-layer-step-evidence (snd (bfs ! k)) (fst (bfs ! k))
      (fri-layer-lengths (length bfs) (len div 2) ! k)
      (fri-layer-indices (length bfs) (idx mod (len div 2))
        (len div 2) ! k)
      (pw * (2 * 2 ^ k))
      (fri-sibling-index
        (fri-layer-lengths (length bfs) (len div 2) ! k)
        (fri-layer-indices (length bfs) (idx mod (len div 2))
          (len div 2) ! k))
      xp xp-path xn xn-path
      (fri-layer-indices (length bfs) (idx mod (len div 2))
        (len div 2) ! k mod
        (fri-layer-lengths (length bfs) (len div 2) ! k div 2))
      v (rest ! k)
    using Cons.premis(7) Suc chunks-eq pow-eq by simp
  show ?thesis
    unfolding pow-eq by (rule raw)
qed
from Cons.IH[OF tail[unfolded out1-eq] rest-layers
  tail-transcript-rest tail-initial-idx-bound tail-all-layer-bounds
  k-bound prev-tail]
obtain sxp-path sxn sxn-path snext where succ-tail:
  fri-layer-step-evidence (snd (bfs ! Suc k))
    (fst (bfs ! Suc k))
    (fri-layer-lengths (length bfs) (len div 2) ! Suc k)
    (fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! Suc k)
    ((pw + pw) * 2 ^ Suc k)
    (fri-sibling-index
      (fri-layer-lengths (length bfs) (len div 2) ! Suc k)
      (fri-layer-indices (length bfs) (idx mod (len div 2))
        (len div 2) ! Suc k))

```

```

    v sxp-path sxn sxn-path
    (fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! Suc k mod
      (fri-layer-lengths (length bfs) (len div 2) ! Suc k div 2))
    snext (rest ! Suc k)
  by blast
have pow-eq: (pw + pw) * (2 * 2 ^ k) = pw * (4 * 2 ^ k)
  by (simp add: algebra-simps)
have succ-tail-raw:
  fri-layer-step-evidence (snd (bfs ! Suc k))
    (fst (bfs ! Suc k))
    (fri-layer-lengths (length bfs) (len div 2) ! Suc k)
    (fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! Suc k)
    ((pw + pw) * (2 * 2 ^ k))
    (fri-sibling-index
      (fri-layer-lengths (length bfs) (len div 2) ! Suc k)
      (fri-layer-indices (length bfs) (idx mod (len div 2))
        (len div 2) ! Suc k))
  v sxp-path sxn sxn-path
  (fri-layer-indices (length bfs) (idx mod (len div 2))
    (len div 2) ! Suc k mod
    (fri-layer-lengths (length bfs) (len div 2) ! Suc k div 2))
  snext (rest ! Suc k)
  using succ-tail by simp
have succ-step:
  fri-layer-step-evidence (snd ((bf # bfs) ! Suc j))
    (fst ((bf # bfs) ! Suc j))
    (fri-layer-lengths (length (bf # bfs)) len ! Suc j)
    (fri-layer-indices (length (bf # bfs)) idx len ! Suc j)
    (pw * 2 ^ Suc j)
    (fri-sibling-index
      (fri-layer-lengths (length (bf # bfs)) len ! Suc j)
      (fri-layer-indices (length (bf # bfs)) idx len ! Suc j))
  v sxp-path sxn sxn-path
  (fri-layer-indices (length (bf # bfs)) idx len ! Suc j mod
    (fri-layer-lengths (length (bf # bfs)) len ! Suc j div 2))
  snext
  (layer-chunks ! Suc j)
  using succ-tail-raw Suc chunks-eq pow-eq
  by (simp add: algebra-simps)
show ?thesis
  using succ-step by blast
qed
qed

lemma verifier-query-round-after-index-composition-recorded-layer-step-evidence:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:

```

```

Some (( ), t) ∈
  set-dist
    (execute
      (verifier-query-round-after-index-program
        fr f-fl f-final as fl final raw) s)
and recorded:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) round-chunk trace-layers
    composition-layers
and transcript: PTranscript s = round-chunk @ PTranscript t
and j-bound: j < length fl
and all-layer-bounds:
  ∧k. k < length fl ⇒
    0 < fri-layer-lengths (length fl) (clength * scale) ! k ∧
    fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! k <
    fri-layer-lengths (length fl) (clength * scale) ! k
shows
  ∃ xp xp-path xn xn-path next-value.
    fri-layer-step-evidence (snd (fl ! j)) (fst (fl ! j))
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! j)
      (2 ^ j)
      (fri-sibling-index
        (fri-layer-lengths (length fl) (clength * scale) ! j)
        (fri-layer-indices (length fl) (index (to-nat raw))
          (clength * scale) ! j))
      xp xp-path xn xn-path
      (fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! j mod
        (fri-layer-lengths (length fl) (clength * scale) ! j div 2))
      next-value
      (composition-layers ! j)
proof —
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1) ∈
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
  and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some (( ), s3) ∈
      set-dist
        (execute

```



```

      (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
and comp-fri:
  Some (c-out, s4) ∈
    set-dist (execute
      (mfold
        (?idx, cp-eval as fv (h ^ ?idx * shift), clength * scale, 1)
        (receive-query-commits fl)) s3)
and assert-comp:
  Some (( ), t) ∈
    set-dist
      (execute
        (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
unfolding verifier-query-round-after-index-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript ?idx fv query-paths query-chunk
and tr-s: PTranscript s = query-chunk @ PTranscript s1
by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  trace-layers-actual:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
by blast
have s3-eq: s3 = s2
using assert-trace unfolding assert-def
by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
  (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers-actual:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk
and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
by blast
have t-eq: t = s4
using assert-comp unfolding assert-def
by (cases case c-out of (-, x, -, -) ⇒ x = final)
  (auto simp: throw-no-outcome)
let ?actual-chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have actual-recorded:
  query-round-fri-layer-transcripts ?idx (map snd f-fl) (map snd fl)
    ?actual-chunk trace-layer-chunks comp-layer-chunks
unfolding query-round-fri-layer-transcripts-def
by (intro exI[of - query-chunk] exI[of - trace-fri-chunk]
    exI[of - comp-fri-chunk] exI[of - fv] exI[of - query-paths] conjI)

```

```

    (use query-chunk-shape trace-layers-actual comp-layers-actual in
      ⟨simp-all add: mult.commute⟩)
  have actual-transcript:
    PTranscript s = ?actual-chunk @ PTranscript t
    using tr-s tr-s1 tr-s3 s3-eq t-eq by simp
  have actual-chunk-eq: ?actual-chunk = round-chunk
    using actual-transcript transcript by simp
  have comp-layer-chunks-eq:
    comp-layer-chunks = composition-layers
    by (rule query-round-fri-layer-transcripts-unique(2)
      [OF actual-recorded[unfolded actual-chunk-eq] recorded])
  have idx-bound: ?idx < clength * scale
    by (rule index-less-domain)
  have map-eq:
    receive-query-commits fl = map fri-layer-opening-step fl
    unfolding receive-query-commits-def fri-layer-opening-step-def
    by (simp add: fun-eq-iff split: prod.splits)
  from mfold-fri-layer-opening-selected-recorded-step-evidence-at
    [OF comp-fri[unfolded map-eq] comp-layers-actual tr-s3 idx-bound
      all-layer-bounds j-bound]
  obtain xp xp-path xn xn-path next-value where step-actual:
    fri-layer-step-evidence (snd (fl ! j)) (fst (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) ?idx (clength * scale) ! j)
    (1 * 2 ^ j)
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) ?idx (clength * scale) ! j))
    xp xp-path xn xn-path
    (fri-layer-indices (length fl) ?idx (clength * scale) ! j mod
      (fri-layer-lengths (length fl) (clength * scale) ! j div 2))
    next-value
    (comp-layer-chunks ! j)
  by blast
  have step:
    fri-layer-step-evidence (snd (fl ! j)) (fst (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) ?idx (clength * scale) ! j)
    (2 ^ j)
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) ?idx (clength * scale) ! j))
    xp xp-path xn xn-path
    (fri-layer-indices (length fl) ?idx (clength * scale) ! j mod
      (fri-layer-lengths (length fl) (clength * scale) ! j div 2))
    next-value
    (composition-layers ! j)
  using step-actual comp-layer-chunks-eq by simp
  show ?thesis

```

using step by blast
qed

lemma *verifier-query-round-after-index-composition-successor-recorded-step-from-previous:*

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes *outcome:*

Some $((\), t) \in$
set-dist
(execute
(verifier-query-round-after-index-program
fr f -fl f -final as fl final raw) s)

and *recorded:*

query-round-fri-layer-transcripts (index (to-nat raw))
(map snd f -fl) (map snd fl) round-chunk trace-layers
composition-layers

and *transcript:* $P\text{Transcript } s = \text{round-chunk } @\ P\text{Transcript } t$

and *j-bound:* $\text{Suc } j < \text{length } fl$

and *all-layer-bounds:*

$\bigwedge k. k < \text{length } fl \implies$
 $0 < \text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! k \wedge$
 $\text{fri-layer-indices } (\text{length } fl) (\text{index } (\text{to-nat } \text{raw}))$
 $(\text{clength} * \text{scale}) ! k <$
 $\text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! k$

and *previous-step:*

fri-layer-step-evidence (snd (fl ! j)) (fst (fl ! j))
(fri-layer-lengths (length fl) (clength * scale) ! j)
(fri-layer-indices (length fl) (index (to-nat raw)))
(clength * scale) ! j)
($2 \wedge j$)
(fri-sibling-index
(fri-layer-lengths (length fl) (clength * scale) ! j)
(fri-layer-indices (length fl) (index (to-nat raw)))
(clength * scale) ! j))
 $xp\ xp\text{-path } xn\ xn\text{-path}$
(fri-layer-indices (length fl) (index (to-nat raw)))
(clength * scale) ! $j \bmod$
(fri-layer-lengths (length fl) (clength * scale) ! $j \text{ div } 2$))
 v
(composition-layers ! j)

shows

$\exists xp\text{-path}'\ xn'\ xn\text{-path}'\ \text{next-value}.$

fri-layer-step-evidence (snd (fl ! Suc j)) (fst (fl ! Suc j))
(fri-layer-lengths (length fl) (clength * scale) ! Suc j)
(fri-layer-indices (length fl) (index (to-nat raw)))
(clength * scale) ! Suc j)
($2 \wedge \text{Suc } j$)
(fri-sibling-index
(fri-layer-lengths (length fl) (clength * scale) ! Suc j)
(fri-layer-indices (length fl) (index (to-nat raw)))

```

      (clength * scale) ! Suc j))
    v xp-path' xn' xn-path'
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! Suc j mod
      (fri-layer-lengths (length fl) (clength * scale) ! Suc j div 2))
    next-value
    (composition-layers ! Suc j)
proof -
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1) ∈
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
  and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some ((), s3) ∈
      set-dist
        (execute
          (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold
          (?idx, cp-eval as fv (h ^ ?idx * shift), clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some ((), t) ∈
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript ?idx fv query-paths query-chunk
  and tr-s: PTranscript s = query-chunk @ PTranscript s1
  by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  trace-layers-actual:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
  and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
  by blast

```

```

have s3-eq:  $s3 = s2$ 
  using assert-trace unfolding assert-def
  by (cases case f-out of  $(-, f-x, -, -) \Rightarrow f-x = f-final$ )
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers-actual:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk
  and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
  by blast
have t-eq:  $t = s4$ 
  using assert-comp unfolding assert-def
  by (cases case c-out of  $(-, x, -, -) \Rightarrow x = final$ )
    (auto simp: throw-no-outcome)
let ?actual-chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have actual-recorded:
  query-round-fri-layer-transcripts ?idx (map snd f-fl) (map snd fl)
  ?actual-chunk trace-layer-chunks comp-layer-chunks
  unfolding query-round-fri-layer-transcripts-def
  by (intro exI[of - query-chunk] exI[of - trace-fri-chunk]
    exI[of - comp-fri-chunk] exI[of - fv] exI[of - query-paths] conjI)
    (use query-chunk-shape trace-layers-actual comp-layers-actual in
       $\langle simp-all add: mult.commute \rangle$ )
have actual-transcript:
  PTranscript s = ?actual-chunk @ PTranscript t
  using tr-s tr-s1 tr-s3 s3-eq t-eq by simp
have actual-chunk-eq: ?actual-chunk = round-chunk
  using actual-transcript transcript by simp
have comp-layer-chunks-eq:
  comp-layer-chunks = composition-layers
  by (rule query-round-fri-layer-transcripts-unique(2)
    [OF actual-recorded[unfolded actual-chunk-eq] recorded])
have idx-bound: ?idx < clength * scale
  by (rule index-less-domain)
have map-eq:
  receive-query-commits fl = map fri-layer-opening-step fl
  unfolding receive-query-commits-def fri-layer-opening-step-def
  by (simp add: fun-eq-iff split: prod.splits)
have previous-step-actual:
  fri-layer-step-evidence (snd (fl ! j)) (fst (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) ?idx (clength * scale) ! j)
    ( $1 * 2^j$ )
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) ?idx (clength * scale) ! j))
  xp xp-path xn xn-path
    (fri-layer-indices (length fl) ?idx (clength * scale) ! j mod

```

```

    (fri-layer-lengths (length fl) (clength * scale) ! j div 2))
  v
  (comp-layer-chunks ! j)
using previous-step comp-layer-chunks-eq by simp
from mfold-fri-layer-opening-successor-recorded-step-from-previous
  [OF comp-fri[unfolded map-eq] comp-layers-actual tr-s3 idx-bound
    all-layer-bounds j-bound previous-step-actual]
obtain sxp-path sxn sxn-path snext where succ-actual:
  fri-layer-step-evidence (snd (fl ! Suc j)) (fst (fl ! Suc j))
  (fri-layer-lengths (length fl) (clength * scale) ! Suc j)
  (fri-layer-indices (length fl) ?idx (clength * scale) ! Suc j)
  (1 * 2 ^ Suc j)
  (fri-sibling-index
    (fri-layer-lengths (length fl) (clength * scale) ! Suc j)
    (fri-layer-indices (length fl) ?idx (clength * scale) ! Suc j))
  v sxp-path sxn sxn-path
  (fri-layer-indices (length fl) ?idx (clength * scale) ! Suc j mod
    (fri-layer-lengths (length fl) (clength * scale) ! Suc j div 2))
  snext
  (comp-layer-chunks ! Suc j)
by blast
have succ:
  fri-layer-step-evidence (snd (fl ! Suc j)) (fst (fl ! Suc j))
  (fri-layer-lengths (length fl) (clength * scale) ! Suc j)
  (fri-layer-indices (length fl) ?idx (clength * scale) ! Suc j)
  (2 ^ Suc j)
  (fri-sibling-index
    (fri-layer-lengths (length fl) (clength * scale) ! Suc j)
    (fri-layer-indices (length fl) ?idx (clength * scale) ! Suc j))
  v sxp-path sxn sxn-path
  (fri-layer-indices (length fl) ?idx (clength * scale) ! Suc j mod
    (fri-layer-lengths (length fl) (clength * scale) ! Suc j div 2))
  snext
  (composition-layers ! Suc j)
using succ-actual comp-layer-chunks-eq by simp
show ?thesis
using succ by blast
qed

lemma accepted-fri-opening-transcript-composition-successor-recorded-step-from-previous-at:
assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers
and out-eq: out = Some (result, final-state)
and round-bound: i < rounds
and layer-bound: Suc j < length composition-roots
and all-layer-bounds:
   $\bigwedge \text{raw } k. k < \text{length composition-roots} \implies$ 

```

$0 < \text{fri-layer-lengths } (\text{length } \text{composition-roots})$
 $(\text{clength} * \text{scale}) ! k \wedge$
 $\text{fri-layer-indices } (\text{length } \text{composition-roots}) (\text{index } (\text{to-nat } \text{raw}))$
 $(\text{clength} * \text{scale}) ! k <$
 $\text{fri-layer-lengths } (\text{length } \text{composition-roots}) (\text{clength} * \text{scale}) ! k$
and *previous-step*:
 $\text{fri-layer-step-evidence } (\text{composition-roots} ! j)$
 $(\text{composition-bs} ! j)$
 $(\text{fri-evidence-layer-len } \text{composition-roots } j)$
 $(\text{fri-evidence-layer-idx } \text{composition-roots } \text{query-idxs } i j)$
 $(2 \wedge j)$
 $(\text{fri-sibling-index } (\text{fri-evidence-layer-len } \text{composition-roots } j)$
 $(\text{fri-evidence-layer-idx } \text{composition-roots } \text{query-idxs } i j))$
 $xp \text{ } xp\text{-path } xn \text{ } xn\text{-path}$
 $(\text{fri-evidence-next-idx } \text{composition-roots } \text{query-idxs } i j)$
 v
 $(\text{composition-round-layers} ! i ! j)$
shows
 $\exists xp\text{-path}' \text{ } xn' \text{ } xn\text{-path}' \text{ } \text{next-value}.$
 $\text{fri-layer-step-evidence } (\text{composition-roots} ! \text{Suc } j)$
 $(\text{composition-bs} ! \text{Suc } j)$
 $(\text{fri-evidence-layer-len } \text{composition-roots } (\text{Suc } j))$
 $(\text{fri-evidence-layer-idx } \text{composition-roots } \text{query-idxs } i (\text{Suc } j))$
 $(2 \wedge \text{Suc } j)$
 $(\text{fri-sibling-index}$
 $(\text{fri-evidence-layer-len } \text{composition-roots } (\text{Suc } j))$
 $(\text{fri-evidence-layer-idx } \text{composition-roots } \text{query-idxs } i$
 $(\text{Suc } j)))$
 $v \text{ } xp\text{-path}' \text{ } xn' \text{ } xn\text{-path}'$
 $(\text{fri-evidence-next-idx } \text{composition-roots } \text{query-idxs } i (\text{Suc } j))$
 next-value
 $(\text{composition-round-layers} ! i ! \text{Suc } j)$
proof –
from *fri-openings* **obtain** $\text{result}' \text{ } \text{final-state}'$ fr *f-fl* as *fl* *query-state*
 $\text{raw-idxs } \text{query-chunks}$ **where**
 $\text{out-def: out} = \text{Some } (\text{result}', \text{final-state}')$
and *trace-roots-eq*: $\text{trace-roots} = \text{map } \text{snd } f\text{-fl}$
and *composition-roots-eq*: $\text{composition-roots} = \text{map } \text{snd } fl$
and *composition-bs-eq*: $\text{composition-bs} = \text{map } \text{fst } fl$
and *outcome*:
 $\text{Some } (\text{result}', \text{final-state}') \in$
 $\text{set-dist } (\text{execute}$
 $(\text{ntimes}$
 $(\text{verifier-query-round-program } fr \text{ } f\text{-fl } \text{trace-final } \text{as } fl$
 $\text{composition-final})$
 $\text{rounds})$
 $\text{query-state})$
and *len-raw*: $\text{length } \text{raw-idxs} = \text{rounds}$
and *query-idxs-eq*:

```

    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state'
and query-chunk:
    ∧i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)
and replay-lookup:
    ∧i. i < rounds ⇒
      fmllookup (HashMap final-state')
        (QueryIndexChallenge (PQueryCounter query-state + i)
          (state-after-query-chunks (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
and recorded:
    ∧i. i < rounds ⇒
      query-round-fri-layer-transcripts (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)
      (trace-round-layers ! i) (composition-round-layers ! i)
unfolding accepted-fri-opening-transcript-def by blast
have result-eq: result' = result
using out-def out-eq by simp
have final-state-eq: final-state' = final-state
using out-def out-eq by simp
have outcome':
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
using outcome result-eq final-state-eq by simp
let ?round =
  verifier-query-round-program fr f-fl trace-final as fl composition-final
from query-opening-ntimes-outcome-decomp-at[OF outcome' round-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist (execute (ntimes ?round i) query-state)
and round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
    Some (suffix, final-state) ∈
      set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
by blast
have x-eq: x = ()
by (cases x) simp

```



```

show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
fix prefix-query-idxs prefix-chunks
assume len-prefix-idxs: length prefix-query-idxs = i
and len-prefix-chunks: length prefix-chunks = i
and transcript-prefix:
  PTranscript query-state =
    List.concat prefix-chunks @ PTranscript s-i
and state-i:
  PState s-i =
    state-after-query-chunks (PState query-state) prefix-chunks i
and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
and prefix-ext: query-state ≤ s-i
and prefix-rounds:
   $\bigwedge k. k < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
from round[unfolded x-eq verifier-query-round-program-alt-def]
obtain raw s0 where
  raw-out:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s-i)
and after:
  Some ((), s-suc) ∈
    set-dist (execute
      (verifier-query-round-after-index-program fr f-fl trace-final as
        fl composition-final raw) s0)
by (auto elim!: set-dist-bindE)
have tr-s0: PTranscript s0 = PTranscript s-i
using receive-query-index-challenge-outcome[OF raw-out] by simp
show ?thesis
proof (rule verifier-query-round-program-outcome[OF round[unfolded x-eq]])
fix raw' idx chunk
assume idx-eq: idx = index (to-nat raw')
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
and transcript-round:
  PTranscript s-i = chunk @ PTranscript s-suc
and state-suc:
  PState s-suc = foldl concat (PState s-i) chunk
and round-ext: s-i ≤ s-suc
and counter-suc:
  PQueryCounter s-suc = Suc (PQueryCounter s-i)
and lookup-suc:
  fmllookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw'

```

```

have raw-eq': raw = raw'
proof -
  have lookup-s0:
    fmlookup (HashMap s0)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw
    using receive-query-index-challenge-outcome[OF raw-out] by simp
  have s0-suc: s0 ≤ s-suc
    using verifier-query-round-after-index-program-outcome[OF after]
    by simp
  have lookup-suc-raw:
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw
    using hash-extension-lookup[OF lookup-s0 s0-suc] .
  show ?thesis
    using lookup-suc lookup-suc-raw by simp
qed
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
  fix suffix-query-idxs suffix-chunks
  assume len-suffix-idxs:
    length suffix-query-idxs = rounds - Suc i
  and len-suffix-chunks:
    length suffix-chunks = rounds - Suc i
  and transcript-suffix:
    PTranscript s-suc =
      List.concat suffix-chunks @ PTranscript final-state
  and suffix-state:
    PState final-state =
      state-after-query-chunks (PState s-suc) suffix-chunks
      (rounds - Suc i)
  and suffix-counter:
    PQueryCounter final-state =
      PQueryCounter s-suc + (rounds - Suc i)
  and suffix-ext: s-suc ≤ final-state
  and suffix-rounds:
     $\bigwedge k. k < rounds - Suc i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
  let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
  have len-selected-chunks: length ?selected-chunks = rounds
    using len-prefix-chunks len-suffix-chunks round-bound by simp
  have selected-chunk-lens:
     $\bigwedge k. k < rounds \implies$ 
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)

```

```

proof —
  fix  $k$ 
  assume  $k\text{-bound}$ :  $k < \text{rounds}$ 
  show
     $\text{length } (?selected\text{-chunks } ! k) =$ 
       $\text{verifier-query-round-transcript-length } (query\text{-idxs } ! k)$ 
       $(\text{map } snd\ f\text{-fl}) (\text{map } snd\ fl)$ 
  proof ( $\text{cases } k < i$ )
    case  $True$ 
    have  $nth\text{-eq}$ :  $?selected\text{-chunks } ! k = \text{prefix-chunks } ! k$ 
      using  $True\ len\text{-prefix-chunks}$  by ( $\text{simp add: } nth\text{-append}$ )
    have
       $\text{length } (\text{prefix-chunks } ! k) =$ 
         $\text{verifier-query-round-transcript-length}$ 
         $(\text{prefix-query-idxs } ! k) (\text{map } snd\ f\text{-fl}) (\text{map } snd\ fl)$ 
      by ( $\text{rule } \text{verifier-query-round-chunk-length}$ 
         $[OF\ \text{prefix-rounds}[OF\ True]])$ 
    then show  $?thesis$ 
      using  $nth\text{-eq}$ 
      by ( $\text{simp add:}$ 
         $\text{verifier-query-round-transcript-length-index-irrelevant}$ )
  next
    case  $False$ 
    show  $?thesis$ 
    proof ( $\text{cases } k = i$ )
      case  $True$ 
      have  $nth\text{-eq}$ :  $?selected\text{-chunks } ! k = \text{chunk}$ 
        using  $True\ len\text{-prefix-chunks}$  by ( $\text{simp add: } nth\text{-append}$ )
      have
         $\text{length } \text{chunk} =$ 
           $\text{verifier-query-round-transcript-length } idx$ 
           $(\text{map } snd\ f\text{-fl}) (\text{map } snd\ fl)$ 
        by ( $\text{rule } \text{verifier-query-round-chunk-length}[OF\ \text{chunk-shape}]$ )
      then show  $?thesis$ 
        using  $nth\text{-eq}$ 
        by ( $\text{simp add:}$ 
           $\text{verifier-query-round-transcript-length-index-irrelevant}$ )
    next
      case  $False$ 
      let  $?k = k - Suc\ i$ 
      have  $ge$ :  $Suc\ i \leq k$ 
        using  $\langle \neg k < i \rangle\ False$  by  $linarith$ 
      have  $k\text{-bound}'$ :  $?k < \text{rounds} - Suc\ i$ 
        using  $ge\ k\text{-bound}$  by  $linarith$ 
      have  $nth\text{-eq}$ :  $?selected\text{-chunks } ! k = \text{suffix-chunks } ! ?k$ 
        using  $ge\ len\text{-prefix-chunks}$  by ( $\text{simp add: } nth\text{-append}$ )
      have
         $\text{length } (\text{suffix-chunks } ! ?k) =$ 
           $\text{verifier-query-round-transcript-length}$ 

```

```

      (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound']])
  then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-f:
   $\bigwedge k. k < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! k)
    (map snd f-fl) (map snd fl) (query-chunks ! k)
  using query-chunk trace-roots-eq composition-roots-eq by simp
have query-chunk-lens:
   $\bigwedge k. k < \text{rounds} \implies$ 
    length (query-chunks ! k) =
    verifier-query-round-transcript-length (query-idxs ! k)
    (map snd f-fl) (map snd fl)
  using query-chunk-f verifier-query-round-chunk-length by blast
have selected-transcript:
  PTranscript query-state =
    List.concat ?selected-chunks @ PTranscript final-state
  using transcript-prefix transcript-round transcript-suffix by simp
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query final-state-eq by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
     $\bigwedge k. k < \text{length } ?\text{selected-chunks} \implies$ 
      length (?selected-chunks ! k) = length (query-chunks ! k)
    using selected-chunk-lens query-chunk-lens len-selected-chunks
    by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
    using concat-eq by simp
  have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks  $\wedge$  ([] :: 'f list) = []
    by (rule concat-append-eq-concat-same-chunk-lengths
        [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
  state-after-query-chunks (PState query-state) query-chunks i
proof -

```

```

have
  state-after-query-chunks (PState query-state) ?selected-chunks i =
    state-after-query-chunks (PState query-state) prefix-chunks i
by (rule state-after-query-chunks-append-prefix
      [OF len-prefix-chunks])
then show ?thesis
  using chunks-eq by simp
qed
have lookup-final:
  fmllookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some raw'
proof -
  have lookup-suc':
    fmllookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
      Some raw'
    using lookup-suc state-i counter-i by simp
  have lookup-final-prefix:
    fmllookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
      Some raw'
    using hash-extension-lookup[OF lookup-suc' suffix-ext] .
  show ?thesis
    using lookup-final-prefix prefix-state-eq by simp
qed
have raw-idx-eq: raw' = raw-idxs ! i
  using lookup-final replay-lookup[OF round-bound] final-state-eq
  by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-idx-eq query-idxs-eq len-raw round-bound by simp
have selected-i: ?selected-chunks ! i = chunk
  using len-prefix-chunks by (simp add: nth-append)
have chunk-i-eq: chunk = query-chunks ! i
  using chunks-eq selected-i by simp
have recorded-i:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) chunk (trace-round-layers ! i)
    (composition-round-layers ! i)
  using recorded[OF round-bound] idx-eq idx-eq-query raw-eq'
    chunk-i-eq trace-roots-eq composition-roots-eq
  by simp
have tr-s0-suc:
  PTranscript s0 = chunk @ PTranscript s-suc

```

```

using tr-s0 transcript-round by simp
have suc-j-bound-fl: Suc j < length fl
  using layer-bound composition-roots-eq by simp
have all-layer-bounds-fl:
   $\bigwedge k. k < \text{length } fl \implies$ 
     $0 < \text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! k \wedge$ 
     $\text{fri-layer-indices } (\text{length } fl) (\text{index } (\text{to-nat } \text{raw}))$ 
     $(\text{clength} * \text{scale}) ! k <$ 
     $\text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! k$ 
  using all-layer-bounds composition-roots-eq by simp
have raw-idx-query:  $\text{index } (\text{to-nat } \text{raw}) = \text{query-idxs} ! i$ 
  using idx-eq idx-eq-query raw-eq' by simp
have previous-step-fl:
  fri-layer-step-evidence (snd (fl ! j)) (fst (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j)
    ( $2 \wedge j$ )
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! j))
    xp xp-path xn xn-path
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j mod
      (fri-layer-lengths (length fl) (clength * scale) ! j div 2))
    v
    (composition-round-layers ! i ! j)
  using previous-step composition-roots-eq composition-bs-eq
    raw-idx-query layer-bound
  unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
    fri-evidence-next-idx-def
  by simp
from verifier-query-round-after-index-composition-successor-recorded-step-from-previous
  [OF after recorded-i tr-s0-suc suc-j-bound-fl
    all-layer-bounds-fl previous-step-fl]
obtain sxp-path srxn srxn-path snext where succ-fl:
  fri-layer-step-evidence (snd (fl ! Suc j))
    (fst (fl ! Suc j))
    (fri-layer-lengths (length fl) (clength * scale) ! Suc j)
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! Suc j)
    ( $2 \wedge \text{Suc } j$ )
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) ! Suc j)
      (fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! Suc j))
    v sxp-path srxn srxn-path
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j mod
      (fri-layer-lengths (length fl) (clength * scale) ! j div 2))

```

```

      (clength * scale) ! Suc j mod
      (fri-layer-lengths (length fl) (clength * scale) ! Suc j
        div 2))
    snext
    (composition-round-layers ! i ! Suc j)
  by blast
have succ:
  fri-layer-step-evidence (composition-roots ! Suc j)
    (composition-bs ! Suc j)
    (fri-evidence-layer-len composition-roots (Suc j))
    (fri-evidence-layer-idx composition-roots query-idxs i
      (Suc j))
    (2 ^ Suc j)
    (fri-sibling-index
      (fri-evidence-layer-len composition-roots (Suc j))
      (fri-evidence-layer-idx composition-roots query-idxs i
        (Suc j)))
    v sxp-path sn sn-path
    (fri-evidence-next-idx composition-roots query-idxs i (Suc j))
    snext
    (composition-round-layers ! i ! Suc j)
  using succ-fl composition-roots-eq composition-bs-eq
    raw-idx-query layer-bound
  unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
    fri-evidence-next-idx-def
  by simp
show ?thesis
  using succ by blast
qed
qed
qed
qed

lemma generic-fri-sampled-successor-conflict-imp-same-layer-with-source-steps:
  assumes successor:
    generic-fri-sampled-successor-opening-conflict roots challenges
    query-idxs round-layers
  and len-challenges: length challenges = length roots
  and source-step:
     $\bigwedge_{\text{source-round layer-idx } v.}$ 
    source-round < length query-idxs  $\implies$ 
    Suc layer-idx < length challenges  $\implies$ 
    generic-fri-round-forced-next-value roots challenges query-idxs
    round-layers source-round layer-idx v  $\implies$ 
     $\exists xp\text{-path } xn \text{ } xn\text{-path.}$ 
    fri-layer-step-evidence
      (roots ! Suc layer-idx)
      (challenges ! Suc layer-idx)
      (fri-evidence-layer-len roots (Suc layer-idx))

```

(fri-evidence-layer-idx roots query-idxs source-round
 (Suc layer-idx))
 (2 $\wedge$ Suc layer-idx)
 (fri-sibling-index
 (fri-evidence-layer-len roots (Suc layer-idx))
 (fri-evidence-layer-idx roots query-idxs source-round
 (Suc layer-idx)))
 v xp-path xn xn-path
 (fri-evidence-next-idx roots query-idxs source-round
 (Suc layer-idx))
 (fri-evidence-next-value roots challenges query-idxs
 source-round (Suc layer-idx) v xn)
 (round-layers ! source-round ! Suc layer-idx)

shows

generic-fri-sampled-same-layer-opening-conflict roots challenges
 query-idxs round-layers

proof –

from generic-fri-sampled-successor-opening-conflictE[OF successor]

obtain source-round target-round layer-idx v xp xp-path xn xn-path

where source-bound: source-round < length query-idxs
and target-bound: target-round < length query-idxs
and layer-bound: Suc layer-idx < length challenges
and forced:

generic-fri-round-forced-next-value roots challenges query-idxs
 round-layers source-round layer-idx v

and target-step:

fri-layer-step-evidence
 (roots ! Suc layer-idx)
 (challenges ! Suc layer-idx)
 (fri-evidence-layer-len roots (Suc layer-idx))
 (fri-evidence-layer-idx roots query-idxs target-round
 (Suc layer-idx))
 (2 $\wedge$ Suc layer-idx)
 (fri-sibling-index
 (fri-evidence-layer-len roots (Suc layer-idx))
 (fri-evidence-layer-idx roots query-idxs target-round
 (Suc layer-idx)))
 xp xp-path xn xn-path
 (fri-evidence-next-idx roots query-idxs target-round
 (Suc layer-idx))
 (fri-evidence-next-value roots challenges query-idxs target-round
 (Suc layer-idx) xp xn)
 (round-layers ! target-round ! Suc layer-idx)

and conflict:

(fri-evidence-next-idx roots query-idxs source-round layer-idx =
 fri-evidence-layer-idx roots query-idxs target-round
 (Suc layer-idx) $\wedge$
 v $\neq$ xp) $\vee$
 (fri-evidence-next-idx roots query-idxs source-round layer-idx =

fri-sibling-index
 (fri-evidence-layer-len roots (Suc layer-idx))
 (fri-evidence-layer-idx roots query-idxs target-round
 (Suc layer-idx)) $\wedge$
 $v \neq xn$)
by blast
from source-step[OF source-bound layer-bound forced]
obtain vp-path vn vn-path **where** source-suc-step:
fri-layer-step-evidence
 (roots ! Suc layer-idx)
 (challenges ! Suc layer-idx)
 (fri-evidence-layer-len roots (Suc layer-idx))
 (fri-evidence-layer-idx roots query-idxs source-round
 (Suc layer-idx))
 ($2 \wedge$ Suc layer-idx)
 (fri-sibling-index
 (fri-evidence-layer-len roots (Suc layer-idx))
 (fri-evidence-layer-idx roots query-idxs source-round
 (Suc layer-idx)))
 v vp-path vn vn-path
 (fri-evidence-next-idx roots query-idxs source-round
 (Suc layer-idx))
 (fri-evidence-next-value roots challenges query-idxs source-round
 (Suc layer-idx) v vn)
 (round-layers ! source-round ! Suc layer-idx)
by blast
have source-idx-suc:
 fri-evidence-layer-idx roots query-idxs source-round (Suc layer-idx) =
 fri-evidence-next-idx roots query-idxs source-round layer-idx
by (rule fri-evidence-layer-idx-Suc)
 (use layer-bound len-challenges **in** simp)
have layer-suc-bound: Suc layer-idx < length challenges
using layer-bound .
have same-conflict:
 (fri-evidence-layer-idx roots query-idxs source-round (Suc layer-idx) =
 fri-evidence-layer-idx roots query-idxs target-round
 (Suc layer-idx)) $\wedge$
 $v \neq xp$) $\vee$
 (fri-evidence-layer-idx roots query-idxs source-round (Suc layer-idx) =
 fri-sibling-index
 (fri-evidence-layer-len roots (Suc layer-idx))
 (fri-evidence-layer-idx roots query-idxs target-round
 (Suc layer-idx)) $\wedge$
 $v \neq xn$)
proof –
from conflict **show** ?thesis
proof
assume c:
 fri-evidence-next-idx roots query-idxs source-round layer-idx =

```

    fri-evidence-layer-idx roots query-idxs target-round
      (Suc layer-idx) ∧
    v ≠ xp
  then show ?thesis
    using source-idx-suc by simp
next
  assume c:
    fri-evidence-next-idx roots query-idxs source-round layer-idx =
      fri-sibling-index
      (fri-evidence-layer-len roots (Suc layer-idx))
      (fri-evidence-layer-idx roots query-idxs target-round
        (Suc layer-idx)) ∧
    v ≠ xn
  then show ?thesis
    using source-idx-suc by simp
qed
qed
show ?thesis
  unfolding generic-fri-sampled-same-layer-opening-conflict-def
  apply (rule exI[where x=source-round])
  apply (rule exI[where x=target-round])
  apply (rule exI[where x=Suc layer-idx])
  apply (rule exI[where x=v])
  apply (rule exI[where x=vp-path])
  apply (rule exI[where x=vn])
  apply (rule exI[where x=vn-path])
  apply (rule exI[where x=xp])
  apply (rule exI[where x=xp-path])
  apply (rule exI[where x=xn])
  apply (rule exI[where x=xn-path])
  apply (intro conjI)
  using source-bound target-bound layer-suc-bound source-suc-step
    target-step same-conflict
  by blast+
qed

lemma generic-fri-sampled-same-layer-conflict-imp-merkle-or-slot-gap:
  assumes conflict:
    generic-fri-sampled-same-layer-opening-conflict roots challenges
      query-idxs round-layers
  and len-challenges: length challenges = length roots
  and auth-step:
    ∧ round-idx layer-idx.
      round-idx < length query-idxs ⇒
      layer-idx < length challenges ⇒
      generic-fri-transcript-step-with-authenticated-chunk roots
        challenges query-idxs round-layers final-state round-idx layer-idx
  shows
    partial-merkle-inconsistency-bad s (Some (result, final-state)) ∨

```

generic-fri-authenticated-slot-recorded-chunk-gap roots query-idxs
round-layers final-state

proof –

from *generic-fri-sampled-same-layer-imp-authenticated-or-recorded-auth-gap*
[OF conflict, of final-state]

consider
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state
| *generic-fri-sampled-same-layer-recorded-auth-gap roots challenges*
query-idxs round-layers final-state
by *blast*

then show *?thesis*

proof *cases*

case *1*

then have *partial-merkle-inconsistency-bad s (Some (result, final-state))*
by *(rule*
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-partial-merkle)
(rule len-challenges)

then show *?thesis* **by** *simp*

next

case *2*

then have *auth-gap:*
generic-fri-sampled-same-layer-authenticated-recorded-auth-gap roots
challenges query-idxs round-layers final-state
by *(rule generic-fri-recorded-auth-gap-imp-authenticated-recorded-auth-gap)*
(rule auth-step)

have *generic-fri-authenticated-slot-recorded-chunk-gap roots query-idxs*
round-layers final-state
by *(rule generic-fri-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap*
[OF auth-gap])

then show *?thesis* **by** *simp*

qed

qed

lemma *composition-fri-verifier-tied-next-value-replay-gap-imp-merkle-or-slot-on-support:*
assumes *support: out ∈ set-dist (execute verify-monad s)*
and *gap:*
composition-fri-verifier-tied-next-value-replay-gap s out
shows
partial-merkle-inconsistency-bad s out ∨
composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
s out

proof –

from *gap* **have** *sampled:*
composition-fri-verifier-tied-sampled-next-value-conflict s out
unfolding *composition-fri-verifier-tied-next-value-replay-gap-def*
by *simp*

from *composition-fri-verifier-tied-sampled-next-value-conflictE [OF sampled]*
obtain *trace-roots trace-bs trace-final fri-dg composition-roots*

```

composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table round-idx round-idx' layer-idx v v'
where fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers
and aligned:
  accepted-with-partial-initial-openings-aligned s out fr trace-roots
  trace-final as fri-dg composition-roots composition-final
  fri-query-idxs trace-openings composition-openings
and trace-cand: partial-trace-table-candidate trace-table trace-openings
and comp-cand:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-not-low:
  ¬ composition-table-low-degree maxDegree composition-table
and round-bound: round-idx < length fri-query-idxs
and round-bound': round-idx' < length fri-query-idxs
and layer-bound: layer-idx < length composition-bs
and same-idx:
  fri-evidence-next-idx composition-roots fri-query-idxs round-idx
  layer-idx =
  fri-evidence-next-idx composition-roots fri-query-idxs round-idx'
  layer-idx
and forced:
  generic-fri-round-forced-next-value composition-roots composition-bs
  fri-query-idxs composition-round-layers round-idx layer-idx v
and forced':
  generic-fri-round-forced-next-value composition-roots composition-bs
  fri-query-idxs composition-round-layers round-idx' layer-idx v'
and neq: v ≠ v'
using sampled
by (elim composition-fri-verifier-tied-sampled-next-value-conflictE)
from fri-openings obtain result final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-fri-opening-transcript-def by blast
have support-some:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
using support out-eq by simp
have challenges:
  accepted-fri-challenges s (Some (result, final-state))
  trace-bs fri-dg composition-bs
using accepted-fri-opening-transcript-challenges[OF fri-openings]
  out-eq by simp
have degree-bound: to-nat fri-dg ≤ maxDegree
by (rule verify-monad-accepted-fri-challenges-degree-bound
  [OF support-some challenges])

```

```

have len-bs-roots: length composition-bs = length composition-roots
  using accepted-fri-opening-transcript-shapes(4)[OF fri-openings] .
have layer-bound-roots: layer-idx < length composition-roots
  using layer-bound len-bs-roots by simp
have len-query: length fri-query-idxs = rounds
  using accepted-fri-opening-transcript-shapes(7)[OF fri-openings] .
have round-bound-rounds: round-idx < rounds
  using round-bound len-query by simp
have round-bound-rounds': round-idx' < rounds
  using round-bound' len-query by simp
have raw-bound:
  fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
  layer-idx <
  fri-evidence-layer-len composition-roots layer-idx
by (rule
  accepted-fri-opening-transcript-composition-evidence-layer-idx-bound
  [OF fri-openings degree-bound round-bound layer-bound-roots])
have raw-bound':
  fri-evidence-layer-idx composition-roots fri-query-idxs round-idx'
  layer-idx <
  fri-evidence-layer-len composition-roots layer-idx
by (rule
  accepted-fri-opening-transcript-composition-evidence-layer-idx-bound
  [OF fri-openings degree-bound round-bound' layer-bound-roots])
have even-len:
  2 dvd fri-evidence-layer-len composition-roots layer-idx
by (rule accepted-fri-opening-transcript-composition-layer-arithmetic(1)
  [OF fri-openings degree-bound layer-bound])
have round-product:
  fri-evidence-layer-len composition-roots layer-idx * 2 ^ layer-idx =
  clength * scale
by (rule accepted-fri-opening-transcript-composition-layer-arithmetic(2)
  [OF fri-openings degree-bound layer-bound])
have auth-step:
   $\bigwedge i j.$ 
   $i < \text{length } \text{fri-query-idxs} \implies$ 
   $j < \text{length } \text{composition-bs} \implies$ 
  generic-fri-transcript-step-with-authenticated-chunk composition-roots
  composition-bs fri-query-idxs composition-round-layers final-state i j
proof -
  fix i j
  assume i-bound:  $i < \text{length } \text{fri-query-idxs}$ 
  and j-bound:  $j < \text{length } \text{composition-bs}$ 
  have i-rounds:  $i < \text{rounds}$ 
  using i-bound len-query by simp
  have j-roots:  $j < \text{length } \text{composition-roots}$ 
  using j-bound len-bs-roots by simp
  show
    generic-fri-transcript-step-with-authenticated-chunk

```

```

    composition-roots composition-bs fri-query-idxs
    composition-round-layers final-state i j
  by (rule
    accepted-fri-opening-transcript-composition-step-with-authenticated-chunk-at-raw
    [OF fri-openings out-eq degree-bound i-rounds j-roots])
qed
show ?thesis
proof (cases
  generic-fri-sampled-same-layer-opening-conflict composition-roots
  composition-bs fri-query-idxs composition-round-layers)
case True
have conflict-route:
  partial-merkle-inconsistency-bad s (Some (result, final-state)) ∨
  generic-fri-authenticated-slot-recorded-chunk-gap composition-roots
  fri-query-idxs composition-round-layers final-state
by (rule generic-fri-sampled-same-layer-conflict-imp-merkle-or-slot-gap
  [OF True len-bs-roots auth-step])
then show ?thesis
proof
  assume partial-merkle-inconsistency-bad s (Some (result, final-state))
  then show ?thesis
    using out-eq by simp
next
  assume slot-gap:
    generic-fri-authenticated-slot-recorded-chunk-gap composition-roots
    fri-query-idxs composition-round-layers final-state
  have route-slot-gap:
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
    s out
  unfolding
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
    rule comp-cand, rule trace-low, rule comp-not-low, rule slot-gap)
  then show ?thesis by simp
qed
next
case False
then have no-same:
  ¬ generic-fri-sampled-same-layer-opening-conflict
  composition-roots composition-bs fri-query-idxs
  composition-round-layers
  by simp
have v = v'
  by (rule generic-fri-forced-next-value-eq-if-no-same-layer-conflict
    [OF no-same round-bound round-bound' layer-bound even-len
    raw-bound raw-bound' round-product same-idx forced forced'])
then show ?thesis
  using neq by simp

```

qed
qed

lemma *composition-fri-verifier-tied-successor-replay-gap-imp-merkle-or-slot-on-support:*

assumes *support: out ∈ set-dist (execute verify-monad s)*

and *gap:*

composition-fri-verifier-tied-successor-replay-gap s out

shows

partial-merkle-inconsistency-bad s out ∨

*composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
s out*

proof —

from *gap* **have** *sampled:*

composition-fri-verifier-tied-sampled-successor-opening-conflict s out

unfolding *composition-fri-verifier-tied-successor-replay-gap-def*

by *simp*

obtain *trace-roots trace-bs trace-final fri-dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as trace-openings composition-openings

trace-table composition-table source-round target-round layer-idx v

xp xp-path xn xn-path

where *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers

and *aligned:*

accepted-with-partial-initial-openings-aligned s out fr trace-roots

trace-final as fri-dg composition-roots composition-final

fri-query-idxs trace-openings composition-openings

and *trace-cand: partial-trace-table-candidate trace-table trace-openings*

and *comp-cand:*

partial-composition-table-candidate composition-table

composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *comp-not-low:*

$\neg$ *composition-table-low-degree maxDegree composition-table*

and *source-bound: source-round < length fri-query-idxs*

and *target-bound: target-round < length fri-query-idxs*

and *layer-bound: Suc layer-idx < length composition-bs*

and *forced:*

generic-fri-round-forced-next-value composition-roots composition-bs

fri-query-idxs composition-round-layers source-round layer-idx v

and *target-step:*

fri-layer-step-evidence

(composition-roots ! Suc layer-idx)

(composition-bs ! Suc layer-idx)

(fri-evidence-layer-len composition-roots (Suc layer-idx))

(fri-evidence-layer-idx composition-roots fri-query-idxs

target-round (Suc layer-idx))

```

(2 ^ Suc layer-idx)
(fri-sibling-index
 (fri-evidence-layer-len composition-roots (Suc layer-idx))
 (fri-evidence-layer-idx composition-roots fri-query-idxs
  target-round (Suc layer-idx)))
xp xp-path xn xn-path
(fri-evidence-next-idx composition-roots fri-query-idxs
 target-round (Suc layer-idx))
(fri-evidence-next-value composition-roots composition-bs
 fri-query-idxs target-round (Suc layer-idx) xp xn)
(composition-round-layers ! target-round ! Suc layer-idx)
and conflict:
(fri-evidence-next-idx composition-roots fri-query-idxs source-round
 layer-idx =
 fri-evidence-layer-idx composition-roots fri-query-idxs target-round
 (Suc layer-idx) ^
 v ≠ xp) ∨
(fri-evidence-next-idx composition-roots fri-query-idxs source-round
 layer-idx =
 fri-sibling-index
 (fri-evidence-layer-len composition-roots (Suc layer-idx))
 (fri-evidence-layer-idx composition-roots fri-query-idxs
  target-round (Suc layer-idx)) ^
 v ≠ xn)
using sampled
by (elim composition-fri-verifier-tied-sampled-successor-opening-conflictE)
from fri-openings obtain result final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-fri-opening-transcript-def by blast
have support-some:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
using support out-eq by simp
have challenges:
  accepted-fri-challenges s (Some (result, final-state))
  trace-bs fri-dg composition-bs
using accepted-fri-opening-transcript-challenges[OF fri-openings]
  out-eq by simp
have degree-bound: to-nat fri-dg ≤ maxDegree
by (rule verify-monad-accepted-fri-challenges-degree-bound
 [OF support-some challenges])
have len-bs-roots: length composition-bs = length composition-roots
using accepted-fri-opening-transcript-shapes(4)[OF fri-openings] .
have len-query: length fri-query-idxs = rounds
using accepted-fri-opening-transcript-shapes(7)[OF fri-openings] .
have sampled-generic:
  generic-fri-sampled-successor-opening-conflict composition-roots
  composition-bs fri-query-idxs composition-round-layers
unfolding generic-fri-sampled-successor-opening-conflict-def
by (intro exI conjI)

```


(rule source-bound, rule target-bound, rule layer-bound,
 rule forced, rule target-step, rule conflict)

have *all-layer-bounds*:

$\bigwedge \text{raw } k. k < \text{length composition-roots} \implies$
 $0 < \text{fri-layer-lengths } (\text{length composition-roots})$
 $(\text{clength} * \text{scale}) ! k \wedge$
 $\text{fri-layer-indices } (\text{length composition-roots}) (\text{index } (\text{to-nat raw}))$
 $(\text{clength} * \text{scale}) ! k <$
 $\text{fri-layer-lengths } (\text{length composition-roots}) (\text{clength} * \text{scale}) ! k$

by (rule accepted-fri-opening-transcript-composition-raw-layer-bound
 [OF fri-openings degree-bound])

have *source-step*:

$\bigwedge \text{source-round layer-idx } v.$
 $\text{source-round} < \text{length fri-query-idxs} \implies$
 $\text{Suc layer-idx} < \text{length composition-bs} \implies$
 $\text{generic-fri-round-forced-next-value composition-roots composition-bs}$
 $\text{fri-query-idxs composition-round-layers source-round layer-idx } v \implies$
 $\exists \text{xp-path } \text{xn } \text{xn-path}.$
 $\text{fri-layer-step-evidence}$
 $(\text{composition-roots} ! \text{Suc layer-idx})$
 $(\text{composition-bs} ! \text{Suc layer-idx})$
 $(\text{fri-evidence-layer-len composition-roots } (\text{Suc layer-idx}))$
 $(\text{fri-evidence-layer-idx composition-roots fri-query-idxs}$
 $\text{source-round } (\text{Suc layer-idx}))$
 $(2 \wedge \text{Suc layer-idx})$
 $(\text{fri-sibling-index}$
 $(\text{fri-evidence-layer-len composition-roots } (\text{Suc layer-idx}))$
 $(\text{fri-evidence-layer-idx composition-roots fri-query-idxs}$
 $\text{source-round } (\text{Suc layer-idx})))$
 $v \text{ xp-path } \text{xn } \text{xn-path}$
 $(\text{fri-evidence-next-idx composition-roots fri-query-idxs}$
 $\text{source-round } (\text{Suc layer-idx}))$
 $(\text{fri-evidence-next-value composition-roots composition-bs}$
 $\text{fri-query-idxs source-round } (\text{Suc layer-idx}) v \text{ xn})$
 $(\text{composition-round-layers} ! \text{source-round} ! \text{Suc layer-idx})$

proof –

fix *source-round layer-idx v*

assume *source-round-bound*: *source-round* < *length fri-query-idxs*

and *layer-bound'*: *Suc layer-idx* < *length composition-bs*

and *forced'*:

$\text{generic-fri-round-forced-next-value composition-roots composition-bs}$
 $\text{fri-query-idxs composition-round-layers source-round layer-idx } v$

have *source-round-rounds*: *source-round* < *rounds*

using *source-round-bound len-query* **by** *simp*

have *layer-bound-roots*: *Suc layer-idx* < *length composition-roots*

using *layer-bound' len-bs-roots* **by** *simp*

from *forced'* **obtain** *fxp fxp-path fxn fxn-path* **where** *previous-step*:

$\text{fri-layer-step-evidence } (\text{composition-roots} ! \text{layer-idx})$
 $(\text{composition-bs} ! \text{layer-idx})$

```

(fri-evidence-layer-len composition-roots layer-idx)
(fri-evidence-layer-idx composition-roots fri-query-idxs
 source-round layer-idx)
(2 ^ layer-idx)
(fri-sibling-index
 (fri-evidence-layer-len composition-roots layer-idx)
 (fri-evidence-layer-idx composition-roots fri-query-idxs
 source-round layer-idx))
fxp fxp-path fxn fxn-path
(fri-evidence-next-idx composition-roots fri-query-idxs
 source-round layer-idx)
v
(composition-round-layers ! source-round ! layer-idx)
unfolding generic-fri-round-forced-next-value-def by blast
from accepted-fri-opening-transcript-composition-successor-recorded-step-from-previous-at
[OF fri-openings out-eq source-round-rounds layer-bound-roots
 all-layer-bounds previous-step]
obtain sxp-path sxn sxn-path snext where succ:
fri-layer-step-evidence (composition-roots ! Suc layer-idx)
 (composition-bs ! Suc layer-idx)
(fri-evidence-layer-len composition-roots (Suc layer-idx))
(fri-evidence-layer-idx composition-roots fri-query-idxs
 source-round (Suc layer-idx))
(2 ^ Suc layer-idx)
(fri-sibling-index
 (fri-evidence-layer-len composition-roots (Suc layer-idx))
 (fri-evidence-layer-idx composition-roots fri-query-idxs
 source-round (Suc layer-idx)))
v sxp-path sxn sxn-path
(fri-evidence-next-idx composition-roots fri-query-idxs
 source-round (Suc layer-idx))
snext
(composition-round-layers ! source-round ! Suc layer-idx)
by blast
have snext-eq:
snext =
fri-evidence-next-value composition-roots composition-bs
 fri-query-idxs source-round (Suc layer-idx) v sxn
using fri-layer-step-evidenceD(3)[OF succ]
unfolding fri-evidence-next-value-def by simp
show
∃ xp-path xn xn-path.
fri-layer-step-evidence
 (composition-roots ! Suc layer-idx)
 (composition-bs ! Suc layer-idx)
(fri-evidence-layer-len composition-roots (Suc layer-idx))
(fri-evidence-layer-idx composition-roots fri-query-idxs
 source-round (Suc layer-idx))
(2 ^ Suc layer-idx)

```

```

    (fri-sibling-index
      (fri-evidence-layer-len composition-roots (Suc layer-idx))
      (fri-evidence-layer-idx composition-roots fri-query-idxs
        source-round (Suc layer-idx)))
    v xp-path xn xn-path
    (fri-evidence-next-idx composition-roots fri-query-idxs
      source-round (Suc layer-idx))
    (fri-evidence-next-value composition-roots composition-bs
      fri-query-idxs source-round (Suc layer-idx) v xn)
    (composition-round-layers ! source-round ! Suc layer-idx)
  using succ snext-eq by blast
qed
have same-layer:
  generic-fri-sampled-same-layer-opening-conflict composition-roots
    composition-bs fri-query-idxs composition-round-layers
  by (rule generic-fri-sampled-successor-conflict-imp-same-layer-with-source-steps
    [OF sampled-generic len-bs-roots source-step])
have auth-step:
   $\bigwedge i j.$ 
   $i < \text{length } \text{fri-query-idxs} \implies$ 
   $j < \text{length } \text{composition-bs} \implies$ 
  generic-fri-transcript-step-with-authenticated-chunk composition-roots
    composition-bs fri-query-idxs composition-round-layers final-state i j
proof -
  fix i j
  assume i-bound:  $i < \text{length } \text{fri-query-idxs}$ 
  and j-bound:  $j < \text{length } \text{composition-bs}$ 
  have i-rounds:  $i < \text{rounds}$ 
  using i-bound len-query by simp
  have j-roots:  $j < \text{length } \text{composition-roots}$ 
  using j-bound len-bs-roots by simp
  show
    generic-fri-transcript-step-with-authenticated-chunk
      composition-roots composition-bs fri-query-idxs
      composition-round-layers final-state i j
  by (rule
    accepted-fri-opening-transcript-composition-step-with-authenticated-chunk-at-raw
    [OF fri-openings out-eq degree-bound i-rounds j-roots])
qed
have route:
  partial-merkle-inconsistency-bad s (Some (result, final-state))  $\vee$ 
  generic-fri-authenticated-slot-recorded-chunk-gap composition-roots
    fri-query-idxs composition-round-layers final-state
  by (rule generic-fri-sampled-same-layer-conflict-imp-merkle-or-slot-gap
    [OF same-layer len-bs-roots auth-step])
then show ?thesis
proof
  assume partial-merkle-inconsistency-bad s (Some (result, final-state))
  then show ?thesis

```

```

    using out-eq by simp
next
assume slot-gap:
  generic-fri-authenticated-slot-recorded-chunk-gap composition-roots
  fri-query-idxs composition-round-layers final-state
have route-slot-gap:
  composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
  s out
unfolding
  composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-def
by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
   rule comp-cand, rule trace-low, rule comp-not-low, rule slot-gap)
then show ?thesis by simp
qed
qed

lemma wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-merkle-and-slot:
  fixes M A :: prob
  assumes merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M
  and slot-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
      s ≤ A
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-successor-replay-gap s) s
      ≤ M + A
proof -
  let ?P = partial-merkle-inconsistency-bad s
  let ?Q = composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s
  have event-le:
    wp-event verify-monad
      (composition-fri-verifier-tied-successor-replay-gap s) s ≤
      wp-event verify-monad (λout. ?P out ∨ ?Q out) s
  by (rule wp-event-mono-on-support)
  (rule
    composition-fri-verifier-tied-successor-replay-gap-imp-merkle-or-slot-on-support)
  also have ... ≤ wp-event verify-monad ?P s +
    wp-event verify-monad ?Q s
  by (rule wp-event-union-bound)
  also have ... ≤ M + A
  by (intro add-mono merkle-bound slot-bound)
  finally show ?thesis .
qed

lemma wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-merkle-and-slot:
  fixes M A :: prob

```

```

assumes merkle-bound:
  wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M
and slot-bound:
  wp-event verify-monad
    (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
    s ≤ A
shows
  wp-event verify-monad
    (composition-fri-verifier-tied-next-value-replay-gap s) s
    ≤ M + A
proof –
  let ?P = partial-merkle-inconsistency-bad s
  let ?Q = composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s
  have event-le:
    wp-event verify-monad
      (composition-fri-verifier-tied-next-value-replay-gap s) s ≤
      wp-event verify-monad (λout. ?P out ∨ ?Q out) s
  by (rule wp-event-mono-on-support)
    (rule
      composition-fri-verifier-tied-next-value-replay-gap-imp-merkle-or-slot-on-support)
  also have ... ≤ wp-event verify-monad ?P s +
    wp-event verify-monad ?Q s
  by (rule wp-event-union-bound)
  also have ... ≤ M + A
  by (intro add-mono merkle-bound slot-bound)
  finally show ?thesis .
qed

end

end

```

```

theory Soundness-FRI-Trace-Replay-Gaps
imports
  Soundness-FRI-Trace-Residuals
  Soundness-FRI-Composition-Replay-Gaps
begin

```

Narrow downstream layer for trace header-tied replay gaps. This mirrors the composition replay-gap layer without growing the already-large trace residual theory.

```

context soundness
begin

```

```

lemma accepted-fri-opening-transcript-trace-layer-arithmetic:
assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers

```

```

    and layer-bound: layer-idx < length trace-bs
  shows 2 dvd fri-evidence-layer-len trace-roots layer-idx
    and fri-evidence-layer-len trace-roots layer-idx * 2 ^ layer-idx =
      clength * scale
proof -
  obtain N where eval-power: clength * scale = 2 ^ N
    using eval-domain-length-power by blast
  have bs-len: length trace-bs = length trace-roots
    using accepted-fri-opening-transcript-shapes(2)[OF fri-openings] .
  have roots-bound: layer-idx < length trace-roots
    using layer-bound bs-len by simp
  have roots-le: length trace-roots ≤ N
proof -
  have len: length trace-roots = ceil-log clength
    using accepted-fri-opening-transcript-shapes(1)[OF fri-openings] .
  have clength ≤ clength * scale
    using scale-pos by simp
  also have ... = 2 ^ N
    by (rule eval-power)
  finally have ceil-log clength ≤ N
    by (rule ceil-log-le-power)
  then show ?thesis
    using len by simp
qed
have suc-le: Suc layer-idx ≤ N
  using roots-bound roots-le by simp
have dvd-eval-suc: (2::nat) ^ Suc layer-idx dvd clength * scale
  using le-imp-power-dvd[OF suc-le] eval-power by simp
show 2 dvd fri-evidence-layer-len trace-roots layer-idx
  unfolding fri-evidence-layer-len-def
  by (rule fri-layer-lengths-even-if-dvd[OF roots-bound dvd-eval-suc])
have le-N: layer-idx ≤ N
  using roots-bound roots-le by simp
have dvd-eval: (2::nat) ^ layer-idx dvd clength * scale
  using le-imp-power-dvd[OF le-N] eval-power by simp
show fri-evidence-layer-len trace-roots layer-idx *
  2 ^ layer-idx = clength * scale
  unfolding fri-evidence-layer-len-def
  by (rule fri-layer-lengths-round-product-if-dvd
    [OF roots-bound dvd-eval])
qed

lemma accepted-fri-opening-transcript-trace-evidence-layer-idx-bound:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  and round-bound: round-idx < length query-idxs
  and layer-bound: layer-idx < length trace-roots

```

shows
fri-evidence-layer-idx trace-roots query-idxs round-idx layer-idx <
fri-evidence-layer-len trace-roots layer-idx

proof –
obtain N **where** *eval-power: clength * scale = 2 ^ N*
using *eval-domain-length-power* **by** *blast*
have *roots-le: length trace-roots ≤ N*
proof –
have *len: length trace-roots = ceil-log clength*
using *accepted-fri-opening-transcript-shapes(1)[OF fri-openings]* .
have *clength ≤ length * scale*
using *scale-pos* **by** *simp*
also have $\dots = 2 ^ N$
by *(rule eval-power)*
finally have *ceil-log clength ≤ N*
by *(rule ceil-log-le-power)*
then show *?thesis*
using *len* **by** *simp*

qed
have *query-bound: query-idxs ! round-idx < clength * scale*
by *(rule accepted-fri-opening-transcript-query-idx-bound*
[OF fri-openings])
(use round-bound in simp)
have *idx-bound:*
fri-layer-indices (length trace-roots) (query-idxs ! round-idx)
*(clength * scale) ! layer-idx <*
*fri-layer-lengths (length trace-roots) (clength * scale) ! layer-idx*
by *(rule fri-layer-indices-nth-bound*
[OF query-bound layer-bound eval-power roots-le])
then show *?thesis*
unfolding *fri-evidence-layer-idx-def fri-evidence-layer-len-def* **by** *simp*

qed

lemma *trace-fri-header-tied-next-value-replay-gap-imp-merkle-or-slot-on-support:*
assumes *support: out ∈ set-dist (execute verify-monad s)*
and *gap: trace-fri-header-tied-next-value-replay-gap s out*
shows
partial-merkle-inconsistency-bad s out ∨
trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s out

proof –
from *gap* **have** *sampled:*
trace-fri-header-tied-sampled-next-value-conflict s out
unfolding *trace-fri-header-tied-next-value-replay-gap-def* **by** *simp*
from *sampled* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as rest trace-openings
trace-table
where *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final

fri-dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *cand:* *partial-trace-table-candidate trace-table trace-openings*
and *not-low:* $\neg$ *trace-table-low-degree trace-table*
and *generic:*
generic-fri-sampled-next-value-conflict trace-roots trace-bs
fri-query-idxs trace-round-layers
unfolding *trace-fri-header-tied-sampled-next-value-conflict-def* **by** *blast*
from *generic-fri-sampled-next-value-conflictE*[*OF generic*]
obtain *round-idx round-idx' layer-idx v v'*
where *round-bound:* *round-idx < length fri-query-idxs*
and *round-bound':* *round-idx' < length fri-query-idxs*
and *layer-bound:* *layer-idx < length trace-bs*
and *same-idx:*
fri-evidence-next-idx trace-roots fri-query-idxs round-idx
layer-idx =
fri-evidence-next-idx trace-roots fri-query-idxs round-idx'
layer-idx
and *forced:*
generic-fri-round-forced-next-value trace-roots trace-bs
fri-query-idxs trace-round-layers round-idx layer-idx v
and *forced':*
generic-fri-round-forced-next-value trace-roots trace-bs
fri-query-idxs trace-round-layers round-idx' layer-idx v'
and *neq:* $v \neq v'$
by *blast*
from *fri-openings* **obtain** *result final-state* **where** *out-eq:*
out = Some (result, final-state)
unfolding *accepted-fri-opening-transcript-def* **by** *blast*
have *len-bs-roots:* *length trace-bs = length trace-roots*
using *accepted-fri-opening-transcript-shapes*(2)[*OF fri-openings*] .
have *layer-bound-roots:* *layer-idx < length trace-roots*
using *layer-bound len-bs-roots* **by** *simp*
have *len-query:* *length fri-query-idxs = rounds*
using *accepted-fri-opening-transcript-shapes*(7)[*OF fri-openings*] .
have *raw-bound:*
fri-evidence-layer-idx trace-roots fri-query-idxs round-idx layer-idx <
fri-evidence-layer-len trace-roots layer-idx
by (*rule accepted-fri-opening-transcript-trace-evidence-layer-idx-bound*
[*OF fri-openings round-bound layer-bound-roots*])
have *raw-bound':*
fri-evidence-layer-idx trace-roots fri-query-idxs round-idx' layer-idx <
fri-evidence-layer-len trace-roots layer-idx


```

    by (rule accepted-fri-opening-transcript-trace-evidence-layer-idx-bound
        [OF fri-openings round-bound' layer-bound-roots])
  have even-len: 2 dvd fri-evidence-layer-len trace-roots layer-idx
    by (rule accepted-fri-opening-transcript-trace-layer-arithmetic(1)
        [OF fri-openings layer-bound])
  have round-product:
    fri-evidence-layer-len trace-roots layer-idx * 2 ^ layer-idx =
      clength * scale
    by (rule accepted-fri-opening-transcript-trace-layer-arithmetic(2)
        [OF fri-openings layer-bound])
  have auth-step:
     $\bigwedge i j.$ 
       $i < \text{length } \text{fri-query-idxs} \implies$ 
       $j < \text{length } \text{trace-bs} \implies$ 
      generic-fri-transcript-step-with-authenticated-chunk trace-roots
        trace-bs fri-query-idxs trace-round-layers final-state i j
  proof -
    fix i j
    assume i-bound:  $i < \text{length } \text{fri-query-idxs}$ 
      and j-bound:  $j < \text{length } \text{trace-bs}$ 
    have i-rounds:  $i < \text{rounds}$ 
      using i-bound len-query by simp
    have j-roots:  $j < \text{length } \text{trace-roots}$ 
      using j-bound len-bs-roots by simp
    show
      generic-fri-transcript-step-with-authenticated-chunk trace-roots
        trace-bs fri-query-idxs trace-round-layers final-state i j
    by (rule
        accepted-fri-opening-transcript-trace-step-with-authenticated-chunk-at-row
        [OF fri-openings out-eq i-rounds j-roots])
  qed
  show ?thesis
  proof (cases
    generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs
      fri-query-idxs trace-round-layers)
    case True
    have route:
      partial-merkle-inconsistency-bad s (Some (result, final-state))  $\vee$ 
      generic-fri-authenticated-slot-recorded-chunk-gap trace-roots
        fri-query-idxs trace-round-layers final-state
    by (rule generic-fri-sampled-same-layer-conflict-imp-merkle-or-slot-gap
        [OF True len-bs-roots auth-step])
    then show ?thesis
  proof
    assume partial-merkle-inconsistency-bad s (Some (result, final-state))
    then show ?thesis
      using out-eq by simp
  next
    assume slot-gap:

```

```

    generic-fri-authenticated-slot-recorded-chunk-gap trace-roots
    fri-query-idxs trace-round-layers final-state
  have route-slot-gap:
    trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s out
  unfolding trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,
     rule not-low, rule slot-gap)
  then show ?thesis by simp
qed
next
case False
then have no-same:
  ¬ generic-fri-sampled-same-layer-opening-conflict trace-roots
  trace-bs fri-query-idxs trace-round-layers
  by simp
have v = v'
  by (rule generic-fri-forced-next-value-eq-if-no-same-layer-conflict
      [OF no-same round-bound round-bound' layer-bound even-len
        raw-bound raw-bound' round-product same-idx forced forced'])
  then show ?thesis
    using neq by simp
qed
qed

lemma wp-trace-fri-header-tied-next-value-replay-gap-bound-from-merkle-and-slot:
  fixes M A :: prob
  assumes merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M
  and slot-bound:
    wp-event verify-monad
      (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s) s ≤ A
  shows
    wp-event verify-monad
      (trace-fri-header-tied-next-value-replay-gap s) s ≤ M + A
proof -
  let ?P = partial-merkle-inconsistency-bad s
  let ?Q = trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s
  have event-le:
    wp-event verify-monad
      (trace-fri-header-tied-next-value-replay-gap s) s ≤
      wp-event verify-monad (λout. ?P out ∨ ?Q out) s
  by (rule wp-event-mono-on-support)
    (rule
      trace-fri-header-tied-next-value-replay-gap-imp-merkle-or-slot-on-support)
  also have ... ≤ wp-event verify-monad ?P s +
    wp-event verify-monad ?Q s
  by (rule wp-event-union-bound)
  also have ... ≤ M + A

```

by (intro add-mono merkle-bound slot-bound)
 finally show ?thesis .
 qed

lemma *verifier-query-round-after-index-trace-successor-recorded-step-from-previous:*
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes *outcome:*
 Some $((\), t) \in$
 set-dist
 (*execute*
 (*verifier-query-round-after-index-program*
 fr f-fl f-final as fl final raw) s)
and *recorded:*
 query-round-fri-layer-transcripts (*index* (*to-nat raw*))
 (*map snd f-fl*) (*map snd fl*) *round-chunk trace-layers*
 composition-layers
and *transcript:* $P\text{Transcript } s = \text{round-chunk } @\ P\text{Transcript } t$
and *j-bound:* $\text{Suc } j < \text{length } f\text{-fl}$
and *all-layer-bounds:*
 $\bigwedge k. k < \text{length } f\text{-fl} \implies$
 $0 < \text{fri-layer-lengths } (\text{length } f\text{-fl})\ (\text{clength} * \text{scale}) ! k \wedge$
 $\text{fri-layer-indices } (\text{length } f\text{-fl})\ (\text{index } (\text{to-nat raw}))$
 $(\text{clength} * \text{scale}) ! k <$
 $\text{fri-layer-lengths } (\text{length } f\text{-fl})\ (\text{clength} * \text{scale}) ! k$
and *previous-step:*
 fri-layer-step-evidence (*snd* ($f\text{-fl} ! j$)) (*fst* ($f\text{-fl} ! j$))
 (*fri-layer-lengths* ($\text{length } f\text{-fl}$) ($\text{clength} * \text{scale}$) ! j)
 (*fri-layer-indices* ($\text{length } f\text{-fl}$) ($\text{index } (\text{to-nat raw})$))
 $(\text{clength} * \text{scale}) ! j$
 $(2 \wedge j)$
 (*fri-sibling-index*
 (*fri-layer-lengths* ($\text{length } f\text{-fl}$) ($\text{clength} * \text{scale}$) ! j)
 (*fri-layer-indices* ($\text{length } f\text{-fl}$) ($\text{index } (\text{to-nat raw})$))
 $(\text{clength} * \text{scale}) ! j$)
 $xp\ xp\text{-path } xn\ xn\text{-path}$
 (*fri-layer-indices* ($\text{length } f\text{-fl}$) ($\text{index } (\text{to-nat raw})$))
 $(\text{clength} * \text{scale}) ! j \bmod$
 $(\text{fri-layer-lengths } (\text{length } f\text{-fl})\ (\text{clength} * \text{scale}) ! j \text{ div } 2)$
 v
 (*trace-layers* ! j)
shows
 $\exists xp\text{-path}'\ xn'\ xn\text{-path}'\ \text{next-value.}$
 fri-layer-step-evidence (*snd* ($f\text{-fl} ! \text{Suc } j$)) (*fst* ($f\text{-fl} ! \text{Suc } j$))
 (*fri-layer-lengths* ($\text{length } f\text{-fl}$) ($\text{clength} * \text{scale}$) ! $\text{Suc } j$)
 (*fri-layer-indices* ($\text{length } f\text{-fl}$) ($\text{index } (\text{to-nat raw})$))
 $(\text{clength} * \text{scale}) ! \text{Suc } j$
 $(2 \wedge \text{Suc } j)$
 (*fri-sibling-index*
 (*fri-layer-lengths* ($\text{length } f\text{-fl}$) ($\text{clength} * \text{scale}$) ! $\text{Suc } j$)

```

    (fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! Suc j))
  v xp-path' xn' xn-path'
  (fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! Suc j mod
    (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j div 2))
  next-value
  (trace-layers ! Suc j)
proof -
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1) ∈
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
  and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some ((), s3) ∈
      set-dist
        (execute
          (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold
          (?idx, cp-eval as fv (h ^ ?idx * shift), clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some ((), t) ∈
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript ?idx fv query-paths query-chunk
  and tr-s: PTranscript s = query-chunk @ PTranscript s1
  by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  trace-layers-actual:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
  and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2

```

```

by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -)  $\Rightarrow$  f-x = f-final)
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers-actual:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk
and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
by blast
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -)  $\Rightarrow$  x = final)
    (auto simp: throw-no-outcome)
let ?actual-chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have actual-recorded:
  query-round-fri-layer-transcripts ?idx (map snd f-fl) (map snd fl)
  ?actual-chunk trace-layer-chunks comp-layer-chunks
unfolding query-round-fri-layer-transcripts-def
by (intro exI[of - query-chunk] exI[of - trace-fri-chunk]
    exI[of - comp-fri-chunk] exI[of - fv] exI[of - query-paths] conjI)
  (use query-chunk-shape trace-layers-actual comp-layers-actual in
     $\langle$ simp-all add: mult.commute $\rangle$ )
have actual-transcript:
  PTranscript s = ?actual-chunk @ PTranscript t
  using tr-s tr-s1 tr-s3 s3-eq t-eq by simp
have actual-chunk-eq: ?actual-chunk = round-chunk
  using actual-transcript transcript by simp
have trace-layer-chunks-eq:
  trace-layer-chunks = trace-layers
  by (rule query-round-fri-layer-transcripts-unique(1)
    [OF actual-recorded[unfolded actual-chunk-eq] recorded])
have idx-bound: ?idx < clength * scale
  by (rule index-less-domain)
have map-eq:
  receive-query-commits f-fl = map fri-layer-opening-step f-fl
  unfolding receive-query-commits-def fri-layer-opening-step-def
  by (simp add: fun-eq-iff split: prod.splits)
have previous-step-actual:
  fri-layer-step-evidence (snd (f-fl ! j)) (fst (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) ?idx (clength * scale) ! j)
  (1 * 2 ^ j)
  (fri-sibling-index
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) ?idx (clength * scale) ! j))
  xp xp-path xn xn-path

```

```

    (fri-layer-indices (length f-fl) ?idx (clength * scale) ! j mod
      (fri-layer-lengths (length f-fl) (clength * scale) ! j div 2))
  v
  (trace-layer-chunks ! j)
using previous-step trace-layer-chunks-eq by simp
from mfold-fri-layer-opening-successor-recorded-step-from-previous
  [OF trace-fri[unfolded map-eq] trace-layers-actual tr-s1 idx-bound
    all-layer-bounds j-bound previous-step-actual]
obtain sxp-path sxn sxn-path snext where succ-actual:
  fri-layer-step-evidence (snd (f-fl ! Suc j)) (fst (f-fl ! Suc j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j)
  (fri-layer-indices (length f-fl) ?idx (clength * scale) ! Suc j)
  (1 * 2 ^ Suc j)
  (fri-sibling-index
    (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j)
    (fri-layer-indices (length f-fl) ?idx (clength * scale) ! Suc j))
  v sxp-path sxn sxn-path
  (fri-layer-indices (length f-fl) ?idx (clength * scale) ! Suc j mod
    (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j div 2))
  snext
  (trace-layer-chunks ! Suc j)
by blast
have succ:
  fri-layer-step-evidence (snd (f-fl ! Suc j)) (fst (f-fl ! Suc j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j)
  (fri-layer-indices (length f-fl) ?idx (clength * scale) ! Suc j)
  (2 ^ Suc j)
  (fri-sibling-index
    (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j)
    (fri-layer-indices (length f-fl) ?idx (clength * scale) ! Suc j))
  v sxp-path sxn sxn-path
  (fri-layer-indices (length f-fl) ?idx (clength * scale) ! Suc j mod
    (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j div 2))
  snext
  (trace-layers ! Suc j)
using succ-actual trace-layer-chunks-eq by simp
show ?thesis
using succ by blast
qed

lemma accepted-fri-opening-transcript-trace-successor-recorded-step-from-previous-at:
assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers
and out-eq: out = Some (result, final-state)
and round-bound: i < rounds
and layer-bound: Suc j < length trace-roots
and all-layer-bounds:

```

$\bigwedge \text{raw } k. k < \text{length trace-roots} \implies$
 $0 < \text{fri-layer-lengths } (\text{length trace-roots})$
 $(\text{clength} * \text{scale}) ! k \wedge$
 $\text{fri-layer-indices } (\text{length trace-roots}) (\text{index } (\text{to-nat raw}))$
 $(\text{clength} * \text{scale}) ! k <$
 $\text{fri-layer-lengths } (\text{length trace-roots}) (\text{clength} * \text{scale}) ! k$
and previous-step:
 $\text{fri-layer-step-evidence } (\text{trace-roots} ! j)$
 $(\text{trace-bs} ! j)$
 $(\text{fri-evidence-layer-len } \text{trace-roots } j)$
 $(\text{fri-evidence-layer-idx } \text{trace-roots } \text{query-idxs } i j)$
 $(2 \wedge j)$
 $(\text{fri-sibling-index } (\text{fri-evidence-layer-len } \text{trace-roots } j)$
 $(\text{fri-evidence-layer-idx } \text{trace-roots } \text{query-idxs } i j))$
 $xp \text{ } xp\text{-path } xn \text{ } xn\text{-path}$
 $(\text{fri-evidence-next-idx } \text{trace-roots } \text{query-idxs } i j)$
 v
 $(\text{trace-round-layers} ! i ! j)$
shows
 $\exists xp\text{-path}' \text{ } xn' \text{ } xn\text{-path}' \text{ next-value.}$
 $\text{fri-layer-step-evidence } (\text{trace-roots} ! \text{Suc } j)$
 $(\text{trace-bs} ! \text{Suc } j)$
 $(\text{fri-evidence-layer-len } \text{trace-roots } (\text{Suc } j))$
 $(\text{fri-evidence-layer-idx } \text{trace-roots } \text{query-idxs } i (\text{Suc } j))$
 $(2 \wedge \text{Suc } j)$
 $(\text{fri-sibling-index}$
 $(\text{fri-evidence-layer-len } \text{trace-roots } (\text{Suc } j))$
 $(\text{fri-evidence-layer-idx } \text{trace-roots } \text{query-idxs } i$
 $(\text{Suc } j)))$
 $v \text{ } xp\text{-path}' \text{ } xn' \text{ } xn\text{-path}'$
 $(\text{fri-evidence-next-idx } \text{trace-roots } \text{query-idxs } i (\text{Suc } j))$
 next-value
 $(\text{trace-round-layers} ! i ! \text{Suc } j)$
proof –
from fri-openings **obtain** $\text{result}' \text{ } \text{final-state}'$ *fr f-fl as fl query-state*
 $\text{raw-idxs } \text{query-chunks}$ **where**
 $\text{out-def: out} = \text{Some } (\text{result}', \text{final-state}')$
and $\text{trace-roots-eq: trace-roots} = \text{map snd } f\text{-fl}$
and $\text{trace-bs-eq: trace-bs} = \text{map fst } f\text{-fl}$
and $\text{composition-roots-eq: composition-roots} = \text{map snd } fl$
and outcome:
 $\text{Some } (\text{result}', \text{final-state}') \in$
 $\text{set-dist } (\text{execute}$
 $(\text{ntimes}$
 $(\text{verifier-query-round-program } fr \text{ } f\text{-fl } \text{trace-final } as \text{ } fl$
 $\text{composition-final})$
 $\text{rounds})$
 $\text{query-state})$
and $\text{len-raw: length raw-idxs} = \text{rounds}$

```

and query-idxs-eq:
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state'
and query-chunk:
   $\bigwedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
and replay-lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmllookup (HashMap final-state')
    (QueryIndexChallenge (PQueryCounter query-state + i)
    (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some (raw-idxs ! i)
and recorded:
   $\bigwedge i. i < \text{rounds} \implies$ 
    query-round-fri-layer-transcripts (query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
    (trace-round-layers ! i) (composition-round-layers ! i)
  unfolding accepted-fri-opening-transcript-def by blast
have result-eq: result' = result
  using out-def out-eq by simp
have final-state-eq: final-state' = final-state
  using out-def out-eq by simp
have outcome':
  Some (result, final-state)  $\in$ 
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
  using outcome result-eq final-state-eq by simp
let ?round =
  verifier-query-round-program fr f-fl trace-final as fl composition-final
from query-opening-ntimes-outcome-decomp-at[OF outcome' round-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i)  $\in$ 
      set-dist (execute (ntimes ?round i) query-state)
  and round:
    Some (x, s-suc)  $\in$  set-dist (execute ?round s-i)
  and suffix:
    Some (suffix, final-state)  $\in$ 
      set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
  by blast
have x-eq: x = ()

```



```

  by (cases x) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
  fix prefix-query-idxs prefix-chunks
  assume len-prefix-idxs: length prefix-query-idxs = i
  and len-prefix-chunks: length prefix-chunks = i
  and transcript-prefix:
    PTranscript query-state =
      List.concat prefix-chunks @ PTranscript s-i
  and state-i:
    PState s-i =
      state-after-query-chunks (PState query-state) prefix-chunks i
  and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
  and prefix-ext: query-state ≤ s-i
  and prefix-rounds:
     $\bigwedge k. k < i \implies$ 
      verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
  from round[unfolded x-eq verifier-query-round-program-alt-def]
  obtain raw s0 where
    raw-out:
      Some (raw, s0) ∈
        set-dist (execute receive-query-index-challenge s-i)
  and after:
    Some ((), s-suc) ∈
      set-dist (execute
        (verifier-query-round-after-index-program fr f-fl trace-final as
          fl composition-final raw) s0)
  by (auto elim!: set-dist-bindE)
  have tr-s0: PTranscript s0 = PTranscript s-i
  using receive-query-index-challenge-outcome[OF raw-out] by simp
show ?thesis
proof (rule verifier-query-round-program-outcome[OF round[unfolded x-eq]])
  fix raw' idx chunk
  assume idx-eq: idx = index (to-nat raw')
  and chunk-shape:
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
  and transcript-round:
    PTranscript s-i = chunk @ PTranscript s-suc
  and lookup-suc:
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw'
  have raw-eq': raw = raw'
  proof -
    have lookup-s0:
      fmlookup (HashMap s0)

```

```

    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
  using receive-query-index-challenge-outcome[OF raw-out] by simp
have s0-suc: s0 ≤ s-suc
  using verifier-query-round-after-index-program-outcome[OF after]
  by simp
have lookup-suc-raw:
  fmllookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
  using hash-extension-lookup[OF lookup-s0 s0-suc] .
show ?thesis
  using lookup-suc lookup-suc-raw by simp
qed
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
  fix suffix-query-idxs suffix-chunks
  assume len-suffix-idxs:
    length suffix-query-idxs = rounds - Suc i
  and len-suffix-chunks:
    length suffix-chunks = rounds - Suc i
  and transcript-suffix:
    PTranscript s-suc =
      List.concat suffix-chunks @ PTranscript final-state
  and suffix-ext: s-suc ≤ final-state
  and suffix-rounds:
     $\bigwedge k. k < rounds - Suc i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
  let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
  have len-selected-chunks: length ?selected-chunks = rounds
    using len-prefix-chunks len-suffix-chunks round-bound by simp
  have selected-chunk-lens:
     $\bigwedge k. k < rounds \implies$ 
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  proof -
    fix k
    assume k-bound: k < rounds
    show
      length (?selected-chunks ! k) =
        verifier-query-round-transcript-length (query-idxs ! k)
          (map snd f-fl) (map snd fl)
    proof (cases k < i)
      case True
      have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
        using True len-prefix-chunks by (simp add: nth-append)

```

```

have
  length (prefix-chunks ! k) =
    verifier-query-round-transcript-length
      (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
  by (rule verifier-query-round-chunk-length
      [OF prefix-rounds[OF True]])
then show ?thesis
  using nth-eq
  by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
next
case False
show ?thesis
proof (cases k = i)
  case True
  have nth-eq: ?selected-chunks ! k = chunk
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length chunk =
      verifier-query-round-transcript-length idx
        (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF chunk-shape])
  then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
next
case False
let ?k = k - Suc i
have ge: Suc i ≤ k
  using ⟨¬ k < i⟩ False by linarith
have k-bound!: ?k < rounds - Suc i
  using ge k-bound by linarith
have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
  using ge len-prefix-chunks by (simp add: nth-append)
have
  length (suffix-chunks ! ?k) =
    verifier-query-round-transcript-length
      (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
  by (rule verifier-query-round-chunk-length
      [OF suffix-rounds[OF k-bound']])
then show ?thesis
  using nth-eq
  by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-f:

```

```

 $\bigwedge k. k < \text{rounds} \implies$ 
  verifier-query-round-chunk (query-idxs ! k)
    (map snd f-fl) (map snd fl) (query-chunks ! k)
using query-chunk trace-roots-eq composition-roots-eq by simp
have query-chunk-lens:
 $\bigwedge k. k < \text{rounds} \implies$ 
  length (query-chunks ! k) =
    verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
using query-chunk-f verifier-query-round-chunk-length by blast
have selected-transcript:
  PTranscript query-state =
    List.concat ?selected-chunks @ PTranscript final-state
using transcript-prefix transcript-round transcript-suffix by simp
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
using selected-transcript transcript-query final-state-eq by simp
have chunks-eq: ?selected-chunks = query-chunks
proof –
  have same-lens:
 $\bigwedge k. k < \text{length } ?\text{selected-chunks} \implies$ 
    length (?selected-chunks ! k) = length (query-chunks ! k)
using selected-chunk-lens query-chunk-lens len-selected-chunks
by simp
have concat-eq-empty:
  List.concat ?selected-chunks @ [] = List.concat query-chunks
using concat-eq by simp
have len-eq: length ?selected-chunks = length query-chunks
using len-selected-chunks len-query-chunks by simp
have ?selected-chunks = query-chunks  $\wedge$  ([] :: 'f list) = []
by (rule concat-append-eq-concat-same-chunk-lengths
    [OF len-eq same-lens concat-eq-empty])
then show ?thesis
by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
proof –
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
      state-after-query-chunks (PState query-state) prefix-chunks i
by (rule state-after-query-chunks-append-prefix
    [OF len-prefix-chunks])
then show ?thesis
using chunks-eq by simp
qed
have lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)

```

```

    (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some raw'
proof -
  have lookup-suc':
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
    Some raw'
  using lookup-suc state-i counter-i by simp
  have lookup-final-prefix:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
    Some raw'
  using hash-extension-lookup[OF lookup-suc' suffix-ext] .
  show ?thesis
    using lookup-final-prefix prefix-state-eq by simp
qed
have raw-idx-eq: raw' = raw-idxs ! i
  using lookup-final replay-lookup[OF round-bound] final-state-eq
  by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-idx-eq query-idxs-eq len-raw round-bound by simp
have selected-i: ?selected-chunks ! i = chunk
  using len-prefix-chunks by (simp add: nth-append)
have chunk-i-eq: chunk = query-chunks ! i
  using chunks-eq selected-i by simp
have recorded-i:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) chunk (trace-round-layers ! i)
    (composition-round-layers ! i)
  using recorded[OF round-bound] idx-eq idx-eq-query raw-eq'
    chunk-i-eq trace-roots-eq composition-roots-eq
  by simp
have tr-s0-suc:
  PTranscript s0 = chunk @ PTranscript s-suc
  using tr-s0 transcript-round by simp
have suc-j-bound-fl: Suc j < length f-fl
  using layer-bound trace-roots-eq by simp
have all-layer-bounds-fl:
   $\bigwedge k. k < \text{length } f\text{-fl} \implies$ 
  0 < fri-layer-lengths (length f-fl) (clength * scale) ! k  $\wedge$ 
  fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! k <
  fri-layer-lengths (length f-fl) (clength * scale) ! k
  using all-layer-bounds trace-roots-eq by simp
have raw-idx-query: index (to-nat raw) = query-idxs ! i

```

```

using idx-eq idx-eq-query raw-eq' by simp
have previous-step-fl:
  fri-layer-step-evidence (snd (f-fl ! j)) (fst (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! j)
    ( $2 \wedge j$ )
    (fri-sibling-index
      (fri-layer-lengths (length f-fl) (clength * scale) ! j)
      (fri-layer-indices (length f-fl) (index (to-nat raw))
        (clength * scale) ! j))
    xp xp-path xn xn-path
    (fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! j mod
      (fri-layer-lengths (length f-fl) (clength * scale) ! j div 2))
    v
    (trace-round-layers ! i ! j)
using previous-step trace-roots-eq trace-bs-eq raw-idx-query
  layer-bound
unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
  fri-evidence-next-idx-def
by simp
from verifier-query-round-after-index-trace-successor-recorded-step-from-previous
  [OF after recorded-i tr-s0-suc suc-j-bound-fl
    all-layer-bounds-fl previous-step-fl]
obtain sxp-path srxn srxn-path snext where succ-fl:
  fri-layer-step-evidence (snd (f-fl ! Suc j))
    (fst (f-fl ! Suc j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j)
    (fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! Suc j)
    ( $2 \wedge \text{Suc } j$ )
    (fri-sibling-index
      (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j)
      (fri-layer-indices (length f-fl) (index (to-nat raw))
        (clength * scale) ! Suc j))
    v sxp-path srxn srxn-path
    (fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! Suc j mod
      (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j
        div 2))
    snext
    (trace-round-layers ! i ! Suc j)
by blast
have succ:
  fri-layer-step-evidence (trace-roots ! Suc j)
    (trace-bs ! Suc j)
    (fri-evidence-layer-len trace-roots (Suc j))
    (fri-evidence-layer-idx trace-roots query-idxs i (Suc j))

```

```

    (2  $\wedge$  Suc j)
    (fri-sibling-index
      (fri-evidence-layer-len trace-roots (Suc j))
      (fri-evidence-layer-idx trace-roots query-idxs i
        (Suc j)))
    v sxp-path sxn sxn-path
    (fri-evidence-next-idx trace-roots query-idxs i (Suc j))
    snext
    (trace-round-layers ! i ! Suc j)
  using succ-fl trace-roots-eq trace-bs-eq raw-idx-query layer-bound
  unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
    fri-evidence-next-idx-def
  by simp
show ?thesis
  using succ by blast
qed
qed
qed
qed

lemma trace-fri-header-tied-successor-replay-gap-imp-merkle-or-slot-on-support:
  assumes support: out  $\in$  set-dist (execute verify-monad s)
  and gap: trace-fri-header-tied-successor-replay-gap s out
  shows
    partial-merkle-inconsistency-bad s out  $\vee$ 
    trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s out
proof -
  from gap have sampled:
    trace-fri-header-tied-sampled-successor-opening-conflict s out
  unfolding trace-fri-header-tied-successor-replay-gap-def by simp
  from sampled obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as rest trace-openings
    trace-table
  where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  and header:
    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest
  and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  and cand: partial-trace-table-candidate trace-table trace-openings
  and not-low:  $\neg$  trace-table-low-degree trace-table
  and generic:
    generic-fri-sampled-successor-opening-conflict trace-roots trace-bs
    fri-query-idxs trace-round-layers

```

unfolding *trace-fri-header-tied-sampled-successor-opening-conflict-def*
by *blast*
from *fri-openings* **obtain** *result final-state* **where** *out-eq*:
out = Some (result, final-state)
unfolding *accepted-fri-opening-transcript-def* **by** *blast*
have *len-bs-roots*: *length trace-bs = length trace-roots*
using *accepted-fri-opening-transcript-shapes(2)*[*OF fri-openings*] .
have *len-query*: *length fri-query-idxs = rounds*
using *accepted-fri-opening-transcript-shapes(7)*[*OF fri-openings*] .
have *all-layer-bounds*:
 $\bigwedge \text{raw } k. k < \text{length trace-roots} \implies$
 $0 < \text{fri-layer-lengths } (\text{length trace-roots})$
 $(\text{clength} * \text{scale}) ! k \wedge$
 $\text{fri-layer-indices } (\text{length trace-roots}) (\text{index } (\text{to-nat raw}))$
 $(\text{clength} * \text{scale}) ! k <$
 $\text{fri-layer-lengths } (\text{length trace-roots}) (\text{clength} * \text{scale}) ! k$
by (*rule accepted-fri-opening-transcript-trace-raw-layer-bound*
[*OF fri-openings*])
have *source-step*:
 $\bigwedge \text{source-round layer-idx } v.$
 $\text{source-round} < \text{length fri-query-idxs} \implies$
 $\text{Suc layer-idx} < \text{length trace-bs} \implies$
 $\text{generic-fri-round-forced-next-value trace-roots trace-bs}$
 $\text{fri-query-idxs trace-round-layers source-round layer-idx } v \implies$
 $\exists \text{xp-path } \text{xn } \text{xn-path}.$
fri-layer-step-evidence
 $(\text{trace-roots} ! \text{Suc layer-idx})$
 $(\text{trace-bs} ! \text{Suc layer-idx})$
 $(\text{fri-evidence-layer-len trace-roots } (\text{Suc layer-idx}))$
 $(\text{fri-evidence-layer-idx trace-roots fri-query-idxs}$
 $\text{source-round } (\text{Suc layer-idx}))$
 $(2 \wedge \text{Suc layer-idx})$
 $(\text{fri-sibling-index}$
 $(\text{fri-evidence-layer-len trace-roots } (\text{Suc layer-idx}))$
 $(\text{fri-evidence-layer-idx trace-roots fri-query-idxs}$
 $\text{source-round } (\text{Suc layer-idx})))$
 $v \text{ xp-path } \text{xn } \text{xn-path}$
 $(\text{fri-evidence-next-idx trace-roots fri-query-idxs}$
 $\text{source-round } (\text{Suc layer-idx}))$
 $(\text{fri-evidence-next-value trace-roots trace-bs}$
 $\text{fri-query-idxs source-round } (\text{Suc layer-idx}) v \text{ xn})$
 $(\text{trace-round-layers} ! \text{source-round} ! \text{Suc layer-idx})$
proof –
fix *source-round layer-idx v*
assume *source-round-bound*: *source-round < length fri-query-idxs*
and *layer-bound'*: *Suc layer-idx < length trace-bs*
and *forced*:
 $\text{generic-fri-round-forced-next-value trace-roots trace-bs}$
 $\text{fri-query-idxs trace-round-layers source-round layer-idx } v$


```

have source-round-rounds: source-round < rounds
  using source-round-bound len-query by simp
have layer-bound-roots: Suc layer-idx < length trace-roots
  using layer-bound' len-bs-roots by simp
from forced obtain fxp fxp-path fxn fxn-path where previous-step:
  fri-layer-step-evidence (trace-roots ! layer-idx)
    (trace-bs ! layer-idx)
    (fri-evidence-layer-len trace-roots layer-idx)
    (fri-evidence-layer-idx trace-roots fri-query-idxs
      source-round layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index
      (fri-evidence-layer-len trace-roots layer-idx)
      (fri-evidence-layer-idx trace-roots fri-query-idxs
        source-round layer-idx))
  fxp fxp-path fxn fxn-path
  (fri-evidence-next-idx trace-roots fri-query-idxs
    source-round layer-idx)
  v
  (trace-round-layers ! source-round ! layer-idx)
unfolding generic-fri-round-forced-next-value-def by blast
from accepted-fri-opening-transcript-trace-successor-recorded-step-from-previous-at
  [OF fri-openings out-eq source-round-rounds layer-bound-roots
    all-layer-bounds previous-step]
obtain sxp-path sxn sxn-path snext where succ:
  fri-layer-step-evidence (trace-roots ! Suc layer-idx)
    (trace-bs ! Suc layer-idx)
    (fri-evidence-layer-len trace-roots (Suc layer-idx))
    (fri-evidence-layer-idx trace-roots fri-query-idxs
      source-round (Suc layer-idx))
    (2 ^ Suc layer-idx)
    (fri-sibling-index
      (fri-evidence-layer-len trace-roots (Suc layer-idx))
      (fri-evidence-layer-idx trace-roots fri-query-idxs
        source-round (Suc layer-idx)))
  v sxp-path sxn sxn-path
  (fri-evidence-next-idx trace-roots fri-query-idxs
    source-round (Suc layer-idx))
  snext
  (trace-round-layers ! source-round ! Suc layer-idx)
by blast
have snext-eq:
  snext =
    fri-evidence-next-value trace-roots trace-bs
      fri-query-idxs source-round (Suc layer-idx) v sxn
using fri-layer-step-evidenceD(3)[OF succ]
unfolding fri-evidence-next-value-def by simp
show
  ∃ xp-path xn xn-path.

```

```

fri-layer-step-evidence
  (trace-roots ! Suc layer-idx)
  (trace-bs ! Suc layer-idx)
  (fri-evidence-layer-len trace-roots (Suc layer-idx))
  (fri-evidence-layer-idx trace-roots fri-query-idxs
    source-round (Suc layer-idx))
  ( $2 \wedge \text{Suc layer-idx}$ )
  (fri-sibling-index
    (fri-evidence-layer-len trace-roots (Suc layer-idx))
    (fri-evidence-layer-idx trace-roots fri-query-idxs
      source-round (Suc layer-idx)))
  v xp-path xn xn-path
  (fri-evidence-next-idx trace-roots fri-query-idxs
    source-round (Suc layer-idx))
  (fri-evidence-next-value trace-roots trace-bs
    fri-query-idxs source-round (Suc layer-idx) v xn)
  (trace-round-layers ! source-round ! Suc layer-idx)
using succ snext-eq by blast
qed
have same-layer:
  generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs
  fri-query-idxs trace-round-layers
by (rule generic-fri-sampled-successor-conflict-imp-same-layer-with-source-steps
  [OF generic len-bs-roots source-step])
have auth-step:
   $\bigwedge i j.$ 
   $i < \text{length } \text{fri-query-idxs} \implies$ 
   $j < \text{length } \text{trace-bs} \implies$ 
  generic-fri-transcript-step-with-authenticated-chunk trace-roots
  trace-bs fri-query-idxs trace-round-layers final-state i j
proof –
  fix i j
  assume i-bound:  $i < \text{length } \text{fri-query-idxs}$ 
  and j-bound:  $j < \text{length } \text{trace-bs}$ 
  have i-rounds:  $i < \text{rounds}$ 
  using i-bound len-query by simp
  have j-roots:  $j < \text{length } \text{trace-roots}$ 
  using j-bound len-bs-roots by simp
  show
  generic-fri-transcript-step-with-authenticated-chunk trace-roots
  trace-bs fri-query-idxs trace-round-layers final-state i j
  by (rule
    accepted-fri-opening-transcript-trace-step-with-authenticated-chunk-at-raw
    [OF fri-openings out-eq i-rounds j-roots])
qed
have route:
  partial-merkle-inconsistency-bad s (Some (result, final-state))  $\vee$ 
  generic-fri-authenticated-slot-recorded-chunk-gap trace-roots
  fri-query-idxs trace-round-layers final-state

```

```

    by (rule generic-fri-sampled-same-layer-conflict-imp-merkle-or-slot-gap
        [OF same-layer len-bs-roots auth-step])
  then show ?thesis
proof
  assume partial-merkle-inconsistency-bad s (Some (result, final-state))
  then show ?thesis
    using out-eq by simp
next
  assume slot-gap:
    generic-fri-authenticated-slot-recorded-chunk-gap trace-roots
    fri-query-idxs trace-round-layers final-state
  have route-slot-gap:
    trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s out
  unfolding trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,
     rule not-low, rule slot-gap)
  then show ?thesis by simp
qed
qed

lemma wp-trace-fri-header-tied-successor-replay-gap-bound-from-merkle-and-slot:
  fixes M A :: prob
  assumes merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M
  and slot-bound:
    wp-event verify-monad
      (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s) s ≤ A
  shows
    wp-event verify-monad
      (trace-fri-header-tied-successor-replay-gap s) s ≤ M + A
proof -
  let ?P = partial-merkle-inconsistency-bad s
  let ?Q = trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s
  have event-le:
    wp-event verify-monad
      (trace-fri-header-tied-successor-replay-gap s) s ≤
      wp-event verify-monad (λout. ?P out ∨ ?Q out) s
  by (rule wp-event-mono-on-support)
    (rule
      trace-fri-header-tied-successor-replay-gap-imp-merkle-or-slot-on-support)
  also have ... ≤ wp-event verify-monad ?P s +
    wp-event verify-monad ?Q s
  by (rule wp-event-union-bound)
  also have ... ≤ M + A
  by (intro add-mono merkle-bound slot-bound)
  finally show ?thesis .
qed

```

end

end

theory *Soundness-FRI-Trace-Sibling-Evidence*
 imports *Soundness-FRI-Value-Aligned-Events*
begin

Proof-only refinement of the trace FRI sibling mismatch branch.

The existing trace candidate predicate records agreement with the trace-query openings only. The first trace FRI sibling value is read from the FRI layer transcript, so the sibling branch cannot be closed from the current candidate predicate alone. This theory isolates the exact missing evidence: a trace candidate must also agree with the recorded first-layer FRI sibling values.

context *soundness*
begin

definition *trace-table-agrees-with-recorded-first-fri-siblings*
 $:: 'f\ list \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow nat\ list \Rightarrow$
 $'f\ list\ list\ list \Rightarrow bool$

where

trace-table-agrees-with-recorded-first-fri-siblings *trace-table*
 trace-roots trace-bs query-idxs trace-round-layers $\longleftrightarrow$
 $(\forall\ round_idx\ xp\ xp_path\ xn\ xn_path.$
 $round_idx < length\ query_idxs \longrightarrow$
 $0 < length\ trace_roots \longrightarrow$
 $0 < length\ trace_bs \longrightarrow$
 fri-layer-step-evidence
 $(trace_roots\ !\ 0)$
 $(trace_bs\ !\ 0)$
 $(fri_evidence_layer_len\ trace_roots\ 0)$
 $(fri_evidence_layer_idx\ trace_roots\ query_idxs\ round_idx\ 0)$
 1
 $(fri_sibling_index\ (fri_evidence_layer_len\ trace_roots\ 0)$
 $(fri_evidence_layer_idx\ trace_roots\ query_idxs\ round_idx\ 0))$
 $xp\ xp_path\ xn\ xn_path$
 $(fri_evidence_next_idx\ trace_roots\ query_idxs\ round_idx\ 0)$
 $(fri_evidence_next_value\ trace_roots\ trace_bs\ query_idxs$
 $round_idx\ 0\ xp\ xn)$
 $(trace_round_layers\ !\ round_idx\ !\ 0) \longrightarrow$
 $trace_table\ !$
 $fri_sibling_index\ (fri_evidence_layer_len\ trace_roots\ 0)$
 $(fri_evidence_layer_idx\ trace_roots\ query_idxs\ round_idx\ 0) =$
 $xn)$

definition *partial-trace-table-candidate-with-recorded-first-fri-siblings*
 $:: 'f\ list \Rightarrow 'f\ authenticated_opening\ list\ list \Rightarrow$

'f list $\Rightarrow$ 'f list $\Rightarrow$ nat list $\Rightarrow$ 'f list list list $\Rightarrow$
 bool

where

partial-trace-table-candidate-with-recorded-first-fri-siblings
trace-table trace-openings trace-roots trace-bs query-idxs
trace-round-layers $\longleftrightarrow$
partial-trace-table-candidate trace-table trace-openings $\wedge$
trace-table-agrees-with-recorded-first-fri-siblings trace-table
trace-roots trace-bs query-idxs trace-round-layers

lemma *partial-trace-table-candidate-with-recorded-first-fri-siblingsD*:

assumes

partial-trace-table-candidate-with-recorded-first-fri-siblings
trace-table trace-openings trace-roots trace-bs query-idxs
trace-round-layers

shows *partial-trace-table-candidate trace-table trace-openings*

and *trace-table-agrees-with-recorded-first-fri-siblings trace-table*
trace-roots trace-bs query-idxs trace-round-layers

using *assms*

unfolding *partial-trace-table-candidate-with-recorded-first-fri-siblings-def*
by *blast+*

definition *trace-fri-header-tied-recorded-sibling-candidate-mismatch*

$::$ ('f, 'a) protocol-channel-scheme $\Rightarrow$
 (unit list $\times$ ('f, 'a) protocol-channel-scheme) option $\Rightarrow$ bool

where

trace-fri-header-tied-recorded-sibling-candidate-mismatch s out
 $\longleftrightarrow$
 $(\exists$ *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table
round-idx xp xp-path xn xn-path result final-state.
out = Some (result, final-state) $\wedge$
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
trace-table-agrees-with-recorded-first-fri-siblings trace-table
trace-roots trace-bs fri-query-idxs trace-round-layers $\wedge$
 $\neg$ *trace-table-low-degree trace-table* $\wedge$
round-idx < length fri-query-idxs $\wedge$
 $0 < \text{length trace-bs}$ $\wedge$
trace-openings ! round-idx $\neq []$ $\wedge$
opening-index (hd (trace-openings ! round-idx)) =
fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0 $\wedge$

opening-value (*hd* (*trace-openings* ! *round-idx*)) = *xp* ∧
fri-layer-step-evidence
(*trace-roots* ! 0)
(*trace-bs* ! 0)
(*fri-evidence-layer-len* *trace-roots* 0)
(*fri-evidence-layer-idx* *trace-roots* *fri-query-idxs* *round-idx* 0)
1
(*fri-sibling-index* (*fri-evidence-layer-len* *trace-roots* 0)
(*fri-evidence-layer-idx* *trace-roots* *fri-query-idxs* *round-idx* 0))
xp xp-path xn xn-path
(*fri-evidence-next-idx* *trace-roots* *fri-query-idxs* *round-idx* 0)
(*fri-evidence-next-value* *trace-roots* *trace-bs* *fri-query-idxs*
round-idx 0 *xp xn*)
(*trace-round-layers* ! *round-idx* ! 0) ∧
trace-table !
fri-sibling-index (*fri-evidence-layer-len* *trace-roots* 0)
(*fri-evidence-layer-idx* *trace-roots* *fri-query-idxs* *round-idx* 0)
≠ *xn*)

definition *trace-fri-header-tied-recorded-sibling-candidate-missing*

:: (*f*, *a*) *protocol-channel-scheme* ⇒
(*unit list* × (*f*, *a*) *protocol-channel-scheme*) *option* ⇒ *bool*

where

trace-fri-header-tied-recorded-sibling-candidate-missing *s out*
 $\longleftrightarrow$
(∃ *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table
round-idx xp xp-path xn xn-path result final-state.
out = *Some* (*result, final-state*) ∧
accepted-fri-opening-transcript *s out trace-roots trace-bs trace-final*
dg composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers ∧
verifier-header-transcript *s fr trace-roots trace-final as dg*
composition-roots composition-final rest ∧
accepted-with-partial-trace-openings *s out fr fri-query-idxs*
trace-openings ∧
partial-trace-table-candidate *trace-table trace-openings* ∧
¬ *trace-table-agrees-with-recorded-first-fri-siblings* *trace-table*
trace-roots trace-bs fri-query-idxs trace-round-layers ∧
¬ *trace-table-low-degree* *trace-table* ∧
round-idx < *length fri-query-idxs* ∧
0 < *length trace-bs* ∧
trace-openings ! *round-idx* ≠ [] ∧
opening-index (*hd* (*trace-openings* ! *round-idx*)) =
fri-evidence-layer-idx *trace-roots* *fri-query-idxs* *round-idx* 0 ∧
opening-value (*hd* (*trace-openings* ! *round-idx*)) = *xp* ∧
fri-layer-step-evidence
(*trace-roots* ! 0)

```

(trace-bs ! 0)
(fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
1
(fri-sibling-index (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
xp xp-path xn xn-path
(fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
(fri-evidence-next-value trace-roots trace-bs fri-query-idxs
  round-idx 0 xp xn)
(trace-round-layers ! round-idx ! 0) ∧
trace-table !
fri-sibling-index (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
≠ xn)

```

lemma *generic-trace-sampled-layer-chain-evidence-recorded-first-sibling-agreement:*

assumes *chain:*

generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers

shows

trace-table-agrees-with-recorded-first-fri-siblings trace-table
trace-roots trace-bs fri-query-idxs trace-round-layers

proof (*unfold trace-table-agrees-with-recorded-first-fri-siblings-def,*
intro allI impI)

fix *round-idx xp xp-path xn xn-path*

assume *round-bound: round-idx < length fri-query-idxs*

and *roots-nonempty: 0 < length trace-roots*

and *challenges-nonempty: 0 < length trace-bs*

and *step:*

```

fri-layer-step-evidence
  (trace-roots ! 0)
  (trace-bs ! 0)
  (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
1
  (fri-sibling-index (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
  xp xp-path xn xn-path
  (fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
  (fri-evidence-next-value trace-roots trace-bs fri-query-idxs
    round-idx 0 xp xn)
  (trace-round-layers ! round-idx ! 0)

```

have *layer0: layers ! 0 = trace-table*

by (*rule generic-fri-sampled-layer-chain-evidenceD(5)[OF chain]*)

have *sampled:*

```

fri-sampled-table-fold
  (trace-bs ! 0)

```

```

(fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
(2 ^ 0)
(doms ! 0)
(layers ! 0)
(trace-round-layers ! round-idx ! 0)
(layers ! Suc 0 !
  fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
by (rule generic-fri-sampled-layer-chain-evidence-sample(2)
  [OF chain round-bound challenges-nonempty])
from sampled obtain yp yp-path yn yn-path
where chunk:
  fri-layer-opening-chunk (fri-evidence-layer-len trace-roots 0)
  yp yp-path yn yn-path (trace-round-layers ! round-idx ! 0)
and match:
  fri-opening-matches-table (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
  (layers ! 0) yp yn
by (elim fri-sampled-table-foldE)
have step-chunk:
  fri-layer-opening-chunk (fri-evidence-layer-len trace-roots 0)
  xp xp-path xn xn-path (trace-round-layers ! round-idx ! 0)
  by (rule fri-layer-step-evidenceD(4)[OF step])
have yn-eq: yn = xn
  using fri-layer-opening-chunk-values-unique(2)[OF chunk step-chunk]
  by simp
have (layers ! 0) !
  fri-sibling-index (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0) =
  yn
  by (rule fri-opening-matches-tableD(4)[OF match])
then show
  trace-table !
  fri-sibling-index (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0) =
  xn
  using layer0 yn-eq by simp
qed

```

lemma *generic-trace-sampled-layer-chain-evidence-augments-partial-trace-candidate:*

assumes *cand: partial-trace-table-candidate trace-table trace-openings*

and chain:

generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
query-idxs trace-round-layers doms layers

shows

partial-trace-table-candidate-with-recorded-first-fri-siblings
trace-table trace-openings trace-roots trace-bs query-idxs
trace-round-layers

unfolding *partial-trace-table-candidate-with-recorded-first-fri-siblings-def*
by (*intro conjI cand*
generic-trace-sampled-layer-chain-evidence-recorded-first-sibling-agreement
[OF chain])

definition *trace-fri-bad-with-header-tied-augmented-partial-candidate*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-bad-with-header-tied-augmented-partial-candidate s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table.}$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{ verifier-header-transcript s fr trace-roots trace-final as dg}$
 $\text{ composition-roots composition-final rest} \wedge$
 $\text{ accepted-with-partial-trace-openings s out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{ partial-trace-table-candidate-with-recorded-first-fri-siblings}$
 $\text{ trace-table trace-openings trace-roots trace-bs fri-query-idxs}$
 $\text{ trace-round-layers} \wedge$
 $\neg \text{ trace-table-low-degree trace-table})$

definition *trace-fri-bad-with-header-tied-augmented-sampled-layer-chain*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-bad-with-header-tied-augmented-sampled-layer-chain s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table}$
 doms layers.
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{ verifier-header-transcript s fr trace-roots trace-final as dg}$
 $\text{ composition-roots composition-final rest} \wedge$
 $\text{ accepted-with-partial-trace-openings s out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{ partial-trace-table-candidate-with-recorded-first-fri-siblings}$
 $\text{ trace-table trace-openings trace-roots trace-bs fri-query-idxs}$
 $\text{ trace-round-layers} \wedge$
 $\neg \text{ trace-table-low-degree trace-table} \wedge$
 $\text{ generic-fri-sampled-layer-chain-evidence trace-table-low-degree}$
 $\text{ trace-table (clength - 1) trace-roots trace-bs trace-final}$

fri-query-idxs trace-round-layers doms layers)

definition *trace-fri-header-tied-augmented-missing-sampled-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-augmented-missing-sampled-layer-chain s out
 $\longleftrightarrow$
trace-fri-bad-with-header-tied-augmented-partial-candidate s out $\wedge$
 $\neg \text{trace-fri-bad-with-header-tied-augmented-sampled-layer-chain } s \text{ out}$

definition *trace-fri-header-tied-unaugmented-partial-candidate*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-unaugmented-partial-candidate s out
 $\longleftrightarrow$
trace-fri-bad-with-header-tied-partial-candidate s out $\wedge$
 $\neg \text{trace-fri-bad-with-header-tied-augmented-partial-candidate } s \text{ out}$

lemma *trace-fri-bad-with-header-tied-augmented-partial-candidate-imp-plain:*

assumes

trace-fri-bad-with-header-tied-augmented-partial-candidate s out

shows *trace-fri-bad-with-header-tied-partial-candidate s out*

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as rest trace-openings trace-table

where *fri:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers

and *header:*

verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest

and *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings

and *cand:*

partial-trace-table-candidate-with-recorded-first-fri-siblings
trace-table trace-openings trace-roots trace-bs fri-query-idxs
trace-round-layers

and *nonlow:* $\neg \text{trace-table-low-degree } \text{trace-table}$

unfolding *trace-fri-bad-with-header-tied-augmented-partial-candidate-def*

by *blast*

have *plain:* *partial-trace-table-candidate trace-table trace-openings*

by *(rule*

partial-trace-table-candidate-with-recorded-first-fri-siblingsD(1)
 $[\text{OF } \text{cand}]$)

show *?thesis*
unfolding *trace-fri-bad-with-header-tied-partial-candidate-def*
by (*intro exI conjI*)
 (*rule fri, rule header, rule partial, rule plain, rule nonlow*)
qed

lemma *trace-fri-bad-with-header-tied-partial-candidate-imp-augmented-or-unaugmented:*
assumes *trace-fri-bad-with-header-tied-partial-candidate s out*
shows
trace-fri-bad-with-header-tied-augmented-partial-candidate s out $\vee$
trace-fri-header-tied-unaugmented-partial-candidate s out
using *assms*
unfolding *trace-fri-header-tied-unaugmented-partial-candidate-def*
by *blast*

lemma *wp-trace-fri-header-tied-partial-candidate-bound-from-augmented-and-unaugmented:*
assumes *augmented-bound:*
wp-event verify-monad
 (*trace-fri-bad-with-header-tied-augmented-partial-candidate s*) *s*
 $\leq A$
and *unaugmented-bound:*
wp-event verify-monad
 (*trace-fri-header-tied-unaugmented-partial-candidate s*) *s* $\leq U$
shows
wp-event verify-monad
 (*trace-fri-bad-with-header-tied-partial-candidate s*) *s* $\leq A + U$
proof –
have *wp-event verify-monad*
 (*trace-fri-bad-with-header-tied-partial-candidate s*) *s* $\leq$
wp-event verify-monad
 ($\lambda out.$
trace-fri-bad-with-header-tied-augmented-partial-candidate s out $\vee$
trace-fri-header-tied-unaugmented-partial-candidate s out) *s*
by (*rule wp-event-mono*)
 (*rule*
trace-fri-bad-with-header-tied-partial-candidate-imp-augmented-or-unaugmented)
also have $\dots \leq$
wp-event verify-monad
 (*trace-fri-bad-with-header-tied-augmented-partial-candidate s*) *s* +
wp-event verify-monad
 (*trace-fri-header-tied-unaugmented-partial-candidate s*) *s*
by (*rule wp-event-union-bound*)
also have $\dots \leq A + U$
by (*rule add-mono[OF augmented-bound unaugmented-bound]*)
finally show *?thesis* .
qed

lemma *trace-fri-bad-with-header-tied-augmented-sampled-layer-chain-imp-plain:*
assumes

trace-fri-bad-with-header-tied-augmented-sampled-layer-chain s out
shows *trace-fri-bad-with-header-tied-sampled-layer-chain s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table doms
layers
where *fri*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *header*:
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
and *partial*:
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *cand*:
partial-trace-table-candidate-with-recorded-first-fri-siblings
trace-table trace-openings trace-roots trace-bs fri-query-idxs
trace-round-layers
and *nonlow*: $\neg$ *trace-table-low-degree trace-table*
and *chain*:
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength – 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
unfolding
trace-fri-bad-with-header-tied-augmented-sampled-layer-chain-def
by *blast*
have *plain*: *partial-trace-table-candidate trace-table trace-openings*
by (*rule*
partial-trace-table-candidate-with-recorded-first-fri-siblingsD(1)
[OF cand])
show *?thesis*
unfolding *trace-fri-bad-with-header-tied-sampled-layer-chain-def*
by (*intro exI conjI*)
(rule fri, rule header, rule partial, rule plain, rule nonlow,
rule chain)
qed

lemma *trace-fri-bad-with-header-tied-sampled-layer-chain-imp-augmented*:
assumes *trace-fri-bad-with-header-tied-sampled-layer-chain s out*
shows *trace-fri-bad-with-header-tied-augmented-sampled-layer-chain s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table doms
layers
where *fri*:

accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final dg composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers
and header:
 verifier-header-transcript s fr trace-roots trace-final as dg
 composition-roots composition-final rest
and partial:
 accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings
and cand: partial-trace-table-candidate trace-table trace-openings
and nonlow: $\neg$ trace-table-low-degree trace-table
and chain:
 generic-fri-sampled-layer-chain-evidence trace-table-low-degree
 trace-table (clength - 1) trace-roots trace-bs trace-final
 fri-query-idxs trace-round-layers doms layers
unfolding trace-fri-bad-with-header-tied-sampled-layer-chain-def
by blast
have augmented:
 partial-trace-table-candidate-with-recorded-first-fri-siblings
 trace-table trace-openings trace-roots trace-bs fri-query-idxs
 trace-round-layers
by (rule
 generic-trace-sampled-layer-chain-evidence-augments-partial-trace-candidate
 [OF cand chain])
show ?thesis
unfolding
 trace-fri-bad-with-header-tied-augmented-sampled-layer-chain-def
by (intro exI conjI)
 (rule fri, rule header, rule partial, rule augmented, rule nonlow,
 rule chain)
qed

lemma wp-trace-fri-header-tied-augmented-sampled-layer-chain-bound-from-plain:
assumes sampled-bound:
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-sampled-layer-chain s) $s \leq R$
shows
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-augmented-sampled-layer-chain s) s
 $\leq R$
proof -
have wp-event verify-monad
 (trace-fri-bad-with-header-tied-augmented-sampled-layer-chain s) s
 $\leq$ wp-event verify-monad
 (trace-fri-bad-with-header-tied-sampled-layer-chain s) s
by (rule wp-event-mono)
 (rule
 trace-fri-bad-with-header-tied-augmented-sampled-layer-chain-imp-plain)
then show ?thesis

by (rule order-trans[*OF* - *sampled-bound*])
 qed

lemma *wp-trace-fri-header-tied-sampled-layer-chain-bound-from-augmented*:
 assumes *sampled-bound*:
 wp-event verify-monad
 (*trace-fri-bad-with-header-tied-augmented-sampled-layer-chain* *s*) *s*
 $\leq R$
 shows
 wp-event verify-monad
 (*trace-fri-bad-with-header-tied-sampled-layer-chain* *s*) *s* $\leq R$
proof –
 have *wp-event verify-monad*
 (*trace-fri-bad-with-header-tied-sampled-layer-chain* *s*) *s*
 $\leq$ *wp-event verify-monad*
 (*trace-fri-bad-with-header-tied-augmented-sampled-layer-chain* *s*) *s*
 by (rule *wp-event-mono*)
 (rule *trace-fri-bad-with-header-tied-sampled-layer-chain-imp-augmented*)
 then show ?thesis
 by (rule order-trans[*OF* - *sampled-bound*])
 qed

lemma *trace-fri-header-tied-augmented-missing-sampled-imp-assignment-obstruction*:
 assumes *missing*:
 trace-fri-header-tied-augmented-missing-sampled-layer-chain *s* *out*
 shows
 trace-fri-header-tied-sampled-layer-assignment-obstruction *s* *out*
proof –
 have *bad*:
 trace-fri-bad-with-header-tied-augmented-partial-candidate *s* *out*
 and *not-sampled*:
 $\neg$ *trace-fri-bad-with-header-tied-augmented-sampled-layer-chain* *s* *out*
 using *missing*
 unfolding *trace-fri-header-tied-augmented-missing-sampled-layer-chain-def*
 by blast+
from *bad* **obtain** *trace-roots* *trace-bs* *trace-final* *dg* *composition-roots*
 composition-bs *composition-final* *fri-query-idxs* *trace-round-layers*
 composition-round-layers *fr* *as* *rest* *trace-openings* *trace-table*
where *fri*:
 accepted-fri-opening-transcript *s* *out* *trace-roots* *trace-bs*
 trace-final *dg* *composition-roots* *composition-bs* *composition-final*
 fri-query-idxs *trace-round-layers* *composition-round-layers*
and *header*:
 verifier-header-transcript *s* *fr* *trace-roots* *trace-final* *as* *dg*
 composition-roots *composition-final* *rest*
and *partial*:
 accepted-with-partial-trace-openings *s* *out* *fr* *fri-query-idxs*
 trace-openings
and *augmented*:

```

    partial-trace-table-candidate-with-recorded-first-fri-siblings
    trace-table trace-openings trace-roots trace-bs fri-query-idxs
    trace-round-layers
and nonlow:  $\neg$  trace-table-low-degree trace-table
unfolding trace-fri-bad-with-header-tied-augmented-partial-candidate-def
by blast
have cand: partial-trace-table-candidate trace-table trace-openings
by (rule
    partial-trace-table-candidate-with-recorded-first-fri-siblingsD(1)
    [OF augmented])
have partial-generic:
    generic-fri-partial-evidence trace-table-low-degree trace-table
    (clength - 1) trace-roots trace-bs trace-final fri-query-idxs
    trace-round-layers
by (rule accepted-fri-opening-transcript-trace-generic-partial-evidence
    [OF fri])
have no-chain:
     $\bigwedge$  doms layers.  $\neg$ 
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers
proof
fix doms layers
assume chain:
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers
have trace-fri-bad-with-header-tied-augmented-sampled-layer-chain s out
unfolding
    trace-fri-bad-with-header-tied-augmented-sampled-layer-chain-def
by (intro exI conjI)
    (rule fri, rule header, rule partial, rule augmented, rule nonlow,
    rule chain)
then show False
using not-sampled by contradiction
qed
have obstruction:
    generic-fri-sampled-layer-assignment-obstruction
    trace-table-low-degree trace-table (clength - 1) trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers
unfolding generic-fri-sampled-layer-assignment-obstruction-def
using partial-generic no-chain by blast
show ?thesis
unfolding trace-fri-header-tied-sampled-layer-assignment-obstruction-def
by (intro exI conjI)
    (rule fri, rule header, rule partial, rule cand, rule nonlow,
    rule obstruction)
qed

```

lemma *wp-trace-fri-header-tied-augmented-missing-sampled-bound-from-assignment-obstruction:*

assumes *obstruction-bound:*

wp-event verify-monad
 $(\text{trace-fri-header-tied-sampled-layer-assignment-obstruction } s) \ s$
 $\leq M$

shows

wp-event verify-monad
 $(\text{trace-fri-header-tied-augmented-missing-sampled-layer-chain } s) \ s$
 $\leq M$

proof –

have *wp-event verify-monad*

$(\text{trace-fri-header-tied-augmented-missing-sampled-layer-chain } s) \ s$
 $\leq \text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-layer-assignment-obstruction } s) \ s$

by (rule *wp-event-mono*)

(rule
 $\text{trace-fri-header-tied-augmented-missing-sampled-imp-assignment-obstruction}$)

then show *?thesis*

by (rule *order-trans*[*OF* - *obstruction-bound*])

qed

lemma *wp-trace-fri-header-tied-augmented-partial-candidate-bound-from-sampled-and-missing:*

assumes *sampler-bound:*

wp-event verify-monad
 $(\text{trace-fri-bad-with-header-tied-augmented-sampled-layer-chain } s) \ s$
 $\leq R$

and *missing-bound:*

wp-event verify-monad
 $(\text{trace-fri-header-tied-augmented-missing-sampled-layer-chain } s) \ s$
 $\leq M$

shows

wp-event verify-monad
 $(\text{trace-fri-bad-with-header-tied-augmented-partial-candidate } s) \ s$
 $\leq R + M$

proof –

have *wp-event verify-monad*

$(\text{trace-fri-bad-with-header-tied-augmented-partial-candidate } s) \ s$
 $\leq \text{wp-event verify-monad}$
 $(\lambda out.$

$\text{trace-fri-bad-with-header-tied-augmented-sampled-layer-chain } s \ out \ \vee$
 $\text{trace-fri-header-tied-augmented-missing-sampled-layer-chain } s \ out) \ s$

unfolding *trace-fri-header-tied-augmented-missing-sampled-layer-chain-def*

by (rule *wp-event-mono*) *blast*

also have $\dots \leq$

wp-event verify-monad
 $(\text{trace-fri-bad-with-header-tied-augmented-sampled-layer-chain } s) \ s +$
wp-event verify-monad

$(\text{trace-fri-header-tied-augmented-missing-sampled-layer-chain } s) \ s$

by (rule *wp-event-union-bound*)

also have $\dots \leq R + M$
 by (rule *add-mono*[*OF sampled-bound missing-bound*])
 finally show *?thesis* .
 qed

lemma *wp-trace-fri-header-tied-augmented-partial-candidate-bound-from-sampled-conflict-and-zero*:
assumes *sampled-bound*:
 wp-event verify-monad
 (*trace-fri-bad-with-header-tied-augmented-sampled-layer-chain s*) *s*
 $\leq R$
and *conflict-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-sampled-assignment-conflict s*) *s* $\leq C$
and *zero-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-zero-round-final-obstruction s*) *s* $\leq Z$
shows
 wp-event verify-monad
 (*trace-fri-bad-with-header-tied-augmented-partial-candidate s*) *s*
 $\leq R + (C + Z)$
proof –
have *obstruction-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-sampled-layer-assignment-obstruction s*) *s*
 $\leq C + Z$
by (rule
 wp-trace-fri-header-tied-assignment-obstruction-bound-from-conflict-and-zero
 [*OF conflict-bound zero-bound*])
have *missing-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-augmented-missing-sampled-layer-chain s*) *s*
 $\leq C + Z$
by (rule
 wp-trace-fri-header-tied-augmented-missing-sampled-bound-from-assignment-obstruction
 [*OF obstruction-bound*])
show *?thesis*
by (rule
 wp-trace-fri-header-tied-augmented-partial-candidate-bound-from-sampled-and-missing
 [*OF sampled-bound missing-bound*])
 qed

lemma *trace-fri-header-tied-augmented-partial-candidate-imp-sampled-conflict-or-zero*:
assumes *bad*:
 trace-fri-bad-with-header-tied-augmented-partial-candidate s out
shows
 trace-fri-bad-with-header-tied-augmented-sampled-layer-chain s out $\vee$
 trace-fri-header-tied-sampled-assignment-conflict s out $\vee$
 trace-fri-header-tied-zero-round-final-obstruction s out
proof (cases

```

    trace-fri-bad-with-header-tied-augmented-sampled-layer-chain s out)
  case True
  then show ?thesis by blast
next
case False
have missing:
  trace-fri-header-tied-augmented-missing-sampled-layer-chain s out
  unfolding trace-fri-header-tied-augmented-missing-sampled-layer-chain-def
  using bad False by blast
have obstruction:
  trace-fri-header-tied-sampled-layer-assignment-obstruction s out
  by (rule
    trace-fri-header-tied-augmented-missing-sampled-imp-assignment-obstruction
    [OF missing])
then show ?thesis
  using trace-fri-header-tied-assignment-obstruction-imp-conflict-or-zero
  by blast
qed

```

lemma *trace-fri-header-tied-recorded-sibling-candidate-mismatch-false:*

$\neg$ *trace-fri-header-tied-recorded-sibling-candidate-mismatch s out*

proof

assume *bad*:

trace-fri-header-tied-recorded-sibling-candidate-mismatch s out

from *bad* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table
round-idx xp xp-path xn xn-path result final-state **where**

fri-openings:

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers

and *agrees*:

trace-table-agrees-with-recorded-first-fri-siblings trace-table
trace-roots trace-bs fri-query-idxs trace-round-layers

and *round-bound*: $\text{round-idx} < \text{length } \text{fri-query-idxs}$

and *bs-nonempty*: $0 < \text{length } \text{trace-bs}$

and *step*:

fri-layer-step-evidence

$(\text{trace-roots} \ ! \ 0)$

$(\text{trace-bs} \ ! \ 0)$

$(\text{fri-evidence-layer-len } \text{trace-roots} \ 0)$

$(\text{fri-evidence-layer-idx } \text{trace-roots} \ \text{fri-query-idxs} \ \text{round-idx} \ 0)$

1

$(\text{fri-sibling-index } (\text{fri-evidence-layer-len } \text{trace-roots} \ 0)$

$(\text{fri-evidence-layer-idx } \text{trace-roots} \ \text{fri-query-idxs} \ \text{round-idx} \ 0))$

xp xp-path xn xn-path

$(\text{fri-evidence-next-idx } \text{trace-roots} \ \text{fri-query-idxs} \ \text{round-idx} \ 0)$

$(\text{fri-evidence-next-value } \text{trace-roots} \ \text{trace-bs} \ \text{fri-query-idxs})$

```

    round-idx 0 xp xn)
  (trace-round-layers ! round-idx ! 0)
and neq:
  trace-table !
  fri-sibling-index (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
  ≠ xn
unfolding trace-fri-header-tied-recorded-sibling-candidate-mismatch-def
by blast
have roots-nonempty: 0 < length trace-roots
using accepted-fri-opening-transcript-shapes(2)[OF fri-openings]
bs-nonempty by simp
have eq:
  trace-table !
  fri-sibling-index (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0) =
  xn
using agrees round-bound roots-nonempty bs-nonempty step
unfolding trace-table-agrees-with-recorded-first-fri-siblings-def
by blast
show False
using eq neq by simp
qed

lemma trace-fri-header-tied-value-aligned-sibling-mismatch-imp-recorded-sibling-missing:
  assumes mismatch:
    trace-fri-header-tied-value-aligned-sibling-mismatch s out
  shows
    trace-fri-header-tied-recorded-sibling-candidate-missing s out
proof -
  from mismatch obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as rest trace-openings trace-table
    round-idx xp xp-path xn xn-path result final-state where
    out-eq: out = Some (result, final-state)
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers
  and header:
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest
  and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  and cand: partial-trace-table-candidate trace-table trace-openings
  and nonlow: ¬ trace-table-low-degree trace-table
  and round-bound: round-idx < length fri-query-idxs
  and bs-nonempty: 0 < length trace-bs

```

```

and openings-nonempty: trace-openings ! round-idx ≠ []
and hd-idx:
  opening-index (hd (trace-openings ! round-idx)) =
    fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0
and hd-value:
  opening-value (hd (trace-openings ! round-idx)) = xp
and step:
  fri-layer-step-evidence
    (trace-roots ! 0)
    (trace-bs ! 0)
    (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    1
    (fri-sibling-index (fri-evidence-layer-len trace-roots 0)
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
    xp xp-path xn xn-path
    (fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
    (fri-evidence-next-value trace-roots trace-bs fri-query-idxs
      round-idx 0 xp xn)
    (trace-round-layers ! round-idx ! 0)
and neg:
  trace-table !
    fri-sibling-index (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    ≠ xn
unfolding trace-fri-header-tied-value-aligned-sibling-mismatch-def
by auto
have roots-nonempty: 0 < length trace-roots
using accepted-fri-opening-transcript-shapes(2)[OF fri-openings]
  bs-nonempty by simp
have missing:
  ¬ trace-table-agrees-with-recorded-first-fri-siblings trace-table
    trace-roots trace-bs fri-query-idxs trace-round-layers
unfolding trace-table-agrees-with-recorded-first-fri-siblings-def
using round-bound roots-nonempty bs-nonempty step neg by blast
show ?thesis
unfolding trace-fri-header-tied-recorded-sibling-candidate-missing-def
by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,
    rule missing, rule nonlow, rule round-bound, rule bs-nonempty,
    rule openings-nonempty, rule hd-idx, rule hd-value, rule step, rule neg)
qed

lemma trace-fri-header-tied-sibling-mismatch-structural-gap-imp-recorded-sibling-missing:
assumes gap: trace-fri-header-tied-sibling-mismatch-structural-gap s out
shows trace-fri-header-tied-recorded-sibling-candidate-missing s out
using gap
unfolding trace-fri-header-tied-sibling-mismatch-structural-gap-def
by (blast intro:

```

trace-fri-header-tied-value-aligned-sibling-mismatch-imp-recorded-sibling-missing)

lemma *trace-fri-header-tied-unaugmented-partial-candidate-imp-missing-or-base-gap:*

assumes *unaugmented:*

trace-fri-header-tied-unaugmented-partial-candidate s out

shows

trace-fri-header-tied-recorded-sibling-candidate-missing s out $\vee$

trace-fri-header-tied-base-value-alignment-gap s out

proof —

from *unaugmented* **have** *plain:*

trace-fri-bad-with-header-tied-partial-candidate s out

and *not-augmented:*

$\neg$ *trace-fri-bad-with-header-tied-augmented-partial-candidate s out*

unfolding *trace-fri-header-tied-unaugmented-partial-candidate-def*

by *simp-all*

from *plain* **obtain** *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as rest trace-openings trace-table

where *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final dg composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers

and *header:*

verifier-header-transcript s fr trace-roots trace-final as dg

composition-roots composition-final rest

and *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs

trace-openings

and *cand: partial-trace-table-candidate trace-table trace-openings*

and *not-low: $\neg$ trace-table-low-degree trace-table*

unfolding *trace-fri-bad-with-header-tied-partial-candidate-def*

by *blast*

have *not-agrees:*

$\neg$ *trace-table-agrees-with-recorded-first-fri-siblings trace-table*

trace-roots trace-bs fri-query-idxs trace-round-layers

proof

assume *agrees:*

trace-table-agrees-with-recorded-first-fri-siblings trace-table

trace-roots trace-bs fri-query-idxs trace-round-layers

have *augmented-candidate:*

partial-trace-table-candidate-with-recorded-first-fri-siblings

trace-table trace-openings trace-roots trace-bs fri-query-idxs

trace-round-layers

unfolding

partial-trace-table-candidate-with-recorded-first-fri-siblings-def

by (*intro conjI cand agrees*)

have *trace-fri-bad-with-header-tied-augmented-partial-candidate s out*

unfolding *trace-fri-bad-with-header-tied-augmented-partial-candidate-def*

by (*intro exI conjI*)

```

    (rule fri-openings, rule header, rule partial,
     rule augmented-candidate, rule not-low)
  then show False
    using not-augmented by contradiction
qed
from not-agrees obtain round-idx xp xp-path xn xn-path where
  round-bound: round-idx < length fri-query-idxs
  and roots-nonempty: 0 < length trace-roots
  and trace-bs-nonempty: 0 < length trace-bs
  and step:
    fri-layer-step-evidence
      (trace-roots ! 0)
      (trace-bs ! 0)
      (fri-evidence-layer-len trace-roots 0)
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
      1
      (fri-sibling-index (fri-evidence-layer-len trace-roots 0)
        (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
      xp xp-path xn xn-path
      (fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
      (fri-evidence-next-value trace-roots trace-bs fri-query-idxs
        round-idx 0 xp xn)
      (trace-round-layers ! round-idx ! 0)
  and sibling-neq:
    trace-table !
    fri-sibling-index (fri-evidence-layer-len trace-roots 0)
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    ≠ xn
  unfolding trace-table-agrees-with-recorded-first-fri-siblings-def
  by blast
have no-match:
  ¬ fri-opening-matches-table (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    trace-table xp xn
proof
  assume match:
    fri-opening-matches-table (fri-evidence-layer-len trace-roots 0)
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    trace-table xp xn
  have trace-table !
    fri-sibling-index (fri-evidence-layer-len trace-roots 0)
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0) =
    xn
  by (rule fri-opening-matches-tableD(4)[OF match])
  then show False
    using sibling-neq by contradiction
qed
have generic-base:
  generic-fri-sampled-base-opening-conflict trace-table trace-roots

```

```

    trace-bs fri-query-idxs trace-round-layers
unfolding generic-fri-sampled-base-opening-conflict-def
by (intro exI conjI)
    (rule trace-bs-nonempty, rule round-bound, rule step, rule no-match)
have sampled-base:
    trace-fri-header-tied-sampled-base-opening-conflict s out
unfolding trace-fri-header-tied-sampled-base-opening-conflict-def
by (intro exI conjI)
    (rule fri-openings, rule header, rule partial, rule cand, rule not-low,
     rule generic-base)
show ?thesis
proof (cases
    trace-fri-header-tied-value-aligned-base-opening-conflict s out)
case True
have trace-fri-header-tied-value-aligned-sibling-mismatch s out
    by (rule trace-fri-header-tied-value-aligned-base-imp-sibling-mismatch
    [OF True])
then show ?thesis
    by (blast intro:
    trace-fri-header-tied-value-aligned-sibling-mismatch-imp-recorded-sibling-missing)
next
case False
then have trace-fri-header-tied-base-value-alignment-gap s out
    using sampled-base
    unfolding trace-fri-header-tied-base-value-alignment-gap-def
    by simp
then show ?thesis by simp
qed
qed

lemma wp-trace-fri-header-tied-unaugmented-partial-candidate-bound-from-recorded-missing-and-base-gap:
fixes M G :: prob
assumes missing-bound:
    wp-event verify-monad
    (trace-fri-header-tied-recorded-sibling-candidate-missing s) s ≤ M
and gap-bound:
    wp-event verify-monad
    (trace-fri-header-tied-base-value-alignment-gap s) s ≤ G
shows
    wp-event verify-monad
    (trace-fri-header-tied-unaugmented-partial-candidate s) s ≤ M + G
proof –
have wp-event verify-monad
    (trace-fri-header-tied-unaugmented-partial-candidate s) s ≤
    wp-event verify-monad
    (λout.
    trace-fri-header-tied-recorded-sibling-candidate-missing s out ∨
    trace-fri-header-tied-base-value-alignment-gap s out) s
by (rule wp-event-mono)

```

```

(rule
  trace-fri-header-tied-unaugmented-partial-candidate-imp-missing-or-base-gap)
also have ... ≤
  wp-event verify-monad
    (trace-fri-header-tied-recorded-sibling-candidate-missing s) s +
  wp-event verify-monad
    (trace-fri-header-tied-base-value-alignment-gap s) s
by (rule wp-event-union-bound)
also have ... ≤ M + G
by (rule add-mono[OF missing-bound gap-bound])
finally show ?thesis .
qed

lemma wp-trace-fri-header-tied-sibling-structural-gap-bound-from-recorded-sibling-missing:
  assumes missing-bound:
    wp-event verify-monad
      (trace-fri-header-tied-recorded-sibling-candidate-missing s) s ≤ M
  shows
    wp-event verify-monad
      (trace-fri-header-tied-sibling-mismatch-structural-gap s) s ≤ M
proof -
  have wp-event verify-monad
    (trace-fri-header-tied-sibling-mismatch-structural-gap s) s ≤
    wp-event verify-monad
      (trace-fri-header-tied-recorded-sibling-candidate-missing s) s
  by (rule wp-event-mono)
  (rule
    trace-fri-header-tied-sibling-mismatch-structural-gap-imp-recorded-sibling-missing)
  then show ?thesis
  by (rule order-trans[OF - missing-bound])
qed

lemma trace-fri-header-tied-sibling-structural-gap-without-recorded-missing-false:
  assumes gap: trace-fri-header-tied-sibling-mismatch-structural-gap s out
  and not-missing:
    ¬ trace-fri-header-tied-recorded-sibling-candidate-missing s out
  shows False
using trace-fri-header-tied-sibling-mismatch-structural-gap-imp-recorded-sibling-missing
  [OF gap] not-missing
by contradiction

lemma wp-trace-fri-header-tied-sibling-structural-gap-without-recorded-missing-zero:
  wp-event verify-monad
    (λout. trace-fri-header-tied-sibling-mismatch-structural-gap s out ∧
      ¬ trace-fri-header-tied-recorded-sibling-candidate-missing s out) s
    ≤ 0
proof -
  have wp-event verify-monad
    (λout. trace-fri-header-tied-sibling-mismatch-structural-gap s out ∧

```


$\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s$
 $\leq \text{wp-event verify-monad } (\lambda\cdot. \text{False}) \ s$
by (rule wp-event-mono)
(auto dest:
 $\text{trace-fri-header-tied-sibling-structural-gap-without-recorded-missing-false}$)
also have ... = 0
unfolding wp-event-def wp-def dist-expect-def **by** simp
finally show ?thesis .
qed

lemma trace-fri-header-tied-value-aligned-sibling-mismatch-without-recorded-missing-imp-partial-merkle:
assumes mismatch:
 $\text{trace-fri-header-tied-value-aligned-sibling-mismatch } s \text{ out}$
and not-missing:
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}$
shows partial-merkle-inconsistency-bad $s \text{ out}$
proof –
have split:
 $\text{partial-merkle-inconsistency-bad } s \text{ out} \vee$
 $\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s \text{ out}$
by (rule
 $\text{trace-fri-header-tied-value-aligned-sibling-mismatch-imp-partial-merkle-or-structural-gap}$
[OF mismatch])
then show ?thesis
using not-missing
 $\text{trace-fri-header-tied-sibling-mismatch-structural-gap-imp-recorded-sibling-missing}$
by blast
qed

lemma wp-trace-fri-header-tied-value-aligned-sibling-mismatch-without-recorded-missing-bound-from-partial-merkle:
assumes merkle-bound:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq P$
shows
 $\text{wp-event verify-monad}$
 $(\lambda \text{out. } \text{trace-fri-header-tied-value-aligned-sibling-mismatch } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s$
 $\leq P$
proof –
have wp-event verify-monad
 $(\lambda \text{out. } \text{trace-fri-header-tied-value-aligned-sibling-mismatch } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s$
 $\leq \text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s$
by (rule wp-event-mono)
(auto dest:
 $\text{trace-fri-header-tied-value-aligned-sibling-mismatch-without-recorded-missing-imp-partial-merkle}$)
then show ?thesis
by (rule order-trans[OF - merkle-bound])
qed

lemma *wp-trace-fri-header-tied-value-aligned-base-without-recorded-missing-bound-from-partial-merkle:*
assumes *merkle-bound:*
wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ P
shows
wp-event verify-monad
 $(\lambda out.$
 $\text{trace-fri-header-tied-value-aligned-base-opening-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) s$
 $\leq P$
proof –
have *wp-event verify-monad*
 $(\lambda out.$
 $\text{trace-fri-header-tied-value-aligned-base-opening-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) s$
 $\leq \text{wp-event verify-monad}$
 $(\lambda out. \text{trace-fri-header-tied-value-aligned-sibling-mismatch } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) s$
by (*rule wp-event-mono*)
 $(\text{auto intro: trace-fri-header-tied-value-aligned-base-imp-sibling-mismatch})$
also have $\dots \leq P$
by (*rule*
wp-trace-fri-header-tied-value-aligned-sibling-mismatch-without-recorded-missing-bound-from-partial-merkle
 $[OF \text{ merkle-bound}]$)
finally show *?thesis* .
qed

lemma *wp-trace-fri-header-tied-sampled-base-without-recorded-missing-bound-from-partial-merkle-and-gap:*
fixes $P \ G :: \text{prob}$
assumes *merkle-bound:*
wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ P
and *gap-bound:*
wp-event verify-monad
 $(\text{trace-fri-header-tied-base-value-alignment-gap } s) s \leq G$
shows
wp-event verify-monad
 $(\lambda out.$
 $\text{trace-fri-header-tied-sampled-base-opening-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) s$
 $\leq P + G$
proof –
have *wp-event verify-monad*
 $(\lambda out.$
 $\text{trace-fri-header-tied-sampled-base-opening-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) s$
 $\leq \text{wp-event verify-monad}$
 $(\lambda out.$
 $(\text{trace-fri-header-tied-value-aligned-base-opening-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \vee$
 $\text{trace-fri-header-tied-base-value-alignment-gap } s \text{ out}) s$

by (rule *wp-event-mono*)
 (auto dest:
 trace-fri-header-tied-sampled-base-imp-value-aligned-or-gap)
also have ... $\leq$
 wp-event verify-monad
 (λ out.
 trace-fri-header-tied-value-aligned-base-opening-conflict s out $\wedge$
 $\neg$ *trace-fri-header-tied-recorded-sibling-candidate-missing s out*) *s* +
 wp-event verify-monad
 (*trace-fri-header-tied-base-value-alignment-gap s*) *s*
by (rule *wp-event-union-bound*)
also have ... $\leq P + G$
by (intro *add-mono gap-bound*
 wp-trace-fri-header-tied-value-aligned-base-without-recorded-missing-bound-from-partial-merkle
 [*OF merkle-bound*])
finally show ?thesis .
qed

lemma *trace-fri-header-tied-recorded-sibling-candidate-missingE*:

assumes *missing*:

trace-fri-header-tied-recorded-sibling-candidate-missing s out

obtains *result final-state trace-roots trace-bs trace-final dg*
 composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers fr as rest trace-openings
 trace-table round-idx xp xp-path xn xn-path

where

out = *Some (result, final-state)*

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
 dg composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers

verifier-header-transcript s fr trace-roots trace-final as dg
 composition-roots composition-final rest

accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings

partial-trace-table-candidate trace-table trace-openings
 $\neg$ *trace-table-agrees-with-recorded-first-fri-siblings trace-table*
 trace-roots trace-bs fri-query-idxs trace-round-layers

$\neg$ *trace-table-low-degree trace-table*

round-idx < *length fri-query-idxs*

 0 < *length trace-bs*

trace-openings ! *round-idx* $\neq$ []
 opening-index (*hd (trace-openings ! round-idx)*) =

fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0

opening-value (*hd (trace-openings ! round-idx)*) = *xp*

fri-layer-step-evidence

 (*trace-roots* ! 0)

 (*trace-bs* ! 0)

 (*fri-evidence-layer-len trace-roots* 0)

 (*fri-evidence-layer-idx trace-roots fri-query-idxs round-idx* 0)

```

1
(fri-sibling-index (fri-evidence-layer-len trace-roots 0)
 (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
xp xp-path xn xn-path
(fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
(fri-evidence-next-value trace-roots trace-bs fri-query-idxs
 round-idx 0 xp xn)
(trace-round-layers ! round-idx ! 0)
trace-table !
fri-sibling-index (fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
≠ xn
using missing
unfolding trace-fri-header-tied-recorded-sibling-candidate-missing-def
apply (elim exE conjE)
apply (rule that)
apply assumption+
done

```

lemma *trace-fri-header-tied-recorded-sibling-candidate-missing-no-same-sampled-chain:*

```

assumes missing:
  trace-fri-header-tied-recorded-sibling-candidate-missing s out
obtains result final-state trace-roots trace-bs trace-final dg
  composition-roots composition-bs composition-final fri-query-idxs
  trace-round-layers composition-round-layers fr as rest trace-openings
  trace-table round-idx xp xp-path xn xn-path
where
  out = Some (result, final-state)
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers
  verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest
  accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  partial-trace-table-candidate trace-table trace-openings
  ¬ trace-table-low-degree trace-table
  ∧ doms layers.
  ¬ generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers

```

proof –

from *missing* **show** *thesis*

proof (rule *trace-fri-header-tied-recorded-sibling-candidate-missingE*)

```

fix result final-state trace-roots trace-bs trace-final dg
  composition-roots composition-bs composition-final fri-query-idxs
  trace-round-layers composition-round-layers fr as rest trace-openings
  trace-table round-idx xp xp-path xn xn-path

```

assume *out-eq*: out = Some (result, final-state)

and *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *header*:
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
and *partial*:
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate*:
partial-trace-table-candidate trace-table trace-openings
and *no-agree*:
 $\neg$ *trace-table-agrees-with-recorded-first-fri-siblings trace-table*
trace-roots trace-bs fri-query-idxs trace-round-layers
and *nonlow*: $\neg$ *trace-table-low-degree trace-table*
have *no-chain*:
 $\bigwedge$ *doms layers*.
 $\neg$ *generic-fri-sampled-layer-chain-evidence trace-table-low-degree*
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
proof
fix *doms layers*
assume *chain*:
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
have *trace-table-agrees-with-recorded-first-fri-siblings trace-table*
trace-roots trace-bs fri-query-idxs trace-round-layers
by (rule
generic-trace-sampled-layer-chain-evidence-recorded-first-sibling-agreement
[*OF chain*])
then show *False*
using *no-agree* **by** *contradiction*
qed
show *thesis*
by (rule *that*[*OF out-eq fri-openings header partial candidate nonlow*
no-chain])
qed
qed

lemma *trace-fri-header-tied-recorded-sibling-candidate-missing-imp-assignment-obstruction*:
assumes *missing*:
trace-fri-header-tied-recorded-sibling-candidate-missing s out
shows
trace-fri-header-tied-sampled-layer-assignment-obstruction s out
proof (rule
trace-fri-header-tied-recorded-sibling-candidate-missing-no-same-sampled-chain
[*OF missing*])

fix *result final-state trace-roots trace-bs trace-final dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as rest trace-openings
trace-table round-idx xp xp-path xn xn-path
assume *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *cand: partial-trace-table-candidate trace-table trace-openings*
and *not-low: $\neg$ trace-table-low-degree trace-table*
and *no-chain:*
 $\bigwedge$ *doms layers.*
 $\neg$ *generic-fri-sampled-layer-chain-evidence trace-table-low-degree*
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
have *partial-generic:*
generic-fri-partial-evidence trace-table-low-degree trace-table
(clength - 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers
by (*rule accepted-fri-opening-transcript-trace-generic-partial-evidence*
[OF fri-openings])
have *obstruction:*
generic-fri-sampled-layer-assignment-obstruction
trace-table-low-degree trace-table (clength - 1) trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers
unfolding *generic-fri-sampled-layer-assignment-obstruction-def*
using *partial-generic no-chain* **by** *blast*
show *trace-fri-header-tied-sampled-layer-assignment-obstruction s out*
unfolding *trace-fri-header-tied-sampled-layer-assignment-obstruction-def*
by (*intro exI conjI*)
(rule fri-openings, rule header, rule partial, rule cand,
rule not-low, rule obstruction)
qed

lemma *trace-fri-header-tied-recorded-sibling-candidate-missing-imp-conflict-or-zero:*
assumes *missing:*
trace-fri-header-tied-recorded-sibling-candidate-missing s out
shows
trace-fri-header-tied-sampled-assignment-conflict s out $\vee$
trace-fri-header-tied-zero-round-final-obstruction s out
by (*rule trace-fri-header-tied-assignment-obstruction-imp-conflict-or-zero*)
(rule
trace-fri-header-tied-recorded-sibling-candidate-missing-imp-assignment-obstruction

[*OF missing*])

lemma *wp-trace-fri-header-tied-recorded-sibling-candidate-missing-bound-from-conflict-and-zero*:

assumes *conflict-bound*:

wp-event verify-monad

(*trace-fri-header-tied-sampled-assignment-conflict s*) $s \leq C$

and *zero-bound*:

wp-event verify-monad

(*trace-fri-header-tied-zero-round-final-obstruction s*) $s \leq Z$

shows

wp-event verify-monad

(*trace-fri-header-tied-recorded-sibling-candidate-missing s*) $s \leq C + Z$

proof –

have *wp-event verify-monad*

(*trace-fri-header-tied-recorded-sibling-candidate-missing s*) $s \leq$

wp-event verify-monad

($\lambda out. trace-fri-header-tied-sampled-assignment-conflict s out \vee$
trace-fri-header-tied-zero-round-final-obstruction s out) s

by (*rule wp-event-mono*)

(*rule*

trace-fri-header-tied-recorded-sibling-candidate-missing-imp-conflict-or-zero)

also have ... $\leq$

wp-event verify-monad

(*trace-fri-header-tied-sampled-assignment-conflict s*) $s +$

wp-event verify-monad

(*trace-fri-header-tied-zero-round-final-obstruction s*) s

by (*rule wp-event-union-bound*)

also have ... $\leq C + Z$

by (*rule add-mono[OF conflict-bound zero-bound]*)

finally show *?thesis* .

qed

end

end

theory *Soundness-FRI-Staged-Replay-Bounds*

imports

Soundness-FRI-Staged-Active-Bounds

Soundness-FRI-Replay-Gap-Assignment

Soundness-FRI-Composition-Replay-Gaps

Soundness-FRI-Trace-Replay-Gaps

Soundness-FRI-Trace-Sibling-Evidence

begin

Staged active FRI bounds that additionally use the replay-gap assignment adapters. This theory is deliberately downstream of both the staged prequery-accounting layer

and the replay-gap layer.

context *soundness*

begin

lemma *wp-trace-fri-header-tied-without-same-without-recorded-missing-bound-from-partial-merkle-and-replay-gaps:*

fixes $P\ G\ N\ S\ F :: \text{prob}$

assumes *merkle-bound:*

wp-event verify-monad (*partial-merkle-inconsistency-bad* s) $s \leq P$

and *alignment-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-base-value-alignment-gap* s) $s \leq G$

and *next-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-next-value-replay-gap* s) $s \leq N$

and *successor-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-successor-replay-gap* s) $s \leq S$

and *final-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-final-value-replay-gap* s) $s \leq F$

shows

wp-event verify-monad

($\lambda out.$

trace-fri-header-tied-sampled-assignment-conflict-without-same-layer

$s\ out \wedge$

$\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s\ out) s$

$\leq (P + G) + N + S + F$

proof –

have *base-bound:*

wp-event verify-monad

($\lambda out.$

trace-fri-header-tied-sampled-base-opening-conflict $s\ out \wedge$

$\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s\ out) s$

$\leq P + G$

by (*rule*

wp-trace-fri-header-tied-sampled-base-without-recorded-missing-bound-from-partial-merkle-and-gap

[*OF merkle-bound alignment-gap-bound*])

have *wp-event verify-monad*

($\lambda out.$

trace-fri-header-tied-sampled-assignment-conflict-without-same-layer

$s\ out \wedge$

$\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s\ out) s$

$\leq \text{wp-event verify-monad}$

($\lambda out.$

(*trace-fri-header-tied-sampled-base-opening-conflict* $s\ out \wedge$

$\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s\ out) \vee$

trace-fri-header-tied-sampled-next-value-conflict $s\ out \vee$

trace-fri-header-tied-sampled-successor-opening-conflict $s\ out \vee$

trace-fri-header-tied-sampled-final-value-conflict $s\ out) s$

by (*rule wp-event-mono*)
 (*auto dest: trace-fri-header-tied-without-same-imp-branch*)
also have ... $\leq$
wp-event verify-monad
 ($\lambda out.$
 trace-fri-header-tied-sampled-base-opening-conflict s out $\wedge$
 $\neg$ *trace-fri-header-tied-recorded-sibling-candidate-missing s out*) *s* +
wp-event verify-monad
 (*trace-fri-header-tied-sampled-next-value-conflict s*) *s* +
wp-event verify-monad
 (*trace-fri-header-tied-sampled-successor-opening-conflict s*) *s* +
wp-event verify-monad
 (*trace-fri-header-tied-sampled-final-value-conflict s*) *s*
by (*rule wp-event-union-bound₄*)
also have ... $\leq (P + G) + N + S + F$
by (*intro add-mono base-bound*
 wp-trace-fri-header-tied-sampled-next-value-bound-from-replay-gap
 [*OF next-gap-bound*]
 wp-trace-fri-header-tied-sampled-successor-bound-from-replay-gap
 [*OF successor-gap-bound*]
 wp-trace-fri-header-tied-sampled-final-value-bound-from-replay-gap
 [*OF final-gap-bound*])
finally show ?thesis .
qed

lemma *wp-trace-fri-header-tied-authenticated-conflict-without-recorded-missing-bound-from-partial-merkle-and-replay*
fixes *P G N S F :: prob*
assumes *merkle-bound:*
 wp-event verify-monad (*partial-merkle-inconsistency-bad s*) *s* $\leq P$
and *alignment-gap-bound:*
 wp-event verify-monad
 (*trace-fri-header-tied-base-value-alignment-gap s*) *s* $\leq G$
and *next-gap-bound:*
 wp-event verify-monad
 (*trace-fri-header-tied-next-value-replay-gap s*) *s* $\leq N$
and *successor-gap-bound:*
 wp-event verify-monad
 (*trace-fri-header-tied-successor-replay-gap s*) *s* $\leq S$
and *final-gap-bound:*
 wp-event verify-monad
 (*trace-fri-header-tied-final-value-replay-gap s*) *s* $\leq F$
shows
 wp-event verify-monad
 ($\lambda out.$
 trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
 s out $\wedge$
 $\neg$ *trace-fri-header-tied-recorded-sibling-candidate-missing s out*) *s*
 $\leq ((P + G) + N + S + F) + P$
proof –

have *without-bound*:
wp-event verify-monad
 $(\lambda out.$
 $\text{trace-fri-header-tied-sampled-assignment-conflict-without-same-layer}$
 $s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s$
 $\leq (P + G) + N + S + F$
by (*rule*
wp-trace-fri-header-tied-without-same-without-recorded-missing-bound-from-partial-merkle-and-replay-gaps
 $[OF \text{ merkle-bound alignment-gap-bound next-gap-bound successor-gap-bound}$
 $\text{final-gap-bound}]$)
have *wp-event verify-monad*
 $(\lambda out.$
 $\text{trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer}$
 $s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s$
 $\leq \text{wp-event verify-monad}$
 $(\lambda out.$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict-without-same-layer}$
 $s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \vee$
 $\text{trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks}$
 $s \text{ out}) \ s$
by (*rule wp-event-mono*)
 $(\text{auto dest:}$
 $\text{trace-fri-header-tied-authenticated-conflict-imp-without-same-or-same-layer})$
also have $\dots \leq$
wp-event verify-monad
 $(\lambda out.$
 $\text{trace-fri-header-tied-sampled-assignment-conflict-without-same-layer}$
 $s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s +$
wp-event verify-monad
 $(\text{trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks } s)$
 s
by (*rule wp-event-union-bound*)
also have $\dots \leq ((P + G) + N + S + F) + P$
by (*intro add-mono without-bound*
 $\text{wp-trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks-bound-from-partial-merkle}$
 $[OF \text{ merkle-bound}]$)
finally show *?thesis* .
qed

lemma *wp-trace-fri-header-tied-sampled-assignment-conflict-without-recorded-missing-bound-from-auth-gap-and-repla*
fixes $A \ P \ G \ N \ S \ F :: \text{prob}$
assumes *auth-gap-bound*:
wp-event verify-monad
 $(\text{trace-fri-header-tied-assignment-auth-gap } s) \ s \leq A$
and *merkle-bound*:

$wp\text{-event verify-monad } (partial\text{-merkle-inconsistency-bad } s) \ s \leq P$
and $alignment\text{-gap-bound}$:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-base-value-alignment-gap } s) \ s \leq G$
and $next\text{-gap-bound}$:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-next-value-replay-gap } s) \ s \leq N$
and $successor\text{-gap-bound}$:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-successor-replay-gap } s) \ s \leq S$
and $final\text{-gap-bound}$:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-final-value-replay-gap } s) \ s \leq F$
shows
 $wp\text{-event verify-monad}$
 $(\lambda out.$
 $\quad trace\text{-fri-header-tied-sampled-assignment-conflict } s \ out \ \wedge$
 $\quad \neg \ trace\text{-fri-header-tied-recorded-sibling-candidate-missing } s \ out) \ s$
 $\leq (((P + G) + N + S + F) + P) + A$
proof –
have $auth\text{-bound}$:
 $wp\text{-event verify-monad}$
 $(\lambda out.$
 $\quad trace\text{-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer}$
 $\quad \quad s \ out \ \wedge$
 $\quad \neg \ trace\text{-fri-header-tied-recorded-sibling-candidate-missing } s \ out) \ s$
 $\leq ((P + G) + N + S + F) + P$
by (rule
 $wp\text{-trace-fri-header-tied-authenticated-conflict-without-recorded-missing-bound-from-partial-merkle-and-replay-gap}$
 $[OF \ merkle\text{-bound } alignment\text{-gap-bound } next\text{-gap-bound } successor\text{-gap-bound}$
 $\quad final\text{-gap-bound}]$)
have $wp\text{-event verify-monad}$
 $(\lambda out.$
 $\quad trace\text{-fri-header-tied-sampled-assignment-conflict } s \ out \ \wedge$
 $\quad \neg \ trace\text{-fri-header-tied-recorded-sibling-candidate-missing } s \ out) \ s$
 $\leq wp\text{-event verify-monad}$
 $(\lambda out.$
 $\quad (trace\text{-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer}$
 $\quad \quad s \ out \ \wedge$
 $\quad \neg \ trace\text{-fri-header-tied-recorded-sibling-candidate-missing } s \ out) \ \vee$
 $\quad trace\text{-fri-header-tied-assignment-auth-gap } s \ out) \ s$
by (rule $wp\text{-event-mono}$)
 $(auto \ dest: \ trace\text{-fri-header-tied-assignment-imp-authenticated-or-auth-gap})$
also have $\dots \leq$
 $wp\text{-event verify-monad}$
 $(\lambda out.$
 $\quad trace\text{-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer}$
 $\quad \quad s \ out \ \wedge$
 $\quad \neg \ trace\text{-fri-header-tied-recorded-sibling-candidate-missing } s \ out) \ s +$

$wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}header\text{-}tied\text{-}assignment\text{-}auth\text{-}gap\ s)\ s$
by $(rule\ wp\text{-}event\text{-}union\text{-}bound)$
also have $\dots \leq (((P + G) + N + S + F) + P) + A$
by $(intro\ add\text{-}mono\ auth\text{-}bound\ auth\text{-}gap\text{-}bound)$
finally show $?thesis$.
qed

lemma $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\text{-}bound\text{-}from\text{-}recorded\text{-}missing\text{-}and\text{-}complement$:
fixes $Missing\ Complement :: prob$
assumes $missing\text{-}bound$:
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}header\text{-}tied\text{-}recorded\text{-}sibling\text{-}candidate\text{-}missing\ s)\ s$
 $\leq Missing$
and $complement\text{-}bound$:
 $wp\text{-}event\ verify\text{-}monad$
 $(\lambda out.$
 $\quad trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\ s\ out \wedge$
 $\quad \neg trace\text{-}fri\text{-}header\text{-}tied\text{-}recorded\text{-}sibling\text{-}candidate\text{-}missing\ s\ out)\ s$
 $\leq Complement$
shows
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\ s)\ s$
 $\leq Missing + Complement$
proof –
have $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\ s)\ s \leq$
 $wp\text{-}event\ verify\text{-}monad$
 $(\lambda out.$
 $\quad trace\text{-}fri\text{-}header\text{-}tied\text{-}recorded\text{-}sibling\text{-}candidate\text{-}missing\ s\ out \vee$
 $\quad (trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\ s\ out \wedge$
 $\quad \neg trace\text{-}fri\text{-}header\text{-}tied\text{-}recorded\text{-}sibling\text{-}candidate\text{-}missing\ s\ out))\ s$
by $(rule\ wp\text{-}event\text{-}mono)\ blast$
also have $\dots \leq$
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}header\text{-}tied\text{-}recorded\text{-}sibling\text{-}candidate\text{-}missing\ s)\ s +$
 $wp\text{-}event\ verify\text{-}monad$
 $(\lambda out.$
 $\quad trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\ s\ out \wedge$
 $\quad \neg trace\text{-}fri\text{-}header\text{-}tied\text{-}recorded\text{-}sibling\text{-}candidate\text{-}missing\ s\ out)\ s$
by $(rule\ wp\text{-}event\text{-}union\text{-}bound)$
also have $\dots \leq Missing + Complement$
by $(intro\ add\text{-}mono\ missing\text{-}bound\ complement\text{-}bound)$
finally show $?thesis$.
qed

lemma $checked\text{-}staged\text{-}security\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}augmented\text{-}sampled\text{-}layer\text{-}chain\text{-}bound\text{-}from\text{-}fresh\text{-}and\text{-}prequery$:
assumes wf : $staged\text{-}budget\text{-}wellformed\ budgets$
and $controlled$: $staged\text{-}adversary\text{-}controlled\ budgets\ A$

and *finite-B*: *finite B*
and *cover*:
 $\wedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-sampled-layer-chain-cover
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad
and *envelope*:
 $\wedge$ *data attacker-state trace-table*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-multiround-bad-sets bad trace-table $\subseteq B$
and *fresh-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
(λs . trace-fri-challenge-list-fresh-hit s B))
adversary-initial-state $\leq$ Fresh
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-augmented-sampled-layer-chain)
adversary-initial-state $\leq$
Fresh +
*hash-relation-budget-value (card B * ceil-log clength)*
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)
proof –
have *sampled*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-sampled-layer-chain)
adversary-initial-state $\leq$
Fresh +
*hash-relation-budget-value (card B * ceil-log clength)*
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)
by (rule
checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-fresh-and-prequery
[OF wf controlled finite-B cover envelope fresh-bound])
have *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-augmented-sampled-layer-chain)
adversary-initial-state $\leq$
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-sampled-layer-chain)

```

    adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def
  by (rule wp-event-mono)
    (auto dest:
      trace-fri-bad-with-header-tied-augmented-sampled-layer-chain-imp-plain
      split: option.splits prod.splits)
  then show ?thesis
    by (rule order-trans[OF - sampled])
qed

lemma checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-augmented-and-unaugmented:
  assumes augmented-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-header-tied-augmented-partial-candidate)
    adversary-initial-state  $\leq$  Augmented
  and unaugmented-bound:
     $\wedge$ data attacker-state.
      Some (data, attacker-state)  $\in$ 
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state)  $\implies$ 
        wp-event verify-monad
          (trace-fri-header-tied-unaugmented-partial-candidate
            (verifier-state-from-adversary attacker-state
              (staged-proof-transcript data)))
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))  $\leq$  Unaugmented
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-header-tied-partial-candidate)
    adversary-initial-state  $\leq$  Augmented + Unaugmented
proof -
  have unaugmented-staged:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-header-tied-unaugmented-partial-candidate)
    adversary-initial-state  $\leq$  Unaugmented
  by (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    (auto simp: trace-fri-header-tied-unaugmented-partial-candidate-def
      unaugmented-bound)
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-partial-candidate)
    adversary-initial-state  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
      ( $\lambda$ out.
        staged-security-with-data-state-verifier-event
          trace-fri-bad-with-header-tied-augmented-partial-candidate out  $\vee$ 

```

```

    staged-security-with-data-state-verifier-event
    trace-fri-header-tied-unaugmented-partial-candidate out)
  adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
by (rule wp-event-mono-on-support)
  (auto dest:
    trace-fri-bad-with-header-tied-partial-candidate-imp-augmented-or-unaugmented
    split: option.splits prod.splits)
also have ...  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-augmented-partial-candidate)
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-header-tied-unaugmented-partial-candidate)
  adversary-initial-state
by (rule wp-event-union-bound)
also have ...  $\leq$  Augmented + Unaugmented
by (rule add-mono[OF augmented-bound unaugmented-staged])
finally show ?thesis .
qed

```

lemma checked-staged-security-trace-fri-header-tied-unaugmented-partial-candidate-bound-from-recorded-missing-and

assumes merkle-bound:

$\wedge$ data attacker-state.

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$

wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Merkle

and missing-bound:

$\wedge$ data attacker-state.

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$

wp-event verify-monad
 (trace-fri-header-tied-recorded-sibling-candidate-missing
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Missing

shows

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event

```

    trace-fri-header-tied-unaugmented-partial-candidate)
  adversary-initial-state ≤ Missing + Merkle
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  fix data attacker-state
  assume support:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have head-neq:
    wp-event verify-monad
      (trace-fri-header-tied-base-head-value-neq-gap ?s) ?s ≤ Merkle
  by (rule
    wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
    (rule merkle-bound[OF support])
  have head-mismatch:
    wp-event verify-monad
      (trace-fri-header-tied-base-head-value-mismatch-gap ?s) ?s ≤ Merkle
  by (rule
    wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
      [OF head-neq])
  have alignment:
    wp-event verify-monad
      (trace-fri-header-tied-base-value-alignment-gap ?s) ?s ≤ Merkle
  by (rule
    wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
      [OF head-mismatch])
  show
    wp-event verify-monad
      (trace-fri-header-tied-unaugmented-partial-candidate ?s) ?s
      ≤ Missing + Merkle
  by (rule
    wp-trace-fri-header-tied-unaugmented-partial-candidate-bound-from-recorded-missing-and-base-gap)
    (rule missing-bound[OF support], rule alignment)
next
  fix s
  show ¬ trace-fri-header-tied-unaugmented-partial-candidate s None
  unfolding trace-fri-header-tied-unaugmented-partial-candidate-def
    trace-fri-bad-with-header-tied-partial-candidate-def
    accepted-fri-opening-transcript-def
  by simp
qed

lemma trace-fri-header-tied-reduction-from-augmented-and-recorded-missing-bounds:
  fixes Augmented Missing Merkle :: prob
  assumes augmented-bound:
    wp-event verify-monad

```


$(\text{trace-fri-bad-with-header-tied-augmented-partial-candidate } s) \ s$
 $\leq \text{Augmented}$
and *missing-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-recorded-sibling-candidate-missing } s) \ s$
 $\leq \text{Missing}$
and *merkle-bound*:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s$
 $\leq \text{Merkle}$
and *total-bound*:
 $\text{Augmented} + (\text{Missing} + \text{Merkle}) \leq \text{trace-fri-error}$
shows *trace-fri-header-tied-reduction* s
proof –
have *head-neq*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-head-value-neq-gap } s) \ s \leq \text{Merkle}$
by (rule
 $\text{wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle}$
 $(\text{rule merkle-bound})$)
have *head-mismatch*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-head-value-mismatch-gap } s) \ s$
 $\leq \text{Merkle}$
by (rule
 $\text{wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap}$
 $[\text{OF head-neq}]$)
have *alignment*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-value-alignment-gap } s) \ s \leq \text{Merkle}$
by (rule
 $\text{wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap}$
 $[\text{OF head-mismatch}]$)
have *unaugmented*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-unaugmented-partial-candidate } s) \ s$
 $\leq \text{Missing} + \text{Merkle}$
by (rule
 $\text{wp-trace-fri-header-tied-unaugmented-partial-candidate-bound-from-recorded-missing-and-base-gap}$
 $(\text{rule missing-bound}, \text{rule alignment})$)
have *partial*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s$
 $\leq \text{Augmented} + (\text{Missing} + \text{Merkle})$
by (rule
 $\text{wp-trace-fri-header-tied-partial-candidate-bound-from-augmented-and-unaugmented}$
 $(\text{rule augmented-bound}, \text{rule unaugmented})$)
show *?thesis*
unfolding *trace-fri-header-tied-reduction-def*
by (rule *order-trans*[*OF partial total-bound*])

qed

lemma *active-trace-fri-header-tied-reductions-from-augmented-and-recorded-missing-bounds:*

fixes *Augmented Missing Merkle ::*

'f staged-proof-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ prob

assumes *augmented-bound:*

$\wedge$ *data attacker-state.*

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-augmented-partial-candidate
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Augmented data attacker-state*

and *missing-bound:*

$\wedge$ *data attacker-state.*

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (trace-fri-header-tied-recorded-sibling-candidate-missing
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Missing data attacker-state*

and *merkle-bound:*

$\wedge$ *data attacker-state.*

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Merkle data attacker-state*

and *total-bound:*

$\wedge$ *data attacker-state.*

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 Augmented data attacker-state +
 (Missing data attacker-state + Merkle data attacker-state)
 $\leq$ trace-fri-error*

and *support:*

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state)
shows
trace-fri-header-tied-reduction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
by (*rule*
trace-fri-header-tied-reduction-from-augmented-and-recorded-missing-bounds)
(rule augmented-bound[OF support], rule missing-bound[OF support],
rule merkle-bound[OF support], rule total-bound[OF support])

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-without-same-residu*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: finite B*
and *cover:*
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-sampled-layer-chain-cover
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad
and *envelope:*
 $\bigwedge \text{data attacker-state trace-table.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-multiround-bad-sets bad trace-table $\subseteq B$
and *fresh-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
($\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B$))
adversary-initial-state $\leq$ Fresh
and *merkle-bound:*
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$ Merkle
and *structural-bound:*
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$

```

wp-event verify-monad
  (trace-fri-header-tied-sibling-mismatch-structural-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ Structural
and zero-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-header-tied-zero-round-final-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ Zero
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state ≤
  (Fresh +
    hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
  (((((Merkle + Structural) + Merkle) + (Merkle + 0) + (Merkle + 0) +
    (Merkle + 0 + 0)) + Merkle) + 0 + Zero)
proof (rule
  checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-and-residual-gaps
  [OF wf controlled finite-B cover envelope fresh-bound])
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show wp-event verify-monad
  (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap ?s) ?s
  ≤ 0
by (rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
  ≤ Merkle
by (rule merkle-bound[OF support])
show wp-event verify-monad
  (trace-fri-header-tied-sibling-mismatch-structural-gap ?s) ?s
  ≤ Structural

```

by (rule structural-bound[OF support])
 have head-neq:
 wp-event verify-monad
 (trace-fri-header-tied-base-head-value-neq-gap ?s) ?s ≤ Merkle
 by (rule
 wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
 (rule merkle-bound[OF support])
 have head-mismatch:
 wp-event verify-monad
 (trace-fri-header-tied-base-head-value-mismatch-gap ?s) ?s
 ≤ Merkle
 by (rule
 wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
 [OF head-neq])
 show wp-event verify-monad
 (trace-fri-header-tied-base-value-alignment-gap ?s) ?s ≤ Merkle
 by (rule
 wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
 [OF head-mismatch])
 show wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap ?s) ?s ≤ Merkle + 0
 by (rule
 wp-trace-fri-header-tied-next-value-replay-gap-bound-from-merkle-and-slot)
 (rule merkle-bound[OF support],
 rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
 show wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap ?s) ?s ≤ Merkle + 0
 by (rule
 wp-trace-fri-header-tied-successor-replay-gap-bound-from-merkle-and-slot)
 (rule merkle-bound[OF support],
 rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
 show wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap ?s) ?s
 ≤ Merkle + 0 + 0
 by (rule
 wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
 (rule merkle-bound[OF support],
 rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero,
 rule wp-trace-fri-header-tied-final-value-selected-step-missing-gap-zero)
 show wp-event verify-monad
 (trace-fri-header-tied-zero-round-final-obstruction ?s) ?s ≤ Zero
 by (rule zero-bound[OF support])
 qed

lemma checked-staged-security-trace-fri-header-tied-zero-round-bound-from-staged-checked-and-merkle:

assumes checked-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-zero-round-checked-final-obstruction)

```

    adversary-initial-state ≤ CheckedZero
and merkle-bound:
 $\wedge$ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (partial-merkle-inconsistency-bad
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ Merkle
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-header-tied-zero-round-final-obstruction)
  adversary-initial-state ≤ CheckedZero + Merkle
proof –
have merkle-staged:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad)
  adversary-initial-state ≤ Merkle
by (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  (auto simp: partial-merkle-inconsistency-bad-def merkle-bound)
have split:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-header-tied-zero-round-final-obstruction)
  adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (λout.
    staged-security-with-data-state-verifier-event
      trace-fri-zero-round-checked-final-obstruction out  $\vee$ 
    staged-security-with-data-state-verifier-event
      partial-merkle-inconsistency-bad out)
  adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
by (rule wp-event-mono-on-support)
  (auto dest:
    trace-fri-header-tied-zero-round-final-obstruction-imp-checked-or-merkle
    split: option.splits prod.splits)
also have ... ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-zero-round-checked-final-obstruction)
  adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event

```

partial-merkle-inconsistency-bad)
adversary-initial-state
by (rule *wp-event-union-bound*)
also have ... $\leq$ *CheckedZero* + *Merkle*
by (intro *add-mono checked-bound merkle-staged*)
finally show ?thesis .
qed

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-staged-checked-zero-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: finite B*
and *cover:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-sampled-layer-chain-cover
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad
and *envelope:*
 $\wedge$ *data attacker-state trace-table.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-multiround-bad-sets bad trace-table $\subseteq B$
and *fresh-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
($\lambda s.$ trace-fri-challenge-list-fresh-hit s B))
adversary-initial-state $\leq$ Fresh
and *merkle-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$ Merkle
and *structural-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-sibling-mismatch-structural-gap

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Structural}$
and *checked-zero-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-zero-round-checked-final-obstruction})$
 $\text{adversary-initial-state} \leq \text{CheckedZero}$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq$
 $(\text{Fresh} +$
 $\text{hash-relation-budget-value } (\text{card } B * \text{ceil-log } \text{clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(((((\text{Merkle} + \text{Structural}) + \text{Merkle}) + (\text{Merkle} + 0) + (\text{Merkle} + 0) +$
 $(\text{Merkle} + 0 + 0)) + \text{Merkle}) + 0 + (\text{CheckedZero} + \text{Merkle}))$
proof –
have *sampled*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-sampled-layer-chain})$
 $\text{adversary-initial-state} \leq$
 $\text{Fresh} +$
 $\text{hash-relation-budget-value } (\text{card } B * \text{ceil-log } \text{clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
by (rule
 $\text{checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-fresh-and-prequery}$
 $[\text{OF wf controlled finite-B cover envelope fresh-bound}])$
have *conflict*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-header-tied-sampled-assignment-conflict})$
 $\text{adversary-initial-state} \leq$
 $(((((\text{Merkle} + \text{Structural}) + \text{Merkle}) + (\text{Merkle} + 0) +$
 $(\text{Merkle} + 0) + (\text{Merkle} + 0 + 0)) + \text{Merkle}) + 0)$
proof (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
fix *data attacker-state*
assume *support*:
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$
let *?s* =
 $\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})$

have *head-neq*:
 wp-event verify-monad
 (*trace-fri-header-tied-base-head-value-neq-gap* ?s) ?s $\leq$ Merkle
by (rule
 wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
 (rule merkle-bound[OF support])
have *head-mismatch*:
 wp-event verify-monad
 (*trace-fri-header-tied-base-head-value-mismatch-gap* ?s) ?s
 $\leq$ Merkle
by (rule
 wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
 [OF *head-neq*])
have *alignment*:
 wp-event verify-monad
 (*trace-fri-header-tied-base-value-alignment-gap* ?s) ?s $\leq$ Merkle
by (rule
 wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
 [OF *head-mismatch*])
have *next-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-next-value-replay-gap* ?s) ?s $\leq$ Merkle + 0
by (rule
 wp-trace-fri-header-tied-next-value-replay-gap-bound-from-merkle-and-slot)
 (rule merkle-bound[OF support],
 rule *wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero*)
have *successor*:
 wp-event verify-monad
 (*trace-fri-header-tied-successor-replay-gap* ?s) ?s $\leq$ Merkle + 0
by (rule
 wp-trace-fri-header-tied-successor-replay-gap-bound-from-merkle-and-slot)
 (rule merkle-bound[OF support],
 rule *wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero*)
have *final*:
 wp-event verify-monad
 (*trace-fri-header-tied-final-value-replay-gap* ?s) ?s
 $\leq$ Merkle + 0 + 0
by (rule
 wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
 (rule merkle-bound[OF support],
 rule *wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero*,
 rule *wp-trace-fri-header-tied-final-value-selected-step-missing-gap-zero*)
show *wp-event verify-monad*
 (*trace-fri-header-tied-sampled-assignment-conflict* ?s) ?s
 $\leq$ (((((Merkle + Structural) + Merkle) + (Merkle + 0) +
 (Merkle + 0) + (Merkle + 0 + 0)) + Merkle) + 0)
by (rule
 wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-slot-recorded-chunk-gap-and-residuals)
 (rule *wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero*,

```

    rule merkle-bound[OF support],
    rule structural-bound[OF support],
    rule alignment, rule next-bound, rule successor, rule final)
next
fix s
show  $\neg$  trace-fri-header-tied-sampled-assignment-conflict s None
  unfolding trace-fri-header-tied-sampled-assignment-conflict-def
    accepted-fri-opening-transcript-def
  by simp
qed
have zero:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-zero-round-final-obstruction)
  adversary-initial-state  $\leq$  CheckedZero + Merkle
  by (rule
    checked-staged-security-trace-fri-header-tied-zero-round-bound-from-staged-checked-and-merkle)
    (rule checked-zero-bound, rule merkle-bound)
show ?thesis
  by (rule
    checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-conflict-and-staged-zero
      [OF sampled conflict zero])
qed

lemma checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-staged-checked-zero-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and cover:
 $\bigwedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    trace-fri-sampled-layer-chain-cover
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) bad
and envelope:
 $\bigwedge$ data attacker-state trace-table.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    trace-fri-multi-round-bad-sets bad trace-table  $\subseteq$  B
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s$ . trace-fri-challenge-list-fresh-hit s B))
  adversary-initial-state  $\leq$  Fresh
and merkle-bound:
 $\bigwedge$ data attacker-state.

```

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ Merkle
and *missing-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-recorded-sibling-candidate-missing
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ Missing
and *checked-zero-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-zero-round-checked-final-obstruction)
adversary-initial-state ≤ CheckedZero
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state ≤
(Fresh +
*hash-relation-budget-value (card B * ceil-log clength)*
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)) +
(((((Merkle + Missing) + Merkle) + (Merkle + 0) + (Merkle + 0) +
(Merkle + 0 + 0)) + Merkle) + 0 + (CheckedZero + Merkle))
proof (rule
checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-staged-checked-zero-and-w
[OF wf controlled finite-B cover envelope fresh-bound merkle-bound - checked-zero-bound])
fix *data attacker-state*
assume *support:*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
show *wp-event verify-monad*
(trace-fri-header-tied-sibling-mismatch-structural-gap ?s) ?s

$\leq \text{Missing}$
by (rule
 $\text{wp-trace-fri-header-tied-sibling-structural-gap-bound-from-recorded-sibling-missing}$
(rule missing-bound[OF support])
qed

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-staged-checked-zero-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: finite B*
and *cover:*
 $\wedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-sampled-layer-chain-cover}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad}$
and *envelope:*
 $\wedge \text{data attacker-state trace-table.}$
Some (data, attacker-state) $\in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *fresh-bound:*
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B))$
 $\text{adversary-initial-state} \leq \text{Fresh}$
and *merkle-bound:*
 $\wedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Merkle}$
and *missing-bound:*
 $\wedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-recorded-sibling-candidate-missing}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$

$(\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})) \leq \text{Missing}$
and *checked-zero-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ (\text{staged-security-with-data-state-verifier-event} \\ \text{trace-fri-zero-round-checked-final-obstruction}) \\ \text{adversary-initial-state} \leq \text{CheckedZero}$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ (\text{staged-security-with-data-state-verifier-event} \\ \text{trace-fri-bad-with-header-tied-partial-candidate}) \\ \text{adversary-initial-state} \leq \\ (\text{Fresh} + \\ \text{hash-relation-budget-value } (\text{card } B * \text{ceil-log } \text{clength}) \\ (\text{staged-attacker-query-budget budgets} + \\ \text{staged-challenge-query-budget})) + \\ ((\text{Missing} + \\ (((\text{Merkle} + \text{Merkle}) + (\text{Merkle} + 0) + (\text{Merkle} + 0) + \\ (\text{Merkle} + 0 + 0)) + \text{Merkle}) + 0)) + \\ (\text{CheckedZero} + \text{Merkle}))$
proof –
have *sampled*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ (\text{staged-security-with-data-state-verifier-event} \\ \text{trace-fri-bad-with-header-tied-sampled-layer-chain}) \\ \text{adversary-initial-state} \leq \\ \text{Fresh} + \\ \text{hash-relation-budget-value } (\text{card } B * \text{ceil-log } \text{clength}) \\ (\text{staged-attacker-query-budget budgets} + \\ \text{staged-challenge-query-budget})$
by (rule
checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-fresh-and-prequery
[OF wf controlled finite-B cover envelope fresh-bound])
have *conflict*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ (\text{staged-security-with-data-state-verifier-event} \\ \text{trace-fri-header-tied-sampled-assignment-conflict}) \\ \text{adversary-initial-state} \leq \\ \text{Missing} + \\ (((\text{Merkle} + \text{Merkle}) + (\text{Merkle} + 0) + (\text{Merkle} + 0) + \\ (\text{Merkle} + 0 + 0)) + \text{Merkle}) + 0$
proof (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
fix *data attacker-state*
assume *support*:
 $\text{Some } (\text{data}, \text{attacker-state}) \in \\ \text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \\ \text{adversary-initial-state})$
let *?s* =
 $\text{verifier-state-from-adversary attacker-state}$

```

    (staged-proof-transcript data)
have head-neq:
  wp-event verify-monad
    (trace-fri-header-tied-base-head-value-neq-gap ?s) ?s ≤ Merkle
  by (rule
    wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
    (rule merkle-bound[OF support])
have head-mismatch:
  wp-event verify-monad
    (trace-fri-header-tied-base-head-value-mismatch-gap ?s) ?s
    ≤ Merkle
  by (rule
    wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
    [OF head-neq])
have alignment:
  wp-event verify-monad
    (trace-fri-header-tied-base-value-alignment-gap ?s) ?s ≤ Merkle
  by (rule
    wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
    [OF head-mismatch])
have next-bound:
  wp-event verify-monad
    (trace-fri-header-tied-next-value-replay-gap ?s) ?s ≤ Merkle + 0
  by (rule
    wp-trace-fri-header-tied-next-value-replay-gap-bound-from-merkle-and-slot)
    (rule merkle-bound[OF support],
    rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have successor:
  wp-event verify-monad
    (trace-fri-header-tied-successor-replay-gap ?s) ?s ≤ Merkle + 0
  by (rule
    wp-trace-fri-header-tied-successor-replay-gap-bound-from-merkle-and-slot)
    (rule merkle-bound[OF support],
    rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have final:
  wp-event verify-monad
    (trace-fri-header-tied-final-value-replay-gap ?s) ?s
    ≤ Merkle + 0 + 0
  by (rule
    wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
    (rule merkle-bound[OF support],
    rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero,
    rule wp-trace-fri-header-tied-final-value-selected-step-missing-gap-zero)
have slot:
  wp-event verify-monad
    (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap ?s) ?s
    ≤ 0
  by (rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have authenticated-recorded-auth:

```

```

wp-event verify-monad
  (trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap ?s)
  ?s ≤ 0
by (rule
  wp-trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap
  [OF slot])
have recorded-auth:
  wp-event verify-monad
    (trace-fri-header-tied-same-layer-recorded-auth-gap ?s) ?s ≤ 0
  by (rule
    wp-trace-fri-header-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap
    [OF authenticated-recorded-auth])
have same-auth:
  wp-event verify-monad
    (trace-fri-header-tied-same-layer-auth-gap ?s) ?s ≤ 0
  by (rule
    wp-trace-fri-header-tied-same-layer-auth-gap-bound-from-recorded-auth-gap
    [OF recorded-auth])
have auth-gap:
  wp-event verify-monad
    (trace-fri-header-tied-assignment-auth-gap ?s) ?s ≤ 0
  by (rule
    wp-trace-fri-header-tied-assignment-auth-gap-bound-from-same-layer-auth-gap
    [OF same-auth])
have complement:
  wp-event verify-monad
    (λout.
      trace-fri-header-tied-sampled-assignment-conflict ?s out ∧
      ¬ trace-fri-header-tied-recorded-sibling-candidate-missing ?s out)
    ?s ≤
    (((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
      (Merkle + 0 + 0)) + Merkle) + 0)
  by (rule
    wp-trace-fri-header-tied-sampled-assignment-conflict-without-recorded-missing-bound-from-auth-gap-and-replay-
    (rule auth-gap, rule merkle-bound[OF support], rule alignment,
      rule next-bound, rule successor, rule final))
show wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict ?s) ?s
  ≤ Missing +
    (((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
      (Merkle + 0 + 0)) + Merkle) + 0)
  by (rule
    wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-recorded-missing-and-complement)
    (rule missing-bound[OF support], rule complement)
next
fix s
show ¬ trace-fri-header-tied-sampled-assignment-conflict s None
  unfolding trace-fri-header-tied-sampled-assignment-conflict-def
    accepted-fri-opening-transcript-def

```

```

    by simp
qed
have zero:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-zero-round-final-obstruction)
  adversary-initial-state ≤ CheckedZero + Merkle
by (rule
  checked-staged-security-trace-fri-header-tied-zero-round-bound-from-staged-checked-and-merkle)
(rule checked-zero-bound, rule merkle-bound)
show ?thesis
by (rule
  checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-conflict-and-staged-zero
    [OF sampled conflict zero])
qed

lemma checked-staged-security-trace-fri-header-tied-augmented-partial-candidate-bound-from-fresh-prequery-staged-ch
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and cover:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  trace-fri-sampled-layer-chain-cover
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) bad
and envelope:
  ∧ data attacker-state trace-table.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  trace-fri-multi-round-bad-sets bad trace-table ⊆ B
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-challenge-list-fresh-hit s B))
  adversary-initial-state ≤ Fresh
and merkle-bound:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  wp-event verify-monad
    (partial-merkle-inconsistency-bad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state

```


$(\text{staged-proof-transcript data}) \leq \text{Merkle}$
and *missing-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-recorded-sibling-candidate-missing}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Missing}$
and *checked-zero-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-zero-round-checked-final-obstruction})$
 $\text{adversary-initial-state} \leq \text{CheckedZero}$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-augmented-partial-candidate})$
 $\text{adversary-initial-state} \leq$
 $(\text{Fresh} +$
 $\text{hash-relation-budget-value (card } B * \text{ceil-log clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $((\text{Missing} +$
 $((((\text{Merkle} + \text{Merkle}) + (\text{Merkle} + 0) + (\text{Merkle} + 0) +$
 $(\text{Merkle} + 0 + 0)) + \text{Merkle}) + 0)) +$
 $(\text{CheckedZero} + \text{Merkle}))$
proof –
have *sampled*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-augmented-sampled-layer-chain})$
 $\text{adversary-initial-state} \leq$
 $\text{Fresh} +$
 $\text{hash-relation-budget-value (card } B * \text{ceil-log clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
by (rule
 $\text{checked-staged-security-trace-fri-header-tied-augmented-sampled-layer-chain-bound-from-fresh-and-prequery}$
 $[\text{OF wf controlled finite-}B \text{ cover envelope fresh-bound}])$
have *conflict*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-header-tied-sampled-assignment-conflict})$
 $\text{adversary-initial-state} \leq$
 $\text{Missing} +$

```

      (((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
        (Merkle + 0 + 0)) + Merkle) + 0)
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have head-neq:
  wp-event verify-monad
    (trace-fri-header-tied-base-head-value-neq-gap ?s) ?s ≤ Merkle
by (rule
  wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
  (rule merkle-bound[OF support])
have head-mismatch:
  wp-event verify-monad
    (trace-fri-header-tied-base-head-value-mismatch-gap ?s) ?s
    ≤ Merkle
by (rule
  wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
  [OF head-neq])
have alignment:
  wp-event verify-monad
    (trace-fri-header-tied-base-value-alignment-gap ?s) ?s ≤ Merkle
by (rule
  wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
  [OF head-mismatch])
have next-bound:
  wp-event verify-monad
    (trace-fri-header-tied-next-value-replay-gap ?s) ?s ≤ Merkle + 0
by (rule
  wp-trace-fri-header-tied-next-value-replay-gap-bound-from-merkle-and-slot)
  (rule merkle-bound[OF support],
  rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have successor:
  wp-event verify-monad
    (trace-fri-header-tied-successor-replay-gap ?s) ?s ≤ Merkle + 0
by (rule
  wp-trace-fri-header-tied-successor-replay-gap-bound-from-merkle-and-slot)
  (rule merkle-bound[OF support],
  rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have final:
  wp-event verify-monad
    (trace-fri-header-tied-final-value-replay-gap ?s) ?s
    ≤ Merkle + 0 + 0
by (rule

```

```

    wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
  (rule merkle-bound[OF support],
    rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero,
    rule wp-trace-fri-header-tied-final-value-selected-step-missing-gap-zero)
have slot:
  wp-event verify-monad
    (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap ?s) ?s
    ≤ 0
  by (rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have authenticated-recorded-auth:
  wp-event verify-monad
    (trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap ?s)
    ?s ≤ 0
  by (rule
    wp-trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap
    [OF slot])
have recorded-auth:
  wp-event verify-monad
    (trace-fri-header-tied-same-layer-recorded-auth-gap ?s) ?s ≤ 0
  by (rule
    wp-trace-fri-header-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap
    [OF authenticated-recorded-auth])
have same-auth:
  wp-event verify-monad
    (trace-fri-header-tied-same-layer-auth-gap ?s) ?s ≤ 0
  by (rule
    wp-trace-fri-header-tied-same-layer-auth-gap-bound-from-recorded-auth-gap
    [OF recorded-auth])
have auth-gap:
  wp-event verify-monad
    (trace-fri-header-tied-assignment-auth-gap ?s) ?s ≤ 0
  by (rule
    wp-trace-fri-header-tied-assignment-auth-gap-bound-from-same-layer-auth-gap
    [OF same-auth])
have complement:
  wp-event verify-monad
    (λout.
      trace-fri-header-tied-sampled-assignment-conflict ?s out ∧
      ¬ trace-fri-header-tied-recorded-sibling-candidate-missing ?s out)
    ?s ≤
    (((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
      (Merkle + 0 + 0)) + Merkle) + 0)
  by (rule
    wp-trace-fri-header-tied-sampled-assignment-conflict-without-recorded-missing-bound-from-auth-gap-and-replay-
    (rule auth-gap, rule merkle-bound[OF support], rule alignment,
      rule next-bound, rule successor, rule final))
show wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict ?s) ?s
  ≤ Missing +

```

```

      (((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
        (Merkle + 0 + 0)) + Merkle) + 0)
  by (rule
    wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-recorded-missing-and-complement)
    (rule missing-bound[OF support], rule complement)
next
  fix s
  show  $\neg$  trace-fri-header-tied-sampled-assignment-conflict s None
    unfolding trace-fri-header-tied-sampled-assignment-conflict-def
      accepted-fri-opening-transcript-def
    by simp
qed
have zero:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-zero-round-final-obstruction)
  adversary-initial-state  $\leq$  CheckedZero + Merkle
  by (rule
    checked-staged-security-trace-fri-header-tied-zero-round-bound-from-staged-checked-and-merkle)
    (rule checked-zero-bound, rule merkle-bound)
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-augmented-partial-candidate)
  adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    ( $\lambda$ out.
      staged-security-with-data-state-verifier-event
        trace-fri-bad-with-header-tied-augmented-sampled-layer-chain out  $\vee$ 
      staged-security-with-data-state-verifier-event
        trace-fri-header-tied-sampled-assignment-conflict out  $\vee$ 
      staged-security-with-data-state-verifier-event
        trace-fri-header-tied-zero-round-final-obstruction out)
  adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def
  by (rule wp-event-mono-on-support)
    (auto dest:
      trace-fri-header-tied-augmented-partial-candidate-imp-sampled-conflict-or-zero
      split: option.splits prod.splits)
also have ...  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-augmented-sampled-layer-chain)
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    ( $\lambda$ out.
      staged-security-with-data-state-verifier-event
        trace-fri-header-tied-sampled-assignment-conflict out  $\vee$ 
      staged-security-with-data-state-verifier-event
        trace-fri-header-tied-zero-round-final-obstruction out)

```

adversary-initial-state
by (rule *wp-event-union-bound*)
also have ... $\leq$
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-augmented-sampled-layer-chain)
adversary-initial-state +
(*wp-event* (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-header-tied-sampled-assignment-conflict)
adversary-initial-state +
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-header-tied-zero-round-final-obstruction)
adversary-initial-state)
by (intro *add-mono order-refl wp-event-union-bound*)
also have ... $\leq$
(*Fresh* +
hash-relation-budget-value (*card* *B* * *ceil-log* *clength*)
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget)) +
(*Missing* +
((((*Merkle* + *Merkle*) + (*Merkle* + 0) + (*Merkle* + 0) +
(*Merkle* + 0 + 0)) + *Merkle*) + 0)) +
(*CheckedZero* + *Merkle*))
by (intro *add-mono sampled conflict zero*)
finally show ?thesis .
qed

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-augmented-route-and-unaugmented*

assumes *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets* *A*

and *finite-B*: *finite* *B*

and *cover*:

$\wedge$ *data* *attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
trace-fri-sampled-layer-chain-cover
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)) *bad*

and *envelope*:

$\wedge$ *data* *attacker-state* *trace-table*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
trace-fri-multi-round-bad-sets *bad* *trace-table* $\subseteq$ *B*

and *fresh-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)

(staged-security-with-data-state-verifier-event
 (λs. trace-fri-challenge-list-fresh-hit s B))
 adversary-initial-state ≤ Fresh
and merkle-bound:
 ∧data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤ Merkle
and missing-bound:
 ∧data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 wp-event verify-monad
 (trace-fri-header-tied-recorded-sibling-candidate-missing
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤ Missing
and checked-zero-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-zero-round-checked-final-obstruction)
 adversary-initial-state ≤ CheckedZero
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state ≤
 ((Fresh +
 hash-relation-budget-value (card B * ceil-log clength)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) +
 ((Missing +
 (((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
 (Merkle + 0 + 0)) + Merkle) + 0)) +
 (CheckedZero + Merkle))) +
 (Missing + Merkle)
proof –
have augmented:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-augmented-partial-candidate)

adversary-initial-state $\leq$
 (Fresh +
 hash-relation-budget-value (card B * ceil-log clength)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) +
 ((Missing +
 (((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
 (Merkle + 0 + 0)) + Merkle) + 0)) +
 (CheckedZero + Merkle))
by (rule
 checked-staged-security-trace-fri-header-tied-augmented-partial-candidate-bound-from-fresh-prequery-staged-check
 [OF wf controlled finite-B cover envelope fresh-bound merkle-bound
 missing-bound checked-zero-bound])
have unaugmented:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-header-tied-unaugmented-partial-candidate)
 adversary-initial-state $\leq$ Missing + Merkle
by (rule
 checked-staged-security-trace-fri-header-tied-unaugmented-partial-candidate-bound-from-recorded-missing-and-me
 (erule merkle-bound, erule missing-bound))
have wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state $\leq$
 wp-event (checked-staged-security-experiment-with-data-state A)
 (λ out.
 staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-augmented-partial-candidate out $\vee$
 staged-security-with-data-state-verifier-event
 trace-fri-header-tied-unaugmented-partial-candidate out)
 adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
by (rule wp-event-mono-on-support)
 (auto dest:
 trace-fri-bad-with-header-tied-partial-candidate-imp-augmented-or-unaugmented
 split: option.splits prod.splits)
also have ... $\leq$
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-augmented-partial-candidate)
 adversary-initial-state +
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-header-tied-unaugmented-partial-candidate)
 adversary-initial-state
by (rule wp-event-union-bound)
also have ... $\leq$
 (Fresh +

```

    hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
    ((Missing +
      (((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
        (Merkle + 0 + 0)) + Merkle) + 0)) +
      (CheckedZero + Merkle))) +
    (Missing + Merkle)
  by (rule add-mono[OF augmented unaugmented])
  finally show ?thesis .
qed

```

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-checked-zero-and-wi*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and cover:
   $\bigwedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
      trace-fri-sampled-layer-chain-cover
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)) bad
and envelope:
   $\bigwedge$ data attacker-state trace-table.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
      trace-fri-multiround-bad-sets bad trace-table  $\subseteq B$ 
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s$ . trace-fri-challenge-list-fresh-hit s B))
    adversary-initial-state  $\leq$  Fresh
and merkle-bound:
   $\bigwedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
      wp-event verify-monad
        (partial-merkle-inconsistency-bad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))  $\leq$  Merkle
and structural-bound:
   $\bigwedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 

```



```

    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-header-tied-sibling-mismatch-structural-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  Structural
and checked-zero-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-zero-round-checked-final-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  CheckedZero
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state  $\leq$ 
  (Fresh +
    hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
  (((((Merkle + Structural) + Merkle) + (Merkle + 0) + (Merkle + 0) +
    (Merkle + 0 + 0)) + Merkle) + 0 + (CheckedZero + Merkle))
proof (rule
  checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-without-same-residuals
  [OF wf controlled finite-B cover envelope fresh-bound merkle-bound
    structural-bound])
fix data attacker-state
assume support:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (trace-fri-header-tied-zero-round-final-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  CheckedZero + Merkle
by (rule wp-trace-fri-header-tied-zero-round-bound-from-checked-and-merkle)
  (rule checked-zero-bound[OF support],
    rule merkle-bound[OF support])
qed

```

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-without-same*

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *finite-B*: $\bigwedge dg. \text{finite } (B \ dg)$

and *cover*:

$\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\implies$
 composition-fri-sampled-layer-chain-cover
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) bad

and *envelope*:

$\bigwedge \text{data attacker-state dg composition-table.}$

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\implies$
 composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \ dg$

and *fresh-bound*:

wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-verifier-event
 ($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$))
 adversary-initial-state $\leq$ Fresh

and *merkle-bound*:

$\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\implies$
wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Merkle

and *missing-bound*:

$\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\implies$
wp-event verify-monad
 (composition-fri-verifier-tied-recorded-base-chunk-auth-missing
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Missing

and *without-same-bound*:

$\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ WithoutSame
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state ≤
(Fresh +
hash-relation-budget-value
($\sum dg \in (UNIV :: 'f \text{ set}).$
*card (B dg) * ceil-log (maxDegree + 1))*
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)) +
((Merkle + Missing) + (Merkle + 0) + (Merkle + 0) + WithoutSame +
Merkle) + 0 + 0)
proof (rule
checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-and-residual-gaps
[OF wf controlled finite-B cover envelope fresh-bound])
fix data attacker-state
assume support:
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
let ?s =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
show wp-event verify-monad
(composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap ?s)
?s ≤ 0
by (rule
wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
≤ Merkle
by (rule merkle-bound[OF support])
show wp-event verify-monad
(composition-fri-verifier-tied-recorded-base-chunk-auth-missing ?s) ?s
≤ Missing
by (rule missing-bound[OF support])
show wp-event verify-monad
(composition-fri-verifier-tied-next-value-replay-gap ?s) ?s
≤ Merkle + 0
by (rule

```

      wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-merkle-and-slot)
(rule merkle-bound[OF support],
rule
  wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
show wp-event verify-monad
  (composition-fri-verifier-tied-successor-replay-gap ?s) ?s
  ≤ Merkle + 0
by (rule
  wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-merkle-and-slot)
(rule merkle-bound[OF support],
rule
  wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
show wp-event verify-monad
  (composition-fri-verifier-tied-final-value-replay-gap ?s) ?s
  ≤ WithoutSame
by (rule
  wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-without-same)
(rule without-same-bound[OF support])
show wp-event verify-monad
  (composition-fri-verifier-tied-zero-round-final-obstruction ?s) ?s
  ≤ 0
by (subst wp-composition-fri-verifier-tied-zero-round-final-obstruction-zero)
simp
qed

```

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-without-miss*

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *finite-B*: $\bigwedge dg. \text{finite } (B \ dg)$

and *cover*:

$\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\implies$
 composition-fri-sampled-layer-chain-cover
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) bad

and *envelope*:

$\bigwedge \text{data attacker-state dg composition-table.}$

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\implies$
 composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \ dg$

and *fresh-bound*:

wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-verifier-event
 ($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$))
 adversary-initial-state ≤ Fresh

and merkle-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Merkle
and without-same-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ WithoutSame
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state $\leq$
 (Fresh +
 hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \text{ set}).$
 card (B dg) * ceil-log (maxDegree + 1))
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) +
 (((Merkle + 0) + (Merkle + 0) + (Merkle + 0) + WithoutSame +
 Merkle) + 0 + 0)
proof (rule
 checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-without-same-resis
 [where Missing = 0])
show staged-budget-wellformed budgets
by (rule wf)
show staged-adversary-controlled budgets A
by (rule controlled)
show $\wedge dg. \text{finite } (B \ dg)$
by (rule finite-B)
show $\wedge$ data attacker-state.
 Some (data, attacker-state)
 $\in$ set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$

composition-fri-sampled-layer-chain-cover
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad
by (rule cover)
show $\bigwedge \text{data attacker-state dg composition-table.}$
Some (data, attacker-state)
 $\in \text{set-dist}$
(execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \text{ dg}$
by (rule envelope)
show *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \text{ B})$)
adversary-initial-state $\leq$ Fresh
by (rule fresh-bound)
show $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state)
 $\in \text{set-dist}$
(execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$ Merkle
by (rule merkle-bound)
show $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state)
 $\in \text{set-dist}$
(execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(composition-fri-verifier-tied-recorded-base-chunk-auth-missing
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ 0
by (subst wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero)
simp
show $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state)
 $\in \text{set-dist}$
(execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{WithoutSame}$
 by (rule without-same-bound)
 qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-zero-slot-mis*

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *finite-B*: $\bigwedge dg. \text{finite } (B \ dg)$
and *cover*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{composition-fri-sampled-layer-chain-cover}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad}$
and *envelope*:
 $\bigwedge \text{data attacker-state dg composition-table.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{composition-fri-multiround-bad-sets bad dg composition-table}$
 $\subseteq B \ dg$
and *fresh-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B))$
 $\text{adversary-initial-state} \leq \text{Fresh}$
and *merkle-bound*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Merkle}$
and *next-bound*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap}$

```

    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ Next
and successor-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (composition-fri-verifier-tied-successor-replay-gap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)) ≤ Successor
and final-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-replay-gap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)) ≤ Final
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-verifier-tied-partial-candidate)
    adversary-initial-state ≤
    (Fresh +
      hash-relation-budget-value
        ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
          card (B dg) * ceil-log (maxDegree + 1))
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget)) +
    (((Merkle + 0) + Next + Successor + Final + Merkle) + 0 + 0)
proof (rule
  checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-and-residual-gaps
    [where Slot = 0 and Missing = 0 and Zero = 0])
show staged-budget-wellformed budgets
  by (rule wf)
show staged-adversary-controlled budgets A
  by (rule controlled)
show  $\wedge dg. \text{finite } (B dg)$ 
  by (rule finite-B)
show  $\wedge$ data attacker-state.
  Some (data, attacker-state)

```


$\in \text{set-dist}$
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\quad \text{adversary-initial-state}) \implies$
 $\text{composition-fri-sampled-layer-chain-cover}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $\quad (\text{staged-proof-transcript data})) \text{ bad}$
by (rule cover)
show $\bigwedge \text{data attacker-state dg composition-table.}$
 $\text{Some } (\text{data}, \text{attacker-state})$
 $\in \text{set-dist}$
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\quad \text{adversary-initial-state}) \implies$
 $\text{composition-fri-multiround-bad-sets bad dg composition-table}$
 $\subseteq B \text{ dg}$
by (rule envelope)
show $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\quad (\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B))$
 $\text{adversary-initial-state} \leq \text{Fresh}$
by (rule fresh-bound)
show $\bigwedge \text{data attacker-state.}$
 $\text{Some } (\text{data}, \text{attacker-state})$
 $\in \text{set-dist}$
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\quad \text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap}$
 $\quad (\text{verifier-state-from-adversary attacker-state}$
 $\quad (\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $\quad (\text{staged-proof-transcript data})) \leq 0$
by (rule
 $\text{wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero})$
show $\bigwedge \text{data attacker-state.}$
 $\text{Some } (\text{data}, \text{attacker-state})$
 $\in \text{set-dist}$
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\quad \text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $\quad (\text{verifier-state-from-adversary attacker-state}$
 $\quad (\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $\quad (\text{staged-proof-transcript data})) \leq \text{Merkle}$
by (rule merkle-bound)
show $\bigwedge \text{data attacker-state.}$
 $\text{Some } (\text{data}, \text{attacker-state})$
 $\in \text{set-dist}$
 $(\text{execute } (\text{checked-staged-transcript-program } A)$

```

    adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-recorded-base-chunk-auth-missing
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq 0$ 
by (subst wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero)
simp
show  $\bigwedge$ data attacker-state.
  Some (data, attacker-state)
   $\in$  set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-next-value-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq \text{Next}$ 
by (rule next-bound)
show  $\bigwedge$ data attacker-state.
  Some (data, attacker-state)
   $\in$  set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-successor-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq \text{Successor}$ 
by (rule successor-bound)
show  $\bigwedge$ data attacker-state.
  Some (data, attacker-state)
   $\in$  set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-final-value-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq \text{Final}$ 
by (rule final-bound)
show  $\bigwedge$ data attacker-state.
  Some (data, attacker-state)
   $\in$  set-dist
    (execute (checked-staged-transcript-program A)

```

```

      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-zero-round-final-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq 0$ 
by (subst wp-composition-fri-verifier-tied-zero-round-final-obstruction-zero)
simp
qed

lemma checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-zero-slot-mis
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
and cover:
 $\bigwedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    composition-fri-sampled-layer-chain-cover
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) bad
and envelope:
 $\bigwedge$ data attacker-state dg composition-table.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    composition-fri-multiround-bad-sets bad dg composition-table
       $\subseteq B \ dg$ 
and fresh-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$ ))
  adversary-initial-state  $\leq \text{Fresh}$ 
and merkle-bound:
 $\bigwedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (partial-merkle-inconsistency-bad
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq \text{Merkle}$ 
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event

```

composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state $\leq$
(Fresh +
hash-relation-budget-value
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \text{ } dg) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $((\text{Merkle} + 0) + (\text{Merkle} + 0) + (\text{Merkle} + 0) + (\text{Merkle} + 0 + 0) +$
 $\text{Merkle}) + 0 + 0)$
proof (rule
checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-zero-slot-missing-a
show *staged-budget-wellformed budgets*
by (rule *wf*)
show *staged-adversary-controlled budgets A*
by (rule *controlled*)
show $\bigwedge dg. \text{finite } (B \text{ } dg)$
by (rule *finite-B*)
show $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state)
 $\in \text{set-dist}$
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
composition-fri-sampled-layer-chain-cover
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad}$
by (rule *cover*)
show $\bigwedge \text{data attacker-state } dg \text{ composition-table.}$
Some (data, attacker-state)
 $\in \text{set-dist}$
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \text{ } dg$
by (rule *envelope*)
show *wp-event (checked-staged-security-experiment-with-data-state A)*
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \text{ } B))$
 $\text{adversary-initial-state} \leq \text{Fresh}$
by (rule *fresh-bound*)
show $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state)
 $\in \text{set-dist}$
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
wp-event verify-monad
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$

```

      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data)) ≤ Merkle
    by (rule merkle-bound)
  show  $\bigwedge$  data attacker-state.
    Some (data, attacker-state)
    ∈ set-dist
      (execute (checked-staged-transcript-program A)
               adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (composition-fri-verifier-tied-next-value-replay-gap
       (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))))
      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data)) ≤ Merkle + 0
  proof -
    fix data attacker-state
    assume support:
      Some (data, attacker-state)
      ∈ set-dist
        (execute (checked-staged-transcript-program A)
                 adversary-initial-state)
    let ?s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    show wp-event verify-monad
      (composition-fri-verifier-tied-next-value-replay-gap ?s) ?s
      ≤ Merkle + 0
    by (rule
        wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-merkle-and-slot)
      (rule merkle-bound[OF support],
       rule
        wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
  qed
  show  $\bigwedge$  data attacker-state.
    Some (data, attacker-state)
    ∈ set-dist
      (execute (checked-staged-transcript-program A)
               adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (composition-fri-verifier-tied-successor-replay-gap
       (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))))
      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data)) ≤ Merkle + 0
  proof -
    fix data attacker-state
    assume support:
      Some (data, attacker-state)
      ∈ set-dist

```

```

      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
show wp-event verify-monad
  (composition-fri-verifier-tied-successor-replay-gap ?s) ?s
  ≤ Merkle + 0
by (rule
  wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-merkle-and-slot)
(rule merkle-bound[OF support],
rule
  wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
qed
show  $\bigwedge$  data attacker-state.
  Some (data, attacker-state)
  ∈ set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-final-value-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ Merkle + 0 + 0
proof -
  fix data attacker-state
  assume support:
    Some (data, attacker-state)
    ∈ set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
show wp-event verify-monad
  (composition-fri-verifier-tied-final-value-replay-gap ?s) ?s
  ≤ Merkle + 0 + 0
by (rule
  wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
(rule merkle-bound[OF support],
rule
  wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero,
rule
  wp-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero)
qed
qed

```

lemma *active-composition-fri-verifier-tied-reductions-from-sampled-and-merkle-bounds:*

```

fixes Sampled Merkle ::
  'f staged-proof-data ⇒ 'f protocol-channel ⇒ prob
assumes sampled-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (composition-fri-bad-with-verifier-tied-sampled-layer-chain
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤ Sampled data attacker-state
and merkle-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (partial-merkle-inconsistency-bad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤ Merkle data attacker-state
and total-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    Sampled data attacker-state +
      (((Merkle data attacker-state + 0) +
        (Merkle data attacker-state + 0) +
        (Merkle data attacker-state + 0) +
        (Merkle data attacker-state + 0 + 0) +
        Merkle data attacker-state) + 0)
    ≤ composition-fri-error
and support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
shows
  composition-fri-verifier-tied-reduction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
have slot:

```

```

wp-event verify-monad
  (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
   ?s) ?s ≤ 0
by (rule
   wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
have next-gap:
wp-event verify-monad
  (composition-fri-verifier-tied-next-value-replay-gap ?s) ?s
  ≤ Merkle data attacker-state + 0
by (rule
   wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-merkle-and-slot)
  (rule merkle-bound[OF support], rule slot)
have successor:
wp-event verify-monad
  (composition-fri-verifier-tied-successor-replay-gap ?s) ?s
  ≤ Merkle data attacker-state + 0
by (rule
   wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-merkle-and-slot)
  (rule merkle-bound[OF support], rule slot)
have final:
wp-event verify-monad
  (composition-fri-verifier-tied-final-value-replay-gap ?s) ?s
  ≤ Merkle data attacker-state + 0 + 0
by (rule
   wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
  (rule merkle-bound[OF support], rule slot,
   rule
    wp-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero)
show ?thesis
by (rule
   composition-fri-verifier-tied-reduction-from-residual-bounds-with-recorded-missing-zero
   [OF sampled-bound[OF support] slot merkle-bound[OF support]
    next-gap successor final total-bound[OF support]])
qed

end

end

```

```

theory Soundness-FRI-Full-Cover-Active-Bounds
imports
  Soundness-FRI-Obligation-Normalization
  Soundness-FRI-Full-Cover-Domains
begin

```

Compact active-route bridges for the full-cover FRI split.

The imported layers already prove:

- a full-cover challenge-list/envelope bound;

- a missing-full-cover domain split;
- active trace/composition residual adapters.

This theory only packages those ingredients in the shape consumed by the same-run public soundness route. It does not add protocol assumptions or change the FRI model.

context *soundness*
begin

lemma *wp-trace-fri-header-tied-sampled-layer-chain-bound-from-envelope-domain-split:*

assumes *future: trace-fri-future-fresh s*
and *envelope:*
 $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *subset: $B \subseteq \text{fri-challenge-space (ceil-log clength)}$*
and *challenge-bound:*
 $\text{nnreal (card } B) / \text{nnreal (CARD('f) } \wedge \text{ceil-log clength)} \leq R$
and *proximity:*
 $\text{wp-event verify-monad (trace-fri-sampled-full-cover-proximity-failure } s \text{ bad)} s \leq P$
and *index:*
 $\text{wp-event verify-monad (trace-fri-sampled-full-cover-index-failure } s) s \leq I$
and *domain-length:*
 $\text{wp-event verify-monad (trace-fri-sampled-full-cover-domain-length-failure } s) s \leq DL$
and *sampled-domain:*
 $\text{wp-event verify-monad (trace-fri-sampled-full-cover-sampled-domain-failure } s) s \leq SD$
shows
 $\text{wp-event verify-monad (trace-fri-bad-with-header-tied-sampled-layer-chain } s) s \leq R + (P + I + (I + DL) + (I + SD))$

proof (*rule*

wp-trace-fri-header-tied-sampled-layer-chain-bound-from-full-cover-and-missing)
show *wp-event verify-monad*
 $(\text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad}) s \leq R$
by (*rule wp-trace-fri-bad-with-full-cover-layer-chain-bound-from-envelope*
 $[\text{OF future envelope subset challenge-bound}]$)
show *wp-event verify-monad*
 $(\text{trace-fri-sampled-missing-full-cover } s \text{ bad}) s \leq P + I + (I + DL) + (I + SD)$
by (*rule wp-trace-fri-sampled-missing-full-cover-bound-from-domain-split*
 $[\text{OF proximity index domain-length sampled-domain}]$)

qed

lemma *wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-envelope-domain-split:*

assumes *future: composition-fri-future-fresh s*
and *envelope:*
 $\bigwedge dg \text{ composition-table.}$

$\text{composition-fri-multi-round-bad-sets bad dg composition-table}$
 $\subseteq B \text{ dg}$
and subset:
 $\bigwedge \text{dg. } B \text{ dg} \subseteq \text{fri-challenge-space (ceil-log (to-nat dg + 1))}$
and challenge-bound:
 $\bigwedge \text{dg. nnreal (card (B dg)) /}$
 $\text{nnreal (CARD('f) } \wedge \text{ceil-log (to-nat dg + 1))} \leq R$
and proximity:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure s bad}) s$
 $\leq P$
and index:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure s}) s \leq I$
and domain-length:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-length-failure s}) s$
 $\leq DL$
and sampled-domain:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-sampled-domain-failure s}) s$
 $\leq SD$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain s}) s$
 $\leq R + (P + I + (I + DL) + (I + SD))$
proof (rule
 $\text{wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-full-cover-and-missing})$
show wp-event verify-monad
 $(\text{composition-fri-bad-with-full-cover-layer-chain s bad}) s \leq R$
by (rule
 $\text{wp-composition-fri-bad-with-full-cover-layer-chain-bound-from-envelope}$
 $[\text{OF future envelope subset challenge-bound}])$
show wp-event verify-monad
 $(\text{composition-fri-sampled-missing-full-cover s bad}) s$
 $\leq P + I + (I + DL) + (I + SD)$
by (rule
 $\text{wp-composition-fri-sampled-missing-full-cover-bound-from-domain-split}$
 $[\text{OF proximity index domain-length sampled-domain}])$
qed

lemma wp-trace-fri-header-tied-sampled-layer-chain-bound-from-envelope-witness-domain:
assumes future: $\text{trace-fri-future-fresh s}$
and envelope:
 $\bigwedge \text{trace-table. trace-fri-multi-round-bad-sets bad trace-table} \subseteq B$
and subset: $B \subseteq \text{fri-challenge-space (ceil-log clength)}$
and challenge-bound:
 $\text{nnreal (card B) / nnreal (CARD('f) } \wedge \text{ceil-log clength)} \leq R$
and proximity:

$wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}proximity\text{-}failure\ s\ bad)\ s \leq P$
and $index$:
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}index\text{-}failure\ s)\ s \leq I$
and $witness\text{-}length$:
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}witness\text{-}length\text{-}failure\ s)\ s \leq W$
and $domain$:
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}domain\text{-}failure\ s)\ s \leq D$
shows
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}bad\text{-}with\text{-}header\text{-}tied\text{-}sampled\text{-}layer\text{-}chain\ s)\ s$
 $\leq R + (P + I + W + D)$
proof (rule
 $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}layer\text{-}chain\text{-}bound\text{-}from\text{-}full\text{-}cover\text{-}and\text{-}missing$)
show $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}bad\text{-}with\text{-}full\text{-}cover\text{-}layer\text{-}chain\ s\ bad)\ s \leq R$
by (rule $wp\text{-}trace\text{-}fri\text{-}bad\text{-}with\text{-}full\text{-}cover\text{-}layer\text{-}chain\text{-}bound\text{-}from\text{-}envelope$
 $[OF\ future\ envelope\ subset\ challenge\text{-}bound]$)
show $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}missing\text{-}full\text{-}cover\ s\ bad)\ s \leq P + I + W + D$
by (rule
 $wp\text{-}trace\text{-}fri\text{-}sampled\text{-}missing\text{-}full\text{-}cover\text{-}bound\text{-}from\text{-}witness\text{-}length\text{-}subcases$
 $[OF\ proximity\ index\ witness\text{-}length\ domain]$)
qed

lemma $wp\text{-}composition\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}layer\text{-}chain\text{-}bound\text{-}from\text{-}envelope\text{-}witness\text{-}domain$:
assumes $future$: $composition\text{-}fri\text{-}future\text{-}fresh\ s$
and $envelope$:
 $\bigwedge dg.\ composition\text{-}table.$
 $composition\text{-}fri\text{-}multi\text{-}round\text{-}bad\text{-}sets\ bad\ dg\ composition\text{-}table$
 $\subseteq B\ dg$
and $subset$:
 $\bigwedge dg.\ B\ dg \subseteq fri\text{-}challenge\text{-}space\ (ceil\text{-}log\ (to\text{-}nat\ dg + 1))$
and $challenge\text{-}bound$:
 $\bigwedge dg.\ nnreal\ (card\ (B\ dg)) /$
 $nnreal\ (CARD('f) \wedge ceil\text{-}log\ (to\text{-}nat\ dg + 1)) \leq R$
and $proximity$:
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}proximity\text{-}failure\ s\ bad)\ s$
 $\leq P$
and $index$:
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}index\text{-}failure\ s)\ s \leq I$
and $witness\text{-}length$:
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}witness\text{-}length\text{-}failure\ s)\ s$

$\leq W$
and domain:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-failure } s) \ s \leq D$
shows
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain } s) \ s$
 $\leq R + (P + I + W + D)$
proof (rule
 $wp\text{-composition-fri-verifier-tied-sampled-layer-chain-bound-from-full-cover-and-missing})$
show $wp\text{-event verify-monad}$
 $(\text{composition-fri-bad-with-full-cover-layer-chain } s \text{ bad}) \ s \leq R$
by (rule
 $wp\text{-composition-fri-bad-with-full-cover-layer-chain-bound-from-envelope}$
 $[OF \text{ future envelope subset challenge-bound}])$
show $wp\text{-event verify-monad}$
 $(\text{composition-fri-sampled-missing-full-cover } s \text{ bad}) \ s$
 $\leq P + I + W + D$
by (rule
 $wp\text{-composition-fri-sampled-missing-full-cover-bound-from-witness-length-subcases}$
 $[OF \text{ proximity index witness-length domain}])$
qed

lemma *trace-fri-header-tied-residual-obligations-from-envelope-domain-split-without-head-slot-and-selected-zero:*

assumes *future: trace-fri-future-fresh s*
and *envelope:*
 $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *subset: $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$*
and *challenge-bound:*
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log clength}) \leq R$
and *proximity:*
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$
and *index:*
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and *domain-length:*
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-length-failure } s) \ s \leq DL$
and *sampled-domain:*
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-sampled-domain-failure } s) \ s \leq SD$
and *zero:*
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq z\text{-b}$
and *merkle:*
 $wp\text{-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq p\text{-b}$
and *structural:*
 $wp\text{-event verify-monad}$

$(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s$
 $\leq h\text{-}b$
and *next-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq n\text{-}b$
and *successor*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq s\text{-}b$
and *total*:
 $(R + (P + I + (I + DL) + (I + SD))) +$
 $(((((p\text{-}b + h\text{-}b) + p\text{-}b) + n\text{-}b + s\text{-}b +$
 $(p\text{-}b + 0 + 0)) + p\text{-}b) + 0 + z\text{-}b)$
 $\leq \text{trace-fri-error}$
shows
 $\text{trace-fri-header-tied-residual-obligations } s$
 $(R + (P + I + (I + DL) + (I + SD))) \ z\text{-}b \ 0 \ p\text{-}b \ h\text{-}b \ p\text{-}b$
 $n\text{-}b \ s\text{-}b \ 0$
by (*rule*
 $\text{trace-fri-header-tied-residual-obligations-from-bounds-without-head-slot-and-selected-zero}$)
(*rule*
 $\text{wp-trace-fri-header-tied-sampled-layer-chain-bound-from-envelope-domain-split}$
 $[\text{OF future envelope subset challenge-bound proximity index}$
 $\text{domain-length sampled-domain}],$
 $\text{rule zero, rule merkle, rule structural, rule next-bound,}$
 $\text{rule successor, rule total})$

lemma *composition-fri-verifier-tied-residual-obligations-from-envelope-domain-split-slot-zero*:
assumes *future: composition-fri-future-fresh s*
and *envelope*:
 $\bigwedge dg \ \text{composition-table.}$
 $\text{composition-fri-multi-round-bad-sets bad } dg \ \text{composition-table}$
 $\subseteq B \ dg$
and *subset*:
 $\bigwedge dg. \ B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$
and *challenge-bound*:
 $\bigwedge dg. \ \text{nnreal } (\text{card } (B \ dg)) /$
 $\text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq R$
and *proximity*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure } s \ \text{bad}) \ s$
 $\leq P$
and *index*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and *domain-length*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-length-failure } s) \ s$
 $\leq DL$
and *sampled-domain*:

$wp\text{-event verify-monad}$
 $(composition\text{-fri-sampled-full-cover-sampled-domain-failure } s) \ s$
 $\leq SD$
and merkle:
 $wp\text{-event verify-monad } (partial\text{-merkle-inconsistency-bad } s) \ s \leq p\text{-}b$
and next-bound:
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-verifier-tied-next-value-replay-gap } s) \ s \leq n\text{-}b$
and successor:
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-verifier-tied-successor-replay-gap } s) \ s \leq s\text{-}b$
and final-bound:
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-verifier-tied-final-value-replay-gap } s) \ s \leq g\text{-}b$
and total:
 $(R + (P + I + (I + DL) + (I + SD))) +$
 $((p\text{-}b + 0) + n\text{-}b + s\text{-}b + g\text{-}b + p\text{-}b) + 0)$
 $\leq composition\text{-fri-error}$
shows
 $composition\text{-fri-verifier-tied-residual-obligations } s$
 $(R + (P + I + (I + DL) + (I + SD))) \ 0 \ p\text{-}b \ n\text{-}b \ s\text{-}b \ g\text{-}b$
by (rule $composition\text{-fri-verifier-tied-residual-obligations-from-bounds-slot-zero}$)
(rule
 $wp\text{-composition-fri-verifier-tied-sampled-layer-chain-bound-from-envelope-domain-split}$
 $[OF \ future \ envelope \ subset \ challenge\text{-bound} \ proximity \ index$
 $domain\text{-length} \ sampled\text{-domain}]$,
rule merkle, rule next-bound, rule successor, rule final-bound,
rule total)

lemma $trace\text{-fri-header-tied-residual-obligations-from-envelope-witness-domain-without-head-slot-and-selected-zero}$:
assumes $future: trace\text{-fri-future-fresh } s$
and envelope:
 $\bigwedge trace\text{-table. } trace\text{-fri-multiround-bad-sets } bad \ trace\text{-table} \subseteq B$
and subset: $B \subseteq fri\text{-challenge-space } (ceil\text{-log } clength)$
and challenge-bound:
 $nnreal \ (card \ B) / nnreal \ (CARD('f) \wedge ceil\text{-log } clength) \leq R$
and proximity:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-sampled-full-cover-proximity-failure } s \ bad) \ s \leq P$
and index:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and witness-length:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-sampled-full-cover-witness-length-failure } s) \ s \leq W$
and domain:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-sampled-full-cover-domain-failure } s) \ s \leq D$
and zero:

$wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq z\text{-b}$
and merkle:
 $wp\text{-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq p\text{-b}$
and structural:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s$
 $\leq h\text{-b}$
and next-bound:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq n\text{-b}$
and successor:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq s\text{-b}$
and total:
 $(R + (P + I + W + D)) +$
 $(((((p\text{-b} + h\text{-b}) + p\text{-b}) + n\text{-b} + s\text{-b} +$
 $(p\text{-b} + 0 + 0)) + p\text{-b}) + 0 + z\text{-b})$
 $\leq \text{trace-fri-error}$
shows
 $\text{trace-fri-header-tied-residual-obligations } s$
 $(R + (P + I + W + D)) \ z\text{-b } 0 \ p\text{-b } h\text{-b } p\text{-b } n\text{-b } s\text{-b } 0$
by (rule
 $\text{trace-fri-header-tied-residual-obligations-from-bounds-without-head-slot-and-selected-zero})$
(rule
 $wp\text{-trace-fri-header-tied-sampled-layer-chain-bound-from-envelope-witness-domain}$
 $[OF \text{ future envelope subset challenge-bound proximity index}$
 $\text{witness-length domain}]$,
rule zero, rule merkle, rule structural, rule next-bound,
rule successor, rule total)

lemma *composition-fri-verifier-tied-residual-obligations-from-envelope-witness-domain-slot-zero:*
assumes future: *composition-fri-future-fresh* s
and envelope:
 $\bigwedge dg \text{ composition-table.}$
 $\text{composition-fri-multi-round-bad-sets bad } dg \text{ composition-table}$
 $\subseteq B \ dg$
and subset:
 $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$
and challenge-bound:
 $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) /$
 $\text{nnreal } (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq R$
and proximity:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s$
 $\leq P$
and index:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure } s) \ s \leq I$

```

and witness-length:
  wp-event verify-monad
    (composition-fri-sampled-full-cover-witness-length-failure s) s
    ≤ W
and domain:
  wp-event verify-monad
    (composition-fri-sampled-full-cover-domain-failure s) s ≤ D
and merkle:
  wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ p-b
and next-bound:
  wp-event verify-monad
    (composition-fri-verifier-tied-next-value-replay-gap s) s ≤ n-b
and successor:
  wp-event verify-monad
    (composition-fri-verifier-tied-successor-replay-gap s) s ≤ s-b
and final-bound:
  wp-event verify-monad
    (composition-fri-verifier-tied-final-value-replay-gap s) s ≤ g-b
and total:
  (R + (P + I + W + D)) +
  (((p-b + 0) + n-b + s-b + g-b + p-b) + 0)
  ≤ composition-fri-error
shows
  composition-fri-verifier-tied-residual-obligations s
  (R + (P + I + W + D)) 0 p-b n-b s-b g-b
by (rule composition-fri-verifier-tied-residual-obligations-from-bounds-slot-zero)
  (rule
    wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-envelope-witness-domain
    [OF future envelope subset challenge-bound proximity index
      witness-length domain],
    rule merkle, rule next-bound, rule successor, rule final-bound,
    rule total)

end

end

```

theory *Soundness-FRI-Full-Cover-Staged-Bounds*

imports

Soundness-FRI-Prequery-Composition-Sampled-Bounds

Soundness-FRI-Full-Cover-Active-Bounds

begin

Staged bounds for the full-cover FRI split.

Unlike the verifier-local full-cover envelope lemmas, these results work in the staged adversary experiment where future FRI keys may have been queried by the attacker. The full-cover part is therefore routed through the existing staged fresh/prequery accounting for FRI challenge-list hits.

context *soundness*
begin

lemma *trace-fri-bad-with-sampled-layer-chain-not-None:*

¬ *trace-fri-bad-with-sampled-layer-chain s None*

unfolding *trace-fri-bad-with-sampled-layer-chain-def*

trace-fri-partial-candidate-opening-evidence-def

accepted-fri-opening-transcript-def

by *blast*

lemma *composition-fri-bad-with-sampled-layer-chain-not-None:*

¬ *composition-fri-bad-with-sampled-layer-chain s None*

unfolding *composition-fri-bad-with-sampled-layer-chain-def*

composition-fri-partial-candidate-opening-evidence-def

accepted-fri-opening-transcript-def

by *blast*

lemma *trace-fri-sampled-missing-full-cover-not-None:*

¬ *trace-fri-sampled-missing-full-cover s bad None*

unfolding *trace-fri-sampled-missing-full-cover-def*

using *trace-fri-bad-with-sampled-layer-chain-not-None* **by** *blast*

lemma *composition-fri-sampled-missing-full-cover-not-None:*

¬ *composition-fri-sampled-missing-full-cover s bad None*

unfolding *composition-fri-sampled-missing-full-cover-def*

using *composition-fri-bad-with-sampled-layer-chain-not-None* **by** *blast*

lemma *checked-staged-security-trace-fri-full-cover-layer-chain-bound-from-fresh-and-prequery:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *finite-B: finite B*

and *envelope:*

$\bigwedge \text{data attacker-state trace-table.}$

Some (data, attacker-state) ∈

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) ⇒

trace-fri-multi-round-bad-sets bad trace-table ⊆ B

and *fresh-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

(λs. trace-fri-challenge-list-fresh-hit s B))

adversary-initial-state ≤ F

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

(λs. trace-fri-bad-with-full-cover-layer-chain s bad))

adversary-initial-state

≤ F +

*hash-relation-budget-value (card B * ceil-log clength)*

```

      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)
proof –
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     (λs. trace-fri-bad-with-full-cover-layer-chain s bad))
    adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       (λs. trace-fri-challenge-list-set-hit s B))
      adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
      set-dist
        (execute (checked-staged-security-experiment-with-data-state A)
         adversary-initial-state)
  and hit:
    staged-security-with-data-state-verifier-event
      (λs. trace-fri-bad-with-full-cover-layer-chain s bad) out
from hit obtain data attacker-state result final-state where
  out-eq: out = Some (((data, attacker-state), result), final-state)
  unfolding staged-security-with-data-state-verifier-event-def
  by (cases out) (auto split: option.splits prod.splits)
have builder:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
     adversary-initial-state)
  using checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
  by blast
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have full:
  trace-fri-bad-with-full-cover-layer-chain ?s bad
    (Some (result, final-state))
  using hit unfolding staged-security-with-data-state-verifier-event-def
    out-eq by simp
have list-hit:
  trace-fri-challenge-list-set-hit ?s B
    (Some (result, final-state))
  by (rule trace-fri-bad-with-full-cover-layer-chain-imp-list-hit
    [OF full])
    (rule envelope[OF builder])
show
  staged-security-with-data-state-verifier-event
    (λs. trace-fri-challenge-list-set-hit s B) out

```

```

    unfolding staged-security-with-data-state-verifier-event-def out-eq
    using list-hit by simp
qed
also have ... ≤
  F +
    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)
  by (rule checked-staged-security-trace-fri-challenge-list-set-hit-bound
      [OF wf controlled finite-B fresh-bound])
  finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-full-cover-layer-chain-bound-from-fresh-and-prequery:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
  and envelope:
     $\bigwedge \text{data attacker-state } dg \text{ composition-table.}$ 
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
      composition-fri-multiround-bad-sets bad dg composition-table
       $\subseteq B \ dg$ 
  and fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$ ))
      adversary-initial-state  $\leq F$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. \text{composition-fri-bad-with-full-cover-layer-chain } s \ \text{bad}$ ))
      adversary-initial-state
       $\leq F +$ 
      hash-relation-budget-value
        ( $\sum dg \in (UNIV :: 'f \ \text{set}).$ 
          card (B dg) * ceil-log (maxDegree + 1))
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)
proof -
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{composition-fri-bad-with-full-cover-layer-chain } s \ \text{bad}$ ))
    adversary-initial-state  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. \text{composition-fri-challenge-list-set-hit } s \ B$ ))
      adversary-initial-state

```

```

proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
      set-dist
        (execute (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  and hit:
    staged-security-with-data-state-verifier-event
      (λs. composition-fri-bad-with-full-cover-layer-chain s bad) out
  from hit obtain data attacker-state result final-state where
    out-eq: out = Some (((data, attacker-state), result), final-state)
  unfolding staged-security-with-data-state-verifier-event-def
  by (cases out) (auto split: option.splits prod.splits)
  have builder:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  using checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
  by blast
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have full:
    composition-fri-bad-with-full-cover-layer-chain ?s bad
      (Some (result, final-state))
  using hit unfolding staged-security-with-data-state-verifier-event-def
    out-eq by simp
  have list-hit:
    composition-fri-challenge-list-set-hit ?s B
      (Some (result, final-state))
  by (rule composition-fri-bad-with-full-cover-layer-chain-imp-list-hit
    [OF full])
    (rule envelope[OF builder])
  show
    staged-security-with-data-state-verifier-event
      (λs. composition-fri-challenge-list-set-hit s B) out
  unfolding staged-security-with-data-state-verifier-event-def out-eq
  using list-hit by simp
qed
also have ... ≤
  F +
    hash-relation-budget-value
      (∑ dg ∈ (UNIV :: 'f set).
        card (B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
  by (rule

```

checked-staged-security-composition-fri-challenge-list-set-hit-bound
[OF wf controlled finite-B fresh-bound])

finally show *?thesis* .

qed

lemma *checked-staged-security-trace-fri-sampled-missing-full-cover-bound-from-domain-split:*

assumes *proximity:*

$\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-sampled-full-cover-proximity-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad)
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ P

and *index:*

$\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-sampled-full-cover-index-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ I

and *domain-length:*

$\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-sampled-full-cover-domain-length-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ DL

and *sampling-domain:*

$\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-sampled-full-cover-sampled-domain-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state

```

      (staged-proof-transcript data)) ≤ SD
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-sampled-missing-full-cover s bad))
  adversary-initial-state ≤ P + I + (I + DL) + (I + SD)
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show ∧s. ¬ trace-fri-sampled-missing-full-cover s bad None
    by (rule trace-fri-sampled-missing-full-cover-not-None)
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show wp-event verify-monad
  (trace-fri-sampled-missing-full-cover ?s bad) ?s
  ≤ P + I + (I + DL) + (I + SD)
by (rule wp-trace-fri-sampled-missing-full-cover-bound-from-domain-split)
  (rule proximity[OF support], rule index[OF support],
    rule domain-length[OF support], rule sampled-domain[OF support])
qed

lemma checked-staged-security-trace-fri-sampled-layer-chain-bound-from-full-cover-fresh-and-domain-split:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B: finite B
  and envelope:
    ∧data attacker-state trace-table.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
        trace-fri-multi-round-bad-sets bad trace-table ⊆ B
  and fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-challenge-list-fresh-hit s B))
    adversary-initial-state ≤ F
  and proximity:
    ∧data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
    wp-event verify-monad
      (trace-fri-sampled-full-cover-proximity-failure
        (verifier-state-from-adversary attacker-state

```

(staged-proof-transcript data)) bad)
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq P$
and index:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-sampled-full-cover-index-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq I$
and domain-length:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-sampled-full-cover-domain-length-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq DL$
and sampled-domain:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-sampled-full-cover-sampled-domain-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq SD$
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-sampled-layer-chain)
 adversary-initial-state
 $\leq$
 (F +
 hash-relation-budget-value (card B * ceil-log clength)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) +
 (P + I + (I + DL) + (I + SD))
proof –
have full-bound:

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. trace\text{-}fri\text{-}bad\text{-}with\text{-}full\text{-}cover\text{-}layer\text{-}chain\ s\ bad$))
adversary-initial-state
 $\leq$
 $F +$
hash-relation-budget-value (*card B * ceil-log clength*)
 (*staged-attacker-query-budget budgets +*
staged-challenge-query-budget)
by (*rule*
checked-staged-security-trace-fri-full-cover-layer-chain-bound-from-fresh-and-prequery
 [*OF wf controlled finite-B envelope fresh-bound*])
have *missing-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. trace\text{-}fri\text{-}sampled\text{-}missing\text{-}full\text{-}cover\ s\ bad$))
adversary-initial-state $\leq P + I + (I + DL) + (I + SD)$
by (*rule*
checked-staged-security-trace-fri-sampled-missing-full-cover-bound-from-domain-split
 [*OF proximity index domain-length sampled-domain*])
have $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-sampled-layer-chain)
adversary-initial-state $\leq$
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state A*)
 ($\lambda out.$
staged-security-with-data-state-verifier-event
 ($\lambda s. trace\text{-}fri\text{-}bad\text{-}with\text{-}full\text{-}cover\text{-}layer\text{-}chain\ s\ bad$) *out* $\vee$
staged-security-with-data-state-verifier-event
 ($\lambda s. trace\text{-}fri\text{-}sampled\text{-}missing\text{-}full\text{-}cover\ s\ bad$) *out*)
adversary-initial-state
by (*rule wp-event-mono-on-support*)
 (*auto simp: staged-security-with-data-state-verifier-event-def*
dest: trace-fri-sampled-imp-full-cover-or-missing
split: option.splits prod.splits)
also have ... $\leq$
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. trace\text{-}fri\text{-}bad\text{-}with\text{-}full\text{-}cover\text{-}layer\text{-}chain\ s\ bad$))
adversary-initial-state $+$
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. trace\text{-}fri\text{-}sampled\text{-}missing\text{-}full\text{-}cover\ s\ bad$))
adversary-initial-state
by (*rule wp-event-union-bound*)
also have ... $\leq$
 ($F +$
hash-relation-budget-value (*card B * ceil-log clength*)
 (*staged-attacker-query-budget budgets +*

$$\text{staged-challenge-query-budget})) +$$

$$(P + I + (I + DL) + (I + SD))$$
by (rule add-mono[*OF full-bound missing-bound*])
 finally show ?thesis .

qed

lemma *checked-staged-security-trace-fri-canonical-sampled-layer-chain-bound-from-full-cover-fresh-and-domain-split:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: finite B*
and *envelope:*
 $\bigwedge \text{data attacker-state trace-table.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *fresh-bound:*
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-challenge-list-fresh-hit s B}))$
 $\text{adversary-initial-state} \leq F$
and *proximity:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-proximity-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq P$
and *index:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$
and *domain-length:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$

```

    (trace-fri-sampled-full-cover-domain-length-failure
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤ DL
and sampled-domain:
 $\wedge$  data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-sampled-full-cover-sampled-domain-failure
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ SD
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-canonical-sampled-layer-chain)
  adversary-initial-state
≤
(F +
  hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)) +
(P + I + (I + DL) + (I + SD))
proof –
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-canonical-sampled-layer-chain)
  adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-sampled-layer-chain)
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    intro: trace-fri-bad-with-canonical-sampled-layer-chain-imp-sampled
    split: option.splits prod.splits)
also have ... ≤
(F +
  hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)) +
(P + I + (I + DL) + (I + SD))
by (rule
  checked-staged-security-trace-fri-sampled-layer-chain-bound-from-full-cover-fresh-and-domain-split
    [OF wf controlled finite-B envelope fresh-bound proximity index

```

$\text{domain-length sampled-domain}]$)
finally show ?thesis .
qed

lemma *checked-staged-security-composition-fri-sampled-missing-full-cover-bound-from-domain-split:*

assumes *proximity:*

$\wedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$

$\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq P$

and *index:*

$\wedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$

$\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$

and *domain-length:*

$\wedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$

$\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq DL$

and *sampled-domain:*

$\wedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$

$\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-sampled-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq SD$

shows
 $wp\text{-event } (checked\text{-staged}\text{-security}\text{-experiment}\text{-with}\text{-data}\text{-state } A)$
 $(staged\text{-security}\text{-with}\text{-data}\text{-state}\text{-verifier}\text{-event}$
 $(\lambda s. composition\text{-fri}\text{-sampled}\text{-missing}\text{-full}\text{-cover } s \text{ bad}))$
 $adversary\text{-initial}\text{-state} \leq P + I + (I + DL) + (I + SD)$
proof (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
show $\bigwedge s. \neg composition\text{-fri}\text{-sampled}\text{-missing}\text{-full}\text{-cover } s \text{ bad None}$
by (rule *composition-fri-sampled-missing-full-cover-not-None*)
next
fix *data attacker-state*
assume *support*:
 $Some (data, attacker\text{-state}) \in$
 $set\text{-dist } (execute (checked\text{-staged}\text{-transcript}\text{-program } A)$
 $adversary\text{-initial}\text{-state})$
let $?s =$
 $verifier\text{-state}\text{-from}\text{-adversary } attacker\text{-state}$
 $(staged\text{-proof}\text{-transcript } data)$
show $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri}\text{-sampled}\text{-missing}\text{-full}\text{-cover } ?s \text{ bad}) ?s$
 $\leq P + I + (I + DL) + (I + SD)$
by (rule
 $wp\text{-composition}\text{-fri}\text{-sampled}\text{-missing}\text{-full}\text{-cover}\text{-bound}\text{-from}\text{-domain}\text{-split})$
(rule *proximity*[*OF support*], rule *index*[*OF support*],
rule *domain-length*[*OF support*], rule *sampled-domain*[*OF support*])
qed

lemma *checked-staged-security-composition-fri-sampled-layer-chain-bound-from-full-cover-fresh-and-domain-split*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *finite-B*: $\bigwedge dg. finite (B dg)$
and *envelope*:
 $\bigwedge data attacker\text{-state } dg \text{ composition}\text{-table}.$
 $Some (data, attacker\text{-state}) \in$
 $set\text{-dist } (execute (checked\text{-staged}\text{-transcript}\text{-program } A)$
 $adversary\text{-initial}\text{-state}) \implies$
 $composition\text{-fri}\text{-multiround}\text{-bad}\text{-sets } bad \text{ dg } composition\text{-table}$
 $\subseteq B dg$
and *fresh-bound*:
 $wp\text{-event } (checked\text{-staged}\text{-security}\text{-experiment}\text{-with}\text{-data}\text{-state } A)$
 $(staged\text{-security}\text{-with}\text{-data}\text{-state}\text{-verifier}\text{-event}$
 $(\lambda s. composition\text{-fri}\text{-challenge}\text{-list}\text{-fresh}\text{-hit } s B))$
 $adversary\text{-initial}\text{-state} \leq F$
and *proximity*:
 $\bigwedge data attacker\text{-state}.$
 $Some (data, attacker\text{-state}) \in$
 $set\text{-dist } (execute (checked\text{-staged}\text{-transcript}\text{-program } A)$
 $adversary\text{-initial}\text{-state}) \implies$
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri}\text{-sampled}\text{-full}\text{-cover}\text{-proximity}\text{-failure}$

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq P$
and *index*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$
and *domain-length*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq DL$
and *sampled-domain*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-sampled-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq SD$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-sampled-layer-chain})$
 $\text{adversary-initial-state}$
 $\leq$
 $(F +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card (B dg) * ceil-log (maxDegree + 1))}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$

$(P + I + (I + DL) + (I + SD))$
proof –
have *full-bound*:
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $(\lambda s. composition\text{-}fri\text{-}bad\text{-}with\text{-}full\text{-}cover\text{-}layer\text{-}chain\ s\ bad))$
 $adversary\text{-}initial\text{-}state$
 $\leq$
 $F +$
 $hash\text{-}relation\text{-}budget\text{-}value$
 $(\sum dg \in (UNIV :: 'f\ set).$
 $card\ (B\ dg) * ceil\text{-}log\ (maxDegree + 1))$
 $(staged\text{-}attacker\text{-}query\text{-}budget\ budgets +$
 $staged\text{-}challenge\text{-}query\text{-}budget)$
by (rule
 $checked\text{-}staged\text{-}security\text{-}composition\text{-}fri\text{-}full\text{-}cover\text{-}layer\text{-}chain\text{-}bound\text{-}from\text{-}fresh\text{-}and\text{-}prequery$
 $[OF\ wf\ controlled\ finite\text{-}B\ envelope\ fresh\text{-}bound])$
have *missing-bound*:
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $(\lambda s. composition\text{-}fri\text{-}sampled\text{-}missing\text{-}full\text{-}cover\ s\ bad))$
 $adversary\text{-}initial\text{-}state \leq P + I + (I + DL) + (I + SD)$
by (rule
 $checked\text{-}staged\text{-}security\text{-}composition\text{-}fri\text{-}sampled\text{-}missing\text{-}full\text{-}cover\text{-}bound\text{-}from\text{-}domain\text{-}split$
 $[OF\ proximity\ index\ domain\text{-}length\ sampled\text{-}domain])$
have $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $composition\text{-}fri\text{-}bad\text{-}with\text{-}sampled\text{-}layer\text{-}chain)$
 $adversary\text{-}initial\text{-}state \leq$
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(\lambda out.$
 $staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $(\lambda s. composition\text{-}fri\text{-}bad\text{-}with\text{-}full\text{-}cover\text{-}layer\text{-}chain\ s\ bad)$
 $out \vee$
 $staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $(\lambda s. composition\text{-}fri\text{-}sampled\text{-}missing\text{-}full\text{-}cover\ s\ bad)\ out)$
 $adversary\text{-}initial\text{-}state$
by (rule *wp-event-mono-on-support*)
 $(auto\ simp: staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event\text{-}def$
 $dest: composition\text{-}fri\text{-}sampled\text{-}imp\text{-}full\text{-}cover\text{-}or\text{-}missing$
 $split: option.splits\ prod.splits)$
also have $\dots \leq$
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $(\lambda s. composition\text{-}fri\text{-}bad\text{-}with\text{-}full\text{-}cover\text{-}layer\text{-}chain\ s\ bad))$
 $adversary\text{-}initial\text{-}state +$
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $(\lambda s. composition\text{-}fri\text{-}sampled\text{-}missing\text{-}full\text{-}cover\ s\ bad))$

adversary-initial-state
by (rule *wp-event-union-bound*)
also have ... $\leq$
 ($F +$
 hash-relation-budget-value
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + (I + DL) + (I + SD))$
by (rule *add-mono[OF full-bound missing-bound]*)
finally show ?thesis .
qed

lemma *checked-staged-security-composition-fri-canonical-sampled-layer-chain-bound-from-full-cover-fresh-and-domain*

assumes *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *finite-B*: $\bigwedge dg. \text{finite } (B \ dg)$
and *envelope*:
 $\bigwedge \text{data attacker-state } dg \text{ composition-table.}$
 Some (*data*, *attacker-state*) $\in$
 set-dist (*execute* (*checked-staged-transcript-program* *A*)
 adversary-initial-state) $\implies$
 composition-fri-multiround-bad-sets *bad* *dg* *composition-table*
 $\subseteq B \ dg$
and *fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 $(\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B))$
 adversary-initial-state $\leq F$
and *proximity*:
 $\bigwedge \text{data attacker-state.}$
 Some (*data*, *attacker-state*) $\in$
 set-dist (*execute* (*checked-staged-transcript-program* *A*)
 adversary-initial-state) $\implies$
 wp-event *verify-monad*
 (*composition-fri-sampled-full-cover-proximity-failure*
 (*verifier-state-from-adversary* *attacker-state*
 (*staged-proof-transcript* *data*)) *bad*)
 (*verifier-state-from-adversary* *attacker-state*
 (*staged-proof-transcript* *data*)) $\leq P$
and *index*:
 $\bigwedge \text{data attacker-state.}$
 Some (*data*, *attacker-state*) $\in$
 set-dist (*execute* (*checked-staged-transcript-program* *A*)
 adversary-initial-state) $\implies$
 wp-event *verify-monad*
 (*composition-fri-sampled-full-cover-index-failure*
 (*verifier-state-from-adversary* *attacker-state*

$(\text{staged-proof-transcript } data)))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript } data)) \leq I$
and domain-length:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript } data)))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript } data)) \leq DL$
and sampling-domain:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-sampled-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript } data)))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript } data)) \leq SD$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-canonical-sampled-layer-chain})$
 $\text{adversary-initial-state}$
 $\leq$
 $(F +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + (I + DL) + (I + SD))$
proof –
have $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-canonical-sampled-layer-chain})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-sampled-layer-chain})$
 $\text{adversary-initial-state}$
by $(\text{rule wp-event-mono-on-support})$
 $(\text{auto simp: staged-security-with-data-state-verifier-event-def}$

intro: composition-fri-bad-with-canonical-sampled-layer-chain-imp-sampled
split: option.splits prod.splits)
also have ... $\leq$
 $(F +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + (I + DL) + (I + SD))$
by (rule
 $\text{checked-staged-security-composition-fri-sampled-layer-chain-bound-from-full-cover-fresh-and-domain-split}$
 $[OF \text{ wf controlled finite-B envelope fresh-bound proximity index}$
 $\text{domain-length sampled-domain}]$)
finally show ?thesis .
qed

lemma *checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-full-cover-fresh-and-domain-split*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: finite B*
and *envelope:*
 $\bigwedge \text{data attacker-state trace-table.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *fresh-bound:*
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B))$
 $\text{adversary-initial-state} \leq F$
and *proximity:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-proximity-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq P$
and *index:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$

(trace-fri-sampled-full-cover-index-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq I$
and domain-length:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-sampled-full-cover-domain-length-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq DL$
and sampled-domain:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-sampled-full-cover-sampled-domain-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq SD$
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-sampled-layer-chain)
 adversary-initial-state
 $\leq$
 (F +
 hash-relation-budget-value (card B * ceil-log clength)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) +
 (P + I + (I + DL) + (I + SD))
proof –
have full-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . trace-fri-bad-with-full-cover-layer-chain s bad))
 adversary-initial-state
 $\leq$
 F +
 hash-relation-budget-value (card B * ceil-log clength)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)

by (rule
checked-staged-security-trace-fri-full-cover-layer-chain-bound-from-fresh-and-prequery
 $[OF \text{ wf controlled finite-}B \text{ envelope fresh-bound}]$)
have *missing-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
 $(\lambda s. \text{ trace-fri-sampled-missing-full-cover } s \text{ bad})$)
adversary-initial-state $\leq P + I + (I + DL) + (I + SD)$)
by (rule
checked-staged-security-trace-fri-sampled-missing-full-cover-bound-from-domain-split
 $[OF \text{ proximity index domain-length sampled-domain}]$)
have *wp-event* (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-sampled-layer-chain)
adversary-initial-state $\leq$
wp-event (*checked-staged-security-experiment-with-data-state* A)
 $(\lambda out.$
staged-security-with-data-state-verifier-event
 $(\lambda s. \text{ trace-fri-bad-with-full-cover-layer-chain } s \text{ bad}) \text{ out} \vee$
staged-security-with-data-state-verifier-event
 $(\lambda s. \text{ trace-fri-sampled-missing-full-cover } s \text{ bad}) \text{ out})$
adversary-initial-state)
by (rule *wp-event-mono-on-support*)
 $(\text{auto simp: staged-security-with-data-state-verifier-event-def}$
 $\text{dest!: trace-fri-header-tied-sampled-layer-chain-imp-sampled-layer-chain}$
 $\text{dest: trace-fri-sampled-imp-full-cover-or-missing}$
 $\text{split: option.splits prod.splits})$
also have $\dots \leq$
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
 $(\lambda s. \text{ trace-fri-bad-with-full-cover-layer-chain } s \text{ bad})$)
adversary-initial-state +
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
 $(\lambda s. \text{ trace-fri-sampled-missing-full-cover } s \text{ bad})$)
adversary-initial-state)
by (rule *wp-event-union-bound*)
also have $\dots \leq$
 $(F +$
hash-relation-budget-value (*card* $B * \text{ceil-log clength}$)
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget)) +
 $(P + I + (I + DL) + (I + SD))$
by (rule *add-mono*[*OF full-bound missing-bound*])
finally show *?thesis* .
qed

lemma *checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-full-cover-fresh-and-dom*
assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: staged-adversary-controlled budgets A
and *finite-B*: $\bigwedge dg. \text{finite } (B \ dg)$
and *envelope*:
 $\bigwedge data \text{ attacker-state } dg \text{ composition-table.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{composition-fri-multiround-bad-sets bad } dg \text{ composition-table}$
 $\subseteq B \ dg$
and *fresh-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B))$
 $\text{adversary-initial-state} \leq F$
and *proximity*:
 $\bigwedge data \text{ attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq P$
and *index*:
 $\bigwedge data \text{ attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$
and *domain-length*:
 $\bigwedge data \text{ attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq DL$
and *sampled-domain*:
 $\bigwedge data \text{ attacker-state.}$

Some $(data, attacker-state) \in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-sampled-full-cover-sampled-domain-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq SD$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-sampled-layer-chain)
 adversary-initial-state
 $\leq$
 ($F +$
 hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \text{ set}).$
 card $(B \ dg) * \text{ceil-log} (\text{maxDegree} + 1)$)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) +
 ($P + I + (I + DL) + (I + SD)$)

proof –

have full-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 ($\lambda s.$ composition-fri-bad-with-full-cover-layer-chain s bad))
 adversary-initial-state
 $\leq$
 $F +$
 hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \text{ set}).$
 card $(B \ dg) * \text{ceil-log} (\text{maxDegree} + 1)$)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)

by (rule
 checked-staged-security-composition-fri-full-cover-layer-chain-bound-from-fresh-and-prequery
 [OF wf controlled finite-B envelope fresh-bound])

have missing-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 ($\lambda s.$ composition-fri-sampled-missing-full-cover s bad))
 adversary-initial-state $\leq P + I + (I + DL) + (I + SD)$

by (rule
 checked-staged-security-composition-fri-sampled-missing-full-cover-bound-from-domain-split
 [OF proximity index domain-length sampled-domain])

have wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-sampled-layer-chain)

```

adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
(λout.
  staged-security-with-data-state-verifier-event
    (λs. composition-fri-bad-with-full-cover-layer-chain s bad)
    out ∨
  staged-security-with-data-state-verifier-event
    (λs. composition-fri-sampled-missing-full-cover s bad)
    out)
adversary-initial-state
by (rule wp-event-mono-on-support)
(auto simp: staged-security-with-data-state-verifier-event-def
  dest!: composition-fri-verifier-tied-sampled-layer-chain-imp-sampled-layer-chain
  dest: composition-fri-sampled-imp-full-cover-or-missing
  split: option.splits prod.splits)
also have ... ≤
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
  (λs. composition-fri-bad-with-full-cover-layer-chain s bad))
adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
  (λs. composition-fri-sampled-missing-full-cover s bad))
adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
(F +
  hash-relation-budget-value
    (∑ dg ∈ (UNIV :: 'f set).
      card (B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)) +
(P + I + (I + DL) + (I + SD))
by (rule add-mono[OF full-bound missing-bound])
finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-sampled-missing-full-cover-bound-from-witness-domain:
  assumes proximity:
    ∧ data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
wp-event verify-monad
  (trace-fri-sampled-full-cover-proximity-failure
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) bad)
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ P

```

and *index*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-full-cover-index-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq I$
and *witness-length*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-full-cover-witness-length-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq W$
and *domain*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-full-cover-domain-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq D$
shows
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s.$ *trace-fri-sampled-missing-full-cover s bad*))
adversary-initial-state $\leq P + I + W + D$
proof (*rule checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
show $\wedge s. \neg \text{trace-fri-sampled-missing-full-cover } s \text{ bad None}$
by (*rule trace-fri-sampled-missing-full-cover-not-None*)
next
fix *data attacker-state*
assume *support*:
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state)
let *?s* =
verifier-state-from-adversary attacker-state

(staged-proof-transcript data)
show wp-event verify-monad
 (trace-fri-sampled-missing-full-cover ?s bad) ?s ≤
 P + I + W + D
by (rule
 wp-trace-fri-sampled-missing-full-cover-bound-from-witness-length-subcases)
 (rule proximity[OF support], rule index[OF support],
 rule witness-length[OF support], rule domain[OF support])
qed

lemma checked-staged-security-composition-fri-sampled-missing-full-cover-bound-from-witness-domain:
assumes proximity:
 ∧data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 wp-event verify-monad
 (composition-fri-sampled-full-cover-proximity-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) bad)
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤ P
and index:
 ∧data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 wp-event verify-monad
 (composition-fri-sampled-full-cover-index-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤ I
and witness-length:
 ∧data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 wp-event verify-monad
 (composition-fri-sampled-full-cover-witness-length-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤ W
and domain:
 ∧data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒


```

wp-event verify-monad
  (composition-fri-sampled-full-cover-domain-failure
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ D
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-sampled-missing-full-cover s bad))
  adversary-initial-state ≤ P + I + W + D
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show ∧s. ¬ composition-fri-sampled-missing-full-cover s bad None
  by (rule composition-fri-sampled-missing-full-cover-not-None)
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show wp-event verify-monad
  (composition-fri-sampled-missing-full-cover ?s bad) ?s ≤
  P + I + W + D
by (rule
  wp-composition-fri-sampled-missing-full-cover-bound-from-witness-length-subcases)
  (rule proximity[OF support], rule index[OF support],
  rule witness-length[OF support], rule domain[OF support])
qed

lemma checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-full-cover-fresh-and-witness-dom
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and envelope:
  ∧data attacker-state trace-table.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  trace-fri-multi-round-bad-sets bad trace-table ⊆ B
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-challenge-list-fresh-hit s B))
  adversary-initial-state ≤ F
and proximity:
  ∧data attacker-state.

```

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event *verify-monad*
(*trace-fri-sampled-full-cover-proximity-failure*
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)) *bad*)
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)) $\leq P$
and *index*:
 $\wedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event *verify-monad*
(*trace-fri-sampled-full-cover-index-failure*
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)))
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)) $\leq I$
and *witness-length*:
 $\wedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event *verify-monad*
(*trace-fri-sampled-full-cover-witness-length-failure*
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)))
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)) $\leq W$
and *domain*:
 $\wedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event *verify-monad*
(*trace-fri-sampled-full-cover-domain-failure*
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)))
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)) $\leq D$
shows
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-sampled-layer-chain)
adversary-initial-state
 $\leq$
(*F* +

```

    hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
    (P + I + W + D)
proof -
have full-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-bad-with-full-cover-layer-chain s bad))
  adversary-initial-state
  ≤
  F +
    hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
by (rule
  checked-staged-security-trace-fri-full-cover-layer-chain-bound-from-fresh-and-prequery
  [OF wf controlled finite-B envelope fresh-bound])
have missing-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-sampled-missing-full-cover s bad))
  adversary-initial-state ≤ P + I + W + D
by (rule
  checked-staged-security-trace-fri-sampled-missing-full-cover-bound-from-witness-domain
  [OF proximity index witness-length domain])
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-sampled-layer-chain)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (λout.
      staged-security-with-data-state-verifier-event
        (λs. trace-fri-bad-with-full-cover-layer-chain s bad) out ∨
      staged-security-with-data-state-verifier-event
        (λs. trace-fri-sampled-missing-full-cover s bad) out)
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    dest!: trace-fri-header-tied-sampled-layer-chain-imp-sampled-layer-chain
    dest: trace-fri-sampled-imp-full-cover-or-missing
    split: option.splits prod.splits)
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-bad-with-full-cover-layer-chain s bad))
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event

```

```

      (λs. trace-fri-sampled-missing-full-cover s bad))
    adversary-initial-state
  by (rule wp-event-union-bound)
also have ... ≤
  (F +
    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)) +
  (P + I + W + D)
  by (rule add-mono[OF full-bound missing-bound])
finally show ?thesis .
qed

```

lemma *checked-staged-security-trace-fri-sampled-layer-chain-bound-from-full-cover-fresh-and-witness-domain:*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and envelope:
  ∧ data attacker-state trace-table.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      trace-fri-multiround-bad-sets bad trace-table ⊆ B
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-challenge-list-fresh-hit s B))
    adversary-initial-state ≤ F
and proximity:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (trace-fri-sampled-full-cover-proximity-failure
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)) bad)
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤ P
and index:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (trace-fri-sampled-full-cover-index-failure
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state

```

$(\text{staged-proof-transcript data}) \leq I$
and *witness-length*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-witness-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq W$
and *domain*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq D$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-sampled-layer-chain})$
 $\text{adversary-initial-state}$
 $\leq$
 $(F +$
 $\text{hash-relation-budget-value (card } B * \text{ceil-log clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + W + D)$
proof –
have *full-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad}))$
 $\text{adversary-initial-state}$
 $\leq$
 $F +$
 $\text{hash-relation-budget-value (card } B * \text{ceil-log clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
by (rule
 $\text{checked-staged-security-trace-fri-full-cover-layer-chain-bound-from-fresh-and-prequery}$
 $[\text{OF wf controlled finite-}B \text{ envelope fresh-bound}])$
have *missing-bound*:

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. trace-fri-sampled-missing-full-cover s bad))
adversary-initial-state ≤ P + I + W + D
by (rule
  checked-staged-security-trace-fri-sampled-missing-full-cover-bound-from-witness-domain
  [OF proximity index witness-length domain])
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-sampled-layer-chain)
adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (λout.
    staged-security-with-data-state-verifier-event
      (λs. trace-fri-bad-with-full-cover-layer-chain s bad) out ∨
    staged-security-with-data-state-verifier-event
      (λs. trace-fri-sampled-missing-full-cover s bad) out)
adversary-initial-state
by (rule wp-event-mono-on-support)
(auto simp: staged-security-with-data-state-verifier-event-def
  dest: trace-fri-sampled-imp-full-cover-or-missing
  split: option.splits prod.splits)
also have ... ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. trace-fri-bad-with-full-cover-layer-chain s bad))
adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. trace-fri-sampled-missing-full-cover s bad))
adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
(F +
  hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)) +
(P + I + W + D)
by (rule add-mono[OF full-bound missing-bound])
finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-full-cover-fresh-and-witn
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: ∧ dg. finite (B dg)
and envelope:
  ∧ data attacker-state dg composition-table.
  Some (data, attacker-state) ∈

```

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \\ \text{adversary-initial-state}) \implies$
 $\text{composition-fri-multiround-bad-sets bad dg composition-table}$
 $\subseteq B \text{ dg}$
and *fresh-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ (\text{staged-security-with-data-state-verifier-event} \\ (\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B)) \\ \text{adversary-initial-state} \leq F$
and *proximity*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \\ \text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure} \\ (\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})) \text{ bad}) \\ (\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})) \leq P$
and *index*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \\ \text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure} \\ (\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data}))) \\ (\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})) \leq I$
and *witness-length*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \\ \text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-witness-length-failure} \\ (\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data}))) \\ (\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})) \leq W$
and *domain*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \\ \text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-failure}$

$$\begin{aligned}
& (\text{verifier-state-from-adversary attacker-state} \\
& \quad (\text{staged-proof-transcript data})) \\
& (\text{verifier-state-from-adversary attacker-state} \\
& \quad (\text{staged-proof-transcript data})) \leq D
\end{aligned}$$

shows

$$\begin{aligned}
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\text{staged-security-with-data-state-verifier-event} \\
& \quad \quad \text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}) \\
& \text{adversary-initial-state} \\
& \leq \\
& (F + \\
& \quad \text{hash-relation-budget-value} \\
& \quad (\sum dg \in (\text{UNIV} :: 'f \text{ set}). \\
& \quad \quad \text{card } (B \text{ dg}) * \text{ceil-log } (\text{maxDegree} + 1)) \\
& \quad (\text{staged-attacker-query-budget budgets} + \\
& \quad \quad \text{staged-challenge-query-budget})) + \\
& (P + I + W + D)
\end{aligned}$$

proof –

have *full-bound*:

$$\begin{aligned}
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\text{staged-security-with-data-state-verifier-event} \\
& \quad \quad (\lambda s. \text{composition-fri-bad-with-full-cover-layer-chain } s \text{ bad})) \\
& \text{adversary-initial-state} \\
& \leq \\
& F + \\
& \quad \text{hash-relation-budget-value} \\
& \quad (\sum dg \in (\text{UNIV} :: 'f \text{ set}). \\
& \quad \quad \text{card } (B \text{ dg}) * \text{ceil-log } (\text{maxDegree} + 1)) \\
& \quad (\text{staged-attacker-query-budget budgets} + \\
& \quad \quad \text{staged-challenge-query-budget})
\end{aligned}$$

by (rule

$$\begin{aligned}
& \text{checked-staged-security-composition-fri-full-cover-layer-chain-bound-from-fresh-and-prequery} \\
& \quad [\text{OF wf controlled finite-B envelope fresh-bound}]
\end{aligned}$$

have *missing-bound*:

$$\begin{aligned}
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\text{staged-security-with-data-state-verifier-event} \\
& \quad \quad (\lambda s. \text{composition-fri-sampled-missing-full-cover } s \text{ bad})) \\
& \text{adversary-initial-state} \leq P + I + W + D
\end{aligned}$$

by (rule

$$\begin{aligned}
& \text{checked-staged-security-composition-fri-sampled-missing-full-cover-bound-from-witness-domain} \\
& \quad [\text{OF proximity index witness-length domain}]
\end{aligned}$$

have *wp-event* (checked-staged-security-experiment-with-data-state A)

$$\begin{aligned}
& \quad (\text{staged-security-with-data-state-verifier-event} \\
& \quad \quad \text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}) \\
& \text{adversary-initial-state} \leq \\
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\lambda \text{out.} \\
& \quad \quad \text{staged-security-with-data-state-verifier-event} \\
& \quad \quad (\lambda s. \text{composition-fri-bad-with-full-cover-layer-chain } s \text{ bad}))
\end{aligned}$$


```

      out  $\vee$ 
      staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-sampled-missing-full-cover  $s$  bad)
      out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    dest!: composition-fri-verifier-tied-sampled-layer-chain-imp-sampled-layer-chain
    dest: composition-fri-sampled-imp-full-cover-or-missing
    split: option.splits prod.splits)
  also have ...  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    (staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-bad-with-full-cover-layer-chain  $s$  bad))
    adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    (staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-sampled-missing-full-cover  $s$  bad))
    adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq$ 
    ( $F$  +
      hash-relation-budget-value
      ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
        card ( $B \ dg$ ) * ceil-log (maxDegree + 1))
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
    ( $P + I + W + D$ )
  by (rule add-mono[OF full-bound missing-bound])
  finally show ?thesis .
qed

end

end

```

theory *Soundness-FRI-Derived-Active-Reductions*

imports

Soundness-FRI-Staged-Replay-Bounds

Soundness-FRI-Full-Cover-Staged-Bounds

begin

Derived active-FRI packaging.

This layer keeps the near-threshold staged replay theory stable. It packages the already proved trace and composition verifier-state reductions into the active-FRI interface consumed by the public route. It does not add protocol assumptions or change the FRI model.

context *soundness*

begin

lemma *trace-fri-header-tied-reduction-from-envelope-witness-domain-without-head-slot-and-selected-zero:*

assumes *future: trace-fri-future-fresh s*

and *envelope:*

$\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$

and *subset: $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$*

and *challenge-bound:*

$\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log clength}) \leq R$

and *proximity:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$

and *index:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-index-failure } s) \ s \leq I$

and *witness-length:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-witness-length-failure } s) \ s \leq W$

and *domain:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-domain-failure } s) \ s \leq D$

and *zero:*

wp-event verify-monad

$(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq z-b$

and *merkle:*

wp-event verify-monad $(\text{partial-merkle-inconsistency-bad } s) \ s \leq p-b$

and *structural:*

wp-event verify-monad

$(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq h-b$

and *next-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq n-b$

and *successor:*

wp-event verify-monad

$(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq s-b$

and *total:*

$(R + (P + I + W + D)) +$
 $(((((p-b + h-b) + p-b) + n-b + s-b +$
 $(p-b + 0 + 0)) + p-b) + 0 + z-b)$
 $\leq \text{trace-fri-error}$

shows *trace-fri-header-tied-reduction s*

by *(rule trace-fri-header-tied-reduction-from-residual-obligations)*

(rule

trace-fri-header-tied-residual-obligations-from-envelope-witness-domain-without-head-slot-and-selected-zero

[OF future envelope subset challenge-bound proximity index

witness-length domain zero merkle structural next-bound successor

total])

lemma *composition-fri-verifier-tied-reduction-from-envelope-witness-domain-slot-zero:*
assumes *future: composition-fri-future-fresh s*
and *envelope:*
 $\bigwedge dg$ *composition-table.*
 $\text{composition-fri-multi-round-bad-sets bad } dg \text{ composition-table}$
 $\subseteq B \ dg$
and *subset:*
 $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$
and *challenge-bound:*
 $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) /$
 $\text{nnreal } (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq R$
and *proximity:*
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s$
 $\leq P$
and *index:*
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and *witness-length:*
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-witness-length-failure } s) \ s$
 $\leq W$
and *domain:*
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-failure } s) \ s \leq D$
and *merkle:*
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq p\text{-}b$
and *next-bound:*
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap } s) \ s \leq n\text{-}b$
and *successor:*
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap } s) \ s \leq s\text{-}b$
and *final-bound:*
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq g\text{-}b$
and *total:*
 $(R + (P + I + W + D)) +$
 $((p\text{-}b + 0) + n\text{-}b + s\text{-}b + g\text{-}b + p\text{-}b) + 0)$
 $\leq \text{composition-fri-error}$
shows *composition-fri-verifier-tied-reduction s*
by (*rule composition-fri-verifier-tied-reduction-from-residual-obligations*)
(*rule*
 $\text{composition-fri-verifier-tied-residual-obligations-from-envelope-witness-domain-slot-zero}$
 $[OF \ \text{future} \ \text{envelope} \ \text{subset} \ \text{challenge-bound} \ \text{proximity} \ \text{index}$
 $\text{witness-length} \ \text{domain} \ \text{merkle} \ \text{next-bound} \ \text{successor} \ \text{final-bound} \ \text{total}]$)

lemma *composition-fri-verifier-tied-reduction-from-envelope-witness-domain-merkle:*
assumes *future: composition-fri-future-fresh s*

and *envelope*:
 $\bigwedge dg \text{ composition-table.}$
 $\text{composition-fri-multiround-bad-sets bad dg composition-table}$
 $\subseteq B dg$
and *subset*:
 $\bigwedge dg. B dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$
and *challenge-bound*:
 $\bigwedge dg. \text{nnreal } (\text{card } (B dg)) /$
 $\text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq R$
and *proximity*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad}) s$
 $\leq P$
and *index*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure } s) s \leq I$
and *witness-length*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-witness-length-failure } s) s$
 $\leq W$
and *domain*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-failure } s) s \leq D$
and *merkle*:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) s \leq M$
and *total*:
 $(R + (P + I + W + D)) +$
 $((M + 0) + (M + 0) + (M + 0) + (M + 0 + 0) + M) + 0)$
 $\leq \text{composition-fri-error}$
shows *composition-fri-verifier-tied-reduction* s
proof –
have *slot*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap } s)$
 $s \leq 0$
by (rule
 $\text{wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero})$
have *next-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap } s) s \leq M + 0$
by (rule
 $\text{wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-merkle-and-slot})$
 $(\text{rule merkle, rule slot})$
have *successor*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap } s) s \leq M + 0$
by (rule
 $\text{wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-merkle-and-slot})$
 $(\text{rule merkle, rule slot})$

have *final-bound*:
 wp-event verify-monad
 (composition-fri-verifier-tied-final-value-replay-gap *s*) *s*
 $\leq M + 0 + 0$
by (rule
 wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
 (rule merkle, rule slot,
 rule
 wp-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero)
show ?thesis
by (rule
 composition-fri-verifier-tied-reduction-from-envelope-witness-domain-slot-zero
 [OF future envelope subset challenge-bound proximity index
 witness-length domain merkle next-bound successor final-bound total])
qed

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-full-cover-domain-conflict-and-zero*

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *finite-B*: finite *B*
and *envelope*:
 $\bigwedge \text{data attacker-state trace-table.}$
 Some (*data*, *attacker-state*) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\implies$
 trace-fri-multiround-bad-sets bad trace-table $\subseteq B$
and *fresh-bound*:
 wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-verifier-event
 ($\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B$))
 adversary-initial-state $\leq F$
and *proximity*:
 $\bigwedge \text{data attacker-state.}$
 Some (*data*, *attacker-state*) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-sampled-full-cover-proximity-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript *data*)) bad)
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript *data*)) $\leq P$
and *index*:
 $\bigwedge \text{data attacker-state.}$
 Some (*data*, *attacker-state*) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-sampled-full-cover-index-failure

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$
and *domain-length*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq DL$
and *sampling-domain*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-sampled-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq SD$
and *conflict-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq C$
and *zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq Z$

shows

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-partial-candidate)
 $adversary\text{-}initial\text{-}state \leq$
 (($F +$
hash-relation-budget-value (*card B * ceil-log clength*)
 (*staged-attacker-query-budget budgets +*
staged-challenge-query-budget)) +
 ($P + I + (I + DL) + (I + SD)$)) + ($C + Z$)

proof (*rule*
checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-conflict-and-staged-zero)
show $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-sampled-layer-chain)
 $adversary\text{-}initial\text{-}state$
 $\leq$
 ($F +$
hash-relation-budget-value (*card B * ceil-log clength*)
 (*staged-attacker-query-budget budgets +*
staged-challenge-query-budget)) +
 ($P + I + (I + DL) + (I + SD)$)

by (*rule*
checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-full-cover-fresh-and-domain-split
 [*OF wf controlled finite-B envelope fresh-bound proximity index*
domain-length sampled-domain])

show $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-header-tied-sampled-assignment-conflict)
 $adversary\text{-}initial\text{-}state \leq C$

by (*rule checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
 (*auto simp: trace-fri-header-tied-sampled-assignment-conflict-def*
accepted-fri-opening-transcript-def conflict-bound)

show $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-header-tied-zero-round-final-obstruction)
 $adversary\text{-}initial\text{-}state \leq Z$

by (*rule checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
 (*auto simp: trace-fri-header-tied-zero-round-final-obstruction-def*
accepted-fri-opening-transcript-def zero-bound)

qed

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-full-cover-witness-domain-conflict-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: finite B*
and *envelope:*
 $\bigwedge data\ attacker\text{-}state\ trace\text{-}table.$
Some (*data, attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program A*)

$\text{adversary-initial-state}) \implies$
 $\text{trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *fresh-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B))$
 $\text{adversary-initial-state} \leq F$
and *proximity*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-proximity-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq P$
and *index*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$
and *witness-length*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-witness-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq W$
and *domain*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$

$(\text{verifier-state-from-adversary attacker-state} \text{ (staged-proof-transcript data)}) \leq D$
and *conflict-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq C$
and *zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq Z$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq$
 $((F +$
 $\text{hash-relation-budget-value (card B * ceil-log clength)}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + W + D)) + (C + Z)$
proof (rule
 $\text{checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-conflict-and-staged-zero})$
show $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-sampled-layer-chain})$
 $\text{adversary-initial-state}$
 $\leq$
 $(F +$
 $\text{hash-relation-budget-value (card B * ceil-log clength)}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + W + D)$
by (rule
 $\text{checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-full-cover-fresh-and-witness-domain}$
 $[OF \text{ wf controlled finite-B envelope fresh-bound proximity index}$

$witness-length\ domain]$
show $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $\quad trace-fri-header-tied-sampled-assignment-conflict)$
 $adversary-initial-state \leq C$
by $(rule\ checked-staged-security-with-data-state-verifier-event-bound-from-cont)$
 $(auto\ simp:\ trace-fri-header-tied-sampled-assignment-conflict-def$
 $\quad accepted-fri-opening-transcript-def\ conflict-bound)$
show $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $\quad trace-fri-header-tied-zero-round-final-obstruction)$
 $adversary-initial-state \leq Z$
by $(rule\ checked-staged-security-with-data-state-verifier-event-bound-from-cont)$
 $(auto\ simp:\ trace-fri-header-tied-zero-round-final-obstruction-def$
 $\quad accepted-fri-opening-transcript-def\ zero-bound)$
qed

lemma $checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-full-cover-domain-conflict-a$
assumes $wf:\ staged-budget-wellformed\ budgets$
and $controlled:\ staged-adversary-controlled\ budgets\ A$
and $finite-B:\ \bigwedge dg.\ finite\ (B\ dg)$
and $envelope:$
 $\bigwedge data\ attacker-state\ dg\ composition-table.$
 $\quad Some\ (data,\ attacker-state) \in$
 $\quad\quad set-dist\ (execute\ (checked-staged-transcript-program\ A)$
 $\quad\quad\quad adversary-initial-state) \implies$
 $\quad composition-fri-multiround-bad-sets\ bad\ dg\ composition-table$
 $\quad \subseteq B\ dg$
and $fresh-bound:$
 $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $\quad (\lambda s.\ composition-fri-challenge-list-fresh-hit\ s\ B))$
 $adversary-initial-state \leq F$
and $proximity:$
 $\bigwedge data\ attacker-state.$
 $\quad Some\ (data,\ attacker-state) \in$
 $\quad\quad set-dist\ (execute\ (checked-staged-transcript-program\ A)$
 $\quad\quad\quad adversary-initial-state) \implies$
 $wp-event\ verify-monad$
 $\quad (composition-fri-sampled-full-cover-proximity-failure$
 $\quad\quad (verifier-state-from-adversary\ attacker-state$
 $\quad\quad\quad (staged-proof-transcript\ data))\ bad)$
 $\quad (verifier-state-from-adversary\ attacker-state$
 $\quad\quad (staged-proof-transcript\ data)) \leq P$
and $index:$
 $\bigwedge data\ attacker-state.$
 $\quad Some\ (data,\ attacker-state) \in$
 $\quad\quad set-dist\ (execute\ (checked-staged-transcript-program\ A)$
 $\quad\quad\quad adversary-initial-state) \implies$

wp-event verify-monad
 (composition-fri-sampled-full-cover-index-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq I$
and domain-length:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
wp-event verify-monad
 (composition-fri-sampled-full-cover-domain-length-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq DL$
and sampled-domain:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
wp-event verify-monad
 (composition-fri-sampled-full-cover-sampled-domain-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq SD$
and conflict-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
wp-event verify-monad
 (composition-fri-verifier-tied-sampled-assignment-conflict
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq C$
and zero-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
wp-event verify-monad
 (composition-fri-verifier-tied-zero-round-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state

$(\text{staged-proof-transcript data}) \leq Z$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-verifier-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq$
 $((F +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card } (B \text{ dg}) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + (I + DL) + (I + SD))) + (C + Z)$
proof (rule
 $\text{checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-sampled-and-missing})$
show $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain})$
 $\text{adversary-initial-state} \leq$
 $(F +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card } (B \text{ dg}) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + (I + DL) + (I + SD))$
by (rule
 $\text{checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-full-cover-fresh-and-domain}$
 $[\text{OF wf controlled finite-B envelope fresh-bound proximity index}$
 $\text{domain-length sampled-domain}])$
fix $\text{data attacker-state}$
assume support:
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$
show $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-missing-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $\leq C + Z$
by (rule
 $\text{wp-composition-fri-verifier-tied-missing-sampled-bound-from-assignment-obstruction})$
(rule
 $\text{wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero}$
 $[\text{OF conflict-bound}[\text{OF support}] \text{ zero-bound}[\text{OF support}]])$
qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-full-cover-witness-domain-c*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *finite-B: $\bigwedge dg. \text{finite } (B \ dg)$*

and *envelope:*

$\bigwedge \text{data attacker-state } dg \text{ composition-table.}$

$\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{composition-fri-multiround-bad-sets bad } dg \text{ composition-table}$
 $\subseteq B \ dg$

and *fresh-bound:*

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B))$
 $\text{adversary-initial-state} \leq F$

and *proximity:*

$\bigwedge \text{data attacker-state.}$

$\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq P$

and *index:*

$\bigwedge \text{data attacker-state.}$

$\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$

and *witness-length:*

$\bigwedge \text{data attacker-state.}$

$\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-witness-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq W$

and domain:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-sampled-full-cover-domain-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq D$
and conflict-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-sampled-assignment-conflict
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq C$
and zero-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-zero-round-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq Z$
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state $\leq$
 ((F +
 hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \text{ set}).$
 card (B dg) * ceil-log (maxDegree + 1))
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) +
 (P + I + W + D)) + (C + Z)
proof (rule
 checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-sampled-and-missing)
show wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event

```

    composition-fri-bad-with-verifier-tied-sampled-layer-chain)
  adversary-initial-state ≤
  (F +
    hash-relation-budget-value
    (∑ dg ∈ (UNIV :: 'f set).
      card (B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)) +
  (P + I + W + D)
by (rule
  checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-full-cover-fresh-and-witness
  [OF wf controlled finite-B envelope fresh-bound proximity index
    witness-length domain])
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (composition-fri-verifier-tied-missing-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  ≤ C + Z
by (rule
  wp-composition-fri-verifier-tied-missing-sampled-bound-from-assignment-obstruction)
(rule
  wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero
  [OF conflict-bound[OF support] zero-bound[OF support]])
qed

lemma checked-staged-security-trace-fri-empty-header-bound-from-full-cover-domain-conflict-and-zero:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B: finite B
  and envelope:
    ∧ data attacker-state trace-table.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      trace-fri-multi-round-bad-sets bad trace-table ⊆ B
  and fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-challenge-list-fresh-hit s B))
      adversary-initial-state ≤ F
  and proximity:
    ∧ data attacker-state.

```

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-full-cover-proximity-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) *bad*)
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq P$

and *index*:

$\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-full-cover-index-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq I$

and *domain-length*:

$\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-full-cover-domain-length-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq DL$

and *sampled-domain*:

$\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-full-cover-sampled-domain-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq SD$

and *conflict-bound*:

$\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-assignment-conflict*


```

    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ C
and zero-bound:
 $\wedge$  data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-zero-round-final-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ Z
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state ≤
  ((F +
    hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
    (P + I + (I + DL) + (I + SD))) + (C + Z)
proof (rule checked-staged-security-trace-fri-empty-header-bound-from-sampled-and-missing)
show wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-sampled-layer-chain)
  adversary-initial-state
  ≤
  (F +
    hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
  (P + I + (I + DL) + (I + SD))
by (rule
  checked-staged-security-trace-fri-sampled-layer-chain-bound-from-full-cover-fresh-and-domain-split
    [OF wf controlled finite-B envelope fresh-bound proximity index
      domain-length sampled-domain])
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have obstruction:

```

$wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}layer\text{-}assignment\text{-}obstruction\ ?s)\ ?s \leq C + Z$
proof –
have $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}layer\text{-}assignment\text{-}obstruction\ ?s)\ ?s \leq$
 $wp\text{-}event\ verify\text{-}monad$
 $(\lambda out. trace\text{-}fri\text{-}sampled\text{-}assignment\text{-}conflict\ ?s\ out \vee$
 $trace\text{-}fri\text{-}zero\text{-}round\text{-}final\text{-}obstruction\ ?s\ out)\ ?s$
by $(rule\ wp\text{-}event\text{-}mono)$
 $(rule$
 $trace\text{-}fri\text{-}sampled\text{-}layer\text{-}assignment\text{-}obstruction\text{-}imp\text{-}conflict\text{-}or\text{-}zero\text{-}round\text{-}final)$
also have $\dots \leq$
 $wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}sampled\text{-}assignment\text{-}conflict\ ?s)\ ?s +$
 $wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}zero\text{-}round\text{-}final\text{-}obstruction\ ?s)\ ?s$
by $(rule\ wp\text{-}event\text{-}union\text{-}bound)$
also have $\dots \leq C + Z$
by $(rule\ add\text{-}mono[OF\ conflict\text{-}bound[OF\ support]$
 $zero\text{-}bound[OF\ support]])$
finally show $?thesis$.
qed
show $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}reachable\text{-}missing\text{-}sampled\text{-}layer\text{-}chain$
 $?s)\ ?s \leq C + Z$
by $(rule\ wp\text{-}trace\text{-}fri\text{-}reachable\text{-}missing\text{-}sampled\text{-}bound\text{-}from\text{-}assignment\text{-}obstruction)$
 $(rule\ obstruction)$
qed

lemma *checked-staged-security-trace-fri-empty-header-bound-from-full-cover-witness-domain-conflict-and-zero:*
assumes $wf: staged\text{-}budget\text{-}wellformed\ budgets$
and $controlled: staged\text{-}adversary\text{-}controlled\ budgets\ A$
and $finite\text{-}B: finite\ B$
and $envelope:$
 $\bigwedge data\ attacker\text{-}state\ trace\text{-}table.$
 $Some\ (data, attacker\text{-}state) \in$
 $set\text{-}dist\ (execute\ (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $trace\text{-}fri\text{-}multi\text{-}round\text{-}bad\text{-}sets\ bad\ trace\text{-}table \subseteq B$
and $fresh\text{-}bound:$
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $(\lambda s. trace\text{-}fri\text{-}challenge\text{-}list\text{-}fresh\text{-}hit\ s\ B))$
 $adversary\text{-}initial\text{-}state \leq F$
and $proximity:$
 $\bigwedge data\ attacker\text{-}state.$
 $Some\ (data, attacker\text{-}state) \in$
 $set\text{-}dist\ (execute\ (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}proximity\text{-}failure$

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq P$
and *index*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$
and *witness-length*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-witness-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq W$
and *domain*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq D$
and *conflict-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq C$
and *zero-bound*:

$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-zero-round-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Z
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$
 ((F +
 hash-relation-budget-value (card B * ceil-log clength)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) +
 (P + I + W + D)) + (C + Z)
proof (rule checked-staged-security-trace-fri-empty-header-bound-from-sampled-and-missing)
show wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-sampled-layer-chain)
 adversary-initial-state
 $\leq$
 (F +
 hash-relation-budget-value (card B * ceil-log clength)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) +
 (P + I + W + D)
by (rule
 checked-staged-security-trace-fri-sampled-layer-chain-bound-from-full-cover-fresh-and-witness-domain
 [OF wf controlled finite-B envelope fresh-bound proximity index
 witness-length domain])
fix data attacker-state
assume support:
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state)
let ?s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
have obstruction:
 wp-event verify-monad
 (trace-fri-sampled-layer-assignment-obstruction ?s) ?s $\leq$ C + Z
proof –
have wp-event verify-monad
 (trace-fri-sampled-layer-assignment-obstruction ?s) ?s $\leq$

```

wp-event verify-monad
  (λout. trace-fri-sampled-assignment-conflict ?s out ∨
    trace-fri-zero-round-final-obstruction ?s out) ?s
by (rule wp-event-mono)
  (rule
    trace-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final)
also have ... ≤
  wp-event verify-monad (trace-fri-sampled-assignment-conflict ?s) ?s +
  wp-event verify-monad (trace-fri-zero-round-final-obstruction ?s) ?s
by (rule wp-event-union-bound)
also have ... ≤ C + Z
by (rule add-mono[OF conflict-bound[OF support]
  zero-bound[OF support]])
finally show ?thesis .
qed
show wp-event verify-monad
  (trace-fri-reachable-missing-sampled-layer-chain
    ?s) ?s ≤ C + Z
by (rule wp-trace-fri-reachable-missing-sampled-bound-from-assignment-obstruction)
  (rule obstruction)
qed

lemma active-fri-reductions-for-from-component-reductions:
  assumes trace-header:
    ∧data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
        trace-fri-header-tied-reduction
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))
  and composition:
    ∧data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
        composition-fri-verifier-tied-reduction
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))
  and empty-trace:
    ∧data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
        trace-fri-empty-header-reduction
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))
  shows active-fri-reductions-for A
  unfolding active-fri-reductions-for-def

```

by (intro allI impI conjI)
 (erule trace-header, erule composition, erule empty-trace)

lemma trace-fri-empty-header-reduction-from-sampled-and-missing-bounds:

fixes $R\ M :: \text{prob}$

assumes sampled-bound:

wp-event verify-monad

(trace-fri-bad-with-sampled-layer-chain s) $s \leq R$

and missing-bound:

wp-event verify-monad

(trace-fri-reachable-missing-sampled-layer-chain s) $s \leq M$

and total-bound:

$R + M \leq \text{trace-fri-error}$

shows trace-fri-empty-header-reduction s

proof –

have reachable:

wp-event verify-monad

(trace-fri-bad-with-reachable-partial-candidate s) $s \leq R + M$

by (rule wp-trace-fri-reachable-bound-from-sampled-and-missing
 [OF sampled-bound missing-bound])

have empty:

wp-event verify-monad

(trace-fri-bad-with-empty-composition-header-candidates s) $s \leq$
 $R + M$

by (rule order-trans[OF - reachable])

(rule wp-event-mono,

rule

trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate)

show ?thesis

unfolding trace-fri-empty-header-reduction-def

by (rule order-trans[OF empty total-bound])

qed

lemma trace-fri-empty-header-reduction-from-sampled-and-assignment-obstruction-bounds:

fixes $R\ M :: \text{prob}$

assumes sampled-bound:

wp-event verify-monad

(trace-fri-bad-with-sampled-layer-chain s) $s \leq R$

and obstruction-bound:

wp-event verify-monad

(trace-fri-sampled-layer-assignment-obstruction s) $s \leq M$

and total-bound:

$R + M \leq \text{trace-fri-error}$

shows trace-fri-empty-header-reduction s

proof (rule trace-fri-empty-header-reduction-from-sampled-and-missing-bounds)

show wp-event verify-monad

(trace-fri-bad-with-sampled-layer-chain s) $s \leq R$

by (rule sampled-bound)

show wp-event verify-monad

$(\text{trace-fri-reachable-missing-sampled-layer-chain } s) \ s \leq M$
by (rule
 $\text{wp-trace-fri-reachable-missing-sampled-bound-from-assignment-obstruction}$
 $[OF \text{ obstruction-bound}]$)
show $R + M \leq \text{trace-fri-error}$
by (rule *total-bound*)
qed

lemma *wp-trace-fri-sampled-layer-assignment-obstruction-bound-from-conflict-and-zero*:
assumes *conflict-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict } s) \ s \leq C$
and *zero-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction } s) \ s \leq Z$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-layer-assignment-obstruction } s) \ s \leq C + Z$
proof –
have $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-layer-assignment-obstruction } s) \ s \leq$
 $\text{wp-event verify-monad}$
 $(\lambda \text{out. trace-fri-sampled-assignment-conflict } s \ \text{out} \ \vee$
 $\text{trace-fri-zero-round-final-obstruction } s \ \text{out}) \ s$
by (rule *wp-event-mono*)
 $(\text{rule trace-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final})$
also have $\dots \leq$
 $\text{wp-event verify-monad } (\text{trace-fri-sampled-assignment-conflict } s) \ s +$
 $\text{wp-event verify-monad } (\text{trace-fri-zero-round-final-obstruction } s) \ s$
by (rule *wp-event-union-bound*)
also have $\dots \leq C + Z$
by (rule *add-mono*[*OF conflict-bound zero-bound*])
finally show *?thesis* .
qed

lemma *trace-fri-empty-header-reduction-from-sampled-conflict-and-zero-bounds*:
fixes $R \ C \ Z :: \text{prob}$
assumes *sampled-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-sampled-layer-chain } s) \ s \leq R$
and *conflict-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict } s) \ s \leq C$
and *zero-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction } s) \ s \leq Z$
and *total-bound*:
 $R + (C + Z) \leq \text{trace-fri-error}$
shows *trace-fri-empty-header-reduction* s

```

proof (rule
  trace-fri-empty-header-reduction-from-sampled-and-assignment-obstruction-bounds)
show wp-event verify-monad
  (trace-fri-bad-with-sampled-layer-chain s)  $s \leq R$ 
by (rule sampled-bound)
show wp-event verify-monad
  (trace-fri-sampled-layer-assignment-obstruction s)  $s \leq C + Z$ 
by (rule
  wp-trace-fri-sampled-layer-assignment-obstruction-bound-from-conflict-and-zero
  [OF conflict-bound zero-bound])
show  $R + (C + Z) \leq \text{trace-fri-error}$ 
by (rule total-bound)
qed

lemma trace-fri-header-tied-reduction-from-augmented-conflict-zero-merkle-bounds:
fixes A C Z M :: prob
assumes augmented-bound:
  wp-event verify-monad
  (trace-fri-bad-with-header-tied-augmented-partial-candidate s)  $s \leq A$ 
and conflict-bound:
  wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict s)  $s \leq C$ 
and zero-bound:
  wp-event verify-monad
  (trace-fri-header-tied-zero-round-final-obstruction s)  $s \leq Z$ 
and merkle-bound:
  wp-event verify-monad (partial-merkle-inconsistency-bad s)  $s \leq M$ 
and total-bound:
   $A + ((C + Z) + M) \leq \text{trace-fri-error}$ 
shows trace-fri-header-tied-reduction s
proof –
have missing-bound:
  wp-event verify-monad
  (trace-fri-header-tied-recorded-sibling-candidate-missing s)  $s \leq C + Z$ 
by (rule
  wp-trace-fri-header-tied-recorded-sibling-candidate-missing-bound-from-conflict-and-zero)
  (rule conflict-bound, rule zero-bound)
show ?thesis
by (rule
  trace-fri-header-tied-reduction-from-augmented-and-recorded-missing-bounds
  [OF augmented-bound missing-bound merkle-bound total-bound])
qed

lemma active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty:
fixes TraceAug TraceMissing TraceMerkle CompSampled CompMerkle ::
  'f staged-proof-data  $\Rightarrow$  'f protocol-channel  $\Rightarrow$  prob
assumes trace-augmented-bound:

```


$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-augmented-partial-candidate
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ TraceAug data attacker-state
and trace-missing-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-header-tied-recorded-sibling-candidate-missing
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 TraceMissing data attacker-state
and trace-merkle-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ TraceMerkle data attacker-state
and trace-total:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 TraceAug data attacker-state +
 (TraceMissing data attacker-state + TraceMerkle data attacker-state)
 $\leq$ trace-fri-error
and composition-sampled-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-sampled-layer-chain
 (verifier-state-from-adversary attacker-state

```

      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
    CompSampled data attacker-state
and composition-merkle-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (partial-merkle-inconsistency-bad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)) ≤ CompMerkle data attacker-state
and composition-total:
 $\wedge$ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    CompSampled data attacker-state +
      (((CompMerkle data attacker-state + 0) +
        (CompMerkle data attacker-state + 0) +
        (CompMerkle data attacker-state + 0) +
        (CompMerkle data attacker-state + 0 + 0) +
        CompMerkle data attacker-state) + 0)
    ≤ composition-fri-error
and empty-trace:
 $\wedge$ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    trace-fri-empty-header-reduction
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
shows active-fri-reductions-for A
proof (rule active-fri-reductions-for-from-component-reductions)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show trace-fri-header-tied-reduction
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
proof –
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)

```

```

have aug:
  wp-event verify-monad
    (trace-fri-bad-with-header-tied-augmented-partial-candidate ?s)
    ?s ≤ TraceAug data attacker-state
  by (rule trace-augmented-bound[OF support])
have missing:
  wp-event verify-monad
    (trace-fri-header-tied-recorded-sibling-candidate-missing ?s)
    ?s ≤ TraceMissing data attacker-state
  by (rule trace-missing-bound[OF support])
have merkle:
  wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
    ≤ TraceMerkle data attacker-state
  by (rule trace-merkle-bound[OF support])
have total:
  TraceAug data attacker-state +
    (TraceMissing data attacker-state + TraceMerkle data attacker-state)
    ≤ trace-fri-error
  by (rule trace-total[OF support])
show ?thesis
  by (rule
    trace-fri-header-tied-reduction-from-augmented-and-recorded-missing-bounds
    [OF aug missing merkle total])
qed
show composition-fri-verifier-tied-reduction
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  by (rule active-composition-fri-verifier-tied-reductions-from-sampled-and-merkle-bounds
    [OF composition-sampled-bound composition-merkle-bound
      composition-total support])
show trace-fri-empty-header-reduction
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  by (rule empty-trace[OF support])
qed

lemma active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty-sampled:
  fixes TraceAug TraceMissing TraceMerkle CompSampled CompMerkle EmptySampled EmptyMissing ::
    'f staged-proof-data ⇒ 'f protocol-channel ⇒ prob
  assumes trace-augmented-bound:
    ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
  wp-event verify-monad
    (trace-fri-bad-with-header-tied-augmented-partial-candidate
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))

```

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{TraceAug data attacker-state}$
and *trace-missing-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-recorded-sibling-candidate-missing}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{TraceMissing data attacker-state}$
and *trace-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{TraceMerkle data attacker-state}$
and *trace-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{TraceAug data attacker-state} +$
 $(\text{TraceMissing data attacker-state} + \text{TraceMerkle data attacker-state})$
 $\leq \text{trace-fri-error}$
and *composition-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{CompSampled data attacker-state}$
and *composition-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{CompMerkle data attacker-state}$
and $\text{composition-total:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{CompSampled data attacker-state} +$
 $((\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0 + 0) +$
 $\text{CompMerkle data attacker-state}) + 0)$
 $\leq \text{composition-fri-error}$
and $\text{empty-sampled-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptySampled data attacker-state}$
and $\text{empty-missing-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-reachable-missing-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyMissing data attacker-state}$
and empty-total:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{EmptySampled data attacker-state} + \text{EmptyMissing data attacker-state}$

$\leq \text{trace-fri-error}$
shows *active-fri-reductions-for A*
proof (*rule active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty*)
fix *data attacker-state*
assume *support*:
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
show *wp-event verify-monad*
(trace-fri-bad-with-header-tied-augmented-partial-candidate ?s) ?s
 $\leq \text{TraceAug data attacker-state}$
by (*rule trace-augmented-bound[OF support]*)
show *wp-event verify-monad*
(trace-fri-header-tied-recorded-sibling-candidate-missing ?s) ?s
 $\leq \text{TraceMissing data attacker-state}$
by (*rule trace-missing-bound[OF support]*)
show *wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s*
 $\leq \text{TraceMerkle data attacker-state}$
by (*rule trace-merkle-bound[OF support]*)
show *TraceAug data attacker-state +*
(TraceMissing data attacker-state + TraceMerkle data attacker-state)
 $\leq \text{trace-fri-error}$
by (*rule trace-total[OF support]*)
show *wp-event verify-monad*
(composition-fri-bad-with-verifier-tied-sampled-layer-chain ?s) ?s
 $\leq \text{CompSampled data attacker-state}$
by (*rule composition-sampled-bound[OF support]*)
show *wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s*
 $\leq \text{CompMerkle data attacker-state}$
by (*rule composition-merkle-bound[OF support]*)
show *CompSampled data attacker-state +*
((CompMerkle data attacker-state + 0) +
(CompMerkle data attacker-state + 0) +
(CompMerkle data attacker-state + 0) +
(CompMerkle data attacker-state + 0 + 0) +
CompMerkle data attacker-state) + 0)
 $\leq \text{composition-fri-error}$
by (*rule composition-total[OF support]*)
show *trace-fri-empty-header-reduction ?s*
by (*rule trace-fri-empty-header-reduction-from-sampled-and-missing-bounds*
[OF empty-sampled-bound[OF support]
empty-missing-bound[OF support] empty-total[OF support]])
qed

lemma *active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty-assignment*:
fixes *TraceAug TraceMissing TraceMerkle CompSampled CompMerkle EmptySampled Empty-*

Obstruction ::

'f staged-proof-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ prob

assumes *trace-augmented-bound:*

$\wedge$ *data attacker-state.*

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-augmented-partial-candidate
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ TraceAug data attacker-state*

and *trace-missing-bound:*

$\wedge$ *data attacker-state.*

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (trace-fri-header-tied-recorded-sibling-candidate-missing
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 TraceMissing data attacker-state*

and *trace-merkle-bound:*

$\wedge$ *data attacker-state.*

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ TraceMerkle data attacker-state*

and *trace-total:*

$\wedge$ *data attacker-state.*

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 TraceAug data attacker-state +
 (TraceMissing data attacker-state + TraceMerkle data attacker-state)
 $\leq$ trace-fri-error*

and *composition-sampled-bound:*

$\wedge$ *data attacker-state.*

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$*

$wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}tied\text{-}sampled\text{-}layer\text{-}chain$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)))$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)) \leq$
 $CompSampled\ data\ attacker\text{-}state$
and $composition\text{-}merkle\text{-}bound$:
 $\wedge data\ attacker\text{-}state.$
 $Some\ (data,\ attacker\text{-}state) \in$
 $set\text{-}dist\ (execute\ (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $wp\text{-}event\ verify\text{-}monad$
 $(partial\text{-}merkle\text{-}inconsistency\text{-}bad$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)))$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)) \leq CompMerkle\ data\ attacker\text{-}state$
and $composition\text{-}total$:
 $\wedge data\ attacker\text{-}state.$
 $Some\ (data,\ attacker\text{-}state) \in$
 $set\text{-}dist\ (execute\ (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $CompSampled\ data\ attacker\text{-}state +$
 $((CompMerkle\ data\ attacker\text{-}state + 0) +$
 $(CompMerkle\ data\ attacker\text{-}state + 0) +$
 $(CompMerkle\ data\ attacker\text{-}state + 0) +$
 $(CompMerkle\ data\ attacker\text{-}state + 0 + 0) +$
 $CompMerkle\ data\ attacker\text{-}state) + 0)$
 $\leq composition\text{-}fri\text{-}error$
and $empty\text{-}sampled\text{-}bound$:
 $\wedge data\ attacker\text{-}state.$
 $Some\ (data,\ attacker\text{-}state) \in$
 $set\text{-}dist\ (execute\ (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}bad\text{-}with\text{-}sampled\text{-}layer\text{-}chain$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)))$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)) \leq$
 $EmptySampled\ data\ attacker\text{-}state$
and $empty\text{-}obstruction\text{-}bound$:
 $\wedge data\ attacker\text{-}state.$
 $Some\ (data,\ attacker\text{-}state) \in$
 $set\text{-}dist\ (execute\ (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}layer\text{-}assignment\text{-}obstruction$


```

    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  EmptyObstruction data attacker-state
and empty-total:
 $\wedge$ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    EmptySampled data attacker-state + EmptyObstruction data attacker-state
    ≤ trace-fri-error
shows active-fri-reductions-for A
proof (rule active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show wp-event verify-monad
  (trace-fri-bad-with-header-tied-augmented-partial-candidate ?s) ?s
  ≤ TraceAug data attacker-state
by (rule trace-augmented-bound[OF support])
show wp-event verify-monad
  (trace-fri-header-tied-recorded-sibling-candidate-missing ?s) ?s
  ≤ TraceMissing data attacker-state
by (rule trace-missing-bound[OF support])
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
  ≤ TraceMerkle data attacker-state
by (rule trace-merkle-bound[OF support])
show TraceAug data attacker-state +
  (TraceMissing data attacker-state + TraceMerkle data attacker-state)
  ≤ trace-fri-error
by (rule trace-total[OF support])
show wp-event verify-monad
  (composition-fri-bad-with-verifier-tied-sampled-layer-chain ?s) ?s
  ≤ CompSampled data attacker-state
by (rule composition-sampled-bound[OF support])
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
  ≤ CompMerkle data attacker-state
by (rule composition-merkle-bound[OF support])
show CompSampled data attacker-state +
  (((CompMerkle data attacker-state + 0) +
    (CompMerkle data attacker-state + 0) +
    (CompMerkle data attacker-state + 0) +
    (CompMerkle data attacker-state + 0 + 0) +

```

```

      CompMerkle data attacker-state) + 0)
    ≤ composition-fri-error
  by (rule composition-total[OF support])
show trace-fri-empty-header-reduction ?s
  by (rule
      trace-fri-empty-header-reduction-from-sampled-and-assignment-obstruction-bounds
      [OF empty-sampled-bound[OF support]
       empty-obstruction-bound[OF support] empty-total[OF support]])
qed

```

lemma *active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty-conflict-zero:*

fixes *TraceAug TraceMissing TraceMerkle CompSampled CompMerkle EmptySampled EmptyConflict EmptyZero ::*

'f staged-proof-data ⇒ 'f protocol-channel ⇒ prob

assumes *trace-augmented-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-bad-with-header-tied-augmented-partial-candidate
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ TraceAug data attacker-state

and *trace-missing-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-recorded-sibling-candidate-missing
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
TraceMissing data attacker-state

and *trace-merkle-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ TraceMerkle data attacker-state

and *trace-total:*

$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 TraceAug data attacker-state +
 (TraceMissing data attacker-state + TraceMerkle data attacker-state)
 $\leq$ trace-fri-error
and composition-sampled-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-sampled-layer-chain
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 CompSampled data attacker-state
and composition-merkle-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ CompMerkle data attacker-state
and composition-total:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 CompSampled data attacker-state +
 (((CompMerkle data attacker-state + 0) +
 (CompMerkle data attacker-state + 0) +
 (CompMerkle data attacker-state + 0) +
 (CompMerkle data attacker-state + 0 + 0) +
 CompMerkle data attacker-state) + 0)
 $\leq$ composition-fri-error
and empty-sampled-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-bad-with-sampled-layer-chain

```

    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  EmptySampled data attacker-state
and empty-conflict-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  wp-event verify-monad
    (trace-fri-sampled-assignment-conflict
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
  EmptyConflict data attacker-state
and empty-zero-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  wp-event verify-monad
    (trace-fri-zero-round-final-obstruction
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
  EmptyZero data attacker-state
and empty-total:
 $\wedge$ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  EmptySampled data attacker-state +
    (EmptyConflict data attacker-state + EmptyZero data attacker-state)
  ≤ trace-fri-error
shows active-fri-reductions-for A
proof (rule active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty-assignment
  [where EmptyObstruction =
     $\lambda$ data attacker-state.
      EmptyConflict data attacker-state + EmptyZero data attacker-state])
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state

```

```

    (staged-proof-transcript data)
  show wp-event verify-monad
    (trace-fri-bad-with-header-tied-augmented-partial-candidate ?s) ?s
    ≤ TraceAug data attacker-state
  by (rule trace-augmented-bound[OF support])
  show wp-event verify-monad
    (trace-fri-header-tied-recorded-sibling-candidate-missing ?s) ?s
    ≤ TraceMissing data attacker-state
  by (rule trace-missing-bound[OF support])
  show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
    ≤ TraceMerkle data attacker-state
  by (rule trace-merkle-bound[OF support])
  show TraceAug data attacker-state +
    (TraceMissing data attacker-state + TraceMerkle data attacker-state)
    ≤ trace-fri-error
  by (rule trace-total[OF support])
  show wp-event verify-monad
    (composition-fri-bad-with-verifier-tied-sampled-layer-chain ?s) ?s
    ≤ CompSampled data attacker-state
  by (rule composition-sampled-bound[OF support])
  show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
    ≤ CompMerkle data attacker-state
  by (rule composition-merkle-bound[OF support])
  show CompSampled data attacker-state +
    (((CompMerkle data attacker-state + 0) +
      (CompMerkle data attacker-state + 0) +
      (CompMerkle data attacker-state + 0) +
      (CompMerkle data attacker-state + 0 + 0) +
      CompMerkle data attacker-state) + 0)
    ≤ composition-fri-error
  by (rule composition-total[OF support])
  show wp-event verify-monad
    (trace-fri-bad-with-sampled-layer-chain ?s) ?s
    ≤ EmptySampled data attacker-state
  by (rule empty-sampled-bound[OF support])
  show wp-event verify-monad
    (trace-fri-sampled-layer-assignment-obstruction ?s) ?s
    ≤ EmptyConflict data attacker-state + EmptyZero data attacker-state
  by (rule
    wp-trace-fri-sampled-layer-assignment-obstruction-bound-from-conflict-and-zero)
  (rule empty-conflict-bound[OF support],
   rule empty-zero-bound[OF support])
  show EmptySampled data attacker-state +
    (EmptyConflict data attacker-state + EmptyZero data attacker-state)
    ≤ trace-fri-error
  by (rule empty-total[OF support])
qed

```

lemma *active-fri-reductions-for-from-trace-augmented-conflict-zero-composition-sampled-and-empty-conflict-zero:*

```

fixes TraceAug TraceConflict TraceZero TraceMerkle CompSampled CompMerkle
      EmptySampled EmptyConflict EmptyZero ::
      'f staged-proof-data  $\Rightarrow$  'f protocol-channel  $\Rightarrow$  prob
assumes trace-augmented-bound:
 $\wedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\Rightarrow$ 
    wp-event verify-monad
      (trace-fri-bad-with-header-tied-augmented-partial-candidate
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq$  TraceAug data attacker-state
and trace-conflict-bound:
 $\wedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\Rightarrow$ 
    wp-event verify-monad
      (trace-fri-header-tied-sampled-assignment-conflict
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq$  TraceConflict data attacker-state
and trace-zero-bound:
 $\wedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\Rightarrow$ 
    wp-event verify-monad
      (trace-fri-header-tied-zero-round-final-obstruction
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq$  TraceZero data attacker-state
and trace-merkle-bound:
 $\wedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\Rightarrow$ 
    wp-event verify-monad
      (partial-merkle-inconsistency-bad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq$  TraceMerkle data attacker-state
and trace-total:
 $\wedge$  data attacker-state.

```

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
TraceAug *data* *attacker-state* +
((*TraceConflict* *data* *attacker-state* + *TraceZero* *data* *attacker-state*) +
TraceMerkle *data* *attacker-state*)
 $\leq$ *trace-fri-error*
and *composition-sampled-bound*:
 $\wedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event *verify-monad*
(*composition-fri-bad-with-verifier-tied-sampled-layer-chain*
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)))
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)) $\leq$
CompSampled *data* *attacker-state*
and *composition-merkle-bound*:
 $\wedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event *verify-monad*
(*partial-merkle-inconsistency-bad*
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)))
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)) $\leq$ *CompMerkle* *data* *attacker-state*
and *composition-total*:
 $\wedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
CompSampled *data* *attacker-state* +
(((*CompMerkle* *data* *attacker-state* + 0) +
(*CompMerkle* *data* *attacker-state* + 0) +
(*CompMerkle* *data* *attacker-state* + 0) +
(*CompMerkle* *data* *attacker-state* + 0 + 0) +
CompMerkle *data* *attacker-state*) + 0)
 $\leq$ *composition-fri-error*
and *empty-sampled-bound*:
 $\wedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event *verify-monad*
(*trace-fri-bad-with-sampled-layer-chain*

```

      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤
    EmptySampled data attacker-state
and empty-conflict-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
                    adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (trace-fri-sampled-assignment-conflict
       (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data)) ≤
      EmptyConflict data attacker-state
and empty-zero-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
                    adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (trace-fri-zero-round-final-obstruction
       (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data)) ≤
      EmptyZero data attacker-state
and empty-total:
 $\wedge$ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
                    adversary-initial-state)  $\implies$ 
    EmptySampled data attacker-state +
      (EmptyConflict data attacker-state + EmptyZero data attacker-state)
    ≤ trace-fri-error
shows active-fri-reductions-for A
proof (rule active-fri-reductions-for-from-component-reductions)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
                    adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show trace-fri-header-tied-reduction ?s
by (rule

```



```

    trace-fri-header-tied-reduction-from-augmented-conflict-zero-merkle-bounds)
  (rule trace-augmented-bound[OF support],
   rule trace-conflict-bound[OF support],
   rule trace-zero-bound[OF support],
   rule trace-merkle-bound[OF support],
   rule trace-total[OF support])
show composition-fri-verifier-tied-reduction ?s
by (rule active-composition-fri-verifier-tied-reductions-from-sampled-and-merkle-bounds
    [OF composition-sampled-bound composition-merkle-bound
     composition-total support])
show trace-fri-empty-header-reduction ?s
by (rule trace-fri-empty-header-reduction-from-sampled-conflict-and-zero-bounds)
    (rule empty-sampled-bound[OF support],
     rule empty-conflict-bound[OF support],
     rule empty-zero-bound[OF support],
     rule empty-total[OF support])
qed

lemma active-fri-reductions-for-from-trace-augmented-composition-canonical-and-empty-conflict-zero:
  fixes TraceAug TraceMissing TraceMerkle CompCanonical CompMerkle EmptyCanonical Emp-
  tyConflict EmptyZero ::
    'f staged-proof-data  $\Rightarrow$  'f protocol-channel  $\Rightarrow$  prob
  assumes trace-augmented-bound:
     $\bigwedge$  data attacker-state.
      Some (data, attacker-state)  $\in$ 
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state)  $\implies$ 
        wp-event verify-monad
          (trace-fri-bad-with-header-tied-augmented-partial-candidate
            (verifier-state-from-adversary attacker-state
              (staged-proof-transcript data)))
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))  $\leq$  TraceAug data attacker-state
  and trace-missing-bound:
     $\bigwedge$  data attacker-state.
      Some (data, attacker-state)  $\in$ 
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state)  $\implies$ 
        wp-event verify-monad
          (trace-fri-header-tied-recorded-sibling-candidate-missing
            (verifier-state-from-adversary attacker-state
              (staged-proof-transcript data)))
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))  $\leq$ 
          TraceMissing data attacker-state
  and trace-merkle-bound:
     $\bigwedge$  data attacker-state.
      Some (data, attacker-state)  $\in$ 
        set-dist (execute (checked-staged-transcript-program A)

```

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{TraceMerkle data attacker-state}$
and trace-total:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{TraceAug data attacker-state} +$
 $(\text{TraceMissing data attacker-state} + \text{TraceMerkle data attacker-state})$
 $\leq \text{trace-fri-error}$
and $\text{composition-canonical-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-canonical-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{CompCanonical data attacker-state}$
and $\text{composition-merkle-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{CompMerkle data attacker-state}$
and $\text{composition-total:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{CompCanonical data attacker-state} +$
 $((\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0 + 0) +$
 $\text{CompMerkle data attacker-state}) + 0)$

$\leq \text{composition-fri-error}$
and *empty-canonical-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-canonical-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyCanonical data attacker-state}$
and *empty-conflict-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyConflict data attacker-state}$
and *empty-zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyZero data attacker-state}$
and *empty-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{EmptyCanonical data attacker-state} +$
 $(\text{EmptyConflict data attacker-state} + \text{EmptyZero data attacker-state})$
 $\leq \text{trace-fri-error}$
shows *active-fri-reductions-for A*
proof (rule
 $\text{active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty-conflict-zero})$
fix *data attacker-state*

```

assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show wp-event verify-monad
  (trace-fri-bad-with-header-tied-augmented-partial-candidate ?s) ?s
  ≤ TraceAug data attacker-state
  by (rule trace-augmented-bound[OF support])
show wp-event verify-monad
  (trace-fri-header-tied-recorded-sibling-candidate-missing ?s) ?s
  ≤ TraceMissing data attacker-state
  by (rule trace-missing-bound[OF support])
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
  ≤ TraceMerkle data attacker-state
  by (rule trace-merkle-bound[OF support])
show TraceAug data attacker-state +
  (TraceMissing data attacker-state + TraceMerkle data attacker-state)
  ≤ trace-fri-error
  by (rule trace-total[OF support])
show wp-event verify-monad
  (composition-fri-bad-with-verifier-tied-sampled-layer-chain ?s) ?s
  ≤ CompCanonical data attacker-state
  by (rule
    wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-sampled)
    (rule wp-composition-fri-sampled-bound-from-canonical,
      rule composition-canonical-bound[OF support])
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
  ≤ CompMerkle data attacker-state
  by (rule composition-merkle-bound[OF support])
show CompCanonical data attacker-state +
  (((CompMerkle data attacker-state + 0) +
    (CompMerkle data attacker-state + 0) +
    (CompMerkle data attacker-state + 0) +
    (CompMerkle data attacker-state + 0 + 0) +
    CompMerkle data attacker-state) + 0)
  ≤ composition-fri-error
  by (rule composition-total[OF support])
show wp-event verify-monad
  (trace-fri-bad-with-sampled-layer-chain ?s) ?s
  ≤ EmptyCanonical data attacker-state
  by (rule wp-trace-fri-sampled-bound-from-canonical)
    (rule empty-canonical-bound[OF support])
show wp-event verify-monad
  (trace-fri-sampled-assignment-conflict ?s) ?s
  ≤ EmptyConflict data attacker-state
  by (rule empty-conflict-bound[OF support])

```

```

show wp-event verify-monad
  (trace-fri-zero-round-final-obstruction ?s) ?s
  ≤ EmptyZero data attacker-state
by (rule empty-zero-bound[OF support])
show EmptyCanonical data attacker-state +
  (EmptyConflict data attacker-state + EmptyZero data attacker-state)
  ≤ trace-fri-error
by (rule empty-total[OF support])
qed

end

end

```

```

theory Soundness-FRI-Sampled-Interface
imports
  Soundness-FRI-Derived-Active-Reductions
begin

```

Sampled-query FRI interface.

This theory names the FRI bad-candidate event that matches the verifier's sampled evidence: authenticated partial openings, sampled fold checks on the canonical domains, final consistency, and a non-low-degree candidate. It is deliberately separate from the full-cover compatibility route, where sampled openings are temporarily strengthened to full fold-index coverage.

```

context soundness
begin

```

```

definition fri-query-index-list-space :: nat list set
where

```

```

  fri-query-index-list-space =
    {query-idxs. length query-idxs = rounds ∧
      set query-idxs ⊆ query-sample-space}

```

```

definition trace-fri-query-index-list-set-hit
  :: ('f, 'a) protocol-channel-scheme ⇒ nat list set ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool

```

```

where
  trace-fri-query-index-list-set-hit s Q out ⇔
    (∃ trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final fri-query-idxs trace-round-layers
      composition-round-layers.
      accepted-fri-opening-transcript s out trace-roots trace-bs
        trace-final dg composition-roots composition-bs composition-final
        fri-query-idxs trace-round-layers composition-round-layers ∧
      fri-query-idxs ∈ Q)

```

definition *composition-fri-query-index-list-set-hit*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{nat list set} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-fri-query-index-list-set-hit } s \ Q \ \text{out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers.}$
 $\text{accepted-fri-opening-transcript } s \ \text{out} \ \text{trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{fri-query-idxs} \in Q)$

definition *query-index-list-entries* $:: \text{nat list set} \Rightarrow \text{nat set}$
where
 $\text{query-index-list-entries } Q = (\bigcup \text{query-idxs} \in Q. \text{set query-idxs})$

lemma *query-index-list-entries-subset-query-sample-space:*
assumes $Q \subseteq \text{fri-query-index-list-space}$
shows $\text{query-index-list-entries } Q \subseteq \text{query-sample-space}$
using *assms*
unfolding *query-index-list-entries-def fri-query-index-list-space-def*
by *auto*

lemma *finite-query-index-list-entries:*
assumes $Q \subseteq \text{fri-query-index-list-space}$
shows $\text{finite } (\text{query-index-list-entries } Q)$
using *query-index-list-entries-subset-query-sample-space[OF assms]*
by (*rule finite-subset*) (*rule finite-query-sample-space*)

lemma *query-index-list-entries-card-le:*
assumes *finite-Q: finite Q*
and *subset: $Q \subseteq \text{fri-query-index-list-space}$*
shows $\text{card } (\text{query-index-list-entries } Q) \leq \text{card } Q * \text{rounds}$
proof –
have
 $\text{card } (\text{query-index-list-entries } Q) \leq (\sum \text{query-idxs} \in Q.$
 $\text{card } (\text{set query-idxs}))$
unfolding *query-index-list-entries-def*
by (*rule card-UN-le*) (*rule finite-Q*)
also have $\dots \leq (\sum \text{query-idxs} \in Q. \text{rounds})$
proof (*rule sum-mono*)
fix *query-idxs*
assume $\text{query-idxs} \in Q$
then have $\text{length query-idxs} = \text{rounds}$
using *subset* **unfolding** *fri-query-index-list-space-def* **by** *auto*
then show $\text{card } (\text{set query-idxs}) \leq \text{rounds}$
using *card-length[of query-idxs]* **by** *simp*

```

qed
also have ... = card Q * rounds
  using finite-Q by simp
finally show ?thesis .
qed

definition generic-fri-sampled-query-candidate-evidence
:: ('f list  $\Rightarrow$  bool)  $\Rightarrow$  ('f list  $\Rightarrow$  bool)  $\Rightarrow$ 
  'f list  $\Rightarrow$  nat  $\Rightarrow$ 
  'f list  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$  nat list  $\Rightarrow$ 
  'f list list list  $\Rightarrow$  'f list list  $\Rightarrow$  bool

where
  generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers  $\longleftrightarrow$ 
  generic-fri-canonical-sampled-layer-chain-evidence low-degree
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers  $\wedge$ 
  candidate-bad candidate-table

lemma generic-fri-sampled-query-candidate-evidenceI:
assumes
  generic-fri-canonical-sampled-layer-chain-evidence low-degree
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers
  candidate-bad candidate-table
shows
  generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers
using assms
unfolding generic-fri-sampled-query-candidate-evidence-def by blast

lemma generic-fri-sampled-query-candidate-evidenceD:
assumes
  generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers
shows
  generic-fri-canonical-sampled-layer-chain-evidence low-degree
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers
  candidate-bad candidate-table
using assms
unfolding generic-fri-sampled-query-candidate-evidence-def by blast+

definition generic-fri-sampled-query-bad-pair-set
:: ('f list  $\Rightarrow$  bool)  $\Rightarrow$  ('f list  $\Rightarrow$  bool)  $\Rightarrow$ 
  'f list  $\Rightarrow$  nat  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$ 

```

$\text{'f list list list} \Rightarrow \text{'f list list} \Rightarrow$
 $(\text{nat list} \times \text{'f list}) \text{ set}$

where

$\text{generic-fri-sampled-query-bad-pair-set low-degree candidate-bad}$
 $\text{candidate-table degree-bound roots final-value round-layers layers} =$
 $\{(query\text{-}idxs, challenges)\}.$
 $\text{generic-fri-sampled-query-candidate-evidence low-degree candidate-bad}$
 $\text{candidate-table degree-bound roots challenges final-value query-idxs}$
 $\text{round-layers layers}\}$

lemma generic-fri-sampled-query-bad-pair-setI:

assumes

$\text{generic-fri-sampled-query-candidate-evidence low-degree candidate-bad}$
 $\text{candidate-table degree-bound roots challenges final-value query-idxs}$
 $\text{round-layers layers}$

shows

$(query\text{-}idxs, challenges) \in$
 $\text{generic-fri-sampled-query-bad-pair-set low-degree candidate-bad}$
 $\text{candidate-table degree-bound roots final-value round-layers layers}$

using *assms* **unfolding** *generic-fri-sampled-query-bad-pair-set-def* **by** *simp*

lemma generic-fri-sampled-query-bad-pair-setD:

assumes

$(query\text{-}idxs, challenges) \in$
 $\text{generic-fri-sampled-query-bad-pair-set low-degree candidate-bad}$
 $\text{candidate-table degree-bound roots final-value round-layers layers}$

shows

$\text{generic-fri-sampled-query-candidate-evidence low-degree candidate-bad}$
 $\text{candidate-table degree-bound roots challenges final-value query-idxs}$
 $\text{round-layers layers}$

using *assms* **unfolding** *generic-fri-sampled-query-bad-pair-set-def* **by** *simp*

lemma generic-fri-sampled-query-bad-pair-set-canonical-evidence:

assumes *pair:*

$(query\text{-}idxs, challenges) \in$
 $\text{generic-fri-sampled-query-bad-pair-set low-degree candidate-bad}$
 $\text{candidate-table degree-bound roots final-value round-layers layers}$

shows

$\text{generic-fri-canonical-sampled-layer-chain-evidence low-degree}$
 $\text{candidate-table degree-bound roots challenges final-value query-idxs}$
 $\text{round-layers layers}$

proof —

have *evidence:*

$\text{generic-fri-sampled-query-candidate-evidence low-degree candidate-bad}$
 $\text{candidate-table degree-bound roots challenges final-value query-idxs}$
 $\text{round-layers layers}$

by (rule *generic-fri-sampled-query-bad-pair-setD*[*OF pair*])

show *?thesis*

by (rule *generic-fri-sampled-query-candidate-evidenceD*(1)[*OF evidence*])

qed

lemma *generic-fri-sampled-query-bad-pair-set-sampled-evidence:*

assumes *pair:*

(query-idxs, challenges) ∈
generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
candidate-table degree-bound roots final-value round-layers layers

shows

generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
(fri-canonical-domains (length challenges)) layers

proof —

have *canonical:*

generic-fri-canonical-sampled-layer-chain-evidence low-degree
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers

by (rule *generic-fri-sampled-query-bad-pair-set-canonical-evidence*
[*OF pair*])

show *?thesis*

by (rule *generic-fri-canonical-sampled-layer-chain-evidenceD*
[*OF canonical*])

qed

lemma *generic-fri-sampled-query-bad-pair-set-partial-evidence:*

assumes *pair:*

(query-idxs, challenges) ∈
generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
candidate-table degree-bound roots final-value round-layers layers

shows

generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers

proof —

have *sampled:*

generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
(fri-canonical-domains (length challenges)) layers

by (rule *generic-fri-sampled-query-bad-pair-set-sampled-evidence*
[*OF pair*])

show *?thesis*

by (rule *generic-fri-sampled-layer-chain-evidenceD(1)*[*OF sampled*])

qed

lemma *generic-fri-sampled-query-bad-pair-set-challenge-length:*

assumes *pair:*

(query-idxs, challenges) ∈
generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
candidate-table degree-bound roots final-value round-layers layers

shows *length challenges = fri-round-count-for-degree-bound degree-bound*

proof —

have *evidence*:
generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers
by (rule *generic-fri-sampled-query-bad-pair-setD*[*OF pair*])
have *canonical*:
generic-fri-canonical-sampled-layer-chain-evidence low-degree
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers
by (rule *generic-fri-sampled-query-candidate-evidenceD*(1)[*OF evidence*])
have *sampled*:
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
(fri-canonical-domains (length challenges)) layers
by (rule *generic-fri-canonical-sampled-layer-chain-evidenceD*
[*OF canonical*])
have *partial*:
generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
by (rule *generic-fri-sampled-layer-chain-evidenceD*(1)[*OF sampled*])
show ?thesis
using *generic-fri-partial-evidence-shapes*(1,2)[*OF partial*] **by** *simp*
qed

lemma *generic-fri-sampled-query-bad-pair-set-round-layers-length*:
assumes *pair*:
(*query-idxs*, *challenges*) $\in$
generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
candidate-table degree-bound roots final-value round-layers layers
shows *length round-layers* = *length query-idxs*
proof –
have *partial*:
generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
by (rule *generic-fri-sampled-query-bad-pair-set-partial-evidence*
[*OF pair*])
have *fri-all-round-layer-evidence roots challenges query-idxs round-layers*
by (rule *generic-fri-partial-evidence-shapes*(3)[*OF partial*])
then show ?thesis
unfolding *fri-all-round-layer-evidence-def* **by** *simp*
qed

lemma *generic-fri-sampled-query-bad-pair-set-sample*:
assumes *pair*:
(*query-idxs*, *challenges*) $\in$
generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
candidate-table degree-bound roots final-value round-layers layers
and *round-bound*: *round-idx* < *length query-idxs*
and *layer-bound*: *layer-idx* < *length challenges*

shows

fri-evidence-next-idx roots query-idxs round-idx layer-idx <
length (layers ! Suc layer-idx)
fri-sampled-table-fold
(challenges ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
(2 ^ layer-idx)
(fri-canonical-domains (length challenges) ! layer-idx)
(layers ! layer-idx)
(round-layers ! round-idx ! layer-idx)
(layers ! Suc layer-idx !
fri-evidence-next-idx roots query-idxs round-idx layer-idx)

proof —

have *sampled*:

generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
(fri-canonical-domains (length challenges)) layers
by (rule *generic-fri-sampled-query-bad-pair-set-sampled-evidence*
[*OF pair*])

show *fri-evidence-next-idx roots query-idxs round-idx layer-idx <*
length (layers ! Suc layer-idx)

by (rule *generic-fri-sampled-layer-chain-evidence-sample(1)*
[*OF sampled round-bound layer-bound*])

show *fri-sampled-table-fold*

(challenges ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
(2 ^ layer-idx)
(fri-canonical-domains (length challenges) ! layer-idx)
(layers ! layer-idx)
(round-layers ! round-idx ! layer-idx)
(layers ! Suc layer-idx !
fri-evidence-next-idx roots query-idxs round-idx layer-idx)
by (rule *generic-fri-sampled-layer-chain-evidence-sample(2)*
[*OF sampled round-bound layer-bound*])

qed

lemma *generic-fri-sampled-query-bad-pair-set-candidate-bad*:

assumes *pair*:

(query-idxs, challenges) ∈
generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
candidate-table degree-bound roots final-value round-layers layers

shows *candidate-bad candidate-table*

proof —

have *evidence*:

generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers

by (rule generic-fri-sampled-query-bad-pair-setD[OF pair])
 show ?thesis
 by (rule generic-fri-sampled-query-candidate-evidenceD(2)[OF evidence])
 qed

lemma generic-fri-sampled-query-bad-pair-set-challenge-space:
 assumes
 (query-idxs, challenges) ∈
 generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
 candidate-table degree-bound roots final-value round-layers layers
 shows challenges ∈
 fri-challenge-space (fri-round-count-for-degree-bound degree-bound)
 using generic-fri-sampled-query-bad-pair-set-challenge-length[OF assms]
 unfolding fri-challenge-space-def by simp

lemma generic-fri-sampled-query-bad-pair-set-subset-challenge-space:
 generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
 candidate-table degree-bound roots final-value round-layers layers
 ⊆ (UNIV :: nat list set) ×
 fri-challenge-space (fri-round-count-for-degree-bound degree-bound)
 by (auto intro: generic-fri-sampled-query-bad-pair-set-challenge-space)

definition fri-query-challenge-pair-query-fiber
 :: (nat list × 'f list) set ⇒ 'f list ⇒ nat list set
 where
 fri-query-challenge-pair-query-fiber P challenges =
 {query-idxs. (query-idxs, challenges) ∈ P}

lemma finite-fri-query-index-list-space:
 finite fri-query-index-list-space
proof –
 have subset:
 fri-query-index-list-space ⊆
 {query-idxs. set query-idxs ⊆ query-sample-space ∧
 length query-idxs = rounds}
 unfolding fri-query-index-list-space-def by auto
 have finite-super:
 finite
 {query-idxs. set query-idxs ⊆ query-sample-space ∧
 length query-idxs = rounds}
 by (rule finite-lists-length-eq[OF finite-query-sample-space])
 show ?thesis
 by (rule finite-subset[OF subset finite-super])
 qed

lemma fri-query-challenge-pair-query-fiber-iff[simp]:
 query-idxs ∈ fri-query-challenge-pair-query-fiber P challenges ⟷
 (query-idxs, challenges) ∈ P
 unfolding fri-query-challenge-pair-query-fiber-def by simp

lemma *fri-query-challenge-pair-query-projection-restricted:*
 $\text{fst } ' (P \cap (\text{fri-query-index-list-space} \times \text{UNIV}))$
 $\subseteq \text{fri-query-index-list-space}$

proof
fix *query-idxs*
assume *query-idxs* $\in$
 $\text{fst } ' (P \cap (\text{fri-query-index-list-space} \times \text{UNIV}))$
then show *query-idxs* $\in \text{fri-query-index-list-space}$
by auto
qed

lemma *finite-fri-query-challenge-pair-query-fiber-restricted:*
finite
 $(\text{fri-query-challenge-pair-query-fiber}$
 $(P \cap (\text{fri-query-index-list-space} \times \text{UNIV})) \text{ challenges})$

proof –
have
 $\text{fri-query-challenge-pair-query-fiber}$
 $(P \cap (\text{fri-query-index-list-space} \times \text{UNIV})) \text{ challenges}$
 $\subseteq \text{fri-query-index-list-space}$
unfolding *fri-query-challenge-pair-query-fiber-def* **by blast**
then show *?thesis*
by (rule *finite-subset*) (rule *finite-fri-query-index-list-space*)
qed

definition *trace-fri-query-challenge-pair-set-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow (\text{nat list} \times 'f \text{ list}) \text{ set} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-query-challenge-pair-set-hit } s \text{ } P \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $(\text{fri-query-idxs, trace-bs}) \in P)$

definition *composition-fri-query-challenge-pair-set-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow (\text{nat list} \times 'f \text{ list}) \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-query-challenge-pair-set-hit } s \text{ } P \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$

$trace-final\ fri-dg\ composition-roots\ composition-bs$
 $composition-final\ fri-query-idxs\ trace-round-layers$
 $composition-round-layers \wedge$
 $(fri-query-idxs, composition-bs) \in P\ fri-dg$

lemma *accepted-fri-opening-transcript-query-index-list-space:*

assumes

$accepted-fri-opening-transcript\ s\ out\ trace-roots\ trace-bs\ trace-final$
 $dg\ composition-roots\ composition-bs\ composition-final\ fri-query-idxs$
 $trace-round-layers\ composition-round-layers$

shows $fri-query-idxs \in fri-query-index-list-space$

proof –

have $len: length\ fri-query-idxs = rounds$

using $accepted-fri-opening-transcript-shapes[OF\ assms]$ **by** *blast*

have $subset: set\ fri-query-idxs \subseteq query-sample-space$

proof

fix idx

assume $idx \in set\ fri-query-idxs$

then show $idx \in query-sample-space$

by (rule $accepted-fri-opening-transcript-query-idx-in-sample-space$
[*OF assms*])

qed

show *?thesis*

unfolding $fri-query-index-list-space-def$

using $len\ subset$ **by** *simp*

qed

lemma *trace-fri-query-challenge-pair-set-hit-restrict-to-query-space:*

assumes $trace-fri-query-challenge-pair-set-hit\ s\ P\ out$

shows

$trace-fri-query-challenge-pair-set-hit\ s$
 $(P \cap (fri-query-index-list-space \times UNIV))\ out$

proof –

from $assms$ **obtain** $trace-roots\ trace-bs\ trace-final\ dg\ composition-roots$
 $composition-bs\ composition-final\ fri-query-idxs\ trace-round-layers$
 $composition-round-layers$ **where**

transcript:

$accepted-fri-opening-transcript\ s\ out\ trace-roots\ trace-bs$
 $trace-final\ dg\ composition-roots\ composition-bs\ composition-final$
 $fri-query-idxs\ trace-round-layers\ composition-round-layers$

and $pair: (fri-query-idxs, trace-bs) \in P$

unfolding $trace-fri-query-challenge-pair-set-hit-def$ **by** *blast*

have $query-space: fri-query-idxs \in fri-query-index-list-space$

by (rule $accepted-fri-opening-transcript-query-index-list-space$
[*OF transcript*])

show *?thesis*

unfolding $trace-fri-query-challenge-pair-set-hit-def$

using $transcript\ pair\ query-space$ **by** *blast*

qed

lemma *composition-fri-query-challenge-pair-set-hit-restrict-to-query-space:*

assumes *composition-fri-query-challenge-pair-set-hit* s P *out*

shows

composition-fri-query-challenge-pair-set-hit s
 $(\lambda dg. P \ dg \cap (\text{fri-query-index-list-space} \times \text{UNIV})) \text{ out}$

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers **where**

transcript:

accepted-fri-opening-transcript s *out* *trace-roots trace-bs*

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers

and *pair:* $(\text{fri-query-idxs}, \text{composition-bs}) \in P$ *fri-dg*

unfolding *composition-fri-query-challenge-pair-set-hit-def* **by** *blast*

have *query-space:* *fri-query-idxs* $\in$ *fri-query-index-list-space*

by $(\text{rule } \text{accepted-fri-opening-transcript-query-index-list-space}$
 $[\text{OF } \text{transcript}])$

show *?thesis*

unfolding *composition-fri-query-challenge-pair-set-hit-def*

using *transcript pair query-space* **by** *blast*

qed

lemma *trace-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit:*

assumes *hit:* *trace-fri-query-challenge-pair-set-hit* s P *out*

and *projection:* $\text{fst} \text{ ' } P \subseteq Q$

shows *trace-fri-query-index-list-set-hit* s Q *out*

proof –

from *hit* **obtain** *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers

where *openings:*

accepted-fri-opening-transcript s *out* *trace-roots trace-bs*

trace-final dg composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers

and *pair:* $(\text{fri-query-idxs}, \text{trace-bs}) \in P$

unfolding *trace-fri-query-challenge-pair-set-hit-def* **by** *blast*

have *fri-query-idxs* $\in$ Q

using *pair projection* **by** *force*

then show *?thesis*

unfolding *trace-fri-query-index-list-set-hit-def*

using *openings* **by** *blast*

qed

lemma *composition-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit:*

assumes *hit:* *composition-fri-query-challenge-pair-set-hit* s P *out*

and *projection:* $\bigwedge dg. \text{fst} \text{ ' } P \ dg \subseteq Q$

shows *composition-fri-query-index-list-set-hit* s Q *out*
proof –
from *hit* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers
where *openings*:
accepted-fri-opening-transcript s *out trace-roots trace-bs*
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *pair*: $(\text{fri-query-idxs}, \text{composition-bs}) \in P$ *fri-dg*
unfolding *composition-fri-query-challenge-pair-set-hit-def* **by** *blast*
have *fri-query-idxs* $\in Q$
using *pair projection[of fri-dg]* **by** *force*
then show *?thesis*
unfolding *composition-fri-query-index-list-set-hit-def*
using *openings* **by** *blast*
qed

lemma *trace-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit*:
assumes *hit*: *trace-fri-query-challenge-pair-set-hit* s P *out*
and *projection*: $\text{snd } P \subseteq B$
shows *trace-fri-challenge-list-set-hit* s B *out*
proof –
from *hit* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers
where *openings*:
accepted-fri-opening-transcript s *out trace-roots trace-bs*
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *pair*: $(\text{fri-query-idxs}, \text{trace-bs}) \in P$
unfolding *trace-fri-query-challenge-pair-set-hit-def* **by** *blast*
have *challenges*:
accepted-fri-challenges s *out trace-bs dg composition-bs*
by $(\text{rule } \text{accepted-fri-opening-transcript-challenges}[\text{OF } \text{openings}])$
have *trace-bs* $\in B$
using *pair projection* **by** *force*
then show *?thesis*
unfolding *trace-fri-challenge-list-set-hit-def*
using *challenges* **by** *blast*
qed

lemma *composition-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit*:
assumes *hit*: *composition-fri-query-challenge-pair-set-hit* s P *out*
and *projection*: $\bigwedge dg. \text{snd } P \ dg \subseteq B \ dg$
shows *composition-fri-challenge-list-set-hit* s B *out*
proof –
from *hit* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers
where *openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *pair*: (*fri-query-idxs*, *composition-bs*) ∈ *P fri-dg*
unfolding *composition-fri-query-challenge-pair-set-hit-def* **by** *blast*
have *challenges*:
accepted-fri-challenges s out trace-bs fri-dg composition-bs
by (*rule accepted-fri-opening-transcript-challenges*[*OF openings*])
have *composition-bs* ∈ *B fri-dg*
using *pair projection*[*of fri-dg*] **by** *force*
then show *?thesis*
unfolding *composition-fri-challenge-list-set-hit-def*
using *challenges* **by** *blast*
qed

definition *trace-fri-sampled-query-pair-set-hit*
 $:: ('f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list list list} \Rightarrow$
 $'f \text{ list list} \Rightarrow (\text{nat list} \times 'f \text{ list}) \text{ set}) \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
trace-fri-sampled-query-pair-set-hit R s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table layers.}$
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table $\wedge$
 $(\text{fri-query-idxs}, \text{trace-bs}) \in$
 $R \text{ trace-table trace-roots trace-final trace-round-layers layers})$

definition *composition-fri-sampled-query-pair-set-hit*
 $:: ('f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list list list} \Rightarrow 'f \text{ list list} \Rightarrow$
 $(\text{nat list} \times 'f \text{ list}) \text{ set}) \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where

composition-fri-sampled-query-pair-set-hit R s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$

f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table layers.
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
(fri-query-idxs, composition-bs) $\in$
R fri-dg composition-table opening-composition-roots composition-final
composition-round-layers layers)

lemma *trace-fri-sampled-query-pair-set-hit-imp-challenge-list-set-hit:*

assumes *hit: trace-fri-sampled-query-pair-set-hit R s out*

and projection:

$\wedge$ trace-table roots final round-layers layers.
snd ‘

R trace-table roots final round-layers layers $\subseteq B$

shows *trace-fri-challenge-list-set-hit s B out*

proof –

from *hit* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table layers

where *evidence:*

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

and pair:

(fri-query-idxs, trace-bs) $\in$
R trace-table trace-roots trace-final trace-round-layers layers

unfolding *trace-fri-sampled-query-pair-set-hit-def* **by** *blast*

have *challenges:*

accepted-fri-challenges s out trace-bs dg composition-bs
by *(rule accepted-fri-opening-transcript-challenges)*
(rule trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence])

have *trace-bs $\in B$*

using *pair projection[of trace-table trace-roots trace-final*
trace-round-layers layers] **by** *force*

then show *?thesis*

unfolding *trace-fri-challenge-list-set-hit-def*

using *challenges* **by** *blast*

qed

lemma *composition-fri-sampled-query-pair-set-hit-imp-challenge-list-set-hit:*

assumes *hit: composition-fri-sampled-query-pair-set-hit R s out*
and projection:
 $\bigwedge \text{fri-dg composition-table roots final round-layers layers.}$
snd ‘
 $R \text{ fri-dg composition-table roots final round-layers layers}$
 $\subseteq B \text{ fri-dg}$
shows *composition-fri-challenge-list-set-hit s B out*
proof –
from *hit* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table layers
where *evidence:*
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
and *pair:*
 $(\text{fri-query-idxs}, \text{composition-bs}) \in$
 $R \text{ fri-dg composition-table opening-composition-roots}$
 $\text{composition-final composition-round-layers layers}$
unfolding *composition-fri-sampled-query-pair-set-hit-def* **by** *auto*
have *challenges:*
accepted-fri-challenges s out trace-bs fri-dg composition-bs
by *(rule accepted-fri-opening-transcript-challenges)*
(rule composition-fri-partial-candidate-opening-evidenceD(1)
[OF evidence])
have *composition-bs* $\in B \text{ fri-dg}$
using *pair projection[of fri-dg composition-table*
opening-composition-roots composition-final composition-round-layers
layers] **by** *force*
then show *?thesis*
unfolding *composition-fri-challenge-list-set-hit-def*
using *challenges* **by** *blast*
qed

lemma *trace-fri-sampled-query-pair-set-hit-imp-query-index-list-set-hit:*
assumes *hit: trace-fri-sampled-query-pair-set-hit R s out*
and projection:
 $\bigwedge \text{trace-table roots final round-layers layers.}$
fst ‘
 $R \text{ trace-table roots final round-layers layers} \subseteq Q$
shows *trace-fri-query-index-list-set-hit s Q out*
proof –

from *hit* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table layers
where *evidence*:
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *pair*:
(fri-query-idxs, trace-bs) ∈
R trace-table trace-roots trace-final trace-round-layers layers
unfolding *trace-fri-sampled-query-pair-set-hit-def* **by** *blast*
have *openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
by *(rule trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence])*
have *fri-query-idxs ∈ Q*
using *pair projection[of trace-table trace-roots trace-final*
trace-round-layers layers] **by** *force*
then show *?thesis*
unfolding *trace-fri-query-index-list-set-hit-def*
using *openings* **by** *blast*
qed

lemma *composition-fri-sampled-query-pair-set-hit-imp-query-index-list-set-hit*:
assumes *hit: composition-fri-sampled-query-pair-set-hit R s out*
and *projection*:
 $\bigwedge \text{fri-dg composition-table roots final round-layers layers.}$
 fst '
 $R \text{ fri-dg composition-table roots final round-layers layers}$
 $\subseteq Q$
shows *composition-fri-query-index-list-set-hit s Q out*
proof –
from *hit* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as declared-dg composition-fri-roots final-value
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table layers
where *evidence*:
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as declared-dg
composition-fri-roots final-value trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table

composition-table
and *pair*:
(fri-query-idxs, composition-bs) ∈
R fri-dg composition-table opening-composition-roots
composition-final composition-round-layers layers
unfolding *composition-fri-sampled-query-pair-set-hit-def* **by** *auto*
have *openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
by *(rule composition-fri-partial-candidate-opening-evidenceD(1)*
[OF evidence])
have *fri-query-idxs ∈ Q*
using *pair projection[of fri-dg composition-table*
opening-composition-roots composition-final composition-round-layers
layers] **by** *force*
then show *?thesis*
unfolding *composition-fri-query-index-list-set-hit-def*
using *openings* **by** *blast*
qed

lemma *trace-fri-sampled-query-pair-set-hit-imp-query-challenge-pair-set-hit*:
assumes *hit: trace-fri-sampled-query-pair-set-hit R s out*
and *projection*:
 $\bigwedge$ *trace-table roots final round-layers layers.*
R trace-table roots final round-layers layers ⊆ P
shows *trace-fri-query-challenge-pair-set-hit s P out*
proof –
from *hit* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table layers
where *evidence*:
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *pair*:
(fri-query-idxs, trace-bs) ∈
R trace-table trace-roots trace-final trace-round-layers layers
unfolding *trace-fri-sampled-query-pair-set-hit-def* **by** *blast*
have *openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
by *(rule trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence])*
have *(fri-query-idxs, trace-bs) ∈ P*

```

    using pair projection[of trace-table trace-roots trace-final
      trace-round-layers layers] by blast
  then show ?thesis
    unfolding trace-fri-query-challenge-pair-set-hit-def
    using openings by blast
qed

lemma composition-fri-sampled-query-pair-set-hit-imp-query-challenge-pair-set-hit:
  assumes hit: composition-fri-sampled-query-pair-set-hit R s out
    and projection:
       $\bigwedge$ fri-dg composition-table roots final round-layers layers.
      R fri-dg composition-table roots final round-layers layers
       $\subseteq$  P fri-dg
  shows composition-fri-query-challenge-pair-set-hit s P out
proof -
  from hit obtain trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as declared-dg composition-fri-roots final-value
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table layers
  where evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as declared-dg
    composition-fri-roots final-value trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  and pair:
    (fri-query-idxs, composition-bs)  $\in$ 
    R fri-dg composition-table opening-composition-roots
    composition-final composition-round-layers layers
  unfolding composition-fri-sampled-query-pair-set-hit-def by auto
  have openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  by (rule composition-fri-partial-candidate-opening-evidenceD(1)
    [OF evidence])
  have (fri-query-idxs, composition-bs)  $\in$  P fri-dg
  using pair projection[of fri-dg composition-table
    opening-composition-roots composition-final composition-round-layers
    layers] by blast
  then show ?thesis
    unfolding composition-fri-query-challenge-pair-set-hit-def
    using openings by blast
qed

```

lemma *trace-fri-sampled-query-pair-set-hit-imp-query-index-list-space-hit:*

assumes *hit: trace-fri-sampled-query-pair-set-hit R s out*

shows

trace-fri-query-index-list-set-hit s fri-query-index-list-space out

proof —

from *hit* **obtain** *trace-roots trace-bs trace-final dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table layers*

where *evidence:*

*trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table*

unfolding *trace-fri-sampled-query-pair-set-hit-def* **by** *blast*

have *openings:*

*accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers*

by *(rule trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence])*

have *fri-query-idxs ∈ fri-query-index-list-space*

by *(rule accepted-fri-opening-transcript-query-index-list-space
[OF openings])*

then show *?thesis*

unfolding *trace-fri-query-index-list-set-hit-def*

using *openings* **by** *blast*

qed

lemma *composition-fri-sampled-query-pair-set-hit-imp-query-index-list-space-hit:*

assumes *hit: composition-fri-sampled-query-pair-set-hit R s out*

shows

composition-fri-query-index-list-set-hit s fri-query-index-list-space out

proof —

from *hit* **obtain** *trace-roots trace-bs trace-final fri-dg
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as declared-dg composition-fri-roots final-value
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table layers*

where *evidence:*

*composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as declared-dg
composition-fri-roots final-value trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table*

unfolding *composition-fri-sampled-query-pair-set-hit-def* **by** *auto*
have *openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
by (rule *composition-fri-partial-candidate-opening-evidenceD(1)*
[*OF evidence*])
have *fri-query-idxs* $\in$ *fri-query-index-list-space*
by (rule *accepted-fri-opening-transcript-query-index-list-space*
[*OF openings*])
then show *?thesis*
unfolding *composition-fri-query-index-list-set-hit-def*
using *openings* **by** *blast*
qed

lemma *trace-fri-sampled-query-pair-set-hit-query-indices*:
assumes *hit: trace-fri-sampled-query-pair-set-hit R s out*
obtains *query-idxs challenges candidate-table roots final round-layers layers*
where
length query-idxs = rounds
 $\forall idx \in \text{set } query-idxs. idx \in query-sample-space$
 $\forall idx \in \text{set } query-idxs. idx < clength * scale$
(*query-idxs, challenges*) $\in$
R candidate-table roots final round-layers layers
proof –
from *hit* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
candidate-table candidate-layers
where *evidence*:
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
candidate-table
and *pair*:
(*fri-query-idxs, trace-bs*) $\in$
R candidate-table trace-roots trace-final trace-round-layers
candidate-layers
unfolding *trace-fri-sampled-query-pair-set-hit-def* **by** *blast*
have *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
by (rule *trace-fri-partial-candidate-opening-evidenceD(1)*[*OF evidence*])
have *len: length fri-query-idxs = rounds*
using *accepted-fri-opening-transcript-shapes[OF fri-openings]* **by** *blast*
have *sample-space: $\forall idx \in \text{set } fri-query-idxs. idx \in query-sample-space$*

proof
 fix idx
 assume $idx \in \text{set } \text{fri-query-idxs}$
 then show $idx \in \text{query-sample-space}$
 by (rule *accepted-fri-opening-transcript-query-idx-in-sample-space*
 [OF *fri-openings*])
 qed
 have *bounds*: $\forall idx \in \text{set } \text{fri-query-idxs}. idx < \text{clength} * \text{scale}$
 using *fri-openings* **unfolding** *accepted-fri-opening-transcript-def*
 by *blast*
 show ?thesis
 by (rule *that*[OF *len sample-space bounds pair*])
 qed

lemma *composition-fri-sampled-query-pair-set-hit-query-indices*:
 assumes *hit*: *composition-fri-sampled-query-pair-set-hit R s out*
 obtains *query-idxs challenges dg candidate-table roots final round-layers*
layers
 where
 length *query-idxs* = *rounds*
 $\forall idx \in \text{set } \text{query-idxs}. idx \in \text{query-sample-space}$
 $\forall idx \in \text{set } \text{query-idxs}. idx < \text{clength} * \text{scale}$
 (*query-idxs*, *challenges*) $\in$
R dg candidate-table roots final round-layers layers
proof –
 from *hit* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as declared-dg composition-fri-roots final-value
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table candidate-table candidate-layers
 where *evidence*:
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as declared-dg
composition-fri-roots final-value trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
candidate-table
 and *pair*:
 (*fri-query-idxs*, *composition-bs*) $\in$
R fri-dg candidate-table opening-composition-roots
composition-final composition-round-layers candidate-layers
unfolding *composition-fri-sampled-query-pair-set-hit-def* **by** *auto*
have *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers

```

    by (rule composition-fri-partial-candidate-opening-evidenceD(1)
        [OF evidence])
  have len: length fri-query-idxs = rounds
    using accepted-fri-opening-transcript-shapes[OF fri-openings] by blast
  have sample-space:  $\forall idx \in \text{set fri-query-idxs}. idx \in \text{query-sample-space}$ 
  proof
    fix idx
    assume idx  $\in$  set fri-query-idxs
    then show idx  $\in$  query-sample-space
      by (rule accepted-fri-opening-transcript-query-idx-in-sample-space
          [OF fri-openings])
    qed
  have bounds:  $\forall idx \in \text{set fri-query-idxs}. idx < \text{clength} * \text{scale}$ 
    using fri-openings unfolding accepted-fri-opening-transcript-def
    by blast
  show ?thesis
    by (rule that[OF len sample-space bounds pair])
  qed

```

lemma wp-trace-fri-sampled-query-pair-set-hit-bound-from-projection:

```

  assumes future: trace-fri-future-fresh s
  and projection:
     $\bigwedge \text{trace-table roots final round-layers layers}.$ 
    snd '
     $R \text{ trace-table roots final round-layers layers} \subseteq B$ 
  and subset:  $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$ 
  and bound:
     $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log clength}) \leq C$ 
  shows
    wp-event verify-monad
      (trace-fri-sampled-query-pair-set-hit R s) s  $\leq C$ 

```

proof –

```

  have wp-event verify-monad
    (trace-fri-sampled-query-pair-set-hit R s) s  $\leq$ 
    wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s
  by (rule wp-event-mono)
    (rule trace-fri-sampled-query-pair-set-hit-imp-challenge-list-set-hit
        [OF - projection])
  also have ...  $\leq \text{nnreal } (\text{card } B) /$ 
     $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log clength})$ 
  by (rule wp-verify-monad-trace-fri-challenge-list-set-bound
      [OF future subset])
  also have ...  $\leq C$ 
  by (rule bound)
  finally show ?thesis .

```

qed

lemma wp-composition-fri-sampled-query-pair-set-hit-bound-from-projection:

```

  fixes C :: prob

```

assumes *future: composition-fri-future-fresh s*
and *projection:*
 $\bigwedge \text{fri-dg composition-table roots final round-layers layers.}$
snd ‘
 $R \text{ fri-dg composition-table roots final round-layers layers}$
 $\subseteq B \text{ fri-dg}$
and *subset:*
 $\bigwedge \text{dg. } B \text{ dg} \subseteq \text{fri-challenge-space (ceil-log (to-nat dg + 1))}$
and *bound:*
 $\bigwedge \text{dg. nnreal (card (B dg)) /}$
 $\text{nnreal (CARD('f) } \wedge \text{ ceil-log (to-nat dg + 1))} \leq C$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-query-pair-set-hit } R \text{ s}) \text{ s} \leq C$
proof –
have $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-query-pair-set-hit } R \text{ s}) \text{ s} \leq$
 $\text{wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s}$
by (rule wp-event-mono)
(rule
 $\text{composition-fri-sampled-query-pair-set-hit-imp-challenge-list-set-hit}$
 $[\text{OF - projection}]$)
also have $\dots \leq C$
by (rule $\text{wp-verify-monad-composition-fri-challenge-list-set-bound}$
 $[\text{OF future subset bound}]$)
finally show *?thesis* .
qed

lemma *wp-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-projection:*
assumes *future: trace-fri-future-fresh s*
and *projection:* $\text{snd } P \subseteq B$
and *subset:* $B \subseteq \text{fri-challenge-space (ceil-log clength)}$
and *bound:*
 $\text{nnreal (card B) / nnreal (CARD('f) } \wedge \text{ ceil-log clength)} \leq C$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-query-challenge-pair-set-hit s P}) \text{ s} \leq C$
proof –
have $\text{wp-event verify-monad}$
 $(\text{trace-fri-query-challenge-pair-set-hit s P}) \text{ s} \leq$
 $\text{wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s}$
by (rule wp-event-mono)
(rule $\text{trace-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit}$
 $[\text{OF - projection}]$)
also have $\dots \leq \text{nnreal (card B) /}$
 $\text{nnreal (CARD('f) } \wedge \text{ ceil-log clength)}$
by (rule $\text{wp-verify-monad-trace-fri-challenge-list-set-bound}$
 $[\text{OF future subset}]$)
also have $\dots \leq C$

by (rule bound)
 finally show ?thesis .
 qed

lemma *wp-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-projection:*

fixes $C :: \text{prob}$
 assumes *future: composition-fri-future-fresh s*
 and *projection: $\bigwedge dg. \text{snd } 'P \ dg \subseteq B \ dg$*
 and *subset:*
 $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$
 and *bound:*
 $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq C$

shows

wp-event verify-monad
 $(\text{composition-fri-query-challenge-pair-set-hit } s \ P) \ s \leq C$

proof –

have *wp-event verify-monad*
 $(\text{composition-fri-query-challenge-pair-set-hit } s \ P) \ s \leq$
wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s
by (rule *wp-event-mono*)
 (rule
composition-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit
 [OF - *projection*])

also have $\dots \leq C$

by (rule *wp-verify-monad-composition-fri-challenge-list-set-bound*
 [OF *future subset bound*])

finally show ?thesis .

qed

definition *trace-fri-sampled-query-bad-candidate*

$:: ('f, 'a) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$

where

trace-fri-sampled-query-bad-candidate s out $\longleftrightarrow$
 $(\exists \text{trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table layers.}$
trace-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
generic-fri-sampled-query-candidate-evidence trace-table-low-degree
 $(\text{Not } \circ \text{trace-table-low-degree}) \text{trace-table } (\text{clength} - 1)$
 $\text{trace-roots trace-bs trace-final fri-query-idxs trace-round-layers}$
 $\text{layers})$

definition *composition-fri-sampled-query-bad-candidate*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-fri-sampled-query-bad-candidate } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table layers.}$
 $\text{composition-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings trace-table}$
 $\text{composition-table} \wedge$
 $\text{generic-fri-sampled-query-candidate-evidence}$
 $(\text{composition-table-low-degree } (\text{to-nat fri-dg}))$
 $(\text{Not} \circ \text{composition-table-low-degree } \text{maxDegree}) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs composition-round-layers layers})$

definition *generic-fri-sampled-query-bad-pair-union*
 $:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow \text{nat} \Rightarrow$
 $(\text{nat list} \times 'f \text{ list}) \text{ set}$
where
 $\text{generic-fri-sampled-query-bad-pair-union low-degree candidate-bad}$
 $\text{degree-bound} =$
 $\{\text{pair. } \exists \text{ candidate-table roots final round-layers layers.}$
 $\text{pair} \in$
 $\text{generic-fri-sampled-query-bad-pair-set low-degree candidate-bad}$
 $\text{candidate-table degree-bound}$
 $\text{roots final round-layers layers}\}$

definition *generic-fri-sampled-query-projection-fraction-bound*
 $:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow \text{nat} \Rightarrow$
 $\text{prob} \Rightarrow \text{bool}$
where
 $\text{generic-fri-sampled-query-projection-fraction-bound low-degree}$
 $\text{candidate-bad degree-bound } C \longleftrightarrow$
 nnreal
 $(\text{card } (\text{snd } '$
 $(\text{generic-fri-sampled-query-bad-pair-union low-degree candidate-bad}$
 $\text{degree-bound} \cap (\text{fri-query-index-list-space} \times \text{UNIV})))) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound degree-bound})$
 $\leq C$

lemma *generic-fri-sampled-query-projection-fraction-boundD*:
assumes
generic-fri-sampled-query-projection-fraction-bound low-degree
candidate-bad degree-bound C
shows

$$\text{nnreal } (\text{card } (\text{snd } ' (\text{generic-fri-sampled-query-bad-pair-union low-degree candidate-bad} \\ \text{degree-bound } \cap (\text{fri-query-index-list-space } \times \text{UNIV})))) / \\ \text{nnreal } (\text{CARD('f)} \wedge \text{fri-round-count-for-degree-bound degree-bound}) \\ \leq C$$

using *assms*
unfolding *generic-fri-sampled-query-projection-fraction-bound-def* .

lemma *generic-fri-sampled-query-projection-fraction-bound-mono*:
assumes
generic-fri-sampled-query-projection-fraction-bound low-degree
candidate-bad degree-bound C
and $C \leq D$
shows
generic-fri-sampled-query-projection-fraction-bound low-degree
candidate-bad degree-bound D
using *assms*
unfolding *generic-fri-sampled-query-projection-fraction-bound-def*
by (*rule order-trans*)

lemma *generic-fri-sampled-query-projection-fraction-bound-from-card*:
assumes *card-bound*:

$$\text{card } (\text{snd } ' (\text{generic-fri-sampled-query-bad-pair-union low-degree candidate-bad} \\ \text{degree-bound } \cap (\text{fri-query-index-list-space } \times \text{UNIV})))) \leq K$$

and *fraction-bound*:

$$\text{nnreal } K / \\ \text{nnreal } (\text{CARD('f)} \wedge \text{fri-round-count-for-degree-bound degree-bound}) \\ \leq C$$

shows
generic-fri-sampled-query-projection-fraction-bound low-degree
candidate-bad degree-bound C
proof –
have *nnreal*

$$(\text{card } (\text{snd } ' (\text{generic-fri-sampled-query-bad-pair-union low-degree candidate-bad} \\ \text{degree-bound } \cap (\text{fri-query-index-list-space } \times \text{UNIV})))) / \\ \text{nnreal } (\text{CARD('f)} \wedge \text{fri-round-count-for-degree-bound degree-bound}) \\ \leq \\ \text{nnreal } K / \\ \text{nnreal } (\text{CARD('f)} \wedge \text{fri-round-count-for-degree-bound degree-bound})$$

using *card-bound* **by** (*rule nnreal-nat-divide-right-mono*)
also have $\dots \leq C$

by (rule fraction-bound)
 finally show ?thesis
 unfolding generic-fri-sampled-query-projection-fraction-bound-def .
 qed

lemma *generic-fri-sampled-query-projection-fraction-bound-from-finite-cover:*

assumes *cover:*

snd ‘

(*generic-fri-sampled-query-bad-pair-union* *low-degree* *candidate-bad*
degree-bound $\cap$ (*fri-query-index-list-space* $\times$ *UNIV*)) $\subseteq$ *B*

and *finite-B:* *finite B*

and *fraction-bound:*

nnreal (*card B*) /

nnreal (*CARD*('f) $\wedge$ *fri-round-count-for-degree-bound* *degree-bound*)
 $\leq$ *C*

shows

generic-fri-sampled-query-projection-fraction-bound *low-degree*
candidate-bad *degree-bound C*

proof (rule *generic-fri-sampled-query-projection-fraction-bound-from-card*)

show *card* (*snd* ‘

(*generic-fri-sampled-query-bad-pair-union* *low-degree* *candidate-bad*
degree-bound $\cap$ (*fri-query-index-list-space* $\times$ *UNIV*)))

$\leq$ *card B*

by (rule *card-mono*[*OF* *finite-B* *cover*])

show *nnreal* (*card B*) /

nnreal (*CARD*('f) $\wedge$ *fri-round-count-for-degree-bound* *degree-bound*)
 $\leq$ *C*

by (rule *fraction-bound*)

qed

definition *trace-fri-sampled-query-bad-pair-union*

$::$ (*nat list* $\times$ '*f list*) *set*

where

trace-fri-sampled-query-bad-pair-union =

generic-fri-sampled-query-bad-pair-union *trace-table-low-degree*
(*Not* $\circ$ *trace-table-low-degree*) (*clength* - 1)

definition *composition-fri-sampled-query-bad-pair-union*

$::$ '*f* $\Rightarrow$ (*nat list* $\times$ '*f list*) *set*

where

composition-fri-sampled-query-bad-pair-union *fri-dg* =

generic-fri-sampled-query-bad-pair-union

(*composition-table-low-degree* (*to-nat* *fri-dg*))

(*Not* $\circ$ *composition-table-low-degree* *maxDegree*) (*to-nat* *fri-dg*)

lemma *generic-fri-sampled-query-bad-pair-unionI:*

assumes

(*query-idxs*, *challenges*) $\in$

generic-fri-sampled-query-bad-pair-set *low-degree* *candidate-bad*

candidate-table degree-bound roots final round-layers layers
shows
(query-idxs, challenges) ∈
generic-fri-sampled-query-bad-pair-union low-degree candidate-bad
degree-bound
using *assms*
unfolding *generic-fri-sampled-query-bad-pair-union-def* **by** *blast*

lemma *generic-fri-sampled-query-bad-pair-union-subset-challenge-space:*
generic-fri-sampled-query-bad-pair-union low-degree candidate-bad
degree-bound
 $\subseteq (UNIV :: nat\ list\ set) \times$
fri-challenge-space (fri-round-count-for-degree-bound degree-bound)
unfolding *generic-fri-sampled-query-bad-pair-union-def*
by (*auto intro: generic-fri-sampled-query-bad-pair-set-challenge-space*)

lemma *generic-fri-sampled-query-bad-pair-union-challenge-projection:*
snd ‘
generic-fri-sampled-query-bad-pair-union low-degree candidate-bad
degree-bound
 $\subseteq$ *fri-challenge-space*
(fri-round-count-for-degree-bound degree-bound)
using *generic-fri-sampled-query-bad-pair-union-subset-challenge-space*
[of low-degree candidate-bad degree-bound]
by *force*

Fallback sanity bound: the restricted projection is always a subset of the full challenge space. This lemma is intentionally not the main FRI low-degree theorem; the useful symbolic bounds must come from a sharper sampled-query FRI argument.

lemma *generic-fri-sampled-query-projection-fraction-bound-trivial:*
generic-fri-sampled-query-projection-fraction-bound low-degree
candidate-bad degree-bound 1
proof –
let *?S =*
snd ‘
(generic-fri-sampled-query-bad-pair-union low-degree candidate-bad
degree-bound ∩ (fri-query-index-list-space × UNIV))
let *?N = CARD('f) ^ fri-round-count-for-degree-bound degree-bound*
have *subset:*
?S ⊆ fri-challenge-space
(fri-round-count-for-degree-bound degree-bound)
using *generic-fri-sampled-query-bad-pair-union-challenge-projection*
[of low-degree candidate-bad degree-bound]
by *blast*
have *card-le: card ?S ≤ ?N*
using *card-mono[OF finite-fri-challenge-space subset]*
by (*simp add: card-fri-challenge-space*)
have *?N ≠ 0*
by *simp*

then have $\text{nnreal } ?N / \text{nnreal } ?N = (1::\text{prob})$
by (rule *nnreal-nat-divide-self*)
moreover have $\text{nnreal } (\text{card } ?S) / \text{nnreal } ?N \leq \text{nnreal } ?N / \text{nnreal } ?N$
using *card-le* **by** (rule *nnreal-nat-divide-right-mono*)
ultimately show *?thesis*
unfolding *generic-fri-sampled-query-projection-fraction-bound-def*
by *simp*
qed

lemma *trace-fri-sampled-query-bad-pair-union-challenge-projection:*
snd ' trace-fri-sampled-query-bad-pair-union $\subseteq$
fri-challenge-space (ceil-log clength)
unfolding *trace-fri-sampled-query-bad-pair-union-def*
using *generic-fri-sampled-query-bad-pair-union-challenge-projection*
[of trace-table-low-degree Not $\circ$ trace-table-low-degree
clength - 1]
by (*simp add: fri-round-count-for-degree-bound-def clength-pos*)

lemma *composition-fri-sampled-query-bad-pair-union-challenge-projection:*
snd ' composition-fri-sampled-query-bad-pair-union fri-dg $\subseteq$
fri-challenge-space (ceil-log (to-nat fri-dg + 1))
unfolding *composition-fri-sampled-query-bad-pair-union-def*
using *generic-fri-sampled-query-bad-pair-union-challenge-projection*
[of composition-table-low-degree (to-nat fri-dg)
Not $\circ$ composition-table-low-degree maxDegree to-nat fri-dg]
by (*simp add: fri-round-count-for-degree-bound-def*)

lemma *trace-fri-restricted-sampled-query-bad-pair-union-challenge-projection:*
snd ' (trace-fri-sampled-query-bad-pair-union $\cap$
(fri-query-index-list-space $\times$ UNIV))
 $\subseteq$ fri-challenge-space (ceil-log clength)
using *trace-fri-sampled-query-bad-pair-union-challenge-projection* **by** *blast*

lemma *composition-fri-restricted-sampled-query-bad-pair-union-challenge-projection:*
snd ' (composition-fri-sampled-query-bad-pair-union fri-dg $\cap$
(fri-query-index-list-space $\times$ UNIV))
 $\subseteq$ fri-challenge-space (ceil-log (to-nat fri-dg + 1))
using *composition-fri-sampled-query-bad-pair-union-challenge-projection*
[of fri-dg]
by *blast*

lemma *trace-fri-sampled-query-bad-candidate-iff-pair-set-hit:*
trace-fri-sampled-query-bad-candidate s out $\longleftrightarrow$
trace-fri-sampled-query-pair-set-hit
(λ trace-table roots final round-layers layers.
generic-fri-sampled-query-bad-pair-set trace-table-low-degree
(Not $\circ$ trace-table-low-degree) trace-table (clength - 1)

```

      roots final round-layers layers)
    s out
unfolding trace-fri-sampled-query-bad-candidate-def
  trace-fri-sampled-query-pair-set-hit-def
  generic-fri-sampled-query-bad-pair-set-def
by blast

lemma composition-fri-sampled-query-bad-candidate-iff-pair-set-hit:
  composition-fri-sampled-query-bad-candidate s out  $\longleftrightarrow$ 
  composition-fri-sampled-query-pair-set-hit
    ( $\lambda$ fri-dg composition-table roots final round-layers layers.
      generic-fri-sampled-query-bad-pair-set
        (composition-table-low-degree (to-nat fri-dg))
        (Not  $\circ$  composition-table-low-degree maxDegree)
        composition-table (to-nat fri-dg) roots final round-layers layers)
    s out
unfolding composition-fri-sampled-query-bad-candidate-def
  composition-fri-sampled-query-pair-set-hit-def
  generic-fri-sampled-query-bad-pair-set-def
by auto

lemma trace-fri-sampled-query-bad-candidate-imp-query-challenge-pair-union-hit:
assumes trace-fri-sampled-query-bad-candidate s out
shows
  trace-fri-query-challenge-pair-set-hit s
  trace-fri-sampled-query-bad-pair-union out
proof –
have hit:
  trace-fri-sampled-query-pair-set-hit
    ( $\lambda$ trace-table roots final round-layers layers.
      generic-fri-sampled-query-bad-pair-set trace-table-low-degree
        (Not  $\circ$  trace-table-low-degree) trace-table (clength – 1)
        roots final round-layers layers)
    s out
using assms trace-fri-sampled-query-bad-candidate-iff-pair-set-hit
by blast
show ?thesis
proof (rule trace-fri-sampled-query-pair-set-hit-imp-query-challenge-pair-set-hit
  [OF hit])
fix trace-table roots final round-layers layers
show
  generic-fri-sampled-query-bad-pair-set trace-table-low-degree
    (Not  $\circ$  trace-table-low-degree) trace-table (clength – 1)
    roots final round-layers layers
 $\subseteq$  trace-fri-sampled-query-bad-pair-union
unfolding trace-fri-sampled-query-bad-pair-union-def
by (auto intro: generic-fri-sampled-query-bad-pair-unionI)
qed
qed

```

lemma *composition-fri-sampled-query-bad-candidate-imp-query-challenge-pair-union-hit:*

assumes *composition-fri-sampled-query-bad-candidate s out*

shows

composition-fri-query-challenge-pair-set-hit s

composition-fri-sampled-query-bad-pair-union out

proof —

have *hit:*

composition-fri-sampled-query-pair-set-hit

$(\lambda \text{fri-dg } \text{composition-table roots final round-layers layers.}$

generic-fri-sampled-query-bad-pair-set

$(\text{composition-table-low-degree } (\text{to-nat fri-dg}))$

$(\text{Not} \circ \text{composition-table-low-degree maxDegree})$

composition-table (to-nat fri-dg) roots final round-layers

layers)

s out

using *assms composition-fri-sampled-query-bad-candidate-iff-pair-set-hit*

by *blast*

show *?thesis*

proof (*rule*

composition-fri-sampled-query-pair-set-hit-imp-query-challenge-pair-set-hit

[OF hit])

fix *fri-dg composition-table roots final round-layers layers*

show

generic-fri-sampled-query-bad-pair-set

$(\text{composition-table-low-degree } (\text{to-nat fri-dg}))$

$(\text{Not} \circ \text{composition-table-low-degree maxDegree})$

composition-table (to-nat fri-dg) roots final round-layers layers

$\subseteq$ *composition-fri-sampled-query-bad-pair-union fri-dg*

unfolding *composition-fri-sampled-query-bad-pair-union-def*

by (*auto intro: generic-fri-sampled-query-bad-pair-unionI*)

qed

qed

lemma *trace-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit:*

assumes *trace-fri-sampled-query-bad-candidate s out*

shows

trace-fri-query-challenge-pair-set-hit s

$(\text{trace-fri-sampled-query-bad-pair-union} \cap$

$(\text{fri-query-index-list-space} \times \text{UNIV})) \text{ out}$

by (*rule trace-fri-query-challenge-pair-set-hit-restrict-to-query-space*)

(*rule*

trace-fri-sampled-query-bad-candidate-imp-query-challenge-pair-union-hit

[OF assms])

lemma *composition-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit:*

assumes *composition-fri-sampled-query-bad-candidate s out*

shows

composition-fri-query-challenge-pair-set-hit s

$(\lambda dg. \text{composition-fri-sampled-query-bad-pair-union } dg \cap$
 $(\text{fri-query-index-list-space} \times \text{UNIV})) \text{ out}$
by $(\text{rule } \text{composition-fri-query-challenge-pair-set-hit-restrict-to-query-space})$
 $(\text{rule}$
 $\text{composition-fri-sampled-query-bad-candidate-imp-query-challenge-pair-union-hit}$
 $[\text{OF } \text{assms}])$

lemma $\text{wp-trace-fri-sampled-query-bad-candidate-iff-pair-set-hit}:$
 $\text{wp-event verify-monad } (\text{trace-fri-sampled-query-bad-candidate } s) \text{ } s =$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-query-pair-set-hit}$
 $(\lambda \text{trace-table roots final round-layers layers.}$
 $\text{generic-fri-sampled-query-bad-pair-set trace-table-low-degree}$
 $(\text{Not} \circ \text{trace-table-low-degree}) \text{ trace-table } (\text{clength} - 1)$
 $\text{roots final round-layers layers})$
 $s) \text{ } s$

proof (rule antisym)
show $\text{wp-event verify-monad } (\text{trace-fri-sampled-query-bad-candidate } s) \text{ } s$
 $\leq \text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-query-pair-set-hit}$
 $(\lambda \text{trace-table roots final round-layers layers.}$
 $\text{generic-fri-sampled-query-bad-pair-set trace-table-low-degree}$
 $(\text{Not} \circ \text{trace-table-low-degree}) \text{ trace-table } (\text{clength} - 1)$
 $\text{roots final round-layers layers})$
 $s) \text{ } s$
by $(\text{rule wp-event-mono})$
 $(\text{use trace-fri-sampled-query-bad-candidate-iff-pair-set-hit in blast})$
show $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-query-pair-set-hit}$
 $(\lambda \text{trace-table roots final round-layers layers.}$
 $\text{generic-fri-sampled-query-bad-pair-set trace-table-low-degree}$
 $(\text{Not} \circ \text{trace-table-low-degree}) \text{ trace-table } (\text{clength} - 1)$
 $\text{roots final round-layers layers})$
 $s) \text{ } s$
 $\leq \text{wp-event verify-monad } (\text{trace-fri-sampled-query-bad-candidate } s) \text{ } s$
by $(\text{rule wp-event-mono})$
 $(\text{use trace-fri-sampled-query-bad-candidate-iff-pair-set-hit in blast})$
qed

lemma $\text{wp-composition-fri-sampled-query-bad-candidate-iff-pair-set-hit}:$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-query-bad-candidate } s) \text{ } s =$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-query-pair-set-hit}$
 $(\lambda \text{fri-dg composition-table roots final round-layers layers.}$
 $\text{generic-fri-sampled-query-bad-pair-set}$
 $(\text{composition-table-low-degree } (\text{to-nat fri-dg}))$
 $(\text{Not} \circ \text{composition-table-low-degree } \text{maxDegree})$
 $\text{composition-table } (\text{to-nat fri-dg}) \text{ roots final round-layers}$

```

      layers)
    s) s
proof (rule antisym)
  show wp-event verify-monad
    (composition-fri-sampled-query-bad-candidate s) s
  ≤ wp-event verify-monad
    (composition-fri-sampled-query-pair-set-hit
      (λfri-dg composition-table roots final round-layers layers.
        generic-fri-sampled-query-bad-pair-set
          (composition-table-low-degree (to-nat fri-dg))
          (Not ∘ composition-table-low-degree maxDegree)
          composition-table (to-nat fri-dg) roots final round-layers
            layers)
        s) s
      s) s
  by (rule wp-event-mono)
    (use composition-fri-sampled-query-bad-candidate-iff-pair-set-hit
      in blast)
  show wp-event verify-monad
    (composition-fri-sampled-query-pair-set-hit
      (λfri-dg composition-table roots final round-layers layers.
        generic-fri-sampled-query-bad-pair-set
          (composition-table-low-degree (to-nat fri-dg))
          (Not ∘ composition-table-low-degree maxDegree)
          composition-table (to-nat fri-dg) roots final round-layers
            layers)
        s) s
      s) s
  ≤ wp-event verify-monad
    (composition-fri-sampled-query-bad-candidate s) s
  by (rule wp-event-mono)
    (use composition-fri-sampled-query-bad-candidate-iff-pair-set-hit
      in blast)
qed

lemma wp-trace-fri-sampled-query-bad-candidate-bound-by-query-challenge-union-hit:
  wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s ≤
  wp-event verify-monad
    (trace-fri-query-challenge-pair-set-hit s
      trace-fri-sampled-query-bad-pair-union) s
by (rule wp-event-mono)
  (rule
    trace-fri-sampled-query-bad-candidate-imp-query-challenge-pair-union-hit)

lemma wp-composition-fri-sampled-query-bad-candidate-bound-by-query-challenge-union-hit:
  wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s ≤
  wp-event verify-monad
    (composition-fri-query-challenge-pair-set-hit s
      composition-fri-sampled-query-bad-pair-union) s
by (rule wp-event-mono)
  (rule

```

composition-fri-sampled-query-bad-candidate-imp-query-challenge-pair-union-hit)

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-by-restricted-query-challenge-union-hit:*

wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s ≤

wp-event verify-monad

(trace-fri-query-challenge-pair-set-hit s

(trace-fri-sampled-query-bad-pair-union ∩

(fri-query-index-list-space × UNIV))) s

by *(rule wp-event-mono)*

(rule

trace-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit)

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-by-restricted-query-challenge-union-hit:*

wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s ≤

wp-event verify-monad

(composition-fri-query-challenge-pair-set-hit s

(λdg. composition-fri-sampled-query-bad-pair-union dg ∩

(fri-query-index-list-space × UNIV))) s

by *(rule wp-event-mono)*

(rule

composition-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit)

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-from-bad-pair-union-projection:*

assumes *future: trace-fri-future-fresh s*

and *bound:*

nnreal (card (snd ‘ trace-fri-sampled-query-bad-pair-union)) /

nnreal (CARD('f) ^ ceil-log clength) ≤ C

shows

wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s ≤ C

proof –

have *wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s*

≤ wp-event verify-monad

(trace-fri-query-challenge-pair-set-hit s

trace-fri-sampled-query-bad-pair-union) s

by *(rule*

wp-trace-fri-sampled-query-bad-candidate-bound-by-query-challenge-union-hit)

also have *... ≤ C*

by *(rule wp-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-projection*

[OF future subset-refl

trace-fri-sampled-query-bad-pair-union-challenge-projection

bound])

finally show *?thesis .*

qed

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-from-bad-pair-union-projection:*

fixes *C :: prob*

assumes *future: composition-fri-future-fresh s*

and *bound:*

∧ dg. nnreal

$$\frac{(\text{card } (\text{snd } \text{'composition-fri-sampled-query-bad-pair-union } dg))}{\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1))} \leq C$$

shows

$\text{wp-event verify-monad } (\text{composition-fri-sampled-query-bad-candidate } s) \ s \leq C$

proof –

have $\text{wp-event verify-monad } (\text{composition-fri-sampled-query-bad-candidate } s) \ s \leq$

$\text{wp-event verify-monad } (\text{composition-fri-query-challenge-pair-set-hit } s$

$\text{composition-fri-sampled-query-bad-pair-union}) \ s$

by (rule

$\text{wp-composition-fri-sampled-query-bad-candidate-bound-by-query-challenge-union-hit})$

also have $\dots \leq C$

by (rule

$\text{wp-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-projection}$

$[\text{OF future subset-refl}$

$\text{composition-fri-sampled-query-bad-pair-union-challenge-projection}$

$\text{bound}])$

finally show ?thesis .

qed

lemma $\text{wp-trace-fri-sampled-query-bad-candidate-bound-from-restricted-bad-pair-union-projection}$:

assumes $\text{future: trace-fri-future-fresh } s$

and bound:

nnreal

$$\frac{(\text{card } (\text{snd } \text{'(trace-fri-sampled-query-bad-pair-union } \cap$$

$$\text{(fri-query-index-list-space } \times \text{ UNIV))}))}{\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } \text{clength})} \leq C$$

shows

$\text{wp-event verify-monad } (\text{trace-fri-sampled-query-bad-candidate } s) \ s \leq C$

proof –

have $\text{wp-event verify-monad } (\text{trace-fri-sampled-query-bad-candidate } s) \ s$

$\leq \text{wp-event verify-monad } (\text{trace-fri-query-challenge-pair-set-hit } s$

$(\text{trace-fri-sampled-query-bad-pair-union } \cap$

$\text{(fri-query-index-list-space } \times \text{ UNIV))}) \ s$

by (rule

$\text{wp-trace-fri-sampled-query-bad-candidate-bound-by-restricted-query-challenge-union-hit})$

also have $\dots \leq C$

by (rule $\text{wp-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-projection}$

$[\text{OF future subset-refl}$

$\text{trace-fri-restricted-sampled-query-bad-pair-union-challenge-projection}$

$\text{bound}])$

finally show ?thesis .

qed

lemma $\text{wp-composition-fri-sampled-query-bad-candidate-bound-from-restricted-bad-pair-union-projection}$:

```

fixes  $C :: \text{prob}$ 
assumes future: composition-fri-future-fresh s
and bound:
   $\bigwedge dg. \text{nnreal}$ 
     $(\text{card } (\text{snd } ' ($ 
       $(\text{composition-fri-sampled-query-bad-pair-union } dg \cap$ 
         $(\text{fri-query-index-list-space} \times \text{UNIV})))) /$ 
       $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq C$ 
shows
   $\text{wp-event verify-monad } (\text{composition-fri-sampled-query-bad-candidate } s) s$ 
     $\leq C$ 
proof –
  have wp-event verify-monad
     $(\text{composition-fri-sampled-query-bad-candidate } s) s \leq$ 
wp-event verify-monad
     $(\text{composition-fri-query-challenge-pair-set-hit } s$ 
       $(\lambda dg. \text{composition-fri-sampled-query-bad-pair-union } dg \cap$ 
         $(\text{fri-query-index-list-space} \times \text{UNIV}))) s$ 
    by (rule
      wp-composition-fri-sampled-query-bad-candidate-bound-by-restricted-query-challenge-union-hit)
  also have  $\dots \leq C$ 
    by (rule
      wp-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-projection
      [OF future subset-refl
        composition-fri-restricted-sampled-query-bad-pair-union-challenge-projection
        bound])
  finally show ?thesis .
qed

lemma trace-fri-sampled-query-bad-pair-union-generic-projection-fraction-bound:
assumes
  generic-fri-sampled-query-projection-fraction-bound
  trace-table-low-degree  $(\text{Not} \circ \text{trace-table-low-degree}) (\text{clength} - 1) C$ 
shows
   $\text{nnreal}$ 
     $(\text{card } (\text{snd } ' ($ 
       $(\text{trace-fri-sampled-query-bad-pair-union} \cap$ 
         $(\text{fri-query-index-list-space} \times \text{UNIV})))) /$ 
       $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } \text{clength}) \leq C$ 
using assms
unfolding generic-fri-sampled-query-projection-fraction-bound-def
  trace-fri-sampled-query-bad-pair-union-def
by (simp add: fri-round-count-for-degree-bound-def clength-pos)

lemma composition-fri-sampled-query-bad-pair-union-generic-projection-fraction-bound:
assumes
  generic-fri-sampled-query-projection-fraction-bound
   $(\text{composition-table-low-degree } (\text{to-nat } dg))$ 
   $(\text{Not} \circ \text{composition-table-low-degree } \text{maxDegree}) (\text{to-nat } dg) C$ 

```


shows

nnreal
(*card* (*snd* ‘
 (*composition-fri-sampled-query-bad-pair-union dg* $\cap$
 (*fri-query-index-list-space* $\times$ *UNIV*)))) /
 nnreal (*CARD*('f) $\wedge$ *ceil-log* (*to-nat dg* + 1)) $\leq C$

using *assms*

unfolding *generic-fri-sampled-query-projection-fraction-bound-def*

composition-fri-sampled-query-bad-pair-union-def

by (*simp add: fri-round-count-for-degree-bound-def*)

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction:*

assumes *future: trace-fri-future-fresh s*

and *generic-bound:*

generic-fri-sampled-query-projection-fraction-bound
trace-table-low-degree (*Not* $\circ$ *trace-table-low-degree*)
(*clength* - 1) *C*

shows

wp-event verify-monad (*trace-fri-sampled-query-bad-candidate s*) *s* $\leq C$

by (*rule*

wp-trace-fri-sampled-query-bad-candidate-bound-from-restricted-bad-pair-union-projection
[*OF future*])

(*rule*

trace-fri-sampled-query-bad-pair-union-generic-projection-fraction-bound
[*OF generic-bound*])

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction:*

fixes *C :: prob*

assumes *future: composition-fri-future-fresh s*

and *generic-bound:*

$\bigwedge dg.$ *generic-fri-sampled-query-projection-fraction-bound*
(*composition-table-low-degree* (*to-nat dg*))
(*Not* $\circ$ *composition-table-low-degree maxDegree*) (*to-nat dg*) *C*

shows

wp-event verify-monad (*composition-fri-sampled-query-bad-candidate s*) *s*
 $\leq C$

by (*rule*

wp-composition-fri-sampled-query-bad-candidate-bound-from-restricted-bad-pair-union-projection
[*OF future*])

(*rule*

composition-fri-sampled-query-bad-pair-union-generic-projection-fraction-bound
[*OF generic-bound*])

lemma *trace-fri-sampled-query-bad-candidate-imp-query-index-list-space-hit:*

assumes *trace-fri-sampled-query-bad-candidate s out*

shows

trace-fri-query-index-list-set-hit s fri-query-index-list-space out

using *assms trace-fri-sampled-query-bad-candidate-iff-pair-set-hit*

trace-fri-sampled-query-pair-set-hit-imp-query-index-list-space-hit

by *blast*

lemma *composition-fri-sampled-query-bad-candidate-imp-query-index-list-space-hit:*

assumes *composition-fri-sampled-query-bad-candidate s out*

shows

composition-fri-query-index-list-set-hit s fri-query-index-list-space out

using *assms composition-fri-sampled-query-bad-candidate-iff-pair-set-hit*

composition-fri-sampled-query-pair-set-hit-imp-query-index-list-space-hit

by *blast*

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-by-query-index-space-hit:*

wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s ≤

wp-event verify-monad

(trace-fri-query-index-list-set-hit s fri-query-index-list-space) s

by *(rule wp-event-mono)*

(rule trace-fri-sampled-query-bad-candidate-imp-query-index-list-space-hit)

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-by-query-index-space-hit:*

wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s ≤

wp-event verify-monad

(composition-fri-query-index-list-set-hit s fri-query-index-list-space) s

by *(rule wp-event-mono)*

(rule

composition-fri-sampled-query-bad-candidate-imp-query-index-list-space-hit)

lemma *trace-fri-sampled-query-bad-candidate-iff-canonical:*

trace-fri-sampled-query-bad-candidate s out $\longleftrightarrow$

trace-fri-bad-with-canonical-sampled-layer-chain s out

proof

assume *trace-fri-sampled-query-bad-candidate s out*

then show *trace-fri-bad-with-canonical-sampled-layer-chain s out*

unfolding *trace-fri-sampled-query-bad-candidate-def*

trace-fri-bad-with-canonical-sampled-layer-chain-def

generic-fri-sampled-query-candidate-evidence-def

by *blast*

next

assume *canonical: trace-fri-bad-with-canonical-sampled-layer-chain s out*

then obtain *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr candidate-query-idxs trace-openings

trace-table layers

where *evidence:*

trace-fri-partial-candidate-opening-evidence s out trace-roots

trace-bs trace-final dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers fr candidate-query-idxs trace-openings

trace-table

and *sampled:*

generic-fri-canonical-sampled-layer-chain-evidence

```

    trace-table-low-degree trace-table (clength - 1) trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers layers
unfolding trace-fri-bad-with-canonical-sampled-layer-chain-def by blast
have not-low: (Not ◦ trace-table-low-degree) trace-table
using trace-fri-partial-candidate-opening-evidenceD(2)[OF evidence]
by (simp add: o-def trace-fri-partial-candidate-evidenceD(3))
have bad:
  generic-fri-sampled-query-candidate-evidence trace-table-low-degree
  (Not ◦ trace-table-low-degree) trace-table (clength - 1)
  trace-roots trace-bs trace-final fri-query-idxs trace-round-layers
  layers
using sampled not-low
unfolding generic-fri-sampled-query-candidate-evidence-def by blast
show trace-fri-sampled-query-bad-candidate s out
unfolding trace-fri-sampled-query-bad-candidate-def
by (intro exI conjI) (rule evidence, rule bad)
qed

```

lemma composition-fri-partial-candidate-opening-evidence-candidate-bad:

assumes

```

  composition-fri-partial-candidate-opening-evidence s out trace-roots
  trace-bs trace-final fri-dg opening-composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr f-fri-roots f-final as dg
  composition-fri-roots final trace-query-idxs trace-openings
  composition-query-idxs composition-openings trace-table
  composition-table

```

shows (Not ◦ composition-table-low-degree maxDegree) composition-table

using composition-fri-partial-candidate-opening-evidenceD(2)[OF assms]

by (simp add: o-def composition-fri-partial-candidate-evidenceD(5))

lemma composition-fri-sampled-query-bad-candidate-iff-canonical:

```

  composition-fri-sampled-query-bad-candidate s out  $\longleftrightarrow$ 
  composition-fri-bad-with-canonical-sampled-layer-chain s out

```

proof (rule iffI)

assume composition-fri-sampled-query-bad-candidate s out

then show composition-fri-bad-with-canonical-sampled-layer-chain s out

unfolding composition-fri-sampled-query-bad-candidate-def

```

  composition-fri-bad-with-canonical-sampled-layer-chain-def
  generic-fri-sampled-query-candidate-evidence-def

```

apply (elim exE conjE)

apply (intro exI conjI)

apply assumption

apply assumption

done

next

assume composition-fri-bad-with-canonical-sampled-layer-chain s out

then show composition-fri-sampled-query-bad-candidate s out

unfolding composition-fri-sampled-query-bad-candidate-def

```

    composition-fri-bad-with-canonical-sampled-layer-chain-def
    generic-fri-sampled-query-candidate-evidence-def
using composition-fri-partial-candidate-opening-evidence-candidate-bad
apply (elim exE conjE)
apply (intro exI conjI)
    apply assumption
    apply assumption
apply (rule composition-fri-partial-candidate-opening-evidence-candidate-bad)
apply assumption
done
qed

```

lemma *wp-trace-fri-sampled-query-bad-candidate-iff-canonical:*
 $wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\ s)\ s =$
 $wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}bad\text{-}with\text{-}canonical\text{-}sampled\text{-}layer\text{-}chain\ s)\ s$

proof (rule antisym)
show $wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\ s)\ s$
 $\leq wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}bad\text{-}with\text{-}canonical\text{-}sampled\text{-}layer\text{-}chain\ s)\ s$
by (rule wp-event-mono)
 (use trace-fri-sampled-query-bad-candidate-iff-canonical **in** blast)
show $wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}bad\text{-}with\text{-}canonical\text{-}sampled\text{-}layer\text{-}chain\ s)\ s$
 $\leq wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\ s)\ s$
by (rule wp-event-mono)
 (use trace-fri-sampled-query-bad-candidate-iff-canonical **in** blast)
qed

lemma *wp-composition-fri-sampled-query-bad-candidate-iff-canonical:*
 $wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\ s)\ s =$
 $wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}bad\text{-}with\text{-}canonical\text{-}sampled\text{-}layer\text{-}chain\ s)\ s$

proof (rule antisym)
show $wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\ s)\ s$
 $\leq wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}bad\text{-}with\text{-}canonical\text{-}sampled\text{-}layer\text{-}chain\ s)\ s$
by (rule wp-event-mono)
 (use composition-fri-sampled-query-bad-candidate-iff-canonical **in** blast)
show $wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}bad\text{-}with\text{-}canonical\text{-}sampled\text{-}layer\text{-}chain\ s)\ s$
 $\leq wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\ s)\ s$
by (rule wp-event-mono)
 (use composition-fri-sampled-query-bad-candidate-iff-canonical **in** blast)
qed

The following bounds expose the current internal sampled-FRI interface: a sampled layer-

chain cover plus a bad-set envelope implies a challenge-list hit, which is then bounded by the existing Fiat-Shamir challenge-list calculus. The cover and envelope parameters are internal proof-layer interfaces; they are not public protocol assumptions.

lemma *trace-fri-sampled-query-bad-candidate-imp-list-hit:*

assumes *sampld: trace-fri-sampled-query-bad-candidate s out*
and *cover: trace-fri-sampled-layer-chain-cover s bad*
and *envelope:*
 $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
shows *trace-fri-challenge-list-set-hit s B out*

proof –

have *canonical: trace-fri-bad-with-canonical-sampled-layer-chain s out*
using *sampld trace-fri-sampled-query-bad-candidate-iff-canonical*
by *blast*
then have *sampld-chain: trace-fri-bad-with-sampled-layer-chain s out*
by *(rule trace-fri-bad-with-canonical-sampled-layer-chain-imp-sampld)*
show *?thesis*
by *(rule trace-fri-bad-with-sampled-layer-chain-imp-list-hit*
[OF sampld-chain cover envelope])

qed

lemma *composition-fri-sampled-query-bad-candidate-imp-list-hit:*

assumes *sampld: composition-fri-sampled-query-bad-candidate s out*
and *cover: composition-fri-sampled-layer-chain-cover s bad*
and *envelope:*
 $\bigwedge dg \text{ composition-table. }$
 $\text{composition-fri-multiround-bad-sets bad dg composition-table}$
 $\subseteq B dg$

shows *composition-fri-challenge-list-set-hit s B out*

proof –

have *canonical:*
composition-fri-bad-with-canonical-sampled-layer-chain s out
using *sampld composition-fri-sampled-query-bad-candidate-iff-canonical*
by *blast*
then have *sampld-chain:*
composition-fri-bad-with-sampled-layer-chain s out
by *(rule composition-fri-bad-with-canonical-sampled-layer-chain-imp-sampld)*
show *?thesis*
by *(rule composition-fri-bad-with-sampled-layer-chain-imp-list-hit*
[OF sampld-chain cover envelope])

qed

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-from-envelope:*

assumes *future: trace-fri-future-fresh s*
and *cover: trace-fri-sampled-layer-chain-cover s bad*
and *envelope:*
 $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *subset: $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$*
and *bound:*
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } \text{clength}) \leq C$

shows
 $wp\text{-event verify-monad } (trace\text{-fri-sampled-query-bad-candidate } s) \ s \leq C$
proof –
have $wp\text{-event verify-monad } (trace\text{-fri-sampled-query-bad-candidate } s) \ s$
 $\leq wp\text{-event verify-monad } (trace\text{-fri-challenge-list-set-hit } s \ B) \ s$
by (rule $wp\text{-event-mono}$)
(rule $trace\text{-fri-sampled-query-bad-candidate-imp-list-hit$
 $[OF - cover\ envelope]$)
also have $\dots \leq nnreal \ (card \ B) \ /$
 $nnreal \ (CARD('f) \wedge \text{ceil-log } clength)$
by (rule $wp\text{-verify-monad-trace-fri-challenge-list-set-bound}$
 $[OF \text{ future subset}]$)
also have $\dots \leq C$
by (rule $bound$)
finally show ?thesis .
qed

lemma $wp\text{-composition-fri-sampled-query-bad-candidate-bound-from-envelope}$:
assumes $future$: $composition\text{-fri-future-fresh } s$
and $cover$: $composition\text{-fri-sampled-layer-chain-cover } s \ bad$
and $envelope$:
 $\bigwedge dg \ composition\text{-table.}$
 $composition\text{-fri-multi-round-bad-sets } bad \ dg \ composition\text{-table}$
 $\subseteq B \ dg$
and $subset$:
 $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$
and $bound$:
 $\bigwedge dg. nnreal \ (card \ (B \ dg)) \ /$
 $nnreal \ (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq C$
shows
 $wp\text{-event verify-monad } (composition\text{-fri-sampled-query-bad-candidate } s) \ s$
 $\leq C$
proof –
have $wp\text{-event verify-monad}$
 $(composition\text{-fri-sampled-query-bad-candidate } s) \ s \leq$
 $wp\text{-event verify-monad } (composition\text{-fri-challenge-list-set-hit } s \ B) \ s$
by (rule $wp\text{-event-mono}$)
(rule $composition\text{-fri-sampled-query-bad-candidate-imp-list-hit}$
 $[OF - cover\ envelope]$)
also have $\dots \leq C$
by (rule $wp\text{-verify-monad-composition-fri-challenge-list-set-bound}$
 $[OF \text{ future subset bound}]$)
finally show ?thesis .
qed

lemma $checked\text{-staged-security-trace-fri-sampled-query-bad-candidate-bound}$:
assumes wf : $staged\text{-budget-wellformed budgets}$
and $controlled$: $staged\text{-adversary-controlled budgets } A$
and $finite\text{-}B$: $finite \ B$

and *cover*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
trace-fri-sampled-layer-chain-cover
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)) *bad*
and *envelope*:
 $\wedge$ *data attacker-state trace-table*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
trace-fri-multiround-bad-sets *bad* *trace-table* $\subseteq B$
and *fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s.$ *trace-fri-challenge-list-fresh-hit* *s* *B*))
adversary-initial-state $\leq F$
shows
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-sampled-query-bad-candidate)
adversary-initial-state $\leq$
F +
hash-relation-budget-value (*card* *B* * *ceil-log* *clength*)
(*staged-attacker-query-budget* *budgets* + *staged-challenge-query-budget*)
proof –
have *wp-event* (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-sampled-query-bad-candidate)
adversary-initial-state $\leq$
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s.$ *trace-fri-challenge-list-set-hit* *s* *B*))
adversary-initial-state
proof (*rule wp-event-mono-on-support*)
fix *out*
assume *support*:
out $\in$
set-dist
(*execute* (*checked-staged-security-experiment-with-data-state* *A*)
adversary-initial-state)
and *hit*:
staged-security-with-data-state-verifier-event
trace-fri-sampled-query-bad-candidate *out*
from *hit* **obtain** *data attacker-state result final-state* **where**
out-eq: *out* = *Some* (((*data*, *attacker-state*), *result*), *final-state*)
unfolding *staged-security-with-data-state-verifier-event-def*

```

    by (cases out) (auto split: option.splits prod.splits)
  have builder:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  using checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
  by blast
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have sampled:
    trace-fri-sampled-query-bad-candidate ?s
      (Some (result, final-state))
  using hit unfolding staged-security-with-data-state-verifier-event-def
    out-eq by simp
  have list-hit:
    trace-fri-challenge-list-set-hit ?s B (Some (result, final-state))
  by (rule trace-fri-sampled-query-bad-candidate-imp-list-hit
    [OF sampled cover[OF builder]])
    (rule envelope[OF builder])
  show
    staged-security-with-data-state-verifier-event
      (λs. trace-fri-challenge-list-set-hit s B) out
  unfolding staged-security-with-data-state-verifier-event-def out-eq
  using list-hit by simp
qed
also have ... ≤
  F +
    hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule checked-staged-security-trace-fri-challenge-list-set-hit-bound
    [OF wf controlled finite-B fresh-bound])
  finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-sampled-query-bad-candidate-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
  and cover:
     $\bigwedge \text{data attacker-state.}$ 
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
    composition-fri-sampled-layer-chain-cover
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) bad
  and envelope:

```


$\wedge$ data attacker-state dg composition-table.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B$ dg
and fresh-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . composition-fri-challenge-list-fresh-hit s B))
 adversary-initial-state $\leq F$
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-sampled-query-bad-candidate)
 adversary-initial-state $\leq$
 F +
 hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \text{ set}).$
 card (B dg) * ceil-log (maxDegree + 1))
 (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof –
have wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-sampled-query-bad-candidate)
 adversary-initial-state $\leq$
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . composition-fri-challenge-list-set-hit s B))
 adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
 out $\in$
 set-dist
 (execute (checked-staged-security-experiment-with-data-state A)
 adversary-initial-state)
and hit:
 staged-security-with-data-state-verifier-event
 composition-fri-sampled-query-bad-candidate out
from hit **obtain** data attacker-state result final-state **where**
 out-eq: out = Some (((data, attacker-state), result), final-state)
unfolding staged-security-with-data-state-verifier-event-def
by (cases out) (auto split: option.splits prod.splits)
have builder:
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state)
using checked-staged-security-experiment-with-data-state-outcomeE

```

    [OF support[unfolded out-eq]]
  by blast
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have sampled:
  composition-fri-sampled-query-bad-candidate ?s
  (Some (result, final-state))
using hit unfolding staged-security-with-data-state-verifier-event-def
  out-eq by simp
have list-hit:
  composition-fri-challenge-list-set-hit ?s B
  (Some (result, final-state))
by (rule composition-fri-sampled-query-bad-candidate-imp-list-hit
    [OF sampled cover[OF builder]])
(rule envelope[OF builder])
show
  staged-security-with-data-state-verifier-event
  (λs. composition-fri-challenge-list-set-hit s B) out
unfolding staged-security-with-data-state-verifier-event-def out-eq
using list-hit by simp
qed
also have ... ≤
  F +
  hash-relation-budget-value
  (∑ dg ∈ (UNIV :: 'f set).
    card (B dg) * ceil-log (maxDegree + 1))
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
by (rule checked-staged-security-composition-fri-challenge-list-set-hit-bound
    [OF wf controlled finite-B fresh-bound])
finally show ?thesis .
qed

lemma active-fri-reductions-for-from-trace-augmented-composition-sampled-query-and-empty-conflict-zero:
  fixes TraceAug TraceMissing TraceMerkle CompSampled CompMerkle EmptySampled Empty-
  Conflict EmptyZero ::
    'f staged-proof-data ⇒ 'f protocol-channel ⇒ prob
  assumes trace-augmented-bound:
    ∧ data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
      wp-event verify-monad
        (trace-fri-bad-with-header-tied-augmented-partial-candidate
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)) ≤ TraceAug data attacker-state
  and trace-missing-bound:

```

$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-header-tied-recorded-sibling-candidate-missing
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 TraceMissing data attacker-state
and trace-merkle-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ TraceMerkle data attacker-state
and trace-total:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 TraceAug data attacker-state +
 (TraceMissing data attacker-state + TraceMerkle data attacker-state)
 $\leq$ trace-fri-error
and composition-sampled-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-sampled-query-bad-candidate
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 CompSampled data attacker-state
and composition-merkle-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{CompMerkle data attacker-state}$
and *composition-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{CompSampled data attacker-state} +$
 $((\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0 + 0) +$
 $\text{CompMerkle data attacker-state}) + 0)$
 $\leq \text{composition-fri-error}$
and *empty-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-query-bad-candidate}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptySampled data attacker-state}$
and *empty-conflict-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyConflict data attacker-state}$
and *empty-zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$

```

    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤
    EmptyZero data attacker-state
and empty-total:
 $\wedge$ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
                     adversary-initial-state)  $\implies$ 
    EmptySampled data attacker-state +
      (EmptyConflict data attacker-state + EmptyZero data attacker-state)
    ≤ trace-fri-error
shows active-fri-reductions-for A
proof (rule
  active-fri-reductions-for-from-trace-augmented-composition-canonical-and-empty-conflict-zero
  [OF trace-augmented-bound trace-missing-bound trace-merkle-bound
   trace-total - composition-merkle-bound composition-total -
   empty-conflict-bound empty-zero-bound empty-total])
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
                   adversary-initial-state)
show wp-event verify-monad
  (composition-fri-bad-with-canonical-sampled-layer-chain
   (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))
  ≤ CompSampled data attacker-state
using composition-sampled-bound[OF support]
by (simp add: wp-composition-fri-sampled-query-bad-candidate-iff-canonical)
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
                   adversary-initial-state)
show wp-event verify-monad
  (trace-fri-bad-with-canonical-sampled-layer-chain
   (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))
  ≤ EmptySampled data attacker-state
using empty-sampled-bound[OF support]
by (simp add: wp-trace-fri-sampled-query-bad-candidate-iff-canonical)
qed

```

definition generic-fri-sampled-query-finite-cover

$:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ list set} \Rightarrow \text{bool}$

where

generic-fri-sampled-query-finite-cover low-degree candidate-bad
 $\text{degree-bound } B \longleftrightarrow$
 $\text{finite } B \wedge$
 snd '
 $(\text{generic-fri-sampled-query-bad-pair-union low-degree candidate-bad}$
 $\text{degree-bound} \cap (\text{fri-query-index-list-space} \times \text{UNIV})) \subseteq B$

lemma *generic-fri-sampled-query-finite-coverD:*

assumes

generic-fri-sampled-query-finite-cover low-degree candidate-bad
 $\text{degree-bound } B$

shows

$\text{finite } B$
 snd '
 $(\text{generic-fri-sampled-query-bad-pair-union low-degree candidate-bad}$
 $\text{degree-bound} \cap (\text{fri-query-index-list-space} \times \text{UNIV})) \subseteq B$

using *assms*

unfolding *generic-fri-sampled-query-finite-cover-def* **by** *simp-all*

lemma *generic-fri-sampled-query-finite-cover-from-candidate-challenge-cover:*

assumes *finite-B: finite B*

and *cover:*

$\bigwedge \text{candidate-table roots final-value round-layers layers query-idxs}$
 challenges.
 $\text{query-idxs} \in \text{fri-query-index-list-space} \implies$
 $\text{generic-fri-sampled-query-candidate-evidence low-degree candidate-bad}$
 $\text{candidate-table degree-bound roots challenges final-value query-idxs}$
 $\text{round-layers layers} \implies$
 $\text{challenges} \in B$

shows

generic-fri-sampled-query-finite-cover low-degree candidate-bad
 $\text{degree-bound } B$

proof $-$

have *subset:*

snd '
 $(\text{generic-fri-sampled-query-bad-pair-union low-degree candidate-bad}$
 $\text{degree-bound} \cap (\text{fri-query-index-list-space} \times \text{UNIV})) \subseteq B$

proof

fix *challenges*

assume $\text{challenges} \in \text{snd '}$

$(\text{generic-fri-sampled-query-bad-pair-union low-degree candidate-bad}$
 $\text{degree-bound} \cap (\text{fri-query-index-list-space} \times \text{UNIV}))$

then obtain *pair* **where** *pair-mem:*

$\text{pair} \in \text{generic-fri-sampled-query-bad-pair-union low-degree}$
 $\text{candidate-bad degree-bound} \cap (\text{fri-query-index-list-space} \times \text{UNIV})$

and *challenges-eq: challenges = snd pair*

by blast
 obtain query-idxs challenges' where pair-eq: pair = (query-idxs, challenges')
 by (cases pair)
 from pair-mem pair-eq challenges-eq
 obtain candidate-table roots final-value round-layers layers
 where query-space: query-idxs $\in$ fri-query-index-list-space
 and pair:
 (query-idxs, challenges) $\in$
 generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
 candidate-table degree-bound roots final-value round-layers layers
 unfolding generic-fri-sampled-query-bad-pair-union-def by auto
 have evidence:
 generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
 candidate-table degree-bound roots challenges final-value query-idxs
 round-layers layers
 by (rule generic-fri-sampled-query-bad-pair-setD[OF pair])
 show challenges $\in B$
 by (rule cover[OF query-space evidence])
 qed
 show ?thesis
 unfolding generic-fri-sampled-query-finite-cover-def
 using finite-B subset by simp
 qed

lemma generic-fri-sampled-query-projection-fraction-bound-from-cover:
 assumes cover:
 generic-fri-sampled-query-finite-cover low-degree candidate-bad
 degree-bound B
 and fraction-bound:
 $\text{nnreal } (\text{card } B) /$
 $\text{nnreal } (\text{CARD}(f) \wedge \text{fri-round-count-for-degree-bound degree-bound})$
 $\leq C$
 shows
 generic-fri-sampled-query-projection-fraction-bound low-degree
 candidate-bad degree-bound C
 by (rule generic-fri-sampled-query-projection-fraction-bound-from-finite-cover)
 (rule generic-fri-sampled-query-finite-coverD(2)[OF cover],
 rule generic-fri-sampled-query-finite-coverD(1)[OF cover],
 rule fraction-bound)

lemma trace-fri-sampled-query-bad-candidate-bound-from-generic-cover:
 assumes future: trace-fri-future-fresh s
 and cover:
 generic-fri-sampled-query-finite-cover trace-table-low-degree
 (Not $\circ$ trace-table-low-degree) (clength - 1) B
 and fraction-bound:
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log clength}) \leq C$
 shows
 wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s $\leq C$

```

proof (rule
  wp-trace-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction
  [OF future])
show
  generic-fri-sampled-query-projection-fraction-bound
  trace-table-low-degree (Not ∘ trace-table-low-degree)
  (clength - 1) C
by (rule generic-fri-sampled-query-projection-fraction-bound-from-cover
  [OF cover])
  (use fraction-bound in
    ⟨simp add: fri-round-count-for-degree-bound-def clength-pos⟩)
qed

lemma composition-fri-sampled-query-bad-candidate-bound-from-generic-cover:
fixes C :: prob
assumes future: composition-fri-future-fresh s
and cover:
  ∧dg. generic-fri-sampled-query-finite-cover
    (composition-table-low-degree (to-nat dg))
    (Not ∘ composition-table-low-degree maxDegree) (to-nat dg) (B dg)
and fraction-bound:
  ∧dg. nnreal (card (B dg)) /
    nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C
shows
  wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s
  ≤ C
proof (rule
  wp-composition-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction
  [OF future])
fix dg
show
  generic-fri-sampled-query-projection-fraction-bound
  (composition-table-low-degree (to-nat dg))
  (Not ∘ composition-table-low-degree maxDegree) (to-nat dg) C
by (rule generic-fri-sampled-query-projection-fraction-bound-from-cover
  [OF cover[of dg]])
  (use fraction-bound[of dg] in
    ⟨simp add: fri-round-count-for-degree-bound-def⟩)
qed

end

end

```

```

theory Soundness-FRI-Query-Aware-Interface
imports Soundness-FRI-Sampled-Interface
begin

```

Query-aware sampled FRI interface.

This theory names the joint query/challenge boundary used by the sampled FRI low-degree reduction. The definitions are generic and are instantiated by the trace and composition FRI routes.

context *soundness*
begin

definition *generic-fri-sampled-query-restricted-bad-pairs*

$:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow \text{nat} \Rightarrow$
 $(\text{nat list} \times 'f \text{ list}) \text{ set}$

where

generic-fri-sampled-query-restricted-bad-pairs *low-degree candidate-bad*
degree-bound =
generic-fri-sampled-query-bad-pair-union *low-degree candidate-bad*
degree-bound $\cap$ (*fri-query-index-list-space* $\times$ *UNIV*)

definition *generic-fri-sampled-query-pair-fraction-bound*

$:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow \text{nat} \Rightarrow$
 $\text{prob} \Rightarrow \text{bool}$

where

generic-fri-sampled-query-pair-fraction-bound *low-degree candidate-bad*
degree-bound $C \longleftrightarrow$
 nnreal
 $(\text{card}$
 $(\text{generic-fri-sampled-query-restricted-bad-pairs } \text{low-degree}$
 $\text{candidate-bad } \text{degree-bound})) /$
 nnreal
 $(\text{card } \text{fri-query-index-list-space} *$
 $\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound } \text{degree-bound})$
 $\leq C$

definition *generic-fri-sampled-query-challenge-projection*

$:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ list set}$

where

generic-fri-sampled-query-challenge-projection *low-degree candidate-bad*
degree-bound =
 $\text{snd } \text{'}$
generic-fri-sampled-query-restricted-bad-pairs *low-degree*
candidate-bad degree-bound

definition *generic-fri-sampled-query-query-fiber*

$:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ list} \Rightarrow \text{nat list set}$

where

generic-fri-sampled-query-query-fiber *low-degree candidate-bad*
degree-bound challenges =
fri-query-challenge-pair-query-fiber
(generic-fri-sampled-query-restricted-bad-pairs *low-degree*
candidate-bad degree-bound) *challenges*

definition *query-rounds-index-list-hit*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{nat list} \Rightarrow \text{nat} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

query-rounds-index-list-hit s *query-idxs* n *out* $\longleftrightarrow$
 $(\exists \text{ results } t \text{ raw-idxs.}$
 $\text{out} = \text{Some } (\text{results}, t) \wedge$
 $\text{length raw-idxs} = n \wedge$
 $\text{length query-idxs} = n \wedge$
 $\text{query-idxs} = \text{map } (\lambda \text{ raw. index } (\text{to-nat raw})) \text{ raw-idxs} \wedge$
 $(\forall i < n. \exists x.$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{QueryIndexChallenge } (\text{PQueryCounter } s + i) x) =$
 $\text{Some } (\text{raw-idxs } ! i)))$

lemma *card-fri-query-index-list-space*:

$\text{card fri-query-index-list-space} = \text{card query-sample-space} \wedge \text{rounds}$
unfolding *fri-query-index-list-space-def*
using *card-lists-length-eq*[*OF finite-query-sample-space, of rounds*]
by (*simp add: conj-commute*)

lemma *query-rounds-index-list-hit-None*[*simp*]:

$\neg \text{query-rounds-index-list-hit } s \text{ query-idxs } n \text{ None}$
unfolding *query-rounds-index-list-hit-def* **by** *simp*

lemma *query-rounds-index-list-hit-tail*:

assumes *head*:
 $\text{Some } ((), s1) \in$
 set-dist
 $(\text{execute}$
 $(\text{verifier-query-round-program fr f-fl f-final as fl final}) s)$
and *tail*:
 $\text{Some } (\text{results}, t) \in$
 set-dist
 $(\text{execute}$
 $(\text{ntimes}$
 $(\text{verifier-query-round-program fr f-fl f-final as fl final}) n)$
 $s1)$
and *hit*:
 $\text{query-rounds-index-list-hit } s \text{ query-idxs } (\text{Suc } n)$
 $(\text{Some } (u \# \text{results}, t))$
shows
 $\text{query-rounds-index-list-hit } s1 \text{ (tl query-idxs) } n$
 $(\text{Some } (\text{results}, t))$

proof –

from *hit* **obtain** *raw-idxs* **where**

len-raw: $\text{length raw-idxs} = \text{Suc } n$

and *len-query*: $\text{length query-idxs} = \text{Suc } n$

```

and query-eq: query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and lookups:
   $\bigwedge i. i < \text{Suc } n \implies \exists x.$ 
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i) x) =
        Some (raw-idxs ! i)
  unfolding query-rounds-index-list-hit-def by auto
have counter-s1: PQueryCounter s1 = Suc (PQueryCounter s)
  using verifier-query-round-program-outcome[OF head] by blast
let ?raw-tail = tl raw-idxs
have len-tail-raw: length ?raw-tail = n
  using len-raw by simp
have len-tail-query: length (tl query-idxs) = n
  using len-query by simp
have tail-eq:
  tl query-idxs = map ( $\lambda$ raw. index (to-nat raw)) ?raw-tail
  using query-eq by (cases raw-idxs; cases query-idxs; simp)
have tail-lookups:
   $\forall i < n. \exists x.$ 
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s1 + i) x) =
        Some (?raw-tail ! i)
proof (intro allI impI)
  fix i
  assume i-bound: i < n
  then obtain x where lookup:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + Suc i) x) =
        Some (raw-idxs ! Suc i)
  using lookups[of Suc i] by auto
have counter-eq: PQueryCounter s1 + i = PQueryCounter s + Suc i
  using counter-s1 by presburger
have tail-raw: ?raw-tail ! i = raw-idxs ! Suc i
  using i-bound len-raw by (cases raw-idxs) auto
show  $\exists x.$ 
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s1 + i) x) =
      Some (?raw-tail ! i)
  by (rule exI[where x=x])
    (use lookup counter-eq tail-raw in simp)
qed
show ?thesis
  unfolding query-rounds-index-list-hit-def
  by (intro exI[of - results] exI[of - t] exI[of - ?raw-tail] conjI)
    (use len-tail-raw len-tail-query tail-eq tail-lookups in simp-all)
qed

lemma query-rounds-index-list-hit-head-index:
  assumes future: query-future-fresh s

```

```

and head:
  Some ((), s1) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
and tail:
  Some (results, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s1)
and hit:
  query-rounds-index-list-hit s query-idxs (Suc n)
    (Some (u # results, t))
shows
  query-round-any-index-set-hit s {query-idxs ! 0} (Some ((), s1))
proof –
from head[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where
  recv: Some (raw, s0) ∈ set-dist (execute receive-query-index-challenge s)
and after:
  Some ((), s1) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program fr f-fl f-final as fl
          final raw) s0)
by (auto elim!: set-dist-bindE)
from hit obtain raw-idxs x where
  len-raw: length raw-idxs = Suc n
and len-query: length query-idxs = Suc n
and query-eq: query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) x) = Some (raw-idxs ! 0)
unfolding query-rounds-index-list-hit-def by auto
have counter-s1: PQueryCounter s1 = Suc (PQueryCounter s)
using verifier-query-round-program-outcome[OF head] by blast
have past: PQueryCounter s < PQueryCounter s1
using counter-s1 by simp
have lookup-s1:
  fmlookup (HashMap s1) (QueryIndexChallenge (PQueryCounter s) x) =
    Some (raw-idxs ! 0)
using ntimes-verifier-query-round-program-preserves-past-query-lookup
  [OF past tail, of x] lookup-t
by simp
have lookup-s0-x:
  fmlookup (HashMap s0) (QueryIndexChallenge (PQueryCounter s) x) =
    Some (raw-idxs ! 0)

```

```

using verifier-query-round-after-index-program-preserves-query-lookup
  [OF after, of PQueryCounter s x] lookup-s1
by simp
have lookup-s0-current:
  fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
using receive-query-index-challenge-outcome[OF recv] by blast
have key-eq:
  QueryIndexChallenge (PQueryCounter s) x =
    QueryIndexChallenge (PQueryCounter s) (PState s)
proof (rule ccontr)
  assume neq:
    QueryIndexChallenge (PQueryCounter s) x ≠
      QueryIndexChallenge (PQueryCounter s) (PState s)
  have lookup-old:
    fmlookup (HashMap s0) (QueryIndexChallenge (PQueryCounter s) x) =
      fmlookup (HashMap s) (QueryIndexChallenge (PQueryCounter s) x)
    by (rule receive-query-index-challenge-preserves-other-lookup
      [OF recv neq])
  have fmlookup (HashMap s) (QueryIndexChallenge (PQueryCounter s) x) =
    None
    using future unfolding query-future-fresh-def by simp
  then show False
    using lookup-s0-x lookup-old by simp
qed
have raw-eq: raw-idxs ! 0 = raw
  using lookup-s0-x lookup-s0-current key-eq by simp
have index (to-nat raw) = query-idxs ! 0
  using query-eq raw-eq len-raw by simp
then show ?thesis
  unfolding query-round-any-index-set-hit-def
  using lookup-s1 raw-eq query-eq len-raw by auto
qed

lemma wp-ntimes-verifier-query-round-program-index-list-bound:
assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and len-query: length query-idxs = n
  and subset: set query-idxs ⊆ query-sample-space
shows
  wp-event
    (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n)
    (query-rounds-index-list-hit s query-idxs n) s ≤
    (1 / nnreal (card query-sample-space)) ^ n
using future len-query subset
proof (induction n arbitrary: s query-idxs)
case 0
show ?case
  using wp-event-le-1

```

```

    [of ntimes (verifier-query-round-program fr f-fl f-final as fl final) 0
      query-rounds-index-list-hit s query-idxs 0 s]
  by simp
next
case (Suc n)
let ?prog = verifier-query-round-program fr f-fl f-final as fl final
let ?p = 1 / nnreal (card query-sample-space)
let ?Q = query-rounds-index-list-hit s query-idxs (Suc n)
let ?Head = query-round-any-index-set-hit s {query-idxs ! 0}
have query0-in: query-idxs ! 0 ∈ query-sample-space
  using Suc.premis(2,3) by (auto intro: nth-mem)
have singleton-subset: {query-idxs ! 0} ⊆ query-sample-space
  using query0-in by simp
have head-bound:
  wp-event ?prog ?Head s ≤ ?p
proof -
  have base:
    wp-event ?prog ?Head s ≤
      nnreal (card {query-idxs ! 0}) /
      nnreal (card query-sample-space)
    by (rule wp-verifier-query-round-program-any-index-set-bound
      [OF Suc.premis(1) raw-bound singleton-subset])
  then show ?thesis
    by simp
qed
show ?case
  unfolding ntimes.simps
proof (rule order-trans)
  show
    wp-event
      (?prog ≧≧ (λu. ntimes ?prog n ≧≧ (λxs. return (u # xs))))
      ?Q s
    ≤ wp-event ?prog ?Head s * (?p ^ n)
  proof (rule wp-event-bind-bound-by-head-and-cont)
    show ¬ ?Q None
      by simp
  next
    fix u s1
    assume head:
      Some (u, s1) ∈ set-dist (execute ?prog s)
      and not-head: ¬ ?Head (Some (u, s1))
    have u-eq: u = ()
      by (cases u) simp
    show
      wp-event
        (ntimes ?prog n ≧≧ (λxs. return (u # xs)))
        ?Q s1 = 0
    proof (rule ccontr)
      assume nonzero:

```

```

wp-event
  (ntimes ?prog n >>= (λxs. return (u # xs)))
  ?Q s1 ≠ 0
have positive:
  0 <
    wp-event
      (ntimes ?prog n >>= (λxs. return (u # xs)))
      ?Q s1
    using nonzero by simp
from wp-event-pos-imp-exists-support[OF positive]
obtain out where out-support:
  out ∈
    set-dist
      (execute
        (ntimes ?prog n >>= (λxs. return (u # xs))) s1)
    and hit: ?Q out
    by blast
from hit obtain ys t0 raw-idxs where out-some: out = Some (ys, t0)
    unfolding query-rounds-index-list-hit-def by blast
have ∃ results t.
  Some (results, t) ∈ set-dist (execute (ntimes ?prog n) s1) ∧
  out = Some (u # results, t)
proof (rule set-dist-bindE[OF out-support[unfolded out-some]])
  fix results t
  assume tail:
    Some (results, t) ∈ set-dist (execute (ntimes ?prog n) s1)
  and ret:
    Some (ys, t0) ∈ set-dist (execute (return (u # results)) t)
  have out-eq: out = Some (u # results, t)
    using ret out-some
    unfolding set-dist-def return.rep-eq dist-return-def
      dist-delta-dist delta-map-def
    by simp
  show ?thesis
    using tail out-eq by blast
qed
then obtain results t where
  tail:
    Some (results, t) ∈ set-dist (execute (ntimes ?prog n) s1)
  and out-eq: out = Some (u # results, t)
  by blast
have head-hit: ?Head (Some (u, s1))
  unfolding u-eq
  by (rule query-rounds-index-list-hit-head-index
    [OF Suc.prem1 head[unfolded u-eq] tail])
    (use hit out-eq u-eq in simp)
then show False
  using not-head by simp
qed

```

```

next
  fix u s1
  assume head:
    Some (u, s1) ∈ set-dist (execute ?prog s)
    and head-hit: ?Head (Some (u, s1))
  have u-eq: u = ()
    by (cases u) simp
  have future-s1: query-future-fresh s1
    by (rule verifier-query-round-program-preserves-query-future-fresh
        [OF Suc.prem1 head[unfolded u-eq]])
  have tail-len: length (tl query-idxs) = n
    using Suc.prem2 by simp
  have tail-subset: set (tl query-idxs) ⊆ query-sample-space
    using Suc.prem3 by (cases query-idxs) auto
  have tail-bound:
    wp-event (ntimes ?prog n)
      (query-rounds-index-list-hit s1 (tl query-idxs) n) s1 ≤
      ?p ^ n
    by (rule Suc.IH[OF future-s1 tail-len tail-subset])
  show
    wp-event
      (ntimes ?prog n ≫ (λxs. return (u # xs)))
      ?Q s1 ≤ ?p ^ n
  proof (rule order-trans[OF - tail-bound])
    show
      wp-event
        (ntimes ?prog n ≫ (λxs. return (u # xs)))
        ?Q s1
      ≤ wp-event (ntimes ?prog n)
        (query-rounds-index-list-hit s1 (tl query-idxs) n) s1
    proof (rule wp-event-bind-bound-by-head-event)
      show
        wp-event (ntimes ?prog n)
          (query-rounds-index-list-hit s1 (tl query-idxs) n) s1
        ≤ wp-event (ntimes ?prog n)
          (query-rounds-index-list-hit s1 (tl query-idxs) n) s1
        by simp
    next
      show
        ?Q None ⇒
          query-rounds-index-list-hit s1 (tl query-idxs) n None
        by simp
    next
      fix results t out
      assume tail:
        Some (results, t) ∈ set-dist (execute (ntimes ?prog n) s1)
        and ret:
          out ∈ set-dist (execute (return (u # results)) t)
        and hit: ?Q out

```



```

    have out-eq: out = Some (u # results, t)
      using ret
      unfolding set-dist-def return.rep-eq dist-return-def
        dist-delta-dist delta-map-def
      by simp
    show query-rounds-index-list-hit s1 (tl query-idxs) n
      (Some (results, t))
    by (rule query-rounds-index-list-hit-tail
      [OF head[unfolded u-eq] tail])
      (use hit out-eq u-eq in simp)
  qed
qed
qed
next
show wp-event ?prog ?Head s * ?p ^ n ≤ ?p ^ Suc n
  using head-bound by (simp add: mult-right-mono)
qed
qed

lemma wp-ntimes-verifier-query-round-program-index-list-set-bound:
  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and finite-Q: finite Q
  and lengths:  $\bigwedge \text{query-idxs}. \text{query-idxs} \in Q \implies \text{length } \text{query-idxs} = n$ 
  and subsets:
     $\bigwedge \text{query-idxs}. \text{query-idxs} \in Q \implies$ 
     $\text{set } \text{query-idxs} \subseteq \text{query-sample-space}$ 
  shows
    wp-event
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n)
      ( $\lambda \text{out}. \exists \text{query-idxs} \in Q.$ 
        query-rounds-index-list-hit s query-idxs n out) s ≤
      nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ n
proof -
  have
    wp-event
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n)
      ( $\lambda \text{out}. \exists \text{query-idxs} \in Q.$ 
        query-rounds-index-list-hit s query-idxs n out) s ≤
      ( $\sum \text{query-idxs} \in Q.$ 
        (1 / nnreal (card query-sample-space)) ^ n)
    by (rule wp-event-finite-UN-bound[OF finite-Q])
      (rule wp-ntimes-verifier-query-round-program-index-list-bound
        [OF future raw-bound lengths subsets])
  also have ... =
    nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ n
  using finite-Q by simp

```

finally show ?thesis .
qed

lemma *wp-ntimes-verifier-query-round-program-fri-index-list-set-bound:*

assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *subset: $Q \subseteq \text{fri-query-index-list-space}$*

shows

wp-event
(*ntimes*
(*verifier-query-round-program fr f-fl f-final as fl final*) *rounds*)
($\lambda \text{out. } \exists \text{ query-idxs} \in Q.$
query-rounds-index-list-hit s query-idxs rounds out) $s \leq$
*nnreal (card Q) **
($1 / \text{nnreal (card query-sample-space)}$) $\wedge$ *rounds*

proof –

have *finite-Q: finite Q*
using *subset* **by** (*rule finite-subset*) (*rule finite-fri-query-index-list-space*)
show ?thesis

proof (*rule wp-ntimes-verifier-query-round-program-index-list-set-bound*
[*OF future raw-bound finite-Q*])

fix *query-idxs*

assume *query-idxs* $\in Q$

then show *length query-idxs = rounds*

using *subset* **unfolding** *fri-query-index-list-space-def* **by** *auto*

next

fix *query-idxs*

assume *query-idxs* $\in Q$

then show *set query-idxs $\subseteq$ query-sample-space*

using *subset* **unfolding** *fri-query-index-list-space-def* **by** *auto*

qed

qed

lemma *generic-fri-sampled-query-query-fiber-entries-subset:*

query-index-list-entries
(*generic-fri-sampled-query-query-fiber low-degree candidate-bad*
degree-bound challenges)
 $\subseteq$ *query-sample-space*

unfolding *query-index-list-entries-def*

generic-fri-sampled-query-query-fiber-def

generic-fri-sampled-query-restricted-bad-pairs-def

fri-query-challenge-pair-query-fiber-def

fri-query-index-list-space-def

by *auto*

lemma *finite-generic-fri-sampled-query-query-fiber-entries:*

finite

(*query-index-list-entries*

(*generic-fri-sampled-query-query-fiber low-degree candidate-bad*

degree-bound challenges))
using generic-fri-sampled-query-query-fiber-entries-subset
by (rule finite-subset) simp

lemma generic-fri-sampled-query-restricted-bad-pairs-subset:
 generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad
 degree-bound
 $\subseteq$ fri-query-index-list-space $\times$
 fri-challenge-space
 (fri-round-count-for-degree-bound degree-bound)
unfolding generic-fri-sampled-query-restricted-bad-pairs-def
using generic-fri-sampled-query-bad-pair-union-subset-challenge-space
 [of low-degree candidate-bad degree-bound]
by blast

lemma finite-generic-fri-sampled-query-restricted-bad-pairs:
 finite
 (generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad
 degree-bound)
proof (rule finite-subset)
show
 generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad
 degree-bound
 $\subseteq$ fri-query-index-list-space $\times$
 fri-challenge-space
 (fri-round-count-for-degree-bound degree-bound)
by (rule generic-fri-sampled-query-restricted-bad-pairs-subset)
show
 finite
 (fri-query-index-list-space $\times$
 fri-challenge-space
 (fri-round-count-for-degree-bound degree-bound))
by (simp add: finite-fri-query-index-list-space
 finite-fri-challenge-space)

qed

lemma generic-fri-sampled-query-pair-fraction-boundD:
assumes
 generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad
 degree-bound C
shows
 nnreal
 (card
 (generic-fri-sampled-query-restricted-bad-pairs low-degree
 candidate-bad degree-bound)) /
 nnreal
 (card fri-query-index-list-space *
 CARD('f) ^ fri-round-count-for-degree-bound degree-bound)
 $\leq C$

using *assms*
unfolding *generic-fri-sampled-query-pair-fraction-bound-def* .

lemma *generic-fri-sampled-query-challenge-projection-subset:*
generic-fri-sampled-query-challenge-projection low-degree candidate-bad
degree-bound
 $\subseteq$ *fri-challenge-space*
(fri-round-count-for-degree-bound degree-bound)
unfolding *generic-fri-sampled-query-challenge-projection-def*
using *generic-fri-sampled-query-restricted-bad-pairs-subset*
[of low-degree candidate-bad degree-bound]
by *force*

lemma *generic-fri-sampled-query-query-fiber-subset:*
generic-fri-sampled-query-query-fiber low-degree candidate-bad
degree-bound challenges
 $\subseteq$ *fri-query-index-list-space*
unfolding *generic-fri-sampled-query-query-fiber-def*
generic-fri-sampled-query-restricted-bad-pairs-def
fri-query-challenge-pair-query-fiber-def
by *auto*

lemma *finite-generic-fri-sampled-query-query-fiber:*
finite
(generic-fri-sampled-query-query-fiber low-degree candidate-bad
degree-bound challenges)
using *generic-fri-sampled-query-query-fiber-subset*
[of low-degree candidate-bad degree-bound challenges]
by *(rule finite-subset) (rule finite-fri-query-index-list-space)*

lemma *generic-fri-sampled-query-query-fiber-entries-card-le:*
card
(query-index-list-entries
(generic-fri-sampled-query-query-fiber low-degree candidate-bad
degree-bound challenges))
 $\leq$
card
(generic-fri-sampled-query-query-fiber low-degree candidate-bad
*degree-bound challenges) * rounds*
by *(rule query-index-list-entries-card-le)*
(rule finite-generic-fri-sampled-query-query-fiber,
rule generic-fri-sampled-query-query-fiber-subset)

lemma *generic-fri-sampled-query-restricted-bad-pairs-as-fiber-Sigma:*
generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad
degree-bound =
 $(\lambda(challenges, query-idxs). (query-idxs, challenges))$ ‘
(SIGMA challenges:
fri-challenge-space (fri-round-count-for-degree-bound degree-bound).

```

    generic-fri-sampled-query-query-fiber low-degree candidate-bad
    degree-bound challenges)
unfolding generic-fri-sampled-query-query-fiber-def
    fri-query-challenge-pair-query-fiber-def
using generic-fri-sampled-query-restricted-bad-pairs-subset
by force

lemma generic-fri-sampled-query-restricted-bad-pairs-card-eq-fiber-sum:
  card
    (generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad
    degree-bound) =
  (∑ challenges ∈
    fri-challenge-space (fri-round-count-for-degree-bound degree-bound).
    card
      (generic-fri-sampled-query-query-fiber low-degree candidate-bad
      degree-bound challenges))

proof –
  let ?S =
    SIGMA challenges:
      fri-challenge-space (fri-round-count-for-degree-bound degree-bound).
      generic-fri-sampled-query-query-fiber low-degree candidate-bad
      degree-bound challenges
  let ?swap = λ(challenges, query-idxs). (query-idxs, challenges)
  have finite-S: finite ?S
    by (rule finite-SigmaI)
      (rule finite-fri-challenge-space,
      rule finite-generic-fri-sampled-query-query-fiber)
  have inj: inj-on ?swap ?S
    unfolding inj-on-def by auto
  have finite-challenge-space:
    finite
      (fri-challenge-space
      (fri-round-count-for-degree-bound degree-bound))
    by (rule finite-fri-challenge-space)
  have finite-fibers:
    ∧challenges. challenges ∈
      fri-challenge-space (fri-round-count-for-degree-bound degree-bound)
    ⇒
    finite
      (generic-fri-sampled-query-query-fiber low-degree candidate-bad
      degree-bound challenges)
    by (rule finite-generic-fri-sampled-query-query-fiber)
  have card-S:
    card ?S =
    (∑ challenges ∈
      fri-challenge-space (fri-round-count-for-degree-bound degree-bound).
      card
        (generic-fri-sampled-query-query-fiber low-degree candidate-bad
        degree-bound challenges))

```

```

by (rule card-SigmaI)
  (use finite-challenge-space finite-fibers in auto)
have card
  (generic-fri-sampled-query-restricted-bad-pairs low-degree
   candidate-bad degree-bound) =
  card (?swap ‘ ?S)
  unfolding generic-fri-sampled-query-restricted-bad-pairs-as-fiber-Sigma
  by simp
also have ... = card ?S
  by (rule card-image[OF inj])
also have ... =
  (∑ challenges ∈
   fri-challenge-space (fri-round-count-for-degree-bound degree-bound).
   card
   (generic-fri-sampled-query-query-fiber low-degree candidate-bad
    degree-bound challenges))
  by (rule card-S)
finally show ?thesis .
qed

```

lemma generic-fri-sampled-query-pair-fraction-bound-from-fiber-sum:

assumes fraction-bound:

```

nnreal
(∑ challenges ∈
 fri-challenge-space (fri-round-count-for-degree-bound degree-bound).
 card
 (generic-fri-sampled-query-query-fiber low-degree candidate-bad
  degree-bound challenges)) /
nnreal
(card fri-query-index-list-space *
 CARD('f) ^ fri-round-count-for-degree-bound degree-bound)
≤ C

```

shows

generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad
degree-bound C

proof –

have card-eq:

```

card
(generic-fri-sampled-query-restricted-bad-pairs low-degree
 candidate-bad degree-bound) =
(∑ challenges ∈
 fri-challenge-space (fri-round-count-for-degree-bound degree-bound).
 card
 (generic-fri-sampled-query-query-fiber low-degree candidate-bad
  degree-bound challenges))
by (rule generic-fri-sampled-query-restricted-bad-pairs-card-eq-fiber-sum)

```

show ?thesis

unfolding generic-fri-sampled-query-pair-fraction-bound-def

using fraction-bound card-eq **by** simp

qed

lemma *generic-fri-sampled-query-pair-fraction-bound-iff-fiber-sum:*

*generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad
degree-bound C*

$\longleftrightarrow$

nnreal

$(\sum_{challenges \in \text{fri-challenge-space } (\text{fri-round-count-for-degree-bound } \text{degree-bound})} \text{card}$

$(\text{generic-fri-sampled-query-query-fiber low-degree candidate-bad}$
 $\text{degree-bound challenges})) /$

nnreal

$(\text{card } \text{fri-query-index-list-space} * \text{CARD}(f) \wedge \text{fri-round-count-for-degree-bound } \text{degree-bound})$

$\leq C$

unfolding *generic-fri-sampled-query-pair-fraction-bound-def*

by (*simp add:*

generic-fri-sampled-query-restricted-bad-pairs-card-eq-fiber-sum)

lemma *trace-fri-sampled-query-restricted-bad-pairs-card-eq-fiber-sum:*

card

$(\text{trace-fri-sampled-query-bad-pair-union} \cap$
 $(\text{fri-query-index-list-space} \times \text{UNIV})) =$

$(\sum_{challenges \in \text{fri-challenge-space } (\text{fri-round-count-for-degree-bound } (\text{clength} - 1))} \text{card}$

$(\text{generic-fri-sampled-query-query-fiber trace-table-low-degree}$
 $(\text{Not} \circ \text{trace-table-low-degree}) (\text{clength} - 1) \text{ challenges}))$

proof –

have *generic:*

card

$(\text{generic-fri-sampled-query-restricted-bad-pairs trace-table-low-degree}$
 $(\text{Not} \circ \text{trace-table-low-degree}) (\text{clength} - 1)) =$

$(\sum_{challenges \in \text{fri-challenge-space } (\text{fri-round-count-for-degree-bound } (\text{clength} - 1))} \text{card}$

$(\text{generic-fri-sampled-query-query-fiber trace-table-low-degree}$
 $(\text{Not} \circ \text{trace-table-low-degree}) (\text{clength} - 1) \text{ challenges}))$

by (*rule generic-fri-sampled-query-restricted-bad-pairs-card-eq-fiber-sum*)

then show *?thesis*

unfolding *trace-fri-sampled-query-bad-pair-union-def*

generic-fri-sampled-query-restricted-bad-pairs-def

by *simp*

qed

lemma *composition-fri-sampled-query-restricted-bad-pairs-card-eq-fiber-sum:*

card

$(\text{composition-fri-sampled-query-bad-pair-union } dg \cap$

$(\text{fri-query-index-list-space} \times \text{UNIV})) =$
 $(\sum \text{challenges} \in$
 $\text{fri-challenge-space } (\text{fri-round-count-for-degree-bound } (\text{to-nat } dg)).$
 card
 $(\text{generic-fri-sampled-query-query-fiber}$
 $(\text{composition-table-low-degree } (\text{to-nat } dg))$
 $(\text{Not} \circ \text{composition-table-low-degree } \text{maxDegree}) (\text{to-nat } dg)$
 $\text{challenges}))$

proof –

have *generic*:

card
 $(\text{generic-fri-sampled-query-restricted-bad-pairs}$
 $(\text{composition-table-low-degree } (\text{to-nat } dg))$
 $(\text{Not} \circ \text{composition-table-low-degree } \text{maxDegree}) (\text{to-nat } dg)) =$
 $(\sum \text{challenges} \in$
 $\text{fri-challenge-space } (\text{fri-round-count-for-degree-bound } (\text{to-nat } dg)).$
 card
 $(\text{generic-fri-sampled-query-query-fiber}$
 $(\text{composition-table-low-degree } (\text{to-nat } dg))$
 $(\text{Not} \circ \text{composition-table-low-degree } \text{maxDegree}) (\text{to-nat } dg)$
 $\text{challenges}))$

by (rule *generic-fri-sampled-query-restricted-bad-pairs-card-eq-fiber-sum*)

then show *?thesis*

unfolding *composition-fri-sampled-query-bad-pair-union-def*
generic-fri-sampled-query-restricted-bad-pairs-def

by *simp*

qed

lemma *trace-fri-sampled-query-pair-fraction-bound-iff-fiber-sum*:

generic-fri-sampled-query-pair-fraction-bound trace-table-low-degree
 $(\text{Not} \circ \text{trace-table-low-degree}) (\text{clength} - 1) C$

$\longleftrightarrow$

nnreal
 $(\sum \text{challenges} \in$
 $\text{fri-challenge-space } (\text{fri-round-count-for-degree-bound } (\text{clength} - 1)).$
 card
 $(\text{generic-fri-sampled-query-query-fiber } \text{trace-table-low-degree}$
 $(\text{Not} \circ \text{trace-table-low-degree}) (\text{clength} - 1) \text{ challenges})) /$
 nnreal
 $(\text{card } \text{fri-query-index-list-space} *$
 $\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound } (\text{clength} - 1))$
 $\leq C$

by (rule *generic-fri-sampled-query-pair-fraction-bound-iff-fiber-sum*)

lemma *composition-fri-sampled-query-pair-fraction-bound-iff-fiber-sum*:

generic-fri-sampled-query-pair-fraction-bound
 $(\text{composition-table-low-degree } (\text{to-nat } dg))$
 $(\text{Not} \circ \text{composition-table-low-degree } \text{maxDegree}) (\text{to-nat } dg) C$

$\longleftrightarrow$

$$\frac{\text{nnreal} \left(\sum \text{challenges} \in \text{fri-challenge-space} \left(\text{fri-round-count-for-degree-bound} \left(\text{to-nat } dg \right) \right) \cdot \text{card} \left(\text{generic-fri-sampled-query-query-fiber} \left(\text{composition-table-low-degree} \left(\text{to-nat } dg \right) \right) \left(\text{Not} \circ \text{composition-table-low-degree } \text{maxDegree} \right) \left(\text{to-nat } dg \right) \text{challenges} \right) \right)}{\text{nnreal} \left(\text{card } \text{fri-query-index-list-space} * \text{CARD}(f) \wedge \text{fri-round-count-for-degree-bound} \left(\text{to-nat } dg \right) \right)} \leq C$$
by (rule generic-fri-sampled-query-pair-fraction-bound-iff-fiber-sum)

lemma *trace-fri-query-index-list-set-hit-imp-query-rounds-any-index-set-hit*:
assumes *hit*: trace-fri-query-index-list-set-hit *s* *Q* *out*
shows
query-rounds-any-index-set-hit *s* (*query-index-list-entries* *Q*) *rounds* *out*
proof –
from *hit* **obtain** *trace-roots* *trace-bs* *trace-final* *dg* *composition-roots* *composition-bs* *composition-final* *query-idxs* *trace-round-layers* *composition-round-layers* **where**
openings:
accepted-fri-opening-transcript *s* *out* *trace-roots* *trace-bs* *trace-final* *dg* *composition-roots* *composition-bs* *composition-final* *query-idxs* *trace-round-layers* *composition-round-layers*
and *query-in*: *query-idxs* $\in$ *Q*
unfolding *trace-fri-query-index-list-set-hit-def* **by** *blast*
from *openings* **obtain** *result* *final-state* *query-start-state* *rest* *raw-idxs* *query-chunks* *trailing* **where**
out-eq: *out* = *Some* (*result*, *final-state*)
and *len-raw*: *length* *raw-idxs* = *rounds*
and *query-idxs-eq*: *query-idxs* = *map* ($\lambda \text{raw. index } (\text{to-nat } \text{raw})$) *raw-idxs*
and *lookup*:
 $\bigwedge i. i < \text{rounds} \implies$
fmlookup (*HashMap* *final-state*)
(*QueryIndexChallenge* (*PQueryCounter* *s* + *i*)
(*state-after-query-chunks* *query-start-state* *query-chunks* *i*)) =
Some (*raw-idxs* ! *i*)
unfolding *accepted-fri-opening-transcript-def*
verifier-query-indices-derived-def
by *blast*
have *zero-bound*: *0* < *rounds*
by (rule *rounds-positive*)
have *query-len*: *length* *query-idxs* = *rounds*
using *query-idxs-eq* *len-raw* **by** *simp*
have *idx-in-entries*:
query-idxs ! *0* $\in$ *query-index-list-entries* *Q*

```

using query-in zero-bound query-len
unfolding query-index-list-entries-def
by (auto intro: nth-mem)
have idx-eq: query-idxs ! 0 = index (to-nat (raw-idxs ! 0))
using query-idxs-eq len-raw zero-bound by simp
show ?thesis
unfolding query-rounds-any-index-set-hit-def
apply (rule exI[where x=result])
apply (rule exI[where x=final-state])
apply (rule exI[where x=0])
apply (rule exI[where x=state-after-query-chunks query-start-state
  query-chunks 0])
apply (rule exI[where x=raw-idxs ! 0])
using out-eq zero-bound lookup[OF zero-bound] idx-in-entries idx-eq
by simp
qed

lemma composition-fri-query-index-list-set-hit-imp-query-rounds-any-index-set-hit:
  assumes hit: composition-fri-query-index-list-set-hit s Q out
  shows
    query-rounds-any-index-set-hit s (query-index-list-entries Q) rounds
    out
proof -
from hit obtain trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final query-idxs trace-round-layers
  composition-round-layers where
    openings:
      accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final dg composition-roots composition-bs composition-final
      query-idxs trace-round-layers composition-round-layers
and query-in: query-idxs ∈ Q
unfolding composition-fri-query-index-list-set-hit-def by blast
from openings obtain result final-state query-start-state rest raw-idxs
  query-chunks trailing where
    out-eq: out = Some (result, final-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq: query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup:
  ∧ i. i < rounds ⇒
    fmllookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks query-start-state query-chunks i)) =
      Some (raw-idxs ! i)
unfolding accepted-fri-opening-transcript-def
  verifier-query-indices-derived-def
by blast
have zero-bound: 0 < rounds
by (rule rounds-positive)
have query-len: length query-idxs = rounds

```

```

    using query-idxs-eq len-raw by simp
  have idx-in-entries:
    query-idxs ! 0 ∈ query-index-list-entries Q
    using query-in zero-bound query-len
    unfolding query-index-list-entries-def
    by (auto intro: nth-mem)
  have idx-eq: query-idxs ! 0 = index (to-nat (raw-idxs ! 0))
    using query-idxs-eq len-raw zero-bound by simp
  show ?thesis
    unfolding query-rounds-any-index-set-hit-def
    apply (rule exI[where x=result])
    apply (rule exI[where x=final-state])
    apply (rule exI[where x=0])
    apply (rule exI[where x=state-after-query-chunks query-start-state
      query-chunks 0])
    apply (rule exI[where x=raw-idxs ! 0])
    using out-eq zero-bound lookup[OF zero-bound] idx-in-entries idx-eq
    by simp
qed

lemma wp-trace-fri-query-index-list-set-hit-bound-from-entries:
  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and subset:  $Q \subseteq \text{fri-query-index-list-space}$ 
  shows
    wp-event verify-monad (trace-fri-query-index-list-set-hit s Q) s ≤
      nnreal rounds *
      (nnreal (card (query-index-list-entries Q)) /
        nnreal (card query-sample-space))
proof -
  have event-bound:
    wp-event verify-monad
      (query-rounds-any-index-set-hit s (query-index-list-entries Q) rounds)
    s ≤
      nnreal rounds *
      (nnreal (card (query-index-list-entries Q)) /
        nnreal (card query-sample-space))
  by (rule wp-verify-monad-query-rounds-any-index-set-bound
    [OF future raw-bound
      query-index-list-entries-subset-query-sample-space[OF subset]])
  show ?thesis
    by (rule order-trans[OF - event-bound]
      (rule wp-event-mono,
        rule trace-fri-query-index-list-set-hit-imp-query-rounds-any-index-set-hit))
qed

lemma wp-composition-fri-query-index-list-set-hit-bound-from-entries:
  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound

```

and *subset*: $Q \subseteq \text{fri-query-index-list-space}$
shows
 $\text{wp-event verify-monad } (\text{composition-fri-query-index-list-set-hit } s \ Q) \ s \leq$
 $\text{nnreal rounds} * (\text{nnreal } (\text{card } (\text{query-index-list-entries } Q)) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}))$
proof –
have *event-bound*:
 $\text{wp-event verify-monad}$
 $(\text{query-rounds-any-index-set-hit } s \ (\text{query-index-list-entries } Q) \ \text{rounds})$
 $s \leq$
 $\text{nnreal rounds} * (\text{nnreal } (\text{card } (\text{query-index-list-entries } Q)) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}))$
by (rule *wp-verify-monad-query-rounds-any-index-set-bound*
 $[\text{OF future raw-bound}$
 $\text{query-index-list-entries-subset-query-sample-space}[\text{OF subset}]])$
show *?thesis*
by (rule *order-trans*[*OF - event-bound*])
 $(\text{rule wp-event-mono},$
 $\text{rule composition-fri-query-index-list-set-hit-imp-query-rounds-any-index-set-hit})$
qed

lemma *generic-fri-sampled-query-pair-fraction-bound-mono*:
assumes
 $\text{generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad}$
 $\text{degree-bound } C$
and $C \leq D$
shows
 $\text{generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad}$
 $\text{degree-bound } D$
using *assms*
unfolding *generic-fri-sampled-query-pair-fraction-bound-def*
by (rule *order-trans*)

lemma *generic-fri-sampled-query-pair-fraction-bound-from-card*:
assumes *card-bound*:
 card
 $(\text{generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad degree-bound}) \leq K$
and *fraction-bound*:
 $\text{nnreal } K /$
 nnreal
 $(\text{card fri-query-index-list-space} * \text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound degree-bound})$
 $\leq C$
shows
 $\text{generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad}$
 $\text{degree-bound } C$

proof –
have *nnreal*
 (*card*
 (*generic-fri-sampled-query-restricted-bad-pairs low-degree*
candidate-bad degree-bound)) /
nnreal
 (*card fri-query-index-list-space* *
CARD('f) ^ fri-round-count-for-degree-bound degree-bound)
 $\leq$
nnreal K /
nnreal
 (*card fri-query-index-list-space* *
CARD('f) ^ fri-round-count-for-degree-bound degree-bound)
using *card-bound* **by** (*rule nnreal-nat-divide-right-mono*)
also have ... $\leq C$
by (*rule fraction-bound*)
finally show ?thesis
unfolding *generic-fri-sampled-query-pair-fraction-bound-def* .
qed

lemma *generic-fri-sampled-query-restricted-bad-pairs-subset-Sigma*:
assumes *projection*:
generic-fri-sampled-query-challenge-projection low-degree candidate-bad
degree-bound $\subseteq B$
shows
generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad
degree-bound
 $\subseteq$
 $(\lambda(\text{challenges}, \text{query-idxs}). (\text{query-idxs}, \text{challenges}))$ ‘
(SIGMA challenges:B.
generic-fri-sampled-query-query-fiber low-degree candidate-bad
degree-bound challenges)

proof
fix *pair*
assume *pair-mem*:
pair $\in$
generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad
degree-bound
obtain *query-idxs challenges* **where** *pair-eq*: *pair = (query-idxs, challenges)*
by (*cases pair*)
have *challenges-in*: *challenges $\in B$*
using *projection pair-mem pair-eq*
unfolding *generic-fri-sampled-query-challenge-projection-def*
by *force*
have *query-in*:
query-idxs $\in$
generic-fri-sampled-query-query-fiber low-degree candidate-bad
degree-bound challenges
using *pair-mem pair-eq*

unfolding *generic-fri-sampled-query-query-fiber-def*
fri-query-challenge-pair-query-fiber-def
by *simp*
have $(challenges, query-idxs) \in$
 $(\text{SIGMA } challenges:B.$
 $generic-fri-sampled-query-query-fiber \text{ low-degree candidate-bad}$
 $degree-bound challenges)$
using *challenges-in query-in by simp*
then show
 $pair \in$
 $(\lambda(challenges, query-idxs). (query-idxs, challenges)) \text{ ‘}$
 $(\text{SIGMA } challenges:B.$
 $generic-fri-sampled-query-query-fiber \text{ low-degree candidate-bad}$
 $degree-bound challenges)$
using *pair-eq by force*
qed

lemma *generic-fri-sampled-query-restricted-bad-pairs-card-le-from-fibers:*
assumes *finite-B: finite B*
and *projection:*
 $generic-fri-sampled-query-challenge-projection \text{ low-degree}$
 $candidate-bad degree-bound \subseteq B$
and *fiber-bound:*
 $\bigwedge challenges. challenges \in B \implies$
 $card$
 $(generic-fri-sampled-query-query-fiber \text{ low-degree candidate-bad}$
 $degree-bound challenges) \leq K$
shows
 $card$
 $(generic-fri-sampled-query-restricted-bad-pairs \text{ low-degree}$
 $candidate-bad degree-bound)$
 $\leq card B * K$
proof –
have *subset:*
 $generic-fri-sampled-query-restricted-bad-pairs \text{ low-degree candidate-bad}$
 $degree-bound$
 $\subseteq$
 $(\lambda(challenges, query-idxs). (query-idxs, challenges)) \text{ ‘}$
 $(\text{SIGMA } challenges:B.$
 $generic-fri-sampled-query-query-fiber \text{ low-degree candidate-bad}$
 $degree-bound challenges)$
by $(rule \text{ generic-fri-sampled-query-restricted-bad-pairs-subset-Sigma}$
 $[OF \text{ projection}])$
have *finite-fibers:*
 $\bigwedge challenges. challenges \in B \implies$
 $finite$
 $(generic-fri-sampled-query-query-fiber \text{ low-degree candidate-bad}$
 $degree-bound challenges)$
by $(rule \text{ finite-generic-fri-sampled-query-query-fiber})$

```

have card-sigma:
  card
    (SIGMA challenges:B.
      generic-fri-sampled-query-query-fiber low-degree candidate-bad
        degree-bound challenges)
    ≤ card B * K
  using finite-B finite-fibers fiber-bound
proof (induction rule: finite-induct)
  case empty
  then show ?case by simp
next
  case (insert challenges B)
  have fiber-le:
    card
      (generic-fri-sampled-query-query-fiber low-degree candidate-bad
        degree-bound challenges) ≤ K
    by (rule insert.prem(2)) simp
  have tail-le:
    card
      (SIGMA challenges:B.
        generic-fri-sampled-query-query-fiber low-degree candidate-bad
          degree-bound challenges)
    ≤ card B * K
    by (rule insert.IH) (use insert.prem in auto)
  have card
    (SIGMA challenges:insert challenges B.
      generic-fri-sampled-query-query-fiber low-degree candidate-bad
        degree-bound challenges)
    =
    card
      (generic-fri-sampled-query-query-fiber low-degree candidate-bad
        degree-bound challenges) +
    card
      (SIGMA challenges:B.
        generic-fri-sampled-query-query-fiber low-degree candidate-bad
          degree-bound challenges)
    using insert.hyps insert.prem by simp
  also have ... ≤ K + card B * K
    using fiber-le tail-le by simp
  also have ... = card (insert challenges B) * K
    using insert.hyps by simp
  finally show ?case .
qed
have finite-sigma:
  finite
    (SIGMA challenges:B.
      generic-fri-sampled-query-query-fiber low-degree candidate-bad
        degree-bound challenges)
  by (rule finite-SigmaI[OF finite-B finite-fibers])

```

then have *finite-swap*:
finite
 $((\lambda(challenges, query-idxs). (query-idxs, challenges)) \text{ ‘ } (SIGMA \text{ challenges:} B. \text{ generic-fri-sampled-query-query-fiber low-degree candidate-bad degree-bound challenges}))$
by *simp*
have *card*
 $(generic-fri-sampled-query-restricted-bad-pairs \text{ low-degree candidate-bad degree-bound})$
 $\leq$
card
 $((\lambda(challenges, query-idxs). (query-idxs, challenges)) \text{ ‘ } (SIGMA \text{ challenges:} B. \text{ generic-fri-sampled-query-query-fiber low-degree candidate-bad degree-bound challenges}))$
by (rule *card-mono*[*OF finite-swap subset*])
also have $\dots \leq$
card
 $(SIGMA \text{ challenges:} B. \text{ generic-fri-sampled-query-query-fiber low-degree candidate-bad degree-bound challenges})$
by (rule *card-image-le*[*OF finite-sigma*])
also have $\dots \leq \text{card } B * K$
by (rule *card-sigma*)
finally show *?thesis* .
qed

lemma *generic-fri-sampled-query-pair-fraction-bound-from-projection-and-fibers*:
assumes *finite-B*: *finite B*
and *projection*:
 $generic-fri-sampled-query-challenge-projection \text{ low-degree candidate-bad degree-bound } \subseteq B$
and *fiber-bound*:
 $\bigwedge challenges. challenges \in B \implies$
card
 $(generic-fri-sampled-query-query-fiber \text{ low-degree candidate-bad degree-bound challenges}) \leq K$
and *fraction-bound*:
 $nnreal (\text{card } B * K) /$
 $nnreal$
 $(\text{card } fri\text{-query-index-list-space} * \text{CARD('f)} \wedge fri\text{-round-count-for-degree-bound degree-bound})$
 $\leq C$
shows
 $generic-fri-sampled-query-pair-fraction-bound \text{ low-degree candidate-bad degree-bound } C$
proof (rule *generic-fri-sampled-query-pair-fraction-bound-from-card*)
show


```

card
  (generic-fri-sampled-query-restricted-bad-pairs low-degree
   candidate-bad degree-bound)
≤ card B * K
by (rule generic-fri-sampled-query-restricted-bad-pairs-card-le-from-fibers
    [OF finite-B projection fiber-bound])
show
  nnreal (card B * K) /
    nnreal
      (card fri-query-index-list-space *
       CARD('f) ^ fri-round-count-for-degree-bound degree-bound)
    ≤ C
by (rule fraction-bound)
qed

end

end

```

```

theory Soundness-FRI-Query-Exact-Verifier
imports Soundness-FRI-Query-Aware-Interface
begin

```

Exact verifier-level consumption of sampled query-index lists.

The query-aware interface defines the strict query-list events and proves exact bounds for the raw verifier query-round sequence. This narrow layer lifts those bounds through the verifier decomposition.

```

context soundness
begin

```

```

lemma query-rounds-index-list-hit-counter-eq:
assumes counter: PQueryCounter t = PQueryCounter s
and hit: query-rounds-index-list-hit s query-idxs n out
shows query-rounds-index-list-hit t query-idxs n out
using assms unfolding query-rounds-index-list-hit-def by auto

```

```

lemma wp-verifier-after-composition-fri-query-index-list-set-bound:
assumes future: query-future-fresh t
and counter: PQueryCounter t = PQueryCounter s
and raw-bound: query-index-raw-preimage-bound
and subset: Q ⊆ fri-query-index-list-space
shows
  wp-event (verifier-after-composition-fri header)
    (λout. ∃ query-idxs ∈ Q.
      query-rounds-index-list-hit s query-idxs rounds out) t ≤
    nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds
proof –

```

```

obtain fr f-fl f-final as dg fl where header-eq:
  header = (fr, f-fl, f-final, as, dg, fl)
  by (cases header) auto
have after-eq:
  verifier-after-composition-fri header =
    (read  $\gg$ 
      ( $\lambda$ final.
        ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds))
  unfolding header-eq verifier-after-composition-fri-def by simp
show ?thesis
  unfolding after-eq
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg (\exists \text{ query-idxs} \in Q.$ 
    query-rounds-index-list-hit s query-idxs rounds None)
  by simp
next
  fix final query-state
  assume read-final:
    Some (final, query-state)  $\in$  set-dist (execute read t)
  have future-query: query-future-fresh query-state
    by (rule read-preserves-query-future-fresh[OF future read-final])
  from read-outcome[OF read-final] obtain rest where
    counter-query-t: PQueryCounter query-state = PQueryCounter t
  by blast
  have counter-query: PQueryCounter query-state = PQueryCounter s
    using counter-query-t counter by simp
  have event-mono:
    wp-event
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      ( $\lambda$ out.  $\exists \text{ query-idxs} \in Q.$ 
        query-rounds-index-list-hit s query-idxs rounds out)
      query-state  $\leq$ 
    wp-event
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      ( $\lambda$ out.  $\exists \text{ query-idxs} \in Q.$ 
        query-rounds-index-list-hit query-state query-idxs rounds out)
      query-state
  proof (rule wp-event-mono)
  fix out
  assume  $\exists \text{ query-idxs} \in Q.$ 
    query-rounds-index-list-hit s query-idxs rounds out
  then obtain query-idxs where
    query-in: query-idxs  $\in$  Q

```

```

    and hit: query-rounds-index-list-hit s query-idxs rounds out
    by blast
  have query-rounds-index-list-hit query-state query-idxs rounds out
    by (rule query-rounds-index-list-hit-counter-eq[OF counter-query hit])
  then show  $\exists \text{ query-idxs} \in Q.$ 
    query-rounds-index-list-hit query-state query-idxs rounds out
    using query-in by blast
qed
also have ...  $\leq$ 
  nnreal (card Q) *
  (1 / nnreal (card query-sample-space))  $\wedge$  rounds
  by (rule wp-ntimes-verifier-query-round-program-fri-index-list-set-bound
    [OF future-query raw-bound subset])
finally show
  wp-event
  (ntimes
    (verifier-query-round-program fr f-fl f-final as fl final)
    rounds)
  ( $\lambda \text{out}.$   $\exists \text{ query-idxs} \in Q.$ 
    query-rounds-index-list-hit s query-idxs rounds out)
  query-state  $\leq$ 
  nnreal (card Q) *
  (1 / nnreal (card query-sample-space))  $\wedge$  rounds .
qed
qed

lemma wp-verify-monad-query-index-list-set-bound:
  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and subset:  $Q \subseteq \text{fri-query-index-list-space}$ 
  shows
    wp-event verify-monad
    ( $\lambda \text{out}.$   $\exists \text{ query-idxs} \in Q.$ 
      query-rounds-index-list-hit s query-idxs rounds out) s  $\leq$ 
    nnreal (card Q) *
    (1 / nnreal (card query-sample-space))  $\wedge$  rounds
  unfolding verify-monad-composition-fri-decomposition
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg (\exists \text{ query-idxs} \in Q.$ 
    query-rounds-index-list-hit s query-idxs rounds None)
  by simp
next
  fix header t
  assume prefix:
    Some (header, t)  $\in$ 
    set-dist (execute verifier-composition-fri-prefix s)
  obtain fr f-fl f-final as dg fl where header-eq:
    header = (fr, f-fl, f-final, as, dg, fl)
  by (cases header) auto

```

```

have future-t: query-future-fresh t
  by (rule verifier-composition-fri-prefix-preserves-query-future-fresh
    [OF future prefix])
have counter-t: PQueryCounter t = PQueryCounter s
  using verifier-composition-fri-prefix-outcome[OF prefix[unfolded header-eq]]
  by simp
show
  wp-event (verifier-after-composition-fri header)
    ( $\lambda out. \exists query\text{-}idxs \in Q.$ 
       $query\text{-}rounds\text{-}index\text{-}list\text{-}hit\ s\ query\text{-}idxs\ rounds\ out$ )  $t \leq$ 
       $nnreal\ (card\ Q) *$ 
       $(1 / nnreal\ (card\ query\text{-}sample\text{-}space)) \wedge rounds$ 
    )
  by (rule wp-verifier-after-composition-fri-query-index-list-set-bound
    [OF future-t counter-t raw-bound subset])
qed

end

end

```

```

theory Soundness-FRI-Query-Exact-Bounds
  imports Soundness-FRI-Query-Exact-Verifier
begin

```

Exact sampled-query bounds for the FRI query-list events.

This layer connects the verifier-level query-index-list consumption theorem to the trace/composition sampled FRI events. It stays separate from the raw verifier decomposition to keep theory finalization predictable.

```

context soundness
begin

```

```

lemma wp-event-finite-union-bound-fri-exact:
  fixes  $B :: 'i \Rightarrow prob$ 
  assumes finite: finite I
  and bounds:  $\bigwedge i. i \in I \implies wp\text{-}event\ m\ (E\ i)\ s \leq B\ i$ 
  shows
     $wp\text{-}event\ m\ (\lambda out. \exists i \in I. E\ i\ out)\ s \leq (\sum i \in I. B\ i)$ 
  using finite bounds
proof (induction I)
  case empty
  show ?case
    unfolding wp-event-def wp-def dist-expect-def by simp
next
  case (insert i I)
  have split:
     $wp\text{-}event\ m\ (\lambda out. \exists j \in insert\ i\ I. E\ j\ out)\ s \leq$ 
     $wp\text{-}event\ m\ (E\ i)\ s +$ 
     $wp\text{-}event\ m\ (\lambda out. \exists j \in I. E\ j\ out)\ s$ 

```

proof –

have $wp\text{-event } m (\lambda out. \exists j \in insert\ i\ I. E\ j\ out) s \leq$
 $wp\text{-event } m (\lambda out. E\ i\ out \vee (\exists j \in I. E\ j\ out)) s$
by $(rule\ wp\text{-event}\text{-mono})\ auto$

also have $\dots \leq$
 $wp\text{-event } m (E\ i) s +$
 $wp\text{-event } m (\lambda out. \exists j \in I. E\ j\ out) s$
by $(rule\ wp\text{-event}\text{-union}\text{-bound})$

finally show $?thesis$.

qed

have $tail$:

$wp\text{-event } m (\lambda out. \exists j \in I. E\ j\ out) s \leq (\sum j \in I. B\ j)$
by $(rule\ insert.IH)\ (use\ insert.prem\ in\ blast)$

have $wp\text{-event } m (E\ i) s +$
 $wp\text{-event } m (\lambda out. \exists j \in I. E\ j\ out) s \leq$
 $B\ i + (\sum j \in I. B\ j)$
by $(intro\ add\text{-mono}\ tail\ insert.prem\ simp)$

also have $\dots = (\sum j \in insert\ i\ I. B\ j)$
using $insert.hyps$ **by** $simp$

finally show $?case$
by $(rule\ order\text{-trans}[OF\ split])$

qed

lemma *weighted-query-fiber-sum-eq*:

fixes $I :: 'i\ set$ **and** $f :: 'i \Rightarrow nat$

assumes fin : $finite\ I$
and $A\text{-pos}$: $0 < A$
and $B\text{-pos}$: $0 < B$

shows
 $(\sum x \in I. (1 / nnreal\ A) * ((1 / nnreal\ B) ^ rounds * nnreal\ (f\ x))) =$
 $nnreal\ (\sum x \in I. f\ x) / (nnreal\ A * nnreal\ B ^ rounds)$

proof –

have $sum\text{-nnreal}$:

$(\sum x \in I. nnreal\ (f\ x)) = nnreal\ (\sum x \in I. f\ x)$
using fin **by** $induction\ simp\text{-all}$

have
 $(\sum x \in I. (1 / nnreal\ A) * ((1 / nnreal\ B) ^ rounds * nnreal\ (f\ x))) =$
 $(\sum x \in I. nnreal\ (f\ x) / (nnreal\ A * nnreal\ B ^ rounds))$
using $A\text{-pos}\ B\text{-pos}$
by $(intro\ sum.cong\ refl)$
 $(transfer,\ simp\ add:\ field\text{-simps}\ power\text{-divide})$

also have $\dots =$
 $(\sum x \in I. nnreal\ (f\ x)) / (nnreal\ A * nnreal\ B ^ rounds)$
by $(rule\ sum\text{-divide}\text{-nnreal}[OF\ fin])$

also have $\dots =$
 $nnreal\ (\sum x \in I. f\ x) / (nnreal\ A * nnreal\ B ^ rounds)$
by $(simp\ add:\ sum\text{-nnreal})$

finally show ?thesis .
qed

lemma *accepted-fri-opening-transcript-imp-query-rounds-index-list-hit*:
assumes *openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers
shows *query-rounds-index-list-hit s query-idxs rounds out*
proof –
from *openings* **obtain** *result final-state query-start-state rest raw-idxs*
query-chunks trailing **where**
out-eq: *out = Some (result, final-state)*
and *len-raw*: *length raw-idxs = rounds*
and *query-idxs-eq*: *query-idxs = map (λraw. index (to-nat raw)) raw-idxs*
and *lookup*:
 $\bigwedge i. i < rounds \implies$
fmlookup (HashMap final-state)
(QueryIndexChallenge (PQueryCounter s + i)
(state-after-query-chunks query-start-state query-chunks i)) =
Some (raw-idxs ! i)
unfolding *accepted-fri-opening-transcript-def*
verifier-query-indices-derived-def
by *blast*
show ?thesis
unfolding *query-rounds-index-list-hit-def*
apply (rule *exI*[**where** *x=result*])
apply (rule *exI*[**where** *x=final-state*])
apply (rule *exI*[**where** *x=raw-idxs*])
using *out-eq len-raw query-idxs-eq lookup*
by *auto*
qed

lemma *trace-fri-query-index-list-set-hit-imp-query-rounds-index-list-hit*:
assumes *hit*: *trace-fri-query-index-list-set-hit s Q out*
shows
 $\exists query-idxs \in Q.$
query-rounds-index-list-hit s query-idxs rounds out
proof –
from *hit* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final query-idxs trace-round-layers
composition-round-layers **where**
openings:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
query-idxs trace-round-layers composition-round-layers
and *query-in*: *query-idxs ∈ Q*
unfolding *trace-fri-query-index-list-set-hit-def* **by** *blast*
have *query-rounds-index-list-hit s query-idxs rounds out*

by (rule accepted-fri-opening-transcript-imp-query-rounds-index-list-hit
 [OF openings])
 then show ?thesis
 using query-in by blast
 qed

lemma composition-fri-query-index-list-set-hit-imp-query-rounds-index-list-hit:
 assumes hit: composition-fri-query-index-list-set-hit s Q out
 shows
 $\exists \text{ query-idxs} \in Q.$
 query-rounds-index-list-hit s query-idxs rounds out
proof –
 from hit obtain trace-roots trace-bs trace-final dg composition-roots
 composition-bs composition-final query-idxs trace-round-layers
 composition-round-layers where
 openings:
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final dg composition-roots composition-bs composition-final
 query-idxs trace-round-layers composition-round-layers
 and query-in: query-idxs $\in Q$
 unfolding composition-fri-query-index-list-set-hit-def by blast
 have query-rounds-index-list-hit s query-idxs rounds out
 by (rule accepted-fri-opening-transcript-imp-query-rounds-index-list-hit
 [OF openings])
 then show ?thesis
 using query-in by blast
 qed

lemma wp-trace-fri-query-index-list-set-hit-exact-bound:
 assumes future: query-future-fresh s
 and raw-bound: query-index-raw-preimage-bound
 and subset: $Q \subseteq \text{fri-query-index-list-space}$
 shows
 $\text{wp-event verify-monad } (\text{trace-fri-query-index-list-set-hit s } Q) \text{ s} \leq$
 $\text{nnreal (card } Q) * (1 / \text{nnreal (card query-sample-space)}) \wedge \text{rounds}$
proof –
 have event-bound:
 wp-event verify-monad
 ($\lambda \text{out. } \exists \text{ query-idxs} \in Q.$
 query-rounds-index-list-hit s query-idxs rounds out) s $\leq$
 $\text{nnreal (card } Q) * (1 / \text{nnreal (card query-sample-space)}) \wedge \text{rounds}$
 by (rule wp-verify-monad-query-index-list-set-bound
 [OF future raw-bound subset])
 show ?thesis
 by (rule order-trans[OF - event-bound])
 (rule wp-event-mono,
 rule trace-fri-query-index-list-set-hit-imp-query-rounds-index-list-hit)

qed

lemma *wp-composition-fri-query-index-list-set-hit-exact-bound:*

assumes *future: query-future-fresh s*

and *raw-bound: query-index-raw-preimage-bound*

and *subset: $Q \subseteq \text{fri-query-index-list-space}$*

shows

wp-event verify-monad (composition-fri-query-index-list-set-hit s Q) s ≤
*nnreal (card Q) **
(1 / nnreal (card query-sample-space)) ^ rounds

proof –

have *event-bound:*

wp-event verify-monad

(λout. ∃ query-idxs ∈ Q.

query-rounds-index-list-hit s query-idxs rounds out) s ≤

*nnreal (card Q) **

(1 / nnreal (card query-sample-space)) ^ rounds

by (*rule wp-verify-monad-query-index-list-set-bound*
[OF future raw-bound subset])

show *?thesis*

by (*rule order-trans[OF - event-bound]*)

(rule wp-event-mono,

rule composition-fri-query-index-list-set-hit-imp-query-rounds-index-list-hit)

qed

lemma *wp-trace-fri-query-challenge-pair-set-hit-bound-from-query-projection:*

assumes *future: query-future-fresh s*

and *raw-bound: query-index-raw-preimage-bound*

and *projection: fst ‘ $P \subseteq Q$*

and *subset: $Q \subseteq \text{fri-query-index-list-space}$*

shows

wp-event verify-monad (trace-fri-query-challenge-pair-set-hit s P) s ≤
*nnreal (card Q) **
(1 / nnreal (card query-sample-space)) ^ rounds

proof –

have *event-bound:*

wp-event verify-monad (trace-fri-query-index-list-set-hit s Q) s ≤

*nnreal (card Q) **

(1 / nnreal (card query-sample-space)) ^ rounds

by (*rule wp-trace-fri-query-index-list-set-hit-exact-bound*
[OF future raw-bound subset])

show *?thesis*

by (*rule order-trans[OF - event-bound]*)

(rule wp-event-mono,

rule trace-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit
[OF - projection])

qed

lemma

wp-composition-fri-query-challenge-pair-set-hit-bound-from-query-projection:
assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *projection: $\bigwedge dg. \text{fst } 'P \text{ dg} \subseteq Q$*
and *subset: $Q \subseteq \text{fri-query-index-list-space}$*
shows
wp-event verify-monad
(composition-fri-query-challenge-pair-set-hit s P) s ≤
*nnreal (card Q) **
(1 / nnreal (card query-sample-space)) ^ rounds
proof –
have *event-bound:*
wp-event verify-monad (composition-fri-query-index-list-set-hit s Q) s ≤
≤
*nnreal (card Q) **
(1 / nnreal (card query-sample-space)) ^ rounds
by (rule *wp-composition-fri-query-index-list-set-hit-exact-bound*
[*OF future raw-bound subset*])
show ?thesis
by (rule *order-trans[OF - event-bound]*)
(rule *wp-event-mono,*
rule *composition-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit*
[*OF - projection*])
qed

lemma *wp-trace-fri-sampled-query-pair-set-hit-bound-from-query-projection:*
assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *projection:*
 $\bigwedge \text{trace-table roots final round-layers layers.}$
 $\text{fst } 'R \text{ trace-table roots final round-layers layers} \subseteq Q$
and *subset: $Q \subseteq \text{fri-query-index-list-space}$*
shows
wp-event verify-monad (trace-fri-sampled-query-pair-set-hit R s) s ≤
*nnreal (card Q) **
(1 / nnreal (card query-sample-space)) ^ rounds
proof –
have *event-bound:*
wp-event verify-monad (trace-fri-query-index-list-set-hit s Q) s ≤
*nnreal (card Q) **
(1 / nnreal (card query-sample-space)) ^ rounds
by (rule *wp-trace-fri-query-index-list-set-hit-exact-bound*
[*OF future raw-bound subset*])
show ?thesis
by (rule *order-trans[OF - event-bound]*)
(rule *wp-event-mono,*
rule *trace-fri-sampled-query-pair-set-hit-imp-query-index-list-set-hit*
[*OF - projection*])
qed

lemma *wp-composition-fri-sampled-query-pair-set-hit-bound-from-query-projection:*

assumes *future: query-future-fresh s*

and *raw-bound: query-index-raw-preimage-bound*

and *projection:*

$\bigwedge dg$ *composition-table roots final round-layers layers.*

fst ‘ R dg composition-table roots final round-layers layers $\subseteq Q$

and *subset: $Q \subseteq \text{fri-query-index-list-space}$*

shows

wp-event verify-monad

(composition-fri-sampled-query-pair-set-hit R s) s $\leq$

*nnreal (card Q) **

(1 / nnreal (card query-sample-space)) $\wedge$ rounds

proof –

have *event-bound:*

wp-event verify-monad (composition-fri-query-index-list-set-hit s Q) s

$\leq$

*nnreal (card Q) **

(1 / nnreal (card query-sample-space)) $\wedge$ rounds

by *(rule wp-composition-fri-query-index-list-set-hit-exact-bound*

[OF future raw-bound subset])

show *?thesis*

by *(rule order-trans[OF - event-bound])*

(rule wp-event-mono,

rule composition-fri-sampled-query-pair-set-hit-imp-query-index-list-set-hit

[OF - projection])

qed

lemma *wp-verifier-after-trace-fri-query-index-list-set-bound:*

assumes *future: query-future-fresh t*

and *counter: PQueryCounter t = PQueryCounter s*

and *raw-bound: query-index-raw-preimage-bound*

and *subset: $Q \subseteq \text{fri-query-index-list-space}$*

shows

wp-event (verifier-after-trace-fri header)

($\lambda out. \exists \text{query-idxs} \in Q.$

query-rounds-index-list-hit s query-idxs rounds out) t $\leq$

*nnreal (card Q) **

(1 / nnreal (card query-sample-space)) $\wedge$ rounds

proof –

obtain *fr f-fl where header-eq: header = (fr, f-fl)*

by *(cases header) auto*

let *?E =*

$\lambda out. \exists \text{query-idxs} \in Q.$

query-rounds-index-list-hit s query-idxs rounds out

let *?C =*

*nnreal (card Q) **

(1 / nnreal (card query-sample-space)) $\wedge$ rounds

have *after-eq:*

```

verifier-after-trace-fri header =
  (read >>=
    (λf-final.
      mmap (replicate (length spec) alpha-round) >>=
        (λas. read >>=
          (λdg. assert (to-nat dg ≤ maxDegree) >>=
            (λ-. ntimes receive-composition-fri-commits
              (ceil-log (to-nat dg + 1)) >>=
                (λfl. read >>=
                  (λfinal.
                    ntimes
                      (verifier-query-round-program fr f-fl f-final as fl
                        final)
                    rounds)))))))
  unfolding header-eq verifier-after-trace-fri-def by simp
show ?thesis
  unfolding after-eq
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?E None
  by simp
next
fix f-final s1
assume read-f-final: Some (f-final, s1) ∈ set-dist (execute read t)
have future-s1: query-future-fresh s1
  by (rule read-preserves-query-future-fresh[OF future read-f-final])
from read-outcome[OF read-f-final] obtain rest1 where
  counter-s1-t: PQueryCounter s1 = PQueryCounter t
  by blast
have counter-s1: PQueryCounter s1 = PQueryCounter s
  using counter-s1-t counter by simp
show
  wp-event
    (mmap (replicate (length spec) alpha-round) >>=
      (λas. read >>=
        (λdg. assert (to-nat dg ≤ maxDegree) >>=
          (λ-. ntimes receive-composition-fri-commits
            (ceil-log (to-nat dg + 1)) >>=
              (λfl. read >>=
                (λfinal.
                  ntimes
                    (verifier-query-round-program fr f-fl f-final as fl
                      final)
                  rounds))))))) ?E s1 ≤ ?C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?E None
  by simp
next
fix as s2
assume alpha-out:

```

```

Some (as, s2) ∈
  set-dist
  (execute (mmap (replicate (length spec) alpha-round)) s1)
have future-s2: query-future-fresh s2
  by (rule mmap-alpha-round-preserves-query-future-fresh
    [OF future-s1 alpha-out])
have counter-s2-s1: PQueryCounter s2 = PQueryCounter s1
  using mmap-alpha-round-outcome[OF alpha-out] by simp
have counter-s2: PQueryCounter s2 = PQueryCounter s
  using counter-s2-s1 counter-s1 by simp
show
  wp-event
  (read >>=
    (λdg. assert (to-nat dg ≤ maxDegree) >>=
      (λ-. ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1)) >>=
          (λfl. read >>=
            (λfinal.
              ntimes
                (verifier-query-round-program fr f-fl f-final as fl
                  final)
              rounds)))))) ?E s2 ≤ ?C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?E None
  by simp
next
  fix dg s3
  assume read-dg: Some (dg, s3) ∈ set-dist (execute read s2)
  have future-s3: query-future-fresh s3
  by (rule read-preserves-query-future-fresh[OF future-s2 read-dg])
  from read-outcome[OF read-dg] obtain rest3 where
    counter-s3-s2: PQueryCounter s3 = PQueryCounter s2
  by blast
  have counter-s3: PQueryCounter s3 = PQueryCounter s
  using counter-s3-s2 counter-s2 by simp
  show
    wp-event
    (assert (to-nat dg ≤ maxDegree) >>=
      (λ-. ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1)) >>=
          (λfl. read >>=
            (λfinal.
              ntimes
                (verifier-query-round-program fr f-fl f-final as fl
                  final)
              rounds)))))) ?E s3 ≤ ?C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?E None
  by simp

```

```

next
  fix unit s4
  assume assert-out:
    Some (unit, s4) ∈
      set-dist (execute (assert (to-nat dg ≤ maxDegree)) s3)
  have s4-eq: s4 = s3
    using assert-out unfolding assert-def
    by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
  have future-s4: query-future-fresh s4
    using future-s3 s4-eq by simp
  have counter-s4: PQueryCounter s4 = PQueryCounter s
    using counter-s3 s4-eq by simp
  show
    wp-event
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1)) >>=
        (λfl. read >>=
          (λfinal.
            ntimes
              (verifier-query-round-program fr f-fl f-final as fl
                final)
            rounds))) ?E s4 ≤ ?C
  proof (rule wp-event-bind-bound-by-cont)
    show ¬ ?E None
      by simp
  next
    fix fl s5
    assume comp-fri:
      Some (fl, s5) ∈
        set-dist
          (execute
            (ntimes receive-composition-fri-commits
              (ceil-log (to-nat dg + 1))) s4)
    have future-s5: query-future-fresh s5
      by (rule ntimes-receive-composition-fri-commits-preserves-query-future-fresh
        [OF comp-fri future-s4])
    have counter-s5-s4: PQueryCounter s5 = PQueryCounter s4
      using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
      by simp
    have counter-s5: PQueryCounter s5 = PQueryCounter s
      using counter-s5-s4 counter-s4 by simp
    show
      wp-event
        (read >>=
          (λfinal.
            ntimes
              (verifier-query-round-program fr f-fl f-final as fl
                final)
            rounds))) ?E s5 ≤ ?C

```

```

proof (rule wp-event-bind-bound-by-cont)
  show  $\neg ?E \text{ None}$ 
    by simp
next
  fix final query-state
  assume read-final:
    Some (final, query-state)  $\in$  set-dist (execute read s5)
  have future-query: query-future-fresh query-state
    by (rule read-preserves-query-future-fresh
      [OF future-s5 read-final])
  from read-outcome[OF read-final] obtain rest where
    counter-query-s5:
      PQueryCounter query-state = PQueryCounter s5
  by blast
  have counter-query:
    PQueryCounter query-state = PQueryCounter s
  using counter-query-s5 counter-s5 by simp
  have event-mono:
    wp-event
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl
          final)
        rounds)
    ?E query-state  $\leq$ 
    wp-event
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl
          final)
        rounds)
    ( $\lambda$ out.  $\exists$  query-idxs  $\in$  Q.
      query-rounds-index-list-hit query-state query-idxs
        rounds out)
    query-state
  proof (rule wp-event-mono)
    fix out
    assume  $\exists$  query-idxs  $\in$  Q.
      query-rounds-index-list-hit s query-idxs rounds out
    then obtain query-idxs where
      query-in: query-idxs  $\in$  Q
    and hit:
      query-rounds-index-list-hit s query-idxs rounds out
    by blast
  have
    query-rounds-index-list-hit query-state query-idxs rounds
      out
  by (rule query-rounds-index-list-hit-counter-eq
    [OF counter-query hit])
  then show  $\exists$  query-idxs  $\in$  Q.
    query-rounds-index-list-hit query-state query-idxs rounds

```

```

      out
      using query-in by blast
    qed
  also have ...  $\leq$  ?C
    by (rule wp-ntimes-verifier-query-round-program-fri-index-list-set-bound
        [OF future-query raw-bound subset])
  finally show
    wp-event
    (ntimes
      (verifier-query-round-program fr f-fl f-final as fl
        final)
      rounds)
    ?E query-state  $\leq$  ?C .
  qed
qed
qed
qed
qed
qed
qed
qed

```

lemma *verifier-trace-fri-prefix-preserves-query-future-fresh-exact:*
assumes *future: query-future-fresh s*
and *prefix:*
Some (header, t) $\in$ set-dist (execute verifier-trace-fri-prefix s)
shows *query-future-fresh t*
proof –
obtain *fr f-fl* **where** *header-eq: header = (fr, f-fl)*
by (cases header) auto
from *prefix[unfolded verifier-trace-fri-prefix-def header-eq]*
obtain *s1* **where**
read-fr: Some (fr, s1) $\in$ set-dist (execute read s)
and *trace-fri:*
Some (f-fl, t) $\in$
set-dist
(execute (ntimes receive-trace-fri-commits (ceil-log clength))
s1)
by (auto elim!: set-dist-bindE)
have *future-s1: query-future-fresh s1*
by (rule read-preserves-query-future-fresh[OF future read-fr])
show ?thesis
by (rule ntimes-receive-trace-fri-commits-preserves-query-future-fresh
 [OF trace-fri future-s1])
qed

lemma *wp-trace-fri-query-challenge-pair-set-hit-fixed-challenge-bound:*
assumes *trace-future: trace-fri-future-fresh s*
and *query-future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*

```

and challenges-space:
  challenges  $\in$  fri-challenge-space (ceil-log clength)
and projection:
  fst ' ( $P \cap (UNIV \times \{challenges\})$ )  $\subseteq Q$ 
and subset:  $Q \subseteq$  fri-query-index-list-space
shows
  wp-event verify-monad
    ( $\lambda out. trace-fri-challenge-list-set-hit\ s\ \{challenges\}\ out \wedge$ 
      trace-fri-query-challenge-pair-set-hit  $s\ P\ out$ )  $s \leq$ 
    ( $1 / nnreal\ (CARD('f) \wedge ceil-log\ clength)$ ) *
    ( $nnreal\ (card\ Q)$ ) *
    ( $1 / nnreal\ (card\ query-sample-space)$ )  $\wedge$  rounds)
proof -
  let ?Hit =
     $\lambda out. trace-fri-challenge-list-set-hit\ s\ \{challenges\}\ out \wedge$ 
      trace-fri-query-challenge-pair-set-hit  $s\ P\ out$ 
  let ?Head =
     $\lambda out. case\ out\ of$ 
      None  $\Rightarrow$  False
      | Some  $((-, f-fl), -) \Rightarrow map\ fst\ f-fl \in \{challenges\}$ 
  let ?D =
     $nnreal\ (card\ Q) *$ 
    ( $1 / nnreal\ (card\ query-sample-space)$ )  $\wedge$  rounds
  have bind-bound:
    wp-event (verifier-trace-fri-prefix  $\gg=$  verifier-after-trace-fri)
      ?Hit  $s \leq$  wp-event verifier-trace-fri-prefix ?Head  $s * ?D$ 
  proof (rule wp-event-bind-bound-by-head-and-cont)
  show  $\neg ?Hit\ None$ 
    unfolding trace-fri-challenge-list-set-hit-def
      trace-fri-query-challenge-pair-set-hit-def
      accepted-fri-opening-transcript-def
    by blast
  next
  fix header prefix-state
  assume prefix:
    Some (header, prefix-state)  $\in$ 
      set-dist (execute verifier-trace-fri-prefix  $s$ )
  and not-head:
     $\neg (case\ Some\ (header, prefix-state)\ of\ None \Rightarrow False$ 
      | Some  $((-, f-fl), -) \Rightarrow map\ fst\ f-fl \in \{challenges\})$ 
  obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
  have neq:  $map\ fst\ f-fl \neq challenges$ 
  using not-head unfolding header-eq by simp
  have zero-le:
    wp-event (verifier-after-trace-fri header)
      ( $\lambda out. trace-fri-challenge-list-set-hit\ s\ \{challenges\}\ out \wedge$ 
        trace-fri-query-challenge-pair-set-hit  $s\ P\ out$ ) prefix-state
       $\leq 0$ 

```



```

proof (rule order-trans)
  show
    wp-event (verifier-after-trace-fri header)
      (λout. trace-fri-challenge-list-set-hit s {challenges} out ∧
        trace-fri-query-challenge-pair-set-hit s P out) prefix-state
      ≤
      wp-event (verifier-after-trace-fri header)
        (λ-. False) prefix-state
proof (rule wp-event-mono-on-support)
  fix out
  assume suffix:
    out ∈
      set-dist (execute (verifier-after-trace-fri header) prefix-state)
  and hit:
    trace-fri-challenge-list-set-hit s {challenges} out ∧
    trace-fri-query-challenge-pair-set-hit s P out
  from hit obtain trace-bs fri-dg composition-bs where
    challenges-hit:
      accepted-fri-challenges s out trace-bs fri-dg composition-bs
  and trace-in: trace-bs ∈ {challenges}
  unfolding trace-fri-challenge-list-set-hit-def
  by blast
  from challenges-hit obtain result final-state where out-eq:
    out = Some (result, final-state)
  unfolding accepted-fri-challenges-def by blast
  have prefix':
    Some ((fr, f-fl), prefix-state) ∈
      set-dist (execute verifier-trace-fri-prefix s)
  using prefix header-eq by simp
  have suffix':
    Some (result, final-state) ∈
      set-dist
        (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
  using suffix header-eq out-eq by simp
  from verifier-after-trace-fri-accepted-fri-challenges[OF prefix' suffix']
  obtain dg' comp-bs' where challenges-prefix:
    accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg' comp-bs'
  by blast
  have trace-eq: trace-bs = map fst f-fl
  using accepted-fri-challenges-unique
    [OF challenges-prefix challenges-hit[unfolded out-eq]]
  by simp
  show False
  using neq trace-eq trace-in by simp
qed
next
show
  wp-event (verifier-after-trace-fri header)

```

```

    (λ-. False) prefix-state ≤ 0
  unfolding wp-event-def wp-def dist-expect-def by simp
qed
show
  wp-event (verifier-after-trace-fri header)
    (λout. trace-fri-challenge-list-set-hit s {challenges} out ∧
      trace-fri-query-challenge-pair-set-hit s P out) prefix-state = 0
  by (rule antisym[OF zero-le]) simp
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix s)
and head:
  (case Some (header, prefix-state) of None ⇒ False
   | Some (-, f-fl), -) ⇒ map fst f-fl ∈ {challenges}
obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
have f-fl-eq: map fst f-fl = challenges
  using head unfolding header-eq by simp
have future-prefix: query-future-fresh prefix-state
  by (rule verifier-trace-fri-prefix-preserves-query-future-fresh-exact
    [OF query-future prefix])
have counter-prefix: PQueryCounter prefix-state = PQueryCounter s
  using verifier-trace-fri-prefix-outcome[OF prefix[unfolded header-eq]]
  by simp
have event-bound:
  wp-event (verifier-after-trace-fri header)
    (λout. ∃ query-idxs ∈ Q.
      query-rounds-index-list-hit s query-idxs rounds out)
    prefix-state ≤ ?D
  by (rule wp-verifier-after-trace-fri-query-index-list-set-bound
    [OF future-prefix counter-prefix raw-bound subset])
have hit-to-query:
  wp-event (verifier-after-trace-fri header)
    (λout. trace-fri-challenge-list-set-hit s {challenges} out ∧
      trace-fri-query-challenge-pair-set-hit s P out) prefix-state
    ≤
  wp-event (verifier-after-trace-fri header)
    (λout. ∃ query-idxs ∈ Q.
      query-rounds-index-list-hit s query-idxs rounds out) prefix-state
proof (rule wp-event-mono-on-support)
fix out
assume suffix:
  out ∈
    set-dist (execute (verifier-after-trace-fri header) prefix-state)
and hit:
  trace-fri-challenge-list-set-hit s {challenges} out ∧
  trace-fri-query-challenge-pair-set-hit s P out

```

```

from hit obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs
  composition-final query-idxs trace-round-layers
  composition-round-layers
and pair: (query-idxs, trace-bs)  $\in P$ 
unfolding trace-fri-query-challenge-pair-set-hit-def by blast
from openings obtain result final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-fri-opening-transcript-def by blast
have prefix':
  Some ((fr, f-fl), prefix-state)  $\in$ 
  set-dist (execute verifier-trace-fri-prefix s)
using prefix header-eq by simp
have suffix':
  Some (result, final-state)  $\in$ 
  set-dist
  (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
using suffix header-eq out-eq by simp
from verifier-after-trace-fri-accepted-fri-challenges[OF prefix' suffix']
obtain dg' comp-bs' where challenges-prefix:
  accepted-fri-challenges s (Some (result, final-state))
  (map fst f-fl) dg' comp-bs'
by blast
have challenges-openings:
  accepted-fri-challenges s out trace-bs fri-dg composition-bs
using openings
unfolding accepted-fri-opening-transcript-def
apply (elim exE conjE)
apply assumption
done
have trace-eq: trace-bs = map fst f-fl
using accepted-fri-challenges-unique
  [OF challenges-prefix challenges-openings[unfolded out-eq]]
by simp
have query-in: query-idxs  $\in Q$ 
proof –
  have query-idxs  $\in$  fst ‘( $P \cap (UNIV \times \{challenges\})$ )
  using pair trace-eq f-fl-eq by force
  then show ?thesis
  using projection by blast
qed
have query-hit:
  query-rounds-index-list-hit s query-idxs rounds out
by (rule accepted-fri-opening-transcript-imp-query-rounds-index-list-hit
  [OF openings])

```

```

show  $\exists \text{query-idxs} \in Q.$ 
  query-rounds-index-list-hit s query-idxs rounds out
  using query-in query-hit by blast
qed
show
  wp-event (verifier-after-trace-fri header)
  ( $\lambda \text{out. trace-fri-challenge-list-set-hit s \{challenges\} out} \wedge$ 
   trace-fri-query-challenge-pair-set-hit s P out) prefix-state
   $\leq ?D$ 
  by (rule order-trans[OF hit-to-query event-bound])
qed
have head-bound:
  wp-event verifier-trace-fri-prefix ?Head s  $\leq$ 
   $1 / \text{nnreal} (\text{CARD}('f) \wedge \text{ceil-log clength})$ 
proof –
  have wp-event verifier-trace-fri-prefix ?Head s  $\leq$ 
   $\text{nnreal} (\text{card } \{challenges\}) / \text{nnreal} (\text{CARD}('f) \wedge \text{ceil-log clength})$ 
  by (rule wp-verifier-trace-fri-prefix-challenge-space-set-bound
   [where  $B = \{challenges\}$ , OF trace-future])
  (use challenges-space in auto)
  then show ?thesis by simp
qed
show ?thesis
  unfolding verify-monad-trace-fri-decomposition
  apply (rule order-trans)
  apply (rule bind-bound)
  apply (rule mult-right-mono[OF head-bound])
  apply simp
  done
qed

lemma wp-trace-fri-query-challenge-pair-set-hit-bound-from-query-fiber-sum:
assumes trace-future: trace-fri-future-fresh s
and query-future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and projection:
   $\bigwedge \text{challenges.}$ 
   $\text{fst } '(P \cap (UNIV \times \{challenges\})) \subseteq$ 
   $Q \text{ challenges}$ 
and subset:  $\bigwedge \text{challenges. } Q \text{ challenges} \subseteq \text{fri-query-index-list-space}$ 
shows
  wp-event verify-monad
  (trace-fri-query-challenge-pair-set-hit s P)  $s \leq$ 
  ( $\sum \text{challenges} \in \text{fri-challenge-space} (\text{ceil-log clength}).$ 
   ( $1 / \text{nnreal} (\text{CARD}('f) \wedge \text{ceil-log clength})$ ) *
   ( $\text{nnreal} (\text{card } (Q \text{ challenges})) *$ 
    ( $1 / \text{nnreal} (\text{card query-sample-space}) \wedge \text{rounds}$ ))
  )
proof –
  let  $?I = \text{fri-challenge-space} (\text{ceil-log clength})$ 

```

```

let ?E =
  λchallenges out.
    trace-fri-challenge-list-set-hit s {challenges} out ∧
    trace-fri-query-challenge-pair-set-hit s P out
let ?B =
  λchallenges.
    (1 / nnreal (CARD('f) ^ ceil-log clength)) *
    (nnreal (card (Q challenges)) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
have split:
  wp-event verify-monad
    (trace-fri-query-challenge-pair-set-hit s P) s ≤
  wp-event verify-monad
    (λout. ∃ challenges ∈ ?I. ?E challenges out) s
proof (rule wp-event-mono)
  fix out
  assume hit: trace-fri-query-challenge-pair-set-hit s P out
  from hit obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers where
  openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final query-idxs trace-round-layers
    composition-round-layers
  unfolding trace-fri-query-challenge-pair-set-hit-def by blast
  have challenges:
    accepted-fri-challenges s out trace-bs fri-dg composition-bs
  using openings
  unfolding accepted-fri-opening-transcript-def
  apply (elim exE conjE)
  apply assumption
  done
  have trace-space: trace-bs ∈ ?I
  by (rule accepted-fri-challenges-trace-space[OF challenges])
  have challenge-hit:
    trace-fri-challenge-list-set-hit s {trace-bs} out
  unfolding trace-fri-challenge-list-set-hit-def
  using challenges by blast
  show ∃ challenges ∈ ?I. ?E challenges out
  using trace-space challenge-hit hit by blast
qed
also have ... ≤ (∑ challenges ∈ ?I. ?B challenges)
proof (rule wp-event-finite-union-bound-fri-exact)
  show finite ?I
  by (rule finite-fri-challenge-space)
next
  fix challenges
  assume challenges: challenges ∈ ?I

```

```

show
  wp-event verify-monad (?E challenges) s ≤ ?B challenges
by (rule wp-trace-fri-query-challenge-pair-set-hit-fixed-challenge-bound
    [OF trace-future query-future raw-bound challenges
      projection subset])
qed
finally show ?thesis .
qed

lemma wp-trace-fri-query-challenge-pair-set-hit-bound-from-query-fibers:
fixes C :: prob
assumes future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and projection:
   $\bigwedge \text{challenges.}$ 
   $\text{fst } ' (P \cap (UNIV \times \{\text{challenges}\})) \subseteq$ 
   $Q \text{ challenges}$ 
and subset:  $\bigwedge \text{challenges. } Q \text{ challenges} \subseteq \text{fri-query-index-list-space}$ 
and bound:
   $\bigwedge \text{challenges.}$ 
   $\text{nnreal (card (} Q \text{ challenges))} *$ 
   $(1 / \text{nnreal (card query-sample-space)}) \wedge \text{rounds} \leq C$ 
shows
  wp-event verify-monad
    (trace-fri-query-challenge-pair-set-hit s P) s ≤ C
unfolding verify-monad-trace-fri-decomposition
proof (rule wp-event-bind-bound-by-cont)
show  $\neg \text{trace-fri-query-challenge-pair-set-hit s P None}$ 
unfolding trace-fri-query-challenge-pair-set-hit-def
  accepted-fri-opening-transcript-def
by blast
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix s)
obtain fr f-fl where header-eq: header = (fr, f-fl)
by (cases header) auto
have future-prefix: query-future-fresh prefix-state
by (rule verifier-trace-fri-prefix-preserves-query-future-fresh-exact
  [OF future prefix])
have counter-prefix: PQueryCounter prefix-state = PQueryCounter s
using verifier-trace-fri-prefix-outcome[OF prefix[unfolded header-eq]]
by simp
let ?Q = Q (map fst f-fl)
have event-bound:
  wp-event (verifier-after-trace-fri header)
    ( $\lambda \text{out. } \exists \text{ query-idxs} \in ?Q.$ 
      query-rounds-index-list-hit s query-idxs rounds out)

```

```

    prefix-state ≤
    nnreal (card ?Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds
  by (rule wp-verifier-after-trace-fri-query-index-list-set-bound
    [OF future-prefix counter-prefix raw-bound subset])
  have pair-to-query:
    wp-event (verifier-after-trace-fri header)
    (trace-fri-query-challenge-pair-set-hit s P) prefix-state ≤
    wp-event (verifier-after-trace-fri header)
    (λout. ∃ query-idxs ∈ ?Q.
      query-rounds-index-list-hit s query-idxs rounds out) prefix-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume suffix:
      out ∈
      set-dist (execute (verifier-after-trace-fri header) prefix-state)
    and hit: trace-fri-query-challenge-pair-set-hit s P out
  from hit obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers where
    openings:
      accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final fri-dg composition-roots composition-bs
      composition-final query-idxs trace-round-layers
      composition-round-layers
    and pair: (query-idxs, trace-bs) ∈ P
    unfolding trace-fri-query-challenge-pair-set-hit-def by blast
  from openings obtain result final-state where out-eq:
    out = Some (result, final-state)
    unfolding accepted-fri-opening-transcript-def by blast
  have prefix':
    Some ((fr, f-fl), prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix s)
    using prefix header-eq by simp
  have suffix':
    Some (result, final-state) ∈
    set-dist
    (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
    using suffix header-eq out-eq by simp
  from verifier-after-trace-fri-accepted-fri-challenges[OF prefix' suffix']
  obtain dg' comp-bs' where challenges-prefix:
    accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg' comp-bs'
  by blast
  have challenges-openings:
    accepted-fri-challenges s out trace-bs fri-dg composition-bs
  using openings
  unfolding accepted-fri-opening-transcript-def
  apply (elim exE conjE)

```

```

    apply assumption
  done
have trace-eq: trace-bs = map fst f-fl
  using accepted-fri-challenges-unique
  [OF challenges-prefix challenges-openings[unfolded out-eq]]
  by simp
have query-in: query-idxs ∈ ?Q
proof -
  have query-idxs ∈
    fst ‘ (P ∩ (UNIV × {map fst f-fl}))
    using pair trace-eq by force
  then show ?thesis
    using projection[of map fst f-fl] by blast
qed
have query-hit:
  query-rounds-index-list-hit s query-idxs rounds out
  by (rule accepted-fri-opening-transcript-imp-query-rounds-index-list-hit
    [OF openings])
show ∃ query-idxs ∈ ?Q.
  query-rounds-index-list-hit s query-idxs rounds out
  using query-in query-hit by blast
qed
show
  wp-event (verifier-after-trace-fri header)
  (trace-fri-query-challenge-pair-set-hit s P) prefix-state ≤ C
  by (rule order-trans[OF pair-to-query])
  (rule order-trans[OF event-bound bound])
qed

lemma wp-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-fibers:
  fixes C-challenge C-query :: prob
  assumes trace-future: trace-fri-future-fresh s
  and query-future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and challenge-subset: B ⊆ fri-challenge-space (ceil-log clength)
  and challenge-bound:
    nnreal (card B) / nnreal (CARD('f) ^ ceil-log clength) ≤
      C-challenge
  and projection:
    ∧challenges.
      fst ‘ ((P ∩ (UNIV × (− B))) ∩
        (UNIV × {challenges})) ⊆ Q challenges
  and subset: ∧challenges. Q challenges ⊆ fri-query-index-list-space
  and query-bound:
    ∧challenges.
      nnreal (card (Q challenges)) *
        (1 / nnreal (card query-sample-space)) ^ rounds ≤ C-query
  shows
    wp-event verify-monad

```



```

    (trace-fri-query-challenge-pair-set-hit s P) s ≤
    C-challenge + C-query
proof –
  let ?Outside = P ∩ (UNIV × (– B))
  have split:
    wp-event verify-monad
      (trace-fri-query-challenge-pair-set-hit s P) s ≤
    wp-event verify-monad
      (λout. trace-fri-challenge-list-set-hit s B out ∨
        trace-fri-query-challenge-pair-set-hit s ?Outside out) s
  proof (rule wp-event-mono)
    fix out
    assume hit: trace-fri-query-challenge-pair-set-hit s P out
    from hit obtain trace-roots trace-bs trace-final fri-dg
      composition-roots composition-bs composition-final query-idxs
      trace-round-layers composition-round-layers where
    openings:
      accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final fri-dg composition-roots composition-bs
      composition-final query-idxs trace-round-layers
      composition-round-layers
    and pair: (query-idxs, trace-bs) ∈ P
    unfolding trace-fri-query-challenge-pair-set-hit-def by blast
  show
    trace-fri-challenge-list-set-hit s B out ∨
    trace-fri-query-challenge-pair-set-hit s ?Outside out
  proof (cases trace-bs ∈ B)
    case True
    have challenges:
      accepted-fri-challenges s out trace-bs fri-dg composition-bs
    using openings
    unfolding accepted-fri-opening-transcript-def
    apply (elim exE conjE)
    apply assumption
    done
    then have trace-fri-challenge-list-set-hit s B out
    using True unfolding trace-fri-challenge-list-set-hit-def by blast
    then show ?thesis by simp
  next
    case False
    have (query-idxs, trace-bs) ∈ ?Outside
    using pair False by simp
    then have trace-fri-query-challenge-pair-set-hit s ?Outside out
    unfolding trace-fri-query-challenge-pair-set-hit-def
    using openings by blast
    then show ?thesis by simp
  qed
qed
also have ... ≤

```

```

    wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s +
    wp-event verify-monad
      (trace-fri-query-challenge-pair-set-hit s ?Outside) s
  by (rule wp-event-union-bound)
also have ... ≤ C-challenge + C-query
proof (rule add-mono)
  show wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s
    ≤ C-challenge
  by (rule order-trans[
      OF wp-verify-monad-trace-fri-challenge-list-set-bound
      [OF trace-future challenge-subset] challenge-bound])
  show
    wp-event verify-monad
      (trace-fri-query-challenge-pair-set-hit s ?Outside) s ≤
      C-query
  by (rule wp-trace-fri-query-challenge-pair-set-hit-bound-from-query-fibers
      [OF query-future raw-bound projection subset query-bound])
qed
finally show ?thesis .
qed

lemma
  wp-composition-fri-query-challenge-pair-set-hit-bound-from-query-fibers:
  fixes C :: prob
  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and projection:
     $\bigwedge dg \text{ challenges. } fst \text{ ' } (P \ dg \cap (UNIV \times \{challenges\})) \subseteq Q \ dg \text{ challenges}$ 
  and subset:  $\bigwedge dg \text{ challenges. } Q \ dg \text{ challenges} \subseteq \text{fri-query-index-list-space}$ 
  and bound:
     $\bigwedge dg \text{ challenges. } nnreal \ (card \ (Q \ dg \text{ challenges})) * (1 / nnreal \ (card \ query\text{-sample-space})) \wedge \text{rounds} \leq C$ 
  shows
    wp-event verify-monad
      (composition-fri-query-challenge-pair-set-hit s P) s ≤ C
  unfolding verify-monad-composition-fri-decomposition
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg \text{composition-fri-query-challenge-pair-set-hit s P None}$ 
  unfolding composition-fri-query-challenge-pair-set-hit-def
    accepted-fri-opening-transcript-def
  by blast
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-composition-fri-prefix s)

```

```

obtain fr f-fl f-final as dg fl where header-eq:
  header = (fr, f-fl, f-final, as, dg, fl)
  by (cases header) auto
have future-prefix: query-future-fresh prefix-state
  by (rule verifier-composition-fri-prefix-preserves-query-future-fresh
    [OF future prefix])
have counter-prefix: PQueryCounter prefix-state = PQueryCounter s
  using verifier-composition-fri-prefix-outcome[OF prefix[unfolded header-eq]]
  by simp
let ?Q = Q dg (map fst fl)
have event-bound:
  wp-event (verifier-after-composition-fri header)
    ( $\lambda out. \exists query\_idxs \in ?Q.$ 
      query-rounds-index-list-hit s query-idxs rounds out)
    prefix-state  $\leq$ 
    nnreal (card ?Q) *
    ( $1 / nnreal (card query\_sample\_space)$ )  $\wedge$  rounds
  by (rule wp-verifier-after-composition-fri-query-index-list-set-bound
    [OF future-prefix counter-prefix raw-bound subset])
have pair-to-query:
  wp-event (verifier-after-composition-fri header)
    (composition-fri-query-challenge-pair-set-hit s P) prefix-state  $\leq$ 
  wp-event (verifier-after-composition-fri header)
    ( $\lambda out. \exists query\_idxs \in ?Q.$ 
      query-rounds-index-list-hit s query-idxs rounds out) prefix-state
proof (rule wp-event-mono-on-support)
fix out
assume suffix:
  out  $\in$ 
    set-dist (execute (verifier-after-composition-fri header)
      prefix-state)
  and hit: composition-fri-query-challenge-pair-set-hit s P out
from hit obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
  openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final query-idxs trace-round-layers
    composition-round-layers
  and pair: (query-idxs, composition-bs)  $\in$  P fri-dg
  unfolding composition-fri-query-challenge-pair-set-hit-def by blast
from openings obtain result final-state where out-eq:
  out = Some (result, final-state)
  unfolding accepted-fri-opening-transcript-def by blast
have prefix':
  Some ((fr, f-fl, f-final, as, dg, fl), prefix-state)  $\in$ 
    set-dist (execute verifier-composition-fri-prefix s)
  using prefix header-eq by simp

```

```

have suffix':
  Some (result, final-state) ∈
    set-dist
      (execute
        (verifier-after-composition-fri
          (fr, f-fl, f-final, as, dg, fl))
        prefix-state)
    using suffix header-eq out-eq by simp
have challenges-prefix:
  accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg (map fst fl)
  by (rule verifier-after-composition-fri-accepted-fri-challenges
    [OF prefix' suffix'])
have challenges-openings:
  accepted-fri-challenges s out trace-bs fri-dg composition-bs
  using openings
  unfolding accepted-fri-opening-transcript-def
  apply (elim exE conjE)
  apply assumption
  done
have eqs: fri-dg = dg ∧ composition-bs = map fst fl
  using accepted-fri-challenges-unique
    [OF challenges-prefix challenges-openings[unfolded out-eq]]
  by simp
have query-in: query-idxs ∈ ?Q
proof –
  have query-idxs ∈
    fst ‘ (P dg ∩ (UNIV × {map fst fl}))
    using pair eqs by force
  then show ?thesis
    using projection[of dg map fst fl] by blast
qed
have query-hit:
  query-rounds-index-list-hit s query-idxs rounds out
  by (rule accepted-fri-opening-transcript-imp-query-rounds-index-list-hit
    [OF openings])
show  $\exists$  query-idxs ∈ ?Q.
  query-rounds-index-list-hit s query-idxs rounds out
  using query-in query-hit by blast
qed
show
  wp-event (verifier-after-composition-fri header)
    (composition-fri-query-challenge-pair-set-hit s P) prefix-state ≤ C
  by (rule order-trans[OF pair-to-query])
    (rule order-trans[OF event-bound bound])
qed

lemma wp-composition-fri-query-challenge-pair-set-hit-fixed-challenge-bound:
  assumes composition-future: composition-fri-future-fresh s

```

```

and query-future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and challenges-space:
  challenges ∈ fri-challenge-space (ceil-log (to-nat dg + 1))
and projection:
  fst ‘ (P dg ∩ (UNIV × {challenges})) ⊆ Q
and subset: Q ⊆ fri-query-index-list-space
shows
  wp-event verify-monad
    (λout.
      composition-fri-challenge-list-set-hit s
        (λdg'. if dg' = dg then {challenges} else {}) out ∧
        composition-fri-query-challenge-pair-set-hit s P out) s ≤
      (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
      (nnreal (card Q) *
        (1 / nnreal (card query-sample-space)) ^ rounds)
proof –
  let ?B = λdg'. if dg' = dg then {challenges} else {}
  let ?Hit =
    λout. composition-fri-challenge-list-set-hit s ?B out ∧
    composition-fri-query-challenge-pair-set-hit s P out
  let ?Head =
    λout. case out of
      None ⇒ False
    | Some ((-, -, -, -, dg', fl), -) ⇒
      dg' = dg ∧ map fst fl = challenges
  let ?D =
    nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds
have bind-bound:
  wp-event
    (verifier-composition-fri-prefix ≫= verifier-after-composition-fri)
    ?Hit s ≤ wp-event verifier-composition-fri-prefix ?Head s * ?D
proof (rule wp-event-bind-bound-by-head-and-cont)
show ¬ ?Hit None
  unfolding composition-fri-challenge-list-set-hit-def
    composition-fri-query-challenge-pair-set-hit-def
    accepted-fri-opening-transcript-def
  by blast
next
  fix header prefix-state
  assume prefix:
    Some (header, prefix-state) ∈
      set-dist (execute verifier-composition-fri-prefix s)
  and not-head:
    ¬ (case Some (header, prefix-state) of None ⇒ False
      | Some ((-, -, -, -, dg', fl), -) ⇒
        dg' = dg ∧ map fst fl = challenges)
  obtain fr f-fl f-final as dg' fl where header-eq:

```

```

header = (fr, f-fl, f-final, as, dg', fl)
by (cases header) auto
have not-fixed: dg' ≠ dg ∨ map fst fl ≠ challenges
using not-head unfolding header-eq by simp
have zero-le:
  wp-event (verifier-after-composition-fri header)
    (λout.
      composition-fri-challenge-list-set-hit s
        (λdg'. if dg' = dg then {challenges} else {}) out ∧
        composition-fri-query-challenge-pair-set-hit s P out) prefix-state
    ≤ 0
proof (rule order-trans)
show
  wp-event (verifier-after-composition-fri header)
    (λout.
      composition-fri-challenge-list-set-hit s
        (λdg'. if dg' = dg then {challenges} else {}) out ∧
        composition-fri-query-challenge-pair-set-hit s P out) prefix-state
    ≤
    wp-event (verifier-after-composition-fri header)
      (λ-. False) prefix-state
proof (rule wp-event-mono-on-support)
fix out
assume suffix:
  out ∈
    set-dist
      (execute (verifier-after-composition-fri header) prefix-state)
and hit:
  composition-fri-challenge-list-set-hit s
    (λdg'. if dg' = dg then {challenges} else {}) out ∧
    composition-fri-query-challenge-pair-set-hit s P out
from hit obtain trace-bs fri-dg composition-bs where
  challenges-hit:
    accepted-fri-challenges s out trace-bs fri-dg composition-bs
and comp-in:
  composition-bs ∈
    (if fri-dg = dg then {challenges} else {})
unfolding composition-fri-challenge-list-set-hit-def
by blast
from challenges-hit obtain result final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-fri-challenges-def by blast
have prefix':
  Some ((fr, f-fl, f-final, as, dg', fl), prefix-state) ∈
    set-dist (execute verifier-composition-fri-prefix s)
using prefix header-eq by simp
have suffix':
  Some (result, final-state) ∈
    set-dist

```

```

      (execute
        (verifier-after-composition-fri
          (fr, f-fl, f-final, as, dg', fl))
        prefix-state)
    using suffix header-eq out-eq by simp
  have challenges-prefix:
    accepted-fri-challenges s (Some (result, final-state))
      (map fst f-fl) dg' (map fst fl)
  by (rule verifier-after-composition-fri-accepted-fri-challenges
      [OF prefix' suffix'])
  have eqs: fri-dg = dg'  $\wedge$  composition-bs = map fst fl
    using accepted-fri-challenges-unique
      [OF challenges-prefix challenges-hit[unfolded out-eq]]
  by simp
  have fixed: fri-dg = dg  $\wedge$  composition-bs = challenges
    using comp-in by (cases fri-dg = dg) auto
  show False
    using not-fixed eqs fixed by simp
qed
next
show
  wp-event (verifier-after-composition-fri header)
    ( $\lambda$ -. False) prefix-state  $\leq 0$ 
  unfolding wp-event-def wp-def dist-expect-def by simp
qed
show
  wp-event (verifier-after-composition-fri header)
    ( $\lambda$ out.
      composition-fri-challenge-list-set-hit s
        ( $\lambda$ dg'. if dg' = dg then {challenges} else {}) out  $\wedge$ 
        composition-fri-query-challenge-pair-set-hit s P out) prefix-state
    = 0
  by (rule antisym[OF zero-le]) simp
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state)  $\in$ 
    set-dist (execute verifier-composition-fri-prefix s)
and head:
  (case Some (header, prefix-state) of None  $\Rightarrow$  False
   | Some ((-, -, -, dg', fl), -)  $\Rightarrow$ 
     dg' = dg  $\wedge$  map fst fl = challenges)
obtain fr f-fl f-final as dg' fl where header-eq:
  header = (fr, f-fl, f-final, as, dg', fl)
by (cases header) auto
have dg-eq: dg' = dg and fl-eq: map fst fl = challenges
  using head unfolding header-eq by simp-all
have future-prefix: query-future-fresh prefix-state
  by (rule verifier-composition-fri-prefix-preserves-query-future-fresh)

```

```

    [OF query-future prefix])
have counter-prefix: PQueryCounter prefix-state = PQueryCounter s
    using verifier-composition-fri-prefix-outcome[OF prefix[unfolded header-eq]]
    by simp
have event-bound:
  wp-event (verifier-after-composition-fri header)
    (λout. ∃ query-idxs ∈ Q.
      query-rounds-index-list-hit s query-idxs rounds out)
    prefix-state ≤ ?D
  by (rule wp-verifier-after-composition-fri-query-index-list-set-bound
    [OF future-prefix counter-prefix raw-bound subset])
have hit-to-query:
  wp-event (verifier-after-composition-fri header)
    (λout.
      composition-fri-challenge-list-set-hit s
        (λdg'. if dg' = dg then {challenges} else {}) out ∧
      composition-fri-query-challenge-pair-set-hit s P out) prefix-state
    ≤
    wp-event (verifier-after-composition-fri header)
      (λout. ∃ query-idxs ∈ Q.
        query-rounds-index-list-hit s query-idxs rounds out) prefix-state
proof (rule wp-event-mono-on-support)
fix out
assume suffix:
  out ∈
    set-dist (execute (verifier-after-composition-fri header)
      prefix-state)
and hit:
  composition-fri-challenge-list-set-hit s
    (λdg'. if dg' = dg then {challenges} else {}) out ∧
  composition-fri-query-challenge-pair-set-hit s P out
from hit obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
  openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final fri-dg composition-roots composition-bs
      composition-final query-idxs trace-round-layers
      composition-round-layers
    and pair: (query-idxs, composition-bs) ∈ P fri-dg
    unfolding composition-fri-query-challenge-pair-set-hit-def by blast
from openings obtain result final-state where out-eq:
  out = Some (result, final-state)
    unfolding accepted-fri-opening-transcript-def by blast
have prefix':
  Some ((fr, f-fl, f-final, as, dg', fl), prefix-state) ∈
    set-dist (execute verifier-composition-fri-prefix s)
    using prefix header-eq by simp
have suffix':

```



```

Some (result, final-state) ∈
  set-dist
    (execute
      (verifier-after-composition-fri
        (fr, f-fl, f-final, as, dg', fl))
      prefix-state)
  using suffix header-eq out-eq by simp
have challenges-prefix:
  accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg' (map fst fl)
  by (rule verifier-after-composition-fri-accepted-fri-challenges
    [OF prefix' suffix'])
have challenges-openings:
  accepted-fri-challenges s out trace-bs fri-dg composition-bs
  using openings
  unfolding accepted-fri-opening-transcript-def
  apply (elim exE conjE)
  apply assumption
  done
have eqs: fri-dg = dg' ∧ composition-bs = map fst fl
  using accepted-fri-challenges-unique
    [OF challenges-prefix challenges-openings[unfolded out-eq]]
  by simp
have query-in: query-idxs ∈ Q
proof -
  have query-idxs ∈ fst ' (P dg ∩ (UNIV × {challenges}))
    using pair eqs dg-eq fl-eq by force
  then show ?thesis
    using projection by blast
qed
have query-hit:
  query-rounds-index-list-hit s query-idxs rounds out
  by (rule accepted-fri-opening-transcript-imp-query-rounds-index-list-hit
    [OF openings])
show ∃ query-idxs ∈ Q.
  query-rounds-index-list-hit s query-idxs rounds out
  using query-in query-hit by blast
qed
show
  wp-event (verifier-after-composition-fri header)
    (λout.
      composition-fri-challenge-list-set-hit s
        (λdg'. if dg' = dg then {challenges} else {}) out ∧
        composition-fri-query-challenge-pair-set-hit s P out) prefix-state
    ≤ ?D
  by (rule order-trans[OF hit-to-query event-bound])
qed
have head-bound:
  wp-event verifier-composition-fri-prefix ?Head s ≤

```

```

1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
proof (rule order-trans)
  let ?SetHead =
    λout. case out of
      None ⇒ False
    | Some ((-, -, -, -, dg', fl), -) ⇒ map fst fl ∈ ?B dg'
  show wp-event verifier-composition-fri-prefix ?Head s ≤
    wp-event verifier-composition-fri-prefix ?SetHead s
  by (rule wp-event-mono-on-support)
    (auto split: option.splits prod.splits)
  show wp-event verifier-composition-fri-prefix ?SetHead s ≤
    1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
  by (rule wp-verifier-composition-fri-prefix-challenge-space-set-bound
    [OF composition-future])
    (use challenges-space in auto)
qed
show ?thesis
  unfolding verify-monad-composition-fri-decomposition
  apply (rule order-trans)
  apply (rule bind-bound)
  apply (rule mult-right-mono[OF head-bound])
  apply simp
done
qed

lemma wp-composition-fri-query-challenge-pair-set-hit-bound-from-query-fiber-sum:
  assumes composition-future: composition-fri-future-fresh s
  and query-future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and projection:
    ∧ dg challenges.
      fst ' (P dg ∩ (UNIV × {challenges})) ⊆
        Q dg challenges
  and subset:
    ∧ dg challenges. Q dg challenges ⊆ fri-query-index-list-space
  shows
    wp-event verify-monad
      (composition-fri-query-challenge-pair-set-hit s P) s ≤
      (∑ dg ∈ UNIV.
        ∑ challenges ∈ fri-challenge-space (ceil-log (to-nat dg + 1)).
          (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
          (nnreal (card (Q dg challenges)) *
            (1 / nnreal (card query-sample-space)) ^ rounds))
proof -
  let ?I =
    SIGMA dg:UNIV. fri-challenge-space (ceil-log (to-nat dg + 1))
  let ?E =
    λp out.
      composition-fri-challenge-list-set-hit s

```

```

    (λdg'. if dg' = fst p then {snd p} else {}) out ∧
    composition-fri-query-challenge-pair-set-hit s P out
  let ?B =
    λp.
      (1 / nnreal (CARD('f) ^ ceil-log (to-nat (fst p) + 1))) *
      (nnreal (card (Q (fst p) (snd p))) *
      (1 / nnreal (card query-sample-space)) ^ rounds)
  have split:
    wp-event verify-monad
      (composition-fri-query-challenge-pair-set-hit s P) s ≤
    wp-event verify-monad
      (λout. ∃ p ∈ ?I. ?E p out) s
  proof (rule wp-event-mono)
    fix out
    assume hit: composition-fri-query-challenge-pair-set-hit s P out
    from hit obtain trace-roots trace-bs trace-final fri-dg
      composition-roots composition-bs composition-final query-idxs
      trace-round-layers composition-round-layers where
    openings:
      accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final fri-dg composition-roots composition-bs
      composition-final query-idxs trace-round-layers
      composition-round-layers
    unfolding composition-fri-query-challenge-pair-set-hit-def by blast
    have challenges:
      accepted-fri-challenges s out trace-bs fri-dg composition-bs
    using openings
    unfolding accepted-fri-opening-transcript-def
    apply (elim exE conjE)
    apply assumption
    done
    have comp-space:
      composition-bs ∈ fri-challenge-space (ceil-log (to-nat fri-dg + 1))
    by (rule accepted-fri-challenges-composition-space[OF challenges])
    have challenge-hit:
      composition-fri-challenge-list-set-hit s
      (λdg'. if dg' = fri-dg then {composition-bs} else {}) out
    unfolding composition-fri-challenge-list-set-hit-def
    using challenges by auto
    have (fri-dg, composition-bs) ∈ ?I
    using comp-space by simp
    then show ∃ p ∈ ?I. ?E p out
  proof (intro bexI[where x = (fri-dg, composition-bs)])
    show ?E (fri-dg, composition-bs) out
    using challenge-hit hit by (simp only: fst-conv snd-conv)
    show (fri-dg, composition-bs) ∈ ?I
    using comp-space by simp
  qed
qed

```

also have ... $\leq (\sum p \in ?I. ?B p)$
proof (rule wp-event-finite-union-bound-fri-exact)
 show finite ?I
 by (simp add: finite-fri-challenge-space)
next
 fix p
 assume p-in: $p \in ?I$
 obtain dg challenges where p-eq: $p = (dg, challenges)$
 by (cases p)
 have challenges:
 challenges $\in$ fri-challenge-space (ceil-log (to-nat dg + 1))
 using p-in p-eq by simp
 have fixed-bound:
 wp-event verify-monad
 ($\lambda out. composition-fri-challenge-list-set-hit s$
 ($\lambda dg'. if dg' = dg then \{challenges\} else \{\}$) out $\wedge$
 composition-fri-query-challenge-pair-set-hit s P out) s
 $\leq 1 / nnreal (CARD('f) \wedge ceil-log (to-nat dg + 1)) *$
 (nnreal (card (Q dg challenges)) *)
 (1 / nnreal (card query-sample-space)) $\wedge$ rounds)
 by (rule
 wp-composition-fri-query-challenge-pair-set-hit-fixed-challenge-bound
 [OF composition-future query-future raw-bound challenges
 projection subset])
 show wp-event verify-monad (?E p) s $\leq ?B p$
 unfolding p-eq using fixed-bound by (simp only: fst-conv snd-conv)
qed
 also have ... =
 ($\sum dg \in UNIV.$
 $\sum challenges \in fri-challenge-space (ceil-log (to-nat dg + 1)).$
 (1 / nnreal (CARD('f) $\wedge$ ceil-log (to-nat dg + 1))) *
 (nnreal (card (Q dg challenges)) *)
 (1 / nnreal (card query-sample-space)) $\wedge$ rounds))
 by (subst sum.Sigma) (simp-all add: finite-fri-challenge-space case-prod-beta)
 finally show ?thesis .
qed

lemma wp-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-fibers:
 fixes C-challenge C-query :: prob
 assumes composition-future: composition-fri-future-fresh s
 and query-future: query-future-fresh s
 and raw-bound: query-index-raw-preimage-bound
 and challenge-subset:
 $\bigwedge dg. B dg \subseteq fri-challenge-space (ceil-log (to-nat dg + 1))$
 and challenge-bound:
 $\bigwedge dg. nnreal (card (B dg)) /$
 $nnreal (CARD('f) \wedge ceil-log (to-nat dg + 1)) \leq C-challenge$
 and projection:
 $\bigwedge dg challenges.$

```

    fst ‘ ((P dg ∩ (UNIV × (− B dg))) ∩
      (UNIV × {challenges})) ⊆ Q dg challenges
  and subset:
    ∧ dg challenges. Q dg challenges ⊆ fri-query-index-list-space
  and query-bound:
    ∧ dg challenges.
      nnreal (card (Q dg challenges)) *
      (1 / nnreal (card query-sample-space)) ^ rounds ≤ C-query
  shows
    wp-event verify-monad
      (composition-fri-query-challenge-pair-set-hit s P) s ≤
      C-challenge + C-query
  proof −
    let ?Outside = λ dg. P dg ∩ (UNIV × (− B dg))
    have split:
      wp-event verify-monad
        (composition-fri-query-challenge-pair-set-hit s P) s ≤
      wp-event verify-monad
        (λ out. composition-fri-challenge-list-set-hit s B out ∨
          composition-fri-query-challenge-pair-set-hit s ?Outside out) s
    proof (rule wp-event-mono)
      fix out
      assume hit: composition-fri-query-challenge-pair-set-hit s P out
      from hit obtain trace-roots trace-bs trace-final fri-dg
        composition-roots composition-bs composition-final query-idxs
        trace-round-layers composition-round-layers where
        openings:
          accepted-fri-opening-transcript s out trace-roots trace-bs
          trace-final fri-dg composition-roots composition-bs
          composition-final query-idxs trace-round-layers
          composition-round-layers
      and pair: (query-idxs, composition-bs) ∈ P fri-dg
      unfolding composition-fri-query-challenge-pair-set-hit-def by blast
    show
      composition-fri-challenge-list-set-hit s B out ∨
      composition-fri-query-challenge-pair-set-hit s ?Outside out
    proof (cases composition-bs ∈ B fri-dg)
      case True
      have challenges:
        accepted-fri-challenges s out trace-bs fri-dg composition-bs
      using openings
      unfolding accepted-fri-opening-transcript-def
      apply (elim exE conjE)
      apply assumption
      done
    then have composition-fri-challenge-list-set-hit s B out
      using True unfolding composition-fri-challenge-list-set-hit-def
      by blast
    then show ?thesis by simp

```

```

next
  case False
  have (query-idxs, composition-bs) ∈ ?Outside fri-dg
    using pair False by simp
  then have
    composition-fri-query-challenge-pair-set-hit s ?Outside out
    unfolding composition-fri-query-challenge-pair-set-hit-def
    using openings by blast
  then show ?thesis by simp
qed
qed
also have ... ≤
  wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s +
  wp-event verify-monad
    (composition-fri-query-challenge-pair-set-hit s ?Outside) s
  by (rule wp-event-union-bound)
also have ... ≤ C-challenge + C-query
proof (rule add-mono)
show
  wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s
    ≤ C-challenge
  by (rule wp-verify-monad-composition-fri-challenge-list-set-bound
    [OF composition-future challenge-subset challenge-bound])
show
  wp-event verify-monad
    (composition-fri-query-challenge-pair-set-hit s ?Outside) s ≤
    C-query
  by (rule
    wp-composition-fri-query-challenge-pair-set-hit-bound-from-query-fibers
    [OF query-future raw-bound projection subset query-bound])
qed
finally show ?thesis .
qed

```

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-fibers*:

fixes *C*-challenge *C*-query :: prob

assumes trace-future: trace-fri-future-fresh *s*

and query-future: query-future-fresh *s*

and raw-bound: query-index-raw-preimage-bound

and challenge-subset: $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$

and challenge-bound:

$$\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}'f) \wedge \text{ceil-log } \text{clength}) \leq \text{C-challenge}$$

and projection:

$$\bigwedge \text{challenges.}$$

fst ‘

$$(((\text{trace-fri-sampled-query-bad-pair-union} \cap (\text{fri-query-index-list-space} \times \text{UNIV})) \cap (\text{UNIV} \times (- B))) \cap$$

$(UNIV \times \{challenges\}) \subseteq Q \text{ challenges}$
and *subset*: $\bigwedge challenges. Q \text{ challenges} \subseteq \text{fri-query-index-list-space}$
and *query-bound*:
 $\bigwedge challenges.$
 $nnreal (\text{card } (Q \text{ challenges})) * (1 / nnreal (\text{card query-sample-space})) \wedge rounds \leq C\text{-query}$
shows
 $wp\text{-event verify-monad } (\text{trace-fri-sampled-query-bad-candidate } s) \ s \leq C\text{-challenge} + C\text{-query}$
proof –
let $?P =$
 $\text{trace-fri-sampled-query-bad-pair-union} \cap (\text{fri-query-index-list-space} \times UNIV)$
have $wp\text{-event verify-monad } (\text{trace-fri-sampled-query-bad-candidate } s) \ s$
 $\leq wp\text{-event verify-monad } (\text{trace-fri-query-challenge-pair-set-hit } s \ ?P) \ s$
by (rule
 $wp\text{-trace-fri-sampled-query-bad-candidate-bound-by-restricted-query-challenge-union-hit}$)
also have $\dots \leq C\text{-challenge} + C\text{-query}$
by (rule
 $wp\text{-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-fibers}$
 $[OF \text{ trace-future query-future raw-bound challenge-subset challenge-bound projection subset query-bound}]$)
finally show $?thesis$.
qed

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-fibers*:

fixes $C\text{-challenge } C\text{-query} :: \text{prob}$
assumes *composition-future*: *composition-fri-future-fresh* s
and *query-future*: *query-future-fresh* s
and *raw-bound*: *query-index-raw-preimage-bound*
and *challenge-subset*:
 $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$
and *challenge-bound*:
 $\bigwedge dg. nnreal (\text{card } (B \ dg)) / nnreal (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq C\text{-challenge}$
and *projection*:
 $\bigwedge dg \text{ challenges.}$
 fst '
 $((((\text{composition-fri-sampled-query-bad-pair-union } dg \cap (\text{fri-query-index-list-space} \times UNIV)) \cap (UNIV \times (- \ B \ dg))) \cap (UNIV \times \{challenges\}))) \subseteq Q \ dg \text{ challenges}$
and *subset*:
 $\bigwedge dg \text{ challenges. } Q \ dg \text{ challenges} \subseteq \text{fri-query-index-list-space}$
and *query-bound*:
 $\bigwedge dg \text{ challenges.}$
 $nnreal (\text{card } (Q \ dg \text{ challenges})) * (1 / nnreal (\text{card query-sample-space})) \wedge rounds \leq C\text{-query}$

shows
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-sampled-query-bad-candidate } s) \ s \leq$
 $C\text{-challenge} + C\text{-query}$

proof –
let $?P =$
 $\lambda dg. composition\text{-fri-sampled-query-bad-pair-union } dg \cap$
 $(fri\text{-query-index-list-space} \times UNIV)$
have $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-sampled-query-bad-candidate } s) \ s \leq$
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-query-challenge-pair-set-hit } s \ ?P) \ s$
by (rule
 $wp\text{-composition-fri-sampled-query-bad-candidate-bound-by-restricted-query-challenge-union-hit})$
also have $\dots \leq C\text{-challenge} + C\text{-query}$
by (rule
 $wp\text{-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-fibers}$
 $[OF \ composition\text{-future } query\text{-future } raw\text{-bound } challenge\text{-subset}$
 $challenge\text{-bound } projection \ subset \ query\text{-bound}]])$
finally show $?thesis$.
qed

lemma $wp\text{-trace-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-fibers}$:
fixes $C\text{-challenge } C\text{-query} :: prob$
assumes $trace\text{-future}: trace\text{-fri-future-fresh } s$
and $query\text{-future}: query\text{-future-fresh } s$
and $raw\text{-bound}: query\text{-index-raw-preimage-bound}$
and $challenge\text{-subset}: B \subseteq fri\text{-challenge-space } (ceil\text{-log } clength)$
and $challenge\text{-bound}$:
 $nnreal \ (card \ B) / nnreal \ (CARD('f) \wedge ceil\text{-log } clength) \leq$
 $C\text{-challenge}$
and $query\text{-bound}$:
 $\bigwedge challenges.$
 $nnreal$
 $(card$
 $(generic\text{-fri-sampled-query-query-fiber } trace\text{-table-low-degree}$
 $(Not \circ trace\text{-table-low-degree}) \ (clength - 1) \ challenges)) \ *$
 $(1 / nnreal \ (card \ query\text{-sample-space})) \wedge rounds \leq C\text{-query}$

shows
 $wp\text{-event } verify\text{-monad } (trace\text{-fri-sampled-query-bad-candidate } s) \ s \leq$
 $C\text{-challenge} + C\text{-query}$

proof (rule
 $wp\text{-trace-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-fibers}$
 $[OF \ trace\text{-future } query\text{-future } raw\text{-bound } challenge\text{-subset } challenge\text{-bound}])$
fix $challenges$
show
 $fst \text{ `}$
 $((trace\text{-fri-sampled-query-bad-pair-union} \cap$
 $(fri\text{-query-index-list-space} \times UNIV))) \cap$


```

    (UNIV × (− B))) ∩
    (UNIV × {challenges}))
  ⊆
  generic-fri-sampled-query-query-fiber trace-table-low-degree
  (Not ∘ trace-table-low-degree) (clength − 1) challenges
unfolding trace-fri-sampled-query-bad-pair-union-def
  generic-fri-sampled-query-query-fiber-def
  generic-fri-sampled-query-restricted-bad-pairs-def
  fri-query-challenge-pair-query-fiber-def
by auto
next
  fix challenges
  show
    generic-fri-sampled-query-query-fiber trace-table-low-degree
    (Not ∘ trace-table-low-degree) (clength − 1) challenges
    ⊆ fri-query-index-list-space
    by (rule generic-fri-sampled-query-query-fiber-subset)
next
  fix challenges
  show
    nnreal
    (card
      (generic-fri-sampled-query-query-fiber trace-table-low-degree
        (Not ∘ trace-table-low-degree) (clength − 1) challenges)) *
    (1 / nnreal (card query-sample-space)) ^ rounds ≤ C-query
    by (rule query-bound)
qed

lemma wp-trace-fri-sampled-query-bad-candidate-bound-from-pair-fraction:
  assumes trace-future: trace-fri-future-fresh s
  and query-future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and fraction:
    generic-fri-sampled-query-pair-fraction-bound trace-table-low-degree
    (Not ∘ trace-table-low-degree) (clength − 1) C
  shows
    wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s ≤ C
proof −
  let ?P =
    trace-fri-sampled-query-bad-pair-union ∩
    (fri-query-index-list-space × UNIV)
  let ?Q =
    λchallenges.
      generic-fri-sampled-query-query-fiber trace-table-low-degree
      (Not ∘ trace-table-low-degree) (clength − 1) challenges
  let ?I = fri-challenge-space (ceil-log clength)
  have sampled-to-pair:
    wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s
    ≤ wp-event verify-monad

```

```

    (trace-fri-query-challenge-pair-set-hit s ?P) s
  by (rule
    wp-trace-fri-sampled-query-bad-candidate-bound-by-restricted-query-challenge-union-hit)
also have ... ≤
  (∑ challenges ∈ ?I.
    (1 / nnreal (CARD('f) ^ ceil-log clength)) *
    (nnreal (card (?Q challenges)) *
    (1 / nnreal (card query-sample-space)) ^ rounds))
proof (rule
  wp-trace-fri-query-challenge-pair-set-hit-bound-from-query-fiber-sum
  [OF trace-future query-future raw-bound])
fix challenges
show fst ' (?P ∩ (UNIV × {challenges})) ⊆
  ?Q challenges
  unfolding trace-fri-sampled-query-bad-pair-union-def
    generic-fri-sampled-query-query-fiber-def
    generic-fri-sampled-query-restricted-bad-pairs-def
    fri-query-challenge-pair-query-fiber-def
  by auto
next
fix challenges
show ?Q challenges ⊆ fri-query-index-list-space
  by (rule generic-fri-sampled-query-query-fiber-subset)
qed
also have ... ≤ C
proof -
  have round-eq:
    fri-round-count-for-degree-bound (clength - Suc 0) = ceil-log clength
  unfolding fri-round-count-for-degree-bound-def
  by (simp add: clength-pos)
  have fiber-fraction:
    nnreal
      (∑ challenges ∈ ?I. card (?Q challenges)) /
      (nnreal (CARD('f) ^ ceil-log clength) *
      nnreal query-sample-space-size ^ rounds)
    ≤ C
  using fraction
  unfolding trace-fri-sampled-query-pair-fraction-bound-iff-fiber-sum
  by (simp add: round-eq card-fri-query-index-list-space
    query-sample-space-size-pos algebra-simps)
  have prob-eq:
    (∑ challenges ∈ ?I.
      (1 / nnreal (CARD('f) ^ ceil-log clength)) *
      (nnreal (card (?Q challenges)) *
      (1 / nnreal (card query-sample-space)) ^ rounds)) =
    nnreal
      (∑ challenges ∈ ?I. card (?Q challenges)) /
      (nnreal (CARD('f) ^ ceil-log clength) *
      nnreal query-sample-space-size ^ rounds)

```

```

using weighted-query-fiber-sum-eq
  [where  $I = ?I$ 
    and  $f = \lambda \text{challenges. card } (?Q \text{ challenges})$ 
    and  $A = \text{CARD}(f) \wedge \text{ceil-log clength}$ 
    and  $B = \text{query-sample-space-size}$ ]
  by (simp add: finite-fri-challenge-space query-sample-space-size-pos
    card-query-sample-space algebra-simps)
show ?thesis
  using fiber-fraction unfolding prob-eq .
qed
finally show ?thesis .
qed

lemma wp-composition-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-fibers:
fixes C-challenge C-query :: prob
assumes composition-future: composition-fri-future-fresh s
and query-future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and challenge-subset:
   $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$ 
and challenge-bound:
   $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) /$ 
   $\text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq C\text{-challenge}$ 
and query-bound:
   $\bigwedge dg \text{ challenges.}$ 
   $\text{nnreal}$ 
   $(\text{card}$ 
     $(\text{generic-fri-sampled-query-query-fiber}$ 
       $(\text{composition-table-low-degree } (to\text{-nat } dg))$ 
       $(\text{Not } \circ \text{ composition-table-low-degree maxDegree } (to\text{-nat } dg)$ 
       $\text{challenges})) *$ 
     $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds} \leq C\text{-query}$ 
  shows
    wp-event verify-monad
     $(\text{composition-fri-sampled-query-bad-candidate } s) \ s \leq$ 
     $C\text{-challenge} + C\text{-query}$ 
proof (rule
  wp-composition-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-fibers
  [OF composition-future query-future raw-bound challenge-subset
    challenge-bound])
fix dg challenges
show
  fst ‘
     $((\text{composition-fri-sampled-query-bad-pair-union } dg \cap$ 
       $(\text{fri-query-index-list-space} \times \text{UNIV})) \cap$ 
       $(\text{UNIV} \times (- \ B \ dg))) \cap$ 
       $(\text{UNIV} \times \{\text{challenges}\}))$ 
     $\subseteq$ 
    generic-fri-sampled-query-query-fiber

```

```

      (composition-table-low-degree (to-nat dg))
      (Not ◦ composition-table-low-degree maxDegree) (to-nat dg)
      challenges
unfolding composition-fri-sampled-query-bad-pair-union-def
      generic-fri-sampled-query-query-fiber-def
      generic-fri-sampled-query-restricted-bad-pairs-def
      fri-query-challenge-pair-query-fiber-def
by auto
next
fix dg challenges
show
  generic-fri-sampled-query-query-fiber
    (composition-table-low-degree (to-nat dg))
    (Not ◦ composition-table-low-degree maxDegree) (to-nat dg)
    challenges
  ⊆ fri-query-index-list-space
by (rule generic-fri-sampled-query-query-fiber-subset)
next
fix dg challenges
show
  nnreal
    (card
      (generic-fri-sampled-query-query-fiber
        (composition-table-low-degree (to-nat dg))
        (Not ◦ composition-table-low-degree maxDegree) (to-nat dg)
        challenges)) *
      (1 / nnreal (card query-sample-space)) ^ rounds ≤ C-query
by (rule query-bound)
qed

lemma wp-composition-fri-sampled-query-bad-candidate-bound-from-pair-fraction-sum:
fixes C :: 'f ⇒ prob
assumes composition-future: composition-fri-future-fresh s
and query-future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and fraction:
  ∧ dg. generic-fri-sampled-query-pair-fraction-bound
    (composition-table-low-degree (to-nat dg))
    (Not ◦ composition-table-low-degree maxDegree) (to-nat dg) (C dg)
shows
  wp-event verify-monad
    (composition-fri-sampled-query-bad-candidate s) s ≤
    (∑ dg ∈ UNIV. C dg)
proof –
let ?P =
  λdg. composition-fri-sampled-query-bad-pair-union dg ∩
    (fri-query-index-list-space × UNIV)
let ?Q =
  λdg challenges.

```

```

    generic-fri-sampled-query-query-fiber
      (composition-table-low-degree (to-nat dg))
      (Not ∘ composition-table-low-degree maxDegree) (to-nat dg)
      challenges
have sampled-to-pair:
  wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s
    ≤ wp-event verify-monad
      (composition-fri-query-challenge-pair-set-hit s ?P) s
by (rule
  wp-composition-fri-sampled-query-bad-candidate-bound-by-restricted-query-challenge-union-hit)
also have ... ≤
  (∑ dg ∈ UNIV.
    ∑ challenges ∈ fri-challenge-space (ceil-log (to-nat dg + 1)).
      (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
      (nnreal (card (?Q dg challenges)) *
      (1 / nnreal (card query-sample-space)) ^ rounds))
proof (rule
  wp-composition-fri-query-challenge-pair-set-hit-bound-from-query-fiber-sum
  [OF composition-future query-future raw-bound])
fix dg challenges
show fst ' (?P dg ∩ (UNIV × {challenges})) ⊆
  ?Q dg challenges
unfolding composition-fri-sampled-query-bad-pair-union-def
  generic-fri-sampled-query-query-fiber-def
  generic-fri-sampled-query-restricted-bad-pairs-def
  fri-query-challenge-pair-query-fiber-def
by auto
next
fix dg challenges
show ?Q dg challenges ⊆ fri-query-index-list-space
by (rule generic-fri-sampled-query-query-fiber-subset)
qed
also have ... ≤ (∑ dg ∈ UNIV. C dg)
proof (rule sum-mono)
fix dg :: 'f
assume dg ∈ UNIV
let ?I = fri-challenge-space (ceil-log (to-nat dg + 1))
have round-eq:
  fri-round-count-for-degree-bound (to-nat dg) =
    ceil-log (to-nat dg + 1)
unfolding fri-round-count-for-degree-bound-def by simp
have fiber-fraction:
  nnreal
    (∑ challenges ∈ ?I. card (?Q dg challenges)) /
    (nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) *
    nnreal query-sample-space-size ^ rounds)
    ≤ C dg
using fraction[of dg]
unfolding composition-fri-sampled-query-pair-fraction-bound-iff-fiber-sum

```

```

    by (simp add: round-eq card-fri-query-index-list-space
        query-sample-space-size-pos algebra-simps)
  have prob-eq:
    ( $\sum challenges \in ?I.$ 
      ( $1 / \text{nnreal } (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1))) *$ 
      ( $\text{nnreal } (card \text{ (?Q } dg \text{ challenges)}) *$ 
      ( $1 / \text{nnreal } (card \text{ query-sample-space})) \wedge \text{rounds})) =$ 
       $\text{nnreal}$ 
      ( $\sum challenges \in ?I. card \text{ (?Q } dg \text{ challenges)}) /$ 
      ( $\text{nnreal } (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) *$ 
       $\text{nnreal } query\text{-sample-space-size} \wedge \text{rounds})$ 
  using weighted-query-fiber-sum-eq
  [where  $I = ?I$ 
    and  $f = \lambda challenges. card \text{ (?Q } dg \text{ challenges)}$ 
    and  $A = CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)$ 
    and  $B = query\text{-sample-space-size}$ ]
  by (simp add: finite-fri-challenge-space query-sample-space-size-pos
      card-query-sample-space algebra-simps)
  show
    ( $\sum challenges \in ?I.$ 
      ( $1 / \text{nnreal } (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1))) *$ 
      ( $\text{nnreal } (card \text{ (?Q } dg \text{ challenges)}) *$ 
      ( $1 / \text{nnreal } (card \text{ query-sample-space})) \wedge \text{rounds}))$ 
       $\leq C \text{ } dg$ 
    using fiber-fraction unfolding prob-eq .
  qed
  finally show ?thesis .
qed

end

end

```

```

theory Soundness-FRI-Query-Index-Staged-Bounds
imports Soundness-FRI-Sampled-Interface
begin

```

Staged query-index accounting for FRI query-list events.

The verifier-local exact query-list bounds still assume query freshness of the transferred verifier state. This layer charges accepted FRI query-list hits to the checked staged transcript query-index target event, using only the query indices actually derived by the verifier.

```

lemma pair-second-set-cover-split:
  assumes  $(x, y) \in P$ 
  shows  $y \in B \vee (x, y) \in P \cap (UNIV \times (- B))$ 
proof (cases  $y \in B$ )
  case True
  then show ?thesis ..

```

```

next
  case False
  then have  $(x, y) \in P \cap (UNIV \times (- B))$ 
    using assms by simp
  then show ?thesis ..
qed

context soundness
begin

lemma accepted-fri-opening-transcript-query-headerE:
  assumes
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  obtains fr as query-state where
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final (PTranscript query-state)
    PState query-state =
    verifier-header-state s fr trace-roots trace-final as dg
    composition-roots composition-final
    verifier-query-indices-derived s out
    (verifier-header-state s fr trace-roots trace-final as dg
    composition-roots composition-final)
    (PTranscript query-state) trace-roots composition-roots query-idxs
  proof -
    show ?thesis
      using assms
      unfolding accepted-fri-opening-transcript-def
      apply (elim exE conjE)
      apply (rule that)
      by (auto intro: Channel-Core.protocol-channel.select-convs(1))
  qed

lemma accepted-fri-opening-transcript-imp-staged-query-index-set-hit:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and builder:
    Some (data, attacker-state)  $\in$ 
      set-dist
      (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  and openings:
    accepted-fri-opening-transcript
    (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
    (Some (result, final-state)) trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers

```

```

and query-in: query-idxs ∈ Q
shows staged-security-with-data-query-index-set-hit
  (query-index-list-entries Q) (Some ((data, result), final-state))
proof –
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from accepted-fri-opening-transcript-query-headerE[OF openings]
obtain fr as query-state where
  header:
    verifier-header-transcript ?s fr trace-roots trace-final as dg
    composition-roots composition-final (PTranscript query-state)
and query-state-hash:
  PState query-state =
    verifier-header-state ?s fr trace-roots trace-final as dg
    composition-roots composition-final
and derived:
    verifier-query-indices-derived ?s (Some (result, final-state))
    (verifier-header-state ?s fr trace-roots trace-final as dg
     composition-roots composition-final)
    (PTranscript query-state) trace-roots composition-roots query-idxs
by blast
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
have header-eq:
  fr = staged-trace-root data ∧
  trace-roots = staged-trace-fri-roots data ∧
  trace-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-roots = staged-composition-fri-roots data ∧
  composition-final = staged-composition-final data ∧
  PTranscript query-state = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
have query-start-eq:
  verifier-header-state ?s fr trace-roots trace-final as dg
    composition-roots composition-final =
    staged-query-start-hash data

```



```

using query-state-hash header-eq
unfolding staged-query-start-hash-def
  staged-composition-fri-start-hash-def
  staged-trace-fri-start-hash-def
  verifier-header-state-def verifier-header-messages-def
  verifier-state-from-adversary-def
by simp
have query-counter: PQueryCounter ?s = 0
  unfolding verifier-state-from-adversary-def by simp
from verifier-query-indices-derivedE[OF derived]
obtain result' final-state' raw-idxs query-chunks trailing where
  out-derived: Some (result, final-state) = Some (result', final-state')
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and len-chunks: length query-chunks = rounds
and concat-eq:
  List.concat query-chunks @ trailing = PTranscript query-state
and round-chunks:
 $\bigwedge i. i < \text{rounds} \implies$ 
  verifier-query-round-chunk (query-idxs ! i)
  trace-roots composition-roots (query-chunks ! i)
and lookups':
 $\bigwedge i. i < \text{rounds} \implies$ 
  fmlookup (HashMap final-state')
  (QueryIndexChallenge (PQueryCounter ?s + i)
    (state-after-query-chunks
      (verifier-header-state ?s fr trace-roots trace-final as dg
        composition-roots composition-final)
      query-chunks i)) =
    Some (raw-idxs ! i)
by metis
have final-state'-eq: final-state' = final-state
  using out-derived by simp
have lookups:
 $\bigwedge i. i < \text{rounds} \implies$ 
  fmlookup (HashMap final-state)
  (QueryIndexChallenge (PQueryCounter ?s + i)
    (state-after-query-chunks
      (verifier-header-state ?s fr trace-roots trace-final as dg
        composition-roots composition-final)
      query-chunks i)) =
    Some (raw-idxs ! i)
  using lookups' final-state'-eq by simp
have len-query: length query-idxs = rounds
  using len-raw query-idxs-eq by simp
have parser-chunk-len:
 $\bigwedge i. i < \text{rounds} \implies$ 
  length (query-chunks ! i) =

```

```

    verifier-query-round-transcript-length (query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
  proof -
    fix i
    assume i-bound: i < rounds
    show
      length (query-chunks ! i) =
        verifier-query-round-transcript-length (query-idxs ! i)
          (staged-trace-fri-roots data)
          (staged-composition-fri-roots data)
    using verifier-query-round-chunk-length[OF round-chunks[OF i-bound]]
      header-eq
    by simp
  qed
  have concat-staged:
    List.concat query-chunks @ trailing =
      List.concat (staged-query-chunks data)
    using concat-eq header-eq by simp
  have chunks-eq: query-chunks = staged-query-chunks data
    using selected-query-chunks-eq-staged
      [OF builder len-query len-chunks concat-staged parser-chunk-len]
    by simp
  from rounds-positive obtain i where i-bound: i < rounds
  by auto
  have idx-in: query-idxs ! i ∈ query-index-list-entries Q
    using query-in i-bound len-query unfolding query-index-list-entries-def
    by (auto intro: nth-mem)
  have lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
    using lookups[OF i-bound] query-counter query-start-eq chunks-eq
    by simp
  have raw-hit:
    index (to-nat (raw-idxs ! i)) ∈ query-index-list-entries Q
    using idx-in query-idxs-eq i-bound len-raw by simp
  show ?thesis
    unfolding staged-security-with-data-query-index-set-hit-def
    using i-bound lookup raw-hit by auto
  qed

  lemma checked-staged-security-trace-fri-query-index-list-set-hit-bound:
    assumes wf: staged-budget-wellformed budgets
      and controlled: staged-adversary-controlled budgets A
    shows
      wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-query-index-list-set-hit s Q))
    adversary-initial-state ≤
    staged-phase-target-error
      (query-index-raw-preimage (query-index-list-entries Q))
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
proof –
  let ?B = query-index-list-entries Q
  let ?projected =
    λout. case out of None ⇒
      staged-security-with-data-query-index-set-hit ?B None
    | Some (((data, -), result), t) ⇒
      staged-security-with-data-query-index-set-hit ?B
        (Some ((data, result), t))
  have event-le:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-query-index-list-set-hit s Q))
      adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?projected adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume out-support:
      out ∈
        set-dist
          (execute (checked-staged-security-experiment-with-data-state A)
            adversary-initial-state)
    assume hit:
      staged-security-with-data-state-verifier-event
        (λs. trace-fri-query-index-list-set-hit s Q) out
    show ?projected out
    proof (cases out)
      case None
      then show ?thesis
        using hit
        unfolding staged-security-with-data-state-verifier-event-def
          staged-security-with-data-query-index-set-hit-def
        by simp
    next
      case (Some pair)
      then obtain data attacker-state result final-state where out-eq:
        out = Some (((data, attacker-state), result), final-state)
      by (cases pair) (auto split: prod.splits)
      let ?s =
        verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)
      have builder:

```

```

Some (data, attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-program A)
    adversary-initial-state)
using out-support
unfolding out-eq checked-staged-security-experiment-with-data-state-def
by (auto elim!: set-dist-bindE)
from hit out-eq have trace-hit:
  trace-fri-query-index-list-set-hit ?s Q
  (Some (result, final-state))
  unfolding staged-security-with-data-state-verifier-event-def by simp
from trace-hit obtain trace-roots trace-bs trace-final dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
  fri:
    accepted-fri-opening-transcript ?s (Some (result, final-state))
    trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final query-idxs trace-round-layers
    composition-round-layers
  and query-in: query-idxs ∈ Q
  unfolding trace-fri-query-index-list-set-hit-def by blast
have staged-hit:
  staged-security-with-data-query-index-set-hit
  (query-index-list-entries Q)
  (Some ((data, result), final-state))
  by (rule
    accepted-fri-opening-transcript-imp-staged-query-index-set-hit
    [OF wf controlled builder fri query-in])
show ?thesis
  unfolding out-eq using staged-hit by simp
qed
qed
also have
  wp-event (checked-staged-security-experiment-with-data-state A)
  ?projected adversary-initial-state =
  wp-event (checked-staged-security-experiment-with-data A)
  (staged-security-with-data-query-index-set-hit ?B)
  adversary-initial-state
  using checked-staged-security-experiment-with-data-event-from-data-state
  [of A staged-security-with-data-query-index-set-hit ?B]
  by simp
also have ... ≤
  staged-phase-target-error
  (query-index-raw-preimage (query-index-list-entries Q))
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule checked-staged-security-with-data-query-index-set-hit-bound
    [OF wf controlled])
finally show ?thesis .
qed

```

lemma *checked-staged-security-composition-fri-query-index-list-set-hit-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
($\lambda s.$ composition-fri-query-index-list-set-hit s Q))
adversary-initial-state $\leq$
staged-phase-target-error
(query-index-raw-preimage (query-index-list-entries Q))
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)

proof –

let *?B = query-index-list-entries Q*

let *?projected =*

$\lambda out.$ case out of None $\Rightarrow$
staged-security-with-data-query-index-set-hit ?B None
| Some (((data, -), result), t) $\Rightarrow$
staged-security-with-data-query-index-set-hit ?B
(Some ((data, result), t))

have *event-le:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
($\lambda s.$ composition-fri-query-index-list-set-hit s Q))
adversary-initial-state $\leq$
wp-event (checked-staged-security-experiment-with-data-state A)
?projected adversary-initial-state

proof (*rule wp-event-mono-on-support*)

fix *out*

assume *out-support:*

out $\in$
set-dist
(execute (checked-staged-security-experiment-with-data-state A)
adversary-initial-state)

assume *hit:*

staged-security-with-data-state-verifier-event
($\lambda s.$ composition-fri-query-index-list-set-hit s Q) out

show *?projected out*

proof (*cases out*)

case *None*

then show *?thesis*

using *hit*

unfolding *staged-security-with-data-state-verifier-event-def*
staged-security-with-data-query-index-set-hit-def

by *simp*

next

case (*Some pair*)

then obtain *data attacker-state result final-state where out-eq:*

```

    out = Some (((data, attacker-state), result), final-state)
  by (cases pair) (auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have builder:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
using out-support
unfolding out-eq checked-staged-security-experiment-with-data-state-def
by (auto elim!: set-dist-bindE)
from hit out-eq have composition-hit:
  composition-fri-query-index-list-set-hit ?s Q
  (Some (result, final-state))
  unfolding staged-security-with-data-state-verifier-event-def by simp
from composition-hit obtain trace-roots trace-bs trace-final dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
  fri:
    accepted-fri-opening-transcript ?s (Some (result, final-state))
    trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final query-idxs trace-round-layers
    composition-round-layers
  and query-in: query-idxs ∈ Q
  unfolding composition-fri-query-index-list-set-hit-def by blast
have staged-hit:
  staged-security-with-data-query-index-set-hit
  (query-index-list-entries Q)
  (Some ((data, result), final-state))
  apply (rule
    accepted-fri-opening-transcript-imp-staged-query-index-set-hit
    [OF wf controlled builder fri query-in])
.
show ?thesis
  unfolding out-eq using staged-hit by simp
qed
qed
also have
  wp-event (checked-staged-security-experiment-with-data-state A)
  ?projected adversary-initial-state =
  wp-event (checked-staged-security-experiment-with-data A)
  (staged-security-with-data-query-index-set-hit ?B)
  adversary-initial-state
  using checked-staged-security-experiment-with-data-event-from-data-state
  [of A staged-security-with-data-query-index-set-hit ?B]
  by simp
also have ... ≤

```

staged-phase-target-error
 (query-index-raw-preimage (query-index-list-entries Q))
 (staged-attacker-query-budget budgets + staged-challenge-query-budget)
by (rule checked-staged-security-with-data-query-index-set-hit-bound
 [OF wf controlled])
finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-query-projection:

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and projection: fst ‘ $P \subseteq Q$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . trace-fri-query-challenge-pair-set-hit s P))
adversary-initial-state $\leq$
staged-phase-target-error
 (query-index-raw-preimage (query-index-list-entries Q))
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)

proof (rule order-trans)

show *wp-event* (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . trace-fri-query-challenge-pair-set-hit s P))
adversary-initial-state $\leq$

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . trace-fri-query-index-list-set-hit s Q))
adversary-initial-state

by (rule wp-event-mono-on-support)
 (auto simp: staged-security-with-data-state-verifier-event-def
 intro:
 trace-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit
 [OF - projection]
 split: option.splits prod.splits)

next

show *wp-event* (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . trace-fri-query-index-list-set-hit s Q))
adversary-initial-state
 $\leq$ *staged-phase-target-error*
 (query-index-raw-preimage (query-index-list-entries Q))
 (staged-attacker-query-budget budgets + staged-challenge-query-budget)
apply (rule checked-staged-security-trace-fri-query-index-list-set-hit-bound
 [OF wf controlled]) .

qed

lemma checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-query-projection:

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *projection*: $\bigwedge dg. fst \text{ ' } P \text{ } dg \subseteq Q$
shows
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s. composition-fri-query-challenge-pair-set-hit \text{ } s \text{ } P$))
adversary-initial-state $\leq$
staged-phase-target-error
(*query-index-row-preimage* (*query-index-list-entries Q*))
(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget)
proof (*rule order-trans*)
show *wp-event* (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s. composition-fri-query-challenge-pair-set-hit \text{ } s \text{ } P$))
adversary-initial-state $\leq$
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s. composition-fri-query-index-list-set-hit \text{ } s \text{ } Q$))
adversary-initial-state
by (*rule wp-event-mono-on-support*)
(*auto simp*: *staged-security-with-data-state-verifier-event-def*
intro:
composition-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit
[*OF* - *projection*]
split: *option.splits prod.splits*)
next
show *wp-event* (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s. composition-fri-query-index-list-set-hit \text{ } s \text{ } Q$))
adversary-initial-state
 $\leq$ *staged-phase-target-error*
(*query-index-row-preimage* (*query-index-list-entries Q*))
(*staged-attacker-query-budget budgets* + *staged-challenge-query-budget*)
by (*rule*
checked-staged-security-composition-fri-query-index-list-set-hit-bound
[*OF wf controlled*])
qed

lemma *checked-staged-security-trace-fri-sampled-query-pair-set-hit-bound-from-query-projection*:
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *projection*:
 $\bigwedge trace-table \text{ } roots \text{ } final \text{ } round-layers \text{ } layers.$
 $fst \text{ ' } R \text{ } trace-table \text{ } roots \text{ } final \text{ } round-layers \text{ } layers \subseteq Q$
shows
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*


```

    (λs. trace-fri-sampled-query-pair-set-hit R s))
  adversary-initial-state ≤
  staged-phase-target-error
    (query-index-raw-preimage (query-index-list-entries Q))
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)
proof (rule order-trans)
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     (λs. trace-fri-sampled-query-pair-set-hit R s))
    adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     (λs. trace-fri-query-index-list-set-hit s Q))
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
     intro:
       trace-fri-sampled-query-pair-set-hit-imp-query-index-list-set-hit
       [OF - projection]
     split: option.splits prod.splits)
next
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     (λs. trace-fri-query-index-list-set-hit s Q))
    adversary-initial-state
    ≤ staged-phase-target-error
      (query-index-raw-preimage (query-index-list-entries Q))
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule checked-staged-security-trace-fri-query-index-list-set-hit-bound
    [OF wf controlled])
qed

lemma checked-staged-security-composition-fri-sampled-query-pair-set-hit-bound-from-query-projection:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and projection:
    ∧fri-dg composition-table roots final round-layers layers.
    fst ‘ R fri-dg composition-table roots final round-layers layers
    ⊆ Q
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       (λs. composition-fri-sampled-query-pair-set-hit R s))
      adversary-initial-state ≤
    staged-phase-target-error
      (query-index-raw-preimage (query-index-list-entries Q))
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)

```

```

proof (rule order-trans)
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-sampled-query-pair-set-hit R s))
    adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-query-index-list-set-hit s Q))
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      intro:
        composition-fri-sampled-query-pair-set-hit-imp-query-index-list-set-hit
        [OF - projection]
        split: option.splits prod.splits)
next
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-query-index-list-set-hit s Q))
    adversary-initial-state
     $\leq$  staged-phase-target-error
      (query-index-raw-preimage (query-index-list-entries Q))
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule
    checked-staged-security-composition-fri-query-index-list-set-hit-bound
    [OF wf controlled])
qed

lemma trace-fri-query-challenge-pair-set-hit-split-by-pair-cover:
  assumes hit: trace-fri-query-challenge-pair-set-hit s P out
  shows
    trace-fri-query-challenge-pair-set-hit s
      ( $P \cap (UNIV \times B)$ ) out  $\vee$ 
    trace-fri-query-challenge-pair-set-hit s
      ( $P \cap (UNIV \times (- B))$ ) out
proof -
  show ?thesis
  using hit
  unfolding trace-fri-query-challenge-pair-set-hit-def
  apply (elim exE conjE)
  apply (subgoal-tac
    ( $\text{fri-query-idxs}, \text{trace-bs}$ )  $\in P \cap (UNIV \times B) \vee$ 
    ( $\text{fri-query-idxs}, \text{trace-bs}$ )  $\in P \cap (UNIV \times (- B))$ )
  apply (erule disjE)
  apply (rule disjI1)
  apply (intro exI conjI)
  apply assumption
  apply assumption
  apply (rule disjI2)

```

```

    apply (intro exI conjI)
    apply assumption
    apply assumption
  by simp
qed

lemma composition-fri-query-challenge-pair-set-hit-split-by-pair-cover:
  assumes hit: composition-fri-query-challenge-pair-set-hit s P out
  shows
    composition-fri-query-challenge-pair-set-hit s
      ( $\lambda dg. P dg \cap (UNIV \times B dg)$ ) out  $\vee$ 
    composition-fri-query-challenge-pair-set-hit s
      ( $\lambda dg. P dg \cap (UNIV \times (- B dg))$ ) out
proof -
  show ?thesis
  using hit
  unfolding composition-fri-query-challenge-pair-set-hit-def
  apply (elim exE conjE)
  apply (subgoal-tac
    ( $fri\text{-}query\text{-}idxs, composition\text{-}bs$ )  $\in P fri\text{-}dg \cap (UNIV \times B fri\text{-}dg) \vee$ 
    ( $fri\text{-}query\text{-}idxs, composition\text{-}bs$ )  $\in$ 
     $P fri\text{-}dg \cap (UNIV \times (- B fri\text{-}dg))$ )
  apply (erule disjE)
  apply (rule disjI1)
  apply (intro exI conjI)
  apply assumption
  apply assumption
  apply (rule disjI2)
  apply (intro exI conjI)
  apply assumption
  apply assumption
  by auto
qed

```

```

lemma checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-projection:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B: finite B
  and fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. trace\text{-}fri\text{-}challenge\text{-}list\text{-}fresh\text{-}hit s B$ ))
      adversary-initial-state  $\leq F$ 
  and projection:
    fst ' $(P \cap (UNIV \times (- B))) \subseteq Q$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. trace\text{-}fri\text{-}query\text{-}challenge\text{-}pair\text{-}set\text{-}hit s P$ ))

```

```

    adversary-initial-state ≤
    F +
    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget) +
    staged-phase-target-error
    (query-index-raw-preimage (query-index-list-entries Q))
    (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)
proof –
  let ?Inside = P ∩ (UNIV × B)
  let ?Outside = P ∩ (UNIV × (− B))
  let ?challenge-event =
    staged-security-with-data-state-verifier-event
    (λs. trace-fri-challenge-list-set-hit s B)
  let ?inside-event =
    staged-security-with-data-state-verifier-event
    (λs. trace-fri-query-challenge-pair-set-hit s ?Inside)
  let ?outside-event =
    staged-security-with-data-state-verifier-event
    (λs. trace-fri-query-challenge-pair-set-hit s ?Outside)
  let ?hbudget =
    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  let ?qbudget =
    staged-phase-target-error
    (query-index-raw-preimage (query-index-list-entries Q))
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  have split:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
    (λs. trace-fri-query-challenge-pair-set-hit s P))
    adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
    (λout.
      ?inside-event out ∨ ?outside-event out)
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume hit:
    staged-security-with-data-state-verifier-event
    (λs. trace-fri-query-challenge-pair-set-hit s P) out
  then show
    ?inside-event out ∨ ?outside-event out
  unfolding staged-security-with-data-state-verifier-event-def
  by (auto
    dest: trace-fri-query-challenge-pair-set-hit-split-by-pair-cover
    split: option.splits prod.splits)
  qed
have union-bound:

```

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (λout. ?inside-event out ∨ ?outside-event out)
  adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  ?inside-event adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
  ?outside-event adversary-initial-state
by (rule wp-event-union-bound)
have split-union-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. trace-fri-query-challenge-pair-set-hit s P))
  adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  ?inside-event adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
  ?outside-event adversary-initial-state
apply (rule order-trans)
apply (rule split)
apply (rule union-bound)
done
have sum-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  ?inside-event adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
  ?outside-event adversary-initial-state ≤ (F + ?hbudget) + ?qbudget
proof -
have challenge-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  ?challenge-event adversary-initial-state ≤ F + ?hbudget
by (rule checked-staged-security-trace-fri-challenge-list-set-hit-bound
  [OF wf controlled finite-B fresh-bound])
have inside-projection: snd ' ?Inside ⊆ B
by force
have inside-to-challenge:
wp-event (checked-staged-security-experiment-with-data-state A)
  ?inside-event adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  ?challenge-event adversary-initial-state
by (rule wp-event-mono-on-support)
(auto simp: staged-security-with-data-state-verifier-event-def
intro:
  trace-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit
  [OF - inside-projection]
split: option.splits prod.splits)
have inside-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  ?inside-event adversary-initial-state ≤ F + ?hbudget
apply (rule order-trans)

```

```

    apply (rule inside-to-challenge)
  apply (rule challenge-bound)
done
have outside-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?outside-event adversary-initial-state ≤ ?qbudget
  by (rule
    checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-query-projection
    [OF wf controlled projection])
show ?thesis
  by (rule add-mono[OF inside-bound outside-bound])
qed
show ?thesis
  using order-trans[OF split-union-bound sum-bound]
  by (simp add: add.assoc)
qed

lemma checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-pr
assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
  and fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$ ))
      adversary-initial-state ≤ F
  and projection:
     $\bigwedge dg. \text{fst } ' (P \ dg \cap (UNIV \times (- \ B \ dg))) \subseteq Q$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P$ ))
    adversary-initial-state ≤
      F +
      hash-relation-budget-value
        ( $\sum dg \in (UNIV :: 'f \ \text{set}).$ 
          card (B dg) * ceil-log (maxDegree + 1))
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget) +
      staged-phase-target-error
        (query-index-raw-preimage (query-index-list-entries Q))
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget)
proof -
  let ?Inside =  $\lambda dg. P \ dg \cap (UNIV \times B \ dg)$ 
  let ?Outside =  $\lambda dg. P \ dg \cap (UNIV \times (- \ B \ dg))$ 
  let ?challenge-event =
    staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{composition-fri-challenge-list-set-hit } s \ B$ )

```

```

let ?inside-event =
  staged-security-with-data-state-verifier-event
    ( $\lambda s.$  composition-fri-query-challenge-pair-set-hit  $s$  ?Inside)
let ?outside-event =
  staged-security-with-data-state-verifier-event
    ( $\lambda s.$  composition-fri-query-challenge-pair-set-hit  $s$  ?Outside)
let ?hbudget =
  hash-relation-budget-value
    ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
      card ( $B \ dg$ ) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
let ?qbudget =
  staged-phase-target-error
    (query-index-raw-preimage (query-index-list-entries  $Q$ ))
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
have split:
  wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    (staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-query-challenge-pair-set-hit  $s$   $P$ ))
  adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    ( $\lambda out.$ 
      ?inside-event out  $\vee$  ?outside-event out)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume hit:
    staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-query-challenge-pair-set-hit  $s$   $P$ ) out
  then show
    ?inside-event out  $\vee$  ?outside-event out
  unfolding staged-security-with-data-state-verifier-event-def
  by (auto
    dest: composition-fri-query-challenge-pair-set-hit-split-by-pair-cover
    split: option.splits prod.splits)
qed
have union-bound:
  wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    ( $\lambda out.$  ?inside-event out  $\vee$  ?outside-event out)
  adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    ?inside-event adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    ?outside-event adversary-initial-state
  by (rule wp-event-union-bound)
have split-union-bound:
  wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    (staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-query-challenge-pair-set-hit  $s$   $P$ ))

```

```

    adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?inside-event adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?outside-event adversary-initial-state
  apply (rule order-trans)
  apply (rule split)
  apply (rule union-bound)
done
have sum-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?inside-event adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?outside-event adversary-initial-state ≤ (F + ?hbudget) + ?qbudget
proof -
  have challenge-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?challenge-event adversary-initial-state ≤ F + ?hbudget
    by (rule
        checked-staged-security-composition-fri-challenge-list-set-hit-bound
        [OF wf controlled finite-B fresh-bound])
  have inside-projection:  $\bigwedge dg. \text{snd } dg \subseteq B \text{ dg}$ 
    by force
  have inside-to-challenge:
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?inside-event adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?challenge-event adversary-initial-state
    by (rule wp-event-mono-on-support)
      (auto simp: staged-security-with-data-state-verifier-event-def
        intro:
          composition-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit
          [OF - inside-projection]
          split: option.splits prod.splits)
  have inside-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?inside-event adversary-initial-state ≤ F + ?hbudget
    apply (rule order-trans)
    apply (rule inside-to-challenge)
    apply (rule challenge-bound)
  done
  have outside-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?outside-event adversary-initial-state ≤ ?qbudget
    by (rule
        checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-query-projection
        [OF wf controlled projection])
  show ?thesis
    by (rule add-mono[OF inside-bound outside-bound])

```



```

qed
show ?thesis
  using order-trans[OF split-union-bound sum-bound]
  by (simp add: add.assoc)
qed

lemma checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-projection
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B: finite B
  and fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B$ ))
      adversary-initial-state  $\leq F$ 
  and projection:
    fst '
      ((trace-fri-sampled-query-bad-pair-union  $\cap$ 
        (fri-query-index-list-space  $\times UNIV$ ))  $\cap$ 
        (UNIV  $\times (- B)$ ))  $\subseteq Q$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-sampled-query-bad-candidate)
      adversary-initial-state  $\leq$ 
      F +
      hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets + staged-challenge-query-budget) +
      staged-phase-target-error
      (query-index-raw-preimage (query-index-list-entries Q))
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
  proof -
    let ?P =
      trace-fri-sampled-query-bad-pair-union  $\cap$ 
      (fri-query-index-list-space  $\times UNIV$ )
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-sampled-query-bad-candidate)
    adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ ?P$ ))
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      intro:
        trace-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit
        split: option.splits prod.splits)

```

also have ... $\leq$
 $F +$
 $\text{hash-relation-budget-value } (\text{card } B * \text{ceil-log } \text{clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}) +$
 $\text{staged-phase-target-error}$
 $(\text{query-index-raw-preimage } (\text{query-index-list-entries } Q))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
by (rule
 $\text{checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-projection}$
 $[OF \text{ wf controlled finite-B fresh-bound projection}])$
finally show ?thesis .
qed

lemma *checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-p*
assumes wf: *staged-budget-wellformed budgets*
and controlled: *staged-adversary-controlled budgets A*
and finite-B: $\bigwedge dg. \text{finite } (B \text{ dg})$
and fresh-bound:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B))$
 $\text{adversary-initial-state} \leq F$
and projection:
 $\bigwedge dg. \text{fst '}$
 $((\text{composition-fri-sampled-query-bad-pair-union } dg \cap$
 $(\text{fri-query-index-list-space} \times \text{UNIV})) \cap$
 $(\text{UNIV} \times (- \ B \ dg))) \subseteq Q$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-sampled-query-bad-candidate})$
 $\text{adversary-initial-state} \leq$
 $F +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card } (B \text{ dg}) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}) +$
 $\text{staged-phase-target-error}$
 $(\text{query-index-raw-preimage } (\text{query-index-list-entries } Q))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
proof –
let ?P =
 $\lambda dg. \text{composition-fri-sampled-query-bad-pair-union } dg \cap$
 $(\text{fri-query-index-list-space} \times \text{UNIV})$
have $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

```

    (staged-security-with-data-state-verifier-event
     composition-fri-sampled-query-bad-candidate)
    adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   (λs. composition-fri-query-challenge-pair-set-hit s ?P))
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
   intro:
    composition-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit
    split: option.splits prod.splits)
also have ... ≤
  F +
  hash-relation-budget-value
  (∑ dg ∈ (UNIV :: 'f set).
   card (B dg) * ceil-log (maxDegree + 1))
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget) +
  staged-phase-target-error
  (query-index-row-preimage (query-index-list-entries Q))
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget)
by (rule
  checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-projection
  [OF wf controlled finite-B fresh-bound projection])
finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-generic-cover:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and cover:
  generic-fri-sampled-query-finite-cover trace-table-low-degree
  (Not ◦ trace-table-low-degree) (clength - 1) B
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   (λs. trace-fri-challenge-list-fresh-hit s B))
  adversary-initial-state ≤ F
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-sampled-query-bad-candidate)
  adversary-initial-state ≤
  F +
  hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof -

```

```

have finite-B: finite B
  by (rule generic-fri-sampled-query-finite-coverD(1)[OF cover])
have projection:
  snd ‘
    (trace-fri-sampled-query-bad-pair-union  $\cap$ 
      (fri-query-index-list-space  $\times$  UNIV))  $\subseteq$  B
  using generic-fri-sampled-query-finite-coverD(2)[OF cover]
  unfolding trace-fri-sampled-query-bad-pair-union-def by simp
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-sampled-query-bad-candidate)
  adversary-initial-state  $\leq$ 
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s.$  trace-fri-challenge-list-set-hit s B))
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    intro:
      trace-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit
      [OF trace-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit
        projection])
  split: option.splits prod.splits)
also have ...  $\leq$ 
  F +
  hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)
  by (rule checked-staged-security-trace-fri-challenge-list-set-hit-bound
    [OF wf controlled finite-B fresh-bound])
finally show ?thesis .
qed

```

```

lemma checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-generic-cover:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and cover:
   $\bigwedge dg.$  generic-fri-sampled-query-finite-cover
    (composition-table-low-degree (to-nat dg))
    (Not  $\circ$  composition-table-low-degree maxDegree) (to-nat dg) (B dg)
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s.$  composition-fri-challenge-list-fresh-hit s B))
  adversary-initial-state  $\leq$  F
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-sampled-query-bad-candidate)

```

```

    adversary-initial-state ≤
    F +
    hash-relation-budget-value
    (∑ dg ∈ (UNIV :: 'f set).
      card (B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)
  proof -
  have finite-B: ∧dg. finite (B dg)
  by (rule generic-fri-sampled-query-finite-coverD(1)[OF cover])
  have projection:
    ∧dg. snd '
      (composition-fri-sampled-query-bad-pair-union dg ∩
       (fri-query-index-list-space × UNIV)) ⊆ B dg
  proof -
  fix dg
  show snd '
    (composition-fri-sampled-query-bad-pair-union dg ∩
     (fri-query-index-list-space × UNIV)) ⊆ B dg
  using generic-fri-sampled-query-finite-coverD(2)[OF cover[of dg]]
  unfolding composition-fri-sampled-query-bad-pair-union-def by simp
  qed
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-fri-sampled-query-bad-candidate)
    adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     (λs. composition-fri-challenge-list-set-hit s B))
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    intro:
      composition-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit
      [OF composition-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit
        projection]
      split: option.splits prod.splits)
  also have ... ≤
    F +
    hash-relation-budget-value
    (∑ dg ∈ (UNIV :: 'f set).
      card (B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)
  by (rule checked-staged-security-composition-fri-challenge-list-set-hit-bound
    [OF wf controlled finite-B fresh-bound])
  finally show ?thesis .
  qed

```

lemma *checked-staged-security-trace-fri-sampled-layer-chain-bound-from-sampled-query:*

assumes *sampled-query:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-sampled-query-bad-candidate)
adversary-initial-state ≤ R

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-sampled-layer-chain)
adversary-initial-state ≤ R

proof —

have *missing:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-sampled-missing-canonical-sampled-layer-chain)
adversary-initial-state ≤ 0

proof (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)

show $\bigwedge s. \neg \text{trace-fri-sampled-missing-canonical-sampled-layer-chain } s \text{ None}$

unfolding *trace-fri-sampled-missing-canonical-sampled-layer-chain-def*
trace-fri-bad-with-sampled-layer-chain-def
accepted-fri-opening-transcript-def
trace-fri-partial-candidate-opening-evidence-def
trace-fri-partial-candidate-evidence-def

by *blast*

next

fix *data attacker-state*

assume

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)

show *wp-event verify-monad*

(trace-fri-sampled-missing-canonical-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ 0

by (rule *wp-trace-fri-missing-canonical-sampled-zero*)

qed

have *wp-event (checked-staged-security-experiment-with-data-state A)*

(staged-security-with-data-state-verifier-event
trace-fri-bad-with-sampled-layer-chain)
adversary-initial-state ≤

wp-event (checked-staged-security-experiment-with-data-state A)
(λout.

staged-security-with-data-state-verifier-event
trace-fri-sampled-query-bad-candidate out ∨
staged-security-with-data-state-verifier-event
trace-fri-sampled-missing-canonical-sampled-layer-chain out)

```

    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      trace-fri-sampled-query-bad-candidate-iff-canonical
      dest: trace-fri-sampled-layer-chain-canonical-or-missing
      split: option.splits prod.splits)
  also have ...  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-sampled-query-bad-candidate)
    adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-sampled-missing-canonical-sampled-layer-chain)
    adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq R + 0$ 
  by (rule add-mono[OF sampled-query missing])
  finally show ?thesis
  by simp
qed

lemma checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-sampled-query:
assumes sampled-query:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-sampled-query-bad-candidate)
  adversary-initial-state  $\leq R$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-verifier-tied-sampled-layer-chain)
  adversary-initial-state  $\leq R$ 
proof –
have missing:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-sampled-missing-canonical-sampled-layer-chain)
  adversary-initial-state  $\leq 0$ 
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
show
   $\bigwedge s. \neg \text{composition-fri-sampled-missing-canonical-sampled-layer-chain}$ 
  s None
unfolding composition-fri-sampled-missing-canonical-sampled-layer-chain-def
  composition-fri-bad-with-sampled-layer-chain-def
  accepted-fri-opening-transcript-def
  composition-fri-partial-candidate-opening-evidence-def
  composition-fri-partial-candidate-evidence-def
by blast

```

```

next
  fix data attacker-state
  assume
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  show wp-event verify-monad
    (composition-fri-sampled-missing-canonical-sampled-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))  $\leq 0$ 
  by (rule wp-composition-fri-missing-canonical-sampled-zero)
qed
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-bad-with-verifier-tied-sampled-layer-chain)
  adversary-initial-state  $\leq$ 
wp-event (checked-staged-security-experiment-with-data-state A)
  ( $\lambda out.$ 
    staged-security-with-data-state-verifier-event
      composition-fri-sampled-query-bad-candidate out  $\vee$ 
      staged-security-with-data-state-verifier-event
        composition-fri-sampled-missing-canonical-sampled-layer-chain out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    composition-fri-sampled-query-bad-candidate-iff-canonical
    dest!: composition-fri-verifier-tied-sampled-layer-chain-imp-sampled-layer-chain
    dest: composition-fri-sampled-layer-chain-canonical-or-missing
    split: option.splits prod.splits)
also have ...  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-sampled-query-bad-candidate)
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-sampled-missing-canonical-sampled-layer-chain)
  adversary-initial-state
  by (rule wp-event-union-bound)
also have ...  $\leq R + 0$ 
  by (rule add-mono[OF sampled-query missing])
finally show ?thesis
  by simp
qed

lemma checked-staged-security-trace-fri-empty-header-bound-from-sampled-query-conflict-and-zero:
  assumes sampled-query:

```


$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-sampled-query-bad-candidate)
 $adversary\text{-}initial\text{-}state \leq R$
and *conflict-bound*:
 $\wedge data$ *attacker-state*.
 Some ($data$, *attacker-state*) $\in$
 $set\text{-}dist$ (*execute* (*checked-staged-transcript-program* A)
 $adversary\text{-}initial\text{-}state$) $\implies$
 $wp\text{-}event$ *verify-monad*
 (*trace-fri-sampled-assignment-conflict*
 (*verifier-state-from-adversary* *attacker-state*
 (*staged-proof-transcript* $data$)))
 (*verifier-state-from-adversary* *attacker-state*
 (*staged-proof-transcript* $data$)) $\leq C$
and *zero-bound*:
 $\wedge data$ *attacker-state*.
 Some ($data$, *attacker-state*) $\in$
 $set\text{-}dist$ (*execute* (*checked-staged-transcript-program* A)
 $adversary\text{-}initial\text{-}state$) $\implies$
 $wp\text{-}event$ *verify-monad*
 (*trace-fri-zero-round-final-obstruction*
 (*verifier-state-from-adversary* *attacker-state*
 (*staged-proof-transcript* $data$)))
 (*verifier-state-from-adversary* *attacker-state*
 (*staged-proof-transcript* $data$)) $\leq Z$
shows
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
 $adversary\text{-}initial\text{-}state \leq R + (C + Z)$
proof (*rule checked-staged-security-trace-fri-empty-header-bound-from-sampled-and-missing*)
show $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-sampled-layer-chain)
 $adversary\text{-}initial\text{-}state \leq R$
by (*rule*
checked-staged-security-trace-fri-sampled-layer-chain-bound-from-sampled-query
 [*OF* *sampled-query*])
next
fix $data$ *attacker-state*
assume *support*:
 Some ($data$, *attacker-state*) $\in$
 $set\text{-}dist$ (*execute* (*checked-staged-transcript-program* A)
 $adversary\text{-}initial\text{-}state$)
let $?s =$
verifier-state-from-adversary *attacker-state*
 (*staged-proof-transcript* $data$)
have *obstruction*:

$wp\text{-event verify-monad}$
 $(\text{trace-fri-sampled-layer-assignment-obstruction } ?s) ?s \leq C + Z$
by (rule
 $wp\text{-trace-fri-sampled-layer-assignment-obstruction-bound-from-conflict-and-zero}$
 $(\text{rule conflict-bound}[OF \text{ support}], \text{rule zero-bound}[OF \text{ support}])$)
show $wp\text{-event verify-monad}$
 $(\text{trace-fri-reachable-missing-sampled-layer-chain } ?s) ?s \leq C + Z$
by (rule $wp\text{-trace-fri-reachable-missing-sampled-bound-from-assignment-obstruction}$)
 $(\text{rule obstruction})$
qed

lemma $wp\text{-composition-fri-verifier-tied-missing-sampled-layer-chain-bound-from-residual-gaps}$:
assumes slot-bound :
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap } s) s$
 $\leq \text{Slot}$
and merkle-bound :
 $wp\text{-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) s$
 $\leq \text{Merkle}$
and missing-bound :
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-recorded-base-chunk-auth-missing } s) s$
 $\leq \text{Missing}$
and next-bound :
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap } s) s$
 $\leq \text{Next}$
and successor-bound :
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap } s) s$
 $\leq \text{Successor}$
and final-bound :
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap } s) s$
 $\leq \text{Final}$
and zero-bound :
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-zero-round-final-obstruction } s) s$
 $\leq \text{Zero}$
shows
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-missing-sampled-layer-chain } s) s$
 $\leq$
 $((\text{Merkle} + \text{Missing}) + \text{Next} + \text{Successor} + \text{Final} + \text{Merkle}) +$
 $\text{Slot} + \text{Zero})$
proof –
have conflict :
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-assignment-conflict } s) s$

$\leq ((\text{Merkle} + \text{Missing}) + \text{Next} + \text{Successor} + \text{Final} + \text{Merkle}) + \text{Slot}$
by (rule
wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-slot-recorded-chunk-gap-and-residuals
 $[\text{OF slot-bound merkle-bound missing-bound next-bound successor-bound final-bound}]$)
have *obstruction*:
wp-event verify-monad
 $(\text{composition-fri-verifier-tied-sampled-layer-assignment-obstruction } s) \ s$
 $\leq$
 $((\text{Merkle} + \text{Missing}) + \text{Next} + \text{Successor} + \text{Final} + \text{Merkle}) +$
 $\text{Slot} + \text{Zero}$
by (rule
wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero
 $(\text{rule conflict, rule zero-bound})$)
show *?thesis*
by (rule
wp-composition-fri-verifier-tied-missing-sampled-bound-from-assignment-obstruction
 $(\text{rule obstruction})$)
qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-sampled-query-and-residual*
assumes *sampled-query*:
wp-event $(\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-sampled-query-bad-candidate})$
 $\text{adversary-initial-state} \leq R$
and *slot-bound*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
wp-event verify-monad
 $(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Slot}$
and *merkle-bound*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
wp-event verify-monad
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Merkle}$
and *missing-bound*:

$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-recorded-base-chunk-auth-missing
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Missing
and next-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-next-value-replay-gap
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Next
and successor-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-successor-replay-gap
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Successor
and final-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-final-value-replay-gap
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Final
and zero-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad

(composition-fri-verifier-tied-zero-round-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Zero

shows

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state $\leq$
 R + (((Merkle + Missing) + Next + Successor + Final + Merkle) +
 Slot + Zero)

proof –

have sampled:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-sampled-layer-chain)
 adversary-initial-state $\leq$ R

by (rule
 checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-sampled-query
 [OF sampled-query])

have missing:

$\bigwedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-missing-sampled-layer-chain
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 (((Merkle + Missing) + Next + Successor + Final + Merkle) +
 Slot + Zero)

proof –

fix data attacker-state

assume support:

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state)

let ?s =

verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)

show wp-event verify-monad
 (composition-fri-verifier-tied-missing-sampled-layer-chain ?s) ?s
 $\leq$
 (((Merkle + Missing) + Next + Successor + Final + Merkle) +
 Slot + Zero)

by (rule

```

      wp-composition-fri-verifier-tied-missing-sampled-layer-chain-bound-from-residual-gaps)
(rule slot-bound[OF support],
rule merkle-bound[OF support],
rule missing-bound[OF support],
rule next-bound[OF support],
rule successor-bound[OF support],
rule final-bound[OF support],
rule zero-bound[OF support])
qed
show ?thesis
by (rule
checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-sampled-and-missing
[OF sampled missing])
qed
end
end

```

```

theory Soundness-Public-Route-Active-Base
imports
  Soundness-Public-Route-Single-Query
  Soundness-FRI-Query-Exact-Bounds
  Soundness-FRI-Query-Index-Staged-Bounds
begin

```

Base active-FRI public-route endpoint.

This theory is deliberately upstream of the downstream FRI route adapters. It exposes the common route theorem from staged active FRI event bounds, without importing the final same-run endpoint theory. This avoids an import cycle when later route layers are consumed by the final public endpoint.

```

context soundness
begin

```

```

definition active-route-transcript-pre-error-for
where

```

```

  active-route-transcript-pre-error-for A =
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-pre-hit
      adversary-initial-state

```

```

definition active-route-current-empty-query-error-for
where

```

```

  active-route-current-empty-query-error-for A =
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
      adversary-initial-state

```

definition *active-route-current-prefix-authenticated-error-for*
where

active-route-current-prefix-authenticated-error-for $A\ i =$
wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* $A\ i$)
 (*checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at*
 i)
adversary-initial-state

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-bounds:*

fixes *trace-fri-error'* *composition-fri-error'*

empty-trace-fri-error' :: *prob*

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets* A

and *trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state $\leq$ *trace-fri-error'*

and *comp-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state $\leq$ *composition-fri-error'*

and *empty-trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *empty-trace-fri-error'*

shows *checked-staged-adversary-acceptance-probability* $A \leq$

hash-collision-budget-value 0

(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +

(*composition-error-bound* +

staged-phase-relation-error *size*

(*staged-alpha-search-queries* *budgets* 0) +

hash-collision-budget-value 0

(*staged-alpha-search-queries* *budgets* 0) +

staged-phase-relation-error *size*

(*staged-alpha-search-queries* *budgets* 0) +

(*active-route-transcript-pre-error-for* A +

staged-concrete-transcript-target-error-bound) +

($\sum i < \text{rounds}$.

active-route-current-prefix-authenticated-error-for $A\ i$ +

(*active-route-transcript-pre-error-for* A +

staged-concrete-transcript-target-error-bound) +

(*active-route-transcript-pre-error-for* A +

staged-concrete-transcript-target-error-bound)) +

$\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(\text{active-route-transcript-pre-error-for } A +$
 $\text{staged-concrete-transcript-target-error-bound}) +$
 $\text{empty-trace-fri-error}' + \text{active-route-current-empty-query-error-for } A)$

proof –

have *transcript-new-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-new-hit}$
 $\text{adversary-initial-state} \leq \text{staged-concrete-transcript-target-error-bound}$

by (rule *checked-staged-security-with-data-state-transcript-new-hit-bound*
[OF *wf controlled*])

have *data-pre-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq \text{active-route-transcript-pre-error-for } A$

by (simp add: *active-route-transcript-pre-error-for-def*)

have *current-empty-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{active-route-current-empty-query-error-for } A$

by (simp add: *active-route-current-empty-query-error-for-def*)

have *prefix-bound*:

$\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at } i)$
 $\text{adversary-initial-state} \leq$
 $\text{active-route-current-prefix-authenticated-error-for } A \ i$

by (simp add: *active-route-current-prefix-authenticated-error-for-def*)

have *trace-path-bound*:

$\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq$
 $\text{active-route-transcript-pre-error-for } A +$
 $\text{staged-concrete-transcript-target-error-bound}$

by (rule
checked-staged-security-with-query-prefix-current-trace-path-output-hit-at-bound-from-transcript
[OF - *data-pre-bound transcript-new-bound*])

have *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at}$
 $\quad i)$
 $\text{adversary-initial-state} \leq$
 $\text{active-route-transcript-pre-error-for } A +$
 $\text{staged-concrete-transcript-target-error-bound}$
by (rule
 $\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at-bound-from-transcript}$
 $[\text{OF - data-pre-bound transcript-new-bound}])$
define $C :: \text{nat} \Rightarrow \text{prob}$ **where**
 $C =$
 $(\lambda i. \text{active-route-current-prefix-authenticated-error-for } A \ i +$
 $\quad (\text{active-route-transcript-pre-error-for } A +$
 $\quad \text{staged-concrete-transcript-target-error-bound}) +$
 $\quad (\text{active-route-transcript-pre-error-for } A +$
 $\quad \text{staged-concrete-transcript-target-error-bound}))$
have *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at}$
 $\quad i)$
 $\text{adversary-initial-state} \leq C \ i$
proof –
fix i
assume $i\text{-bound}: i < \text{rounds}$
show
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at}$
 $\quad i)$
 $\text{adversary-initial-state} \leq C \ i$
unfolding $C\text{-def}$
by (rule
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-structured-}$
 $[\text{OF wf controlled } i\text{-bound prefix-bound}[\text{OF } i\text{-bound}]$
 $\quad \text{trace-path-bound}[\text{OF } i\text{-bound}] \text{ composition-path-bound}[\text{OF } i\text{-bound}]])$
qed
have *route*:
 $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\quad \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\quad \text{staged-phase-relation-error size}$
 $\quad (\text{staged-alpha-search-queries budgets } 0) +$

```

    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    (active-route-transcript-pre-error-for A +
     staged-concrete-transcript-target-error-bound)) +
    trace-fri-error' + composition-fri-error' +
    (∑ i<rounds. C i) +
    (composition-error-bound +
     staged-phase-relation-error size
     (staged-alpha-search-queries budgets 0) +
     hash-collision-budget-value 0
     (staged-alpha-search-queries budgets 0) +
     staged-phase-relation-error size
     (staged-alpha-search-queries budgets 0) +
     (active-route-transcript-pre-error-for A +
      staged-concrete-transcript-target-error-bound) +
     empty-trace-fri-error' +
     active-route-current-empty-query-error-for A)
  by (rule
    stark-soundness-from-transcript-indexed-partial-opening-route-from-sampled-transcript-relevant-drift-only-single-
    [OF false-statement wf controlled round-bound data-pre-bound
     current-empty-bound trace-fri-bound comp-fri-bound
     empty-trace-fri-bound])
  then show ?thesis
  unfolding C-def by (simp add: algebra-simps)
qed

end

end

```

```

theory Soundness-FRI-Sampled-Interface-Conflict-Zero
  imports Soundness-FRI-Sampled-Interface
begin

```

Conflict/zero active-FRI wrapper for the sampled-query interface.

This narrow layer keeps the sampled-interface theory from growing further while exposing the cleaner trace route that does not carry a separate recorded-sibling-missing component.

```

context soundness
begin

```

```

lemma active-fri-reductions-for-from-trace-augmented-conflict-zero-composition-sampled-query-and-empty-conflict-zero
  fixes TraceAug TraceConflict TraceZero TraceMerkle CompSampled CompMerkle
    EmptySampled EmptyConflict EmptyZero ::
    'f staged-proof-data ⇒ 'f protocol-channel ⇒ prob
  assumes trace-augmented-bound:

```

$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-augmented-partial-candidate
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ TraceAug data attacker-state
and trace-conflict-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ TraceConflict data attacker-state
and trace-zero-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-header-tied-zero-round-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ TraceZero data attacker-state
and trace-merkle-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ TraceMerkle data attacker-state
and trace-total:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 TraceAug data attacker-state +

$((\text{TraceConflict data attacker-state} + \text{TraceZero data attacker-state}) + \text{TraceMerkle data attacker-state})$
 $\leq \text{trace-fri-error}$
and *composition-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-query-bad-candidate}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{CompSampled data attacker-state}$
and *composition-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{CompMerkle data attacker-state}$
and *composition-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{CompSampled data attacker-state} +$
 $((\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0 + 0) +$
 $\text{CompMerkle data attacker-state}) + 0)$
 $\leq \text{composition-fri-error}$
and *empty-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-query-bad-candidate}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$

```

    EmptySampled data attacker-state
and empty-conflict-bound:
 $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-sampled-assignment-conflict
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$ 
    EmptyConflict data attacker-state
and empty-zero-bound:
 $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-zero-round-final-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$ 
    EmptyZero data attacker-state
and empty-total:
 $\wedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
    EmptySampled data attacker-state +
      (EmptyConflict data attacker-state + EmptyZero data attacker-state)
       $\leq$  trace-fri-error
shows active-fri-reductions-for A
proof (rule
  active-fri-reductions-for-from-trace-augmented-conflict-zero-composition-sampled-and-empty-conflict-zero)
fix data attacker-state
assume support:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show wp-event verify-monad
  (trace-fri-bad-with-header-tied-augmented-partial-candidate ?s) ?s
   $\leq$  TraceAug data attacker-state
by (rule trace-augmented-bound[OF support])
show wp-event verify-monad

```

```

    (trace-fri-header-tied-sampled-assignment-conflict ?s) ?s
    ≤ TraceConflict data attacker-state
  by (rule trace-conflict-bound[OF support])
show wp-event verify-monad
    (trace-fri-header-tied-zero-round-final-obstruction ?s) ?s
    ≤ TraceZero data attacker-state
  by (rule trace-zero-bound[OF support])
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
    ≤ TraceMerkle data attacker-state
  by (rule trace-merkle-bound[OF support])
show TraceAug data attacker-state +
    ((TraceConflict data attacker-state + TraceZero data attacker-state) +
     TraceMerkle data attacker-state)
    ≤ trace-fri-error
  by (rule trace-total[OF support])
show wp-event verify-monad
    (composition-fri-bad-with-verifier-tied-sampled-layer-chain ?s) ?s
    ≤ CompSampled data attacker-state
  by (rule wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-sampled)
    (rule wp-composition-fri-sampled-bound-from-canonical,
     simp add: wp-composition-fri-sampled-query-bad-candidate-iff-canonical)
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
    ≤ CompMerkle data attacker-state
  by (rule composition-merkle-bound[OF support])
show CompSampled data attacker-state +
    (((CompMerkle data attacker-state + 0) +
     (CompMerkle data attacker-state + 0) +
     (CompMerkle data attacker-state + 0) +
     (CompMerkle data attacker-state + 0 + 0) +
     CompMerkle data attacker-state) + 0)
    ≤ composition-fri-error
  by (rule composition-total[OF support])
show wp-event verify-monad
    (trace-fri-bad-with-sampled-layer-chain ?s) ?s
    ≤ EmptySampled data attacker-state
  by (rule wp-trace-fri-sampled-bound-from-canonical)
    (insert empty-sampled-bound[OF support],
     simp add: wp-trace-fri-sampled-query-bad-candidate-iff-canonical)
show wp-event verify-monad
    (trace-fri-sampled-assignment-conflict ?s) ?s
    ≤ EmptyConflict data attacker-state
  by (rule empty-conflict-bound[OF support])
show wp-event verify-monad
    (trace-fri-zero-round-final-obstruction ?s) ?s
    ≤ EmptyZero data attacker-state
  by (rule empty-zero-bound[OF support])
show EmptySampled data attacker-state +
    (EmptyConflict data attacker-state + EmptyZero data attacker-state)

```

```

      ≤ trace-fri-error
    by (rule empty-total[OF support])
qed

end

end

```

```

theory Soundness-Public-Route-Same-Run
imports
  Soundness-Public-Route-Active-Base
  Soundness-FRI-Sampled-Interface-Conflict-Zero
begin

```

Public-route infrastructure for the staged soundness theorem. This layer assembles the checked current-empty and transcript-empty reductions and states them in the plain locale *soundness*. FRI losses are expressed as concrete staged verifier-event probabilities that are bounded in later FRI layers.

```

context soundness
begin

```

```

lemma current-query-route-bound-from-active-fri-bounds-internal:
  fixes trace-fri-error' composition-fri-error'
    empty-trace-fri-error' :: prob
  assumes false-statement: ¬ exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-header-tied-partial-candidate)
      adversary-initial-state ≤ trace-fri-error'
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-verifier-tied-partial-candidate)
      adversary-initial-state ≤ composition-fri-error'
  and empty-trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-empty-composition-header-candidates)
      adversary-initial-state ≤ empty-trace-fri-error'
  shows checked-staged-adversary-acceptance-probability A ≤
    hash-collision-budget-value 0
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget + verifier-hash-query-budget) +
    (composition-error-bound +
      staged-phase-relation-error size

```

(staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 ($\sum i < \text{rounds}$.
 1 +
 (1 + staged-concrete-transcript-target-error-bound) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 trace-fri-error' + composition-fri-error' +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound) +
 empty-trace-fri-error' + 1)
proof –
 let ?exact =
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 ($\sum i < \text{rounds}$.
 active-route-current-prefix-authenticated-error-for A i +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 trace-fri-error' + composition-fri-error' +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +

$staged-concrete-transcript-target-error-bound) +$
 $empty-trace-fri-error' + active-route-current-empty-query-error-for A)$
have *route*:
checked-staged-adversary-acceptance-probability $A \leq ?exact$
by (rule
stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-bounds
 $[OF\ false-statement\ wf\ controlled\ trace-fri-bound\ comp-fri-bound$
 $empty-trace-fri-bound]$)
have *exact-le-route*:
 $?exact \leq$
 $hash-collision-budget-value\ 0$
 $(staged-attacker-query-budget\ budgets +$
 $staged-challenge-query-budget + verifier-hash-query-budget) +$
 $(composition-error-bound +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $(1 + staged-concrete-transcript-target-error-bound)) +$
 $(\sum_{i < rounds}.$
 $1 +$
 $(1 + staged-concrete-transcript-target-error-bound) +$
 $(1 + staged-concrete-transcript-target-error-bound)) +$
 $trace-fri-error' + composition-fri-error' +$
 $(composition-error-bound +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $(1 + staged-concrete-transcript-target-error-bound) +$
 $empty-trace-fri-error' + 1)$
unfolding *active-route-transcript-pre-error-for-def*
active-route-current-prefix-authenticated-error-for-def
active-route-current-empty-query-error-for-def
by (intro *add-mono sum-mono order-refl wp-event-le-1*)
show *?thesis*
by (rule *order-trans*[*OF route exact-le-route*])
qed

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-staged-active-fri-bounds*:
fixes *trace-bad* ::
 $'f\ list \Rightarrow nat \Rightarrow 'f\ list \Rightarrow 'f\ set$
and *composition-bad* ::
 $'f \Rightarrow 'f\ list \Rightarrow nat \Rightarrow 'f\ list \Rightarrow 'f\ set$
and *trace-B* :: $'f\ list\ set$

and *composition-B* :: 'f $\Rightarrow$ 'f list set
and *trace-fresh* *trace-merkle* *trace-missing* *empty-fresh*
empty-assignment *composition-fresh* *composition-merkle* :: prob
and *trace-fri-error'* *composition-fri-error'*
empty-trace-fri-error' :: prob
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *trace-finite*: *finite* *trace-B*
and *trace-cover*:
 $\wedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\Rightarrow$
trace-fri-sampled-layer-chain-cover
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)) *trace-bad*
and *trace-envelope*:
 $\wedge$ *data* *attacker-state* *trace-table*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\Rightarrow$
trace-fri-multiround-bad-sets *trace-bad* *trace-table* $\subseteq$ *trace-B*
and *trace-fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
(λs . *trace-fri-challenge-list-fresh-hit* *s* *trace-B*))
adversary-initial-state $\leq$ *trace-fresh*
and *trace-merkle-bound*:
 $\wedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\Rightarrow$
wp-event *verify-monad*
(*partial-merkle-inconsistency-bad*
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)))
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)) $\leq$ *trace-merkle*
and *trace-missing-bound*:
 $\wedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\Rightarrow$
wp-event *verify-monad*
(*trace-fri-header-tied-recorded-sibling-candidate-missing*
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)))
(*verifier-state-from-adversary* *attacker-state*

$(\text{staged-proof-transcript data})) \leq \text{trace-missing}$
and *trace-total*:
 $(\text{trace-fresh} +$
 $\quad \text{hash-relation-budget-value (card trace-B * ceil-log clength)}$
 $\quad (\text{staged-attacker-query-budget budgets} +$
 $\quad \quad \text{staged-challenge-query-budget})) +$
 $((\text{trace-missing} +$
 $\quad (((\text{trace-merkle} + \text{trace-merkle}) + (\text{trace-merkle} + 0) +$
 $\quad (\text{trace-merkle} + 0) + (\text{trace-merkle} + 0 + 0)) +$
 $\quad \text{trace-merkle}) + 0)) +$
 $(1 + \text{trace-merkle})) +$
 $(\text{trace-missing} + \text{trace-merkle})$
 $\leq \text{trace-fri-error'}$
and *empty-fresh-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\quad (\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \text{ trace-B}))$
 $\text{adversary-initial-state} \leq \text{empty-fresh}$
and *empty-assignment-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\quad \text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-layer-assignment-obstruction}$
 $\quad (\text{verifier-state-from-adversary attacker-state}$
 $\quad \quad (\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $\quad (\text{staged-proof-transcript data})) \leq \text{empty-assignment}$
and *empty-total*:
 $(\text{empty-fresh} +$
 $\quad \text{hash-relation-budget-value (card trace-B * ceil-log clength)}$
 $\quad (\text{staged-attacker-query-budget budgets} +$
 $\quad \quad \text{staged-challenge-query-budget})) + \text{empty-assignment}$
 $\leq \text{empty-trace-fri-error'}$
and *composition-finite*: $\wedge \text{dg. finite (composition-B dg)}$
and *composition-cover*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\quad \text{adversary-initial-state})} \implies$
 $\text{composition-fri-sampled-layer-chain-cover}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $\quad (\text{staged-proof-transcript data})) \text{ composition-bad}$
and *composition-envelope*:
 $\wedge \text{data attacker-state dg composition-table.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\quad \text{adversary-initial-state})} \implies$

$\text{composition-fri-multiround-bad-sets } \text{composition-bad } dg \text{ composition-table}$
 $\subseteq \text{composition-B } dg$
and $\text{composition-fresh-bound:}$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \text{ composition-B}))$
 $\text{adversary-initial-state} \leq \text{composition-fresh}$
and $\text{composition-merkle-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-merkle}$
and $\text{composition-total:}$
 $(\text{composition-fresh} +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card } (\text{composition-B } dg) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $((\text{composition-merkle} + 0) +$
 $(\text{composition-merkle} + 0) + (\text{composition-merkle} + 0) +$
 $(\text{composition-merkle} + 0 + 0) + \text{composition-merkle}) + 0 + 0)$
 $\leq \text{composition-fri-error'}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\sum i < \text{rounds.}$
 $1 +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $\text{trace-fri-error' } + \text{composition-fri-error' } +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$

```

hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
(1 + staged-concrete-transcript-target-error-bound) +
empty-trace-fri-error' + 1)
proof -
have trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state ≤ trace-fri-error'
proof (rule order-trans)
show wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state ≤
    ((trace-fresh +
      hash-relation-budget-value (card trace-B * ceil-log clength)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
    ((trace-missing +
      (((trace-merkle + trace-merkle) + (trace-merkle + 0) +
        (trace-merkle + 0) + (trace-merkle + 0 + 0)) +
        trace-merkle) + 0))) +
    (1 + trace-merkle))) +
    (trace-missing + trace-merkle)
by (rule
  checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-augmented-route-and-unaugmented-s
  [OF wf controlled trace-finite trace-cover trace-envelope
    trace-fresh-bound trace-merkle-bound trace-missing-bound
    wp-event-le-1])
show ... ≤ trace-fri-error'
by (rule trace-total)
qed
have empty-trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state ≤ empty-trace-fri-error'
proof (rule order-trans)
show wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state ≤
    (empty-fresh +
      hash-relation-budget-value (card trace-B * ceil-log clength)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) + empty-assignment

```

```

    by (rule
      checked-staged-security-trace-fri-empty-header-bound-from-fresh-prequery-and-assignment-obstruction
      [OF wf controlled trace-finite trace-cover trace-envelope
        empty-fresh-bound empty-assignment-bound])
  show ... ≤ empty-trace-fri-error'
    by (rule empty-total)
qed
have comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-verifier-tied-partial-candidate)
  adversary-initial-state ≤ composition-fri-error'
proof (rule order-trans)
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-verifier-tied-partial-candidate)
  adversary-initial-state ≤
    (composition-fresh +
      hash-relation-budget-value
      (∑ dg ∈ (UNIV :: 'f set).
        card (composition-B dg) * ceil-log (maxDegree + 1))
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
    (((composition-merkle + 0) +
      (composition-merkle + 0) + (composition-merkle + 0) +
      (composition-merkle + 0 + 0) + composition-merkle) + 0 + 0)
  by (rule
    checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-zero-slot-missin
    [OF wf controlled composition-finite composition-cover
      composition-envelope composition-fresh-bound composition-merkle-bound])
  show ... ≤ composition-fri-error'
    by (rule composition-total)
qed
show ?thesis
  by (rule
    current-query-route-bound-from-active-fri-bounds-internal
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      empty-trace-fri-bound])
qed

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-residual-obliga
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and residuals:
  active-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft
  Rc Ac Pc Nc Sc Gc
shows checked-staged-adversary-acceptance-probability A ≤
  hash-collision-budget-value 0

```

(staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 ($\sum_{i < \text{rounds.}}$
 1 +
 (1 + staged-concrete-transcript-target-error-bound) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 trace-fri-error + composition-fri-error +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound) +
 trace-fri-error + 1)

proof –

have trace-fri-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state $\leq$ trace-fri-error

by (rule active-fri-residual-obligations-staged-trace-bound[OF residuals])

have comp-fri-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state $\leq$ composition-fri-error

by (rule
 active-fri-residual-obligations-staged-composition-bound[OF residuals])

have empty-trace-fri-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ trace-fri-error

by (rule
 active-fri-residual-obligations-staged-empty-header-bound[OF residuals])

show ?thesis

by (rule
 current-query-route-bound-from-active-fri-bounds-internal
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound])

empty-trace-fri-bound)

qed

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-residual-compo*

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *trace-residuals*:

active-trace-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft

and *composition-residuals*:

active-composition-fri-residual-obligations-for A Rc Ac Pc Nc Sc Gc

and *empty-trace*: *active-empty-trace-fri-reduction-for A*

shows *checked-staged-adversary-acceptance-probability A* $\leq$

hash-collision-budget-value 0

(*staged-attacker-query-budget budgets* +

staged-challenge-query-budget + *verifier-hash-query-budget*) +

(*composition-error-bound* +

staged-phase-relation-error size

(*staged-alpha-search-queries budgets 0*) +

hash-collision-budget-value 0

(*staged-alpha-search-queries budgets 0*) +

staged-phase-relation-error size

(*staged-alpha-search-queries budgets 0*) +

(*1* + *staged-concrete-transcript-target-error-bound*)) +

($\sum$ *i* < *rounds*.

1 +

(*1* + *staged-concrete-transcript-target-error-bound*) +

(*1* + *staged-concrete-transcript-target-error-bound*)) +

trace-fri-error + *composition-fri-error* +

(*composition-error-bound* +

staged-phase-relation-error size

(*staged-alpha-search-queries budgets 0*) +

hash-collision-budget-value 0

(*staged-alpha-search-queries budgets 0*) +

staged-phase-relation-error size

(*staged-alpha-search-queries budgets 0*) +

(*1* + *staged-concrete-transcript-target-error-bound*) +

trace-fri-error + *1*)

proof –

have *residuals*:

active-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft

Rc Ac Pc Nc Sc Gc

by (rule *active-fri-residual-obligations-from-components*

[OF *trace-residuals composition-residuals empty-trace*])

show *?thesis*

by (rule

stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-residual-obligations

[OF *false-statement wf controlled residuals*])

qed

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-reductions:*

assumes *false-statement:* $\neg \text{exists-valid-trace}$

and *wf:* *staged-budget-wellformed budgets*

and *controlled:* *staged-adversary-controlled budgets A*

and *active-fri:* *active-fri-reductions-for A*

shows *checked-staged-adversary-acceptance-probability A ≤*

hash-collision-budget-value 0

(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +

(composition-error-bound +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

hash-collision-budget-value 0

(staged-alpha-search-queries budgets 0) +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

(1 + staged-concrete-transcript-target-error-bound)) +

($\sum_{i < \text{rounds.}}$

1 +

(1 + staged-concrete-transcript-target-error-bound) +

(1 + staged-concrete-transcript-target-error-bound)) +

trace-fri-error + composition-fri-error +

(composition-error-bound +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

hash-collision-budget-value 0

(staged-alpha-search-queries budgets 0) +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

(1 + staged-concrete-transcript-target-error-bound) +

trace-fri-error + 1)

proof –

have *trace-fri-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

trace-fri-bad-with-header-tied-partial-candidate)

adversary-initial-state ≤ trace-fri-error

by (rule *active-fri-reductions-staged-trace-bound*[OF *active-fri*])

have *comp-fri-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

composition-fri-bad-with-verifier-tied-partial-candidate)

adversary-initial-state ≤ composition-fri-error

by (rule *active-fri-reductions-staged-composition-bound*[OF *active-fri*])

have *empty-trace-fri-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

trace-fri-bad-with-empty-composition-header-candidates)

$\text{adversary-initial-state} \leq \text{trace-fri-error}$
by (rule *active-fri-reductions-staged-empty-header-bound*[*OF active-fri*])
show ?thesis
by (rule
 $\text{current-query-route-bound-from-active-fri-bounds-internal}$
 $[\text{OF false-statement wf controlled trace-fri-bound comp-fri-bound}$
 $\text{empty-trace-fri-bound}]$)
qed

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-derived-active-fri-compo*
fixes *TraceAug TraceMissing TraceMerkle CompSampled CompMerkle EmptySampled Emp-*
tyMissing ::

$'f \text{ staged-proof-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{prob}$

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets A*

and *trace-augmented-bound*:

$\wedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$

$\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \Rightarrow$

$\text{wp-event verify-monad}$

$(\text{trace-fri-bad-with-header-tied-augmented-partial-candidate}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{TraceAug data attacker-state}$

and *trace-missing-bound*:

$\wedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$

$\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \Rightarrow$

$\text{wp-event verify-monad}$

$(\text{trace-fri-header-tied-recorded-sibling-candidate-missing}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$

$\text{TraceMissing data attacker-state}$

and *trace-merkle-bound*:

$\wedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$

$\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \Rightarrow$

$\text{wp-event verify-monad}$

$(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$

$(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data})) \leq \text{TraceMerkle data attacker-state}$
and *trace-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{TraceAug data attacker-state} +$
 $(\text{TraceMissing data attacker-state} + \text{TraceMerkle data attacker-state})$
 $\leq \text{trace-fri-error}$
and *composition-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{CompSampled data attacker-state}$
and *composition-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{CompMerkle data attacker-state}$
and *composition-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{CompSampled data attacker-state} +$
 $((\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0 + 0) +$
 $\text{CompMerkle data attacker-state}) + 0)$
 $\leq \text{composition-fri-error}$
and *empty-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$

$$\begin{aligned}
& \text{wp-event verify-monad} \\
& \quad (\text{trace-fri-bad-with-sampled-layer-chain} \\
& \quad \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad \quad (\text{staged-proof-transcript data}))) \\
& \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad (\text{staged-proof-transcript data})) \leq \\
& \quad \text{EmptySampled data attacker-state} \\
& \text{and empty-missing-bound:} \\
& \wedge \text{data attacker-state.} \\
& \quad \text{Some (data, attacker-state)} \in \\
& \quad \quad \text{set-dist (execute (checked-staged-transcript-program A)} \\
& \quad \quad \quad \text{adversary-initial-state)} \implies \\
& \quad \text{wp-event verify-monad} \\
& \quad \quad (\text{trace-fri-reachable-missing-sampled-layer-chain} \\
& \quad \quad \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad \quad \quad (\text{staged-proof-transcript data}))) \\
& \quad \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad \quad (\text{staged-proof-transcript data})) \leq \\
& \quad \quad \text{EmptyMissing data attacker-state} \\
& \text{and empty-total:} \\
& \wedge \text{data attacker-state.} \\
& \quad \text{Some (data, attacker-state)} \in \\
& \quad \quad \text{set-dist (execute (checked-staged-transcript-program A)} \\
& \quad \quad \quad \text{adversary-initial-state)} \implies \\
& \quad \quad \text{EmptySampled data attacker-state} + \text{EmptyMissing data attacker-state} \\
& \quad \leq \text{trace-fri-error} \\
& \text{shows checked-staged-adversary-acceptance-probability A} \leq \\
& \quad \text{hash-collision-budget-value 0} \\
& \quad \quad (\text{staged-attacker-query-budget budgets} + \\
& \quad \quad \quad \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) + \\
& \quad (\text{composition-error-bound} + \\
& \quad \quad \text{staged-phase-relation-error size} \\
& \quad \quad \quad (\text{staged-alpha-search-queries budgets 0}) + \\
& \quad \text{hash-collision-budget-value 0} \\
& \quad \quad (\text{staged-alpha-search-queries budgets 0}) + \\
& \quad \quad \text{staged-phase-relation-error size} \\
& \quad \quad \quad (\text{staged-alpha-search-queries budgets 0}) + \\
& \quad (1 + \text{staged-concrete-transcript-target-error-bound})) + \\
& \quad (\sum_{i < \text{rounds.}} \\
& \quad \quad 1 + \\
& \quad \quad (1 + \text{staged-concrete-transcript-target-error-bound}) + \\
& \quad \quad (1 + \text{staged-concrete-transcript-target-error-bound})) + \\
& \quad \text{trace-fri-error} + \text{composition-fri-error} + \\
& \quad (\text{composition-error-bound} + \\
& \quad \quad \text{staged-phase-relation-error size} \\
& \quad \quad \quad (\text{staged-alpha-search-queries budgets 0}) + \\
& \quad \text{hash-collision-budget-value 0} \\
& \quad \quad (\text{staged-alpha-search-queries budgets 0}) + \\
& \quad \quad \text{staged-phase-relation-error size}
\end{aligned}$$

$(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $\text{trace-fri-error} + 1)$

proof –

have *active-fri: active-fri-reductions-for A*

by (rule
active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty-sampled
 $[OF \text{ trace-augmented-bound trace-missing-bound trace-merkle-bound}$
 $\text{trace-total composition-sampled-bound composition-merkle-bound}$
 $\text{composition-total empty-sampled-bound empty-missing-bound}$
 $\text{empty-total}]$)

show *?thesis*

by (rule
stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-reductions
 $[OF \text{ false-statement wf controlled active-fri}]$)

qed

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-derived-active-fri-conflict*

fixes *TraceAug TraceMissing TraceMerkle CompSampled CompMerkle EmptySampled Empty-*
Conflict EmptyZero ::

'f staged-proof-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ prob

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *trace-augmented-bound:*
 $\wedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-bad-with-header-tied-augmented-partial-candidate
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$ TraceAug data attacker-state

and *trace-missing-bound:*
 $\wedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-recorded-sibling-candidate-missing
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
TraceMissing data attacker-state

and *trace-merkle-bound:*
 $\wedge \text{data attacker-state.}$

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ TraceMerkle data attacker-state
and *trace-total:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
TraceAug data attacker-state +
(TraceMissing data attacker-state + TraceMerkle data attacker-state)
 $\leq$ *trace-fri-error*
and *composition-sampled-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(composition-fri-bad-with-verifier-tied-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
CompSampled data attacker-state
and *composition-merkle-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ CompMerkle data attacker-state
and *composition-total:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
CompSampled data attacker-state +
((CompMerkle data attacker-state + 0) +
(CompMerkle data attacker-state + 0) +
(CompMerkle data attacker-state + 0) +

$(\text{CompMerkle data attacker-state} + 0 + 0) +$
 $\text{CompMerkle data attacker-state}) + 0)$
 $\leq \text{composition-fri-error}$
and *empty-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptySampled data attacker-state}$
and *empty-conflict-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyConflict data attacker-state}$
and *empty-zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyZero data attacker-state}$
and *empty-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{EmptySampled data attacker-state} +$
 $(\text{EmptyConflict data attacker-state} + \text{EmptyZero data attacker-state})$
 $\leq \text{trace-fri-error}$
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0

(staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 ($\sum_{i < \text{rounds.}}$
 1 +
 (1 + staged-concrete-transcript-target-error-bound) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 trace-fri-error + composition-fri-error +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound) +
 trace-fri-error + 1)

proof –

have active-fri: active-fri-reductions-for A

by (rule
 active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty-conflict-zero
 [OF trace-augmented-bound trace-missing-bound trace-merkle-bound
 trace-total composition-sampled-bound composition-merkle-bound
 composition-total empty-sampled-bound empty-conflict-bound
 empty-zero-bound empty-total])

show ?thesis

by (rule
 stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-reductions
 [OF false-statement wf controlled active-fri])

qed

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-derived-active-fri-canonical

fixes TraceAug TraceMissing TraceMerkle CompCanonical CompMerkle EmptyCanonical EmptyConflict EmptyZero ::

'f staged-proof-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ prob

assumes false-statement: $\neg$ exists-valid-trace

and wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and trace-augmented-bound:

$\bigwedge$ data attacker-state.

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A))

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-augmented-partial-candidate}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{TraceAug data attacker-state}$
and $\text{trace-missing-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-recorded-sibling-candidate-missing}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{TraceMissing data attacker-state}$
and $\text{trace-merkle-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{TraceMerkle data attacker-state}$
and trace-total:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})} \implies$
 $\text{TraceAug data attacker-state} +$
 $(\text{TraceMissing data attacker-state} + \text{TraceMerkle data attacker-state})$
 $\leq \text{trace-fri-error}$
and $\text{composition-canonical-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-canonical-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$

CompCanonical data attacker-state
and *composition-merkle-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$ CompMerkle data attacker-state
and *composition-total:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
CompCanonical data attacker-state +
((CompMerkle data attacker-state + 0) +
(CompMerkle data attacker-state + 0) +
(CompMerkle data attacker-state + 0) +
(CompMerkle data attacker-state + 0 + 0) +
CompMerkle data attacker-state) + 0)
 $\leq$ *composition-fri-error*
and *empty-canonical-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-bad-with-canonical-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
EmptyCanonical data attacker-state
and *empty-conflict-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-sampled-assignment-conflict
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
EmptyConflict data attacker-state
and *empty-zero-bound:*

$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-zero-round-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 EmptyZero data attacker-state
and empty-total:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 EmptyCanonical data attacker-state +
 (EmptyConflict data attacker-state + EmptyZero data attacker-state)
 $\leq$ trace-fri-error
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 ($\sum$ i < rounds.
 1 +
 (1 + staged-concrete-transcript-target-error-bound) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 trace-fri-error + composition-fri-error +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound) +
 trace-fri-error + 1)
proof –
have active-fri: active-fri-reductions-for A
by (rule
 active-fri-reductions-for-from-trace-augmented-composition-canonical-and-empty-conflict-zero

```

[OF trace-augmented-bound trace-missing-bound trace-merkle-bound
  trace-total composition-canonical-bound composition-merkle-bound
  composition-total empty-canonical-bound empty-conflict-bound
  empty-zero-bound empty-total])
show ?thesis
by (rule
  stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-reductions
  [OF false-statement wf controlled active-fri])
qed

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-sampled-query-fri-compo
fixes TraceAug TraceConflict TraceZero TraceMerkle CompSampled CompMerkle EmptySam-
pled EmptyConflict EmptyZero ::
  'f staged-proof-data  $\Rightarrow$  'f protocol-channel  $\Rightarrow$  prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-augmented-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\Rightarrow$ 
    wp-event verify-monad
      (trace-fri-bad-with-header-tied-augmented-partial-candidate
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq$  TraceAug data attacker-state
and trace-conflict-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\Rightarrow$ 
    wp-event verify-monad
      (trace-fri-header-tied-sampled-assignment-conflict
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq$  TraceConflict data attacker-state
and trace-zero-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\Rightarrow$ 
    wp-event verify-monad
      (trace-fri-header-tied-zero-round-final-obstruction
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state

```

$(\text{staged-proof-transcript data})) \leq \text{TraceZero data attacker-state}$
and *trace-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{TraceMerkle data attacker-state}$
and *trace-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{TraceAug data attacker-state} +$
 $((\text{TraceConflict data attacker-state} + \text{TraceZero data attacker-state}) +$
 $\text{TraceMerkle data attacker-state})$
 $\leq \text{trace-fri-error}$
and *composition-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-query-bad-candidate}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{CompSampled data attacker-state}$
and *composition-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{CompMerkle data attacker-state}$
and *composition-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$

$\text{CompSampled data attacker-state} +$
 $((\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0 + 0) +$
 $\text{CompMerkle data attacker-state}) + 0)$
 $\leq \text{composition-fri-error}$
and empty-sampled-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-query-bad-candidate}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptySampled data attacker-state}$
and empty-conflict-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyConflict data attacker-state}$
and empty-zero-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyZero data attacker-state}$
and empty-total:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{EmptySampled data attacker-state} +$

$(\text{EmptyConflict data attacker-state} + \text{EmptyZero data attacker-state})$
 $\leq \text{trace-fri-error}$
shows *checked-staged-adversary-acceptance-probability* $A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\sum_{i < \text{rounds.}} 1 +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $\text{trace-fri-error} + 1)$
proof –
have *active-fri: active-fri-reductions-for* A
by (rule
 $\text{active-fri-reductions-for-from-trace-augmented-conflict-zero-composition-sampled-query-and-empty-conflict-zero}$
 $[\text{OF trace-augmented-bound trace-conflict-bound trace-zero-bound}$
 $\text{trace-merkle-bound trace-total composition-sampled-bound composition-merkle-bound}$
 $\text{composition-total empty-sampled-bound empty-conflict-bound}$
 $\text{empty-zero-bound empty-total}]$)
show *?thesis*
by (rule
 $\text{stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-reductions}$
 $[\text{OF false-statement wf controlled active-fri}]$)
qed

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-sampled-query-projection*
fixes *TraceAug TraceConflict TraceZero TraceMerkle CompSampled CompMerkle EmptySam-*
pled EmptyConflict EmptyZero ::
 $'f \text{ staged-proof-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{prob}$
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets* A

and *trace-augmented-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-bad-with-header-tied-augmented-partial-candidate*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *TraceAug data attacker-state*
and *trace-conflict-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-header-tied-sampled-assignment-conflict*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *TraceConflict data attacker-state*
and *trace-zero-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-header-tied-zero-round-final-obstruction*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *TraceZero data attacker-state*
and *trace-merkle-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*partial-merkle-inconsistency-bad*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *TraceMerkle data attacker-state*
and *trace-total*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

$\text{TraceAug data attacker-state} +$
 $((\text{TraceConflict data attacker-state} + \text{TraceZero data attacker-state}) +$
 $\text{TraceMerkle data attacker-state})$
 $\leq \text{trace-fri-error}$
and *composition-sampled-future*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{composition-fri-future-fresh}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *composition-sampled-generic*:
 $\wedge \text{data attacker-state dg.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{generic-fri-sampled-query-projection-fraction-bound}$
 $(\text{composition-table-low-degree (to-nat dg)})$
 $(\text{Not } \circ \text{ composition-table-low-degree maxDegree) (to-nat dg)}$
 $(\text{CompSampled data attacker-state})$
and *composition-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{CompMerkle data attacker-state}$
and *composition-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{CompSampled data attacker-state} +$
 $((((\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0 + 0) +$
 $\text{CompMerkle data attacker-state}) + 0)$
 $\leq \text{composition-fri-error}$
and *empty-sampled-future*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$

trace-fri-future-fresh
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
and *empty-sampled-generic*:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 generic-fri-sampled-query-projection-fraction-bound
 trace-table-low-degree (Not $\circ$ trace-table-low-degree)
 (clength - 1) (EmptySampled data attacker-state)
and *empty-conflict-bound*:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-sampled-assignment-conflict
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 EmptyConflict data attacker-state
and *empty-zero-bound*:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-zero-round-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 EmptyZero data attacker-state
and *empty-total*:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 EmptySampled data attacker-state +
 (EmptyConflict data attacker-state + EmptyZero data attacker-state)
 $\leq$ trace-fri-error
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size

```

    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    (1 + staged-concrete-transcript-target-error-bound)) +
    (Σ i<rounds.
      1 +
      (1 + staged-concrete-transcript-target-error-bound) +
      (1 + staged-concrete-transcript-target-error-bound)) +
    trace-fri-error + composition-fri-error +
    (composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      (1 + staged-concrete-transcript-target-error-bound) +
      trace-fri-error + 1)
proof (rule
  stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-sampled-query-fri-components
  [OF false-statement wf controlled trace-augmented-bound
    trace-conflict-bound trace-zero-bound trace-merkle-bound trace-total -
    composition-merkle-bound composition-total - empty-conflict-bound
    empty-zero-bound empty-total])
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
show
  wp-event verify-monad (composition-fri-sampled-query-bad-candidate ?s)
  ?s ≤ CompSampled data attacker-state
by (rule
  wp-composition-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction
  [OF composition-sampled-future[OF support]])
  (rule composition-sampled-generic[OF support])
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state

```

```

    (staged-proof-transcript data)
  show
    wp-event verify-monad (trace-fri-sampled-query-bad-candidate ?s)
      ?s ≤ EmptySampled data attacker-state
  by (rule
    wp-trace-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction
    [OF empty-sampled-future[OF support]])
    (rule empty-sampled-generic[OF support])
qed

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-trace-active-and-staged-s
fixes composition-B :: 'f ⇒ 'f list set
and composition-Q empty-Q :: nat list set
and CompSampled CompMerkle EmptySampled EmptyConflict EmptyZero
    composition-fresh empty-fresh :: prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-header-tied-partial-candidate)
      adversary-initial-state ≤ trace-fri-error
and composition-finite:  $\bigwedge dg. \text{finite } (\text{composition-B } dg)$ 
and composition-fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \text{ composition-B}$ ))
      adversary-initial-state ≤ composition-fresh
and composition-projection:
     $\bigwedge dg. \text{fst } '
      ((\text{composition-fri-sampled-query-bad-pair-union } dg \cap
        (\text{fri-query-index-list-space} \times \text{UNIV})) \cap
        (\text{UNIV} \times (\neg \text{composition-B } dg))) \subseteq \text{composition-Q}$ 
and composition-sampled-total:
    composition-fresh +
    hash-relation-budget-value
    ( $\sum dg \in (\text{UNIV} :: 'f \text{ set}).$ 
      card (composition-B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget) +
    staged-phase-target-error
    (query-index-raw-preimage
      (query-index-list-entries composition-Q))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
    ≤ CompSampled
and composition-merkle-bound:
     $\bigwedge \text{data attacker-state.}$ 

```

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*partial-merkle-inconsistency-bad*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *CompMerkle*
and *composition-total*:
CompSampled +
(((*CompMerkle* + 0) + (*CompMerkle* + 0) + (*CompMerkle* + 0) +
(*CompMerkle* + 0 + 0) + *CompMerkle*) + 0 + 0)
 $\leq$ *composition-fri-error*
and *empty-finite*: *finite empty-B*
and *empty-fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s.$ *trace-fri-challenge-list-fresh-hit s empty-B*))
adversary-initial-state $\leq$ *empty-fresh*
and *empty-projection*:
fst ‘
((*trace-fri-sampled-query-bad-pair-union* $\cap$
(*fri-query-index-list-space* $\times$ *UNIV*)) $\cap$
(*UNIV* $\times$ ($-$ *empty-B*))) $\subseteq$ *empty-Q*
and *empty-sampled-total*:
empty-fresh +
hash-relation-budget-value (*card empty-B* * *ceil-log clength*)
(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget) +
staged-phase-target-error
(*query-index-raw-preimage* (*query-index-list-entries empty-Q*))
(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget)
 $\leq$ *EmptySampled*
and *empty-conflict-bound*:
 $\bigwedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-assignment-conflict*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *EmptyConflict*
and *empty-zero-bound*:
 $\bigwedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{EmptyZero}$
and empty-total:
 $\text{EmptySampled} + (\text{EmptyConflict} + \text{EmptyZero}) \leq \text{trace-fri-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\sum_{i < \text{rounds.}} 1 +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $\text{trace-fri-error} + 1)$
proof –
have $\text{composition-sampled:}$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-sampled-query-bad-candidate})$
 $\text{adversary-initial-state} \leq \text{CompSampled}$
by $(\text{rule order-trans}$
 $[\text{OF checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-}$
 $[\text{OF wf controlled composition-finite composition-fresh-bound}$
 $\text{composition-projection}]$
 $\text{composition-sampled-total}])$
have comp-fri-bound:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state $\leq$ *composition-fri-error*

proof –

have *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state $\leq$
CompSampled +
 $((CompMerkle + 0) + (CompMerkle + 0) + (CompMerkle + 0) +$
 $(CompMerkle + 0 + 0) + CompMerkle) + 0 + 0$

apply (*rule*
checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-sampled-query-and-residual-g
[OF composition-sampled])

apply (*rule wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero*)

apply (*rule composition-merkle-bound*)

apply *assumption*

apply (*rule wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero*)

apply (*rule wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-merkle-and-slot*)

apply (*rule composition-merkle-bound*)

apply *assumption*

apply (*rule wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero*)

apply (*rule wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-merkle-and-slot*)

apply (*rule composition-merkle-bound*)

apply *assumption*

apply (*rule wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero*)

apply (*rule wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step*)

apply (*rule composition-merkle-bound*)

apply *assumption*

apply (*rule wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero*)

apply (*rule wp-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero*)

apply (*subst wp-composition-fri-verifier-tied-zero-round-final-obstruction-zero*)

apply *simp*

done

then show *?thesis*
by (*rule order-trans[OF - composition-total]*)

qed

have *empty-sampled:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-sampled-query-bad-candidate)
adversary-initial-state $\leq$ *EmptySampled*

by (*rule order-trans*
 $[OF \text{ checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-project}$
 $[OF \text{ wf controlled empty-finite empty-fresh-bound empty-projection}]$
empty-sampled-total])

have *empty-trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event

trace-fri-bad-with-empty-composition-header-candidates
adversary-initial-state $\leq$ *trace-fri-error*
proof –
have *wp-event* (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *EmptySampled* + (*EmptyConflict* + *EmptyZero*)
by (*rule*
checked-staged-security-trace-fri-empty-header-bound-from-sampled-query-conflict-and-zero
[*OF empty-sampled empty-conflict-bound empty-zero-bound*])
then show *?thesis*
by (*rule order-trans*[*OF - empty-total*])
qed
show *?thesis*
by (*rule*
current-query-route-bound-from-active-fri-bounds-internal
[*OF false-statement wf controlled trace-fri-bound comp-fri-bound*
empty-trace-fri-bound])
qed
theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-full-cover-domain-conflict*
fixes *trace-bad* ::
'f list $\Rightarrow$ *nat* $\Rightarrow$ *'f list* $\Rightarrow$ *'f set*
and *composition-bad* ::
'f $\Rightarrow$ *'f list* $\Rightarrow$ *nat* $\Rightarrow$ *'f list* $\Rightarrow$ *'f set*
and *trace-B* :: *'f list set*
and *composition-B* :: *'f* $\Rightarrow$ *'f list set*
and *trace-fresh trace-proximity trace-index trace-domain-length*
trace-sampled-domain trace-conflict trace-zero
composition-fresh composition-proximity composition-index
composition-domain-length composition-sampled-domain
composition-conflict composition-zero
empty-conflict empty-zero :: *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-finite*: *finite trace-B*
and *trace-envelope*:
 $\bigwedge$ *data attacker-state trace-table*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-multi-round-bad-sets trace-bad trace-table $\subseteq$ trace-B
and *trace-fresh-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
*(λs . *trace-fri-challenge-list-fresh-hit s trace-B*))*
adversary-initial-state $\leq$ *trace-fresh*
and *trace-proximity-bound*:

$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-sampled-full-cover-proximity-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) trace-bad)
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ trace-proximity
and trace-index-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-sampled-full-cover-index-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ trace-index
and trace-domain-length-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-sampled-full-cover-domain-length-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ trace-domain-length
and trace-sampled-domain-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-sampled-full-cover-sampled-domain-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ trace-sampled-domain
and trace-conflict-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad

$(\text{trace-fri-header-tied-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{trace-conflict}$
and trace-zero-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{trace-zero}$
and trace-total:
 $((\text{trace-fresh} +$
 $\text{hash-relation-budget-value (card trace-B * ceil-log clength)}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(\text{trace-proximity} + \text{trace-index} +$
 $(\text{trace-index} + \text{trace-domain-length}) +$
 $(\text{trace-index} + \text{trace-sampled-domain}))) +$
 $(\text{trace-conflict} + \text{trace-zero})$
 $\leq \text{trace-fri-error}$
and $\text{composition-finite: } \wedge \text{dg. finite (composition-B dg)}$
and $\text{composition-envelope:}$
 $\wedge \text{data attacker-state dg composition-table.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{composition-fri-multiround-bad-sets composition-bad dg composition-table}$
 $\subseteq \text{composition-B dg}$
and $\text{composition-fresh-bound:}$
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-challenge-list-fresh-hit s composition-B}))$
 $\text{adversary-initial-state} \leq \text{composition-fresh}$
and $\text{composition-proximity-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ composition-bad})$
 $(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data})) \leq \text{composition-proximity}$
and *composition-index-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-index}$
and *composition-domain-length-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-domain-length}$
and *composition-sampled-domain-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-sampled-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-sampled-domain}$
and *composition-conflict-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-conflict}$
and *composition-zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-zero}$
and $\text{composition-total:}$
 $((\text{composition-fresh} +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card} (\text{composition-B } dg) * \text{ceil-log} (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(\text{composition-proximity} + \text{composition-index} +$
 $(\text{composition-index} + \text{composition-domain-length}) +$
 $(\text{composition-index} + \text{composition-sampled-domain}))) +$
 $(\text{composition-conflict} + \text{composition-zero})$
 $\leq \text{composition-fri-error}$
and $\text{empty-conflict-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some} (data, \text{attacker-state}) \in$
 $\text{set-dist} (\text{execute} (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{empty-conflict}$
and empty-zero-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some} (data, \text{attacker-state}) \in$
 $\text{set-dist} (\text{execute} (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{empty-zero}$
and empty-total:
 $((\text{trace-fresh} +$
 $\text{hash-relation-budget-value} (\text{card trace-B} * \text{ceil-log clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(\text{trace-proximity} + \text{trace-index} +$
 $(\text{trace-index} + \text{trace-domain-length}) +$
 $(\text{trace-index} + \text{trace-sampled-domain}))) +$

$(\text{empty-conflict} + \text{empty-zero})$
 $\leq \text{trace-fri-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\sum_{i < \text{rounds.}} 1 +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $\text{trace-fri-error} + 1)$
proof –
have trace-fri-bound:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
proof –
have $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq$
 $((\text{trace-fresh} +$
 $\text{hash-relation-budget-value } (\text{card trace-B} * \text{ceil-log clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(\text{trace-proximity} + \text{trace-index} +$
 $(\text{trace-index} + \text{trace-domain-length}) +$
 $(\text{trace-index} + \text{trace-sampled-domain}))) +$
 $(\text{trace-conflict} + \text{trace-zero})$
by (rule
 $\text{checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-full-cover-domain-conflict-and-zero}$

```

    [where bad = trace-bad];
    (rule wf | rule controlled | rule trace-finite |
     rule trace-envelope | rule trace-fresh-bound |
     rule trace-proximity-bound | rule trace-index-bound |
     rule trace-domain-length-bound | rule trace-sampled-domain-bound |
     rule trace-conflict-bound | rule trace-zero-bound))
  then show ?thesis
    by (rule order-trans[OF - trace-total])
qed
have comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-fri-bad-with-verifier-tied-partial-candidate)
  adversary-initial-state ≤ composition-fri-error
proof -
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-fri-bad-with-verifier-tied-partial-candidate)
  adversary-initial-state ≤
    ((composition-fresh +
     hash-relation-budget-value
     (∑ dg ∈ (UNIV :: 'f set).
      card (composition-B dg) * ceil-log (maxDegree + 1))
     (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)) +
     (composition-proximity + composition-index +
      (composition-index + composition-domain-length) +
      (composition-index + composition-sampled-domain))) +
     (composition-conflict + composition-zero))
  by (rule
    checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-full-cover-domain-conflict-and
    [where bad = composition-bad];
    (rule wf | rule controlled | rule composition-finite |
     rule composition-envelope | rule composition-fresh-bound |
     rule composition-proximity-bound | rule composition-index-bound |
     rule composition-domain-length-bound |
     rule composition-sampled-domain-bound |
     rule composition-conflict-bound | rule composition-zero-bound))
  then show ?thesis
    by (rule order-trans[OF - composition-total])
qed
have empty-trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state ≤ trace-fri-error
proof -
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event

```

```

    trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state ≤
  ((trace-fresh +
    hash-relation-budget-value (card trace-B * ceil-log clength)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)) +
    (trace-proximity + trace-index +
      (trace-index + trace-domain-length) +
      (trace-index + trace-sampled-domain))) +
    (empty-conflict + empty-zero)
  by (rule
    checked-staged-security-trace-fri-empty-header-bound-from-full-cover-domain-conflict-and-zero
    [where bad = trace-bad];
    (rule wf | rule controlled | rule trace-finite |
      rule trace-envelope | rule trace-fresh-bound |
      rule trace-proximity-bound | rule trace-index-bound |
      rule trace-domain-length-bound | rule trace-sampled-domain-bound |
      rule empty-conflict-bound | rule empty-zero-bound))
  then show ?thesis
    by (rule order-trans[OF - empty-total])
qed
show ?thesis
  by (rule
    current-query-route-bound-from-active-fri-bounds-internal
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      empty-trace-fri-bound])
qed

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-full-cover-domain-conflict
fixes trace-bad ::
  'f list ⇒ nat ⇒ 'f list ⇒ 'f set
and composition-bad ::
  'f ⇒ 'f list ⇒ nat ⇒ 'f list ⇒ 'f set
and trace-fresh trace-proximity trace-index trace-domain-length
  trace-sampled-domain trace-conflict trace-zero
  composition-fresh composition-proximity composition-index
  composition-domain-length composition-sampled-domain
  composition-conflict composition-zero
  empty-conflict empty-zero :: prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs.
        trace-fri-challenge-list-fresh-hit s
        (fri-challenge-space (ceil-log clength))))
  adversary-initial-state ≤ trace-fresh

```

and *trace-proximity-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-full-cover-proximity-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) *trace-bad*)
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *trace-proximity*

and *trace-index-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-full-cover-index-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *trace-index*

and *trace-domain-length-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-full-cover-domain-length-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *trace-domain-length*

and *trace-sampled-domain-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-full-cover-sampled-domain-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *trace-sampled-domain*

and *trace-conflict-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

$\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{trace-conflict}$
and trace-zero-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{trace-zero}$
and trace-total:
 $((\text{trace-fresh} +$
 $\text{hash-relation-budget-value}$
 $(\text{card (fri-challenge-space (ceil-log clength))} * \text{ceil-log clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(\text{trace-proximity} + \text{trace-index} +$
 $(\text{trace-index} + \text{trace-domain-length}) +$
 $(\text{trace-index} + \text{trace-sampled-domain}))) +$
 $(\text{trace-conflict} + \text{trace-zero})$
 $\leq \text{trace-fri-error}$
and $\text{composition-fresh-bound:}$
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s.$
 $\text{composition-fri-challenge-list-fresh-hit s}$
 $(\lambda dg. \text{fri-challenge-space (ceil-log (to-nat dg} + 1))))))$
 $\text{adversary-initial-state} \leq \text{composition-fresh}$
and $\text{composition-proximity-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ composition-bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-proximity}$
and $\text{composition-index-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$

$set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$
 $wp-event \ verify-monad$
 $(composition-fri-sampled-full-cover-index-failure$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)))$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)) \leq composition-index$
and $composition-domain-length-bound:$
 $\wedge data \ attacker-state.$
 $Some \ (data, \ attacker-state) \in$
 $set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$
 $wp-event \ verify-monad$
 $(composition-fri-sampled-full-cover-domain-length-failure$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)))$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)) \leq composition-domain-length$
and $composition-sampled-domain-bound:$
 $\wedge data \ attacker-state.$
 $Some \ (data, \ attacker-state) \in$
 $set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$
 $wp-event \ verify-monad$
 $(composition-fri-sampled-full-cover-sampled-domain-failure$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)))$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)) \leq composition-sampled-domain$
and $composition-conflict-bound:$
 $\wedge data \ attacker-state.$
 $Some \ (data, \ attacker-state) \in$
 $set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$
 $wp-event \ verify-monad$
 $(composition-fri-verifier-tied-sampled-assignment-conflict$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)))$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)) \leq composition-conflict$
and $composition-zero-bound:$
 $\wedge data \ attacker-state.$
 $Some \ (data, \ attacker-state) \in$
 $set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$
 $wp-event \ verify-monad$
 $(composition-fri-verifier-tied-zero-round-final-obstruction$
 $(verifier-state-from-adversary \ attacker-state$

$(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-zero}$
and composition-total :
 $((\text{composition-fresh} +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card} (\text{fri-challenge-space} (\text{ceil-log} (\text{to-nat } dg + 1))) *$
 $\text{ceil-log} (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(\text{composition-proximity} + \text{composition-index} +$
 $(\text{composition-index} + \text{composition-domain-length}) +$
 $(\text{composition-index} + \text{composition-sampled-domain}))) +$
 $(\text{composition-conflict} + \text{composition-zero})$
 $\leq \text{composition-fri-error}$
and $\text{empty-conflict-bound}$:
 $\wedge \text{data attacker-state.}$
 $\text{Some} (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist} (\text{execute} (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{empty-conflict}$
and empty-zero-bound :
 $\wedge \text{data attacker-state.}$
 $\text{Some} (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist} (\text{execute} (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{empty-zero}$
and empty-total :
 $((\text{trace-fresh} +$
 $\text{hash-relation-budget-value}$
 $(\text{card} (\text{fri-challenge-space} (\text{ceil-log } \text{clength})) * \text{ceil-log } \text{clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(\text{trace-proximity} + \text{trace-index} +$
 $(\text{trace-index} + \text{trace-domain-length}) +$
 $(\text{trace-index} + \text{trace-sampled-domain}))) +$
 $(\text{empty-conflict} + \text{empty-zero})$
 $\leq \text{trace-fri-error}$

shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
 (1 + *staged-concrete-transcript-target-error-bound*)) +
 ($\sum_{i < \text{rounds.}}$
 1 +
 (1 + *staged-concrete-transcript-target-error-bound*) +
 (1 + *staged-concrete-transcript-target-error-bound*)) +
trace-fri-error + *composition-fri-error* +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
 (1 + *staged-concrete-transcript-target-error-bound*) +
trace-fri-error + 1)

proof (rule

stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-full-cover-domain-conflict-zero-c

[**where** *trace-B* = *fri-challenge-space* (*ceil-log* *clength*)

and *composition-B* =

$\lambda dg. \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$

and *trace-bad* = *trace-bad*

and *composition-bad* = *composition-bad*])

show $\neg \text{exists-valid-trace}$

by (rule *false-statement*)

show *staged-budget-wellformed* *budgets*

by (rule *wf*)

show *staged-adversary-controlled* *budgets* A

by (rule *controlled*)

show *finite* (*fri-challenge-space* (*ceil-log* *clength*))

by *simp*

show $\bigwedge \text{data attacker-state trace-table.}$

Some (*data*, *attacker-state*) $\in$

set-dist (*execute* (*checked-staged-transcript-program* A)

adversary-initial-state) $\implies$

trace-fri-multi-round-bad-sets *trace-bad* *trace-table*

$\subseteq$ *fri-challenge-space* (*ceil-log* *clength*)

by (rule *trace-fri-multi-round-bad-sets-subset*)

show *wp-event* (*checked-staged-security-experiment-with-data-state* A)

(staged-security-with-data-state-verifier-event
 (λs.
 trace-fri-challenge-list-fresh-hit s
 (fri-challenge-space (ceil-log clength))))
 adversary-initial-state ≤ trace-fresh
by (rule trace-fresh-bound)
show ∧ data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 wp-event verify-monad
 (trace-fri-sampled-full-cover-proximity-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) trace-bad)
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤ trace-proximity
by (rule trace-proximity-bound)
show ∧ data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 wp-event verify-monad
 (trace-fri-sampled-full-cover-index-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤ trace-index
by (rule trace-index-bound)
show ∧ data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 wp-event verify-monad
 (trace-fri-sampled-full-cover-domain-length-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤ trace-domain-length
by (rule trace-domain-length-bound)
show ∧ data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 wp-event verify-monad
 (trace-fri-sampled-full-cover-sampled-domain-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤ trace-sampled-domain

by (rule trace-sampled-domain-bound)
show $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-sampled-assignment-conflict
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$ trace-conflict
by (rule trace-conflict-bound)
show $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-zero-round-final-obstruction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$ trace-zero
by (rule trace-zero-bound)
show ((trace-fresh +
hash-relation-budget-value
(card (fri-challenge-space (ceil-log clength)) * ceil-log clength)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)) +
(trace-proximity + trace-index +
(trace-index + trace-domain-length) +
(trace-index + trace-sampled-domain))) +
(trace-conflict + trace-zero)
 $\leq$ trace-fri-error
by (rule trace-total)
show $\bigwedge \text{dg. finite (fri-challenge-space (ceil-log (to-nat dg + 1)))}$
by simp
show $\bigwedge \text{data attacker-state dg composition-table.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
composition-fri-multiround-bad-sets composition-bad dg
composition-table
 $\subseteq$ fri-challenge-space (ceil-log (to-nat dg + 1))
by (rule composition-fri-multiround-bad-sets-subset)
show wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
($\lambda s.$
composition-fri-challenge-list-fresh-hit s
($\lambda \text{dg. fri-challenge-space (ceil-log (to-nat dg + 1))$)))

$adversary-initial-state \leq composition-fresh$
by (rule *composition-fresh-bound*)
show $\bigwedge data \text{ attacker-state.}$
 $Some (data, attacker-state) \in$
 $set-dist (execute (checked-staged-transcript-program A)$
 $adversary-initial-state) \implies$
 $wp-event \text{ verify-monad}$
 $(composition-fri-sampled-full-cover-proximity-failure$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data)) composition-bad)$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data)) \leq composition-proximity$
by (rule *composition-proximity-bound*)
show $\bigwedge data \text{ attacker-state.}$
 $Some (data, attacker-state) \in$
 $set-dist (execute (checked-staged-transcript-program A)$
 $adversary-initial-state) \implies$
 $wp-event \text{ verify-monad}$
 $(composition-fri-sampled-full-cover-index-failure$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data)))$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data)) \leq composition-index$
by (rule *composition-index-bound*)
show $\bigwedge data \text{ attacker-state.}$
 $Some (data, attacker-state) \in$
 $set-dist (execute (checked-staged-transcript-program A)$
 $adversary-initial-state) \implies$
 $wp-event \text{ verify-monad}$
 $(composition-fri-sampled-full-cover-domain-length-failure$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data)))$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data)) \leq composition-domain-length$
by (rule *composition-domain-length-bound*)
show $\bigwedge data \text{ attacker-state.}$
 $Some (data, attacker-state) \in$
 $set-dist (execute (checked-staged-transcript-program A)$
 $adversary-initial-state) \implies$
 $wp-event \text{ verify-monad}$
 $(composition-fri-sampled-full-cover-sampled-domain-failure$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data)))$
 $(verifier-state-from-adversary attacker-state$
 $(staged-proof-transcript data)) \leq composition-sampled-domain$
by (rule *composition-sampled-domain-bound*)
show $\bigwedge data \text{ attacker-state.}$
 $Some (data, attacker-state) \in$
 $set-dist (execute (checked-staged-transcript-program A)$

```

    adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-sampled-assignment-conflict
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  composition-conflict
by (rule composition-conflict-bound)
show  $\bigwedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-zero-round-final-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  composition-zero
by (rule composition-zero-bound)
show ((composition-fresh +
  hash-relation-budget-value
    ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
      card (fri-challenge-space (ceil-log (to-nat dg + 1))) *
      ceil-log (maxDegree + 1)))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)) +
  (composition-proximity + composition-index +
    (composition-index + composition-domain-length) +
    (composition-index + composition-sampled-domain))) +
  (composition-conflict + composition-zero)
 $\leq$  composition-fri-error
by (rule composition-total)
show  $\bigwedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-sampled-assignment-conflict
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  empty-conflict
by (rule empty-conflict-bound)
show  $\bigwedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-zero-round-final-obstruction

```


$(\text{verifier-state-from-adversary } \text{attacker-state} \text{ (staged-proof-transcript data)}))$
 $(\text{verifier-state-from-adversary } \text{attacker-state} \text{ (staged-proof-transcript data)}) \leq \text{empty-zero}$
by (rule empty-zero-bound)
show $((\text{trace-fresh} + \text{hash-relation-budget-value} \text{ (card (fri-challenge-space (ceil-log clength)) * ceil-log clength)} \text{ (staged-attacker-query-budget budgets + staged-challenge-query-budget))} + \text{trace-proximity} + \text{trace-index} + \text{trace-index} + \text{trace-domain-length} + \text{trace-index} + \text{trace-sampled-domain})) + \text{empty-conflict} + \text{empty-zero}) \leq \text{trace-fri-error}$
by (rule empty-total)
qed

definition *concrete-trace-fri-header-error-for*
where

concrete-trace-fri-header-error-for $A =$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \text{ (staged-security-with-data-state-verifier-event trace-fri-bad-with-header-tied-partial-candidate) adversary-initial-state}$

definition *concrete-composition-fri-error-for*
where

concrete-composition-fri-error-for $A =$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \text{ (staged-security-with-data-state-verifier-event composition-fri-bad-with-verifier-tied-partial-candidate) adversary-initial-state}$

definition *concrete-trace-fri-empty-error-for*
where

concrete-trace-fri-empty-error-for $A =$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \text{ (staged-security-with-data-state-verifier-event trace-fri-bad-with-empty-composition-header-candidates) adversary-initial-state}$

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route:*

assumes *false-statement:* $\neg \text{exists-valid-trace}$
and *wf:* *staged-budget-wellformed* budgets
and *controlled:* *staged-adversary-controlled* $\text{budgets } A$
shows *checked-staged-adversary-acceptance-probability* $A \leq \text{hash-collision-budget-value } 0 \text{ (staged-attacker-query-budget budgets} +$

$$\begin{aligned}
& \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) + \\
& (\text{composition-error-bound} + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{hash-collision-budget-value } 0 \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& (1 + \text{staged-concrete-transcript-target-error-bound})) + \\
& (\sum_{i < \text{rounds.}} \\
& \quad 1 + \\
& \quad (1 + \text{staged-concrete-transcript-target-error-bound}) + \\
& \quad (1 + \text{staged-concrete-transcript-target-error-bound})) + \\
& \text{concrete-trace-fri-header-error-for } A + \\
& \quad \text{concrete-composition-fri-error-for } A + \\
& (\text{composition-error-bound} + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{hash-collision-budget-value } 0 \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& (1 + \text{staged-concrete-transcript-target-error-bound}) + \\
& \quad \text{concrete-trace-fri-empty-error-for } A + 1)
\end{aligned}$$

proof –

have *trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)

(*staged-security-with-data-state-verifier-event*

trace-fri-bad-with-header-tied-partial-candidate)

adversary-initial-state $\leq$ *concrete-trace-fri-header-error-for* *A*

by (*simp add: concrete-trace-fri-header-error-for-def*)

have *comp-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)

(*staged-security-with-data-state-verifier-event*

composition-fri-bad-with-verifier-tied-partial-candidate)

adversary-initial-state $\leq$ *concrete-composition-fri-error-for* *A*

by (*simp add: concrete-composition-fri-error-for-def*)

have *empty-trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)

(*staged-security-with-data-state-verifier-event*

trace-fri-bad-with-empty-composition-header-candidates)

adversary-initial-state $\leq$ *concrete-trace-fri-empty-error-for* *A*

by (*simp add: concrete-trace-fri-empty-error-for-def*)

show *?thesis*

by (*rule*

current-query-route-bound-from-active-fri-bounds-internal

[*OF false-statement wf controlled trace-fri-bound comp-fri-bound*

empty-trace-fri-bound])

qed

theorem *stark-soundness-same-run-endpoint:*
assumes *false-statement:* $\neg$ *exists-valid-trace*
and *wf:* *staged-budget-wellformed budgets*
and *controlled:* *staged-adversary-controlled budgets A*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(1 + staged-concrete-transcript-target-error-bound)) +
*($\sum$ *i* < rounds.*
1 +
(1 + staged-concrete-transcript-target-error-bound) +
(1 + staged-concrete-transcript-target-error-bound)) +
concrete-trace-fri-header-error-for A +
concrete-composition-fri-error-for A +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(1 + staged-concrete-transcript-target-error-bound) +
concrete-trace-fri-empty-error-for A + 1)
by (*rule*
stark-soundness-from-current-public-prefix-current-query-relevant-drift-route
[OF false-statement wf controlled])
end

hide-fact

soundness.stark-soundness-same-run-endpoint
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-staged-active-fri-bound
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-bounds
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-reductions
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-derived-active-fri-comp
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-derived-active-fri-conf
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-derived-active-fri-cano
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-sampled-query-fri-com
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-sampled-query-projecti

soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-trace-active-and-staged
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-full-cover-domain-conf
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-full-cover-domain-conf
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-residual-oblig
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-residual-com

end

theory *Soundness-Public-Route-Concrete-FRI-Endpoint*
imports *Soundness-Public-Route-Same-Run*
begin

Public-route endpoint with concrete FRI losses expanded one step below the top-level bad-event probabilities.

This theory is retained as infrastructure for the current sharpened endpoint: downstream theories reuse the reachable verifier-event envelope definitions and one-step concrete FRI bounds. The active public theorem is exported by `Soundness_FRI_Trace_Restricted_Query_Endpoint.th`

context *soundness*
begin

definition *reachable-verifier-event-bound-for*
where

reachable-verifier-event-bound-for A E =
 $\text{Max } (\text{insert } 0$
 $((\lambda \text{out. case out of}$
 $\quad \text{None} \Rightarrow 0$
 $\quad | \text{Some } y \Rightarrow$
 $\quad \quad \text{let } s =$
 $\quad \quad \quad \text{verifier-state-from-adversary } (\text{snd } y)$
 $\quad \quad \quad (\text{staged-proof-transcript } (\text{fst } y))$
 $\quad \quad \text{in wp-event verify-monad } (E \ s) \ s) \text{ ‘}$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\quad \text{adversary-initial-state}))$

lemma *reachable-verifier-event-bound-for-ge:*

assumes *support:*
 $\text{Some } (\text{data, attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\quad \text{adversary-initial-state})$

shows

$\text{wp-event verify-monad}$
 $(E$
 $\quad (\text{verifier-state-from-adversary attacker-state}$
 $\quad \quad (\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $\quad (\text{staged-proof-transcript data}))$

$\leq \text{reachable-verifier-event-bound-for } A \ E$
proof –
let $?F =$
 $(\lambda \text{out. case out of}$
 $\quad \text{None} \Rightarrow 0$
 $\quad | \text{Some } y \Rightarrow$
 $\quad \quad \text{let } s =$
 $\quad \quad \quad \text{verifier-state-from-adversary (snd } y)$
 $\quad \quad \quad (\text{staged-proof-transcript (fst } y))$
 $\quad \quad \text{in wp-event verify-monad (E } s) s)$
let $?S =$
 $\quad \text{insert } 0$
 $\quad (?F \ ‘$
 $\quad \quad \text{set-dist}$
 $\quad \quad (\text{execute (checked-staged-transcript-program } A)$
 $\quad \quad \quad \text{adversary-initial-state}))$
have $\text{finite-S: finite } ?S$
by simp
have $\text{mem: } ?F (\text{Some (data, attacker-state)}) \in ?S$
using support by blast
have $\text{le: } ?F (\text{Some (data, attacker-state)}) \leq \text{Max } ?S$
by $(\text{rule Max-ge[OF finite-S mem]})$
have $F\text{-eq:}$
 $\quad ?F (\text{Some (data, attacker-state)}) =$
 $\quad \text{wp-event verify-monad}$
 $\quad (E$
 $\quad \quad (\text{verifier-state-from-adversary attacker-state}$
 $\quad \quad \quad (\text{staged-proof-transcript data})))$
 $\quad (\text{verifier-state-from-adversary attacker-state}$
 $\quad \quad (\text{staged-proof-transcript data}))$
by $(\text{simp add: Let-def})$
show $?thesis$
unfolding $\text{reachable-verifier-event-bound-for-def}$
using le F-eq by simp
qed

definition $\text{trace-fri-full-challenge-bad}$

where

$\text{trace-fri-full-challenge-bad} - - - = \text{UNIV}$

definition $\text{composition-fri-full-challenge-bad}$

where

$\text{composition-fri-full-challenge-bad} - - - = \text{UNIV}$

definition $\text{concrete-trace-fri-full-cover-domain-error-for}$

where

$\text{concrete-trace-fri-full-cover-domain-error-for budgets } A =$
 $((\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $\quad (\text{staged-security-with-data-state-verifier-event}$

$(\lambda s.$
 $\quad \text{trace-fri-challenge-list-fresh-hit } s$
 $\quad (\text{fri-challenge-space } (\text{ceil-log } \text{clength}))))$
 $\text{adversary-initial-state} +$
 $\text{hash-relation-budget-value}$
 $(\text{card } (\text{fri-challenge-space } (\text{ceil-log } \text{clength})) * \text{ceil-log } \text{clength})$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\quad \text{staged-challenge-query-budget})) +$
 $(\text{reachable-verifier-event-bound-for } A$
 $\quad (\lambda s.$
 $\quad \quad \text{trace-fri-sampled-full-cover-proximity-failure } s$
 $\quad \quad \text{trace-fri-full-challenge-bad}) +$
 $\text{reachable-verifier-event-bound-for } A$
 $\quad \text{trace-fri-sampled-full-cover-index-failure} +$
 $(\text{reachable-verifier-event-bound-for } A$
 $\quad \quad \text{trace-fri-sampled-full-cover-index-failure} +$
 $\quad \text{reachable-verifier-event-bound-for } A$
 $\quad \quad \text{trace-fri-sampled-full-cover-domain-length-failure}) +$
 $(\text{reachable-verifier-event-bound-for } A$
 $\quad \quad \text{trace-fri-sampled-full-cover-index-failure} +$
 $\quad \text{reachable-verifier-event-bound-for } A$
 $\quad \quad \text{trace-fri-sampled-full-cover-sampled-domain-failure}))))$

definition *concrete-trace-fri-header-error-for'*

where

$\text{concrete-trace-fri-header-error-for'} \text{ budgets } A =$
 $\text{concrete-trace-fri-full-cover-domain-error-for } \text{budgets } A +$
 $(\text{reachable-verifier-event-bound-for } A$
 $\quad \text{trace-fri-header-tied-sampled-assignment-conflict} +$
 $\text{reachable-verifier-event-bound-for } A$
 $\quad \text{trace-fri-header-tied-zero-round-final-obstruction})$

definition *concrete-trace-fri-empty-error-for'*

where

$\text{concrete-trace-fri-empty-error-for'} \text{ budgets } A =$
 $\text{concrete-trace-fri-full-cover-domain-error-for } \text{budgets } A +$
 $(\text{reachable-verifier-event-bound-for } A$
 $\quad \text{trace-fri-sampled-assignment-conflict} +$
 $\text{reachable-verifier-event-bound-for } A$
 $\quad \text{trace-fri-zero-round-final-obstruction})$

definition *concrete-composition-fri-full-cover-domain-error-for*

where

$\text{concrete-composition-fri-full-cover-domain-error-for } \text{budgets } A =$
 $((\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\quad (\text{staged-security-with-data-state-verifier-event}$
 $\quad (\lambda s.$
 $\quad \quad \text{composition-fri-challenge-list-fresh-hit } s$
 $\quad \quad (\lambda dg. \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))))))$

adversary-initial-state +
hash-relation-budget-value
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (fri\text{-challenge-space } (ceil\text{-log } (to\text{-nat } dg + 1))) *$
 $\text{ceil-log } (maxDegree + 1))$
 $(staged\text{-attacker-query-budget } budgets +$
 $\text{staged-challenge-query-budget})) +$
 $(reachable\text{-verifier-event-bound-for } A$
 $(\lambda s.$
 $\text{composition-fri-sampled-full-cover-proximity-failure } s$
 $\text{composition-fri-full-challenge-bad}) +$
 $reachable\text{-verifier-event-bound-for } A$
 $\text{composition-fri-sampled-full-cover-index-failure } +$
 $(reachable\text{-verifier-event-bound-for } A$
 $\text{composition-fri-sampled-full-cover-index-failure } +$
 $reachable\text{-verifier-event-bound-for } A$
 $\text{composition-fri-sampled-full-cover-domain-length-failure}) +$
 $(reachable\text{-verifier-event-bound-for } A$
 $\text{composition-fri-sampled-full-cover-index-failure } +$
 $reachable\text{-verifier-event-bound-for } A$
 $\text{composition-fri-sampled-full-cover-sampled-domain-failure})))$

definition *concrete-composition-fri-error-for'*

where

concrete-composition-fri-error-for' *budgets A* =
concrete-composition-fri-full-cover-domain-error-for *budgets A* +
 $(reachable\text{-verifier-event-bound-for } A$
 $\text{composition-fri-verifier-tied-sampled-assignment-conflict } +$
 $reachable\text{-verifier-event-bound-for } A$
 $\text{composition-fri-verifier-tied-zero-round-final-obstruction})$

lemma *concrete-trace-fri-header-error-for'-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

$wp\text{-event } (checked\text{-staged-security-experiment-with-data-state } A)$
 $(staged\text{-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $adversary\text{-initial-state} \leq \text{concrete-trace-fri-header-error-for' } budgets A$

unfolding *concrete-trace-fri-header-error-for'-def*

concrete-trace-fri-full-cover-domain-error-for-def

by (*rule*

checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-full-cover-domain-conflict-and-zero

[**where** *bad* = *trace-fri-full-challenge-bad*

and *B* = *fri-challenge-space* (*ceil-log* *clength*),

OF wf controlled)]

(*auto intro!*: *reachable-verifier-event-bound-for-ge*

dest: trace-fri-multi-round-bad-sets-subset[THEN subsetD])

lemma *concrete-trace-fri-empty-error-for'-bound:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state ≤ concrete-trace-fri-empty-error-for' budgets A
unfolding *concrete-trace-fri-empty-error-for'-def*
concrete-trace-fri-full-cover-domain-error-for-def
by (*rule*
checked-staged-security-trace-fri-empty-header-bound-from-full-cover-domain-conflict-and-zero
[where bad = trace-fri-full-challenge-bad
and B = fri-challenge-space (ceil-log clength),
OF wf controlled])
(auto intro!: reachable-verifier-event-bound-for-ge
dest: trace-fri-multiround-bad-sets-subset[THEN subsetD])

lemma *concrete-composition-fri-error-for'-bound:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state ≤ concrete-composition-fri-error-for' budgets A
unfolding *concrete-composition-fri-error-for'-def*
concrete-composition-fri-full-cover-domain-error-for-def
by (*rule*
checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-full-cover-domain-conflict-and-zero
[where bad = composition-fri-full-challenge-bad
and B = λdg. fri-challenge-space (ceil-log (to-nat dg + 1)),
OF wf controlled])
(auto intro!: reachable-verifier-event-bound-for-ge
dest: composition-fri-multiround-bad-sets-subset[THEN subsetD])

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route:*
assumes *false-statement: ¬ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows *checked-staged-adversary-acceptance-probability A ≤*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +

staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 (*1 + staged-concrete-transcript-target-error-bound*)) +
 ($\sum i < \text{rounds}$.
 1 +
 (*1 + staged-concrete-transcript-target-error-bound*) +
 (*1 + staged-concrete-transcript-target-error-bound*))) +
concrete-trace-fri-header-error-for' budgets A +
concrete-composition-fri-error-for' budgets A +
 (*composition-error-bound +*
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 (*1 + staged-concrete-transcript-target-error-bound*) +
 concrete-trace-fri-empty-error-for' budgets A + 1)
 by (rule
 current-query-route-bound-from-active-fri-bounds-internal
 [*OF false-statement wf controlled*
 concrete-trace-fri-header-error-for'-bound[OF wf controlled]
 concrete-composition-fri-error-for'-bound[OF wf controlled]
 concrete-trace-fri-empty-error-for'-bound[OF wf controlled]])

theorem *stark-soundness-concrete-fri-endpoint:*

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows *checked-staged-adversary-acceptance-probability A $\leq$*

hash-collision-budget-value 0

(*staged-attacker-query-budget budgets +*
 staged-challenge-query-budget + verifier-hash-query-budget) +

(*composition-error-bound +*

staged-phase-relation-error size

 (*staged-alpha-search-queries budgets 0*) +

hash-collision-budget-value 0

 (*staged-alpha-search-queries budgets 0*) +

staged-phase-relation-error size

 (*staged-alpha-search-queries budgets 0*) +

 (*1 + staged-concrete-transcript-target-error-bound*)) +

($\sum i < \text{rounds}$.

1 +

 (*1 + staged-concrete-transcript-target-error-bound*) +

 (*1 + staged-concrete-transcript-target-error-bound*))) +

concrete-trace-fri-header-error-for' budgets A +

concrete-composition-fri-error-for' budgets A +

(*composition-error-bound +*

staged-phase-relation-error size

```

    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    (1 + staged-concrete-transcript-target-error-bound) +
    concrete-trace-fri-empty-error-for' budgets A + 1)
  by (rule
    stark-soundness-from-current-public-prefix-current-query-relevant-drift-route
    [OF false-statement wf controlled])

end

hide-fact
  soundness.stark-soundness-concrete-fri-endpoint
  soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route

end

theory Soundness-FRI-Query-Fiber-Staged-Bounds
  imports
    Soundness-FRI-Query-Index-Staged-Bounds
    Soundness-FRI-Query-Aware-Interface
  begin

  Staged sampled-query FRI bounds from generic query fibers.

  The checked staged query-index accounting currently consumes a fixed query target.
  This layer derives such a target from the union of the generic per-challenge query fibers
  outside a chosen challenge-list cover. It keeps the query-aware FRI boundary explicit
  without changing the protocol or the random-oracle model.

  context soundness
  begin

  definition generic-fri-sampled-query-outside-query-cover
    :: ('f list  $\Rightarrow$  bool)  $\Rightarrow$  ('f list  $\Rightarrow$  bool)  $\Rightarrow$  nat  $\Rightarrow$ 
       'f list set  $\Rightarrow$  nat list set
  where
    generic-fri-sampled-query-outside-query-cover low-degree candidate-bad
      degree-bound B =
      ( $\bigcup$  challenges  $\in$ 
        fri-challenge-space (fri-round-count-for-degree-bound degree-bound) - B.
        generic-fri-sampled-query-query-fiber low-degree candidate-bad
        degree-bound challenges)

  definition generic-fri-sampled-query-missing-full-cover-fiber
    :: ('f list  $\Rightarrow$  bool)  $\Rightarrow$  ('f list  $\Rightarrow$  bool)  $\Rightarrow$  nat  $\Rightarrow$ 
       'f list  $\Rightarrow$  nat list set
  where

```

generic-fri-sampled-query-missing-full-cover-fiber low-degree candidate-bad
degree-bound challenges =
 $\{ \text{query-idxs} \in \text{fri-query-index-list-space}.$
 $\exists \text{ candidate-table roots final-value round-layers layers}.$
generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers $\wedge$
 $\neg \text{ fri-sampled-layers-cover-fold-indices roots query-idxs} \}$

lemma *generic-fri-sampled-query-missing-full-cover-fiber-subset:*
generic-fri-sampled-query-missing-full-cover-fiber low-degree candidate-bad
degree-bound challenges $\subseteq$ fri-query-index-list-space
unfolding *generic-fri-sampled-query-missing-full-cover-fiber-def* **by** *auto*

lemma *finite-generic-fri-sampled-query-missing-full-cover-fiber:*
finite
(generic-fri-sampled-query-missing-full-cover-fiber low-degree
candidate-bad degree-bound challenges)
using *generic-fri-sampled-query-missing-full-cover-fiber-subset*
by *(rule finite-subset) (rule finite-fri-query-index-list-space)*

lemma *generic-fri-sampled-query-query-fiber-subset-missing-full-cover:*
assumes *full-cover-imp:*
 $\wedge \text{ candidate-table roots final-value round-layers layers query-idxs}.$
 $\text{query-idxs} \in \text{fri-query-index-list-space} \implies$
generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers $\implies$
fri-sampled-layers-cover-fold-indices roots query-idxs $\implies$
challenges $\in B$
and *challenges-notin: challenges $\notin B$*
shows
generic-fri-sampled-query-query-fiber low-degree candidate-bad
degree-bound challenges $\subseteq$
generic-fri-sampled-query-missing-full-cover-fiber low-degree
candidate-bad degree-bound challenges

proof
fix *query-idxs*
assume *query-in:*
 $\text{query-idxs} \in$
generic-fri-sampled-query-query-fiber low-degree candidate-bad
degree-bound challenges
then have *query-space: query-idxs $\in$ fri-query-index-list-space*
using *generic-fri-sampled-query-query-fiber-subset* **by** *blast*
from *query-in* **obtain** *candidate-table roots final-value round-layers layers*
where *evidence:*
generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers

unfolding *generic-fri-sampled-query-query-fiber-def*
generic-fri-sampled-query-restricted-bad-pairs-def
generic-fri-sampled-query-bad-pair-union-def
generic-fri-sampled-query-bad-pair-set-def
fri-query-challenge-pair-query-fiber-def
by *auto*
have *not-cover*: $\neg$ *fri-sampled-layers-cover-fold-indices roots query-idxs*
proof
assume *cover*: *fri-sampled-layers-cover-fold-indices roots query-idxs*
have *challenges* $\in B$
by (*rule full-cover-imp*[*OF query-space evidence cover*])
then show *False*
using *challenges-notin* **by** *simp*
qed
show *query-idxs* $\in$
generic-fri-sampled-query-missing-full-cover-fiber low-degree
candidate-bad degree-bound challenges
unfolding *generic-fri-sampled-query-missing-full-cover-fiber-def*
by (*intro CollectI conjI exI*)
(rule query-space, rule evidence, rule not-cover)
qed

lemma *generic-fri-sampled-query-query-fiber-entries-card-le-missing-full-cover*:
assumes *full-cover-imp*:
 $\bigwedge$ *candidate-table roots final-value round-layers layers query-idxs.*
query-idxs $\in$ *fri-query-index-list-space* $\implies$
generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers $\implies$
fri-sampled-layers-cover-fold-indices roots query-idxs $\implies$
challenges $\in B$
and *challenges-notin*: *challenges* $\notin B$
shows
card
(query-index-list-entries
(generic-fri-sampled-query-query-fiber low-degree candidate-bad
degree-bound challenges))
 $\leq$
card
(query-index-list-entries
(generic-fri-sampled-query-missing-full-cover-fiber low-degree
candidate-bad degree-bound challenges))
proof –
have *subset*:
generic-fri-sampled-query-query-fiber low-degree candidate-bad
degree-bound challenges $\subseteq$
generic-fri-sampled-query-missing-full-cover-fiber low-degree
candidate-bad degree-bound challenges
by (*rule generic-fri-sampled-query-query-fiber-subset-missing-full-cover*

$[OF \text{ full-cover-imp challenges-notin}]$
have *entries-subset*:
 $\text{query-index-list-entries}$
 $(\text{generic-fri-sampled-query-query-fiber low-degree candidate-bad}$
 $\text{degree-bound challenges})$
 $\subseteq$
 $\text{query-index-list-entries}$
 $(\text{generic-fri-sampled-query-missing-full-cover-fiber low-degree}$
 $\text{candidate-bad degree-bound challenges})$
using *subset unfolding query-index-list-entries-def* **by** *blast*
show *?thesis*
by (*rule card-mono*[*OF - entries-subset*])
 $(\text{rule finite-subset}[OF - \text{finite-query-sample-space}],$
 $\text{unfold query-index-list-entries-def}$
 $\text{generic-fri-sampled-query-missing-full-cover-fiber-def}$
 $\text{fri-query-index-list-space-def, auto})$
qed

lemma *generic-fri-sampled-query-outside-query-cover-subset*:
 $\text{generic-fri-sampled-query-outside-query-cover low-degree candidate-bad}$
 $\text{degree-bound } B \subseteq \text{fri-query-index-list-space}$
unfolding *generic-fri-sampled-query-outside-query-cover-def*
by (*auto intro: generic-fri-sampled-query-query-fiber-subset*[*THEN subsetD*])

lemma *finite-generic-fri-sampled-query-outside-query-cover*:
 finite
 $(\text{generic-fri-sampled-query-outside-query-cover low-degree candidate-bad}$
 $\text{degree-bound } B)$
using *generic-fri-sampled-query-outside-query-cover-subset*
by (*rule finite-subset*) (*rule finite-fri-query-index-list-space*)

lemma *generic-fri-sampled-query-outside-query-cover-card-le-from-fibers*:
 card
 $(\text{generic-fri-sampled-query-outside-query-cover low-degree candidate-bad}$
 $\text{degree-bound } B)$
 $\leq$
 $(\sum \text{challenges} \in$
 $\text{fri-challenge-space } (\text{fri-round-count-for-degree-bound degree-bound}) - B.$
 card
 $(\text{generic-fri-sampled-query-query-fiber low-degree candidate-bad}$
 $\text{degree-bound challenges}))$
unfolding *generic-fri-sampled-query-outside-query-cover-def*
by (*rule card-UN-le*) *simp*

lemma *generic-fri-sampled-query-outside-query-cover-entries-card-le-from-fibers*:
 card
 $(\text{query-index-list-entries}$
 $(\text{generic-fri-sampled-query-outside-query-cover low-degree candidate-bad}$
 $\text{degree-bound } B))$

$$\leq$$

$$\left(\sum_{\text{card}} \text{challenges} \in \text{fri-challenge-space } (\text{fri-round-count-for-degree-bound } \text{degree-bound}) - B. \right.$$

$$\left. (\text{generic-fri-sampled-query-query-fiber low-degree candidate-bad degree-bound challenges}) \right) * \text{rounds}$$
proof –

let $?Q =$

$$\text{generic-fri-sampled-query-outside-query-cover low-degree candidate-bad degree-bound } B$$
let $?K =$

$$\left(\sum_{\text{card}} \text{challenges} \in \text{fri-challenge-space } (\text{fri-round-count-for-degree-bound } \text{degree-bound}) - B. \right.$$

$$\left. (\text{generic-fri-sampled-query-query-fiber low-degree candidate-bad degree-bound challenges}) \right)$$
have $\text{entries-le: card } (\text{query-index-list-entries } ?Q) \leq \text{card } ?Q * \text{rounds}$
by (rule $\text{query-index-list-entries-card-le}$)

 (rule $\text{finite-generic-fri-sampled-query-outside-query-cover}$,

 rule $\text{generic-fri-sampled-query-outside-query-cover-subset}$)

have $\text{cover-le: card } ?Q \leq ?K$
by (rule $\text{generic-fri-sampled-query-outside-query-cover-card-le-from-fibers}$)

have $\text{card } ?Q * \text{rounds} \leq ?K * \text{rounds}$
by (rule $\text{mult-right-mono}[OF \text{ cover-le}] \text{ simp}$)

then show $?thesis$
using entries-le **by** linarith

qed

lemma $\text{trace-fri-sampled-query-outside-projection-subset}$:

fst '

$$((\text{trace-fri-sampled-query-bad-pair-union} \cap$$

$$(\text{fri-query-index-list-space} \times \text{UNIV})) \cap$$

$$(\text{UNIV} \times (- B)))$$

$$\subseteq$$

$$\text{generic-fri-sampled-query-outside-query-cover trace-table-low-degree}$$

$$(\text{Not} \circ \text{trace-table-low-degree}) (\text{clength} - 1) B$$
unfolding $\text{trace-fri-sampled-query-bad-pair-union-def}$

$$\text{generic-fri-sampled-query-outside-query-cover-def}$$

$$\text{generic-fri-sampled-query-query-fiber-def}$$

$$\text{generic-fri-sampled-query-restricted-bad-pairs-def}$$

$$\text{fri-query-challenge-pair-query-fiber-def}$$
using $\text{generic-fri-sampled-query-bad-pair-union-subset-challenge-space}$

$$[\text{of trace-table-low-degree Not} \circ \text{trace-table-low-degree clength} - 1]$$
by force

definition $\text{composition-fri-sampled-query-outside-query-cover}$

$:: ('f \Rightarrow 'f \text{ list set}) \Rightarrow \text{nat list set}$

where

$\text{composition-fri-sampled-query-outside-query-cover } B =$

$(\bigcup dg \in (UNIV :: 'f \text{ set}).$
 $\text{generic-fri-sampled-query-outside-query-cover}$
 $(\text{composition-table-low-degree } (to\text{-nat } dg))$
 $(Not \circ \text{composition-table-low-degree } maxDegree)$
 $(to\text{-nat } dg) (B \text{ } dg))$

lemma *composition-fri-sampled-query-outside-query-cover-subset*:
 $\text{composition-fri-sampled-query-outside-query-cover } B$
 $\subseteq \text{fri-query-index-list-space}$
unfolding *composition-fri-sampled-query-outside-query-cover-def*
by (*auto intro*:
 $\text{generic-fri-sampled-query-outside-query-cover-subset}[THEN \text{subsetD}]$)

lemma *finite-composition-fri-sampled-query-outside-query-cover*:
 $\text{finite } (\text{composition-fri-sampled-query-outside-query-cover } B)$
using *composition-fri-sampled-query-outside-query-cover-subset*
by (*rule finite-subset*) (*rule finite-fri-query-index-list-space*)

lemma *composition-fri-sampled-query-outside-query-cover-card-le-from-fibers*:
 $\text{card } (\text{composition-fri-sampled-query-outside-query-cover } B)$
 $\leq$
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\sum \text{challenges} \in$
 $\text{fri-challenge-space } (\text{fri-round-count-for-degree-bound } (to\text{-nat } dg)) -$
 $B \text{ } dg.$
 card
 $(\text{generic-fri-sampled-query-query-fiber}$
 $(\text{composition-table-low-degree } (to\text{-nat } dg))$
 $(Not \circ \text{composition-table-low-degree } maxDegree)$
 $(to\text{-nat } dg) \text{ challenges}))$

proof –
let $?Q = \text{composition-fri-sampled-query-outside-query-cover } B$
let $?G = \lambda dg.$
 $\text{generic-fri-sampled-query-outside-query-cover}$
 $(\text{composition-table-low-degree } (to\text{-nat } dg))$
 $(Not \circ \text{composition-table-low-degree } maxDegree)$
 $(to\text{-nat } dg) (B \text{ } dg)$
let $?K = \lambda dg.$
 $(\sum \text{challenges} \in$
 $\text{fri-challenge-space } (\text{fri-round-count-for-degree-bound } (to\text{-nat } dg)) -$
 $B \text{ } dg.$
 card
 $(\text{generic-fri-sampled-query-query-fiber}$
 $(\text{composition-table-low-degree } (to\text{-nat } dg))$
 $(Not \circ \text{composition-table-low-degree } maxDegree)$
 $(to\text{-nat } dg) \text{ challenges}))$
have $\text{card } ?Q \leq (\sum dg \in (UNIV :: 'f \text{ set}). \text{card } (?G \text{ } dg))$
unfolding *composition-fri-sampled-query-outside-query-cover-def*
by (*rule card-UN-le*) *simp*

also have ... $\leq (\sum dg \in (UNIV :: 'f \text{ set}). ?K \ dg)$
by (rule *sum-mono*)
(rule *generic-fri-sampled-query-outside-query-cover-card-le-from-fibers*)
finally show *?thesis* .
qed

lemma *composition-fri-sampled-query-outside-query-cover-entries-card-le-from-fibers:*
card
(query-index-list-entries
(composition-fri-sampled-query-outside-query-cover *B*))
 $\leq$
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\sum challenges \in$
fri-challenge-space (*fri-round-count-for-degree-bound* (*to-nat dg*)) -
B dg.
card
(*generic-fri-sampled-query-query-fiber*
(*composition-table-low-degree* (*to-nat dg*))
(*Not* $\circ$ *composition-table-low-degree maxDegree*)
(*to-nat dg challenges*)) * *rounds*

proof -
let *?Q* = *composition-fri-sampled-query-outside-query-cover B*
let *?K* =
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\sum challenges \in$
fri-challenge-space (*fri-round-count-for-degree-bound* (*to-nat dg*)) -
B dg.
card
(*generic-fri-sampled-query-query-fiber*
(*composition-table-low-degree* (*to-nat dg*))
(*Not* $\circ$ *composition-table-low-degree maxDegree*)
(*to-nat dg challenges*))
have *entries-le:* card (query-index-list-entries *?Q*) $\leq$ card *?Q* * *rounds*
by (rule *query-index-list-entries-card-le*)
(rule *finite-composition-fri-sampled-query-outside-query-cover*,
rule *composition-fri-sampled-query-outside-query-cover-subset*)
have *cover-le:* card *?Q* $\leq$ *?K*
by (rule *composition-fri-sampled-query-outside-query-cover-card-le-from-fibers*)
have card *?Q* * *rounds* $\leq$ *?K* * *rounds*
by (rule *mult-right-mono[OF cover-le]*) *simp*
then show *?thesis*
using *entries-le* **by** *linarith*
qed

lemma *composition-fri-sampled-query-outside-projection-subset:*
fst '
((*composition-fri-sampled-query-bad-pair-union dg* $\cap$
(*fri-query-index-list-space* $\times$ *UNIV*)) $\cap$
(*UNIV* $\times$ ($-$ *B dg*)))

$\subseteq$ *composition-fri-sampled-query-outside-query-cover B*
unfolding *composition-fri-sampled-query-bad-pair-union-def*
composition-fri-sampled-query-outside-query-cover-def
generic-fri-sampled-query-outside-query-cover-def
generic-fri-sampled-query-query-fiber-def
generic-fri-sampled-query-restricted-bad-pairs-def
fri-query-challenge-pair-query-fiber-def
using *generic-fri-sampled-query-bad-pair-union-subset-challenge-space*
[*of composition-table-low-degree (to-nat dg)*
Not $\circ$ composition-table-low-degree maxDegree to-nat dg]
by force

lemma *checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-outsid*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *finite-B: finite B*

and *fresh-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

($\lambda s.$ trace-fri-challenge-list-fresh-hit s B))

adversary-initial-state $\leq$ F

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

trace-fri-sampled-query-bad-candidate)

adversary-initial-state $\leq$

F +

*hash-relation-budget-value (card B * ceil-log clength)*

(staged-attacker-query-budget budgets + staged-challenge-query-budget) +

staged-phase-target-error

(query-index-raw-preimage

(query-index-list-entries

(generic-fri-sampled-query-outside-query-cover

trace-table-low-degree (Not $\circ$ trace-table-low-degree)

(clength - 1) B)))

(staged-attacker-query-budget budgets +

staged-challenge-query-budget)

by *(rule*

checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-projection

[OF wf controlled finite-B fresh-bound

trace-fri-sampled-query-outside-projection-subset])

lemma *checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-outsid*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *finite-B: finite B*

and *fresh-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

```

    ( $\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B$ ))
  adversary-initial-state  $\leq F$ 
and query-fraction:
  nnreal
    (card
      (query-index-list-entries
        (generic-fri-sampled-query-outside-query-cover
          trace-table-low-degree (Not  $\circ$  trace-table-low-degree)
          (clength - 1) B))) /
      nnreal (card query-sample-space)  $\leq Q$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-sampled-query-bad-candidate)
  adversary-initial-state  $\leq$ 
    F +
    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget) +
    nnreal (staged-attacker-query-budget budgets +
      staged-challenge-query-budget) * Q
proof -
  let ?Qset =
    generic-fri-sampled-query-outside-query-cover trace-table-low-degree
    (Not  $\circ$  trace-table-low-degree) (clength - 1) B
  have base:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-sampled-query-bad-candidate)
    adversary-initial-state  $\leq$ 
      F +
      hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets + staged-challenge-query-budget) +
      staged-phase-target-error
      (query-index-raw-preimage (query-index-list-entries ?Qset))
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
  by (rule
    checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-outside-fil
    [OF wf controlled finite-B fresh-bound])
  have subset:
    query-index-list-entries ?Qset  $\subseteq$  query-sample-space
  by (rule query-index-list-entries-subset-query-sample-space)
    (rule generic-fri-sampled-query-outside-query-cover-subset)
  have target-bound:
    staged-phase-target-error
      (query-index-raw-preimage (query-index-list-entries ?Qset))
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
     $\leq$ 

```

$nnreal$ ($staged\text{-}attacker\text{-}query\text{-}budget$ $budgets$ +
 $staged\text{-}challenge\text{-}query\text{-}budget$) * Q
by ($rule$ $staged\text{-}phase\text{-}query\text{-}index\text{-}target\text{-}error\text{-}query\text{-}bound$
 $[OF$ $query\text{-}index\text{-}raw\text{-}preimage\text{-}bound\text{-}from\text{-}sampler\text{-}wellformed$ $subset$
 $query\text{-}fraction]$)
show $?thesis$
by ($rule$ $order\text{-}trans[OF$ $base]$)
 $(use$ $target\text{-}bound$ **in** $simp$)
qed

lemma $checked\text{-}staged\text{-}security\text{-}trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\text{-}bound\text{-}from\text{-}generic\text{-}challenge\text{-}cover\text{-}and\text{-}outsid$

assumes wf : $staged\text{-}budget\text{-}wellformed$ $budgets$

and $controlled$: $staged\text{-}adversary\text{-}controlled$ $budgets$ A

and $finite\text{-}B$: $finite$ B

and $fresh\text{-}bound$:

$wp\text{-}event$ ($checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state$ A)

($staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$

($\lambda s. trace\text{-}fri\text{-}challenge\text{-}list\text{-}fresh\text{-}hit$ s B))

$adversary\text{-}initial\text{-}state \leq F$

and $query\text{-}fraction$:

$nnreal$

($(\sum$ $challenges \in$

$fri\text{-}challenge\text{-}space$ ($fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound$ ($clength - 1$)) -
 B .

$card$

($generic\text{-}fri\text{-}sampled\text{-}query\text{-}query\text{-}fiber$ $trace\text{-}table\text{-}low\text{-}degree$

($Not \circ trace\text{-}table\text{-}low\text{-}degree$) ($clength - 1$) $challenges$)) *
 $rounds$) /

$nnreal$ ($card$ $query\text{-}sample\text{-}space$) $\leq Q$

shows

$wp\text{-}event$ ($checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state$ A)

($staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$

$trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate$)

$adversary\text{-}initial\text{-}state \leq$

$F +$

$hash\text{-}relation\text{-}budget\text{-}value$ ($card$ $B * ceil\text{-}log$ $clength$)

($staged\text{-}attacker\text{-}query\text{-}budget$ $budgets + staged\text{-}challenge\text{-}query\text{-}budget$) +

$nnreal$ ($staged\text{-}attacker\text{-}query\text{-}budget$ $budgets +$

$staged\text{-}challenge\text{-}query\text{-}budget$) * Q

proof ($rule$

$checked\text{-}staged\text{-}security\text{-}trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\text{-}bound\text{-}from\text{-}generic\text{-}challenge\text{-}cover\text{-}and\text{-}outside\text{-}fiber$

$[OF$ wf $controlled$ $finite\text{-}B$ $fresh\text{-}bound]$)

let $?Entries =$

$query\text{-}index\text{-}list\text{-}entries$

($generic\text{-}fri\text{-}sampled\text{-}query\text{-}outside\text{-}query\text{-}cover$ $trace\text{-}table\text{-}low\text{-}degree$

($Not \circ trace\text{-}table\text{-}low\text{-}degree$) ($clength - 1$) B)

let $?K =$

($\sum$ $challenges \in$

$fri\text{-}challenge\text{-}space$ ($fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound$ ($clength - 1$)) -

B .
card
 (generic-fri-sampled-query-query-fiber trace-table-low-degree
 (Not $\circ$ trace-table-low-degree) (clength - 1) challenges)) *
rounds
have card-le: *card* ?Entries $\leq$?K
by (rule
 generic-fri-sampled-query-outside-query-cover-entries-card-le-from-fibers)
have nnreal (card ?Entries) / nnreal (card query-sample-space)
 $\leq$ nnreal ?K / nnreal (card query-sample-space)
using card-le **by** (rule nnreal-nat-divide-right-mono)
also have ... $\leq$ Q
by (rule query-fraction)
finally show
 nnreal (card ?Entries) / nnreal (card query-sample-space) $\leq$ Q .
qed

lemma checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: $\bigwedge dg. \text{finite } (B \text{ dg})$
and fresh-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 ($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \text{ B}$))
 adversary-initial-state $\leq$ F
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-sampled-query-bad-candidate)
 adversary-initial-state $\leq$
 F +
 hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \text{ set}).$
 card (B dg) * ceil-log (maxDegree + 1))
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget) +
 staged-phase-target-error
 (query-index-raw-preimage
 (query-index-list-entries
 (composition-fri-sampled-query-outside-query-cover B)))
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
by (rule
 checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-projection
 [OF wf controlled finite-B fresh-bound
 composition-fri-sampled-query-outside-projection-subset])

lemma checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *finite-B*: $\bigwedge dg. \text{finite } (B \text{ } dg)$
and *fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \text{ } B$))
adversary-initial-state $\leq F$
and *query-fraction*:
nnreal
(card
(query-index-list-entries
(composition-fri-sampled-query-outside-query-cover *B*))) /
nnreal (card *query-sample-space*) $\leq Q$
shows
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
composition-fri-sampled-query-bad-candidate)
adversary-initial-state $\leq$
F +
hash-relation-budget-value
($\sum dg \in (UNIV :: 'f \text{ set}).$
card (*B* *dg*) * *ceil-log* (*maxDegree* + 1))
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget) +
nnreal (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget) * *Q*
proof –
let *?Qset* = *composition-fri-sampled-query-outside-query-cover* *B*
have *base*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
composition-fri-sampled-query-bad-candidate)
adversary-initial-state $\leq$
F +
hash-relation-budget-value
($\sum dg \in (UNIV :: 'f \text{ set}).$
card (*B* *dg*) * *ceil-log* (*maxDegree* + 1))
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget) +
staged-phase-target-error
(*query-index-raw-preimage* (*query-index-list-entries* *?Qset*))
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget)
by (rule
checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-ou
[*OF wf controlled finite-B fresh-bound*])
have *subset*:
query-index-list-entries *?Qset* $\subseteq$ *query-sample-space*

by (rule *query-index-list-entries-subset-query-sample-space*)
 (rule *composition-fri-sampled-query-outside-query-cover-subset*)
have *target-bound*:
staged-phase-target-error
 (query-index-row-preimage (query-index-list-entries ?Qset))
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
 ≤
 nnreal (staged-attacker-query-budget budgets +
 staged-challenge-query-budget) * Q
by (rule *staged-phase-query-index-target-error-query-bound*
 [OF query-index-row-preimage-bound-from-sampler-wellformed subset
 query-fraction])
show ?thesis
by (rule *order-trans*[OF base])
 (use *target-bound* **in** *simp*)
qed

lemma *checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and*

assumes *wf*: *staged-budget-wellformed* budgets
and *controlled*: *staged-adversary-controlled* budgets A
and *finite-B*: $\bigwedge dg. \text{finite } (B \ dg)$
and *fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$))
adversary-initial-state ≤ F
and *query-fraction*:
 nnreal
 (($\sum dg \in (UNIV :: 'f \ set).$
 $\sum challenges \in$
 fri-challenge-space (*fri-round-count-for-degree-bound* (to-nat dg)) –
 B dg.
 card
 (*generic-fri-sampled-query-query-fiber*
 (*composition-table-low-degree* (to-nat dg))
 (Not ◦ *composition-table-low-degree* maxDegree)
 (to-nat dg) challenges)) * rounds) /
 nnreal (*card* *query-sample-space*) ≤ Q

shows

wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-sampled-query-bad-candidate)
adversary-initial-state ≤
 F +
hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \ set).$
 card (B dg) * *ceil-log* (maxDegree + 1))
 (staged-attacker-query-budget budgets +

```

      staged-challenge-query-budget) +
      nnreal (staged-attacker-query-budget budgets +
      staged-challenge-query-budget) * Q
proof (rule
  checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-outside
  [OF wf controlled finite-B fresh-bound])
let ?Entries =
  query-index-list-entries
  (composition-fri-sampled-query-outside-query-cover B)
let ?K =
  (∑ dg ∈ (UNIV :: 'f set).
    ∑ challenges ∈
      fri-challenge-space (fri-round-count-for-degree-bound (to-nat dg)) -
      B dg.
    card
      (generic-fri-sampled-query-query-fiber
        (composition-table-low-degree (to-nat dg))
        (Not ∘ composition-table-low-degree maxDegree)
        (to-nat dg) challenges)) * rounds
have card-le: card ?Entries ≤ ?K
by (rule
  composition-fri-sampled-query-outside-query-cover-entries-card-le-from-fibers)
have nnreal (card ?Entries) / nnreal (card query-sample-space)
  ≤ nnreal ?K / nnreal (card query-sample-space)
using card-le by (rule nnreal-nat-divide-right-mono)
also have ... ≤ Q
by (rule query-fraction)
finally show
  nnreal (card ?Entries) / nnreal (card query-sample-space) ≤ Q .
qed

end

end

```

```

theory Soundness-FRI-Query-List-Prequery
imports
  Stark-Core.Merkle-Tree
  Soundness-Core-Events
  Soundness-FRI-Query-Exact-Bounds
  Soundness-FRI-Query-Index-Staged-Bounds
begin

```

Query-list prequery accounting for FRI. The lemmas in this theory relate verifier-generated query index lists to random-oracle budget events used in the staged soundness proof.

```

context soundness
begin

```

lemma *set-dist-return-valueD*:
assumes $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (\text{return } x) \ s)$
shows $y = x$
using *assms*
unfolding *set-dist-def return.rep-eq dist-return-def*
by (*simp add: dist-delta-dist delta-map-def*)

lemma *set-dist-return-stateD*:
assumes $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (\text{return } x) \ s)$
shows $t = s$
using *assms*
unfolding *set-dist-def return.rep-eq dist-return-def*
by (*simp add: dist-delta-dist delta-map-def*)

lemma *set-dist-bind-return-consE*:
assumes *support*:
 $\text{out} \in \text{set-dist } (\text{execute } (m \gg (\lambda xs. \text{return } (u \# xs))) \ s)$
and *out-eq*: $\text{out} = \text{Some } (ys, t0)$
obtains *results t where*
 $\text{Some } (\text{results}, t) \in \text{set-dist } (\text{execute } m \ s)$
 $ys = u \# \text{results}$
 $t0 = t$
 $\text{out} = \text{Some } (u \# \text{results}, t)$
proof –
from *support[unfolded out-eq]*
obtain *results t where tail*:
 $\text{Some } (\text{results}, t) \in \text{set-dist } (\text{execute } m \ s)$
and *ret*:
 $\text{Some } (ys, t0) \in \text{set-dist } (\text{execute } (\text{return } (u \# \text{results})) \ t)$
by (*auto elim!: set-dist-bindE*)
have *ys-eq*: $ys = u \# \text{results}$
by (*rule set-dist-return-valueD[OF ret]*)
have *t0-eq*: $t0 = t$
by (*rule set-dist-return-stateD[OF ret]*)
have *out-eq'*: $\text{out} = \text{Some } (u \# \text{results}, t)$
using *out-eq ys-eq t0-eq* **by** *simp*
show *?thesis*
by (*rule that[OF tail ys-eq t0-eq out-eq']*)
qed

lemma *set-dist-bind-return-cons-SomeE*:
assumes *support*:
 $\text{Some } (ys, t) \in \text{set-dist } (\text{execute } (m \gg (\lambda xs. \text{return } (u \# xs))) \ s)$
obtains *xs where*
 $\text{Some } (xs, t) \in \text{set-dist } (\text{execute } m \ s)$
 $ys = u \# xs$
proof –
from *support* **obtain** *xs t0 where tail*:
 $\text{Some } (xs, t0) \in \text{set-dist } (\text{execute } m \ s)$


```

    and ys-eq:  $ys = u \# xs$ 
    and t-eq:  $t = t0$ 
    by (elim set-dist-bind-return-consE) simp
  have tail-t:  $\text{Some } (xs, t) \in \text{set-dist } (\text{execute } m \ s)$ 
    by (subst t-eq, rule tail)
  show ?thesis
    by (rule that[OF tail-t ys-eq])
qed

lemma wp-event-nonzero-imp-exists-support:
  assumes wp-event  $m \ P \ s \neq 0$ 
  obtains out where  $\text{out} \in \text{set-dist } (\text{execute } m \ s)$  and  $P \ \text{out}$ 
  by (rule wp-event-pos-imp-exists-support)
    (rule zero-less-iff-neq-zero[THEN iffD2], rule assms)

lemma wp-event-false [simp]:
  wp-event  $m \ (\lambda-. \text{False}) \ s = 0$ 
  unfolding wp-event-def wp-def dist-expect-def
  by simp

lemma wp-event-bind-return-cons-mono:
  fixes  $m :: ('a \text{ list}, 's) \text{ state-monad}$ 
  assumes none-imp:  $Q \ \text{None} \implies R \ \text{None}$ 
  and some-imp:
     $\bigwedge xs \ t. \text{Some } (xs, t) \in \text{set-dist } (\text{execute } m \ s) \implies$ 
     $Q \ (\text{Some } (u \# xs, t)) \implies R \ (\text{Some } (xs, t))$ 
  shows
     $\text{wp-event } (m \gg (\lambda xs. \text{return } (u \# xs))) \ Q \ s \leq$ 
     $\text{wp-event } m \ R \ s$ 
  proof (rule wp-event-bind-bound-by-head-event)
    show  $\text{wp-event } m \ R \ s \leq \text{wp-event } m \ R \ s$ 
      by (rule order.refl)
  next
    show  $Q \ \text{None} \implies R \ \text{None}$ 
      by (rule none-imp)
  next
    fix  $xs \ t \ \text{out}$ 
    assume support:  $\text{Some } (xs, t) \in \text{set-dist } (\text{execute } m \ s)$ 
    and ret:  $\text{out} \in \text{set-dist } (\text{execute } (\text{return } (u \# xs)) \ t)$ 
    and q:  $Q \ \text{out}$ 
    have out-eq:  $\text{out} = \text{Some } (u \# xs, t)$ 
      using ret
    unfolding set-dist-def return.rep-eq dist-return-def
      dist-delta-dist delta-map-def
    by simp
    have q-some:  $Q \ (\text{Some } (u \# xs, t))$ 
      using q by (simp only: out-eq)
    show  $R \ (\text{Some } (xs, t))$ 
      by (rule some-imp[OF support q-some])

```

qed

lemma *eq-Suc-imp-less-nat*:

assumes $m = \text{Suc } n$
shows $n < m$
by (*subst assms, rule lessI*)

lemma *list-length-SucE*:

assumes $\text{length } xs = \text{Suc } n$
obtains $x \ xs'$ **where** $xs = x \# xs'$
using *assms* **by** (*cases xs*) *simp-all*

lemma *state-after-query-chunks-cons-shift*:

assumes $s1 = \text{state-after-query-chunks } s \ (\text{chunk} \# \text{chunks}) \ 1$
shows
 $\text{state-after-query-chunks } s1 \ \text{chunks } n =$
 $\text{state-after-query-chunks } s \ (\text{chunk} \# \text{chunks}) \ (\text{Suc } n)$
using *assms* **unfolding** *state-after-query-chunks-def* **by** *simp*

lemma *state-after-query-chunks-cons-shift-index*:

assumes $s1 = \text{state-after-query-chunks } s \ (\text{chunk} \# \text{chunks}) \ 1$
shows
 $\text{state-after-query-chunks } s1 \ \text{chunks } i =$
 $\text{state-after-query-chunks } s \ (\text{chunk} \# \text{chunks}) \ (\text{Suc } i)$
using *assms* **unfolding** *state-after-query-chunks-def* **by** *simp*

lemma *state-after-query-chunks-one-cons*:

$\text{state-after-query-chunks } s \ (\text{chunk} \# \text{chunks}) \ 1 = \text{foldl concat } s \ \text{chunk}$
unfolding *state-after-query-chunks-def* **by** *simp*

end

Query-list prequery accounting for sampled FRI.

The exact verifier-local sampled-query bounds are list-valued: they bound the probability of drawing a whole query-index list from a target set. In the staged adversary experiment the verifier reuses the adversary's oracle map, so an exact staged theorem needs a fresh/prequeried split for the whole query-index list. This layer provides the conservative relation-budget side of that split without changing the oracle model.

context *soundness*

begin

lemma *verifier-query-round-program-alt-outcomeE*:

assumes *outcome*:
 $\text{Some } ((), t) \in$
 set-dist
 $(\text{execute}$
 $(\text{verifier-query-round-program } \text{fr } f\text{-fl } f\text{-final } \text{as } \text{fl } \text{final}) \ s)$
obtains *raw s0* **where**

$\text{Some } (raw, s0) \in \text{set-dist } (\text{execute receive-query-index-challenge } s)$
 $\text{Some } ((), t) \in$
 set-dist
 $(\text{execute}$
 $(\text{verifier-query-round-after-index-program } fr \ f-fl \ f-final \text{ as } fl$
 $\text{final } raw) \ s0)$
using $\text{outcome}[\text{unfolded verifier-query-round-program-alt-def}]$
by $(\text{auto elim! : set-dist-bindE})$

lemma *verifier-query-round-parts-counter*:

assumes recv :
 $\text{Some } (raw, s0) \in \text{set-dist } (\text{execute receive-query-index-challenge } s)$
and **after**:
 $\text{Some } ((), t) \in$
 set-dist
 $(\text{execute}$
 $(\text{verifier-query-round-after-index-program } fr \ f-fl \ f-final \text{ as } fl$
 $\text{final } raw) \ s0)$
shows $PQueryCounter \ t = \text{Suc } (PQueryCounter \ s)$
proof –
have $\text{counter-s0} : PQueryCounter \ s0 = \text{Suc } (PQueryCounter \ s)$
using $\text{receive-query-index-challenge-counter-outcome}[OF \ \text{recv}]$ **by** *simp*
have $\text{counter-t-s0} : PQueryCounter \ t = PQueryCounter \ s0$
by $(\text{rule conjunct2}[OF \ \text{verifier-query-round-after-index-program-outcome}$
 $[OF \ \text{after}]])$
show *?thesis*
proof (rule HOL.trans)
show $PQueryCounter \ t = PQueryCounter \ s0$
by $(\text{rule counter-t-s0})$
show $PQueryCounter \ s0 = \text{Suc } (PQueryCounter \ s)$
by (rule counter-s0)
qed
qed

definition *query-index-raw-list-preimage* $:: \text{nat list set} \Rightarrow 'f \text{ list set}$

where
 $\text{query-index-raw-list-preimage } Q =$
 $\{\text{rows. length rows} = \text{rounds} \wedge$
 $\text{map } (\lambda \text{raw. index } (\text{to-nat } \text{raw})) \ \text{rows} \in Q\}$

definition *query-index-raw-list-position-values*

$:: \text{nat list set} \Rightarrow \text{nat} \Rightarrow 'f \text{ set}$
where
 $\text{query-index-raw-list-position-values } Q \ i =$
 $\{\text{raw. } \exists \text{rows} \in \text{query-index-raw-list-preimage } Q.$
 $i < \text{rounds} \wedge \text{raw} = \text{rows} ! i\}$

definition *query-index-raw-list-relation-fiber-bound*

$:: \text{nat list set} \Rightarrow \text{nat}$

where

query-index-raw-list-relation-fiber-bound $Q =$
 $(\sum i < \text{rounds}. \text{card } (\text{query-index-raw-list-position-values } Q \ i))$

definition *staged-transcript-query-index-list-set-hit*

$:: \text{nat list set} \Rightarrow$
 $(\text{'f staged-proof-data} \times \text{'f protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

staged-transcript-query-index-list-set-hit $Q \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (\text{data}, \text{attacker-state}) \Rightarrow$
 $(\exists \text{raw-idxs query-idxs}.$
 $\text{length raw-idxs} = \text{rounds} \wedge$
 $\text{query-idxs} \in Q \wedge$
 $\text{query-idxs} = \text{map } (\lambda \text{raw}. \text{index } (\text{to-nat raw})) \text{ raw-idxs} \wedge$
 $(\forall i < \text{rounds}.$
 $\text{fmlookup } (\text{HashMap attacker-state})$
 $(\text{QueryIndexChallenge } i$
 $(\text{state-after-query-chunks}$
 $(\text{staged-query-start-hash data})$
 $(\text{staged-query-chunks data } i)) =$
 $\text{Some } (\text{raw-idxs ! } i))))$

definition *staged-security-with-data-query-index-list-set-hit*

$:: \text{nat list set} \Rightarrow$
 $((\text{'f staged-proof-data} \times \text{'f protocol-channel}) \times$
 $\text{unit list}) \times \text{'f protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

staged-security-with-data-query-index-list-set-hit $Q \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((\text{data}, -), \text{result}), \text{final-state}) \Rightarrow$
 $(\exists \text{raw-idxs query-idxs}.$
 $\text{length raw-idxs} = \text{rounds} \wedge$
 $\text{query-idxs} \in Q \wedge$
 $\text{query-idxs} = \text{map } (\lambda \text{raw}. \text{index } (\text{to-nat raw})) \text{ raw-idxs} \wedge$
 $(\forall i < \text{rounds}.$
 $\text{fmlookup } (\text{HashMap final-state})$
 $(\text{QueryIndexChallenge } i$
 $(\text{state-after-query-chunks}$
 $(\text{staged-query-start-hash data})$
 $(\text{staged-query-chunks data } i)) =$
 $\text{Some } (\text{raw-idxs ! } i))))$

definition *staged-security-with-data-state-query-index-list-path-fresh*

$:: \text{nat list set} \Rightarrow$
 $((\text{'f staged-proof-data} \times \text{'f protocol-channel}) \times$
 $\text{unit list}) \times \text{'f protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

staged-security-with-data-state-query-index-list-path-fresh Q $out \longleftrightarrow$
 (case out of
 None $\Rightarrow$ *False*
 | *Some* (((*data*, *attacker-state*), *result*), *final-state*) $\Rightarrow$
 ($\exists$ *raw-idxs* *query-idxs*.
 length *raw-idxs* = *rounds* $\wedge$
 query-idxs $\in Q$ $\wedge$
 query-idxs = *map* (λ *raw*. *index* (*to-nat* *raw*)) *raw-idxs* $\wedge$
 ($\forall$ $i < rounds$.
 fmlookup (*HashMap* *final-state*)
 (*QueryIndexChallenge* i
 (*state-after-query-chunks*
 (*staged-query-start-hash* *data*)
 (*staged-query-chunks* *data*) i)) =
 Some (*raw-idxs* ! i) $\wedge$
 fmlookup
 (*HashMap*
 (*verifier-state-from-adversary* *attacker-state*
 (*staged-proof-transcript* *data*)))
 (*QueryIndexChallenge* i
 (*state-after-query-chunks*
 (*staged-query-start-hash* *data*)
 (*staged-query-chunks* *data*) i)) =
 None)))

definition *checked-staged-query-index-list-relation*

$:: 'f$ *staged-adversary* $\Rightarrow$ *nat list set* $\Rightarrow$
 'f *protocol-hash-input* $\Rightarrow$ *'f* $\Rightarrow$ *bool*

where

checked-staged-query-index-list-relation A Q x $y \longleftrightarrow$
 ($\exists$ *data* *attacker-state* *raw-idxs* i .
 Some (*data*, *attacker-state*) $\in$
 set-dist
 (*execute* (*checked-staged-transcript-program* A)
 adversary-initial-state) $\wedge$
 raw-idxs $\in$ *query-index-raw-list-preimage* Q $\wedge$
 $i < rounds$ $\wedge$
 $x =$ *QueryIndexChallenge* i
 (*state-after-query-chunks*
 (*staged-query-start-hash* *data*)
 (*staged-query-chunks* *data*) i) $\wedge$
 $y =$ *raw-idxs* ! i)

definition *staged-security-with-data-state-query-index-list-prequery-hit*

$:: 'f$ *staged-adversary* $\Rightarrow$ *nat list set* $\Rightarrow$
 (((*'f* *staged-proof-data* $\times$ *'f* *protocol-channel*) $\times$
 unit list) $\times$ *'f* *protocol-channel*) *option* $\Rightarrow$ *bool*

where

staged-security-with-data-state-query-index-list-prequery-hit A Q out
 $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((data, attacker-state), result), final-state) $\Rightarrow$
 hash-relation-hit
 (checked-staged-query-index-list-relation A Q)
 adversary-initial-state attacker-state)

definition *query-index-list-path-fresh*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{nat} \Rightarrow 'f \text{ list list} \Rightarrow$
 $'f \text{ list} \Rightarrow \text{bool}$

where

query-index-list-path-fresh s n chunks raws $\longleftrightarrow$
 $\text{length raws} = n \wedge$
 $\text{length chunks} = n \wedge$
 $(\forall i < n.$
 fmlookup (HashMap s)
 (QueryIndexChallenge (PQueryCounter s + i)
 (state-after-query-chunks (PState s) chunks i)) = None)

lemma *query-index-list-path-fresh-head-fresh:*

assumes

query-index-list-path-fresh s (Suc n) chunks (raw # raws)

shows

fmlookup (HashMap s)
 (QueryIndexChallenge (PQueryCounter s) (PState s)) = None

using *assms* **unfolding** *query-index-list-path-fresh-def*

state-after-query-chunks-def

by *auto*

lemma *query-index-list-path-fresh-lookupD:*

assumes *fresh: query-index-list-path-fresh s n chunks raws*

and *i-bound: i < n*

shows

fmlookup (HashMap s)
 (QueryIndexChallenge (PQueryCounter s + i)
 (state-after-query-chunks (PState s) chunks i)) = None

proof –

have *all:*

$\forall j < n.$

fmlookup (HashMap s)
 (QueryIndexChallenge (PQueryCounter s + j)
 (state-after-query-chunks (PState s) chunks j)) = None

using *fresh* **unfolding** *query-index-list-path-fresh-def*

by (*elim conjE*)

show *?thesis*

by (*rule all[rule-format, OF i-bound]*)

qed

definition *query-rounds-raw-list-path-hit*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
query-rounds-raw-list-path-hit $s \text{ raws } n \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some (results, t)} \Rightarrow$
 $(\exists \text{ chunks.}$
 $\text{length raws} = n \wedge$
 $\text{length chunks} = n \wedge$
 $P\text{State } t = \text{state-after-query-chunks } (P\text{State } s) \text{ chunks } n \wedge$
 $\text{query-index-list-path-fresh } s \text{ } n \text{ chunks raws} \wedge$
 $(\forall i < n.$
 $\text{fmlookup (HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s + i)$
 $(\text{state-after-query-chunks } (P\text{State } s) \text{ chunks } i)) =$
 $\text{Some (raws ! i)})))$

lemma *query-rounds-raw-list-path-hitI*:
assumes *len-raws*: $\text{length raws} = n$
and *len-chunks*: $\text{length chunks} = n$
and *state*:
 $P\text{State } t = \text{state-after-query-chunks } (P\text{State } s) \text{ chunks } n$
and *fresh*: $\text{query-index-list-path-fresh } s \text{ } n \text{ chunks raws}$
and *lookup*:
 $\bigwedge i. i < n \implies$
 $\text{fmlookup (HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s + i)$
 $(\text{state-after-query-chunks } (P\text{State } s) \text{ chunks } i)) =$
 Some (raws ! i)
shows *query-rounds-raw-list-path-hit* $s \text{ raws } n \text{ (Some (results, t))}$
proof –
have *rhs-eq*:
 $\text{query-rounds-raw-list-path-hit } s \text{ raws } n \text{ (Some (results, t))} =$
 $(\exists \text{ chunks.}$
 $\text{length raws} = n \wedge$
 $\text{length chunks} = n \wedge$
 $P\text{State } t = \text{state-after-query-chunks } (P\text{State } s) \text{ chunks } n \wedge$
 $\text{query-index-list-path-fresh } s \text{ } n \text{ chunks raws} \wedge$
 $(\forall i < n.$
 $\text{fmlookup (HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s + i)$
 $(\text{state-after-query-chunks } (P\text{State } s) \text{ chunks } i)) =$
 $\text{Some (raws ! i)})))$
unfolding *query-rounds-raw-list-path-hit-def* **by** *simp*
show *?thesis*
by (*subst rhs-eq*, *rule exI[of - chunks]*),

intro conjI allI impI,
 rule len-raws, rule len-chunks, rule state,
 rule fresh, rule lookup)

qed

lemma *query-rounds-raw-list-path-hit-cons-SomeE*:

assumes

query-rounds-raw-list-path-hit s (raw # raws) (Suc n)
(Some (ys, t))

obtains *chunks* **where**

length raws = n
length chunks = Suc n
PState t = state-after-query-chunks (PState s) chunks (Suc n)
query-index-list-path-fresh s (Suc n) chunks (raw # raws)
 $\bigwedge i. i < \text{Suc } n \implies$
fmlookup (HashMap t)
(QueryIndexChallenge (PQueryCounter s + i)
(state-after-query-chunks (PState s) chunks i)) =
Some ((raw # raws) ! i)

proof –

have *hit-eq*:

query-rounds-raw-list-path-hit s (raw # raws) (Suc n)
(Some (ys, t)) =
 $(\exists \text{chunks.}$
 $\text{length (raw \# raws) = Suc } n \wedge$
 $\text{length chunks = Suc } n \wedge$
 $\text{PState } t = \text{state-after-query-chunks (PState } s) \text{ chunks (Suc } n) \wedge$
 $\text{query-index-list-path-fresh } s \text{ (Suc } n) \text{ chunks (raw \# raws) } \wedge$
 $(\forall i < \text{Suc } n.$
 $\text{fmlookup (HashMap t)}$
 $\text{(QueryIndexChallenge (PQueryCounter } s + i)$
 $\text{(state-after-query-chunks (PState } s) \text{ chunks } i)) =$
 $\text{Some ((raw \# raws) ! i)))$

unfolding *query-rounds-raw-list-path-hit-def* **by** *simp*

have *ex-chunks*:

$\exists \text{chunks.}$
 $\text{length (raw \# raws) = Suc } n \wedge$
 $\text{length chunks = Suc } n \wedge$
 $\text{PState } t = \text{state-after-query-chunks (PState } s) \text{ chunks (Suc } n) \wedge$
 $\text{query-index-list-path-fresh } s \text{ (Suc } n) \text{ chunks (raw \# raws) } \wedge$
 $(\forall i < \text{Suc } n.$
 $\text{fmlookup (HashMap t)}$
 $\text{(QueryIndexChallenge (PQueryCounter } s + i)$
 $\text{(state-after-query-chunks (PState } s) \text{ chunks } i)) =$
 $\text{Some ((raw \# raws) ! i))$

using *assms* **by** (*simp only: hit-eq*)

then obtain *chunks* **where** *len-raws-full*: *length (raw # raws) = Suc n*

and *len-chunks*: *length chunks = Suc n*

and *state*: *PState t = state-after-query-chunks (PState s) chunks (Suc n)*


```

and fresh: query-index-list-path-fresh s (Suc n) chunks (raw # raws)
and lookup-all:
   $\forall i < \text{Suc } n.$ 
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) chunks i)) =
        Some ((raw # raws) ! i)
  by blast
have len-raws: length raws = n
  using len-raws-full by simp
have lookup:
   $\wedge i. i < \text{Suc } n \implies$ 
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) chunks i)) =
        Some ((raw # raws) ! i)
  using lookup-all by blast
show ?thesis
  by (rule that[OF len-raws len-chunks state fresh lookup])
qed

```

definition *query-round-any-row-set-hit*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ set} \Rightarrow$
 $(\text{unit} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
query-round-any-row-set-hit *s* *B* *out* $\longleftrightarrow$
 (*case* *out* of
None $\Rightarrow$ *False*
 | *Some* (*-, t*) $\Rightarrow$
 ($\exists \text{ raw} \in B.$
 fmlookup (*HashMap* *t*)
 (*QueryIndexChallenge* (*PQueryCounter* *s*) (*PState* *s*)) =
 Some *raw*))

lemma *query-round-any-row-set-hit-None*:
 $\neg \text{query-round-any-row-set-hit } s \ B \ \text{None}$
unfolding *query-round-any-row-set-hit-def* **by** *simp*

lemma *query-round-any-row-set-hit-NoneE*:
assumes *query-round-any-row-set-hit* *s* *B* *None*
shows *P*
using *assms* *query-round-any-row-set-hit-None* **by** *contradiction*

lemma *query-round-any-row-set-hitI*:
assumes *raw* $\in B$
and *fmlookup* (*HashMap* *t*)
 (*QueryIndexChallenge* (*PQueryCounter* *s*) (*PState* *s*)) = *Some* *raw*
shows *query-round-any-row-set-hit* *s* *B* (*Some* (*u*, *t*))
proof –

have *hit-eq*:
 $\text{query-round-any-row-set-hit } s \ B \ (\text{Some } (u, t)) =$
 $(\exists \text{ raw} \in B.$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some raw})$
unfolding *query-round-any-row-set-hit-def* **by** *simp*
show *?thesis*
by (*subst hit-eq*, *rule bexI[of - raw]*, *rule assms(2)*, *rule assms(1)*)
qed

lemma *query-round-any-row-set-hit-SomeE*:
assumes *query-round-any-row-set-hit* $s \ B \ (\text{Some } (u, t))$
obtains *raw* **where**
 $\text{raw} \in B$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some raw}$
proof –
have *hit-eq*:
 $\text{query-round-any-row-set-hit } s \ B \ (\text{Some } (u, t)) =$
 $(\exists \text{ raw} \in B.$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some raw})$
unfolding *query-round-any-row-set-hit-def* **by** *simp*
have $\exists \text{ raw} \in B.$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some raw}$
using *assms* **by** (*simp only: hit-eq*)
then show *?thesis*
by (*blast intro: that*)
qed

lemma *receive-query-index-challenge-current-lookupD*:
assumes $\text{Some } (\text{raw}, s0) \in \text{set-dist } (\text{execute receive-query-index-challenge } s)$
shows
 $\text{fmlookup } (\text{HashMap } s0)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some raw}$
proof –
have *recv-info*:
 $s \leq s0 \wedge$
 $P\text{State } s0 = P\text{State } s \wedge$
 $P\text{Transcript } s0 = P\text{Transcript } s \wedge$
 $\text{fmlookup } (\text{HashMap } s0)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some raw}$
by (*rule receive-query-index-challenge-outcome[OF assms]*)
show *?thesis*
by (*rule conjunct2[OF conjunct2[OF conjunct2[OF recv-info]]]*)
qed

lemma *verifier-query-round-after-index-program-preserves-current-lookupD*:

```

assumes after:
  Some (u, t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program fr f-fl f-final as fl
          final raw) s0)
and lookup:
  fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
shows
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof –
  let ?key = QueryIndexChallenge (PQueryCounter s) (PState s)
  have u-eq: u = ()
    by (cases u) simp
  have preserve:
    fmlookup (HashMap t) ?key = fmlookup (HashMap s0) ?key
    by (rule verifier-query-round-after-index-program-preserves-query-lookup
      [OF after[unfolded u-eq]])
  show ?thesis
  proof (rule HOL.trans)
    show fmlookup (HashMap t) ?key = fmlookup (HashMap s0) ?key
      by (rule preserve)
    show fmlookup (HashMap s0) ?key = Some raw
      by (rule lookup)
  qed
qed

lemma wp-receive-query-index-challenge-raw-set-le:
  assumes fresh:
    fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = None
  shows
    wp-event receive-query-index-challenge
      ( $\lambda out. case\ out\ of\ None \Rightarrow False \mid Some\ (raw, -) \Rightarrow raw \in B$ )
       $s \leq nnreal\ (card\ B) / nnreal\ size$ 
  proof –
    have eq:
      wp-event receive-query-index-challenge
        ( $\lambda out. case\ out\ of\ None \Rightarrow False \mid Some\ (raw, -) \Rightarrow raw \in B$ )
         $s = nnreal\ (card\ B) / nnreal\ size$ 
      by (rule wp-receive-query-index-challenge-fresh-set[OF fresh])
    show ?thesis
      by (subst eq, rule order-refl)
  qed

lemma finite-query-index-raw-list-preimage[simp]:
  finite (query-index-raw-list-preimage Q)

```

```

proof –
  have query-index-raw-list-preimage  $Q \subseteq \{xs. \text{length } xs = \text{rounds}\}$ 
    unfolding query-index-raw-list-preimage-def by auto
  then show ?thesis
    apply (rule finite-subset) by (simp add: finite-list-length)
qed

lemma finite-query-index-raw-list-position-values[simp]:
  finite (query-index-raw-list-position-values Q i)
proof –
  have query-index-raw-list-position-values Q i  $\subseteq$  UNIV
    by simp
  then show ?thesis
    by (rule finite-subset) simp
qed

lemma query-index-list-path-fresh-tail-lookup:
  assumes fresh:
    query-index-list-path-fresh s (Suc n) (chunk # chunks) (raw # raws)
  and head:
    Some  $((\ ), s1) \in$ 
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and head-chunk:
    PState s1 = foldl concat (PState s) chunk
  and head-counter:
    PQueryCounter s1 = Suc (PQueryCounter s)
  and i-bound:  $i < n$ 
  shows
    fmlookup (HashMap s1)
      (QueryIndexChallenge (PQueryCounter s1 + i)
        (state-after-query-chunks (PState s1) chunks i)) = None
proof –
  from verifier-query-round-program-alt-outcomeE[OF head]
  obtain raw-idx s0 where recv:
    Some (raw-idx, s0) \in set-dist (execute receive-query-index-challenge s)
  and after:
    Some  $((\ ), s1) \in$ 
      set-dist
        (execute
          (verifier-query-round-after-index-program fr f-fl f-final as fl
            final raw-idx s0))
    by blast
  let ?tail-key =
    QueryIndexChallenge (PQueryCounter s1 + i)
      (state-after-query-chunks (PState s1) chunks i)
  let ?full-key =
    QueryIndexChallenge (PQueryCounter s + Suc i)

```

```

    (state-after-query-chunks (PState s) (chunk # chunks) (Suc i))
  have key-eq: ?tail-key = ?full-key
    using head-chunk head-counter
    unfolding state-after-query-chunks-def
    by simp
  have suc-i-bound: Suc i < Suc n
    using i-bound by simp
  have full-none:
    fmlookup (HashMap s) ?full-key = None
    by (rule query-index-list-path-fresh-lookupD[OF fresh suc-i-bound])
  have neg:
    ?full-key ≠ QueryIndexChallenge (PQueryCounter s) (PState s)
    by simp
  have lookup-s0:
    fmlookup (HashMap s0) ?full-key = None
  proof -
    have preserve:
      fmlookup (HashMap s0) ?full-key =
        fmlookup (HashMap s) ?full-key
      by (rule receive-query-index-challenge-preserves-other-lookup
        [OF recv neg])
    show ?thesis
    proof (rule HOL.trans)
      show fmlookup (HashMap s0) ?full-key =
        fmlookup (HashMap s) ?full-key
        by (rule preserve)
      show fmlookup (HashMap s) ?full-key = None
        by (rule full-none)
    qed
  qed
  have lookup-s1:
    fmlookup (HashMap s1) ?full-key = None
  proof -
    have preserve:
      fmlookup (HashMap s1) ?full-key =
        fmlookup (HashMap s0) ?full-key
      by (rule verifier-query-round-after-index-program-preserves-query-lookup
        [OF after])
    show ?thesis
    proof (rule HOL.trans)
      show fmlookup (HashMap s1) ?full-key =
        fmlookup (HashMap s0) ?full-key
        by (rule preserve)
      show fmlookup (HashMap s0) ?full-key = None
        by (rule lookup-s0)
    qed
  qed
  show ?thesis
    unfolding key-eq by (rule lookup-s1)

```

qed

lemma *query-index-list-path-fresh-tail*:
 assumes *fresh*:
 query-index-list-path-fresh *s* (*Suc* *n*) (*chunk* # *chunks*) (*raw* # *raws*)
 and *head*:
 Some (*()*, *s1*) ∈
 set-dist
 (*execute*
 (*verifier-query-round-program* *fr* *f-fl* *f-final* *as* *fl* *final*) *s*)
 and *head-chunk*:
 PState *s1* = *foldl concat* (*PState* *s*) *chunk*
 and *head-counter*:
 PQueryCounter *s1* = *Suc* (*PQueryCounter* *s*)
 shows *query-index-list-path-fresh* *s1* *n* *chunks* *raws*
proof –
 have *len-raws*: *length* *raws* = *n*
 using *fresh* **unfolding** *query-index-list-path-fresh-def* **by** *simp*
 have *len-chunks*: *length* *chunks* = *n*
 using *fresh* **unfolding** *query-index-list-path-fresh-def* **by** *simp*
 have *lookup-none*:
 $\bigwedge i. i < n \implies$
 fmlookup (*HashMap* *s1*)
 (*QueryIndexChallenge* (*PQueryCounter* *s1* + *i*)
 (*state-after-query-chunks* (*PState* *s1*) *chunks* *i*)) = *None*
 by (*rule* *query-index-list-path-fresh-tail-lookup*
 [*OF* *fresh* *head* *head-chunk* *head-counter*])
 show *?thesis*
 unfolding *query-index-list-path-fresh-def*
 using *len-raws* *len-chunks* *lookup-none* **by** *simp*
qed

lemma *query-rounds-first-tail-key-fresh-after-head*:
 assumes *fresh*:
 query-index-list-path-fresh *s* (*Suc* *n*) *chunks* (*raw* # *raws*)
 and *head*:
 Some (*()*, *s1*) ∈
 set-dist
 (*execute*
 (*verifier-query-round-program* *fr* *f-fl* *f-final* *as* *fl* *final*) *s*)
 and *one-bound*: *1* < *Suc* *n*
 and *head-counter*: *PQueryCounter* *s1* = *Suc* (*PQueryCounter* *s*)
 shows
 fmlookup (*HashMap* *s1*)
 (*QueryIndexChallenge* (*PQueryCounter* *s1*)
 (*state-after-query-chunks* (*PState* *s*) *chunks* *1*)) = *None*
proof –
 let *?y* = *state-after-query-chunks* (*PState* *s*) *chunks* *1*
 let *?full-key* = *QueryIndexChallenge* (*PQueryCounter* *s* + *1*) *?y*

```

have full-none-s: fmlookup (HashMap s) ?full-key = None
  by (rule query-index-list-path-fresh-lookupD[OF fresh one-bound])
from verifier-query-round-program-alt-outcomeE[OF head]
obtain r s0 where recv:
  Some (r, s0) ∈ set-dist (execute receive-query-index-challenge s)
and after:
  Some (( ), s1) ∈
    set-dist
    (execute
      (verifier-query-round-after-index-program fr f-fl f-final as fl
        final r) s0)
by blast
have neq:
  ?full-key ≠ QueryIndexChallenge (PQueryCounter s) (PState s)
by simp
have s0-none:
  fmlookup (HashMap s0) ?full-key = None
proof –
  have preserve:
    fmlookup (HashMap s0) ?full-key =
      fmlookup (HashMap s) ?full-key
    by (rule receive-query-index-challenge-preserves-other-lookup
      [OF recv neq])
  show ?thesis
proof (rule HOL.trans)
  show fmlookup (HashMap s0) ?full-key =
    fmlookup (HashMap s) ?full-key
    by (rule preserve)
  show fmlookup (HashMap s) ?full-key = None
    by (rule full-none-s)
  qed
qed
have s1-none: fmlookup (HashMap s1) ?full-key = None
proof –
  have preserve:
    fmlookup (HashMap s1) ?full-key =
      fmlookup (HashMap s0) ?full-key
    by (rule verifier-query-round-after-index-program-preserves-query-lookup
      [OF after])
  show ?thesis
proof (rule HOL.trans)
  show fmlookup (HashMap s1) ?full-key =
    fmlookup (HashMap s0) ?full-key
    by (rule preserve)
  show fmlookup (HashMap s0) ?full-key = None
    by (rule s0-none)
  qed
qed
show ?thesis

```

```

using head-counter s1-none by simp
qed

lemma query-rounds-raw-list-path-hit-tail-head-state:
assumes head:
  Some ((), s1) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
and tail:
  Some (results, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s1)
and len-chunks-suc: length chunks = Suc n
and state-t:
  PState t = state-after-query-chunks (PState s) chunks (Suc n)
and fresh:
  query-index-list-path-fresh s (Suc n) chunks (raw # rows)
and lookups:
   $\bigwedge i. i < \text{Suc } n \implies$ 
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) chunks i)) =
      Some ((raw # rows) ! i)
and chunks-eq: chunks = chunk # chunks-tail
and head-counter: PQueryCounter s1 = Suc (PQueryCounter s)
shows PState s1 = state-after-query-chunks (PState s) chunks 1
proof (cases n)
case 0
then have results = [] t = s1
  using tail by simp-all
then show ?thesis
  using state-t 0 unfolding chunks-eq state-after-query-chunks-def by simp
next
case (Suc m)
from tail[unfolded Suc ntimes.simps] obtain u' s2 rest t0 where tail-head:
  Some (u', s2) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s1)
and tail-rest0:
  Some (rest, t0) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) m)

```



```

      s2)
    and ret:
      Some (results, t) ∈ set-dist (execute (return (u' # rest)) t0)
    by (auto elim!: set-dist-bindE)
  have results-eq: results = u' # rest
  by (rule set-dist-return-valueD[OF ret])
  have t-eq: t = t0
  by (rule set-dist-return-stateD[OF ret])
  have tail-rest:
    Some (rest, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) m)
          s2)
  by (subst t-eq, rule tail-rest0)
  have u'-eq: u' = ()
  by (cases u') simp
  let ?y = state-after-query-chunks (PState s) chunks 1
  let ?key = QueryIndexChallenge (PQueryCounter s1) ?y
  have full-none-s1:
    fmlookup (HashMap s1) ?key = None
  by (rule query-rounds-first-tail-key-fresh-after-head
    [OF fresh head - head-counter])
    (use Suc in simp)
  have lookup-t:
    fmlookup (HashMap t) ?key = Some (raws ! 0)
  using lookups[of 1] Suc head-counter
  unfolding chunks-eq state-after-query-chunks-def by simp
  have key-eq:
    ?key = QueryIndexChallenge (PQueryCounter s1) (PState s1)
  proof (rule ccontr)
    assume neg:
      ?key ≠ QueryIndexChallenge (PQueryCounter s1) (PState s1)
    have tail-head-unit:
      Some ((), s2) ∈
        set-dist
          (execute
            (verifier-query-round-program fr f-fl f-final as fl final) s1)
    using tail-head u'-eq by simp
  from verifier-query-round-program-alt-outcomeE[OF tail-head-unit]
  obtain r s0 where recv:
    Some (r, s0) ∈ set-dist (execute receive-query-index-challenge s1)
  and after:
    Some ((), s2) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program fr f-fl f-final as fl
            final r) s0)

```

```

    by blast
  have s0-none:
    fmlookup (HashMap s0) ?key = None
    using receive-query-index-challenge-preserves-other-lookup[OF recv neq]
    full-none-s1
    by simp
  have s2-none: fmlookup (HashMap s2) ?key = None
  proof -
    have preserve:
      fmlookup (HashMap s2) ?key = fmlookup (HashMap s0) ?key
      by (rule verifier-query-round-after-index-program-preserves-query-lookup
        [OF after])
    show ?thesis
    proof (rule HOL.trans)
      show fmlookup (HashMap s2) ?key = fmlookup (HashMap s0) ?key
        by (rule preserve)
      show fmlookup (HashMap s0) ?key = None
        by (rule s0-none)
    qed
  qed
  from verifier-query-round-program-outcome[OF tail-head[unfolded u'-eq]]
  obtain r2::'f and idx2::nat and chunk2::'f list where
    counter-s2: PQueryCounter s2 = Suc (PQueryCounter s1)
    by blast
  have lookup-pres:
    fmlookup (HashMap t) ?key = fmlookup (HashMap s2) ?key
    by (rule ntimes-verifier-query-round-program-preserves-past-query-lookup
      [OF - tail-rest])
    (use counter-s2 in simp)
  show False
    using lookup-t s2-none lookup-pres by simp
  qed
  then show ?thesis
    by simp
  qed

end

context soundness
begin

lemma query-rounds-raw-list-path-hit-tail:
  assumes head:
    Some ((), s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results, t) ∈

```

```

    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s1)
  and hit:
    query-rounds-raw-list-path-hit s (raw # rows) (Suc n)
      (Some (u # results, t))
  shows query-rounds-raw-list-path-hit s1 rows n (Some (results, t))
proof -
  from query-rounds-raw-list-path-hit-cons-SomeE[OF hit] obtain chunks where
    len-rows: length rows = n
  and len-chunks-suc: length chunks = Suc n
  and state-t:
    PState t = state-after-query-chunks (PState s) chunks (Suc n)
  and fresh:
    query-index-list-path-fresh s (Suc n) chunks (raw # rows)
  and lookups:
     $\bigwedge i. i < \text{Suc } n \implies$ 
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) chunks i)) =
    Some ((raw # rows) ! i)
  by blast
  obtain chunk chunks-tail where chunks-eq: chunks = chunk # chunks-tail
  using len-chunks-suc by (cases chunks) auto
  have len-chunks-tail: length chunks-tail = n
  using len-chunks-suc unfolding chunks-eq by simp
  from verifier-query-round-program-outcome[OF head]
  obtain raw0::'f and idx::nat and chunk0:: 'f list where
    head-counter: PQueryCounter s1 = Suc (PQueryCounter s)
  by blast
  have head-state:
    PState s1 = state-after-query-chunks (PState s) chunks 1
  by (rule query-rounds-raw-list-path-hit-tail-head-state
    [OF head tail len-chunks-suc state-t fresh lookups chunks-eq
      head-counter])
  have fresh-tail:
    query-index-list-path-fresh s1 n chunks-tail rows
proof -
  have fresh-chunks:
    query-index-list-path-fresh s (Suc n) (chunk # chunks-tail)
      (raw # rows)
  by (subst chunks-eq[symmetric], rule fresh)
  have head-chunk: PState s1 = foldl concat (PState s) chunk
proof -
  have head-state-cons:
    PState s1 =
      state-after-query-chunks (PState s) (chunk # chunks-tail) 1

```

```

    by (subst chunks-eq[symmetric], rule head-state)
  show ?thesis
    by (subst state-after-query-chunks-one-cons[symmetric],
        rule head-state-cons)
qed
show ?thesis
  by (rule query-index-list-path-fresh-tail
      [OF fresh-chunks head head-chunk head-counter])
qed
have state-tail:
  PState t =
    state-after-query-chunks (PState s1) chunks-tail n
proof -
  have shift:
    state-after-query-chunks (PState s1) chunks-tail n =
      state-after-query-chunks (PState s) (chunk # chunks-tail) (Suc n)
  by (rule state-after-query-chunks-cons-shift)
    (use head-state chunks-eq in simp)
  show ?thesis
    by (subst shift, subst chunks-eq[symmetric], rule state-t)
qed
have lookup-tail:
   $\bigwedge i. i < n \implies$ 
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s1 + i)
      (state-after-query-chunks (PState s1) chunks-tail i)) =
    Some (raws ! i)
proof -
  fix i
  assume i-bound:  $i < n$ 
  have key-eq:
    QueryIndexChallenge (PQueryCounter s1 + i)
      (state-after-query-chunks (PState s1) chunks-tail i) =
    QueryIndexChallenge (PQueryCounter s + Suc i)
      (state-after-query-chunks (PState s) chunks (Suc i))
  proof -
    have st-shift:
      state-after-query-chunks (PState s1) chunks-tail i =
        state-after-query-chunks (PState s) (chunk # chunks-tail) (Suc i)
    by (rule state-after-query-chunks-cons-shift-index)
      (use head-state chunks-eq in simp)
    show ?thesis
      using head-counter st-shift unfolding chunks-eq by simp
  qed
  show fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s1 + i)
      (state-after-query-chunks (PState s1) chunks-tail i)) =
    Some (raws ! i)
  using lookups[of Suc i] i-bound key-eq by simp

```

```

qed
show ?thesis
  by (rule query-rounds-raw-list-path-hitI
      [OF len-raws len-chunks-tail state-tail fresh-tail lookup-tail])
qed

end

context soundness
begin

lemma query-rounds-raw-list-path-hit-head:
  assumes head:
    Some ((), s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) n)
          s1)
  and hit:
    query-rounds-raw-list-path-hit s (raw # raws) (Suc n)
    (Some (u # results, t))
  shows query-round-any-raw-set-hit s {raw} (Some ((), s1))
proof -
  have lookup-t:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  proof -
    from query-rounds-raw-list-path-hit-cons-SomeE[OF hit]
    obtain chunks where lookup:
       $\bigwedge i. i < \text{Suc } n \implies$ 
      fmlookup (HashMap t)
        (QueryIndexChallenge (PQueryCounter s + i)
          (state-after-query-chunks (PState s) chunks i)) =
        Some ((raw # raws) ! i)
    by blast
  have lookup0:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + 0)
        (state-after-query-chunks (PState s) chunks 0)) =
    Some ((raw # raws) ! 0)
  using lookup[of 0] by simp
  show ?thesis
    using lookup0 unfolding state-after-query-chunks-def by simp

```

qed
from *verifier-query-round-program-alt-outcomeE*[*OF head*]
obtain *raw0 s0* **where**
recv:
Some (raw0, s0) ∈ set-dist (execute receive-query-index-challenge s)
and after:
Some ((), s1) ∈
set-dist
(execute
(verifier-query-round-after-index-program fr f-fl f-final as fl
final raw0) s0)
by *blast*
have *counter-s1: PQueryCounter s1 = Suc (PQueryCounter s)*
by (*rule verifier-query-round-parts-counter*[*OF recv after*])
have *past: PQueryCounter s < PQueryCounter s1*
by (*rule eq-Suc-imp-less-nat*[*OF counter-s1*])
have *lookup-s1:*
fmlookup (HashMap s1)
(QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof –
let *?key = QueryIndexChallenge (PQueryCounter s) (PState s)*
have *preserve:*
fmlookup (HashMap t) ?key = fmlookup (HashMap s1) ?key
by (*rule ntimes-verifier-query-round-program-preserves-past-query-lookup*
[OF past tail])
show *?thesis*
proof (*rule HOL.trans*)
show *fmlookup (HashMap s1) ?key = fmlookup (HashMap t) ?key*
by (*rule HOL.sym*[*OF preserve*])
show *fmlookup (HashMap t) ?key = Some raw*
by (*rule lookup-t*)
qed
qed
show *?thesis*
by (*rule query-round-any-raw-set-hitI*[*OF singletonI lookup-s1*])
qed

lemma *verifier-query-round-program-raw-set-hit-imp-raw-in-Some:*
assumes *recv:*
Some (raw, s0) ∈ set-dist (execute receive-query-index-challenge s)
and after:
Some (u, t) ∈
set-dist
(execute
(verifier-query-round-after-index-program fr f-fl f-final as fl
final raw) s0)
and *hit: query-round-any-raw-set-hit s B (Some (u, t))*
shows *raw ∈ B*
proof –

```

from query-round-any-row-set-hit-SomeE[OF hit]
obtain raw' where
  raw'-in: raw' ∈ B
  and lookup-t:
    fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw'
  by blast
have lookup-s0:
  fmlookup (HashMap s0)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  by (rule receive-query-index-challenge-current-lookupD[OF recv])
have lookup-t-current:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  by (rule verifier-query-round-after-index-program-preserves-current-lookupD
    [OF after lookup-s0])
have some-eq: Some raw = Some raw'
  using lookup-t-current lookup-t by simp
have raw-eq: raw = raw'
  using some-eq by simp
show ?thesis
  by (subst raw-eq, rule raw'-in)
qed

```

```

lemma verifier-query-round-program-row-set-hit-imp-row-in:
assumes recv:
  Some (raw, s0) ∈ set-dist (execute receive-query-index-challenge s)
and after:
  out ∈
    set-dist
    (execute
      (verifier-query-round-after-index-program fr f-fl f-final as fl
        final raw) s0)
  and hit: query-round-any-row-set-hit s B out
shows raw ∈ B
proof –
have rhs:
  (case out of
    None ⇒ False
    | Some (·, t) ⇒
      (∃ raw' ∈ B.
        fmlookup (HashMap t)
        (QueryIndexChallenge (PQueryCounter s) (PState s)) =
          Some raw'))
  using hit by (simp only: query-round-any-row-set-hit-def)
obtain u t where out-eq: out = Some (u, t)
proof (cases out)
case None
have False

```

```

    using rhs None by simp
  then show ?thesis
    by (rule FalseE)
next
case (Some pair)
then obtain a b where out = Some (a, b)
  by (cases pair) auto
then show ?thesis
  by (rule that)
qed
have after-some:
  Some (u, t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program fr f-fl f-final as fl
          final raw) s0)
  by (subst out-eq[symmetric]) (rule after)
have hit-some: query-round-any-row-set-hit s B (Some (u, t))
  by (subst out-eq[symmetric]) (rule hit)
show ?thesis
  by (rule verifier-query-round-program-row-set-hit-imp-row-in-Some
    [OF recv after-some hit-some])
qed

lemma wp-verifier-query-round-program-any-row-set-bound:
  assumes fresh:
    fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = None
  shows
    wp-event (verifier-query-round-program fr f-fl f-final as fl final)
      (query-round-any-row-set-hit s B) s ≤
      nnreal (card B) / nnreal size
  unfolding verifier-query-round-program-alt-def
proof (rule wp-event-bind-bound-by-head-event)
  show wp-event receive-query-index-challenge
    (λout. case out of None ⇒ False | Some (raw, -) ⇒ raw ∈ B)
    s ≤ nnreal (card B) / nnreal size
  by (rule wp-receive-query-index-challenge-row-set-le[OF fresh])
next
  show query-round-any-row-set-hit s B None ⇒
    (case None of None ⇒ False | Some (raw, -) ⇒ raw ∈ B)
  by (rule query-round-any-row-set-hit-NoneE)
next
  fix raw s0 out
  assume recv:
    Some (raw, s0) ∈ set-dist (execute receive-query-index-challenge s)
  and out:
    out ∈
      set-dist

```



```

      (execute
        (verifier-query-round-after-index-program fr f-fl f-final as fl
          final raw) s0)
    and hit: query-round-any-row-set-hit s B out
  show (case Some (raw, s0) of None  $\Rightarrow$  False | Some (raw, -)  $\Rightarrow$  raw  $\in$  B)
  proof -
    have raw-in: raw  $\in$  B
    by (rule verifier-query-round-program-row-set-hit-imp-row-in
      [where s=s and B=B and raw=raw and s0=s0 and out=out
        and fr=fr and f-fl=f-fl and f-final=f-final and as=as
        and fl=fl and final=final, OF recv out hit])
    then show ?thesis
      by simp
  qed
qed

```

```

lemma query-rounds-row-list-path-hit-SomeE:
  assumes query-rounds-row-list-path-hit s rows n out
  obtains results t where out = Some (results, t)
  proof -
    show ?thesis
    proof (cases out)
      case None
      have none-eq: query-rounds-row-list-path-hit s rows n None = False
        unfolding query-rounds-row-list-path-hit-def by simp
      have hit-none: query-rounds-row-list-path-hit s rows n None
        by (subst None[symmetric], rule assms)
      then show ?thesis
        using hit-none by (simp only: none-eq)
    next
      case (Some pair)
      then obtain results t where out = Some (results, t)
        by (cases pair) auto
      then show ?thesis
        by (rule that)
    qed
  qed

```

```

lemma query-rounds-row-list-path-hit-some-fresh-currentD:
  assumes hit:
    query-rounds-row-list-path-hit s (raw # rows-tail) (Suc n)
    (Some (ys, t))
  shows
    fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = None
  proof -
    have some-eq:
      query-rounds-row-list-path-hit s (raw # rows-tail) (Suc n)
      (Some (ys, t)) =

```

$(\exists \text{ chunks.}$
 $\text{length } (\text{raw} \# \text{ rows-tail}) = \text{Suc } n \wedge$
 $\text{length } \text{chunks} = \text{Suc } n \wedge$
 $P\text{State } t = \text{state-after-query-chunks } (P\text{State } s) \text{ chunks } (\text{Suc } n) \wedge$
 $\text{query-index-list-path-fresh } s (\text{Suc } n) \text{ chunks } (\text{raw} \# \text{ rows-tail}) \wedge$
 $(\forall i < \text{Suc } n.$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s + i)$
 $(\text{state-after-query-chunks } (P\text{State } s) \text{ chunks } i)) =$
 $\text{Some } ((\text{raw} \# \text{ rows-tail}) ! i)))$
unfolding *query-rounds-row-list-path-hit-def* **by** *simp*
have *ex-chunks*:
 $\exists \text{ chunks.}$
 $\text{length } (\text{raw} \# \text{ rows-tail}) = \text{Suc } n \wedge$
 $\text{length } \text{chunks} = \text{Suc } n \wedge$
 $P\text{State } t = \text{state-after-query-chunks } (P\text{State } s) \text{ chunks } (\text{Suc } n) \wedge$
 $\text{query-index-list-path-fresh } s (\text{Suc } n) \text{ chunks } (\text{raw} \# \text{ rows-tail}) \wedge$
 $(\forall i < \text{Suc } n.$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s + i)$
 $(\text{state-after-query-chunks } (P\text{State } s) \text{ chunks } i)) =$
 $\text{Some } ((\text{raw} \# \text{ rows-tail}) ! i)))$
using *hit* **by** (*simp only: some-eq*)
then obtain *chunks* **where** *fresh-path*:
 $\text{query-index-list-path-fresh } s (\text{Suc } n) \text{ chunks } (\text{raw} \# \text{ rows-tail})$
by (*elim exE conjE*)
show *?thesis*
by (*rule query-index-list-path-fresh-head-fresh[OF fresh-path]*)
qed

lemma *query-rounds-row-list-path-hit-some-head-lookupD*:
assumes *hit*:
 $\text{query-rounds-row-list-path-hit } s (\text{raw} \# \text{ rows}) (\text{Suc } n)$
 $(\text{Some } (ys, t))$
shows
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some } \text{raw}$
proof –
have *some-eq*:
 $\text{query-rounds-row-list-path-hit } s (\text{raw} \# \text{ rows}) (\text{Suc } n)$
 $(\text{Some } (ys, t)) =$
 $(\exists \text{ chunks.}$
 $\text{length } (\text{raw} \# \text{ rows}) = \text{Suc } n \wedge$
 $\text{length } \text{chunks} = \text{Suc } n \wedge$
 $P\text{State } t = \text{state-after-query-chunks } (P\text{State } s) \text{ chunks } (\text{Suc } n) \wedge$
 $\text{query-index-list-path-fresh } s (\text{Suc } n) \text{ chunks } (\text{raw} \# \text{ rows}) \wedge$
 $(\forall i < \text{Suc } n.$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s + i)$

```

      (state-after-query-chunks (PState s) chunks i)) =
      Some ((raw # rows) ! i)))
unfolding query-rounds-raw-list-path-hit-def by simp
have ex-chunks:
   $\exists$  chunks.
    length (raw # rows) = Suc n  $\wedge$ 
    length chunks = Suc n  $\wedge$ 
    PState t = state-after-query-chunks (PState s) chunks (Suc n)  $\wedge$ 
    query-index-list-path-fresh s (Suc n) chunks (raw # rows)  $\wedge$ 
    ( $\forall i < \text{Suc } n.$ 
      fmllookup (HashMap t)
        (QueryIndexChallenge (PQueryCounter s + i)
          (state-after-query-chunks (PState s) chunks i)) =
          Some ((raw # rows) ! i))
    using hit by (simp only: some-eq)
then obtain chunks where lookup-all:
   $\forall i < \text{Suc } n.$ 
    fmllookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) chunks i)) =
      Some ((raw # rows) ! i)
    by (elim exE conjE)
have lookup0:
  fmllookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + 0)
      (state-after-query-chunks (PState s) chunks 0)) =
    Some ((raw # rows) ! 0)
proof –
  have 0 < Suc n
  by simp
  from lookup-all this show ?thesis
  by blast
qed
show ?thesis
  using lookup0 unfolding state-after-query-chunks-def by simp
qed

lemma query-rounds-raw-list-path-hit-bind-headD:
assumes head:
  Some (u, s1)  $\in$ 
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
and support:
  out  $\in$ 
    set-dist
      (execute
        (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n
           $\gg$  ( $\lambda xs.$  return (u # xs)))) s1)

```

```

and hit:
  query-rounds-raw-list-path-hit s (raw # raws-tail) (Suc n) out
shows query-round-any-raw-set-hit s {raw} (Some (u, s1))
proof –
  let ?tail =
    ntimes (verifier-query-round-program fr f-fl f-final as fl final) n
from hit obtain ys t0 where out-some: out = Some (ys, t0)
  by (rule query-rounds-raw-list-path-hit-SomeE)
from support out-some obtain results t where tail:
  Some (results, t) ∈ set-dist (execute ?tail s1)
  and out-eq: out = Some (u # results, t)
  by (elim set-dist-bind-return-consE)
have u-eq: u = ()
  by (cases u) simp
have head-unit:
  Some ((), s1) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
  using head u-eq by simp
have hit-head-out:
  query-rounds-raw-list-path-hit s (raw # raws-tail) (Suc n)
    (Some (u # results, t))
  by (subst out-eq[symmetric], rule hit)
have query-round-any-raw-set-hit s {raw} (Some ((), s1))
  by (rule query-rounds-raw-list-path-hit-head
    [OF head-unit tail hit-head-out])
then show ?thesis
  using u-eq by simp
qed

lemma query-rounds-raw-list-path-hit-cont-zero-if-not-head:
assumes head:
  Some (u, s1) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
and not-head:
  ¬ (fmlookup (HashMap s)
    (QueryIndexChallenge (PQueryCounter s) (PState s))) = None ∧
    query-round-any-raw-set-hit s {raw} (Some (u, s1)))
shows
  wp-event
    (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n
      ≫ (λxs. return (u # xs)))
    (query-rounds-raw-list-path-hit s (raw # raws-tail) (Suc n))
    s1 = 0
proof (rule ccontr)
  let ?tail =

```

```

    ntimes (verifier-query-round-program fr f-fl f-final as fl final) n
assume nonzero:
    wp-event (?tail  $\gg$  ( $\lambda xs.$  return ( $u \# xs$ )))
      (query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n))
      s1  $\neq$  0
from wp-event-nonzero-imp-exists-support[OF nonzero]
obtain out where out-support:
    out  $\in$  set-dist (execute (?tail  $\gg$  ( $\lambda xs.$  return ( $u \# xs$ ))) s1)
and hit:
    query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n) out
by blast
from hit obtain ys t0 where out-some: out = Some (ys, t0)
by (rule query-rounds-raw-list-path-hit-SomeE)
have hit-some:
    query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n)
      (Some (ys, t0))
by (subst out-some[symmetric], rule hit)
have fresh-current:
    fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = None
by (rule query-rounds-raw-list-path-hit-some-fresh-currentD[OF hit-some])
have raw-head:
    query-round-any-raw-set-hit s {raw} (Some (u, s1))
by (rule query-rounds-raw-list-path-hit-bind-headD[OF head out-support hit])
show False
using not-head fresh-current raw-head by blast
qed

lemma query-rounds-raw-list-path-hit-cont-le-tail:
assumes head:
    Some (u, s1)  $\in$ 
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
shows
    wp-event
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n
         $\gg$  ( $\lambda xs.$  return ( $u \# xs$ )))
      (query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n))
      s1
     $\leq$  wp-event
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n)
      (query-rounds-raw-list-path-hit s1 rows-tail n) s1
proof –
let ?tail =
    ntimes (verifier-query-round-program fr f-fl f-final as fl final) n
have u-eq: u = ()
by (cases u) simp
show ?thesis

```

```

proof (rule wp-event-bind-bound-by-head-event)
  show wp-event ?tail
    (query-rounds-raw-list-path-hit s1 rows-tail n) s1
  ≤ wp-event ?tail
    (query-rounds-raw-list-path-hit s1 rows-tail n) s1
  by simp
next
  show
    query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n) None
    ⇒ query-rounds-raw-list-path-hit s1 rows-tail n None
  unfolding query-rounds-raw-list-path-hit-def by simp
next
  fix results t out
  assume tail:
    Some (results, t) ∈ set-dist (execute ?tail s1)
  and ret: out ∈ set-dist (execute (return (u # results)) t)
  and hit:
    query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n) out
from ret obtain out-eq:
  out = Some (u # results, t)
  unfolding set-dist-def return.rep-eq dist-return-def
    dist-delta-dist delta-map-def
  by simp
  show query-rounds-raw-list-path-hit s1 rows-tail n
    (Some (results, t))
  by (rule query-rounds-raw-list-path-hit-tail
    [OF head[unfolded u-eq] tail])
    (use hit out-eq u-eq in simp)
qed
qed

lemma wp-ntimes-verifier-query-round-program-raw-list-path-hit-bound:
  assumes len-rows: length rows = n
  shows
    wp-event
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n)
      (query-rounds-raw-list-path-hit s rows n) s ≤
      (1 / nnreal size) ^ n
  using len-rows
proof (induction n arbitrary: s rows)
  case 0
  then show ?case
    unfolding query-rounds-raw-list-path-hit-def
      query-index-list-path-fresh-def state-after-query-chunks-def
    by (simp add: wp-event-def wpsimps)
next
  case (Suc n)
  obtain raw rows-tail where rows-eq: rows = raw # rows-tail
  by (rule list-length-SucE[OF Suc.premss])

```

```

have len-tail: length raws-tail = n
  using Suc.premis unfolding raws-eq by simp
let ?prog = verifier-query-round-program fr f-fl f-final as fl final
let ?tail =
  ntimes ?prog n
let ?Q = query-rounds-raw-list-path-hit s raws (Suc n)
let ?fresh =
  fmlookup (HashMap s)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = None
let ?Head =
  λout. ?fresh ∧ query-round-any-raw-set-hit s {raw} out
have head-bound:
  wp-event ?prog ?Head s ≤ 1 / nnreal size
proof (cases ?fresh)
case True
have mono:
  wp-event ?prog ?Head s ≤
    wp-event ?prog (query-round-any-raw-set-hit s {raw}) s
  by (rule wp-event-mono) simp
also have ... ≤ nnreal (card {raw}) / nnreal size
  by (rule wp-verifier-query-round-program-any-raw-set-bound[OF True])
also have ... = 1 / nnreal size
  by simp
finally show ?thesis .
next
case False
have impossible: ?Head = (λ-. False)
  using False by auto
have zero: wp-event ?prog ?Head s = 0
  by (simp only: impossible wp-event-false)
then show ?thesis
  by simp
qed
show ?case
  unfolding ntimes.simps raws-eq
proof (rule order-trans)
have none-Q:
  ¬ query-rounds-raw-list-path-hit s (raw # raws-tail) (Suc n) None
  by (simp add: query-rounds-raw-list-path-hit-def)
show
  wp-event
    (?prog ≫ (λu. ?tail ≫ (λxs. return (u # xs))))
    (query-rounds-raw-list-path-hit s (raw # raws-tail) (Suc n)) s
  ≤ wp-event ?prog ?Head s * ((1 / nnreal size) ^ n)
proof (rule wp-event-bind-bound-by-head-and-cont
  [where m = ?prog
    and k = λu. ?tail ≫ (λxs. return (u # xs))
    and Q = query-rounds-raw-list-path-hit s (raw # raws-tail) (Suc n)
    and P = ?Head

```

```

    and  $D = (1 / \text{nnreal size}) \wedge n$ ,
    OF none-Q])
  fix u s1
  assume head:
    Some (u, s1) ∈ set-dist (execute ?prog s)
  and not-head:  $\neg ?\text{Head} (\text{Some } (u, s1))$ 
  show
    wp-event (?tail  $\gg (\lambda xs. \text{return } (u \# xs))$ )
    (query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n))
    s1 = 0
  by (rule query-rounds-raw-list-path-hit-cont-zero-if-not-head
    [OF head not-head])
next
  fix u s1
  assume head:
    Some (u, s1) ∈ set-dist (execute ?prog s)
  and head-hit:  $?\text{Head} (\text{Some } (u, s1))$ 
  have tail-bound:
    wp-event ?tail
    (query-rounds-raw-list-path-hit s1 rows-tail n) s1 ≤
    (1 / nnreal size)  $\wedge n$ 
  by (rule Suc.IH[OF len-tail])
  show
    wp-event (?tail  $\gg (\lambda xs. \text{return } (u \# xs))$ )
    (query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n))
    s1 ≤ (1 / nnreal size)  $\wedge n$ 
  proof (rule order-trans[OF - tail-bound])
    show
      wp-event (?tail  $\gg (\lambda xs. \text{return } (u \# xs))$ )
      (query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n))
      s1
      ≤ wp-event ?tail
      (query-rounds-raw-list-path-hit s1 rows-tail n) s1
    proof (rule wp-event-bind-return-cons-mono
      [where m = ?tail
        and Q =
          query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n)
        and R = query-rounds-raw-list-path-hit s1 rows-tail n
        and u = u])
      show
        query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n)
        None  $\implies$ 
        query-rounds-raw-list-path-hit s1 rows-tail n None
      unfolding query-rounds-raw-list-path-hit-def by simp
    next
      fix xs t
      assume tail:
        Some (xs, t) ∈ set-dist (execute ?tail s1)
      and hit:

```



```

      query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n)
      (Some (u # xs, t))
    have u-eq: u = ()
    by (cases u) simp
  show query-rounds-raw-list-path-hit s1 rows-tail n
    (Some (xs, t))
    by (rule query-rounds-raw-list-path-hit-tail
        [OF head[unfolded u-eq] tail])
        (use hit u-eq in simp)
  qed
  qed
  qed
next
  show wp-event ?prog ?Head s * (1 / nnreal size) ^ n
    ≤ (1 / nnreal size) ^ Suc n
    using head-bound by (simp add: mult-right-mono)
  qed
qed

lemma wp-ntimes-verifier-query-round-program-raw-list-preimage-path-hit-bound:
  shows
    wp-event
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      (λout. ∃ rows ∈ query-index-raw-list-preimage Q.
        query-rounds-raw-list-path-hit s rows rounds out)
      s ≤
      nnreal (card (query-index-raw-list-preimage Q)) *
      (1 / nnreal size) ^ rounds
  proof -
    have finite-rows: finite (query-index-raw-list-preimage Q)
    by (rule finite-query-index-raw-list-preimage)
    have
      wp-event
        (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
        (λout. ∃ rows ∈ query-index-raw-list-preimage Q.
          query-rounds-raw-list-path-hit s rows rounds out)
        s ≤
        (∑ rows ∈ query-index-raw-list-preimage Q.
          (1 / nnreal size) ^ rounds)
    by (rule wp-event-finite-UN-bound[OF finite-rows])
        (rule wp-ntimes-verifier-query-round-program-raw-list-path-hit-bound,
          simp add: query-index-raw-list-preimage-def)
    also have ... =
      nnreal (card (query-index-raw-list-preimage Q)) *
      (1 / nnreal size) ^ rounds
    using finite-rows by simp
  finally show ?thesis .

```

qed

lemma *checked-staged-query-index-list-relation-fiber-card-bound:*

$\text{card } \{y. \text{checked-staged-query-index-list-relation } A \ Q \ x \ y\}$
 $\leq \text{query-index-raw-list-relation-fiber-bound } Q$

proof –

let $?values =$

$(\bigcup i < \text{rounds}. \text{query-index-raw-list-position-values } Q \ i)$

have *subset:*

$\{y. \text{checked-staged-query-index-list-relation } A \ Q \ x \ y\} \subseteq ?values$

unfolding *checked-staged-query-index-list-relation-def*

query-index-raw-list-position-values-def

by *blast*

have *finite-values: finite ?values*

by *simp*

have $\text{card } \{y. \text{checked-staged-query-index-list-relation } A \ Q \ x \ y\}$

$\leq \text{card } ?values$

by $(\text{rule } \text{card-mono}[OF \ \text{finite-values } \text{subset}])$

also have $\dots \leq$

$(\sum i < \text{rounds}. \text{card } (\text{query-index-raw-list-position-values } Q \ i))$

apply $(\text{rule } \text{card-UN-le})$ **by** *simp*

also have $\dots = \text{query-index-raw-list-relation-fiber-bound } Q$

unfolding *query-index-raw-list-relation-fiber-bound-def* **by** *simp*

finally show *?thesis* .

qed

lemma *hash-relation-program-checked-staged-transcript-query-index-list:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

hash-relation-program

$(\text{checked-staged-query-index-list-relation } A \ Q)$

$(\text{query-index-raw-list-relation-fiber-bound } Q)$

$(\text{staged-attacker-query-budget } \text{budgets} + \text{staged-challenge-query-budget})$

$(\text{checked-staged-transcript-program } A)$

by $(\text{rule } \text{hash-relation-program-checked-staged-transcript-program}$

$[OF \ wf \ \text{controlled}])$

$(\text{rule } \text{checked-staged-query-index-list-relation-fiber-card-bound})$

lemma *staged-transcript-query-index-list-set-hit-imp-relation-hit:*

assumes *support:*

$\text{Some } (data, \text{attacker-state}) \in$

set-dist

$(\text{execute } (\text{checked-staged-transcript-program } A)$

$\text{adversary-initial-state})$

and *hit:*

$\text{staged-transcript-query-index-list-set-hit } Q$

$(\text{Some } (data, \text{attacker-state}))$

shows

```

    hash-relation-hit
      (checked-staged-query-index-list-relation A Q)
      adversary-initial-state attacker-state
  proof -
  from hit obtain raw-idxs query-idxs where
    len-raw: length raw-idxs = rounds
  and query-in: query-idxs ∈ Q
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookup:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap attacker-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)
  unfolding staged-transcript-query-index-list-set-hit-def by auto
  have raw-in: raw-idxs ∈ query-index-raw-list-preimage Q
  unfolding query-index-raw-list-preimage-def
  using len-raw query-in query-idxs-eq by blast
  let ?x =
    QueryIndexChallenge 0
    (state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) 0)
  have zero-bound: 0 < rounds
  by (rule rounds-positive)
  have initial-none: fmllookup (HashMap adversary-initial-state) ?x = None
  by (simp add: adversary-initial-state-def)
  have final-lookup:
    fmllookup (HashMap attacker-state) ?x = Some (raw-idxs ! 0)
  using lookup[OF zero-bound] by simp
  have rel:
    checked-staged-query-index-list-relation A Q ?x (raw-idxs ! 0)
  unfolding checked-staged-query-index-list-relation-def
  by (intro exI[of - data] exI[of - attacker-state]
    exI[of - raw-idxs] exI[of - 0] conjI)
    (use support raw-in zero-bound in simp-all)
  show ?thesis
  unfolding hash-relation-hit-def
  by (intro exI[of - ?x] exI[of - raw-idxs ! 0] conjI)
    (use initial-none final-lookup rel in simp-all)
qed

```

```

lemma staged-transcript-query-index-list-set-hit-imp-relation-hit-event:
  assumes support:
    out ∈
      set-dist
      (execute (checked-staged-transcript-program A)

```

```

      adversary-initial-state)
    and hit: staged-transcript-query-index-list-set-hit Q out
  shows
    hash-relation-hit-event
      (checked-staged-query-index-list-relation A Q)
      adversary-initial-state out
  proof (cases out)
  case None
  then show ?thesis
    using hit unfolding staged-transcript-query-index-list-set-hit-def
      hash-relation-hit-event-def
    by simp
  next
  case (Some result)
  then obtain data attacker-state where out-eq:
    out = Some (data, attacker-state)
  by (cases result) simp
  have support-some:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  using support by (simp only: out-eq)
  have hit-some:
    staged-transcript-query-index-list-set-hit Q
      (Some (data, attacker-state))
  using hit by (simp only: out-eq)
  have rel-hit:
    hash-relation-hit
      (checked-staged-query-index-list-relation A Q)
      adversary-initial-state attacker-state
  by (rule staged-transcript-query-index-list-set-hit-imp-relation-hit
    [OF support-some hit-some])
  show ?thesis
    unfolding out-eq hash-relation-hit-event-def
    using rel-hit by simp
qed

```

lemma *staged-transcript-query-index-list-set-hit-bound:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
wp-event (checked-staged-transcript-program A)
(staged-transcript-query-index-list-set-hit Q)
adversary-initial-state ≤
hash-relation-budget-value
(query-index-raw-list-relation-fiber-bound Q)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)

```

proof –
  have event-mono:
    wp-event (checked-staged-transcript-program A)
      (staged-transcript-query-index-list-set-hit Q)
      adversary-initial-state ≤
    wp-event (checked-staged-transcript-program A)
      (hash-relation-hit-event
        (checked-staged-query-index-list-relation A Q)
        adversary-initial-state)
      adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
    and hit: staged-transcript-query-index-list-set-hit Q out
  show hash-relation-hit-event
    (checked-staged-query-index-list-relation A Q)
    adversary-initial-state out
  by (rule staged-transcript-query-index-list-set-hit-imp-relation-hit-event
    [OF support hit])
  qed
  have relation-bound:
    wp-event (checked-staged-transcript-program A)
      (hash-relation-hit-event
        (checked-staged-query-index-list-relation A Q)
        adversary-initial-state)
      adversary-initial-state ≤
      hash-relation-budget-value
        (query-index-raw-list-relation-fiber-bound Q)
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget)
  proof –
  have program:
    hash-relation-program
      (checked-staged-query-index-list-relation A Q)
      (query-index-raw-list-relation-fiber-bound Q)
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
      (checked-staged-transcript-program A)
  by (rule hash-relation-program-checked-staged-transcript-query-index-list
    [OF wf controlled])
  then show ?thesis
    unfolding hash-relation-program-def hash-relation-budget-def by blast
  qed
  show ?thesis
  by (rule order-trans[OF event-mono relation-bound])
  qed

```

lemma *checked-staged-security-with-data-query-index-list-set-bound-from-transcript*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *transcript-bound*:

wp-event (checked-staged-transcript-program A)
(staged-transcript-query-index-list-set-hit Q)
adversary-initial-state ≤ C

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-query-index-list-set-hit Q)
adversary-initial-state ≤ C

unfolding *checked-staged-security-experiment-with-data-state-def*

proof (*rule wp-event-bind-bound-by-head-event[OF transcript-bound]*)

show *staged-security-with-data-query-index-list-set-hit Q None ⇒*
staged-transcript-query-index-list-set-hit Q None

unfolding *staged-security-with-data-query-index-list-set-hit-def*
staged-transcript-query-index-list-set-hit-def

by *simp*

next

fix *data attacker-state out*

assume *builder*:

Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)

and *cont*:

out ∈
set-dist
(execute
(get ≫
(λs. put
(verifier-state-from-adversary s
(staged-proof-transcript data)) ≫
(λ-.
verify-monad ≫
(λresult. return ((data, s), result))))
attacker-state)

and *hit*: *staged-security-with-data-query-index-list-set-hit Q out*

from *checked-staged-transcript-program-outcome-query-lookups*
[OF wf controlled builder]

obtain *attacker-raw-idxs attacker-query-idxs where*
len-attacker-raw: *length attacker-raw-idxs = rounds*

and *attacker-query-idxs-eq*:

attacker-query-idxs =
map (λraw. index (to-nat raw)) attacker-raw-idxs

and *lookup-attacker*:

∀ j < rounds.
fmlookup (HashMap attacker-state)

```

    (QueryIndexChallenge j
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) j)) =
    Some (attacker-raw-idxs ! j)
  by blast
from hit obtain data' attacker-state' result final-state raw-idxs query-idxs where
  out-eq: out = Some (((data', attacker-state'), result), final-state))
and len-raw: length raw-idxs = rounds
and query-in: query-idxs ∈ Q
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup-final:
  ∧ i. i < rounds ⇒
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data')
          (staged-query-chunks data') i)) =
      Some (raw-idxs ! i)
unfolding staged-security-with-data-query-index-list-set-hit-def
by (auto split: option.splits prod.splits)
from cont[unfolded out-eq] obtain verifier-result where
  verifier:
    Some (verifier-result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
and data'-eq: data' = data
and attacker-state'-eq: attacker-state' = attacker-state
by (auto elim!: set-dist-bindE)
have ext:
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data) ≤ final-state
by (rule verify-monad-hash-extends[OF verifier])
have raw-eq:
  ∧ i. i < rounds ⇒ attacker-raw-idxs ! i = raw-idxs ! i
proof –
  fix i
  assume i-bound: i < rounds
  have lookup-initial:
    fmlookup
      (HashMap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)

```

```

      (staged-query-chunks data) i)) =
      Some (attacker-raw-idxs ! i)
    using lookup-attacker i-bound by simp
  have lookup-final-from-initial:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (attacker-raw-idxs ! i)
    by (rule hash-extension-lookup[OF lookup-initial ext])
  show attacker-raw-idxs ! i = raw-idxs ! i
    using lookup-final-from-initial lookup-final[OF i-bound] data'-eq
    by simp
qed
have query-idxs-attacker:
  query-idxs = map (λraw. index (to-nat raw)) attacker-raw-idxs
proof (rule nth-equalityI)
  show length query-idxs =
    length (map (λraw. index (to-nat raw)) attacker-raw-idxs)
    using len-raw len-attacker-raw query-idxs-eq by simp
next
  fix i
  assume i-bound:
    i < length query-idxs
  then have i-rounds: i < rounds
    using len-raw query-idxs-eq by simp
  then show query-idxs ! i =
    map (λraw. index (to-nat raw)) attacker-raw-idxs ! i
    using query-idxs-eq raw-eq[OF i-rounds] len-raw apply simp
    using len-attacker-raw by auto
qed
show staged-transcript-query-index-list-set-hit Q
  (Some (data, attacker-state))
  unfolding staged-transcript-query-index-list-set-hit-def
  using len-attacker-raw query-in query-idxs-attacker lookup-attacker
  by auto
qed

lemma checked-staged-security-with-data-query-index-list-set-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-query-index-list-set-hit Q)
      adversary-initial-state ≤
      hash-relation-budget-value
        (query-index-raw-list-relation-fiber-bound Q)
        (staged-attacker-query-budget budgets +

```


staged-challenge-query-budget)
by (rule checked-staged-security-with-data-query-index-list-set-bound-from-transcript
 [OF wf controlled staged-transcript-query-index-list-set-hit-bound
 [OF wf controlled]])

lemma checked-staged-security-with-data-state-query-index-list-fresh-or-prequeried:

assumes support:

Some (((data, attacker-state), result), final-state) ∈
 set-dist
 (execute (checked-staged-security-experiment-with-data-state A)
 adversary-initial-state)

and hit:

staged-security-with-data-query-index-list-set-hit Q
 (Some (((data, attacker-state), result), final-state))

shows

staged-security-with-data-state-query-index-list-path-fresh Q
 (Some (((data, attacker-state), result), final-state)) ∨
 hash-relation-hit
 (checked-staged-query-index-list-relation A Q)
 adversary-initial-state attacker-state

proof –

from checked-staged-security-experiment-with-data-state-outcomeE[OF support]

obtain builder verifier **where**

builder:

Some (data, attacker-state) ∈
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state)

and verifier:

Some (result, final-state) ∈
 set-dist
 (execute verify-monad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))

by blast

from hit **obtain** raw-idxs query-idxs **where**

len-raw: length raw-idxs = rounds

and query-in: query-idxs ∈ Q

and query-idxs-eq:

query-idxs = map (λraw. index (to-nat raw)) raw-idxs

and lookup-final:

∧ i. i < rounds ⇒
 fmllookup (HashMap final-state)
 (QueryIndexChallenge i
 (state-after-query-chunks
 (staged-query-start-hash data)
 (staged-query-chunks data) i)) =
 Some (raw-idxs ! i)

unfolding staged-security-with-data-query-index-list-set-hit-def

```

  by auto
let ?vstate =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have raw-in: raw-idxs ∈ query-index-raw-list-preimage Q
  unfolding query-index-raw-list-preimage-def
  using len-raw query-in query-idxs-eq by blast
have ext: ?vstate ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
show ?thesis
proof (cases
  ∀ i < rounds.
    fmllookup (HashMap ?vstate)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        None)
  case True
  then have
    staged-security-with-data-state-query-index-list-path-fresh Q
      (Some (((data, attacker-state), result), final-state))
    unfolding staged-security-with-data-state-query-index-list-path-fresh-def
    using len-raw query-in query-idxs-eq lookup-final
    by (auto intro: exI[of - raw-idxs] exI[of - query-idxs])
  then show ?thesis by simp
next
case False
then obtain i old where
  i-bound: i < rounds
  and lookup-initial:
    fmllookup (HashMap ?vstate)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        Some old
  by (metis option.exhaust)
let ?key =
  QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i)
have lookup-final-old:
  fmllookup (HashMap final-state) ?key = Some old
  by (rule hash-extension-lookup[OF lookup-initial ext])
have old-eq: old = raw-idxs ! i
  using lookup-final-old lookup-final[OF i-bound] by simp
have lookup-attacker:
  fmllookup (HashMap attacker-state) ?key = Some (raw-idxs ! i)

```

```

using lookup-initial old-eq
unfolding verifier-state-from-adversary-def by simp
have relation:
  checked-staged-query-index-list-relation A Q ?key (raw-idxs ! i)
unfolding checked-staged-query-index-list-relation-def
by (intro exI[of - data] exI[of - attacker-state]
      exI[of - raw-idxs] exI[of - i] conjI)
    (use builder raw-in i-bound in simp-all)
have initial-none:
  fmlookup (HashMap adversary-initial-state) ?key = None
by (simp add: adversary-initial-state-def)
have
  hash-relation-hit
    (checked-staged-query-index-list-relation A Q)
    adversary-initial-state attacker-state
unfolding hash-relation-hit-def
by (intro exI[of - ?key] exI[of - raw-idxs ! i] conjI)
    (use initial-none lookup-attacker relation in simp-all)
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-data-state-query-index-list-prequery-hit-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-prequery-hit A Q)
    adversary-initial-state ≤
    hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound Q)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
unfolding checked-staged-security-experiment-with-data-state-def
proof (rule wp-event-bind-bound-by-head-event)
show wp-event (checked-staged-transcript-program A)
  (hash-relation-hit-event
    (checked-staged-query-index-list-relation A Q)
    adversary-initial-state)
  adversary-initial-state
  ≤ hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof -
have program:
  hash-relation-program
    (checked-staged-query-index-list-relation A Q)
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)

```

```

    (checked-staged-transcript-program A)
  by (rule hash-relation-program-checked-staged-transcript-query-index-list
    [OF wf controlled])
  then show ?thesis
    unfolding hash-relation-program-def hash-relation-budget-def by blast
qed
next
show
  staged-security-with-data-state-query-index-list-prequery-hit A Q None
     $\Rightarrow$ 
    hash-relation-hit-event
    (checked-staged-query-index-list-relation A Q)
    adversary-initial-state None
  unfolding staged-security-with-data-state-query-index-list-prequery-hit-def
    hash-relation-hit-event-def
  by simp
next
fix data attacker-state out
assume cont:
  out  $\in$ 
  set-dist
  (execute
    (get  $\gg$ 
      ( $\lambda s$ . put
        (verifier-state-from-adversary s
          (staged-proof-transcript data))  $\gg$ 
        ( $\lambda$ -. verify-monad  $\gg$ 
          ( $\lambda$ result. return ((data, s), result))))))
    attacker-state)
assume prequery:
  staged-security-with-data-state-query-index-list-prequery-hit A Q out
show
  hash-relation-hit-event
  (checked-staged-query-index-list-relation A Q)
  adversary-initial-state (Some (data, attacker-state))
proof (cases out)
case None
  then show ?thesis
    using prequery
    unfolding staged-security-with-data-state-query-index-list-prequery-hit-def
  by simp
next
case (Some packed)
from cont Some have cont-some:
  Some packed  $\in$ 
  set-dist
  (execute
    (get  $\gg$ 
      ( $\lambda s$ . put

```

```

      (verifier-state-from-adversary s
        (staged-proof-transcript data)) >>=
      (λ-. verify-monad >>=
        (λresult. return ((data, s), result))))
    attacker-state)
  by simp
obtain packed-value packed-state where packed-pair:
  packed = (packed-value, packed-state)
  by (cases packed)
from cont-some[unfolded packed-pair] obtain captured get-state where get-out:
  Some (captured, get-state) ∈ set-dist (execute get attacker-state)
and after-get0:
  Some (packed-value, packed-state) ∈
    set-dist
      (execute
        (put
          (verifier-state-from-adversary captured
            (staged-proof-transcript data)) >>=
            (λ-. verify-monad >>=
              (λresult. return ((data, captured), result))))
          get-state)
        by (rule set-dist-bindE) (rule that)
have captured-eq: captured = attacker-state
and get-state-eq: get-state = attacker-state
using get-out
unfolding set-dist-def get.rep-eq dist-get-def
by (simp-all add: dist-delta-dist delta-map-def)
have after-get:
  Some packed ∈
    set-dist
      (execute
        (put
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)) >>=
            (λ-. verify-monad >>=
              (λresult. return ((data, attacker-state), result))))
          attacker-state)
        using after-get0 captured-eq get-state-eq packed-pair by simp
from after-get[unfolded packed-pair] obtain unit put-state where put-out:
  Some (unit, put-state) ∈
    set-dist
      (execute
        (put
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
          attacker-state)
and after-put0:
  Some (packed-value, packed-state) ∈
    set-dist

```

```

      (execute
        (verify-monad  $\gg$ 
          ( $\lambda$ result. return ((data, attacker-state), result)))
        put-state)
    by (rule set-dist-bindE) (rule that)
  have put-state-eq:
    put-state =
      verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)
  using put-out
  unfolding set-dist-def put.rep-eq dist-put-def
  by (simp add: dist-delta-dist delta-map-def)
  have after-put:
    Some packed  $\in$ 
      set-dist
        (execute
          (verify-monad  $\gg$ 
            ( $\lambda$ result. return ((data, attacker-state), result)))
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
  using after-put0 put-state-eq packed-pair by simp
  from after-put[unfolded packed-pair] obtain result final-state where verify-out:
    Some (result, final-state)  $\in$ 
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
  and ret-out:
    Some (packed-value, packed-state)  $\in$ 
      set-dist (execute (return ((data, attacker-state), result)) final-state)
  by (rule set-dist-bindE) (rule that)
  have packed-eq: packed = (((data, attacker-state), result), final-state)
  proof -
    have value-eq: packed-value = ((data, attacker-state), result)
      by (rule set-dist-return-valueD[OF ret-out])
    have state-eq: packed-state = final-state
      by (rule set-dist-return-stateD[OF ret-out])
    show ?thesis
      using packed-pair value-eq state-eq by simp
  qed
  have out-eq:
    out = Some (((data, attacker-state), result), final-state)
  using Some packed-eq by simp
  from prequery have hit:
    hash-relation-hit
      (checked-staged-query-index-list-relation A Q)
      adversary-initial-state attacker-state
  unfolding out-eq
    staged-security-with-data-state-query-index-list-prequery-hit-def

```

```

    by simp
  then show ?thesis
    using hit unfolding hash-relation-hit-event-def by simp
qed
qed

lemma checked-staged-security-with-data-state-query-index-list-hit-bound-from-path-fresh:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and path-fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-index-list-path-fresh Q)
    adversary-initial-state  $\leq$  C
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-query-index-list-set-hit Q)
    adversary-initial-state  $\leq$ 
      C +
      hash-relation-budget-value
        (query-index-raw-list-relation-fiber-bound Q)
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget)
  proof -
    have event-mono:
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-with-data-query-index-list-set-hit Q)
      adversary-initial-state  $\leq$ 
      wp-event (checked-staged-security-experiment-with-data-state A)
        ( $\lambda$ out.
          staged-security-with-data-state-query-index-list-path-fresh Q out  $\vee$ 
          staged-security-with-data-state-query-index-list-prequery-hit A Q out)
      adversary-initial-state
    proof (rule wp-event-mono-on-support)
      fix out
      assume support:
        out  $\in$ 
        set-dist
        (execute (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
      and hit: staged-security-with-data-query-index-list-set-hit Q out
      show
        staged-security-with-data-state-query-index-list-path-fresh Q out  $\vee$ 
        staged-security-with-data-state-query-index-list-prequery-hit A Q out
    proof (cases out)
      case None
      then show ?thesis
        using hit unfolding staged-security-with-data-query-index-list-set-hit-def
        by simp
    next

```

```

case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed) (auto split: prod.splits)
have split:
  staged-security-with-data-state-query-index-list-path-fresh Q
    (Some (((data, attacker-state), result), final-state))  $\vee$ 
  hash-relation-hit
    (checked-staged-query-index-list-relation A Q)
  adversary-initial-state attacker-state
apply (rule
  checked-staged-security-with-data-state-query-index-list-fresh-or-prequeried)
using support hit unfolding out-eq by simp-all
then show ?thesis
  unfolding out-eq
    staged-security-with-data-state-query-index-list-prequery-hit-def
  by simp
qed
qed
have union-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    ( $\lambda out.$ 
    staged-security-with-data-state-query-index-list-path-fresh Q out  $\vee$ 
    staged-security-with-data-state-query-index-list-prequery-hit A Q out)
  adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-path-fresh Q)
  adversary-initial-state  $+$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-prequery-hit A Q)
  adversary-initial-state
by (rule wp-event-union-bound)
have prequery-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-prequery-hit A Q)
  adversary-initial-state  $\leq$ 
  hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets  $+$ 
    staged-challenge-query-budget)
by (rule
  checked-staged-security-with-data-state-query-index-list-prequery-hit-bound
  [OF wf controlled])
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-query-index-list-set-hit Q)
  adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-path-fresh Q)
  adversary-initial-state  $+$ 

```



```

    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-index-list-prequery-hit A Q)
      adversary-initial-state
  by (rule order-trans[OF event-mono union-bound])
also have ... ≤
  C +
  hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)
  by (rule add-mono[OF path-fresh-bound prequery-bound])
finally show ?thesis .
qed

```

lemma *accepted-fri-opening-transcript-imp-accepted-transcript-shape:*
assumes *fri:*
 accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
 dg composition-roots composition-bs composition-final query-idxs
 trace-round-layers composition-round-layers
shows $\exists$ *as. accepted-transcript-shape s out as query-idxs*
using *fri*
unfolding *accepted-fri-opening-transcript-def*
 accepted-transcript-shape-def
by *blast*

lemma *checked-staged-security-trace-fri-query-index-list-set-hit-bound-list:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 ($\lambda s. \text{trace-fri-query-index-list-set-hit } s \ Q$))
 adversary-initial-state ≤
 hash-relation-budget-value
 (query-index-raw-list-relation-fiber-bound Q)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
proof (rule *order-trans*)
show *wp-event (checked-staged-security-experiment-with-data-state A)*
 (staged-security-with-data-state-verifier-event
 ($\lambda s. \text{trace-fri-query-index-list-set-hit } s \ Q$))
 adversary-initial-state ≤
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-query-index-list-set-hit Q)
 adversary-initial-state
proof (rule *wp-event-mono-on-support*)
fix *out*
assume *support:*
 out ∈

```

    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
  and hit:
    staged-security-with-data-state-verifier-event
    ( $\lambda s$ . trace-fri-query-index-list-set-hit s Q) out
show staged-security-with-data-query-index-list-set-hit Q out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding staged-security-with-data-state-verifier-event-def
      staged-security-with-data-query-index-list-set-hit-def
    by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed) (auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from hit out-eq have trace-hit:
    trace-fri-query-index-list-set-hit ?s Q
    (Some (result, final-state))
    unfolding staged-security-with-data-state-verifier-event-def by simp
  from trace-hit obtain trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers where
    fri:
      accepted-fri-opening-transcript ?s (Some (result, final-state))
      trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final query-idxs trace-round-layers
      composition-round-layers
    and query-in: query-idxs  $\in$  Q
    unfolding trace-fri-query-index-list-set-hit-def by blast
  from accepted-fri-opening-transcript-imp-accepted-transcript-shape[OF fri]
  obtain as where shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
    by blast
  from checked-staged-security-with-data-state-accepted-shape-query-keys
    [OF wf controlled support[unfolded out-eq] shape]
  obtain raw-idxs where
    len-raw: length raw-idxs = rounds
    and query-idxs-eq:
      query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
    and lookup:
       $\bigwedge i. i < \text{rounds} \implies$ 

```

```

      fmlookup (HashMap final-state)
      (QueryIndexChallenge i
       (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
    by blast
  show ?thesis
    unfolding out-eq staged-security-with-data-query-index-list-set-hit-def
    using len-raw query-in query-idxs-eq lookup by auto
qed
qed
next
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-query-index-list-set-hit Q)
    adversary-initial-state
    ≤ hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound Q)
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule checked-staged-security-with-data-query-index-list-set-hit-bound
      [OF wf controlled])
qed

lemma checked-staged-security-composition-fri-query-index-list-set-hit-bound-list:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       (λs. composition-fri-query-index-list-set-hit s Q))
      adversary-initial-state ≤
      hash-relation-budget-value
        (query-index-raw-list-relation-fiber-bound Q)
        (staged-attacker-query-budget budgets +
         staged-challenge-query-budget)
  proof (rule order-trans)
    show wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       (λs. composition-fri-query-index-list-set-hit s Q))
      adversary-initial-state ≤
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-with-data-query-index-list-set-hit Q)
      adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out ∈
      set-dist
      (execute (checked-staged-security-experiment-with-data-state A)

```

```

    adversary-initial-state)
  and hit:
    staged-security-with-data-state-verifier-event
    (λs. composition-fri-query-index-list-set-hit s Q) out
  show staged-security-with-data-query-index-list-set-hit Q out
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding staged-security-with-data-state-verifier-event-def
      staged-security-with-data-query-index-list-set-hit-def
    by simp
  next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed) (auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from hit out-eq have composition-hit:
    composition-fri-query-index-list-set-hit ?s Q
    (Some (result, final-state))
  unfolding staged-security-with-data-state-verifier-event-def by simp
  from composition-hit obtain trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers where
  fri:
    accepted-fri-opening-transcript ?s (Some (result, final-state))
    trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final query-idxs trace-round-layers
    composition-round-layers
  and query-in: query-idxs ∈ Q
  unfolding composition-fri-query-index-list-set-hit-def by blast
  from accepted-fri-opening-transcript-imp-accepted-transcript-shape[OF fri]
  obtain as where shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
  by blast
  from checked-staged-security-with-data-state-accepted-shape-query-keys
    [OF wf controlled support[unfolded out-eq] shape]
  obtain raw-idxs where
    len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookup:
    ∧i. i < rounds ⇒
      fmlookup (HashMap final-state)
      (QueryIndexChallenge i

```

```

      (state-after-query-chunks
       (staged-query-start-hash data)
       (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  by blast
show ?thesis
  unfolding out-eq staged-security-with-data-query-index-list-set-hit-def
  using len-raw query-in query-idxs-eq lookup by auto
qed
qed
next
show wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-query-index-list-set-hit Q)
  adversary-initial-state
  ≤ hash-relation-budget-value
  (query-index-raw-list-relation-fiber-bound Q)
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule checked-staged-security-with-data-query-index-list-set-hit-bound
      [OF wf controlled])
qed

lemma checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-list:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and projection: fst ' P ⊆ Q
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       (λs. trace-fri-query-challenge-pair-set-hit s P))
      adversary-initial-state ≤
      hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound Q)
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)
  proof (rule order-trans)
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     (λs. trace-fri-query-challenge-pair-set-hit s P))
    adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       (λs. trace-fri-query-index-list-set-hit s Q))
      adversary-initial-state
  by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
      intro:
        trace-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit
        [OF - projection]
        split: option.splits prod.splits)

```

next
show *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
($\lambda s.$ trace-fri-query-index-list-set-hit s Q))
adversary-initial-state
 $\leq$ *hash-relation-budget-value*
(query-index-raw-list-relation-fiber-bound Q)
(staged-attacker-query-budget budgets + staged-challenge-query-budget)
by *(rule checked-staged-security-trace-fri-query-index-list-set-hit-bound-list*
 $[OF\ wf\ controlled])$
qed

lemma *checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-list:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *projection: $\bigwedge dg.$ fst ‘ $P\ dg \subseteq Q$*
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
($\lambda s.$ composition-fri-query-challenge-pair-set-hit s P))
adversary-initial-state $\leq$
hash-relation-budget-value
(query-index-raw-list-relation-fiber-bound Q)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)

proof *(rule order-trans)*
show *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
($\lambda s.$ composition-fri-query-challenge-pair-set-hit s P))
adversary-initial-state $\leq$
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
($\lambda s.$ composition-fri-query-index-list-set-hit s Q))
adversary-initial-state
by *(rule wp-event-mono-on-support)*
(auto simp: staged-security-with-data-state-verifier-event-def
intro:
composition-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit
 $[OF\ -\ projection]$
split: option.splits prod.splits)

next
show *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
($\lambda s.$ composition-fri-query-index-list-set-hit s Q))
adversary-initial-state
 $\leq$ *hash-relation-budget-value*
(query-index-raw-list-relation-fiber-bound Q)
(staged-attacker-query-budget budgets + staged-challenge-query-budget)
by *(rule*

checked-staged-security-composition-fri-query-index-list-set-hit-bound-list
[OF wf controlled])

qed

lemma *checked-staged-security-trace-fri-query-index-list-set-hit-bound-from-path-fresh:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *path-fresh-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-query-index-list-path-fresh Q)
adversary-initial-state ≤ C
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
(λs. trace-fri-query-index-list-set-hit s Q))
adversary-initial-state ≤
C +
hash-relation-budget-value
(query-index-raw-list-relation-fiber-bound Q)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)
proof (rule *order-trans*)
show *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
(λs. trace-fri-query-index-list-set-hit s Q))
adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-query-index-list-set-hit Q)
adversary-initial-state
proof (rule *wp-event-mono-on-support*)
fix *out*
assume *support:*
out ∈
set-dist
(execute (checked-staged-security-experiment-with-data-state A)
adversary-initial-state)
and *hit:*
staged-security-with-data-state-verifier-event
(λs. trace-fri-query-index-list-set-hit s Q) out
show *staged-security-with-data-query-index-list-set-hit Q out*
proof (cases *out*)
case *None*
then show *?thesis*
using *hit*
unfolding *staged-security-with-data-state-verifier-event-def*
staged-security-with-data-query-index-list-set-hit-def
by *simp*
next
case (*Some packed*)

```

then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed) (auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from hit out-eq have trace-hit:
  trace-fri-query-index-list-set-hit ?s Q
  (Some (result, final-state))
  unfolding staged-security-with-data-state-verifier-event-def by simp
from trace-hit obtain trace-roots trace-bs trace-final dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
  fri:
    accepted-fri-opening-transcript ?s (Some (result, final-state))
    trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final query-idxs trace-round-layers
    composition-round-layers
  and query-in: query-idxs ∈ Q
  unfolding trace-fri-query-index-list-set-hit-def by blast
from accepted-fri-opening-transcript-imp-accepted-transcript-shape[OF fri]
obtain as where shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as
  query-idxs
  by blast
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support[unfolded out-eq] shape]
obtain raw-idxs where
  len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookup:
     $\bigwedge i. i < \text{rounds} \implies$ 
    fmllookup (HashMap final-state)
    (QueryIndexChallenge i
    (state-after-query-chunks
    (staged-query-start-hash data)
    (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  by blast
show ?thesis
  unfolding out-eq staged-security-with-data-query-index-list-set-hit-def
  using len-raw query-in query-idxs-eq lookup by auto
qed
qed
next
show wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-query-index-list-set-hit Q)
  adversary-initial-state

```


$\leq C +$
 $\text{hash-relation-budget-value}$
 $(\text{query-index-raw-list-relation-fiber-bound } Q)$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
by (rule
 $\text{checked-staged-security-with-data-state-query-index-list-hit-bound-from-path-fresh}$
 $[OF \text{ wf controlled path-fresh-bound}])$
qed

lemma *checked-staged-security-composition-fri-query-index-list-set-hit-bound-from-path-fresh*:
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *path-fresh-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-query-index-list-path-fresh } Q)$
 $\text{adversary-initial-state} \leq C$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-query-index-list-set-hit } s \ Q))$
 $\text{adversary-initial-state} \leq$
 $C +$
 $\text{hash-relation-budget-value}$
 $(\text{query-index-raw-list-relation-fiber-bound } Q)$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
proof (rule *order-trans*)
show $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-query-index-list-set-hit } s \ Q))$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-query-index-list-set-hit } Q)$
 $\text{adversary-initial-state}$
proof (rule *wp-event-mono-on-support*)
fix *out*
assume *support*:
 $\text{out} \in$
 set-dist
 $(\text{execute } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{adversary-initial-state})$
and *hit*:
 $\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-query-index-list-set-hit } s \ Q) \ \text{out}$
show $\text{staged-security-with-data-query-index-list-set-hit } Q \ \text{out}$
proof (cases *out*)
case *None*
then show *?thesis*

```

using hit
unfolding staged-security-with-data-state-verifier-event-def
  staged-security-with-data-query-index-list-set-hit-def
by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed) (auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from hit out-eq have composition-hit:
  composition-fri-query-index-list-set-hit ?s Q
  (Some (result, final-state))
unfolding staged-security-with-data-state-verifier-event-def by simp
from composition-hit obtain trace-roots trace-bs trace-final dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
  fri:
  accepted-fri-opening-transcript ?s (Some (result, final-state))
  trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final query-idxs trace-round-layers
  composition-round-layers
and query-in: query-idxs ∈ Q
unfolding composition-fri-query-index-list-set-hit-def by blast
from accepted-fri-opening-transcript-imp-accepted-transcript-shape[OF fri]
obtain as where shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as
  query-idxs
by blast
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support[unfolded out-eq] shape]
obtain raw-idxs where
  len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
  fmllookup (HashMap final-state)
  (QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data)
      (staged-query-chunks data) i)) =
  Some (raw-idxs ! i)
by blast
show ?thesis
unfolding out-eq staged-security-with-data-query-index-list-set-hit-def
using len-raw query-in query-idxs-eq lookup by auto

```

```

qed
qed
next
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-query-index-list-set-hit Q)
    adversary-initial-state
    ≤ C +
      hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound Q)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
  by (rule
    checked-staged-security-with-data-state-query-index-list-hit-bound-from-path-fresh
    [OF wf controlled path-fresh-bound])
qed

lemma checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-path-fresh:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and projection: fst ' P ⊆ Q
  and path-fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-index-list-path-fresh Q)
      adversary-initial-state ≤ C
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-query-challenge-pair-set-hit s P))
      adversary-initial-state ≤
        C +
        hash-relation-budget-value
        (query-index-raw-list-relation-fiber-bound Q)
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget)
proof (rule order-trans)
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-query-challenge-pair-set-hit s P))
    adversary-initial-state ≤
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-with-data-state-verifier-event
          (λs. trace-fri-query-index-list-set-hit s Q))
        adversary-initial-state
  by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    intro:
      trace-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit
      [OF - projection]
      split: option.splits prod.splits)

```

next
show *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
($\lambda s.$ trace-fri-query-index-list-set-hit s Q))
adversary-initial-state
 $\leq C +$
hash-relation-budget-value
(query-index-raw-list-relation-fiber-bound Q)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)
by (rule
checked-staged-security-trace-fri-query-index-list-set-hit-bound-from-path-fresh
[OF wf controlled path-fresh-bound])

qed

lemma *checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-path-fresh:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *projection: $\bigwedge dg.$ fst ' P $dg \subseteq Q$*

and *path-fresh-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-query-index-list-path-fresh Q)

adversary-initial-state $\leq C$

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

($\lambda s.$ composition-fri-query-challenge-pair-set-hit s P))

adversary-initial-state $\leq$

C +

hash-relation-budget-value

(query-index-raw-list-relation-fiber-bound Q)

(staged-attacker-query-budget budgets +

staged-challenge-query-budget)

proof (rule order-trans)

show *wp-event (checked-staged-security-experiment-with-data-state A)*

(staged-security-with-data-state-verifier-event

($\lambda s.$ composition-fri-query-challenge-pair-set-hit s P))

adversary-initial-state $\leq$

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

($\lambda s.$ composition-fri-query-index-list-set-hit s Q))

adversary-initial-state

by (rule wp-event-mono-on-support)

(auto simp: staged-security-with-data-state-verifier-event-def

intro:

composition-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit

[OF - projection]

split: option.splits prod.splits)

next

```

show wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. composition-fri-query-index-list-set-hit s Q))
  adversary-initial-state
  ≤ C +
    hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
  by (rule
    checked-staged-security-composition-fri-query-index-list-set-hit-bound-from-path-fresh
    [OF wf controlled path-fresh-bound])
qed

end

end

```

```

theory Soundness-FRI-Query-List-Exact-Product
  imports Soundness-FRI-Query-List-Prequery
begin

```

Exact product accounting for sampled FRI query-index lists.

The preceding layer proves the product theorem in raw field space. This layer converts raw-list preimages to query-index-list targets using the current exact-uniformity assumptions for the modulo sampler.

```

context soundness
begin

```

```

definition verifier-after-composition-fri-raw-list-path-hit
  :: ('f, 'a) protocol-channel-scheme ⇒ nat list set ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
  verifier-after-composition-fri-raw-list-path-hit s Q out ⟷
    (∃ final query-state rows.
      Some (final, query-state) ∈ set-dist (execute read s) ∧
      rows ∈ query-index-raw-list-preimage Q ∧
      query-rounds-raw-list-path-hit query-state rows rounds out)

```

```

lemma verifier-after-composition-fri-raw-list-path-hitI:
  assumes read-final:
    Some (final, query-state) ∈ set-dist (execute read s)
  and raw-in: rows ∈ query-index-raw-list-preimage Q
  and len-rows: length rows = rounds
  and len-chunks: length chunks = rounds
  and state:
    PState t = state-after-query-chunks (PState query-state) chunks rounds
  and fresh:

```

```

    query-index-list-path-fresh query-state rounds chunks raws
and lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) chunks i)) =
      Some (raws ! i)
  and out-eq: out = Some (results, t)
shows verifier-after-composition-fri-raw-list-path-hit s Q out
proof –
  have path: query-rounds-raw-list-path-hit query-state raws rounds out
  unfolding out-eq
  by (rule query-rounds-raw-list-path-hitI
    [OF len-raws len-chunks state fresh lookup])
  show ?thesis
  unfolding verifier-after-composition-fri-raw-list-path-hit-def
  by (intro exI[of - final] exI[of - query-state] exI[of - raws]
    conjI read-final raw-in path)
qed

```

lemma staged-security-with-data-state-query-index-list-path-fresh-SomeE:

```

assumes hit:
  staged-security-with-data-state-query-index-list-path-fresh Q
  (Some (((data, attacker-state), result), final-state))
obtains raw-idxs query-idxs where
  length raw-idxs = rounds
  query-idxs  $\in$  Q
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup
      (HashMap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
    None

```

```

proof –
  have hit-ex:
     $\exists$  raw-idxs query-idxs.
      length raw-idxs = rounds  $\wedge$ 

```

```

query-idxs ∈ Q ∧
query-idxs = map (λraw. index (to-nat raw)) raw-idxs ∧
(∀ i < rounds.
  fmllookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i) ∧
  fmllookup
    (HashMap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    None)
using hit
unfolding staged-security-with-data-state-query-index-list-path-fresh-def
by simp
from hit-ex obtain raw-idxs query-idxs where len:
  length raw-idxs = rounds
and in-Q: query-idxs ∈ Q
and map-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookups:
  ∀ i < rounds.
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i) ∧
    fmllookup
      (HashMap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      None
by blast
show ?thesis
proof (rule that[OF len in-Q map-eq])
  fix i
  assume i < rounds
  then show

```

```

    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  using lookups by blast
next
fix i
assume i < rounds
then show
  fmlookup
    (HashMap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
  None
  using lookups by blast
qed
qed

lemma staged-security-with-data-state-query-index-list-path-fresh-None[simp]:
  ¬ staged-security-with-data-state-query-index-list-path-fresh Q None
unfolding staged-security-with-data-state-query-index-list-path-fresh-def
by simp

lemma staged-security-with-data-state-query-index-list-path-fresh-caseD:
assumes hit:
  staged-security-with-data-state-query-index-list-path-fresh Q
  (case out of
    None ⇒ None
  | Some (result, final-state) ⇒
    Some (((data, attacker-state), result), final-state))
shows
  ∃ result final-state raw-idxs query-idxs.
  out = Some (result, final-state) ∧
  length raw-idxs = rounds ∧
  query-idxs ∈ Q ∧
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs ∧
  (∀ i < rounds.
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)) ∧

```



```

    (∀ i < rounds.
      fmllookup
        (HashMap
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))))
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
      None)
  proof (cases out)
  case None
  then show ?thesis
    using hit by simp
next
case (Some pair)
then obtain result final-state where out-eq:
  out = Some (result, final-state)
by (cases pair) simp
have hit-ex:
  ∃ raw-idxs query-idxs.
    length raw-idxs = rounds ∧
    query-idxs ∈ Q ∧
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs ∧
    (∀ i < rounds.
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
      Some (raw-idxs ! i) ∧
      fmllookup
        (HashMap
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))))
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
      None)
  using hit out-eq
  unfolding staged-security-with-data-state-query-index-list-path-fresh-def
  by simp
from hit-ex obtain raw-idxs query-idxs where
  length raw-idxs = rounds
  query-idxs ∈ Q
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  ∀ i < rounds.
    fmllookup (HashMap final-state)

```

```

      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
    Some (raw-idxs ! i) ∧
  fmlookup
    (HashMap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
  None
  by blast
then show ?thesis
  using out-eq by blast
qed

lemma staged-security-with-data-state-query-index-list-path-fresh-caseE:
assumes hit:
  staged-security-with-data-state-query-index-list-path-fresh Q
  (case out of
    None ⇒ None
  | Some (result, final-state) ⇒
    Some (((data, attacker-state), result), final-state))
obtains result final-state raw-idxs query-idxs where
  out = Some (result, final-state)
  length raw-idxs = rounds
  query-idxs ∈ Q
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  ∧ i. i < rounds ⇒
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  ∧ i. i < rounds ⇒
    fmlookup
      (HashMap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
    None
proof (cases out)

```

```

case None
then have False
  using hit by simp
then show ?thesis by simp
next
case (Some pair)
then obtain result final-state where out-eq:
  out = Some (result, final-state)
  by (cases pair) simp
have hit-some:
  staged-security-with-data-state-query-index-list-path-fresh Q
    (Some (((data, attacker-state), result), final-state))
  using hit out-eq by simp
from staged-security-with-data-state-query-index-list-path-fresh-SomeE
  [OF hit-some]
obtain raw-idxs query-idxs where
  len: length raw-idxs = rounds
  and in-Q: query-idxs ∈ Q
  and map-eq: query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookup:
     $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
  and fresh:
     $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup
      (HashMap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      None
  by blast
show ?thesis
  by (rule that[OF out-eq len in-Q map-eq lookup fresh])
qed

lemma verifier-header-transcript-from-prefix-read:
assumes len-trace: length f-roots = ceil-log clength
and len-alpha: length as = length spec
and len-comp: length comp-roots = ceil-log (to-nat dg + 1)
and transcript-prefix:
  PTranscript s =

```

```

    [fr] @ f-roots @ [f-final] @ as @ [dg] @
    comp-roots @ PTranscript prefix-state
  and read-prefix: PTranscript prefix-state = final # rest
  and query-rest: PTranscript query-state = rest
shows
  verifier-header-transcript s fr f-roots f-final as dg
  comp-roots final (PTranscript query-state)
proof -
  have transcript:
    PTranscript s =
      verifier-header-messages fr f-roots f-final as dg comp-roots final @
      PTranscript query-state
  using transcript-prefix read-prefix query-rest
  unfolding verifier-header-messages-def
  by simp
show ?thesis
  unfolding verifier-header-transcript-def
  using transcript len-trace len-alpha len-comp by simp
qed

```

lemma *verifier-composition-fri-prefix-outcome-basicD*:

assumes *outcome*:

Some ((fr, f-fl, f-final, as, dg, fl), t) ∈
set-dist (execute verifier-composition-fri-prefix s)

shows

length f-fl = ceil-log clength ∧
length as = length spec ∧
length fl = ceil-log (to-nat dg + 1) ∧
PTranscript s =
[fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
map snd fl @ PTranscript t ∧
PState t =
foldl concat
(concat
(foldl concat
(concat
(foldl concat (concat (PState s) fr) (map snd f-fl))
f-final)
as)
dg)
(map snd fl) ∧
PQueryCounter t = PQueryCounter s

proof -

have *full*: *length f-fl = ceil-log clength ∧*
length as = length spec ∧
length fl = ceil-log (to-nat dg + 1) ∧
PTranscript s =
[fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
map snd fl @ PTranscript t ∧

```

PState t =
  foldl concat
    (concat
      (foldl concat
        (concat
          (foldl concat (concat (PState s) fr) (map snd f-fl))
            f-final)
          as)
        dg)
      (map snd fl) ∧
  s ≤ t ∧
  PQueryCounter t = PQueryCounter s ∧
  (∀ i < length f-fl.
    fmlookup (HashMap t)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr)
          (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))) ∧
  (∀ i < length fl.
    fmlookup (HashMap t)
      (CompositionFriChallenge (PCompositionFriCounter s + i)
        (foldl concat
          (concat
            (foldl concat
              (concat
                (foldl concat (concat (PState s) fr) (map snd f-fl))
                  f-final)
                as)
              dg)
            (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i)))
  by (rule verifier-composition-fri-prefix-outcome[OF outcome])
then show ?thesis by simp
qed

```

lemma *alpha-round-preserves-query-lookup*:

assumes *outcome*: Some (a, t) ∈ set-dist (execute alpha-round s)

shows

fmlookup (HashMap t) (QueryIndexChallenge i x) =

fmlookup (HashMap s) (QueryIndexChallenge i x)

proof –

from *outcome* **obtain** a0 s0 a1 s1 s2 **where**

rand: Some (a0, s0) ∈ set-dist (execute receive-alpha-challenge s)

and *read-a*: Some (a1, s1) ∈ set-dist (execute read s0)

and *assert-a*: Some ((), s2) ∈ set-dist (execute (assert (a0 = a1)) s1)

and *ret-a*: Some (a, t) ∈ set-dist (execute (return a1) s2)

unfolding *alpha-round-def*

by (auto simp: Let-def elim!: set-dist-bindE)

have *neq*:

```

    QueryIndexChallenge i x ≠
      AlphaChallenge (PAlphaCounter s) (PState s)
  by simp
have rand-lookup:
  fmlookup (HashMap s0) (QueryIndexChallenge i x) =
    fmlookup (HashMap s) (QueryIndexChallenge i x)
  by (rule receive-alpha-challenge-preserves-other-lookup[OF rand neq])
have read-lookup:
  HashMap s1 = HashMap s0
  by (rule read-preserves-hash-map[OF read-a])
have s2-eq: s2 = s1
  using assert-a unfolding assert-def
  by (cases a0 = a1) (auto simp: throw-no-outcome)
have t-eq: t = s2
  using ret-a by simp
show ?thesis
  using rand-lookup read-lookup unfolding t-eq s2-eq by simp
qed

```

```

lemma receive-trace-fri-commits-preserves-query-lookup:
  assumes outcome:
    Some ((b, r), t) ∈ set-dist (execute receive-trace-fri-commits s)
  shows
    fmlookup (HashMap t) (QueryIndexChallenge i x) =
      fmlookup (HashMap s) (QueryIndexChallenge i x)
proof —
  from outcome obtain s-read where
    read-r: Some (r, s-read) ∈ set-dist (execute read s)
  and rand-b:
    Some (b, t) ∈ set-dist (execute receive-trace-fri-challenge s-read)
  unfolding receive-trace-fri-commits-def receive-fri-commits-with-def
  by (auto elim!: set-dist-bindE)
have read-lookup:
  HashMap s-read = HashMap s
  by (rule read-preserves-hash-map[OF read-r])
have neq:
  QueryIndexChallenge i x ≠
    TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)
  by simp
have rand-lookup:
  fmlookup (HashMap t) (QueryIndexChallenge i x) =
    fmlookup (HashMap s-read) (QueryIndexChallenge i x)
  by (rule receive-trace-fri-challenge-preserves-other-lookup
      [OF rand-b neq])
show ?thesis
  using rand-lookup read-lookup by simp
qed

```

```

lemma receive-composition-fri-commits-preserves-query-lookup:

```

```

assumes outcome:
  Some ((b, r), t) ∈
    set-dist (execute receive-composition-fri-commits s)
shows
  fmlookup (HashMap t) (QueryIndexChallenge i x) =
    fmlookup (HashMap s) (QueryIndexChallenge i x)
proof –
from outcome obtain s-read where
  read-r: Some (r, s-read) ∈ set-dist (execute read s)
and rand-b:
  Some (b, t) ∈
    set-dist (execute receive-composition-fri-challenge s-read)
unfolding receive-composition-fri-commits-def receive-fri-commits-with-def
by (auto elim!: set-dist-bindE)
have read-lookup:
  HashMap s-read = HashMap s
by (rule read-preserves-hash-map[OF read-r])
have neg:
  QueryIndexChallenge i x ≠
    CompositionFriChallenge
      (PCompositionFriCounter s-read) (PState s-read)
by simp
have rand-lookup:
  fmlookup (HashMap t) (QueryIndexChallenge i x) =
    fmlookup (HashMap s-read) (QueryIndexChallenge i x)
by (rule receive-composition-fri-challenge-preserves-other-lookup
  [OF rand-b neg])
show ?thesis
  using rand-lookup read-lookup by simp
qed

```

```

lemma verifier-composition-fri-prefix-preserves-query-lookup:
assumes outcome:
  Some ((fr, f-fl, f-final, as, dg, fl), t) ∈
    set-dist (execute verifier-composition-fri-prefix s)
shows
  fmlookup (HashMap t) (QueryIndexChallenge i x) =
    fmlookup (HashMap s) (QueryIndexChallenge i x)
proof –
let ?key = QueryIndexChallenge i x
from outcome obtain s1 s2 s3 s4 s5 s6 s7 where
  read-fr: Some (fr, s1) ∈ set-dist (execute read s)
and trace-fri:
  Some (f-fl, s2) ∈
    set-dist (execute
      (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
and read-f-final: Some (f-final, s3) ∈ set-dist (execute read s2)
and alpha-out:
  Some (as, s4) ∈

```

```

    set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
and read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
and degree-assert:
    Some ((), s6) ∈
    set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
and comp-fri:
    Some (fl, s7) ∈
    set-dist (execute
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1))) s6)
and ret:
    Some ((fr, f-fl, f-final, as, dg, fl), t) ∈
    set-dist (execute (return (fr, f-fl, f-final, as, dg, fl)) s7)
unfolding verifier-composition-fri-prefix-def
by (auto elim!: set-dist-bindE)
have lookup-s1:
  fmlookup (HashMap s1) ?key = fmlookup (HashMap s) ?key
  using read-preserves-hash-map[OF read-fr] by simp
have lookup-s2:
  fmlookup (HashMap s2) ?key = fmlookup (HashMap s1) ?key
proof (rule ntimes-preserves-lookup[OF - trace-fri])
  fix y u v
  assume out: Some (y, v) ∈ set-dist (execute receive-trace-fri-commits u)
  obtain b r where y-eq: y = (b, r)
  by (cases y) simp
  show fmlookup (HashMap v) ?key = fmlookup (HashMap u) ?key
  by (rule receive-trace-fri-commits-preserves-query-lookup
    [OF out[unfolded y-eq]])
qed
have lookup-s3:
  fmlookup (HashMap s3) ?key = fmlookup (HashMap s2) ?key
  using read-preserves-hash-map[OF read-f-final] by simp
have lookup-s4:
  fmlookup (HashMap s4) ?key = fmlookup (HashMap s3) ?key
proof (rule mmap-preserves-lookup[OF - alpha-out])
  fix m y u v
  assume m-in: m ∈ set (replicate (length spec) alpha-round)
  and out: Some (y, v) ∈ set-dist (execute m u)
  then have m-eq: m = alpha-round
  by simp
  show fmlookup (HashMap v) ?key = fmlookup (HashMap u) ?key
  using alpha-round-preserves-query-lookup[OF out[unfolded m-eq]] .
qed
have lookup-s5:
  fmlookup (HashMap s5) ?key = fmlookup (HashMap s4) ?key
  using read-preserves-hash-map[OF read-dg] by simp
have s6-eq: s6 = s5
  using degree-assert unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)

```



```

have lookup-s7:
  fmlookup (HashMap s7) ?key = fmlookup (HashMap s6) ?key
proof (rule ntimes-preserves-lookup[OF - comp-fri])
  fix y u v
  assume out: Some (y, v) ∈
    set-dist (execute receive-composition-fri-commits u)
  obtain b r where y-eq: y = (b, r)
  by (cases y) simp
  show fmlookup (HashMap v) ?key = fmlookup (HashMap u) ?key
  by (rule receive-composition-fri-commits-preserves-query-lookup
    [OF out[unfolded y-eq]])
qed
have t-eq: t = s7
  using ret by simp
show ?thesis
  using lookup-s1 lookup-s2 lookup-s3 lookup-s4 lookup-s5 lookup-s7
  unfolding t-eq s6-eq by simp
qed

lemma query-index-raw-singleton-card-uniform:
  assumes b-in: b ∈ query-sample-space
  shows card (query-index-raw-preimage {b}) =
    size div query-sample-space-size
proof -
  have subset: {b} ⊆ query-sample-space
  using b-in by simp
  have card (query-index-raw-preimage {b}) =
    (size div query-sample-space-size) * card {b}
  unfolding card-query-index-raw-preimage-eq-nat-preimage
  apply (rule card-query-index-nat-preimage-uniform-range)
  apply (rule to-nat-range)
  using query-sample-space-size-dvd subset
  unfolding query-sample-space-size-def by simp-all
  then show ?thesis
  by simp
qed

lemma query-index-raw-list-fixed-preimage-card-bound:
  assumes len: length query-idxs = rounds
  and subset: set query-idxs ⊆ query-sample-space
  shows card {raws.
    length raws = rounds ∧
    map (λraw. index (to-nat raw)) raws = query-idxs}
    ≤ (size div query-sample-space-size) ^ rounds
  using len subset
proof (induction rounds arbitrary: query-idxs)
  case 0
  then have {raws.
    length raws = 0 ∧

```

```

    map (λraw. index (to-nat raw)) rows = query-idxs} = {}
  by auto
  then show ?case by simp
next
case (Suc n)
obtain q qs where query-eq: query-idxs = q # qs
  using Suc.premis(1) by (cases query-idxs) auto
have len-qs: length qs = n
  using Suc.premis(1) unfolding query-eq by simp
have q-in: q ∈ query-sample-space
  using Suc.premis(2) unfolding query-eq by simp
have qs-subset: set qs ⊆ query-sample-space
  using Suc.premis(2) unfolding query-eq by simp
let ?A = {rows.
  length rows = Suc n ∧
  map (λraw. index (to-nat raw)) rows = q # qs}
let ?B = query-index-row-preimage {q}
let ?C = {rows.
  length rows = n ∧
  map (λraw. index (to-nat raw)) rows = qs}
have A-eq: ?A = (λ(raw, rows). raw # rows) ‘ (?B × ?C)
proof
  show ?A ⊆ (λ(raw, rows). raw # rows) ‘ (?B × ?C)
  proof
    fix rows
    assume rows-in: rows ∈ ?A
    then obtain raw tail where rows-eq: rows = raw # tail
      by (cases rows) auto
    have raw-in: raw ∈ ?B
      using rows-in unfolding rows-eq query-index-row-preimage-def by simp
    have tail-in: tail ∈ ?C
      using rows-in unfolding rows-eq by simp
    show rows ∈ (λ(raw, rows). raw # rows) ‘ (?B × ?C)
      unfolding rows-eq by (intro image-eqI[of - - (raw, tail)])
      (use raw-in tail-in in simp-all)
  qed
next
  show (λ(raw, rows). raw # rows) ‘ (?B × ?C) ⊆ ?A
  unfolding query-index-row-preimage-def by auto
qed
have inj: inj-on (λ(raw, rows). raw # rows) (?B × ?C)
  unfolding inj-on-def by auto
have finite-B: finite ?B
  by simp
have finite-C: finite ?C
proof -
  have ?C ⊆ {xs. length xs = n}
  by auto
  then show ?thesis

```

```

    by (auto intro: List.finite-list-length)
qed
have card ?A = card (( $\lambda$ (raw, rows). raw # rows) ‘ ( $?B \times ?C$ ))
  unfolding A-eq by simp
also have ... = card ( $?B \times ?C$ )
  by (rule card-image[OF inj])
also have ... = card ?B * card ?C
  using finite-B finite-C by simp
also have ...  $\leq$ 
  (size div query-sample-space-size) *
  (size div query-sample-space-size)  $^n$ 
proof (rule mult-mono)
  show card ?B  $\leq$  size div query-sample-space-size
    using query-index-raw-singleton-card-uniform[OF q-in] by simp
  show card ?C  $\leq$  (size div query-sample-space-size)  $^n$ 
    by (rule Suc.IH[OF len-qs qs-subset])
  show 0  $\leq$  size div query-sample-space-size by simp
  show 0  $\leq$  card ?C by simp
qed
also have ... = (size div query-sample-space-size)  $^{Suc\ n}$ 
  by simp
finally show ?case
  unfolding query-eq .
qed

lemma query-index-raw-list-preimage-card-bound:
  assumes subset:  $Q \subseteq \text{fri-query-index-list-space}$ 
  shows card (query-index-raw-list-preimage Q)
     $\leq$  card Q * (size div query-sample-space-size)  $^{rounds}$ 
proof -
  let ?fiber =  $\lambda \text{query-idxs}. \{\text{rows}.$ 
    length rows = rounds  $\wedge$ 
    map ( $\lambda \text{raw}. \text{index (to-nat raw)}$ ) rows = query-idxs $\}$ 
  have finite-Q: finite Q
    using subset by (rule finite-subset) (rule finite-fri-query-index-list-space)
  have preimage-eq:
    query-index-raw-list-preimage Q = ( $\bigcup \text{query-idxs} \in Q. ?\text{fiber query-idxs}$ )
    unfolding query-index-raw-list-preimage-def by auto
  have finite-fiber:  $\bigwedge \text{query-idxs}. \text{finite } (?\text{fiber query-idxs})$ 
  proof -
    fix query-idxs
    have ?fiber query-idxs  $\subseteq \{\text{xs}. \text{length xs} = \text{rounds}\}$ 
      by auto
    then show finite (?fiber query-idxs)
      by (auto intro: List.finite-list-length)
  qed
  have card (query-index-raw-list-preimage Q) =
    card ( $\bigcup \text{query-idxs} \in Q. ?\text{fiber query-idxs}$ )
    unfolding preimage-eq ..

```

```

also have ...  $\leq$  ( $\sum query\_idxs \in Q. card \ (?fiber \ query\_idxs)$ )
  using finite-Q by (auto intro: card-UN-le)
also have ...  $\leq$ 
  ( $\sum query\_idxs \in Q.$ 
    ( $size \ div \ query\_sample\_space\_size$ )  $\wedge$  rounds)
proof (rule sum-mono)
  fix query_idx
  assume query-in: query_idx  $\in$  Q
  have len: length query_idx = rounds
    using query-in subset unfolding fri-query-index-list-space-def by auto
  have entries: set query_idx  $\subseteq$  query-sample-space
    using query-in subset unfolding fri-query-index-list-space-def by auto
  show card (?fiber query_idx)
     $\leq$  ( $size \ div \ query\_sample\_space\_size$ )  $\wedge$  rounds
    by (rule query-index-row-list-fixed-preimage-card-bound[OF len entries])
qed
also have ... = card Q * ( $size \ div \ query\_sample\_space\_size$ )  $\wedge$  rounds
  using finite-Q by simp
finally show ?thesis .
qed

```

lemma query-index-row-list-preimage-probability-bound:

```

assumes subset: Q  $\subseteq$  fri-query-index-list-space
shows
  nnreal (card (query-index-row-list-preimage Q)) *
    ( $1 \ / \ nnreal \ size$ )  $\wedge$  rounds
 $\leq$  nnreal (card Q) *
  ( $1 \ / \ nnreal \ (card \ query\_sample\_space)$ )  $\wedge$  rounds

```

proof –

```

let ?d = size div query-sample-space-size
have card-le:
  card (query-index-row-list-preimage Q)
     $\leq$  card Q * ?d  $\wedge$  rounds
  by (rule query-index-row-list-preimage-card-bound[OF subset])
have dvd: query-sample-space-size dvd size
  using query-sample-space-size-dvd
  unfolding query-sample-space-size-def by simp
have size-eq: size = ?d * query-sample-space-size
  using dvd query-sample-space-size-pos by (simp add: dvd-eq-mod-eq-0)
have size-pos: 0 < size
  using size-card by simp
have d-pos: 0 < ?d
proof (cases ?d)
  case 0
  then show ?thesis
    using size-eq size-pos by simp
next
  case (Suc n)
  then show ?thesis by simp

```

```

qed
have le:
  nnreal (card (query-index-raw-list-preimage Q)) *
    (1 / nnreal size) ^ rounds
≤ nnreal (card Q * ?d ^ rounds) *
  (1 / nnreal size) ^ rounds
using card-le apply (simp add: mult-right-mono)
by (metis mult-right-mono of-nat-le-iff of-nat-mult of-nat-power zero-least)
have prod-eq:
  nnreal (card Q * ?d ^ rounds) *
    (1 / nnreal size) ^ rounds =
  nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds
proof -
  have card-space-eq:
    card query-sample-space = query-sample-space-size
  by (rule card-query-sample-space)
  have base-eq:
    nnreal ?d * (1 / nnreal size) =
      1 / nnreal (card query-sample-space)
  proof (rule nn2real-eq-iff[THEN iffD1])
    have real-base:
      real ?d / real size = 1 / real query-sample-space-size
    proof -
      have real-size-eq:
        real size = real ?d * real query-sample-space-size
      using arg-cong[OF size-eq, of real]
      by (simp add: of-nat-mult)
      have real ?d / real size =
        real ?d / (real ?d * real query-sample-space-size)
      using real-size-eq by simp
      also have ... = 1 / real query-sample-space-size
      using d-pos query-sample-space-size-pos
      by (simp add: field-simps)
      finally show ?thesis .
    qed
  show nn2real (nnreal ?d * (1 / nnreal size)) =
    nn2real (1 / nnreal (card query-sample-space))
  proof -
    have lhs-eq:
      nn2real (nnreal ?d * (1 / nnreal size)) =
        real ?d / real size
    proof -
      have nn2real (nnreal ?d * (1 / nnreal size)) =
        real ?d * (1 / real size)
      by (simp only: nn2real-mult nn2real-nnreal
        nn2real-divide nn2real-1)
      also have ... = real ?d / real size
      by (simp add: divide-inverse)
    qed
  qed

```

```

    finally show ?thesis .
qed
have rhs-eq:
  nn2real (1 / nnreal (card query-sample-space)) =
    1 / real query-sample-space-size
  unfolding card-space-eq
  by (simp only: nn2real-divide nn2real-1 nn2real-nnreal)
show ?thesis
  using real-base lhs-eq rhs-eq by simp
qed
qed
have
  nnreal (card Q * ?d ^ rounds) * (1 / nnreal size) ^ rounds =
  nnreal (card Q) * (nnreal ?d * (1 / nnreal size)) ^ rounds
  by (simp add: of-nat-mult of-nat-power power-mult-distrib
    ac-simps)
also have ... =
  nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds
  using base-eq by simp
finally show ?thesis .
qed
show ?thesis
  using le prod-eq by simp
qed

lemma wp-ntimes-verifier-query-round-program-query-list-path-product-bound:
  assumes subset:  $Q \subseteq \text{fri-query-index-list-space}$ 
  shows
    wp-event
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      ( $\lambda \text{out. } \exists \text{raws} \in \text{query-index-raw-list-preimage } Q.
        \text{query-rounds-raw-list-path-hit } s \text{ raws rounds out}$ )
       $s \leq$ 
      nnreal (card Q) *
        (1 / nnreal (card query-sample-space)) ^ rounds
  proof -
    have raw-bound:
      wp-event
        (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
        ( $\lambda \text{out. } \exists \text{raws} \in \text{query-index-raw-list-preimage } Q.
          \text{query-rounds-raw-list-path-hit } s \text{ raws rounds out}$ )
         $s \leq$ 
        nnreal (card (query-index-raw-list-preimage Q)) *
          (1 / nnreal size) ^ rounds
      by (rule wp-ntimes-verifier-query-round-program-raw-list-preimage-path-hit-bound)
    also have ...  $\leq$ 

```

$nnreal (card Q) * (1 / nnreal (card query-sample-space)) ^ rounds$
 by (rule query-index-raw-list-preimage-probability-bound[OF subset])
 finally show ?thesis .
 qed

lemma read-outcome-unique-state:
 assumes out1: Some (x, t) ∈ set-dist (execute read s)
 and out2: Some (y, u) ∈ set-dist (execute read s)
 shows x = y ∧ t = u
proof –
 from read-outcome[OF out1] obtain rest where
 tr: PTranscript s = x # rest
 by blast
 have out1-eq:
 t = s(PState := concat (PState s) x, PTranscript := rest)
 using read-nonempty-outcome[OF out1 tr] by simp
 have out2-eq:
 y = x ∧
 u = s(PState := concat (PState s) x, PTranscript := rest)
 using read-nonempty-outcome[OF out2 tr] by simp
 then show ?thesis
 using out1-eq by simp
 qed

lemma wp-verifier-after-composition-fri-raw-list-path-product-bound:
 assumes subset: Q ⊆ fri-query-index-list-space
 shows
 wp-event (verifier-after-composition-fri header)
 (verifier-after-composition-fri-raw-list-path-hit s Q) s ≤
 nnreal (card Q) *
 (1 / nnreal (card query-sample-space)) ^ rounds
proof –
 obtain fr f-fl f-final as dg fl where header-eq:
 header = (fr, f-fl, f-final, as, dg, fl)
 by (cases header) auto
 have after-eq:
 verifier-after-composition-fri header =
 (read ≧
 (λfinal.
 ntimes
 (verifier-query-round-program fr f-fl f-final as fl final)
 rounds))
 unfolding header-eq verifier-after-composition-fri-def by simp
 show ?thesis
 unfolding after-eq
proof (rule wp-event-bind-bound-by-cont)
 show ¬ verifier-after-composition-fri-raw-list-path-hit s Q None
 unfolding verifier-after-composition-fri-raw-list-path-hit-def

```

      query-rounds-raw-list-path-hit-def
    by simp
next
fix final query-state
assume read-final:
  Some (final, query-state) ∈ set-dist (execute read s)
have mono:
  wp-event
  (ntimes
    (verifier-query-round-program fr f-fl f-final as fl final)
    rounds)
  (verifier-after-composition-fri-raw-list-path-hit s Q)
  query-state ≤
  wp-event
  (ntimes
    (verifier-query-round-program fr f-fl f-final as fl final)
    rounds)
  (λout. ∃ raws ∈ query-index-raw-list-preimage Q.
    query-rounds-raw-list-path-hit query-state raws rounds out)
  query-state
proof (rule wp-event-mono-on-support)
fix out
assume hit:
  verifier-after-composition-fri-raw-list-path-hit s Q out
from hit obtain final' query-state' raws where
  read':
    Some (final', query-state') ∈ set-dist (execute read s)
  and raw-in: raws ∈ query-index-raw-list-preimage Q
  and path:
    query-rounds-raw-list-path-hit query-state' raws rounds out
  unfolding verifier-after-composition-fri-raw-list-path-hit-def
  by blast
have query-state' = query-state
  using read-outcome-unique-state[OF read' read-final] by simp
then show ∃ raws ∈ query-index-raw-list-preimage Q.
  query-rounds-raw-list-path-hit query-state raws rounds out
  using raw-in path by blast
qed
also have ... ≤
  nnreal (card Q) *
  (1 / nnreal (card query-sample-space)) ^ rounds
by (rule
  wp-ntimes-verifier-query-round-program-query-list-path-product-bound
  [OF subset])
finally show
  wp-event
  (ntimes
    (verifier-query-round-program fr f-fl f-final as fl final)
    rounds)

```



```

      (verifier-after-composition-fri-raw-list-path-hit s Q)
      query-state
    ≤ nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds .
qed
qed

lemma verifier-after-composition-fri-query-index-list-path-fresh-imp-raw-hit-after-prefix:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and prefix:
  Some (header, prefix-state) ∈
    set-dist
      (execute verifier-composition-fri-prefix
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
and suffix:
  out ∈ set-dist
    (execute (verifier-after-composition-fri header) prefix-state)
and hit:
  staged-security-with-data-state-query-index-list-path-fresh Q
  (case out of
    None ⇒ None
    | Some (result, final-state) ⇒
      Some (((data, attacker-state), result), final-state))
shows verifier-after-composition-fri-raw-list-path-hit prefix-state Q out
proof (cases out)
case None
then show ?thesis
  using hit by simp
next
case (Some result-state)
then obtain result final-state where
  out-eq: out = Some (result, final-state)
by (cases result-state) simp
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have hit-some:
  staged-security-with-data-state-query-index-list-path-fresh Q
    (Some (((data, attacker-state), result), final-state))
using hit out-eq by simp
from staged-security-with-data-state-query-index-list-path-fresh-SomeE
  [OF hit-some]

```

```

obtain raw-idxs-hit query-idxs-hit where
  len-hit: length raw-idxs-hit = rounds
and query-hit-in: query-idxs-hit ∈ Q
and query-hit-eq:
  query-idxs-hit = map (λraw. index (to-nat raw)) raw-idxs-hit
and lookup-hit:
   $\bigwedge i. i < rounds \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data i)) =
        Some (raw-idxs-hit ! i))
and fresh-hit:
   $\bigwedge i. i < rounds \implies$ 
    fmlookup (HashMap ?s)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data i)) =
        None)
by blast
obtain fr f-fl f-final as dg fl where header-eq:
  header = (fr, f-fl, f-final, as, dg, fl)
by (cases header) auto
from suffix[unfolded out-eq header-eq verifier-after-composition-fri-def]
obtain final query-state where
  read-final:
    Some (final, query-state) ∈ set-dist (execute read prefix-state)
and query-out:
  Some (result, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
        query-state)
by (auto elim! set-dist-bindE)
from ntimes-verifier-query-rounds-outcome[OF query-out]
obtain raw-idxs query-idxs query-chunks where
  len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-chunks: length query-chunks = rounds
and tr-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and state-final:
  PState final-state =

```

```

    state-after-query-chunks (PState query-state) query-chunks rounds
and round-chunks:
   $\bigwedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
and lookups:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmllookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
    (state-after-query-chunks
    (PState query-state) query-chunks i)) =
    Some (raw-idxs ! i)
by blast
from read-outcome[OF read-final] have query-counter:
  PQueryCounter query-state = PQueryCounter prefix-state
by blast
have prefix-basic:
  length f-fl = ceil-log clength  $\wedge$ 
  length as = length spec  $\wedge$ 
  length fl = ceil-log (to-nat dg + 1)  $\wedge$ 
  PTranscript ?s =
    [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
    map snd fl @ PTranscript prefix-state  $\wedge$ 
  PState prefix-state =
    foldl concat
    (concat
    (foldl concat
    (concat
    (foldl concat (concat (PState ?s) fr) (map snd f-fl))
    f-final)
    as)
    dg)
    (map snd fl)  $\wedge$ 
  PQueryCounter prefix-state = PQueryCounter ?s
by (rule verifier-composition-fri-prefix-outcome-basicD
  [OF prefix[unfolded header-eq]])
have prefix-counter: PQueryCounter prefix-state = PQueryCounter ?s
using prefix-basic by simp
have local-counter: PQueryCounter query-state = 0
using query-counter prefix-counter
unfolding verifier-state-from-adversary-def by simp
have prefix-res:
  length f-fl = ceil-log clength  $\wedge$ 
  length as = length spec  $\wedge$ 
  length fl = ceil-log (to-nat dg + 1)  $\wedge$ 
  PTranscript ?s =
    [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
    map snd fl @ PTranscript prefix-state  $\wedge$ 
  PState prefix-state =

```

```

    foldl concat
      (concat
        (foldl concat
          (concat
            (foldl concat (concat (PState ?s) fr) (map snd f-fl))
            f-final)
          as)
        dg)
      (map snd fl)
  using prefix-basic by simp
from read-outcome[OF read-final] obtain rest-query where
  tr-prefix: PTranscript prefix-state = final # rest-query
and state-query:
  PState query-state = concat (PState prefix-state) final
and tr-read-query: PTranscript query-state = rest-query
by blast
have lookup-query:
   $\bigwedge i x. \text{fmlookup} (\text{HashMap } \text{query-state})$ 
    (QueryIndexChallenge i x) =
    fmlookup (HashMap ?s) (QueryIndexChallenge i x)
proof -
  have query-state-eq:
    query-state =
      prefix-state(PState := concat (PState prefix-state) final,
        PTranscript := rest-query)
  using read-nonempty-outcome[OF read-final tr-prefix] by simp
have hash-prefix:
   $\bigwedge i x. \text{fmlookup} (\text{HashMap } \text{prefix-state})$ 
    (QueryIndexChallenge i x) =
    fmlookup (HashMap ?s) (QueryIndexChallenge i x)
  by (rule verifier-composition-fri-prefix-preserves-query-lookup
    [OF prefix[unfolded header-eq]])
show  $\bigwedge i x. \text{fmlookup} (\text{HashMap } \text{query-state})$ 
  (QueryIndexChallenge i x) =
  fmlookup (HashMap ?s) (QueryIndexChallenge i x)
  using query-state-eq hash-prefix by simp
qed
have len-trace-actual: length (map snd f-fl) = ceil-log clength
  using prefix-res by simp
have len-alpha-actual: length as = length spec
  using prefix-res by simp
have len-comp-actual:
  length (map snd fl) = ceil-log (to-nat dg + 1)
  using prefix-res by simp
have transcript-prefix-actual:
  PTranscript ?s =
    [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
    map snd fl @ PTranscript prefix-state
  using prefix-res by simp

```

```

have header-actual:
  verifier-header-transcript ?s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
by (rule verifier-header-transcript-from-prefix-read
    [OF len-trace-actual len-alpha-actual len-comp-actual
      transcript-prefix-actual tr-prefix tr-read-query])
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
have header-unique:
  fr = staged-trace-root data ∧
  map snd f-fl = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  map snd fl = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  PTranscript query-state = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header-actual staged-header]
by simp
have prefix-state-query:
  PState query-state = staged-query-start-hash data
using state-query prefix-res header-unique
unfolding staged-query-start-hash-def
  staged-composition-fri-start-hash-def
  staged-trace-fri-start-hash-def
by simp
have chunks-eq: query-chunks = staged-query-chunks data
proof –
from checked-staged-transcript-program-query-chunks-match-verifier-lengths
  [OF builder]
obtain staged-query-idxs where
  staged-match:
    staged-query-chunks-match-verifier-lengths data staged-query-idxs
by blast
have query-idxs-len: length query-idxs = rounds
using query-idxs-eq len-raw by simp
have staged-match-query-idxs:
  staged-query-chunks-match-verifier-lengths data query-idxs
by (rule staged-query-chunks-match-verifier-lengths-transfer

```

```

    [OF staged-match query-idxs-len])
have parser-chunk-len:
   $\bigwedge i. i < \text{rounds} \implies$ 
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)
proof -
  fix i
  assume i-bound: i < rounds
  show
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)
  using verifier-query-round-chunk-length
    [OF round-chunks[OF i-bound]] header-unique
  by simp
qed
have concat-staged:
  List.concat query-chunks @ PTranscript final-state =
    List.concat (staged-query-chunks data)
  using tr-query header-unique by simp
then show ?thesis
  using query-chunks-eq-staged-if-matching-lengths
    [OF len-chunks concat-staged parser-chunk-len staged-match-query-idxs]
  by simp
qed
have raw-eq:
   $\bigwedge i. i < \text{rounds} \implies \text{raw-idxs} ! i = \text{raw-idxs-hit} ! i$ 
  using lookups lookup-hit local-counter prefix-state-query chunks-eq
  by simp
have raw-idxs-eq: raw-idxs = raw-idxs-hit
  by (rule nth-equalityI)
    (use len-raw len-hit raw-eq in simp-all)
have raw-in: raw-idxs  $\in$  query-index-raw-list-preimage Q
  unfolding query-index-raw-list-preimage-def
  using len-raw raw-idxs-eq query-hit-in query-hit-eq by blast
have local-fresh:
  query-index-list-path-fresh query-state rounds query-chunks raw-idxs
  unfolding query-index-list-path-fresh-def
  using len-raw len-chunks fresh-hit local-counter prefix-state-query
    chunks-eq lookup-query
  by simp
show ?thesis
  by (rule verifier-after-composition-fri-raw-list-path-hitI
    [where final=final and query-state=query-state and raws=raw-idxs
      and chunks=query-chunks and t=final-state and results=result,
      OF read-final raw-in len-raw len-chunks state-final local-fresh

```

lookups out-eq])

qed

lemma *verifier-after-composition-fri-query-index-list-path-fresh-product-bound-after-prefix:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *subset: $Q \subseteq \text{fri-query-index-list-space}$*
and *builder:*
Some (data, attacker-state) $\in$
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix:*
Some (header, prefix-state) $\in$
set-dist
(execute verifier-composition-fri-prefix
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))))
shows
wp-event (verifier-after-composition-fri header)
($\lambda out.$
staged-security-with-data-state-query-index-list-path-fresh Q
(case out of
None $\Rightarrow$ None
| Some (result, final-state) $\Rightarrow$
Some (((data, attacker-state), result), final-state)))
prefix-state
 *$\leq \text{nnreal (card Q) *}$*
(1 / nnreal (card query-sample-space)) $\wedge$ rounds
proof –
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
let *?P =*
 $\lambda out.$
staged-security-with-data-state-query-index-list-path-fresh Q
(case out of
None $\Rightarrow$ None
| Some (result, final-state) $\Rightarrow$
Some (((data, attacker-state), result), final-state)))
have *event-mono:*
wp-event (verifier-after-composition-fri header)
?P prefix-state
 $\leq$
wp-event (verifier-after-composition-fri header)
(verifier-after-composition-fri-raw-list-path-hit prefix-state Q)
prefix-state
proof (rule *wp-event-mono-on-support*)
fix *out*

```

assume suffix:
  out ∈
    set-dist
      (execute (verifier-after-composition-fri header) prefix-state)
and hit:
  staged-security-with-data-state-query-index-list-path-fresh Q
    (case out of
      None ⇒ None
      | Some (result, final-state) ⇒
        Some (((data, attacker-state), result), final-state))
show verifier-after-composition-fri-raw-list-path-hit prefix-state Q out
by (rule
  verifier-after-composition-fri-query-index-list-path-fresh-imp-raw-hit-after-prefix
  [OF wf controlled builder prefix suffix hit])
qed
also have ... ≤
  nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds
by (rule wp-verifier-after-composition-fri-raw-list-path-product-bound
  [OF subset])
finally show ?thesis .
qed

lemma verify-monad-query-index-list-path-fresh-product-bound-after-builder:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and subset: Q ⊆ fri-query-index-list-space
and builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
shows
  wp-event verify-monad
    (λout.
      staged-security-with-data-state-query-index-list-path-fresh Q
      (case out of
        None ⇒ None
        | Some (result, final-state) ⇒
          Some (((data, attacker-state), result), final-state)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
    ≤ nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds
proof –
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
note verify-decomp = verify-monad-composition-fri-decomposition

```



```

show ?thesis
  unfolding verify-decomp
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg$  staged-security-with-data-state-query-index-list-path-fresh Q
    (case None of None  $\Rightarrow$  None
      | Some (result, final-state)  $\Rightarrow$ 
        Some (((data, attacker-state), result), final-state))
    by simp
next
  fix header prefix-state
  assume prefix:
    Some (header, prefix-state)  $\in$ 
      set-dist (execute verifier-composition-fri-prefix ?s)
  show
    wp-event (verifier-after-composition-fri header)
      ( $\lambda$ out.
        staged-security-with-data-state-query-index-list-path-fresh Q
          (case out of
            None  $\Rightarrow$  None
            | Some (result, final-state)  $\Rightarrow$ 
              Some (((data, attacker-state), result), final-state))))
        prefix-state
       $\leq$  nnreal (card Q) *
        (1 / nnreal (card query-sample-space))  $\wedge$  rounds
    by (rule
      verifier-after-composition-fri-query-index-list-path-fresh-product-bound-after-prefix
      [OF wf controlled subset builder prefix])
qed
qed

lemma checked-staged-security-with-data-state-query-index-list-path-fresh-product-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and subset:  $Q \subseteq$  fri-query-index-list-space
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-index-list-path-fresh Q)
      adversary-initial-state  $\leq$ 
        nnreal (card Q) *
          (1 / nnreal (card query-sample-space))  $\wedge$  rounds
proof -
  let ?P = staged-security-with-data-state-query-index-list-path-fresh Q
  have none:  $\neg$  ?P None
    by simp
  have cont-bound:
     $\bigwedge$ data attacker-state.
      Some (data, attacker-state)  $\in$ 
        set-dist
          (execute (checked-staged-transcript-program A)

```

```

      adversary-initial-state)  $\Rightarrow$ 
wp-event verify-monad
  ( $\lambda$ out.
    ?P
    (case out of
      None  $\Rightarrow$  None
      | Some (result, final-state)  $\Rightarrow$ 
        Some (((data, attacker-state), result), final-state)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
     $\leq$  nnreal (card Q) *
      (1 / nnreal (card query-sample-space))  $\wedge$  rounds
  by (rule verify-monad-query-index-list-path-fresh-product-bound-after-builder
    [OF wf controlled subset])
show ?thesis
  by (rule checked-staged-security-with-data-state-bound-from-data-cont
    [OF none cont-bound])
qed

```

lemma *checked-staged-security-with-data-state-query-index-list-hit-exact-product-bound:*

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and subset: $Q \subseteq \text{fri-query-index-list-space}$

shows

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-query-index-list-set-hit Q)
  adversary-initial-state  $\leq$ 
  nnreal (card Q) *
    (1 / nnreal (card query-sample-space))  $\wedge$  rounds +
  hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
  by (rule
    checked-staged-security-with-data-state-query-index-list-hit-bound-from-path-fresh
    [OF wf controlled
      checked-staged-security-with-data-state-query-index-list-path-fresh-product-bound
      [OF wf controlled subset]])

```

lemma *checked-staged-security-trace-fri-query-index-list-set-hit-exact-product-bound:*

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and subset: $Q \subseteq \text{fri-query-index-list-space}$

shows

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda$ s. trace-fri-query-index-list-set-hit s Q))
  adversary-initial-state  $\leq$ 
  nnreal (card Q) *

```

$(1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds} +$
 $\text{hash-relation-budget-value}$
 $(\text{query-index-raw-list-relation-fiber-bound } Q)$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget})$
by (*rule*
 $\text{checked-staged-security-trace-fri-query-index-list-set-hit-bound-from-path-fresh}$
 $[\text{OF } \text{wf controlled}$
 $\text{checked-staged-security-with-data-state-query-index-list-path-fresh-product-bound}$
 $[\text{OF } \text{wf controlled subset}]])$

lemma *checked-staged-security-composition-fri-query-index-list-set-hit-exact-product-bound:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *subset: $Q \subseteq \text{fri-query-index-list-space}$*

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-query-index-list-set-hit } s \ Q))$
 $\text{adversary-initial-state} \leq$
 $\text{nnreal } (\text{card } Q) *$
 $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds} +$
 $\text{hash-relation-budget-value}$
 $(\text{query-index-raw-list-relation-fiber-bound } Q)$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget})$

by (*rule*
 $\text{checked-staged-security-composition-fri-query-index-list-set-hit-bound-from-path-fresh}$
 $[\text{OF } \text{wf controlled}$
 $\text{checked-staged-security-with-data-state-query-index-list-path-fresh-product-bound}$
 $[\text{OF } \text{wf controlled subset}]])$

lemma *checked-staged-security-trace-fri-query-challenge-pair-set-hit-exact-product-bound:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *projection: $\text{fst } P \subseteq Q$*
and *subset: $Q \subseteq \text{fri-query-index-list-space}$*

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ P))$
 $\text{adversary-initial-state} \leq$
 $\text{nnreal } (\text{card } Q) *$
 $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds} +$
 $\text{hash-relation-budget-value}$
 $(\text{query-index-raw-list-relation-fiber-bound } Q)$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget})$

by (*rule*

checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-path-fresh
 $[OF \text{ wf controlled projection}$
checked-staged-security-with-data-state-query-index-list-path-fresh-product-bound
 $[OF \text{ wf controlled subset}]])$

lemma *checked-staged-security-composition-fri-query-challenge-pair-set-hit-exact-product-bound:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *projection: $\bigwedge dg. \text{fst } dg \subseteq Q$*
and *subset: $Q \subseteq \text{fri-query-index-list-space}$*

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
($\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \text{ } P$))
adversary-initial-state $\leq$
*nnreal (card Q) **
(1 / nnreal (card query-sample-space)) $\wedge$ rounds +
hash-relation-budget-value
(query-index-raw-list-relation-fiber-bound Q)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)

by (rule

checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-path-fresh
 $[OF \text{ wf controlled projection}$
checked-staged-security-with-data-state-query-index-list-path-fresh-product-bound
 $[OF \text{ wf controlled subset}]])$

end

end

theory *Soundness-FRI-Query-Fiber-Exact-Staged-Bounds*

imports

Soundness-FRI-Query-Fiber-Staged-Bounds
Soundness-FRI-Query-List-Exact-Product

begin

Exact query-list staged bounds for the outside-query-cover branch of sampled FRI query events.

This layer keeps the challenge-cover split and uses exact product query-list accounting for the outside branch.

context *soundness*

begin

lemma *checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-list-exact*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: finite B*

and *fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B$))
adversary-initial-state $\leq F$
and *projection*:
 $\text{fst} \ ' \ (P \cap (UNIV \times (- \ B))) \subseteq Q$
and *subset*: $Q \subseteq \text{fri-query-index-list-space}$
shows
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ P$))
adversary-initial-state $\leq$
 $F +$
hash-relation-budget-value (*card* *B* * *ceil-log clength*)
 (*staged-attacker-query-budget* *budgets* + *staged-challenge-query-budget*) +
 (*nnreal* (*card* *Q*) *
 ($1 / \text{nnreal} (\text{card } \text{query-sample-space})$) $\wedge$ *rounds* +
hash-relation-budget-value
 (*query-index-row-list-relation-fiber-bound* *Q*)
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget))
proof –
let *?Outside* = $P \cap (UNIV \times (- \ B))$
have *split*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ P$))
adversary-initial-state $\leq$
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 ($\lambda out. \text{staged-security-with-data-state-verifier-event}$
 ($\lambda s. \text{trace-fri-challenge-list-set-hit } s \ B$) *out* $\vee$
staged-security-with-data-state-verifier-event
 ($\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ ?Outside$) *out*)
adversary-initial-state
proof (*rule wp-event-mono-on-support*)
fix *out*
assume *hit*:
staged-security-with-data-state-verifier-event
 ($\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ P$) *out*
then show
staged-security-with-data-state-verifier-event
 ($\lambda s. \text{trace-fri-challenge-list-set-hit } s \ B$) *out* $\vee$
staged-security-with-data-state-verifier-event
 ($\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ ?Outside$) *out*
proof (*cases out*)
case *None*
then show *?thesis*

```

    using hit unfolding staged-security-with-data-state-verifier-event-def
    by simp
next
case (Some full)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases full) (auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have pair-hit:
  trace-fri-query-challenge-pair-set-hit ?s P
  (Some (result, final-state))
  using hit out-eq
  unfolding staged-security-with-data-state-verifier-event-def by simp
from pair-hit obtain trace-roots trace-bs trace-final dg
  composition-roots composition-bs composition-final fri-query-idxs
  trace-round-layers composition-round-layers
where openings:
  accepted-fri-opening-transcript ?s (Some (result, final-state))
  trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs
  trace-round-layers composition-round-layers
and pair: (fri-query-idxs, trace-bs) ∈ P
  unfolding trace-fri-query-challenge-pair-set-hit-def by blast
show ?thesis
proof (cases trace-bs ∈ B)
case True
have challenges:
  accepted-fri-challenges ?s (Some (result, final-state))
  trace-bs dg composition-bs
  by (rule accepted-fri-opening-transcript-challenges[OF openings])
have trace-fri-challenge-list-set-hit ?s B
  (Some (result, final-state))
  unfolding trace-fri-challenge-list-set-hit-def
  using challenges True by blast
then show ?thesis
  using out-eq unfolding staged-security-with-data-state-verifier-event-def
  by simp
next
case False
have outside-pair: (fri-query-idxs, trace-bs) ∈ ?Outside
  using pair False by simp
have trace-fri-query-challenge-pair-set-hit ?s ?Outside
  (Some (result, final-state))
  unfolding trace-fri-query-challenge-pair-set-hit-def
  using openings outside-pair by blast
then show ?thesis
  using out-eq unfolding staged-security-with-data-state-verifier-event-def

```

```

      by simp
    qed
  qed
  qed
  also have ... ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-challenge-list-set-hit s B))
      adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-query-challenge-pair-set-hit s ?Outside))
      adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤
    (F +
      hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
    (nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds +
      hash-relation-budget-value
        (query-index-raw-list-relation-fiber-bound Q)
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget))
  proof (rule add-mono)
    show wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-challenge-list-set-hit s B))
      adversary-initial-state
      ≤ F +
        hash-relation-budget-value (card B * ceil-log clength)
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget)
    by (rule checked-staged-security-trace-fri-challenge-list-set-hit-bound
      [OF wf controlled finite-B fresh-bound])
    show wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-query-challenge-pair-set-hit s ?Outside))
      adversary-initial-state
      ≤ nnreal (card Q) *
        (1 / nnreal (card query-sample-space)) ^ rounds +
        hash-relation-budget-value
          (query-index-raw-list-relation-fiber-bound Q)
          (staged-attacker-query-budget budgets +
            staged-challenge-query-budget)
    by (rule
      checked-staged-security-trace-fri-query-challenge-pair-set-hit-exact-product-bound
      [OF wf controlled projection subset])

```

qed
 finally show ?thesis
 by (simp add: add.assoc)
 qed

lemma *checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-his*

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *finite-B*: $\bigwedge dg. \text{finite } (B \ dg)$
and *fresh-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$))
adversary-initial-state $\leq F$

and *projection*:

$\bigwedge dg. \text{fst } ' (P \ dg \cap (UNIV \times (- \ B \ dg))) \subseteq Q$

and *subset*: $Q \subseteq \text{fri-query-index-list-space}$

shows

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P$))
adversary-initial-state $\leq$
 $F +$
hash-relation-budget-value
 $(\sum dg \in (UNIV :: 'f \ \text{set}).$
 $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget}) +$
 $(\text{nnreal } (\text{card } Q) * (1 / \text{nnreal } (\text{card } \text{query-sample-space})) ^ \text{rounds} +$
hash-relation-budget-value
 $(\text{query-index-raw-list-relation-fiber-bound } Q)$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget}))$

proof –

let *?Outside* = $\lambda dg. P \ dg \cap (UNIV \times (- \ B \ dg))$

have *split*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P$))
adversary-initial-state $\leq$
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 ($\lambda out.$
 $\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-challenge-list-set-hit } s \ B) \ out \ \vee$
 $\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ ?Outside) \ out)$
adversary-initial-state

proof (rule *wp-event-mono-on-support*)


```

fix out
assume hit:
  staged-security-with-data-state-verifier-event
  ( $\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P$ ) out
then show
  staged-security-with-data-state-verifier-event
  ( $\lambda s. \text{composition-fri-challenge-list-set-hit } s \ B$ ) out  $\vee$ 
  staged-security-with-data-state-verifier-event
  ( $\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ ?\text{Outside}$ ) out
proof (cases out)
  case None
  then show ?thesis
    using hit unfolding staged-security-with-data-state-verifier-event-def
    by simp
next
  case (Some full)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases full) (auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have pair-hit:
    composition-fri-query-challenge-pair-set-hit ?s P
    (Some (result, final-state))
    using hit out-eq
    unfolding staged-security-with-data-state-verifier-event-def by simp
  from pair-hit obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers
  where openings:
    accepted-fri-opening-transcript ?s (Some (result, final-state))
    trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers
  and pair: (fri-query-idxs, composition-bs)  $\in P$  fri-dg
    unfolding composition-fri-query-challenge-pair-set-hit-def by blast
  show ?thesis
  proof (cases composition-bs  $\in B$  fri-dg)
    case True
    have challenges:
      accepted-fri-challenges ?s (Some (result, final-state))
      trace-bs fri-dg composition-bs
      by (rule accepted-fri-opening-transcript-challenges[OF openings])
    have composition-fri-challenge-list-set-hit ?s B
      (Some (result, final-state))
      unfolding composition-fri-challenge-list-set-hit-def
      using challenges True by blast
    then show ?thesis

```

```

    using out-eq unfolding staged-security-with-data-state-verifier-event-def
    by simp
next
case False
have outside-pair:
  (fri-query-idxs, composition-bs) ∈ ?Outside fri-dg
  using pair False by simp
have composition-fri-query-challenge-pair-set-hit ?s ?Outside
  (Some (result, final-state))
  unfolding composition-fri-query-challenge-pair-set-hit-def
  using openings outside-pair by blast
then show ?thesis
  using out-eq unfolding staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
qed
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-challenge-list-set-hit s B))
    adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-query-challenge-pair-set-hit s ?Outside))
    adversary-initial-state
  by (rule wp-event-union-bound)
also have ... ≤
  (F +
    hash-relation-budget-value
      (∑ dg ∈ (UNIV :: 'f set).
        card (B dg) * ceil-log (maxDegree + 1))
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
    (nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds +
      hash-relation-budget-value
        (query-index-raw-list-relation-fiber-bound Q)
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget))
  proof (rule add-mono)
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-challenge-list-set-hit s B))
    adversary-initial-state
    ≤ F +
      hash-relation-budget-value
        (∑ dg ∈ (UNIV :: 'f set).
          card (B dg) * ceil-log (maxDegree + 1))

```

```

      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)
    by (rule
        checked-staged-security-composition-fri-challenge-list-set-hit-bound
        [OF wf controlled finite-B fresh-bound])
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     (λs. composition-fri-query-challenge-pair-set-hit s ?Outside))
  adversary-initial-state
    ≤ nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds +
    hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound Q)
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)
  by (rule
      checked-staged-security-composition-fri-query-challenge-pair-set-hit-exact-product-bound
      [OF wf controlled projection subset])
qed
finally show ?thesis
  by (simp add: add.assoc)
qed

lemma checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-outsides
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B: finite B
  and fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       (λs. trace-fri-challenge-list-fresh-hit s B))
    adversary-initial-state ≤ F
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-sampled-query-bad-candidate)
    adversary-initial-state ≤
      F +
      hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets + staged-challenge-query-budget) +
      (nnreal
       (card
        (generic-fri-sampled-query-outside-query-cover
         trace-table-low-degree (Not ∘ trace-table-low-degree)
         (clength - 1) B)) *
        (1 / nnreal (card query-sample-space)) ^ rounds +
       hash-relation-budget-value
        (query-index-raw-list-relation-fiber-bound
         (generic-fri-sampled-query-outside-query-cover

```

```

      trace-table-low-degree (Not  $\circ$  trace-table-low-degree)
      (clength - 1) B))
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget))
proof -
  let ?P =
    trace-fri-sampled-query-bad-pair-union  $\cap$ 
    (fri-query-index-list-space  $\times$  UNIV)
  let ?Q =
    generic-fri-sampled-query-outside-query-cover trace-table-low-degree
    (Not  $\circ$  trace-table-low-degree) (clength - 1) B
  have sampled-to-pair:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-sampled-query-bad-candidate)
    adversary-initial-state  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     ( $\lambda s$ . trace-fri-query-challenge-pair-set-hit s ?P))
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
     intro:
      trace-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit
      split: option.splits prod.splits)
  also have ...  $\leq$ 
    F +
    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget) +
    (nnreal (card ?Q) *
     (1 / nnreal (card query-sample-space))  $\wedge$  rounds +
     hash-relation-budget-value
     (query-index-row-list-relation-fiber-bound ?Q)
     (staged-attacker-query-budget budgets +
      staged-challenge-query-budget))
  by (rule
    checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-list-exact
    [OF wf controlled finite-B fresh-bound
     trace-fri-sampled-query-outside-projection-subset
     generic-fri-sampled-query-outside-query-cover-subset])
  finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B:  $\bigwedge dg$ . finite (B dg)
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (staged-security-with-data-state-verifier-event
      ( $\lambda s$ . composition-fri-challenge-list-fresh-hit  $s$   $B$ ))
    adversary-initial-state  $\leq F$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state  $A$ )
      (staged-security-with-data-state-verifier-event
        composition-fri-sampled-query-bad-candidate)
    adversary-initial-state  $\leq$ 
       $F +$ 
      hash-relation-budget-value
        ( $\sum dg \in (UNIV :: 'f \text{ set})$ .
          card ( $B \ dg$ ) * ceil-log (maxDegree + 1))
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget) +
      (nnreal (card (composition-fri-sampled-query-outside-query-cover  $B$ )) *
        (1 / nnreal (card query-sample-space)) ^ rounds +
      hash-relation-budget-value
        (query-index-raw-list-relation-fiber-bound
          (composition-fri-sampled-query-outside-query-cover  $B$ ))
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget))
  proof -
  let ?P =
     $\lambda dg$ . composition-fri-sampled-query-bad-pair-union  $dg \cap$ 
      (fri-query-index-list-space  $\times UNIV$ )
  let ?Q = composition-fri-sampled-query-outside-query-cover  $B$ 
  have sampled-to-pair:
    wp-event (checked-staged-security-experiment-with-data-state  $A$ )
      (staged-security-with-data-state-verifier-event
        composition-fri-sampled-query-bad-candidate)
    adversary-initial-state  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state  $A$ )
      (staged-security-with-data-state-verifier-event
        ( $\lambda s$ . composition-fri-query-challenge-pair-set-hit  $s$  ?P))
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      intro:
        composition-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit
        split: option.splits prod.splits)
  also have ...  $\leq$ 
     $F +$ 
    hash-relation-budget-value
      ( $\sum dg \in (UNIV :: 'f \text{ set})$ .
        card ( $B \ dg$ ) * ceil-log (maxDegree + 1))
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget) +
    (nnreal (card ?Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds +

```

```

    hash-relation-budget-value
    (query-index-row-list-relation-fiber-bound ?Q)
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget))
  by (rule
    checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-list-exa
    [OF wf controlled finite-B fresh-bound
     composition-fri-sampled-query-outside-projection-subset
     composition-fri-sampled-query-outside-query-cover-subset])
  finally show ?thesis .
qed

end

end

```

theory *Soundness-FRI-Query-Challenge-Fresh-Split*
imports *Soundness-FRI-Query-Fiber-Exact-Staged-Bounds*
begin

Staged split for FRI query/challenge pair events.

The exact pair-fraction route must keep the query/challenge pair event until the challenge-list probability has been accounted for. This layer separates only the challenge prequery side event, leaving the fresh branch as the original pair hit conjoined with path freshness of the relevant FRI challenge keys.

context *soundness*
begin

lemma *verifier-after-trace-fri-concrete-header-transcript*:
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes *prefix*:
 $\text{Some } ((fr, f-fl), \text{prefix-state}) \in$
 $\text{set-dist } (\text{execute verifier-trace-fri-prefix } s)$
and *suffix*:
 $\text{Some } (\text{result}, \text{final-state}) \in$
 set-dist
 $(\text{execute } (\text{verifier-after-trace-fri } (fr, f-fl)) \text{ prefix-state})$
obtains $f\text{-final as dg fl final query-state}$ **where**
 $\text{verifier-header-transcript } s \text{ fr } (\text{map snd } f\text{-fl}) \text{ f-final as dg}$
 $(\text{map snd fl}) \text{ final } (P\text{Transcript query-state})$
 $\text{Some } (\text{result}, \text{final-state}) \in$
 set-dist
 $(\text{execute}$
 $(\text{ntimes}$
 $(\text{verifier-query-round-program } fr \text{ f-fl } f\text{-final as fl final})$
 $\text{rounds})$
 $\text{query-state})$
proof –

```

have prefix-res:
  PTranscript s = [fr] @ map snd f-fl @ PTranscript prefix-state ∧
  PState prefix-state =
    foldl concat (concat (PState s) fr) (map snd f-fl)
  using verifier-trace-fri-prefix-outcome[OF prefix] by simp
have len-f-fl: length f-fl = ceil-log clength
  using verifier-trace-fri-prefix-outcome[OF prefix] by simp
from suffix obtain f-final s3 as s4 dg s5 s6 fl s7 final query-state where
  read-trace-final:
    Some (f-final, s3) ∈ set-dist (execute read prefix-state)
  and alpha-out:
    Some (as, s4) ∈
      set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
  and read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
  and degree-assert:
    Some ((), s6) ∈
      set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
  and comp-fri:
    Some (fl, s7) ∈
      set-dist (execute
        (ntimes receive-composition-fri-commits
          (ceil-log (to-nat dg + 1))) s6)
  and read-final:
    Some (final, query-state) ∈ set-dist (execute read s7)
  and query-out:
    Some (result, final-state) ∈
      set-dist (execute
        (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
        query-state)
  unfolding verifier-after-trace-fri-def
  by (auto elim!: set-dist-bindE)
from read-outcome[OF read-trace-final] obtain rest3 where
  tr-prefix: PTranscript prefix-state = f-final # rest3
  and tr-s3: PTranscript s3 = rest3
  by blast
have alpha-res:
  length as = length spec ∧
  PTranscript s3 = as @ PTranscript s4
  using mmap-alpha-round-outcome[OF alpha-out] by simp
from read-outcome[OF read-dg] obtain rest5 where
  tr-s4: PTranscript s4 = dg # rest5
  and tr-s5: PTranscript s5 = rest5
  by blast
have s6-eq: s6 = s5
  using degree-assert unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
have comp-res:
  length fl = ceil-log (to-nat dg + 1) ∧

```

PTranscript s6 = map snd fl @ PTranscript s7
using *ntimes-receive-composition-fri-commits-outcome[OF comp-fri]*
by *simp*
from *read-outcome[OF read-final]* **obtain** *rest-query* **where**
tr-s7: PTranscript s7 = final # rest-query
and *tr-query: PTranscript query-state = rest-query*
by *blast*
have *header:*
verifier-header-transcript s fr (map snd f-fl) f-final as dg
(map snd fl) final (PTranscript query-state)
using *prefix-res len-f-fl tr-prefix tr-s3 alpha-res tr-s4 tr-s5 s6-eq comp-res*
tr-s7 tr-query
unfolding *verifier-header-transcript-def verifier-header-messages-def*
by *simp*
show *thesis*
by *(rule that[OF header query-out])*
qed

lemma *verifier-after-trace-fri-header-agrees-with-accepted-challenges:*

assumes *prefix:*
Some ((fr, f-fl), prefix-state) ∈
set-dist (execute verifier-trace-fri-prefix s)
and *suffix:*
Some (result, final-state) ∈
set-dist
(execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
and *accepted:*
accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
comp-bs

shows *trace-bs = map fst f-fl*

proof —

from *verifier-after-trace-fri-accepted-fri-challenges[OF prefix suffix]*
obtain *dg' comp-bs'* **where** *replay:*
accepted-fri-challenges s (Some (result, final-state))
(map fst f-fl) dg' comp-bs'
by *blast*
show *?thesis*
using *accepted-fri-challenges-unique[OF replay accepted]* **by** *simp*
qed

lemma *verifier-after-trace-fri-header-roots-agree-with-fresh-hit:*

assumes *prefix:*
Some ((fr, f-fl), prefix-state) ∈
set-dist (execute verifier-trace-fri-prefix s)
and *suffix:*
Some (result, final-state) ∈
set-dist
(execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
and *fresh:*


```

    trace-fri-challenge-list-fresh-hit s B
    (Some (result, final-state))
shows trace-fri-challenge-path-fresh s fr (map snd f-fl)
    (length (map snd f-fl))
proof –
from fresh obtain trace-bs dg comp-bs fr' trace-roots' trace-final' as'
    composition-roots' final' rest' where
    accepted:
    accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
    comp-bs
and header-hit:
    verifier-header-transcript s fr' trace-roots' trace-final' as' dg
    composition-roots' final' rest'
and fresh-path:
    trace-fri-challenge-path-fresh s fr' trace-roots'
    (length trace-roots')
unfolding trace-fri-challenge-list-fresh-hit-def by blast
from verifier-after-trace-fri-accepted-fri-challenges[OF prefix suffix]
obtain dg0 comp-bs0 where replay:
    accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg0 comp-bs0
by blast
have dg-eq: dg = dg0
    using accepted-fri-challenges-unique[OF accepted replay] by simp
from verifier-after-trace-fri-concrete-header-transcript[OF prefix suffix]
obtain f-final0::'f and as0:: 'f list and dg1::'f and fl0:: ('f × 'f) list and final0:: 'f and
    query-state0:: ('f, 'a) protocol-channel-scheme where
    header-concrete:
    verifier-header-transcript s fr (map snd f-fl) f-final0 as0 dg1
    (map snd fl0) final0 (PTranscript query-state0)
by metis
from accepted-fri-challenges-headerE[OF replay]
obtain fr0::'f and f-fl0:: ('f × 'f) list and f-final0'::'f and as0':: 'f list and fl0':: ('f × 'f)
    list and final0'::'f and rest0':: 'f list where
    header-replay:
    verifier-header-transcript s fr0 (map snd f-fl0) f-final0' as0'
    dg0 (map snd fl0') final0' rest0'
by metis
have replay-unique:
    fr0 = fr ∧ map snd f-fl0 = map snd f-fl ∧ dg0 = dg1
    using verifier-header-transcript-unique[OF header-replay header-concrete]
by simp
from accepted-fri-challenges-headerE[OF accepted]
obtain fr1::'f and f-fl1:: ('f × 'f) list and f-final1::'f and as1:: 'f list and fl1:: ('f × 'f) list
and final1::'f and rest1 where
    header-accepted:
    verifier-header-transcript s fr1 (map snd f-fl1) f-final1 as1 dg
    (map snd fl1) final1 rest1
by metis

```

```

have accepted-unique-header:
  fr1 = fr0  $\wedge$  map snd f-fl1 = map snd f-fl0
  using verifier-header-transcript-unique[OF header-accepted header-replay]
    dg-eq
  by simp
have hit-unique:
  fr' = fr1  $\wedge$  trace-roots' = map snd f-fl1
  using verifier-header-transcript-unique[OF header-hit header-accepted]
    by simp
show ?thesis
  using fresh-path hit-unique accepted-unique-header replay-unique by simp
qed

```

lemma *verifier-after-composition-fri-concrete-header-transcript*:

fixes *s* :: ('f, 'a) *protocol-channel-scheme*

assumes *prefix*:

Some ((*fr*, *f-fl*, *f-final*, *as*, *dg*, *fl*), *prefix-state*) $\in$
set-dist (*execute verifier-composition-fri-prefix s*)

and *suffix*:

Some (*result*, *final-state*) $\in$
set-dist
(*execute*
(*verifier-after-composition-fri*
(*fr*, *f-fl*, *f-final*, *as*, *dg*, *fl*))
prefix-state)

obtains *final query-state* **where**

verifier-header-transcript s fr (*map snd f-fl*) *f-final as dg*
(*map snd fl*) *final* (*PTranscript query-state*)
Some (*result*, *final-state*) $\in$
set-dist
(*execute*
(*ntimes*
(*verifier-query-round-program fr f-fl f-final as fl final*)
rounds)
query-state)

proof –

have *prefix-res*:

PTranscript s =
[*fr*] @ *map snd f-fl* @ [*f-final*] @ *as* @ [*dg*] @
map snd fl @ *PTranscript prefix-state*

using *verifier-composition-fri-prefix-outcome*[*OF prefix*] **by** *simp*

from *suffix* **obtain** *final query-state* **where**

read-final:
Some (*final*, *query-state*) $\in$
set-dist (*execute read prefix-state*)

and *query-out*:

Some (*result*, *final-state*) $\in$
set-dist
(*execute*

```

      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      query-state)
  unfolding verifier-after-composition-fri-def
  by (auto elim!: set-dist-bindE)
from read-outcome[OF read-final] obtain rest-query where
  tr-prefix: PTranscript prefix-state = final # rest-query
  and tr-query: PTranscript query-state = rest-query
  by blast
have len-f-fl: length f-fl = ceil-log clength
  using verifier-composition-fri-prefix-outcome[OF prefix] by simp
have len-as: length as = length spec
  using verifier-composition-fri-prefix-outcome[OF prefix] by simp
have len-fl: length fl = ceil-log (to-nat dg + 1)
  using verifier-composition-fri-prefix-outcome[OF prefix] by simp
have header:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
  using prefix-res len-f-fl len-as len-fl tr-prefix tr-query
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
show thesis
  by (rule that[OF header query-out])
qed

```

lemma *verifier-after-composition-fri-header-agrees-with-accepted-challenges:*

```

  assumes prefix:
    Some ((fr, f-fl, f-final, as, dg, fl), prefix-state) ∈
      set-dist (execute verifier-composition-fri-prefix s)
  and suffix:
    Some (result, final-state) ∈
      set-dist
        (execute
          (verifier-after-composition-fri
            (fr, f-fl, f-final, as, dg, fl))
          prefix-state)
  and accepted:
    accepted-fri-challenges s (Some (result, final-state)) trace-bs dg'
      comp-bs
  shows dg' = dg ∧ comp-bs = map fst fl
proof -
  have replay:
    accepted-fri-challenges s (Some (result, final-state))
      (map fst f-fl) dg (map fst fl)
  by (rule verifier-after-composition-fri-accepted-fri-challenges
      [OF prefix suffix])
show ?thesis
  using accepted-fri-challenges-unique[OF replay accepted] by simp

```

qed

lemma *verifier-after-composition-fri-header-roots-agree-with-fresh-hit*:

assumes *prefix*:

*Some ((fr, f-fl, f-final, as, dg, fl), prefix-state) ∈
set-dist (execute verifier-composition-fri-prefix s)*

and *suffix*:

*Some (result, final-state) ∈
set-dist
(execute
(verifier-after-composition-fri
(fr, f-fl, f-final, as, dg, fl))
prefix-state)*

and *fresh*:

*composition-fri-challenge-list-fresh-hit s B
(Some (result, final-state))*

shows *composition-fri-challenge-path-fresh s fr (map snd f-fl) f-final
as dg (map snd fl) (length (map snd fl))*

proof —

from *fresh* **obtain** *trace-bs dg' comp-bs fr' trace-roots' trace-final' as'
composition-roots' final' rest'* **where**

accepted:

*accepted-fri-challenges s (Some (result, final-state)) trace-bs dg'
comp-bs*

and *header-hit*:

*verifier-header-transcript s fr' trace-roots' trace-final' as' dg'
composition-roots' final' rest'*

and *fresh-path*:

*composition-fri-challenge-path-fresh s fr' trace-roots' trace-final'
as' dg' composition-roots' (length composition-roots')*

unfolding *composition-fri-challenge-list-fresh-hit-def* **by** *blast*

from *verifier-after-composition-fri-concrete-header-transcript[OF prefix suffix]*

obtain *final0:: 'f* **and** *query-state0:: ('f, 'a) protocol-channel-scheme* **where**
header-concrete:

*verifier-header-transcript s fr (map snd f-fl) f-final as dg
(map snd fl) final0 (PTranscript query-state0)*

by *blast*

have *fr' = fr* $\wedge$

trace-roots' = map snd f-fl $\wedge$

trace-final' = f-final $\wedge$

as' = as $\wedge$

dg' = dg $\wedge$

composition-roots' = map snd fl

using *verifier-header-transcript-unique[OF header-hit header-concrete]*

by *simp*

then show *?thesis*

using *fresh-path* **by** *simp*

qed

definition *staged-security-trace-fri-pair-challenge-fresh*

$:: (\text{nat list} \times 'f \text{ list}) \text{ set} \Rightarrow 'f \text{ list set} \Rightarrow$
 $((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times$
 $\text{unit list}) \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

staged-security-trace-fri-pair-challenge-fresh $P B \text{ out} \longleftrightarrow$
staged-security-with-data-state-verifier-event
 $(\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s P) \text{ out} \wedge$
staged-security-with-data-state-verifier-event
 $(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s B) \text{ out}$

definition *staged-security-composition-fri-pair-challenge-fresh*

$:: ('f \Rightarrow (\text{nat list} \times 'f \text{ list}) \text{ set}) \Rightarrow$
 $('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times$
 $\text{unit list}) \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

staged-security-composition-fri-pair-challenge-fresh $P B \text{ out} \longleftrightarrow$
staged-security-with-data-state-verifier-event
 $(\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s P) \text{ out} \wedge$
staged-security-with-data-state-verifier-event
 $(\lambda s. \text{composition-fri-challenge-list-fresh-hit } s B) \text{ out}$

lemma *checked-staged-security-trace-fri-query-challenge-pair-set-hit-split-challenge-fresh:*

assumes *projection:* $\text{snd} \text{ ' } P \subseteq B$

shows

wp-event (*checked-staged-security-experiment-with-data-state* A)
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s P))$
adversary-initial-state $\leq$
wp-event (*checked-staged-security-experiment-with-data-state* A)
 $(\text{staged-security-trace-fri-pair-challenge-fresh } P B)$
adversary-initial-state $+$
wp-event (*checked-staged-security-experiment-with-data-state* A)
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-challenge-list-prequery-hit } s B))$
adversary-initial-state

proof –

have *event-mono:*

wp-event (*checked-staged-security-experiment-with-data-state* A)
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s P))$
adversary-initial-state $\leq$
wp-event (*checked-staged-security-experiment-with-data-state* A)
 $(\lambda \text{out.}$
 $\text{staged-security-trace-fri-pair-challenge-fresh } P B \text{ out} \vee$
 $\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-challenge-list-prequery-hit } s B) \text{ out})$
adversary-initial-state

```

proof (rule wp-event-mono-on-support)
  fix out
  assume hit:
    staged-security-with-data-state-verifier-event
    ( $\lambda s.$  trace-fri-query-challenge-pair-set-hit  $s$   $P$ ) out
  show
    staged-security-trace-fri-pair-challenge-fresh  $P$   $B$  out  $\vee$ 
    staged-security-with-data-state-verifier-event
    ( $\lambda s.$  trace-fri-challenge-list-prequery-hit  $s$   $B$ ) out
  proof (cases out)
    case None
    then show ?thesis
      using hit unfolding staged-security-with-data-state-verifier-event-def
      by simp
  next
    case (Some packed)
    then obtain data attacker-state result final-state where out-eq:
      out = Some (((data, attacker-state), result), final-state)
      by (cases packed) (auto split: prod.splits)
    let ?s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    from hit out-eq have pair-hit:
      trace-fri-query-challenge-pair-set-hit ?s  $P$ 
      (Some (result, final-state))
    unfolding staged-security-with-data-state-verifier-event-def by simp
    have challenge-hit:
      trace-fri-challenge-list-set-hit ?s  $B$ 
      (Some (result, final-state))
    by (rule trace-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit
      [OF pair-hit projection])
    from trace-fri-challenge-list-set-hit-imp-fresh-or-prequery
      [OF challenge-hit]
    show ?thesis
    proof
      assume fresh:
        trace-fri-challenge-list-fresh-hit ?s  $B$ 
        (Some (result, final-state))
      then have staged-security-trace-fri-pair-challenge-fresh  $P$   $B$  out
        using hit out-eq unfolding
          staged-security-trace-fri-pair-challenge-fresh-def
          staged-security-with-data-state-verifier-event-def
        by simp
      then show ?thesis by simp
    next
      assume prequery:
        trace-fri-challenge-list-prequery-hit ?s  $B$ 
        (Some (result, final-state))
      then have

```

```

    staged-security-with-data-state-verifier-event
    (λs. trace-fri-challenge-list-prequery-hit s B) out
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
  then show ?thesis by simp
qed
qed
qed
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-trace-fri-pair-challenge-fresh P B)
    adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-challenge-list-prequery-hit s B))
    adversary-initial-state
  by (rule wp-event-union-bound)
finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-fresh:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B: finite B
  and projection: snd ' P ⊆ B
  and fresh-branch-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-trace-fri-pair-challenge-fresh P B)
      adversary-initial-state ≤ F
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-query-challenge-pair-set-hit s P))
      adversary-initial-state ≤
      F +
      hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof -
  have split:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-query-challenge-pair-set-hit s P))
      adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-trace-fri-pair-challenge-fresh P B)
      adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-challenge-list-prequery-hit s B))

```

```

    adversary-initial-state
  by (rule
    checked-staged-security-trace-fri-query-challenge-pair-set-hit-split-challenge-fresh
    [OF projection])
  have prequery-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-challenge-list-prequery-hit s B))
    adversary-initial-state ≤
    hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule checked-staged-security-trace-fri-verifier-prequery-hit-bound
    [OF wf controlled finite-B])
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-query-challenge-pair-set-hit s P))
    adversary-initial-state ≤
    F +
    hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule order-trans[OF split])
    (rule add-mono[OF fresh-branch-bound prequery-bound])
  then show ?thesis .
qed

lemma checked-staged-security-composition-fri-query-challenge-pair-set-hit-split-challenge-fresh:
  assumes projection:  $\bigwedge dg. \text{snd } 'P \text{ dg} \subseteq B \text{ dg}$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. composition-fri-query-challenge-pair-set-hit s P))
    adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-composition-fri-pair-challenge-fresh P B)
    adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. composition-fri-challenge-list-prequery-hit s B))
    adversary-initial-state
proof -
  have event-mono:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. composition-fri-query-challenge-pair-set-hit s P))
    adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (λout.
        staged-security-composition-fri-pair-challenge-fresh P B out ∨
        staged-security-with-data-state-verifier-event

```



```

      ( $\lambda s.$  composition-fri-challenge-list-prequery-hit  $s$   $B$ )  $out$ )
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix  $out$ 
    assume  $hit$ :
      staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-query-challenge-pair-set-hit  $s$   $P$ )  $out$ 
    show
      staged-security-composition-fri-pair-challenge-fresh  $P$   $B$   $out \vee$ 
      staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-challenge-list-prequery-hit  $s$   $B$ )  $out$ 
    proof (cases  $out$ )
      case None
      then show ?thesis
        using  $hit$  unfolding staged-security-with-data-state-verifier-event-def
        by simp
    next
      case (Some packed)
      then obtain  $data$   $attacker-state$   $result$   $final-state$  where  $out-eq$ :
         $out = Some (((data, attacker-state), result), final-state)$ 
        by (cases packed) (auto split: prod.splits)
      let ? $s$  =
        verifier-state-from-adversary  $attacker-state$ 
        (staged-proof-transcript  $data$ )
      from  $hit$   $out-eq$  have  $pair-hit$ :
        composition-fri-query-challenge-pair-set-hit ? $s$   $P$ 
        (Some ( $result$ ,  $final-state$ ))
      unfolding staged-security-with-data-state-verifier-event-def by simp
      have  $challenge-hit$ :
        composition-fri-challenge-list-set-hit ? $s$   $B$ 
        (Some ( $result$ ,  $final-state$ ))
      by (rule
        composition-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit
        [OF  $pair-hit$  projection])
      from composition-fri-challenge-list-set-hit-imp-fresh-or-prequery
        [OF  $challenge-hit$ ]
      show ?thesis
    proof
      assume  $fresh$ :
        composition-fri-challenge-list-fresh-hit ? $s$   $B$ 
        (Some ( $result$ ,  $final-state$ ))
      then have
        staged-security-composition-fri-pair-challenge-fresh  $P$   $B$   $out$ 
      using  $hit$   $out-eq$  unfolding
        staged-security-composition-fri-pair-challenge-fresh-def
        staged-security-with-data-state-verifier-event-def
      by simp
      then show ?thesis by simp
    next

```

```

assume prequery:
  composition-fri-challenge-list-prequery-hit ?s B
  (Some (result, final-state))
then have
  staged-security-with-data-state-verifier-event
  (λs. composition-fri-challenge-list-prequery-hit s B) out
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
  then show ?thesis by simp
qed
qed
qed
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-composition-fri-pair-challenge-fresh P B)
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   (λs. composition-fri-challenge-list-prequery-hit s B))
  adversary-initial-state
  by (rule wp-event-union-bound)
finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-fresh:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
and projection:  $\bigwedge dg. \text{snd } 'P \ dg \subseteq B \ dg$ 
and fresh-branch-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-composition-fri-pair-challenge-fresh P B)
  adversary-initial-state ≤ F
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   (λs. composition-fri-query-challenge-pair-set-hit s P))
  adversary-initial-state ≤
  F +
  hash-relation-budget-value
  ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
   card (B dg) * ceil-log (maxDegree + 1))
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof –
have split:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   (λs. composition-fri-query-challenge-pair-set-hit s P))
  adversary-initial-state ≤

```

```

    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-composition-fri-pair-challenge-fresh P B)
      adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s.$  composition-fri-challenge-list-prequery-hit s B))
      adversary-initial-state
  by (rule
    checked-staged-security-composition-fri-query-challenge-pair-set-hit-split-challenge-fresh
    [OF projection])
  have prequery-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s.$  composition-fri-challenge-list-prequery-hit s B))
      adversary-initial-state  $\leq$ 
      hash-relation-budget-value
        ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
          card (B dg) * ceil-log (maxDegree + 1))
        (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule checked-staged-security-composition-fri-verifier-prequery-hit-bound
    [OF wf controlled finite-B])
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-query-challenge-pair-set-hit s P))
    adversary-initial-state  $\leq$ 
    F +
    hash-relation-budget-value
      ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
        card (B dg) * ceil-log (maxDegree + 1))
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule order-trans[OF split])
    (rule add-mono[OF fresh-branch-bound prequery-bound])
  then show ?thesis .
qed

end

end

```

```

theory Soundness-FRI-Query-Challenge-Actual-Product
imports Soundness-FRI-Query-Challenge-Fresh-Split
begin

```

Verifier-local pair-product bounds with actual FRI challenge-path freshness.

This replaces the global trace-FRI future-fresh premise in the fixed challenge product bound by the concrete freshness evidence carried by the trace FRI challenge-list freshness event.

```

context soundness

```

begin

lemma *verifier-trace-fri-prefix-preserves-query-lookup*:

assumes *outcome*:

Some ((fr, f-fl), t) ∈

set-dist (execute verifier-trace-fri-prefix s)

shows

fmlookup (HashMap t) (QueryIndexChallenge i x) =

fmlookup (HashMap s) (QueryIndexChallenge i x)

proof –

let *?key = QueryIndexChallenge i x*

from *outcome* **obtain** *s1 s2* **where**

read-fr: Some (fr, s1) ∈ set-dist (execute read s)

and *trace-fri*:

Some (f-fl, s2) ∈

set-dist (execute

(ntimes receive-trace-fri-commits (ceil-log clength)) s1)

and *ret*:

Some ((fr, f-fl), t) ∈ set-dist (execute (return (fr, f-fl)) s2)

unfolding *verifier-trace-fri-prefix-def*

by (*auto elim!: set-dist-bindE*)

have *lookup-s1*:

fmlookup (HashMap s1) ?key = fmlookup (HashMap s) ?key

using *read-preserves-hash-map[OF read-fr]* **by** *simp*

have *lookup-s2*:

fmlookup (HashMap s2) ?key = fmlookup (HashMap s1) ?key

proof (*rule ntimes-preserves-lookup[OF - trace-fri]*)

fix *y u v*

assume *out: Some (y, v) ∈ set-dist (execute receive-trace-fri-commits u)*

obtain *b r* **where** *y-eq: y = (b, r)*

by (*cases y*) *simp*

show *fmlookup (HashMap v) ?key = fmlookup (HashMap u) ?key*

by (*rule receive-trace-fri-commits-preserves-query-lookup*
[OF out[unfolded y-eq]])

qed

have *t-eq: t = s2*

using *ret* **by** *simp*

show *?thesis*

using *lookup-s1 lookup-s2* **unfolding** *t-eq* **by** *simp*

qed

lemma *wp-trace-fri-query-challenge-pair-set-hit-fixed-challenge-actual-fresh-bound*:

assumes *query-future: query-future-fresh s*

and *raw-bound: query-index-raw-preimage-bound*

and *challenges-space*:

challenges ∈ fri-challenge-space (ceil-log clength)

and *projection*:

fst ‘ (Pairs ∩ (UNIV × {challenges})) ⊆ Queries

and *subset: Queries ⊆ fri-query-index-list-space*

shows

wp-event verify-monad
 $(\lambda out. \text{trace-fri-challenge-list-fresh-hit } s \ \{challenges\} \ out \wedge$
 $\text{trace-fri-query-challenge-pair-set-hit } s \ Pairs \ out) \ s \leq$
 $(1 / \text{nnreal } (CARD('f) \wedge \text{ceil-log } clength)) \ast$
 $(\text{nnreal } (card \ Queries) \ast$
 $(1 / \text{nnreal } (card \ query-sample-space)) \wedge \text{rounds})$

proof –

let *?Hit* =
 $\lambda out. \text{trace-fri-challenge-list-fresh-hit } s \ \{challenges\} \ out \wedge$
 $\text{trace-fri-query-challenge-pair-set-hit } s \ Pairs \ out$

let *?Head* =
 $\lambda out. \text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } ((fr, f-fl), -) \Rightarrow$
 $\text{map fst } f-fl = challenges \wedge$
 $\text{trace-fri-challenge-path-fresh } s \ fr \ (\text{map snd } f-fl)$
 $(\text{length } (\text{map snd } f-fl))$

let *?D* =
 $\text{nnreal } (card \ Queries) \ast$
 $(1 / \text{nnreal } (card \ query-sample-space)) \wedge \text{rounds}$

have *bind-bound*:

wp-event verify-monad ?Hit s ≤
wp-event verifier-trace-fri-prefix ?Head s * *?D*

unfolding *verify-monad-trace-fri-decomposition*

proof (rule *wp-event-bind-bound-by-head-and-cont*)

show $\neg ?Hit \text{ None}$

unfolding *trace-fri-challenge-list-fresh-hit-def*
trace-fri-query-challenge-pair-set-hit-def
accepted-fri-challenges-def
accepted-fri-opening-transcript-def

by *blast*

next

fix *header prefix-state*

assume *prefix*:

Some (header, prefix-state) ∈
 $\text{set-dist } (\text{execute verifier-trace-fri-prefix } s)$
and *not-head*: $\neg ?Head \ (\text{Some } (header, \text{prefix-state}))$

obtain *fr f-fl* **where** *header-eq*: *header* = (*fr*, *f-fl*)

by (*cases header*) *auto*

have *not-head'*:

$\text{map fst } f-fl \neq challenges \vee$
 $\neg \text{trace-fri-challenge-path-fresh } s \ fr \ (\text{map snd } f-fl)$
 $(\text{length } (\text{map snd } f-fl))$

using *not-head* **unfolding** *header-eq* **by** *simp*

have *zero-le*:

wp-event (verifier-after-trace-fri header) ?Hit prefix-state ≤ 0

proof (rule *order-trans*)

show *wp-event (verifier-after-trace-fri header) ?Hit prefix-state* ≤

```

    wp-event (verifier-after-trace-fri header)
    (λ-. False) prefix-state
proof (rule wp-event-mono-on-support)
fix out
assume suffix:
    out ∈
    set-dist (execute (verifier-after-trace-fri header) prefix-state)
and hit: ?Hit out
from hit obtain result :: unit list and final-state where out-eq:
    out = Some (result, final-state)
unfolding trace-fri-challenge-list-fresh-hit-def
    accepted-fri-challenges-def
by blast
have prefix':
    Some ((fr, f-fl), prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix s)
using prefix header-eq by simp
have suffix':
    Some (result, final-state) ∈
    set-dist
    (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
using suffix header-eq out-eq by simp
from hit obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers where
    openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final query-idxs trace-round-layers
    composition-round-layers
unfolding trace-fri-query-challenge-pair-set-hit-def by blast
have pair-challenges:
    accepted-fri-challenges s out trace-bs fri-dg composition-bs
by (rule accepted-fri-opening-transcript-challenges[OF openings])
from hit obtain trace-bs' dg' comp-bs' where
    fresh-challenges:
    accepted-fri-challenges s out trace-bs' dg' comp-bs'
and trace-bs'-in: trace-bs' ∈ {challenges}
unfolding trace-fri-challenge-list-fresh-hit-def by blast
have trace-bs-eq: trace-bs = trace-bs'
using accepted-fri-challenges-unique
    [OF pair-challenges[unfolded out-eq]
    fresh-challenges[unfolded out-eq]]
by simp
have prefix-trace-eq: trace-bs = map fst f-fl
by (rule verifier-after-trace-fri-header-agrees-with-accepted-challenges
    [OF prefix' suffix' pair-challenges[unfolded out-eq]])
have fresh-hit:
    trace-fri-challenge-list-fresh-hit s {challenges}

```

```

    (Some (result, final-state))
  using hit out-eq by simp
have concrete-fresh:
  trace-fri-challenge-path-fresh s fr (map snd f-fl)
  (length (map snd f-fl))
  by (rule verifier-after-trace-fri-header-roots-agree-with-fresh-hit
    [OF prefix' suffix' fresh-hit])
have head-holds:
  map fst f-fl = challenges ∧
  trace-fri-challenge-path-fresh s fr (map snd f-fl)
  (length (map snd f-fl))
  using prefix-trace-eq trace-bs-eq trace-bs'-in concrete-fresh by simp
show False
  using not-head' head-holds by blast
qed
next
show wp-event (verifier-after-trace-fri header)
  (λ-. False) prefix-state ≤ 0
  unfolding wp-event-def wp-def dist-expect-def by simp
qed
show wp-event (verifier-after-trace-fri header) ?Hit prefix-state = 0
  by (rule antisym[OF zero-le]) simp
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state) ∈
  set-dist (execute verifier-trace-fri-prefix s)
  and head: ?Head (Some (header, prefix-state))
obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
have f-fl-eq: map fst f-fl = challenges
  using head unfolding header-eq by simp
have future-prefix: query-future-fresh prefix-state
  by (rule verifier-trace-fri-prefix-preserves-query-future-fresh-exact
    [OF query-future prefix])
have counter-prefix: PQueryCounter prefix-state = PQueryCounter s
  using verifier-trace-fri-prefix-outcome[OF prefix[unfolded header-eq]]
  by simp
have event-bound:
  wp-event (verifier-after-trace-fri header)
  (λout. ∃ query-idxs ∈ Queries.
    query-rounds-index-list-hit s query-idxs rounds out)
  prefix-state ≤ ?D
  by (rule wp-verifier-after-trace-fri-query-index-list-set-bound
    [OF future-prefix counter-prefix raw-bound subset])
have hit-to-query:
  wp-event (verifier-after-trace-fri header) ?Hit prefix-state ≤
  wp-event (verifier-after-trace-fri header)
  (λout. ∃ query-idxs ∈ Queries.

```

query-rounds-index-list-hit s query-idxs rounds out) *prefix-state*
proof (rule *wp-event-mono-on-support*)
fix *out*
assume *suffix*:
out ∈
set-dist (*execute* (*verifier-after-trace-fri header*) *prefix-state*)
and *hit*: ?*Hit out*
from *hit* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers **where**
openings:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final query-idxs trace-round-layers
composition-round-layers
and *pair*: (*query-idxs*, *trace-bs*) ∈ *Pairs*
unfolding *trace-fri-query-challenge-pair-set-hit-def* **by** *blast*
from *openings* **obtain** *result :: unit list* **and** *final-state* **where** *out-eq*:
out = *Some* (*result*, *final-state*)
unfolding *accepted-fri-opening-transcript-def* **by** *blast*
have *prefix'*:
Some ((*fr*, *f-fl*), *prefix-state*) ∈
set-dist (*execute* *verifier-trace-fri-prefix s*)
using *prefix header-eq* **by** *simp*
have *suffix'*:
Some (*result*, *final-state*) ∈
set-dist
(*execute* (*verifier-after-trace-fri* (*fr*, *f-fl*)) *prefix-state*)
using *suffix header-eq out-eq* **by** *simp*
have *challenges-openings*:
accepted-fri-challenges s out trace-bs fri-dg composition-bs
by (rule *accepted-fri-opening-transcript-challenges*[*OF openings*])
have *trace-eq*: *trace-bs* = *map fst f-fl*
by (rule *verifier-after-trace-fri-header-agrees-with-accepted-challenges*
[*OF prefix' suffix' challenges-openings*[*unfolded out-eq*]])
have *query-in*: *query-idxs* ∈ *Queries*
proof –
have *query-idxs* ∈ *fst* ‘(*Pairs* ∩ (*UNIV* × {*challenges*}))
using *pair trace-eq f-fl-eq* **by** *force*
then show ?*thesis*
using *projection* **by** *blast*
qed
have *query-hit*:
query-rounds-index-list-hit s query-idxs rounds out
by (rule *accepted-fri-opening-transcript-imp-query-rounds-index-list-hit*
[*OF openings*])
show ∃ *query-idxs* ∈ *Queries*.
query-rounds-index-list-hit s query-idxs rounds out
using *query-in query-hit* **by** *blast*

qed
show $wp\text{-event } (verifier\text{-after}\text{-trace}\text{-fri } header) \text{ ?Hit prefix}\text{-state} \leq ?D$
by (rule order-trans[*OF hit-to-query event-bound*])
qed
have *head-bound*:
 $wp\text{-event } verifier\text{-trace}\text{-fri}\text{-prefix } ?Head \ s \leq$
 $1 / nnreal \ (CARD('f) \wedge \text{ceil-log } clength)$
proof –
let *?HeadSet* =
 $\lambda out. \text{ case out of}$
 $None \Rightarrow False$
 $| Some \ ((fr, f\text{-}fl), -) \Rightarrow$
 $map \ fst \ f\text{-}fl \in \{challenges\} \wedge$
 $trace\text{-fri}\text{-challenge}\text{-path}\text{-fresh } s \ fr \ (map \ snd \ f\text{-}fl)$
 $(length \ (map \ snd \ f\text{-}fl))$
have *subset*: $\{challenges\} \subseteq fri\text{-challenge}\text{-space } (ceil\text{-log } clength)$
using *challenges-space* **by** *simp*
have *mono*:
 $wp\text{-event } verifier\text{-trace}\text{-fri}\text{-prefix } ?Head \ s \leq$
 $wp\text{-event } verifier\text{-trace}\text{-fri}\text{-prefix } ?HeadSet \ s$
by (rule *wp-event-mono*) (auto split: *option.splits prod.splits*)
have $wp\text{-event } verifier\text{-trace}\text{-fri}\text{-prefix } ?Head \ s \leq$
 $nnreal \ (card \ \{challenges\}) /$
 $nnreal \ (CARD('f) \wedge \text{ceil-log } clength)$
by (rule order-trans[*OF mono*])
 $(rule \ wp\text{-verifier}\text{-trace}\text{-fri}\text{-prefix}\text{-challenge}\text{-list}\text{-actual}\text{-fresh}\text{-bound}$
 $[OF \ subset])$
then show *?thesis*
by *simp*
qed
have $wp\text{-event } verify\text{-monad } ?Hit \ s \leq$
 $wp\text{-event } verifier\text{-trace}\text{-fri}\text{-prefix } ?Head \ s * ?D$
by (rule *bind-bound*)
also have $\dots \leq$
 $(1 / nnreal \ (CARD('f) \wedge \text{ceil-log } clength)) * ?D$
by (rule *mult-right-mono*[*OF head-bound*]) *simp*
finally show *?thesis* .
qed

lemma *wp-trace-fri-query-challenge-pair-set-hit-challenge-fresh-fiber-sum*:
assumes *query-future*: *query-future-fresh* *s*
and *raw-bound*: *query-index-raw-preimage-bound*
and *challenge-subset*: $B \subseteq fri\text{-challenge}\text{-space } (ceil\text{-log } clength)$
and *projection*:
 $\bigwedge challenges. challenges \in B \Rightarrow$
 $fst \ ' \ (P \cap (UNIV \times \{challenges\})) \subseteq Q \ challenges$
and *subset*:
 $\bigwedge challenges. challenges \in B \Rightarrow$

$Q \text{ challenges} \subseteq \text{fri-query-index-list-space}$
shows
 $\text{wp-event verify-monad}$
 $(\lambda \text{out. trace-fri-challenge-list-fresh-hit } s \text{ } B \text{ out} \wedge$
 $\text{trace-fri-query-challenge-pair-set-hit } s \text{ } P \text{ out}) \text{ } s \leq$
 $(\sum \text{challenges} \in B.$
 $(1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log clength})) *$
 $(\text{nnreal } (\text{card } (Q \text{ challenges}))) *$
 $(1 / \text{nnreal } (\text{card query-sample-space})) \wedge \text{rounds}))$
proof –
let $?E =$
 $\lambda \text{challenges out.}$
 $\text{trace-fri-challenge-list-fresh-hit } s \text{ } \{\text{challenges}\} \text{ out} \wedge$
 $\text{trace-fri-query-challenge-pair-set-hit } s \text{ } P \text{ out}$
let $?C =$
 $\lambda \text{challenges.}$
 $(1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log clength})) *$
 $(\text{nnreal } (\text{card } (Q \text{ challenges}))) *$
 $(1 / \text{nnreal } (\text{card query-sample-space})) \wedge \text{rounds}))$
have *split*:
 $\text{wp-event verify-monad}$
 $(\lambda \text{out. trace-fri-challenge-list-fresh-hit } s \text{ } B \text{ out} \wedge$
 $\text{trace-fri-query-challenge-pair-set-hit } s \text{ } P \text{ out}) \text{ } s \leq$
 $\text{wp-event verify-monad}$
 $(\lambda \text{out. } \exists \text{challenges} \in B. ?E \text{ challenges out}) \text{ } s$
proof (rule *wp-event-mono*)
fix *out*
assume *hit*:
 $\text{trace-fri-challenge-list-fresh-hit } s \text{ } B \text{ out} \wedge$
 $\text{trace-fri-query-challenge-pair-set-hit } s \text{ } P \text{ out}$
then obtain $\text{trace-bs} :: 'f \text{ list}$ **and** $\text{dg} :: 'f$ **and** $\text{comp-bs} :: 'f \text{ list}$
and $\text{fr} :: 'f$ **and** $\text{trace-roots} :: 'f \text{ list}$ **and** $\text{trace-final} :: 'f$
and $\text{as} :: 'f \text{ list}$ **and** $\text{composition-roots} :: 'f \text{ list}$
and $\text{final} :: 'f$ **and** $\text{rest} :: 'f \text{ list}$ **where**
accepted:
 $\text{accepted-fri-challenges } s \text{ out trace-bs dg comp-bs}$
and *header*:
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots final rest}$
and $\text{trace-bs-in: trace-bs} \in B$
and *fresh-path*:
 $\text{trace-fri-challenge-path-fresh } s \text{ fr trace-roots}$
 $(\text{length trace-roots})$
unfolding *trace-fri-challenge-list-fresh-hit-def* **by** *blast*
have *singleton-fresh*:
 $\text{trace-fri-challenge-list-fresh-hit } s \text{ } \{\text{trace-bs}\} \text{ out}$
unfolding *trace-fri-challenge-list-fresh-hit-def*
proof (*intro exI conjI*)
show $\text{accepted-fri-challenges } s \text{ out trace-bs dg comp-bs}$

```

    by (rule accepted)
  show verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots final rest
    by (rule header)
  show trace-bs ∈ {trace-bs}
    by simp
  show trace-fri-challenge-path-fresh s fr trace-roots
    (length trace-roots)
    by (rule fresh-path)
qed

show ∃ challenges ∈ B. ?E challenges out
  using singleton-fresh hit trace-bs-in
  by (intro bexI[of - trace-bs]) simp-all
qed
also have ... ≤ (∑ challenges ∈ B. ?C challenges)
proof (rule wp-event-finite-union-bound-fri-exact)
  show finite B
    using challenge-subset by (rule finite-subset)
    (rule finite-fri-challenge-space)
next
  fix challenges
  assume challenges-in: challenges ∈ B
  have challenges-space:
    challenges ∈ fri-challenge-space (ceil-log clength)
    using challenge-subset challenges-in by blast
  show wp-event verify-monad (?E challenges) s ≤ ?C challenges
    by (rule
      wp-trace-fri-query-challenge-pair-set-hit-fixed-challenge-actual-fresh-bound
      [OF query-future raw-bound challenges-space
        projection[OF challenges-in] subset[OF challenges-in]])
qed
finally show ?thesis .
qed

lemma wp-composition-fri-query-challenge-pair-set-hit-fixed-challenge-actual-fresh-bound:
  assumes query-future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and challenges-space:
    challenges ∈ fri-challenge-space (ceil-log (to-nat dg + 1))
  and projection:
    fst ‘ (Pairs dg ∩ (UNIV × {challenges})) ⊆ Queries
  and subset: Queries ⊆ fri-query-index-list-space
  shows
    wp-event verify-monad
      (λout.
        composition-fri-challenge-list-fresh-hit s
          (λdg'. if dg' = dg then {challenges} else {})) out ∧
        composition-fri-query-challenge-pair-set-hit s Pairs out) s ≤

```

```

(1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
  (nnreal (card Queries) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
proof -
  let ?B = λdg'. if dg' = dg then {challenges} else {}
  let ?Hit =
    λout. composition-fri-challenge-list-fresh-hit s ?B out ∧
      composition-fri-query-challenge-pair-set-hit s Pairs out
  let ?Head =
    λout. case out of
      None ⇒ False
    | Some ((fr, f-fl, f-final, as, dg', fl), -) ⇒
      dg' = dg ∧ map fst fl = challenges ∧
        composition-fri-challenge-path-fresh s fr (map snd f-fl)
        f-final as dg' (map snd fl) (length (map snd fl))
  let ?D =
    nnreal (card Queries) *
      (1 / nnreal (card query-sample-space)) ^ rounds
  have bind-bound:
    wp-event verify-monad ?Hit s ≤
      wp-event verifier-composition-fri-prefix ?Head s * ?D
  unfolding verify-monad-composition-fri-decomposition
  proof (rule wp-event-bind-bound-by-head-and-cont)
  show ¬ ?Hit None
  unfolding composition-fri-challenge-list-fresh-hit-def
    composition-fri-query-challenge-pair-set-hit-def
    accepted-fri-challenges-def
    accepted-fri-opening-transcript-def
  by blast
next
  fix header prefix-state
  assume prefix:
    Some (header, prefix-state) ∈
      set-dist (execute verifier-composition-fri-prefix s)
  and not-head: ¬ ?Head (Some (header, prefix-state))
  obtain fr f-fl f-final as dg' fl where header-eq:
    header = (fr, f-fl, f-final, as, dg', fl)
  by (cases header) auto
  have not-head':
    dg' ≠ dg ∨ map fst fl ≠ challenges ∨
      ¬ composition-fri-challenge-path-fresh s fr (map snd f-fl)
      f-final as dg' (map snd fl) (length (map snd fl))
  using not-head unfolding header-eq by simp
  have zero-le:
    wp-event (verifier-after-composition-fri header) ?Hit prefix-state ≤ 0
  proof (rule order-trans)
  show wp-event (verifier-after-composition-fri header) ?Hit prefix-state ≤
    wp-event (verifier-after-composition-fri header)
      (λ-. False) prefix-state

```

```

proof (rule wp-event-mono-on-support)
  fix out
  assume suffix:
    out ∈
      set-dist
        (execute (verifier-after-composition-fri header) prefix-state)
  and hit: ?Hit out
  from hit obtain result :: unit list and final-state where out-eq:
    out = Some (result, final-state)
  unfolding composition-fri-challenge-list-fresh-hit-def
    accepted-fri-challenges-def
  by blast
  have prefix':
    Some ((fr, f-fl, f-final, as, dg', fl), prefix-state) ∈
      set-dist (execute verifier-composition-fri-prefix s)
  using prefix header-eq by simp
  have suffix':
    Some (result, final-state) ∈
      set-dist
        (execute
          (verifier-after-composition-fri
            (fr, f-fl, f-final, as, dg', fl))
          prefix-state)
  using suffix header-eq out-eq by simp
  from hit obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers where
    openings:
      accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final fri-dg composition-roots composition-bs
      composition-final query-idxs trace-round-layers
      composition-round-layers
  unfolding composition-fri-query-challenge-pair-set-hit-def by blast
  have pair-challenges:
    accepted-fri-challenges s out trace-bs fri-dg composition-bs
  by (rule accepted-fri-opening-transcript-challenges[OF openings])
  from hit obtain trace-bs' dg'' comp-bs' where
    fresh-challenges:
      accepted-fri-challenges s out trace-bs' dg'' comp-bs'
  and comp-bs'-in:
    comp-bs' ∈ (if dg'' = dg then {challenges} else {})
  unfolding composition-fri-challenge-list-fresh-hit-def by blast
  have challenge-eqs:
    fri-dg = dg'' ∧ composition-bs = comp-bs'
  using accepted-fri-challenges-unique
    [OF pair-challenges[unfolded out-eq]
      fresh-challenges[unfolded out-eq]]
  by simp
  have prefix-eqs:

```

```

    fri-dg = dg'  $\wedge$  composition-bs = map fst fl
  by (rule verifier-after-composition-fri-header-agrees-with-accepted-challenges
      [OF prefix' suffix' pair-challenges[unfolded out-eq]])
have fresh-hit:
  composition-fri-challenge-list-fresh-hit s ?B
  (Some (result, final-state))
  using hit out-eq by simp
have concrete-fresh:
  composition-fri-challenge-path-fresh s fr (map snd f-fl)
  f-final as dg' (map snd fl) (length (map snd fl))
  by (rule verifier-after-composition-fri-header-roots-agree-with-fresh-hit
      [OF prefix' suffix' fresh-hit])
have dg-eq': dg'' = dg
  using comp-bs'-in by (cases dg'' = dg) auto
have comp-bs'-eq: comp-bs' = map fst fl
  using challenge-eqs prefix-eqs by simp
have comp-bs'-challenges: comp-bs' = challenges
  using comp-bs'-in dg-eq' by simp
have dg'-eq: dg' = dg
  using challenge-eqs prefix-eqs dg-eq' by simp
have fl-challenges: map fst fl = challenges
  using comp-bs'-eq comp-bs'-challenges by simp
have head-holds:
  dg' = dg  $\wedge$  map fst fl = challenges  $\wedge$ 
  composition-fri-challenge-path-fresh s fr (map snd f-fl)
  f-final as dg' (map snd fl) (length (map snd fl))
  using dg'-eq fl-challenges concrete-fresh by simp
show False
  using not-head' head-holds by blast
qed
next
show wp-event (verifier-after-composition-fri header)
  ( $\lambda$ -. False) prefix-state  $\leq 0$ 
  unfolding wp-event-def wp-def dist-expect-def by simp
qed
show wp-event (verifier-after-composition-fri header) ?Hit prefix-state = 0
  by (rule antisym[OF zero-le]) simp
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state)  $\in$ 
  set-dist (execute verifier-composition-fri-prefix s)
  and head: ?Head (Some (header, prefix-state))
obtain fr f-fl f-final as dg' fl where header-eq:
  header = (fr, f-fl, f-final, as, dg', fl)
  by (cases header) auto
have dg-eq: dg' = dg and fl-eq: map fst fl = challenges
  using head unfolding header-eq by simp-all
have future-prefix: query-future-fresh prefix-state

```

by (rule verifier-composition-fri-prefix-preserves-query-future-fresh
 [OF query-future prefix])
have counter-prefix: PQueryCounter prefix-state = PQueryCounter s
using verifier-composition-fri-prefix-outcome[OF prefix[unfolded header-eq]]
by simp
have event-bound:
 wp-event (verifier-after-composition-fri header)
 (λ out. $\exists$ query-idxs $\in$ Queries.
 query-rounds-index-list-hit s query-idxs rounds out)
 prefix-state $\leq$?D
by (rule wp-verifier-after-composition-fri-query-index-list-set-bound
 [OF future-prefix counter-prefix raw-bound subset])
have hit-to-query:
 wp-event (verifier-after-composition-fri header) ?Hit prefix-state $\leq$
 wp-event (verifier-after-composition-fri header)
 (λ out. $\exists$ query-idxs $\in$ Queries.
 query-rounds-index-list-hit s query-idxs rounds out) prefix-state
proof (rule wp-event-mono-on-support)
fix out
assume suffix:
 out $\in$
 set-dist
 (execute (verifier-after-composition-fri header) prefix-state)
and hit: ?Hit out
from hit **obtain** trace-roots trace-bs trace-final fri-dg
 composition-roots composition-bs composition-final query-idxs
 trace-round-layers composition-round-layers **where**
 openings:
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final query-idxs trace-round-layers
 composition-round-layers
and pair: (query-idxs, composition-bs) $\in$ Pairs fri-dg
unfolding composition-fri-query-challenge-pair-set-hit-def **by** blast
from openings **obtain** result :: unit list **and** final-state **where** out-eq:
 out = Some (result, final-state)
unfolding accepted-fri-opening-transcript-def **by** blast
have prefix':
 Some ((fr, f-fl, f-final, as, dg', fl), prefix-state) $\in$
 set-dist (execute verifier-composition-fri-prefix s)
using prefix header-eq **by** simp
have suffix':
 Some (result, final-state) $\in$
 set-dist
 (execute
 (verifier-after-composition-fri
 (fr, f-fl, f-final, as, dg', fl))
 prefix-state)
using suffix header-eq out-eq **by** simp

```

have challenges-openings:
  accepted-fri-challenges s out trace-bs fri-dg composition-bs
  by (rule accepted-fri-opening-transcript-challenges[OF openings])
have eqs: fri-dg = dg'  $\wedge$  composition-bs = map fst fl
  by (rule verifier-after-composition-fri-header-agrees-with-accepted-challenges
    [OF prefix' suffix' challenges-openings[unfolded out-eq]])
have query-in: query-idxs  $\in$  Queries
proof -
  have query-idxs  $\in$  fst ' (Pairs dg  $\cap$  (UNIV  $\times$  {challenges}))
  using pair eqs dg-eq fl-eq by force
  then show ?thesis
  using projection by blast
qed
have query-hit:
  query-rounds-index-list-hit s query-idxs rounds out
  by (rule accepted-fri-opening-transcript-imp-query-rounds-index-list-hit
    [OF openings])
show  $\exists$  query-idxs  $\in$  Queries.
  query-rounds-index-list-hit s query-idxs rounds out
  using query-in query-hit by blast
qed
show wp-event (verifier-after-composition-fri header) ?Hit prefix-state  $\leq$ 
  ?D
  by (rule order-trans[OF hit-to-query event-bound])
qed
have head-bound:
  wp-event verifier-composition-fri-prefix ?Head s  $\leq$ 
  1 / nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg + 1))
proof -
  let ?B =  $\lambda$ dg'. if dg' = dg then {challenges} else {}
  let ?HeadSet =
     $\lambda$ out. case out of
      None  $\Rightarrow$  False
    | Some ((fr, f-fl, f-final, as, dg', fl), -)  $\Rightarrow$ 
      map fst fl  $\in$  ?B dg'  $\wedge$ 
      composition-fri-challenge-path-fresh s fr (map snd f-fl)
      f-final as dg' (map snd fl) (length (map snd fl))
  have subset:
     $\bigwedge$ dg'. ?B dg'  $\subseteq$ 
    fri-challenge-space (ceil-log (to-nat dg' + 1))
  using challenges-space by auto
  have bound:
     $\bigwedge$ dg'. nnreal (card (?B dg')) /
    nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg' + 1))  $\leq$ 
    1 / nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg + 1))
  using challenges-space by auto
  have mono:
    wp-event verifier-composition-fri-prefix ?Head s  $\leq$ 
    wp-event verifier-composition-fri-prefix ?HeadSet s

```



```

    by (rule wp-event-mono) (auto split: option.splits prod.splits)
  show ?thesis
    by (rule order-trans[OF mono])
      (rule wp-verifier-composition-fri-prefix-challenge-list-actual-fresh-bound
        [OF subset bound])
  qed
  have wp-event verify-monad ?Hit s ≤
    wp-event verifier-composition-fri-prefix ?Head s * ?D
  by (rule bind-bound)
  also have ... ≤
    (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) * ?D
  by (rule mult-right-mono[OF head-bound]) simp
  finally show ?thesis .
qed

lemma wp-composition-fri-query-challenge-pair-set-hit-challenge-fresh-fiber-sum:
  assumes query-future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and challenge-subset:
     $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$ 
  and projection:
     $\bigwedge dg \text{ challenges. } \text{challenges} \in B \ dg \implies$ 
     $\text{fst } ' (P \ dg \cap (UNIV \times \{\text{challenges}\})) \subseteq$ 
     $Q \ dg \text{ challenges}$ 
  and subset:
     $\bigwedge dg \text{ challenges. } \text{challenges} \in B \ dg \implies$ 
     $Q \ dg \text{ challenges} \subseteq \text{fri-query-index-list-space}$ 
  shows
    wp-event verify-monad
    ( $\lambda out. \text{composition-fri-challenge-list-fresh-hit } s \ B \ out \wedge$ 
       $\text{composition-fri-query-challenge-pair-set-hit } s \ P \ out) \ s \leq$ 
    ( $\sum dg \in UNIV. \sum \text{challenges} \in B \ dg.$ 
       $(1 / \text{nnreal } (\text{CARD}('f) ^ \text{ceil-log } (\text{to-nat } dg + 1))) *$ 
       $(\text{nnreal } (\text{card } (Q \ dg \text{ challenges}))) *$ 
       $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) ^ \text{rounds}))$ 
  proof -
    let ?I = SIGMA dg:UNIV. B dg
    let ?E =
       $\lambda p \text{ out. } \text{composition-fri-challenge-list-fresh-hit } s$ 
       $(\lambda dg'. \text{if } dg' = \text{fst } p \text{ then } \{\text{snd } p\} \text{ else } \{\}) \text{ out } \wedge$ 
       $\text{composition-fri-query-challenge-pair-set-hit } s \ P \ out$ 
    let ?C =
       $\lambda p.$ 
       $(1 / \text{nnreal } (\text{CARD}('f) ^ \text{ceil-log } (\text{to-nat } (\text{fst } p) + 1))) *$ 
       $(\text{nnreal } (\text{card } (Q \ (\text{fst } p) \ (\text{snd } p)))) *$ 
       $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) ^ \text{rounds})$ 
  end

```

have *split*:
wp-event verify-monad
 $(\lambda out. \text{composition-fri-challenge-list-fresh-hit } s \ B \ out \wedge$
 $\text{composition-fri-query-challenge-pair-set-hit } s \ P \ out) \ s \leq$
wp-event verify-monad
 $(\lambda out. \exists p \in ?I. ?E \ p \ out) \ s$
proof (rule *wp-event-mono*)
fix *out*
assume *hit*:
 $\text{composition-fri-challenge-list-fresh-hit } s \ B \ out \wedge$
 $\text{composition-fri-query-challenge-pair-set-hit } s \ P \ out$
then obtain *trace-bs dg comp-bs fr trace-roots trace-final as*
 $\text{composition-roots final rest}$ **where**
accepted:
 $\text{accepted-fri-challenges } s \ out \ \text{trace-bs } dg \ \text{comp-bs}$
and *comp-bs-in*: $\text{comp-bs} \in B \ dg$
and *header*:
 $\text{verifier-header-transcript } s \ fr \ \text{trace-roots } \text{trace-final} \ as \ dg$
 $\text{composition-roots final rest}$
and *fresh-path*:
 $\text{composition-fri-challenge-path-fresh } s \ fr \ \text{trace-roots } \text{trace-final}$
 $as \ dg \ \text{composition-roots } (\text{length } \text{composition-roots})$
unfolding *composition-fri-challenge-list-fresh-hit-def* **by** *blast*
have *singleton-fresh*:
 $\text{composition-fri-challenge-list-fresh-hit } s$
 $(\lambda dg'. \text{if } dg' = dg \text{ then } \{\text{comp-bs}\} \text{ else } \{\}) \ out$
unfolding *composition-fri-challenge-list-fresh-hit-def*
proof (intro *exI conjI*)
show *accepted-fri-challenges* $s \ out \ \text{trace-bs } dg \ \text{comp-bs}$
by (rule *accepted*)
show *verifier-header-transcript* $s \ fr \ \text{trace-roots } \text{trace-final} \ as \ dg$
 $\text{composition-roots final rest}$
by (rule *header*)
show $\text{comp-bs} \in (\text{if } dg = dg \text{ then } \{\text{comp-bs}\} \text{ else } \{\})$
by *simp*
show *composition-fri-challenge-path-fresh* $s \ fr \ \text{trace-roots } \text{trace-final}$
 $as \ dg \ \text{composition-roots } (\text{length } \text{composition-roots})$
by (rule *fresh-path*)
qed
have *p-in*: $(dg, \text{comp-bs}) \in ?I$
using *comp-bs-in* **by** *simp*
have *singleton-fresh'*:
 $\text{composition-fri-challenge-list-fresh-hit } s$
 $(\lambda dg'. \text{if } dg' = fst \ (dg, \text{comp-bs}) \text{ then } \{snd \ (dg, \text{comp-bs})\}$
 $\text{else } \{\}) \ out$
using *singleton-fresh* **by** (*simp only: fst-conv snd-conv*)
have *e-hit*: $?E \ (dg, \text{comp-bs}) \ out$
using *singleton-fresh' hit* **by** *simp*
show $\exists p \in ?I. ?E \ p \ out$

```

    using p-in e-hit by blast
qed
also have ...  $\leq (\sum p \in ?I. ?C p)$ 
proof (rule wp-event-finite-union-bound-fri-exact)
  show finite ?I
    apply (rule finite-SigmaI)
    using challenge-subset finite-fri-challenge-space
    apply (auto intro: finite-subset)
    by (meson finite-fri-challenge-space rev-finite-subset)
next
fix p
assume p-in:  $p \in ?I$ 
obtain dg challenges where p-eq:  $p = (dg, challenges)$ 
  by (cases p) auto
have challenges-in:  $challenges \in B dg$ 
  using p-in p-eq by simp
have challenges-space:
  challenges  $\in$  fri-challenge-space (ceil-log (to-nat dg + 1))
  using challenge-subset challenges-in by blast
show wp-event verify-monad ( $?E p$ )  $s \leq ?C p$ 
  unfolding p-eq
  apply (rule wp-composition-fri-query-challenge-pair-set-hit-fixed-challenge-actual-fresh-bound)
  using query-future raw-bound challenges-space
    projection[OF challenges-in] subset[OF challenges-in]
  by auto
qed
finally have bound:
  wp-event verify-monad
    ( $\lambda out. composition-fri-challenge-list-fresh-hit s B out \wedge$ 
       $composition-fri-query-challenge-pair-set-hit s P out$ )  $s \leq$ 
    ( $\sum p \in ?I. ?C p$ ) .
have finite-B:  $\bigwedge dg. finite (B dg)$ 
  by (meson challenge-subset finite-fri-challenge-space finite-subset)
have sigma-eq:
  ( $\sum p \in ?I. ?C p$ ) =
  ( $\sum dg \in UNIV. \sum challenges \in B dg.$ 
    ( $1 / nnreal (CARD('f) \wedge ceil-log (to-nat dg + 1))$ ) *
    ( $nnreal (card (Q dg challenges))$ ) *
    ( $1 / nnreal (card query-sample-space)$ )  $\wedge$  rounds))
  by (subst sum.Sigma) (simp-all add: finite-B case-prod-beta)
show ?thesis
  using bound sigma-eq by simp
qed

```

definition staged-security-trace-fri-pair-challenge-query-path-fresh

```

:: (nat list  $\times$  'f list) set  $\Rightarrow$  'f list set  $\Rightarrow$ 
  nat list set  $\Rightarrow$ 
  (((('f staged-proof-data  $\times$  'f protocol-channel)  $\times$ 

```

$\text{unit list}) \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{staged-security-trace-fri-pair-challenge-query-path-fresh } P \ B \ Q \ \text{out}$
 $\longleftrightarrow$
 $\text{staged-security-trace-fri-pair-challenge-fresh } P \ B \ \text{out} \wedge$
 $\text{staged-security-with-data-state-query-index-list-path-fresh } Q \ \text{out}$
definition $\text{staged-security-composition-fri-pair-challenge-query-path-fresh}$
 $:: ('f \Rightarrow (\text{nat list} \times 'f \text{ list}) \text{ set}) \Rightarrow$
 $('f \Rightarrow 'f \text{ list set}) \Rightarrow \text{nat list set} \Rightarrow$
 $((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times$
 $\text{unit list}) \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{staged-security-composition-fri-pair-challenge-query-path-fresh } P \ B \ Q \ \text{out}$
 $\longleftrightarrow$
 $\text{staged-security-composition-fri-pair-challenge-fresh } P \ B \ \text{out} \wedge$
 $\text{staged-security-with-data-state-query-index-list-path-fresh } Q \ \text{out}$
definition $\text{verifier-after-trace-fri-raw-list-path-hit}$
 $:: 'f \times ('f \times 'f) \text{ list} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{nat list set} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{verifier-after-trace-fri-raw-list-path-hit header } s \ Q \ \text{out} \longleftrightarrow$
 $(\exists \text{ fr f-fl f-final s3 as s4 dg s5 s6 fl s7 final query-state rows.}$
 $\text{header} = (\text{fr}, \text{f-fl}) \wedge$
 $\text{Some} (\text{f-final}, \text{s3}) \in \text{set-dist} (\text{execute read } s) \wedge$
 $\text{Some} (\text{as}, \text{s4}) \in$
 $\text{set-dist} (\text{execute} (\text{mmap} (\text{replicate} (\text{length spec}) \text{alpha-round})) \text{s3}) \wedge$
 $\text{Some} (\text{dg}, \text{s5}) \in \text{set-dist} (\text{execute read } \text{s4}) \wedge$
 $\text{Some} ((), \text{s6}) \in$
 $\text{set-dist} (\text{execute} (\text{assert} (\text{to-nat dg} \leq \text{maxDegree})) \text{s5}) \wedge$
 $\text{Some} (\text{fl}, \text{s7}) \in$
 $\text{set-dist} (\text{execute}$
 $(\text{ntimes receive-composition-fri-commits}$
 $(\text{ceil-log} (\text{to-nat dg} + 1))) \text{s6}) \wedge$
 $\text{Some} (\text{final}, \text{query-state}) \in \text{set-dist} (\text{execute read } \text{s7}) \wedge$
 $\text{rows} \in \text{query-index-raw-list-preimage } Q \wedge$
 $\text{query-rounds-raw-list-path-hit query-state rows rounds out})$
lemma $\text{verifier-after-trace-fri-staged-query-path-fresh-imp-raw-list-path-hit:}$
assumes $\text{wf: staged-budget-wellformed budgets}$
and $\text{controlled: staged-adversary-controlled budgets } A$
and builder:
 $\text{Some} (\text{data}, \text{attacker-state}) \in$
 set-dist
 $(\text{execute} (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$
and prefix:

```

    Some ((fr, f-fl), prefix-state) ∈
      set-dist
        (execute verifier-trace-fri-prefix
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
  and suffix:
    out ∈
      set-dist
        (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
  and hit:
    staged-security-with-data-state-query-index-list-path-fresh Q
    (case out of
      None ⇒ None
      | Some (result, final-state) ⇒
        Some (((data, attacker-state), result), final-state))
  shows verifier-after-trace-fri-raw-list-path-hit (fr, f-fl) prefix-state Q out
proof (cases out)
case None
then show ?thesis
using hit
unfolding staged-security-with-data-state-query-index-list-path-fresh-def
by simp
next
case (Some pair)
then obtain result :: unit list and final-state where out-eq:
  out = Some (result, final-state)
by (cases pair) simp
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from suffix[unfolded out-eq verifier-after-trace-fri-def]
obtain f-final s3 as s4 dg s5 s6 fl s7 final query-state where
  read-trace-final:
    Some (f-final, s3) ∈ set-dist (execute read prefix-state)
  and alpha-out:
    Some (as, s4) ∈
      set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
  and read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
  and degree-assert:
    Some ((), s6) ∈
      set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
  and comp-fri:
    Some (fl, s7) ∈
      set-dist (execute
        (ntimes receive-composition-fri-commits
          (ceil-log (to-nat dg + 1)))) s6)
  and read-final:
    Some (final, query-state) ∈ set-dist (execute read s7)
  and query-out:

```

```

Some (result, final-state) ∈
  set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      query-state)
  by (auto elim!: set-dist-bindE)
from ntimes-verifier-query-rounds-outcome[OF query-out]
obtain raw-idxs query-idxs query-chunks where
  len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-chunks: length query-chunks = rounds
and tr-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and state-final:
  PState final-state =
    state-after-query-chunks (PState query-state) query-chunks rounds
and round-chunks:
  ∧i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
and lookups:
  ∧i. i < rounds ⇒
    fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks
        (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
by blast
have trace-prefix-res:
  length f-fl = ceil-log clength ∧
  PTranscript ?s = [fr] @ map snd f-fl @ PTranscript prefix-state ∧
  PState prefix-state =
    foldl concat (concat (PState ?s) fr) (map snd f-fl) ∧
  PQueryCounter prefix-state = PQueryCounter ?s
using verifier-trace-fri-prefix-outcome[OF prefix] by simp
from read-outcome[OF read-trace-final] obtain rest3 where
  tr-prefix: PTranscript prefix-state = f-final # rest3
and tr-s3: PTranscript s3 = rest3
and state-s3: PState s3 = concat (PState prefix-state) f-final
and counter-s3: PQueryCounter s3 = PQueryCounter prefix-state
by blast
have alpha-res:
  length as = length spec ∧
  PTranscript s3 = as @ PTranscript s4 ∧
  PState s4 = foldl concat (PState s3) as ∧

```

```

    PQueryCounter s4 = PQueryCounter s3
    using mmap-alpha-round-outcome[OF alpha-out] by simp
  from read-outcome[OF read-dg] obtain rest5 where
    tr-s4: PTranscript s4 = dg # rest5
    and tr-s5: PTranscript s5 = rest5
    and state-s5: PState s5 = concat (PState s4) dg
    and counter-s5: PQueryCounter s5 = PQueryCounter s4
    by blast
  have s6-eq: s6 = s5
    using degree-assert unfolding assert-def
    by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
  have comp-res:
    length fl = ceil-log (to-nat dg + 1) ∧
    PTranscript s6 = map snd fl @ PTranscript s7 ∧
    PState s7 = foldl concat (PState s6) (map snd fl) ∧
    PQueryCounter s7 = PQueryCounter s6
    using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
    by simp
  from read-outcome[OF read-final] obtain rest-query where
    tr-s7: PTranscript s7 = final # rest-query
    and tr-read-query: PTranscript query-state = rest-query
    and state-query: PState query-state = concat (PState s7) final
    and counter-query-s7: PQueryCounter query-state = PQueryCounter s7
    by blast
  have local-counter: PQueryCounter query-state = 0
    using trace-prefix-res counter-s3 alpha-res counter-s5 s6-eq comp-res
    counter-query-s7
    unfolding verifier-state-from-adversary-def by simp
  have prefix-lookup:
    ∧ i x. fmlookup (HashMap prefix-state)
      (QueryIndexChallenge i x) =
      fmlookup (HashMap ?s) (QueryIndexChallenge i x)
    by (rule verifier-trace-fri-prefix-preserves-query-lookup[OF prefix])
  have s3-lookup:
    HashMap s3 = HashMap prefix-state
    by (rule read-preserves-hash-map[OF read-trace-final])
  have s4-lookup:
    ∧ i x. fmlookup (HashMap s4) (QueryIndexChallenge i x) =
    fmlookup (HashMap s3) (QueryIndexChallenge i x)
  proof -
    fix i x
    show fmlookup (HashMap s4) (QueryIndexChallenge i x) =
    fmlookup (HashMap s3) (QueryIndexChallenge i x)
  proof (rule mmap-preserves-lookup[OF - alpha-out])
    fix m y u v
    assume m-in: m ∈ set (replicate (length spec) alpha-round)
    and out: Some (y, v) ∈ set-dist (execute m u)
    then have m-eq: m = alpha-round
    by simp
  end

```

```

show fmlookup (HashMap v) (QueryIndexChallenge i x) =
  fmlookup (HashMap u) (QueryIndexChallenge i x)
  using alpha-round-preserves-query-lookup[OF out[unfolded m-eq]] .
qed
qed
have s5-lookup:
  HashMap s5 = HashMap s4
  by (rule read-preserves-hash-map[OF read-dg])
have s7-lookup:
   $\bigwedge i x. \text{fmlookup (HashMap s7) (QueryIndexChallenge i x) =}$ 
    fmlookup (HashMap s6) (QueryIndexChallenge i x)
proof -
  fix i x
  show fmlookup (HashMap s7) (QueryIndexChallenge i x) =
    fmlookup (HashMap s6) (QueryIndexChallenge i x)
  proof (rule ntimes-preserves-lookup[OF - comp-fri])
    fix y u v
    assume out:
      Some (y, v)  $\in$ 
        set-dist (execute receive-composition-fri-commits u)
    obtain b r where y-eq: y = (b, r)
    by (cases y) simp
    show fmlookup (HashMap v) (QueryIndexChallenge i x) =
      fmlookup (HashMap u) (QueryIndexChallenge i x)
    by (rule receive-composition-fri-commits-preserves-query-lookup
      [OF out[unfolded y-eq]])
  qed
qed
have query-lookup:
  HashMap query-state = HashMap s7
  by (rule read-preserves-hash-map[OF read-final])
have header-actual:
  verifier-header-transcript ?s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
  using trace-prefix-res tr-prefix tr-s3 alpha-res tr-s4 tr-s5 s6-eq
    comp-res tr-s7 tr-read-query
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript

```



```

    [OF wf controlled builder])
have header-unique:
  fr = staged-trace-root data ∧
  map snd f-fl = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  map snd fl = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  PTranscript query-state = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header-actual staged-header]
by simp
have prefix-state-query:
  PState query-state = staged-query-start-hash data
using trace-prefix-res state-s3 alpha-res state-s5 s6-eq comp-res
  state-query header-unique
unfolding staged-query-start-hash-def
  staged-composition-fri-start-hash-def
  staged-trace-fri-start-hash-def
by simp
have chunks-eq: query-chunks = staged-query-chunks data
proof –
from checked-staged-transcript-program-query-chunks-match-verifier-lengths
  [OF builder]
obtain staged-query-idxs where
  staged-match:
    staged-query-chunks-match-verifier-lengths data staged-query-idxs
by blast
have query-idxs-len: length query-idxs = rounds
using query-idxs-eq len-row by simp
have staged-match-query-idxs:
  staged-query-chunks-match-verifier-lengths data query-idxs
by (rule staged-query-chunks-match-verifier-lengths-transfer
  [OF staged-match query-idxs-len])
have parser-chunk-len:
  ∧i. i < rounds ⇒
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)
proof –
fix i
assume i-bound: i < rounds
show
  length (query-chunks ! i) =
    verifier-query-round-transcript-length (query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
using verifier-query-round-chunk-length[OF round-chunks[OF i-bound]]

```

```

    header-unique
  by simp
qed
have concat-staged:
  List.concat query-chunks @ PTranscript final-state =
    List.concat (staged-query-chunks data)
  using tr-query header-unique by simp
then show ?thesis
  using query-chunks-eq-staged-if-matching-lengths
    [OF len-chunks concat-staged parser-chunk-len
      staged-match-query-idxs]
  by simp
qed
from hit out-eq obtain raw-idxs-hit query-idxs-hit where
  query-hit-in: query-idxs-hit ∈ Q
  and len-hit: length raw-idxs-hit = rounds
  and query-hit-eq:
    query-idxs-hit =
      map (λraw. index (to-nat raw)) raw-idxs-hit
  and lookup-hit:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs-hit ! i)
  and fresh-hit:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap ?s)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        None
  unfolding staged-security-with-data-state-query-index-list-path-fresh-def
  by auto
have raw-eq:
  ∧ i. i < rounds ⇒ raw-idxs ! i = raw-idxs-hit ! i
  using lookups lookup-hit local-counter prefix-state-query chunks-eq
  by simp
have raw-idxs-eq: raw-idxs = raw-idxs-hit
  by (rule nth-equalityI)
  (use len-raw len-hit raw-eq in simp-all)
have raw-in: raw-idxs ∈ query-index-raw-list-preimage Q
  unfolding query-index-raw-list-preimage-def
  using len-raw raw-idxs-eq query-hit-in query-hit-eq by blast
have query-lookup-start:
  ∧ i x. fmllookup (HashMap query-state) (QueryIndexChallenge i x) =

```

```

    fmlookup (HashMap ?s) (QueryIndexChallenge i x)
using prefix-lookup s3-lookup s4-lookup s5-lookup s6-eq
    s7-lookup query-lookup by simp
have local-fresh:
    query-index-list-path-fresh query-state rounds query-chunks raw-idxs
    unfolding query-index-list-path-fresh-def
proof (intro conjI allI impI)
    show length raw-idxs = rounds
    using len-raw .
next
    show length query-chunks = rounds
    using len-chunks .
next
    fix i
    assume i-bound: i < rounds
    let ?x =
        state-after-query-chunks (staged-query-start-hash data)
        (staged-query-chunks data) i
    have fmlookup (HashMap query-state)
        (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
        fmlookup (HashMap ?s) (QueryIndexChallenge i ?x)
    using local-counter prefix-state-query chunks-eq
        query-lookup-start[of i ?x] by simp
    also have ... = None
    by (rule fresh-hit[OF i-bound])
    finally show
        fmlookup (HashMap query-state)
        (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
        None .
qed
have path:
    query-rounds-raw-list-path-hit query-state raw-idxs rounds out
    unfolding query-rounds-raw-list-path-hit-def out-eq
    using len-raw len-chunks state-final local-fresh lookups
    by auto
have witness:
    (fr, f-fl) = (fr, f-fl) ∧
    Some (f-final, s3) ∈ set-dist (execute read prefix-state) ∧
    Some (as, s4) ∈
        set-dist (execute (mmap (replicate (length spec) alpha-round)) s3) ∧
    Some (dg, s5) ∈ set-dist (execute read s4) ∧
    Some ((), s6) ∈
        set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5) ∧
    Some (fl, s7) ∈
        set-dist (execute
            (ntimes receive-composition-fri-commits
            (ceil-log (to-nat dg + 1)))) s6) ∧

```

Some (final, query-state) ∈ set-dist (execute read s7) ∧
raw-idxs ∈ query-index-raw-list-preimage Q ∧
query-rounds-raw-list-path-hit query-state raw-idxs rounds out
using *read-trace-final alpha-out read-dg degree-assert comp-fri*
read-final raw-in path **by** *simp*
show *?thesis*
unfolding *verifier-after-trace-fri-raw-list-path-hit-def*
using *witness* **by** *blast*
qed

lemma *verifier-after-composition-fri-staged-query-path-fresh-imp-raw-list-path-hit:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *builder:*
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix:*
Some (header, prefix-state) ∈
set-dist
(execute
verifier-composition-fri-prefix
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))))
and *suffix:*
out ∈
set-dist
(execute (verifier-after-composition-fri header) prefix-state)
and *hit:*
staged-security-with-data-state-query-index-list-path-fresh Q
(case out of
None ⇒ None
| Some (result, final-state) ⇒
Some (((data, attacker-state), result), final-state)))
shows *verifier-after-composition-fri-raw-list-path-hit prefix-state Q out*
proof *(cases out)*
case *None*
then show *?thesis*
using *hit*
unfolding *staged-security-with-data-state-query-index-list-path-fresh-def*
by *simp*
next
case *(Some pair)*
then obtain *result :: unit list* **and** *final-state* **where** *out-eq:*
out = Some (result, final-state)
by *(cases pair) simp*
let *?s =*
verifier-state-from-adversary attacker-state

```

    (staged-proof-transcript data)
obtain fr f-fl f-final as dg fl where header-eq:
    header = (fr, f-fl, f-final, as, dg, fl)
    by (cases header) auto
from suffix[unfolded out-eq header-eq verifier-after-composition-fri-def]
obtain final query-state where
    read-final:
        Some (final, query-state) ∈ set-dist (execute read prefix-state)
    and query-out:
        Some (result, final-state) ∈
            set-dist
                (execute
                    (ntimes
                        (verifier-query-round-program fr f-fl f-final as fl final)
                        rounds)
                    query-state)
    by (auto elim!: set-dist-bindE)
from ntimes-verifier-query-rounds-outcome[OF query-out]
obtain raw-idxs query-idxs query-chunks where
    len-raw: length raw-idxs = rounds
    and query-idxs-eq:
        query-idxs = map (λraw. index (to-nat raw)) raw-idxs
    and len-chunks: length query-chunks = rounds
    and tr-query:
        PTranscript query-state =
            List.concat query-chunks @ PTranscript final-state
    and state-final:
        PState final-state =
            state-after-query-chunks (PState query-state) query-chunks rounds
    and round-chunks:
        ∧ i. i < rounds ⇒
            verifier-query-round-chunk (query-idxs ! i)
                (map snd f-fl) (map snd fl) (query-chunks ! i)
    and lookups:
        ∧ i. i < rounds ⇒
            fmlookup (HashMap final-state)
                (QueryIndexChallenge (PQueryCounter query-state + i)
                    (state-after-query-chunks
                        (PState query-state) query-chunks i)) =
                Some (raw-idxs ! i)
    by blast
from read-outcome[OF read-final] have query-counter:
    PQueryCounter query-state = PQueryCounter prefix-state
    by blast
have prefix-counter:
    PQueryCounter prefix-state = PQueryCounter ?s
    using verifier-composition-fri-prefix-outcome
        [OF prefix[unfolded header-eq]]
    by simp

```

```

have local-counter: PQueryCounter query-state = 0
  using query-counter prefix-counter
  unfolding verifier-state-from-adversary-def by simp
have prefix-lookup:
   $\wedge i x$ . fmlookup (HashMap prefix-state)
    (QueryIndexChallenge i x) =
    fmlookup (HashMap ?s) (QueryIndexChallenge i x)
  by (rule verifier-composition-fri-prefix-preserves-query-lookup
    [OF prefix[unfolded header-eq]])
have read-lookup: HashMap query-state = HashMap prefix-state
  by (rule read-preserves-hash-map[OF read-final])
have prefix-res:
  length f-fl = ceil-log clength  $\wedge$ 
  length as = length spec  $\wedge$ 
  length fl = ceil-log (to-nat dg + 1)  $\wedge$ 
  PTranscript ?s =
    [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
    map snd fl @ PTranscript prefix-state  $\wedge$ 
  PState prefix-state =
    foldl concat
      (concat
        (foldl concat
          (concat
            (foldl concat (concat (PState ?s) fr) (map snd f-fl))
              f-final)
            as)
          dg)
        (map snd fl))
  using verifier-composition-fri-prefix-outcome
    [OF prefix[unfolded header-eq]]
  by simp
from read-outcome[OF read-final] obtain rest-query where
  tr-prefix: PTranscript prefix-state = final # rest-query
  and state-query: PState query-state = concat (PState prefix-state) final
  and tr-read-query: PTranscript query-state = rest-query
  by blast
have header-actual:
  verifier-header-transcript ?s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
  unfolding verifier-header-transcript-def verifier-header-messages-def
  using prefix-res tr-prefix tr-read-query by simp
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)

```

```

    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
have header-unique:
  fr = staged-trace-root data ∧
  map snd f-fl = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  map snd fl = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  PTranscript query-state = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header-actual staged-header]
by simp
have prefix-state-query:
  PState query-state = staged-query-start-hash data
using state-query prefix-res header-unique
unfolding staged-query-start-hash-def
  staged-composition-fri-start-hash-def
  staged-trace-fri-start-hash-def
by simp
have chunks-eq: query-chunks = staged-query-chunks data
proof -
  from checked-staged-transcript-program-query-chunks-match-verifier-lengths
    [OF builder]
  obtain staged-query-idxs where
    staged-match:
      staged-query-chunks-match-verifier-lengths data staged-query-idxs
  by blast
have query-idxs-len: length query-idxs = rounds
  using query-idxs-eq len-raw by simp
have staged-match-query-idxs:
  staged-query-chunks-match-verifier-lengths data query-idxs
  by (rule staged-query-chunks-match-verifier-lengths-transfer
      [OF staged-match query-idxs-len])
have parser-chunk-len:
  ∧ i. i < rounds ⇒
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
proof -
  fix i
  assume i-bound: i < rounds
  show
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
      (staged-trace-fri-roots data)

```

```

      (staged-composition-fri-roots data)
    using verifier-query-round-chunk-length[OF round-chunks[OF i-bound]]
      header-unique
    by simp
qed
have concat-staged:
  List.concat query-chunks @ PTranscript final-state =
    List.concat (staged-query-chunks data)
  using tr-query header-unique by simp
then show ?thesis
  using query-chunks-eq-staged-if-matching-lengths
    [OF len-chunks concat-staged parser-chunk-len
      staged-match-query-idxs]
  by simp
qed
from hit out-eq obtain raw-idxs-hit query-idxs-hit where
  query-hit-in: query-idxs-hit ∈ Q
  and len-hit: length raw-idxs-hit = rounds
  and query-hit-eq:
    query-idxs-hit =
      map (λraw. index (to-nat raw)) raw-idxs-hit
  and lookup-hit:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs-hit ! i)
  and fresh-hit:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap ?s)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        None
  unfolding staged-security-with-data-state-query-index-list-path-fresh-def
  by auto
have raw-eq:
  ∧ i. i < rounds ⇒ raw-idxs ! i = raw-idxs-hit ! i
  using lookups lookup-hit local-counter prefix-state-query chunks-eq
  by simp
have raw-idxs-eq: raw-idxs = raw-idxs-hit
  by (rule nth-equalityI)
  (use len-raw len-hit raw-eq in simp-all)
have raw-in: raw-idxs ∈ query-index-raw-list-preimage Q
  unfolding query-index-raw-list-preimage-def
  using len-raw raw-idxs-eq query-hit-in query-hit-eq by blast

```



```

have query-lookup-start:
   $\wedge i x.$  fmlookup (HashMap query-state) (QueryIndexChallenge i x) =
    fmlookup (HashMap ?s) (QueryIndexChallenge i x)
  using prefix-lookup read-lookup by simp
have local-fresh:
  query-index-list-path-fresh query-state rounds query-chunks raw-idxs
  unfolding query-index-list-path-fresh-def
proof (intro conjI allI impI)
  show length raw-idxs = rounds
    using len-raw .
next
  show length query-chunks = rounds
    using len-chunks .
next
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  let ?x =
    state-after-query-chunks (staged-query-start-hash data)
    (staged-query-chunks data) i
  have fmlookup (HashMap query-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    fmlookup (HashMap ?s) (QueryIndexChallenge i ?x)
  using local-counter prefix-state-query chunks-eq
    query-lookup-start[of i ?x] by simp
  also have ... = None
    by (rule fresh-hit[OF i-bound])
  finally show
    fmlookup (HashMap query-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
      None .
qed
have path:
  query-rounds-raw-list-path-hit query-state raw-idxs rounds out
  unfolding query-rounds-raw-list-path-hit-def out-eq
  using len-raw len-chunks state-final local-fresh lookups
  by auto
show ?thesis
  unfolding verifier-after-composition-fri-raw-list-path-hit-def
  by (intro exI[of - final] exI[of - query-state]
    exI[of - raw-idxs] conjI)
    (use read-final raw-in path in simp-all)
qed

lemma staged-security-trace-fri-pair-challenge-fresh-imp-query-index-list-hit:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:

```

```

    out ∈
      set-dist
        (execute (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
    and projection: fst ‘ P ⊆ Q
    and hit: staged-security-trace-fri-pair-challenge-fresh P B out
  shows staged-security-with-data-query-index-list-set-hit Q out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding staged-security-trace-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
    staged-security-with-data-query-index-list-set-hit-def
  by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed) (auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from hit out-eq have pair-hit:
  trace-fri-query-challenge-pair-set-hit ?s P
  (Some (result, final-state))
  unfolding staged-security-trace-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
  by simp
have trace-hit:
  trace-fri-query-index-list-set-hit ?s Q
  (Some (result, final-state))
  by (rule trace-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit
    [OF pair-hit projection])
from trace-hit obtain trace-roots trace-bs trace-final dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
  fri:
    accepted-fri-opening-transcript ?s (Some (result, final-state))
    trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final query-idxs trace-round-layers
    composition-round-layers
  and query-in: query-idxs ∈ Q
  unfolding trace-fri-query-index-list-set-hit-def by blast
from accepted-fri-opening-transcript-imp-accepted-transcript-shape[OF fri]
obtain as where shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as query-idxs
  by blast
from checked-staged-security-with-data-state-accepted-shape-query-keys

```

```

    [OF wf controlled support[unfolded out-eq] shape]
obtain raw-idxs where
  len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and lookup:
   $\bigwedge i. i < rounds \implies$ 
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)
by blast
show ?thesis
  unfolding out-eq staged-security-with-data-query-index-list-set-hit-def
  using len-raw query-in query-idxs-eq lookup by auto
qed

lemma staged-security-composition-fri-pair-challenge-fresh-imp-query-index-list-hit:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out  $\in$ 
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
and projection:  $\bigwedge dg. \text{fst } P \text{ dg} \subseteq Q$ 
and hit: staged-security-composition-fri-pair-challenge-fresh P B out
shows staged-security-with-data-query-index-list-set-hit Q out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding staged-security-composition-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
    staged-security-with-data-query-index-list-set-hit-def
  by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed) (auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from hit out-eq have pair-hit:
  composition-fri-query-challenge-pair-set-hit ?s P
    (Some (result, final-state))

```

```

unfolding staged-security-composition-fri-pair-challenge-fresh-def
  staged-security-with-data-state-verifier-event-def
by simp
have composition-hit:
  composition-fri-query-index-list-set-hit ?s Q
  (Some (result, final-state))
by (rule
  composition-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit
  [OF pair-hit projection])
from composition-hit obtain trace-roots trace-bs trace-final dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
  fri:
  accepted-fri-opening-transcript ?s (Some (result, final-state))
  trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final query-idxs trace-round-layers
  composition-round-layers
and query-in: query-idxs  $\in$  Q
unfolding composition-fri-query-index-list-set-hit-def by blast
from accepted-fri-opening-transcript-imp-accepted-transcript-shape[OF fri]
obtain as where shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as query-idxs
by blast
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support[unfolded out-eq] shape]
obtain raw-idxs where
  len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
  fmlookup (HashMap final-state)
  (QueryIndexChallenge i
  (state-after-query-chunks
  (staged-query-start-hash data)
  (staged-query-chunks data) i)) =
  Some (raw-idxs ! i)
by blast
show ?thesis
unfolding out-eq staged-security-with-data-query-index-list-set-hit-def
using len-raw query-in query-idxs-eq lookup by auto
qed

lemma checked-staged-security-trace-fri-pair-challenge-fresh-bound-from-query-path-fresh:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and projection: fst '  $P \subseteq Q$ 
and path-branch-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)

```

$(\text{staged-security-trace-fri-pair-challenge-query-path-fresh } P \ B \ Q)$
 $\text{adversary-initial-state} \leq C$

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-trace-fri-pair-challenge-fresh } P \ B)$
 $\text{adversary-initial-state} \leq$
 $C +$
 $\text{hash-relation-budget-value}$
 $(\text{query-index-raw-list-relation-fiber-bound } Q)$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$

proof –

have *event-mono*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-trace-fri-pair-challenge-fresh } P \ B)$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\lambda \text{out.}$
 $\text{staged-security-trace-fri-pair-challenge-query-path-fresh } P \ B \ Q \ \text{out} \ \vee$
 $\text{staged-security-with-data-state-query-index-list-prequery-hit } A \ Q \ \text{out})$
 $\text{adversary-initial-state}$

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support*:

$\text{out} \in$
 set-dist
 $(\text{execute } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{adversary-initial-state})$

and *hit*: $\text{staged-security-trace-fri-pair-challenge-fresh } P \ B \ \text{out}$

have *query-hit*:

$\text{staged-security-with-data-query-index-list-set-hit } Q \ \text{out}$
by (rule
 $\text{staged-security-trace-fri-pair-challenge-fresh-imp-query-index-list-hit}$
 $[\text{OF wf controlled support projection hit}])$

show

$\text{staged-security-trace-fri-pair-challenge-query-path-fresh } P \ B \ Q \ \text{out} \ \vee$
 $\text{staged-security-with-data-state-query-index-list-prequery-hit } A \ Q \ \text{out}$

proof (cases *out*)

case *None*

then show *?thesis*

using *query-hit*
unfolding $\text{staged-security-with-data-query-index-list-set-hit-def}$
by *simp*

next

case (*Some packed*)

then obtain *data attacker-state result final-state* **where** *out-eq*:

$\text{out} = \text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state})$
by (cases *packed*) (auto split: *prod.splits*)

have *split*:

```

    staged-security-with-data-state-query-index-list-path-fresh Q
      (Some (((data, attacker-state), result), final-state)) ∨
    hash-relation-hit
      (checked-staged-query-index-list-relation A Q)
    adversary-initial-state attacker-state
  apply (rule
    checked-staged-security-with-data-state-query-index-list-fresh-or-prequeried)
  using support query-hit unfolding out-eq by simp-all
then show ?thesis
proof
  assume fresh:
    staged-security-with-data-state-query-index-list-path-fresh Q
      (Some (((data, attacker-state), result), final-state))
  have pair-fresh:
    staged-security-trace-fri-pair-challenge-fresh P B
      (Some (((data, attacker-state), result), final-state))
  using hit out-eq by simp
  then have
    staged-security-trace-fri-pair-challenge-query-path-fresh P B Q
      out
  unfolding out-eq
    staged-security-trace-fri-pair-challenge-query-path-fresh-def
  using fresh pair-fresh by simp
  then show ?thesis ..
next
  assume prequery:
    hash-relation-hit
      (checked-staged-query-index-list-relation A Q)
    adversary-initial-state attacker-state
  then have
    staged-security-with-data-state-query-index-list-prequery-hit A Q
      out
  unfolding out-eq
    staged-security-with-data-state-query-index-list-prequery-hit-def
  by simp
  then show ?thesis ..
qed
qed
qed
have union-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (λout.
      staged-security-trace-fri-pair-challenge-query-path-fresh P B Q out ∨
      staged-security-with-data-state-query-index-list-prequery-hit A Q out)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-trace-fri-pair-challenge-query-path-fresh P B Q)
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)

```

```

      (staged-security-with-data-state-query-index-list-prequery-hit A Q)
      adversary-initial-state
    by (rule wp-event-union-bound)
  have prequery-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-index-list-prequery-hit A Q)
      adversary-initial-state ≤
      hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound Q)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
    by (rule
      checked-staged-security-with-data-state-query-index-list-prequery-hit-bound
      [OF wf controlled])
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-trace-fri-pair-challenge-fresh P B)
    adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-trace-fri-pair-challenge-query-path-fresh P B Q)
      adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-index-list-prequery-hit A Q)
      adversary-initial-state
    by (rule order-trans[OF event-mono union-bound])
  also have ... ≤
    C +
    hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
    by (rule add-mono[OF path-branch-bound prequery-bound])
  finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-pair-challenge-fresh-bound-from-query-path-fresh:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and projection:  $\bigwedge dg. \text{fst } dg \subseteq P \implies dg \subseteq Q$ 
  and path-branch-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-composition-fri-pair-challenge-query-path-fresh P B Q)
      adversary-initial-state ≤ C
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-composition-fri-pair-challenge-fresh P B)
      adversary-initial-state ≤
      C +
      hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound Q)

```

```

      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)
proof –
  have event-mono:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-composition-fri-pair-challenge-fresh P B)
      adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (λout.
        staged-security-composition-fri-pair-challenge-query-path-fresh P B Q out ∨
        staged-security-with-data-state-query-index-list-prequery-hit A Q out)
      adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
      set-dist
        (execute (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  and hit: staged-security-composition-fri-pair-challenge-fresh P B out
have query-hit:
  staged-security-with-data-query-index-list-set-hit Q out
  by (rule
    staged-security-composition-fri-pair-challenge-fresh-imp-query-index-list-hit
    [OF wf controlled support projection hit])
show
  staged-security-composition-fri-pair-challenge-query-path-fresh P B Q out ∨
  staged-security-with-data-state-query-index-list-prequery-hit A Q out
proof (cases out)
  case None
  then show ?thesis
    using query-hit
    unfolding staged-security-with-data-query-index-list-set-hit-def
    by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed) (auto split: prod.splits)
  have split:
    staged-security-with-data-state-query-index-list-path-fresh Q
      (Some (((data, attacker-state), result), final-state)) ∨
    hash-relation-hit
      (checked-staged-query-index-list-relation A Q)
      adversary-initial-state attacker-state
  apply (rule
    checked-staged-security-with-data-state-query-index-list-fresh-or-prequeried)
  using support query-hit unfolding out-eq by simp-all
  then show ?thesis

```



```

proof
  assume fresh:
    staged-security-with-data-state-query-index-list-path-fresh Q
    (Some (((data, attacker-state), result), final-state))
  have pair-fresh:
    staged-security-composition-fri-pair-challenge-fresh P B
    (Some (((data, attacker-state), result), final-state))
  using hit out-eq by simp
  then have
    staged-security-composition-fri-pair-challenge-query-path-fresh
      P B Q out
  unfolding out-eq
    staged-security-composition-fri-pair-challenge-query-path-fresh-def
  using fresh pair-fresh by simp
  then show ?thesis ..
next
  assume prequery:
    hash-relation-hit
    (checked-staged-query-index-list-relation A Q)
    adversary-initial-state attacker-state
  then have
    staged-security-with-data-state-query-index-list-prequery-hit A Q
    out
  unfolding out-eq
    staged-security-with-data-state-query-index-list-prequery-hit-def
  by simp
  then show ?thesis ..
qed
qed
qed
have union-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  ( $\lambda out.$ 
    staged-security-composition-fri-pair-challenge-query-path-fresh P B Q out  $\vee$ 
    staged-security-with-data-state-query-index-list-prequery-hit A Q out)
  adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-composition-fri-pair-challenge-query-path-fresh P B Q)
  adversary-initial-state  $+$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-query-index-list-prequery-hit A Q)
  adversary-initial-state
by (rule wp-event-union-bound)
have prequery-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-query-index-list-prequery-hit A Q)
  adversary-initial-state  $\leq$ 
  hash-relation-budget-value
  (query-index-raw-list-relation-fiber-bound Q)

```

(staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
by (rule
 checked-staged-security-with-data-state-query-index-list-prequery-hit-bound
 [OF wf controlled])
have wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-composition-fri-pair-challenge-fresh P B)
 adversary-initial-state ≤
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-composition-fri-pair-challenge-query-path-fresh P B Q)
 adversary-initial-state +
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-query-index-list-prequery-hit A Q)
 adversary-initial-state
by (rule order-trans[OF event-mono union-bound])
also have ... ≤
 C +
 hash-relation-budget-value
 (query-index-raw-list-relation-fiber-bound Q)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
by (rule add-mono[OF path-branch-bound prequery-bound])
finally show ?thesis **by** simp
qed

lemma checked-staged-security-composition-fri-pair-challenge-query-path-fresh-fixed-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and challenges-space:
 challenges ∈ fri-challenge-space (ceil-log (to-nat dg + 1))
and projection:
 fst ‘ (P dg ∩ (UNIV × {challenges})) ⊆ Q
and subset: Q ⊆ fri-query-index-list-space
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-composition-fri-pair-challenge-query-path-fresh P
 (λdg'. if dg' = dg then {challenges} else {}) Q)
 adversary-initial-state ≤
 (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
 (nnreal (card Q) *
 (1 / nnreal (card query-sample-space)) ^ rounds)
proof (rule checked-staged-security-with-data-state-bound-from-data-cont)
let ?B = λdg'. if dg' = dg then {challenges} else {}
let ?D =
 nnreal (card Q) *
 (1 / nnreal (card query-sample-space)) ^ rounds
show ¬ staged-security-composition-fri-pair-challenge-query-path-fresh
 P ?B Q None
unfolding

```

    staged-security-composition-fri-pair-challenge-query-path-fresh-def
    staged-security-composition-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
  by simp
next
fix data attacker-state
assume builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
let ?B = λdg'. if dg' = dg then {challenges} else {}
let ?Event =
  λout.
    staged-security-composition-fri-pair-challenge-query-path-fresh P ?B Q
    (case out of
      None ⇒ None
      | Some (result, final-state) ⇒
        Some (((data, attacker-state), result), final-state))
let ?Head =
  λout. case out of
    None ⇒ False
    | Some ((fr, f-fl, f-final, as, dg', fl), -) ⇒
      dg' = dg ∧ map fst fl = challenges ∧
      composition-fri-challenge-path-fresh ?s fr (map snd f-fl)
      f-final as dg' (map snd fl) (length (map snd fl))
show wp-event verify-monad ?Event ?s ≤
  (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
  (nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
unfolding verify-monad-composition-fri-decomposition
proof (rule order-trans)
show wp-event
  (verifier-composition-fri-prefix ≫=
    verifier-after-composition-fri)
  ?Event ?s
≤ wp-event verifier-composition-fri-prefix ?Head ?s *
  (nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
proof (rule wp-event-bind-bound-by-head-and-cont)
show ¬ ?Event None
unfolding
  staged-security-composition-fri-pair-challenge-query-path-fresh-def
  staged-security-composition-fri-pair-challenge-fresh-def
  staged-security-with-data-state-verifier-event-def
by simp

```

```

next
  fix header prefix-state
  assume prefix:
    Some (header, prefix-state) ∈
      set-dist (execute verifier-composition-fri-prefix ?s)
  and not-head:  $\neg$  ?Head (Some (header, prefix-state))
  obtain fr f-fl f-final as dg' fl where header-eq:
    header = (fr, f-fl, f-final, as, dg', fl)
  by (cases header) auto
  have not-head':
    dg'  $\neq$  dg  $\vee$  map fst fl  $\neq$  challenges  $\vee$ 
       $\neg$  composition-fri-challenge-path-fresh ?s fr (map snd f-fl)
      f-final as dg' (map snd fl) (length (map snd fl))
  using not-head unfolding header-eq by simp
  show wp-event (verifier-after-composition-fri header) ?Event
    prefix-state = 0
  proof (rule antisym)
    show wp-event (verifier-after-composition-fri header) ?Event
      prefix-state  $\leq$  0
    proof (rule order-trans)
      show wp-event (verifier-after-composition-fri header) ?Event
        prefix-state  $\leq$ 
          wp-event (verifier-after-composition-fri header)
            ( $\lambda$ -. False) prefix-state
    proof (rule wp-event-mono-on-support)
      fix out
      assume suffix:
        out ∈
          set-dist
            (execute (verifier-after-composition-fri header)
              prefix-state)
      and hit: ?Event out
      show False
      proof (cases out)
        case None
        then show ?thesis
          using hit
          unfolding
            staged-security-composition-fri-pair-challenge-query-path-fresh-def
            staged-security-composition-fri-pair-challenge-fresh-def
            staged-security-with-data-state-verifier-event-def
          by simp
      next
        case (Some pair)
        then obtain result :: unit list and final-state where out-eq:
          out = Some (result, final-state)
        by (cases pair) simp
      from hit out-eq have fresh:
        composition-fri-challenge-list-fresh-hit ?s ?B

```

```

    (Some (result, final-state))
and pair-hit:
composition-fri-query-challenge-pair-set-hit ?s P
    (Some (result, final-state))
unfolding
    staged-security-composition-fri-pair-challenge-query-path-fresh-def
    staged-security-composition-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
by simp-all
have prefix':
    Some ((fr, f-fl, f-final, as, dg', fl), prefix-state) ∈
        set-dist (execute verifier-composition-fri-prefix ?s)
using prefix header-eq by simp
have suffix':
    Some (result, final-state) ∈
        set-dist
        (execute
            (verifier-after-composition-fri
                (fr, f-fl, f-final, as, dg', fl))
            prefix-state)
using suffix header-eq out-eq by simp
from pair-hit obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers where
openings:
    accepted-fri-opening-transcript ?s
    (Some (result, final-state)) trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final query-idxs trace-round-layers
    composition-round-layers
unfolding composition-fri-query-challenge-pair-set-hit-def
by blast
have pair-challenges:
    accepted-fri-challenges ?s (Some (result, final-state))
    trace-bs fri-dg composition-bs
by (rule accepted-fri-opening-transcript-challenges
    [OF openings])
from fresh obtain trace-bs' dg'' comp-bs' where
fresh-challenges:
    accepted-fri-challenges ?s (Some (result, final-state))
    trace-bs' dg'' comp-bs'
and comp-bs'-in:
    comp-bs' ∈
        (if dg'' = dg then {challenges} else {})
unfolding composition-fri-challenge-list-fresh-hit-def
by blast
have challenge-egs:
    fri-dg = dg'' ∧ composition-bs = comp-bs'
using accepted-fri-challenges-unique

```

```

    [OF pair-challenges fresh-challenges]
  by simp
  have prefix-eqs:
    fri-dg = dg'  $\wedge$  composition-bs = map fst fl
  by (rule
    verifier-after-composition-fri-header-agrees-with-accepted-challenges
    [OF prefix' suffix' pair-challenges])
  have concrete-fresh:
    composition-fri-challenge-path-fresh ?s fr (map snd f-fl)
    f-final as dg' (map snd fl) (length (map snd fl))
  by (rule
    verifier-after-composition-fri-header-roots-agree-with-fresh-hit
    [OF prefix' suffix' fresh])
  have dg''-eq: dg'' = dg
    using comp-bs'-in by (cases dg'' = dg) auto
  have dg'-eq: dg' = dg
    using challenge-eqs prefix-eqs dg''-eq by simp
  have fl-challenges: map fst fl = challenges
    using comp-bs'-in prefix-eqs challenge-eqs dg''-eq by simp
  have head-holds:
    dg' = dg  $\wedge$  map fst fl = challenges  $\wedge$ 
    composition-fri-challenge-path-fresh ?s fr (map snd f-fl)
    f-final as dg' (map snd fl) (length (map snd fl))
    using dg'-eq fl-challenges concrete-fresh by simp
  show ?thesis
    using not-head' head-holds by blast
qed
qed
next
  show wp-event (verifier-after-composition-fri header)
    ( $\lambda$ -. False) prefix-state  $\leq$  0
    unfolding wp-event-def wp-def dist-expect-def by simp
qed
next
  show 0  $\leq$ 
    wp-event (verifier-after-composition-fri header) ?Event
    prefix-state
  by simp
qed
next
  fix header prefix-state
  assume prefix:
    Some (header, prefix-state)  $\in$ 
    set-dist (execute verifier-composition-fri-prefix ?s)
  and head: ?Head (Some (header, prefix-state))
  have event-mono:
    wp-event (verifier-after-composition-fri header) ?Event
    prefix-state  $\leq$ 
    wp-event (verifier-after-composition-fri header)

```

```

    (verifier-after-composition-fri-raw-list-path-hit prefix-state Q)
    prefix-state
proof (rule wp-event-mono-on-support)
  fix out
  assume suffix:
    out ∈
      set-dist
      (execute (verifier-after-composition-fri header) prefix-state)
  and hit: ?Event out
from hit have path-hit:
    staged-security-with-data-state-query-index-list-path-fresh Q
    (case out of
      None ⇒ None
    | Some (result, final-state) ⇒
      Some (((data, attacker-state), result), final-state))
  unfolding
    staged-security-composition-fri-pair-challenge-query-path-fresh-def
  by simp
show
  verifier-after-composition-fri-raw-list-path-hit prefix-state Q
  out
  by (rule
    verifier-after-composition-fri-staged-query-path-fresh-imp-raw-list-path-hit
    [OF wf controlled builder prefix suffix path-hit])
qed
also have ... ≤
  nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds
  by (rule wp-verifier-after-composition-fri-raw-list-path-product-bound
    [OF subset])
finally show
  wp-event (verifier-after-composition-fri header) ?Event
  prefix-state
  ≤ nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds .
qed
next
show wp-event verifier-composition-fri-prefix ?Head ?s *
  (nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
  ≤
  (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
  (nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
proof (rule mult-right-mono)
show wp-event verifier-composition-fri-prefix ?Head ?s ≤
  1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
proof –
  let ?B = λdg'. if dg' = dg then {challenges} else {}

```

```

let ?HeadSet =
   $\lambda out. \text{case out of}$ 
    None  $\Rightarrow$  False
  | Some ((fr, f-fl, f-final, as, dg', fl), -)  $\Rightarrow$ 
    map fst fl  $\in$  ?B dg'  $\wedge$ 
    composition-fri-challenge-path-fresh ?s fr (map snd f-fl)
    f-final as dg' (map snd fl) (length (map snd fl))
have mono:
  wp-event verifier-composition-fri-prefix ?Head ?s  $\leq$ 
  wp-event verifier-composition-fri-prefix ?HeadSet ?s
by (rule wp-event-mono) (auto split: option.splits prod.splits)
have subset:
   $\bigwedge dg'. ?B dg' \subseteq$ 
    fri-challenge-space (ceil-log (to-nat dg' + 1))
using challenges-space by auto
have bound:
   $\bigwedge dg'. \text{nnreal (card (?B dg'))} /$ 
    nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg' + 1))  $\leq$ 
    1 / nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg + 1))
using challenges-space by auto
have set-bound:
  wp-event verifier-composition-fri-prefix ?HeadSet ?s  $\leq$ 
  1 / nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg + 1))
by (rule
  wp-verifier-composition-fri-prefix-challenge-list-actual-fresh-bound
  [OF subset bound])
show ?thesis
by (rule order-trans[OF mono set-bound])
qed
show 0  $\leq$  nnreal (card Q) *
  (1 / nnreal (card query-sample-space))  $\wedge$  rounds
by simp
qed
qed
qed

```

lemma checked-staged-security-composition-fri-pair-challenge-query-path-fresh-fiber-sum:

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and challenge-subset:

$\bigwedge dg. B dg \subseteq \text{fri-challenge-space (ceil-log (to-nat dg + 1))}$

and projection:

$\bigwedge dg \text{ challenges.}$

challenges $\in B dg \implies$

fst ' (P dg $\cap$ (UNIV $\times$ {challenges})) $\subseteq$

Q dg challenges

and subset:

$\bigwedge dg \text{ challenges. challenges} \in B dg \implies$

Q dg challenges $\subseteq \text{fri-query-index-list-space}$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-composition-fri-pair-challenge-query-path-fresh P
B
 $(\bigcup dg \in UNIV. \bigcup challenges \in B dg. Q dg challenges))$
adversary-initial-state $\leq$
 $(\sum dg \in UNIV.$
 $\sum challenges \in B dg.$
 $(1 / nnreal (CARD('f) \wedge ceil-log (to-nat dg + 1))) *$
 $(nnreal (card (Q dg challenges)) *$
 $(1 / nnreal (card query-sample-space)) \wedge rounds))$

proof –

let $?Q = (\bigcup dg \in UNIV. \bigcup challenges \in B dg. Q dg challenges)$
let $?I = SIGMA dg:UNIV. B dg$
let $?E =$
 $\lambda p.$
 $staged-security-composition-fri-pair-challenge-query-path-fresh P$
 $(\lambda dg'. \text{if } dg' = fst\ p \text{ then } \{snd\ p\} \text{ else } \{\})$
 $(Q (fst\ p) (snd\ p))$
let $?C =$
 $\lambda p.$
 $(1 / nnreal (CARD('f) \wedge ceil-log (to-nat (fst\ p) + 1))) *$
 $(nnreal (card (Q (fst\ p) (snd\ p))) *$
 $(1 / nnreal (card query-sample-space)) \wedge rounds)$

have *split*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-composition-fri-pair-challenge-query-path-fresh P B ?Q)
adversary-initial-state $\leq$
wp-event (checked-staged-security-experiment-with-data-state A)
 $(\lambda out. \exists p \in ?I. ?E\ p\ out)$
adversary-initial-state

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support*:

out $\in$
set-dist
 $(execute (checked-staged-security-experiment-with-data-state A)$
 $adversary-initial-state)$

and *hit*:

staged-security-composition-fri-pair-challenge-query-path-fresh P B ?Q
out

show $\exists p \in ?I. ?E\ p\ out$

proof (cases *out*)

case *None*

then show *?thesis*

using *hit*

unfolding

staged-security-composition-fri-pair-challenge-query-path-fresh-def
staged-security-composition-fri-pair-challenge-fresh-def

```

    staged-security-with-data-state-verifier-event-def
  by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed) (auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from hit out-eq have pair-hit:
  composition-fri-query-challenge-pair-set-hit ?s P
  (Some (result, final-state))
and fresh:
  composition-fri-challenge-list-fresh-hit ?s B
  (Some (result, final-state))
and path:
  staged-security-with-data-state-query-index-list-path-fresh ?Q out
unfolding
  staged-security-composition-fri-pair-challenge-query-path-fresh-def
  staged-security-composition-fri-pair-challenge-fresh-def
  staged-security-with-data-state-verifier-event-def
by simp-all
from fresh obtain trace-bs dg comp-bs fr trace-roots trace-final as
  composition-roots final rest where
  accepted:
    accepted-fri-challenges ?s (Some (result, final-state))
    trace-bs dg comp-bs
and comp-bs-in: comp-bs ∈ B dg
and header:
  verifier-header-transcript ?s fr trace-roots trace-final as dg
  composition-roots final rest
and fresh-path:
  composition-fri-challenge-path-fresh ?s fr trace-roots
  trace-final as dg composition-roots (length composition-roots)
unfolding composition-fri-challenge-list-fresh-hit-def by blast
from pair-hit obtain trace-roots' trace-bs' trace-final' fri-dg
  composition-roots' composition-bs' composition-final' query-idxs
  trace-round-layers composition-round-layers where
  openings:
    accepted-fri-opening-transcript ?s
    (Some (result, final-state)) trace-roots' trace-bs'
    trace-final' fri-dg composition-roots' composition-bs'
    composition-final' query-idxs trace-round-layers
    composition-round-layers
and pair: (query-idxs, composition-bs') ∈ P fri-dg
unfolding composition-fri-query-challenge-pair-set-hit-def by blast
have pair-challenges:
  accepted-fri-challenges ?s (Some (result, final-state))

```

```

    trace-bs' fri-dg composition-bs'
  by (rule accepted-fri-opening-transcript-challenges[OF openings])
have eqs: fri-dg = dg  $\wedge$  composition-bs' = comp-bs
  using accepted-fri-challenges-unique[OF pair-challenges accepted]
  by simp
have query-in: query-idxs  $\in$  Q dg comp-bs
proof -
  have query-idxs  $\in$ 
    fst ' (P dg  $\cap$  (UNIV  $\times$  {comp-bs}))
  using pair eqs by force
  then show ?thesis
    using projection[OF comp-bs-in] by blast
qed
from path out-eq obtain raw-idxs query-idxs-path where
  len-raw: length raw-idxs = rounds
  and query-path-in: query-idxs-path  $\in$  ?Q
  and query-path-eq:
    query-idxs-path =
      map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
  and lookup:
     $\bigwedge i. i < \text{rounds} \implies$ 
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)
  and fresh-lookup:
     $\bigwedge i. i < \text{rounds} \implies$ 
      fmllookup (HashMap ?s)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        None
  unfolding staged-security-with-data-state-query-index-list-path-fresh-def
  by auto
from accepted-fri-opening-transcript-imp-accepted-transcript-shape[OF openings]
obtain as-shape where shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  as-shape query-idxs
  by blast
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support[unfolded out-eq] shape]
obtain raw-idxs-pair where
  len-raw-pair: length raw-idxs-pair = rounds
  and query-idxs-eq:
    query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs-pair
  and lookup-pair:

```

```

 $\bigwedge i. i < \text{rounds} \implies$ 
   $\text{fmlookup } (\text{HashMap } \text{final-state})$ 
   $(\text{QueryIndexChallenge } i$ 
     $(\text{state-after-query-chunks}$ 
       $(\text{staged-query-start-hash } \text{data})$ 
       $(\text{staged-query-chunks } \text{data}) \ i)) =$ 
     $\text{Some } (\text{raw-idxs-pair } ! \ i)$ 
  by blast
have raw-eq:
   $\bigwedge i. i < \text{rounds} \implies \text{raw-idxs } ! \ i = \text{raw-idxs-pair } ! \ i$ 
  using lookup lookup-pair by simp
have raw-idxs-eq:  $\text{raw-idxs} = \text{raw-idxs-pair}$ 
  by  $(\text{rule } \text{nth-equalityI})$ 
   $(\text{use } \text{len-raw } \text{len-raw-pair } \text{raw-eq} \text{ in } \text{simp-all})$ 
have path-witness:
   $\text{length } \text{raw-idxs} = \text{rounds} \wedge$ 
   $\text{query-idxs} \in Q \text{ dg comp-bs} \wedge$ 
   $\text{query-idxs} = \text{map } (\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})) \ \text{raw-idxs} \wedge$ 
   $(\forall i < \text{rounds}.$ 
     $\text{fmlookup } (\text{HashMap } \text{final-state})$ 
     $(\text{QueryIndexChallenge } i$ 
       $(\text{state-after-query-chunks}$ 
         $(\text{staged-query-start-hash } \text{data})$ 
         $(\text{staged-query-chunks } \text{data}) \ i)) =$ 
       $\text{Some } (\text{raw-idxs } ! \ i) \wedge$ 
     $\text{fmlookup } (\text{HashMap } ?s)$ 
     $(\text{QueryIndexChallenge } i$ 
       $(\text{state-after-query-chunks}$ 
         $(\text{staged-query-start-hash } \text{data})$ 
         $(\text{staged-query-chunks } \text{data}) \ i)) =$ 
     $\text{None})$ 
  using len-raw raw-idxs-eq query-in query-idxs-eq lookup
  fresh-lookup by auto
have path-body:
   $\exists \text{raw-idxs } \text{query-idxs}.$ 
   $\text{length } \text{raw-idxs} = \text{rounds} \wedge$ 
   $\text{query-idxs} \in Q \text{ dg comp-bs} \wedge$ 
   $\text{query-idxs} = \text{map } (\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})) \ \text{raw-idxs} \wedge$ 
   $(\forall i < \text{rounds}.$ 
     $\text{fmlookup } (\text{HashMap } \text{final-state})$ 
     $(\text{QueryIndexChallenge } i$ 
       $(\text{state-after-query-chunks}$ 
         $(\text{staged-query-start-hash } \text{data})$ 
         $(\text{staged-query-chunks } \text{data}) \ i)) =$ 
     $\text{Some } (\text{raw-idxs } ! \ i) \wedge$ 
     $\text{fmlookup } (\text{HashMap } ?s)$ 
     $(\text{QueryIndexChallenge } i$ 
       $(\text{state-after-query-chunks}$ 
         $(\text{staged-query-start-hash } \text{data})$ 

```

```

      (staged-query-chunks data) i)) =
      None)
    using path-witness by blast
  have path-single:
    staged-security-with-data-state-query-index-list-path-fresh
      (Q dg comp-bs) out
    unfolding out-eq
      staged-security-with-data-state-query-index-list-path-fresh-def
    using path-body by simp
  have fresh-single:
    composition-fri-challenge-list-fresh-hit ?s
      ( $\lambda dg'. \text{if } dg' = dg \text{ then } \{comp\text{-}bs\} \text{ else } \{\}$ )
      (Some (result, final-state))
  proof -
    have witness:
      accepted-fri-challenges ?s (Some (result, final-state))
      trace-bs dg comp-bs  $\wedge$ 
      verifier-header-transcript ?s fr trace-roots trace-final as dg
      composition-roots final rest  $\wedge$ 
      comp-bs  $\in$  ( $\text{if } dg = dg \text{ then } \{comp\text{-}bs\} \text{ else } \{\}$ )  $\wedge$ 
      composition-fri-challenge-path-fresh ?s fr trace-roots
      trace-final as dg composition-roots (length composition-roots)
    using accepted header fresh-path by simp
  show ?thesis
    unfolding composition-fri-challenge-list-fresh-hit-def
    using witness by blast
qed
have singleton:
  ?E (dg, comp-bs) out
proof -
  have fresh-single-pair:
    composition-fri-challenge-list-fresh-hit ?s
      ( $\lambda dg'. \text{if } dg' = dg \text{ then } \{snd (dg, comp\text{-}bs)\} \text{ else } \{\}$ )
      (Some (result, final-state))
  proof -
    have
      ( $\lambda dg'. \text{if } dg' = dg \text{ then } \{comp\text{-}bs\} \text{ else } \{\}$ ) =
      ( $\lambda dg'. \text{if } dg' = dg \text{ then } \{snd (dg, comp\text{-}bs)\} \text{ else } \{\}$ )
      by (rule ext) (simp add: prod.sel)
    then show ?thesis
      using fresh-single by simp
  qed
show ?thesis
unfolding out-eq
  staged-security-composition-fri-pair-challenge-query-path-fresh-def
  staged-security-composition-fri-pair-challenge-fresh-def
  staged-security-with-data-state-verifier-event-def
  using pair-hit fresh-single-pair path-single out-eq by simp
qed

```

```

show ?thesis
  by (intro beI[of - (dg, comp-bs)])
    (use comp-bs-in singleton in simp-all)
qed
qed
also have ...  $\leq (\sum p \in ?I. ?C p)$ 
proof (rule wp-event-finite-union-bound-fri-exact)
  show finite ?I
    by (rule finite-SigmaI)
      (use challenge-subset finite-fri-challenge-space in
        ‹auto intro: finite-subset[OF challenge-subset]›)
next
  fix p
  assume p-in:  $p \in ?I$ 
  obtain dg challenges where p-eq:  $p = (dg, challenges)$ 
    by (cases p) auto
  have challenges-in:  $challenges \in B\ dg$ 
    using p-in p-eq by simp
  have challenges-space:
     $challenges \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$ 
    using challenge-subset[of dg] challenges-in by blast
  have fixed-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-composition-fri-pair-challenge-query-path-fresh P
        ( $\lambda dg'. \text{if } dg' = dg \text{ then } \{challenges\} \text{ else } \{\}$ )
        (Q dg challenges))
      adversary-initial-state  $\leq$ 
      ( $(1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1))) *$ 
        ( $\text{nnreal } (\text{card } (Q\ dg\ challenges)) *$ 
          ( $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds}$ ))
      by (rule
        checked-staged-security-composition-fri-pair-challenge-query-path-fresh-fixed-bound
        [where  $dg = dg$  and  $challenges = challenges$ 
          and  $Q = Q\ dg\ challenges$  and  $P = P$ ,
          OF wf controlled challenges-space
          projection[OF challenges-in] subset[OF challenges-in]])
  have event-eq:
    ?E p =
      staged-security-composition-fri-pair-challenge-query-path-fresh P
        ( $\lambda dg'. \text{if } dg' = dg \text{ then } \{challenges\} \text{ else } \{\}$ )
        (Q dg challenges)
  unfolding p-eq
    staged-security-composition-fri-pair-challenge-query-path-fresh-def
    staged-security-composition-fri-pair-challenge-fresh-def
    composition-fri-challenge-list-fresh-hit-def
  by (rule ext) (simp add: prod.sel)
have cost-eq:
  ?C p =
    ( $(1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1))) *$ 

```

```

      (nnreal (card (Q dg challenges)) *
       (1 / nnreal (card query-sample-space)) ^ rounds)
    unfolding p-eq by simp
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (?E p) adversary-initial-state ≤ ?C p
    using fixed-bound unfolding event-eq cost-eq .
qed
finally have bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-composition-fri-pair-challenge-query-path-fresh P
     B ?Q)
  adversary-initial-state ≤ (∑ p ∈ ?I. ?C p) .
have finite-B: ∧ dg. finite (B dg)
  by (meson challenge-subset finite-fri-challenge-space finite-subset)
have sigma-eq:
  (∑ p ∈ ?I. ?C p) =
  (∑ dg ∈ UNIV.
   ∑ challenges ∈ B dg.
   (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
   (nnreal (card (Q dg challenges)) *
    (1 / nnreal (card query-sample-space)) ^ rounds))
  by (subst sum.Sigma) (simp-all add: finite-B case-prod-beta)
show ?thesis
  using bound sigma-eq by simp
qed

end

end
end

```

```

theory Soundness-FRI-Query-Challenge-Trace-Staged-Product
imports Soundness-FRI-Query-Challenge-Actual-Product
begin

```

Trace-side staged product accounting for the challenge-fresh/query-path-fresh FRI pair branch.

This theory is intentionally narrow. The preceding layer contains the verifier-local actual-fresh product bounds and the raw-list path bridges. Here we package the trace bridge into staged fixed-challenge and finite-cover product bounds.

```

context soundness
begin

```

```

lemma verifier-after-trace-fri-query-index-list-path-fresh-imp-actual-raw-path:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and builder:
  Some (data, attacker-state) ∈
  set-dist

```

```

      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
    and prefix:
      Some ((fr, f-fl), prefix-state) ∈
        set-dist
        (execute verifier-trace-fri-prefix
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))))
    and read-trace-final:
      Some (f-final, s3) ∈ set-dist (execute read prefix-state)
    and alpha-out:
      Some (as, s4) ∈
        set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
    and read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
    and degree-assert:
      Some ((), s6) ∈
        set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
    and comp-fri:
      Some (fl, s7) ∈
        set-dist (execute
          (ntimes receive-composition-fri-commits
            (ceil-log (to-nat dg + 1))) s6)
    and read-final:
      Some (final, query-state) ∈ set-dist (execute read s7)
    and query-out:
      Some (result, final-state) ∈
        set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          query-state)
    and hit:
      staged-security-with-data-state-query-index-list-path-fresh Q
      (Some (((data, attacker-state), result), final-state))
  shows ∃ raws ∈ query-index-raw-list-preimage Q.
    query-rounds-raw-list-path-hit query-state raws rounds
    (Some (result, final-state))
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from ntimes-verifier-query-rounds-outcome[OF query-out]
  obtain raw-idxs query-idxs query-chunks where
    len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-chunks: length query-chunks = rounds
  and tr-query:

```



```

PTranscript query-state =
  List.concat query-chunks @ PTranscript final-state
and state-final:
  PState final-state =
    state-after-query-chunks (PState query-state) query-chunks rounds
and round-chunks:
   $\bigwedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
and lookups:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmllookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
    (state-after-query-chunks
    (PState query-state) query-chunks i)) =
    Some (raw-idxs ! i)
by blast
have trace-prefix-res:
  length f-fl = ceil-log clength  $\wedge$ 
  PTranscript ?s = [fr] @ map snd f-fl @ PTranscript prefix-state  $\wedge$ 
  PState prefix-state =
    foldl concat (concat (PState ?s) fr) (map snd f-fl)  $\wedge$ 
  PQueryCounter prefix-state = PQueryCounter ?s
  using verifier-trace-fri-prefix-outcome[OF prefix] by simp
from read-outcome[OF read-trace-final] obtain rest3 where
  tr-prefix: PTranscript prefix-state = f-final # rest3
  and tr-s3: PTranscript s3 = rest3
  and state-s3: PState s3 = concat (PState prefix-state) f-final
  and counter-s3: PQueryCounter s3 = PQueryCounter prefix-state
  by blast
have alpha-res:
  length as = length spec  $\wedge$ 
  PTranscript s3 = as @ PTranscript s4  $\wedge$ 
  PState s4 = foldl concat (PState s3) as  $\wedge$ 
  PQueryCounter s4 = PQueryCounter s3
  using mmap-alpha-round-outcome[OF alpha-out] by simp
from read-outcome[OF read-dg] obtain rest5 where
  tr-s4: PTranscript s4 = dg # rest5
  and tr-s5: PTranscript s5 = rest5
  and state-s5: PState s5 = concat (PState s4) dg
  and counter-s5: PQueryCounter s5 = PQueryCounter s4
  by blast
have s6-eq: s6 = s5
  using degree-assert unfolding assert-def
  by (cases to-nat dg  $\leq$  maxDegree) (auto simp: throw-no-outcome)
have comp-res:
  length fl = ceil-log (to-nat dg + 1)  $\wedge$ 
  PTranscript s6 = map snd fl @ PTranscript s7  $\wedge$ 
  PState s7 = foldl concat (PState s6) (map snd fl)  $\wedge$ 

```

```

    PQueryCounter s7 = PQueryCounter s6
    using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
    by simp
  from read-outcome[OF read-final] obtain rest-query where
    tr-s7: PTranscript s7 = final # rest-query
    and tr-read-query: PTranscript query-state = rest-query
    and state-query: PState query-state = concat (PState s7) final
    and counter-query-s7: PQueryCounter query-state = PQueryCounter s7
  by blast
  have local-counter: PQueryCounter query-state = 0
    using trace-prefix-res counter-s3 alpha-res counter-s5 s6-eq comp-res
      counter-query-s7
    unfolding verifier-state-from-adversary-def by simp
  have prefix-lookup:
     $\bigwedge i x. \text{fmlookup} (\text{HashMap prefix-state})$ 
      (QueryIndexChallenge i x) =
      fmlookup (HashMap ?s) (QueryIndexChallenge i x)
    by (rule verifier-trace-fri-prefix-preserves-query-lookup[OF prefix])
  have s3-lookup:
    HashMap s3 = HashMap prefix-state
    by (rule read-preserves-hash-map[OF read-trace-final])
  have s4-lookup:
     $\bigwedge i x. \text{fmlookup} (\text{HashMap s4}) (\text{QueryIndexChallenge } i \ x) =$ 
      fmlookup (HashMap s3) (QueryIndexChallenge i x)
  proof -
    fix i x
    show fmlookup (HashMap s4) (QueryIndexChallenge i x) =
      fmlookup (HashMap s3) (QueryIndexChallenge i x)
    proof (rule mmap-preserves-lookup[OF - alpha-out])
      fix m y u v
      assume m-in:  $m \in \text{set } (\text{replicate } (\text{length spec}) \text{ alpha-round})$ 
      and out:  $\text{Some } (y, v) \in \text{set-dist } (\text{execute } m \ u)$ 
      then have m-eq:  $m = \text{alpha-round}$ 
      by simp
      show fmlookup (HashMap v) (QueryIndexChallenge i x) =
        fmlookup (HashMap u) (QueryIndexChallenge i x)
      using alpha-round-preserves-query-lookup[OF out[unfolded m-eq]] .
    qed
  qed
  have s5-lookup:
    HashMap s5 = HashMap s4
    by (rule read-preserves-hash-map[OF read-dg])
  have s7-lookup:
     $\bigwedge i x. \text{fmlookup} (\text{HashMap s7}) (\text{QueryIndexChallenge } i \ x) =$ 
      fmlookup (HashMap s6) (QueryIndexChallenge i x)
  proof -
    fix i x
    show fmlookup (HashMap s7) (QueryIndexChallenge i x) =
      fmlookup (HashMap s6) (QueryIndexChallenge i x)

```

```

proof (rule ntimes-preserves-lookup[OF - comp-fri])
  fix y u v
  assume out:
    Some (y, v) ∈
      set-dist (execute receive-composition-fri-commits u)
  obtain b r where y-eq: y = (b, r)
    by (cases y simp)
  show fmlookup (HashMap v) (QueryIndexChallenge i x) =
    fmlookup (HashMap u) (QueryIndexChallenge i x)
    by (rule receive-composition-fri-commits-preserves-query-lookup
      [OF out[unfolded y-eq]])
  qed
qed
have query-lookup:
  HashMap query-state = HashMap s7
  by (rule read-preserves-hash-map[OF read-final])
have header-actual:
  verifier-header-transcript ?s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
  using trace-prefix-res tr-prefix tr-s3 alpha-res tr-s4 tr-s5 s6-eq
    comp-res tr-s7 tr-read-query
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
have header-unique:
  fr = staged-trace-root data ∧
  map snd f-fl = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  map snd fl = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  PTranscript query-state = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF header-actual staged-header]
  by simp
have prefix-state-query:
  PState query-state = staged-query-start-hash data
  using trace-prefix-res state-s3 alpha-res state-s5 s6-eq comp-res

```

```

    state-query header-unique
unfolding staged-query-start-hash-def
    staged-composition-fri-start-hash-def
    staged-trace-fri-start-hash-def
by simp
have chunks-eq: query-chunks = staged-query-chunks data
proof -
from checked-staged-transcript-program-query-chunks-match-verifier-lengths
    [OF builder]
obtain staged-query-idxs where
    staged-match:
        staged-query-chunks-match-verifier-lengths data staged-query-idxs
by blast
have query-idxs-len: length query-idxs = rounds
using query-idxs-eq len-raw by simp
have staged-match-query-idxs:
    staged-query-chunks-match-verifier-lengths data query-idxs
by (rule staged-query-chunks-match-verifier-lengths-transfer
    [OF staged-match query-idxs-len])
have parser-chunk-len:
     $\bigwedge i. i < \text{rounds} \implies$ 
        length (query-chunks ! i) =
            verifier-query-round-transcript-length (query-idxs ! i)
                (staged-trace-fri-roots data)
                (staged-composition-fri-roots data)
proof -
fix i
assume i-bound: i < rounds
show
    length (query-chunks ! i) =
        verifier-query-round-transcript-length (query-idxs ! i)
            (staged-trace-fri-roots data)
            (staged-composition-fri-roots data)
using verifier-query-round-chunk-length[OF round-chunks[OF i-bound]]
    header-unique
by simp
qed
have concat-staged:
    List.concat query-chunks @ PTranscript final-state =
        List.concat (staged-query-chunks data)
using tr-query header-unique by simp
then show ?thesis
using query-chunks-eq-staged-if-matching-lengths
    [OF len-chunks concat-staged parser-chunk-len
        staged-match-query-idxs]
by simp
qed
from hit obtain raw-idxs-hit query-idxs-hit where
    query-hit-in: query-idxs-hit  $\in Q$ 

```

```

and len-hit: length raw-idxs-hit = rounds
and query-hit-eq:
  query-idxs-hit =
    map ( $\lambda$ raw. index (to-nat raw)) raw-idxs-hit
and lookup-hit:
   $\wedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        Some (raw-idxs-hit ! i)
and fresh-hit:
   $\wedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap ?s)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        None
unfolding staged-security-with-data-state-query-index-list-path-fresh-def
by auto
have raw-eq:
   $\wedge i. i < \text{rounds} \implies \text{raw-idxs} ! i = \text{raw-idxs-hit} ! i$ 
using lookups lookup-hit local-counter prefix-state-query chunks-eq
by simp
have raw-idxs-eq: raw-idxs = raw-idxs-hit
by (rule nth-equalityI)
  (use len-raw len-hit raw-eq in simp-all)
have raw-in: raw-idxs  $\in$  query-index-raw-list-preimage Q
unfolding query-index-raw-list-preimage-def
using len-raw raw-idxs-eq query-hit-in query-hit-eq by blast
have query-lookup-start:
   $\wedge i x. \text{fmlookup} (\text{HashMap } \text{query-state}) (\text{QueryIndexChallenge } i x) =$ 
    fmlookup (HashMap ?s) (QueryIndexChallenge i x)
using prefix-lookup s3-lookup s4-lookup s5-lookup s6-eq
  s7-lookup query-lookup by simp
have local-fresh:
  query-index-list-path-fresh query-state rounds query-chunks raw-idxs
unfolding query-index-list-path-fresh-def
proof (intro conjI allI impI)
  show length raw-idxs = rounds
    using len-raw .
next
  show length query-chunks = rounds
    using len-chunks .
next
  fix i
  assume i-bound: i < rounds

```

```

let ?x =
  state-after-query-chunks (staged-query-start-hash data)
    (staged-query-chunks data) i
have fmlookup (HashMap query-state)
  (QueryIndexChallenge (PQueryCounter query-state + i)
    (state-after-query-chunks (PState query-state) query-chunks i)) =
  fmlookup (HashMap ?s) (QueryIndexChallenge i ?x)
using local-counter prefix-state-query chunks-eq
  query-lookup-start[of i ?x] by simp
also have ... = None
by (rule fresh-hit[OF i-bound])
finally show
  fmlookup (HashMap query-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    None .
qed
have path:
  query-rounds-raw-list-path-hit query-state raw-idxs rounds
    (Some (result, final-state))
unfolding query-rounds-raw-list-path-hit-def
using len-raw len-chunks state-final local-fresh lookups
by auto
show ?thesis
using raw-in path by blast
qed

lemma verifier-after-trace-fri-query-index-list-path-fresh-product-bound-after-prefix:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and subset:  $Q \subseteq \text{fri-query-index-list-space}$ 
and builder:
  Some (data, attacker-state)  $\in$ 
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and prefix:
  Some (header, prefix-state)  $\in$ 
    set-dist
      (execute verifier-trace-fri-prefix
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
shows
  wp-event (verifier-after-trace-fri header)
    ( $\lambda$ out.
      staged-security-with-data-state-query-index-list-path-fresh Q
        (case out of
          None  $\Rightarrow$  None
          | Some (result, final-state)  $\Rightarrow$ 

```

```

      Some (((data, attacker-state), result), final-state)))
    prefix-state
  ≤ nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds
proof -
obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
let ?P =
  λout.
    staged-security-with-data-state-query-index-list-path-fresh Q
    (case out of
      None ⇒ None
      | Some (result, final-state) ⇒
        Some (((data, attacker-state), result), final-state))
let ?C =
  nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds
have prefix':
  Some ((fr, f-fl), prefix-state) ∈
    set-dist
    (execute verifier-trace-fri-prefix
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
  using prefix header-eq by simp
have after-eq:
  verifier-after-trace-fri header =
    (read >=
      (λf-final.
        mmap (replicate (length spec) alpha-round) >=
          (λas. read >=
            (λdg. assert (to-nat dg ≤ maxDegree) >=
              (λ-. ntimes receive-composition-fri-commits
                (ceil-log (to-nat dg + 1)) >=
                  (λfl. read >=
                    (λfinal.
                      ntimes
                        (verifier-query-round-program fr f-fl f-final as fl
                          final)
                      rounds))))))))
  unfolding header-eq verifier-after-trace-fri-def by simp
show ?thesis
  unfolding after-eq
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?P None
    unfolding staged-security-with-data-state-query-index-list-path-fresh-def
    by simp
next
  fix f-final s3
  assume read-trace-final:

```

Some (f-final, s3) ∈ set-dist (execute read prefix-state)
show
wp-event
 (mmap (replicate (length spec) alpha-round) $\gg=$
 (λas. read $\gg=$
 (λdg. assert (to-nat dg ≤ maxDegree) $\gg=$
 (λ-. ntimes receive-composition-fri-commits
 (ceil-log (to-nat dg + 1)) $\gg=$
 (λfl. read $\gg=$
 (λfinal.
 ntimes
 (verifier-query-round-program fr f-fl f-final as fl
 final)
 rounds)))))) ?P s3 ≤ ?C
proof (rule wp-event-bind-bound-by-cont)
show $\neg$?P None
unfolding staged-security-with-data-state-query-index-list-path-fresh-def
by simp
next
fix as s4
assume alpha-out:
Some (as, s4) ∈
set-dist
(execute (mmap (replicate (length spec) alpha-round)) s3)
show
wp-event
 (read $\gg=$
 (λdg. assert (to-nat dg ≤ maxDegree) $\gg=$
 (λ-. ntimes receive-composition-fri-commits
 (ceil-log (to-nat dg + 1)) $\gg=$
 (λfl. read $\gg=$
 (λfinal.
 ntimes
 (verifier-query-round-program fr f-fl f-final as fl
 final)
 rounds)))))) ?P s4 ≤ ?C
proof (rule wp-event-bind-bound-by-cont)
show $\neg$?P None
unfolding staged-security-with-data-state-query-index-list-path-fresh-def
by simp
next
fix dg s5
assume read-dg:
Some (dg, s5) ∈ set-dist (execute read s4)
show
wp-event
 (assert (to-nat dg ≤ maxDegree) $\gg=$
 (λ-. ntimes receive-composition-fri-commits
 (ceil-log (to-nat dg + 1)) $\gg=$


```

      (λfl. read >>=
        (λfinal.
          ntimes
            (verifier-query-round-program fr f-fl f-final as fl
              final)
            rounds)))) ?P s5 ≤ ?C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?P None
    unfolding staged-security-with-data-state-query-index-list-path-fresh-def
    by simp
next
  fix unit s6
  assume degree-assert:
    Some (unit, s6) ∈
      set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
  show
    wp-event
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1)) >>=
          (λfl. read >>=
            (λfinal.
              ntimes
                (verifier-query-round-program fr f-fl f-final as fl
                  final)
                rounds)))) ?P s6 ≤ ?C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?P None
    unfolding staged-security-with-data-state-query-index-list-path-fresh-def
    by simp
next
  fix fl s7
  assume comp-fri:
    Some (fl, s7) ∈
      set-dist
        (execute
          (ntimes receive-composition-fri-commits
            (ceil-log (to-nat dg + 1)))) s6)
  show
    wp-event
      (read >>=
        (λfinal.
          ntimes
            (verifier-query-round-program fr f-fl f-final as fl
              final)
            rounds)) ?P s7 ≤ ?C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?P None
    unfolding staged-security-with-data-state-query-index-list-path-fresh-def
    by simp

```

```

next
  fix final query-state
  assume read-final:
    Some (final, query-state) ∈ set-dist (execute read s7)
  have degree-assert-unit:
    Some ((), s6) ∈
      set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
  using degree-assert by (cases unit) simp
  have mono:
    wp-event
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
    ?P query-state ≤
    wp-event
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
    (λout. ∃ rows ∈ query-index-raw-list-preimage Q.
      query-rounds-raw-list-path-hit query-state rows rounds out)
    query-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume query-out:
    out ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl
              final)
            rounds)
          query-state)
  and hit: ?P out
  show ∃ rows ∈ query-index-raw-list-preimage Q.
    query-rounds-raw-list-path-hit query-state rows rounds out
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding staged-security-with-data-state-query-index-list-path-fresh-def
    by simp
  next
  case (Some pair)
  then obtain result final-state where out-eq:
    out = Some (result, final-state)
  by (cases pair) simp
  have query-out':
    Some (result, final-state) ∈
      set-dist

```

```

      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl
            final)
          rounds)
        query-state)
    using query-out out-eq by simp
  have hit':
    staged-security-with-data-state-query-index-list-path-fresh Q
      (Some (((data, attacker-state), result), final-state))
    using hit out-eq by simp
  show ?thesis
    using
      verifier-after-trace-fri-query-index-list-path-fresh-imp-actual-raw-path
      [OF wf controlled builder prefix' read-trace-final
        alpha-out read-dg degree-assert-unit comp-fri read-final
        query-out' hit]
    unfolding out-eq .
qed
qed
also have ... ≤ ?C
  by (rule
    wp-ntimes-verifier-query-round-program-query-list-path-product-bound
    [OF subset])
finally show
  wp-event
    (ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds)
  ?P query-state ≤ ?C .
qed
qed
qed
qed
qed
qed
qed

```

lemma *checked-staged-security-trace-fri-pair-challenge-query-path-fresh-fixed-bound:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *challenges-space:*
 challenges ∈ fri-challenge-space (ceil-log clength)
and *projection:*
 fst ‘ (P ∩ (UNIV × {challenges})) ⊆ Q
and *subset: Q ⊆ fri-query-index-list-space*
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-trace-fri-pair-challenge-query-path-fresh P

```

      {challenges} Q)
    adversary-initial-state ≤
      (1 / nnreal (CARD('f) ^ ceil-log clength)) *
      (nnreal (card Q) *
        (1 / nnreal (card query-sample-space)) ^ rounds)
  proof (rule checked-staged-security-with-data-state-bound-from-data-cont)
  let ?D =
    nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds
  show ¬ staged-security-trace-fri-pair-challenge-query-path-fresh
    P {challenges} Q None
  unfolding staged-security-trace-fri-pair-challenge-query-path-fresh-def
    staged-security-trace-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
  by simp
next
fix data attacker-state
assume builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
let ?Event =
  λout.
    staged-security-trace-fri-pair-challenge-query-path-fresh P
    {challenges} Q
    (case out of
      None ⇒ None
      | Some (result, final-state) ⇒
        Some (((data, attacker-state), result), final-state))
let ?Head =
  λout. case out of
    None ⇒ False
    | Some ((fr, f-fl), -) ⇒
      map fst f-fl = challenges ∧
      trace-fri-challenge-path-fresh ?s fr (map snd f-fl)
      (length (map snd f-fl))
show wp-event verify-monad ?Event ?s ≤
  (1 / nnreal (CARD('f) ^ ceil-log clength)) *
  (nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
unfolding verify-monad-trace-fri-decomposition
proof (rule order-trans)
show wp-event
  (verifier-trace-fri-prefix ≫= verifier-after-trace-fri)
  ?Event ?s

```

```

≤ wp-event verifier-trace-fri-prefix ?Head ?s *
  (nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
proof (rule wp-event-bind-bound-by-head-and-cont)
show ¬ ?Event None
  unfolding staged-security-trace-fri-pair-challenge-query-path-fresh-def
    staged-security-trace-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
  by simp
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix ?s)
  and not-head: ¬ ?Head (Some (header, prefix-state))
obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
have not-head':
  map fst f-fl ≠ challenges ∨
    ¬ trace-fri-challenge-path-fresh ?s fr (map snd f-fl)
    (length (map snd f-fl))
  using not-head unfolding header-eq by simp
show wp-event (verifier-after-trace-fri header) ?Event prefix-state =
  0
proof (rule antisym)
show wp-event (verifier-after-trace-fri header) ?Event
  prefix-state ≤ 0
proof (rule order-trans)
show wp-event (verifier-after-trace-fri header) ?Event
  prefix-state ≤
    wp-event (verifier-after-trace-fri header)
    (λ-. False) prefix-state
proof (rule wp-event-mono-on-support)
fix out
assume suffix:
  out ∈
    set-dist
    (execute (verifier-after-trace-fri header) prefix-state)
  and hit: ?Event out
show False
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    staged-security-trace-fri-pair-challenge-query-path-fresh-def
    staged-security-trace-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
  by simp

```

```

next
case (Some pair)
then obtain result final-state where out-eq:
  out = Some (result, final-state)
  by (cases pair) simp
from hit out-eq have fresh:
  trace-fri-challenge-list-fresh-hit ?s {challenges}
  (Some (result, final-state))
and pair-hit:
  trace-fri-query-challenge-pair-set-hit ?s P
  (Some (result, final-state))
unfolding
  staged-security-trace-fri-pair-challenge-query-path-fresh-def
  staged-security-trace-fri-pair-challenge-fresh-def
  staged-security-with-data-state-verifier-event-def
by simp-all
have prefix':
  Some ((fr, f-fl), prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix ?s)
using prefix header-eq by simp
have suffix':
  Some (result, final-state) ∈
    set-dist
      (execute (verifier-after-trace-fri (fr, f-fl))
        prefix-state)
using suffix header-eq out-eq by simp
from pair-hit obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
openings:
  accepted-fri-opening-transcript ?s
  (Some (result, final-state)) trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs
  composition-final query-idxs trace-round-layers
  composition-round-layers
unfolding trace-fri-query-challenge-pair-set-hit-def by blast
have pair-challenges:
  accepted-fri-challenges ?s (Some (result, final-state))
  trace-bs fri-dg composition-bs
by (rule accepted-fri-opening-transcript-challenges
  [OF openings])
from fresh obtain trace-bs' dg' comp-bs' where
  fresh-challenges:
    accepted-fri-challenges ?s (Some (result, final-state))
    trace-bs' dg' comp-bs'
and trace-bs'-in: trace-bs' ∈ {challenges}
unfolding trace-fri-challenge-list-fresh-hit-def by blast
have trace-bs-eq: trace-bs = trace-bs'
using accepted-fri-challenges-unique

```

```

    [OF pair-challenges fresh-challenges]
  by simp
  have prefix-trace-eq: trace-bs = map fst f-fl
  by (rule
    verifier-after-trace-fri-header-agrees-with-accepted-challenges
    [OF prefix' suffix' pair-challenges])
  have concrete-fresh:
    trace-fri-challenge-path-fresh ?s fr (map snd f-fl)
    (length (map snd f-fl))
  by (rule verifier-after-trace-fri-header-roots-agree-with-fresh-hit
    [OF prefix' suffix' fresh])
  have head-holds:
    map fst f-fl = challenges ∧
    trace-fri-challenge-path-fresh ?s fr (map snd f-fl)
    (length (map snd f-fl))
  using prefix-trace-eq trace-bs-eq trace-bs'-in
  concrete-fresh by simp
  show ?thesis
  using not-head' head-holds by blast
qed
qed
next
  show wp-event (verifier-after-trace-fri header)
    (λ-. False) prefix-state ≤ 0
  unfolding wp-event-def wp-def dist-expect-def by simp
qed
next
  show 0 ≤ wp-event (verifier-after-trace-fri header) ?Event
    prefix-state
  by simp
qed
next
  fix header prefix-state
  assume prefix:
    Some (header, prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix ?s)
  and head: ?Head (Some (header, prefix-state))
  obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
  have event-mono:
    wp-event (verifier-after-trace-fri header) ?Event prefix-state ≤
    wp-event (verifier-after-trace-fri header)
    (λout.
      staged-security-with-data-state-query-index-list-path-fresh Q
      (case out of
        None ⇒ None
      | Some (result, final-state) ⇒
        Some (((data, attacker-state), result), final-state)))
    prefix-state

```

```

    by (rule wp-event-mono-on-support)
      (auto simp: staged-security-trace-fri-pair-challenge-query-path-fresh-def)
  also have ... ≤
    nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds
  by (rule
      verifier-after-trace-fri-query-index-list-path-fresh-product-bound-after-prefix
      [OF wf controlled subset builder prefix])
  finally show wp-event (verifier-after-trace-fri header) ?Event
    prefix-state ≤
    nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds .
qed
next
show wp-event verifier-trace-fri-prefix ?Head ?s *
  (nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
  ≤
  (1 / nnreal (CARD('f) ^ ceil-log clength)) *
  (nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
proof (rule mult-right-mono)
  show wp-event verifier-trace-fri-prefix ?Head ?s ≤
    1 / nnreal (CARD('f) ^ ceil-log clength)
  proof -
    let ?HeadSet =
      λout. case out of
        None ⇒ False
      | Some ((fr, f-fl), -) ⇒
        map fst f-fl ∈ {challenges} ∧
        trace-fri-challenge-path-fresh ?s fr (map snd f-fl)
        (length (map snd f-fl))
    have mono:
      wp-event verifier-trace-fri-prefix ?Head ?s ≤
      wp-event verifier-trace-fri-prefix ?HeadSet ?s
    by (rule wp-event-mono) (auto split: option.splits prod.splits)
    have set-bound:
      wp-event verifier-trace-fri-prefix ?HeadSet ?s ≤
      nnreal (card {challenges}) /
      nnreal (CARD('f) ^ ceil-log clength)
    by (rule
        wp-verifier-trace-fri-prefix-challenge-list-actual-fresh-bound
        [where B = {challenges}])
      (use challenges-space in auto)
    show ?thesis
      using order-trans[OF mono set-bound] by simp
  qed
show 0 ≤ nnreal (card Q) *
  (1 / nnreal (card query-sample-space)) ^ rounds

```



```

    by simp
  qed
qed
qed
lemma checked-staged-security-trace-fri-pair-challenge-query-path-fresh-fiber-sum:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and challenge-subset:  $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$ 
  and projection:
     $\bigwedge \text{challenges. challenges} \in B \implies$ 
     $\text{fst } '(P \cap (UNIV \times \{\text{challenges}\})) \subseteq Q \text{ challenges}$ 
  and subset:
     $\bigwedge \text{challenges. challenges} \in B \implies$ 
     $Q \text{ challenges} \subseteq \text{fri-query-index-list-space}$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-trace-fri-pair-challenge-query-path-fresh P B
    ( $\bigcup \text{challenges} \in B. Q \text{ challenges}$ ))
    adversary-initial-state  $\leq$ 
    ( $\sum \text{challenges} \in B.$ 
    ( $1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } \text{clength})) *$ 
    ( $\text{nnreal } (\text{card } (Q \text{ challenges})) *$ 
    ( $1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds}$ ))
  proof -
    let ?Q = ( $\bigcup \text{challenges} \in B. Q \text{ challenges}$ )
    let ?E =
       $\lambda \text{challenges.}$ 
      staged-security-trace-fri-pair-challenge-query-path-fresh P
      {challenges} (Q challenges)
    let ?C =
       $\lambda \text{challenges.}$ 
      ( $1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } \text{clength})) *$ 
      ( $\text{nnreal } (\text{card } (Q \text{ challenges})) *$ 
      ( $1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds}$ )
    have split:
      wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-trace-fri-pair-challenge-query-path-fresh P B ?Q)
      adversary-initial-state  $\leq$ 
      wp-event (checked-staged-security-experiment-with-data-state A)
      ( $\lambda \text{out. } \exists \text{challenges} \in B. ?E \text{ challenges out}$ )
      adversary-initial-state
    proof (rule wp-event-mono-on-support)
      fix out
      assume support:
        out  $\in$ 
        set-dist
        (execute (checked-staged-security-experiment-with-data-state A)

```

```

    adversary-initial-state)
  and hit:
    staged-security-trace-fri-pair-challenge-query-path-fresh P B ?Q out
  show  $\exists$  challenges  $\in B$ . ?E challenges out
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding staged-security-trace-fri-pair-challenge-query-path-fresh-def
      staged-security-trace-fri-pair-challenge-fresh-def
      staged-security-with-data-state-verifier-event-def
    by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed) (auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from hit out-eq have pair-hit:
  trace-fri-query-challenge-pair-set-hit ?s P
  (Some (result, final-state))
and fresh:
  trace-fri-challenge-list-fresh-hit ?s B
  (Some (result, final-state))
and path:
  staged-security-with-data-state-query-index-list-path-fresh ?Q out
unfolding staged-security-trace-fri-pair-challenge-query-path-fresh-def
  staged-security-trace-fri-pair-challenge-fresh-def
  staged-security-with-data-state-verifier-event-def
by simp-all
from fresh obtain trace-bs dg comp-bs fr trace-roots trace-final as
  composition-roots final rest where
  accepted:
    accepted-fri-challenges ?s (Some (result, final-state))
    trace-bs dg comp-bs
and trace-bs-in: trace-bs  $\in B$ 
and header:
  verifier-header-transcript ?s fr trace-roots trace-final as dg
  composition-roots final rest
and fresh-path:
  trace-fri-challenge-path-fresh ?s fr trace-roots
  (length trace-roots)
unfolding trace-fri-challenge-list-fresh-hit-def by blast
from pair-hit obtain trace-roots' trace-bs' trace-final' fri-dg
  composition-roots' composition-bs' composition-final' query-idxs
  trace-round-layers composition-round-layers where
  openings:

```

```

    accepted-fri-opening-transcript ?s
    (Some (result, final-state)) trace-roots' trace-bs'
    trace-final' fri-dg composition-roots' composition-bs'
    composition-final' query-idxs trace-round-layers
    composition-round-layers
  and pair: (query-idxs, trace-bs') ∈ P
  unfolding trace-fri-query-challenge-pair-set-hit-def by blast
have pair-challenges:
  accepted-fri-challenges ?s (Some (result, final-state))
  trace-bs' fri-dg composition-bs'
  by (rule accepted-fri-opening-transcript-challenges[OF openings])
have trace-eq: trace-bs' = trace-bs
  using accepted-fri-challenges-unique[OF pair-challenges accepted]
  by simp
have query-in: query-idxs ∈ Q trace-bs
proof -
  have query-idxs ∈ fst ' (P ∩ (UNIV × {trace-bs}))
    using pair trace-eq by force
  then show ?thesis
    using projection[OF trace-bs-in] by blast
qed
from path out-eq obtain raw-idxs query-idxs-path where
  len-raw: length raw-idxs = rounds
  and query-path-in: query-idxs-path ∈ ?Q
  and query-path-eq:
    query-idxs-path =
      map (λraw. index (to-nat raw)) raw-idxs
  and lookup:
    ∧i. i < rounds ⇒
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)
  and fresh-lookup:
    ∧i. i < rounds ⇒
      fmllookup (HashMap ?s)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        None
  unfolding staged-security-with-data-state-query-index-list-path-fresh-def
  by auto
from accepted-fri-opening-transcript-imp-accepted-transcript-shape[OF openings]
obtain as-shape where shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  as-shape query-idxs

```

```

by blast
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support[unfolded out-eq] shape]
obtain raw-idxs-pair where
  len-raw-pair: length raw-idxs-pair = rounds
and query-idxs-eq:
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs-pair
and lookup-pair:
   $\bigwedge i. i < rounds \implies$ 
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs-pair ! i)
by blast
have raw-eq:
   $\bigwedge i. i < rounds \implies$  raw-idxs ! i = raw-idxs-pair ! i
  using lookup lookup-pair by simp
have raw-idxs-eq: raw-idxs = raw-idxs-pair
  by (rule nth-equalityI)
  (use len-raw len-raw-pair raw-eq in simp-all)
have path-witness:
  length raw-idxs = rounds  $\wedge$ 
  query-idxs  $\in Q$  trace-bs  $\wedge$ 
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs  $\wedge$ 
  ( $\forall i < rounds.$ 
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)  $\wedge$ 
    fmllookup (HashMap ?s)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      None)
  using len-raw raw-idxs-eq query-in query-idxs-eq lookup
  fresh-lookup by auto
have path-body:
   $\exists$  raw-idxs query-idxs.
  length raw-idxs = rounds  $\wedge$ 
  query-idxs  $\in Q$  trace-bs  $\wedge$ 
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs  $\wedge$ 
  ( $\forall i < rounds.$ 
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i

```

```

      (state-after-query-chunks
       (staged-query-start-hash data)
       (staged-query-chunks data) i)) =
    Some (raw-idxs ! i) ∧
    fmllookup (HashMap ?s)
      (QueryIndexChallenge i
       (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    None)
  using path-witness by blast
have path-single:
  staged-security-with-data-state-query-index-list-path-fresh
    (Q trace-bs) out
  unfolding out-eq
    staged-security-with-data-state-query-index-list-path-fresh-def
  using path-body by simp
have fresh-single:
  trace-fri-challenge-list-fresh-hit ?s {trace-bs}
    (Some (result, final-state))
proof -
  have witness:
    accepted-fri-challenges ?s (Some (result, final-state))
      trace-bs dg comp-bs ∧
    verifier-header-transcript ?s fr trace-roots trace-final as dg
      composition-roots final rest ∧
    trace-bs ∈ {trace-bs} ∧
    trace-fri-challenge-path-fresh ?s fr trace-roots
      (length trace-roots)
  using accepted header fresh-path by simp
show ?thesis
  unfolding trace-fri-challenge-list-fresh-hit-def
  using witness by blast
qed
have singleton:
  ?E trace-bs out
  unfolding out-eq
    staged-security-trace-fri-pair-challenge-query-path-fresh-def
    staged-security-trace-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
  using pair-hit fresh-single path-single out-eq by simp
show ?thesis
  using singleton trace-bs-in by blast
qed
qed
also have ... ≤ (∑ challenges ∈ B. ?C challenges)
proof (rule wp-event-finite-union-bound-fri-exact)
  show finite B
  using challenge-subset by (rule finite-subset)

```

```

    (rule finite-fri-challenge-space)
next
  fix challenges
  assume challenges-in: challenges ∈ B
  have challenges-space:
    challenges ∈ fri-challenge-space (ceil-log clength)
  using challenge-subset challenges-in by blast
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (?E challenges) adversary-initial-state ≤ ?C challenges
  by (rule
    checked-staged-security-trace-fri-pair-challenge-query-path-fresh-fixed-bound
    [OF wf controlled challenges-space
      projection[OF challenges-in] subset[OF challenges-in]])
qed
finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-query-challenge-pair-set-hit-challenge-weighted-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and challenge-subset: B ⊆ fri-challenge-space (ceil-log clength)
  and challenge-projection: snd ' P ⊆ B
  and query-projection:
    ∧ challenges.
      challenges ∈ B ⇒
      fst ' (P ∩ (UNIV × {challenges})) ⊆ Q challenges
  and subset:
    ∧ challenges. challenges ∈ B ⇒
    Q challenges ⊆ fri-query-index-list-space
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-query-challenge-pair-set-hit s P))
      adversary-initial-state ≤
      (∑ challenges ∈ B.
        (1 / nnreal (CARD('f) ^ ceil-log clength)) *
        (nnreal (card (Q challenges)) *
          (1 / nnreal (card query-sample-space)) ^ rounds)) +
      hash-relation-budget-value
        (query-index-raw-list-relation-fiber-bound
          (⋃ challenges ∈ B. Q challenges))
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget) +
      hash-relation-budget-value (card B * ceil-log clength)
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget)
proof -
  let ?Q = (⋃ challenges ∈ B. Q challenges)
  let ?Path =

```

```

( $\sum$  challenges  $\in B$ .
  (1 / nnreal (CARD('f)  $\wedge$  ceil-log clength)) *
  (nnreal (card (Q challenges)) *
    (1 / nnreal (card query-sample-space))  $\wedge$  rounds))
let ?QueryPre =
  hash-relation-budget-value
  (query-index-raw-list-relation-fiber-bound ?Q)
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)
let ?ChallengePre =
  hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)
have finite-B: finite B
  using challenge-subset by (rule finite-subset)
  (rule finite-fri-challenge-space)
have fst-projection: fst ' P  $\subseteq$  ?Q
proof
  fix q
  assume q-in: q  $\in$  fst ' P
  then obtain c where pair: (q, c)  $\in$  P
  by auto
  have c-in: c  $\in$  B
  using challenge-projection pair by force
  have q  $\in$  fst ' (P  $\cap$  (UNIV  $\times$  {c}))
  using pair by force
  then have q  $\in$  Q c
  using query-projection[OF c-in] by blast
  then show q  $\in$  ?Q
  using c-in by blast
qed
have path-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-trace-fri-pair-challenge-query-path-fresh P B ?Q)
  adversary-initial-state  $\leq$  ?Path
  by (rule
    checked-staged-security-trace-fri-pair-challenge-query-path-fresh-fiber-sum
    [OF wf controlled challenge-subset query-projection subset])
have fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-trace-fri-pair-challenge-fresh P B)
  adversary-initial-state  $\leq$  ?Path + ?QueryPre
  by (rule
    checked-staged-security-trace-fri-pair-challenge-fresh-bound-from-query-path-fresh
    [OF wf controlled fst-projection path-bound])
have pair-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda$ s. trace-fri-query-challenge-pair-set-hit s P))

```

```

    adversary-initial-state ≤
    (?Path + ?QueryPre) + ?ChallengePre
  by (rule
    checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-fresh
    [OF wf controlled finite-B challenge-projection fresh-bound])
  then show ?thesis
    by (simp add: add.assoc)
qed

lemma checked-staged-security-composition-fri-query-challenge-pair-set-hit-challenge-weighted-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and challenge-subset:
     $\bigwedge dg. B\ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$ 
  and challenge-projection:  $\bigwedge dg. \text{snd } 'P\ dg \subseteq B\ dg$ 
  and query-projection:
     $\bigwedge dg \text{ challenges.}$ 
     $\text{challenges} \in B\ dg \implies$ 
     $\text{fst } '(P\ dg \cap (UNIV \times \{\text{challenges}\})) \subseteq$ 
     $Q\ dg \text{ challenges}$ 
  and subset:
     $\bigwedge dg \text{ challenges. challenges} \in B\ dg \implies$ 
     $Q\ dg \text{ challenges} \subseteq \text{fri-query-index-list-space}$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s\ P$ ))
    adversary-initial-state ≤
    ( $\sum dg \in UNIV.$ 
       $\sum \text{challenges} \in B\ dg.$ 
      ( $1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1))$ ) *
      ( $\text{nnreal } (\text{card } (Q\ dg \text{ challenges}))$ ) *
      ( $1 / \text{nnreal } (\text{card } \text{query-sample-space})$ )  $\wedge$  rounds)) +
    hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound
        ( $\bigcup dg \in UNIV. \bigcup \text{challenges} \in B\ dg.$ 
           $Q\ dg \text{ challenges}$ ))
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget) +
      hash-relation-budget-value
        ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
           $\text{card } (B\ dg) * \text{ceil-log } (\text{maxDegree} + 1)$ )
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget)

proof -
  let ?Q =
    ( $\bigcup dg \in UNIV. \bigcup \text{challenges} \in B\ dg. Q\ dg \text{ challenges}$ )
  let ?Path =
    ( $\sum dg \in UNIV.$ 

```



```


$$\sum \text{challenges} \in B \text{ dg.}$$


$$(1 / \text{nnreal} (\text{CARD}('f) \wedge \text{ceil-log} (\text{to-nat } dg + 1))) * \\
(\text{nnreal} (\text{card } (Q \text{ dg challenges})) * \\
(1 / \text{nnreal} (\text{card } \text{query-sample-space})) \wedge \text{rounds}))$$

let ?QueryPre =
  hash-relation-budget-value
  (query-index-raw-list-relation-fiber-bound ?Q)
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget)
let ?ChallengePre =
  hash-relation-budget-value
  ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
   card (B dg) * ceil-log (maxDegree + 1))
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget)
have finite-B:  $\bigwedge dg. \text{finite } (B \text{ dg})$ 
by (rule finite-subset[OF challenge-subset])
  (rule finite-fri-challenge-space)
have fst-projection:  $\bigwedge dg. \text{fst } ' P \text{ dg} \subseteq ?Q$ 
proof
  fix dg q
  assume q-in:  $q \in \text{fst } ' P \text{ dg}$ 
  then obtain c where pair:  $(q, c) \in P \text{ dg}$ 
  by auto
  have c-in:  $c \in B \text{ dg}$ 
  using challenge-projection[of dg] pair by force
  have  $q \in \text{fst } '(P \text{ dg} \cap (UNIV \times \{c\}))$ 
  using pair by force
  then have  $q \in Q \text{ dg } c$ 
  using query-projection[OF c-in] by blast
  then show  $q \in ?Q$ 
  using c-in by blast
qed
have path-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-composition-fri-pair-challenge-query-path-fresh P
   B ?Q)
  adversary-initial-state  $\leq ?Path$ 
by (rule
  checked-staged-security-composition-fri-pair-challenge-query-path-fresh-fiber-sum
  [OF wf controlled challenge-subset query-projection subset])
have fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-composition-fri-pair-challenge-fresh P B)
  adversary-initial-state  $\leq ?Path + ?QueryPre$ 
by (rule
  checked-staged-security-composition-fri-pair-challenge-fresh-bound-from-query-path-fresh
  [OF wf controlled fst-projection path-bound])
have pair-bound:

```

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. composition\text{-}fri\text{-}query\text{-}challenge\text{-}pair\text{-}set\text{-}hit\ s\ P$))
 $adversary\text{-}initial\text{-}state \leq$
 ($?Path + ?QueryPre$) $+ ?ChallengePre$
by (*rule*
checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-fresh
 [*OF wf controlled finite-B challenge-projection fresh-bound*])
then show *?thesis*
by (*simp add: add.assoc*)
qed

lemma *checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-pair-fraction-challenge-weighted:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *fraction:*
generic-fri-sampled-query-pair-fraction-bound trace-table-low-degree
 (*Not* $\circ$ *trace-table-low-degree*) (*clength* $- 1$) C

shows

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-sampled-query-bad-candidate)
 $adversary\text{-}initial\text{-}state \leq$
 $C +$
hash-relation-budget-value
 (*query-index-raw-list-relation-fiber-bound*
 ($\bigcup challenges \in fri\text{-}challenge\text{-}space$ (*ceil-log clength*)).
generic-fri-sampled-query-query-fiber trace-table-low-degree
 (*Not* $\circ$ *trace-table-low-degree*) (*clength* $- 1$) *challenges*))
 (*staged-attacker-query-budget budgets* $+$
staged-challenge-query-budget) $+$
hash-relation-budget-value
 (*card* (*fri-challenge-space* (*ceil-log clength*))) $\ast$ *ceil-log clength*)
 (*staged-attacker-query-budget budgets* $+$
staged-challenge-query-budget)

proof $-$

let $?P =$
trace-fri-sampled-query-bad-pair-union $\cap$
 (*fri-query-index-list-space* $\times UNIV$)
let $?B = fri\text{-}challenge\text{-}space$ (*ceil-log clength*)
let $?Q =$
 $\lambda challenges.$
generic-fri-sampled-query-query-fiber trace-table-low-degree
 (*Not* $\circ$ *trace-table-low-degree*) (*clength* $- 1$) *challenges*
let $?Path =$
 ($\sum challenges \in ?B.$
 ($1 / nnreal$ (*CARD*(*f*) $\wedge$ *ceil-log clength*))) $\ast$
 ($nnreal$ (*card* ($?Q$ *challenges*))) $\ast$
 ($1 / nnreal$ (*card query-sample-space*)) $\wedge$ *rounds*)

```

let ?QueryPre =
  hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound
      ( $\bigcup$  challenges  $\in$  ?B. ?Q challenges))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
let ?ChallengePre =
  hash-relation-budget-value
    (card ?B * ceil-log clength)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
have sampled-to-pair:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-sampled-query-bad-candidate)
  adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s$ . trace-fri-query-challenge-pair-set-hit s ?P))
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    intro:
      trace-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit
      split: option.splits prod.splits)
also have ...  $\leq$  ?Path + ?QueryPre + ?ChallengePre
proof –
  have challenge-subset:
    ?B  $\subseteq$  fri-challenge-space (ceil-log clength)
  by simp
have challenge-projection: snd ‘ ?P  $\subseteq$  ?B
  by (rule trace-fri-restricted-sampled-query-bad-pair-union-challenge-projection)
have query-projection:
   $\bigwedge$  challenges. challenges  $\in$  ?B  $\implies$ 
    fst ‘ (?P  $\cap$  (UNIV  $\times$  {challenges}))  $\subseteq$  ?Q challenges
  by (auto
    intro: generic-fri-sampled-query-query-fiber-subset
    simp: generic-fri-sampled-query-query-fiber-def
      generic-fri-sampled-query-restricted-bad-pairs-def
      fri-query-challenge-pair-query-fiber-def
      trace-fri-sampled-query-bad-pair-union-def)
have subset-Q:
   $\bigwedge$  challenges. challenges  $\in$  ?B  $\implies$ 
    ?Q challenges  $\subseteq$  fri-query-index-list-space
  by (rule generic-fri-sampled-query-query-fiber-subset)
show ?thesis
  by (rule
    checked-staged-security-trace-fri-query-challenge-pair-set-hit-challenge-weighted-bound
    [OF wf controlled challenge-subset challenge-projection]

```

```

      query-projection subset-Q])
qed
also have ... ≤ C + ?QueryPre + ?ChallengePre
proof -
  have round-eq:
    fri-round-count-for-degree-bound (clength - Suc 0) = ceil-log clength
  unfolding fri-round-count-for-degree-bound-def
  by (simp add: clength-pos)
  have fiber-fraction:
    nnreal
      (∑ challenges ∈ ?B. card (?Q challenges)) /
      (nnreal (CARD('f) ^ ceil-log clength) *
        nnreal query-sample-space-size ^ rounds)
    ≤ C
  using fraction
  unfolding trace-fri-sampled-query-pair-fraction-bound-iff-fiber-sum
  by (simp add: round-eq card-fri-query-index-list-space
    query-sample-space-size-pos algebra-simps)
  have prob-eq:
    ?Path =
    nnreal
      (∑ challenges ∈ ?B. card (?Q challenges)) /
      (nnreal (CARD('f) ^ ceil-log clength) *
        nnreal query-sample-space-size ^ rounds)
  using weighted-query-fiber-sum-eq
  [where I = ?B
    and f = λchallenges. card (?Q challenges)
    and A = CARD('f) ^ ceil-log clength
    and B = query-sample-space-size]
  by (simp add: finite-fri-challenge-space query-sample-space-size-pos
    card-query-sample-space algebra-simps)
  have ?Path ≤ C
    using fiber-fraction unfolding prob-eq .
  then show ?thesis
    by (simp add: add-mono)
qed
finally show ?thesis .
qed

```

lemma *checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-pair-fraction-challenge-weight*
fixes $C :: 'f \Rightarrow \text{prob}$
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *fraction:*
 $\bigwedge dg. \text{generic-fri-sampled-query-pair-fraction-bound}$
 $(\text{composition-table-low-degree } (to\text{-nat } dg))$
 $(\text{Not } \circ \text{composition-table-low-degree } \text{maxDegree}) (to\text{-nat } dg) (C \ dg)$
shows
 $wp\text{-event } (\text{checked-staged-security-experiment-with-data-state } A)$

```

(staged-security-with-data-state-verifier-event
 composition-fri-sampled-query-bad-candidate)
adversary-initial-state ≤
(∑ dg ∈ UNIV. C dg) +
hash-relation-budget-value
(query-index-raw-list-relation-fiber-bound
 (∪ dg ∈ UNIV.
  ∪ challenges ∈ fri-challenge-space (ceil-log (to-nat dg + 1)).
  generic-fri-sampled-query-query-fiber
   (composition-table-low-degree (to-nat dg))
   (Not ∘ composition-table-low-degree maxDegree)
   (to-nat dg) challenges))
(staged-attacker-query-budget budgets +
 staged-challenge-query-budget) +
hash-relation-budget-value
(∑ dg ∈ (UNIV :: 'f set).
 card (fri-challenge-space (ceil-log (to-nat dg + 1))) *
 ceil-log (maxDegree + 1))
(staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
proof -
let ?P =
  λdg. composition-fri-sampled-query-bad-pair-union dg ∩
    (fri-query-index-list-space × UNIV)
let ?B =
  λdg. fri-challenge-space (ceil-log (to-nat dg + 1))
let ?Q =
  λdg challenges.
    generic-fri-sampled-query-query-fiber
    (composition-table-low-degree (to-nat dg))
    (Not ∘ composition-table-low-degree maxDegree)
    (to-nat dg) challenges
let ?Path =
  (∑ dg ∈ UNIV.
   ∑ challenges ∈ ?B dg.
   (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
   (nnreal (card (?Q dg challenges)) *
    (1 / nnreal (card query-sample-space)) ^ rounds))
let ?QueryPre =
  hash-relation-budget-value
  (query-index-raw-list-relation-fiber-bound
   (∪ dg ∈ UNIV. ∪ challenges ∈ ?B dg. ?Q dg challenges))
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget)
let ?ChallengePre =
  hash-relation-budget-value
  (∑ dg ∈ (UNIV :: 'f set). card (?B dg) * ceil-log (maxDegree + 1))
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget)

```

```

have sampled-to-pair:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-sampled-query-bad-candidate)
    adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-query-challenge-pair-set-hit s ?P))
    adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    intro:
      composition-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit
      split: option.splits prod.splits)
also have ... ≤ ?Path + ?QueryPre + ?ChallengePre
proof –
  have challenge-subset:
     $\bigwedge dg. ?B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$ 
  by simp
  have challenge-projection:  $\bigwedge dg. \text{snd } ' ?P \ dg \subseteq ?B \ dg$ 
  by (rule
    composition-fri-restricted-sampled-query-bad-pair-union-challenge-projection)
  have query-projection:
     $\bigwedge dg \text{ challenges. challenges } \in ?B \ dg \implies$ 
     $\text{fst } ' (?P \ dg \cap (UNIV \times \{\text{challenges}\})) \subseteq$ 
     $?Q \ dg \text{ challenges}$ 
  by (auto
    intro: generic-fri-sampled-query-query-fiber-subset
    simp: generic-fri-sampled-query-query-fiber-def
      generic-fri-sampled-query-restricted-bad-pairs-def
      fri-query-challenge-pair-query-fiber-def
      composition-fri-sampled-query-bad-pair-union-def)
  have subset-Q:
     $\bigwedge dg \text{ challenges. challenges } \in ?B \ dg \implies$ 
     $?Q \ dg \text{ challenges } \subseteq \text{fri-query-index-list-space}$ 
  by (rule generic-fri-sampled-query-query-fiber-subset)
  show ?thesis
  by (rule
    checked-staged-security-composition-fri-query-challenge-pair-set-hit-challenge-weighted-bound
    [OF wf controlled challenge-subset challenge-projection
      query-projection subset-Q])
qed
also have ... ≤ ( $\sum dg \in UNIV. C \ dg$ ) + ?QueryPre + ?ChallengePre
proof –
  have path-le:
     $?Path \leq (\sum dg \in UNIV. C \ dg)$ 
  proof (rule sum-mono)
    fix dg :: 'f
    assume dg ∈ UNIV

```

```

let ?I = ?B dg
have round-eq:
  fri-round-count-for-degree-bound (to-nat dg) =
    ceil-log (to-nat dg + 1)
  unfolding fri-round-count-for-degree-bound-def by simp
have fiber-fraction:
  nnreal
    (∑ challenges ∈ ?I. card (?Q dg challenges)) /
    (nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) *
      nnreal query-sample-space-size ^ rounds)
    ≤ C dg
  using fraction[of dg]
  unfolding composition-fri-sampled-query-pair-fraction-bound-iff-fiber-sum
  by (simp add: round-eq card-fri-query-index-list-space
    query-sample-space-size-pos algebra-simps)
have prob-eq:
  (∑ challenges ∈ ?I.
    (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
    (nnreal (card (?Q dg challenges)) *
      (1 / nnreal (card query-sample-space)) ^ rounds)) =
  nnreal
    (∑ challenges ∈ ?I. card (?Q dg challenges)) /
    (nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) *
      nnreal query-sample-space-size ^ rounds)
  using weighted-query-fiber-sum-eq
  [where I = ?I
    and f = λchallenges. card (?Q dg challenges)
    and A = CARD('f) ^ ceil-log (to-nat dg + 1)
    and B = query-sample-space-size]
  by (simp add: finite-fri-challenge-space query-sample-space-size-pos
    card-query-sample-space algebra-simps)
show
  (∑ challenges ∈ ?I.
    (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
    (nnreal (card (?Q dg challenges)) *
      (1 / nnreal (card query-sample-space)) ^ rounds))
    ≤ C dg
  using fiber-fraction unfolding prob-eq .
qed
then show ?thesis
  by (simp add: add-mono)
qed
finally show ?thesis .
qed

end

end

```

```

theory Soundness-FRI-Sampled-Challenge-Cover
imports
  Soundness-FRI-Query-Challenge-Trace-Staged-Product
  Soundness-FRI-Internal-Bounds
begin

```

Generic sampled-query FRI challenge-cover reductions.

This layer is deliberately independent of trace/composition naming. The intended later FRI low-degree proof has to show that every sampled bad candidate produces a challenge list in an algebraic multiround bad set. The lemmas here turn exactly that fact, plus local round cardinality bounds, into the finite-cover/fraction interface consumed by the staged route.

```

lemma nnreal-nat-mult-divide-cancel-left:
  fixes a b c :: nat
  assumes  $0 < b$ 
  shows  $\text{nnreal } (a * b) / \text{nnreal } (b * c) = \text{nnreal } a / \text{nnreal } c$ 
proof -
  have b-ne:  $(\text{real } b) \neq 0$ 
    using assms by simp
  have  $\text{nn2real } (\text{nnreal } (a * b) / \text{nnreal } (b * c)) =$ 
     $\text{real } (a * b) / \text{real } (b * c)$ 
    by (simp only: nn2real-divide nn2real-nnreal)
  also have  $\dots = (\text{real } a * \text{real } b) / (\text{real } b * \text{real } c)$ 
    by (simp add: of-nat-mult)
  also have  $\dots = \text{real } a / \text{real } c$ 
    using b-ne by (simp add: field-simps)
  also have  $\dots = \text{nn2real } (\text{nnreal } a / \text{nnreal } c)$ 
    by (simp only: nn2real-divide nn2real-nnreal)
  finally have
     $\text{nn2real } (\text{nnreal } (a * b) / \text{nnreal } (b * c)) =$ 
     $\text{nn2real } (\text{nnreal } a / \text{nnreal } c) .$ 
  then show ?thesis
    by (simp only: nn2real-eq-iff)
qed

```

```

context soundness
begin

```

```

definition trace-fri-sampled-query-no-full-cover-failure-candidate
  ::  $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow$ 
     $(f \text{ list} \Rightarrow \text{nat} \Rightarrow f \text{ list} \Rightarrow f \text{ set}) \Rightarrow$ 
     $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$ 
where
  trace-fri-sampled-query-no-full-cover-failure-candidate s bad out
     $\longleftrightarrow$ 
     $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$ 
       $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$ 

```


composition-round-layers fr *candidate-query-idxs* *trace-openings*
trace-table *layers*.
trace-fri-partial-candidate-opening-evidence *s* out *trace-roots*
trace-bs *trace-final* *dg* *composition-roots* *composition-bs*
composition-final *fri-query-idxs* *trace-round-layers*
composition-round-layers fr *candidate-query-idxs* *trace-openings*
trace-table $\wedge$
generic-fri-sampled-query-candidate-evidence *trace-table-low-degree*
 $(\text{Not} \circ \text{trace-table-low-degree})$ *trace-table* $(\text{clength} - 1)$
trace-roots *trace-bs* *trace-final* *fri-query-idxs* *trace-round-layers*
layers $\wedge$
 $\neg$ *generic-fri-sampled-full-cover-failure* *trace-table-low-degree*
trace-table $(\text{clength} - 1)$ *trace-roots* *trace-bs* *trace-final*
fri-query-idxs *trace-round-layers*
 $(\text{fri-canonical-domains } (\text{length } \text{trace-bs}))$ *layers*
 $(\text{bad } \text{trace-table})$

definition *composition-fri-sampled-query-no-full-cover-failure-candidate*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-query-no-full-cover-failure-candidate *s* bad out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots } \text{trace-bs } \text{trace-final } \text{fri-dg}$
 $\text{opening-composition-roots } \text{composition-bs } \text{composition-final}$
 $\text{fri-query-idxs } \text{trace-round-layers } \text{composition-round-layers } \text{fr}$
 $\text{f-fri-roots } \text{f-final } \text{as } \text{dg } \text{composition-fri-roots } \text{final}$
 $\text{trace-query-idxs } \text{trace-openings } \text{composition-query-idxs}$
 $\text{composition-openings } \text{trace-table } \text{composition-table } \text{layers.}$
composition-fri-partial-candidate-opening-evidence *s* out *trace-roots*
trace-bs *trace-final* *fri-dg* *opening-composition-roots* *composition-bs*
composition-final *fri-query-idxs* *trace-round-layers*
composition-round-layers fr *f-fri-roots* *f-final* as *dg*
composition-fri-roots *final* *trace-query-idxs* *trace-openings*
composition-query-idxs *composition-openings* *trace-table*
composition-table $\wedge$
generic-fri-sampled-query-candidate-evidence
 $(\text{composition-table-low-degree } (\text{to-nat } \text{fri-dg}))$
 $(\text{Not} \circ \text{composition-table-low-degree } \text{maxDegree})$ *composition-table*
 $(\text{to-nat } \text{fri-dg})$ *opening-composition-roots* *composition-bs*
composition-final *fri-query-idxs* *composition-round-layers* *layers* $\wedge$
 $\neg$ *generic-fri-sampled-full-cover-failure*
 $(\text{composition-table-low-degree } (\text{to-nat } \text{fri-dg}))$ *composition-table*
 $(\text{to-nat } \text{fri-dg})$ *opening-composition-roots* *composition-bs*
composition-final *fri-query-idxs* *composition-round-layers*
 $(\text{fri-canonical-domains } (\text{length } \text{composition-bs}))$ *layers*
 $(\text{bad } \text{fri-dg } \text{composition-table})$

definition *trace-fri-sampled-query-no-full-cover-failure-pairs*

$:: ('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{nat list} \times 'f \text{ list}) \text{ set}$

where

trace-fri-sampled-query-no-full-cover-failure-pairs bad =
 $(\{(query\text{-}idxs, challenges).\}$
 $\exists candidate\text{-}table \text{ roots final-value round-layers layers.}$
 $generic\text{-}fri\text{-}sampled\text{-}query\text{-}candidate\text{-}evidence \text{ trace-table-low-degree}$
 $(Not \circ \text{trace-table-low-degree}) \text{ candidate-table (clength} - 1)$
 $\text{roots challenges final-value query-idxs round-layers layers} \wedge$
 $\neg generic\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}failure \text{ trace-table-low-degree}$
 $\text{candidate-table (clength} - 1) \text{ roots challenges final-value}$
 $\text{query-idxs round-layers (fri-canonical-domains (length challenges))}$
 $\text{layers (bad candidate-table)}\}$
 $\cap (\text{fri-query-index-list-space} \times UNIV))$

definition *trace-fri-sampled-query-no-full-cover-failure-query-fiber*

$:: ('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $'f \text{ list} \Rightarrow \text{nat list set}$

where

trace-fri-sampled-query-no-full-cover-failure-query-fiber bad
 $challenges =$
 $fri\text{-}query\text{-}challenge\text{-}pair\text{-}query\text{-}fiber$
 $(\text{trace-fri-sampled-query-no-full-cover-failure-pairs bad})$
 $challenges$

definition *composition-fri-sampled-query-no-full-cover-failure-pairs*

$:: ('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $'f \Rightarrow (\text{nat list} \times 'f \text{ list}) \text{ set}$

where

composition-fri-sampled-query-no-full-cover-failure-pairs bad dg =
 $(\{(query\text{-}idxs, challenges).\}$
 $\exists candidate\text{-}table \text{ roots final-value round-layers layers.}$
 $generic\text{-}fri\text{-}sampled\text{-}query\text{-}candidate\text{-}evidence$
 $(\text{composition-table-low-degree (to-nat dg)})$
 $(Not \circ \text{composition-table-low-degree maxDegree})$
 $\text{candidate-table (to-nat dg) roots challenges final-value}$
 $\text{query-idxs round-layers layers} \wedge$
 $\neg generic\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}failure$
 $(\text{composition-table-low-degree (to-nat dg)}) \text{ candidate-table}$
 $(\text{to-nat dg) roots challenges final-value query-idxs round-layers}$
 $(\text{fri-canonical-domains (length challenges)}) \text{ layers}$
 $(\text{bad dg candidate-table})\}$
 $\cap (\text{fri-query-index-list-space} \times UNIV))$

definition *composition-fri-sampled-query-no-full-cover-failure-query-fiber*

$:: ('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow \text{nat list set}$

where

composition-fri-sampled-query-no-full-cover-failure-query-fiber bad dg
challenges =
fri-query-challenge-pair-query-fiber
(composition-fri-sampled-query-no-full-cover-failure-pairs bad dg)
challenges

lemma *trace-fri-sampled-query-no-full-cover-failure-pairs-subset:*

trace-fri-sampled-query-no-full-cover-failure-pairs bad
 $\subseteq$ *trace-fri-sampled-query-bad-pair-union* $\cap$
(fri-query-index-list-space $\times$ UNIV)

proof

fix *pair*

assume *pair-in:*

pair $\in$ *trace-fri-sampled-query-no-full-cover-failure-pairs bad*

obtain *query-idxs challenges* **where** *pair-eq: pair = (query-idxs, challenges)*

by (*cases pair*)

from *pair-in pair-eq* **obtain** *candidate-table roots final-value round-layers*

layers **where** *query-space: query-idxs $\in$ fri-query-index-list-space*

and *evidence:*

generic-fri-sampled-query-candidate-evidence trace-table-low-degree

(Not $\circ$ trace-table-low-degree) candidate-table (clength $- 1$)

roots challenges final-value query-idxs round-layers layers

unfolding *trace-fri-sampled-query-no-full-cover-failure-pairs-def*

by *blast*

have *pair-set:*

(query-idxs, challenges) $\in$

generic-fri-sampled-query-bad-pair-set trace-table-low-degree

(Not $\circ$ trace-table-low-degree) candidate-table (clength $- 1$)

roots final-value round-layers layers

by (*rule generic-fri-sampled-query-bad-pair-setI [OF evidence]*)

then have *union:*

(query-idxs, challenges) $\in$ trace-fri-sampled-query-bad-pair-union

unfolding *trace-fri-sampled-query-bad-pair-union-def*

by (*rule generic-fri-sampled-query-bad-pair-unionI*)

show

pair $\in$ trace-fri-sampled-query-bad-pair-union $\cap$

(fri-query-index-list-space $\times$ UNIV)

using *pair-eq query-space union* **by** *simp*

qed

lemma *trace-fri-sampled-query-no-full-cover-failure-pairs-challenge-projection:*

snd ' trace-fri-sampled-query-no-full-cover-failure-pairs bad

$\subseteq$ *fri-challenge-space (ceil-log clength)*

using *trace-fri-sampled-query-no-full-cover-failure-pairs-subset*

trace-fri-restricted-sampled-query-bad-pair-union-challenge-projection

by *blast*

lemma *trace-fri-sampled-query-no-full-cover-failure-query-fiber-subset:*

trace-fri-sampled-query-no-full-cover-failure-query-fiber bad challenges

$\subseteq$ *fri-query-index-list-space*
unfolding *trace-fri-sampled-query-no-full-cover-failure-query-fiber-def*
using *trace-fri-sampled-query-no-full-cover-failure-pairs-subset*
by (*auto simp: fri-query-challenge-pair-query-fiber-def*)

lemma *composition-fri-sampled-query-no-full-cover-failure-pairs-subset:*
composition-fri-sampled-query-no-full-cover-failure-pairs bad dg
 $\subseteq$ *composition-fri-sampled-query-bad-pair-union dg* $\cap$
(fri-query-index-list-space $\times$ UNIV)

proof
fix *pair*
assume *pair-in:*
pair $\in$ composition-fri-sampled-query-no-full-cover-failure-pairs
bad dg
obtain *query-idxs challenges* **where** *pair-eq: pair = (query-idxs, challenges)*
by (*cases pair*)
from *pair-in pair-eq* **obtain** *candidate-table roots final-value round-layers*
layers **where** *query-space: query-idxs $\in$ fri-query-index-list-space*
and *evidence:*
generic-fri-sampled-query-candidate-evidence
(composition-table-low-degree (to-nat dg))
(Not $\circ$ composition-table-low-degree maxDegree)
candidate-table (to-nat dg) roots challenges final-value
query-idxs round-layers layers
unfolding *composition-fri-sampled-query-no-full-cover-failure-pairs-def*
by *blast*
have *pair-set:*
(query-idxs, challenges) $\in$
generic-fri-sampled-query-bad-pair-set
(composition-table-low-degree (to-nat dg))
(Not $\circ$ composition-table-low-degree maxDegree)
candidate-table (to-nat dg) roots final-value round-layers layers
by (*rule generic-fri-sampled-query-bad-pair-setI[OF evidence]*)
then have *union:*
(query-idxs, challenges) $\in$
composition-fri-sampled-query-bad-pair-union dg
unfolding *composition-fri-sampled-query-bad-pair-union-def*
by (*rule generic-fri-sampled-query-bad-pair-unionI*)
show
pair $\in$ composition-fri-sampled-query-bad-pair-union dg $\cap$
(fri-query-index-list-space $\times$ UNIV)
using *pair-eq query-space union* **by** *simp*
qed

lemma *composition-fri-sampled-query-no-full-cover-failure-pairs-challenge-projection:*
snd ‘ composition-fri-sampled-query-no-full-cover-failure-pairs bad dg
 $\subseteq$ *fri-challenge-space (ceil-log (to-nat dg + 1))*
using *composition-fri-sampled-query-no-full-cover-failure-pairs-subset*
composition-fri-restricted-sampled-query-bad-pair-union-challenge-projection

by blast

lemma *composition-fri-sampled-query-no-full-cover-failure-query-fiber-subset:*
 composition-fri-sampled-query-no-full-cover-failure-query-fiber bad dg
 challenges
 $\subseteq$ *fri-query-index-list-space*
unfolding *composition-fri-sampled-query-no-full-cover-failure-query-fiber-def*
using *composition-fri-sampled-query-no-full-cover-failure-pairs-subset*
by (*auto simp: fri-query-challenge-pair-query-fiber-def*)

lemma *trace-fri-sampled-query-no-full-cover-failure-candidate-imp-pair-hit:*
assumes
 trace-fri-sampled-query-no-full-cover-failure-candidate s bad out
shows

trace-fri-query-challenge-pair-set-hit s
 (*trace-fri-sampled-query-no-full-cover-failure-pairs bad*) *out*

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table layers **where** *evidence:*
 trace-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table

and *candidate:*

generic-fri-sampled-query-candidate-evidence trace-table-low-degree
 (*Not* $\circ$ *trace-table-low-degree*) *trace-table (clength - 1)*
 trace-roots trace-bs trace-final fri-query-idxs trace-round-layers
 layers

and *no-failure:*

$\neg$ *generic-fri-sampled-full-cover-failure trace-table-low-degree*
 trace-table (clength - 1) trace-roots trace-bs trace-final
 fri-query-idxs trace-round-layers
 (*fri-canonical-domains (length trace-bs)*) *layers*
 (*bad trace-table*)

unfolding *trace-fri-sampled-query-no-full-cover-failure-candidate-def*
by *fastforce*

have *transcript:*

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
 dg composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers

using *evidence*

unfolding *trace-fri-partial-candidate-opening-evidence-def* **by** *blast*

have *query-space:*

fri-query-idxs $\in$ *fri-query-index-list-space*

by (*rule accepted-fri-opening-transcript-query-index-list-space*
 [*OF transcript*])

```

have pair:
  (fri-query-idxs, trace-bs) ∈
    trace-fri-sampled-query-no-full-cover-failure-pairs bad
  unfolding trace-fri-sampled-query-no-full-cover-failure-pairs-def
  using candidate no-failure query-space by blast
show ?thesis
  unfolding trace-fri-query-challenge-pair-set-hit-def
  using transcript pair by blast
qed

lemma composition-fri-sampled-query-no-full-cover-failure-candidate-imp-pair-hit:
  assumes
    composition-fri-sampled-query-no-full-cover-failure-candidate s bad out
  shows
    composition-fri-query-challenge-pair-set-hit s
    (composition-fri-sampled-query-no-full-cover-failure-pairs bad) out
proof –
  from assms obtain trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table layers where
  evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  and candidate:
    generic-fri-sampled-query-candidate-evidence
    (composition-table-low-degree (to-nat fri-dg))
    (Not ◦ composition-table-low-degree maxDegree) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers layers
  and no-failure:
    ¬ generic-fri-sampled-full-cover-failure
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
    (fri-canonical-domains (length composition-bs)) layers
    (bad fri-dg composition-table)
  unfolding composition-fri-sampled-query-no-full-cover-failure-candidate-def
  by fastforce
have transcript:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    fri-dg opening-composition-roots composition-bs composition-final

```

```

    fri-query-idxs trace-round-layers composition-round-layers
  using evidence
  unfolding composition-fri-partial-candidate-opening-evidence-def by blast
have query-space:
  fri-query-idxs ∈ fri-query-index-list-space
  by (rule accepted-fri-opening-transcript-query-index-list-space
    [OF transcript])
have pair:
  (fri-query-idxs, composition-bs) ∈
    composition-fri-sampled-query-no-full-cover-failure-pairs bad fri-dg
  unfolding composition-fri-sampled-query-no-full-cover-failure-pairs-def
  using candidate no-failure query-space by blast
show ?thesis
  unfolding composition-fri-query-challenge-pair-set-hit-def
  using transcript pair by blast
qed

```

```

lemma trace-fri-sampled-query-no-full-cover-failure-candidate-not-None[simp]:
  ¬ trace-fri-sampled-query-no-full-cover-failure-candidate s bad None
  unfolding trace-fri-sampled-query-no-full-cover-failure-candidate-def
    trace-fri-partial-candidate-opening-evidence-def
    accepted-fri-opening-transcript-def
  by simp

```

```

lemma composition-fri-sampled-query-no-full-cover-failure-candidate-not-None[simp]:
  ¬ composition-fri-sampled-query-no-full-cover-failure-candidate s bad
    None
  unfolding composition-fri-sampled-query-no-full-cover-failure-candidate-def
    composition-fri-partial-candidate-opening-evidence-def
    accepted-fri-opening-transcript-def
  by simp

```

```

lemma trace-fri-sampled-query-no-full-cover-failure-candidate-imp-bad-candidate:
  assumes
    trace-fri-sampled-query-no-full-cover-failure-candidate s bad out
  shows trace-fri-sampled-query-bad-candidate s out
  using assms
  unfolding trace-fri-sampled-query-no-full-cover-failure-candidate-def
    trace-fri-sampled-query-bad-candidate-def
  by force

```

```

lemma composition-fri-sampled-query-no-full-cover-failure-candidate-imp-bad-candidate:
  assumes
    composition-fri-sampled-query-no-full-cover-failure-candidate s bad out
  shows composition-fri-sampled-query-bad-candidate s out
  using assms
  unfolding
    composition-fri-sampled-query-no-full-cover-failure-candidate-def
    composition-fri-sampled-query-bad-candidate-def

```

by force

lemma *trace-fri-sampled-query-bad-candidate-imp-no-full-cover-or-failure:*

assumes *sampld: trace-fri-sampled-query-bad-candidate s out*

shows

trace-fri-sampled-query-no-full-cover-failure-candidate s bad out $\vee$
trace-fri-sampled-full-cover-failure s bad out

proof –

from *sampld* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table layers

where *evidence:*

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

and *candidate:*

generic-fri-sampled-query-candidate-evidence trace-table-low-degree
(Not $\circ$ trace-table-low-degree) trace-table (clength – 1)
trace-roots trace-bs trace-final fri-query-idxs trace-round-layers
layers

unfolding *trace-fri-sampled-query-bad-candidate-def* **by** *blast*

have *canonical:*

generic-fri-canonical-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength – 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers layers

by (rule *generic-fri-sampled-query-candidate-evidenceD(1)*
[OF *candidate*])

have *chain:*

generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength – 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers
(fri-canonical-domains (length trace-bs)) layers

proof –

have *sampld-chain:*

generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength – 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers
(fri-canonical-domains (length trace-bs)) layers

by (rule *generic-fri-canonical-sampled-layer-chain-evidenceD*
[OF *canonical*])

then show *?thesis* .

qed

show *?thesis*

proof (cases

generic-fri-sampled-full-cover-failure trace-table-low-degree
trace-table (clength – 1) trace-roots trace-bs trace-final


```

    fri-query-idxs trace-round-layers
    (fri-canonical-domains (length trace-bs)) layers
    (bad trace-table))
  case False
  then have
    trace-fri-sampled-query-no-full-cover-failure-candidate s bad out
    unfolding trace-fri-sampled-query-no-full-cover-failure-candidate-def
    using evidence candidate by blast
  then show ?thesis by simp
next
case True
then have trace-fri-sampled-full-cover-failure s bad out
  unfolding trace-fri-sampled-full-cover-failure-def
  using evidence chain by blast
then show ?thesis by simp
qed
qed

lemma composition-fri-sampled-query-bad-candidate-imp-no-full-cover-or-failure:
  assumes sampled: composition-fri-sampled-query-bad-candidate s out
  shows
    composition-fri-sampled-query-no-full-cover-failure-candidate s bad out  $\vee$ 
    composition-fri-sampled-full-cover-failure s bad out
proof -
  from sampled obtain trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table layers
  where evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  and candidate:
    generic-fri-sampled-query-candidate-evidence
    (composition-table-low-degree (to-nat fri-dg))
    (Not  $\circ$  composition-table-low-degree maxDegree) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers layers
  unfolding composition-fri-sampled-query-bad-candidate-def by meson
have canonical:
  generic-fri-canonical-sampled-layer-chain-evidence
  (composition-table-low-degree (to-nat fri-dg)) composition-table
  (to-nat fri-dg) opening-composition-roots composition-bs

```

```

    composition-final fri-query-idxs composition-round-layers layers
  by (rule generic-fri-sampled-query-candidate-evidenceD(1)
    [OF candidate])
  have chain:
    generic-fri-sampled-layer-chain-evidence
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers
      (fri-canonical-domains (length composition-bs)) layers
  proof -
    have sampled-chain:
      generic-fri-sampled-layer-chain-evidence
        (composition-table-low-degree (to-nat fri-dg)) composition-table
        (to-nat fri-dg) opening-composition-roots composition-bs
        composition-final fri-query-idxs composition-round-layers
        (fri-canonical-domains (length composition-bs)) layers
      by (rule generic-fri-canonical-sampled-layer-chain-evidenceD
        [OF canonical])
    then show ?thesis .
  qed
  show ?thesis
  proof (cases
    generic-fri-sampled-full-cover-failure
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers
      (fri-canonical-domains (length composition-bs)) layers
      (bad fri-dg composition-table))
  case False
  then have
    composition-fri-sampled-query-no-full-cover-failure-candidate s bad out
  unfolding
    composition-fri-sampled-query-no-full-cover-failure-candidate-def
  using evidence candidate by meson
  then show ?thesis by simp
  next
  case True
  then have composition-fri-sampled-full-cover-failure s bad out
  unfolding composition-fri-sampled-full-cover-failure-def
  using evidence chain by meson
  then show ?thesis by simp
  qed
  qed

lemma wp-trace-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-failure:
  assumes no-failure-bound:
    wp-event verify-monad
      (trace-fri-sampled-query-no-full-cover-failure-candidate s bad) s
    ≤ N

```

and failure-bound:

$$wp\text{-event verify-monad} \quad (trace\text{-fri-sampled-full-cover-failure } s \text{ bad}) \ s \leq F$$
shows

$$wp\text{-event verify-monad} \quad (trace\text{-fri-sampled-query-bad-candidate } s) \ s \leq N + F$$
proof –
have $wp\text{-event verify-monad} \quad (trace\text{-fri-sampled-query-bad-candidate } s) \ s$

$$\leq wp\text{-event verify-monad} \quad (\lambda out.$$

$$trace\text{-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad out} \vee$$

$$trace\text{-fri-sampled-full-cover-failure } s \text{ bad out}) \ s$$

by (rule $wp\text{-event-mono}$)
(rule $trace\text{-fri-sampled-query-bad-candidate-imp-no-full-cover-or-failure}$)
also have $\dots \leq$

$$wp\text{-event verify-monad} \quad (trace\text{-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad}) \ s +$$

$$wp\text{-event verify-monad} \quad (trace\text{-fri-sampled-full-cover-failure } s \text{ bad}) \ s$$

by (rule $wp\text{-event-union-bound}$)
also have $\dots \leq N + F$
by (rule $add\text{-mono}[OF \ no\text{-failure-bound} \ failure\text{-bound}]$)
finally show $?thesis$.
qed

lemma $wp\text{-composition-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-failure}$:
assumes $no\text{-failure-bound}$:

$$wp\text{-event verify-monad} \quad (composition\text{-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad}) \ s \leq N$$
and failure-bound:

$$wp\text{-event verify-monad} \quad (composition\text{-fri-sampled-full-cover-failure } s \text{ bad}) \ s \leq F$$
shows

$$wp\text{-event verify-monad} \quad (composition\text{-fri-sampled-query-bad-candidate } s) \ s \leq N + F$$
proof –
have $wp\text{-event verify-monad} \quad (composition\text{-fri-sampled-query-bad-candidate } s) \ s$

$$\leq wp\text{-event verify-monad} \quad (\lambda out.$$

$$composition\text{-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad out} \vee$$

$$composition\text{-fri-sampled-full-cover-failure } s \text{ bad out}) \ s$$

by (rule $wp\text{-event-mono}$)
(rule $composition\text{-fri-sampled-query-bad-candidate-imp-no-full-cover-or-failure}$)
also have $\dots \leq$

$$wp\text{-event verify-monad}$$

```

      (composition-fri-sampled-query-no-full-cover-failure-candidate s bad)
    s +
    wp-event verify-monad
      (composition-fri-sampled-full-cover-failure s bad) s
  by (rule wp-event-union-bound)
also have ... ≤ N + F
  by (rule add-mono[OF no-failure-bound failure-bound])
finally show ?thesis .
qed

```

lemma *checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-failure:*

assumes *no-failure-bound:*

$\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒

wp-event verify-monad

(trace-fri-sampled-query-no-full-cover-failure-candidate
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad)
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ N

and *failure-bound:*

$\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒

wp-event verify-monad

(trace-fri-sampled-full-cover-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad)
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ F

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-sampled-query-bad-candidate)
adversary-initial-state ≤ N + F

proof (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)

show $\bigwedge s. \neg \text{trace-fri-sampled-query-bad-candidate } s \text{ None}$

unfolding *trace-fri-sampled-query-bad-candidate-def*

trace-fri-partial-candidate-opening-evidence-def

accepted-fri-opening-transcript-def

by *simp*

next

fix *data attacker-state*

assume *support:*

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)

```

    adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  show wp-event verify-monad (trace-fri-sampled-query-bad-candidate ?s) ?s
    ≤ N + F
  by (rule
    wp-trace-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-failure)
    (rule no-failure-bound[OF support], rule failure-bound[OF support])
qed

lemma checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-failure:
  assumes no-failure-bound:
    ∧data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (composition-fri-sampled-query-no-full-cover-failure-candidate
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)) bad)
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤ N
  and failure-bound:
    ∧data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (composition-fri-sampled-full-cover-failure
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)) bad)
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤ F
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-sampled-query-bad-candidate)
      adversary-initial-state ≤ N + F
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show ∧s. ¬ composition-fri-sampled-query-bad-candidate s None
  unfolding composition-fri-sampled-query-bad-candidate-def
    composition-fri-partial-candidate-opening-evidence-def
    accepted-fri-opening-transcript-def
  by simp
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈

```

```

    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  show wp-event verify-monad
    (composition-fri-sampled-query-bad-candidate ?s) ?s ≤ N + F
  by (rule
    wp-composition-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-failure)
    (rule no-failure-bound[OF support], rule failure-bound[OF support])
qed

lemma checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-staged-no-full-cover-or-failure:
assumes no-failure-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-sampled-query-no-full-cover-failure-candidate s bad))
  adversary-initial-state ≤ N
and failure-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-sampled-full-cover-failure s bad))
  adversary-initial-state ≤ F
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-sampled-query-bad-candidate)
  adversary-initial-state ≤ N + F
proof –
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-sampled-query-bad-candidate)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs out.
        trace-fri-sampled-query-no-full-cover-failure-candidate s bad
        out ∨
        trace-fri-sampled-full-cover-failure s bad out))
  adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
      dest: trace-fri-sampled-query-bad-candidate-imp-no-full-cover-or-failure
      split: option.splits prod.splits)
  also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (λout.
      staged-security-with-data-state-verifier-event
        (λs. trace-fri-sampled-query-no-full-cover-failure-candidate s

```

```

      bad) out  $\vee$ 
      staged-security-with-data-state-verifier-event
      ( $\lambda s$ . trace-fri-sampled-full-cover-failure s bad) out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      split: option.splits prod.splits)
  also have ...  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s$ . trace-fri-sampled-query-no-full-cover-failure-candidate s
          bad))
      adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s$ . trace-fri-sampled-full-cover-failure s bad))
      adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq N + F$ 
  by (rule add-mono[OF no-failure-bound failure-bound])
  finally show ?thesis .
qed

```

lemma *checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-staged-no-full-cover-or-failure*

assumes *no-failure-bound*:

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s$ .
      composition-fri-sampled-query-no-full-cover-failure-candidate s
        bad))
adversary-initial-state  $\leq N$ 

```

and *failure-bound*:

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s$ . composition-fri-sampled-full-cover-failure s bad))
adversary-initial-state  $\leq F$ 

```

shows

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-sampled-query-bad-candidate)
adversary-initial-state  $\leq N + F$ 

```

proof –

```

have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-sampled-query-bad-candidate)
adversary-initial-state  $\leq$ 
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s$  out.

```

```

      composition-fri-sampled-query-no-full-cover-failure-candidate s
      bad out ∨
      composition-fri-sampled-full-cover-failure s bad out))
    adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
      dest:
        composition-fri-sampled-query-bad-candidate-imp-no-full-cover-or-failure
        split: option.splits prod.splits)
  also have ... ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
    (λout.
      staged-security-with-data-state-verifier-event
      (λs.
        composition-fri-sampled-query-no-full-cover-failure-candidate s
        bad) out ∨
      staged-security-with-data-state-verifier-event
      (λs. composition-fri-sampled-full-cover-failure s bad) out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      split: option.splits prod.splits)
  also have ... ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs.
        composition-fri-sampled-query-no-full-cover-failure-candidate s
        bad))
    adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-sampled-full-cover-failure s bad))
    adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤ N + F
  by (rule add-mono[OF no-failure-bound failure-bound])
  finally show ?thesis .
qed

```

lemma *checked-staged-security-trace-fri-sampled-query-no-full-cover-failure-candidate-bound-from-pair-fraction-chall*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *fraction:*

generic-fri-sampled-query-pair-fraction-bound trace-table-low-degree
(Not ◦ trace-table-low-degree) (clength - 1) C

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
(λs. trace-fri-sampled-query-no-full-cover-failure-candidate s bad))


```

adversary-initial-state ≤
C +
hash-relation-budget-value
(query-index-raw-list-relation-fiber-bound
(⋃ challenges ∈ fri-challenge-space (ceil-log clength).
generic-fri-sampled-query-query-fiber trace-table-low-degree
(Not ∘ trace-table-low-degree) (clength - 1) challenges))
(staged-attacker-query-budget budgets +
staged-challenge-query-budget) +
hash-relation-budget-value
(card (fri-challenge-space (ceil-log clength)) * ceil-log clength)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)
proof -
have wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
(λs. trace-fri-sampled-query-no-full-cover-failure-candidate s bad))
adversary-initial-state
≤ wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-sampled-query-bad-candidate)
adversary-initial-state
by (rule wp-event-mono)
(auto simp: staged-security-with-data-state-verifier-event-def
intro:
trace-fri-sampled-query-no-full-cover-failure-candidate-imp-bad-candidate
split: option.splits prod.splits)
also have ... ≤
C +
hash-relation-budget-value
(query-index-raw-list-relation-fiber-bound
(⋃ challenges ∈ fri-challenge-space (ceil-log clength).
generic-fri-sampled-query-query-fiber trace-table-low-degree
(Not ∘ trace-table-low-degree) (clength - 1) challenges))
(staged-attacker-query-budget budgets +
staged-challenge-query-budget) +
hash-relation-budget-value
(card (fri-challenge-space (ceil-log clength)) * ceil-log clength)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)
by (rule
checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-pair-fraction-challenge-weighted
[OF wf controlled fraction])
finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-sampled-query-no-full-cover-failure-candidate-bound-from-restricted-pairs:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A

```

shows

$wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $(\lambda s. trace\text{-}fri\text{-}sampled\text{-}query\text{-}no\text{-}full\text{-}cover\text{-}failure\text{-}candidate \ s \ bad))$
 $adversary\text{-}initial\text{-}state \leq$
 $(\sum challenges \in fri\text{-}challenge\text{-}space \ (ceil\text{-}log \ clength).$
 $(1 / nnreal \ (CARD('f) \wedge ceil\text{-}log \ clength)) *$
 $(nnreal$
 $(card$
 $(trace\text{-}fri\text{-}sampled\text{-}query\text{-}no\text{-}full\text{-}cover\text{-}failure\text{-}query\text{-}fiber$
 $bad \ challenges)) *$
 $(1 / nnreal \ (card \ query\text{-}sample\text{-}space)) \wedge rounds)) +$
 $hash\text{-}relation\text{-}budget\text{-}value$
 $(query\text{-}index\text{-}raw\text{-}list\text{-}relation\text{-}fiber\text{-}bound$
 $(\bigcup challenges \in fri\text{-}challenge\text{-}space \ (ceil\text{-}log \ clength).$
 $trace\text{-}fri\text{-}sampled\text{-}query\text{-}no\text{-}full\text{-}cover\text{-}failure\text{-}query\text{-}fiber$
 $bad \ challenges))$
 $(staged\text{-}attacker\text{-}query\text{-}budget \ budgets +$
 $staged\text{-}challenge\text{-}query\text{-}budget) +$
 $hash\text{-}relation\text{-}budget\text{-}value$
 $(card \ (fri\text{-}challenge\text{-}space \ (ceil\text{-}log \ clength)) * ceil\text{-}log \ clength)$
 $(staged\text{-}attacker\text{-}query\text{-}budget \ budgets +$
 $staged\text{-}challenge\text{-}query\text{-}budget)$

proof –

let $?P = trace\text{-}fri\text{-}sampled\text{-}query\text{-}no\text{-}full\text{-}cover\text{-}failure\text{-}pairs \ bad$
let $?B = fri\text{-}challenge\text{-}space \ (ceil\text{-}log \ clength)$
let $?Q =$
 $trace\text{-}fri\text{-}sampled\text{-}query\text{-}no\text{-}full\text{-}cover\text{-}failure\text{-}query\text{-}fiber \ bad$
have $event\text{-}le:$
 $wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $(\lambda s. trace\text{-}fri\text{-}sampled\text{-}query\text{-}no\text{-}full\text{-}cover\text{-}failure\text{-}candidate \ s \ bad))$
 $adversary\text{-}initial\text{-}state \leq$
 $wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $(\lambda s. trace\text{-}fri\text{-}query\text{-}challenge\text{-}pair\text{-}set\text{-}hit \ s \ ?P))$
 $adversary\text{-}initial\text{-}state$
by $(rule \ wp\text{-}event\text{-}mono\text{-}on\text{-}support)$
 $(auto \ simp: staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event\text{-}def$
 $intro:$
 $trace\text{-}fri\text{-}sampled\text{-}query\text{-}no\text{-}full\text{-}cover\text{-}failure\text{-}candidate\text{-}imp\text{-}pair\text{-}hit$
 $split: option.splits \ prod.splits)$
have $pair\text{-}bound:$
 $wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $(\lambda s. trace\text{-}fri\text{-}query\text{-}challenge\text{-}pair\text{-}set\text{-}hit \ s \ ?P))$
 $adversary\text{-}initial\text{-}state \leq$
 $(\sum challenges \in ?B.$
 $(1 / nnreal \ (CARD('f) \wedge ceil\text{-}log \ clength)) *$

```

      (nnreal (card (?Q challenges)) *
       (1 / nnreal (card query-sample-space)) ^ rounds)) +
hash-relation-budget-value
  (query-index-raw-list-relation-fiber-bound
   (⋃ challenges ∈ ?B. ?Q challenges))
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget) +
hash-relation-budget-value (card ?B * ceil-log clength)
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget)
proof (rule
  checked-staged-security-trace-fri-query-challenge-pair-set-hit-challenge-weighted-bound
  [OF wf controlled])
show ?B ⊆ fri-challenge-space (ceil-log clength)
  by simp
next
show snd ‘ ?P ⊆ ?B
  by (rule
    trace-fri-sampled-query-no-full-cover-failure-pairs-challenge-projection)
next
fix challenges
assume challenges ∈ ?B
show fst ‘ (?P ∩ (UNIV × {challenges})) ⊆
  ?Q challenges
  unfolding trace-fri-sampled-query-no-full-cover-failure-query-fiber-def
    fri-query-challenge-pair-query-fiber-def
  by force
next
fix challenges
assume challenges ∈ ?B
show ?Q challenges ⊆ fri-query-index-list-space
  by (rule
    trace-fri-sampled-query-no-full-cover-failure-query-fiber-subset)
qed
show ?thesis
  by (rule order-trans[OF event-le pair-bound])
qed

```

lemma checked-staged-security-composition-fri-sampled-query-no-full-cover-failure-candidate-bound-from-restricted-p
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs.
 composition-fri-sampled-query-no-full-cover-failure-candidate s
 bad))
 adversary-initial-state ≤
 (∑ dg ∈ UNIV.

$\sum challenges \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1)).$
 $(1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1))) *$
 $(\text{nnreal}$
 $(\text{card}$
 $(\text{composition-fri-sampled-query-no-full-cover-failure-query-fiber}$
 $\text{bad } dg \text{ challenges})) *$
 $(1 / \text{nnreal } (\text{card query-sample-space})) \wedge \text{rounds})) +$
 $\text{hash-relation-budget-value}$
 $(\text{query-index-row-list-relation-fiber-bound}$
 $(\bigcup dg \in \text{UNIV.}$
 $\bigcup challenges \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1)).$
 $\text{composition-fri-sampled-query-no-full-cover-failure-query-fiber}$
 $\text{bad } dg \text{ challenges}))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}) +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card } (\text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))) *$
 $\text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
proof –
let $?P =$
 $\text{composition-fri-sampled-query-no-full-cover-failure-pairs bad}$
let $?B =$
 $\lambda dg. \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$
let $?Q =$
 $\text{composition-fri-sampled-query-no-full-cover-failure-query-fiber bad}$
have event-le:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s.$
 $\text{composition-fri-sampled-query-no-full-cover-failure-candidate } s$
 $\text{bad}))$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s ?P))$
 $\text{adversary-initial-state}$
by $(\text{rule wp-event-mono-on-support})$
 $(\text{auto simp: staged-security-with-data-state-verifier-event-def}$
 intro:
 $\text{composition-fri-sampled-query-no-full-cover-failure-candidate-imp-pair-hit}$
 $\text{split: option.splits prod.splits})$
have pair-bound:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s ?P))$
 $\text{adversary-initial-state} \leq$

```

( $\sum dg \in UNIV.$ 
 $\sum challenges \in ?B dg.$ 
  ( $1 / \text{nnreal } (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1))) *$ 
  ( $\text{nnreal } (card (?Q dg challenges)) *$ 
  ( $1 / \text{nnreal } (card \text{ query-sample-space})) \wedge \text{rounds})) +$ 
  hash-relation-budget-value
  ( $\text{query-index-raw-list-relation-fiber-bound}$ 
  ( $\bigcup dg \in UNIV. \bigcup challenges \in ?B dg.$ 
     $?Q dg challenges$ ))
  ( $\text{staged-attacker-query-budget budgets} +$ 
   $\text{staged-challenge-query-budget}$ ) +
  hash-relation-budget-value
  ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
     $card (?B dg) * \text{ceil-log } (maxDegree + 1))$ 
  ( $\text{staged-attacker-query-budget budgets} +$ 
   $\text{staged-challenge-query-budget}$ )
proof (rule
  checked-staged-security-composition-fri-query-challenge-pair-set-hit-challenge-weighted-bound
  [OF wf controlled])
fix dg
show  $?B dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$ 
  by simp
next
fix dg
show  $snd \text{ ' } ?P dg \subseteq ?B dg$ 
  by (rule
    composition-fri-sampled-query-no-full-cover-failure-pairs-challenge-projection)
next
fix dg challenges
assume  $challenges \in ?B dg$ 
show  $\text{fst ' } (?P dg \cap (UNIV \times \{challenges\})) \subseteq$ 
   $?Q dg challenges$ 
  unfolding
    composition-fri-sampled-query-no-full-cover-failure-query-fiber-def
    fri-query-challenge-pair-query-fiber-def
  by force
next
fix dg challenges
assume  $challenges \in ?B dg$ 
show  $?Q dg challenges \subseteq \text{fri-query-index-list-space}$ 
  by (rule
    composition-fri-sampled-query-no-full-cover-failure-query-fiber-subset)
qed
show ?thesis
  by (rule order-trans[OF event-le pair-bound])
qed

lemma checked-staged-security-composition-fri-sampled-query-no-full-cover-failure-candidate-bound-from-pair-fraction
  fixes  $C :: 'f \Rightarrow \text{prob}$ 

```

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *fraction*:
 $\wedge dg.$ generic-fri-sampled-query-pair-fraction-bound
 $(\text{composition-table-low-degree } (to\text{-nat } dg))$
 $(Not \circ \text{composition-table-low-degree } maxDegree) (to\text{-nat } dg) (C \ dg)$
shows
 $wp\text{-event } (checked\text{-staged-security-experiment-with-data-state } A)$
 $(staged\text{-security-with-data-state-verifier-event}$
 $(\lambda s.$
 $\text{composition-fri-sampled-query-no-full-cover-failure-candidate } s$
 $\text{bad}))$
 $adversary\text{-initial-state} \leq$
 $(\sum dg \in UNIV. C \ dg) +$
 $hash\text{-relation-budget-value}$
 $(query\text{-index-raw-list-relation-fiber-bound}$
 $(\bigcup dg \in UNIV.$
 $\bigcup challenges \in fri\text{-challenge-space } (ceil\text{-log } (to\text{-nat } dg + 1)).$
 $generic\text{-fri-sampled-query-query-fiber}$
 $(\text{composition-table-low-degree } (to\text{-nat } dg))$
 $(Not \circ \text{composition-table-low-degree } maxDegree)$
 $(to\text{-nat } dg) \ challenges))$
 $(staged\text{-attacker-query-budget } budgets +$
 $\text{staged-challenge-query-budget}) +$
 $hash\text{-relation-budget-value}$
 $(\sum dg \in (UNIV :: 'f \ set).$
 $card \ (fri\text{-challenge-space } (ceil\text{-log } (to\text{-nat } dg + 1))) *$
 $ceil\text{-log } (maxDegree + 1))$
 $(staged\text{-attacker-query-budget } budgets +$
 $\text{staged-challenge-query-budget})$
proof –
have $wp\text{-event } (checked\text{-staged-security-experiment-with-data-state } A)$
 $(staged\text{-security-with-data-state-verifier-event}$
 $(\lambda s.$
 $\text{composition-fri-sampled-query-no-full-cover-failure-candidate } s$
 $\text{bad}))$
 $adversary\text{-initial-state}$
 $\leq wp\text{-event } (checked\text{-staged-security-experiment-with-data-state } A)$
 $(staged\text{-security-with-data-state-verifier-event}$
 $\text{composition-fri-sampled-query-bad-candidate})$
 $adversary\text{-initial-state}$
by $(rule \ wp\text{-event-mono})$
 $(auto \ simp: staged\text{-security-with-data-state-verifier-event-def}$
 $intro:$
 $\text{composition-fri-sampled-query-no-full-cover-failure-candidate-imp-bad-candidate}$
 $split: option.splits \ prod.splits)$
also have $\dots \leq$
 $(\sum dg \in UNIV. C \ dg) +$
 $hash\text{-relation-budget-value}$

```

    (query-index-raw-list-relation-fiber-bound
      ( $\bigcup dg \in UNIV.$ 
        ( $\bigcup challenges \in fri\text{-}challenge\text{-}space$  ( $ceil\text{-}log$  ( $to\text{-}nat$   $dg + 1$ )).
          generic-fri-sampled-query-query-fiber
            ( $composition\text{-}table\text{-}low\text{-}degree$  ( $to\text{-}nat$   $dg$ ))
            ( $Not \circ composition\text{-}table\text{-}low\text{-}degree$   $maxDegree$ )
            ( $to\text{-}nat$   $dg$ )  $challenges$ ))
        ( $staged\text{-}attacker\text{-}query\text{-}budget$   $budgets +$ 
           $staged\text{-}challenge\text{-}query\text{-}budget$ ) +
         $hash\text{-}relation\text{-}budget\text{-}value$ 
        ( $\sum dg \in (UNIV :: 'f\ set).$ 
           $card$  ( $fri\text{-}challenge\text{-}space$  ( $ceil\text{-}log$  ( $to\text{-}nat$   $dg + 1$ ))) *
           $ceil\text{-}log$  ( $maxDegree + 1$ ))
        ( $staged\text{-}attacker\text{-}query\text{-}budget$   $budgets +$ 
           $staged\text{-}challenge\text{-}query\text{-}budget$ )
      )
    by (rule
      checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-pair-fraction-challenge-weighted
      [OF  $wf$  controlled fraction])
    finally show ?thesis .
  qed

lemma finite-generic-fri-bad-challenge-lists:
  finite (generic-fri-bad-challenge-lists  $n$   $bad$ )
proof (rule finite-subset)
  show generic-fri-bad-challenge-lists  $n$   $bad \subseteq fri\text{-}challenge\text{-}space$   $n$ 
    by (rule generic-fri-bad-challenge-lists-subset)
  show finite ( $fri\text{-}challenge\text{-}space$   $n$ )
    by (rule finite-fri-challenge-space)
  qed

lemma generic-fri-sampled-query-finite-cover-from-bad-challenge-lists:
  assumes cover:
     $\bigwedge candidate\text{-}table$   $roots$   $final\text{-}value$   $round\text{-}layers$   $layers$   $query\text{-}idxs$ 
       $challenges.$ 
     $query\text{-}idxs \in fri\text{-}query\text{-}index\text{-}list\text{-}space \implies$ 
    generic-fri-sampled-query-candidate-evidence  $low\text{-}degree$   $candidate\text{-}bad$ 
       $candidate\text{-}table$   $degree\text{-}bound$   $roots$   $challenges$   $final\text{-}value$   $query\text{-}idxs$ 
       $round\text{-}layers$   $layers \implies$ 
     $challenges \in$ 
      generic-fri-bad-challenge-lists
      ( $fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound$   $degree\text{-}bound$ )  $bad$ 
  shows
    generic-fri-sampled-query-finite-cover  $low\text{-}degree$   $candidate\text{-}bad$ 
       $degree\text{-}bound$ 
    (generic-fri-bad-challenge-lists
      ( $fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound$   $degree\text{-}bound$ )  $bad$ )
proof (rule generic-fri-sampled-query-finite-cover-from-candidate-challenge-cover)
  show finite
    (generic-fri-bad-challenge-lists

```

```

    (fri-round-count-for-degree-bound degree-bound) bad)
  by (rule finite-generic-fri-bad-challenge-lists)
fix candidate-table roots final-value round-layers layers query-idxs
challenges
assume query-space: query-idxs ∈ fri-query-index-list-space
and evidence:
  generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
  candidate-table degree-bound roots challenges final-value query-idxs
  round-layers layers
show challenges ∈
  generic-fri-bad-challenge-lists
  (fri-round-count-for-degree-bound degree-bound) bad
by (rule cover[OF query-space evidence])
qed

lemma generic-fri-sampled-query-projection-fraction-bound-from-bad-challenge-round-bounds:
  assumes cover:
     $\bigwedge$  candidate-table roots final-value round-layers layers query-idxs
    challenges.
    query-idxs ∈ fri-query-index-list-space  $\implies$ 
    generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers  $\implies$ 
    challenges ∈
    generic-fri-bad-challenge-lists
    (fri-round-count-for-degree-bound degree-bound) bad
  and local-bound:
     $\bigwedge i$  prefix.
     $i < \text{fri-round-count-for-degree-bound degree-bound} \implies$ 
    length prefix =  $i \implies$ 
    card (bad i prefix)  $\leq$  bd i
  and fraction-bound:
    nnreal
     $(\sum i \in \{..<\text{fri-round-count-for-degree-bound degree-bound}\}.$ 
     $\text{CARD('f)} \wedge i * \text{bd } i *$ 
     $\text{CARD('f)} \wedge$ 
     $(\text{fri-round-count-for-degree-bound degree-bound} - \text{Suc } i)) /$ 
    nnreal (CARD('f)  $\wedge$  fri-round-count-for-degree-bound degree-bound)
     $\leq C$ 
  shows
    generic-fri-sampled-query-projection-fraction-bound low-degree
    candidate-bad degree-bound C
proof -
  let ?n = fri-round-count-for-degree-bound degree-bound
  let ?B = generic-fri-bad-challenge-lists ?n bad
  let ?K =
     $(\sum i \in \{..<?n\}.$ 
     $\text{CARD('f)} \wedge i * \text{bd } i * \text{CARD('f)} \wedge (?n - \text{Suc } i))$ 
  have cover-B:

```


generic-fri-sampled-query-finite-cover low-degree candidate-bad
degree-bound ?B
by (rule *generic-fri-sampled-query-finite-cover-from-bad-challenge-lists*
[OF cover])
have *card-bound: card ?B ≤ ?K*
by (rule *generic-fri-bad-challenge-lists-card-bound-from-round-bounds*)
(rule local-bound)
have *ratio-bound:*
nnreal (card ?B) / nnreal (CARD('f) ^ ?n) ≤ C
proof –
have *nnreal (card ?B) / nnreal (CARD('f) ^ ?n)*
≤ nnreal ?K / nnreal (CARD('f) ^ ?n)
using *card-bound* **by** (rule *nnreal-nat-divide-right-mono*)
also have *... ≤ C*
by (rule *fraction-bound*)
finally show *?thesis .*
qed
show *?thesis*
by (rule *generic-fri-sampled-query-projection-fraction-bound-from-cover*
[OF cover-B ratio-bound])
qed

lemma *generic-fri-sampled-query-projection-fraction-bound-from-one-step-disagreement-subset:*
assumes *cover:*
 $\bigwedge$ *candidate-table roots final-value round-layers layers query-idxs*
challenges.
query-idxs ∈ fri-query-index-list-space ⇒
generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers ⇒
challenges ∈
generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound degree-bound) bad
and *local-subset:*
 $\bigwedge$ *i prefix.*
i < fri-round-count-for-degree-bound degree-bound ⇒
length prefix = i ⇒
bad i prefix ⊆
fri-one-step-disagreement-challenges
(len i) (pw i) (layer i prefix) (claimed i prefix) (domains i prefix)
and *fraction-bound:*
nnreal
 $(\sum i \in \{..<\text{fri-round-count-for-degree-bound degree-bound}\}.$
 $\text{CARD('f)} ^ i * (\text{len } i \text{ div } 2) *$
 $\text{CARD('f)} ^$
 $(\text{fri-round-count-for-degree-bound degree-bound} - \text{Suc } i)) /$
 $\text{nnreal } (\text{CARD('f)} ^ \text{fri-round-count-for-degree-bound degree-bound})$
 $\leq C$
shows

generic-fri-sampled-query-projection-fraction-bound low-degree
candidate-bad degree-bound C
proof (rule
generic-fri-sampled-query-projection-fraction-bound-from-bad-challenge-round-bounds
[OF cover - fraction-bound])
fix $i :: \text{nat}$ **and** $\text{prefix} :: 'f \text{ list}$
assume $i < \text{fri-round-count-for-degree-bound degree-bound}$
and $\text{prefix} : \text{length prefix} = i$
have $\text{card} (\text{bad } i \text{ prefix}) \leq$
 card
 $(\text{fri-one-step-disagreement-challenges}$
 $(\text{len } i) (\text{pw } i) (\text{layer } i \text{ prefix}) (\text{claimed } i \text{ prefix})$
 $(\text{domains } i \text{ prefix}))$
by (rule *card-mono*) (*simp-all add: local-subset[OF i prefix]*)
also have $\dots \leq \text{len } i \text{ div } 2$
by (rule *fri-one-step-disagreement-challenges-card-bound-local*)
finally show $\text{card} (\text{bad } i \text{ prefix}) \leq \text{len } i \text{ div } 2$.
qed

lemma *generic-fri-sampled-query-projection-fraction-bound-from-one-step-disagreement-subset-exact:*
assumes *cover:*
 $\bigwedge \text{candidate-table roots final-value round-layers layers query-idxs}$
 challenges.
 $\text{query-idxs} \in \text{fri-query-index-list-space} \implies$
 $\text{generic-fri-sampled-query-candidate-evidence low-degree candidate-bad}$
 $\text{candidate-table degree-bound roots challenges final-value query-idxs}$
 $\text{round-layers layers} \implies$
 $\text{challenges} \in$
 $\text{generic-fri-bad-challenge-lists}$
 $(\text{fri-round-count-for-degree-bound degree-bound}) \text{ bad}$
and *local-subset:*
 $\bigwedge i \text{ prefix.}$
 $i < \text{fri-round-count-for-degree-bound degree-bound} \implies$
 $\text{length prefix} = i \implies$
 $\text{bad } i \text{ prefix} \subseteq$
 $\text{fri-one-step-disagreement-challenges}$
 $(\text{len } i) (\text{pw } i) (\text{layer } i \text{ prefix}) (\text{claimed } i \text{ prefix}) (\text{domains } i \text{ prefix})$
shows
 $\text{generic-fri-sampled-query-projection-fraction-bound low-degree}$
 $\text{candidate-bad degree-bound}$
 $(\text{fri-one-step-disagreement-round-fraction}$
 $(\text{fri-round-count-for-degree-bound degree-bound}) \text{ len})$
proof (rule
generic-fri-sampled-query-projection-fraction-bound-from-one-step-disagreement-subset)
fix $\text{candidate-table roots final-value round-layers layers query-idxs challenges}$
assume $\text{query-idxs} : \text{query-idxs} \in \text{fri-query-index-list-space}$
and *evidence:*
 $\text{generic-fri-sampled-query-candidate-evidence low-degree candidate-bad}$
 $\text{candidate-table degree-bound roots challenges final-value query-idxs}$

```

    round-layers layers
  show challenges ∈
    generic-fri-bad-challenge-lists
    (fri-round-count-for-degree-bound degree-bound) bad
  by (rule cover[OF query-idxs evidence])
next
  fix i :: nat and prefix :: 'f list
  assume i: i < fri-round-count-for-degree-bound degree-bound
    and prefix: length prefix = i
  show bad i prefix ⊆
    fri-one-step-disagreement-challenges
    (len i) (pw i) (layer i prefix) (claimed i prefix) (domains i prefix)
  by (rule local-subset[OF i prefix])
next
  show nreal
    (∑ i ∈ {..fri-round-count-for-degree-bound degree-bound}.
      CARD('f) ^ i * (len i div 2) *
      CARD('f) ^
        (fri-round-count-for-degree-bound degree-bound - Suc i)) /
    nreal (CARD('f) ^ fri-round-count-for-degree-bound degree-bound)
    ≤ fri-one-step-disagreement-round-fraction
      (fri-round-count-for-degree-bound degree-bound) len
  by (rule fri-one-step-disagreement-round-fraction-refl-bound)
qed

```

lemma *generic-fri-sampled-query-pair-fraction-bound-from-bad-challenge-round-bounds:*

assumes cover:

$\bigwedge$ candidate-table roots final-value round-layers layers query-idxs
challenges.

query-idxs ∈ fri-query-index-list-space $\implies$

generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers $\implies$

challenges ∈

generic-fri-bad-challenge-lists

(fri-round-count-for-degree-bound degree-bound) bad

and local-bound:

$\bigwedge$ i prefix.

i < fri-round-count-for-degree-bound degree-bound $\implies$

length prefix = i $\implies$

card (bad i prefix) ≤ bd i

and fraction-bound:

nreal

(∑ i ∈ {..*fri-round-count-for-degree-bound degree-bound*}.
CARD('f) ^ i * bd i *
CARD('f) ^
(fri-round-count-for-degree-bound degree-bound - Suc i)) /
nreal (CARD('f) ^ fri-round-count-for-degree-bound degree-bound)
≤ C

shows
generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad degree-bound C

proof –
 let $?n = \text{fri-round-count-for-degree-bound degree-bound}$
 let $?B = \text{generic-fri-bad-challenge-lists } ?n \text{ bad}$
 let $?K =$
 $(\sum i \in \{..<?n\}.$
 $\text{CARD}(f) \wedge i * \text{bd } i * \text{CARD}(f) \wedge (?n - \text{Suc } i))$

have *projection*:
generic-fri-sampled-query-challenge-projection low-degree candidate-bad degree-bound $\subseteq ?B$

proof
fix *challenges*
assume *challenges-in*:
challenges $\in$
generic-fri-sampled-query-challenge-projection low-degree candidate-bad degree-bound
then obtain *query-idxs* **where** *pair*:
 $(\text{query-idxs}, \text{challenges}) \in$
generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad degree-bound
unfolding *generic-fri-sampled-query-challenge-projection-def* **by** *force*
then have *query-space*: *query-idxs $\in$ fri-query-index-list-space*
unfolding *generic-fri-sampled-query-restricted-bad-pairs-def* **by** *simp*
have *pair-union*:
 $(\text{query-idxs}, \text{challenges}) \in$
generic-fri-sampled-query-bad-pair-union low-degree candidate-bad degree-bound
using *pair*
unfolding *generic-fri-sampled-query-restricted-bad-pairs-def* **by** *simp*
then obtain *candidate-table roots final-value round-layers layers* **where**
pair-set:
 $(\text{query-idxs}, \text{challenges}) \in$
generic-fri-sampled-query-bad-pair-set low-degree candidate-bad candidate-table degree-bound roots final-value round-layers layers
unfolding *generic-fri-sampled-query-bad-pair-union-def* **by** *auto*
have *evidence*:
generic-fri-sampled-query-candidate-evidence low-degree candidate-bad candidate-table degree-bound roots challenges final-value query-idxs round-layers layers
by $(\text{rule } \text{generic-fri-sampled-query-bad-pair-setD}[\text{OF } \text{pair-set}])$
show *challenges $\in ?B$*
by $(\text{rule } \text{cover}[\text{OF } \text{query-space evidence}])$

qed
have *card-bound*: *card $?B \leq ?K$*
by $(\text{rule } \text{generic-fri-bad-challenge-lists-card-bound-from-round-bounds})$
 $(\text{rule } \text{local-bound})$
have *ratio-bound*:

```

nnreal (card ?B * card fri-query-index-list-space) /
  nnreal (card fri-query-index-list-space * CARD('f) ^ ?n)
  ≤ C
proof -
  have card-q-pos: 0 < card fri-query-index-list-space
    by (simp add: card-fri-query-index-list-space
      query-sample-space-size-pos)
  then have card-q-ne: card fri-query-index-list-space ≠ 0
    by simp
  have nnreal (card ?B * card fri-query-index-list-space) /
    nnreal (card fri-query-index-list-space * CARD('f) ^ ?n) =
    nnreal (card ?B) / nnreal (CARD('f) ^ ?n)
    by (rule nnreal-nat-mult-divide-cancel-left[OF card-q-pos])
  also have ... ≤ nnreal ?K / nnreal (CARD('f) ^ ?n)
    using card-bound by (rule nnreal-nat-divide-right-mono)
  also have ... ≤ C
    by (rule fraction-bound)
  finally show ?thesis .
qed
show ?thesis
  by (rule generic-fri-sampled-query-pair-fraction-bound-from-projection-and-fibers
    [OF finite-generic-fri-bad-challenge-lists projection - ratio-bound])
  (meson card-mono finite-fri-query-index-list-space
    generic-fri-sampled-query-query-fiber-subset)
qed

lemma generic-fri-sampled-query-pair-fraction-bound-from-one-step-disagreement-subset:
assumes cover:
  ∧ candidate-table roots final-value round-layers layers query-idxs
    challenges.
  query-idxs ∈ fri-query-index-list-space ⇒
  generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers ⇒
  challenges ∈
    generic-fri-bad-challenge-lists
    (fri-round-count-for-degree-bound degree-bound) bad
and local-subset:
  ∧ i prefix.
  i < fri-round-count-for-degree-bound degree-bound ⇒
  length prefix = i ⇒
  bad i prefix ⊆
    fri-one-step-disagreement-challenges
    (len i) (pw i) (layer i prefix) (claimed i prefix) (domains i prefix)
and fraction-bound:
  nnreal
  (∑ i ∈ {..

```

$$\frac{(\text{fri-round-count-for-degree-bound } \text{degree-bound} - \text{Suc } i))}{\text{nnreal } (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound } \text{degree-bound})}$$

$$\leq C$$

shows

generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad degree-bound C

proof (rule

generic-fri-sampled-query-pair-fraction-bound-from-bad-challenge-round-bounds
 $[\text{OF cover} - \text{fraction-bound}]$)

fix $i :: \text{nat}$ **and** $\text{prefix} :: 'f \text{ list}$

assume $i: i < \text{fri-round-count-for-degree-bound } \text{degree-bound}$
and $\text{prefix}: \text{length } \text{prefix} = i$

have $\text{card } (\text{bad } i \text{ prefix}) \leq$
 card
 $(\text{fri-one-step-disagreement-challenges}$
 $(\text{len } i) (\text{pw } i) (\text{layer } i \text{ prefix}) (\text{claimed } i \text{ prefix})$
 $(\text{domains } i \text{ prefix}))$

by (rule *card-mono*) (*simp-all add: local-subset[OF i prefix]*)

also have $\dots \leq \text{len } i \text{ div } 2$

by (rule *fri-one-step-disagreement-challenges-card-bound-local*)

finally show $\text{card } (\text{bad } i \text{ prefix}) \leq \text{len } i \text{ div } 2$.

qed

lemma *generic-fri-sampled-query-pair-fraction-bound-from-one-step-disagreement-subset-exact:*

assumes *cover:*

$\bigwedge \text{candidate-table roots final-value round-layers layers query-idxs}$
 challenges.

$\text{query-idxs} \in \text{fri-query-index-list-space} \implies$
 $\text{generic-fri-sampled-query-candidate-evidence low-degree candidate-bad}$
 $\text{candidate-table degree-bound roots challenges final-value query-idxs}$
 $\text{round-layers layers} \implies$
 $\text{challenges} \in$
 $\text{generic-fri-bad-challenge-lists}$
 $(\text{fri-round-count-for-degree-bound } \text{degree-bound}) \text{ bad}$

and *local-subset:*

$\bigwedge i \text{ prefix.}$
 $i < \text{fri-round-count-for-degree-bound } \text{degree-bound} \implies$
 $\text{length } \text{prefix} = i \implies$
 $\text{bad } i \text{ prefix} \subseteq$
 $\text{fri-one-step-disagreement-challenges}$
 $(\text{len } i) (\text{pw } i) (\text{layer } i \text{ prefix}) (\text{claimed } i \text{ prefix}) (\text{domains } i \text{ prefix})$

shows

generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad degree-bound
 $(\text{fri-one-step-disagreement-round-fraction}$
 $(\text{fri-round-count-for-degree-bound } \text{degree-bound}) \text{ len})$

proof (rule

generic-fri-sampled-query-pair-fraction-bound-from-one-step-disagreement-subset)

fix *candidate-table roots final-value round-layers layers query-idxs challenges*

```

assume query-idxs: query-idxs ∈ fri-query-index-list-space
and evidence:
  generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
  candidate-table degree-bound roots challenges final-value query-idxs
  round-layers layers
show challenges ∈
  generic-fri-bad-challenge-lists
  (fri-round-count-for-degree-bound degree-bound) bad
by (rule cover[OF query-idxs evidence])
next
fix i :: nat and prefix :: 'f list
assume i: i < fri-round-count-for-degree-bound degree-bound
and prefix: length prefix = i
show bad i prefix ⊆
  fri-one-step-disagreement-challenges
  (len i) (pw i) (layer i prefix) (claimed i prefix) (domains i prefix)
by (rule local-subset[OF i prefix])
next
show nnreal
  (∑ i ∈ {..fri-round-count-for-degree-bound degree-bound}.
    CARD('f) ^ i * (len i div 2) *
    CARD('f) ^
      (fri-round-count-for-degree-bound degree-bound - Suc i)) /
  nnreal (CARD('f) ^ fri-round-count-for-degree-bound degree-bound)
  ≤ fri-one-step-disagreement-round-fraction
    (fri-round-count-for-degree-bound degree-bound) len
by (rule fri-one-step-disagreement-round-fraction-refl-bound)
qed

lemma trace-fri-sampled-query-pair-fraction-bound-from-bad-challenge-round-bounds:
assumes cover:
  ∧ candidate-table roots final-value round-layers layers query-idxs
  challenges.
  query-idxs ∈ fri-query-index-list-space ⇒
  generic-fri-sampled-query-candidate-evidence trace-table-low-degree
  (Not ○ trace-table-low-degree) candidate-table (clength - 1)
  roots challenges final-value query-idxs round-layers layers ⇒
  challenges ∈
  generic-fri-bad-challenge-lists
  (fri-round-count-for-degree-bound (clength - 1)) bad
and local-bound:
  ∧ i prefix.
  i < fri-round-count-for-degree-bound (clength - 1) ⇒
  length prefix = i ⇒
  card (bad i prefix) ≤ bd i
and fraction-bound:
  nnreal
  (∑ i ∈ {..fri-round-count-for-degree-bound (clength - 1)}).
    CARD('f) ^ i * bd i *

```

$$\begin{aligned}
& \text{CARD}('f) \wedge \\
& \quad (\text{fri-round-count-for-degree-bound } (\text{clength} - 1) - \text{Suc } i)) / \\
& \text{nnreal } (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound } (\text{clength} - 1)) \\
& \leq C
\end{aligned}$$

shows

generic-fri-sampled-query-pair-fraction-bound trace-table-low-degree
(Not $\circ$ trace-table-low-degree) (clength - 1) C

by (rule
generic-fri-sampled-query-pair-fraction-bound-from-bad-challenge-round-bounds
[OF cover local-bound fraction-bound])

lemma *generic-fri-sampled-query-pair-fraction-bound-from-full-cover-round-bounds:*
assumes *full-cover:*
 $\bigwedge$ *candidate-table roots final-value round-layers layers query-idxs*
challenges.
query-idxs $\in$ fri-query-index-list-space $\implies$
generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers $\implies$
 $\exists$ *doms. generic-fri-sampled-layer-chain-full-cover low-degree*
candidate-table degree-bound roots challenges final-value query-idxs
round-layers doms layers bad

and *local-bound:*
 $\bigwedge i$ *prefix.*
i < fri-round-count-for-degree-bound degree-bound $\implies$
length prefix = i $\implies$
card (bad i prefix) $\leq$ bd i

and *fraction-bound:*
nnreal
 $(\sum i \in \{..<\text{fri-round-count-for-degree-bound degree-bound}\}.$
 $\text{CARD}('f) \wedge i * \text{bd } i *$
 $\text{CARD}('f) \wedge$
 $(\text{fri-round-count-for-degree-bound degree-bound} - \text{Suc } i)) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound degree-bound})$
 $\leq C$

shows
generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad
degree-bound C

proof (rule
generic-fri-sampled-query-pair-fraction-bound-from-bad-challenge-round-bounds
[OF - local-bound fraction-bound])

fix *candidate-table roots final-value round-layers layers query-idxs*
challenges

assume *query-space: query-idxs $\in$ fri-query-index-list-space*
and *evidence:*
generic-fri-sampled-query-candidate-evidence low-degree
candidate-bad candidate-table degree-bound roots challenges
final-value query-idxs round-layers layers

obtain *doms where cover:*

*generic-fri-sampled-layer-chain-full-cover low-degree candidate-table
 degree-bound roots challenges final-value query-idxs round-layers doms
 layers bad*
using *full-cover[OF query-space evidence]* **by** *blast*
have *bad:*
challenges ∈ generic-fri-bad-challenge-lists (length challenges) bad
by (rule *generic-fri-sampled-layer-chain-full-cover-bad-challenge-list*
[OF cover])
have *canonical:*
*generic-fri-canonical-sampled-layer-chain-evidence low-degree
 candidate-table degree-bound roots challenges final-value query-idxs
 round-layers layers*
by (rule *generic-fri-sampled-query-candidate-evidenceD(1)[OF evidence]*)
have *sampled:*
*generic-fri-sampled-layer-chain-evidence low-degree candidate-table
 degree-bound roots challenges final-value query-idxs round-layers
 (fri-canonical-domains (length challenges)) layers*
by (rule *generic-fri-canonical-sampled-layer-chain-evidenceD*
[OF canonical])
have *partial:*
*generic-fri-partial-evidence low-degree candidate-table degree-bound
 roots challenges final-value query-idxs round-layers*
by (rule *generic-fri-sampled-layer-chain-evidenceD(1)[OF sampled]*)
have *len:*
length challenges = fri-round-count-for-degree-bound degree-bound
using *generic-fri-partial-evidence-shapes(1,2)[OF partial]* **by** *simp*
show *challenges ∈*
generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound degree-bound) bad
using *bad* **by** (*simp add: len*)
qed

lemma *generic-fri-sampled-query-candidate-evidence-full-cover-if-no-failure:*

assumes *evidence:*

*generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
 candidate-table degree-bound roots challenges final-value query-idxs
 round-layers layers*

and *no-failure:*

$\neg$ *generic-fri-sampled-full-cover-failure low-degree candidate-table
 degree-bound roots challenges final-value query-idxs round-layers
 (fri-canonical-domains (length challenges)) layers bad*

shows

$\exists$ *doms. generic-fri-sampled-layer-chain-full-cover low-degree
 candidate-table degree-bound roots challenges final-value query-idxs
 round-layers doms layers bad*

proof —

have *canonical:*

*generic-fri-canonical-sampled-layer-chain-evidence low-degree
 candidate-table degree-bound roots challenges final-value query-idxs*

```

    round-layers layers
  by (rule generic-fri-sampled-query-candidate-evidenceD(1)[OF evidence])
have sampled:
  generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    (fri-canonical-domains (length challenges)) layers
  by (rule generic-fri-canonical-sampled-layer-chain-evidenceD
    [OF canonical])
show ?thesis
  using sampled no-failure
  unfolding generic-fri-sampled-layer-chain-full-cover-def
    generic-fri-sampled-full-cover-failure-def
  by blast
qed

lemma generic-fri-sampled-query-pair-fraction-bound-from-no-full-cover-failure-round-bounds:
  assumes no-failure:
     $\bigwedge$  candidate-table roots final-value round-layers layers query-idxs
      challenges.
      query-idxs  $\in$  fri-query-index-list-space  $\implies$ 
      generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
        candidate-table degree-bound roots challenges final-value query-idxs
        round-layers layers  $\implies$ 
       $\neg$  generic-fri-sampled-full-cover-failure low-degree candidate-table
        degree-bound roots challenges final-value query-idxs round-layers
        (fri-canonical-domains (length challenges)) layers bad
  and local-bound:
     $\bigwedge i$  prefix.
       $i < \text{fri-round-count-for-degree-bound degree-bound} \implies$ 
      length prefix =  $i \implies$ 
      card (bad i prefix)  $\leq$  bd i
  and fraction-bound:
    nnreal
       $(\sum i \in \{..<\text{fri-round-count-for-degree-bound degree-bound}\}.$ 
        CARD('f)  $\wedge$  i * bd i *
        CARD('f)  $\wedge$ 
        (fri-round-count-for-degree-bound degree-bound - Suc i)) /
      nnreal (CARD('f)  $\wedge$  fri-round-count-for-degree-bound degree-bound)
     $\leq C$ 
  shows
    generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad
      degree-bound C
proof (rule
  generic-fri-sampled-query-pair-fraction-bound-from-full-cover-round-bounds
  [OF - local-bound fraction-bound])
fix candidate-table roots final-value round-layers layers query-idxs
  challenges
assume query-space: query-idxs  $\in$  fri-query-index-list-space
and evidence:

```

generic-fri-sampled-query-candidate-evidence low-degree
candidate-bad candidate-table degree-bound roots challenges
final-value query-idxs round-layers layers
show $\exists$ doms. *generic-fri-sampled-layer-chain-full-cover low-degree*
candidate-table degree-bound roots challenges final-value query-idxs
round-layers doms layers bad
by (rule *generic-fri-sampled-query-candidate-evidence-full-cover-if-no-failure*
 $[OF\ evidence\ no-failure[OF\ query-space\ evidence]]$)
qed

lemma *trace-fri-sampled-query-pair-fraction-bound-from-full-cover-round-bounds:*

assumes *full-cover:*

$\bigwedge$ *candidate-table roots final-value round-layers layers query-idxs*
challenges.
query-idxs $\in$ *fri-query-index-list-space* $\implies$
generic-fri-sampled-query-candidate-evidence trace-table-low-degree
 $(Not \circ trace-table-low-degree)\ candidate-table\ (clength - 1)$
roots challenges final-value query-idxs round-layers layers $\implies$
 $\exists$ doms. *generic-fri-sampled-layer-chain-full-cover*
trace-table-low-degree candidate-table (clength - 1) roots
challenges final-value query-idxs round-layers doms layers bad

and *local-bound:*

$\bigwedge i$ *prefix.*
 $i < fri-round-count-for-degree-bound\ (clength - 1) \implies$
 $length\ prefix = i \implies$
 $card\ (bad\ i\ prefix) \leq bd\ i$

and *fraction-bound:*

$nnreal$
 $(\sum i \in \{..<fri-round-count-for-degree-bound\ (clength - 1)\}.$
 $CARD('f) \wedge i * bd\ i *$
 $CARD('f) \wedge$
 $(fri-round-count-for-degree-bound\ (clength - 1) - Suc\ i)) /$
 $nnreal\ (CARD('f) \wedge fri-round-count-for-degree-bound\ (clength - 1))$
 $\leq C$

shows

generic-fri-sampled-query-pair-fraction-bound trace-table-low-degree
 $(Not \circ trace-table-low-degree)\ (clength - 1)\ C$

by (rule

generic-fri-sampled-query-pair-fraction-bound-from-full-cover-round-bounds
 $[OF\ full-cover\ local-bound\ fraction-bound]$)

lemma *trace-fri-sampled-query-pair-fraction-bound-from-no-full-cover-failure-round-bounds:*

assumes *no-failure:*

$\bigwedge$ *candidate-table roots final-value round-layers layers query-idxs*
challenges.
query-idxs $\in$ *fri-query-index-list-space* $\implies$
generic-fri-sampled-query-candidate-evidence trace-table-low-degree
 $(Not \circ trace-table-low-degree)\ candidate-table\ (clength - 1)$
roots challenges final-value query-idxs round-layers layers $\implies$

$\neg$ generic-fri-sampled-full-cover-failure trace-table-low-degree
 candidate-table (clength - 1) roots challenges final-value
 query-idxs round-layers (fri-canonical-domains (length challenges))
 layers bad
and local-bound:
 $\bigwedge i$ prefix.
 $i < \text{fri-round-count-for-degree-bound (clength - 1)} \implies$
 $\text{length prefix} = i \implies$
 $\text{card (bad i prefix)} \leq \text{bd } i$
and fraction-bound:
 nnreal
 $(\sum i \in \{..<\text{fri-round-count-for-degree-bound (clength - 1)}\}.$
 $\text{CARD('f)} \wedge^i * \text{bd } i *$
 $\text{CARD('f)} \wedge$
 $(\text{fri-round-count-for-degree-bound (clength - 1)} - \text{Suc } i)) /$
 $\text{nnreal (CARD('f)} \wedge \text{fri-round-count-for-degree-bound (clength - 1))}$
 $\leq C$
shows
 generic-fri-sampled-query-pair-fraction-bound trace-table-low-degree
 (Not $\circ$ trace-table-low-degree) (clength - 1) C
by (rule
 generic-fri-sampled-query-pair-fraction-bound-from-no-full-cover-failure-round-bounds
 [OF no-failure local-bound fraction-bound])

lemma composition-fri-sampled-query-pair-fraction-bound-from-bad-challenge-round-bounds:
fixes dg :: 'f
assumes cover:
 $\bigwedge$ candidate-table roots final-value round-layers layers query-idxs
 challenges.
 $\text{query-idxs} \in \text{fri-query-index-list-space} \implies$
 generic-fri-sampled-query-candidate-evidence
 (composition-table-low-degree (to-nat dg))
 (Not $\circ$ composition-table-low-degree maxDegree) candidate-table
 (to-nat dg) roots challenges final-value query-idxs round-layers
 layers $\implies$
 challenges $\in$
 generic-fri-bad-challenge-lists
 (fri-round-count-for-degree-bound (to-nat dg)) (bad dg)
and local-bound:
 $\bigwedge dg$ i prefix.
 $i < \text{fri-round-count-for-degree-bound (to-nat dg)} \implies$
 $\text{length prefix} = i \implies$
 $\text{card (bad dg i prefix)} \leq \text{bd dg } i$
and fraction-bound:
 $\bigwedge dg.$
 nnreal
 $(\sum i \in \{..<\text{fri-round-count-for-degree-bound (to-nat dg)}\}.$
 $\text{CARD('f)} \wedge^i * \text{bd dg } i *$
 $\text{CARD('f)} \wedge$

$$\frac{(\text{fri-round-count-for-degree-bound } (\text{to-nat } dg) - \text{Suc } i))}{\text{nnreal } (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound } (\text{to-nat } dg))} \leq C \text{ } dg$$

shows

generic-fri-sampled-query-pair-fraction-bound
(composition-table-low-degree (to-nat dg))
(Not o composition-table-low-degree maxDegree) (to-nat dg) (C dg)

proof (rule
generic-fri-sampled-query-pair-fraction-bound-from-bad-challenge-round-bounds)

fix *candidate-table roots final-value round-layers layers query-idxs*
challenges

assume *query-space: query-idxs ∈ fri-query-index-list-space*

and *evidence:*
generic-fri-sampled-query-candidate-evidence
(composition-table-low-degree (to-nat dg))
(Not o composition-table-low-degree maxDegree) candidate-table
(to-nat dg) roots challenges final-value query-idxs round-layers
layers

show *challenges ∈*
generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound (to-nat dg)) (bad dg)

by (rule *cover[OF query-space evidence]*)

next

fix *i* **and** *prefix :: 'f list*

assume *i: i < fri-round-count-for-degree-bound (to-nat dg)*

and *len: length prefix = i*

show *card (bad dg i prefix) ≤ bd dg i*

by (rule *local-bound[OF i len]*)

next

show *nnreal*
 $(\sum i \in \{..<\text{fri-round-count-for-degree-bound } (\text{to-nat } dg)\}.$
 $\text{CARD}('f) \wedge i * \text{bd } dg \text{ } i *$
 $\text{CARD}('f) \wedge$
 $(\text{fri-round-count-for-degree-bound } (\text{to-nat } dg) - \text{Suc } i)) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound } (\text{to-nat } dg))$
 $\leq C \text{ } dg$
by (rule *fraction-bound*)

qed

lemma *composition-fri-sampled-query-pair-fraction-bound-from-full-cover-round-bounds:*

fixes *dg :: 'f*

assumes *full-cover:*
 $\bigwedge \text{candidate-table roots final-value round-layers layers query-idxs}$
challenges.
query-idxs ∈ fri-query-index-list-space $\implies$
generic-fri-sampled-query-candidate-evidence
(composition-table-low-degree (to-nat dg))
(Not o composition-table-low-degree maxDegree) candidate-table
(to-nat dg) roots challenges final-value query-idxs round-layers

$layers \Rightarrow$
 $\exists$ doms. generic-fri-sampled-layer-chain-full-cover
 (composition-table-low-degree (to-nat dg)) candidate-table
 (to-nat dg) roots challenges final-value query-idxs round-layers doms
 layers (bad dg)
and local-bound:
 $\wedge dg$ i prefix.
 $i < \text{fri-round-count-for-degree-bound (to-nat dg)} \Rightarrow$
 $\text{length prefix} = i \Rightarrow$
 $\text{card (bad dg i prefix)} \leq \text{bd dg i}$
and fraction-bound:
 $\wedge dg.$
 $nnreal$
 $(\sum i \in \{..<\text{fri-round-count-for-degree-bound (to-nat dg)}\}.$
 $\text{CARD('f)} \wedge i * \text{bd dg i} *$
 $\text{CARD('f)} \wedge$
 $(\text{fri-round-count-for-degree-bound (to-nat dg)} - \text{Suc i})) /$
 $nnreal (\text{CARD('f)} \wedge \text{fri-round-count-for-degree-bound (to-nat dg)})$
 $\leq C dg$
shows
 generic-fri-sampled-query-pair-fraction-bound
 (composition-table-low-degree (to-nat dg))
 (Not $\circ$ composition-table-low-degree maxDegree) (to-nat dg) (C dg)
proof (rule
 generic-fri-sampled-query-pair-fraction-bound-from-full-cover-round-bounds)
fix candidate-table roots final-value round-layers layers query-idxs
 challenges
assume query-space: query-idxs $\in$ fri-query-index-list-space
and evidence:
 generic-fri-sampled-query-candidate-evidence
 (composition-table-low-degree (to-nat dg))
 (Not $\circ$ composition-table-low-degree maxDegree) candidate-table
 (to-nat dg) roots challenges final-value query-idxs round-layers
 layers
show $\exists$ doms. generic-fri-sampled-layer-chain-full-cover
 (composition-table-low-degree (to-nat dg)) candidate-table
 (to-nat dg) roots challenges final-value query-idxs round-layers doms
 layers (bad dg)
by (rule full-cover[OF query-space evidence])
next
fix i and prefix :: 'f list
assume i: $i < \text{fri-round-count-for-degree-bound (to-nat dg)}$
and len: $\text{length prefix} = i$
show $\text{card (bad dg i prefix)} \leq \text{bd dg i}$
by (rule local-bound[OF i len])
next
show nnreal
 $(\sum i \in \{..<\text{fri-round-count-for-degree-bound (to-nat dg)}\}.$
 $\text{CARD('f)} \wedge i * \text{bd dg i} *$

$$\frac{\text{CARD}('f) \wedge (\text{fri-round-count-for-degree-bound } (to\text{-nat } dg) - \text{Suc } i))}{\text{nnreal } (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound } (to\text{-nat } dg))} \leq C \text{ dg}$$
by (rule fraction-bound)

qed

lemma *composition-fri-sampled-query-pair-fraction-bound-from-no-full-cover-failure-round-bounds:*
fixes $dg :: 'f$

assumes *no-failure:*

$\bigwedge$ candidate-table roots final-value round-layers layers query-idxs
 challenges.
 $query\text{-idxs} \in \text{fri-query-index-list-space} \implies$
generic-fri-sampled-query-candidate-evidence
 $(\text{composition-table-low-degree } (to\text{-nat } dg))$
 $(\text{Not} \circ \text{composition-table-low-degree } \text{maxDegree}) \text{ candidate-table}$
 $(to\text{-nat } dg) \text{ roots challenges final-value query-idxs round-layers}$
 $\text{layers} \implies$
 $\neg$ *generic-fri-sampled-full-cover-failure*
 $(\text{composition-table-low-degree } (to\text{-nat } dg)) \text{ candidate-table}$
 $(to\text{-nat } dg) \text{ roots challenges final-value query-idxs round-layers}$
 $(\text{fri-canonical-domains } (\text{length challenges})) \text{ layers } (\text{bad } dg)$

and *local-bound:*

$\bigwedge dg \text{ i prefix.}$
 $i < \text{fri-round-count-for-degree-bound } (to\text{-nat } dg) \implies$
 $\text{length prefix} = i \implies$
 $\text{card } (\text{bad } dg \text{ i prefix}) \leq \text{bd } dg \text{ i}$

and *fraction-bound:*

$\bigwedge dg.$
 nnreal
 $(\sum i \in \{..<\text{fri-round-count-for-degree-bound } (to\text{-nat } dg)\}).$
 $\text{CARD}('f) \wedge i * \text{bd } dg \text{ i} *$
 $\text{CARD}('f) \wedge$
 $(\text{fri-round-count-for-degree-bound } (to\text{-nat } dg) - \text{Suc } i)) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound } (to\text{-nat } dg))$
 $\leq C \text{ dg}$

shows

generic-fri-sampled-query-pair-fraction-bound
 $(\text{composition-table-low-degree } (to\text{-nat } dg))$
 $(\text{Not} \circ \text{composition-table-low-degree } \text{maxDegree}) (to\text{-nat } dg) (C \text{ dg})$

proof (rule

generic-fri-sampled-query-pair-fraction-bound-from-no-full-cover-failure-round-bounds)

fix candidate-table roots final-value round-layers layers query-idxs
 challenges

assume *query-space:* $query\text{-idxs} \in \text{fri-query-index-list-space}$

and *evidence:*

generic-fri-sampled-query-candidate-evidence
 $(\text{composition-table-low-degree } (to\text{-nat } dg))$
 $(\text{Not} \circ \text{composition-table-low-degree } \text{maxDegree}) \text{ candidate-table}$

```

      (to-nat dg) roots challenges final-value query-idxs round-layers
      layers
show  $\neg$  generic-fri-sampled-full-cover-failure
      (composition-table-low-degree (to-nat dg)) candidate-table
      (to-nat dg) roots challenges final-value query-idxs round-layers
      (fri-canonical-domains (length challenges)) layers (bad dg)
      by (rule no-failure[OF query-space evidence])
next
fix i and prefix :: 'f list
assume i: i < fri-round-count-for-degree-bound (to-nat dg)
      and len: length prefix = i
show card (bad dg i prefix)  $\leq$  bd dg i
      by (rule local-bound[OF i len])
next
show nnreal
      ( $\sum i \in \{..<\text{fri-round-count-for-degree-bound (to-nat dg)}\}$ .
      CARD('f)  $\wedge$  i * bd dg i *
      CARD('f)  $\wedge$ 
      (fri-round-count-for-degree-bound (to-nat dg) - Suc i)) /
      nnreal (CARD('f)  $\wedge$  fri-round-count-for-degree-bound (to-nat dg))
       $\leq$  C dg
      by (rule fraction-bound)
qed

```

lemma wp-trace-fri-sampled-query-bad-candidate-bound-from-bad-challenge-round-bounds:

```

assumes future: trace-fri-future-fresh s
and cover:
 $\bigwedge$  candidate-table roots final-value round-layers layers query-idxs
      challenges.
      query-idxs  $\in$  fri-query-index-list-space  $\implies$ 
      generic-fri-sampled-query-candidate-evidence trace-table-low-degree
      (Not  $\circ$  trace-table-low-degree) candidate-table (clength - 1)
      roots challenges final-value query-idxs round-layers layers  $\implies$ 
      challenges  $\in$ 
      generic-fri-bad-challenge-lists
      (fri-round-count-for-degree-bound (clength - 1)) bad
and local-bound:
 $\bigwedge$  i prefix.
      i < fri-round-count-for-degree-bound (clength - 1)  $\implies$ 
      length prefix = i  $\implies$ 
      card (bad i prefix)  $\leq$  bd i
and fraction-bound:
      nnreal
      ( $\sum i \in \{..<\text{fri-round-count-for-degree-bound (clength - 1)}\}$ .
      CARD('f)  $\wedge$  i * bd i *
      CARD('f)  $\wedge$ 
      (fri-round-count-for-degree-bound (clength - 1) - Suc i)) /
      nnreal (CARD('f)  $\wedge$  fri-round-count-for-degree-bound (clength - 1))
       $\leq$  C

```


shows
 $wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\ s)\ s$
 $\leq C$

proof (rule
 $wp\text{-}trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\text{-}bound\text{-}from\text{-}generic\text{-}projection\text{-}fraction$
 $[OF\ future]$)

show
 $generic\text{-}fri\text{-}sampled\text{-}query\text{-}projection\text{-}fraction\text{-}bound$
 $trace\text{-}table\text{-}low\text{-}degree\ (Not\ \circ\ trace\text{-}table\text{-}low\text{-}degree)$
 $(clength - 1)\ C$

by (rule
 $generic\text{-}fri\text{-}sampled\text{-}query\text{-}projection\text{-}fraction\text{-}bound\text{-}from\text{-}bad\text{-}challenge\text{-}round\text{-}bounds$
 $[OF\ cover\ local\text{-}bound\ fraction\text{-}bound]$)

qed

lemma $wp\text{-}trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\text{-}bound\text{-}from\text{-}one\text{-}step\text{-}disagreement\text{-}subset$:
assumes $future$: $trace\text{-}fri\text{-}future\text{-}fresh\ s$
and $cover$:
 $\bigwedge candidate\text{-}table\ roots\ final\text{-}value\ round\text{-}layers\ layers\ query\text{-}idxs$
 $challenges.$
 $query\text{-}idxs \in fri\text{-}query\text{-}index\text{-}list\text{-}space \implies$
 $generic\text{-}fri\text{-}sampled\text{-}query\text{-}candidate\text{-}evidence\ trace\text{-}table\text{-}low\text{-}degree$
 $(Not\ \circ\ trace\text{-}table\text{-}low\text{-}degree)\ candidate\text{-}table\ (clength - 1)$
 $roots\ challenges\ final\text{-}value\ query\text{-}idxs\ round\text{-}layers\ layers \implies$
 $challenges \in$
 $generic\text{-}fri\text{-}bad\text{-}challenge\text{-}lists$
 $(fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound\ (clength - 1))\ bad$

and $local\text{-}subset$:
 $\bigwedge i\ prefix.$
 $i < fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound\ (clength - 1) \implies$
 $length\ prefix = i \implies$
 $bad\ i\ prefix \subseteq$
 $fri\text{-}one\text{-}step\text{-}disagreement\text{-}challenges$
 $(len\ i)\ (pw\ i)\ (layer\ i\ prefix)\ (claimed\ i\ prefix)\ (domains\ i\ prefix)$

and $fraction\text{-}bound$:
 $nnreal$
 $(\sum i \in \{..<fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound\ (clength - 1)\}).$
 $CARD('f) \wedge i * (len\ i\ div\ 2) *$
 $CARD('f) \wedge$
 $(fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound\ (clength - 1) - Suc\ i)) /$
 $nnreal\ (CARD('f) \wedge fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound\ (clength - 1))$
 $\leq C$

shows
 $wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\ s)\ s$
 $\leq C$

proof (rule
 $wp\text{-}trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\text{-}bound\text{-}from\text{-}generic\text{-}projection\text{-}fraction$
 $[OF\ future]$)

show

generic-fri-sampled-query-projection-fraction-bound
trace-table-low-degree (Not o trace-table-low-degree)
(clength - 1) C
by (rule
generic-fri-sampled-query-projection-fraction-bound-from-one-step-disagreement-subset
[OF cover local-subset fraction-bound])
qed

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-from-one-step-disagreement-subset-exact:*
assumes *future: trace-fri-future-fresh s*
and *cover:*
 $\bigwedge$ *candidate-table roots final-value round-layers layers query-idxs*
challenges.
query-idxs ∈ fri-query-index-list-space ⇒
generic-fri-sampled-query-candidate-evidence trace-table-low-degree
(Not o trace-table-low-degree) candidate-table (clength - 1)
roots challenges final-value query-idxs round-layers layers ⇒
challenges ∈
generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound (clength - 1)) bad
and *local-subset:*
 $\bigwedge$ *i prefix.*
i < fri-round-count-for-degree-bound (clength - 1) ⇒
length prefix = i ⇒
bad i prefix ⊆
fri-one-step-disagreement-challenges
(len i) (pw i) (layer i prefix) (claimed i prefix) (domains i prefix)
shows
wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s
 $\leq$
fri-one-step-disagreement-round-fraction
(fri-round-count-for-degree-bound (clength - 1)) len
proof (rule *wp-trace-fri-sampled-query-bad-candidate-bound-from-one-step-disagreement-subset*)
show *trace-fri-future-fresh s*
by (rule *future*)
next
fix *candidate-table roots final-value round-layers layers query-idxs challenges*
assume *query-idxs: query-idxs ∈ fri-query-index-list-space*
and *evidence:*
generic-fri-sampled-query-candidate-evidence trace-table-low-degree
(Not o trace-table-low-degree) candidate-table (clength - 1)
roots challenges final-value query-idxs round-layers layers
show *challenges ∈*
generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound (clength - 1)) bad
by (rule *cover[OF query-idxs evidence]*)
next
fix *i :: nat and prefix :: 'f list*
assume *i: i < fri-round-count-for-degree-bound (clength - 1)*

and *prefix*: *length prefix = i*
show *bad i prefix* $\subseteq$
fri-one-step-disagreement-challenges
(len i) (pw i) (layer i prefix) (claimed i prefix) (domains i prefix)
by (*rule local-subset*[*OF i prefix*])
next
show *nnreal*
 $(\sum_{i \in \{..<\text{fri-round-count-for-degree-bound}(\text{clength} - 1)\}} \text{CARD}(f) \wedge i * (\text{len } i \text{ div } 2) * \text{CARD}(f) \wedge (\text{fri-round-count-for-degree-bound}(\text{clength} - 1) - \text{Suc } i)) / \text{nnreal}(\text{CARD}(f) \wedge \text{fri-round-count-for-degree-bound}(\text{clength} - 1)) \leq \text{fri-one-step-disagreement-round-fraction}(\text{fri-round-count-for-degree-bound}(\text{clength} - 1)) \text{ len}$
by (*rule fri-one-step-disagreement-round-fraction-refl-bound*)
qed

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-from-canonical-one-step-disagreement-subset*:
assumes *future*: *trace-fri-future-fresh s*
and *cover*:
 $\bigwedge \text{candidate-table roots final-value round-layers layers query-idxs challenges.}$
 $\text{query-idxs} \in \text{fri-query-index-list-space} \implies$
 $\text{generic-fri-sampled-query-candidate-evidence trace-table-low-degree}(\text{Not} \circ \text{trace-table-low-degree}) \text{ candidate-table}(\text{clength} - 1) \text{ roots challenges final-value query-idxs round-layers layers} \implies$
 $\text{challenges} \in \text{generic-fri-bad-challenge-lists}(\text{fri-round-count-for-degree-bound}(\text{clength} - 1)) \text{ bad}$
and *local-subset*:
 $\bigwedge i \text{ prefix.}$
 $i < \text{fri-round-count-for-degree-bound}(\text{clength} - 1) \implies$
 $\text{length prefix} = i \implies$
 $\text{bad } i \text{ prefix} \subseteq \text{fri-one-step-disagreement-challenges}(\text{trace-fri-canonical-layer-len } i) (\text{pw } i) (\text{layer } i \text{ prefix}) (\text{claimed } i \text{ prefix}) (\text{domains } i \text{ prefix})$
shows
 $\text{wp-event verify-monad}(\text{trace-fri-sampled-query-bad-candidate } s) s \leq \text{concrete-trace-fri-bad-challenge-fraction}$
using
 $\text{wp-trace-fri-sampled-query-bad-candidate-bound-from-one-step-disagreement-subset-exact}[\text{OF future cover local-subset}]$
by (*simp add*: *concrete-trace-fri-bad-challenge-fraction-def*)

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-from-bad-challenge-round-bounds*:
assumes *future*: *composition-fri-future-fresh s*
and *cover*:
 $\bigwedge dg \text{ candidate-table roots final-value round-layers layers query-idxs}$

challenges.
query-idxs $\in$ *fri-query-index-list-space* $\implies$
generic-fri-sampled-query-candidate-evidence
(composition-table-low-degree (to-nat dg))
(Not $\circ$ composition-table-low-degree maxDegree) candidate-table
(to-nat dg) roots challenges final-value query-idxs round-layers
layers $\implies$
challenges $\in$
generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound (to-nat dg)) (bad dg)
and *local-bound:*
 $\bigwedge dg \ i \text{ prefix.}$
 $i < \text{fri-round-count-for-degree-bound (to-nat dg)} \implies$
 $\text{length prefix} = i \implies$
 $\text{card (bad dg i prefix)} \leq \text{bd dg i}$
and *fraction-bound:*
 $\bigwedge dg.$
 nnreal
 $(\sum i \in \{.. < \text{fri-round-count-for-degree-bound (to-nat dg)}\}.$
 $\text{CARD('f)} \wedge i * \text{bd dg i} *$
 $\text{CARD('f)} \wedge$
 $(\text{fri-round-count-for-degree-bound (to-nat dg)} - \text{Suc } i)) /$
 $\text{nnreal (CARD('f)} \wedge \text{fri-round-count-for-degree-bound (to-nat dg))}$
 $\leq C$
shows
 $\text{wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s}$
 $\leq C$
proof (rule
wp-composition-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction
[OF future])
fix *dg* :: 'f
show
generic-fri-sampled-query-projection-fraction-bound
(composition-table-low-degree (to-nat dg))
(Not $\circ$ composition-table-low-degree maxDegree)
(to-nat dg) C
proof (rule
generic-fri-sampled-query-projection-fraction-bound-from-bad-challenge-round-bounds)
fix *candidate-table roots final-value round-layers layers query-idxs*
challenges
assume *query-space: query-idxs* $\in$ *fri-query-index-list-space*
and *evidence:*
generic-fri-sampled-query-candidate-evidence
(composition-table-low-degree (to-nat dg))
(Not $\circ$ composition-table-low-degree maxDegree) candidate-table
(to-nat dg) roots challenges final-value query-idxs round-layers
layers
show *challenges* $\in$
generic-fri-bad-challenge-lists

```

    (fri-round-count-for-degree-bound (to-nat dg)) (bad dg)
  by (rule cover[OF query-space evidence])
next
fix i and prefix :: 'f list
assume i: i < fri-round-count-for-degree-bound (to-nat dg)
  and len: length prefix = i
show card (bad dg i prefix) ≤ bd dg i
  by (rule local-bound[OF i len])
next
show nnreal
  (∑ i ∈ {.. $\text{fri-round-count-for-degree-bound (to-nat dg)}$ }.
    CARD('f) ^ i * bd dg i *
    CARD('f) ^
      (fri-round-count-for-degree-bound (to-nat dg) - Suc i)) /
  nnreal (CARD('f) ^ fri-round-count-for-degree-bound (to-nat dg))
≤ C
  by (rule fraction-bound)
qed
qed

lemma wp-composition-fri-sampled-query-bad-candidate-bound-from-one-step-disagreement-subset:
  assumes future: composition-fri-future-fresh s
  and cover:
    ∧ dg candidate-table roots final-value round-layers layers query-idxs
      challenges.
    query-idxs ∈ fri-query-index-list-space ⇒
    generic-fri-sampled-query-candidate-evidence
      (composition-table-low-degree (to-nat dg))
      (Not ∘ composition-table-low-degree maxDegree) candidate-table
      (to-nat dg) roots challenges final-value query-idxs round-layers
      layers ⇒
    challenges ∈
      generic-fri-bad-challenge-lists
      (fri-round-count-for-degree-bound (to-nat dg)) (bad dg)
  and local-subset:
    ∧ dg i prefix.
    i < fri-round-count-for-degree-bound (to-nat dg) ⇒
    length prefix = i ⇒
    bad dg i prefix ⊆
      fri-one-step-disagreement-challenges
      (len dg i) (pw dg i) (layer dg i prefix)
      (claimed dg i prefix) (domains dg i prefix)
  and fraction-bound:
    ∧ dg.
    nnreal
      (∑ i ∈ {.. $\text{fri-round-count-for-degree-bound (to-nat dg)}$ }.
        CARD('f) ^ i * (len dg i div 2) *
        CARD('f) ^
          (fri-round-count-for-degree-bound (to-nat dg) - Suc i)) /

```

```

      nnreal (CARD('f) ^ fri-round-count-for-degree-bound (to-nat dg))
    ≤ C
  shows
    wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s
    ≤ C
  proof (rule
    wp-composition-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction
    [OF future])
  fix dg :: 'f
  show
    generic-fri-sampled-query-projection-fraction-bound
      (composition-table-low-degree (to-nat dg))
      (Not ∘ composition-table-low-degree maxDegree)
      (to-nat dg) C
  proof (rule
    generic-fri-sampled-query-projection-fraction-bound-from-one-step-disagreement-subset)
  fix candidate-table roots final-value round-layers layers query-idxs
    challenges
  assume query-space: query-idxs ∈ fri-query-index-list-space
  and evidence:
    generic-fri-sampled-query-candidate-evidence
      (composition-table-low-degree (to-nat dg))
      (Not ∘ composition-table-low-degree maxDegree) candidate-table
      (to-nat dg) roots challenges final-value query-idxs round-layers
      layers
  show challenges ∈
    generic-fri-bad-challenge-lists
      (fri-round-count-for-degree-bound (to-nat dg)) (bad dg)
  by (rule cover[OF query-space evidence])
  next
  fix i and prefix :: 'f list
  assume i: i < fri-round-count-for-degree-bound (to-nat dg)
  and prefix: length prefix = i
  show bad dg i prefix ⊆
    fri-one-step-disagreement-challenges
      (len dg i) (pw dg i) (layer dg i prefix)
      (claimed dg i prefix) (domains dg i prefix)
  by (rule local-subset[OF i prefix])
  next
  show nnreal
    (∑ i ∈ {.. $\text{fri-round-count-for-degree-bound (to-nat dg)}$ }.
      CARD('f) ^ i * (len dg i div 2) *
      CARD('f) ^
        (fri-round-count-for-degree-bound (to-nat dg) - Suc i)) /
    nnreal (CARD('f) ^ fri-round-count-for-degree-bound (to-nat dg))
    ≤ C
  by (rule fraction-bound)
qed
qed

```

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-from-one-step-disagreement-subset-exact-bound:*
assumes *future: composition-fri-future-fresh s*
and *cover:*
 $\bigwedge dg$ *candidate-table roots final-value round-layers layers query-idxs*
challenges.
query-idxs $\in$ *fri-query-index-list-space* $\implies$
generic-fri-sampled-query-candidate-evidence
(composition-table-low-degree (to-nat dg))
(Not $\circ$ composition-table-low-degree maxDegree) candidate-table
(to-nat dg) roots challenges final-value query-idxs round-layers
layers $\implies$
challenges $\in$
generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound (to-nat dg)) (bad dg)
and *local-subset:*
 $\bigwedge dg$ *i prefix.*
i $<$ *fri-round-count-for-degree-bound (to-nat dg)* $\implies$
length prefix = i $\implies$
bad dg i prefix $\subseteq$
fri-one-step-disagreement-challenges
(len dg i) (pw dg i) (layer dg i prefix)
(claimed dg i prefix) (domains dg i prefix)
and *fraction-bound:*
 $\bigwedge dg.$
fri-one-step-disagreement-round-fraction
(fri-round-count-for-degree-bound (to-nat dg)) (len dg)
 $\leq C$
shows
wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s
 $\leq C$
proof (*rule*
wp-composition-fri-sampled-query-bad-candidate-bound-from-one-step-disagreement-subset
[OF future cover local-subset])
fix *dg*
show *nnreal*
 $(\sum i \in \{..<fri-round-count-for-degree-bound (to-nat dg)\}.$
 $CARD('f) \wedge i * (len dg i \text{ div } 2) *$
 $CARD('f) \wedge$
 $(fri-round-count-for-degree-bound (to-nat dg) - Suc i)) /$
 $nnreal (CARD('f) \wedge fri-round-count-for-degree-bound (to-nat dg))$
 $\leq C$
using *fraction-bound[of dg]*
by (*simp add: fri-one-step-disagreement-round-fraction-def*
fri-bad-challenge-round-fraction-def
fri-one-step-disagreement-round-mass-def
fri-bad-challenge-round-mass-def)
qed

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-from-canonical-one-step-disagreement-subset:*
assumes *future: composition-fri-future-fresh s*
and *cover:*
 $\bigwedge dg$ *candidate-table roots final-value round-layers layers query-idxs*
challenges.
query-idxs $\in$ *fri-query-index-list-space* $\implies$
generic-fri-sampled-query-candidate-evidence
(composition-table-low-degree (to-nat dg))
(Not $\circ$ composition-table-low-degree maxDegree) candidate-table
(to-nat dg) roots challenges final-value query-idxs round-layers
layers $\implies$
challenges $\in$
generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound (to-nat dg)) (bad dg)
and *local-subset:*
 $\bigwedge dg$ *i prefix.*
i $<$ *fri-round-count-for-degree-bound (to-nat dg)* $\implies$
length prefix = i $\implies$
bad dg i prefix $\subseteq$
fri-one-step-disagreement-challenges
(composition-fri-canonical-layer-len dg i) (pw dg i)
(layer dg i prefix) (claimed dg i prefix) (domains dg i prefix)
shows
wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s
 $\leq$ *concrete-composition-fri-bad-challenge-fraction*
proof (rule
wp-composition-fri-sampled-query-bad-candidate-bound-from-one-step-disagreement-subset-exact-bound)
show *composition-fri-future-fresh s*
by (rule *future*)
next
fix *dg candidate-table roots final-value round-layers layers query-idxs*
challenges
assume *query-idxs: query-idxs* $\in$ *fri-query-index-list-space*
and *evidence:*
generic-fri-sampled-query-candidate-evidence
(composition-table-low-degree (to-nat dg))
(Not $\circ$ composition-table-low-degree maxDegree) candidate-table
(to-nat dg) roots challenges final-value query-idxs round-layers
layers
show *challenges* $\in$
generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound (to-nat dg)) (bad dg)
by (rule *cover[OF query-idxs evidence]*)
next
fix *dg and i :: nat and prefix :: 'f list*
assume *i: i* $<$ *fri-round-count-for-degree-bound (to-nat dg)*
and *prefix: length prefix = i*
show *bad dg i prefix* $\subseteq$
fri-one-step-disagreement-challenges


```

      (composition-fri-canonical-layer-len dg i) (pw dg i)
      (layer dg i prefix) (claimed dg i prefix) (domains dg i prefix)
    by (rule local-subset[OF i prefix])
  next
  fix dg
  show fri-one-step-disagreement-round-fraction
    (fri-round-count-for-degree-bound (to-nat dg))
    (composition-fri-canonical-layer-len dg)
    ≤ concrete-composition-fri-bad-challenge-fraction
  by (rule concrete-composition-fri-bad-challenge-fraction-ge)
qed

end

end

```

```

theory Soundness-FRI-Zero-Round-Query-List-Bounds
imports
  Soundness-FRI-Zero-Round-Target
  Soundness-FRI-Query-List-Exact-Product
begin

```

Exact staged query-list bounds for fixed zero-round FRI targets.

The target table and final value must already be fixed before query sampling. This theory deliberately does not close the remaining candidate/verifier alignment gap; it only packages the exact-product query accounting for the fixed targets isolated in *Soundness-FRI-Zero-Round-Target*.

```

context soundness
begin

```

```

definition fri-zero-round-first-query-index-lists
  :: 'f list ⇒ 'f ⇒ nat list set
where
  fri-zero-round-first-query-index-lists table final =
    {query-idxs ∈ fri-query-index-list-space.
     query-idxs ! 0 ∈ fri-final-value-agreement-set table final}

```

```

lemma fri-zero-round-first-query-index-lists-subset:
  fri-zero-round-first-query-index-lists table final
    ⊆ fri-query-index-list-space
unfolding fri-zero-round-first-query-index-lists-def by blast

```

```

lemma finite-fri-zero-round-first-query-index-lists[simp]:
  finite (fri-zero-round-first-query-index-lists table final)
by (rule finite-subset
    [OF fri-zero-round-first-query-index-lists-subset])
    (rule finite-fri-query-index-list-space)

```

lemma *fri-zero-round-first-query-index-listsD*:
assumes *query-idxs* $\in$
fri-zero-round-first-query-index-lists table final
shows *query-idxs* $\in$ *fri-query-index-list-space*
and *query-idxs ! 0* $\in$ *fri-final-value-agreement-set table final*
using *assms*
unfolding *fri-zero-round-first-query-index-lists-def* **by** *blast+*

lemma *fri-zero-round-first-query-index-lists-card-le*:
 $\text{card } (\text{fri-zero-round-first-query-index-lists table final}) \leq$
 $\text{card } (\text{fri-final-value-agreement-set table final}) * \text{card query-sample-space} \wedge (\text{rounds} - 1)$

proof –
let *?T* = *fri-zero-round-first-query-index-lists table final*
let *?Tail* =
 $\{xs. \text{set } xs \subseteq \text{query-sample-space} \wedge \text{length } xs = \text{rounds} - 1\}$
let *?F* = $\lambda \text{query-idxs}. (\text{query-idxs ! } 0, \text{tl query-idxs})$
have *image-subset*:
 $?F ' ?T \subseteq \text{fri-final-value-agreement-set table final} \times ?Tail$

proof
fix *y*
assume $y \in ?F ' ?T$
then obtain *query-idxs* **where** *query-idxs-in*: *query-idxs* $\in ?T$
and *y-def*: $y = ?F \text{ query-idxs}$
by *blast*
have *query-space*: *query-idxs* $\in$ *fri-query-index-list-space*
and *first-in*:
 $\text{query-idxs ! } 0 \in \text{fri-final-value-agreement-set table final}$
using *fri-zero-round-first-query-index-listsD[OF query-idxs-in]*
by *blast+*
have *len*: $\text{length query-idxs} = \text{rounds}$
and *set-subset*: $\text{set query-idxs} \subseteq \text{query-sample-space}$
using *query-space* **unfolding** *fri-query-index-list-space-def* **by** *auto*
have *tail-len*: $\text{length } (\text{tl query-idxs}) = \text{rounds} - 1$
using *len* *rounds-positive* **by** $(\text{cases query-idxs}) \text{ simp-all}$
have *tail-set*: $\text{set } (\text{tl query-idxs}) \subseteq \text{set query-idxs}$
by $(\text{cases query-idxs}) \text{ auto}$
have *tail-subset*: $\text{set } (\text{tl query-idxs}) \subseteq \text{query-sample-space}$
using *tail-set* *set-subset* **by** *blast*
show $y \in \text{fri-final-value-agreement-set table final} \times ?Tail$
using *y-def* *first-in* *tail-len* *tail-subset* **by** *simp*

qed
have *inj*: *inj-on* *?F* *?T*
proof (*rule inj-onI*)
fix *xs* *ys*
assume *xs-in*: *xs* $\in ?T$
and *ys-in*: *ys* $\in ?T$

```

    and eq: ?F xs = ?F ys
have xs-len: length xs = rounds
    and ys-len: length ys = rounds
    using fri-zero-round-first-query-index-listsD(1)[OF xs-in]
    fri-zero-round-first-query-index-listsD(1)[OF ys-in]
    unfolding fri-query-index-list-space-def by auto
have xs-ne: xs ≠ []
    and ys-ne: ys ≠ []
    using xs-len ys-len rounds-positive by auto
have head-eq: xs ! 0 = ys ! 0
    using eq by simp
have tail-eq: tl xs = tl ys
    using eq by simp
from xs-ne obtain x xs' where xs-def: xs = x # xs'
    by (cases xs) auto
from ys-ne obtain y ys' where ys-def: ys = y # ys'
    by (cases ys) auto
show xs = ys
    using head-eq tail-eq unfolding xs-def ys-def by simp
qed
have card ?T = card (?F ' ?T)
    using inj by (rule card-image[symmetric])
also have ... ≤
    card (fri-final-value-agreement-set table final × ?Tail)
proof (rule card-mono)
    show finite (fri-final-value-agreement-set table final × ?Tail)
        by (intro finite-cartesian-product finite-fri-final-value-agreement-set
            finite-lists-length-eq[OF finite-query-sample-space])
    show ?F ' ?T ⊆ fri-final-value-agreement-set table final × ?Tail
        by (rule image-subset)
qed
also have ... =
    card (fri-final-value-agreement-set table final) * card ?Tail
    by (simp add: card-cartesian-product)
also have card ?Tail = card query-sample-space ^ (rounds - 1)
    by (rule card-lists-length-eq[OF finite-query-sample-space])
finally show ?thesis .
qed

lemma fri-zero-round-first-query-index-lists-card-bound-from-small:
  assumes small:
    card (fri-final-value-agreement-set table final) <
      query-sample-space-size
  shows card (fri-zero-round-first-query-index-lists table final) ≤
    (query-sample-space-size - 1) *
      query-sample-space-size ^ (rounds - 1)
proof -
  have agreement-le:
    card (fri-final-value-agreement-set table final) ≤

```

```

      query-sample-space-size - 1
    using small by linarith
  have card (fri-zero-round-first-query-index-lists table final) ≤
    card (fri-final-value-agreement-set table final) *
    card query-sample-space ^ (rounds - 1)
  by (rule fri-zero-round-first-query-index-lists-card-le)
  also have ... ≤
    (query-sample-space-size - 1) *
    query-sample-space-size ^ (rounds - 1)
  using agreement-le by simp
  finally show ?thesis .
qed

```

definition *trace-fri-zero-round-query-list-target-hit*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-zero-round-query-list-target-hit } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-table trace-final.}$
 $\text{trace-fri-query-index-list-set-hit } s$
 $(\text{fri-zero-round-first-query-index-lists trace-table trace-final})$
 $\text{out} \wedge$
 $\text{card (fri-final-value-agreement-set trace-table trace-final)} <$
 $\text{query-sample-space-size})$

lemma *trace-fri-zero-round-query-list-target-hitE*:
assumes *trace-fri-zero-round-query-list-target-hit s out*
obtains *trace-table trace-final* **where**
 $\text{trace-fri-query-index-list-set-hit } s$
 $(\text{fri-zero-round-first-query-index-lists trace-table trace-final})$
 out
 $\text{card (fri-final-value-agreement-set trace-table trace-final)} <$
 $\text{query-sample-space-size}$
using *assms*
unfolding *trace-fri-zero-round-query-list-target-hit-def* **by** *blast*

lemma *trace-fri-zero-round-checked-first-query-target-hit-imp-query-list-target*:
assumes *trace-fri-zero-round-checked-first-query-target-hit s out*
shows *trace-fri-zero-round-query-list-target-hit s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr trace-openings trace-table}$
where *fri-openings*:
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers}$
and *first-hit*:
 $\text{fri-query-idxs} ! 0 \in$

```

    fri-final-value-agreement-set trace-table trace-final
  and card-lt:
    card (fri-final-value-agreement-set trace-table trace-final) <
      query-sample-space-size
  unfolding trace-fri-zero-round-checked-first-query-target-hit-def
  by blast
have query-space:
  fri-query-idxs ∈ fri-query-index-list-space
  by (rule accepted-fri-opening-transcript-query-index-list-space
    [OF fri-openings])
have query-target:
  fri-query-idxs ∈
    fri-zero-round-first-query-index-lists trace-table trace-final
  using query-space first-hit
  unfolding fri-zero-round-first-query-index-lists-def by simp
have hit:
  trace-fri-query-index-list-set-hit s
    (fri-zero-round-first-query-index-lists trace-table trace-final) out
  unfolding trace-fri-query-index-list-set-hit-def
  using fri-openings query-target by blast
show ?thesis
  unfolding trace-fri-zero-round-query-list-target-hit-def
  using hit card-lt by blast
qed

lemma trace-fri-zero-round-candidate-first-query-target-hit-imp-query-list-target-or-alignment-gap:
  assumes trace-fri-zero-round-candidate-first-query-target-hit s out
  shows
    trace-fri-zero-round-query-list-target-hit s out ∨
    trace-fri-zero-round-candidate-first-query-alignment-gap s out
proof (rule trace-fri-zero-round-candidate-first-query-target-hitE[OF assms])
  fix trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
  assume evidence:
    trace-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
  and rounds-empty: length trace-bs = 0
  and not-final:
    ¬ fri-final-constant-consistent trace-table trace-final
  and candidate-first-hit:
    candidate-query-idxs ! 0 ∈
      fri-final-value-agreement-set trace-table trace-final
  and card-lt:
    card (fri-final-value-agreement-set trace-table trace-final) <

```

```

    query-sample-space-size
have fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final dg composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers
by (rule trace-fri-partial-candidate-opening-evidenceD(1)
  [OF evidence])
show ?thesis
proof (cases fri-query-idxs ! 0 ∈
  fri-final-value-agreement-set trace-table trace-final)
case True
have query-space:
  fri-query-idxs ∈ fri-query-index-list-space
by (rule accepted-fri-opening-transcript-query-index-list-space
  [OF fri-openings])
have query-target:
  fri-query-idxs ∈
    fri-zero-round-first-query-index-lists trace-table trace-final
using query-space True
unfolding fri-zero-round-first-query-index-lists-def by simp
have hit:
  trace-fri-query-index-list-set-hit s
  (fri-zero-round-first-query-index-lists trace-table trace-final)
  out
unfolding trace-fri-query-index-list-set-hit-def
using fri-openings query-target by blast
have trace-fri-zero-round-query-list-target-hit s out
unfolding trace-fri-zero-round-query-list-target-hit-def
using hit card-lt by blast
then show ?thesis by blast
next
case False
have gap:
  trace-fri-zero-round-candidate-first-query-alignment-gap s out
unfolding trace-fri-zero-round-candidate-first-query-alignment-gap-def
by (intro exI conjI)
  (rule evidence, rule rounds-empty, rule not-final,
  rule candidate-first-hit, rule False, rule card-lt)
then show ?thesis by blast
qed
qed

lemma trace-fri-zero-round-final-obstruction-imp-query-list-target-or-candidate-gaps:
  assumes trace-fri-zero-round-final-obstruction s out
  shows
    trace-fri-zero-round-query-list-target-hit s out ∨
    trace-fri-zero-round-candidate-first-query-value-gap s out ∨
    trace-fri-zero-round-candidate-first-query-alignment-gap s out
proof (rule trace-fri-zero-round-final-obstruction-agreement-set-card-lt

```

```

[OF assms])
fix trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
assume evidence:
    trace-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
and rounds-empty: length trace-bs = 0
and not-final:
     $\neg$  fri-final-constant-consistent trace-table trace-final
and card-lt:
    card (fri-final-value-agreement-set trace-table trace-final) <
        query-sample-space-size
show ?thesis
proof (cases candidate-query-idxs ! 0 ∈
    fri-final-value-agreement-set trace-table trace-final)
case True
have candidate-hit:
    trace-fri-zero-round-candidate-first-query-target-hit s out
    unfolding trace-fri-zero-round-candidate-first-query-target-hit-def
    by (intro exI conjI)
        (rule evidence, rule rounds-empty, rule not-final, rule True,
        rule card-lt)
then show ?thesis
    using
        trace-fri-zero-round-candidate-first-query-target-hit-imp-query-list-target-or-alignment-gap
    by blast
next
case False
have value-gap:
    trace-fri-zero-round-candidate-first-query-value-gap s out
    unfolding trace-fri-zero-round-candidate-first-query-value-gap-def
    by (intro exI conjI)
        (rule evidence, rule rounds-empty, rule not-final, rule False,
        rule card-lt)
then show ?thesis by blast
qed
qed

lemma wp-trace-fri-zero-round-final-obstruction-bound-from-query-list-target-and-candidate-gaps:
assumes target-bound:
    wp-event verify-monad
    (trace-fri-zero-round-query-list-target-hit s)  $s \leq Q$ 
and value-bound:
    wp-event verify-monad

```

```

    (trace-fri-zero-round-candidate-first-query-value-gap s) s ≤ V
  and alignment-bound:
    wp-event verify-monad
    (trace-fri-zero-round-candidate-first-query-alignment-gap s) s ≤ A
  shows
    wp-event verify-monad
    (trace-fri-zero-round-final-obstruction s) s ≤ Q + V + A
proof -
  have event-le:
    wp-event verify-monad
    (trace-fri-zero-round-final-obstruction s) s ≤
    wp-event verify-monad
    (λout.
      trace-fri-zero-round-query-list-target-hit s out ∨
      trace-fri-zero-round-candidate-first-query-value-gap s out ∨
      trace-fri-zero-round-candidate-first-query-alignment-gap s out) s
  by (rule wp-event-mono)
  (rule trace-fri-zero-round-final-obstruction-imp-query-list-target-or-candidate-gaps)
  also have ... ≤
    wp-event verify-monad
    (trace-fri-zero-round-query-list-target-hit s) s +
    wp-event verify-monad
    (λout.
      trace-fri-zero-round-candidate-first-query-value-gap s out ∨
      trace-fri-zero-round-candidate-first-query-alignment-gap s out) s
  by (rule wp-event-union-bound)
  also have ... ≤
    wp-event verify-monad
    (trace-fri-zero-round-query-list-target-hit s) s +
    (wp-event verify-monad
      (trace-fri-zero-round-candidate-first-query-value-gap s) s +
      wp-event verify-monad
      (trace-fri-zero-round-candidate-first-query-alignment-gap s) s)
  by (intro add-mono order.refl wp-event-union-bound)
  also have ... ≤ Q + (V + A)
  by (intro add-mono target-bound value-bound alignment-bound)
  also have ... = Q + V + A
  by (simp add: add.assoc)
  finally show ?thesis .
qed

```

```

lemma checked-staged-security-trace-fri-zero-round-query-list-target-exact-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs.
        trace-fri-query-index-list-set-hit s

```


$$\begin{aligned} & \text{(fri-zero-round-first-query-index-lists table final))} \\ \text{adversary-initial-state} & \leq \\ \text{nnreal} & \\ & (\text{card (fri-zero-round-first-query-index-lists table final)}) * \\ & (1 / \text{nnreal (card query-sample-space)}) ^ \text{rounds} + \\ \text{hash-relation-budget-value} & \\ & (\text{query-index-raw-list-relation-fiber-bound} \\ & \text{(fri-zero-round-first-query-index-lists table final)}) \\ & (\text{staged-attacker-query-budget budgets} + \\ & \text{staged-challenge-query-budget}) \\ \text{by (rule checked-staged-security-trace-fri-query-index-list-set-hit-exact-product-bound} & \\ \text{[OF wf controlled fri-zero-round-first-query-index-lists-subset])} & \end{aligned}$$

lemma *checked-staged-security-trace-fri-zero-round-query-list-target-small-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *small:*

$$\begin{aligned} & \text{card (fri-final-value-agreement-set table final)} < \\ & \text{query-sample-space-size} \end{aligned}$$

shows

$$\begin{aligned} & \text{wp-event (checked-staged-security-experiment-with-data-state A)} \\ & (\text{staged-security-with-data-state-verifier-event} \\ & (\lambda s. \\ & \text{trace-fri-query-index-list-set-hit s} \\ & \text{(fri-zero-round-first-query-index-lists table final)})) \\ \text{adversary-initial-state} & \leq \\ \text{nnreal} & \\ & ((\text{query-sample-space-size} - 1) * \\ & \text{query-sample-space-size} ^ (\text{rounds} - 1)) * \\ & (1 / \text{nnreal (card query-sample-space)}) ^ \text{rounds} + \\ \text{hash-relation-budget-value} & \\ & (\text{query-index-raw-list-relation-fiber-bound} \\ & \text{(fri-zero-round-first-query-index-lists table final)}) \\ & (\text{staged-attacker-query-budget budgets} + \\ & \text{staged-challenge-query-budget}) \end{aligned}$$

proof –

have *exact:*

$$\begin{aligned} & \text{wp-event (checked-staged-security-experiment-with-data-state A)} \\ & (\text{staged-security-with-data-state-verifier-event} \\ & (\lambda s. \\ & \text{trace-fri-query-index-list-set-hit s} \\ & \text{(fri-zero-round-first-query-index-lists table final)})) \\ \text{adversary-initial-state} & \leq \\ \text{nnreal} & \\ & (\text{card (fri-zero-round-first-query-index-lists table final)}) * \\ & (1 / \text{nnreal (card query-sample-space)}) ^ \text{rounds} + \\ \text{hash-relation-budget-value} & \\ & (\text{query-index-raw-list-relation-fiber-bound} \\ & \text{(fri-zero-round-first-query-index-lists table final)}) \end{aligned}$$

```

      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)
    by (rule
        checked-staged-security-trace-fri-zero-round-query-list-target-exact-bound
        [OF wf controlled])
  have card-le:
    card (fri-zero-round-first-query-index-lists table final) ≤
      (query-sample-space-size - 1) *
      query-sample-space-size ^ (rounds - 1)
  by (rule fri-zero-round-first-query-index-lists-card-bound-from-small
      [OF small])
  have product-le:
    nnreal
      (card (fri-zero-round-first-query-index-lists table final)) *
      (1 / nnreal (card query-sample-space)) ^ rounds ≤
    nnreal
      ((query-sample-space-size - 1) *
       query-sample-space-size ^ (rounds - 1)) *
      (1 / nnreal (card query-sample-space)) ^ rounds
  proof -
    have card-nn:
      nnreal (card (fri-zero-round-first-query-index-lists table final)) ≤
      nnreal
        ((query-sample-space-size - 1) *
         query-sample-space-size ^ (rounds - 1))
    using of-nat-mono[OF card-le, where 'a = nnreal] by simp
  show ?thesis
    by (rule mult-right-mono[OF card-nn]) simp
qed
show ?thesis
  by (rule order-trans[OF exact add-mono[OF product-le order-refl]])
qed

end

end

```

theory *Soundness-FRI-Zero-Round-Agreement-Set-Bounds*
imports *Soundness-FRI-Zero-Round-Query-List-Bounds*
begin

Global agreement-set bound for the zero-round trace FRI target.

The verifier-local event *trace-fri-zero-round-query-list-target-hit* is existential in a candidate table and final value. The query-list target depends on these witnesses only through the agreement set inside *query-sample-space*. This layer replaces the arbitrary-list existential by a finite union over agreement sets, preserving the exact query-list product accounting for each fixed set.

context *soundness*
begin

definition *fri-zero-round-first-query-index-lists-for-set*
 $:: \text{nat set} \Rightarrow \text{nat list set}$

where
fri-zero-round-first-query-index-lists-for-set $S =$
 $\{\text{query-idxs} \in \text{fri-query-index-list-space}. \text{query-idxs} ! 0 \in S\}$

definition *fri-zero-round-small-agreement-sets* $:: \text{nat set set}$

where
fri-zero-round-small-agreement-sets $=$
 $\{S. S \subseteq \text{query-sample-space} \wedge \text{card } S < \text{query-sample-space-size}\}$

definition *trace-fri-zero-round-agreement-set-target-error-for*

where
trace-fri-zero-round-agreement-set-target-error-for $\text{budgets} =$
 $(\sum S \in \text{fri-zero-round-small-agreement-sets}.$
 $\text{nnreal } (\text{card } (\text{fri-zero-round-first-query-index-lists-for-set } S)) *$
 $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) ^ \text{rounds} +$
 $\text{hash-relation-budget-value}$
 $(\text{query-index-row-list-relation-fiber-bound}$
 $(\text{fri-zero-round-first-query-index-lists-for-set } S))$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget}))$

lemma *finite-fri-zero-round-small-agreement-sets[simp]:*

finite fri-zero-round-small-agreement-sets

proof —

have *fri-zero-round-small-agreement-sets* $\subseteq \text{Pow } \text{query-sample-space}$

unfolding *fri-zero-round-small-agreement-sets-def* **by** *blast*

then show *?thesis*

by (rule *finite-subset*) *simp*

qed

lemma *fri-zero-round-first-query-index-lists-for-set-subset:*

fri-zero-round-first-query-index-lists-for-set S

$\subseteq \text{fri-query-index-list-space}$

unfolding *fri-zero-round-first-query-index-lists-for-set-def* **by** *blast*

lemma *trace-fri-zero-round-table-target-eq-agreement-set-target:*

fri-zero-round-first-query-index-lists *table* *final* $=$

fri-zero-round-first-query-index-lists-for-set

(*fri-final-value-agreement-set* *table* *final*)

unfolding *fri-zero-round-first-query-index-lists-def*

fri-zero-round-first-query-index-lists-for-set-def

by *simp*

lemma *checked-staged-security-trace-fri-zero-round-agreement-set-target-bound:*

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *S-in*: $S \in \text{fri-zero-round-small-agreement-sets}$
shows
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s.$
trace-fri-query-index-list-set-hit s
(*fri-zero-round-first-query-index-lists-for-set S*)))
adversary-initial-state $\leq$
nnreal (*card* (*fri-zero-round-first-query-index-lists-for-set S*)) *
($1 / \text{nnreal} (\text{card } \text{query-sample-space})$) $^{\text{rounds}}$ +
hash-relation-budget-value
(*query-index-row-list-relation-fiber-bound*
(*fri-zero-round-first-query-index-lists-for-set S*))
(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget)
by (*rule checked-staged-security-trace-fri-query-index-list-set-hit-exact-product-bound*
[*OF wf controlled fri-zero-round-first-query-index-lists-for-set-subset*])

lemma *checked-staged-security-trace-fri-zero-round-query-list-target-bound-from-agreement-sets*:

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
shows
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-zero-round-query-list-target-hit)
adversary-initial-state $\leq$
trace-fri-zero-round-agreement-set-target-error-for budgets

proof –

let *?M* = *checked-staged-security-experiment-with-data-state A*
let *?A* = *fri-zero-round-small-agreement-sets*
let *?P* =
 $\lambda S. \text{staged-security-with-data-state-verifier-event}$
($\lambda s.$
trace-fri-query-index-list-set-hit s
(*fri-zero-round-first-query-index-lists-for-set S*))

have *event-le*:

wp-event ?M
(*staged-security-with-data-state-verifier-event*
trace-fri-zero-round-query-list-target-hit)
adversary-initial-state $\leq$
wp-event ?M ($\lambda \text{out}. \exists S \in ?A. ?P S \text{ out}$)
adversary-initial-state

proof (*rule wp-event-mono-on-support*)

fix *out*

assume

out $\in \text{set-dist} (\text{execute } ?M \text{ adversary-initial-state})$
and *hit*:

```

    staged-security-with-data-state-verifier-event
    trace-fri-zero-round-query-list-target-hit out
show  $\exists S \in ?A. ?P \ S \ out$ 
proof (cases out)
  case None
  then show ?thesis
    using hit unfolding staged-security-with-data-state-verifier-event-def
    trace-fri-zero-round-query-list-target-hit-def
    by simp
next
case (Some out-data)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases out-data, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have local-hit:
  trace-fri-zero-round-query-list-target-hit ?s
  (Some (result, final-state))
  using hit unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
then obtain trace-table trace-final where query-hit:
  trace-fri-query-index-list-set-hit ?s
  (fri-zero-round-first-query-index-lists trace-table trace-final)
  (Some (result, final-state))
  and small:
    card (fri-final-value-agreement-set trace-table trace-final)
    < query-sample-space-size
  by (rule trace-fri-zero-round-query-list-target-hitE)
let ?S = fri-final-value-agreement-set trace-table trace-final
have S-in: ?S  $\in$  ?A
  unfolding fri-zero-round-small-agreement-sets-def
  using fri-final-value-agreement-set-subset small by simp
have query-hit':
  trace-fri-query-index-list-set-hit ?s
  (fri-zero-round-first-query-index-lists-for-set ?S)
  (Some (result, final-state))
  using query-hit
  by (simp add:
    trace-fri-zero-round-table-target-eq-agreement-set-target)
have ?P ?S out
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  using query-hit' by simp
show ?thesis
  using S-in  $\langle ?P \ ?S \ out \rangle$  by blast
qed
qed
have union-bound:

```

```

wp-event ?M (λout. ∃ S ∈ ?A. ?P S out)
adversary-initial-state ≤
(∑ S ∈ ?A.
  nnreal (card (fri-zero-round-first-query-index-lists-for-set S)) *
    (1 / nnreal (card query-sample-space)) ^ rounds +
  hash-relation-budget-value
  (query-index-raw-list-relation-fiber-bound
    (fri-zero-round-first-query-index-lists-for-set S))
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget))
proof (rule wp-event-finite-UN-bound)
  show finite ?A
  by simp
next
  fix S
  assume S-in: S ∈ ?A
  show wp-event ?M (?P S) adversary-initial-state ≤
    nnreal (card (fri-zero-round-first-query-index-lists-for-set S)) *
      (1 / nnreal (card query-sample-space)) ^ rounds +
    hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound
        (fri-zero-round-first-query-index-lists-for-set S))
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
  by (rule checked-staged-security-trace-fri-zero-round-agreement-set-target-bound
    [OF wf controlled S-in])
qed
show ?thesis
  by (rule order-trans[OF event-le])
    (use union-bound in
      ⟨simp add: trace-fri-zero-round-agreement-set-target-error-for-def⟩)
qed

end

end

```

```

theory Soundness-FRI-Trace-Empty-Verifier-Tied
imports
  Soundness-FRI-Trace-Untied-Cases
  Soundness-FRI-Zero-Round-Query-List-Bounds
begin

```

Verifier-tied normalization for the empty-composition-header trace FRI branch.

The broad empty-header branch can carry a non-low candidate built from existential partial openings. This layer separates that branch into the verifier-checked zero-round obstruction, Merkle inconsistency, or the existing candidate-transfer residual. It deliberately avoids complete-table reconstruction from sampled openings.

```

context soundness
begin

lemma trace-zero-round-candidate-low-degree-if-final-consistent:
  assumes rounds-empty:  $\text{length trace-bs} = 0$ 
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  and candidate:
    partial-trace-table-candidate trace-table trace-openings
  and final:
    fri-final-constant-consistent trace-table trace-final
  shows trace-table-low-degree trace-table
proof -
  have len-trace-bs:  $\text{length trace-bs} = \text{ceil-log clength}$ 
    using accepted-fri-opening-transcript-shapes(1,2)[OF fri-openings]
    by simp
  have ceil-zero:  $\text{ceil-log clength} = 0$ 
    using rounds-empty len-trace-bs by simp
  have clength-le-one:  $\text{clength} \leq 1$ 
    using ceil-zero unfolding ceil-log-def
    by (cases  $\text{clength} \leq 1$ ) simp-all
  have clength-eq:  $\text{clength} = 1$ 
    using clength-le-one clength-pos by simp
  have len-table:  $\text{length trace-table} = \text{length eval-domain}$ 
    using candidate eval-domain-length
    unfolding partial-trace-table-candidate-def
    by simp
  have table-eq:
     $\text{trace-table} = \text{map (poly [:trace-final:]) eval-domain}$ 
proof (rule nth-equalityI)
  show  $\text{length trace-table} =$ 
     $\text{length (map (poly [:trace-final:]) eval-domain)}$ 
    using len-table by simp
next
  fix i
  assume i-bound:  $i < \text{length trace-table}$ 
  have trace-table ! i = trace-final
    by (rule fri-final-constant-consistent-nth[OF final i-bound])
  then show trace-table ! i =
     $\text{map (poly [:trace-final:]) eval-domain} ! i$ 
    using i-bound len-table by simp
qed
show ?thesis
  unfolding trace-table-low-degree-def
  by (intro exI[of - [:trace-final:]] conjI)
    (use clength-eq table-eq in simp-all)
qed

```

lemma *trace-fri-zero-round-final-obstruction-imp-checked-merkle-transfer-or-header:*
assumes *support: out ∈ set-dist (execute verify-monad s)*
and *obstruction: trace-fri-zero-round-final-obstruction s out*
shows
trace-fri-zero-round-checked-final-obstruction s out ∨
partial-merkle-inconsistency-bad s out ∨
trace-fri-header-tied-candidate-transfer-gap s out ∨
trace-fri-bad-with-header-tied-partial-candidate s out
proof –
from *trace-fri-zero-round-final-obstructionE*[*OF obstruction*]
obtain *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr0 candidate-query-idxs trace-openings0
trace-table0 **where** *evidence:*
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr0 candidate-query-idxs trace-openings0
trace-table0
and *rounds-empty: length trace-bs = 0*
and *not-final0:*
¬ fri-final-constant-consistent trace-table0 trace-final
by *blast*
have *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
using *trace-fri-partial-candidate-opening-evidenceD(1)*[*OF evidence*] .
from *fri-openings* **obtain** *result final-state* **where** *out-eq:*
out = Some (result, final-state)
unfolding *accepted-fri-opening-transcript-def* **by** *blast*
from *accepted-fri-opening-transcript-headerE*[*OF fri-openings*]
obtain *fr as rest* **where** *header:*
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
by *blast*
show *?thesis*
proof (*cases partial-merkle-inconsistency-bad s out*)
case *True*
then show *?thesis* **by** *simp*
next
case *no-merkle: False*
from *accepted-fri-opening-transcript-trace-openings-for-query-indices*
[*OF fri-openings header out-eq*]
obtain *trace-openings* **where** *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
by *blast*


```

obtain trace-table where candidate:
  partial-trace-table-candidate trace-table trace-openings
using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
  [OF partial[unfolded out-eq] no-merkle[unfolded out-eq]]
by blast
show ?thesis
proof (cases trace-table-low-degree trace-table)
  case True
  then show ?thesis
  proof (cases trace-fri-bad-with-header-tied-partial-candidate s out)
    case True
    then show ?thesis by simp
  next
  case no-header: False
  have reachable:
    trace-fri-bad-with-reachable-partial-candidate s out
    using trace-fri-zero-round-final-obstruction-imp-header-tied-or-gap
    [OF obstruction] no-header
    unfolding trace-fri-reachable-header-tie-gap-def by simp
  have gap: trace-fri-reachable-header-tie-gap s out
    unfolding trace-fri-reachable-header-tie-gap-def
    using reachable no-header by simp
  have transfer:
    trace-fri-header-tied-candidate-transfer-gap s out
    by (rule trace-fri-header-candidate-low-imp-transfer-gap
      [OF gap fri-openings header partial candidate True])
    then show ?thesis by simp
  qed
next
  case not-low: False
  have not-final:
     $\neg$  fri-final-constant-consistent trace-table trace-final
  proof
    assume final:
      fri-final-constant-consistent trace-table trace-final
    have low:
      trace-table-low-degree trace-table
      by (rule
        trace-zero-round-candidate-low-degree-if-final-consistent
        [OF rounds-empty fri-openings candidate final])
    then show False
      using not-low by contradiction
  qed
have header-zero:
  trace-fri-header-tied-zero-round-final-obstruction s out
  unfolding trace-fri-header-tied-zero-round-final-obstruction-def
  by (intro exI conjI)
    (rule fri-openings, rule header, rule partial, rule candidate,
      rule not-low, rule rounds-empty, rule not-final)

```

```

    then show ?thesis
    using
      trace-fri-header-tied-zero-round-final-obstruction-imp-checked-or-merkle
    by blast
  qed
qed
qed

```

lemma *wp-trace-fri-zero-round-checked-final-obstruction-bound-from-query-list-target:*

assumes *target-bound:*

wp-event verify-monad

$(\text{trace-fri-zero-round-query-list-target-hit } s) \ s \leq Q$

shows

wp-event verify-monad

$(\text{trace-fri-zero-round-checked-final-obstruction } s) \ s \leq Q$

proof –

have *wp-event verify-monad*

$(\text{trace-fri-zero-round-checked-final-obstruction } s) \ s \leq$

wp-event verify-monad

$(\text{trace-fri-zero-round-checked-first-query-target-hit } s) \ s$

by (rule *wp-event-mono*)

(rule *trace-fri-zero-round-checked-final-obstruction-imp-first-query-target-hit*)

also have $\dots \leq$

wp-event verify-monad

$(\text{trace-fri-zero-round-query-list-target-hit } s) \ s$

by (rule *wp-event-mono*)

(rule *trace-fri-zero-round-checked-first-query-target-hit-imp-query-list-target*)

also have $\dots \leq Q$

by (rule *target-bound*)

finally show ?thesis .

qed

lemma *wp-trace-fri-zero-round-final-obstruction-bound-from-checked-merkle-transfer-header:*

fixes $H \ M \ Q \ T :: \text{prob}$

assumes *checked-bound:*

wp-event verify-monad

$(\text{trace-fri-zero-round-checked-final-obstruction } s) \ s \leq Q$

and *merkle-bound:*

wp-event verify-monad (*partial-merkle-inconsistency-bad* s) $s \leq M$

and *transfer-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-candidate-transfer-gap } s) \ s \leq T$

and *header-bound:*

wp-event verify-monad

$(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s \leq H$

shows

wp-event verify-monad

$(\text{trace-fri-zero-round-final-obstruction } s) \ s \leq Q + M + T + H$

proof –

have *wp-event verify-monad*
 (*trace-fri-zero-round-final-obstruction s*) *s* ≤
wp-event verify-monad
 (λ*out*.
 trace-fri-zero-round-checked-final-obstruction s out ∨
 partial-merkle-inconsistency-bad s out ∨
 trace-fri-header-tied-candidate-transfer-gap s out ∨
 trace-fri-bad-with-header-tied-partial-candidate s out) *s*
by (rule *wp-event-mono-on-support*)
 (rule *trace-fri-zero-round-final-obstruction-imp-checked-merkle-transfer-or-header*)
also have ... ≤
wp-event verify-monad
 (*trace-fri-zero-round-checked-final-obstruction s*) *s* +
wp-event verify-monad (*partial-merkle-inconsistency-bad s*) *s* +
wp-event verify-monad
 (*trace-fri-header-tied-candidate-transfer-gap s*) *s* +
wp-event verify-monad
 (*trace-fri-bad-with-header-tied-partial-candidate s*) *s*
by (rule *wp-event-union-bound4*)
also have ... ≤ *Q* + *M* + *T* + *H*
by (intro *add-mono checked-bound merkle-bound transfer-bound*
 header-bound)
finally show ?thesis .
qed

lemma *trace-fri-header-tied-candidate-transfer-gap-imp-header-or-untied*:
assumes *transfer*: *trace-fri-header-tied-candidate-transfer-gap s out*
shows
 trace-fri-bad-with-header-tied-partial-candidate s out ∨
 trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
proof (cases *trace-fri-bad-with-header-tied-partial-candidate s out*)
case *True*
then show ?thesis **by** *simp*
next
case *no-header*: *False*
from *transfer* **obtain** *trace-roots trace-bs trace-final fri-dg*
 composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers fr as rest trace-openings
 trace-table **where**
raw: *trace-fri-bad-with-reachable-partial-candidate s out*
and *fri-openings*:
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
and *header*:
 verifier-header-transcript s fr trace-roots trace-final as fri-dg
 composition-roots composition-final rest
and *partial*:

```

    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  and candidate:
    partial-trace-table-candidate trace-table trace-openings
  and low: trace-table-low-degree trace-table
  unfolding trace-fri-header-tied-candidate-transfer-gap-def by blast
have gap: trace-fri-reachable-header-tie-gap s out
  unfolding trace-fri-reachable-header-tie-gap-def
  using raw no-header by simp
have low-gap: trace-fri-header-tied-low-candidate-gap s out
  unfolding trace-fri-header-tied-low-candidate-gap-def
  by (intro exI conjI)
  (rule gap, rule fri-openings, rule header, rule partial, rule candidate,
   rule low)
have untied:
  trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
  by (rule trace-fri-header-tied-low-candidate-gap-imp-untied-reachable-witness
    [OF low-gap])
then show ?thesis by simp
qed

```

lemma *wp-trace-fri-header-tied-candidate-transfer-gap-bound-from-header-and-untied:*

```

  fixes H U :: prob
  assumes header-bound:
    wp-event verify-monad
      (trace-fri-bad-with-header-tied-partial-candidate s) s ≤ H
  and untied-bound:
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
      s ≤ U

```

shows

```

  wp-event verify-monad
    (trace-fri-header-tied-candidate-transfer-gap s) s ≤ H + U

```

proof –

```

  have wp-event verify-monad
    (trace-fri-header-tied-candidate-transfer-gap s) s ≤
  wp-event verify-monad
    (λout.
      trace-fri-bad-with-header-tied-partial-candidate s out ∨
      trace-fri-header-tied-low-candidate-untied-reachable-witness-gap
        s out) s
  by (rule wp-event-mono)
  (rule trace-fri-header-tied-candidate-transfer-gap-imp-header-or-untied)
  also have ... ≤
  wp-event verify-monad
    (trace-fri-bad-with-header-tied-partial-candidate s) s +
  wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
    s

```

```

    by (rule wp-event-union-bound)
  also have ...  $\leq H + U$ 
    by (rule add-mono[OF header-bound untied-bound])
  finally show ?thesis .
qed

end

end

```

theory *Soundness-Public-Route-Concrete-FRI-Sharpened*

imports

Soundness-Public-Route-Concrete-FRI-Endpoint
Soundness-FRI-Query-Fiber-Staged-Bounds
Soundness-FRI-Prequery-Challenge-Bounds
Soundness-FRI-Sampled-Challenge-Cover
Soundness-FRI-Staged-Replay-Bounds
Soundness-FRI-Zero-Round-Agreement-Set-Bounds
Soundness-FRI-Trace-Empty-Verifier-Tied

begin

First-stage API for sharpening the concrete FRI endpoint.

The public endpoint already removed the FRI soundness assumptions by using explicit staged verifier-event error definitions. This theory factors those definitions into named challenge, full-cover/domain, conflict, and zero-round components. At this stage the components are definitionally equal to the endpoint terms; later proof layers can replace individual components by sharper product/fiber or residual sums without touching the public theorem interface.

context *soundness*

begin

definition *trace-fri-challenge-prequery-error-for*

where

trace-fri-challenge-prequery-error-for budgets $A =$
 (*wp-event* (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s.$
 trace-fri-challenge-list-fresh-hit s
 (*fri-challenge-space* (*ceil-log* *clength*))))
adversary-initial-state +
hash-relation-budget-value
 (*card* (*fri-challenge-space* (*ceil-log* *clength*))) * *ceil-log* *clength*)
 (*staged-attacker-query-budget* *budgets* +
 staged-challenge-query-budget))

definition *trace-fri-full-cover-domain-residual-error-for*

where

$\text{trace-fri-full-cover-domain-residual-error-for } A =$
 $(\text{reachable-verifier-event-bound-for } A$
 $(\lambda s.$
 $\quad \text{trace-fri-sampled-full-cover-proximity-failure } s$
 $\quad \text{trace-fri-full-challenge-bad}) +$
 $\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-sampled-full-cover-index-failure} +$
 $(\text{reachable-verifier-event-bound-for } A$
 $\quad \text{trace-fri-sampled-full-cover-index-failure} +$
 $\text{reachable-verifier-event-bound-for } A$
 $\quad \text{trace-fri-sampled-full-cover-domain-length-failure}) +$
 $(\text{reachable-verifier-event-bound-for } A$
 $\quad \text{trace-fri-sampled-full-cover-index-failure} +$
 $\text{reachable-verifier-event-bound-for } A$
 $\quad \text{trace-fri-sampled-full-cover-sampled-domain-failure}))$

definition *sharpened-trace-fri-full-cover-domain-error-for*
where

$\text{sharpened-trace-fri-full-cover-domain-error-for budgets } A =$
 $\text{trace-fri-challenge-prequery-error-for budgets } A +$
 $\text{trace-fri-full-cover-domain-residual-error-for } A$

definition *trace-fri-header-conflict-zero-error-for*
where

$\text{trace-fri-header-conflict-zero-error-for } A =$
 $(\text{reachable-verifier-event-bound-for } A$
 $\quad \text{trace-fri-header-tied-sampled-assignment-conflict} +$
 $\text{reachable-verifier-event-bound-for } A$
 $\quad \text{trace-fri-header-tied-zero-round-final-obstruction})$

definition *trace-fri-header-replay-conflict-error-for*
where

$\text{trace-fri-header-replay-conflict-error-for } A =$
 $(\text{let } M = \text{reachable-verifier-event-bound-for } A$
 $\quad \text{partial-merkle-inconsistency-bad};$
 $\quad H = \text{reachable-verifier-event-bound-for } A$
 $\quad \text{trace-fri-header-tied-recorded-sibling-candidate-missing}$
 $\text{in } H + (((M + M) + (M + 0) + (M + 0) + (M + 0 + 0)) + M) + 0))$

definition *trace-fri-controlled-merkle-error-for*
where

$\text{trace-fri-controlled-merkle-error-for budgets} =$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\quad \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget})$

definition *trace-fri-header-replay-conflict-zero-error-for*
where

$\text{trace-fri-header-replay-conflict-zero-error-for budgets } A =$

trace-fri-header-replay-conflict-error-for A +
(trace-fri-zero-round-agreement-set-target-error-for budgets +
trace-fri-controlled-merkle-error-for budgets)

definition *sharpened-trace-fri-header-error-for*

where

sharpened-trace-fri-header-error-for budgets A =
sharpened-trace-fri-full-cover-domain-error-for budgets A +
trace-fri-header-conflict-zero-error-for A

definition *trace-fri-empty-conflict-zero-error-for*

where

trace-fri-empty-conflict-zero-error-for A =
(reachable-verifier-event-bound-for A
trace-fri-sampled-assignment-conflict +
reachable-verifier-event-bound-for A
trace-fri-zero-round-final-obstruction)

definition *trace-fri-empty-small-conflict-zero-error-for*

where

trace-fri-empty-small-conflict-zero-error-for budgets A =
(reachable-verifier-event-bound-for A
trace-fri-sampled-assignment-conflict +
trace-fri-zero-round-agreement-set-target-error-for budgets +
trace-fri-controlled-merkle-error-for budgets +
reachable-verifier-event-bound-for A
trace-fri-bad-with-header-tied-partial-candidate +
(trace-fri-controlled-merkle-error-for budgets +
(reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-untied-root-gap +
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-untied-query-gap)) +
reachable-verifier-event-bound-for A
trace-fri-bad-with-header-tied-partial-candidate)

definition *sharpened-trace-fri-empty-error-for*

where

sharpened-trace-fri-empty-error-for budgets A =
sharpened-trace-fri-full-cover-domain-error-for budgets A +
trace-fri-empty-conflict-zero-error-for A

definition *composition-fri-challenge-prequery-error-for*

where

composition-fri-challenge-prequery-error-for budgets A =
(wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
(λs.
composition-fri-challenge-list-fresh-hit s
(λdg. fri-challenge-space (ceil-log (to-nat dg + 1))))))

adversary-initial-state +
hash-relation-budget-value
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (fri\text{-challenge-space } (ceil\text{-log } (to\text{-nat } dg + 1))) *$
 $ceil\text{-log } (maxDegree + 1))$
 $(staged\text{-attacker-query-budget } budgets +$
 $staged\text{-challenge-query-budget}))$

definition *composition-fri-full-cover-domain-residual-error-for*
where

composition-fri-full-cover-domain-residual-error-for $A =$
 $(reachable\text{-verifier-event-bound-for } A$
 $(\lambda s.$
 $\text{composition-fri-sampled-full-cover-proximity-failure } s$
 $\text{composition-fri-full-challenge-bad}) +$
 $reachable\text{-verifier-event-bound-for } A$
 $\text{composition-fri-sampled-full-cover-index-failure} +$
 $(reachable\text{-verifier-event-bound-for } A$
 $\text{composition-fri-sampled-full-cover-index-failure} +$
 $reachable\text{-verifier-event-bound-for } A$
 $\text{composition-fri-sampled-full-cover-domain-length-failure}) +$
 $(reachable\text{-verifier-event-bound-for } A$
 $\text{composition-fri-sampled-full-cover-index-failure} +$
 $reachable\text{-verifier-event-bound-for } A$
 $\text{composition-fri-sampled-full-cover-sampled-domain-failure}))$

definition *sharpened-composition-fri-full-cover-domain-error-for*
where

sharpened-composition-fri-full-cover-domain-error-for $budgets\ A =$
 $\text{composition-fri-challenge-prequery-error-for } budgets\ A +$
 $\text{composition-fri-full-cover-domain-residual-error-for } A$

definition *composition-fri-conflict-zero-error-for*
where

composition-fri-conflict-zero-error-for $A =$
 $(reachable\text{-verifier-event-bound-for } A$
 $\text{composition-fri-verifier-tied-sampled-assignment-conflict} +$
 $reachable\text{-verifier-event-bound-for } A$
 $\text{composition-fri-verifier-tied-zero-round-final-obstruction})$

definition *sharpened-composition-fri-error-for*
where

sharpened-composition-fri-error-for $budgets\ A =$
 $\text{sharpened-composition-fri-full-cover-domain-error-for } budgets\ A +$
 $\text{composition-fri-conflict-zero-error-for } A$

definition *composition-one-step-query-fraction-for*

$:: ('f \Rightarrow nat \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow prob$

where

composition-one-step-query-fraction-for comp-bad =
nnreal
 $((\sum dg \in (UNIV :: 'f \text{ set}).$
 $\sum challenges \in$
 $fri\text{-}challenge\text{-}space (fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound (to\text{-}nat dg)) -$
 $generic\text{-}fri\text{-}bad\text{-}challenge\text{-}lists$
 $(fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound (to\text{-}nat dg))$
 $(comp\text{-}bad dg).$
card
 $(generic\text{-}fri\text{-}sampled\text{-}query\text{-}query\text{-}fiber$
 $(composition\text{-}table\text{-}low\text{-}degree (to\text{-}nat dg))$
 $(Not \circ composition\text{-}table\text{-}low\text{-}degree maxDegree)$
 $(to\text{-}nat dg) challenges)) * rounds) /$
nnreal (*card query-sample-space*)

definition *composition-one-step-challenge-hash-count* :: *nat*
where

composition-one-step-challenge-hash-count =
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $fri\text{-}one\text{-}step\text{-}disagreement\text{-}round\text{-}mass$
 $(fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound (to\text{-}nat dg))$
 $(composition\text{-}fri\text{-}canonical\text{-}layer\text{-}len dg) *$
 $ceil\text{-}log (maxDegree + 1))$

definition *composition-one-step-sampled-error-for*
where

composition-one-step-sampled-error-for budgets comp-bad =
 $concrete\text{-}composition\text{-}fri\text{-}bad\text{-}challenge\text{-}fraction +$
 $hash\text{-}relation\text{-}budget\text{-}value$
 $composition\text{-}one\text{-}step\text{-}challenge\text{-}hash\text{-}count$
 $(staged\text{-}attacker\text{-}query\text{-}budget budgets +$
 $staged\text{-}challenge\text{-}query\text{-}budget) +$
 $nnreal (staged\text{-}attacker\text{-}query\text{-}budget budgets +$
 $staged\text{-}challenge\text{-}query\text{-}budget) *$
 $composition\text{-}one\text{-}step\text{-}query\text{-}fraction\text{-}for comp\text{-}bad$

definition *composition-one-step-reachable-merkle-error-for*
where

composition-one-step-reachable-merkle-error-for A =
 $reachable\text{-}verifier\text{-}event\text{-}bound\text{-}for A partial\text{-}merkle\text{-}inconsistency\text{-}bad$

definition *composition-one-step-controlled-merkle-error-for*
where

composition-one-step-controlled-merkle-error-for budgets =
 $hash\text{-}collision\text{-}budget\text{-}value 0$
 $(staged\text{-}attacker\text{-}query\text{-}budget budgets +$
 $staged\text{-}challenge\text{-}query\text{-}budget + verifier\text{-}hash\text{-}query\text{-}budget)$

definition *composition-one-step-verifier-tied-error-for*

where

composition-one-step-verifier-tied-error-for budgets A comp-bad =
composition-one-step-sampled-error-for budgets comp-bad +
 (((*composition-one-step-reachable-merkle-error-for A + 0*) +
 (*composition-one-step-reachable-merkle-error-for A + 0*) +
 (*composition-one-step-reachable-merkle-error-for A + 0*) +
 (*composition-one-step-reachable-merkle-error-for A + 0 + 0*) +
composition-one-step-reachable-merkle-error-for A) + 0 + 0)

definition *composition-one-step-verifier-tied-controlled-error-for*

where

composition-one-step-verifier-tied-controlled-error-for budgets comp-bad =
composition-one-step-sampled-error-for budgets comp-bad +
 (((*composition-one-step-controlled-merkle-error-for budgets + 0*) +
 (*composition-one-step-controlled-merkle-error-for budgets + 0*) +
 (*composition-one-step-controlled-merkle-error-for budgets + 0*) +
 (*composition-one-step-controlled-merkle-error-for budgets + 0 + 0*) +
composition-one-step-controlled-merkle-error-for budgets) + 0 + 0)

definition *trace-one-step-query-fraction-for*

:: (nat ⇒ 'f list ⇒ 'f set) ⇒ prob

where

trace-one-step-query-fraction-for empty-bad =
 nnreal
 ((∑ challenges ∈
 fri-challenge-space (fri-round-count-for-degree-bound (clength - 1)) -
 generic-fri-bad-challenge-lists
 (*fri-round-count-for-degree-bound (clength - 1)) empty-bad.*
 card
 (*generic-fri-sampled-query-query-fiber trace-table-low-degree*
 (*Not* ∘ *trace-table-low-degree*) (clength - 1) challenges)) *
 rounds) /
 nnreal (card query-sample-space)

definition *trace-one-step-challenge-hash-count* :: nat

where

trace-one-step-challenge-hash-count =
fri-one-step-disagreement-round-mass
 (*fri-round-count-for-degree-bound (clength - 1)*)
trace-fri-canonical-layer-len *
ceil-log clength

definition *trace-one-step-disagreement-bad-for*

:: (nat ⇒ nat) ⇒
 (nat ⇒ 'f list ⇒ 'f list) ⇒
 (nat ⇒ 'f list ⇒ 'f list) ⇒
 (nat ⇒ 'f list ⇒ 'f list) ⇒
 nat ⇒ 'f list ⇒ 'f set

where

trace-one-step-disagreement-bad-for pw layer claimed doms i prefix =
fri-one-step-disagreement-challenges
 (trace-fri-canonical-layer-len i) (pw i)
 (layer i prefix) (claimed i prefix) (doms i prefix)

definition *composition-one-step-disagreement-bad-for*

:: ('f ⇒ nat ⇒ nat) ⇒
 ('f ⇒ nat ⇒ 'f list ⇒ 'f list) ⇒
 ('f ⇒ nat ⇒ 'f list ⇒ 'f list) ⇒
 ('f ⇒ nat ⇒ 'f list ⇒ 'f list) ⇒
 'f ⇒ nat ⇒ 'f list ⇒ 'f set

where

composition-one-step-disagreement-bad-for pw layer claimed doms dg i prefix =
fri-one-step-disagreement-challenges
 (composition-fri-canonical-layer-len dg i) (pw dg i)
 (layer dg i prefix) (claimed dg i prefix) (doms dg i prefix)

definition *trace-one-step-sampled-error-for*

where

trace-one-step-sampled-error-for budgets empty-bad =
concrete-trace-fri-bad-challenge-fraction +
hash-relation-budget-value
trace-one-step-challenge-hash-count
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget) +
 nnreal (staged-attacker-query-budget budgets +
 staged-challenge-query-budget) *
trace-one-step-query-fraction-for empty-bad

definition *trace-empty-one-step-error-for*

where

trace-empty-one-step-error-for budgets A empty-bad =
trace-one-step-sampled-error-for budgets empty-bad +
trace-fri-empty-small-conflict-zero-error-for budgets A

definition *trace-header-one-step-error-for*

where

trace-header-one-step-error-for budgets A trace-bad =
trace-one-step-sampled-error-for budgets trace-bad +
trace-fri-header-replay-conflict-zero-error-for budgets A

definition *trace-one-step-split-pw* :: nat ⇒ nat

where

trace-one-step-split-pw - = 0

definition *trace-one-step-split-layer*

:: nat ⇒ 'f list ⇒ 'f list

where

trace-one-step-split-layer i - =

replicate (trace-fri-canonical-layer-len i) 0

definition *trace-one-step-split-claimed*

:: nat $\Rightarrow$ 'f list $\Rightarrow$ 'f list

where

*trace-one-step-split-claimed i - =
replicate (trace-fri-canonical-layer-len i) 1*

definition *trace-one-step-split-domain*

:: nat $\Rightarrow$ 'f list $\Rightarrow$ 'f list

where

*trace-one-step-split-domain i - =
replicate (trace-fri-canonical-layer-len i) 1*

definition *trace-one-step-split-bad*

:: nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set

where

*trace-one-step-split-bad =
trace-one-step-disagreement-bad-for
trace-one-step-split-pw trace-one-step-split-layer
trace-one-step-split-claimed trace-one-step-split-domain*

definition *composition-one-step-split-pw*

:: 'f $\Rightarrow$ nat $\Rightarrow$ nat

where

composition-one-step-split-pw - - = 0

definition *composition-one-step-split-layer*

:: 'f $\Rightarrow$ nat $\Rightarrow$ 'f list $\Rightarrow$ 'f list

where

*composition-one-step-split-layer dg i - =
replicate (composition-fri-canonical-layer-len dg i) 0*

definition *composition-one-step-split-claimed*

:: 'f $\Rightarrow$ nat $\Rightarrow$ 'f list $\Rightarrow$ 'f list

where

*composition-one-step-split-claimed dg i - =
replicate (composition-fri-canonical-layer-len dg i) 1*

definition *composition-one-step-split-domain*

:: 'f $\Rightarrow$ nat $\Rightarrow$ 'f list $\Rightarrow$ 'f list

where

*composition-one-step-split-domain dg i - =
replicate (composition-fri-canonical-layer-len dg i) 1*

definition *composition-one-step-split-bad*

:: 'f $\Rightarrow$ nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set

where

composition-one-step-split-bad =

composition-one-step-disagreement-bad-for
composition-one-step-split-pw composition-one-step-split-layer
composition-one-step-split-claimed composition-one-step-split-domain

definition *concrete-trace-header-one-step-error-for*

where

concrete-trace-header-one-step-error-for budgets $A =$
trace-header-one-step-error-for budgets A
trace-one-step-split-bad

definition *concrete-composition-one-step-error-for*

where

concrete-composition-one-step-error-for budgets $A =$
composition-one-step-verifier-tied-controlled-error-for budgets
composition-one-step-split-bad

definition *concrete-trace-empty-one-step-error-for*

where

concrete-trace-empty-one-step-error-for budgets $A =$
trace-empty-one-step-error-for budgets A
trace-one-step-split-bad

lemma *sharpened-trace-fri-full-cover-domain-error-for-eq:*

sharpened-trace-fri-full-cover-domain-error-for budgets $A =$
concrete-trace-fri-full-cover-domain-error-for budgets A

by (*simp add:*

sharpened-trace-fri-full-cover-domain-error-for-def
trace-fri-challenge-prequery-error-for-def
trace-fri-full-cover-domain-residual-error-for-def
concrete-trace-fri-full-cover-domain-error-for-def)

lemma *sharpened-trace-fri-header-error-for-eq:*

sharpened-trace-fri-header-error-for budgets $A =$
concrete-trace-fri-header-error-for' budgets A

by (*simp add:*

sharpened-trace-fri-header-error-for-def
sharpened-trace-fri-full-cover-domain-error-for-eq
trace-fri-header-conflict-zero-error-for-def
concrete-trace-fri-header-error-for'-def)

lemma *sharpened-trace-fri-empty-error-for-eq:*

sharpened-trace-fri-empty-error-for budgets $A =$
concrete-trace-fri-empty-error-for' budgets A

by (*simp add:*

sharpened-trace-fri-empty-error-for-def
sharpened-trace-fri-full-cover-domain-error-for-eq
trace-fri-empty-conflict-zero-error-for-def
concrete-trace-fri-empty-error-for'-def)

lemma *sharpened-composition-fri-full-cover-domain-error-for-eq*:
sharpened-composition-fri-full-cover-domain-error-for budgets $A =$
concrete-composition-fri-full-cover-domain-error-for budgets A
by (*simp add*:
sharpened-composition-fri-full-cover-domain-error-for-def
composition-fri-challenge-prequery-error-for-def
composition-fri-full-cover-domain-residual-error-for-def
concrete-composition-fri-full-cover-domain-error-for-def)

lemma *sharpened-composition-fri-error-for-eq*:
sharpened-composition-fri-error-for budgets $A =$
concrete-composition-fri-error-for' budgets A
by (*simp add*:
sharpened-composition-fri-error-for-def
sharpened-composition-fri-full-cover-domain-error-for-eq
composition-fri-conflict-zero-error-for-def
concrete-composition-fri-error-for'-def)

lemma *sharpened-trace-fri-header-error-for-bound*:
assumes *wf*: *staged-budget-wellformed* budgets
and *controlled*: *staged-adversary-controlled* budgets A
shows
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state $\leq$ *sharpened-trace-fri-header-error-for* budgets A
using *concrete-trace-fri-header-error-for'-bound*[*OF wf controlled*]
by (*simp add*: *sharpened-trace-fri-header-error-for-eq*)

lemma *sharpened-trace-fri-empty-error-for-bound*:
assumes *wf*: *staged-budget-wellformed* budgets
and *controlled*: *staged-adversary-controlled* budgets A
shows
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *sharpened-trace-fri-empty-error-for* budgets A
using *concrete-trace-fri-empty-error-for'-bound*[*OF wf controlled*]
by (*simp add*: *sharpened-trace-fri-empty-error-for-eq*)

lemma *sharpened-composition-fri-error-for-bound*:
assumes *wf*: *staged-budget-wellformed* budgets
and *controlled*: *staged-adversary-controlled* budgets A
shows
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state $\leq$ *sharpened-composition-fri-error-for* budgets A
using *concrete-composition-fri-error-for'-bound*[*OF wf controlled*]

by (simp add: sharpened-composition-fri-error-for-eq)

lemma composition-one-step-sampled-error-for-bound:

fixes comp-bad :: 'f $\Rightarrow$ nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and comp-subset:

$\bigwedge dg\ i\ prefix.$

$i < \text{fri-round-count-for-degree-bound } (to\text{-nat } dg) \implies$

$\text{length } prefix = i \implies$

$\text{comp-bad } dg\ i\ prefix \subseteq$

$\text{fri-one-step-disagreement-challenges}$

$(\text{composition-fri-canonical-layer-len } dg\ i)\ (\text{comp-pw } dg\ i)$

$(\text{comp-layer } dg\ i\ prefix)\ (\text{comp-claimed } dg\ i\ prefix)$

$(\text{comp-dom } dg\ i\ prefix)$

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

$(\text{staged-security-with-data-state-verifier-event}$

$\text{composition-fri-sampled-query-bad-candidate})$

$\text{adversary-initial-state}$

$\leq \text{composition-one-step-sampled-error-for budgets comp-bad}$

proof –

let ?B =

$\lambda dg. \text{generic-fri-bad-challenge-lists}$

$(\text{fri-round-count-for-degree-bound } (to\text{-nat } dg))\ (\text{comp-bad } dg)$

let ?Q = composition-one-step-query-fraction-for comp-bad

have round-bound:

$\bigwedge dg\ i\ prefix.$

$i < \text{fri-round-count-for-degree-bound } (to\text{-nat } dg) \implies$

$\text{length } prefix = i \implies$

$\text{card } (\text{comp-bad } dg\ i\ prefix)$

$\leq \text{composition-fri-canonical-layer-len } dg\ i\ \text{div } 2$

proof –

fix dg i **and** prefix :: 'f list

assume i: $i < \text{fri-round-count-for-degree-bound } (to\text{-nat } dg)$

and prefix-len: $\text{length } prefix = i$

have card (comp-bad dg i prefix) $\leq$

card

$(\text{fri-one-step-disagreement-challenges}$

$(\text{composition-fri-canonical-layer-len } dg\ i)\ (\text{comp-pw } dg\ i)$

$(\text{comp-layer } dg\ i\ prefix)\ (\text{comp-claimed } dg\ i\ prefix)$

$(\text{comp-dom } dg\ i\ prefix))$

by (rule card-mono) (simp-all add: comp-subset[OF i prefix-len])

also have ... $\leq \text{composition-fri-canonical-layer-len } dg\ i\ \text{div } 2$

by (rule fri-one-step-disagreement-challenges-card-bound-local)

finally show card (comp-bad dg i prefix)

$\leq \text{composition-fri-canonical-layer-len } dg\ i\ \text{div } 2$.

qed

have fraction-bound:

```

 $\wedge dg. \text{nnreal}$ 
 $(\sum i \in \{.. < \text{fri-round-count-for-degree-bound } (to\text{-nat } dg)\}.$ 
 $\text{CARD}(f) \wedge^i *$ 
 $(\text{composition-fri-canonical-layer-len } dg \ i \ \text{div } 2) *$ 
 $\text{CARD}(f) \wedge$ 
 $(\text{fri-round-count-for-degree-bound } (to\text{-nat } dg) - \text{Suc } i)) /$ 
 $\text{nnreal } (\text{CARD}(f) \wedge \text{fri-round-count-for-degree-bound } (to\text{-nat } dg))$ 
 $\leq \text{concrete-composition-fri-bad-challenge-fraction}$ 
proof –
fix  $dg$ 
show  $\text{nnreal}$ 
 $(\sum i \in \{.. < \text{fri-round-count-for-degree-bound } (to\text{-nat } dg)\}.$ 
 $\text{CARD}(f) \wedge^i *$ 
 $(\text{composition-fri-canonical-layer-len } dg \ i \ \text{div } 2) *$ 
 $\text{CARD}(f) \wedge$ 
 $(\text{fri-round-count-for-degree-bound } (to\text{-nat } dg) - \text{Suc } i)) /$ 
 $\text{nnreal } (\text{CARD}(f) \wedge \text{fri-round-count-for-degree-bound } (to\text{-nat } dg))$ 
 $\leq \text{concrete-composition-fri-bad-challenge-fraction}$ 
using  $\text{concrete-composition-fri-bad-challenge-fraction-ge}[of \ dg]$ 
by ( $\text{simp add: fri-one-step-disagreement-round-fraction-def}$ 
 $\text{fri-bad-challenge-round-fraction-def}$ 
 $\text{fri-one-step-disagreement-round-mass-def}$ 
 $\text{fri-bad-challenge-round-mass-def}$ )
qed
have  $\text{fresh-bound:}$ 
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$ 
 $(\text{staged-security-with-data-state-verifier-event}$ 
 $(\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ ?B))$ 
 $\text{adversary-initial-state}$ 
 $\leq \text{concrete-composition-fri-bad-challenge-fraction}$ 
by ( $\text{rule}$ 
 $\text{checked-staged-security-composition-fri-bad-challenge-list-fresh-hit-bound-from-round-bounds}$ 
 $[OF \ \text{round-bound fraction-bound}]$ )
have  $\text{fiber-fraction:}$ 
 $\text{nnreal}$ 
 $((\sum dg \in \text{UNIV}.$ 
 $\sum \text{challenges} \in$ 
 $\text{fri-challenge-space } (\text{fri-round-count-for-degree-bound } (to\text{-nat } dg)) -$ 
 $?B \ dg.$ 
 $\text{card}$ 
 $(\text{generic-fri-sampled-query-query-fiber}$ 
 $(\text{composition-table-low-degree } (to\text{-nat } dg))$ 
 $(\text{Not } \circ \text{composition-table-low-degree } \text{maxDegree})$ 
 $(to\text{-nat } dg) \ \text{challenges})) *$ 
 $\text{rounds}) /$ 
 $\text{nnreal } (\text{card query-sample-space}) \leq ?Q$ 
by ( $\text{simp add: composition-one-step-query-fraction-for-def}$ )
have  $\text{sampled:}$ 
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$ 

```



```

    (staged-security-with-data-state-verifier-event
     composition-fri-sampled-query-bad-candidate)
  adversary-initial-state
≤
concrete-composition-fri-bad-challenge-fraction +
hash-relation-budget-value
(∑ dg ∈ UNIV. card (?B dg) * ceil-log (maxDegree + 1))
(staged-attacker-query-budget budgets +
 staged-challenge-query-budget) +
nnreal
(staged-attacker-query-budget budgets +
 staged-challenge-query-budget) * ?Q
by (rule
  checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-ou
  [OF wf controlled finite-generic-fri-bad-challenge-lists
   fresh-bound fiber-fraction])
have card-le:
  (∑ dg ∈ UNIV. card (?B dg) * ceil-log (maxDegree + 1))
  ≤ composition-one-step-challenge-hash-count
proof (unfold composition-one-step-challenge-hash-count-def, rule sum-mono)
fix dg :: 'f
have raw:
  card (?B dg) ≤
  (∑ i ∈ {..<fri-round-count-for-degree-bound (to-nat dg)} .
   CARD('f) ^ i *
   (composition-fri-canonical-layer-len dg i div 2) *
   CARD('f) ^
   (fri-round-count-for-degree-bound (to-nat dg) - Suc i))
proof (rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds)
fix i :: nat and prefix :: 'f list
assume i: i < fri-round-count-for-degree-bound (to-nat dg)
  and prefix-len: length prefix = i
show card (comp-bad dg i prefix)
  ≤ composition-fri-canonical-layer-len dg i div 2
  by (rule round-bound[OF i prefix-len])
qed
have card (?B dg) ≤
  fri-one-step-disagreement-round-mass
  (fri-round-count-for-degree-bound (to-nat dg))
  (composition-fri-canonical-layer-len dg)
using raw
by (simp add: fri-one-step-disagreement-round-mass-def
  fri-bad-challenge-round-mass-def)
then show card (?B dg) * ceil-log (maxDegree + 1)
  ≤ fri-one-step-disagreement-round-mass
  (fri-round-count-for-degree-bound (to-nat dg))
  (composition-fri-canonical-layer-len dg) *
  ceil-log (maxDegree + 1)
  by (rule mult-right-mono) simp

```

```

qed
have budget-le:
  hash-relation-budget-value
    ( $\sum_{dg \in UNIV}. \text{card } (?B \ dg) * \text{ceil-log } (\text{maxDegree} + 1)$ )
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)
  ≤
  hash-relation-budget-value
    composition-one-step-challenge-hash-count
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)
  by (rule hash-relation-budget-value-mono-left[OF card-le])
show ?thesis
  by (rule order-trans[OF sampled])
    (use budget-le in
     ‹simp add: composition-one-step-sampled-error-for-def›)
qed

lemma trace-one-step-sampled-error-for-bound:
  fixes empty-bad :: nat  $\Rightarrow$  'f list  $\Rightarrow$  'f set
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and empty-subset:
     $\bigwedge i \text{ prefix}. i < \text{fri-round-count-for-degree-bound } (\text{clength} - 1) \Rightarrow$ 
    length prefix = i  $\Rightarrow$ 
    empty-bad i prefix  $\subseteq$ 
    fri-one-step-disagreement-challenges
      (trace-fri-canonical-layer-len i) (empty-pw i)
      (empty-layer i prefix) (empty-claimed i prefix)
      (empty-dom i prefix)
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-sampled-query-bad-candidate)
      adversary-initial-state
    ≤ trace-one-step-sampled-error-for budgets empty-bad
proof -
  let ?B =
    generic-fri-bad-challenge-lists
    (fri-round-count-for-degree-bound (clength - 1)) empty-bad
  let ?Q = trace-one-step-query-fraction-for empty-bad
  have round-bound:
     $\bigwedge i \text{ prefix}. i < \text{fri-round-count-for-degree-bound } (\text{clength} - 1) \Rightarrow$ 
    length prefix = i  $\Rightarrow$ 
    card (empty-bad i prefix) ≤ trace-fri-canonical-layer-len i div 2
proof -
  fix i :: nat and prefix :: 'f list

```

```

assume  $i: i < \text{fri-round-count-for-degree-bound } (\text{clength} - 1)$ 
and  $\text{prefix-len: length prefix} = i$ 
have  $\text{card } (\text{empty-bad } i \text{ prefix}) \leq$ 
   $\text{card}$ 
   $(\text{fri-one-step-disagreement-challenges}$ 
     $(\text{trace-fri-canonical-layer-len } i) (\text{empty-pw } i)$ 
     $(\text{empty-layer } i \text{ prefix}) (\text{empty-claimed } i \text{ prefix})$ 
     $(\text{empty-dom } i \text{ prefix}))$ 
  by  $(\text{rule card-mono}) (\text{simp-all add: empty-subset}[OF i \text{ prefix-len}])$ 
also have  $\dots \leq \text{trace-fri-canonical-layer-len } i \text{ div } 2$ 
  by  $(\text{rule fri-one-step-disagreement-challenges-card-bound-local})$ 
finally show  $\text{card } (\text{empty-bad } i \text{ prefix})$ 
   $\leq \text{trace-fri-canonical-layer-len } i \text{ div } 2 .$ 
qed
have  $\text{fresh-bound:}$ 
   $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$ 
   $(\text{staged-security-with-data-state-verifier-event}$ 
     $(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \text{ ?}B))$ 
   $\text{adversary-initial-state}$ 
   $\leq \text{concrete-trace-fri-bad-challenge-fraction}$ 
proof  $-$ 
have  $\text{base:}$ 
   $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$ 
   $(\text{staged-security-with-data-state-verifier-event}$ 
     $(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \text{ ?}B))$ 
   $\text{adversary-initial-state}$ 
   $\leq \text{nnreal}$ 
   $(\sum_{i \in \{.. < \text{fri-round-count-for-degree-bound } (\text{clength} - 1)\}}.$ 
     $\text{CARD('f)} \wedge i * (\text{trace-fri-canonical-layer-len } i \text{ div } 2) *$ 
     $\text{CARD('f)} \wedge$ 
     $(\text{fri-round-count-for-degree-bound } (\text{clength} - 1) - \text{Suc } i)) /$ 
     $\text{nnreal } (\text{CARD('f)} \wedge \text{fri-round-count-for-degree-bound } (\text{clength} - 1))$ 
  by  $(\text{rule}$ 
     $\text{checked-staged-security-trace-fri-bad-challenge-list-fresh-hit-bound-from-round-bounds}$ 
     $[OF \text{ round-bound}])$ 
show  $\text{?thesis}$ 
by  $(\text{rule order-trans}[OF \text{ base}])$ 
   $(\text{simp add: concrete-trace-fri-bad-challenge-fraction-def}$ 
     $\text{fri-one-step-disagreement-round-fraction-def}$ 
     $\text{fri-bad-challenge-round-fraction-def}$ 
     $\text{fri-one-step-disagreement-round-mass-def}$ 
     $\text{fri-bad-challenge-round-mass-def})$ 
qed
have  $\text{fiber-fraction:}$ 
   $\text{nnreal}$ 
   $((\sum \text{challenges} \in$ 
     $\text{fri-challenge-space } (\text{fri-round-count-for-degree-bound } (\text{clength} - 1)) -$ 
     $\text{?}B.$ 
     $\text{card}$ 

```

```

    (generic-fri-sampled-query-query-fiber trace-table-low-degree
      (Not  $\circ$  trace-table-low-degree) (clength - 1) challenges)) *
    rounds) /
    nnreal (card query-sample-space)  $\leq$  ?Q
  by (simp add: trace-one-step-query-fraction-for-def)
have sampled:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-sampled-query-bad-candidate)
  adversary-initial-state
   $\leq$ 
  concrete-trace-fri-bad-challenge-fraction +
  hash-relation-budget-value (card ?B * ceil-log clength)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget) +
  nnreal
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget) * ?Q
  by (rule
    checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-outside-fib
      [OF wf controlled finite-generic-fri-bad-challenge-lists
        fresh-bound fiber-fraction])
have card-le:
  card ?B * ceil-log clength  $\leq$  trace-one-step-challenge-hash-count
proof -
  have raw:
    card ?B  $\leq$ 
    ( $\sum i \in \{..< \text{fri-round-count-for-degree-bound (clength - 1)}\}$ .
      CARD('f)  $\wedge$  i * (trace-fri-canonical-layer-len i div 2) *
      CARD('f)  $\wedge$ 
      (fri-round-count-for-degree-bound (clength - 1) - Suc i))
  proof (rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds)
    fix i :: nat and prefix :: 'f list
    assume i: i < fri-round-count-for-degree-bound (clength - 1)
      and prefix-len: length prefix = i
    show card (empty-bad i prefix)
       $\leq$  trace-fri-canonical-layer-len i div 2
    by (rule round-bound[OF i prefix-len])
  qed
have card ?B  $\leq$ 
  fri-one-step-disagreement-round-mass
    (fri-round-count-for-degree-bound (clength - 1))
    trace-fri-canonical-layer-len
  using raw
  by (simp add: fri-one-step-disagreement-round-mass-def
    fri-bad-challenge-round-mass-def)
then show ?thesis
  unfolding trace-one-step-challenge-hash-count-def
  by (rule mult-right-mono) simp

```

```

qed
have budget-le:
  hash-relation-budget-value (card ?B * ceil-log clength)
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)
    ≤
    hash-relation-budget-value trace-one-step-challenge-hash-count
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)
  by (rule hash-relation-budget-value-mono-left[OF card-le])
show ?thesis
  by (rule order-trans[OF sampled])
    (use budget-le in ⟨simp add: trace-one-step-sampled-error-for-def⟩)
qed

lemma checked-staged-security-trace-fri-empty-header-bound-from-staged-sampled-conflict-and-zero:
  assumes sampled-query:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-sampled-query-bad-candidate)
    adversary-initial-state ≤ R
  and conflict-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-sampled-assignment-conflict)
    adversary-initial-state ≤ C
  and zero-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-zero-round-final-obstruction)
    adversary-initial-state ≤ Z
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state ≤ R + (C + Z)
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?sampled =
    staged-security-with-data-state-verifier-event
    trace-fri-bad-with-sampled-layer-chain
  let ?missing =
    staged-security-with-data-state-verifier-event
    trace-fri-reachable-missing-sampled-layer-chain
  let ?conflict =
    staged-security-with-data-state-verifier-event
    trace-fri-sampled-assignment-conflict
  let ?zero =
    staged-security-with-data-state-verifier-event

```

```

    trace-fri-zero-round-final-obstruction
  have sampled:
    wp-event ?M ?sampled adversary-initial-state ≤ R
  by (rule
    checked-staged-security-trace-fri-sampled-layer-chain-bound-from-sampled-query
    [OF sampled-query])
  have missing-split:
    wp-event ?M ?missing adversary-initial-state ≤
    wp-event ?M (λout. ?conflict out ∨ ?zero out)
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out ∈ set-dist (execute ?M adversary-initial-state)
      and missing: ?missing out
    show ?conflict out ∨ ?zero out
  proof (cases out)
    case None
    then show ?thesis
      using missing unfolding staged-security-with-data-state-verifier-event-def
      by simp
  next
    case (Some out-data)
    then obtain data attacker-state result final-state where out-eq:
      out = Some (((data, attacker-state), result), final-state)
    by (cases out-data, auto split: prod.splits)
    let ?s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    from checked-staged-security-experiment-with-data-state-outcomeE
      [OF support[unfolded out-eq]]
    have verify-support:
      Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
    by blast
    have missing-local:
      trace-fri-reachable-missing-sampled-layer-chain ?s
      (Some (result, final-state))
    using missing unfolding out-eq
      staged-security-with-data-state-verifier-event-def
    by simp
    have obstruction:
      trace-fri-sampled-layer-assignment-obstruction ?s
      (Some (result, final-state))
    by (rule
      trace-fri-reachable-missing-sampled-imp-assignment-obstruction-on-support
      [OF verify-support missing-local])
    then have
      trace-fri-sampled-assignment-conflict ?s
      (Some (result, final-state)) ∨

```

```

    trace-fri-zero-round-final-obstruction ?s
    (Some (result, final-state))
  by (rule
    trace-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final)
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
have missing-bound:
  wp-event ?M ?missing adversary-initial-state  $\leq C + Z$ 
proof -
  have wp-event ?M ?missing adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda$ out. ?conflict out  $\vee$  ?zero out)
      adversary-initial-state
  by (rule missing-split)
  also have ...  $\leq$ 
    wp-event ?M ?conflict adversary-initial-state +
    wp-event ?M ?zero adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq C + Z$ 
  by (rule add-mono[OF conflict-bound zero-bound])
  finally show ?thesis .
qed
have split:
  wp-event ?M
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state  $\leq$ 
  wp-event ?M ( $\lambda$ out. ?sampled out  $\vee$  ?missing out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    dest:
      trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate
      trace-fri-reachable-imp-sampled-or-missing-layer-chain
      split: option.splits prod.splits)
  also have ...  $\leq$ 
    wp-event ?M ?sampled adversary-initial-state +
    wp-event ?M ?missing adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq R + (C + Z)$ 
  by (rule add-mono[OF sampled missing-bound])
  finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-header-tied-candidate-transfer-gap-bound-from-staged-header-and-untied:
  fixes H U :: prob
  assumes header-bound:

```

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state  $\leq H$ 
and untied-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap)
  adversary-initial-state  $\leq U$ 
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-header-tied-candidate-transfer-gap)
  adversary-initial-state  $\leq H + U$ 
proof –
let ?M = checked-staged-security-experiment-with-data-state A
let ?header =
  staged-security-with-data-state-verifier-event
  trace-fri-bad-with-header-tied-partial-candidate
let ?untied =
  staged-security-with-data-state-verifier-event
  trace-fri-header-tied-low-candidate-untied-reachable-witness-gap
have wp-event ?M
  (staged-security-with-data-state-verifier-event
    trace-fri-header-tied-candidate-transfer-gap)
  adversary-initial-state  $\leq$ 
  wp-event ?M ( $\lambda$ out. ?header out  $\vee$  ?untied out)
  adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
by (rule wp-event-mono-on-support)
  (auto dest:
    trace-fri-header-tied-candidate-transfer-gap-imp-header-or-untied
    split: option.splits prod.splits)
also have ...  $\leq$ 
  wp-event ?M ?header adversary-initial-state +
  wp-event ?M ?untied adversary-initial-state
by (rule wp-event-union-bound)
also have ...  $\leq H + U$ 
by (rule add-mono[OF header-bound untied-bound])
finally show ?thesis .
qed

```

```

lemma checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-staged-checked-merkle-transfer-head
fixes H M Q T :: prob
assumes checked-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-zero-round-checked-final-obstruction)
  adversary-initial-state  $\leq Q$ 

```


and merkle-bound:
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
partial-merkle-inconsistency-bad)
adversary-initial-state $\leq M$
and transfer-bound:
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-header-tied-candidate-transfer-gap)
adversary-initial-state $\leq T$
and header-bound:
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state $\leq H$
shows
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-zero-round-final-obstruction)
adversary-initial-state $\leq Q + M + T + H$
proof –
let ?*M* = *checked-staged-security-experiment-with-data-state A*
let ?*checked* =
staged-security-with-data-state-verifier-event
trace-fri-zero-round-checked-final-obstruction
let ?*merkle* =
staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad
let ?*transfer* =
staged-security-with-data-state-verifier-event
trace-fri-header-tied-candidate-transfer-gap
let ?*header* =
staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-partial-candidate
have *wp-event* ?*M*
 (*staged-security-with-data-state-verifier-event*
trace-fri-zero-round-final-obstruction)
adversary-initial-state $\leq$
wp-event ?*M*
 ($\lambda out.$
 ?*checked out* $\vee$?*merkle out* $\vee$?*transfer out* $\vee$?*header out*)
adversary-initial-state
proof (*rule wp-event-mono-on-support*)
fix *out*
assume *support*:
out $\in$ *set-dist* (*execute* ?*M* *adversary-initial-state*)
and *obstruction*:
staged-security-with-data-state-verifier-event
trace-fri-zero-round-final-obstruction out

```

show ?checked out  $\vee$  ?merkle out  $\vee$  ?transfer out  $\vee$  ?header out
proof (cases out)
  case None
  then show ?thesis
    using obstruction
    unfolding staged-security-with-data-state-verifier-event-def
    by simp
next
  case (Some out-data)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases out-data, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
  have verify-support:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)
    by blast
  have obstruction-local:
    trace-fri-zero-round-final-obstruction ?s
    (Some (result, final-state))
    using obstruction
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
  have
    trace-fri-zero-round-checked-final-obstruction ?s
    (Some (result, final-state))  $\vee$ 
    partial-merkle-inconsistency-bad ?s
    (Some (result, final-state))  $\vee$ 
    trace-fri-header-tied-candidate-transfer-gap ?s
    (Some (result, final-state))  $\vee$ 
    trace-fri-bad-with-header-tied-partial-candidate ?s
    (Some (result, final-state))
    by (rule
      trace-fri-zero-round-final-obstruction-imp-checked-merkle-transfer-or-header
      [OF verify-support obstruction-local])
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
also have ...  $\leq$ 
  wp-event ?M ?checked adversary-initial-state +
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?transfer adversary-initial-state +
  wp-event ?M ?header adversary-initial-state
  by (rule wp-event-union-bound4)

```

also have $\dots \leq Q + M + T + H$
 by (intro add-mono checked-bound merkle-bound transfer-bound
 header-bound)
 finally show ?thesis .
 qed

lemma *checked-staged-security-trace-fri-zero-round-checked-final-obstruction-bound-from-agreement-sets:*
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-zero-round-checked-final-obstruction)
 adversary-initial-state $\leq$
 trace-fri-zero-round-agreement-set-target-error-for budgets
proof –
have event-le:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-zero-round-checked-final-obstruction)
 adversary-initial-state $\leq$
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-zero-round-query-list-target-hit)
 adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
by (rule wp-event-mono-on-support)
 (auto
 intro:
 trace-fri-zero-round-checked-first-query-target-hit-imp-query-list-target
 dest:
 trace-fri-zero-round-checked-final-obstruction-imp-first-query-target-hit
 split: option.splits prod.splits)
show ?thesis
by (rule order-trans[OF event-le])
 (rule checked-staged-security-trace-fri-zero-round-query-list-target-bound-from-agreement-sets
 [OF wf controlled])
 qed

lemma *checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets:*
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-zero-round-final-obstruction)
 adversary-initial-state $\leq$
 trace-fri-zero-round-agreement-set-target-error-for budgets +
 trace-fri-controlled-merkle-error-for budgets +

```

    reachable-verifier-event-bound-for A
    trace-fri-bad-with-header-tied-partial-candidate +
    (trace-fri-controlled-merkle-error-for budgets +
    (reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-root-gap +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-query-gap)) +
    reachable-verifier-event-bound-for A
    trace-fri-bad-with-header-tied-partial-candidate
proof –
have checked:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
    trace-fri-zero-round-checked-final-obstruction)
    adversary-initial-state ≤
    trace-fri-zero-round-agreement-set-target-error-for budgets
by (rule checked-staged-security-trace-fri-zero-round-checked-final-obstruction-bound-from-agreement-sets
    [OF wf controlled])
have merkle:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad)
    adversary-initial-state ≤
    trace-fri-controlled-merkle-error-for budgets
unfolding trace-fri-controlled-merkle-error-for-def
by (rule
    checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
    [OF wf controlled])
have header:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate)
    adversary-initial-state ≤
    reachable-verifier-event-bound-for A
    trace-fri-bad-with-header-tied-partial-candidate
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
show  $\bigwedge s. \neg \text{trace-fri-bad-with-header-tied-partial-candidate } s \text{ None}$ 
unfolding trace-fri-bad-with-header-tied-partial-candidate-def
    accepted-fri-opening-transcript-def
by simp
next
fix data attacker-state
assume support: Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
    adversary-initial-state)
show wp-event verify-monad
    (trace-fri-bad-with-header-tied-partial-candidate
    (verifier-state-from-adversary attacker-state)

```

```

    (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  ≤ reachable-verifier-event-bound-for A
    trace-fri-bad-with-header-tied-partial-candidate
  by (rule reachable-verifier-event-bound-for-ge[OF support])
qed
have hash:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      hash-map-output-collision-bad)
  adversary-initial-state ≤
    trace-fri-controlled-merkle-error-for budgets
  unfolding trace-fri-controlled-merkle-error-for-def
  by (rule
    checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
      [OF wf controlled])
have root:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-untied-root-gap)
  adversary-initial-state ≤
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-root-gap
  proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    show  $\bigwedge s. \neg \text{trace-fri-header-tied-low-candidate-untied-root-gap } s \text{ None}$ 
    unfolding trace-fri-header-tied-low-candidate-untied-root-gap-def
      trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
      accepted-fri-opening-transcript-def
    by simp
  next
  fix data attacker-state
  assume support: Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  show wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-root-gap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    ≤ reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-untied-root-gap
  by (rule reachable-verifier-event-bound-for-ge[OF support])
qed
have query:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event

```

```

    trace-fri-header-tied-low-candidate-untied-query-gap)
  adversary-initial-state ≤
  reachable-verifier-event-bound-for A
  trace-fri-header-tied-low-candidate-untied-query-gap
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
show  $\bigwedge s. \neg \text{trace-fri-header-tied-low-candidate-untied-query-gap } s \text{ None}$ 
  unfolding trace-fri-header-tied-low-candidate-untied-query-gap-def
    trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
    accepted-fri-opening-transcript-def
  by simp
next
fix data attacker-state
assume support: Some (data, attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-program A)
    adversary-initial-state)
show wp-event verify-monad
  (trace-fri-header-tied-low-candidate-untied-query-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  ≤ reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-query-gap
  by (rule reachable-verifier-event-bound-for-ge[OF support])
qed
have untied:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-untied-reachable-witness-gap)
  adversary-initial-state ≤
  trace-fri-controlled-merkle-error-for budgets +
  (reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-root-gap +
    reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-untied-query-gap)
proof -
let ?M = checked-staged-security-experiment-with-data-state A
let ?untied =
  staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap
let ?hash =
  staged-security-with-data-state-verifier-event
    hash-map-output-collision-bad
let ?root =
  staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-untied-root-gap
let ?query =
  staged-security-with-data-state-verifier-event

```

```

    trace-fri-header-tied-low-candidate-untied-query-gap
have split:
  wp-event ?M ?untied adversary-initial-state ≤
    wp-event ?M (λout. ?hash out ∨ ?root out ∨ ?query out)
    adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
by (rule wp-event-mono-on-support)
  (auto
    dest:
      trace-fri-header-tied-low-candidate-untied-gap-imp-hash-root-or-query
      split: option.splits prod.splits)
also have ... ≤
  wp-event ?M ?hash adversary-initial-state +
  wp-event ?M (λout. ?root out ∨ ?query out)
  adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
  wp-event ?M ?hash adversary-initial-state +
  (wp-event ?M ?root adversary-initial-state +
  wp-event ?M ?query adversary-initial-state)
by (intro add-mono order-refl wp-event-union-bound)
also have ... ≤
  trace-fri-controlled-merkle-error-for budgets +
  (reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-root-gap +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-query-gap)
by (intro add-mono hash root query)
finally show ?thesis .
qed
have transfer:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-header-tied-candidate-transfer-gap)
  adversary-initial-state ≤
  reachable-verifier-event-bound-for A
  trace-fri-bad-with-header-tied-partial-candidate +
  (trace-fri-controlled-merkle-error-for budgets +
  (reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-root-gap +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-query-gap))
by (rule
  checked-staged-security-trace-fri-header-tied-candidate-transfer-gap-bound-from-staged-header-and-untied
  [OF header untied])
have raw:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-zero-round-final-obstruction)

```

$adversary-initial-state \leq$
 $trace-fri-zero-round-agreement-set-target-error-for\ budgets +$
 $trace-fri-controlled-merkle-error-for\ budgets +$
 $(reachable-verifier-event-bound-for\ A$
 $\quad trace-fri-bad-with-header-tied-partial-candidate +$
 $\quad (trace-fri-controlled-merkle-error-for\ budgets +$
 $\quad \quad (reachable-verifier-event-bound-for\ A$
 $\quad \quad \quad trace-fri-header-tied-low-candidate-untied-root-gap +$
 $\quad \quad \quad \quad reachable-verifier-event-bound-for\ A$
 $\quad \quad \quad \quad \quad trace-fri-header-tied-low-candidate-untied-query-gap))) +$
 $reachable-verifier-event-bound-for\ A$
 $\quad trace-fri-bad-with-header-tied-partial-candidate$
by (rule
 $checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-staged-checked-merkle-transfer-header$
 $[OF\ checked\ merkle\ transfer\ header])$
show ?thesis
by (rule order-trans[$OF\ raw$]) (simp add: algebra-simps)
qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-staged-sampled-conflict-and*
assumes *sampled-bound:*
 $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $\quad composition-fri-bad-with-verifier-tied-sampled-layer-chain)$
 $adversary-initial-state \leq R$
and *conflict-bound:*
 $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $\quad composition-fri-verifier-tied-sampled-assignment-conflict)$
 $adversary-initial-state \leq C$
and *zero-bound:*
 $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $\quad composition-fri-verifier-tied-zero-round-final-obstruction)$
 $adversary-initial-state \leq Z$
shows
 $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $\quad composition-fri-bad-with-verifier-tied-partial-candidate)$
 $adversary-initial-state \leq R + (C + Z)$
proof –
let ?M = *checked-staged-security-experiment-with-data-state A*
let ?sampled =
 $staged-security-with-data-state-verifier-event$
 $\quad composition-fri-bad-with-verifier-tied-sampled-layer-chain$
let ?missing =
 $staged-security-with-data-state-verifier-event$
 $\quad composition-fri-verifier-tied-missing-sampled-layer-chain$
let ?conflict =


```

    staged-security-with-data-state-verifier-event
    composition-fri-verifier-tied-sampled-assignment-conflict
let ?zero =
    staged-security-with-data-state-verifier-event
    composition-fri-verifier-tied-zero-round-final-obstruction
have missing-split:
    wp-event ?M ?missing adversary-initial-state ≤
    wp-event ?M (λout. ?conflict out ∨ ?zero out)
    adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈ set-dist (execute ?M adversary-initial-state)
    and missing: ?missing out
  show ?conflict out ∨ ?zero out
  proof (cases out)
    case None
    then show ?thesis
      using missing unfolding staged-security-with-data-state-verifier-event-def
      by simp
  next
    case (Some out-data)
    then obtain data attacker-state result final-state where out-eq:
      out = Some (((data, attacker-state), result), final-state)
      by (cases out-data, auto split: prod.splits)
    let ?s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    from checked-staged-security-experiment-with-data-state-outcomeE
      [OF support[unfolded out-eq]]
    have verify-support:
      Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
      by blast
    have missing-local:
      composition-fri-verifier-tied-missing-sampled-layer-chain ?s
      (Some (result, final-state))
      using missing unfolding out-eq
      staged-security-with-data-state-verifier-event-def
      by simp
    have obstruction:
      composition-fri-verifier-tied-sampled-layer-assignment-obstruction
      ?s (Some (result, final-state))
      by (rule
        composition-fri-verifier-tied-missing-sampled-imp-assignment-obstruction
        [OF missing-local])
    then have
      composition-fri-verifier-tied-sampled-assignment-conflict ?s
      (Some (result, final-state)) ∨
      composition-fri-verifier-tied-zero-round-final-obstruction ?s

```

```

      (Some (result, final-state))
    by (rule
      composition-fri-verifier-tied-assignment-obstruction-imp-conflict-or-zero-on-support
      [OF verify-support])
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
have missing-bound:
  wp-event ?M ?missing adversary-initial-state  $\leq C + Z$ 
proof -
  have wp-event ?M ?missing adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda$ out. ?conflict out  $\vee$  ?zero out)
      adversary-initial-state
    by (rule missing-split)
  also have ...  $\leq$ 
    wp-event ?M ?conflict adversary-initial-state +
    wp-event ?M ?zero adversary-initial-state
    by (rule wp-event-union-bound)
  also have ...  $\leq C + Z$ 
    by (rule add-mono[OF conflict-bound zero-bound])
  finally show ?thesis .
qed
have split:
  wp-event ?M
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-verifier-tied-partial-candidate)
    adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda$ out. ?sampled out  $\vee$  ?missing out)
      adversary-initial-state
  unfolding composition-fri-verifier-tied-missing-sampled-layer-chain-def
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      split: option.splits prod.splits)
  also have ...  $\leq$ 
    wp-event ?M ?sampled adversary-initial-state +
    wp-event ?M ?missing adversary-initial-state
    by (rule wp-event-union-bound)
  also have ...  $\leq R + (C + Z)$ 
    by (rule add-mono[OF sampled-bound missing-bound])
  finally show ?thesis .
qed

lemma checked-staged-security-partial-merkle-inconsistency-bad-bound-by-composition-one-step-controlled-merkle-err
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (staged-security-with-data-state-verifier-event
     partial-merkle-inconsistency-bad)
    adversary-initial-state  $\leq$ 
    composition-one-step-controlled-merkle-error-for budgets
unfolding composition-one-step-controlled-merkle-error-for-def
by (rule
     checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
     [OF wf controlled])

lemma checked-staged-security-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap)
  adversary-initial-state  $\leq$  (0::prob)
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg$ 
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s None
  unfolding
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-def
  by simp
next
  fix data attacker-state
  assume Some (data, attacker-state)  $\in$ 
    set-dist
    (execute (checked-staged-transcript-program A)
     adversary-initial-state)
  show wp-event verify-monad
    (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data))
     $\leq 0$ 
  by (rule
    wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
qed

lemma checked-staged-security-composition-fri-verifier-tied-next-value-replay-gap-bound-by-controlled-merkle:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-fri-verifier-tied-next-value-replay-gap)
    adversary-initial-state  $\leq$ 
    composition-one-step-controlled-merkle-error-for budgets + 0
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?merkle =

```

```

    staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad
  let ?slot =
    staged-security-with-data-state-verifier-event
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
  have split:
    wp-event ?M
    (staged-security-with-data-state-verifier-event
     composition-fri-verifier-tied-next-value-replay-gap)
    adversary-initial-state ≤
    wp-event ?M (λout. ?merkle out ∨ ?slot out)
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out ∈ set-dist (execute ?M adversary-initial-state)
    and gap:
      staged-security-with-data-state-verifier-event
      composition-fri-verifier-tied-next-value-replay-gap out
    show ?merkle out ∨ ?slot out
    proof (cases out)
      case None
      then show ?thesis
        using gap unfolding staged-security-with-data-state-verifier-event-def
        by simp
    next
      case (Some out-data)
      then obtain data attacker-state result final-state where out-eq:
        out = Some (((data, attacker-state), result), final-state)
        by (cases out-data, auto split: prod.splits)
      let ?s =
        verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)
      from checked-staged-security-experiment-with-data-state-outcomeE
        [OF support[unfolded out-eq]]
      have verify-support:
        Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
        by blast
      have gap-local:
        composition-fri-verifier-tied-next-value-replay-gap ?s
        (Some (result, final-state))
      using gap unfolding out-eq
        staged-security-with-data-state-verifier-event-def
      by simp
    have
      partial-merkle-inconsistency-bad ?s
      (Some (result, final-state)) ∨
      composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
      ?s (Some (result, final-state))

```

```

    by (rule
      composition-fri-verifier-tied-next-value-replay-gap-imp-merkle-or-slot-on-support
      [OF verify-support gap-local])
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
also have ... ≤
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?slot adversary-initial-state
  by (rule wp-event-union-bound)
also have ... ≤
  composition-one-step-controlled-merkle-error-for budgets + 0
  by (intro add-mono
    checked-staged-security-partial-merkle-inconsistency-bad-bound-by-composition-one-step-controlled-merkle-error
    [OF wf controlled]
    checked-staged-security-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
  finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-verifier-tied-successor-replay-gap-bound-by-controlled-merkle:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-verifier-tied-successor-replay-gap)
      adversary-initial-state ≤
      composition-one-step-controlled-merkle-error-for budgets + 0
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?merkle =
    staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad
  let ?slot =
    staged-security-with-data-state-verifier-event
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
  have split:
    wp-event ?M
      (staged-security-with-data-state-verifier-event
        composition-fri-verifier-tied-successor-replay-gap)
      adversary-initial-state ≤
    wp-event ?M (λout. ?merkle out ∨ ?slot out)
      adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out ∈ set-dist (execute ?M adversary-initial-state)

```

```

and gap:
  staged-security-with-data-state-verifier-event
  composition-fri-verifier-tied-successor-replay-gap out
show ?merkle out  $\vee$  ?slot out
proof (cases out)
  case None
  then show ?thesis
    using gap unfolding staged-security-with-data-state-verifier-event-def
    by simp
next
case (Some out-data)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases out-data, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF support[unfolded out-eq]]
have verify-support:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)
  by blast
have gap-local:
  composition-fri-verifier-tied-successor-replay-gap ?s
  (Some (result, final-state))
  using gap unfolding out-eq
  staged-security-with-data-state-verifier-event-def
  by simp
have
  partial-merkle-inconsistency-bad ?s
  (Some (result, final-state))  $\vee$ 
  composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
  ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-successor-replay-gap-imp-merkle-or-slot-on-support
    [OF verify-support gap-local])
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
also have ...  $\leq$ 
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?slot adversary-initial-state
  by (rule wp-event-union-bound)
also have ...  $\leq$ 
  composition-one-step-controlled-merkle-error-for budgets + 0
  by (intro add-mono
    checked-staged-security-partial-merkle-inconsistency-bad-bound-by-composition-one-step-controlled-merkle-error

```

```

    [OF wf controlled]
    checked-staged-security-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
  finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-verifier-tied-final-value-selected-step-missing-gap)
  adversary-initial-state ≤ (0::prob)
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg$ 
    composition-fri-verifier-tied-final-value-selected-step-missing-gap s None
  unfolding
    composition-fri-verifier-tied-final-value-selected-step-missing-gap-def
    composition-fri-verifier-tied-final-value-authenticated-replay-gap-def
    accepted-fri-opening-transcript-def
  by simp
next
  fix data attacker-state
  assume Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  show wp-event verify-monad
    (composition-fri-verifier-tied-final-value-selected-step-missing-gap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    ≤ 0
  by (rule
    wp-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero)
qed

lemma checked-staged-security-composition-fri-verifier-tied-final-value-replay-gap-bound-by-controlled-merkle:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-verifier-tied-final-value-replay-gap)
    adversary-initial-state ≤
      composition-one-step-controlled-merkle-error-for budgets + 0 + 0
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?merkle =
    staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad

```

```

let ?slot =
  staged-security-with-data-state-verifier-event
  composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
let ?selected =
  staged-security-with-data-state-verifier-event
  composition-fri-verifier-tied-final-value-selected-step-missing-gap
have split:
  wp-event ?M
  (staged-security-with-data-state-verifier-event
   composition-fri-verifier-tied-final-value-replay-gap)
  adversary-initial-state ≤
  wp-event ?M
  (λout. ?merkle out ∨ ?slot out ∨ ?selected out)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out ∈ set-dist (execute ?M adversary-initial-state)
and gap:
  staged-security-with-data-state-verifier-event
  composition-fri-verifier-tied-final-value-replay-gap out
show ?merkle out ∨ ?slot out ∨ ?selected out
proof (cases out)
case None
then show ?thesis
  using gap unfolding staged-security-with-data-state-verifier-event-def
  by simp
next
case (Some out-data)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases out-data, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from checked-staged-security-experiment-with-data-state-outcomeE
[OF support[unfolded out-eq]]
have verify-support:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
have gap-local:
  composition-fri-verifier-tied-final-value-replay-gap ?s
  (Some (result, final-state))
  using gap unfolding out-eq
  staged-security-with-data-state-verifier-event-def
  by simp
have
  partial-merkle-inconsistency-bad ?s
  (Some (result, final-state)) ∨

```



```

    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-final-value-authenticated-replay-gap
    ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-final-value-replay-gap-imp-merkle-or-slot-or-authenticated-on-support
    [OF verify-support gap-local])
then show ?thesis
proof
  assume partial-merkle-inconsistency-bad ?s
    (Some (result, final-state))
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
next
  assume tail:
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-final-value-authenticated-replay-gap
    ?s (Some (result, final-state))
  then show ?thesis
  proof
    assume
      composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
      ?s (Some (result, final-state))
    then show ?thesis
      unfolding out-eq staged-security-with-data-state-verifier-event-def
      by simp
  next
    assume auth:
      composition-fri-verifier-tied-final-value-authenticated-replay-gap
      ?s (Some (result, final-state))
    have selected:
      composition-fri-verifier-tied-final-value-selected-step-missing-gap
      ?s (Some (result, final-state))
    by (rule
      composition-fri-verifier-tied-final-value-authenticated-replay-gap-imp-selected-step-missing
      [OF auth])
    then show ?thesis
      unfolding out-eq staged-security-with-data-state-verifier-event-def
      by simp
  qed
qed
qed
qed
also have ... ≤
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?slot adversary-initial-state +
  wp-event ?M ?selected adversary-initial-state

```

```

proof –
  have wp-event ?M
    ( $\lambda out. ?merkle\ out \vee ?slot\ out \vee ?selected\ out$ )
    adversary-initial-state  $\leq$ 
    wp-event ?M ?merkle adversary-initial-state +
    wp-event ?M ( $\lambda out. ?slot\ out \vee ?selected\ out$ )
    adversary-initial-state
  by (rule wp-event-union-bound)
also have ...  $\leq$ 
  wp-event ?M ?merkle adversary-initial-state +
  (wp-event ?M ?slot adversary-initial-state +
  wp-event ?M ?selected adversary-initial-state)
  by (intro add-mono order-refl wp-event-union-bound)
finally show ?thesis
  by (simp add: add.assoc)
qed
also have ...  $\leq$ 
  composition-one-step-controlled-merkle-error-for budgets + 0 + 0
by (intro add-mono
  checked-staged-security-partial-merkle-inconsistency-bad-bound-by-composition-one-step-controlled-merkle-error
  [OF wf controlled]
  checked-staged-security-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero
  checked-staged-security-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero)
finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
  composition-fri-verifier-tied-recorded-base-chunk-auth-missing)
  adversary-initial-state  $\leq$  (0::prob)
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
show  $\bigwedge s. \neg$ 
  composition-fri-verifier-tied-recorded-base-chunk-auth-missing s None
unfolding
  composition-fri-verifier-tied-recorded-base-chunk-auth-missing-def
  accepted-fri-opening-transcript-def
by simp
next
fix data attacker-state
assume Some (data, attacker-state)  $\in$ 
  set-dist
  (execute (checked-staged-transcript-program A)
  adversary-initial-state)
show wp-event verify-monad
  (composition-fri-verifier-tied-recorded-base-chunk-auth-missing
  (verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state

```

```

      (staged-proof-transcript data))
    ≤ 0
  by (rule
      wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero)
qed

```

lemma *checked-staged-security-composition-fri-verifier-tied-sampled-assignment-conflict-bound-by-controlled-merkle:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-verifier-tied-sampled-assignment-conflict)
  adversary-initial-state ≤
  (((composition-one-step-controlled-merkle-error-for budgets + 0) +
    (composition-one-step-controlled-merkle-error-for budgets + 0) +
    (composition-one-step-controlled-merkle-error-for budgets + 0) +
    (composition-one-step-controlled-merkle-error-for budgets + 0 + 0) +
    composition-one-step-controlled-merkle-error-for budgets) + 0)

```

proof —

let *?M = checked-staged-security-experiment-with-data-state A*

let *?conflict =*

```

  staged-security-with-data-state-verifier-event
  composition-fri-verifier-tied-sampled-assignment-conflict

```

let *?merkle =*

```

  staged-security-with-data-state-verifier-event
  partial-merkle-inconsistency-bad

```

let *?slot =*

```

  staged-security-with-data-state-verifier-event
  composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap

```

let *?next =*

```

  staged-security-with-data-state-verifier-event
  composition-fri-verifier-tied-next-value-replay-gap

```

let *?successor =*

```

  staged-security-with-data-state-verifier-event
  composition-fri-verifier-tied-successor-replay-gap

```

let *?final =*

```

  staged-security-with-data-state-verifier-event
  composition-fri-verifier-tied-final-value-replay-gap

```

have *split:*

```

wp-event ?M ?conflict adversary-initial-state ≤
wp-event ?M
  (λout.
    ?merkle out ∨ ?slot out ∨ ?next out ∨
    ?successor out ∨ ?final out ∨ ?merkle out ∨ ?slot out)
  adversary-initial-state

```

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support:*

```

    out ∈ set-dist (execute ?M adversary-initial-state)
  and conflict: ?conflict out
show ?merkle out ∨ ?slot out ∨ ?next out ∨
    ?successor out ∨ ?final out ∨ ?merkle out ∨ ?slot out
proof (cases out)
  case None
  then show ?thesis
    using conflict unfolding staged-security-with-data-state-verifier-event-def
    by simp
next
  case (Some out-data)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases out-data, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
  have verify-support:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
    by blast
  have conflict-local:
    composition-fri-verifier-tied-sampled-assignment-conflict ?s
    (Some (result, final-state))
    using conflict unfolding out-eq
    staged-security-with-data-state-verifier-event-def
    by simp
  have local-split:
    partial-merkle-inconsistency-bad ?s
    (Some (result, final-state)) ∨
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-next-value-replay-gap
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-successor-replay-gap
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-final-value-replay-gap
    ?s (Some (result, final-state)) ∨
    partial-merkle-inconsistency-bad ?s
    (Some (result, final-state)) ∨
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
    ?s (Some (result, final-state))
  proof -
    have auth-or-gap:
      composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
      ?s (Some (result, final-state)) ∨
      composition-fri-verifier-tied-assignment-auth-gap
      ?s (Some (result, final-state))

```

```

    by (rule
        composition-fri-verifier-tied-assignment-imp-authenticated-or-auth-gap
        [OF conflict-local])
  then show ?thesis
proof
  assume auth:
    composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
    ?s (Some (result, final-state))
  have without-or-same:
    composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
    ?s (Some (result, final-state))
  by (rule
      composition-fri-verifier-tied-authenticated-conflict-imp-without-same-or-same-layer
      [OF auth])
  then show ?thesis
proof
  assume without:
    composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
    ?s (Some (result, final-state))
  then have branches:
    composition-fri-verifier-tied-sampled-base-opening-conflict
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-sampled-next-value-conflict
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-sampled-successor-opening-conflict
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-sampled-final-value-conflict
    ?s (Some (result, final-state))
  using composition-fri-verifier-tied-without-same-iff-branches
  by blast
  then show ?thesis
proof
  assume base:
    composition-fri-verifier-tied-sampled-base-opening-conflict
    ?s (Some (result, final-state))
  have base-split:
    composition-fri-verifier-tied-authenticated-base-opening-conflict
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-base-chunk-auth-gap
    ?s (Some (result, final-state))
  by (rule
      composition-fri-verifier-tied-sampled-base-imp-authenticated-or-auth-gap
      [OF base])
  then show ?thesis
proof
  assume auth-base:
    composition-fri-verifier-tied-authenticated-base-opening-conflict

```

```

      ?s (Some (result, final-state))
    then have
      partial-merkle-inconsistency-bad ?s
        (Some (result, final-state))
      by (rule
        composition-fri-verifier-tied-authenticated-base-conflict-imp-partial-merkle)
    then show ?thesis by simp
  next
    assume gap:
      composition-fri-verifier-tied-base-chunk-auth-gap
      ?s (Some (result, final-state))
    then have missing:
      composition-fri-verifier-tied-recorded-base-chunk-auth-missing
      ?s (Some (result, final-state))
    by (rule
      composition-fri-verifier-tied-base-chunk-auth-gap-imp-recorded-missing)
    then have
      composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
      ?s (Some (result, final-state))
    by (rule
      composition-fri-verifier-tied-recorded-base-chunk-auth-missing-imp-slot-recorded-chunk-gap)
    then show ?thesis by simp
  qed
next
  assume rest:
    composition-fri-verifier-tied-sampled-next-value-conflict
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-sampled-successor-opening-conflict
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-sampled-final-value-conflict
    ?s (Some (result, final-state))
  then show ?thesis
proof
  assume next-conflict:
    composition-fri-verifier-tied-sampled-next-value-conflict
    ?s (Some (result, final-state))
  then have
    composition-fri-verifier-tied-next-value-replay-gap
    ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-sampled-next-value-imp-replay-gap)
  then show ?thesis by simp
next
  assume tail:
    composition-fri-verifier-tied-sampled-successor-opening-conflict
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-sampled-final-value-conflict
    ?s (Some (result, final-state))
  then show ?thesis

```

```

proof
  assume successor:
    composition-fri-verifier-tied-sampled-successor-opening-conflict
    ?s (Some (result, final-state))
  then have
    composition-fri-verifier-tied-successor-replay-gap
    ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-sampled-successor-imp-replay-gap)
  then show ?thesis by simp
next
  assume final:
    composition-fri-verifier-tied-sampled-final-value-conflict
    ?s (Some (result, final-state))
  then have
    composition-fri-verifier-tied-final-value-replay-gap
    ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-sampled-final-value-imp-replay-gap)
  then show ?thesis by simp
qed
qed
qed
next
  assume same:
    composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
    ?s (Some (result, final-state))
  then have
    partial-merkle-inconsistency-bad ?s
    (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-imp-partial-merkle)
  then show ?thesis by simp
qed
next
  assume gap:
    composition-fri-verifier-tied-assignment-auth-gap
    ?s (Some (result, final-state))
  have same-gap:
    composition-fri-verifier-tied-same-layer-auth-gap
    ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-assignment-auth-gap-imp-same-layer-auth-gap
    [OF gap])
  have recorded-gap:
    composition-fri-verifier-tied-same-layer-recorded-auth-gap
    ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-same-layer-auth-gap-imp-recorded-auth-gap

```

```

    [OF same-gap])
  have authenticated-gap:
    composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap
    ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-same-layer-recorded-auth-gap-imp-authenticated-recorded-auth-gap-on-support
    [OF verify-support recorded-gap])
  have
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
    ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap
    [OF authenticated-gap])
  then show ?thesis by simp
qed
qed
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
have next-bound:
  wp-event ?M ?next adversary-initial-state ≤
    composition-one-step-controlled-merkle-error-for budgets
  using
    checked-staged-security-composition-fri-verifier-tied-next-value-replay-gap-bound-by-controlled-merkle
    [OF wf controlled]
  by simp
have successor-bound:
  wp-event ?M ?successor adversary-initial-state ≤
    composition-one-step-controlled-merkle-error-for budgets
  using
    checked-staged-security-composition-fri-verifier-tied-successor-replay-gap-bound-by-controlled-merkle
    [OF wf controlled]
  by simp
have final-bound:
  wp-event ?M ?final adversary-initial-state ≤
    composition-one-step-controlled-merkle-error-for budgets
  using
    checked-staged-security-composition-fri-verifier-tied-final-value-replay-gap-bound-by-controlled-merkle
    [OF wf controlled]
  by simp
have split-bound:
  wp-event ?M
  (λout.
    ?merkle out ∨ ?slot out ∨ ?next out ∨
    ?successor out ∨ ?final out ∨ ?merkle out ∨ ?slot out)
  adversary-initial-state ≤
    wp-event ?M ?merkle adversary-initial-state +

```



```

      (wp-event ?M ?slot adversary-initial-state +
      (wp-event ?M ?next adversary-initial-state +
      (wp-event ?M ?successor adversary-initial-state +
      (wp-event ?M ?final adversary-initial-state +
      wp-event ?M ?merkle adversary-initial-state))))
proof -
  let ?nodup =
    λout.
      ?merkle out ∨ ?slot out ∨ ?next out ∨
      ?successor out ∨ ?final out ∨ ?merkle out
have dedup:
  wp-event ?M
    (λout.
      ?merkle out ∨ ?slot out ∨ ?next out ∨
      ?successor out ∨ ?final out ∨ ?merkle out ∨ ?slot out)
  adversary-initial-state ≤
  wp-event ?M ?nodup adversary-initial-state
by (rule wp-event-mono) auto
have union-bound:
  wp-event ?M ?nodup adversary-initial-state ≤
  wp-event ?M ?merkle adversary-initial-state +
  (wp-event ?M ?slot adversary-initial-state +
  (wp-event ?M ?next adversary-initial-state +
  (wp-event ?M ?successor adversary-initial-state +
  (wp-event ?M ?final adversary-initial-state +
  wp-event ?M ?merkle adversary-initial-state))))
proof -
  let ?tail-final = λout. ?final out ∨ ?merkle out
  let ?tail-successor = λout. ?successor out ∨ ?tail-final out
  let ?tail-next = λout. ?next out ∨ ?tail-successor out
  let ?tail-slot = λout. ?slot out ∨ ?tail-next out
  have final-bound':
    wp-event ?M ?tail-final adversary-initial-state ≤
    wp-event ?M ?final adversary-initial-state +
    wp-event ?M ?merkle adversary-initial-state
  by (rule wp-event-union-bound)
  have successor-bound':
    wp-event ?M ?tail-successor adversary-initial-state ≤
    wp-event ?M ?successor adversary-initial-state +
    (wp-event ?M ?final adversary-initial-state +
    wp-event ?M ?merkle adversary-initial-state)
  proof -
    have wp-event ?M ?tail-successor adversary-initial-state ≤
    wp-event ?M ?successor adversary-initial-state +
    wp-event ?M ?tail-final adversary-initial-state
    by (rule wp-event-union-bound)
    also have ... ≤
    wp-event ?M ?successor adversary-initial-state +
    (wp-event ?M ?final adversary-initial-state +

```

```

    wp-event ?M ?merkle adversary-initial-state)
  by (intro add-mono order-refl final-bound')
  finally show ?thesis .
qed
have next-bound':
  wp-event ?M ?tail-next adversary-initial-state ≤
    wp-event ?M ?next adversary-initial-state +
    (wp-event ?M ?successor adversary-initial-state +
    (wp-event ?M ?final adversary-initial-state +
    wp-event ?M ?merkle adversary-initial-state))
proof -
  have wp-event ?M ?tail-next adversary-initial-state ≤
    wp-event ?M ?next adversary-initial-state +
    wp-event ?M ?tail-successor adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤
    wp-event ?M ?next adversary-initial-state +
    (wp-event ?M ?successor adversary-initial-state +
    (wp-event ?M ?final adversary-initial-state +
    wp-event ?M ?merkle adversary-initial-state))
  by (intro add-mono order-refl successor-bound')
  finally show ?thesis .
qed
have slot-bound':
  wp-event ?M ?tail-slot adversary-initial-state ≤
    wp-event ?M ?slot adversary-initial-state +
    (wp-event ?M ?next adversary-initial-state +
    (wp-event ?M ?successor adversary-initial-state +
    (wp-event ?M ?final adversary-initial-state +
    wp-event ?M ?merkle adversary-initial-state)))
proof -
  have wp-event ?M ?tail-slot adversary-initial-state ≤
    wp-event ?M ?slot adversary-initial-state +
    wp-event ?M ?tail-next adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤
    wp-event ?M ?slot adversary-initial-state +
    (wp-event ?M ?next adversary-initial-state +
    (wp-event ?M ?successor adversary-initial-state +
    (wp-event ?M ?final adversary-initial-state +
    wp-event ?M ?merkle adversary-initial-state)))
  by (intro add-mono order-refl next-bound')
  finally show ?thesis .
qed
have wp-event ?M ?nodup adversary-initial-state ≤
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?tail-slot adversary-initial-state
  by (rule wp-event-union-bound)
also have ... ≤

```

```

    wp-event ?M ?merkle adversary-initial-state +
    (wp-event ?M ?slot adversary-initial-state +
    (wp-event ?M ?next adversary-initial-state +
    (wp-event ?M ?successor adversary-initial-state +
    (wp-event ?M ?final adversary-initial-state +
    wp-event ?M ?merkle adversary-initial-state))))
  by (intro add-mono order-refl slot-bound')
  finally show ?thesis .
qed
show ?thesis
  by (rule order-trans[OF dedup union-bound])
qed
have split-sum-bound:
  wp-event ?M ?merkle adversary-initial-state +
  (wp-event ?M ?slot adversary-initial-state +
  (wp-event ?M ?next adversary-initial-state +
  (wp-event ?M ?successor adversary-initial-state +
  (wp-event ?M ?final adversary-initial-state +
  wp-event ?M ?merkle adversary-initial-state)))) ≤
  composition-one-step-controlled-merkle-error-for budgets +
  (0 +
  (composition-one-step-controlled-merkle-error-for budgets +
  (composition-one-step-controlled-merkle-error-for budgets +
  (composition-one-step-controlled-merkle-error-for budgets +
  composition-one-step-controlled-merkle-error-for budgets))))
  by (intro add-mono
    checked-staged-security-partial-merkle-inconsistency-bad-bound-by-composition-one-step-controlled-merkle-error
    [OF wf controlled]
    checked-staged-security-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero
    next-bound successor-bound final-bound)
have wp-event ?M ?conflict adversary-initial-state ≤
  wp-event ?M
  (λout.
    ?merkle out ∨ ?slot out ∨ ?next out ∨
    ?successor out ∨ ?final out ∨ ?merkle out ∨ ?slot out)
  adversary-initial-state
  by (rule split)
also have ... ≤
  wp-event ?M ?merkle adversary-initial-state +
  (wp-event ?M ?slot adversary-initial-state +
  (wp-event ?M ?next adversary-initial-state +
  (wp-event ?M ?successor adversary-initial-state +
  (wp-event ?M ?final adversary-initial-state +
  wp-event ?M ?merkle adversary-initial-state))))
  by (rule split-bound)
also have ... ≤
  composition-one-step-controlled-merkle-error-for budgets +
  (0 +
  (composition-one-step-controlled-merkle-error-for budgets +

```

```

      (composition-one-step-controlled-merkle-error-for budgets +
      (composition-one-step-controlled-merkle-error-for budgets +
      composition-one-step-controlled-merkle-error-for budgets))))
    by (rule split-sum-bound)
  finally show ?thesis
    by (simp add: add.assoc)
qed

```

lemma *composition-one-step-verifier-tied-controlled-error-for-bound:*

```

fixes comp-bad :: 'f  $\Rightarrow$  nat  $\Rightarrow$  'f list  $\Rightarrow$  'f set
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and comp-subset:
   $\bigwedge dg\ i\ prefix.$ 
     $i < fri-round-count-for-degree-bound\ (to-nat\ dg) \implies$ 
     $length\ prefix = i \implies$ 
     $comp-bad\ dg\ i\ prefix \subseteq$ 
     $fri-one-step-disagreement-challenges$ 
     $(composition-fri-canonical-layer-len\ dg\ i)\ (comp-pw\ dg\ i)$ 
     $(comp-layer\ dg\ i\ prefix)\ (comp-claimed\ dg\ i\ prefix)$ 
     $(comp-dom\ dg\ i\ prefix)$ 

```

shows

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-fri-bad-with-verifier-tied-partial-candidate)
  adversary-initial-state
 $\leq$  composition-one-step-verifier-tied-controlled-error-for budgets
  comp-bad

```

proof —

have *sampled-query:*

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-fri-sampled-query-bad-candidate)
  adversary-initial-state
 $\leq$  composition-one-step-sampled-error-for budgets comp-bad
by (rule composition-one-step-sampled-error-for-bound
      [OF wf controlled comp-subset])

```

have *sampled:*

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-fri-bad-with-verifier-tied-sampled-layer-chain)
  adversary-initial-state
 $\leq$  composition-one-step-sampled-error-for budgets comp-bad
by (rule
      checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-sampled-query
      [OF sampled-query])

```

have *conflict:*

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event

```

```

    composition-fri-verifier-tied-sampled-assignment-conflict)
  adversary-initial-state ≤
  (((composition-one-step-controlled-merkle-error-for budgets + 0) +
    (composition-one-step-controlled-merkle-error-for budgets + 0) +
    (composition-one-step-controlled-merkle-error-for budgets + 0) +
    (composition-one-step-controlled-merkle-error-for budgets + 0 + 0) +
    composition-one-step-controlled-merkle-error-for budgets) + 0)
  by (rule
    checked-staged-security-composition-fri-verifier-tied-sampled-assignment-conflict-bound-by-controlled-merkle
    [OF wf controlled])
  have zero:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-verifier-tied-zero-round-final-obstruction)
    adversary-initial-state ≤ (0::prob)
  proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    show  $\bigwedge s. \neg$ 
      composition-fri-verifier-tied-zero-round-final-obstruction s None
    unfolding composition-fri-verifier-tied-zero-round-final-obstruction-def
      accepted-fri-opening-transcript-def
    by simp
  next
  fix data attacker-state
  assume support: Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  show wp-event verify-monad
    (composition-fri-verifier-tied-zero-round-final-obstruction
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    ≤ 0
  by (subst
      wp-composition-fri-verifier-tied-zero-round-final-obstruction-zero)
    simp
  qed
  have branch:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-verifier-tied-partial-candidate)
    adversary-initial-state ≤
    composition-one-step-sampled-error-for budgets comp-bad +
    (((composition-one-step-controlled-merkle-error-for budgets + 0) +
      (composition-one-step-controlled-merkle-error-for budgets + 0) +
      (composition-one-step-controlled-merkle-error-for budgets + 0) +
      (composition-one-step-controlled-merkle-error-for budgets + 0 + 0) +
      composition-one-step-controlled-merkle-error-for budgets) + 0) + 0)

```

```

by (rule
  checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-staged-sampled-conflict-and-zero
  [OF sampled conflict zero])
show ?thesis
using branch
by (simp add: composition-one-step-verifier-tied-controlled-error-for-def)
qed

lemma composition-one-step-verifier-tied-error-for-bound:
fixes comp-bad :: 'f  $\Rightarrow$  nat  $\Rightarrow$  'f list  $\Rightarrow$  'f set
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and comp-subset:
 $\bigwedge$  dg i prefix.
  i < fri-round-count-for-degree-bound (to-nat dg)  $\implies$ 
  length prefix = i  $\implies$ 
  comp-bad dg i prefix  $\subseteq$ 
    fri-one-step-disagreement-challenges
      (composition-fri-canonical-layer-len dg i) (comp-pw dg i)
      (comp-layer dg i prefix) (comp-claimed dg i prefix)
      (comp-dom dg i prefix)
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-verifier-tied-partial-candidate)
    adversary-initial-state
   $\leq$  composition-one-step-verifier-tied-error-for budgets A comp-bad
proof -
have sampled:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-sampled-query-bad-candidate)
    adversary-initial-state
   $\leq$  composition-one-step-sampled-error-for budgets comp-bad
by (rule composition-one-step-sampled-error-for-bound
  [OF wf controlled comp-subset])
have merkle-bound:
 $\bigwedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  wp-event verify-monad
    (partial-merkle-inconsistency-bad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
   $\leq$  composition-one-step-reachable-merkle-error-for A
unfolding composition-one-step-reachable-merkle-error-for-def

```

```

    by (rule reachable-verifier-event-bound-for-ge, assumption)
  have branch:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-verifier-tied-partial-candidate)
      adversary-initial-state
    ≤ composition-one-step-sampled-error-for budgets comp-bad +
      (((composition-one-step-reachable-merkle-error-for A + 0) +
        (composition-one-step-reachable-merkle-error-for A + 0) +
        (composition-one-step-reachable-merkle-error-for A + 0) +
        (composition-one-step-reachable-merkle-error-for A + 0 + 0) +
        composition-one-step-reachable-merkle-error-for A) + 0 + 0)
  apply (rule
    checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-sampled-query-and-residual-gap
    [OF sampled])
  apply (rule wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
  apply (rule merkle-bound)
  apply assumption
  apply (rule wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero)
  apply (rule wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-merkle-and-slot)
  apply (rule merkle-bound)
  apply assumption
  apply (rule wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
  apply (rule wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-merkle-and-slot)
  apply (rule merkle-bound)
  apply assumption
  apply (rule wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
  apply (rule wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
  apply (rule merkle-bound)
  apply assumption
  apply (rule wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
  apply (rule wp-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero)
  apply (subst wp-composition-fri-verifier-tied-zero-round-final-obstruction-zero)
  apply simp
  done
  show ?thesis
    using branch
    by (simp add: composition-one-step-verifier-tied-error-for-def)
qed

```

```

lemma trace-empty-one-step-error-for-bound:
  fixes empty-bad :: nat ⇒ 'f list ⇒ 'f set
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and empty-subset:
     $\bigwedge i \text{ prefix.}$ 
     $i < \text{fri-round-count-for-degree-bound } (\text{clength} - 1) \implies$ 
     $\text{length prefix} = i \implies$ 
     $\text{empty-bad } i \text{ prefix} \subseteq$ 

```

```

    fri-one-step-disagreement-challenges
      (trace-fri-canonical-layer-len i) (empty-pw i)
      (empty-layer i prefix) (empty-claimed i prefix)
      (empty-dom i prefix)
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-empty-composition-header-candidates)
      adversary-initial-state
    ≤ trace-empty-one-step-error-for budgets A empty-bad
  proof -
  have sampled:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-sampled-query-bad-candidate)
      adversary-initial-state
    ≤ trace-one-step-sampled-error-for budgets empty-bad
  by (rule trace-one-step-sampled-error-for-bound
      [OF wf controlled empty-subset])
  have conflict-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-sampled-assignment-conflict)
      adversary-initial-state
    ≤ reachable-verifier-event-bound-for A
      trace-fri-sampled-assignment-conflict
  proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg \text{trace-fri-sampled-assignment-conflict } s \text{ None}$ 
  unfolding trace-fri-sampled-assignment-conflict-def
    trace-fri-partial-candidate-opening-evidence-def
    accepted-fri-opening-transcript-def
  by simp
  next
  fix data attacker-state
  assume support:  $\text{Some } (data, \text{attacker-state}) \in \text{set-dist}$ 
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  show wp-event verify-monad
    (trace-fri-sampled-assignment-conflict
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    ≤ reachable-verifier-event-bound-for A
      trace-fri-sampled-assignment-conflict
  by (rule reachable-verifier-event-bound-for-ge[OF support])
qed
have zero-bound:

```


$wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-zero-round-final-obstruction)
adversary-initial-state
 $\leq$ *trace-fri-zero-round-agreement-set-target-error-for budgets* +
trace-fri-controlled-merkle-error-for budgets +
reachable-verifier-event-bound-for A
trace-fri-bad-with-header-tied-partial-candidate +
 (*trace-fri-controlled-merkle-error-for budgets* +
 (*reachable-verifier-event-bound-for A*
trace-fri-header-tied-low-candidate-untied-root-gap +
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-untied-query-gap)) +
reachable-verifier-event-bound-for A
trace-fri-bad-with-header-tied-partial-candidate
by (*rule checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets*
[OF wf controlled])
have *branch*:
 $wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state
 $\leq$ *trace-one-step-sampled-error-for budgets empty-bad* +
 (*reachable-verifier-event-bound-for A*
trace-fri-sampled-assignment-conflict +
trace-fri-zero-round-agreement-set-target-error-for budgets +
trace-fri-controlled-merkle-error-for budgets +
reachable-verifier-event-bound-for A
trace-fri-bad-with-header-tied-partial-candidate +
 (*trace-fri-controlled-merkle-error-for budgets* +
 (*reachable-verifier-event-bound-for A*
trace-fri-header-tied-low-candidate-untied-root-gap +
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-untied-query-gap)) +
reachable-verifier-event-bound-for A
trace-fri-bad-with-header-tied-partial-candidate)
proof –
have *raw*:
 $wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state
 $\leq$ *trace-one-step-sampled-error-for budgets empty-bad* +
 (*reachable-verifier-event-bound-for A*
trace-fri-sampled-assignment-conflict +
 (*trace-fri-zero-round-agreement-set-target-error-for budgets* +
trace-fri-controlled-merkle-error-for budgets +
reachable-verifier-event-bound-for A
trace-fri-bad-with-header-tied-partial-candidate +

```

    (trace-fri-controlled-merkle-error-for budgets +
    (reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-root-gap +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-query-gap)) +
    reachable-verifier-event-bound-for A
    trace-fri-bad-with-header-tied-partial-candidate))
  by (rule
    checked-staged-security-trace-fri-empty-header-bound-from-staged-sampled-conflict-and-zero
    [OF sampled conflict-bound zero-bound])
  show ?thesis
    by (rule order-trans[OF raw]) (simp add: algebra-simps)
qed
show ?thesis
  using branch
  by (simp add: trace-empty-one-step-error-for-def
    trace-fri-empty-small-conflict-zero-error-for-def)
qed

lemma trace-header-one-step-error-for-bound:
  fixes trace-bad :: nat  $\Rightarrow$  'f list  $\Rightarrow$  'f set
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-subset:
     $\bigwedge i$  prefix.
     $i < \text{fri-round-count-for-degree-bound } (\text{clength} - 1) \implies$ 
     $\text{length prefix} = i \implies$ 
     $\text{trace-bad } i \text{ prefix} \subseteq$ 
    fri-one-step-disagreement-challenges
    (trace-fri-canonical-layer-len i) (empty-pw i)
    (empty-layer i prefix) (empty-claimed i prefix)
    (empty-dom i prefix)
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate)
    adversary-initial-state
     $\leq$  trace-header-one-step-error-for budgets A trace-bad
proof -
  have sampled-query:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
    trace-fri-sampled-query-bad-candidate)
    adversary-initial-state
     $\leq$  trace-one-step-sampled-error-for budgets trace-bad
  by (rule trace-one-step-sampled-error-for-bound
    [OF wf controlled trace-subset])
  have sampled-plain:
    wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-sampled-layer-chain)
  adversary-initial-state
≤ trace-one-step-sampled-error-for budgets trace-bad
by (rule
   checked-staged-security-trace-fri-sampled-layer-chain-bound-from-sampled-query
   [OF sampled-query])
have sampled-header:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-header-tied-sampled-layer-chain)
  adversary-initial-state
≤ trace-one-step-sampled-error-for budgets trace-bad
proof (rule order-trans[OF - sampled-plain])
show wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-bad-with-header-tied-sampled-layer-chain)
  adversary-initial-state
≤ wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-bad-with-sampled-layer-chain)
  adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
   dest: trace-fri-header-tied-sampled-layer-chain-imp-sampled-layer-chain
   split: option.splits prod.splits)
qed
have conflict-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-header-tied-sampled-assignment-conflict)
  adversary-initial-state
≤ trace-fri-header-replay-conflict-error-for A
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
                  adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
let ?M =
  reachable-verifier-event-bound-for A partial-merkle-inconsistency-bad
let ?H =
  reachable-verifier-event-bound-for A
  trace-fri-header-tied-recorded-sibling-candidate-missing
have merkle-bound:
  wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s

```

$\leq ?M$
by (rule reachable-verifier-event-bound-for-ge[OF support])
have recorded-missing-bound:
 wp-event verify-monad
 (trace-fri-header-tied-recorded-sibling-candidate-missing ?s) ?s
 $\leq ?H$
by (rule reachable-verifier-event-bound-for-ge[OF support])
have head-neq:
 wp-event verify-monad
 (trace-fri-header-tied-base-head-value-neq-gap ?s) ?s $\leq ?M$
by (rule
 wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
 (rule merkle-bound)
have head-mismatch:
 wp-event verify-monad
 (trace-fri-header-tied-base-head-value-mismatch-gap ?s) ?s
 $\leq ?M$
by (rule
 wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
 [OF head-neq])
have alignment:
 wp-event verify-monad
 (trace-fri-header-tied-base-value-alignment-gap ?s) ?s $\leq ?M$
by (rule
 wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
 [OF head-mismatch])
have next-bound:
 wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap ?s) ?s $\leq ?M + 0$
by (rule
 wp-trace-fri-header-tied-next-value-replay-gap-bound-from-merkle-and-slot)
 (rule merkle-bound,
 rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have successor:
 wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap ?s) ?s $\leq ?M + 0$
by (rule
 wp-trace-fri-header-tied-successor-replay-gap-bound-from-merkle-and-slot)
 (rule merkle-bound,
 rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have final:
 wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap ?s) ?s $\leq ?M + 0 + 0$
by (rule
 wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
 (rule merkle-bound,
 rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero,
 rule wp-trace-fri-header-tied-final-value-selected-step-missing-gap-zero)
have slot:

```

wp-event verify-monad
  (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap ?s) ?s
  ≤ 0
by (rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have authenticated-recorded-auth:
  wp-event verify-monad
    (trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap ?s)
    ?s ≤ 0
  by (rule
    wp-trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap
    [OF slot])
have recorded-auth:
  wp-event verify-monad
    (trace-fri-header-tied-same-layer-recorded-auth-gap ?s) ?s ≤ 0
  by (rule
    wp-trace-fri-header-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap
    [OF authenticated-recorded-auth])
have same-auth:
  wp-event verify-monad
    (trace-fri-header-tied-same-layer-auth-gap ?s) ?s ≤ 0
  by (rule
    wp-trace-fri-header-tied-same-layer-auth-gap-bound-from-recorded-auth-gap
    [OF recorded-auth])
have auth-gap:
  wp-event verify-monad
    (trace-fri-header-tied-assignment-auth-gap ?s) ?s ≤ 0
  by (rule
    wp-trace-fri-header-tied-assignment-auth-gap-bound-from-same-layer-auth-gap
    [OF same-auth])
have complement:
  wp-event verify-monad
    (λout.
      trace-fri-header-tied-sampled-assignment-conflict ?s out ∧
      ¬ trace-fri-header-tied-recorded-sibling-candidate-missing ?s out)
    ?s ≤
    (((?M + ?M) + (?M + 0) + (?M + 0) + (?M + 0 + 0)) + ?M) + 0)
  by (rule
    wp-trace-fri-header-tied-sampled-assignment-conflict-without-recorded-missing-bound-from-auth-gap-and-replay-
    (rule auth-gap, rule merkle-bound, rule alignment,
      rule next-bound, rule successor, rule final))
show wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict ?s) ?s
  ≤ trace-fri-header-replay-conflict-error-for A
unfolding trace-fri-header-replay-conflict-error-for-def Let-def
by (rule
  wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-recorded-missing-and-complement)
  (rule recorded-missing-bound, rule complement)
next
fix s

```

```

show  $\neg$  trace-fri-header-tied-sampled-assignment-conflict s None
  unfolding trace-fri-header-tied-sampled-assignment-conflict-def
    accepted-fri-opening-transcript-def
  by simp
qed
have zero-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-zero-round-final-obstruction)
    adversary-initial-state
 $\leq$  trace-fri-zero-round-agreement-set-target-error-for budgets +
  trace-fri-controlled-merkle-error-for budgets
proof  $-$ 
  have checked:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-zero-round-checked-final-obstruction)
      adversary-initial-state
     $\leq$  trace-fri-zero-round-agreement-set-target-error-for budgets
  by (rule checked-staged-security-trace-fri-zero-round-checked-final-obstruction-bound-from-agreement-sets
    [OF wf controlled])
  have merkle:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        partial-merkle-inconsistency-bad)
      adversary-initial-state
     $\leq$  trace-fri-controlled-merkle-error-for budgets
  unfolding trace-fri-controlled-merkle-error-for-def
  by (rule
    checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
    [OF wf controlled])
  have split:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-header-tied-zero-round-final-obstruction)
      adversary-initial-state
     $\leq$  wp-event (checked-staged-security-experiment-with-data-state A)
      ( $\lambda out.$ 
        staged-security-with-data-state-verifier-event
          trace-fri-zero-round-checked-final-obstruction out  $\vee$ 
          staged-security-with-data-state-verifier-event
            partial-merkle-inconsistency-bad out)
        adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def
  by (rule wp-event-mono-on-support)
    (auto dest:
      trace-fri-header-tied-zero-round-final-obstruction-imp-checked-or-merkle
      split: option.splits prod.splits)
also have ...  $\leq$ 

```

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-zero-round-checked-final-obstruction)
  adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad)
  adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
  trace-fri-zero-round-agreement-set-target-error-for budgets +
  trace-fri-controlled-merkle-error-for budgets
by (rule add-mono[OF checked merkle])
finally show ?thesis .
qed
have branch:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state
≤ trace-one-step-sampled-error-for budgets trace-bad +
  (trace-fri-header-replay-conflict-error-for A +
    (trace-fri-zero-round-agreement-set-target-error-for budgets +
      trace-fri-controlled-merkle-error-for budgets))
by (rule
  checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-conflict-and-staged-zero
    [OF sampled-header conflict-bound zero-bound])
show ?thesis
using branch
by (simp add: trace-header-one-step-error-for-def
  trace-fri-header-replay-conflict-zero-error-for-def)
qed

definition trace-empty-header-refined-zero-error-for
where
  trace-empty-header-refined-zero-error-for budgets A empty-bad =
    (trace-fri-zero-round-agreement-set-target-error-for budgets +
      trace-fri-controlled-merkle-error-for budgets +
      (trace-header-one-step-error-for budgets A empty-bad +
        (trace-fri-controlled-merkle-error-for budgets +
          (reachable-verifier-event-bound-for A
            trace-fri-header-tied-low-candidate-untied-root-gap +
            reachable-verifier-event-bound-for A
              trace-fri-header-tied-low-candidate-untied-query-gap)))) +
      trace-header-one-step-error-for budgets A empty-bad)

definition trace-empty-header-refined-one-step-error-for
where
  trace-empty-header-refined-one-step-error-for budgets A empty-bad =

```

trace-one-step-sampled-error-for budgets empty-bad +
(reachable-verifier-event-bound-for A
trace-fri-sampled-assignment-conflict +
trace-empty-header-refined-zero-error-for budgets A empty-bad)

definition *concrete-trace-empty-header-refined-one-step-error-for*
where

concrete-trace-empty-header-refined-one-step-error-for budgets A =
trace-empty-header-refined-one-step-error-for budgets A
trace-one-step-split-bad

lemma *checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets-and-header:*
fixes $H :: \text{prob}$

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *header-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state $\leq H$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-zero-round-final-obstruction)
adversary-initial-state $\leq$
trace-fri-zero-round-agreement-set-target-error-for budgets +
trace-fri-controlled-merkle-error-for budgets +
(H +
(trace-fri-controlled-merkle-error-for budgets +
(reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-untied-root-gap +
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-untied-query-gap))) +
H

proof –

have *checked:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-zero-round-checked-final-obstruction)
adversary-initial-state $\leq$
trace-fri-zero-round-agreement-set-target-error-for budgets

by (rule

checked-staged-security-trace-fri-zero-round-checked-final-obstruction-bound-from-agreement-sets
[OF wf controlled])

have *merkle:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state $\leq$

trace-fri-controlled-merkle-error-for budgets
unfolding *trace-fri-controlled-merkle-error-for-def*
by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[OF wf controlled])
have *hash*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
hash-map-output-collision-bad)
adversary-initial-state ≤
trace-fri-controlled-merkle-error-for budgets
unfolding *trace-fri-controlled-merkle-error-for-def*
by (rule
checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
[OF wf controlled])
have *root*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-header-tied-low-candidate-untied-root-gap)
adversary-initial-state ≤
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-untied-root-gap
proof (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
show $\bigwedge s. \neg \text{trace-fri-header-tied-low-candidate-untied-root-gap } s \text{ None}$
unfolding *trace-fri-header-tied-low-candidate-untied-root-gap-def*
trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
accepted-fri-opening-transcript-def
by *simp*
next
fix *data attacker-state*
assume *support: Some (data, attacker-state) ∈*
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
show *wp-event verify-monad*
(trace-fri-header-tied-low-candidate-untied-root-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
 $\leq$ *reachable-verifier-event-bound-for A*
trace-fri-header-tied-low-candidate-untied-root-gap
by (rule *reachable-verifier-event-bound-for-ge[OF support]*)
qed
have *query*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-header-tied-low-candidate-untied-query-gap)
adversary-initial-state ≤

```

    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-query-gap
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg \text{trace-fri-header-tied-low-candidate-untied-query-gap } s \text{ None}$ 
    unfolding trace-fri-header-tied-low-candidate-untied-query-gap-def
      trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
      accepted-fri-opening-transcript-def
    by simp
next
  fix data attacker-state
  assume support: Some (data, attacker-state)  $\in$ 
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  show wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-query-gap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
     $\leq$  reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-untied-query-gap
    by (rule reachable-verifier-event-bound-for-ge[OF support])
qed
let ?M = checked-staged-security-experiment-with-data-state A
let ?untied =
  staged-security-with-data-state-verifier-event
  trace-fri-header-tied-low-candidate-untied-reachable-witness-gap
let ?hash =
  staged-security-with-data-state-verifier-event
  hash-map-output-collision-bad
let ?root =
  staged-security-with-data-state-verifier-event
  trace-fri-header-tied-low-candidate-untied-root-gap
let ?query =
  staged-security-with-data-state-verifier-event
  trace-fri-header-tied-low-candidate-untied-query-gap
have untied:
  wp-event ?M ?untied adversary-initial-state  $\leq$ 
    trace-fri-controlled-merkle-error-for budgets +
    (reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-untied-root-gap +
      reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-untied-query-gap)
proof -
  have wp-event ?M ?untied adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda \text{out. ?hash out} \vee \text{?root out} \vee \text{?query out}$ )
      adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def

```

```

by (rule wp-event-mono-on-support)
  (auto
    dest:
      trace-fri-header-tied-low-candidate-untied-gap-imp-hash-root-or-query
      split: option.splits prod.splits)
also have ... ≤
  wp-event ?M ?hash adversary-initial-state +
  wp-event ?M (λout. ?root out ∨ ?query out)
  adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
  wp-event ?M ?hash adversary-initial-state +
  (wp-event ?M ?root adversary-initial-state +
    wp-event ?M ?query adversary-initial-state)
by (intro add-mono order-refl wp-event-union-bound)
also have ... ≤
  trace-fri-controlled-merkle-error-for budgets +
  (reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-root-gap +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-query-gap)
by (intro add-mono hash root query)
finally show ?thesis .
qed
have transfer:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-header-tied-candidate-transfer-gap)
  adversary-initial-state ≤
  H +
  (trace-fri-controlled-merkle-error-for budgets +
    (reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-untied-root-gap +
      reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-untied-query-gap))
by (rule
  checked-staged-security-trace-fri-header-tied-candidate-transfer-gap-bound-from-staged-header-and-untied
  [OF header-bound untied])
show ?thesis
by (rule
  checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-staged-checked-merkle-transfer-header
  [OF checked merkle transfer header-bound])
qed

lemma trace-empty-header-refined-one-step-error-for-bound:
  fixes empty-bad :: nat ⇒ 'f list ⇒ 'f set
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and empty-subset:

```

$\bigwedge i \text{ prefix.}$
 $i < \text{fri-round-count-for-degree-bound } (\text{clength} - 1) \implies$
 $\text{length prefix} = i \implies$
 $\text{empty-bad } i \text{ prefix} \subseteq$
 $\text{fri-one-step-disagreement-challenges}$
 $(\text{trace-fri-canonical-layer-len } i) (\text{empty-pw } i)$
 $(\text{empty-layer } i \text{ prefix}) (\text{empty-claimed } i \text{ prefix})$
 $(\text{empty-dom } i \text{ prefix})$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state}$
 $\leq \text{trace-empty-header-refined-one-step-error-for budgets } A \text{ empty-bad}$
proof –
have *sampled*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-sampled-query-bad-candidate})$
 $\text{adversary-initial-state}$
 $\leq \text{trace-one-step-sampled-error-for budgets empty-bad}$
by (rule *trace-one-step-sampled-error-for-bound*
 $[\text{OF } \text{wf controlled empty-subset}])$
have *conflict-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-sampled-assignment-conflict})$
 $\text{adversary-initial-state}$
 $\leq \text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-sampled-assignment-conflict}$
proof (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
show $\bigwedge s. \neg \text{trace-fri-sampled-assignment-conflict } s \text{ None}$
unfolding *trace-fri-sampled-assignment-conflict-def*
 $\text{trace-fri-partial-candidate-opening-evidence-def}$
 $\text{accepted-fri-opening-transcript-def}$
by *simp*
next
fix *data attacker-state*
assume *support*: $\text{Some } (data, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$
show $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $\leq \text{reachable-verifier-event-bound-for } A$

$\text{trace-fri-sampled-assignment-conflict}$
by (rule *reachable-verifier-event-bound-for-ge*[*OF support*])
qed
have *header-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
 $\text{trace-fri-bad-with-header-tied-partial-candidate}$)
 $\text{adversary-initial-state}$
 $\leq \text{trace-header-one-step-error-for budgets A empty-bad}$
by (rule *trace-header-one-step-error-for-bound*
[*OF wf controlled empty-subset*])
have *zero-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
 $\text{trace-fri-zero-round-final-obstruction}$)
 $\text{adversary-initial-state}$
 $\leq \text{trace-empty-header-refined-zero-error-for budgets A empty-bad}$
unfolding *trace-empty-header-refined-zero-error-for-def*
by (rule
 $\text{checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets-and-header}$
[*OF wf controlled header-bound*])
have *branch*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
 $\text{trace-fri-bad-with-empty-composition-header-candidates}$)
 $\text{adversary-initial-state}$
 $\leq \text{trace-one-step-sampled-error-for budgets empty-bad} +$
($\text{reachable-verifier-event-bound-for A}$
 $\text{trace-fri-sampled-assignment-conflict} +$
 $\text{trace-empty-header-refined-zero-error-for budgets A empty-bad}$)
by (rule
 $\text{checked-staged-security-trace-fri-empty-header-bound-from-staged-sampled-conflict-and-zero}$
[*OF sampled conflict-bound zero-bound*])
show *?thesis*
using *branch*
by (*simp add: trace-empty-header-refined-one-step-error-for-def*)
qed

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-one-step-fri-components*
fixes *trace-bad empty-bad* :: $\text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}$
and *comp-bad* :: $'f \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}$
assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-subset*:
 $\bigwedge i \text{ prefix.}$
 $i < \text{fri-round-count-for-degree-bound (clength - 1)} \implies$
 $\text{length prefix} = i \implies$
 $\text{trace-bad } i \text{ prefix} \subseteq$

fri-one-step-disagreement-challenges
 (trace-fri-canonical-layer-len *i*) (empty-pw *i*)
 (empty-layer *i* prefix) (empty-claimed *i* prefix)
 (empty-dom *i* prefix)

and *comp-subset*:

$\bigwedge dg \ i \text{ prefix.}$
 $i < \text{fri-round-count-for-degree-bound } (to\text{-nat } dg) \implies$
 $\text{length prefix} = i \implies$
 $\text{comp-bad } dg \ i \text{ prefix} \subseteq$
fri-one-step-disagreement-challenges
 (composition-fri-canonical-layer-len *dg i*) (comp-pw *dg i*)
 (comp-layer *dg i* prefix) (comp-claimed *dg i* prefix)
 (comp-dom *dg i* prefix)

and *empty-subset*:

$\bigwedge i \text{ prefix.}$
 $i < \text{fri-round-count-for-degree-bound } (clength - 1) \implies$
 $\text{length prefix} = i \implies$
 $\text{empty-bad } i \text{ prefix} \subseteq$
fri-one-step-disagreement-challenges
 (trace-fri-canonical-layer-len *i*) (empty-pw *i*)
 (empty-layer *i* prefix) (empty-claimed *i* prefix)
 (empty-dom *i* prefix)

shows checked-staged-adversary-acceptance-probability $A \leq$

hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 ($\sum i < \text{rounds.}$
 active-route-current-prefix-authenticated-error-for $A \ i$ +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 trace-header-one-step-error-for budgets A trace-bad +
 composition-one-step-verifier-tied-controlled-error-for budgets
 comp-bad +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +

staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
trace-empty-header-refined-one-step-error-for budgets A empty-bad +
active-route-current-empty-query-error-for A)

proof –

have *trace-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state
 $\leq$ *trace-header-one-step-error-for budgets A trace-bad*

by (rule *trace-header-one-step-error-for-bound*
 $[OF\ wf\ controlled\ trace-subset]$)

have *comp-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state
 $\leq$ *composition-one-step-verifier-tied-controlled-error-for budgets*
comp-bad

by (rule *composition-one-step-verifier-tied-controlled-error-for-bound*
 $[OF\ wf\ controlled\ comp-subset]$)

have *empty-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state
 $\leq$ *trace-empty-header-refined-one-step-error-for budgets A empty-bad*

by (rule *trace-empty-header-refined-one-step-error-for-bound*
 $[OF\ wf\ controlled\ empty-subset]$)

show *?thesis*

by (rule
stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-bounds
 $[OF\ false-statement\ wf\ controlled\ trace-bound\ comp-bound\ empty-bound]$)

qed

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-one-step-endpoint*:

assumes *false-statement: $\neg\ exists-valid-trace$*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows *checked-staged-adversary-acceptance-probability A $\leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +

hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
($\sum i < \text{rounds}$.
active-route-current-prefix-authenticated-error-for A i +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
concrete-trace-header-one-step-error-for budgets A +
concrete-composition-one-step-error-for budgets A +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
concrete-trace-empty-header-refined-one-step-error-for budgets A +
active-route-current-empty-query-error-for A)
unfolding *concrete-trace-header-one-step-error-for-def*
concrete-composition-one-step-error-for-def
concrete-trace-empty-header-refined-one-step-error-for-def
by (rule
stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-one-step-fri-components
[**where** *trace-bad = trace-one-step-split-bad*
and *comp-bad = composition-one-step-split-bad*
and *empty-bad = trace-one-step-split-bad*
and *empty-pw = trace-one-step-split-pw*
and *empty-layer = trace-one-step-split-layer*
and *empty-claimed = trace-one-step-split-claimed*
and *empty-dom = trace-one-step-split-domain*
and *comp-pw = composition-one-step-split-pw*
and *comp-layer = composition-one-step-split-layer*
and *comp-claimed = composition-one-step-split-claimed*
and *comp-dom = composition-one-step-split-domain,*
OF false-statement wf controlled])
(simp-all add:
trace-one-step-split-bad-def
composition-one-step-split-bad-def
trace-one-step-disagreement-bad-for-def
composition-one-step-disagreement-bad-for-def)

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-sharpened:*

assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
shows *checked-staged-adversary-acceptance-probability* *A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget *budgets* +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error *size*
(staged-alpha-search-queries *budgets* 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries *budgets* 0) +
staged-phase-relation-error *size*
(staged-alpha-search-queries *budgets* 0) +
(active-route-transcript-pre-error-for *A* +
staged-concrete-transcript-target-error-bound)) +
($\sum i < \text{rounds}$.
active-route-current-prefix-authenticated-error-for *A* *i* +
(active-route-transcript-pre-error-for *A* +
staged-concrete-transcript-target-error-bound) +
(active-route-transcript-pre-error-for *A* +
staged-concrete-transcript-target-error-bound)) +
sharpened-trace-fri-header-error-for *budgets* *A* +
sharpened-composition-fri-error-for *budgets* *A* +
(composition-error-bound +
staged-phase-relation-error *size*
(staged-alpha-search-queries *budgets* 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries *budgets* 0) +
staged-phase-relation-error *size*
(staged-alpha-search-queries *budgets* 0) +
(active-route-transcript-pre-error-for *A* +
staged-concrete-transcript-target-error-bound) +
sharpened-trace-fri-empty-error-for *budgets* *A* +
active-route-current-empty-query-error-for *A*)
by (rule
stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-bounds
[OF *false-statement* *wf* *controlled*
sharpened-trace-fri-header-error-for-bound[OF *wf* *controlled*]
sharpened-composition-fri-error-for-bound[OF *wf* *controlled*]
sharpened-trace-fri-empty-error-for-bound[OF *wf* *controlled*]])

theorem *stark-soundness-concrete-fri-sharpened-endpoint*:

assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
shows *checked-staged-adversary-acceptance-probability* *A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget *budgets* +

```

    staged-challenge-query-budget + verifier-hash-query-budget) +
  (composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    (active-route-transcript-pre-error-for A +
      staged-concrete-transcript-target-error-bound)) +
  (Σ i < rounds.
    active-route-current-prefix-authenticated-error-for A i +
    (active-route-transcript-pre-error-for A +
      staged-concrete-transcript-target-error-bound) +
    (active-route-transcript-pre-error-for A +
      staged-concrete-transcript-target-error-bound)) +
  concrete-trace-header-one-step-error-for budgets A +
  concrete-composition-one-step-error-for budgets A +
  (composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    (active-route-transcript-pre-error-for A +
      staged-concrete-transcript-target-error-bound) +
    concrete-trace-empty-header-refined-one-step-error-for budgets A +
    active-route-current-empty-query-error-for A)
  by (rule
    stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-one-step-endpoint
    [OF false-statement wf controlled])

end

hide-fact
  soundness.stark-soundness-concrete-fri-sharpened-endpoint
  soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-sharpened

end

```

```

theory Soundness-FRI-Canonical-Full-Cover-Failure
  imports Soundness-FRI-Sampled-Challenge-Cover
begin

```

Canonical full-cover failure events for sampled-query FRI.

The broad sampled full-cover failure predicates quantify over arbitrary domain witnesses. The sampled-query split, however, produces the failure on the canonical FRI domains.

This layer names that narrower event so later bounds do not have to prove facts about unused arbitrary domain entries.

context *soundness*

begin

definition *trace-fri-sampled-canonical-full-cover-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-sampled-canonical-full-cover-failure s bad out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{ trace-table layers.}$
trace-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{ trace-bs trace-final dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{ trace-table} \wedge$
generic-fri-sampled-query-candidate-evidence trace-table-low-degree
 $(\text{Not} \circ \text{trace-table-low-degree}) \text{ trace-table} (\text{clength} - 1)$
 $\text{ trace-roots trace-bs trace-final fri-query-idxs trace-round-layers}$
 $\text{ layers} \wedge$
generic-fri-sampled-full-cover-failure trace-table-low-degree
 $\text{ trace-table} (\text{clength} - 1) \text{ trace-roots trace-bs trace-final}$
 $\text{ fri-query-idxs trace-round-layers}$
 $(\text{fri-canonical-domains} (\text{length trace-bs})) \text{ layers}$
 $(\text{bad trace-table}))$

definition *composition-fri-sampled-canonical-full-cover-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-canonical-full-cover-failure s bad out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{ opening-composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{ f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table layers.}$
composition-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{ trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{ composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings trace-table}$

composition-table $\wedge$
generic-fri-sampled-query-candidate-evidence
(composition-table-low-degree (to-nat fri-dg))
(Not $\circ$ composition-table-low-degree maxDegree) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers layers $\wedge$
generic-fri-sampled-full-cover-failure
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
(fri-canonical-domains (length composition-bs)) layers
(bad fri-dg composition-table))

lemma *trace-fri-sampled-canonical-full-cover-failure-not-None*[simp]:

$\neg$ *trace-fri-sampled-canonical-full-cover-failure s bad None*

unfolding *trace-fri-sampled-canonical-full-cover-failure-def*

trace-fri-partial-candidate-opening-evidence-def

accepted-fri-opening-transcript-def

by *simp*

lemma *composition-fri-sampled-canonical-full-cover-failure-not-None*[simp]:

$\neg$ *composition-fri-sampled-canonical-full-cover-failure s bad None*

unfolding *composition-fri-sampled-canonical-full-cover-failure-def*

composition-fri-partial-candidate-opening-evidence-def

accepted-fri-opening-transcript-def

by *simp*

lemma *trace-fri-sampled-canonical-full-cover-failure-imp-broad*:

assumes *trace-fri-sampled-canonical-full-cover-failure s bad out*

shows *trace-fri-sampled-full-cover-failure s bad out*

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr candidate-query-idxs trace-openings

trace-table layers

where *evidence*:

trace-fri-partial-candidate-opening-evidence s out trace-roots

trace-bs trace-final dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers fr candidate-query-idxs trace-openings

trace-table

and *candidate*:

generic-fri-sampled-query-candidate-evidence trace-table-low-degree

(Not $\circ$ trace-table-low-degree) trace-table (clength — 1)

trace-roots trace-bs trace-final fri-query-idxs trace-round-layers

layers

and *failure*:

generic-fri-sampled-full-cover-failure trace-table-low-degree

trace-table (clength — 1) trace-roots trace-bs trace-final

```

    fri-query-idxs trace-round-layers
    (fri-canonical-domains (length trace-bs)) layers
    (bad trace-table)
  unfolding trace-fri-sampled-canonical-full-cover-failure-def by blast
have canonical:
  generic-fri-canonical-sampled-layer-chain-evidence trace-table-low-degree
  trace-table (clength - 1) trace-roots trace-bs trace-final
  fri-query-idxs trace-round-layers layers
  by (rule generic-fri-sampled-query-candidate-evidenceD(1)[OF candidate])
have sampled:
  generic-fri-sampled-layer-chain-evidence trace-table-low-degree
  trace-table (clength - 1) trace-roots trace-bs trace-final
  fri-query-idxs trace-round-layers
  (fri-canonical-domains (length trace-bs)) layers
  by (rule generic-fri-canonical-sampled-layer-chain-evidenceD
    [OF canonical])
show ?thesis
  unfolding trace-fri-sampled-full-cover-failure-def
  using evidence sampled failure by blast
qed

lemma composition-fri-sampled-canonical-full-cover-failure-imp-broad:
  assumes composition-fri-sampled-canonical-full-cover-failure s bad out
  shows composition-fri-sampled-full-cover-failure s bad out
proof -
  from assms obtain trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table layers
  where evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  and candidate:
    generic-fri-sampled-query-candidate-evidence
    (composition-table-low-degree (to-nat fri-dg))
    (Not o composition-table-low-degree maxDegree) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers layers
  and failure:
    generic-fri-sampled-full-cover-failure
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs

```

```

    composition-final fri-query-idxs composition-round-layers
    (fri-canonical-domains (length composition-bs)) layers
    (bad fri-dg composition-table)
unfolding composition-fri-sampled-canonical-full-cover-failure-def
by meson
have canonical:
  generic-fri-canonical-sampled-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers layers
  by (rule generic-fri-sampled-query-candidate-evidenceD(1)[OF candidate])
have sampled:
  generic-fri-sampled-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
    (fri-canonical-domains (length composition-bs)) layers
  by (rule generic-fri-canonical-sampled-layer-chain-evidenceD
    [OF canonical])
show ?thesis
  unfolding composition-fri-sampled-full-cover-failure-def
  using evidence sampled failure by meson
qed

lemma generic-fri-canonical-sampled-layer-chain-next-idx-current-bound:
assumes chain:
  generic-fri-sampled-layer-chain-evidence low-degree candidate-table
  degree-bound roots challenges final-value query-idxs round-layers
  (fri-canonical-domains (length challenges)) layers
and round-bound: round-idx < length query-idxs
and layer-bound: layer-idx < length challenges
shows
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
  length (fri-canonical-domain-at layer-idx)
proof –
have fold:
  fri-sampled-table-fold (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-canonical-domains (length challenges) ! layer-idx)
    (layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  by (rule generic-fri-sampled-layer-chain-evidence-sample(2)
    [OF chain round-bound layer-bound])
from fold obtain xp xp-path xn xn-path where match:
  fri-opening-matches-table

```

```

    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (layers ! layer-idx) xp xn
  by (elim fri-sampled-table-foldE)
have partial:
  generic-fri-partial-evidence low-degree candidate-table degree-bound
  roots challenges final-value query-idxs round-layers
  by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
have roots-len: length challenges = length roots
  by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
have roots-bound: layer-idx < length roots
  using layer-bound roots-len by simp
have len-eq:
  fri-evidence-layer-len roots layer-idx =
    length (fri-canonical-domain-at layer-idx)
  unfolding fri-evidence-layer-len-def fri-canonical-domain-at-length
  using fri-layer-lengths-nth-div[OF roots-bound, of clength * scale]
  by simp
have raw-bound:
  fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
    length (fri-canonical-domain-at layer-idx)
  using fri-opening-matches-tableD(1)[OF match] unfolding len-eq .
show ?thesis
proof (cases fri-evidence-layer-len roots layer-idx div 2 = 0)
  case True
  then show ?thesis
    using raw-bound unfolding fri-evidence-next-idx-def by simp
next
  case False
  have half-bound:
    fri-evidence-layer-len roots layer-idx div 2 ≤
      length (fri-canonical-domain-at layer-idx)
    unfolding len-eq by simp
  have
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
      fri-evidence-layer-len roots layer-idx div 2
    unfolding fri-evidence-next-idx-def
    by (rule mod-less-divisor) (use False in simp)
  then show ?thesis
    using half-bound by linarith
qed
qed

lemma generic-fri-canonical-sampled-layer-chain-next-idx-successor-bound:
  assumes chain:
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    (fri-canonical-domains (length challenges)) layers
  and no-shape:

```

```

    ¬ generic-fri-full-cover-shape-failure roots challenges
      (fri-canonical-domains (length challenges)) layers
  and round-bound: round-idx < length query-idxs
  and layer-bound: layer-idx < length roots
shows
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (fri-canonical-domains (length challenges) ! Suc layer-idx)
proof -
  have partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
      roots challenges final-value query-idxs round-layers
    by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
  have roots-len: length challenges = length roots
    by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
  have layer-challenge-bound: layer-idx < length challenges
    using layer-bound roots-len by simp
  have no-arith:
    ¬ generic-fri-full-cover-arithmetic-shape-failure roots challenges
    using no-shape
    unfolding generic-fri-full-cover-shape-failure-def
      generic-fri-full-cover-arithmetic-shape-failure-def
    by blast
  have even-len:
    2 dvd fri-evidence-layer-len roots layer-idx
    using no-arith layer-challenge-bound
    unfolding generic-fri-full-cover-arithmetic-shape-failure-def by blast
  have roots-bound: layer-idx < length roots
    by (rule layer-bound)
  have len-eq:
    fri-evidence-layer-len roots layer-idx =
      length (fri-canonical-domain-at layer-idx)
    unfolding fri-evidence-layer-len-def fri-canonical-domain-at-length
    using fri-layer-lengths-nth-div[OF roots-bound, of clength * scale]
    by simp
  have next-dom-eq:
    fri-canonical-domains (length challenges) ! Suc layer-idx =
      fri-canonical-domain-at (Suc layer-idx)
    by (rule fri-canonical-domains-nth)
      (use layer-challenge-bound in simp)
  have next-len:
    length (fri-canonical-domains (length challenges) ! Suc layer-idx) =
      fri-evidence-layer-len roots layer-idx div 2
proof -
  have length (fri-canonical-domains (length challenges) ! Suc layer-idx) =
    (clength * scale) div 2 ^ Suc layer-idx
    unfolding next-dom-eq fri-canonical-domain-at-length by simp
  also have ... = ((clength * scale) div 2 ^ layer-idx) div 2
    by (metis div-mult2-eq mult.commute power-Suc)
  also have ... = fri-evidence-layer-len roots layer-idx div 2

```



```

unfolding fri-evidence-layer-len-def
using fri-layer-lengths-nth-div[OF roots-bound, of clength * scale]
by simp
finally show ?thesis .
qed
have raw-current:
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (fri-canonical-domain-at layer-idx)
by (rule generic-fri-canonical-sampled-layer-chain-next-idx-current-bound
    [OF chain round-bound layer-challenge-bound])
have half-pos:
  0 < fri-evidence-layer-len roots layer-idx div 2
using raw-current even-len unfolding len-eq by auto
have
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    fri-evidence-layer-len roots layer-idx div 2
unfolding fri-evidence-next-idx-def
by (rule mod-less-divisor[OF half-pos])
then show ?thesis
unfolding next-len .
qed

lemma generic-fri-canonical-sampled-domain-failure-imp-shape:
assumes chain:
  generic-fri-sampled-layer-chain-evidence low-degree candidate-table
  degree-bound roots challenges final-value query-idxs round-layers
  (fri-canonical-domains (length challenges)) layers
and sampled-domain:
  generic-fri-full-cover-sampled-domain-failure roots query-idxs
  (fri-canonical-domains (length challenges))
shows
  generic-fri-full-cover-shape-failure roots challenges
  (fri-canonical-domains (length challenges)) layers
proof (rule ccontr)
assume no-shape:
   $\neg$  generic-fri-full-cover-shape-failure roots challenges
  (fri-canonical-domains (length challenges)) layers
have partial:
  generic-fri-partial-evidence low-degree candidate-table degree-bound
  roots challenges final-value query-idxs round-layers
by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
have roots-len: length challenges = length roots
by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
have roots-bound: length roots  $\leq$  length challenges
using roots-len by simp
have no-sampled:
   $\neg$  generic-fri-full-cover-sampled-domain-failure roots query-idxs
  (fri-canonical-domains (length challenges))
proof (rule fri-canonical-domains-no-sampled-domain-failure

```

```

    [OF roots-bound])
  fix round-idx layer-idx
  assume round-bound: round-idx < length query-idxs
    and layer-bound: layer-idx < length roots
  show fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (fri-canonical-domain-at layer-idx)
    by (rule generic-fri-canonical-sampled-layer-chain-next-idx-current-bound
      [OF chain round-bound])
      (use layer-bound roots-len in simp)
  next
  fix round-idx layer-idx
  assume round-bound: round-idx < length query-idxs
    and layer-bound: layer-idx < length roots
  show fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (fri-canonical-domains (length challenges) ! Suc layer-idx)
    by (rule
      generic-fri-canonical-sampled-layer-chain-next-idx-successor-bound
      [OF chain no-shape round-bound layer-bound])
  qed
  show False
    using sampled-domain no-sampled by contradiction
  qed

lemma generic-fri-canonical-domain-failure-imp-index-or-shape:
  assumes chain:
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    (fri-canonical-domains (length challenges)) layers
  and domain:
    generic-fri-full-cover-domain-failure roots challenges
    (fri-canonical-domains (length challenges))
  shows
    generic-fri-full-cover-index-failure roots query-idxs ∨
    generic-fri-full-cover-shape-failure roots challenges
    (fri-canonical-domains (length challenges)) layers
  proof (cases generic-fri-full-cover-index-failure roots query-idxs)
    case True
    then show ?thesis by simp
  next
  case no-index: False
  have partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
    by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
  have roots-len: length challenges = length roots
    by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
  have sampled-domain:
    generic-fri-full-cover-sampled-domain-failure roots query-idxs
    (fri-canonical-domains (length challenges))

```

using *generic-fri-full-cover-domain-failure-imp-index-or-sampled-domain*
 [OF roots-len domain, of query-idxs] no-index
by *blast*
then have *shape*:
generic-fri-full-cover-shape-failure roots challenges
(fri-canonical-domains (length challenges)) layers
by (rule *generic-fri-canonical-sampled-domain-failure-imp-shape*
 [OF chain])
then show ?thesis **by** *simp*
qed

lemma *trace-fri-sampled-canonical-full-cover-failure-split-no-domain*:
assumes *trace-fri-sampled-canonical-full-cover-failure s bad out*
shows

trace-fri-sampled-full-cover-proximity-failure s bad out $\vee$
trace-fri-sampled-full-cover-index-failure s out $\vee$
trace-fri-sampled-full-cover-shape-failure s out $\vee$
trace-fri-sampled-full-cover-start-low-failure s out

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table layers

where *evidence*:

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

and *candidate*:

generic-fri-sampled-query-candidate-evidence trace-table-low-degree
(Not $\circ$ trace-table-low-degree) trace-table (clength — 1)
trace-roots trace-bs trace-final fri-query-idxs trace-round-layers
layers

and *failure*:

generic-fri-sampled-full-cover-failure trace-table-low-degree
trace-table (clength — 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers
(fri-canonical-domains (length trace-bs)) layers
(bad trace-table)

unfolding *trace-fri-sampled-canonical-full-cover-failure-def* **by** *blast*

have *canonical*:

generic-fri-canonical-sampled-layer-chain-evidence
trace-table-low-degree trace-table (clength — 1) trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers layers

by (rule *generic-fri-sampled-query-candidate-evidenceD(1)*
 [OF candidate])

have *chain*:

generic-fri-sampled-layer-chain-evidence trace-table-low-degree

```

    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers
    (fri-canonical-domains (length trace-bs)) layers
  by (rule generic-fri-canonical-sampled-layer-chain-evidenceD
      [OF canonical])
have cases:
  generic-fri-full-cover-proximity-failure (clength - 1) trace-bs
  (fri-canonical-domains (length trace-bs)) layers
  (bad trace-table) ∨
  generic-fri-full-cover-index-failure trace-roots fri-query-idxs ∨
  generic-fri-full-cover-shape-failure trace-roots trace-bs
  (fri-canonical-domains (length trace-bs)) layers ∨
  generic-fri-full-cover-domain-failure trace-roots trace-bs
  (fri-canonical-domains (length trace-bs)) ∨
  generic-fri-full-cover-start-low-failure (clength - 1)
  trace-table ∨
  generic-fri-full-cover-final-low-failure (clength - 1) trace-bs
  (fri-canonical-domains (length trace-bs)) layers
  by (rule generic-fri-sampled-full-cover-failure-cases[OF failure])
from cases show ?thesis
proof
  assume proximity:
    generic-fri-full-cover-proximity-failure (clength - 1) trace-bs
    (fri-canonical-domains (length trace-bs)) layers
    (bad trace-table)
  then show ?thesis
    unfolding trace-fri-sampled-full-cover-proximity-failure-def
    using evidence chain by blast
next
  assume rest:
    generic-fri-full-cover-index-failure trace-roots fri-query-idxs ∨
    generic-fri-full-cover-shape-failure trace-roots trace-bs
    (fri-canonical-domains (length trace-bs)) layers ∨
    generic-fri-full-cover-domain-failure trace-roots trace-bs
    (fri-canonical-domains (length trace-bs)) ∨
    generic-fri-full-cover-start-low-failure (clength - 1)
    trace-table ∨
    generic-fri-full-cover-final-low-failure (clength - 1) trace-bs
    (fri-canonical-domains (length trace-bs)) layers
  then show ?thesis
proof
  assume index:
    generic-fri-full-cover-index-failure trace-roots fri-query-idxs
  then show ?thesis
    unfolding trace-fri-sampled-full-cover-index-failure-def
    using evidence chain by blast
next
  assume rest':
    generic-fri-full-cover-shape-failure trace-roots trace-bs

```

```

    (fri-canonical-domains (length trace-bs)) layers ∨
    generic-fri-full-cover-domain-failure trace-roots trace-bs
    (fri-canonical-domains (length trace-bs)) ∨
    generic-fri-full-cover-start-low-failure (clength - 1)
    trace-table ∨
    generic-fri-full-cover-final-low-failure (clength - 1) trace-bs
    (fri-canonical-domains (length trace-bs)) layers
then show ?thesis
proof
  assume shape:
    generic-fri-full-cover-shape-failure trace-roots trace-bs
    (fri-canonical-domains (length trace-bs)) layers
  then show ?thesis
    unfolding trace-fri-sampled-full-cover-shape-failure-def
    using evidence chain by blast
next
  assume rest'':
    generic-fri-full-cover-domain-failure trace-roots trace-bs
    (fri-canonical-domains (length trace-bs)) ∨
    generic-fri-full-cover-start-low-failure (clength - 1)
    trace-table ∨
    generic-fri-full-cover-final-low-failure (clength - 1)
    trace-bs (fri-canonical-domains (length trace-bs)) layers
  then show ?thesis
proof
  assume domain:
    generic-fri-full-cover-domain-failure trace-roots trace-bs
    (fri-canonical-domains (length trace-bs))
  have index-or-shape:
    generic-fri-full-cover-index-failure trace-roots
    fri-query-idxs ∨
    generic-fri-full-cover-shape-failure trace-roots trace-bs
    (fri-canonical-domains (length trace-bs)) layers
  by (rule generic-fri-canonical-domain-failure-imp-index-or-shape
    [OF chain domain])
  then show ?thesis
proof
  assume generic-fri-full-cover-index-failure trace-roots
    fri-query-idxs
  then show ?thesis
    unfolding trace-fri-sampled-full-cover-index-failure-def
    using evidence chain by blast
next
  assume generic-fri-full-cover-shape-failure trace-roots
    trace-bs (fri-canonical-domains (length trace-bs)) layers
  then show ?thesis
    unfolding trace-fri-sampled-full-cover-shape-failure-def
    using evidence chain by blast
qed

```

```

next
  assume rest''':
    generic-fri-full-cover-start-low-failure (clength - 1)
    trace-table  $\vee$ 
    generic-fri-full-cover-final-low-failure (clength - 1)
    trace-bs (fri-canonical-domains (length trace-bs)) layers
  then show ?thesis
proof
  assume start:
    generic-fri-full-cover-start-low-failure (clength - 1)
    trace-table
  then show ?thesis
    unfolding trace-fri-sampled-full-cover-start-low-failure-def
    using evidence chain by blast
next
  assume final:
    generic-fri-full-cover-final-low-failure (clength - 1)
    trace-bs (fri-canonical-domains (length trace-bs)) layers
  have final-event:
    trace-fri-sampled-full-cover-final-low-failure s out
    unfolding trace-fri-sampled-full-cover-final-low-failure-def
    using evidence chain final by blast
  then have shape-event:
    trace-fri-sampled-full-cover-shape-failure s out
    by (rule trace-fri-sampled-full-cover-final-low-imp-shape)
  then show ?thesis by simp
qed
qed
qed
qed
qed
qed

lemma composition-fri-sampled-canonical-full-cover-failure-split-no-domain:
  assumes composition-fri-sampled-canonical-full-cover-failure s bad out
  shows
    composition-fri-sampled-full-cover-proximity-failure s bad out  $\vee$ 
    composition-fri-sampled-full-cover-index-failure s out  $\vee$ 
    composition-fri-sampled-full-cover-shape-failure s out  $\vee$ 
    composition-fri-sampled-full-cover-start-low-failure s out
proof -
  from assms obtain trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table layers
  where evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots

```

trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
and *candidate:*
generic-fri-sampled-query-candidate-evidence
(composition-table-low-degree (to-nat fri-dg))
(Not o composition-table-low-degree maxDegree) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers layers
and *failure:*
generic-fri-sampled-full-cover-failure
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
(fri-canonical-domains (length composition-bs)) layers
(bad fri-dg composition-table)
unfolding *composition-fri-sampled-canonical-full-cover-failure-def*
by *meson*
have *canonical:*
generic-fri-canonical-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers layers
by *(rule generic-fri-sampled-query-candidate-evidenceD(1)*
[OF candidate])
have *chain:*
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
(fri-canonical-domains (length composition-bs)) layers
by *(rule generic-fri-canonical-sampled-layer-chain-evidenceD*
[OF canonical])
have *cases:*
generic-fri-full-cover-proximity-failure (to-nat fri-dg)
composition-bs (fri-canonical-domains (length composition-bs))
layers (bad fri-dg composition-table) ∨
generic-fri-full-cover-index-failure opening-composition-roots
fri-query-idxs ∨
generic-fri-full-cover-shape-failure opening-composition-roots
composition-bs (fri-canonical-domains (length composition-bs))
layers ∨
generic-fri-full-cover-domain-failure opening-composition-roots
composition-bs (fri-canonical-domains (length composition-bs)) ∨
generic-fri-full-cover-start-low-failure (to-nat fri-dg)
composition-table ∨

```

    generic-fri-full-cover-final-low-failure (to-nat fri-dg) composition-bs
      (fri-canonical-domains (length composition-bs)) layers
  by (rule generic-fri-sampled-full-cover-failure-cases[OF failure])
from cases show ?thesis
proof
  assume proximity:
    generic-fri-full-cover-proximity-failure (to-nat fri-dg)
      composition-bs (fri-canonical-domains (length composition-bs))
      layers (bad fri-dg composition-table)
  then show ?thesis
    unfolding composition-fri-sampled-full-cover-proximity-failure-def
    using evidence chain by meson
next
assume rest:
  generic-fri-full-cover-index-failure opening-composition-roots
    fri-query-idxs ∨
  generic-fri-full-cover-shape-failure opening-composition-roots
    composition-bs (fri-canonical-domains (length composition-bs))
    layers ∨
  generic-fri-full-cover-domain-failure opening-composition-roots
    composition-bs (fri-canonical-domains (length composition-bs)) ∨
  generic-fri-full-cover-start-low-failure (to-nat fri-dg)
    composition-table ∨
  generic-fri-full-cover-final-low-failure (to-nat fri-dg)
    composition-bs (fri-canonical-domains (length composition-bs))
    layers
then show ?thesis
proof
  assume index:
    generic-fri-full-cover-index-failure opening-composition-roots
      fri-query-idxs
  then show ?thesis
    unfolding composition-fri-sampled-full-cover-index-failure-def
    using evidence chain by meson
next
assume rest':
  generic-fri-full-cover-shape-failure opening-composition-roots
    composition-bs (fri-canonical-domains (length composition-bs))
    layers ∨
  generic-fri-full-cover-domain-failure opening-composition-roots
    composition-bs (fri-canonical-domains (length composition-bs)) ∨
  generic-fri-full-cover-start-low-failure (to-nat fri-dg)
    composition-table ∨
  generic-fri-full-cover-final-low-failure (to-nat fri-dg)
    composition-bs (fri-canonical-domains (length composition-bs))
    layers
then show ?thesis
proof
  assume shape:

```



```

    generic-fri-full-cover-shape-failure opening-composition-roots
      composition-bs (fri-canonical-domains (length composition-bs))
      layers
  then show ?thesis
    unfolding composition-fri-sampled-full-cover-shape-failure-def
    using evidence chain by meson
next
assume rest'':
  generic-fri-full-cover-domain-failure opening-composition-roots
    composition-bs (fri-canonical-domains (length composition-bs)) ∨
  generic-fri-full-cover-start-low-failure (to-nat fri-dg)
    composition-table ∨
  generic-fri-full-cover-final-low-failure (to-nat fri-dg)
    composition-bs (fri-canonical-domains (length composition-bs))
    layers
  then show ?thesis
proof
  assume domain:
    generic-fri-full-cover-domain-failure opening-composition-roots
      composition-bs (fri-canonical-domains (length composition-bs))
  have index-or-shape:
    generic-fri-full-cover-index-failure opening-composition-roots
      fri-query-idxs ∨
    generic-fri-full-cover-shape-failure opening-composition-roots
      composition-bs (fri-canonical-domains (length composition-bs))
    layers
  by (rule generic-fri-canonical-domain-failure-imp-index-or-shape
      [OF chain domain])
  then show ?thesis
proof
  assume generic-fri-full-cover-index-failure
    opening-composition-roots fri-query-idxs
  then show ?thesis
    unfolding composition-fri-sampled-full-cover-index-failure-def
    using evidence chain by meson
next
  assume generic-fri-full-cover-shape-failure
    opening-composition-roots composition-bs
      (fri-canonical-domains (length composition-bs)) layers
  then show ?thesis
    unfolding composition-fri-sampled-full-cover-shape-failure-def
    using evidence chain by meson
qed
next
assume rest''':
  generic-fri-full-cover-start-low-failure (to-nat fri-dg)
    composition-table ∨
  generic-fri-full-cover-final-low-failure (to-nat fri-dg)
    composition-bs (fri-canonical-domains (length composition-bs))

```

```

    layers
  then show ?thesis
proof
  assume start:
    generic-fri-full-cover-start-low-failure (to-nat fri-dg)
    composition-table
  then show ?thesis
  unfolding
    composition-fri-sampled-full-cover-start-low-failure-def
  using evidence chain by meson
next
  assume final:
    generic-fri-full-cover-final-low-failure (to-nat fri-dg)
    composition-bs (fri-canonical-domains (length composition-bs))
    layers
  have final-event:
    composition-fri-sampled-full-cover-final-low-failure s out
  unfolding
    composition-fri-sampled-full-cover-final-low-failure-def
  using evidence chain final by meson
  then have shape-event:
    composition-fri-sampled-full-cover-shape-failure s out
  by (rule composition-fri-sampled-full-cover-final-low-imp-shape)
  then show ?thesis by simp
qed
qed
qed
qed
qed
qed
qed

lemma wp-trace-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases:
  assumes proximity:
    wp-event verify-monad
    (trace-fri-sampled-full-cover-proximity-failure s bad) s ≤ P
  and index:
    wp-event verify-monad
    (trace-fri-sampled-full-cover-index-failure s) s ≤ I
  and shape:
    wp-event verify-monad
    (trace-fri-sampled-full-cover-shape-failure s) s ≤ Sh
  shows
    wp-event verify-monad
    (trace-fri-sampled-canonical-full-cover-failure s bad) s
    ≤ P + I + Sh
proof -
  have wp-event verify-monad
    (trace-fri-sampled-canonical-full-cover-failure s bad) s
    ≤ wp-event verify-monad

```

($\lambda out.$
 $trace\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}proximity\text{-}failure\ s\ bad\ out \vee$
 $trace\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}index\text{-}failure\ s\ out \vee$
 $trace\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}shape\text{-}failure\ s\ out \vee$
 $trace\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}start\text{-}low\text{-}failure\ s\ out) \ s$
 by (rule *wp-event-mono*)
 (rule *trace-fri-sampled-canonical-full-cover-failure-split-no-domain*)
 also have ... $\leq$
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}proximity\text{-}failure\ s\ bad) \ s +$
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}index\text{-}failure\ s) \ s +$
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}shape\text{-}failure\ s) \ s +$
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}start\text{-}low\text{-}failure\ s) \ s$
 by (rule *wp-event-union-bound₄*)
 also have ... $\leq P + I + Sh + 0$
 using *proximity index shape*
 $wp\text{-}trace\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}start\text{-}low\text{-}failure\text{-}zero[of\ s]$
 by (*intro add-mono*)
 finally show *?thesis*
 by *simp*
 qed

lemma *wp-composition-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases:*
 assumes *proximity:*
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}proximity\text{-}failure\ s\ bad) \ s \leq P$
 and *index:*
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}index\text{-}failure\ s) \ s \leq I$
 and *shape:*
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}shape\text{-}failure\ s) \ s \leq Sh$
 shows
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}sampled\text{-}canonical\text{-}full\text{-}cover\text{-}failure\ s\ bad) \ s$
 $\leq P + I + Sh$

proof –
 have $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}sampled\text{-}canonical\text{-}full\text{-}cover\text{-}failure\ s\ bad) \ s$
 $\leq wp\text{-}event\ verify\text{-}monad$
 ($\lambda out.$
 $composition\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}proximity\text{-}failure\ s\ bad\ out \vee$
 $composition\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}index\text{-}failure\ s\ out \vee$
 $composition\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}shape\text{-}failure\ s\ out \vee$
 $composition\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}start\text{-}low\text{-}failure\ s\ out) \ s$
 by (rule *wp-event-mono*)

(rule
 composition-fri-sampled-canonical-full-cover-failure-split-no-domain)
 also have ... $\leq$
 wp-event verify-monad
 (composition-fri-sampled-full-cover-proximity-failure s bad) s +
 wp-event verify-monad
 (composition-fri-sampled-full-cover-index-failure s) s +
 wp-event verify-monad
 (composition-fri-sampled-full-cover-shape-failure s) s +
 wp-event verify-monad
 (composition-fri-sampled-full-cover-start-low-failure s) s
 by (rule wp-event-union-bound4)
 also have ... $\leq P + I + Sh + 0$
 using proximity index shape
 wp-composition-fri-sampled-full-cover-start-low-failure-zero[of s]
 by (intro add-mono)
 finally show ?thesis
 by simp
 qed

lemma wp-trace-fri-sampled-canonical-full-cover-failure-bound-from-witness-shape:

assumes proximity:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-proximity-failure s bad) s $\leq P$
 and index:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-index-failure s) s $\leq I$
 and witness:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-witness-length-failure s) s $\leq W$
 shows
 wp-event verify-monad
 (trace-fri-sampled-canonical-full-cover-failure s bad) s
 $\leq P + I + W$
 by (rule
 wp-trace-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases
 [OF proximity index])
 (rule wp-trace-fri-sampled-full-cover-shape-bound-from-witness-length
 [OF witness])

lemma wp-composition-fri-sampled-canonical-full-cover-failure-bound-from-witness-shape:

assumes proximity:
 wp-event verify-monad
 (composition-fri-sampled-full-cover-proximity-failure s bad) s $\leq P$
 and index:
 wp-event verify-monad
 (composition-fri-sampled-full-cover-index-failure s) s $\leq I$
 and witness:
 wp-event verify-monad

$(\text{composition-fri-sampled-full-cover-witness-length-failure } s) \ s$
 $\leq W$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-canonical-full-cover-failure } s \text{ bad}) \ s$
 $\leq P + I + W$
by (rule
 $\text{wp-composition-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases}$
 $[\text{OF proximity index}]$)
(rule
 $\text{wp-composition-fri-sampled-full-cover-shape-bound-from-witness-length}$
 $[\text{OF witness}]$)

lemma *checked-staged-security-trace-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases:*

assumes *proximity:*

$\wedge \text{data attacker-state.}$

$\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$

$\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-proximity-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq P$

and *index:*

$\wedge \text{data attacker-state.}$

$\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$

$\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$

and *shape:*

$\wedge \text{data attacker-state.}$

$\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$

$\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-shape-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq Sh$

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

```

    (staged-security-with-data-state-verifier-event
     (λs. trace-fri-sampled-canonical-full-cover-failure s bad))
    adversary-initial-state ≤ P + I + Sh
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show
    ∧s. ¬ trace-fri-sampled-canonical-full-cover-failure s bad None
  by simp
next
  fix data attacker-state
  assume support:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  show wp-event verify-monad
    (trace-fri-sampled-canonical-full-cover-failure ?s bad) ?s
    ≤ P + I + Sh
  by (rule
    wp-trace-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases)
    (rule proximity[OF support], rule index[OF support],
     rule shape[OF support])
qed

lemma checked-staged-security-trace-fri-sampled-canonical-full-cover-failure-bound-from-witness-shape:
  assumes proximity:
    ∧data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
        wp-event verify-monad
          (trace-fri-sampled-full-cover-proximity-failure
            (verifier-state-from-adversary attacker-state
              (staged-proof-transcript data)) bad)
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)) ≤ P
  and index:
    ∧data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
        wp-event verify-monad
          (trace-fri-sampled-full-cover-index-failure
            (verifier-state-from-adversary attacker-state
              (staged-proof-transcript data)))
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)) ≤ I
  and witness:

```

$\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-witness-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq W$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-sampled-canonical-full-cover-failure } s \text{ bad}))$
 $\text{adversary-initial-state} \leq P + I + W$
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
show
 $\wedge s. \neg \text{trace-fri-sampled-canonical-full-cover-failure } s \text{ bad None}$
by simp
next
fix data attacker-state
assume support:
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)}$
let ?s =
 $\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})$
show wp-event verify-monad
 $(\text{trace-fri-sampled-canonical-full-cover-failure ?s bad}) \text{ ?s}$
 $\leq P + I + W$
by (rule
 $\text{wp-trace-fri-sampled-canonical-full-cover-failure-bound-from-witness-shape})$
 $(\text{rule proximity[OF support]}, \text{rule index[OF support]},$
 $\text{rule witness[OF support]})$
qed

lemma checked-staged-security-composition-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases:

assumes proximity:

$\wedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$

$\text{set-dist (execute (checked-staged-transcript-program } A)$

$\text{adversary-initial-state)} \implies$

$\text{wp-event verify-monad}$

$(\text{composition-fri-sampled-full-cover-proximity-failure}$

$(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data})) \text{ bad})$

$(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data})) \leq P$

and *index*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$
and *shape*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-shape-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq Sh$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-sampled-canonical-full-cover-failure } s \text{ bad}))$
 $\text{adversary-initial-state} \leq P + I + Sh$
proof (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
show
 $\wedge s. \neg \text{composition-fri-sampled-canonical-full-cover-failure } s \text{ bad}$
 None
by *simp*
next
fix *data attacker-state*
assume *support*:
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)}$
let *?s* =
 $\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})$
show $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-canonical-full-cover-failure } ?s \text{ bad}) \text{ ?s}$
 $\leq P + I + Sh$
by (rule
 $\text{wp-composition-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases})$
 $(\text{rule proximity}[OF \text{ support}], \text{rule index}[OF \text{ support}],$
 $\text{rule shape}[OF \text{ support}])$
qed

lemma *checked-staged-security-composition-fri-sampled-canonical-full-cover-failure-bound-from-witness-shape*:

assumes *proximity*:

$\wedge$ *data attacker-state*.

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒

wp-event verify-monad
(composition-fri-sampled-full-cover-proximity-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad)
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ P

and *index*:

$\wedge$ *data attacker-state*.

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒

wp-event verify-monad
(composition-fri-sampled-full-cover-index-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ I

and *witness*:

$\wedge$ *data attacker-state*.

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒

wp-event verify-monad
(composition-fri-sampled-full-cover-witness-length-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ W

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
($\lambda s.$ composition-fri-sampled-canonical-full-cover-failure s bad))
adversary-initial-state ≤ P + I + W

proof (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)

show

$\wedge s. \neg$ *composition-fri-sampled-canonical-full-cover-failure s bad*
None

by *simp*

next

fix *data attacker-state*

assume *support*:

Some (data, attacker-state) ∈

```

    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  show wp-event verify-monad
    (composition-fri-sampled-canonical-full-cover-failure ?s bad) ?s
    ≤ P + I + W
  by (rule
    wp-composition-fri-sampled-canonical-full-cover-failure-bound-from-witness-shape)
    (rule proximity[OF support], rule index[OF support],
    rule witness[OF support])
qed

lemma trace-fri-sampled-query-bad-candidate-imp-no-full-cover-or-canonical-failure:
  assumes sampled: trace-fri-sampled-query-bad-candidate s out
  shows
    trace-fri-sampled-query-no-full-cover-failure-candidate s bad out ∨
    trace-fri-sampled-canonical-full-cover-failure s bad out
  proof -
    from sampled obtain trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr candidate-query-idxs trace-openings
      trace-table layers
    where evidence:
      trace-fri-partial-candidate-opening-evidence s out trace-roots
      trace-bs trace-final dg composition-roots composition-bs
      composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr candidate-query-idxs trace-openings
      trace-table
    and candidate:
      generic-fri-sampled-query-candidate-evidence trace-table-low-degree
      (Not ∘ trace-table-low-degree) trace-table (clength - 1)
      trace-roots trace-bs trace-final fri-query-idxs trace-round-layers
      layers
    unfolding trace-fri-sampled-query-bad-candidate-def by blast
  show ?thesis
  proof (cases
    generic-fri-sampled-full-cover-failure trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers
      (fri-canonical-domains (length trace-bs)) layers
      (bad trace-table))
    case False
    then have
      trace-fri-sampled-query-no-full-cover-failure-candidate s bad out
    unfolding trace-fri-sampled-query-no-full-cover-failure-candidate-def
    using evidence candidate by blast
  then show ?thesis by simp
  end

```

```

next
  case True
  then have
    trace-fri-sampled-canonical-full-cover-failure s bad out
    unfolding trace-fri-sampled-canonical-full-cover-failure-def
    using evidence candidate by blast
  then show ?thesis by simp
qed
qed

lemma composition-fri-sampled-query-bad-candidate-imp-no-full-cover-or-canonical-failure:
  assumes sampled: composition-fri-sampled-query-bad-candidate s out
  shows
    composition-fri-sampled-query-no-full-cover-failure-candidate s bad out  $\vee$ 
    composition-fri-sampled-canonical-full-cover-failure s bad out
proof -
  from sampled obtain trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table layers
  where evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  and candidate:
    generic-fri-sampled-query-candidate-evidence
    (composition-table-low-degree (to-nat fri-dg))
    (Not  $\circ$  composition-table-low-degree maxDegree) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers layers
  unfolding composition-fri-sampled-query-bad-candidate-def by meson
show ?thesis
proof (cases
  generic-fri-sampled-full-cover-failure
  (composition-table-low-degree (to-nat fri-dg)) composition-table
  (to-nat fri-dg) opening-composition-roots composition-bs
  composition-final fri-query-idxs composition-round-layers
  (fri-canonical-domains (length composition-bs)) layers
  (bad fri-dg composition-table))
case False
then have
  composition-fri-sampled-query-no-full-cover-failure-candidate s bad out
  unfolding

```

```

      composition-fri-sampled-query-no-full-cover-failure-candidate-def
    using evidence candidate by meson
  then show ?thesis by simp
next
case True
then have
  composition-fri-sampled-canonical-full-cover-failure s bad out
  unfolding composition-fri-sampled-canonical-full-cover-failure-def
  using evidence candidate by meson
  then show ?thesis by simp
qed
qed

lemma wp-trace-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-canonical-failure:
  assumes no-failure-bound:
    wp-event verify-monad
      (trace-fri-sampled-query-no-full-cover-failure-candidate s bad) s
      ≤ N
  and failure-bound:
    wp-event verify-monad
      (trace-fri-sampled-canonical-full-cover-failure s bad) s ≤ F
  shows
    wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s
      ≤ N + F
proof -
  have wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s
    ≤ wp-event verify-monad
      (λout.
        trace-fri-sampled-query-no-full-cover-failure-candidate s bad out ∨
        trace-fri-sampled-canonical-full-cover-failure s bad out) s
  by (rule wp-event-mono)
  (rule
    trace-fri-sampled-query-bad-candidate-imp-no-full-cover-or-canonical-failure)
  also have ... ≤
    wp-event verify-monad
      (trace-fri-sampled-query-no-full-cover-failure-candidate s bad) s +
    wp-event verify-monad
      (trace-fri-sampled-canonical-full-cover-failure s bad) s
  by (rule wp-event-union-bound)
  also have ... ≤ N + F
  by (rule add-mono[OF no-failure-bound failure-bound])
  finally show ?thesis .
qed

```

```

lemma wp-composition-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-canonical-failure:
  assumes no-failure-bound:
    wp-event verify-monad
      (composition-fri-sampled-query-no-full-cover-failure-candidate s bad) s
      ≤ N

```

and *failure-bound*:

$$\text{wp-event verify-monad } (\text{composition-fri-sampled-canonical-full-cover-failure } s \text{ bad}) \ s \leq F$$
shows

$$\text{wp-event verify-monad } (\text{composition-fri-sampled-query-bad-candidate } s) \ s \leq N + F$$
proof –
have $\text{wp-event verify-monad } (\text{composition-fri-sampled-query-bad-candidate } s) \ s \leq \text{wp-event verify-monad } (\lambda \text{out. } \text{composition-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad out} \vee \text{composition-fri-sampled-canonical-full-cover-failure } s \text{ bad out}) \ s$
by (rule *wp-event-mono*)
(rule *composition-fri-sampled-query-bad-candidate-imp-no-full-cover-or-canonical-failure*)
also have $\dots \leq \text{wp-event verify-monad } (\text{composition-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad}) \ s + \text{wp-event verify-monad } (\text{composition-fri-sampled-canonical-full-cover-failure } s \text{ bad}) \ s$
by (rule *wp-event-union-bound*)
also have $\dots \leq N + F$
by (rule *add-mono[OF no-failure-bound failure-bound]*)
finally show *?thesis* .
qed

lemma *checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-canonical-failure*:

assumes *no-failure-bound*:

$$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \ (\text{staged-security-with-data-state-verifier-event } (\lambda s. \text{trace-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad})) \ \text{adversary-initial-state} \leq N$$
and *failure-bound*:

$$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \ (\text{staged-security-with-data-state-verifier-event } (\lambda s. \text{trace-fri-sampled-canonical-full-cover-failure } s \text{ bad})) \ \text{adversary-initial-state} \leq F$$
shows

$$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \ (\text{staged-security-with-data-state-verifier-event } \text{trace-fri-sampled-query-bad-candidate}) \ \text{adversary-initial-state} \leq N + F$$
proof –
have $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \ (\text{staged-security-with-data-state-verifier-event } \text{trace-fri-sampled-query-bad-candidate}) \ \text{adversary-initial-state} \leq$

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (λout.
    staged-security-with-data-state-verifier-event
      (λs. trace-fri-sampled-query-no-full-cover-failure-candidate s
        bad) out ∨
    staged-security-with-data-state-verifier-event
      (λs. trace-fri-sampled-canonical-full-cover-failure s bad) out)
  adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
    dest:
      trace-fri-sampled-query-bad-candidate-imp-no-full-cover-or-canonical-failure
      split: option.splits prod.splits)
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-sampled-query-no-full-cover-failure-candidate s bad))
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-sampled-canonical-full-cover-failure s bad))
  adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤ N + F
by (rule add-mono[OF no-failure-bound failure-bound])
finally show ?thesis .
qed

```

lemma *checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-canonical-f*

assumes *no-failure-bound*:

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs.
      composition-fri-sampled-query-no-full-cover-failure-candidate s
        bad))
  adversary-initial-state ≤ N

```

and *failure-bound*:

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. composition-fri-sampled-canonical-full-cover-failure s bad))
  adversary-initial-state ≤ F

```

shows

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-sampled-query-bad-candidate)
  adversary-initial-state ≤ N + F

```

proof —

```

have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event

```

```

    composition-fri-sampled-query-bad-candidate)
  adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
(λout.
  staged-security-with-data-state-verifier-event
  (λs.
    composition-fri-sampled-query-no-full-cover-failure-candidate s
    bad) out ∨
  staged-security-with-data-state-verifier-event
  (λs. composition-fri-sampled-canonical-full-cover-failure s bad)
  out)
  adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
  dest:
    composition-fri-sampled-query-bad-candidate-imp-no-full-cover-or-canonical-failure
    split: option.splits prod.splits)
also have ... ≤
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
(λs.
  composition-fri-sampled-query-no-full-cover-failure-candidate s
  bad))
  adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
(λs. composition-fri-sampled-canonical-full-cover-failure s bad))
  adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤ N + F
by (rule add-mono[OF no-failure-bound failure-bound])
finally show ?thesis .
qed

end

end

```

```

theory Soundness-FRI-Trace-Transcript-Consistent
imports Soundness-FRI-Trace-Candidate-Blocker
begin

```

Trace-only verifier-consumed FRI evidence.

The broad reachable trace FRI predicate is root-parametric: it accepts any authenticated trace root present in the final verifier state. This layer names the verifier-consumed alternative, where the trace root and query indices are those appearing in the accepted FRI opening transcript. It does not change the protocol or the public theorem interface.

```

context soundness

```

begin

definition *accepted-with-partial-trace-openings-verifier-consumed*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow 'f \text{ list list list} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow \text{bool}$

where

accepted-with-partial-trace-openings-verifier-consumed s out
 trace-roots trace-bs trace-final dg composition-roots composition-bs
 composition-final query-idxs trace-round-layers
 composition-round-layers fr as trace-openings $\longleftrightarrow$
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
 dg composition-roots composition-bs composition-final query-idxs
 trace-round-layers composition-round-layers $\wedge$
 $(\exists \text{ rest.}$
 verifier-header-transcript s fr trace-roots trace-final as dg
 composition-roots composition-final rest) $\wedge$
 accepted-with-partial-trace-openings s out fr query-idxs trace-openings

lemma *accepted-with-partial-trace-openings-verifier-consumedD:*

assumes

accepted-with-partial-trace-openings-verifier-consumed s out
 trace-roots trace-bs trace-final dg composition-roots composition-bs
 composition-final query-idxs trace-round-layers
 composition-round-layers fr as trace-openings

shows

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
 dg composition-roots composition-bs composition-final query-idxs
 trace-round-layers composition-round-layers
 $\exists \text{ rest. } \text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 composition-roots composition-final rest
 accepted-with-partial-trace-openings s out fr query-idxs trace-openings

using *assms*

unfolding *accepted-with-partial-trace-openings-verifier-consumed-def*

by *simp-all*

definition *trace-fri-bad-with-verifier-consumed-partial-candidate*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-bad-with-verifier-consumed-partial-candidate s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr as trace-openings trace-table.

accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ trace-table-low-degree trace-table)

lemma *trace-fri-bad-with-verifier-consumed-imp-header-tied:*

assumes *trace-fri-bad-with-verifier-consumed-partial-candidate s out*

shows *trace-fri-bad-with-header-tied-partial-candidate s out*

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings trace-table **where**
consumed:

accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings

and *candidate:*

partial-trace-table-candidate trace-table trace-openings

and *not-low:* $\neg$ *trace-table-low-degree trace-table*

unfolding *trace-fri-bad-with-verifier-consumed-partial-candidate-def*
by *blast*

have *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers

by (rule *accepted-with-partial-trace-openings-verifier-consumedD(1)*)
 $[OF\ consumed]$)

from *accepted-with-partial-trace-openings-verifier-consumedD(2)[OF consumed]*

obtain *rest* **where** *header:*

verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest

by *blast*

have *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings

by (rule *accepted-with-partial-trace-openings-verifier-consumedD(3)*)
 $[OF\ consumed]$)

show *?thesis*

unfolding *trace-fri-bad-with-header-tied-partial-candidate-def*

by (*intro exI conjI*)

(rule fri-openings, rule header, rule partial, rule candidate,
rule not-low)

qed

lemma *trace-fri-bad-with-header-tied-imp-verifier-consumed:*

assumes *trace-fri-bad-with-header-tied-partial-candidate s out*

shows *trace-fri-bad-with-verifier-consumed-partial-candidate s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table **where**
fri-openings:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate:*
partial-trace-table-candidate trace-table trace-openings
and *not-low:* $\neg$ *trace-table-low-degree trace-table*
unfolding *trace-fri-bad-with-header-tied-partial-candidate-def*
by *blast*
have *consumed:*
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
unfolding *accepted-with-partial-trace-openings-verifier-consumed-def*
by (*intro conjI exI*)
(*rule fri-openings, rule header, rule partial*)
show *?thesis*
unfolding *trace-fri-bad-with-verifier-consumed-partial-candidate-def*
by (*intro exI conjI*)
(*rule consumed, rule candidate, rule not-low*)
qed

lemma *trace-fri-bad-with-verifier-consumed-iff-header-tied:*
trace-fri-bad-with-verifier-consumed-partial-candidate s out
 $\longleftrightarrow$ *trace-fri-bad-with-header-tied-partial-candidate s out*
proof
assume *trace-fri-bad-with-verifier-consumed-partial-candidate s out*
then show *trace-fri-bad-with-header-tied-partial-candidate s out*
by (*rule trace-fri-bad-with-verifier-consumed-imp-header-tied*)
next
assume *trace-fri-bad-with-header-tied-partial-candidate s out*
then show *trace-fri-bad-with-verifier-consumed-partial-candidate s out*
by (*rule trace-fri-bad-with-header-tied-imp-verifier-consumed*)
qed

lemma *trace-fri-header-tied-reduction-verifier-consumed-bound:*
assumes *trace-fri-header-tied-reduction s*

shows
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}consumed\text{-}partial\text{-}candidate\ s)\ s \leq$
 $trace\text{-}fri\text{-}error$

proof –
have *mono*:
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}consumed\text{-}partial\text{-}candidate\ s)\ s \leq$
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}bad\text{-}with\text{-}header\text{-}tied\text{-}partial\text{-}candidate\ s)\ s$
by (rule *wp-event-mono*)
(rule *trace-fri-bad-with-verifier-consumed-imp-header-tied*)
also have $\dots \leq trace\text{-}fri\text{-}error$
by (rule *trace-fri-header-tied-reductionD[OF assms]*)
finally show *?thesis* .

qed

lemma *trace-fri-verifier-consumed-reduction-staged-bound*:
assumes *reductions*:
 $\bigwedge data\ attacker\text{-}state.$
 $Some\ (data,\ attacker\text{-}state) \in$
 $set\text{-}dist\ (execute\ (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $trace\text{-}fri\text{-}header\text{-}tied\text{-}reduction$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data))$

shows
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $trace\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}consumed\text{-}partial\text{-}candidate)$
 $adversary\text{-}initial\text{-}state \leq trace\text{-}fri\text{-}error$

proof –
have *mono*:
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $trace\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}consumed\text{-}partial\text{-}candidate)$
 $adversary\text{-}initial\text{-}state \leq$
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $trace\text{-}fri\text{-}bad\text{-}with\text{-}header\text{-}tied\text{-}partial\text{-}candidate)$
 $adversary\text{-}initial\text{-}state$

proof (rule *wp-event-mono*)
fix *out*
assume *consumed*:
 $staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $trace\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}consumed\text{-}partial\text{-}candidate\ out$
then show
 $staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $trace\text{-}fri\text{-}bad\text{-}with\text{-}header\text{-}tied\text{-}partial\text{-}candidate\ out$

```

proof (cases out)
  case None
  then show ?thesis
    using consumed
    unfolding staged-security-with-data-state-verifier-event-def
    by simp
next
  case (Some payload)
  obtain data attacker-state result final-state where payload:
    payload = (((data, attacker-state), result), final-state)
    by (cases payload) auto
  have consumed':
    trace-fri-bad-with-verifier-consumed-partial-candidate
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state))
    using consumed Some payload
    unfolding staged-security-with-data-state-verifier-event-def
    by simp
  have header:
    trace-fri-bad-with-header-tied-partial-candidate
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state))
    by (rule trace-fri-bad-with-verifier-consumed-imp-header-tied
      [OF consumed'])
  show ?thesis
    using Some payload header
    unfolding staged-security-with-data-state-verifier-event-def
    by simp
  qed
qed
also have  $\dots \leq \text{trace-fri-error}$ 
  by (rule trace-fri-header-tied-reduction-staged-bound[OF reductions])
  finally show ?thesis .
qed

end

end

```

```

theory Soundness-FRI-Trace-Empty-Sampled-Assignment-Split
imports
  Soundness-Public-Route-Concrete-FRI-Sharpened
  Soundness-FRI-Canonical-Full-Cover-Failure
  Soundness-FRI-Trace-Transcript-Consistent
  Soundness-Conceptual-Query-Empty-Current
begin

```

First split for the remaining plain trace empty-branch sampled-assignment conflict. The plain event is intentionally broader than the low-degree sampled-query bad-candidate event: it does not require the conflicting candidate table to be non-low-degree. This layer separates the aligned verifier-tied part from the residual that still needs a Merkle/replay explanation.

context *soundness*
begin

lemma *prob-le-refl*: $(x::prob) \leq x$
by *transfer simp*

definition *trace-fri-sampled-assignment-verifier-untied-gap*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-sampled-assignment-verifier-untied-gap } s \text{ out} \longleftrightarrow$
 $\text{trace-fri-sampled-assignment-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-verifier-tied-sampled-assignment-conflict } s \text{ out}$

lemma *trace-fri-sampled-assignment-conflict-imp-verifier-tied-or-untied*:
assumes *trace-fri-sampled-assignment-conflict* $s \text{ out}$
shows
 $\text{trace-fri-verifier-tied-sampled-assignment-conflict } s \text{ out} \vee$
 $\text{trace-fri-sampled-assignment-verifier-untied-gap } s \text{ out}$
using *assms*
unfolding *trace-fri-sampled-assignment-verifier-untied-gap-def* **by** *blast*

lemma *wp-trace-fri-sampled-assignment-conflict-bound-from-verifier-tied-and-untied*:
assumes *verifier-tied-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-verifier-tied-sampled-assignment-conflict } s) s \leq V$
and *untied-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-verifier-untied-gap } s) s \leq U$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict } s) s \leq V + U$
proof –
have *wp-event verify-monad*
 $(\text{trace-fri-sampled-assignment-conflict } s) s \leq$
 $\text{wp-event verify-monad}$
 $(\lambda \text{out.}$
 $\text{trace-fri-verifier-tied-sampled-assignment-conflict } s \text{ out} \vee$
 $\text{trace-fri-sampled-assignment-verifier-untied-gap } s \text{ out}) s$
by *(rule wp-event-mono)*
 $(\text{rule trace-fri-sampled-assignment-conflict-imp-verifier-tied-or-untied})$
also have $\dots \leq$
 $\text{wp-event verify-monad}$

```

      (trace-fri-verifier-tied-sampled-assignment-conflict s) s +
    wp-event verify-monad
      (trace-fri-sampled-assignment-verifier-untied-gap s) s
  by (rule wp-event-union-bound)
also have ... ≤ V + U
  by (rule add-mono[OF verifier-tied-bound untied-bound])
finally show ?thesis .
qed

```

lemma *checked-staged-security-trace-fri-sampled-assignment-conflict-bound-from-verifier-tied-and-untied:*

assumes *verifier-tied-bound:*

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-verifier-tied-sampled-assignment-conflict)
adversary-initial-state ≤ V

```

and *untied-bound:*

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-sampled-assignment-verifier-untied-gap)
adversary-initial-state ≤ U

```

shows

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-sampled-assignment-conflict)
adversary-initial-state ≤ V + U

```

proof –

let *?M* = *checked-staged-security-experiment-with-data-state A*

let *?plain* =

```

  staged-security-with-data-state-verifier-event
  trace-fri-sampled-assignment-conflict

```

let *?tied* =

```

  staged-security-with-data-state-verifier-event
  trace-fri-verifier-tied-sampled-assignment-conflict

```

let *?untied* =

```

  staged-security-with-data-state-verifier-event
  trace-fri-sampled-assignment-verifier-untied-gap

```

have *wp-event ?M ?plain adversary-initial-state ≤*

```

  wp-event ?M (λout. ?tied out ∨ ?untied out)
  adversary-initial-state

```

unfolding *staged-security-with-data-state-verifier-event-def*

by (rule *wp-event-mono*)

(auto dest:

```

  trace-fri-sampled-assignment-conflict-imp-verifier-tied-or-untied
  split: option.splits prod.splits)

```

also have ... ≤

```

  wp-event ?M ?tied adversary-initial-state +
  wp-event ?M ?untied adversary-initial-state

```

by (rule *wp-event-union-bound*)

also have ... ≤ V + U

by (rule add-mono[OF verifier-tied-bound untied-bound])
 finally show ?thesis .
 qed

definition *trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer*

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$
 (unit list $\times$ ('f, 'a) protocol-channel-scheme) option $\Rightarrow$ bool

where

trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
 s out $\longleftrightarrow$
 ($\exists$ trace-roots trace-bs trace-final dg composition-roots
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr trace-openings trace-table.
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final dg composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers $\wedge$
 accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings $\wedge$
 partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ trace-table-low-degree trace-table $\wedge$
 generic-fri-sampled-assignment-conflict-without-same-layer
 trace-table trace-roots trace-bs trace-final fri-query-idxs
 trace-round-layers)

definition *trace-fri-verifier-tied-same-layer-auth-gap*

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$
 (unit list $\times$ ('f, 'a) protocol-channel-scheme) option $\Rightarrow$ bool

where

trace-fri-verifier-tied-same-layer-auth-gap s out $\longleftrightarrow$
 ($\exists$ trace-roots trace-bs trace-final dg composition-roots
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr trace-openings trace-table result
 final-state.
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final dg composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers $\wedge$
 out = Some (result, final-state) $\wedge$
 accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings $\wedge$
 partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ trace-table-low-degree trace-table $\wedge$
 generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs
 fri-query-idxs trace-round-layers $\wedge$
 $\neg$ generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
 trace-roots trace-bs fri-query-idxs trace-round-layers final-state)

lemma *trace-fri-verifier-tied-conflict-imp-authenticated-or-auth-gap:*

assumes *trace-fri-verifier-tied-sampled-assignment-conflict* s out
shows

```

    trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
      s out  $\vee$ 
    trace-fri-verifier-tied-same-layer-auth-gap s out
proof –
from assms obtain trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr trace-openings trace-table
where fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final dg composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers
and partial:
  accepted-with-partial-trace-openings s out fr fri-query-idxs
  trace-openings
and cand: partial-trace-table-candidate trace-table trace-openings
and not-low:  $\neg$  trace-table-low-degree trace-table
and conflict:
  generic-fri-sampled-assignment-conflict trace-table trace-roots
  trace-bs trace-final fri-query-idxs trace-round-layers
unfolding trace-fri-verifier-tied-sampled-assignment-conflict-def
by blast
from fri-openings obtain result final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-fri-opening-transcript-def by blast
have split:
  generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
  trace-table trace-roots trace-bs trace-final fri-query-idxs
  trace-round-layers final-state  $\vee$ 
  (generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs
   fri-query-idxs trace-round-layers  $\wedge$ 
    $\neg$  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
   trace-roots trace-bs fri-query-idxs trace-round-layers final-state)
by (rule
  generic-fri-sampled-assignment-conflict-imp-authenticated-or-same-layer-auth-gap
  [OF conflict])
then show ?thesis
proof
assume auth:
  generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
  trace-table trace-roots trace-bs trace-final fri-query-idxs
  trace-round-layers final-state
have trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
  s out
unfolding
  trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
by (intro exI conjI)
  (rule fri-openings, rule out-eq, rule partial, rule cand,
   rule not-low, rule auth)
then show ?thesis by simp

```



```

next
  assume gap:
    generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs
    fri-query-idxs trace-round-layers  $\wedge$ 
     $\neg$  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    trace-roots trace-bs fri-query-idxs trace-round-layers final-state
  have trace-fri-verifier-tied-same-layer-auth-gap s out
    unfolding trace-fri-verifier-tied-same-layer-auth-gap-def
    by (intro exI conjI)
      (rule fri-openings, rule out-eq, rule partial, rule cand,
       rule not-low, use gap in simp-all)
  then show ?thesis by simp
qed
qed

lemma wp-trace-fri-verifier-tied-conflict-bound-from-authenticated-and-auth-gap:
  fixes A G :: prob
  assumes auth-bound:
    wp-event verify-monad
      (trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
    s  $\leq$  A
  and gap-bound:
    wp-event verify-monad
      (trace-fri-verifier-tied-same-layer-auth-gap s) s  $\leq$  G
  shows
    wp-event verify-monad
      (trace-fri-verifier-tied-sampled-assignment-conflict s) s  $\leq$  A + G
proof -
  have wp-event verify-monad
    (trace-fri-verifier-tied-sampled-assignment-conflict s) s  $\leq$ 
    wp-event verify-monad
      ( $\lambda$ out.
        trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
          s out  $\vee$ 
          trace-fri-verifier-tied-same-layer-auth-gap s out) s
    by (rule wp-event-mono)
      (rule trace-fri-verifier-tied-conflict-imp-authenticated-or-auth-gap)
  also have ...  $\leq$ 
    wp-event verify-monad
      (trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
      s +
      wp-event verify-monad (trace-fri-verifier-tied-same-layer-auth-gap s) s
    by (rule wp-event-union-bound)
  also have ...  $\leq$  A + G
    by (rule add-mono[OF auth-bound gap-bound])
  finally show ?thesis .
qed

```

lemma trace-fri-verifier-tied-authenticated-conflict-imp-without-same-or-same-layer:

assumes

trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out

shows

trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out $\vee$
trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
s out

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-openings trace-table result
final-state

where *fri-openings*:

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers

and *out-eq*: *out* = *Some (result, final-state)*

and *partial*:

accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings

and *cand*: *partial-trace-table-candidate trace-table trace-openings*

and *not-low*: $\neg$ *trace-table-low-degree trace-table*

and *conflict*:

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
trace-table trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers final-state

unfolding

trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def

by *blast*

have *split*:

generic-fri-sampled-assignment-conflict-without-same-layer
trace-table trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers $\vee$
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
trace-roots trace-bs fri-query-idxs trace-round-layers final-state

by (*rule*

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-split
[*OF conflict*])

then show *?thesis*

proof

assume *without*:

generic-fri-sampled-assignment-conflict-without-same-layer
trace-table trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers

have *trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer*

s out

unfolding

trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def

```

    by (intro exI conjI)
      (rule fri-openings, rule partial, rule cand, rule not-low,
       rule without)
  then show ?thesis by simp
next
assume same:
  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
  trace-roots trace-bs fri-query-idxs trace-round-layers final-state
have trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
  s out
  unfolding
    trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-def
  by (intro exI conjI)
    (rule fri-openings, rule out-eq, rule partial, rule cand,
     rule not-low, rule same)
  then show ?thesis by simp
qed
qed

lemma wp-trace-fri-verifier-tied-authenticated-conflict-bound-from-without-same-and-merkle:
  assumes without-bound:
    wp-event verify-monad
      (trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s)
    s ≤ W
  and merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M
  shows
    wp-event verify-monad
      (trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
    s ≤ W + M
proof -
  have wp-event verify-monad
    (trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
    s ≤
    wp-event verify-monad
      (λout.
        trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
          s out ∨
          trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
            s out) s
  by (rule wp-event-mono)
    (rule
      trace-fri-verifier-tied-authenticated-conflict-imp-without-same-or-same-layer)
  also have ... ≤
    wp-event verify-monad
      (trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s)
    s +
    wp-event verify-monad
      (trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks s)

```

s
by (rule *wp-event-union-bound*)
also have $\dots \leq W + M$
by (rule *add-mono*[*OF without-bound*
wp-trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-bound-from-partial-merkle
[OF merkle-bound]])
finally show *?thesis* .
qed

lemma *checked-staged-security-trace-fri-verifier-tied-conflict-bound-from-authenticated-and-auth-gap:*

assumes *auth-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer)
adversary-initial-state ≤ Auth

and *gap-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-verifier-tied-same-layer-auth-gap)
adversary-initial-state ≤ Gap

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-verifier-tied-sampled-assignment-conflict)
adversary-initial-state ≤ Auth + Gap

proof —

let *?M = checked-staged-security-experiment-with-data-state A*

let *?conflict =*

staged-security-with-data-state-verifier-event
trace-fri-verifier-tied-sampled-assignment-conflict

let *?auth =*

staged-security-with-data-state-verifier-event
trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer

let *?gap =*

staged-security-with-data-state-verifier-event
trace-fri-verifier-tied-same-layer-auth-gap

have *wp-event ?M ?conflict adversary-initial-state ≤*

wp-event ?M (λout. ?auth out ∨ ?gap out) adversary-initial-state

unfolding *staged-security-with-data-state-verifier-event-def*

by (rule *wp-event-mono*)

(auto dest:

trace-fri-verifier-tied-conflict-imp-authenticated-or-auth-gap
split: option.splits prod.splits)

also have $\dots \leq$

wp-event ?M ?auth adversary-initial-state +

wp-event ?M ?gap adversary-initial-state

by (rule *wp-event-union-bound*)

also have $\dots \leq Auth + Gap$

by (rule *add-mono*[*OF auth-bound gap-bound*])

finally show ?thesis .
qed

lemma *checked-staged-security-trace-fri-verifier-tied-authenticated-conflict-bound-from-without-same-and-merkle:*
assumes *without-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer)
adversary-initial-state ≤ W

and *merkle-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state ≤ M

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer)
adversary-initial-state ≤ W + M

proof –

let ?M = *checked-staged-security-experiment-with-data-state A*

let ?auth =

staged-security-with-data-state-verifier-event
trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer

let ?without =

staged-security-with-data-state-verifier-event
trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer

let ?merkle =

staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad

have *wp-event ?M ?auth adversary-initial-state ≤*

wp-event ?M (λout. ?without out ∨ ?merkle out)
adversary-initial-state

proof (rule *wp-event-mono*)

fix *out*

assume *auth: ?auth out*

show *?without out ∨ ?merkle out*

proof (cases *out*)

case *None*

then show ?thesis

using *auth*

unfolding *staged-security-with-data-state-verifier-event-def*

trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def

by *simp*

next

case (*Some packed*)

then obtain *data attacker-state result final-state* **where** *out-eq:*

out = Some (((data, attacker-state), result), final-state)

by (cases *packed*, auto *split: prod.splits*)

```

let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have local-auth:
  trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
  ?s (Some (result, final-state))
using auth
unfolding out-eq staged-security-with-data-state-verifier-event-def
by simp
have split:
  trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
  ?s (Some (result, final-state)) ∨
  trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
  ?s (Some (result, final-state))
by (rule
  trace-fri-verifier-tied-authenticated-conflict-imp-without-same-or-same-layer
  [OF local-auth])
then show ?thesis
proof
  assume without:
    trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
    ?s (Some (result, final-state))
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
next
  assume same:
    trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
    ?s (Some (result, final-state))
  have partial-merkle-inconsistency-bad ?s
    (Some (result, final-state))
  by (rule
    trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-imp-partial-merkle
    [OF same])
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
qed
also have ... ≤
  wp-event ?M ?without adversary-initial-state +
  wp-event ?M ?merkle adversary-initial-state
  by (rule wp-event-union-bound)
also have ... ≤ W + M
  by (rule add-mono[OF without-bound merkle-bound])
finally show ?thesis .
qed

```

lemma *checked-staged-security-trace-fri-verifier-tied-conflict-bound-from-without-same-merkle-and-auth-gap:*

fixes $W\ M\ G :: \text{prob}$

assumes *without-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer)
adversary-initial-state $\leq W$

and *merkle-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state $\leq M$

and *gap-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-verifier-tied-same-layer-auth-gap)
adversary-initial-state $\leq G$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-verifier-tied-sampled-assignment-conflict)
adversary-initial-state $\leq (W + M) + G$

proof (rule

checked-staged-security-trace-fri-verifier-tied-conflict-bound-from-authenticated-and-auth-gap)

show *wp-event (checked-staged-security-experiment-with-data-state A)*

(staged-security-with-data-state-verifier-event
trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer)
adversary-initial-state $\leq W + M$

by (rule

checked-staged-security-trace-fri-verifier-tied-authenticated-conflict-bound-from-without-same-and-merkle
[OF without-bound merkle-bound])

show *wp-event (checked-staged-security-experiment-with-data-state A)*

(staged-security-with-data-state-verifier-event
trace-fri-verifier-tied-same-layer-auth-gap)
adversary-initial-state $\leq G$

by (rule *gap-bound*)

qed

definition *trace-fri-empty-sampled-assignment-structural-error-for*

where

trace-fri-empty-sampled-assignment-structural-error-for budgets A =
((reachable-verifier-event-bound-for A
trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer +
trace-fri-controlled-merkle-error-for budgets) +
reachable-verifier-event-bound-for A
trace-fri-verifier-tied-same-layer-auth-gap) +
reachable-verifier-event-bound-for A
trace-fri-sampled-assignment-verifier-untied-gap

definition *trace-empty-header-openings-zero-error-for*

where

trace-empty-header-openings-zero-error-for budgets *A* *empty-bad* =
 (*trace-fri-zero-round-agreement-set-target-error-for* budgets +
trace-fri-controlled-merkle-error-for budgets +
 (*trace-header-one-step-error-for* budgets *A* *empty-bad* +
reachable-verifier-event-bound-for *A*
trace-fri-header-tied-low-candidate-untied-openings-gap) +
trace-header-one-step-error-for budgets *A* *empty-bad*)

definition *trace-fri-header-tied-low-candidate-nonconsumed-openings-gap*

:: ('f, 'a) *protocol-channel-scheme* $\Rightarrow$
 (*unit list* $\times$ ('f, 'a) *protocol-channel-scheme*) *option* $\Rightarrow$ *bool*

where

trace-fri-header-tied-low-candidate-nonconsumed-openings-gap *s* *out*
 $\longleftrightarrow$
 ($\exists$ *trace-roots* *trace-bs* *trace-final* *fri-dg* *composition-roots*
composition-bs *composition-final* (*fri-query-idxs* :: *nat list*)
trace-round-layers *composition-round-layers* (*fr* :: 'f) *as*
trace-openings *trace-table* (*fr0* :: 'f) (*query-idxs0* :: *nat list*)
trace-openings0 *trace-table0* .
trace-fri-reachable-header-tie-gap *s* *out* $\wedge$
accepted-with-partial-trace-openings-verifier-consumed *s* *out*
trace-roots *trace-bs* *trace-final* *fri-dg* *composition-roots*
composition-bs *composition-final* *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *as* *trace-openings* $\wedge$
partial-trace-table-candidate *trace-table* *trace-openings* $\wedge$
trace-table-low-degree *trace-table* $\wedge$
trace-fri-partial-candidate-evidence *s* *out* *fr0* *query-idxs0*
trace-openings0 *trace-table0* $\wedge$
(*fr0* $\neq$ *fr* $\vee$ *query-idxs0* $\neq$ *fri-query-idxs*) $\wedge$
trace-openings0 $\neq$ *trace-openings*)

definition *trace-fri-header-tied-low-candidate-nonconsumed-root-gap*

:: ('f, 'a) *protocol-channel-scheme* $\Rightarrow$
 (*unit list* $\times$ ('f, 'a) *protocol-channel-scheme*) *option* $\Rightarrow$ *bool*

where

trace-fri-header-tied-low-candidate-nonconsumed-root-gap *s* *out*
 $\longleftrightarrow$
 ($\exists$ *trace-roots* *trace-bs* *trace-final* *fri-dg* *composition-roots*
composition-bs *composition-final* (*fri-query-idxs* :: *nat list*)
trace-round-layers *composition-round-layers* (*fr* :: 'f) *as*
trace-openings *trace-table* (*fr0* :: 'f) (*query-idxs0* :: *nat list*)
trace-openings0 *trace-table0* .
trace-fri-reachable-header-tie-gap *s* *out* $\wedge$
accepted-with-partial-trace-openings-verifier-consumed *s* *out*
trace-roots *trace-bs* *trace-final* *fri-dg* *composition-roots*
composition-bs *composition-final* *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *as* *trace-openings* $\wedge$

partial-trace-table-candidate *trace-table* *trace-openings* $\wedge$
trace-table-low-degree *trace-table* $\wedge$
trace-fri-partial-candidate-evidence *s* *out* *fr0* *query-idxs0*
trace-openings0 *trace-table0* $\wedge$
fr0 $\neq$ *fr* $\wedge$
trace-openings0 $\neq$ *trace-openings*)

definition *trace-fri-header-tied-low-candidate-nonconsumed-query-gap*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-low-candidate-nonconsumed-query-gap *s* *out*
 $\longleftrightarrow$
 $(\exists \text{ trace-roots } \text{trace-bs } \text{trace-final } \text{fri-dg } \text{composition-roots}$
 $\text{composition-bs } \text{composition-final } (\text{fri-query-idxs} :: \text{nat list})$
 $\text{trace-round-layers } \text{composition-round-layers } (\text{fr} :: 'f) \text{ as}$
 $\text{trace-openings } \text{trace-table } (\text{fr0} :: 'f) (\text{query-idxs0} :: \text{nat list})$
 $\text{trace-openings0 } \text{trace-table0}.$
 $\text{trace-fri-reachable-header-tie-gap } s \text{ out} \wedge$
 $\text{accepted-with-partial-trace-openings-verifier-consumed } s \text{ out}$
 $\text{trace-roots } \text{trace-bs } \text{trace-final } \text{fri-dg } \text{composition-roots}$
 $\text{composition-bs } \text{composition-final } \text{fri-query-idxs } \text{trace-round-layers}$
 $\text{composition-round-layers } \text{fr as trace-openings} \wedge$
 $\text{partial-trace-table-candidate } \text{trace-table } \text{trace-openings} \wedge$
 $\text{trace-table-low-degree } \text{trace-table} \wedge$
 $\text{trace-fri-partial-candidate-evidence } s \text{ out } \text{fr0 } \text{query-idxs0}$
 $\text{trace-openings0 } \text{trace-table0} \wedge$
 $\text{query-idxs0} \neq \text{fri-query-idxs} \wedge$
 $\text{trace-openings0} \neq \text{trace-openings})$

definition *trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap *s* *out*
 $\longleftrightarrow$
 $(\exists \text{ trace-roots } \text{trace-bs } \text{trace-final } \text{fri-dg } \text{composition-roots}$
 $\text{composition-bs } \text{composition-final } (\text{fri-query-idxs} :: \text{nat list})$
 $\text{trace-round-layers } \text{composition-round-layers } (\text{fr} :: 'f) \text{ as}$
 $\text{trace-openings } \text{trace-table } (\text{fr0} :: 'f) (\text{query-idxs0} :: \text{nat list})$
 $\text{trace-openings0 } \text{trace-table0}.$
 $\text{trace-fri-reachable-header-tie-gap } s \text{ out} \wedge$
 $\text{accepted-with-partial-trace-openings-verifier-consumed } s \text{ out}$
 $\text{trace-roots } \text{trace-bs } \text{trace-final } \text{fri-dg } \text{composition-roots}$
 $\text{composition-bs } \text{composition-final } \text{fri-query-idxs } \text{trace-round-layers}$
 $\text{composition-round-layers } \text{fr as trace-openings} \wedge$
 $\text{partial-trace-table-candidate } \text{trace-table } \text{trace-openings} \wedge$
 $\text{trace-table-low-degree } \text{trace-table} \wedge$
 $\text{trace-fri-partial-candidate-evidence } s \text{ out } \text{fr0 } \text{query-idxs0}$

$\text{trace-openings0 trace-table0} \wedge$
 $\text{fr0} \neq \text{fr} \wedge$
 $\text{query-idxs0} = \text{fri-query-idxs} \wedge$
 $\text{trace-openings0} \neq \text{trace-openings}$

definition *trace-empty-header-nonnormalized-zero-error-for*

where

$\text{trace-empty-header-nonnormalized-zero-error-for budgets } A \text{ empty-bad} =$
 $(\text{trace-fri-zero-round-agreement-set-target-error-for budgets} +$
 $\text{trace-fri-controlled-merkle-error-for budgets} +$
 $(\text{trace-fri-controlled-merkle-error-for budgets} +$
 $((\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-header-tied-low-candidate-nonconsumed-query-gap} +$
 $\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap}) +$
 $\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-header-tied-low-candidate-nonconsumed-query-gap})) +$
 $\text{trace-header-one-step-error-for budgets } A \text{ empty-bad})$

definition *trace-fri-zero-round-verifier-consumed-low-candidate-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-zero-round-verifier-consumed-low-candidate-gap } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings trace-table.}$
 $\text{trace-fri-zero-round-final-obstruction } s \text{ out} \wedge$
 $\text{trace-fri-reachable-header-tie-gap } s \text{ out} \wedge$
 $\text{accepted-with-partial-trace-openings-verifier-consumed } s \text{ out}$
 $\text{trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings} \wedge$
 $\text{length trace-bs} = 0 \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{trace-table-low-degree trace-table})$

lemma *trace-fri-header-tied-low-candidate-nonconsumed-root-gapE:*

assumes *trace-fri-header-tied-low-candidate-nonconsumed-root-gap* $s \text{ out}$

obtains *trace-roots trace-bs trace-final fri-dg composition-roots*
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings trace-table}$
 $\text{fr0 query-idxs0 trace-openings0 trace-table0}$ **where**
 $\text{trace-fri-reachable-header-tie-gap } s \text{ out}$
 $\text{accepted-with-partial-trace-openings-verifier-consumed } s \text{ out}$
 $\text{trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings}$

partial-trace-table-candidate trace-table trace-openings
trace-table-low-degree trace-table
trace-fri-partial-candidate-evidence s out fr0 query-idxs0
trace-openings0 trace-table0
fr0 $\neq$ fr
trace-openings0 $\neq$ trace-openings
using *assms*
unfolding *trace-fri-header-tied-low-candidate-nonconsumed-root-gap-def*
by *blast*

lemma *trace-fri-header-tied-low-candidate-nonconsumed-query-gapE:*
assumes *trace-fri-header-tied-low-candidate-nonconsumed-query-gap s out*
obtains *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings trace-table
fr0 query-idxs0 trace-openings0 trace-table0 **where**
trace-fri-reachable-header-tie-gap s out
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
partial-trace-table-candidate trace-table trace-openings
trace-table-low-degree trace-table
trace-fri-partial-candidate-evidence s out fr0 query-idxs0
trace-openings0 trace-table0
query-idxs0 $\neq$ fri-query-idxs
trace-openings0 $\neq$ trace-openings
using *assms*
unfolding *trace-fri-header-tied-low-candidate-nonconsumed-query-gap-def*
by *blast*

lemma *trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gapE:*
assumes *trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap s out*
obtains *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings trace-table
fr0 query-idxs0 trace-openings0 trace-table0 **where**
trace-fri-reachable-header-tie-gap s out
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
partial-trace-table-candidate trace-table trace-openings
trace-table-low-degree trace-table
trace-fri-partial-candidate-evidence s out fr0 query-idxs0
trace-openings0 trace-table0
fr0 $\neq$ fr
query-idxs0 = fri-query-idxs

$trace-openings0 \neq trace-openings$
using *assms*
unfolding
 $trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap-def$
by *blast*

lemma *trace-fri-zero-round-verifier-consumed-low-candidate-gapE*:

assumes
 $trace-fri-zero-round-verifier-consumed-low-candidate-gap\ s\ out$
obtains $trace-roots\ trace-bs\ trace-final\ fri-dg\ composition-roots$
 $composition-bs\ composition-final\ fri-query-idxs\ trace-round-layers$
 $composition-round-layers\ fr\ as\ trace-openings\ trace-table$ **where**
 $trace-fri-zero-round-final-obstruction\ s\ out$
 $trace-fri-reachable-header-tie-gap\ s\ out$
 $accepted-with-partial-trace-openings-verifier-consumed\ s\ out$
 $trace-roots\ trace-bs\ trace-final\ fri-dg\ composition-roots$
 $composition-bs\ composition-final\ fri-query-idxs\ trace-round-layers$
 $composition-round-layers\ fr\ as\ trace-openings$
 $length\ trace-bs = 0$
 $partial-trace-table-candidate\ trace-table\ trace-openings$
 $trace-table-low-degree\ trace-table$
using *assms*
unfolding $trace-fri-zero-round-verifier-consumed-low-candidate-gap-def$
by *blast*

lemma *trace-fri-zero-round-verifier-consumed-low-candidate-gap-imp-candidate-available*:

assumes
 $trace-fri-zero-round-verifier-consumed-low-candidate-gap\ s\ out$
shows $trace-fri-header-tied-candidate-available-gap\ s\ out$
proof (rule $trace-fri-zero-round-verifier-consumed-low-candidate-gapE$
 $[OF\ assms]$)
fix $trace-roots\ trace-bs\ trace-final\ fri-dg\ composition-roots$
 $composition-bs\ composition-final\ fri-query-idxs\ trace-round-layers$
 $composition-round-layers\ fr\ as\ trace-openings\ trace-table$
assume $gap: trace-fri-reachable-header-tie-gap\ s\ out$
and *consumed*:
 $accepted-with-partial-trace-openings-verifier-consumed\ s\ out$
 $trace-roots\ trace-bs\ trace-final\ fri-dg\ composition-roots$
 $composition-bs\ composition-final\ fri-query-idxs\ trace-round-layers$
 $composition-round-layers\ fr\ as\ trace-openings$
and *candidate*:
 $partial-trace-table-candidate\ trace-table\ trace-openings$
have *fri-openings*:
 $accepted-fri-opening-transcript\ s\ out\ trace-roots\ trace-bs$
 $trace-final\ fri-dg\ composition-roots\ composition-bs\ composition-final$
 $fri-query-idxs\ trace-round-layers\ composition-round-layers$
by (rule $accepted-with-partial-trace-openings-verifier-consumedD(1)$
 $[OF\ consumed]$)
from $accepted-with-partial-trace-openings-verifier-consumedD(2)$

[OF consumed]
obtain *rest* **where** *header*:
 verifier-header-transcript s fr trace-roots trace-final as fri-dg
 composition-roots composition-final rest
by *blast*
have *partial*:
 accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings
by (rule *accepted-with-partial-trace-openings-verifier-consumedD*(3)
 [OF consumed])
show *trace-fri-header-tied-candidate-available-gap s out*
unfolding *trace-fri-header-tied-candidate-available-gap-def*
by (intro *exI conjI*)
 (rule *gap*, rule *fri-openings*, rule *header*, rule *partial*,
 rule *candidate*)
qed

lemma *trace-fri-zero-round-verifier-consumed-low-candidate-gap-imp-untied-openings*:
assumes
 trace-fri-zero-round-verifier-consumed-low-candidate-gap s out
shows *trace-fri-header-tied-low-candidate-untied-openings-gap s out*
proof (rule *trace-fri-zero-round-verifier-consumed-low-candidate-gapE*
 [OF *assms*])
fix *trace-roots trace-bs trace-final fri-dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr as trace-openings trace-table
assume *gap*: *trace-fri-reachable-header-tie-gap s out*
and *consumed*:
 accepted-with-partial-trace-openings-verifier-consumed s out
 trace-roots trace-bs trace-final fri-dg composition-roots
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr as trace-openings
and *candidate*:
 partial-trace-table-candidate trace-table trace-openings
and *low*: *trace-table-low-degree trace-table*
have *fri-openings*:
 accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
 fri-dg composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers
by (rule *accepted-with-partial-trace-openings-verifier-consumedD*(1)
 [OF *consumed*])
from *accepted-with-partial-trace-openings-verifier-consumedD*(2)
 [OF *consumed*]
obtain *rest* **where** *header*:
 verifier-header-transcript s fr trace-roots trace-final as fri-dg
 composition-roots composition-final rest
by *blast*
have *partial*:
 accepted-with-partial-trace-openings s out fr fri-query-idxs

```

    trace-openings
  by (rule accepted-with-partial-trace-openings-verifier-consumedD(3))
    [OF consumed])
  have low-gap: trace-fri-header-tied-low-candidate-gap s out
    unfolding trace-fri-header-tied-low-candidate-gap-def
  by (intro exI conjI)
    (rule gap, rule fri-openings, rule header, rule partial,
     rule candidate, rule low)
  have untied:
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
  by (rule trace-fri-header-tied-low-candidate-gap-imp-untied-reachable-witness
    [OF low-gap])
  show trace-fri-header-tied-low-candidate-untied-openings-gap s out
  by (rule trace-fri-header-tied-low-candidate-untied-gap-imp-openings
    [OF untied])
qed

lemma accepted-with-partial-trace-openings-query-index-list-space:
  assumes partial:
    accepted-with-partial-trace-openings s out fr query-idxs trace-openings
  shows query-idxs ∈ fri-query-index-list-space
proof -
  have len: length query-idxs = rounds
  by (rule accepted-with-partial-trace-openings-shapes(1)[OF partial])
  have subset: set query-idxs ⊆ query-sample-space
proof
  fix idx
  assume idx-in: idx ∈ set query-idxs
  then obtain i where i-bound: i < length query-idxs
    and idx-eq: query-idxs ! i = idx
    unfolding in-set-conv-nth by blast
  have i-round: i < rounds
  using i-bound len by simp
  obtain result final-state where out-eq: out = Some (result, final-state)
  and auth:
     $\bigwedge j. j < rounds \implies$ 
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! j) final-state
  using partial unfolding accepted-with-partial-trace-openings-def by blast
  have indices:
    map opening-index (trace-openings ! i) = powers-scaled idx
  using accepted-with-partial-trace-openings-shapes(3)[OF partial i-round]
    idx-eq by simp
  show idx ∈ query-sample-space
  by (rule partial-authenticated-trace-openings-query-index-in-sample-space
    [OF indices auth[OF i-round]])
qed
show ?thesis
  unfolding fri-query-index-list-space-def

```

using len subset by simp
qed

lemma *trace-fri-header-tied-low-candidate-nonconsumed-root-gap-shapes:*
assumes *trace-fri-header-tied-low-candidate-nonconsumed-root-gap s out*
obtains *fri-query-idxs query-idxs0* **where**
fri-query-idxs $\in$ *fri-query-index-list-space*
query-idxs0 $\in$ *fri-query-index-list-space*

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings trace-table
fr0 query-idxs0 trace-openings0 trace-table0 **where**
consumed:
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
and *evidence0:*
trace-fri-partial-candidate-evidence s out fr0 query-idxs0
trace-openings0 trace-table0
and *mismatch: fr0 $\neq$ fr*
by (rule *trace-fri-header-tied-low-candidate-nonconsumed-root-gapE*)
have *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
fri-dg composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers
by (rule *accepted-with-partial-trace-openings-verifier-consumedD*(1)
 $[OF\ consumed]$)
have *verifier-space:*
fri-query-idxs $\in$ *fri-query-index-list-space*
by (rule *accepted-fri-opening-transcript-query-index-list-space*
 $[OF\ fri-openings]$)
have *alternate-space: query-idxs0* $\in$ *fri-query-index-list-space*
by (rule *accepted-with-partial-trace-openings-query-index-list-space*)
(rule *trace-fri-partial-candidate-evidenceD*(1) $[OF\ evidence0]$)
show *?thesis*
by (rule *that* $[OF\ verifier-space\ alternate-space]$)

qed

lemma *trace-fri-header-tied-low-candidate-nonconsumed-query-gap-shapes:*
assumes *trace-fri-header-tied-low-candidate-nonconsumed-query-gap s out*
obtains *fri-query-idxs query-idxs0* **where**
fri-query-idxs $\in$ *fri-query-index-list-space*
query-idxs0 $\in$ *fri-query-index-list-space*
query-idxs0 $\neq$ *fri-query-idxs*

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as trace-openings trace-table
fr0 query-idxs0 trace-openings0 trace-table0 **where**
consumed:
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
and *evidence0:*
trace-fri-partial-candidate-evidence s out fr0 query-idxs0
trace-openings0 trace-table0
and *mismatch: query-idxs0 $\neq$ fri-query-idxs*
by (rule *trace-fri-header-tied-low-candidate-nonconsumed-query-gapE*)
have *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
fri-dg composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers
by (rule *accepted-with-partial-trace-openings-verifier-consumedD(1)*
[OF consumed])
have *verifier-space:*
fri-query-idxs $\in$ fri-query-index-list-space
by (rule *accepted-fri-opening-transcript-query-index-list-space*
[OF fri-openings])
have *alternate-space: query-idxs0 $\in$ fri-query-index-list-space*
by (rule *accepted-with-partial-trace-openings-query-index-list-space*)
(rule *trace-fri-partial-candidate-evidenceD(1)[OF evidence0]*)
show *?thesis*
by (rule *that[of fri-query-idxs query-idxs0,*
OF verifier-space alternate-space mismatch])
qed

lemma *trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap-shapes:*
assumes
trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap s out
obtains *fri-query-idxs query-idxs0* **where**
fri-query-idxs $\in$ fri-query-index-list-space
query-idxs0 $\in$ fri-query-index-list-space
query-idxs0 = fri-query-idxs
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings trace-table
fr0 query-idxs0 trace-openings0 trace-table0 **where**
consumed:
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
and *evidence0:*
trace-fri-partial-candidate-evidence s out fr0 query-idxs0

trace-openings0 trace-table0
and *mismatch: fr0 ≠ fr*
and *query-eq: query-idxs0 = fri-query-idxs*
by (rule *trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gapE*)
have *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
fri-dg composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers
by (rule *accepted-with-partial-trace-openings-verifier-consumedD(1)*
[*OF consumed*])
have *verifier-space:*
fri-query-idxs ∈ fri-query-index-list-space
by (rule *accepted-fri-opening-transcript-query-index-list-space*
[*OF fri-openings*])
have *alternate-space: query-idxs0 ∈ fri-query-index-list-space*
by (rule *accepted-with-partial-trace-openings-query-index-list-space*)
(rule *trace-fri-partial-candidate-evidenceD(1)*[*OF evidence0*])
show *?thesis*
by (rule *that*[*OF verifier-space alternate-space query-eq*])
qed

lemma *trace-fri-header-tied-low-candidate-nonconsumed-root-gap-imp-query-or-irrelevant-root:*
assumes *trace-fri-header-tied-low-candidate-nonconsumed-root-gap s out*
shows
trace-fri-header-tied-low-candidate-nonconsumed-query-gap s out ∨
trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap
s out
proof —
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings trace-table
fr0 query-idxs0 trace-openings0 trace-table0 **where**
gap: trace-fri-reachable-header-tie-gap s out
and *consumed:*
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
and *candidate: partial-trace-table-candidate trace-table trace-openings*
and *low: trace-table-low-degree trace-table*
and *evidence0:*
trace-fri-partial-candidate-evidence s out fr0 query-idxs0
trace-openings0 trace-table0
and *root-mismatch: fr0 ≠ fr*
and *openings-mismatch: trace-openings0 ≠ trace-openings*
by (rule *trace-fri-header-tied-low-candidate-nonconsumed-root-gapE*)
show *?thesis*
proof (*cases query-idxs0 ≠ fri-query-idxs*)
case *True*

```

have trace-fri-header-tied-low-candidate-nonconsumed-query-gap s out
  unfolding trace-fri-header-tied-low-candidate-nonconsumed-query-gap-def
  by (intro exI conjI)
    (rule gap, rule consumed, rule candidate, rule low, rule evidence0,
      rule True, rule openings-mismatch)
  then show ?thesis by simp
next
case False
then have query-eq: query-idxs0 = fri-query-idxs
  by simp
have trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap
  s out
  unfolding
    trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap-def
  by (intro exI conjI)
    (rule gap, rule consumed, rule candidate, rule low, rule evidence0,
      rule root-mismatch, rule query-eq, rule openings-mismatch)
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-trace-fri-header-tied-low-candidate-nonconsumed-root-gap-bound-from-query-and-irrel
assumes query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-nonconsumed-query-gap)
  adversary-initial-state  $\leq Q$ 
and irrelevant-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap)
  adversary-initial-state  $\leq I$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-nonconsumed-root-gap)
  adversary-initial-state  $\leq Q + I$ 
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?root =
    staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-nonconsumed-root-gap
  let ?query =
    staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-nonconsumed-query-gap
  let ?irrelevant =
    staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap
  have wp-event ?M ?root adversary-initial-state  $\leq$ 

```

$wp\text{-}event\ ?M\ (\lambda out. \ ?query\ out \vee \ ?irrelevant\ out)$
 $adversary\text{-}initial\text{-}state$
unfolding $staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event\text{-}def$
by (rule $wp\text{-}event\text{-}mono\text{-}on\text{-}support$)
(auto dest:
 $trace\text{-}fri\text{-}header\text{-}tied\text{-}low\text{-}candidate\text{-}nonconsumed\text{-}root\text{-}gap\text{-}imp\text{-}query\text{-}or\text{-}irrelevant\text{-}root$
 $split: option.splits\ prod.splits$)
also have $\dots \leq$
 $wp\text{-}event\ ?M\ ?query\ adversary\text{-}initial\text{-}state +$
 $wp\text{-}event\ ?M\ ?irrelevant\ adversary\text{-}initial\text{-}state$
by (rule $wp\text{-}event\text{-}union\text{-}bound$)
also have $\dots \leq Q + I$
by (rule $add\text{-}mono[OF\ query\text{-}bound\ irrelevant\text{-}bound]$)
finally show $?thesis$.
qed

lemma $trace\text{-}fri\text{-}header\text{-}tied\text{-}low\text{-}candidate\text{-}nonconsumed\text{-}openings\text{-}gap\text{-}imp\text{-}root\text{-}or\text{-}query$:
assumes $trace\text{-}fri\text{-}header\text{-}tied\text{-}low\text{-}candidate\text{-}nonconsumed\text{-}openings\text{-}gap\ s\ out$
shows

$trace\text{-}fri\text{-}header\text{-}tied\text{-}low\text{-}candidate\text{-}nonconsumed\text{-}root\text{-}gap\ s\ out \vee$
 $trace\text{-}fri\text{-}header\text{-}tied\text{-}low\text{-}candidate\text{-}nonconsumed\text{-}query\text{-}gap\ s\ out$

proof —

from $assms$ **obtain** $trace\text{-}roots\ trace\text{-}bs\ trace\text{-}final\ fri\text{-}dg\ composition\text{-}roots$
 $composition\text{-}bs\ composition\text{-}final\ fri\text{-}query\text{-}idxs$
 $trace\text{-}round\text{-}layers\ composition\text{-}round\text{-}layers\ fr\ as$
 $trace\text{-}openings\ trace\text{-}table\ fr0\ query\text{-}idxs0\ trace\text{-}openings0\ trace\text{-}table0$

where

gap : $trace\text{-}fri\text{-}reachable\text{-}header\text{-}tie\text{-}gap\ s\ out$

and $consumed$:

$accepted\text{-}with\text{-}partial\text{-}trace\text{-}openings\text{-}verifier\text{-}consumed\ s\ out$
 $trace\text{-}roots\ trace\text{-}bs\ trace\text{-}final\ fri\text{-}dg\ composition\text{-}roots$
 $composition\text{-}bs\ composition\text{-}final\ fri\text{-}query\text{-}idxs\ trace\text{-}round\text{-}layers$
 $composition\text{-}round\text{-}layers\ fr\ as\ trace\text{-}openings$

and $candidate$: $partial\text{-}trace\text{-}table\text{-}candidate\ trace\text{-}table\ trace\text{-}openings$

and low : $trace\text{-}table\text{-}low\text{-}degree\ trace\text{-}table$

and $evidence0$:

$trace\text{-}fri\text{-}partial\text{-}candidate\text{-}evidence\ s\ out\ fr0\ query\text{-}idxs0$
 $trace\text{-}openings0\ trace\text{-}table0$

and $mismatch$: $fr0 \neq fr \vee query\text{-}idxs0 \neq fri\text{-}query\text{-}idxs$

and $openings\text{-}mismatch$: $trace\text{-}openings0 \neq trace\text{-}openings$

unfolding $trace\text{-}fri\text{-}header\text{-}tied\text{-}low\text{-}candidate\text{-}nonconsumed\text{-}openings\text{-}gap\text{-}def$

by $blast$

from $mismatch$ **show** $?thesis$

proof

assume $root\text{-}mismatch$: $fr0 \neq fr$

have $trace\text{-}fri\text{-}header\text{-}tied\text{-}low\text{-}candidate\text{-}nonconsumed\text{-}root\text{-}gap\ s\ out$

unfolding $trace\text{-}fri\text{-}header\text{-}tied\text{-}low\text{-}candidate\text{-}nonconsumed\text{-}root\text{-}gap\text{-}def$

apply (rule $exI[where\ x=trace\text{-}roots]$)

apply (rule $exI[where\ x=trace\text{-}bs]$)

```

  apply (rule exI[where x=trace-final])
  apply (rule exI[where x=fri-dg])
  apply (rule exI[where x=composition-roots])
  apply (rule exI[where x=composition-bs])
  apply (rule exI[where x=composition-final])
  apply (rule exI[where x=fri-query-idxs])
  apply (rule exI[where x=trace-round-layers])
  apply (rule exI[where x=composition-round-layers])
  apply (rule exI[where x=fr])
  apply (rule exI[where x=as])
  apply (rule exI[where x=trace-openings])
  apply (rule exI[where x=trace-table])
  apply (rule exI[where x=fr0])
  apply (rule exI[where x=query-idxs0])
  apply (rule exI[where x=trace-openings0])
  apply (rule exI[where x=trace-table0])
  using gap consumed candidate low evidence0 root-mismatch openings-mismatch
  by blast
then show ?thesis by simp
next
assume query-mismatch: query-idxs0 ≠ fri-query-idxs
have trace-fri-header-tied-low-candidate-nonconsumed-query-gap s out
  unfolding trace-fri-header-tied-low-candidate-nonconsumed-query-gap-def
  apply (rule exI[where x=trace-roots])
  apply (rule exI[where x=trace-bs])
  apply (rule exI[where x=trace-final])
  apply (rule exI[where x=fri-dg])
  apply (rule exI[where x=composition-roots])
  apply (rule exI[where x=composition-bs])
  apply (rule exI[where x=composition-final])
  apply (rule exI[where x=fri-query-idxs])
  apply (rule exI[where x=trace-round-layers])
  apply (rule exI[where x=composition-round-layers])
  apply (rule exI[where x=fr])
  apply (rule exI[where x=as])
  apply (rule exI[where x=trace-openings])
  apply (rule exI[where x=trace-table])
  apply (rule exI[where x=fr0])
  apply (rule exI[where x=query-idxs0])
  apply (rule exI[where x=trace-openings0])
  apply (rule exI[where x=trace-table0])
  using gap consumed candidate low evidence0 query-mismatch openings-mismatch
  by blast
then show ?thesis by simp
qed
qed

lemma trace-fri-header-tied-low-candidate-nonconsumed-root-gap-imp-untied-root-gap:
  assumes trace-fri-header-tied-low-candidate-nonconsumed-root-gap s out

```

shows *trace-fri-header-tied-low-candidate-untied-root-gap s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
trace-table fr0 query-idxs0 trace-openings0 trace-table0
where *gap: trace-fri-reachable-header-tie-gap s out*
and *consumed:*
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
and *candidate: partial-trace-table-candidate trace-table trace-openings*
and *low: trace-table-low-degree trace-table*
and *evidence0:*
trace-fri-partial-candidate-evidence s out fr0 query-idxs0
trace-openings0 trace-table0
and *root-mismatch: fr0 ≠ fr*
by (rule *trace-fri-header-tied-low-candidate-nonconsumed-root-gapE*)
have *transcript:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *header-ex:*
 $\exists$ *rest. verifier-header-transcript s fr trace-roots trace-final as*
fri-dg composition-roots composition-final rest
and *accepted:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
by (rule *accepted-with-partial-trace-openings-verifier-consumedD*
 $[OF\ consumed]) +$
from *header-ex* **obtain** *rest* **where** *header:*
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
by *blast*
have *accepted0:*
accepted-with-partial-trace-openings s out fr0 query-idxs0
trace-openings0
and *candidate0: partial-trace-table-candidate trace-table0 trace-openings0*
and *not-low0: ¬ trace-table-low-degree trace-table0*
by (rule *trace-fri-partial-candidate-evidenceD* $[OF\ evidence0]) +$
show *?thesis*
unfolding *trace-fri-header-tied-low-candidate-untied-root-gap-def*
by (rule *trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI*
 $[OF\ gap\ transcript\ header\ accepted\ candidate\ low\ accepted0\ candidate0$
 $not-low0])$
(simp add: root-mismatch)
qed

lemma *trace-fri-header-tied-low-candidate-nonconsumed-query-gap-imp-untied-query-gap*:
assumes *trace-fri-header-tied-low-candidate-nonconsumed-query-gap s out*
shows *trace-fri-header-tied-low-candidate-untied-query-gap s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
trace-table fr0 query-idxs0 trace-openings0 trace-table0
where *gap*: *trace-fri-reachable-header-tie-gap s out*
and *consumed*:
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
and *candidate*: *partial-trace-table-candidate trace-table trace-openings*
and *low*: *trace-table-low-degree trace-table*
and *evidence0*:
trace-fri-partial-candidate-evidence s out fr0 query-idxs0
trace-openings0 trace-table0
and *query-mismatch*: *query-idxs0 ≠ fri-query-idxs*
by (rule *trace-fri-header-tied-low-candidate-nonconsumed-query-gapE*)
have *transcript*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *header-ex*:
 $\exists$ *rest*. *verifier-header-transcript s fr trace-roots trace-final as*
fri-dg composition-roots composition-final rest
and *accepted*:
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
by (rule *accepted-with-partial-trace-openings-verifier-consumedD*
 $[OF\ consumed]$) +
from *header-ex* **obtain** *rest* **where** *header*:
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
by *blast*
have *accepted0*:
accepted-with-partial-trace-openings s out fr0 query-idxs0
trace-openings0
and *candidate0*: *partial-trace-table-candidate trace-table0 trace-openings0*
and *not-low0*: $\neg$ *trace-table-low-degree trace-table0*
by (rule *trace-fri-partial-candidate-evidenceD* $[OF\ evidence0]$) +
show *?thesis*
unfolding *trace-fri-header-tied-low-candidate-untied-query-gap-def*
by (rule *trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI*
 $[OF\ gap\ transcript\ header\ accepted\ candidate\ low\ accepted0\ candidate0]$)

```

      not-low0]])
    (simp add: query-mismatch)
qed

lemma checked-staged-security-trace-fri-header-tied-low-candidate-nonconsumed-openings-gap-bound-from-root-query
  assumes root-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-header-tied-low-candidate-nonconsumed-root-gap)
    adversary-initial-state  $\leq R$ 
  and query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-header-tied-low-candidate-nonconsumed-query-gap)
    adversary-initial-state  $\leq Q$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-header-tied-low-candidate-nonconsumed-openings-gap)
    adversary-initial-state  $\leq R + Q$ 
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?nonconsumed =
    staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-nonconsumed-openings-gap
  let ?root =
    staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-nonconsumed-root-gap
  let ?query =
    staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-nonconsumed-query-gap
  have split:
    wp-event ?M ?nonconsumed adversary-initial-state  $\leq$ 
      wp-event ?M ( $\lambda$ out. ?root out  $\vee$  ?query out) adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def
  by (rule wp-event-mono-on-support)
    (auto dest:
      trace-fri-header-tied-low-candidate-nonconsumed-openings-gap-imp-root-or-query
      split: option.splits prod.splits)
  also have ...  $\leq$ 
    wp-event ?M ?root adversary-initial-state +
    wp-event ?M ?query adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq R + Q$ 
  by (rule add-mono[OF root-bound query-bound])
  finally show ?thesis .
qed

```

lemma trace-fri-zero-round-final-obstruction-imp-checked-merkle-low-consumed-or-header:

assumes *support*: $out \in \text{set-dist} \text{ (execute verify-monad } s)$
and *obstruction*: $\text{trace-fri-zero-round-final-obstruction } s \text{ out}$
shows
 $\text{trace-fri-zero-round-checked-final-obstruction } s \text{ out} \vee$
 $\text{partial-merkle-inconsistency-bad } s \text{ out} \vee$
 $\text{trace-fri-zero-round-verifier-consumed-low-candidate-gap } s \text{ out} \vee$
 $\text{trace-fri-bad-with-header-tied-partial-candidate } s \text{ out}$
proof –
from $\text{trace-fri-zero-round-final-obstructionE}[OF \text{ obstruction}]$
obtain $\text{trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr0 candidate-query-idxs trace-openings0}$
 trace-table0 **where** *evidence*:
 $\text{trace-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr0 candidate-query-idxs trace-openings0}$
 trace-table0
and *rounds-empty*: $\text{length trace-bs} = 0$
by $(\text{rule trace-fri-zero-round-final-obstructionE}[OF \text{ obstruction}])$
have *fri-openings*:
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers}$
using $\text{trace-fri-partial-candidate-opening-evidenceD}(1)[OF \text{ evidence}]$.
from *fri-openings* **obtain** *result final-state* **where** *out-eq*:
 $out = \text{Some}(\text{result}, \text{final-state})$
unfolding $\text{accepted-fri-opening-transcript-def}$ **by** *blast*
from $\text{accepted-fri-opening-transcript-headerE}[OF \text{ fri-openings}]$
obtain *fr as rest* **where** *header*:
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final rest}$
by *blast*
show *?thesis*
proof $(\text{cases partial-merkle-inconsistency-bad } s \text{ out})$
case *True*
then show *?thesis* **by** *simp*
next
case *no-merkle*: *False*
from $\text{accepted-fri-opening-transcript-trace-openings-for-query-indices}$
 $[OF \text{ fri-openings header out-eq}]$
obtain *trace-openings* **where** *partial*:
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 trace-openings
by *blast*
obtain *trace-table* **where** *candidate*:
 $\text{partial-trace-table-candidate trace-table trace-openings}$
using $\text{accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad}$
 $[OF \text{ partial}[unfolding \text{ out-eq}] \text{ no-merkle}[unfolding \text{ out-eq}]]$


```

  by blast
show ?thesis
proof (cases trace-table-low-degree trace-table)
  case True
  then show ?thesis
  proof (cases trace-fri-bad-with-header-tied-partial-candidate s out)
    case True
    then show ?thesis by simp
  next
  case no-header: False
  have reachable:
    trace-fri-bad-with-reachable-partial-candidate s out
  using trace-fri-zero-round-final-obstruction-imp-header-tied-or-gap
    [OF obstruction] no-header
  unfolding trace-fri-reachable-header-tie-gap-def by simp
  have gap: trace-fri-reachable-header-tie-gap s out
  unfolding trace-fri-reachable-header-tie-gap-def
  using reachable no-header by simp
  have consumed:
    accepted-with-partial-trace-openings-verifier-consumed s out
    trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as
    trace-openings
  unfolding accepted-with-partial-trace-openings-verifier-consumed-def
  by (intro conjI exI)
    (rule fri-openings, rule header, rule partial)
  have low-gap:
    trace-fri-zero-round-verifier-consumed-low-candidate-gap s out
  unfolding
    trace-fri-zero-round-verifier-consumed-low-candidate-gap-def
  by (intro exI conjI)
    (rule obstruction, rule gap, rule consumed, rule rounds-empty,
    rule candidate, rule True)
  then show ?thesis by simp
qed
next
case not-low: False
have not-final:
  ¬ fri-final-constant-consistent trace-table trace-final
proof
  assume final:
    fri-final-constant-consistent trace-table trace-final
  have low:
    trace-table-low-degree trace-table
  by (rule
    trace-zero-round-candidate-low-degree-if-final-consistent
    [OF rounds-empty fri-openings candidate final])
  then show False

```

```

    using not-low by contradiction
qed
have header-zero:
  trace-fri-header-tied-zero-round-final-obstruction s out
  unfolding trace-fri-header-tied-zero-round-final-obstruction-def
  by (intro exI conjI)
    (rule fri-openings, rule header, rule partial, rule candidate,
     rule not-low, rule rounds-empty, rule not-final)
then show ?thesis
  using
    trace-fri-header-tied-zero-round-final-obstruction-imp-checked-or-merkle
  by blast
qed
qed
qed

lemma checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-staged-checked-merkle-low-consumed
  fixes Q M L H :: prob
  assumes checked-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-zero-round-checked-final-obstruction)
    adversary-initial-state  $\leq$  Q
  and merkle-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       partial-merkle-inconsistency-bad)
    adversary-initial-state  $\leq$  M
  and low-consumed-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-zero-round-verifier-consumed-low-candidate-gap)
    adversary-initial-state  $\leq$  L
  and header-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-bad-with-header-tied-partial-candidate)
    adversary-initial-state  $\leq$  H
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-zero-round-final-obstruction)
    adversary-initial-state  $\leq$  Q + M + L + H
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?zero =
    staged-security-with-data-state-verifier-event
    trace-fri-zero-round-final-obstruction
  let ?checked =

```

```

    staged-security-with-data-state-verifier-event
    trace-fri-zero-round-checked-final-obstruction
let ?merkle =
    staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad
let ?low =
    staged-security-with-data-state-verifier-event
    trace-fri-zero-round-verifier-consumed-low-candidate-gap
let ?header =
    staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate
have wp-event ?M ?zero adversary-initial-state ≤
    wp-event ?M
    (λout. ?checked out ∨ ?merkle out ∨ ?low out ∨
      ?header out)
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
    out ∈ set-dist (execute ?M adversary-initial-state)
    and zero: ?zero out
show ?checked out ∨ ?merkle out ∨ ?low out ∨ ?header out
proof (cases out)
case None
then show ?thesis
    using zero unfolding staged-security-with-data-state-verifier-event-def
    by simp
next
case (Some out-data)
then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases out-data, auto split: prod.splits)
let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
have verify-support:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
    by blast
have zero-local:
    trace-fri-zero-round-final-obstruction ?s
    (Some (result, final-state))
    using zero unfolding out-eq
    staged-security-with-data-state-verifier-event-def
    by simp
have
    trace-fri-zero-round-checked-final-obstruction ?s
    (Some (result, final-state)) ∨

```

```

    partial-merkle-inconsistency-bad ?s
    (Some (result, final-state)) ∨
    trace-fri-zero-round-verifier-consumed-low-candidate-gap ?s
    (Some (result, final-state)) ∨
    trace-fri-bad-with-header-tied-partial-candidate ?s
    (Some (result, final-state))
  by (rule
    trace-fri-zero-round-final-obstruction-imp-checked-merkle-low-consumed-or-header
    [OF verify-support zero-local])
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
also have ... ≤
  wp-event ?M ?checked adversary-initial-state +
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?low adversary-initial-state +
  wp-event ?M ?header adversary-initial-state
  by (rule wp-event-union-bound4)
also have ... ≤ Q + M + L + H
  by (intro add-mono checked-bound merkle-bound low-consumed-bound
    header-bound)
finally show ?thesis .
qed

lemma trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-imp-nonconsumed:
  assumes
    trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap
    s out
  shows
    trace-fri-header-tied-low-candidate-nonconsumed-openings-gap s out
proof -
  from assms obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as rest trace-openings
    trace-table fr0 query-idxs0 trace-openings0 trace-table0 where
    gap: trace-fri-reachable-header-tie-gap s out
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  and header:
    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest
  and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings

```

and *candidate*:
 partial-trace-table-candidate trace-table trace-openings
and *low*: *trace-table-low-degree trace-table*
and *partial0*:
 accepted-with-partial-trace-openings s out fr0 query-idxs0
 trace-openings0
and *candidate0*:
 partial-trace-table-candidate trace-table0 trace-openings0
and *not-low0*: $\neg$ *trace-table-low-degree trace-table0*
and *mismatch*:
 fr0 $\neq$ *fr* $\vee$ *query-idxs0* $\neq$ *fri-query-idxs*
and *openings-mismatch*:
 trace-openings0 $\neq$ *trace-openings*
unfolding
 trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-def
 trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
by *blast*
have *consumed*:
 accepted-with-partial-trace-openings-verifier-consumed s out
 trace-roots trace-bs trace-final fri-dg composition-roots
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr as trace-openings
unfolding *accepted-with-partial-trace-openings-verifier-consumed-def*
by (*intro conjI exI*)
 (*rule fri-openings, rule header, rule partial*)
have *evidence0*:
 trace-fri-partial-candidate-evidence s out fr0 query-idxs0
 trace-openings0 trace-table0
unfolding *trace-fri-partial-candidate-evidence-def*
by (*intro conjI*)
 (*rule partial0, rule candidate0, rule not-low0*)
show *?thesis*
unfolding *trace-fri-header-tied-low-candidate-nonconsumed-openings-gap-def*
by (*intro exI conjI*)
 (*rule gap, rule consumed, rule candidate, rule low, rule evidence0,*
 rule mismatch, rule openings-mismatch)
qed

lemma *checked-staged-security-trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-bound-from-no*

assumes *nonconsumed-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-header-tied-low-candidate-nonconsumed-openings-gap)
adversary-initial-state $\leq N$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap)
adversary-initial-state $\leq N$

```

proof (rule order-trans[OF - nonconsumed-bound])
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap)
    adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-header-tied-low-candidate-nonconsumed-openings-gap)
    adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def
  by (rule wp-event-mono-on-support)
    (auto dest:
     trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-imp-nonconsumed
     split: option.splits prod.splits)
qed

lemma checked-staged-security-trace-fri-sampled-assignment-conflict-bound-structural:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-sampled-assignment-conflict)
      adversary-initial-state  $\leq$ 
      trace-fri-empty-sampled-assignment-structural-error-for budgets A
proof –
  have without:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer)
      adversary-initial-state  $\leq$ 
      reachable-verifier-event-bound-for A
      trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
  by (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    (simp add:
     trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
     accepted-fri-opening-transcript-def,
     rule reachable-verifier-event-bound-for-ge)
  have merkle:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       partial-merkle-inconsistency-bad)
      adversary-initial-state  $\leq$ 
      trace-fri-controlled-merkle-error-for budgets
  unfolding trace-fri-controlled-merkle-error-for-def
  by (rule
     checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
     [OF wf controlled])
  have gap:

```

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-verifier-tied-same-layer-auth-gap)
adversary-initial-state ≤
  reachable-verifier-event-bound-for A
  trace-fri-verifier-tied-same-layer-auth-gap
by (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  (simp add: trace-fri-verifier-tied-same-layer-auth-gap-def
    accepted-fri-opening-transcript-def,
    rule reachable-verifier-event-bound-for-ge)
have tied:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-verifier-tied-sampled-assignment-conflict)
  adversary-initial-state ≤
    (reachable-verifier-event-bound-for A
      trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer +
      trace-fri-controlled-merkle-error-for budgets) +
    reachable-verifier-event-bound-for A
    trace-fri-verifier-tied-same-layer-auth-gap
by (rule
  checked-staged-security-trace-fri-verifier-tied-conflict-bound-from-without-same-merkle-and-auth-gap
    [OF without merkle gap])
have untied:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-sampled-assignment-verifier-untied-gap)
  adversary-initial-state ≤
    reachable-verifier-event-bound-for A
    trace-fri-sampled-assignment-verifier-untied-gap
by (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  (simp add: trace-fri-sampled-assignment-verifier-untied-gap-def
    trace-fri-sampled-assignment-conflict-def
    trace-fri-partial-candidate-opening-evidence-def
    accepted-fri-opening-transcript-def,
    rule reachable-verifier-event-bound-for-ge)
show ?thesis
unfolding trace-fri-empty-sampled-assignment-structural-error-for-def
by (rule
  checked-staged-security-trace-fri-sampled-assignment-conflict-bound-from-verifier-tied-and-untied
    [OF tied untied])
qed

lemma checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets-header-and-openings
fixes H :: prob
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and header-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)

```

(staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state $\leq H$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-zero-round-final-obstruction)
 adversary-initial-state $\leq$
 trace-fri-zero-round-agreement-set-target-error-for budgets +
 trace-fri-controlled-merkle-error-for budgets +
 (H +
 reachable-verifier-event-bound-for A
 trace-fri-header-tied-low-candidate-untied-openings-gap) +
 H

proof –

have checked:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-zero-round-checked-final-obstruction)
 adversary-initial-state $\leq$
 trace-fri-zero-round-agreement-set-target-error-for budgets

by (rule
 checked-staged-security-trace-fri-zero-round-checked-final-obstruction-bound-from-agreement-sets
 [OF wf controlled])

have merkle:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 partial-merkle-inconsistency-bad)
 adversary-initial-state $\leq$
 trace-fri-controlled-merkle-error-for budgets

unfolding trace-fri-controlled-merkle-error-for-def

by (rule
 checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
 [OF wf controlled])

have openings:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-header-tied-low-candidate-untied-openings-gap)
 adversary-initial-state $\leq$
 reachable-verifier-event-bound-for A
 trace-fri-header-tied-low-candidate-untied-openings-gap

proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)

show $\bigwedge s. \neg$

trace-fri-header-tied-low-candidate-untied-openings-gap s None
by (simp add: trace-fri-header-tied-low-candidate-untied-openings-gap-def
 trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
 accepted-fri-opening-transcript-def)

next

fix data attacker-state


```

assume support:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
show wp-event verify-monad
  (trace-fri-header-tied-low-candidate-untied-openings-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  ≤ reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-openings-gap
  by (rule reachable-verifier-event-bound-for-ge[OF support])
qed
have untied:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-untied-reachable-witness-gap)
  adversary-initial-state ≤
    reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-untied-openings-gap
proof –
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-untied-reachable-witness-gap)
  adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-header-tied-low-candidate-untied-openings-gap)
      adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def
  by (rule wp-event-mono-on-support)
    (auto dest: trace-fri-header-tied-low-candidate-untied-gap-imp-openings
      split: option.splits prod.splits)
  also have ... ≤
    reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-untied-openings-gap
  by (rule openings)
  finally show ?thesis .
qed
have transfer:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-candidate-transfer-gap)
  adversary-initial-state ≤
    H +
    reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-untied-openings-gap

```

```

    by (rule
      checked-staged-security-trace-fri-header-tied-candidate-transfer-gap-bound-from-staged-header-and-untied
      [OF header-bound untied])
  show ?thesis
    by (rule
      checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-staged-checked-merkle-transfer-header
      [OF checked merkle transfer header-bound])
qed

lemma checked-staged-security-trace-fri-header-tied-low-candidate-untied-openings-gap-bound-from-hash-and-nonnor
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-header-tied-low-candidate-untied-openings-gap)
    adversary-initial-state ≤
    trace-fri-controlled-merkle-error-for budgets +
    ((reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
      reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
      reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-nonconsumed-query-gap)
  proof -
    let ?M = checked-staged-security-experiment-with-data-state A
    let ?openings =
      staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-untied-openings-gap
    let ?hash =
      staged-security-with-data-state-verifier-event
      hash-map-output-collision-bad
    let ?nonnormal =
      staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap
    let ?nonconsumed =
      staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-nonconsumed-openings-gap
    let ?root =
      staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-nonconsumed-root-gap
    let ?query =
      staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-nonconsumed-query-gap
    let ?irrelevant =
      staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap
    have hash:
      wp-event ?M ?hash adversary-initial-state ≤

```

trace-fri-controlled-merkle-error-for budgets
unfolding *trace-fri-controlled-merkle-error-for-def*
by (rule
checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
[OF wf controlled])
have *root*:
wp-event ?M ?root adversary-initial-state ≤
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-nonconsumed-root-gap
by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
(simp add: *trace-fri-header-tied-low-candidate-nonconsumed-root-gap-def*
accepted-with-partial-trace-openings-verifier-consumed-def
trace-fri-partial-candidate-evidence-def
accepted-fri-opening-transcript-def,
rule *reachable-verifier-event-bound-for-ge*)
have *query*:
wp-event ?M ?query adversary-initial-state ≤
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-nonconsumed-query-gap
by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
(simp add: *trace-fri-header-tied-low-candidate-nonconsumed-query-gap-def*
accepted-with-partial-trace-openings-verifier-consumed-def
trace-fri-partial-candidate-evidence-def
accepted-fri-opening-transcript-def,
rule *reachable-verifier-event-bound-for-ge*)
have *irrelevant*:
wp-event ?M ?irrelevant adversary-initial-state ≤
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap
by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
(simp add:
trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap-def
accepted-with-partial-trace-openings-verifier-consumed-def
trace-fri-partial-candidate-evidence-def
accepted-fri-opening-transcript-def,
rule *reachable-verifier-event-bound-for-ge*)
have *root-refined*:
wp-event ?M ?root adversary-initial-state ≤
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap
by (rule
checked-staged-security-trace-fri-header-tied-low-candidate-nonconsumed-root-gap-bound-from-query-and-irrelevant
[OF query irrelevant])
have *nonconsumed-root-query*:
wp-event ?M ?nonconsumed adversary-initial-state ≤
(*reachable-verifier-event-bound-for A*
trace-fri-header-tied-low-candidate-nonconsumed-query-gap +

```

    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-query-gap
  by (rule
    checked-staged-security-trace-fri-header-tied-low-candidate-nonconsumed-openings-gap-bound-from-root-query
    [OF root-refined query])
  have nonnormal-root-query:
    wp-event ?M ?nonnormal adversary-initial-state ≤
      (reachable-verifier-event-bound-for A
        trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
        reachable-verifier-event-bound-for A
        trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
        reachable-verifier-event-bound-for A
        trace-fri-header-tied-low-candidate-nonconsumed-query-gap
  by (rule
    checked-staged-security-trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-bound-from-nonconsumed-root-query
    [OF nonconsumed-root-query])
  have wp-event ?M ?openings adversary-initial-state ≤
    wp-event ?M (λout. ?hash out ∨ ?nonnormal out)
    adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def
  by (rule wp-event-mono-on-support)
    (auto
      dest:
        trace-fri-header-tied-low-candidate-untied-openings-gap-imp-normalized-or-nonnormalized
        trace-fri-header-tied-low-candidate-untied-normalized-openings-gap-imp-hash-collision
        split: option.splits prod.splits)
  also have ... ≤
    wp-event ?M ?hash adversary-initial-state +
    wp-event ?M ?nonnormal adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤
    trace-fri-controlled-merkle-error-for budgets +
    ((reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
      reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
      reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-nonconsumed-query-gap)
  by (rule add-mono[OF hash nonnormal-root-query])
  finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-zero-round-verifier-consumed-low-candidate-gap-bound-from-hash-and-nonnormal
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (staged-security-with-data-state-verifier-event
      trace-fri-zero-round-verifier-consumed-low-candidate-gap)
  adversary-initial-state ≤
  trace-fri-controlled-merkle-error-for budgets +
  ((reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-query-gap)
proof –
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-zero-round-verifier-consumed-low-candidate-gap)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-untied-openings-gap)
  adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
by (rule wp-event-mono-on-support)
  (auto
    dest:
      trace-fri-zero-round-verifier-consumed-low-candidate-gap-imp-untied-openings
    split: option.splits prod.splits)
also have ... ≤
  trace-fri-controlled-merkle-error-for budgets +
  ((reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-query-gap)
by (rule
  checked-staged-security-trace-fri-header-tied-low-candidate-untied-openings-gap-bound-from-hash-and-nonnormal
  [OF wf controlled])
finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets-header-and-nonnormal
fixes H :: prob
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and header-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state ≤ H
shows

```

$wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad trace\text{-}fri\text{-}zero\text{-}round\text{-}final\text{-}obstruction)$
 $adversary\text{-}initial\text{-}state \leq$
 $trace\text{-}fri\text{-}zero\text{-}round\text{-}agreement\text{-}set\text{-}target\text{-}error\text{-}for \ budgets +$
 $trace\text{-}fri\text{-}controlled\text{-}merkle\text{-}error\text{-}for \ budgets +$
 $(trace\text{-}fri\text{-}controlled\text{-}merkle\text{-}error\text{-}for \ budgets +$
 $\quad ((reachable\text{-}verifier\text{-}event\text{-}bound\text{-}for \ A$
 $\quad \quad trace\text{-}fri\text{-}header\text{-}tied\text{-}low\text{-}candidate\text{-}nonconsumed\text{-}query\text{-}gap +$
 $\quad \quad reachable\text{-}verifier\text{-}event\text{-}bound\text{-}for \ A$
 $\quad \quad trace\text{-}fri\text{-}header\text{-}tied\text{-}low\text{-}candidate\text{-}nonconsumed\text{-}irrelevant\text{-}root\text{-}gap) +$
 $\quad \quad reachable\text{-}verifier\text{-}event\text{-}bound\text{-}for \ A$
 $\quad \quad trace\text{-}fri\text{-}header\text{-}tied\text{-}low\text{-}candidate\text{-}nonconsumed\text{-}query\text{-}gap))) +$
 H
proof –
have *checked*:
 $wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad trace\text{-}fri\text{-}zero\text{-}round\text{-}checked\text{-}final\text{-}obstruction)$
 $adversary\text{-}initial\text{-}state \leq$
 $trace\text{-}fri\text{-}zero\text{-}round\text{-}agreement\text{-}set\text{-}target\text{-}error\text{-}for \ budgets$
by (rule
 $checked\text{-}staged\text{-}security\text{-}trace\text{-}fri\text{-}zero\text{-}round\text{-}checked\text{-}final\text{-}obstruction\text{-}bound\text{-}from\text{-}agreement\text{-}sets$
 $\quad [OF \ wf \ controlled])$
have *merkle*:
 $wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad partial\text{-}merkle\text{-}inconsistency\text{-}bad)$
 $adversary\text{-}initial\text{-}state \leq$
 $trace\text{-}fri\text{-}controlled\text{-}merkle\text{-}error\text{-}for \ budgets$
unfolding $trace\text{-}fri\text{-}controlled\text{-}merkle\text{-}error\text{-}for\text{-}def$
by (rule
 $checked\text{-}staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}partial\text{-}merkle\text{-}inconsistency\text{-}bad\text{-}bound\text{-}controlled$
 $\quad [OF \ wf \ controlled])$
have *openings*:
 $wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad trace\text{-}fri\text{-}zero\text{-}round\text{-}verifier\text{-}consumed\text{-}low\text{-}candidate\text{-}gap)$
 $adversary\text{-}initial\text{-}state \leq$
 $trace\text{-}fri\text{-}controlled\text{-}merkle\text{-}error\text{-}for \ budgets +$
 $((reachable\text{-}verifier\text{-}event\text{-}bound\text{-}for \ A$
 $\quad \quad trace\text{-}fri\text{-}header\text{-}tied\text{-}low\text{-}candidate\text{-}nonconsumed\text{-}query\text{-}gap +$
 $\quad \quad reachable\text{-}verifier\text{-}event\text{-}bound\text{-}for \ A$
 $\quad \quad trace\text{-}fri\text{-}header\text{-}tied\text{-}low\text{-}candidate\text{-}nonconsumed\text{-}irrelevant\text{-}root\text{-}gap) +$
 $\quad \quad reachable\text{-}verifier\text{-}event\text{-}bound\text{-}for \ A$
 $\quad \quad trace\text{-}fri\text{-}header\text{-}tied\text{-}low\text{-}candidate\text{-}nonconsumed\text{-}query\text{-}gap)$
by (rule
 $checked\text{-}staged\text{-}security\text{-}trace\text{-}fri\text{-}zero\text{-}round\text{-}verifier\text{-}consumed\text{-}low\text{-}candidate\text{-}gap\text{-}bound\text{-}from\text{-}hash\text{-}and\text{-}nonnorm$
 $\quad [OF \ wf \ controlled])$

have zero:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
trace-fri-zero-round-final-obstruction)
adversary-initial-state $\leq$
trace-fri-zero-round-agreement-set-target-error-for budgets +
trace-fri-controlled-merkle-error-for budgets +
(trace-fri-controlled-merkle-error-for budgets +
((reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-nonconsumed-query-gap)) +
H
by (rule
checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-staged-checked-merkle-low-consumed-he
[OF checked merkle openings header-bound])
show ?thesis
by (rule zero)
qed

definition *trace-empty-header-sampled-split-one-step-error-for*
where
trace-empty-header-sampled-split-one-step-error-for budgets A empty-bad =
trace-one-step-sampled-error-for budgets empty-bad +
(trace-fri-empty-sampled-assignment-structural-error-for budgets A +
trace-empty-header-openings-zero-error-for budgets A empty-bad)

definition *concrete-trace-empty-header-sampled-split-one-step-error-for*
where
concrete-trace-empty-header-sampled-split-one-step-error-for budgets A =
trace-empty-header-sampled-split-one-step-error-for budgets A
trace-one-step-split-bad

lemma *trace-empty-header-sampled-split-one-step-error-for-bound:*
fixes *empty-bad :: nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *empty-subset:*
 $\bigwedge i$ prefix.
 $i < \text{fri-round-count-for-degree-bound } (\text{clength} - 1) \implies$
length prefix = $i \implies$
empty-bad i prefix $\subseteq$
fri-one-step-disagreement-challenges
(trace-fri-canonical-layer-len i) (empty-pw i)
(empty-layer i prefix) (empty-claimed i prefix)
(empty-dom i prefix)
shows

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state
 $\leq$ *trace-empty-header-sampled-split-one-step-error-for budgets* A *empty-bad*

proof –

have *sampled*:

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-sampled-query-bad-candidate)
adversary-initial-state
 $\leq$ *trace-one-step-sampled-error-for budgets* *empty-bad*
by (rule *trace-one-step-sampled-error-for-bound*
 [*OF wf controlled empty-subset*])

have *conflict-bound*:

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-sampled-assignment-conflict)
adversary-initial-state
 $\leq$ *trace-fri-empty-sampled-assignment-structural-error-for budgets* A
by (rule
checked-staged-security-trace-fri-sampled-assignment-conflict-bound-structural
 [*OF wf controlled*])

have *header-bound*:

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state
 $\leq$ *trace-header-one-step-error-for budgets* A *empty-bad*
by (rule *trace-header-one-step-error-for-bound*
 [*OF wf controlled empty-subset*])

have *zero-bound*:

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-zero-round-final-obstruction)
adversary-initial-state
 $\leq$ *trace-empty-header-openings-zero-error-for budgets* A *empty-bad*
unfolding *trace-empty-header-openings-zero-error-for-def*
by (rule
checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets-header-and-openings
 [*OF wf controlled header-bound*])

have *branch*:

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state
 $\leq$ *trace-one-step-sampled-error-for budgets* *empty-bad* +
 (*trace-fri-empty-sampled-assignment-structural-error-for budgets* A +
trace-empty-header-openings-zero-error-for budgets A *empty-bad*)

by (rule
 checked-staged-security-trace-fri-empty-header-bound-from-staged-sampled-conflict-and-zero
 [OF sampled conflict-bound zero-bound])
 show ?thesis
 using branch
 by (simp add: trace-empty-header-sampled-split-one-step-error-for-def)
 qed

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-sampled-split-one-step-fr
fixes trace-bad empty-bad :: nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set
and comp-bad :: 'f $\Rightarrow$ nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-subset:
 $\bigwedge i$ prefix.
 $i < \text{fri-round-count-for-degree-bound } (\text{clength } - 1) \Rightarrow$
 $\text{length prefix} = i \Rightarrow$
 $\text{trace-bad } i \text{ prefix} \subseteq$
 fri-one-step-disagreement-challenges
 (trace-fri-canonical-layer-len i) (empty-pw i)
 (empty-layer i prefix) (empty-claimed i prefix)
 (empty-dom i prefix)
and comp-subset:
 $\bigwedge dg$ i prefix.
 $i < \text{fri-round-count-for-degree-bound } (\text{to-nat } dg) \Rightarrow$
 $\text{length prefix} = i \Rightarrow$
 $\text{comp-bad } dg \text{ i prefix} \subseteq$
 fri-one-step-disagreement-challenges
 (composition-fri-canonical-layer-len dg i) (comp-pw dg i)
 (comp-layer dg i prefix) (comp-claimed dg i prefix)
 (comp-dom dg i prefix)
and empty-subset:
 $\bigwedge i$ prefix.
 $i < \text{fri-round-count-for-degree-bound } (\text{clength } - 1) \Rightarrow$
 $\text{length prefix} = i \Rightarrow$
 $\text{empty-bad } i \text{ prefix} \subseteq$
 fri-one-step-disagreement-challenges
 (trace-fri-canonical-layer-len i) (empty-pw i)
 (empty-layer i prefix) (empty-claimed i prefix)
 (empty-dom i prefix)
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0

(staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 ($\sum i < \text{rounds}$.
 active-route-current-prefix-authenticated-error-for A i +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 trace-header-one-step-error-for budgets A trace-bad +
 composition-one-step-verifier-tied-controlled-error-for budgets
 comp-bad +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 trace-empty-header-sampled-split-one-step-error-for budgets A empty-bad +
 active-route-current-empty-query-error-for A)

proof –

have trace-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state
 $\leq$ trace-header-one-step-error-for budgets A trace-bad

by (rule trace-header-one-step-error-for-bound
 [OF wf controlled trace-subset])

have comp-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state
 $\leq$ composition-one-step-verifier-tied-controlled-error-for budgets
 comp-bad

by (rule composition-one-step-verifier-tied-controlled-error-for-bound
 [OF wf controlled comp-subset])

have empty-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state
 $\leq$ trace-empty-header-sampled-split-one-step-error-for budgets A empty-bad

by (rule trace-empty-header-sampled-split-one-step-error-for-bound
 [OF wf controlled empty-subset])
 show ?thesis
 by (rule
 stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-bounds
 [OF false-statement wf controlled trace-bound comp-bound empty-bound])
 qed

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-sampled-split-endpoint:
 assumes false-statement: $\neg$ exists-valid-trace
 and wf: staged-budget-wellformed budgets
 and controlled: staged-adversary-controlled budgets A
 shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 ($\sum i < \text{rounds.}$
 active-route-current-prefix-authenticated-error-for A i +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 concrete-trace-header-one-step-error-for budgets A +
 concrete-composition-one-step-error-for budgets A +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 concrete-trace-empty-header-sampled-split-one-step-error-for budgets A +
 active-route-current-empty-query-error-for A)
 unfolding concrete-trace-header-one-step-error-for-def
 concrete-composition-one-step-error-for-def
 concrete-trace-empty-header-sampled-split-one-step-error-for-def
 by (rule
 stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-sampled-split-one-step-fri-comp

[where $\text{trace-bad} = \text{trace-one-step-split-bad}$
 and $\text{comp-bad} = \text{composition-one-step-split-bad}$
 and $\text{empty-bad} = \text{trace-one-step-split-bad}$
 and $\text{empty-pw} = \text{trace-one-step-split-pw}$
 and $\text{empty-layer} = \text{trace-one-step-split-layer}$
 and $\text{empty-claimed} = \text{trace-one-step-split-claimed}$
 and $\text{empty-dom} = \text{trace-one-step-split-domain}$
 and $\text{comp-pw} = \text{composition-one-step-split-pw}$
 and $\text{comp-layer} = \text{composition-one-step-split-layer}$
 and $\text{comp-claimed} = \text{composition-one-step-split-claimed}$
 and $\text{comp-dom} = \text{composition-one-step-split-domain}$,
 OF false-statement wf controlled])
 (simp-all add:
 $\text{trace-one-step-split-bad-def}$
 $\text{composition-one-step-split-bad-def}$
 $\text{trace-one-step-disagreement-bad-for-def}$
 $\text{composition-one-step-disagreement-bad-for-def}$)

definition *active-route-current-prefix-conceptual-residual-error-for*
where

active-route-current-prefix-conceptual-residual-error-for
 ($A :: 'f \text{ staged-adversary}$) ($i :: \text{nat}$) =
 wp-event
 ($\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i$)
 checked-staged-security-with-query-prefix-conceptual-context-residual-at
 adversary-initial-state

definition *active-route-current-prefix-conceptual-hit-residual-error-for*
where

active-route-current-prefix-conceptual-hit-residual-error-for
 ($A :: 'f \text{ staged-adversary}$) ($i :: \text{nat}$) =
 wp-event
 ($\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i$)
 ($\lambda \text{out.}$
 checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
 $i \ \text{out} \wedge$
 checked-staged-security-with-query-prefix-conceptual-context-residual-at
 out)
 adversary-initial-state

definition *active-route-current-prefix-conceptual-bound-for*
where

active-route-current-prefix-conceptual-bound-for
 ($\text{budgets} :: \text{staged-budgets}$) ($A :: 'f \text{ staged-adversary}$)
 ($i :: \text{nat}$) =
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound +
 active-route-current-prefix-conceptual-hit-residual-error-for $A \ i$

lemma *active-route-current-prefix-authenticated-error-for-le-conceptual-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

shows

active-route-current-prefix-authenticated-error-for A i ≤

active-route-current-prefix-conceptual-bound-for budgets A i

proof –

let *?M = checked-staged-security-experiment-with-query-prefix-data-state A i*

let *?Hit =*

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
i

let *?Conceptual =*

checked-staged-security-with-query-prefix-conceptual-target-hit

let *?Residual =*

λout. ?Hit out ∧
checked-staged-security-with-query-prefix-conceptual-context-residual-at
out

have *event-le:*

wp-event ?M ?Hit adversary-initial-state ≤

wp-event ?M (λout. ?Conceptual out ∨ ?Residual out)

adversary-initial-state

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support: out ∈ set-dist (execute ?M adversary-initial-state)*

and *hit: ?Hit out*

from *checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-imp-conceptual-or-residual-on-suppo*

[OF wf controlled i-bound support hit]

show *?Conceptual out ∨ ?Residual out*

using *hit by blast*

qed

have *conceptual-bound:*

wp-event ?M ?Conceptual adversary-initial-state ≤

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) + query-error-bound

by (rule

checked-staged-security-with-query-prefix-conceptual-target-hit-bound

[OF wf controlled i-bound])

have *union-bound:*

wp-event ?M (λout. ?Conceptual out ∨ ?Residual out)

adversary-initial-state ≤

wp-event ?M ?Conceptual adversary-initial-state +

wp-event ?M ?Residual adversary-initial-state

by (rule *wp-event-union-bound*)

have *residual-bound:*

wp-event ?M ?Residual adversary-initial-state ≤

active-route-current-prefix-conceptual-hit-residual-error-for A i

by (*simp add:*

$\text{active-route-current-prefix-conceptual-hit-residual-error-for-def}$)
have sum-bound :
 $\text{wp-event } ?M \text{ ?Conceptual adversary-initial-state} +$
 $\text{wp-event } ?M \text{ ?Residual adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{active-route-current-prefix-conceptual-hit-residual-error-for } A \ i$
by ($\text{intro add-mono conceptual-bound residual-bound}$)
show $?thesis$
unfolding $\text{active-route-current-prefix-authenticated-error-for-def}$
 $\text{active-route-current-prefix-conceptual-bound-for-def}$
by ($\text{rule order-trans[OF event-le]}$)
 $(\text{rule order-trans[OF union-bound sum-bound]})$
qed

definition $\text{active-route-current-empty-conceptual-residual-error-for}$
 $:: 'f \text{ itself} \Rightarrow 'f \text{ staged-adversary} \Rightarrow \text{prob}$
where
 $\text{active-route-current-empty-conceptual-residual-error-for}$
 $(- :: 'f \text{ itself})$
 $(A :: 'f \text{ staged-adversary}) =$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ 0)$
 $(\text{checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at}$
 $(0 :: \text{nat}))$
 $\text{adversary-initial-state}$

definition $\text{active-route-current-empty-conceptual-bound-for}$
where
 $\text{active-route-current-empty-conceptual-bound-for}$
 $(\text{budgets} :: \text{staged-budgets}) (A :: 'f \text{ staged-adversary}) =$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } 0 + 1) +$
 $\text{query-error-bound} +$
 $\text{active-route-current-empty-conceptual-residual-error-for TYPE('f) } A +$
 $(\text{active-route-transcript-pre-error-for } A +$
 $\text{staged-concrete-transcript-target-error-bound})$

lemma $\text{active-route-current-empty-query-error-for-le-conceptual-bound}$:
assumes $\text{wf: staged-budget-wellformed budgets}$
and $\text{controlled: staged-adversary-controlled budgets } A$
shows
 $\text{active-route-current-empty-query-error-for } A \leq$
 $\text{active-route-current-empty-conceptual-bound-for budgets } A$
proof –
have $i\text{-bound: } 0 < \text{rounds}$
using rounds-positive **by** simp
have residual-bound :

$wp\text{-}event$
 (checked-staged-security-experiment-with-query-prefix-data-state A 0)
 (checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
 (0 :: nat))
 $adversary\text{-}initial\text{-}state \leq$
 $active\text{-}route\text{-}current\text{-}empty\text{-}conceptual\text{-}residual\text{-}error\text{-}for\ TYPE('f) A$
unfolding active-route-current-empty-conceptual-residual-error-for-def
by (rule wp-event-mono) simp
have pre-bound:
 $wp\text{-}event$ (checked-staged-security-experiment-with-data-state A)
 $staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}transcript\text{-}pre\text{-}hit$
 $adversary\text{-}initial\text{-}state \leq active\text{-}route\text{-}transcript\text{-}pre\text{-}error\text{-}for A$
by (simp add: active-route-transcript-pre-error-for-def)
have new-bound:
 $wp\text{-}event$ (checked-staged-security-experiment-with-data-state A)
 $staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}transcript\text{-}new\text{-}hit$
 $adversary\text{-}initial\text{-}state \leq staged\text{-}concrete\text{-}transcript\text{-}target\text{-}error\text{-}bound$
by (rule checked-staged-security-with-data-state-transcript-new-hit-bound
 [OF wf controlled])
show ?thesis
unfolding active-route-current-empty-query-error-for-def
 $active\text{-}route\text{-}current\text{-}empty\text{-}conceptual\text{-}bound\text{-}for\text{-}def$
by (rule
 checked-staged-security-with-data-state-current-empty-query-opening-hit-at-bound-from-conceptual-empty-residual
 [OF wf controlled i-bound residual-bound pre-bound new-bound])
qed

definition trace-no-full-cover-sampled-query-error-for

where

$trace\text{-}no\text{-}full\text{-}cover\text{-}sampled\text{-}query\text{-}error\text{-}for$ budgets bad =
 $(\sum challenges \in fri\text{-}challenge\text{-}space\ (ceil\text{-}log\ clength).$
 $(1 / nnreal\ (CARD('f) \wedge ceil\text{-}log\ clength)) *$
 $(nnreal$
 $(card$
 $(trace\text{-}fri\text{-}sampled\text{-}query\text{-}no\text{-}full\text{-}cover\text{-}failure\text{-}query\text{-}fiber$
 $bad\ challenges)) *$
 $(1 / nnreal\ (card\ query\text{-}sample\text{-}space)) \wedge rounds)) +$
 $hash\text{-}relation\text{-}budget\text{-}value$
 $(query\text{-}index\text{-}raw\text{-}list\text{-}relation\text{-}fiber\text{-}bound$
 $(\bigcup challenges \in fri\text{-}challenge\text{-}space\ (ceil\text{-}log\ clength).$
 $trace\text{-}fri\text{-}sampled\text{-}query\text{-}no\text{-}full\text{-}cover\text{-}failure\text{-}query\text{-}fiber$
 $bad\ challenges))$
 $(staged\text{-}attacker\text{-}query\text{-}budget\ budgets + staged\text{-}challenge\text{-}query\text{-}budget) +$
 $hash\text{-}relation\text{-}budget\text{-}value$
 $(card\ (fri\text{-}challenge\text{-}space\ (ceil\text{-}log\ clength)) * ceil\text{-}log\ clength)$
 $(staged\text{-}attacker\text{-}query\text{-}budget\ budgets + staged\text{-}challenge\text{-}query\text{-}budget)$

definition trace-canonical-full-cover-shape-failure-error-for

where

$\text{trace-canonical-full-cover-shape-failure-error-for } A \text{ bad} =$
 $\text{reachable-verifier-event-bound-for } A$
 $(\lambda s. \text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad}) +$
 $\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-sampled-full-cover-index-failure} +$
 $\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-sampled-full-cover-shape-failure}$

definition *trace-restricted-sampled-query-error-for*
where

$\text{trace-restricted-sampled-query-error-for budgets } A \text{ bad} =$
 $\text{trace-no-full-cover-sampled-query-error-for budgets } \text{bad} +$
 $\text{trace-canonical-full-cover-shape-failure-error-for } A \text{ bad}$

lemma *trace-restricted-sampled-query-error-for-bound:*

assumes *wf: staged-budget-wellformed budgets*

and controlled: *staged-adversary-controlled budgets A*

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-sampled-query-bad-candidate})$
 $\text{adversary-initial-state}$
 $\leq \text{trace-restricted-sampled-query-error-for budgets } A \text{ bad}$

proof –

have *no-full:*

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad}))$
 $\text{adversary-initial-state}$

$\leq \text{trace-no-full-cover-sampled-query-error-for budgets } \text{bad}$

unfolding *trace-no-full-cover-sampled-query-error-for-def*

by (rule

$\text{checked-staged-security-trace-fri-sampled-query-no-full-cover-failure-candidate-bound-from-restricted-pairs}$
 $[\text{OF wf controlled}])$

have *failure:*

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-sampled-canonical-full-cover-failure } s \text{ bad}))$
 $\text{adversary-initial-state}$

$\leq \text{trace-canonical-full-cover-shape-failure-error-for } A \text{ bad}$

unfolding *trace-canonical-full-cover-shape-failure-error-for-def*

proof (rule

$\text{checked-staged-security-trace-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases})$

fix *data attacker-state*

assume *support:*

$\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$

show *wp-event verify-monad*


```

    (trace-fri-sampled-full-cover-proximity-failure
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) bad)
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data))
  ≤ reachable-verifier-event-bound-for A
  (λs. trace-fri-sampled-full-cover-proximity-failure s bad)
  by (rule reachable-verifier-event-bound-for-ge[OF support])
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (trace-fri-sampled-full-cover-index-failure
   (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))
  ≤ reachable-verifier-event-bound-for A
  trace-fri-sampled-full-cover-index-failure
  by (rule reachable-verifier-event-bound-for-ge[OF support])
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (trace-fri-sampled-full-cover-shape-failure
   (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))
  ≤ reachable-verifier-event-bound-for A
  trace-fri-sampled-full-cover-shape-failure
  by (rule reachable-verifier-event-bound-for-ge[OF support])
qed
have sampled:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-sampled-query-bad-candidate)
    adversary-initial-state
  ≤ trace-no-full-cover-sampled-query-error-for budgets bad +
    trace-canonical-full-cover-shape-failure-error-for A bad
  by (rule
    checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-canonical-failure
      [OF no-full failure])

```

show *?thesis*
using *sampled*
by (*simp add: trace-restricted-sampled-query-error-for-def*)
qed

definition *concrete-sampled-split-stark-soundness-bound-for*
where

concrete-sampled-split-stark-soundness-bound-for budgets $A =$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* budgets +
 staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
 staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* budgets 0) +
 staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
 (*active-route-transcript-pre-error-for* A +
 staged-concrete-transcript-target-error-bound)) +
 ($\sum i < \text{rounds}.$
 active-route-current-prefix-conceptual-bound-for budgets A i +
 (*active-route-transcript-pre-error-for* A +
 staged-concrete-transcript-target-error-bound) +
 (*active-route-transcript-pre-error-for* A +
 staged-concrete-transcript-target-error-bound)) +
concrete-trace-header-one-step-error-for budgets A +
concrete-composition-one-step-error-for budgets A +
 (*composition-error-bound* +
 staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* budgets 0) +
 staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
 (*active-route-transcript-pre-error-for* A +
 staged-concrete-transcript-target-error-bound) +
concrete-trace-empty-header-sampled-split-one-step-error-for budgets A +
active-route-current-empty-conceptual-bound-for budgets A)

theorem *stark-soundness-trace-empty-sampled-split-endpoint:*

assumes *false-statement:* $\neg \text{exists-valid-trace}$

and *wf:* *staged-budget-wellformed* budgets

and *controlled:* *staged-adversary-controlled* budgets A

shows *checked-staged-adversary-acceptance-probability* $A \leq$
concrete-sampled-split-stark-soundness-bound-for budgets A

proof –

have *old-bound:*

checked-staged-adversary-acceptance-probability $A \leq$

hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
($\sum_{i < \text{rounds}}$.
active-route-current-prefix-authenticated-error-for A i +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
concrete-trace-header-one-step-error-for budgets A +
concrete-composition-one-step-error-for budgets A +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
concrete-trace-empty-header-sampled-split-one-step-error-for budgets A +
active-route-current-empty-query-error-for A)
by (rule
stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-sampled-split-endpoint
[OF false-statement wf controlled])
have *prefix-terms:*
($\sum_{i < \text{rounds}}$.
active-route-current-prefix-authenticated-error-for A i +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound))
 $\leq$
($\sum_{i < \text{rounds}}$.
active-route-current-prefix-conceptual-bound-for budgets A i +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound))
by (intro sum-mono add-mono order-refl

$\text{active-route-current-prefix-authenticated-error-for-le-conceptual-bound}$
 $[OF \text{ wf controlled}]$)
 simp
have empty-term :
 $\text{active-route-current-empty-query-error-for } A \leq$
 $\text{active-route-current-empty-conceptual-bound-for budgets } A$
by (rule $\text{active-route-current-empty-query-error-for-le-conceptual-bound}$
 $[OF \text{ wf controlled}]$)
have old-le-new :
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(\text{active-route-transcript-pre-error-for } A +$
 $\text{staged-concrete-transcript-target-error-bound})) +$
 $(\sum_{i < \text{rounds.}}$
 $\text{active-route-current-prefix-authenticated-error-for } A \text{ } i +$
 $(\text{active-route-transcript-pre-error-for } A +$
 $\text{staged-concrete-transcript-target-error-bound}) +$
 $(\text{active-route-transcript-pre-error-for } A +$
 $\text{staged-concrete-transcript-target-error-bound})) +$
 $\text{concrete-trace-header-one-step-error-for budgets } A +$
 $\text{concrete-composition-one-step-error-for budgets } A +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(\text{active-route-transcript-pre-error-for } A +$
 $\text{staged-concrete-transcript-target-error-bound}) +$
 $\text{concrete-trace-empty-header-sampled-split-one-step-error-for budgets } A +$
 $\text{active-route-current-empty-query-error-for } A)$
 $\leq \text{concrete-sampled-split-stark-soundness-bound-for budgets } A$
unfolding $\text{concrete-sampled-split-stark-soundness-bound-for-def}$
by (intro add-mono order-refl prefix-terms empty-term)
show $?thesis$
by (rule $\text{order-trans}[OF \text{ old-bound old-le-new}]$)
qed
end

hide-fact

soundness.stark-soundness-trace-empty-sampled-split-endpoint

soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-sampled-split-endpoint

end

theory *Soundness-FRI-Trace-Restricted-Query-Endpoint*

imports *Soundness-FRI-Trace-Empty-Sampled-Assignment-Split*

begin

Public entry point for the staged STARK soundness theorem. The theorem in this theory combines the staged adversary experiment, Merkle authentication reductions, query bounds, composition randomization bounds, and the FRI query/challenge accounting into one explicit probability bound.

The bound uses restricted trace and composition product-accounting terms and the proved zero-round trace empty-header estimates from the preceding FRI theories.

context *soundness*

begin

definition *trace-header-restricted-one-step-error-for*

where

trace-header-restricted-one-step-error-for budgets A trace-bad =
trace-restricted-sampled-query-error-for budgets A trace-bad +
trace-fri-header-replay-conflict-zero-error-for budgets A

definition *concrete-trace-header-restricted-one-step-error-for*

where

concrete-trace-header-restricted-one-step-error-for budgets A =
trace-header-restricted-one-step-error-for budgets A
*(λ -. *trace-one-step-split-bad*)*

lemma *trace-fri-header-tied-sampled-assignment-conflict-replay-bound:*

up-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-header-tied-sampled-assignment-conflict)
adversary-initial-state

$\leq$ *trace-fri-header-replay-conflict-error-for A*

proof (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)

fix *data attacker-state*

assume *support:*

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)

let *?s =*

verifier-state-from-adversary attacker-state
(staged-proof-transcript data)

let *?M =*

```

    reachable-verifier-event-bound-for A partial-merkle-inconsistency-bad
let ?H =
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-recorded-sibling-candidate-missing
have merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
    ≤ ?M
    by (rule reachable-verifier-event-bound-for-ge[OF support])
have recorded-missing-bound:
    wp-event verify-monad
    (trace-fri-header-tied-recorded-sibling-candidate-missing ?s) ?s
    ≤ ?H
    by (rule reachable-verifier-event-bound-for-ge[OF support])
have head-neq:
    wp-event verify-monad
    (trace-fri-header-tied-base-head-value-neq-gap ?s) ?s ≤ ?M
    by (rule
        wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
        (rule merkle-bound)
have head-mismatch:
    wp-event verify-monad
    (trace-fri-header-tied-base-head-value-mismatch-gap ?s) ?s
    ≤ ?M
    by (rule
        wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
        [OF head-neq])
have alignment:
    wp-event verify-monad
    (trace-fri-header-tied-base-value-alignment-gap ?s) ?s ≤ ?M
    by (rule
        wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
        [OF head-mismatch])
have next-bound:
    wp-event verify-monad
    (trace-fri-header-tied-next-value-replay-gap ?s) ?s ≤ ?M + 0
    by (rule
        wp-trace-fri-header-tied-next-value-replay-gap-bound-from-merkle-and-slot)
        (rule merkle-bound,
        rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have successor:
    wp-event verify-monad
    (trace-fri-header-tied-successor-replay-gap ?s) ?s ≤ ?M + 0
    by (rule
        wp-trace-fri-header-tied-successor-replay-gap-bound-from-merkle-and-slot)
        (rule merkle-bound,
        rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have final:
    wp-event verify-monad
    (trace-fri-header-tied-final-value-replay-gap ?s) ?s ≤ ?M + 0 + 0

```

by (rule
 $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}final\text{-}value\text{-}replay\text{-}gap\text{-}bound\text{-}from\text{-}merkle\text{-}slot\text{-}and\text{-}selected\text{-}step$)
(rule $merkle\text{-}bound$,
rule $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}authenticated\text{-}slot\text{-}recorded\text{-}chunk\text{-}gap\text{-}zero$,
rule $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}final\text{-}value\text{-}selected\text{-}step\text{-}missing\text{-}gap\text{-}zero$)
have $slot$:
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}header\text{-}tied\text{-}authenticated\text{-}slot\text{-}recorded\text{-}chunk\text{-}gap\ ?s)\ ?s$
 ≤ 0
by (rule $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}authenticated\text{-}slot\text{-}recorded\text{-}chunk\text{-}gap\text{-}zero$)
have $authenticated\text{-}recorded\text{-}auth$:
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}header\text{-}tied\text{-}same\text{-}layer\text{-}authenticated\text{-}recorded\text{-}auth\text{-}gap\ ?s)$
 $?s \leq 0$
by (rule
 $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}same\text{-}layer\text{-}authenticated\text{-}recorded\text{-}auth\text{-}gap\text{-}bound\text{-}from\text{-}slot\text{-}recorded\text{-}chunk\text{-}gap$
 $[OF\ slot]$)
have $recorded\text{-}auth$:
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}header\text{-}tied\text{-}same\text{-}layer\text{-}recorded\text{-}auth\text{-}gap\ ?s)\ ?s \leq 0$
by (rule
 $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}same\text{-}layer\text{-}recorded\text{-}auth\text{-}gap\text{-}bound\text{-}from\text{-}authenticated\text{-}recorded\text{-}auth\text{-}gap$
 $[OF\ authenticated\text{-}recorded\text{-}auth]$)
have $same\text{-}auth$:
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}header\text{-}tied\text{-}same\text{-}layer\text{-}auth\text{-}gap\ ?s)\ ?s \leq 0$
by (rule
 $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}same\text{-}layer\text{-}auth\text{-}gap\text{-}bound\text{-}from\text{-}recorded\text{-}auth\text{-}gap$
 $[OF\ recorded\text{-}auth]$)
have $auth\text{-}gap$:
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}header\text{-}tied\text{-}assignment\text{-}auth\text{-}gap\ ?s)\ ?s \leq 0$
by (rule
 $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}assignment\text{-}auth\text{-}gap\text{-}bound\text{-}from\text{-}same\text{-}layer\text{-}auth\text{-}gap$
 $[OF\ same\text{-}auth]$)
have $complement$:
 $wp\text{-}event\ verify\text{-}monad$
 $(\lambda out.$
 $trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\ ?s\ out \wedge$
 $\neg trace\text{-}fri\text{-}header\text{-}tied\text{-}recorded\text{-}sibling\text{-}candidate\text{-}missing\ ?s\ out)$
 $?s \leq$
 $(((((?M + ?M) + (?M + 0) + (?M + 0) + (?M + 0 + 0)) + ?M) + 0))$
by (rule
 $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\text{-}without\text{-}recorded\text{-}missing\text{-}bound\text{-}from\text{-}auth\text{-}gap\text{-}and\text{-}replay\text{-}g$
(rule $auth\text{-}gap$, rule $merkle\text{-}bound$, rule $alignment$,
rule $next\text{-}bound$, rule $successor$, rule $final$)
show $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\ ?s)\ ?s$
 $\leq trace\text{-}fri\text{-}header\text{-}replay\text{-}conflict\text{-}error\text{-}for\ A$

unfolding *trace-fri-header-replay-conflict-error-for-def Let-def*
by (rule
wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-recorded-missing-and-complement)
(rule *recorded-missing-bound*, rule *complement*)
next
fix *s*
show $\neg$ *trace-fri-header-tied-sampled-assignment-conflict s None*
unfolding *trace-fri-header-tied-sampled-assignment-conflict-def*
accepted-fri-opening-transcript-def
by *simp*
qed

lemma *trace-fri-header-tied-zero-round-final-obstruction-agreement-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(*staged-security-with-data-state-verifier-event*
trace-fri-header-tied-zero-round-final-obstruction)
adversary-initial-state
 $\leq$ *trace-fri-zero-round-agreement-set-target-error-for budgets* +
trace-fri-controlled-merkle-error-for budgets

proof —

have *checked:*

wp-event (checked-staged-security-experiment-with-data-state A)
(*staged-security-with-data-state-verifier-event*
trace-fri-zero-round-checked-final-obstruction)
adversary-initial-state
 $\leq$ *trace-fri-zero-round-agreement-set-target-error-for budgets*
by (rule
checked-staged-security-trace-fri-zero-round-checked-final-obstruction-bound-from-agreement-sets
[*OF wf controlled*])

have *merkle:*

wp-event (checked-staged-security-experiment-with-data-state A)
(*staged-security-with-data-state-verifier-event*
partial-merkle-inconsistency-bad)
adversary-initial-state
 $\leq$ *trace-fri-controlled-merkle-error-for budgets*

unfolding *trace-fri-controlled-merkle-error-for-def*

by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[*OF wf controlled*])

have *split:*

wp-event (checked-staged-security-experiment-with-data-state A)
(*staged-security-with-data-state-verifier-event*
trace-fri-header-tied-zero-round-final-obstruction)
adversary-initial-state
 $\leq$ *wp-event (checked-staged-security-experiment-with-data-state A)*
($\lambda out.$


```

    staged-security-with-data-state-verifier-event
    trace-fri-zero-round-checked-final-obstruction out ∨
    staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad out)
  adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
by (rule wp-event-mono-on-support)
  (auto dest:
    trace-fri-header-tied-zero-round-final-obstruction-imp-checked-or-merkle
    split: option.splits prod.splits)
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-zero-round-checked-final-obstruction)
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad)
  adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
  trace-fri-zero-round-agreement-set-target-error-for budgets +
  trace-fri-controlled-merkle-error-for budgets
by (rule add-mono[OF checked merkle])
finally show ?thesis .
qed

```

```

lemma trace-header-restricted-one-step-error-for-bound:
  fixes trace-bad :: 'f list ⇒ nat ⇒ 'f list ⇒ 'f set
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-partial-candidate)
    adversary-initial-state
    ≤ trace-header-restricted-one-step-error-for budgets A trace-bad
proof –
  have sampled-query:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-sampled-query-bad-candidate)
    adversary-initial-state
    ≤ trace-restricted-sampled-query-error-for budgets A trace-bad
  by (rule trace-restricted-sampled-query-error-for-bound[OF wf controlled])
  have sampled-plain:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-sampled-layer-chain)

```

adversary-initial-state
 $\leq$ *trace-restricted-sampled-query-error-for budgets A trace-bad*
by (rule
checked-staged-security-trace-fri-sampled-layer-chain-bound-from-sampled-query
[OF sampled-query])
have *sampled-header*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-sampled-layer-chain)
adversary-initial-state
 $\leq$ *trace-restricted-sampled-query-error-for budgets A trace-bad*
proof (rule *order-trans[OF - sampled-plain]*)
show *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-sampled-layer-chain)
adversary-initial-state
 $\leq$ *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-sampled-layer-chain)
adversary-initial-state
by (rule *wp-event-mono*)
(auto simp: staged-security-with-data-state-verifier-event-def
dest: trace-fri-header-tied-sampled-layer-chain-imp-sampled-layer-chain
split: option.splits prod.splits)
qed
have *conflict-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-header-tied-sampled-assignment-conflict)
adversary-initial-state
 $\leq$ *trace-fri-header-replay-conflict-error-for A*
by (rule *trace-fri-header-tied-sampled-assignment-conflict-replay-bound*)
have *zero-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-header-tied-zero-round-final-obstruction)
adversary-initial-state
 $\leq$ *trace-fri-zero-round-agreement-set-target-error-for budgets +*
trace-fri-controlled-merkle-error-for budgets
by (rule
trace-fri-header-tied-zero-round-final-obstruction-agreement-bound
[OF wf controlled])
have *branch*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state
 $\leq$ *trace-restricted-sampled-query-error-for budgets A trace-bad +*
(trace-fri-header-replay-conflict-error-for A +

(trace-fri-zero-round-agreement-set-target-error-for budgets +
 trace-fri-controlled-merkle-error-for budgets))
 by (rule
 checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-conflict-and-staged-zero
 [OF sampled-header conflict-bound zero-bound])
 show ?thesis
 using branch
 by (simp add: trace-header-restricted-one-step-error-for-def
 trace-fri-header-replay-conflict-zero-error-for-def)
 qed

definition trace-empty-header-restricted-openings-zero-error-for
 where

trace-empty-header-restricted-openings-zero-error-for budgets A empty-bad =
 (trace-fri-zero-round-agreement-set-target-error-for budgets +
 trace-fri-controlled-merkle-error-for budgets +
 (trace-header-restricted-one-step-error-for budgets A (λ-. empty-bad) +
 reachable-verifier-event-bound-for A
 trace-fri-header-tied-low-candidate-untied-openings-gap) +
 trace-header-restricted-one-step-error-for budgets A (λ-. empty-bad))

definition trace-empty-header-restricted-nonnormalized-zero-error-for
 where

trace-empty-header-restricted-nonnormalized-zero-error-for budgets A empty-bad =
 (trace-fri-zero-round-agreement-set-target-error-for budgets +
 trace-fri-controlled-merkle-error-for budgets +
 (trace-fri-controlled-merkle-error-for budgets +
 ((reachable-verifier-event-bound-for A
 trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
 reachable-verifier-event-bound-for A
 trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
 reachable-verifier-event-bound-for A
 trace-fri-header-tied-low-candidate-nonconsumed-query-gap)) +
 trace-header-restricted-one-step-error-for budgets A (λ-. empty-bad))

definition trace-empty-header-restricted-sampled-split-one-step-error-for
 where

trace-empty-header-restricted-sampled-split-one-step-error-for budgets A empty-bad =
 trace-restricted-sampled-query-error-for budgets A (λ-. empty-bad) +
 (trace-fri-empty-sampled-assignment-structural-error-for budgets A +
 min (trace-empty-header-restricted-openings-zero-error-for budgets A empty-bad)
 (trace-empty-header-restricted-nonnormalized-zero-error-for budgets A empty-bad))

definition concrete-trace-empty-header-restricted-sampled-split-one-step-error-for
 where

concrete-trace-empty-header-restricted-sampled-split-one-step-error-for budgets A =
 trace-empty-header-restricted-sampled-split-one-step-error-for budgets A
 trace-one-step-split-bad

lemma *trace-empty-header-restricted-sampled-split-one-step-error-for-bound*:
fixes *empty-bad* :: nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set
assumes *wf*: *staged-budget-wellformed* budgets
and *controlled*: *staged-adversary-controlled* budgets *A*
shows
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state
 $\leq$ *trace-empty-header-restricted-sampled-split-one-step-error-for*
 budgets *A* *empty-bad*

proof –
have *sampled*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-sampled-query-bad-candidate)
 adversary-initial-state
 $\leq$ *trace-restricted-sampled-query-error-for* budgets *A* (λ -. *empty-bad*)
by (rule *trace-restricted-sampled-query-error-for-bound* [*OF wf controlled*])
have *conflict-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-sampled-assignment-conflict)
 adversary-initial-state
 $\leq$ *trace-fri-empty-sampled-assignment-structural-error-for* budgets *A*
by (rule
 checked-staged-security-trace-fri-sampled-assignment-conflict-bound-structural
 [*OF wf controlled*])
have *header-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state
 $\leq$ *trace-header-restricted-one-step-error-for* budgets *A* (λ -. *empty-bad*)
by (rule *trace-header-restricted-one-step-error-for-bound*
 [*OF wf controlled*])
have *zero-openings-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-zero-round-final-obstruction)
 adversary-initial-state
 $\leq$ *trace-empty-header-restricted-openings-zero-error-for* budgets *A* *empty-bad*
unfolding *trace-empty-header-restricted-openings-zero-error-for-def*
by (rule
 checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets-header-and-openings
 [*OF wf controlled header-bound*])
have *zero-nonnormalized-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*

$\text{trace-fri-zero-round-final-obstruction})$
 $\text{adversary-initial-state}$
 $\leq \text{trace-empty-header-restricted-nonnormalized-zero-error-for budgets } A \text{ empty-bad}$
unfolding $\text{trace-empty-header-restricted-nonnormalized-zero-error-for-def}$
by (rule
 $\text{checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets-header-and-nonnormalized}$
 $[\text{OF wf controlled header-bound}]$)
have zero-bound:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-zero-round-final-obstruction})$
 $\text{adversary-initial-state}$
 $\leq \min (\text{trace-empty-header-restricted-openings-zero-error-for budgets } A \text{ empty-bad})$
 $(\text{trace-empty-header-restricted-nonnormalized-zero-error-for budgets } A \text{ empty-bad})$
using zero-openings-bound zero-nonnormalized-bound **by** simp
have branch:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state}$
 $\leq \text{trace-restricted-sampled-query-error-for budgets } A (\lambda \cdot \text{empty-bad}) +$
 $(\text{trace-fri-empty-sampled-assignment-structural-error-for budgets } A +$
 $\min (\text{trace-empty-header-restricted-openings-zero-error-for budgets } A \text{ empty-bad})$
 $(\text{trace-empty-header-restricted-nonnormalized-zero-error-for budgets } A \text{ empty-bad}))$
by (rule
 $\text{checked-staged-security-trace-fri-empty-header-bound-from-staged-sampled-conflict-and-zero}$
 $[\text{OF sampled conflict-bound zero-bound}]$)
show ?thesis
using branch
by (simp add:
 $\text{trace-empty-header-restricted-sampled-split-one-step-error-for-def})$
qed

definition $\text{composition-no-full-cover-sampled-query-error-for}$
where
 $\text{composition-no-full-cover-sampled-query-error-for budgets bad} =$
 $(\sum_{dg \in \text{UNIV}.}$
 $\sum_{\text{challenges} \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))}.$
 $(1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1))) *$
 $(\text{nnreal}$
 $(\text{card}$
 $(\text{composition-fri-sampled-query-no-full-cover-failure-query-fiber}$
 $\text{bad } dg \text{ challenges})) *$
 $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds})) +$
 $\text{hash-relation-budget-value}$
 $(\text{query-index-raw-list-relation-fiber-bound}$
 $(\bigcup_{dg \in \text{UNIV}.}$
 $\bigcup_{\text{challenges} \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))}.$
 $\text{composition-fri-sampled-query-no-full-cover-failure-query-fiber}$

$$\begin{aligned}
& \text{bad } dg \text{ challenges})) \\
& (\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget}) + \\
& \text{hash-relation-budget-value} \\
& (\sum dg \in (UNIV :: 'f \text{ set}). \\
& \quad \text{card (fri-challenge-space (ceil-log (to-nat dg} + 1))) * \\
& \quad \text{ceil-log (maxDegree} + 1)) \\
& (\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget})
\end{aligned}$$

definition *composition-canonical-full-cover-shape-failure-error-for*
where

$$\begin{aligned}
& \text{composition-canonical-full-cover-shape-failure-error-for } A \text{ bad} = \\
& \text{reachable-verifier-event-bound-for } A \\
& (\lambda s. \text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad}) + \\
& \text{reachable-verifier-event-bound-for } A \\
& \text{composition-fri-sampled-full-cover-index-failure} + \\
& \text{reachable-verifier-event-bound-for } A \\
& \text{composition-fri-sampled-full-cover-shape-failure}
\end{aligned}$$

definition *composition-restricted-sampled-query-error-for*
where

$$\begin{aligned}
& \text{composition-restricted-sampled-query-error-for budgets } A \text{ bad} = \\
& \text{composition-no-full-cover-sampled-query-error-for budgets bad} + \\
& \text{composition-canonical-full-cover-shape-failure-error-for } A \text{ bad}
\end{aligned}$$

lemma *composition-restricted-sampled-query-error-for-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

$$\begin{aligned}
& \text{wp-event (checked-staged-security-experiment-with-data-state } A) \\
& (\text{staged-security-with-data-state-verifier-event} \\
& \quad \text{composition-fri-sampled-query-bad-candidate}) \\
& \text{adversary-initial-state} \\
& \leq \text{composition-restricted-sampled-query-error-for budgets } A \text{ bad}
\end{aligned}$$

proof –

have *no-full:*

$$\begin{aligned}
& \text{wp-event (checked-staged-security-experiment-with-data-state } A) \\
& (\text{staged-security-with-data-state-verifier-event} \\
& \quad (\lambda s. \\
& \quad \quad \text{composition-fri-sampled-query-no-full-cover-failure-candidate } s \\
& \quad \quad \text{bad})) \\
& \text{adversary-initial-state}
\end{aligned}$$

$\leq$ *composition-no-full-cover-sampled-query-error-for budgets bad*

unfolding *composition-no-full-cover-sampled-query-error-for-def*

by (rule

$$\begin{aligned}
& \text{checked-staged-security-composition-fri-sampled-query-no-full-cover-failure-candidate-bound-from-restricted-pairs} \\
& [\text{OF wf controlled}]
\end{aligned}$$

have *failure:*

$$\begin{aligned}
& \text{wp-event (checked-staged-security-experiment-with-data-state } A) \\
& (\text{staged-security-with-data-state-verifier-event}
\end{aligned}$$

```

    (λs. composition-fri-sampled-canonical-full-cover-failure s bad))
  adversary-initial-state
≤ composition-canonical-full-cover-shape-failure-error-for A bad
unfolding composition-canonical-full-cover-shape-failure-error-for-def
proof (rule
  checked-staged-security-composition-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (composition-fri-sampled-full-cover-proximity-failure
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) bad)
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
≤ reachable-verifier-event-bound-for A
  (λs. composition-fri-sampled-full-cover-proximity-failure s bad)
by (rule reachable-verifier-event-bound-for-ge[OF support])
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (composition-fri-sampled-full-cover-index-failure
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
≤ reachable-verifier-event-bound-for A
  composition-fri-sampled-full-cover-index-failure
by (rule reachable-verifier-event-bound-for-ge[OF support])
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (composition-fri-sampled-full-cover-shape-failure
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
≤ reachable-verifier-event-bound-for A
  composition-fri-sampled-full-cover-shape-failure

```

by (rule reachable-verifier-event-bound-for-ge[OF support])
 qed
 have sampled:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-sampled-query-bad-candidate)
 adversary-initial-state
 ≤ composition-no-full-cover-sampled-query-error-for budgets bad +
 composition-canonical-full-cover-shape-failure-error-for A bad
 by (rule
 checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-canonical-failure
 [OF no-full failure])
 show ?thesis
 using sampled
 by (simp add: composition-restricted-sampled-query-error-for-def)
 qed

definition composition-restricted-verifier-tied-controlled-error-for
 where
 composition-restricted-verifier-tied-controlled-error-for budgets A comp-bad =
 composition-restricted-sampled-query-error-for budgets A comp-bad +
 (((composition-one-step-controlled-merkle-error-for budgets + 0) +
 (composition-one-step-controlled-merkle-error-for budgets + 0) +
 (composition-one-step-controlled-merkle-error-for budgets + 0) +
 (composition-one-step-controlled-merkle-error-for budgets + 0 + 0) +
 composition-one-step-controlled-merkle-error-for budgets) + 0 + 0)

definition concrete-composition-restricted-one-step-error-for
 where
 concrete-composition-restricted-one-step-error-for budgets A =
 composition-restricted-verifier-tied-controlled-error-for budgets A
 (λdg -. composition-one-step-split-bad dg)

lemma composition-restricted-verifier-tied-controlled-error-for-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state
 ≤ composition-restricted-verifier-tied-controlled-error-for budgets A
 comp-bad

proof –
have sampled-query:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-sampled-query-bad-candidate)
 adversary-initial-state

$\leq$ *composition-restricted-sampled-query-error-for budgets A comp-bad*
by (rule *composition-restricted-sampled-query-error-for-bound*
[*OF wf controlled*])
have *sampled*:
wp-event (checked-staged-security-experiment-with-data-state A)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-verifier-tied-sampled-layer-chain)
adversary-initial-state
 $\leq$ *composition-restricted-sampled-query-error-for budgets A comp-bad*
by (rule
checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-sampled-query
[*OF sampled-query*])
have *conflict*:
wp-event (checked-staged-security-experiment-with-data-state A)
(*staged-security-with-data-state-verifier-event*
composition-fri-verifier-tied-sampled-assignment-conflict)
adversary-initial-state $\leq$
(((*composition-one-step-controlled-merkle-error-for budgets + 0*) +
(*composition-one-step-controlled-merkle-error-for budgets + 0*) +
(*composition-one-step-controlled-merkle-error-for budgets + 0*) +
(*composition-one-step-controlled-merkle-error-for budgets + 0 + 0*) +
composition-one-step-controlled-merkle-error-for budgets) + *0*)
by (rule
checked-staged-security-composition-fri-verifier-tied-sampled-assignment-conflict-bound-by-controlled-merkle
[*OF wf controlled*])
have *zero*:
wp-event (checked-staged-security-experiment-with-data-state A)
(*staged-security-with-data-state-verifier-event*
composition-fri-verifier-tied-zero-round-final-obstruction)
adversary-initial-state $\leq$ (*0::prob*)
by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
(*simp-all add*:
composition-fri-verifier-tied-zero-round-final-obstruction-false
wp-composition-fri-verifier-tied-zero-round-final-obstruction-zero)
have *branch*:
wp-event (checked-staged-security-experiment-with-data-state A)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state $\leq$
composition-restricted-sampled-query-error-for budgets A comp-bad +
((((*composition-one-step-controlled-merkle-error-for budgets + 0*) +
(*composition-one-step-controlled-merkle-error-for budgets + 0*) +
(*composition-one-step-controlled-merkle-error-for budgets + 0*) +
(*composition-one-step-controlled-merkle-error-for budgets + 0 + 0*) +
composition-one-step-controlled-merkle-error-for budgets) + *0*) + *0*)
by (rule
checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-staged-sampled-conflict-and-zero
[*OF sampled conflict zero*])
show ?thesis

using branch
 by (simp add:
 composition-restricted-verifier-tied-controlled-error-for-def)
 qed

definition *active-route-current-prefix-conceptual-prefix-collision-hit-at*
where

active-route-current-prefix-conceptual-prefix-collision-hit-at i out
 $\longleftrightarrow$
checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
i out $\wedge$
 (case out of
 None $\Rightarrow$ False
 | Some (((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result), final-state) $\Rightarrow$
 hash-map-output-collision prefix-state)

definition *active-route-current-prefix-conceptual-algebraic-residual-hit-at*
where

active-route-current-prefix-conceptual-algebraic-residual-hit-at i out
 $\longleftrightarrow$
checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
i out $\wedge$
 (case out of
 None $\Rightarrow$ False
 | Some (((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result), final-state) $\Rightarrow$
 $\neg$ trace-table-low-degree
 (query-prefix-trace-conceptual-table prefix prefix-state) $\vee$
 $\neg$ composition-table-low-degree maxDegree
 (query-prefix-composition-conceptual-table prefix prefix-state) $\vee$
 all-queries-consistent
 (query-prefix-trace-conceptual-table prefix prefix-state)
 (query-prefix-composition-conceptual-table prefix prefix-state)
 (sq α -alphas prefix))

definition *active-route-current-prefix-conceptual-algebraic-residual-error-for*
where

active-route-current-prefix-conceptual-algebraic-residual-error-for A i =
wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*active-route-current-prefix-conceptual-algebraic-residual-hit-at i*)
 adversary-initial-state

definition *active-route-current-prefix-conceptual-trace-low-degree-bad-at*
where

active-route-current-prefix-conceptual-trace-low-degree-bad-at i out
 $\longleftrightarrow$
active-route-current-prefix-conceptual-algebraic-residual-hit-at i out $\wedge$

(case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 $\neg$ trace-table-low-degree
 (query-prefix-trace-conceptual-table prefix prefix-state))

definition *active-route-current-prefix-conceptual-composition-low-degree-bad-at*
where

active-route-current-prefix-conceptual-composition-low-degree-bad-at i out
 $\longleftrightarrow$
active-route-current-prefix-conceptual-algebraic-residual-hit-at i out $\wedge$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 $\neg$ composition-table-low-degree maxDegree
 (query-prefix-composition-conceptual-table prefix prefix-state))

definition *active-route-current-prefix-conceptual-alpha-bad-set-hit-at*
where

active-route-current-prefix-conceptual-alpha-bad-set-hit-at i out
 $\longleftrightarrow$
active-route-current-prefix-conceptual-algebraic-residual-hit-at i out $\wedge$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 spp-alphas prefix $\in$
 composition-trace-bad-alpha-space
 (query-prefix-trace-conceptual-table prefix prefix-state))

definition *active-route-current-prefix-conceptual-trace-low-degree-error-for*
where

active-route-current-prefix-conceptual-trace-low-degree-error-for A i =
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (active-route-current-prefix-conceptual-trace-low-degree-bad-at i)
 adversary-initial-state

definition *active-route-current-prefix-conceptual-composition-low-degree-error-for*
where

active-route-current-prefix-conceptual-composition-low-degree-error-for A i =
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (active-route-current-prefix-conceptual-composition-low-degree-bad-at i)
 adversary-initial-state

definition *active-route-current-prefix-conceptual-alpha-bad-set-error-for*

where

active-route-current-prefix-conceptual-alpha-bad-set-error-for A i =
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(active-route-current-prefix-conceptual-alpha-bad-set-hit-at i)
adversary-initial-state

lemma *active-route-current-prefix-conceptual-alpha-bad-set-hit-at-imp-data-state-alpha-prefix-bad-set-hit-on-support:*

assumes *support:*

Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)

and *candidate:*

∧ alpha-prefix alpha-prefix-state.
alpha-prefix =
(sqp-trace-root prefix, sqp-trace-fri-roots prefix,
sqp-trace-fri-challenges prefix, sqp-trace-final prefix) ⇒
Some (alpha-prefix, alpha-prefix-state) ∈
set-dist
(execute (staged-alpha-prefix-program A)
adversary-initial-state) ⇒
query-prefix-trace-conceptual-table prefix prefix-state ∈
alpha-prefix-trace-table-candidates alpha-prefix-state
(sqp-trace-root prefix)

and *hit:*

active-route-current-prefix-conceptual-alpha-bad-set-hit-at i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))

shows

checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
composition-trace-bad-alpha-space
(Some (((data, attacker-state), result), final-state))

proof –

obtain *alpha-prefix alpha-prefix-state* **where** *alpha-prefix-eq:*

alpha-prefix =
(sqp-trace-root prefix, sqp-trace-fri-roots prefix,
sqp-trace-fri-challenges prefix, sqp-trace-final prefix)

and *alpha-support:*

Some (alpha-prefix, alpha-prefix-state) ∈
set-dist
(execute (staged-alpha-prefix-program A) adversary-initial-state)

by (rule *checked-staged-security-with-query-prefix-data-state-alpha-prefix-support*
 $[OF\ support]$)

have *table-candidate:*

query-prefix-trace-conceptual-table prefix prefix-state ∈
alpha-prefix-trace-table-candidates alpha-prefix-state

```

    (sqp-trace-root prefix)
  by (rule candidate[OF alpha-prefix-eq alpha-support])
have alphas-eq: staged-alphas data = sqp-alphas prefix
  by (rule checked-staged-security-with-query-prefix-data-state-alphas-eq
      [OF support])
have alpha-bad:
  sqp-alphas prefix ∈
    composition-trace-bad-alpha-space
    (query-prefix-trace-conceptual-table prefix prefix-state)
  using hit
  unfolding active-route-current-prefix-conceptual-alpha-bad-set-hit-at-def
  by simp
have staged-alpha-bad:
  staged-alphas data ∈
    composition-trace-bad-alpha-space
    (query-prefix-trace-conceptual-table prefix prefix-state)
  using alpha-bad alphas-eq by simp
have staged-alpha-in-space: staged-alphas data ∈ alpha-space
  using staged-alpha-bad composition-trace-bad-alpha-space-subset-alpha-space
  by blast
have union-hit:
  staged-alphas data ∈
    alpha-prefix-union-bad-sets alpha-prefix-state
    composition-trace-bad-alpha-space (staged-trace-root data)
proof -
  have root-eq: staged-trace-root data = sqp-trace-root prefix
    using support
    unfolding
      checked-staged-security-experiment-with-query-prefix-data-state-def
      checked-staged-after-query-prefix-receive-with-verifier-def
      checked-staged-after-query-prefix-receive-def
    by (auto elim!: set-dist-bindE split: prod.splits)
  have table-candidate':
    query-prefix-trace-conceptual-table prefix prefix-state ∈
      alpha-prefix-trace-table-candidates alpha-prefix-state
      (staged-trace-root data)
    using table-candidate root-eq by simp
  show ?thesis
    unfolding alpha-prefix-union-bad-sets-def
    using table-candidate' staged-alpha-bad staged-alpha-in-space
    by auto
qed
have prefix-components:
  staged-trace-root data = fst alpha-prefix ∧
  staged-trace-fri-roots data = fst (snd alpha-prefix) ∧
  staged-trace-fri-challenges data = fst (snd (snd alpha-prefix)) ∧
  staged-trace-final data = snd (snd (snd alpha-prefix))
  using support alpha-prefix-eq
  unfolding

```

```

checked-staged-security-experiment-with-query-prefix-data-state-def
checked-staged-after-query-prefix-receive-with-verifier-def
checked-staged-after-query-prefix-receive-def
by (auto elim!: set-dist-bindE split: prod.splits)
show ?thesis
unfolding checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-def
using alpha-support prefix-components union-hit
by auto
qed

lemma active-route-current-prefix-conceptual-alpha-bad-set-error-for-bound-from-data-state-alpha-prefix:
  assumes i-bound:  $i < \text{rounds}$ 
  and candidate:
     $\bigwedge \text{prefix prefix-state raw raw-state data attacker-state result}$ 
     $\text{final-state alpha-prefix alpha-prefix-state.}$ 
     $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$ 
     $((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}) \in$ 
     $\text{set-dist}$ 
     $(\text{execute}$ 
     $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$ 
     $\text{adversary-initial-state}) \implies$ 
     $\text{alpha-prefix} =$ 
     $(\text{sqp-trace-root prefix}, \text{sqp-trace-fri-roots prefix},$ 
     $\text{sqp-trace-fri-challenges prefix}, \text{sqp-trace-final prefix}) \implies$ 
     $\text{Some } (\text{alpha-prefix}, \text{alpha-prefix-state}) \in$ 
     $\text{set-dist}$ 
     $(\text{execute } (\text{staged-alpha-prefix-program } A)$ 
     $\text{adversary-initial-state}) \implies$ 
     $\text{query-prefix-trace-conceptual-table prefix prefix-state} \in$ 
     $\text{alpha-prefix-trace-table-candidates alpha-prefix-state}$ 
     $(\text{sqp-trace-root prefix})$ 
  and data-state-bound:
     $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$ 
     $(\text{checked-staged-security-with-data-state-alpha-prefix-bad-set-hit } A$ 
     $\text{composition-trace-bad-alpha-space})$ 
     $\text{adversary-initial-state} \leq C$ 
  shows
    active-route-current-prefix-conceptual-alpha-bad-set-error-for  $A \ i \leq C$ 
proof -
  let ?Data =
    checked-staged-security-with-data-state-alpha-prefix-bad-set-hit  $A$ 
    composition-trace-bad-alpha-space
  let ?Projected =
     $\lambda \text{out. case out of}$ 
     $\text{None} \Rightarrow ?Data \ \text{None}$ 
     $| \ \text{Some } (\text{packed}, t) \Rightarrow ?Data \ (\text{Some } (\text{snd packed}, t))$ 
  let ?E = active-route-current-prefix-conceptual-alpha-bad-set-hit-at  $i$ 
  have event-le:
    wp-event

```

```

    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    ?E adversary-initial-state ≤
wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    ?Projected adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
    and hit: ?E out
  show ?Projected out
  proof (cases out)
    case None
    then show ?thesis
      using hit
      unfolding active-route-current-prefix-conceptual-alpha-bad-set-hit-at-def
      by simp
  next
    case (Some packed)
    then obtain prefix prefix-state raw raw-state data attacker-state result
      final-state where out-eq:
      out =
        Some (((((prefix, prefix-state), raw), raw-state),
          ((data, attacker-state), result)), final-state)
      by (cases packed) (auto split: prod.splits)
    have support':
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state) ∈
        set-dist
        (execute
          (checked-staged-security-experiment-with-query-prefix-data-state A i)
          adversary-initial-state)
    using support unfolding out-eq .
    have hit':
      active-route-current-prefix-conceptual-alpha-bad-set-hit-at i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
    using hit unfolding out-eq .
    have data-hit:
      ?Data (Some (((data, attacker-state), result), final-state))
    by (rule
      active-route-current-prefix-conceptual-alpha-bad-set-hit-at-imp-data-state-alpha-prefix-bad-set-hit-on-support
      [OF support' - hit'])
      (rule candidate[OF support'])
    show ?thesis

```

unfolding *out-eq* **using** *data-hit* **by** *simp*
qed
qed
have *projection*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
 ?*Projected adversary-initial-state* =
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 ?*Data adversary-initial-state*
using *checked-staged-security-experiment-with-query-prefix-data-state-projection-event*
 [*OF i-bound*, *of A ?Data*]
by *simp*
show *?thesis*
unfolding *active-route-current-prefix-conceptual-alpha-bad-set-error-for-def*
by (*rule order-trans*[*OF event-le*])
 (*simp add: projection data-state-bound*)
qed

definition *active-route-current-prefix-split-algebraic-residual-bound-for*
where

active-route-current-prefix-split-algebraic-residual-bound-for *A i* =
active-route-current-prefix-conceptual-trace-low-degree-error-for *A i* +
active-route-current-prefix-conceptual-composition-low-degree-error-for *A i* +
active-route-current-prefix-conceptual-alpha-bad-set-error-for *A i*

definition *active-route-current-prefix-split-conceptual-bound-for*
where

active-route-current-prefix-split-conceptual-bound-for *budgets A i* =
staged-phase-relation-error size
 (*staged-query-search-queries* *budgets i + 1*) +
query-error-bound +
hash-collision-budget-value 0
 (*staged-query-search-queries* *budgets i + 1*) +
active-route-current-prefix-split-algebraic-residual-bound-for *A i*

lemma *active-route-current-prefix-conceptual-prefix-collision-hit-at-bound*:

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*

shows

wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
 (*active-route-current-prefix-conceptual-prefix-collision-hit-at* *i*)
 adversary-initial-state ≤
 hash-collision-budget-value 0
 (*staged-query-search-queries* *budgets i + 1*)

proof –

have *i-le: i ≤ rounds*
using *i-bound* **by** *simp*


```

have prefix-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    checked-staged-query-prefix-hash-map-output-collision
    adversary-initial-state  $\leq$ 
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1)
by (rule checked-staged-query-prefix-hash-map-output-collision-bound
  [OF wf controlled i-le])
show ?thesis
proof (rule
  checked-staged-security-experiment-with-query-prefix-data-state-bound-by-prefix
  [OF prefix-bound])
show
  active-route-current-prefix-conceptual-prefix-collision-hit-at i None
     $\implies$  checked-staged-query-prefix-hash-map-output-collision None
unfolding
  active-route-current-prefix-conceptual-prefix-collision-hit-at-def
  checked-staged-query-prefix-hash-map-output-collision-def
by simp
next
fix prefix-out t out
assume
  Some (prefix-out, t) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
and cont:
  out ∈
    set-dist
    (execute
      (get  $\gg$ 
        ( $\lambda$ prefix-state.
          checked-staged-after-query-prefix-receive-with-verifier A i
            prefix-out  $\gg$ 
            ( $\lambda$ result. return ((prefix-out, prefix-state), result))))
      t)
and event:
  active-route-current-prefix-conceptual-prefix-collision-hit-at i out
show
  checked-staged-query-prefix-hash-map-output-collision
    (Some (prefix-out, t))
using cont event
unfolding
  active-route-current-prefix-conceptual-prefix-collision-hit-at-def
  checked-staged-query-prefix-hash-map-output-collision-def
by (auto elim!: set-dist-bindE split: option.splits prod.splits)
qed
qed

```

lemma *active-route-current-prefix-conceptual-hit-residual-split*:

assumes

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
i out

and

checked-staged-security-with-query-prefix-conceptual-context-residual-at
out

shows

active-route-current-prefix-conceptual-prefix-collision-hit-at i out $\vee$
active-route-current-prefix-conceptual-algebraic-residual-hit-at i out

using *assms*

unfolding

active-route-current-prefix-conceptual-prefix-collision-hit-at-def
active-route-current-prefix-conceptual-algebraic-residual-hit-at-def
checked-staged-security-with-query-prefix-conceptual-context-residual-at-def

by (*cases out*) (*auto split: prod.splits*)

lemma *active-route-current-prefix-conceptual-algebraic-residual-split-on-support*:

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *support*:

Some (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*) $\in$
set-dist
(*execute*
(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
adversary-initial-state)

and *hit*:

active-route-current-prefix-conceptual-algebraic-residual-hit-at i
(*Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*))

shows

active-route-current-prefix-conceptual-trace-low-degree-bad-at i
(*Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*)) $\vee$
active-route-current-prefix-conceptual-composition-low-degree-bad-at i
(*Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*)) $\vee$
active-route-current-prefix-conceptual-alpha-bad-set-hit-at i
(*Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*))

proof –

let *?trace* = *query-prefix-trace-conceptual-table prefix prefix-state*

let *?composition* =

query-prefix-composition-conceptual-table prefix prefix-state

from *hit* **have** *residual*:

$\neg \text{trace-table-low-degree } ?\text{trace} \vee$
 $\neg \text{composition-table-low-degree } \text{maxDegree } ?\text{composition} \vee$
all-queries-consistent ?trace ?composition (sqp-alphas prefix)

unfolding

```

    active-route-current-prefix-conceptual-algebraic-residual-hit-at-def
  by simp
show ?thesis
proof (cases trace-table-low-degree ?trace)
  case False
  then show ?thesis
    using hit
    unfolding active-route-current-prefix-conceptual-trace-low-degree-bad-at-def
    by simp
next
case trace-low: True
show ?thesis
proof (cases composition-table-low-degree maxDegree ?composition)
  case False
  then show ?thesis
    using hit
    unfolding
      active-route-current-prefix-conceptual-composition-low-degree-bad-at-def
    by simp
next
case composition-low: True
from residual trace-low composition-low have all-queries:
  all-queries-consistent ?trace ?composition (sqp-alphas prefix)
  by simp
have prefix-support:
  Some ((prefix, prefix-state), prefix-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-with-state A i)
        adversary-initial-state)
  by (rule
    checked-staged-security-with-query-prefix-data-state-prefix-support
    [OF support])
have alpha-len: length (sqp-alphas prefix) = length spec
  by (rule checked-staged-query-prefix-with-state-alphas-length
    [OF prefix-support])
have alpha-bad:
  sqp-alphas prefix ∈ composition-trace-bad-alpha-space ?trace
  by (rule
    low-degree-all-queries-consistent-imp-composition-trace-bad-alpha-space
    [OF false-statement alpha-len trace-low composition-low
      all-queries])
then show ?thesis
  using hit
  unfolding
    active-route-current-prefix-conceptual-alpha-bad-set-hit-at-def
  by simp
qed
qed
qed

```

lemma *active-route-current-prefix-conceptual-algebraic-residual-bound-from-split:*

assumes *false-statement:* $\neg \text{exists-valid-trace}$

shows

active-route-current-prefix-conceptual-algebraic-residual-error-for A i
 $\leq$ *active-route-current-prefix-split-algebraic-residual-bound-for A i*

proof –

let *?M* = *checked-staged-security-experiment-with-query-prefix-data-state A i*

let *?Algebraic* =

active-route-current-prefix-conceptual-algebraic-residual-hit-at i

let *?Trace* =

active-route-current-prefix-conceptual-trace-low-degree-bad-at i

let *?Composition* =

active-route-current-prefix-conceptual-composition-low-degree-bad-at i

let *?Alpha* =

active-route-current-prefix-conceptual-alpha-bad-set-hit-at i

have *event-le:*

wp-event ?M ?Algebraic adversary-initial-state $\leq$

wp-event ?M ($\lambda \text{out. ?Trace out} \vee \text{?Composition out} \vee \text{?Alpha out}$)
adversary-initial-state

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support:* *out* $\in$ *set-dist* (*execute ?M adversary-initial-state*)

and *hit:* *?Algebraic out*

show *?Trace out* $\vee$ *?Composition out* $\vee$ *?Alpha out*

proof (*cases out*)

case *None*

then show *?thesis*

using *hit*

unfolding

active-route-current-prefix-conceptual-algebraic-residual-hit-at-def

by *simp*

next

case (*Some packed*)

then obtain *prefix prefix-state raw raw-state data attacker-state result*
final-state **where** *out-eq:*

out =

Some ((((((*prefix, prefix-state*), *raw*), *raw-state*),
((*data, attacker-state*), *result*))), *final-state*)

by (*cases packed*) (*auto split: prod.splits*)

have *support':*

Some ((((((*prefix, prefix-state*), *raw*), *raw-state*),
((*data, attacker-state*), *result*))), *final-state*) $\in$
set-dist (*execute ?M adversary-initial-state*)

using *support* **unfolding** *out-eq* .

have *hit':*

?Algebraic

(*Some* ((((((*prefix, prefix-state*), *raw*), *raw-state*),
((*data, attacker-state*), *result*))), *final-state*))

```

    using hit unfolding out-eq .
show ?thesis
  unfolding out-eq
  by (rule
    active-route-current-prefix-conceptual-algebraic-residual-split-on-support
    [OF false-statement support' hit'])
qed
qed
have union-bound:
  wp-event ?M ( $\lambda out. ?Trace\ out \vee ?Composition\ out \vee ?Alpha\ out$ )
  adversary-initial-state  $\leq$ 
  wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M ?Composition adversary-initial-state +
  wp-event ?M ?Alpha adversary-initial-state
proof -
  have wp-event ?M
    ( $\lambda out. ?Trace\ out \vee ?Composition\ out \vee ?Alpha\ out$ )
    adversary-initial-state  $\leq$ 
    wp-event ?M ?Trace adversary-initial-state +
    wp-event ?M ( $\lambda out. ?Composition\ out \vee ?Alpha\ out$ )
    adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq$ 
    wp-event ?M ?Trace adversary-initial-state +
    (wp-event ?M ?Composition adversary-initial-state +
    wp-event ?M ?Alpha adversary-initial-state)
  by (intro add-mono order-refl wp-event-union-bound)
  finally show ?thesis
    by (simp add: add.assoc)
qed
show ?thesis
  unfolding
    active-route-current-prefix-conceptual-algebraic-residual-error-for-def
    active-route-current-prefix-split-algebraic-residual-bound-for-def
    active-route-current-prefix-conceptual-trace-low-degree-error-for-def
    active-route-current-prefix-conceptual-composition-low-degree-error-for-def
    active-route-current-prefix-conceptual-alpha-bad-set-error-for-def
  by (rule order-trans[OF event-le union-bound])
qed

lemma active-route-current-prefix-authenticated-error-for-le-split-conceptual-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < rounds$ 
  and false-statement:  $\neg exists\text{-}valid\text{-}trace$ 
  shows
    active-route-current-prefix-authenticated-error-for A i  $\leq$ 
    active-route-current-prefix-split-conceptual-bound-for budgets A i
proof -

```

```

let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i
let ?Hit =
  checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
    i
let ?Conceptual =
  checked-staged-security-with-query-prefix-conceptual-target-hit
let ?Collision =
  active-route-current-prefix-conceptual-prefix-collision-hit-at i
let ?Algebraic =
  active-route-current-prefix-conceptual-algebraic-residual-hit-at i
have event-le:
  wp-event ?M ?Hit adversary-initial-state ≤
  wp-event ?M
  (λout. ?Conceptual out ∨ ?Collision out ∨ ?Algebraic out)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support: out ∈ set-dist (execute ?M adversary-initial-state)
  and hit: ?Hit out
  have conceptual-or-residual:
    ?Conceptual out ∨
    checked-staged-security-with-query-prefix-conceptual-context-residual-at
      out
  by (rule
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-imp-conceptual-or-residual-on-support
    [where budgets=budgets and A=A and i=i and out=out,
      OF wf controlled i-bound support hit])
  show ?Conceptual out ∨ ?Collision out ∨ ?Algebraic out
  using conceptual-or-residual
  proof (elim disjE)
  assume conceptual: ?Conceptual out
  then show ?thesis by simp
  next
  assume residual:
    checked-staged-security-with-query-prefix-conceptual-context-residual-at
      out
  have ?Collision out ∨ ?Algebraic out
  by (rule active-route-current-prefix-conceptual-hit-residual-split
    [OF hit residual])
  then show ?thesis by simp
  qed
qed
have conceptual-bound:
  wp-event ?M ?Conceptual adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) + query-error-bound
  by (rule
    checked-staged-security-with-query-prefix-conceptual-target-hit-bound
    [OF wf controlled i-bound])

```

have *collision-bound*:
 $wp\text{-event } ?M \text{ ?Collision adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1)$
by (rule
 $\text{active-route-current-prefix-conceptual-prefix-collision-hit-at-bound}$
 $[OF \text{ wf controlled } i\text{-bound}]$)
have *algebraic-bound0*:
 $\text{active-route-current-prefix-conceptual-algebraic-residual-error-for } A \ i \leq$
 $\text{active-route-current-prefix-split-algebraic-residual-bound-for } A \ i$
by (rule
 $\text{active-route-current-prefix-conceptual-algebraic-residual-bound-from-split}$
 $[OF \text{ false-statement}]$)
have *algebraic-bound*:
 $wp\text{-event } ?M \text{ ?Algebraic adversary-initial-state} \leq$
 $\text{active-route-current-prefix-split-algebraic-residual-bound-for } A \ i$
using *algebraic-bound0*
unfolding $\text{active-route-current-prefix-conceptual-algebraic-residual-error-for-def}$
by *simp*
have *union-bound*:
 $wp\text{-event } ?M$
 $(\lambda out. \text{ ?Conceptual out} \vee \text{ ?Collision out} \vee \text{ ?Algebraic out})$
 $\text{adversary-initial-state} \leq$
 $wp\text{-event } ?M \text{ ?Conceptual adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?Collision adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?Algebraic adversary-initial-state}$
proof –
have
 $wp\text{-event } ?M$
 $(\lambda out. \text{ ?Conceptual out} \vee \text{ ?Collision out} \vee \text{ ?Algebraic out})$
 $\text{adversary-initial-state} \leq$
 $wp\text{-event } ?M \text{ ?Conceptual adversary-initial-state} +$
 $wp\text{-event } ?M (\lambda out. \text{ ?Collision out} \vee \text{ ?Algebraic out})$
 $\text{adversary-initial-state}$
by (rule *wp-event-union-bound*)
also have $\dots \leq$
 $wp\text{-event } ?M \text{ ?Conceptual adversary-initial-state} +$
 $(wp\text{-event } ?M \text{ ?Collision adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?Algebraic adversary-initial-state})$
by (intro *add-mono order-refl wp-event-union-bound*)
finally show *?thesis*
by (*simp add: add.assoc*)
qed
have *sum-bound*:
 $wp\text{-event } ?M \text{ ?Conceptual adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?Collision adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?Algebraic adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$

```

    query-error-bound +
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1) +
    active-route-current-prefix-split-algebraic-residual-bound-for A i
proof -
  have tail-bound:
    wp-event ?M ?Collision adversary-initial-state +
    wp-event ?M ?Algebraic adversary-initial-state ≤
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1) +
    active-route-current-prefix-split-algebraic-residual-bound-for A i
  by (intro add-mono collision-bound algebraic-bound)
  have
    wp-event ?M ?Conceptual adversary-initial-state +
    (wp-event ?M ?Collision adversary-initial-state +
     wp-event ?M ?Algebraic adversary-initial-state) ≤
    (staged-phase-relation-error size
     (staged-query-search-queries budgets i + 1) +
     query-error-bound) +
    (hash-collision-budget-value 0
     (staged-query-search-queries budgets i + 1) +
     active-route-current-prefix-split-algebraic-residual-bound-for A i)
  by (intro add-mono conceptual-bound tail-bound)
  then show ?thesis
  by (simp add: add.assoc)
qed
show ?thesis
  unfolding active-route-current-prefix-authenticated-error-for-def
    active-route-current-prefix-split-conceptual-bound-for-def
  by (rule order-trans[OF event-le])
    (rule order-trans[OF union-bound sum-bound])
qed

definition active-route-current-empty-conceptual-prefix-collision-hit-at
where
  active-route-current-empty-conceptual-prefix-collision-hit-at
    (- :: 'f itself) out
   $\longleftrightarrow$ 
  checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
    0 out  $\wedge$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)  $\Rightarrow$ 
    hash-map-output-collision prefix-state)

definition active-route-current-empty-conceptual-trace-low-degree-bad-at
where
  active-route-current-empty-conceptual-trace-low-degree-bad-at

```


$(- :: 'f \text{ itself}) \text{ out}$
 $\longleftrightarrow$
checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
 $0 \text{ out} \wedge$
(case out of
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}) \Rightarrow$
 $\neg \text{trace-table-low-degree}$
 $(\text{query-prefix-trace-conceptual-table } \text{prefix } \text{prefix-state}))$

definition *active-route-current-empty-conceptual-alpha-bad-set-hit-at*

where

active-route-current-empty-conceptual-alpha-bad-set-hit-at
 $(- :: 'f \text{ itself}) \text{ out}$
 $\longleftrightarrow$
checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
 $0 \text{ out} \wedge$
(case out of
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}) \Rightarrow$
 $\text{sqp-alphas } \text{prefix} \in$
 $\text{composition-trace-bad-alpha-space}$
 $(\text{query-prefix-trace-conceptual-table } \text{prefix } \text{prefix-state}))$

definition *active-route-current-empty-conceptual-prefix-collision-error-for*

where

active-route-current-empty-conceptual-prefix-collision-error-for $A =$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ 0)$
 $(\text{active-route-current-empty-conceptual-prefix-collision-hit-at}$
 $\text{TYPE}(f))$
 $\text{adversary-initial-state}$

definition *active-route-current-empty-conceptual-trace-low-degree-error-for*

where

active-route-current-empty-conceptual-trace-low-degree-error-for $A =$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ 0)$
 $(\text{active-route-current-empty-conceptual-trace-low-degree-bad-at}$
 $\text{TYPE}(f))$
 $\text{adversary-initial-state}$

definition *active-route-current-empty-conceptual-alpha-bad-set-error-for*

where

active-route-current-empty-conceptual-alpha-bad-set-error-for $A =$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ 0)$

(*active-route-current-empty-conceptual-alpha-bad-set-hit-at*
TYPE('f'))
adversary-initial-state

lemma *active-route-current-empty-conceptual-alpha-bad-set-hit-at-imp-data-state-alpha-prefix-bad-set-hit-on-support:*

assumes *support:*

Some (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*) ∈
set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state A 0*)
adversary-initial-state)

and *candidate:*

∧ *alpha-prefix alpha-prefix-state.*
alpha-prefix =
 (*sqp-trace-root prefix*, *sqp-trace-fri-roots prefix*,
sqp-trace-fri-challenges prefix, *sqp-trace-final prefix*) ⇒
Some (*alpha-prefix*, *alpha-prefix-state*) ∈
set-dist
 (*execute* (*staged-alpha-prefix-program A*)
adversary-initial-state) ⇒
query-prefix-trace-conceptual-table prefix prefix-state ∈
alpha-prefix-trace-table-candidates alpha-prefix-state
 (*sqp-trace-root prefix*)

and *hit:*

active-route-current-empty-conceptual-alpha-bad-set-hit-at
TYPE('f')
(Some (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*))

shows

checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
composition-trace-bad-alpha-space
(Some (((*data*, *attacker-state*), *result*), *final-state*))

proof –

obtain *alpha-prefix alpha-prefix-state* **where** *alpha-prefix-eq:*

alpha-prefix =
 (*sqp-trace-root prefix*, *sqp-trace-fri-roots prefix*,
sqp-trace-fri-challenges prefix, *sqp-trace-final prefix*)

and *alpha-support:*

Some (*alpha-prefix*, *alpha-prefix-state*) ∈
set-dist
 (*execute* (*staged-alpha-prefix-program A*) *adversary-initial-state*)

by (rule *checked-staged-security-with-query-prefix-data-state-alpha-prefix-support*
 [OF *support*])

have *table-candidate:*

query-prefix-trace-conceptual-table prefix prefix-state ∈
alpha-prefix-trace-table-candidates alpha-prefix-state
 (*sqp-trace-root prefix*)

by (rule *candidate*[OF *alpha-prefix-eq alpha-support*])

```

have alphas-eq: staged-alphas data = sqp-alphas prefix
  by (rule checked-staged-security-with-query-prefix-data-state-alphas-eq
    [OF support])
have alpha-bad:
  sqp-alphas prefix ∈
    composition-trace-bad-alpha-space
      (query-prefix-trace-conceptual-table prefix prefix-state)
using hit
unfolding active-route-current-empty-conceptual-alpha-bad-set-hit-at-def
by simp
have staged-alpha-bad:
  staged-alphas data ∈
    composition-trace-bad-alpha-space
      (query-prefix-trace-conceptual-table prefix prefix-state)
using alpha-bad alphas-eq by simp
have staged-alpha-in-space: staged-alphas data ∈ alpha-space
using staged-alpha-bad composition-trace-bad-alpha-space-subset-alpha-space
by blast
have root-eq: staged-trace-root data = sqp-trace-root prefix
using support
unfolding
  checked-staged-security-experiment-with-query-prefix-data-state-def
  checked-staged-after-query-prefix-receive-with-verifier-def
  checked-staged-after-query-prefix-receive-def
by (auto elim!: set-dist-bindE split: prod.splits)
have union-hit:
  staged-alphas data ∈
    alpha-prefix-union-bad-sets alpha-prefix-state
      composition-trace-bad-alpha-space (staged-trace-root data)
proof –
  have table-candidate':
    query-prefix-trace-conceptual-table prefix prefix-state ∈
      alpha-prefix-trace-table-candidates alpha-prefix-state
        (staged-trace-root data)
    using table-candidate root-eq by simp
show ?thesis
unfolding alpha-prefix-union-bad-sets-def
using table-candidate' staged-alpha-bad staged-alpha-in-space
by auto
qed
have prefix-components:
  staged-trace-root data = fst alpha-prefix ∧
    staged-trace-fri-roots data = fst (snd alpha-prefix) ∧
    staged-trace-fri-challenges data = fst (snd (snd alpha-prefix)) ∧
    staged-trace-final data = snd (snd (snd alpha-prefix))
using support alpha-prefix-eq
unfolding
  checked-staged-security-experiment-with-query-prefix-data-state-def
  checked-staged-after-query-prefix-receive-with-verifier-def

```

```

    checked-staged-after-query-prefix-receive-def
  by (auto elim!: set-dist-bindE split: prod.splits)
show ?thesis
  unfolding checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-def
  using alpha-support prefix-components union-hit
  by auto
qed

lemma active-route-current-empty-conceptual-alpha-bad-set-error-for-bound-from-data-state-alpha-prefix:
  assumes candidate:
     $\bigwedge \text{prefix prefix-state raw raw-state data attacker-state result}$ 
     $\text{final-state alpha-prefix alpha-prefix-state.}$ 
     $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$ 
     $((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}) \in$ 
     $\text{set-dist}$ 
     $(\text{execute}$ 
     $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ 0)$ 
     $\text{adversary-initial-state}) \implies$ 
     $\text{alpha-prefix} =$ 
     $(\text{sqp-trace-root prefix}, \text{sqp-trace-fri-roots prefix},$ 
     $\text{sqp-trace-fri-challenges prefix}, \text{sqp-trace-final prefix}) \implies$ 
     $\text{Some } (\text{alpha-prefix}, \text{alpha-prefix-state}) \in$ 
     $\text{set-dist}$ 
     $(\text{execute } (\text{staged-alpha-prefix-program } A)$ 
     $\text{adversary-initial-state}) \implies$ 
     $\text{query-prefix-trace-conceptual-table prefix prefix-state} \in$ 
     $\text{alpha-prefix-trace-table-candidates alpha-prefix-state}$ 
     $(\text{sqp-trace-root prefix})$ 
  and data-state-bound:
     $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$ 
     $(\text{checked-staged-security-with-data-state-alpha-prefix-bad-set-hit } A$ 
     $\text{composition-trace-bad-alpha-space})$ 
     $\text{adversary-initial-state} \leq C$ 
  shows
     $\text{active-route-current-empty-conceptual-alpha-bad-set-error-for } A \leq C$ 
proof -
  let ?Data =
     $\text{checked-staged-security-with-data-state-alpha-prefix-bad-set-hit } A$ 
     $\text{composition-trace-bad-alpha-space}$ 
  let ?Projected =
     $\lambda \text{out. case out of}$ 
     $\text{None} \Rightarrow ?Data \ \text{None}$ 
     $| \ \text{Some } (\text{packed}, t) \Rightarrow ?Data \ (\text{Some } (\text{snd packed}, t))$ 
  let ?E =
     $\text{active-route-current-empty-conceptual-alpha-bad-set-hit-at}$ 
     $\text{TYPE('f)}$ 
  have event-le:
     $\text{wp-event}$ 
     $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ 0)$ 

```

```

    ?E adversary-initial-state ≤
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A 0)
  ?Projected adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A 0)
        adversary-initial-state)
  and hit: ?E out
show ?Projected out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding active-route-current-empty-conceptual-alpha-bad-set-hit-at-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
  final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
have support':
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A 0)
        adversary-initial-state)
  using support unfolding out-eq .
have hit':
  active-route-current-empty-conceptual-alpha-bad-set-hit-at
    TYPE('f)
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  using hit unfolding out-eq .
have data-hit:
  ?Data (Some (((data, attacker-state), result), final-state))
  by (rule
    active-route-current-empty-conceptual-alpha-bad-set-hit-at-imp-data-state-alpha-prefix-bad-set-hit-on-support
      [OF support' - hit'])
    (rule candidate[OF support'])
show ?thesis

```

unfolding *out-eq* **using** *data-hit* **by** *simp*
qed
qed
have *projection*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A* 0)
 ?*Projected adversary-initial-state* =
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 ?*Data adversary-initial-state*
using *checked-staged-security-experiment-with-query-prefix-data-state-projection-event*
 [*OF rounds-positive*, of *A* ?*Data*]
by *simp*
show ?*thesis*
unfolding *active-route-current-empty-conceptual-alpha-bad-set-error-for-def*
by (*rule order-trans*[*OF event-le*])
 (*simp add: projection data-state-bound*)
qed

definition *active-route-current-empty-split-residual-bound-for*
where

active-route-current-empty-split-residual-bound-for *A* =
active-route-current-empty-conceptual-prefix-collision-error-for *A* +
active-route-current-empty-conceptual-trace-low-degree-error-for *A* +
active-route-current-empty-conceptual-alpha-bad-set-error-for *A*

definition *active-route-current-empty-split-conceptual-bound-for*
where

active-route-current-empty-split-conceptual-bound-for *budgets* *A* =
staged-phase-relation-error size
 (*staged-query-search-queries* *budgets* 0 + 1) +
query-error-bound +
active-route-current-empty-split-residual-bound-for *A* +
 (*active-route-transcript-pre-error-for* *A* +
 staged-concrete-transcript-target-error-bound)

lemma *active-route-current-empty-conceptual-prefix-collision-hit-at-bound*:

assumes *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
shows

active-route-current-empty-conceptual-prefix-collision-error-for *A* ≤
hash-collision-budget-value 0
 (*staged-query-search-queries* *budgets* 0 + 1)

proof –

have *i-le*: 0 ≤ *rounds*
by *simp*

have *prefix-bound*:

wp-event (*checked-staged-query-prefix-receive-with-state* *A* 0)
checked-staged-query-prefix-hash-map-output-collision
adversary-initial-state ≤

```

    hash-collision-budget-value 0
    (staged-query-search-queries budgets 0 + 1)
  by (rule checked-staged-query-prefix-hash-map-output-collision-bound
      [OF wf controlled i-le])
show ?thesis
  unfolding active-route-current-empty-conceptual-prefix-collision-error-for-def
proof (rule
  checked-staged-security-experiment-with-query-prefix-data-state-bound-by-prefix
  [OF prefix-bound])
show
  active-route-current-empty-conceptual-prefix-collision-hit-at
  TYPE('f) None
   $\implies$  checked-staged-query-prefix-hash-map-output-collision None
  unfolding
    active-route-current-empty-conceptual-prefix-collision-hit-at-def
    checked-staged-query-prefix-hash-map-output-collision-def
  by simp
next
fix prefix-out t out
assume
  Some (prefix-out, t)  $\in$ 
  set-dist
  (execute (checked-staged-query-prefix-receive-with-state A 0)
    adversary-initial-state)
and cont:
  out  $\in$ 
  set-dist
  (execute
    (get  $\gg$ 
      ( $\lambda$ prefix-state.
        checked-staged-after-query-prefix-receive-with-verifier A 0
        prefix-out  $\gg$ 
        ( $\lambda$ result. return ((prefix-out, prefix-state), result))))
    t)
and event:
  active-route-current-empty-conceptual-prefix-collision-hit-at
  TYPE('f) out
show
  checked-staged-query-prefix-hash-map-output-collision
  (Some (prefix-out, t))
using cont event
unfolding
  active-route-current-empty-conceptual-prefix-collision-hit-at-def
  checked-staged-query-prefix-hash-map-output-collision-def
by (auto elim!: set-dist-bindE split: option.splits prod.splits)
qed
qed

```

lemma *active-route-current-empty-conceptual-residual-split-on-support:*

```

assumes false-statement:  $\neg$  exists-valid-trace
and support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)  $\in$ 
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A 0)
      adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
    0
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
shows
  active-route-current-empty-conceptual-prefix-collision-hit-at
    TYPE('f)
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))  $\vee$ 
  active-route-current-empty-conceptual-trace-low-degree-bad-at
    TYPE('f)
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))  $\vee$ 
  active-route-current-empty-conceptual-alpha-bad-set-hit-at
    TYPE('f)
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof –
  let ?trace = query-prefix-trace-conceptual-table prefix prefix-state
  let ?composition = replicate (scale * clength) (sqp-composition-final prefix)
  from hit have residual:
    hash-map-output-collision prefix-state  $\vee$ 
     $\neg$  trace-table-low-degree ?trace  $\vee$ 
    all-queries-consistent ?trace ?composition (sqp-alphas prefix)
  unfolding
    checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at-def
    checked-staged-security-with-query-prefix-conceptual-empty-context-residual-at-def
  by simp
  show ?thesis
  proof (cases hash-map-output-collision prefix-state)
    case True
    then show ?thesis
      using hit
      unfolding
        active-route-current-empty-conceptual-prefix-collision-hit-at-def
      by simp
  next
    case clean: False
    show ?thesis
    proof (cases trace-table-low-degree ?trace)

```



```

case False
then show ?thesis
  using hit
  unfolding
    active-route-current-empty-conceptual-trace-low-degree-bad-at-def
  by simp
next
case trace-low: True
from residual clean trace-low have all-queries:
  all-queries-consistent ?trace ?composition (sqp-alphas prefix)
  by simp
have prefix-support:
  Some ((prefix, prefix-state), prefix-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-with-state A 0)
      adversary-initial-state)
  by (rule
    checked-staged-security-with-query-prefix-data-state-prefix-support
    [OF support])
have alpha-len: length (sqp-alphas prefix) = length spec
  by (rule checked-staged-query-prefix-with-state-alphas-length
    [OF prefix-support])
have alpha-bad:
  sqp-alphas prefix ∈ composition-trace-bad-alpha-space ?trace
  by (rule
    low-degree-all-queries-consistent-imp-composition-trace-bad-alpha-space
    [OF false-statement alpha-len trace-low
      constant-composition-table-low-degree all-queries])
then show ?thesis
  using hit
  unfolding
    active-route-current-empty-conceptual-alpha-bad-set-hit-at-def
  by simp
qed
qed
qed

lemma active-route-current-empty-conceptual-residual-bound-from-split:
  assumes false-statement: ¬ exists-valid-trace
  shows
    active-route-current-empty-conceptual-residual-error-for TYPE('f) A ≤
      active-route-current-empty-split-residual-bound-for A
proof –
  let ?M = checked-staged-security-experiment-with-query-prefix-data-state A 0
  let ?Residual =
    λout.
      checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
        0 out
  let ?Collision =

```

```

    active-route-current-empty-conceptual-prefix-collision-hit-at
      TYPE('f)
  let ?Trace =
    active-route-current-empty-conceptual-trace-low-degree-bad-at
      TYPE('f)
  let ?Alpha =
    active-route-current-empty-conceptual-alpha-bad-set-hit-at
      TYPE('f)
  have event-le:
    wp-event ?M ?Residual adversary-initial-state ≤
    wp-event ?M (λout. ?Collision out ∨ ?Trace out ∨ ?Alpha out)
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support: out ∈ set-dist (execute ?M adversary-initial-state)
    and hit: ?Residual out
    show ?Collision out ∨ ?Trace out ∨ ?Alpha out
  proof (cases out)
    case None
    then show ?thesis
      using hit
      unfolding
        checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at-def
        checked-staged-security-with-query-prefix-conceptual-empty-context-residual-at-def
      by simp
  next
    case (Some packed)
    then obtain prefix prefix-state raw raw-state data attacker-state result
      final-state where out-eq:
        out =
          Some (((((prefix, prefix-state), raw), raw-state),
            ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
    have support':
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state) ∈
      set-dist (execute ?M adversary-initial-state)
    using support unfolding out-eq .
    have hit':
      ?Residual
        (Some (((((prefix, prefix-state), raw), raw-state),
          ((data, attacker-state), result)), final-state))
    using hit unfolding out-eq .
    show ?thesis
      unfolding out-eq
    by (rule
      active-route-current-empty-conceptual-residual-split-on-support
      [OF false-statement support' hit'])
  qed

```

```

qed
have union-bound:
  wp-event ?M ( $\lambda out. ?Collision\ out \vee ?Trace\ out \vee ?Alpha\ out$ )
    adversary-initial-state  $\leq$ 
    wp-event ?M ?Collision adversary-initial-state +
    wp-event ?M ?Trace adversary-initial-state +
    wp-event ?M ?Alpha adversary-initial-state
proof -
  have
    wp-event ?M
      ( $\lambda out. ?Collision\ out \vee ?Trace\ out \vee ?Alpha\ out$ )
        adversary-initial-state  $\leq$ 
        wp-event ?M ?Collision adversary-initial-state +
        wp-event ?M ( $\lambda out. ?Trace\ out \vee ?Alpha\ out$ )
          adversary-initial-state
    by (rule wp-event-union-bound)
  also have ...  $\leq$ 
    wp-event ?M ?Collision adversary-initial-state +
    (wp-event ?M ?Trace adversary-initial-state +
     wp-event ?M ?Alpha adversary-initial-state)
    by (intro add-mono order-refl wp-event-union-bound)
  finally show ?thesis
    by (simp add: add.assoc)
qed
show ?thesis
  unfolding
    active-route-current-empty-conceptual-residual-error-for-def
    active-route-current-empty-split-residual-bound-for-def
    active-route-current-empty-conceptual-prefix-collision-error-for-def
    active-route-current-empty-conceptual-trace-low-degree-error-for-def
    active-route-current-empty-conceptual-alpha-bad-set-error-for-def
  by (rule order-trans[OF event-le union-bound])
qed

lemma active-route-current-empty-query-error-for-le-split-conceptual-bound:
  assumes false-statement:  $\neg$  exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    active-route-current-empty-query-error-for A  $\leq$ 
    active-route-current-empty-split-conceptual-bound-for budgets A
proof -
  have i-bound:  $0 < rounds$ 
  using rounds-positive by simp
  have residual-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A 0)
      (checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
       0)

```

adversary-initial-state $\leq$
active-route-current-empty-split-residual-bound-for A
using *active-route-current-empty-conceptual-residual-bound-from-split*
[OF false-statement]
unfolding *active-route-current-empty-conceptual-residual-error-for-def* .
have *pre-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq$ *active-route-transcript-pre-error-for A*
by (*simp add: active-route-transcript-pre-error-for-def*)
have *new-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-new-hit
adversary-initial-state $\leq$ *staged-concrete-transcript-target-error-bound*
by (*rule checked-staged-security-with-data-state-transcript-new-hit-bound*
[OF wf controlled])
show *?thesis*
unfolding *active-route-current-empty-query-error-for-def*
active-route-current-empty-split-conceptual-bound-for-def
by (*rule*
checked-staged-security-with-data-state-current-empty-query-opening-hit-at-bound-from-conceptual-empty-residual
[OF wf controlled i-bound residual-bound pre-bound new-bound])
qed

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-restricted-trace-endpoint:*

assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
($\sum i < \text{rounds}.$
active-route-current-prefix-split-conceptual-bound-for budgets A i +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
concrete-trace-header-restricted-one-step-error-for budgets A +
concrete-composition-restricted-one-step-error-for budgets A +

(composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 concrete-trace-empty-header-restricted-sampled-split-one-step-error-for budgets A +
 active-route-current-empty-split-conceptual-bound-for budgets A)

proof –

have trace-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state
 $\leq$ concrete-trace-header-restricted-one-step-error-for budgets A
unfolding concrete-trace-header-restricted-one-step-error-for-def
by (rule trace-header-restricted-one-step-error-for-bound
 [OF wf controlled])

have comp-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state
 $\leq$ concrete-composition-restricted-one-step-error-for budgets A
unfolding concrete-composition-restricted-one-step-error-for-def
by (rule composition-restricted-verifier-tied-controlled-error-for-bound
 [OF wf controlled])

have empty-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state
 $\leq$ concrete-trace-empty-header-restricted-sampled-split-one-step-error-for
 budgets A
unfolding
 concrete-trace-empty-header-restricted-sampled-split-one-step-error-for-def
by (rule trace-empty-header-restricted-sampled-split-one-step-error-for-bound
 [OF wf controlled])

have route:

checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +

hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
($\sum i < \text{rounds}.$
active-route-current-prefix-authenticated-error-for A i +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
concrete-trace-header-restricted-one-step-error-for budgets A +
concrete-composition-restricted-one-step-error-for budgets A +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
concrete-trace-empty-header-restricted-sampled-split-one-step-error-for budgets A +
active-route-current-empty-query-error-for A)
by (*rule*
stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-bounds
[OF false-statement wf controlled trace-bound comp-bound empty-bound])
have *prefix-terms:*
($\sum i < \text{rounds}.$
active-route-current-prefix-authenticated-error-for A i +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound))
 $\leq$
($\sum i < \text{rounds}.$
active-route-current-prefix-split-conceptual-bound-for budgets A i +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound))
by (*intro sum-mono add-mono order-refl*
active-route-current-prefix-authenticated-error-for-le-split-conceptual-bound
[OF wf controlled - false-statement])
simp
have *empty-term:*
active-route-current-empty-query-error-for A $\leq$
active-route-current-empty-split-conceptual-bound-for budgets A

by (*rule active-route-current-empty-query-error-for-le-split-conceptual-bound*
[OF false-statement wf controlled])
have *route-le*:
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
($\sum i < \text{rounds}.$
active-route-current-prefix-authenticated-error-for A i +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
concrete-trace-header-restricted-one-step-error-for budgets A +
concrete-composition-restricted-one-step-error-for budgets A +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
concrete-trace-empty-header-restricted-sampled-split-one-step-error-for budgets A +
active-route-current-empty-query-error-for A)
 $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
($\sum i < \text{rounds}.$
active-route-current-prefix-split-conceptual-bound-for budgets A i +

(active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 concrete-trace-header-restricted-one-step-error-for budgets A +
 concrete-composition-restricted-one-step-error-for budgets A +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 concrete-trace-empty-header-restricted-sampled-split-one-step-error-for budgets A +
 active-route-current-empty-split-conceptual-bound-for budgets A)
by (intro add-mono order-refl prefix-terms empty-term)
show ?thesis
by (rule order-trans[OF route route-le])
qed

definition concrete-restricted-trace-stark-soundness-bound-for
where

concrete-restricted-trace-stark-soundness-bound-for budgets A =
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 ($\sum i < \text{rounds}.$
 active-route-current-prefix-split-conceptual-bound-for budgets A i +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 concrete-trace-header-restricted-one-step-error-for budgets A +
 concrete-composition-restricted-one-step-error-for budgets A +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0

(staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 concrete-trace-empty-header-restricted-sampled-split-one-step-error-for budgets A +
 active-route-current-empty-split-conceptual-bound-for budgets A)

theorem *stark-soundness*:

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

shows *checked-staged-adversary-acceptance-probability A* $\leq$
concrete-restricted-trace-stark-soundness-bound-for budgets A

unfolding *concrete-restricted-trace-stark-soundness-bound-for-def*

by (*rule*

stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-restricted-trace-endpoint
[OF false-statement wf controlled])

end

end

theory *Stark-Document*

imports

Soundness-FRI-Trace-Restricted-Query-Endpoint

Stark-Completeness.Completeness

Stark-Examples.Square-Sequence

begin

Document aggregation theory for the generated Isabelle proof document. The main proof entry points remain the completeness theorem, the executable example, and the public soundness endpoint imported above.

end